

FORM

v5.0.0-beta.1-207-geebd42d

Generated by Doxygen 1.9.8

1 Data Structure Index	1
1.1 Data Structures	1
2 File Index	5
2.1 File List	5
3 Data Structure Documentation	7
3.1 AEdge Class Reference	7
3.1.1 Detailed Description	7
3.1.2 Constructor & Destructor Documentation	8
3.1.2.1 AEdge()	8
3.1.2.2 ~AEdge()	8
3.1.3 Field Documentation	8
3.1.3.1 cand	8
3.1.3.2 nodes	8
3.1.3.3 nlegs	8
3.1.3.4 ptcl	8
3.1.3.5 DUMMYPADDING	9
3.2 AllGlobals Struct Reference	9
3.2.1 Detailed Description	10
3.2.2 Field Documentation	10
3.2.2.1 M	10
3.2.2.2 Cc	10
3.2.2.3 S	10
3.2.2.4 R	10
3.2.2.5 N	11
3.2.2.6 O	11
3.2.2.7 P	11
3.2.2.8 T	11
3.2.2.9 X	11
3.3 ANode Class Reference	11
3.3.1 Detailed Description	12
3.3.2 Constructor & Destructor Documentation	12
3.3.2.1 ANode()	12
3.3.2.2 ~ANode()	12
3.3.3 Member Function Documentation	12
3.3.3.1 newleg()	12
3.3.4 Field Documentation	13
3.3.4.1 deg	13
3.3.4.2 nlegs	13
3.3.4.3 anodes	13
3.3.4.4 aedges	13
3.3.4.5 aelegs	13

3.3.4.6 cand	13
3.4 ARGBUFFER Struct Reference	14
3.4.1 Detailed Description	14
3.4.2 Field Documentation	14
3.4.2.1 buffer	14
3.4.2.2 dollar	14
3.4.2.3 size	15
3.4.2.4 type	15
3.4.2.5 dummy	15
3.5 Assign Class Reference	15
3.5.1 Detailed Description	17
3.5.2 Constructor & Destructor Documentation	17
3.5.2.1 Assign()	17
3.5.2.2 ~Assign()	17
3.5.3 Member Function Documentation	17
3.5.3.1 prCand()	17
3.5.3.2 checkAG()	17
3.5.3.3 assignAllVertices()	18
3.5.3.4 selectVertex()	18
3.5.3.5 selectVertexSimp()	18
3.5.3.6 selectLeg()	18
3.5.3.7 assignVertex()	18
3.5.3.8 allAssigned()	18
3.5.3.9 fromMGraph()	18
3.5.3.10 addEdge()	19
3.5.3.11 connect()	19
3.5.3.12 fillEGraph()	19
3.5.3.13 reordLeg()	19
3.5.3.14 getLegParticle()	19
3.5.3.15 legEdgeParticle()	20
3.5.3.16 legPart()	20
3.5.3.17 candPart()	20
3.5.3.18 selUnAssVertex()	20
3.5.3.19 selUnAssVertexSimp()	20
3.5.3.20 selUnAssLeg()	20
3.5.3.21 assignVertex()	21
3.5.3.22 assignPLeg()	21
3.5.3.23 isOrdPLeg()	21
3.5.3.24 detEdge()	21
3.5.3.25 candPartClassify()	21
3.5.3.26 updateCandNode()	21
3.5.3.27 checkOrderCpl()	22

3.5.3.28 isOrdLegs()	22
3.5.3.29 isIsomorphic()	22
3.5.3.30 cmpPermGraph()	22
3.5.3.31 cmpNodes()	22
3.5.3.32 edgeSym()	22
3.5.3.33 saveCouple()	23
3.5.3.34 restoreCouple()	23
3.5.3.35 subCouple()	23
3.5.4 Field Documentation	23
3.5.4.1 egraph	23
3.5.4.2 mgraph	23
3.5.4.3 sproc	23
3.5.4.4 proc	23
3.5.4.5 model	24
3.5.4.6 opt	24
3.5.4.7 astack	24
3.5.4.8 pnclass	24
3.5.4.9 orbits	24
3.5.4.10 nNodes	24
3.5.4.11 nEdges	24
3.5.4.12 nExtern	24
3.5.4.13 nETotal	25
3.5.4.14 nAGraphs	25
3.5.4.15 wAGraphs	25
3.5.4.16 nAOPI	25
3.5.4.17 wAOPI	25
3.5.4.18 nodes	25
3.5.4.19 edges	25
3.5.4.20 checkpoint0	25
3.5.4.21 cplleft	26
3.6 AStack Class Reference	26
3.6.1 Detailed Description	27
3.6.2 Constructor & Destructor Documentation	27
3.6.2.1 AStack()	27
3.6.2.2 ~AStack()	27
3.6.3 Member Function Documentation	27
3.6.3.1 setAGraph()	27
3.6.3.2 checkPoint()	27
3.6.3.3 restore()	27
3.6.3.4 prStack()	28
3.6.3.5 pushNode()	28
3.6.3.6 pushEdge()	28

3.6.4 Field Documentation	28
3.6.4.1 agraph	28
3.6.4.2 nStack	28
3.6.4.3 nStackP	28
3.6.4.4 nSize	28
3.6.4.5 eStack	29
3.6.4.6 eStackP	29
3.6.4.7 eSize	29
3.7 bit_field Struct Reference	29
3.7.1 Detailed Description	29
3.7.2 Field Documentation	30
3.7.2.1 bit_0	30
3.7.2.2 bit_1	30
3.7.2.3 bit_2	30
3.7.2.4 bit_3	30
3.7.2.5 bit_4	30
3.7.2.6 bit_5	30
3.7.2.7 bit_6	30
3.7.2.8 bit_7	31
3.8 BrAcKeTiNdEx Struct Reference	31
3.8.1 Detailed Description	31
3.8.2 Field Documentation	32
3.8.2.1 start	32
3.8.2.2 next	32
3.8.2.3 bracket	32
3.8.2.4 termsinbracket	32
3.9 BrAcKeTiNfO Struct Reference	32
3.9.1 Detailed Description	33
3.9.2 Field Documentation	33
3.9.2.1 indexbuffer	33
3.9.2.2 bracketbuffer	33
3.9.2.3 bracketbuffersize	33
3.9.2.4 indexbuffersize	33
3.9.2.5 bracketfill	34
3.9.2.6 indexfill	34
3.9.2.7 SortType	34
3.10 BracketInfo Struct Reference	34
3.10.1 Detailed Description	35
3.10.2 Constructor & Destructor Documentation	35
3.10.2.1 BracketInfo()	35
3.10.3 Member Function Documentation	35
3.10.3.1 operator<()	35

3.10.4 Field Documentation	35
3.10.4.1 pattern	35
3.10.4.2 num_terms	35
3.10.4.3 dummy	36
3.10.4.4 p	36
3.11 bufIPstruct Struct Reference	36
3.11.1 Detailed Description	37
3.11.2 Field Documentation	37
3.11.2.1 i	37
3.11.2.2 e	37
3.12 C_const Struct Reference	37
3.12.1 Detailed Description	42
3.12.2 Field Documentation	42
3.12.2.1 separators	42
3.12.2.2 StoreFileSize	42
3.12.2.3 dollarnames	42
3.12.2.4 exprnames	42
3.12.2.5 varnames	42
3.12.2.6 ChannelList	43
3.12.2.7 DubiousList	43
3.12.2.8 FunctionList	43
3.12.2.9 ExpressionList	43
3.12.2.10 IndexList	43
3.12.2.11 SetElementList	43
3.12.2.12 SetList	44
3.12.2.13 SymbolList	44
3.12.2.14 VectorList	44
3.12.2.15 PotModDolList	44
3.12.2.16 ModOptDolList	44
3.12.2.17 TableBaseList	44
3.12.2.18 cbufList	45
3.12.2.19 AutoSymbolList	45
3.12.2.20 AutoIndexList	45
3.12.2.21 AutoVectorList	45
3.12.2.22 AutoFunctionList	45
3.12.2.23 autonames	45
3.12.2.24 Symbols	45
3.12.2.25 Indices	46
3.12.2.26 Vectors	46
3.12.2.27 Functions	46
3.12.2.28 activenames	46
3.12.2.29 Streams	46

3.12.2.30 CurrentStream	46
3.12.2.31 SwitchArray	46
3.12.2.32 models	47
3.12.2.33 SwitchHeap	47
3.12.2.34 termstack	47
3.12.2.35 termsortstack	47
3.12.2.36 cmod	47
3.12.2.37 powmod	47
3.12.2.38 modpowers	48
3.12.2.39 halfmod	48
3.12.2.40 ProtoType	48
3.12.2.41 WildC	48
3.12.2.42 IfHeap	48
3.12.2.43 IfCount	48
3.12.2.44 IfStack	48
3.12.2.45 iBuffer	49
3.12.2.46 iPointer	49
3.12.2.47 iStop	49
3.12.2.48 LabelNames	49
3.12.2.49 FixIndices	49
3.12.2.50 termsumcheck	49
3.12.2.51 WildcardNames	50
3.12.2.52 Labels	50
3.12.2.53 tokens	50
3.12.2.54 toptokens	50
3.12.2.55 endoftokens	50
3.12.2.56 tokenarglevel	50
3.12.2.57 modinverses	51
3.12.2.58 Fortran90Kind	51
3.12.2.59 MultiBracketBuf	51
3.12.2.60 extrasym	51
3.12.2.61 doloopstack	51
3.12.2.62 doloopnest	51
3.12.2.63 CheckpointRunAfter	51
3.12.2.64 CheckpointRunBefore	52
3.12.2.65 IfSumCheck	52
3.12.2.66 CommutInSet	52
3.12.2.67 TestValue	52
3.12.2.68 argstack	52
3.12.2.69 insidestack	52
3.12.2.70 inexprstack	52
3.12.2.71 iBufferSize	53

3.12.2.72 TransEname	53
3.12.2.73 ProcessBucketSize	53
3.12.2.74 mProcessBucketSize	53
3.12.2.75 CModule	53
3.12.2.76 ThreadBucketSize	53
3.12.2.77 CheckpointStamp	53
3.12.2.78 CheckpointInterval	54
3.12.2.79 cbufnum	54
3.12.2.80 AutoDeclareFlag	54
3.12.2.81 NoShowInput	54
3.12.2.82 ShortStats	54
3.12.2.83 compiletype	54
3.12.2.84 firstconstindex	54
3.12.2.85 insidefirst	55
3.12.2.86 minsidefirst	55
3.12.2.87 wildflag	55
3.12.2.88 NumLabels	55
3.12.2.89 MaxLabels	55
3.12.2.90 IDefDim	55
3.12.2.91 IDefDim4	55
3.12.2.92 NumWildcardNames	55
3.12.2.93 WildcardBufferSize	56
3.12.2.94 MaxIf	56
3.12.2.95 NumStreams	56
3.12.2.96 MaxNumStreams	56
3.12.2.97 firstctypemessage	56
3.12.2.98 tablecheck	56
3.12.2.99 idoption	56
3.12.2.100 BottomLevel	56
3.12.2.101 CompileLevel	57
3.12.2.102 TokensWriteFlag	57
3.12.2.103 UnsureDollarMode	57
3.12.2.104 outsidefun	57
3.12.2.105 funpowers	57
3.12.2.106 WarnFlag	57
3.12.2.107 StatsFlag	57
3.12.2.108 NamesFlag	57
3.12.2.109 CodesFlag	58
3.12.2.110 SetupFlag	58
3.12.2.111 SortType	58
3.12.2.112 ISortType	58
3.12.2.113 ThreadStats	58

3.12.2.114 FinalStats	58
3.12.2.115 OldParallelStats	58
3.12.2.116 ThreadsFlag	58
3.12.2.117 ThreadBalancing	59
3.12.2.118 ThreadSortFileSynch	59
3.12.2.119 ProcessStats	59
3.12.2.120 BracketNormalize	59
3.12.2.121 maxtermlevel	59
3.12.2.122 dumnumflag	59
3.12.2.123 bracketindexflag	59
3.12.2.124 parallelfld	59
3.12.2.125 mparallelfld	60
3.12.2.126 inparallelfld	60
3.12.2.127 partodofld	60
3.12.2.128 properorderflag	60
3.12.2.129 vetofilling	60
3.12.2.130 tablefilling	60
3.12.2.131 vetotablebasefill	60
3.12.2.132 exprfillwarning	60
3.12.2.133 lhdollarflag	61
3.12.2.134 NoCompress	61
3.12.2.135 IsFortran90	61
3.12.2.136 MultiBracketLevels	61
3.12.2.137 topolynomialflag	61
3.12.2.138 ffbufnum	61
3.12.2.139 OldFactArgFlag	61
3.12.2.140 MemDebugFlag	61
3.12.2.141 OldGCDflag	62
3.12.2.142 WTimeStatsFlag	62
3.12.2.143 SortReallocateFlag	62
3.12.2.144 PrintBacktraceFlag	62
3.12.2.145 FlintPolyFlag	62
3.12.2.146 HumanStatsFlag	62
3.12.2.147 doloopstacksize	62
3.12.2.148 dolooplevel	62
3.12.2.149 CheckpointFlag	63
3.12.2.150 SizeCommutelnSet	63
3.12.2.151 origin	63
3.12.2.152 vectorlikeLHS	63
3.12.2.153 nummodels	63
3.12.2.154 modelspace	63
3.12.2.155 ModelLevel	63

3.12.2.156 argsumcheck	64
3.12.2.157 insidesumcheck	64
3.12.2.158 inexprsumcheck	64
3.12.2.159 RepSumCheck	64
3.12.2.160 IUniTrace	64
3.12.2.161 RepLevel	64
3.12.2.162 arglevel	64
3.12.2.163 insidelevel	64
3.12.2.164 inexprlevel	65
3.12.2.165 termlevel	65
3.12.2.166 MustTestTable	65
3.12.2.167 DumNum	65
3.12.2.168 ncmmod	65
3.12.2.169 npowmod	65
3.12.2.170 modmode	65
3.12.2.171 nhalfmod	65
3.12.2.172 DirtPow	66
3.12.2.173 IUnitTrace	66
3.12.2.174 NwildC	66
3.12.2.175 ComDefer	66
3.12.2.176 CollectFun	66
3.12.2.177 AltCollectFun	66
3.12.2.178 OutputMode	66
3.12.2.179 Cnumpows	66
3.12.2.180 OutputSpaces	67
3.12.2.181 OutNumberType	67
3.12.2.182 DidClean	67
3.12.2.183 IfLevel	67
3.12.2.184 WhileLevel	67
3.12.2.185 SwitchLevel	67
3.12.2.186 SwitchInArray	67
3.12.2.187 MaxSwitch	67
3.12.2.188 LogHandle	68
3.12.2.189 LineLength	68
3.12.2.190 StoreHandle	68
3.12.2.191 HideLevel	68
3.12.2.192 IPolyFun	68
3.12.2.193 IPolyFunInv	68
3.12.2.194 IPolyFunType	68
3.12.2.195 IPolyFunExp	68
3.12.2.196 IPolyFunVar	69
3.12.2.197 IPolyFunPow	69

3.12.2.198 SymChangeFlag	69
3.12.2.199 CollectPercentage	69
3.12.2.200 ShortStatsMax	69
3.12.2.201 extrasymbols	69
3.12.2.202 PolyRatFunChanged	69
3.12.2.203 ToBelFactors	69
3.12.2.204 InnerTest	70
3.12.2.205 Commercial	70
3.12.2.206 debugFlags	70
3.13 CbUf Struct Reference	70
3.13.1 Detailed Description	71
3.13.2 Field Documentation	71
3.13.2.1 Buffer	71
3.13.2.2 Top	71
3.13.2.3 Pointer	72
3.13.2.4 lhs	72
3.13.2.5 rhs	72
3.13.2.6 CanCommu	72
3.13.2.7 NumTerms	72
3.13.2.8 numdum	73
3.13.2.9 dimension	73
3.13.2.10 boomlijst	73
3.13.2.11 BufferSize	73
3.13.2.12 numlhs	73
3.13.2.13 numrhs	73
3.13.2.14 maxlhs	74
3.13.2.15 maxrhs	74
3.13.2.16 mnumlhs	74
3.13.2.17 mnumrhs	74
3.13.2.18 numtree	74
3.13.2.19 rootnum	74
3.13.2.20 MaxTreeSize	74
3.14 ChAnNeL Struct Reference	75
3.14.1 Detailed Description	75
3.14.2 Field Documentation	75
3.14.2.1 name	75
3.14.2.2 handle	75
3.15 COST Struct Reference	75
3.15.1 Detailed Description	76
3.15.2 Field Documentation	76
3.15.2.1 add	76
3.15.2.2 mul	76

3.15.2.3 div	76
3.15.2.4 pow	76
3.16 CSEEq Struct Reference	76
3.16.1 Detailed Description	76
3.16.2 Member Function Documentation	77
3.16.2.1 operator()	77
3.17 CSEHash Struct Reference	77
3.17.1 Detailed Description	77
3.17.2 Member Function Documentation	77
3.17.2.1 operator()	77
3.18 dbase Struct Reference	78
3.18.1 Detailed Description	78
3.18.2 Field Documentation	78
3.18.2.1 info	78
3.18.2.2 mode	79
3.18.2.3 tablenameessize	79
3.18.2.4 topnumber	79
3.18.2.5 tablenameefill	79
3.18.2.6 rwmode	79
3.18.2.7 iblocks	79
3.18.2.8 nblocks	79
3.18.2.9 handle	79
3.18.2.10 name	80
3.18.2.11 fullname	80
3.18.2.12 tablenames	80
3.19 DGraph Struct Reference	80
3.19.1 Detailed Description	81
3.19.2 Field Documentation	81
3.19.2.1 nnodes	81
3.19.2.2 nedges	81
3.19.2.3 nodes	81
3.19.2.4 edges	81
3.20 DICTIONARY Struct Reference	81
3.20.1 Detailed Description	82
3.20.2 Field Documentation	82
3.20.2.1 elements	82
3.20.2.2 name	82
3.20.2.3 sizeelements	82
3.20.2.4 numelements	82
3.20.2.5 numbers	82
3.20.2.6 variables	83
3.20.2.7 characters	83

3.20.2.8 funwith	83
3.20.2.9 gnumelements	83
3.20.2.10 ranges	83
3.21 DICTIONARY_ELEMENT Struct Reference	83
3.21.1 Detailed Description	83
3.21.2 Field Documentation	84
3.21.2.1 lhs	84
3.21.2.2 rhs	84
3.21.2.3 type	84
3.21.2.4 size	84
3.22 DiStRiBuTe Struct Reference	84
3.22.1 Detailed Description	85
3.22.2 Field Documentation	85
3.22.2.1 obj1	85
3.22.2.2 obj2	85
3.22.2.3 out	85
3.22.2.4 sign	85
3.22.2.5 n1	85
3.22.2.6 n2	85
3.22.2.7 n	86
3.22.2.8 cycle	86
3.23 DNode Struct Reference	86
3.23.1 Detailed Description	86
3.23.2 Field Documentation	86
3.23.2.1 extloop	86
3.23.2.2 intrct	86
3.24 dollar_buf Struct Reference	87
3.24.1 Detailed Description	87
3.25 DoLIaRS Struct Reference	87
3.25.1 Detailed Description	87
3.25.2 Field Documentation	88
3.25.2.1 where	88
3.25.2.2 factors	88
3.25.2.3 size	88
3.25.2.4 name	88
3.25.2.5 type	88
3.25.2.6 node	88
3.25.2.7 index	88
3.25.2.8 zero	89
3.25.2.9 numdummies	89
3.25.2.10 nfactors	89
3.26 DoLoOp Struct Reference	89

3.26.1 Detailed Description	90
3.26.2 Field Documentation	90
3.26.2.1 p	90
3.26.2.2 name	90
3.26.2.3 vars	91
3.26.2.4 contents	91
3.26.2.5 dollarname	91
3.26.2.6 startlinenumber	91
3.26.2.7 firstnum	91
3.26.2.8 lastnum	91
3.26.2.9 incnum	91
3.26.2.10 type	92
3.26.2.11 NoShowInput	92
3.26.2.12 errorsinloop	92
3.26.2.13 firstloopcall	92
3.26.2.14 firstdollar	92
3.26.2.15 lastdollar	92
3.26.2.16 incdollar	92
3.26.2.17 NumPreTypes	92
3.26.2.18 PriefLevel	93
3.26.2.19 PreSwitchLevel	93
3.27 DuBiOuS Struct Reference	93
3.27.1 Detailed Description	93
3.27.2 Field Documentation	93
3.27.2.1 name	93
3.27.2.2 node	93
3.27.2.3 dummy	94
3.28 ECand Class Reference	94
3.28.1 Detailed Description	94
3.28.2 Constructor & Destructor Documentation	94
3.28.2.1 ECand()	94
3.28.2.2 ~ECand()	94
3.28.3 Member Function Documentation	95
3.28.3.1 prECand()	95
3.28.4 Field Documentation	95
3.28.4.1 det	95
3.28.4.2 nplist	95
3.28.4.3 plist	95
3.29 EEdge Class Reference	95
3.29.1 Detailed Description	96
3.29.2 Constructor & Destructor Documentation	96
3.29.2.1 EEdge() [1/2]	96

3.29.2.2 EEdge() [2/2]	96
3.29.2.3 ~EEdge()	97
3.29.3 Member Function Documentation	97
3.29.3.1 copy()	97
3.29.3.2 setId()	97
3.29.3.3 print()	97
3.29.3.4 setType()	97
3.29.3.5 setLMom()	97
3.29.3.6 setEMom()	98
3.29.4 Field Documentation	98
3.29.4.1 egraph	98
3.29.4.2 id	98
3.29.4.3 ext	98
3.29.4.4 ptcl	98
3.29.4.5 deleted	98
3.29.4.6 nodes	98
3.29.4.7 nlegs	99
3.29.4.8 emom	99
3.29.4.9 lmom	99
3.29.4.10 extMom	99
3.29.4.11 cut	99
3.29.4.12 visited	99
3.29.4.13 conid	99
3.29.4.14 edtype	99
3.29.4.15 opicomp	100
3.29.4.16 dir	100
3.30 EFLine Class Reference	100
3.30.1 Detailed Description	100
3.30.2 Constructor & Destructor Documentation	100
3.30.2.1 EFLine()	100
3.30.3 Member Function Documentation	101
3.30.3.1 print()	101
3.30.4 Field Documentation	101
3.30.4.1 elist	101
3.30.4.2 ftype	101
3.30.4.3 fkind	101
3.30.4.4 nlist	101
3.31 EGraph Class Reference	102
3.31.1 Detailed Description	103
3.31.2 Constructor & Destructor Documentation	104
3.31.2.1 EGraph()	104
3.31.2.2 ~EGraph()	104

3.31.3 Member Function Documentation	104
3.31.3.1 copy()	104
3.31.3.2 print()	104
3.31.3.3 fromDGraph()	104
3.31.3.4 fromMGraph()	104
3.31.3.5 isOptE()	105
3.31.3.6 setExtern()	105
3.31.3.7 isExternal()	105
3.31.3.8 isFermion()	105
3.31.3.9 setExtLoop()	105
3.31.3.10 endSetExtLoop()	105
3.31.3.11 connComp()	106
3.31.3.12 connVisit()	106
3.31.3.13 biconnE()	106
3.31.3.14 biinitE()	106
3.31.3.15 bisearchE()	106
3.31.3.16 findRoot()	106
3.31.3.17 dirEdge()	107
3.31.3.18 extMomConsv()	107
3.31.3.19 cmpMom()	107
3.31.3.20 groupLMom()	107
3.31.3.21 chkMomConsv()	107
3.31.3.22 prFLines()	107
3.31.3.23 getFLines()	108
3.31.3.24 fltrace()	108
3.31.3.25 addFLine()	108
3.31.3.26 legParticle()	108
3.31.4 Field Documentation	108
3.31.4.1 opt	108
3.31.4.2 model	108
3.31.4.3 proc	109
3.31.4.4 sproc	109
3.31.4.5 mgraph	109
3.31.4.6 econn	109
3.31.4.7 nodes	109
3.31.4.8 edges	109
3.31.4.9 mld	109
3.31.4.10 ald	109
3.31.4.11 sld	110
3.31.4.12 gSubld	110
3.31.4.13 assigned	110
3.31.4.14 fsign	110

3.31.4.15 nsym	110
3.31.4.16 esym	110
3.31.4.17 nsym1	110
3.31.4.18 extperm	110
3.31.4.19 multp	111
3.31.4.20 pld	111
3.31.4.21 sNodes	111
3.31.4.22 sEdges	111
3.31.4.23 sMaxdeg	111
3.31.4.24 sLoops	111
3.31.4.25 nNodes	111
3.31.4.26 nEdges	111
3.31.4.27 nExtern	112
3.31.4.28 maxdeg	112
3.31.4.29 nLoops	112
3.31.4.30 totalc	112
3.31.4.31 nopicomp	112
3.31.4.32 opi2plp	112
3.31.4.33 nopi2p	112
3.31.4.34 nadj2ptv	112
3.31.4.35 DUMMYPADDING	113
3.31.4.36 bidef	113
3.31.4.37 bilow	113
3.31.4.38 extMom	113
3.31.4.39 bconn	113
3.31.4.40 bicount	113
3.31.4.41 loopm	113
3.31.4.42 opiCount	113
3.31.4.43 flines	114
3.31.4.44 nflines	114
3.31.4.45 DUMMYPADDING1	114
3.32 ENode Class Reference	114
3.32.1 Detailed Description	115
3.32.2 Constructor & Destructor Documentation	115
3.32.2.1 ENode() [1/2]	115
3.32.2.2 ENode() [2/2]	115
3.32.2.3 ~ENode()	115
3.32.3 Member Function Documentation	116
3.32.3.1 initAss()	116
3.32.3.2 setId()	116
3.32.3.3 copy()	116
3.32.3.4 setExtern()	116

3.32.3.5 setType()	116
3.32.3.6 print()	116
3.32.4 Field Documentation	117
3.32.4.1 egraph	117
3.32.4.2 edges	117
3.32.4.3 id	117
3.32.4.4 maxdeg	117
3.32.4.5 deg	117
3.32.4.6 extloop	117
3.32.4.7 ndtype	117
3.32.4.8 intrct	118
3.32.4.9 klow	118
3.32.4.10 visited	118
3.32.4.11 DUMMYPADDING	118
3.33 EStack Class Reference	118
3.33.1 Detailed Description	118
3.33.2 Constructor & Destructor Documentation	119
3.33.2.1 EStack()	119
3.33.2.2 ~EStack()	119
3.33.3 Member Function Documentation	119
3.33.3.1 print()	119
3.33.4 Field Documentation	119
3.33.4.1 edgen	119
3.33.4.2 det	119
3.33.4.3 nplist	119
3.33.4.4 plist	120
3.34 ExPrEsSiOn Struct Reference	120
3.34.1 Detailed Description	121
3.34.2 Field Documentation	121
3.34.2.1 onfile	121
3.34.2.2 prototype	121
3.34.2.3 size	121
3.34.2.4 renum	122
3.34.2.5 bracketinfo	122
3.34.2.6 newbracketinfo	122
3.34.2.7 renumlists	122
3.34.2.8 inmem	122
3.34.2.9 counter	122
3.34.2.10 name	122
3.34.2.11 hidelevel	123
3.34.2.12 vflags	123
3.34.2.13 printfldg	123

3.34.2.14 status	123
3.34.2.15 replace	123
3.34.2.16 node	123
3.34.2.17 whichbuffer	123
3.34.2.18 namesize	123
3.34.2.19 compression	124
3.34.2.20 numdummies	124
3.34.2.21 numfactors	124
3.34.2.22 sizeprototype	124
3.34.2.23 uflags	124
3.35 FaCdOLaR Struct Reference	124
3.35.1 Detailed Description	124
3.35.2 Field Documentation	125
3.35.2.1 where	125
3.35.2.2 size	125
3.35.2.3 type	125
3.35.2.4 value	125
3.36 factorized_poly Class Reference	125
3.36.1 Detailed Description	126
3.36.2 Member Function Documentation	126
3.36.2.1 add_factor()	126
3.36.2.2 tostring()	126
3.36.3 Field Documentation	126
3.36.3.1 factor	126
3.36.3.2 power	126
3.37 FGInput Struct Reference	126
3.37.1 Detailed Description	127
3.37.2 Field Documentation	127
3.37.2.1 ninitl	127
3.37.2.2 initln	127
3.37.2.3 initlc	127
3.37.2.4 nfinal	127
3.37.2.5 finaln	127
3.37.2.6 finalc	127
3.37.2.7 coupl	128
3.38 FiLe Struct Reference	128
3.38.1 Detailed Description	129
3.38.2 Field Documentation	129
3.38.2.1 PPosition	129
3.38.2.2 filesize	129
3.38.2.3 PObuffer	129
3.38.2.4 PStop	129

3.38.2.5 POfill	129
3.38.2.6 POfull	129
3.38.2.7 name	130
3.38.2.8 numblocks	130
3.38.2.9 inbuffer	130
3.38.2.10 POfsize	130
3.38.2.11 handle	130
3.38.2.12 active	130
3.39 FiLeDaTa Struct Reference	131
3.39.1 Detailed Description	131
3.39.2 Field Documentation	132
3.39.2.1 Index	132
3.39.2.2 Fill	132
3.39.2.3 Position	132
3.39.2.4 Handle	132
3.39.2.5 dirtyflag	132
3.40 FiLeInDeX Struct Reference	133
3.40.1 Detailed Description	133
3.40.2 Field Documentation	134
3.40.2.1 next	134
3.40.2.2 number	134
3.40.2.3 expression	134
3.40.2.4 empty	134
3.41 FixedGlobals Struct Reference	134
3.41.1 Detailed Description	135
3.41.2 Field Documentation	135
3.41.2.1 Operation	135
3.41.2.2 OperaFind	135
3.41.2.3 VarType	135
3.41.2.4 ExprStat	136
3.41.2.5 FunNam	136
3.41.2.6 swmes	136
3.41.2.7 fname	136
3.41.2.8 fname2	136
3.41.2.9 s_one	136
3.41.2.10 fnamebase	136
3.41.2.11 fname2base	136
3.41.2.12 fname2size	137
3.41.2.13 fname2size	137
3.41.2.14 cTable	137
3.42 fixedset Struct Reference	137
3.42.1 Detailed Description	137

3.42.2 Field Documentation	137
3.42.2.1 name	137
3.42.2.2 description	138
3.42.2.3 type	138
3.42.2.4 dimension	138
3.43 flint::fmpz Class Reference	138
3.43.1 Detailed Description	138
3.43.2 Constructor & Destructor Documentation	138
3.43.2.1 fmpz()	138
3.43.2.2 ~fmpz()	139
3.43.3 Member Function Documentation	139
3.43.3.1 print()	139
3.43.4 Field Documentation	139
3.43.4.1 d	139
3.44 Fraction Class Reference	139
3.44.1 Detailed Description	140
3.44.2 Constructor & Destructor Documentation	140
3.44.2.1 Fraction() $[1/2]$	140
3.44.2.2 Fraction() $[2/2]$	140
3.44.3 Member Function Documentation	140
3.44.3.1 print()	140
3.44.3.2 setValue() $[1/2]$	140
3.44.3.3 setValue() $[2/2]$	140
3.44.3.4 add() $[1/2]$	141
3.44.3.5 add() $[2/2]$	141
3.44.3.6 sub()	141
3.44.3.7 gcd()	141
3.44.3.8 normal()	141
3.44.3.9 isEq()	141
3.44.4 Field Documentation	142
3.44.4.1 num	142
3.44.4.2 den	142
3.44.4.3 ratio	142
3.45 FUN_INFO Struct Reference	142
3.45.1 Detailed Description	142
3.45.2 Field Documentation	143
3.45.2.1 location	143
3.45.2.2 numargs	143
3.45.2.3 numfunnies	143
3.45.2.4 numwildcards	143
3.45.2.5 symmet	143
3.45.2.6 tensor	143

3.45.2.7 commute	143
3.46 FuNcTiOn Struct Reference	144
3.46.1 Detailed Description	144
3.46.2 Field Documentation	145
3.46.2.1 tabl	145
3.46.2.2 symminfo	145
3.46.2.3 name	145
3.46.2.4 commute	145
3.46.2.5 complex	145
3.46.2.6 number	146
3.46.2.7 flags	146
3.46.2.8 spec	146
3.46.2.9 symmetric	146
3.46.2.10 node	146
3.46.2.11 namesize	147
3.46.2.12 dimension	147
3.46.2.13 maxnumargs	147
3.46.2.14 minnumargs	147
3.47 gcd_heuristic_failed Class Reference	147
3.47.1 Detailed Description	147
3.48 HANDLERS Struct Reference	147
3.48.1 Detailed Description	148
3.48.2 Field Documentation	148
3.48.2.1 newlogonly	148
3.48.2.2 newhandle	148
3.48.2.3 oldhandle	148
3.48.2.4 oldlogonly	148
3.48.2.5 oldprinttype	148
3.48.2.6 oldsilent	149
3.49 IInput Struct Reference	149
3.49.1 Detailed Description	149
3.49.2 Field Documentation	149
3.49.2.1 name	149
3.49.2.2 icode	149
3.49.2.3 nplistn	149
3.49.2.4 plistn	150
3.49.2.5 plistc	150
3.49.2.6 cvallist	150
3.50 InDeX Struct Reference	150
3.50.1 Detailed Description	150
3.50.2 Field Documentation	150
3.50.2.1 name	150

3.50.2.2 type	151
3.50.2.3 dimension	151
3.50.2.4 number	151
3.50.2.5 flags	151
3.50.2.6 nmin4	151
3.50.2.7 node	151
3.50.2.8 namesize	151
3.51 indexblock Struct Reference	152
3.51.1 Detailed Description	152
3.51.2 Field Documentation	152
3.51.2.1 flags	152
3.51.2.2 previousblock	152
3.51.2.3 position	152
3.51.2.4 objects	153
3.52 InDeXeNtRy Struct Reference	153
3.52.1 Detailed Description	154
3.52.2 Field Documentation	154
3.52.2.1 position	154
3.52.2.2 length	154
3.52.2.3 variables	154
3.52.2.4 CompressSize	154
3.52.2.5 nsymbols	154
3.52.2.6 nindices	155
3.52.2.7 nvectors	155
3.52.2.8 nfunctions	155
3.52.2.9 size	155
3.52.2.10 name	155
3.53 iniinfo Struct Reference	156
3.53.1 Detailed Description	156
3.53.2 Field Documentation	156
3.53.2.1 entriesinindex	156
3.53.2.2 numberofindexblocks	156
3.53.2.3 firstindexblock	156
3.53.2.4 lastindexblock	156
3.53.2.5 numberoftables	157
3.53.2.6 numberofnamesblocks	157
3.53.2.7 firstnameblock	157
3.53.2.8 lastnameblock	157
3.54 INSIDEINFO Struct Reference	157
3.54.1 Detailed Description	158
3.54.2 Field Documentation	158
3.54.2.1 buffer	158

3.54.2.2 oldcompiletype	158
3.54.2.3 oldparallelflag	158
3.54.2.4 oldnumpotmoddollars	158
3.54.2.5 size	158
3.54.2.6 numdollars	158
3.54.2.7 oldcbuf	159
3.54.2.8 oldrbuf	159
3.54.2.9 inscbuf	159
3.54.2.10 oldcnumlhs	159
3.55 Interaction Class Reference	159
3.55.1 Detailed Description	160
3.55.2 Constructor & Destructor Documentation	160
3.55.2.1 Interaction()	160
3.55.2.2 ~Interaction()	160
3.55.3 Member Function Documentation	161
3.55.3.1 prInteraction()	161
3.55.4 Field Documentation	161
3.55.4.1 mdl	161
3.55.4.2 name	161
3.55.4.3 plist	161
3.55.4.4 clist	161
3.55.4.5 slist	161
3.55.4.6 id	161
3.55.4.7 csum	162
3.55.4.8 nlegs	162
3.55.4.9 loop	162
3.55.4.10 icode	162
3.55.4.11 nplist	162
3.55.4.12 nclist	162
3.55.4.13 nslist	162
3.56 KEYWORD Struct Reference	163
3.56.1 Detailed Description	163
3.56.2 Field Documentation	163
3.56.2.1 name	163
3.56.2.2 func	163
3.56.2.3 type	163
3.56.2.4 flags	163
3.57 KEYWORDV Struct Reference	164
3.57.1 Detailed Description	164
3.57.2 Field Documentation	164
3.57.2.1 name	164
3.57.2.2 var	164

3.57.2.3 type	164
3.57.2.4 flags	164
3.58 LIST Struct Reference	165
3.58.1 Detailed Description	165
3.58.2 Field Documentation	165
3.58.2.1 lijst	165
3.58.2.2 message	165
3.58.2.3 num	165
3.58.2.4 maxnum	166
3.58.2.5 size	166
3.58.2.6 numglobal	166
3.58.2.7 numtemp	166
3.58.2.8 numclear	166
3.59 longMultiStruct Struct Reference	167
3.59.1 Detailed Description	167
3.59.2 Field Documentation	167
3.59.2.1 buffer	167
3.59.2.2 bufpos	167
3.59.2.3 packpos	167
3.59.2.4 nPacks	168
3.59.2.5 lastLen	168
3.59.2.6 next	168
3.60 M_const Struct Reference	168
3.60.1 Detailed Description	172
3.60.2 Field Documentation	172
3.60.2.1 zeropos	172
3.60.2.2 S0	172
3.60.2.3 gcmmod	172
3.60.2.4 gpowmod	172
3.60.2.5 TempDir	173
3.60.2.6 TempSortDir	173
3.60.2.7 IncDir	173
3.60.2.8 InputFileName	173
3.60.2.9 LogFileName	173
3.60.2.10 OutBuffer	173
3.60.2.11 Path	173
3.60.2.12 SetupDir	173
3.60.2.13 SetupFile	174
3.60.2.14 gFortran90Kind	174
3.60.2.15 gextrasym	174
3.60.2.16 ggextrasym	174
3.60.2.17 oldnumextrasymbols	174

3.60.2.18 SpectatorFiles	174
3.60.2.19 MaxTer	174
3.60.2.20 CompressSize	174
3.60.2.21 ScratSize	175
3.60.2.22 HideSize	175
3.60.2.23 SizeStoreCache	175
3.60.2.24 MaxStreamSize	175
3.60.2.25 SIOsize	175
3.60.2.26 SLargeSize	175
3.60.2.27 SSmallEsize	175
3.60.2.28 SSmallSize	175
3.60.2.29 STermsInSmall	176
3.60.2.30 MaxBracketBufferSize	176
3.60.2.31 hProcessBucketSize	176
3.60.2.32 gProcessBucketSize	176
3.60.2.33 shmWinSize	176
3.60.2.34 OldChildTime	176
3.60.2.35 OldSecTime	176
3.60.2.36 OldMilliTime	176
3.60.2.37 WorkSize	177
3.60.2.38 gThreadBucketSize	177
3.60.2.39 ggThreadBucketSize	177
3.60.2.40 SumTime	177
3.60.2.41 SpectatorSize	177
3.60.2.42 TimeLimit	177
3.60.2.43 FileOnlyFlag	177
3.60.2.44 Interact	177
3.60.2.45 MaxParLevel	178
3.60.2.46 OutBufSize	178
3.60.2.47 SMaxFpatches	178
3.60.2.48 SMaxPatches	178
3.60.2.49 StdOut	178
3.60.2.50 ginsidefirst	178
3.60.2.51 gDefDim	178
3.60.2.52 gDefDim4	178
3.60.2.53 NumFixedSets	179
3.60.2.54 NumFixedFunctions	179
3.60.2.55 rbufnum	179
3.60.2.56 dbufnum	179
3.60.2.57 sbufnum	179
3.60.2.58 zbufnum	179
3.60.2.59 SkipClears	179

3.60.2.60 gTokensWriteFlag	179
3.60.2.61 gfunpowers	180
3.60.2.62 gStatsFlag	180
3.60.2.63 gNamesFlag	180
3.60.2.64 gCodesFlag	180
3.60.2.65 gSortType	180
3.60.2.66 gproperorderflag	180
3.60.2.67 hparallelflag	180
3.60.2.68 gparallelflag	180
3.60.2.69 totalnumberofthreads	181
3.60.2.70 gSizeCommuteInSet	181
3.60.2.71 gThreadStats	181
3.60.2.72 ggThreadStats	181
3.60.2.73 gFinalStats	181
3.60.2.74 ggFinalStats	181
3.60.2.75 gThreadsFlag	181
3.60.2.76 ggThreadsFlag	181
3.60.2.77 gThreadBalancing	182
3.60.2.78 ggThreadBalancing	182
3.60.2.79 gThreadSortFileSynch	182
3.60.2.80 ggThreadSortFileSynch	182
3.60.2.81 gProcessStats	182
3.60.2.82 ggProcessStats	182
3.60.2.83 gOldParallelStats	182
3.60.2.84 ggOldParallelStats	182
3.60.2.85 maxFlevels	183
3.60.2.86 resetTimeOnClear	183
3.60.2.87 gcNumDollars	183
3.60.2.88 MultiRun	183
3.60.2.89 gNoSpacesInNumbers	183
3.60.2.90 ggNoSpacesInNumbers	183
3.60.2.91 gIsFortran90	183
3.60.2.92 PrintTotalSize	183
3.60.2.93 fbufferSize	184
3.60.2.94 gOldFactArgFlag	184
3.60.2.95 ggOldFactArgFlag	184
3.60.2.96 gnumextrasym	184
3.60.2.97 ggnumextrasym	184
3.60.2.98 NumSpectatorFiles	184
3.60.2.99 SizeForSpectatorFiles	184
3.60.2.100 gOldGCDflag	184
3.60.2.101 ggOldGCDflag	185

3.60.2.102 gWTimeStatsFlag	185
3.60.2.103 ggWTimeStatsFlag	185
3.60.2.104 jumpratio	185
3.60.2.105 MaxTal	185
3.60.2.106 IndDum	185
3.60.2.107 DumInd	185
3.60.2.108 Willnd	185
3.60.2.109 gncmod	186
3.60.2.110 gnpowmod	186
3.60.2.111 gmodmode	186
3.60.2.112 gUnitTrace	186
3.60.2.113 gOutputMode	186
3.60.2.114 gOutputSpaces	186
3.60.2.115 gOutNumberType	186
3.60.2.116 gCnumpows	186
3.60.2.117 gUniTrace	187
3.60.2.118 MaxWildcards	187
3.60.2.119 mTraceDum	187
3.60.2.120 OffsetIndex	187
3.60.2.121 OffsetVector	187
3.60.2.122 RepMax	187
3.60.2.123 LogType	187
3.60.2.124 ggStatsFlag	187
3.60.2.125 gLineLength	188
3.60.2.126 qError	188
3.60.2.127 FortranCont	188
3.60.2.128 HoldFlag	188
3.60.2.129 Ordering	188
3.60.2.130 silent	188
3.60.2.131 tracebackflag	188
3.60.2.132 expnum	188
3.60.2.133 denomnum	189
3.60.2.134 facnum	189
3.60.2.135 invfacnum	189
3.60.2.136 sumnum	189
3.60.2.137 sumpnum	189
3.60.2.138 OldOrderFlag	189
3.60.2.139 termfunnum	189
3.60.2.140 matchfunnum	189
3.60.2.141 countfunnum	190
3.60.2.142 gPolyFun	190
3.60.2.143 gPolyFunInv	190

3.60.2.144 gPolyFunType	190
3.60.2.145 gPolyFunExp	190
3.60.2.146 gPolyFunVar	190
3.60.2.147 gPolyFunPow	190
3.60.2.148 dollarzero	190
3.60.2.149 atstartup	191
3.60.2.150 exitflag	191
3.60.2.151 NumStoreCaches	191
3.60.2.152 gIndentSpace	191
3.60.2.153 ggIndentSpace	191
3.60.2.154 gShortStatsMax	191
3.60.2.155 ggShortStatsMax	191
3.60.2.156 gextrasyms	192
3.60.2.157 ggextrasyms	192
3.60.2.158 zerorhs	192
3.60.2.159 onerhs	192
3.60.2.160 havesortdir	192
3.60.2.161 vectorzero	192
3.60.2.162 ClearStore	192
3.60.2.163 BracketFactors	192
3.60.2.164 FromStdin	193
3.60.2.165 IgnoreDeprecation	193
3.61 MCBLOCK Class Reference	193
3.61.1 Detailed Description	193
3.61.2 Constructor & Destructor Documentation	193
3.61.2.1 MCBLOCK()	193
3.61.2.2 ~MCBLOCK()	194
3.61.3 Member Function Documentation	194
3.61.3.1 init()	194
3.61.4 Field Documentation	194
3.61.4.1 edges	194
3.61.4.2 nmedges	194
3.61.4.3 nartps	194
3.61.4.4 loop	194
3.61.4.5 DUMMYPADDING	195
3.62 MCBridge Class Reference	195
3.62.1 Detailed Description	195
3.62.2 Constructor & Destructor Documentation	195
3.62.2.1 MCBridge()	195
3.62.2.2 ~MCBridge()	195
3.62.3 Field Documentation	195
3.62.3.1 nodes	195

3.62.3.2 next	196
3.63 MConn Class Reference	196
3.63.1 Detailed Description	197
3.63.2 Constructor & Destructor Documentation	197
3.63.2.1 MConn()	197
3.63.2.2 ~MConn()	197
3.63.3 Member Function Documentation	198
3.63.3.1 init()	198
3.63.3.2 pushNode()	198
3.63.3.3 pushEdge()	198
3.63.3.4 addOPlc()	198
3.63.3.5 addBridge()	198
3.63.3.6 addArtic()	198
3.63.3.7 addBlock()	199
3.63.3.8 addBlockSelf()	199
3.63.3.9 print()	199
3.63.4 Field Documentation	199
3.63.4.1 opics	199
3.63.4.2 bridges	199
3.63.4.3 blocks	199
3.63.4.4 articuls	199
3.63.4.5 opisp	200
3.63.4.6 opistk	200
3.63.4.7 blksp	200
3.63.4.8 blkstk	200
3.63.4.9 snodes	200
3.63.4.10 sedges	200
3.63.4.11 nopic	200
3.63.4.12 nlpopic	200
3.63.4.13 nctopic	201
3.63.4.14 nbridges	201
3.63.4.15 ne0bridges	201
3.63.4.16 ne1bridges	201
3.63.4.17 nselfloops	201
3.63.4.18 nblocks	201
3.63.4.19 na1blocks	201
3.63.4.20 narticuls	201
3.63.4.21 neblocks	202
3.63.4.22 nopisp	202
3.63.4.23 opistkptr	202
3.63.4.24 nblksp	202
3.63.4.25 blkstkptr	202

3.63.4.26 DUMMYPADDING	202
3.64 MCOpi Class Reference	202
3.64.1 Detailed Description	203
3.64.2 Constructor & Destructor Documentation	203
3.64.2.1 MCOpi()	203
3.64.2.2 ~MCOpi()	203
3.64.3 Member Function Documentation	203
3.64.3.1 init()	203
3.64.4 Field Documentation	203
3.64.4.1 nodes	203
3.64.4.2 nnodes	204
3.64.4.3 nlegs	204
3.64.4.4 next	204
3.64.4.5 nedges	204
3.64.4.6 loop	204
3.64.4.7 ctloop	204
3.65 MGraph Class Reference	205
3.65.1 Detailed Description	206
3.65.2 Constructor & Destructor Documentation	206
3.65.2.1 MGraph() [1/2]	206
3.65.2.2 MGraph() [2/2]	207
3.65.2.3 ~MGraph()	207
3.65.3 Member Function Documentation	207
3.65.3.1 init()	207
3.65.3.2 generate()	207
3.65.3.3 isExternal()	207
3.65.3.4 printAdjMat()	207
3.65.3.5 print()	208
3.65.3.6 isConnected()	208
3.65.3.7 visit()	208
3.65.3.8 isIsomorphic()	208
3.65.3.9 permMat()	208
3.65.3.10 compMat()	208
3.65.3.11 refineClass()	209
3.65.3.12 bisearchME()	209
3.65.3.13 biconnME()	209
3.65.3.14 isOptM()	209
3.65.3.15 connectClass()	209
3.65.3.16 connectNode()	209
3.65.3.17 connectLeg()	210
3.65.3.18 newGraph()	210
3.65.4 Field Documentation	210

3.65.4.1 opt	210
3.65.4.2 group	210
3.65.4.3 orbits	210
3.65.4.4 nodes	210
3.65.4.5 mld	211
3.65.4.6 clist	211
3.65.4.7 pld	211
3.65.4.8 nNodes	211
3.65.4.9 nEdges	211
3.65.4.10 nLoops	211
3.65.4.11 nExtern	211
3.65.4.12 nClasses	211
3.65.4.13 mindeg	212
3.65.4.14 maxdeg	212
3.65.4.15 adjMat	212
3.65.4.16 curcl	212
3.65.4.17 egraph	212
3.65.4.18 nsym	212
3.65.4.19 esym	212
3.65.4.20 cDiag	212
3.65.4.21 c1PI	213
3.65.4.22 cNoTadpole	213
3.65.4.23 cNoTadBlock	213
3.65.4.24 c1PINoTadBlock	213
3.65.4.25 wscon	213
3.65.4.26 wsopi	213
3.65.4.27 ngen	213
3.65.4.28 ngconn	213
3.65.4.29 nCallRefine	214
3.65.4.30 discardRefine	214
3.65.4.31 discardDisc	214
3.65.4.32 discardIso	214
3.65.4.33 opi	214
3.65.4.34 opiloop	214
3.65.4.35 extself	214
3.65.4.36 selfloop	214
3.65.4.37 tadpole	215
3.65.4.38 tadblock	215
3.65.4.39 block	215
3.65.4.40 DUMMYPADDING1	215
3.65.4.41 mconn	215
3.65.4.42 modmat	215

3.65.4.43 bidef	215
3.65.4.44 bilow	215
3.65.4.45 bicount	216
3.65.4.46 DUMMYPADDING2	216
3.66 MiNmAx Struct Reference	216
3.66.1 Detailed Description	216
3.66.2 Field Documentation	216
3.66.2.1 mini	216
3.66.2.2 maxi	216
3.66.2.3 size	217
3.67 MInput Struct Reference	217
3.67.1 Detailed Description	217
3.67.2 Field Documentation	217
3.67.2.1 name	217
3.67.2.2 defpart	217
3.67.2.3 ncouple	217
3.67.2.4 cnamlist	218
3.68 MNode Class Reference	218
3.68.1 Detailed Description	218
3.68.2 Constructor & Destructor Documentation	218
3.68.2.1 MNode() [1/2]	218
3.68.2.2 MNode() [2/2]	219
3.68.3 Field Documentation	219
3.68.3.1 id	219
3.68.3.2 deg	219
3.68.3.3 cls	219
3.68.3.4 extloop	219
3.68.3.5 cmindeg	219
3.68.3.6 cmaxdeg	220
3.68.3.7 freelg	220
3.68.3.8 visited	220
3.69 MNodeClass Class Reference	220
3.69.1 Detailed Description	221
3.69.2 Constructor & Destructor Documentation	221
3.69.2.1 MNodeClass()	221
3.69.2.2 ~MNodeClass()	221
3.69.3 Member Function Documentation	221
3.69.3.1 init()	221
3.69.3.2 copy()	221
3.69.3.3 clCmp()	221
3.69.3.4 printMat()	222
3.69.3.5 mkFlist()	222

3.69.3.6 mkNdCl()	222
3.69.3.7 mkClMat()	222
3.69.3.8 incMat()	222
3.69.3.9 cmpMNCArry()	222
3.69.3.10 reorder()	223
3.69.4 Field Documentation	223
3.69.4.1 clmat	223
3.69.4.2 clist	223
3.69.4.3 ndcl	223
3.69.4.4 flist	223
3.69.4.5 clord	223
3.69.4.6 cmindeg	223
3.69.4.7 cmaxdeg	224
3.69.4.8 nNodes	224
3.69.4.9 nClasses	224
3.69.4.10 maxdeg	224
3.69.4.11 flg0	224
3.69.4.12 flg1	224
3.69.4.13 flg2	224
3.70 Model Class Reference	225
3.70.1 Detailed Description	226
3.70.2 Constructor & Destructor Documentation	226
3.70.2.1 Model()	226
3.70.2.2 ~Model()	226
3.70.3 Member Function Documentation	226
3.70.3.1 prModel()	226
3.70.3.2 addParticle()	226
3.70.3.3 addParticleEnd()	227
3.70.3.4 addInteraction()	227
3.70.3.5 addInteractionEnd()	227
3.70.3.6 findParticleName()	227
3.70.3.7 findParticleCode()	227
3.70.3.8 findInteractionName()	227
3.70.3.9 findInteractionCode()	227
3.70.3.10 particleName()	228
3.70.3.11 particleCode()	228
3.70.3.12 normalParticle()	228
3.70.3.13 antiParticle()	228
3.70.3.14 allParticles()	228
3.70.3.15 findMClass()	228
3.70.3.16 prParticleArray()	228
3.70.4 Field Documentation	229

3.70.4.1 name	229
3.70.4.2 cnlist	229
3.70.4.3 ncouple	229
3.70.4.4 nParticles	229
3.70.4.5 particles	229
3.70.4.6 pdef	229
3.70.4.7 nallPart	229
3.70.4.8 allPart	230
3.70.4.9 nintPart	230
3.70.4.10 intPart	230
3.70.4.11 nInteracts	230
3.70.4.12 interacts	230
3.70.4.13 vdef	230
3.70.4.14 maxnlegs	230
3.70.4.15 maxcpl	230
3.70.4.16 maxloop	231
3.70.4.17 cplgcp	231
3.70.4.18 cplglg	231
3.70.4.19 cplgnvl	231
3.70.4.20 cplgvl	231
3.70.4.21 ncplgcp	231
3.70.4.22 defpart	231
3.70.4.23 skipFLine	231
3.70.4.24 DUMMY_PADDING	232
3.71 MoDeL Struct Reference	232
3.71.1 Detailed Description	233
3.71.2 Field Documentation	233
3.71.2.1 vertices	233
3.71.2.2 couplings	233
3.71.2.3 name	233
3.71.2.4 grccmodel	233
3.71.2.5 legcouple	233
3.71.2.6 nparticles	233
3.71.2.7 nvertices	234
3.71.2.8 invertices	234
3.71.2.9 sizevertices	234
3.71.2.10 sizecouplings	234
3.71.2.11 ncouplings	234
3.71.2.12 error	234
3.71.2.13 dummy	234
3.72 MODNUM Struct Reference	235
3.72.1 Detailed Description	235

3.72.2 Field Documentation	235
3.72.2.1 a	235
3.72.2.2 m	235
3.72.2.3 na	235
3.72.2.4 nm	235
3.73 MoDoPtDoLIaRS Struct Reference	236
3.73.1 Detailed Description	236
3.73.2 Field Documentation	236
3.73.2.1 number	236
3.73.2.2 type	236
3.74 monomial_larger Class Reference	236
3.74.1 Detailed Description	236
3.74.2 Constructor & Destructor Documentation	236
3.74.2.1 monomial_larger()	236
3.74.3 Member Function Documentation	237
3.74.3.1 operator()()	237
3.75 MORbits Class Reference	237
3.75.1 Detailed Description	237
3.75.2 Constructor & Destructor Documentation	237
3.75.2.1 MORbits()	237
3.75.2.2 ~MORbits()	238
3.75.3 Member Function Documentation	238
3.75.3.1 print()	238
3.75.3.2 initPerm()	238
3.75.3.3 fromPerm()	238
3.75.3.4 toOrbits()	238
3.75.4 Field Documentation	238
3.75.4.1 nOrbits	238
3.75.4.2 nNodes	239
3.75.4.3 nd2or	239
3.75.4.4 or2nd	239
3.75.4.5 flist	239
3.76 flint::mpoly Class Reference	239
3.76.1 Detailed Description	239
3.76.2 Constructor & Destructor Documentation	240
3.76.2.1 mpoly()	240
3.76.2.2 ~mpoly()	240
3.76.3 Member Function Documentation	240
3.76.3.1 print()	240
3.76.4 Field Documentation	240
3.76.4.1 d	240
3.77 flint::mpoly_ctx Class Reference	240

3.77.1 Detailed Description	241
3.77.2 Constructor & Destructor Documentation	241
3.77.2.1 mpoly_ctx()	241
3.77.2.2 ~mpoly_ctx()	241
3.77.3 Field Documentation	241
3.77.3.1 d	241
3.78 flint::mpoly_factor Class Reference	241
3.78.1 Detailed Description	241
3.78.2 Constructor & Destructor Documentation	242
3.78.2.1 mpoly_factor()	242
3.78.2.2 ~mpoly_factor()	242
3.78.3 Field Documentation	242
3.78.3.1 d	242
3.79 N_const Struct Reference	242
3.79.1 Detailed Description	245
3.79.2 Field Documentation	245
3.79.2.1 OldPosIn	245
3.79.2.2 OldPosOut	245
3.79.2.3 theposition	245
3.79.2.4 EndNest	245
3.79.2.5 Frozen	245
3.79.2.6 FullProto	246
3.79.2.7 cTerm	246
3.79.2.8 RepPoint	246
3.79.2.9 WildValue	246
3.79.2.10 WildStop	246
3.79.2.11 argaddress	246
3.79.2.12 RepFunList	246
3.79.2.13 patstop	246
3.79.2.14 terstop	247
3.79.2.15 terstart	247
3.79.2.16 terfirstcomm	247
3.79.2.17 DumFound	247
3.79.2.18 DumPlace	247
3.79.2.19 DumFunPlace	247
3.79.2.20 UsedSymbol	247
3.79.2.21 UsedVector	247
3.79.2.22 UsedIndex	248
3.79.2.23 UsedFunction	248
3.79.2.24 MaskPointer	248
3.79.2.25 ForFindOnly	248
3.79.2.26 findTerm	248

3.79.2.27 findPattern	248
3.79.2.28 dummyrenumlist	248
3.79.2.29 funargs	248
3.79.2.30 funlocs	249
3.79.2.31 funinds	249
3.79.2.32 NoScrat2	249
3.79.2.33 ReplaceScrat	249
3.79.2.34 tracestack	249
3.79.2.35 selecttermundo	249
3.79.2.36 patternbuffer	249
3.79.2.37 termbuffer	249
3.79.2.38 PoinScrat	250
3.79.2.39 FunScrat	250
3.79.2.40 RenumScrat	250
3.79.2.41 FunInfo	250
3.79.2.42 SplitScrat	250
3.79.2.43 SplitScrat1	250
3.79.2.44 FunSorts	250
3.79.2.45 SoScratC	250
3.79.2.46 listinprint	251
3.79.2.47 currentTerm	251
3.79.2.48 arglist	251
3.79.2.49 tlistbuf	251
3.79.2.50 compressSpace	251
3.79.2.51 SHcombi	251
3.79.2.52 poly_vars	251
3.79.2.53 cmod	251
3.79.2.54 SHvar	252
3.79.2.55 deferskipped	252
3.79.2.56 InScrat	252
3.79.2.57 SplitScratSize	252
3.79.2.58 InScrat1	252
3.79.2.59 SplitScratSize1	252
3.79.2.60 ninterms	252
3.79.2.61 SHcombisize	252
3.79.2.62 NumTotWildArgs	253
3.79.2.63 UseFindOnly	253
3.79.2.64 UsedOtherFind	253
3.79.2.65 ErrorInDollar	253
3.79.2.66 numfargs	253
3.79.2.67 numflocs	253
3.79.2.68 nargs	253

3.79.2.69 tohunt	253
3.79.2.70 numoffuns	254
3.79.2.71 funisize	254
3.79.2.72 RSize	254
3.79.2.73 numtracesctack	254
3.79.2.74 intracestack	254
3.79.2.75 numfuninfo	254
3.79.2.76 NumFunSorts	254
3.79.2.77 MaxFunSorts	254
3.79.2.78 arglistsize	255
3.79.2.79 tlistsize	255
3.79.2.80 filenum	255
3.79.2.81 compressSize	255
3.79.2.82 polysortflag	255
3.79.2.83 nogroundlevel	255
3.79.2.84 subsubveto	255
3.79.2.85 MaxRenumScratch	255
3.79.2.86 oldtype	256
3.79.2.87 oldvalue	256
3.79.2.88 NumWild	256
3.79.2.89 RepFunNum	256
3.79.2.90 DisOrderFlag	256
3.79.2.91 WildDirt	256
3.79.2.92 NumFound	256
3.79.2.93 WildReserve	256
3.79.2.94 TelnFun	257
3.79.2.95 TeSuOut	257
3.79.2.96 WildArgs	257
3.79.2.97 WildEat	257
3.79.2.98 PolyNormFlag	257
3.79.2.99 PolyFunTodo	257
3.79.2.100 sizeselecttermundo	257
3.79.2.101 patternbuffersize	257
3.79.2.102 numlistinprint	258
3.79.2.103 ncmod	258
3.79.2.104 ExpectedSign	258
3.79.2.105 SignCheck	258
3.79.2.106 IndDum	258
3.79.2.107 poly_num_vars	258
3.79.2.108 idfunctionflag	258
3.79.2.109 poly_vars_type	259
3.79.2.110 tryterm	259

3.80 nameblock Struct Reference	259
3.80.1 Detailed Description	259
3.80.2 Field Documentation	259
3.80.2.1 previousblock	259
3.80.2.2 position	259
3.80.2.3 names	260
3.81 NaMeNode Struct Reference	260
3.81.1 Detailed Description	260
3.81.2 Field Documentation	260
3.81.2.1 name	260
3.81.2.2 parent	260
3.81.2.3 left	261
3.81.2.4 right	261
3.81.2.5 balance	261
3.81.2.6 type	261
3.81.2.7 number	261
3.82 NaMeSpAcE Struct Reference	262
3.82.1 Detailed Description	262
3.82.2 Field Documentation	262
3.82.2.1 previous	262
3.82.2.2 next	262
3.82.2.3 usernames	263
3.82.2.4 name	263
3.83 NaMeTree Struct Reference	263
3.83.1 Detailed Description	264
3.83.2 Field Documentation	264
3.83.2.1 namenode	264
3.83.2.2 namebuffer	264
3.83.2.3 nodesize	264
3.83.2.4 nodefill	264
3.83.2.5 namesize	264
3.83.2.6 namefill	265
3.83.2.7 oldnamefill	265
3.83.2.8 oldnodefill	265
3.83.2.9 globalnamefill	265
3.83.2.10 globalnodefill	265
3.83.2.11 clearnamefill	265
3.83.2.12 clearnodefill	266
3.83.2.13 headnode	266
3.84 NCand Class Reference	266
3.84.1 Detailed Description	266
3.84.2 Constructor & Destructor Documentation	266

3.84.2.1 NCand()	266
3.84.2.2 ~NCand()	267
3.84.3 Member Function Documentation	267
3.84.3.1 prNCand()	267
3.84.4 Field Documentation	267
3.84.4.1 deg	267
3.84.4.2 st	267
3.84.4.3 nilist	267
3.84.4.4 ilist	267
3.85 NCInput Struct Reference	268
3.85.1 Detailed Description	268
3.85.2 Field Documentation	268
3.85.2.1 clddeg	268
3.85.2.2 clnum	268
3.85.2.3 cltyp	268
3.85.2.4 cmind	268
3.85.2.5 cmaxd	268
3.85.2.6 ptcl	269
3.85.2.7 cple	269
3.86 NeStInG Struct Reference	269
3.86.1 Detailed Description	269
3.86.2 Field Documentation	269
3.86.2.1 termsize	269
3.86.2.2 funsize	269
3.86.2.3 argsize	270
3.87 NoDe Struct Reference	270
3.87.1 Detailed Description	270
3.87.2 Field Documentation	270
3.87.2.1 left	270
3.87.2.2 rght	271
3.87.2.3 lloser	271
3.87.2.4 rloser	271
3.87.2.5 lsrc	271
3.87.2.6 rsrc	271
3.88 node Struct Reference	271
3.88.1 Detailed Description	272
3.88.2 Constructor & Destructor Documentation	272
3.88.2.1 node() [1/2]	272
3.88.2.2 node() [2/2]	272
3.88.3 Member Function Documentation	272
3.88.3.1 cmp()	272
3.88.3.2 operator()()	273

3.88.3.3 calcHash()	273
3.88.4 Field Documentation	273
3.88.4.1 data	273
3.88.4.2 l	273
3.88.4.3 r	273
3.88.4.4 sign	273
3.88.4.5 hash	273
3.89 NodeEq Struct Reference	274
3.89.1 Detailed Description	274
3.89.2 Member Function Documentation	274
3.89.2.1 operator()	274
3.90 NodeHash Struct Reference	274
3.90.1 Detailed Description	274
3.90.2 Member Function Documentation	274
3.90.2.1 operator()	274
3.91 NStack Class Reference	275
3.91.1 Detailed Description	275
3.91.2 Constructor & Destructor Documentation	275
3.91.2.1 NStack()	275
3.91.2.2 ~NStack()	275
3.91.3 Member Function Documentation	275
3.91.3.1 print()	275
3.91.4 Field Documentation	275
3.91.4.1 noden	275
3.91.4.2 deg	276
3.91.4.3 st	276
3.91.4.4 nilist	276
3.91.4.5 ilist	276
3.92 O_const Struct Reference	276
3.92.1 Detailed Description	278
3.92.2 Field Documentation	278
3.92.2.1 SaveData	278
3.92.2.2 SaveHeader	278
3.92.2.3 OptimizeResult	278
3.92.2.4 OutputLine	279
3.92.2.5 OutStop	279
3.92.2.6 OutFill	279
3.92.2.7 bracket	279
3.92.2.8 termbuf	279
3.92.2.9 tabstring	279
3.92.2.10 wpos	279
3.92.2.11 wpoin	279

3.92.2.12 DollarOutBuffer	280
3.92.2.13 CurBufWrt	280
3.92.2.14 FlipWORD	280
3.92.2.15 FlipLONG	280
3.92.2.16 FlipPOS	280
3.92.2.17 FlipPOINTER	280
3.92.2.18 ResizeData	280
3.92.2.19 ResizeWORD	280
3.92.2.20 ResizeNCWORD	281
3.92.2.21 ResizeLONG	281
3.92.2.22 ResizePOS	281
3.92.2.23 ResizePOINTER	281
3.92.2.24 CheckPower	281
3.92.2.25 RenumberVec	281
3.92.2.26 Dictionaries	281
3.92.2.27 tensorList	281
3.92.2.28 inscheme	282
3.92.2.29 NumInBrack	282
3.92.2.30 wlen	282
3.92.2.31 DollarOutSizeBuffer	282
3.92.2.32 DollarInOutBuffer	282
3.92.2.33 Optimize	282
3.92.2.34 OutInBuffer	282
3.92.2.35 NoSpacesInNumbers	282
3.92.2.36 BlockSpaces	283
3.92.2.37 CurrentDictionary	283
3.92.2.38 SizeDictionaries	283
3.92.2.39 NumDictionaries	283
3.92.2.40 CurDictNumbers	283
3.92.2.41 CurDictVariables	283
3.92.2.42 CurDictSpecials	283
3.92.2.43 CurDictFunWithArgs	283
3.92.2.44 CurDictNumberWarning	284
3.92.2.45 CurDictNotInFunctions	284
3.92.2.46 CurDictInDollars	284
3.92.2.47 gNumDictionaries	284
3.92.2.48 schemenum	284
3.92.2.49 transFlag	284
3.92.2.50 powerFlag	284
3.92.2.51 mpower	284
3.92.2.52 resizeFlag	285
3.92.2.53 bufferedInd	285

3.92.2.54 OutSkip	285
3.92.2.55 IsBracket	285
3.92.2.56 InFbrack	285
3.92.2.57 PrintType	285
3.92.2.58 FortFirst	285
3.92.2.59 DoubleFlag	285
3.92.2.60 IndentSpace	286
3.92.2.61 FactorMode	286
3.92.2.62 FactorNum	286
3.92.2.63 ErrorBlock	286
3.92.2.64 OptimizationLevel	286
3.92.2.65 FortDotChar	286
3.93 objects Struct Reference	287
3.93.1 Detailed Description	287
3.93.2 Field Documentation	287
3.93.2.1 position	287
3.93.2.2 size	287
3.93.2.3 date	287
3.93.2.4 tablenumber	287
3.93.2.5 uncompressed	288
3.93.2.6 spare1	288
3.93.2.7 spare2	288
3.93.2.8 spare3	288
3.93.2.9 element	288
3.94 OptDef Struct Reference	288
3.94.1 Detailed Description	288
3.94.2 Field Documentation	289
3.94.2.1 name	289
3.94.2.2 mean	289
3.94.2.3 defaultv	289
3.94.2.4 DUMMY_PADDING	289
3.95 optimization Class Reference	289
3.95.1 Detailed Description	290
3.95.2 Description	290
3.95.3 Member Function Documentation	290
3.95.3.1 operator<()	290
3.95.4 Field Documentation	291
3.95.4.1 type	291
3.95.4.2 arg1	291
3.95.4.3 arg2	291
3.95.4.4 improve	291
3.95.4.5 coeff	291

3.95.4.6 eqnidxs	291
3.96 OPTIMIZE Struct Reference	292
3.96.1 Detailed Description	292
3.96.2 Field Documentation	292
3.96.2.1 fval	292
3.96.2.2 ival	292
3.96.2.3 horner	293
3.96.2.4 hornerdirection	293
3.96.2.5 method	293
3.96.2.6 mctstimelimit	293
3.96.2.7 mctsnumexpand	293
3.96.2.8 mctsnumkeep	293
3.96.2.9 mctsnumrepeat	293
3.96.2.10 greedytimelimit	293
3.96.2.11 greedyminnum	294
3.96.2.12 greedymaxperc	294
3.96.2.13 printstats	294
3.96.2.14 debugflags	294
3.96.2.15 schemeflags	294
3.96.2.16 mctsdecaymode	294
3.96.2.17 salter	294
3.96.2.18 spare	295
3.97 OPTIMIZERESULT Struct Reference	295
3.97.1 Detailed Description	295
3.97.2 Field Documentation	295
3.97.2.1 code	295
3.97.2.2 nameofexpr	295
3.97.2.3 codesize	296
3.97.2.4 exprnr	296
3.97.2.5 minvar	296
3.97.2.6 maxvar	296
3.98 Options Class Reference	296
3.98.1 Detailed Description	297
3.98.2 Constructor & Destructor Documentation	297
3.98.2.1 Options()	297
3.98.2.2 ~Options()	298
3.98.3 Member Function Documentation	298
3.98.3.1 setDefaultValue()	298
3.98.3.2 setValue()	298
3.98.3.3 getValue()	298
3.98.3.4 print()	298
3.98.3.5 setOutputF()	298

3.98.3.6 setOutputP()	299
3.98.3.7 printLevel()	299
3.98.3.8 printModel()	299
3.98.3.9 setOutMG()	299
3.98.3.10 setOutAG()	299
3.98.3.11 setEndMG()	299
3.98.3.12 setErExit()	300
3.98.3.13 getDef()	300
3.98.3.14 begin()	300
3.98.3.15 end()	300
3.98.3.16 beginProc()	300
3.98.3.17 endProc()	300
3.98.3.18 beginSubProc()	300
3.98.3.19 endSubProc()	301
3.98.3.20 newMGraph()	301
3.98.3.21 newAGraph()	301
3.98.3.22 outModel()	301
3.98.4 Field Documentation	301
3.98.4.1 model	301
3.98.4.2 proc	301
3.98.4.3 sproc	301
3.98.4.4 out	302
3.98.4.5 outmg	302
3.98.4.6 endmg	302
3.98.4.7 outag	302
3.98.4.8 argmg	302
3.98.4.9 argemg	302
3.98.4.10 argag	302
3.98.4.11 values	302
3.98.4.12 DUMMYPADDING	303
3.98.4.13 time0	303
3.98.4.14 time1	303
3.99 Output Class Reference	303
3.99.1 Detailed Description	304
3.99.2 Constructor & Destructor Documentation	304
3.99.2.1 Output()	304
3.99.2.2 ~Output()	305
3.99.3 Member Function Documentation	305
3.99.3.1 setOutgrf()	305
3.99.3.2 setOutgrp()	305
3.99.3.3 outBeginF()	305
3.99.3.4 outBeginP()	305

3.99.3.5 outEndF()	305
3.99.3.6 outEndP()	306
3.99.3.7 outProcBeginF()	306
3.99.3.8 outProcBeginP()	306
3.99.3.9 outProcBegin0()	306
3.99.3.10 outSProcBeginF()	306
3.99.3.11 outSProcBeginP()	306
3.99.3.12 outProcEndF()	306
3.99.3.13 outProcEndP()	307
3.99.3.14 outEGraphF()	307
3.99.3.15 outEGraphP()	307
3.99.3.16 outModelF()	307
3.99.3.17 outModelP()	307
3.99.4 Field Documentation	307
3.99.4.1 opt	307
3.99.4.2 model	307
3.99.4.3 proc	308
3.99.4.4 sproc	308
3.99.4.5 outgrf	308
3.99.4.6 outgrfp	308
3.99.4.7 outgrp	308
3.99.4.8 outgrpp	308
3.99.4.9 proclid	308
3.99.4.10 outproc	309
3.100 P_const Struct Reference	309
3.100.1 Detailed Description	310
3.100.2 Field Documentation	311
3.100.2.1 DollarList	311
3.100.2.2 PreVarList	311
3.100.2.3 LoopList	311
3.100.2.4 ProcList	311
3.100.2.5 inside	311
3.100.2.6 firstnamespace	311
3.100.2.7 lastnamespace	311
3.100.2.8 fullname	312
3.100.2.9 PreSwitchStrings	312
3.100.2.10 preStart	312
3.100.2.11 preStop	312
3.100.2.12 preFill	312
3.100.2.13 procedureExtension	312
3.100.2.14 cprocedureExtension	312
3.100.2.15 PreAssignStack	312

3.100.2.16 PelfStack	313
3.100.2.17 PreSwitchModes	313
3.100.2.18 PreTypes	313
3.100.2.19 StopWatchZero	313
3.100.2.20 InOutBuf	313
3.100.2.21 pSize	313
3.100.2.22 PreAssignFlag	313
3.100.2.23 PreContinuation	313
3.100.2.24 PreproFlag	314
3.100.2.25 iBufError	314
3.100.2.26 PreOut	314
3.100.2.27 PreSwitchLevel	314
3.100.2.28 NumPreSwitchStrings	314
3.100.2.29 MaxPreTypes	314
3.100.2.30 NumPreTypes	314
3.100.2.31 MaxPelfLevel	314
3.100.2.32 PelfLevel	315
3.100.2.33 PreInsideLevel	315
3.100.2.34 DelayPrevar	315
3.100.2.35 AllowDelay	315
3.100.2.36 lhdollarerror	315
3.100.2.37 eat	315
3.100.2.38 gNumPre	315
3.100.2.39 PreDebug	315
3.100.2.40 OpenDictionary	316
3.100.2.41 PreAssignLevel	316
3.100.2.42 MaxPreAssignLevel	316
3.100.2.43 fullnamesize	316
3.100.2.44 DebugFlag	316
3.100.2.45 preError	316
3.100.2.46 ComChar	316
3.100.2.47 cComChar	317
3.101 ParallelVars Struct Reference	317
3.101.1 Detailed Description	318
3.101.2 Field Documentation	318
3.101.2.1 slavebuf	318
3.101.2.2 sbuf	318
3.101.2.3 rbufs	318
3.101.2.4 me	318
3.101.2.5 numtasks	318
3.101.2.6 parallel	318
3.101.2.7 rhsInParallel	319

3.101.2.8 mkSlaveInfile	319
3.101.2.9 exprbufsize	319
3.101.2.10 exprtodo	319
3.101.2.11 log	319
3.101.2.12 numsbuffs	319
3.101.2.13 numrbuffs	319
3.102 PaRtl Struct Reference	320
3.102.1 Detailed Description	320
3.102.2 Field Documentation	320
3.102.2.1 psize	320
3.102.2.2 args	320
3.102.2.3 nargs	320
3.102.2.4 nfun	320
3.102.2.5 numargs	321
3.102.2.6 numpart	321
3.102.2.7 where	321
3.103 PaRtlcLe Struct Reference	321
3.103.1 Detailed Description	321
3.103.2 Field Documentation	321
3.103.2.1 number	321
3.103.2.2 spin	322
3.103.2.3 mass	322
3.103.2.4 type	322
3.104 Particle Class Reference	322
3.104.1 Detailed Description	323
3.104.2 Constructor & Destructor Documentation	323
3.104.2.1 Particle()	323
3.104.2.2 ~Particle()	323
3.104.3 Member Function Documentation	324
3.104.3.1 particleName()	324
3.104.3.2 particleCode()	324
3.104.3.3 aparticle()	324
3.104.3.4 prParticle()	324
3.104.3.5 isNeutral()	324
3.104.3.6 typeName()	324
3.104.3.7 typeGName()	325
3.104.4 Field Documentation	325
3.104.4.1 mdl	325
3.104.4.2 name	325
3.104.4.3 aname	325
3.104.4.4 id	325
3.104.4.5 ptype	325

3.104.4.6 neutral	325
3.104.4.7 pcode	326
3.104.4.8 acode	326
3.104.4.9 cmindeg	326
3.104.4.10 cmaxdeg	326
3.104.4.11 extonly	326
3.105 PeRmUtE Struct Reference	326
3.105.1 Detailed Description	327
3.105.2 Field Documentation	327
3.105.2.1 objects	327
3.105.2.2 sign	327
3.105.2.3 n	327
3.105.2.4 cycle	327
3.106 PeRmUtEp Struct Reference	327
3.106.1 Detailed Description	328
3.106.2 Field Documentation	328
3.106.2.1 objects	328
3.106.2.2 sign	328
3.106.2.3 n	328
3.106.2.4 cycle	328
3.107 PF_BUFFER Struct Reference	328
3.107.1 Detailed Description	329
3.107.2 Field Documentation	329
3.107.2.1 buff	329
3.107.2.2 fill	329
3.107.2.3 full	329
3.107.2.4 stop	329
3.107.2.5 status	330
3.107.2.6 retstat	330
3.107.2.7 request	330
3.107.2.8 type	330
3.107.2.9 index	330
3.107.2.10 tag	330
3.107.2.11 from	330
3.107.2.12 numbufs	330
3.107.2.13 active	331
3.108 PGInput Struct Reference	331
3.108.1 Detailed Description	331
3.108.2 Field Documentation	331
3.108.2.1 cdeg	331
3.108.2.2 ctyp	331
3.108.2.3 cnum	331

3.108.2.4 cple	332
3.108.2.5 pname	332
3.108.2.6 pcode	332
3.109 PInput Struct Reference	332
3.109.1 Detailed Description	332
3.109.2 Field Documentation	332
3.109.2.1 name	332
3.109.2.2 aname	333
3.109.2.3 pcode	333
3.109.2.4 acode	333
3.109.2.5 ptypen	333
3.109.2.6 ptypec	333
3.109.2.7 extonly	333
3.110 PNodeClass Class Reference	334
3.110.1 Detailed Description	334
3.110.2 Constructor & Destructor Documentation	335
3.110.2.1 PNodeClass() [1/2]	335
3.110.2.2 PNodeClass() [2/2]	335
3.110.2.3 ~PNodeClass()	335
3.110.3 Member Function Documentation	335
3.110.3.1 prPNodeClass()	335
3.110.3.2 prElem()	335
3.110.4 Field Documentation	336
3.110.4.1 sproc	336
3.110.4.2 deg	336
3.110.4.3 type	336
3.110.4.4 particle	336
3.110.4.5 couple	336
3.110.4.6 cmindeg	336
3.110.4.7 cmaxdeg	336
3.110.4.8 count	337
3.110.4.9 cl2nd	337
3.110.4.10 nd2cl	337
3.110.4.11 cl2mcl	337
3.110.4.12 nnodes	337
3.110.4.13 nclass	337
3.111 flint::poly Class Reference	337
3.111.1 Detailed Description	338
3.111.2 Constructor & Destructor Documentation	338
3.111.2.1 poly()	338
3.111.2.2 ~poly()	338
3.111.3 Member Function Documentation	338

3.111.3.1 print()	338
3.111.4 Field Documentation	338
3.111.4.1 d	338
3.112 poly Class Reference	339
3.112.1 Detailed Description	340
3.112.2 Constructor & Destructor Documentation	340
3.112.2.1 poly() [1/3]	340
3.112.2.2 poly() [2/3]	341
3.112.2.3 poly() [3/3]	341
3.112.2.4 ~poly()	341
3.112.3 Member Function Documentation	341
3.112.3.1 operator+=()	341
3.112.3.2 operator-=()	341
3.112.3.3 operator*=()	341
3.112.3.4 operator/=()	342
3.112.3.5 operator%=()	342
3.112.3.6 operator+()	342
3.112.3.7 operator-()	342
3.112.3.8 operator*()	342
3.112.3.9 operator/()	342
3.112.3.10 operator%()	342
3.112.3.11 operator==()	343
3.112.3.12 operator!=()	343
3.112.3.13 operator=()	343
3.112.3.14 operator[]() [1/2]	343
3.112.3.15 operator[]() [2/2]	343
3.112.3.16 termscopy()	343
3.112.3.17 check_memory()	343
3.112.3.18 expand_memory()	344
3.112.3.19 is_zero()	344
3.112.3.20 is_one()	344
3.112.3.21 is_integer()	344
3.112.3.22 is_monomial()	344
3.112.3.23 is_dense_univariate()	344
3.112.4 Description	344
3.112.5 Notes	345
3.112.5.1 sign()	345
3.112.5.2 degree()	345
3.112.5.3 total_degree()	345
3.112.5.4 first_variable()	345
3.112.5.5 number_of_terms()	345
3.112.5.6 all_variables()	345

3.112.5.7 integer_lcoeff()	346
3.112.5.8 lcoeff_univar()	346
3.112.5.9 lcoeff_multivar()	346
3.112.5.10 coefficient()	346
3.112.5.11 derivative()	346
3.112.5.12 setmod()	346
3.112.5.13 coefficients_modulo()	346
3.112.5.14 simple_poly() ^[1/2]	347
3.112.5.15 simple_poly() ^[2/2]	347
3.112.5.16 get_variables()	347
3.112.5.17 argument_to_poly()	347
3.112.5.18 poly_to_argument()	347
3.112.5.19 poly_to_argument_with_den()	348
3.112.5.20 size_of_form_notation()	348
3.112.5.21 size_of_form_notation_with_den()	348
3.112.5.22 normalize()	348
3.112.5.23 from_coefficient_list()	348
3.112.5.24 to_coefficient_list()	348
3.112.5.25 coefficient_list_divmod()	349
3.112.5.26 to_string()	349
3.112.5.27 monomial_compare()	349
3.112.5.28 last_monomial_index()	349
3.112.5.29 last_monomial()	349
3.112.5.30 compare_degree_vector()	349
3.112.5.31 degree_vector()	349
3.112.5.32 add()	350
3.112.5.33 sub()	350
3.112.5.34 mul()	350
3.112.6 Description	350
3.112.6.1 div()	351
3.112.6.2 mod()	351
3.112.6.3 divmod()	351
3.112.7 Description	351
3.112.7.1 divides()	352
3.112.7.2 mul_one_term()	352
3.112.7.3 mul_univar()	352
3.112.7.4 mul_heap()	352
3.112.8 Description	352
3.112.8.1 divmod_one_term()	353
3.112.8.2 divmod_univar()	353
3.112.9 Description	354
3.112.9.1 divmod_heap()	354

3.112.10 Description	354
3.112.10.1 push_heap()	355
3.112.10.2 pop_heap()	355
3.112.11 Field Documentation	356
3.112.11.1 terms	356
3.112.11.2 size_of_terms	356
3.112.11.3 modp	356
3.112.11.4 modn	356
3.113 flint::poly_factor Class Reference	356
3.113.1 Detailed Description	356
3.113.2 Constructor & Destructor Documentation	357
3.113.2.1 poly_factor()	357
3.113.2.2 ~poly_factor()	357
3.113.3 Field Documentation	357
3.113.3.1 d	357
3.114 POLYMOD Struct Reference	357
3.114.1 Detailed Description	357
3.114.2 Field Documentation	358
3.114.2.1 coefs	358
3.114.2.2 numsym	358
3.114.2.3 arraysize	358
3.114.2.4 polysize	358
3.114.2.5 modnum	358
3.115 PoSiTiOn Struct Reference	358
3.115.1 Detailed Description	358
3.115.2 Field Documentation	359
3.115.2.1 p1	359
3.116 PRELOAD Struct Reference	359
3.116.1 Detailed Description	359
3.116.2 Field Documentation	359
3.116.2.1 buffer	359
3.116.2.2 size	359
3.117 pReVaR Struct Reference	360
3.117.1 Detailed Description	360
3.117.2 Field Documentation	360
3.117.2.1 name	360
3.117.2.2 value	360
3.117.2.3 argnames	360
3.117.2.4 nargs	361
3.117.2.5 wildarg	361
3.118 PROCEDURE Struct Reference	361
3.118.1 Detailed Description	362

3.118.2 Field Documentation	362
3.118.2.1 p	362
3.118.2.2 name	362
3.118.2.3 loadmode	362
3.119 Process Class Reference	362
3.119.1 Detailed Description	363
3.119.2 Constructor & Destructor Documentation	364
3.119.2.1 Process()	364
3.119.2.2 ~Process()	364
3.119.3 Member Function Documentation	364
3.119.3.1 prProcess()	364
3.119.3.2 mkSProcess()	364
3.119.4 Field Documentation	364
3.119.4.1 model	364
3.119.4.2 opt	365
3.119.4.3 mgrcount	365
3.119.4.4 agrcount	365
3.119.4.5 initlPart	365
3.119.4.6 finalPart	365
3.119.4.7 id	365
3.119.4.8 ninitl	365
3.119.4.9 nfinal	365
3.119.4.10 ctotat	366
3.119.4.11 nExtern	366
3.119.4.12 loop	366
3.119.4.13 maxnlegs	366
3.119.4.14 clist	366
3.119.4.15 nSubproc	366
3.119.4.16 sptbl	366
3.119.4.17 sproc	366
3.119.4.18 astack	367
3.119.4.19 ngraphs	367
3.119.4.20 nopi	367
3.119.4.21 wgraphs	367
3.119.4.22 wopi	367
3.119.4.23 nMGraphs	367
3.119.4.24 nMOPI	367
3.119.4.25 wMGraphs	367
3.119.4.26 wMOPI	368
3.119.4.27 nAGraphs	368
3.119.4.28 nAOPI	368
3.119.4.29 wAGraphs	368

3.119.4.30 wAOPI	368
3.119.4.31 sec	368
3.120 R_const Struct Reference	369
3.120.1 Detailed Description	370
3.120.2 Field Documentation	371
3.120.2.1 StoreData	371
3.120.2.2 Fscr	371
3.120.2.3 FoStage4	371
3.120.2.4 DefPosition	371
3.120.2.5 infile	371
3.120.2.6 outfile	371
3.120.2.7 hidefile	371
3.120.2.8 CompressBuffer	372
3.120.2.9 ComprTop	372
3.120.2.10 CompressPointer	372
3.120.2.11 CompareRoutine	372
3.120.2.12 wranfia	372
3.120.2.13 moebiustable	372
3.120.2.14 OldTime	372
3.120.2.15 InInBuf	372
3.120.2.16 InHiBuf	373
3.120.2.17 pWorkSize	373
3.120.2.18 lWorkSize	373
3.120.2.19 posWorkSize	373
3.120.2.20 wranfseed	373
3.120.2.21 NoCompress	373
3.120.2.22 gzipCompress	373
3.120.2.23 Cnumlhs	373
3.120.2.24 outtohide	374
3.120.2.25 wranfcall	374
3.120.2.26 wranfnpair1	374
3.120.2.27 wranfnpair2	374
3.120.2.28 GetFile	374
3.120.2.29 KeptInHold	374
3.120.2.30 BracketOn	374
3.120.2.31 MaxBracket	374
3.120.2.32 CurDum	375
3.120.2.33 DeferFlag	375
3.120.2.34 TePos	375
3.120.2.35 sLevel	375
3.120.2.36 Stage4Name	375
3.120.2.37 GetOneFile	375

3.120.2.38 PolyFun	375
3.120.2.39 PolyFunInv	375
3.120.2.40 PolyFunType	376
3.120.2.41 PolyFunExp	376
3.120.2.42 PolyFunVar	376
3.120.2.43 PolyFunPow	376
3.120.2.44 Eside	376
3.120.2.45 MaxDum	376
3.120.2.46 level	376
3.120.2.47 expchanged	376
3.120.2.48 expflags	377
3.120.2.49 CurExpr	377
3.120.2.50 SortType	377
3.120.2.51 ShortSortCount	377
3.120.2.52 modeloptions	377
3.120.2.53 funoffset	377
3.120.2.54 moebiustablesizes	377
3.121 ReNuMbEr Struct Reference	378
3.121.1 Detailed Description	378
3.121.2 Field Documentation	378
3.121.2.1 startposition	378
3.121.2.2 symb	379
3.121.2.3 indi	379
3.121.2.4 vect	379
3.121.2.5 func	379
3.121.2.6 symnum	379
3.121.2.7 indnum	380
3.121.2.8 vecnum	380
3.121.2.9 funnum	380
3.122 S_const Struct Reference	380
3.122.1 Detailed Description	381
3.122.2 Field Documentation	381
3.122.2.1 MaxExprSize	381
3.122.2.2 OldOnFile	381
3.122.2.3 OldNumFactors	381
3.122.2.4 Oldvflags	382
3.122.2.5 Olduflags	382
3.122.2.6 NumOldOnFile	382
3.122.2.7 NumOldNumFactors	382
3.122.2.8 MultiThreaded	382
3.122.2.9 Balancing	382
3.122.2.10 ExecMode	382

3.122.2.11 CollectOverFlag	383
3.123 SeTs Struct Reference	383
3.123.1 Detailed Description	383
3.123.2 Field Documentation	383
3.123.2.1 name	383
3.123.2.2 type	383
3.123.2.3 first	383
3.123.2.4 last	384
3.123.2.5 node	384
3.123.2.6 namesize	384
3.123.2.7 dimension	384
3.123.2.8 flags	384
3.124 SETUPPARAMETERS Struct Reference	384
3.124.1 Detailed Description	385
3.124.2 Field Documentation	385
3.124.2.1 parameter	385
3.124.2.2 type	385
3.124.2.3 flags	385
3.124.2.4 value	385
3.125 SGroup Class Reference	385
3.125.1 Detailed Description	386
3.125.2 Constructor & Destructor Documentation	386
3.125.2.1 SGroup()	386
3.125.2.2 ~SGroup()	386
3.125.3 Member Function Documentation	386
3.125.3.1 print()	386
3.125.3.2 newGroup()	387
3.125.3.3 clearGroup()	387
3.125.3.4 delGroup()	387
3.125.3.5 genNext()	387
3.125.3.6 addGroup()	387
3.125.3.7 nElem()	387
3.125.3.8 nextElem()	387
3.125.4 Field Documentation	388
3.125.4.1 size	388
3.125.4.2 nelem	388
3.125.4.3 elem	388
3.125.4.4 nnodes	388
3.125.4.5 neclass	388
3.125.4.6 eclass	388
3.125.4.7 cgen	388
3.125.4.8 csav	389

3.125.4.9 permg	389
3.125.4.10 perms	389
3.125.4.11 pgr	389
3.125.4.12 pgq	389
3.125.4.13 psr	389
3.125.4.14 psq	389
3.126 SHvariables Struct Reference	390
3.126.1 Detailed Description	390
3.126.2 Field Documentation	390
3.126.2.1 outterm	390
3.126.2.2 outfun	390
3.126.2.3 incoef	390
3.126.2.4 stop1	390
3.126.2.5 stop2	391
3.126.2.6 ststop1	391
3.126.2.7 ststop2	391
3.126.2.8 finishuf	391
3.126.2.9 do_uffle	391
3.126.2.10 combilast	391
3.126.2.11 nincoef	391
3.126.2.12 level	391
3.126.2.13 thefunction	392
3.126.2.14 option	392
3.127 sOrT Struct Reference	392
3.127.1 Detailed Description	394
3.127.2 Field Documentation	394
3.127.2.1 file	394
3.127.2.2 SizeInFile	394
3.127.2.3 lBuffer	394
3.127.2.4 lTop	394
3.127.2.5 lFill	394
3.127.2.6 used	394
3.127.2.7 sBuffer	395
3.127.2.8 sTop	395
3.127.2.9 sTop2	395
3.127.2.10 sHalf	395
3.127.2.11 sFill	395
3.127.2.12 sPointer	395
3.127.2.13 PoinFill	395
3.127.2.14 cBuffer	395
3.127.2.15 Patches	396
3.127.2.16 pStop	396

3.127.2.17	poina	396
3.127.2.18	poin2a	396
3.127.2.19	ktoi	396
3.127.2.20	tree	396
3.127.2.21	fPatches	396
3.127.2.22	inPatches	396
3.127.2.23	fPatchesStop	397
3.127.2.24	iPatches	397
3.127.2.25	f	397
3.127.2.26	ff	397
3.127.2.27	sTerms	397
3.127.2.28	LargeSize	397
3.127.2.29	SmallSize	397
3.127.2.30	SmallEsize	397
3.127.2.31	TermsInSmall	398
3.127.2.32	Terms2InSmall	398
3.127.2.33	GenTerms	398
3.127.2.34	TermsLeft	398
3.127.2.35	GenSpace	398
3.127.2.36	SpaceLeft	398
3.127.2.37	putinsize	398
3.127.2.38	ninterms	398
3.127.2.39	MaxPatches	399
3.127.2.40	MaxFpatches	399
3.127.2.41	type	399
3.127.2.42	IPatch	399
3.127.2.43	fPatchN1	399
3.127.2.44	PolyWise	399
3.127.2.45	PolyFlag	399
3.127.2.46	cBufferSize	399
3.127.2.47	maxtermsize	400
3.127.2.48	newmaxtermsize	400
3.127.2.49	outputmode	400
3.127.2.50	stagelevel	400
3.127.2.51	fPatchN	400
3.127.2.52	inNum	400
3.127.2.53	stage4	400
3.128	SpecTatoR Struct Reference	401
3.128.1	Detailed Description	401
3.128.2	Field Documentation	401
3.128.2.1	position	401
3.128.2.2	readpos	402

3.128.2.3 fh	402
3.128.2.4 name	402
3.128.2.5 exprnumber	402
3.128.2.6 flags	402
3.129 SProcess Class Reference	402
3.129.1 Detailed Description	404
3.129.2 Constructor & Destructor Documentation	404
3.129.2.1 SProcess() [1/2]	404
3.129.2.2 SProcess() [2/2]	404
3.129.2.3 ~SProcess()	404
3.129.3 Member Function Documentation	404
3.129.3.1 prSProcess()	404
3.129.3.2 generate()	405
3.129.3.3 assign()	405
3.129.3.4 toMNodeClass()	405
3.129.3.5 match()	405
3.129.3.6 endMGraph()	405
3.129.3.7 endAGraph()	405
3.129.3.8 resultMGraph()	406
3.129.3.9 resultAGraph()	406
3.129.4 Field Documentation	406
3.129.4.1 model	406
3.129.4.2 proc	406
3.129.4.3 opt	406
3.129.4.4 pnclass	406
3.129.4.5 astack	407
3.129.4.6 mgraph	407
3.129.4.7 egraph	407
3.129.4.8 agraph	407
3.129.4.9 cl2nd	407
3.129.4.10 nd2cl	407
3.129.4.11 nclass	407
3.129.4.12 ninitl	407
3.129.4.13 nfinal	408
3.129.4.14 nvert	408
3.129.4.15 clist	408
3.129.4.16 id	408
3.129.4.17 loop	408
3.129.4.18 nNodes	408
3.129.4.19 nEdges	408
3.129.4.20 nExtern	408
3.129.4.21 ncouple	409

3.129.4.22 tCouple	409
3.129.4.23 DUMMYPadding	409
3.129.4.24 mgrcount	409
3.129.4.25 agrcount	409
3.129.4.26 extperm	409
3.129.4.27 nMGraphs	409
3.129.4.28 nMOPI	409
3.129.4.29 wMGraphs	410
3.129.4.30 wMOPI	410
3.129.4.31 nAGraphs	410
3.129.4.32 nAOPI	410
3.129.4.33 wAGraphs	410
3.129.4.34 wAOPI	410
3.130 StOrEcAcHe Struct Reference	411
3.130.1 Detailed Description	411
3.130.2 Field Documentation	411
3.130.2.1 position	411
3.130.2.2 toppos	412
3.130.2.3 next	412
3.130.2.4 buffer	412
3.131 STOREHEADER Struct Reference	412
3.131.1 Detailed Description	413
3.131.2 Field Documentation	413
3.131.2.1 headermark	413
3.131.2.2 lenWORD	413
3.131.2.3 lenLONG	413
3.131.2.4 lenPOS	413
3.131.2.5 lenPOINTER	414
3.131.2.6 endianness	414
3.131.2.7 sSym	414
3.131.2.8 sInd	414
3.131.2.9 sVec	414
3.131.2.10 sFun	415
3.131.2.11 maxpower	415
3.131.2.12 wildoffset	415
3.131.2.13 revision	415
3.131.2.14 reserved	415
3.132 Stream Struct Reference	416
3.132.1 Detailed Description	416
3.132.2 Field Documentation	416
3.132.2.1 fileposition	416
3.132.2.2 linenumber	417

3.132.2.3 prevline	417
3.132.2.4 buffer	417
3.132.2.5 pointer	417
3.132.2.6 top	417
3.132.2.7 FoldName	417
3.132.2.8 name	418
3.132.2.9 pname	418
3.132.2.10 buffersize	418
3.132.2.11 bufferposition	418
3.132.2.12 inbuffer	418
3.132.2.13 previous	418
3.132.2.14 handle	418
3.132.2.15 type	419
3.132.2.16 prevars	419
3.132.2.17 previousNoShowInput	419
3.132.2.18 eqnum	419
3.132.2.19 afterwards	419
3.132.2.20 olddelay	419
3.132.2.21 oldnoshowinput	419
3.132.2.22 isnextchar	419
3.132.2.23 nextchar	420
3.132.2.24 reserved	420
3.133 SuBbUf Struct Reference	420
3.133.1 Detailed Description	420
3.133.2 Field Documentation	420
3.133.2.1 subexpnum	420
3.133.2.2 buffernum	420
3.134 SWITCH Struct Reference	421
3.134.1 Detailed Description	421
3.134.2 Field Documentation	421
3.134.2.1 table	421
3.134.2.2 defaultcase	422
3.134.2.3 endswitch	422
3.134.2.4 typetable	422
3.134.2.5 maxcase	422
3.134.2.6 mincase	422
3.134.2.7 numcases	422
3.134.2.8 tablesize	422
3.134.2.9 caseoffset	422
3.134.2.10 iflevel	423
3.134.2.11 whilelevel	423
3.134.2.12 nestingsum	423

3.134.2.13 padding	423
3.135 SWITCHTABLE Struct Reference	423
3.135.1 Detailed Description	423
3.135.2 Field Documentation	423
3.135.2.1 ncase	423
3.135.2.2 value	424
3.135.2.3 compbuffer	424
3.136 SyMbOI Struct Reference	424
3.136.1 Detailed Description	424
3.136.2 Field Documentation	424
3.136.2.1 name	424
3.136.2.2 minpower	424
3.136.2.3 maxpower	425
3.136.2.4 complex	425
3.136.2.5 number	425
3.136.2.6 flags	425
3.136.2.7 node	425
3.136.2.8 namesize	425
3.136.2.9 dimension	425
3.137 T_const Struct Reference	426
3.137.1 Detailed Description	428
3.137.2 Field Documentation	428
3.137.2.1 S0	428
3.137.2.2 SS	428
3.137.2.3 Nest	428
3.137.2.4 NestStop	428
3.137.2.5 NestPoin	429
3.137.2.6 BrackBuf	429
3.137.2.7 StoreCache	429
3.137.2.8 StoreCacheAlloc	429
3.137.2.9 pWorkSpace	429
3.137.2.10 IWorkSpace	429
3.137.2.11 posWorkSpace	429
3.137.2.12 WorkSpace	429
3.137.2.13 WorkTop	430
3.137.2.14 WorkPointer	430
3.137.2.15 RepCount	430
3.137.2.16 RepTop	430
3.137.2.17 WildArgTaken	430
3.137.2.18 factorials	430
3.137.2.19 small_power_n	430
3.137.2.20 small_power	430

3.137.2.21 bernoullis	431
3.137.2.22 primelist	431
3.137.2.23 pfac	431
3.137.2.24 pBer	431
3.137.2.25 TMaddr	431
3.137.2.26 WildMask	431
3.137.2.27 previousEfactor	431
3.137.2.28 TermMemHeap	431
3.137.2.29 NumberMemHeap	432
3.137.2.30 CacheNumberMemHeap	432
3.137.2.31 bracketinfo	432
3.137.2.32 ListPoly	432
3.137.2.33 ListSymbols	432
3.137.2.34 NumMem	432
3.137.2.35 TopologiesTerm	432
3.137.2.36 TopologiesStart	432
3.137.2.37 partitions	433
3.137.2.38 sBer	433
3.137.2.39 pWorkPointer	433
3.137.2.40 IWorkPointer	433
3.137.2.41 posWorkPointer	433
3.137.2.42 InNumMem	433
3.137.2.43 sfact	433
3.137.2.44 mfac	433
3.137.2.45 ebufnum	434
3.137.2.46 fbufnum	434
3.137.2.47 allbufnum	434
3.137.2.48 aebufnum	434
3.137.2.49 idallflag	434
3.137.2.50 idallnum	434
3.137.2.51 idallmaxnum	434
3.137.2.52 WildcardBufferSize	434
3.137.2.53 TermMemMax	435
3.137.2.54 TermMemTop	435
3.137.2.55 NumberMemMax	435
3.137.2.56 NumberMemTop	435
3.137.2.57 CacheNumberMemMax	435
3.137.2.58 CacheNumberMemTop	435
3.137.2.59 bracketindexflag	435
3.137.2.60 optentimes	435
3.137.2.61 ListSymbolsSize	436
3.137.2.62 NumListSymbols	436

3.137.2.63 numpoly	436
3.137.2.64 LeaveNegative	436
3.137.2.65 TrimPower	436
3.137.2.66 small_power_maxx	436
3.137.2.67 small_power_maxn	436
3.137.2.68 dummysubexp	436
3.137.2.69 comsym	437
3.137.2.70 comnum	437
3.137.2.71 comfun	437
3.137.2.72 comind	437
3.137.2.73 MinVecArg	437
3.137.2.74 FunArg	437
3.137.2.75 locwildvalue	437
3.137.2.76 mulpat	437
3.137.2.77 proexp	438
3.137.2.78 TMout	438
3.137.2.79 TMbuff	438
3.137.2.80 TMdolfac	438
3.137.2.81 nfac	438
3.137.2.82 nBer	438
3.137.2.83 mBer	438
3.137.2.84 PolyAct	438
3.137.2.85 RecFlag	439
3.137.2.86 inprimelist	439
3.137.2.87 sizeprimelist	439
3.137.2.88 fromindex	439
3.137.2.89 setinterntopo	439
3.137.2.90 setexterntopo	439
3.137.2.91 TopologiesLevel	439
3.137.2.92 TopologiesOptions	440
3.138 T_EEdge Class Reference	440
3.138.1 Detailed Description	440
3.138.2 Field Documentation	440
3.138.2.1 nodes	440
3.138.2.2 ext	440
3.138.2.3 momn	440
3.138.2.4 momc	441
3.138.2.5 padding	441
3.139 T_EGraph Class Reference	441
3.139.1 Detailed Description	442
3.139.2 Constructor & Destructor Documentation	442
3.139.2.1 T_EGraph()	442

3.139.2.2 <code>~T_EGraph()</code>	442
3.139.3 Member Function Documentation	442
3.139.3.1 <code>print()</code>	442
3.139.3.2 <code>init()</code>	443
3.139.3.3 <code>setExtern()</code>	443
3.139.3.4 <code>endSetExtern()</code>	443
3.139.4 Field Documentation	443
3.139.4.1 <code>gld</code>	443
3.139.4.2 <code>pId</code>	443
3.139.4.3 <code>nNodes</code>	443
3.139.4.4 <code>nEdges</code>	444
3.139.4.5 <code>maxdeg</code>	444
3.139.4.6 <code>nExtern</code>	444
3.139.4.7 <code>opi</code>	444
3.139.4.8 <code>nsym</code>	444
3.139.4.9 <code>esym</code>	444
3.139.4.10 <code>nodes</code>	444
3.139.4.11 <code>edges</code>	445
3.140 <code>T_ENode</code> Class Reference	445
3.140.1 Detailed Description	445
3.140.2 Field Documentation	445
3.140.2.1 <code>deg</code>	445
3.140.2.2 <code>ext</code>	445
3.140.2.3 <code>edges</code>	445
3.141 <code>T_MGraph</code> Class Reference	446
3.141.1 Detailed Description	447
3.141.2 Constructor & Destructor Documentation	447
3.141.2.1 <code>T_MGraph()</code>	447
3.141.2.2 <code>~T_MGraph()</code>	447
3.141.3 Member Function Documentation	447
3.141.3.1 <code>generate()</code>	447
3.141.4 Field Documentation	447
3.141.4.1 <code>pId</code>	447
3.141.4.2 <code>nNodes</code>	447
3.141.4.3 <code>nEdges</code>	448
3.141.4.4 <code>nLoops</code>	448
3.141.4.5 <code>nodes</code>	448
3.141.4.6 <code>clist</code>	448
3.141.4.7 <code>nClasses</code>	448
3.141.4.8 <code>mindeg</code>	448
3.141.4.9 <code>maxdeg</code>	448
3.141.4.10 <code>selOPI</code>	448

3.141.4.11 ndiag	449
3.141.4.12 n1PI	449
3.141.4.13 nBridges	449
3.141.4.14 c1PI	449
3.141.4.15 adjMat	449
3.141.4.16 nsym	449
3.141.4.17 esym	449
3.141.4.18 wsum	449
3.141.4.19 wsopi	450
3.141.4.20 curcl	450
3.141.4.21 egraph	450
3.141.4.22 ngen	450
3.141.4.23 ngconn	450
3.141.4.24 nCallRefine	450
3.141.4.25 discardOrd	450
3.141.4.26 discardRefine	450
3.141.4.27 discardDisc	451
3.141.4.28 discardIso	451
3.142 T_MNode Class Reference	451
3.142.1 Detailed Description	451
3.142.2 Constructor & Destructor Documentation	451
3.142.2.1 T_MNode()	451
3.142.3 Field Documentation	452
3.142.3.1 id	452
3.142.3.2 deg	452
3.142.3.3 cls	452
3.142.3.4 ext	452
3.142.3.5 freelg	452
3.142.3.6 visited	452
3.143 T_MNodeClass Class Reference	453
3.143.1 Detailed Description	453
3.143.2 Constructor & Destructor Documentation	453
3.143.2.1 T_MNodeClass()	453
3.143.2.2 ~T_MNodeClass()	453
3.143.3 Member Function Documentation	454
3.143.3.1 init()	454
3.143.3.2 copy()	454
3.143.3.3 clCmp()	454
3.143.3.4 printMat()	454
3.143.3.5 mkFlist()	454
3.143.3.6 mkNdCI()	454
3.143.3.7 mkCIMat()	455

3.143.3.8 incMat()	455
3.143.3.9 chkOrd()	455
3.143.3.10 cmpArray()	455
3.143.4 Field Documentation	455
3.143.4.1 nNodes	455
3.143.4.2 nClasses	455
3.143.4.3 clist	456
3.143.4.4 ndcl	456
3.143.4.5 clmat	456
3.143.4.6 flist	456
3.143.4.7 maxdeg	456
3.143.4.8 forallignment	456
3.144 TaBIEbAsE Struct Reference	457
3.144.1 Detailed Description	457
3.144.2 Field Documentation	457
3.144.2.1 fillpoint	457
3.144.2.2 current	458
3.144.2.3 name	458
3.144.2.4 tablenumbers	458
3.144.2.5 subindex	458
3.144.2.6 numtables	458
3.145 TaBIEbAsEsUbInDeX Struct Reference	458
3.145.1 Detailed Description	459
3.145.2 Field Documentation	459
3.145.2.1 where	459
3.145.2.2 size	459
3.146 TaBIEs Struct Reference	459
3.146.1 Detailed Description	460
3.146.2 Field Documentation	460
3.146.2.1 tablepointers	460
3.146.2.2 prototype	461
3.146.2.3 pattern	461
3.146.2.4 mm	461
3.146.2.5 flags	461
3.146.2.6 boomlijst	461
3.146.2.7 argtail	462
3.146.2.8 spare	462
3.146.2.9 buffers	462
3.146.2.10 totind	462
3.146.2.11 reserved	462
3.146.2.12 defined	463
3.146.2.13 mdefined	463

3.146.2.14 prototypeSize	463
3.146.2.15 numind	463
3.146.2.16 bounds	463
3.146.2.17 strict	463
3.146.2.18 sparse	464
3.146.2.19 numtree	464
3.146.2.20 rootnum	464
3.146.2.21 MaxTreeSize	464
3.146.2.22 bufnum	464
3.146.2.23 buffersize	465
3.146.2.24 buffersfill	465
3.146.2.25 tablenum	465
3.146.2.26 mode	465
3.146.2.27 numdummies	465
3.147 TERMINFO Struct Reference	466
3.147.1 Detailed Description	466
3.147.2 Field Documentation	466
3.147.2.1 term	466
3.147.2.2 currentModel	466
3.147.2.3 currentMODEL	466
3.147.2.4 legcouple	466
3.147.2.5 numtopo	467
3.147.2.6 numdia	467
3.147.2.7 diaoffset	467
3.147.2.8 level	467
3.147.2.9 externalset	467
3.147.2.10 internalset	467
3.147.2.11 numextern	467
3.147.2.12 flags	468
3.148 ToPoTyPe Struct Reference	468
3.148.1 Detailed Description	468
3.148.2 Field Documentation	469
3.148.2.1 vert	469
3.148.2.2 vertmax	469
3.148.2.3 opt	469
3.148.2.4 cldeg	469
3.148.2.5 cnum	469
3.148.2.6 clex	469
3.148.2.7 cmin	469
3.148.2.8 cmaxd	470
3.148.2.9 ncl	470
3.148.2.10 nvert [1/2]	470

3.148.2.11 nloops	470
3.148.2.12 nlegs	470
3.148.2.13 npadding	470
3.148.2.14 nvert [2/2]	470
3.148.2.15 sopi	471
3.149 TrAcEn Struct Reference	471
3.149.1 Detailed Description	471
3.149.2 Field Documentation	471
3.149.2.1 accu	471
3.149.2.2 accup	471
3.149.2.3 temp	472
3.149.2.4 perm	472
3.149.2.5 inlist	472
3.149.2.6 sgn	472
3.149.2.7 num	472
3.149.2.8 level	472
3.149.2.9 factor	472
3.149.2.10 allsign	473
3.150 TrAcEs Struct Reference	473
3.150.1 Detailed Description	474
3.150.2 Field Documentation	474
3.150.2.1 accu	474
3.150.2.2 accup	474
3.150.2.3 temp	474
3.150.2.4 perm	474
3.150.2.5 inlist	474
3.150.2.6 nt3	474
3.150.2.7 nt4	475
3.150.2.8 j3	475
3.150.2.9 j4	475
3.150.2.10 e3	475
3.150.2.11 e4	475
3.150.2.12 eers	475
3.150.2.13 mepf	475
3.150.2.14 mdel	475
3.150.2.15 pepf	476
3.150.2.16 pdel	476
3.150.2.17 sgn	476
3.150.2.18 stap	476
3.150.2.19 step1	476
3.150.2.20 kstep	476
3.150.2.21 mdum	476

3.150.2.22 gamm	476
3.150.2.23 ad	477
3.150.2.24 a3	477
3.150.2.25 a4	477
3.150.2.26 lc3	477
3.150.2.27 lc4	477
3.150.2.28 sign1	477
3.150.2.29 sign2	477
3.150.2.30 gamma5	477
3.150.2.31 num	478
3.150.2.32 level	478
3.150.2.33 factor	478
3.150.2.34 allsign	478
3.150.2.35 finalstep	478
3.151 tree Struct Reference	478
3.151.1 Detailed Description	479
3.151.2 Field Documentation	479
3.151.2.1 parent	479
3.151.2.2 left	479
3.151.2.3 right	479
3.151.2.4 value	479
3.151.2.5 blnce	480
3.151.2.6 usage	480
3.152 tree_node Class Reference	480
3.152.1 Detailed Description	481
3.152.2 Constructor & Destructor Documentation	481
3.152.2.1 tree_node() [1/2]	481
3.152.2.2 tree_node() [2/2]	481
3.152.3 Member Function Documentation	481
3.152.3.1 operator=()	481
3.152.4 Field Documentation	481
3.152.4.1 childs	481
3.152.4.2 sum_results	482
3.152.4.3 num_visits	482
3.152.4.4 var	482
3.152.4.5 finished	482
3.153 VaRrEnUm Struct Reference	482
3.153.1 Detailed Description	482
3.153.2 Field Documentation	483
3.153.2.1 start	483
3.153.2.2 lo	483
3.153.2.3 hi	483

3.154 VeCtOr Struct Reference	483
3.154.1 Detailed Description	484
3.154.2 Field Documentation	484
3.154.2.1 name	484
3.154.2.2 complex	484
3.154.2.3 number	484
3.154.2.4 flags	484
3.154.2.5 node	484
3.154.2.6 namesize	484
3.154.2.7 dimension	485
3.155 VeRtEx Struct Reference	485
3.155.1 Detailed Description	485
3.155.2 Field Documentation	485
3.155.2.1 particles	485
3.155.2.2 couplings	486
3.155.2.3 nparticles	486
3.155.2.4 ncouplings	486
3.155.2.5 type	486
3.155.2.6 error	486
3.155.2.7 externonly	486
3.155.2.8 spare	486
3.156 X_const Struct Reference	487
3.156.1 Detailed Description	487
3.156.2 Field Documentation	487
3.156.2.1 currentPrompt	487
3.156.2.2 shellname	487
3.156.2.3 stderrname	487
3.156.2.4 timeout	487
3.156.2.5 killSignal	488
3.156.2.6 killWholeGroup	488
3.156.2.7 daemonize	488
3.156.2.8 currentExternalChannel	488
4 File Documentation	489
4.1 argument.c File Reference	489
4.1.1 Detailed Description	490
4.1.2 Macro Definition Documentation	490
4.1.2.1 NEWORDER	490
4.1.3 Function Documentation	490
4.1.3.1 execarg()	490
4.1.3.2 execterm()	490
4.1.3.3 ArgumentImplode()	490

4.1.3.4 ArgumentExplode()	491
4.1.3.5 ArgFactorize()	491
4.1.3.6 FindArg()	491
4.1.3.7 InsertArg()	491
4.1.3.8 CleanupArgCache()	492
4.1.3.9 ArgSymbolMerge()	492
4.1.3.10 ArgDotproductMerge()	492
4.1.3.11 TakeArgContent()	493
4.1.3.12 MakeInteger()	493
4.1.3.13 MakeMod()	494
4.1.3.14 SortWeights()	494
4.2 argument.c	495
4.3 bugtool.c File Reference	535
4.3.1 Detailed Description	535
4.3.2 Function Documentation	536
4.3.2.1 ExprStatus()	536
4.4 bugtool.c	536
4.5 checkpoint.c File Reference	537
4.5.1 Detailed Description	538
4.5.2 Macro Definition Documentation	538
4.5.2.1 CACHED_SNAPSHOT	538
4.5.2.2 CACHE_SIZE	538
4.5.2.3 R_FREE	539
4.5.2.4 R_FREE_NAMETREE	539
4.5.2.5 R_FREE_STREAM	539
4.5.2.6 R_SET	539
4.5.2.7 R_COPY_B	539
4.5.2.8 S_WRITE_B	540
4.5.2.9 S_FLUSH_B	540
4.5.2.10 R_COPY_S	540
4.5.2.11 S_WRITE_S	540
4.5.2.12 R_COPY_LIST	540
4.5.2.13 S_WRITE_LIST	541
4.5.2.14 R_COPY_NAMETREE	541
4.5.2.15 S_WRITE_NAMETREE	541
4.5.2.16 S_WRITE_DOLLAR	541
4.5.2.17 ANNOUNCE	542
4.5.3 Function Documentation	542
4.5.3.1 CheckRecoveryFile()	542
4.5.3.2 DeleteRecoveryFile()	542
4.5.3.3 RecoveryFilename()	542
4.5.3.4 InitRecovery()	543

4.5.3.5 fwrite_cached()	543
4.5.3.6 flush_cache()	543
4.5.3.7 DoRecovery()	543
4.5.3.8 DoCheckpoint()	544
4.5.4 Variable Documentation	545
4.5.4.1 cache_buffer	545
4.5.4.2 cache_fill	545
4.6 checkpoint.c	545
4.7 comexpr.c File Reference	581
4.7.1 Detailed Description	582
4.7.2 Function Documentation	582
4.7.2.1 CoLocal()	582
4.7.2.2 CoGlobal()	582
4.7.2.3 CoLocalFactorized()	582
4.7.2.4 CoGlobalFactorized()	582
4.7.2.5 DoExpr()	582
4.7.2.6 ColdOld()	583
4.7.2.7 Cold()	583
4.7.2.8 ColdNew()	583
4.7.2.9 CoDisorder()	583
4.7.2.10 CoMany()	583
4.7.2.11 CoMulti()	583
4.7.2.12 ColfMatch()	583
4.7.2.13 ColfNoMatch()	584
4.7.2.14 CoOnce()	584
4.7.2.15 CoOnly()	584
4.7.2.16 CoSelect()	584
4.7.2.17 ColdExpression()	584
4.7.2.18 CoMultiply()	584
4.7.2.19 CoFill()	584
4.7.2.20 CoFillExpression()	585
4.7.2.21 CoPrintTable()	585
4.7.2.22 CoAssign()	585
4.7.2.23 CoDeallocateTable()	585
4.8 comexpr.c	585
4.9 compcomm.c File Reference	609
4.9.1 Detailed Description	612
4.9.2 Function Documentation	612
4.9.2.1 CoFormat()	612
4.9.2.2 CoCollect()	612
4.9.2.3 setonoff()	612
4.9.2.4 CoCompress()	613

4.9.2.5 CoFlags()	613
4.9.2.6 CoOff()	613
4.9.2.7 CoOn()	613
4.9.2.8 CoInsideFirst()	613
4.9.2.9 CoProperCount()	613
4.9.2.10 CoDelete()	613
4.9.2.11 CoKeep()	614
4.9.2.12 CoFixIndex()	614
4.9.2.13 CoMetric()	614
4.9.2.14 DoPrint()	614
4.9.2.15 CoPrint()	614
4.9.2.16 CoPrintB()	614
4.9.2.17 CoNPrint()	614
4.9.2.18 CoPushHide()	615
4.9.2.19 CoPopHide()	615
4.9.2.20 SetExprCases()	615
4.9.2.21 SetExpr()	615
4.9.2.22 CoDrop()	615
4.9.2.23 CoNoDrop()	615
4.9.2.24 CoSkip()	616
4.9.2.25 CoNoSkip()	616
4.9.2.26 CoHide()	616
4.9.2.27 CoIntoHide()	616
4.9.2.28 CoNoIntoHide()	616
4.9.2.29 CoNoHide()	616
4.9.2.30 CoUnHide()	616
4.9.2.31 CoNoUnHide()	617
4.9.2.32 AddToCom()	617
4.9.2.33 AddComString()	617
4.9.2.34 Add2ComStrings()	617
4.9.2.35 CoDiscard()	617
4.9.2.36 CoContract()	617
4.9.2.37 CoGoTo()	618
4.9.2.38 CoLabel()	618
4.9.2.39 DoArgument()	618
4.9.2.40 CoArgument()	618
4.9.2.41 CoEndArgument()	618
4.9.2.42 CoInside()	618
4.9.2.43 CoEndInside()	618
4.9.2.44 CoNormalize()	619
4.9.2.45 CoMakeInteger()	619
4.9.2.46 CoSplitArg()	619

4.9.2.47 CoSplitFirstArg()	619
4.9.2.48 CoSplitLastArg()	619
4.9.2.49 CoFactArg()	619
4.9.2.50 DoSymmetrize()	619
4.9.2.51 CoSymmetrize()	620
4.9.2.52 CoAntiSymmetrize()	620
4.9.2.53 CoCycleSymmetrize()	620
4.9.2.54 CoRCycleSymmetrize()	620
4.9.2.55 CoWrite()	620
4.9.2.56 CoNWrite()	620
4.9.2.57 CoRatio()	620
4.9.2.58 CoRedefine()	621
4.9.2.59 CoRenumber()	621
4.9.2.60 CoSum()	621
4.9.2.61 CoToTensor()	621
4.9.2.62 CoToVector()	621
4.9.2.63 CoTrace4()	621
4.9.2.64 CoTraceN()	621
4.9.2.65 CoChisholm()	622
4.9.2.66 DoChain()	622
4.9.2.67 CoChainin()	622
4.9.2.68 CoChainout()	622
4.9.2.69 CoExit()	622
4.9.2.70 CoInParallel()	622
4.9.2.71 CoNotInParallel()	622
4.9.2.72 DoInParallel()	623
4.9.2.73 CoInExpression()	623
4.9.2.74 CoEndInExpression()	623
4.9.2.75 CoSetExitFlag()	623
4.9.2.76 CoTryReplace()	623
4.9.2.77 CoModulus()	623
4.9.2.78 CoRepeat()	623
4.9.2.79 CoEndRepeat()	624
4.9.2.80 DoBrackets()	624
4.9.2.81 CoBracket()	624
4.9.2.82 CoAntiBracket()	624
4.9.2.83 CoMultiBracket()	624
4.9.2.84 CountComp()	624
4.9.2.85 Colf()	624
4.9.2.86 CoElse()	625
4.9.2.87 CoElseif()	625
4.9.2.88 CoEndIf()	625

4.9.2.89 CoWhile()	625
4.9.2.90 CoEndWhile()	625
4.9.2.91 DoFindLoop()	625
4.9.2.92 CoFindLoop()	625
4.9.2.93 CoReplaceLoop()	626
4.9.2.94 CoFunPowers()	626
4.9.2.95 CoUnitTrace()	626
4.9.2.96 CoTerm()	626
4.9.2.97 CoEndTerm()	626
4.9.2.98 CoSort()	626
4.9.2.99 CoPolyFun()	626
4.9.2.100 CoPolyRatFun()	627
4.9.2.101 CoMerge()	627
4.9.2.102 CoStuffle()	627
4.9.2.103 CoProcessBucket()	627
4.9.2.104 CoThreadBucket()	627
4.9.2.105 DoArgPlode()	627
4.9.2.106 CoArgExplode()	627
4.9.2.107 CoArgImplode()	628
4.9.2.108 CoClearTable()	628
4.9.2.109 CoDenominators()	628
4.9.2.110 CoDropCoefficient()	628
4.9.2.111 CoDropSymbols()	628
4.9.2.112 CoToPolynomial()	628
4.9.2.113 CoFromPolynomial()	628
4.9.2.114 CoArgToExtraSymbol()	629
4.9.2.115 CoExtraSymbols()	629
4.9.2.116 GetIfDollarFactor()	629
4.9.2.117 GetDoParam()	629
4.9.2.118 CoDo()	629
4.9.2.119 CoEndDo()	629
4.9.2.120 CoFactDollar()	630
4.9.2.121 CoFactorize()	630
4.9.2.122 CoNFactorize()	630
4.9.2.123 CoUnFactorize()	630
4.9.2.124 CoNUnFactorize()	630
4.9.2.125 DoFactorize()	630
4.9.2.126 CoOptimizeOption()	630
4.9.2.127 CoPutInside()	631
4.9.2.128 CoAntiPutInside()	631
4.9.2.129 DoPutInside()	631
4.9.2.130 CoSwitch()	631

4.9.2.131 CoCase()	631
4.9.2.132 CoBreak()	631
4.9.2.133 CoDefault()	631
4.9.2.134 CoEndSwitch()	632
4.9.2.135 CoSetUserFlag()	632
4.9.2.136 CoClearUserFlag()	632
4.9.2.137 CoCreateAllLoops()	632
4.9.2.138 CoCreateAllPaths()	632
4.9.2.139 CoCreateAll()	632
4.10 compcomm.c	633
4.11 compiler.c File Reference	722
4.11.1 Detailed Description	723
4.11.2 Macro Definition Documentation	723
4.11.2.1 OPTION0	723
4.11.2.2 OPTION1	723
4.11.2.3 OPTION2	724
4.11.2.4 REDUCESUBEXPBUFFERS	724
4.11.3 Function Documentation	724
4.11.3.1 inictable()	724
4.11.3.2 findcommand()	724
4.11.3.3 ParenthesesTest()	724
4.11.3.4 SkipAName()	724
4.11.3.5 IsRHS()	725
4.11.3.6 IsIdStatement()	725
4.11.3.7 CompileAlgebra()	725
4.11.3.8 CompileStatement()	725
4.11.3.9 TestTables()	725
4.11.3.10 CompileSubExpressions()	725
4.11.3.11 CodeGenerator()	726
4.11.3.12 CompleteTerm()	726
4.11.3.13 CodeFactors()	726
4.11.3.14 GenerateFactors()	726
4.11.4 Variable Documentation	726
4.11.4.1 alfatable1	726
4.11.4.2 subexpbuffers	726
4.11.4.3 topsubexpbuffers	727
4.11.4.4 insubexpbuffers	727
4.12 compiler.c	727
4.13 compress.c File Reference	753
4.13.1 Detailed Description	753
4.14 compress.c	754
4.15 comtool.c File Reference	762

4.15.1 Detailed Description	763
4.15.2 Function Documentation	763
4.15.2.1 inicbufs()	763
4.15.2.2 finishcbuf()	763
4.15.2.3 clearcbuf()	764
4.15.2.4 DoubleCbuffer()	764
4.15.2.5 AddLHS()	764
4.15.2.6 AddRHS()	765
4.15.2.7 AddNtoL()	765
4.15.2.8 AddNtoC()	766
4.15.2.9 InsTree()	767
4.15.2.10 FindTree()	767
4.15.2.11 RedoTree()	767
4.15.2.12 ClearTree()	767
4.15.2.13 IniFbuffer()	768
4.15.2.14 numcommute()	768
4.16 comtool.c	768
4.17 comtool.h File Reference	775
4.17.1 Detailed Description	776
4.18 comtool.h	776
4.19 declare.h File Reference	777
4.19.1 Detailed Description	800
4.19.2 Macro Definition Documentation	800
4.19.2.1 MaX	800
4.19.2.2 MiN	800
4.19.2.3 ABS	800
4.19.2.4 SGN	800
4.19.2.5 REDLENG	801
4.19.2.6 INCLENG	801
4.19.2.7 GETCOEF	801
4.19.2.8 GETSTOP	801
4.19.2.9 StuffAdd	801
4.19.2.10 EXCHN	801
4.19.2.11 EXCH	802
4.19.2.12 TOKENTOLINE	802
4.19.2.13 UngetFromStream	802
4.19.2.14 AddLineFeed	802
4.19.2.15 TryRecover	802
4.19.2.16 UngetChar	802
4.19.2.17 ParseNumber	803
4.19.2.18 ParseSign	803
4.19.2.19 ParseSignedNumber	803

4.19.2.20 NCOPY	803
4.19.2.21 NCOPYI	803
4.19.2.22 NCOPYB	804
4.19.2.23 NCOPYI32	804
4.19.2.24 WCOPY	804
4.19.2.25 NeedNumber	804
4.19.2.26 SKIPBLANKS	804
4.19.2.27 FLUSHCONSOLE	805
4.19.2.28 SKIPBRA1	805
4.19.2.29 SKIPBRA2	805
4.19.2.30 SKIPBRA3	805
4.19.2.31 SKIPBRA4	805
4.19.2.32 SKIPBRA5	806
4.19.2.33 CYCLE1	806
4.19.2.34 AddToCB	806
4.19.2.35 EXCHINOUT	806
4.19.2.36 BACKINOUT	806
4.19.2.37 CopyArg	807
4.19.2.38 FILLARG	807
4.19.2.39 COPYARG	807
4.19.2.40 ZEROARG	807
4.19.2.41 FILLFUN	807
4.19.2.42 COPYFUN	807
4.19.2.43 COPYFUN3	808
4.19.2.44 FILLFUN3	808
4.19.2.45 FILLSUB	808
4.19.2.46 COPYSUB	808
4.19.2.47 FILLEXPR	808
4.19.2.48 NEXTARG	808
4.19.2.49 COPY1ARG	809
4.19.2.50 ZeroFillRange	809
4.19.2.51 TABLESIZE	809
4.19.2.52 WORDDIF	809
4.19.2.53 wsizeof	810
4.19.2.54 VARNAME	810
4.19.2.55 DOLLARNAME	810
4.19.2.56 EXPRNAME	810
4.19.2.57 PREV	810
4.19.2.58 Terminate	810
4.19.2.59 SETERROR	810
4.19.2.60 DUMMYUSE	811
4.19.2.61 ADDPOS	811

4.19.2.62 SETBASELENGTH	811
4.19.2.63 SETBASEPOSITION	811
4.19.2.64 ISEQUALPOSINC	811
4.19.2.65 ISGEPOSINC	811
4.19.2.66 DIVPOS	812
4.19.2.67 MULPOS	812
4.19.2.68 DIFPOS	812
4.19.2.69 DIFBASE	812
4.19.2.70 ADD2POS	812
4.19.2.71 PUTZERO	812
4.19.2.72 BASEPOSITION	813
4.19.2.73 SETSTARTPOS	813
4.19.2.74 NOTSTARTPOS	813
4.19.2.75 ISMINPOS	813
4.19.2.76 ISEQUALPOS	813
4.19.2.77 ISNOTEQUALPOS	813
4.19.2.78 ISLESSPOS	814
4.19.2.79 ISGEPOS	814
4.19.2.80 ISNOTZEROPOS	814
4.19.2.81 ISZEROPOS	814
4.19.2.82 ISPOSPOS	814
4.19.2.83 ISNEGPOS	814
4.19.2.84 TOLONG	814
4.19.2.85 Add2Com	815
4.19.2.86 Add3Com	815
4.19.2.87 Add4Com	815
4.19.2.88 Add5Com	815
4.19.2.89 WantAddPointers	815
4.19.2.90 WantAddLongs	816
4.19.2.91 WantAddPositions	816
4.19.2.92 FORM_INLINE	816
4.19.2.93 MEMORYMACROS	816
4.19.2.94 TermMalloc	816
4.19.2.95 NumberMalloc	816
4.19.2.96 CacheNumberMalloc	817
4.19.2.97 TermFree	817
4.19.2.98 NumberFree	817
4.19.2.99 CacheNumberFree	817
4.19.2.100 NestingChecksum	817
4.19.2.101 MesNesting	817
4.19.2.102 MarkPolyRatFunDirty	818
4.19.2.103 PUSHPREASSIGNLEVEL	818

4.19.2.104 POPPREASSIGNLEVEL	818
4.19.2.105 EXTERNLOCK	818
4.19.2.106 INILOCK	818
4.19.2.107 LOCK	819
4.19.2.108 UNLOCK	819
4.19.2.109 EXTERNRWLOCK	819
4.19.2.110 INIRWLOCK	819
4.19.2.111 RWLOCKR	819
4.19.2.112 RWLOCKW	819
4.19.2.113 UNRWLOCK	819
4.19.2.114 MLOCK	820
4.19.2.115 MUNLOCK	820
4.19.2.116 GETIDENTITY	820
4.19.2.117 GETBIDENTITY	820
4.19.2.118 M_alloc	820
4.19.2.119 CompareTerms	820
4.19.2.120 FiniShuffle	820
4.19.2.121 DoShuffle	821
4.19.3 Typedef Documentation	821
4.19.3.1 WRITEBUFTOEXTCHANNEL	821
4.19.3.2 GETCFROMEXTCHANNEL	821
4.19.3.3 SETTERMINATORFOREXTCHANNEL	821
4.19.3.4 SETKILLMODEFOREXTCHANNEL	821
4.19.3.5 WRITEFILE	821
4.19.3.6 GETTERM	821
4.19.4 Function Documentation	822
4.19.4.1 TELLFILE()	822
4.19.4.2 StartVariables()	822
4.19.4.3 CodeToLine()	822
4.19.4.4 AddArrayIndex()	823
4.19.4.5 FindInIndex()	823
4.19.4.6 NextFileIndex()	823
4.19.4.7 PasteTerm()	823
4.19.4.8 StrCopy()	824
4.19.4.9 WrtPower()	824
4.19.4.10 AccumGCD()	824
4.19.4.11 AddArgs()	824
4.19.4.12 AddCoef()	825
4.19.4.13 AddLong()	825
4.19.4.14 AddPLon()	825
4.19.4.15 AddPoly()	826
4.19.4.16 AddRat()	827

4.19.4.17 AddToLine()	827
4.19.4.18 AddWild()	827
4.19.4.19 BigLong()	827
4.19.4.20 BinomGen()	827
4.19.4.21 CheckWild()	828
4.19.4.22 Chisholm()	828
4.19.4.23 CleanExpr()	828
4.19.4.24 CleanUp()	828
4.19.4.25 ClearWild()	828
4.19.4.26 CompareFunctions()	828
4.19.4.27 Commute()	829
4.19.4.28 DetCommu()	829
4.19.4.29 DoesCommu()	829
4.19.4.30 CompArg()	829
4.19.4.31 CompCoef()	829
4.19.4.32 CompGroup()	830
4.19.4.33 Compare1()	830
4.19.4.34 CountDo()	831
4.19.4.35 CountFun()	831
4.19.4.36 DimensionSubterm()	831
4.19.4.37 DimensionTerm()	831
4.19.4.38 DimensionExpression()	831
4.19.4.39 Deferred()	831
4.19.4.40 DeleteStore()	832
4.19.4.41 DetCurDum()	832
4.19.4.42 DetVars()	833
4.19.4.43 Distribute()	833
4.19.4.44 DivLong()	833
4.19.4.45 DivRat()	833
4.19.4.46 Divvy()	833
4.19.4.47 DoDelta()	834
4.19.4.48 DoDelta3()	834
4.19.4.49 TestPartitions()	834
4.19.4.50 DoPartitions()	834
4.19.4.51 CoCanonicalize()	834
4.19.4.52 DoCanonicalize()	834
4.19.4.53 GenTopologies()	835
4.19.4.54 GenDiagrams()	835
4.19.4.55 DoTopologyCanonicalize()	835
4.19.4.56 DoShattering()	835
4.19.4.57 GenerateTopologies()	835
4.19.4.58 DoTableExpansion()	836

4.19.4.59 DoDistrib()	836
4.19.4.60 DoShuffle()	836
4.19.4.61 DoPermutations()	836
4.19.4.62 Shuffle()	836
4.19.4.63 FinishShuffle()	836
4.19.4.64 DoStuffle()	837
4.19.4.65 Stuffle()	837
4.19.4.66 FinishStuffle()	837
4.19.4.67 StuffRootAdd()	837
4.19.4.68 TestUse()	837
4.19.4.69 FindTB()	837
4.19.4.70 CheckTableDeclarations()	838
4.19.4.71 Apply()	838
4.19.4.72 ApplyExec()	838
4.19.4.73 ApplyReset()	838
4.19.4.74 TableReset()	838
4.19.4.75 ReWorkT()	838
4.19.4.76 GetIfDollarNum()	839
4.19.4.77 FindVar()	839
4.19.4.78 DolfStatement()	839
4.19.4.79 DoOnePow()	839
4.19.4.80 DoRevert()	840
4.19.4.81 DoSumF1()	840
4.19.4.82 DoSumF2()	841
4.19.4.83 DoTheta()	841
4.19.4.84 EndSort()	841
4.19.4.85 EntVar()	842
4.19.4.86 EpfCon()	842
4.19.4.87 EpfFind()	842
4.19.4.88 EpfGen()	843
4.19.4.89 EqualArg()	843
4.19.4.90 Factorial()	843
4.19.4.91 Bernoulli()	843
4.19.4.92 FactorIn()	843
4.19.4.93 FactorInExpr()	844
4.19.4.94 FindAll()	844
4.19.4.95 FindMulti()	844
4.19.4.96 FindOnce()	844
4.19.4.97 FindOnly()	844
4.19.4.98 FindRest()	844
4.19.4.99 FindrNumber()	845
4.19.4.100 FiniLine()	845

4.19.4.101 FiniTerm()	845
4.19.4.102 FlushOut()	845
4.19.4.103 FunLevel()	846
4.19.4.104 AdjustRenumScratch()	846
4.19.4.105 GarbHand()	846
4.19.4.106 GcdLong()	847
4.19.4.107 LcmLong()	847
4.19.4.108 Generator()	847
4.19.4.109 GetBinom()	848
4.19.4.110 GetFromStore()	848
4.19.4.111 GetLong()	849
4.19.4.112 GetMoreTerms()	849
4.19.4.113 GetMoreFromMem()	849
4.19.4.114 GetOneTerm()	849
4.19.4.115 GetTable()	849
4.19.4.116 GetTerm()	849
4.19.4.117 Glue()	850
4.19.4.118 InFunction()	850
4.19.4.119 IniLine()	851
4.19.4.120 IniVars()	851
4.19.4.121 InsertTerm()	851
4.19.4.122 LongToLine()	852
4.19.4.123 MakeDirty()	852
4.19.4.124 MarkDirty()	852
4.19.4.125 PolyFunDirty()	852
4.19.4.126 PolyFunClean()	853
4.19.4.127 MakeModTable()	853
4.19.4.128 MatchE()	853
4.19.4.129 MatchCy()	853
4.19.4.130 FunMatchCy()	853
4.19.4.131 FunMatchSy()	853
4.19.4.132 MatchArgument()	854
4.19.4.133 MatchFunction()	854
4.19.4.134 MergePatches()	854
4.19.4.135 MesCerr()	855
4.19.4.136 MesComp()	855
4.19.4.137 Modulus()	855
4.19.4.138 MoveDummies()	855
4.19.4.139 MulLong()	856
4.19.4.140 MulRat()	856
4.19.4.141 Mully()	856
4.19.4.142 MultDo()	856

4.19.4.143 NewSort()	856
4.19.4.144 ExtraSymbol()	857
4.19.4.145 Normalize()	857
4.19.4.146 BracketNormalize()	857
4.19.4.147 DropCoefficient()	857
4.19.4.148 DropSymbols()	857
4.19.4.149 PutInside()	857
4.19.4.150 OpenTemp()	858
4.19.4.151 Pack()	858
4.19.4.152 PasteFile()	858
4.19.4.153 Permute()	859
4.19.4.154 PermuteP()	859
4.19.4.155 PolyFunMul()	859
4.19.4.156 PopVariables()	860
4.19.4.157 PrepPoly()	860
4.19.4.158 Processor()	861
4.19.4.159 Product()	862
4.19.4.160 PrtLong()	862
4.19.4.161 PrtTerms()	863
4.19.4.162 PrintDeprecation()	863
4.19.4.163 PrintRunningTime()	863
4.19.4.164 GetRunningTime()	863
4.19.4.165 PutBracket()	863
4.19.4.166 PutIn()	863
4.19.4.167 PutInStore()	864
4.19.4.168 PutOut()	864
4.19.4.169 Quotient()	865
4.19.4.170 RaisPow()	865
4.19.4.171 RaisPowCached()	865
4.19.5 Description	865
4.19.6 Notes	866
4.19.6.1 RaisPowMod()	866
4.19.6.2 NormalModulus()	866
4.19.6.3 MakeInverses()	866
4.19.6.4 GetModInverses()	867
4.19.6.5 GetLongModInverses()	867
4.19.6.6 RatToLine()	867
4.19.6.7 RatioFind()	867
4.19.6.8 RatioGen()	868
4.19.6.9 ReNumber()	868
4.19.6.10 ReadSnum()	868
4.19.6.11 Remain10()	868

4.19.6.12 Remain4()	868
4.19.6.13 ResetScratch()	868
4.19.6.14 ResolveSet()	869
4.19.6.15 RevertScratch()	869
4.19.6.16 ScanFunctions()	869
4.19.6.17 SeekScratch()	869
4.19.6.18 SetEndScratch()	869
4.19.6.19 SetEndHScratch()	869
4.19.6.20 SetFileIndex()	870
4.19.6.21 Sflush()	870
4.19.6.22 Simplify()	870
4.19.6.23 SortWild()	871
4.19.6.24 LocateBase()	871
4.19.6.25 SplitMerge()	871
4.19.6.26 StoreTerm()	872
4.19.6.27 SubPLon()	873
4.19.6.28 Substitute()	873
4.19.6.29 SymFind()	873
4.19.6.30 SymGen()	874
4.19.6.31 Symmetrize()	874
4.19.6.32 FullSymmetrize()	874
4.19.6.33 TakeModulus()	874
4.19.6.34 TakeNormalModulus()	874
4.19.6.35 TalToLine()	875
4.19.6.36 TenVec()	875
4.19.6.37 TenVecFind()	875
4.19.6.38 TermRenumber()	875
4.19.6.39 TestDrop()	876
4.19.6.40 PutInVflags()	876
4.19.6.41 TestMatch()	876
4.19.6.42 TestSub()	877
4.19.6.43 TimeCPU()	877
4.19.6.44 TimeChildren()	878
4.19.6.45 TimeWallClock()	878
4.19.6.46 ToStorage()	878
4.19.6.47 TokenToLine()	878
4.19.6.48 Trace4()	878
4.19.6.49 Trace4Gen()	879
4.19.6.50 Trace4no()	879
4.19.6.51 TraceFind()	879
4.19.6.52 TraceN()	879
4.19.6.53 TraceNgen()	879

4.19.6.54 TraceNno()	879
4.19.6.55 Traces()	880
4.19.6.56 Trick()	880
4.19.6.57 TryDo()	880
4.19.6.58 UnPack()	880
4.19.6.59 VarStore()	880
4.19.6.60 WildFill()	881
4.19.6.61 WriteAll()	881
4.19.6.62 WriteOne()	881
4.19.6.63 WriteArgument()	881
4.19.6.64 WriteExpression()	881
4.19.6.65 WriteInnerTerm()	881
4.19.6.66 WriteLists()	882
4.19.6.67 WriteSetup()	882
4.19.6.68 WriteStats()	882
4.19.6.69 WriteSubTerm()	883
4.19.6.70 WriteTerm()	883
4.19.6.71 execarg()	883
4.19.6.72 execterm()	883
4.19.6.73 SpecialCleanup()	883
4.19.6.74 SetMods()	883
4.19.6.75 UnSetMods()	884
4.19.6.76 DoExecute()	884
4.19.6.77 SetScratch()	884
4.19.6.78 Warning()	884
4.19.6.79 HighWarning()	884
4.19.6.80 SpareTable()	884
4.19.6.81 strDup1()	885
4.19.6.82 Malloc()	885
4.19.6.83 Malloc1()	885
4.19.6.84 DoTail()	885
4.19.6.85 OpenInput()	885
4.19.6.86 PutPreVar()	885
4.19.6.87 Error0()	886
4.19.6.88 Error1()	886
4.19.6.89 Error2()	886
4.19.6.90 ReadFromStream()	886
4.19.6.91 GetFromStream()	887
4.19.6.92 LookInStream()	887
4.19.6.93 OpenStream()	887
4.19.6.94 LocateFile()	887
4.19.6.95 CloseStream()	887

4.19.6.96 PositionStream()	887
4.19.6.97 ReverseStatements()	888
4.19.6.98 ProcessOption()	888
4.19.6.99 DoSetups()	888
4.19.6.100 TerminateImpl()	888
4.19.6.101 GetNode()	888
4.19.6.102 AddName()	888
4.19.6.103 GetName()	889
4.19.6.104 GetFunction()	889
4.19.6.105 GetNumber()	889
4.19.6.106 GetLastExprName()	889
4.19.6.107 GetAutoName()	889
4.19.6.108 GetVar()	889
4.19.6.109 MakeDubious()	890
4.19.6.110 GetOName()	890
4.19.6.111 DumpTree()	890
4.19.6.112 DumpNode()	890
4.19.6.113 LinkTree()	890
4.19.6.114 CopyTree()	890
4.19.6.115 CompactifyTree()	891
4.19.6.116 MakeNameTree()	891
4.19.6.117 FreeNameTree()	891
4.19.6.118 AddExpression()	891
4.19.6.119 AddSymbol()	891
4.19.6.120 AddDollar()	891
4.19.6.121 ReplaceDollar()	892
4.19.6.122 DollarRaiseLow()	892
4.19.6.123 AddVector()	892
4.19.6.124 AddDubious()	892
4.19.6.125 AddIndex()	892
4.19.6.126 DoDimension()	892
4.19.6.127 AddFunction()	893
4.19.6.128 CoCommutInSet()	893
4.19.6.129 CoFunction()	893
4.19.6.130 TestName()	893
4.19.6.131 AddSet()	893
4.19.6.132 DoElements()	893
4.19.6.133 DoTempSet()	894
4.19.6.134 NameConflict()	894
4.19.6.135 OpenFile()	894
4.19.6.136 OpenAddFile()	894
4.19.6.137 ReOpenFile()	894

4.19.6.138 CreateFile()	894
4.19.6.139 CreateLogFile()	894
4.19.6.140 CloseFile()	895
4.19.6.141 CopyFile()	895
4.19.6.142 CreateHandle()	895
4.19.6.143 ReadFile()	895
4.19.6.144 ReadPosFile()	895
4.19.6.145 WriteFileToFile()	896
4.19.6.146 SeekFile()	896
4.19.6.147 TellFile()	896
4.19.6.148 FlushFile()	896
4.19.6.149 GetPosFile()	896
4.19.6.150 SetPosFile()	896
4.19.6.151 SynchFile()	897
4.19.6.152 TruncateFile()	897
4.19.6.153 GetChannel()	897
4.19.6.154 GetAppendChannel()	897
4.19.6.155 CloseChannel()	897
4.19.6.156 inictable()	897
4.19.6.157 findcommand()	897
4.19.6.158 inicbufs()	898
4.19.6.159 StartFiles()	898
4.19.6.160 MakeDate()	898
4.19.6.161 PreProcessor()	898
4.19.6.162 FromList()	898
4.19.6.163 FromOList()	899
4.19.6.164 FromVarList()	899
4.19.6.165 DoubleList()	899
4.19.6.166 DoubleLList()	899
4.19.6.167 DoubleBuffer()	899
4.19.6.168 ExpandBuffer()	899
4.19.6.169 iexp()	900
4.19.6.170 IsLikeVector()	900
4.19.6.171 AreArgsEqual()	900
4.19.6.172 CompareArgs()	900
4.19.6.173 SkipField()	900
4.19.6.174 StrCmp()	900
4.19.6.175 StrlCmp()	901
4.19.6.176 StrHlCmp()	901
4.19.6.177 StrlCont()	901
4.19.6.178 CmpArray()	901
4.19.6.179 ConWord()	901

4.19.6.180 StrLen()	901
4.19.6.181 GetPreVar()	902
4.19.6.182 ToGeneral()	902
4.19.6.183 ToPolyFunGeneral()	902
4.19.6.184 ToFast()	902
4.19.6.185 GetSetupPar()	902
4.19.6.186 RecalcSetups()	902
4.19.6.187 AllocSetups()	903
4.19.6.188 AllocSort()	903
4.19.6.189 AllocSortFileName()	903
4.19.6.190 LoadInputFile()	903
4.19.6.191 GetInput()	903
4.19.6.192 ClearPushback()	903
4.19.6.193 GetChar()	904
4.19.6.194 CharOut()	904
4.19.6.195 UnsetAllowDelay()	904
4.19.6.196 PopPreVars()	904
4.19.6.197 IniModule()	904
4.19.6.198 IniSpecialModule()	904
4.19.6.199 ModuleInstruction()	904
4.19.6.200 PreProInstruction()	905
4.19.6.201 LoadInstruction()	905
4.19.6.202 LoadStatement()	905
4.19.6.203 FindKeyWord()	905
4.19.6.204 FindInKeyWord()	905
4.19.6.205 DoDefine()	905
4.19.6.206 DoRedefine()	906
4.19.6.207 TheDefine()	906
4.19.6.208 TheUndefine()	906
4.19.6.209 ClearMacro()	907
4.19.6.210 DoUndefine()	907
4.19.6.211 DoInclude()	907
4.19.6.212 DoReverseInclude()	907
4.19.6.213 Include()	907
4.19.6.214 DoExternal()	907
4.19.6.215 DoToExternal()	907
4.19.6.216 DoFromExternal()	908
4.19.6.217 DoPrompt()	908
4.19.6.218 DoSetExternal()	908
4.19.6.219 DoSetExternalAttr()	908
4.19.6.220 DoRmExternal()	908
4.19.6.221 DoFactDollar()	908

4.19.6.222 GetDollarNumber()	908
4.19.6.223 DoSetRandom()	909
4.19.6.224 DoOptimize()	909
4.19.6.225 DoClearOptimize()	909
4.19.6.226 DoSkipExtraSymbols()	909
4.19.6.227 DoTimeOutAfter()	909
4.19.6.228 DoMessage()	909
4.19.6.229 DoPreOut()	909
4.19.6.230 DoPreAppend()	910
4.19.6.231 DoPreCreate()	910
4.19.6.232 DoPreAssign()	910
4.19.6.233 DoPreBreak()	910
4.19.6.234 DoPreDefault()	910
4.19.6.235 DoPreSwitch()	910
4.19.6.236 DoPreEndSwitch()	910
4.19.6.237 DoPreCase()	911
4.19.6.238 DoPreShow()	911
4.19.6.239 DoPreExchange()	911
4.19.6.240 DoSystem()	911
4.19.6.241 DoPipe()	911
4.19.6.242 StartPrepro()	911
4.19.6.243 Dolfdef()	911
4.19.6.244 Dolfydef()	912
4.19.6.245 Dolfndef()	912
4.19.6.246 DoElse()	912
4.19.6.247 DoElseif()	912
4.19.6.248 DoEndif()	912
4.19.6.249 DoTerminate()	912
4.19.6.250 Dolf()	912
4.19.6.251 DoCall()	913
4.19.6.252 DoDebug()	913
4.19.6.253 DoContinueDo()	913
4.19.6.254 DoDo()	913
4.19.6.255 DoBreakDo()	913
4.19.6.256 DoEnddo()	913
4.19.6.257 DoEndprocedure()	913
4.19.6.258 DoInside()	914
4.19.6.259 DoEndInside()	914
4.19.6.260 DoProcedure()	914
4.19.6.261 DoPrePrintTimes()	914
4.19.6.262 DoPreWrite()	914
4.19.6.263 DoPreClose()	914

4.19.6.264 DoPreRemove()	914
4.19.6.265 DoPreSortReallocate()	915
4.19.6.266 DoCommentChar()	915
4.19.6.267 DoPreExtension()	915
4.19.6.268 DoPreReset()	915
4.19.6.269 WriteString()	915
4.19.6.270 WriteUnfinString()	915
4.19.6.271 AddToString()	916
4.19.6.272 PreCalc()	916
4.19.6.273 PreEval()	916
4.19.6.274 NumToStr()	916
4.19.6.275 PreCmp()	916
4.19.6.276 PreEq()	917
4.19.6.277 pParseObject()	917
4.19.6.278 PriefEval()	917
4.19.6.279 EvalPrief()	917
4.19.6.280 PreLoad()	917
4.19.6.281 PreSkip()	918
4.19.6.282 EndOfToken()	918
4.19.6.283 SetSpecialMode()	918
4.19.6.284 MakeGlobal()	918
4.19.6.285 ExecModule()	918
4.19.6.286 ExecStore()	918
4.19.6.287 FullCleanup()	919
4.19.6.288 DoExecStatement()	919
4.19.6.289 DoPipeStatement()	919
4.19.6.290 DoPolyfun()	919
4.19.6.291 DoPolyratfun()	919
4.19.6.292 CompileStatement()	919
4.19.6.293 ToToken()	919
4.19.6.294 GetDollar()	920
4.19.6.295 MesWork()	920
4.19.6.296 MesCall()	920
4.19.6.297 NumCopy()	920
4.19.6.298 LongCopy()	920
4.19.6.299 LongLongCopy()	920
4.19.6.300 ReserveTempFiles()	921
4.19.6.301 PrintTerm()	921
4.19.6.302 PrintTermC()	921
4.19.6.303 PrintSubTerm()	921
4.19.6.304 PrintWords()	921
4.19.6.305 PrintSeq()	921

4.19.6.306 ExpandTripleDots()	922
4.19.6.307 ComPress()	922
4.19.6.308 StageSort()	922
4.19.6.309 M_free()	923
4.19.6.310 ClearWildcardNames()	923
4.19.6.311 AddWildcardName()	923
4.19.6.312 GetWildcardName()	923
4.19.6.313 Globalize()	923
4.19.6.314 ResetVariables()	923
4.19.6.315 AddToPreTypes()	923
4.19.6.316 MessPreNesting()	924
4.19.6.317 GetStreamPosition()	924
4.19.6.318 DoubleCbuffer()	924
4.19.6.319 AddLHS()	924
4.19.6.320 AddRHS()	925
4.19.6.321 AddNtoL()	925
4.19.6.322 AddNtoC()	926
4.19.6.323 DoubleIfBuffers()	927
4.19.6.324 CreateStream()	927
4.19.6.325 setonoff()	927
4.19.6.326 DoPrint()	927
4.19.6.327 SetExpr()	927
4.19.6.328 AddToCom()	927
4.19.6.329 Add2ComStrings()	928
4.19.6.330 DoSymmetrize()	928
4.19.6.331 DoArgument()	928
4.19.6.332 ArgFactorize()	928
4.19.6.333 TakeArgContent()	928
4.19.6.334 MakeInteger()	929
4.19.6.335 MakeMod()	929
4.19.6.336 FindArg()	930
4.19.6.337 InsertArg()	930
4.19.6.338 CleanupArgCache()	931
4.19.6.339 ArgSymbolMerge()	931
4.19.6.340 ArgDotproductMerge()	931
4.19.6.341 SortWeights()	932
4.19.6.342 DoBrackets()	932
4.19.6.343 DoPutInside()	932
4.19.6.344 CountComp()	932
4.19.6.345 CoAntiBracket()	933
4.19.6.346 CoAntiSymmetrize()	933
4.19.6.347 DoArgPlode()	933

4.19.6.348 CoArgExplode()	933
4.19.6.349 CoArgImplode()	933
4.19.6.350 CoArgument()	933
4.19.6.351 CoInside()	933
4.19.6.352 ExecInside()	934
4.19.6.353 CoInExpression()	934
4.19.6.354 CoInParallel()	934
4.19.6.355 CoNotInParallel()	934
4.19.6.356 DoInParallel()	934
4.19.6.357 CoEndInExpression()	934
4.19.6.358 CoBracket()	934
4.19.6.359 CoPutInside()	935
4.19.6.360 CoAntiPutInside()	935
4.19.6.361 CoMultiBracket()	935
4.19.6.362 CoCFunction()	935
4.19.6.363 CoCTensor()	935
4.19.6.364 CoCollect()	935
4.19.6.365 CoCompress()	935
4.19.6.366 CoContract()	936
4.19.6.367 CoCycleSymmetrize()	936
4.19.6.368 CoDelete()	936
4.19.6.369 CoTableBase()	936
4.19.6.370 CoApply()	936
4.19.6.371 CoDenominators()	936
4.19.6.372 CoDimension()	936
4.19.6.373 CoDiscard()	937
4.19.6.374 CoDisorder()	937
4.19.6.375 CoDrop()	937
4.19.6.376 CoDropCoefficient()	937
4.19.6.377 CoDropSymbols()	937
4.19.6.378 CoElse()	937
4.19.6.379 CoElseif()	937
4.19.6.380 CoEndArgument()	938
4.19.6.381 CoEndInside()	938
4.19.6.382 CoEndIf()	938
4.19.6.383 CoEndRepeat()	938
4.19.6.384 CoEndTerm()	938
4.19.6.385 CoEndWhile()	938
4.19.6.386 CoExit()	938
4.19.6.387 CoFactArg()	939
4.19.6.388 CoFactDollar()	939
4.19.6.389 CoFactorize()	939

4.19.6.390 CoNFactorize()	939
4.19.6.391 CoUnFactorize()	939
4.19.6.392 CoNUnFactorize()	939
4.19.6.393 DoFactorize()	939
4.19.6.394 CoFill()	940
4.19.6.395 CoFillExpression()	940
4.19.6.396 CoFixIndex()	940
4.19.6.397 CoFormat()	940
4.19.6.398 CoGlobal()	940
4.19.6.399 CoGlobalFactorized()	940
4.19.6.400 CoGoTo()	940
4.19.6.401 Cold()	941
4.19.6.402 ColdNew()	941
4.19.6.403 ColdOld()	941
4.19.6.404 Colf()	941
4.19.6.405 ColfMatch()	941
4.19.6.406 ColfNoMatch()	941
4.19.6.407 ColIndex()	941
4.19.6.408 ColInsideFirst()	942
4.19.6.409 CoKeep()	942
4.19.6.410 CoLabel()	942
4.19.6.411 CoLoad()	942
4.19.6.412 CoLocal()	942
4.19.6.413 CoLocalFactorized()	942
4.19.6.414 CoMany()	942
4.19.6.415 CoMerge()	943
4.19.6.416 CoStuffle()	943
4.19.6.417 CoMetric()	943
4.19.6.418 CoModOption()	943
4.19.6.419 CoModuleOption()	943
4.19.6.420 CoModulus()	943
4.19.6.421 CoMulti()	943
4.19.6.422 CoMultiply()	944
4.19.6.423 CoNFunction()	944
4.19.6.424 CoNPrint()	944
4.19.6.425 CoNTensor()	944
4.19.6.426 CoNWrite()	944
4.19.6.427 CoNoDrop()	944
4.19.6.428 CoNoSkip()	944
4.19.6.429 CoNormalize()	945
4.19.6.430 CoMakeInteger()	945
4.19.6.431 CoFlags()	945

4.19.6.432 CoOff()	945
4.19.6.433 CoOn()	945
4.19.6.434 CoOnce()	945
4.19.6.435 CoOnly()	945
4.19.6.436 CoOptimizeOption()	946
4.19.6.437 CoPolyFun()	946
4.19.6.438 CoPolyRatFun()	946
4.19.6.439 CoPrint()	946
4.19.6.440 CoPrintB()	946
4.19.6.441 CoProperCount()	946
4.19.6.442 CoUnitTrace()	946
4.19.6.443 CoRCycleSymmetrize()	947
4.19.6.444 CoRatio()	947
4.19.6.445 CoRedefine()	947
4.19.6.446 CoRenumber()	947
4.19.6.447 CoRepeat()	947
4.19.6.448 CoSave()	947
4.19.6.449 CoSelect()	947
4.19.6.450 CoSet()	948
4.19.6.451 CoSetExitFlag()	948
4.19.6.452 CoSkip()	948
4.19.6.453 CoProcessBucket()	948
4.19.6.454 CoPushHide()	948
4.19.6.455 CoPopHide()	948
4.19.6.456 CoHide()	948
4.19.6.457 CoIntoHide()	949
4.19.6.458 CoNoIntoHide()	949
4.19.6.459 CoNoHide()	949
4.19.6.460 CoUnHide()	949
4.19.6.461 CoNoUnHide()	949
4.19.6.462 CoSort()	949
4.19.6.463 CoSplitArg()	949
4.19.6.464 CoSplitFirstArg()	950
4.19.6.465 CoSplitLastArg()	950
4.19.6.466 CoSum()	950
4.19.6.467 CoSymbol()	950
4.19.6.468 CoSymmetrize()	950
4.19.6.469 DoTable()	950
4.19.6.470 CoTable()	950
4.19.6.471 CoTerm()	951
4.19.6.472 CoNTable()	951
4.19.6.473 CoCTable()	951

4.19.6.474 EmptyTable()	951
4.19.6.475 CoToTensor()	951
4.19.6.476 CoToVector()	951
4.19.6.477 CoTrace4()	951
4.19.6.478 CoTraceN()	952
4.19.6.479 CoChisholm()	952
4.19.6.480 CoTransform()	952
4.19.6.481 CoClearTable()	952
4.19.6.482 DoChain()	952
4.19.6.483 CoChainin()	952
4.19.6.484 CoChainout()	952
4.19.6.485 CoTryReplace()	953
4.19.6.486 CoVector()	953
4.19.6.487 CoWhile()	953
4.19.6.488 CoWrite()	953
4.19.6.489 CoAuto()	953
4.19.6.490 CoSwitch()	953
4.19.6.491 CoCase()	953
4.19.6.492 CoBreak()	954
4.19.6.493 CoDefault()	954
4.19.6.494 CoEndSwitch()	954
4.19.6.495 CoTBaddto()	954
4.19.6.496 CoTBaudit()	954
4.19.6.497 CoTBCleanup()	954
4.19.6.498 CoTBcreate()	954
4.19.6.499 CoTBenter()	955
4.19.6.500 CoTBhelp()	955
4.19.6.501 CoTBload()	955
4.19.6.502 CoTBoff()	955
4.19.6.503 CoTBon()	955
4.19.6.504 CoTBopen()	955
4.19.6.505 CoTBreplace()	955
4.19.6.506 CoTBuse()	956
4.19.6.507 CoTestUse()	956
4.19.6.508 CoThreadBucket()	956
4.19.6.509 AddComString()	956
4.19.6.510 CompileAlgebra()	956
4.19.6.511 IsIdStatement()	956
4.19.6.512 IsRHS()	957
4.19.6.513 ParenthesesTest()	957
4.19.6.514 tokenize()	957
4.19.6.515 WriteTokens()	957

4.19.6.516 simp1token()	957
4.19.6.517 simpwtoken()	957
4.19.6.518 simp2token()	957
4.19.6.519 simp3atoken()	958
4.19.6.520 simp3btoken()	958
4.19.6.521 simp4token()	958
4.19.6.522 simp5token()	958
4.19.6.523 simp6token()	958
4.19.6.524 SkipAName()	958
4.19.6.525 TestTables()	959
4.19.6.526 GetLabel()	959
4.19.6.527 ColdExpression()	959
4.19.6.528 CoAssign()	959
4.19.6.529 DoExpr()	959
4.19.6.530 CompileSubExpressions()	959
4.19.6.531 CodeGenerator()	960
4.19.6.532 CompleteTerm()	960
4.19.6.533 CodeFactors()	960
4.19.6.534 GenerateFactors()	960
4.19.6.535 InsTree()	960
4.19.6.536 FindTree()	960
4.19.6.537 RedoTree()	961
4.19.6.538 ClearTree()	961
4.19.6.539 CatchDollar()	961
4.19.6.540 AssignDollar()	961
4.19.6.541 WriteDollarToBuffer()	961
4.19.6.542 WriteDollarFactorToBuffer()	961
4.19.6.543 AddToDollarBuffer()	962
4.19.6.544 PutTermInDollar()	962
4.19.6.545 TermAssign()	962
4.19.6.546 WildDollars()	962
4.19.6.547 numcommute()	962
4.19.6.548 FullRenummer()	962
4.19.6.549 Lus()	963
4.19.6.550 FindLus()	963
4.19.6.551 CoReplaceLoop()	963
4.19.6.552 CoFindLoop()	963
4.19.6.553 DoFindLoop()	963
4.19.6.554 CoFunPowers()	963
4.19.6.555 SortTheList()	964
4.19.6.556 MatchIsPossible()	964
4.19.6.557 StudyPattern()	964

4.19.6.558 DolToTensor()	964
4.19.6.559 DolToFunction()	964
4.19.6.560 DolToVector()	964
4.19.6.561 DolToNumber()	964
4.19.6.562 DolToSymbol()	965
4.19.6.563 DolToIndex()	965
4.19.6.564 DolToLong()	965
4.19.6.565 DollarFactorize()	965
4.19.6.566 CoPrintTable()	965
4.19.6.567 CoDeallocateTable()	965
4.19.6.568 CleanDollarFactors()	965
4.19.6.569 TakeDollarContent()	966
4.19.6.570 MakeDollarInteger()	966
4.19.6.571 MakeDollarMod()	967
4.19.6.572 GetDolNum()	967
4.19.6.573 AddPotModdollar()	967
4.19.6.574 Optimize()	968
4.19.7 Description	968
4.19.7.1 ClearOptimize()	969
4.19.8 Description	969
4.19.8.1 TakeLongRoot()	970
4.19.8.2 TakeRatRoot()	970
4.19.8.3 MakeRational()	970
4.19.8.4 MakeLongRational()	970
4.19.8.5 ClearTableTree()	971
4.19.8.6 InsTableTree()	971
4.19.8.7 RedoTableTree()	971
4.19.8.8 FindTableTree()	971
4.19.8.9 finishcbuf()	971
4.19.8.10 clearcbuf()	972
4.19.8.11 CleanUpSort()	972
4.19.8.12 AllocFileHandle()	972
4.19.8.13 DeAllocFileHandle()	972
4.19.8.14 LowerSortLevel()	972
4.19.8.15 PolyRatFunSpecial()	973
4.19.8.16 SimpleSplitMergeRec()	973
4.19.8.17 SimpleSplitMerge()	973
4.19.8.18 BinarySearch()	973
4.19.8.19 InsideDollar()	973
4.19.8.20 DolToTerms()	973
4.19.8.21 EvalDoLoopArg()	974
4.19.8.22 SetExprCases()	974

4.19.8.23 TestSelect()	974
4.19.8.24 SubsInAll()	975
4.19.8.25 TransferBuffer()	975
4.19.8.26 TakeIDfunction()	975
4.19.8.27 MakeSetupAllocs()	975
4.19.8.28 TryFileSetups()	975
4.19.8.29 ExchangeExpressions()	975
4.19.8.30 ExchangeDollars()	976
4.19.8.31 GetFirstBracket()	976
4.19.8.32 GetFirstTerm()	976
4.19.8.33 GetContent()	976
4.19.8.34 CleanupTerm()	977
4.19.8.35 ContentMerge()	977
4.19.8.36 PriefDollarEval()	977
4.19.8.37 TermsInDollar()	977
4.19.8.38 SizeOfDollar()	977
4.19.8.39 TermsInExpression()	977
4.19.8.40 SizeOfExpression()	977
4.19.8.41 TranslateExpression()	978
4.19.8.42 IsSetMember()	978
4.19.8.43 IsMultipleOf()	978
4.19.8.44 TwoExprCompare()	978
4.19.8.45 UpdatePositions()	978
4.19.8.46 M_print()	978
4.19.8.47 M_check1()	979
4.19.8.48 PrintTime()	979
4.19.8.49 FindBracket()	979
4.19.8.50 PutBracketInIndex()	979
4.19.8.51 ClearBracketIndex()	979
4.19.8.52 OpenBracketIndex()	979
4.19.8.53 DoNoParallel()	979
4.19.8.54 DoParallel()	980
4.19.8.55 DoModSum()	980
4.19.8.56 DoModMax()	980
4.19.8.57 DoModMin()	980
4.19.8.58 DoModLocal()	980
4.19.8.59 DoModDollar()	980
4.19.8.60 DoProcessBucket()	980
4.19.8.61 DoinParallel()	981
4.19.8.62 DonotinParallel()	981
4.19.8.63 FlipTable()	981
4.19.8.64 ChainIn()	981

4.19.8.65 ChainOut()	981
4.19.8.66 ArgumentImplode()	981
4.19.8.67 ArgumentExplode()	982
4.19.8.68 DenToFunction()	982
4.19.8.69 HowMany()	982
4.19.8.70 RemoveDollars()	982
4.19.8.71 CountTerms1()	982
4.19.8.72 TermsInBracket()	982
4.19.8.73 Crash()	983
4.19.8.74 str_dup()	983
4.19.8.75 convertblock()	983
4.19.8.76 convertnamesblock()	983
4.19.8.77 convertiniinfo()	983
4.19.8.78 ReadIndex()	983
4.19.8.79 WriteIndexBlock()	984
4.19.8.80 WriteNamesBlock()	984
4.19.8.81 WriteIndex()	984
4.19.8.82 WriteIniInfo()	984
4.19.8.83 ReadIniInfo()	984
4.19.8.84 AddToIndex()	984
4.19.8.85 GetDbase()	985
4.19.8.86 OpenDbase()	985
4.19.8.87 ReadObject()	985
4.19.8.88 ReadObjObject()	985
4.19.8.89 ExistsObject()	985
4.19.8.90 DeleteObject()	985
4.19.8.91 WriteObject()	986
4.19.8.92 AddObject()	986
4.19.8.93 NewDbase()	986
4.19.8.94 FreeTableBase()	986
4.19.8.95 ComposeTableNames()	986
4.19.8.96 PutTableNames()	986
4.19.8.97 AddTableName()	987
4.19.8.98 GetTableName()	987
4.19.8.99 FindTableNumber()	987
4.19.8.100 TryEnvironment()	987
4.19.8.101 CopyExpression()	987
4.19.8.102 set_in()	987
4.19.8.103 set_set()	988
4.19.8.104 set_del()	988
4.19.8.105 set_sub()	988
4.19.8.106 DoPreAddSeparator()	988

4.19.8.107 DoPreRmSeparator()	988
4.19.8.108 openExternalChannel()	988
4.19.8.109 initPresetExternalChannels()	989
4.19.8.110 closeExternalChannel()	989
4.19.8.111 selectExternalChannel()	989
4.19.8.112 getCurrentExternalChannel()	989
4.19.8.113 closeAllExternalChannels()	989
4.19.8.114 defineChannel()	989
4.19.8.115 writeToChannel()	990
4.19.8.116 ReleaseTB()	990
4.19.8.117 SymbolNormalize()	990
4.19.8.118 TestFunFlag()	990
4.19.8.119 CompareSymbols()	990
4.19.8.120 CompareHSymbols()	991
4.19.8.121 NextPrime()	991
4.19.8.122 Moebius()	991
4.19.8.123 wranf()	991
4.19.8.124 iranf()	991
4.19.8.125 iniwranf()	992
4.19.8.126 PreRandom()	992
4.19.8.127 TreatPolyRatFun()	992
4.19.8.128 ReadSaveHeader()	992
4.19.8.129 ReadSaveIndex()	992
4.19.8.130 ReadSaveExpression()	993
4.19.8.131 ReadSaveTerm32()	994
4.19.8.132 ReadSaveVariables()	995
4.19.8.133 WriteStoreHeader()	996
4.19.8.134 InitRecovery()	996
4.19.8.135 CheckRecoveryFile()	996
4.19.8.136 DeleteRecoveryFile()	997
4.19.8.137 RecoveryFilename()	997
4.19.8.138 DoRecovery()	997
4.19.8.139 DoCheckpoint()	998
4.19.8.140 NumberMallocAddMemory()	999
4.19.8.141 CacheNumberMallocAddMemory()	999
4.19.8.142 TermMallocAddMemory()	999
4.19.8.143 ExprStatus()	999
4.19.8.144 iniTools()	999
4.19.8.145 TestTerm()	999
4.19.8.146 RunTransform()	1000
4.19.8.147 RunEncode()	1000
4.19.8.148 RunDecode()	1000

4.19.8.149 RunReplace()	1000
4.19.8.150 RunImplode()	1001
4.19.8.151 RunExplode()	1001
4.19.8.152 TestArgNum()	1001
4.19.8.153 PutArgInScratch()	1001
4.19.8.154 ReadRange()	1001
4.19.8.155 FindRange()	1001
4.19.8.156 RunPermute()	1002
4.19.8.157 RunReverse()	1002
4.19.8.158 RunCycle()	1002
4.19.8.159 RunAddArg()	1002
4.19.8.160 RunMulArg()	1002
4.19.8.161 RunIsLyndon()	1002
4.19.8.162 RunToLyndon()	1003
4.19.8.163 RunDropArg()	1003
4.19.8.164 RunSelectArg()	1003
4.19.8.165 RunDedup()	1003
4.19.8.166 RunZtoHArg()	1003
4.19.8.167 RunHtoZArg()	1003
4.19.8.168 NormPolyTerm()	1004
4.19.8.169 ConvertToPoly()	1004
4.19.8.170 LocalConvertToPoly()	1004
4.19.8.171 ConvertFromPoly()	1004
4.19.8.172 FindSubterm()	1004
4.19.8.173 FindLocalSubterm()	1005
4.19.8.174 PrintSubtermList()	1005
4.19.8.175 PrintExtraSymbol()	1005
4.19.8.176 FindSubexpression()	1005
4.19.8.177 UpdateMaxSize()	1005
4.19.8.178 CoToPolynomial()	1005
4.19.8.179 CoFromPolynomial()	1006
4.19.8.180 CoArgToExtraSymbol()	1006
4.19.8.181 CoExtraSymbols()	1006
4.19.8.182 GetDoParam()	1006
4.19.8.183 GetIfDollarFactor()	1006
4.19.8.184 CoDo()	1006
4.19.8.185 CoEndDo()	1007
4.19.8.186 ExtraSymFun()	1007
4.19.8.187 PruneExtraSymbols()	1007
4.19.8.188 IniFbuffer()	1007
4.19.8.189 GCDfunction()	1007
4.19.8.190 GCDfunction3()	1007

4.19.8.191 ReadPolyRatFun()	1008
4.19.8.192 FromPolyRatFun()	1008
4.19.8.193 PolyDiv()	1008
4.19.8.194 GCDclean()	1008
4.19.8.195 TakeSymbolContent()	1008
4.19.8.196 GCDterms()	1009
4.19.8.197 PutExtraSymbols()	1009
4.19.8.198 TakeExtraSymbols()	1009
4.19.8.199 MultiplyWithTerm()	1009
4.19.8.200 TakeContent()	1009
4.19.8.201 MergeSymbolLists()	1010
4.19.8.202 MergeDotproductLists()	1010
4.19.8.203 CreateExpression()	1010
4.19.8.204 DIVfunction()	1010
4.19.8.205 MULfunc()	1011
4.19.8.206 ConvertArgument()	1011
4.19.8.207 ExpandRat()	1011
4.19.8.208 InvPoly()	1011
4.19.8.209 TestDoLoop()	1011
4.19.8.210 TestEndDoLoop()	1011
4.19.8.211 poly_gcd()	1012
4.19.9 Description	1012
4.19.10 Notes	1012
4.19.10.1 poly_div()	1012
4.19.10.2 poly_rem()	1013
4.19.10.3 poly_inverse()	1013
4.19.10.4 poly_mul()	1013
4.19.10.5 poly_ratfun_add()	1013
4.19.11 Description	1013
4.19.12 Notes	1014
4.19.12.1 poly_ratfun_normalize()	1014
4.19.13 Description	1014
4.19.14 Notes	1015
4.19.14.1 poly_factorize_argument()	1015
4.19.15 Description	1015
4.19.16 Notes	1016
4.19.16.1 poly_factorize_dollar()	1016
4.19.17 Description	1016
4.19.18 Notes	1016
4.19.18.1 poly_factorize_expression()	1017
4.19.19 Description	1017
4.19.20 Notes	1017

4.19.20.1 poly_unfactorize_expression()	1018
4.19.21 Description	1018
4.19.22 Notes	1018
4.19.22.1 poly_free_poly_vars()	1019
4.19.22.2 optimize_print_code()	1019
4.19.23 Description	1019
4.19.23.1 DoPreAdd()	1019
4.19.23.2 DoPreUseDictionary()	1019
4.19.23.3 DoPreCloseDictionary()	1019
4.19.23.4 DoPreOpenDictionary()	1020
4.19.23.5 RemoveDictionary()	1020
4.19.23.6 UnSetDictionary()	1020
4.19.23.7 SetDictionaryOptions()	1020
4.19.23.8 AddToDictionary()	1020
4.19.23.9 AddDictionary()	1020
4.19.23.10 FindDictionary()	1020
4.19.23.11 IsExponentSign()	1021
4.19.23.12 IsMultiplySign()	1021
4.19.23.13 TransformRational()	1021
4.19.23.14 WriteDictionary()	1021
4.19.23.15 ShrinkDictionary()	1021
4.19.23.16 MultiplyToLine()	1021
4.19.23.17 FindSymbol()	1021
4.19.23.18 FindVector()	1022
4.19.23.19 FindIndex()	1022
4.19.23.20 FindFunction()	1022
4.19.23.21 FindFunWithArgs()	1022
4.19.23.22 FindExtraSymbol()	1022
4.19.23.23 DictToBytes()	1022
4.19.23.24 DictFromBytes()	1022
4.19.23.25 CoCreateSpectator()	1023
4.19.23.26 CoToSpectator()	1023
4.19.23.27 CoRemoveSpectator()	1023
4.19.23.28 CoEmptySpectator()	1023
4.19.23.29 CoCopySpectator()	1023
4.19.23.30 PutInSpectator()	1023
4.19.23.31 ClearSpectators()	1023
4.19.23.32 GetFromSpectator()	1024
4.19.23.33 FlushSpectators()	1024
4.19.23.34 ReadFromScratch()	1024
4.19.23.35 AddToScratch()	1024
4.19.23.36 DoPreAppendPath()	1024

4.19.23.37 DoPrePrependPath()	1025
4.19.23.38 DoSwitch()	1025
4.19.23.39 DoEndSwitch()	1025
4.19.23.40 FindCase()	1025
4.19.23.41 SwitchSplitMergeRec()	1025
4.19.23.42 SwitchSplitMerge()	1025
4.19.23.43 DoubleSwitchBuffers()	1026
4.19.23.44 DistrN()	1026
4.19.23.45 DoNamespace()	1026
4.19.23.46 DoEndNamespace()	1026
4.19.23.47 DoUse()	1026
4.19.23.48 SkipName()	1026
4.19.23.49 ConstructName()	1027
4.19.23.50 DoSetUserFlag()	1027
4.19.23.51 DoClearUserFlag()	1027
4.19.23.52 CreateVertex()	1027
4.19.23.53 ReadParticle()	1027
4.19.23.54 CoModel()	1027
4.19.23.55 CoParticle()	1028
4.19.23.56 CoVertex()	1028
4.19.23.57 CoEndModel()	1028
4.19.23.58 LoadModel()	1028
4.19.23.59 CoSetUserFlag()	1028
4.19.23.60 CoClearUserFlag()	1028
4.19.23.61 CoCreateAllLoops()	1028
4.19.23.62 CoCreateAllPaths()	1029
4.19.23.63 CoCreateAll()	1029
4.19.23.64 AllLoops()	1029
4.19.23.65 StartLoops()	1029
4.19.23.66 GenLoops()	1029
4.19.23.67 LoopOutput()	1030
4.19.23.68 AllPaths()	1030
4.19.23.69 GenPaths()	1030
4.19.23.70 PathOutput()	1030
4.20 declare.h	1031
4.21 diagrams.c File Reference	1051
4.21.1 Detailed Description	1051
4.21.2 Function Documentation	1052
4.21.2.1 CoCanonicalize()	1052
4.21.2.2 DoCanonicalize()	1052
4.21.2.3 DoTopologyCanonicalize()	1052
4.21.2.4 DoShattering()	1052

4.22 diagrams.c	1052
4.23 diawrap.cc	1061
4.24 dict.c File Reference	1073
4.24.1 Detailed Description	1074
4.24.2 Function Documentation	1074
4.24.2.1 TransformRational()	1074
4.24.2.2 IsMultiplySign()	1074
4.24.2.3 IsExponentSign()	1075
4.24.2.4 FindSymbol()	1075
4.24.2.5 FindVector()	1075
4.24.2.6 FindIndex()	1075
4.24.2.7 FindFunction()	1075
4.24.2.8 FindFunWithArgs()	1075
4.24.2.9 FindExtraSymbol()	1075
4.24.2.10 FindDictionary()	1076
4.24.2.11 AddDictionary()	1076
4.24.2.12 AddToDictionary()	1076
4.24.2.13 UseDictionary()	1076
4.24.2.14 SetDictionaryOptions()	1076
4.24.2.15 UnSetDictionary()	1076
4.24.2.16 RemoveDictionary()	1077
4.24.2.17 ShrinkDictionary()	1077
4.24.2.18 DoPreOpenDictionary()	1077
4.24.2.19 DoPreCloseDictionary()	1077
4.24.2.20 DoPreUseDictionary()	1077
4.24.2.21 DoPreAdd()	1077
4.24.2.22 DictToBytes()	1077
4.24.2.23 DictFromBytes()	1078
4.25 dict.c	1078
4.26 dollar.c File Reference	1091
4.26.1 Detailed Description	1092
4.26.2 Macro Definition Documentation	1092
4.26.2.1 STEP2	1092
4.26.3 Function Documentation	1093
4.26.3.1 CatchDollar()	1093
4.26.3.2 AssignDollar()	1093
4.26.3.3 WriteDollarToBuffer()	1093
4.26.3.4 WriteDollarFactorToBuffer()	1093
4.26.3.5 AddToDollarBuffer()	1093
4.26.3.6 TermAssign()	1093
4.26.3.7 PutTermInDollar()	1094
4.26.3.8 WildDollars()	1094

4.26.3.9 DolToTensor()	1094
4.26.3.10 DolToFunction()	1094
4.26.3.11 DolToVector()	1094
4.26.3.12 DolToNumber()	1094
4.26.3.13 DolToSymbol()	1094
4.26.3.14 DolToIndex()	1095
4.26.3.15 DolToTerms()	1095
4.26.3.16 DolToLong()	1095
4.26.3.17 ExecInside()	1095
4.26.3.18 InsideDollar()	1095
4.26.3.19 ExchangeDollars()	1095
4.26.3.20 TermsInDollar()	1095
4.26.3.21 SizeOfDollar()	1096
4.26.3.22 PelfDollarEval()	1096
4.26.3.23 TranslateExpression()	1096
4.26.3.24 IsSetMember()	1096
4.26.3.25 IsMultipleOf()	1096
4.26.3.26 TwoExprCompare()	1096
4.26.3.27 DollarRaiseLow()	1097
4.26.3.28 EvalDoLoopArg()	1097
4.26.3.29 TestDoLoop()	1097
4.26.3.30 TestEndDoLoop()	1098
4.26.3.31 DollarFactorize()	1098
4.26.3.32 CleanDollarFactors()	1098
4.26.3.33 TakeDollarContent()	1098
4.26.3.34 MakeDollarInteger()	1098
4.26.3.35 MakeDollarMod()	1099
4.26.3.36 GetDolNum()	1100
4.26.3.37 AddPotModdollar()	1100
4.27 dollar.c	1101
4.28 evaluate.c	1146
4.29 execute.c File Reference	1154
4.29.1 Detailed Description	1155
4.29.2 Macro Definition Documentation	1155
4.29.2.1 CURRENTBRACKET	1155
4.29.2.2 BRACKETCURRENTEXPR	1155
4.29.2.3 BRACKETOTHEREXPR	1156
4.29.2.4 NOBRACKETACTIVE	1156
4.29.3 Function Documentation	1156
4.29.3.1 CleanExpr()	1156
4.29.3.2 PopVariables()	1156
4.29.3.3 MakeGlobal()	1156

4.29.3.4 TestDrop()	1156
4.29.3.5 PutInVflags()	1156
4.29.3.6 DoExecute()	1157
4.29.3.7 PutBracket()	1157
4.29.3.8 SpecialCleanup()	1157
4.29.3.9 SetMods()	1157
4.29.3.10 UnSetMods()	1157
4.29.3.11 ExchangeExpressions()	1157
4.29.3.12 GetFirstBracket()	1158
4.29.3.13 GetFirstTerm()	1158
4.29.3.14 GetContent()	1159
4.29.3.15 CleanupTerm()	1159
4.29.3.16 ContentMerge()	1159
4.29.3.17 TermsInExpression()	1159
4.29.3.18 SizeOfExpression()	1160
4.29.3.19 UpdatePositions()	1160
4.29.3.20 CountTerms1()	1160
4.29.3.21 TermsInBracket()	1160
4.30 execute.c	1160
4.31 extcmd.c File Reference	1192
4.31.1 Detailed Description	1192
4.31.2 Function Documentation	1193
4.31.2.1 openExternalChannel()	1193
4.31.2.2 initPresetExternalChannels()	1193
4.31.2.3 closeExternalChannel()	1193
4.31.2.4 selectExternalChannel()	1193
4.31.2.5 getCurrentExternalChannel()	1193
4.31.2.6 closeAllExternalChannels()	1193
4.32 extcmd.c	1194
4.33 factor.c File Reference	1213
4.33.1 Detailed Description	1213
4.33.2 Function Documentation	1213
4.33.2.1 FactorIn()	1213
4.33.2.2 FactorInExpr()	1213
4.34 factor.c	1214
4.35 findpat.c File Reference	1224
4.35.1 Detailed Description	1224
4.35.2 Function Documentation	1224
4.35.2.1 FindOnly()	1224
4.35.2.2 FindOnce()	1225
4.35.2.3 FindMulti()	1225
4.35.2.4 FindRest()	1225

4.36 findpat.c	1225
4.37 flintinterface.cc File Reference	1241
4.37.1 Detailed Description	1242
4.37.2 Macro Definition Documentation	1242
4.37.2.1 IFW	1242
4.38 flintinterface.cc	1242
4.39 flintinterface.h File Reference	1268
4.39.1 Detailed Description	1270
4.39.2 Typedef Documentation	1270
4.39.2.1 var_map_t	1270
4.39.3 Function Documentation	1270
4.39.3.1 cleanup()	1270
4.39.3.2 cleanup_master()	1270
4.39.3.3 divmod_mpoly()	1270
4.39.3.4 divmod_poly()	1270
4.39.3.5 factorize_mpoly()	1271
4.39.3.6 factorize_poly()	1271
4.39.3.7 form_sort()	1271
4.39.3.8 from_argument_mpoly()	1271
4.39.3.9 from_argument_poly()	1271
4.39.3.10 fmpz_get_form()	1272
4.39.3.11 fmpz_set_form()	1272
4.39.3.12 gcd_mpoly()	1272
4.39.3.13 gcd_poly()	1272
4.39.3.14 get_variables()	1272
4.39.3.15 inverse_poly()	1273
4.39.3.16 mul_mpoly()	1273
4.39.3.17 mul_poly()	1273
4.39.3.18 ratfun_add_mpoly()	1273
4.39.3.19 ratfun_add_poly()	1273
4.39.3.20 ratfun_normalize_mpoly()	1274
4.39.3.21 ratfun_normalize_poly()	1274
4.39.3.22 ratfun_read_mpoly()	1274
4.39.3.23 ratfun_read_poly()	1274
4.39.3.24 startup_init()	1274
4.39.3.25 to_argument_mpoly() [1/2]	1275
4.39.3.26 to_argument_mpoly() [2/2]	1275
4.39.3.27 to_argument_poly() [1/2]	1275
4.39.3.28 to_argument_poly() [2/2]	1275
4.39.3.29 simplify_fmpz()	1276
4.39.3.30 simplify_fmpz_poly()	1276
4.39.3.31 fix_sign_fmpz_mpoly_ratfun()	1276

4.39.3.32 fix_sign_fmpz_poly_ratfun()	1276
4.40 flintinterface.h	1276
4.41 flintwrap.cc File Reference	1278
4.41.1 Detailed Description	1279
4.41.2 Function Documentation	1279
4.41.2.1 flint_final_cleanup_thread()	1279
4.41.2.2 flint_final_cleanup_master()	1279
4.41.2.3 flint_div()	1279
4.41.2.4 flint_factorize_argument()	1279
4.41.2.5 flint_factorize_dollar()	1280
4.41.2.6 flint_gcd()	1280
4.41.2.7 flint_inverse()	1280
4.41.2.8 flint_mul()	1280
4.41.2.9 flint_ratfun_add()	1280
4.41.2.10 flint_ratfun_normalize()	1280
4.41.2.11 flint_rem()	1281
4.41.2.12 flint_startup_init()	1281
4.42 flintwrap.cc	1281
4.43 float.c File Reference	1284
4.43.1 Detailed Description	1286
4.43.2 Macro Definition Documentation	1286
4.43.2.1 WITHCUTOFF	1286
4.43.2.2 GMPSPREAD	1286
4.43.3 Function Documentation	1286
4.43.3.1 Form_mpf_init()	1286
4.43.3.2 Form_mpf_clear()	1286
4.43.3.3 Form_mpf_set_prec_raw()	1286
4.43.3.4 FormtoZ()	1287
4.43.3.5 ZtoForm()	1287
4.43.3.6 FloatToInteger()	1287
4.43.3.7 IntegerToFloat()	1287
4.43.3.8 FloatToRat()	1287
4.43.3.9 AddFloats()	1287
4.43.3.10 MulFloats()	1288
4.43.3.11 DivFloats()	1288
4.43.3.12 AddRatToFloat()	1288
4.43.3.13 MulRatToFloat()	1288
4.43.3.14 SimpleDelta()	1288
4.43.3.15 SimpleDeltaC()	1289
4.43.3.16 SingleTable()	1289
4.43.3.17 DoubleTable()	1289
4.43.3.18 EndTable()	1289

4.43.3.19 deltaMZV()	1289
4.43.3.20 deltaEuler()	1290
4.43.3.21 deltaEulerC()	1290
4.43.3.22 CalculateMZVhalf()	1290
4.43.3.23 CalculateMZV()	1290
4.43.3.24 CalculateEuler()	1290
4.43.3.25 ExpandMZV()	1290
4.43.3.26 ExpandEuler()	1291
4.43.3.27 PackFloat()	1291
4.43.3.28 UnpackFloat()	1291
4.43.3.29 RatToFloat()	1291
4.43.3.30 Form_mpf_empty()	1291
4.43.3.31 TestFloat()	1291
4.43.3.32 FloatFunToRat()	1292
4.43.3.33 ZeroTable()	1292
4.43.3.34 ReadFloat()	1292
4.43.3.35 CheckFloat()	1292
4.43.3.36 SetFloatPrecision()	1292
4.43.3.37 PrintFloat()	1292
4.43.3.38 SetupMZVTables()	1293
4.43.3.39 SetupMPFTables()	1293
4.43.3.40 ClearMZVTables()	1293
4.43.3.41 CoToFloat()	1293
4.43.3.42 CoToRat()	1293
4.43.3.43 ToFloat()	1293
4.43.3.44 ToRat()	1293
4.43.3.45 AddWithFloat()	1294
4.43.3.46 MergeWithFloat()	1294
4.43.3.47 EvaluateEuler()	1294
4.43.3.48 CoEvaluate()	1294
4.44 float.c	1294
4.45 form3.h File Reference	1323
4.45.1 Detailed Description	1325
4.45.2 Macro Definition Documentation	1325
4.45.2.1 MAJORVERSION	1325
4.45.2.2 MINORVERSION	1325
4.45.2.3 PRODUCTIONDATE	1326
4.45.2.4 BETAVERSION	1326
4.45.2.5 inline	1326
4.45.2.6 NDEBUG	1326
4.45.2.7 STATIC_ASSERT	1326
4.45.2.8 STATIC_ASSERT__1	1326

4.45.2.9	STATIC_ASSERT__2	1326
4.45.2.10	STATIC_ASSERT__3	1327
4.45.2.11	TOPBITONLY	1327
4.45.2.12	TOPLONGBITONLY	1327
4.45.2.13	SPECMASK	1327
4.45.2.14	WILDMASK	1327
4.45.2.15	WORDMASK	1327
4.45.2.16	AWORDMASK	1327
4.45.2.17	FULLMAX	1328
4.45.2.18	MAXPOSITIVE	1328
4.45.2.19	MAXLONG	1328
4.45.2.20	MAXPOSITIVE2	1328
4.45.2.21	MAXPOSITIVE4	1328
4.45.2.22	form_alignof	1328
4.45.2.23	PADDUMMY	1328
4.45.2.24	PADPOSITION	1329
4.45.2.25	PADPOINTER	1329
4.45.2.26	PADLONG	1329
4.45.2.27	PADINT	1330
4.45.2.28	PADWORD	1330
4.45.2.29	WITHSORTBOTS	1330
4.45.2.30	FILES	1330
4.45.2.31	Uopen	1330
4.45.2.32	Uflush	1330
4.45.2.33	Uclose	1331
4.45.2.34	Uread	1331
4.45.2.35	Uwrite	1331
4.45.2.36	Usetbuf	1331
4.45.2.37	Useek	1331
4.45.2.38	Utell	1331
4.45.2.39	Ugetpos	1332
4.45.2.40	Usetpos	1332
4.45.2.41	Usync	1332
4.45.2.42	Utruncate	1332
4.45.2.43	Ustdout	1332
4.45.2.44	MAX_OPEN_FILES	1332
4.45.2.45	bzero	1332
4.45.2.46	GetPID	1333
4.45.3	Typedef Documentation	1333
4.45.3.1	SBYTE	1333
4.45.3.2	UBYTE	1333
4.45.3.3	UINT	1333

4.45.3.4 RLONG	1333
4.45.3.5 MLONG	1333
4.45.3.6 BOOL	1333
4.46 form3.h	1334
4.47 fsizes.h File Reference	1340
4.47.1 Detailed Description	1342
4.47.2 Macro Definition Documentation	1342
4.47.2.1 MAXPRENAMESIZE	1342
4.47.2.2 MAXPOWER	1342
4.47.2.3 MAXVARIABLES	1342
4.47.2.4 MAXDOLLARVARIABLES	1342
4.47.2.5 WILDOFFSET	1342
4.47.2.6 MAXINNAMETREE	1343
4.47.2.7 MAXDUMMIES	1343
4.47.2.8 WORKBUFFER	1343
4.47.2.9 MAXTER	1343
4.47.2.10 HALFMAX	1343
4.47.2.11 MAXSUBEXPRESSIONS	1343
4.47.2.12 MAXFILESTREAMSIZE	1343
4.47.2.13 MAXENAME	1343
4.47.2.14 MAXSAVEFUNCTION	1344
4.47.2.15 MAXPARLEVEL	1344
4.47.2.16 MAXNUMBERSIZE	1344
4.47.2.17 MAXREPEAT	1344
4.47.2.18 NORMSIZE	1344
4.47.2.19 INITNODESIZE	1344
4.47.2.20 INITNAMESIZE	1344
4.47.2.21 NUMFIXED	1344
4.47.2.22 MAXNEST	1345
4.47.2.23 MAXMATCH	1345
4.47.2.24 MAXIF	1345
4.47.2.25 SIZEFACS	1345
4.47.2.26 NUMFACS	1345
4.47.2.27 MAXLOOPS	1345
4.47.2.28 MAXLABELS	1345
4.47.2.29 COMMERCIALSIZE	1345
4.47.2.30 MAXFLAGS	1346
4.47.2.31 COMPRESSBUFFER	1346
4.47.2.32 FORTRANCONTINUATIONLINES	1346
4.47.2.33 MAXLEVELS	1346
4.47.2.34 MAXLHS	1346
4.47.2.35 MAXWILDC	1346

4.47.2.36 NUMTABLEENTRIES	1346
4.47.2.37 COMPILERBUFFER	1346
4.47.2.38 SMALLBUFFER	1347
4.47.2.39 SMALLOVERFLOW	1347
4.47.2.40 TERMSSMALL	1347
4.47.2.41 LARGEBUFFER	1347
4.47.2.42 SCRATCHSIZE	1347
4.47.2.43 MAXPATCHES	1347
4.47.2.44 MAXFPATCHES	1347
4.47.2.45 SORTIOSIZE	1347
4.47.2.46 SSMALLBUFFER	1348
4.47.2.47 SSMALLOVERFLOW	1348
4.47.2.48 STERMSSMALL	1348
4.47.2.49 SLARGEBUFFER	1348
4.47.2.50 SMAXPATCHES	1348
4.47.2.51 SMAXFPATCHES	1348
4.47.2.52 SSORTIOSIZE	1348
4.47.2.53 SPECTATORSIZE	1348
4.47.2.54 MAXFLEVELS	1349
4.47.2.55 COMPINC	1349
4.47.2.56 MAXNUMSIZE	1349
4.47.2.57 MAXBRACKETBUFFERSIZE	1349
4.47.2.58 SFHSIZE	1349
4.47.2.59 DEFAULTPROCESSBUCKETSIZE	1349
4.47.2.60 SHMWINSIZE	1349
4.47.2.61 TABLEEXTENSION	1349
4.47.2.62 GZIPDEFAULT	1350
4.47.2.63 DEFAULTTHREADS	1350
4.47.2.64 DEFAULTTHREADBUCKETSIZE	1350
4.47.2.65 DEFAULTTHREADLOADBALANCING	1350
4.47.2.66 THREADSCRATCHSIZE	1350
4.47.2.67 THREADSCRATCHOUTSIZE	1350
4.47.2.68 MAXTABLECOMBUF	1350
4.47.2.69 MAXCOMBUFRHS	1350
4.47.2.70 NUMSTORECACHES	1351
4.47.2.71 SIZESTORECACHE	1351
4.47.2.72 INDENTSPACE	1351
4.47.2.73 MULTIINDENTSPACE	1351
4.47.2.74 MAXMULTIBRACKETLEVELS	1351
4.47.2.75 FBUFFERSIZE	1351
4.47.2.76 NPAIR1	1351
4.47.2.77 NPAIR2	1351

4.47.2.78 MAXLINELENGTH	1352
4.47.2.79 MINALLOC	1352
4.47.2.80 JUMPRATIO	1352
4.47.2.81 MAXPARTICLES	1352
4.47.2.82 MAXCOUPLINGS	1352
4.47.2.83 NUMOPTIONS	1352
4.47.2.84 MAXLEGS	1352
4.48 fsizes.h	1353
4.49 ftypes.h File Reference	1355
4.49.1 Detailed Description	1367
4.49.2 Macro Definition Documentation	1367
4.49.2.1 WITHOUTERROR	1367
4.49.2.2 WITHERROR	1367
4.49.2.3 FILESTREAM	1368
4.49.2.4 PREVARSTREAM	1368
4.49.2.5 PREREADSTREAM	1368
4.49.2.6 PIPESTREAM	1368
4.49.2.7 PRECALCSTREAM	1368
4.49.2.8 DOLLARSTREAM	1368
4.49.2.9 PREREADSTREAM2	1368
4.49.2.10 EXTERNALCHANNELSTREAM	1368
4.49.2.11 PREREADSTREAM3	1369
4.49.2.12 REVERSEFILESTREAM	1369
4.49.2.13 INPUTSTREAM	1369
4.49.2.14 ENDOFSTREAM	1369
4.49.2.15 ENDOFINPUT	1369
4.49.2.16 SUBROUTINEFILE	1369
4.49.2.17 PROCEDUREFILE	1369
4.49.2.18 HEADERFILE	1369
4.49.2.19 SETUPFILE	1370
4.49.2.20 TABLEBASEFILE	1370
4.49.2.21 FIRSTMODULE	1370
4.49.2.22 GLOBALMODULE	1370
4.49.2.23 SORTMODULE	1370
4.49.2.24 STOREMODULE	1370
4.49.2.25 CLEARMODULE	1370
4.49.2.26 ENDMODULE	1370
4.49.2.27 POLYFUN	1371
4.49.2.28 NOPARALLEL_DOLLAR	1371
4.49.2.29 NOPARALLEL_RHS	1371
4.49.2.30 NOPARALLEL_CONVPOLY	1371
4.49.2.31 NOPARALLEL_SPECTATOR	1371

4.49.2.32 NOPARALLEL_USER	1371
4.49.2.33 NOPARALLEL_TBLDOLLAR	1371
4.49.2.34 NOPARALLEL_NPROC	1371
4.49.2.35 PARALLELFLAG	1372
4.49.2.36 PRENOACTION	1372
4.49.2.37 PRERAISEAFTER	1372
4.49.2.38 PRELOWERAFTER	1372
4.49.2.39 WITHSEMICOLON	1372
4.49.2.40 WITHOUTSEMICOLON	1372
4.49.2.41 MODULEINSTACK	1372
4.49.2.42 EXECUTINGIF	1372
4.49.2.43 LOOKINGFORELSE	1373
4.49.2.44 LOOKINGFORENDIF	1373
4.49.2.45 NEWSTATEMENT	1373
4.49.2.46 OLDSTATEMENT	1373
4.49.2.47 EXECUTINGPRESWITCH	1373
4.49.2.48 SEARCHINGPRECASE	1373
4.49.2.49 SEARCHINGPREENDSWITCH	1373
4.49.2.50 PREPROONLY	1373
4.49.2.51 DUMPTOCOMPILER	1374
4.49.2.52 DUMPOUTTERMS	1374
4.49.2.53 DUMPINTERMS	1374
4.49.2.54 DUMPTOSORT	1374
4.49.2.55 DUMPTOPARALLEL	1374
4.49.2.56 THREADSDEBUG	1374
4.49.2.57 ERROROUT	1374
4.49.2.58 INPUTOUT	1374
4.49.2.59 STATSOUT	1375
4.49.2.60 EXPRSOUT	1375
4.49.2.61 WRITEOUT	1375
4.49.2.62 EXTERNALCHANNELOUT	1375
4.49.2.63 NUMERICALLOOP	1375
4.49.2.64 LISTEDLOOP	1375
4.49.2.65 ONEEXPRESSION	1375
4.49.2.66 PRETYPENONE	1375
4.49.2.67 PRETYPEIF	1376
4.49.2.68 PRETYPEDO	1376
4.49.2.69 PRETYPEPROCEDURE	1376
4.49.2.70 PRETYPESWITCH	1376
4.49.2.71 PRETYPEINSIDE	1376
4.49.2.72 DECLARATION	1376
4.49.2.73 SPECIFICATION	1376

4.49.2.74 DEFINITION	1376
4.49.2.75 STATEMENT	1377
4.49.2.76 TOOOUTPUT	1377
4.49.2.77 ATENDOFMODULE	1377
4.49.2.78 MIXED2	1377
4.49.2.79 MIXED	1377
4.49.2.80 NOAUTO	1377
4.49.2.81 PARTEST	1377
4.49.2.82 WITHAUTO	1377
4.49.2.83 ALLVARIABLES	1378
4.49.2.84 SYMBOLONLY	1378
4.49.2.85 INDEXONLY	1378
4.49.2.86 VECTORONLY	1378
4.49.2.87 FUNCTIONONLY	1378
4.49.2.88 SETONLY	1378
4.49.2.89 EXPRESSIONONLY	1378
4.49.2.90 CDELETE	1378
4.49.2.91 ANYTYPE	1379
4.49.2.92 CSYMBOL	1379
4.49.2.93 CINDEX	1379
4.49.2.94 CVECTOR	1379
4.49.2.95 CFUNCTION	1379
4.49.2.96 CSET	1379
4.49.2.97 CEXPRESSION	1379
4.49.2.98 CDOTPRODUCT	1379
4.49.2.99 CNUMBER	1380
4.49.2.100 CSUBEXP	1380
4.49.2.101 CDELTA	1380
4.49.2.102 CDOLLAR	1380
4.49.2.103 CDUBIOUS	1380
4.49.2.104 CRANGE	1380
4.49.2.105 CMODEL	1380
4.49.2.106 CVECTOR1	1380
4.49.2.107 CDOUBLEDOT	1381
4.49.2.108 TSYMBOL	1381
4.49.2.109 TINDEX	1381
4.49.2.110 TVECTOR	1381
4.49.2.111 TFUNCTION	1381
4.49.2.112 TSET	1381
4.49.2.113 TEXPRESSON	1381
4.49.2.114 TDOTPRODUCT	1381
4.49.2.115 TNUMBER	1382

4.49.2.116 TSUBEXP	1382
4.49.2.117 TDELTA	1382
4.49.2.118 TDOLLAR	1382
4.49.2.119 TDUBIOUS	1382
4.49.2.120 LPARENTHESIS	1382
4.49.2.121 RPARENTHESIS	1382
4.49.2.122 TWILDCARD	1382
4.49.2.123 TWILDARG	1383
4.49.2.124 TDOT	1383
4.49.2.125 LBRACE	1383
4.49.2.126 RBRACE	1383
4.49.2.127 TCOMMA	1383
4.49.2.128 TFUNOPEN	1383
4.49.2.129 TFUNCLOSE	1383
4.49.2.130 TMULTIPLY	1383
4.49.2.131 TDIVIDE	1384
4.49.2.132 TPOWER	1384
4.49.2.133 TPLUS	1384
4.49.2.134 TMINUS	1384
4.49.2.135 TNOT	1384
4.49.2.136 TENDOFIT	1384
4.49.2.137 TSETOPEN	1384
4.49.2.138 TSETCLOSE	1384
4.49.2.139 TGENINDEX	1385
4.49.2.140 TCONJUGATE	1385
4.49.2.141 LRPARENTHESSES	1385
4.49.2.142 TNUMBER1	1385
4.49.2.143 TPOWER1	1385
4.49.2.144 TEMPTY	1385
4.49.2.145 TSETNUM	1385
4.49.2.146 TSGAMMA	1385
4.49.2.147 TSETDOL	1386
4.49.2.148 TFLOAT	1386
4.49.2.149 TYPEISFUN	1386
4.49.2.150 TYPEISSUB	1386
4.49.2.151 TYPEISMYSTERY	1386
4.49.2.152 LHSIDEX	1386
4.49.2.153 LHSIDE	1386
4.49.2.154 RHSIDE	1386
4.49.2.155 FORTRANMODE	1387
4.49.2.156 REDUCEMODE	1387
4.49.2.157 MAPLEMODE	1387

4.49.2.158 MATHEMATICAMODE	1387
4.49.2.159 CMODE	1387
4.49.2.160 VORTTRANMODE	1387
4.49.2.161 PFORTTRANMODE	1387
4.49.2.162 DOUBLEFORTTRANMODE	1387
4.49.2.163 DOUBLEPRECISIONFLAG	1388
4.49.2.164 NODOUBLEMASK	1388
4.49.2.165 QUADRUPLEFORTTRANMODE	1388
4.49.2.166 QUADRUPLEPRECISIONFLAG	1388
4.49.2.167 ALLINTEGERDOUBLE	1388
4.49.2.168 NOQUADMASK	1388
4.49.2.169 NORMALFORMAT	1388
4.49.2.170 NOSPACEFORMAT	1388
4.49.2.171 ISNOTFORTTRAN90	1389
4.49.2.172 ISFORTTRAN90	1389
4.49.2.173 ALSOREVERSE	1389
4.49.2.174 CHISHOLM	1389
4.49.2.175 NOTRICK	1389
4.49.2.176 SORTLOWFIRST	1389
4.49.2.177 SORTHIGHFIRST	1389
4.49.2.178 SORTPOWERFIRST	1389
4.49.2.179 SORTANTIPOWER	1390
4.49.2.180 STATSSPLITMERGE	1390
4.49.2.181 STATSMERGETOFILE	1390
4.49.2.182 STATSPOSTSORT	1390
4.49.2.183 NOCHECKLOGTYPE	1390
4.49.2.184 CHECKLOGTYPE	1390
4.49.2.185 NMIN4SHIFT	1390
4.49.2.186 SYMBOL	1390
4.49.2.187 DOTPRODUCT	1391
4.49.2.188 VECTOR	1391
4.49.2.189 INDEX	1391
4.49.2.190 EXPRESSION	1391
4.49.2.191 SUBEXPRESSION	1391
4.49.2.192 DOLLAREXPRESSION	1391
4.49.2.193 SETSET	1391
4.49.2.194 ARGWILD	1391
4.49.2.195 MINVECTOR	1392
4.49.2.196 SETEXP	1392
4.49.2.197 DOLLAREXP2	1392
4.49.2.198 HAAKJE0	1392
4.49.2.199 FUNCTION	1392

4.49.2.200 TMPPOLYFUN	1392
4.49.2.201 ARGFIELD	1392
4.49.2.202 SNUMBER	1392
4.49.2.203 LNUMBER	1393
4.49.2.204 HAAKJE	1393
4.49.2.205 DELTA	1393
4.49.2.206 EXPONENT	1393
4.49.2.207 DENOMINATOR	1393
4.49.2.208 SETFUNCTION	1393
4.49.2.209 GAMMA	1393
4.49.2.210 GAMMAI	1393
4.49.2.211 GAMMAFIVE	1394
4.49.2.212 GAMMASIX	1394
4.49.2.213 GAMMASEVEN	1394
4.49.2.214 SUMF1	1394
4.49.2.215 SUMF2	1394
4.49.2.216 DUMFUN	1394
4.49.2.217 REPLACEMENT	1394
4.49.2.218 REVERSEFUNCTION	1394
4.49.2.219 DISTRIBUTION	1395
4.49.2.220 DELTA3	1395
4.49.2.221 DUMMYFUN	1395
4.49.2.222 DUMMYTEN	1395
4.49.2.223 LEVICIVITA	1395
4.49.2.224 FACTORIAL	1395
4.49.2.225 INVERSEFACTORIAL	1395
4.49.2.226 BINOMIAL	1395
4.49.2.227 NUMARGSFUN	1396
4.49.2.228 SIGNFUN	1396
4.49.2.229 MODFUNCTION	1396
4.49.2.230 MOD2FUNCTION	1396
4.49.2.231 MINFUNCTION	1396
4.49.2.232 MAXFUNCTION	1396
4.49.2.233 ABSFUNCTION	1396
4.49.2.234 SIGFUNCTION	1396
4.49.2.235 INTFUNCTION	1397
4.49.2.236 THETA	1397
4.49.2.237 THETA2	1397
4.49.2.238 DELTA2	1397
4.49.2.239 DELTAP	1397
4.49.2.240 BERNOULLIFUNCTION	1397
4.49.2.241 COUNTFUNCTION	1397

4.49.2.242 MATCHFUNCTION	1397
4.49.2.243 PATTERNFUNCTION	1398
4.49.2.244 TERMFUNCTION	1398
4.49.2.245 CONJUGATION	1398
4.49.2.246 ROOTFUNCTION	1398
4.49.2.247 TABLEFUNCTION	1398
4.49.2.248 FIRSTBRACKET	1398
4.49.2.249 TERMSINEXPR	1398
4.49.2.250 NUMTERMSFUN	1398
4.49.2.251 GCDFUNCTION	1399
4.49.2.252 DIVFUNCTION	1399
4.49.2.253 REMFUNCTION	1399
4.49.2.254 MAXPOWEROF	1399
4.49.2.255 MINPOWEROF	1399
4.49.2.256 TABLESTUB	1399
4.49.2.257 FACTORIN	1399
4.49.2.258 TERMSINBRACKET	1399
4.49.2.259 WILDARGFUN	1400
4.49.2.260 SQRTFUNCTION	1400
4.49.2.261 LNFUNCTION	1400
4.49.2.262 SINFUNCTION	1400
4.49.2.263 COSFUNCTION	1400
4.49.2.264 TANFUNCTION	1400
4.49.2.265 ASINFUNCTION	1400
4.49.2.266 ACOSFUNCTION	1400
4.49.2.267 ATANFUNCTION	1401
4.49.2.268 ATAN2FUNCTION	1401
4.49.2.269 SINHFUNCTION	1401
4.49.2.270 COSHFUNCTION	1401
4.49.2.271 TANHFUNCTION	1401
4.49.2.272 ASINHFUNCTION	1401
4.49.2.273 ACOSHFUNCTION	1401
4.49.2.274 ATANHFUNCTION	1401
4.49.2.275 LI2FUNCTION	1402
4.49.2.276 LINFUNCTION	1402
4.49.2.277 EXTRASYMFUN	1402
4.49.2.278 RANDOMFUNCTION	1402
4.49.2.279 RANPERM	1402
4.49.2.280 NUMFACTORS	1402
4.49.2.281 FIRSTTERM	1402
4.49.2.282 CONTENTTERM	1402
4.49.2.283 PRIMENUMBER	1403

4.49.2.284 EXTEUCLIDEAN	1403
4.49.2.285 MAKERATIONAL	1403
4.49.2.286 INVERSEFUNCTION	1403
4.49.2.287 IDFUNCTION	1403
4.49.2.288 PUTFIRST	1403
4.49.2.289 PERMUTATIONS	1403
4.49.2.290 PARTITIONS	1403
4.49.2.291 MULFUNCTION	1404
4.49.2.292 MOEBIUS	1404
4.49.2.293 TOPOLOGIES	1404
4.49.2.294 DIAGRAMS	1404
4.49.2.295 TOPO	1404
4.49.2.296 NODEFUNCTION	1404
4.49.2.297 EDGE	1404
4.49.2.298 SIZEOFFUNCTION	1404
4.49.2.299 BLOCK	1405
4.49.2.300 ONEPI	1405
4.49.2.301 PHI	1405
4.49.2.302 MAXBUILTINFUNCTION	1405
4.49.2.303 FIRSTUSERFUNCTION	1405
4.49.2.304 ALLFUNCTIONS	1405
4.49.2.305 ALLMZVFUNCTIONS	1405
4.49.2.306 ISYMBOL	1405
4.49.2.307 PISYMBOL	1406
4.49.2.308 COEFFSYMBOL	1406
4.49.2.309 NUMERATORSYMBOL	1406
4.49.2.310 DENOMINATORSYMBOL	1406
4.49.2.311 WILDARGSYMBOL	1406
4.49.2.312 DIMENSIONSYMBOL	1406
4.49.2.313 FACTORSYMBOL	1406
4.49.2.314 SEPARATESYMBOL	1406
4.49.2.315 EESYMBOL	1407
4.49.2.316 EMSYMBOL	1407
4.49.2.317 BUILTINSYMBOLS	1407
4.49.2.318 FIRSTUSERSYMBOL	1407
4.49.2.319 BUILTININDICES	1407
4.49.2.320 BUILTINVECTORS	1407
4.49.2.321 BUILTINDOLLARS	1407
4.49.2.322 WILDARGVECTOR	1407
4.49.2.323 WILDARGINDEX	1408
4.49.2.324 TYPEEXPRESSION	1408
4.49.2.325 TYPEIDNEW	1408

4.49.2.326 TYPEIDOLD	1408
4.49.2.327 TYPEOPERATION	1408
4.49.2.328 TYPEREPEAT	1408
4.49.2.329 TYPEENDREPEAT	1408
4.49.2.330 TYPECOUNT	1408
4.49.2.331 TYPEMULT	1409
4.49.2.332 TYPEGOTO	1409
4.49.2.333 TYPEDISCARD	1409
4.49.2.334 TYPEIF	1409
4.49.2.335 TYPEELSE	1409
4.49.2.336 TYPEELIF	1409
4.49.2.337 TYPEENDIF	1409
4.49.2.338 TYPESUM	1409
4.49.2.339 TYPECHISHOLM	1410
4.49.2.340 TYPEREVERSE	1410
4.49.2.341 TYPEARG	1410
4.49.2.342 TYPENORM	1410
4.49.2.343 TYPENORM2	1410
4.49.2.344 TYPENORM3	1410
4.49.2.345 TYPEEXIT	1410
4.49.2.346 TYPESETEXIT	1410
4.49.2.347 TYPEPRINT	1411
4.49.2.348 TYPEFPRINT	1411
4.49.2.349 TYPEREDEFPRE	1411
4.49.2.350 TYPESPLITARG	1411
4.49.2.351 TYPESPLITARG2	1411
4.49.2.352 TYPEFACTARG	1411
4.49.2.353 TYPEFACTARG2	1411
4.49.2.354 TYPETRY	1411
4.49.2.355 TYPEASSIGN	1412
4.49.2.356 TYPERENUMBER	1412
4.49.2.357 TYPESUMFIX	1412
4.49.2.358 TYPEFINDLOOP	1412
4.49.2.359 TYPEUNRAVEL	1412
4.49.2.360 TYPEADJUSTBOUNDS	1412
4.49.2.361 TYPEINSIDE	1412
4.49.2.362 TYPETERM	1412
4.49.2.363 TYPESORT	1413
4.49.2.364 TYPEDETCURDUM	1413
4.49.2.365 TYPEINEXPRESSION	1413
4.49.2.366 TYPESPLITFIRSTARG	1413
4.49.2.367 TYPESPLITLASTARG	1413

4.49.2.368 TYPEMERGE	1413
4.49.2.369 TYPETESTUSE	1413
4.49.2.370 TYPEAPPLY	1413
4.49.2.371 TYPEAPPLYRESET	1414
4.49.2.372 TYPECHAININ	1414
4.49.2.373 TYPECHAINOUT	1414
4.49.2.374 TYPENORM4	1414
4.49.2.375 TYPEFACTOR	1414
4.49.2.376 TYPEARGIMPLODE	1414
4.49.2.377 TYPEARGEXPLODE	1414
4.49.2.378 TYPEDENOMINATORS	1414
4.49.2.379 TYPESTUFFLE	1415
4.49.2.380 TYPEDROPCOEFFICIENT	1415
4.49.2.381 TYPETRANSFORM	1415
4.49.2.382 TYPETOPOLYNOMIAL	1415
4.49.2.383 TYPEFROMPOLYNOMIAL	1415
4.49.2.384 TYPEDOLOOP	1415
4.49.2.385 TYPEENDDOLOOP	1415
4.49.2.386 TYPEDROPSYMBOLS	1415
4.49.2.387 TYPEPUTINSIDE	1416
4.49.2.388 TYPETOSPECTATOR	1416
4.49.2.389 TYPEARGTOEXTRASymbol	1416
4.49.2.390 TYPECANONICALIZE	1416
4.49.2.391 TYPESWITCH	1416
4.49.2.392 TYPEENDSWITCH	1416
4.49.2.393 TYPESETUSERFLAG	1416
4.49.2.394 TYPECLEARUSERFLAG	1416
4.49.2.395 TYPEALLLOOPS	1417
4.49.2.396 TYPEALLPATHS	1417
4.49.2.397 TAKETRACE	1417
4.49.2.398 CONTRACT	1417
4.49.2.399 RATIO	1417
4.49.2.400 SYMMETRIZE	1417
4.49.2.401 TENVEC	1417
4.49.2.402 SUMNUM1	1417
4.49.2.403 SUMNUM2	1418
4.49.2.404 WILDDUMMY	1418
4.49.2.405 SYMTONUM	1418
4.49.2.406 SYMTOSYM	1418
4.49.2.407 SYMTOSUB	1418
4.49.2.408 VECTOMIN	1418
4.49.2.409 VECTOVEC	1418

4.49.2.410 VECTOSUB	1418
4.49.2.411 INDTOIND	1419
4.49.2.412 INDTOSUB	1419
4.49.2.413 FUNTOFUN	1419
4.49.2.414 ARGTOARG	1419
4.49.2.415 ARLTOARL	1419
4.49.2.416 EXPTOEXP	1419
4.49.2.417 FROMBRAC	1419
4.49.2.418 FROMSET	1419
4.49.2.419 SETTONUM	1420
4.49.2.420 WILDCARDS	1420
4.49.2.421 SETNUMBER	1420
4.49.2.422 LOADDOLLAR	1420
4.49.2.423 NUMTONUM	1420
4.49.2.424 NUMTOSYM	1420
4.49.2.425 NUMTOIND	1420
4.49.2.426 NUMTOSUB	1420
4.49.2.427 EATTENSOR	1421
4.49.2.428 CLEANFLAG	1421
4.49.2.429 DIRTYFLAG	1421
4.49.2.430 DIRTYSYMFLAG	1421
4.49.2.431 MUSTCLEANPRF	1421
4.49.2.432 SUBTERMUSED1	1421
4.49.2.433 SUBTERMUSED2	1421
4.49.2.434 ALLDIRTY	1421
4.49.2.435 ARGHEAD	1422
4.49.2.436 FUNHEAD	1422
4.49.2.437 SUBEXPSIZE	1422
4.49.2.438 EXPRHEAD	1422
4.49.2.439 TYPEARGHEADSIZE	1422
4.49.2.440 SKIP	1422
4.49.2.441 DROP	1422
4.49.2.442 HIDE	1422
4.49.2.443 UNHIDE	1423
4.49.2.444 INTOHIDE	1423
4.49.2.445 LOCALEXPRESSION	1423
4.49.2.446 SKIPLEXPRESSION	1423
4.49.2.447 DROPEXPRESSION	1423
4.49.2.448 DROPEDEXPRESSION	1423
4.49.2.449 GLOBALEXPRESSION	1423
4.49.2.450 SKIPGEXPRESSION	1423
4.49.2.451 DROPGEXPRESSION	1424

4.49.2.452 STOREDEXPRESSION	1424
4.49.2.453 HIDDENLEXPRESSION	1424
4.49.2.454 HIDDENGEXPRESSION	1424
4.49.2.455 INCEXPRESSION	1424
4.49.2.456 HIDELEXPRESSION	1424
4.49.2.457 HIDEGEXPRESSION	1424
4.49.2.458 DROPHLEXPRESSION	1424
4.49.2.459 DROPHGEXPRESSION	1425
4.49.2.460 UNHIDELEXPRESSION	1425
4.49.2.461 UNHIDEGEXPRESSION	1425
4.49.2.462 INTOHIDELEXPRESSION	1425
4.49.2.463 INTOHIDEGEXPRESSION	1425
4.49.2.464 SPECTATOREXPRESSION	1425
4.49.2.465 DROPSPECTATOREXPRESSION	1425
4.49.2.466 SKIPUNHIDELEXPRESSION	1425
4.49.2.467 SKIPUNHIDEGEXPRESSION	1426
4.49.2.468 PRINTOFF	1426
4.49.2.469 PRINTON	1426
4.49.2.470 PRINTCONTENTS	1426
4.49.2.471 PRINTCONTENT	1426
4.49.2.472 PRINTLFILE	1426
4.49.2.473 PRINTONETERM	1426
4.49.2.474 PRINTONEFUNCTION	1426
4.49.2.475 PRINTALL	1427
4.49.2.476 REGULAREXPRESSION	1427
4.49.2.477 REDEFINEDEXPRESSION	1427
4.49.2.478 NEWLYDEFINEDEXPRESSION	1427
4.49.2.479 VERTEXFUNCTION	1427
4.49.2.480 GENERALFUNCTION	1427
4.49.2.481 TENSORFUNCTION	1427
4.49.2.482 GAMMAFUNCTION	1427
4.49.2.483 POS_	1428
4.49.2.484 POS0_	1428
4.49.2.485 NEG_	1428
4.49.2.486 NEG0_	1428
4.49.2.487 EVEN_	1428
4.49.2.488 ODD_	1428
4.49.2.489 Z_	1428
4.49.2.490 SYMBOL_	1428
4.49.2.491 FIXED_	1429
4.49.2.492 INDEX_	1429
4.49.2.493 Q_	1429

4.49.2.494 DUMMYINDEX_	1429
4.49.2.495 VECTOR_	1429
4.49.2.496 GAMMA1	1429
4.49.2.497 GAMMA5	1429
4.49.2.498 GAMMA6	1429
4.49.2.499 GAMMA7	1430
4.49.2.500 FUNNYVEC	1430
4.49.2.501 FUNNYWILD	1430
4.49.2.502 SUMMEDIND	1430
4.49.2.503 NOINDEX	1430
4.49.2.504 FUNNYDOLLAR	1430
4.49.2.505 EMPTYINDEX	1430
4.49.2.506 MINSPEC	1430
4.49.2.507 USEDFLAG	1431
4.49.2.508 DUMMYFLAG	1431
4.49.2.509 MAINSORT	1431
4.49.2.510 FUNCTIONSORT	1431
4.49.2.511 SUBSORT	1431
4.49.2.512 FLOATMODE	1431
4.49.2.513 RATIONALMODE	1431
4.49.2.514 NUMSPECSETS	1431
4.49.2.515 ISZERO	1432
4.49.2.516 ISUNMODIFIED	1432
4.49.2.517 ISCOMPRESSED	1432
4.49.2.518 ISINRHS	1432
4.49.2.519 ISFACTORIZED	1432
4.49.2.520 TOBEFACTORED	1432
4.49.2.521 TOBEUNFACTORED	1432
4.49.2.522 KEEPZERO	1432
4.49.2.523 VARTYPENONE	1433
4.49.2.524 VARTYPECOMPLEX	1433
4.49.2.525 VARTYPEIMAGINARY	1433
4.49.2.526 VARTYPEEROOTOFFUNITY	1433
4.49.2.527 VARTYPEMINUS	1433
4.49.2.528 CYCLESYMMETRIC	1433
4.49.2.529 RCYCLESYMMETRIC	1433
4.49.2.530 SYMMETRIC	1433
4.49.2.531 ANTISYMMETRIC	1434
4.49.2.532 REVERSEORDER	1434
4.49.2.533 SUBMULTI	1434
4.49.2.534 SUBONCE	1434
4.49.2.535 SUBONLY	1434

4.49.2.536 SUBMANY	1434
4.49.2.537 SUBVECTOR	1434
4.49.2.538 SUBSELECT	1434
4.49.2.539 SUBALL	1435
4.49.2.540 SUBMASK	1435
4.49.2.541 SUBDISORDER	1435
4.49.2.542 SUBAFTER	1435
4.49.2.543 SUBAFTERNOT	1435
4.49.2.544 IDHEAD	1435
4.49.2.545 DOLLARFLAG	1435
4.49.2.546 NORMALIZEFLAG	1435
4.49.2.547 GIDENT	1436
4.49.2.548 GFIVE	1436
4.49.2.549 GPLUS	1436
4.49.2.550 GMINUS	1436
4.49.2.551 LONGNUMBER	1436
4.49.2.552 MATCH	1436
4.49.2.553 COEFFI	1436
4.49.2.554 SUBEXPR	1436
4.49.2.555 MULTIPLEOF	1437
4.49.2.556 IFDOLLAR	1437
4.49.2.557 IFEXPRESSION	1437
4.49.2.558 IFDOLLAREXTRA	1437
4.49.2.559 IFISFACTORIZED	1437
4.49.2.560 IFOCCURS	1437
4.49.2.561 IFUSERFLAG	1437
4.49.2.562 IFFLOATNUMBER	1437
4.49.2.563 GREATER	1438
4.49.2.564 GREATEREQUAL	1438
4.49.2.565 LESS	1438
4.49.2.566 LESSEQUAL	1438
4.49.2.567 EQUAL	1438
4.49.2.568 NOTEQUAL	1438
4.49.2.569 ORCOND	1438
4.49.2.570 ANDCOND	1438
4.49.2.571 DUMMY	1439
4.49.2.572 SORT	1439
4.49.2.573 STORE	1439
4.49.2.574 END	1439
4.49.2.575 GLOBAL	1439
4.49.2.576 CLEAR	1439
4.49.2.577 VECTBIT	1439

4.49.2.578 DOTPBIT	1439
4.49.2.579 FUNBIT	1440
4.49.2.580 SETBIT	1440
4.49.2.581 EXTRAPARAMETER	1440
4.49.2.582 GENCOMMUTE	1440
4.49.2.583 GENNONCOMMUTE	1440
4.49.2.584 NAMENOTFOUND	1440
4.49.2.585 DOLUNDEFINED	1440
4.49.2.586 DOLNUMBER	1440
4.49.2.587 DOLARGUMENT	1441
4.49.2.588 DOLSUBTERM	1441
4.49.2.589 DOLTERMS	1441
4.49.2.590 DOLWILDARGS	1441
4.49.2.591 DOLINDEX	1441
4.49.2.592 DOLZERO	1441
4.49.2.593 FINDLOOP	1441
4.49.2.594 REPLACELOOP	1441
4.49.2.595 NOFUNPOWERS	1442
4.49.2.596 COMFUNPOWERS	1442
4.49.2.597 ALLFUNPOWERS	1442
4.49.2.598 PROPERORDERFLAG	1442
4.49.2.599 REGULAR	1442
4.49.2.600 FINISH	1442
4.49.2.601 POLYADD	1442
4.49.2.602 POLYSUB	1442
4.49.2.603 POLYMUL	1443
4.49.2.604 POLYDIV	1443
4.49.2.605 POLYREM	1443
4.49.2.606 POLYGCD	1443
4.49.2.607 POLYINTFAC	1443
4.49.2.608 POLYNORM	1443
4.49.2.609 MODNONE	1443
4.49.2.610 MODSUM	1443
4.49.2.611 MODMAX	1444
4.49.2.612 MODMIN	1444
4.49.2.613 MODLOCAL	1444
4.49.2.614 ELEMENTUSED	1444
4.49.2.615 ELEMENTLOADED	1444
4.49.2.616 POSNEG	1444
4.49.2.617 INVERSE TABLE	1444
4.49.2.618 COEFFICIENTSONLY	1444
4.49.2.619 ALSOPOWERS	1445

4.49.2.620 ALSOFUNARGS	1445
4.49.2.621 ALSODOLLARS	1445
4.49.2.622 NOINVERSES	1445
4.49.2.623 POSITIVEONLY	1445
4.49.2.624 UNPACK	1445
4.49.2.625 NOUNPACK	1445
4.49.2.626 FROMFUNCTION	1445
4.49.2.627 VARNAMES	1446
4.49.2.628 AUTONAMES	1446
4.49.2.629 EXPRNAMES	1446
4.49.2.630 DOLLARNAMES	1446
4.49.2.631 DUMMYBUFFER	1446
4.49.2.632 ALLARGS	1446
4.49.2.633 NUMARG	1446
4.49.2.634 ARGRANGE	1446
4.49.2.635 MAKEARGS	1447
4.49.2.636 MAXRANGEINDICATOR	1447
4.49.2.637 REPLACEARG	1447
4.49.2.638 ENCODEARG	1447
4.49.2.639 DECODEARG	1447
4.49.2.640 IMplodeARG	1447
4.49.2.641 EXPLODEARG	1447
4.49.2.642 PERMUTEARG	1447
4.49.2.643 REVERSEARG	1448
4.49.2.644 CYCLEARG	1448
4.49.2.645 ISLYNDON	1448
4.49.2.646 ISLYNDONR	1448
4.49.2.647 TOLYNDON	1448
4.49.2.648 TOLYNDONR	1448
4.49.2.649 ADDARG	1448
4.49.2.650 MULTIPLYARG	1448
4.49.2.651 DROPARG	1449
4.49.2.652 SELECTARG	1449
4.49.2.653 DEDUPARG	1449
4.49.2.654 ZTOHARG	1449
4.49.2.655 HTOZARG	1449
4.49.2.656 BASECODE	1449
4.49.2.657 YESLYNDON	1449
4.49.2.658 NOLYNDON	1449
4.49.2.659 TOPOLYNOMIALFLAG	1450
4.49.2.660 FACTARGFLAG	1450
4.49.2.661 OLDFACTARG	1450

4.49.2.662 NEWFACTARG	1450
4.49.2.663 FROMMODULEOPTION	1450
4.49.2.664 FROMPOINTINSTRUCTION	1450
4.49.2.665 EXTRASYMBOL	1450
4.49.2.666 REGULARSYMBOL	1450
4.49.2.667 EXPRESSIONNUMBER	1451
4.49.2.668 O_NONE	1451
4.49.2.669 O_CSE	1451
4.49.2.670 O_CSEGREEDY	1451
4.49.2.671 O_GREEDY	1451
4.49.2.672 O_OCCURRENCE	1451
4.49.2.673 O_MCTS	1451
4.49.2.674 O_SIMULATED_ANNEALING	1451
4.49.2.675 O_FORWARD	1452
4.49.2.676 O_BACKWARD	1452
4.49.2.677 O_FORWARDORBACKWARD	1452
4.49.2.678 O_FORWARDANDBACKWARD	1452
4.49.2.679 OPTHEAD	1452
4.49.2.680 DOALL	1452
4.49.2.681 ONLYFUNCTIONS	1452
4.49.2.682 INUSE	1452
4.49.2.683 COULDCOMMUTE	1453
4.49.2.684 DOESNOTCOMMUTE	1453
4.49.2.685 DICT_NONUMBERS	1453
4.49.2.686 DICT_INTEGERONLY	1453
4.49.2.687 DICT_RATIONALONLY	1453
4.49.2.688 DICT_ALLNUMBERS	1453
4.49.2.689 DICT_NOVARIABLES	1453
4.49.2.690 DICT_DOVARIABLES	1453
4.49.2.691 DICT_NOSPECIALS	1454
4.49.2.692 DICT_DOSPECIALS	1454
4.49.2.693 DICT_NOFUNWITHARGS	1454
4.49.2.694 DICT_DOFUNWITHARGS	1454
4.49.2.695 DICT_NOTINDOLLARS	1454
4.49.2.696 DICT_INDOLLARS	1454
4.49.2.697 DICT_INTEGERNUMBER	1454
4.49.2.698 DICT_RATIONALNUMBER	1454
4.49.2.699 DICT_SYMBOL	1455
4.49.2.700 DICT_VECTOR	1455
4.49.2.701 DICT_INDEX	1455
4.49.2.702 DICT_FUNCTION	1455
4.49.2.703 DICT_FUNCTION_WITH_ARGUMENTS	1455

4.49.2.704	DICT_SPECIALCHARACTER	1455
4.49.2.705	DICT_RANGE	1455
4.49.2.706	READSPECTATORFLAG	1455
4.49.2.707	GLOBALSPECTATORFLAG	1456
4.49.2.708	ORDEREDSET	1456
4.49.2.709	DENSETABLE	1456
4.49.2.710	SPARSETABLE	1456
4.49.2.711	ONEPARTICLEIRREDUCIBLE	1456
4.49.2.712	WITHINSERTIONS	1456
4.49.2.713	NOTADPOLES	1456
4.49.2.714	WITHSYMMETRIZE	1456
4.49.2.715	TOPOLOGIESONLY	1457
4.49.2.716	NONODES	1457
4.49.2.717	WITHEDGES	1457
4.49.2.718	CHECKEXTERN	1457
4.49.2.719	WITHBLOCKS	1457
4.49.2.720	WITHONEPI	1457
4.49.2.721	NOSNAILS	1457
4.49.2.722	NOEXTSELF	1457
4.49.3	Typedef Documentation	1458
4.49.3.1	PVFUNWP	1458
4.49.3.2	PVFUNV	1458
4.49.3.3	CFUN	1458
4.49.3.4	TFUN	1458
4.49.3.5	TFUN1	1458
4.50	ftypes.h	1459
4.51	function.c File Reference	1471
4.51.1	Detailed Description	1472
4.51.2	Function Documentation	1472
4.51.2.1	MakeDirty()	1472
4.51.2.2	MarkDirty()	1472
4.51.2.3	PolyFunDirty()	1472
4.51.2.4	PolyFunClean()	1472
4.51.2.5	Symmetrize()	1472
4.51.2.6	CompGroup()	1473
4.51.2.7	FullSymmetrize()	1473
4.51.2.8	SymGen()	1473
4.51.2.9	SymFind()	1473
4.51.2.10	ChainIn()	1473
4.51.2.11	ChainOut()	1473
4.51.2.12	MatchFunction()	1474
4.51.2.13	ScanFunctions()	1474

4.52 function.c	1474
4.53 fwin.h File Reference	1497
4.53.1 Detailed Description	1497
4.53.2 Macro Definition Documentation	1497
4.53.2.1 LINEFEED	1497
4.53.2.2 CARRIAGEReturn	1497
4.53.2.3 WITHRETURN	1497
4.53.2.4 WITHSYSTEM	1497
4.53.2.5 P_term	1498
4.53.2.6 SEPARATOR	1498
4.53.2.7 ALTSEPARATOR	1498
4.53.2.8 PATHSEPARATOR	1498
4.53.2.9 WITH_ENV	1498
4.54 fwin.h	1498
4.55 gentopo.cc	1499
4.56 gentopo.h	1517
4.57 grcc.cc	1519
4.58 grcc.h	1638
4.59 grccparam.h	1653
4.60 if.c File Reference	1656
4.60.1 Detailed Description	1656
4.60.2 Function Documentation	1657
4.60.2.1 GetIfDollarNum()	1657
4.60.2.2 FindVar()	1657
4.60.2.3 DoIfStatement()	1657
4.60.2.4 HowMany()	1657
4.60.2.5 DoubleIfBuffers()	1657
4.60.2.6 DoSwitch()	1657
4.60.2.7 DoEndSwitch()	1658
4.60.2.8 FindCase()	1658
4.60.2.9 DoubleSwitchBuffers()	1658
4.60.2.10 SwitchSplitMergeRec()	1658
4.60.2.11 SwitchSplitMerge()	1658
4.61 if.c	1659
4.62 index.c File Reference	1673
4.62.1 Detailed Description	1673
4.62.2 Function Documentation	1673
4.62.2.1 FindBracket()	1673
4.62.2.2 PutBracketInIndex()	1674
4.62.2.3 ClearBracketIndex()	1674
4.62.2.4 OpenBracketIndex()	1674
4.62.2.5 PutInside()	1674

4.63 index.c	1674
4.64 inivar.h File Reference	1683
4.64.1 Detailed Description	1683
4.64.2 Variable Documentation	1683
4.64.2.1 FG	1683
4.64.2.2 A	1683
4.64.2.3 fixedsets	1684
4.64.2.4 BufferForOutput	1684
4.64.2.5 setupfilename	1684
4.65 inivar.h	1684
4.66 lus.c File Reference	1688
4.66.1 Detailed Description	1688
4.66.2 Function Documentation	1688
4.66.2.1 Lus()	1688
4.66.2.2 FindLus()	1689
4.66.2.3 SortTheList()	1689
4.66.2.4 AllLoops()	1689
4.66.2.5 StartLoops()	1689
4.66.2.6 GenLoops()	1689
4.66.2.7 LoopOutput()	1690
4.66.2.8 AllPaths()	1690
4.66.2.9 GenPaths()	1690
4.66.2.10 PathOutput()	1690
4.66.2.11 AllOnePI()	1690
4.66.2.12 RemoveBridges()	1691
4.66.2.13 TakeOneLine()	1691
4.66.2.14 OutputOnePI()	1691
4.67 lus.c	1691
4.68 mallocprotect.h	1710
4.69 message.c File Reference	1714
4.69.1 Detailed Description	1715
4.69.2 Function Documentation	1715
4.69.2.1 Error0()	1715
4.69.2.2 Error1()	1715
4.69.2.3 Error2()	1715
4.69.2.4 MesWork()	1716
4.69.2.5 MesPrint()	1716
4.69.2.6 Warning()	1716
4.69.2.7 HighWarning()	1716
4.69.2.8 MesCall()	1716
4.69.2.9 MesCerr()	1716
4.69.2.10 MesComp()	1716

4.69.2.11 PrintTerm()	1717
4.69.2.12 PrintTermC()	1717
4.69.2.13 PrintSubTerm()	1717
4.69.2.14 PrintWords()	1717
4.69.2.15 PrintSeq()	1717
4.70 message.c	1718
4.71 minos.c File Reference	1729
4.71.1 Detailed Description	1730
4.71.2 Macro Definition Documentation	1730
4.71.2.1 CFD	1730
4.71.2.2 CTD	1730
4.71.3 Function Documentation	1731
4.71.3.1 minosread()	1731
4.71.3.2 minoswrite()	1731
4.71.3.3 str_dup()	1731
4.71.3.4 convertblock()	1731
4.71.3.5 convertnamesblock()	1731
4.71.3.6 convertiniinfo()	1732
4.71.3.7 LocateBase()	1732
4.71.3.8 ReadIndex()	1732
4.71.3.9 WriteIndexBlock()	1732
4.71.3.10 WriteNamesBlock()	1732
4.71.3.11 WriteIndex()	1732
4.71.3.12 WriteIniInfo()	1733
4.71.3.13 ReadIniInfo()	1733
4.71.3.14 GetDbase()	1733
4.71.3.15 NewDbase()	1733
4.71.3.16 FreeTableBase()	1733
4.71.3.17 ComposeTableNames()	1733
4.71.3.18 OpenDbase()	1733
4.71.3.19 AddTableName()	1734
4.71.3.20 GetTableName()	1734
4.71.3.21 PutTableNames()	1734
4.71.3.22 AddToIndex()	1734
4.71.3.23 AddObject()	1734
4.71.3.24 FindTableNumber()	1734
4.71.3.25 WriteObject()	1735
4.71.3.26 ReadObject()	1735
4.71.3.27 ReadijObject()	1735
4.71.3.28 ExistsObject()	1735
4.71.3.29 DeleteObject()	1735
4.71.4 Variable Documentation	1736

4.71.4.1 withoutflush	1736
4.72 minos.c	1736
4.73 minos.h File Reference	1753
4.73.1 Detailed Description	1754
4.73.2 Macro Definition Documentation	1754
4.73.2.1 MAXBASES	1754
4.73.2.2 NUMOBJECTS	1755
4.73.2.3 MAXINDEXSIZE	1755
4.73.2.4 NAMETABLESIZE	1755
4.73.2.5 ELEMENTSIZE	1755
4.73.2.6 TODISK	1755
4.73.2.7 FROMDISK	1755
4.73.2.8 MDIRTYFLAG	1755
4.73.2.9 MCLEANFLAG	1755
4.73.2.10 INANDOUT	1756
4.73.2.11 INPUTONLY	1756
4.73.2.12 OUTPUTONLY	1756
4.73.2.13 NOCOMPRESS	1756
4.73.3 Function Documentation	1756
4.73.3.1 minosread()	1756
4.73.3.2 minoswrite()	1756
4.73.4 Variable Documentation	1757
4.73.4.1 withoutflush	1757
4.74 minos.h	1757
4.75 model.c	1758
4.76 module.c File Reference	1764
4.76.1 Detailed Description	1764
4.76.2 Function Documentation	1765
4.76.2.1 ModuleInstruction()	1765
4.76.2.2 CoModuleOption()	1765
4.76.2.3 CoModOption()	1765
4.76.2.4 SetSpecialMode()	1765
4.76.2.5 ExecModule()	1765
4.76.2.6 ExecStore()	1765
4.76.2.7 FullCleanUp()	1766
4.76.2.8 DoPolyfun()	1766
4.76.2.9 DoPolyratfun()	1766
4.76.2.10 DoNoParallel()	1766
4.76.2.11 DoParallel()	1766
4.76.2.12 DoModSum()	1766
4.76.2.13 DoModMax()	1766
4.76.2.14 DoModMin()	1767

4.76.2.15 DoModLocal()	1767
4.76.2.16 DoProcessBucket()	1767
4.76.2.17 DoModDollar()	1767
4.76.2.18 DoinParallel()	1767
4.76.2.19 DonotinParallel()	1767
4.76.2.20 DoExecStatement()	1767
4.76.2.21 DoPipeStatement()	1768
4.77 module.c	1768
4.78 mpi.c File Reference	1777
4.78.1 Detailed Description	1778
4.78.2 Macro Definition Documentation	1779
4.78.2.1 PF_PACKSIZE	1779
4.78.2.2 MPI_ERRCODE_CHECK	1779
4.78.3 Function Documentation	1779
4.78.3.1 PF_RealTime()	1779
4.78.3.2 PF_LibInit()	1780
4.78.3.3 PF_LibTerminate()	1780
4.78.3.4 PF_Probe()	1780
4.78.3.5 PF_ISendSbuf()	1781
4.78.3.6 PF_RecvWbuf()	1781
4.78.3.7 PF_IRecvRbuf()	1782
4.78.3.8 PF_WaitRbuf()	1782
4.78.3.9 PF_Bcast()	1783
4.78.3.10 PF_RawSend()	1783
4.78.3.11 PF_RawRecv()	1783
4.78.3.12 PF_RawProbe()	1784
4.78.3.13 PF_PrintPackBuf()	1784
4.78.3.14 PF_PreparePack()	1785
4.78.3.15 PF_Pack()	1785
4.78.3.16 PF_Unpack()	1786
4.78.3.17 PF_PackString()	1786
4.78.3.18 PF_UnpackString()	1787
4.78.3.19 PF_Send()	1787
4.78.3.20 PF_Receive()	1788
4.78.3.21 PF_Broadcast()	1788
4.78.3.22 PF_PrepareLongSinglePack()	1789
4.78.3.23 PF_LongSinglePack()	1789
4.78.3.24 PF_LongSingleUnpack()	1789
4.78.3.25 PF_LongSingleSend()	1790
4.78.3.26 PF_LongSingleReceive()	1790
4.78.3.27 PF_PrepareLongMultiPack()	1791
4.78.3.28 PF_LongMultiPackImpl()	1791

4.78.3.29 PF_LongMultiUnpackImpl()	1792
4.78.3.30 PF_LongMultiBroadcast()	1793
4.78.4 Variable Documentation	1793
4.78.4.1 PF_maxDollarChunkSize	1793
4.79 mpi.c	1794
4.80 mpidbg.h File Reference	1811
4.80.1 Detailed Description	1812
4.80.2 Macro Definition Documentation	1813
4.80.2.1 MPIDBG_RANK	1813
4.80.2.2 MPIDBG_Out	1813
4.80.2.3 MPIDBG_EXTARG	1813
4.80.2.4 MPI_Send	1813
4.80.2.5 MPI_Recv	1813
4.80.2.6 MPI_Bsend	1813
4.80.2.7 MPI_Ssend	1813
4.80.2.8 MPI_Rsend	1814
4.80.2.9 MPI_Isend	1814
4.80.2.10 MPI_Ibsend	1814
4.80.2.11 MPI_Issend	1814
4.80.2.12 MPI_Irsend	1814
4.80.2.13 MPI_Irecv	1814
4.80.2.14 MPI_Wait	1814
4.80.2.15 MPI_Test	1815
4.80.2.16 MPI_Waitany	1815
4.80.2.17 MPI_Testany	1815
4.80.2.18 MPI_Waitall	1815
4.80.2.19 MPI_Testall	1815
4.80.2.20 MPI_Waitsome	1815
4.80.2.21 MPI_Testsome	1815
4.80.2.22 MPI_Iprobe	1816
4.80.2.23 MPI_Probe	1816
4.80.2.24 MPI_Cancel	1816
4.80.2.25 MPI_Test_cancelled	1816
4.80.2.26 MPI_Barrier	1816
4.80.2.27 MPI_Bcast	1816
4.81 mpidbg.h	1817
4.82 mytime.cc	1825
4.83 mytime.h	1825
4.84 names.c File Reference	1826
4.84.1 Detailed Description	1827
4.84.2 Function Documentation	1827
4.84.2.1 GetNode()	1827

4.84.2.2 AddName()	1827
4.84.2.3 GetName()	1828
4.84.2.4 GetFunction()	1828
4.84.2.5 GetNumber()	1828
4.84.2.6 GetLastExprName()	1828
4.84.2.7 GetOName()	1828
4.84.2.8 GetAutoName()	1828
4.84.2.9 GetVar()	1829
4.84.2.10 EntVar()	1829
4.84.2.11 GetDollar()	1829
4.84.2.12 DumpTree()	1829
4.84.2.13 DumpNode()	1829
4.84.2.14 CompactifyTree()	1830
4.84.2.15 CopyTree()	1830
4.84.2.16 LinkTree()	1830
4.84.2.17 MakeNameTree()	1830
4.84.2.18 FreeNameTree()	1830
4.84.2.19 ClearWildcardNames()	1830
4.84.2.20 AddWildcardName()	1831
4.84.2.21 GetWildcardName()	1831
4.84.2.22 AddSymbol()	1831
4.84.2.23 CoSymbol()	1831
4.84.2.24 AddIndex()	1831
4.84.2.25 CoIndex()	1831
4.84.2.26 DoDimension()	1832
4.84.2.27 CoDimension()	1832
4.84.2.28 AddVector()	1832
4.84.2.29 CoVector()	1832
4.84.2.30 AddFunction()	1832
4.84.2.31 CoCommutelnSet()	1832
4.84.2.32 CoFunction()	1833
4.84.2.33 CoNFunction()	1833
4.84.2.34 CoCFunction()	1833
4.84.2.35 CoNTensor()	1833
4.84.2.36 CoCTensor()	1833
4.84.2.37 DoTable()	1833
4.84.2.38 CoTable()	1834
4.84.2.39 CoNTable()	1834
4.84.2.40 CoCTable()	1834
4.84.2.41 EmptyTable()	1834
4.84.2.42 AddSet()	1834
4.84.2.43 DoElements()	1834

4.84.2.44 CoSet()	1835
4.84.2.45 DoTempSet()	1835
4.84.2.46 CoAuto()	1835
4.84.2.47 AddDollar()	1835
4.84.2.48 ReplaceDollar()	1835
4.84.2.49 AddDubious()	1835
4.84.2.50 MakeDubious()	1836
4.84.2.51 NameConflict()	1836
4.84.2.52 AddExpression()	1836
4.84.2.53 GetLabel()	1836
4.84.2.54 ResetVariables()	1836
4.84.2.55 RemoveDollars()	1836
4.84.2.56 Globalize()	1837
4.84.2.57 TestName()	1837
4.85 names.c	1837
4.86 normal.c File Reference	1875
4.86.1 Detailed Description	1875
4.86.2 Macro Definition Documentation	1876
4.86.2.1 MAXNUMBEROFNONCOMTERMS	1876
4.86.3 Function Documentation	1876
4.86.3.1 CompareFunctions()	1876
4.86.3.2 Commute()	1876
4.86.3.3 Normalize()	1876
4.86.3.4 ExtraSymbol()	1876
4.86.3.5 DoTheta()	1876
4.86.3.6 DoDelta()	1877
4.86.3.7 DoRevert()	1877
4.86.3.8 DetCommu()	1877
4.86.3.9 DoesCommu()	1877
4.86.3.10 TreatPolyRatFun()	1877
4.86.3.11 DropCoefficient()	1877
4.86.3.12 DropSymbols()	1877
4.86.3.13 SymbolNormalize()	1878
4.86.3.14 TestFunFlag()	1878
4.86.3.15 BracketNormalize()	1878
4.87 normal.c	1878
4.88 notation.c File Reference	1941
4.88.1 Detailed Description	1941
4.88.2 Function Documentation	1941
4.88.2.1 NormPolyTerm()	1941
4.88.2.2 ConvertToPoly()	1942
4.88.2.3 LocalConvertToPoly()	1942

4.88.2.4 ConvertFromPoly()	1942
4.88.2.5 FindSubterm()	1942
4.88.2.6 FindLocalSubterm()	1943
4.88.2.7 PrintSubtermList()	1943
4.88.2.8 PrintExtraSymbol()	1943
4.88.2.9 FindSubexpression()	1943
4.88.2.10 ExtraSymFun()	1943
4.88.2.11 PruneExtraSymbols()	1943
4.89 notation.c	1944
4.90 opera.c File Reference	1957
4.90.1 Detailed Description	1958
4.90.2 Function Documentation	1958
4.90.2.1 EpfFind()	1958
4.90.2.2 EpfCon()	1958
4.90.2.3 EpfGen()	1959
4.90.2.4 Trick()	1959
4.90.2.5 Trace4no()	1959
4.90.2.6 Trace4()	1959
4.90.2.7 Trace4Gen()	1959
4.90.2.8 TraceNno()	1960
4.90.2.9 TraceN()	1960
4.90.2.10 TraceNgen()	1960
4.90.2.11 Traces()	1960
4.90.2.12 TraceFind()	1960
4.90.2.13 Chisholm()	1960
4.90.2.14 TenVecFind()	1961
4.90.2.15 TenVec()	1961
4.91 opera.c	1961
4.92 optimize.cc File Reference	1990
4.92.1 Detailed Description	1992
4.92.2 Function Documentation	1992
4.92.2.1 print_instr()	1992
4.92.2.2 my_random_shuffle()	1992
4.92.3 Description	1992
4.92.3.1 get_expression()	1992
4.92.4 Description	1993
4.92.4.1 get_brackets()	1993
4.92.5 Description	1993
4.92.5.1 count_operators() [1/2]	1994
4.92.6 Description	1994
4.92.6.1 count_operators() [2/2]	1994
4.92.7 Description	1994

4.92.7.1 occurrence_order()	1994
4.92.8 Description	1994
4.92.8.1 getpower()	1995
4.92.9 Description	1995
4.92.10 Note	1995
4.92.11 Description	1995
4.92.11.1 fixarg()	1995
4.92.12 Description	1996
4.92.12.1 GcdLong_fix_args()	1996
4.92.12.2 DivLong_fix_args()	1996
4.92.12.3 build_Horner_tree()	1996
4.92.13 Description	1997
4.92.13.1 term_compare()	1997
4.92.14 Description	1997
4.92.14.1 Horner_tree()	1998
4.92.15 Description	1998
4.92.15.1 print_tree()	1998
4.92.15.2 hash_range()	1998
4.92.15.3 generate_instructions()	1999
4.92.16 Description	1999
4.92.17 Implementation details	1999
4.92.17.1 count_operators_cse()	2000
4.92.17.2 buildTree()	2000
4.92.17.3 count_operators_cse_topdown()	2000
4.92.17.4 simulated_annealing()	2000
4.92.17.5 find_Horner_MCTS_expand_tree()	2000
4.92.17.6 find_Horner_MCTS()	2000
4.92.18 Description	2001
4.92.18.1 merge_operators()	2001
4.92.19 Description	2001
4.92.20 Implementation details	2002
4.92.20.1 find_optimizations()	2002
4.92.21 Description	2002
4.92.21.1 do_optimization()	2002
4.92.22 Description	2003
4.92.22.1 partial_factorize()	2003
4.92.23 Description	2003
4.92.23.1 optimize_greedy()	2003
4.92.24 Description	2004
4.92.24.1 recycle_variables()	2004
4.92.25 Description	2004
4.92.26 Implementation details	2005

4.92.26.1 optimize_expression_given_Horner()	2005
4.92.27 Description	2005
4.92.27.1 generate_output()	2006
4.92.28 Description	2006
4.92.28.1 generate_expression()	2006
4.92.29 Description	2006
4.92.29.1 optimize_print_code()	2007
4.92.30 Description	2007
4.92.30.1 Optimize()	2007
4.92.31 Description	2008
4.92.31.1 ClearOptimize()	2009
4.92.32 Description	2009
4.92.33 Variable Documentation	2010
4.92.33.1 OPER_ADD	2010
4.92.33.2 OPER_MUL	2010
4.92.33.3 OPER_COMMA	2010
4.92.33.4 optimize_expr	2010
4.92.33.5 optimize_best_Horner_schemes	2010
4.92.33.6 optimize_num_vars	2011
4.92.33.7 optimize_best_num_oper	2011
4.92.33.8 optimize_best_instr	2011
4.92.33.9 optimize_best_vars	2011
4.92.33.10 mcts_factorized	2011
4.92.33.11 mcts_separated	2011
4.92.33.12 mcts_vars	2011
4.92.33.13 mcts_root	2011
4.92.33.14 mcts_expr_score	2012
4.92.33.15 mcts_best_schemes	2012
4.93 optimize.cc	2012
4.94 parallel.c File Reference	2064
4.94.1 Detailed Description	2066
4.94.2 Macro Definition Documentation	2066
4.94.2.1 PRINTFBUF	2066
4.94.2.2 SWAP	2067
4.94.2.3 PACK_LONG	2067
4.94.2.4 UNPACK_LONG	2067
4.94.2.5 CHECK	2067
4.94.2.6 _CHECK	2068
4.94.2.7 __CHECK	2068
4.94.2.8 DBGOUT	2068
4.94.2.9 DBGOUT_NINTERMS	2068
4.94.2.10 PF_STATS_SIZE	2068

4.94.2.11 PF_SNDFILEBUFSIZE	2069
4.94.2.12 recvBuffer	2069
4.94.3 Typedef Documentation	2069
4.94.3.1 NODE	2069
4.94.4 Function Documentation	2069
4.94.4.1 PF_RealTime()	2069
4.94.4.2 PF_LibInit()	2069
4.94.4.3 PF_LibTerminate()	2070
4.94.4.4 PF_Probe()	2070
4.94.4.5 PF_RecvWbuf()	2071
4.94.4.6 PF_IRecvRbuf()	2071
4.94.4.7 PF_WaitRbuf()	2071
4.94.4.8 PF_RawSend()	2072
4.94.4.9 PF_RawRecv()	2072
4.94.4.10 PF_RawProbe()	2073
4.94.4.11 PF_EndSort()	2073
4.94.4.12 PF_Deferred()	2074
4.94.4.13 PF_Processor()	2075
4.94.4.14 PF_Init()	2076
4.94.4.15 PF_Terminate()	2077
4.94.4.16 PF_GetSlaveTimes()	2078
4.94.4.17 PF_BroadcastNumber()	2078
4.94.4.18 PF_BroadcastBuffer()	2079
4.94.4.19 PF_BroadcastString()	2080
4.94.4.20 PF_BroadcastPreDollar()	2080
4.94.4.21 PF_CollectModifiedDollars()	2081
4.94.4.22 PF_BroadcastModifiedDollars()	2082
4.94.4.23 PF_BroadcastRedefinedPreVars()	2083
4.94.4.24 PF_StoreInsideInfo()	2084
4.94.4.25 PF_RestoreInsideInfo()	2084
4.94.4.26 PF_BroadcastCBuf()	2084
4.94.4.27 PF_BroadcastExpFlags()	2085
4.94.4.28 PF_BroadcastExpr()	2085
4.94.4.29 PF_BroadcastRHS()	2086
4.94.4.30 PF_InParallelProcessor()	2086
4.94.4.31 PF_SendFile()	2086
4.94.4.32 PF_RecvFile()	2087
4.94.4.33 PF_MLock()	2088
4.94.4.34 PF_MUnlock()	2088
4.94.4.35 PF_WriteFileToFile()	2088
4.94.4.36 PF_FlushStdOutBuffer()	2089
4.94.4.37 PF_FreeErrorMessageBuffers()	2089

4.94.5 Variable Documentation	2089
4.94.5.1 PF	2089
4.95 parallel.c	2090
4.96 parallel.h File Reference	2138
4.96.1 Detailed Description	2140
4.96.2 Macro Definition Documentation	2140
4.96.2.1 MASTER	2140
4.96.2.2 PF_RESET	2140
4.96.2.3 PF_TIME	2141
4.96.2.4 PF_TERM_MSGTAG	2141
4.96.2.5 PF_ENDSORT_MSGTAG	2141
4.96.2.6 PF_DOLLAR_MSGTAG	2141
4.96.2.7 PF_BUFFER_MSGTAG	2141
4.96.2.8 PF_ENDBUFFER_MSGTAG	2141
4.96.2.9 PF_READY_MSGTAG	2141
4.96.2.10 PF_DATA_MSGTAG	2142
4.96.2.11 PF_EMPTY_MSGTAG	2142
4.96.2.12 PF_STDOUT_MSGTAG	2142
4.96.2.13 PF_LOG_MSGTAG	2142
4.96.2.14 PF_OPT_MCTS_MSGTAG	2142
4.96.2.15 PF_OPT_HORNER_MSGTAG	2142
4.96.2.16 PF_OPT_COLLECT_MSGTAG	2142
4.96.2.17 PF_MISC_MSGTAG	2142
4.96.2.18 GNUC_PREREQ	2143
4.96.2.19 OMPI_SKIP_MPICXX	2143
4.96.2.20 indices	2143
4.96.2.21 PF_ANY_SOURCE	2143
4.96.2.22 PF_ANY_MSGTAG	2143
4.96.2.23 PF_COMM	2143
4.96.2.24 PF_BYTE	2143
4.96.2.25 PF_INT	2144
4.96.2.26 PF_LongMultiPack	2144
4.96.2.27 PF_LongMultiUnpack	2144
4.96.3 Function Documentation	2144
4.96.3.1 PF_ISendSbuf()	2144
4.96.3.2 PF_Bcast()	2145
4.96.3.3 PF_RawSend()	2145
4.96.3.4 PF_RawRecv()	2145
4.96.3.5 PF_PreparePack()	2146
4.96.3.6 PF_Pack()	2146
4.96.3.7 PF_Unpack()	2147
4.96.3.8 PF_PackString()	2147

4.96.3.9 PF_UnpackString()	2148
4.96.3.10 PF_Send()	2148
4.96.3.11 PF_Receive()	2149
4.96.3.12 PF_Broadcast()	2149
4.96.3.13 PF_PrepareLongSinglePack()	2150
4.96.3.14 PF_LongSinglePack()	2150
4.96.3.15 PF_LongSingleUnpack()	2150
4.96.3.16 PF_LongSingleSend()	2151
4.96.3.17 PF_LongSingleReceive()	2151
4.96.3.18 PF_PrepareLongMultiPack()	2152
4.96.3.19 PF_LongMultiPackImpl()	2152
4.96.3.20 PF_LongMultiUnpackImpl()	2153
4.96.3.21 PF_LongMultiBroadcast()	2154
4.96.3.22 PF_EndSort()	2154
4.96.3.23 PF_Deferred()	2155
4.96.3.24 PF_Processor()	2156
4.96.3.25 PF_Init()	2157
4.96.3.26 PF_Terminate()	2158
4.96.3.27 PF_GetSlaveTimes()	2159
4.96.3.28 PF_BroadcastNumber()	2159
4.96.3.29 PF_BroadcastBuffer()	2160
4.96.3.30 PF_BroadcastString()	2161
4.96.3.31 PF_BroadcastPreDollar()	2161
4.96.3.32 PF_CollectModifiedDollars()	2162
4.96.3.33 PF_BroadcastModifiedDollars()	2163
4.96.3.34 PF_BroadcastRedefinedPreVars()	2164
4.96.3.35 PF_BroadcastCBuf()	2165
4.96.3.36 PF_BroadcastExpFlags()	2165
4.96.3.37 PF_StoreInsideInfo()	2166
4.96.3.38 PF_RestoreInsideInfo()	2166
4.96.3.39 PF_BroadcastExpr()	2166
4.96.3.40 PF_BroadcastRHS()	2167
4.96.3.41 PF_InParallelProcessor()	2167
4.96.3.42 PF_SendFile()	2167
4.96.3.43 PF_RecvFile()	2168
4.96.3.44 PF_MLock()	2169
4.96.3.45 PF_MUnlock()	2169
4.96.3.46 PF_WriteFileToFile()	2169
4.96.3.47 PF_FlushStdOutBuffer()	2170
4.96.4 Variable Documentation	2170
4.96.4.1 PF	2170
4.96.4.2 PF_maxDollarChunkSize	2170

4.97 parallel.h	2171
4.98 pattern.c File Reference	2174
4.98.1 Detailed Description	2174
4.98.2 Macro Definition Documentation	2175
4.98.2.1 PutInBuffers	2175
4.98.3 Function Documentation	2175
4.98.3.1 TestMatch()	2175
4.98.3.2 Substitute()	2176
4.98.3.3 FindAll()	2176
4.98.3.4 TestSelect()	2176
4.98.3.5 SubsInAll()	2177
4.98.3.6 TransferBuffer()	2177
4.98.3.7 TakeIDfunction()	2177
4.99 pattern.c	2177
4.100 poly.cc	2204
4.101 poly.h	2237
4.102 polyfact.cc File Reference	2240
4.102.1 Detailed Description	2241
4.102.2 Function Documentation	2241
4.102.2.1 operator<<() [1/2]	2241
4.102.2.2 operator<<() [2/2]	2241
4.103 polyfact.cc	2242
4.104 polyfact.h	2257
4.105 polygcd.cc File Reference	2259
4.105.1 Detailed Description	2259
4.105.2 Function Documentation	2259
4.105.2.1 gcd_heuristic_possible()	2259
4.105.3 Description	2260
4.105.4 Notes	2260
4.105.4.1 gcd_linear_helper()	2260
4.106 polygcd.cc	2260
4.107 polygcd.h	2276
4.108 polywrap.cc File Reference	2278
4.108.1 Detailed Description	2279
4.108.2 Function Documentation	2279
4.108.2.1 poly_determine_modulus()	2279
4.108.3 Description	2279
4.108.4 Notes	2279
4.108.4.1 poly_gcd()	2279
4.108.5 Description	2279
4.108.6 Notes	2280
4.108.6.1 poly_divmod()	2280

4.108.6.2 poly_div()	2280
4.108.6.3 poly_rem()	2280
4.108.6.4 poly_ratfun_read()	2281
4.108.7 Description	2281
4.108.8 Notes	2281
4.108.8.1 poly_sort()	2281
4.108.9 Description	2281
4.108.10 Notes	2282
4.108.10.1 poly_ratfun_add()	2282
4.108.11 Description	2282
4.108.12 Notes	2283
4.108.12.1 poly_ratfun_normalize()	2283
4.108.13 Description	2283
4.108.14 Notes	2284
4.108.14.1 poly_fix_minus_signs()	2284
4.108.14.2 poly_factorize()	2284
4.108.15 Description	2285
4.108.16 Notes	2285
4.108.16.1 poly_factorize_argument()	2285
4.108.17 Description	2285
4.108.18 Notes	2286
4.108.18.1 poly_factorize_dollar()	2286
4.108.19 Description	2286
4.108.20 Notes	2286
4.108.20.1 poly_factorize_expression()	2287
4.108.21 Description	2287
4.108.22 Notes	2287
4.108.22.1 poly_unfactorize_expression()	2288
4.108.23 Description	2288
4.108.24 Notes	2288
4.108.24.1 poly_inverse()	2289
4.108.24.2 poly_mul()	2289
4.108.24.3 poly_free_poly_vars()	2289
4.108.25 Variable Documentation	2289
4.108.25.1 POLYWRAP_DENOMPOWER_INCREASE_FACTOR	2289
4.109 polywrap.cc	2289
4.110 portsignals.h File Reference	2311
4.110.1 Detailed Description	2312
4.110.2 Macro Definition Documentation	2312
4.110.2.1 FATAL_SIG_ERROR	2312
4.110.2.2 NSIG	2312
4.110.2.3 SIGSEGV	2312

4.110.2.4 SIGFPE	2313
4.110.2.5 SIGILL	2313
4.110.2.6 SIGEMT	2313
4.110.2.7 SIGSYS	2313
4.110.2.8 SIGPIPE	2313
4.110.2.9 SIGLOST	2313
4.110.2.10 SIGXCPU	2313
4.110.2.11 SIGXFSZ	2313
4.110.2.12 SIGTERM	2314
4.110.2.13 SIGINT	2314
4.110.2.14 SIGQUIT	2314
4.110.2.15 SIGHUP	2314
4.110.2.16 SIGALRM	2314
4.110.2.17 SIGVTALRM	2314
4.110.2.18 SIGPROF	2314
4.111 portsignals.h	2315
4.112 pre.c File Reference	2316
4.112.1 Detailed Description	2318
4.112.2 Macro Definition Documentation	2319
4.112.2.1 STRINGIFY	2319
4.112.2.2 STRINGIFY__	2319
4.112.2.3 SKIPBUFSIZE	2319
4.112.2.4 KILL	2319
4.112.2.5 KILLALL	2319
4.112.2.6 DAEMON	2319
4.112.2.7 SHELL	2319
4.112.2.8 STDERR	2320
4.112.2.9 TRUE_EXPR	2320
4.112.2.10 FALSE_EXPR	2320
4.112.2.11 NOSHELL	2320
4.112.2.12 TERMINAL	2320
4.112.3 Function Documentation	2320
4.112.3.1 GetInput()	2320
4.112.3.2 ClearPushback()	2320
4.112.3.3 GetChar()	2321
4.112.3.4 CharOut()	2321
4.112.3.5 UnsetAllowDelay()	2321
4.112.3.6 GetPreVar()	2321
4.112.3.7 PutPreVar()	2321
4.112.3.8 PopPreVars()	2322
4.112.3.9 IniModule()	2322
4.112.3.10 IniSpecialModule()	2322

4.112.3.11 PreProcessor()	2322
4.112.3.12 PreProInstruction()	2322
4.112.3.13 LoadInstruction()	2322
4.112.3.14 LoadStatement()	2323
4.112.3.15 ExpandTripleDots()	2323
4.112.3.16 FindKeyWord()	2323
4.112.3.17 FindInKeyWord()	2323
4.112.3.18 TheDefine()	2323
4.112.3.19 DoCommentChar()	2324
4.112.3.20 DoPreAssign()	2324
4.112.3.21 DoDefine()	2324
4.112.3.22 DoRedefine()	2324
4.112.3.23 ClearMacro()	2325
4.112.3.24 TheUndefine()	2325
4.112.3.25 DoUndefine()	2325
4.112.3.26 DoInclude()	2325
4.112.3.27 DoReverseInclude()	2325
4.112.3.28 Include()	2325
4.112.3.29 DoPreExchange()	2325
4.112.3.30 DoCall()	2326
4.112.3.31 DoDebug()	2326
4.112.3.32 DoTerminate()	2326
4.112.3.33 DoContinueDo()	2326
4.112.3.34 DoDo()	2326
4.112.3.35 DoBreakDo()	2326
4.112.3.36 DoElse()	2326
4.112.3.37 DoElseif()	2327
4.112.3.38 DoEnddo()	2327
4.112.3.39 DoEndif()	2327
4.112.3.40 DoEndprocedure()	2327
4.112.3.41 Dolf()	2327
4.112.3.42 Dolfdef()	2327
4.112.3.43 Dolfydef()	2327
4.112.3.44 Dolfndef()	2328
4.112.3.45 DoInside()	2328
4.112.3.46 DoEndInside()	2328
4.112.3.47 DoMessage()	2328
4.112.3.48 DoPipe()	2328
4.112.3.49 DoProcExtension()	2328
4.112.3.50 DoPreOut()	2328
4.112.3.51 DoPrePrintTimes()	2329
4.112.3.52 DoPreSortReallocate()	2329

4.112.3.53 DoPreAppend()	2329
4.112.3.54 DoPreCreate()	2329
4.112.3.55 DoPreRemove()	2329
4.112.3.56 DoPreClose()	2329
4.112.3.57 DoPreWrite()	2329
4.112.3.58 DoProcedure()	2330
4.112.3.59 DoPreBreak()	2330
4.112.3.60 DoPreCase()	2330
4.112.3.61 DoPreDefault()	2330
4.112.3.62 DoPreEndSwitch()	2330
4.112.3.63 DoPreSwitch()	2330
4.112.3.64 DoPreShow()	2330
4.112.3.65 DoSystem()	2331
4.112.3.66 PreLoad()	2331
4.112.3.67 PreSkip()	2331
4.112.3.68 StartPrepro()	2331
4.112.3.69 EvalPrelf()	2331
4.112.3.70 PrelfEval()	2331
4.112.3.71 PreCmp()	2332
4.112.3.72 PreEq()	2332
4.112.3.73 pParseObject()	2332
4.112.3.74 PreCalc()	2332
4.112.3.75 PreEval()	2332
4.112.3.76 AddToPreTypes()	2333
4.112.3.77 MessPreNesting()	2333
4.112.3.78 DoPreAddSeparator()	2333
4.112.3.79 DoPreRmSeparator()	2333
4.112.3.80 DoExternal()	2333
4.112.3.81 DoPrompt()	2333
4.112.3.82 DoSetExternal()	2333
4.112.3.83 DoSetExternalAttr()	2334
4.112.3.84 DoRmExternal()	2334
4.112.3.85 DoFromExternal()	2334
4.112.3.86 DoToExternal()	2334
4.112.3.87 defineChannel()	2334
4.112.3.88 writeToChannel()	2334
4.112.3.89 DoFactDollar()	2335
4.112.3.90 GetDollarNumber()	2335
4.112.3.91 DoSetRandom()	2335
4.112.3.92 DoOptimize()	2335
4.112.3.93 DoClearOptimize()	2335
4.112.3.94 DoSkipExtraSymbols()	2335

4.112.3.95 DoPreReset()	2335
4.112.3.96 DoPreAppendPath()	2336
4.112.3.97 DoPrePrependPath()	2336
4.112.3.98 DoTimeOutAfter()	2336
4.112.3.99 DoNamespace()	2336
4.112.3.100 DoEndNamespace()	2336
4.112.3.101 SkipName()	2336
4.112.3.102 ConstructName()	2337
4.112.3.103 DoUse()	2337
4.112.3.104 UserFlags()	2337
4.112.3.105 DoClearUserFlag()	2337
4.112.3.106 DoSetUserFlag()	2337
4.113 pre.c	2338
4.114 proces.c File Reference	2427
4.114.1 Detailed Description	2427
4.114.2 Macro Definition Documentation	2428
4.114.2.1 DONE	2428
4.114.3 Function Documentation	2428
4.114.3.1 Processor()	2428
4.114.3.2 TestSub()	2429
4.114.3.3 InFunction()	2429
4.114.3.4 InsertTerm()	2430
4.114.3.5 PasteFile()	2432
4.114.3.6 PasteTerm()	2433
4.114.3.7 FiniTerm()	2433
4.114.3.8 Generator()	2434
4.114.3.9 DoOnePow()	2435
4.114.3.10 Deferred()	2436
4.114.3.11 PrepPoly()	2437
4.114.3.12 PolyFunMul()	2438
4.114.4 Variable Documentation	2439
4.114.4.1 printscratch	2439
4.115 proces.c	2439
4.116 ratio.c File Reference	2504
4.116.1 Detailed Description	2505
4.116.2 Function Documentation	2505
4.116.2.1 RatioFind()	2505
4.116.2.2 RatioGen()	2505
4.116.2.3 BinomGen()	2506
4.116.2.4 DoSumF1()	2506
4.116.2.5 Glue()	2506
4.116.2.6 DoSumF2()	2506

4.116.2.7 GCDfunction()	2506
4.116.2.8 GCDfunction3()	2507
4.116.2.9 PutExtraSymbols()	2507
4.116.2.10 TakeExtraSymbols()	2507
4.116.2.11 MultiplyWithTerm()	2507
4.116.2.12 TakeContent()	2507
4.116.2.13 MergeSymbolLists()	2508
4.116.2.14 MergeDotproductLists()	2508
4.116.2.15 CreateExpression()	2508
4.116.2.16 GCDterms()	2508
4.116.2.17 ReadPolyRatFun()	2509
4.116.2.18 FromPolyRatFun()	2509
4.116.2.19 TakeSymbolContent()	2509
4.116.2.20 GCDclean()	2509
4.116.2.21 PolyDiv()	2510
4.116.2.22 DIVfunction()	2510
4.116.2.23 MULfunc()	2510
4.116.2.24 ConvertArgument()	2510
4.116.2.25 ExpandRat()	2510
4.116.2.26 InvPoly()	2510
4.116.3 Variable Documentation	2511
4.116.3.1 divrem	2511
4.116.3.2 TheErrorMessage	2511
4.117 ratio.c	2511
4.118 reken.c File Reference	2553
4.118.1 Detailed Description	2555
4.118.2 Macro Definition Documentation	2555
4.118.2.1 GCDMAX	2555
4.118.2.2 NEWTRICK	2555
4.118.2.3 COPYLONG	2555
4.118.2.4 WARMUP	2556
4.118.3 Function Documentation	2556
4.118.3.1 Pack()	2556
4.118.3.2 UnPack()	2556
4.118.3.3 Mully()	2556
4.118.3.4 Divvy()	2556
4.118.3.5 AddRat()	2557
4.118.3.6 MulRat()	2557
4.118.3.7 DivRat()	2557
4.118.3.8 Simplify()	2557
4.118.3.9 AccumGCD()	2557
4.118.3.10 TakeRatRoot()	2558

4.118.3.11 AddLong()	2558
4.118.3.12 AddPLon()	2558
4.118.3.13 SubPLon()	2558
4.118.3.14 MulLong()	2558
4.118.3.15 BigLong()	2559
4.118.3.16 DivLong()	2559
4.118.3.17 RaisPow()	2559
4.118.3.18 RaisPowCached()	2559
4.118.4 Description	2559
4.118.5 Notes	2560
4.118.5.1 RaisPowMod()	2560
4.118.5.2 NormalModulus()	2560
4.118.5.3 MakeInverses()	2560
4.118.5.4 GetModInverses()	2561
4.118.5.5 GetLongModInverses()	2561
4.118.5.6 Product()	2561
4.118.5.7 Quotient()	2561
4.118.5.8 Remain10()	2562
4.118.5.9 Remain4()	2562
4.118.5.10 PrtLong()	2562
4.118.5.11 GetLong()	2562
4.118.5.12 GcdLong()	2562
4.118.5.13 GetBinom()	2563
4.118.5.14 LcmLong()	2563
4.118.5.15 TakeLongRoot()	2563
4.118.5.16 MakeRational()	2563
4.118.5.17 MakeLongRational()	2563
4.118.5.18 CompCoef()	2564
4.118.5.19 Modulus()	2564
4.118.5.20 TakeModulus()	2564
4.118.5.21 TakeNormalModulus()	2564
4.118.5.22 MakeModTable()	2564
4.118.5.23 Factorial()	2565
4.118.5.24 Bernoulli()	2565
4.118.5.25 NextPrime()	2565
4.118.5.26 Moebius()	2565
4.118.5.27 iniwranf()	2565
4.118.5.28 wranf()	2566
4.118.5.29 iranf()	2566
4.118.5.30 PreRandom()	2566
4.119 reken.c	2566
4.120 reshuf.c File Reference	2612

4.120.1 Detailed Description	2612
4.120.2 Macro Definition Documentation	2613
4.120.2.1 NEWCODE	2613
4.120.3 Function Documentation	2613
4.120.3.1 ReNumber()	2613
4.120.3.2 FunLevel()	2613
4.120.3.3 DetCurDum()	2613
4.120.3.4 FullRenumber()	2613
4.120.3.5 MoveDummies()	2613
4.120.3.6 AdjustRenumScratch()	2614
4.120.3.7 CountDo()	2614
4.120.3.8 CountFun()	2614
4.120.3.9 DimensionSubterm()	2614
4.120.3.10 DimensionTerm()	2614
4.120.3.11 DimensionExpression()	2614
4.120.3.12 MultDo()	2615
4.120.3.13 TryDo()	2615
4.120.3.14 DoDistrib()	2615
4.120.3.15 EqualArg()	2615
4.120.3.16 DoDelta3()	2615
4.120.3.17 TestPartitions()	2615
4.120.3.18 DoPartitions()	2616
4.120.3.19 DoPermutations()	2616
4.120.3.20 DoShuffle()	2616
4.120.3.21 Shuffle()	2616
4.120.3.22 FinishShuffle()	2616
4.120.3.23 DoStuffle()	2616
4.120.3.24 Stuffle()	2617
4.120.3.25 FinishStuffle()	2617
4.120.3.26 StuffRootAdd()	2617
4.121 reshuf.c	2617
4.122 sch.c File Reference	2653
4.122.1 Detailed Description	2654
4.122.2 Macro Definition Documentation	2654
4.122.2.1 va_dcl	2654
4.122.2.2 va_start	2654
4.122.2.3 va_end	2655
4.122.2.4 va_arg	2655
4.122.3 Typedef Documentation	2655
4.122.3.1 va_list	2655
4.122.4 Function Documentation	2655
4.122.4.1 StrCopy()	2655

4.122.4.2 AddToLine()	2655
4.122.4.3 FiniLine()	2655
4.122.4.4 IniLine()	2656
4.122.4.5 LongToLine()	2656
4.122.4.6 RatToLine()	2656
4.122.4.7 TalToLine()	2656
4.122.4.8 TokenToLine()	2656
4.122.4.9 CodeToLine()	2656
4.122.4.10 MultiplyToLine()	2657
4.122.4.11 AddArrayIndex()	2657
4.122.4.12 PrtTerms()	2657
4.122.4.13 WrtPower()	2657
4.122.4.14 PrintTime()	2657
4.122.4.15 WriteLists()	2657
4.122.4.16 WriteDictionary()	2657
4.122.4.17 WriteArgument()	2658
4.122.4.18 WriteSubTerm()	2658
4.122.4.19 WriteInnerTerm()	2658
4.122.4.20 WriteTerm()	2658
4.122.4.21 WriteExpression()	2658
4.122.4.22 WriteAll()	2658
4.122.4.23 WriteOne()	2659
4.123 sch.c	2659
4.124 setfile.c File Reference	2692
4.124.1 Detailed Description	2693
4.124.2 Macro Definition Documentation	2693
4.124.2.1 NUMERICALVALUE	2693
4.124.2.2 STRINGVALUE	2693
4.124.2.3 PATHVALUE	2694
4.124.2.4 ONOFFVALUE	2694
4.124.2.5 DEFINEVALUE	2694
4.124.2.6 SETBUFSIZE	2694
4.124.3 Function Documentation	2694
4.124.3.1 DoSetups()	2694
4.124.3.2 ProcessOption()	2694
4.124.3.3 GetSetupPar()	2694
4.124.3.4 RecalcSetups()	2695
4.124.3.5 AllocSetups()	2695
4.124.3.6 WriteSetup()	2695
4.124.3.7 AllocSort()	2695
4.124.3.8 AllocSortFileName()	2695
4.124.3.9 AllocFileHandle()	2695

4.124.3.10 DeAllocFileHandle()	2696
4.124.3.11 MakeSetupAllocs()	2696
4.124.3.12 TryFileSetups()	2696
4.124.3.13 TryEnvironment()	2696
4.124.4 Variable Documentation	2696
4.124.4.1 curdirp	2696
4.124.4.2 cursortdirp	2696
4.124.4.3 commentchar	2696
4.124.4.4 dotchar	2697
4.124.4.5 highfirst	2697
4.124.4.6 lowfirst	2697
4.124.4.7 procedureextension	2697
4.124.4.8 setupparameters	2697
4.125 setfile.c	2697
4.126 smart.c File Reference	2712
4.126.1 Detailed Description	2712
4.126.2 Function Documentation	2713
4.126.2.1 StudyPattern()	2713
4.126.2.2 MatchIsPossible()	2713
4.127 smart.c	2713
4.128 sort.c File Reference	2718
4.128.1 Detailed Description	2719
4.128.2 Macro Definition Documentation	2719
4.128.2.1 NEWSPLITMERGE	2719
4.128.2.2 NEWSPLITMERGETIMSORT	2719
4.128.2.3 HUMANSTRLEN	2720
4.128.2.4 HUMANSUFFLEN	2720
4.128.2.5 INSLENGTH	2720
4.128.3 Function Documentation	2720
4.128.3.1 HumanString()	2720
4.128.3.2 WriteStats()	2720
4.128.3.3 NewSort()	2721
4.128.3.4 EndSort()	2721
4.128.3.5 PutIn()	2722
4.128.3.6 Sflush()	2723
4.128.3.7 PutOut()	2723
4.128.3.8 FlushOut()	2724
4.128.3.9 AddCoef()	2725
4.128.3.10 AddPoly()	2725
4.128.3.11 AddArgs()	2726
4.128.3.12 Compare1()	2727
4.128.3.13 CompareSymbols()	2728

4.128.3.14 CompareHSymbols()	2728
4.128.3.15 ComPress()	2728
4.128.3.16 SplitMerge()	2729
4.128.3.17 GarbHand()	2730
4.128.3.18 MergePatches()	2730
4.128.3.19 StoreTerm()	2731
4.128.3.20 StageSort()	2732
4.128.3.21 SortWild()	2732
4.128.3.22 CleanUpSort()	2733
4.128.3.23 LowerSortLevel()	2733
4.128.3.24 PolyRatFunSpecial()	2733
4.128.3.25 SimpleSplitMergeRec()	2733
4.128.3.26 SimpleSplitMerge()	2733
4.128.3.27 BinarySearch()	2734
4.128.4 Variable Documentation	2734
4.128.4.1 numcompares	2734
4.128.4.2 toterms	2734
4.128.4.3 humanTermsSuffix	2734
4.128.4.4 humanBytesSuffix	2734
4.129 sort.c	2735
4.130 spectator.c File Reference	2790
4.130.1 Detailed Description	2791
4.130.2 Function Documentation	2791
4.130.2.1 CoCreateSpectator()	2791
4.130.2.2 CoToSpectator()	2791
4.130.2.3 CoRemoveSpectator()	2791
4.130.2.4 CoEmptySpectator()	2791
4.130.2.5 PutInSpectator()	2791
4.130.2.6 FlushSpectators()	2792
4.130.2.7 CoCopySpectator()	2792
4.130.2.8 GetFromSpectator()	2792
4.130.2.9 ClearSpectators()	2792
4.131 spectator.c	2792
4.132 startup.c File Reference	2800
4.132.1 Detailed Description	2801
4.132.2 Macro Definition Documentation	2802
4.132.2.1 STRINGIFY	2802
4.132.2.2 STRINGIFY__	2802
4.132.2.3 FORMNAME	2802
4.132.2.4 VERSIONSTR__	2802
4.132.2.5 VERSIONSTR	2802
4.132.2.6 TAKEPATH	2802

4.132.2.7 DEFAULTFNAMELENGTH	2802
4.132.3 Function Documentation	2803
4.132.3.1 DoTail()	2803
4.132.3.2 OpenInput()	2803
4.132.3.3 ReserveTempFiles()	2803
4.132.3.4 StartVariables()	2803
4.132.3.5 StartMore()	2804
4.132.3.6 IniVars()	2804
4.132.3.7 main()	2804
4.132.3.8 CleanUp()	2804
4.132.3.9 TerminateImpl()	2804
4.132.3.10 PrintDeprecation()	2804
4.132.3.11 PrintRunningTime()	2805
4.132.3.12 GetRunningTime()	2805
4.132.4 Variable Documentation	2805
4.132.4.1 emptystring	2805
4.132.4.2 defaulttempfilename	2805
4.133 startup.c	2806
4.134 store.c File Reference	2831
4.134.1 Detailed Description	2832
4.134.2 Macro Definition Documentation	2832
4.134.2.1 SAVEREVISION	2832
4.134.3 Function Documentation	2832
4.134.3.1 OpenTemp()	2832
4.134.3.2 SeekScratch()	2832
4.134.3.3 SetEndScratch()	2832
4.134.3.4 SetEndHScratch()	2833
4.134.3.5 SetScratch()	2833
4.134.3.6 RevertScratch()	2833
4.134.3.7 ResetScratch()	2833
4.134.3.8 ReadFromScratch()	2833
4.134.3.9 AddToScratch()	2833
4.134.3.10 CoSave()	2834
4.134.3.11 CoLoad()	2834
4.134.3.12 DeleteStore()	2834
4.134.3.13 PutInStore()	2834
4.134.3.14 GetTerm()	2834
4.134.3.15 GetOneTerm()	2834
4.134.3.16 GetMoreTerms()	2835
4.134.3.17 GetMoreFromMem()	2835
4.134.3.18 GetFromStore()	2835
4.134.3.19 DetVars()	2835

4.134.3.20 ToStorage()	2835
4.134.3.21 NextFileIndex()	2835
4.134.3.22 SetFileIndex()	2836
4.134.3.23 VarStore()	2836
4.134.3.24 TermRenumbr()	2836
4.134.3.25 FindrNumber()	2837
4.134.3.26 FindInIndex()	2837
4.134.3.27 GetTable()	2837
4.134.3.28 CopyExpression()	2837
4.134.3.29 WriteStoreHeader()	2837
4.134.3.30 ReadSaveHeader()	2838
4.134.3.31 ReadSaveIndex()	2838
4.134.3.32 ReadSaveVariables()	2839
4.134.3.33 ReadSaveTerm32()	2840
4.134.3.34 ReadSaveExpression()	2841
4.135 store.c	2841
4.136 structs.h File Reference	2899
4.136.1 Detailed Description	2902
4.136.2 Macro Definition Documentation	2903
4.136.2.1 INFILEINDEX	2903
4.136.2.2 EMPTYININDEX	2903
4.136.2.3 PHEAD	2903
4.136.2.4 PHEAD0	2903
4.136.2.5 BHEAD	2903
4.136.2.6 BHEAD0	2903
4.136.3 Typedef Documentation	2904
4.136.3.1 INDEXENTRY	2904
4.136.3.2 FILEINDEX	2904
4.136.3.3 VARRENUM	2904
4.136.3.4 RENUMBER	2904
4.136.3.5 NAMENODE	2904
4.136.3.6 NAMETREE	2904
4.136.3.7 COMPTREE	2905
4.136.3.8 TABLES	2905
4.136.3.9 FUNCTIONS	2905
4.136.3.10 FILEHANDLE	2905
4.136.3.11 STREAM	2905
4.136.3.12 TRACES	2905
4.136.3.13 TRACEN	2906
4.136.3.14 PREVAR	2906
4.136.3.15 DOLOOP	2906
4.136.3.16 set_of_char	2906

4.136.3.17 one_byte	2906
4.136.3.18 FINISHUFFLE	2906
4.136.3.19 DO_UFFLE	2906
4.136.3.20 COMPAREDUMMY	2907
4.136.3.21 CBUF	2907
4.136.3.22 CHANNEL	2907
4.136.3.23 NESTING	2907
4.136.3.24 STORECACHE	2907
4.136.3.25 PERM	2907
4.136.3.26 PERMP	2907
4.136.3.27 DISTRIBUTE	2908
4.136.3.28 PARTI	2908
4.136.3.29 SORTING	2908
4.136.3.30 ALLGLOBALS	2908
4.136.3.31 WCN	2908
4.136.3.32 WCN2	2908
4.136.3.33 COMPARE	2908
4.136.3.34 FIXEDGLOBALS	2909
4.137 structs.h	2909
4.138 symmetr.c File Reference	2935
4.138.1 Detailed Description	2935
4.138.2 Function Documentation	2935
4.138.2.1 MatchE()	2935
4.138.2.2 Permute()	2936
4.138.2.3 PermuteP()	2936
4.138.2.4 Distribute()	2936
4.138.2.5 MatchCy()	2936
4.138.2.6 FunMatchCy()	2936
4.138.2.7 FunMatchSy()	2936
4.138.2.8 MatchArgument()	2937
4.139 symmetr.c	2937
4.140 tables.c File Reference	2964
4.140.1 Detailed Description	2965
4.140.2 Function Documentation	2965
4.140.2.1 ClearTableTree()	2965
4.140.2.2 InsTableTree()	2965
4.140.2.3 RedoTableTree()	2965
4.140.2.4 FindTableTree()	2965
4.140.2.5 DoTableExpansion()	2966
4.140.2.6 CoTableBase()	2966
4.140.2.7 FlipTable()	2966
4.140.2.8 SpareTable()	2966

4.140.2.9 FindTB()	2966
4.140.2.10 CoTBcreate()	2966
4.140.2.11 CoTBopen()	2966
4.140.2.12 CoTBaddto()	2967
4.140.2.13 CoTBenter()	2967
4.140.2.14 CoTestUse()	2967
4.140.2.15 CheckTableDeclarations()	2967
4.140.2.16 CoTBload()	2967
4.140.2.17 TestUse()	2967
4.140.2.18 CoTBaudit()	2967
4.140.2.19 CoTBon()	2968
4.140.2.20 CoTBoff()	2968
4.140.2.21 CoTBCleanup()	2968
4.140.2.22 CoTBreplace()	2968
4.140.2.23 CoTBuse()	2968
4.140.2.24 CoApply()	2968
4.140.2.25 CoTBhelp()	2968
4.140.2.26 ReWorkT()	2969
4.140.2.27 Apply()	2969
4.140.2.28 ApplyExec()	2969
4.140.2.29 ApplyReset()	2969
4.140.2.30 TableReset()	2969
4.140.2.31 ReleaseTB()	2969
4.140.3 Variable Documentation	2970
4.140.3.1 helptb	2970
4.141 tables.c	2970
4.142 threads.c File Reference	2997
4.142.1 Detailed Description	2997
4.143 threads.c	2997
4.144 token.c File Reference	3051
4.144.1 Detailed Description	3052
4.144.2 Macro Definition Documentation	3052
4.144.2.1 CHECKPOLY	3052
4.144.3 Function Documentation	3052
4.144.3.1 tokenize()	3052
4.144.3.2 WriteTokens()	3052
4.144.3.3 simp1token()	3052
4.144.3.4 simpwtoken()	3053
4.144.3.5 simp2token()	3053
4.144.3.6 simp3atoken()	3053
4.144.3.7 simp3btoken()	3053
4.144.3.8 simp4token()	3053

4.144.3.9 simp5token()	3053
4.144.3.10 simp6token()	3054
4.144.4 Variable Documentation	3054
4.144.4.1 ttypes	3054
4.145 token.c	3054
4.146 tools.c File Reference	3079
4.146.1 Detailed Description	3081
4.146.2 Macro Definition Documentation	3081
4.146.2.1 FILLVALUE	3081
4.146.2.2 TERMMEMSTARTNUM	3081
4.146.2.3 TERMEXTRAWORDS	3082
4.146.2.4 NUMBERMEMSTARTNUM	3082
4.146.2.5 NUMBEREXTRAWORDS	3082
4.146.2.6 DODOUBLE	3082
4.146.2.7 DOEXPAND	3082
4.146.3 Function Documentation	3083
4.146.3.1 LoadInputFile()	3083
4.146.3.2 ReadFromStream()	3083
4.146.3.3 GetFromStream()	3083
4.146.3.4 LookInStream()	3083
4.146.3.5 OpenStream()	3083
4.146.3.6 LocateFile()	3083
4.146.3.7 CloseStream()	3084
4.146.3.8 CreateStream()	3084
4.146.3.9 GetStreamPosition()	3084
4.146.3.10 PositionStream()	3084
4.146.3.11 ReverseStatements()	3084
4.146.3.12 StartFiles()	3084
4.146.3.13 OpenFile()	3084
4.146.3.14 OpenAddFile()	3085
4.146.3.15 ReOpenFile()	3085
4.146.3.16 CreateFile()	3085
4.146.3.17 CreateLogFile()	3085
4.146.3.18 CloseFile()	3085
4.146.3.19 CopyFile()	3085
4.146.3.20 CreateHandle()	3086
4.146.3.21 ReadFile()	3086
4.146.3.22 ReadPosFile()	3086
4.146.3.23 WriteFileToFile()	3086
4.146.3.24 SeekFile()	3086
4.146.3.25 TellFile()	3086
4.146.3.26 TELLFILE()	3087

4.146.3.27 FlushFile()	3087
4.146.3.28 GetPosFile()	3087
4.146.3.29 SetPosFile()	3087
4.146.3.30 SynchFile()	3087
4.146.3.31 TruncateFile()	3087
4.146.3.32 GetChannel()	3088
4.146.3.33 GetAppendChannel()	3088
4.146.3.34 CloseChannel()	3088
4.146.3.35 UpdateMaxSize()	3088
4.146.3.36 StrCmp()	3088
4.146.3.37 StrLCmp()	3088
4.146.3.38 StrHICmp()	3089
4.146.3.39 StrLCont()	3089
4.146.3.40 CmpArray()	3089
4.146.3.41 ConWord()	3089
4.146.3.42 StrLen()	3089
4.146.3.43 NumToStr()	3089
4.146.3.44 WriteString()	3090
4.146.3.45 WriteUnfinString()	3090
4.146.3.46 AddToString()	3090
4.146.3.47 strDup1()	3090
4.146.3.48 EndOfToken()	3090
4.146.3.49 ToToken()	3090
4.146.3.50 SkipField()	3091
4.146.3.51 ReadSnum()	3091
4.146.3.52 NumCopy()	3091
4.146.3.53 LongCopy()	3091
4.146.3.54 LongLongCopy()	3091
4.146.3.55 MakeDate()	3091
4.146.3.56 set_in()	3092
4.146.3.57 set_set()	3092
4.146.3.58 set_del()	3092
4.146.3.59 set_sub()	3092
4.146.3.60 iniTools()	3092
4.146.3.61 Malloc()	3092
4.146.3.62 Malloc1()	3093
4.146.3.63 M_free()	3093
4.146.3.64 M_check1()	3093
4.146.3.65 M_print()	3093
4.146.3.66 TermMallocAddMemory()	3093
4.146.3.67 NumberMallocAddMemory()	3093
4.146.3.68 CacheNumberMallocAddMemory()	3093

4.146.3.69 FromList()	3094
4.146.3.70 From0List()	3094
4.146.3.71 FromVarList()	3094
4.146.3.72 DoubleList()	3094
4.146.3.73 DoubleLList()	3094
4.146.3.74 DoubleBuffer()	3094
4.146.3.75 ExpandBuffer()	3095
4.146.3.76 iexp()	3095
4.146.3.77 ToGeneral()	3095
4.146.3.78 ToFast()	3095
4.146.3.79 ToPolyFunGeneral()	3095
4.146.3.80 IsLikeVector()	3095
4.146.3.81 AreArgsEqual()	3096
4.146.3.82 CompareArgs()	3096
4.146.3.83 CompArg()	3096
4.146.3.84 TimeWallClock()	3096
4.146.3.85 TimeChildren()	3096
4.146.3.86 TimeCPU()	3097
4.146.3.87 Crash()	3097
4.146.3.88 TestTerm()	3097
4.146.3.89 DistrN()	3098
4.146.4 Variable Documentation	3098
4.146.4.1 filelist	3098
4.146.4.2 numinfilelist	3098
4.146.4.3 filelistsize	3098
4.146.4.4 WriteFile	3098
4.147 tools.c	3099
4.148 topowrap.cc File Reference	3145
4.148.1 Detailed Description	3146
4.148.2 Macro Definition Documentation	3146
4.148.2.1 MAXPOINTS	3146
4.148.3 Function Documentation	3146
4.148.3.1 GenerateVertices()	3146
4.148.3.2 GenerateTopologies()	3146
4.148.3.3 toForm()	3146
4.149 topowrap.cc	3147
4.150 transform.c File Reference	3150
4.150.1 Detailed Description	3151
4.150.2 Function Documentation	3151
4.150.2.1 CoTransform()	3151
4.150.2.2 RunTransform()	3151
4.150.2.3 RunEncode()	3151

4.150.2.4 RunDecode()	3151
4.150.2.5 RunReplace()	3152
4.150.2.6 RunImplode()	3152
4.150.2.7 RunExplode()	3152
4.150.2.8 RunPermute()	3152
4.150.2.9 RunReverse()	3152
4.150.2.10 RunDedup()	3152
4.150.2.11 RunCycle()	3153
4.150.2.12 RunAddArg()	3153
4.150.2.13 RunMulArg()	3153
4.150.2.14 RunIsLyndon()	3153
4.150.2.15 RunToLyndon()	3153
4.150.2.16 RunDropArg()	3153
4.150.2.17 RunSelectArg()	3154
4.150.2.18 RunZtoHArg()	3154
4.150.2.19 RunHtoZArg()	3154
4.150.2.20 TestArgNum()	3154
4.150.2.21 PutArgInScratch()	3154
4.150.2.22 ReadRange()	3154
4.150.2.23 FindRange()	3155
4.151 transform.c	3155
4.152 unix.h File Reference	3196
4.152.1 Detailed Description	3196
4.152.2 Macro Definition Documentation	3197
4.152.2.1 LINEFEED	3197
4.152.2.2 CARRIAGERETURN	3197
4.152.2.3 WITHPIPE	3197
4.152.2.4 WITHSYSTEM	3197
4.152.2.5 WITHEXTERNALCHANNEL	3197
4.152.2.6 TRAP SIGNALS	3197
4.152.2.7 P_term	3197
4.152.2.8 SEPARATOR	3198
4.152.2.9 ALTSEPARATOR	3198
4.152.2.10 PATHSEPARATOR	3198
4.152.2.11 WITH_ENV	3198
4.152.2.12 WITH_ALARM	3198
4.153 unix.h	3198
4.154 unixfile.c File Reference	3199
4.154.1 Detailed Description	3199
4.155 unixfile.c	3199
4.156 variable.h File Reference	3202
4.156.1 Detailed Description	3204

4.156.2 Macro Definition Documentation	3204
4.156.2.1 chartype	3204
4.156.2.2 Procedures	3204
4.156.2.3 NumProcedures	3204
4.156.2.4 MaxProcedures	3204
4.156.2.5 DoLoops	3205
4.156.2.6 NumDoLoops	3205
4.156.2.7 MaxDoLoops	3205
4.156.2.8 PreVar	3205
4.156.2.9 NumPre	3205
4.156.2.10 MaxNumPre	3205
4.156.2.11 SetElements	3205
4.156.2.12 Sets	3205
4.156.2.13 functions	3206
4.156.2.14 indices	3206
4.156.2.15 symbols	3206
4.156.2.16 vectors	3206
4.156.2.17 tablebases	3206
4.156.2.18 NumFunctions	3206
4.156.2.19 NumIndices	3206
4.156.2.20 NumSymbols	3206
4.156.2.21 NumVectors	3207
4.156.2.22 NumSets	3207
4.156.2.23 NumSetElements	3207
4.156.2.24 NumTableBases	3207
4.156.2.25 GlobalFunctions	3207
4.156.2.26 GlobalIndices	3207
4.156.2.27 GlobalSymbols	3207
4.156.2.28 GlobalVectors	3207
4.156.2.29 GlobalSets	3208
4.156.2.30 GlobalSetElements	3208
4.156.2.31 cbuf	3208
4.156.2.32 channels	3208
4.156.2.33 NumOutputChannels	3208
4.156.2.34 Dollars	3208
4.156.2.35 NumDollars	3208
4.156.2.36 Dubious	3208
4.156.2.37 NumDubious	3209
4.156.2.38 Expressions	3209
4.156.2.39 NumExpressions	3209
4.156.2.40 autofunctions	3209
4.156.2.41 autoindices	3209

4.156.2.42 autosymbols	3209
4.156.2.43 autovectors	3209
4.156.2.44 xsymbol	3209
4.156.2.45 numxsymbol	3210
4.156.2.46 PotModdollars	3210
4.156.2.47 NumPotModdollars	3210
4.156.2.48 ModOptdollars	3210
4.156.2.49 NumModOptdollars	3210
4.156.2.50 BUG	3210
4.156.2.51 AC	3210
4.156.2.52 AM	3210
4.156.2.53 AN	3211
4.156.2.54 AO	3211
4.156.2.55 AP	3211
4.156.2.56 AR	3211
4.156.2.57 AS	3211
4.156.2.58 AT	3211
4.156.2.59 AX	3211
4.156.3 Variable Documentation	3212
4.156.3.1 WriteFile	3212
4.156.3.2 A	3212
4.156.3.3 FG	3212
4.156.3.4 fixedsets	3212
4.156.3.5 setupfilename	3212
4.157 variable.h	3213
4.158 vector.h File Reference	3214
4.158.1 Detailed Description	3216
4.158.2 Macro Definition Documentation	3216
4.158.2.1 VectorStruct	3216
4.158.2.2 Vector	3216
4.158.2.3 DeclareVector	3217
4.158.2.4 VectorInit	3217
4.158.2.5 VectorFree	3217
4.158.2.6 VectorPtr	3218
4.158.2.7 VectorFront	3218
4.158.2.8 VectorBack	3218
4.158.2.9 VectorSize	3219
4.158.2.10 VectorCapacity	3219
4.158.2.11 VectorEmpty	3219
4.158.2.12 VectorClear	3220
4.158.2.13 VectorReserve	3220
4.158.2.14 VectorPushBack	3221

4.158.2.15 VectorPushBacks	3221
4.158.2.16 VectorPopBack	3221
4.158.2.17 VectorInsert	3222
4.158.2.18 VectorInserts	3222
4.158.2.19 VectorErase	3223
4.158.2.20 VectorErases	3223
4.159 vector.h	3223
4.160 wildcard.c File Reference	3227
4.160.1 Detailed Description	3227
4.160.2 Macro Definition Documentation	3227
4.160.2.1 DEBUG	3227
4.160.3 Function Documentation	3228
4.160.3.1 WildFill()	3228
4.160.3.2 ResolveSet()	3228
4.160.3.3 ClearWild()	3228
4.160.3.4 AddWild()	3228
4.160.3.5 CheckWild()	3228
4.160.3.6 DenToFunction()	3229
4.161 wildcard.c	3229

Chapter 1

Data Structure Index

1.1 Data Structures

Here are the data structures with brief descriptions:

AEdge	7
AllGlobals	9
ANode	11
ARGBUFFER	14
Assign	15
AStack	26
bit_field	29
BrAcKeTiNdEx	31
BrAcKeTiNfO	32
BracketInfo	34
bufIPstruct	36
C_const	37
CbUf	70
ChAnNeL	75
COST	75
CSEEq	76
CSEHash	77
dbase	78
DGraph	80
DICTIONARY	81
DICTIONARY_ELEMENT	83
DiStRiBuTe	84
DNode	86
dollar_buf	87
DoLIaR\$	87
DoLoOp	89
DuBiOuS	93
ECand	94
EEdge	95
EFLine	100
EGraph	102
ENode	114
EStack	118
ExPrEsSiOn	120
FaCdOILaR	124

factorized_poly	125
FGInput	126
FiLe	128
FiLeDaTa	131
FiLeInDeX	133
FixedGlobals	134
fixedset	137
flint::fmpz	138
Fraction	139
FUN_INFO	142
FuNcTiOn	144
gcd_heuristic_failed	147
HANDLERS	147
IInput	149
InDeX	150
indexblock	152
InDeXeNtRy	153
iniinfo	156
INSIDEINFO	157
Interaction	159
KEYWORD	163
KEYWORDV	164
LIST	165
longMultiStruct	167
M_const	168
MCBlock	193
MCBridge	195
MConn	196
MCOpi	202
MGraph	205
MiNmAx	216
MInput	217
MNode	218
MNodeClass	220
Model	225
MoDeL	232
MODNUM	235
MoDoPtDoLIaRS	236
monomial_larger	236
MORbits	237
flint::mpoly	239
flint::mpoly_ctx	240
flint::mpoly_factor	241
N_const	242
nameblock	259
NaMeNode	260
NaMeSpAcE	262
NaMeTree	263
NCand	266
NCInput	268
NeStInG	269
NoDe	270
node	271
NodeEq	274
NodeHash	274
NStack	275
O_const	276
objects	287

OptDef	288
optimization	289
OPTIMIZE	292
OPTIMIZERESULT	295
Options	296
Output	303
P_const	309
ParallelVars	317
PaRtl	320
PaRtlcLe	321
Particle	322
PeRmUtE	326
PeRmUtEp	327
PF_BUFFER	328
PGInput	331
PInput	332
PNodeClass	334
flint::poly	337
poly	339
flint::poly_factor	356
POLYMOD	357
PoSiTiOn	358
PRELOAD	359
pReVaR	360
PROCEDURE	361
Process	362
R_const	369
ReNuMbEr	378
S_const	380
SeTs	383
SETUPPARAMETERS	384
SGroup	385
SHvariables	390
sOrT	392
SpecTatoR	401
SProcess	402
StOrEcAcHe	411
STOREHEADER	412
StreaM	416
SuBbUf	420
SWITCH	421
SWITCHTABLE	423
SyMbOl	424
T_const	426
T_EEdge	440
T_EGraph	441
T_ENode	445
T_MGraph	446
T_MNode	451
T_MNodeClass	453
TaBIEbAsE	457
TaBIEbAsEsUbInDeX	458
TaBIes	459
TERMINFO	466
ToPoTyPe	468
TrAcEn	471
TrAcEs	473
tree	478

tree_node	480
VaRrEnUm	482
VeCtOr	483
VeRtEx	485
X_const	487

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

argument.c	489
bugtool.c	535
checkpoint.c	537
comexpr.c	581
compcomm.c	609
compiler.c	722
compress.c	753
comtool.c	762
comtool.h	775
declare.h	777
diagrams.c	1051
diawrap.cc	1061
dict.c	1073
dollar.c	1091
evaluate.c	1146
execute.c	1154
extcmd.c	1192
factor.c	1213
findpat.c	1224
flintinterface.cc	1241
flintinterface.h	1268
flintwrap.cc	1278
float.c	1284
form3.h	1323
fsizes.h	1340
ftypes.h	1355
function.c	1471
fwin.h	1497
gentopo.cc	1499
gentopo.h	1517
grcc.cc	1519
grcc.h	1638
grccparam.h	1653
if.c	1656
index.c	1673

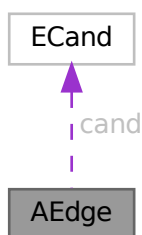
inivar.h	1683
lus.c	1688
mallocprotect.h	1710
message.c	1714
minos.c	1729
minos.h	1753
model.c	1758
module.c	1764
mpi.c	1777
mpidbg.h	1811
mytime.cc	1825
mytime.h	1825
names.c	1826
normal.c	1875
notation.c	1941
opera.c	1957
optimize.cc	1990
parallel.c	2064
parallel.h	2138
pattern.c	2174
poly.cc	2204
poly.h	2237
polyfact.cc	2240
polyfact.h	2257
polygcd.cc	2259
polygcd.h	2276
polywrap.cc	2278
portsignals.h	2311
pre.c	2316
proces.c	2427
ratio.c	2504
reken.c	2553
reshuf.c	2612
sch.c	2653
setfile.c	2692
smart.c	2712
sort.c	2718
spectator.c	2790
startup.c	2800
store.c	2831
structs.h	2899
symmetr.c	2935
tables.c	2964
threads.c	2997
token.c	3051
tools.c	3079
topowrap.cc	3145
transform.c	3150
unix.h	3196
unixfile.c	3199
variable.h	3202
vector.h	3214
wildcard.c	3227

Chapter 3

Data Structure Documentation

3.1 AEdge Class Reference

Collaboration diagram for AEdge:



Public Member Functions

- [AEdge](#) (int n0, int l0, int n1, int l1)

Data Fields

- [ECand](#) * [cand](#)
- int [nodes](#) [2]
- int [nlegs](#) [2]
- int [ptcl](#)
- int [DUMMY_PADDING](#)

3.1.1 Detailed Description

Definition at line [1102](#) of file [grcc.h](#).

3.1.2 Constructor & Destructor Documentation

3.1.2.1 AEdge()

```
AEdge::AEdge (
    int  n0,
    int  l0,
    int  n1,
    int  l1 )
```

Definition at line [7413](#) of file [grcc.cc](#).

3.1.2.2 ~AEdge()

```
AEdge::~AEdge (
    void  )
```

Definition at line [7424](#) of file [grcc.cc](#).

3.1.3 Field Documentation

3.1.3.1 cand

```
ECand* AEdge::cand
```

Definition at line [1109](#) of file [grcc.h](#).

3.1.3.2 nodes

```
int AEdge::nodes[2]
```

Definition at line [1110](#) of file [grcc.h](#).

3.1.3.3 nlegs

```
int AEdge::nlegs[2]
```

Definition at line [1111](#) of file [grcc.h](#).

3.1.3.4 ptcl

```
int AEdge::ptcl
```

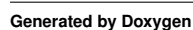
Definition at line [1112](#) of file [grcc.h](#).

```
int AEdge::DUMMYPADDING
```

The documentation for this class was generated from the following files:

- ## 3.2 AllGlobals Struct Reference

Collaboration diagram for AllGlobals:



Public Member Functions

- **PADPOSITION** (0, 0, 0, 0, sizeof(struct [P_const](#))+sizeof(struct [T_const](#))+sizeof(struct [X_const](#)))

Data Fields

- struct [M_const](#) M
- struct [C_const](#) Cc
- struct [S_const](#) S
- struct [R_const](#) R
- struct [N_const](#) N
- struct [O_const](#) O
- struct [P_const](#) P
- struct [T_const](#) T
- struct [X_const](#) X

3.2.1 Detailed Description

Without pthreads (FORM) the ALLGLOBALS struct has all the global variables

Definition at line 2620 of file [structs.h](#).

3.2.2 Field Documentation

3.2.2.1 M

```
struct M\_const AllGlobals::M
```

Definition at line 2621 of file [structs.h](#).

3.2.2.2 Cc

```
struct C\_const AllGlobals::Cc
```

Definition at line 2622 of file [structs.h](#).

3.2.2.3 S

```
struct S\_const AllGlobals::S
```

Definition at line 2623 of file [structs.h](#).

3.2.2.4 R

```
struct R\_const AllGlobals::R
```

Definition at line 2624 of file [structs.h](#).

3.2.2.5 N

```
struct N_const AllGlobals::N
```

Definition at line 2625 of file [structs.h](#).

3.2.2.6 O

```
struct O_const AllGlobals::O
```

Definition at line 2626 of file [structs.h](#).

3.2.2.7 P

```
struct P_const AllGlobals::P
```

Definition at line 2627 of file [structs.h](#).

3.2.2.8 T

```
struct T_const AllGlobals::T
```

Definition at line 2628 of file [structs.h](#).

3.2.2.9 X

```
struct X_const AllGlobals::X
```

Definition at line 2629 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.3 ANode Class Reference

Collaboration diagram for ANode:



Public Member Functions

- [ANode](#) (int dg)
- int [newleg](#) (void)

Data Fields

- int [deg](#)
- int [nlegs](#)
- int * [anodes](#)
- int * [aedges](#)
- int * [aelegs](#)
- [NCand](#) * [cand](#)

3.3.1 Detailed Description

Definition at line [1078](#) of file [grcc.h](#).

3.3.2 Constructor & Destructor Documentation

3.3.2.1 ANode()

```
ANode::ANode (  
    int dg )
```

Definition at line [7358](#) of file [grcc.cc](#).

3.3.2.2 ~ANode()

```
ANode::~ANode (  
    void )
```

Definition at line [7376](#) of file [grcc.cc](#).

3.3.3 Member Function Documentation

3.3.3.1 newleg()

```
int ANode::newleg (  
    void )
```

Definition at line [7393](#) of file [grcc.cc](#).

3.3.4 Field Documentation

3.3.4.1 deg

```
int ANode::deg
```

Definition at line 1088 of file [grcc.h](#).

3.3.4.2 nlegs

```
int ANode::nlegs
```

Definition at line 1089 of file [grcc.h](#).

3.3.4.3 anodes

```
int* ANode::anodes
```

Definition at line 1090 of file [grcc.h](#).

3.3.4.4 aedges

```
int* ANode::aedges
```

Definition at line 1091 of file [grcc.h](#).

3.3.4.5 aelegs

```
int* ANode::aelegs
```

Definition at line 1092 of file [grcc.h](#).

3.3.4.6 cand

```
NCand* ANode::cand
```

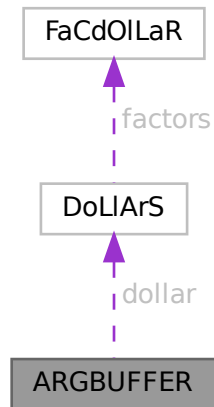
Definition at line 1093 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.4 ARGBUFFER Struct Reference

Collaboration diagram for ARGBUFFER:



Data Fields

- WORD * [buffer](#)
- DOLLARS [dollar](#)
- LONG [size](#)
- int [type](#)
- int [dummy](#)

3.4.1 Detailed Description

Definition at line [601](#) of file [ratio.c](#).

3.4.2 Field Documentation

3.4.2.1 buffer

```
WORD* ARGBUFFER::buffer
```

Definition at line [602](#) of file [ratio.c](#).

3.4.2.2 dollar

```
DOLLARS ARGBUFFER::dollar
```

Definition at line [603](#) of file [ratio.c](#).

Public Member Functions

- [Assign](#) ([SProcess](#) *sprc, [MGraph](#) *mgr, [PNodeClass](#) *pnc)
- void [prCand](#) (const char *msg)
- void [checkAG](#) (const char *msg)
- Bool [assignAllVertices](#) (void)
- Bool [selectVertex](#) (void)
- Bool [selectVertexSimp](#) (int lastv)
- Bool [selectLeg](#) (int v, int lastlg)
- Bool [assignVertex](#) (int v)
- Bool [allAssigned](#) (void)
- Bool [fromMGraph](#) (void)
- void [addEdge](#) (int n0, int n1, int nplist, int *plist)
- void [connect](#) (int n0, int l0, int eg, int el, int n1, int l1)
- Bool [fillEGraph](#) (int aid, [BigInt](#) nsym, [BigInt](#) esym, [BigInt](#) nsym1)
- int * [reordLeg](#) (int n, int *reord, int *plist, int *used)
- int [getLegParticle](#) (int n, int ln)
- int [legEdgeParticle](#) (int n, int ln, int pt)
- int [legPart](#) (int v, int lg, int nplst, int *plst, int *rlist, const int size)
- int [candPart](#) (int v, int ln, int *plist, const int size)
- int [selUnAssVertex](#) (void)
- int [selUnAssVertexSimp](#) (int lastv)
- int [selUnAssLeg](#) (int v, int lastlg)
- [NCandSt](#) [assignVertex](#) (int v, int ia)
- Bool [assignPLeg](#) (int n, int ln, int pt)
- Bool [isOrdPLeg](#) (int n, int ln, int pt)
- Bool [detEdge](#) (int e)
- Bool [candPartClassify](#) (int v, int *npdass, int *pdass, int *npuass, int *puass, const int size)
- Bool [updateCandNode](#) (int v)
- Bool [checkOrderCpl](#) (void)
- Bool [isOrdLegs](#) (void)
- Bool [isIsomorphic](#) ([MNodeClass](#) *cl, [BigInt](#) *nsym, [BigInt](#) *esym, [BigInt](#) *nsym1)
- int [cmpPermGraph](#) (int *p, [MNodeClass](#) *cl)
- int [cmpNodes](#) (int nd0, int nd1, [MNodeClass](#) *cn)
- [BigInt](#) [edgeSym](#) (void)
- void [saveCouple](#) (int *sav)
- void [restoreCouple](#) (int *sav)
- Bool [subCouple](#) (int *cpl)

Data Fields

- [EGraph](#) * egraph
- [MGraph](#) * mgraph
- [SProcess](#) * sprc
- [Process](#) * proc
- [Model](#) * model
- [Options](#) * opt
- [AStack](#) * astack
- [PNodeClass](#) * pnclass
- [MOrbits](#) * orbits
- int [nNodes](#)
- int [nEdges](#)
- int [nExtern](#)
- int [nETotal](#)

- [BigInt](#) [nAGraphs](#)
- [Fraction](#) [wAGraphs](#)
- [BigInt](#) [nAOPI](#)
- [Fraction](#) [wAOPI](#)
- [ANode](#) ** [nodes](#)
- [AEdge](#) ** [edges](#)
- [CheckPt](#) [checkpoint0](#)
- [int](#) [cplleft](#) [[GRCC_MAXNCPLG](#)]

3.5.1 Detailed Description

Definition at line [1121](#) of file [grcc.h](#).

3.5.2 Constructor & Destructor Documentation

3.5.2.1 Assign()

```
Assign::Assign (  
    SProcess * sprc,  
    MGraph * mgr,  
    PNodeClass * pnc )
```

Definition at line [7433](#) of file [grcc.cc](#).

3.5.2.2 ~Assign()

```
Assign::~Assign (  
    void )
```

Definition at line [7533](#) of file [grcc.cc](#).

3.5.3 Member Function Documentation

3.5.3.1 prCand()

```
void Assign::prCand (  
    const char * msg )
```

Definition at line [7553](#) of file [grcc.cc](#).

3.5.3.2 checkAG()

```
void Assign::checkAG (  
    const char * msg )
```

Definition at line [7589](#) of file [grcc.cc](#).

3.5.3.3 assignAllVertices()

```
Bool Assign::assignAllVertices (
    void )
```

Definition at line [7613](#) of file [grcc.cc](#).

3.5.3.4 selectVertex()

```
Bool Assign::selectVertex (
    void )
```

Definition at line [7686](#) of file [grcc.cc](#).

3.5.3.5 selectVertexSimp()

```
Bool Assign::selectVertexSimp (
    int lastv )
```

Definition at line [7651](#) of file [grcc.cc](#).

3.5.3.6 selectLeg()

```
Bool Assign::selectLeg (
    int v,
    int lastlg )
```

Definition at line [7721](#) of file [grcc.cc](#).

3.5.3.7 assignVertex()

```
Bool Assign::assignVertex (
    int v )
```

Definition at line [7823](#) of file [grcc.cc](#).

3.5.3.8 allAssigned()

```
Bool Assign::allAssigned (
    void )
```

Definition at line [7946](#) of file [grcc.cc](#).

3.5.3.9 fromMGraph()

```
Bool Assign::fromMGraph (
    void )
```

Definition at line [8050](#) of file [grcc.cc](#).

3.5.3.10 addEdge()

```
void Assign::addEdge (
    int n0,
    int n1,
    int nplist,
    int * plist )
```

Definition at line 8196 of file [grcc.cc](#).

3.5.3.11 connect()

```
void Assign::connect (
    int n0,
    int l0,
    int eg,
    int el,
    int n1,
    int l1 )
```

Definition at line 8241 of file [grcc.cc](#).

3.5.3.12 fillEGraph()

```
Bool Assign::fillEGraph (
    int aid,
    BigInt nsym,
    BigInt esym,
    BigInt nsym1 )
```

Definition at line 8270 of file [grcc.cc](#).

3.5.3.13 reordLeg()

```
int * Assign::reordLeg (
    int n,
    int * reord,
    int * plist,
    int * used )
```

Definition at line 8417 of file [grcc.cc](#).

3.5.3.14 getLegParticle()

```
int Assign::getLegParticle (
    int n,
    int ln )
```

Definition at line 8492 of file [grcc.cc](#).

3.5.3.15 legEdgeParticle()

```
int Assign::legEdgeParticle (
    int n,
    int ln,
    int pt )
```

Definition at line 8519 of file [grcc.cc](#).

3.5.3.16 legPart()

```
int Assign::legPart (
    int v,
    int lg,
    int nplst,
    int * plst,
    int * rlist,
    const int size )
```

Definition at line 8539 of file [grcc.cc](#).

3.5.3.17 candPart()

```
int Assign::candPart (
    int v,
    int ln,
    int * plist,
    const int size )
```

Definition at line 8555 of file [grcc.cc](#).

3.5.3.18 selUnAssVertex()

```
int Assign::selUnAssVertex (
    void )
```

Definition at line 8608 of file [grcc.cc](#).

3.5.3.19 selUnAssVertexSimp()

```
int Assign::selUnAssVertexSimp (
    int lastv )
```

Definition at line 8588 of file [grcc.cc](#).

3.5.3.20 selUnAssLeg()

```
int Assign::selUnAssLeg (
    int v,
    int lastlg )
```

Definition at line 8637 of file [grcc.cc](#).

3.5.3.21 assignVertex()

```
NCandSt Assign::assignVertex (
    int v,
    int ia )
```

Definition at line 8676 of file [grcc.cc](#).

3.5.3.22 assignPLeg()

```
Bool Assign::assignPLeg (
    int n,
    int ln,
    int pt )
```

Definition at line 8722 of file [grcc.cc](#).

3.5.3.23 isOrdPLeg()

```
Bool Assign::isOrdPLeg (
    int n,
    int ln,
    int pt )
```

Definition at line 8798 of file [grcc.cc](#).

3.5.3.24 detEdge()

```
Bool Assign::detEdge (
    int e )
```

Definition at line 8777 of file [grcc.cc](#).

3.5.3.25 candPartClassify()

```
Bool Assign::candPartClassify (
    int v,
    int * npdass,
    int * pdass,
    int * npuass,
    int * puass,
    const int size )
```

Definition at line 8842 of file [grcc.cc](#).

3.5.3.26 updateCandNode()

```
Bool Assign::updateCandNode (
    int v )
```

Definition at line 8879 of file [grcc.cc](#).

3.5.3.27 checkOrderCpl()

```
Bool Assign::checkOrderCpl (
    void )
```

Definition at line 9018 of file [grcc.cc](#).

3.5.3.28 isOrdLegs()

```
Bool Assign::isOrdLegs (
    void )
```

Definition at line 9056 of file [grcc.cc](#).

3.5.3.29 isIsomorphic()

```
Bool Assign::isIsomorphic (
    MNodeClass * cl,
    BigInt * nsym,
    BigInt * esym,
    BigInt * nsym1 )
```

Definition at line 9094 of file [grcc.cc](#).

3.5.3.30 cmpPermGraph()

```
int Assign::cmpPermGraph (
    int * p,
    MNodeClass * cl )
```

Definition at line 9176 of file [grcc.cc](#).

3.5.3.31 cmpNodes()

```
int Assign::cmpNodes (
    int nd0,
    int nd1,
    MNodeClass * cn )
```

Definition at line 9274 of file [grcc.cc](#).

3.5.3.32 edgeSym()

```
BigInt Assign::edgeSym (
    void )
```

Definition at line 9300 of file [grcc.cc](#).

3.5.3.33 saveCouple()

```
void Assign::saveCouple (
    int * sav )
```

Definition at line 7783 of file [grcc.cc](#).

3.5.3.34 restoreCouple()

```
void Assign::restoreCouple (
    int * sav )
```

Definition at line 7795 of file [grcc.cc](#).

3.5.3.35 subCouple()

```
Bool Assign::subCouple (
    int * cpl )
```

Definition at line 7807 of file [grcc.cc](#).

3.5.4 Field Documentation

3.5.4.1 egraph

```
EGraph* Assign::egraph
```

Definition at line 1126 of file [grcc.h](#).

3.5.4.2 mgraph

```
MGraph* Assign::mgraph
```

Definition at line 1127 of file [grcc.h](#).

3.5.4.3 sproc

```
SProcess* Assign::sproc
```

Definition at line 1128 of file [grcc.h](#).

3.5.4.4 proc

```
Process* Assign::proc
```

Definition at line 1129 of file [grcc.h](#).

3.5.4.5 model

`Model*` Assign::model

Definition at line 1130 of file [grcc.h](#).

3.5.4.6 opt

`Options*` Assign::opt

Definition at line 1131 of file [grcc.h](#).

3.5.4.7 astack

`AStack*` Assign::astack

Definition at line 1132 of file [grcc.h](#).

3.5.4.8 pnclass

`PNodeClass*` Assign::pnclass

Definition at line 1133 of file [grcc.h](#).

3.5.4.9 orbits

`MOrbits*` Assign::orbits

Definition at line 1134 of file [grcc.h](#).

3.5.4.10 nNodes

`int` Assign::nNodes

Definition at line 1136 of file [grcc.h](#).

3.5.4.11 nEdges

`int` Assign::nEdges

Definition at line 1137 of file [grcc.h](#).

3.5.4.12 nExtern

`int` Assign::nExtern

Definition at line 1138 of file [grcc.h](#).

3.5.4.13 nETotal

`int Assign::nETotal`

Definition at line 1139 of file [grcc.h](#).

3.5.4.14 nAGraphs

`BigInt Assign::nAGraphs`

Definition at line 1141 of file [grcc.h](#).

3.5.4.15 wAGraphs

`Fraction Assign::wAGraphs`

Definition at line 1142 of file [grcc.h](#).

3.5.4.16 nAOPI

`BigInt Assign::nAOPI`

Definition at line 1143 of file [grcc.h](#).

3.5.4.17 wAOPI

`Fraction Assign::wAOPI`

Definition at line 1144 of file [grcc.h](#).

3.5.4.18 nodes

`ANode** Assign::nodes`

Definition at line 1146 of file [grcc.h](#).

3.5.4.19 edges

`AEdge** Assign::edges`

Definition at line 1147 of file [grcc.h](#).

3.5.4.20 checkpoint0

`CheckPt Assign::checkpoint0`

Definition at line 1149 of file [grcc.h](#).

3.5.4.21 cplleft

```
int Assign::cplleft[GRCC_MAXNCPLG]
```

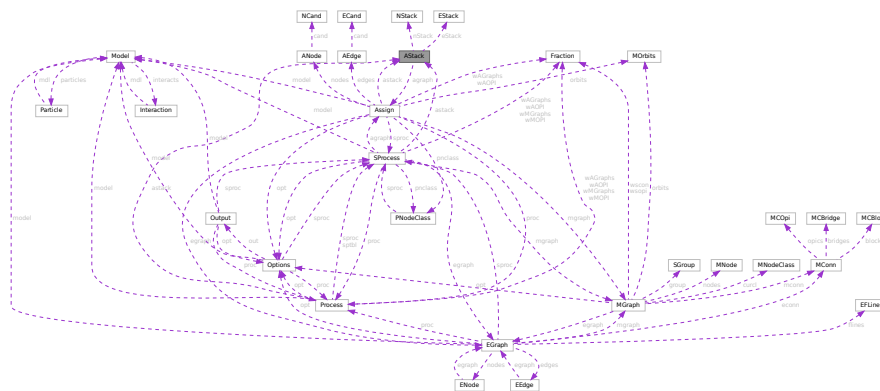
Definition at line 1151 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.6 AStack Class Reference

Collaboration diagram for AStack:



Public Member Functions

- [AStack](#) (int nSize, int eSize)
- void [setAGraph](#) ([Assign](#) *ag)
- void [checkPoint](#) ([CheckPt](#) sav)
- void [restore](#) ([CheckPt](#) sav)
- void [restoreMsg](#) ([CheckPt](#) sav, const char *msg)
- void [prStack](#) (void)
- void [pushNode](#) (int n)
- void [pushEdge](#) (int e)

Data Fields

- [Assign](#) * [agraph](#)
- [NStack](#) ** [nStack](#)
- int [nStackP](#)
- int [nSize](#)
- [EStack](#) ** [eStack](#)
- int [eStackP](#)
- int [eSize](#)

3.6.1 Detailed Description

Definition at line 1305 of file [grcc.h](#).

3.6.2 Constructor & Destructor Documentation

3.6.2.1 AStack()

```
AStack::AStack (
    int nSize,
    int eSize )
```

Definition at line 9470 of file [grcc.cc](#).

3.6.2.2 ~AStack()

```
AStack::~~AStack (
    void )
```

Definition at line 9502 of file [grcc.cc](#).

3.6.3 Member Function Documentation

3.6.3.1 setAGraph()

```
void AStack::setAGraph (
    Assign * ag )
```

Definition at line 9530 of file [grcc.cc](#).

3.6.3.2 checkPoint()

```
void AStack::checkPoint (
    CheckPt sav )
```

Definition at line 9595 of file [grcc.cc](#).

3.6.3.3 restore()

```
void AStack::restore (
    CheckPt sav )
```

Definition at line 9639 of file [grcc.cc](#).

3.6.3.4 prStack()

```
void AStack::prStack (
    void )
```

Definition at line [9662](#) of file [grcc.cc](#).

3.6.3.5 pushNode()

```
void AStack::pushNode (
    int n )
```

Definition at line [9536](#) of file [grcc.cc](#).

3.6.3.6 pushEdge()

```
void AStack::pushEdge (
    int e )
```

Definition at line [9566](#) of file [grcc.cc](#).

3.6.4 Field Documentation

3.6.4.1 agraph

```
Assign* AStack::agraph
```

Definition at line [1309](#) of file [grcc.h](#).

3.6.4.2 nStack

```
NStack** AStack::nStack
```

Definition at line [1311](#) of file [grcc.h](#).

3.6.4.3 nStackP

```
int AStack::nStackP
```

Definition at line [1312](#) of file [grcc.h](#).

3.6.4.4 nSize

```
int AStack::nSize
```

Definition at line [1313](#) of file [grcc.h](#).

3.6.4.5 eStack

```
EStack** AStack::eStack
```

Definition at line 1315 of file [grcc.h](#).

3.6.4.6 eStackP

```
int AStack::eStackP
```

Definition at line 1316 of file [grcc.h](#).

3.6.4.7 eSize

```
int AStack::eSize
```

Definition at line 1317 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.7 bit_field Struct Reference

```
#include <structs.h>
```

Data Fields

- UBYTE [bit_0](#): 1
- UBYTE [bit_1](#): 1
- UBYTE [bit_2](#): 1
- UBYTE [bit_3](#): 1
- UBYTE [bit_4](#): 1
- UBYTE [bit_5](#): 1
- UBYTE [bit_6](#): 1
- UBYTE [bit_7](#): 1

3.7.1 Detailed Description

The struct [bit_field](#) is used by [set_in](#), [set_set](#), [set_del](#) and [set_sub](#). They in turn are used in [pre.c](#) to toggle bits that indicate whether a character can be used as a separator of function arguments. This facility is used in the communication with external channels.

Definition at line 927 of file [structs.h](#).

3.7.2 Field Documentation

3.7.2.1 bit_0

```
UBYTE bit_field::bit_0
```

Definition at line 928 of file [structs.h](#).

3.7.2.2 bit_1

```
UBYTE bit_field::bit_1
```

Definition at line 929 of file [structs.h](#).

3.7.2.3 bit_2

```
UBYTE bit_field::bit_2
```

Definition at line 930 of file [structs.h](#).

3.7.2.4 bit_3

```
UBYTE bit_field::bit_3
```

Definition at line 931 of file [structs.h](#).

3.7.2.5 bit_4

```
UBYTE bit_field::bit_4
```

Definition at line 932 of file [structs.h](#).

3.7.2.6 bit_5

```
UBYTE bit_field::bit_5
```

Definition at line 933 of file [structs.h](#).

3.7.2.7 bit_6

```
UBYTE bit_field::bit_6
```

Definition at line 934 of file [structs.h](#).

3.7.2.8 bit_7

```
UBYTE bit_field::bit_7
```

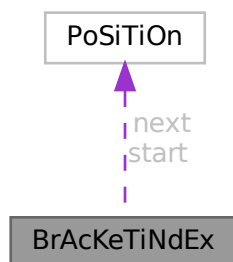
Definition at line 935 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.8 BrAcKeTiNdEx Struct Reference

Collaboration diagram for BrAcKeTiNdEx:



Public Member Functions

- **PADPOSITION** (0, 2, 0, 0, 0)

Data Fields

- [POSITION](#) start
- [POSITION](#) next
- LONG [bracket](#)
- LONG [termsinbracket](#)

3.8.1 Detailed Description

Definition at line 317 of file [structs.h](#).

3.8.2 Field Documentation

3.8.2.1 start

`POSITION BrAcKeTiNdEx::start`

Definition at line 318 of file [structs.h](#).

3.8.2.2 next

`POSITION BrAcKeTiNdEx::next`

Definition at line 319 of file [structs.h](#).

3.8.2.3 bracket

`LONG BrAcKeTiNdEx::bracket`

Definition at line 320 of file [structs.h](#).

3.8.2.4 termsinbracket

`LONG BrAcKeTiNdEx::termsinbracket`

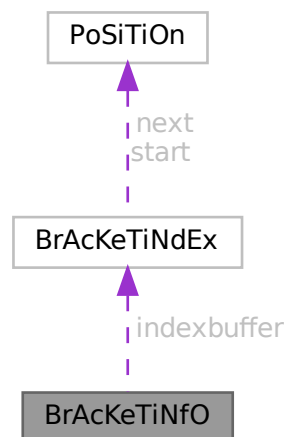
Definition at line 321 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.9 BrAcKeTiNfO Struct Reference

Collaboration diagram for BrAcKeTiNfO:



Data Fields

- [BRACKETINDEX](#) * [indexbuffer](#)
- WORD * [bracketbuffer](#)
- LONG [bracketbuffersize](#)
- LONG [indexbuffersize](#)
- LONG [bracketfill](#)
- LONG [indexfill](#)
- WORD [SortType](#)

3.9.1 Detailed Description

Definition at line [329](#) of file [structs.h](#).

3.9.2 Field Documentation

3.9.2.1 indexbuffer

```
BRACKETINDEX* BrAcKeTiNfO::indexbuffer
```

[D]

Definition at line [330](#) of file [structs.h](#).

Referenced by [DoRecovery\(\)](#), and [FlushOut\(\)](#).

3.9.2.2 bracketbuffer

```
WORD* BrAcKeTiNfO::bracketbuffer
```

[D]

Definition at line [331](#) of file [structs.h](#).

Referenced by [DoRecovery\(\)](#).

3.9.2.3 bracketbuffersize

```
LONG BrAcKeTiNfO::bracketbuffersize
```

Definition at line [332](#) of file [structs.h](#).

3.9.2.4 indexbuffersize

```
LONG BrAcKeTiNfO::indexbuffersize
```

Definition at line [333](#) of file [structs.h](#).

3.9.2.5 bracketfill

```
LONG BrAcKeTiNfO::bracketfill
```

Definition at line 334 of file [structs.h](#).

3.9.2.6 indexfill

```
LONG BrAcKeTiNfO::indexfill
```

Definition at line 335 of file [structs.h](#).

3.9.2.7 SortType

```
WORD BrAcKeTiNfO::SortType
```

The sorting criterium used (like POWERFIRST etc)

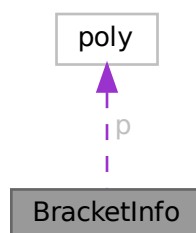
Definition at line 336 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.10 BracketInfo Struct Reference

Collaboration diagram for BracketInfo:



Public Member Functions

- [BracketInfo](#) (const std::vector< int > &pattern, int num_terms, const [poly](#) *p)
- bool [operator<](#) (const [BracketInfo](#) &rhs) const

Data Fields

- `std::vector< int >` [pattern](#)
- `int` [num_terms](#)
- `int` [dummy](#) = 0
- `const` [poly](#) * [p](#)

3.10.1 Detailed Description

Definition at line [1389](#) of file [polygcd.cc](#).

3.10.2 Constructor & Destructor Documentation

3.10.2.1 BracketInfo()

```
BracketInfo::BracketInfo (
    const std::vector< int > & pattern,
    int num_terms,
    const poly * p ) [inline]
```

Definition at line [1394](#) of file [polygcd.cc](#).

3.10.3 Member Function Documentation

3.10.3.1 operator<()

```
bool BracketInfo::operator< (
    const BracketInfo & rhs ) const [inline]
```

Definition at line [1395](#) of file [polygcd.cc](#).

3.10.4 Field Documentation

3.10.4.1 pattern

```
std::vector<int> BracketInfo::pattern
```

Definition at line [1390](#) of file [polygcd.cc](#).

3.10.4.2 num_terms

```
int BracketInfo::num_terms
```

Definition at line [1391](#) of file [polygcd.cc](#).

3.10.4.3 dummy

```
int BracketInfo::dummy = 0
```

Definition at line 1391 of file [polygcd.cc](#).

3.10.4.4 p

```
const poly* BracketInfo::p
```

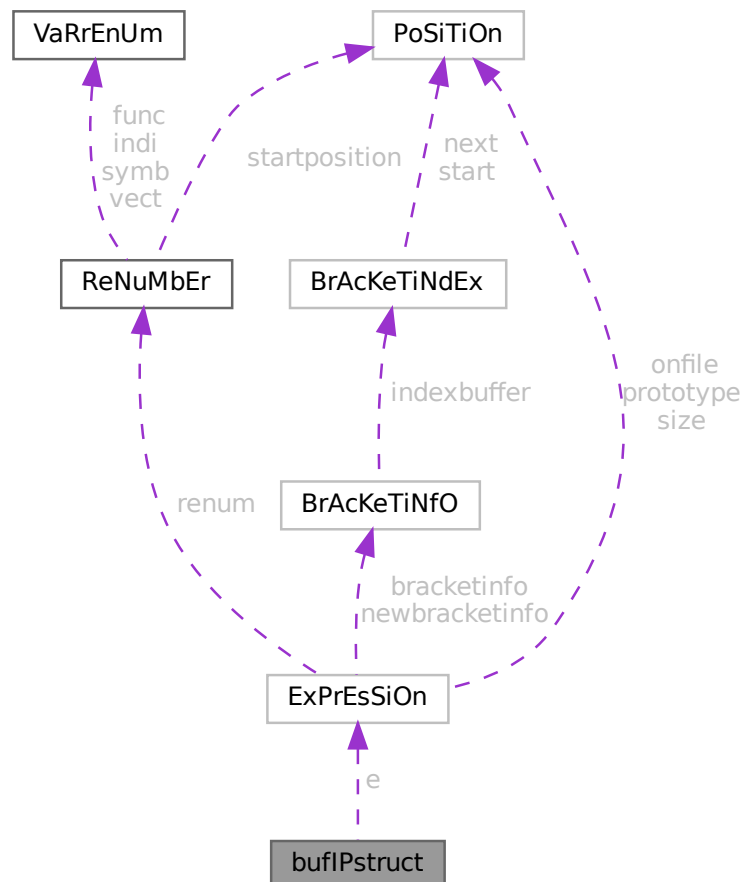
Definition at line 1392 of file [polygcd.cc](#).

The documentation for this struct was generated from the following file:

- [polygcd.cc](#)

3.11 bufIPstruct Struct Reference

Collaboration diagram for bufIPstruct:



Data Fields

- LONG `i`
- struct `ExPrEsSiOn e`

3.11.1 Detailed Description

Definition at line 3921 of file [parallel.c](#).

3.11.2 Field Documentation**3.11.2.1 `i`**

```
LONG bufIPstruct::i
```

Definition at line 3922 of file [parallel.c](#).

3.11.2.2 `e`

```
struct ExPrEsSiOn bufIPstruct::e
```

Definition at line 3923 of file [parallel.c](#).

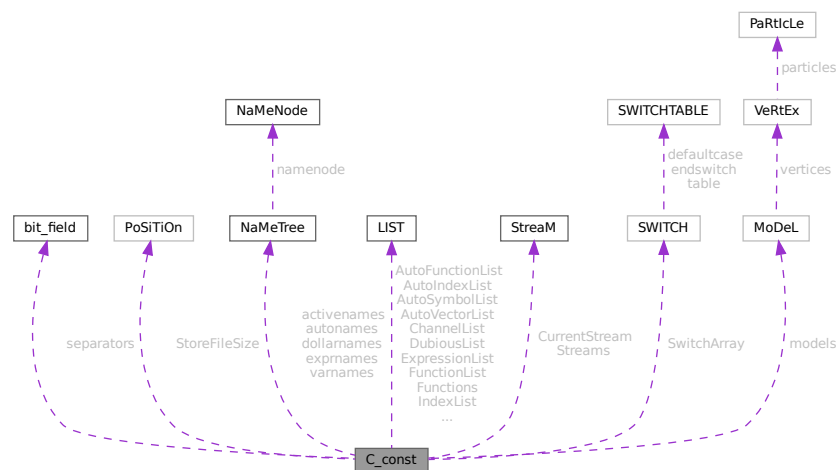
The documentation for this struct was generated from the following file:

- [parallel.c](#)

3.12 C_const Struct Reference

```
#include <structs.h>
```

Collaboration diagram for C_const:



Public Member Functions

- **PADPOSITION** (48, 8+3 *MAXNEST, 75, 48+3 *MAXNEST+MAXREPEAT, COMMERCIALSIZE+MAXFLAGS+4+sizeof(LIST) *17)

Data Fields

- [set_of_char](#) separators
- [POSITION](#) StoreFileSize
- [NAMETREE](#) * [dollarnames](#)
- [NAMETREE](#) * [exprnames](#)
- [NAMETREE](#) * [varnames](#)
- [LIST](#) ChannelList
- [LIST](#) DubiousList
- [LIST](#) FunctionList
- [LIST](#) ExpressionList
- [LIST](#) IndexList
- [LIST](#) SetElementList
- [LIST](#) SetList
- [LIST](#) SymbolList
- [LIST](#) VectorList
- [LIST](#) PotModDolList
- [LIST](#) ModOptDolList
- [LIST](#) TableBaseList
- [LIST](#) cbufList
- [LIST](#) AutoSymbolList
- [LIST](#) AutoIndexList
- [LIST](#) AutoVectorList
- [LIST](#) AutoFunctionList
- [NAMETREE](#) * [autonames](#)
- [LIST](#) * [Symbols](#)
- [LIST](#) * [Indices](#)
- [LIST](#) * [Vectors](#)
- [LIST](#) * [Functions](#)
- [NAMETREE](#) ** [activenames](#)
- [STREAM](#) * [Streams](#)
- [STREAM](#) * [CurrentStream](#)
- [SWITCH](#) * [SwitchArray](#)
- [MODEL](#) ** [models](#)
- [WORD](#) * [SwitchHeap](#)
- [LONG](#) * [termstack](#)
- [LONG](#) * [termsortstack](#)
- [UWORD](#) * [cmod](#)
- [UWORD](#) * [powmod](#)
- [UWORD](#) * [modpowers](#)
- [UWORD](#) * [halfmod](#)
- [WORD](#) * [ProtoType](#)
- [WORD](#) * [WildC](#)
- [LONG](#) * [IfHeap](#)
- [LONG](#) * [IfCount](#)
- [LONG](#) * [IfStack](#)
- [UBYTE](#) * [iBuffer](#)
- [UBYTE](#) * [iPointer](#)
- [UBYTE](#) * [iStop](#)

- UBYTE ** [LabelNames](#)
- WORD * [FixIndices](#)
- WORD * [termsumcheck](#)
- UBYTE * [WildcardNames](#)
- int * [Labels](#)
- SBYTE * [tokens](#)
- SBYTE * [toptokens](#)
- SBYTE * [endoftokens](#)
- WORD * [tokenarglevel](#)
- UWORD * [modinverses](#)
- UBYTE * [Fortran90Kind](#)
- WORD ** [MultiBracketBuf](#)
- UBYTE * [extrasym](#)
- WORD * [doloopstack](#)
- WORD * [doloopnest](#)
- char * [CheckpointRunAfter](#)
- char * [CheckpointRunBefore](#)
- WORD * [IfSumCheck](#)
- WORD * [CommutelnSet](#)
- UBYTE * [TestValue](#)
- LONG [argstack](#) [MAXNEST]
- LONG [insidestack](#) [MAXNEST]
- LONG [inexprstack](#) [MAXNEST]
- LONG [iBufferSize](#)
- LONG [TransEname](#)
- LONG [ProcessBucketSize](#)
- LONG [mProcessBucketSize](#)
- LONG [CModule](#)
- LONG [ThreadBucketSize](#)
- LONG [CheckpointStamp](#)
- LONG [CheckpointInterval](#)
- int [cbufnum](#)
- int [AutoDeclareFlag](#)
- int [NoShowInput](#)
- int [ShortStats](#)
- int [compiletype](#)
- int [firstconstindex](#)
- int [insidefirst](#)
- int [minsidefirst](#)
- int [wildflag](#)
- int [NumLabels](#)
- int [MaxLabels](#)
- int [IDefDim](#)
- int [IDefDim4](#)
- int [NumWildcardNames](#)
- int [WildcardBufferSize](#)
- int [MaxIf](#)
- int [NumStreams](#)
- int [MaxNumStreams](#)
- int [firstctypemessage](#)
- int [tablecheck](#)
- int [idooption](#)
- int [BottomLevel](#)
- int [CompileLevel](#)
- int [TokensWriteFlag](#)

- int [UnsureDollarMode](#)
- int [outsidefun](#)
- int [funpowers](#)
- int [WarnFlag](#)
- int [StatsFlag](#)
- int [NamesFlag](#)
- int [CodesFlag](#)
- int [SetupFlag](#)
- int [SortType](#)
- int [ISortType](#)
- int [ThreadStats](#)
- int [FinalStats](#)
- int [OldParallelStats](#)
- int [ThreadsFlag](#)
- int [ThreadBalancing](#)
- int [ThreadSortFileSynch](#)
- int [ProcessStats](#)
- int [BracketNormalize](#)
- int [maxtermlevel](#)
- int [dumnumflag](#)
- int [bracketindexflag](#)
- int [parallelflag](#)
- int [mparallelflag](#)
- int [inparallelflag](#)
- int [partodoflag](#)
- int [properorderflag](#)
- int [vetofilling](#)
- int [tablefilling](#)
- int [vetotablebasefill](#)
- int [exprfillwarning](#)
- int [lhdollarflag](#)
- int [NoCompress](#)
- int [IsFortran90](#)
- int [MultiBracketLevels](#)
- int [topolynomialflag](#)
- int [ffbufnum](#)
- int [OldFactArgFlag](#)
- int [MemDebugFlag](#)
- int [OldGCDflag](#)
- int [WTimeStatsFlag](#)
- int [SortReallocateFlag](#)
- int [PrintBacktraceFlag](#)
- int [FlintPolyFlag](#)
- int [HumanStatsFlag](#)
- int [doloopstacksize](#)
- int [dolooplevel](#)
- int [CheckpointFlag](#)
- int [SizeCommutelnSet](#)
- int [origin](#)
- int [vectorlikeLHS](#)
- int [nummodels](#)
- int [modelspace](#)
- int [ModelLevel](#)
- WORD [argsumcheck](#) [MAXNEST]
- WORD [insidesumcheck](#) [MAXNEST]

- WORD [inexprsumcheck](#) [MAXNEST]
- WORD [RepSumCheck](#) [MAXREPEAT]
- WORD [IUniTrace](#) [4]
- WORD [RepLevel](#)
- WORD [arglevel](#)
- WORD [insidelevel](#)
- WORD [inexprlevel](#)
- WORD [termlevel](#)
- WORD [MustTestTable](#)
- WORD [DumNum](#)
- WORD [ncmod](#)
- WORD [npowmod](#)
- WORD [modmode](#)
- WORD [nhalfmod](#)
- WORD [DirtPow](#)
- WORD [IUnitTrace](#)
- WORD [NwildC](#)
- WORD [ComDefer](#)
- WORD [CollectFun](#)
- WORD [AltCollectFun](#)
- WORD [OutputMode](#)
- WORD [Cnumpows](#)
- WORD [OutputSpaces](#)
- WORD [OutNumberType](#)
- WORD [DidClean](#)
- WORD [IfLevel](#)
- WORD [WhileLevel](#)
- WORD [SwitchLevel](#)
- WORD [SwitchInArray](#)
- WORD [MaxSwitch](#)
- WORD [LogHandle](#)
- WORD [LineLength](#)
- WORD [StoreHandle](#)
- WORD [HideLevel](#)
- WORD [IPolyFun](#)
- WORD [IPolyFunInv](#)
- WORD [IPolyFunType](#)
- WORD [IPolyFunExp](#)
- WORD [IPolyFunVar](#)
- WORD [IPolyFunPow](#)
- WORD [SymChangeFlag](#)
- WORD [CollectPercentage](#)
- WORD [ShortStatsMax](#)
- WORD [extrasymbols](#)
- WORD [PolyRatFunChanged](#)
- WORD [ToBeInFactors](#)
- WORD [InnerTest](#)
- UBYTE [Commercial](#) [COMMERCIALSIZE+2]
- UBYTE [debugFlags](#) [MAXFLAGS+2]

3.12.1 Detailed Description

The `C_const` struct is part of the global data and resides in the `ALLGLOBALS` struct A under the name #C. We see it used with the macro AC as in AC.exprnames. It contains variables that involve the compiler and objects set during compilation.

Definition at line 1697 of file `structs.h`.

3.12.2 Field Documentation

3.12.2.1 separators

`set_of_char C_const::separators`

Separators in #call and #do

Definition at line 1698 of file `structs.h`.

3.12.2.2 StoreFileSize

`POSITION C_const::StoreFileSize`

Definition at line 1699 of file `structs.h`.

3.12.2.3 dollarnames

`NAMETREE* C_const::dollarnames`

[D] Names of dollar variables

Definition at line 1700 of file `structs.h`.

3.12.2.4 exprnames

`NAMETREE* C_const::exprnames`

[D] Names of expressions

Definition at line 1701 of file `structs.h`.

3.12.2.5 varnames

`NAMETREE* C_const::varnames`

[D] Names of regular variables

Definition at line 1702 of file `structs.h`.

3.12.2.6 ChannelList

`LIST C_const::ChannelList`

Used for the #write statement. Contains [CHANNEL](#)

Definition at line [1703](#) of file [structs.h](#).

3.12.2.7 DubiousList

`LIST C_const::DubiousList`

List of dubious variables. Contains DUBIOUSV. If not empty -> no execution

Definition at line [1705](#) of file [structs.h](#).

3.12.2.8 FunctionList

`LIST C_const::FunctionList`

List of functions and properties. Contains [FUNCTIONS](#)

Definition at line [1707](#) of file [structs.h](#).

3.12.2.9 ExpressionList

`LIST C_const::ExpressionList`

List of expressions, locations etc.

Definition at line [1708](#) of file [structs.h](#).

3.12.2.10 IndexList

`LIST C_const::IndexList`

List of indices

Definition at line [1709](#) of file [structs.h](#).

3.12.2.11 SetElementList

`LIST C_const::SetElementList`

List of all elements of all sets

Definition at line [1710](#) of file [structs.h](#).

3.12.2.12 SetList

`LIST C_const::SetList`

List of the sets

Definition at line 1711 of file [structs.h](#).

3.12.2.13 SymbolList

`LIST C_const::SymbolList`

List of the symbols and their properties

Definition at line 1712 of file [structs.h](#).

3.12.2.14 VectorList

`LIST C_const::VectorList`

List of the vectors

Definition at line 1713 of file [structs.h](#).

3.12.2.15 PotModDolList

`LIST C_const::PotModDolList`

Potentially changed dollars

Definition at line 1714 of file [structs.h](#).

3.12.2.16 ModOptDolList

`LIST C_const::ModOptDolList`

Module Option Dollars list

Definition at line 1715 of file [structs.h](#).

3.12.2.17 TableBaseList

`LIST C_const::TableBaseList`

TableBase list

Definition at line 1716 of file [structs.h](#).

3.12.2.18 cbufList

`LIST C_const::cbufList`

List of compiler buffers

Definition at line 1720 of file [structs.h](#).

3.12.2.19 AutoSymbolList

`LIST C_const::AutoSymbolList`

Definition at line 1724 of file [structs.h](#).

3.12.2.20 AutoIndexList

`LIST C_const::AutoIndexList`

Definition at line 1725 of file [structs.h](#).

3.12.2.21 AutoVectorList

`LIST C_const::AutoVectorList`

Definition at line 1726 of file [structs.h](#).

3.12.2.22 AutoFunctionList

`LIST C_const::AutoFunctionList`

Definition at line 1727 of file [structs.h](#).

3.12.2.23 autonames

`NAMETREE* C_const::autonames`

[D] Names in autodeclare

Definition at line 1728 of file [structs.h](#).

3.12.2.24 Symbols

`LIST* C_const::Symbols`

Definition at line 1730 of file [structs.h](#).

3.12.2.25 Indices

`LIST* C_const::Indices`

Definition at line 1732 of file [structs.h](#).

3.12.2.26 Vectors

`LIST* C_const::Vectors`

Definition at line 1733 of file [structs.h](#).

3.12.2.27 Functions

`LIST* C_const::Functions`

Definition at line 1734 of file [structs.h](#).

3.12.2.28 activenames

`NAMETREE** C_const::activenames`

Definition at line 1735 of file [structs.h](#).

3.12.2.29 Streams

`STREAM* C_const::Streams`

(C) Pointer for AutoDeclare statement. Points either to varnames or autonames. [D] The input streams.

Definition at line 1737 of file [structs.h](#).

3.12.2.30 CurrentStream

`STREAM* C_const::CurrentStream`

(C) The current input stream. Streams are: do loop, file, prevariable. points into Streams memory.

Definition at line 1738 of file [structs.h](#).

3.12.2.31 SwitchArray

`SWITCH* C_const::SwitchArray`

Definition at line 1740 of file [structs.h](#).

3.12.2.32 models

MODEL** C_const::models

Definition at line 1741 of file [structs.h](#).

3.12.2.33 SwitchHeap

WORD* C_const::SwitchHeap

Definition at line 1742 of file [structs.h](#).

3.12.2.34 termstack

LONG* C_const::termstack

[D] Last term statement {offset}

Definition at line 1743 of file [structs.h](#).

3.12.2.35 termsortstack

LONG* C_const::termsortstack

[D] Last sort statement {offset}

Definition at line 1744 of file [structs.h](#).

3.12.2.36 cmod

UWORD* C_const::cmod

[D] Local setting of modulus. Pointer to value.

Definition at line 1745 of file [structs.h](#).

3.12.2.37 powmod

UWORD* C_const::powmod

Local setting printing as powers. Points into cmod memory

Definition at line 1746 of file [structs.h](#).

3.12.2.38 modpowers

UWORD* C_const::modpowers

[D] The conversion table for mod-> powers.

Definition at line 1747 of file [structs.h](#).

3.12.2.39 halfmod

UWORD* C_const::halfmod

Definition at line 1748 of file [structs.h](#).

3.12.2.40 ProtoType

WORD* C_const::ProtoType

Definition at line 1749 of file [structs.h](#).

3.12.2.41 WildC

WORD* C_const::WildC

Definition at line 1750 of file [structs.h](#).

3.12.2.42 IfHeap

LONG* C_const::IfHeap

[D] Keeps track of where to go in if

Definition at line 1751 of file [structs.h](#).

3.12.2.43 IfCount

LONG* C_const::IfCount

[D] Keeps track of where to go in if

Definition at line 1752 of file [structs.h](#).

3.12.2.44 IfStack

LONG* C_const::IfStack

Keeps track of where to go in if. Points into IfHeap-memory

Definition at line 1753 of file [structs.h](#).

3.12.2.45 iBuffer

```
UBYTE* C_const::iBuffer
```

[D] Compiler input buffer

Definition at line 1754 of file [structs.h](#).

3.12.2.46 iPointer

```
UBYTE* C_const::iPointer
```

Running pointer in the compiler input buffer

Definition at line 1755 of file [structs.h](#).

3.12.2.47 iStop

```
UBYTE* C_const::iStop
```

Top of iBuffer

Definition at line 1756 of file [structs.h](#).

3.12.2.48 LabelNames

```
UBYTE** C_const::LabelNames
```

[D] List of names in label statements

Definition at line 1757 of file [structs.h](#).

3.12.2.49 FixIndices

```
WORD* C_const::FixIndices
```

[D] Buffer of fixed indices

Definition at line 1758 of file [structs.h](#).

3.12.2.50 termsumcheck

```
WORD* C_const::termsumcheck
```

[D] Checking of nesting

Definition at line 1759 of file [structs.h](#).

3.12.2.51 WildcardNames

```
UBYTE* C_const::WildcardNames
```

[D] Names of ?a variables

Definition at line 1760 of file [structs.h](#).

3.12.2.52 Labels

```
int* C_const::Labels
```

Label information for during run. Pointer into LabelNames memory.

Definition at line 1761 of file [structs.h](#).

3.12.2.53 tokens

```
SBYTE* C_const::tokens
```

[D] Array with tokens for tokenizer

Definition at line 1762 of file [structs.h](#).

3.12.2.54 toptokens

```
SBYTE* C_const::toptokens
```

Top of tokens

Definition at line 1763 of file [structs.h](#).

3.12.2.55 endoftokens

```
SBYTE* C_const::endoftokens
```

End of the actual tokens

Definition at line 1764 of file [structs.h](#).

3.12.2.56 tokenarglevel

```
WORD* C_const::tokenarglevel
```

[D] Keeps track of function arguments

Definition at line 1765 of file [structs.h](#).

3.12.2.57 modinverses

UWORD* C_const::modinverses

Definition at line 1766 of file [structs.h](#).

3.12.2.58 Fortran90Kind

UBYTE* C_const::Fortran90Kind

Definition at line 1767 of file [structs.h](#).

3.12.2.59 MultiBracketBuf

WORD** C_const::MultiBracketBuf

Definition at line 1768 of file [structs.h](#).

3.12.2.60 extrasym

UBYTE* C_const::extrasym

Definition at line 1769 of file [structs.h](#).

3.12.2.61 doloopstack

WORD* C_const::doloopstack

Definition at line 1770 of file [structs.h](#).

3.12.2.62 doloopnest

WORD* C_const::doloopnest

Definition at line 1771 of file [structs.h](#).

3.12.2.63 CheckpointRunAfter

char* C_const::CheckpointRunAfter

[D] Filename of script to be executed *before* creating the snapshot. =0 if no script shall be executed.

Definition at line 1772 of file [structs.h](#).

3.12.2.64 CheckpointRunBefore

```
char* C_const::CheckpointRunBefore
```

[D] Filename of script to be executed *after* having created the snapshot. =0 if no script shall be executed.

Definition at line 1774 of file [structs.h](#).

3.12.2.65 IfSumCheck

```
WORD* C_const::IfSumCheck
```

[D] Keeps track of if-nesting

Definition at line 1776 of file [structs.h](#).

3.12.2.66 CommuteInSet

```
WORD* C_const::CommuteInSet
```

Definition at line 1777 of file [structs.h](#).

3.12.2.67 TestValue

```
UBYTE* C_const::TestValue
```

Definition at line 1778 of file [structs.h](#).

3.12.2.68 argstack

```
LONG C_const::argstack[MAXNEST]
```

Definition at line 1786 of file [structs.h](#).

3.12.2.69 insidestack

```
LONG C_const::insidestack[MAXNEST]
```

Definition at line 1787 of file [structs.h](#).

3.12.2.70 inexprstack

```
LONG C_const::inexprstack[MAXNEST]
```

Definition at line 1788 of file [structs.h](#).

3.12.2.71 iBufferSize

LONG C_const::iBufferSize

Definition at line 1789 of file [structs.h](#).

3.12.2.72 TransName

LONG C_const::TransName

Definition at line 1790 of file [structs.h](#).

3.12.2.73 ProcessBucketSize

LONG C_const::ProcessBucketSize

Definition at line 1792 of file [structs.h](#).

3.12.2.74 mProcessBucketSize

LONG C_const::mProcessBucketSize

Definition at line 1793 of file [structs.h](#).

3.12.2.75 CModule

LONG C_const::CModule

Definition at line 1794 of file [structs.h](#).

3.12.2.76 ThreadBucketSize

LONG C_const::ThreadBucketSize

Definition at line 1795 of file [structs.h](#).

3.12.2.77 CheckpointStamp

LONG C_const::CheckpointStamp

Timestamp of the last created snapshot (set to Timer(0)).

Definition at line 1796 of file [structs.h](#).

3.12.2.78 CheckpointInterval

```
LONG C_const::CheckpointInterval
```

Time interval in milliseconds for snapshots. =0 if snapshots shall be created at the end of every module.

Definition at line 1797 of file [structs.h](#).

3.12.2.79 cbufnum

```
int C_const::cbufnum
```

Current compiler buffer

Definition at line 1805 of file [structs.h](#).

3.12.2.80 AutoDeclareFlag

```
int C_const::AutoDeclareFlag
```

Definition at line 1806 of file [structs.h](#).

3.12.2.81 NoShowInput

```
int C_const::NoShowInput
```

(C) Mode of looking for names. Set to NOAUTO (=0) or WITHAUTO (=2), cf. AutoDeclare statement

Definition at line 1808 of file [structs.h](#).

3.12.2.82 ShortStats

```
int C_const::ShortStats
```

Definition at line 1809 of file [structs.h](#).

3.12.2.83 compiletype

```
int C_const::compiletype
```

Definition at line 1810 of file [structs.h](#).

3.12.2.84 firstconstindex

```
int C_const::firstconstindex
```

Definition at line 1811 of file [structs.h](#).

3.12.2.85 insidefirst

```
int C_const::insidefirst
```

Definition at line 1812 of file [structs.h](#).

3.12.2.86 minsidefirst

```
int C_const::minsidefirst
```

Definition at line 1813 of file [structs.h](#).

3.12.2.87 wildflag

```
int C_const::wildflag
```

Definition at line 1814 of file [structs.h](#).

3.12.2.88 NumLabels

```
int C_const::NumLabels
```

Definition at line 1815 of file [structs.h](#).

3.12.2.89 MaxLabels

```
int C_const::MaxLabels
```

Definition at line 1816 of file [structs.h](#).

3.12.2.90 IDefDim

```
int C_const::lDefDim
```

Definition at line 1817 of file [structs.h](#).

3.12.2.91 IDefDim4

```
int C_const::lDefDim4
```

Definition at line 1818 of file [structs.h](#).

3.12.2.92 NumWildcardNames

```
int C_const::NumWildcardNames
```

Definition at line 1819 of file [structs.h](#).

3.12.2.93 WildcardBufferSize

```
int C_const::WildcardBufferSize
```

Definition at line 1820 of file [structs.h](#).

3.12.2.94 MaxIf

```
int C_const::MaxIf
```

Definition at line 1821 of file [structs.h](#).

3.12.2.95 NumStreams

```
int C_const::NumStreams
```

Definition at line 1822 of file [structs.h](#).

3.12.2.96 MaxNumStreams

```
int C_const::MaxNumStreams
```

Definition at line 1823 of file [structs.h](#).

3.12.2.97 firstctypemessage

```
int C_const::firstctypemessage
```

Definition at line 1824 of file [structs.h](#).

3.12.2.98 tablecheck

```
int C_const::tablecheck
```

Definition at line 1825 of file [structs.h](#).

3.12.2.99 idoption

```
int C_const::idoption
```

Definition at line 1826 of file [structs.h](#).

3.12.2.100 BottomLevel

```
int C_const::BottomLevel
```

Definition at line 1827 of file [structs.h](#).

3.12.2.101 CompileLevel

```
int C_const::CompileLevel
```

Definition at line 1828 of file [structs.h](#).

3.12.2.102 TokensWriteFlag

```
int C_const::TokensWriteFlag
```

Definition at line 1829 of file [structs.h](#).

3.12.2.103 UnsureDollarMode

```
int C_const::UnsureDollarMode
```

Definition at line 1830 of file [structs.h](#).

3.12.2.104 outsidefun

```
int C_const::outsidefun
```

Definition at line 1831 of file [structs.h](#).

3.12.2.105 funpowers

```
int C_const::funpowers
```

Definition at line 1832 of file [structs.h](#).

3.12.2.106 WarnFlag

```
int C_const::WarnFlag
```

Definition at line 1833 of file [structs.h](#).

3.12.2.107 StatsFlag

```
int C_const::StatsFlag
```

Definition at line 1834 of file [structs.h](#).

3.12.2.108 NamesFlag

```
int C_const::NamesFlag
```

Definition at line 1835 of file [structs.h](#).

3.12.2.109 CodesFlag

```
int C_const::CodesFlag
```

Definition at line 1836 of file [structs.h](#).

3.12.2.110 SetupFlag

```
int C_const::SetupFlag
```

Definition at line 1837 of file [structs.h](#).

3.12.2.111 SortType

```
int C_const::SortType
```

Definition at line 1838 of file [structs.h](#).

3.12.2.112 lSortType

```
int C_const::lSortType
```

Definition at line 1839 of file [structs.h](#).

3.12.2.113 ThreadStats

```
int C_const::ThreadStats
```

Definition at line 1840 of file [structs.h](#).

3.12.2.114 FinalStats

```
int C_const::FinalStats
```

Definition at line 1841 of file [structs.h](#).

3.12.2.115 OldParallelStats

```
int C_const::OldParallelStats
```

Definition at line 1842 of file [structs.h](#).

3.12.2.116 ThreadsFlag

```
int C_const::ThreadsFlag
```

Definition at line 1843 of file [structs.h](#).

3.12.2.117 ThreadBalancing

```
int C_const::ThreadBalancing
```

Definition at line 1844 of file [structs.h](#).

3.12.2.118 ThreadSortFileSynch

```
int C_const::ThreadSortFileSynch
```

Definition at line 1845 of file [structs.h](#).

3.12.2.119 ProcessStats

```
int C_const::ProcessStats
```

Definition at line 1846 of file [structs.h](#).

3.12.2.120 BracketNormalize

```
int C_const::BracketNormalize
```

Definition at line 1847 of file [structs.h](#).

3.12.2.121 maxtermlevel

```
int C_const::maxtermlevel
```

Definition at line 1848 of file [structs.h](#).

3.12.2.122 dumnumflag

```
int C_const::dumnumflag
```

Definition at line 1849 of file [structs.h](#).

3.12.2.123 bracketindexflag

```
int C_const::bracketindexflag
```

Definition at line 1850 of file [structs.h](#).

3.12.2.124 parallelfag

```
int C_const::parallelfag
```

Definition at line 1851 of file [structs.h](#).

3.12.2.125 mparallelflag

```
int C_const::mparallelflag
```

Definition at line 1852 of file [structs.h](#).

3.12.2.126 inparallelflag

```
int C_const::inparallelflag
```

Definition at line 1853 of file [structs.h](#).

3.12.2.127 partodoflag

```
int C_const::partodoflag
```

Definition at line 1854 of file [structs.h](#).

3.12.2.128 properorderflag

```
int C_const::properorderflag
```

Definition at line 1855 of file [structs.h](#).

3.12.2.129 vetofilling

```
int C_const::vetofilling
```

Definition at line 1856 of file [structs.h](#).

3.12.2.130 tablefilling

```
int C_const::tablefilling
```

Definition at line 1857 of file [structs.h](#).

3.12.2.131 vetotablebasefill

```
int C_const::vetotablebasefill
```

Definition at line 1858 of file [structs.h](#).

3.12.2.132 exprfillwarning

```
int C_const::exprfillwarning
```

Definition at line 1859 of file [structs.h](#).

3.12.2.133 lhdollarflag

```
int C_const::lhdollarflag
```

Definition at line 1860 of file [structs.h](#).

3.12.2.134 NoCompress

```
int C_const::NoCompress
```

Definition at line 1861 of file [structs.h](#).

3.12.2.135 IsFortran90

```
int C_const::IsFortran90
```

Definition at line 1862 of file [structs.h](#).

3.12.2.136 MultiBracketLevels

```
int C_const::MultiBracketLevels
```

Definition at line 1863 of file [structs.h](#).

3.12.2.137 topolynomialflag

```
int C_const::topolynomialflag
```

Definition at line 1864 of file [structs.h](#).

3.12.2.138 ffbufnum

```
int C_const::ffbufnum
```

Definition at line 1865 of file [structs.h](#).

3.12.2.139 OldFactArgFlag

```
int C_const::OldFactArgFlag
```

Definition at line 1866 of file [structs.h](#).

3.12.2.140 MemDebugFlag

```
int C_const::MemDebugFlag
```

Definition at line 1867 of file [structs.h](#).

3.12.2.141 OldGCDflag

```
int C_const::OldGCDflag
```

Definition at line 1868 of file [structs.h](#).

3.12.2.142 WTimeStatsFlag

```
int C_const::WTimeStatsFlag
```

Definition at line 1869 of file [structs.h](#).

3.12.2.143 SortReallocateFlag

```
int C_const::SortReallocateFlag
```

Definition at line 1870 of file [structs.h](#).

3.12.2.144 PrintBacktraceFlag

```
int C_const::PrintBacktraceFlag
```

Definition at line 1874 of file [structs.h](#).

3.12.2.145 FlintPolyFlag

```
int C_const::FlintPolyFlag
```

Definition at line 1875 of file [structs.h](#).

3.12.2.146 HumanStatsFlag

```
int C_const::HumanStatsFlag
```

Definition at line 1876 of file [structs.h](#).

3.12.2.147 doloopstacksize

```
int C_const::doloopstacksize
```

Definition at line 1877 of file [structs.h](#).

3.12.2.148 dolooplevel

```
int C_const::dolooplevel
```

Definition at line 1878 of file [structs.h](#).

3.12.2.149 CheckpointFlag

```
int C_const::CheckpointFlag
```

Tells preprocessor whether checkpoint code must executed. -1 : do recovery from snapshot, set by command line option; 0 : do nothing; 1 : create snapshots, set by On checkpoint statement

Definition at line 1879 of file [structs.h](#).

3.12.2.150 SizeCommuteInSet

```
int C_const::SizeCommuteInSet
```

Definition at line 1883 of file [structs.h](#).

3.12.2.151 origin

```
int C_const::origin
```

Definition at line 1888 of file [structs.h](#).

3.12.2.152 vectorlikeLHS

```
int C_const::vectorlikeLHS
```

Definition at line 1889 of file [structs.h](#).

3.12.2.153 nummodels

```
int C_const::nummodels
```

Definition at line 1890 of file [structs.h](#).

3.12.2.154 modelspace

```
int C_const::modelspace
```

Definition at line 1891 of file [structs.h](#).

3.12.2.155 ModelLevel

```
int C_const::ModelLevel
```

Definition at line 1892 of file [structs.h](#).

3.12.2.156 argsumcheck

WORD C_const::argsumcheck[MAXNEST]

Definition at line 1893 of file [structs.h](#).

3.12.2.157 insidesumcheck

WORD C_const::insidesumcheck[MAXNEST]

Definition at line 1894 of file [structs.h](#).

3.12.2.158 inexprsumcheck

WORD C_const::inexprsumcheck[MAXNEST]

Definition at line 1895 of file [structs.h](#).

3.12.2.159 RepSumCheck

WORD C_const::RepSumCheck[MAXREPEAT]

Definition at line 1896 of file [structs.h](#).

3.12.2.160 IUniTrace

WORD C_const::lUniTrace[4]

Definition at line 1897 of file [structs.h](#).

3.12.2.161 RepLevel

WORD C_const::RepLevel

Definition at line 1898 of file [structs.h](#).

3.12.2.162 arglevel

WORD C_const::arglevel

Definition at line 1899 of file [structs.h](#).

3.12.2.163 insidelevel

WORD C_const::insidelevel

Definition at line 1900 of file [structs.h](#).

3.12.2.164 inexprlevel

WORD C_const::inexprlevel

Definition at line 1901 of file [structs.h](#).

3.12.2.165 termlevel

WORD C_const::termlevel

Definition at line 1902 of file [structs.h](#).

3.12.2.166 MustTestTable

WORD C_const::MustTestTable

Definition at line 1903 of file [structs.h](#).

3.12.2.167 DumNum

WORD C_const::DumNum

Definition at line 1904 of file [structs.h](#).

3.12.2.168 ncmmod

WORD C_const::ncmmod

Definition at line 1905 of file [structs.h](#).

3.12.2.169 npowmod

WORD C_const::npowmod

Definition at line 1906 of file [structs.h](#).

3.12.2.170 modmode

WORD C_const::modmode

Definition at line 1907 of file [structs.h](#).

3.12.2.171 nhalfmod

WORD C_const::nhalfmod

Definition at line 1908 of file [structs.h](#).

3.12.2.172 DirtPow

WORD C_const::DirtPow

Definition at line 1909 of file [structs.h](#).

3.12.2.173 lUnitTrace

WORD C_const::lUnitTrace

Definition at line 1910 of file [structs.h](#).

3.12.2.174 NwildC

WORD C_const::NwildC

Definition at line 1911 of file [structs.h](#).

3.12.2.175 ComDefer

WORD C_const::ComDefer

Definition at line 1912 of file [structs.h](#).

3.12.2.176 CollectFun

WORD C_const::CollectFun

Definition at line 1913 of file [structs.h](#).

3.12.2.177 AltCollectFun

WORD C_const::AltCollectFun

Definition at line 1914 of file [structs.h](#).

3.12.2.178 OutputMode

WORD C_const::OutputMode

Definition at line 1915 of file [structs.h](#).

3.12.2.179 Cnumpows

WORD C_const::Cnumpows

Definition at line 1916 of file [structs.h](#).

3.12.2.180 OutputSpaces

WORD C_const::OutputSpaces

Definition at line 1917 of file [structs.h](#).

3.12.2.181 OutNumberType

WORD C_const::OutNumberType

Definition at line 1918 of file [structs.h](#).

3.12.2.182 DidClean

WORD C_const::DidClean

Definition at line 1919 of file [structs.h](#).

3.12.2.183 IfLevel

WORD C_const::IfLevel

Definition at line 1920 of file [structs.h](#).

3.12.2.184 WhileLevel

WORD C_const::WhileLevel

Definition at line 1921 of file [structs.h](#).

3.12.2.185 SwitchLevel

WORD C_const::SwitchLevel

Definition at line 1922 of file [structs.h](#).

3.12.2.186 SwitchInArray

WORD C_const::SwitchInArray

Definition at line 1923 of file [structs.h](#).

3.12.2.187 MaxSwitch

WORD C_const::MaxSwitch

Definition at line 1924 of file [structs.h](#).

3.12.2.188 LogHandle

WORD C_const::LogHandle

Definition at line 1925 of file [structs.h](#).

3.12.2.189 LineLength

WORD C_const::LineLength

Definition at line 1926 of file [structs.h](#).

3.12.2.190 StoreHandle

WORD C_const::StoreHandle

Definition at line 1927 of file [structs.h](#).

3.12.2.191 HideLevel

WORD C_const::HideLevel

Definition at line 1928 of file [structs.h](#).

3.12.2.192 IPolyFun

WORD C_const::lPolyFun

Definition at line 1929 of file [structs.h](#).

3.12.2.193 IPolyFunInv

WORD C_const::lPolyFunInv

Definition at line 1930 of file [structs.h](#).

3.12.2.194 IPolyFunType

WORD C_const::lPolyFunType

Definition at line 1931 of file [structs.h](#).

3.12.2.195 IPolyFunExp

WORD C_const::lPolyFunExp

Definition at line 1932 of file [structs.h](#).

3.12.2.196 IPolyFunVar

WORD C_const::lPolyFunVar

Definition at line 1933 of file [structs.h](#).

3.12.2.197 IPolyFunPow

WORD C_const::lPolyFunPow

Definition at line 1934 of file [structs.h](#).

3.12.2.198 SymChangeFlag

WORD C_const::SymChangeFlag

Definition at line 1935 of file [structs.h](#).

3.12.2.199 CollectPercentage

WORD C_const::CollectPercentage

Definition at line 1936 of file [structs.h](#).

3.12.2.200 ShortStatsMax

WORD C_const::ShortStatsMax

Definition at line 1937 of file [structs.h](#).

3.12.2.201 extrasymbols

WORD C_const::extrasymbols

Definition at line 1938 of file [structs.h](#).

3.12.2.202 PolyRatFunChanged

WORD C_const::PolyRatFunChanged

Definition at line 1939 of file [structs.h](#).

3.12.2.203 ToBeInFactors

WORD C_const::ToBeInFactors

Definition at line 1940 of file [structs.h](#).

3.12.2.204 InnerTest

```
WORD C_const::InnerTest
```

Definition at line 1941 of file [structs.h](#).

3.12.2.205 Commercial

```
UBYTE C_const::Commercial[COMMERCIALSIZE+2]
```

Definition at line 1945 of file [structs.h](#).

3.12.2.206 debugFlags

```
UBYTE C_const::debugFlags[MAXFLAGS+2]
```

Definition at line 1946 of file [structs.h](#).

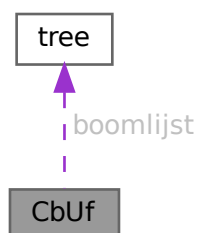
The documentation for this struct was generated from the following file:

- [structs.h](#)

3.13 CbUf Struct Reference

```
#include <structs.h>
```

Collaboration diagram for CbUf:



Data Fields

- WORD * [Buffer](#)
- WORD * [Top](#)
- WORD * [Pointer](#)
- WORD ** [lhs](#)
- WORD ** [rhs](#)
- LONG * [CanCommu](#)
- LONG * [NumTerms](#)
- WORD * [numdum](#)
- WORD * [dimension](#)
- COMPTREE * [boomlijst](#)
- LONG [BufferSize](#)
- int [numlhs](#)
- int [numrhs](#)
- int [maxlhs](#)
- int [maxrhs](#)
- int [mnumlhs](#)
- int [mnumrhs](#)
- int [numtree](#)
- int [rootnum](#)
- int [MaxTreeSize](#)

3.13.1 Detailed Description

The CBUF struct is used by the compiler. It is a compiler buffer of which since version 3.0 there can be many.

Definition at line [992](#) of file [structs.h](#).

3.13.2 Field Documentation**3.13.2.1 Buffer**

```
WORD* CbUf::Buffer
```

[D] Size in BufferSize

Definition at line [993](#) of file [structs.h](#).

Referenced by [CleanupArgCache\(\)](#), [clearcbuf\(\)](#), [DoubleCbuffer\(\)](#), [finishcbuf\(\)](#), [inicbufs\(\)](#), [PF_BroadcastCBuf\(\)](#), and [Processor\(\)](#).

3.13.2.2 Top

```
WORD* CbUf::Top
```

pointer to the end of the Buffer memory

Definition at line [994](#) of file [structs.h](#).

Referenced by [AddNtoC\(\)](#), [AddNtoL\(\)](#), [DoubleCbuffer\(\)](#), [finishcbuf\(\)](#), [inicbufs\(\)](#), and [PF_BroadcastCBuf\(\)](#).

3.13.2.3 Pointer

```
WORD* CbUf::Pointer
```

pointer into the Buffer memory

Definition at line 995 of file [structs.h](#).

Referenced by [AddLHS\(\)](#), [AddNtoC\(\)](#), [AddNtoL\(\)](#), [AddRHS\(\)](#), [CleanupArgCache\(\)](#), [clearcbuf\(\)](#), [DoubleCbuffer\(\)](#), [finishcbuf\(\)](#), [Generator\(\)](#), [inicbufs\(\)](#), [PF_BroadcastCBuf\(\)](#), and [Processor\(\)](#).

3.13.2.4 lhs

```
WORD** CbUf::lhs
```

[D] Size in maxlhs. list of pointers into Buffer.

Definition at line 996 of file [structs.h](#).

Referenced by [AddLHS\(\)](#), [DoubleCbuffer\(\)](#), [finishcbuf\(\)](#), [Generator\(\)](#), [inicbufs\(\)](#), [PF_BroadcastCBuf\(\)](#), [Processor\(\)](#), and [TestMatch\(\)](#).

3.13.2.5 rhs

```
WORD** CbUf::rhs
```

[D] Size in maxrhs. list of pointers into Buffer.

Definition at line 997 of file [structs.h](#).

Referenced by [AddRHS\(\)](#), [CleanupArgCache\(\)](#), [clearcbuf\(\)](#), [DoubleCbuffer\(\)](#), [finishcbuf\(\)](#), [Generator\(\)](#), [inicbufs\(\)](#), [IniFbuffer\(\)](#), [PF_BroadcastCBuf\(\)](#), and [Processor\(\)](#).

3.13.2.6 CanCommu

```
LONG* CbUf::CanCommu
```

points into rhs memory behind WORD* area.

Definition at line 998 of file [structs.h](#).

Referenced by [AddRHS\(\)](#), [finishcbuf\(\)](#), [inicbufs\(\)](#), [IniFbuffer\(\)](#), and [PF_BroadcastCBuf\(\)](#).

3.13.2.7 NumTerms

```
LONG* CbUf::NumTerms
```

points into rhs memory behind CanCommu area

Definition at line 999 of file [structs.h](#).

Referenced by [AddRHS\(\)](#), [finishcbuf\(\)](#), [inicbufs\(\)](#), [IniFbuffer\(\)](#), and [PF_BroadcastCBuf\(\)](#).

3.13.2.8 numdum

```
WORD* CbUf::numdum
```

points into rhs memory behind NumTerms

Definition at line 1000 of file [structs.h](#).

Referenced by [AddRHS\(\)](#), [inicbufs\(\)](#), [IniFbuffer\(\)](#), and [PF_BroadcastCBuf\(\)](#).

3.13.2.9 dimension

```
WORD* CbUf::dimension
```

points into rhs memory behind numdum

Definition at line 1001 of file [structs.h](#).

Referenced by [AddRHS\(\)](#), [inicbufs\(\)](#), [IniFbuffer\(\)](#), and [PF_BroadcastCBuf\(\)](#).

3.13.2.10 boomlijst

```
COMPTREE* CbUf::boomlijst
```

[D] Number elements in MaxTreeSize

Definition at line 1002 of file [structs.h](#).

Referenced by [CleanupArgCache\(\)](#), [clearcbuf\(\)](#), [finishcbuf\(\)](#), [inicbufs\(\)](#), [IniFbuffer\(\)](#), and [PF_BroadcastCBuf\(\)](#).

3.13.2.11 BufferSize

```
LONG CbUf::BufferSize
```

Number of allocated WORD's in Buffer

Definition at line 1003 of file [structs.h](#).

Referenced by [DoubleCbuffer\(\)](#), [finishcbuf\(\)](#), [inicbufs\(\)](#), and [PF_BroadcastCBuf\(\)](#).

3.13.2.12 numlhs

```
int CbUf::numlhs
```

Definition at line 1004 of file [structs.h](#).

3.13.2.13 numrhs

```
int CbUf::numrhs
```

Definition at line 1005 of file [structs.h](#).

3.13.2.14 maxlhs

```
int CbUf::maxlhs
```

Definition at line 1006 of file [structs.h](#).

3.13.2.15 maxrhs

```
int CbUf::maxrhs
```

Definition at line 1007 of file [structs.h](#).

3.13.2.16 mnumlhs

```
int CbUf::mnumlhs
```

Definition at line 1008 of file [structs.h](#).

3.13.2.17 mnumrhs

```
int CbUf::mnumrhs
```

Definition at line 1009 of file [structs.h](#).

3.13.2.18 numtree

```
int CbUf::numtree
```

Definition at line 1010 of file [structs.h](#).

3.13.2.19 rootnum

```
int CbUf::rootnum
```

Definition at line 1011 of file [structs.h](#).

3.13.2.20 MaxTreeSize

```
int CbUf::MaxTreeSize
```

Definition at line 1012 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.14 ChAnNeL Struct Reference

```
#include <structs.h>
```

Data Fields

- char * [name](#)
- int [handle](#)

3.14.1 Detailed Description

When we read input from text files we have to remember not only their handle but also their name. This is needed for error messages. Hence we call such a file a channel and reserve a struct of type [CHANNEL](#) to allow to lay this link.

Definition at line [1023](#) of file [structs.h](#).

3.14.2 Field Documentation

3.14.2.1 name

```
char* ChAnNeL::name
```

[D] Name of the channel

Definition at line [1024](#) of file [structs.h](#).

3.14.2.2 handle

```
int ChAnNeL::handle
```

File handle

Definition at line [1025](#) of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.15 COST Struct Reference

Data Fields

- LONG [add](#)
- LONG [mul](#)
- LONG [div](#)
- LONG [pow](#)

3.15.1 Detailed Description

Definition at line [1294](#) of file [structs.h](#).

3.15.2 Field Documentation

3.15.2.1 add

```
LONG COST::add
```

Definition at line [1295](#) of file [structs.h](#).

3.15.2.2 mul

```
LONG COST::mul
```

Definition at line [1296](#) of file [structs.h](#).

3.15.2.3 div

```
LONG COST::div
```

Definition at line [1297](#) of file [structs.h](#).

3.15.2.4 pow

```
LONG COST::pow
```

Definition at line [1298](#) of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.16 CSEEq Struct Reference

Public Member Functions

- bool [operator\(\)](#) (const vector< WORD > &lhs, const vector< WORD > &rhs) const

3.16.1 Detailed Description

Definition at line [1060](#) of file [optimize.cc](#).

3.16.2 Member Function Documentation

3.16.2.1 operator()

```
bool CSEEq::operator() (
    const vector< WORD > & lhs,
    const vector< WORD > & rhs ) const [inline]
```

Definition at line 1061 of file [optimize.cc](#).

The documentation for this struct was generated from the following file:

- [optimize.cc](#)

3.17 CSEHash Struct Reference

Public Member Functions

- `size_t operator()` (const vector< WORD > &n) const

3.17.1 Detailed Description

Definition at line 1054 of file [optimize.cc](#).

3.17.2 Member Function Documentation

3.17.2.1 operator()

```
size_t CSEHash::operator() (
    const vector< WORD > & n ) const [inline]
```

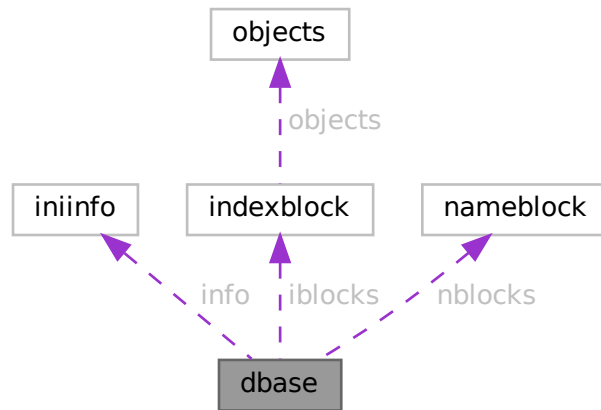
Definition at line 1055 of file [optimize.cc](#).

The documentation for this struct was generated from the following file:

- [optimize.cc](#)

3.18 dbase Struct Reference

Collaboration diagram for dbase:



Data Fields

- `INIINFO` `info`
- `MLONG` `mode`
- `MLONG` `tablenameessize`
- `MLONG` `topnumber`
- `MLONG` `tablenameefill`
- `MLONG` `rwmode`
- `INDEXBLOCK` `** iblocks`
- `NAMESBLOCK` `** nblocks`
- `FILE` `* handle`
- `char *` `name`
- `char *` `fullname`
- `char *` `tablenamees`

3.18.1 Detailed Description

Definition at line 121 of file `minos.h`.

3.18.2 Field Documentation

3.18.2.1 `info`

`INIINFO` `dbase::info`

Definition at line 122 of file `minos.h`.

3.18.2.2 mode

MLONG dbase::mode

Definition at line 123 of file [minos.h](#).

3.18.2.3 tablenamessize

MLONG dbase::tablenamessize

Definition at line 124 of file [minos.h](#).

3.18.2.4 topnumber

MLONG dbase::topnumber

Definition at line 125 of file [minos.h](#).

3.18.2.5 tablenameefill

MLONG dbase::tablenameefill

Definition at line 126 of file [minos.h](#).

3.18.2.6 rwmode

MLONG dbase::rwmode

Definition at line 127 of file [minos.h](#).

3.18.2.7 iblocks

INDEXBLOCK** dbase::iblocks

Definition at line 128 of file [minos.h](#).

3.18.2.8 nblocks

NAMESBLOCK** dbase::nblocks

Definition at line 129 of file [minos.h](#).

3.18.2.9 handle

FILE* dbase::handle

Definition at line 130 of file [minos.h](#).

3.18.2.10 name

```
char* dbase::name
```

Definition at line 131 of file [minos.h](#).

3.18.2.11 fullname

```
char* dbase::fullname
```

Definition at line 132 of file [minos.h](#).

3.18.2.12 tablenames

```
char* dbase::tablenames
```

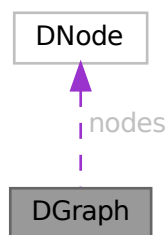
Definition at line 133 of file [minos.h](#).

The documentation for this struct was generated from the following file:

- [minos.h](#)

3.19 DGraph Struct Reference

Collaboration diagram for DGraph:



Data Fields

- int [nnodes](#)
- int [nedges](#)
- [DNode](#) * [nodes](#)
- [DEdge](#) * [edges](#)

3.19.1 Detailed Description

Definition at line 199 of file [grccparam.h](#).

3.19.2 Field Documentation

3.19.2.1 nnodes

```
int DGraph::nnodes
```

Definition at line 200 of file [grccparam.h](#).

3.19.2.2 nedges

```
int DGraph::nedges
```

Definition at line 201 of file [grccparam.h](#).

3.19.2.3 nodes

```
DNode* DGraph::nodes
```

Definition at line 202 of file [grccparam.h](#).

3.19.2.4 edges

```
DEdge* DGraph::edges
```

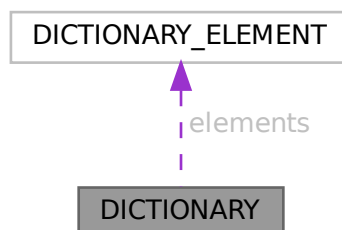
Definition at line 203 of file [grccparam.h](#).

The documentation for this struct was generated from the following file:

- [grccparam.h](#)

3.20 DICTIONARY Struct Reference

Collaboration diagram for DICTIONARY:



Data Fields

- [DICTIONARY_ELEMENT](#) ** [elements](#)
- UBYTE * [name](#)
- int [sizeelements](#)
- int [numelements](#)
- int [numbers](#)
- int [variables](#)
- int [characters](#)
- int [funwith](#)
- int [gnumelements](#)
- int [ranges](#)

3.20.1 Detailed Description

Definition at line [1361](#) of file [structs.h](#).

3.20.2 Field Documentation

3.20.2.1 elements

```
DICTIONARY_ELEMENT** DICTIONARY::elements
```

Definition at line [1362](#) of file [structs.h](#).

3.20.2.2 name

```
UBYTE* DICTIONARY::name
```

Definition at line [1363](#) of file [structs.h](#).

3.20.2.3 sizeelements

```
int DICTIONARY::sizeelements
```

Definition at line [1364](#) of file [structs.h](#).

3.20.2.4 numelements

```
int DICTIONARY::numelements
```

Definition at line [1365](#) of file [structs.h](#).

3.20.2.5 numbers

```
int DICTIONARY::numbers
```

Definition at line [1366](#) of file [structs.h](#).

3.20.2.6 variables

```
int DICTIONARY::variables
```

Definition at line 1367 of file [structs.h](#).

3.20.2.7 characters

```
int DICTIONARY::characters
```

Definition at line 1368 of file [structs.h](#).

3.20.2.8 funwith

```
int DICTIONARY::funwith
```

Definition at line 1369 of file [structs.h](#).

3.20.2.9 gnumelements

```
int DICTIONARY::gnumelements
```

Definition at line 1370 of file [structs.h](#).

3.20.2.10 ranges

```
int DICTIONARY::ranges
```

Definition at line 1371 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.21 DICTIONARY_ELEMENT Struct Reference

Data Fields

- WORD * [lhs](#)
- WORD * [rhs](#)
- int [type](#)
- int [size](#)

3.21.1 Detailed Description

Definition at line 1354 of file [structs.h](#).

3.21.2 Field Documentation

3.21.2.1 lhs

```
WORD* DICTIONARY_ELEMENT::lhs
```

Definition at line 1355 of file [structs.h](#).

3.21.2.2 rhs

```
WORD* DICTIONARY_ELEMENT::rhs
```

Definition at line 1356 of file [structs.h](#).

3.21.2.3 type

```
int DICTIONARY_ELEMENT::type
```

Definition at line 1357 of file [structs.h](#).

3.21.2.4 size

```
int DICTIONARY_ELEMENT::size
```

Definition at line 1358 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.22 DiStRiBuTe Struct Reference

```
#include <structs.h>
```

Data Fields

- WORD * [obj1](#)
- WORD * [obj2](#)
- WORD * [out](#)
- WORD [sign](#)
- WORD [n1](#)
- WORD [n2](#)
- WORD [n](#)
- WORD [cycle](#) [MAXMATCH]

3.22.1 Detailed Description

The struct DISTRIBUTE is used to help the pattern matcher when matching antisymmetric tensors.

Definition at line 1103 of file [structs.h](#).

3.22.2 Field Documentation

3.22.2.1 obj1

```
WORD* DiStRiBuTe::obj1
```

Definition at line 1104 of file [structs.h](#).

3.22.2.2 obj2

```
WORD* DiStRiBuTe::obj2
```

Definition at line 1105 of file [structs.h](#).

3.22.2.3 out

```
WORD* DiStRiBuTe::out
```

Definition at line 1106 of file [structs.h](#).

3.22.2.4 sign

```
WORD DiStRiBuTe::sign
```

Definition at line 1107 of file [structs.h](#).

3.22.2.5 n1

```
WORD DiStRiBuTe::n1
```

Definition at line 1108 of file [structs.h](#).

3.22.2.6 n2

```
WORD DiStRiBuTe::n2
```

Definition at line 1109 of file [structs.h](#).

3.22.2.7 n

```
WORD DiStRiBuTe::n
```

Definition at line 1110 of file [structs.h](#).

3.22.2.8 cycle

```
WORD DiStRiBuTe::cycle[MAXMATCH]
```

Definition at line 1111 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.23 DNode Struct Reference

Data Fields

- int [extloop](#)
- int [intrct](#)

3.23.1 Detailed Description

Definition at line 192 of file [grccparam.h](#).

3.23.2 Field Documentation

3.23.2.1 extloop

```
int DNode::extloop
```

Definition at line 193 of file [grccparam.h](#).

3.23.2.2 intrct

```
int DNode::intrct
```

Definition at line 194 of file [grccparam.h](#).

The documentation for this struct was generated from the following file:

- [grccparam.h](#)

3.24 dollar_buf Struct Reference

3.24.1 Detailed Description

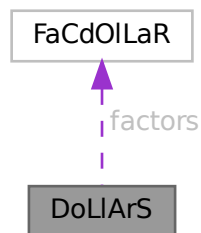
Definition at line 2479 of file [parallel.c](#).

The documentation for this struct was generated from the following file:

- [parallel.c](#)

3.25 DoLIaRS Struct Reference

Collaboration diagram for DoLIaRS:



Data Fields

- WORD * [where](#)
- [FACDOLLAR](#) * [factors](#)
- LONG [size](#)
- LONG [name](#)
- WORD [type](#)
- WORD [node](#)
- WORD [index](#)
- WORD [zero](#)
- WORD [numdummies](#)
- WORD [nfactors](#)

3.25.1 Detailed Description

Definition at line 530 of file [structs.h](#).

3.25.2 Field Documentation

3.25.2.1 where

WORD* DoLLArS::where

Definition at line 531 of file [structs.h](#).

3.25.2.2 factors

FACDOLLAR* DoLLArS::factors

Definition at line 532 of file [structs.h](#).

3.25.2.3 size

LONG DoLLArS::size

Definition at line 537 of file [structs.h](#).

3.25.2.4 name

LONG DoLLArS::name

Definition at line 538 of file [structs.h](#).

3.25.2.5 type

WORD DoLLArS::type

Definition at line 539 of file [structs.h](#).

3.25.2.6 node

WORD DoLLArS::node

Definition at line 540 of file [structs.h](#).

3.25.2.7 index

WORD DoLLArS::index

Definition at line 541 of file [structs.h](#).

3.25.2.8 zero

```
WORD DoLlArS::zero
```

Definition at line 542 of file [structs.h](#).

3.25.2.9 numdummies

```
WORD DoLlArS::numdummies
```

Definition at line 543 of file [structs.h](#).

3.25.2.10 nfactors

```
WORD DoLlArS::nfactors
```

Definition at line 544 of file [structs.h](#).

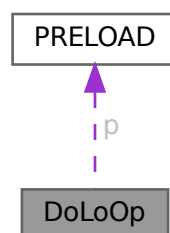
The documentation for this struct was generated from the following file:

- [structs.h](#)

3.26 DoLoOp Struct Reference

```
#include <structs.h>
```

Collaboration diagram for DoLoOp:



Data Fields

- [PRELOAD](#) [p](#)
- UBYTE * [name](#)
- UBYTE * [vars](#)
- UBYTE * [contents](#)
- UBYTE * [dollarname](#)
- LONG [startlinenumber](#)
- LONG [firstnum](#)
- LONG [lastnum](#)
- LONG [incnum](#)
- int [type](#)
- int [NoShowInput](#)
- int [errorsinloop](#)
- int [firstloopcall](#)
- WORD [firstdollar](#)
- WORD [lastdollar](#)
- WORD [incdollar](#)
- WORD [NumPreTypes](#)
- WORD [PrefLevel](#)
- WORD [PreSwitchLevel](#)

3.26.1 Detailed Description

Each preprocessor do loop has a struct of type DOLOOP to keep track of all relevant parameters like where the beginning of the loop is, what the boundaries, increment and value of the loop parameter are, etc. Also we keep the whole loop inside a buffer of type [PRELOAD](#)

Definition at line [897](#) of file [structs.h](#).

3.26.2 Field Documentation

3.26.2.1 [p](#)

[PRELOAD](#) [DoLoOp](#)::[p](#)

size, name and buffer

Definition at line [898](#) of file [structs.h](#).

3.26.2.2 [name](#)

UBYTE* [DoLoOp](#)::[name](#)

pointer into [PRELOAD](#) buffer

Definition at line [899](#) of file [structs.h](#).

3.26.2.3 vars

```
UBYTE* DoLoOp::vars
```

Definition at line 900 of file [structs.h](#).

3.26.2.4 contents

```
UBYTE* DoLoOp::contents
```

Definition at line 901 of file [structs.h](#).

3.26.2.5 dollarname

```
UBYTE* DoLoOp::dollarname
```

For loop over terms in expression. Allocated with Malloc1()

Definition at line 902 of file [structs.h](#).

3.26.2.6 startlinenumber

```
LONG DoLoOp::startlinenumber
```

Definition at line 903 of file [structs.h](#).

3.26.2.7 firstnum

```
LONG DoLoOp::firstnum
```

Definition at line 904 of file [structs.h](#).

3.26.2.8 lastnum

```
LONG DoLoOp::lastnum
```

Definition at line 905 of file [structs.h](#).

3.26.2.9 incnum

```
LONG DoLoOp::incnum
```

Definition at line 906 of file [structs.h](#).

3.26.2.10 type

```
int DoLoOp::type
```

Definition at line 907 of file [structs.h](#).

3.26.2.11 NoShowInput

```
int DoLoOp::NoShowInput
```

Definition at line 908 of file [structs.h](#).

3.26.2.12 errorsinloop

```
int DoLoOp::errorsinloop
```

Definition at line 909 of file [structs.h](#).

3.26.2.13 firstloopcall

```
int DoLoOp::firstloopcall
```

Definition at line 910 of file [structs.h](#).

3.26.2.14 firstdollar

```
WORD DoLoOp::firstdollar
```

Definition at line 911 of file [structs.h](#).

3.26.2.15 lastdollar

```
WORD DoLoOp::lastdollar
```

Definition at line 912 of file [structs.h](#).

3.26.2.16 incdollar

```
WORD DoLoOp::incdollar
```

Definition at line 913 of file [structs.h](#).

3.26.2.17 NumPreTypes

```
WORD DoLoOp::NumPreTypes
```

Definition at line 914 of file [structs.h](#).

3.26.2.18 PreIfLevel

WORD DoLoOp::PreIfLevel

Definition at line 915 of file [structs.h](#).

3.26.2.19 PreSwitchLevel

WORD DoLoOp::PreSwitchLevel

Definition at line 916 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.27 DuBiOuS Struct Reference

Data Fields

- LONG [name](#)
- WORD [node](#)
- WORD [dummy](#)

3.27.1 Detailed Description

Definition at line 515 of file [structs.h](#).

3.27.2 Field Documentation

3.27.2.1 name

LONG DuBiOuS::name

Definition at line 516 of file [structs.h](#).

3.27.2.2 node

WORD DuBiOuS::node

Definition at line 517 of file [structs.h](#).

3.27.2.3 dummy

WORD DuBiOuS::dummy

Definition at line 518 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.28 ECand Class Reference

Public Member Functions

- [ECand](#) (int dt, int nplist, int *plst)
- void [prECand](#) (const char *msg)

Data Fields

- Bool [det](#)
- int [nplist](#)
- int [plist](#) [GRCC_MAXMPARTICLES2]

3.28.1 Detailed Description

Definition at line 1062 of file [grcc.h](#).

3.28.2 Constructor & Destructor Documentation

3.28.2.1 ECand()

```
ECand::ECand (  
    int dt,  
    int nplist,  
    int * plst )
```

Definition at line 7325 of file [grcc.cc](#).

3.28.2.2 ~ECand()

```
ECand::~~ECand (  
    void )
```

Definition at line 7345 of file [grcc.cc](#).

Public Member Functions

- [EEdge](#) ([EGraph](#) *egrph, int nedges, int nloops)
- void [copy](#) ([EEdge](#) *ee)
- void [setId](#) ([EGraph](#) *egrph, const int eid)
- void [print](#) (void)
- void [setType](#) (int typ)
- void [setLMom](#) (int k, int dir)
- void [setEMom](#) (int nedges, int *extn, int dir)

Data Fields

- [EGraph](#) * [egraph](#)
- int [id](#)
- int [ext](#)
- int [ptcl](#)
- int [deleted](#)
- int [nodes](#) [2]
- int [nlegs](#) [2]
- int * [emom](#)
- int * [lmom](#)
- int * [extMom](#)
- Bool [cut](#)
- int [visited](#)
- int [conid](#)
- int [edtype](#)
- int [opicom](#)
- int [dir](#)

3.29.1 Detailed Description

Definition at line [538](#) of file [grcc.h](#).

3.29.2 Constructor & Destructor Documentation

3.29.2.1 EEdge() [1/2]

```
EEdge::EEdge (
    void )
```

Definition at line [5640](#) of file [grcc.cc](#).

3.29.2.2 EEdge() [2/2]

```
EEdge::EEdge (
    EGraph * egrph,
    int nedges,
    int nloops )
```

Definition at line [5659](#) of file [grcc.cc](#).

3.29.2.3 ~EEdge()

```
EEdge::~~EEdge (
    void )
```

Definition at line [5678](#) of file [grcc.cc](#).

3.29.3 Member Function Documentation

3.29.3.1 copy()

```
void EEdge::copy (
    EEdge * ee )
```

Definition at line [5689](#) of file [grcc.cc](#).

3.29.3.2 setId()

```
void EEdge::setId (
    EGraph * egrph,
    const int eid )
```

Definition at line [5701](#) of file [grcc.cc](#).

3.29.3.3 print()

```
void EEdge::print (
    void )
```

Definition at line [5737](#) of file [grcc.cc](#).

3.29.3.4 setType()

```
void EEdge::setType (
    int typ )
```

Definition at line [5708](#) of file [grcc.cc](#).

3.29.3.5 setLMom()

```
void EEdge::setLMom (
    int k,
    int dir )
```

Definition at line [5731](#) of file [grcc.cc](#).

3.29.3.6 setEMom()

```
void EEdge::setEMom (
    int nedges,
    int * extn,
    int dir )
```

Definition at line [5719](#) of file [grcc.cc](#).

3.29.4 Field Documentation

3.29.4.1 egraph

```
EGraph* EEdge::egraph
```

Definition at line [541](#) of file [grcc.h](#).

3.29.4.2 id

```
int EEdge::id
```

Definition at line [543](#) of file [grcc.h](#).

3.29.4.3 ext

```
int EEdge::ext
```

Definition at line [544](#) of file [grcc.h](#).

3.29.4.4 ptcl

```
int EEdge::ptcl
```

Definition at line [545](#) of file [grcc.h](#).

3.29.4.5 deleted

```
int EEdge::deleted
```

Definition at line [546](#) of file [grcc.h](#).

3.29.4.6 nodes

```
int EEdge::nodes[2]
```

Definition at line [547](#) of file [grcc.h](#).

3.29.4.7 nlegs

```
int EEdge::nlegs[2]
```

Definition at line 548 of file [grcc.h](#).

3.29.4.8 emom

```
int* EEdge::emom
```

Definition at line 551 of file [grcc.h](#).

3.29.4.9 lmom

```
int* EEdge::lmom
```

Definition at line 552 of file [grcc.h](#).

3.29.4.10 extMom

```
int* EEdge::extMom
```

Definition at line 553 of file [grcc.h](#).

3.29.4.11 cut

```
Bool EEdge::cut
```

Definition at line 555 of file [grcc.h](#).

3.29.4.12 visited

```
int EEdge::visited
```

Definition at line 556 of file [grcc.h](#).

3.29.4.13 conid

```
int EEdge::conid
```

Definition at line 557 of file [grcc.h](#).

3.29.4.14 edtype

```
int EEdge::edtype
```

Definition at line 558 of file [grcc.h](#).

3.29.4.15 opicomp

```
int EEdge::opicomp
```

Definition at line 559 of file [grcc.h](#).

3.29.4.16 dir

```
int EEdge::dir
```

Definition at line 560 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.30 EFLine Class Reference

Public Member Functions

- void [print](#) (const char *msg)

Data Fields

- int [elist](#) [GRCC_MAXNODES]
- FLType [ftype](#)
- int [fkind](#)
- int [nlist](#)

3.30.1 Detailed Description

Definition at line 583 of file [grcc.h](#).

3.30.2 Constructor & Destructor Documentation

3.30.2.1 EFLine()

```
EFLine::EFLine (  
    void )
```

Definition at line 5779 of file [grcc.cc](#).

3.30.3 Member Function Documentation

3.30.3.1 print()

```
void EFLine::print (
    const char * msg )
```

Definition at line 5787 of file [grcc.cc](#).

3.30.4 Field Documentation

3.30.4.1 elist

```
int EFLine::elist[GRCC_MAXNODES]
```

Definition at line 585 of file [grcc.h](#).

3.30.4.2 ftype

```
FLType EFLine::ftype
```

Definition at line 586 of file [grcc.h](#).

3.30.4.3 fkind

```
int EFLine::fkind
```

Definition at line 587 of file [grcc.h](#).

3.30.4.4 nlist

```
int EFLine::nlist
```

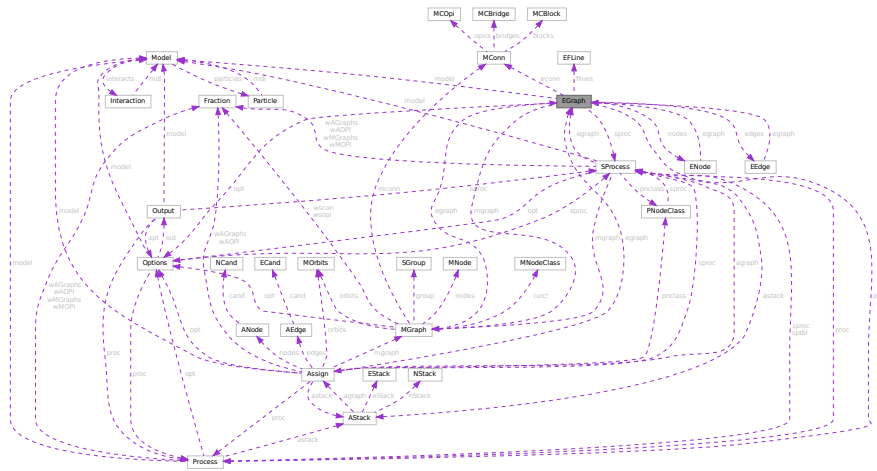
Definition at line 588 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.31 EGraph Class Reference

Collaboration diagram for EGraph:



Public Member Functions

- [EGraph](#) (int nnodes, int nedges, int mxdeg)
- void [copy](#) ([EGraph](#) *eg)
- void [print](#) (void)
- void [fromDGraph](#) ([DGraph](#) *dg)
- void [fromMGraph](#) ([MGraph](#) *mgraph)
- Bool [isOptE](#) (void)
- [ENode](#) * [setExtern](#) (int n0, int pt, int ndtp)
- Bool [isExternal](#) (int nd)
- Bool [isFermion](#) (int nd)
- void [setExtLoop](#) (int nd, int val)
- void [endSetExtLoop](#) (void)
- int [connComp](#) (void)
- int [connVisit](#) (int nd, int ncc)
- void [biconnE](#) (void)
- void [biinitE](#) (void)
- void [bisearchE](#) (int nd, int *extlst, int *intlst, int *opiext, int *opiloop)
- int [findRoot](#) (void)
- int [dirEdge](#) (int n, int e)
- void [extMomConsv](#) (void)
- int [cmpMom](#) (int *lm0, int *em0, int *lm1, int *em1)
- int [groupLMom](#) (int *grp, int *ed2gr)
- void [chkMomConsv](#) (void)
- void [prFLines](#) (void)
- void [getFLines](#) (void)
- int [fltrace](#) (int fk, int nd0, int *fl)
- void [addFLine](#) (const FLType ft, int fk, int nfl, int *fl)
- int [legParticle](#) (int ed, int lg)

Data Fields

- [Options](#) * [opt](#)
- [Model](#) * [model](#)
- [Process](#) * [proc](#)
- [SProcess](#) * [sproc](#)
- [MGraph](#) * [mgraph](#)
- [MConn](#) * [econn](#)
- [ENode](#) ** [nodes](#)
- [EEdge](#) ** [edges](#)
- [BigInt](#) [mld](#)
- [BigInt](#) [ald](#)
- [BigInt](#) [sld](#)
- [BigInt](#) [gSubld](#)
- [Bool](#) [assigned](#)
- [int](#) [fsign](#)
- [BigInt](#) [nsym](#)
- [BigInt](#) [esym](#)
- [BigInt](#) [nsym1](#)
- [BigInt](#) [extperm](#)
- [BigInt](#) [multp](#)
- [int](#) [pld](#)
- [int](#) [sNodes](#)
- [int](#) [sEdges](#)
- [int](#) [sMaxdeg](#)
- [int](#) [sLoops](#)
- [int](#) [nNodes](#)
- [int](#) [nEdges](#)
- [int](#) [nExtern](#)
- [int](#) [maxdeg](#)
- [int](#) [nLoops](#)
- [int](#) [totalc](#)
- [int](#) [nopicomp](#)
- [int](#) [opi2plp](#)
- [int](#) [nopi2p](#)
- [int](#) [nadj2ptv](#)
- [int](#) [DUMMYPADDING](#)
- [int](#) * [bidef](#)
- [int](#) * [bilow](#)
- [int](#) * [extMom](#)
- [int](#) [bconn](#)
- [int](#) [bicount](#)
- [int](#) [loopm](#)
- [int](#) [opiCount](#)
- [EFLine](#) * [flines](#) [GRCC_MAXFLINES]
- [int](#) [nflines](#)
- [int](#) [DUMMYPADDING1](#)

3.31.1 Detailed Description

Definition at line 595 of file [grcc.h](#).

3.31.2 Constructor & Destructor Documentation

3.31.2.1 EGraph()

```
EGraph::EGraph (
    int  nnodes,
    int  nedges,
    int  mxdeg )
```

Definition at line [5812](#) of file [grcc.cc](#).

3.31.2.2 ~EGraph()

```
EGraph::~~EGraph (
    void )
```

Definition at line [5873](#) of file [grcc.cc](#).

3.31.3 Member Function Documentation

3.31.3.1 copy()

```
void EGraph::copy (
    EGraph * eg )
```

Definition at line [5904](#) of file [grcc.cc](#).

3.31.3.2 print()

```
void EGraph::print (
    void )
```

Definition at line [6269](#) of file [grcc.cc](#).

3.31.3.3 fromDGraph()

```
void EGraph::fromDGraph (
    DGraph * dg )
```

Definition at line [5969](#) of file [grcc.cc](#).

3.31.3.4 fromMGraph()

```
void EGraph::fromMGraph (
    MGraph * mgraph )
```

Definition at line [6074](#) of file [grcc.cc](#).

3.31.3.5 isOptE()

```
Bool EGraph::isOptE (
    void )
```

Definition at line 6440 of file [grcc.cc](#).

3.31.3.6 setExtern()

```
ENode * EGraph::setExtern (
    int n0,
    int pt,
    int ndtp )
```

Definition at line 6222 of file [grcc.cc](#).

3.31.3.7 isExternal()

```
Bool EGraph::isExternal (
    int nd ) [inline]
```

Definition at line 667 of file [grcc.h](#).

3.31.3.8 isFermion()

```
int EGraph::isFermion (
    int nd )
```

Definition at line 7056 of file [grcc.cc](#).

3.31.3.9 setExtLoop()

```
void EGraph::setExtLoop (
    int nd,
    int val )
```

Definition at line 5946 of file [grcc.cc](#).

3.31.3.10 endSetExtLoop()

```
void EGraph::endSetExtLoop (
    void )
```

Definition at line 5954 of file [grcc.cc](#).

3.31.3.11 connComp()

```
int EGraph::connComp (
    void )
```

Definition at line 6586 of file [grcc.cc](#).

3.31.3.12 connVisit()

```
int EGraph::connVisit (
    int nd,
    int ncc )
```

Definition at line 6618 of file [grcc.cc](#).

3.31.3.13 biconnE()

```
void EGraph::biconnE (
    void )
```

Definition at line 6639 of file [grcc.cc](#).

3.31.3.14 biinitE()

```
void EGraph::biinitE (
    void )
```

Definition at line 6708 of file [grcc.cc](#).

3.31.3.15 bisearchE()

```
void EGraph::bisearchE (
    int nd,
    int * extlst,
    int * intlst,
    int * opiext,
    int * opiloop )
```

Definition at line 6744 of file [grcc.cc](#).

3.31.3.16 findRoot()

```
int EGraph::findRoot (
    void )
```

Definition at line 6537 of file [grcc.cc](#).

3.31.3.17 dirEdge()

```
int EGraph::dirEdge (
    int n,
    int e )
```

Definition at line 6347 of file [grcc.cc](#).

3.31.3.18 extMomConsv()

```
void EGraph::extMomConsv (
    void )
```

Definition at line 6949 of file [grcc.cc](#).

3.31.3.19 cmpMom()

```
int EGraph::cmpMom (
    int * lm0,
    int * em0,
    int * lm1,
    int * em1 )
```

Definition at line 6357 of file [grcc.cc](#).

3.31.3.20 groupLMom()

```
int EGraph::groupLMom (
    int * grp,
    int * ed2gr )
```

Definition at line 6407 of file [grcc.cc](#).

3.31.3.21 chkMomConsv()

```
void EGraph::chkMomConsv (
    void )
```

Definition at line 6977 of file [grcc.cc](#).

3.31.3.22 prFLines()

```
void EGraph::prFLines (
    void )
```

Definition at line 6334 of file [grcc.cc](#).

3.31.3.23 getFLines()

```
void EGraph::getFLines (
    void )
```

Definition at line 7154 of file [grcc.cc](#).

3.31.3.24 fltrace()

```
int EGraph::fltrace (
    int fk,
    int nd0,
    int * fl )
```

Definition at line 7069 of file [grcc.cc](#).

3.31.3.25 addFLine()

```
void EGraph::addFLine (
    const FLType ft,
    int fk,
    int nfl,
    int * fl )
```

Definition at line 7240 of file [grcc.cc](#).

3.31.3.26 legParticle()

```
int EGraph::legParticle (
    int ed,
    int lg )
```

Definition at line 7048 of file [grcc.cc](#).

3.31.4 Field Documentation

3.31.4.1 opt

[Options*](#) EGraph::opt

Definition at line 597 of file [grcc.h](#).

3.31.4.2 model

[Model*](#) EGraph::model

Definition at line 598 of file [grcc.h](#).

3.31.4.3 proc

`Process*` EGraph::proc

Definition at line 599 of file [grcc.h](#).

3.31.4.4 sproc

`SProcess*` EGraph::sproc

Definition at line 600 of file [grcc.h](#).

3.31.4.5 mgraph

`MGraph*` EGraph::mgraph

Definition at line 601 of file [grcc.h](#).

3.31.4.6 econn

`MConn*` EGraph::econn

Definition at line 602 of file [grcc.h](#).

3.31.4.7 nodes

`ENode**` EGraph::nodes

Definition at line 604 of file [grcc.h](#).

3.31.4.8 edges

`EEdge**` EGraph::edges

Definition at line 605 of file [grcc.h](#).

3.31.4.9 mId

`BigInt` EGraph::mId

Definition at line 607 of file [grcc.h](#).

3.31.4.10 aId

`BigInt` EGraph::aId

Definition at line 608 of file [grcc.h](#).

3.31.4.11 sId

```
BigInt EGraph::sId
```

Definition at line 609 of file [grcc.h](#).

3.31.4.12 gSubId

```
BigInt EGraph::gSubId
```

Definition at line 610 of file [grcc.h](#).

3.31.4.13 assigned

```
Bool EGraph::assigned
```

Definition at line 611 of file [grcc.h](#).

3.31.4.14 fsign

```
int EGraph::fsign
```

Definition at line 613 of file [grcc.h](#).

3.31.4.15 nsym

```
BigInt EGraph::nsym
```

Definition at line 614 of file [grcc.h](#).

3.31.4.16 esym

```
BigInt EGraph::esym
```

Definition at line 614 of file [grcc.h](#).

3.31.4.17 nsym1

```
BigInt EGraph::nsym1
```

Definition at line 615 of file [grcc.h](#).

3.31.4.18 extperm

```
BigInt EGraph::extperm
```

Definition at line 616 of file [grcc.h](#).

3.31.4.19 multp

```
BigInt EGraph::multp
```

Definition at line 617 of file [grcc.h](#).

3.31.4.20 pld

```
int EGraph::pId
```

Definition at line 619 of file [grcc.h](#).

3.31.4.21 sNodes

```
int EGraph::sNodes
```

Definition at line 621 of file [grcc.h](#).

3.31.4.22 sEdges

```
int EGraph::sEdges
```

Definition at line 622 of file [grcc.h](#).

3.31.4.23 sMaxdeg

```
int EGraph::sMaxdeg
```

Definition at line 623 of file [grcc.h](#).

3.31.4.24 sLoops

```
int EGraph::sLoops
```

Definition at line 624 of file [grcc.h](#).

3.31.4.25 nNodes

```
int EGraph::nNodes
```

Definition at line 626 of file [grcc.h](#).

3.31.4.26 nEdges

```
int EGraph::nEdges
```

Definition at line 627 of file [grcc.h](#).

3.31.4.27 nExtern

```
int EGraph::nExtern
```

Definition at line 628 of file [grcc.h](#).

3.31.4.28 maxdeg

```
int EGraph::maxdeg
```

Definition at line 629 of file [grcc.h](#).

3.31.4.29 nLoops

```
int EGraph::nLoops
```

Definition at line 630 of file [grcc.h](#).

3.31.4.30 totalc

```
int EGraph::totalc
```

Definition at line 631 of file [grcc.h](#).

3.31.4.31 nopicomp

```
int EGraph::nopicomp
```

Definition at line 634 of file [grcc.h](#).

3.31.4.32 opi2plp

```
int EGraph::opi2plp
```

Definition at line 635 of file [grcc.h](#).

3.31.4.33 nopi2p

```
int EGraph::nopi2p
```

Definition at line 636 of file [grcc.h](#).

3.31.4.34 nadj2ptv

```
int EGraph::nadj2ptv
```

Definition at line 637 of file [grcc.h](#).

3.31.4.35 DUMMYPADDING

```
int EGraph::DUMMYPADDING
```

Definition at line 639 of file [grcc.h](#).

3.31.4.36 bidef

```
int* EGraph::bidef
```

Definition at line 641 of file [grcc.h](#).

3.31.4.37 bilow

```
int* EGraph::bilow
```

Definition at line 642 of file [grcc.h](#).

3.31.4.38 extMom

```
int* EGraph::extMom
```

Definition at line 643 of file [grcc.h](#).

3.31.4.39 bconn

```
int EGraph::bconn
```

Definition at line 644 of file [grcc.h](#).

3.31.4.40 bicount

```
int EGraph::bicount
```

Definition at line 645 of file [grcc.h](#).

3.31.4.41 loopm

```
int EGraph::loopm
```

Definition at line 646 of file [grcc.h](#).

3.31.4.42 opiCount

```
int EGraph::opiCount
```

Definition at line 647 of file [grcc.h](#).

3.31.4.43 flines

[EFLine*](#) EGraph::flines[GRCC_MAXFLINES]

Definition at line 650 of file [grcc.h](#).

3.31.4.44 nflines

int EGraph::nflines

Definition at line 651 of file [grcc.h](#).

3.31.4.45 DUMMY_PADDING1

int EGraph::DUMMY_PADDING1

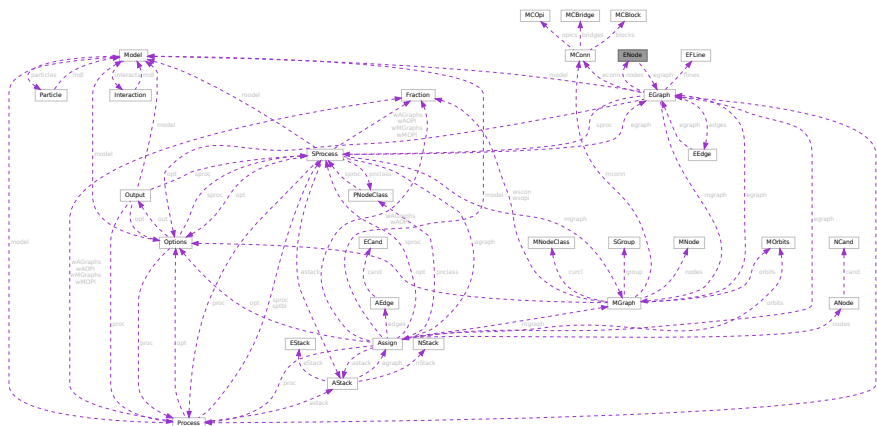
Definition at line 653 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.32 ENode Class Reference

Collaboration diagram for ENode:



Public Member Functions

- [ENode](#) ([EGraph](#) *egrph, int loops, int sdeg)
- void [initAss](#) ([EGraph](#) *egrph, int nid, int sdg)
- void [setId](#) ([EGraph](#) *egrph, const int nid)
- void [copy](#) ([ENode](#) *en)
- void [setExtern](#) (int typ, int pt)
- void [setType](#) (int typ)
- void [print](#) (void)

Data Fields

- [EGraph](#) * [egraph](#)
- int * [edges](#)
- int [id](#)
- int [maxdeg](#)
- int [deg](#)
- int [extloop](#)
- int [ndtype](#)
- int [intrct](#)
- int * [klow](#)
- int [visited](#)
- int [DUMMYPADDING](#)

3.32.1 Detailed Description

Definition at line [504](#) of file [grcc.h](#).

3.32.2 Constructor & Destructor Documentation

3.32.2.1 ENode() [1/2]

```
ENode::ENode (  
    void )
```

Definition at line [5526](#) of file [grcc.cc](#).

3.32.2.2 ENode() [2/2]

```
ENode::ENode (  
    EGraph * egrph,  
    int loops,  
    int sdeg )
```

Definition at line [5542](#) of file [grcc.cc](#).

3.32.2.3 ~ENode()

```
ENode::~ENode (  
    void )
```

Definition at line [5562](#) of file [grcc.cc](#).

3.32.3 Member Function Documentation

3.32.3.1 initAss()

```
void ENode::initAss (
    EGraph * egrph,
    int nid,
    int sdg )
```

Definition at line 5586 of file [grcc.cc](#).

3.32.3.2 setId()

```
void ENode::setId (
    EGraph * egrph,
    const int nid )
```

Definition at line 5594 of file [grcc.cc](#).

3.32.3.3 copy()

```
void ENode::copy (
    ENode * en )
```

Definition at line 5573 of file [grcc.cc](#).

3.32.3.4 setExtern()

```
void ENode::setExtern (
    int typ,
    int pt )
```

Definition at line 5601 of file [grcc.cc](#).

3.32.3.5 setType()

```
void ENode::setType (
    int typ )
```

Definition at line 5608 of file [grcc.cc](#).

3.32.3.6 print()

```
void ENode::print (
    void )
```

Definition at line 5621 of file [grcc.cc](#).

3.32.4 Field Documentation

3.32.4.1 egraph

`EGraph*` `ENode::egraph`

Definition at line 506 of file [grcc.h](#).

3.32.4.2 edges

`int*` `ENode::edges`

Definition at line 507 of file [grcc.h](#).

3.32.4.3 id

`int` `ENode::id`

Definition at line 510 of file [grcc.h](#).

3.32.4.4 maxdeg

`int` `ENode::maxdeg`

Definition at line 511 of file [grcc.h](#).

3.32.4.5 deg

`int` `ENode::deg`

Definition at line 512 of file [grcc.h](#).

3.32.4.6 extloop

`int` `ENode::extloop`

Definition at line 513 of file [grcc.h](#).

3.32.4.7 ndtype

`int` `ENode::ndtype`

Definition at line 514 of file [grcc.h](#).

3.32.4.8 intrct

```
int ENode::intrct
```

Definition at line 515 of file [grcc.h](#).

3.32.4.9 klow

```
int* ENode::klow
```

Definition at line 518 of file [grcc.h](#).

3.32.4.10 visited

```
int ENode::visited
```

Definition at line 519 of file [grcc.h](#).

3.32.4.11 DUMMYPADDING

```
int ENode::DUMMYPADDING
```

Definition at line 521 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.33 EStack Class Reference

Public Member Functions

- void [print](#) (const char *msg)

Data Fields

- int [edgen](#)
- int [det](#)
- int [nplist](#)
- int [plist](#) [GRCC_MAXMPARTICLES2]

3.33.1 Detailed Description

Definition at line 1288 of file [grcc.h](#).

3.33.2 Constructor & Destructor Documentation

3.33.2.1 EStack()

```
EStack::EStack ( ) [inline]
```

Definition at line 1297 of file [grcc.h](#).

3.33.2.2 ~EStack()

```
EStack::~~EStack ( ) [inline]
```

Definition at line 1298 of file [grcc.h](#).

3.33.3 Member Function Documentation

3.33.3.1 print()

```
void EStack::print (
    const char * msg )
```

Definition at line 9463 of file [grcc.cc](#).

3.33.4 Field Documentation

3.33.4.1 edgen

```
int EStack::edgen
```

Definition at line 1292 of file [grcc.h](#).

3.33.4.2 det

```
int EStack::det
```

Definition at line 1293 of file [grcc.h](#).

3.33.4.3 nplist

```
int EStack::nplist
```

Definition at line 1294 of file [grcc.h](#).

3.33.4.4 plist

```
int EStack::plist[GRCC_MAXMPARTICLES2]
```

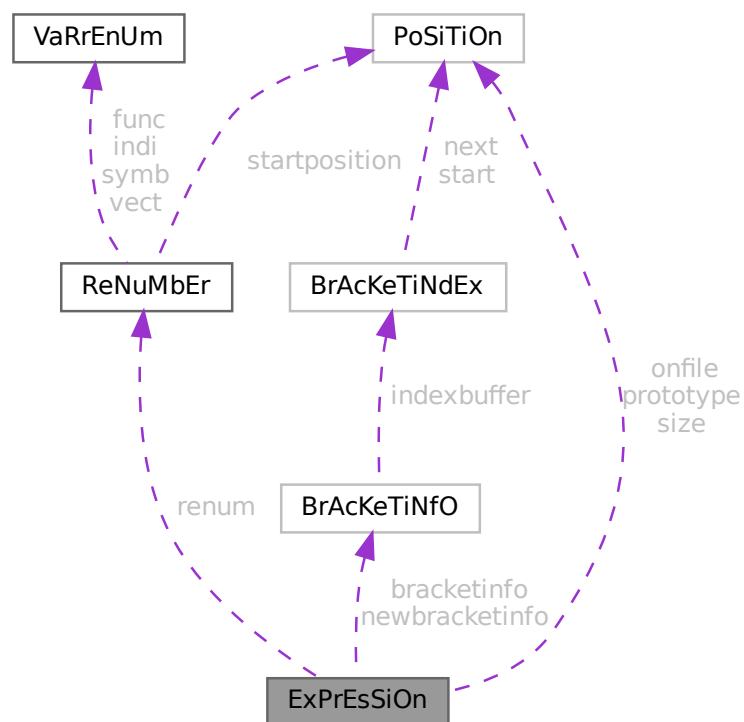
Definition at line 1295 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.34 ExPrEsSiOn Struct Reference

Collaboration diagram for ExPrEsSiOn:



Public Member Functions

- **PADPOSITION** (5, 2, 0, 13, 0)

Data Fields

- [POSITION onfile](#)
- [POSITION prototype](#)
- [POSITION size](#)
- [RENUMBER renum](#)
- [BRACKETINFO * bracketinfo](#)
- [BRACKETINFO * newbracketinfo](#)
- [WORD * renumlists](#)
- [WORD * inmem](#)
- [LONG counter](#)
- [LONG name](#)
- [WORD hidelevel](#)
- [WORD vflags](#)
- [WORD printflag](#)
- [WORD status](#)
- [WORD replace](#)
- [WORD node](#)
- [WORD whichbuffer](#)
- [WORD namesize](#)
- [WORD compression](#)
- [WORD numdummies](#)
- [WORD numfactors](#)
- [WORD sizeprototype](#)
- [WORD uflags](#)

3.34.1 Detailed Description

Definition at line [391](#) of file [structs.h](#).

3.34.2 Field Documentation

3.34.2.1 onfile

[POSITION](#) ExPrEsSiOn::onfile

Definition at line [392](#) of file [structs.h](#).

3.34.2.2 prototype

[POSITION](#) ExPrEsSiOn::prototype

Definition at line [393](#) of file [structs.h](#).

3.34.2.3 size

[POSITION](#) ExPrEsSiOn::size

Definition at line [394](#) of file [structs.h](#).

3.34.2.4 renum

`RENUMBER ExPrEsSiOn::renum`

Definition at line 395 of file [structs.h](#).

3.34.2.5 bracketinfo

`BRACKETINFO* ExPrEsSiOn::bracketinfo`

Definition at line 396 of file [structs.h](#).

3.34.2.6 newbracketinfo

`BRACKETINFO* ExPrEsSiOn::newbracketinfo`

Definition at line 397 of file [structs.h](#).

3.34.2.7 renumlists

`WORD* ExPrEsSiOn::renumlists`

Allocated only for threaded version if variables exist, else points to AN.dummyrenumlist

Definition at line 398 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#).

3.34.2.8 inmem

`WORD* ExPrEsSiOn::inmem`

Definition at line 400 of file [structs.h](#).

3.34.2.9 counter

`LONG ExPrEsSiOn::counter`

Definition at line 401 of file [structs.h](#).

3.34.2.10 name

`LONG ExPrEsSiOn::name`

Definition at line 402 of file [structs.h](#).

3.34.2.11 hidelevel

WORD ExPrEsSiOn::hidelevel

Definition at line 403 of file [structs.h](#).

3.34.2.12 vflags

WORD ExPrEsSiOn::vflags

Definition at line 404 of file [structs.h](#).

3.34.2.13 printflag

WORD ExPrEsSiOn::printflag

Definition at line 405 of file [structs.h](#).

3.34.2.14 status

WORD ExPrEsSiOn::status

Definition at line 406 of file [structs.h](#).

3.34.2.15 replace

WORD ExPrEsSiOn::replace

Definition at line 407 of file [structs.h](#).

3.34.2.16 node

WORD ExPrEsSiOn::node

Definition at line 408 of file [structs.h](#).

3.34.2.17 whichbuffer

WORD ExPrEsSiOn::whichbuffer

Definition at line 409 of file [structs.h](#).

3.34.2.18 namesize

WORD ExPrEsSiOn::namesize

Definition at line 410 of file [structs.h](#).

3.34.2.19 compression

WORD ExPrEsSiOn::compression

Definition at line 411 of file [structs.h](#).

3.34.2.20 numdummies

WORD ExPrEsSiOn::numdummies

Definition at line 412 of file [structs.h](#).

3.34.2.21 numfactors

WORD ExPrEsSiOn::numfactors

Definition at line 413 of file [structs.h](#).

3.34.2.22 sizeprototype

WORD ExPrEsSiOn::sizeprototype

Definition at line 414 of file [structs.h](#).

3.34.2.23 uflags

WORD ExPrEsSiOn::uflags

Definition at line 415 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.35 FaCdOILaR Struct Reference

Data Fields

- WORD * [where](#)
- LONG [size](#)
- WORD [type](#)
- WORD [value](#)

3.35.1 Detailed Description

Definition at line 522 of file [structs.h](#).

3.35.2 Field Documentation

3.35.2.1 where

WORD* FaCdOlLaR::where

Definition at line 523 of file [structs.h](#).

3.35.2.2 size

LONG FaCdOlLaR::size

Definition at line 524 of file [structs.h](#).

3.35.2.3 type

WORD FaCdOlLaR::type

Definition at line 525 of file [structs.h](#).

3.35.2.4 value

WORD FaCdOlLaR::value

Definition at line 526 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.36 factorized_poly Class Reference

Public Member Functions

- void [add_factor](#) (const [poly](#) &f, int p=1)
- const std::string [tostring](#) () const

Data Fields

- std::vector< [poly](#) > [factor](#)
- std::vector< int > [power](#)

Friends

- std::ostream & [operator](#)<< (std::ostream &out, const [poly](#) &p)

3.36.1 Detailed Description

Definition at line 54 of file [polyfact.h](#).

3.36.2 Member Function Documentation

3.36.2.1 `add_factor()`

```
void factorized_poly::add_factor (
    const poly & f,
    int p = 1 )
```

Definition at line 125 of file [polyfact.cc](#).

3.36.2.2 `tostring()`

```
const string factorized_poly::tostring ( ) const
```

Definition at line 59 of file [polyfact.cc](#).

3.36.3 Field Documentation

3.36.3.1 `factor`

```
std::vector<poly> factorized_poly::factor
```

Definition at line 59 of file [polyfact.h](#).

3.36.3.2 `power`

```
std::vector<int> factorized_poly::power
```

Definition at line 60 of file [polyfact.h](#).

The documentation for this class was generated from the following files:

- [polyfact.h](#)
- [polyfact.cc](#)

3.37 FGInput Struct Reference

Data Fields

- long [ninitl](#)
- const char * [initln](#) [GRCC_MAXNODES]
- int [initlc](#) [GRCC_MAXNODES]
- long [nfinal](#)
- const char * [finaln](#) [GRCC_MAXNODES]
- int [finalc](#) [GRCC_MAXNODES]
- int [coupl](#) [GRCC_MAXNCPLG]

3.37.1 Detailed Description

Definition at line 165 of file [grccparam.h](#).

3.37.2 Field Documentation

3.37.2.1 ninitl

```
long FGInput::ninitl
```

Definition at line 166 of file [grccparam.h](#).

3.37.2.2 initln

```
const char* FGInput::initln[GRCC_MAXNODES]
```

Definition at line 167 of file [grccparam.h](#).

3.37.2.3 initlc

```
int FGInput::initlc[GRCC_MAXNODES]
```

Definition at line 168 of file [grccparam.h](#).

3.37.2.4 nfinal

```
long FGInput::nfinal
```

Definition at line 169 of file [grccparam.h](#).

3.37.2.5 finaln

```
const char* FGInput::finaln[GRCC_MAXNODES]
```

Definition at line 170 of file [grccparam.h](#).

3.37.2.6 finalc

```
int FGInput::finalc[GRCC_MAXNODES]
```

Definition at line 171 of file [grccparam.h](#).

3.37.2.7 coupl

```
int FGInput::coupl[GRCC_MAXNCPLG]
```

Definition at line 172 of file [grccparam.h](#).

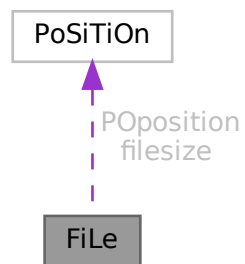
The documentation for this struct was generated from the following file:

- [grccparam.h](#)

3.38 FiLe Struct Reference

```
#include <structs.h>
```

Collaboration diagram for FiLe:



Public Member Functions

- **PADPOSITION** (5, 3, 2, 0, 0)

Data Fields

- [POSITION](#) [POposition](#)
- [POSITION](#) [filesize](#)
- WORD * [PObuffer](#)
- WORD * [POstop](#)
- WORD * [POfill](#)
- WORD * [POfull](#)
- char * [name](#)
- ULONG [numblocks](#)
- ULONG [inbuffer](#)
- LONG [POsize](#)
- int [handle](#)
- int [active](#)

3.38.1 Detailed Description

The type FILEHANDLE is the struct that controls all relevant information of a file, whether it is open or not. The file may even not yet exist. There is a system of caches (PObuffer) and as long as the information to be written still fits inside the cache the file may never be created. There are variables that can store information about different types of files, like scratch files or sort files. Depending on what is available in the system we may also have information about gzip compression (currently sort file only) or locks (TFORM).

Definition at line 682 of file [structs.h](#).

3.38.2 Field Documentation

3.38.2.1 PPosition

`POSITION FiLe::PPosition`

Definition at line 683 of file [structs.h](#).

3.38.2.2 filesize

`POSITION FiLe::filesize`

Definition at line 684 of file [structs.h](#).

3.38.2.3 PObuffer

`WORD* FiLe::PObuffer`

Definition at line 685 of file [structs.h](#).

3.38.2.4 PStop

`WORD* FiLe::PStop`

Definition at line 686 of file [structs.h](#).

3.38.2.5 POfill

`WORD* FiLe::POfill`

Definition at line 687 of file [structs.h](#).

3.38.2.6 POfull

`WORD* FiLe::POfull`

Definition at line 688 of file [structs.h](#).

3.38.2.7 name

```
char* FiLe::name
```

Definition at line 695 of file [structs.h](#).

3.38.2.8 numblocks

```
ULONG FiLe::numblocks
```

Definition at line 700 of file [structs.h](#).

3.38.2.9 inbuffer

```
ULONG FiLe::inbuffer
```

Definition at line 701 of file [structs.h](#).

3.38.2.10 PSize

```
LONG FiLe::PSize
```

Definition at line 702 of file [structs.h](#).

3.38.2.11 handle

```
int FiLe::handle
```

Our own handle. Equal -1 if no file exists.

Definition at line 710 of file [structs.h](#).

Referenced by [DoOnePow\(\)](#), [FlushOut\(\)](#), [GetFirstTerm\(\)](#), [MergePatches\(\)](#), [poly_factorize_expression\(\)](#), [poly_unfactorize_expression\(\)](#), [PutIn\(\)](#), [PutOut\(\)](#), [Sflush\(\)](#), and [StageSort\(\)](#).

3.38.2.12 active

```
int FiLe::active
```

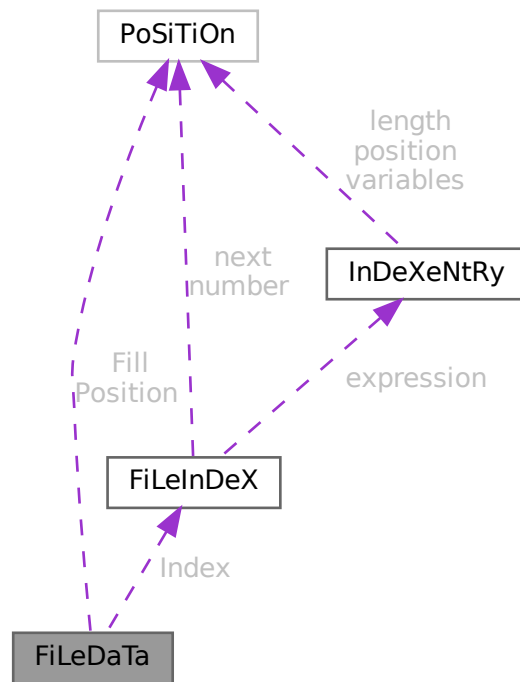
Definition at line 711 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.39 FiLeDaTa Struct Reference

Collaboration diagram for FiLeDaTa:



Public Member Functions

- **PADPOSITION** (0, 0, 0, 2, 0)

Data Fields

- [FILEINDEX](#) Index
- [POSITION](#) Fill
- [POSITION](#) Position
- [WORD](#) Handle
- [WORD](#) dirtyflag

3.39.1 Detailed Description

Definition at line 151 of file [structs.h](#).

3.39.2 Field Documentation

3.39.2.1 Index

`FILEINDEX` `FileDaTa::Index`

Definition at line 152 of file [structs.h](#).

3.39.2.2 Fill

`POSITION` `FileDaTa::Fill`

Definition at line 153 of file [structs.h](#).

3.39.2.3 Position

`POSITION` `FileDaTa::Position`

Definition at line 154 of file [structs.h](#).

3.39.2.4 Handle

`WORD` `FileDaTa::Handle`

Definition at line 155 of file [structs.h](#).

3.39.2.5 dirtyflag

`WORD` `FileDaTa::dirtyflag`

Definition at line 156 of file [structs.h](#).

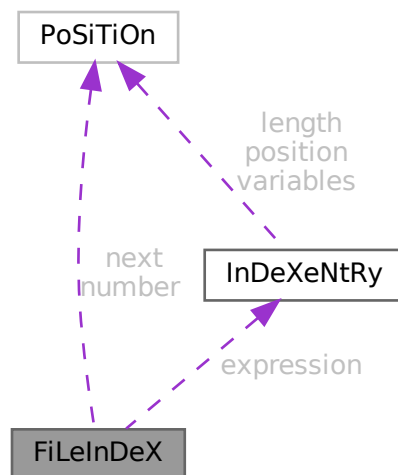
The documentation for this struct was generated from the following file:

- [structs.h](#)

3.40 FiLeInDeX Struct Reference

```
#include <structs.h>
```

Collaboration diagram for FiLeInDeX:



Data Fields

- [POSITION](#) next
- [POSITION](#) number
- [INDEXENTRY](#) expression [[INFILEINDEX](#)]
- [SBYTE](#) empty [[EMPTYININDEX](#)]

3.40.1 Detailed Description

Defines the structure of a file index in store-files and save-files.

It contains several entries (see struct [InDeXeNtRy](#)) up to a maximum of [INFILEINDEX](#).

The variable number has been made of type [POSITION](#) to avoid padding problems with some types of computers/↔ OS and keep system independence of the .sav files.

This struct is always 512 bytes long.

Definition at line [138](#) of file [structs.h](#).

3.40.2 Field Documentation

3.40.2.1 next

`POSITION FileInDeX::next`

Position of next FILEINDEX if any

Definition at line 139 of file [structs.h](#).

Referenced by [ReadSaveIndex\(\)](#).

3.40.2.2 number

`POSITION FileInDeX::number`

Number of used entries in this index

Definition at line 140 of file [structs.h](#).

Referenced by [ReadSaveIndex\(\)](#).

3.40.2.3 expression

`INDEXENTRY FileInDeX::expression[INFILEINDEX]`

File index entries

Definition at line 141 of file [structs.h](#).

3.40.2.4 empty

`SBYTE FileInDeX::empty[EMPTYININDEX]`

Padding to 512 bytes

Definition at line 142 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.41 FixedGlobals Struct Reference

```
#include <structs.h>
```


Data Fields

- WCN [Operation](#) [8]
- WCN2 [OperaFind](#) [6]
- char * [VarType](#) [10]
- char * [ExprStat](#) [21]
- char * [FunNam](#) [2]
- char * [swmes](#) [3]
- char * [fname](#)
- char * [fname2](#)
- UBYTE * [s_one](#)
- WORD [fnamebase](#)
- WORD [fname2base](#)
- WORD [fnamesize](#)
- WORD [fname2size](#)
- UINT [cTable](#) [256]

3.41.1 Detailed Description

The FIXEDGLOBALS struct is an anachronism. It started as the struct with global variables that needed initialization. It contains the elements `Operation` and `OperaFind` which define a very early way of automatically jumping to the proper operation. We find the results of it in parts of the file [opera.c](#). Later operations were treated differently in a more transparent way. We never changed the existing code. The most important part is currently the `cTable` which is used intensively in the compiler.

Definition at line [2674](#) of file [structs.h](#).

3.41.2 Field Documentation

3.41.2.1 Operation

```
WCN FixedGlobals::Operation[8]
```

Definition at line [2675](#) of file [structs.h](#).

3.41.2.2 OperaFind

```
WCN2 FixedGlobals::OperaFind[6]
```

Definition at line [2676](#) of file [structs.h](#).

3.41.2.3 VarType

```
char* FixedGlobals::VarType[10]
```

Definition at line [2677](#) of file [structs.h](#).

3.41.2.4 ExprStat

```
char* FixedGlobals::ExprStat[21]
```

Definition at line 2678 of file [structs.h](#).

3.41.2.5 FunNam

```
char* FixedGlobals::FunNam[2]
```

Definition at line 2679 of file [structs.h](#).

3.41.2.6 swmes

```
char* FixedGlobals::swmes[3]
```

Definition at line 2680 of file [structs.h](#).

3.41.2.7 fname

```
char* FixedGlobals::fname
```

Definition at line 2681 of file [structs.h](#).

3.41.2.8 fname2

```
char* FixedGlobals::fname2
```

Definition at line 2682 of file [structs.h](#).

3.41.2.9 s_one

```
UBYTE* FixedGlobals::s_one
```

Definition at line 2683 of file [structs.h](#).

3.41.2.10 fnamebase

```
WORD FixedGlobals::fnamebase
```

Definition at line 2684 of file [structs.h](#).

3.41.2.11 fname2base

```
WORD FixedGlobals::fname2base
```

Definition at line 2685 of file [structs.h](#).

3.41.2.12 fnamesize

WORD FixedGlobals::fnamesize

Definition at line 2686 of file [structs.h](#).

3.41.2.13 fname2size

WORD FixedGlobals::fname2size

Definition at line 2687 of file [structs.h](#).

3.41.2.14 cTable

UINT FixedGlobals::cTable[256]

Definition at line 2688 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.42 fixedset Struct Reference

Data Fields

- char * [name](#)
- char * [description](#)
- int [type](#)
- int [dimension](#)

3.42.1 Detailed Description

Definition at line 571 of file [structs.h](#).

3.42.2 Field Documentation

3.42.2.1 name

char* fixedset::name

Definition at line 572 of file [structs.h](#).

3.42.2.2 description

```
char* fixedset::description
```

Definition at line 573 of file [structs.h](#).

3.42.2.3 type

```
int fixedset::type
```

Definition at line 574 of file [structs.h](#).

3.42.2.4 dimension

```
int fixedset::dimension
```

Definition at line 575 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.43 flint::fmpz Class Reference

Public Member Functions

- void [print](#) (const string &text)

Data Fields

- [fmpz_t](#) [d](#)

3.43.1 Detailed Description

Definition at line 54 of file [flintinterface.h](#).

3.43.2 Constructor & Destructor Documentation

3.43.2.1 fmpz()

```
flint::fmpz::fmpz ( ) [inline]
```

Definition at line 57 of file [flintinterface.h](#).

3.43.2.2 ~fmpz()

```
flint::fmpz::~~fmpz ( ) [inline]
```

Definition at line 58 of file [flintinterface.h](#).

3.43.3 Member Function Documentation

3.43.3.1 print()

```
void flint::fmpz::print (
    const string & text ) [inline]
```

Definition at line 59 of file [flintinterface.h](#).

3.43.4 Field Documentation

3.43.4.1 d

```
fmpz_t flint::fmpz::d
```

Definition at line 56 of file [flintinterface.h](#).

The documentation for this class was generated from the following file:

- [flintinterface.h](#)

3.44 Fraction Class Reference

Public Member Functions

- [Fraction](#) (BigInt n, BigInt d)
- void [print](#) (const char *msg)
- void [setValue](#) (BigInt n, BigInt d)
- void [setValue](#) ([Fraction](#) &f)
- void [add](#) (BigInt n, BigInt d)
- void [add](#) ([Fraction](#) f)
- void [sub](#) ([Fraction](#) f)
- BigInt [gcd](#) (BigInt n0, BigInt n1)
- void [normal](#) (void)
- Bool [isEq](#) ([Fraction](#) f)

Data Fields

- BigInt [num](#)
- BigInt [den](#)
- double [ratio](#)

3.44.1 Detailed Description

Definition at line [182](#) of file [grcc.h](#).

3.44.2 Constructor & Destructor Documentation

3.44.2.1 Fraction() [1/2]

```
Fraction::Fraction ( ) [inline]
```

Definition at line [187](#) of file [grcc.h](#).

3.44.2.2 Fraction() [2/2]

```
Fraction::Fraction (
    BigInt n,
    BigInt d )
```

Definition at line [9680](#) of file [grcc.cc](#).

3.44.3 Member Function Documentation

3.44.3.1 print()

```
void Fraction::print (
    const char * msg )
```

Definition at line [9691](#) of file [grcc.cc](#).

3.44.3.2 setValue() [1/2]

```
void Fraction::setValue (
    BigInt n,
    BigInt d )
```

Definition at line [9703](#) of file [grcc.cc](#).

3.44.3.3 setValue() [2/2]

```
void Fraction::setValue (
    Fraction & f )
```

Definition at line [9711](#) of file [grcc.cc](#).

3.44.3.4 add() [1/2]

```
void Fraction::add (
    BigInt n,
    BigInt d )
```

Definition at line 9756 of file [grcc.cc](#).

3.44.3.5 add() [2/2]

```
void Fraction::add (
    Fraction f )
```

Definition at line 9779 of file [grcc.cc](#).

3.44.3.6 sub()

```
void Fraction::sub (
    Fraction f )
```

Definition at line 9788 of file [grcc.cc](#).

3.44.3.7 gcd()

```
BigInt Fraction::gcd (
    BigInt n0,
    BigInt n1 )
```

Definition at line 9719 of file [grcc.cc](#).

3.44.3.8 normal()

```
void Fraction::normal (
    void )
```

Definition at line 9739 of file [grcc.cc](#).

3.44.3.9 isEq()

```
Bool Fraction::isEq (
    Fraction f )
```

Definition at line 9797 of file [grcc.cc](#).

3.44.4 Field Documentation

3.44.4.1 num

```
BigInt Fraction::num
```

Definition at line 184 of file [grcc.h](#).

3.44.4.2 den

```
BigInt Fraction::den
```

Definition at line 184 of file [grcc.h](#).

3.44.4.3 ratio

```
double Fraction::ratio
```

Definition at line 185 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.45 FUN_INFO Struct Reference

```
#include <structs.h>
```

Data Fields

- WORD * [location](#)
- int [numargs](#)
- int [numfunnies](#)
- int [numwildcards](#)
- int [symmet](#)
- int [tensor](#)
- int [commute](#)

3.45.1 Detailed Description

The struct [FUN_INFO](#) is used for information about functions in the file [smart.c](#) which is supposed to intelligently look for patterns in complicated wildcard situations involving symmetric functions.

Definition at line 608 of file [structs.h](#).

3.45.2 Field Documentation

3.45.2.1 location

```
WORD* FUN_INFO::location
```

Definition at line 609 of file [structs.h](#).

3.45.2.2 numargs

```
int FUN_INFO::numargs
```

Definition at line 610 of file [structs.h](#).

3.45.2.3 numfunnies

```
int FUN_INFO::numfunnies
```

Definition at line 611 of file [structs.h](#).

3.45.2.4 numwildcards

```
int FUN_INFO::numwildcards
```

Definition at line 612 of file [structs.h](#).

3.45.2.5 symmet

```
int FUN_INFO::symmet
```

Definition at line 613 of file [structs.h](#).

3.45.2.6 tensor

```
int FUN_INFO::tensor
```

Definition at line 614 of file [structs.h](#).

3.45.2.7 commute

```
int FUN_INFO::commute
```

Definition at line 615 of file [structs.h](#).

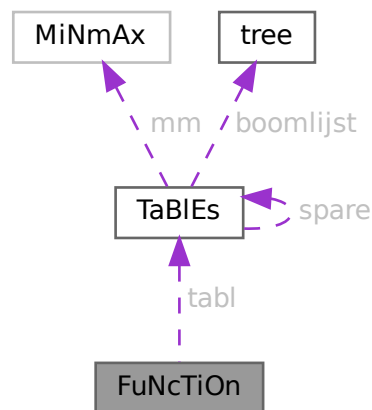
The documentation for this struct was generated from the following file:

- [structs.h](#)

3.46 FuNcTiOn Struct Reference

```
#include <structs.h>
```

Collaboration diagram for FuNcTiOn:



Data Fields

- [TABLES](#) `tabl`
- LONG `symminfo`
- LONG `name`
- WORD `commute`
- WORD `complex`
- WORD `number`
- WORD `flags`
- WORD `spec`
- WORD `symmetric`
- WORD `node`
- WORD `namesize`
- WORD `dimension`
- WORD `maxnumargs`
- WORD `minnumargs`

3.46.1 Detailed Description

Contains all information about a function. Also used for tables. It is used in the [LIST](#) elements of AC.

Definition at line [477](#) of file [structs.h](#).

3.46.2 Field Documentation

3.46.2.1 `tabl`

`TABLES FuNcTiOn::tabl`

Used if redefined as table. != 0 if function is a table

Definition at line 478 of file [structs.h](#).

3.46.2.2 `symminfo`

`LONG FuNcTiOn::symminfo`

Info regarding symm properties offset in buffer

Definition at line 479 of file [structs.h](#).

3.46.2.3 `name`

`LONG FuNcTiOn::name`

Location in namebuffer of [NAMETREE](#)

Definition at line 480 of file [structs.h](#).

Referenced by [StartVariables\(\)](#).

3.46.2.4 `commute`

`WORD FuNcTiOn::commute`

Commutation properties

Definition at line 481 of file [structs.h](#).

3.46.2.5 `complex`

`WORD FuNcTiOn::complex`

Properties under complex conjugation

Definition at line 482 of file [structs.h](#).

3.46.2.6 number

WORD FuNcTiOn::number

Number when stored in file

Definition at line [483](#) of file [structs.h](#).

Referenced by [ReadSaveIndex\(\)](#).

3.46.2.7 flags

WORD FuNcTiOn::flags

Used to indicate usage when storing

Definition at line [484](#) of file [structs.h](#).

3.46.2.8 spec

WORD FuNcTiOn::spec

Regular, Tensor, etc. See [FunSpecs](#).

Definition at line [485](#) of file [structs.h](#).

Referenced by [ReadSaveVariables\(\)](#).

3.46.2.9 symmetric

WORD FuNcTiOn::symmetric

0 if symmetric properties

Definition at line [486](#) of file [structs.h](#).

Referenced by [StartVariables\(\)](#).

3.46.2.10 node

WORD FuNcTiOn::node

Location in namenode of [NAMETREE](#)

Definition at line [487](#) of file [structs.h](#).

3.46.2.11 namesize

WORD FuNcTiOn::namesize

Length of the name

Definition at line 488 of file [structs.h](#).

3.46.2.12 dimension

WORD FuNcTiOn::dimension

Definition at line 489 of file [structs.h](#).

3.46.2.13 maxnumargs

WORD FuNcTiOn::maxnumargs

Definition at line 490 of file [structs.h](#).

3.46.2.14 minnumargs

WORD FuNcTiOn::minnumargs

Definition at line 491 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.47 gcd_heuristic_failed Class Reference

3.47.1 Detailed Description

Definition at line 33 of file [polygcd.h](#).

The documentation for this class was generated from the following file:

- [polygcd.h](#)

3.48 HANDLERS Struct Reference

```
#include <structs.h>
```

Data Fields

- WORD [newlogonly](#)
- WORD [newhandle](#)
- WORD [oldhandle](#)
- WORD [oldlogonly](#)
- WORD [oldprinttype](#)
- WORD [oldsilent](#)

3.48.1 Detailed Description

The struct [HANDLERS](#) is used in the communication with external channels.

Definition at line [964](#) of file [structs.h](#).

3.48.2 Field Documentation

3.48.2.1 newlogonly

```
WORD HANDLERS::newlogonly
```

Definition at line [965](#) of file [structs.h](#).

3.48.2.2 newhandle

```
WORD HANDLERS::newhandle
```

Definition at line [966](#) of file [structs.h](#).

3.48.2.3 oldhandle

```
WORD HANDLERS::oldhandle
```

Definition at line [967](#) of file [structs.h](#).

3.48.2.4 oldlogonly

```
WORD HANDLERS::oldlogonly
```

Definition at line [968](#) of file [structs.h](#).

3.48.2.5 oldprinttype

```
WORD HANDLERS::oldprinttype
```

Definition at line [969](#) of file [structs.h](#).

3.48.2.6 oldsilent

```
WORD HANDLERS::oldsilent
```

Definition at line 970 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.49 Input Struct Reference

Data Fields

- const char * [name](#)
- int [icode](#)
- int [nplistn](#)
- const char * [plistn](#) [GRCC_MAXLEGS]
- int [plistc](#) [GRCC_MAXLEGS]
- int [cvallist](#) [GRCC_MAXNCPLG]

3.49.1 Detailed Description

Definition at line 137 of file [grccparam.h](#).

3.49.2 Field Documentation

3.49.2.1 name

```
const char* IInput::name
```

Definition at line 138 of file [grccparam.h](#).

3.49.2.2 icode

```
int IInput::icode
```

Definition at line 139 of file [grccparam.h](#).

3.49.2.3 nplistn

```
int IInput::nplistn
```

Definition at line 140 of file [grccparam.h](#).

3.49.2.4 plistn

```
const char* IInput::plistn[GRCC_MAXLEGS]
```

Definition at line 141 of file [grccparam.h](#).

3.49.2.5 plistc

```
int IInput::plistc[GRCC_MAXLEGS]
```

Definition at line 142 of file [grccparam.h](#).

3.49.2.6 cvallist

```
int IInput::cvallist[GRCC_MAXNCPLG]
```

Definition at line 143 of file [grccparam.h](#).

The documentation for this struct was generated from the following file:

- [grccparam.h](#)

3.50 InDeX Struct Reference

Data Fields

- LONG [name](#)
- WORD [type](#)
- WORD [dimension](#)
- WORD [number](#)
- WORD [flags](#)
- WORD [nmin4](#)
- WORD [node](#)
- WORD [namesize](#)

3.50.1 Detailed Description

Definition at line 445 of file [structs.h](#).

3.50.2 Field Documentation

3.50.2.1 name

```
LONG InDeX::name
```

Definition at line 446 of file [structs.h](#).

3.50.2.2 type

WORD InDeX::type

Definition at line 447 of file [structs.h](#).

3.50.2.3 dimension

WORD InDeX::dimension

Definition at line 448 of file [structs.h](#).

3.50.2.4 number

WORD InDeX::number

Definition at line 449 of file [structs.h](#).

3.50.2.5 flags

WORD InDeX::flags

Definition at line 450 of file [structs.h](#).

3.50.2.6 nmin4

WORD InDeX::nmin4

Definition at line 451 of file [structs.h](#).

3.50.2.7 node

WORD InDeX::node

Definition at line 452 of file [structs.h](#).

3.50.2.8 namesize

WORD InDeX::namesize

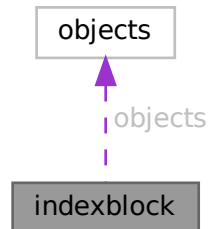
Definition at line 453 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.51 indexblock Struct Reference

Collaboration diagram for indexblock:



Data Fields

- MLONG [flags](#)
- MLONG [previousblock](#)
- MLONG [position](#)
- [OBJECTS objects](#) [NUMOBJECTS]

3.51.1 Detailed Description

Definition at line [108](#) of file [minos.h](#).

3.51.2 Field Documentation

3.51.2.1 flags

MLONG `indexblock::flags`

Definition at line [109](#) of file [minos.h](#).

3.51.2.2 previousblock

MLONG `indexblock::previousblock`

Definition at line [110](#) of file [minos.h](#).

3.51.2.3 position

MLONG `indexblock::position`

Definition at line [111](#) of file [minos.h](#).

3.51.2.4 objects

`OBJECTS indexblock::objects[NUMOBJECTS]`

Definition at line 112 of file [minos.h](#).

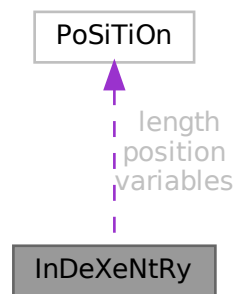
The documentation for this struct was generated from the following file:

- [minos.h](#)

3.52 InDeXeNtRy Struct Reference

```
#include <structs.h>
```

Collaboration diagram for InDeXeNtRy:



Public Member Functions

- **PADPOSITION** (0, 1, 0, 5, MAXENAME+1)

Data Fields

- [POSITION](#) position
- [POSITION](#) length
- [POSITION](#) variables
- LONG [CompressSize](#)
- WORD [nsymbols](#)
- WORD [nindices](#)
- WORD [nvectors](#)
- WORD [nfunctions](#)
- WORD [size](#)
- SBYTE [name](#) [MAXENAME+1]

3.52.1 Detailed Description

Defines the structure of an entry in a file index (see struct [FiLeInDeX](#)).

It represents one expression in the file.

Definition at line [100](#) of file [structs.h](#).

3.52.2 Field Documentation

3.52.2.1 position

`POSITION InDeXeNtRy::position`

Position of the expression itself

Definition at line [101](#) of file [structs.h](#).

3.52.2.2 length

`POSITION InDeXeNtRy::length`

Length of the expression itself

Definition at line [102](#) of file [structs.h](#).

3.52.2.3 variables

`POSITION InDeXeNtRy::variables`

Position of the list with variables

Definition at line [103](#) of file [structs.h](#).

3.52.2.4 CompressSize

`LONG InDeXeNtRy::CompressSize`

Size of buffer before compress

Definition at line [104](#) of file [structs.h](#).

3.52.2.5 nsymbols

`WORD InDeXeNtRy::nsymbols`

Number of symbols in the list

Definition at line [105](#) of file [structs.h](#).

Referenced by [ReadSaveVariables\(\)](#).

3.52.2.6 nindices

WORD InDeXeNtRy::nindices

Number of indices in the list

Definition at line 106 of file [structs.h](#).

Referenced by [ReadSaveVariables\(\)](#).

3.52.2.7 nvectors

WORD InDeXeNtRy::nvectors

Number of vectors in the list

Definition at line 107 of file [structs.h](#).

Referenced by [ReadSaveVariables\(\)](#).

3.52.2.8 nfunctions

WORD InDeXeNtRy::nfunctions

Number of functions in the list

Definition at line 108 of file [structs.h](#).

Referenced by [ReadSaveVariables\(\)](#).

3.52.2.9 size

WORD InDeXeNtRy::size

Size of variables field

Definition at line 109 of file [structs.h](#).

3.52.2.10 name

SBYTE InDeXeNtRy::name[MAXENAME+1]

Name of expression

Definition at line 110 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.53 iniinfo Struct Reference

Data Fields

- MLONG [entriesinindex](#)
- MLONG [numberofindexblocks](#)
- MLONG [firstindexblock](#)
- MLONG [lastindexblock](#)
- MLONG [numberoftables](#)
- MLONG [numberofnamesblocks](#)
- MLONG [firstnameblock](#)
- MLONG [lastnameblock](#)

3.53.1 Detailed Description

Definition at line [83](#) of file [minos.h](#).

3.53.2 Field Documentation

3.53.2.1 entriesinindex

```
MLONG iniinfo::entriesinindex
```

Definition at line [85](#) of file [minos.h](#).

3.53.2.2 numberofindexblocks

```
MLONG iniinfo::numberofindexblocks
```

Definition at line [86](#) of file [minos.h](#).

3.53.2.3 firstindexblock

```
MLONG iniinfo::firstindexblock
```

Definition at line [87](#) of file [minos.h](#).

3.53.2.4 lastindexblock

```
MLONG iniinfo::lastindexblock
```

Definition at line [88](#) of file [minos.h](#).

3.53.2.5 numberoftables

MLONG iniinfo::numberoftables

Definition at line 89 of file [minos.h](#).

3.53.2.6 numberofnamesblocks

MLONG iniinfo::numberofnamesblocks

Definition at line 90 of file [minos.h](#).

3.53.2.7 firstnameblock

MLONG iniinfo::firstnameblock

Definition at line 91 of file [minos.h](#).

3.53.2.8 lastnameblock

MLONG iniinfo::lastnameblock

Definition at line 92 of file [minos.h](#).

The documentation for this struct was generated from the following file:

- [minos.h](#)

3.54 INSIDEINFO Struct Reference

```
#include <structs.h>
```

Data Fields

- WORD * [buffer](#)
- int [oldcompletype](#)
- int [oldparallelflag](#)
- int [oldnumpotmoddollars](#)
- WORD [size](#)
- WORD [numdollars](#)
- WORD [oldcbuf](#)
- WORD [oldrbuf](#)
- WORD [inscbuf](#)
- WORD [oldcnumlhs](#)

3.54.1 Detailed Description

Used for #inside

Definition at line [854](#) of file [structs.h](#).

3.54.2 Field Documentation

3.54.2.1 buffer

```
WORD*  INSIDEINFO::buffer
```

Definition at line [855](#) of file [structs.h](#).

3.54.2.2 oldcompiletype

```
int  INSIDEINFO::oldcompiletype
```

Definition at line [856](#) of file [structs.h](#).

3.54.2.3 oldparalleflag

```
int  INSIDEINFO::oldparalleflag
```

Definition at line [857](#) of file [structs.h](#).

3.54.2.4 oldnumpotmoddollars

```
int  INSIDEINFO::oldnumpotmoddollars
```

Definition at line [858](#) of file [structs.h](#).

3.54.2.5 size

```
WORD  INSIDEINFO::size
```

Definition at line [859](#) of file [structs.h](#).

3.54.2.6 numdollars

```
WORD  INSIDEINFO::numdollars
```

Definition at line [860](#) of file [structs.h](#).

3.54.2.7 olddbuf

WORD INSIDEINFO::olddbuf

Definition at line 861 of file [structs.h](#).

3.54.2.8 oldrbuf

WORD INSIDEINFO::oldrbuf

Definition at line 862 of file [structs.h](#).

3.54.2.9 inscbuf

WORD INSIDEINFO::inscbuf

Definition at line 863 of file [structs.h](#).

3.54.2.10 oldcnumlhs

WORD INSIDEINFO::oldcnumlhs

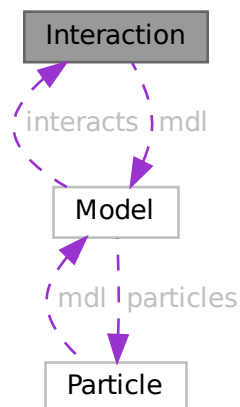
Definition at line 864 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.55 Interaction Class Reference

Collaboration diagram for Interaction:



Public Member Functions

- [Interaction](#) ([Model](#) *modl, int iid, const char *nam, int icode, int *cpl, int nlgs, int *plst, int csm, int lp)
- void [prlInteraction](#) (void)

Data Fields

- [Model](#) * [mdl](#)
- char * [name](#)
- int * [plist](#)
- int * [clist](#)
- int * [slist](#)
- int [id](#)
- int [csum](#)
- int [nlegs](#)
- int [loop](#)
- int [icode](#)
- int [nplist](#)
- int [nclist](#)
- int [nslst](#)

3.55.1 Detailed Description

Definition at line [236](#) of file [grcc.h](#).

3.55.2 Constructor & Destructor Documentation

3.55.2.1 Interaction()

```
Interaction::Interaction (  
    Model * modl,  
    int iid,  
    const char * nam,  
    int icode,  
    int * cpl,  
    int nlgs,  
    int * plst,  
    int csm,  
    int lp )
```

Definition at line [1570](#) of file [grcc.cc](#).

3.55.2.2 ~Interaction()

```
Interaction::~~Interaction (  
    void )
```

Definition at line [1691](#) of file [grcc.cc](#).

3.55.3 Member Function Documentation

3.55.3.1 prInteraction()

```
void Interaction::prInteraction (
    void )
```

Definition at line 1706 of file [grcc.cc](#).

3.55.4 Field Documentation

3.55.4.1 mdl

```
Model* Interaction::mdl
```

Definition at line 238 of file [grcc.h](#).

3.55.4.2 name

```
char* Interaction::name
```

Definition at line 239 of file [grcc.h](#).

3.55.4.3 plist

```
int* Interaction::plist
```

Definition at line 240 of file [grcc.h](#).

3.55.4.4 clist

```
int* Interaction::clist
```

Definition at line 241 of file [grcc.h](#).

3.55.4.5 slist

```
int* Interaction::slist
```

Definition at line 242 of file [grcc.h](#).

3.55.4.6 id

```
int Interaction::id
```

Definition at line 244 of file [grcc.h](#).

3.55.4.7 csum

```
int Interaction::csum
```

Definition at line [245](#) of file [grcc.h](#).

3.55.4.8 nlegs

```
int Interaction::nlegs
```

Definition at line [246](#) of file [grcc.h](#).

3.55.4.9 loop

```
int Interaction::loop
```

Definition at line [247](#) of file [grcc.h](#).

3.55.4.10 icode

```
int Interaction::icode
```

Definition at line [248](#) of file [grcc.h](#).

3.55.4.11 nplist

```
int Interaction::nplist
```

Definition at line [250](#) of file [grcc.h](#).

3.55.4.12 nclist

```
int Interaction::nclist
```

Definition at line [251](#) of file [grcc.h](#).

3.55.4.13 nslist

```
int Interaction::nslist
```

Definition at line [252](#) of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.56 KEYWORD Struct Reference

```
#include <structs.h>
```

Data Fields

- char * [name](#)
- TFUN [func](#)
- int [type](#)
- int [flags](#)

3.56.1 Detailed Description

The [KEYWORD](#) struct defines names of commands/statements and the routine to be called when they are encountered by the compiler or preprocessor.

Definition at line [222](#) of file [structs.h](#).

3.56.2 Field Documentation

3.56.2.1 name

```
char* KEYWORD::name
```

Definition at line [223](#) of file [structs.h](#).

3.56.2.2 func

```
TFUN KEYWORD::func
```

Definition at line [224](#) of file [structs.h](#).

3.56.2.3 type

```
int KEYWORD::type
```

Definition at line [225](#) of file [structs.h](#).

3.56.2.4 flags

```
int KEYWORD::flags
```

Definition at line [226](#) of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.57 KEYWORDV Struct Reference

```
#include <structs.h>
```

Data Fields

- char * [name](#)
- int * [var](#)
- int [type](#)
- int [flags](#)

3.57.1 Detailed Description

The [KEYWORDV](#) struct defines names of commands/statements and the variable to be affected when they are encountered by the compiler or preprocessor.

Definition at line [234](#) of file [structs.h](#).

3.57.2 Field Documentation

3.57.2.1 name

```
char* KEYWORDV::name
```

Definition at line [235](#) of file [structs.h](#).

3.57.2.2 var

```
int* KEYWORDV::var
```

Definition at line [236](#) of file [structs.h](#).

3.57.2.3 type

```
int KEYWORDV::type
```

Definition at line [237](#) of file [structs.h](#).

3.57.2.4 flags

```
int KEYWORDV::flags
```

Definition at line [238](#) of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.58 LIST Struct Reference

```
#include <structs.h>
```

Data Fields

- void * [lijst](#)
- char * [message](#)
- int [num](#)
- int [maxnum](#)
- int [size](#)
- int [numglobal](#)
- int [numtemp](#)
- int [numclear](#)

3.58.1 Detailed Description

Much information is stored in arrays of which we can double the size if the array proves to be too small. Such arrays are controlled by a variable of type [LIST](#). The routines that expand the lists are in the file [tools.c](#)

Definition at line [205](#) of file [structs.h](#).

3.58.2 Field Documentation

3.58.2.1 lijst

```
void* LIST::lijst
```

[D] Holds space for "maxnum" elements of size "size" each

Definition at line [206](#) of file [structs.h](#).

3.58.2.2 message

```
char* LIST::message
```

Text for Malloc1 when allocating lijst. Set to constant string.

Definition at line [207](#) of file [structs.h](#).

3.58.2.3 num

```
int LIST::num
```

Number of elements in lijst.

Definition at line [208](#) of file [structs.h](#).

3.58.2.4 maxnum

```
int LIST::maxnum
```

Maximum number of elements in lijst.

Definition at line [209](#) of file [structs.h](#).

3.58.2.5 size

```
int LIST::size
```

Size of one element in lijst.

Definition at line [210](#) of file [structs.h](#).

3.58.2.6 numglobal

```
int LIST::numglobal
```

Marker for position when .global is executed.

Definition at line [211](#) of file [structs.h](#).

3.58.2.7 numtemp

```
int LIST::numtemp
```

At the moment only needed for sets and setstore.

Definition at line [212](#) of file [structs.h](#).

3.58.2.8 numclear

```
int LIST::numclear
```

Only for the clear instruction.

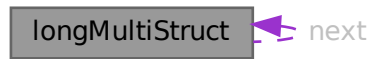
Definition at line [213](#) of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.59 longMultiStruct Struct Reference

Collaboration diagram for longMultiStruct:



Data Fields

- UBYTE * [buffer](#)
- int [bufpos](#)
- int [packpos](#)
- int [nPacks](#)
- int [lastLen](#)
- struct [longMultiStruct](#) * [next](#)

3.59.1 Detailed Description

Definition at line [1013](#) of file [mpi.c](#).

3.59.2 Field Documentation

3.59.2.1 buffer

```
UBYTE* longMultiStruct::buffer
```

Definition at line [1014](#) of file [mpi.c](#).

3.59.2.2 bufpos

```
int longMultiStruct::bufpos
```

Definition at line [1015](#) of file [mpi.c](#).

3.59.2.3 packpos

```
int longMultiStruct::packpos
```

Definition at line [1016](#) of file [mpi.c](#).

3.59.2.4 nPacks

```
int longMultiStruct::nPacks
```

Definition at line 1017 of file [mpi.c](#).

3.59.2.5 lastLen

```
int longMultiStruct::lastLen
```

Definition at line 1018 of file [mpi.c](#).

3.59.2.6 next

```
struct longMultiStruct* longMultiStruct::next
```

Definition at line 1021 of file [mpi.c](#).

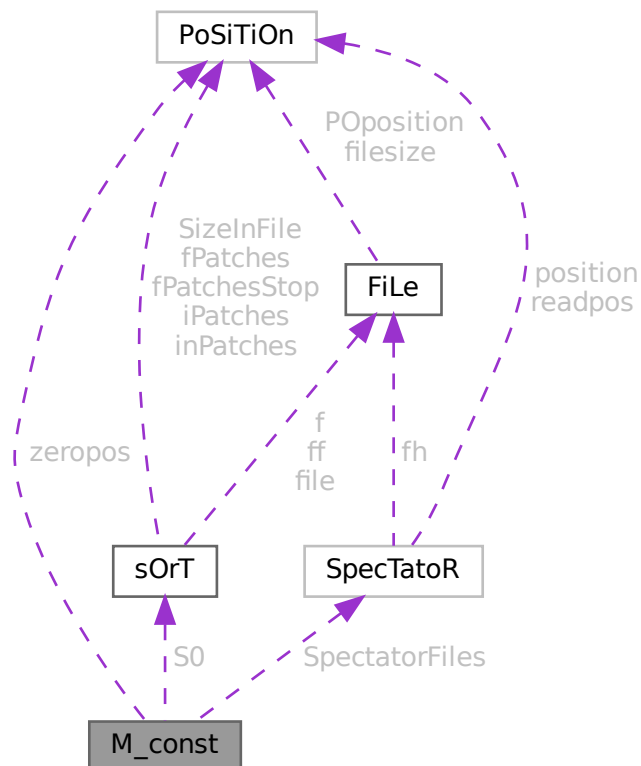
The documentation for this struct was generated from the following file:

- [mpi.c](#)

3.60 M_const Struct Reference

```
#include <structs.h>
```

Collaboration diagram for M_const:



Public Member Functions

- **PADPOSITION** (17, 28, 62, 84, 2)

Data Fields

- [POSITION](#) zeropos
- [SORTING](#) * [S0](#)
- [UWORD](#) * [gcmmod](#)
- [UWORD](#) * [gpowmod](#)
- [UBYTE](#) * [TempDir](#)
- [UBYTE](#) * [TempSortDir](#)
- [UBYTE](#) * [IncDir](#)
- [UBYTE](#) * [InputFileName](#)
- [UBYTE](#) * [LogFileName](#)
- [UBYTE](#) * [OutBuffer](#)
- [UBYTE](#) * [Path](#)
- [UBYTE](#) * [SetupDir](#)
- [UBYTE](#) * [SetupFile](#)
- [UBYTE](#) * [gFortran90Kind](#)
- [UBYTE](#) * [gextrasym](#)
- [UBYTE](#) * [ggextrasym](#)
- [UBYTE](#) * [oldnumextrasymbols](#)
- [SPECTATOR](#) * [SpectatorFiles](#)
- [LONG](#) [MaxTer](#)
- [LONG](#) [CompressSize](#)
- [LONG](#) [ScratSize](#)
- [LONG](#) [HideSize](#)
- [LONG](#) [SizeStoreCache](#)
- [LONG](#) [MaxStreamSize](#)
- [LONG](#) [SIOsize](#)
- [LONG](#) [SLargeSize](#)
- [LONG](#) [SSmallEsize](#)
- [LONG](#) [SSmallSize](#)
- [LONG](#) [STermsInSmall](#)
- [LONG](#) [MaxBracketBufferSize](#)
- [LONG](#) [hProcessBucketSize](#)
- [LONG](#) [gProcessBucketSize](#)
- [LONG](#) [shmWinSize](#)
- [LONG](#) [OldChildTime](#)
- [LONG](#) [OldSecTime](#)
- [LONG](#) [OldMilliTime](#)
- [LONG](#) [WorkSize](#)
- [LONG](#) [gThreadBucketSize](#)
- [LONG](#) [ggThreadBucketSize](#)
- [LONG](#) [SumTime](#)
- [LONG](#) [SpectatorSize](#)
- [LONG](#) [TimeLimit](#)
- [int](#) [FileOnlyFlag](#)
- [int](#) [Interact](#)
- [int](#) [MaxParLevel](#)
- [int](#) [OutBufSize](#)
- [int](#) [SMaxFpatches](#)
- [int](#) [SMaxPatches](#)

- int StdOut
- int ginsidefirst
- int gDefDim
- int gDefDim4
- int NumFixedSets
- int NumFixedFunctions
- int rbufnum
- int dbufnum
- int sbufnum
- int zbufnum
- int SkipClears
- int gTokensWriteFlag
- int gfunpowers
- int gStatsFlag
- int gNamesFlag
- int gCodesFlag
- int gSortType
- int gproperorderflag
- int hparallelflag
- int gparallelflag
- int totalnumberofthreads
- int gSizeCommutelnSet
- int gThreadStats
- int ggThreadStats
- int gFinalStats
- int ggFinalStats
- int gThreadsFlag
- int ggThreadsFlag
- int gThreadBalancing
- int ggThreadBalancing
- int gThreadSortFileSynch
- int ggThreadSortFileSynch
- int gProcessStats
- int ggProcessStats
- int gOldParallelStats
- int ggOldParallelStats
- int maxFlevels
- int resetTimeOnClear
- int gcNumDollars
- int MultiRun
- int gNoSpacesInNumbers
- int ggNoSpacesInNumbers
- int gIsFortran90
- int PrintTotalSize
- int fbufferSize
- int gOldFactArgFlag
- int ggOldFactArgFlag
- int gnumextrasym
- int ggnumextrasym
- int NumSpectatorFiles
- int SizeForSpectatorFiles
- int gOldGCDflag
- int ggOldGCDflag
- int gWTimeStatsFlag
- int ggWTimeStatsFlag

- int [jumpratio](#)
- WORD [MaxTal](#)
- WORD [IndDum](#)
- WORD [DumInd](#)
- WORD [WillInd](#)
- WORD [gncmod](#)
- WORD [gnpowmod](#)
- WORD [gmodmode](#)
- WORD [gUnitTrace](#)
- WORD [gOutputMode](#)
- WORD [gOutputSpaces](#)
- WORD [gOutNumberType](#)
- WORD [gCnumpows](#)
- WORD [gUniTrace](#) [4]
- WORD [MaxWildcards](#)
- WORD [mTraceDum](#)
- WORD [OffsetIndex](#)
- WORD [OffsetVector](#)
- WORD [RepMax](#)
- WORD [LogType](#)
- WORD [ggStatsFlag](#)
- WORD [gLineLength](#)
- WORD [qError](#)
- WORD [FortranCont](#)
- WORD [HoldFlag](#)
- WORD [Ordering](#) [15]
- WORD [silent](#)
- WORD [tracebackflag](#)
- WORD [expnum](#)
- WORD [denomnum](#)
- WORD [facnum](#)
- WORD [invfacnum](#)
- WORD [sumnum](#)
- WORD [sumpnum](#)
- WORD [OldOrderFlag](#)
- WORD [termfunnum](#)
- WORD [matchfunnum](#)
- WORD [countfunnum](#)
- WORD [gPolyFun](#)
- WORD [gPolyFunInv](#)
- WORD [gPolyFunType](#)
- WORD [gPolyFunExp](#)
- WORD [gPolyFunVar](#)
- WORD [gPolyFunPow](#)
- WORD [dollarzero](#)
- WORD [atstartup](#)
- WORD [exitflag](#)
- WORD [NumStoreCaches](#)
- WORD [gIndentSpace](#)
- WORD [ggIndentSpace](#)
- WORD [gShortStatsMax](#)
- WORD [ggShortStatsMax](#)
- WORD [gextrasymbols](#)
- WORD [ggextrasymbols](#)
- WORD [zerorhs](#)

- WORD [onerhs](#)
- WORD [havesortdir](#)
- WORD [vectorzero](#)
- WORD [ClearStore](#)
- WORD [BracketFactors](#) [8]
- BOOL [FromStdin](#)
- BOOL [IgnoreDeprecation](#)

3.60.1 Detailed Description

The [M_const](#) struct is part of the global data and resides in the [ALLGLOBALS](#) struct A under the name #M. We see it used with the macro AM as in AM.S0. It contains global settings at startup or .clear.

Definition at line [1424](#) of file [structs.h](#).

3.60.2 Field Documentation

3.60.2.1 zeropos

```
POSITION M_const::zeropos
```

Definition at line [1425](#) of file [structs.h](#).

3.60.2.2 S0

```
SORTING* M_const::S0
```

[D] The main sort buffer

Definition at line [1426](#) of file [structs.h](#).

3.60.2.3 gcmmod

```
UWORD* M_const::gcmmod
```

Global setting of modulus. Uses AC.cmod's memory

Definition at line [1427](#) of file [structs.h](#).

3.60.2.4 gpowmod

```
UWORD* M_const::gpowmod
```

Global setting printing as powers. Uses AC.cmod's memory

Definition at line [1428](#) of file [structs.h](#).

3.60.2.5 TempDir

```
UBYTE* M_const::TempDir
```

Definition at line 1429 of file [structs.h](#).

3.60.2.6 TempSortDir

```
UBYTE* M_const::TempSortDir
```

Definition at line 1430 of file [structs.h](#).

3.60.2.7 IncDir

```
UBYTE* M_const::IncDir
```

Definition at line 1431 of file [structs.h](#).

3.60.2.8 InputFileName

```
UBYTE* M_const::InputFileName
```

Definition at line 1432 of file [structs.h](#).

3.60.2.9 LogFileName

```
UBYTE* M_const::LogFileName
```

Definition at line 1433 of file [structs.h](#).

3.60.2.10 OutBuffer

```
UBYTE* M_const::OutBuffer
```

Definition at line 1434 of file [structs.h](#).

3.60.2.11 Path

```
UBYTE* M_const::Path
```

Definition at line 1435 of file [structs.h](#).

3.60.2.12 SetupDir

```
UBYTE* M_const::SetupDir
```

Definition at line 1436 of file [structs.h](#).

3.60.2.13 SetupFile

UBYTE* M_const::SetupFile

Definition at line 1437 of file [structs.h](#).

3.60.2.14 gFortran90Kind

UBYTE* M_const::gFortran90Kind

Definition at line 1438 of file [structs.h](#).

3.60.2.15 gextrasym

UBYTE* M_const::gextrasym

Definition at line 1439 of file [structs.h](#).

3.60.2.16 ggextrasym

UBYTE* M_const::ggextrasym

Definition at line 1440 of file [structs.h](#).

3.60.2.17 oldnumextrasymbols

UBYTE* M_const::oldnumextrasymbols

Definition at line 1441 of file [structs.h](#).

3.60.2.18 SpectatorFiles

SPECTATOR* M_const::SpectatorFiles

Definition at line 1442 of file [structs.h](#).

3.60.2.19 MaxTer

LONG M_const::MaxTer

Definition at line 1450 of file [structs.h](#).

3.60.2.20 CompressSize

LONG M_const::CompressSize

Definition at line 1451 of file [structs.h](#).

3.60.2.21 ScratSize

LONG M_const::SkratSize

Definition at line 1452 of file [structs.h](#).

3.60.2.22 HideSize

LONG M_const::HideSize

Definition at line 1453 of file [structs.h](#).

3.60.2.23 SizeStoreCache

LONG M_const::SizeStoreCache

Definition at line 1454 of file [structs.h](#).

3.60.2.24 MaxStreamSize

LONG M_const::MaxStreamSize

Definition at line 1455 of file [structs.h](#).

3.60.2.25 SIOsize

LONG M_const::SIOsize

Definition at line 1456 of file [structs.h](#).

3.60.2.26 SLargeSize

LONG M_const::SLargeSize

Definition at line 1457 of file [structs.h](#).

3.60.2.27 SSmallEsize

LONG M_const::SSmallEsize

Definition at line 1458 of file [structs.h](#).

3.60.2.28 SSmallSize

LONG M_const::SSmallSize

Definition at line 1459 of file [structs.h](#).

3.60.2.29 STermsInSmall

LONG M_const::STermsInSmall

Definition at line 1460 of file [structs.h](#).

3.60.2.30 MaxBracketBufferSize

LONG M_const::MaxBracketBufferSize

Definition at line 1461 of file [structs.h](#).

3.60.2.31 hProcessBucketSize

LONG M_const::hProcessBucketSize

Definition at line 1462 of file [structs.h](#).

3.60.2.32 gProcessBucketSize

LONG M_const::gProcessBucketSize

Definition at line 1463 of file [structs.h](#).

3.60.2.33 shmWinSize

LONG M_const::shmWinSize

Definition at line 1464 of file [structs.h](#).

3.60.2.34 OldChildTime

LONG M_const::OldChildTime

Definition at line 1465 of file [structs.h](#).

3.60.2.35 OldSecTime

LONG M_const::OldSecTime

Definition at line 1466 of file [structs.h](#).

3.60.2.36 OldMilliTime

LONG M_const::OldMilliTime

Definition at line 1467 of file [structs.h](#).

3.60.2.37 WorkSize

LONG M_const::WorkSize

Definition at line 1468 of file [structs.h](#).

3.60.2.38 gThreadBucketSize

LONG M_const::gThreadBucketSize

Definition at line 1469 of file [structs.h](#).

3.60.2.39 ggThreadBucketSize

LONG M_const::ggThreadBucketSize

Definition at line 1470 of file [structs.h](#).

3.60.2.40 SumTime

LONG M_const::SumTime

Definition at line 1471 of file [structs.h](#).

3.60.2.41 SpectatorSize

LONG M_const::SpectatorSize

Definition at line 1472 of file [structs.h](#).

3.60.2.42 TimeLimit

LONG M_const::TimeLimit

Definition at line 1473 of file [structs.h](#).

3.60.2.43 FileOnlyFlag

int M_const::FileOnlyFlag

Definition at line 1480 of file [structs.h](#).

3.60.2.44 Interact

int M_const::Interact

Definition at line 1481 of file [structs.h](#).

3.60.2.45 MaxParLevel

```
int M_const::MaxParLevel
```

Definition at line [1482](#) of file [structs.h](#).

3.60.2.46 OutBufSize

```
int M_const::OutBufSize
```

Definition at line [1483](#) of file [structs.h](#).

3.60.2.47 SMaxFpatches

```
int M_const::SMaxFpatches
```

Definition at line [1484](#) of file [structs.h](#).

3.60.2.48 SMaxPatches

```
int M_const::SMaxPatches
```

Definition at line [1485](#) of file [structs.h](#).

3.60.2.49 StdOut

```
int M_const::StdOut
```

Definition at line [1486](#) of file [structs.h](#).

3.60.2.50 ginsidefirst

```
int M_const::ginsidefirst
```

Definition at line [1487](#) of file [structs.h](#).

3.60.2.51 gDefDim

```
int M_const::gDefDim
```

Definition at line [1488](#) of file [structs.h](#).

3.60.2.52 gDefDim4

```
int M_const::gDefDim4
```

Definition at line [1489](#) of file [structs.h](#).

3.60.2.53 NumFixedSets

```
int M_const::NumFixedSets
```

Definition at line 1490 of file [structs.h](#).

3.60.2.54 NumFixedFunctions

```
int M_const::NumFixedFunctions
```

Definition at line 1491 of file [structs.h](#).

3.60.2.55 rbufnum

```
int M_const::rbufnum
```

Definition at line 1492 of file [structs.h](#).

3.60.2.56 dbufnum

```
int M_const::dbufnum
```

Definition at line 1493 of file [structs.h](#).

3.60.2.57 sbufnum

```
int M_const::sbufnum
```

Definition at line 1494 of file [structs.h](#).

3.60.2.58 zbufnum

```
int M_const::zbufnum
```

Definition at line 1495 of file [structs.h](#).

3.60.2.59 SkipClears

```
int M_const::SkipClears
```

Definition at line 1496 of file [structs.h](#).

3.60.2.60 gTokensWriteFlag

```
int M_const::gTokensWriteFlag
```

Definition at line 1497 of file [structs.h](#).

3.60.2.61 gfunpowers

```
int M_const::gfunpowers
```

Definition at line [1498](#) of file [structs.h](#).

3.60.2.62 gStatsFlag

```
int M_const::gStatsFlag
```

Definition at line [1499](#) of file [structs.h](#).

3.60.2.63 gNamesFlag

```
int M_const::gNamesFlag
```

Definition at line [1500](#) of file [structs.h](#).

3.60.2.64 gCodesFlag

```
int M_const::gCodesFlag
```

Definition at line [1501](#) of file [structs.h](#).

3.60.2.65 gSortType

```
int M_const::gSortType
```

Definition at line [1502](#) of file [structs.h](#).

3.60.2.66 gproperorderflag

```
int M_const::gproperorderflag
```

Definition at line [1503](#) of file [structs.h](#).

3.60.2.67 hparallelflag

```
int M_const::hparallelflag
```

Definition at line [1504](#) of file [structs.h](#).

3.60.2.68 gparallelflag

```
int M_const::gparallelflag
```

Definition at line [1505](#) of file [structs.h](#).

3.60.2.69 totalnumberofthreads

```
int M_const::totalnumberofthreads
```

Definition at line 1506 of file [structs.h](#).

3.60.2.70 gSizeCommuteInSet

```
int M_const::gSizeCommuteInSet
```

Definition at line 1507 of file [structs.h](#).

3.60.2.71 gThreadStats

```
int M_const::gThreadStats
```

Definition at line 1508 of file [structs.h](#).

3.60.2.72 ggThreadStats

```
int M_const::ggThreadStats
```

Definition at line 1509 of file [structs.h](#).

3.60.2.73 gFinalStats

```
int M_const::gFinalStats
```

Definition at line 1510 of file [structs.h](#).

3.60.2.74 ggFinalStats

```
int M_const::ggFinalStats
```

Definition at line 1511 of file [structs.h](#).

3.60.2.75 gThreadsFlag

```
int M_const::gThreadsFlag
```

Definition at line 1512 of file [structs.h](#).

3.60.2.76 ggThreadsFlag

```
int M_const::ggThreadsFlag
```

Definition at line 1513 of file [structs.h](#).

3.60.2.77 gThreadBalancing

```
int M_const::gThreadBalancing
```

Definition at line 1514 of file [structs.h](#).

3.60.2.78 ggThreadBalancing

```
int M_const::ggThreadBalancing
```

Definition at line 1515 of file [structs.h](#).

3.60.2.79 gThreadSortFileSynch

```
int M_const::gThreadSortFileSynch
```

Definition at line 1516 of file [structs.h](#).

3.60.2.80 ggThreadSortFileSynch

```
int M_const::ggThreadSortFileSynch
```

Definition at line 1517 of file [structs.h](#).

3.60.2.81 gProcessStats

```
int M_const::gProcessStats
```

Definition at line 1518 of file [structs.h](#).

3.60.2.82 ggProcessStats

```
int M_const::ggProcessStats
```

Definition at line 1519 of file [structs.h](#).

3.60.2.83 gOldParallelStats

```
int M_const::gOldParallelStats
```

Definition at line 1520 of file [structs.h](#).

3.60.2.84 ggOldParallelStats

```
int M_const::ggOldParallelStats
```

Definition at line 1521 of file [structs.h](#).

3.60.2.85 maxFlevels

```
int M_const::maxFlevels
```

Definition at line 1522 of file [structs.h](#).

3.60.2.86 resetTimeOnClear

```
int M_const::resetTimeOnClear
```

Definition at line 1523 of file [structs.h](#).

3.60.2.87 gcNumDollars

```
int M_const::gcNumDollars
```

Definition at line 1524 of file [structs.h](#).

3.60.2.88 MultiRun

```
int M_const::MultiRun
```

Definition at line 1525 of file [structs.h](#).

3.60.2.89 gNoSpacesInNumbers

```
int M_const::gNoSpacesInNumbers
```

Definition at line 1526 of file [structs.h](#).

3.60.2.90 ggNoSpacesInNumbers

```
int M_const::ggNoSpacesInNumbers
```

Definition at line 1527 of file [structs.h](#).

3.60.2.91 gIsFortran90

```
int M_const::gIsFortran90
```

Definition at line 1528 of file [structs.h](#).

3.60.2.92 PrintTotalSize

```
int M_const::PrintTotalSize
```

Definition at line 1529 of file [structs.h](#).

3.60.2.93 fbufferSize

```
int M_const::fbufferSize
```

Definition at line 1530 of file [structs.h](#).

3.60.2.94 gOldFactArgFlag

```
int M_const::gOldFactArgFlag
```

Definition at line 1531 of file [structs.h](#).

3.60.2.95 ggOldFactArgFlag

```
int M_const::ggOldFactArgFlag
```

Definition at line 1532 of file [structs.h](#).

3.60.2.96 gnumextrasym

```
int M_const::gnumextrasym
```

Definition at line 1533 of file [structs.h](#).

3.60.2.97 ggnumextrasym

```
int M_const::ggnumextrasym
```

Definition at line 1534 of file [structs.h](#).

3.60.2.98 NumSpectatorFiles

```
int M_const::NumSpectatorFiles
```

Definition at line 1535 of file [structs.h](#).

3.60.2.99 SizeForSpectatorFiles

```
int M_const::SizeForSpectatorFiles
```

Definition at line 1536 of file [structs.h](#).

3.60.2.100 gOldGCDflag

```
int M_const::gOldGCDflag
```

Definition at line 1537 of file [structs.h](#).

3.60.2.101 ggOldGCDflag

```
int M_const::ggOldGCDflag
```

Definition at line 1538 of file [structs.h](#).

3.60.2.102 gWTimeStatsFlag

```
int M_const::gWTimeStatsFlag
```

Definition at line 1539 of file [structs.h](#).

3.60.2.103 ggWTimeStatsFlag

```
int M_const::ggWTimeStatsFlag
```

Definition at line 1540 of file [structs.h](#).

3.60.2.104 jumpratio

```
int M_const::jumpratio
```

Definition at line 1541 of file [structs.h](#).

3.60.2.105 MaxTal

```
WORD M_const::MaxTal
```

Definition at line 1542 of file [structs.h](#).

3.60.2.106 IndDum

```
WORD M_const::IndDum
```

Definition at line 1543 of file [structs.h](#).

3.60.2.107 DumInd

```
WORD M_const::DumInd
```

Definition at line 1544 of file [structs.h](#).

3.60.2.108 Willnd

```
WORD M_const::Willnd
```

Definition at line 1545 of file [structs.h](#).

3.60.2.109 gncmod

WORD M_const::gncmod

Definition at line 1546 of file [structs.h](#).

3.60.2.110 gnpowmod

WORD M_const::gnpowmod

Definition at line 1547 of file [structs.h](#).

3.60.2.111 gmodmode

WORD M_const::gmodmode

Definition at line 1548 of file [structs.h](#).

3.60.2.112 gUnitTrace

WORD M_const::gUnitTrace

Definition at line 1549 of file [structs.h](#).

3.60.2.113 gOutputMode

WORD M_const::gOutputMode

Definition at line 1550 of file [structs.h](#).

3.60.2.114 gOutputSpaces

WORD M_const::gOutputSpaces

Definition at line 1551 of file [structs.h](#).

3.60.2.115 gOutNumberType

WORD M_const::gOutNumberType

Definition at line 1552 of file [structs.h](#).

3.60.2.116 gCnumpows

WORD M_const::gCnumpows

Definition at line 1553 of file [structs.h](#).

3.60.2.117 gUniTrace

WORD M_const::gUniTrace[4]

Definition at line 1554 of file [structs.h](#).

3.60.2.118 MaxWildcards

WORD M_const::MaxWildcards

Definition at line 1555 of file [structs.h](#).

3.60.2.119 mTraceDum

WORD M_const::mTraceDum

Definition at line 1556 of file [structs.h](#).

3.60.2.120 OffsetIndex

WORD M_const::OffsetIndex

Definition at line 1557 of file [structs.h](#).

3.60.2.121 OffsetVector

WORD M_const::OffsetVector

Definition at line 1558 of file [structs.h](#).

3.60.2.122 RepMax

WORD M_const::RepMax

Definition at line 1559 of file [structs.h](#).

3.60.2.123 LogType

WORD M_const::LogType

Definition at line 1560 of file [structs.h](#).

3.60.2.124 ggStatsFlag

WORD M_const::ggStatsFlag

Definition at line 1561 of file [structs.h](#).

3.60.2.125 gLineLength

WORD M_const::gLineLength

Definition at line 1562 of file [structs.h](#).

3.60.2.126 qError

WORD M_const::qError

Definition at line 1563 of file [structs.h](#).

3.60.2.127 FortranCont

WORD M_const::FortranCont

Definition at line 1564 of file [structs.h](#).

3.60.2.128 HoldFlag

WORD M_const::HoldFlag

Definition at line 1565 of file [structs.h](#).

3.60.2.129 Ordering

WORD M_const::Ordering[15]

Definition at line 1566 of file [structs.h](#).

3.60.2.130 silent

WORD M_const::silent

Definition at line 1567 of file [structs.h](#).

3.60.2.131 tracebackflag

WORD M_const::tracebackflag

Definition at line 1568 of file [structs.h](#).

3.60.2.132 expnum

WORD M_const::expnum

Definition at line 1569 of file [structs.h](#).

3.60.2.133 denomnum

WORD M_const::denomnum

Definition at line 1570 of file [structs.h](#).

3.60.2.134 facnum

WORD M_const::facnum

Definition at line 1571 of file [structs.h](#).

3.60.2.135 invfacnum

WORD M_const::invfacnum

Definition at line 1572 of file [structs.h](#).

3.60.2.136 sumnum

WORD M_const::sumnum

Definition at line 1573 of file [structs.h](#).

3.60.2.137 sumpnum

WORD M_const::sumpnum

Definition at line 1574 of file [structs.h](#).

3.60.2.138 OldOrderFlag

WORD M_const::OldOrderFlag

Definition at line 1575 of file [structs.h](#).

3.60.2.139 termfunnum

WORD M_const::termfunnum

Definition at line 1576 of file [structs.h](#).

3.60.2.140 matchfunnum

WORD M_const::matchfunnum

Definition at line 1577 of file [structs.h](#).

3.60.2.141 countfunnum

WORD M_const::countfunnum

Definition at line 1578 of file [structs.h](#).

3.60.2.142 gPolyFun

WORD M_const::gPolyFun

Definition at line 1579 of file [structs.h](#).

3.60.2.143 gPolyFunInv

WORD M_const::gPolyFunInv

Definition at line 1580 of file [structs.h](#).

3.60.2.144 gPolyFunType

WORD M_const::gPolyFunType

Definition at line 1581 of file [structs.h](#).

3.60.2.145 gPolyFunExp

WORD M_const::gPolyFunExp

Definition at line 1582 of file [structs.h](#).

3.60.2.146 gPolyFunVar

WORD M_const::gPolyFunVar

Definition at line 1583 of file [structs.h](#).

3.60.2.147 gPolyFunPow

WORD M_const::gPolyFunPow

Definition at line 1584 of file [structs.h](#).

3.60.2.148 dollarzero

WORD M_const::dollarzero

Definition at line 1585 of file [structs.h](#).

3.60.2.149 atstartup

WORD M_const::atstartup

Definition at line 1586 of file [structs.h](#).

3.60.2.150 exitflag

WORD M_const::exitflag

Definition at line 1587 of file [structs.h](#).

3.60.2.151 NumStoreCaches

WORD M_const::NumStoreCaches

Definition at line 1588 of file [structs.h](#).

3.60.2.152 gIndentSpace

WORD M_const::gIndentSpace

Definition at line 1589 of file [structs.h](#).

3.60.2.153 ggIndentSpace

WORD M_const::ggIndentSpace

Definition at line 1590 of file [structs.h](#).

3.60.2.154 gShortStatsMax

WORD M_const::gShortStatsMax

For On FewerStatistics 10;

Definition at line 1591 of file [structs.h](#).

3.60.2.155 ggShortStatsMax

WORD M_const::ggShortStatsMax

For On FewerStatistics 10;

Definition at line 1592 of file [structs.h](#).

3.60.2.156 gextrasymbols

WORD M_const::gextrasymbols

Definition at line 1593 of file [structs.h](#).

3.60.2.157 ggextrasymbols

WORD M_const::ggextrasymbols

Definition at line 1594 of file [structs.h](#).

3.60.2.158 zerorhs

WORD M_const::zerorhs

Definition at line 1595 of file [structs.h](#).

3.60.2.159 onerhs

WORD M_const::onerhs

Definition at line 1596 of file [structs.h](#).

3.60.2.160 havessortdir

WORD M_const::havessortdir

Definition at line 1597 of file [structs.h](#).

3.60.2.161 vectorzero

WORD M_const::vectorzero

Definition at line 1598 of file [structs.h](#).

3.60.2.162 ClearStore

WORD M_const::ClearStore

Definition at line 1599 of file [structs.h](#).

3.60.2.163 BracketFactors

WORD M_const::BracketFactors[8]

Definition at line 1600 of file [structs.h](#).

3.60.2.164 FromStdin

```
BOOL M_const::FromStdin
```

Definition at line 1601 of file [structs.h](#).

3.60.2.165 IgnoreDeprecation

```
BOOL M_const::IgnoreDeprecation
```

Definition at line 1602 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.61 MCBLOCK Class Reference

Public Member Functions

- void [init](#) (void)

Data Fields

- Edge2n * [edges](#)
- int [nedges](#)
- int [nartps](#)
- int [loop](#)
- int [DUMMYPADDING](#)

3.61.1 Detailed Description

Definition at line 933 of file [grcc.h](#).

3.61.2 Constructor & Destructor Documentation

3.61.2.1 MCBLOCK()

```
MCBlock::MCBlock (  
    void )
```

Definition at line 5106 of file [grcc.cc](#).

3.61.2.2 ~MCBlock()

```
MCBlock::~MCBlock (
    void )
```

Definition at line [5112](#) of file [grcc.cc](#).

3.61.3 Member Function Documentation

3.61.3.1 init()

```
void MCBlock::init (
    void )
```

Definition at line [5117](#) of file [grcc.cc](#).

3.61.4 Field Documentation

3.61.4.1 edges

```
Edge2n* MCBlock::edges
```

Definition at line [935](#) of file [grcc.h](#).

3.61.4.2 nmedges

```
int MCBlock::nmedges
```

Definition at line [936](#) of file [grcc.h](#).

3.61.4.3 nartps

```
int MCBlock::nartps
```

Definition at line [937](#) of file [grcc.h](#).

3.61.4.4 loop

```
int MCBlock::loop
```

Definition at line [938](#) of file [grcc.h](#).

3.61.4.5 DUMMYPADDING

```
int MCBlock::DUMMYPADDING
```

Definition at line 940 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.62 MCBridge Class Reference

Data Fields

- [Edge2n](#) [nodes](#)
- [int](#) [next](#)

3.62.1 Detailed Description

Definition at line 921 of file [grcc.h](#).

3.62.2 Constructor & Destructor Documentation

3.62.2.1 MCBridge()

```
MCBridge::MCBridge (  
    void )
```

Definition at line 5094 of file [grcc.cc](#).

3.62.2.2 ~MCBridge()

```
MCBridge::~~MCBridge (  
    void )
```

Definition at line 5099 of file [grcc.cc](#).

3.62.3 Field Documentation

3.62.3.1 nodes

```
Edge2n MCBridge::nodes
```

Definition at line 923 of file [grcc.h](#).

3.62.3.2 next

```
int MCBridge::next
```

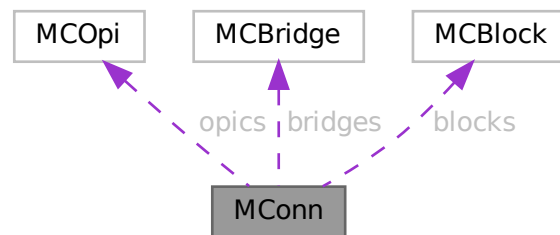
Definition at line 924 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.63 MConn Class Reference

Collaboration diagram for MConn:



Public Member Functions

- [MConn](#) (int nnod, int nedg)
- void [init](#) (void)
- void [pushNode](#) (int nd)
- void [pushEdge](#) (int n0, int n1)
- void [addOPic](#) ([MCOpI](#) *mopi, int stp)
- void [addBridge](#) (int n0, int n1, int nex, int nextot)
- void [addArtic](#) (int nd, int mul)
- void [addBlock](#) ([MCBlock](#) *eblk, int stp)
- void [addBlockSelf](#) (int nd, int mul)
- void [print](#) (void)

Data Fields

- [MCOpi](#) * [opics](#)
- [MCBridge](#) * [bridges](#)
- [MCBlock](#) * [blocks](#)
- int * [articuls](#)
- int * [opisp](#)
- int * [opistk](#)
- [Edge2n](#) * [blksp](#)
- [Edge2n](#) * [blkstk](#)
- int [snodes](#)
- int [sedges](#)
- int [nopic](#)
- int [nlpopic](#)
- int [nctopic](#)
- int [nbridges](#)
- int [ne0bridges](#)
- int [ne1bridges](#)
- int [nselfloops](#)
- int [nblocks](#)
- int [na1blocks](#)
- int [narticuls](#)
- int [neblocks](#)
- int [nopisp](#)
- int [opistkptr](#)
- int [nblksp](#)
- int [blkstkptr](#)
- int [DUMMY_PADDING](#)

3.63.1 Detailed Description

Definition at line [950](#) of file [grcc.h](#).

3.63.2 Constructor & Destructor Documentation**3.63.2.1 MConn()**

```
MConn::MConn (
    int nnod,
    int nedg )
```

Definition at line [5128](#) of file [grcc.cc](#).

3.63.2.2 ~MConn()

```
MConn::~MConn (
    void )
```

Definition at line [5147](#) of file [grcc.cc](#).

3.63.3 Member Function Documentation

3.63.3.1 init()

```
void MConn::init (
    void )
```

Definition at line 5161 of file [grcc.cc](#).

3.63.3.2 pushNode()

```
void MConn::pushNode (
    int nd )
```

Definition at line 5189 of file [grcc.cc](#).

3.63.3.3 pushEdge()

```
void MConn::pushEdge (
    int n0,
    int n1 )
```

Definition at line 5198 of file [grcc.cc](#).

3.63.3.4 addOPic()

```
void MConn::addOPic (
    MCOpi * mopi,
    int stp )
```

Definition at line 5214 of file [grcc.cc](#).

3.63.3.5 addBridge()

```
void MConn::addBridge (
    int n0,
    int n1,
    int nex,
    int nextot )
```

Definition at line 5240 of file [grcc.cc](#).

3.63.3.6 addArtic()

```
void MConn::addArtic (
    int nd,
    int mul )
```

Definition at line 5257 of file [grcc.cc](#).

3.63.3.7 addBlock()

```
void MConn::addBlock (
    MCBLOCK * eblk,
    int stp )
```

Definition at line 5268 of file [grcc.cc](#).

3.63.3.8 addBlockSelf()

```
void MConn::addBlockSelf (
    int nd,
    int mul )
```

Definition at line 5292 of file [grcc.cc](#).

3.63.3.9 print()

```
void MConn::print (
    void )
```

Definition at line 5310 of file [grcc.cc](#).

3.63.4 Field Documentation

3.63.4.1 opics

```
MCOpI* MConn::opics
```

Definition at line 953 of file [grcc.h](#).

3.63.4.2 bridges

```
MCBridge* MConn::bridges
```

Definition at line 954 of file [grcc.h](#).

3.63.4.3 blocks

```
MCBlock* MConn::blocks
```

Definition at line 955 of file [grcc.h](#).

3.63.4.4 articuls

```
int* MConn::articuls
```

Definition at line 956 of file [grcc.h](#).

3.63.4.5 opisp

```
int* MConn::opisp
```

Definition at line 958 of file [grcc.h](#).

3.63.4.6 opistk

```
int* MConn::opistk
```

Definition at line 959 of file [grcc.h](#).

3.63.4.7 blksp

```
Edge2n* MConn::blksp
```

Definition at line 960 of file [grcc.h](#).

3.63.4.8 blkstk

```
Edge2n* MConn::blkstk
```

Definition at line 961 of file [grcc.h](#).

3.63.4.9 snodes

```
int MConn::snodes
```

Definition at line 962 of file [grcc.h](#).

3.63.4.10 sedges

```
int MConn::sedges
```

Definition at line 963 of file [grcc.h](#).

3.63.4.11 nopic

```
int MConn::nopic
```

Definition at line 966 of file [grcc.h](#).

3.63.4.12 nlpopic

```
int MConn::nlpopic
```

Definition at line 967 of file [grcc.h](#).

3.63.4.13 nctopic

```
int MConn::nctopic
```

Definition at line 968 of file [grcc.h](#).

3.63.4.14 nbridges

```
int MConn::nbridges
```

Definition at line 969 of file [grcc.h](#).

3.63.4.15 ne0bridges

```
int MConn::ne0bridges
```

Definition at line 970 of file [grcc.h](#).

3.63.4.16 ne1bridges

```
int MConn::ne1bridges
```

Definition at line 971 of file [grcc.h](#).

3.63.4.17 nselfloops

```
int MConn::nselfloops
```

Definition at line 972 of file [grcc.h](#).

3.63.4.18 nblocks

```
int MConn::nblocks
```

Definition at line 975 of file [grcc.h](#).

3.63.4.19 na1blocks

```
int MConn::na1blocks
```

Definition at line 976 of file [grcc.h](#).

3.63.4.20 narticuls

```
int MConn::narticuls
```

Definition at line 977 of file [grcc.h](#).

3.63.4.21 neblocks

```
int MConn::neblocks
```

Definition at line 978 of file [grcc.h](#).

3.63.4.22 nopisp

```
int MConn::nopisp
```

Definition at line 981 of file [grcc.h](#).

3.63.4.23 opistkptr

```
int MConn::opistkptr
```

Definition at line 982 of file [grcc.h](#).

3.63.4.24 nblksp

```
int MConn::nblksp
```

Definition at line 983 of file [grcc.h](#).

3.63.4.25 blkstkptr

```
int MConn::blkstkptr
```

Definition at line 984 of file [grcc.h](#).

3.63.4.26 DUMMYPADDING

```
int MConn::DUMMYPADDING
```

Definition at line 986 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.64 MCOpi Class Reference

Public Member Functions

- void [init](#) (void)

Data Fields

- int * [nodes](#)
- int [nnodes](#)
- int [nlegs](#)
- int [next](#)
- int [nedges](#)
- int [loop](#)
- int [ctloop](#)

3.64.1 Detailed Description

Definition at line [902](#) of file [grcc.h](#).

3.64.2 Constructor & Destructor Documentation

3.64.2.1 MCOpI()

```
MCOpI::MCOpI (  
    void )
```

Definition at line [5067](#) of file [grcc.cc](#).

3.64.2.2 ~MCOpI()

```
MCOpI::~~MCOpI (  
    void )
```

Definition at line [5073](#) of file [grcc.cc](#).

3.64.3 Member Function Documentation

3.64.3.1 init()

```
void MCOpI::init (  
    void )
```

Definition at line [5079](#) of file [grcc.cc](#).

3.64.4 Field Documentation

3.64.4.1 nodes

```
int* MCOpI::nodes
```

Definition at line [904](#) of file [grcc.h](#).

3.64.4.2 nnodes

```
int MCOpi::nnodes
```

Definition at line 905 of file [grcc.h](#).

3.64.4.3 nlegs

```
int MCOpi::nlegs
```

Definition at line 906 of file [grcc.h](#).

3.64.4.4 next

```
int MCOpi::next
```

Definition at line 907 of file [grcc.h](#).

3.64.4.5 nedges

```
int MCOpi::nedges
```

Definition at line 908 of file [grcc.h](#).

3.64.4.6 loop

```
int MCOpi::loop
```

Definition at line 909 of file [grcc.h](#).

3.64.4.7 ctloop

```
int MCOpi::ctloop
```

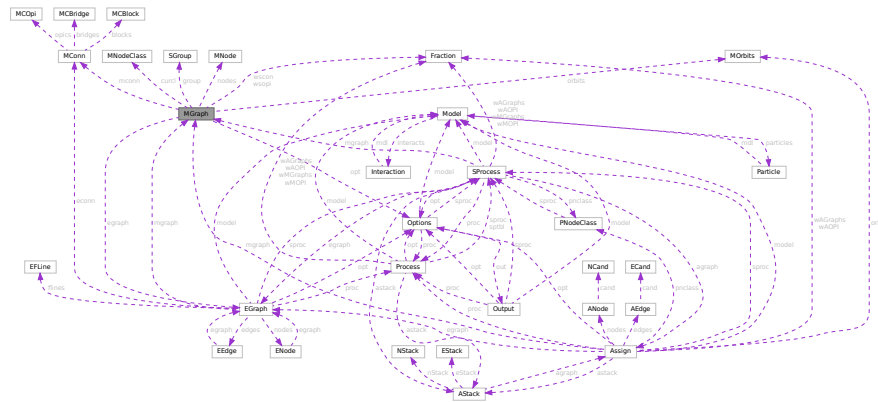
Definition at line 910 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.65 MGraph Class Reference

Collaboration diagram for MGraph:



Public Member Functions

- [MGraph](#) (int pid, int ncl, [NCInput](#) *mgi, [Options](#) *opt)
- [MGraph](#) (int pid, int ncl, int *cldeg, int *clnum, int *clexl, int *cmind, int *cmaxd, [Options](#) *opt)
- void [init](#) (void)
- [BigInt](#) [generate](#) (void)
- [Bool](#) [isExternal](#) (int nd)
- void [printAdjMat](#) ([MNodeClass](#) *cl)
- void [print](#) (void)
- [Bool](#) [isConnected](#) (void)
- [Bool](#) [visit](#) (int nd)
- [Bool](#) [isIsomorphic](#) ([MNodeClass](#) *cl)
- void [permMat](#) (int size, int *perm, int **mat0, int **mat1)
- int [compMat](#) (int size, int **mat0, int **mat1)
- [MNodeClass](#) * [refineClass](#) ([MNodeClass](#) *cl)
- void [bisearchME](#) (int nd, int pd, int ned, [MCopi](#) *mopi, [MCBlock](#) *mblk, int *next, int *nart)
- void [biconnME](#) (void)
- [Bool](#) [isOptM](#) (void)
- void [connectClass](#) ([MNodeClass](#) *cl)
- void [connectNode](#) (int sc, int ss, [MNodeClass](#) *cl)
- void [connectLeg](#) (int sc, int sn, int tc, int ts, [MNodeClass](#) *cl)
- void [newGraph](#) ([MNodeClass](#) *cl)

Data Fields

- [Options](#) * opt
- [SGroup](#) * group
- [MObits](#) * orbits
- [MNode](#) ** nodes
- [BigInt](#) mld
- int * clist
- int pld
- int nNodes

- int [nEdges](#)
- int [nLoops](#)
- int [nExtern](#)
- int [nClasses](#)
- int [mindeg](#)
- int [maxdeg](#)
- int ** [adjMat](#)
- [MNodeClass](#) * [curcl](#)
- [EGraph](#) * [egraph](#)
- [BigInt](#) [nsym](#)
- [BigInt](#) [esym](#)
- [BigInt](#) [cDiag](#)
- [BigInt](#) [c1PI](#)
- [BigInt](#) [cNoTadpole](#)
- [BigInt](#) [cNoTadBlock](#)
- [BigInt](#) [c1PINoTadBlock](#)
- [Fraction](#) [wscon](#)
- [Fraction](#) [wsopi](#)
- [BigInt](#) [ngen](#)
- [BigInt](#) [ngconn](#)
- [BigInt](#) [nCallRefine](#)
- [BigInt](#) [discardRefine](#)
- [BigInt](#) [discardDisc](#)
- [BigInt](#) [discardIso](#)
- [Bool](#) [opi](#)
- [Bool](#) [opiloop](#)
- [Bool](#) [extself](#)
- [Bool](#) [selfloop](#)
- [Bool](#) [tadpole](#)
- [Bool](#) [tadblock](#)
- [Bool](#) [block](#)
- int [DUMMYPADDING1](#)
- [MConn](#) * [mconn](#)
- int ** [modmat](#)
- int * [bidef](#)
- int * [bilow](#)
- int [bicount](#)
- int [DUMMYPADDING2](#)

3.65.1 Detailed Description

Definition at line [761](#) of file [grcc.h](#).

3.65.2 Constructor & Destructor Documentation

3.65.2.1 MGraph() [1/2]

```
MGraph::MGraph (
    int pid,
    int ncl,
    NCInput * mg,
    Options * opt )
```

Definition at line [3732](#) of file [grcc.cc](#).

3.65.2.2 MGraph() [2/2]

```
MGraph::MGraph (
    int pid,
    int ncl,
    int * cldeg,
    int * clnum,
    int * clexl,
    int * cmind,
    int * cmaxd,
    Options * opt )
```

Definition at line 3649 of file [grcc.cc](#).

3.65.2.3 ~MGraph()

```
MGraph::~MGraph (
    void )
```

Definition at line 3860 of file [grcc.cc](#).

3.65.3 Member Function Documentation

3.65.3.1 init()

```
void MGraph::init (
    void )
```

Definition at line 3817 of file [grcc.cc](#).

3.65.3.2 generate()

```
BigInt MGraph::generate (
    void )
```

Definition at line 4479 of file [grcc.cc](#).

3.65.3.3 isExternal()

```
Bool MGraph::isExternal (
    int nd ) [inline]
```

Definition at line 841 of file [grcc.h](#).

3.65.3.4 printAdjMat()

```
void MGraph::printAdjMat (
    MNodeClass * cl )
```

Definition at line 3889 of file [grcc.cc](#).

3.65.3.5 print()

```
void MGraph::print (
    void )
```

Definition at line [3908](#) of file [grcc.cc](#).

3.65.3.6 isConnected()

```
Bool MGraph::isConnected (
    void )
```

Definition at line [3932](#) of file [grcc.cc](#).

3.65.3.7 visit()

```
Bool MGraph::visit (
    int nd )
```

Definition at line [3956](#) of file [grcc.cc](#).

3.65.3.8 isIsomorphic()

```
Bool MGraph::isIsomorphic (
    MNodeClass * cl )
```

Definition at line [3981](#) of file [grcc.cc](#).

3.65.3.9 permMat()

```
void MGraph::permMat (
    int size,
    int * perm,
    int ** mat0,
    int ** mat1 )
```

Definition at line [4039](#) of file [grcc.cc](#).

3.65.3.10 compMat()

```
int MGraph::compMat (
    int size,
    int ** mat0,
    int ** mat1 )
```

Definition at line [4053](#) of file [grcc.cc](#).

3.65.3.11 refineClass()

```
MNodeClass * MGraph::refineClass (
    MNodeClass * cl )
```

Definition at line 4071 of file [grcc.cc](#).

3.65.3.12 bisearchME()

```
void MGraph::bisearchME (
    int nd,
    int pd,
    int ned,
    MCOpi * mopi,
    MCBLOCK * mblk,
    int * next,
    int * nart )
```

Definition at line 4239 of file [grcc.cc](#).

3.65.3.13 biconnME()

```
void MGraph::biconnME (
    void )
```

Definition at line 4153 of file [grcc.cc](#).

3.65.3.14 isOptM()

```
Bool MGraph::isOptM (
    void )
```

Definition at line 4731 of file [grcc.cc](#).

3.65.3.15 connectClass()

```
void MGraph::connectClass (
    MNodeClass * cl )
```

Definition at line 4505 of file [grcc.cc](#).

3.65.3.16 connectNode()

```
void MGraph::connectNode (
    int sc,
    int ss,
    MNodeClass * cl )
```

Definition at line 4537 of file [grcc.cc](#).

3.65.3.17 connectLeg()

```
void MGraph::connectLeg (
    int sc,
    int sn,
    int tc,
    int ts,
    MNodeClass * cl )
```

Definition at line [4558](#) of file [grcc.cc](#).

3.65.3.18 newGraph()

```
void MGraph::newGraph (
    MNodeClass * cl )
```

Definition at line [4815](#) of file [grcc.cc](#).

3.65.4 Field Documentation

3.65.4.1 opt

[Options*](#) MGraph::opt

Definition at line [766](#) of file [grcc.h](#).

3.65.4.2 group

[SGroup*](#) MGraph::group

Definition at line [769](#) of file [grcc.h](#).

3.65.4.3 orbits

[MOrbits*](#) MGraph::orbits

Definition at line [770](#) of file [grcc.h](#).

3.65.4.4 nodes

[MNode**](#) MGraph::nodes

Definition at line [772](#) of file [grcc.h](#).

3.65.4.5 mId

```
BigInt MGraph::mId
```

Definition at line 773 of file [grcc.h](#).

3.65.4.6 clist

```
int* MGraph::clist
```

Definition at line 774 of file [grcc.h](#).

3.65.4.7 pId

```
int MGraph::pId
```

Definition at line 775 of file [grcc.h](#).

3.65.4.8 nNodes

```
int MGraph::nNodes
```

Definition at line 776 of file [grcc.h](#).

3.65.4.9 nEdges

```
int MGraph::nEdges
```

Definition at line 777 of file [grcc.h](#).

3.65.4.10 nLoops

```
int MGraph::nLoops
```

Definition at line 778 of file [grcc.h](#).

3.65.4.11 nExtern

```
int MGraph::nExtern
```

Definition at line 779 of file [grcc.h](#).

3.65.4.12 nClasses

```
int MGraph::nClasses
```

Definition at line 780 of file [grcc.h](#).

3.65.4.13 mindeg

```
int MGraph::mindeg
```

Definition at line 781 of file [grcc.h](#).

3.65.4.14 maxdeg

```
int MGraph::maxdeg
```

Definition at line 782 of file [grcc.h](#).

3.65.4.15 adjMat

```
int** MGraph::adjMat
```

Definition at line 785 of file [grcc.h](#).

3.65.4.16 curcl

```
MNodeClass* MGraph::curcl
```

Definition at line 786 of file [grcc.h](#).

3.65.4.17 egraph

```
EGraph* MGraph::egraph
```

Definition at line 787 of file [grcc.h](#).

3.65.4.18 nsym

```
BigInt MGraph::nsym
```

Definition at line 788 of file [grcc.h](#).

3.65.4.19 esym

```
BigInt MGraph::esym
```

Definition at line 789 of file [grcc.h](#).

3.65.4.20 cDiag

```
BigInt MGraph::cDiag
```

Definition at line 792 of file [grcc.h](#).

3.65.4.21 c1PI

BigInt MGraph::c1PI

Definition at line 793 of file [grcc.h](#).

3.65.4.22 cNoTadpole

BigInt MGraph::cNoTadpole

Definition at line 794 of file [grcc.h](#).

3.65.4.23 cNoTadBlock

BigInt MGraph::cNoTadBlock

Definition at line 795 of file [grcc.h](#).

3.65.4.24 c1PInoTadBlock

BigInt MGraph::c1PInoTadBlock

Definition at line 796 of file [grcc.h](#).

3.65.4.25 wscon

Fraction MGraph::wscon

Definition at line 797 of file [grcc.h](#).

3.65.4.26 wsopi

Fraction MGraph::wsopi

Definition at line 798 of file [grcc.h](#).

3.65.4.27 ngen

BigInt MGraph::ngen

Definition at line 801 of file [grcc.h](#).

3.65.4.28 ngconn

BigInt MGraph::ngconn

Definition at line 802 of file [grcc.h](#).

3.65.4.29 nCallRefine

`BigInt MGraph::nCallRefine`

Definition at line 804 of file [grcc.h](#).

3.65.4.30 discardRefine

`BigInt MGraph::discardRefine`

Definition at line 805 of file [grcc.h](#).

3.65.4.31 discardDisc

`BigInt MGraph::discardDisc`

Definition at line 806 of file [grcc.h](#).

3.65.4.32 discardIso

`BigInt MGraph::discardIso`

Definition at line 807 of file [grcc.h](#).

3.65.4.33 opi

`Bool MGraph::opi`

Definition at line 810 of file [grcc.h](#).

3.65.4.34 opiloop

`Bool MGraph::opiloop`

Definition at line 811 of file [grcc.h](#).

3.65.4.35 extself

`Bool MGraph::extself`

Definition at line 812 of file [grcc.h](#).

3.65.4.36 selfloop

`Bool MGraph::selfloop`

Definition at line 813 of file [grcc.h](#).

3.65.4.37 tadpole

`Bool MGraph::tadpole`

Definition at line 814 of file [grcc.h](#).

3.65.4.38 tadblock

`Bool MGraph::tadblock`

Definition at line 815 of file [grcc.h](#).

3.65.4.39 block

`Bool MGraph::block`

Definition at line 816 of file [grcc.h](#).

3.65.4.40 DUMMYPADDING1

`int MGraph::DUMMYPADDING1`

Definition at line 818 of file [grcc.h](#).

3.65.4.41 mconn

`MConn* MGraph::mconn`

Definition at line 821 of file [grcc.h](#).

3.65.4.42 modmat

`int** MGraph::modmat`

Definition at line 824 of file [grcc.h](#).

3.65.4.43 bidef

`int* MGraph::bidef`

Definition at line 827 of file [grcc.h](#).

3.65.4.44 bilow

`int* MGraph::bilow`

Definition at line 828 of file [grcc.h](#).

3.65.4.45 bicount

```
int MGraph::bicount
```

Definition at line 829 of file [grcc.h](#).

3.65.4.46 DUMMYPADDING2

```
int MGraph::DUMMYPADDING2
```

Definition at line 831 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.66 MiNmAx Struct Reference

Data Fields

- WORD [mini](#)
- WORD [maxi](#)
- WORD [size](#)

3.66.1 Detailed Description

Definition at line 307 of file [structs.h](#).

3.66.2 Field Documentation

3.66.2.1 mini

```
WORD MiNmAx::mini
```

Minimum value

Definition at line 308 of file [structs.h](#).

3.66.2.2 maxi

```
WORD MiNmAx::maxi
```

Maximum value

Definition at line 309 of file [structs.h](#).

3.66.2.3 size

```
WORD MinMax::size
```

Value of one unit in this position.

Definition at line 310 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.67 MInput Struct Reference

Data Fields

- const char * [name](#)
- int [defpart](#)
- int [ncouple](#)
- const char * [cnamlist](#) [GRCC_MAXNCPLG]

3.67.1 Detailed Description

Definition at line 119 of file [grccparam.h](#).

3.67.2 Field Documentation

3.67.2.1 name

```
const char* MInput::name
```

Definition at line 120 of file [grccparam.h](#).

3.67.2.2 defpart

```
int MInput::defpart
```

Definition at line 121 of file [grccparam.h](#).

3.67.2.3 ncouple

```
int MInput::ncouple
```

Definition at line 123 of file [grccparam.h](#).

3.67.2.4 cnamlist

```
const char* MInput::cnamlist[GRCC_MAXNCPLG]
```

Definition at line 124 of file [grccparam.h](#).

The documentation for this struct was generated from the following file:

- [grccparam.h](#)

3.68 MNode Class Reference

Public Member Functions

- [MNode](#) (int id, int clss, [NCInput](#) *mgi)
- [MNode](#) (int id, int deg, int extloop, int clss, int cmind, int cmaxd)

Data Fields

- int [id](#)
- int [deg](#)
- int [clss](#)
- int [extloop](#)
- int [cminddeg](#)
- int [cmaxdeg](#)
- int [freelg](#)
- int [visited](#)

3.68.1 Detailed Description

Definition at line 739 of file [grcc.h](#).

3.68.2 Constructor & Destructor Documentation

3.68.2.1 MNode() [1/2]

```
MNode::MNode (  
    int id,  
    int clss,  
    NCInput * mgi )
```

Definition at line 3605 of file [grcc.cc](#).

3.68.2.2 MNode() [2/2]

```
MNode::MNode (
    int id,
    int deg,
    int extloop,
    int cls,
    int cmin,
    int cmax )
```

Definition at line [3626](#) of file [grcc.cc](#).

3.68.3 Field Documentation

3.68.3.1 id

```
int MNode::id
```

Definition at line [741](#) of file [grcc.h](#).

3.68.3.2 deg

```
int MNode::deg
```

Definition at line [742](#) of file [grcc.h](#).

3.68.3.3 cls

```
int MNode::cls
```

Definition at line [743](#) of file [grcc.h](#).

3.68.3.4 extloop

```
int MNode::extloop
```

Definition at line [744](#) of file [grcc.h](#).

3.68.3.5 cmindeg

```
int MNode::cmindeg
```

Definition at line [745](#) of file [grcc.h](#).

3.68.3.6 cmaxdeg

```
int MNode::cmaxdeg
```

Definition at line 746 of file [grcc.h](#).

3.68.3.7 freelg

```
int MNode::freelg
```

Definition at line 748 of file [grcc.h](#).

3.68.3.8 visited

```
int MNode::visited
```

Definition at line 749 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.69 MNodeClass Class Reference

Public Member Functions

- [MNodeClass](#) (int nnodes, int nclasses)
- void [init](#) (int *cl, int mxdeg, int **adjmat)
- void [copy](#) ([MNodeClass](#) *mnc)
- int [clCmp](#) (int nd0, int nd1, int cn)
- void [printMat](#) (void)
- void [mkFlist](#) (void)
- void [mkNdCl](#) (void)
- void [mkClMat](#) (int **adjmat)
- void [incMat](#) (int nd, int td, int val)
- int [cmpMNCArray](#) (int *a0, int *a1, int ma)
- void [reorder](#) ([MGraph](#) *mg)

Data Fields

- int [clmat](#) [GRCC_MAXNODES][GRCC_MAXNODES]
- int [clist](#) [GRCC_MAXNODES]
- int [ndcl](#) [GRCC_MAXNODES]
- int [flist](#) [GRCC_MAXNODES+1]
- int [clord](#) [GRCC_MAXNODES]
- int [cmindeg](#) [GRCC_MAXNODES]
- int [cmaxdeg](#) [GRCC_MAXNODES]
- int [nNodes](#)
- int [nClasses](#)
- int [maxdeg](#)
- int [flg0](#)
- int [flg1](#)
- int [flg2](#)

3.69.1 Detailed Description

Definition at line 865 of file [grcc.h](#).

3.69.2 Constructor & Destructor Documentation

3.69.2.1 MNodeClass()

```
MNodeClass::MNodeClass (
    int nnodes,
    int nclasses )
```

Definition at line 3358 of file [grcc.cc](#).

3.69.2.2 ~MNodeClass()

```
MNodeClass::~MNodeClass (
    void )
```

Definition at line 3387 of file [grcc.cc](#).

3.69.3 Member Function Documentation

3.69.3.1 init()

```
void MNodeClass::init (
    int * cl,
    int mxdeg,
    int ** adjmat )
```

Definition at line 3394 of file [grcc.cc](#).

3.69.3.2 copy()

```
void MNodeClass::copy (
    MNodeClass * mnc )
```

Definition at line 3418 of file [grcc.cc](#).

3.69.3.3 clCmp()

```
int MNodeClass::clCmp (
    int nd0,
    int nd1,
    int cn )
```

Definition at line 3474 of file [grcc.cc](#).

3.69.3.4 printMat()

```
void MNodeClass::printMat (
    void )
```

Definition at line [3525](#) of file [grcc.cc](#).

3.69.3.5 mkFlist()

```
void MNodeClass::mkFlist (
    void )
```

Definition at line [3442](#) of file [grcc.cc](#).

3.69.3.6 mkNdCl()

```
void MNodeClass::mkNdCl (
    void )
```

Definition at line [3458](#) of file [grcc.cc](#).

3.69.3.7 mkClMat()

```
void MNodeClass::mkClMat (
    int ** adjmat )
```

Definition at line [3496](#) of file [grcc.cc](#).

3.69.3.8 incMat()

```
void MNodeClass::incMat (
    int nd,
    int td,
    int val )
```

Definition at line [3516](#) of file [grcc.cc](#).

3.69.3.9 cmpMNCArray()

```
int MNodeClass::cmpMNCArray (
    int * a0,
    int * a1,
    int ma )
```

Definition at line [3556](#) of file [grcc.cc](#).

3.69.3.10 reorder()

```
void MNodeClass::reorder (
    MGraph * mg )
```

Definition at line 3572 of file [grcc.cc](#).

3.69.4 Field Documentation

3.69.4.1 clmat

```
int MNodeClass::clmat[GRCC_MAXNODES][GRCC_MAXNODES]
```

Definition at line 867 of file [grcc.h](#).

3.69.4.2 clist

```
int MNodeClass::clist[GRCC_MAXNODES]
```

Definition at line 868 of file [grcc.h](#).

3.69.4.3 ndcl

```
int MNodeClass::ndcl[GRCC_MAXNODES]
```

Definition at line 869 of file [grcc.h](#).

3.69.4.4 flist

```
int MNodeClass::flist[GRCC_MAXNODES+1]
```

Definition at line 870 of file [grcc.h](#).

3.69.4.5 clord

```
int MNodeClass::clord[GRCC_MAXNODES]
```

Definition at line 871 of file [grcc.h](#).

3.69.4.6 cmindeg

```
int MNodeClass::cmindeg[GRCC_MAXNODES]
```

Definition at line 872 of file [grcc.h](#).

3.69.4.7 cmaxdeg

```
int MNodeClass::cmaxdeg[GRCC_MAXNODES]
```

Definition at line 873 of file [grcc.h](#).

3.69.4.8 nNodes

```
int MNodeClass::nNodes
```

Definition at line 874 of file [grcc.h](#).

3.69.4.9 nClasses

```
int MNodeClass::nClasses
```

Definition at line 875 of file [grcc.h](#).

3.69.4.10 maxdeg

```
int MNodeClass::maxdeg
```

Definition at line 876 of file [grcc.h](#).

3.69.4.11 flg0

```
int MNodeClass::flg0
```

Definition at line 878 of file [grcc.h](#).

3.69.4.12 flg1

```
int MNodeClass::flg1
```

Definition at line 879 of file [grcc.h](#).

3.69.4.13 flg2

```
int MNodeClass::flg2
```

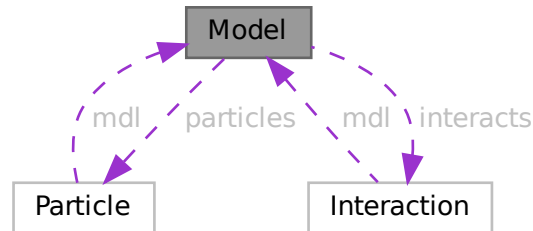
Definition at line 880 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.70 Model Class Reference

Collaboration diagram for Model:



Public Member Functions

- [Model](#) ([MInput](#) *minp)
- void [prModel](#) (void)
- void [addParticle](#) ([PInput](#) *pinp)
- void [addParticleEnd](#) (void)
- void [addInteraction](#) ([IInput](#) *iinp)
- void [addInteractionEnd](#) (void)
- int [findParticleName](#) (const char *name)
- int [findParticleCode](#) (int pcd)
- int [findInteractionName](#) (const char *name)
- int [findInteractionCode](#) (int icd)
- char * [particleName](#) (int p)
- int [particleCode](#) (int p)
- int [normalParticle](#) (int pt)
- int [antiParticle](#) (int pt)
- int * [allParticles](#) (int *len)
- int [findMClass](#) (const int cpl, const int dgr)
- void [prParticleArray](#) (int n, int *a, const char *msg)

Data Fields

- char * [name](#)
- char ** [cnlist](#)
- int [ncouple](#)
- int [nParticles](#)
- [Particle](#) ** [particles](#)
- int [pdef](#)
- int [nallPart](#)
- int [allPart](#) [GRCC_MAXMPARTICLES2]
- int [nintPart](#)
- int [intPart](#) [GRCC_MAXMPARTICLES2]
- int [nInteracts](#)

- [Interaction](#) ** [interacts](#)
- int [vdef](#)
- int [maxnlegs](#)
- int [maxcpl](#)
- int [maxloop](#)
- int * [cplgcp](#)
- int * [cplglg](#)
- int * [cplgnvl](#)
- int ** [cplgvl](#)
- int [ncplgcp](#)
- int [defpart](#)
- int [skipFLine](#)
- int [DUMMYPADDING](#)

3.70.1 Detailed Description

Definition at line [262](#) of file [grcc.h](#).

3.70.2 Constructor & Destructor Documentation

3.70.2.1 Model()

```
Model::Model (
    MInput * minp )
```

Definition at line [1726](#) of file [grcc.cc](#).

3.70.2.2 ~Model()

```
Model::~~Model (
    void )
```

Definition at line [1787](#) of file [grcc.cc](#).

3.70.3 Member Function Documentation

3.70.3.1 prModel()

```
void Model::prModel (
    void )
```

Definition at line [1827](#) of file [grcc.cc](#).

3.70.3.2 addParticle()

```
void Model::addParticle (
    PInput * pinp )
```

Definition at line [1876](#) of file [grcc.cc](#).

3.70.3.3 addParticleEnd()

```
void Model::addParticleEnd (
    void )
```

Definition at line 1918 of file [grcc.cc](#).

3.70.3.4 addInteraction()

```
void Model::addInteraction (
    IInput * iinp )
```

Definition at line 1956 of file [grcc.cc](#).

3.70.3.5 addInteractionEnd()

```
void Model::addInteractionEnd (
    void )
```

Definition at line 2084 of file [grcc.cc](#).

3.70.3.6 findParticleName()

```
int Model::findParticleName (
    const char * name )
```

Definition at line 2157 of file [grcc.cc](#).

3.70.3.7 findParticleCode()

```
int Model::findParticleCode (
    int pcd )
```

Definition at line 2175 of file [grcc.cc](#).

3.70.3.8 findInteractionName()

```
int Model::findInteractionName (
    const char * name )
```

Definition at line 2294 of file [grcc.cc](#).

3.70.3.9 findInteractionCode()

```
int Model::findInteractionCode (
    int icd )
```

Definition at line 2307 of file [grcc.cc](#).

3.70.3.10 `particleName()`

```
char * Model::particleName (
    int p )
```

Definition at line [2190](#) of file [grcc.cc](#).

3.70.3.11 `particleCode()`

```
int Model::particleCode (
    int p )
```

Definition at line [2210](#) of file [grcc.cc](#).

3.70.3.12 `normalParticle()`

```
int Model::normalParticle (
    int pt )
```

Definition at line [2256](#) of file [grcc.cc](#).

3.70.3.13 `antiParticle()`

```
int Model::antiParticle (
    int pt )
```

Definition at line [2273](#) of file [grcc.cc](#).

3.70.3.14 `allParticles()`

```
int * Model::allParticles (
    int * len )
```

Definition at line [2287](#) of file [grcc.cc](#).

3.70.3.15 `findMClass()`

```
int Model::findMClass (
    const int cpl,
    const int dgr )
```

Definition at line [2243](#) of file [grcc.cc](#).

3.70.3.16 `prParticleArray()`

```
void Model::prParticleArray (
    int n,
    int * a,
    const char * msg )
```

Definition at line [2228](#) of file [grcc.cc](#).

3.70.4 Field Documentation

3.70.4.1 name

```
char* Model::name
```

Definition at line 264 of file [grcc.h](#).

3.70.4.2 cnlist

```
char** Model::cnlist
```

Definition at line 265 of file [grcc.h](#).

3.70.4.3 ncouple

```
int Model::ncouple
```

Definition at line 266 of file [grcc.h](#).

3.70.4.4 nParticles

```
int Model::nParticles
```

Definition at line 268 of file [grcc.h](#).

3.70.4.5 particles

```
Particle** Model::particles
```

Definition at line 269 of file [grcc.h](#).

3.70.4.6 pdef

```
int Model::pdef
```

Definition at line 270 of file [grcc.h](#).

3.70.4.7 nallPart

```
int Model::nallPart
```

Definition at line 273 of file [grcc.h](#).

3.70.4.8 allPart

```
int Model::allPart[GRCC_MAXMPARTICLES2]
```

Definition at line 274 of file [grcc.h](#).

3.70.4.9 nintPart

```
int Model::nintPart
```

Definition at line 277 of file [grcc.h](#).

3.70.4.10 intPart

```
int Model::intPart[GRCC_MAXMPARTICLES2]
```

Definition at line 278 of file [grcc.h](#).

3.70.4.11 nInteracts

```
int Model::nInteracts
```

Definition at line 280 of file [grcc.h](#).

3.70.4.12 interacts

```
Interaction** Model::interacts
```

Definition at line 281 of file [grcc.h](#).

3.70.4.13 vdef

```
int Model::vdef
```

Definition at line 282 of file [grcc.h](#).

3.70.4.14 maxnlegs

```
int Model::maxnlegs
```

Definition at line 284 of file [grcc.h](#).

3.70.4.15 maxcpl

```
int Model::maxcpl
```

Definition at line 285 of file [grcc.h](#).

3.70.4.16 maxloop

```
int Model::maxloop
```

Definition at line 286 of file [grcc.h](#).

3.70.4.17 cplgcp

```
int* Model::cplgcp
```

Definition at line 289 of file [grcc.h](#).

3.70.4.18 cplglg

```
int* Model::cplglg
```

Definition at line 290 of file [grcc.h](#).

3.70.4.19 cplgnvl

```
int* Model::cplgnvl
```

Definition at line 291 of file [grcc.h](#).

3.70.4.20 cplgvl

```
int** Model::cplgvl
```

Definition at line 292 of file [grcc.h](#).

3.70.4.21 ncplgcp

```
int Model::ncplgcp
```

Definition at line 293 of file [grcc.h](#).

3.70.4.22 defpart

```
int Model::defpart
```

Definition at line 295 of file [grcc.h](#).

3.70.4.23 skipFLine

```
int Model::skipFLine
```

Definition at line 296 of file [grcc.h](#).

3.70.4.24 DUMMYPADDING

```
int Model::DUMMYPADDING
```

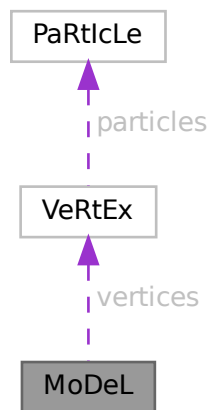
Definition at line 298 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.71 MoDeL Struct Reference

Collaboration diagram for MoDeL:



Data Fields

- [VERTEX](#) ** [vertices](#)
- WORD * [couplings](#)
- UBYTE * [name](#)
- void * [grccmodel](#)
- WORD [legcouple](#) [MAXLEGS+2]
- WORD [nparticles](#)
- WORD [nvertices](#)
- WORD [invertices](#)
- WORD [sizevertices](#)
- WORD [sizecouplings](#)
- WORD [ncouplings](#)
- WORD [error](#)
- WORD [dummy](#)

3.71.1 Detailed Description

Definition at line 637 of file [structs.h](#).

3.71.2 Field Documentation

3.71.2.1 vertices

```
VERTEX** MoDeL::vertices
```

Definition at line 638 of file [structs.h](#).

3.71.2.2 couplings

```
WORD* MoDeL::couplings
```

Definition at line 639 of file [structs.h](#).

3.71.2.3 name

```
UBYTE* MoDeL::name
```

Definition at line 640 of file [structs.h](#).

3.71.2.4 grccmodel

```
void* MoDeL::grccmodel
```

Definition at line 641 of file [structs.h](#).

3.71.2.5 legcouple

```
WORD MoDeL::legcouple[MAXLEGS+2]
```

Definition at line 642 of file [structs.h](#).

3.71.2.6 nparticles

```
WORD MoDeL::nparticles
```

Definition at line 643 of file [structs.h](#).

3.71.2.7 nvertices

WORD MoDeL::nvertices

Definition at line 644 of file [structs.h](#).

3.71.2.8 invertices

WORD MoDeL::invertices

Definition at line 645 of file [structs.h](#).

3.71.2.9 sizevertices

WORD MoDeL::sizevertices

Definition at line 646 of file [structs.h](#).

3.71.2.10 sizecouplings

WORD MoDeL::sizecouplings

Definition at line 647 of file [structs.h](#).

3.71.2.11 ncouplings

WORD MoDeL::ncouplings

Definition at line 648 of file [structs.h](#).

3.71.2.12 error

WORD MoDeL::error

Definition at line 649 of file [structs.h](#).

3.71.2.13 dummy

WORD MoDeL::dummy

Definition at line 650 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.72 MODNUM Struct Reference

Data Fields

- UWORD * [a](#)
- UWORD * [m](#)
- WORD [na](#)
- WORD [nm](#)

3.72.1 Detailed Description

Definition at line [1301](#) of file [structs.h](#).

3.72.2 Field Documentation

3.72.2.1 [a](#)

UWORD* MODNUM::a

Definition at line [1302](#) of file [structs.h](#).

3.72.2.2 [m](#)

UWORD* MODNUM::m

Definition at line [1303](#) of file [structs.h](#).

3.72.2.3 [na](#)

WORD MODNUM::na

Definition at line [1304](#) of file [structs.h](#).

3.72.2.4 [nm](#)

WORD MODNUM::nm

Definition at line [1305](#) of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.73 MoDoPtDoLIaRS Struct Reference

Data Fields

- WORD [number](#)
- WORD [type](#)

3.73.1 Detailed Description

Definition at line [556](#) of file [structs.h](#).

3.73.2 Field Documentation

3.73.2.1 number

WORD MoDoPtDoLIaRS::number

Definition at line [560](#) of file [structs.h](#).

3.73.2.2 type

WORD MoDoPtDoLIaRS::type

Definition at line [561](#) of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.74 monomial_larger Class Reference

Public Member Functions

- bool [operator\(\)](#) (const WORD *a, const WORD *b)

3.74.1 Detailed Description

Definition at line [179](#) of file [poly.h](#).

3.74.2 Constructor & Destructor Documentation

3.74.2.1 monomial_larger()

```
monomial_larger::monomial_larger ( ) [inline]
```

Definition at line [184](#) of file [poly.h](#).

3.74.3 Member Function Documentation

3.74.3.1 operator()

```
bool monomial_larger::operator() (
    const WORD * a,
    const WORD * b ) [inline]
```

Definition at line 190 of file [poly.h](#).

The documentation for this class was generated from the following file:

- [poly.h](#)

3.75 MORbits Class Reference

Public Member Functions

- void [print](#) (void)
- void [initPerm](#) (int nnodes)
- void [fromPerm](#) (int *perm)
- void [toOrbits](#) (void)

Data Fields

- int [nOrbits](#)
- int [nNodes](#)
- int [nd2or](#) [GRCC_MAXNODES]
- int [or2nd](#) [GRCC_MAXNODES]
- int [flist](#) [GRCC_MAXNODES+1]

3.75.1 Detailed Description

Definition at line 1005 of file [grcc.h](#).

3.75.2 Constructor & Destructor Documentation

3.75.2.1 MORbits()

```
MORbits::MORbits (
    void )
```

Definition at line 4964 of file [grcc.cc](#).

3.75.2.2 ~MOrbits()

```
MOrbits::~MOrbits (
    void )
```

Definition at line [4971](#) of file [grcc.cc](#).

3.75.3 Member Function Documentation

3.75.3.1 print()

```
void MOrbits::print (
    void )
```

Definition at line [4976](#) of file [grcc.cc](#).

3.75.3.2 initPerm()

```
void MOrbits::initPerm (
    int nnodes )
```

Definition at line [4989](#) of file [grcc.cc](#).

3.75.3.3 fromPerm()

```
void MOrbits::fromPerm (
    int * perm )
```

Definition at line [5000](#) of file [grcc.cc](#).

3.75.3.4 toOrbits()

```
void MOrbits::toOrbits (
    void )
```

Definition at line [5039](#) of file [grcc.cc](#).

3.75.4 Field Documentation

3.75.4.1 nOrbits

```
int MOrbits::nOrbits
```

Definition at line [1017](#) of file [grcc.h](#).

3.75.4.2 nNodes

```
int MORbits::nNodes
```

Definition at line 1018 of file [grcc.h](#).

3.75.4.3 nd2or

```
int MORbits::nd2or[GRCC_MAXNODES]
```

Definition at line 1019 of file [grcc.h](#).

3.75.4.4 or2nd

```
int MORbits::or2nd[GRCC_MAXNODES]
```

Definition at line 1020 of file [grcc.h](#).

3.75.4.5 flist

```
int MORbits::flist[GRCC_MAXNODES+1]
```

Definition at line 1021 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.76 flint::mpoly Class Reference

Public Member Functions

- [mpoly](#) (fmpz_mpoly_ctx_struct *ctx_in)
- void [print](#) (const string &text)

Data Fields

- fmpz_mpoly_t [d](#)

3.76.1 Detailed Description

Definition at line 76 of file [flintinterface.h](#).

3.76.2 Constructor & Destructor Documentation

3.76.2.1 mpoly()

```
flint::mpoly::mpoly (
    fmpz_mpoly_ctx_struct * ctx_in ) [inline], [explicit]
```

Definition at line 81 of file [flintinterface.h](#).

3.76.2.2 ~mpoly()

```
flint::mpoly::~~mpoly ( ) [inline]
```

Definition at line 82 of file [flintinterface.h](#).

3.76.3 Member Function Documentation

3.76.3.1 print()

```
void flint::mpoly::print (
    const string & text ) [inline]
```

Definition at line 83 of file [flintinterface.h](#).

3.76.4 Field Documentation

3.76.4.1 d

```
fmpz_mpoly_t flint::mpoly::d
```

Definition at line 80 of file [flintinterface.h](#).

The documentation for this class was generated from the following file:

- [flintinterface.h](#)

3.77 flint::mpoly_ctx Class Reference

Public Member Functions

- [mpoly_ctx](#) (int64_t nvars)

Data Fields

- [fmpz_mpoly_ctx_t](#) d

3.77.1 Detailed Description

Definition at line 99 of file [flintinterface.h](#).

3.77.2 Constructor & Destructor Documentation

3.77.2.1 mpoly_ctx()

```
flint::mpoly_ctx::mpoly_ctx (
    int64_t nvars ) [inline], [explicit]
```

Definition at line 102 of file [flintinterface.h](#).

3.77.2.2 ~mpoly_ctx()

```
flint::mpoly_ctx::~~mpoly_ctx ( ) [inline]
```

Definition at line 103 of file [flintinterface.h](#).

3.77.3 Field Documentation

3.77.3.1 d

```
fmprz_mpoly_ctx_t flint::mpoly_ctx::d
```

Definition at line 101 of file [flintinterface.h](#).

The documentation for this class was generated from the following file:

- [flintinterface.h](#)

3.78 flint::mpoly_factor Class Reference

Public Member Functions

- [mpoly_factor](#) (fmprz_mpoly_ctx_struct *ctx_in)

Data Fields

- fmprz_mpoly_factor_t [d](#)

3.78.1 Detailed Description

Definition at line 89 of file [flintinterface.h](#).

3.78.2 Constructor & Destructor Documentation

3.78.2.1 mpoly_factor()

```
flint::mpoly_factor::mpoly_factor (
    fmpz_mpoly_ctx_struct * ctx_in ) [inline], [explicit]
```

Definition at line 94 of file [flintinterface.h](#).

3.78.2.2 ~mpoly_factor()

```
flint::mpoly_factor::~~mpoly_factor ( ) [inline]
```

Definition at line 97 of file [flintinterface.h](#).

3.78.3 Field Documentation

3.78.3.1 d

```
fmpz_mpoly_factor_t flint::mpoly_factor::d
```

Definition at line 93 of file [flintinterface.h](#).

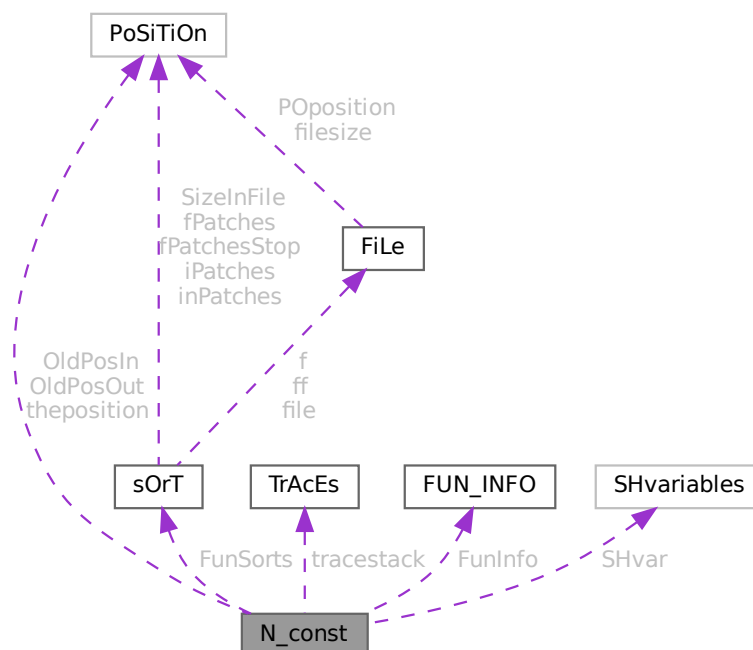
The documentation for this class was generated from the following file:

- [flintinterface.h](#)

3.79 N_const Struct Reference

```
#include <structs.h>
```

Collaboration diagram for N_const:



Public Member Functions

- **PADPOSITION** (50, 7, 23, 26, sizeof([SHvariables](#)))

Data Fields

- [POSITION](#) OldPosIn
- [POSITION](#) OldPosOut
- [POSITION](#) theposition
- WORD * [EndNest](#)
- WORD * [Frozen](#)
- WORD * [FullProto](#)
- WORD * [cTerm](#)
- int * [RepPoint](#)
- WORD * [WildValue](#)
- WORD * [WildStop](#)
- WORD * [argaddress](#)
- WORD * [RepFunList](#)
- WORD * [patstop](#)
- WORD * [terstop](#)
- WORD * [terstart](#)
- WORD * [terfirstcomm](#)
- WORD * [DumFound](#)
- WORD ** [DumPlace](#)
- WORD ** [DumFunPlace](#)
- WORD * [UsedSymbol](#)
- WORD * [UsedVector](#)
- WORD * [UsedIndex](#)
- WORD * [UsedFunction](#)
- WORD * [MaskPointer](#)
- WORD * [ForFindOnly](#)
- WORD * [findTerm](#)
- WORD * [findPattern](#)
- WORD * [dummyrenumlist](#)
- int * [funargs](#)
- WORD ** [funlocs](#)
- int * [funinds](#)
- UWORD * [NoScrat2](#)
- WORD * [ReplaceScrat](#)
- [TRACES](#) * [tracestack](#)
- WORD * [selecttermundo](#)
- WORD * [patternbuffer](#)
- WORD * [termbuffer](#)
- WORD ** [PoinScratch](#)
- WORD ** [FunScratch](#)
- WORD * [RenumScratch](#)
- [FUN_INFO](#) * [FunInfo](#)
- WORD ** [SplitScratch](#)
- WORD ** [SplitScratch1](#)
- [SORTING](#) ** [FunSorts](#)
- UWORD * [SoScratC](#)
- WORD * [listinprint](#)
- WORD * [currentTerm](#)
- WORD ** [arglist](#)

- int * [tlistbuf](#)
- WORD * [compressSpace](#)
- UWORD * [SHcombi](#)
- WORD * [poly_vars](#)
- UWORD * [cmod](#)
- [SHvariables](#) SHvar
- LONG [deferskipped](#)
- LONG [InScratch](#)
- LONG [SplitScratchSize](#)
- LONG [InScratch1](#)
- LONG [SplitScratchSize1](#)
- LONG [ninterms](#)
- LONG [SHcombisize](#)
- int [NumTotWildArgs](#)
- int [UseFindOnly](#)
- int [UsedOtherFind](#)
- int [ErrorInDollar](#)
- int [numfargs](#)
- int [numflocs](#)
- int [nargs](#)
- int [tohunt](#)
- int [numoffuns](#)
- int [funisize](#)
- int [Rssize](#)
- int [numtracesctack](#)
- int [intracestack](#)
- int [numfuninfo](#)
- int [NumFunSorts](#)
- int [MaxFunSorts](#)
- int [arglistsize](#)
- int [tlistsize](#)
- int [filenum](#)
- int [compressSize](#)
- int [polysortflag](#)
- int [nogroundlevel](#)
- int [subsubveto](#)
- WORD [MaxRenumScratch](#)
- WORD [oldtype](#)
- WORD [oldvalue](#)
- WORD [NumWild](#)
- WORD [RepFunNum](#)
- WORD [DisOrderFlag](#)
- WORD [WildDirt](#)
- WORD [NumFound](#)
- WORD [WildReserve](#)
- WORD [TelInFun](#)
- WORD [TeSuOut](#)
- WORD [WildArgs](#)
- WORD [WildEat](#)
- WORD [PolyNormFlag](#)
- WORD [PolyFunTodo](#)
- WORD [sizeselecttermundo](#)
- WORD [patternbuffersize](#)
- WORD [numlistinprint](#)
- WORD [ncmod](#)

- WORD [ExpectedSign](#)
- WORD [SignCheck](#)
- WORD [IndDum](#)
- WORD [poly_num_vars](#)
- WORD [idfunctionflag](#)
- WORD [poly_vars_type](#)
- WORD [tryterm](#)

3.79.1 Detailed Description

The [N_const](#) struct is part of the global data and resides either in the ALLGLOBALS struct A, or the ALLPRIVATES struct B (TFORM) under the name N We see it used with the macro AN as in AN.RepFunNum It has variables that are private to each thread and are used as temporary storage during the expansion of the terms tree.

Definition at line [2276](#) of file [structs.h](#).

3.79.2 Field Documentation

3.79.2.1 OldPosIn

[POSITION](#) [N_const::OldPosIn](#)

Definition at line [2277](#) of file [structs.h](#).

3.79.2.2 OldPosOut

[POSITION](#) [N_const::OldPosOut](#)

Definition at line [2278](#) of file [structs.h](#).

3.79.2.3 theposition

[POSITION](#) [N_const::theposition](#)

Definition at line [2279](#) of file [structs.h](#).

3.79.2.4 EndNest

[WORD*](#) [N_const::EndNest](#)

Definition at line [2280](#) of file [structs.h](#).

3.79.2.5 Frozen

[WORD*](#) [N_const::Frozen](#)

Definition at line [2281](#) of file [structs.h](#).

3.79.2.6 FullProto

WORD* N_const::FullProto

Definition at line 2282 of file [structs.h](#).

3.79.2.7 cTerm

WORD* N_const::cTerm

Definition at line 2283 of file [structs.h](#).

3.79.2.8 RepPoint

int* N_const::RepPoint

Definition at line 2284 of file [structs.h](#).

3.79.2.9 WildValue

WORD* N_const::WildValue

Definition at line 2285 of file [structs.h](#).

3.79.2.10 WildStop

WORD* N_const::WildStop

Definition at line 2286 of file [structs.h](#).

3.79.2.11 argaddress

WORD* N_const::argaddress

Definition at line 2287 of file [structs.h](#).

3.79.2.12 RepFunList

WORD* N_const::RepFunList

Definition at line 2288 of file [structs.h](#).

3.79.2.13 patstop

WORD* N_const::patstop

Definition at line 2289 of file [structs.h](#).

3.79.2.14 terstop

WORD* N_const::terstop

Definition at line 2290 of file [structs.h](#).

3.79.2.15 terstart

WORD* N_const::terstart

Definition at line 2291 of file [structs.h](#).

3.79.2.16 terfirstcomm

WORD* N_const::terfirstcomm

Definition at line 2292 of file [structs.h](#).

3.79.2.17 DumFound

WORD* N_const::DumFound

Definition at line 2293 of file [structs.h](#).

3.79.2.18 DumPlace

WORD** N_const::DumPlace

Definition at line 2294 of file [structs.h](#).

3.79.2.19 DumFunPlace

WORD** N_const::DumFunPlace

Definition at line 2295 of file [structs.h](#).

3.79.2.20 UsedSymbol

WORD* N_const::UsedSymbol

Definition at line 2296 of file [structs.h](#).

3.79.2.21 UsedVector

WORD* N_const::UsedVector

Definition at line 2297 of file [structs.h](#).

3.79.2.22 UsedIndex

WORD* N_const::UsedIndex

Definition at line 2298 of file [structs.h](#).

3.79.2.23 UsedFunction

WORD* N_const::UsedFunction

Definition at line 2299 of file [structs.h](#).

3.79.2.24 MaskPointer

WORD* N_const::MaskPointer

Definition at line 2300 of file [structs.h](#).

3.79.2.25 ForFindOnly

WORD* N_const::ForFindOnly

Definition at line 2301 of file [structs.h](#).

3.79.2.26 findTerm

WORD* N_const::findTerm

Definition at line 2302 of file [structs.h](#).

3.79.2.27 findPattern

WORD* N_const::findPattern

Definition at line 2303 of file [structs.h](#).

3.79.2.28 dummyrenumlist

WORD* N_const::dummyrenumlist

Definition at line 2308 of file [structs.h](#).

3.79.2.29 funargs

int* N_const::funargs

Definition at line 2309 of file [structs.h](#).

3.79.2.30 funlocs

WORD** N_const::funlocs

Definition at line 2310 of file [structs.h](#).

3.79.2.31 funinds

int* N_const::funinds

Definition at line 2311 of file [structs.h](#).

3.79.2.32 NoScrat2

UWORD* N_const::NoScrat2

Definition at line 2312 of file [structs.h](#).

3.79.2.33 ReplaceScrat

WORD* N_const::ReplaceScrat

Definition at line 2313 of file [structs.h](#).

3.79.2.34 tracestack

TRACES* N_const::tracestack

Definition at line 2314 of file [structs.h](#).

3.79.2.35 selecttermundo

WORD* N_const::selecttermundo

Definition at line 2315 of file [structs.h](#).

3.79.2.36 patternbuffer

WORD* N_const::patternbuffer

Definition at line 2316 of file [structs.h](#).

3.79.2.37 termbuffer

WORD* N_const::termbuffer

Definition at line 2317 of file [structs.h](#).

3.79.2.38 PoinScratch

WORD** N_const::PoinScratch

Definition at line 2318 of file [structs.h](#).

3.79.2.39 FunScratch

WORD** N_const::FunScratch

Definition at line 2319 of file [structs.h](#).

3.79.2.40 RenumScratch

WORD* N_const::RenumScratch

Definition at line 2320 of file [structs.h](#).

3.79.2.41 FunInfo

FUN_INFO* N_const::FunInfo

Definition at line 2321 of file [structs.h](#).

3.79.2.42 SplitScratch

WORD** N_const::SplitScratch

Definition at line 2322 of file [structs.h](#).

3.79.2.43 SplitScratch1

WORD** N_const::SplitScratch1

Definition at line 2323 of file [structs.h](#).

3.79.2.44 FunSorts

SORTING** N_const::FunSorts

Definition at line 2324 of file [structs.h](#).

3.79.2.45 SoScratC

UWORD* N_const::SoScratC

Definition at line 2325 of file [structs.h](#).

3.79.2.46 listinprint

WORD* N_const::listinprint

Definition at line 2326 of file [structs.h](#).

3.79.2.47 currentTerm

WORD* N_const::currentTerm

Definition at line 2327 of file [structs.h](#).

3.79.2.48 arglist

WORD** N_const::arglist

Definition at line 2328 of file [structs.h](#).

3.79.2.49 tlistbuf

int* N_const::tlistbuf

Definition at line 2329 of file [structs.h](#).

3.79.2.50 compressSpace

WORD* N_const::compressSpace

Definition at line 2333 of file [structs.h](#).

3.79.2.51 SHcombi

UWORD* N_const::SHcombi

Definition at line 2338 of file [structs.h](#).

3.79.2.52 poly_vars

WORD* N_const::poly_vars

Definition at line 2339 of file [structs.h](#).

3.79.2.53 cmod

UWORD* N_const::cmod

Definition at line 2340 of file [structs.h](#).

3.79.2.54 SHvar

`SHvariables N_const::SHvar`

Definition at line 2341 of file [structs.h](#).

3.79.2.55 deferskipped

`LONG N_const::deferskipped`

Definition at line 2342 of file [structs.h](#).

3.79.2.56 InScratch

`LONG N_const::InScratch`

Definition at line 2343 of file [structs.h](#).

3.79.2.57 SplitScratchSize

`LONG N_const::SplitScratchSize`

Definition at line 2344 of file [structs.h](#).

3.79.2.58 InScratch1

`LONG N_const::InScratch1`

Definition at line 2345 of file [structs.h](#).

3.79.2.59 SplitScratchSize1

`LONG N_const::SplitScratchSize1`

Definition at line 2346 of file [structs.h](#).

3.79.2.60 ninterms

`LONG N_const::ninterms`

Definition at line 2347 of file [structs.h](#).

3.79.2.61 SHcombisize

`LONG N_const::SHcombisize`

Definition at line 2356 of file [structs.h](#).

3.79.2.62 NumTotWildArgs

```
int N_const::NumTotWildArgs
```

Definition at line [2357](#) of file [structs.h](#).

3.79.2.63 UseFindOnly

```
int N_const::UseFindOnly
```

Definition at line [2358](#) of file [structs.h](#).

3.79.2.64 UsedOtherFind

```
int N_const::UsedOtherFind
```

Definition at line [2359](#) of file [structs.h](#).

3.79.2.65 ErrorInDollar

```
int N_const::ErrorInDollar
```

Definition at line [2360](#) of file [structs.h](#).

3.79.2.66 numfargs

```
int N_const::numfargs
```

Definition at line [2361](#) of file [structs.h](#).

3.79.2.67 numflocs

```
int N_const::numflocs
```

Definition at line [2362](#) of file [structs.h](#).

3.79.2.68 nargs

```
int N_const::nargs
```

Definition at line [2363](#) of file [structs.h](#).

3.79.2.69 tohunt

```
int N_const::tohunt
```

Definition at line [2364](#) of file [structs.h](#).

3.79.2.70 numoffuns

```
int N_const::numoffuns
```

Definition at line [2365](#) of file [structs.h](#).

3.79.2.71 funisize

```
int N_const::funisize
```

Definition at line [2366](#) of file [structs.h](#).

3.79.2.72 RSize

```
int N_const::RSize
```

Definition at line [2367](#) of file [structs.h](#).

3.79.2.73 numtracesctack

```
int N_const::numtracesctack
```

Definition at line [2368](#) of file [structs.h](#).

3.79.2.74 intracestack

```
int N_const::intracestack
```

Definition at line [2369](#) of file [structs.h](#).

3.79.2.75 numfuninfo

```
int N_const::numfuninfo
```

Definition at line [2370](#) of file [structs.h](#).

3.79.2.76 NumFunSorts

```
int N_const::NumFunSorts
```

Definition at line [2371](#) of file [structs.h](#).

3.79.2.77 MaxFunSorts

```
int N_const::MaxFunSorts
```

Definition at line [2372](#) of file [structs.h](#).

3.79.2.78 arglistsize

```
int N_const::arglistsize
```

Definition at line [2373](#) of file [structs.h](#).

3.79.2.79 tlistsize

```
int N_const::tlistsize
```

Definition at line [2374](#) of file [structs.h](#).

3.79.2.80 filenum

```
int N_const::filenum
```

Definition at line [2375](#) of file [structs.h](#).

3.79.2.81 compressSize

```
int N_const::compressSize
```

Definition at line [2376](#) of file [structs.h](#).

3.79.2.82 polysortflag

```
int N_const::polysortflag
```

Definition at line [2377](#) of file [structs.h](#).

3.79.2.83 nogroundlevel

```
int N_const::nogroundlevel
```

Definition at line [2378](#) of file [structs.h](#).

3.79.2.84 subsubveto

```
int N_const::subsubveto
```

Definition at line [2379](#) of file [structs.h](#).

3.79.2.85 MaxRenumScratch

```
WORD N_const::MaxRenumScratch
```

Definition at line [2380](#) of file [structs.h](#).

3.79.2.86 oldtype

WORD N_const::oldtype

Definition at line 2381 of file [structs.h](#).

3.79.2.87 oldvalue

WORD N_const::oldvalue

Definition at line 2382 of file [structs.h](#).

3.79.2.88 NumWild

WORD N_const::NumWild

Definition at line 2383 of file [structs.h](#).

3.79.2.89 RepFunNum

WORD N_const::RepFunNum

Definition at line 2384 of file [structs.h](#).

3.79.2.90 DisOrderFlag

WORD N_const::DisOrderFlag

Definition at line 2385 of file [structs.h](#).

3.79.2.91 WildDirt

WORD N_const::WildDirt

Definition at line 2386 of file [structs.h](#).

3.79.2.92 NumFound

WORD N_const::NumFound

Definition at line 2387 of file [structs.h](#).

3.79.2.93 WildReserve

WORD N_const::WildReserve

Definition at line 2388 of file [structs.h](#).

3.79.2.94 TeInFun

WORD N_const::TeInFun

Definition at line 2389 of file [structs.h](#).

3.79.2.95 TeSuOut

WORD N_const::TeSuOut

Definition at line 2390 of file [structs.h](#).

3.79.2.96 WildArgs

WORD N_const::WildArgs

Definition at line 2391 of file [structs.h](#).

3.79.2.97 WildEat

WORD N_const::WildEat

Definition at line 2392 of file [structs.h](#).

3.79.2.98 PolyNormFlag

WORD N_const::PolyNormFlag

Definition at line 2393 of file [structs.h](#).

3.79.2.99 PolyFunTodo

WORD N_const::PolyFunTodo

Definition at line 2394 of file [structs.h](#).

3.79.2.100 sizeselecttermundo

WORD N_const::sizeselecttermundo

Definition at line 2395 of file [structs.h](#).

3.79.2.101 patternbuffersize

WORD N_const::patternbuffersize

Definition at line 2396 of file [structs.h](#).

3.79.2.102 numlistinprint

WORD N_const::numlistinprint

Definition at line [2397](#) of file [structs.h](#).

3.79.2.103 ncmmod

WORD N_const::ncmmod

Definition at line [2398](#) of file [structs.h](#).

3.79.2.104 ExpectedSign

WORD N_const::ExpectedSign

Definition at line [2399](#) of file [structs.h](#).

3.79.2.105 SignCheck

WORD N_const::SignCheck

Used in pattern matching of antisymmetric functions

Definition at line [2400](#) of file [structs.h](#).

3.79.2.106 IndDum

WORD N_const::IndDum

Used in pattern matching of antisymmetric functions

Definition at line [2401](#) of file [structs.h](#).

3.79.2.107 poly_num_vars

WORD N_const::poly_num_vars

Definition at line [2402](#) of file [structs.h](#).

3.79.2.108 idfunctionflag

WORD N_const::idfunctionflag

Definition at line [2403](#) of file [structs.h](#).

3.79.2.109 poly_vars_type

WORD N_const::poly_vars_type

Definition at line 2404 of file [structs.h](#).

3.79.2.110 tryterm

WORD N_const::tryterm

Definition at line 2405 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.80 nameblock Struct Reference

Data Fields

- MLONG [previousblock](#)
- MLONG [position](#)
- char [names](#) [NAMETABLESIZE]

3.80.1 Detailed Description

Definition at line 115 of file [minos.h](#).

3.80.2 Field Documentation

3.80.2.1 previousblock

MLONG nameblock::previousblock

Definition at line 116 of file [minos.h](#).

3.80.2.2 position

MLONG nameblock::position

Definition at line 117 of file [minos.h](#).

3.80.2.3 names

```
char nameblock::names[NAMETABLESIZE]
```

Definition at line 118 of file [minos.h](#).

The documentation for this struct was generated from the following file:

- [minos.h](#)

3.81 NaMeNode Struct Reference

```
#include <structs.h>
```

Data Fields

- LONG [name](#)
- WORD [parent](#)
- WORD [left](#)
- WORD [right](#)
- WORD [balance](#)
- WORD [type](#)
- WORD [number](#)

3.81.1 Detailed Description

The names of variables are kept in an array. Elements of type [NAMENODE](#) define a tree (that is kept balanced) that make it easy and fast to look for variables. See also [NAMETREE](#).

Definition at line 247 of file [structs.h](#).

3.81.2 Field Documentation

3.81.2.1 name

```
LONG NaMeNode::name
```

Offset into [NAMETREE::namebuffer](#).

Definition at line 248 of file [structs.h](#).

3.81.2.2 parent

```
WORD NaMeNode::parent
```

== -1 if no parent.

Definition at line 249 of file [structs.h](#).

3.81.2.3 left

WORD NaMeNode::left

=-1 if no child.

Definition at line 250 of file [structs.h](#).

3.81.2.4 right

WORD NaMeNode::right

=-1 if no child.

Definition at line 251 of file [structs.h](#).

3.81.2.5 balance

WORD NaMeNode::balance

Used for the balancing of the tree.

Definition at line 252 of file [structs.h](#).

3.81.2.6 type

WORD NaMeNode::type

Type associated with the name. See [compiler types](#).

Definition at line 253 of file [structs.h](#).

3.81.2.7 number

WORD NaMeNode::number

Number of variable in [LIST](#)'s like for example [C_const::SymbolList](#).

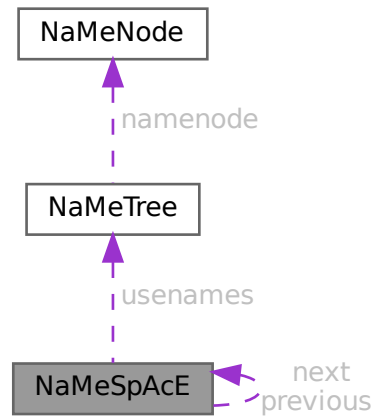
Definition at line 254 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.82 NaMeSpAcE Struct Reference

Collaboration diagram for NaMeSpAcE:



Data Fields

- struct [NaMeSpAcE](#) * [previous](#)
- struct [NaMeSpAcE](#) * [next](#)
- [NAMETREE](#) * [usernames](#)
- UBYTE * [name](#)

3.82.1 Detailed Description

Definition at line [658](#) of file [structs.h](#).

3.82.2 Field Documentation

3.82.2.1 previous

```
struct NaMeSpAcE* NaMeSpAcE::previous
```

Definition at line [659](#) of file [structs.h](#).

3.82.2.2 next

```
struct NaMeSpAcE* NaMeSpAcE::next
```

Definition at line [660](#) of file [structs.h](#).

3.82.2.3 usernames

`NAMETREE* NaMeSpAcE::usernames`

Definition at line 661 of file [structs.h](#).

3.82.2.4 name

`UBYTE* NaMeSpAcE::name`

Definition at line 662 of file [structs.h](#).

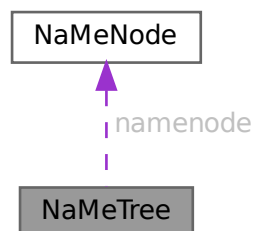
The documentation for this struct was generated from the following file:

- [structs.h](#)

3.83 NaMeTree Struct Reference

```
#include <structs.h>
```

Collaboration diagram for NaMeTree:



Data Fields

- `NAMENODE * namenode`
- `UBYTE * namebuffer`
- `LONG nodesize`
- `LONG nodefill`
- `LONG namesize`
- `LONG namefill`
- `LONG oldnamefill`
- `LONG oldnodefill`
- `LONG globalnamefill`
- `LONG globalnodefill`
- `LONG clearnamefill`
- `LONG clearnodefill`
- `WORD headnode`

3.83.1 Detailed Description

A struct of type [NAMETREE](#) controls a complete (balanced) tree of names for the compiler. The compiler maintains several of such trees and the system has been set up in such a way that one could define more of them if we ever want to work with local name spaces.

Definition at line [265](#) of file [structs.h](#).

3.83.2 Field Documentation

3.83.2.1 namenode

```
NAMENODE* NaMeTree::namenode
```

[D] Vector of [NAMENODE](#)'s. Number of elements is [nodesize](#). =0 if no memory has been allocated.

Definition at line [266](#) of file [structs.h](#).

3.83.2.2 namebuffer

```
UBYTE* NaMeTree::namebuffer
```

[D] Buffer that holds all the name strings referred to by the [NAMENODE](#)'s. Allocation size is [namesize](#). =0 if no memory has been allocated.

Definition at line [268](#) of file [structs.h](#).

3.83.2.3 nodesize

```
LONG NaMeTree::nodesize
```

Maximum number of elements in [namenode](#).

Definition at line [271](#) of file [structs.h](#).

3.83.2.4 nodefill

```
LONG NaMeTree::nodefill
```

Number of currently used nodes in [namenode](#).

Definition at line [272](#) of file [structs.h](#).

3.83.2.5 namesize

```
LONG NaMeTree::namesize
```

Allocation size of [namebuffer](#) in bytes.

Definition at line [273](#) of file [structs.h](#).

3.83.2.6 namefill

```
LONG NaMeTree::namefill
```

Number of bytes occupied.

Definition at line 274 of file [structs.h](#).

3.83.2.7 oldnamefill

```
LONG NaMeTree::oldnamefill
```

UNUSED

Definition at line 275 of file [structs.h](#).

3.83.2.8 oldnodefill

```
LONG NaMeTree::oldnodefill
```

UNUSED

Definition at line 276 of file [structs.h](#).

3.83.2.9 globalnamefill

```
LONG NaMeTree::globalnamefill
```

Set by .global statement to the value of [namefill](#). When a .store command is processed, this value will be used to reset namefill.

Definition at line 277 of file [structs.h](#).

3.83.2.10 globalnodefill

```
LONG NaMeTree::globalnodefill
```

Same usage as [globalnamefill](#), but for nodefill.

Definition at line 279 of file [structs.h](#).

3.83.2.11 clearnamefill

```
LONG NaMeTree::clearnamefill
```

Marks the reset point used by the .clear statement.

Definition at line 280 of file [structs.h](#).

3.83.2.12 clearnodefill

```
LONG NaMeTree::clearnodefill
```

Marks the reset point used by the .clear statement.

Definition at line 281 of file [structs.h](#).

3.83.2.13 headnode

```
WORD NaMeTree::headnode
```

Offset in [namenode](#) of head node. ==-1 if tree is empty.

Definition at line 282 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.84 NCand Class Reference

Public Member Functions

- [NCand](#) (const NCandSt sta, const int dega, const int nilst, int *ilst)
- void [prNCand](#) (const char *msg)

Data Fields

- int [deg](#)
- NCandSt [st](#)
- int [nilist](#)
- int [ilst](#) [GRCC_MAXMINTERACT]

3.84.1 Detailed Description

Definition at line 1044 of file [grcc.h](#).

3.84.2 Constructor & Destructor Documentation

3.84.2.1 NCand()

```
NCand::NCand (
    const NCandSt sta,
    const int dega,
    const int nilst,
    int * ilst )
```

Definition at line 7285 of file [grcc.cc](#).

3.84.2.2 ~NCand()

```
NCand::~NCand (
    void )
```

Definition at line 7312 of file [grcc.cc](#).

3.84.3 Member Function Documentation

3.84.3.1 prNCand()

```
void NCand::prNCand (
    const char * msg )
```

Definition at line 7317 of file [grcc.cc](#).

3.84.4 Field Documentation

3.84.4.1 deg

```
int NCand::deg
```

Definition at line 1048 of file [grcc.h](#).

3.84.4.2 st

```
NCandSt NCand::st
```

Definition at line 1049 of file [grcc.h](#).

3.84.4.3 nilist

```
int NCand::nilist
```

Definition at line 1050 of file [grcc.h](#).

3.84.4.4 ilist

```
int NCand::ilist[GRCC_MAXMINTERACT]
```

Definition at line 1051 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.85 NCInput Struct Reference

Data Fields

- int [cldeg](#)
- int [clnum](#)
- int [cltyp](#)
- int [cmind](#)
- int [cmaxd](#)
- int [ptcl](#)
- int [cple](#)

3.85.1 Detailed Description

Definition at line [149](#) of file [grccparam.h](#).

3.85.2 Field Documentation

3.85.2.1 cldeg

```
int NCInput::cldeg
```

Definition at line [151](#) of file [grccparam.h](#).

3.85.2.2 clnum

```
int NCInput::clnum
```

Definition at line [152](#) of file [grccparam.h](#).

3.85.2.3 cltyp

```
int NCInput::cltyp
```

Definition at line [153](#) of file [grccparam.h](#).

3.85.2.4 cmind

```
int NCInput::cmind
```

Definition at line [154](#) of file [grccparam.h](#).

3.85.2.5 cmaxd

```
int NCInput::cmaxd
```

Definition at line [155](#) of file [grccparam.h](#).

3.85.2.6 ptcl

```
int NCInput::ptcl
```

Definition at line 158 of file [grccparam.h](#).

3.85.2.7 cple

```
int NCInput::cple
```

Definition at line 159 of file [grccparam.h](#).

The documentation for this struct was generated from the following file:

- [grccparam.h](#)

3.86 NeStInG Struct Reference

```
#include <structs.h>
```

Data Fields

- WORD * [termsize](#)
- WORD * [funsize](#)
- WORD * [argsize](#)

3.86.1 Detailed Description

The NESTING struct is used when we enter the argument of functions and there is the possibility that we have to change something there. Because functions can be nested we have to keep track of all levels of functions in case we have to move the outer layers to make room for a larger function argument.

Definition at line 1053 of file [structs.h](#).

3.86.2 Field Documentation

3.86.2.1 termsize

```
WORD* NeStInG::termsize
```

Definition at line 1054 of file [structs.h](#).

3.86.2.2 funsize

```
WORD* NeStInG::funsize
```

Definition at line 1055 of file [structs.h](#).

3.86.2.3 argsize

WORD* NeStInG::argsize

Definition at line 1056 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.87 NoDe Struct Reference

Collaboration diagram for NoDe:



Data Fields

- struct [NoDe](#) * [left](#)
- struct [NoDe](#) * [right](#)
- int [lloser](#)
- int [rloser](#)
- int [lsrc](#)
- int [rsrc](#)

3.87.1 Detailed Description

A node for the tree of losers in the final sorting on the master.

Definition at line 265 of file [parallel.c](#).

3.87.2 Field Documentation

3.87.2.1 left

struct [NoDe](#)* NoDe::left

Definition at line 266 of file [parallel.c](#).

3.87.2.2 right

```
struct NoDe* NoDe::right
```

Definition at line 267 of file [parallel.c](#).

3.87.2.3 lloser

```
int NoDe::lloser
```

Definition at line 268 of file [parallel.c](#).

3.87.2.4 rloser

```
int NoDe::rloser
```

Definition at line 269 of file [parallel.c](#).

3.87.2.5 lsrc

```
int NoDe::lsrc
```

Definition at line 270 of file [parallel.c](#).

3.87.2.6 rsrc

```
int NoDe::rsrc
```

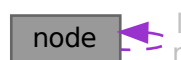
Definition at line 271 of file [parallel.c](#).

The documentation for this struct was generated from the following file:

- [parallel.c](#)

3.88 node Struct Reference

Collaboration diagram for node:



Public Member Functions

- [node](#) (const WORD *data)
- int [cmp](#) (const struct [node](#) *rhs) const
- bool [operator\(\)](#) (const struct [node](#) *lhs, const struct [node](#) *rhs) const
- void [calcHash](#) ()

Data Fields

- const WORD * [data](#)
- struct [node](#) * [l](#)
- struct [node](#) * [r](#)
- WORD [sign](#)
- UWORD [hash](#)

3.88.1 Detailed Description

Definition at line [1496](#) of file [optimize.cc](#).

3.88.2 Constructor & Destructor Documentation

3.88.2.1 [node\(\)](#) [1/2]

```
node::node ( ) [inline]
```

Definition at line [1503](#) of file [optimize.cc](#).

3.88.2.2 [node\(\)](#) [2/2]

```
node::node (
    const WORD * data ) [inline]
```

Definition at line [1504](#) of file [optimize.cc](#).

3.88.3 Member Function Documentation

3.88.3.1 [cmp\(\)](#)

```
int node::cmp (
    const struct node * rhs ) const [inline]
```

Definition at line [1508](#) of file [optimize.cc](#).

3.88.3.2 operator()

```
bool node::operator() (
    const struct node * lhs,
    const struct node * rhs ) const [inline]
```

Definition at line 1532 of file [optimize.cc](#).

3.88.3.3 calcHash()

```
void node::calcHash ( ) [inline]
```

Definition at line 1537 of file [optimize.cc](#).

3.88.4 Field Documentation

3.88.4.1 data

```
const WORD* node::data
```

Definition at line 1497 of file [optimize.cc](#).

3.88.4.2 l

```
struct node* node::l
```

Definition at line 1498 of file [optimize.cc](#).

3.88.4.3 r

```
struct node* node::r
```

Definition at line 1499 of file [optimize.cc](#).

3.88.4.4 sign

```
WORD node::sign
```

Definition at line 1500 of file [optimize.cc](#).

3.88.4.5 hash

```
UWORD node::hash
```

Definition at line 1501 of file [optimize.cc](#).

The documentation for this struct was generated from the following file:

- [optimize.cc](#)

3.89 NodeEq Struct Reference

Public Member Functions

- `bool operator()` (const `NODE` *lhs, const `NODE` *rhs) const

3.89.1 Detailed Description

Definition at line 1558 of file [optimize.cc](#).

3.89.2 Member Function Documentation

3.89.2.1 operator()

```
bool NodeEq::operator() (
    const NODE * lhs,
    const NODE * rhs ) const [inline]
```

Definition at line 1559 of file [optimize.cc](#).

The documentation for this struct was generated from the following file:

- [optimize.cc](#)

3.90 NodeHash Struct Reference

Public Member Functions

- `size_t operator()` (const `NODE` *n) const

3.90.1 Detailed Description

Definition at line 1552 of file [optimize.cc](#).

3.90.2 Member Function Documentation

3.90.2.1 operator()

```
size_t NodeHash::operator() (
    const NODE * n ) const [inline]
```

Definition at line 1553 of file [optimize.cc](#).

The documentation for this struct was generated from the following file:

- [optimize.cc](#)

3.91 NStack Class Reference

Public Member Functions

- void [print](#) (const char *msg)

Data Fields

- int [noden](#)
- int [deg](#)
- NCandSt [st](#)
- int [nilist](#)
- int [ilist](#) [GRCC_MAXMINTERACT]

3.91.1 Detailed Description

Definition at line 1270 of file [grcc.h](#).

3.91.2 Constructor & Destructor Documentation

3.91.2.1 NStack()

```
NStack::NStack ( ) [inline]
```

Definition at line 1280 of file [grcc.h](#).

3.91.2.2 ~NStack()

```
NStack::~~NStack ( ) [inline]
```

Definition at line 1281 of file [grcc.h](#).

3.91.3 Member Function Documentation

3.91.3.1 print()

```
void NStack::print (
    const char * msg )
```

Definition at line 9456 of file [grcc.cc](#).

3.91.4 Field Documentation

3.91.4.1 noden

```
int NStack::noden
```

Definition at line 1274 of file [grcc.h](#).

3.91.4.2 deg

```
int NStack::deg
```

Definition at line 1275 of file [grcc.h](#).

3.91.4.3 st

```
NCandSt NStack::st
```

Definition at line 1276 of file [grcc.h](#).

3.91.4.4 nilist

```
int NStack::nilist
```

Definition at line 1277 of file [grcc.h](#).

3.91.4.5 ilist

```
int NStack::ilist[GRCC_MAXMINTERACT]
```

Definition at line 1278 of file [grcc.h](#).

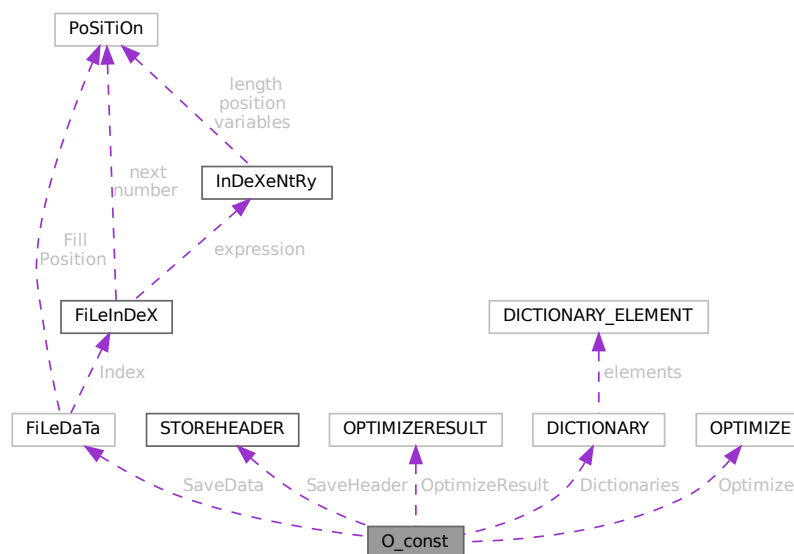
The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.92 O_const Struct Reference

```
#include <structs.h>
```

Collaboration diagram for O_const:



Public Member Functions

- **PADPOSITION** (25, 4, 36, 17, 1)

Data Fields

- FILEDATA SaveData
- STOREHEADER SaveHeader
- OPTIMIZERESULT OptimizeResult
- UBYTE * OutputLine
- UBYTE * OutStop
- UBYTE * OutFill
- WORD * bracket
- WORD * termbuf
- WORD * tabstring
- UBYTE * wpos
- UBYTE * wpoin
- UBYTE * DollarOutBuffer
- UBYTE * CurBufWrt
- void(* FlipWORD)(UBYTE *)
- void(* FlipLONG)(UBYTE *)
- void(* FlipPOS)(UBYTE *)
- void(* FlipPOINTER)(UBYTE *)
- void(* ResizeData)(UBYTE *, int, UBYTE *, int)
- void(* ResizeWORD)(UBYTE *, UBYTE *)
- void(* ResizeNCWORD)(UBYTE *, UBYTE *)
- void(* ResizeLONG)(UBYTE *, UBYTE *)
- void(* ResizePOS)(UBYTE *, UBYTE *)
- void(* ResizePOINTER)(UBYTE *, UBYTE *)
- void(* CheckPower)(UBYTE *)
- void(* RenumVec)(UBYTE *)
- DICTIONARY ** Dictionaries
- UBYTE * tensorList
- WORD * inscheme
- LONG NumInBrack
- LONG wlen
- LONG DollarOutSizeBuffer
- LONG DollarInOutBuffer
- OPTIMIZE Optimize
- int OutInBuffer
- int NoSpacesInNumbers
- int BlockSpaces
- int CurrentDictionary
- int SizeDictionaries
- int NumDictionaries
- int CurDictNumbers
- int CurDictVariables
- int CurDictSpecials
- int CurDictFunWithArgs
- int CurDictNumberWarning
- int CurDictNotInFunctions
- int CurDictInDollars
- int gNumDictionaries
- WORD schemenum

- WORD [transFlag](#)
- WORD [powerFlag](#)
- WORD [mpower](#)
- WORD [resizeFlag](#)
- WORD [bufferedInd](#)
- WORD [OutSkip](#)
- WORD [IsBracket](#)
- WORD [InFbrack](#)
- WORD [PrintType](#)
- WORD [FortFirst](#)
- WORD [DoubleFlag](#)
- WORD [IndentSpace](#)
- WORD [FactorMode](#)
- WORD [FactorNum](#)
- WORD [ErrorBlock](#)
- WORD [OptimizationLevel](#)
- UBYTE [FortDotChar](#)

3.92.1 Detailed Description

The [O_const](#) struct is part of the global data and resides in the ALLGLOBALS struct A under the name O We see it used with the macro AO as in AO.OutputLine It contains variables that involve the writing of text output and save/store files.

Definition at line [2453](#) of file [structs.h](#).

3.92.2 Field Documentation

3.92.2.1 SaveData

[FILEDATA](#) [O_const::SaveData](#)

Definition at line [2454](#) of file [structs.h](#).

3.92.2.2 SaveHeader

[STOREHEADER](#) [O_const::SaveHeader](#)

Definition at line [2455](#) of file [structs.h](#).

3.92.2.3 OptimizeResult

[OPTIMIZERESULT](#) [O_const::OptimizeResult](#)

Definition at line [2456](#) of file [structs.h](#).

3.92.2.4 OutputLine

UBYTE* O_const::OutputLine

Definition at line 2457 of file [structs.h](#).

3.92.2.5 OutStop

UBYTE* O_const::OutStop

Definition at line 2458 of file [structs.h](#).

3.92.2.6 OutFill

UBYTE* O_const::OutFill

Definition at line 2459 of file [structs.h](#).

3.92.2.7 bracket

WORD* O_const::bracket

Definition at line 2460 of file [structs.h](#).

3.92.2.8 termbuf

WORD* O_const::termbuf

Definition at line 2461 of file [structs.h](#).

3.92.2.9 tabstring

WORD* O_const::tabstring

Definition at line 2462 of file [structs.h](#).

3.92.2.10 wpos

UBYTE* O_const::wpos

Definition at line 2463 of file [structs.h](#).

3.92.2.11 wpoin

UBYTE* O_const::wpoin

Definition at line 2464 of file [structs.h](#).

3.92.2.12 DollarOutBuffer

```
UBYTE* O_const::DollarOutBuffer
```

Definition at line [2465](#) of file [structs.h](#).

3.92.2.13 CurBufWrt

```
UBYTE* O_const::CurBufWrt
```

Definition at line [2466](#) of file [structs.h](#).

3.92.2.14 FlipWORD

```
void(* O_const::FlipWORD) (UBYTE *)
```

Definition at line [2467](#) of file [structs.h](#).

3.92.2.15 FlipLONG

```
void(* O_const::FlipLONG) (UBYTE *)
```

Definition at line [2468](#) of file [structs.h](#).

3.92.2.16 FlipPOS

```
void(* O_const::FlipPOS) (UBYTE *)
```

Definition at line [2469](#) of file [structs.h](#).

3.92.2.17 FlipPOINTER

```
void(* O_const::FlipPOINTER) (UBYTE *)
```

Definition at line [2470](#) of file [structs.h](#).

3.92.2.18 ResizeData

```
void(* O_const::ResizeData) (UBYTE *, int, UBYTE *, int)
```

Definition at line [2471](#) of file [structs.h](#).

3.92.2.19 ResizeWORD

```
void(* O_const::ResizeWORD) (UBYTE *, UBYTE *)
```

Definition at line [2472](#) of file [structs.h](#).

3.92.2.20 ResizeNCWORD

```
void(* O_const::ResizeNCWORD) (UBYTE *, UBYTE *)
```

Definition at line 2473 of file [structs.h](#).

3.92.2.21 ResizeLONG

```
void(* O_const::ResizeLONG) (UBYTE *, UBYTE *)
```

Definition at line 2474 of file [structs.h](#).

3.92.2.22 ResizePOS

```
void(* O_const::ResizePOS) (UBYTE *, UBYTE *)
```

Definition at line 2475 of file [structs.h](#).

3.92.2.23 ResizePOINTER

```
void(* O_const::ResizePOINTER) (UBYTE *, UBYTE *)
```

Definition at line 2476 of file [structs.h](#).

3.92.2.24 CheckPower

```
void(* O_const::CheckPower) (UBYTE *)
```

Definition at line 2477 of file [structs.h](#).

3.92.2.25 RenumberVec

```
void(* O_const::ReNUMBERVec) (UBYTE *)
```

Definition at line 2478 of file [structs.h](#).

3.92.2.26 Dictionaries

```
DICTIoNARY** O_const::Dictionaries
```

Definition at line 2479 of file [structs.h](#).

3.92.2.27 tensorList

```
UBYTE* O_const::tensorList
```

Definition at line 2480 of file [structs.h](#).

3.92.2.28 inscheme

```
WORD* O_const::inscheme
```

Definition at line 2481 of file [structs.h](#).

3.92.2.29 NumInBrack

```
LONG O_const::NumInBrack
```

Definition at line 2487 of file [structs.h](#).

3.92.2.30 wlen

```
LONG O_const::wlen
```

Definition at line 2488 of file [structs.h](#).

3.92.2.31 DollarOutSizeBuffer

```
LONG O_const::DollarOutSizeBuffer
```

Definition at line 2489 of file [structs.h](#).

3.92.2.32 DollarInOutBuffer

```
LONG O_const::DollarInOutBuffer
```

Definition at line 2490 of file [structs.h](#).

3.92.2.33 Optimize

```
OPTIMIZE O_const::Optimize
```

Definition at line 2495 of file [structs.h](#).

3.92.2.34 OutInBuffer

```
int O_const::OutInBuffer
```

Definition at line 2496 of file [structs.h](#).

3.92.2.35 NoSpacesInNumbers

```
int O_const::NoSpacesInNumbers
```

Definition at line 2497 of file [structs.h](#).

3.92.2.36 BlockSpaces

```
int O_const::BlockSpaces
```

Definition at line 2498 of file [structs.h](#).

3.92.2.37 CurrentDictionary

```
int O_const::CurrentDictionary
```

Definition at line 2499 of file [structs.h](#).

3.92.2.38 SizeDictionaries

```
int O_const::SizeDictionaries
```

Definition at line 2500 of file [structs.h](#).

3.92.2.39 NumDictionaries

```
int O_const::NumDictionaries
```

Definition at line 2501 of file [structs.h](#).

3.92.2.40 CurDictNumbers

```
int O_const::CurDictNumbers
```

Definition at line 2502 of file [structs.h](#).

3.92.2.41 CurDictVariables

```
int O_const::CurDictVariables
```

Definition at line 2503 of file [structs.h](#).

3.92.2.42 CurDictSpecials

```
int O_const::CurDictSpecials
```

Definition at line 2504 of file [structs.h](#).

3.92.2.43 CurDictFunWithArgs

```
int O_const::CurDictFunWithArgs
```

Definition at line 2505 of file [structs.h](#).

3.92.2.44 CurDictNumberWarning

```
int O_const::CurDictNumberWarning
```

Definition at line 2506 of file [structs.h](#).

3.92.2.45 CurDictNotInFunctions

```
int O_const::CurDictNotInFunctions
```

Definition at line 2507 of file [structs.h](#).

3.92.2.46 CurDictInDollars

```
int O_const::CurDictInDollars
```

Definition at line 2508 of file [structs.h](#).

3.92.2.47 gNumDictionaries

```
int O_const::gNumDictionaries
```

Definition at line 2509 of file [structs.h](#).

3.92.2.48 schemenum

```
WORD O_const::schemenum
```

Definition at line 2513 of file [structs.h](#).

3.92.2.49 transFlag

```
WORD O_const::transFlag
```

Definition at line 2514 of file [structs.h](#).

3.92.2.50 powerFlag

```
WORD O_const::powerFlag
```

Definition at line 2515 of file [structs.h](#).

3.92.2.51 mpower

```
WORD O_const::mpower
```

Definition at line 2516 of file [structs.h](#).

3.92.2.52 resizeFlag

WORD O_const::resizeFlag

Definition at line 2517 of file [structs.h](#).

3.92.2.53 bufferedInd

WORD O_const::bufferedInd

Definition at line 2518 of file [structs.h](#).

3.92.2.54 OutSkip

WORD O_const::OutSkip

Definition at line 2519 of file [structs.h](#).

3.92.2.55 IsBracket

WORD O_const::IsBracket

Definition at line 2520 of file [structs.h](#).

3.92.2.56 InFbrack

WORD O_const::InFbrack

Definition at line 2521 of file [structs.h](#).

3.92.2.57 PrintType

WORD O_const::PrintType

Definition at line 2522 of file [structs.h](#).

3.92.2.58 FortFirst

WORD O_const::FortFirst

Definition at line 2523 of file [structs.h](#).

3.92.2.59 DoubleFlag

WORD O_const::DoubleFlag

Definition at line 2524 of file [structs.h](#).

3.92.2.60 IndentSpace

WORD O_const::IndentSpace

Definition at line 2525 of file [structs.h](#).

3.92.2.61 FactorMode

WORD O_const::FactorMode

Definition at line 2526 of file [structs.h](#).

3.92.2.62 FactorNum

WORD O_const::FactorNum

Definition at line 2527 of file [structs.h](#).

3.92.2.63 ErrorBlock

WORD O_const::ErrorBlock

Definition at line 2528 of file [structs.h](#).

3.92.2.64 OptimizationLevel

WORD O_const::OptimizationLevel

Definition at line 2529 of file [structs.h](#).

3.92.2.65 FortDotChar

UBYTE O_const::FortDotChar

Definition at line 2530 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.93 objects Struct Reference

Data Fields

- MLONG [position](#)
- MLONG [size](#)
- MLONG [date](#)
- MLONG [tablename](#)
- MLONG [uncompressed](#)
- MLONG [spare1](#)
- MLONG [spare2](#)
- MLONG [spare3](#)
- char [element](#) [ELEMENTSIZE]

3.93.1 Detailed Description

Definition at line [95](#) of file [minos.h](#).

3.93.2 Field Documentation

3.93.2.1 position

MLONG `objects::position`

Definition at line [97](#) of file [minos.h](#).

3.93.2.2 size

MLONG `objects::size`

Definition at line [98](#) of file [minos.h](#).

3.93.2.3 date

MLONG `objects::date`

Definition at line [99](#) of file [minos.h](#).

3.93.2.4 tablename

MLONG `objects::tablename`

Definition at line [100](#) of file [minos.h](#).

3.93.2.5 uncompressed

```
MLONG objects::uncompressed
```

Definition at line 101 of file [minos.h](#).

3.93.2.6 spare1

```
MLONG objects::spare1
```

Definition at line 102 of file [minos.h](#).

3.93.2.7 spare2

```
MLONG objects::spare2
```

Definition at line 103 of file [minos.h](#).

3.93.2.8 spare3

```
MLONG objects::spare3
```

Definition at line 104 of file [minos.h](#).

3.93.2.9 element

```
char objects::element[ELEMENTSIZE]
```

Definition at line 105 of file [minos.h](#).

The documentation for this struct was generated from the following file:

- [minos.h](#)

3.94 OptDef Struct Reference

Data Fields

- const char * [name](#)
- const char * [mean](#)
- int [defaultv](#)
- int [DUMMY_PADDING](#)

3.94.1 Detailed Description

Definition at line 101 of file [grccparam.h](#).

3.94.2 Field Documentation

3.94.2.1 name

```
const char* OptDef::name
```

Definition at line 102 of file [grccparam.h](#).

3.94.2.2 mean

```
const char* OptDef::mean
```

Definition at line 103 of file [grccparam.h](#).

3.94.2.3 defaultv

```
int OptDef::defaultv
```

Definition at line 104 of file [grccparam.h](#).

3.94.2.4 DUMMYPADDING

```
int OptDef::DUMMYPADDING
```

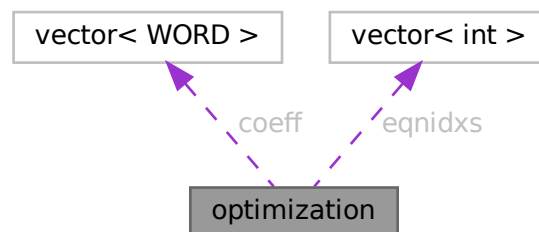
Definition at line 105 of file [grccparam.h](#).

The documentation for this struct was generated from the following file:

- [grccparam.h](#)

3.95 optimization Class Reference

Collaboration diagram for optimization:



Public Member Functions

- bool `operator<` (const `optimization` &a) const

Data Fields

- int `type`
- int `arg1`
- int `arg2`
- int `improve`
- vector< WORD > `coeff`
- vector< int > `eqnidxs`

3.95.1 Detailed Description

class Optimization

3.95.2 Description

This object represents an optimization. Its type is a number in the range 0 to 5. Depending on this type, the variables `arg1`, `arg2` and `coeff` indicate:

type==0 : optimization of the form $x[\text{arg1}] \wedge \text{arg2}$ (coeff=empty) type==1 : optimization of the form $x[\text{arg1}] * x[\text{arg2}]$ (coeff=empty) type==2 : optimization of the form $x[\text{arg1}] * \text{coeff}$ (arg2=0) type==3 : optimization of the form $x[\text{arg1}] + \text{coeff}$ (arg2=0) type==4 : optimization of the form $x[\text{arg1}] + x[\text{arg2}]$ (coeff=empty) type==5 : optimization of the form $x[\text{arg1}] - x[\text{arg2}]$ (coeff=empty)

Here, "`x[arg]`" represents a symbol (if positive) or an extrasymbol (if negative). The represented symbol's id is $\text{ABS}(x[\text{arg}]) - 1$.

"eqns" is a list of equation, where this optimization can be performed.

"improve" is the total improvement of this optimization.

Definition at line 2670 of file `optimize.cc`.

3.95.3 Member Function Documentation

3.95.3.1 `operator<()`

```
bool optimization::operator< (
    const optimization & a ) const [inline]
```

Definition at line 2676 of file `optimize.cc`.

3.95.4 Field Documentation

3.95.4.1 type

```
int optimization::type
```

Definition at line 2672 of file [optimize.cc](#).

3.95.4.2 arg1

```
int optimization::arg1
```

Definition at line 2672 of file [optimize.cc](#).

3.95.4.3 arg2

```
int optimization::arg2
```

Definition at line 2672 of file [optimize.cc](#).

3.95.4.4 improve

```
int optimization::improve
```

Definition at line 2672 of file [optimize.cc](#).

3.95.4.5 coeff

```
vector<WORD> optimization::coeff
```

Definition at line 2673 of file [optimize.cc](#).

3.95.4.6 eqnidxs

```
vector<int> optimization::eqnidxs
```

Definition at line 2674 of file [optimize.cc](#).

The documentation for this class was generated from the following file:

- [optimize.cc](#)

3.96 OPTIMIZE Struct Reference

Data Fields

- union {
 float [fval](#)
 int [ival](#) [2]
} **mctsconstant**
- int [horner](#)
- int [hornerdirection](#)
- int [method](#)
- int [mctstimelimit](#)
- int [mctsnumexpand](#)
- int [mctsnumkeep](#)
- int [mctsnumrepeat](#)
- int [greedytimelimit](#)
- int [greedyminnum](#)
- int [greedymaxperc](#)
- int [printstats](#)
- int [debugflags](#)
- int [schemeflags](#)
- int [mctsdecaymode](#)
- int [salter](#)
- union {
 float [fval](#)
 int [ival](#) [2]
} **saMaxT**
- union {
 float [fval](#)
 int [ival](#) [2]
} **saMinT**
- int [spare](#)

3.96.1 Detailed Description

Definition at line [1312](#) of file [structs.h](#).

3.96.2 Field Documentation

3.96.2.1 fval

```
float OPTIMIZE::fval
```

Definition at line [1314](#) of file [structs.h](#).

3.96.2.2 ival

```
int OPTIMIZE::ival[2]
```

Definition at line [1315](#) of file [structs.h](#).

3.96.2.3 horner

```
int OPTIMIZE::horner
```

Definition at line 1317 of file [structs.h](#).

3.96.2.4 hornerdirection

```
int OPTIMIZE::hornerdirection
```

Definition at line 1318 of file [structs.h](#).

3.96.2.5 method

```
int OPTIMIZE::method
```

Definition at line 1319 of file [structs.h](#).

3.96.2.6 mctstimelimit

```
int OPTIMIZE::mctstimelimit
```

Definition at line 1320 of file [structs.h](#).

3.96.2.7 mctsnumexpand

```
int OPTIMIZE::mctsnumexpand
```

Definition at line 1321 of file [structs.h](#).

3.96.2.8 mctsnumkeep

```
int OPTIMIZE::mctsnumkeep
```

Definition at line 1322 of file [structs.h](#).

3.96.2.9 mctsnumrepeat

```
int OPTIMIZE::mctsnumrepeat
```

Definition at line 1323 of file [structs.h](#).

3.96.2.10 greedytimelimit

```
int OPTIMIZE::greedytimelimit
```

Definition at line 1324 of file [structs.h](#).

3.96.2.11 greedyminnum

```
int OPTIMIZE::greedyminnum
```

Definition at line 1325 of file [structs.h](#).

3.96.2.12 greedymaxperc

```
int OPTIMIZE::greedymaxperc
```

Definition at line 1326 of file [structs.h](#).

3.96.2.13 printstats

```
int OPTIMIZE::printstats
```

Definition at line 1327 of file [structs.h](#).

3.96.2.14 debugflags

```
int OPTIMIZE::debugflags
```

Definition at line 1328 of file [structs.h](#).

3.96.2.15 schemeflags

```
int OPTIMIZE::schemeflags
```

Definition at line 1329 of file [structs.h](#).

3.96.2.16 mctsdecaymode

```
int OPTIMIZE::mctsdecaymode
```

Definition at line 1330 of file [structs.h](#).

3.96.2.17 salter

```
int OPTIMIZE::saIter
```

Definition at line 1331 of file [structs.h](#).

3.96.2.18 spare

```
int OPTIMIZE::spare
```

Definition at line 1340 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.97 OPTIMIZERESULT Struct Reference

Public Member Functions

- **PADPOSITION** (2, 1, 0, 3, 0)

Data Fields

- WORD * [code](#)
- UBYTE * [nameofexpr](#)
- LONG [codesize](#)
- WORD [exprnr](#)
- WORD [minvar](#)
- WORD [maxvar](#)

3.97.1 Detailed Description

Definition at line 1343 of file [structs.h](#).

3.97.2 Field Documentation

3.97.2.1 code

```
WORD* OPTIMIZERESULT::code
```

Definition at line 1344 of file [structs.h](#).

3.97.2.2 nameofexpr

```
UBYTE* OPTIMIZERESULT::nameofexpr
```

Definition at line 1345 of file [structs.h](#).

Public Member Functions

- void [setDefault](#)Value (void)
- void [setValue](#) (int ind, int val)
- int [getValue](#) (int ind)
- void [print](#) (void)
- void [setOutputF](#) (Bool outgrf, const char *fname)
- void [setOutputP](#) (Bool outgrp, const char *fname)
- void [printLevel](#) (int l)
- void [printModel](#) (void)
- void [setOutMG](#) (OutEGB *omg, void *pt)
- void [setOutAG](#) (OutEG *oag, void *pt)
- void [setEndMG](#) (OutEGB *omg, void *pt)
- void [setErExit](#) (ErExit *ere, void *pt)
- const [OptDef](#) * [getDef](#) (void)
- void [begin](#) ([Model](#) *mdl)
- void [end](#) (void)
- void [beginProc](#) ([Process](#) *prc)
- void [endProc](#) (void)
- void [beginSubProc](#) ([SProcess](#) *sprc)
- void [endSubProc](#) (void)
- void [newMGraph](#) ([MGraph](#) *mgr)
- void [newAGraph](#) ([EGraph](#) *egr)
- void [outModel](#) (void)

Data Fields

- [Model](#) * [model](#)
- [Process](#) * [proc](#)
- [SProcess](#) * [sprc](#)
- [Output](#) * [out](#)
- OutEGB * [outmg](#)
- OutEGB * [endmg](#)
- OutEG * [outag](#)
- void * [argmg](#)
- void * [argemg](#)
- void * [argag](#)
- int [values](#) [GRCC_OPT_Size]
- int [DUMMY_PADDING](#)
- double [time0](#)
- double [time1](#)

3.98.1 Detailed Description

Definition at line 88 of file [grcc.h](#).

3.98.2 Constructor & Destructor Documentation**3.98.2.1 Options()**

```
Options::Options (
    void )
```

Definition at line 117 of file [grcc.cc](#).

3.98.2.2 ~Options()

```
Options::~~Options (
    void )
```

Definition at line 149 of file [grcc.cc](#).

3.98.3 Member Function Documentation

3.98.3.1 setDefaultValue()

```
void Options::setDefaultValue (
    void )
```

Definition at line 161 of file [grcc.cc](#).

3.98.3.2 setValue()

```
void Options::setValue (
    int ind,
    int val )
```

Definition at line 212 of file [grcc.cc](#).

3.98.3.3 getValue()

```
int Options::getValue (
    int ind )
```

Definition at line 227 of file [grcc.cc](#).

3.98.3.4 print()

```
void Options::print (
    void )
```

Definition at line 241 of file [grcc.cc](#).

3.98.3.5 setOutputF()

```
void Options::setOutputF (
    Bool outgrf,
    const char * fname )
```

Definition at line 264 of file [grcc.cc](#).

3.98.3.6 setOutputP()

```
void Options::setOutputP (
    Bool outgrp,
    const char * fname )
```

Definition at line 271 of file [grcc.cc](#).

3.98.3.7 printLevel()

```
void Options::printLevel (
    int l )
```

Definition at line 206 of file [grcc.cc](#).

3.98.3.8 printModel()

```
void Options::printModel (
    void )
```

Definition at line 278 of file [grcc.cc](#).

3.98.3.9 setOutMG()

```
void Options::setOutMG (
    OutEGB * omg,
    void * pt )
```

Definition at line 178 of file [grcc.cc](#).

3.98.3.10 setOutAG()

```
void Options::setOutAG (
    OutEG * oag,
    void * pt )
```

Definition at line 192 of file [grcc.cc](#).

3.98.3.11 setEndMG()

```
void Options::setEndMG (
    OutEGB * omg,
    void * pt )
```

Definition at line 185 of file [grcc.cc](#).

3.98.3.12 setErExit()

```
void Options::setErExit (
    ErExit * ere,
    void * pt )
```

Definition at line 199 of file [grcc.cc](#).

3.98.3.13 getDef()

```
const OptDef * Options::getDef (
    void )
```

Definition at line 257 of file [grcc.cc](#).

3.98.3.14 begin()

```
void Options::begin (
    Model * mdl )
```

Definition at line 299 of file [grcc.cc](#).

3.98.3.15 end()

```
void Options::end (
    void )
```

Definition at line 319 of file [grcc.cc](#).

3.98.3.16 beginProc()

```
void Options::beginProc (
    Process * prc )
```

Definition at line 351 of file [grcc.cc](#).

3.98.3.17 endProc()

```
void Options::endProc (
    void )
```

Definition at line 367 of file [grcc.cc](#).

3.98.3.18 beginSubProc()

```
void Options::beginSubProc (
    SProcess * sprc )
```

Definition at line 456 of file [grcc.cc](#).

3.98.3.19 endSubProc()

```
void Options::endSubProc (
    void )
```

Definition at line 477 of file [grcc.cc](#).

3.98.3.20 newMGraph()

```
void Options::newMGraph (
    MGraph * mgr )
```

Definition at line 546 of file [grcc.cc](#).

3.98.3.21 newAGraph()

```
void Options::newAGraph (
    EGraph * egr )
```

Definition at line 583 of file [grcc.cc](#).

3.98.3.22 outModel()

```
void Options::outModel (
    void )
```

Definition at line 290 of file [grcc.cc](#).

3.98.4 Field Documentation

3.98.4.1 model

```
Model* Options::model
```

Definition at line 91 of file [grcc.h](#).

3.98.4.2 proc

```
Process* Options::proc
```

Definition at line 92 of file [grcc.h](#).

3.98.4.3 sproc

```
SProcess* Options::sproc
```

Definition at line 93 of file [grcc.h](#).

3.98.4.4 out

`Output* Options::out`

Definition at line 94 of file [grcc.h](#).

3.98.4.5 outmg

`OutEGB* Options::outmg`

Definition at line 95 of file [grcc.h](#).

3.98.4.6 endmg

`OutEGB* Options::endmg`

Definition at line 96 of file [grcc.h](#).

3.98.4.7 outag

`OutEG* Options::outag`

Definition at line 97 of file [grcc.h](#).

3.98.4.8 argmg

`void* Options::argmg`

Definition at line 98 of file [grcc.h](#).

3.98.4.9 argemg

`void* Options::argemg`

Definition at line 99 of file [grcc.h](#).

3.98.4.10 argag

`void* Options::argag`

Definition at line 100 of file [grcc.h](#).

3.98.4.11 values

`int Options::values[GRCC_OPT_Size]`

Definition at line 103 of file [grcc.h](#).

Public Member Functions

- [Output](#) ([Options](#) *optn)
- void [setOutgrf](#) (const char *fname)
- void [setOutgrp](#) (const char *fname)
- Bool [outBeginF](#) ([Model](#) *mdl, Bool pr)
- Bool [outBeginP](#) ([Model](#) *mdl, Bool pr)
- void [outEndF](#) (void)
- void [outEndP](#) (void)
- void [outProcBeginF](#) ([Process](#) *prc)
- void [outProcBeginP](#) ([Process](#) *prc)
- void [outProcBegin0](#) (int next, int couple, int loop)
- void [outSProcBeginF](#) ([SProcess](#) *sprc)
- void [outSProcBeginP](#) ([SProcess](#) *sprc)
- void [outProcEndF](#) (void)
- void [outProcEndP](#) (void)
- void [outEGraphF](#) ([EGraph](#) *egraph)
- void [outEGraphP](#) ([EGraph](#) *egraph)
- void [outModelF](#) (void)
- void [outModelP](#) (void)

Data Fields

- [Options](#) * opt
- [Model](#) * model
- [Process](#) * proc
- [SProcess](#) * sprc
- char * [outgrf](#)
- FILE * [outgrfp](#)
- char * [outgrp](#)
- FILE * [outgrpp](#)
- int [procl](#)
- Bool [outproc](#)

3.99.1 Detailed Description

Definition at line 144 of file [grcc.h](#).

3.99.2 Constructor & Destructor Documentation

3.99.2.1 Output()

```
Output::Output (
    Options * optn )
```

Definition at line 637 of file [grcc.cc](#).

3.99.2.2 ~Output()

```
Output::~~Output (
    void )
```

Definition at line [653](#) of file [grcc.cc](#).

3.99.3 Member Function Documentation

3.99.3.1 setOutgrf()

```
void Output::setOutgrf (
    const char * fname )
```

Definition at line [684](#) of file [grcc.cc](#).

3.99.3.2 setOutgrp()

```
void Output::setOutgrp (
    const char * fname )
```

Definition at line [701](#) of file [grcc.cc](#).

3.99.3.3 outBeginF()

```
Bool Output::outBeginF (
    Model * mdl,
    Bool pr )
```

Definition at line [718](#) of file [grcc.cc](#).

3.99.3.4 outBeginP()

```
Bool Output::outBeginP (
    Model * mdl,
    Bool pr )
```

Definition at line [756](#) of file [grcc.cc](#).

3.99.3.5 outEndF()

```
void Output::outEndF (
    void )
```

Definition at line [787](#) of file [grcc.cc](#).

3.99.3.6 outEndP()

```
void Output::outEndP (
    void )
```

Definition at line 803 of file [grcc.cc](#).

3.99.3.7 outProcBeginF()

```
void Output::outProcBeginF (
    Process * prc )
```

Definition at line 819 of file [grcc.cc](#).

3.99.3.8 outProcBeginP()

```
void Output::outProcBeginP (
    Process * prc )
```

Definition at line 864 of file [grcc.cc](#).

3.99.3.9 outProcBegin0()

```
void Output::outProcBegin0 (
    int next,
    int couple,
    int loop )
```

Definition at line 886 of file [grcc.cc](#).

3.99.3.10 outSProcBeginF()

```
void Output::outSProcBeginF (
    SProcess * sprc )
```

Definition at line 918 of file [grcc.cc](#).

3.99.3.11 outSProcBeginP()

```
void Output::outSProcBeginP (
    SProcess * sprc )
```

Definition at line 973 of file [grcc.cc](#).

3.99.3.12 outProcEndF()

```
void Output::outProcEndF (
    void )
```

Definition at line 993 of file [grcc.cc](#).

3.99.3.13 outProcEndP()

```
void Output::outProcEndP (
    void )
```

Definition at line 1012 of file [grcc.cc](#).

3.99.3.14 outEGraphF()

```
void Output::outEGraphF (
    EGraph * egraph )
```

Definition at line 1024 of file [grcc.cc](#).

3.99.3.15 outEGraphP()

```
void Output::outEGraphP (
    EGraph * egraph )
```

Definition at line 1152 of file [grcc.cc](#).

3.99.3.16 outModelF()

```
void Output::outModelF (
    void )
```

Definition at line 1254 of file [grcc.cc](#).

3.99.3.17 outModelP()

```
void Output::outModelP (
    void )
```

Definition at line 1335 of file [grcc.cc](#).

3.99.4 Field Documentation

3.99.4.1 opt

[Options*](#) Output::opt

Definition at line 147 of file [grcc.h](#).

3.99.4.2 model

[Model*](#) Output::model

Definition at line 148 of file [grcc.h](#).

3.99.4.3 proc

`Process*` Output::proc

Definition at line 149 of file [grcc.h](#).

3.99.4.4 sproc

`SProcess*` Output::sproc

Definition at line 150 of file [grcc.h](#).

3.99.4.5 outgrf

`char*` Output::outgrf

Definition at line 151 of file [grcc.h](#).

3.99.4.6 outgrfp

`FILE*` Output::outgrfp

Definition at line 152 of file [grcc.h](#).

3.99.4.7 outgrp

`char*` Output::outgrp

Definition at line 153 of file [grcc.h](#).

3.99.4.8 outgrpp

`FILE*` Output::outgrpp

Definition at line 154 of file [grcc.h](#).

3.99.4.9 proclId

`int` Output::procId

Definition at line 155 of file [grcc.h](#).

3.99.4.10 outproc

```
Bool Output::outproc
```

Definition at line 156 of file [grcc.h](#).

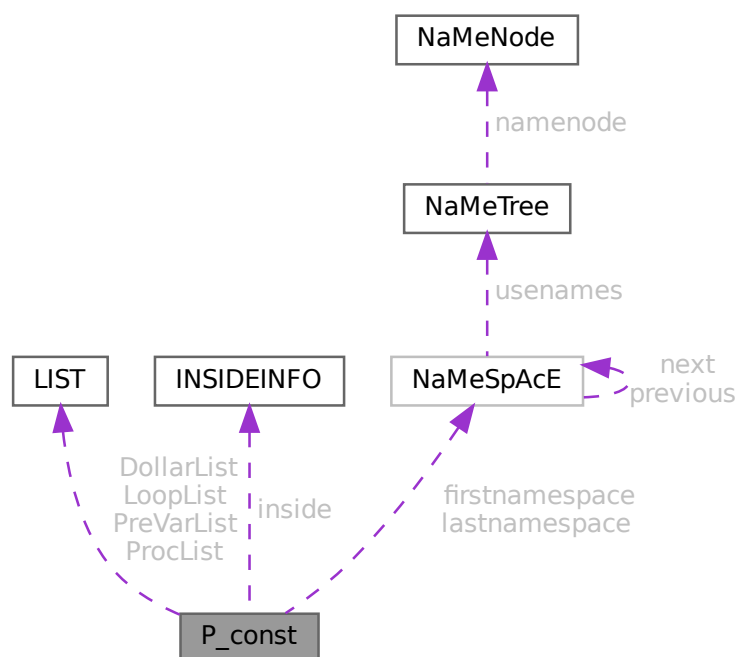
The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.100 P_const Struct Reference

```
#include <structs.h>
```

Collaboration diagram for P_const:



Data Fields

- [LIST DollarList](#)
- [LIST PreVarList](#)
- [LIST LoopList](#)
- [LIST ProcList](#)
- [INSIDEINFO inside](#)

- [NAMESPACE](#) * [firstnamespace](#)
- [NAMESPACE](#) * [lastnamespace](#)
- [UBYTE](#) * [fullName](#)
- [UBYTE](#) ** [PreSwitchStrings](#)
- [UBYTE](#) * [preStart](#)
- [UBYTE](#) * [preStop](#)
- [UBYTE](#) * [preFill](#)
- [UBYTE](#) * [procedureExtension](#)
- [UBYTE](#) * [cprocedureExtension](#)
- [LONG](#) * [PreAssignStack](#)
- [int](#) * [PrelfStack](#)
- [int](#) * [PreSwitchModes](#)
- [int](#) * [PreTypes](#)
- [LONG](#) [StopWatchZero](#)
- [LONG](#) [InOutBuf](#)
- [LONG](#) [pSize](#)
- [int](#) [PreAssignFlag](#)
- [int](#) [PreContinuation](#)
- [int](#) [PreproFlag](#)
- [int](#) [iBufError](#)
- [int](#) [PreOut](#)
- [int](#) [PreSwitchLevel](#)
- [int](#) [NumPreSwitchStrings](#)
- [int](#) [MaxPreTypes](#)
- [int](#) [NumPreTypes](#)
- [int](#) [MaxPrelfLevel](#)
- [int](#) [PrelfLevel](#)
- [int](#) [PreInsideLevel](#)
- [int](#) [DelayPrevar](#)
- [int](#) [AllowDelay](#)
- [int](#) [lhdollarerror](#)
- [int](#) [eat](#)
- [int](#) [gNumPre](#)
- [int](#) [PreDebug](#)
- [int](#) [OpenDictionary](#)
- [int](#) [PreAssignLevel](#)
- [int](#) [MaxPreAssignLevel](#)
- [int](#) [fullnamesize](#)
- [WORD](#) [DebugFlag](#)
- [WORD](#) [preError](#)
- [UBYTE](#) [ComChar](#)
- [UBYTE](#) [cComChar](#)

3.100.1 Detailed Description

The [P_const](#) struct is part of the global data and resides in the ALLGLOBALS struct A under the name P We see it used with the macro AP as in AP.InOutBuf It contains objects that have dealings with the preprocessor.

Definition at line [1628](#) of file [structs.h](#).

3.100.2 Field Documentation

3.100.2.1 DollarList

`LIST P_const::DollarList`

Definition at line 1629 of file [structs.h](#).

3.100.2.2 PreVarList

`LIST P_const::PreVarList`

Definition at line 1631 of file [structs.h](#).

3.100.2.3 LoopList

`LIST P_const::LoopList`

Definition at line 1633 of file [structs.h](#).

3.100.2.4 ProcList

`LIST P_const::ProcList`

Definition at line 1634 of file [structs.h](#).

3.100.2.5 inside

`INSIDEINFO P_const::inside`

Definition at line 1635 of file [structs.h](#).

3.100.2.6 firstnamespace

`NAMESPACE* P_const::firstnamespace`

Definition at line 1636 of file [structs.h](#).

3.100.2.7 lastnamespace

`NAMESPACE* P_const::lastnamespace`

Definition at line 1637 of file [structs.h](#).

3.100.2.8 fullname

```
UBYTE* P_const::fullname
```

Definition at line 1638 of file [structs.h](#).

3.100.2.9 PreSwitchStrings

```
UBYTE** P_const::PreSwitchStrings
```

Definition at line 1639 of file [structs.h](#).

3.100.2.10 preStart

```
UBYTE* P_const::preStart
```

Definition at line 1640 of file [structs.h](#).

3.100.2.11 preStop

```
UBYTE* P_const::preStop
```

Definition at line 1641 of file [structs.h](#).

3.100.2.12 preFill

```
UBYTE* P_const::preFill
```

Definition at line 1642 of file [structs.h](#).

3.100.2.13 procedureExtension

```
UBYTE* P_const::procedureExtension
```

Definition at line 1643 of file [structs.h](#).

3.100.2.14 cprocedureExtension

```
UBYTE* P_const::cprocedureExtension
```

Definition at line 1644 of file [structs.h](#).

3.100.2.15 PreAssignStack

```
LONG* P_const::PreAssignStack
```

Definition at line 1645 of file [structs.h](#).

3.100.2.16 PreIfStack

```
int* P_const::PreIfStack
```

Definition at line 1646 of file [structs.h](#).

3.100.2.17 PreSwitchModes

```
int* P_const::PreSwitchModes
```

Definition at line 1647 of file [structs.h](#).

3.100.2.18 PreTypes

```
int* P_const::PreTypes
```

Definition at line 1648 of file [structs.h](#).

3.100.2.19 StopWatchZero

```
LONG P_const::StopWatchZero
```

Definition at line 1652 of file [structs.h](#).

3.100.2.20 InOutBuf

```
LONG P_const::InOutBuf
```

Definition at line 1653 of file [structs.h](#).

3.100.2.21 pSize

```
LONG P_const::pSize
```

Definition at line 1654 of file [structs.h](#).

3.100.2.22 PreAssignFlag

```
int P_const::PreAssignFlag
```

Definition at line 1655 of file [structs.h](#).

3.100.2.23 PreContinuation

```
int P_const::PreContinuation
```

Definition at line 1656 of file [structs.h](#).

3.100.2.24 PreproFlag

```
int P_const::PreproFlag
```

Definition at line 1657 of file [structs.h](#).

3.100.2.25 iBufError

```
int P_const::iBufError
```

Definition at line 1658 of file [structs.h](#).

3.100.2.26 PreOut

```
int P_const::PreOut
```

Definition at line 1659 of file [structs.h](#).

3.100.2.27 PreSwitchLevel

```
int P_const::PreSwitchLevel
```

Definition at line 1660 of file [structs.h](#).

3.100.2.28 NumPreSwitchStrings

```
int P_const::NumPreSwitchStrings
```

Definition at line 1661 of file [structs.h](#).

3.100.2.29 MaxPreTypes

```
int P_const::MaxPreTypes
```

Definition at line 1662 of file [structs.h](#).

3.100.2.30 NumPreTypes

```
int P_const::NumPreTypes
```

Definition at line 1663 of file [structs.h](#).

3.100.2.31 MaxPreIfLevel

```
int P_const::MaxPreIfLevel
```

Definition at line 1664 of file [structs.h](#).

3.100.2.32 PreIfLevel

```
int P_const::PreIfLevel
```

Definition at line 1665 of file [structs.h](#).

3.100.2.33 PreInsideLevel

```
int P_const::PreInsideLevel
```

Definition at line 1666 of file [structs.h](#).

3.100.2.34 DelayPrevar

```
int P_const::DelayPrevar
```

Definition at line 1667 of file [structs.h](#).

3.100.2.35 AllowDelay

```
int P_const::AllowDelay
```

Definition at line 1668 of file [structs.h](#).

3.100.2.36 lhdollarerror

```
int P_const::lhdollarerror
```

Definition at line 1669 of file [structs.h](#).

3.100.2.37 eat

```
int P_const::eat
```

Definition at line 1670 of file [structs.h](#).

3.100.2.38 gNumPre

```
int P_const::gNumPre
```

Definition at line 1671 of file [structs.h](#).

3.100.2.39 PreDebug

```
int P_const::PreDebug
```

Definition at line 1672 of file [structs.h](#).

3.100.2.40 OpenDictionary

```
int P_const::OpenDictionary
```

Definition at line 1673 of file [structs.h](#).

3.100.2.41 PreAssignLevel

```
int P_const::PreAssignLevel
```

Definition at line 1674 of file [structs.h](#).

3.100.2.42 MaxPreAssignLevel

```
int P_const::MaxPreAssignLevel
```

Definition at line 1675 of file [structs.h](#).

3.100.2.43 fullnamesize

```
int P_const::fullnamesize
```

Definition at line 1676 of file [structs.h](#).

3.100.2.44 DebugFlag

```
WORD P_const::DebugFlag
```

Definition at line 1677 of file [structs.h](#).

3.100.2.45 preError

```
WORD P_const::preError
```

Definition at line 1678 of file [structs.h](#).

3.100.2.46 ComChar

```
UBYTE P_const::ComChar
```

Definition at line 1679 of file [structs.h](#).

3.100.2.47 cComChar

```
UBYTE P_const::cComChar
```

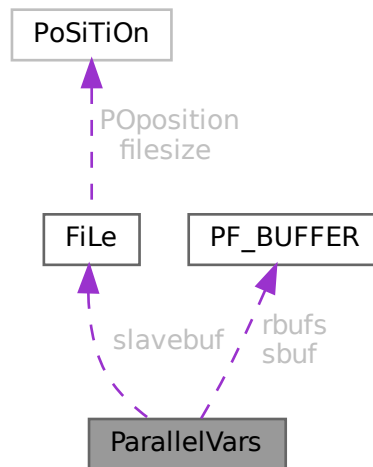
Definition at line 1680 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.101 ParallelVars Struct Reference

Collaboration diagram for ParallelVars:



Public Member Functions

- **PADPOSITION** (2, 0, 8, 2, 0)

Data Fields

- [FILEHANDLE](#) [slavebuf](#)
- [PF_BUFFER](#) * [sbuf](#)
- [PF_BUFFER](#) ** [rbufs](#)
- int [me](#)
- int [numtasks](#)
- int [parallel](#)
- int [rhsInParallel](#)
- int [mkSlaveInfile](#)
- int [exprbufsize](#)
- int [exprtodo](#)
- int [log](#)
- WORD [numsbufs](#)
- WORD [numrbufs](#)

3.101.1 Detailed Description

Definition at line 170 of file [parallel.h](#).

3.101.2 Field Documentation

3.101.2.1 slavebuf

```
FILEHANDLE ParallelVars::slavebuf
```

Definition at line 171 of file [parallel.h](#).

3.101.2.2 sbuf

```
PF_BUFFER* ParallelVars::sbuf
```

Definition at line 173 of file [parallel.h](#).

3.101.2.3 rbufs

```
PF_BUFFER** ParallelVars::rbufs
```

Definition at line 174 of file [parallel.h](#).

3.101.2.4 me

```
int ParallelVars::me
```

Definition at line 175 of file [parallel.h](#).

3.101.2.5 numtasks

```
int ParallelVars::numtasks
```

Definition at line 176 of file [parallel.h](#).

3.101.2.6 parallel

```
int ParallelVars::parallel
```

Definition at line 177 of file [parallel.h](#).

3.101.2.7 rhsInParallel

```
int ParallelVars::rhsInParallel
```

Definition at line 179 of file [parallel.h](#).

3.101.2.8 mkSlaveInfile

```
int ParallelVars::mkSlaveInfile
```

Definition at line 180 of file [parallel.h](#).

3.101.2.9 exprbufsize

```
int ParallelVars::exprbufsize
```

Definition at line 181 of file [parallel.h](#).

3.101.2.10 exptodo

```
int ParallelVars::exptodo
```

Definition at line 182 of file [parallel.h](#).

3.101.2.11 log

```
int ParallelVars::log
```

Definition at line 183 of file [parallel.h](#).

3.101.2.12 numsbufs

```
WORD ParallelVars::numsbufs
```

Definition at line 184 of file [parallel.h](#).

3.101.2.13 numrbufs

```
WORD ParallelVars::numrbufs
```

Definition at line 185 of file [parallel.h](#).

The documentation for this struct was generated from the following file:

- [parallel.h](#)

3.102 PaRtl Struct Reference

```
#include <structs.h>
```

Data Fields

- WORD * [psize](#)
- WORD * [args](#)
- WORD * [nargs](#)
- WORD * [nfun](#)
- WORD [numargs](#)
- WORD [numpart](#)
- WORD [where](#)

3.102.1 Detailed Description

The struct PARTI is used to help determining whether a `partition_` function can be replaced.

Definition at line [1120](#) of file [structs.h](#).

3.102.2 Field Documentation

3.102.2.1 psize

```
WORD* PaRtI::psize
```

Definition at line [1121](#) of file [structs.h](#).

3.102.2.2 args

```
WORD* PaRtI::args
```

Definition at line [1122](#) of file [structs.h](#).

3.102.2.3 nargs

```
WORD* PaRtI::nargs
```

Definition at line [1123](#) of file [structs.h](#).

3.102.2.4 nfun

```
WORD* PaRtI::nfun
```

Definition at line [1124](#) of file [structs.h](#).

3.102.2.5 numargs

WORD PaRtI::numargs

Definition at line 1125 of file [structs.h](#).

3.102.2.6 numpart

WORD PaRtI::numpart

Definition at line 1126 of file [structs.h](#).

3.102.2.7 where

WORD PaRtI::where

Definition at line 1127 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.103 PaRtlcLe Struct Reference

Data Fields

- WORD [number](#)
- WORD [spin](#)
- WORD [mass](#)
- WORD [type](#)

3.103.1 Detailed Description

Definition at line 619 of file [structs.h](#).

3.103.2 Field Documentation

3.103.2.1 number

WORD PaRtIcLe::number

Definition at line 620 of file [structs.h](#).

3.103.2.2 spin

WORD PaRtIcLe::spin

Definition at line 621 of file [structs.h](#).

3.103.2.3 mass

WORD PaRtIcLe::mass

Definition at line 622 of file [structs.h](#).

3.103.2.4 type

WORD PaRtIcLe::type

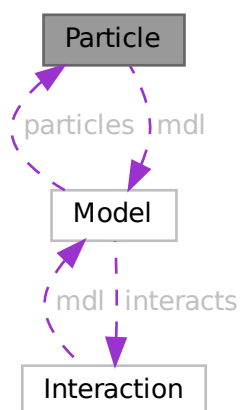
Definition at line 623 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.104 Particle Class Reference

Collaboration diagram for Particle:



Public Member Functions

- [Particle](#) ([Model](#) *modl, int pid, [PInput](#) *pinp)
- char * [particleName](#) (int p)
- int [particleCode](#) (int p)
- char * [interactionName](#) (int p)
- char * [aparticle](#) (void)
- void [prParticle](#) (void)
- int [isNeutral](#) (void)
- const char * [typeName](#) (void)
- const char * [typeGName](#) (void)

Data Fields

- [Model](#) * [mdl](#)
- char * [name](#)
- char * [aname](#)
- int [id](#)
- int [ptype](#)
- int [neutral](#)
- int [pcode](#)
- int [acode](#)
- int [cmindeg](#)
- int [cmaxdeg](#)
- int [extonly](#)

3.104.1 Detailed Description

Definition at line 205 of file [grcc.h](#).

3.104.2 Constructor & Destructor Documentation

3.104.2.1 Particle()

```
Particle::Particle (  
    Model * modl,  
    int pid,  
    PInput * pinp )
```

Definition at line 1422 of file [grcc.cc](#).

3.104.2.2 ~Particle()

```
Particle::~~Particle (  
    void )
```

Definition at line 1499 of file [grcc.cc](#).

3.104.3 Member Function Documentation

3.104.3.1 `particleName()`

```
char * Particle::particleName (  
    int p )
```

Definition at line 1506 of file [grcc.cc](#).

3.104.3.2 `particleCode()`

```
int Particle::particleCode (  
    int p )
```

Definition at line 1516 of file [grcc.cc](#).

3.104.3.3 `aparticle()`

```
char * Particle::aparticle (  
    void )
```

Definition at line 1544 of file [grcc.cc](#).

3.104.3.4 `prParticle()`

```
void Particle::prParticle (  
    void )
```

Definition at line 1556 of file [grcc.cc](#).

3.104.3.5 `isNeutral()`

```
int Particle::isNeutral (  
    void )
```

Definition at line 1526 of file [grcc.cc](#).

3.104.3.6 `typeName()`

```
const char * Particle::typeName (  
    void )
```

Definition at line 1532 of file [grcc.cc](#).

3.104.3.7 typeGName()

```
const char * Particle::typeGName (  
    void )
```

Definition at line 1538 of file [grcc.cc](#).

3.104.4 Field Documentation

3.104.4.1 mdl

```
Model* Particle::mdl
```

Definition at line 207 of file [grcc.h](#).

3.104.4.2 name

```
char* Particle::name
```

Definition at line 208 of file [grcc.h](#).

3.104.4.3 aname

```
char* Particle::aname
```

Definition at line 209 of file [grcc.h](#).

3.104.4.4 id

```
int Particle::id
```

Definition at line 210 of file [grcc.h](#).

3.104.4.5 ptype

```
int Particle::ptype
```

Definition at line 211 of file [grcc.h](#).

3.104.4.6 neutral

```
int Particle::neutral
```

Definition at line 212 of file [grcc.h](#).

3.104.4.7 pcode

```
int Particle::pcode
```

Definition at line 213 of file [grcc.h](#).

3.104.4.8 acode

```
int Particle::acode
```

Definition at line 214 of file [grcc.h](#).

3.104.4.9 cmindeg

```
int Particle::cmindeg
```

Definition at line 215 of file [grcc.h](#).

3.104.4.10 cmaxdeg

```
int Particle::cmaxdeg
```

Definition at line 216 of file [grcc.h](#).

3.104.4.11 extonly

```
int Particle::extonly
```

Definition at line 218 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.105 PeRmUtE Struct Reference

```
#include <structs.h>
```

Data Fields

- WORD * [objects](#)
- WORD [sign](#)
- WORD [n](#)
- WORD [cycle](#) [MAXMATCH]

3.105.1 Detailed Description

The struct PERM is used to generate all permutations when the pattern matcher has to try to match (anti)symmetric functions.

Definition at line 1078 of file [structs.h](#).

3.105.2 Field Documentation

3.105.2.1 objects

```
WORD* PeRmUtE::objects
```

Definition at line 1079 of file [structs.h](#).

3.105.2.2 sign

```
WORD PeRmUtE::sign
```

Definition at line 1080 of file [structs.h](#).

3.105.2.3 n

```
WORD PeRmUtE::n
```

Definition at line 1081 of file [structs.h](#).

3.105.2.4 cycle

```
WORD PeRmUtE::cycle[MAXMATCH]
```

Definition at line 1082 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.106 PeRmUtEp Struct Reference

```
#include <structs.h>
```

Data Fields

- WORD ** [objects](#)
- WORD [sign](#)
- WORD [n](#)
- WORD [cycle](#) [MAXMATCH]

3.106.1 Detailed Description

Like struct PERM but works with pointers.

Definition at line [1090](#) of file [structs.h](#).

3.106.2 Field Documentation

3.106.2.1 objects

```
WORD** PeRmUtEp::objects
```

Definition at line [1091](#) of file [structs.h](#).

3.106.2.2 sign

```
WORD PeRmUtEp::sign
```

Definition at line [1092](#) of file [structs.h](#).

3.106.2.3 n

```
WORD PeRmUtEp::n
```

Definition at line [1093](#) of file [structs.h](#).

3.106.2.4 cycle

```
WORD PeRmUtEp::cycle[MAXMATCH]
```

Definition at line [1094](#) of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.107 PF_BUFFER Struct Reference

```
#include <parallel.h>
```

Data Fields

- WORD ** [buff](#)
- WORD ** [fill](#)
- WORD ** [full](#)
- WORD ** [stop](#)
- MPI_Status * [status](#)
- MPI_Status * [retstat](#)
- MPI_Request * [request](#)
- MPI_Datatype * [type](#)
- int * [index](#)
- int * [tag](#)
- int * [from](#)
- int [numbufs](#)
- int [active](#)

3.107.1 Detailed Description

A struct for nonblocking, unbuffered send of the sorted terms in the PObuffers back to the master using several "rotating" PObuffers.

Definition at line [148](#) of file [parallel.h](#).

3.107.2 Field Documentation

3.107.2.1 buff

```
WORD** PF_BUFFER::buff
```

Definition at line [149](#) of file [parallel.h](#).

3.107.2.2 fill

```
WORD** PF_BUFFER::fill
```

Definition at line [150](#) of file [parallel.h](#).

3.107.2.3 full

```
WORD** PF_BUFFER::full
```

Definition at line [151](#) of file [parallel.h](#).

3.107.2.4 stop

```
WORD** PF_BUFFER::stop
```

Definition at line [152](#) of file [parallel.h](#).

3.107.2.5 status

```
MPI_Status* PF_BUFFER::status
```

Definition at line 153 of file [parallel.h](#).

3.107.2.6 retstat

```
MPI_Status* PF_BUFFER::retstat
```

Definition at line 154 of file [parallel.h](#).

3.107.2.7 request

```
MPI_Request* PF_BUFFER::request
```

Definition at line 155 of file [parallel.h](#).

3.107.2.8 type

```
MPI_Datatype* PF_BUFFER::type
```

Definition at line 156 of file [parallel.h](#).

3.107.2.9 index

```
int* PF_BUFFER::index
```

Definition at line 157 of file [parallel.h](#).

3.107.2.10 tag

```
int* PF_BUFFER::tag
```

Definition at line 158 of file [parallel.h](#).

3.107.2.11 from

```
int* PF_BUFFER::from
```

Definition at line 159 of file [parallel.h](#).

3.107.2.12 numbufs

```
int PF_BUFFER::numbufs
```

Definition at line 160 of file [parallel.h](#).

3.107.2.13 active

```
int PF_BUFFER::active
```

Definition at line 161 of file [parallel.h](#).

The documentation for this struct was generated from the following file:

- [parallel.h](#)

3.108 PGInput Struct Reference

Data Fields

- int [cdeg](#)
- int [ctyp](#)
- int [cnum](#)
- int [cple](#)
- const char * [pname](#)
- long [pcode](#)

3.108.1 Detailed Description

Definition at line 176 of file [grccparam.h](#).

3.108.2 Field Documentation

3.108.2.1 cdeg

```
int PGInput::cdeg
```

Definition at line 177 of file [grccparam.h](#).

3.108.2.2 ctyp

```
int PGInput::ctyp
```

Definition at line 178 of file [grccparam.h](#).

3.108.2.3 cnum

```
int PGInput::cnum
```

Definition at line 179 of file [grccparam.h](#).

3.108.2.4 cple

```
int PGInput::cple
```

Definition at line 180 of file [grccparam.h](#).

3.108.2.5 pname

```
const char* PGInput::pname
```

Definition at line 182 of file [grccparam.h](#).

3.108.2.6 pcode

```
long PGInput::pcode
```

Definition at line 184 of file [grccparam.h](#).

The documentation for this struct was generated from the following file:

- [grccparam.h](#)

3.109 PInput Struct Reference

Data Fields

- const char * [name](#)
- const char * [aname](#)
- int [pcode](#)
- int [acode](#)
- const char * [ptypen](#)
- int [ptypec](#)
- int [extonly](#)

3.109.1 Detailed Description

Definition at line 127 of file [grccparam.h](#).

3.109.2 Field Documentation

3.109.2.1 name

```
const char* PInput::name
```

Definition at line 128 of file [grccparam.h](#).

3.109.2.2 aname

```
const char* PInput::aname
```

Definition at line 129 of file [grccparam.h](#).

3.109.2.3 pcode

```
int PInput::pcode
```

Definition at line 130 of file [grccparam.h](#).

3.109.2.4 acode

```
int PInput::acode
```

Definition at line 131 of file [grccparam.h](#).

3.109.2.5 ptypen

```
const char* PInput::ptypen
```

Definition at line 132 of file [grccparam.h](#).

3.109.2.6 ptypec

```
int PInput::ptypec
```

Definition at line 133 of file [grccparam.h](#).

3.109.2.7 extonly

```
int PInput::extonly
```

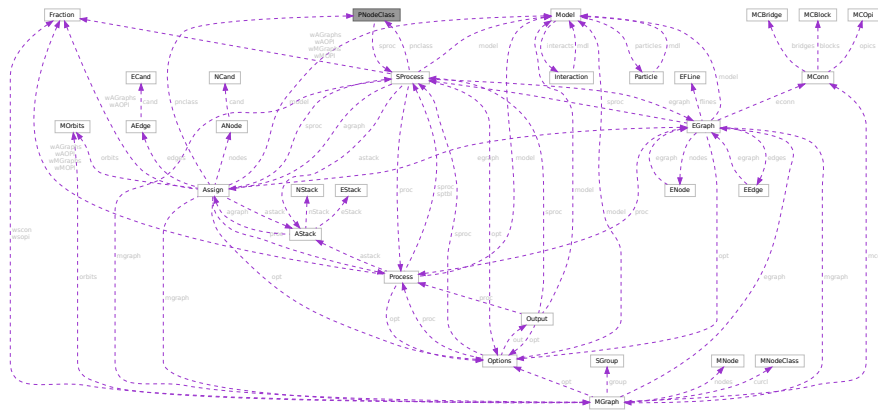
Definition at line 134 of file [grccparam.h](#).

The documentation for this struct was generated from the following file:

- [grccparam.h](#)

3.110 PNodeClass Class Reference

Collaboration diagram for PNodeClass:



Public Member Functions

- [PNodeClass](#) ([SProcess](#) *sprc, int nnods, int nclss, [NCInput](#) *cls)
- [PNodeClass](#) ([SProcess](#) *sprc, int nnods, int nclss, int *dgs, int *typ, int *ptcl, int *cpl, int *cnt, int *cmind, int *cmaxd)
- void [prPNodeClass](#) (void)
- void [prElem](#) (int e)

Data Fields

- [SProcess](#) * [sprc](#)
- int * [deg](#)
- int * [type](#)
- int * [particle](#)
- int * [couple](#)
- int * [cmindeg](#)
- int * [cmaxdeg](#)
- int * [count](#)
- int * [cl2nd](#)
- int * [nd2cl](#)
- int * [cl2mcl](#)
- int [nnodes](#)
- int [nclss](#)

3.110.1 Detailed Description

Definition at line 325 of file [grcc.h](#).

3.110.2 Constructor & Destructor Documentation

3.110.2.1 PNodeClass() [1/2]

```
PNodeClass::PNodeClass (
    SProcess * sprc,
    int nnods,
    int nclss,
    NCInput * cls )
```

Definition at line 2325 of file [grcc.cc](#).

3.110.2.2 PNodeClass() [2/2]

```
PNodeClass::PNodeClass (
    SProcess * sprc,
    int nnods,
    int nclss,
    int * dgs,
    int * typ,
    int * ptcl,
    int * cpl,
    int * cnt,
    int * cmind,
    int * cmaxd )
```

Definition at line 2399 of file [grcc.cc](#).

3.110.2.3 ~PNodeClass()

```
PNodeClass::~~PNodeClass (
    void )
```

Definition at line 2477 of file [grcc.cc](#).

3.110.3 Member Function Documentation

3.110.3.1 prPNodeClass()

```
void PNodeClass::prPNodeClass (
    void )
```

Definition at line 2492 of file [grcc.cc](#).

3.110.3.2 prElem()

```
void PNodeClass::prElem (
    int e )
```

Definition at line 2504 of file [grcc.cc](#).

3.110.4 Field Documentation

3.110.4.1 `sproc`

`SProcess*` `PNodeClass::sproc`

Definition at line 327 of file [grcc.h](#).

3.110.4.2 `deg`

`int*` `PNodeClass::deg`

Definition at line 328 of file [grcc.h](#).

3.110.4.3 `type`

`int*` `PNodeClass::type`

Definition at line 329 of file [grcc.h](#).

3.110.4.4 `particle`

`int*` `PNodeClass::particle`

Definition at line 330 of file [grcc.h](#).

3.110.4.5 `couple`

`int*` `PNodeClass::couple`

Definition at line 331 of file [grcc.h](#).

3.110.4.6 `cmindeg`

`int*` `PNodeClass::cmindeg`

Definition at line 332 of file [grcc.h](#).

3.110.4.7 `cmaxdeg`

`int*` `PNodeClass::cmaxdeg`

Definition at line 333 of file [grcc.h](#).

3.110.4.8 count

```
int* PNodeClass::count
```

Definition at line 334 of file [grcc.h](#).

3.110.4.9 cl2nd

```
int* PNodeClass::cl2nd
```

Definition at line 335 of file [grcc.h](#).

3.110.4.10 nd2cl

```
int* PNodeClass::nd2cl
```

Definition at line 336 of file [grcc.h](#).

3.110.4.11 cl2mcl

```
int* PNodeClass::cl2mcl
```

Definition at line 337 of file [grcc.h](#).

3.110.4.12 nnodes

```
int PNodeClass::nnodes
```

Definition at line 338 of file [grcc.h](#).

3.110.4.13 nclass

```
int PNodeClass::nclass
```

Definition at line 339 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.111 flint::poly Class Reference

Public Member Functions

- void [print](#) (const string &text)

Data Fields

- `mpz_poly_t` [d](#)

3.111.1 Detailed Description

Definition at line [61](#) of file [flintinterface.h](#).

3.111.2 Constructor & Destructor Documentation

3.111.2.1 `poly()`

```
flint::poly::poly ( ) [inline]
```

Definition at line [64](#) of file [flintinterface.h](#).

3.111.2.2 `~poly()`

```
flint::poly::~~poly ( ) [inline]
```

Definition at line [65](#) of file [flintinterface.h](#).

3.111.3 Member Function Documentation

3.111.3.1 `print()`

```
void flint::poly::print (
    const string & text ) [inline]
```

Definition at line [66](#) of file [flintinterface.h](#).

3.111.4 Field Documentation

3.111.4.1 `d`

```
mpz_poly_t flint::poly::d
```

Definition at line [63](#) of file [flintinterface.h](#).

The documentation for this class was generated from the following file:

- [flintinterface.h](#)

3.112 poly Class Reference

Public Member Functions

- [poly](#) (PHEAD int, WORD=-1, WORD=1)
- [poly](#) (PHEAD const UWORD *, WORD, WORD=-1, WORD=1)
- [poly](#) (const [poly](#) &, WORD=-1, WORD=1)
- [poly](#) & [operator+=](#) (const [poly](#) &)
- [poly](#) & [operator-=](#) (const [poly](#) &)
- [poly](#) & [operator*=](#) (const [poly](#) &)
- [poly](#) & [operator/=](#) (const [poly](#) &)
- [poly](#) & [operator%=>](#) (const [poly](#) &)
- const [poly](#) [operator+](#) (const [poly](#) &) const
- const [poly](#) [operator-](#) (const [poly](#) &) const
- const [poly](#) [operator*](#) (const [poly](#) &) const
- const [poly](#) [operator/](#) (const [poly](#) &) const
- const [poly](#) [operator%](#) (const [poly](#) &) const
- bool [operator==](#) (const [poly](#) &) const
- bool [operator!=](#) (const [poly](#) &) const
- [poly](#) & [operator=](#) (const [poly](#) &)
- WORD & [operator\[\]](#) (int)
- const WORD & [operator\[\]](#) (int) const
- void [termscopy](#) (const WORD *, int, int)
- void [check_memory](#) (int)
- void [expand_memory](#) (int)
- bool [is_zero](#) () const
- bool [is_one](#) () const
- bool [is_integer](#) () const
- bool [is_monomial](#) () const
- int [is_dense_univariate](#) () const
- int [sign](#) () const
- int [degree](#) (int) const
- int [total_degree](#) () const
- int [first_variable](#) () const
- int [number_of_terms](#) () const
- const std::vector< int > [all_variables](#) () const
- const [poly](#) [integer_lcoeff](#) () const
- const [poly](#) [lcoeff_univar](#) (int) const
- const [poly](#) [lcoeff_multivar](#) (int) const
- const [poly](#) [coefficient](#) (int, int) const
- const [poly](#) [derivative](#) (int) const
- void [setmod](#) (WORD, WORD=1)
- void [coefficients_modulo](#) (UWORD *, WORD, bool)
- int [size_of_form_notation](#) () const
- int [size_of_form_notation_with_den](#) (WORD) const
- const [poly](#) & [normalize](#) ()
- const std::string [to_string](#) () const
- WORD [last_monomial_index](#) () const
- WORD * [last_monomial](#) () const
- int [compare_degree_vector](#) (const [poly](#) &) const
- std::vector< int > [degree_vector](#) () const
- int [compare_degree_vector](#) (const std::vector< int > &) const

Static Public Member Functions

- static const [poly simple_poly](#) (PHEAD int, int=0, int=1, int=0, int=1)
- static const [poly simple_poly](#) (PHEAD int, const [poly](#) &, int=1, int=0, int=1)
- static void [get_variables](#) (PHEAD const std::vector< WORD * > &, bool, bool)
- static const [poly argument_to_poly](#) (PHEAD WORD *, bool, bool, [poly](#) *den=NULL)
- static void [poly_to_argument](#) (const [poly](#) &, WORD *, bool)
- static void [poly_to_argument_with_den](#) (const [poly](#) &, WORD, const UWORD *, WORD *, bool)
- static const [poly from_coefficient_list](#) (PHEAD const std::vector< WORD > &, int, WORD)
- static const std::vector< WORD > [to_coefficient_list](#) (const [poly](#) &)
- static const std::vector< WORD > [coefficient_list_divmod](#) (const std::vector< WORD > &, const std::vector< WORD > &, WORD, int)
- static int [monomial_compare](#) (PHEAD const WORD *, const WORD *)
- static void [add](#) (const [poly](#) &, const [poly](#) &, [poly](#) &)
- static void [sub](#) (const [poly](#) &, const [poly](#) &, [poly](#) &)
- static void [mul](#) (const [poly](#) &, const [poly](#) &, [poly](#) &)
- static void [div](#) (const [poly](#) &, const [poly](#) &, [poly](#) &)
- static void [mod](#) (const [poly](#) &, const [poly](#) &, [poly](#) &)
- static void [divmod](#) (const [poly](#) &, const [poly](#) &, [poly](#) &, [poly](#) &, bool only_divides)
- static bool [divides](#) (const [poly](#) &, const [poly](#) &)
- static void [mul_one_term](#) (const [poly](#) &, const [poly](#) &, [poly](#) &)
- static void [mul_univar](#) (const [poly](#) &, const [poly](#) &, [poly](#) &, int)
- static void [mul_heap](#) (const [poly](#) &, const [poly](#) &, [poly](#) &)
- static void [divmod_one_term](#) (const [poly](#) &, const [poly](#) &, [poly](#) &, [poly](#) &, bool)
- static void [divmod_univar](#) (const [poly](#) &, const [poly](#) &, [poly](#) &, [poly](#) &, int, bool)
- static void [divmod_heap](#) (const [poly](#) &, const [poly](#) &, [poly](#) &, [poly](#) &, bool, bool, bool &)
- static void [push_heap](#) (PHEAD WORD **, int)
- static void [pop_heap](#) (PHEAD WORD **, int)

Data Fields

- WORD * [terms](#)
- LONG [size_of_terms](#)
- WORD [modp](#)
- WORD [modn](#)

3.112.1 Detailed Description

Definition at line 53 of file [poly.h](#).

3.112.2 Constructor & Destructor Documentation

3.112.2.1 [poly\(\)](#) [1/3]

```
poly::poly (
    PHEAD int a,
    WORD modp = -1,
    WORD modn = 1 )
```

Definition at line 54 of file [poly.cc](#).

3.112.2.2 poly() [2/3]

```
poly::poly (
    PHEAD const UWORD * a,
    WORD na,
    WORD modp = -1,
    WORD modn = 1 )
```

Definition at line 83 of file [poly.cc](#).

3.112.2.3 poly() [3/3]

```
poly::poly (
    const poly & a,
    WORD modp = -1,
    WORD modn = 1 )
```

Definition at line 111 of file [poly.cc](#).

3.112.2.4 ~poly()

```
poly::~~poly ( )
```

Definition at line 133 of file [poly.cc](#).

3.112.3 Member Function Documentation

3.112.3.1 operator+=()

```
poly & poly::operator+= (
    const poly & a )
```

Definition at line 1966 of file [poly.cc](#).

3.112.3.2 operator-=()

```
poly & poly::operator-= (
    const poly & a )
```

Definition at line 1967 of file [poly.cc](#).

3.112.3.3 operator*=()

```
poly & poly::operator*= (
    const poly & a )
```

Definition at line 1968 of file [poly.cc](#).

3.112.3.4 operator/=()

```
poly & poly::operator/= (
    const poly & a )
```

Definition at line 1969 of file [poly.cc](#).

3.112.3.5 operator%=()

```
poly & poly::operator%= (
    const poly & a )
```

Definition at line 1970 of file [poly.cc](#).

3.112.3.6 operator+()

```
const poly poly::operator+ (
    const poly & a ) const
```

Definition at line 1959 of file [poly.cc](#).

3.112.3.7 operator-()

```
const poly poly::operator- (
    const poly & a ) const
```

Definition at line 1960 of file [poly.cc](#).

3.112.3.8 operator*()

```
const poly poly::operator* (
    const poly & a ) const
```

Definition at line 1961 of file [poly.cc](#).

3.112.3.9 operator/()

```
const poly poly::operator/ (
    const poly & a ) const
```

Definition at line 1962 of file [poly.cc](#).

3.112.3.10 operator%()

```
const poly poly::operator% (
    const poly & a ) const
```

Definition at line 1963 of file [poly.cc](#).

3.112.3.11 operator==()

```
bool poly::operator== (
    const poly & a ) const
```

Definition at line 1973 of file [poly.cc](#).

3.112.3.12 operator"!=()"

```
bool poly::operator!= (
    const poly & a ) const
```

Definition at line 1979 of file [poly.cc](#).

3.112.3.13 operator=()

```
poly & poly::operator= (
    const poly & a )
```

Definition at line 1939 of file [poly.cc](#).

3.112.3.14 operator[]() [1/2]

```
WORD & poly::operator[] (
    int i ) [inline]
```

Definition at line 249 of file [poly.h](#).

3.112.3.15 operator[]() [2/2]

```
const WORD & poly::operator[] (
    int i ) const [inline]
```

Definition at line 253 of file [poly.h](#).

3.112.3.16 termscopy()

```
void poly::termscopy (
    const WORD * source,
    int dest,
    int num ) [inline]
```

Definition at line 260 of file [poly.h](#).

3.112.3.17 check_memory()

```
void poly::check_memory (
    int i ) [inline]
```

Definition at line 242 of file [poly.h](#).

3.112.3.18 `expand_memory()`

```
void poly::expand_memory (
    int i )
```

Definition at line 149 of file [poly.cc](#).

3.112.3.19 `is_zero()`

```
bool poly::is_zero ( ) const
```

Definition at line 2209 of file [poly.cc](#).

3.112.3.20 `is_one()`

```
bool poly::is_one ( ) const
```

Definition at line 2219 of file [poly.cc](#).

3.112.3.21 `is_integer()`

```
bool poly::is_integer ( ) const
```

Definition at line 2239 of file [poly.cc](#).

3.112.3.22 `is_monomial()`

```
bool poly::is_monomial ( ) const
```

Definition at line 2259 of file [poly.cc](#).

3.112.3.23 `is_dense_univariate()`

```
int poly::is_dense_univariate ( ) const
```

Dense univariate detection

3.112.4 Description

This method returns whether the polynomial is dense and univariate. The possible return values are:

-2 is not dense univariate -1 is no variables $n \geq 0$ is univariate in n

3.112.5 Notes

A univariate polynomial is considered dense iff more than half of the coefficients `a_0...a_deg` are non-zero.

Definition at line 2284 of file [poly.cc](#).

Referenced by [divmod\(\)](#), and [mul\(\)](#).

3.112.5.1 `sign()`

```
int poly::sign ( ) const
```

Definition at line 2039 of file [poly.cc](#).

3.112.5.2 `degree()`

```
int poly::degree (
    int x ) const
```

Definition at line 2050 of file [poly.cc](#).

3.112.5.3 `total_degree()`

```
int poly::total_degree ( ) const
```

Definition at line 2063 of file [poly.cc](#).

3.112.5.4 `first_variable()`

```
int poly::first_variable ( ) const
```

Definition at line 2000 of file [poly.cc](#).

3.112.5.5 `number_of_terms()`

```
int poly::number_of_terms ( ) const
```

Definition at line 1986 of file [poly.cc](#).

3.112.5.6 `all_variables()`

```
const vector< int > poly::all_variables ( ) const
```

Definition at line 2016 of file [poly.cc](#).

3.112.5.7 integer_lcoeff()

```
const poly poly::integer_lcoeff ( ) const
```

Definition at line 2083 of file [poly.cc](#).

3.112.5.8 lcoeff_univar()

```
const poly poly::lcoeff_univar (
    int x ) const
```

Definition at line 2133 of file [poly.cc](#).

3.112.5.9 lcoeff_multivar()

```
const poly poly::lcoeff_multivar (
    int x ) const
```

Definition at line 2140 of file [poly.cc](#).

3.112.5.10 coefficient()

```
const poly poly::coefficient (
    int x,
    int n ) const
```

Definition at line 2105 of file [poly.cc](#).

3.112.5.11 derivative()

```
const poly poly::derivative (
    int x ) const
```

Definition at line 2172 of file [poly.cc](#).

3.112.5.12 setmod()

```
void poly::setmod (
    WORD _modp,
    WORD _modn = 1 )
```

Definition at line 179 of file [poly.cc](#).

3.112.5.13 coefficients_modulo()

```
void poly::coefficients_modulo (
    UWORD * a,
    WORD na,
    bool small )
```

Definition at line 215 of file [poly.cc](#).

3.112.5.14 simple_poly() [1/2]

```
const poly poly::simple_poly (
    PHEAD int x,
    int a = 0,
    int b = 1,
    int p = 0,
    int n = 1 ) [static]
```

Definition at line 2317 of file [poly.cc](#).

3.112.5.15 simple_poly() [2/2]

```
const poly poly::simple_poly (
    PHEAD int x,
    const poly & a,
    int b = 1,
    int p = 0,
    int n = 1 ) [static]
```

Definition at line 2356 of file [poly.cc](#).

3.112.5.16 get_variables()

```
void poly::get_variables (
    PHEAD const std::vector< WORD * > & ,
    bool ,
    bool ) [static]
```

Definition at line 2395 of file [poly.cc](#).

3.112.5.17 argument_to_poly()

```
const poly poly::argument_to_poly (
    PHEAD WORD * e,
    bool with_arghead,
    bool sort_univar,
    poly * den = NULL ) [static]
```

Definition at line 2498 of file [poly.cc](#).

3.112.5.18 poly_to_argument()

```
void poly::poly_to_argument (
    const poly & a,
    WORD * res,
    bool with_arghead ) [static]
```

Definition at line 2611 of file [poly.cc](#).

3.112.5.19 poly_to_argument_with_den()

```
void poly::poly_to_argument_with_den (
    const poly & a,
    WORD nden,
    const UWORD * den,
    WORD * res,
    bool with_arghead ) [static]
```

Definition at line 2683 of file [poly.cc](#).

3.112.5.20 size_of_form_notation()

```
int poly::size_of_form_notation ( ) const
```

Definition at line 2766 of file [poly.cc](#).

3.112.5.21 size_of_form_notation_with_den()

```
int poly::size_of_form_notation_with_den (
    WORD nden ) const
```

Definition at line 2795 of file [poly.cc](#).

3.112.5.22 normalize()

```
const poly & poly::normalize ( )
```

Definition at line 362 of file [poly.cc](#).

3.112.5.23 from_coefficient_list()

```
const poly poly::from_coefficient_list (
    PHEAD const std::vector< WORD > & ,
    int ,
    WORD ) [static]
```

Definition at line 2885 of file [poly.cc](#).

3.112.5.24 to_coefficient_list()

```
const vector< WORD > poly::to_coefficient_list (
    const poly & a ) [static]
```

Definition at line 2823 of file [poly.cc](#).

3.112.5.25 coefficient_list_divmod()

```
const vector< WORD > poly::coefficient_list_divmod (
    const std::vector< WORD > & ,
    const std::vector< WORD > & ,
    WORD ,
    int ) [static]
```

Definition at line 2846 of file [poly.cc](#).

3.112.5.26 to_string()

```
const string poly::to_string ( ) const
```

Definition at line 267 of file [poly.cc](#).

3.112.5.27 monomial_compare()

```
int poly::monomial_compare (
    PHEAD const WORD * a,
    const WORD * b ) [static]
```

Definition at line 350 of file [poly.cc](#).

3.112.5.28 last_monomial_index()

```
WORD poly::last_monomial_index ( ) const
```

Definition at line 465 of file [poly.cc](#).

3.112.5.29 last_monomial()

```
WORD * poly::last_monomial ( ) const
```

Definition at line 472 of file [poly.cc](#).

3.112.5.30 compare_degree_vector()

```
int poly::compare_degree_vector (
    const poly & b ) const
```

Definition at line 481 of file [poly.cc](#).

3.112.5.31 degree_vector()

```
std::vector< int > poly::degree_vector ( ) const
```

Definition at line 503 of file [poly.cc](#).

3.112.5.32 add()

```
void poly::add (
    const poly & a,
    const poly & b,
    poly & c ) [static]
```

Definition at line 538 of file [poly.cc](#).

3.112.5.33 sub()

```
void poly::sub (
    const poly & a,
    const poly & b,
    poly & c ) [static]
```

Definition at line 622 of file [poly.cc](#).

3.112.5.34 mul()

```
void poly::mul (
    const poly & a,
    const poly & b,
    poly & c ) [static]
```

Polynomial multiplication

3.112.6 Description

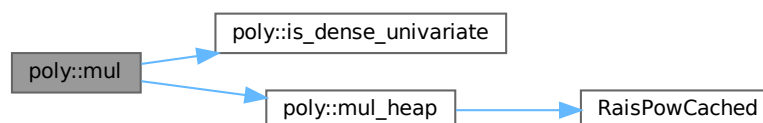
This routine determines which multiplication routine to use for multiplying two polynomials. The logic is as follows:

- If a or b consists of only one term, call `mul_one_term`;
- Otherwise, if both are univariate and dense, call `mul_univar`;
- Otherwise, call `mul_heap`.

Definition at line 1195 of file [poly.cc](#).

References [is_dense_univariate\(\)](#), and [mul_heap\(\)](#).

Here is the call graph for this function:



3.112.6.1 div()

```
void poly::div (
    const poly & a,
    const poly & b,
    poly & c ) [static]
```

Definition at line 1915 of file [poly.cc](#).

3.112.6.2 mod()

```
void poly::mod (
    const poly & a,
    const poly & b,
    poly & c ) [static]
```

Definition at line 1927 of file [poly.cc](#).

3.112.6.3 divmod()

```
void poly::divmod (
    const poly & a,
    const poly & b,
    poly & q,
    poly & r,
    bool only_divides ) [static]
```

Polynomial division

3.112.7 Description

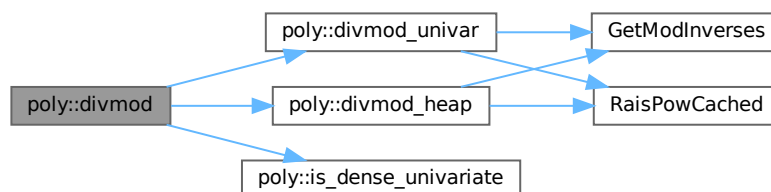
This routine determines which division routine to use for dividing two polynomials. The logic is as follows:

- If b consists of only one term, call `divmod_one_term`;
- Otherwise, if both are univariate and dense, call `divmod_univar`;
- Otherwise, call `divmod_heap`.

Definition at line 1856 of file [poly.cc](#).

References [divmod_heap\(\)](#), [divmod_univar\(\)](#), and [is_dense_univariate\(\)](#).

Here is the call graph for this function:



3.112.7.1 divides()

```
bool poly::divides (
    const poly & a,
    const poly & b ) [static]
```

Definition at line 1902 of file [poly.cc](#).

3.112.7.2 mul_one_term()

```
void poly::mul_one_term (
    const poly & a,
    const poly & b,
    poly & c ) [static]
```

Definition at line 755 of file [poly.cc](#).

3.112.7.3 mul_univar()

```
void poly::mul_univar (
    const poly & a,
    const poly & b,
    poly & c,
    int var ) [static]
```

Definition at line 829 of file [poly.cc](#).

3.112.7.4 mul_heap()

```
void poly::mul_heap (
    const poly & a,
    const poly & b,
    poly & c ) [static]
```

Multiplication of polynomials with a heap

3.112.8 Description

Multiplies two multivariate polynomials. The next element of the product is efficiently determined by using a heap. If the product of the maximum power in all variables is small, a hash table is used to add equal terms for extra speed.

A heap element h is formatted as follows:

- h[0] = index in a
- h[1] = index in b
- h[2] = hash code (-1 if no hash is used)
- h[3] = length of coefficient with sign
- h[4...4+AN.poly_num_vars-1] = powers

- $h[4+AN.poly_num_vars...4+h[3]-1]$ = coefficient

Definition at line 961 of file [poly.cc](#).

References [RaisPowCached\(\)](#).

Referenced by [mul\(\)](#).

Here is the call graph for this function:



3.112.8.1 `divmod_one_term()`

```
void poly::divmod_one_term (  
    const poly & a,  
    const poly & b,  
    poly & q,  
    poly & r,  
    bool only_divides ) [static]
```

Definition at line 1245 of file [poly.cc](#).

3.112.8.2 `divmod_univar()`

```
void poly::divmod_univar (  
    const poly & a,  
    const poly & b,  
    poly & q,  
    poly & r,  
    int var,  
    bool only_divides ) [static]
```

Division of dense univariate polynomials.

3.112.9 Description

Divides two dense univariate polynomials. For each power, the method collects all terms that result in that power.

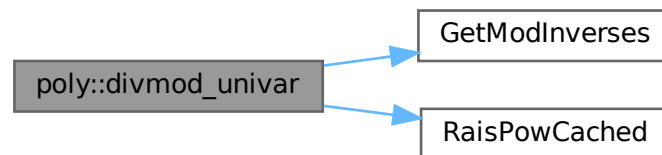
Relevant formula $[Q=A/B, P=\text{SUM}(p_i \cdot x^i), n=\text{deg}(A), m=\text{deg}(B)]: q_k = [a_{m+k} - \text{SUM}(i=k+1 \dots n-m, b_{m+k-i} \cdot q_i)] / b_m$

Definition at line 1376 of file [poly.cc](#).

References [GetModInverses\(\)](#), and [RaisPowCached\(\)](#).

Referenced by [divmod\(\)](#).

Here is the call graph for this function:



3.112.9.1 divmod_heap()

```

void poly::divmod_heap (
    const poly & a,
    const poly & b,
    poly & q,
    poly & r,
    bool only_divides,
    bool check_div,
    bool & div_fail ) [static]
  
```

Division of polynomials with a heap

3.112.10 Description

Divides two multivariate polynomials. The next element of the quotient/remainder is efficiently determined by using a heap. If the product of the maximum power in all variables is small, a hash table is used to add equal terms for extra speed.

If the input flag `check_div` is set then if the result of any coefficient division results in a non-zero remainder (indicating that division has failed over the integers) the output flag `div_fail` will be set and the division will be terminated early (`q`, `r` will be incorrect). If `check_div` is not set then non-zero remainders from coefficient division will be written into `r`.

A heap element `h` is formatted as follows:

- $h[0]$ = index in a
- $h[1]$ = index in b
- $h[2] = -1$ (no hash is used)
- $h[3]$ = length of coefficient with sign
- $h[4 \dots 4 + \text{AN.poly_num_vars} - 1]$ = powers
- $h[4 + \text{AN.poly_num_vars} \dots 4 + h[3] - 1]$ = coefficient

Note: the hashing trick as in multiplication cannot be used easily, since there is no tight upperbound on the exponents in the answer.

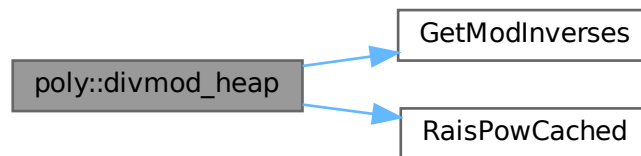
For details, see M. Monagan, "Polynomial Division using Dynamic Array, Heaps, and Packed Exponent Vectors"

Definition at line 1579 of file [poly.cc](#).

References [GetModInverses\(\)](#), and [RaisPowCached\(\)](#).

Referenced by [divmod\(\)](#).

Here is the call graph for this function:



3.112.10.1 `push_heap()`

```
void poly::push_heap (
    PHEAD WORD ** heap,
    int n ) [static]
```

Definition at line 739 of file [poly.cc](#).

3.112.10.2 `pop_heap()`

```
void poly::pop_heap (
    PHEAD WORD ** heap,
    int n ) [static]
```

Definition at line 707 of file [poly.cc](#).

3.112.11 Field Documentation

3.112.11.1 terms

WORD* poly::terms

Definition at line 62 of file [poly.h](#).

3.112.11.2 size_of_terms

LONG poly::size_of_terms

Definition at line 63 of file [poly.h](#).

3.112.11.3 modp

WORD poly::modp

Definition at line 64 of file [poly.h](#).

3.112.11.4 modn

WORD poly::modn

Definition at line 64 of file [poly.h](#).

The documentation for this class was generated from the following files:

- [poly.h](#)
- [poly.cc](#)

3.113 flint::poly_factor Class Reference

Data Fields

- [fmpz_poly_factor_t](#) [d](#)

3.113.1 Detailed Description

Definition at line 70 of file [flintinterface.h](#).

3.113.2 Constructor & Destructor Documentation

3.113.2.1 `poly_factor()`

```
flint::poly_factor::poly_factor ( ) [inline]
```

Definition at line 73 of file [flintinterface.h](#).

3.113.2.2 `~poly_factor()`

```
flint::poly_factor::~~poly_factor ( ) [inline]
```

Definition at line 74 of file [flintinterface.h](#).

3.113.3 Field Documentation

3.113.3.1 `d`

```
fmprz_poly_factor_t flint::poly_factor::d
```

Definition at line 72 of file [flintinterface.h](#).

The documentation for this class was generated from the following file:

- [flintinterface.h](#)

3.114 POLYMOD Struct Reference

```
#include <structs.h>
```

Data Fields

- WORD * [coefs](#)
- WORD [numsym](#)
- WORD [arraysize](#)
- WORD [polysize](#)
- WORD [modnum](#)

3.114.1 Detailed Description

The [POLYMOD](#) struct controls one univariate polynomial of which the coefficients have been taken modulus a (prime) number that fits inside a variable of type WORD. The polynomial is stored as an array of coefficients of size WORD.

Definition at line 1268 of file [structs.h](#).

3.114.2 Field Documentation

3.114.2.1 `coefs`

`WORD* POLYMOD::coefs`

Definition at line 1269 of file [structs.h](#).

3.114.2.2 `numsym`

`WORD POLYMOD::numsym`

Definition at line 1270 of file [structs.h](#).

3.114.2.3 `arraysize`

`WORD POLYMOD::arraysize`

Definition at line 1271 of file [structs.h](#).

3.114.2.4 `polysize`

`WORD POLYMOD::polysize`

Definition at line 1272 of file [structs.h](#).

3.114.2.5 `modnum`

`WORD POLYMOD::modnum`

Definition at line 1273 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.115 PoSiTiOn Struct Reference

Data Fields

- `off_t p1`

3.115.1 Detailed Description

Definition at line 59 of file [structs.h](#).

3.115.2 Field Documentation

3.115.2.1 p1

```
off_t PoSiTiOn::p1
```

Definition at line 60 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.116 PRELOAD Struct Reference

```
#include <structs.h>
```

Data Fields

- UBYTE * [buffer](#)
- LONG [size](#)

3.116.1 Detailed Description

Used by the preprocessor to load the contents of a doloop or a procedure. The struct [PRELOAD](#) is used both in the DOLOOP and [PROCEDURE](#) structs.

Definition at line 873 of file [structs.h](#).

3.116.2 Field Documentation

3.116.2.1 buffer

```
UBYTE* PRELOAD::buffer
```

Definition at line 874 of file [structs.h](#).

3.116.2.2 size

```
LONG PRELOAD::size
```

Definition at line 875 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.117 pReVaR Struct Reference

```
#include <structs.h>
```

Data Fields

- UBYTE * [name](#)
- UBYTE * [value](#)
- UBYTE * [argnames](#)
- int [nargs](#)
- int [wildarg](#)

3.117.1 Detailed Description

An element of the type PREVAR is needed for each preprocessor variable.

Definition at line [841](#) of file [structs.h](#).

3.117.2 Field Documentation

3.117.2.1 name

```
UBYTE* pReVaR::name
```

allocated

Definition at line [842](#) of file [structs.h](#).

3.117.2.2 value

```
UBYTE* pReVaR::value
```

points into memory of name

Definition at line [843](#) of file [structs.h](#).

3.117.2.3 argnames

```
UBYTE* pReVaR::argnames
```

names of arguments, zero separated. points into memory of name

Definition at line [844](#) of file [structs.h](#).

3.117.2.4 nargs

```
int pReVaR::nargs
```

0 = regular, >= 1: number of macro arguments. total number

Definition at line 845 of file [structs.h](#).

3.117.2.5 wildarg

```
int pReVaR::wildarg
```

The number of a potential ?var. If none: 0. wildarg<nargs

Definition at line 846 of file [structs.h](#).

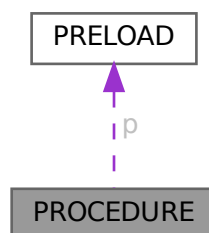
The documentation for this struct was generated from the following file:

- [structs.h](#)

3.118 PROCEDURE Struct Reference

```
#include <structs.h>
```

Collaboration diagram for PROCEDURE:



Data Fields

- [PRELOAD](#) p
- UBYTE * [name](#)
- int [loadmode](#)

Public Member Functions

- [Process](#) (int pid, [Model](#) *model, [Options](#) *opt, int nin, int *initlPart, int nfin, int *finalPart, int *coupling)
- **Process** (int pid, [Model](#) *model, [Options](#) *opt, [FGInput](#) *fgi)
- void [prProcess](#) (void)
- void [mkSProcess](#) (void)

Data Fields

- [Model](#) * [model](#)
- [Options](#) * [opt](#)
- [BigInt](#) [mgrcount](#)
- [BigInt](#) [agrcount](#)
- int * [initlPart](#)
- int * [finalPart](#)
- int [id](#)
- int [ninitl](#)
- int [nfinal](#)
- int [ctotal](#)
- int [nExtern](#)
- int [loop](#)
- int [maxnlegs](#)
- int [clist](#) [GRCC_MAXNCPLG]
- int [nSubproc](#)
- [SProcess](#) * [sptbl](#) [GRCC_MAXSUBPROCS]
- [SProcess](#) * [sproc](#)
- [AStack](#) * [astack](#)
- [BigInt](#) [ngraphs](#)
- [BigInt](#) [nopi](#)
- [BigInt](#) [wgraphs](#)
- [BigInt](#) [wopi](#)
- [BigInt](#) [nMGraphs](#)
- [BigInt](#) [nMOPI](#)
- [Fraction](#) [wMGraphs](#)
- [Fraction](#) [wMOPI](#)
- [BigInt](#) [nAGraphs](#)
- [BigInt](#) [nAOPI](#)
- [Fraction](#) [wAGraphs](#)
- [Fraction](#) [wAOPI](#)
- double [sec](#)

3.119.1 Detailed Description

Definition at line 424 of file [grcc.h](#).

3.119.2 Constructor & Destructor Documentation

3.119.2.1 Process()

```
Process::Process (
    int pid,
    Model * model,
    Options * opt,
    int nin,
    int * initlPart,
    int nfin,
    int * finalPart,
    int * coupling )
```

Definition at line 3036 of file [grcc.cc](#).

3.119.2.2 ~Process()

```
Process::~~Process (
    void )
```

Definition at line 3134 of file [grcc.cc](#).

3.119.3 Member Function Documentation

3.119.3.1 prProcess()

```
void Process::prProcess (
    void )
```

Definition at line 3142 of file [grcc.cc](#).

3.119.3.2 mkSProcess()

```
void Process::mkSProcess (
    void )
```

Definition at line 3150 of file [grcc.cc](#).

3.119.4 Field Documentation

3.119.4.1 model

```
Model* Process::model
```

Definition at line 426 of file [grcc.h](#).

3.119.4.2 opt

`Options* Process::opt`

Definition at line 427 of file [grcc.h](#).

3.119.4.3 mgrcount

`BigInt Process::mgrcount`

Definition at line 428 of file [grcc.h](#).

3.119.4.4 agrcount

`BigInt Process::agrcount`

Definition at line 429 of file [grcc.h](#).

3.119.4.5 initlPart

`int* Process::initlPart`

Definition at line 430 of file [grcc.h](#).

3.119.4.6 finalPart

`int* Process::finalPart`

Definition at line 431 of file [grcc.h](#).

3.119.4.7 id

`int Process::id`

Definition at line 433 of file [grcc.h](#).

3.119.4.8 ninitl

`int Process::ninitl`

Definition at line 434 of file [grcc.h](#).

3.119.4.9 nfinal

`int Process::nfinal`

Definition at line 435 of file [grcc.h](#).

3.119.4.10 ctotat

```
int Process::ctotat
```

Definition at line 436 of file [grcc.h](#).

3.119.4.11 nExtern

```
int Process::nExtern
```

Definition at line 437 of file [grcc.h](#).

3.119.4.12 loop

```
int Process::loop
```

Definition at line 438 of file [grcc.h](#).

3.119.4.13 maxnlegs

```
int Process::maxnlegs
```

Definition at line 439 of file [grcc.h](#).

3.119.4.14 clist

```
int Process::clist[GRCC_MAXNCPLG]
```

Definition at line 440 of file [grcc.h](#).

3.119.4.15 nSubproc

```
int Process::nSubproc
```

Definition at line 444 of file [grcc.h](#).

3.119.4.16 sptbl

```
SProcess* Process::sptbl[GRCC_MAXSUBPROCS]
```

Definition at line 445 of file [grcc.h](#).

3.119.4.17 sproc

```
SProcess* Process::sproc
```

Definition at line 446 of file [grcc.h](#).

3.119.4.18 astack

`AStack*` `Process::astack`

Definition at line 449 of file [grcc.h](#).

3.119.4.19 ngraphs

`BigInt` `Process::ngraphs`

Definition at line 452 of file [grcc.h](#).

3.119.4.20 nopi

`BigInt` `Process::nopi`

Definition at line 453 of file [grcc.h](#).

3.119.4.21 wgraphs

`BigInt` `Process::wgraphs`

Definition at line 454 of file [grcc.h](#).

3.119.4.22 wopi

`BigInt` `Process::wopi`

Definition at line 455 of file [grcc.h](#).

3.119.4.23 nMGraphs

`BigInt` `Process::nMGraphs`

Definition at line 458 of file [grcc.h](#).

3.119.4.24 nMOPI

`BigInt` `Process::nMOPI`

Definition at line 459 of file [grcc.h](#).

3.119.4.25 wMGraphs

`Fraction` `Process::wMGraphs`

Definition at line 460 of file [grcc.h](#).

3.119.4.26 wMOPI

`Fraction` `Process::wMOPI`

Definition at line 461 of file [grcc.h](#).

3.119.4.27 nAGraphs

`BigInt` `Process::nAGraphs`

Definition at line 463 of file [grcc.h](#).

3.119.4.28 nAOPI

`BigInt` `Process::nAOPI`

Definition at line 464 of file [grcc.h](#).

3.119.4.29 wAGraphs

`Fraction` `Process::wAGraphs`

Definition at line 465 of file [grcc.h](#).

3.119.4.30 wAOPI

`Fraction` `Process::wAOPI`

Definition at line 466 of file [grcc.h](#).

3.119.4.31 sec

`double` `Process::sec`

Definition at line 468 of file [grcc.h](#).

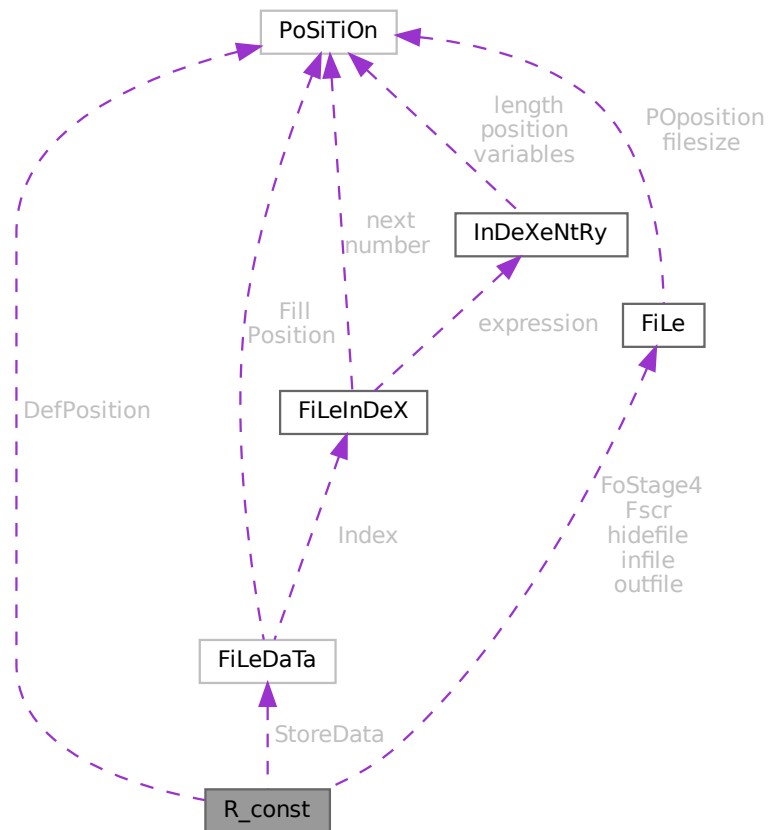
The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.120 R_const Struct Reference

```
#include <structs.h>
```

Collaboration diagram for R_const:



Public Member Functions

- **PADPOSITION** (8, 7, 7, 27, 0)

Data Fields

- [FILEDATA](#) `StoreData`
- [FILEHANDLE](#) `Fscr` [3]
- [FILEHANDLE](#) `FoStage4` [2]
- [POSITION](#) `DefPosition`
- [FILEHANDLE](#) * `infile`
- [FILEHANDLE](#) * `outfile`
- [FILEHANDLE](#) * `hidefile`
- [WORD](#) * `CompressBuffer`
- [WORD](#) * `ComprTop`

- WORD * [CompressPointer](#)
- COMPAREDUMMY [CompareRoutine](#)
- ULONG * [wranfia](#)
- SBYTE * [moebiustable](#)
- LONG [OldTime](#)
- LONG [InInBuf](#)
- LONG [InHiBuf](#)
- LONG [pWorkSize](#)
- LONG [IWorkSize](#)
- LONG [posWorkSize](#)
- ULONG [wranfseed](#)
- int [NoCompress](#)
- int [gzipCompress](#)
- int [Cnumlhs](#)
- int [outtohide](#)
- int [wranfcall](#)
- int [wranfnpair1](#)
- int [wranfnpair2](#)
- WORD [GetFile](#)
- WORD [KeptInHold](#)
- WORD [BracketOn](#)
- WORD [MaxBracket](#)
- WORD [CurDum](#)
- WORD [DeferFlag](#)
- WORD [TePos](#)
- WORD [sLevel](#)
- WORD [Stage4Name](#)
- WORD [GetOneFile](#)
- WORD [PolyFun](#)
- WORD [PolyFunInv](#)
- WORD [PolyFunType](#)
- WORD [PolyFunExp](#)
- WORD [PolyFunVar](#)
- WORD [PolyFunPow](#)
- WORD [Eside](#)
- WORD [MaxDum](#)
- WORD [level](#)
- WORD [expchanged](#)
- WORD [expflags](#)
- WORD [CurExpr](#)
- WORD [SortType](#)
- WORD [ShortSortCount](#)
- WORD [modeloptions](#)
- WORD [funoffset](#)
- WORD [moebiustablesizes](#)

3.120.1 Detailed Description

The [R_const](#) struct is part of the global data and resides either in the ALLGLOBALS struct A, or the ALLPRIVATE struct B (TFORM) under the name R We see it used with the macro AR as in AR.infile It has the variables that define the running environment and that should be transferred with a term in a multithreaded run.

Definition at line [2029](#) of file [structs.h](#).

3.120.2 Field Documentation

3.120.2.1 StoreData

`FILEDATA R_const::StoreData`

Definition at line 2030 of file [structs.h](#).

3.120.2.2 Fscr

`FILEHANDLE R_const::Fscr[3]`

Definition at line 2031 of file [structs.h](#).

3.120.2.3 FoStage4

`FILEHANDLE R_const::FoStage4[2]`

Definition at line 2032 of file [structs.h](#).

3.120.2.4 DefPosition

`POSITION R_const::DefPosition`

Definition at line 2033 of file [structs.h](#).

3.120.2.5 infile

`FILEHANDLE* R_const::infile`

Definition at line 2034 of file [structs.h](#).

3.120.2.6 outfile

`FILEHANDLE* R_const::outfile`

Definition at line 2035 of file [structs.h](#).

3.120.2.7 hidefile

`FILEHANDLE* R_const::hidefile`

Definition at line 2036 of file [structs.h](#).

3.120.2.8 CompressBuffer

WORD* R_const::CompressBuffer

Definition at line 2038 of file [structs.h](#).

3.120.2.9 ComprTop

WORD* R_const::ComprTop

Definition at line 2039 of file [structs.h](#).

3.120.2.10 CompressPointer

WORD* R_const::CompressPointer

Definition at line 2040 of file [structs.h](#).

3.120.2.11 CompareRoutine

COMPAREDUMMY R_const::CompareRoutine

Definition at line 2041 of file [structs.h](#).

3.120.2.12 wranfia

ULONG* R_const::wranfia

Definition at line 2042 of file [structs.h](#).

3.120.2.13 moebiustable

SBYTE* R_const::moebiustable

Definition at line 2043 of file [structs.h](#).

3.120.2.14 OldTime

LONG R_const::OldTime

Definition at line 2045 of file [structs.h](#).

3.120.2.15 InInBuf

LONG R_const::InInBuf

Definition at line 2046 of file [structs.h](#).

3.120.2.16 InHiBuf

LONG R_const::InHiBuf

Definition at line 2047 of file [structs.h](#).

3.120.2.17 pWorkSize

LONG R_const::pWorkSize

Definition at line 2048 of file [structs.h](#).

3.120.2.18 lWorkSize

LONG R_const::lWorkSize

Definition at line 2049 of file [structs.h](#).

3.120.2.19 posWorkSize

LONG R_const::posWorkSize

Definition at line 2050 of file [structs.h](#).

3.120.2.20 wranfseed

ULONG R_const::wranfseed

Definition at line 2051 of file [structs.h](#).

3.120.2.21 NoCompress

int R_const::NoCompress

Definition at line 2052 of file [structs.h](#).

3.120.2.22 gzipCompress

int R_const::gzipCompress

Definition at line 2053 of file [structs.h](#).

3.120.2.23 Cnumlhs

int R_const::Cnumlhs

Definition at line 2054 of file [structs.h](#).

3.120.2.24 outtohide

```
int R_const::outtohide
```

Definition at line 2055 of file [structs.h](#).

3.120.2.25 wranfcall

```
int R_const::wranfcall
```

Definition at line 2059 of file [structs.h](#).

3.120.2.26 wranfnpair1

```
int R_const::wranfnpair1
```

Definition at line 2060 of file [structs.h](#).

3.120.2.27 wranfnpair2

```
int R_const::wranfnpair2
```

Definition at line 2061 of file [structs.h](#).

3.120.2.28 GetFile

```
WORD R_const::GetFile
```

Definition at line 2067 of file [structs.h](#).

3.120.2.29 KeptInHold

```
WORD R_const::KeptInHold
```

Definition at line 2068 of file [structs.h](#).

3.120.2.30 BracketOn

```
WORD R_const::BracketOn
```

Definition at line 2069 of file [structs.h](#).

3.120.2.31 MaxBracket

```
WORD R_const::MaxBracket
```

Definition at line 2070 of file [structs.h](#).

3.120.2.32 CurDum

WORD R_const::CurDum

Definition at line 2071 of file [structs.h](#).

3.120.2.33 DeferFlag

WORD R_const::DeferFlag

Definition at line 2072 of file [structs.h](#).

3.120.2.34 TePos

WORD R_const::TePos

Definition at line 2073 of file [structs.h](#).

3.120.2.35 sLevel

WORD R_const::sLevel

Definition at line 2074 of file [structs.h](#).

3.120.2.36 Stage4Name

WORD R_const::Stage4Name

Definition at line 2075 of file [structs.h](#).

3.120.2.37 GetOneFile

WORD R_const::GetOneFile

Definition at line 2076 of file [structs.h](#).

3.120.2.38 PolyFun

WORD R_const::PolyFun

Definition at line 2077 of file [structs.h](#).

3.120.2.39 PolyFunInv

WORD R_const::PolyFunInv

Definition at line 2078 of file [structs.h](#).

3.120.2.40 PolyFunType

WORD R_const::PolyFunType

Definition at line 2079 of file [structs.h](#).

3.120.2.41 PolyFunExp

WORD R_const::PolyFunExp

Definition at line 2080 of file [structs.h](#).

3.120.2.42 PolyFunVar

WORD R_const::PolyFunVar

Definition at line 2081 of file [structs.h](#).

3.120.2.43 PolyFunPow

WORD R_const::PolyFunPow

Definition at line 2082 of file [structs.h](#).

3.120.2.44 Eside

WORD R_const::Eside

Definition at line 2083 of file [structs.h](#).

3.120.2.45 MaxDum

WORD R_const::MaxDum

Definition at line 2084 of file [structs.h](#).

3.120.2.46 level

WORD R_const::level

Definition at line 2085 of file [structs.h](#).

3.120.2.47 expchanged

WORD R_const::expchanged

Definition at line 2086 of file [structs.h](#).

3.120.2.48 expflags

WORD R_const::expflags

Definition at line 2087 of file [structs.h](#).

3.120.2.49 CurExpr

WORD R_const::CurExpr

Definition at line 2088 of file [structs.h](#).

3.120.2.50 SortType

WORD R_const::SortType

Definition at line 2089 of file [structs.h](#).

3.120.2.51 ShortSortCount

WORD R_const::ShortSortCount

Definition at line 2090 of file [structs.h](#).

3.120.2.52 modeloptions

WORD R_const::modeloptions

Definition at line 2091 of file [structs.h](#).

3.120.2.53 funoffset

WORD R_const::funoffset

Definition at line 2092 of file [structs.h](#).

3.120.2.54 moebiustablesizesize

WORD R_const::moebiustablesizesize

Definition at line 2093 of file [structs.h](#).

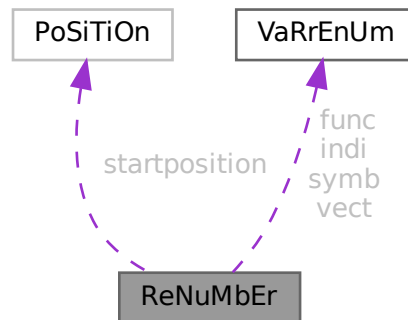
The documentation for this struct was generated from the following file:

- [structs.h](#)

3.121 ReNuMbEr Struct Reference

```
#include <structs.h>
```

Collaboration diagram for ReNuMbEr:



Public Member Functions

- **PADPOSITION** (4, 0, 0, 0, sizeof([VARRENUM](#)) *4)

Data Fields

- [POSITION](#) startposition
- [VARRENUM](#) symb
- [VARRENUM](#) indi
- [VARRENUM](#) vect
- [VARRENUM](#) func
- WORD * [symnum](#)
- WORD * [indnum](#)
- WORD * [vecnum](#)
- WORD * [funnum](#)

3.121.1 Detailed Description

Only symb.lo gets dynamically allocated. All other pointers points into this memory.

Definition at line [178](#) of file [structs.h](#).

3.121.2 Field Documentation

3.121.2.1 startposition

[POSITION](#) ReNuMbEr::startposition

Definition at line [179](#) of file [structs.h](#).

3.121.2.2 symb

`VARRENUM ReNuMbEr::symb`

Symbols

Definition at line 181 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#), [Generator\(\)](#), [GetFirstTerm\(\)](#), and [TermRenumber\(\)](#).

3.121.2.3 indi

`VARRENUM ReNuMbEr::indi`

Indices

Definition at line 182 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#), and [TermRenumber\(\)](#).

3.121.2.4 vect

`VARRENUM ReNuMbEr::vect`

Vectors

Definition at line 183 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#), and [TermRenumber\(\)](#).

3.121.2.5 func

`VARRENUM ReNuMbEr::func`

Functions

Definition at line 184 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#), and [TermRenumber\(\)](#).

3.121.2.6 symnum

`WORD* ReNuMbEr::symnum`

Renumbered symbols

Definition at line 186 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#), and [TermRenumber\(\)](#).

3.121.2.7 indnum

```
WORD* ReNuMbEr::indnum
```

Renumbered indices

Definition at line 187 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#), and [TermRenumber\(\)](#).

3.121.2.8 vecnum

```
WORD* ReNuMbEr::vecnum
```

Renumbered vectors

Definition at line 188 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#), and [TermRenumber\(\)](#).

3.121.2.9 funnum

```
WORD* ReNuMbEr::funnum
```

Renumbered functions

Definition at line 189 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#), and [TermRenumber\(\)](#).

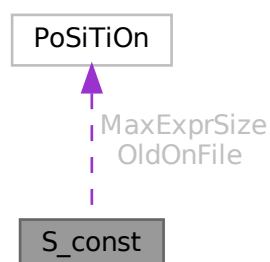
The documentation for this struct was generated from the following file:

- [structs.h](#)

3.122 S_const Struct Reference

```
#include <structs.h>
```

Collaboration diagram for S_const:



Public Member Functions

- **PADPOSITION** (4, 0, 4, 2, 0)

Data Fields

- [POSITION](#) MaxExprSize
- [POSITION](#) * OldOnFile
- [WORD](#) * OldNumFactors
- [WORD](#) * Oldvflags
- [WORD](#) * Olduflags
- [int](#) NumOldOnFile
- [int](#) NumOldNumFactors
- [int](#) MultiThreaded
- [int](#) Balancing
- [WORD](#) ExecMode
- [WORD](#) CollectOverFlag

3.122.1 Detailed Description

The [S_const](#) struct is part of the global data and resides in the ALLGLOBALS struct A under the name S We see it used with the macro AS as in AS.ExecMode It has some variables used by the master in multithreaded runs

Definition at line [1977](#) of file [structs.h](#).

3.122.2 Field Documentation

3.122.2.1 MaxExprSize

[POSITION](#) S_const::MaxExprSize

Definition at line [1978](#) of file [structs.h](#).

3.122.2.2 OldOnFile

[POSITION](#)* S_const::OldOnFile

Definition at line [1984](#) of file [structs.h](#).

3.122.2.3 OldNumFactors

[WORD](#)* S_const::OldNumFactors

Definition at line [1985](#) of file [structs.h](#).

3.122.2.4 Oldvflags

```
WORD* S_const::Oldvflags
```

Definition at line 1986 of file [structs.h](#).

3.122.2.5 Olduflags

```
WORD* S_const::Olduflags
```

Definition at line 1987 of file [structs.h](#).

3.122.2.6 NumOldOnFile

```
int S_const::NumOldOnFile
```

Definition at line 1991 of file [structs.h](#).

3.122.2.7 NumOldNumFactors

```
int S_const::NumOldNumFactors
```

Definition at line 1992 of file [structs.h](#).

3.122.2.8 MultiThreaded

```
int S_const::MultiThreaded
```

Definition at line 1993 of file [structs.h](#).

3.122.2.9 Balancing

```
int S_const::Balancing
```

Definition at line 2000 of file [structs.h](#).

3.122.2.10 ExecMode

```
WORD S_const::ExecMode
```

Definition at line 2001 of file [structs.h](#).

3.122.2.11 CollectOverFlag

WORD S_const::CollectOverFlag

Definition at line 2003 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.123 SeTs Struct Reference

Data Fields

- LONG [name](#)
- WORD [type](#)
- WORD [first](#)
- WORD [last](#)
- WORD [node](#)
- WORD [namesize](#)
- WORD [dimension](#)
- WORD [flags](#)

3.123.1 Detailed Description

Definition at line 499 of file [structs.h](#).

3.123.2 Field Documentation

3.123.2.1 name

LONG SeTs::name

Definition at line 500 of file [structs.h](#).

3.123.2.2 type

WORD SeTs::type

Definition at line 501 of file [structs.h](#).

3.123.2.3 first

WORD SeTs::first

Definition at line 502 of file [structs.h](#).

3.123.2.4 last

WORD SeTs::last

Definition at line 503 of file [structs.h](#).

3.123.2.5 node

WORD SeTs::node

Definition at line 504 of file [structs.h](#).

3.123.2.6 namesize

WORD SeTs::namesize

Definition at line 505 of file [structs.h](#).

3.123.2.7 dimension

WORD SeTs::dimension

Definition at line 506 of file [structs.h](#).

3.123.2.8 flags

WORD SeTs::flags

Definition at line 507 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.124 SETUPPARAMETERS Struct Reference

```
#include <structs.h>
```

Data Fields

- UBYTE * [parameter](#)
- int [type](#)
- int [flags](#)
- LONG [value](#)

3.124.1 Detailed Description

Each setup parameter has one element of the struct [SETUPPARAMETERS](#) assigned to it. By binary search in the array of them we can then locate the proper element by name. We have to assume that two ints make a long and either one or two longs make a pointer. The long before the ints and padding gives a problem in the initialization.

Definition at line [1038](#) of file [structs.h](#).

3.124.2 Field Documentation

3.124.2.1 parameter

```
UBYTE* SETUPPARAMETERS::parameter
```

Definition at line [1039](#) of file [structs.h](#).

3.124.2.2 type

```
int SETUPPARAMETERS::type
```

Definition at line [1040](#) of file [structs.h](#).

3.124.2.3 flags

```
int SETUPPARAMETERS::flags
```

Definition at line [1041](#) of file [structs.h](#).

3.124.2.4 value

```
LONG SETUPPARAMETERS::value
```

Definition at line [1042](#) of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.125 SGroup Class Reference

Public Member Functions

- void [print](#) (void)
- void [newGroup](#) (int nelm, int nclss, int *clss)
- void [clearGroup](#) (void)
- void [delGroup](#) (void)
- int * [genNext](#) (void)
- void [addGroup](#) (int *p)
- BigInt [nElem](#) (void)
- int * [nextElem](#) (void)

Data Fields

- BigInt [size](#)
- BigInt [nelem](#)
- int ** [elem](#)
- int [nnodes](#)
- int [neclass](#)
- int [eclass](#) [GRCC_MAXNODES]
- int [cgen](#)
- int [csav](#)
- int [permg](#) [GRCC_MAXNODES]
- int [perms](#) [GRCC_MAXNODES]
- int [pgr](#) [GRCC_MAXNODES]
- int [pgq](#) [GRCC_MAXNODES]
- int [psr](#) [GRCC_MAXNODES]
- int [psq](#) [GRCC_MAXNODES]

3.125.1 Detailed Description

Definition at line [700](#) of file [grcc.h](#).

3.125.2 Constructor & Destructor Documentation

3.125.2.1 SGroup()

```
SGroup::SGroup (  
    void )
```

Definition at line [5376](#) of file [grcc.cc](#).

3.125.2.2 ~SGroup()

```
SGroup::~~SGroup (  
    void )
```

Definition at line [5385](#) of file [grcc.cc](#).

3.125.3 Member Function Documentation

3.125.3.1 print()

```
void SGroup::print (  
    void )
```

Definition at line [5402](#) of file [grcc.cc](#).

3.125.3.2 newGroup()

```
void SGroup::newGroup (
    int nelm,
    int nclss,
    int * clss )
```

Definition at line 5417 of file [grcc.cc](#).

3.125.3.3 clearGroup()

```
void SGroup::clearGroup (
    void )
```

Definition at line 5442 of file [grcc.cc](#).

3.125.3.4 delGroup()

```
void SGroup::delGroup (
    void )
```

Definition at line 5450 of file [grcc.cc](#).

3.125.3.5 genNext()

```
int * SGroup::genNext (
    void )
```

Definition at line 5466 of file [grcc.cc](#).

3.125.3.6 addGroup()

```
void SGroup::addGroup (
    int * p )
```

Definition at line 5476 of file [grcc.cc](#).

3.125.3.7 nElem()

```
BigInt SGroup::nElem (
    void )
```

Definition at line 5493 of file [grcc.cc](#).

3.125.3.8 nextElem()

```
int * SGroup::nextElem (
    void )
```

Definition at line 5499 of file [grcc.cc](#).

3.125.4 Field Documentation

3.125.4.1 size

```
BigInt SGroup::size
```

Definition at line 702 of file [grcc.h](#).

3.125.4.2 nelem

```
BigInt SGroup::nelem
```

Definition at line 703 of file [grcc.h](#).

3.125.4.3 elem

```
int** SGroup::elem
```

Definition at line 704 of file [grcc.h](#).

3.125.4.4 nnodes

```
int SGroup::nnodes
```

Definition at line 705 of file [grcc.h](#).

3.125.4.5 neclass

```
int SGroup::neclass
```

Definition at line 706 of file [grcc.h](#).

3.125.4.6 eclass

```
int SGroup::eclass[GRCC_MAXNODES]
```

Definition at line 707 of file [grcc.h](#).

3.125.4.7 cgen

```
int SGroup::cgen
```

Definition at line 708 of file [grcc.h](#).

3.125.4.8 csav

```
int SGroup::csav
```

Definition at line 709 of file [grcc.h](#).

3.125.4.9 permg

```
int SGroup::permg[GRCC_MAXNODES]
```

Definition at line 710 of file [grcc.h](#).

3.125.4.10 perms

```
int SGroup::perms[GRCC_MAXNODES]
```

Definition at line 711 of file [grcc.h](#).

3.125.4.11 pgr

```
int SGroup::pgr[GRCC_MAXNODES]
```

Definition at line 713 of file [grcc.h](#).

3.125.4.12 pgq

```
int SGroup::pgq[GRCC_MAXNODES]
```

Definition at line 714 of file [grcc.h](#).

3.125.4.13 psr

```
int SGroup::psr[GRCC_MAXNODES]
```

Definition at line 715 of file [grcc.h](#).

3.125.4.14 psq

```
int SGroup::psq[GRCC_MAXNODES]
```

Definition at line 716 of file [grcc.h](#).

The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.126 SHvariables Struct Reference

Data Fields

- WORD * [outterm](#)
- WORD * [outfun](#)
- WORD * [incoef](#)
- WORD * [stop1](#)
- WORD * [stop2](#)
- WORD * [ststop1](#)
- WORD * [ststop2](#)
- FINISHUFFLE [finishuf](#)
- DO_UFFLE [do_uffle](#)
- LONG [combilast](#)
- WORD [nincoef](#)
- WORD [level](#)
- WORD [thefunction](#)
- WORD [option](#)

3.126.1 Detailed Description

Definition at line [1276](#) of file [structs.h](#).

3.126.2 Field Documentation

3.126.2.1 outterm

```
WORD* SHvariables::outterm
```

Definition at line [1277](#) of file [structs.h](#).

3.126.2.2 outfun

```
WORD* SHvariables::outfun
```

Definition at line [1278](#) of file [structs.h](#).

3.126.2.3 incoef

```
WORD* SHvariables::incoef
```

Definition at line [1279](#) of file [structs.h](#).

3.126.2.4 stop1

```
WORD* SHvariables::stop1
```

Definition at line [1280](#) of file [structs.h](#).

3.126.2.5 stop2

WORD* SHvariables::stop2

Definition at line 1281 of file [structs.h](#).

3.126.2.6 ststop1

WORD* SHvariables::ststop1

Definition at line 1282 of file [structs.h](#).

3.126.2.7 ststop2

WORD* SHvariables::ststop2

Definition at line 1283 of file [structs.h](#).

3.126.2.8 finishuf

FINISHUFFLE SHvariables::finishuf

Definition at line 1284 of file [structs.h](#).

3.126.2.9 do_uffle

DO_UFFLE SHvariables::do_uffle

Definition at line 1285 of file [structs.h](#).

3.126.2.10 combilast

LONG SHvariables::combilast

Definition at line 1286 of file [structs.h](#).

3.126.2.11 nincoef

WORD SHvariables::nincoef

Definition at line 1287 of file [structs.h](#).

3.126.2.12 level

WORD SHvariables::level

Definition at line 1288 of file [structs.h](#).

3.126.2.13 thefunction

WORD SHvariables::thefunction

Definition at line 1289 of file [structs.h](#).

3.126.2.14 option

WORD SHvariables::option

Definition at line 1290 of file [structs.h](#).

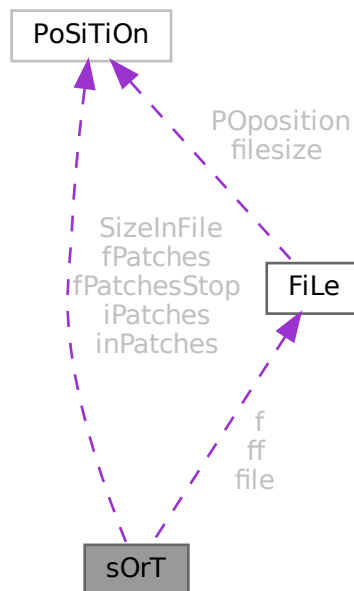
The documentation for this struct was generated from the following file:

- [structs.h](#)

3.127 sOrT Struct Reference

```
#include <structs.h>
```

Collaboration diagram for sOrT:



Public Member Functions

- **PADPOSITION** (24, 12, 12, 3, 0)

Data Fields

- [FILEHANDLE](#) [file](#)
- [POSITION](#) [SizeInFile](#) [3]
- WORD * [lBuffer](#)
- WORD * [lTop](#)
- WORD * [lFill](#)
- WORD * [used](#)
- WORD * [sBuffer](#)
- WORD * [sTop](#)
- WORD * [sTop2](#)
- WORD * [sHalf](#)
- WORD * [sFill](#)
- WORD ** [sPointer](#)
- WORD ** [PoinFill](#)
- WORD * [cBuffer](#)
- WORD ** [Patches](#)
- WORD ** [pStop](#)
- WORD ** [poina](#)
- WORD ** [poin2a](#)
- WORD * [ktoi](#)
- WORD * [tree](#)
- [POSITION](#) * [fPatches](#)
- [POSITION](#) * [inPatches](#)
- [POSITION](#) * [fPatchesStop](#)
- [POSITION](#) * [iPatches](#)
- [FILEHANDLE](#) * [f](#)
- [FILEHANDLE](#) ** [ff](#)
- LONG [sTerms](#)
- LONG [LargeSize](#)
- LONG [SmallSize](#)
- LONG [SmallEsize](#)
- LONG [TermsInSmall](#)
- LONG [Terms2InSmall](#)
- LONG [GenTerms](#)
- LONG [TermsLeft](#)
- LONG [GenSpace](#)
- LONG [SpaceLeft](#)
- LONG [putinsize](#)
- LONG [ninterms](#)
- int [MaxPatches](#)
- int [MaxFpatches](#)
- int [type](#)
- int [lPatch](#)
- int [fPatchN1](#)
- int [PolyWise](#)
- int [PolyFlag](#)
- int [cBufferSize](#)
- int [maxtermsize](#)
- int [newmaxtermsize](#)
- int [outputmode](#)
- int [stagelevel](#)
- WORD [fPatchN](#)
- WORD [inNum](#)
- WORD [stage4](#)

3.127.1 Detailed Description

The struct SORTING is used to control a sort operation. It includes a small and a large buffer and arrays for keeping track of various stages of the (merge) sorts. Each sort level has its own struct and different levels can have different sizes for its arrays. Also different threads have their own set of SORTING structs.

Definition at line 1140 of file [structs.h](#).

3.127.2 Field Documentation

3.127.2.1 file

```
FILEHANDLE sOrT::file
```

Definition at line 1141 of file [structs.h](#).

3.127.2.2 SizeInFile

```
POSITION sOrT::SizeInFile[3]
```

Definition at line 1142 of file [structs.h](#).

3.127.2.3 lBuffer

```
WORD* sOrT::lBuffer
```

Definition at line 1143 of file [structs.h](#).

3.127.2.4 lTop

```
WORD* sOrT::lTop
```

Definition at line 1144 of file [structs.h](#).

3.127.2.5 lFill

```
WORD* sOrT::lFill
```

Definition at line 1145 of file [structs.h](#).

3.127.2.6 used

```
WORD* sOrT::used
```

Definition at line 1146 of file [structs.h](#).

3.127.2.7 sBuffer

WORD* sOrT::sBuffer

Definition at line 1147 of file [structs.h](#).

3.127.2.8 sTop

WORD* sOrT::sTop

Definition at line 1148 of file [structs.h](#).

3.127.2.9 sTop2

WORD* sOrT::sTop2

Definition at line 1149 of file [structs.h](#).

3.127.2.10 sHalf

WORD* sOrT::sHalf

Definition at line 1150 of file [structs.h](#).

3.127.2.11 sFill

WORD* sOrT::sFill

Definition at line 1151 of file [structs.h](#).

3.127.2.12 sPointer

WORD** sOrT::sPointer

Definition at line 1152 of file [structs.h](#).

3.127.2.13 PoinFill

WORD** sOrT::PoinFill

Definition at line 1153 of file [structs.h](#).

3.127.2.14 cBuffer

WORD* sOrT::cBuffer

Definition at line 1154 of file [structs.h](#).

3.127.2.15 Patches

```
WORD** sOrT::Patches
```

Definition at line 1155 of file [structs.h](#).

3.127.2.16 pStop

```
WORD** sOrT::pStop
```

Definition at line 1156 of file [structs.h](#).

3.127.2.17 poina

```
WORD** sOrT::poina
```

Definition at line 1157 of file [structs.h](#).

3.127.2.18 poin2a

```
WORD** sOrT::poin2a
```

Definition at line 1158 of file [structs.h](#).

3.127.2.19 ktoi

```
WORD* sOrT::ktoi
```

Definition at line 1159 of file [structs.h](#).

3.127.2.20 tree

```
WORD* sOrT::tree
```

Definition at line 1160 of file [structs.h](#).

3.127.2.21 fPatches

```
POSITION* sOrT::fPatches
```

Definition at line 1166 of file [structs.h](#).

3.127.2.22 inPatches

```
POSITION* sOrT::inPatches
```

Definition at line 1167 of file [structs.h](#).

3.127.2.23 fPatchesStop

`POSITION* sOrT::fPatchesStop`

Definition at line 1168 of file [structs.h](#).

3.127.2.24 iPatches

`POSITION* sOrT::iPatches`

Definition at line 1169 of file [structs.h](#).

3.127.2.25 f

`FILEHANDLE* sOrT::f`

Definition at line 1170 of file [structs.h](#).

3.127.2.26 ff

`FILEHANDLE** sOrT::ff`

Definition at line 1171 of file [structs.h](#).

3.127.2.27 sTerms

`LONG sOrT::sTerms`

Definition at line 1172 of file [structs.h](#).

3.127.2.28 LargeSize

`LONG sOrT::LargeSize`

Definition at line 1173 of file [structs.h](#).

3.127.2.29 SmallSize

`LONG sOrT::SmallSize`

Definition at line 1174 of file [structs.h](#).

3.127.2.30 SmallEsize

`LONG sOrT::SmallEsize`

Definition at line 1175 of file [structs.h](#).

3.127.2.31 TermsInSmall

LONG sOrT::TermsInSmall

Definition at line 1176 of file [structs.h](#).

3.127.2.32 Terms2InSmall

LONG sOrT::Terms2InSmall

Definition at line 1177 of file [structs.h](#).

3.127.2.33 GenTerms

LONG sOrT::GenTerms

Definition at line 1178 of file [structs.h](#).

3.127.2.34 TermsLeft

LONG sOrT::TermsLeft

Definition at line 1179 of file [structs.h](#).

3.127.2.35 GenSpace

LONG sOrT::GenSpace

Definition at line 1180 of file [structs.h](#).

3.127.2.36 SpaceLeft

LONG sOrT::SpaceLeft

Definition at line 1181 of file [structs.h](#).

3.127.2.37 putinsize

LONG sOrT::putinsize

Definition at line 1182 of file [structs.h](#).

3.127.2.38 ninterms

LONG sOrT::ninterms

Definition at line 1183 of file [structs.h](#).

3.127.2.39 MaxPatches

```
int sOrT::MaxPatches
```

Definition at line 1184 of file [structs.h](#).

3.127.2.40 MaxFpatches

```
int sOrT::MaxFpatches
```

Definition at line 1185 of file [structs.h](#).

3.127.2.41 type

```
int sOrT::type
```

Definition at line 1186 of file [structs.h](#).

3.127.2.42 lPatch

```
int sOrT::lPatch
```

Definition at line 1187 of file [structs.h](#).

3.127.2.43 fPatchN1

```
int sOrT::fPatchN1
```

Definition at line 1188 of file [structs.h](#).

3.127.2.44 PolyWise

```
int sOrT::PolyWise
```

Definition at line 1189 of file [structs.h](#).

3.127.2.45 PolyFlag

```
int sOrT::PolyFlag
```

Definition at line 1190 of file [structs.h](#).

3.127.2.46 cBufferSize

```
int sOrT::cBufferSize
```

Definition at line 1191 of file [structs.h](#).

3.127.2.47 maxtermsize

```
int sOrT::maxtermsize
```

Definition at line 1192 of file [structs.h](#).

3.127.2.48 newmaxtermsize

```
int sOrT::newmaxtermsize
```

Definition at line 1193 of file [structs.h](#).

3.127.2.49 outputmode

```
int sOrT::outputmode
```

Definition at line 1194 of file [structs.h](#).

3.127.2.50 stagelevel

```
int sOrT::stagelevel
```

Definition at line 1195 of file [structs.h](#).

3.127.2.51 fPatchN

```
WORD sOrT::fPatchN
```

Definition at line 1196 of file [structs.h](#).

3.127.2.52 inNum

```
WORD sOrT::inNum
```

Definition at line 1197 of file [structs.h](#).

3.127.2.53 stage4

```
WORD sOrT::stage4
```

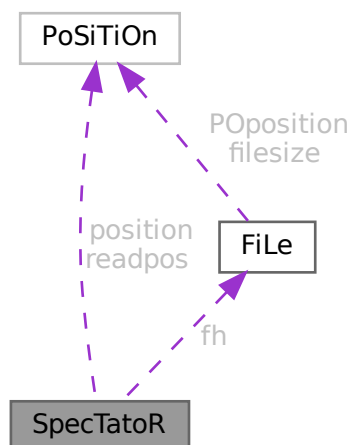
Definition at line 1198 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.128 SpecTatoR Struct Reference

Collaboration diagram for SpecTatoR:



Public Member Functions

- **PADPOSITION** (2, 0, 0, 2, 0)

Data Fields

- [POSITION](#) position
- [POSITION](#) readpos
- [FILEHANDLE](#) * fh
- char * name
- [WORD](#) exprnumber
- [WORD](#) flags

3.128.1 Detailed Description

Definition at line 765 of file [structs.h](#).

3.128.2 Field Documentation

3.128.2.1 position

[POSITION](#) SpecTatoR::position

Definition at line 766 of file [structs.h](#).

Public Member Functions

- [SProcess](#) ([Model](#) *mdl, [Process](#) *prc, [Options](#) *opts, int sid, int *clst, int ncls, [NCInput](#) *cls)
- [SProcess](#) ([Model](#) *mdl, [Process](#) *prc, [Options](#) *opts, int sid, int *clst, int ncls, int *cdeg, int *ctyp, int *ptcl, int *cpl, int *cnum, int *cmind, int *cmaxd)
- void [prSProcess](#) (void)
- [BigInt](#) [generate](#) (void)
- void [assign](#) ([MGraph](#) *mgr)
- int [toMNodeClass](#) (int *ctyp, int *cldeg, int *clnum, int *cmind, int *cmaxd)
- [PNodeClass](#) * [match](#) ([MGraph](#) *mgr)
- void [endMGraph](#) ([MGraph](#) *mgr)
- void [endAGraph](#) ([EGraph](#) *egr)
- void [resultMGraph](#) ([BigInt](#) nmgraphs, [Fraction](#) mwsum, [BigInt](#) nmopi, [Fraction](#) mwopi)
- void [resultAGraph](#) ([BigInt](#) nagraphs, [Fraction](#) awsum, [BigInt](#) naopi, [Fraction](#) awopi)

Data Fields

- [Model](#) * [model](#)
- [Process](#) * [proc](#)
- [Options](#) * [opt](#)
- [PNodeClass](#) * [pnclass](#)
- [AStack](#) * [astack](#)
- [MGraph](#) * [mgraph](#)
- [EGraph](#) * [egraph](#)
- [Assign](#) * [agraph](#)
- int * [cl2nd](#)
- int * [nd2cl](#)
- int [nclass](#)
- int [ninitl](#)
- int [nfinal](#)
- int [nvert](#)
- int [clist](#) [GRCC_MAXNCPLG]
- int [id](#)
- int [loop](#)
- int [nNodes](#)
- int [nEdges](#)
- int [nExtern](#)
- int [ncouple](#)
- int [tCouple](#)
- int [DUMMY_PADDING](#)
- [BigInt](#) [mgrcount](#)
- [BigInt](#) [agrcount](#)
- [BigInt](#) [extperm](#)
- [BigInt](#) [nMGraphs](#)
- [BigInt](#) [nMOPI](#)
- [Fraction](#) [wMGraphs](#)
- [Fraction](#) [wMOPI](#)
- [BigInt](#) [nAGraphs](#)
- [BigInt](#) [nAOPI](#)
- [Fraction](#) [wAGraphs](#)
- [Fraction](#) [wAOPI](#)

3.129.1 Detailed Description

Definition at line 350 of file [grcc.h](#).

3.129.2 Constructor & Destructor Documentation

3.129.2.1 SProcess() [1/2]

```
SProcess::SProcess (
    Model * mdl,
    Process * prc,
    Options * opts,
    int sid,
    int * clst,
    int ncls,
    NCInput * cls )
```

Definition at line 2695 of file [grcc.cc](#).

3.129.2.2 SProcess() [2/2]

```
SProcess::SProcess (
    Model * mdl,
    Process * prc,
    Options * opts,
    int sid,
    int * clst,
    int ncls,
    int * cdeg,
    int * ctyp,
    int * ptcl,
    int * cpl,
    int * cnum,
    int * cmind,
    int * cmaxd )
```

Definition at line 2526 of file [grcc.cc](#).

3.129.2.3 ~SProcess()

```
SProcess::~~SProcess (
    void )
```

Definition at line 2857 of file [grcc.cc](#).

3.129.3 Member Function Documentation

3.129.3.1 prSProcess()

```
void SProcess::prSProcess (
    void )
```

Definition at line 2865 of file [grcc.cc](#).

3.129.3.2 generate()

```
BigInt SProcess::generate (
    void )
```

Definition at line [2877](#) of file [grcc.cc](#).

3.129.3.3 assign()

```
void SProcess::assign (
    MGraph * mgr )
```

Definition at line [2921](#) of file [grcc.cc](#).

3.129.3.4 toMNodeClass()

```
int SProcess::toMNodeClass (
    int * ctyp,
    int * cldeg,
    int * clnum,
    int * cmind,
    int * cmaxd )
```

Definition at line [2902](#) of file [grcc.cc](#).

3.129.3.5 match()

```
PNodeClass * SProcess::match (
    MGraph * mgr )
```

Definition at line [2938](#) of file [grcc.cc](#).

3.129.3.6 endMGraph()

```
void SProcess::endMGraph (
    MGraph * mgr )
```

Definition at line [3017](#) of file [grcc.cc](#).

3.129.3.7 endAGraph()

```
void SProcess::endAGraph (
    EGraph * egr )
```

Definition at line [3027](#) of file [grcc.cc](#).

3.129.3.8 resultMGraph()

```
void SProcess::resultMGraph (
    BigInt nmgraphs,
    Fraction mwsum,
    BigInt nmopi,
    Fraction mwopi )
```

Definition at line 2999 of file [grcc.cc](#).

3.129.3.9 resultAGraph()

```
void SProcess::resultAGraph (
    BigInt nagraphs,
    Fraction awsum,
    BigInt naopi,
    Fraction awopi )
```

Definition at line 3008 of file [grcc.cc](#).

3.129.4 Field Documentation

3.129.4.1 model

```
Model* SProcess::model
```

Definition at line 357 of file [grcc.h](#).

3.129.4.2 proc

```
Process* SProcess::proc
```

Definition at line 358 of file [grcc.h](#).

3.129.4.3 opt

```
Options* SProcess::opt
```

Definition at line 359 of file [grcc.h](#).

3.129.4.4 pnclass

```
PNodeClass* SProcess::pnclass
```

Definition at line 360 of file [grcc.h](#).

3.129.4.5 astack

`AStack* SProcess::astack`

Definition at line 361 of file [grcc.h](#).

3.129.4.6 mgraph

`MGraph* SProcess::mgraph`

Definition at line 363 of file [grcc.h](#).

3.129.4.7 egraph

`EGraph* SProcess::egraph`

Definition at line 364 of file [grcc.h](#).

3.129.4.8 agraph

`Assign* SProcess::agraph`

Definition at line 365 of file [grcc.h](#).

3.129.4.9 cl2nd

`int* SProcess::cl2nd`

Definition at line 367 of file [grcc.h](#).

3.129.4.10 nd2cl

`int* SProcess::nd2cl`

Definition at line 369 of file [grcc.h](#).

3.129.4.11 nclass

`int SProcess::nclass`

Definition at line 370 of file [grcc.h](#).

3.129.4.12 ninitl

`int SProcess::ninitl`

Definition at line 372 of file [grcc.h](#).

3.129.4.13 nfinal

```
int SProcess::nfinal
```

Definition at line 373 of file [grcc.h](#).

3.129.4.14 nvert

```
int SProcess::nvert
```

Definition at line 374 of file [grcc.h](#).

3.129.4.15 clist

```
int SProcess::clist[GRCC_MAXNCPLG]
```

Definition at line 376 of file [grcc.h](#).

3.129.4.16 id

```
int SProcess::id
```

Definition at line 377 of file [grcc.h](#).

3.129.4.17 loop

```
int SProcess::loop
```

Definition at line 378 of file [grcc.h](#).

3.129.4.18 nNodes

```
int SProcess::nNodes
```

Definition at line 379 of file [grcc.h](#).

3.129.4.19 nEdges

```
int SProcess::nEdges
```

Definition at line 380 of file [grcc.h](#).

3.129.4.20 nExtern

```
int SProcess::nExtern
```

Definition at line 381 of file [grcc.h](#).

3.129.4.21 ncouple

```
int SProcess::ncouple
```

Definition at line 382 of file [grcc.h](#).

3.129.4.22 tCouple

```
int SProcess::tCouple
```

Definition at line 383 of file [grcc.h](#).

3.129.4.23 DUMMYPADDING

```
int SProcess::DUMMYPADDING
```

Definition at line 385 of file [grcc.h](#).

3.129.4.24 mgrcount

```
BigInt SProcess::mgrcount
```

Definition at line 387 of file [grcc.h](#).

3.129.4.25 agrcount

```
BigInt SProcess::agrcount
```

Definition at line 388 of file [grcc.h](#).

3.129.4.26 extperm

```
BigInt SProcess::extperm
```

Definition at line 389 of file [grcc.h](#).

3.129.4.27 nMGraphs

```
BigInt SProcess::nMGraphs
```

Definition at line 392 of file [grcc.h](#).

3.129.4.28 nMOPI

```
BigInt SProcess::nMOPI
```

Definition at line 393 of file [grcc.h](#).

3.129.4.29 wMGraphs

`Fraction SProcess::wMGraphs`

Definition at line 394 of file [grcc.h](#).

3.129.4.30 wMOPI

`Fraction SProcess::wMOPI`

Definition at line 395 of file [grcc.h](#).

3.129.4.31 nAGraphs

`BigInt SProcess::nAGraphs`

Definition at line 397 of file [grcc.h](#).

3.129.4.32 nAOPI

`BigInt SProcess::nAOPI`

Definition at line 398 of file [grcc.h](#).

3.129.4.33 wAGraphs

`Fraction SProcess::wAGraphs`

Definition at line 399 of file [grcc.h](#).

3.129.4.34 wAOPI

`Fraction SProcess::wAOPI`

Definition at line 400 of file [grcc.h](#).

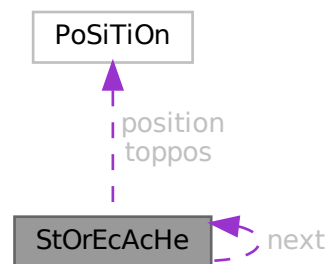
The documentation for this class was generated from the following files:

- [grcc.h](#)
- [grcc.cc](#)

3.130 StOrEcAcHe Struct Reference

```
#include <structs.h>
```

Collaboration diagram for StOrEcAcHe:



Public Member Functions

- **PADPOSITION** (1, 0, 0, 2, 0)

Data Fields

- **POSITION** position
- **POSITION** toppos
- struct **StOrEcAcHe** * next
- **WORD** buffer [2]

3.130.1 Detailed Description

The struct of type STORECACHE is used by a caching system for reading terms from stored expressions. Each thread should have its own system of caches.

Definition at line 1065 of file [structs.h](#).

3.130.2 Field Documentation

3.130.2.1 position

POSITION StOrEcAcHe::position

Definition at line 1066 of file [structs.h](#).

3.130.2.2 toppos

`POSITION StOrEcAcHe::toppos`

Definition at line 1067 of file [structs.h](#).

3.130.2.3 next

`struct StOrEcAcHe* StOrEcAcHe::next`

Definition at line 1068 of file [structs.h](#).

3.130.2.4 buffer

`WORD StOrEcAcHe::buffer[2]`

Definition at line 1069 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.131 STOREHEADER Struct Reference

```
#include <structs.h>
```

Data Fields

- UBYTE [headermark](#) [8]
- UBYTE [lenWORD](#)
- UBYTE [lenLONG](#)
- UBYTE [lenPOS](#)
- UBYTE [lenPOINTER](#)
- UBYTE [endianness](#) [16]
- UBYTE [sSym](#)
- UBYTE [sInd](#)
- UBYTE [sVec](#)
- UBYTE [sFun](#)
- UBYTE [maxpower](#) [16]
- UBYTE [wildoffset](#) [16]
- UBYTE [revision](#)
- UBYTE [reserved](#) [512-8-4-16-4-16-16-1]

3.131.1 Detailed Description

Defines the structure of the file header for store-files and save-files.

The first 8 bytes serve as a unique mark to identity save-files that contain such a header. Older versions of FORM don't have this header and will write the POSITION of the next file index (struct [FiLeInDeX](#)) here, which is always different from this pattern.

It is always 512 bytes long.

Definition at line 75 of file [structs.h](#).

3.131.2 Field Documentation

3.131.2.1 headermark

```
UBYTE STOREHEADER::headermark[8]
```

Pattern for header identification. Old versions of FORM have a maximum sizeof(POSITION) of 8

Definition at line 76 of file [structs.h](#).

3.131.2.2 lenWORD

```
UBYTE STOREHEADER::lenWORD
```

Number of bytes for WORD

Definition at line 78 of file [structs.h](#).

Referenced by [WriteStoreHeader\(\)](#).

3.131.2.3 lenLONG

```
UBYTE STOREHEADER::lenLONG
```

Number of bytes for LONG

Definition at line 79 of file [structs.h](#).

Referenced by [WriteStoreHeader\(\)](#).

3.131.2.4 lenPOS

```
UBYTE STOREHEADER::lenPOS
```

Number of bytes for POSITION

Definition at line 80 of file [structs.h](#).

Referenced by [WriteStoreHeader\(\)](#).

3.131.2.5 lenPOINTER

```
UBYTE STOREHEADER::lenPOINTER
```

Number of bytes for void *

Definition at line 81 of file [structs.h](#).

Referenced by [WriteStoreHeader\(\)](#).

3.131.2.6 endianness

```
UBYTE STOREHEADER::endianness[16]
```

Used to determine endianness, sizeof(int) should be <= 16

Definition at line 82 of file [structs.h](#).

Referenced by [WriteStoreHeader\(\)](#).

3.131.2.7 sSym

```
UBYTE STOREHEADER::sSym
```

sizeof(struct SyMbOl)

Definition at line 83 of file [structs.h](#).

Referenced by [WriteStoreHeader\(\)](#).

3.131.2.8 sInd

```
UBYTE STOREHEADER::sInd
```

sizeof(struct InDeX)

Definition at line 84 of file [structs.h](#).

Referenced by [WriteStoreHeader\(\)](#).

3.131.2.9 sVec

```
UBYTE STOREHEADER::sVec
```

sizeof(struct VeCtOr)

Definition at line 85 of file [structs.h](#).

Referenced by [WriteStoreHeader\(\)](#).

3.131.2.10 sFun

```
UBYTE STOREHEADER::sFun
```

sizeof(struct FuNcTiOn)

Definition at line 86 of file [structs.h](#).

Referenced by [WriteStoreHeader\(\)](#).

3.131.2.11 maxpower

```
UBYTE STOREHEADER::maxpower[16]
```

Maximum power, see MAXPOWER

Definition at line 87 of file [structs.h](#).

Referenced by [WriteStoreHeader\(\)](#).

3.131.2.12 wildoffset

```
UBYTE STOREHEADER::wildoffset[16]
```

WILDOFFSET macro

Definition at line 88 of file [structs.h](#).

Referenced by [WriteStoreHeader\(\)](#).

3.131.2.13 revision

```
UBYTE STOREHEADER::revision
```

Revision number of save-file system

Definition at line 89 of file [structs.h](#).

3.131.2.14 reserved

```
UBYTE STOREHEADER::reserved[512-8-4-16-4-16-16-1]
```

Padding to 512 bytes

Definition at line 90 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.132 Stream Struct Reference

```
#include <structs.h>
```

Public Member Functions

- **PADPOSITION** (6, 3, 9, 0, 4)

Data Fields

- off_t [fileposition](#)
- off_t [linenumber](#)
- off_t [prevline](#)
- UBYTE * [buffer](#)
- UBYTE * [pointer](#)
- UBYTE * [top](#)
- UBYTE * [FoldName](#)
- UBYTE * [name](#)
- UBYTE * [pname](#)
- LONG [buffersize](#)
- LONG [bufferposition](#)
- LONG [inbuffer](#)
- int [previous](#)
- int [handle](#)
- int [type](#)
- int [prevars](#)
- int [previousNoShowInput](#)
- int [eqnum](#)
- int [afterwards](#)
- int [olddelay](#)
- int [oldnoshowinput](#)
- UBYTE [isnextchar](#)
- UBYTE [nextchar](#) [2]
- UBYTE [reserved](#)

3.132.1 Detailed Description

Input is read from 'streams' which are represented by objects of type STREAM. A stream can be a file, a do-loop, a procedure, the string value of a preprocessor variable When a new stream is opened we have to keep information about where to fall back in the parent stream to allow this to happen even in the middle of reading names etc as would be the case with a`i'b

Definition at line [736](#) of file [structs.h](#).

3.132.2 Field Documentation

3.132.2.1 fileposition

```
off_t Stream::fileposition
```

Definition at line [737](#) of file [structs.h](#).

3.132.2.2 `linenumber`

```
off_t Stream::linenumber
```

Definition at line [738](#) of file [structs.h](#).

3.132.2.3 `prevline`

```
off_t Stream::prevline
```

Definition at line [739](#) of file [structs.h](#).

3.132.2.4 `buffer`

```
UBYTE* Stream::buffer
```

[D] Size in buffersize

Definition at line [740](#) of file [structs.h](#).

3.132.2.5 `pointer`

```
UBYTE* Stream::pointer
```

pointer into buffer memory

Definition at line [741](#) of file [structs.h](#).

3.132.2.6 `top`

```
UBYTE* Stream::top
```

pointer into buffer memory

Definition at line [742](#) of file [structs.h](#).

3.132.2.7 `FoldName`

```
UBYTE* Stream::FoldName
```

[D]

Definition at line [743](#) of file [structs.h](#).

3.132.2.8 name

```
UBYTE* Stream::name
```

[D]

Definition at line 744 of file [structs.h](#).

3.132.2.9 pname

```
UBYTE* Stream::pname
```

for DOLLARSTREAM and PREVARSTREAM it points always to name, else it is undefined

Definition at line 745 of file [structs.h](#).

3.132.2.10 buffersize

```
LONG Stream::buffersize
```

Definition at line 747 of file [structs.h](#).

3.132.2.11 bufferposition

```
LONG Stream::bufferposition
```

Definition at line 748 of file [structs.h](#).

3.132.2.12 inbuffer

```
LONG Stream::inbuffer
```

Definition at line 749 of file [structs.h](#).

3.132.2.13 previous

```
int Stream::previous
```

Definition at line 750 of file [structs.h](#).

3.132.2.14 handle

```
int Stream::handle
```

Definition at line 751 of file [structs.h](#).

3.132.2.15 type

```
int Stream::type
```

Definition at line 752 of file [structs.h](#).

3.132.2.16 prevars

```
int Stream::prevars
```

Definition at line 753 of file [structs.h](#).

3.132.2.17 previousNoShowInput

```
int Stream::previousNoShowInput
```

Definition at line 754 of file [structs.h](#).

3.132.2.18 eqnum

```
int Stream::eqnum
```

Definition at line 755 of file [structs.h](#).

3.132.2.19 afterwards

```
int Stream::afterwards
```

Definition at line 756 of file [structs.h](#).

3.132.2.20 olddelay

```
int Stream::olddelay
```

Definition at line 757 of file [structs.h](#).

3.132.2.21 oldnoshowinput

```
int Stream::oldnoshowinput
```

Definition at line 758 of file [structs.h](#).

3.132.2.22 isnextchar

```
UBYTE Stream::isnextchar
```

Definition at line 759 of file [structs.h](#).

3.132.2.23 nextchar

```
UBYTE Stream::nextchar[2]
```

Definition at line 760 of file [structs.h](#).

3.132.2.24 reserved

```
UBYTE Stream::reserved
```

Definition at line 761 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.133 SuBbUf Struct Reference

Data Fields

- WORD [subexpnum](#)
- WORD [buffernum](#)

3.133.1 Detailed Description

Definition at line 257 of file [compiler.c](#).

3.133.2 Field Documentation

3.133.2.1 subexpnum

```
WORD SuBbUf::subexpnum
```

Definition at line 258 of file [compiler.c](#).

3.133.2.2 buffernum

```
WORD SuBbUf::buffernum
```

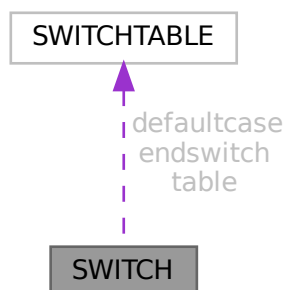
Definition at line 259 of file [compiler.c](#).

The documentation for this struct was generated from the following file:

- [compiler.c](#)

3.134 SWITCH Struct Reference

Collaboration diagram for SWITCH:



Data Fields

- [SWITCHTABLE * table](#)
- [SWITCHTABLE defaultcase](#)
- [SWITCHTABLE endswitch](#)
- WORD [typetable](#)
- WORD [maxcase](#)
- WORD [mincase](#)
- WORD [numcases](#)
- WORD [tablesize](#)
- WORD [caseoffset](#)
- WORD [iflevel](#)
- WORD [whilelevel](#)
- WORD [nestingsum](#)
- WORD [padding](#)

3.134.1 Detailed Description

Definition at line [1380](#) of file [structs.h](#).

3.134.2 Field Documentation

3.134.2.1 table

[SWITCHTABLE*](#) [SWITCH::table](#)

Definition at line [1381](#) of file [structs.h](#).

3.134.2.2 defaultcase

`SWITCHTABLE SWITCH::defaultcase`

Definition at line 1382 of file [structs.h](#).

3.134.2.3 endswitch

`SWITCHTABLE SWITCH::endswitch`

Definition at line 1383 of file [structs.h](#).

3.134.2.4 typetable

`WORD SWITCH::typetable`

Definition at line 1384 of file [structs.h](#).

3.134.2.5 maxcase

`WORD SWITCH::maxcase`

Definition at line 1385 of file [structs.h](#).

3.134.2.6 mincase

`WORD SWITCH::mincase`

Definition at line 1386 of file [structs.h](#).

3.134.2.7 numcases

`WORD SWITCH::numcases`

Definition at line 1387 of file [structs.h](#).

3.134.2.8 tablesize

`WORD SWITCH::tablesize`

Definition at line 1388 of file [structs.h](#).

3.134.2.9 caseoffset

`WORD SWITCH::caseoffset`

Definition at line 1389 of file [structs.h](#).

3.134.2.10 iflevel

WORD SWITCH::iflevel

Definition at line 1390 of file [structs.h](#).

3.134.2.11 whilelevel

WORD SWITCH::whilelevel

Definition at line 1391 of file [structs.h](#).

3.134.2.12 nestingsum

WORD SWITCH::nestingsum

Definition at line 1392 of file [structs.h](#).

3.134.2.13 padding

WORD SWITCH::padding

Definition at line 1393 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.135 SWITCHTABLE Struct Reference

Data Fields

- WORD [ncase](#)
- WORD [value](#)
- WORD [compbuffer](#)

3.135.1 Detailed Description

Definition at line 1374 of file [structs.h](#).

3.135.2 Field Documentation

3.135.2.1 ncase

WORD SWITCHTABLE::ncase

Definition at line 1375 of file [structs.h](#).

3.135.2.2 value

WORD SWITCHTABLE::value

Definition at line 1376 of file [structs.h](#).

3.135.2.3 compbuffer

WORD SWITCHTABLE::compbuffer

Definition at line 1377 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.136 SyMbOl Struct Reference

Data Fields

- LONG [name](#)
- WORD [minpower](#)
- WORD [maxpower](#)
- WORD [complex](#)
- WORD [number](#)
- WORD [flags](#)
- WORD [node](#)
- WORD [namesize](#)
- WORD [dimension](#)

3.136.1 Detailed Description

Definition at line 428 of file [structs.h](#).

3.136.2 Field Documentation

3.136.2.1 name

LONG SyMbOl::name

Definition at line 429 of file [structs.h](#).

3.136.2.2 minpower

WORD SyMbOl::minpower

Definition at line 430 of file [structs.h](#).

3.136.2.3 maxpower

WORD SyMbOl::maxpower

Definition at line 431 of file [structs.h](#).

3.136.2.4 complex

WORD SyMbOl::complex

Definition at line 432 of file [structs.h](#).

3.136.2.5 number

WORD SyMbOl::number

Definition at line 433 of file [structs.h](#).

3.136.2.6 flags

WORD SyMbOl::flags

Definition at line 434 of file [structs.h](#).

3.136.2.7 node

WORD SyMbOl::node

Definition at line 435 of file [structs.h](#).

3.136.2.8 namesize

WORD SyMbOl::namesize

Definition at line 436 of file [structs.h](#).

3.136.2.9 dimension

WORD SyMbOl::dimension

Definition at line 437 of file [structs.h](#).

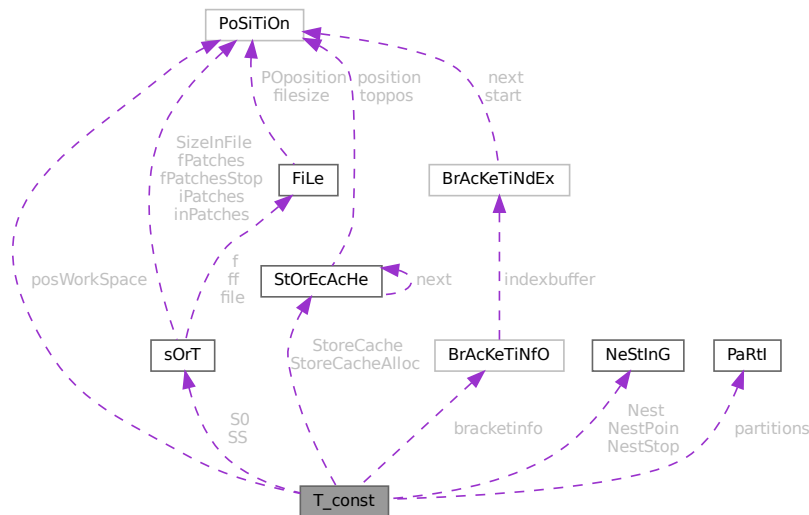
The documentation for this struct was generated from the following file:

- [structs.h](#)

3.137 T_const Struct Reference

```
#include <structs.h>
```

Collaboration diagram for T_const:



Data Fields

- `SORTING * S0`
- `SORTING * SS`
- `NESTING Nest`
- `NESTING NestStop`
- `NESTING NestPoin`
- `WORD * BrackBuf`
- `STORECACHE StoreCache`
- `STORECACHE StoreCacheAlloc`
- `WORD ** pWorkspace`
- `LONG * lWorkspace`
- `POSITION * posWorkspace`
- `WORD * Workspace`
- `WORD * WorkTop`
- `WORD * WorkPointer`
- `int * RepCount`
- `int * RepTop`
- `WORD * WildArgTaken`
- `UWORD * factorials`
- `WORD * small_power_n`
- `UWORD ** small_power`
- `UWORD * bernoullis`
- `WORD * primelist`
- `LONG * pfac`
- `LONG * pBer`
- `WORD * TMaddr`

- WORD * WildMask
- WORD * previousEfactor
- WORD ** TermMemHeap
- UWORD ** NumberMemHeap
- UWORD ** CacheNumberMemHeap
- BRACKETINFO * bracketinfo
- WORD ** ListPoly
- WORD * ListSymbols
- UWORD * NumMem
- WORD * TopologiesTerm
- WORD * TopologiesStart
- PARTI partitions
- LONG sBer
- LONG pWorkPointer
- LONG IWorkPointer
- LONG posWorkPointer
- LONG lnNumMem
- int sfact
- int mfac
- int ebufnum
- int fbufnum
- int allbufnum
- int aebufnum
- int idallflag
- int idallnum
- int idallmaxnum
- int WildcardBufferSize
- int TermMemMax
- int TermMemTop
- int NumberMemMax
- int NumberMemTop
- int CacheNumberMemMax
- int CacheNumberMemTop
- int bracketindexflag
- int optentimes
- int ListSymbolsSize
- int NumListSymbols
- int numpoly
- int LeaveNegative
- int TrimPower
- WORD small_power_maxx
- WORD small_power_maxn
- WORD dummysubexp [SUBEXPSIZE+4]
- WORD comsym [8]
- WORD comnum [4]
- WORD comfun [FUNHEAD+4]
- WORD comind [7]
- WORD MinVecArg [7+ARGHEAD]
- WORD FunArg [4+ARGHEAD+FUNHEAD]
- WORD locwildvalue [SUBEXPSIZE]
- WORD mulpat [SUBEXPSIZE+5]
- WORD proexp [SUBEXPSIZE+5]
- WORD TMout [40]
- WORD TMbuff
- WORD TMdolfac

- WORD [nfac](#)
- WORD [nBer](#)
- WORD [mBer](#)
- WORD [PolyAct](#)
- WORD [RecFlag](#)
- WORD [inprimelist](#)
- WORD [sizeprimelist](#)
- WORD [fromindex](#)
- WORD [setinterntopo](#)
- WORD [setexterntopo](#)
- WORD [TopologiesLevel](#)
- WORD [TopologiesOptions](#) [2]

3.137.1 Detailed Description

The [T_const](#) struct is part of the global data and resides either in the ALLGLOBALS struct A, or the ALLPRIVATES struct B (TFORM) under the name T We see it used with the macro AT as in AT.WorkPointer It has variables that are private to each thread, most of which have to be defined at startup.

Definition at line [2121](#) of file [structs.h](#).

3.137.2 Field Documentation

3.137.2.1 S0

```
SORTING* T_const::S0
```

Definition at line [2125](#) of file [structs.h](#).

3.137.2.2 SS

```
SORTING* T_const::SS
```

Definition at line [2126](#) of file [structs.h](#).

3.137.2.3 Nest

```
NESTING T_const::Nest
```

Definition at line [2127](#) of file [structs.h](#).

3.137.2.4 NestStop

```
NESTING T_const::NestStop
```

Definition at line [2128](#) of file [structs.h](#).

3.137.2.5 NestPoin

`NESTING T_const::NestPoin`

Definition at line 2129 of file [structs.h](#).

3.137.2.6 BrackBuf

`WORD* T_const::BrackBuf`

Definition at line 2130 of file [structs.h](#).

3.137.2.7 StoreCache

`STORECACHE T_const::StoreCache`

Definition at line 2131 of file [structs.h](#).

3.137.2.8 StoreCacheAlloc

`STORECACHE T_const::StoreCacheAlloc`

Definition at line 2132 of file [structs.h](#).

3.137.2.9 pWorkSpace

`WORD** T_const::pWorkSpace`

Definition at line 2133 of file [structs.h](#).

3.137.2.10 lWorkSpace

`LONG* T_const::lWorkSpace`

Definition at line 2134 of file [structs.h](#).

3.137.2.11 posWorkSpace

`POSITION* T_const::posWorkSpace`

Definition at line 2135 of file [structs.h](#).

3.137.2.12 WorkSpace

`WORD* T_const::WorkSpace`

Definition at line 2136 of file [structs.h](#).

3.137.2.13 WorkTop

WORD* T_const::WorkTop

Definition at line 2137 of file [structs.h](#).

3.137.2.14 WorkPointer

WORD* T_const::WorkPointer

Definition at line 2138 of file [structs.h](#).

3.137.2.15 RepCount

int* T_const::RepCount

Definition at line 2139 of file [structs.h](#).

3.137.2.16 RepTop

int* T_const::RepTop

Definition at line 2140 of file [structs.h](#).

3.137.2.17 WildArgTaken

WORD* T_const::WildArgTaken

Definition at line 2141 of file [structs.h](#).

3.137.2.18 factorials

UWORD* T_const::factorials

Definition at line 2142 of file [structs.h](#).

3.137.2.19 small_power_n

WORD* T_const::small_power_n

Definition at line 2143 of file [structs.h](#).

3.137.2.20 small_power

UWORD** T_const::small_power

Definition at line 2144 of file [structs.h](#).

3.137.2.21 bernoullis

UWORD* T_const::bernoullis

Definition at line 2145 of file [structs.h](#).

3.137.2.22 primelist

WORD* T_const::primelist

Definition at line 2146 of file [structs.h](#).

3.137.2.23 pfac

LONG* T_const::pfac

Definition at line 2147 of file [structs.h](#).

3.137.2.24 pBer

LONG* T_const::pBer

Definition at line 2148 of file [structs.h](#).

3.137.2.25 TMaddr

WORD* T_const::TMaddr

Definition at line 2149 of file [structs.h](#).

3.137.2.26 WildMask

WORD* T_const::WildMask

Definition at line 2150 of file [structs.h](#).

3.137.2.27 previousEfactor

WORD* T_const::previousEfactor

Definition at line 2151 of file [structs.h](#).

3.137.2.28 TermMemHeap

WORD** T_const::TermMemHeap

Definition at line 2152 of file [structs.h](#).

3.137.2.29 NumberMemHeap

UWORD** T_const::NumberMemHeap

Definition at line 2153 of file [structs.h](#).

3.137.2.30 CacheNumberMemHeap

UWORD** T_const::CacheNumberMemHeap

Definition at line 2154 of file [structs.h](#).

3.137.2.31 bracketinfo

BRACKETINFO* T_const::bracketinfo

Definition at line 2155 of file [structs.h](#).

3.137.2.32 ListPoly

WORD** T_const::ListPoly

Definition at line 2156 of file [structs.h](#).

3.137.2.33 ListSymbols

WORD* T_const::ListSymbols

Definition at line 2157 of file [structs.h](#).

3.137.2.34 NumMem

UWORD* T_const::NumMem

Definition at line 2158 of file [structs.h](#).

3.137.2.35 TopologiesTerm

WORD* T_const::TopologiesTerm

Definition at line 2159 of file [structs.h](#).

3.137.2.36 TopologiesStart

WORD* T_const::TopologiesStart

Definition at line 2160 of file [structs.h](#).

3.137.2.37 partitions

`PARTI T_const::partitions`

Definition at line 2169 of file [structs.h](#).

3.137.2.38 sBer

`LONG T_const::sBer`

Definition at line 2170 of file [structs.h](#).

3.137.2.39 pWorkPointer

`LONG T_const::pWorkPointer`

Definition at line 2171 of file [structs.h](#).

3.137.2.40 lWorkPointer

`LONG T_const::lWorkPointer`

Definition at line 2172 of file [structs.h](#).

3.137.2.41 posWorkPointer

`LONG T_const::posWorkPointer`

Definition at line 2173 of file [structs.h](#).

3.137.2.42 InNumMem

`LONG T_const::InNumMem`

Definition at line 2174 of file [structs.h](#).

3.137.2.43 sfact

`int T_const::sfact`

Definition at line 2175 of file [structs.h](#).

3.137.2.44 mfac

`int T_const::mfac`

Definition at line 2176 of file [structs.h](#).

3.137.2.45 ebufnum

```
int T_const::ebufnum
```

Definition at line 2177 of file [structs.h](#).

3.137.2.46 fbufnum

```
int T_const::fbufnum
```

Definition at line 2178 of file [structs.h](#).

3.137.2.47 allbufnum

```
int T_const::allbufnum
```

Definition at line 2179 of file [structs.h](#).

3.137.2.48 aebufnum

```
int T_const::aebufnum
```

Definition at line 2180 of file [structs.h](#).

3.137.2.49 idallflag

```
int T_const::idallflag
```

Definition at line 2181 of file [structs.h](#).

3.137.2.50 idallnum

```
int T_const::idallnum
```

Definition at line 2182 of file [structs.h](#).

3.137.2.51 idallmaxnum

```
int T_const::idallmaxnum
```

Definition at line 2183 of file [structs.h](#).

3.137.2.52 WildcardBufferSize

```
int T_const::WildcardBufferSize
```

Definition at line 2184 of file [structs.h](#).

3.137.2.53 TermMemMax

```
int T_const::TermMemMax
```

Definition at line 2193 of file [structs.h](#).

3.137.2.54 TermMemTop

```
int T_const::TermMemTop
```

Definition at line 2194 of file [structs.h](#).

3.137.2.55 NumberMemMax

```
int T_const::NumberMemMax
```

Definition at line 2195 of file [structs.h](#).

3.137.2.56 NumberMemTop

```
int T_const::NumberMemTop
```

Definition at line 2196 of file [structs.h](#).

3.137.2.57 CacheNumberMemMax

```
int T_const::CacheNumberMemMax
```

Definition at line 2197 of file [structs.h](#).

3.137.2.58 CacheNumberMemTop

```
int T_const::CacheNumberMemTop
```

Definition at line 2198 of file [structs.h](#).

3.137.2.59 bracketindexflag

```
int T_const::bracketindexflag
```

Definition at line 2199 of file [structs.h](#).

3.137.2.60 optentimes

```
int T_const::optentimes
```

Definition at line 2200 of file [structs.h](#).

3.137.2.61 ListSymbolsSize

```
int T_const::ListSymbolsSize
```

Definition at line 2201 of file [structs.h](#).

3.137.2.62 NumListSymbols

```
int T_const::NumListSymbols
```

Definition at line 2202 of file [structs.h](#).

3.137.2.63 numpoly

```
int T_const::numpoly
```

Definition at line 2203 of file [structs.h](#).

3.137.2.64 LeaveNegative

```
int T_const::LeaveNegative
```

Definition at line 2204 of file [structs.h](#).

3.137.2.65 TrimPower

```
int T_const::TrimPower
```

Definition at line 2205 of file [structs.h](#).

3.137.2.66 small_power_maxx

```
WORD T_const::small_power_maxx
```

Definition at line 2206 of file [structs.h](#).

3.137.2.67 small_power_maxn

```
WORD T_const::small_power_maxn
```

Definition at line 2207 of file [structs.h](#).

3.137.2.68 dummysubexp

```
WORD T_const::dummysubexp[SUBEXPSIZE+4]
```

Definition at line 2208 of file [structs.h](#).

3.137.2.69 comsym

```
WORD T_const::comsym[8]
```

Definition at line 2209 of file [structs.h](#).

3.137.2.70 comnum

```
WORD T_const::comnum[4]
```

Definition at line 2210 of file [structs.h](#).

3.137.2.71 comfun

```
WORD T_const::comfun[FUNHEAD+4]
```

Definition at line 2211 of file [structs.h](#).

3.137.2.72 comind

```
WORD T_const::comind[7]
```

Definition at line 2213 of file [structs.h](#).

3.137.2.73 MinVecArg

```
WORD T_const::MinVecArg[7+ARGHEAD]
```

Definition at line 2214 of file [structs.h](#).

3.137.2.74 FunArg

```
WORD T_const::FunArg[4+ARGHEAD+FUNHEAD]
```

Definition at line 2215 of file [structs.h](#).

3.137.2.75 locwildvalue

```
WORD T_const::locwildvalue[SUBEXPSIZE]
```

Definition at line 2216 of file [structs.h](#).

3.137.2.76 mulpat

```
WORD T_const::mulpat[SUBEXPSIZE+5]
```

Definition at line 2217 of file [structs.h](#).

3.137.2.77 proexp

```
WORD T_const::proexp[SUBEXPSIZE+5]
```

Definition at line 2219 of file [structs.h](#).

3.137.2.78 TMout

```
WORD T_const::TMout[40]
```

Definition at line 2220 of file [structs.h](#).

3.137.2.79 TMbuff

```
WORD T_const::TMbuff
```

Definition at line 2221 of file [structs.h](#).

3.137.2.80 TMdolfac

```
WORD T_const::TMdolfac
```

Definition at line 2222 of file [structs.h](#).

3.137.2.81 nfac

```
WORD T_const::nfac
```

Definition at line 2223 of file [structs.h](#).

3.137.2.82 nBer

```
WORD T_const::nBer
```

Definition at line 2224 of file [structs.h](#).

3.137.2.83 mBer

```
WORD T_const::mBer
```

Definition at line 2225 of file [structs.h](#).

3.137.2.84 PolyAct

```
WORD T_const::PolyAct
```

Definition at line 2226 of file [structs.h](#).

3.137.2.85 RecFlag

WORD T_const::RecFlag

Definition at line 2227 of file [structs.h](#).

3.137.2.86 inprimelist

WORD T_const::inprimelist

Definition at line 2228 of file [structs.h](#).

3.137.2.87 sizeprimelist

WORD T_const::sizeprimelist

Definition at line 2229 of file [structs.h](#).

3.137.2.88 fromindex

WORD T_const::fromindex

Definition at line 2230 of file [structs.h](#).

3.137.2.89 setinterntopo

WORD T_const::setinterntopo

Definition at line 2231 of file [structs.h](#).

3.137.2.90 setexterntopo

WORD T_const::setexterntopo

Definition at line 2232 of file [structs.h](#).

3.137.2.91 TopologiesLevel

WORD T_const::TopologiesLevel

Definition at line 2233 of file [structs.h](#).

3.137.2.92 TopologiesOptions

WORD T_const::TopologiesOptions[2]

Definition at line 2234 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.138 T_EEdge Class Reference

Data Fields

- int [nodes](#) [2]
- int [ext](#)
- int [momn](#)
- char [momc](#) [2]
- char [padding](#) [6]

3.138.1 Detailed Description

Definition at line 24 of file [gentopo.h](#).

3.138.2 Field Documentation

3.138.2.1 nodes

int T_EEdge::nodes[2]

Definition at line 26 of file [gentopo.h](#).

3.138.2.2 ext

int T_EEdge::ext

Definition at line 27 of file [gentopo.h](#).

3.138.2.3 momn

int T_EEdge::momn

Definition at line 28 of file [gentopo.h](#).

3.138.2.4 momc

```
char T_EEdge::momc[2]
```

Definition at line 29 of file [gentopo.h](#).

3.138.2.5 padding

```
char T_EEdge::padding[6]
```

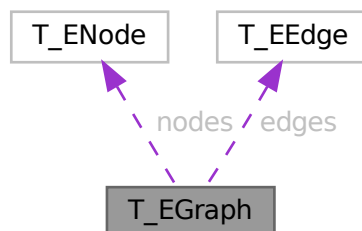
Definition at line 31 of file [gentopo.h](#).

The documentation for this class was generated from the following file:

- [gentopo.h](#)

3.139 T_EGraph Class Reference

Collaboration diagram for T_EGraph:



Public Member Functions

- [T_EGraph](#) (int nnodes, int nedges, int mxdeg)
- void [print](#) (void)
- void [init](#) (int pid, long gid, int **adjmat, int sopi, BigInt nsym, BigInt esym)
- void [setExtern](#) (int nd, int val)
- void [endSetExtern](#) (void)

Data Fields

- long `gld`
- int `pld`
- int `nNodes`
- int `nEdges`
- int `maxdeg`
- int `nExtern`
- int `opi`
- `BigInt` `nsym`
- `BigInt` `esym`
- `T_ENode` * `nodes`
- `T_EEdge` * `edges`

3.139.1 Detailed Description

Definition at line 34 of file [gentopo.h](#).

3.139.2 Constructor & Destructor Documentation

3.139.2.1 T_EGraph()

```
T_EGraph::T_EGraph (
    int nnodes,
    int nedges,
    int mxdeg )
```

Definition at line 88 of file [gentopo.cc](#).

3.139.2.2 ~T_EGraph()

```
T_EGraph::~T_EGraph ( )
```

Definition at line 109 of file [gentopo.cc](#).

3.139.3 Member Function Documentation

3.139.3.1 print()

```
void T_EGraph::print (
    void )
```

Definition at line 200 of file [gentopo.cc](#).

3.139.3.2 init()

```
void T_EGraph::init (
    int pid,
    long gid,
    int ** adjmat,
    int sopi,
    BigInt nsym,
    BigInt esym )
```

Definition at line 124 of file [gentopo.cc](#).

3.139.3.3 setExtern()

```
void T_EGraph::setExtern (
    int nd,
    int val )
```

Definition at line 181 of file [gentopo.cc](#).

3.139.3.4 endSetExtern()

```
void T_EGraph::endSetExtern (
    void )
```

Definition at line 187 of file [gentopo.cc](#).

3.139.4 Field Documentation

3.139.4.1 gId

```
long T_EGraph::gId
```

Definition at line 36 of file [gentopo.h](#).

3.139.4.2 pId

```
int T_EGraph::pId
```

Definition at line 37 of file [gentopo.h](#).

3.139.4.3 nNodes

```
int T_EGraph::nNodes
```

Definition at line 38 of file [gentopo.h](#).

3.139.4.4 nEdges

```
int T_EGraph::nEdges
```

Definition at line 39 of file [gentopo.h](#).

3.139.4.5 maxdeg

```
int T_EGraph::maxdeg
```

Definition at line 40 of file [gentopo.h](#).

3.139.4.6 nExtern

```
int T_EGraph::nExtern
```

Definition at line 41 of file [gentopo.h](#).

3.139.4.7 opi

```
int T_EGraph::opi
```

Definition at line 43 of file [gentopo.h](#).

3.139.4.8 nsym

```
BigInt T_EGraph::nsym
```

Definition at line 44 of file [gentopo.h](#).

3.139.4.9 esym

```
BigInt T_EGraph::esym
```

Definition at line 44 of file [gentopo.h](#).

3.139.4.10 nodes

```
T\_ENode\* T_EGraph::nodes
```

Definition at line 46 of file [gentopo.h](#).

3.139.4.11 edges

`T_EEdge* T_EGraph::edges`

Definition at line 47 of file [gentopo.h](#).

The documentation for this class was generated from the following files:

- [gentopo.h](#)
- [gentopo.cc](#)

3.140 T_ENode Class Reference

Data Fields

- `int` [deg](#)
- `int` [ext](#)
- `int *` [edges](#)

3.140.1 Detailed Description

Definition at line 17 of file [gentopo.h](#).

3.140.2 Field Documentation

3.140.2.1 deg

`int T_ENode::deg`

Definition at line 19 of file [gentopo.h](#).

3.140.2.2 ext

`int T_ENode::ext`

Definition at line 20 of file [gentopo.h](#).

3.140.2.3 edges

`int* T_ENode::edges`

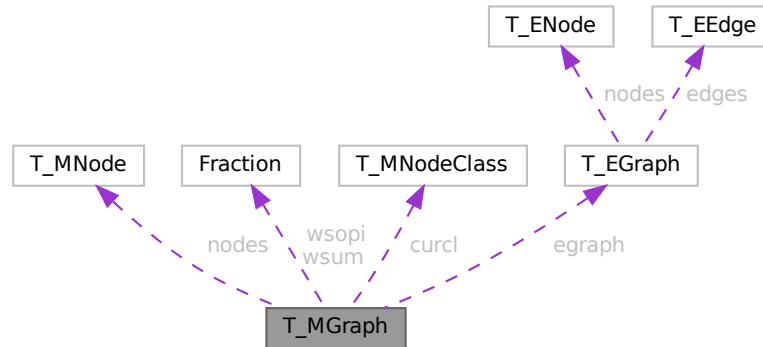
Definition at line 21 of file [gentopo.h](#).

The documentation for this class was generated from the following file:

- [gentopo.h](#)

3.141 T_MGraph Class Reference

Collaboration diagram for T_MGraph:



Public Member Functions

- [T_MGraph](#) (int pid, int ncl, int *cldeg, int *clnum, int *clext, int sopi)
- long [generate](#) (void)

Data Fields

- int [pId](#)
- int [nNodes](#)
- int [nEdges](#)
- int [nLoops](#)
- [T_MNode](#) ** [nodes](#)
- int * [clist](#)
- int [nClasses](#)
- int [mindeg](#)
- int [maxdeg](#)
- int [selOPI](#)
- long [ndiag](#)
- long [n1PI](#)
- int [nBridges](#)
- int [c1PI](#)
- int ** [adjMat](#)
- BigInt [nsym](#)
- BigInt [esym](#)
- [Fraction](#) [wsum](#)
- [Fraction](#) [wsopi](#)
- [T_MNodeClass](#) * [curcl](#)
- [T_EGraph](#) * [egraph](#)
- long [ngen](#)
- long [ngconn](#)
- long [nCallRefine](#)
- long [discardOrd](#)
- long [discardRefine](#)
- long [discardDisc](#)
- long [discardIso](#)

3.141.1 Detailed Description

Definition at line 84 of file [gentopo.h](#).

3.141.2 Constructor & Destructor Documentation

3.141.2.1 T_MGraph()

```
T_MGraph::T_MGraph (
    int pid,
    int ncl,
    int * cldeg,
    int * clnum,
    int * clext,
    int sopi )
```

Definition at line 473 of file [gentopo.cc](#).

3.141.2.2 ~T_MGraph()

```
T_MGraph::~T_MGraph ( )
```

Definition at line 572 of file [gentopo.cc](#).

3.141.3 Member Function Documentation

3.141.3.1 generate()

```
long T_MGraph::generate (
    void )
```

Definition at line 911 of file [gentopo.cc](#).

3.141.4 Field Documentation

3.141.4.1 pld

```
int T_MGraph::pId
```

Definition at line 89 of file [gentopo.h](#).

3.141.4.2 nNodes

```
int T_MGraph::nNodes
```

Definition at line 90 of file [gentopo.h](#).

3.141.4.3 nEdges

```
int T_MGraph::nEdges
```

Definition at line 91 of file [gentopo.h](#).

3.141.4.4 nLoops

```
int T_MGraph::nLoops
```

Definition at line 92 of file [gentopo.h](#).

3.141.4.5 nodes

```
T_MNode** T_MGraph::nodes
```

Definition at line 93 of file [gentopo.h](#).

3.141.4.6 clist

```
int* T_MGraph::clist
```

Definition at line 94 of file [gentopo.h](#).

3.141.4.7 nClasses

```
int T_MGraph::nClasses
```

Definition at line 95 of file [gentopo.h](#).

3.141.4.8 mindeg

```
int T_MGraph::mindeg
```

Definition at line 97 of file [gentopo.h](#).

3.141.4.9 maxdeg

```
int T_MGraph::maxdeg
```

Definition at line 98 of file [gentopo.h](#).

3.141.4.10 selOPI

```
int T_MGraph::selOPI
```

Definition at line 100 of file [gentopo.h](#).

3.141.4.11 ndiag

```
long T_MGraph::ndiag
```

Definition at line 103 of file [gentopo.h](#).

3.141.4.12 n1PI

```
long T_MGraph::n1PI
```

Definition at line 104 of file [gentopo.h](#).

3.141.4.13 nBridges

```
int T_MGraph::nBridges
```

Definition at line 105 of file [gentopo.h](#).

3.141.4.14 c1PI

```
int T_MGraph::c1PI
```

Definition at line 108 of file [gentopo.h](#).

3.141.4.15 adjMat

```
int** T_MGraph::adjMat
```

Definition at line 109 of file [gentopo.h](#).

3.141.4.16 nsym

```
BigInt T_MGraph::nsym
```

Definition at line 110 of file [gentopo.h](#).

3.141.4.17 esym

```
BigInt T_MGraph::esym
```

Definition at line 111 of file [gentopo.h](#).

3.141.4.18 wsum

```
Fraction T_MGraph::wsum
```

Definition at line 112 of file [gentopo.h](#).

3.141.4.19 wsopi

`Fraction T_MGraph::wsopi`

Definition at line 113 of file [gentopo.h](#).

3.141.4.20 curcl

`T_MNodeClass* T_MGraph::curcl`

Definition at line 114 of file [gentopo.h](#).

3.141.4.21 egraph

`T_EGraph* T_MGraph::egraph`

Definition at line 115 of file [gentopo.h](#).

3.141.4.22 ngen

`long T_MGraph::ngen`

Definition at line 118 of file [gentopo.h](#).

3.141.4.23 ngconn

`long T_MGraph::ngconn`

Definition at line 119 of file [gentopo.h](#).

3.141.4.24 nCallRefine

`long T_MGraph::nCallRefine`

Definition at line 121 of file [gentopo.h](#).

3.141.4.25 discardOrd

`long T_MGraph::discardOrd`

Definition at line 122 of file [gentopo.h](#).

3.141.4.26 discardRefine

`long T_MGraph::discardRefine`

Definition at line 123 of file [gentopo.h](#).

3.141.4.27 discardDisc

```
long T_MGraph::discardDisc
```

Definition at line 124 of file [gentopo.h](#).

3.141.4.28 discardIso

```
long T_MGraph::discardIso
```

Definition at line 125 of file [gentopo.h](#).

The documentation for this class was generated from the following files:

- [gentopo.h](#)
- [gentopo.cc](#)

3.142 T_MNode Class Reference

Public Member Functions

- [T_MNode](#) (int id, int deg, int ext, int clss)

Data Fields

- int [id](#)
- int [deg](#)
- int [clss](#)
- int [ext](#)
- int [freelg](#)
- int [visited](#)

3.142.1 Detailed Description

Definition at line 61 of file [gentopo.h](#).

3.142.2 Constructor & Destructor Documentation

3.142.2.1 T_MNode()

```
T_MNode::T_MNode (
    int id,
    int deg,
    int ext,
    int clss )
```

Definition at line 237 of file [gentopo.cc](#).

3.142.3 Field Documentation

3.142.3.1 id

```
int T_MNode::id
```

Definition at line 63 of file [gentopo.h](#).

3.142.3.2 deg

```
int T_MNode::deg
```

Definition at line 64 of file [gentopo.h](#).

3.142.3.3 clss

```
int T_MNode::clss
```

Definition at line 65 of file [gentopo.h](#).

3.142.3.4 ext

```
int T_MNode::ext
```

Definition at line 66 of file [gentopo.h](#).

3.142.3.5 freeIg

```
int T_MNode::freeIg
```

Definition at line 68 of file [gentopo.h](#).

3.142.3.6 visited

```
int T_MNode::visited
```

Definition at line 69 of file [gentopo.h](#).

The documentation for this class was generated from the following files:

- [gentopo.h](#)
- [gentopo.cc](#)

3.143 T_MNodeClass Class Reference

Public Member Functions

- [T_MNodeClass](#) (int nnodes, int ncl)
- void [init](#) (int *cl, int mxdeg, int **adjmat)
- void [copy](#) ([T_MNodeClass](#) *mnc)
- int [clCmp](#) (int nd0, int nd1, int cn)
- void [printMat](#) (void)
- void [mkFlist](#) (void)
- void [mkNdCl](#) (void)
- void [mkClMat](#) (int **adjmat)
- void [incMat](#) (int nd, int td, int val)
- Bool [chkOrd](#) (int nd, int ndc, [T_MNodeClass](#) *cl, int *dtcl)
- int [cmpArray](#) (int *a0, int *a1, int ma)

Data Fields

- int [nNodes](#)
- int [nClasses](#)
- int * [clist](#)
- int * [ndcl](#)
- int ** [clmat](#)
- int * [flist](#)
- int [maxdeg](#)
- int [foralignment](#)

3.143.1 Detailed Description

Definition at line 249 of file [gentopo.cc](#).

3.143.2 Constructor & Destructor Documentation

3.143.2.1 T_MNodeClass()

```
T_MNodeClass::T_MNodeClass (
    int nnodes,
    int ncl )
```

Definition at line 275 of file [gentopo.cc](#).

3.143.2.2 ~T_MNodeClass()

```
T_MNodeClass::~T_MNodeClass ( )
```

Definition at line 290 of file [gentopo.cc](#).

3.143.3 Member Function Documentation

3.143.3.1 init()

```
void T_MNodeClass::init (
    int * cl,
    int mxdeg,
    int ** adjmat )
```

Definition at line 302 of file [gentopo.cc](#).

3.143.3.2 copy()

```
void T_MNodeClass::copy (
    T_MNodeClass * mnc )
```

Definition at line 315 of file [gentopo.cc](#).

3.143.3.3 clCmp()

```
int T_MNodeClass::clCmp (
    int nd0,
    int nd1,
    int cn )
```

Definition at line 364 of file [gentopo.cc](#).

3.143.3.4 printMat()

```
void T_MNodeClass::printMat (
    void )
```

Definition at line 435 of file [gentopo.cc](#).

3.143.3.5 mkFlist()

```
void T_MNodeClass::mkFlist (
    void )
```

Definition at line 336 of file [gentopo.cc](#).

3.143.3.6 mkNdCl()

```
void T_MNodeClass::mkNdCl (
    void )
```

Definition at line 350 of file [gentopo.cc](#).

3.143.3.7 mkClMat()

```
void T_MNodeClass::mkClMat (
    int ** adjmat )
```

Definition at line 384 of file [gentopo.cc](#).

3.143.3.8 incMat()

```
void T_MNodeClass::incMat (
    int nd,
    int td,
    int val )
```

Definition at line 398 of file [gentopo.cc](#).

3.143.3.9 chkOrd()

```
Bool T_MNodeClass::chkOrd (
    int nd,
    int ndc,
    T_MNodeClass * cl,
    int * dtcl )
```

Definition at line 405 of file [gentopo.cc](#).

3.143.3.10 cmpArray()

```
int T_MNodeClass::cmpArray (
    int * a0,
    int * a1,
    int ma )
```

Definition at line 458 of file [gentopo.cc](#).

3.143.4 Field Documentation

3.143.4.1 nNodes

```
int T_MNodeClass::nNodes
```

Definition at line 251 of file [gentopo.cc](#).

3.143.4.2 nClasses

```
int T_MNodeClass::nClasses
```

Definition at line 252 of file [gentopo.cc](#).

3.143.4.3 clist

```
int* T_MNodeClass::clist
```

Definition at line [253](#) of file [gentopo.cc](#).

3.143.4.4 ndcl

```
int* T_MNodeClass::ndcl
```

Definition at line [254](#) of file [gentopo.cc](#).

3.143.4.5 clmat

```
int** T_MNodeClass::clmat
```

Definition at line [255](#) of file [gentopo.cc](#).

3.143.4.6 flist

```
int* T_MNodeClass::flist
```

Definition at line [256](#) of file [gentopo.cc](#).

3.143.4.7 maxdeg

```
int T_MNodeClass::maxdeg
```

Definition at line [257](#) of file [gentopo.cc](#).

3.143.4.8 forallignment

```
int T_MNodeClass::forallignment
```

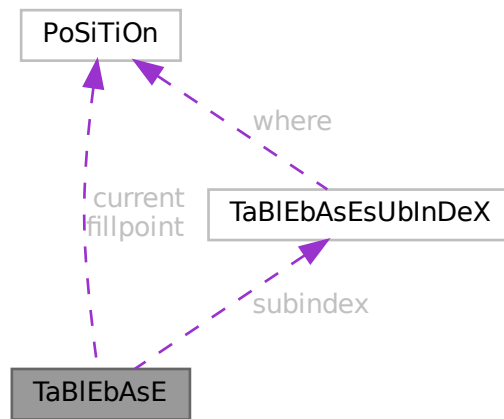
Definition at line [258](#) of file [gentopo.cc](#).

The documentation for this class was generated from the following file:

- [gentopo.cc](#)

3.144 TaBIEbAsE Struct Reference

Collaboration diagram for TaBIEbAsE:



Public Member Functions

- **PADPOSITION** (3, 0, 1, 0, 0)

Data Fields

- [POSITION](#) fillpoint
- [POSITION](#) current
- `UBYTE * name`
- `int * tablenumbers`
- [TABLEBASESUBINDEX](#) * `subindex`
- `int numtables`

3.144.1 Detailed Description

Definition at line 592 of file [structs.h](#).

3.144.2 Field Documentation

3.144.2.1 fillpoint

[POSITION](#) `TaBIEbAsE::fillpoint`

Definition at line 593 of file [structs.h](#).

3.144.2.2 current

`POSITION TaBlEbAsE::current`

Definition at line 594 of file [structs.h](#).

3.144.2.3 name

`UBYTE* TaBlEbAsE::name`

Definition at line 595 of file [structs.h](#).

3.144.2.4 tablenumbers

`int* TaBlEbAsE::tablenumbers`

Definition at line 596 of file [structs.h](#).

3.144.2.5 subindex

`TABLEBASESUBINDEX* TaBlEbAsE::subindex`

Definition at line 597 of file [structs.h](#).

3.144.2.6 numtables

`int TaBlEbAsE::numtables`

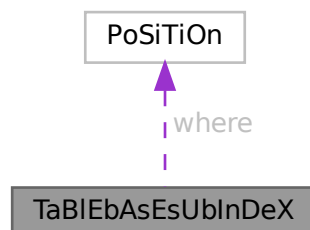
Definition at line 598 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.145 TaBlEbAsEsUbInDeX Struct Reference

Collaboration diagram for TaBlEbAsEsUbInDeX:



Public Member Functions

- **PADPOSITION** (0, 1, 0, 0, 0)

Data Fields

- **POSITION** where
- **LONG** size

3.145.1 Detailed Description

Definition at line 582 of file [structs.h](#).

3.145.2 Field Documentation

3.145.2.1 where

POSITION TaBlEbAsEsUbInDeX::where

Definition at line 583 of file [structs.h](#).

3.145.2.2 size

LONG TaBlEbAsEsUbInDeX::size

Definition at line 584 of file [structs.h](#).

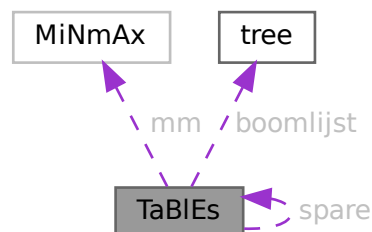
The documentation for this struct was generated from the following file:

- [structs.h](#)

3.146 TaBlEs Struct Reference

```
#include <structs.h>
```

Collaboration diagram for TaBlEs:



Data Fields

- WORD * [tablepointers](#)
- WORD * [prototype](#)
- WORD * [pattern](#)
- [MINMAX](#) * [mm](#)
- WORD * [flags](#)
- [COMPTREE](#) * [boomlijst](#)
- UBYTE * [argtail](#)
- struct [TaBleS](#) * [spare](#)
- WORD * [buffers](#)
- LONG [totind](#)
- LONG [reserved](#)
- LONG [defined](#)
- LONG [mdefined](#)
- int [prototypeSize](#)
- int [numind](#)
- int [bounds](#)
- int [strict](#)
- int [sparse](#)
- int [numtree](#)
- int [rootnum](#)
- int [MaxTreeSize](#)
- WORD [bufnum](#)
- WORD [bufferssize](#)
- WORD [buffersfill](#)
- WORD [tablenum](#)
- WORD [mode](#)
- WORD [numdummies](#)

3.146.1 Detailed Description

[buffers](#), [mm](#), [flags](#), and [prototype](#) are always dynamically allocated, [tablepointers](#) only if needed (=0 if unallocated), [boomlijst](#) and [argtail](#) only for sparse tables.

Allocation is done for both the normal and the stub instance ([spare](#)), except for [prototype](#) and [argtail](#) which share memory.

Definition at line [350](#) of file [structs.h](#).

3.146.2 Field Documentation

3.146.2.1 [tablepointers](#)

```
WORD* TaBleS::tablepointers
```

[D] Start in [tablepointers](#) table.

Definition at line [351](#) of file [structs.h](#).

Referenced by [DoRecovery\(\)](#).

3.146.2.2 prototype

WORD* TaBlEs::prototype

[D] The wildcard prototyping for arguments

Definition at line 356 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#).

3.146.2.3 pattern

WORD* TaBlEs::pattern

The pattern with which to match the arguments

Definition at line 357 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#).

3.146.2.4 mm

MINMAX* TaBlEs::mm

[D] Array bounds, dimension by dimension. # elements = numind.

Definition at line 359 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#).

3.146.2.5 flags

WORD* TaBlEs::flags

[D] Is element in use ? etc. # elements = numind.

Definition at line 360 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#).

3.146.2.6 boomlijst

COMPTREE* TaBlEs::boomlijst

[D] Tree for searching in sparse tables

Definition at line 361 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#).

3.146.2.7 argtail

```
UBYTE* TaBles::argtail
```

[D] The arguments in characters. Starts for tablebase with parenthesis to indicate tail

Definition at line 362 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#).

3.146.2.8 spare

```
struct TaBles* TaBles::spare
```

[D] For tablebase. Alternatingly stubs and real

Definition at line 364 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#).

3.146.2.9 buffers

```
WORD* TaBles::buffers
```

[D] When we use more than one compiler buffer.

Definition at line 365 of file [structs.h](#).

Referenced by [AddRHS\(\)](#), and [DoRecovery\(\)](#).

3.146.2.10 totind

```
LONG TaBles::totind
```

Total number requested

Definition at line 366 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#).

3.146.2.11 reserved

```
LONG TaBles::reserved
```

Total reservation in tablepointers for sparse

Definition at line 367 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#).

3.146.2.12 defined

```
LONG TaBlEs::defined
```

Number of table elements that are defined

Definition at line 368 of file [structs.h](#).

3.146.2.13 mdefined

```
LONG TaBlEs::mdefined
```

Same as defined but after .global

Definition at line 369 of file [structs.h](#).

3.146.2.14 prototypeSize

```
int TaBlEs::prototypeSize
```

Size of allocated memory for prototype in bytes.

Definition at line 370 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#).

3.146.2.15 numind

```
int TaBlEs::numind
```

Number of array indices

Definition at line 371 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#).

3.146.2.16 bounds

```
int TaBlEs::bounds
```

Array bounds check on/off.

Definition at line 372 of file [structs.h](#).

3.146.2.17 strict

```
int TaBlEs::strict
```

>0: all must be defined. <0: undefined not substitute

Definition at line 373 of file [structs.h](#).

3.146.2.18 sparse

```
int TaBlEs::sparse
```

0 --> sparse table

Definition at line 374 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#).

3.146.2.19 numtree

```
int TaBlEs::numtree
```

For the tree for sparse tables

Definition at line 375 of file [structs.h](#).

3.146.2.20 rootnum

```
int TaBlEs::rootnum
```

For the tree for sparse tables

Definition at line 376 of file [structs.h](#).

3.146.2.21 MaxTreeSize

```
int TaBlEs::MaxTreeSize
```

For the tree for sparse tables

Definition at line 377 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#).

3.146.2.22 bufnum

```
WORD TaBlEs::bufnum
```

Each table potentially its own buffer

Definition at line 378 of file [structs.h](#).

Referenced by [AddRHS\(\)](#).

3.146.2.23 buffersize

WORD TaBlEs::buffersize

When we use more than one compiler buffer

Definition at line 379 of file [structs.h](#).

Referenced by [AddRHS\(\)](#), and [DoRecovery\(\)](#).

3.146.2.24 buffersfill

WORD TaBlEs::buffersfill

When we use more than one compiler buffer

Definition at line 380 of file [structs.h](#).

Referenced by [AddRHS\(\)](#).

3.146.2.25 tablenum

WORD TaBlEs::tablenum

For testing of tableuse

Definition at line 381 of file [structs.h](#).

3.146.2.26 mode

WORD TaBlEs::mode

0: normal, 1: stub

Definition at line 382 of file [structs.h](#).

3.146.2.27 numdummies

WORD TaBlEs::numdummies

Definition at line 383 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.147 TERMINFO Struct Reference

Data Fields

- WORD * [term](#)
- void * [currentModel](#)
- void * [currentMODEL](#)
- WORD * [legcouple](#) [MAXLEGS+1]
- LONG [numtopo](#)
- LONG [numdia](#)
- WORD [diaoffset](#)
- WORD [level](#)
- WORD [externalset](#)
- WORD [internalset](#)
- WORD [numextern](#)
- WORD [flags](#)

3.147.1 Detailed Description

Definition at line [1396](#) of file [structs.h](#).

3.147.2 Field Documentation

3.147.2.1 term

```
WORD* TERMINFO::term
```

Definition at line [1397](#) of file [structs.h](#).

3.147.2.2 currentModel

```
void* TERMINFO::currentModel
```

Definition at line [1398](#) of file [structs.h](#).

3.147.2.3 currentMODEL

```
void* TERMINFO::currentMODEL
```

Definition at line [1399](#) of file [structs.h](#).

3.147.2.4 legcouple

```
WORD* TERMINFO::legcouple[MAXLEGS+1]
```

Definition at line [1400](#) of file [structs.h](#).

3.147.2.5 numtopo

LONG TERMINFO::numtopo

Definition at line 1401 of file [structs.h](#).

3.147.2.6 numdia

LONG TERMINFO::numdia

Definition at line 1402 of file [structs.h](#).

3.147.2.7 diaoffset

WORD TERMINFO::diaoffset

Definition at line 1403 of file [structs.h](#).

3.147.2.8 level

WORD TERMINFO::level

Definition at line 1404 of file [structs.h](#).

3.147.2.9 externalset

WORD TERMINFO::externalset

Definition at line 1405 of file [structs.h](#).

3.147.2.10 internalset

WORD TERMINFO::internalset

Definition at line 1406 of file [structs.h](#).

3.147.2.11 numextern

WORD TERMINFO::numextern

Definition at line 1407 of file [structs.h](#).

3.147.2.12 flags

WORD TERMINFO::flags

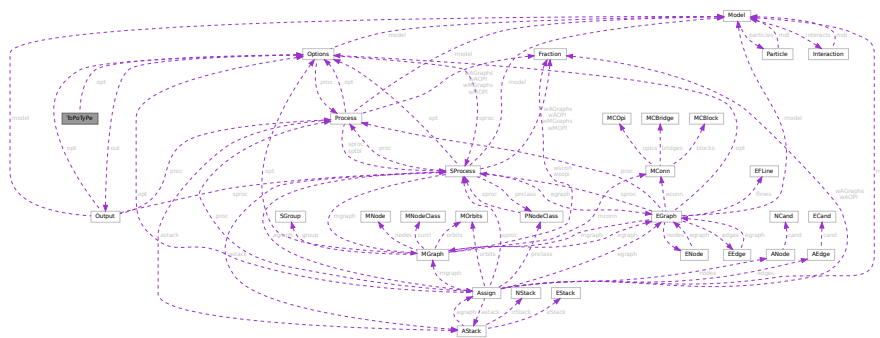
Definition at line 1408 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.148 ToPoTyPe Struct Reference

Collaboration diagram for ToPoTyPe:



Data Fields

- WORD * [vert](#)
- WORD * [vertmax](#)
- [Options](#) * [opt](#)
- int [cldeg](#) [MAXPOINTS]
- int [clnum](#) [MAXPOINTS]
- int [clex](#) [MAXPOINTS]
- int [cmin](#) [MAXLEGS+1]
- int [cmax](#) [MAXLEGS+1]
- int [ncl](#)
- int [nvert](#)
- int [nloops](#)
- int [nlegs](#)
- int [npadding](#)
- WORD [nvert](#)
- WORD [sopi](#)

3.148.1 Detailed Description

Definition at line 12 of file [diawrap.cc](#).

3.148.2 Field Documentation

3.148.2.1 vert

WORD * ToPoTyPe::vert

Definition at line 13 of file [diawrap.cc](#).

3.148.2.2 vertmax

WORD * ToPoTyPe::vertmax

Definition at line 14 of file [diawrap.cc](#).

3.148.2.3 opt

Options* ToPoTyPe::opt

Definition at line 15 of file [diawrap.cc](#).

3.148.2.4 cldeg

int ToPoTyPe::cldeg

Definition at line 16 of file [diawrap.cc](#).

3.148.2.5 clnum

int ToPoTyPe::clnum

Definition at line 16 of file [diawrap.cc](#).

3.148.2.6 clext

int ToPoTyPe::clext

Definition at line 16 of file [diawrap.cc](#).

3.148.2.7 cmind

int ToPoTyPe::cmind[MAXLEGS+1]

Definition at line 17 of file [diawrap.cc](#).

3.148.2.8 cmaxd

```
int ToPoTyPe::cmaxd[MAXLEGS+1]
```

Definition at line 17 of file [diawrap.cc](#).

3.148.2.9 ncl

```
int ToPoTyPe::ncl
```

Definition at line 18 of file [diawrap.cc](#).

3.148.2.10 nvert [1/2]

```
int ToPoTyPe::nvert
```

Definition at line 18 of file [diawrap.cc](#).

3.148.2.11 nloops

```
int ToPoTyPe::nloops
```

Definition at line 48 of file [topowrap.cc](#).

3.148.2.12 nlegs

```
int ToPoTyPe::nlegs
```

Definition at line 48 of file [topowrap.cc](#).

3.148.2.13 npadding

```
int ToPoTyPe::npadding
```

Definition at line 48 of file [topowrap.cc](#).

3.148.2.14 nvert [2/2]

```
WORD ToPoTyPe::nvert
```

Definition at line 51 of file [topowrap.cc](#).

3.148.2.15 sopi

WORD ToPoTyPe::sopi

Definition at line 52 of file [topowrap.cc](#).

The documentation for this struct was generated from the following files:

- [diawrap.cc](#)
- [topowrap.cc](#)

3.149 TrAcEn Struct Reference

```
#include <structs.h>
```

Data Fields

- WORD * [accu](#)
- WORD * [accup](#)
- WORD * [termp](#)
- WORD * [perm](#)
- WORD * [inlist](#)
- WORD [sgn](#)
- WORD [num](#)
- WORD [level](#)
- WORD [factor](#)
- WORD [allsign](#)

3.149.1 Detailed Description

The struct TRACEN keeps track of the progress during the expansion of a 4-dimensional trace. Each time a term gets generated the expansion tree continues in the next statement. When it returns it has to know where to continue.

Definition at line 822 of file [structs.h](#).

3.149.2 Field Documentation

3.149.2.1 accu

WORD* TrAcEn::accu

Definition at line 823 of file [structs.h](#).

3.149.2.2 accup

WORD* TrAcEn::accup

Definition at line 824 of file [structs.h](#).

3.149.2.3 temp

WORD* TrAcEn::temp

Definition at line 825 of file [structs.h](#).

3.149.2.4 perm

WORD* TrAcEn::perm

Definition at line 826 of file [structs.h](#).

3.149.2.5 inlist

WORD* TrAcEn::inlist

Definition at line 827 of file [structs.h](#).

3.149.2.6 sgn

WORD TrAcEn::sgn

Definition at line 828 of file [structs.h](#).

3.149.2.7 num

WORD TrAcEn::num

Definition at line 828 of file [structs.h](#).

3.149.2.8 level

WORD TrAcEn::level

Definition at line 828 of file [structs.h](#).

3.149.2.9 factor

WORD TrAcEn::factor

Definition at line 828 of file [structs.h](#).

3.149.2.10 allsign

WORD TrAcEn::allsign

Definition at line 828 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.150 TrAcEs Struct Reference

```
#include <structs.h>
```

Data Fields

- WORD * [accu](#)
- WORD * [accup](#)
- WORD * [termp](#)
- WORD * [perm](#)
- WORD * [inlist](#)
- WORD * [nt3](#)
- WORD * [nt4](#)
- WORD * [j3](#)
- WORD * [j4](#)
- WORD * [e3](#)
- WORD * [e4](#)
- WORD * [eers](#)
- WORD * [mepf](#)
- WORD * [mdel](#)
- WORD * [pepf](#)
- WORD * [pdel](#)
- WORD [sgn](#)
- WORD [stap](#)
- WORD [step1](#)
- WORD [kstep](#)
- WORD [mdum](#)
- WORD [gamm](#)
- WORD [ad](#)
- WORD [a3](#)
- WORD [a4](#)
- WORD [lc3](#)
- WORD [lc4](#)
- WORD [sign1](#)
- WORD [sign2](#)
- WORD [gamma5](#)
- WORD [num](#)
- WORD [level](#)
- WORD [factor](#)
- WORD [allsign](#)
- WORD [finalstep](#)

3.150.1 Detailed Description

The struct TRACES keeps track of the progress during the expansion of a 4-dimensional trace. Each time a term gets generated the expansion tree continues in the next statement. When it returns it has to know where to continue. The 4-dimensional traces are more complicated than the n-dimensional traces (see TRACEN) because of the extra tricks that can be used. They are responsible for the shorter final expressions.

Definition at line 789 of file [structs.h](#).

3.150.2 Field Documentation

3.150.2.1 `accu`

```
WORD* TrAcEs::accu
```

Definition at line 790 of file [structs.h](#).

3.150.2.2 `accup`

```
WORD* TrAcEs::accup
```

Definition at line 791 of file [structs.h](#).

3.150.2.3 `termp`

```
WORD* TrAcEs::termp
```

Definition at line 792 of file [structs.h](#).

3.150.2.4 `perm`

```
WORD* TrAcEs::perm
```

Definition at line 793 of file [structs.h](#).

3.150.2.5 `inlist`

```
WORD* TrAcEs::inlist
```

Definition at line 794 of file [structs.h](#).

3.150.2.6 `nt3`

```
WORD* TrAcEs::nt3
```

Definition at line 795 of file [structs.h](#).

3.150.2.7 nt4

WORD* TrAcEs::nt4

Definition at line 796 of file [structs.h](#).

3.150.2.8 j3

WORD* TrAcEs::j3

Definition at line 797 of file [structs.h](#).

3.150.2.9 j4

WORD* TrAcEs::j4

Definition at line 798 of file [structs.h](#).

3.150.2.10 e3

WORD* TrAcEs::e3

Definition at line 799 of file [structs.h](#).

3.150.2.11 e4

WORD* TrAcEs::e4

Definition at line 800 of file [structs.h](#).

3.150.2.12 eers

WORD* TrAcEs::eers

Definition at line 801 of file [structs.h](#).

3.150.2.13 mepf

WORD* TrAcEs::mepf

Definition at line 802 of file [structs.h](#).

3.150.2.14 mdel

WORD* TrAcEs::mdel

Definition at line 803 of file [structs.h](#).

3.150.2.15 pef

WORD* TrAcEs::pepf

Definition at line [804](#) of file [structs.h](#).

3.150.2.16 pdel

WORD* TrAcEs::pdel

Definition at line [805](#) of file [structs.h](#).

3.150.2.17 sgn

WORD TrAcEs::sgn

Definition at line [806](#) of file [structs.h](#).

3.150.2.18 stap

WORD TrAcEs::stap

Definition at line [807](#) of file [structs.h](#).

3.150.2.19 step1

WORD TrAcEs::step1

Definition at line [808](#) of file [structs.h](#).

3.150.2.20 kstep

WORD TrAcEs::kstep

Definition at line [808](#) of file [structs.h](#).

3.150.2.21 mdum

WORD TrAcEs::mdum

Definition at line [808](#) of file [structs.h](#).

3.150.2.22 gamm

WORD TrAcEs::gamm

Definition at line [809](#) of file [structs.h](#).

3.150.2.23 ad

WORD TrAcEs::ad

Definition at line 809 of file [structs.h](#).

3.150.2.24 a3

WORD TrAcEs::a3

Definition at line 809 of file [structs.h](#).

3.150.2.25 a4

WORD TrAcEs::a4

Definition at line 809 of file [structs.h](#).

3.150.2.26 lc3

WORD TrAcEs::lc3

Definition at line 809 of file [structs.h](#).

3.150.2.27 lc4

WORD TrAcEs::lc4

Definition at line 809 of file [structs.h](#).

3.150.2.28 sign1

WORD TrAcEs::sign1

Definition at line 810 of file [structs.h](#).

3.150.2.29 sign2

WORD TrAcEs::sign2

Definition at line 810 of file [structs.h](#).

3.150.2.30 gamma5

WORD TrAcEs::gamma5

Definition at line 810 of file [structs.h](#).

3.150.2.31 num

WORD TrAcEs::num

Definition at line 810 of file [structs.h](#).

3.150.2.32 level

WORD TrAcEs::level

Definition at line 810 of file [structs.h](#).

3.150.2.33 factor

WORD TrAcEs::factor

Definition at line 810 of file [structs.h](#).

3.150.2.34 allsign

WORD TrAcEs::allsign

Definition at line 810 of file [structs.h](#).

3.150.2.35 finalstep

WORD TrAcEs::finalstep

Definition at line 811 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.151 tree Struct Reference

```
#include <structs.h>
```

Data Fields

- int [parent](#)
- int [left](#)
- int [right](#)
- int [value](#)
- int [blnce](#)
- int [usage](#)

3.151.1 Detailed Description

The subexpressions in the compiler are kept track of in a (balanced) tree to reduce the need for subexpressions and hence save much space in large rhs expressions (like when we have xxxxxx occurrences of objects like $f(x+1, x+1)$ in which each $x+1$ becomes a subexpression. The struct that controls this tree is COMPTREE.

Definition at line 294 of file [structs.h](#).

3.151.2 Field Documentation

3.151.2.1 parent

```
int tree::parent
```

Index of parent

Definition at line 295 of file [structs.h](#).

Referenced by [IniFbuffer\(\)](#).

3.151.2.2 left

```
int tree::left
```

Left child (if not -1)

Definition at line 296 of file [structs.h](#).

Referenced by [IniFbuffer\(\)](#).

3.151.2.3 right

```
int tree::right
```

Right child (if not -1)

Definition at line 297 of file [structs.h](#).

Referenced by [IniFbuffer\(\)](#).

3.151.2.4 value

```
int tree::value
```

The object to be sorted and searched

Definition at line 298 of file [structs.h](#).

Referenced by [IniFbuffer\(\)](#).

3.151.2.5 blnce

```
int tree::blnce
```

Balance factor

Definition at line 299 of file [structs.h](#).

Referenced by [IniFbuffer\(\)](#).

3.151.2.6 usage

```
int tree::usage
```

Number of uses in some types of trees

Definition at line 300 of file [structs.h](#).

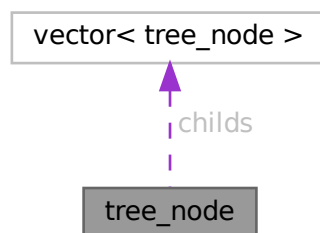
Referenced by [CleanupArgCache\(\)](#), and [IniFbuffer\(\)](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.152 tree_node Class Reference

Collaboration diagram for tree_node:



Public Member Functions

- [tree_node](#) (int _var=0)
- [tree_node](#) (const [tree_node](#) &other)
- [tree_node](#) & [operator=](#) (const [tree_node](#) &other)

Data Fields

- vector< [tree_node](#) > [childs](#)
- double [sum_results](#)
- int [num_visits](#)
- WORD [var](#)
- bool [finished](#)

3.152.1 Detailed Description

Definition at line 125 of file [optimize.cc](#).

3.152.2 Constructor & Destructor Documentation

3.152.2.1 tree_node() [1/2]

```
tree_node::tree_node (  
    int _var = 0 ) [inline]
```

Definition at line 134 of file [optimize.cc](#).

3.152.2.2 tree_node() [2/2]

```
tree_node::tree_node (  
    const tree\_node & other ) [inline]
```

Definition at line 137 of file [optimize.cc](#).

3.152.3 Member Function Documentation

3.152.3.1 operator=()

```
tree\_node & tree_node::operator= (  
    const tree\_node & other ) [inline]
```

Definition at line 144 of file [optimize.cc](#).

3.152.4 Field Documentation

3.152.4.1 childs

```
vector<tree\_node> tree_node::childs
```

Definition at line 127 of file [optimize.cc](#).

3.152.4.2 `sum_results`

```
double tree_node::sum_results
```

Definition at line 128 of file [optimize.cc](#).

3.152.4.3 `num_visits`

```
int tree_node::num_visits
```

Definition at line 129 of file [optimize.cc](#).

3.152.4.4 `var`

```
WORD tree_node::var
```

Definition at line 130 of file [optimize.cc](#).

3.152.4.5 `finished`

```
bool tree_node::finished
```

Definition at line 131 of file [optimize.cc](#).

The documentation for this class was generated from the following file:

- [optimize.cc](#)

3.153 `VaRrEnUm` Struct Reference

```
#include <structs.h>
```

Data Fields

- WORD * [start](#)
- WORD * [lo](#)
- WORD * [hi](#)

3.153.1 Detailed Description

Contains the pointers to an array in which a binary search will be performed.

Definition at line 166 of file [structs.h](#).

3.153.2 Field Documentation

3.153.2.1 start

```
WORD* VaRrEnUm::start
```

Start point for search. Points inbetween lo and hi

Definition at line 167 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#).

3.153.2.2 lo

```
WORD* VaRrEnUm::lo
```

Start of memory area

Definition at line 168 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#), [Generator\(\)](#), and [GetFirstTerm\(\)](#).

3.153.2.3 hi

```
WORD* VaRrEnUm::hi
```

End of memory area

Definition at line 169 of file [structs.h](#).

Referenced by [DoRecovery\(\)](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.154 VeCtOr Struct Reference

Data Fields

- LONG [name](#)
- WORD [complex](#)
- WORD [number](#)
- WORD [flags](#)
- WORD [node](#)
- WORD [namesize](#)
- WORD [dimension](#)

3.154.1 Detailed Description

Definition at line [461](#) of file [structs.h](#).

3.154.2 Field Documentation

3.154.2.1 name

```
LONG VeCtOr::name
```

Definition at line [462](#) of file [structs.h](#).

3.154.2.2 complex

```
WORD VeCtOr::complex
```

Definition at line [463](#) of file [structs.h](#).

3.154.2.3 number

```
WORD VeCtOr::number
```

Definition at line [464](#) of file [structs.h](#).

3.154.2.4 flags

```
WORD VeCtOr::flags
```

Definition at line [465](#) of file [structs.h](#).

3.154.2.5 node

```
WORD VeCtOr::node
```

Definition at line [466](#) of file [structs.h](#).

3.154.2.6 namesize

```
WORD VeCtOr::namesize
```

Definition at line [467](#) of file [structs.h](#).

3.154.2.7 dimension

WORD VeCtOr::dimension

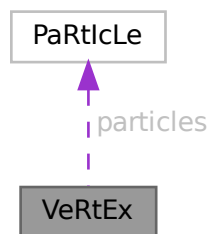
Definition at line 468 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.155 VeRtEx Struct Reference

Collaboration diagram for VeRtEx:



Data Fields

- [PARTICLE particles](#) [MAXPARTICLES]
- WORD [couplings](#) [2 *MAXCOUPLINGS]
- WORD [nparticles](#)
- WORD [ncouplings](#)
- WORD [type](#)
- WORD [error](#)
- WORD [externonly](#)
- WORD [spare](#)

3.155.1 Detailed Description

Definition at line 626 of file [structs.h](#).

3.155.2 Field Documentation

3.155.2.1 particles

[PARTICLE](#) VeRtEx::particles [MAXPARTICLES]

Definition at line 627 of file [structs.h](#).

3.155.2.2 couplings

WORD VeRtEx::couplings[2 *MAXCOUPLINGS]

Definition at line 628 of file [structs.h](#).

3.155.2.3 nparticles

WORD VeRtEx::nparticles

Definition at line 629 of file [structs.h](#).

3.155.2.4 ncouplings

WORD VeRtEx::ncouplings

Definition at line 630 of file [structs.h](#).

3.155.2.5 type

WORD VeRtEx::type

Definition at line 631 of file [structs.h](#).

3.155.2.6 error

WORD VeRtEx::error

Definition at line 632 of file [structs.h](#).

3.155.2.7 externonly

WORD VeRtEx::externonly

Definition at line 633 of file [structs.h](#).

3.155.2.8 spare

WORD VeRtEx::spare

Definition at line 634 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

3.156 X_const Struct Reference

```
#include <structs.h>
```

Data Fields

- `UBYTE * currentPrompt`
- `UBYTE * shellname`
- `UBYTE * stderrname`
- `int timeout`
- `int killSignal`
- `int killWholeGroup`
- `int daemonize`
- `int currentExternalChannel`

3.156.1 Detailed Description

The `X_const` struct is part of the global data and resides in the `ALLGLOBALS` struct `A` under the name `X`. We see it used with the macro `AX` as in `AX.timeout`. It contains variables that involve communication with external programs.

Definition at line 2559 of file `structs.h`.

3.156.2 Field Documentation

3.156.2.1 currentPrompt

```
UBYTE* X_const::currentPrompt
```

Definition at line 2560 of file `structs.h`.

3.156.2.2 shellname

```
UBYTE* X_const::shellname
```

Definition at line 2561 of file `structs.h`.

3.156.2.3 stderrname

```
UBYTE* X_const::stderrname
```

Definition at line 2563 of file `structs.h`.

3.156.2.4 timeout

```
int X_const::timeout
```

Definition at line 2565 of file `structs.h`.

3.156.2.5 killSignal

```
int X_const::killSignal
```

Definition at line 2568 of file [structs.h](#).

3.156.2.6 killWholeGroup

```
int X_const::killWholeGroup
```

Definition at line 2569 of file [structs.h](#).

3.156.2.7 daemonize

```
int X_const::daemonize
```

Definition at line 2571 of file [structs.h](#).

3.156.2.8 currentExternalChannel

```
int X_const::currentExternalChannel
```

Definition at line 2572 of file [structs.h](#).

The documentation for this struct was generated from the following file:

- [structs.h](#)

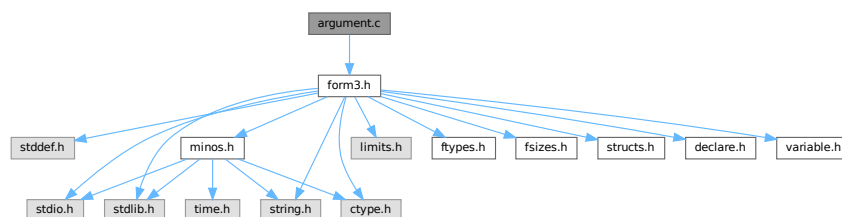
Chapter 4

File Documentation

4.1 argument.c File Reference

```
#include "form3.h"
```

Include dependency graph for argument.c:



Macros

- `#define` [NEWORDER](#)

Functions

- WORD [execarg](#) (PHEAD WORD *term, WORD level)
- WORD [execterm](#) (PHEAD WORD *term, WORD level)
- int [ArgumentImplode](#) (PHEAD WORD *term, WORD *thelist)
- int [ArgumentExplode](#) (PHEAD WORD *term, WORD *thelist)
- int [ArgFactorize](#) (PHEAD WORD *argin, WORD *argout)
- WORD [FindArg](#) (PHEAD WORD *a)
- WORD [InsertArg](#) (PHEAD WORD *argin, WORD *argout, int par)
- int [CleanupArgCache](#) (PHEAD WORD bufnum)
- int [ArgSymbolMerge](#) (WORD *t1, WORD *t2)
- int [ArgDotproductMerge](#) (WORD *t1, WORD *t2)
- WORD * [TakeArgContent](#) (PHEAD WORD *argin, WORD *argout)
- WORD * [MakeInteger](#) (PHEAD WORD *argin, WORD *argout, WORD *argfree)
- WORD * [MakeMod](#) (PHEAD WORD *argin, WORD *argout, WORD *argfree)
- void [SortWeights](#) (LONG *weights, LONG *extraspace, WORD number)

4.1.1 Detailed Description

Contains the routines that deal with the execution phase of the argument and related statements (like term)

Definition in file [argument.c](#).

4.1.2 Macro Definition Documentation

4.1.2.1 NEWORDER

```
#define NEWORDER
```

Factorizes an argument in general notation (meaning that the first word of the argument is a positive size indicator)
Input (argin): pointer to the complete argument [Output](#) (argout): Pointer to where the output should be written. This is in the WorkSpace Return value should be negative if anything goes wrong.

The notation of the output should be a string of arguments terminated by the number zero.

Originally we sorted in a way that the constants came last. This gave conflicts with the dollar and expression factorizations (in the expressions we wanted the zero first and then followed by the constants).

Definition at line [2058](#) of file [argument.c](#).

4.1.3 Function Documentation

4.1.3.1 `execarg()`

```
WORD execarg (
    PHEAD WORD * term,
    WORD level )
```

Definition at line [56](#) of file [argument.c](#).

4.1.3.2 `execterm()`

```
WORD execterm (
    PHEAD WORD * term,
    WORD level )
```

Definition at line [1769](#) of file [argument.c](#).

4.1.3.3 `ArgumentImplode()`

```
int ArgumentImplode (
    PHEAD WORD * term,
    WORD * thelist )
```

Definition at line [1845](#) of file [argument.c](#).

4.1.3.4 ArgumentExplode()

```
int ArgumentExplode (
    PHEAD WORD * term,
    WORD * thelist )
```

Definition at line 1949 of file [argument.c](#).

4.1.3.5 ArgFactorize()

```
int ArgFactorize (
    PHEAD WORD * argin,
    WORD * argout )
```

Definition at line 2060 of file [argument.c](#).

4.1.3.6 FindArg()

```
WORD FindArg (
    PHEAD WORD * a )
```

Looks the argument up in the (workers) table. If it is found the number in the table is returned (plus one to make it positive). If it is not found we look in the compiler provided table. If it is found - the number in the table is returned (minus one to make it negative). If in neither table we return zero.

Definition at line 2513 of file [argument.c](#).

4.1.3.7 InsertArg()

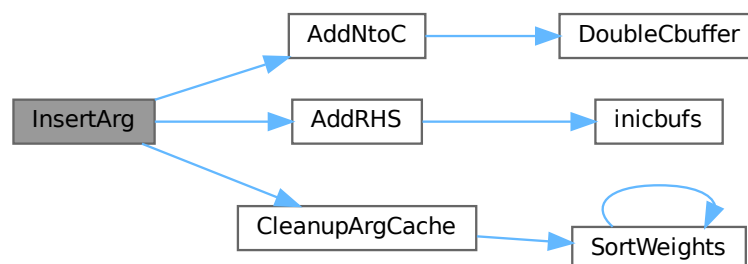
```
WORD InsertArg (
    PHEAD WORD * argin,
    WORD * argout,
    int par )
```

Inserts the argument into the (workers) table. If the table is too full we eliminate half of it. The eliminated elements are the ones that have not been used most recently, weighted by their total use and age(?). If par == 0 it inserts in the regular factorization cache If par == 1 it inserts in the cache defined with the FactorCache statement

Definition at line 2537 of file [argument.c](#).

References [AddNtoC\(\)](#), [AddRHS\(\)](#), and [CleanupArgCache\(\)](#).

Here is the call graph for this function:



4.1.3.8 CleanupArgCache()

```
int CleanupArgCache (
    PHEAD WORD bufnum )
```

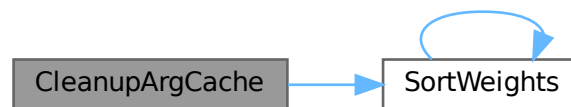
Cleans up the argument factorization cache. We throw half the elements. For a weight of what we want to keep we use the product of usage and the number in the buffer.

Definition at line 2572 of file [argument.c](#).

References [CbUf::boomlijst](#), [CbUf::Buffer](#), [CbUf::Pointer](#), [CbUf::rhs](#), [SortWeights\(\)](#), and [tree::usage](#).

Referenced by [InsertArg\(\)](#).

Here is the call graph for this function:



4.1.3.9 ArgSymbolMerge()

```
int ArgSymbolMerge (
    WORD * t1,
    WORD * t2 )
```

Definition at line 2643 of file [argument.c](#).

4.1.3.10 ArgDotproductMerge()

```
int ArgDotproductMerge (
    WORD * t1,
    WORD * t2 )
```

Definition at line 2698 of file [argument.c](#).

4.1.3.11 TakeArgContent()

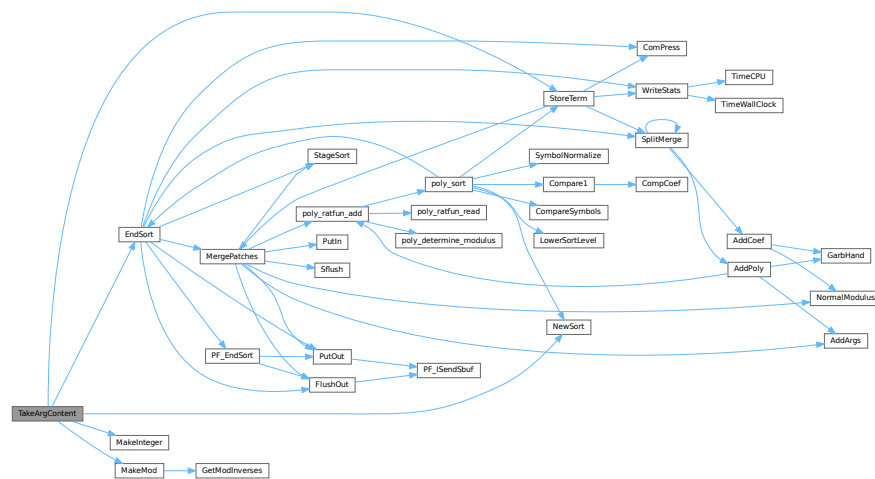
```
WORD * TakeArgContent (
    PHEAD WORD * argin,
    WORD * argout )
```

Implements part of the old ExecArg in which we take common factors from arguments with more than one term. The common pieces are put in argout as a sequence of arguments. The part with the multiple terms that are now relative prime is put in argfree which is allocated via TermMalloc and is given as the return value. The difference with the old code is that negative powers are always removed. Hence it is as in MakeInteger in which only numerators will be left: now only zero or positive powers will be remaining.

Definition at line 2766 of file [argument.c](#).

References [EndSort\(\)](#), [MakeInteger\(\)](#), [MakeMod\(\)](#), [NewSort\(\)](#), and [StoreTerm\(\)](#).

Here is the call graph for this function:



4.1.3.12 MakeInteger()

```
WORD * MakeInteger (
    PHEAD WORD * argin,
    WORD * argout,
    WORD * argfree )
```

For normalizing everything to integers we have to determine for all elements of this argument the LCM of the denominators and the GCD of the numerators. The input argument is in argin. The number that comes out should go to argout. The new pointer in the argout buffer is the return value. The normalized argument is in argfree.

Definition at line 3312 of file [argument.c](#).

Referenced by [TakeArgContent\(\)](#).

4.1.3.13 MakeMod()

```
WORD * MakeMod (
    PHEAD WORD * argin,
    WORD * argout,
    WORD * argfree )
```

Similar to MakeInteger but now with modulus arithmetic using only a one WORD 'prime'. We make the coefficient of the first term in the argument equal to one. Already the coefficients are taken modulus AN.cmod and AN.ncmod == 1

Definition at line 3483 of file [argument.c](#).

References [GetModInverses\(\)](#).

Referenced by [TakeArgContent\(\)](#).

Here is the call graph for this function:



4.1.3.14 SortWeights()

```
void SortWeights (
    LONG * weights,
    LONG * extraspace,
    WORD number )
```

Sorts an array of LONGS in the same way SplitMerge (in [sort.c](#)) works We use gradual division in two.

Definition at line 3528 of file [argument.c](#).

References [SortWeights\(\)](#).

Referenced by [CleanupArgCache\(\)](#), and [SortWeights\(\)](#).

Here is the call graph for this function:



4.2 argument.c

[Go to the documentation of this file.](#)

```

00001
00007 /* #[ License : */
00008 /*
00009 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00010 *   When using this file you are requested to refer to the publication
00011 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00012 *   This is considered a matter of courtesy as the development was paid
00013 *   for by FOM the Dutch physics granting agency and we would like to
00014 *   be able to track its scientific use to convince FOM of its value
00015 *   for the community.
00016 *
00017 *   This file is part of FORM.
00018 *
00019 *   FORM is free software: you can redistribute it and/or modify it under the
00020 *   terms of the GNU General Public License as published by the Free Software
00021 *   Foundation, either version 3 of the License, or (at your option) any later
00022 *   version.
00023 *
00024 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00025 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00026 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00027 *   details.
00028 *
00029 *   You should have received a copy of the GNU General Public License along
00030 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00031 */
00032 /* #[ License : */
00033
00034 /*
00035 *   #[ include : argument.c
00036 */
00037
00038 #include "form3.h"
00039
00040 /*
00041 *   #[ include :
00042 *   #[ execarg :
00043
00044 *   Executes the subset of statements in an argument environment.
00045 *   The calling routine should be of the type
00046 *   if ( C->lhs[level][0] == TYPEARG ) {
00047 *       if ( execarg(term,level) ) goto GenCall;
00048 *       level = C->lhs[level][2];
00049 *       goto SkipCount;
00050 *   }
00051 *   Note that there will be cases in which extra space is needed.
00052 *   In addition the compare with C->numlhs isn't very fine, because we
00053 *   need to insert a different value (C->lhs[level][2]).
00054 */
00055
00056 WORD execarg(PHEAD WORD *term, WORD level)
00057 {
00058     GETBIDENTITY
00059     WORD *t, *r, *m, *v;
00060     WORD *start, *stop, *rstop, *r1, *r2 = 0, *r3 = 0, *r4, *r5, *r6, *r7, *r8, *r9;
00061     WORD *mm, *mstop, *rnext, *rr, *factor, type, ngcd, nq;
00062     CBUF *C = cbuf+AM.rbufnum, *CC = cbuf+AT.ebufnum;
00063     WORD i, j, k, oldnumlhs = AR.Cnumlhs, count, action = 0, olddefer = AR.DeferFlag;
00064     WORD oldnumrhs = CC->numrhs, size, pow, jj;
00065     LONG oldcpointer = CC->Pointer - CC->Buffer, oldp2, oldppointer = AT.pWorkPointer, lp;
00066     WORD *oldwork = AT.WorkPointer, *oldwork2, scale, renorm;
00067     WORD kLCM = 0, kGCD = 0, kGCD2, kkLCM = 0, jLCM = 0, jGCD, sign = 1;
00068     int ii, didpolyratfun;
00069     UWORD *EAscrat, *GCDBuffer = 0, *GCDBuffer2 = 0, *LCMbuffer = 0, *LCMb = 0, *LCMc = 0;
00070     AT.WorkPointer += *term;
00071     start = C->lhs[level];
00072     AR.Cnumlhs = start[2];
00073     stop = start + start[1];
00074     type = *start;
00075     scale = start[4];
00076     renorm = start[5];
00077     start += TYPEARGHEADSIZE;
00078
00079     #[ Dollars :
00080
00081     if ( renorm && start[1] != 0 ) { /* We have to evaluate $ symbols inside () */
00082         t = start+1; factor = oldwork2 = v = AT.WorkPointer;
00083         i = *t; t++;
00084         *v++ = i+3; i--; NCOPY(v,t,i);
00085         *v++ = 1; *v++ = 1; *v++ = 3;
00086         AT.WorkPointer = v;
00087         start = t; AR.Eside = LHSIDEX;

```

```

00088     NewSort(BHEAD0);
00089     if ( Generator(BHEAD factor,AR.Cnumlhs) ) {
00090         LowerSortLevel();
00091         AT.WorkPointer = oldwork;
00092         return(-1);
00093     }
00094     AT.WorkPointer = v;
00095     if ( EndSort(BHEAD factor,0) < 0 ) {}
00096     if ( *factor && *(factor+*factor) != 0 ) {
00097         MLOCK(ErrorMessageLock);
00098         MesPrint("%$ in () does not evaluate into a single term");
00099         MUNLOCK(ErrorMessageLock);
00100         return(-1);
00101     }
00102     AR.Eside = RHSIDE;
00103     if ( *factor > 0 ) {
00104         v = factor+*factor;
00105         v -= ABS(v[-1]);
00106         *factor = v-factor;
00107     }
00108     AT.WorkPointer = v;
00109 }
00110 else {
00111     if ( *start < 0 ) {
00112         factor = start + 1;
00113         start += -*start;
00114     }
00115     else factor = 0;
00116 }
00117 /*
00118 #] Dollars :
00119 */
00120 t = term;
00121 r = t + *t;
00122 rstop = r - ABS(r[-1]);
00123 t++;
00124 /*
00125 #[ Argument detection : + argument statement
00126 */
00127 /*
00128 Allocate Numbers for MakeInteger here, for re-use in case multiple
00129 functions are treated in the same term.
00130 */
00131 if ( type == TYPENORM4 ) {
00132     GCDbuffer = NumberMalloc("execarg");
00133     GCDbuffer2 = NumberMalloc("execarg");
00134     LCMbuffer = NumberMalloc("execarg");
00135     LCMb = NumberMalloc("execarg"); LCMc = NumberMalloc("execarg");
00136 }
00137 didpolyratfun = 0;
00138 while ( t < rstop ) {
00139     if ( *t >= FUNCTION && functions[*t-FUNCTION].spec <= 0 ) {
00140 /*
00141         We have a function. First count the number of arguments.
00142         Tensors are excluded.
00143 */
00144         count = 0;
00145         v = t;
00146         m = t + FUNHEAD;
00147         r = t + t[1];
00148         while ( m < r ) {
00149             count++;
00150             NEXTARG(m)
00151         }
00152         if ( count <= 0 ) { t += t[1]; continue; }
00153 /*
00154         Now we take the arguments one by one and test for a match
00155 */
00156         for ( i = 1; i <= count; i++ ) {
00157             m = start;
00158             while ( m < stop ) {
00159                 r = m + m[1];
00160                 j = *r++;
00161                 if ( j > 1 ) {
00162                     while ( --j > 0 ) {
00163                         if ( *r == i ) goto RightNum;
00164                         r++;
00165                     }
00166                     m = r;
00167                     continue;
00168                 }
00169             }
00170             RightNum:
00171             if ( m[1] == 2 ) {
00172 #ifdef WITHFLOAT
00173                 if ( *t != FLOATFUN || AT.aux_ == 0 || TestFloat(t) == 0 )
00174                     {

```

```

00175             m += 2;
00176             m += *m;
00177             goto HaveTodo;
00178         }
00179     }
00180     else {
00181         r = m + m[1];
00182         m += 2;
00183         while ( m < r ) {
00184             if ( *m == CSET ) {
00185                 r1 = SetElements + Sets[m[1]].first;
00186                 r2 = SetElements + Sets[m[1]].last;
00187                 while ( r1 < r2 ) {
00188                     if ( *r1++ == *t ) goto HaveTodo;
00189                 }
00190             }
00191             else if ( m[1] == *t ) goto HaveTodo;
00192             m += 2;
00193         }
00194     }
00195     m += *m;
00196 }
00197 continue;
00198 HaveTodo:
00199 /*
00200     If we come here we have to do the argument i (first is 1).
00201 */
00202 sign = 1;
00203 action = 1;
00204 if ( *t == AR.PolyFun ) didpolyratfun = 1;
00205 v[2] |= DIRTYFLAG;
00206 r = t + FUNHEAD;
00207 j = i;
00208 while ( --j > 0 ) { NEXTARG(r) }
00209 if ( ( type == TYPESPLITARG ) || ( type == TYPESPLITFIRSTARG )
00210     || ( type == TYPESPLITLASTARG ) ) {
00211     if ( *t > FUNCTION && *r > 0 ) {
00212         WantAddPointers(2);
00213         AT.pWorkSpace[AT.pWorkPointer++] = t;
00214         AT.pWorkSpace[AT.pWorkPointer++] = r;
00215     }
00216     continue;
00217 }
00218 else if ( type == TYPESPLITARG2 ) {
00219     if ( *t > FUNCTION && *r > 0 ) {
00220         WantAddPointers(2);
00221         AT.pWorkSpace[AT.pWorkPointer++] = t;
00222         AT.pWorkSpace[AT.pWorkPointer++] = r;
00223     }
00224     continue;
00225 }
00226 else if ( type == TYPEFACTARG || type == TYPEFACTARG2 ) {
00227     if ( *t > FUNCTION || *t == DENOMINATOR ) {
00228         if ( *r > 0 ) {
00229             mm = r + ARGHEAD; mstop = r + *r;
00230             if ( mm + *mm < mstop ) {
00231                 WantAddPointers(2);
00232                 AT.pWorkSpace[AT.pWorkPointer++] = t;
00233                 AT.pWorkSpace[AT.pWorkPointer++] = r;
00234                 continue;
00235             }
00236             if ( *mm == 1+ABS(mstop[-1]) ) continue;
00237             if ( mstop[-3] != 1 || mstop[-2] != 1
00238                 || mstop[-1] != 3 ) {
00239                 WantAddPointers(2);
00240                 AT.pWorkSpace[AT.pWorkPointer++] = t;
00241                 AT.pWorkSpace[AT.pWorkPointer++] = r;
00242                 continue;
00243             }
00244             GETSTOP(mm,mstop); mm++;
00245             if ( mm + mm[1] < mstop ) {
00246                 WantAddPointers(2);
00247                 AT.pWorkSpace[AT.pWorkPointer++] = t;
00248                 AT.pWorkSpace[AT.pWorkPointer++] = r;
00249                 continue;
00250             }
00251             if ( *mm == SYMBOL && ( mm[1] > 4 ||
00252                 ( mm[3] != 1 && mm[3] != -1 ) ) ) {
00253                 WantAddPointers(2);
00254                 AT.pWorkSpace[AT.pWorkPointer++] = t;
00255                 AT.pWorkSpace[AT.pWorkPointer++] = r;
00256                 continue;
00257             }
00258             else if ( *mm == DOTPRODUCT && ( mm[1] > 5 ||
00259                 ( mm[4] != 1 && mm[4] != -1 ) ) ) {
00260                 WantAddPointers(2);
00261                 AT.pWorkSpace[AT.pWorkPointer++] = t;

```

```

00262         AT.pWorkSpace[AT.pWorkPointer++] = r;
00263         continue;
00264     }
00265     else if ( ( *mm == DELTA || *mm == VECTOR )
00266         && mm[1] > 4 ) {
00267         WantAddPointers(2);
00268         AT.pWorkSpace[AT.pWorkPointer++] = t;
00269         AT.pWorkSpace[AT.pWorkPointer++] = r;
00270         continue;
00271     }
00272     }
00273     else if ( factor && *factor == 4 && factor[2] == 1 ) {
00274         WantAddPointers(2);
00275         AT.pWorkSpace[AT.pWorkPointer++] = t;
00276         AT.pWorkSpace[AT.pWorkPointer++] = r;
00277         continue;
00278     }
00279     else if ( factor && *factor == 0
00280     && ( *r == -SNUMBER && r[1] != 1 ) ) {
00281         WantAddPointers(2);
00282         AT.pWorkSpace[AT.pWorkPointer++] = t;
00283         AT.pWorkSpace[AT.pWorkPointer++] = r;
00284         continue;
00285     }
00286     else if ( *r == -MINVECTOR ) {
00287         WantAddPointers(2);
00288         AT.pWorkSpace[AT.pWorkPointer++] = t;
00289         AT.pWorkSpace[AT.pWorkPointer++] = r;
00290         continue;
00291     }
00292     }
00293     continue;
00294 }
00295 else if ( type == TYPENORM || type == TYPENORM2 || type == TYPENORM3 || type ==
TYPENORM4 ) {
00296     if ( *r < 0 ) {
00297         WORD rone;
00298         if ( *r == -MINVECTOR ) { rone = -1; *r = -INDEX; }
00299         else if ( *r != -SNUMBER || r[1] == 1 || r[1] == 0 ) continue;
00300         else { rone = r[1]; r[1] = 1; }
00301     /*
00302     Now we must multiply the general coefficient by r[1]
00303     */
00304     if ( scale && ( factor == 0 || *factor ) ) {
00305         action = 1;
00306         v[2] |= DIRTYFLAG;
00307         if ( rone < 0 ) {
00308             if ( type == TYPENORM3 ) k = 1;
00309             else k = -1;
00310             rone = -rone;
00311         }
00312         else k = 1;
00313         r1 = term + *term;
00314         size = r1[-1];
00315         size = REDLENG(size);
00316         if ( scale > 0 ) {
00317             for ( jj = 0; jj < scale; jj++ ) {
00318                 if ( Mully(BHEAD (UWORD *)rstop,&size,(UWORD *)(&rone),k) )
00319                     goto execargerr;
00320             }
00321         }
00322         else {
00323             for ( jj = 0; jj > scale; jj-- ) {
00324                 if ( Divvy(BHEAD (UWORD *)rstop,&size,(UWORD *)(&rone),k) )
00325                     goto execargerr;
00326             }
00327         }
00328         size = INCLENG(size);
00329         k = size < 0 ? -size: size;
00330         rstop[k-1] = size;
00331         *term = (WORD)(rstop - term) + k;
00332     }
00333     continue;
00334 }
00335 /*
00336 Now we have to find a reference term.
00337 If factor is defined and *factor != 0 we have to
00338 look for the first term that matches the pattern exactly
00339 Otherwise the first term plays this role
00340 If its coefficient is not one,
00341 we must set up a division of the whole argument by
00342 this coefficient, and a multiplication of the term
00343 when the type is not equal to TYPENORM2.
00344 We first multiply the coefficient of the term.
00345 Then we set up the division.
00346
00347 First find the magic term

```

```

00348 */
00349
00350 /*
00351
00352     For normalizing everything to integers we have to
00353     determine for all elements of this argument the LCM of
00354     the denominators and the GCD of the numerators.
00355     The buffers have been allocated already.
00356 */
00357
00358     r4 = r + *r;
00359     r1 = r + ARGHEAD;
00360
00361     First take the first term to load up the LCM and the GCD
00362
00363     r2 = r1 + *r1;
00364     j = r2[-1];
00365     if ( j < 0 ) sign = -1;
00366     r3 = r2 - ABS(j);
00367     k = REDLENG(j);
00368     if ( k < 0 ) k = -k;
00369     while ( ( k > 1 ) && ( r3[k-1] == 0 ) ) k--;
00370     for ( kGCD = 0; kGCD < k; kGCD++ ) GCDbuffer[kGCD] = r3[kGCD];
00371     k = REDLENG(j);
00372     if ( k < 0 ) k = -k;
00373     r3 += k;
00374     while ( ( k > 1 ) && ( r3[k-1] == 0 ) ) k--;
00375     for ( kLCM = 0; kLCM < k; kLCM++ ) LCMbuffer[kLCM] = r3[kLCM];
00376     r1 = r2;
00377
00378     Now go through the rest of the terms in this argument.
00379
00380     while ( r1 < r4 ) {
00381         r2 = r1 + *r1;
00382         j = r2[-1];
00383         r3 = r2 - ABS(j);
00384         k = REDLENG(j);
00385         if ( k < 0 ) k = -k;
00386         while ( ( k > 1 ) && ( r3[k-1] == 0 ) ) k--;
00387         if ( ( ( GCDbuffer[0] == 1 ) && ( kGCD == 1 ) ) ) {
00388             GCD is already 1
00389         }
00390         else if ( ( ( k != 1 ) || ( r3[0] != 1 ) ) ) {
00391             if ( GcdLong(BHEAD GCDbuffer,kGCD, (UWORD *)r3,k,GCDbuffer2,&kGCD2) ) {
00392                 NumberFree(GCDbuffer,"execarg");
00393                 NumberFree(GCDbuffer2,"execarg");
00394                 NumberFree(LCMbuffer,"execarg");
00395                 NumberFree(LCMb,"execarg"); NumberFree(LCMc,"execarg");
00396                 goto execargerr;
00397             }
00398             kGCD = kGCD2;
00399             for ( ii = 0; ii < kGCD; ii++ ) GCDbuffer[ii] = GCDbuffer2[ii];
00400         }
00401         else {
00402             kGCD = 1; GCDbuffer[0] = 1;
00403         }
00404         k = REDLENG(j);
00405         if ( k < 0 ) k = -k;
00406         r3 += k;
00407         while ( ( k > 1 ) && ( r3[k-1] == 0 ) ) k--;
00408         if ( ( ( LCMbuffer[0] == 1 ) && ( kLCM == 1 ) ) ) {
00409             for ( kLCM = 0; kLCM < k; kLCM++ )
00410                 LCMbuffer[kLCM] = r3[kLCM];
00411         }
00412         else if ( ( ( k != 1 ) || ( r3[0] != 1 ) ) ) {
00413             if ( GcdLong(BHEAD LCMbuffer,kLCM, (UWORD *)r3,k,LCMb,&kLCM) ) {
00414                 NumberFree(GCDbuffer,"execarg"); NumberFree(GCDbuffer2,"execarg");
00415                 NumberFree(LCMbuffer,"execarg"); NumberFree(LCMb,"execarg");
00416                 NumberFree(LCMc,"execarg");
00417                 goto execargerr;
00418             }
00419             DivLong( (UWORD *)r3,k,LCMb,kLCM,LCMb,&kLCM,LCMc,&jLCM);
00420             Mullong(LCMbuffer,kLCM,LCMb,kLCM,LCMc,&jLCM);
00421             for ( kLCM = 0; kLCM < jLCM; kLCM++ )
00422                 LCMbuffer[kLCM] = LCMc[kLCM];
00423         }
00424         else {} /* LCM doesn't change */
00425         r1 = r2;
00426     }
00427
00428     Now put the factor together: GCD/LCM
00429
00430     r3 = (WORD *) (GCDbuffer);
00431     if ( kGCD == kLCM ) {
00432         for ( jGCD = 0; jGCD < kGCD; jGCD++ )
00433             r3[jGCD+kGCD] = LCMbuffer[jGCD];
00434     }
00435     k = kGCD;

```

```

00434     }
00435     else if ( kGCD > kLCM ) {
00436         for ( jGCD = 0; jGCD < kLCM; jGCD++ )
00437             r3[jGCD+kGCD] = LCMbuffer[jGCD];
00438         for ( jGCD = kLCM; jGCD < kGCD; jGCD++ )
00439             r3[jGCD+kGCD] = 0;
00440         k = kGCD;
00441     }
00442     else {
00443         for ( jGCD = kGCD; jGCD < kLCM; jGCD++ )
00444             r3[jGCD] = 0;
00445         for ( jGCD = 0; jGCD < kLCM; jGCD++ )
00446             r3[jGCD+kLCM] = LCMbuffer[jGCD];
00447         k = kLCM;
00448     }
00449
00450     j = 2*k+1;
00451 /*
00452     Now we have to correct the overall factor
00453 */
00454     if ( scale && ( factor == 0 || *factor > 0 ) )
00455         goto ScaledVariety;
00456 /*
00457     The if was added 28-nov-2012 to give MakeInteger also
00458     the (0) option.
00459 */
00460     if ( scale && ( factor == 0 || *factor ) ) {
00461         size = term[*term-1];
00462         size = REDLENG(size);
00463         if ( MulRat(BHEAD (UWORD *)rstop,size,(UWORD *)r3,k,
00464             (UWORD *)rstop,&size) ) goto execargerr;
00465         size = INCLENG(size);
00466         k = size < 0 ? -size: size;
00467         rstop[k-1] = size*sign;
00468         *term = (WORD)(rstop - term) + k;
00469     }
00470 }
00471 else {
00472     if ( factor && *factor >= 1 ) {
00473         r4 = r + *r;
00474         r1 = r + ARGHEAD;
00475         while ( r1 < r4 ) {
00476             r2 = r1 + *r1;
00477             r3 = r2 - ABS(r2[-1]);
00478             j = r3 - r1;
00479             r5 = factor;
00480             if ( j != *r5 ) { r1 = r2; continue; }
00481             r5++; r6 = r1+1;
00482             while ( --j > 0 ) {
00483                 if ( *r5 != *r6 ) break;
00484                 r5++; r6++;
00485             }
00486             if ( j > 0 ) { r1 = r2; continue; }
00487             break;
00488         }
00489         if ( r1 >= r4 ) continue;
00490     }
00491     else {
00492         r1 = r + ARGHEAD;
00493         r2 = r1 + *r1;
00494         r3 = r2 - ABS(r2[-1]);
00495     }
00496     if ( *r3 == 1 && r3[1] == 1 ) {
00497         if ( r2[-1] == 3 ) continue;
00498         if ( r2[-1] == -3 && type == TYPENORM3 ) continue;
00499     }
00500     action = 1;
00501     v[2] |= DIRTYFLAG;
00502     j = r2[-1];
00503     k = REDLENG(j);
00504     if ( j < 0 ) j = -j;
00505     if ( type == TYPENORM && scale && ( factor == 0 || *factor ) ) {
00506 /*
00507         Now we correct the overall factor
00508 */
00509         ScaledVariety;
00510         size = term[*term-1];
00511         size = REDLENG(size);
00512         if ( scale > 0 ) {
00513             for ( jj = 0; jj < scale; jj++ ) {
00514                 if ( MulRat(BHEAD (UWORD *)rstop,size,(UWORD *)r3,k,
00515                     (UWORD *)rstop,&size) ) goto execargerr;
00516             }
00517         }
00518         else {
00519             for ( jj = 0; jj > scale; jj-- ) {
00520                 if ( DivRat(BHEAD (UWORD *)rstop,size,(UWORD *)r3,k,

```

```

00521                                     (UWORD *)rstop,&size) ) goto execargerr;
00522                                     }
00523                                     }
00524                                     size = INCLENG(size);
00525                                     k = size < 0 ? -size: size;
00526                                     rstop[k-1] = size*sign;
00527                                     *term = (WORD)(rstop - term) + k;
00528                                     }
00529                                     }
00530 /*
00531 We generate a statement for adapting all terms in the
00532 argument successively
00533 */
00534 r4 = AddRHS(AT.ebufnum,1);
00535 while ( (r4+j+12) > CC->Top ) r4 = DoubleCbuffer(AT.ebufnum,r4,3);
00536 *r4++ = j+1;
00537 i = (j-1)/2; /* was (j-1)*2 ????? 17-oct-2017 */
00538 for ( k = 0; k < i; k++ ) *r4++ = r3[i+k];
00539 for ( k = 0; k < i; k++ ) *r4++ = r3[k];
00540 if ( ( type == TYPENORM3 ) || ( type == TYPENORM4 ) ) *r4++ = j*sign;
00541 else *r4++ = r3[j-1];
00542 *r4++ = 0;
00543 CC->rhs[CC->numrhs+1] = r4;
00544 CC->Pointer = r4;
00545 AT.mulpat[5] = CC->numrhs;
00546 AT.mulpat[7] = AT.ebufnum;
00547 }
00548 else if ( type == TYPEARGTOEXTRASymbol ) {
00549 WORD n;
00550 if ( r[0] < 0 ) {
00551 /* The argument is in the fast notation. */
00552 WORD tmp[Max(9,FUNHEAD+5)];
00553 switch ( r[0] ) {
00554 case -SNUMBER:
00555     if ( r[1] == 0 ) {
00556         tmp[0] = 0;
00557     }
00558     else {
00559         tmp[0] = 4;
00560         tmp[1] = ABS(r[1]);
00561         tmp[2] = 1;
00562         tmp[3] = r[1] > 0 ? 3 : -3;
00563         tmp[4] = 0;
00564     }
00565     break;
00566 case -SYMBOL:
00567     tmp[0] = 8;
00568     tmp[1] = SYMBOL;
00569     tmp[2] = 4;
00570     tmp[3] = r[1];
00571     tmp[4] = 1;
00572     tmp[5] = 1;
00573     tmp[6] = 1;
00574     tmp[7] = 3;
00575     tmp[8] = 0;
00576     break;
00577 case -INDEX:
00578 case -VECTOR:
00579 case -MINVECTOR:
00580     tmp[0] = 7;
00581     tmp[1] = INDEX;
00582     tmp[2] = 3;
00583     tmp[3] = r[1];
00584     tmp[4] = 1;
00585     tmp[5] = 1;
00586     tmp[6] = r[0] != -MINVECTOR ? 3 : -3;
00587     tmp[7] = 0;
00588     break;
00589 default:
00590     if ( r[0] <= -FUNCTION ) {
00591         tmp[0] = FUNHEAD+4;
00592         tmp[1] = -r[0];
00593         tmp[2] = FUNHEAD;
00594         ZeroFillRange(tmp,3,1+FUNHEAD);
00595         tmp[FUNHEAD+1] = 1;
00596         tmp[FUNHEAD+2] = 1;
00597         tmp[FUNHEAD+3] = 3;
00598         tmp[FUNHEAD+4] = 0;
00599         break;
00600     }
00601     else {
00602         MLOCK(ErrorMessageLock);
00603         MesPrint("Unknown fast notation found (TYPEARGTOEXTRASymbol)");
00604         MUNLOCK(ErrorMessageLock);
00605         return(-1);
00606     }
00607 }

```

```

00608         n = FindSubexpression(tmp);
00609     }
00610     else {
00611         /*
00612          * NOTE: writing to r[r[0]] is legal. As long as we work
00613          * in a part of the term, at least the coefficient of
00614          * the term must follow.
00615          */
00616         WORD old_rr0 = r[r[0]];
00617         r[r[0]] = 0; /* zero-terminated */
00618         n = FindSubexpression(r+ARGHEAD);
00619         r[r[0]] = old_rr0;
00620     }
00621     /* Put the new argument in the work space. */
00622     if ( AT.WorkPointer+2 > AT.WorkTop ) {
00623         MLOCK(ErrorMessageLock);
00624         MesWork();
00625         MUNLOCK(ErrorMessageLock);
00626         return(-1);
00627     }
00628     r1 = AT.WorkPointer;
00629     if ( scale ) { /* means "tonumber" */
00630         r1[0] = -SNUMBER;
00631         r1[1] = n;
00632     }
00633     else {
00634         r1[0] = -SYMBOL;
00635         r1[1] = MAXVARIABLES-n;
00636     }
00637     /* We need r2, r3, m and k to shift the data. */
00638     r2 = r + (r[0] > 0 ? r[0] : r[0] <= -FUNCTION ? 1 : 2);
00639     r3 = r;
00640     m = r1+ARGHEAD+2;
00641     k = 2;
00642     goto do_shift;
00643 }
00644 r3 = r;
00645 AR.DeferFlag = 0;
00646 if ( *r > 0 ) {
00647     NewSort(BHEAD0);
00648     action = 1;
00649     r2 = r + *r;
00650     r += ARGHEAD;
00651     while ( r < r2 ) { /* Sum over the terms */
00652         m = AT.WorkPointer;
00653         j = *r;
00654         while ( --j >= 0 ) *m++ = *r++;
00655         r1 = AT.WorkPointer;
00656         AT.WorkPointer = m;
00657     /*
00658      * What to do with dummy indices?
00659      */
00660     if ( type == TYPENORM || type == TYPENORM2 || type == TYPENORM3 || type ==
TYPENORM4 ) {
00661         if ( MultDo(BHEAD r1,AT.mulpat) ) goto execargerr;
00662         AT.WorkPointer = r1 + *r1;
00663     }
00664     if ( Generator(BHEAD r1,level) ) goto execargerr;
00665     AT.WorkPointer = r1;
00666 }
00667 }
00668 else {
00669     r2 = r + (( *r <= -FUNCTION ) ? 1:2);
00670     r1 = AT.WorkPointer;
00671     ToGeneral(r,r1,0);
00672     m = r1 + ARGHEAD;
00673     AT.WorkPointer = r1 + *r1;
00674     NewSort(BHEAD0);
00675     action = 1;
00676 /*
00677  * What to do with dummy indices?
00678  */
00679     if ( type == TYPENORM || type == TYPENORM2 || type == TYPENORM3 || type ==
TYPENORM4 ) {
00680         if ( MultDo(BHEAD m,AT.mulpat) ) goto execargerr;
00681         AT.WorkPointer = m + *m;
00682     }
00683     if ( (*m != 0) && Generator(BHEAD m,level) ) goto execargerr;
00684     AT.WorkPointer = r1;
00685 }
00686 if ( EndSort(BHEAD AT.WorkPointer+ARGHEAD,1) < 0 ) goto execargerr;
00687 AR.DeferFlag = olddefer;
00688 /*
00689  * Now shift the sorted entity over the old argument.
00690  */
00691 m = AT.WorkPointer+ARGHEAD;
00692 while ( *m ) m += *m;

```



```

00693         k = WORDDIF(m,AT.WorkPointer);
00694         *AT.WorkPointer = k;
00695         AT.WorkPointer[1] = 0;
00696         if ( ToFast(AT.WorkPointer,AT.WorkPointer) ) {
00697             if ( *AT.WorkPointer <= -FUNCTION ) k = 1;
00698             else k = 2;
00699         }
00700     do_shift:
00701         if ( *r3 > 0 ) j = k - *r3;
00702         else if ( *r3 <= -FUNCTION ) j = k - 1;
00703         else j = k - 2;
00704
00705         t[1] += j;
00706         action = 1;
00707         v[2] |= DIRTYFLAG;
00708         if ( j > 0 ) {
00709             r = m + j;
00710             while ( m > AT.WorkPointer ) *--r = *--m;
00711             AT.WorkPointer = r;
00712             m = term + *term;
00713             r = m + j;
00714             while ( m > r2 ) *--r = *--m;
00715         }
00716         else if ( j < 0 ) {
00717             r = r2 + j;
00718             r1 = term + *term;
00719             while ( r2 < r1 ) *r++ = *r2++;
00720         }
00721         r = r3;
00722         m = AT.WorkPointer;
00723         NCOPY(r,m,k);
00724         *term += j;
00725         rstop += j;
00726         CC->numrhs = oldnumrhs;
00727         CC->Pointer = CC->Buffer + oldcpointer;
00728     }
00729 }
00730 t += t[1];
00731 }
00732 /*
00733     If TYPENORM4, we allocated Number buffers before the above while loop. Free them.
00734 */
00735 if ( type == TYPENORM4 ) {
00736     NumberFree(GCDBuffer,"execarg");
00737     NumberFree(GCDBuffer2,"execarg");
00738     NumberFree(LCMBuffer,"execarg");
00739     NumberFree(LCMb,"execarg"); NumberFree(LCMc,"execarg");
00740 }
00741 if ( didpolyratfun ) {
00742     PolyFunDirty(BHEAD term);
00743     didpolyratfun = 0;
00744 }
00745 /*
00746 #] Argument detection :
00747 #[ SplitArg : + varieties
00748 */
00749 if ( ( type == TYPESPLITARG || type == TYPESPLITARG2
00750     || type == TYPESPLITFIRSTARG || type == TYPESPLITLASTARG ) &&
00751     AT.pWorkPointer > oldppointer ) {
00752     t = term+1;
00753     r1 = AT.WorkPointer + 1;
00754     lp = oldppointer;
00755     while ( t < rstop ) {
00756         if ( lp < AT.pWorkPointer && t == AT.pWorkspace[lp] ) {
00757             v = t;
00758             m = t + FUNHEAD;
00759             r = t + t[1];
00760             r2 = r1; while ( t < m ) *r1++ = *t++;
00761             while ( m < r ) {
00762                 t = m;
00763                 NEXTARG(m)
00764                 if ( lp >= AT.pWorkPointer || t != AT.pWorkspace[lp+1] ) {
00765                     if ( *t > 0 ) t[1] = 0;
00766                     while ( t < m ) *r1++ = *t++;
00767                     continue;
00768                 }
00769             }
00770             /*
00771             Now we have a nontrivial argument that should be done.
00772             */
00773             lp += 2;
00774             action = 1;
00775             v[2] |= DIRTYFLAG;
00776             r3 = t + *t;
00777             t += ARGHEAD;
00778             if ( type == TYPESPLITFIRSTARG ) {
00779                 r4 = r1; r5 = t; r7 = oldwork;
00780                 *r1++ = *t + ARGHEAD;

```

```

00780         for ( i = 1; i < ARGHEAD; i++ ) *r1++ = 0;
00781         j = 0;
00782         while ( t < r3 ) {
00783             i = *t;
00784             if ( j == 0 ) {
00785                 NCOPY(r7,t,i)
00786                 j++;
00787             }
00788             else {
00789                 NCOPY(r1,t,i)
00790             }
00791         }
00792         *r4 = r1 - r4;
00793         if ( j ) {
00794             if ( ToFast(r4,r4) ) {
00795                 r1 = r4;
00796                 if ( *r1 > -FUNCTION ) r1++;
00797                 r1++;
00798             }
00799             r7 = oldwork;
00800             while ( --j >= 0 ) {
00801                 r4 = r1; i = *r7;
00802                 *r1++ = i+ARGHEAD; *r1++ = 0;
00803                 FILLARG(r1);
00804                 NCOPY(r1,r7,i)
00805                 if ( ToFast(r4,r4) ) {
00806                     r1 = r4;
00807                     if ( *r1 > -FUNCTION ) r1++;
00808                     r1++;
00809                 }
00810             }
00811         }
00812         t = r3;
00813     }
00814     else if ( type == TYPESPLITLASTARG ) {
00815         r4 = r1; r5 = t; r7 = oldwork;
00816         *r1++ = *t + ARGHEAD;
00817         for ( i = 1; i < ARGHEAD; i++ ) *r1++ = 0;
00818         j = 0;
00819         while ( t < r3 ) {
00820             i = *t;
00821             if ( t+i >= r3 ) {
00822                 NCOPY(r7,t,i)
00823                 j++;
00824             }
00825             else {
00826                 NCOPY(r1,t,i)
00827             }
00828         }
00829         *r4 = r1 - r4;
00830         if ( j ) {
00831             if ( ToFast(r4,r4) ) {
00832                 r1 = r4;
00833                 if ( *r1 > -FUNCTION ) r1++;
00834                 r1++;
00835             }
00836             r7 = oldwork;
00837             while ( --j >= 0 ) {
00838                 r4 = r1; i = *r7;
00839                 *r1++ = i+ARGHEAD; *r1++ = 0;
00840                 FILLARG(r1);
00841                 NCOPY(r1,r7,i)
00842                 if ( ToFast(r4,r4) ) {
00843                     r1 = r4;
00844                     if ( *r1 > -FUNCTION ) r1++;
00845                     r1++;
00846                 }
00847             }
00848         }
00849         t = r3;
00850     }
00851     else if ( factor == 0 || ( type == TYPESPLITARG2 && *factor == 0 ) ) {
00852         while ( t < r3 ) {
00853             r4 = r1;
00854             *r1++ = *t + ARGHEAD;
00855             for ( i = 1; i < ARGHEAD; i++ ) *r1++ = 0;
00856             i = *t;
00857             while ( --i >= 0 ) *r1++ = *t++;
00858             if ( ToFast(r4,r4) ) {
00859                 r1 = r4;
00860                 if ( *r1 > -FUNCTION ) r1++;
00861                 r1++;
00862             }
00863         }
00864     }
00865     else if ( type == TYPESPLITARG2 ) {
00866         /*

```

```

00867                                     Here we better put the pattern matcher at work?
00868                                     Remember: there are no wildcards.
00869  */
00870
00871 WORD *oRepFunList = AN.RepFunList;
00872 WORD *oWildMask = AT.WildMask, *oWildValue = AN.WildValue;
00873 AN.WildValue = AT.locwildvalue; AT.WildMask = AT.locwildvalue+2;
00874 AN.NumWild = 0;
00875 r4 = r1; r5 = t; r7 = oldwork;
00876 *r1++ = *t + ARGHEAD;
00877 for ( i = 1; i < ARGHEAD; i++ ) *r1++ = 0;
00878 j = 0;
00879 while ( t < r3 ) {
00880     AN.UseFindOnly = 0; oldwork2 = AT.WorkPointer;
00881     AN.RepFunList = r1;
00882     AT.WorkPointer = r1+AN.RepFunNum+2;
00883     i = *t;
00884     if ( FindRest(BHEAD t,factor) &&
00885         ( AN.UsedOtherFind || FindOnce(BHEAD t,factor) ) ) {
00886         NCOPY(r7,t,i)
00887         j++;
00888     }
00889     else if ( factor[0] == FUNHEAD+1 && factor[1] >= FUNCTION ) {
00890         WORD *rr1 = t+1, *rr2 = t+i;
00891         rr2 -= ABS(rr2[-1]);
00892         while ( rr1 < rr2 ) {
00893             if ( *rr1 == factor[1] ) break;
00894             rr1 += rr1[1];
00895         }
00896         if ( rr1 < rr2 ) {
00897             NCOPY(r7,t,i)
00898             j++;
00899         }
00900         else {
00901             NCOPY(r1,t,i)
00902         }
00903     }
00904     else {
00905         NCOPY(r1,t,i)
00906     }
00907     AT.WorkPointer = oldwork2;
00908 }
00909 AN.RepFunList = oRepFunList;
00910 *r4 = r1 - r4;
00911 if ( j ) {
00912     if ( ToFast(r4,r4) ) {
00913         r1 = r4;
00914         if ( *r1 > -FUNCTION ) r1++;
00915         r1++;
00916     }
00917     r7 = oldwork;
00918     while ( --j >= 0 ) {
00919         r4 = r1; i = *r7;
00920         *r1++ = i+ARGHEAD; *r1++ = 0;
00921         FILLARG(r1);
00922         NCOPY(r1,r7,i)
00923         if ( ToFast(r4,r4) ) {
00924             r1 = r4;
00925             if ( *r1 > -FUNCTION ) r1++;
00926             r1++;
00927         }
00928     }
00929 }
00930 t = r3;
00931 AT.WildMask = oWildMask; AN.WildValue = oWildValue;
00932 }
00933 else {
00934     /*
00935     This code deals with splitting off a single term
00936     */
00937     r4 = r1; r5 = t;
00938     *r1++ = *t + ARGHEAD;
00939     for ( i = 1; i < ARGHEAD; i++ ) *r1++ = 0;
00940     j = 0;
00941     while ( t < r3 ) {
00942         r6 = t + *t; r6 -= ABS(r6[-1]);
00943         if ( (r6 - t) == *factor ) {
00944             k = *factor - 1;
00945             for ( ; k > 0; k-- ) {
00946                 if ( t[k] != factor[k] ) break;
00947             }
00948             if ( k <= 0 ) {
00949                 j = r3 - t; t += *t; continue;
00950             }
00951         }
00952         else if ( (r6 - t) == 1 && *factor == 0 ) {
00953             j = r3 - t; t += *t; continue;
00954         }
00955     }

```

```

00954             i = *t;
00955             NCOPY(r1,t,i)
00956         }
00957         *r4 = r1 - r4;
00958         if ( j ) {
00959             if ( ToFast(r4,r4) ) {
00960                 r1 = r4;
00961                 if ( *r1 > -FUNCTION ) r1++;
00962                 r1++;
00963             }
00964             t = r3 - j;
00965             r4 = r1;
00966             *r1++ = *t + ARGHEAD;
00967             for ( i = 1; i < ARGHEAD; i++ ) *r1++ = 0;
00968             i = *t;
00969             while ( --i >= 0 ) *r1++ = *t++;
00970             if ( ToFast(r4,r4) ) {
00971                 r1 = r4;
00972                 if ( *r1 > -FUNCTION ) r1++;
00973                 r1++;
00974             }
00975         }
00976         t = r3;
00977     }
00978 }
00979     r2[1] = r1 - r2;
00980 }
00981 else {
00982     r = t + t[1];
00983     while ( t < r ) *r1++ = *t++;
00984 }
00985 }
00986 r = term + *term;
00987 while ( t < r ) *r1++ = *t++;
00988 m = AT.WorkPointer;
00989 i = m[0] = r1 - m;
00990 t = term;
00991 while ( --i >= 0 ) *t++ = *m++;
00992 if ( AT.WorkPointer < m ) AT.WorkPointer = m;
00993 }
00994 /*
00995 #] SplitArg :
00996 #[ FACTARG :
00997 */
00998 if ( ( type == TYPEFACTARG || type == TYPEFACTARG2 ) &&
00999     AT.pWorkPointer > oldppointer ) {
01000     t = term+1;
01001     r1 = AT.WorkPointer + 1;
01002     lp = oldppointer;
01003     while ( t < rstop ) {
01004         if ( lp < AT.pWorkPointer && AT.pWorkspace[lp] == t ) {
01005             v = t;
01006             m = t + FUNHEAD;
01007             r = t + t[1];
01008             r2 = r1; while ( t < m ) *r1++ = *t++;
01009             while ( m < r ) {
01010                 rr = t = m;
01011                 NEXTARG(m)
01012                 if ( lp >= AT.pWorkPointer || AT.pWorkspace[lp+1] != t ) {
01013                     if ( *t > 0 ) t[1] = 0;
01014                     while ( t < m ) *r1++ = *t++;
01015                     continue;
01016                 }
01017             }
01018             /*
01019             Now we have a nontrivial argument that should be studied.
01020             Try to find common factors.
01021             */
01022             lp += 2;
01023             if ( *t < 0 ) {
01024                 if ( factor && ( *factor == 0 && *t == -SNUMBER ) ) {
01025                     *r1++ = *t++;
01026                     if ( *t == 0 ) *r1++ = *t++;
01027                     else { *r1++ = 1; t++; }
01028                     continue;
01029                 }
01030                 else if ( factor && *factor == 4 && factor[2] == 1 ) {
01031                     if ( *t == -SNUMBER ) {
01032                         if ( factor[3] < 0 || t[1] >= 0 ) {
01033                             while ( t < m ) *r1++ = *t++;
01034                         }
01035                         else {
01036                             *r1++ = -SNUMBER; *r1++ = -1;
01037                             *r1++ = *t++; *r1++ = -*t++;
01038                         }
01039                     }
01040                     else {
01041                         while ( t < m ) *r1++ = *t++;
01042                     }
01043                 }
01044             }
01045         }
01046     }

```

```

01041             *r1++ = -SNUMBER; *r1++ = 1;
01042         }
01043         continue;
01044     }
01045     else if ( *t == -MINVECTOR ) {
01046         if ( AC.OldFactArgFlag == NEWFACTARG ) {
01047             *r1++ = -SNUMBER; *r1++ = -1;
01048             *r1++ = -VECTOR; t++; *r1++ = *t++;
01049         }
01050         else {
01051             *r1++ = -VECTOR; t++; *r1++ = *t++;
01052             *r1++ = -SNUMBER; *r1++ = -1;
01053             *r1++ = -SNUMBER; *r1++ = 1;
01054         }
01055         continue;
01056     }
01057 }
01058 /*
01059 Now we have a nontrivial argument
01060 */
01061 r3 = t + *t;
01062 t += ARGHEAD; r5 = t; /* Store starting point */
01063 /* We have terms from r5 to r3 */
01064 if ( r5+r5 == r3 && factor ) { /* One term only */
01065     if ( *factor == 0 ) {
01066         GETSTOP(t,r6);
01067         r9 = r1; *r1++ = 0; *r1++ = 1;
01068         FILLARG(r1);
01069         *r1++ = (r6-t)+3; t++;
01070         while ( t < r6 ) *r1++ = *t++;
01071         *r1++ = 1; *r1++ = 1; *r1++ = 3;
01072         *r9 = r1-r9;
01073         if ( ToFast(r9,r9) ) {
01074             if ( *r9 <= -FUNCTION ) r1 = r9+1;
01075             else r1 = r9+2;
01076         }
01077         t = r3; continue;
01078     }
01079     if ( factor[0] == 4 && factor[2] == 1 ) {
01080         GETSTOP(t,r6);
01081         r7 = r1; *r1++ = (r6-t)+3+ARGHEAD; *r1++ = 0;
01082         FILLARG(r1);
01083         *r1++ = (r6-t)+3; t++;
01084         while ( t < r6 ) *r1++ = *t++;
01085         *r1++ = 1; *r1++ = 1; *r1++ = 3;
01086         if ( ToFast(r7,r7) ) {
01087             if ( *r7 <= -FUNCTION ) r1 = r7+1;
01088             else r1 = r7+2;
01089         }
01090         if ( r3[-1] < 0 && factor[3] > 0 ) {
01091             *r1++ = -SNUMBER; *r1++ = -1;
01092             if ( r3[-1] == -3 && r3[-2] == 1
01093                 && ( r3[-3] & MAXPOSITIVE ) == r3[-3] ) {
01094                 *r1++ = -SNUMBER; *r1++ = r3[-3];
01095             }
01096             else {
01097                 *r1++ = (r3-r6)+1+ARGHEAD;
01098                 *r1++ = 0;
01099                 FILLARG(r1);
01100                 *r1++ = (r3-r6+1);
01101                 while ( t < r3 ) *r1++ = *t++;
01102                 r1[-1] = -r1[-1];
01103             }
01104         }
01105         else {
01106             if ( ( r3[-1] == -3 || r3[-1] == 3 )
01107                 && r3[-2] == 1
01108                 && ( r3[-3] & MAXPOSITIVE ) == r3[-3] ) {
01109                 *r1++ = -SNUMBER; *r1++ = r3[-3];
01110                 if ( r3[-1] < 0 ) r1[-1] = - r1[-1];
01111             }
01112             else {
01113                 *r1++ = (r3-r6)+1+ARGHEAD;
01114                 *r1++ = 0;
01115                 FILLARG(r1);
01116                 *r1++ = (r3-r6+1);
01117                 while ( t < r3 ) *r1++ = *t++;
01118             }
01119         }
01120         t = r3; continue;
01121     }
01122 }
01123 /*
01124 Now we take the first term and look for its pieces
01125 inside the other terms.
01126
01127 It is at this point that a more general factorization

```

```

01128 routine could take over (allowing for writing the output
01129 properly of course).
01130 */
01131 if ( AC.OldFactArgFlag == NEWFACTARG ) {
01132 if ( factor == 0 ) {
01133     WORD *oldworkpointer2 = AT.WorkPointer;
01134     AT.WorkPointer = r1 + AM.MaxTer+FUNHEAD;
01135     if ( ArgFactorize(BHEAD t-ARGHEAD,r1) < 0 ) {
01136         MesCall("ExecArg");
01137         return(-1);
01138     }
01139     AT.WorkPointer = oldworkpointer2;
01140     t = r3;
01141     while ( *r1 ) { NEXTARG(r1) }
01142 }
01143 else {
01144     rnext = t + *t;
01145     GETSTOP(t,r6);
01146     t++;
01147     t = r5; pow = 1;
01148     while ( t < r3 ) {
01149         t += *t; if ( t[-1] > 0 ) { pow = 0; break; }
01150     }
01151 }
01152 /*
01153 We have to add here the code for computing the GCD
01154 and to divide it out.
01155
01156 # Numerical factor :
01157 */
01158 t = r5;
01159 EAscrat = (UWORD *) (TermMalloc("execarg"));
01160 if ( t + *t == r3 ) {
01161     if ( factor == 0 || *factor > 2 ) {
01162         if ( pow > 0 ) {
01163             *r1++ = -SNUMBER; *r1++ = -1;
01164             t = r5;
01165             while ( t < r3 ) {
01166                 t += *t; t[-1] = -t[-1];
01167             }
01168             t = rr; *r1++ = *t++; *r1++ = 1; t++;
01169             COPYARG(r1,t);
01170             while ( t < m ) *r1++ = *t++;
01171         }
01172     }
01173     else {
01174         GETSTOP(t,r6);
01175         ngcd = t[t[0]-1];
01176         i = abs(ngcd)-1;
01177         while ( --i >= 0 ) EAscrat[i] = r6[i];
01178         t += *t;
01179         while ( t < r3 ) {
01180             GETSTOP(t,r6);
01181             i = t[t[0]-1];
01182             if ( AccumGCD(BHEAD EAscrat,&ngcd, (UWORD *)r6,i) ) goto execargerr;
01183             if ( ngcd == 3 && EAscrat[0] == 1 && EAscrat[1] == 1 ) break;
01184             t += *t;
01185         }
01186     }
01187     if ( ngcd != 3 || EAscrat[0] != 1 || EAscrat[1] != 1 )
01188     {
01189         if ( pow ) ngcd = -ngcd;
01190         t = r5; r9 = r1; *r1++ = t[-ARGHEAD]; *r1++ = 1;
01191         FILLARG(r1); ngcd = REDLENG(ngcd);
01192         while ( t < r3 ) {
01193             GETSTOP(t,r6);
01194             r7 = t; r8 = r1;
01195             while ( r7 < r6 ) *r1++ = *r7++;
01196             t += *t;
01197             i = REDLENG(t[-1]);
01198             if ( DivRat(BHEAD (UWORD *)r6,i,EAscrat,ngcd, (UWORD *)r1,&nq) ) goto
01199 execargerr;
01200             nq = INCLENG(nq);
01201             i = ABS(nq)-1;
01202             r1 += i; *r1++ = nq; *r8 = r1-r8;
01203         }
01204         *r9 = r1-r9;
01205         ngcd = INCLENG(ngcd);
01206         i = ABS(ngcd)-1;
01207         if ( factor && *factor == 0 ) {}
01208         else if ( ( factor && factor[0] == 4 && factor[2] == 1
01209 && factor[3] == -3 ) || pow == 0 ) {
01210             r9 = r1; *r1++ = ARGHEAD+2+i; *r1++ = 0;
01211             FILLARG(r1); *r1++ = i+2;
01212             for ( j = 0; j < i; j++ ) *r1++ = EAscrat[j];
01213             *r1++ = ngcd;

```

```

01214         if ( ToFast(r9,r9) ) r1 = r9+2;
01215     }
01216     else if ( factor && factor[0] == 4 && factor[2] == 1
01217 && factor[3] > 0 && pow ) {
01218         if ( ngcd < 0 ) ngcd = -ngcd;
01219         *r1++ = -SNUMBER; *r1++ = -1;
01220         r9 = r1; *r1++ = ARGHEAD+2+i; *r1++ = 0;
01221         FILLARG(r1); *r1++ = i+2;
01222         for ( j = 0; j < i; j++ ) *r1++ = EAscrat[j];
01223         *r1++ = ngcd;
01224         if ( ToFast(r9,r9) ) r1 = r9+2;
01225     }
01226     else {
01227         if ( ngcd < 0 ) ngcd = -ngcd;
01228         if ( pow ) { *r1++ = -SNUMBER; *r1++ = -1; }
01229         if ( ngcd != 3 || EAscrat[0] != 1 || EAscrat[1] != 1 ) {
01230             r9 = r1; *r1++ = ARGHEAD+2+i; *r1++ = 0;
01231             FILLARG(r1); *r1++ = i+2;
01232             for ( j = 0; j < i; j++ ) *r1++ = EAscrat[j];
01233             *r1++ = ngcd;
01234             if ( ToFast(r9,r9) ) r1 = r9+2;
01235         }
01236     }
01237 }
01238 /*
01239     #] Numerical factor :
01240     else {
01241 onetermnew;;
01242
01243         if ( factor == 0 || *factor > 2 ) {
01244             if ( pow > 0 ) {
01245                 *r1++ = -SNUMBER; *r1++ = -1;
01246                 t = r5;
01247                 while ( t < r3 ) {
01248                     t += *t; t[-1] = -t[-1];
01249                 }
01250             }
01251             t = rr; *r1++ = *t++; *r1++ = 1; t++;
01252             COPYARG(r1,t);
01253             while ( t < m ) *r1++ = *t++;
01254         }
01255     }
01256 onetermnew;;
01257 */
01258     }
01259     TermFree(EAscrat,"execarg");
01260 }
01261 }
01262 else { /* AC.OldFactArgFlag is ON */
01263 {
01264     WORD *mnext, ncom;
01265     rnext = t + *t;
01266     GETSTOP(t,r6);
01267     t++;
01268     if ( factor == 0 ) {
01269         while ( t < r6 ) {
01270 /*
01271     #[ SYMBOL :
01272 */
01273         if ( *t == SYMBOL ) {
01274             r7 = t; r8 = t + t[1]; t += 2;
01275             while ( t < r8 ) {
01276                 pow = t[1];
01277                 mm = rnext;
01278                 while ( mm < r3 ) {
01279                     mnext = mm + *mm;
01280                     GETSTOP(mm,mstop); mm++;
01281                     while ( mm < mstop ) {
01282                         if ( *mm != SYMBOL ) mm += mm[1];
01283                         else break;
01284                     }
01285                     if ( *mm == SYMBOL ) {
01286                         mstop = mm + mm[1]; mm += 2;
01287                         while ( *mm != *t && mm < mstop ) mm += 2;
01288                         if ( mm >= mstop ) pow = 0;
01289                         else if ( pow > 0 && mm[1] > 0 ) {
01290                             if ( mm[1] < pow ) pow = mm[1];
01291                         }
01292                         else if ( pow < 0 && mm[1] < 0 ) {
01293                             if ( mm[1] > pow ) pow = mm[1];
01294                         }
01295                         else pow = 0;
01296                     }
01297                     else pow = 0;
01298                     if ( pow == 0 ) break;
01299                     mm = mnext;
01300                 }

```

```

01301         if ( pow == 0 ) { t += 2; continue; }
01302     /*
01303     We have a factor
01304     */
01305     action = 1; i = pow;
01306     if ( i > 0 ) {
01307         while ( --i >= 0 ) {
01308             *r1++ = -SYMBOL;
01309             *r1++ = *t;
01310         }
01311     }
01312     else {
01313         while ( i++ < 0 ) {
01314             *r1++ = 8 + ARGHEAD;
01315             for ( j = 1; j < ARGHEAD; j++ ) *r1++ = 0;
01316             *r1++ = 8; *r1++ = SYMBOL;
01317             *r1++ = 4; *r1++ = *t; *r1++ = -1;
01318             *r1++ = 1; *r1++ = 1; *r1++ = 3;
01319         }
01320     }
01321     /*
01322     Now we have to remove the symbols
01323     */
01324     t[1] -= pow;
01325     mm = rnext;
01326     while ( mm < r3 ) {
01327         mnext = mm + *mm;
01328         GETSTOP(mm,mstop); mm++;
01329         while ( mm < mstop ) {
01330             if ( *mm != SYMBOL ) mm += mm[1];
01331             else break;
01332         }
01333         mstop = mm + mm[1]; mm += 2;
01334         while ( mm < mstop && *mm != *t ) mm += 2;
01335         mm[1] -= pow;
01336         mm = mnext;
01337     }
01338     t += 2;
01339 }
01340
01341 /*
01342 #] SYMBOL :
01343 #[ DOTPRODUCT :
01344 */
01345 else if ( *t == DOTPRODUCT ) {
01346     r7 = t; r8 = t + t[1]; t += 2;
01347     while ( t < r8 ) {
01348         pow = t[2];
01349         mm = rnext;
01350         while ( mm < r3 ) {
01351             mnext = mm + *mm;
01352             GETSTOP(mm,mstop); mm++;
01353             while ( mm < mstop ) {
01354                 if ( *mm != DOTPRODUCT ) mm += mm[1];
01355                 else break;
01356             }
01357             if ( *mm == DOTPRODUCT ) {
01358                 mstop = mm + mm[1]; mm += 2;
01359                 while ( ( *mm != *t || mm[1] != t[1] )
01360                     && mm < mstop ) mm += 3;
01361                 if ( mm >= mstop ) pow = 0;
01362                 else if ( pow > 0 && mm[2] > 0 ) {
01363                     if ( mm[2] < pow ) pow = mm[2];
01364                 }
01365                 else if ( pow < 0 && mm[2] < 0 ) {
01366                     if ( mm[2] > pow ) pow = mm[2];
01367                 }
01368                 else pow = 0;
01369             }
01370             else pow = 0;
01371             if ( pow == 0 ) break;
01372             mm = mnext;
01373         }
01374     }
01375     if ( pow == 0 ) { t += 3; continue; }
01376     /*
01377     We have a factor
01378     */
01379     action = 1; i = pow;
01380     if ( i > 0 ) {
01381         while ( --i >= 0 ) {
01382             *r1++ = 9 + ARGHEAD;
01383             for ( j = 1; j < ARGHEAD; j++ ) *r1++ = 0;
01384             *r1++ = 9; *r1++ = DOTPRODUCT;
01385             *r1++ = 5; *r1++ = *t; *r1++ = t[1]; *r1++ = 1;
01386             *r1++ = 1; *r1++ = 1; *r1++ = 3;
01387         }
01388     }

```



```

01388         else {
01389             while ( i++ < 0 ) {
01390                 *r1++ = 9 + ARGHEAD;
01391                 for ( j = 1; j < ARGHEAD; j++ ) *r1++ = 0;
01392                 *r1++ = 9; *r1++ = DOTPRODUCT;
01393                 *r1++ = 5; *r1++ = *t; *r1++ = t[1]; *r1++ = -1;
01394                 *r1++ = 1; *r1++ = 1; *r1++ = 3;
01395             }
01396         }
01397     /*
01398     Now we have to remove the dotproducts
01399     */
01400     t[2] -= pow;
01401     mm = rnext;
01402     while ( mm < r3 ) {
01403         mnext = mm + *mm;
01404         GETSTOP(mm,mstop); mm++;
01405         while ( mm < mstop ) {
01406             if ( *mm != DOTPRODUCT ) mm += mm[1];
01407             else break;
01408         }
01409         mstop = mm + mm[1]; mm += 2;
01410         while ( mm < mstop && ( *mm != *t
01411             || mm[1] != t[1] ) ) mm += 3;
01412         mm[2] -= pow;
01413         mm = mnext;
01414     }
01415     t += 3;
01416 }
01417 }
01418 /*
01419 #] DOTPRODUCT :
01420 #[ DELTA/VECTOR :
01421 */
01422     else if ( *t == DELTA || *t == VECTOR ) {
01423         r7 = t; r8 = t + t[1]; t += 2;
01424         while ( t < r8 ) {
01425             mm = rnext;
01426             pow = 1;
01427             while ( mm < r3 ) {
01428                 mnext = mm + *mm;
01429                 GETSTOP(mm,mstop); mm++;
01430                 while ( mm < mstop ) {
01431                     if ( *mm != *r7 ) mm += mm[1];
01432                     else break;
01433                 }
01434                 if ( *mm == *r7 ) {
01435                     mstop = mm + mm[1]; mm += 2;
01436                     while ( ( *mm != *t || mm[1] != t[1] )
01437                         && mm < mstop ) mm += 2;
01438                     if ( mm >= mstop ) pow = 0;
01439                 }
01440                 else pow = 0;
01441                 if ( pow == 0 ) break;
01442                 mm = mnext;
01443             }
01444             if ( pow == 0 ) { t += 2; continue; }
01445         /*
01446         We have a factor
01447         */
01448         action = 1;
01449         *r1++ = 8 + ARGHEAD;
01450         for ( j = 1; j < ARGHEAD; j++ ) *r1++ = 0;
01451         *r1++ = 8; *r1++ = *r7;
01452         *r1++ = 4; *r1++ = *t; *r1++ = t[1];
01453         *r1++ = 1; *r1++ = 1; *r1++ = 3;
01454     /*
01455     Now we have to remove the delta's/vectors
01456     */
01457     mm = rnext;
01458     while ( mm < r3 ) {
01459         mnext = mm + *mm;
01460         GETSTOP(mm,mstop); mm++;
01461         while ( mm < mstop ) {
01462             if ( *mm != *r7 ) mm += mm[1];
01463             else break;
01464         }
01465         mstop = mm + mm[1]; mm += 2;
01466         while ( mm < mstop && (
01467             *mm != *t || mm[1] != t[1] ) ) mm += 2;
01468         *mm = mm[1] = NOINDEX;
01469         mm = mnext;
01470     }
01471     *t = t[1] = NOINDEX;
01472     t += 2;
01473 }
01474 }

```

```

01475 /*
01476 #] DELTA/VECTOR :
01477 #[ INDEX :
01478 */
01479
01480     else if ( *t == INDEX ) {
01481         r7 = t; r8 = t + t[1]; t += 2;
01482         while ( t < r8 ) {
01483             mm = rnext;
01484             pow = 1;
01485             while ( mm < r3 ) {
01486                 mnext = mm + *mm;
01487                 GETSTOP(mm,mstop); mm++;
01488                 while ( mm < mstop ) {
01489                     if ( *mm != *r7 ) mm += mm[1];
01490                     else break;
01491                 }
01492                 if ( *mm == *r7 ) {
01493                     mstop = mm + mm[1]; mm += 2;
01494                     while ( *mm != *t
01495                         && mm < mstop ) mm++;
01496                     if ( mm >= mstop ) pow = 0;
01497                 }
01498                 else pow = 0;
01499                 if ( pow == 0 ) break;
01500                 mm = mnext;
01501             }
01502             if ( pow == 0 ) { t++; continue; }
01503
01504             We have a factor
01505
01506             action = 1;
01507
01508             The next looks like an error.
01509             We should have here a VECTOR or INDEX like object
01510
01511             *r1++ = 7 + ARGHEAD;
01512             for ( j = 1; j < ARGHEAD; j++ ) *r1++ = 0;
01513             *r1++ = 7; *r1++ = *r7;
01514             *r1++ = 3; *r1++ = *t;
01515             *r1++ = 1; *r1++ = 1; *r1++ = 3;
01516
01517             Replace this by: (11-apr-2007)
01518
01519             if ( *t < 0 ) { *r1++ = -VECTOR; }
01520             else { *r1++ = -INDEX; }
01521             *r1++ = *t;
01522
01523             Now we have to remove the index
01524
01525             *t = NOINDEX;
01526             mm = rnext;
01527             while ( mm < r3 ) {
01528                 mnext = mm + *mm;
01529                 GETSTOP(mm,mstop); mm++;
01530                 while ( mm < mstop ) {
01531                     if ( *mm != *r7 ) mm += mm[1];
01532                     else break;
01533                 }
01534                 mstop = mm + mm[1]; mm += 2;
01535                 while ( mm < mstop &&
01536                     *mm != *t ) mm += 1;
01537                 *mm = NOINDEX;
01538                 mm = mnext;
01539             }
01540             t += 1;
01541         }
01542     }
01543
01544 #] INDEX :
01545 #[ FUNCTION :
01546 */
01547
01548     else if ( *t >= FUNCTION ) {
01549
01550         In the next code we should actually look inside
01551         the DENOMINATOR or EXPONENT for noncommuting objects
01552
01553         if ( *t >= FUNCTION &&
01554             functions[*t-FUNCTION].commute == 0 ) ncom = 0;
01555         else ncom = 1;
01556         if ( ncom ) {
01557             mm = r5 + 1;
01558             while ( mm < t && ( *mm == DUMMYFUN
01559                 || *mm == DUMMYTEN ) ) mm += mm[1];
01560             if ( mm < t ) { t += t[1]; continue; }
01561         }
01562         mm = rnext; pow = 1;
01563         while ( mm < r3 ) {

```

```

01562             mnext = mm + *mm;
01563             GETSTOP(mm,mstop); mm++;
01564             while ( mm < mstop ) {
01565                 if ( *mm == *t && mm[1] == t[1] ) {
01566                     for ( i = 2; i < t[1]; i++ ) {
01567                         if ( mm[i] != t[i] ) break;
01568                     }
01569                     if ( i >= t[1] )
01570                         { mm += mm[1]; goto nextmterm; }
01571                 }
01572                 if ( ncom && *mm != DUMMYFUN && *mm != DUMMYTEN )
01573                     { pow = 0; break; }
01574                 mm += mm[1];
01575             }
01576             if ( mm >= mstop ) pow = 0;
01577             if ( pow == 0 ) break;
01578 nextmterm:
01579             mm = mnext;
01580             if ( pow == 0 ) { t += t[1]; continue; }
01581 /*
01582 Copy the function
01583 */
01584 action = 1;
01585 *r1++ = t[1] + 4 + ARGHEAD;
01586 for ( i = 1; i < ARGHEAD; i++ ) *r1++ = 0;
01587 *r1++ = t[1] + 4;
01588 for ( i = 0; i < t[1]; i++ ) *r1++ = t[i];
01589 *r1++ = 1; *r1++ = 1; *r1++ = 3;
01590 /*
01591 Now we have to take out the functions
01592 */
01593 mm = rnext;
01594 while ( mm < r3 ) {
01595     mnext = mm + *mm;
01596     GETSTOP(mm,mstop); mm++;
01597     while ( mm < mstop ) {
01598         if ( *mm == *t && mm[1] == t[1] ) {
01599             for ( i = 2; i < t[1]; i++ ) {
01600                 if ( mm[i] != t[i] ) break;
01601             }
01602             if ( i >= t[1] ) {
01603                 if ( functions[*t-FUNCTION].spec > 0 )
01604                     *mm = DUMMYTEN;
01605                 else
01606                     *mm = DUMMYFUN;
01607                 mm += mm[1];
01608                 goto nextterm;
01609             }
01610         }
01611         mm += mm[1];
01612     }
01613 nextterm:
01614     mm = mnext;
01615     if ( functions[*t-FUNCTION].spec > 0 )
01616         *t = DUMMYTEN;
01617     else
01618         *t = DUMMYFUN;
01619     action = 1;
01620     v[2] = DIRTYFLAG;
01621     t += t[1];
01622 }
01623 /*
01624 #] FUNCTION :
01625 */
01626 else {
01627     t += t[1];
01628 }
01629 }
01630 }
01631 t = r5; pow = 1;
01632 while ( t < r3 ) {
01633     t += *t; if ( t[-1] > 0 ) { pow = 0; break; }
01634 }
01635 /*
01636 We have to add here the code for computing the GCD
01637 and to divide it out.
01638 */
01639 /*
01640 #[ Numerical factor :
01641 */
01642 t = r5;
01643 EAscrat = (UWORD *) (TermMalloc("execarg"));
01644 if ( t + *t == r3 ) goto oneterm;
01645 GETSTOP(t,r6);
01646 ngcd = t[t[0]-1];
01647 i = abs(ngcd)-1;
01648 while ( --i >= 0 ) EAscrat[i] = r6[i];

```

```

01649         t += *t;
01650         while ( t < r3 ) {
01651             GETSTOP(t,r6);
01652             i = t[t[0]-1];
01653             if ( AccumGCD(BHEAD EAscrat,&ngcd,(UWORD *)r6,i) ) goto execargerr;
01654             if ( ngcd == 3 && EAscrat[0] == 1 && EAscrat[1] == 1 ) break;
01655             t += *t;
01656         }
01657         if ( ngcd != 3 || EAscrat[0] != 1 || EAscrat[1] != 1 ) {
01658             if ( pow ) ngcd = -ngcd;
01659             t = r5; r9 = r1; *r1++ = t[-ARGHEAD]; *r1++ = 1;
01660             FILLARG(r1); ngcd = REDLENG(ngcd);
01661             while ( t < r3 ) {
01662                 GETSTOP(t,r6);
01663                 r7 = t; r8 = r1;
01664                 while ( r7 < r6 ) *r1++ = *r7++;
01665                 t += *t;
01666                 i = REDLENG(t[-1]);
01667                 if ( DivRat(BHEAD (UWORD *)r6,i,EAscrat,ngcd,(UWORD *)r1,&nq) ) goto
execargerr;
01668                 nq = INCLENG(nq);
01669                 i = ABS(nq)-1;
01670                 r1 += i; *r1++ = nq; *r8 = r1-r8;
01671             }
01672             *r9 = r1-r9;
01673             ngcd = INCLENG(ngcd);
01674             i = ABS(ngcd)-1;
01675             if ( factor && *factor == 0 ) {}
01676             else if ( ( factor && factor[0] == 4 && factor[2] == 1
01677 && factor[3] == -3 ) || pow == 0 ) {
01678                 r9 = r1; *r1++ = ARGHEAD+2+i; *r1++ = 0;
01679                 FILLARG(r1); *r1++ = i+2;
01680                 for ( j = 0; j < i; j++ ) *r1++ = EAscrat[j];
01681                 *r1++ = ngcd;
01682                 if ( ToFast(r9,r9) ) r1 = r9+2;
01683             }
01684             else if ( factor && factor[0] == 4 && factor[2] == 1
01685 && factor[3] > 0 && pow ) {
01686                 if ( ngcd < 0 ) ngcd = -ngcd;
01687                 *r1++ = -SNUMBER; *r1++ = -1;
01688                 r9 = r1; *r1++ = ARGHEAD+2+i; *r1++ = 0;
01689                 FILLARG(r1); *r1++ = i+2;
01690                 for ( j = 0; j < i; j++ ) *r1++ = EAscrat[j];
01691                 *r1++ = ngcd;
01692                 if ( ToFast(r9,r9) ) r1 = r9+2;
01693             }
01694             else {
01695                 if ( ngcd < 0 ) ngcd = -ngcd;
01696                 if ( pow ) { *r1++ = -SNUMBER; *r1++ = -1; }
01697                 if ( ngcd != 3 || EAscrat[0] != 1 || EAscrat[1] != 1 ) {
01698                     r9 = r1; *r1++ = ARGHEAD+2+i; *r1++ = 0;
01699                     FILLARG(r1); *r1++ = i+2;
01700                     for ( j = 0; j < i; j++ ) *r1++ = EAscrat[j];
01701                     *r1++ = ngcd;
01702                     if ( ToFast(r9,r9) ) r1 = r9+2;
01703                 }
01704             }
01705         }
01706         /*
01707         #] Numerical factor :
01708         */
01709         else {
01710 oneterm:;
01711             if ( factor == 0 || *factor > 2 ) {
01712                 if ( pow > 0 ) {
01713                     *r1++ = -SNUMBER; *r1++ = -1;
01714                     t = r5;
01715                     while ( t < r3 ) {
01716                         t += *t; t[-1] = -t[-1];
01717                     }
01718                 }
01719                 t = rr; *r1++ = *t++; *r1++ = 1; t++;
01720                 COPYARG(r1,t);
01721                 while ( t < m ) *r1++ = *t++;
01722             }
01723         }
01724         TermFree(EAscrat,"execarg");
01725     }
01726     } /* AC.OldFactArgFlag */
01727 }
01728 /* r1 is fout in ons voorbeeld. */
01729     r2[1] = r1 - r2;
01730     action = 1;
01731     v[2] = DIRTYFLAG;
01732 }
01733 else {
01734     r = t + t[1];

```

```

01735         while ( t < r ) *r1++ = *t++;
01736     }
01737 }
01738 r = term + *term;
01739 while ( t < r ) *r1++ = *t++;
01740 m = AT.WorkPointer;
01741 i = m[0] = r1 - m;
01742 t = term;
01743 while ( --i >= 0 ) *t++ = *m++;
01744 if ( AT.WorkPointer < t ) AT.WorkPointer = t;
01745 }
01746 /*
01747 #] FACTARG :
01748 */
01749 AR.Cnumlhs = oldnumlhs;
01750 if ( action && Normalize(BHEAD term) ) goto execargerr;
01751 AT.WorkPointer = oldwork;
01752 if ( AT.WorkPointer < term + *term ) AT.WorkPointer = term + *term;
01753 AT.pWorkPointer = oldppointer;
01754 return(action);
01755 execargerr:
01756 AT.WorkPointer = oldwork;
01757 AT.pWorkPointer = oldppointer;
01758 MLOCK(ErrorMessageLock);
01759 MesCall("execarg");
01760 MUNLOCK(ErrorMessageLock);
01761 return(-1);
01762 }
01763
01764 /*
01765 #] execarg :
01766 #[ execterm :
01767 */
01768
01769 WORD execterm(PHEAD WORD *term, WORD level)
01770 {
01771     GETBIDENTITY
01772     CBUF *C = cbuf+AM.rbufnum;
01773     WORD oldnumlhs = AR.Cnumlhs;
01774     WORD maxisat = C->lhs[level][2];
01775     WORD *buffer1 = 0;
01776     WORD *oldworkpointer = AT.WorkPointer;
01777     WORD *t1, i;
01778     WORD olddeferflag = AR.DeferFlag, tryterm = 0;
01779     AR.DeferFlag = 0;
01780     do {
01781         AR.Cnumlhs = C->lhs[level][3];
01782         NewSort(BHEAD0);
01783     /*
01784         Normally for function arguments we do not use PolyFun/PolyRatFun.
01785         Hence NewSort sets the corresponding variables to zero.
01786         Here we overwrite that.
01787     */
01788     AN.FunSorts[AR.sLevel]->PolyFlag = ( AR.PolyFun != 0 ) ? AR.PolyFunType: 0;
01789     if ( AR.PolyFun == 0 ) { AN.FunSorts[AR.sLevel]->PolyFlag = 0; }
01790     else if ( AR.PolyFunType == 1 ) { AN.FunSorts[AR.sLevel]->PolyFlag = 1; }
01791     else if ( AR.PolyFunType == 2 ) {
01792         if ( AR.PolyFunExp == 2 ) AN.FunSorts[AR.sLevel]->PolyFlag = 1;
01793         else AN.FunSorts[AR.sLevel]->PolyFlag = 2;
01794     }
01795     if ( buffer1 ) {
01796         term = buffer1;
01797         while ( *term ) {
01798             t1 = oldworkpointer;
01799             i = *term; while ( --i >= 0 ) *t1++ = *term++;
01800             AT.WorkPointer = t1;
01801             if ( Generator(BHEAD oldworkpointer,level) ) goto exectermerr;
01802         }
01803     }
01804     else {
01805         if ( Generator(BHEAD term,level) ) goto exectermerr;
01806     }
01807     if ( buffer1 ) {
01808         if ( tryterm ) { TermFree(buffer1,"buffer in sort statement"); tryterm = 0; }
01809         else { M_free((void *)buffer1,"buffer in sort statement"); }
01810         buffer1 = 0;
01811     }
01812     AN.tryterm = 1;
01813     if ( EndSort(BHEAD (WORD *)((void *)&buffer1),2) < 0 ) goto exectermerr;
01814     tryterm = AN.tryterm; AN.tryterm = 0;
01815     level = AR.Cnumlhs;
01816 } while ( AR.Cnumlhs < maxisat );
01817 AR.Cnumlhs = oldnumlhs;
01818 AR.DeferFlag = olddeferflag;
01819 term = buffer1;
01820 while ( *term ) {
01821     t1 = oldworkpointer;

```

```

01822         i = *term; while ( --i >= 0 ) *tl++ = *term++;
01823         AT.WorkPointer = tl;
01824         if ( Generator(BHEAD oldworkpointer,level) ) goto exectermerr;
01825     }
01826     if ( tryterm ) { TermFree(buffer1,"buffer in term statement"); tryterm = 0; }
01827     else { M_free(buffer1,"buffer in term statement"); }
01828     buffer1 = 0;
01829     AT.WorkPointer = oldworkpointer;
01830     return(0);
01831 exectermerr:
01832     AT.WorkPointer = oldworkpointer;
01833     AR.DeferFlag = olddeferflag;
01834     MLOCK(ErrorMessageLock);
01835     MesCall("execterm");
01836     MUNLOCK(ErrorMessageLock);
01837     return(-1);
01838 }
01839
01840 /*
01841 #] execterm :
01842 #[ ArgumentImplode :
01843 */
01844
01845 int ArgumentImplode(PHEAD WORD *term, WORD *thelist)
01846 {
01847     GETBIDENTITY
01848     WORD *liststart, *liststop, *inlist;
01849     WORD *w, *t, *tend, *tstop, *tt, *ttstop, *ttt, ncount, i;
01850     int action = 0;
01851     liststop = thelist + thelist[1];
01852     liststart = thelist + 2;
01853     t = term;
01854     tend = t + *t;
01855     tstop = tend - ABS(tend[-1]);
01856     t++;
01857     while ( t < tstop ) {
01858         if ( *t >= FUNCTION ) {
01859             inlist = liststart;
01860             while ( inlist < liststop && *inlist != *t ) inlist += inlist[1];
01861             if ( inlist < liststop ) {
01862                 tt = t; ttstop = t + t[1]; w = AT.WorkPointer;
01863                 for ( i = 0; i < FUNHEAD; i++ ) *w++ = *tt++;
01864                 while ( tt < ttstop ) {
01865                     ncount = 0;
01866                     if ( *tt == -SNUMBER && tt[1] == 0 ) {
01867                         ncount = 1; ttt = tt; tt += 2;
01868                         while ( tt < ttstop && *tt == -SNUMBER && tt[1] == 0 ) {
01869                             ncount++; tt += 2;
01870                         }
01871                     }
01872                     if ( ncount > 0 ) {
01873                         if ( tt < ttstop && *tt == -SNUMBER && ( tt[1] == 1 || tt[1] == -1 ) ) {
01874                             *w++ = -SNUMBER;
01875                             *w++ = (ncount+1) * tt[1];
01876                             tt += 2;
01877                             action = 1;
01878                         }
01879                         else if ( ( tt[0] == tt[ARGHEAD] + ARGHEAD )
01880                             && ( ABS(tt[tt[0]-1]) == 3 )
01881                             && ( tt[tt[0]-2] == 1 )
01882                             && ( tt[tt[0]-3] == 1 ) ) { /* Single term with coef +/- 1 */
01883                             i = *tt; NCOPY(w,tt,i)
01884                             w[-3] = ncount+1;
01885                             action = 1;
01886                         }
01887                     }
01888                     else if ( *tt == -SYMBOL ) {
01889                         *w++ = ARGHEAD+8;
01890                         *w++ = 0;
01891                         FILLARG(w)
01892                         *w++ = 8;
01893                         *w++ = SYMBOL;
01894                         *w++ = tt[1];
01895                         *w++ = 1;
01896                         *w++ = ncount+1; *w++ = 1; *w++ = 3;
01897                         tt += 2;
01898                         action = 1;
01899                     }
01900                     else if ( *tt <= -FUNCTION ) {
01901                         *w++ = ARGHEAD+FUNHEAD+4;
01902                         *w++ = 0;
01903                         FILLARG(w)
01904                         *w++ = -*tt++;
01905                         *w++ = FUNHEAD+4;
01906                         FILLFUN(w)
01907                         *w++ = ncount+1; *w++ = 1; *w++ = 3;
01908                         action = 1;
01909                     }
01910                 }
01911             }
01912         }
01913     }
01914 }

```

```

01909         else {
01910             while ( ttt < tt ) *w++ = *ttt++;
01911             if ( tt < ttstop && *tt == -SNUMBER ) {
01912                 *w++ = *tt++; *w++ = *tt++;
01913             }
01914         }
01915     }
01916     else if ( *tt <= -FUNCTION ) {
01917         *w++ = *ttt++;
01918     }
01919     else if ( *tt < 0 ) {
01920         *w++ = *ttt++;
01921         *w++ = *ttt++;
01922     }
01923     else {
01924         i = *tt; NCOPY(w,tt,i)
01925     }
01926 }
01927 AT.WorkPointer[1] = w - AT.WorkPointer;
01928 while ( tt < tend ) *w++ = *tt++;
01929 ttt = AT.WorkPointer; tt = t;
01930 while ( ttt < w ) *ttt++ = *ttt++;
01931 term[0] = tt - term;
01932 AT.WorkPointer = tt;
01933 tend = tt; tstop = tt - ABS(tt[-1]);
01934 }
01935 }
01936 t += t[1];
01937 }
01938 if ( action ) {
01939     if ( Normalize(BHEAD term) ) return(-1);
01940 }
01941 return(0);
01942 }
01943
01944 /*
01945 #] ArgumentImplode :
01946 #[ ArgumentExplode :
01947 */
01948
01949 int ArgumentExplode(PHEAD WORD *term, WORD *thelist)
01950 {
01951     GETBIDENTITY
01952     WORD *liststart, *liststop, *inlist, *old;
01953     WORD *w, *t, *tend, *tstop, *tt, *ttstop, *ttt, ncount, i;
01954     int action = 0;
01955     LONG x;
01956     liststop = thelist + thelist[1];
01957     liststart = thelist + 2;
01958     t = term;
01959     tend = t + *t;
01960     tstop = tend - ABS(tend[-1]);
01961     t++;
01962     while ( t < tstop ) {
01963         if ( *t >= FUNCTION ) {
01964             inlist = liststart;
01965             while ( inlist < liststop && *inlist != *t ) inlist += inlist[1];
01966             if ( inlist < liststop ) {
01967                 tt = t; ttstop = t + t[1]; w = AT.WorkPointer;
01968                 for ( i = 0; i < FUNHEAD; i++ ) *w++ = *tt++;
01969                 while ( tt < ttstop ) {
01970                     if ( *tt == -SNUMBER && tt[1] != 0 ) {
01971                         if ( tt[1] < AM.MaxTer/((WORD)sizeof(WORD)*4)
01972                             && tt[1] > -(AM.MaxTer/((WORD)sizeof(WORD)*4))
01973                             && ( tt[1] > 1 || tt[1] < -1 ) ) {
01974                             ncount = ABS(tt[1]);
01975                             while ( ncount > 1 ) {
01976                                 *w++ = -SNUMBER; *w++ = 0; ncount--;
01977                             }
01978                             *w++ = -SNUMBER;
01979                             if ( tt[1] < 0 ) *w++ = -1;
01980                             else *w++ = 1;
01981                             tt += 2;
01982                             action = 1;
01983                         }
01984                     }
01985                     else {
01986                         *w++ = *ttt++; *w++ = *ttt++;
01987                     }
01988                 }
01989                 else if ( *tt <= -FUNCTION ) {
01990                     *w++ = *ttt++;
01991                 }
01992                 else if ( *tt < 0 ) {
01993                     *w++ = *ttt++;
01994                     *w++ = *ttt++;
01995                 }
01996                 else if ( tt[0] == tt[ARGHEAD]+ARGHEAD ) {

```

```

01996             ttt = tt + tt[0] - 1;
01997             i = (ABS(ttt[0])-1)/2;
01998             if ( i > 1 ) {
01999 TooMany:
02000                 old = AN.currentTerm;
02001                 AN.currentTerm = term;
02002                 MesPrint("Too many arguments in output of ArgExplode");
02003                 MesPrint("Term = %t");
02004                 AN.currentTerm = old;
02005                 return(-1);
02006             }
02007             if ( ttt[-1] != 1 ) goto NoExplode;
02008             x = ttt[-2];
02009             if ( 2*x > (AT.WorkTop-w)-*term ) goto TooMany;
02010             ncount = x - 1;
02011             while ( ncount > 0 ) {
02012                 *w++ = -SNUMBER; *w++ = 0; ncount--;
02013             }
02014             ttt[-2] = 1;
02015             i = *tt; NCOPY(w,tt,i)
02016             action = 1;
02017         }
02018         else {
02019 NoExplode:
02020             i = *tt; NCOPY(w,tt,i)
02021         }
02022         AT.WorkPointer[1] = w - AT.WorkPointer;
02023         while ( tt < tend ) *w++ = *tt++;
02024         ttt = AT.WorkPointer; tt = t;
02025         while ( ttt < w ) *tt++ = *ttt++;
02026         term[0] = tt - term;
02027         AT.WorkPointer = tt;
02028         tend = tt; tstop = tt - ABS(tt[-1]);
02029     }
02030 }
02031 t += t[1];
02032 }
02033 if ( action ) {
02034     if ( Normalize(BHEAD term) ) return(-1);
02035 }
02036 return(0);
02037 }
02038
02039 /*
02040 #] ArgumentExplode :
02041 #[ ArgFactorize :
02042 */
02058 #define NEWORDER
02059
02060 int ArgFactorize(PHEAD WORD *argin, WORD *argout)
02061 {
02062     /*
02063     #] step 0 : Declarations and initializations
02064     */
02065     WORD *argfree, *argextra, *argcopy, *t, *tstop, *a, *a1, *a2;
02066     #ifndef NEWORDER
02067     WORD *tt;
02068     #endif
02069     WORD startebuf = cbuf[AT.ebufnum].numrhs,oldword;
02070     WORD oldsorttype = AR.SortType, numargs;
02071     int error = 0, action = 0, i, ii, number, sign = 1;
02072
02073     *argout = 0;
02074     /*
02075     #] step 0 :
02076     #[ step 1 : Take care of ordering
02077     */
02078     AR.SortType = SORTHIGHFIRST;
02079     if ( oldsorttype != AR.SortType ) {
02080         NewSort(BHEAD0);
02081         oldword = argin[*argin]; argin[*argin] = 0;
02082         t = argin+ARGHEAD;
02083         while ( *t ) {
02084             tstop = t + *t;
02085             if ( AN.ncmod != 0 ) {
02086                 if ( AN.ncmod != 1 || ( (WORD)AN.cmod[0] < 0 ) ) {
02087                     MLOCK(ErrorMessageLock);
02088                     MesPrint("Factorization modulus a number, greater than a WORD not implemented.");
02089                     MUNLOCK(ErrorMessageLock);
02090                     Terminate(-1);
02091                 }
02092             }
02093             if ( Modulus(t) ) {
02094                 MLOCK(ErrorMessageLock);
02095                 MesCall("ArgFactorize");
02096                 MUNLOCK(ErrorMessageLock);
02097                 Terminate(-1);
02098             }
02099         }
02100     }

```



```

02098         if ( !*t ) { t = tstop; continue; }
02099     }
02100     StoreTerm(BHEAD t);
02101     t = tstop;
02102 }
02103 /* par = 1, in case the arg has more than SubTermsInSmall terms */
02104 EndSort(BHEAD argin+ARGHEAD,1);
02105 argin[*argin] = oldword;
02106 }
02107 /*
02108 #] step 1 :
02109 #[ step 2 : take out the 'content'.
02110 */
02111 argfree = TakeArgContent(BHEAD argin,argout);
02112 {
02113     al = argout;
02114     while ( *al ) {
02115         if ( al[0] == -SNUMBER && ( al[1] == 1 || al[1] == -1 ) ) {
02116             if ( al[1] == -1 ) { sign = -sign; al[1] = 1; }
02117             if ( al[2] ) {
02118                 a = t = al+2; while ( *t ) NEXTARG(t);
02119                 i = t - al-2;
02120                 t = al; NCOPY(t,a,i);
02121                 *t = 0;
02122                 continue;
02123             }
02124             else {
02125                 al[0] = 0;
02126             }
02127             break;
02128         }
02129         else if ( al[0] == FUNHEAD+ARGHEAD+4 && al[ARGHEAD] == FUNHEAD+4
02130             && al[*al-1] == 3 && al[*al-2] == 1 && al[*al-3] == 1
02131             && al[ARGHEAD+1] >= FUNCTION ) {
02132             a = t = al+*al; while ( *t ) NEXTARG(t);
02133             i = t - a;
02134             *al = -al[ARGHEAD+1]; t = al+1; NCOPY(t,a,i);
02135             *t = 0;
02136         }
02137         NEXTARG(al);
02138     }
02139 }
02140 if ( argfree == 0 ) {
02141     argfree = argin;
02142 }
02143 else if ( argfree[0] == ( argfree[ARGHEAD]+ARGHEAD ) ) {
02144     Normalize(BHEAD argfree+ARGHEAD);
02145     argfree[0] = argfree[ARGHEAD]+ARGHEAD;
02146     argfree[1] = 0;
02147     if ( ( argfree[0] == ARGHEAD+4 ) && ( argfree[ARGHEAD+3] == 3 )
02148         && ( argfree[ARGHEAD+1] == 1 ) && ( argfree[ARGHEAD+2] == 1 ) ) {
02149         goto return0;
02150     }
02151 }
02152 else {
02153 /*
02154     The way we took out objects is rather brutish. We have to
02155     normalize
02156 */
02157     NewSort(BHEAD0);
02158     t = argfree+ARGHEAD;
02159     while ( *t ) {
02160         tstop = t + *t;
02161         Normalize(BHEAD t);
02162         StoreTerm(BHEAD t);
02163         t = tstop;
02164     }
02165     /* par = 1, in case the arg has more than SubTermsInSmall terms */
02166     EndSort(BHEAD argfree+ARGHEAD,1);
02167     t = argfree+ARGHEAD;
02168     while ( *t ) t += *t;
02169     *argfree = t - argfree;
02170 }
02171 /*
02172 #] step 2 :
02173 #[ step 3 : look whether we have done this one already.
02174 */
02175 if ( ( number = FindArg(BHEAD argfree) ) != 0 ) {
02176     if ( number > 0 ) t = cbuf[AT.fbufnum].rhs[number-1];
02177     else t = cbuf[AC.ffbufnum].rhs[-number-1];
02178 /*
02179     Now position on the result. Remember we have in the cache:
02180         inputarg,0,outputargs,0
02181     t is currently at inputarg. *inputarg is always positive.
02182     in principle this holds also for the arguments in the output
02183     but we take no risks here (in case of future developments).
02184 */

```

```

02185         t += *t; t++;
02186         tstop = t;
02187         ii = 0;
02188         while ( *tstop ) {
02189             if ( *tstop == -SNUMBER && tstop[1] == -1 ) {
02190                 sign = -sign; ii += 2;
02191             }
02192             NEXTARG(tstop);
02193         }
02194         a = argout; while ( *a ) NEXTARG(a);
02195 #ifndef NEWORDER
02196         if ( sign == -1 ) { *a++ = -SNUMBER; *a++ = -1; *a = 0; sign = 1; }
02197 #endif
02198         i = tstop - t - ii;
02199         ii = a - argout;
02200         a2 = a; a1 = a + i;
02201         *a1 = 0;
02202         while ( ii > 0 ) { *--a1 = *--a2; ii--; }
02203         a = argout;
02204         while ( *t ) {
02205             if ( *t == -SNUMBER && t[1] == -1 ) { t += 2; }
02206             else { COPY1ARG(a,t) }
02207         }
02208         goto return0;
02209     }
02210 /*
02211     #] step 3 :
02212     #[ step 4 : invoke ConvertToPoly
02213
02214         We make a copy first in case there are no factors
02215 */
02216     argcopy = TermMalloc("argcopy");
02217     for ( i = 0; i <= *argfree; i++ ) argcopy[i] = argfree[i];
02218
02219     tstop = argfree + *argfree;
02220     {
02221         WORD sumcommu = 0;
02222         t = argfree + ARGHEAD;
02223         while ( t < tstop ) {
02224             sumcommu += DoesCommu(t);
02225             t += *t;
02226         }
02227         if ( sumcommu > 1 ) {
02228             MesPrint("ERROR: Cannot factorize an argument with more than one noncommuting object");
02229             Terminate(-1);
02230         }
02231     }
02232     t = argfree + ARGHEAD;
02233
02234     while ( t < tstop ) {
02235         if ( ( t[1] != SYMBOL ) && ( *t != (ABS(t[*t-1])+1) ) ) {
02236             action = 1; break;
02237         }
02238         t += *t;
02239     }
02240     if ( action ) {
02241         t = argfree + ARGHEAD;
02242         argextra = AT.WorkPointer;
02243         NewSort(BHEAD0);
02244         while ( t < tstop ) {
02245             if ( LocalConvertToPoly(BHEAD t,argextra,startebuf,0) < 0 ) {
02246                 error = -1;
02247             }
02248             getout:
02249                 AR.SortType = oldsorttype;
02250                 TermFree(argcopy,"argcopy");
02251                 if ( argfree != argin ) TermFree(argfree,"argfree");
02252                 MesCall("ArgFactorize");
02253                 Terminate(-1);
02254                 return(-1);
02255             }
02256             StoreTerm(BHEAD argextra);
02257             t += *t; argextra += *argextra;
02258         }
02259         /* par = 1, in case the arg has more than SubTermsInSmall terms */
02260         if ( EndSort(BHEAD argfree+ARGHEAD,1) < 0 ) { error = -2; goto getout; }
02261         t = argfree + ARGHEAD;
02262         while ( *t > 0 ) t += *t;
02263         *argfree = t - argfree;
02264     }
02265 /*
02266     #] step 4 :
02267     #[ step 5 : If not in the tables, we have to do this by hard work.
02268 */
02269     a = argout;
02270     while ( *a ) NEXTARG(a);
02271     if ( poly_factorize_argument(BHEAD argfree,a) < 0 ) {

```

```

02272         MesCall("ArgFactorize");
02273         error = -1;
02274     }
02275     /*
02276     #] step 5 :
02277     #[ step 6 : use now ConvertFromPoly
02278
02279         Be careful: there should be more than one argument now.
02280     */
02281     if ( error == 0 && action ) {
02282         a1 = a; NEXTARG(a1);
02283         if ( *a1 != 0 ) {
02284             CBUF *C = cbuf+AC.cbunum;
02285             CBUF *CC = cbuf+AT.ebunum;
02286             WORD *oldworkpointer = AT.WorkPointer;
02287             WORD *argcopy2 = TermMalloc("argcopy2"), *a1, *a2;
02288             a1 = a; a2 = argcopy2;
02289             while ( *a1 ) {
02290                 if ( *a1 < 0 ) {
02291                     if ( *a1 > -FUNCTION ) *a2++ = *a1++;
02292                     *a2++ = *a1++; *a2 = 0;
02293                     continue;
02294                 }
02295                 t = a1 + ARGHEAD;
02296                 tstop = a1 + *a1;
02297                 argextra = AT.WorkPointer;
02298                 NewSort(BHEAD0);
02299                 while ( t < tstop ) {
02300                     if ( ConvertFromPoly(BHEAD t,argextra,numxsymbol,CC->numrhs-startebuf+numxsymbol
02301 ,startebuf-numxsymbol,1) <= 0 ) {
02302                         TermFree(argcopy2,"argcopy2");
02303                         LowerSortLevel();
02304                         error = -3;
02305                         goto getout;
02306                     }
02307                     t += *t;
02308                     AT.WorkPointer = argextra + *argextra;
02309                 /*
02310                 ConvertFromPoly leaves terms with subexpressions. Hence:
02311             */
02312                 if ( Generator(BHEAD argextra,C->numlhs) ) {
02313                     TermFree(argcopy2,"argcopy2");
02314                     LowerSortLevel();
02315                     error = -4;
02316                     goto getout;
02317                 }
02318                 AT.WorkPointer = oldworkpointer;
02319                 /* par = 1, in case the factor has more than SubTermsInSmall terms */
02320                 if ( EndSort(BHEAD a2+ARGHEAD,1) < 0 ) { error = -5; goto getout; }
02321                 t = a2+ARGHEAD; while ( *t ) t += *t;
02322                 *a2 = t - a2; a2[1] = 0; ZEROARG(a2);
02323                 ToFast(a2,a2); NEXTARG(a2);
02324                 a1 = tstop;
02325             }
02326             i = a2 - argcopy2;
02327             a2 = argcopy2; a1 = a;
02328             NCOPY(a1,a2,i);
02329             *a1 = 0;
02330             TermFree(argcopy2,"argcopy2");
02331         /*
02332         Erase the entries we made temporarily in cbuf[AT.ebunum]
02333     */
02334         CC->numrhs = startebuf;
02335     }
02336     else { /* no factorization. recover the argument from before step 3. */
02337         for ( i = 0; i <= *argcopy; i++ ) a[i] = argcopy[i];
02338     }
02339 }
02340 /*
02341 #] step 6 :
02342 #[ step 7 : Add this one to the tables.
02343
02344         Possibly drop some elements in the tables
02345         when they become too full.
02346     */
02347     if ( error == 0 && AN.ncmod == 0 ) {
02348         if ( InsertArg(BHEAD argcopy,a,0) < 0 ) { error = -1; }
02349     }
02350     /*
02351     #] step 7 :
02352     #[ step 8 : Clean up and return.
02353
02354         Change the order of the arguments in argout and a.
02355         Use argcopy as spare space.
02356     */
02357     ii = a - argout;

```

```

02359     for ( i = 0; i < ii; i++ ) argcopy[i] = argout[i];
02360     al = a;
02361     while ( *a1 ) {
02362         if ( *a1 == -SNUMBER && a1[1] < 0 ) {
02363             sign = -sign; a1[1] = -a1[1];
02364             if ( a1[1] == 1 ) {
02365                 a2 = a1+2; while ( *a2 ) NEXTARG(a2);
02366                 i = a2-a1-2; a2 = a1+2;
02367                 NCOPY(a1,a2,i);
02368                 *a1 = 0;
02369             }
02370             while ( *a1 ) NEXTARG(a1);
02371             break;
02372         }
02373         else {
02374             if ( *a1 > 0 && *a1 == a1[ARGHEAD]+ARGHEAD && a1[*a1-1] < 0 ) {
02375                 a1[*a1-1] = -a1[*a1-1]; sign = -sign;
02376             }
02377             if ( *a1 == ARGHEAD+4 && a1[ARGHEAD+1] == 1 && a1[ARGHEAD+2] == 1 ) {
02378                 a2 = a1+ARGHEAD+4; while ( *a2 ) NEXTARG(a2);
02379                 i = a2-a1-ARGHEAD-4; a2 = a1+ARGHEAD+4;
02380                 NCOPY(a1,a2,i);
02381                 *a1 = 0;
02382                 break;
02383             }
02384             while ( *a1 ) NEXTARG(a1);
02385             break;
02386         }
02387         NEXTARG(a1);
02388     }
02389     i = a1 - a;
02390     a2 = argout;
02391     NCOPY(a2,a,i);
02392     for ( i = 0; i < ii; i++ ) *a2++ = argcopy[i];
02393     #ifndef NEWORDER
02394     if ( sign == -1 ) { *a2++ = -SNUMBER; *a2++ = -1; sign = 1; }
02395     #endif
02396     *a2 = 0;
02397     TermFree(argcopy,"argcopy");
02398     return 0;
02399     if ( argfree != argin ) TermFree(argfree,"argfree");
02400     if ( oldsorttype != AR.SortType ) {
02401         AR.SortType = oldsorttype;
02402         a = argout;
02403         while ( *a ) {
02404             if ( *a > 0 ) {
02405                 NewSort(BHEAD0);
02406                 oldword = a[*a]; a[*a] = 0;
02407                 t = a+ARGHEAD;
02408                 while ( *t ) {
02409                     tstop = t + *t;
02410                     StoreTerm(BHEAD t);
02411                     t = tstop;
02412                 }
02413                 /* par = 1, in case the factor has more than SubTermsInSmall terms */
02414                 EndSort(BHEAD a+ARGHEAD,1);
02415                 a[*a] = oldword;
02416                 a += *a;
02417             }
02418             else { NEXTARG(a); }
02419         }
02420     }
02421     #ifdef NEWORDER
02422     t = argout; numargs = 0;
02423     while ( *t ) {
02424         tt = t;
02425         NEXTARG(tt);
02426         if ( *tt == ABS(t[-1])+1+ARGHEAD && sign == -1 ) { t[-1] = -t[-1]; sign = 1; }
02427         else if ( *tt == -SNUMBER && sign == -1 ) { tt[1] = -tt[1]; sign = 1; }
02428         numargs++;
02429     }
02430     if ( sign == -1 ) {
02431         *t++ = -SNUMBER; *t++ = -1; *t = 0; sign = 1; numargs++;
02432     }
02433     #else
02434     /*
02435     Now we have to sort the arguments
02436     First have the number of 'nontrivial/nonnumerical' arguments
02437     Then make a piece of code like in FullSymmetrize with that number
02438     of arguments to be symmetrized.
02439     Put a function in front
02440     Call the Symmetrize routine
02441     */
02442     t = argout; numargs = 0;
02443     while ( *t && *t != -SNUMBER && ( *t < 0 || ( ABS(t[*t-1]) != *t-1 ) ) ) {
02444         NEXTARG(t);
02445         numargs++;

```

```

02446     }
02447 #endif
02448     if ( numargs > 1 ) {
02449         WORD *Lijst;
02450         WORD x[3];
02451         x[0] = argout[-FUNHEAD];
02452         x[1] = argout[-FUNHEAD+1];
02453         x[2] = argout[-FUNHEAD+2];
02454         while ( *t ) { NEXTARG(t); }
02455         argout[-FUNHEAD] = SQRFUNCTION;
02456         argout[-FUNHEAD+1] = t-argout+FUNHEAD;
02457         argout[-FUNHEAD+2] = 0;
02458         AT.WorkPointer = t+1;
02459         Lijst = AT.WorkPointer;
02460         for ( i = 0; i < numargs; i++ ) Lijst[i] = i;
02461         AT.WorkPointer += numargs;
02462         error = Symmetrize(BHEAD argout-FUNHEAD,Lijst,numargs,1,SYMMETRIC);
02463         AT.WorkPointer = Lijst;
02464         argout[-FUNHEAD] = x[0];
02465         argout[-FUNHEAD+1] = x[1];
02466         argout[-FUNHEAD+2] = x[2];
02467 #ifdef NEWWORDER
02468 /*
02469      Now we have to get a potential numerical argument to the first position
02470 */
02471     tstop = argout; while ( *tstop ) { NEXTARG(tstop); }
02472     t = argout; number = 0;
02473     while ( *t ) {
02474         tt = t; NEXTARG(t);
02475         if ( *tt == -SNUMBER ) {
02476             if ( number == 0 ) break;
02477             x[0] = tt[1];
02478             while ( tt > argout ) { ---t = ---tt; }
02479             argout[0] = -SNUMBER; argout[1] = x[0];
02480             break;
02481         }
02482         else if ( *tt == ABS(t[-1])+1+ARGHEAD ) {
02483             if ( number == 0 ) break;
02484             ii = t - tt;
02485             for ( i = 0; i < ii; i++ ) tstop[i] = tt[i];
02486             while ( tt > argout ) { ---t = ---tt; }
02487             for ( i = 0; i < ii; i++ ) argout[i] = tstop[i];
02488             *tstop = 0;
02489             break;
02490         }
02491         number++;
02492     }
02493 #endif
02494     }
02495 /*
02496     #] step 8 :
02497 */
02498     return(error);
02499 }
02500
02501 /*
02502     #] ArgFactorize :
02503     #[ FindArg :
02504 */
02505 WORD FindArg(PHEAD WORD *a)
02506 {
02507     int number;
02508     if ( AN.ncmod != 0 ) return(0); /* no room for mod stuff */
02509     number = FindTree(AT.fbufnum,a);
02510     if ( number >= 0 ) return(number+1);
02511     number = FindTree(AC.ffbufnum,a);
02512     if ( number >= 0 ) return(-number-1);
02513     return(0);
02514 }
02515
02516 /*
02517     #] FindArg :
02518     #[ InsertArg :
02519 */
02520 WORD InsertArg(PHEAD WORD *argin, WORD *argout,int par)
02521 {
02522     CBUF *C;
02523     WORD *a, i, bufnum;
02524     if ( par == 0 ) {
02525         bufnum = AT.fbufnum;
02526         C = cbuf+bufnum;
02527         if ( C->numrhs >= (C->maxrhs-2) ) CleanupArgCache(BHEAD AT.fbufnum);
02528     }
02529     else if ( par == 1 ) {
02530         bufnum = AC.ffbufnum;
02531         C = cbuf+bufnum;
02532     }
02533 }

```

```

02550     else { return(-1); }
02551     AddRHS(bufnum,1);
02552     AddNtoC(bufnum,*argin,argin,1);
02553     AddToCB(C,0)
02554     a = argout; while ( *a ) NEXTARG(a);
02555     i = a - argout;
02556     AddNtoC(bufnum,i,argout,2);
02557     AddToCB(C,0)
02558     return(InsTree(bufnum,C->numrhs));
02559 }
02560
02561 /*
02562 #] InsertArg :
02563 #[ CleanupArgCache :
02564 */
02572 int CleanupArgCache(PHEAD WORD bufnum)
02573 {
02574     CBUF *C = cbuf + bufnum;
02575     COMPTREE *boomlijst = C->boomlijst;
02576     LONG *weights = (LONG *)Malloc1(2*(C->numrhs+1)*sizeof(LONG),"CleanupArgCache");
02577     LONG w, whalf, *extraweights;
02578     WORD *a, *to, *from;
02579     int i,j,k;
02580     for ( i = 1; i <= C->numrhs; i++ ) {
02581         weights[i] = ((LONG)i) * (boomlijst[i].usage);
02582     }
02583     /*
02584         Now sort the weights and determine the halfway weight
02585     */
02586     extraweights = weights+C->numrhs+1;
02587     SortWeights(weights+1,extraweights,C->numrhs);
02588     whalf = weights[C->numrhs/2+1];
02589     /*
02590         We should drop everybody with a weight < whalf.
02591     */
02592     to = C->Buffer;
02593     k = 1;
02594     for ( i = 1; i <= C->numrhs; i++ ) {
02595         from = C->rhs[i]; w = ((LONG)i) * (boomlijst[i].usage);
02596         if ( w >= whalf ) {
02597             if ( i < C->numrhs-1 ) {
02598                 if ( to == from ) {
02599                     to = C->rhs[i+1];
02600                 }
02601                 else {
02602                     j = C->rhs[i+1] - from;
02603                     C->rhs[k] = to;
02604                     NCOPY(to,from,j)
02605                 }
02606             }
02607             else if ( to == from ) {
02608                 to += *to + 1; while ( *to ) NEXTARG(to); to++;
02609             }
02610             else {
02611                 a = from; a += *a+1; while ( *a ) NEXTARG(a); a++;
02612                 j = a - from;
02613                 C->rhs[k] = to;
02614                 NCOPY(to,from,j)
02615             }
02616             weights[k++] = boomlijst[i].usage;
02617         }
02618     }
02619     C->numrhs = --k;
02620     C->Pointer = to;
02621     /*
02622         Next we need to rebuild the tree.
02623         Note that this can probably be done much faster by using the
02624         remains of the old tree !!!!!!!!!!!!!!!
02625     */
02626     ClearTree(AT.fbufnum);
02627     for ( i = 1; i <= k; i++ ) {
02628         InsTree(AT.fbufnum,i);
02629         boomlijst[i].usage = weights[i];
02630     }
02631     /*
02632         And cleanup
02633     */
02634     M_free(weights,"CleanupArgCache");
02635     return(0);
02636 }
02637
02638 /*
02639 #] CleanupArgCache :
02640 #[ ArgSymbolMerge :
02641 */
02642
02643 int ArgSymbolMerge(WORD *t1, WORD *t2)

```

```

02644 {
02645     WORD *t1e = t1+t1[1];
02646     WORD *t2e = t2+t2[1];
02647     WORD *t1a = t1+2;
02648     WORD *t2a = t2+2;
02649     WORD *t3;
02650     while ( t1a < t1e && t2a < t2e ) {
02651         if ( *t1a < *t2a ) {
02652             if ( t1a[1] >= 0 ) {
02653                 t3 = t1a+2;
02654                 while ( t3 < t1e ) { t3[-2] = *t3; t3[-1] = t3[1]; t3 += 2; }
02655                 t1e -= 2;
02656             }
02657             else t1a += 2;
02658         }
02659         else if ( *t1a > *t2a ) {
02660             if ( t2a[1] >= 0 ) t2a += 2;
02661             else {
02662                 t3 = t1e;
02663                 while ( t3 > t1a ) { *t3 = t3[-2]; t3[1] = t3[-1]; t3 -= 2; }
02664                 *t1a++ = *t2a++;
02665                 *t1a++ = *t2a++;
02666                 t1e += 2;
02667             }
02668         }
02669         else {
02670             if ( t2a[1] < t1a[1] ) t1a[1] = t2a[1];
02671             t1a += 2; t2a += 2;
02672         }
02673     }
02674     while ( t2a < t2e ) {
02675         if ( t2a[1] < 0 ) {
02676             *t1a++ = *t2a++;
02677             *t1a++ = *t2a++;
02678         }
02679         else t2a += 2;
02680     }
02681     while ( t1a < t1e ) {
02682         if ( t1a[1] >= 0 ) {
02683             t3 = t1a+2;
02684             while ( t3 < t1e ) { t3[-2] = *t3; t3[-1] = t3[1]; t3 += 2; }
02685             t1e -= 2;
02686         }
02687         else t1a += 2;
02688     }
02689     t1[1] = t1a - t1;
02690     return(0);
02691 }
02692
02693 /*
02694 #] ArgSymbolMerge :
02695 #[ ArgDotproductMerge :
02696 */
02697
02698 int ArgDotproductMerge(WORD *t1, WORD *t2)
02699 {
02700     WORD *t1e = t1+t1[1];
02701     WORD *t2e = t2+t2[1];
02702     WORD *t1a = t1+2;
02703     WORD *t2a = t2+2;
02704     WORD *t3;
02705     while ( t1a < t1e && t2a < t2e ) {
02706         if ( *t1a < *t2a || ( *t1a == *t2a && t1a[1] < t2a[1] ) ) {
02707             if ( t1a[2] >= 0 ) {
02708                 t3 = t1a+3;
02709                 while ( t3 < t1e ) { t3[-3] = *t3; t3[-2] = t3[1]; t3[-1] = t3[2]; t3 += 3; }
02710                 t1e -= 3;
02711             }
02712             else t1a += 3;
02713         }
02714         else if ( *t1a > *t2a || ( *t1a == *t2a && t1a[1] > t2a[1] ) ) {
02715             if ( t2a[2] >= 0 ) t2a += 3;
02716             else {
02717                 t3 = t1e;
02718                 while ( t3 > t1a ) { *t3 = t3[-3]; t3[1] = t3[-2]; t3[2] = t3[-1]; t3 -= 3; }
02719                 *t1a++ = *t2a++;
02720                 *t1a++ = *t2a++;
02721                 *t1a++ = *t2a++;
02722                 t1e += 3;
02723             }
02724         }
02725         else {
02726             if ( t2a[2] < t1a[2] ) t1a[2] = t2a[2];
02727             t1a += 3; t2a += 3;
02728         }
02729     }
02730     while ( t2a < t2e ) {

```

```

02731         if ( t2a[2] < 0 ) {
02732             *t1a++ = *t2a++;
02733             *t1a++ = *t2a++;
02734             *t1a++ = *t2a++;
02735         }
02736         else t2a += 3;
02737     }
02738     while ( t1a < t1e ) {
02739         if ( t1a[2] >= 0 ) {
02740             t3 = t1a+3;
02741             while ( t3 < t1e ) { t3[-3] = *t3; t3[-2] = t3[1]; t3[-1] = t3[2]; t3 += 3; }
02742             t1e -= 3;
02743         }
02744         else t1a += 2;
02745     }
02746     t1[1] = t1a - t1;
02747     return(0);
02748 }
02749
02750 /*
02751     #] ArgDotproductMerge :
02752     #[ TakeArgContent :
02753 */
02766 WORD *TakeArgContent(PHEAD WORD *argin, WORD *argout)
02767 {
02768     GETBIDENTITY
02769     WORD *t, *rnext, *r1, *r2, *r3, *r5, *r6, *r7, *r8, *r9;
02770     WORD pow, *mm, *mnext, *mstop, *argin2 = argin, *argin3 = argin, *argfree;
02771     WORD ncom;
02772     int j, i, act;
02773     r5 = t = argin + ARGHEAD;
02774     r3 = argin + *argin;
02775     rnext = t + *t;
02776     GETSTOP(t,r6);
02777     r1 = argout;
02778     t++;
02779 /*
02780     First pass: arrange everything but the symbols and dotproducts.
02781     They need separate treatment because we have to take out negative
02782     powers.
02783 */
02784     while ( t < r6 ) {
02785 /*
02786         #[ DELTA/VECTOR :
02787 */
02788         if ( *t == DELTA || *t == VECTOR ) {
02789             r7 = t; r8 = t + t[1]; t += 2;
02790             while ( t < r8 ) {
02791                 mm = rnext;
02792                 pow = 1;
02793                 while ( mm < r3 ) {
02794                     mnext = mm + *mm;
02795                     GETSTOP(mm,mstop); mm++;
02796                     while ( mm < mstop ) {
02797                         if ( *mm != *r7 ) mm += mm[1];
02798                         else break;
02799                     }
02800                     if ( *mm == *r7 ) {
02801                         mstop = mm + mm[1]; mm += 2;
02802                         while ( ( *mm != *t || mm[1] != t[1] )
02803                             && mm < mstop ) mm += 2;
02804                         if ( mm >= mstop ) pow = 0;
02805                     }
02806                     else pow = 0;
02807                     if ( pow == 0 ) break;
02808                     mm = mnext;
02809                 }
02810                 if ( pow == 0 ) { t += 2; continue; }
02811 /*
02812                 We have a factor
02813 */
02814                 *r1++ = 8 + ARGHEAD;
02815                 for ( j = 1; j < ARGHEAD; j++ ) *r1++ = 0;
02816                 *r1++ = 8; *r1++ = *r7;
02817                 *r1++ = 4; *r1++ = *t; *r1++ = t[1];
02818                 *r1++ = 1; *r1++ = 1; *r1++ = 3;
02819                 argout = r1;
02820 /*
02821                 Now we have to remove the delta's/vectors
02822 */
02823                 mm = rnext;
02824                 while ( mm < r3 ) {
02825                     mnext = mm + *mm;
02826                     GETSTOP(mm,mstop); mm++;
02827                     while ( mm < mstop ) {
02828                         if ( *mm != *r7 ) mm += mm[1];
02829                         else break;

```



```

02830         }
02831         mstop = mm + mm[1]; mm += 2;
02832         while ( mm < mstop && (
02833             *mm != *t || mm[1] != t[1] ) ) mm += 2;
02834         *mm = mm[1] = NOINDEX;
02835         mm = mnext;
02836     }
02837     *t = t[1] = NOINDEX;
02838     t += 2;
02839 }
02840 }
02841 /*
02842     #] DELTA/VECTOR :
02843     #[ INDEX :
02844 */
02845 else if ( *t == INDEX ) {
02846     r7 = t; r8 = t + t[1]; t += 2;
02847     while ( t < r8 ) {
02848         mm = rnext;
02849         pow = 1;
02850         while ( mm < r3 ) {
02851             mnext = mm + *mm;
02852             GETSTOP(mm,mstop); mm++;
02853             while ( mm < mstop ) {
02854                 if ( *mm != *r7 ) mm += mm[1];
02855                 else break;
02856             }
02857             if ( *mm == *r7 ) {
02858                 mstop = mm + mm[1]; mm += 2;
02859                 while ( *mm != *t
02860                     && mm < mstop ) mm++;
02861                 if ( mm >= mstop ) pow = 0;
02862             }
02863             else pow = 0;
02864             if ( pow == 0 ) break;
02865             mm = mnext;
02866         }
02867         if ( pow == 0 ) { t++; continue; }
02868     /*
02869     We have a factor
02870 */
02871     if ( *t < 0 ) { *r1++ = -VECTOR; }
02872     else          { *r1++ = -INDEX; }
02873     *r1++ = *t;
02874     argout = r1;
02875 /*
02876     Now we have to remove the index
02877 */
02878     *t = NOINDEX;
02879     mm = rnext;
02880     while ( mm < r3 ) {
02881         mnext = mm + *mm;
02882         GETSTOP(mm,mstop); mm++;
02883         while ( mm < mstop ) {
02884             if ( *mm != *r7 ) mm += mm[1];
02885             else break;
02886         }
02887         mstop = mm + mm[1]; mm += 2;
02888         while ( mm < mstop &&
02889             *mm != *t ) mm += 1;
02890         *mm = NOINDEX;
02891         mm = mnext;
02892     }
02893     t += 1;
02894 }
02895 }
02896 /*
02897     #] INDEX :
02898     #[ FUNCTION :
02899 */
02900 else if ( *t >= FUNCTION ) {
02901 /*
02902     In the next code we should actually look inside
02903     the DENOMINATOR or EXPONENT for noncommuting objects
02904 */
02905     if ( *t >= FUNCTION &&
02906         functions[*t-FUNCTION].commute == 0 ) ncom = 0;
02907     else ncom = 1;
02908     if ( ncom ) {
02909         mm = r5 + 1;
02910         while ( mm < t && ( *mm == DUMMYFUN
02911             || *mm == DUMMYTEN ) ) mm += mm[1];
02912         if ( mm < t ) { t += t[1]; continue; }
02913     }
02914     mm = rnext; pow = 1;
02915     while ( mm < r3 ) {
02916         mnext = mm + *mm;

```

```

02917         GETSTOP(mm,mstop); mm++;
02918         while ( mm < mstop ) {
02919             if ( *mm == *t && mm[1] == t[1] ) {
02920                 for ( i = 2; i < t[1]; i++ ) {
02921                     if ( mm[i] != t[i] ) break;
02922                 }
02923                 if ( i >= t[1] )
02924                     { mm += mm[1]; goto nexttterm; }
02925             }
02926             if ( ncom && *mm != DUMMYFUN && *mm != DUMMYTEN )
02927                 { pow = 0; break; }
02928             mm += mm[1];
02929         }
02930         if ( mm >= mstop ) pow = 0;
02931         if ( pow == 0 ) break;
02932 nexttterm:    mm = mnext;
02933     }
02934     if ( pow == 0 ) { t += t[1]; continue; }
02935 /*
02936     Copy the function
02937 */
02938     *r1++ = t[1] + 4 + ARGHEAD;
02939     for ( i = 1; i < ARGHEAD; i++ ) *r1++ = 0;
02940     *r1++ = t[1] + 4;
02941     for ( i = 0; i < t[1]; i++ ) *r1++ = t[i];
02942     *r1++ = 1; *r1++ = 1; *r1++ = 3;
02943     argout = r1;
02944 /*
02945     Now we have to take out the functions
02946 */
02947     mm = rnext;
02948     while ( mm < r3 ) {
02949         mnext = mm + *mm;
02950         GETSTOP(mm,mstop); mm++;
02951         while ( mm < mstop ) {
02952             if ( *mm == *t && mm[1] == t[1] ) {
02953                 for ( i = 2; i < t[1]; i++ ) {
02954                     if ( mm[i] != t[i] ) break;
02955                 }
02956                 if ( i >= t[1] ) {
02957                     if ( functions[*t-FUNCTION].spec > 0 )
02958                         *mm = DUMMYTEN;
02959                     else
02960                         *mm = DUMMYFUN;
02961                     mm += mm[1];
02962                     goto nexttterm;
02963                 }
02964             }
02965             mm += mm[1];
02966         }
02967 nexttterm:    mm = mnext;
02968     }
02969     if ( functions[*t-FUNCTION].spec > 0 )
02970         *t = DUMMYTEN;
02971     else
02972         *t = DUMMYFUN;
02973     t += t[1];
02974 }
02975 /*
02976     #] FUNCTION :
02977 */
02978     else {
02979         t += t[1];
02980     }
02981 }
02982 /*
02983     #[ SYMBOL :
02984
02985     Now collect all symbols. We can use the space after r1 as storage
02986 */
02987     t = argin+ARGHEAD;
02988     rnext = t + *t;
02989     r2 = r1;
02990     while ( t < r3 ) {
02991         GETSTOP(t,r6);
02992         t++;
02993         act = 0;
02994         while ( t < r6 ) {
02995             if ( *t == SYMBOL ) {
02996                 act = 1;
02997                 i = t[1];
02998                 NCOPY(r2,t,i)
02999             }
03000             else { t += t[1]; }
03001         }
03002         if ( act == 0 ) {
03003             *r2++ = SYMBOL; *r2++ = 2;

```

```

03004     }
03005     t = rnext; rnext = rnext + *rnext;
03006 }
03007 *r2 = 0;
03008 argin2 = argin;
03009 /*
03010     Now we have a list of all symbols as a sequence of SYMBOL subterms.
03011     Any symbol that is absent in a subterm has power zero.
03012     We now need a list of all minimum powers.
03013     This can be done by subsequent merges.
03014 */
03015 r7 = r1;          /* The first object into which we merge. */
03016 r8 = r7 + r7[1]; /* The object that gets merged into r7. */
03017 while ( *r8 ) {
03018     r2 = r8 + r8[1]; /* Next object */
03019     ArgSymbolMerge(r7,r8);
03020     r8 = r2;
03021 }
03022 /*
03023     Now we have to divide by the object in r7 and take it apart as factors.
03024     The division can be simple if there are no negative powers.
03025 */
03026 if ( r7[1] > 2 ) {
03027     r8 = r7+2;
03028     r2 = r7 + r7[1];
03029     act = 0;
03030     pow = 0;
03031     while ( r8 < r2 ) {
03032         if ( r8[1] < 0 ) { act = 1; pow += -r8[1]*(ARGHEAD+8); }
03033         else { pow += 2*r8[1]; }
03034         r8 += 2;
03035     }
03036 /*
03037     The amount of space we need to move r7 is given in pow
03038 */
03039 if ( act == 0 ) { /* this can be done 'in situ' */
03040     t = argin + ARGHEAD;
03041     while ( t < r3 ) {
03042         rnext = t + *t;
03043         GETSTOP(t,r6);
03044         t++;
03045         while ( t < r6 ) {
03046             if ( *t != SYMBOL ) { t += t[1]; continue; }
03047             r8 = r7+2; r9 = t + t[1]; t += 2;
03048             while ( ( t < r9 ) && ( r8 < r2 ) ) {
03049                 if ( *t == *r8 ) {
03050                     t[1] -= r8[1]; t += 2; r8 += 2;
03051                 }
03052                 else { /* *t must be < than *r8 !!! */
03053                     t += 2;
03054                 }
03055             }
03056             t = r9;
03057         }
03058         t = rnext;
03059     }
03060 /*
03061     And now the factors that go to argout.
03062     First we have to move r7 out of the way.
03063 */
03064 r8 = r7+pow; i = r7[1];
03065 while ( --i >= 0 ) r8[i] = r7[i];
03066 r2 += pow;
03067 r8 += 2;
03068 while ( r8 < r2 ) {
03069     for ( i = 0; i < r8[1]; i++ ) { *r1++ = -SYMBOL; *r1++ = *r8; }
03070     r8 += 2;
03071 }
03072 }
03073 else { /* this needs a new location */
03074     argin2 = TermMalloc("TakeArgContent2");
03075 /*
03076     We have to multiply the inverse of r7 into argin
03077     The answer should go to argin2.
03078 */
03079 r5 = argin2; *r5++ = 0; *r5++ = 0; FILLARG(r5);
03080 t = argin+ARGHEAD;
03081 while ( t < r3 ) {
03082     rnext = t + *t;
03083     GETSTOP(t,r6);
03084     r9 = r5;
03085     *r5++ = *t++ + r7[1];
03086     while ( t < r6 ) *r5++ = *t++;
03087     i = r7[1] - 2; r8 = r7+2;
03088     *r5++ = r7[0]; *r5++ = r7[1];
03089     while ( i > 0 ) { *r5++ = *r8++; *r5++ = -*r8++; i -= 2; }
03090     while ( t < rnext ) *r5++ = *t++;

```

```

03091         Normalize(BHEAD r9);
03092         r5 = r9 + *r9;
03093     }
03094     *r5 = 0;
03095     *argin2 = r5-argin2;
03096 /*
03097     We may have to sort the terms in argin2.
03098 */
03099     NewSort(BHEAD0);
03100     t = argin2+ARGHEAD;
03101     while ( *t ) {
03102         StoreTerm(BHEAD t);
03103         t += *t;
03104     }
03105     t = argin2+ARGHEAD;
03106     if ( EndSort(BHEAD t,0) < 0 ) goto Irreg;
03107     while ( *t ) t += *t;
03108     *argin2 = t - argin2;
03109     r3 = t;
03110 /*
03111     And now the factors that go to argout.
03112     First we have to move r7 out of the way.
03113 */
03114     r8 = r7+pow; i = r7[1];
03115     while ( --i >= 0 ) r8[i] = r7[i];
03116     r2 += pow;
03117     r8 += 2;
03118     while ( r8 < r2 ) {
03119         if ( r8[1] >= 0 ) {
03120             for ( i = 0; i < r8[1]; i++ ) { *r1++ = -SYMBOL; *r1++ = *r8; }
03121         }
03122         else {
03123             for ( i = 0; i < -r8[1]; i++ ) {
03124                 *r1++ = ARGHEAD+8; *r1++ = 0;
03125                 FILLARG(r1);
03126                 *r1++ = 8; *r1++ = SYMBOL; *r1++ = 4; *r1++ = *r8;
03127                 *r1++ = -1; *r1++ = 1; *r1++ = 1; *r1++ = 3;
03128             }
03129             r8 += 2;
03130         }
03131     }
03132     argout = r1;
03133 }
03134 }
03135 /*
03136     #] SYMBOL :
03137     #[ DOTPRODUCT :
03138
03139     Now collect all dotproducts. We can use the space after r1 as storage
03140 */
03141     t = argin2+ARGHEAD;
03142     rnext = t + *t;
03143     r2 = r1;
03144     while ( t < r3 ) {
03145         GETSTOP(t,r6);
03146         t++;
03147         act = 0;
03148         while ( t < r6 ) {
03149             if ( *t == DOTPRODUCT ) {
03150                 act = 1;
03151                 i = t[1];
03152                 NCOPY(r2,t,i)
03153             }
03154             else { t += t[1]; }
03155         }
03156         if ( act == 0 ) {
03157             *r2++ = DOTPRODUCT; *r2++ = 2;
03158         }
03159         t = rnext; rnext = rnext + *rnext;
03160     }
03161     *r2 = 0;
03162     argin3 = argin2;
03163 /*
03164     Now we have a list of all dotproducts as a sequence of DOTPRODUCT
03165     subterms. Any dotproduct that is absent in a subterm has power zero.
03166     We now need a list of all minimum powers.
03167     This can be done by subsequent merges.
03168 */
03169     r7 = r1; /* The first object into which we merge. */
03170     r8 = r7 + r7[1]; /* The object that gets merged into r7. */
03171     while ( *r8 ) {
03172         r2 = r8 + r8[1]; /* Next object */
03173         ArgDotproductMerge(r7,r8);
03174         r8 = r2;
03175     }
03176 /*
03177     Now we have to divide by the object in r7 and take it apart as factors.

```

```

03178         The division can be simple if there are no negative powers.
03179  */
03180     if ( r7[1] > 2 ) {
03181         r8 = r7+2;
03182         r2 = r7 + r7[1];
03183         act = 0;
03184         pow = 0;
03185         while ( r8 < r2 ) {
03186             if ( r8[2] < 0 ) { pow += -r8[2]*(ARGHEAD+9); }
03187             else { pow += r8[2]*(ARGHEAD+9); }
03188             r8 += 3;
03189         }
03190  /*
03191     The amount of space we need to move r7 is given in pow
03192     For dotproducts we always need a new location
03193  */
03194     {
03195         argin3 = TermMalloc("TakeArgContent3");
03196  /*
03197     We have to multiply the inverse of r7 into argin
03198     The answer should go to argin2.
03199  */
03200         r5 = argin3; *r5++ = 0; *r5++ = 0; FILLARG(r5);
03201         t = argin2+ARGHEAD;
03202         while ( t < r3 ) {
03203             rnext = t + *t;
03204             GETSTOP(t,r6);
03205             r9 = r5;
03206             *r5++ = *t++ + r7[1];
03207             while ( t < r6 ) *r5++ = *t++;
03208             i = r7[1] - 2; r8 = r7+2;
03209             *r5++ = r7[0]; *r5++ = r7[1];
03210             while ( i > 0 ) { *r5++ = *r8++; *r5++ = *r8++; *r5++ = -*r8++; i -= 3; }
03211             while ( t < rnext ) *r5++ = *t++;
03212             Normalize(BHEAD r9);
03213             r5 = r9 + *r9;
03214         }
03215         *r5 = 0;
03216         *argin3 = r5-argin3;
03217  /*
03218     We may have to sort the terms in argin3.
03219  */
03220         NewSort(BHEAD0);
03221         t = argin3+ARGHEAD;
03222         while ( *t ) {
03223             StoreTerm(BHEAD t);
03224             t += *t;
03225         }
03226         t = argin3+ARGHEAD;
03227         if ( EndSort(BHEAD t,0) < 0 ) goto Irreg;
03228         while ( *t ) t += *t;
03229         *argin3 = t - argin3;
03230         r3 = t;
03231  /*
03232     And now the factors that go to argout.
03233     First we have to move r7 out of the way.
03234  */
03235         r8 = r7+pow; i = r7[1];
03236         while ( --i >= 0 ) r8[i] = r7[i];
03237         r2 += pow;
03238         r8 += 2;
03239         while ( r8 < r2 ) {
03240             for ( i = ABS(r8[2]); i > 0; i-- ) {
03241                 *r1++ = ARGHEAD+9; *r1++ = 0; FILLARG(r1);
03242                 *r1++ = 9; *r1++ = DOTPRODUCT; *r1++ = 5; *r1++ = *r8;
03243                 *r1++ = r8[1]; *r1++ = r8[2] < 0 ? -1: 1;
03244                 *r1++ = 1; *r1++ = 1; *r1++ = 3;
03245             }
03246             r8 += 3;
03247         }
03248         argout = r1;
03249     }
03250 }
03251  /*
03252     #] DOTPRODUCT :
03253
03254     We have now in argin3 the argument stripped of negative powers and
03255     common factors. The only thing left to deal with is to make the
03256     coefficients integer. For that we have to find the LCM of the denominators
03257     and the GCD of the numerators. And to start with, the sign.
03258     We force the sign of the first term to be positive.
03259  */
03260     t = argin3 + ARGHEAD; pow = 1;
03261     t += *t;
03262     if ( t[-1] < 0 ) {
03263         pow = 0;
03264         t[-1] = -t[-1];

```

```

03265         while ( t < r3 ) {
03266             t += *t; t[-1] = -t[-1];
03267         }
03268     }
03269 /*
03270 Now the GCD of the numerators and the LCM of the denominators:
03271 */
03272     argfree = TermMalloc("TakeArgContent1");
03273     if ( AN.cmod != 0 ) {
03274         r1 = MakeMod(BHEAD argin3,r1,argfree);
03275     }
03276     else {
03277         r1 = MakeInteger(BHEAD argin3,r1,argfree);
03278     }
03279     if ( pow == 0 ) {
03280         *r1++ = -SNUMBER;
03281         *r1++ = -1;
03282     }
03283     *r1 = 0;
03284 /*
03285 Cleanup
03286 */
03287     if ( argin3 != argin2 ) TermFree(argin3,"TakeArgContent3");
03288     if ( argin2 != argin ) TermFree(argin2,"TakeArgContent2");
03289     return(argfree);
03290 Irreg:
03291     MesPrint("Irregularity while sorting argument in TakeArgContent");
03292     if ( argin3 != argin2 ) TermFree(argin3,"TakeArgContent3");
03293     if ( argin2 != argin ) TermFree(argin2,"TakeArgContent2");
03294     Terminate(-1);
03295     return(0);
03296 }
03297
03298 /*
03299 #] TakeArgContent :
03300 #[ MakeInteger :
03301 */
03312 WORD *MakeInteger(PHEAD WORD *argin,WORD *argout,WORD *argfree)
03313 {
03314     UWORD *GCDBuffer, *GCDBuffer2, *LCMBuffer, *LCMb, *LCMc;
03315     WORD *r, *r1, *r2, *r3, *r4, *r5, *rnext, i, k, j;
03316     WORD kGCD, kLCM, kGCD2, kkLCM, jLCM, jGCD;
03317     GCDBuffer = NumberMalloc("MakeInteger");
03318     GCDBuffer2 = NumberMalloc("MakeInteger");
03319     LCMBuffer = NumberMalloc("MakeInteger");
03320     LCMb = NumberMalloc("MakeInteger");
03321     LCMc = NumberMalloc("MakeInteger");
03322     r4 = argin + *argin;
03323     r = argin + ARGHEAD;
03324 /*
03325 First take the first term to load up the LCM and the GCD
03326 */
03327     r2 = r + *r;
03328     j = r2[-1];
03329     r3 = r2 - ABS(j);
03330     k = REDLENG(j);
03331     if ( k < 0 ) k = -k;
03332     while ( ( k > 1 ) && ( r3[k-1] == 0 ) ) k--;
03333     for ( kGCD = 0; kGCD < k; kGCD++ ) GCDBuffer[kGCD] = r3[kGCD];
03334     k = REDLENG(j);
03335     if ( k < 0 ) k = -k;
03336     r3 += k;
03337     while ( ( k > 1 ) && ( r3[k-1] == 0 ) ) k--;
03338     for ( kLCM = 0; kLCM < k; kLCM++ ) LCMBuffer[kLCM] = r3[kLCM];
03339     r1 = r2;
03340 /*
03341 Now go through the rest of the terms in this argument.
03342 */
03343     while ( r1 < r4 ) {
03344         r2 = r1 + *r1;
03345         j = r2[-1];
03346         r3 = r2 - ABS(j);
03347         k = REDLENG(j);
03348         if ( k < 0 ) k = -k;
03349         while ( ( k > 1 ) && ( r3[k-1] == 0 ) ) k--;
03350         if ( ( GCDBuffer[0] == 1 ) && ( kGCD == 1 ) ) {
03351 /*
03352 GCD is already 1
03353 */
03354         }
03355         else if ( ( k != 1 ) || ( r3[0] != 1 ) ) {
03356             if ( GcdLong(BHEAD GCDBuffer,kGCD,(UWORD *)r3,k,GCDBuffer2,&kGCD2) ) {
03357                 NumberFree(GCDBuffer,"MakeInteger");
03358                 NumberFree(GCDBuffer2,"MakeInteger");
03359                 NumberFree(LCMBuffer,"MakeInteger");
03360                 NumberFree(LCMb,"MakeInteger"); NumberFree(LCMc,"MakeInteger");
03361                 goto MakeIntegerErr;

```

```

03362         }
03363         kGCD = kGCD2;
03364         for ( i = 0; i < kGCD; i++ ) GCDbuffer[i] = GCDbuffer2[i];
03365     }
03366     else {
03367         kGCD = 1; GCDbuffer[0] = 1;
03368     }
03369     k = REDLENG(j);
03370     if ( k < 0 ) k = -k;
03371     r3 += k;
03372     while ( ( k > 1 ) && ( r3[k-1] == 0 ) ) k--;
03373     if ( ( ( LCMbuffer[0] == 1 ) && ( kLCM == 1 ) ) ) {
03374         for ( kLCM = 0; kLCM < k; kLCM++ )
03375             LCMbuffer[kLCM] = r3[kLCM];
03376     }
03377     else if ( ( k != 1 ) || ( r3[0] != 1 ) ) {
03378         if ( GcdLong(BHEAD LCMbuffer, kLCM, (UWORD *)r3, k, LCMb, &kLCM) ) {
03379             NumberFree(GCDbuffer, "MakeInteger"); NumberFree(GCDbuffer2, "MakeInteger");
03380             NumberFree(LCMbuffer, "MakeInteger"); NumberFree(LCMb, "MakeInteger");
03381             NumberFree(LCMc, "MakeInteger");
03382             goto MakeIntegerErr;
03383         }
03384         DivLong((UWORD *)r3, k, LCMb, kLCM, LCMb, &kLCM, LCMc, &jLCM);
03385         Mullong(LCMbuffer, kLCM, LCMb, kLCM, LCMc, &jLCM);
03386         for ( kLCM = 0; kLCM < jLCM; kLCM++ )
03387             LCMbuffer[kLCM] = LCMc[kLCM];
03388     }
03389     else {} /* LCM doesn't change */
03390     r1 = r2;
03391 }
03392 /*
03393  Now put the factor together: GCD/LCM
03394 */
03395 r3 = (WORD *) (GCDbuffer);
03396 if ( kGCD == kLCM ) {
03397     for ( jGCD = 0; jGCD < kGCD; jGCD++ )
03398         r3[jGCD+kGCD] = LCMbuffer[jGCD];
03399     k = kGCD;
03400 }
03401 else if ( kGCD > kLCM ) {
03402     for ( jGCD = 0; jGCD < kLCM; jGCD++ )
03403         r3[jGCD+kGCD] = LCMbuffer[jGCD];
03404     for ( jGCD = kLCM; jGCD < kGCD; jGCD++ )
03405         r3[jGCD+kGCD] = 0;
03406     k = kGCD;
03407 }
03408 else {
03409     for ( jGCD = kGCD; jGCD < kLCM; jGCD++ )
03410         r3[jGCD] = 0;
03411     for ( jGCD = 0; jGCD < kLCM; jGCD++ )
03412         r3[jGCD+kLCM] = LCMbuffer[jGCD];
03413     k = kLCM;
03414 }
03415 j = 2*k+1;
03416 /*
03417  Now we have to write this to argout
03418 */
03419 if ( ( j == 3 ) && ( r3[1] == 1 ) && ( (WORD) (r3[0]) > 0 ) ) {
03420     *argout = -SNUMBER;
03421     argout[1] = r3[0];
03422     r1 = argout+2;
03423 }
03424 else {
03425     r1 = argout;
03426     *r1++ = j+1+ARGHEAD; *r1++ = 0; FILLARG(r1);
03427     *r1++ = j+1; r2 = r3;
03428     for ( i = 0; i < k; i++ ) { *r1++ = *r2++; *r1++ = *r2++; }
03429     *r1++ = j;
03430 }
03431 /*
03432  Next we have to take the factor out from the argument.
03433  This cannot be done in location, because the denominator stuff can make
03434  coefficients longer.
03435 */
03436 r2 = argfree + 2; FILLARG(r2)
03437 while ( r < r4 ) {
03438     rnext = r + *r;
03439     j = ABS(rnext[-1]);
03440     r5 = rnext - j;
03441     r3 = r2;
03442     while ( r < r5 ) *r2++ = *r++;
03443     j = (j-1)/2; /* reduced length. Remember, k is the other red length */
03444     if ( DivRat(BHEAD (UWORD *)r5, j, GCDbuffer, k, (UWORD *)r2, &i) ) {
03445         goto MakeIntegerErr;
03446     }
03447     i = 2*i+1;
03448     r2 = r2 + i;

```

```

03448         if ( rnext[-1] < 0 ) r2[-1] = -i;
03449     else r2[-1] = i;
03450     *r3 = r2-r3;
03451     r = rnext;
03452 }
03453 *r2 = 0;
03454 argfree[0] = r2-argfree;
03455 argfree[1] = 0;
03456 /*
03457 Cleanup
03458 */
03459 NumberFree(LCMC,"MakeInteger");
03460 NumberFree(LCMb,"MakeInteger");
03461 NumberFree(LCMbuffer,"MakeInteger");
03462 NumberFree(GCDbuffer2,"MakeInteger");
03463 NumberFree(GCDbuffer,"MakeInteger");
03464 return(r1);
03465
03466 MakeIntegerErr:
03467 MesCall("MakeInteger");
03468 Terminate(-1);
03469 return(0);
03470 }
03471
03472 /*
03473 #] MakeInteger :
03474 #[ MakeMod :
03475 */
03483 WORD *MakeMod(PHEAD WORD *argin,WORD *argout,WORD *argfree)
03484 {
03485     WORD *r, *instop, *r1, *m, x, xx, ix, ip;
03486     int i;
03487     r = argin; instop = r + *r; r += ARGHEAD;
03488     x = r[*r-3];
03489     if ( r[*r-1] < 0 ) x += AN.cmod[0];
03490     if ( GetModInverses(x,(WORD)(AN.cmod[0]),&ix,&ip) ) {
03491         Terminate(-1);
03492     }
03493     argout[0] = -SNUMBER;
03494     argout[1] = x;
03495     argout[2] = 0;
03496     r1 = argout+2;
03497 /*
03498     Now we have to multiply all coefficients by ix.
03499     This does not make things longer, but we should keep to the conventions
03500     of MakeInteger.
03501 */
03502     m = argfree + ARGHEAD;
03503     while ( r < instop ) {
03504         xx = r[*r-3]; if ( r[*r-1] < 0 ) xx += AN.cmod[0];
03505         xx = (WORD)((((LONG)xx)*ix) % AN.cmod[0]);
03506         if ( xx != 0 ) {
03507             i = *r; NCOPY(m,r,i);
03508             m[-3] = xx; m[-1] = 3;
03509         }
03510         else { r += *r; }
03511     }
03512     *m = 0;
03513     *argfree = m - argfree;
03514     argfree[1] = 0;
03515     argfree += 2; FILLARG(argfree);
03516     return(r1);
03517 }
03518
03519 /*
03520 #] MakeMod :
03521 #[ SortWeights :
03522 */
03528 void SortWeights(LONG *weights, LONG *extraspace, WORD number)
03529 {
03530     LONG w, *fill, *from1, *from2;
03531     int n1,n2,i;
03532     if ( number >= 4 ) {
03533         n1 = number/2; n2 = number - n1;
03534         SortWeights(weights,extraspace,n1);
03535         SortWeights(weights+n1,extraspace,n2);
03536 /*
03537         We copy the first patch to the extra space. Then we merge
03538         Note that a potential remaining n2 objects are already in place.
03539 */
03540         for ( i = 0; i < n1; i++ ) extraspace[i] = weights[i];
03541         fill = weights; from1 = extraspace; from2 = weights+n1;
03542         while ( n1 > 0 && n2 > 0 ) {
03543             if ( *from1 <= *from2 ) { *fill++ = *from1++; n1--; }
03544             else { *fill++ = *from2++; n2--; }
03545         }
03546         while ( n1 > 0 ) { *fill++ = *from1++; n1--; }

```



```

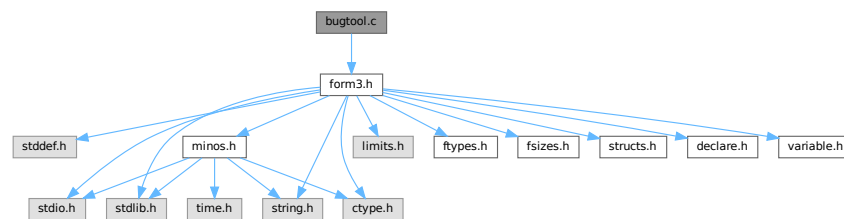
03547     }
03548     /*
03549     Special cases
03550     */
03551     else if ( number == 3 ) { /* 6 permutations of which one is trivial */
03552         if ( weights[0] > weights[1] ) {
03553             if ( weights[1] > weights[2] ) {
03554                 w = weights[0]; weights[0] = weights[2]; weights[2] = w;
03555             }
03556             else if ( weights[0] > weights[2] ) {
03557                 w = weights[0]; weights[0] = weights[1];
03558                 weights[1] = weights[2]; weights[2] = w;
03559             }
03560             else {
03561                 w = weights[0]; weights[0] = weights[1]; weights[1] = w;
03562             }
03563         }
03564         else if ( weights[0] > weights[2] ) {
03565             w = weights[0]; weights[0] = weights[2];
03566             weights[2] = weights[1]; weights[1] = w;
03567         }
03568         else if ( weights[1] > weights[2] ) {
03569             w = weights[1]; weights[1] = weights[2]; weights[2] = w;
03570         }
03571     }
03572     else if ( number == 2 ) {
03573         if ( weights[0] > weights[1] ) {
03574             w = weights[0]; weights[0] = weights[1]; weights[1] = w;
03575         }
03576     }
03577     return;
03578 }
03579
03580 /*
03581 #] SortWeights :
03582 */

```

4.3 bugtool.c File Reference

#include "form3.h"

Include dependency graph for bugtool.c:



Functions

- void [ExprStatus](#) ([EXPRESSIONS](#) e)

4.3.1 Detailed Description

Low level routines for debugging

Definition in file [bugtool.c](#).

4.3.2 Function Documentation

4.3.2.1 ExprStatus()

```
void ExprStatus (
    EXPRESSIONS e )
```

Definition at line 66 of file [bugtool.c](#).

4.4 bugtool.c

[Go to the documentation of this file.](#)

```
00001
00006 /* #[] License : */
00007 /*
00008 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00009 * When using this file you are requested to refer to the publication
00010 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00011 * This is considered a matter of courtesy as the development was paid
00012 * for by FOM the Dutch physics granting agency and we would like to
00013 * be able to track its scientific use to convince FOM of its value
00014 * for the community.
00015 *
00016 * This file is part of FORM.
00017 *
00018 * FORM is free software: you can redistribute it and/or modify it under the
00019 * terms of the GNU General Public License as published by the Free Software
00020 * Foundation, either version 3 of the License, or (at your option) any later
00021 * version.
00022 *
00023 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00024 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00025 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00026 * details.
00027 *
00028 * You should have received a copy of the GNU General Public License along
00029 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00030 */
00031 /* #[] License : */
00032
00033 /*
00034 #[] Includes :
00035 */
00036
00037 #include "form3.h"
00038
00039 /*
00040 #[] Includes :
00041 #[] ExprStatus :
00042 */
00043
00044 static UBYTE *statusexpr[] = {
00045     (UBYTE *) "LOCALEXPRESSION"
00046     , (UBYTE *) "SKIPLEXPRESSION"
00047     , (UBYTE *) "DROPLEXPRESSION"
00048     , (UBYTE *) "DROPPEDEXPRESSION"
00049     , (UBYTE *) "GLOBALEXPRESSION"
00050     , (UBYTE *) "SKIPGEXPRESSION"
00051     , (UBYTE *) "DROPGEXPRESSION"
00052     , (UBYTE *) "UNKNOWN"
00053     , (UBYTE *) "STOREDEXPRESSION"
00054     , (UBYTE *) "HIDDENLEXPRESSION"
00055     , (UBYTE *) "HIDELEXPRESSION"
00056     , (UBYTE *) "DROPHLEXPRESSION"
00057     , (UBYTE *) "UNHIDELEXPRESSION"
00058     , (UBYTE *) "HIDDENGEXPRESSION"
00059     , (UBYTE *) "HIDEGEXPRESSION"
00060     , (UBYTE *) "DROPHGEXPRESSION"
00061     , (UBYTE *) "UNHIDEGEXPRESSION"
00062     , (UBYTE *) "INTOHIDELEXPRESSION"
00063     , (UBYTE *) "INTOHIDEGEXPRESSION"
00064 };
00065
00066 void ExprStatus(EXPRESSIONS e)
00067 {
00068     MesPrint("Expression %s(%d) has status %s(%d,%d). Buffer: %d, Position: %15p",
```

```

00069      AC.exprnames->namebuffer+e->name, (WORD) (e->Expressions),
00070      statusexpr[e->status], e->status, e->hidelevel,
00071      e->whichbuffer, &(e->onfile));
00072  }
00073
00074  /*
00075   #] ExprStatus :
00076   */

```

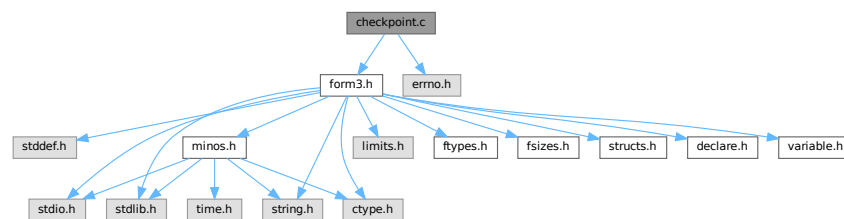
4.5 checkpoint.c File Reference

```

#include "form3.h"
#include <errno.h>

```

Include dependency graph for checkpoint.c:



Macros

- #define [CACHED_SNAPSHOT](#)
- #define [CACHE_SIZE](#) 4096
- #define [R_FREE](#)(ARG) if (ARG) M_free(ARG, #ARG);
- #define [R_FREE_NAMETREE](#)(ARG)
- #define [R_FREE_STREAM](#)(ARG)
- #define [R_SET](#)(VAR, TYPE) VAR = *((TYPE*)p); p = (unsigned char*)p + sizeof(TYPE);
- #define [R_COPY_B](#)(VAR, SIZE, CAST)
- #define [S_WRITE_B](#)(BUF, LEN) if (fwrite_cached(BUF, 1, LEN, fd) != (size_t)(LEN)) return(__LINE__);
- #define [S_FLUSH_B](#) if (flush_cache(fd) != 1) return(__LINE__);
- #define [R_COPY_S](#)(VAR, CAST)
- #define [S_WRITE_S](#)(STR)
- #define [R_COPY_LIST](#)(ARG)
- #define [S_WRITE_LIST](#)(LST)
- #define [R_COPY_NAMETREE](#)(ARG)
- #define [S_WRITE_NAMETREE](#)(ARG)
- #define [S_WRITE_DOLLAR](#)(ARG)
- #define [ANNOUNCE](#)(str)

Functions

- int [CheckRecoveryFile](#) (void)
- void [DeleteRecoveryFile](#) (void)
- char * [RecoveryFilename](#) (void)
- void [InitRecovery](#) (void)
- size_t [fwrite_cached](#) (const void *ptr, size_t size, size_t nmemb, FILE *fd)
- size_t [flush_cache](#) (FILE *fd)
- int [DoRecovery](#) (int *moduletype)
- void [DoCheckpoint](#) (int moduletype)

Variables

- unsigned char `cache_buffer` [CACHE_SIZE]
- size_t `cache_fill` = 0

4.5.1 Detailed Description

Contains all functions that deal with the recovery mechanism controlled and activated by the On Checkpoint switch.

The main function are DoCheckpoint, DoRecovery, and DoSnapshot. If the checkpoints are activated DoCheckpoint is called every time a module is finished executing. If the conditions for the creation of a recovery snapshot are met DoCheckpoint calls DoSnapshot. DoRecovery is called once when FORM starts up with the command line argument -R. Most of the other code contains debugging facilities that are only compiled if the macro PRINTDEBUG is defined.

The recovery mechanism is atomic, i.e. only if everything went well, the final recovery file is created (and the older one overwritten) in a single step (copying). If some errors occur, a warning is issued and the program continues without having created a new recovery file. The only situation in which the creation of the recovery data leads to a termination of the running program is if not enough disk or memory space is left.

For ParFORM each slave creates its own recovery file, sends it to the master and then it deletes the recovery file. The master stores all the recovery files and on recovery it feeds these files to the slaves. It is nearly impossible to recover after some MPI fault so ParFORM terminates on any recovery failure.

DoRecovery and DoSnapshot do the loading and saving of the recovery data, respectively. Every change in one functions needs to be accompanied by the appropriate change in the other function. The structure of both functions is quite similar. They handle the relevant global structs one after the other and then care about the copying of the hide and scratch files.

The names of the recovery, scratch and hide files are hard-coded in the variables in fold "filenames and system commands".

If the global structs AM,AP,AC,AR are changed, DoRecovery and DoSnapshot usually also have to be changed. Some structs are read/written as a whole (AP,AC), some are read/written only partly as a selection of their individual elements (AM,AR). If AM or AR have been changed by adding or removing an element that is important for the runtime status, then the reading/writing statements have to be added to or removed from DoRecovery and DoSnapshot. If AP or AC are changed, then for non-pointer variables (in the case of a struct it also means that none of its elements is a pointer) nothing has to be changed in the functions here. If pointers are involved, extra code has to be added (or removed). See the comments of DoRecovery and DoSnapshot.

Definition in file [checkpoint.c](#).

4.5.2 Macro Definition Documentation

4.5.2.1 CACHED_SNAPSHOT

```
#define CACHED_SNAPSHOT
```

Definition at line 1214 of file [checkpoint.c](#).

4.5.2.2 CACHE_SIZE

```
#define CACHE_SIZE 4096
```

Definition at line 1216 of file [checkpoint.c](#).

4.5.2.3 R_FREE

```
#define R_FREE(  
    ARG )    if ( ARG ) M_free(ARG, #ARG);
```

Definition at line 1281 of file [checkpoint.c](#).

4.5.2.4 R_FREE_NAMETREE

```
#define R_FREE_NAMETREE(  
    ARG )
```

Value:

```
R_FREE(ARG->namenode); \  
R_FREE(ARG->namebuffer); \  
R_FREE(ARG);
```

Definition at line 1284 of file [checkpoint.c](#).

4.5.2.5 R_FREE_STREAM

```
#define R_FREE_STREAM(  
    ARG )
```

Value:

```
R_FREE(ARG.buffer); \  
R_FREE(ARG.FoldName); \  
R_FREE(ARG.name);
```

Definition at line 1289 of file [checkpoint.c](#).

4.5.2.6 R_SET

```
#define R_SET(  
    VAR,  
    TYPE )    VAR = *((TYPE*)p); p = (unsigned char*)p + sizeof(TYPE);
```

Definition at line 1296 of file [checkpoint.c](#).

4.5.2.7 R_COPY_B

```
#define R_COPY_B(  
    VAR,  
    SIZE,  
    CAST )
```

Value:

```
VAR = (CAST)Malloc1(SIZE, #VAR); \  
memcpy(VAR, p, SIZE); p = (unsigned char*)p + SIZE;
```

Definition at line 1301 of file [checkpoint.c](#).

4.5.2.8 S_WRITE_B

```
#define S_WRITE_B(
    BUF,
    LEN ) if ( fwrite_cached(BUF, 1, LEN, fd) != (size_t) (LEN) ) return(__LINE__);
```

Definition at line 1305 of file [checkpoint.c](#).

4.5.2.9 S_FLUSH_B

```
#define S_FLUSH_B if ( flush_cache(fd) != 1 ) return(__LINE__);
```

Definition at line 1308 of file [checkpoint.c](#).

4.5.2.10 R_COPY_S

```
#define R_COPY_S(
    VAR,
    CAST )
```

Value:

```
if ( VAR ) { \
    VAR = (CAST)Malloc1(strlen(p)+1,"R_COPY_S"); \
    strcpy((char*)VAR, p); p = (unsigned char*)p + strlen(p) + 1; \
}
```

Definition at line 1313 of file [checkpoint.c](#).

4.5.2.11 S_WRITE_S

```
#define S_WRITE_S(
    STR )
```

Value:

```
if ( STR ) { \
    l = strlen((char*)STR) + 1; \
    if ( fwrite_cached(STR, 1, l, fd) != (size_t)l ) return(__LINE__); \
}
```

Definition at line 1319 of file [checkpoint.c](#).

4.5.2.12 R_COPY_LIST

```
#define R_COPY_LIST(
    ARG )
```

Value:

```
if ( ARG.maxnum ) { \
    R_COPY_B(ARG.lijst, ARG.size*ARG.maxnum, void*) \
}
```

Definition at line 1327 of file [checkpoint.c](#).

4.5.2.13 S_WRITE_LIST

```
#define S_WRITE_LIST(
    LST )
```

Value:

```
if ( LST.maxnum ) { \
    S_WRITE_B((char*)LST.lijst, LST.maxnum*LST.size) \
}
```

Definition at line 1332 of file [checkpoint.c](#).

4.5.2.14 R_COPY_NAMETREE

```
#define R_COPY_NAMETREE(
    ARG )
```

Value:

```
R_COPY_B(ARG, sizeof(NAMETREE), NAMETREE*); \
if ( ARG->namenode ) { \
    R_COPY_B(ARG->namenode, ARG->nodesize*sizeof(NAMENODE), NAMENODE*); \
} \
if ( ARG->namebuffer ) { \
    R_COPY_B(ARG->namebuffer, ARG->namesize, UBYTE*); \
}
```

Definition at line 1339 of file [checkpoint.c](#).

4.5.2.15 S_WRITE_NAMETREE

```
#define S_WRITE_NAMETREE(
    ARG )
```

Value:

```
S_WRITE_B(ARG, sizeof(NAMETREE)); \
if ( ARG->namenode ) { \
    S_WRITE_B(ARG->namenode, ARG->nodesize*sizeof(struct NaMeNode)); \
} \
if ( ARG->namebuffer ) { \
    S_WRITE_B(ARG->namebuffer, ARG->namesize); \
}
```

Definition at line 1348 of file [checkpoint.c](#).

4.5.2.16 S_WRITE_DOLLAR

```
#define S_WRITE_DOLLAR(
    ARG )
```

Value:

```
if ( ARG.size && ARG.where && ARG.where != &(AM.dollarzero) ) { \
    S_WRITE_B(ARG.where, ARG.size*sizeof(WORD)) \
}
```

Definition at line 1359 of file [checkpoint.c](#).

4.5.2.17 ANNOUNCE

```
#define ANNOUNCE(  
    str )
```

Definition at line 1370 of file [checkpoint.c](#).

4.5.3 Function Documentation

4.5.3.1 CheckRecoveryFile()

```
int CheckRecoveryFile (  
    void )
```

Checks whether a snapshot/recovery file exists. Returns 1 if it exists, 0 otherwise.

Definition at line 278 of file [checkpoint.c](#).

References [RecoveryFilename\(\)](#).

Here is the call graph for this function:



4.5.3.2 DeleteRecoveryFile()

```
void DeleteRecoveryFile (  
    void )
```

Deletes the recovery files. It is called by `CleanUp()` in the case of a successful completion.

Definition at line 333 of file [checkpoint.c](#).

4.5.3.3 RecoveryFilename()

```
char * RecoveryFilename (  
    void )
```

Returns pointer to recovery filename.

Definition at line 364 of file [checkpoint.c](#).

Referenced by [CheckRecoveryFile\(\)](#), and [DoRecovery\(\)](#).

4.5.3.4 InitRecovery()

```
void InitRecovery (
    void )
```

Sets up the strings for the filenames of the recovery files. This functions should only be called once to avoid memory leaks and after AM.TempDir has been initialized.

Definition at line 399 of file [checkpoint.c](#).

4.5.3.5 fwrite_cached()

```
size_t fwrite_cached (
    const void * ptr,
    size_t size,
    size_t nmemb,
    FILE * fd )
```

Definition at line 1222 of file [checkpoint.c](#).

4.5.3.6 flush_cache()

```
size_t flush_cache (
    FILE * fd )
```

Definition at line 1247 of file [checkpoint.c](#).

4.5.3.7 DoRecovery()

```
int DoRecovery (
    int * moduletype )
```

Reads from the recovery file and restores all necessary variables and states in FORM, so that the execution can recommence in preprocessor() as if no restart of FORM had occurred.

The recovery file is read into memory as a whole. The pointer p then points into this memory at the next non-processed data. The macros by which variables are restored, like R_SET, automatically increase p appropriately.

If something goes wrong, the function returns with a non-zero value.

Allocated memory that would be lost when overwriting the global structs with data from the file is freed first. A major part of the code deals with the restoration of pointers. The idiom we use is to memorize the original pointer value (org), allocate new memory and copy the data from the file into this memory, calculate the offset between the old pointer value and the new allocated memory position (ofs), and then correct all affected pointers (+=ofs).

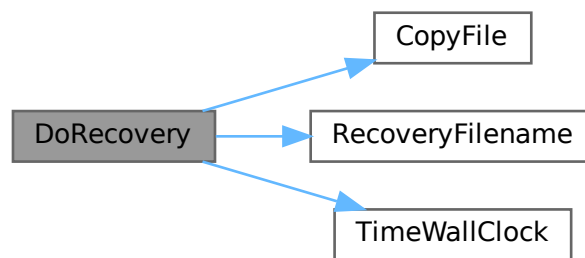
We rely on the fact that several variables (especially in AM) are already assigned the correct values by the startup functions. That means, in principle, that a change in the setup files between snapshot creation and recovery will be noticed.

Definition at line 1402 of file [checkpoint.c](#).

References [TaBIEs::argtail](#), [TaBIEs::boomlijst](#), [BrAcKeTiNfO::bracketbuffer](#), [TaBIEs::buffers](#), [TaBIEs::bufferssize](#), [CopyFile\(\)](#), [TaBIEs::flags](#), [ReNuMbEr::func](#), [ReNuMbEr::funnum](#), [VaRrEnUm::hi](#), [BrAcKeTiNfO::indexbuffer](#),

[ReNuMbEr::indi](#), [ReNuMbEr::indnum](#), [VaRrEnUm::lo](#), [TaBIes::MaxTreeSize](#), [TaBIes::mm](#), [TaBIes::numind](#), [TaBIes::pattern](#), [TaBIes::prototype](#), [TaBIes::prototypeSize](#), [RecoveryFilename\(\)](#), [ExPrEsSiOn::renumlists](#), [TaBIes::reserved](#), [TaBIes::spare](#), [TaBIes::sparse](#), [VaRrEnUm::start](#), [ReNuMbEr::symb](#), [ReNuMbEr::symnum](#), [TaBIes::tablepointers](#), [TimeWallClock\(\)](#), [TaBIes::totind](#), [ReNuMbEr::vecnum](#), and [ReNuMbEr::vect](#).

Here is the call graph for this function:



4.5.3.8 DoCheckpoint()

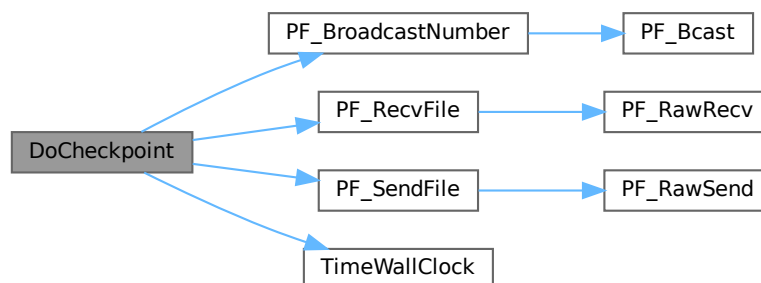
```
void DoCheckpoint (
    int moduletype )
```

Checks whether a snapshot should be done. Calls `DoSnapshot()` to create the snapshot.

Definition at line 3111 of file [checkpoint.c](#).

References [PF_BroadcastNumber\(\)](#), [PF_RecvFile\(\)](#), [PF_SendFile\(\)](#), and [TimeWallClock\(\)](#).

Here is the call graph for this function:



4.5.4 Variable Documentation

4.5.4.1 cache_buffer

```
unsigned char cache_buffer[CACHE_SIZE]
```

Definition at line 1219 of file [checkpoint.c](#).

4.5.4.2 cache_fill

```
size_t cache_fill = 0
```

Definition at line 1220 of file [checkpoint.c](#).

4.6 checkpoint.c

[Go to the documentation of this file.](#)

```
00001 /*
00002      #[ Explanations :
00003 */
00052 /*
00053      #[ Explanations :
00054      #[ License :
00055 *
00056 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00057 *   When using this file you are requested to refer to the publication
00058 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00059 *   This is considered a matter of courtesy as the development was paid
00060 *   for by FOM the Dutch physics granting agency and we would like to
00061 *   be able to track its scientific use to convince FOM of its value
00062 *   for the community.
00063 *
00064 *   This file is part of FORM.
00065 *
00066 *   FORM is free software: you can redistribute it and/or modify it under the
00067 *   terms of the GNU General Public License as published by the Free Software
00068 *   Foundation, either version 3 of the License, or (at your option) any later
00069 *   version.
00070 *
00071 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00072 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00073 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00074 *   details.
00075 *
00076 *   You should have received a copy of the GNU General Public License along
00077 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00078 */
00079 /*
00080      #[ License :
00081      #[ Includes :
00082 */
00083
00084 #include "form3.h"
00085
00086 #include <errno.h>
00087
00088 /*
00089 #define PRINTDEBUG
00090 */
00091
00092 /*
00093 #define PRINTTIMEMARKS
00094 */
00095
00096 /*
00097      #[ Includes :
00098      #[ filenames and system commands :
00099 */
00100
00104 #ifndef WITHMPI
00105 #define BASENAME_FMT "%c%04dFORMrecv"
```

```

00111 static char BaseName[] = BASENAME_FMT;
00112 #else
00113 static char *BaseName = "FORMrecv";
00114 #endif
00118 static char *recoveryfile = 0;
00124 static char *intermedfile = 0;
00128 static char *sortfile = 0;
00132 static char *hidefile = 0;
00136 static char *storefile = 0;
00137
00142 static int done_snapshot = 0;
00143
00144 #ifdef WITHMPI
00149 static int PF_fmt_pos;
00150
00155 static const char *PF_recoveryfile(char prefix, int id, int intermed)
00156 {
00157     /*
00158      * Assume that InitRecovery() has been already called, namely
00159      * recoveryfile, intermedfile and PF_fmt_pos are already initialized.
00160      */
00161     static char *tmp_recovery = NULL;
00162     static char *tmp_intermed = NULL;
00163     char *tmp, c;
00164     if ( tmp_recovery == NULL ) {
00165         if ( PF.numtasks > 9999 ) { /* see BASENAME_FMT */
00166             MesPrint("Checkpoint: too many number of processors.");
00167             Terminate(-1);
00168         }
00169         tmp_recovery = (char *)Malloc1(strlen(recoveryfile) + strlen(intermedfile) + 2,
"PF_recoveryfile");
00170         tmp_intermed = tmp_recovery + strlen(recoveryfile) + 1;
00171         strcpy(tmp_recovery, recoveryfile);
00172         strcpy(tmp_intermed, intermedfile);
00173     }
00174     tmp = intermed ? tmp_intermed : tmp_recovery;
00175     c = tmp[PF_fmt_pos + 13]; /* The magic number 13 comes from BASENAME_FMT. */
00176     sprintf(tmp + PF_fmt_pos, BASENAME_FMT, prefix, id);
00177     tmp[PF_fmt_pos + 13] = c;
00178     return tmp;
00179 }
00180 #endif
00181
00182 /*
00183  #] filenames and system commands :
00184  #[ CheckRecoveryFile :
00185  */
00186
00191 #ifdef WITHMPI
00192
00198 static int PF_CheckRecoveryFile()
00199 {
00200     int i,ret=0;
00201     FILE *fd;
00202     /* Check if the recovery file for the master exists. */
00203     if ( PF.me == MASTER ) {
00204         if ( (fd = fopen(recoveryfile, "r")) ) {
00205             fclose(fd);
00206             PF_BroadcastNumber(1);
00207         }
00208         else {
00209             PF_BroadcastNumber(0);
00210             return 0;
00211         }
00212     }
00213     else {
00214         if ( !PF_BroadcastNumber(0) )
00215             return 0;
00216     }
00217     /* Now the main part. */
00218     if (PF.me == MASTER){
00219         /*We have to have recovery files for the master and all the slaves:*/
00220         for(i=1; i<PF.numtasks;i++){
00221             const char *tmpnam = PF_recoveryfile('m', i, 0);
00222             if ( (fd = fopen(tmpnam, "r")) )
00223                 fclose(fd);
00224             else
00225                 break;
00226         }/*for(i=0; i<PF.numtasks;i++)*/
00227         if(i!=PF.numtasks){/*some files are absent*/
00228             int j;
00229             /*Send all slaves failure*/
00230             for(j=1; j<PF.numtasks;j++){
00231                 ret=PF_SendFile(j, NULL);
00232                 if(ret<0)
00233                     return(-1);
00234             }

```

```

00235         if(i==0)
00236             return(0);/*Ok, no recovery files at all.*/
00237         /*The master recovery file exists but some slave files are absent*/
00238         MesPrint("The file %s exists but some of the slave recovery files are absent.",
00239                 RecoveryFilename());
00240         return(-1);
00241     }/*if(i!=PF.numtasks)*/
00242     /*All the files are here.*/
00243     /*Send all slaves success and files:*/
00244     for(i=1; i<PF.numtasks;i++){
00245         const char *tmpnam = PF_recoveryfile('m', i, 0);
00246         fd = fopen(tmpnam, "r");
00247         ret=PF_SendFile(i, fd);/*if fd==NULL, PF_SendFile seds to a slave the failure tag*/
00248         if(fd == NULL)
00249             return(-1);
00250         else
00251             fclose(fd);
00252         if(ret<0)
00253             return(-1);
00254     }/*for(i=0; i<PF.numtasks;i++)*/
00255     return(1);
00256 }/*if (PF.me == MASTER)*/
00257 /*Slave:*/
00258 /*Get the answer from the master:*/
00259 fd=fopen(recoveryfile,"w");
00260 if(fd == NULL) {
00261     MesPrint("Failed to open %s in write mode in process %w", recoveryfile);
00262     return(-1);
00263 }
00264 ret=PF_RecvFile(MASTER,fd);
00265 if(ret<0)
00266     return(-1);
00267 fclose(fd);
00268 if(ret==0){
00269     /*Nothing is found by the master*/
00270     remove(recoveryfile);
00271     return(0);
00272 }
00273 /*Recovery file is successfully transferred*/
00274 return(1);
00275 }
00276 #endif
00277
00278 int CheckRecoveryFile(void)
00279 {
00280     int ret = 0;
00281 #ifdef WITHMPI
00282     ret = PF_CheckRecoveryFile();
00283 #else
00284     FILE *fd;
00285     if ( ( fd = fopen(recoveryfile, "r") ) ) {
00286         fclose(fd);
00287         ret = 1;
00288     }
00289 #endif
00290     if ( ret < 0 ){/*In ParFORM CheckRecoveryFile() may return a fatal error.*/
00291         MesPrint("Fail checking recovery file");
00292         Terminate(-1);
00293     }
00294     else if ( ret > 0 ) {
00295         if ( AC.CheckpointFlag != -1 ) {
00296             /* recovery file exists but recovery option is not given */
00297 #ifdef WITHMPI
00298             if ( PF.me == MASTER ) {
00299 #endif
00300                 MesPrint("The recovery file %s exists, but the recovery option -R has not been given!",
00301                         RecoveryFilename());
00302                 MesPrint("FORM will be terminated to avoid unintentional loss of data.");
00303                 MesPrint("Delete the recovery file manually, if you want to start FORM without
00304 recovery.");
00305 #ifdef WITHMPI
00306             }
00307             if(PF.me != MASTER)
00308                 remove(RecoveryFilename());
00309 #endif
00310             Terminate(-1);
00311         }
00312         else {
00313             if ( AC.CheckpointFlag == -1 ) {
00314                 /* recovery option given but recovery file does not exist */
00315 #ifdef WITHMPI
00316                 if ( PF.me == MASTER )
00317 #endif
00318                 MesPrint("Option -R for recovery has been given, but the recovery file %s does not
00319 exist!", RecoveryFilename());
00320                 Terminate(-1);

```

```

00319     }
00320 }
00321 return(ret);
00322 }
00323
00324 /*
00325 #] CheckRecoveryFile :
00326 #[ DeleteRecoveryFile :
00327 */
00328
00333 void DeleteRecoveryFile(void)
00334 {
00335     if ( done_snapshot ) {
00336         remove(recoveryfile);
00337 #ifdef WITHMPI
00338         if( PF.me == MASTER){
00339             int i;
00340             for(i=1; i<PF.numtasks;i++){
00341                 const char *tmpnam = PF_recoveryfile('m', i, 0);
00342                 remove(tmpnam);
00343             }/*for(i=1; i<PF.numtasks;i++)*/
00344             remove(storefile);
00345             remove(sortfile);
00346             remove(hidefile);
00347         }/*if( PF.me == MASTER)*/
00348 #else
00349         remove(storefile);
00350         remove(sortfile);
00351         remove(hidefile);
00352 #endif
00353     }
00354 }
00355
00356 /*
00357 #] DeleteRecoveryFile :
00358 #[ RecoveryFilename :
00359 */
00360
00364 char *RecoveryFilename(void)
00365 {
00366     return(recoveryfile);
00367 }
00368
00369 /*
00370 #] RecoveryFilename :
00371 #[ InitRecovery :
00372 */
00373
00377 static char *InitName(char *str, char *ext)
00378 {
00379     char *s, *d = str;
00380     if ( AM.TempDir ) {
00381         s = (char*)AM.TempDir;
00382         while ( *s ) { *d++ = *s++; }
00383         *d++ = SEPARATOR;
00384     }
00385     s = BaseName;
00386     while ( *s ) { *d++ = *s++; }
00387     *d++ = '.';
00388     s = ext;
00389     while ( *s ) { *d++ = *s++; }
00390     *d++ = 0;
00391     return d;
00392 }
00393
00399 void InitRecovery(void)
00400 {
00401     int lenpath = AM.TempDir ? strlen((char*)AM.TempDir)+1 : 0;
00402 #ifdef WITHMPI
00403     sprintf(BaseName,BASENAME_FMT,(PF.me == MASTER)?'m':'s',PF.me);
00404     /*Now BaseName has a form ?XXXXFORMrecv where ? == 'm' for master and 's' for slave,
00405        XXXX is a zero - padded PF.me*/
00406     PF_fmt_pos = lenpath;
00407 #endif
00408     recoveryfile = (char*)Malloc1(5*(lenpath+strlen(BaseName)+4+1),"InitRecovery");
00409     intermedfile = InitName(recoveryfile, "tmp");
00410     sortfile      = InitName(intermedfile, "XXX");
00411     hidefile      = InitName(sortfile, "out");
00412     storefile     = InitName(hidefile, "hid");
00413     InitName(storefile, "str");
00414 }
00415
00416 /*
00417 #] InitRecovery :
00418 #[ Debugging :
00419 */
00420

```

```

00421 #ifdef PRINTDEBUG
00422
00423 void print_BYTE(void *p)
00424 {
00425     UBYTE h = (UBYTE) (*(UBYTE*)p) >> 4;
00426     UBYTE l = (UBYTE) (*(UBYTE*)p) & 0x0F;
00427     if ( h > 9 ) h += 55; else h += 48;
00428     if ( l > 9 ) l += 55; else l += 48;
00429     printf("%c%c ", h, l);
00430 }
00431
00432 static void print_STR(UBYTE *p)
00433 {
00434     if ( p ) {
00435         MesPrint("%s", (char*)p);
00436     }
00437     else {
00438         MesPrint("NULL");
00439     }
00440 }
00441
00442 static void print_WORDB(WORD *buf, WORD *top)
00443 {
00444     LONG size = top-buf;
00445     int i;
00446     while ( size > 0 ) {
00447         if ( size > MAXPOSITIVE ) i = MAXPOSITIVE;
00448         else i = size;
00449         size -= i;
00450         MesPrint("%a",i,buf);
00451         buf += i;
00452     }
00453 }
00454
00455 static void print_VOIDP(void *p, size_t size)
00456 {
00457     int i;
00458     if ( p ) {
00459         while ( size > 0 ) {
00460             if ( size > MAXPOSITIVE ) i = MAXPOSITIVE;
00461             else i = size;
00462             size -= i;
00463             MesPrint("%b",i,(UBYTE *)p);
00464             p = ((UBYTE *)p)+i;
00465         }
00466     }
00467     else {
00468         MesPrint("NULL");
00469     }
00470 }
00471
00472 static void print_CHARS(UBYTE *p, size_t size)
00473 {
00474     int i;
00475     while ( size > 0 ) {
00476         if ( size > MAXPOSITIVE ) i = MAXPOSITIVE;
00477         else i = size;
00478         size -= i;
00479         MesPrint("%C",i,(char *)p);
00480         p += i;
00481     }
00482 }
00483
00484 static void print_WORDV(WORD *p, size_t size)
00485 {
00486     int i;
00487     if ( p ) {
00488         while ( size > 0 ) {
00489             if ( size > MAXPOSITIVE ) i = MAXPOSITIVE;
00490             else i = size;
00491             size -= i;
00492             MesPrint("%a",i,p);
00493             p += i;
00494         }
00495     }
00496     else {
00497         MesPrint("NULL");
00498     }
00499 }
00500
00501 static void print_INTV(int *p, size_t size)
00502 {
00503     int iarray[8];
00504     WORD i = 0;
00505     if ( p ) {
00506         while ( size > 0 ) {
00507             if ( i >= 8 ) {

```

```

00508         MesPrint("%I",i,iarray);
00509         i = 0;
00510     }
00511     iarray[i++] = *p++;
00512     size--;
00513 }
00514 if ( i > 0 ) MesPrint("%I",i,iarray);
00515 }
00516 else {
00517     MesPrint("NULL");
00518 }
00519 }
00520
00521 static void print_LONGV(LONG *p, size_t size)
00522 {
00523     LONG larray[8];
00524     WORD i = 0;
00525     if ( p ) {
00526         while ( size > 0 ) {
00527             if ( i >= 8 ) {
00528                 MesPrint("%I",i,larray);
00529                 i = 0;
00530             }
00531             larray[i++] = *p++;
00532             size--;
00533         }
00534         if ( i > 0 ) MesPrint("%I",i,larray);
00535     }
00536     else {
00537         MesPrint("NULL");
00538     }
00539 }
00540
00541 static void print_PRELOAD(PRELOAD *l)
00542 {
00543     if ( l->size ) {
00544         print_CHARS(l->buffer, l->size);
00545     }
00546     MesPrint("%ld", l->size);
00547 }
00548
00549 static void print_PREVAR(PREVAR *l)
00550 {
00551     MesPrint("%s", l->name);
00552     print_STR(l->value);
00553     if ( l->nargs ) print_STR(l->argnames);
00554     MesPrint("%d", l->nargs);
00555     MesPrint("%d", l->wildarg);
00556 }
00557
00558 static void print_DOLLARS(DOLLARS l)
00559 {
00560     print_VOIDP(l->where, l->size);
00561     MesPrint("%ld", l->size);
00562     MesPrint("%ld", l->name);
00563     MesPrint("%s", AC.dollarnames->namebuffer+l->name);
00564     MesPrint("%d", l->type);
00565     MesPrint("%d", l->node);
00566     MesPrint("%d", l->index);
00567     MesPrint("%d", l->zero);
00568     MesPrint("%d", l->numdummies);
00569     MesPrint("%d", l->nfactors);
00570 }
00571
00572 static void print_LIST(LIST *l)
00573 {
00574     print_VOIDP(l->lijst, l->size);
00575     MesPrint("%s", l->message);
00576     MesPrint("%d", l->num);
00577     MesPrint("%d", l->maxnum);
00578     MesPrint("%d", l->size);
00579     MesPrint("%d", l->numglobal);
00580     MesPrint("%d", l->numtemp);
00581     MesPrint("%d", l->numclear);
00582 }
00583
00584 static void print_DOLOOP(DOLOOP *l)
00585 {
00586     print_PRELOAD(&(l->p));
00587     print_STR(l->name);
00588     if ( l->type != NUMERICALLOOP ) {
00589         print_STR(l->vars);
00590     }
00591     print_STR(l->contents);
00592     if ( l->type != LISTEDLOOP && l->type != NUMERICALLOOP ) {
00593         print_STR(l->dollarname);
00594     }

```



```

00595     MesPrint("%l", l->startlinenumber);
00596     MesPrint("%l", l->firstnum);
00597     MesPrint("%l", l->lastnum);
00598     MesPrint("%l", l->incnum);
00599     MesPrint("%d", l->type);
00600     MesPrint("%d", l->NoShowInput);
00601     MesPrint("%d", l->errorsinloop);
00602     MesPrint("%d", l->firstloopcall);
00603 }
00604
00605 static void print_PROCEDURE(PROCEDURE *l)
00606 {
00607     if ( l->loadmode != 1 ) {
00608         print_PRELOAD(&(l->p));
00609     }
00610     print_STR(l->name);
00611     MesPrint("%d", l->loadmode);
00612 }
00613
00614 static void print_NAMETREE(NAMETREE *t)
00615 {
00616     int i;
00617     for ( i=0; i<t->nodefill; ++i ) {
00618         MesPrint("%l %d %d %d %d %d %d\n", t->namenode[i].name,
00619             t->namenode[i].parent, t->namenode[i].left, t->namenode[i].right,
00620             t->namenode[i].balance, t->namenode[i].type, t->namenode[i].number );
00621     }
00622     print_CHARS(t->namebuffer, t->namefill);
00623     MesPrint("%l", t->namesize);
00624     MesPrint("%l", t->namefill);
00625     MesPrint("%l", t->nodesize);
00626     MesPrint("%l", t->nodefill);
00627     MesPrint("%l", t->oldnamefill);
00628     MesPrint("%l", t->oldnodefill);
00629     MesPrint("%l", t->globalnamefill);
00630     MesPrint("%l", t->globalnodefill);
00631     MesPrint("%l", t->clearnamefill);
00632     MesPrint("%l", t->clearnodefill);
00633     MesPrint("%d", t->headnode);
00634 }
00635
00636 void print_CBUF(CBUF *c)
00637 {
00638     int i;
00639     print_WORDV(c->Buffer, c->BufferSize);
00640     /*
00641     MesPrint("%x", c->Buffer);
00642     MesPrint("%x", c->lhs);
00643     MesPrint("%x", c->rhs);
00644     */
00645     for ( i=0; i<c->numlhs; ++i ) {
00646         if ( c->lhs[i] ) MesPrint("%d", *(c->lhs[i]));
00647     }
00648     for ( i=0; i<c->numrhs; ++i ) {
00649         if ( c->rhs[i] ) MesPrint("%d", *(c->rhs[i]));
00650     }
00651     MesPrint("%l", *c->CanCommu);
00652     MesPrint("%l", *c->NumTerms);
00653     MesPrint("%d", *c->numdum);
00654     for ( i=0; i<c->MaxTreeSize; ++i ) {
00655         MesPrint("%d %d %d %d %d", c->boomlijst[i].parent, c->boomlijst[i].left,
c->boomlijst[i].right,
00656             c->boomlijst[i].value, c->boomlijst[i].blnce);
00657     }
00658 }
00659
00660 static void print_STREAM(STREAM *t)
00661 {
00662     print_CHARS(t->buffer, t->inbuffer);
00663     MesPrint("%l", (LONG)(t->pointer-t->buffer));
00664     print_STR(t->FoldName);
00665     print_STR(t->name);
00666     if ( t->type == PREVARSTREAM || t->type == DOLLARSTREAM ) {
00667         print_STR(t->pname);
00668     }
00669     MesPrint("%l", (LONG)t->fileposition);
00670     MesPrint("%l", (LONG)t->linenumber);
00671     MesPrint("%l", (LONG)t->prevline);
00672     MesPrint("%l", t->buffersize);
00673     MesPrint("%l", t->bufferposition);
00674     MesPrint("%l", t->inbuffer);
00675     MesPrint("%d", t->previous);
00676     MesPrint("%d", t->handle);
00677     switch ( t->type ) {
00678         case FILESTREAM: MesPrint("%d == FILESTREAM", t->type); break;
00679         case PREVARSTREAM: MesPrint("%d == PREVARSTREAM", t->type); break;
00680         case PREREADSTREAM: MesPrint("%d == PREREADSTREAM", t->type); break;

```

```

00681         case PIPESTREAM: MesPrint("%d == PIPESTREAM", t->type); break;
00682         case PRECALCSTREAM: MesPrint("%d == PRECALCSTREAM", t->type); break;
00683         case DOLLARSTREAM: MesPrint("%d == DOLLARSTREAM", t->type); break;
00684         case PREREADSTREAM2: MesPrint("%d == PREREADSTREAM2", t->type); break;
00685         case EXTERNALCHANNELSTREAM: MesPrint("%d == EXTERNALCHANNELSTREAM", t->type); break;
00686         case PREREADSTREAM3: MesPrint("%d == PREREADSTREAM3", t->type); break;
00687         default: MesPrint("%d == UNKNOWN", t->type);
00688     }
00689 }
00690
00691 static void print_M()
00692 {
00693     MesPrint("%%% M_const");
00694     MesPrint("%d", *AM.gcmmod);
00695     MesPrint("%d", *AM.gpowmod);
00696     print_STR(AM.TempDir);
00697     print_STR(AM.TempSortDir);
00698     print_STR(AM.IncDir);
00699     print_STR(AM.InputFileName);
00700     print_STR(AM.LogFileName);
00701     print_STR(AM.OutBuffer);
00702     print_STR(AM.Path);
00703     print_STR(AM.SetupDir);
00704     print_STR(AM.SetupFile);
00705     MesPrint("--MARK 1");
00706     MesPrint("%l", (LONG)BASEPOSITION(AM.zeropos));
00707 #ifdef WITHPTHREADS
00708     MesPrint("%l", AM.ThreadScratSize);
00709     MesPrint("%l", AM.ThreadScratOutSize);
00710 #endif
00711     MesPrint("%l", AM.MaxTer);
00712     MesPrint("%l", AM.CompressSize);
00713     MesPrint("%l", AM.ScratSize);
00714     MesPrint("%l", AM.SizeStoreCache);
00715     MesPrint("%l", AM.MaxStreamSize);
00716     MesPrint("%l", AM.SIOsize);
00717     MesPrint("%l", AM.SLargeSize);
00718     MesPrint("%l", AM.SSmallEsize);
00719     MesPrint("%l", AM.SSmallSize);
00720     MesPrint("--MARK 2");
00721     MesPrint("%l", AM.STermsInSmall);
00722     MesPrint("%l", AM.MaxBracketBufferSize);
00723     MesPrint("%l", AM.hProcessBucketSize);
00724     MesPrint("%l", AM.gProcessBucketSize);
00725     MesPrint("%l", AM.shmWinSize);
00726     MesPrint("%l", AM.OldChildTime);
00727     MesPrint("%l", AM.OldSecTime);
00728     MesPrint("%l", AM.OldMilliTime);
00729     MesPrint("%l", AM.WorkSize);
00730     MesPrint("%l", AM.gThreadBucketSize);
00731     MesPrint("--MARK 3");
00732     MesPrint("%l", AM.ggThreadBucketSize);
00733     MesPrint("%d", AM.FileOnlyFlag);
00734     MesPrint("%d", AM.Interact);
00735     MesPrint("%d", AM.MaxParLevel);
00736     MesPrint("%d", AM.OutBufSize);
00737     MesPrint("%d", AM.SMaxFpatches);
00738     MesPrint("%d", AM.SMaxPatches);
00739     MesPrint("%d", AM.StdOut);
00740     MesPrint("%d", AM.ginsidefirst);
00741     MesPrint("%d", AM.gDefDim);
00742     MesPrint("%d", AM.gDefDim4);
00743     MesPrint("--MARK 4");
00744     MesPrint("%d", AM.NumFixedSets);
00745     MesPrint("%d", AM.NumFixedFunctions);
00746     MesPrint("%d", AM.rbufnum);
00747     MesPrint("%d", AM.dbufnum);
00748     MesPrint("%d", AM.SkipClears);
00749     MesPrint("%d", AM.gfunpowers);
00750     MesPrint("%d", AM.gStatsFlag);
00751     MesPrint("%d", AM.gNamesFlag);
00752     MesPrint("%d", AM.gCodesFlag);
00753     MesPrint("%d", AM.gTokensWriteFlag);
00754     MesPrint("%d", AM.gSortType);
00755     MesPrint("%d", AM.gproperorderflag);
00756     MesPrint("--MARK 5");
00757     MesPrint("%d", AM.hparallelflag);
00758     MesPrint("%d", AM.gparallelflag);
00759     MesPrint("%d", AM.totalnumberofthreads);
00760     MesPrint("%d", AM.gSizeCommuteInSet);
00761     MesPrint("%d", AM.gThreadStats);
00762     MesPrint("%d", AM.ggThreadStats);
00763     MesPrint("%d", AM.gFinalStats);
00764     MesPrint("%d", AM.ggFinalStats);
00765     MesPrint("%d", AM.gThreadsFlag);
00766     MesPrint("%d", AM.ggThreadsFlag);
00767     MesPrint("%d", AM.gThreadBalancing);

```

```

00768     MesPrint("%d", AM.ggThreadBalancing);
00769     MesPrint("%d", AM.gThreadSortFileSynch);
00770     MesPrint("%d", AM.ggThreadSortFileSynch);
00771     MesPrint("%d", AM.gProcessStats);
00772     MesPrint("%d", AM.ggProcessStats);
00773     MesPrint("%d", AM.gOldParallelStats);
00774     MesPrint("%d", AM.ggOldParallelStats);
00775     MesPrint("%d", AM.gWTimeStatsFlag);
00776     MesPrint("%d", AM.ggWTimeStatsFlag);
00777     MesPrint("%d", AM.maxFlevels);
00778     MesPrint("--MARK 6");
00779     MesPrint("%d", AM.resetTimeOnClear);
00780     MesPrint("%d", AM.gcNumDollars);
00781     MesPrint("%d", AM.MultiRun);
00782     MesPrint("%d", AM.gNoSpacesInNumbers);
00783     MesPrint("%d", AM.ggNoSpacesInNumbers);
00784     MesPrint("%d", AM.MaxTal);
00785     MesPrint("%d", AM.IndDum);
00786     MesPrint("%d", AM.DumInd);
00787     MesPrint("%d", AM.WilInd);
00788     MesPrint("%d", AM.gncmod);
00789     MesPrint("%d", AM.gnpowmod);
00790     MesPrint("%d", AM.gmodmode);
00791     MesPrint("--MARK 7");
00792     MesPrint("%d", AM.gUnitTrace);
00793     MesPrint("%d", AM.gOutputMode);
00794     MesPrint("%d", AM.gCnumpows);
00795     MesPrint("%d", AM.gOutputSpaces);
00796     MesPrint("%d", AM.gOutNumberType);
00797     MesPrint("%d %d %d %d", AM.gUniTrace[0], AM.gUniTrace[1], AM.gUniTrace[2], AM.gUniTrace[3]);
00798     MesPrint("%d", AM.MaxWildcards);
00799     MesPrint("%d", AM.mTraceDum);
00800     MesPrint("%d", AM.OffsetIndex);
00801     MesPrint("%d", AM.OffsetVector);
00802     MesPrint("%d", AM.RepMax);
00803     MesPrint("%d", AM.LogType);
00804     MesPrint("%d", AM.ggStatsFlag);
00805     MesPrint("%d", AM.gLineLength);
00806     MesPrint("%d", AM.qError);
00807     MesPrint("--MARK 8");
00808     MesPrint("%d", AM.FortranCont);
00809     MesPrint("%d", AM.HoldFlag);
00810     MesPrint("%d %d %d %d %d", AM.Ordering[0], AM.Ordering[1], AM.Ordering[2], AM.Ordering[3],
AM.Ordering[4]);
00811     MesPrint("%d %d %d %d %d", AM.Ordering[5], AM.Ordering[6], AM.Ordering[7], AM.Ordering[8],
AM.Ordering[9]);
00812     MesPrint("%d %d %d %d %d", AM.Ordering[10], AM.Ordering[11], AM.Ordering[12], AM.Ordering[13],
AM.Ordering[14]);
00813     MesPrint("%d", AM.silent);
00814     MesPrint("%d", AM.tracebackflag);
00815     MesPrint("%d", AM.expnum);
00816     MesPrint("%d", AM.denomnum);
00817     MesPrint("%d", AM.facnum);
00818     MesPrint("%d", AM.invfacnum);
00819     MesPrint("%d", AM.sumnum);
00820     MesPrint("%d", AM.sumpnum);
00821     MesPrint("--MARK 9");
00822     MesPrint("%d", AM.OldOrderFlag);
00823     MesPrint("%d", AM.termfunnum);
00824     MesPrint("%d", AM.matchfunnum);
00825     MesPrint("%d", AM.countfunnum);
00826     MesPrint("%d", AM.gPolyFun);
00827     MesPrint("%d", AM.gPolyFunInv);
00828     MesPrint("%d", AM.gPolyFunType);
00829     MesPrint("%d", AM.gPolyFunExp);
00830     MesPrint("%d", AM.gPolyFunVar);
00831     MesPrint("%d", AM.gPolyFunPow);
00832     MesPrint("--MARK 10");
00833     MesPrint("%d", AM.dollarzero);
00834     MesPrint("%d", AM.atstartup);
00835     MesPrint("%d", AM.exitflag);
00836     MesPrint("%d", AM.NumStoreCaches);
00837     MesPrint("%d", AM.gIndentSpace);
00838     MesPrint("%d", AM.ggIndentSpace);
00839     MesPrint("%d", AM.gShortStatsMax);
00840     MesPrint("%d", AM.ggShortStatsMax);
00841     MesPrint("--MARK 11");
00842     MesPrint("%d", AM.FromStdin);
00843     MesPrint("%d", AM.IgnoreDeprecation);
00844     MesPrint("%%% END M_const");
00845     /* fflush(0); */
00846 }
00847
00848 static void print_P()
00849 {
00850     int i;
00851     MesPrint("%%% P_const");

```

```

00852     print_LIST(&AP.DollarList);
00853     for ( i=0; i<AP.DollarList.num; ++i ) {
00854         print_DOLLARS(&(Dollars[i]));
00855     }
00856     MesPrint("--MARK 1");
00857     print_LIST(&AP.PreVarList);
00858     for ( i=0; i<AP.PreVarList.num; ++i ) {
00859         print_PREVAR(&(PreVar[i]));
00860     }
00861     MesPrint("--MARK 2");
00862     print_LIST(&AP.LoopList);
00863     for ( i=0; i<AP.LoopList.num; ++i ) {
00864         print_DOLOOP(&(DoLoops[i]));
00865     }
00866     MesPrint("--MARK 3");
00867     print_LIST(&AP.ProcList);
00868     for ( i=0; i<AP.ProcList.num; ++i ) {
00869         print_PROCEDURE(&(Procedures[i]));
00870     }
00871     MesPrint("--MARK 4");
00872     for ( i=0; i<=AP.PreSwitchLevel; ++i ) {
00873         print_STR(AP.PreSwitchStrings[i]);
00874     }
00875     MesPrint("%l", AP.preStop-AP.preStart);
00876     if ( AP.preFill ) MesPrint("%l", AP.preFill-AP.preStart);
00877     print_CHARS(AP.preStart, AP.pSize);
00878     MesPrint("%s", AP.procedureExtension);
00879     MesPrint("%s", AP.cprocedureExtension);
00880     print_INTV(AP.PreIfStack, AP.MaxPreIfLevel);
00881     print_INTV(AP.PreSwitchModes, AP.NumPreSwitchStrings+1);
00882     print_INTV(AP.PreTypes, AP.NumPreTypes+1);
00883     MesPrint("%d", AP.PreAssignFlag);
00884     MesPrint("--MARK 5");
00885     MesPrint("%d", AP.PreContinuation);
00886     MesPrint("%l", AP.InOutBuf);
00887     MesPrint("%l", AP.pSize);
00888     MesPrint("%d", AP.PreproFlag);
00889     MesPrint("%d", AP.iBufError);
00890     MesPrint("%d", AP.PreOut);
00891     MesPrint("%d", AP.PreSwitchLevel);
00892     MesPrint("%d", AP.NumPreSwitchStrings);
00893     MesPrint("%d", AP.MaxPreTypes);
00894     MesPrint("--MARK 6");
00895     MesPrint("%d", AP.NumPreTypes);
00896     MesPrint("%d", AP.DelayPrevar);
00897     MesPrint("%d", AP.AllowDelay);
00898     MesPrint("%d", AP.lhdollarerror);
00899     MesPrint("%d", AP.eat);
00900     MesPrint("%d", AP.gNumPre);
00901     MesPrint("%d", AP.PreDebug);
00902     MesPrint("--MARK 7");
00903     MesPrint("%d", AP.DebugFlag);
00904     MesPrint("%d", AP.preError);
00905     MesPrint("%C", 1, &(AP.ComChar));
00906     MesPrint("%C", 1, &(AP.cComChar));
00907     MesPrint("%%%" END P_const");
00908     /* fflush(0); */
00909 }
00910
00911 static void print_C()
00912 {
00913     int i;
00914     UBYTE buf[40], *t;
00915     MesPrint("%%%" C_const");
00916     for ( i=0; i<32; ++i ) {
00917         t = buf;
00918         t = NumCopy((WORD) (AC.separators[i].bit_7), t);
00919         t = NumCopy((WORD) (AC.separators[i].bit_6), t);
00920         t = NumCopy((WORD) (AC.separators[i].bit_5), t);
00921         t = NumCopy((WORD) (AC.separators[i].bit_4), t);
00922         t = NumCopy((WORD) (AC.separators[i].bit_3), t);
00923         t = NumCopy((WORD) (AC.separators[i].bit_2), t);
00924         t = NumCopy((WORD) (AC.separators[i].bit_1), t);
00925         t = NumCopy((WORD) (AC.separators[i].bit_0), t);
00926         MesPrint("%s ", buf);
00927     }
00928     print_NAMETREE(AC.dollarnames);
00929     print_NAMETREE(AC.exprnames);
00930     print_NAMETREE(AC.varnames);
00931     MesPrint("--MARK 1");
00932     print_LIST(&AC.ChannelList);
00933     for ( i=0; i<AC.ChannelList.num; ++i ) {
00934         MesPrint("%s %d", channels[i].name, channels[i].handle);
00935     }
00936     MesPrint("--MARK 2");
00937     print_LIST(&AC.DubiousList);
00938     MesPrint("--MARK 3");

```

```

00939     print_LIST(&AC.FunctionList);
00940     for ( i=0; i<AC.FunctionList.num; ++i ) {
00941         if ( functions[i].tabl ) {
00942             }
00943             MesPrint("%1", functions[i].symminfo);
00944             MesPrint("%1", functions[i].name);
00945             MesPrint("%d", functions[i].namesize);
00946         }
00947     MesPrint("--MARK 4");
00948     print_LIST(&AC.ExpressionList);
00949     print_LIST(&AC.IndexList);
00950     print_LIST(&AC.SetElementList);
00951     print_LIST(&AC.SetList);
00952     MesPrint("--MARK 5");
00953     print_LIST(&AC.SymbolList);
00954     print_LIST(&AC.VectorList);
00955     print_LIST(&AC.PotModDolList);
00956     print_LIST(&AC.ModOptDolList);
00957     print_LIST(&AC.TableBaseList);
00958
00959
00960 /*
00961     print_LIST(&AC.cbufList);
00962     for ( i=0; i<AC.cbufList.num; ++i ) {
00963         MesPrint("cbufList.num == %d", i);
00964         print_CBUF(cbuf+i);
00965     }
00966     MesPrint("%d", AC.cbufnum);
00967 */
00968     MesPrint("--MARK 6");
00969
00970     print_LIST(&AC.AutoSymbolList);
00971     print_LIST(&AC.AutoIndexList);
00972     print_LIST(&AC.AutoVectorList);
00973     print_LIST(&AC.AutoFunctionList);
00974
00975     print_NAMETREE(AC.autonames);
00976     MesPrint("--MARK 7");
00977
00978     print_LIST(AC.Symbols);
00979     print_LIST(AC.Indices);
00980     print_LIST(AC.Vectors);
00981     print_LIST(AC.Functions);
00982     MesPrint("--MARK 8");
00983
00984     print_NAMETREE(*AC.activenames);
00985
00986     MesPrint("--MARK 9");
00987
00988     MesPrint("%d", AC.AutoDeclareFlag);
00989
00990     for ( i=0; i<AC.NumStreams; ++i ) {
00991         MesPrint("Stream %d\n", i);
00992         print_STREAM(AC.Streams+i);
00993     }
00994     print_STREAM(AC.CurrentStream);
00995     MesPrint("--MARK 10");
00996
00997     print_LONGV(AC.termstack, AC.maxtermlevel);
00998     print_LONGV(AC.termstack, AC.maxtermlevel);
00999     print_VOIDP(AC.cmod, AM.MaxTal*4*sizeof(UWORD));
01000     print_WORDV((WORD *) (AC.cmod), 1);
01001     print_WORDV((WORD *) (AC.powmod), 1);
01002     print_WORDV((WORD *) AC.modpowers, 1);
01003     print_WORDV((WORD *) AC.halfmod, 1);
01004     MesPrint("--MARK 10-2");
01005     /*
01006     print_WORDV(AC.ProtoType, AC.ProtoType[1]);
01007     print_WORDV(AC.WildC, 1);
01008     */
01009
01010     MesPrint("--MARK 11");
01011     /* IfHeap ... Labels */
01012
01013     print_CHARS((UBYTE *) AC.tokens, AC.toptokens-AC.tokens);
01014     MesPrint("%1", AC.endoftokens-AC.tokens);
01015     print_WORDV(AC.tokenarglevel, AM.MaxParLevel);
01016     print_WORDV((WORD *) AC.modinverses, ABS(AC.ncmod));
01017 #ifdef WITHPTHREADS
01018     print_LONGV(AC.inputnumbers, AC.sizefirstnum+AC.sizefirstnum*sizeof(WORD)/sizeof(LONG));
01019     print_WORDV(AC.pfirstnum, 1);
01020 #endif
01021     MesPrint("--MARK 12");
01022     print_LONGV(AC.argstack, MAXNEST);
01023     print_LONGV(AC.insidestack, MAXNEST);
01024     print_LONGV(AC.inexprstack, MAXNEST);
01025     MesPrint("%1", AC.iBufferSize);

```

```
01026     MesPrint("%l", AC.TransEname);
01027     MesPrint("%l", AC.ProcessBucketSize);
01028     MesPrint("%l", AC.mProcessBucketSize);
01029     MesPrint("%l", AC.CModule);
01030     MesPrint("%l", AC.ThreadBucketSize);
01031     MesPrint("%d", AC.NoShowInput);
01032     MesPrint("%d", AC.ShortStats);
01033     MesPrint("%d", AC.compiletype);
01034     MesPrint("%d", AC.firstconstindex);
01035     MesPrint("%d", AC.insidefirst);
01036     MesPrint("%d", AC.minsidefirst);
01037     MesPrint("%d", AC.wildflag);
01038     MesPrint("%d", AC.NumLabels);
01039     MesPrint("%d", AC.MaxLabels);
01040     MesPrint("--MARK 13");
01041     MesPrint("%d", AC.lDefDim);
01042     MesPrint("%d", AC.lDefDim4);
01043     MesPrint("%d", AC.NumWildcardNames);
01044     MesPrint("%d", AC.WildcardBufferSize);
01045     MesPrint("%d", AC.MaxIf);
01046     MesPrint("%d", AC.NumStreams);
01047     MesPrint("%d", AC.MaxNumStreams);
01048     MesPrint("%d", AC.firstctypemessage);
01049     MesPrint("%d", AC.tablecheck);
01050     MesPrint("%d", AC.idoption);
01051     MesPrint("%d", AC.BottomLevel);
01052     MesPrint("%d", AC.CompileLevel);
01053     MesPrint("%d", AC.TokensWriteFlag);
01054     MesPrint("%d", AC.UnsureDollarMode);
01055     MesPrint("%d", AC.outsidefun);
01056     MesPrint("%d", AC.funpowers);
01057     MesPrint("--MARK 14");
01058     MesPrint("%d", AC.WarnFlag);
01059     MesPrint("%d", AC.StatsFlag);
01060     MesPrint("%d", AC.NamesFlag);
01061     MesPrint("%d", AC.CodesFlag);
01062     MesPrint("%d", AC.TokensWriteFlag);
01063     MesPrint("%d", AC.SetupFlag);
01064     MesPrint("%d", AC.SortType);
01065     MesPrint("%d", AC.lSortType);
01066     MesPrint("%d", AC.ThreadStats);
01067     MesPrint("%d", AC.FinalStats);
01068     MesPrint("%d", AC.ThreadsFlag);
01069     MesPrint("%d", AC.ThreadBalancing);
01070     MesPrint("%d", AC.ThreadSortFileSynch);
01071     MesPrint("%d", AC.ProcessStats);
01072     MesPrint("%d", AC.OldParallelStats);
01073     MesPrint("%d", AC.WTimeStatsFlag);
01074     MesPrint("%d", AC.BraceNormalizer);
01075     MesPrint("%d", AC.maxtermlevel);
01076     MesPrint("%d", AC.dumnumflag);
01077     MesPrint("--MARK 15");
01078     MesPrint("%d", AC.bracketindexflag);
01079     MesPrint("%d", AC.parallelflag);
01080     MesPrint("%d", AC.mparallelflag);
01081     MesPrint("%d", AC.properorderflag);
01082     MesPrint("%d", AC.vetofilling);
01083     MesPrint("%d", AC.tablefilling);
01084     MesPrint("%d", AC.vetotablebasefill);
01085     MesPrint("%d", AC.exprfillwarning);
01086     MesPrint("%d", AC.lhdollarflag);
01087     MesPrint("%d", AC.NoCompress);
01088 #ifdef WITHPTHREADS
01089     MesPrint("%d", AC.numpfirstnum);
01090     MesPrint("%d", AC.sizepfirstnum);
01091 #endif
01092     MesPrint("%d", AC.RepLevel);
01093     MesPrint("%d", AC.arglevel);
01094     MesPrint("%d", AC.insidelevel);
01095     MesPrint("%d", AC.inexprlevel);
01096     MesPrint("%d", AC.termlevel);
01097     MesPrint("--MARK 16");
01098     print_WORDV(AC.argsumcheck, MAXNEST);
01099     print_WORDV(AC.insidesumcheck, MAXNEST);
01100     print_WORDV(AC.inexprsumcheck, MAXNEST);
01101     MesPrint("%d", AC.MustTestTable);
01102     MesPrint("%d", AC.DumNum);
01103     MesPrint("%d", AC.ncmod);
01104     MesPrint("%d", AC.npowmod);
01105     MesPrint("%d", AC.modmode);
01106     MesPrint("%d", AC.nhalfmod);
01107     MesPrint("%d", AC.DirtPow);
01108     MesPrint("%d", AC.lUnitTrace);
01109     MesPrint("%d", AC.NwildC);
01110     MesPrint("%d", AC.ComDefer);
01111     MesPrint("%d", AC.CollectFun);
01112     MesPrint("%d", AC.AltCollectFun);
```

```

01113     MesPrint("--MARK 17");
01114     MesPrint("%d", AC.OutputMode);
01115     MesPrint("%d", AC.Cnumpows);
01116     MesPrint("%d", AC.OutputSpaces);
01117     MesPrint("%d", AC.OutNumberType);
01118     print_WORDV(AC.lUniTrace, 4);
01119     print_WORDV(AC.RepSumCheck, MAXREPEAT);
01120     MesPrint("%d", AC.DidClean);
01121     MesPrint("%d", AC.IfLevel);
01122     MesPrint("%d", AC.WhileLevel);
01123     print_WORDV(AC.IfSumCheck, (AC.MaxIf+1));
01124     MesPrint("%d", AC.LogHandle);
01125     MesPrint("%d", AC.LineLength);
01126     MesPrint("%d", AC.StoreHandle);
01127     MesPrint("%d", AC.HideLevel);
01128     MesPrint("%d", AC.lPolyFun);
01129     MesPrint("%d", AC.lPolyFunInv);
01130     MesPrint("%d", AC.lPolyFunType);
01131     MesPrint("%d", AC.lPolyFunExp);
01132     MesPrint("%d", AC.lPolyFunVar);
01133     MesPrint("%d", AC.lPolyFunPow);
01134     MesPrint("%d", AC.SymChangeFlag);
01135     MesPrint("%d", AC.CollectPercentage);
01136     MesPrint("%d", AC.ShortStatsMax);
01137     MesPrint("--MARK 18");
01138
01139     print_CHARS(AC.Commercial, COMMERCIALSIZE+2);
01140
01141     MesPrint("%", AC.CheckpointFlag);
01142     MesPrint("%l", AC.CheckpointStamp);
01143     print_STR((unsigned char*)AC.CheckpointRunAfter);
01144     print_STR((unsigned char*)AC.CheckpointRunBefore);
01145     MesPrint("%l", AC.CheckpointInterval);
01146
01147     MesPrint("%%% END C_const");
01148     /* fflush(0); */
01149 }
01150
01151 static void print_R()
01152 {
01153     GETIDENTITY
01154     int i;
01155     MesPrint("%%% R_const");
01156     MesPrint("%l", (LONG)(AR.infile-AR.Fscr));
01157     MesPrint("%s", AR.infile->name);
01158     MesPrint("%l", (LONG)(AR.outfile-AR.Fscr));
01159     MesPrint("%s", AR.outfile->name);
01160     MesPrint("%l", AR.hidefile-AR.Fscr);
01161     MesPrint("%s", AR.hidefile->name);
01162     for ( i=0; i<3; ++i ) {
01163         MesPrint("FSCR %d", i);
01164         print_WORDB(AR.Fscr[i].PObuffer, AR.Fscr[i].POfull);
01165     }
01166     /* ... */
01167     MesPrint("%l", AR.OldTime);
01168     MesPrint("%l", AR.InInBuf);
01169     MesPrint("%l", AR.InHiBuf);
01170     MesPrint("%l", AR.pWorkSize);
01171     MesPrint("%l", AR.lWorkSize);
01172     MesPrint("%l", AR.posWorkSize);
01173     MesPrint("%d", AR.NoCompress);
01174     MesPrint("%d", AR.gzipCompress);
01175     MesPrint("%d", AR.Cnumlhs);
01176 #ifdef WITHPTHREADS
01177     MesPrint("%d", AR.exprtodo);
01178 #endif
01179     MesPrint("%d", AR.GetFile);
01180     MesPrint("%d", AR.KeptInHold);
01181     MesPrint("%d", AR.BraceOn);
01182     MesPrint("%d", AR.MaxBracket);
01183     MesPrint("%d", AR.CurDum);
01184     MesPrint("%d", AR.DeferFlag);
01185     MesPrint("%d", AR.TePos);
01186     MesPrint("%d", AR.sLevel);
01187     MesPrint("%d", AR.Stage4Name);
01188     MesPrint("%d", AR.GetOneFile);
01189     MesPrint("%d", AR.PolyFun);
01190     MesPrint("%d", AR.PolyFunInv);
01191     MesPrint("%d", AR.PolyFunType);
01192     MesPrint("%d", AR.PolyFunExp);
01193     MesPrint("%d", AR.PolyFunVar);
01194     MesPrint("%d", AR.PolyFunPow);
01195     MesPrint("%d", AR.Eside);
01196     MesPrint("%d", AR.MaxDum);
01197     MesPrint("%d", AR.level);
01198     MesPrint("%d", AR.expchanged);
01199     MesPrint("%d", AR.expflags);

```

```

01200     MesPrint("%d", AR.CurExpr);
01201     MesPrint("%d", AR.SortType);
01202     MesPrint("%d", AR.ShortSortCount);
01203     MesPrint("%%% END R_const");
01204 /*  fflush(0); */
01205 }
01206
01207 #endif /* ifdef PRINTDEBUG */
01208
01209 /*
01210     #] Debugging :
01211     #[ Cached file operation functions :
01212 */
01213
01214 #define CACHED_SNAPSHOT
01215
01216 #define CACHE_SIZE 4096
01217
01218 #ifndef CACHED_SNAPSHOT
01219 unsigned char cache_buffer[CACHE_SIZE];
01220 size_t cache_fill = 0;
01221
01222 size_t fwrite_cached(const void *ptr, size_t size, size_t nmemb, FILE *fd)
01223 {
01224     size_t fullsize = size*nmemb;
01225     if ( fullsize+cache_fill >= CACHE_SIZE ) {
01226         size_t overlap = CACHE_SIZE-cache_fill;
01227         memcpy(cache_buffer+cache_fill, (unsigned char*)ptr, overlap);
01228         if ( fwrite(cache_buffer, 1, CACHE_SIZE, fd) != CACHE_SIZE ) return 0;
01229         fullsize -= overlap;
01230         if ( fullsize >= CACHE_SIZE ) {
01231             cache_fill = fullsize % CACHE_SIZE;
01232             if ( cache_fill ) memcpy(cache_buffer, (unsigned char*)ptr+overlap+fullsize-cache_fill,
cache_fill);
01233             if ( fwrite((unsigned char*)ptr+overlap, 1, fullsize-cache_fill, fd) !=
fullsize-cache_fill ) return 0;
01234         }
01235         else {
01236             memcpy(cache_buffer, (unsigned char*)ptr+overlap, fullsize);
01237             cache_fill = fullsize;
01238         }
01239     }
01240     else {
01241         memcpy(cache_buffer+cache_fill, (unsigned char*)ptr, fullsize);
01242         cache_fill += fullsize;
01243     }
01244     return nmemb;
01245 }
01246
01247 size_t flush_cache(FILE *fd)
01248 {
01249     if ( cache_fill ) {
01250         size_t retval = fwrite(cache_buffer, 1, cache_fill, fd);
01251         if ( retval != cache_fill ) {
01252             cache_fill = 0;
01253             return 0;
01254         }
01255         cache_fill = 0;
01256     }
01257     return 1;
01258 }
01259 #else
01260 size_t fwrite_cached(const void *ptr, size_t size, size_t nmemb, FILE *fd)
01261 {
01262     return fwrite(ptr, size, nmemb, fd);
01263 }
01264
01265 size_t flush_cache(FILE *fd)
01266 {
01267     DUMMYUSE(fd)
01268     return 1;
01269 }
01270 #endif
01271
01272 /*
01273     #] Cached file operation functions :
01274     #[ Helper Macros :
01275 */
01276
01277 /* some helper macros to streamline the code in DoSnapshot() and DoRecovery() */
01278
01279 /* freeing memory */
01280
01281 #define R_FREE(ARG) \
01282     if ( ARG ) M_free(ARG, #ARG);
01283
01284 #define R_FREE_NAMETREE(ARG) \

```



```

01285     R_FREE(ARG->namenode); \
01286     R_FREE(ARG->namebuffer); \
01287     R_FREE(ARG);
01288
01289 #define R_FREE_STREAM(ARG) \
01290     R_FREE(ARG.buffer); \
01291     R_FREE(ARG.FoldName); \
01292     R_FREE(ARG.name);
01293
01294 /* reading a single variable */
01295
01296 #define R_SET(VAR,TYPE) \
01297     VAR = *((TYPE*)p); p = (unsigned char*)p + sizeof(TYPE);
01298
01299 /* general buffer */
01300
01301 #define R_COPY_B(VAR,SIZE,CAST) \
01302     VAR = (CAST)Malloc1(SIZE,#VAR); \
01303     memcpy(VAR, p, SIZE); p = (unsigned char*)p + SIZE;
01304
01305 #define S_WRITE_B(BUF,LEN) \
01306     if ( fwrite_cached(BUF, 1, LEN, fd) != (size_t)(LEN) ) return(__LINE__);
01307
01308 #define S_FLUSH_B \
01309     if ( flush_cache(fd) != 1 ) return(__LINE__);
01310
01311 /* character strings */
01312
01313 #define R_COPY_S(VAR,CAST) \
01314     if ( VAR ) { \
01315         VAR = (CAST)Malloc1(strlen(p)+1,"R_COPY_S"); \
01316         strcpy((char*)VAR, p); p = (unsigned char*)p + strlen(p) + 1; \
01317     }
01318
01319 #define S_WRITE_S(STR) \
01320     if ( STR ) { \
01321         l = strlen((char*)STR) + 1; \
01322         if ( fwrite_cached(STR, 1, l, fd) != (size_t)l ) return(__LINE__); \
01323     }
01324
01325 /* LIST */
01326
01327 #define R_COPY_LIST(ARG) \
01328     if ( ARG.maxnum ) { \
01329         R_COPY_B(ARG.lijst, ARG.size*ARG.maxnum, void*) \
01330     }
01331
01332 #define S_WRITE_LIST(LST) \
01333     if ( LST.maxnum ) { \
01334         S_WRITE_B((char*)LST.lijst, LST.maxnum*LST.size) \
01335     }
01336
01337 /* NAMETREE */
01338
01339 #define R_COPY_NAMETREE(ARG) \
01340     R_COPY_B(ARG, sizeof(NAMETREE), NAMETREE*); \
01341     if ( ARG->namenode ) { \
01342         R_COPY_B(ARG->namenode, ARG->nodesize*sizeof(NAMENODE), NAMENODE*); \
01343     } \
01344     if ( ARG->namebuffer ) { \
01345         R_COPY_B(ARG->namebuffer, ARG->namesize, UBYTE*); \
01346     }
01347
01348 #define S_WRITE_NAMETREE(ARG) \
01349     S_WRITE_B(ARG, sizeof(NAMETREE)); \
01350     if ( ARG->namenode ) { \
01351         S_WRITE_B(ARG->namenode, ARG->nodesize*sizeof(struct NaMeNode)); \
01352     } \
01353     if ( ARG->namebuffer ) { \
01354         S_WRITE_B(ARG->namebuffer, ARG->namesize); \
01355     }
01356
01357 /* DOLLAR */
01358
01359 #define S_WRITE_DOLLAR(ARG) \
01360     if ( ARG.size && ARG.where && ARG.where != &(AM.dollarzero) ) { \
01361         S_WRITE_B(ARG.where, ARG.size*sizeof(WORD)) \
01362     }
01363
01364 /* Printing time marks with ANNOUNCE macro */
01365
01366 #ifdef PRINTTIMEMARKS
01367     time_t announce_time;
01368     #define ANNOUNCE(str) time(&announce_time); MesPrint("TIMEMARK %s %s", ctime(&announce_time), #str);
01369 #else
01370     #define ANNOUNCE(str)
01371 #endif

```

```

01372
01373 /*
01374 #] Helper Macros :
01375 #[ DoRecovery :
01376 */
01377
01402 int DoRecovery(int *moduletype)
01403 {
01404     GETIDENTITY
01405     FILE *fd;
01406     POSITION pos;
01407     void *buf, *p;
01408     LONG size, l;
01409     int i, j;
01410     UBYTE *org;
01411     char *namebufout, *namebufhide;
01412     LONG ofs;
01413     void *oldAMdollarzero;
01414     LIST PotModDolListBackup;
01415     LIST ModOptDolListBackup;
01416     WORD oldLogHandle;
01417
01418     MesPrint("Recovering ... %"); fflush(0);
01419
01420     if ( !(fd = fopen(recoveryfile, "r")) ) return(__LINE__);
01421
01422     /* load the complete recovery file into a buffer */
01423     if ( fread(&pos, sizeof(POSITION), 1, fd) != 1 ) return(__LINE__);
01424     size = BASEPOSITION(pos) - sizeof(POSITION);
01425     buf = Malloc1(size, "recovery buffer");
01426     if ( fread(buf, size, 1, fd) != 1 ) return(__LINE__);
01427
01428     /* pointer p will go through the buffer in the following */
01429     p = buf;
01430
01431     /* read moduletype */
01432     R_SET(*moduletype, int);
01433
01434     /*#[ AM : */
01435
01436     /* only certain elements will be restored. the rest of AM should have gotten
01437      * the correct values at startup. */
01438
01439     R_SET(AM.hparallelflag, int);
01440     R_SET(AM.gparallelflag, int);
01441     R_SET(AM.gCodesFlag, int);
01442     R_SET(AM.gNamesFlag, int);
01443     R_SET(AM.gStatsFlag, int);
01444     R_SET(AM.gTokensWriteFlag, int);
01445     R_SET(AM.gNoSpacesInNumbers, int);
01446     R_SET(AM.gIndentSpace, WORD);
01447     R_SET(AM.gUnitTrace, WORD);
01448     R_SET(AM.gDefDim, int);
01449     R_SET(AM.gDefDim4, int);
01450     R_SET(AM.gncmod, WORD);
01451     R_SET(AM.gnpowmod, WORD);
01452     R_SET(AM.gmodmode, WORD);
01453     R_SET(AM.gOutputMode, WORD);
01454     R_SET(AM.gCnumpows, WORD);
01455     R_SET(AM.gOutputSpaces, WORD);
01456     R_SET(AM.gOutNumberType, WORD);
01457     R_SET(AM.gfunpowers, int);
01458     R_SET(AM.gPolyFun, WORD);
01459     R_SET(AM.gPolyFunInv, WORD);
01460     R_SET(AM.gPolyFunType, WORD);
01461     R_SET(AM.gPolyFunExp, WORD);
01462     R_SET(AM.gPolyFunVar, WORD);
01463     R_SET(AM.gPolyFunPow, WORD);
01464     R_SET(AM.gProcessBucketSize, LONG);
01465     R_SET(AM.OldChildTime, LONG);
01466     R_SET(AM.OldSecTime, LONG);
01467     R_SET(AM.OldMilliTime, LONG);
01468     R_SET(AM.gproperorderflag, int);
01469     R_SET(AM.gThreadBucketSize, LONG);
01470     R_SET(AM.gSizeCommuteInSet, int);
01471     R_SET(AM.gThreadStats, int);
01472     R_SET(AM.gFinalStats, int);
01473     R_SET(AM.gThreadsFlag, int);
01474     R_SET(AM.gThreadBalancing, int);
01475     R_SET(AM.gThreadSortFileSynch, int);
01476     R_SET(AM.gProcessStats, int);
01477     R_SET(AM.gOldParallelStats, int);
01478     R_SET(AM.gSortType, int);
01479     R_SET(AM.gShortStatsMax, WORD);
01480     R_SET(AM.gIsFortran90, int);
01481     R_SET(oldAMdollarzero, void*);
01482     R_FREE(AM.gFortran90Kind);

```

```

01483     R_SET(AM.gFortran90Kind, UBYTE *);
01484     R_COPY_S(AM.gFortran90Kind, UBYTE *);
01485
01486     R_COPY_S(AM.gextrasymp, UBYTE *);
01487     R_COPY_S(AM.ggextrasymp, UBYTE *);
01488
01489     R_SET(AM.PrintTotalSize, int);
01490     R_SET(AM.fbuffersize, int);
01491     R_SET(AM.gOldFactArgFlag, int);
01492     R_SET(AM.ggOldFactArgFlag, int);
01493
01494     R_SET(AM.gnumextrasymp, int);
01495     R_SET(AM.ggnumextrasymp, int);
01496     R_SET(AM.NumSpectatorFiles, int);
01497     R_SET(AM.SizeForSpectatorFiles, int);
01498     R_SET(AM.gOldGCDflag, int);
01499     R_SET(AM.ggOldGCDflag, int);
01500     R_SET(AM.gWTimeStatsFlag, int);
01501
01502     R_FREE(AM.Path);
01503     R_SET(AM.Path, UBYTE *);
01504     R_COPY_S(AM.Path, UBYTE *);
01505
01506     R_SET(AM.FromStdin, BOOL);
01507     R_SET(AM.IgnoreDeprecation, BOOL);
01508
01509 #ifdef PRINTDEBUG
01510     print_M();
01511 #endif
01512
01513     /*#] AM : */
01514     /*#[ AC : */
01515
01516     /* #[ AC free pointers */
01517
01518     /* AC will be overwritten by data from the recovery file, therefore
01519      * dynamically allocated memory must be freed first. */
01520
01521     R_FREE_NAMETREE(AC.dollarnames);
01522     R_FREE_NAMETREE(AC.exprnames);
01523     R_FREE_NAMETREE(AC.varnames);
01524     for ( i=0; i<AC.ChannellList.num; ++i ) {
01525         R_FREE(channels[i].name);
01526     }
01527     R_FREE(AC.ChannellList.lijst);
01528     R_FREE(AC.DubiousList.lijst);
01529     for ( i=0; i<AC.FunctionList.num; ++i ) {
01530         TABLES T = functions[i].tabl;
01531         if ( T ) {
01532             R_FREE(T->buffers);
01533             R_FREE(T->mm);
01534             R_FREE(T->flags);
01535             R_FREE(T->prototype);
01536             R_FREE(T->tablepointers);
01537             if ( T->sparse ) {
01538                 R_FREE(T->boomlijst);
01539                 R_FREE(T->argtail);
01540             }
01541             if ( T->spare ) {
01542                 R_FREE(T->spare->buffers);
01543                 R_FREE(T->spare->mm);
01544                 R_FREE(T->spare->flags);
01545                 R_FREE(T->spare->tablepointers);
01546                 if ( T->spare->sparse ) {
01547                     R_FREE(T->spare->boomlijst);
01548                 }
01549                 R_FREE(T->spare);
01550             }
01551             R_FREE(T);
01552         }
01553     }
01554     R_FREE(AC.FunctionList.lijst);
01555     for ( i=0; i<AC.ExpressionList.num; ++i ) {
01556         if ( Expressions[i].renum ) {
01557             R_FREE(Expressions[i].renum->symb.lo);
01558             R_FREE(Expressions[i].renum);
01559         }
01560         if ( Expressions[i].bracketinfo ) {
01561             R_FREE(Expressions[i].bracketinfo->indexbuffer);
01562             R_FREE(Expressions[i].bracketinfo->bracketbuffer);
01563             R_FREE(Expressions[i].bracketinfo);
01564         }
01565         if ( Expressions[i].newbracketinfo ) {
01566             R_FREE(Expressions[i].newbracketinfo->indexbuffer);
01567             R_FREE(Expressions[i].newbracketinfo->bracketbuffer);
01568             R_FREE(Expressions[i].newbracketinfo);
01569         }

```

```

01570         if ( Expressions[i].renumlists != AN.dummyrenumlist ) {
01571             R_FREE(Expressions[i].renumlists);
01572         }
01573         R_FREE(Expressions[i].inmem);
01574     }
01575     R_FREE(AC.ExpressionList.lijst);
01576     R_FREE(AC.IndexList.lijst);
01577     R_FREE(AC.SetElementList.lijst);
01578     R_FREE(AC.SetList.lijst);
01579     R_FREE(AC.SymbolList.lijst);
01580     R_FREE(AC.VectorList.lijst);
01581     for ( i=0; i<AC.TableBaseList.num; ++i ) {
01582         R_FREE(tablebases[i].iblocks);
01583         R_FREE(tablebases[i].nblocks);
01584         R_FREE(tablebases[i].name);
01585         R_FREE(tablebases[i].fullname);
01586         R_FREE(tablebases[i].tablenames);
01587     }
01588     R_FREE(AC.TableBaseList.lijst);
01589     for ( i=0; i<AC.cbufList.num; ++i ) {
01590         R_FREE(cbuf[i].Buffer);
01591         R_FREE(cbuf[i].lhs);
01592         R_FREE(cbuf[i].rhs);
01593         R_FREE(cbuf[i].boomlijst);
01594     }
01595     R_FREE(AC.cbufList.lijst);
01596     R_FREE(AC.AutoSymbolList.lijst);
01597     R_FREE(AC.AutoIndexList.lijst);
01598     R_FREE(AC.AutoVectorList.lijst);
01599     /* Tables cannot be auto-declared, therefore no extra code here */
01600     R_FREE(AC.AutoFunctionList.lijst);
01601     R_FREE_NAMETREE(AC.autonames);
01602     for ( i=0; i<AC.NumStreams; ++i ) {
01603         R_FREE_STREAM(AC.Streams[i]);
01604     }
01605     R_FREE(AC.Streams);
01606     R_FREE(AC.termstack);
01607     R_FREE(AC.termstortstack);
01608     R_FREE(AC.cmod);
01609     R_FREE(AC.modpowers);
01610     R_FREE(AC.halfmod);
01611     R_FREE(AC.IfHeap);
01612     R_FREE(AC.IfCount);
01613     R_FREE(AC.iBuffer);
01614     for ( i=0; i<AC.NumLabels; ++i ) {
01615         R_FREE(AC.LabelNames[i]);
01616     }
01617     R_FREE(AC.LabelNames);
01618     R_FREE(AC.FixIndices);
01619     R_FREE(AC.termsumcheck);
01620     R_FREE(AC.WildcardNames);
01621     R_FREE(AC.tokens);
01622     R_FREE(AC.tokenarglevel);
01623     R_FREE(AC.modinverses);
01624     R_FREE(AC.Fortran90Kind);
01625 #ifdef WITHPTHREADS
01626     R_FREE(AC.inputnumbers);
01627 #endif
01628     R_FREE(AC.IfSumCheck);
01629     R_FREE(AC.CommuteInSet);
01630     R_FREE(AC.CheckpointRunAfter);
01631     R_FREE(AC.CheckpointRunBefore);
01632
01633     /* #] AC free pointers */
01634
01635     /* backup some lists in order to restore it to the initial setup */
01636     PotModDolListBackup = AC.PotModDolList;
01637     ModOptDolListBackup = AC.ModOptDolList;
01638     oldLogHandle = AC.LogHandle;
01639
01640     /* first we copy AC as a whole and then restore the pointer structures step
01641        by step. */
01642
01643     AC = *((struct C_const*)p); p = (unsigned char*)p + sizeof(struct C_const);
01644
01645     R_COPY_NAMETREE(AC.dollarnames);
01646     R_COPY_NAMETREE(AC.exprnames);
01647     R_COPY_NAMETREE(AC.varnames);
01648
01649     R_COPY_LIST(AC.ChannelList);
01650     for ( i=0; i<AC.ChannelList.num; ++i ) {
01651         R_COPY_S(channels[i].name, char*);
01652         channels[i].handle = ReOpenFile(channels[i].name);
01653     }
01654     AC.ChannelList.message = "channel buffer";
01655
01656     AC.DubiousList.lijst = 0;

```

```

01657     AC.DubiousList.message = "ambiguous variable";
01658     AC.DubiousList.num =
01659     AC.DubiousList.maxnum =
01660     AC.DubiousList.numglobal =
01661     AC.DubiousList.numtemp =
01662     AC.DubiousList.numclear = 0;
01663
01664     R_COPY_LIST(AC.FunctionList);
01665     for ( i=0; i<AC.FunctionList.num; ++i ) {
01666         if ( functions[i].tabl ) {
01667             TABLES tabl;
01668             R_COPY_B(tabl, sizeof(struct TableEs), TABLES);
01669             functions[i].tabl = tabl;
01670             if ( tabl->tablepointers ) {
01671                 if ( tabl->sparse ) {
01672                     R_COPY_B(tabl->tablepointers,
01673                             tabl->reserved*sizeof(WORD)*(tabl->numind+TABLEEXTENSION),
01674                             WORD*);
01675                 }
01676                 else {
01677                     R_COPY_B(tabl->tablepointers,
01678                             TABLEEXTENSION*sizeof(WORD)*(tabl->totind), WORD*);
01679                 }
01680             }
01681             org = (UBYTE*)tabl->prototype;
01682             #ifdef WITHPTHREADS
01683             R_COPY_B(tabl->prototype, tabl->prototypeSize, WORD*);
01684             ofs = (UBYTE*)tabl->prototype - org;
01685             for ( j=0; j<AM.totalnumberofthreads; ++j ) {
01686                 if ( tabl->prototype[j] ) {
01687                     tabl->prototype[j] = (WORD*)((UBYTE*)tabl->prototype[j] + ofs);
01688                 }
01689             }
01690             if ( tabl->pattern ) {
01691                 tabl->pattern = (WORD*)((UBYTE*)tabl->pattern + ofs);
01692                 for ( j=0; j<AM.totalnumberofthreads; ++j ) {
01693                     if ( tabl->pattern[j] ) {
01694                         tabl->pattern[j] = (WORD*)((UBYTE*)tabl->pattern[j] + ofs);
01695                     }
01696                 }
01697             }
01698             #else
01699             ofs = tabl->pattern - tabl->prototype;
01700             R_COPY_B(tabl->prototype, tabl->prototypeSize, WORD*);
01701             if ( tabl->pattern ) {
01702                 tabl->pattern = tabl->prototype + ofs;
01703             }
01704             #endif
01705             R_COPY_B(tabl->mm, tabl->numind*(LONG)sizeof(MINMAX), MINMAX*);
01706             R_COPY_B(tabl->flags, tabl->numind*(LONG)sizeof(WORD), WORD*);
01707             if ( tabl->sparse ) {
01708                 R_COPY_B(tabl->boomlijst, tabl->MaxTreeSize*(LONG)sizeof(COMPTREE), COMPTREE*);
01709                 R_COPY_S(tabl->argtail, UBYTE*);
01710             }
01711             R_COPY_B(tabl->buffers, tabl->bufferssize*(LONG)sizeof(WORD), WORD*);
01712             if ( tabl->spare ) {
01713                 TABLES spare;
01714                 R_COPY_B(spare, sizeof(struct TableEs), TABLES);
01715                 tabl->spare = spare;
01716                 if ( spare->tablepointers ) {
01717                     if ( spare->sparse ) {
01718                         R_COPY_B(spare->tablepointers,
01719                                 spare->reserved*sizeof(WORD)*(spare->numind+TABLEEXTENSION),
01720                                 WORD*);
01721                     }
01722                     else {
01723                         R_COPY_B(spare->tablepointers,
01724                                 TABLEEXTENSION*sizeof(WORD)*(spare->totind), WORD*);
01725                     }
01726                 }
01727                 spare->prototype = tabl->prototype;
01728                 spare->pattern = tabl->pattern;
01729                 R_COPY_B(spare->mm, spare->numind*(LONG)sizeof(MINMAX), MINMAX*);
01730                 R_COPY_B(spare->flags, spare->numind*(LONG)sizeof(WORD), WORD*);
01731                 if ( tabl->sparse ) {
01732                     R_COPY_B(spare->boomlijst, spare->MaxTreeSize*(LONG)sizeof(COMPTREE), COMPTREE*);
01733                     spare->argtail = tabl->argtail;
01734                 }
01735                 spare->spare = tabl;
01736                 R_COPY_B(spare->buffers, spare->bufferssize*(LONG)sizeof(WORD), WORD*);
01737             }
01738         }
01739     }
01740     AC.FunctionList.message = "function";
01741
01742     R_COPY_LIST(AC.ExpressionList);
01743     for ( i=0; i<AC.ExpressionList.num; ++i ) {

```

```

01744     EXPRESSIONS ex = Expressions + i;
01745     if ( ex->renum ) {
01746         R_COPY_B(ex->renum, sizeof(struct ReNuMbEr), RENUMBER);
01747         org = (UBYTE*)ex->renum->symb.lo;
01748         R_SET(size, size_t);
01749         R_COPY_B(ex->renum->symb.lo, size, WORD*);
01750         ofs = (UBYTE*)ex->renum->symb.lo - org;
01751         ex->renum->symb.start = (WORD*) ((UBYTE*)ex->renum->symb.start + ofs);
01752         ex->renum->symb.hi = (WORD*) ((UBYTE*)ex->renum->symb.hi + ofs);
01753         ex->renum->indi.lo = (WORD*) ((UBYTE*)ex->renum->indi.lo + ofs);
01754         ex->renum->indi.start = (WORD*) ((UBYTE*)ex->renum->indi.start + ofs);
01755         ex->renum->indi.hi = (WORD*) ((UBYTE*)ex->renum->indi.hi + ofs);
01756         ex->renum->vect.lo = (WORD*) ((UBYTE*)ex->renum->vect.lo + ofs);
01757         ex->renum->vect.start = (WORD*) ((UBYTE*)ex->renum->vect.start + ofs);
01758         ex->renum->vect.hi = (WORD*) ((UBYTE*)ex->renum->vect.hi + ofs);
01759         ex->renum->func.lo = (WORD*) ((UBYTE*)ex->renum->func.lo + ofs);
01760         ex->renum->func.start = (WORD*) ((UBYTE*)ex->renum->func.start + ofs);
01761         ex->renum->func.hi = (WORD*) ((UBYTE*)ex->renum->func.hi + ofs);
01762         ex->renum->symnum = (WORD*) ((UBYTE*)ex->renum->symnum + ofs);
01763         ex->renum->indnum = (WORD*) ((UBYTE*)ex->renum->indnum + ofs);
01764         ex->renum->vecnum = (WORD*) ((UBYTE*)ex->renum->vecnum + ofs);
01765         ex->renum->funnum = (WORD*) ((UBYTE*)ex->renum->funnum + ofs);
01766     }
01767     if ( ex->bracketinfo ) {
01768         R_COPY_B(ex->bracketinfo, sizeof(BRACKETINFO), BRACKETINFO*);
01769         R_COPY_B(ex->bracketinfo->indexbuffer,
01770 ex->bracketinfo->indexbuffersize*sizeof(BRACKETINDEX), BRACKETINDEX*);
01771         R_COPY_B(ex->bracketinfo->bracketbuffer, ex->bracketinfo->bracketbuffersize*sizeof(WORD),
01772 WORD*);
01773     }
01774     if ( ex->newbracketinfo ) {
01775         R_COPY_B(ex->newbracketinfo, sizeof(BRACKETINFO), BRACKETINFO*);
01776         R_COPY_B(ex->newbracketinfo->indexbuffer,
01777 ex->newbracketinfo->indexbuffersize*sizeof(BRACKETINDEX), BRACKETINDEX*);
01778         R_COPY_B(ex->newbracketinfo->bracketbuffer,
01779 ex->newbracketinfo->bracketbuffersize*sizeof(WORD), WORD*);
01780     }
01781 #ifdef WITHPTHREADS
01782     ex->renumlists = 0;
01783 #else
01784     ex->renumlists = AN.dummyrenumlist;
01785 #endif
01786     if ( ex->inmem ) {
01787         R_SET(size, size_t);
01788         R_COPY_B(ex->inmem, size, WORD*);
01789     }
01790     AC.ExpressionList.message = "expression";
01791     R_COPY_LIST(AC.IndexList);
01792     AC.IndexList.message = "index";
01793     R_COPY_LIST(AC.SetElementList);
01794     AC.SetElementList.message = "set element";
01795     R_COPY_LIST(AC.SetList);
01796     AC.SetList.message = "set";
01797     R_COPY_LIST(AC.SymbolList);
01798     AC.SymbolList.message = "symbol";
01799     R_COPY_LIST(AC.VectorList);
01800     AC.VectorList.message = "vector";
01801     AC.PotModDolList = PotModDolListBackup;
01802     AC.ModOptDolList = ModOptDolListBackup;
01803     R_COPY_LIST(AC.TableBaseList);
01804     for ( i=0; i<AC.TableBaseList.num; ++i ) {
01805         if ( tablebases[i].iblocks ) {
01806             R_COPY_B(tablebases[i].iblocks,
01807 tablebases[i].info.numberofindexblocks*sizeof(INDEXBLOCK*), INDEXBLOCK**);
01808             for ( j=0; j<tablebases[i].info.numberofindexblocks; ++j ) {
01809                 if ( tablebases[i].iblocks[j] ) {
01810                     R_COPY_B(tablebases[i].iblocks[j], sizeof(INDEXBLOCK), INDEXBLOCK*);
01811                 }
01812             }
01813             if ( tablebases[i].nblocks ) {
01814                 R_COPY_B(tablebases[i].nblocks,
01815 tablebases[i].info.numberofnamesblocks*sizeof(NAMESBLOCK*), NAMESBLOCK**);
01816                 for ( j=0; j<tablebases[i].info.numberofindexblocks; ++j ) {
01817                     if ( tablebases[i].nblocks[j] ) {
01818                         R_COPY_B(tablebases[i].nblocks[j], sizeof(NAMESBLOCK), NAMESBLOCK*);
01819                     }
01820                 }
01821             }
01822             /* reopen file */
01823             if ( ( tablebases[i].handle = fopen(tablebases[i].fullname, "r+b") ) == NULL ) {
01824                 MesPrint("ERROR: Could not reopen tablebase %s!", tablebases[i].name);
01825                 Terminate(-1);
01826             }
01827         }
01828     }

```

```

01825     }
01826     R_COPY_S(tablebases[i].name,char*);
01827     R_COPY_S(tablebases[i].fullname,char*);
01828     R_COPY_S(tablebases[i].tablenames,char*);
01829 }
01830 AC.TableBaseList.message = "list of tablebases";
01831
01832 R_COPY_LIST(AC.cbufList);
01833 for ( i=0; i<AC.cbufList.num; ++i ) {
01834     org = (UBYTE*)cbuf[i].Buffer;
01835     R_COPY_B(cbuf[i].Buffer, cbuf[i].BufferSize*sizeof(WORD), WORD*);
01836     ofs = (UBYTE*)cbuf[i].Buffer - org;
01837     cbuf[i].Top = (WORD*)((UBYTE*)cbuf[i].Top + ofs);
01838     cbuf[i].Pointer = (WORD*)((UBYTE*)cbuf[i].Pointer + ofs);
01839     R_COPY_B(cbuf[i].lhs, cbuf[i].maxlhs*(LONG)sizeof(WORD*), WORD**);
01840     for ( j=1; j<=cbuf[i].numlhs; ++j ) {
01841         if ( cbuf[i].lhs[j] ) cbuf[i].lhs[j] = (WORD*)((UBYTE*)cbuf[i].lhs[j] + ofs);
01842     }
01843     org = (UBYTE*)cbuf[i].rhs;
01844     R_COPY_B(cbuf[i].rhs, cbuf[i].maxrhs*(LONG)(sizeof(WORD*)+2*sizeof(LONG)+2*sizeof(WORD)),
WORD**);
01845     for ( j=1; j<=cbuf[i].numrhs; ++j ) {
01846         if ( cbuf[i].rhs[j] ) cbuf[i].rhs[j] = (WORD*)((UBYTE*)cbuf[i].rhs[j] + ofs);
01847     }
01848     ofs = (UBYTE*)cbuf[i].rhs - org;
01849     cbuf[i].CanCommu = (LONG*)((UBYTE*)cbuf[i].CanCommu + ofs);
01850     cbuf[i].NumTerms = (LONG*)((UBYTE*)cbuf[i].NumTerms + ofs);
01851     cbuf[i].numdum = (WORD*)((UBYTE*)cbuf[i].numdum + ofs);
01852     cbuf[i].dimension = (WORD*)((UBYTE*)cbuf[i].dimension + ofs);
01853     if ( cbuf[i].boomlijst ) {
01854         R_COPY_B(cbuf[i].boomlijst, cbuf[i].MaxTreeSize*sizeof(COMPTREE), COMPTREE*);
01855     }
01856 }
01857 AC.cbufList.message = "compiler buffer";
01858
01859 R_COPY_LIST(AC.AutoSymbolList);
01860 AC.AutoSymbolList.message = "autosymbol";
01861 R_COPY_LIST(AC.AutoIndexList);
01862 AC.AutoIndexList.message = "autoindex";
01863 R_COPY_LIST(AC.AutoVectorList);
01864 AC.AutoVectorList.message = "autovector";
01865 R_COPY_LIST(AC.AutoFunctionList);
01866 AC.AutoFunctionList.message = "autofunction";
01867
01868 R_COPY_NAMETREE(AC.autonames);
01869
01870 AC.Symbols = &(AC.SymbolList);
01871 AC.Indices = &(AC.IndexList);
01872 AC.Vectors = &(AC.VectorList);
01873 AC.Functions = &(AC.FunctionList);
01874 AC.activenames = &(AC.varnames);
01875
01876 org = (UBYTE*)AC.Streams;
01877 R_COPY_B(AC.Streams, AC.MaxNumStreams*(LONG)sizeof(STREAM), STREAM*);
01878 for ( i=0; i<AC.NumStreams; ++i ) {
01879     if ( AC.Streams[i].type != FILESTREAM ) {
01880         UBYTE *org2;
01881         org2 = AC.Streams[i].buffer;
01882         if ( AC.Streams[i].inbuffer ) {
01883             R_COPY_B(AC.Streams[i].buffer, AC.Streams[i].inbuffer, UBYTE*);
01884         }
01885         ofs = AC.Streams[i].buffer - org2;
01886         AC.Streams[i].pointer += ofs;
01887         AC.Streams[i].top += ofs;
01888     }
01889     else {
01890         p = (unsigned char*)p + AC.Streams[i].inbuffer;
01891     }
01892     AC.Streams[i].buffersize = AC.Streams[i].inbuffer;
01893     R_COPY_S(AC.Streams[i].FoldName,UBYTE*);
01894     R_COPY_S(AC.Streams[i].name,UBYTE*);
01895     if ( AC.Streams[i].type == PREVARSTREAM || AC.Streams[i].type == DOLLARSTREAM ) {
01896         AC.Streams[i].pname = AC.Streams[i].name;
01897     }
01898     else if ( AC.Streams[i].type == FILESTREAM ) {
01899         UBYTE *org2;
01900         org2 = AC.Streams[i].buffer;
01901         AC.Streams[i].buffer = (UBYTE*)Malloc1(AC.Streams[i].buffersize, "buffer");
01902         ofs = AC.Streams[i].buffer - org2;
01903         AC.Streams[i].pointer += ofs;
01904         AC.Streams[i].top += ofs;
01905
01906         /* open file except for already opened main input file */
01907         if ( i ) {
01908             AC.Streams[i].handle = OpenFile((char *) (AC.Streams[i].name));
01909             if ( AC.Streams[i].handle == -1 ) {
01910                 MesPrint("ERROR: Could not reopen stream %s!",AC.Streams[i].name);

```

```

01911         Terminate(-1);
01912     }
01913 }
01914
01915     PUTZERO(pos);
01916     ADDPOS(pos, AC.Streams[i].bufferposition);
01917     SeekFile(AC.Streams[i].handle, &pos, SEEK_SET);
01918
01919     AC.Streams[i].inbuffer = ReadFile(AC.Streams[i].handle, AC.Streams[i].buffer,
AC.Streams[i].inbuffer);
01920
01921     SETBASEPOSITION(pos, AC.Streams[i].fileposition);
01922     SeekFile(AC.Streams[i].handle, &pos, SEEK_SET);
01923 }
01924 /*
01925  * Ideally, we should check if we have a type PREREADSTREAM, PREREADSTREAM2, and
01926  * PRECALCSTREAM here. If so, we should free element name and point it
01927  * to the name element of the embracing stream's struct. In practice,
01928  * this is undoable without adding new data to STREAM. Since we create
01929  * only a small memory leak here (some few byte for each existing
01930  * stream of these types) and only once when we do a recovery, we
01931  * tolerate this leak and keep it STREAM as it is.
01932  */
01933 }
01934 ofs = (UBYTE*)AC.Streams - org;
01935 AC.CurrentStream = (STREAM*)((UBYTE*)AC.CurrentStream + ofs);
01936
01937 if ( AC.termstack ) {
01938     R_COPY_B(AC.termstack, AC.maxtermlevel*(LONG)sizeof(LONG), LONG*);
01939 }
01940
01941 if ( AC.termstack ) {
01942     R_COPY_B(AC.termstack, AC.maxtermlevel*(LONG)sizeof(LONG), LONG*);
01943 }
01944
01945 /* exception: here we also change values from struct AM */
01946 R_COPY_B(AC.cmod, AM.MaxTal*4*(LONG)sizeof(WORD), WORD*);
01947 AM.gcmmod = AC.cmod + AM.MaxTal;
01948 AC.powmod = AM.gcmmod + AM.MaxTal;
01949 AM.gpowmod = AC.powmod + AM.MaxTal;
01950
01951 AC.modpowers = 0;
01952 AC.halfmod = 0;
01953
01954 /* we don't care about AC.ProtoType/WildC */
01955
01956 if ( AC.IfHeap ) {
01957     ofs = AC.IfStack - AC.IfHeap;
01958     R_COPY_B(AC.IfHeap, (LONG)sizeof(LONG)*(AC.MaxIf+1), LONG*);
01959     AC.IfStack = AC.IfHeap + ofs;
01960     R_COPY_B(AC.IfCount, (LONG)sizeof(LONG)*(AC.MaxIf+1), LONG*);
01961 }
01962
01963 org = AC.iBuffer;
01964 size = AC.iStop - AC.iBuffer + 2;
01965 R_COPY_B(AC.iBuffer, size, UBYTE*);
01966 ofs = AC.iBuffer - org;
01967 AC.iPointer += ofs;
01968 AC.iStop += ofs;
01969
01970 if ( AC.LabelNames ) {
01971     org = (UBYTE*)AC.LabelNames;
01972     R_COPY_B(AC.LabelNames, AC.MaxLabels*(LONG)(sizeof(UBYTE*)+sizeof(WORD)), UBYTE*);
01973     for ( i=0; i<AC.NumLabels; ++i ) {
01974         R_COPY_S(AC.LabelNames[i], UBYTE*);
01975     }
01976     ofs = (UBYTE*)AC.LabelNames - org;
01977     AC.Labels = (int*)((UBYTE*)AC.Labels + ofs);
01978 }
01979
01980 R_COPY_B(AC.FixIndices, AM.OffsetIndex*(LONG)sizeof(WORD), WORD*);
01981
01982 if ( AC.termsumcheck ) {
01983     R_COPY_B(AC.termsumcheck, AC.maxtermlevel*(LONG)sizeof(WORD), WORD*);
01984 }
01985
01986 R_COPY_B(AC.WildcardNames, AC.WildcardBufferSize, UBYTE*);
01987
01988 if ( AC.tokens ) {
01989     size = AC.toptokens - AC.tokens;
01990     if ( size ) {
01991         org = (UBYTE*)AC.tokens;
01992         R_COPY_B(AC.tokens, size, SBYTE*);
01993         ofs = (UBYTE*)AC.tokens - org;
01994         AC.endoftokens += ofs;
01995         AC.toptokens += ofs;
01996     }

```



```

01997         else {
01998             AC.tokens = 0;
01999             AC.endoftokens = AC.tokens;
02000             AC.toptokens = AC.tokens;
02001         }
02002     }
02003
02004     R_COPY_B(AC.tokenarglevel, AM.MaxParLevel*(LONG)sizeof(WORD), WORD*);
02005
02006     AC.modinverses = 0;
02007
02008     R_COPY_S(AC.Fortran90Kind, UBYTE *);
02009
02010 #ifdef WITHPTHREADS
02011     if ( AC.inputnumbers ) {
02012         org = (UBYTE*)AC.inputnumbers;
02013         R_COPY_B(AC.inputnumbers, AC.sizepfirstnum*(LONG)(sizeof(WORD)+sizeof(LONG)), LONG*);
02014         ofs = (UBYTE*)AC.inputnumbers - org;
02015         AC.pfirstnum = (WORD*)((UBYTE*)AC.pfirstnum + ofs);
02016     }
02017     AC.halfmodlock = dummylock;
02018 #endif /* ifdef WITHPTHREADS */
02019
02020     if ( AC.IfSumCheck ) {
02021         R_COPY_B(AC.IfSumCheck, (LONG)sizeof(WORD)*(AC.MaxIf+1), WORD*);
02022     }
02023     if ( AC.CommuteInSet ) {
02024         R_COPY_B(AC.CommuteInSet, (LONG)sizeof(WORD)*(AC.SizeCommuteInSet+1), WORD*);
02025     }
02026
02027     AC.LogHandle = oldLogHandle;
02028
02029     R_COPY_S(AC.CheckpointRunAfter, char*);
02030     R_COPY_S(AC.CheckpointRunBefore, char*);
02031
02032     R_COPY_S(AC.extrasym, UBYTE *);
02033
02034 #ifdef PRINTDEBUG
02035     print_C();
02036 #endif
02037
02038     /*#] AC : */
02039     /*#[ AP : */
02040
02041     /* #[ AP free pointers */
02042
02043     /* AP will be overwritten by data from the recovery file, therefore
02044      * dynamically allocated memory must be freed first. */
02045
02046     for ( i=0; i<AP.DollarList.num; ++i ) {
02047         if ( Dollars[i].size ) {
02048             R_FREE(Dollars[i].where);
02049         }
02050         CleanDollarFactors(Dollars+i);
02051     }
02052     R_FREE(AP.DollarList.lijst);
02053
02054     for ( i=0; i<AP.PreVarList.num; ++i ) {
02055         R_FREE(PreVar[i].name);
02056     }
02057     R_FREE(AP.PreVarList.lijst);
02058
02059     for ( i=0; i<AP.LoopList.num; ++i ) {
02060         R_FREE(DoLoops[i].p.buffer);
02061         if ( DoLoops[i].dollarname ) {
02062             R_FREE(DoLoops[i].dollarname);
02063         }
02064     }
02065     R_FREE(AP.LoopList.lijst);
02066
02067     for ( i=0; i<AP.ProcList.num; ++i ) {
02068         R_FREE(Procedures[i].p.buffer);
02069         R_FREE(Procedures[i].name);
02070     }
02071     R_FREE(AP.ProcList.lijst);
02072
02073     for ( i=1; i<=AP.PreSwitchLevel; ++i ) {
02074         R_FREE(AP.PreSwitchStrings[i]);
02075     }
02076     R_FREE(AP.PreSwitchStrings);
02077     R_FREE(AP.preStart);
02078     R_FREE(AP.procedureExtension);
02079     R_FREE(AP.cprocedureExtension);
02080     R_FREE(AP.PreIfStack);
02081     R_FREE(AP.PreSwitchModes);
02082     R_FREE(AP.PreTypes);
02083

```

```

02084      /* #] AP free pointers */
02085
02086      /* first we copy AP as a whole and then restore the pointer structures step
02087      by step. */
02088
02089      AP = *((struct P_const*)p); p = (unsigned char*)p + sizeof(struct P_const);
02090 #ifdef WITHPTHREADS
02091      AP.PreVarLock = dummylock;
02092 #endif
02093
02094      R_COPY_LIST(AP.DollarList);
02095      for ( i=0; i<AP.DollarList.num; ++i ) {
02096          DOLLARS d = Dollars + i;
02097          size = d->size * sizeof(WORD);
02098          if ( size && d->where && d->where != oldAMDollarzero ) {
02099              R_COPY_B(d->where, size, void*);
02100          }
02101 #ifdef WITHPTHREADS
02102          d->pthreadlockread = dummylock;
02103          d->pthreadlockwrite = dummylock;
02104 #endif
02105          if ( d->nfactors > 1 ) {
02106              R_COPY_B(d->factors, sizeof(FACDOLLAR)*d->nfactors, FACDOLLAR*);
02107              for ( j = 0; j < d->nfactors; j++ ) {
02108                  if ( d->factors[j].size > 0 ) {
02109                      R_COPY_B(d->factors[j].where, sizeof(WORD)*(d->factors[j].size+1), WORD*);
02110                  }
02111              }
02112          }
02113      }
02114      AP.DollarList.message = "$-variable";
02115
02116      R_COPY_LIST(AP.PreVarList);
02117      for ( i=0; i<AP.PreVarList.num; ++i ) {
02118          R_SET(size, size_t);
02119          org = PreVar[i].name;
02120          R_COPY_B(PreVar[i].name, size, UBYTE*);
02121          ofs = PreVar[i].name - org;
02122          if ( PreVar[i].value ) PreVar[i].value += ofs;
02123          if ( PreVar[i].argnames ) PreVar[i].argnames += ofs;
02124      }
02125      AP.PreVarList.message = "PreVariable";
02126
02127      R_COPY_LIST(AP.LoopList);
02128      for ( i=0; i<AP.LoopList.num; ++i ) {
02129          org = DoLoops[i].p.buffer;
02130          R_COPY_B(DoLoops[i].p.buffer, DoLoops[i].p.size, UBYTE*);
02131          ofs = DoLoops[i].p.buffer - org;
02132          if ( DoLoops[i].name ) DoLoops[i].name += ofs;
02133          if ( DoLoops[i].vars ) DoLoops[i].vars += ofs;
02134          if ( DoLoops[i].contents ) DoLoops[i].contents += ofs;
02135          if ( DoLoops[i].type == ONEEXPRESSION ) {
02136              R_COPY_S(DoLoops[i].dollarname, UBYTE*);
02137          }
02138      }
02139      AP.LoopList.message = "doloop";
02140
02141      R_COPY_LIST(AP.ProcList);
02142      for ( i=0; i<AP.ProcList.num; ++i ) {
02143          if ( Procedures[i].p.size ) {
02144              if ( Procedures[i].loadmode != 1 ) {
02145                  R_COPY_B(Procedures[i].p.buffer, Procedures[i].p.size, UBYTE*);
02146              }
02147              else {
02148                  R_SET(j, int);
02149                  Procedures[i].p.buffer = Procedures[j].p.buffer;
02150              }
02151          }
02152          R_COPY_S(Procedures[i].name, UBYTE*);
02153      }
02154      AP.ProcList.message = "procedure";
02155
02156      size = (AP.NumPreSwitchStrings+1)*(LONG)sizeof(UBYTE*);
02157      R_COPY_B(AP.PreSwitchStrings, size, UBYTE*);
02158      for ( i=1; i<=AP.PreSwitchLevel; ++i ) {
02159          R_COPY_S(AP.PreSwitchStrings[i], UBYTE*);
02160      }
02161
02162      org = AP.preStart;
02163      R_COPY_B(AP.preStart, AP.pSize, UBYTE*);
02164      ofs = AP.preStart - org;
02165      if ( AP.preFill ) AP.preFill += ofs;
02166      if ( AP.preStop ) AP.preStop += ofs;
02167
02168      R_COPY_S(AP.procedureExtension, UBYTE*);
02169      R_COPY_S(AP.cprocedureExtension, UBYTE*);
02170

```

```

02171     R_COPY_B(AP.PreAssignStack, AP.MaxPreAssignLevel*(LONG)sizeof(LONG), LONG*);
02172     R_COPY_B(AP.PreIfStack, AP.MaxPreIfLevel*(LONG)sizeof(int), int*);
02173     R_COPY_B(AP.PreSwitchModes, (AP.NumPreSwitchStrings+1)*(LONG)sizeof(int), int*);
02174     R_COPY_B(AP.PreTypes, (AP.MaxPreTypes+1)*(LONG)sizeof(int), int*);
02175
02176 #ifdef PRINTDEBUG
02177     print_P();
02178 #endif
02179
02180     /*#] AP : */
02181     /*#[ AR : */
02182
02183     R_SET(ofs, LONG);
02184     if ( ofs ) {
02185         AR.infile = AR.Fscr+1;
02186         AR.outfile = AR.Fscr;
02187         AR.hidefile = AR.Fscr+2;
02188     }
02189     else {
02190         AR.infile = AR.Fscr;
02191         AR.outfile = AR.Fscr+1;
02192         AR.hidefile = AR.Fscr+2;
02193     }
02194
02195     /* #[ AR free pointers */
02196
02197     /* Parts of AR will be overwritten by data from the recovery file, therefore
02198      * dynamically allocated memory must be freed first. */
02199
02200     namebufout = AR.outfile->name;
02201     namebufhide = AR.hidefile->name;
02202     R_FREE(AR.outfile->PObuffer);
02203 #ifdef WITHZLIB
02204     R_FREE(AR.outfile->zsp);
02205     R_FREE(AR.outfile->ziobuffer);
02206 #endif
02207     namebufhide = AR.hidefile->name;
02208     R_FREE(AR.hidefile->PObuffer);
02209 #ifdef WITHZLIB
02210     R_FREE(AR.hidefile->zsp);
02211     R_FREE(AR.hidefile->ziobuffer);
02212 #endif
02213     /* no files should be opened -> nothing to do with handle */
02214
02215     /* #] AR free pointers */
02216
02217     /* outfile */
02218     R_SET(*AR.outfile, FILEHANDLE);
02219     org = (UBYTE*)AR.outfile->PObuffer;
02220     size = AR.outfile->POfull - AR.outfile->PObuffer;
02221     AR.outfile->PObuffer = (WORD*)Malloc1(AR.outfile->POsize, "PObuffer");
02222     if ( size ) {
02223         memcpy(AR.outfile->PObuffer, p, size*sizeof(WORD));
02224         p = (unsigned char*)p + size*sizeof(WORD);
02225     }
02226     ofs = (UBYTE*)AR.outfile->PObuffer - org;
02227     AR.outfile->POstop = (WORD*)((UBYTE*)AR.outfile->POstop + ofs);
02228     AR.outfile->POfill = (WORD*)((UBYTE*)AR.outfile->POfill + ofs);
02229     AR.outfile->POfull = (WORD*)((UBYTE*)AR.outfile->POfull + ofs);
02230     AR.outfile->name = namebufout;
02231 #ifdef WITHPTHREADS
02232     AR.outfile->wPObuffer = AR.outfile->PObuffer;
02233     AR.outfile->wPOstop = AR.outfile->POstop;
02234     AR.outfile->wPOfill = AR.outfile->POfill;
02235     AR.outfile->wPOfull = AR.outfile->POfull;
02236 #endif
02237 #ifdef WITHZLIB
02238     /* zsp and ziobuffer will be allocated when used */
02239     AR.outfile->zsp = 0;
02240     AR.outfile->ziobuffer = 0;
02241 #endif
02242     /* reopen old outfile */
02243 #ifdef WITHMPI
02244     if (PF.me==MASTER)
02245 #endif
02246     if ( AR.outfile->handle >= 0 ) {
02247         if ( CopyFile(sortfile, AR.outfile->name) ) {
02248             MesPrint("ERROR: Could not copy old output sort file %s!", sortfile);
02249             Terminate(-1);
02250         }
02251         AR.outfile->handle = ReOpenFile(AR.outfile->name);
02252         if ( AR.outfile->handle == -1 ) {
02253             MesPrint("ERROR: Could not reopen output sort file %s!", AR.outfile->name);
02254             Terminate(-1);
02255         }
02256         SeekFile(AR.outfile->handle, &AR.outfile->POposition, SEEK_SET);
02257     }

```

```

02258
02259  /* hidefile */
02260  R_SET(*AR.hidefile, FILEHANDLE);
02261  AR.hidefile->name = namebufhide;
02262  if ( AR.hidefile->PObuffer ) {
02263      org = (UBYTE*)AR.hidefile->PObuffer;
02264      size = AR.hidefile->POfull - AR.hidefile->PObuffer;
02265      AR.hidefile->PObuffer = (WORD*)Malloc1(AR.hidefile->POsize, "PObuffer");
02266      if ( size ) {
02267          memcpy(AR.hidefile->PObuffer, p, size*sizeof(WORD));
02268          p = (unsigned char*)p + size*sizeof(WORD);
02269      }
02270      ofs = (UBYTE*)AR.hidefile->PObuffer - org;
02271      AR.hidefile->POstop = (WORD*)((UBYTE*)AR.hidefile->POstop + ofs);
02272      AR.hidefile->POfill = (WORD*)((UBYTE*)AR.hidefile->POfill + ofs);
02273      AR.hidefile->POfull = (WORD*)((UBYTE*)AR.hidefile->POfull + ofs);
02274  #ifndef WITHPTHREADS
02275      AR.hidefile->wPObuffer = AR.hidefile->PObuffer;
02276      AR.hidefile->wPOstop = AR.hidefile->POstop;
02277      AR.hidefile->wPOfill = AR.hidefile->POfill;
02278      AR.hidefile->wPOfull = AR.hidefile->POfull;
02279  #endif
02280  }
02281  #ifdef WITHZLIB
02282      /* zsp and ziobuffer will be allocated when used */
02283      AR.hidefile->zsp = 0;
02284      AR.hidefile->ziobuffer = 0;
02285  #endif
02286  /* reopen old hidefile */
02287  if ( AR.hidefile->handle >= 0 ) {
02288      if ( CopyFile(hidefile, AR.hidefile->name) ) {
02289          MesPrint("ERROR: Could not copy old hide file %s!",hidefile);
02290          Terminate(-1);
02291      }
02292      AR.hidefile->handle = ReOpenFile(AR.hidefile->name);
02293      if ( AR.hidefile->handle == -1 ) {
02294          MesPrint("ERROR: Could not reopen hide file %s!",AR.hidefile->name);
02295          Terminate(-1);
02296      }
02297      SeekFile(AR.hidefile->handle, &AR.hidefile->POposition, SEEK_SET);
02298  }
02299
02300  /* store file */
02301  R_SET(pos, POSITION);
02302  if ( ISNOTZEROPOS(pos) ) {
02303      CloseFile(AR.StoreData.Handle);
02304      R_SET(AR.StoreData, FILEDATA);
02305      if ( CopyFile(storefile, FG.fname) ) {
02306          MesPrint("ERROR: Could not copy old store file %s!",storefile);
02307          Terminate(-1);
02308      }
02309      AR.StoreData.Handle = (WORD)ReOpenFile(FG.fname);
02310      SeekFile(AR.StoreData.Handle, &AR.StoreData.Position, SEEK_SET);
02311  }
02312
02313  R_SET(AR.DefPosition, POSITION);
02314  R_SET(AR.OldTime, LONG);
02315  R_SET(AR.InInBuf, LONG);
02316  R_SET(AR.InHiBuf, LONG);
02317
02318  R_SET(AR.NoCompress, int);
02319  R_SET(AR.gzipCompress, int);
02320
02321  R_SET(AR.outtohide, int);
02322
02323  R_SET(AR.GetFile, WORD);
02324  R_SET(AR.KeptInHold, WORD);
02325  R_SET(AR.BracketOn, WORD);
02326  R_SET(AR.MaxBracket, WORD);
02327  R_SET(AR.CurDum, WORD);
02328  R_SET(AR.DeferFlag, WORD);
02329  R_SET(AR.TePos, WORD);
02330  R_SET(AR.sLevel, WORD);
02331  R_SET(AR.Stage4Name, WORD);
02332  R_SET(AR.GetOneFile, WORD);
02333  R_SET(AR.PolyFun, WORD);
02334  R_SET(AR.PolyFunInv, WORD);
02335  R_SET(AR.PolyFunType, WORD);
02336  R_SET(AR.PolyFunExp, WORD);
02337  R_SET(AR.PolyFunVar, WORD);
02338  R_SET(AR.PolyFunPow, WORD);
02339  R_SET(AR.Eside, WORD);
02340  R_SET(AR.MaxDum, WORD);
02341  R_SET(AR.level, WORD);
02342  R_SET(AR.expchanged, WORD);
02343  R_SET(AR.expflags, WORD);
02344  R_SET(AR.CurExpr, WORD);

```

```

02345     R_SET(AR.SortType, WORD);
02346     R_SET(AR.ShortSortCount, WORD);
02347
02348     /* this is usually done in Process(), but sometimes FORM does not
02349        end up executing Process() before it uses the AR.CompressPointer,
02350        so we need to explicitly set it here. */
02351     AR.CompressPointer = AR.CompressBuffer;
02352
02353 #ifdef WITHPTHREADS
02354     for ( j = 0; j < AM.totalnumberofthreads; j++ ) {
02355         R_SET(AB[j]->R.wranfnpair1, int);
02356         R_SET(AB[j]->R.wranfnpair2, int);
02357         R_SET(AB[j]->R.wranfcall, int);
02358         R_SET(AB[j]->R.wranfseed, ULONG);
02359         R_SET(AB[j]->R.wranfia, ULONG*);
02360         if ( AB[j]->R.wranfia ) {
02361             R_COPY_B(AB[j]->R.wranfia, sizeof(ULONG)*AB[j]->R.wranfnpair2, ULONG*);
02362         }
02363     }
02364 #else
02365     R_SET(AR.wranfnpair1, int);
02366     R_SET(AR.wranfnpair2, int);
02367     R_SET(AR.wranfcall, int);
02368     R_SET(AR.wranfseed, ULONG);
02369     R_SET(AR.wranfia, ULONG*);
02370     if ( AR.wranfia ) {
02371         R_COPY_B(AR.wranfia, sizeof(ULONG)*AR.wranfnpair2, ULONG*);
02372     }
02373 #endif
02374
02375 #ifdef PRINTDEBUG
02376     print_R();
02377 #endif
02378
02379     /*#] AR : */
02380     /*#[ AO :*/
02381     /*
02382     We copy all non-pointer variables.
02383     */
02384     l = sizeof(A.O) - ((UBYTE *)(&(A.O.NumInBrack))-(UBYTE *)(&A.O));
02385     memcpy(&(A.O.NumInBrack), p, l); p = (unsigned char*)p + l;
02386     /*
02387     Now the variables in OptimizeResult
02388     */
02389     memcpy(&(A.O.OptimizeResult), p, sizeof(OPTIMIZERESULT));
02390     p = (unsigned char*)p + sizeof(OPTIMIZERESULT);
02391
02392     if ( A.O.OptimizeResult.codesize > 0 ) {
02393         R_COPY_B(A.O.OptimizeResult.code, A.O.OptimizeResult.codesize*sizeof(WORD), WORD *);
02394     }
02395     R_COPY_S(A.O.OptimizeResult.nameofexpr, UBYTE *);
02396     /*
02397     And now the dictionaries. We know how many there are. We also know
02398     how many elements the array AO.Dictionaries should have.
02399     */
02400     if ( AO.SizeDictionaries > 0 ) {
02401         AO.Dictionaries = (DICTIONARY **)Malloc1(AO.SizeDictionaries*sizeof(DICTIONARY *),
02402            "Dictionaries");
02403         for ( i = 0; i < AO.NumDictionaries; i++ ) {
02404             R_SET(i, LONG)
02405             AO.Dictionaries[i] = DictFromBytes(p);
02406             p = (char *)p + l;
02407         }
02408     }
02409     /*#] AO :*/
02410 #ifdef WITHMPI
02411     /*#[ PF : */
02412     /*Block*/
02413     int numtasks;
02414     R_SET(numtasks, int);
02415     if(numtasks!=PF.numtasks){
02416         MesPrint("%d number of tasks expected instead of %d; use mpirun -np %d",
02417             numtasks, PF.numtasks, numtasks);
02418         if(PF.me!=MASTER)
02419             remove(RecoveryFilename());
02420         Terminate(-1);
02421     }
02422     /*Block*/
02423     R_SET(PF.rhsInParallel, int);
02424     R_SET(PF.exprbufsize, int);
02425     R_SET(PF.log, int);
02426     /*#] PF : */
02427 #endif
02428
02429 #ifdef WITHPTHREADS
02430     /* read timing information of individual threads */
02431     R_SET(i, int);

```

```

02432     for ( j=1; j<AM.totalnumberofthreads; ++j ) {
02433         /* ... and correcting OldTime */
02434         AB[j]->R.OldTime = -(*(LONG*)p+j));
02435     }
02436     WriteTimerInfo((LONG*)p, (LONG *) ((unsigned char*)p + i*(LONG)sizeof(LONG)));
02437     p = (unsigned char*)p + 2*i*(LONG)sizeof(LONG);
02438 #endif /* ifdef WITHPTHREADS */
02439
02440     if ( fclose(fd) ) return(__LINE__);
02441
02442     M_free(buf, "recovery buffer");
02443
02444     /* cares about data in S_const */
02445     UpdatePositions();
02446     AT.SS = AT.S0;
02447 /*
02448     Set the checkpoint parameter right for the next checkpoint.
02449 */
02450     AC.CheckpointStamp = TimeWallClock(1);
02451
02452     done_snapshot = 1;
02453     MesPrint("done."); fflush(0);
02454
02455     return(0);
02456 }
02457
02458 /*
02459     #] DoRecovery :
02460     #[ DoSnapshot :
02461 */
02462
02463 static int DoSnapshot(int moduletype)
02464 {
02465     GETIDENTITY
02466     FILE *fd;
02467     POSITION pos;
02468     int i, j;
02469     LONG l;
02470     WORD *w;
02471     void *adr;
02472 #ifdef WITHPTHREADS
02473     LONG *longp, *longpp;
02474 #endif /* ifdef WITHPTHREADS */
02475
02476     MesPrint("Saving recovery point ... %"); fflush(0);
02477 #ifdef PRINTTIMEMARKS
02478     MesPrint("\n");
02479 #endif
02480
02481     if ( !(fd = fopen(intermedfile, "wb")) ) return(__LINE__);
02482
02483     /* reserve space in the file for a length field */
02484     if ( fwrite(&pos, 1, sizeof(POSITION), fd) != sizeof(POSITION) ) return(__LINE__);
02485
02486     /* write moduletype */
02487     if ( fwrite(&moduletype, 1, sizeof(int), fd) != sizeof(int) ) return(__LINE__);
02488
02489     /*#[ AM :*/
02490
02491     /* since most values don't change during execution, AM doesn't need to be
02492      * written as a whole. all values will be correctly set when starting up
02493      * anyway. only the exceptions need to be taken care of. see MakeGlobal()
02494      * and PopVariables() in execute.c. */
02495
02496     ANNOUNCE(AM)
02497     S_WRITE_B(&AM.hparallelflag, sizeof(int));
02498     S_WRITE_B(&AM.gparallelflag, sizeof(int));
02499     S_WRITE_B(&AM.gCodesFlag, sizeof(int));
02500     S_WRITE_B(&AM.gNamesFlag, sizeof(int));
02501     S_WRITE_B(&AM.gStatsFlag, sizeof(int));
02502     S_WRITE_B(&AM.gTokensWriteFlag, sizeof(int));
02503     S_WRITE_B(&AM.gNoSpacesInNumbers, sizeof(int));
02504     S_WRITE_B(&AM.gIndentSpace, sizeof(WORD));
02505     S_WRITE_B(&AM.gUnitTrace, sizeof(WORD));
02506     S_WRITE_B(&AM.gDefDim, sizeof(int));
02507     S_WRITE_B(&AM.gDefDim4, sizeof(int));
02508     S_WRITE_B(&AM.gncmod, sizeof(WORD));
02509     S_WRITE_B(&AM.gnpowmod, sizeof(WORD));
02510     S_WRITE_B(&AM.gmodmode, sizeof(WORD));
02511     S_WRITE_B(&AM.gOutputMode, sizeof(WORD));
02512     S_WRITE_B(&AM.gCnumpows, sizeof(WORD));
02513     S_WRITE_B(&AM.gOutputSpaces, sizeof(WORD));
02514     S_WRITE_B(&AM.gOutNumberType, sizeof(WORD));
02515     S_WRITE_B(&AM.gfunpowers, sizeof(int));
02516     S_WRITE_B(&AM.gPolyFun, sizeof(WORD));
02517     S_WRITE_B(&AM.gPolyFunInv, sizeof(WORD));
02518     S_WRITE_B(&AM.gPolyFunType, sizeof(WORD));

```

```

02534     S_WRITE_B(&AM.gPolyFunExp, sizeof(WORD));
02535     S_WRITE_B(&AM.gPolyFunVar, sizeof(WORD));
02536     S_WRITE_B(&AM.gPolyFunPow, sizeof(WORD));
02537     S_WRITE_B(&AM.gProcessBucketSize, sizeof(LONG));
02538     S_WRITE_B(&AM.OldChildTime, sizeof(LONG));
02539     S_WRITE_B(&AM.OldSecTime, sizeof(LONG));
02540     S_WRITE_B(&AM.OldMilliTime, sizeof(LONG));
02541     S_WRITE_B(&AM.gproperorderflag, sizeof(int));
02542     S_WRITE_B(&AM.gThreadBucketSize, sizeof(LONG));
02543     S_WRITE_B(&AM.gSizeCommuteInSet, sizeof(int));
02544     S_WRITE_B(&AM.gThreadStats, sizeof(int));
02545     S_WRITE_B(&AM.gFinalStats, sizeof(int));
02546     S_WRITE_B(&AM.gThreadsFlag, sizeof(int));
02547     S_WRITE_B(&AM.gThreadBalancing, sizeof(int));
02548     S_WRITE_B(&AM.gThreadSortFileSynch, sizeof(int));
02549     S_WRITE_B(&AM.gProcessStats, sizeof(int));
02550     S_WRITE_B(&AM.gOldParallelStats, sizeof(int));
02551     S_WRITE_B(&AM.gSortType, sizeof(int));
02552     S_WRITE_B(&AM.gShortStatsMax, sizeof(WORD));
02553     S_WRITE_B(&AM.gIsFortran90, sizeof(int));
02554     adr = &AM.dollarzero;
02555     S_WRITE_B(adr, sizeof(void*));
02556     S_WRITE_B(&AM.gFortran90Kind, sizeof(UBYTE *));
02557     S_WRITE_S(AM.gFortran90Kind);
02558
02559     S_WRITE_S(AM.gextrasymp);
02560     S_WRITE_S(AM.ggextrasymp);
02561
02562     S_WRITE_B(&AM.PrintTotalSize, sizeof(int));
02563     S_WRITE_B(&AM.fbuffersize, sizeof(int));
02564     S_WRITE_B(&AM.gOldFactArgFlag, sizeof(int));
02565     S_WRITE_B(&AM.ggOldFactArgFlag, sizeof(int));
02566
02567     S_WRITE_B(&AM.gnumextrasymp, sizeof(int));
02568     S_WRITE_B(&AM.ggnumextrasymp, sizeof(int));
02569     S_WRITE_B(&AM.NumSpectatorFiles, sizeof(int));
02570     S_WRITE_B(&AM.SizeForSpectatorFiles, sizeof(int));
02571     S_WRITE_B(&AM.gOldGCDflag, sizeof(int));
02572     S_WRITE_B(&AM.ggOldGCDflag, sizeof(int));
02573     S_WRITE_B(&AM.gWTimeStatsFlag, sizeof(int));
02574
02575     S_WRITE_B(&AM.Path, sizeof(UBYTE *));
02576     S_WRITE_S(AM.Path);
02577
02578     S_WRITE_B(&AM.FromStdin, sizeof(BOOL));
02579     S_WRITE_B(&AM.IgnoreDeprecation, sizeof(BOOL));
02580
02581     /*#] AM :*/
02582     /*#[ AC :*/
02583
02584     /* we write AC as a whole and then write all additional data step by step.
02585      * AC.DubiousList doesn't need to be treated, because it should be empty. */
02586
02587     ANNOUNCE(AC)
02588     S_WRITE_B(&AC, sizeof(struct C_const));
02589
02590     S_WRITE_NAMETREE(AC.dollarnames);
02591     S_WRITE_NAMETREE(AC.exprnames);
02592     S_WRITE_NAMETREE(AC.varnames);
02593
02594     S_WRITE_LIST(AC.ChannelList);
02595     for ( i=0; i<AC.ChannelList.num; ++i ) {
02596         S_WRITE_S(channels[i].name);
02597     }
02598
02599     ANNOUNCE(AC.FunctionList)
02600     S_WRITE_LIST(AC.FunctionList);
02601     for ( i=0; i<AC.FunctionList.num; ++i ) {
02602         /* if the function is a table */
02603         if ( functions[i].tabl ) {
02604             TABLES tabl = functions[i].tabl;
02605             S_WRITE_B(tabl, sizeof(struct TabLEs));
02606             if ( tabl->tablepointers ) {
02607                 if ( tabl->sparse ) {
02608                     /* sparse tables. reserved holds number of allocated
02609                      * elements. the size of an element is numind plus
02610                      * TABLEEXTENSION times the size of WORD. */
02611                     S_WRITE_B(tabl->tablepointers,
02612                               tabl->reserved*sizeof(WORD)*(tabl->numind+TABLEEXTENSION));
02613                 }
02614                 else {
02615                     /* matrix like tables. */
02616                     S_WRITE_B(tabl->tablepointers,
02617                               TABLEEXTENSION*sizeof(WORD)*(tabl->totind));
02618                 }
02619             }
02620             S_WRITE_B(tabl->prototype, tabl->prototypeSize);

```

```

02621     S_WRITE_B(tabl->mm, tabl->numind*(LONG)sizeof(MINMAX));
02622     S_WRITE_B(tabl->flags, tabl->numind*(LONG)sizeof(WORD));
02623     if ( tabl->sparse ) {
02624         S_WRITE_B(tabl->boomlijst, tabl->MaxTreeSize*(LONG)sizeof(COMPTREE));
02625         S_WRITE_S(tabl->argtail);
02626     }
02627     S_WRITE_B(tabl->buffers, tabl->bufferssize*(LONG)sizeof(WORD));
02628     if ( tabl->sparse ) {
02629         TABLES spare = tabl->spare;
02630         S_WRITE_B(spare, sizeof(struct TaBLeS));
02631         if ( spare->tablepointers ) {
02632             if ( spare->sparse ) {
02633                 /* sparse tables */
02634                 S_WRITE_B(spare->tablepointers,
02635                     spare->reserved*sizeof(WORD)*(spare->numind+TABLEEXTENSION));
02636             }
02637             else {
02638                 /* matrix like tables */
02639                 S_WRITE_B(spare->tablepointers,
02640                     TABLEEXTENSION*sizeof(WORD)*(spare->totind));
02641             }
02642         }
02643         S_WRITE_B(spare->mm, spare->numind*(LONG)sizeof(MINMAX));
02644         S_WRITE_B(spare->flags, spare->numind*(LONG)sizeof(WORD));
02645         if ( spare->sparse ) {
02646             S_WRITE_B(spare->boomlijst, spare->MaxTreeSize*(LONG)sizeof(COMPTREE));
02647         }
02648         S_WRITE_B(spare->buffers, spare->bufferssize*(LONG)sizeof(WORD));
02649     }
02650 }
02651 }
02652
02653 ANNOUNCE(AC.ExpressionList)
02654 S_WRITE_LIST(AC.ExpressionList);
02655 for ( i=0; i<AC.ExpressionList.num; ++i ) {
02656     EXPRESSIONS ex = Expressions + i;
02657     if ( ex->renum ) {
02658         S_WRITE_B(ex->renum, sizeof(struct ReNuMbEr));
02659         /* there is one dynamically allocated buffer for struct ReNuMbEr and
02660          * symb.lo points to its beginning. the size of the buffer is not
02661          * stored anywhere but we know it is 2*sizeof(WORD)*N, where N is
02662          * the number of all vectors, indices, functions and symbols. since
02663          * funum points into the buffer at a distance 2N-[Number of
02664          * functions] from symb.lo (see GetTable() in store.c), we can
02665          * calculate the buffer size by some pointer arithmetic. the size is
02666          * then written to the file. */
02667         l = ex->renum->funnum - ex->renum->symb.lo;
02668         l += ex->renum->funnum - ex->renum->func.lo;
02669         S_WRITE_B(&l, sizeof(size_t));
02670         S_WRITE_B(ex->renum->symb.lo, l);
02671     }
02672     if ( ex->bracketinfo ) {
02673         S_WRITE_B(ex->bracketinfo, sizeof(BRACKETINFO));
02674         S_WRITE_B(ex->bracketinfo->indexbuffer,
02675             ex->bracketinfo->indexbuffersize*sizeof(BRACKETINDEX));
02676         S_WRITE_B(ex->bracketinfo->bracketbuffer,
02677             ex->bracketinfo->bracketbuffersize*sizeof(WORD));
02678     }
02679     if ( ex->newbracketinfo ) {
02680         S_WRITE_B(ex->newbracketinfo, sizeof(BRACKETINFO));
02681         S_WRITE_B(ex->newbracketinfo->indexbuffer,
02682             ex->newbracketinfo->indexbuffersize*sizeof(BRACKETINDEX));
02683         S_WRITE_B(ex->newbracketinfo->bracketbuffer,
02684             ex->newbracketinfo->bracketbuffersize*sizeof(WORD));
02685     }
02686     /* don't need to write ex->renumlists */
02687     if ( ex->inmem ) {
02688         /* size of the inmem buffer has to be determined. we use the fact
02689          * that the end of an expression is marked by a zero. */
02690         w = ex->inmem;
02691         while ( *w++ );
02692         l = w - ex->inmem;
02693         S_WRITE_B(&l, sizeof(size_t));
02694         S_WRITE_B(ex->inmem, l);
02695     }
02696 }
02697
02698 ANNOUNCE(AC.IndexList)
02699 S_WRITE_LIST(AC.IndexList);
02700 S_WRITE_LIST(AC.SetElementList);
02701 S_WRITE_LIST(AC.SetList);
02702 S_WRITE_LIST(AC.SymbolList);
02703 S_WRITE_LIST(AC.VectorList);
02704
02705 ANNOUNCE(AC.TableBaseList)
02706 S_WRITE_LIST(AC.TableBaseList);
02707 for ( i=0; i<AC.TableBaseList.num; ++i ) {

```



```

02704      /* see struct dbase in minos.h */
02705      if ( tablebases[i].iblocks ) {
02706          S_WRITE_B(tablebases[i].iblocks, tablebases[i].info.numberofindexblocks *
sizeof(INDEXBLOCK*));
02707          for ( j=0; j<tablebases[i].info.numberofindexblocks; ++j ) {
02708              if ( tablebases[i].iblocks[j] ) {
02709                  S_WRITE_B(tablebases[i].iblocks[j], sizeof(INDEXBLOCK));
02710              }
02711          }
02712      }
02713      if ( tablebases[i].nblocks ) {
02714          S_WRITE_B(tablebases[i].nblocks, tablebases[i].info.numberofnamesblocks *
sizeof(NAMESBLOCK*));
02715          for ( j=0; j<tablebases[i].info.numberofnamesblocks; ++j ) {
02716              if ( tablebases[i].nblocks[j] ) {
02717                  S_WRITE_B(tablebases[i].nblocks[j], sizeof(NAMESBLOCK));
02718              }
02719          }
02720      }
02721      S_WRITE_S(tablebases[i].name);
02722      S_WRITE_S(tablebases[i].fullname);
02723      S_WRITE_S(tablebases[i].tablenames);
02724  }
02725
02726  ANNOUNCE(AC.cbufList)
02727  S_WRITE_LIST(AC.cbufList);
02728  for ( i=0; i<AC.cbufList.num; ++i ) {
02729      S_WRITE_B(cbuf[i].Buffer, cbuf[i].BufferSize*sizeof(WORD));
02730      /* see inicbufs in comtool.c */
02731      S_WRITE_B(cbuf[i].lhs, cbuf[i].maxlhs*(LONG)sizeof(WORD*));
02732      S_WRITE_B(cbuf[i].rhs, cbuf[i].maxrhs*(LONG)(sizeof(WORD*)+2*sizeof(LONG)+2*sizeof(WORD)));
02733      if ( cbuf[i].boomlijst ) {
02734          S_WRITE_B(cbuf[i].boomlijst, cbuf[i].MaxTreeSize*(LONG)sizeof(COMPTREE));
02735      }
02736  }
02737
02738  S_WRITE_LIST(AC.AutoSymbolList);
02739  S_WRITE_LIST(AC.AutoIndexList);
02740  S_WRITE_LIST(AC.AutoVectorList);
02741  S_WRITE_LIST(AC.AutoFunctionList);
02742
02743  S_WRITE_NAMETREE(AC.autonames);
02744
02745  ANNOUNCE(AC.Streams)
02746  S_WRITE_B(AC.Streams, AC.MaxNumStreams*(LONG)sizeof(STREAM));
02747  for ( i=0; i<AC.NumStreams; ++i ) {
02748      if ( AC.Streams[i].inbuffer ) {
02749          S_WRITE_B(AC.Streams[i].buffer, AC.Streams[i].inbuffer);
02750      }
02751      S_WRITE_S(AC.Streams[i].FoldName);
02752      S_WRITE_S(AC.Streams[i].name);
02753  }
02754
02755  if ( AC.termstack ) {
02756      S_WRITE_B(AC.termstack, AC.maxtermlevel*(LONG)sizeof(LONG));
02757  }
02758
02759  if ( AC.termstortstack ) {
02760      S_WRITE_B(AC.termstortstack, AC.maxtermlevel*(LONG)sizeof(LONG));
02761  }
02762
02763  S_WRITE_B(AC.cmod, AM.MaxTal*4*(LONG)sizeof(UWORD));
02764
02765  if ( AC.IfHeap ) {
02766      S_WRITE_B(AC.IfHeap, (LONG)sizeof(LONG)*(AC.MaxIf+1));
02767      S_WRITE_B(AC.IfCount, (LONG)sizeof(LONG)*(AC.MaxIf+1));
02768  }
02769
02770  l = AC.iStop - AC.iBuffer + 2;
02771  S_WRITE_B(AC.iBuffer, l);
02772
02773  if ( AC.LabelNames ) {
02774      S_WRITE_B(AC.LabelNames, AC.MaxLabels*(LONG)(sizeof(UBYTE*)+sizeof(WORD)));
02775      for ( i=0; i<AC.NumLabels; ++i ) {
02776          S_WRITE_S(AC.LabelNames[i]);
02777      }
02778  }
02779
02780  S_WRITE_B(AC.FixIndices, AM.OffsetIndex*(LONG)sizeof(WORD));
02781
02782  if ( AC.termsumcheck ) {
02783      S_WRITE_B(AC.termsumcheck, AC.maxtermlevel*(LONG)sizeof(WORD));
02784  }
02785
02786  S_WRITE_B(AC.WildcardNames, AC.WildcardBufferSize);
02787
02788  if ( AC.tokens ) {

```

```

02789         l = AC.toptokens - AC.tokens;
02790         if ( l ) {
02791             S_WRITE_B(AC.tokens, l);
02792         }
02793     }
02794
02795     S_WRITE_B(AC.tokenarglevel, AM.MaxParLevel*(LONG)sizeof(WORD));
02796
02797     S_WRITE_S(AC.Fortran90Kind);
02798
02799 #ifdef WITHPTHREADS
02800     if ( AC.inputnumbers ) {
02801         S_WRITE_B(AC.inputnumbers, AC.sizepfirstnum*(LONG) (sizeof(WORD)+sizeof(LONG)));
02802     }
02803 #endif /* ifdef WITHPTHREADS */
02804
02805     if ( AC.IfSumCheck ) {
02806         S_WRITE_B(AC.IfSumCheck, (LONG)sizeof(WORD)*(AC.MaxIf+1));
02807     }
02808     if ( AC.CommuteInSet ) {
02809         S_WRITE_B(AC.CommuteInSet, (LONG)sizeof(WORD)*(AC.SizeCommuteInSet+1));
02810     }
02811
02812     S_WRITE_S(AC.CheckpointRunAfter);
02813     S_WRITE_S(AC.CheckpointRunBefore);
02814
02815     S_WRITE_S(AC.extrasym);
02816
02817     /*# AC :*/
02818     /*# AP :*/
02819
02820     /* we write AP as a whole and then write all additional data step by step. */
02821
02822     ANNOUNCE(AP)
02823     S_WRITE_B(&AP, sizeof(struct P_const));
02824
02825     S_WRITE_LIST(AP.DollarList);
02826     for ( i=0; i<AP.DollarList.num; ++i ) {
02827         DOLLARS d = Dollars + i;
02828         S_WRITE_DOLLAR(Dollars[i]);
02829         if ( d->nfactors > 1 ) {
02830             S_WRITE_B(&(d->factors),sizeof(FACDOLLAR)*d->nfactors);
02831             for ( j = 0; j < d->nfactors; j++ ) {
02832                 if ( d->factors[j].size > 0 ) {
02833                     S_WRITE_B(&(d->factors[i].where),sizeof(WORD)*(d->factors[j].size+1));
02834                 }
02835             }
02836         }
02837     }
02838
02839     S_WRITE_LIST(AP.PreVarList);
02840     for ( i=0; i<AP.PreVarList.num; ++i ) {
02841         /* there is one dynamically allocated buffer in struct pReVar holding
02842          * the strings name, value and several argnames. the size of the buffer
02843          * can be calculated by adding up their string lengths. */
02844         l = strlen((char*)PreVar[i].name) + 1;
02845         if ( PreVar[i].value ) {
02846             l += strlen((char*)(PreVar[i].name+1)) + 1;
02847         }
02848         for ( j=0; j<PreVar[i].nargs; ++j ) {
02849             l += strlen((char*)(PreVar[i].name+1)) + 1;
02850         }
02851         S_WRITE_B(&l, sizeof(size_t));
02852         S_WRITE_B(PreVar[i].name, l);
02853     }
02854
02855     ANNOUNCE(AP.LoopList)
02856     S_WRITE_LIST(AP.LoopList);
02857     for ( i=0; i<AP.LoopList.num; ++i ) {
02858         S_WRITE_B(DoLoops[i].p.buffer, DoLoops[i].p.size);
02859         if ( DoLoops[i].type == ONEEXPRESSION ) {
02860             /* do loops with an expression keep this expression in a dynamically
02861              * allocated buffer in dollarname */
02862             S_WRITE_S(DoLoops[i].dollarname);
02863         }
02864     }
02865
02866     S_WRITE_LIST(AP.ProcList);
02867     for ( i=0; i<AP.ProcList.num; ++i ) {
02868         if ( Procedures[i].p.size ) {
02869             if ( Procedures[i].loadmode != 1 ) {
02870                 S_WRITE_B(Procedures[i].p.buffer, Procedures[i].p.size);
02871             }
02872             else {
02873                 for ( j=0; j<AP.ProcList.num; ++j ) {
02874                     if ( Procedures[i].p.buffer == Procedures[j].p.buffer ) {
02875                         break;

```

```

02876         }
02877     }
02878     if ( j == AP.ProcList.num ) {
02879         MesPrint("Error writing procedures to recovery file!");
02880     }
02881     S_WRITE_B(&j, sizeof(int));
02882 }
02883 }
02884 S_WRITE_S(Procedures[i].name);
02885 }
02886
02887 S_WRITE_B(AP.PreSwitchStrings, (AP.NumPreSwitchStrings+1)*(LONG)sizeof(UBYTE*));
02888 for ( i=1; i<=AP.PreSwitchLevel; ++i ) {
02889     S_WRITE_S(AP.PreSwitchStrings[i]);
02890 }
02891
02892 S_WRITE_B(AP.preStart, AP.pSize);
02893
02894 S_WRITE_S(AP.procedureExtension);
02895 S_WRITE_S(AP.cprocedureExtension);
02896
02897 S_WRITE_B(AP.PreAssignStack, AP.MaxPreAssignLevel*(LONG)sizeof(LONG));
02898 S_WRITE_B(AP.PreIfStack, AP.MaxPreIfLevel*(LONG)sizeof(int));
02899 S_WRITE_B(AP.PreSwitchModes, (AP.NumPreSwitchStrings+1)*(LONG)sizeof(int));
02900 S_WRITE_B(AP.PreTypes, (AP.MaxPreTypes+1)*(LONG)sizeof(int));
02901
02902 /*#] AP :*/
02903 /*#[ AR :*/
02904
02905 ANNOUNCE(AR)
02906 /* to remember which entry in AR.Fscr corresponds to the infile */
02907 l = AR.infile - AR.Fscr;
02908 S_WRITE_B(&l, sizeof(LONG));
02909
02910 /* write the FILEHANDLES */
02911 S_WRITE_B(AR.outfile, sizeof(FILEHANDLE));
02912 l = AR.outfile->POfull - AR.outfile->PObuffer;
02913 if ( l ) {
02914     S_WRITE_B(AR.outfile->PObuffer, l*sizeof(WORD));
02915 }
02916 S_WRITE_B(AR.hidefile, sizeof(FILEHANDLE));
02917 l = AR.hidefile->POfull - AR.hidefile->PObuffer;
02918 if ( l ) {
02919     S_WRITE_B(AR.hidefile->PObuffer, l*sizeof(WORD));
02920 }
02921
02922 S_WRITE_B(&AR.StoreData.Fill, sizeof(POSITION));
02923 if ( ISNOTZEROPOS(AR.StoreData.Fill) ) {
02924     S_WRITE_B(&AR.StoreData, sizeof(FILEDATA));
02925 }
02926
02927 S_WRITE_B(&AR.DefPosition, sizeof(POSITION));
02928
02929 l = TimeCPU(1); l = -l;
02930 S_WRITE_B(&l, sizeof(LONG));
02931
02932 ANNOUNCE(AR.InInBuf)
02933 S_WRITE_B(&AR.InInBuf, sizeof(LONG));
02934 S_WRITE_B(&AR.InHiBuf, sizeof(LONG));
02935
02936 S_WRITE_B(&AR.NoCompress, sizeof(int));
02937 S_WRITE_B(&AR.zipCompress, sizeof(int));
02938
02939 S_WRITE_B(&AR.outtohide, sizeof(int));
02940
02941 S_WRITE_B(&AR.GetFile, sizeof(WORD));
02942 S_WRITE_B(&AR.KeptInHold, sizeof(WORD));
02943 S_WRITE_B(&AR.BraceOn, sizeof(WORD));
02944 S_WRITE_B(&AR.MaxBracket, sizeof(WORD));
02945 S_WRITE_B(&AR.CurDum, sizeof(WORD));
02946 S_WRITE_B(&AR.DeferFlag, sizeof(WORD));
02947 S_WRITE_B(&AR.TePos, sizeof(WORD));
02948 S_WRITE_B(&AR.sLevel, sizeof(WORD));
02949 S_WRITE_B(&AR.Stage4Name, sizeof(WORD));
02950 S_WRITE_B(&AR.GetOneFile, sizeof(WORD));
02951 S_WRITE_B(&AR.PolyFun, sizeof(WORD));
02952 S_WRITE_B(&AR.PolyFunInv, sizeof(WORD));
02953 S_WRITE_B(&AR.PolyFunType, sizeof(WORD));
02954 S_WRITE_B(&AR.PolyFunExp, sizeof(WORD));
02955 S_WRITE_B(&AR.PolyFunVar, sizeof(WORD));
02956 S_WRITE_B(&AR.PolyFunPow, sizeof(WORD));
02957 S_WRITE_B(&AR.Eside, sizeof(WORD));
02958 S_WRITE_B(&AR.MaxDum, sizeof(WORD));
02959 S_WRITE_B(&AR.level, sizeof(WORD));
02960 S_WRITE_B(&AR.expchanged, sizeof(WORD));
02961 S_WRITE_B(&AR.expflags, sizeof(WORD));
02962 S_WRITE_B(&AR.CurExpr, sizeof(WORD));

```

```

02963     S_WRITE_B(&AR.SortType, sizeof(WORD));
02964     S_WRITE_B(&AR.ShortSortCount, sizeof(WORD));
02965
02966 #ifdef WITHPTHREADS
02967     for ( j = 0; j < AM.totalnumberofthreads; j++ ) {
02968         S_WRITE_B(&(AB[j]->R.wranfnpair1), sizeof(int));
02969         S_WRITE_B(&(AB[j]->R.wranfnpair2), sizeof(int));
02970         S_WRITE_B(&(AB[j]->R.wranfcall), sizeof(int));
02971         S_WRITE_B(&(AB[j]->R.wranfseed), sizeof(ULONG));
02972         S_WRITE_B(&(AB[j]->R.wranfia), sizeof(ULONG *));
02973         if ( AB[j]->R.wranfia ) {
02974             S_WRITE_B(AB[j]->R.wranfia, sizeof(ULONG)*AB[j]->R.wranfnpair2);
02975         }
02976     }
02977 #else
02978     S_WRITE_B(&(AR.wranfnpair1), sizeof(int));
02979     S_WRITE_B(&(AR.wranfnpair2), sizeof(int));
02980     S_WRITE_B(&(AR.wranfcall), sizeof(int));
02981     S_WRITE_B(&(AR.wranfseed), sizeof(ULONG));
02982     S_WRITE_B(&(AR.wranfia), sizeof(ULONG *));
02983     if ( AR.wranfia ) {
02984         S_WRITE_B(AR.wranfia, sizeof(ULONG)*AR.wranfnpair2);
02985     }
02986 #endif
02987
02988     /*#] AR :*/
02989     /*#] AO :*/
02990     /*
02991     We copy all non-pointer variables.
02992     */
02993     ANNOUNCE(AO)
02994     l = sizeof(A.O) - ((UBYTE *)(&(A.O.NumInBrack)) - (UBYTE *)(&(A.O)));
02995     S_WRITE_B(&(A.O.NumInBrack), l);
02996     /*
02997     Now the variables in OptimizeResult
02998     */
02999     S_WRITE_B(&(A.O.OptimizeResult), sizeof(OPTIMIZERESULT));
03000     if ( A.O.OptimizeResult.codesize > 0 ) {
03001         S_WRITE_B(A.O.OptimizeResult.code, A.O.OptimizeResult.codesize*sizeof(WORD));
03002     }
03003     S_WRITE_S(A.O.OptimizeResult.nameofexpr);
03004     /*
03005     And now the dictionaries.
03006     We write each dictionary to a buffer and get the size of that buffer.
03007     Then we write the size and the buffer.
03008     */
03009     for ( i = 0; i < AO.NumDictionaries; i++ ) {
03010         l = DictToBytes(AO.Dictionaries[i], (UBYTE *) (AT.WorkPointer));
03011         S_WRITE_B(&l, sizeof(LONG));
03012         S_WRITE_B(AT.WorkPointer, l);
03013     }
03014
03015     /*#] AO :*/
03016     /*#] PF :*/
03017 #ifdef WITHMPI
03018     S_WRITE_B(&PF.numtasks, sizeof(int));
03019     S_WRITE_B(&PF.rhsInParallel, sizeof(int));
03020     S_WRITE_B(&PF.exprbufsize, sizeof(int));
03021     S_WRITE_B(&PF.log, sizeof(int));
03022 #endif
03023     /*#] PF :*/
03024
03025 #ifdef WITHPTHREADS
03026
03027     ANNOUNCE(GetTimerInfo)
03028     /*
03029     write timing information of individual threads
03030     */
03031     i = GetTimerInfo(&longp, &longpp);
03032     S_WRITE_B(&i, sizeof(int));
03033     S_WRITE_B(longp, i*(LONG)sizeof(LONG));
03034     S_WRITE_B(&i, sizeof(int));
03035     S_WRITE_B(longpp, i*(LONG)sizeof(LONG));
03036
03037 #endif
03038
03039     S_FLUSH_B /* because we will call fwrite() directly in the following code */
03040
03041     /* save length of data at the beginning of the file */
03042     ANNOUNCE(file length)
03043     SETBASEPOSITION(pos, (ftell(fd)));
03044     fseek(fd, 0, SEEK_SET);
03045     if ( fwrite(&pos, 1, sizeof(POSITION), fd) != sizeof(POSITION) ) return(__LINE__);
03046     fseek(fd, BASEPOSITION(pos), SEEK_SET);
03047
03048     ANNOUNCE(file close)
03049     if ( fclose(fd) ) return(__LINE__);

```

```

03050 #ifdef WITHMPI
03051     if ( PF.me == MASTER ) {
03052 #endif
03053 /*
03054     copy store file if necessary
03055 */
03056     ANNOUNCE(copy store file)
03057     if ( ISNOTZEROPOS(AR.StoreData.Fill) ) {
03058         if ( CopyFile(FG.fname, storefile) ) return(__LINE__);
03059     }
03060 /*
03061     copy sort file if necessary
03062 */
03063     ANNOUNCE(copy sort file)
03064     if ( AR.outfile->handle >= 0 ) {
03065         if ( CopyFile(AR.outfile->name, sortfile) ) return(__LINE__);
03066     }
03067 /*
03068     copy hide file if necessary
03069 */
03070     ANNOUNCE(copy hide file)
03071     if ( AR.hidefile->handle >= 0 ) {
03072         if ( CopyFile(AR.hidefile->name, hidefile) ) return(__LINE__);
03073     }
03074 #ifdef WITHMPI
03075     }
03076 /*
03077     * For ParFORM, the renaming will be performed after the master got
03078     * all recovery files from the slaves.
03079 */
03080 #else
03081 /*
03082     make the intermediate file the recovery file
03083 */
03084     ANNOUNCE(rename intermediate file)
03085     if ( rename(intermedfile, recoveryfile) ) return(__LINE__);
03086
03087     done_snapshot = 1;
03088
03089     MesPrint("done."); fflush(0);
03090 #endif
03091
03092 #ifdef PRINTDEBUG
03093     print_M();
03094     print_C();
03095     print_P();
03096     print_R();
03097 #endif
03098
03099     return(0);
03100 }
03101
03102 /*
03103 #] DoSnapshot :
03104 #[ DoCheckpoint :
03105 */
03106
03111 void DoCheckpoint(int moduletype)
03112 {
03113     int error;
03114     LONG timestamp = TimeWallClock(1);
03115 #ifdef WITHMPI
03116     if (PF.me == MASTER){
03117 #endif
03118         if ( timestamp - AC.CheckpointStamp >= AC.CheckpointInterval ) {
03119             char argbuf[20];
03120             int retvalue = 0;
03121             if ( AC.CheckpointRunBefore ) {
03122                 size_t l1, l2;
03123                 char *str;
03124                 l1 = strlen(AC.CheckpointRunBefore);
03125                 NumToStr((UBYTE*)argbuf, AC.CModule);
03126                 l2 = strlen(argbuf);
03127                 str = (char*)Malloc(1+l1+l2+2, "callbefore");
03128                 strcpy(str, AC.CheckpointRunBefore);
03129                 *(str+l1) = ' ';
03130                 strcpy(str+l1+1, argbuf);
03131                 retvalue = system(str);
03132                 M_free(str, "callbefore");
03133                 if ( retvalue ) {
03134                     MesPrint("Script returned error -> no recovery file will be created.");
03135                 }
03136             }
03137 #ifdef WITHMPI
03138             /* Confirm slaves to make snapshots. */
03139             PF_BroadcastNumber(retvalue == 0);
03140 #endif
03141         }
03142 #endif

```

```

03141         if ( retvalue == 0 ) {
03142             if ( (error = DoSnapshot(moduletype)) ) {
03143                 MesPrint("Error creating recovery files: %d", error);
03144             }
03145 #ifdef WITHMPI
03146         {
03147             int i;
03148             /*get recovery files from slaves:*/
03149             for(i=1; i<PF.numtasks;i++){
03150                 FILE *fd;
03151                 const char *tmpnam = PF_recoveryfile('m', i, 1);
03152                 fd = fopen(tmpnam, "w");
03153                 if(fd == NULL){
03154                     MesPrint("Error opening recovery file for slave %d",i);
03155                     Terminate(-1);
03156                 }/*if(fd == NULL)*/
03157                 retvalue=PF_RecvFile(i,fd);
03158                 if(retvalue<=0){
03159                     MesPrint("Error receiving recovery file from slave %d",i);
03160                     Terminate(-1);
03161                 }/*if(retvalue<=0)*/
03162                 fclose(fd);
03163             }/*for(i=0; i<PF.numtasks;i++)*/
03164             /*
03165              * Make the intermediate files the recovery files.
03166              */
03167             ANNOUNCE(rename intermediate file)
03168             for ( i = 0; i < PF.numtasks; i++ ) {
03169                 const char *src = PF_recoveryfile('m', i, 1);
03170                 const char *dst = PF_recoveryfile('m', i, 0);
03171                 if ( rename(src, dst) ) {
03172                     MesPrint("Error renaming recovery file %s -> %s", src, dst);
03173                 }
03174             }
03175             done_snapshot = 1;
03176             MesPrint("done."); fflush(0);
03177         }
03178 #endif
03179     }
03180     if ( AC.CheckpointRunAfter ) {
03181         size_t l1, l2;
03182         char *str;
03183         l1 = strlen(AC.CheckpointRunAfter);
03184         NumToStr((UBYTE*)argbuf, AC.CModule);
03185         l2 = strlen(argbuf);
03186         str = (char*)Malloc1(l1+l2+2, "callafter");
03187         strcpy(str, AC.CheckpointRunAfter);
03188         *(str+l1) = ' ';
03189         strcpy(str+l1+1, argbuf);
03190         retvalue = system(str);
03191         M_free(str, "callafter");
03192         if ( retvalue ) {
03193             MesPrint("Error calling script after recovery.");
03194         }
03195     }
03196     AC.CheckpointStamp = TimeWallClock(1);
03197 }
03198 #ifdef WITHMPI
03199 else/* timestamp - AC.CheckpointStamp < AC.CheckpointInterval*/
03200 /* The slaves don't need to make snapshots. */
03201 PF_BroadcastNumber(0);
03202 }
03203 }/*if(PF.me == MASTER)*/
03204 else{/*Slave*/
03205     int i;
03206     /* Check if the slave needs to make a snapshot. */
03207     if ( PF_BroadcastNumber(0) ) {
03208         error = DoSnapshot(moduletype);
03209         if(error == 0){
03210             FILE *fd;
03211             /*
03212              * Send the recovery file to the master. Note that no renaming
03213              * has been performed and what we have to send is actually sitting
03214              * in the intermediate file.
03215              */
03216             fd = fopen(intermedfile, "r");
03217             i=PF_SendFile(MASTER, fd);/*if fd=NULL, PF_SendFile seds to a slave the failure tag*/
03218             if(fd == NULL)
03219                 Terminate(-1);
03220             fclose(fd);
03221             if(i<=0)
03222                 Terminate(-1);
03223             /*Now the slave need not the recovery file so remove it:*/
03224             remove(intermedfile);
03225         }
03226         else{
03227             /*send the error tag to the master:*/

```

```

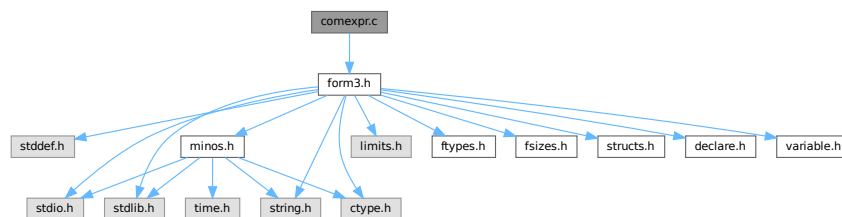
03228         PF_SendFile(MASTER, NULL); /*if fd==NULL, PF_SendFile seds to a slave the failure tag*/
03229         Terminate(-1);
03230     }
03231     done_snapshot = 1;
03232     } /*if (tag=PF_DATA_MSGTAG)*/
03233     } /*if (PF.me != MASTER)*/
03234 #endif
03235 }
03236
03237 /*
03238     #] DoCheckpoint :
03239 */

```

4.7 comexpr.c File Reference

```
#include "form3.h"
```

Include dependency graph for comexpr.c:



Data Structures

- struct **id_options**

Functions

- int **CoLocal** (UBYTE *inp)
- int **CoGlobal** (UBYTE *inp)
- int **CoLocalFactorized** (UBYTE *inp)
- int **CoGlobalFactorized** (UBYTE *inp)
- int **DoExpr** (UBYTE *inp, int type, int par)
- int **ColdOld** (UBYTE *inp)
- int **Cold** (UBYTE *inp)
- int **ColdNew** (UBYTE *inp)
- int **CoDisorder** (UBYTE *inp)
- int **CoMany** (UBYTE *inp)
- int **CoMulti** (UBYTE *inp)
- int **ColfMatch** (UBYTE *inp)
- int **ColfNoMatch** (UBYTE *inp)
- int **CoOnce** (UBYTE *inp)
- int **CoOnly** (UBYTE *inp)
- int **CoSelect** (UBYTE *inp)
- int **ColdExpression** (UBYTE *inp, int type)
- int **CoMultiply** (UBYTE *inp)
- int **CoFill** (UBYTE *inp)
- int **CoFillExpression** (UBYTE *inp)
- int **CoPrintTable** (UBYTE *inp)
- int **CoAssign** (UBYTE *inp)
- int **CoDeallocateTable** (UBYTE *inp)

4.7.1 Detailed Description

Compiler routines for statements that involve algebraic expressions. These involve definitions, id-statements, the multiply statement and the fill statement.

Definition in file [comexpr.c](#).

4.7.2 Function Documentation

4.7.2.1 CoLocal()

```
int CoLocal (
    UBYTE * inp )
```

Definition at line 66 of file [comexpr.c](#).

4.7.2.2 CoGlobal()

```
int CoGlobal (
    UBYTE * inp )
```

Definition at line 73 of file [comexpr.c](#).

4.7.2.3 CoLocalFactorized()

```
int CoLocalFactorized (
    UBYTE * inp )
```

Definition at line 80 of file [comexpr.c](#).

4.7.2.4 CoGlobalFactorized()

```
int CoGlobalFactorized (
    UBYTE * inp )
```

Definition at line 87 of file [comexpr.c](#).

4.7.2.5 DoExpr()

```
int DoExpr (
    UBYTE * inp,
    int type,
    int par )
```

Definition at line 96 of file [comexpr.c](#).

4.7.2.6 ColdOld()

```
int ColdOld (
    UBYTE * inp )
```

Definition at line 384 of file [comexpr.c](#).

4.7.2.7 Cold()

```
int Cold (
    UBYTE * inp )
```

Definition at line 395 of file [comexpr.c](#).

4.7.2.8 ColdNew()

```
int ColdNew (
    UBYTE * inp )
```

Definition at line 406 of file [comexpr.c](#).

4.7.2.9 CoDisorder()

```
int CoDisorder (
    UBYTE * inp )
```

Definition at line 417 of file [comexpr.c](#).

4.7.2.10 CoMany()

```
int CoMany (
    UBYTE * inp )
```

Definition at line 428 of file [comexpr.c](#).

4.7.2.11 CoMulti()

```
int CoMulti (
    UBYTE * inp )
```

Definition at line 439 of file [comexpr.c](#).

4.7.2.12 ColfMatch()

```
int ColfMatch (
    UBYTE * inp )
```

Definition at line 450 of file [comexpr.c](#).

4.7.2.13 ColfNoMatch()

```
int ColfNoMatch (
    UBYTE * inp )
```

Definition at line 461 of file [comexpr.c](#).

4.7.2.14 CoOnce()

```
int CoOnce (
    UBYTE * inp )
```

Definition at line 472 of file [comexpr.c](#).

4.7.2.15 CoOnly()

```
int CoOnly (
    UBYTE * inp )
```

Definition at line 483 of file [comexpr.c](#).

4.7.2.16 CoSelect()

```
int CoSelect (
    UBYTE * inp )
```

Definition at line 494 of file [comexpr.c](#).

4.7.2.17 ColdExpression()

```
int ColdExpression (
    UBYTE * inp,
    int type )
```

Definition at line 507 of file [comexpr.c](#).

4.7.2.18 CoMultiply()

```
int CoMultiply (
    UBYTE * inp )
```

Definition at line 1031 of file [comexpr.c](#).

4.7.2.19 CoFill()

```
int CoFill (
    UBYTE * inp )
```

Definition at line 1070 of file [comexpr.c](#).

4.7.2.20 CoFillExpression()

```
int CoFillExpression (
    UBYTE * inp )
```

Definition at line 1356 of file [comexpr.c](#).

4.7.2.21 CoPrintTable()

```
int CoPrintTable (
    UBYTE * inp )
```

Definition at line 1748 of file [comexpr.c](#).

4.7.2.22 CoAssign()

```
int CoAssign (
    UBYTE * inp )
```

Definition at line 1924 of file [comexpr.c](#).

4.7.2.23 CoDeallocateTable()

```
int CoDeallocateTable (
    UBYTE * inp )
```

Definition at line 1982 of file [comexpr.c](#).

4.8 comexpr.c

[Go to the documentation of this file.](#)

```
00001
00008 /* # [ License : */
00009 /*
00010 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00011 * When using this file you are requested to refer to the publication
00012 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00013 * This is considered a matter of courtesy as the development was paid
00014 * for by FOM the Dutch physics granting agency and we would like to
00015 * be able to track its scientific use to convince FOM of its value
00016 * for the community.
00017 *
00018 * This file is part of FORM.
00019 *
00020 * FORM is free software: you can redistribute it and/or modify it under the
00021 * terms of the GNU General Public License as published by the Free Software
00022 * Foundation, either version 3 of the License, or (at your option) any later
00023 * version.
00024 *
00025 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00026 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00027 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00028 * details.
00029 *
00030 * You should have received a copy of the GNU General Public License along
00031 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00032 */
00033 /* # ] License : */
00034
00035 /*
```

```

00036     # [ Includes : compi2.c
00037
00038     File contains most of what has to do with compiling expressions.
00039     Main supporting file: token.c
00040 */
00041
00042 #include "form3.h"
00043
00044 static struct id_options {
00045     UBYTE *name;
00046     int code;
00047     int dummy;
00048 } IdOptions[] = {
00049     { (UBYTE *) "multi",      SUBMULTI      , 0 }
00050     , { (UBYTE *) "many",      SUBMANY       , 0 }
00051     , { (UBYTE *) "only",      SUBONLY       , 0 }
00052     , { (UBYTE *) "once",      SUBONCE       , 0 }
00053     , { (UBYTE *) "ifmatch",   SUBAFTER      , 0 }
00054     , { (UBYTE *) "ifnomatch", SUBAFTERNOT  , 0 }
00055     , { (UBYTE *) "ifnotmatch", SUBAFTERNOT  , 0 }
00056     , { (UBYTE *) "disorder",  SUBDISORDER , 0 }
00057     , { (UBYTE *) "select",    SUBSELECT    , 0 }
00058     , { (UBYTE *) "all",       SUBALL       , 0 }
00059 };
00060
00061 /*
00062 #] Includes :
00063 # [ CoLocal :
00064 */
00065
00066 int CoLocal(UBYTE *inp) { return(DoExpr(inp, LOCALEXPRESSION, 0)); }
00067
00068 /*
00069 #] CoLocal :
00070 # [ CoGlobal :
00071 */
00072
00073 int CoGlobal(UBYTE *inp) { return(DoExpr(inp, GLOBALEXPRESSION, 0)); }
00074
00075 /*
00076 #] CoGlobal :
00077 # [ CoLocalFactorized :
00078 */
00079
00080 int CoLocalFactorized(UBYTE *inp) { return(DoExpr(inp, LOCALEXPRESSION, 1)); }
00081
00082 /*
00083 #] CoLocalFactorized :
00084 # [ CoGlobalFactorized :
00085 */
00086
00087 int CoGlobalFactorized(UBYTE *inp) { return(DoExpr(inp, GLOBALEXPRESSION, 1)); }
00088
00089 /*
00090 #] CoGlobalFactorized :
00091 # [ DoExpr:
00092
00093
00094 */
00095
00096 int DoExpr(UBYTE *inp, int type, int par)
00097 {
00098     GETIDENTITY
00099     int error = 0;
00100     UBYTE *p, *q, c;
00101     WORD *w, i, j = 0, c1, c2, *OldWork = AT.WorkPointer, osize;
00102     WORD jold = 0;
00103     POSITION pos;
00104     while ( *inp == ',' ) inp++;
00105     if ( par ) AC.ToBeInFactors = 1;
00106     else AC.ToBeInFactors = 0;
00107     p = inp;
00108     while ( *p && *p != '=' ) {
00109         if ( *p == '(' ) SKIPBRA4(p)
00110         else if ( *p == '{' ) SKIPBRA5(p)
00111         else if ( *p == '[' ) SKIPBRA1(p)
00112         else p++;
00113     }
00114     if ( *p ) { /* Variety with the = sign */
00115         if ( ( q = SkipAName(inp) ) == 0 || q[-1] == '_' ) {
00116             MesPrint("%Illegal name for expression");
00117             error = 1;
00118             if ( q[-1] == '_' ) {
00119                 while ( FG.cTable[*q] < 2 || *q == '_' ) q++;
00120             }
00121         }
00122         else {

```

```

00123     c = *q; *q = 0;
00124     if ( GetVar(inp,&c1,&c2,ALLVARIABLES,NOAUTO) != NAMENOTFOUND ) {
00125         if ( c1 == CEXPRESSION ) {
00126             if ( Expressions[c2].status == STOREDEXPRESSION ) {
00127                 MesPrint("&Illegal attempt to overwrite a stored expression");
00128                 error = 1;
00129             }
00130             else {
00131                 HighWarning("Expression is replaced by new definition");
00132                 if ( AO.OptimizeResult.nameofexpr != NULL &&
00133                     StrCmp(inp,AO.OptimizeResult.nameofexpr) == 0 ) {
00134                     ClearOptimize();
00135                 }
00136                 if ( Expressions[c2].status != DROPEDEXPRESSION ) {
00137                     w = &(Expressions[c2].status);
00138                     if ( *w == LOCALEXPRESSION || *w == SKIPLEXPRESSION )
00139                         *w = DROPEXPRESSION;
00140                     else if ( *w == GLOBALEXPRESSION || *w == SKIPGEXPRESSION )
00141                         *w = DROPGEXPRESSION;
00142                     else if ( *w == HIDDENLEXPRESSION )
00143                         *w = DROPHLEXPRESSION;
00144                     else if ( *w == HIDDENGEXPRESSION )
00145                         *w = DROPHGEXPRESSION;
00146                 }
00147                 AC.TransEname = Expressions[c2].name;
00148                 j = EntVar(CEXPRESSION,0,type,0,0,0);
00149                 Expressions[j].node = Expressions[c2].node;
00150                 Expressions[c2].replace = j;
00151             }
00152         }
00153         else {
00154             MesPrint("&name of expression is also name of a variable");
00155             error = 1;
00156             j = EntVar(CEXPRESSION,inp,type,0,0,0);
00157         }
00158         jold = c2;
00159     }
00160     else {
00161         /*
00162             Here we have to worry about reuse of the expression in the
00163             same module. That will need AS.Oldvflags but that may not
00164             be defined or have the proper value.
00165         */
00166         j = EntVar(CEXPRESSION,inp,type,0,0,0);
00167         jold = j;
00168     }
00169     *q = c;
00170     OldWork = w = AT.WorkPointer;
00171     *w++ = TYPEEXPRESSION;
00172     *w++ = 3+SUBEXPSIZE;
00173     *w++ = j;
00174     AC.ProtoType = w;
00175     AR.CurExpr = j; /* Block expression j */
00176     *w++ = SUBEXPRESSION;
00177     *w++ = SUBEXPSIZE;
00178     *w++ = j;
00179     *w++ = 1;
00180     *w++ = AC.cbunum;
00181     FILLSUB(w)
00182
00183     if ( c == '(' ) {
00184         while ( *q == ',' || *q == '(' ) {
00185             inp = q+1;
00186             if ( ( q = SkipAName(inp) ) == 0 ) {
00187                 MesPrint("&Illegal name for expression argument");
00188                 error = 1;
00189                 q = p - 1;
00190                 break;
00191             }
00192             c = *q; *q = 0;
00193             if ( GetVar(inp,&c1,&c2,ALLVARIABLES,WITHAUTO) < 0 ) c1 = -1;
00194             switch ( c1 ) {
00195                 case CSYMBOL :
00196                     *w++ = SYMTOSYM; *w++ = 4; *w++ = c2; *w++ = 0;
00197                     break;
00198                 case CINDEXT :
00199                     *w++ = INDTOIND; *w++ = 4;
00200                     *w++ = c2 + AM.OffsetIndex; *w++ = 0;
00201                     break;
00202                 case CVECTOR :
00203                     *w++ = VECTOVEC; *w++ = 4;
00204                     *w++ = c2 + AM.OffsetVector; *w++ = 0;
00205                     break;
00206                 case CFUNCTION :
00207                     *w++ = FUNTOFUN; *w++ = 4; *w++ = c2 + FUNCTION; *w++ = 0;
00208                     break;
00209                 default :

```

```

00210             MesPrint("&Illegal expression parameter: %s",inp);
00211             error = 1;
00212             break;
00213         }
00214         *q = c;
00215     }
00216     if ( *q != ')' || q+1 != p ) {
00217         MesPrint("&Illegal use of arguments for expression");
00218         error = 1;
00219     }
00220     AC.ProtoType[1] = w - AC.ProtoType;
00221 }
00222 else if ( c != '=' ) {
00223 /*
00224     The dummy accepted L F := RHS;
00225 */
00226     MesPrint("&Illegal LHS for expression definition");
00227     error = 1;
00228 }
00229 *w++ = 1;
00230 *w++ = 1;
00231 *w++ = 3;
00232 *w++ = 0;
00233 SeekScratch(AR.outfile,&pos);
00234 Expressions[j].counter = 1;
00235 Expressions[j].onfile = pos;
00236 Expressions[j].whichbuffer = 0;
00237 #ifdef PARALLELCODE
00238     Expressions[j].partodo = AC.inparallelfalg;
00239 #endif
00240 OldWork[2] = w - OldWork - 3;
00241 AT.WorkPointer = w;
00242 /*
00243     Writing the expression prototype to disk and to the compiler
00244     buffer is done only after the RHS has been compiled because
00245     we don't know the number of the main level RHS yet.
00246 */
00247 }
00248 inp = p+1;
00249 ClearWildcardNames();
00250 osize = AC.ProtoType[1]; AC.ProtoType[1] = SUBEXPSIZE;
00251 PutInVflags(jold);
00252 if ( ( i = CompileAlgebra(inp,RHSIDE,AC.ProtoType) ) < 0 ) {
00253     AC.ProtoType[1] = osize;
00254     error = 1;
00255 }
00256 else if ( error == 0 ) {
00257     AC.ProtoType[1] = osize;
00258     AC.ProtoType[2] = i;
00259     if ( PutOut(BHEAD OldWork+2,&pos,AR.outfile,0) < 0 ) {
00260         MesPrint("&Cannot create expression");
00261         error = -1;
00262     }
00263     else {
00264         Expressions[j].sizeprototype = OldWork[2];
00265         OldWork[2] = 4+SUBEXPSIZE;
00266         OldWork[4] = SUBEXPSIZE;
00267         OldWork[5] = i;
00268         OldWork[SUBEXPSIZE+3] = 1;
00269         OldWork[SUBEXPSIZE+4] = 1;
00270         OldWork[SUBEXPSIZE+5] = 3;
00271         OldWork[SUBEXPSIZE+6] = 0;
00272         if ( PutOut(BHEAD OldWork+2,&pos,AR.outfile,0) < 0 ) {
00273             || FlushOut(&pos,AR.outfile,0) ) {
00274                 MesPrint("&Cannot create expression");
00275                 error = -1;
00276             }
00277             AR.outfile->POfull = AR.outfile->POfill;
00278         }
00279         OldWork[2] = j;
00280 /*
00281     Seems unnecessary (13-feb-2018)
00282
00283     AddNtoL(OldWork[1],OldWork);
00284 */
00285     AT.WorkPointer = OldWork;
00286     if ( AC.dumnumflag ) Add2Com(TYPEDETCURDUM)
00287 }
00288 AC.ToBeInFactors = 0;
00289 }
00290 else { /* Variety in which expressions change property */
00291 /*
00292     This code got a major revision because it didn't
00293     take hidden expressions into account. (1-jun-2010 JV)
00294 */
00295     do {
00296         if ( ( q = SkipAName(inp) ) == 0 ) {

```

```

00297         MesPrint("&Illegal name(s) for expression(s)");
00298         return(1);
00299     }
00300     c = *q; *q = 0;
00301     if ( GetName(AC.exprnames,inp,&c2,NOAUTO) == NAMENOTFOUND ) {
00302         MesPrint("&%s is not a valid expression",inp);
00303         error = 1;
00304     }
00305     else {
00306         w = &(Expressions[c2].status);
00307         if ( type == LOCALEXPRESSION ) {
00308             switch ( *w ) {
00309                 case GLOBALEXPRESSION:
00310                     *w = LOCALEXPRESSION;
00311                     break;
00312                 case SKIPGEXPRESSION:
00313                     *w = SKIPLEXPRESSION;
00314                     break;
00315                 case DROPGEXPRESSION:
00316                     *w = DROPLEXPRESSION;
00317                     break;
00318                 case HIDDENGEXPRESSION:
00319                     *w = HIDDENLEXPRESSION;
00320                     break;
00321                 case HIDEGEXPRESSION:
00322                     *w = HIDELEXPRESSION;
00323                     break;
00324                 case UNHIDEGEXPRESSION:
00325                     *w = UNHIDELEXPRESSION;
00326                     break;
00327                 case INTOHIDEGEXPRESSION:
00328                     *w = INTOHIDELEXPRESSION;
00329                     break;
00330                 case DROPHGEXPRESSION:
00331                     *w = DROPHLEXPRESSION;
00332                     break;
00333             }
00334         }
00335         else if ( type == GLOBALEXPRESSION ) {
00336             switch ( *w ) {
00337                 case LOCALEXPRESSION:
00338                     *w = GLOBALEXPRESSION;
00339                     break;
00340                 case SKIPLEXPRESSION:
00341                     *w = SKIPGEXPRESSION;
00342                     break;
00343                 case DROPLEXPRESSION:
00344                     *w = DROPGEXPRESSION;
00345                     break;
00346                 case HIDDENLEXPRESSION:
00347                     *w = HIDDENGEXPRESSION;
00348                     break;
00349                 case HIDELEXPRESSION:
00350                     *w = HIDEGEXPRESSION;
00351                     break;
00352                 case UNHIDELEXPRESSION:
00353                     *w = UNHIDEGEXPRESSION;
00354                     break;
00355                 case INTOHIDELEXPRESSION:
00356                     *w = INTOHIDEGEXPRESSION;
00357                     break;
00358                 case DROPHLEXPRESSION:
00359                     *w = DROPHGEXPRESSION;
00360                     break;
00361             }
00362         }
00363         /*
00364             old code
00365             if ( type != LOCALEXPRESSION || *w != STOREDEXPRESSION )
00366                 *w = type;
00367         */
00368     }
00369     *q = c; inp = q+1;
00370 } while ( c == ',' );
00371 if ( c ) {
00372     MesPrint("&Illegal object in local or global redefinition");
00373     error = 1;
00374 }
00375 }
00376 return(error);
00377 }
00378
00379 /*
00380 #] DoExpr:
00381 #[ CoIdOld :
00382 */
00383

```

```
00384 int CoIdOld(UBYTE *inp)
00385 {
00386     AC.idoption = 0;
00387     return(CoIdExpression(inp, TYPEIDOLD));
00388 }
00389
00390 /*
00391     #] CoIdOld :
00392     #[ CoId :
00393 */
00394
00395 int CoId(UBYTE *inp)
00396 {
00397     AC.idoption = 0;
00398     return(CoIdExpression(inp, TYPEIDNEW));
00399 }
00400
00401 /*
00402     #] CoId :
00403     #[ CoIdNew :
00404 */
00405
00406 int CoIdNew(UBYTE *inp)
00407 {
00408     AC.idoption = 0;
00409     return(CoIdExpression(inp, TYPEIDNEW));
00410 }
00411
00412 /*
00413     #] CoIdNew :
00414     #[ CoDisorder :
00415 */
00416
00417 int CoDisorder(UBYTE *inp)
00418 {
00419     AC.idoption = SUBDISORDER;
00420     return(CoIdExpression(inp, TYPEIDNEW));
00421 }
00422
00423 /*
00424     #] CoDisorder :
00425     #[ CoMany :
00426 */
00427
00428 int CoMany(UBYTE *inp)
00429 {
00430     AC.idoption = SUBMANY;
00431     return(CoIdExpression(inp, TYPEIDNEW));
00432 }
00433
00434 /*
00435     #] CoMany :
00436     #[ CoMulti :
00437 */
00438
00439 int CoMulti(UBYTE *inp)
00440 {
00441     AC.idoption = SUBMULTI;
00442     return(CoIdExpression(inp, TYPEIDNEW));
00443 }
00444
00445 /*
00446     #] CoMulti :
00447     #[ CoIfMatch :
00448 */
00449
00450 int CoIfMatch(UBYTE *inp)
00451 {
00452     AC.idoption = SUBAFTER;
00453     return(CoIdExpression(inp, TYPEIDNEW));
00454 }
00455
00456 /*
00457     #] CoIfMatch :
00458     #[ CoIfNoMatch :
00459 */
00460
00461 int CoIfNoMatch(UBYTE *inp)
00462 {
00463     AC.idoption = SUBAFTERNOT;
00464     return(CoIdExpression(inp, TYPEIDNEW));
00465 }
00466
00467 /*
00468     #] CoIfNoMatch :
00469     #[ CoOnce :
00470 */
```



```

00471
00472 int CoOnce(UBYTE *inp)
00473 {
00474     AC.idoption = SUBONCE;
00475     return(CoIdExpression(inp,TYPEIDNEW));
00476 }
00477
00478 /*
00479     #] CoOnce :
00480     #[ CoOnly :
00481 */
00482
00483 int CoOnly(UBYTE *inp)
00484 {
00485     AC.idoption = SUBONLY;
00486     return(CoIdExpression(inp,TYPEIDNEW));
00487 }
00488
00489 /*
00490     #] CoOnly :
00491     #[ CoSelect :
00492 */
00493
00494 int CoSelect(UBYTE *inp)
00495 {
00496     AC.idoption = SUBSELECT;
00497     return(CoIdExpression(inp,TYPEIDNEW));
00498 }
00499
00500 /*
00501     #] CoSelect :
00502     #[ CoIdExpression :
00503
00504     First finish dealing with secondary keywords
00505 */
00506
00507 int CoIdExpression(UBYTE *inp, int type)
00508 {
00509     GETIDENTITY
00510     int i, j, idhead, error = 0, MinusSign = 0, opt, retcode;
00511     WORD *w, *s, *m, *mm, *ww, *FirstWork, *OldWork, cl, numsets = 0,
00512         oldnumrhs, *ow, oldEside;
00513     UBYTE *p, *pp, c;
00514     CBUF *C = cbuf+AC.cbufnum;
00515     LONG oldcpointer, x;
00516     FirstWork = OldWork = AT.WorkPointer;
00517 /*
00518     Don't forget to change in StudyPattern if we change/add_to the
00519     following setup.
00520     if ( type == TYPEIF ) idhead = IDHEAD-1;
00521     else
00522 */
00523     idhead = IDHEAD;
00524     AR.CurExpr = -1;
00525     w = AT.WorkPointer;
00526     *w++ = type;
00527     *w++ = idhead + SUBEXPSIZE;
00528     w++;
00529     if ( idhead >= IDHEAD ) *w++ = -1;
00530 #if IDHEAD > 4
00531     for ( i = 4; i < idhead; i++ ) *w++ = 0;
00532 #endif
00533     while ( *inp == ',' ) inp++;
00534     p = inp;
00535     if ( AC.idoption == SUBSELECT ) {
00536         p--;
00537         goto findsets;
00538     }
00539     else if ( ( AC.idoption == SUBAFTER ) || ( AC.idoption == SUBAFTERNOT ) ) {
00540         while ( *p && *p != '=' && *p != ',' ) {
00541             if ( *p == '(' ) SKIPBRA4(p)
00542             else if ( *p == '{' ) SKIPBRA5(p)
00543             else if ( *p == '[' ) SKIPBRA1(p)
00544             else p++;
00545         }
00546         if ( *p == '=' || *inp != '-' || inp[1] != '>' ) {
00547             MesPrint("&Illegal use if if[no]match in id statement");
00548             error = 1; goto AllDone;
00549         }
00550         if ( *p == 0 ) {
00551             MesPrint("&id-statement without = sign");
00552             error = 1; goto AllDone;
00553         }
00554         inp += 2; pp = inp;
00555         goto readlabel;
00556     }
00557     for(;;) {

```

```

00558     while ( *p && *p != '=' && *p != ',' ) {
00559         if ( *p == '(' ) SKIPBRA4(p)
00560         else if ( *p == '{' ) SKIPBRA5(p)
00561         else if ( *p == '[' ) SKIPBRA1(p)
00562         else p++;
00563     }
00564     if ( *p == '=' ) break;
00565     if ( *p == 0 ) {
00566         MesPrint("&id-statement without = sign");
00567         error = 1; goto AllDone;
00568     }
00569     /*
00570     We have either a secondary option or a syntax error
00571     */
00572     pp = inp;
00573     while ( FG.cTable[*pp] == 0 ) pp++;
00574     c = *pp; *pp = 0;
00575     i = sizeof(IdOptions)/sizeof(struct id_options);
00576     while ( --i >= 0 ) {
00577         if ( StrICmp(inp,IdOptions[i].name) == 0 ) break;
00578     }
00579     if ( i < 0 ) {
00580         MesPrint("&Illegal option %s in id-statement",inp);
00581         *pp = c; error = 1; p++; inp = p; continue;
00582     }
00583     opt = IdOptions[i].code;
00584     *pp = c;
00585     inp = pp+1;
00586     switch ( opt ) {
00587         case SUBDISORDER:
00588             if ( pp != p ) goto IllField;
00589             AC.idoption |= SUBDISORDER;
00590             p++; inp = p;
00591             break;
00592         case SUBSELECT:
00593             if ( p != pp ) goto IllField;
00594             if ( ( AC.idoption & SUBMASK ) != 0 ) {
00595                 if ( AC.idoption == SUBMULTI && type == TYPEIF ) {}
00596                 else {
00597                     MesPrint("&Conflicting options in id-statement");
00598                     error = 1;
00599                 }
00600             }
00601 findsets:;
00602     /*
00603     Now we read the sets
00604     */
00605     numsets = 0;
00606     for(;;) {
00607         inp = ++p;
00608         while ( *p && *p != '=' && *p != ',' ) {
00609             if ( *p == '(' ) SKIPBRA4(p)
00610             else if ( *p == '{' ) SKIPBRA5(p)
00611             else if ( *p == '[' ) SKIPBRA1(p)
00612             else p++;
00613         }
00614         if ( *p == '=' ) break;
00615         if ( *p == 0 ) {
00616             MesPrint("&id-statement without = sign");
00617             error = 1; goto AllDone;
00618         }
00619     /*
00620     We have a set at inp.
00621     */
00622         if ( *inp == '{' ) {
00623             if ( p[-1] != '}' ) {
00624                 c = *p; *p = 0;
00625                 MesPrint("&Illegal temporary set: %s",inp);
00626                 error = 1; *p = c;
00627             }
00628             else {
00629                 inp++;
00630                 c = p[-1]; p[-1] = 0;
00631                 c1 = DoTempSet(inp,p-1);
00632                 *w++ = c1;
00633                 p[-1] = c;
00634                 numsets++;
00635                 if ( w[-1] < 0 ) error = 1;
00636             }
00637         }
00638         else {
00639             c = *p; *p = 0;
00640             if ( GetName(AC.varnames,inp,&c1,NOAUTO) != CSET ) {
00641                 MesPrint("&%s is not a set",inp);
00642                 error = 1;
00643             }
00644             else {

```

```

00645             if ( c1 < AM.NumFixedSets ) {
00646                 MesPrint("&Built in sets are not allowed in the select option");
00647                 error = 1;
00648             }
00649             else if ( Sets[c1].type == CRANGE ) {
00650                 MesPrint("&Ranged sets are not allowed in the select option");
00651                 error = 1;
00652             }
00653             numsets++;
00654             *w++ = c1;
00655         }
00656         *p = c;
00657     }
00658 }
00659 /*
00660     Now exchange the positions a bit.
00661     Regular stuff at OldWork, numsets sets at FirstWork[idhead]
00662 */
00663 OldWork = w;
00664 for ( i = 0; i < idhead; i++ ) *w++ = FirstWork[i];
00665 AC.idoption = SUBSELECT;
00666 break;
00667 case SUBAFTER:
00668 case SUBAFTERNOT:
00669     if ( type == TYPEIF ) {
00670         MesPrint("&The if[no]match->label option is not allowed in an if statement");
00671         error = 1; goto AllDone;
00672     }
00673     if ( pp[0] != '-' || pp[1] != '>' ) goto IllField;
00674     pp += 2; /* points now at the label */
00675     inp = pp;
00676     AC.idoption |= opt;
00677 readlabel:
00678     while ( FG.cTable[*pp] <= 1 ) pp++;
00679     if ( pp != p ) {
00680         c = *p; *p = 0;
00681         MesPrint("&Illegal label %s in if[no]match option of id-statement",inp);
00682         *p = c; error = 1; inp = p+1; continue;
00683     }
00684     c = *p; *p = 0;
00685     OldWork[3] = GetLabel(inp);
00686     *p++ = c; inp = p;
00687     break;
00688 case SUBALL:
00689     x = 0;
00690     if ( *pp == '(' ) {
00691         if ( FG.cTable[*inp] == 1 ) {
00692             while ( *inp >= '0' && *inp <= '9' ) x = 10*x+*inp++ - '0';
00693         }
00694         else {
00695             pp++;
00696             while ( FG.cTable[*inp] == 0 ) inp++;
00697             c = *inp; *inp = 0;
00698             if ( StrICont(pp,(UBYTE *)"normalize") != 0 ) goto IllOpt;
00699             *inp = c;
00700             OldWork[4] |= NORMALIZEFLAG;
00701         }
00702         if ( *inp != ')' || inp+1 != p ) {
00703             c = *inp; *inp = 0;
00704 IllOpt:
00705             MesPrint("&Illegal ALL option in id-statement: ",pp);
00706             *inp++ = c;
00707             error = 1;
00708             continue;
00709         }
00710         pp = inp;
00711         inp = pp+1;
00712     }
00713 /*
00714     Note that the following statement limits x to
00715 */
00716     if ( x > MAXPOSITIVE ) {
00717         MesPrint("&Requested maximum number of matches %1 in ALL option in id-statement is
greater than %1 ",x,MAXPOSITIVE);
00718         error = 1;
00719     }
00720     OldWork[5] = x;
00721     if ( type != TYPEIDNEW ) {
00722         if ( type == TYPEIDOLD ) {
00723             MesPrint("&Requested ALL option not allowed in idold/also statement.");
00724             error = 1;
00725         }
00726         else if ( type == TYPEIF ) {
00727             MesPrint("&Requested ALL option not allowed in if(match())");
00728             error = 1;
00729         }
00730         else {

```

```

00731             MesPrint("&ALL option only allowed in regular id-statement.");
00732             error = 1;
00733         }
00734     }
00735     p++; inp = p;
00736     AC.idoption = opt;
00737     break;
00738     default:
00739         if ( pp != p ) {
00740     IllField:
00741             c = *p; *p = 0;
00742             MesPrint("&Illegal optionfield %s in id-statement",inp);
00743             *p = c; error = 1; inp = p+1; continue;
00744         }
00745         i = AC.idoption & SUBMASK;
00746         if ( i && i != opt ) {
00747             MesPrint("&Conflicting options in id-statement");
00748             error = 1; continue;
00749         }
00750         else AC.idoption |= opt;
00751         while ( *p == ',' ) p++;
00752         inp = p;
00753         break;
00754     }
00755     if ( ( AC.idoption & SUBMASK ) == 0 ) AC.idoption |= SUBMULTI;
00756     OldWork[2] = AC.idoption;
00757     /*
00758     Now we have a field till the = sign
00759     Now the subexpression prototype
00760     */
00761     AC.ProtoType = w;
00762     *w++ = SUBEXPRESSION;
00763     *w++ = SUBEXPSIZE;
00764     *w++ = C->numrhs+1;
00765     *w++ = 1;
00766     *w++ = AC.cbufnum;
00767     FILLSUB(w)
00768     AC.WildC = w;
00769     AC.NwildC = 0;
00770     AT.WorkPointer = s = w + 4*AM.MaxWildcards + 8;
00771     /*
00772     Now read the LHS
00773     */
00774     ClearWildcardNames();
00775     oldcpointer = AddLHS(AC.cbufnum) - C->Buffer;
00776
00777     *p = 0;
00778     oldnumrhs = C->numrhs;
00779     if ( ( retcode = CompileAlgebra(inp,LHSIDE,AC.ProtoType) ) < 0 ) { error = 1; }
00780     else AC.ProtoType[2] = retcode;
00781     *p = '='; inp = p+1;
00782     AT.WorkPointer = s;
00783     if ( AC.NwildC && SortWild(w,AC.NwildC) ) error = 1;
00784
00785     /* Make the LHS pointers ready */
00786
00787     OldWork[1] = AC.WildC-OldWork;
00788     OldWork[idhead+1] = OldWork[1] - idhead;
00789     w = AC.WildC;
00790     AT.WorkPointer = w;
00791     s = C->rhs[C->numrhs];
00792     /*
00793     Now check whether wildcards get converted to dollars (for PARALLEL)
00794     */
00795     {
00796         WORD *tw, *twstop;
00797         tw = AC.ProtoType; twstop = tw + tw[1]; tw += SUBEXPSIZE;
00798         while ( tw < twstop ) {
00799             if ( *tw == LOADDOLLAR ) {
00800                 AddPotModdollar(tw[2]);
00801             }
00802             tw += tw[1];
00803         }
00804     }
00805     /*
00806     We have the expression in the compiler buffers.
00807     The main level is at lhs[numlhs]
00808     The partial lhs (including ProtoType) is in OldWork (in Workspace)
00809     We need to load the result at w after the prototype
00810     Because these sort routines don't use the Workspace
00811     there should not be a conflict
00812     */
00813     if ( !error && *s == 0 ) {
00814     IllLeft:MesPrint("&Illegal LHS");
00815         AC.lhdollarflag = 0;
00816         return(1);
00817     }

```

```

00818     if ( !error && *(s+s) != 0 ) {
00819         MesPrint("&LHS should be one term only");
00820         return(1);
00821     }
00822     if ( error == 0 ) {
00823         WORD oldpolyfun = AR.PolyFun;
00824         if ( NewSort(BHEAD0) || NewSort(BHEAD0) ) {
00825             if ( !error ) error = 1;
00826             return(error);
00827         }
00828         AN.RepPoint = AT.RepCount + 1;
00829         ow = (WORD *) ((UBYTE *) (AT.WorkPointer)) + AM.MaxTer);
00830         mm = s; ww = ow; i = *mm;
00831         while ( --i >= 0 ) { *ww++ = *mm++; } AT.WorkPointer = ww;
00832         AC.lhdollarflag = 0; oldEside = AR.Eside; AR.Eside = LHSIDE;
00833         AR.Cnumlhs = C->numlhs;
00834         AR.PolyFun = 0;
00835         if ( Generator(BHEAD ow,C->numlhs) ) {
00836             AR.Eside = oldEside;
00837             LowerSortLevel(); LowerSortLevel(); AR.PolyFun = oldpolyfun; goto IllLeft;
00838         }
00839         AR.Eside = oldEside;
00840         AT.WorkPointer = w;
00841         if ( EndSort(BHEAD w,0) < 0 ) { LowerSortLevel(); AR.PolyFun = oldpolyfun; goto IllLeft; }
00842         AR.PolyFun = oldpolyfun;
00843         if ( *w == 0 || *(w+w) != 0 ) {
00844             MesPrint("&LHS must be one term");
00845             AC.lhdollarflag = 0;
00846             return(1);
00847         }
00848         LowerSortLevel();
00849         if ( AC.lhdollarflag ) MarkDirty(w,DIRTYFLAG);
00850     }
00851     AT.WorkPointer = w + *w;
00852     AC.DumNum = 0;
00853 /*
00854     Everything is now after OldWork. We can pop the compilerbuffer.
00855     Next test for illegal things like a coefficient
00856     At this point we have:
00857     w = the term of the LHS
00858 */
00859     C->Pointer = C->Buffer + oldcpointer;
00860     C->numrhs = oldnumrhs;
00861     C->numlhs--;
00862
00863     m = w + *w - 3;
00864     AC.vectorlikeLHS = 0;
00865     if ( !error ) {
00866         if ( m[2] != 3 || m[1] != 1 || *m != 1 ) {
00867             if ( *m == 1 && m[1] == 1 && m[2] == -3 ) {
00868                 MinusSign = 1;
00869             }
00870             else {
00871                 MesPrint("&Coefficient in LHS");
00872                 error = 1;
00873                 AC.DumNum = 0;
00874                 *w -= ABS(m[2])-3;
00875             }
00876         }
00877         if ( *w == 7 && w[1] == INDEX && w[3] < 0 ) {
00878             if ( ( AC.idoption & SUBMASK ) != 0 && ( AC.idoption & SUBMASK ) !=
00879                 SUBMULTI ) {
00880                 MesPrint("&Illegal option for substitution of a vector");
00881                 error = 1;
00882             }
00883             AC.DumNum = AM.IndDum;
00884             OldWork[2] = ( OldWork[2] - ( OldWork[2] & SUBMASK ) ) | SUBVECTOR;
00885             c1 = w[3];
00886             /* We overwrite the LHS */
00887             *w++ = INDTOIND;
00888             *w++ = 4;
00889             *w++ = AC.DumNum + WILDOffset;
00890             *w++ = 0;
00891             w[0] = 5;
00892             w[1] = VECTOR;
00893             w[2] = 4;
00894             w[3] = c1;
00895             w[4] = AC.DumNum + WILDOffset;
00896             OldWork[idhead+1] = w - OldWork - idhead;
00897             AC.vectorlikeLHS = 1;
00898         }
00899         else {
00900             AC.DumNum = 0;
00901             *w -= 3;
00902             i = OldWork[2] & SUBMASK;
00903             m = w + *w;
00904             if ( i == 0 || i == SUBMULTI ) {

```

```

00905         s = w+1;
00906         while ( s < m ) {
00907             if ( *s == SYMBOL ) {
00908                 j = s[1]/2; s += 2;
00909                 while ( --j >= 0 ) {
00910                     if ( ABS(s[1]) > 2*MAXPOWER ) {
00911                         OldWork[2] = ( OldWork[2] - i ) | SUBONCE;
00912                         break;
00913                     }
00914                     s += 2;
00915                 }
00916                 if ( j >= 0 ) break;
00917             }
00918             else if ( *s == DOTPRODUCT ) {
00919                 j = s[1]/3; s += 2;
00920                 while ( --j >= 0 ) {
00921                     if ( ABS(s[2]) > 2*MAXPOWER ) {
00922                         OldWork[2] = ( OldWork[2] - i ) | SUBONCE;
00923                         break;
00924                     }
00925                     else if ( s[1] >= -(2*WILDOFFSET) || s[0] >= -(2*WILDOFFSET) ) {
00926                         OldWork[2] = ( OldWork[2] - i ) | SUBMANY;
00927                         i = SUBMANY;
00928                     }
00929                     s += 3;
00930                 }
00931                 if ( j >= 0 ) break;
00932             }
00933             else {
00934                 OldWork[2] = ( OldWork[2] - i ) | SUBMANY;
00935                 break;
00936             }
00937         }
00938     }
00939     if ( ( OldWork[2] & SUBMASK ) == 0 ) OldWork[2] |= SUBMULTI;
00940 }
00941 if ( ( OldWork[2] & SUBMASK ) == SUBSELECT ) {
00942     /*
00943         Paste the SETSET information after the pattern.
00944         Important note: We will still get function information for the
00945         smart patternmatching after it. To distinguish them we need to have
00946         that SETSET != m*n+1 in which m is the number of words per function
00947         and n the number of functions. Currently (29-may-1997) m = 4.
00948     */
00949     *m++ = SETSET;
00950     *m++ = numsets+2;
00951     s = FirstWork + idhead;
00952     while ( --numsets >= 0 ) *m++ = *s++;
00953 }
00954 else {
00955     m = w + *w;
00956 }
00957 }
00958 /*
00959     We keep the whole thing in OldWork for the moment.
00960     We still have to add the number of the RHS expression.
00961     There is also some opportunity now to be smart about the pattern.
00962     This is needed for complicated wilddcarding with symmetric functions.
00963     We do this in a special routine during compile time to make sure
00964     that we loose as little time as possible (during running) if there
00965     is no need to be smart.
00966 */
00967 *m++ = 0;
00968 OldWork[1] = m - OldWork;
00969 AC.ProtoType = OldWork+idhead;
00970 if ( !error ) {
00971     if ( StudyPattern(OldWork) ) error = 1;
00972 }
00973 AT.WorkPointer = OldWork + OldWork[1];
00974 if ( AC.lhdollarflag ) OldWork[4] |= DOLLARFLAG;
00975 AC.lhdollarflag = 0;
00976 /*
00977     Test whether the id/idold configuration is fine.
00978 */
00979 if ( type == TYPEIDOLD ) {
00980     WORD ci = C->numlhs;
00981     while ( ci >= 1 ) {
00982         if ( C->lhs[ci][0] == TYPEIDNEW ) {
00983             if ( (C->lhs[ci][2] & SUBMASK) == SUBALL ) {
00984                 MesPrint("&Idold/also cannot follow an id,all statement.");
00985                 error = 1;
00986             }
00987             break;
00988         }
00989         else if ( C->lhs[ci][0] == TYPEDETCURDUM ) { ci--; continue; }
00990         else if ( C->lhs[ci][0] == TYPEIDOLD ) { ci--; continue; }
00991         else ci = 0;

```

```

00992     }
00993     if ( ci < 1 ) {
00994         MesPrint("&Idold/also should follow an id/idnew statement.");
00995         error = 1;
00996     }
00997 }
00998 /*
00999 Now the right hand side.
01000 */
01001 if ( type != TYPEIF ) {
01002     if ( ( retcode = CompileAlgebra(inp,RHSIDE,AC.ProtoType) ) < 0 ) error = 1;
01003     else {
01004         AC.ProtoType[2] = retcode;
01005         AC.DumNum = 0;
01006         if ( MinusSign ) { /* Flip the sign of the RHS */
01007             w = C->rhs[retcode];
01008             while ( *w ) { w += *w; w[-1] = -w[-1]; }
01009         }
01010         if ( AC.dumnumflag ) Add2Com(TYPEDETCURDUM)
01011     }
01012 }
01013 /*
01014 Actual adding happens only now after numrhs insertion
01015 */
01016 if ( !error ) { AddNtoL(OldWork[1],OldWork); }
01017 AllDone:
01018 AC.lhdollarflag = 0;
01019 AT.WorkPointer = FirstWork;
01020 return(error);
01021 }
01022
01023 /*
01024 #] CoIdExpression :
01025 #[ CoMultiply :
01026 */
01027
01028 static WORD mularray[13] = { TYPEMULT, SUBEXPSIZE+3, 0, SUBEXPRESSION,
01029     SUBEXPSIZE, 0, 1, 0, 0, 0, 0, 0, 0 };
01030
01031 int CoMultiply(UBYTE *inp)
01032 {
01033     UBYTE *p;
01034     int error = 0, RetCode;
01035     mularray[2] = 0; /* right multiply is default */
01036     while ( *inp == ',' ) inp++;
01037     /* if ( inp[-1] == '-' || inp[-1] == '+' ) inp--; */
01038     p = SkipField(inp,0);
01039     if ( *p ) {
01040         *p = 0;
01041         if ( StrICont(inp,(UBYTE *)"left") == 0 ) mularray[2] = 1;
01042         else if ( StrICont(inp,(UBYTE *)"right") == 0 ) mularray[2] = 0;
01043         else {
01044             MesPrint("&Illegal option in multiply statement or ; forgotten.");
01045             return(1);
01046         }
01047         *p = ',';
01048         inp = p + 1;
01049     }
01050     ClearWildcardNames();
01051     while ( *inp == ',' ) inp++;
01052     AC.ProtoType = mularray+3;
01053     mularray[7] = AC.cbufnum;
01054     if ( ( RetCode = CompileAlgebra(inp,RHSIDE,AC.ProtoType) ) < 0 ) error = 1;
01055     else {
01056         mularray[5] = RetCode;
01057         AddNtoL(SUBEXPSIZE+3,mularray);
01058         if ( AC.dumnumflag ) Add2Com(TYPEDETCURDUM)
01059     }
01060     return(error);
01061 }
01062
01063 /*
01064 #] CoMultiply :
01065 #[ CoFill :
01066
01067 Special additions for tablebase-like tables added 12-aug-2002
01068 */
01069
01070 int CoFill(UBYTE *inp)
01071 {
01072     GETIDENTITY
01073     WORD error = 0, x, xx, funnum, type, *oldwp = AT.WorkPointer;
01074     int i, oldcbufnum = AC.cbufnum, nofill = 0, numover, redef = 0;
01075     WORD *w, *wold, *Tprototype;
01076     UBYTE *p = inp, c, *inpl;
01077     TABLES T = 0, oldT;
01078     LONG newreservation, sum = 0;

```

```

01079     UBYTE *p1, *p2, *p3, *p4, *fake = 0;
01080     int tablestub = 0;
01081     if ( AC.exprfillwarning == 1 ) AC.exprfillwarning = 0;
01082 /*
01083     Read the name of the function and test that it is in the table.
01084 */
01085     p1 = inp;
01086     if ( ( p = SkipAName(inp) ) == 0 ) return(1);
01087     p2 = p;
01088     c = *p; *p = 0;
01089     if ( ( GetVar(inp,&type,&funnum,CFUNCTION,WITHAUTO) == NAMENOTFOUND )
01090 || ( T = functions[funnum].tabl ) == 0 || ( T->numind > 0 && c != '(' ) ) {
01091         MesPrint("%s should be a table with argument(s)",inp);
01092         *p = c; return(1);
01093     }
01094     oldT = T;
01095     *p++ = c;
01096     if ( T->numind == 0 ) {
01097         if ( c == '(' ) {
01098             if ( *p != ')' ) {
01099                 c = *p; *p = 0;
01100                 MesPrint("%s should be a table without arguments",inp);
01101                 *p = c; return(1);
01102             }
01103             else { p++; }
01104         }
01105         else { p--; }
01106         sum = 0;
01107         p3 = p;
01108         goto andagain;
01109     }
01110     w = oldwp;
01111     if ( T->numind < 0 ) { /* Pick up the first index */
01112         ParseSignedNumber(xx,p);
01113         if ( FG.cTable[p[-1]] != 1 || *p != ',' || xx < 1 || ( xx > ( -T->numind - 1 ) ) ) {
01114             MesPrint("&No valid number of table indices in *-table fill statement.");
01115             return(1);
01116         }
01117         *w++ = xx;
01118         p++;
01119     }
01120     else { xx = T->numind; }
01121     for ( sum = 0, i = 0; i < xx; i++ ) {
01122         ParseSignedNumber(x,p);
01123         if ( FG.cTable[p[-1]] != 1 || ( *p != ',' && *p != ')' ) ) {
01124             MesPrint("&Table arguments in fill statement should be numbers");
01125             return(1);
01126         }
01127         if ( T->sparse ) *w++ = x;
01128         else if ( x < T->mm[i].mini || x > T->mm[i].maxi ) {
01129             MesPrint("&Value %d for argument %d of table out of bounds",x,i+1);
01130             error = 1; nofill = 1;
01131         }
01132         else sum += ( x - T->mm[i].mini ) * T->mm[i].size;
01133         if ( *p == ')' ) break;
01134         p++;
01135     }
01136     p3 = p;
01137     if ( T->numind < 0 ) {
01138         for ( ; i < ABS(T->numind)-1; i++ ) *w++ = 0;
01139         xx = -T->numind;
01140     }
01141     if ( *p != ')' || i < ( xx - 1 ) ) {
01142         MesPrint("&Incorrect number of table arguments in fill statement. Should be %d",
01143             ,T->numind);
01144         error = 1; nofill = 1;
01145     }
01146     AT.WorkPointer = w;
01147     if ( T->sparse == 0 ) sum *= TABLEEXTENSION;
01148 andagain:;
01149     AC.cbunum = T->bunum;
01150     if ( T->sparse ) {
01151         i = FindTableTree(T,oldwp,1);
01152         if ( i >= 0 ) {
01153             sum = i + ABS(T->numind);
01154             if ( tablestub == 0 && ( ( T->sparse & 2 ) == 2 ) && ( T->mode != 0 )
01155                 && ( AC.vetotablebasefill == 0 ) ) {
01156 /*
01157                 This redefinition does not need a new stub
01158 */
01159                 functions[funnum].tabl = T = T->sparse;
01160                 tablestub = 1;
01161                 goto andagain;
01162             }
01163             redef = 1;
01164             goto redef;
01165         }

```



```

01166     if ( T->totind >= T->reserved ) {
01167         if ( T->reserved == 0 ) newreservation = 20;
01168         else newreservation = T->reserved;
01169         while ( T->totind >= newreservation && newreservation < MAXTABLECOMBUF )
01170             newreservation = 2*newreservation;
01171         if ( newreservation > MAXTABLECOMBUF ) newreservation = MAXTABLECOMBUF;
01172         if ( T->totind >= newreservation ) {
01173             MesPrint("@More than %ld elements in sparse table",MAXTABLECOMBUF);
01174             AC.cbufnum = oldcbufnum;
01175             Terminate(-1);
01176         }
01177         wold = (WORD *)Malloc1(newreservation*sizeof(WORD)*
01178                               (ABS(T->numind)+TABLEEXTENSION),"tablepointers");
01179         for ( i = T->reserved*(ABS(T->numind)+TABLEEXTENSION)-1; i >= 0; i-- )
01180             wold[i] = T->tablepointers[i];
01181         if ( T->tablepointers ) M_free(T->tablepointers,"tablepointers");
01182         T->tablepointers = wold;
01183         T->reserved = newreservation;
01184     }
01185     w = oldwp;
01186     for ( sum = T->totind*(ABS(T->numind)+TABLEEXTENSION), i = 0; i < ABS(T->numind); i++ ) {
01187         T->tablepointers[sum++] = *w++;
01188     }
01189     InsTableTree(T,T->tablepointers+sum-ABS(T->numind));
01190 #if TABLEEXTENSION == 2
01191     T->tablepointers[sum+TABLEEXTENSION-1] = -1; /* New element! */
01192 #else
01193     T->tablepointers[sum+1] = T->bufnum;
01194     T->tablepointers[sum+2] = -1;
01195     T->tablepointers[sum+3] = -1;
01196     T->tablepointers[sum+4] = 0;
01197     T->tablepointers[sum+5] = 0;
01198 #endif
01199 }
01200     else {
01201         if ( !nofill && T->tablepointers[sum] >= 0 ) {
01202             redef::
01203                 if ( AC.vetofilling ) nofill = 1;
01204                 else {
01205                     Warning("Table element was already defined. New definition will be used");
01206                 }
01207         }
01208 #if TABLEEXTENSION == 2
01209         T->tablepointers[sum+TABLEEXTENSION-1] = -1; /* New element! */
01210 #else
01211         T->tablepointers[sum+1] = T->bufnum;
01212         T->tablepointers[sum+2] = -1;
01213         T->tablepointers[sum+3] = -1;
01214         T->tablepointers[sum+4] = 0;
01215         T->tablepointers[sum+5] = 0;
01216 #endif
01217     }
01218     if ( T->numind ) { p++; }
01219     if ( *p != '=' ) {
01220         MesPrint("&Fill statement misses = sign after the table element");
01221         AC.cbufnum = oldcbufnum;
01222         AT.WorkPointer = oldwp;
01223         functions[funnum].tabl = oldT;
01224         return(1);
01225     }
01226     if ( tablestub == 0 && T->mode == 1 && AC.vetotablebasefill == 0 ) {
01227         /*
01228          Here we construct a righthandside from the indices and the wildcards
01229         */
01230         int numfake;
01231         tablestub = 1;
01232         p4 = T->argtail;
01233         while ( *p4 ) p4++;
01234         numfake = (p4-T->argtail)+(p3-p1)+10;
01235
01236         fake = (UBYTE *)Malloc1(numfake*sizeof(UBYTE),"Fill fake rhs");
01237         p = fake;
01238         *p++ = 't'; *p++ = 'b'; *p++ = 'l'; *p++ = '_'; *p++ = '(';
01239         p4 = p1; while ( p4 < p2 ) *p++ = *p4++; *p++ = ',';
01240         p4 = p2+1; while ( p4 < p3 ) *p++ = *p4++;
01241         if ( T->argtail ) {
01242             p4 = T->argtail + 1;
01243             while ( FG.cTable[*p4] == 1 ) p4++;
01244             while ( *p4 ) {
01245                 if ( *p4 == '?' && p[-1] != ',' ) {
01246                     p4++;
01247                     if ( FG.cTable[*p4] == 0 || *p4 == '$' || *p4 == '[' ) {
01248                         p4 = SkipAName(p4);
01249                         if ( *p4 == '[' ) {
01250                             SKIPBRA1(p4);
01251                         }
01252                     }

```

```

01253         else if ( *p4 == '{' ) {
01254             SKIPBRA2(p4);
01255         }
01256         else if ( *p4 ) { *p++ = *p4++; continue; }
01257     }
01258     else *p++ = *p4++;
01259 }
01260 }
01261 *p++ = '>';
01262 *p = 0;
01263 inpl = fake;
01264 }
01265 else {
01266     inpl = ++p;
01267 }
01268 c = 0;
01269 /*
01270 Now we have the indices and p points to the rhs.
01271 */
01272 numover = 0;
01273 AC.tablefilling = funnum;
01274 while ( *inpl ) {
01275     p = SkipField(inpl,0);
01276     c = *p; *p = 0;
01277 #ifdef WITHPTHREADS
01278     Tprototype = T->prototype[0];
01279 #else
01280     Tprototype = T->prototype;
01281 #endif
01282     if ( ( i = CompileAlgebra(inpl,RHSIDE,Tprototype) ) < 0 ) { error = 1; i = 0; }
01283     if ( !nofill ) {
01284         T->tablepointers[sum] = i;
01285         T->tablepointers[sum+1] = T->bufnum;
01286     }
01287     AC.DumNum = 0;
01288     *p = c;
01289     if ( T->sparse || c == 0 ) break;
01290     inpl = ++p;
01291 #if ( TABLEEXTENSION == 2 )
01292     sum++;
01293 #else
01294     sum += 2;
01295 #endif
01296     if ( !nofill && T->tablepointers[sum] >= 0 ) numover++;
01297 #if ( TABLEEXTENSION == 2 )
01298     sum++;
01299 #else
01300     sum += TABLEEXTENSION-2;
01301 #endif
01302 }
01303 if ( AC.exprfillwarning == 1 ) {
01304     AC.exprfillwarning = 2;
01305     Warning("Use of expressions and/or $variables in Fill statements is potentially very
dangerous.");
01306 }
01307 AC.tablefilling = 0;
01308 if ( T->sparse && c != 0 ) {
01309     MesPrint("&In sparse tables one can fill only one element at a time");
01310     error = 1;
01311 }
01312 else if ( numover ) {
01313     if ( numover == 1 )
01314         Warning("one element was overwritten. New definition will be used");
01315     else if ( AC.WarnFlag )
01316         MesPrint("&Warning: %d elements were overwritten. New definitions will be used",numover);
01317 }
01318 if ( T->sparse ) {
01319     if ( redef == 0 ) T->totind++;
01320 }
01321 else T->defined++;
01322 /*
01323 NumSets = AC.SetList.numtemp;
01324 NumSetElements = AC.SetElementList.numtemp;
01325 */
01326 if ( fake ) {
01327     M_free(fake,"Fill fake rhs");
01328     fake = 0;
01329     functions[funnum].tabl = T = T->sparse;
01330     p = p3;
01331     goto andagain;
01332 }
01333 AC.cbunum = oldcbunum;
01334 AC.SymChangeFlag = 1;
01335 AT.WorkPointer = oldwp;
01336 functions[funnum].tabl = oldT;
01337 return(error);
01338 }

```

```

01339
01340 /*
01341 #] CoFill :
01342 #[ CoFillExpression :
01343
01344 Syntax: FillExpression table = expression(x1,...,xn);
01345 The arguments should have been bracketed. Each corresponds to one
01346 of the dimensions of the table. Then the bracket with x1^2*x3^4
01347 will fill the (2,0,4) element of the table (if n=3 of course).
01348 Brackets that don't fit will be skipped. It just gives a warning.
01349
01350 New option (13-jul-2005)
01351 Syntax: FillExpression table = expression(f);
01352 The table indices are arguments of the function f which should
01353 have been bracketed before.
01354 */
01355
01356 int CoFillExpression(UBYTE *inp)
01357 {
01358     GETIDENTITY
01359     UBYTE *p, c;
01360     WORD type, funnum, expnum, symnum, numsym = 0, *oldwork = AT.WorkPointer;
01361     WORD *brackets, *term, brasize, *b, *m, *w, *pw, *tstop, zero = 0;
01362     WORD oldcbuf = AC.cbufnum, curelement = 0;
01363     int weneedit, i, j, numzero, pow, numfirst;
01364     TABLES T = 0;
01365     LONG newreservation, numcommu, sum;
01366     POSITION oldposition;
01367     FILEHANDLE *fi;
01368     CBUF *C;
01369     WORD numdummies;
01370
01371     AN.IndDum = AM.IndDum;
01372     if ( ( p = SkipAName(inp) ) == 0 ) return(1);
01373     c = *p; *p = 0;
01374     if ( ( GetVar(inp,&type,&funnum,CFUNCTION,NOAUTO) == NAMENOTFOUND )
01375     || ( T = functions[funnum].tabl ) == 0 ) {
01376         MesPrint("%s should be a previously declared table",inp);
01377         *p = c; return(1);
01378     }
01379     *p++ = c;
01380     if ( T->spare ) T = T->spare;
01381     C = cbuf + T->bufnum;
01382     if ( c != '=' ) {
01383         MesPrint("%sNo = sign in FillExpression statement");
01384         return(1);
01385     }
01386     inp = p;
01387     if ( ( p = SkipAName(inp) ) == 0 ) return(1);
01388     c = *p; *p = 0;
01389     if ( ( type = GetName(AC.exprnames,inp,&expnum,NOAUTO) ) == NAMENOTFOUND
01390     || c != '(' || (
01391         Expressions[expnum].status != LOCALEXPRESSION &&
01392         Expressions[expnum].status != SKIPLEXPRESSION &&
01393         Expressions[expnum].status != DROPLEXPRESSION &&
01394         Expressions[expnum].status != GLOBALEXPRESSION &&
01395         Expressions[expnum].status != SKIPGEXPRESSION &&
01396         Expressions[expnum].status != DROPGEXPRESSION ) ) {
01397         MesPrint("%s should be an active expression with arguments",inp);
01398         *p = c; return(1);
01399     }
01400     if ( Expressions[expnum].inmem ) {
01401         MesPrint("%s cannot be used in a FillExpression statement in the same %n\
01402 module that it has been redefined",inp);
01403         *p = c; return(1);
01404     }
01405     *p++ = c;
01406     while ( *p ) {
01407         inp = p;
01408         if ( ( p = SkipAName(inp) ) == 0 ) return(1);
01409         c = *p; *p = 0;
01410
01411         if ( GetVar(inp,&type,&symnum,-1,NOAUTO) == NAMENOTFOUND ) {
01412             MesPrint("%s should be a previously declared symbol or function",inp);
01413             *p = c; return(1);
01414         }
01415         else if ( type == CSYMBOL ) {
01416             *p++ = c;
01417             *AT.WorkPointer++ = symnum;
01418             numsym++;
01419         }
01420         else if ( type == CFUNCTION ) {
01421             numsym = -1;
01422             *p++ = c;
01423             if ( c != ')' ) {
01424                 MesPrint("%sArgument should be a single function or a list of symbols");
01425                 return(1);

```

```

01426         }
01427         symnum += FUNCTION;
01428         *AT.WorkPointer++ = symnum;
01429     }
01430     else {
01431         MesPrint("%&s should be a previously declared symbol or function",inp);
01432         *p = c; return(1);
01433     }
01434     if ( c == ' ' ) break;
01435     if ( c != ',' ) {
01436         MesPrint("%Illegal separator in FillExpression statement");
01437         goto noway;
01438     }
01439 }
01440 if ( *p ) {
01441     MesPrint("%Illegal end of FillExpression statement");
01442     goto noway;
01443 }
01444 /*
01445     We have the number of the table in funnum.
01446     The number of the expression in expnum, the table struct in T
01447     and either the numbers of the symbols in oldwork (there are numsym of them)
01448     or the number of the function in oldwork (just one and numsym = -1).
01449     We don't sort them!!!!
01450 */
01451 if ( ( numsym > 0 ) && ( ABS(T->numind) != numsym ) ) {
01452     MesPrint("%This table needs %d symbols for its array indices");
01453     goto noway;
01454 }
01455 EXCHINOUT
01456 #ifdef WITHMPI
01457 /*
01458     * The workers can't access to the data of the input expression. We need to
01459     * broadcast it to all the workers.
01460 */
01461 PF_BroadcastExpr(&Expressions[expnum], AR.infile);
01462 if ( PF.me == MASTER ) {
01463     /*
01464         * Restore the file position on the master.
01465     */
01466     POSITION pos;
01467     SetEndScratch(AR.infile, &pos);
01468 }
01469 #endif
01470 fi = AR.infile;
01471 if ( fi->handle >= 0 ) {
01472     PUTZERO(oldposition);
01473     SeekFile(fi->handle,&oldposition,SEEK_CUR);
01474     SetScratch(fi,&(Expressions[expnum].onfile));
01475     if ( ISNEGPOS(Expressions[expnum].onfile) ) {
01476         MesPrint("%File error in FillExpression");
01477         BACKINOUT
01478         goto noway;
01479     }
01480 }
01481 else {
01482 /*
01483     Note: Because everything fits inside memory we never get problems
01484     with excessive file sizes.
01485 */
01486     SETBASEPOSITION(oldposition,(UBYTE *) (fi->POfill)-(UBYTE *) (fi->PObuffer));
01487     fi->POfill = (WORD *) ((UBYTE *) (fi->PObuffer) + BASEPOSITION(Expressions[expnum].onfile));
01488 }
01489 pw = AT.WorkPointer;
01490 if ( numsym < 0 ) { brackets = pw + 1; }
01491 else { brackets = pw + numsym; }
01492 brasize = -1; wenedit = 0; /* stands for we need it */
01493 term = (WORD *) ((UBYTE *) (brackets)) + AM.MaxTer;
01494 AT.WorkPointer = (WORD *) ((UBYTE *) (term)) + AM.MaxTer;
01495 AC.cbufnum = T->bufnum;
01496 AC.tablefilling = funnum;
01497 if ( GetTerm(BHEAD term) > 0 ) { /* Skip prototype */
01498     while ( GetTerm(BHEAD term) > 0 ) {
01499         GETSTOP(term,tstop);
01500         w = m = term + 1;
01501         while ( m < tstop && *m != HAAKJE ) m += m[1];
01502         if ( *m != HAAKJE ) {
01503             MesPrint("%Illegal attempt to put an expression without brackets in a table");
01504             BACKINOUT
01505             goto noway;
01506         }
01507         if ( brasize == m - w ) {
01508             b = brackets;
01509             while ( *b == *w && w < m ) { b++; w++; }
01510             if ( w == m ) { /* Same as current bracket. Copy. */
01511                 if ( wenedit ) {
01512                     m += m[1] - 1;

```

```

01513         *m = *term - (m-term);
01514         AddNtoC(AC.cbufnum,*m,m,3);
01515         numdummies = DetCurDum(BHEAD term) - AM.IndDum;
01516         if ( numdummies > T->numdummies ) T->numdummies = numdummies;
01517     }
01518     continue; /* Next term */
01519 }
01520 }
01521 if ( weneedit ) {
01522     AddNtoC(AC.cbufnum,1,&zero,4); /* Terminate old bracket */
01523     numcommu = numcommute(C->rhs[curelement],&(C->NumTerms[curelement]));
01524     C->CanCommu[curelement] = numcommu;
01525 }
01526 b = brackets; w = term + 1;
01527 if ( numsym < 0 ) pw = oldwork + 1;
01528 else pw = oldwork + numsym;
01529 while ( w < m ) *b++ = *w++;
01530 brasize = b - brackets;
01531 /*
01532 Now compute the element. See whether we need it
01533 */
01534 if ( numsym < 0 ) {
01535     WORD *bb, bnum;
01536     if ( *brackets != symnum || brasize != brackets[1] ) {
01537         weneedit = 0; continue; /* Cannot work! */
01538     }
01539 /*
01540 Now count the number of arguments and whether they are numbers
01541 */
01542 b = brackets + FUNHEAD;
01543 bb = brackets+brackets[1];
01544 i = 0;
01545 if ( T->numind < 0 ) {
01546     bnum = b[1]+1;
01547     if ( bnum > -T->numind ) {
01548         weneedit = 0; continue; /* Cannot work! */
01549     }
01550 }
01551 else bnum = T->numind;
01552 while ( b < bb ) {
01553     if ( *b != -SNUMBER ) break;
01554     i++;
01555     b += 2;
01556 }
01557 if ( b < bb || i != bnum ) {
01558     weneedit = 0; continue; /* Cannot work! */
01559 }
01560 }
01561 else if ( brasize > 0 && ( *brackets != SYMBOL
01562 || brackets[1] < brasize || (brackets[1]-2) > numsym*2 ) ) {
01563     weneedit = 0; continue; /* Cannot work! */
01564 }
01565 numzero = 0; sum = 0;
01566 numfirst = 0;
01567 if ( numsym > 0 ) {
01568     for ( i = 0; i < numsym; i++ ) {
01569         if ( brasize > 0 ) {
01570             b = brackets + 2; j = brackets[1]-2;
01571             while ( j > 0 ) {
01572                 if ( *b == oldwork[i] ) break;
01573                 j -= 2; b += 2;
01574             }
01575             if ( j <= 0 ) { /* it was not there */
01576                 numzero++; pow = 0;
01577                 if ( 2*numzero+brackets[1]-2 > numsym*2 ) {
01578                     weneedit = 0; goto nextterm;
01579                 }
01580             }
01581             else pow = b[1];
01582         }
01583         else pow = 0;
01584         if ( T->sparse ) {
01585             if ( T->numind < 0 ) {
01586                 if ( i == 0 ) {
01587                     numfirst = pow;
01588                     if ( pow > -T->numind ) {
01589                         weneedit = 0; goto nextterm;
01590                     }
01591                 }
01592                 else if ( i > pow ) {
01593                     weneedit = 0; goto nextterm;
01594                 }
01595             }
01596             *pw++ = pow;
01597         }
01598         else if ( pow < T->mm[i].mini || pow > T->mm[i].maxi ) {
01599             weneedit = 0; goto nextterm;

```

```

01600         }
01601         else sum += ( pow - T->mm[i].mini ) * T->mm[i].size;
01602     }
01603 }
01604 else {
01605     WORD xx;
01606     b = brackets + FUNHEAD;
01607     sum = 0;
01608 /*
01609     Now scan the arguments of the function.
01610     We did check already the number and type of the arguments.
01611 */
01612     xx = (brackets[1]-FUNHEAD)/2;
01613     for ( i = 0; i < xx; i++ ) {
01614         pow = b[1];
01615         b += 2;
01616         if ( T->sparse ) {
01617             if ( T->numind < 0 ) {
01618                 if ( i == 0 ) {
01619                     numfirst = pow;
01620                     if ( pow >= -T->numind ) {
01621                         weneedit = 0; goto nextterm;
01622                     }
01623                 }
01624                 *pw++ = pow;
01625             }
01626             else if ( pow < T->mm[i].mini || pow > T->mm[i].maxi ) {
01627                 weneedit = 0; goto nextterm;
01628             }
01629             else sum += ( pow - T->mm[i].mini ) * T->mm[i].size;
01630         }
01631     }
01632     if ( T->numind < 0 ) {
01633         for ( i = numfirst+1; i < -T->numind; i++ ) *pw++ = 0;
01634     }
01635     weneedit = 1;
01636     if ( T->sparse ) {
01637         if ( numsym < 0 ) pw = oldwork + 1;
01638         else pw = oldwork + ABS(T->numind);
01639         i = FindTableTree(T,pw,1);
01640         if ( i >= 0 ) {
01641             sum = i+ABS(T->numind);
01642             /*
01643             Wrong!!!!
01644             C->rhs[T->tablepointers[sum]] = C->Pointer;
01645             */
01646             C->Pointer--; /* Back up over the zero */
01647             goto newentry;
01648         }
01649         if ( T->totind >= T->reserved ) {
01650             if ( T->reserved == 0 ) newreservation = 20;
01651             else newreservation = T->reserved;
01652             /*---Copied from Fill-----*/
01653             while ( T->totind >= newreservation && newreservation < MAXTABLECOMBUF )
01654                 newreservation = 2*newreservation;
01655             if ( newreservation > MAXTABLECOMBUF ) newreservation = MAXTABLECOMBUF;
01656             if ( T->totind >= newreservation ) {
01657                 MesPrint("@More than %ld elements in sparse table",MAXTABLECOMBUF);
01658                 AC.cbufnum = oldcbuf;
01659                 AT.WorkPointer = oldwork;
01660                 Terminate(-1);
01661             }
01662             /*---Copied from Fill-----*/
01663             if ( T->totind >= newreservation ) {
01664                 MesPrint("@More than %ld elements in sparse table",MAXTABLECOMBUF);
01665                 AC.cbufnum = oldcbuf;
01666                 AT.WorkPointer = oldwork;
01667                 Terminate(-1);
01668             }
01669             w = (WORD *)Malloca1(newreservation*sizeof(WORD)*
01670                 (ABS(T->numind)+TABLEEXTENSION),"tablepointers");
01671             for ( i = T->reserved*(ABS(T->numind)+TABLEEXTENSION)-1; i >= 0; i-- )
01672                 w[i] = T->tablepointers[i];
01673             if ( T->tablepointers ) M_free(T->tablepointers,"tablepointers");
01674             T->tablepointers = w;
01675             T->reserved = newreservation;
01676         }
01677         if ( numsym < 0 ) pw = oldwork + 1;
01678         else pw = oldwork + numsym;
01679         for ( sum = T->totind*(ABS(T->numind)+TABLEEXTENSION), i = 0; i < ABS(T->numind); i++
01680     ) {
01681         T->tablepointers[sum++] = *pw++;
01682     }
01683     InsTableTree(T,T->tablepointers+sum-ABS(T->numind));
01684     (T->totind)++;
01685     #if ( TABLEEXTENSION != 2 )

```

```

01686         else {
01687             sum *= TABLEEXTENSION;
01688         }
01689 #endif
01690 /*
01691     Start a new entry. Copy the element.
01692 */
01693     AddRHS(T->bufnum,0);
01694     T->tablepointers[sum] = C->numrhs;
01695 #if ( TABLEEXTENSION == 2 )
01696     T->tablepointers[sum+TABLEEXTENSION-1] = -1;
01697 #else
01698     T->tablepointers[sum+1] = T->bufnum;
01699     T->tablepointers[sum+2] = -1;
01700     T->tablepointers[sum+3] = -1;
01701     T->tablepointers[sum+4] = 0;
01702     T->tablepointers[sum+5] = 0;
01703 #endif
01704 newentry: if ( *m == HAAKJE ) { m += m[1] - 1; }
01705           else m--;
01706           *m = *term - (m-term);
01707           AddNtoC(AC.cbufnum,*m,m,5);
01708           curelement = T->tablepointers[sum];
01709 nextterm:
01710     }
01711     if ( weneedit ) {
01712         AddNtoC(AC.cbufnum,1,&zero,6); /* Terminate old bracket */
01713         numcommu = numcommute(C->rhs[curelement],&(C->NumTerms[curelement]));
01714         C->CanCommu[curelement] = numcommu;
01715     }
01716 }
01717 if ( fi->handle >= 0 ) {
01718     SetScratch(fi,&(oldposition));
01719 }
01720 else {
01721     fi->POfill = (WORD *) ((UBYTE *) (fi->PObuffer) + BASEPOSITION(oldposition));
01722 }
01723 BACKINOUT
01724 AC.cbufnum = oldcbuf;
01725 AC.tablefilling = 0;
01726 AT.WorkPointer = oldwork;
01727 return(0);
01728 noway:
01729 BACKINOUT
01730 AC.cbufnum = oldcbuf;
01731 AC.tablefilling = 0;
01732 AT.WorkPointer = oldwork;
01733 return(1);
01734 }
01735
01736 /*
01737 #] CoFillExpression :
01738 #[ CoPrintTable :
01739
01740 Syntax
01741     PrintTable [+f] [+s] tablename [>[>] file];
01742     All defined elements are written with individual Fill statements.
01743     If a file is specified, the result is written to file only.
01744     The flags of the print statement apply as much as possible.
01745     We make use of the regular write routines.
01746 */
01747
01748 int CoPrintTable(UBYTE *inp)
01749 {
01750     GETIDENTITY
01751     int fflag = 0, sflag = 0, addflag = 0, error = 0, sum, i, j;
01752     UBYTE *filename, *p, c, buffer[100], *s, *oldoutputline = AO.OutputLine;
01753     WORD type, funnum, *expr, *m, num;
01754     TABLES T = 0;
01755     WORD oldSkip = AO.OutSkip, oldMode = AC.OutputMode, oldHandle = AC.LogHandle;
01756     WORD oldType = AO.PrintType, *oldwork = AT.WorkPointer;
01757     UBYTE *oldFill = AO.OutFill, *oldLine = AO.OutputLine;
01758 #ifdef WITHMPI
01759     if ( PF.me != MASTER ) return 0;
01760 #endif
01761 /*
01762     First the flags
01763 */
01764     while ( *inp == '+' ) {
01765         inp++;
01766         if ( *inp == 'f' || *inp == 'F' ) { fflag = 1; inp++; }
01767         else if ( *inp == 's' || *inp == 'S' ) { sflag = PRINTONETERM; inp++; }
01768         else {
01769             MesPrint("%Illegal + option in PrintTable statement");
01770             error = 1; inp++;
01771         }
01772         while ( *inp != ',' && *inp && *inp != '+' ) {

```

```

01773         if ( !error ) {
01774             if ( *inp ) {
01775                 MesPrint("&Illegal + option in PrintTable statement");
01776                 inp++;
01777             }
01778             else {
01779                 MesPrint("&Unfinished PrintTable statement");
01780                 return(1);
01781             }
01782             error = 1;
01783         }
01784         inp++;
01785     }
01786     if ( *inp == ',' ) inp++;
01787 }
01788 /*
01789 Now the name of the table
01790 */
01791 if ( ( p = SkipAName(inp) ) == 0 ) return(1);
01792 c = *p; *p = 0;
01793 if ( ( GetVar(inp,&type,&funnum,CFUNCTION,NOAUTO) == NAMENOTFOUND )
01794 || ( T = functions[funnum].tabl ) == 0 ) {
01795     MesPrint("&%s should be a previously declared table",inp);
01796     *p = c; return(1);
01797 }
01798 if ( T->sparse && T->mode == 1 ) T = T->sparse;
01799 *p++ = c;
01800 /*
01801 Check for a filename. Runs to the end of the statement.
01802 */
01803 filename = 0;
01804 if ( c == '>' ) {
01805     if ( *p == '>' ) { addflag = 1; p++; }
01806     filename = p;
01807 }
01808 else filename = 0;
01809
01810 if ( filename ) {
01811     if ( addflag ) AC.LogHandle = OpenAddFile((char *)filename);
01812     else AC.LogHandle = CreateFile((char *)filename);
01813     if ( AC.LogHandle < 0 ) {
01814         MesPrint("&Cannot open file '%s' properly",filename);
01815         error = 1; goto finally;
01816     }
01817     AO.PrintType = PRINTLFILE;
01818 }
01819 else if ( fflag && AC.LogHandle >= 0 ) {
01820     AO.PrintType = PRINTLFILE;
01821 }
01822 AO.OutFill = AO.OutputLine = (UBYTE *)AT.WorkPointer;
01823 AT.WorkPointer += 2*AC.LineLength;
01824
01825 AO.PrintType |= sflag;
01826 AC.OutputMode = 0;
01827 AO.IsBracket = 0;
01828 AO.OutSkip = 0;
01829 AR.DeferFlag = 0;
01830 AC.outsidefun = 1;
01831 if ( AC.LogHandle == oldHandle ) FiniLine();
01832 AO.OutputLine = AO.OutFill = (UBYTE *)Malloc1(AC.LineLength+20,"PrintTable");
01833 AO.OutStop = AO.OutFill + AC.LineLength;
01834 for ( i = 0; i < T->totind; i++ ) {
01835     if ( !T->sparse && T->tablepointers[i*TABLEEXTENSION] < 0 ) continue;
01836     TokenToLine((UBYTE *)"Fill ");
01837     TokenToLine((UBYTE *) (VARNAME(functions,funnum)));
01838     TokenToLine((UBYTE *) "(");
01839     AO.OutSkip = 3;
01840     if ( T->sparse ) {
01841         sum = i * ( T->numind + TABLEEXTENSION );
01842         for ( j = 0; j < T->numind; j++, sum++ ) {
01843             if ( j > 0 ) TokenToLine((UBYTE *)",");
01844             num = T->tablepointers[sum];
01845             s = buffer; s = NumCopy(num,s);
01846             TokenToLine(buffer);
01847         }
01848         expr = cbuf[T->bufnum].rhs[T->tablepointers[sum]];
01849     }
01850     else {
01851         for ( j = 0; j < T->numind; j++ ) {
01852             if ( j > 0 ) {
01853                 TokenToLine((UBYTE *)",");
01854                 num = T->mm[j].mini + ( i % T->mm[j-1].size ) / T->mm[j].size;
01855             }
01856             else {
01857                 num = T->mm[j].mini + i / T->mm[j].size;
01858             }
01859             s = buffer; s = NumCopy(num,s);

```



```

01860         TokenToLine(buffer);
01861     }
01862     expr = cbuf[T->bufnum].rhs[T->tablepointers[TABLEEXTENSION*i]];
01863 }
01864 TOKENTOLINE("=", "=");
01865 if ( sflag ) {
01866     FiniLine();
01867     if ( AC.OutputSpaces != NOSPACEFORMAT ) TokenToLine((UBYTE *) " ");
01868 }
01869 m = expr;
01870 /*
01871     WORD lbrac, first;
01872     lbrac = 0; first = 1;
01873     while ( *m ) {
01874         if ( WriteTerm(m,&lbrac,first,1,0) ) {
01875             MesPrint("Error while writing table");
01876             error = 1;
01877             goto finally;
01878         }
01879         first = 0;
01880         m += *m;
01881     }
01882     if ( first ) { TOKENTOLINE(" 0","0") }
01883     else if ( lbrac ) { TOKENTOLINE(")","") }
01884 */
01885     while ( *m ) m += *m;
01886     if ( m > expr ) {
01887         if ( WriteExpression(expr, (LONG) (m-expr)) ) { error = 1; goto finally; }
01888         AO.OutSkip = 0;
01889     }
01890     else {
01891         TokenToLine((UBYTE *) "0");
01892     }
01893     TokenToLine((UBYTE *) ";" );
01894     FiniLine();
01895 }
01896 M_free(AO.OutputLine, "PrintTable");
01897 AO.OutputLine = AO.OutFill = oldoutputline;
01898 /*
01899     Reset the file pointers and parameters if any. Close file if needed.
01900 */
01901 finally:
01902     AO.OutSkip = oldSkip;
01903     AC.OutputMode = oldMode;
01904     AC.LogHandle = oldHandle;
01905     AO.PrintType = oldType;
01906     AO.OutFill = oldFill;
01907     AO.OutputLine = oldLine;
01908     AT.WorkPointer = oldwork;
01909     AC.outsidefun = 0;
01910     return(error);
01911 }
01912
01913 /*
01914     #] CoPrintTable :
01915     #[ CoAssign :
01916
01917     This statement has an easy syntax:
01918         $name = expression
01919 */
01920
01921 static WORD AssignLHS[14] = { TYPEASSIGN, 3+SUBEXPSIZE, 0,
01922                             SUBEXPRESSION, SUBEXPSIZE, 0, 1, 0, 0,0,0,0,0 };
01923
01924 int CoAssign(UBYTE *inp)
01925 {
01926     int error = 0, retcode;
01927     UBYTE *name, c;
01928     WORD number;
01929     if ( *inp != '$' ) {
01930 nolhs: MesPrint("&assign statement should have a dollar variable in the LHS");
01931         return(1);
01932     }
01933     inp++; name = inp;
01934     if ( FG.cTable[*inp] != 0 ) goto nolhs;
01935     while ( FG.cTable[*inp] < 2 ) inp++;
01936     if ( AP.PreAssignFlag == 2 ) {
01937         if ( *inp == '_' ) inp++;
01938     }
01939     if ( ( *inp == ',' && inp[1] != '=' ) && ( *inp != '=' ) ) {
01940         MesPrint("&assign statement should have only a dollar variable in the LHS");
01941         return(1);
01942     }
01943     c = *inp;
01944     *inp = 0;
01945     if ( GetName(AC.dollarnames, name, &number, NOAUTO) == NAMENOTFOUND ) {
01946         number = AddDollar(name, DOLUNDEFINED, 0, 0);

```

```

01947     }
01948     *inp = c;
01949     if ( c == ',' ) inp++;
01950     *inp++ = '=';
01951     if ( *inp == ',' ) inp++;
01952 /*
01953     Fake a Prototype and read the RHS
01954 */
01955     AssignLHS[7] = AC.cbunum;
01956     retcode = CompileAlgebra(inp,RHSIDE,(AssignLHS+3));
01957     if ( retcode < 0 ) error = 1;
01958     AC.DumNum = 0;
01959 /*
01960     Now add the LHS
01961 */
01962     AssignLHS[2] = number;
01963     AssignLHS[5] = retcode;
01964     AddNtoL(AssignLHS[1],AssignLHS);
01965 /*
01966     Add to the list of potentially modified dollars (for PARALLEL)
01967 */
01968     AddPotModdollar(number);
01969     return(error);
01970 }
01971
01972 /*
01973     #] CoAssign :
01974     #[ CoDeallocateTable :
01975
01976     Syntax: DeallocateTable tablename(s);
01977     Should work only for sparse tables.
01978     Action: Cleans all definitions of elements of a table as if there have
01979            never been any fill statements.
01980 */
01981
01982 int CoDeallocateTable(UBYTE *inp)
01983 {
01984     UBYTE *p, c;
01985     TABLES T = 0;
01986     WORD type, funnum, i;
01987     c = *inp;
01988     while ( c ) {
01989         while ( *inp == ',' ) inp++;
01990         if ( *inp == 0 ) break;
01991         if ( ( p = SkipAName(inp) ) == 0 ) return(1);
01992         c = *p; *p = 0;
01993         if ( ( GetVar(inp,&type,&funnum,CFUNCTION,NOAUTO) == NAMENOTFOUND )
01994             || ( T = functions[funnum].tabl ) == 0 ) {
01995             MesPrint("%s should be a previously declared table",inp);
01996             *p = c; return(1);
01997         }
01998         if ( T->sparse == 0 ) {
01999             MesPrint("%s should be a sparse table",inp);
02000             *p = c; return(1);
02001         }
02002         if ( T->tablepointers ) M_free(T->tablepointers,"tablepointers");
02003         ClearTableTree(T);
02004         for ( i = 0; i < T->buffersfill; i++ ) { /* was <= */
02005             finishcbuf(T->buffers[i]);
02006         }
02007         T->bufnum = inicbufs();
02008         T->buffersfill = 0;
02009         T->buffers[T->buffersfill++] = T->bufnum;
02010         T->tablepointers = 0;
02011         T->boomlijst = 0;
02012         T->totind = 0;
02013         T->reserved = 0;
02014
02015         if ( T->sparse ) {
02016             TABLES TT = T->sparse;
02017             if ( TT->tablepointers ) M_free(TT->tablepointers,"tablepointers");
02018             ClearTableTree(TT);
02019             for ( i = 0; i < TT->buffersfill; i++ ) { /* was <= */
02020                 finishcbuf(TT->buffers[i]);
02021             }
02022             TT->bufnum = inicbufs();
02023             TT->buffersfill = 0;
02024             TT->buffers[T->buffersfill++] = T->bufnum;
02025             TT->tablepointers = 0;
02026             TT->boomlijst = 0;
02027             TT->totind = 0;
02028             TT->reserved = 0;
02029         }
02030         *p++ = c;
02031         inp = p;
02032     }
02033     return(0);

```

```

02034 }
02035
02036 /*
02037  #] CoDeallocateTable :
02038  #[ CoFactorCache :
02039 */
02040 /*
02050 int CoFactorCache(UBYTE *inp)
02051 {
02052     Code to be added in due time
02053     We need to read 'expression', get its terms through Generator and sort them.
02054     We store the result in the WorkSpace in argument notation.
02055     This will be argin.
02056     Then we do the same with the sequence of factors. They form argout.
02057     The whole is put in the buffer with the call
02058         InsertArg(BHEAD argin,argout,1)
02059     return(0);
02060 }
02061 */
02062
02063 /*
02064  #] CoFactorCache :
02065 */

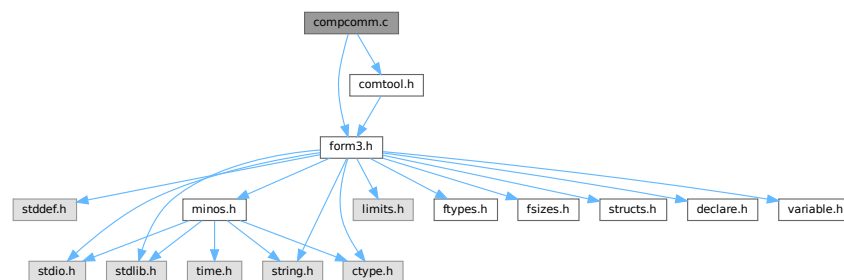
```

4.9 compcomm.c File Reference

```
#include "form3.h"
```

```
#include "comtool.h"
```

Include dependency graph for compcomm.c:



Functions

- int [CoFormat](#) (UBYTE *s)
- int [CoCollect](#) (UBYTE *s)
- int [setonoff](#) (UBYTE *s, int *flag, int onvalue, int offvalue)
- int [CoCompress](#) (UBYTE *s)
- int [CoFlags](#) (UBYTE *s, int value)
- int [CoOff](#) (UBYTE *s)
- int [CoOn](#) (UBYTE *s)
- int [ColInsideFirst](#) (UBYTE *s)
- int [CoProperCount](#) (UBYTE *s)
- int [CoDelete](#) (UBYTE *s)
- int [CoKeep](#) (UBYTE *s)
- int [CoFixIndex](#) (UBYTE *s)
- int [CoMetric](#) (UBYTE *s)
- int [DoPrint](#) (UBYTE *s, int par)
- int [CoPrint](#) (UBYTE *s)

- int [CoPrintB](#) (UBYTE *s)
- int [CoNPrint](#) (UBYTE *s)
- int [CoPushHide](#) (UBYTE *s)
- int [CoPopHide](#) (UBYTE *s)
- int [SetExprCases](#) (int par, int setunset, int val)
- int [SetExpr](#) (UBYTE *s, int setunset, int par)
- int [CoDrop](#) (UBYTE *s)
- int [CoNoDrop](#) (UBYTE *s)
- int [CoSkip](#) (UBYTE *s)
- int [CoNoSkip](#) (UBYTE *s)
- int [CoHide](#) (UBYTE *inp)
- int [CoIntoHide](#) (UBYTE *inp)
- int [CoNoIntoHide](#) (UBYTE *inp)
- int [CoNoHide](#) (UBYTE *inp)
- int [CoUnHide](#) (UBYTE *inp)
- int [CoNoUnHide](#) (UBYTE *inp)
- void [AddToCom](#) (int n, WORD *array)
- int [AddComString](#) (int n, WORD *array, UBYTE *thestring, int par)
- int [Add2ComStrings](#) (int n, WORD *array, UBYTE *string1, UBYTE *string2)
- int [CoDiscard](#) (UBYTE *s)
- int [CoContract](#) (UBYTE *s)
- int [CoGoTo](#) (UBYTE *inp)
- int [CoLabel](#) (UBYTE *inp)
- int [DoArgument](#) (UBYTE *s, int par)
- int [CoArgument](#) (UBYTE *s)
- int [CoEndArgument](#) (UBYTE *s)
- int [CoInside](#) (UBYTE *s)
- int [CoEndInside](#) (UBYTE *s)
- int [CoNormalize](#) (UBYTE *s)
- int [CoMakeInteger](#) (UBYTE *s)
- int [CoSplitArg](#) (UBYTE *s)
- int [CoSplitFirstArg](#) (UBYTE *s)
- int [CoSplitLastArg](#) (UBYTE *s)
- int [CoFactArg](#) (UBYTE *s)
- int [DoSymmetrize](#) (UBYTE *s, int par)
- int [CoSymmetrize](#) (UBYTE *s)
- int [CoAntiSymmetrize](#) (UBYTE *s)
- int [CoCycleSymmetrize](#) (UBYTE *s)
- int [CoRCycleSymmetrize](#) (UBYTE *s)
- int [CoWrite](#) (UBYTE *s)
- int [CoNWrite](#) (UBYTE *s)
- int [CoRatio](#) (UBYTE *s)
- int [CoRedefine](#) (UBYTE *s)
- int [CoRenumber](#) (UBYTE *s)
- int [CoSum](#) (UBYTE *s)
- int [CoToTensor](#) (UBYTE *s)
- int [CoToVector](#) (UBYTE *s)
- int [CoTrace4](#) (UBYTE *s)
- int [CoTraceN](#) (UBYTE *s)
- int [CoChisholm](#) (UBYTE *s)
- int [DoChain](#) (UBYTE *s, int option)
- int [CoChainin](#) (UBYTE *s)
- int [CoChainout](#) (UBYTE *s)
- int [CoExit](#) (UBYTE *s)
- int [CoInParallel](#) (UBYTE *s)

- int [CoNotInParallel](#) (UBYTE *s)
- int [DoInParallel](#) (UBYTE *s, int par)
- int [CoInExpression](#) (UBYTE *s)
- int [CoEndInExpression](#) (UBYTE *s)
- int [CoSetExitFlag](#) (UBYTE *s)
- int [CoTryReplace](#) (UBYTE *p)
- int [CoModulus](#) (UBYTE *inp)
- int [CoRepeat](#) (UBYTE *inp)
- int [CoEndRepeat](#) (UBYTE *inp)
- int [DoBrackets](#) (UBYTE *inp, int par)
- int [CoBracket](#) (UBYTE *inp)
- int [CoAntiBracket](#) (UBYTE *inp)
- int [CoMultiBracket](#) (UBYTE *inp)
- WORD * [CountComp](#) (UBYTE *inp, WORD *to)
- int [Colf](#) (UBYTE *inp)
- int [CoElse](#) (UBYTE *p)
- int [CoElseif](#) (UBYTE *inp)
- int [CoEndIf](#) (UBYTE *inp)
- int [CoWhile](#) (UBYTE *inp)
- int [CoEndWhile](#) (UBYTE *inp)
- int [DoFindLoop](#) (UBYTE *inp, int mode)
- int [CoFindLoop](#) (UBYTE *inp)
- int [CoReplaceLoop](#) (UBYTE *inp)
- int [CoFunPowers](#) (UBYTE *inp)
- int [CoUnitTrace](#) (UBYTE *s)
- int [CoTerm](#) (UBYTE *s)
- int [CoEndTerm](#) (UBYTE *s)
- int [CoSort](#) (UBYTE *s)
- int [CoPolyFun](#) (UBYTE *s)
- int [CoPolyRatFun](#) (UBYTE *s)
- int [CoMerge](#) (UBYTE *inp)
- int [CoStuffle](#) (UBYTE *inp)
- int [CoProcessBucket](#) (UBYTE *s)
- int [CoThreadBucket](#) (UBYTE *s)
- int [DoArgPlode](#) (UBYTE *s, int par)
- int [CoArgExplode](#) (UBYTE *s)
- int [CoArgImplode](#) (UBYTE *s)
- int [CoClearTable](#) (UBYTE *s)
- int [CoDenominators](#) (UBYTE *s)
- int [CoDropCoefficient](#) (UBYTE *s)
- int [CoDropSymbols](#) (UBYTE *s)
- int [CoToPolynomial](#) (UBYTE *inp)
- int [CoFromPolynomial](#) (UBYTE *inp)
- int [CoArgToExtraSymbol](#) (UBYTE *s)
- int [CoExtraSymbols](#) (UBYTE *inp)
- WORD * [GetIfDollarFactor](#) (UBYTE **inp, WORD *w)
- UBYTE * [GetDoParam](#) (UBYTE *inp, WORD **wp, int par)
- int [CoDo](#) (UBYTE *inp)
- int [CoEndDo](#) (UBYTE *inp)
- int [CoFactDollar](#) (UBYTE *inp)
- int [CoFactorize](#) (UBYTE *s)
- int [CoNFactorize](#) (UBYTE *s)
- int [CoUnFactorize](#) (UBYTE *s)
- int [CoNUnFactorize](#) (UBYTE *s)
- int [DoFactorize](#) (UBYTE *s, int par)

- int [CoOptimizeOption](#) (UBYTE *s)
- int [CoPutInside](#) (UBYTE *inp)
- int [CoAntiPutInside](#) (UBYTE *inp)
- int [DoPutInside](#) (UBYTE *inp, int par)
- int [CoSwitch](#) (UBYTE *s)
- int [CoCase](#) (UBYTE *s)
- int [CoBreak](#) (UBYTE *s)
- int [CoDefault](#) (UBYTE *s)
- int [CoEndSwitch](#) (UBYTE *s)
- int [CoSetUserFlag](#) (UBYTE *s)
- int [CoClearUserFlag](#) (UBYTE *s)
- int [CoCreateAllLoops](#) (UBYTE *s)
- int [CoCreateAllPaths](#) (UBYTE *s)
- int [CoCreateAll](#) (UBYTE *s)

4.9.1 Detailed Description

Compiler routines for most statements that don't involve algebraic expressions. Exceptions: all routines involving declarations are in the file [names.c](#) When making new statements one can add the compiler routines here and have a look whether there is already a routine that is similar. In that case one can make a copy and modify it.

Definition in file [compcomm.c](#).

4.9.2 Function Documentation

4.9.2.1 CoFormat()

```
int CoFormat (
    UBYTE * s )
```

Definition at line [153](#) of file [compcomm.c](#).

4.9.2.2 CoCollect()

```
int CoCollect (
    UBYTE * s )
```

Definition at line [428](#) of file [compcomm.c](#).

4.9.2.3 setonoff()

```
int setonoff (
    UBYTE * s,
    int * flag,
    int onvalue,
    int offvalue )
```

Definition at line [490](#) of file [compcomm.c](#).

4.9.2.4 CoCompress()

```
int CoCompress (
    UBYTE * s )
```

Definition at line 506 of file [compcomm.c](#).

4.9.2.5 CoFlags()

```
int CoFlags (
    UBYTE * s,
    int value )
```

Definition at line 555 of file [compcomm.c](#).

4.9.2.6 CoOff()

```
int CoOff (
    UBYTE * s )
```

Definition at line 590 of file [compcomm.c](#).

4.9.2.7 CoOn()

```
int CoOn (
    UBYTE * s )
```

Definition at line 656 of file [compcomm.c](#).

4.9.2.8 CoInsideFirst()

```
int CoInsideFirst (
    UBYTE * s )
```

Definition at line 928 of file [compcomm.c](#).

4.9.2.9 CoProperCount()

```
int CoProperCount (
    UBYTE * s )
```

Definition at line 935 of file [compcomm.c](#).

4.9.2.10 CoDelete()

```
int CoDelete (
    UBYTE * s )
```

Definition at line 942 of file [compcomm.c](#).

4.9.2.11 CoKeep()

```
int CoKeep (
    UBYTE * s )
```

Definition at line 987 of file [compcomm.c](#).

4.9.2.12 CoFixIndex()

```
int CoFixIndex (
    UBYTE * s )
```

Definition at line 999 of file [compcomm.c](#).

4.9.2.13 CoMetric()

```
int CoMetric (
    UBYTE * s )
```

Definition at line 1033 of file [compcomm.c](#).

4.9.2.14 DoPrint()

```
int DoPrint (
    UBYTE * s,
    int par )
```

Definition at line 1041 of file [compcomm.c](#).

4.9.2.15 CoPrint()

```
int CoPrint (
    UBYTE * s )
```

Definition at line 1257 of file [compcomm.c](#).

4.9.2.16 CoPrintB()

```
int CoPrintB (
    UBYTE * s )
```

Definition at line 1264 of file [compcomm.c](#).

4.9.2.17 CoNPrint()

```
int CoNPrint (
    UBYTE * s )
```

Definition at line 1271 of file [compcomm.c](#).

4.9.2.18 CoPushHide()

```
int CoPushHide (
    UBYTE * s )
```

Definition at line 1278 of file [compcomm.c](#).

4.9.2.19 CoPopHide()

```
int CoPopHide (
    UBYTE * s )
```

Definition at line 1322 of file [compcomm.c](#).

4.9.2.20 SetExprCases()

```
int SetExprCases (
    int par,
    int setunset,
    int val )
```

Definition at line 1357 of file [compcomm.c](#).

4.9.2.21 SetExpr()

```
int SetExpr (
    UBYTE * s,
    int setunset,
    int par )
```

Definition at line 1512 of file [compcomm.c](#).

4.9.2.22 CoDrop()

```
int CoDrop (
    UBYTE * s )
```

Definition at line 1557 of file [compcomm.c](#).

4.9.2.23 CoNoDrop()

```
int CoNoDrop (
    UBYTE * s )
```

Definition at line 1564 of file [compcomm.c](#).

4.9.2.24 CoSkip()

```
int CoSkip (
    UBYTE * s )
```

Definition at line 1571 of file [compcomm.c](#).

4.9.2.25 CoNoSkip()

```
int CoNoSkip (
    UBYTE * s )
```

Definition at line 1578 of file [compcomm.c](#).

4.9.2.26 CoHide()

```
int CoHide (
    UBYTE * inp )
```

Definition at line 1585 of file [compcomm.c](#).

4.9.2.27 CoIntoHide()

```
int CoIntoHide (
    UBYTE * inp )
```

Definition at line 1603 of file [compcomm.c](#).

4.9.2.28 CoNoIntoHide()

```
int CoNoIntoHide (
    UBYTE * inp )
```

Definition at line 1621 of file [compcomm.c](#).

4.9.2.29 CoNoHide()

```
int CoNoHide (
    UBYTE * inp )
```

Definition at line 1628 of file [compcomm.c](#).

4.9.2.30 CoUnHide()

```
int CoUnHide (
    UBYTE * inp )
```

Definition at line 1635 of file [compcomm.c](#).

4.9.2.31 CoNoUnHide()

```
int CoNoUnHide (
    UBYTE * inp )
```

Definition at line 1642 of file [compcomm.c](#).

4.9.2.32 AddToCom()

```
void AddToCom (
    int n,
    WORD * array )
```

Definition at line 1649 of file [compcomm.c](#).

4.9.2.33 AddComString()

```
int AddComString (
    int n,
    WORD * array,
    UBYTE * thestring,
    int par )
```

Definition at line 1664 of file [compcomm.c](#).

4.9.2.34 Add2ComStrings()

```
int Add2ComStrings (
    int n,
    WORD * array,
    UBYTE * string1,
    UBYTE * string2 )
```

Definition at line 1723 of file [compcomm.c](#).

4.9.2.35 CoDiscard()

```
int CoDiscard (
    UBYTE * s )
```

Definition at line 1764 of file [compcomm.c](#).

4.9.2.36 CoContract()

```
int CoContract (
    UBYTE * s )
```

Definition at line 1787 of file [compcomm.c](#).

4.9.2.37 CoGoTo()

```
int CoGoTo (
    UBYTE * inp )
```

Definition at line 1816 of file [compcomm.c](#).

4.9.2.38 CoLabel()

```
int CoLabel (
    UBYTE * inp )
```

Definition at line 1835 of file [compcomm.c](#).

4.9.2.39 DoArgument()

```
int DoArgument (
    UBYTE * s,
    int par )
```

Definition at line 1862 of file [compcomm.c](#).

4.9.2.40 CoArgument()

```
int CoArgument (
    UBYTE * s )
```

Definition at line 2096 of file [compcomm.c](#).

4.9.2.41 CoEndArgument()

```
int CoEndArgument (
    UBYTE * s )
```

Definition at line 2103 of file [compcomm.c](#).

4.9.2.42 CoInside()

```
int CoInside (
    UBYTE * s )
```

Definition at line 2129 of file [compcomm.c](#).

4.9.2.43 CoEndInside()

```
int CoEndInside (
    UBYTE * s )
```

Definition at line 2136 of file [compcomm.c](#).

4.9.2.44 CoNormalize()

```
int CoNormalize (
    UBYTE * s )
```

Definition at line 2162 of file [compcomm.c](#).

4.9.2.45 CoMakeInteger()

```
int CoMakeInteger (
    UBYTE * s )
```

Definition at line 2169 of file [compcomm.c](#).

4.9.2.46 CoSplitArg()

```
int CoSplitArg (
    UBYTE * s )
```

Definition at line 2176 of file [compcomm.c](#).

4.9.2.47 CoSplitFirstArg()

```
int CoSplitFirstArg (
    UBYTE * s )
```

Definition at line 2183 of file [compcomm.c](#).

4.9.2.48 CoSplitLastArg()

```
int CoSplitLastArg (
    UBYTE * s )
```

Definition at line 2190 of file [compcomm.c](#).

4.9.2.49 CoFactArg()

```
int CoFactArg (
    UBYTE * s )
```

Definition at line 2197 of file [compcomm.c](#).

4.9.2.50 DoSymmetrize()

```
int DoSymmetrize (
    UBYTE * s,
    int par )
```

Definition at line 2217 of file [compcomm.c](#).

4.9.2.51 CoSymmetrize()

```
int CoSymmetrize (  
    UBYTE * s )
```

Definition at line [2337](#) of file [compcomm.c](#).

4.9.2.52 CoAntiSymmetrize()

```
int CoAntiSymmetrize (  
    UBYTE * s )
```

Definition at line [2344](#) of file [compcomm.c](#).

4.9.2.53 CoCycleSymmetrize()

```
int CoCycleSymmetrize (  
    UBYTE * s )
```

Definition at line [2351](#) of file [compcomm.c](#).

4.9.2.54 CoRCycleSymmetrize()

```
int CoRCycleSymmetrize (  
    UBYTE * s )
```

Definition at line [2358](#) of file [compcomm.c](#).

4.9.2.55 CoWrite()

```
int CoWrite (  
    UBYTE * s )
```

Definition at line [2365](#) of file [compcomm.c](#).

4.9.2.56 CoNWrite()

```
int CoNWrite (  
    UBYTE * s )
```

Definition at line [2390](#) of file [compcomm.c](#).

4.9.2.57 CoRatio()

```
int CoRatio (  
    UBYTE * s )
```

Definition at line [2417](#) of file [compcomm.c](#).

4.9.2.58 CoRedefine()

```
int CoRedefine (
    UBYTE * s )
```

Definition at line [2457](#) of file [compcomm.c](#).

4.9.2.59 CoRenumber()

```
int CoRenumber (
    UBYTE * s )
```

Definition at line [2562](#) of file [compcomm.c](#).

4.9.2.60 CoSum()

```
int CoSum (
    UBYTE * s )
```

Definition at line [2583](#) of file [compcomm.c](#).

4.9.2.61 CoToTensor()

```
int CoToTensor (
    UBYTE * s )
```

Definition at line [2703](#) of file [compcomm.c](#).

4.9.2.62 CoToVector()

```
int CoToVector (
    UBYTE * s )
```

Definition at line [2871](#) of file [compcomm.c](#).

4.9.2.63 CoTrace4()

```
int CoTrace4 (
    UBYTE * s )
```

Definition at line [2941](#) of file [compcomm.c](#).

4.9.2.64 CoTraceN()

```
int CoTraceN (
    UBYTE * s )
```

Definition at line [3028](#) of file [compcomm.c](#).

4.9.2.65 CoChisholm()

```
int CoChisholm (
    UBYTE * s )
```

Definition at line 3088 of file [compcomm.c](#).

4.9.2.66 DoChain()

```
int DoChain (
    UBYTE * s,
    int option )
```

Definition at line 3170 of file [compcomm.c](#).

4.9.2.67 CoChainin()

```
int CoChainin (
    UBYTE * s )
```

Definition at line 3214 of file [compcomm.c](#).

4.9.2.68 CoChainout()

```
int CoChainout (
    UBYTE * s )
```

Definition at line 3226 of file [compcomm.c](#).

4.9.2.69 CoExit()

```
int CoExit (
    UBYTE * s )
```

Definition at line 3236 of file [compcomm.c](#).

4.9.2.70 CoInParallel()

```
int CoInParallel (
    UBYTE * s )
```

Definition at line 3263 of file [compcomm.c](#).

4.9.2.71 CoNotInParallel()

```
int CoNotInParallel (
    UBYTE * s )
```

Definition at line 3273 of file [compcomm.c](#).

4.9.2.72 DoInParallel()

```
int DoInParallel (
    UBYTE * s,
    int par )
```

Definition at line 3288 of file [compcomm.c](#).

4.9.2.73 CoInExpression()

```
int CoInExpression (
    UBYTE * s )
```

Definition at line 3357 of file [compcomm.c](#).

4.9.2.74 CoEndInExpression()

```
int CoEndInExpression (
    UBYTE * s )
```

Definition at line 3410 of file [compcomm.c](#).

4.9.2.75 CoSetExitFlag()

```
int CoSetExitFlag (
    UBYTE * s )
```

Definition at line 3436 of file [compcomm.c](#).

4.9.2.76 CoTryReplace()

```
int CoTryReplace (
    UBYTE * p )
```

Definition at line 3450 of file [compcomm.c](#).

4.9.2.77 CoModulus()

```
int CoModulus (
    UBYTE * inp )
```

Definition at line 3553 of file [compcomm.c](#).

4.9.2.78 CoRepeat()

```
int CoRepeat (
    UBYTE * inp )
```

Definition at line 3691 of file [compcomm.c](#).

4.9.2.79 CoEndRepeat()

```
int CoEndRepeat (
    UBYTE * inp )
```

Definition at line [3714](#) of file [compcomm.c](#).

4.9.2.80 DoBrackets()

```
int DoBrackets (
    UBYTE * inp,
    int par )
```

Definition at line [3752](#) of file [compcomm.c](#).

4.9.2.81 CoBracket()

```
int CoBracket (
    UBYTE * inp )
```

Definition at line [3879](#) of file [compcomm.c](#).

4.9.2.82 CoAntiBracket()

```
int CoAntiBracket (
    UBYTE * inp )
```

Definition at line [3887](#) of file [compcomm.c](#).

4.9.2.83 CoMultiBracket()

```
int CoMultiBracket (
    UBYTE * inp )
```

Definition at line [3898](#) of file [compcomm.c](#).

4.9.2.84 CountComp()

```
WORD * CountComp (
    UBYTE * inp,
    WORD * to )
```

Definition at line [4030](#) of file [compcomm.c](#).

4.9.2.85 Colf()

```
int CoIf (
    UBYTE * inp )
```

Definition at line [4220](#) of file [compcomm.c](#).

4.9.2.86 CoElse()

```
int CoElse (
    UBYTE * p )
```

Definition at line 4862 of file [compcomm.c](#).

4.9.2.87 CoElseIf()

```
int CoElseIf (
    UBYTE * inp )
```

Definition at line 4889 of file [compcomm.c](#).

4.9.2.88 CoEndIf()

```
int CoEndIf (
    UBYTE * inp )
```

Definition at line 4916 of file [compcomm.c](#).

4.9.2.89 CoWhile()

```
int CoWhile (
    UBYTE * inp )
```

Definition at line 4961 of file [compcomm.c](#).

4.9.2.90 CoEndWhile()

```
int CoEndWhile (
    UBYTE * inp )
```

Definition at line 4982 of file [compcomm.c](#).

4.9.2.91 DoFindLoop()

```
int DoFindLoop (
    UBYTE * inp,
    int mode )
```

Definition at line 5010 of file [compcomm.c](#).

4.9.2.92 CoFindLoop()

```
int CoFindLoop (
    UBYTE * inp )
```

Definition at line 5127 of file [compcomm.c](#).

4.9.2.93 CoReplaceLoop()

```
int CoReplaceLoop (
    UBYTE * inp )
```

Definition at line [5135](#) of file [compcomm.c](#).

4.9.2.94 CoFunPowers()

```
int CoFunPowers (
    UBYTE * inp )
```

Definition at line [5155](#) of file [compcomm.c](#).

4.9.2.95 CoUnitTrace()

```
int CoUnitTrace (
    UBYTE * s )
```

Definition at line [5182](#) of file [compcomm.c](#).

4.9.2.96 CoTerm()

```
int CoTerm (
    UBYTE * s )
```

Definition at line [5217](#) of file [compcomm.c](#).

4.9.2.97 CoEndTerm()

```
int CoEndTerm (
    UBYTE * s )
```

Definition at line [5265](#) of file [compcomm.c](#).

4.9.2.98 CoSort()

```
int CoSort (
    UBYTE * s )
```

Definition at line [5292](#) of file [compcomm.c](#).

4.9.2.99 CoPolyFun()

```
int CoPolyFun (
    UBYTE * s )
```

Definition at line [5331](#) of file [compcomm.c](#).

4.9.2.100 CoPolyRatFun()

```
int CoPolyRatFun (
    UBYTE * s )
```

Definition at line 5378 of file [compcomm.c](#).

4.9.2.101 CoMerge()

```
int CoMerge (
    UBYTE * inp )
```

Definition at line 5548 of file [compcomm.c](#).

4.9.2.102 CoStuffle()

```
int CoStuffle (
    UBYTE * inp )
```

Definition at line 5604 of file [compcomm.c](#).

4.9.2.103 CoProcessBucket()

```
int CoProcessBucket (
    UBYTE * s )
```

Definition at line 5659 of file [compcomm.c](#).

4.9.2.104 CoThreadBucket()

```
int CoThreadBucket (
    UBYTE * s )
```

Definition at line 5677 of file [compcomm.c](#).

4.9.2.105 DoArgPlode()

```
int DoArgPlode (
    UBYTE * s,
    int par )
```

Definition at line 5707 of file [compcomm.c](#).

4.9.2.106 CoArgExplode()

```
int CoArgExplode (
    UBYTE * s )
```

Definition at line 5763 of file [compcomm.c](#).

4.9.2.107 CoArgImplode()

```
int CoArgImplode (
    UBYTE * s )
```

Definition at line 5770 of file [compcomm.c](#).

4.9.2.108 CoClearTable()

```
int CoClearTable (
    UBYTE * s )
```

Definition at line 5777 of file [compcomm.c](#).

4.9.2.109 CoDenominators()

```
int CoDenominators (
    UBYTE * s )
```

Definition at line 5859 of file [compcomm.c](#).

4.9.2.110 CoDropCoefficient()

```
int CoDropCoefficient (
    UBYTE * s )
```

Definition at line 5888 of file [compcomm.c](#).

4.9.2.111 CoDropSymbols()

```
int CoDropSymbols (
    UBYTE * s )
```

Definition at line 5902 of file [compcomm.c](#).

4.9.2.112 CoToPolynomial()

```
int CoToPolynomial (
    UBYTE * inp )
```

Definition at line 5925 of file [compcomm.c](#).

4.9.2.113 CoFromPolynomial()

```
int CoFromPolynomial (
    UBYTE * inp )
```

Definition at line 6000 of file [compcomm.c](#).

4.9.2.114 CoArgToExtraSymbol()

```
int CoArgToExtraSymbol (
    UBYTE * s )
```

Definition at line [6026](#) of file [compcomm.c](#).

4.9.2.115 CoExtraSymbols()

```
int CoExtraSymbols (
    UBYTE * inp )
```

Definition at line [6076](#) of file [compcomm.c](#).

4.9.2.116 GetIfDollarFactor()

```
WORD * GetIfDollarFactor (
    UBYTE ** inp,
    WORD * w )
```

Definition at line [6149](#) of file [compcomm.c](#).

4.9.2.117 GetDoParam()

```
UBYTE * GetDoParam (
    UBYTE * inp,
    WORD ** wp,
    int par )
```

Definition at line [6207](#) of file [compcomm.c](#).

4.9.2.118 CoDo()

```
int CoDo (
    UBYTE * inp )
```

Definition at line [6284](#) of file [compcomm.c](#).

4.9.2.119 CoEndDo()

```
int CoEndDo (
    UBYTE * inp )
```

Definition at line [6387](#) of file [compcomm.c](#).

4.9.2.120 CoFactDollar()

```
int CoFactDollar (
    UBYTE * inp )
```

Definition at line 6418 of file [compcomm.c](#).

4.9.2.121 CoFactorize()

```
int CoFactorize (
    UBYTE * s )
```

Definition at line 6447 of file [compcomm.c](#).

4.9.2.122 CoNFactorize()

```
int CoNFactorize (
    UBYTE * s )
```

Definition at line 6454 of file [compcomm.c](#).

4.9.2.123 CoUnFactorize()

```
int CoUnFactorize (
    UBYTE * s )
```

Definition at line 6461 of file [compcomm.c](#).

4.9.2.124 CoNUnFactorize()

```
int CoNUnFactorize (
    UBYTE * s )
```

Definition at line 6468 of file [compcomm.c](#).

4.9.2.125 DoFactorize()

```
int DoFactorize (
    UBYTE * s,
    int par )
```

Definition at line 6475 of file [compcomm.c](#).

4.9.2.126 CoOptimizeOption()

```
int CoOptimizeOption (
    UBYTE * s )
```

Definition at line 6608 of file [compcomm.c](#).

4.9.2.127 CoPutInside()

```
int CoPutInside (
    UBYTE * inp )
```

Definition at line 7044 of file [compcomm.c](#).

4.9.2.128 CoAntiPutInside()

```
int CoAntiPutInside (
    UBYTE * inp )
```

Definition at line 7045 of file [compcomm.c](#).

4.9.2.129 DoPutInside()

```
int DoPutInside (
    UBYTE * inp,
    int par )
```

Definition at line 7047 of file [compcomm.c](#).

4.9.2.130 CoSwitch()

```
int CoSwitch (
    UBYTE * s )
```

Definition at line 7167 of file [compcomm.c](#).

4.9.2.131 CoCase()

```
int CoCase (
    UBYTE * s )
```

Definition at line 7213 of file [compcomm.c](#).

4.9.2.132 CoBreak()

```
int CoBreak (
    UBYTE * s )
```

Definition at line 7263 of file [compcomm.c](#).

4.9.2.133 CoDefault()

```
int CoDefault (
    UBYTE * s )
```

Definition at line 7289 of file [compcomm.c](#).

4.9.2.134 CoEndSwitch()

```
int CoEndSwitch (
    UBYTE * s )
```

Definition at line [7316](#) of file [compcomm.c](#).

4.9.2.135 CoSetUserFlag()

```
int CoSetUserFlag (
    UBYTE * s )
```

Definition at line [7384](#) of file [compcomm.c](#).

4.9.2.136 CoClearUserFlag()

```
int CoClearUserFlag (
    UBYTE * s )
```

Definition at line [7412](#) of file [compcomm.c](#).

4.9.2.137 CoCreateAllLoops()

```
int CoCreateAllLoops (
    UBYTE * s )
```

Definition at line [7449](#) of file [compcomm.c](#).

4.9.2.138 CoCreateAllPaths()

```
int CoCreateAllPaths (
    UBYTE * s )
```

Definition at line [7569](#) of file [compcomm.c](#).

4.9.2.139 CoCreateAll()

```
int CoCreateAll (
    UBYTE * s )
```

Definition at line [7697](#) of file [compcomm.c](#).

4.10 compcomm.c

[Go to the documentation of this file.](#)

```

00001
00010 /* #[ License : */
00011 /*
00012 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00013 *   When using this file you are requested to refer to the publication
00014 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00015 *   This is considered a matter of courtesy as the development was paid
00016 *   for by FOM the Dutch physics granting agency and we would like to
00017 *   be able to track its scientific use to convince FOM of its value
00018 *   for the community.
00019 *
00020 *   This file is part of FORM.
00021 *
00022 *   FORM is free software: you can redistribute it and/or modify it under the
00023 *   terms of the GNU General Public License as published by the Free Software
00024 *   Foundation, either version 3 of the License, or (at your option) any later
00025 *   version.
00026 *
00027 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00028 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00029 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00030 *   details.
00031 *
00032 *   You should have received a copy of the GNU General Public License along
00033 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00034 */
00035 /* #] License : */
00036 /*
00037 *   #[ includes :
00038 */
00039
00040 #include "form3.h"
00041 #include "comtool.h"
00042 #ifdef WITHFLOAT
00043 #include <gmp.h>
00044 #endif
00045
00046 static KEYWORD formatoptions[] = {
00047     {"allfloat",      (TFUN)0,    ALLINTEGERDOUBLE,    0}
00048     ,{"c",            (TFUN)0,    CMODE,                0}
00049     ,{"doublefortran", (TFUN)0,    DOUBLEFORTRANMODE,  0}
00050     ,{"float",         (TFUN)0,    0,                    2}
00051 #ifdef WITHFLOAT
00052     ,{"floatprecision", (TFUN)0,    0,                    5}
00053 #endif
00054     ,{"fortran",       (TFUN)0,    FORTRANMODE,          0}
00055     ,{"fortran90",     (TFUN)0,    FORTRANMODE,          4}
00056     ,{"maple",         (TFUN)0,    MAPLEMODE,            0}
00057     ,{"mathematica",   (TFUN)0,    MATHEMATICAMODE,      0}
00058     ,{"normal",        (TFUN)0,    NORMALFORMAT,         1}
00059     ,{"nospaces",      (TFUN)0,    NOSPACEFORMAT,        3}
00060     ,{"pfortran",      (TFUN)0,    PFORTRANMODE,         0}
00061     ,{"quadfortran",   (TFUN)0,    QUADRUPLEFORTRANMODE, 0}
00062     ,{"quadruplefortran", (TFUN)0,    QUADRUPLEFORTRANMODE, 0}
00063     ,{"rational",      (TFUN)0,    RATIONALMODE,         1}
00064     ,{"reduce",        (TFUN)0,    REDUCEMODE,           0}
00065     ,{"spaces",        (TFUN)0,    NORMALFORMAT,         3}
00066     ,{"vortran",       (TFUN)0,    VORTRANMODE,          0}
00067 };
00068
00069 static KEYWORD trace4options[] = {
00070     {"contract",       (TFUN)0,    CHISHOLM,             0}
00071     ,{"nocontract",    (TFUN)0,    0,                    CHISHOLM}
00072     ,{"nosymmetrize",  (TFUN)0,    0,                    ALSOREVERSE}
00073     ,{"notrick",       (TFUN)0,    NOTRICK,              0}
00074     ,{"symmetrize",    (TFUN)0,    ALSOREVERSE,          0}
00075     ,{"trick",         (TFUN)0,    0,                    NOTRICK}
00076 };
00077
00078 static KEYWORD chisoptions[] = {
00079     {"nosymmetrize", (TFUN)0,    0,                    ALSOREVERSE}
00080     ,{"symmetrize",  (TFUN)0,    ALSOREVERSE,          0}
00081 };
00082
00083 static KEYWORDV writeoptions[] = {
00084     {"stats",          &(AC.StatsFlag), 1,            0}
00085     ,{"statistics",    &(AC.StatsFlag), 1,            0}
00086     ,{"shortstats",    &(AC.ShortStats), 1,           0}
00087     ,{"shortstatistics",&(AC.ShortStats), 1,           0}
00088     ,{"warnings",      &(AC.WarnFlag), 1,            0}
00089     ,{"allwarnings",   &(AC.WarnFlag), 2,            0}
00090     ,{"setup",         &(AC.SetupFlag), 1,           0}

```

```

00091     ,{"names",          &(AC.NamesFlag),    1,      0}
00092     ,{"allnames",       &(AC.NamesFlag),    2,      0}
00093     ,{"codes",          &(AC.CodesFlag),    1,      0}
00094     ,{"highfirst",      &(AC.SortType),    SORTHIGHFIRST,  SORTLOWFIRST}
00095     ,{"lowfirst",       &(AC.SortType),    SORTLOWFIRST,  SORTHIGHFIRST}
00096     ,{"powerfirst",     &(AC.SortType),    SORTPOWERFIRST, SORTHIGHFIRST}
00097     ,{"tokens",         &(AC.TokensWriteFlag),1,      0}
00098 };
00099
00100 static KEYWORDV onoffoptions[] = {
00101     {"compress",        &(AC.NoCompress),  0,      1}
00102     ,{"checkpoint",     &(AC.CheckpointFlag), 1,      0}
00103     ,{"insidefirst",    &(AC.insidefirst), 1,      0}
00104     ,{"propercount",    &(AC.BottomLevel), 1,      0}
00105     ,{"stats",          &(AC.StatsFlag),    1,      0}
00106     ,{"statistics",     &(AC.StatsFlag),    1,      0}
00107     ,{"shortstats",     &(AC.ShortStats),  1,      0}
00108     ,{"shortstatistics",&(AC.ShortStats),  1,      0}
00109     ,{"names",          &(AC.NamesFlag),    1,      0}
00110     ,{"allnames",       &(AC.NamesFlag),    2,      0}
00111     ,{"warnings",       &(AC.WarnFlag),    1,      0}
00112     ,{"allwarnings",   &(AC.WarnFlag),    2,      0}
00113     ,{"highfirst",      &(AC.SortType),    SORTHIGHFIRST,  SORTLOWFIRST}
00114     ,{"lowfirst",       &(AC.SortType),    SORTLOWFIRST,  SORTHIGHFIRST}
00115     ,{"powerfirst",     &(AC.SortType),    SORTPOWERFIRST, SORTHIGHFIRST}
00116     ,{"setup",          &(AC.SetupFlag),    1,      0}
00117     ,{"codes",          &(AC.CodesFlag),    1,      0}
00118     ,{"tokens",         &(AC.TokensWriteFlag),1,0}
00119     ,{"properorder",    &(AC.properorderflag),1,0}
00120     ,{"threadloadbalancing",&(AC.ThreadBalancing),1, 0}
00121     ,{"threads",        &(AC.ThreadsFlag),1, 0}
00122     ,{"threadsortfilesynch",&(AC.ThreadSortFileSynch),1, 0}
00123     ,{"threadstats",    &(AC.ThreadStats),1, 0}
00124     ,{"finalstats",     &(AC.FinalStats),1, 0}
00125     ,{"fewerstats",     &(AC.ShortStatsMax), 10, 0}
00126     ,{"fewerstatistics",&(AC.ShortStatsMax), 10, 0}
00127     ,{"processstats",   &(AC.ProcessStats),1, 0}
00128     ,{"oldparallelstats",&(AC.OldParallelStats),1,0}
00129     ,{"parallel",       &(AC.parallelflag),PARALLELFLAG,NOPARALLEL_USER}
00130     ,{"nospacesinnumbers",&(AO.NoSpacesInNumbers),1,0}
00131     ,{"indent",         &(AO.IndentSpace),INDENTSPACE,0}
00132     ,{"totalsize",      &(AM.PrintTotalSize), 1, 0}
00133     ,{"flag",           (int *)&(AC.debugFlags), 1, 0}
00134     ,{"oldfactarg",     &(AC.OldFactArgFlag), 1, 0}
00135     ,{"memdebugflag",   &(AC.MemDebugFlag), 1, 0}
00136     ,{"oldgcd",         &(AC.OldGCDflag), 1, 0}
00137     ,{"innertest",     &(AC.InnerTest), 1, 0}
00138     ,{"wtimestats",    &(AC.WTimeStatsFlag), 1, 0}
00139     ,{"sortreallocate", &(AC.SortReallocateFlag), 1, 0}
00140     ,{"backtrace",     &(AC.PrintBacktraceFlag), 1, 0}
00141     ,{"flint",         &(AC.FlintPolyFlag), 1, 0}
00142     ,{"humanstats",    &(AC.HumanStatsFlag), 1, 0}
00143     ,{"humanstatistics",&(AC.HumanStatsFlag), 1, 0}
00144 };
00145
00146 static WORD one = 1;
00147
00148 /*
00149  #] includes :
00150  #[ CoFormat :
00151  */
00152
00153 int CoFormat(UBYTE *s)
00154 {
00155     int error = 0, x;
00156     KEYWORD *key;
00157     UBYTE *ss;
00158     while ( *s == ' ' || *s == ',' ) s++;
00159     if ( *s == 0 ) {
00160         AC.OutputMode = 72;
00161         AC.OutputSpaces = NORMALFORMAT;
00162         return(error);
00163     }
00164     /*
00165     First the optimization level
00166     */
00167     if ( *s == 'O' || *s == 'o' ) {
00168         if ( ( FG.cTable[s[1]] == 1 ) ||
00169             ( s[1] == '=' && FG.cTable[s[2]] == 1 ) ) {
00170             s++; if ( *s == '=' ) s++;
00171             x = 0;
00172             while ( *s >= '0' && *s <= '9' ) x = 10*x + *s++ - '0';
00173             while ( *s == ',' ) s++;
00174             AO.OptimizationLevel = x;
00175             AO.Optimize.greedytimelimit = 0;
00176             AO.Optimize.mctstimelimit = 0;
00177             AO.Optimize.printstats = 0;

```

```

00178     AO.Optimize.debugflags = 0;
00179     AO.Optimize.schemeflags = 0;
00180     AO.Optimize.mctsdecaymode = 1; /* default is decreasing C_p with iteration number */
00181     if ( AO.inscheme ) {
00182         M_free(AO.inscheme,"Horner input scheme");
00183         AO.inscheme = 0; AO.schemenum = 0;
00184     }
00185     switch ( x ) {
00186     case 0:
00187         break;
00188     case 1:
00189         AO.Optimize.mctsconstant.fval = -1.0;
00190         AO.Optimize.horner = O_OCCURRENCE;
00191         AO.Optimize.hornerdirection = O_FORWARDORBACKWARD;
00192         AO.Optimize.method = O_CSE;
00193         break;
00194     case 2:
00195         AO.Optimize.horner = O_OCCURRENCE;
00196         AO.Optimize.hornerdirection = O_FORWARDORBACKWARD;
00197         AO.Optimize.method = O_GREEDY;
00198         AO.Optimize.greedyminnum = 10;
00199         AO.Optimize.greedymaxperc = 5;
00200         break;
00201     case 3:
00202         AO.Optimize.mctsconstant.fval = 1.0;
00203         AO.Optimize.horner = O_MCTS;
00204         AO.Optimize.hornerdirection = O_FORWARDORBACKWARD;
00205         AO.Optimize.method = O_GREEDY;
00206         AO.Optimize.mctsnumexpand = 1000;
00207         AO.Optimize.mctsnumkeep = 10;
00208         AO.Optimize.mctsnumrepeat = 1;
00209         AO.Optimize.greedyminnum = 10;
00210         AO.Optimize.greedymaxperc = 5;
00211         break;
00212     case 4:
00213         AO.Optimize.horner = O_SIMULATED_ANNEALING;
00214         AO.Optimize.saIter = 1000;
00215         AO.Optimize.saMaxT.fval = 2000;
00216         AO.Optimize.saMinT.fval = 1;
00217         break;
00218     default:
00219         error = 1;
00220         MesPrint("&Illegal optimization specification in format statement");
00221         break;
00222     }
00223     if ( error == 0 && *s != 0 && x > 0 ) return(CoOptimizeOption(s));
00224     return(error);
00225 }
00226 #ifndef EXPOPT
00227 { UBYTE c;
00228   ss = s;
00229   while ( FG.cTable[*s] == 0 ) s++;
00230   c = *s; *s = 0;
00231   if ( StrICont(ss,(UBYTE *)"optimize") == 0 ) {
00232       *s = c;
00233       while ( *s == ',' ) s++;
00234       if ( *s == '=' ) s++;
00235       AO.OptimizationLevel = 3;
00236       AO.Optimize.mctsconstant.fval = 1.0;
00237       AO.Optimize.horner = O_MCTS;
00238       AO.Optimize.hornerdirection = O_FORWARDORBACKWARD;
00239       AO.Optimize.method = O_GREEDY;
00240       AO.Optimize.mctstimelimit = 0;
00241       AO.Optimize.mctsnumexpand = 1000;
00242       AO.Optimize.mctsnumkeep = 10;
00243       AO.Optimize.mctsnumrepeat = 1;
00244       AO.Optimize.greedytimelimit = 0;
00245       AO.Optimize.greedyminnum = 10;
00246       AO.Optimize.greedymaxperc = 5;
00247       AO.Optimize.printstats = 0;
00248       AO.Optimize.debugflags = 0;
00249       AO.Optimize.schemeflags = 0;
00250       AO.Optimize.mctsdecaymode = 1;
00251       if ( AO.inscheme ) {
00252           M_free(AO.inscheme,"Horner input scheme");
00253           AO.inscheme = 0; AO.schemenum = 0;
00254       }
00255       return(CoOptimizeOption(s));
00256   }
00257   else {
00258       error = 1;
00259       MesPrint("&Illegal optimization specification in format statement");
00260       return(error);
00261   }
00262 }
00263 #endif
00264 }

```

```

00265     else if ( FG.cTable[*s] == 1 ) {
00266         x = 0;
00267         while ( FG.cTable[*s] == 1 ) x = 10*x + *s++ - '0';
00268         if ( x <= 0 || x >= MAXLINELENGTH ) {
00269             error = 1;
00270             MesPrint("&Illegal value for linesize: %d",x);
00271             x = 72;
00272         }
00273         if ( x < 39 ) {
00274             MesPrint(" ... Too small value for linesize corrected to 39");
00275             x = 39;
00276         }
00277         AO.DoubleFlag = 0;
00278     /*
00279     The next line resets the mode to normal. Because the special modes
00280     reset the line length we have a little problem with the special modes
00281     and customized line length. We try to improve by removing the next line
00282     */
00283     /*
00284     AC.OutputMode = 0;
00285     AC.LineLength = x;
00286     if ( *s != 0 ) {
00287         error = 1;
00288         MesPrint("&Illegal linesize field in format statement");
00289     }
00290     else {
00291         key = FindKeyWord(s,formatoptions,
00292             sizeof(formatoptions)/sizeof(KEYWORD));
00293         if ( key ) {
00294             if ( key->type == FORTRANMODE || key->type == PFORTRANMODE || key->type ==
DOUBLEFORTRANMODE
00295                 || key->type == QUADRUPLEFORTRANMODE || key->type == VORTRANMODE ) {
00296                 if ( AC.LineLength > 72 ) AC.LineLength = 72;
00297             }
00298             if ( key->flags == 0 ) {
00299                 if ( key->type == FORTRANMODE || key->type == PFORTRANMODE
00300                     || key->type == DOUBLEFORTRANMODE || key->type == ALLINTEGERDOUBLE
00301                     || key->type == QUADRUPLEFORTRANMODE || key->type == VORTRANMODE ) {
00302                     AC.IsFortran90 = ISNOTFORTRAN90;
00303                     if ( AC.Fortran90Kind ) {
00304                         M_free(AC.Fortran90Kind,"Fortran90 Kind");
00305                         AC.Fortran90Kind = 0;
00306                     }
00307                 }
00308                 if ( ( key->type == ALLINTEGERDOUBLE ) && AO.DoubleFlag != 0 ) {
00309                     AO.DoubleFlag |= 4;
00310                 }
00311                 else {
00312                     AO.DoubleFlag = 0;
00313                     AC.OutputMode = key->type & NODOUBLEMASK;
00314                     if ( ( key->type & DOUBLEPRECISIONFLAG ) != 0 ) {
00315                         AO.DoubleFlag = 1;
00316                     }
00317                     else if ( ( key->type & QUADRUPLEPRECISIONFLAG ) != 0 ) {
00318                         AO.DoubleFlag = 2;
00319                     }
00320                 }
00321             }
00322             else if ( key->flags == 1 ) {
00323                 AC.OutputMode = AC.OutNumberType = key->type;
00324             }
00325             else if ( key->flags == 2 ) {
00326                 while ( FG.cTable[*s] == 0 ) s++;
00327                 if ( *s == 0 ) AC.OutNumberType = 10;
00328                 else if ( *s == ',' ) {
00329                     s++;
00330                     x = 0;
00331                     while ( FG.cTable[*s] == 1 ) x = 10*x + *s++ - '0';
00332                     if ( *s != 0 ) {
00333                         error = 1;
00334                         MesPrint("&Illegal float format specifier");
00335                     }
00336                     else {
00337                         if ( x < 3 ) {
00338                             x = 3;
00339                             MesPrint("& ... float format value corrected to 3");
00340                         }
00341                         if ( x > 100 ) {
00342                             x = 100;
00343                             MesPrint("& ... float format value corrected to 100");
00344                         }
00345                         AC.OutNumberType = x;
00346                     }
00347                 }
00348             }
00349             else if ( key->flags == 3 ) {
00350                 AC.OutputSpaces = key->type;

```

```

00351     }
00352     else if ( key->flags == 4 ) {
00353         AC.IsFortran90 = ISFORTRAN90;
00354         if ( AC.Fortran90Kind ) {
00355             M_free(AC.Fortran90Kind,"Fortran90 Kind");
00356             AC.Fortran90Kind = 0;
00357         }
00358         while ( FG.cTable[*s] <= 1 ) s++;
00359         if ( *s == ',' ) {
00360             s++; ss = s;
00361             while ( *ss && *ss != ',' ) ss++;
00362             if ( *ss == ',' ) {
00363                 MesPrint("&No white space or comma's allowed in Fortran90 option: %s",s);
00364             }
00365             else {
00366                 AC.Fortran90Kind = strDup1(s,"Fortran90 Kind");
00367             }
00368         }
00369         AO.DoubleFlag = 0;
00370         AC.OutputMode = key->type & NODOUBLEMASK;
00371     }
00372 #ifdef WITHFLOAT
00373     else if ( key->flags == 5 ) {
00374         /*
00375             Syntax: Format FloatPrecision number;
00376             Format FloatPrecision off;
00377         */
00378         while ( FG.cTable[*s] == 0 ) s++;
00379         while ( *s == ' ' || *s == '\t' || *s == ',' ) s++;
00380         if ( *s == 0 ) {
00381             AO.FloatPrec = 0;
00382         }
00383         else if ( tolower(*s) == 'o' && tolower(s[1]) == 'f'
00384             && tolower(s[2]) == 'f' ) {
00385             ss = s;
00386             s += 3;
00387             while ( *s == ' ' || *s == '\t' || *s == ',' ) s++;
00388             if ( *s ) { s = ss; goto WrongOption; }
00389             AO.FloatPrec = -1;
00390         }
00391         else if ( FG.cTable[*s] == 1 ) {
00392             ss = s;
00393             AO.FloatPrec = 0;
00394             while ( *s <= '9' && *s >= '0' )
00395                 AO.FloatPrec = 10*AO.FloatPrec + (*s++ - '0');
00396             while ( *s == ' ' || *s == '\t' || *s == ',' ) s++;
00397             if ( *s ) { s = ss; goto WrongOption; }
00398         }
00399         else {
00400 WrongOption: MesPrint("&Illegal option in Format FloatPrecision: %s",s);
00401             error = 1;
00402         }
00403     }
00404 #endif
00405 }
00406 else if ( ( *s == 'c' || *s == 'C' ) && ( FG.cTable[s[1]] == 1 ) ) {
00407     UBYTE *ss = s+1;
00408     WORD x = 0;
00409     while ( *ss >= '0' && *ss <= '9' ) x = 10*x + *ss++ - '0';
00410     if ( *ss != 0 ) goto Unknown;
00411     AC.OutputMode = CMODE;
00412     AC.Cnumpows = x;
00413 }
00414 else {
00415 Unknown: MesPrint("&Unknown option: %s",s); error = 1;
00416 }
00417 }
00418 return(error);
00419 }
00420
00421 /*
00422 #] CoFormat :
00423 #[ CoCollect :
00424
00425 Collect,functionname
00426 */
00427
00428 int CoCollect(UBYTE *s)
00429 {
00430     /* -----change 17-feb-2003 Added percentage */
00431     WORD numfun;
00432     int type,x = 0;
00433     UBYTE *t = SkipAName(s), *t1, *t2;
00434     AC.AltCollectFun = 0;
00435     if ( t == 0 ) goto syntaxerror;
00436     t1 = t; while ( *t1 == ',' || *t1 == ' ' || *t1 == '\t' ) t1++;

```

```

00437     *t = 0; t = t1;
00438     if ( *t1 && ( FG.cTable[*t1] == 0 || *t1 == '[' ) ) {
00439         t2 = SkipAName(t1);
00440         if ( t2 == 0 ) goto syntaxerror;
00441         t = t2;
00442         while ( *t == ',' || *t == ' ' || *t == '\\t' ) t++;
00443         *t2 = 0;
00444     }
00445     else t1 = 0;
00446     if ( *t && FG.cTable[*t] == 1 ) {
00447         while ( *t >= '0' && *t <= '9' ) x = 10*x + *t++ - '0';
00448         if ( x > 100 ) x = 100;
00449         while ( *t == ',' || *t == ' ' || *t == '\\t' ) t++;
00450         if ( *t ) goto syntaxerror;
00451     }
00452     else {
00453         if ( *t ) goto syntaxerror;
00454         x = 100;
00455     }
00456     if ( ( ( type = GetName(AC.varnames,s,&numfun,WITHAUTO) ) != CFUNCTION )
00457     || ( functions[numfun].spec != 0 ) ) {
00458         MesPrint("%s should be a regular function",s);
00459         if ( type < 0 ) {
00460             if ( GetName(AC.exprnames,s,&numfun,NOAUTO) == NAMENOTFOUND )
00461                 AddFunction(s,0,0,0,0,0,-1,-1);
00462         }
00463         return(1);
00464     }
00465     AC.CollectFun = numfun+FUNCTION;
00466     AC.CollectPercentage = (WORD)x;
00467     if ( t1 ) {
00468         if ( ( ( type = GetName(AC.varnames,t1,&numfun,WITHAUTO) ) != CFUNCTION )
00469         || ( functions[numfun].spec != 0 ) ) {
00470             MesPrint("%s should be a regular function",t1);
00471             if ( type < 0 ) {
00472                 if ( GetName(AC.exprnames,t1,&numfun,NOAUTO) == NAMENOTFOUND )
00473                     AddFunction(t1,0,0,0,0,0,-1,-1);
00474             }
00475             return(1);
00476         }
00477         AC.AltCollectFun = numfun+FUNCTION;
00478     }
00479     return(0);
00480 syntaxerror:
00481     MesPrint("&Collect statement needs one or two functions (and a percentage) for its argument(s)");
00482     return(1);
00483 }
00484
00485 /*
00486 #] CoCollect :
00487 #[ setonoff :
00488 */
00489
00490 int setonoff(UBYTE *s, int *flag, int onvalue, int offvalue)
00491 {
00492     if ( StrICmp(s,(UBYTE *)"on") == 0 ) *flag = onvalue;
00493     else if ( StrICmp(s,(UBYTE *)"off") == 0 ) *flag = offvalue;
00494     else {
00495         MesPrint("&Unknown option: %s, on or off expected",s);
00496         return(1);
00497     }
00498     return(0);
00499 }
00500
00501 /*
00502 #] setonoff :
00503 #[ CoCompress :
00504 */
00505
00506 int CoCompress(UBYTE *s)
00507 {
00508     GETIDENTITY
00509     UBYTE *t, c;
00510     if ( StrICmp(s,(UBYTE *)"on") == 0 ) {
00511         AC.NoCompress = 0;
00512         AR.gzipCompress = 0;
00513     }
00514     else if ( StrICmp(s,(UBYTE *)"off") == 0 ) {
00515         AC.NoCompress = 1;
00516         AR.gzipCompress = 0;
00517     }
00518     else {
00519         t = s; while ( FG.cTable[*t] <= 1 ) t++;
00520         c = *t; *t = 0;
00521         if ( StrICmp(s,(UBYTE *)"gzip") == 0 ) {
00522 #ifndef WITHZLIB
00523             Warning("gzip compression not supported on this platform");

```



```

00524 #endif
00525     s = t; *s = c;
00526     if ( *s == 0 ) {
00527         AR.gzipCompress = GZIPDEFAULT; /* Normally should be 6 */
00528         return(0);
00529     }
00530     while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
00531     t = s;
00532     if ( FG.cTable[*s] == 1 ) {
00533         AR.gzipCompress = *s - '0';
00534         s++;
00535         while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
00536         if ( *s == 0 ) return(0);
00537     }
00538     MesPrint("&Unknown gzip option: %s, a digit was expected",t);
00539     return(1);
00540
00541 }
00542 else {
00543     MesPrint("&Unknown option: %s, on, off or gzip expected",s);
00544     return(1);
00545 }
00546 }
00547 return(0);
00548 }
00549
00550 /*
00551  #] CoCompress :
00552  #[ CoFlags :
00553  */
00554
00555 int CoFlags(UBYTE *s,int value)
00556 {
00557     int i, error = 0;
00558     if ( *s != ',' ) {
00559         MesPrint("&Proper syntax is: On/Off Flag,number[s];");
00560         error = 1;
00561     }
00562     while ( *s == ',' ) {
00563         do { s++; } while ( *s == ',' );
00564         i = 0;
00565         if ( FG.cTable[*s] != 1 ) {
00566             MesPrint("&Proper syntax is: On/Off Flag,number[s];");
00567             error = 1;
00568             break;
00569         }
00570         while ( FG.cTable[*s] == 1 ) { i = 10*i + *s++ - '0'; }
00571         if ( i <= 0 || i > MAXFLAGS ) {
00572             MesPrint("&The number of a flag in On/Off Flag should be in the range
00573 0-%d", (int)MAXFLAGS);
00574             error = 1;
00575             break;
00576         }
00577         AC.debugFlags[i] = value;
00578     }
00579     if ( *s ) {
00580         MesPrint("&Proper syntax is: On/Off Flag,number[s];");
00581         error = 1;
00582     }
00583     return(error);
00584 }
00585 /*
00586  #] CoFlags :
00587  #[ CoOff :
00588  */
00589
00590 int CoOff(UBYTE *s)
00591 {
00592     GETIDENTITY
00593     UBYTE *t, c;
00594     int i, num = sizeof(onoffoptions)/sizeof(KEYWORD);
00595     for (;;) {
00596         while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
00597         if ( *s == 0 ) return(0);
00598         if ( chartype[*s] != 0 ) {
00599             MesPrint("&Illegal character or option encountered in OFF statement");
00600             return(-1);
00601         }
00602         t = s; while ( chartype[*s] == 0 ) s++;
00603         c = *s; *s = 0;
00604         for ( i = 0; i < num; i++ ) {
00605             if ( StrICont(t, (UBYTE *) (onoffoptions[i].name)) == 0 ) break;
00606         }
00607         if ( i >= num ) {
00608             MesPrint("&Unrecognized option in OFF statement: %s",t);
00609             *s = c; return(-1);

```

```

00610     }
00611     else if ( StrICont(t, (UBYTE *)"compress") == 0 ) {
00612         AR.gzipCompress = 0;
00613     }
00614     else if ( StrICont(t, (UBYTE *)"checkpoint") == 0 ) {
00615         PrintDeprecation("the checkpoint mechanism", "issues/626");
00616         AC.CheckpointInterval = 0;
00617         if ( AC.CheckpointRunBefore ) { free(AC.CheckpointRunBefore); AC.CheckpointRunBefore =
NULL; }
00618         if ( AC.CheckpointRunAfter ) { free(AC.CheckpointRunAfter); AC.CheckpointRunAfter = NULL;
}
00619         if ( AC.NoShowInput == 0 ) MesPrint("Checkpoints deactivated.");
00620     }
00621     else if ( StrICont(t, (UBYTE *)"threads") == 0 ) {
00622         AS.MultiThreaded = 0;
00623     }
00624     else if ( StrICont(t, (UBYTE *)"flag") == 0 ) {
00625         *s = c;
00626         return(CoFlags(s, 0));
00627     }
00628     else if ( StrICont(t, (UBYTE *)"innertest") == 0 ) {
00629         *s = c;
00630         AC.InnerTest = 0;
00631         if ( AC.TestValue ) {
00632             M_free(AC.TestValue, "InnerTest");
00633             AC.TestValue = 0;
00634         }
00635     }
00636     else if ( StrICont(t, (UBYTE *)"sortreallocate") == 0 ) {
00637         if ( AC.SortReallocateFlag == 2 ) {
00638             /* The flag has been set by #sortreallocate, and it was off before. Leave it as 2,
00639              so that the reallocation still happens in the current module. It will be turned
00640              off after the reallocation is done. */
00641             return(0);
00642         }
00643     }
00644     *s = c;
00645     *onoffoptions[i].var = onoffoptions[i].flags;
00646     AR.SortType = AC.SortType;
00647     AC.mparallelfalg = AC.parallelfalg | AM.hparallelfalg;
00648 }
00649 }
00650
00651 /*
00652 #] CoOff :
00653 #[ CoOn :
00654 */
00655
00656 int CoOn(UBYTE *s)
00657 {
00658     GETIDENTITY
00659     UBYTE *t, c;
00660     int i, num = sizeof(onoffoptions)/sizeof(KEYWORD);
00661     LONG interval;
00662     for (;;) {
00663         while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
00664         if ( *s == 0 ) return(0);
00665         if ( chartype[*s] != 0 ) {
00666             MesPrint("&Illegal character or option encountered in ON statement");
00667             return(-1);
00668         }
00669         t = s; while ( chartype[*s] == 0 ) s++;
00670         c = *s; *s = 0;
00671         for ( i = 0; i < num; i++ ) {
00672             if ( StrICont(t, (UBYTE *) (onoffoptions[i].name)) == 0 ) break;
00673         }
00674         if ( i >= num ) {
00675             MesPrint("&Unrecognized option in ON statement: %s", t);
00676             *s = c; return(-1);
00677         }
00678         if ( StrICont(t, (UBYTE *)"backtrace") == 0 ) {
00679             #ifndef ENABLE_BACKTRACE
00680                 Warning("backtrace not supported on this platform");
00681             #endif
00682         }
00683         else if ( StrICont(t, (UBYTE *)"compress") == 0 ) {
00684             AR.gzipCompress = 0;
00685             *s = c;
00686             while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
00687             if ( *s ) {
00688                 t = s;
00689                 while ( FG.cTable[*s] <= 1 ) s++;
00690                 c = *s; *s = 0;
00691                 if ( StrICmp(t, (UBYTE *)"gzip") == 0 ) {
00692                     #ifndef WITHZLIB
00693                         Warning("gzip compression not supported on this platform");
00694                     #endif

```

```

00695 #ifdef WITHZSTD
00696 /* If gzip is specified, turn off zstd compression. zlib still goes via the
    wrapper. */
00697 ZWRAP_useZSTDcompression(0);
00698 #endif
00699 }
00700 else if ( StrICmp(t, (UBYTE *) "zstd") == 0 ) {
00701 #ifdef WITHZSTD
00702     ZWRAP_useZSTDcompression(1);
00703 #else
00704     Warning("zstd compression not supported on this platform");
00705 #endif
00706 }
00707 else {
00708     MesPrint("&Unrecognized option in ON compress statement: %s", t);
00709     return(-1);
00710 }
00711 /* Whether we are using zlib or zstd, accept and use a compression level. */
00712 *s = c;
00713 while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
00714 if ( FG.cTable[*s] == 1 ) {
00715     AR.gzipCompress = *s++ - '0';
00716     while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
00717     if ( *s ) {
00718         MesPrint("&Unrecognized option in ON compress gzip/zstd statement: %s", t);
00719         return(-1);
00720     }
00721 }
00722 else if ( *s == 0 ) {
00723     AR.gzipCompress = GZIPDEFAULT;
00724 }
00725 else {
00726     MesPrint("&Unrecognized option in ON compress gzip/zstd statement: %s, single
digit expected", t);
00727     return(-1);
00728 }
00729 }
00730 }
00731 else if ( StrICont(t, (UBYTE *) "checkpoint") == 0 ) {
00732     PrintDeprecation("the checkpoint mechanism", "issues/626");
00733     AC.CheckpointInterval = 0;
00734     if ( AC.CheckpointRunBefore ) { free(AC.CheckpointRunBefore); AC.CheckpointRunBefore =
NULL; }
00735     if ( AC.CheckpointRunAfter ) { free(AC.CheckpointRunAfter); AC.CheckpointRunAfter = NULL;
}
00736 *s = c;
00737 while ( *s ) {
00738     while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
00739     if ( FG.cTable[*s] == 1 ) {
00740         interval = 0;
00741         t = s;
00742         do { interval = 10*interval + *s++ - '0'; } while ( FG.cTable[*s] == 1 );
00743         if ( *s == 's' || *s == 'S' ) {
00744             s++;
00745         }
00746         else if ( *s == 'm' || *s == 'M' ) {
00747             interval *= 60; s++;
00748         }
00749         else if ( *s == 'h' || *s == 'H' ) {
00750             interval *= 3600; s++;
00751         }
00752         else if ( *s == 'd' || *s == 'D' ) {
00753             interval *= 86400; s++;
00754         }
00755         if ( *s != ',' && FG.cTable[*s] != 6 && FG.cTable[*s] != 10 ) {
00756             MesPrint("&Unrecognized time interval in ON Checkpoint statement: %s", t);
00757             return(-1);
00758         }
00759         AC.CheckpointInterval = interval * 100; /* in 1/100 of seconds */
00760     }
00761     else if ( FG.cTable[*s] == 0 ) {
00762         int type;
00763         t = s;
00764         while ( FG.cTable[*s] == 0 ) s++;
00765         c = *s; *s = 0;
00766         if ( StrICmp(t, (UBYTE *) "run") == 0 ) {
00767             type = 3;
00768         }
00769         else if ( StrICmp(t, (UBYTE *) "runafter") == 0 ) {
00770             type = 2;
00771         }
00772         else if ( StrICmp(t, (UBYTE *) "runbefore") == 0 ) {
00773             type = 1;
00774         }
00775         else {
00776             MesPrint("&Unrecognized option in ON Checkpoint statement: %s", t);
00777             *s = c; return(-1);

```

```

00778     }
00779     *s = c;
00780     if ( *s != '=' && FG.cTable[*s+1] != 9 ) {
00781         MesPrint("&Unrecognized option in ON Checkpoint statement: %s", t);
00782         return(-1);
00783     }
00784     ++s;
00785     t = ++s;
00786     while ( *s ) {
00787         if ( FG.cTable[*s] == 9 ) {
00788             c = *s; *s = 0;
00789             if ( type & 1 ) {
00790                 if ( AC.CheckpointRunBefore ) {
00791                     free(AC.CheckpointRunBefore); AC.CheckpointRunBefore = NULL;
00792                 }
00793                 if ( s-t > 0 ) {
00794                     AC.CheckpointRunBefore = Malloc1(s-t+1, "AC.CheckpointRunBefore");
00795                     StrCopy(t, (UBYTE*)AC.CheckpointRunBefore);
00796                 }
00797             }
00798             if ( type & 2 ) {
00799                 if ( AC.CheckpointRunAfter ) {
00800                     free(AC.CheckpointRunAfter); AC.CheckpointRunAfter = NULL;
00801                 }
00802                 if ( s-t > 0 ) {
00803                     AC.CheckpointRunAfter = Malloc1(s-t+1, "AC.CheckpointRunAfter");
00804                     StrCopy(t, (UBYTE*)AC.CheckpointRunAfter);
00805                 }
00806             }
00807             *s = c;
00808             break;
00809         }
00810         ++s;
00811     }
00812     if ( FG.cTable[*s] != 9 ) {
00813         MesPrint("&Unrecognized option in ON Checkpoint statement: %s", t);
00814         return(-1);
00815     }
00816     ++s;
00817 }
00818
00819 /*
00820 if ( AC.NoShowInput == 0 ) {
00821     MesPrint("Checkpoints activated.");
00822     if ( AC.CheckpointInterval ) {
00823         MesPrint("-> Minimum saving interval: %1 seconds.", AC.CheckpointInterval/100);
00824     }
00825     else {
00826         MesPrint("-> No minimum saving interval given. Saving after EVERY module.");
00827     }
00828     if ( AC.CheckpointRunBefore ) {
00829         MesPrint("-> Calling script \"%s\" before saving.", AC.CheckpointRunBefore);
00830     }
00831     if ( AC.CheckpointRunAfter ) {
00832         MesPrint("-> Calling script \"%s\" after saving.", AC.CheckpointRunAfter);
00833     }
00834 }
00835 */
00836 }
00837 else if ( StrICont(t, (UBYTE *) "indentSpace") == 0 ) {
00838     *s = c;
00839     while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
00840     if ( *s ) {
00841         i = 0;
00842         while ( FG.cTable[*s] == 1 ) { i = 10*i + *s++ - '0'; }
00843         if ( *s ) {
00844             MesPrint("&Unrecognized option in ON IndentSpace statement: %s", t);
00845             return(-1);
00846         }
00847         if ( i > 40 ) {
00848             Warning("IndentSpace parameter adjusted to 40");
00849             i = 40;
00850         }
00851         AO.IndentSpace = i;
00852     }
00853     else {
00854         AO.IndentSpace = AM.ggIndentSpace;
00855     }
00856     return(0);
00857 }
00858 else if ( ( StrICont(t, (UBYTE *) "fewerstats") == 0 ) ||
00859           ( StrICont(t, (UBYTE *) "fewerstatistics") == 0 ) ) {
00860     *s = c;
00861     while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
00862     if ( *s ) {
00863         i = 0;
00864         while ( FG.cTable[*s] == 1 ) { i = 10*i + *s++ - '0'; }

```

```

00865         if ( *s ) {
00866             MesPrint("&Unrecognized option in ON FewerStatistics statement: %s",t);
00867             return(-1);
00868         }
00869         if ( i > AM.S0->MaxPatches ) {
00870             if ( AC.WarnFlag )
00871                 MesPrint("&Warning: FewerStatistics parameter greater than MaxPatches(=%d).
Adjusted to %d"
00872                         ,AM.S0->MaxPatches, (AM.S0->MaxPatches+1)/2);
00873             i = (AM.S0->MaxPatches+1)/2;
00874         }
00875         AC.ShortStatsMax = i;
00876     }
00877     else {
00878         AC.ShortStatsMax = 10; /* default value */
00879     }
00880     return(0);
00881 }
00882 else if ( StrICont(t, (UBYTE *) "threads") == 0 ) {
00883     if ( AM.totalnumberofthreads > 1 ) AS.MultiThreaded = 1;
00884 }
00885 else if ( StrICont(t, (UBYTE *) "flag") == 0 ) {
00886     *s = c;
00887     return(CoFlags(s,1));
00888 }
00889 else if ( StrICont(t, (UBYTE *) "innertest") == 0 ) {
00890     UBYTE *t;
00891     *s = c;
00892     while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
00893     if ( *s ) {
00894         t = s; while ( *t ) t++;
00895         while ( t[-1] == ' ' || t[-1] == '\t' ) t--;
00896         c = *t; *t = 0;
00897         if ( AC.TestValue ) M_free(AC.TestValue, "InnerTest");
00898         AC.TestValue = strDupl(s, "InnerTest");
00899         *t = c;
00900         s = t;
00901         while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
00902     }
00903     else {
00904         if ( AC.TestValue ) {
00905             M_free(AC.TestValue, "InnerTest");
00906             AC.TestValue = 0;
00907         }
00908     }
00909 }
00910 else if ( StrICont(t, (UBYTE *) "flint") == 0 ) {
00911 #ifndef WITHFLINT
00912     MesPrint("&Warning: FORM was not built with FLINT support.");
00913     MesPrint("Statement has no effect.");
00914 #endif
00915 }
00916 else { *s = c; }
00917 *onoffoptions[i].var = onoffoptions[i].type;
00918 AR.SortType = AC.SortType;
00919 AC.mparallelfalg = AC.parallelfalg | AM.hparallelfalg;
00920 }
00921 }
00922
00923 /*
00924 #] CoOn :
00925 #[ CoInsideFirst :
00926 */
00927
00928 int CoInsideFirst(UBYTE *s) { return(setonoff(s,&AC.insidefirst,1,0)); }
00929
00930 /*
00931 #] CoInsideFirst :
00932 #[ CoProperCount :
00933 */
00934
00935 int CoProperCount(UBYTE *s) { return(setonoff(s,&AC.BottomLevel,1,0)); }
00936
00937 /*
00938 #] CoProperCount :
00939 #[ CoDelete :
00940 */
00941
00942 int CoDelete(UBYTE *s)
00943 {
00944     int error = 0;
00945     if ( StrICmp(s, (UBYTE *) "storage") == 0 ) {
00946         if ( DeleteStore(1) < 0 ) {
00947             MesPrint("&Cannot restart storage file");
00948             error = 1;
00949         }
00950     }
00951 }

```

```

00951     else {
00952         UBYTE *t = s, c;
00953         while ( *t && *t != ',' && *t != '>' ) t++;
00954         c = *t; *t = 0;
00955         if ( ( StrICmp(s, (UBYTE *)"extrasymbols") == 0 )
00956             || ( StrICmp(s, (UBYTE *)"extrasymbol") == 0 ) ) {
00957             WORD x = 0;
00958             /*
00959              * Either deletes all extra symbols or deletes above a given number
00960              */
00961             *t = c; s = t;
00962             if ( *s == '>' ) {
00963                 s++;
00964                 if ( FG.cTable[*s] != 1 ) goto unknown;
00965                 while ( *s <= '9' && *s >= '0' ) x = 10*x + *s++ - '0';
00966                 if ( *s ) goto unknown;
00967             }
00968             else if ( *s ) goto unknown;
00969             if ( x < AM.gnumextrasyms ) x = AM.gnumextrasyms;
00970             PruneExtraSymbols(x);
00971         }
00972     }
00973     *t = c;
00974 unknown:
00975     MesPrint("&Unknown option: %s",s);
00976     error = 1;
00977 }
00978 }
00979 return(error);
00980 }
00981 /*
00982 #] CoDelete :
00983 #[ CoKeep :
00984 */
00985 int CoKeep(UBYTE *s)
00986 {
00987     if ( StrICmp(s, (UBYTE *)"brackets") == 0 ) AC.ComDefer = 1;
00988     else { MesPrint("&Unknown option: '%s'",s); return(1); }
00989     return(0);
00990 }
00991 /*
00992 #] CoKeep :
00993 #[ CoFixIndex :
00994 */
00995 int CoFixIndex(UBYTE *s)
00996 {
00997     int x, y, error = 0;
00998     while ( *s ) {
00999         if ( FG.cTable[*s] != 1 ) {
01000             proper: MesPrint("&Proper syntax is: FixIndex,number:value[,number,value];");
01001             return(1);
01002         }
01003         ParseNumber(x,s)
01004         if ( *s != ':' ) goto proper;
01005         s++;
01006         if ( *s != '-' && *s != '+' && FG.cTable[*s] != 1 ) goto proper;
01007         ParseSignedNumber(y,s)
01008         if ( *s && *s != ',' ) goto proper;
01009         while ( *s == ',' ) s++;
01010         if ( x >= AM.OffsetIndex ) {
01011             MesPrint("&Fixed index out of allowed range. Change ConstIndex in setup file?");
01012             MesPrint("&Current value of ConstIndex = %d",AM.OffsetIndex-1);
01013             error = 1;
01014         }
01015         if ( y != (int)((WORD)y) ) {
01016             MesPrint("&Value of d_(%d,%d) outside range for this computer",x,x);
01017             error = 1;
01018         }
01019         if ( error == 0 ) AC.FixIndices[x] = y;
01020     }
01021     return(error);
01022 }
01023 /*
01024 #] CoFixIndex :
01025 #[ CoMetric :
01026 */
01027 int CoMetric(UBYTE *s)
01028 { DUMMYUSE(s); MesPrint("&The metric statement does not do anything yet"); return(1); }
01029 /*
01030 #] CoMetric :

```

```

01038     #[ DoPrint :
01039     */
01040
01041 int DoPrint(UBYTE *s, int par)
01042 {
01043     int i, error = 0, numdol = 0, type;
01044     WORD handle = -1;
01045     UBYTE *name, c, *t;
01046     EXPRESSIONS e;
01047     WORD numexpr, tofile = 0, *w, par2 = 0;
01048     CBUF *C = cbuf + AC.cbufnum;
01049     while ( *s == ',' ) s++;
01050     if ( ( *s == '+' || *s == '-' ) && ( s[1] == 'f' || s[1] == 'F' ) ) {
01051         t = s + 2; while ( *t == ' ' || *t == ',' ) t++;
01052         if ( *t == '"' ) {
01053             if ( *s == '+' ) { tofile = 1; handle = AC.LogHandle; }
01054             s = t;
01055         }
01056     }
01057     else if ( *s == '<' ) {
01058         UBYTE *filename;
01059         s++; filename = s;
01060         while ( *s && *s != '>' ) s++;
01061         if ( *s == 0 ) {
01062             MesPrint("&Improper filename in print statement");
01063             return(1);
01064         }
01065         *s++ = 0;
01066         tofile = 1;
01067         if ( ( handle = GetChannel((char *)filename,1) ) < 0 ) return(1);
01068         SKIPBLANKS(s) if ( *s == ',' ) s++; SKIPBLANKS(s)
01069         if ( *s == '+' && ( s[1] == 's' || s[1] == 'S' ) ) {
01070             s += 2;
01071             par2 |= PRINTONETERM;
01072             if ( *s == 's' || *s == 'S' ) {
01073                 s++;
01074                 par2 |= PRINTONEFUNCTION;
01075                 if ( *s == 's' || *s == 'S' ) {
01076                     s++;
01077                     par2 |= PRINTALL;
01078                 }
01079             }
01080             SKIPBLANKS(s) if ( *s == ',' ) s++; SKIPBLANKS(s)
01081         }
01082     }
01083     if ( par == PRINTON && *s == '"' ) {
01084         WORD code[3];
01085         if ( tofile == 1 ) code[0] = TYPEFPRINT;
01086         else code[0] = TYPEPRINT;
01087         code[1] = handle;
01088         code[2] = par2;
01089         s++; name = s;
01090         while ( *s && *s != '"' ) {
01091             if ( *s == '\\' ) s++;
01092             if ( *s == '%' && s[1] == '$' ) numdol++;
01093             s++;
01094         }
01095         if ( *s != '"' ) {
01096             MesPrint("&String in print statement should be enclosed in \"");
01097             return(1);
01098         }
01099         *s = 0;
01100         AddComString(3,code,name,1);
01101         *s++ = '"';
01102         while ( *s == ',' ) {
01103             s++;
01104             if ( *s == '$' ) {
01105                 s++; name = s; while ( FG.cTable[*s] <= 1 ) s++;
01106                 c = *s; *s = 0;
01107                 type = GetName(AC.dollarnames,name,&numexpr,NOAUTO);
01108                 if ( type == NAMENOTFOUND ) {
01109                     MesPrint("&$ variable %s not (yet) defined",name);
01110                     error = 1;
01111                 }
01112                 else {
01113                     C->lhs[C->numlhs][1] += 2;
01114                     *(C->Pointer)++ = DOLLAREXPRESSION;
01115                     *(C->Pointer)++ = numexpr;
01116                     numdol--;
01117                 }
01118             }
01119             else {
01120                 MesPrint("&Illegal object in print statement");
01121                 error = 1;
01122                 return(error);
01123             }
01124             *s = c;

```

```

01125         if ( c == '[' ) {
01126             w = C->Pointer;
01127             s++;
01128             s = GetDoParam(s,&(C->Pointer),-1);
01129             if ( s == 0 ) return(1);
01130             if ( *s != ']' ) {
01131                 MesPrint("&unmatched [] in $ factor");
01132                 return(1);
01133             }
01134             C->lhs[C->numlhs][1] += C->Pointer - w;
01135             s++;
01136         }
01137     }
01138     if ( *s != 0 ) {
01139         MesPrint("&Illegal object in print statement");
01140         error = 1;
01141     }
01142     if ( numdol > 0 ) {
01143         MesPrint("&More $ variables asked for than provided");
01144         error = 1;
01145     }
01146     *(C->Pointer)++ = 0;
01147     return(error);
01148 }
01149 if ( *s == 0 ) { /* All active expressions */
01150 AllExpr:
01151     for ( e = Expressions, i = NumExpressions; i > 0; i--, e++ ) {
01152         if ( e->status == LOCALEXPRESSION || e->status ==
01153             GLOBALEXPRESSION || e->status == UNHIDELEXPRESSION
01154             || e->status == UNHIDEEXPRESSION ) e->printflag = par;
01155     }
01156     return(error);
01157 }
01158 while ( *s ) {
01159     if ( *s == '+' ) {
01160         s++;
01161         if ( tolower(*s) == 'f' ) par |= PRINTLFILE;
01162         else if ( tolower(*s) == 's' ) {
01163             if ( tolower(s[1]) == 's' ) {
01164                 if ( tolower(s[2]) == 's' ) {
01165                     par |= PRINTONEFUNCTION | PRINTONETERM | PRINTALL;
01166                     s++;
01167                 }
01168                 else if ( ( par & 3 ) < 2 ) par |= PRINTONEFUNCTION | PRINTONETERM;
01169                 s++;
01170             }
01171             else {
01172                 if ( ( par & 3 ) < 2 ) par |= PRINTONETERM;
01173             }
01174         }
01175         else {
01176 illeg:
01177             MesPrint("&Illegal option in (n)print statement");
01178             error = 1;
01179         }
01180         s++;
01181         if ( *s == 0 ) goto AllExpr;
01182     }
01183     else if ( *s == '-' ) {
01184         s++;
01185         if ( tolower(*s) == 'f' ) par &= ~PRINTLFILE;
01186         else if ( tolower(*s) == 's' ) {
01187             if ( tolower(s[1]) == 's' ) {
01188                 if ( tolower(s[2]) == 's' ) {
01189                     par &= ~PRINTALL;
01190                     s++;
01191                 }
01192                 else if ( ( par & 3 ) < 2 ) {
01193                     par &= ~PRINTONEFUNCTION;
01194                     par &= ~PRINTALL;
01195                 }
01196                 s++;
01197             }
01198             else {
01199                 if ( ( par & 3 ) < 2 ) {
01200                     par &= ~PRINTONETERM;
01201                     par &= ~PRINTONEFUNCTION;
01202                     par &= ~PRINTALL;
01203                 }
01204             }
01205             else goto illeg;
01206             s++;
01207             if ( *s == 0 ) goto AllExpr;
01208         }
01209     }
01210     else if ( FG.cTable[*s] == 0 || *s == '[' ) {
01211         name = s;
01212         if ( ( s = SkipAName(s) ) == 0 ) {

```



```

01212         MesPrint("&Improper name in (n)print statement");
01213         return(1);
01214     }
01215     c = *s; *s = 0;
01216     if ( ( GetName(AC.exprnames,name,&numexpr,NOAUTO) == CEXPRESSION )
01217         && ( Expressions[numexpr].status == LOCALEXPRESSION
01218             || Expressions[numexpr].status == GLOBALEXPRESSION ) ) {
01219 FoundExpr;;
01220         if ( c == '[' && s[1] == ']' ) {
01221             Expressions[numexpr].printflag = par | PRINTCONTENTS;
01222             *s++ = c; c = *++s;
01223         }
01224         else
01225             Expressions[numexpr].printflag = par;
01226     }
01227     else if ( GetLastExprName(name,&numexpr)
01228         && ( Expressions[numexpr].status == LOCALEXPRESSION
01229             || Expressions[numexpr].status == GLOBALEXPRESSION
01230             || Expressions[numexpr].status == UNHIDELEXPRESSION
01231             || Expressions[numexpr].status == UNHIDEGEXPRESSION
01232         ) ) {
01233         goto FoundExpr;
01234     }
01235     else {
01236         MesPrint("%&s is not the name of an active expression",name);
01237         error = 1;
01238     }
01239     *s++ = c;
01240     if ( c == 0 ) return(0);
01241     if ( c == '-' || c == '+' ) s--;
01242 }
01243 else if ( *s == ',' ) s++;
01244 else {
01245     MesPrint("&Illegal object in (n)print statement");
01246     return(1);
01247 }
01248 }
01249 return(0);
01250 }
01251
01252 /*
01253 #] DoPrint :
01254 #[ CoPrint :
01255 */
01256
01257 int CoPrint(UBYTE *s) { return(DoPrint(s,PRINTON)); }
01258
01259 /*
01260 #] CoPrint :
01261 #[ CoPrintB :
01262 */
01263
01264 int CoPrintB(UBYTE *s) { return(DoPrint(s,PRINTCONTENT)); }
01265
01266 /*
01267 #] CoPrintB :
01268 #[ CoNPrint :
01269 */
01270
01271 int CoNPrint(UBYTE *s) { return(DoPrint(s,PRINTOFF)); }
01272
01273 /*
01274 #] CoNPrint :
01275 #[ CoPushHide :
01276 */
01277
01278 int CoPushHide(UBYTE *s)
01279 {
01280     GETIDENTITY
01281     WORD *ScratchBuf;
01282     int i;
01283     if ( AR.Fscr[2].PObuffer == 0 ) {
01284         ScratchBuf = (WORD *)Malloc1(AM.HideSize*sizeof(WORD),"hidesize");
01285         AR.Fscr[2].POsize = AM.HideSize * sizeof(WORD);
01286         AR.Fscr[2].POfull = AR.Fscr[2].POfill = AR.Fscr[2].PObuffer = ScratchBuf;
01287         AR.Fscr[2].POstop = AR.Fscr[2].PObuffer + AM.HideSize;
01288         PUTZERO(AR.Fscr[2].POposition);
01289     }
01290     while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
01291     AC.HideLevel += 2;
01292     if ( *s ) {
01293         MesPrint("&PushHide statement should have no arguments");
01294         return(-1);
01295     }
01296     for ( i = 0; i < NumExpressions; i++ ) {
01297         switch ( Expressions[i].status ) {
01298             case DROPLEXPRESSION:

```

```

01299         case SKIPLEXPRESSION:
01300         case LOCALEXPRESSION:
01301             Expressions[i].status = HIDELEXPRESSION;
01302             Expressions[i].hidelevel = AC.HideLevel-1;
01303             break;
01304         case DROPGEXPRESSION:
01305         case SKIPGEXPRESSION:
01306         case GLOBALEXPRESSION:
01307             Expressions[i].status = HIDEGEXPRESSION;
01308             Expressions[i].hidelevel = AC.HideLevel-1;
01309             break;
01310         default:
01311             break;
01312     }
01313 }
01314 return(0);
01315 }
01316
01317 /*
01318 #] CoPushHide :
01319 #[ CoPopHide :
01320 */
01321
01322 int CoPopHide(UBYTE *s)
01323 {
01324     int i;
01325     while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
01326     if ( AC.HideLevel <= 0 ) {
01327         MesPrint("&PopHide statement without corresponding PushHide statement");
01328         return(-1);
01329     }
01330     AC.HideLevel -= 2;
01331     if ( *s ) {
01332         MesPrint("&PopHide statement should have no arguments");
01333         return(-1);
01334     }
01335     for ( i = 0; i < NumExpressions; i++ ) {
01336         switch ( Expressions[i].status ) {
01337             case HIDDENLEXPRESSION:
01338                 if ( Expressions[i].hidelevel > AC.HideLevel )
01339                     Expressions[i].status = UNHIDELEXPRESSION;
01340                 break;
01341             case HIDDENGEXPRESSION:
01342                 if ( Expressions[i].hidelevel > AC.HideLevel )
01343                     Expressions[i].status = UNHIDEGEXPRESSION;
01344                 break;
01345             default:
01346                 break;
01347         }
01348     }
01349     return(0);
01350 }
01351
01352 /*
01353 #] CoPopHide :
01354 #[ SetExprCases :
01355 */
01356
01357 int SetExprCases(int par, int setunset, int val)
01358 {
01359     switch ( par ) {
01360         case SKIP:
01361             switch ( val ) {
01362                 case SKIPLEXPRESSION:
01363                     if ( !setunset ) val = LOCALEXPRESSION;
01364                     break;
01365                 case SKIPGEXPRESSION:
01366                     if ( !setunset ) val = GLOBALEXPRESSION;
01367                     break;
01368                 case LOCALEXPRESSION:
01369                     if ( setunset ) val = SKIPLEXPRESSION;
01370                     break;
01371                 case GLOBALEXPRESSION:
01372                     if ( setunset ) val = SKIPGEXPRESSION;
01373                     break;
01374                 case INTOHIDEGEXPRESSION:
01375                 case INTOHIDELEXPRESSION:
01376                 default:
01377                     break;
01378             }
01379             break;
01380         case DROP:
01381             switch ( val ) {
01382                 case SKIPLEXPRESSION:
01383                 case LOCALEXPRESSION:
01384                 case HIDELEXPRESSION:
01385                     if ( setunset ) val = DROPLEXPRESSION;

```

```

01386         break;
01387     case DROPEXPRESSION:
01388         if ( !setunset ) val = LOCALEXPRESSION;
01389         break;
01390     case SKIPGEXPRESSION:
01391     case GLOBALEXPRESSION:
01392     case HIDEGEXPRESSION:
01393         if ( setunset ) val = DROPGEXPRESSION;
01394         break;
01395     case DROPGEXPRESSION:
01396         if ( !setunset ) val = GLOBALEXPRESSION;
01397         break;
01398     case HIDDENLEXPRESSION:
01399     case UNHIDELEXPRESSION:
01400         if ( setunset ) val = DROPHLEXPRESSION;
01401         break;
01402     case HIDDENGEXPRESSION:
01403     case UNHIDEGEXPRESSION:
01404         if ( setunset ) val = DROPHGEXPRESSION;
01405         break;
01406     case DROPHLEXPRESSION:
01407         if ( !setunset ) val = HIDDENLEXPRESSION;
01408         break;
01409     case DROPHGEXPRESSION:
01410         if ( !setunset ) val = HIDDENGEXPRESSION;
01411         break;
01412     case INTOHIDEGEXPRESSION:
01413     case INTOHIDELEXPRESSION:
01414     default:
01415         break;
01416     }
01417     break;
01418 case HIDE:
01419     switch ( val ) {
01420     case DROPEXPRESSION:
01421     case SKIPLEXPRESSION:
01422     case LOCALEXPRESSION:
01423         if ( setunset ) val = HIDELEXPRESSION;
01424         break;
01425     case HIDELEXPRESSION:
01426         if ( !setunset ) val = LOCALEXPRESSION;
01427         break;
01428     case DROPGEXPRESSION:
01429     case SKIPGEXPRESSION:
01430     case GLOBALEXPRESSION:
01431         if ( setunset ) val = HIDEGEXPRESSION;
01432         break;
01433     case HIDEGEXPRESSION:
01434         if ( !setunset ) val = GLOBALEXPRESSION;
01435         break;
01436     case INTOHIDEGEXPRESSION:
01437     case INTOHIDELEXPRESSION:
01438     default:
01439         break;
01440     }
01441     break;
01442 case UNHIDE:
01443     switch ( val ) {
01444     case HIDDENLEXPRESSION:
01445     case DROPHLEXPRESSION:
01446         if ( setunset ) val = UNHIDELEXPRESSION;
01447         break;
01448     case UNHIDELEXPRESSION:
01449         if ( !setunset ) val = HIDDENLEXPRESSION;
01450         break;
01451     case HIDDENGEXPRESSION:
01452     case DROPHGEXPRESSION:
01453         if ( setunset ) val = UNHIDEGEXPRESSION;
01454         break;
01455     case UNHIDEGEXPRESSION:
01456         if ( !setunset ) val = HIDDENGEXPRESSION;
01457         break;
01458     case INTOHIDEGEXPRESSION:
01459     case INTOHIDELEXPRESSION:
01460     default:
01461         break;
01462     }
01463     break;
01464 case INTOHIDE:
01465     switch ( val ) {
01466     case HIDDENLEXPRESSION:
01467     case HIDDENGEXPRESSION:
01468         MesPrint("&Expression is already hidden");
01469         return(-1);
01470     case DROPHLEXPRESSION:
01471     case DROPHGEXPRESSION:
01472     case UNHIDELEXPRESSION:

```

```

01473         case UNHIDEGEXPRESSION:
01474             if ( setunset ) {
01475                 MesPrint("&Cannot unhide/drop and put intohide expression in the same
module");
01476                 return(-1);
01477             }
01478             break;
01479         case LOCALEXPRESSION:
01480         case DROPLEXPRESSION:
01481         case SKIPLEXPRESSION:
01482         case HIDELEXPRESSION:
01483             if ( setunset ) val = INTOHIDELEXPRESSION;
01484             break;
01485         case GLOBALEXPRESSION:
01486         case DROPGEXPRESSION:
01487         case SKIPGEXPRESSION:
01488         case HIDEGEXPRESSION:
01489             if ( setunset ) val = INTOHIDEGEXPRESSION;
01490             break;
01491         case INTOHIDELEXPRESSION:
01492             if ( !setunset ) val = LOCALEXPRESSION;
01493             break;
01494         case INTOHIDEGEXPRESSION:
01495             if ( !setunset ) val = GLOBALEXPRESSION;
01496             break;
01497         default:
01498             break;
01499     }
01500     break;
01501 default:
01502     break;
01503 }
01504 return(val);
01505 }
01506
01507 /*
01508 #] SetExprCases :
01509 #[ SetExpr :
01510 */
01511
01512 int SetExpr(UBYTE *s, int setunset, int par)
01513 {
01514     WORD *w, numexpr;
01515     int error = 0, i;
01516     UBYTE *name, c;
01517     if ( *s == 0 ) {
01518         for ( i = 0; i < NumExpressions; i++ ) {
01519             w = &(Expressions[i].status);
01520             *w = SetExprCases(par,setunset,*w);
01521             if ( *w < 0 ) error = 1;
01522             if ( ( par == HIDE || par == INTOHIDE ) && setunset == 1 )
01523                 Expressions[i].hidelevel = AC.HideLevel;
01524         }
01525         return(0);
01526     }
01527     while ( *s ) {
01528         if ( *s == ',' ) { s++; continue; }
01529         if ( *s == '0' ) { s++; continue; }
01530         name = s;
01531         if ( ( s = SkipAName(s) ) == 0 ) {
01532             MesPrint("&Improper name for an expression: '%s'",name);
01533             return(1);
01534         }
01535         c = *s; *s = 0;
01536         if ( GetName(AC.exprnames,name,&numexpr,NOAUTO) == CEXPRESSION ) {
01537             w = &(Expressions[numexpr].status);
01538             *w = SetExprCases(par,setunset,*w);
01539             if ( *w < 0 ) error = 1;
01540             if ( ( par == HIDE || par == INTOHIDE ) && setunset == 1 )
01541                 Expressions[numexpr].hidelevel = AC.HideLevel;
01542         }
01543         else if ( GetName(AC.varnames,name,&numexpr,NOAUTO) != NAMENOTFOUND ) {
01544             MesPrint("&%s is not an expression",name);
01545             error = 1;
01546         }
01547         *s = c;
01548     }
01549     return(error);
01550 }
01551
01552 /*
01553 #] SetExpr :
01554 #[ CoDrop :
01555 */
01556
01557 int CoDrop(UBYTE *s) { return(SetExpr(s,1,DROP)); }
01558

```

```

01559 /*
01560      #] CoDrop :
01561      #[ CoNoDrop :
01562 */
01563
01564 int CoNoDrop(UBYTE *s) { return(SetExpr(s,0,DROP)); }
01565
01566 /*
01567      #] CoNoDrop :
01568      #[ CoSkip :
01569 */
01570
01571 int CoSkip(UBYTE *s) { return(SetExpr(s,1,SKIP)); }
01572
01573 /*
01574      #] CoSkip :
01575      #[ CoNoSkip :
01576 */
01577
01578 int CoNoSkip(UBYTE *s) { return(SetExpr(s,0,SKIP)); }
01579
01580 /*
01581      #] CoNoSkip :
01582      #[ CoHide :
01583 */
01584
01585 int CoHide(UBYTE *inp) {
01586     GETIDENTITY
01587     WORD *ScratchBuf;
01588     if ( AR.Fscr[2].PObuffer == 0 ) {
01589         ScratchBuf = (WORD *)Malloc1(AM.HideSize*sizeof(WORD),"hidesize");
01590         AR.Fscr[2].POsize = AM.HideSize * sizeof(WORD);
01591         AR.Fscr[2].POfull = AR.Fscr[2].POfill = AR.Fscr[2].PObuffer = ScratchBuf;
01592         AR.Fscr[2].POstop = AR.Fscr[2].PObuffer + AM.HideSize;
01593         PUTZERO(AR.Fscr[2].POposition);
01594     }
01595     return(SetExpr(inp,1,HIDE));
01596 }
01597
01598 /*
01599      #] CoHide :
01600      #[ CoIntoHide :
01601 */
01602
01603 int CoIntoHide(UBYTE *inp) {
01604     GETIDENTITY
01605     WORD *ScratchBuf;
01606     if ( AR.Fscr[2].PObuffer == 0 ) {
01607         ScratchBuf = (WORD *)Malloc1(AM.HideSize*sizeof(WORD),"hidesize");
01608         AR.Fscr[2].POsize = AM.HideSize * sizeof(WORD);
01609         AR.Fscr[2].POfull = AR.Fscr[2].POfill = AR.Fscr[2].PObuffer = ScratchBuf;
01610         AR.Fscr[2].POstop = AR.Fscr[2].PObuffer + AM.HideSize;
01611         PUTZERO(AR.Fscr[2].POposition);
01612     }
01613     return(SetExpr(inp,1,INTOHIDE));
01614 }
01615
01616 /*
01617      #] CoIntoHide :
01618      #[ CoNoIntoHide :
01619 */
01620
01621 int CoNoIntoHide(UBYTE *inp) { return(SetExpr(inp,0,INTOHIDE)); }
01622
01623 /*
01624      #] CoNoIntoHide :
01625      #[ CoNoHide :
01626 */
01627
01628 int CoNoHide(UBYTE *inp) { return(SetExpr(inp,0,HIDE)); }
01629
01630 /*
01631      #] CoNoHide :
01632      #[ CoUnHide :
01633 */
01634
01635 int CoUnHide(UBYTE *inp) { return(SetExpr(inp,1,UNHIDE)); }
01636
01637 /*
01638      #] CoUnHide :
01639      #[ CoNoUnHide :
01640 */
01641
01642 int CoNoUnHide(UBYTE *inp) { return(SetExpr(inp,0,UNHIDE)); }
01643
01644 /*
01645      #] CoNoUnHide :

```

```

01646     #[ AddToCom :
01647 */
01648
01649 void AddToCom(int n, WORD *array)
01650 {
01651     CBUF *C = cbuf+AC.cbufnum;
01652 #ifdef COMPUFDEBBUG
01653     MesPrint(" %a",n,array);
01654 #endif
01655     while ( C->Pointer+n >= C->Top ) DoubleCbuffer(AC.cbufnum,C->Pointer,18);
01656     while ( --n >= 0 ) *(C->Pointer)++ = *array++;
01657 }
01658
01659 /*
01660 #] AddToCom :
01661 #[ AddComString :
01662 */
01663
01664 int AddComString(int n, WORD *array, UBYTE *thestring, int par)
01665 {
01666     CBUF *C = cbuf+AC.cbufnum;
01667     UBYTE *s = thestring, *w;
01668 #ifdef COMPUFDEBBUG
01669     WORD *cc;
01670     UBYTE *ww;
01671 #endif
01672     int i, numchars = 0, size, zeroes;
01673     while ( *s ) {
01674         if ( *s == '\\\\' ) s++;
01675         else if ( par == 1 &&
01676             ( ( *s == '%' && s[1] != 't' && s[1] != 'T' && s[1] != '$' &&
01677               s[1] != 'w' && s[1] != 'W' && s[1] != 'r' && s[1] != 0 ) || *s == '#'
01678             || *s == '@' || *s == '&' ) ) {
01679             numchars++;
01680         }
01681         s++; numchars++;
01682     }
01683     AddLHS(AC.cbufnum);
01684     size = numchars/sizeof(WORD)+1;
01685     while ( C->Pointer+size+n+2 >= C->Top ) DoubleCbuffer(AC.cbufnum,C->Pointer,19);
01686 #ifdef COMPUFDEBBUG
01687     cc = C->Pointer;
01688 #endif
01689     *(C->Pointer)++ = array[0];
01690     *(C->Pointer)++ = size+n+2;
01691     for ( i = 1; i < n; i++ ) *(C->Pointer)++ = array[i];
01692     *(C->Pointer)++ = size;
01693 #ifdef COMPUFDEBBUG
01694     ww =
01695 #endif
01696     w = (UBYTE *) (C->Pointer);
01697     zeroes = size*sizeof(WORD)-numchars;
01698     s = thestring;
01699     while ( *s ) {
01700         if ( *s == '\\\\' ) s++;
01701         else if ( par == 1 && ( *s == '%' &&
01702             s[1] != 't' && s[1] != 'T' && s[1] != '$' &&
01703             s[1] != 'w' && s[1] != 'W' && s[1] != 'r' && s[1] != 0 ) || *s == '#'
01704             || *s == '@' || *s == '&' ) ) {
01705             *w++ = '%';
01706         }
01707         *w++ = *s++;
01708     }
01709     while ( --zeroes >= 0 ) *w++ = 0;
01710     C->Pointer += size;
01711 #ifdef COMPUFDEBBUG
01712     MesPrint("LH: %a",size+1+n,cc);
01713     MesPrint(" %s",thestring);
01714 #endif
01715     return(0);
01716 }
01717
01718 /*
01719 #] AddComString :
01720 #[ Add2ComStrings :
01721 */
01722
01723 int Add2ComStrings(int n, WORD *array, UBYTE *string1, UBYTE *string2)
01724 {
01725     CBUF *C = cbuf+AC.cbufnum;
01726     UBYTE *s1 = string1, *s2 = string2, *w;
01727     int i, num1chars = 0, num2chars = 0, size1, size2, zeroes1, zeroes2;
01728     AddLHS(AC.cbufnum);
01729     while ( *s1 ) { s1++; num1chars++; }
01730     size1 = num1chars/sizeof(WORD)+1;
01731     if ( *s2 ) {
01732         while ( *s2 ) { s2++; num2chars++; }

```

```

01733     size2 = num2chars/sizeof(WORD)+1;
01734 }
01735 else size2 = 0;
01736 while ( C->Pointer+size1+size2+n+3 >= C->Top ) DoubleCbuffer (AC.cbunum,C->Pointer,20);
01737 *(C->Pointer)++ = array[0];
01738 *(C->Pointer)++ = size1+size2+n+3;
01739 for ( i = 1; i < n; i++ ) *(C->Pointer)++ = array[i];
01740 *(C->Pointer)++ = size1;
01741 w = (UBYTE *) (C->Pointer);
01742 zeroes1 = size1*sizeof(WORD)-num1chars;
01743 s1 = string1;
01744 while ( *s1 ) { *w++ = *s1++; }
01745 while ( --zeroes1 >= 0 ) *w++ = 0;
01746 C->Pointer += size1;
01747 *(C->Pointer)++ = size2;
01748 if ( size2 ) {
01749     w = (UBYTE *) (C->Pointer);
01750     zeroes2 = size2*sizeof(WORD)-num2chars;
01751     s2 = string2;
01752     while ( *s2 ) { *w++ = *s2++; }
01753     while ( --zeroes2 >= 0 ) *w++ = 0;
01754     C->Pointer += size2;
01755 }
01756 return(0);
01757 }
01758
01759 /*
01760 #] Add2ComStrings :
01761 #[ CoDiscard :
01762 */
01763
01764 int CoDiscard(UBYTE *s)
01765 {
01766     if ( *s == 0 ) {
01767         Add2Com(TYPEDISCARD)
01768         return(0);
01769     }
01770     MesPrint("&Illegal argument in discard statement: '%s'",s);
01771     return(1);
01772 }
01773
01774 /*
01775 #] CoDiscard :
01776 #[ CoContract :
01777
01778 Syntax:
01779     Contract
01780     Contract:#
01781     Contract #
01782     Contract:##, #
01783 */
01784
01785 static WORD carray[5] = { TYPEOPERATION,5,CONTRACT,0,0 };
01786
01787 int CoContract(UBYTE *s)
01788 {
01789     int x;
01790     if ( *s == ':' ) {
01791         s++;
01792         ParseNumber(x,s)
01793         if ( *s != ',' && *s ) {
01794 proper:         MesPrint("&Illegal number in contract statement");
01795                 return(1);
01796             }
01797             if ( *s ) s++;
01798             carray[4] = x;
01799         }
01800         else carray[4] = 0;
01801         if ( FG.cTable[*s] == 1 ) {
01802             ParseNumber(x,s)
01803             if ( *s ) goto proper;
01804             carray[3] = x;
01805         }
01806         else if ( *s ) goto proper;
01807         else carray[3] = -1;
01808         return(AddNtoL(5,carray));
01809     }
01810
01811 /*
01812 #] CoContract :
01813 #[ CoGoTo :
01814 */
01815
01816 int CoGoTo(UBYTE *inp)
01817 {
01818     UBYTE *s = inp;
01819     int x;

```

```

01820     while ( FG.cTable[*s] <= 1 ) s++;
01821     if ( *s ) {
01822         MesPrint("&Label should be an alpha-numeric string");
01823         return(1);
01824     }
01825     x = GetLabel(inp);
01826     Add3Com(TYPEGOTO,x);
01827     return(0);
01828 }
01829
01830 /*
01831 #] CoGoTo :
01832 #[ CoLabel :
01833 */
01834
01835 int CoLabel(UBYTE *inp)
01836 {
01837     UBYTE *s = inp;
01838     int x;
01839     while ( FG.cTable[*s] <= 1 ) s++;
01840     if ( *s ) {
01841         MesPrint("&Label should be an alpha-numeric string");
01842         return(1);
01843     }
01844     x = GetLabel(inp);
01845     if ( AC.Labels[x] >= 0 ) {
01846         MesPrint("&Label %s defined more than once",AC.LabelNames[x]);
01847         return(1);
01848     }
01849     AC.Labels[x] = cbuf[AC.cbufnum].numlhs;
01850     return(0);
01851 }
01852
01853 /*
01854 #] CoLabel :
01855 #[ DoArgument :
01856
01857 Layout:
01858     par,full size,numlhs(+1),par,scale
01859     scale is for normalize
01860 */
01861
01862 int DoArgument(UBYTE *s, int par)
01863 {
01864     GETIDENTITY
01865     UBYTE *name, *t, *v, c;
01866     WORD *oldworkpointer = AT.WorkPointer, *w, *ww, number, *scale;
01867     int error = 0, zeroflag, type, x;
01868     AC.lhdollarflag = 0;
01869     while ( *s == ',' ) s++;
01870     w = AT.WorkPointer;
01871     *w++ = par;
01872     w++;
01873     switch ( par ) {
01874         case TYPEARG:
01875             if ( AC.arglevel >= MAXNEST ) {
01876                 MesPrint("@Nesting of argument statements more than %d levels"
01877                     , (WORD)MAXNEST);
01878                 return(-1);
01879             }
01880             AC.argsumcheck[AC.arglevel] = NestingChecksum();
01881             AC.argstack[AC.arglevel] = cbuf[AC.cbufnum].Pointer
01882                 - cbuf[AC.cbufnum].Buffer + 2;
01883             AC.arglevel++;
01884             *w++ = cbuf[AC.cbufnum].numlhs;
01885             break;
01886         case TYPENORM:
01887         case TYPENORM4:
01888         case TYPESPLITARG:
01889         case TYPESPLITFIRSTARG:
01890         case TYPESPLITLASTARG:
01891         case TYPEFACTARG:
01892         case TYPEARGTOEXTRASymbol:
01893             *w++ = cbuf[AC.cbufnum].numlhs+1;
01894             break;
01895     }
01896     *w++ = par;
01897     scale = w;
01898     *w++ = 1;
01899     *w++ = 0;
01900     if ( *s == '^' ) {
01901         s++; ParseSignedNumber(x,s)
01902         while ( *s == ',' ) s++;
01903         *scale = x;
01904     }
01905     if ( *s == '(' ) {
01906         t = s+1; SKIPBRA3(s)      /* We did check the brackets already */

```



```

01907     if ( par == TYPEARG ) {
01908         MesPrint("&Illegal () entry in argument statement");
01909         error = 1; s++; goto skipbracks;
01910     }
01911     else if ( par == TYPESPLITFIRSTARG ) {
01912         MesPrint("&Illegal () entry in splitfirstarg statement");
01913         error = 1; s++; goto skipbracks;
01914     }
01915     else if ( par == TYPESPLITLASTARG ) {
01916         MesPrint("&Illegal () entry in splitlastarg statement");
01917         error = 1; s++; goto skipbracks;
01918     }
01919     v = t;
01920     while ( v < s ) {
01921         if ( *v == '?' ) {
01922             MesPrint("&Wildcarding not allowed in this type of statement");
01923             error = 1; break;
01924         }
01925         v++;
01926     }
01927     v = s++;
01928     if ( *t == '(' && v[-1] == ')' ) {
01929         t++; v--;
01930         if ( par == TYPESPLITARG ) oldworkpointer[0] = TYPESPLITARG2;
01931         else if ( par == TYPEFACTARG ) oldworkpointer[0] = TYPEFACTARG2;
01932         else if ( par == TYPENORM4 ) oldworkpointer[0] = TYPENORM4;
01933         else if ( par == TYPENORM ) {
01934             if ( *t == '-' ) { oldworkpointer[0] = TYPENORM3; t++; }
01935             else { oldworkpointer[0] = TYPENORM2; *scale = 0; }
01936         }
01937     }
01938     if ( error == 0 ) {
01939         CBUF *C = cbuf+AC.cbunum;
01940         WORD oldnumrhs = C->numrhs, oldnumlhs = C->numlhs;
01941         WORD prototype[SUBEXPSIZE+40]; /* Up to 10 nested sums! */
01942         WORD *m, *mm;
01943         int i, retcode;
01944         LONG oldpointer = C->Pointer - C->Buffer;
01945         *v = 0;
01946         prototype[0] = SUBEXPRESSION;
01947         prototype[1] = SUBEXPSIZE;
01948         prototype[2] = C->numrhs+1;
01949         prototype[3] = 1;
01950         prototype[4] = AC.cbunum;
01951         AT.WorkPointer += TYPEARGHEADSIZE+1;
01952         AddLHS(AC.cbunum);
01953         if ( ( retcode = CompileAlgebra(t,LHSIDE,prototype) ) < 0 )
01954             error = 1;
01955         else {
01956             prototype[2] = retcode;
01957             ww = C->lhs[retcode];
01958             AC.lhdollarflag = 0;
01959             if ( *ww == 0 ) {
01960                 *w++ = -2; *w++ = 0;
01961             }
01962             else if ( ww[ww[0]] != 0 ) {
01963                 MesPrint("&There should be only one term between ()");
01964                 error = 1;
01965             }
01966             else if ( NewSort(BHEAD0) ) { if ( !error ) error = 1; }
01967             else if ( NewSort(BHEAD0) ) {
01968                 LowerSortLevel();
01969                 if ( !error ) error = 1;
01970             }
01971             else {
01972                 AN.RepPoint = AT.RepCount + 1;
01973                 m = AT.WorkPointer;
01974                 mm = ww; i = *mm;
01975                 while ( --i >= 0 ) *m++ = *mm++;
01976                 mm = AT.WorkPointer; AT.WorkPointer = m;
01977                 AR.Cnumlhs = C->numlhs;
01978                 if ( Generator(BHEAD mm,C->numlhs) ) {
01979                     LowerSortLevel(); error = 1;
01980                 }
01981                 else if ( EndSort(BHEAD mm,0) < 0 ) {
01982                     error = 1;
01983                     AT.WorkPointer = mm;
01984                 }
01985                 else if ( *mm == 0 ) {
01986                     *w++ = -2; *w++ = 0;
01987                     AT.WorkPointer = mm;
01988                 }
01989                 else if ( mm[mm[0]] != 0 ) {
01990                     error = 1;
01991                     AT.WorkPointer = mm;
01992                 }
01993                 else {

```

```

01994         AT.WorkPointer = mm;
01995         m = mm+*mm;
01996         if ( par == TYPEFACTARG ) {
01997             if ( *mm != ABS(m[-1])+1 ) {
01998                 *mm -= ABS(m[-1]); /* Strip coefficient */
01999             }
02000             mm[-1] = -*mm-1; w += *mm+1;
02001         }
02002         else {
02003             *mm -= ABS(m[-1]); /* Strip coefficient */
02004         /*
02005             if ( *mm == 1 ) { *w++ = -2; *w++ = 0; }
02006             else
02007         /*
02008                 { mm[-1] = -*mm-1; w += *mm+1; }
02009             }
02010             oldworkpointer[1] = w - oldworkpointer;
02011         }
02012         LowerSortLevel();
02013     }
02014     oldworkpointer[5] = AC.lhdollarflag;
02015 }
02016 *v = ' ';
02017 C->numrhs = oldnumrhs;
02018 C->numlhs = oldnumlhs;
02019 C->Pointer = C->Buffer + oldpointer;
02020 }
02021 }
02022 skipbracks:
02023     if ( *s == 0 ) { *w++ = 0; *w++ = 2; *w++ = 1; }
02024     else {
02025         do {
02026             if ( *s == ',' ) { s++; continue; }
02027             ww = w; *w++ = 0; w++;
02028             if ( FG.cTable[*s] > 1 && *s != '[' && *s != '{' ) {
02029                 MesPrint("&Illegal parameters in statement");
02030                 error = 1;
02031                 break;
02032             }
02033             while ( FG.cTable[*s] == 0 || *s == '[' || *s == '{' ) {
02034                 if ( *s == '{' ) {
02035                     name = s+1;
02036                     SKIPBRA2(s)
02037                     c = *s; *s = 0;
02038                     number = DoTempSet(name,s);
02039                     name--; *s++ = c; c = *s; *s = 0;
02040                     goto doset;
02041                 }
02042                 else {
02043                     name = s;
02044                     if ( ( s = SkipAName(s) ) == 0 ) {
02045                         MesPrint("&Illegal name '%s'",name);
02046                         return(1);
02047                     }
02048                     c = *s; *s = 0;
02049                     if ( ( type = GetName(AC.varnames,name,&number,WITHAUTO) ) == CSET ) {
02050                         if ( Sets[number].type != CFUNCTION ) goto nofun;
02051                         *w++ = CSET; *w++ = number;
02052                     }
02053                     else if ( type == CFUNCTION ) {
02054                         *w++ = CFUNCTION; *w++ = number + FUNCTION;
02055                     }
02056                     else {
02057                         MesPrint("&%s is not a function or a set of functions",
02058                             name);
02059                         error = 1;
02060                     }
02061                 }
02062                 *s = c;
02063                 while ( *s == ',' ) s++;
02064             }
02065             ww[1] = w - ww;
02066             ww = w; w++; zeroflag = 0;
02067             while ( FG.cTable[*s] == 1 ) {
02068                 ParseNumber(x,s)
02069                 if ( *s && *s != ',' ) {
02070                     MesPrint("&Illegal separator after number");
02071                     error = 1;
02072                     while ( *s && *s != ',' ) s++;
02073                 }
02074                 while ( *s == ',' ) s++;
02075                 if ( x == 0 ) zeroflag = 1;
02076                 if ( !zeroflag ) *w++ = (WORD)x;
02077             }
02078             *ww = w - ww;
02079         } while ( *s );
02080     }

```

```

02081     oldworkpointer[1] = w - oldworkpointer;
02082     if ( par == TYPEARG ) { /* To make sure. The Pointer might move in the future */
02083         AC.argstack[AC.arglevel-1] = cbuf[AC.cbunum].Pointer
02084             - cbuf[AC.cbunum].Buffer + 2;
02085     }
02086     AddNtoL(oldworkpointer[1],oldworkpointer);
02087     AT.WorkPointer = oldworkpointer;
02088     return(error);
02089 }
02090
02091 /*
02092     #] DoArgument :
02093     #[ CoArgument :
02094 */
02095
02096 int CoArgument(UBYTE *s) { return(DoArgument(s,TYPEARG)); }
02097
02098 /*
02099     #] CoArgument :
02100     #[ CoEndArgument :
02101 */
02102
02103 int CoEndArgument(UBYTE *s)
02104 {
02105     CBUF *C = cbuf+AC.cbunum;
02106     while ( *s == ',' ) s++;
02107     if ( *s ) {
02108         MesPrint("%Illegal syntax for EndArgument statement");
02109         return(1);
02110     }
02111     if ( AC.arglevel <= 0 ) {
02112         MesPrint("&EndArgument without corresponding Argument statement");
02113         return(1);
02114     }
02115     AC.arglevel--;
02116     cbuf[AC.cbunum].Buffer[AC.argstack[AC.arglevel]] = C->numlhs;
02117     if ( AC.argsumcheck[AC.arglevel] != NestingChecksum() ) {
02118         MesNesting();
02119         return(1);
02120     }
02121     return(0);
02122 }
02123
02124 /*
02125     #] CoEndArgument :
02126     #[ CoInside :
02127 */
02128
02129 int CoInside(UBYTE *s) { return(ExecInside(s)); }
02130
02131 /*
02132     #] CoInside :
02133     #[ CoEndInside :
02134 */
02135
02136 int CoEndInside(UBYTE *s)
02137 {
02138     CBUF *C = cbuf+AC.cbunum;
02139     while ( *s == ',' ) s++;
02140     if ( *s ) {
02141         MesPrint("%Illegal syntax for EndInside statement");
02142         return(1);
02143     }
02144     if ( AC.insidelevel <= 0 ) {
02145         MesPrint("&EndInside without corresponding Inside statement");
02146         return(1);
02147     }
02148     AC.insidelevel--;
02149     cbuf[AC.cbunum].Buffer[AC.insidestack[AC.insidelevel]] = C->numlhs;
02150     if ( AC.insidesumcheck[AC.insidelevel] != NestingChecksum() ) {
02151         MesNesting();
02152         return(1);
02153     }
02154     return(0);
02155 }
02156
02157 /*
02158     #] CoEndInside :
02159     #[ CoNormalize :
02160 */
02161
02162 int CoNormalize(UBYTE *s) { return(DoArgument(s,TYPENORM)); }
02163
02164 /*
02165     #] CoNormalize :
02166     #[ CoMakeInteger :
02167 */

```

```

02168
02169 int CoMakeInteger(UBYTE *s) { return(DoArgument(s,TYPENORM4)); }
02170
02171 /*
02172 #] CoMakeInteger :
02173 #[ CoSplitArg :
02174 */
02175
02176 int CoSplitArg(UBYTE *s) { return(DoArgument(s,TYPESPLITARG)); }
02177
02178 /*
02179 #] CoSplitArg :
02180 #[ CoSplitFirstArg :
02181 */
02182
02183 int CoSplitFirstArg(UBYTE *s) { return(DoArgument(s,TYPESPLITFIRSTARG)); }
02184
02185 /*
02186 #] CoSplitFirstArg :
02187 #[ CoSplitLastArg :
02188 */
02189
02190 int CoSplitLastArg(UBYTE *s) { return(DoArgument(s,TYPESPLITLASTARG)); }
02191
02192 /*
02193 #] CoSplitLastArg :
02194 #[ CoFactArg :
02195 */
02196
02197 int CoFactArg(UBYTE *s) {
02198     if ( ( AC.topolynomialflag & TOPOLYNOMIALFLAG ) != 0 ) {
02199         MesPrint("&ToPolynomial statement and FactArg statement are not allowed in the same module");
02200         return(1);
02201     }
02202     AC.topolynomialflag |= FACTARGFLAG;
02203     return(DoArgument(s,TYPEFACTARG));
02204 }
02205
02206 /*
02207 #] CoFactArg :
02208 #[ DoSymmetrize :
02209
02210     Syntax:
02211     Symmetrize Fun[:[number]] [Fields]      -> par = 0;
02212     AntiSymmetrize Fun[:[number]] [Fields]  -> par = 1;
02213     CycleSymmetrize Fun[:[number]] [Fields] -> par = 2;
02214     RCycleSymmetrize Fun[:[number]] [Fields]-> par = 3;
02215 */
02216
02217 int DoSymmetrize(UBYTE *s, int par)
02218 {
02219     GETIDENTITY
02220     int extra = 0, error = 0, err, fix, x, groupsize, num, i;
02221     UBYTE *name, c;
02222     WORD funnum, *w, *ww, type;
02223     for(;;) {
02224         name = s;
02225         if ( ( s = SkipAName(s) ) == 0 ) {
02226             MesPrint("&Improper function name");
02227             return(1);
02228         }
02229         c = *s; *s = 0;
02230         if ( c != ',' || ( FG.cTable[s[1]] != 0 && s[1] != '(' ) ) break;
02231         if ( par <= 0 && StrICmp(name,(UBYTE *)"cyclic") == 0 ) extra = 2;
02232         else if ( par <= 0 && StrICmp(name,(UBYTE *)"rcyclic") == 0 ) extra = 6;
02233         else {
02234             MesPrint("&Illegal option: '%s'",name);
02235             error = 1;
02236         }
02237         *s++ = c;
02238     }
02239     if ( ( err = GetVar(name,&type,&funnum,CFUNCTION,WITHAUTO) ) == NAMENOTFOUND ) {
02240         MesPrint("&Undefined function: %s",name);
02241         AddFunction(name,0,0,0,0,0,-1,-1);
02242         *s++ = c;
02243         return(1);
02244     }
02245     funnum += FUNCTION;
02246     if ( err == -1 ) error = 1;
02247     *s = c;
02248     if ( *s == ':' ) {
02249         s++;
02250         if ( *s == ',' || *s == '(' || *s == 0 ) fix = -1;
02251         else if ( FG.cTable[*s] == 1 ) {
02252             ParseNumber(fix,s)
02253             if ( fix == 0 )
02254                 Warning("Restriction to zero arguments removed");

```

```

02255     }
02256     else {
02257         MesPrint("&Illegal character after :");
02258         return(1);
02259     }
02260 }
02261 else fix = 0;
02262 w = AT.WorkPointer;
02263 *w++ = TYPEOPERATION;
02264 w++;
02265 *w++ = SYMMETRIZE;
02266 *w++ = par | extra;
02267 *w++ = funnum;
02268 *w++ = fix;
02269 /*
02270 And now the argument lists. We have either ,#,#,... or (#,#,...,#),(#,...
02271 */
02272 w += 2; ww = w; groupsize = -1;
02273 while ( *s == ',' ) s++;
02274 while ( *s ) {
02275     if ( *s == '(' ) {
02276         s++; num = 0;
02277         while ( *s && *s != ')' ) {
02278             if ( *s == ',' ) { s++; continue; }
02279             if ( FG.cTable[*s] != 1 ) goto illarg;
02280             ParseNumber(x,s)
02281             if ( x <= 0 || ( fix > 0 && x > fix ) ) goto illnum;
02282             num++;
02283             *w++ = x-1;
02284         }
02285         if ( *s == 0 ) {
02286             MesPrint("&Improper termination of statement");
02287             return(1);
02288         }
02289         if ( groupsize < 0 ) groupsize = num;
02290         else if ( groupsize != num ) goto group;
02291         s++;
02292     }
02293     else if ( FG.cTable[*s] == 1 ) {
02294         if ( groupsize < 0 ) groupsize = 1;
02295         else if ( groupsize != 1 ) {
02296             group: MesPrint("&All groups should have the same number of arguments");
02297                 return(1);
02298         }
02299         ParseNumber(x,s)
02300         if ( x <= 0 || ( fix > 0 && x > fix ) ) {
02301             illnum: MesPrint("&Illegal argument number: %d",x);
02302                 return(1);
02303         }
02304         *w++ = x-1;
02305     }
02306     else {
02307         illarg: MesPrint("&Illegal argument");
02308                 return(1);
02309     }
02310     while ( *s == ',' ) s++;
02311 }
02312 /*
02313 Now the completion
02314 */
02315 if ( w == ww ) {
02316     ww[-1] = 1;
02317     ww[-2] = 0;
02318     if ( fix > 0 ) {
02319         for ( i = 0; i < fix; i++ ) *w++ = i;
02320         ww[-2] = fix; /* Bugfix 31-oct-2001. Reported by York Schroeder */
02321     }
02322 }
02323 else {
02324     ww[-1] = groupsize;
02325     ww[-2] = (w-ww)/groupsize;
02326 }
02327 AT.WorkPointer[1] = w - AT.WorkPointer;
02328 AddNtoL(AT.WorkPointer[1],AT.WorkPointer);
02329 return(error);
02330 }
02331
02332 /*
02333 #] DoSymmetrize :
02334 #[ CoSymmetrize :
02335 */
02336
02337 int CoSymmetrize(UBYTE *s) { return(DoSymmetrize(s,SYMMETRIC)); }
02338
02339 /*
02340 #] CoSymmetrize :
02341 #[ CoAntiSymmetrize :

```

```

02342 */
02343
02344 int CoAntiSymmetrize(UBYTE *s) { return(DoSometrize(s,ANTISYMMETRIC)); }
02345
02346 /*
02347     #] CoAntiSymmetrize :
02348     #[ CoCycleSymmetrize :
02349 */
02350
02351 int CoCycleSymmetrize(UBYTE *s) { return(DoSometrize(s,CYCLESYMMETRIC)); }
02352
02353 /*
02354     #] CoCycleSymmetrize :
02355     #[ CoRCycleSymmetrize :
02356 */
02357
02358 int CoRCycleSymmetrize(UBYTE *s) { return(DoSometrize(s,RCYCLESYMMETRIC)); }
02359
02360 /*
02361     #] CoRCycleSymmetrize :
02362     #[ CoWrite :
02363 */
02364
02365 int CoWrite(UBYTE *s)
02366 {
02367     GETIDENTITY
02368     UBYTE *option;
02369     KEYWORDV *key;
02370     option = s;
02371     if ( ( ( s = SkipAName(s) ) == 0 ) || *s != 0 ) {
02372         MesPrint("&Proper use of write statement is: write option");
02373         return(1);
02374     }
02375     key = (KEYWORDV *)FindInKeyWord(option, (KEYWORD
*)writeoptions,sizeof(writeoptions)/sizeof(KEYWORD));
02376     if ( key == 0 ) {
02377         MesPrint("&Unrecognized option in write statement");
02378         return(1);
02379     }
02380     *key->var = key->type;
02381     AR.SortType = AC.SortType;
02382     return(0);
02383 }
02384
02385 /*
02386     #] CoWrite :
02387     #[ CoNWrite :
02388 */
02389
02390 int CoNWrite(UBYTE *s)
02391 {
02392     GETIDENTITY
02393     UBYTE *option;
02394     KEYWORDV *key;
02395     option = s;
02396     if ( ( ( s = SkipAName(s) ) == 0 ) || *s != 0 ) {
02397         MesPrint("&Proper use of nwrite statement is: nwrite option");
02398         return(1);
02399     }
02400     key = (KEYWORDV *)FindInKeyWord(option, (KEYWORD
*)writeoptions,sizeof(writeoptions)/sizeof(KEYWORD));
02401     if ( key == 0 ) {
02402         MesPrint("&Unrecognized option in nwrite statement");
02403         return(1);
02404     }
02405     *key->var = key->flags;
02406     AR.SortType = AC.SortType;
02407     return(0);
02408 }
02409
02410 /*
02411     #] CoNWrite :
02412     #[ CoRatio :
02413 */
02414
02415 static WORD ratstring[6] = { TYPEOPERATION, 6, RATIO, 0, 0, 0 };
02416
02417 int CoRatio(UBYTE *s)
02418 {
02419     UBYTE c, *t;
02420     int i, type, error = 0;
02421     WORD numsym, *rs;
02422     rs = ratstring+3;
02423     for ( i = 0; i < 3; i++ ) {
02424         if ( *s ) {
02425             t = s;
02426             s = SkipAName(s);

```

```

02427         c = *s; *s = 0;
02428         if ( ( ( type = GetName(AC.varnames,t,&numsym,WITHAUTO) ) != CSYMBOL )
02429             && type != CDUBIOUS ) {
02430             MesPrint("&%s is not a symbol",t);
02431             error = 4;
02432             if ( type < 0 ) numsym = AddSymbol(t,-MAXPOWER,MAXPOWER,0,0);
02433         }
02434         *s = c;
02435         if ( *s == ',' ) s++;
02436     }
02437     else {
02438         if ( error == 0 )
02439             MesPrint("&The ratio statement needs three symbols for its arguments");
02440         error++;
02441         numsym = 0;
02442     }
02443     *rs++ = numsym;
02444 }
02445 AddNtoL(6, ratstring);
02446 return(error);
02447 }
02448
02449 /*
02450 #] CoRatio :
02451 #[ CoRedefine :
02452
02453 We have a preprocessor variable and a (new) value for it.
02454 This value is inside a string that must be stored.
02455 */
02456
02457 int CoRedefine(UBYTE *s)
02458 {
02459     UBYTE *name, c, *args = 0;
02460     int numprevar;
02461     WORD code[2];
02462     name = s;
02463     if ( FG.cTable[*s] || ( s = SkipAName(s) ) == 0 || s[-1] == '_' ) {
02464         MesPrint("&Illegal name for preprocessor variable in redefine statement");
02465         return(1);
02466     }
02467     c = *s; *s = 0;
02468     for ( numprevar = NumPre-1; numprevar >= 0; numprevar-- ) {
02469         if ( StrCmp(name, PreVar[numprevar].name) == 0 ) break;
02470     }
02471     if ( numprevar < 0 ) {
02472         MesPrint("&There is no preprocessor variable with the name '%s'", name);
02473         *s = c;
02474         return(1);
02475     }
02476     *s = c;
02477
02478 /*
02479 The next code worries about arguments.
02480 It is a direct copy of the code in TheDefine in the preprocessor.
02481 */
02482 if ( *s == '(' ) { /* arguments. scan for correctness */
02483     s++; args = s;
02484     for (;;) {
02485         if ( chartype[*s] != 0 ) goto illarg;
02486         s++;
02487         while ( chartype[*s] <= 1 ) s++;
02488         while ( *s == ' ' || *s == '\t' ) s++;
02489         if ( *s == ')' ) break;
02490         if ( *s != ',' ) goto illargs;
02491         s++;
02492         while ( *s == ' ' || *s == '\t' ) s++;
02493     }
02494     *s++ = 0;
02495     while ( *s == ' ' || *s == '\t' ) s++;
02496 }
02497 while ( *s == ',' ) s++;
02498 if ( *s != '"' ) {
02499     MesPrint("&Value for %s should be enclosed in double quotes",
02500             PreVar[numprevar].name);
02501     return(1);
02502 }
02503 s++; name = s; /* actually name points to the new string */
02504 while ( *s && *s != '"' ) { if ( *s == '\\' ) s++; s++; }
02505 if ( *s != '"' ) goto encl;
02506 *s = 0;
02507 code[0] = TYPEREDEFPRE; code[1] = numprevar;
02508
02509 /*
02510 AddComString(2, code, name, 0);
02511
02512 Add2ComStrings(2, code, name, args);
02513 *s = '"';
02514 #ifdef PARALLELLECODE
02515 */

```

```

02514     Now we prepare the input numbering system for pthreads.
02515     We need a list of preprocessor variables that are redefined in this
02516     module.
02517 */
02518 {
02519     int j;
02520     WORD *newpf;
02521     LONG *newin;
02522     for ( j = 0; j < AC.numpfirstnum; j++ ) {
02523         if ( numprevar == AC.pfirstnum[j] ) break;
02524     }
02525     if ( j >= AC.numpfirstnum ) { /* add to list */
02526         if ( j >= AC.sizepfirstnum ) {
02527             if ( AC.sizepfirstnum <= 0 ) { AC.sizepfirstnum = 10; }
02528             else { AC.sizepfirstnum = 2 * AC.sizepfirstnum; }
02529             newin = (LONG *)Malloc1(AC.sizepfirstnum*(sizeof(WORD)+sizeof(LONG)), "AC.pfirstnum");
02530             newpf = (WORD *) (newin+AC.sizepfirstnum);
02531             for ( j = 0; j < AC.numpfirstnum; j++ ) {
02532                 newpf[j] = AC.pfirstnum[j];
02533                 newin[j] = AC.inputnumbers[j];
02534             }
02535             if ( AC.inputnumbers ) M_free(AC.inputnumbers, "AC.pfirstnum");
02536             AC.inputnumbers = newin;
02537             AC.pfirstnum = newpf;
02538         }
02539         AC.pfirstnum[AC.numpfirstnum] = numprevar;
02540         AC.inputnumbers[AC.numpfirstnum] = -1;
02541         AC.numpfirstnum++;
02542     }
02543 }
02544 #endif
02545     return(0);
02546 illarg:;
02547     MesPrint("&Illegally formed name in argument of redefine statement");
02548     return(1);
02549 illargs:;
02550     MesPrint("&Illegally formed arguments in redefine statement");
02551     return(1);
02552 }
02553
02554 /*
02555     #] CoRedefine :
02556     #[ CoRenumber :
02557
02558     renumber    or renumber,0    Only exchanges (n^2 until no improvement)
02559     renumber,1    All permutations (could be slow)
02560 */
02561
02562 int CoRenumber(UBYTE *s)
02563 {
02564     int x;
02565     UBYTE *inp;
02566     while ( *s == ',' ) s++;
02567     inp = s;
02568     if ( *s == 0 ) { x = 0; }
02569     else ParseNumber(x,s)
02570     if ( *s == 0 && x >= 0 && x <= 1 ) {
02571         Add3Com(TYPERENNUMBER,x);
02572         return(0);
02573     }
02574     MesPrint("&Illegal argument in Renumber statement: '%s'",inp);
02575     return(1);
02576 }
02577
02578 /*
02579     #] CoRenumber :
02580     #[ CoSum :
02581 */
02582
02583 int CoSum(UBYTE *s)
02584 {
02585     CBUF *C = cbuf+AC.cbufnum;
02586     UBYTE *ss = 0, c, *t;
02587     int error = 0, i = 0, type, x;
02588     WORD numindex,number;
02589     while ( *s ) {
02590         t = s;
02591         if ( *s == '$' ) {
02592             t++; s++; while ( FG.cTable[*s] < 2 ) s++;
02593             c = *s; *s = 0;
02594             if ( ( number = GetDollar(t) ) < 0 ) {
02595                 MesPrint("&Undefined variable %s",t);
02596                 if ( !error ) error = 1;
02597                 number = AddDollar(t,0,0,0);
02598             }
02599             numindex = -number;
02600         }

```



```

02601     else {
02602         if ( ( s = SkipAName(s) ) == 0 ) return(1);
02603         c = *s; *s = 0;
02604         if ( ( ( type = GetOName(AC.exprnames,t,&numindex,NOAUTO) ) != NAMENOTFOUND )
02605         || ( ( type = GetOName(AC.varnames,t,&numindex,WITHAUTO) ) != CINDEX ) ) {
02606             if ( type != NAMENOTFOUND ) error = NameConflict( type,t);
02607             else {
02608                 MesPrint("&%s should have been declared as an index",t);
02609                 error = 1;
02610                 numindex = AddIndex(s,AC.lDefDim,AC.lDefDim4) + AM.OffsetIndex;
02611             }
02612         }
02613     }
02614     Add3Com(TYPESUM,numindex);
02615     i = 3; *s = c;
02616     if ( *s == 0 ) break;
02617     if ( *s != ',' ) {
02618         MesPrint("&Illegal separator between objects in sum statement.");
02619         return(1);
02620     }
02621     s++;
02622     if ( FG.cTable[*s] == 0 || *s == '[' || *s == '$' ) {
02623         while ( FG.cTable[*s] == 0 || *s == '[' || *s == '$' ) {
02624             if ( *s == '$' ) {
02625                 s++;
02626                 ss = t = s;
02627                 while ( FG.cTable[*s] < 2 ) s++;
02628                 c = *s; *s = 0;
02629                 if ( ( number = GetDollar(t) ) < 0 ) {
02630                     MesPrint("&Undefined variable %s",t);
02631                     if ( !error ) error = 1;
02632                     number = AddDollar(t,0,0,0);
02633                 }
02634                 numindex = -number;
02635             }
02636             else {
02637                 ss = t = s;
02638                 if ( ( s = SkipAName(s) ) == 0 ) return(1);
02639                 c = *s; *s = 0;
02640                 if ( ( ( type = GetOName(AC.exprnames,t,&numindex,NOAUTO) ) != NAMENOTFOUND )
02641                 || ( ( type = GetOName(AC.varnames,t,&numindex,WITHAUTO) ) != CINDEX ) ) {
02642                     if ( type != NAMENOTFOUND ) error = NameConflict( type,t);
02643                     else {
02644                         MesPrint("&%s should have been declared as an index",t);
02645                         error = 1;
02646                         numindex = AddIndex(s,AC.lDefDim,AC.lDefDim4) + AM.OffsetIndex;
02647                     }
02648                 }
02649             }
02650             AddToCB(C,numindex)
02651             i++;
02652             C->Pointer[-i+1] = i;
02653             *s = c;
02654             if ( *s == 0 ) return(error);
02655             if ( *s != ',' ) {
02656                 MesPrint("&Illegal separator between objects in sum statement.");
02657                 return(1);
02658             }
02659             s++;
02660         }
02661         if ( FG.cTable[*s] == 1 ) {
02662             C->Pointer[-i+1]--; C->Pointer--; s = ss;
02663         }
02664     }
02665     else if ( FG.cTable[*s] == 1 ) {
02666         while ( FG.cTable[*s] == 1 ) {
02667             t = s;
02668             x = *s++ - '0';
02669             while( FG.cTable[*s] == 1 ) x = 10*x + *s++ - '0';
02670             if ( *s && *s != ',' ) {
02671                 MesPrint("&%s is not a legal fixed index",t);
02672                 return(1);
02673             }
02674             else if ( x >= AM.OffsetIndex ) {
02675                 MesPrint("&%d is too large to be a fixed index",x);
02676                 error = 1;
02677             }
02678             else {
02679                 AddToCB(C,x)
02680                 i++;
02681                 C->Pointer[-i] = TYPESUMFIX;
02682                 C->Pointer[-i+1] = i;
02683             }
02684             if ( *s == 0 ) break;
02685             s++;
02686         }
02687     }

```

```

02688         else {
02689             MesPrint("&Illegal object in sum statement");
02690             error = 1;
02691         }
02692     }
02693     return(error);
02694 }
02695
02696 /*
02697 #] CoSum :
02698 #[ CoToTensor :
02699 */
02700
02701 static WORD cttarray[7] = { TYPEOPERATION,7,TENVEC,0,0,1,0 };
02702
02703 int CoToTensor(UBYTE *s)
02704 {
02705     UBYTE c, *t;
02706     int type, j, nargs, error = 0;
02707     WORD number, dol[2] = { 0, 0 };
02708     cttarray[1] = 6; /* length */
02709     cttarray[3] = 0; /* tensor */
02710     cttarray[4] = 0; /* vector */
02711     cttarray[5] = 1; /* option flags */
02712     /* cttarray[6] = 0; set veto */
02713     /*
02714      Count the number of the arguments. The validity of them is not checked here.
02715     */
02716     nargs = 0;
02717     t = s;
02718     for (;;) {
02719         while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
02720         if ( *s == 0 ) break;
02721         if ( *s == '!' ) {
02722             s++;
02723             if ( *s == '{' ) {
02724                 SKIPBRA2(s)
02725                 s++;
02726             } else {
02727                 if ( ( s = SkipAName(s) ) == 0 ) goto syntax_error;
02728             }
02729         } else {
02730             if ( ( s = SkipAName(s) ) == 0 ) goto syntax_error;
02731         }
02732         nargs++;
02733     }
02734     if ( nargs < 2 ) goto not_enough_arguments;
02735     s = t;
02736     /*
02737     Parse options, which are given as the arguments except the last two.
02738     */
02739     for ( j = 2; j < nargs; j++ ) {
02740         while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
02741         if ( *s == '!' ) {
02742             /*
02743              Handle !set or ![vector,...]. Note: If two or more sets are
02744              specified, then only the last one is used.
02745             */
02746             s++;
02747             cttarray[1] = 7;
02748             cttarray[5] |= 8;
02749             if ( FG.cTable[*s] == 0 || *s == '[' || *s == '_' ) {
02750                 t = s;
02751                 if ( ( s = SkipAName(s) ) == 0 ) goto syntax_error;
02752                 c = *s; *s = 0;
02753                 type = GetName(AC.varnames,t,&number,WITHAUTO);
02754                 if ( type == CVECTOR ) {
02755                     /*
02756                      As written in the manual, "!"p" (without "{}") should work.
02757                     */
02758                     cttarray[6] = DoTempSet(t,s);
02759                     *s = c;
02760                     goto check_tempset;
02761                 }
02762                 else if ( type != CSET ) {
02763                     MesPrint("&%s is not the name of a set or a vector",t);
02764                     error = 1;
02765                 }
02766                 *s = c;
02767                 cttarray[6] = number;
02768             }
02769             else if ( *s == '{' ) {
02770                 t = ++s; SKIPBRA2(s) *s = 0;
02771                 cttarray[6] = DoTempSet(t,s);
02772                 *s++ = ' ';
02773             }
02774             check_tempset:
02775             if ( cttarray[6] < 0 ) {

```

```

02775         error = 1;
02776     }
02777     if ( AC.wildflag ) {
02778         MesPrint("&Improper use of wildcard(s) in set specification");
02779         error = 1;
02780     }
02781 }
02782 } else {
02783 /*
02784     Other options.
02785 */
02786     t = s;
02787     if ( ( s = SkipAName(s) ) == 0 ) goto syntax_error;
02788     c = *s; *s = 0;
02789     if ( StrICmp(t, (UBYTE *)"nosquare") == 0 ) cttarray[5] |= 2;
02790     else if ( StrICmp(t, (UBYTE *)"functions") == 0 ) cttarray[5] |= 4;
02791     else {
02792         MesPrint("&Unrecognized option in ToTensor statement: '%s'",t);
02793         *s = c;
02794         return(1);
02795     }
02796     *s = c;
02797 }
02798 }
02799 /*
02800     Now parse a vector and a tensor. The ordering doesn't matter.
02801 */
02802 for ( j = 0; j < 2; j++ ) {
02803     while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
02804     t = s;
02805     if ( ( s = SkipAName(s) ) == 0 ) goto syntax_error;
02806     c = *s; *s = 0;
02807     if ( t[0] == '$' ) {
02808         dol[j] = GetDollar(t+1);
02809         if ( dol[j] < 0 ) dol[j] = AddDollar(t+1,DOLUNDEFINED,0,0);
02810     } else {
02811         type = GetName(AC.varnames,t,&number,WITHAUTO);
02812         if ( type == CVECTOR ) {
02813             cttarray[4] = number + AM.OffsetVector;
02814         }
02815         else if ( type == CFUNCTION && ( functions[number].spec > 0 ) ) {
02816             cttarray[3] = number + FUNCTION;
02817         }
02818         else {
02819             MesPrint("&%s is not a vector or a tensor",t);
02820             error = 1;
02821         }
02822     }
02823     *s = c;
02824 }
02825 if ( cttarray[3] == 0 || cttarray[4] == 0 ) {
02826     if ( dol[0] == 0 && dol[1] == 0 ) {
02827         goto not_enough_arguments;
02828     }
02829     else if ( cttarray[3] ) {
02830         if ( dol[1] ) cttarray[4] = dol[1];
02831         else if ( dol[0] ) cttarray[4] = dol[0];
02832         else {
02833             goto not_enough_arguments;
02834         }
02835     }
02836     else if ( cttarray[4] ) {
02837         if ( dol[1] ) cttarray[3] = -dol[1];
02838         else if ( dol[0] ) cttarray[3] = -dol[0];
02839         else {
02840             goto not_enough_arguments;
02841         }
02842     }
02843     else {
02844         if ( dol[0] == 0 || dol[1] == 0 ) {
02845             goto not_enough_arguments;
02846         }
02847         else {
02848             cttarray[3] = -dol[0]; cttarray[4] = dol[1];
02849         }
02850     }
02851 }
02852 AddNtoL(cttarray[1],cttarray);
02853 return(error);
02854
02855 syntax_error:
02856     MesPrint("&Syntax error in ToTensor statement");
02857     return(1);
02858
02859 not_enough_arguments:
02860     MesPrint("&ToTensor statement needs a vector and a tensor");
02861     return(1);

```

```

02862 }
02863
02864 /*
02865     #] CoToTensor :
02866     #[ CoToVector :
02867 */
02868
02869 static WORD ctvarray[6] = { TYPEOPERATION,6,TENVEC,0,0,0 };
02870
02871 int CoToVector(UBYTE *s)
02872 {
02873     UBYTE *t, c;
02874     int j, type, error = 0;
02875     WORD number, dol[2];
02876     dol[0] = dol[1] = 0;
02877     ctvarray[3] = ctvarray[4] = ctvarray[5] = 0;
02878     for ( j = 0; j < 2; j++ ) {
02879         t = s;
02880         if ( ( s = SkipAName(s) ) == 0 ) {
02881 proper:      MesPrint("&Arguments of ToVector statement should be a vector and a tensor");
02882             return(1);
02883         }
02884         c = *s; *s = 0;
02885         if ( *t == '$' ) {
02886             dol[j] = GetDollar(t+1);
02887             if ( dol[j] < 0 ) dol[j] = AddDollar(t+1,DOLUNDEFINED,0,0);
02888         }
02889         else if ( ( type = GetName(AC.varnames,t,&number,WITHAUTO) ) == CVECTOR )
02890             ctvarray[4] = number + AM.OffsetVector;
02891         else if ( type == CFUNCTION && ( functions[number].spec > 0 ) )
02892             ctvarray[3] = number+FUNCTION;
02893         else {
02894             MesPrint("&%s is not a vector or a tensor",t);
02895             error = 1;
02896         }
02897         *s = c; if ( *s && *s != ',' ) goto proper;
02898         if ( *s ) s++;
02899     }
02900     if ( *s != 0 ) goto proper;
02901     if ( ctvarray[3] == 0 || ctvarray[4] == 0 ) {
02902         if ( dol[0] == 0 && dol[1] == 0 ) {
02903             MesPrint("&ToVector statement needs a vector and a tensor");
02904             error = 1;
02905         }
02906         else if ( ctvarray[3] ) {
02907             if ( dol[1] ) ctvarray[4] = dol[1];
02908             else if ( dol[0] ) ctvarray[4] = dol[0];
02909             else {
02910                 MesPrint("&ToVector statement needs a vector and a tensor");
02911                 error = 1;
02912             }
02913         }
02914         else if ( ctvarray[4] ) {
02915             if ( dol[1] ) ctvarray[3] = -dol[1];
02916             else if ( dol[0] ) ctvarray[3] = -dol[0];
02917             else {
02918                 MesPrint("&ToVector statement needs a vector and a tensor");
02919                 error = 1;
02920             }
02921         }
02922         else {
02923             if ( dol[0] == 0 || dol[1] == 0 ) {
02924                 MesPrint("&ToVector statement needs a vector and a tensor");
02925                 error = 1;
02926             }
02927             else {
02928                 ctvarray[3] = -dol[0]; ctvarray[4] = dol[1];
02929             }
02930         }
02931     }
02932     AddNtoL(6,ctvarray);
02933     return(error);
02934 }
02935
02936 /*
02937     #] CoToVector :
02938     #[ CoTrace4 :
02939 */
02940
02941 int CoTrace4(UBYTE *s)
02942 {
02943     int error = 0, type, option = CHISHOLM;
02944     UBYTE *t, c;
02945     WORD numindex, one = 1;
02946     KEYWORD *key;
02947     for (;;) {
02948         t = s;

```

```

02949         if ( FG.cTable[*s] == 1 ) break;
02950         if ( ( s = SkipAName(s) ) == 0 ) {
02951 proper:    MesPrint("&Proper syntax for Trace4 is 'Trace4[,options],index;'");
02952             return(1);
02953         }
02954         if ( *s == 0 ) break;
02955         c = *s; *s = 0;
02956         if ( ( key = FindKeyWord(t,trace4options,
02957             sizeof(trace4options)/sizeof(KEYWORD)) ) == 0 ) break;
02958         else {
02959             option |= key->type;
02960             option &= ~key->flags;
02961         }
02962         if ( ( *s++ = c ) != ',' ) {
02963             MesPrint("&Illegal separator in Trace4 statement");
02964             return(1);
02965         }
02966         if ( *s == 0 ) goto proper;
02967     }
02968     s = t;
02969     if ( FG.cTable[*s] == 1 ) {
02970 retry:
02971         ParseNumber(numindex,s)
02972         if ( *s != 0 ) {
02973             MesPrint("&Last argument of Trace4 should be an index");
02974             return(1);
02975         }
02976         if ( numindex >= AM.OffsetIndex ) {
02977             MesPrint("&fixed index >= %d. Change value of OffsetIndex in setup file"
02978                 ,AM.OffsetIndex);
02979             return(1);
02980         }
02981     }
02982     else if ( *s == '$' ) {
02983         if ( ( type = GetName(AC.dollarnames,s+1,&numindex,NOAUTO) ) == CDOLLAR )
02984             numindex = -numindex;
02985         else {
02986             MesPrint("&%s is undefined",s);
02987             numindex = AddDollar(s+1,DOLINDEX,&one,1);
02988             return(1);
02989         }
02990 tests:   s = SkipAName(s);
02991         if ( *s != 0 ) {
02992             MesPrint("&Trace4 should have a single index or $variable for its argument");
02993             return(1);
02994         }
02995     }
02996     else if ( ( type = GetName(AC.varnames,s,&numindex,WITHAUTO) ) == CINDEX ) {
02997         numindex += AM.OffsetIndex;
02998         goto tests;
02999     }
03000     else if ( type != -1 ) {
03001         if ( type != CDUBIOUS ) {
03002             if ( ( FG.cTable[*s] != 0 ) && ( *s != '[' ) ) {
03003                 if ( *s == '+' && FG.cTable[s[1]] == 1 ) { s++; goto retry; }
03004                 goto proper;
03005             }
03006             NameConflict(type,s);
03007             type = MakeDubious(AC.varnames,s,&numindex);
03008         }
03009         return(1);
03010     }
03011     else {
03012         MesPrint("&%s is not an index",s);
03013         numindex = AddIndex(s,AC.lDefDim,AC.lDefDim4) + AM.OffsetIndex;
03014         return(1);
03015     }
03016     if ( error ) return(error);
03017     if ( ( option & CHISHOLM ) != 0 )
03018         Add4Com(TYPECHISHOLM,numindex,(option & ALSOREVERSE));
03019     Add5Com(TYPEOPERATION,TAKETRACE,4 + (option & NOTRICK),numindex);
03020     return(0);
03021 }
03022
03023 /*
03024 #] CoTrace4 :
03025 #[ CoTraceN :
03026 */
03027
03028 int CoTraceN(UBYTE *s)
03029 {
03030     WORD numindex, one = 1;
03031     int type;
03032     if ( FG.cTable[*s] == 1 ) {
03033 retry:
03034         ParseNumber(numindex,s)
03035         if ( *s != 0 ) {

```

```

03036 proper:      MesPrint("&TraceN should have a single index for its argument");
03037               return(1);
03038           }
03039           if ( numindex >= AM.OffsetIndex ) {
03040               MesPrint("&fixed index >= %d. Change value of OffsetIndex in setup file"
03041                       ,AM.OffsetIndex);
03042               return(1);
03043           }
03044       }
03045       else if ( *s == '$' ) {
03046           if ( ( type = GetName(AC.dollarnames,s+1,&numindex,NOAUTO) ) == CDOLLAR )
03047               numindex = -numindex;
03048           else {
03049               MesPrint("&%s is undefined",s);
03050               numindex = AddDollar(s+1,DOLINDEX,&one,1);
03051               return(1);
03052           }
03053       tests:  s = SkipAName(s);
03054           if ( *s != 0 ) {
03055               MesPrint("&TraceN should have a single index or $variable for its argument");
03056               return(1);
03057           }
03058       }
03059       else if ( ( type = GetName(AC.varnames,s,&numindex,WITHAUTO) ) == CINDEX ) {
03060           numindex += AM.OffsetIndex;
03061           goto tests;
03062       }
03063       else if ( type != -1 ) {
03064           if ( type != CDUBIOUS ) {
03065               if ( ( FG.cTable[*s] != 0 ) && ( *s != '[' ) ) {
03066                   if ( *s == '+' && FG.cTable[s[1]] == 1 ) { s++; goto retry; }
03067                   goto proper;
03068               }
03069               NameConflict(type,s);
03070               type = MakeDubious(AC.varnames,s,&numindex);
03071           }
03072           return(1);
03073       }
03074       else {
03075           MesPrint("&%s is not an index",s);
03076           numindex = AddIndex(s,AC.lDefDim,AC.lDefDim4) + AM.OffsetIndex;
03077           return(1);
03078       }
03079       Add5Com(TYPEOPERATION,TAKETRACE,0,numindex);
03080       return(0);
03081   }
03082   /*
03083   #] CoTraceN :
03084   #[ CoChisholm :
03085   */
03086   /*
03087   int CoChisholm(UBYTE *s)
03088   {
03089       int error = 0, type, option = CHISHOLM;
03090       UBYTE *t, c;
03091       WORD numindex, one = 1;
03092       KEYWORD *key;
03093       for (;;) {
03094           t = s;
03095           if ( FG.cTable[*s] == 1 ) break;
03096           if ( ( s = SkipAName(s) ) == 0 ) {
03097               proper:  MesPrint("&Proper syntax for Chisholm is 'Chisholm[,options],index;'");
03098                       return(1);
03099           }
03100           if ( *s == 0 ) break;
03101           c = *s; *s = 0;
03102           if ( ( key = FindKeyWord(t,chisoptions,
03103                                   sizeof(chisoptions)/sizeof(KEYWORD)) ) == 0 ) break;
03104           else {
03105               option |= key->type;
03106               option &= ~key->flags;
03107           }
03108           if ( ( *s++ = c ) != ',' ) {
03109               MesPrint("&Illegal separator in Chisholm statement");
03110               return(1);
03111           }
03112           if ( *s == 0 ) goto proper;
03113       }
03114       s = t;
03115       if ( FG.cTable[*s] == 1 ) {
03116           ParseNumber(numindex,s)
03117           if ( *s != 0 ) {
03118               MesPrint("&Last argument of Chisholm should be an index");
03119               return(1);
03120           }
03121       }
03122       if ( numindex >= AM.OffsetIndex ) {

```

```

03123         MesPrint("&fixed index >= %d. Change value of OffsetIndex in setup file"
03124         ,AM.OffsetIndex);
03125         return(1);
03126     }
03127 }
03128 else if ( *s == '$' ) {
03129     if ( ( type = GetName(AC.dollarnames,s+1,&numindex,NOAUTO) ) == CDOLLAR )
03130         numindex = -numindex;
03131     else {
03132         MesPrint("&%s is undefined",s);
03133         numindex = AddDollar(s+1,DOLINDEX,&one,1);
03134         return(1);
03135     }
03136 tests: s = SkipAName(s);
03137     if ( *s != 0 ) {
03138         MesPrint("&Chisholm should have a single index or $variable for its argument");
03139         return(1);
03140     }
03141 }
03142 else if ( ( type = GetName(AC.varnames,s,&numindex,WITHAUTO) ) == CINDEX ) {
03143     numindex += AM.OffsetIndex;
03144     goto tests;
03145 }
03146 else if ( type != -1 ) {
03147     if ( type != CDUBIOUS ) {
03148         NameConflict(type,s);
03149         type = MakeDubious(AC.varnames,s,&numindex);
03150     }
03151     return(1);
03152 }
03153 else {
03154     MesPrint("&%s is not an index",s);
03155     numindex = AddIndex(s,AC.lDefDim,AC.lDefDim4) + AM.OffsetIndex;
03156     return(1);
03157 }
03158 if ( error ) return(error);
03159 Add4Com(TYPECHISHOLM,numindex,(option & ALSOREVERSE));
03160 return(0);
03161 }
03162
03163 /*
03164 #] CoChisholm :
03165 #[ DoChain :
03166
03167 Syntax: Chainxx functionname;
03168 */
03169
03170 int DoChain(UBYTE *s, int option)
03171 {
03172     WORD numfunc,type;
03173     if ( *s == '$' ) {
03174         if ( ( type = GetName(AC.dollarnames,s+1,&numfunc,NOAUTO) ) == CDOLLAR )
03175             numfunc = -numfunc;
03176         else {
03177             MesPrint("&%s is undefined",s);
03178             numfunc = AddDollar(s+1,DOLINDEX,&one,1);
03179             return(1);
03180         }
03181 tests: s = SkipAName(s);
03182     if ( *s != 0 ) {
03183         MesPrint("&ChainIn/ChainOut should have a single function or $variable for its argument");
03184         return(1);
03185     }
03186 }
03187 else if ( ( type = GetName(AC.varnames,s,&numfunc,WITHAUTO) ) == CFUNCTION ) {
03188     numfunc += FUNCTION;
03189     goto tests;
03190 }
03191 else if ( type != -1 ) {
03192     if ( type != CDUBIOUS ) {
03193         NameConflict(type,s);
03194         type = MakeDubious(AC.varnames,s,&numfunc);
03195     }
03196     return(1);
03197 }
03198 else {
03199     MesPrint("&%s is not a function",s);
03200     numfunc = AddFunction(s,0,0,0,0,0,-1,-1) + FUNCTION;
03201     return(1);
03202 }
03203 Add3Com(option,numfunc);
03204 return(0);
03205 }
03206
03207 /*
03208 #] DoChain :
03209 #[ CoChainin :

```

```

03210
03211     Syntax: Chainin functionname;
03212 */
03213
03214 int CoChainin(UBYTE *s)
03215 {
03216     return(DoChain(s,TYPECHAININ));
03217 }
03218
03219 /*
03220     #] CoChainin :
03221     #[ CoChainout :
03222
03223     Syntax: Chainout functionname;
03224 */
03225
03226 int CoChainout(UBYTE *s)
03227 {
03228     return(DoChain(s,TYPECHAINOUT));
03229 }
03230
03231 /*
03232     #] CoChainout :
03233     #[ CoExit :
03234 */
03235
03236 int CoExit(UBYTE *s)
03237 {
03238     UBYTE *name;
03239     WORD code = TYPEEXIT;
03240     while ( *s == ',' ) s++;
03241     if ( *s == 0 ) {
03242         Add3Com(TYPEEXIT,0);
03243         return(0);
03244     }
03245     name = s+1;
03246     s++;
03247     while ( *s ) { if ( *s == '\\' ) s++; s++; }
03248     if ( name[-1] != '"' || s[-1] != '"' ) {
03249         MesPrint("&Illegal syntax for exit statement");
03250         return(1);
03251     }
03252     s[-1] = 0;
03253     AddComString(1,&code,name,0);
03254     s[-1] = '"';
03255     return(0);
03256 }
03257
03258 /*
03259     #] CoExit :
03260     #[ CoInParallel :
03261 */
03262
03263 int CoInParallel(UBYTE *s)
03264 {
03265     return(DoInParallel(s,1));
03266 }
03267
03268 /*
03269     #] CoInParallel :
03270     #[ CoNotInParallel :
03271 */
03272
03273 int CoNotInParallel(UBYTE *s)
03274 {
03275     return(DoInParallel(s,0));
03276 }
03277
03278 /*
03279     #] CoNotInParallel :
03280     #[ DoInParallel :
03281
03282     InParallel;
03283     InParallel,names;
03284     NotInParallel;
03285     NotInParallel,names;
03286 */
03287
03288 int DoInParallel(UBYTE *s, int par)
03289 {
03290     #ifdef PARALLELCODE
03291         EXPRESSIONS e;
03292         WORD i;
03293     #endif
03294     WORD number;
03295     UBYTE *t, c;
03296     int error = 0;

```



```

03297 #ifndef WITHPTHREADS
03298     DUMMYUSE(par);
03299 #endif
03300     if ( *s == 0 ) {
03301         AC.inparallelflag = par;
03302 #ifdef PARALLELCODE
03303         for ( i = NumExpressions-1; i >= 0; i-- ) {
03304             e = Expressions+i;
03305             if ( e->status == LOCALEXPRESSION || e->status == GLOBALEXPRESSION
03306                 || e->status == UNHIDELEXPRESSION || e->status == UNHIDEGEXPRESSION
03307             ) {
03308                 e->partodo = par;
03309             }
03310         }
03311 #endif
03312     }
03313     else {
03314         for(;;) { /* Look for a (comma separated) list of variables */
03315             while ( *s == ',' ) s++;
03316             if ( *s == 0 ) break;
03317             if ( *s == '[' || FG.cTable[*s] == 0 ) {
03318                 t = s;
03319                 if ( ( s = SkipAName(s) ) == 0 ) {
03320                     MesPrint("&Improper name for an expression: '%s'",t);
03321                     return(1);
03322                 }
03323                 c = *s; *s = 0;
03324                 if ( GetName(AC.exprnames,t,&number,NOAUTO) == CEXPRESSION ) {
03325 #ifdef PARALLELCODE
03326                     e = Expressions+number;
03327                     if ( e->status == LOCALEXPRESSION || e->status == GLOBALEXPRESSION
03328                         || e->status == UNHIDELEXPRESSION || e->status == UNHIDEGEXPRESSION
03329                     ) {
03330                         e->partodo = par;
03331                     }
03332 #endif
03333                 }
03334                 else if ( GetName(AC.varnames,t,&number,NOAUTO) != NAMENOTFOUND ) {
03335                     MesPrint("&%%s is not an expression",t);
03336                     error = 1;
03337                 }
03338                 *s = c;
03339             }
03340             else {
03341                 MesPrint("&Illegal object in InExpression statement");
03342                 error = 1;
03343                 while ( *s && *s != ',' ) s++;
03344                 if ( *s == 0 ) break;
03345             }
03346         }
03347     }
03348     }
03349     return(error);
03350 }
03351
03352 /*
03353 #] DoInParallel :
03354 #[ CoInExpression :
03355 */
03356
03357 int CoInExpression(UBYTE *s)
03358 {
03359     GETIDENTITY
03360     UBYTE *t, c;
03361     WORD *w, number;
03362     int error = 0;
03363     w = AT.WorkPointer;
03364     if ( AC.inexprlevel >= MAXNEST ) {
03365         MesPrint("@Nesting of inexpression statements more than %d levels", (WORD)MAXNEST);
03366         return(-1);
03367     }
03368     AC.inexprsumcheck[AC.inexprlevel] = NestingChecksum();
03369     AC.inexprstack[AC.inexprlevel] = cbuf[AC.cbunum].Pointer
03370         - cbuf[AC.cbunum].Buffer + 2;
03371     AC.inexprlevel++;
03372     *w++ = TYPEINEXPRESSION;
03373     w++; w++;
03374     for(;;) { /* Look for a (comma separated) list of variables */
03375         while ( *s == ',' ) s++;
03376         if ( *s == 0 ) break;
03377         if ( *s == '[' || FG.cTable[*s] == 0 ) {
03378             t = s;
03379             if ( ( s = SkipAName(s) ) == 0 ) {
03380                 MesPrint("&Improper name for an expression: '%s'",t);
03381                 return(1);
03382             }
03383             c = *s; *s = 0;

```

```

03384         if ( GetName(AC.exprnames,t,&number,NOAUTO) == CEXPRESSION ) {
03385             *w++ = number;
03386         }
03387         else if ( GetName(AC.varnames,t,&number,NOAUTO) != NAMENOTFOUND ) {
03388             MesPrint("& %s is not an expression",t);
03389             error = 1;
03390         }
03391         *s = c;
03392     }
03393     else {
03394         MesPrint("& Illegal object in InExpression statement");
03395         error = 1;
03396         while ( *s && *s != ',' ) s++;
03397         if ( *s == 0 ) break;
03398     }
03399 }
03400 AT.WorkPointer[1] = w - AT.WorkPointer;
03401 AddNtoL(AT.WorkPointer[1],AT.WorkPointer);
03402 return(error);
03403 }
03404
03405 /*
03406 #] CoInExpression :
03407 #[ CoEndInExpression :
03408 */
03409 int CoEndInExpression(UBYTE *s)
03410 {
03411     CBUF *C = cbuf+AC.cbunum;
03412     while ( *s == ',' ) s++;
03413     if ( *s ) {
03414         MesPrint("& Illegal syntax for EndInExpression statement");
03415         return(1);
03416     }
03417     if ( AC.inexprlevel <= 0 ) {
03418         MesPrint("& EndInExpression without corresponding InExpression statement");
03419         return(1);
03420     }
03421     AC.inexprlevel--;
03422     cbuf[AC.cbunum].Buffer[AC.inexprstack[AC.inexprlevel]] = C->numlhs;
03423     if ( AC.inexprsumcheck[AC.inexprlevel] != NestingChecksum() ) {
03424         MesNesting();
03425         return(1);
03426     }
03427     return(0);
03428 }
03429 }
03430
03431 /*
03432 #] CoEndInExpression :
03433 #[ CoSetExitFlag :
03434 */
03435 int CoSetExitFlag(UBYTE *s)
03436 {
03437     if ( *s ) {
03438         MesPrint("& Illegal syntax for the SetExitFlag statement");
03439         return(1);
03440     }
03441     Add2Com(TYPESETEXIT);
03442     return(0);
03443 }
03444 }
03445
03446 /*
03447 #] CoSetExitFlag :
03448 #[ CoTryReplace :
03449 */
03450 int CoTryReplace(UBYTE *p)
03451 {
03452     GETIDENTITY
03453     UBYTE *name, c;
03454     WORD *w, error = 0, i, which = -1, c1, minvec = 0;
03455     w = AT.WorkPointer;
03456     *w++ = TYPETRY;
03457     *w++ = 3;
03458     *w++ = 0;
03459     *w++ = REPLACEMENT;
03460     *w++ = FUNHEAD;
03461     FILLFUN(w)
03462     /*
03463     Now we have to read a function argument for the replace_ function.
03464     Current arguments that we allow involve only single arguments
03465     that do not expand further. No brackets!
03466     */
03467     while ( *p ) {
03468         /*
03469             No numbers yet
03470         */

```

```

03471     if ( *p == '-' && minvec == 0 && which == (CVECTOR+1) ) {
03472         minvec = 1; p++;
03473     }
03474     if ( *p == '[' || FG.cTable[*p] == 0 ) {
03475         name = p;
03476         if ( ( p = SkipAName(p) ) == 0 ) return(1);
03477         c = *p; *p = 0;
03478         i = GetName(AC.varnames,name,&c1,WITHAUTO);
03479         if ( which >= 0 && i >= 0 && i != CDUBIOUS && which != (i+1) ) {
03480             MesPrint("&Illegal combination of objects in TryReplace");
03481             error = 1;
03482         }
03483         else if ( minvec && i != CVECTOR && i != CDUBIOUS ) {
03484             MesPrint("&Currently a - sign can be used only with a vector in TryReplace");
03485             error = 1;
03486         }
03487         else switch ( i ) {
03488             case CSYMBOL: *w++ = -SYMBOL; *w++ = c1; break;
03489             case CVECTOR:
03490                 if ( minvec ) *w++ = -MINVECTOR;
03491                 else *w++ = -VECTOR;
03492                 *w++ = c1 + AM.OffsetVector;
03493                 minvec = 0;
03494                 break;
03495             case CINDEXX: *w++ = -INDEX; *w++ = c1 + AM.OffsetIndex;
03496                 if ( c1 >= AM.WilInd && c == '?' ) { *p++ = c; c = *p; }
03497                 break;
03498             case CFUNCTION: *w++ = -c1-FUNCTION; break;
03499             case CDUBIOUS: minvec = 0; error = 1; break;
03500             default:
03501                 MesPrint("&Illegal object type in TryReplace: %s",name);
03502                 error = 1;
03503                 i = 0;
03504                 break;
03505         }
03506         if ( which < 0 ) which = i+1;
03507         else which = -1;
03508         *p = c;
03509         if ( *p == ',' ) p++;
03510         continue;
03511     }
03512     else {
03513         MesPrint("&Illegal object in TryReplace");
03514         error = 1;
03515         while ( *p && *p != ',' ) {
03516             if ( *p == '(' ) SKIPBRA3(p)
03517             else if ( *p == '{' ) SKIPBRA2(p)
03518             else if ( *p == '[' ) SKIPBRA1(p)
03519             else p++;
03520         }
03521     }
03522     if ( *p == ',' ) p++;
03523     if ( which < 0 ) which = 0;
03524     else which = -1;
03525 }
03526 if ( which >= 0 ) {
03527     MesPrint("&Odd number of arguments in TryReplace");
03528     error = 1;
03529 }
03530 i = w - AT.WorkPointer;
03531 AT.WorkPointer[1] = i;
03532 AT.WorkPointer[2] = i - 3;
03533 AT.WorkPointer[4] = i - 3;
03534 AddNtoL((int)i,AT.WorkPointer);
03535 return(error);
03536 }
03537
03538 /*
03539 #] CoTryReplace :
03540 #[ CoModulus :
03541
03542 Old syntax: Modulus [-] number [:number]
03543 New syntax: Modulus [option(s)] number
03544 Options are: NoFunctions/CoefficientsOnly/AlsoFunctions
03545             PlusMin/Positive
03546             InverseTable
03547             PrintPowersOf(number)
03548             AlsoPowers/NoPowers
03549             AlsoDollars/NoDollars
03550 Notice: We change the defaults. This may cause problems to some.
03551 */
03552
03553 int CoModulus(UBYTE *inp)
03554 {
03555     GETIDENTITY
03556     int Retval = 0, sign = 1;
03557     UBYTE *p, c;

```

```

03558     while ( *inp == ',' || *inp == ' ' || *inp == '\t' ) inp++;
03559     if ( *inp == 0 ) {
03560 SwitchOff:
03561         if ( AC.modpowers ) M_free(AC.modpowers,"AC.modpowers");
03562         AC.modpowers = 0;
03563         AN.ncmod = AC.ncmod = 0;
03564         if ( AC.halfmod ) M_free(AC.halfmod,"halfmod");
03565         AC.halfmod = 0; AC.nhalfmod = 0;
03566         if ( AC.modinverses ) M_free(AC.modinverses,"modinverses");
03567         AC.modinverses = 0;
03568         AC.modmode = 0;
03569         return(0);
03570     }
03571 #ifdef WITHFLOAT
03572     if ( AT.aux_ != 0 ) {
03573         MesPrint("&Simultaneous use of floating point and modulus arithmetic makes no sense.");
03574         Retval = 1;
03575     }
03576 #endif
03577     AC.modmode = 0;
03578     if ( *inp == '-' ) {
03579         sign = -1;
03580         inp++;
03581     }
03582     else {
03583         while ( FG.cTable[*inp] == 0 ) {
03584             p = inp;
03585             while ( FG.cTable[*inp] == 0 ) inp++;
03586             c = *inp; *inp = 0;
03587             if ( StrICmp(p,(UBYTE *)"nofunctions") == 0 ) {
03588                 AC.modmode &= ~ALSOFUNARGS;
03589             }
03590             else if ( StrICmp(p,(UBYTE *)"alsofunctions") == 0 ) {
03591                 AC.modmode |= ALSOFUNARGS;
03592             }
03593             else if ( StrICmp(p,(UBYTE *)"coefficientsonly") == 0 ) {
03594                 AC.modmode &= ~ALSOFUNARGS;
03595                 AC.modmode &= ~ALSOPOWERS;
03596                 sign = -1;
03597             }
03598             else if ( StrICmp(p,(UBYTE *)"plusmin") == 0 ) {
03599                 AC.modmode |= POSNEG;
03600             }
03601             else if ( StrICmp(p,(UBYTE *)"positive") == 0 ) {
03602                 AC.modmode &= ~POSNEG;
03603             }
03604             else if ( StrICmp(p,(UBYTE *)"inversetable") == 0 ) {
03605                 AC.modmode |= INVERSETABLE;
03606             }
03607             else if ( StrICmp(p,(UBYTE *)"noinversetable") == 0 ) {
03608                 AC.modmode &= ~INVERSETABLE;
03609             }
03610             else if ( StrICmp(p,(UBYTE *)"nodollars") == 0 ) {
03611                 AC.modmode &= ~ALSODOLLARS;
03612             }
03613             else if ( StrICmp(p,(UBYTE *)"alsodollars") == 0 ) {
03614                 AC.modmode |= ALSODOLLARS;
03615             }
03616             else if ( StrICmp(p,(UBYTE *)"printpowersof") == 0 ) {
03617                 *inp = c;
03618                 if ( *inp != '(' ) {
03619 badsyntax:
03620                     MesPrint("&Bad syntax in argument of PrintPowersOf(number) in Modulus statement");
03621                     return(1);
03622                 }
03623                 while ( *inp == ',' || *inp == ' ' || *inp == '\t' ) inp++;
03624                 inp++; p = inp;
03625                 if ( FG.cTable[*inp] != 1 ) goto badsyntax;
03626                 do { inp++; } while ( FG.cTable[*inp] == 1 );
03627                 c = *inp; *inp = 0;
03628                 if ( GetLong(p,(UWORD *)AC.powmod,&AC.npowmod) ) Retval = -1;
03629                 if ( TakeModulus((UWORD *)AC.powmod,&AC.npowmod,AC.cmod,AC.ncmod,NOUNPACK) ) Retval = -1;
03630                 if ( AC.npowmod == 0 ) {
03631                     MesPrint("&Improper value for generator");
03632                     Retval = -1;
03633                 }
03634                 if ( MakeModTable() ) Retval = -1;
03635                 AC.DirtPow = 1;
03636                 *inp = c;
03637                 while ( *inp == ',' || *inp == ' ' || *inp == '\t' ) inp++;
03638                 if ( *inp != '(' ) goto badsyntax;
03639                 inp++;
03640                 c = *inp;
03641             }
03642             else if ( StrICmp(p,(UBYTE *)"alsopowers") == 0 ) {
03643                 AC.modmode |= ALSOPOWERS;
03644                 sign = 1;

```

```

03645     }
03646     else if ( StrICmp(p, (UBYTE *) "nopowers") == 0 ) {
03647         AC.modmode &= ~ALSOPOWERS;
03648         sign = -1;
03649     }
03650     else {
03651         MesPrint("&Unrecognized option %s in Modulus statement", inp);
03652         return(1);
03653     }
03654     *inp = c;
03655     while ( *inp == ',' || *inp == ' ' || *inp == '\t' ) inp++;
03656     if ( *inp == 0 ) {
03657         MesPrint("&Modulus statement with no value!!!");
03658         return(1);
03659     }
03660 }
03661 }
03662 p = inp;
03663 if ( FG.cTable[*inp] != 1 ) {
03664     MesPrint("&Invalid value for modulus:%s", inp);
03665     if ( AC.modpowers ) M_free(AC.modpowers, "AC.modpowers");
03666     AC.modpowers = 0;
03667     AN.ncmod = AC.ncmod = 0;
03668     if ( AC.halfmod ) M_free(AC.halfmod, "halfmod");
03669     AC.halfmod = 0; AC.nhalfmod = 0;
03670     if ( AC.modinverses ) M_free(AC.modinverses, "modinverses");
03671     AC.modinverses = 0;
03672     return(1);
03673 }
03674 do { inp++; } while ( FG.cTable[*inp] == 1 );
03675 c = *inp; *inp = 0;
03676 Retval = GetLong(p, (UWORD *) AC.cmod, &AC.ncmod);
03677 if ( Retval == 0 && AC.ncmod == 0 ) goto SwitchOff;
03678 if ( sign < 0 ) AC.ncmod = -AC.ncmod;
03679 AN.ncmod = AC.ncmod;
03680 if ( ( AC.modmode & INVERSETABLE ) != 0 ) MakeInverses();
03681 if ( AC.halfmod ) M_free(AC.halfmod, "halfmod");
03682 AC.halfmod = 0; AC.nhalfmod = 0;
03683 return(Retval);
03684 }
03685
03686 /*
03687 #] CoModulus :
03688 #[ CoRepeat :
03689 */
03690
03691 int CoRepeat(UBYTE *inp)
03692 {
03693     int error = 0;
03694     AC.RepSumCheck[AC.RepLevel] = NestingChecksum();
03695     AC.RepLevel++;
03696     if ( AC.RepLevel > AM.RepMax ) {
03697         MesPrint("&Too many repeat levels. Maximum is %d", AM.RepMax);
03698         return(1);
03699     }
03700     Add3Com(TYPEREPEAT, -1) /* Means indefinite */
03701     while ( *inp == ' ' || *inp == ',' || *inp == '\t' ) inp++;
03702     if ( *inp ) {
03703         error = CompileStatement(inp);
03704         if ( CoEndRepeat(inp) ) error = 1;
03705     }
03706     return(error);
03707 }
03708
03709 /*
03710 #] CoRepeat :
03711 #[ CoEndRepeat :
03712 */
03713
03714 int CoEndRepeat(UBYTE *inp)
03715 {
03716     CBUF *C = cbuf+AC.cbunum;
03717     int level, error = 0, repeatlevel = 0;
03718     DUMMYUSE(inp);
03719     AC.RepLevel--;
03720     if ( AC.RepLevel < 0 ) {
03721         MesPrint("&EndRepeat without Repeat");
03722         AC.RepLevel = 0;
03723         return(1);
03724     }
03725     else if ( AC.RepSumCheck[AC.RepLevel] != NestingChecksum() ) {
03726         MesNesting();
03727         error = 1;
03728     }
03729     level = C->numlhs+1;
03730     while ( level > 0 ) {
03731         if ( C->lhs[--level][0] == TYPEREPEAT ) {

```

```

03732         if ( repeatlevel == 0 ) {
03733             Add3Com(TYPEENDREPEAT,level)
03734             return(error);
03735         }
03736         repeatlevel--;
03737     }
03738     else if ( C->lhs[level][0] == TYPEENDREPEAT ) repeatlevel++;
03739 }
03740 return(1);
03741 }
03742
03743 /*
03744 #] CoEndRepeat :
03745 #[ DoBrackets :
03746
03747     Reads in the bracket information.
03748     Storage is in the form of a regular term.
03749     No subterms and arguments are allowed.
03750 */
03751
03752 int DoBrackets(UBYTE *inp, int par)
03753 {
03754     GETIDENTITY
03755     UBYTE *p, *pp, c;
03756     WORD *to, i, type, *w, error = 0;
03757     WORD c1,c2, *WorkSave;
03758     int bflag;
03759     p = inp;
03760     WorkSave = to = AT.WorkPointer;
03761     to++;
03762     if ( AT.BrackBuf == 0 ) {
03763         AR.MaxBracket = 100;
03764         AT.BrackBuf = (WORD *)Malloca(sizeof(WORD)*(AR.MaxBracket+1),"bracket buffer");
03765     }
03766     *AT.BrackBuf = 0;
03767     AR.BracketOn = 0;
03768     AC.bracketindexflag = 0;
03769     AT.bracketindexflag = 0;
03770     if ( *p == '+' || *p == '-' ) p++;
03771     if ( p[-1] == ',' && *p ) p--;
03772     if ( p[-1] == '+' && *p ) { bflag = 1; if ( *p != ',' ) { *--p = ','; } }
03773     else if ( p[-1] == '-' && *p ) { bflag = -1; if ( *p != ',' ) { *--p = ','; } }
03774     else bflag = 0;
03775     while ( *p == ',' ) {
03776 redo: AR.BracketOn++;
03777         while ( *p == ',' ) p++;
03778         if ( *p == 0 ) break;
03779         if ( *p == '0' ) {
03780             p++; while ( *p == '0' ) p++;
03781             continue;
03782         }
03783         inp = pp = p;
03784         p = SkipAName(p);
03785         if ( p == 0 ) return(1);
03786         c = *p;
03787         *p = 0;
03788         type = GetName(AC.varnames,inp,&c1,WITHAUTO);
03789         if ( c == '.' ) {
03790             if ( type == CVECTOR || type == CDUBIOUS ) {
03791                 *p++ = c;
03792                 inp = p;
03793                 p = SkipAName(p);
03794                 if ( p == 0 ) return(1);
03795                 c = *p;
03796                 *p = 0;
03797                 type = GetName(AC.varnames,inp,&c2,WITHAUTO);
03798                 if ( type != CVECTOR && type != CDUBIOUS ) {
03799                     MesPrint("&Not a vector in dotproduct in bracket statement: %s",inp);
03800                     error = 1;
03801                 }
03802                 else type = CDOTPRODUCT;
03803             }
03804             else {
03805                 MesPrint("&Illegal use of . after %s in bracket statement",inp);
03806                 error = 1;
03807                 *p++ = c;
03808                 goto redo;
03809             }
03810         }
03811         switch ( type ) {
03812             case CSYMBOL :
03813                 *to++ = SYMBOL; *to++ = 4; *to++ = c1; *to++ = 1; break;
03814             case CVECTOR :
03815                 *to++ = INDEX; *to++ = 3; *to++ = AM.OffsetVector + c1; break;
03816             case CFUNCTION :
03817                 *to++ = c1+FUNCTION; *to++ = FUNHEAD; *to++ = 0;
03818                 FILLFUN3(to)

```

```

03819         break;
03820     case CDOTPRODUCT :
03821         *to++ = DOTPRODUCT; *to++ = 5; *to++ = c1 + AM.OffsetVector;
03822         *to++ = c2 + AM.OffsetVector; *to++ = 1; break;
03823     case CDELTA :
03824         *to++ = DELTA; *to++ = 4; *to++ = EMPTYINDEX; *to++ = EMPTYINDEX; break;
03825     case CSET :
03826         *to++ = SETSET; *to++ = 4; *to++ = c1; *to++ = Sets[c1].type; break;
03827     default :
03828         MesPrint("&Illegal bracket request for %s",pp);
03829         error = 1; break;
03830     }
03831     *p = c;
03832 }
03833 if ( *p ) {
03834     MesCerr("separator",p);
03835     AC.BracketNormalize = 0;
03836     AT.WorkPointer = WorkSave;
03837     error = 1;
03838     return(error);
03839 }
03840 *to++ = 1; *to++ = 1; *to++ = 3;
03841 *AT.WorkPointer = to - AT.WorkPointer;
03842 AT.WorkPointer = to;
03843 AC.BracketNormalize = 1;
03844 if ( BracketNormalize(BHEAD WorkSave) ) { error = 1; AR.BracketOn = 0; }
03845 else {
03846     w = WorkSave;
03847     if ( *w == 4 || !*w ) { AR.BracketOn = 0; }
03848     else {
03849         i = *(w+w-1);
03850         if ( i < 0 ) i = -i;
03851         *w -= i;
03852         i = *w;
03853         if ( i > AR.MaxBracket ) {
03854             WORD *newbuf;
03855             newbuf = (WORD *)Malloca(sizeof(WORD)*(i+1),"bracket buffer");
03856             AR.MaxBracket = i;
03857             if ( AT.BrackBuf != 0 ) M_free(AT.BrackBuf,"bracket buffer");
03858             AT.BrackBuf = newbuf;
03859         }
03860         to = AT.BrackBuf;
03861         NCOPY(to,w,i);
03862     }
03863 }
03864 AC.BracketNormalize = 0;
03865 if ( par == 1 ) AR.BracketOn = -AR.BracketOn;
03866 if ( error == 0 ) {
03867     AC.bracketindexflag = biflag;
03868     AT.bracketindexflag = biflag;
03869 }
03870 AT.WorkPointer = WorkSave;
03871 return(error);
03872 }
03873
03874 /*
03875 #] DoBrackets :
03876 #[ CoBracket :
03877 */
03878
03879 int CoBracket(UBYTE *inp)
03880 { return(DoBrackets(inp,0)); }
03881
03882 /*
03883 #] CoBracket :
03884 #[ CoAntiBracket :
03885 */
03886
03887 int CoAntiBracket(UBYTE *inp)
03888 { return(DoBrackets(inp,1)); }
03889
03890 /*
03891 #] CoAntiBracket :
03892 #[ CoMultiBracket :
03893
03894 Syntax:
03895     MultiBracket:{A|B} bracketinfo:...:{A|B} bracketinfo;
03896 */
03897
03898 int CoMultiBracket(UBYTE *inp)
03899 {
03900     GETIDENTITY
03901     int i, error = 0, error1, type, num;
03902     UBYTE *s, c;
03903     WORD *to, *from;
03904
03905     if ( *inp != ':' ) {

```

```

03906         MesPrint("&Illegal Multiple Bracket separator: %s",inp);
03907         return(1);
03908     }
03909     inp++;
03910     if ( AC.MultiBracketBuf == 0 ) {
03911         AC.MultiBracketBuf = (WORD **)Malloca1(sizeof(WORD *)*MAXMULTIBRACKETLEVELS,"multi bracket
buffer");
03912         for ( i = 0; i < MAXMULTIBRACKETLEVELS; i++ ) {
03913             AC.MultiBracketBuf[i] = 0;
03914         }
03915     }
03916     else {
03917         for ( i = 0; i < MAXMULTIBRACKETLEVELS; i++ ) {
03918             if ( AC.MultiBracketBuf[i] ) {
03919                 M_free(AC.MultiBracketBuf[i],"bracket buffer i");
03920                 AC.MultiBracketBuf[i] = 0;
03921             }
03922         }
03923         AC.MultiBracketLevels = 0;
03924     }
03925     AC.MultiBracketLevels = 0;
03926     /*
03927         Start with disabling the regular brackets.
03928     */
03929     if ( AT.BrackBuf == 0 ) {
03930         AR.MaxBracket = 100;
03931         AT.BrackBuf = (WORD *)Malloca1(sizeof(WORD)*(AR.MaxBracket+1),"bracket buffer");
03932     }
03933     *AT.BrackBuf = 0;
03934     AR.BracketOn = 0;
03935     AC.bracketindexflag = 0;
03936     AT.bracketindexflag = 0;
03937     /*
03938         Now loop through the various levels, separated by the colons.
03939     */
03940     for ( i = 0; i < MAXMULTIBRACKETLEVELS; i++ ) {
03941         if ( *inp == 0 ) goto RegEnd;
03942         /*
03943             1: skip to ':', determine bracket or antibracket
03944         */
03945         s = inp;
03946         while ( *s && *s != ':' ) {
03947             if ( *s == '[' ) { SKIPBRA1(s) s++; }
03948             else if ( *s == '{' ) { SKIPBRA2(s) s++; }
03949             else s++;
03950         }
03951         c = *s; *s = 0;
03952         if ( StrICont(inp,(UBYTE *)"antibrackets") == 0 ) { type = 1; }
03953         else if ( StrICont(inp,(UBYTE *)"brackets") == 0 ) { type = 0; }
03954         else {
03955             MesPrint("&Illegal (anti)bracket specification in MultiBracket statement");
03956             if ( error == 0 ) error = 1;
03957             goto NextLevel;
03958         }
03959         while ( FG.cTable[*inp] == 0 ) inp++;
03960         if ( *inp != ',' ) {
03961             MesPrint("&Illegal separator after (anti)bracket specification in MultiBracket
statement");
03962             if ( error == 0 ) error = 1;
03963             goto NextLevel;
03964         }
03965         inp++;
03966         /*
03967             2: call DoBrackets.
03968         */
03969         error1 = DoBrackets(inp, type);
03970         if ( error < 0 ) return(error1);
03971         if ( error1 > error ) error = error1;
03972         /*
03973             3: copy bracket information to the multi bracket arrays
03974         */
03975         if ( AR.BracketOn ) {
03976             num = AT.BrackBuf[0];
03977             to = AC.MultiBracketBuf[i] = (WORD *)Malloca1((num+2)*sizeof(WORD),"bracket buffer i");
03978             from = AT.BrackBuf;
03979             *to++ = AR.BracketOn;
03980             NCOPY(to,from,num);
03981             *to = 0;
03982         }
03983         /*
03984             4: set ready for the next level
03985         */
03986     NextLevel:
03987         *s = c; if ( c == ':' ) s++;
03988         inp = s;
03989         *AT.BrackBuf = 0;
03990         AR.BracketOn = 0;

```



```

03991     }
03992     if ( *inp != 0 ) {
03993         MesPrint("&More than %d levels in MultiBracket statement", (WORD)MAXMULTIBRACKETLEVELS);
03994         if ( error == 0 ) error = 1;
03995     }
03996 RegEnd:
03997     AC.MultiBracketLevels = i;
03998     *AT.BrackBuf = 0;
03999     AR.BraceOn = 0;
04000     AC.bracketindexflag = 0;
04001     AT.bracketindexflag = 0;
04002     return(error);
04003 }
04004
04005 /*
04006 #] CoMultiBracket :
04007 #[ CountComp :
04008
04009     This routine reads the count statement. The syntax is:
04010     count minimum,object,size[,object,size]
04011     Objects can be:
04012         symbol
04013         dotproduct
04014         vector
04015         function
04016     Vectors can have the auxiliary flags:
04017         +v +f +d +?setname
04018
04019     Output for the compiler:
04020     TYPECOUNT,size,minimum,objects
04021     with the objects:
04022     SYMBOL,4,number,size
04023     DOTPRODUCT,5,v1,v2,size
04024     FUNCTION,4,number,size
04025     VECTOR,5,number,bits,size or VECTOR,6,number,bits,setnumber,size
04026
04027     Currently only used in the if statement
04028 */
04029
04030 WORD *CountComp(UBYTE *inp, WORD *to)
04031 {
04032     GETIDENTITY
04033     UBYTE *p, c;
04034     WORD *w, mini = 0, type, c1, c2;
04035     int error = 0;
04036     p = inp;
04037     w = to;
04038     AR.Eside = 2;
04039     *w++ = TYPECOUNT;
04040     *w++ = 0;
04041     *w++ = 0;
04042     while ( *p == ',' ) {
04043         p++; inp = p;
04044         if ( *p == '[' || FG.cTable[*p] == 0 ) {
04045             if ( ( p = SkipAName(inp) ) == 0 ) return(0);
04046             c = *p; *p = 0;
04047             type = GetName(AC.varnames,inp,&c1,WITHAUTO);
04048             if ( c == '.' ) {
04049                 if ( type == CVECTOR || type == CDUBIOUS ) {
04050                     *p++ = c;
04051                     inp = p;
04052                     p = SkipAName(p);
04053                     if ( p == 0 ) return(0);
04054                     c = *p;
04055                     *p = 0;
04056                     type = GetName(AC.varnames,inp,&c2,WITHAUTO);
04057                     if ( type != CVECTOR && type != CDUBIOUS ) {
04058                         MesPrint("&Not a vector in dotproduct in if statement: %s",inp);
04059                         error = 1;
04060                     }
04061                     else type = CDOTPRODUCT;
04062                 }
04063                 else {
04064                     MesPrint("&Illegal use of . after %s in if statement",inp);
04065                     if ( type == NAMENOTFOUND )
04066                         MesPrint("&%s is not a properly declared variable",inp);
04067                     error = 1;
04068                     *p++ = c;
04069                     while ( *p && *p != ')' && *p != ',' ) p++;
04070                     if ( *p == ',' && FG.cTable[p[1]] == 1 ) {
04071                         p++;
04072                         while ( *p && *p != ')' && *p != ',' ) p++;
04073                     }
04074                     continue;
04075                 }
04076             }
04077             *p = c;

```

```

04078         switch ( type ) {
04079             case CSYMBOL:
04080                 *w++ = SYMBOL; *w++ = 4; *w++ = c1;
04081 Sgetnum:         if ( *p != ',' ) {
04082                     MesCerr("sequence",p);
04083                     while ( *p && *p != ')' && *p != ',' ) p++;
04084                     error = 1;
04085                 }
04086                 p++; inp = p;
04087                 ParseSignedNumber(mini,p)
04088                 if ( FG.cTable[p[-1]] != 1 || ( *p && *p != ')' && *p != ',' ) ) {
04089                     while ( *p && *p != ')' && *p != ',' ) p++;
04090                     error = 1;
04091                     c = *p; *p = 0;
04092                     MesPrint("&Improper value in count: %s",inp);
04093                     *p = c;
04094                     while ( *p && *p != ')' && *p != ',' ) p++;
04095                 }
04096                 *w++ = mini;
04097                 break;
04098             case CFUNCTION:
04099                 *w++ = FUNCTION; *w++ = 4; *w++ = c1+FUNCTION; goto Sgetnum;
04100             case CDOTPRODUCT:
04101                 *w++ = DOTPRODUCT; *w++ = 5;
04102                 *w++ = c2 + AM.OffsetVector;
04103                 *w++ = c1 + AM.OffsetVector;
04104                 goto Sgetnum;
04105             case CVECTOR:
04106                 *w++ = VECTOR; *w++ = 5;
04107                 *w++ = c1 + AM.OffsetVector;
04108                 if ( *p == ',' ) {
04109                     *w++ = VECTBIT | DOTPBIT | FUNBIT;
04110                     goto Sgetnum;
04111                 }
04112             else if ( *p == '+' ) {
04113                 p++;
04114                 *w = 0;
04115                 while ( *p && *p != ',' ) {
04116                     if ( *p == 'v' || *p == 'V' ) {
04117                         *w |= VECTBIT; p++;
04118                     }
04119                     else if ( *p == 'd' || *p == 'D' ) {
04120                         *w |= DOTPBIT; p++;
04121                     }
04122                     else if ( *p == 'f' || *p == 'F'
04123                             || *p == 't' || *p == 'T' ) {
04124                         *w |= FUNBIT; p++;
04125                     }
04126                     else if ( *p == '?' ) {
04127                         p++; inp = p;
04128                         if ( *p == '{' ) { /* } */
04129                             SKIPBRA2(p)
04130                             if ( p == 0 ) return(0);
04131                             if ( ( c1 = DoTempSet(inp+1,p) ) < 0 ) return(0);
04132                             if ( Sets[c1].type != CFUNCTION ) {
04133                                 MesPrint("&set type conflict: Function expected");
04134                                 return(0);
04135                             }
04136                             type = CSET;
04137                             c = ++p;
04138                         }
04139                         else {
04140                             p = SkipAName(p);
04141                             if ( p == 0 ) return(0);
04142                             c = *p; *p = 0;
04143                             type = GetName(AC.varnames,inp,&c1,WITHAUTO);
04144                         }
04145                         if ( type != CSET && type != CDUBIOUS ) {
04146                             MesPrint("&%s is not a set",inp);
04147                             error = 1;
04148                         }
04149                         w[-2] = 6;
04150                         *w++ |= SETBIT;
04151                         *w++ = c1;
04152                         *p = c;
04153                         goto Sgetnum;
04154                     }
04155                     else {
04156                         MesCerr("specifier for vector",p);
04157                         error = 1;
04158                     }
04159                 }
04160                 w++;
04161                 goto Sgetnum;
04162             }
04163             else {
04164                 MesCerr("specifier for vector",p);

```

```

04165         while ( *p && *p != ')' && *p != ',' ) p++;
04166         error = 1;
04167         *w++ = VECTBIT | DOTPBIT | FUNBIT;
04168         goto Sgetnum;
04169     }
04170     case CDUBIOUS:
04171         goto skipfield;
04172     default:
04173         *p = 0;
04174         MesPrint("%s is not a symbol, function, vector or dotproduct",inp);
04175         error = 1;
04176 skipfield:
04177         while ( *p && *p != ')' && *p != ',' ) p++;
04178         if ( *p && FG.cTable[p[1]] == 1 ) {
04179             p++;
04180             while ( *p && *p != ')' && *p != ',' ) p++;
04181             break;
04182         }
04183     }
04184     else {
04185         MesCerr("name",p);
04186         while ( *p && *p != ',' ) p++;
04187         error = 1;
04188     }
04189 }
04190 to[1] = w-to;
04191 if ( *p == ')' ) p++;
04192 if ( *p ) { MesCerr("end of statement",p); return(0); }
04193 if ( error ) return(0);
04194 return(w);
04195 }
04196
04197 /*
04198 #] CountComp :
04199 #[ CoIf :
04200
04201     Reads the if statement: There must be a pair of parentheses.
04202     Much work is delegated to the routines in compi2 and CountComp.
04203     The goto is kept hanging as it is forward.
04204     The address in which the label must be written is pushed on
04205     the AC.IfStack.
04206
04207     Here we allow statements of the type
04208     if ( condition ) single statement;
04209     compile the if statement.
04210     test character at end
04211     if not ; or )
04212     copy the statement after the proper parenthesis to the
04213     beginning of the AC.iBuffer.
04214     Have it compiled.
04215     generate an endif statement.
04216 */
04217
04218 static UWORD *CIsratC = 0;
04219
04220 int CoIf(UBYTE *inp)
04221 {
04222     GETIDENTITY
04223     int error = 0, level;
04224     WORD *w, *ww, *u, *s, *OldWork, *OldSpace = AT.WorkSpace;
04225     WORD gotexp = 0; /* Indicates whether there can be a condition */
04226     WORD lenpp, lenlev, ncoef, i, number;
04227     UBYTE *p, *pp, *ppp, c;
04228     CBUF *C = cbuf+AC.cbunum;
04229     LONG x;
04230 #ifdef WITHFLOAT
04231     int spec;
04232 #endif
04233     if ( *inp == '(' && inp[1] == ',' ) inp += 2;
04234     else if ( *inp == '(' ) inp++; /* Usually we enter at the bracket */
04235
04236     if ( CIsratC == 0 )
04237         CIsratC = (UWORD *)Malloca1((AM.MaxTal+2)*sizeof(UWORD),"CoIf");
04238     lenpp = 0;
04239     lenlev = 1;
04240     if ( AC.IfLevel >= AC.MaxIf ) DoubleIfBuffers();
04241     AC.IfCount[lenpp++] = 0;
04242 /*
04243     IfStack is used for organizing the 'go to' for the various if levels
04244 */
04245     *AC.IfStack++ = C->Pointer-C->Buffer+2;
04246 /*
04247     IfSumCheck is used to test for illegal nesting of if, argument or repeat.
04248 */
04249     AC.IfSumCheck[AC.IfLevel] = NestingChecksum();
04250     AC.IfLevel++;
04251     w = OldWork = AT.WorkPointer;

```

```

04252     *w++ = TYPEIF;
04253     w += 2;
04254     p = inp;
04255     for(;;) {
04256         inp = p;
04257         level = 0;
04258 ReDo:
04259         if ( FG.cTable[*p] == 1 ) { /* Number */
04260             if ( gotexp == 1 ) { MesCerr("position for ",p); error = 1; }
04261 #ifdef WITHFLOAT
04262             pp = CheckFloat(p,&spec);
04263             if ( pp > p ) { /* Got one */
04264 HaveFloat:
04265                 if ( spec == 1 ) { /* is zero */
04266                     *w++ = LONGNUMBER; *w++ = 3; *w++ = 0;
04267                 }
04268                 else if ( spec == -1 ) {
04269                     MesPrint("&The floating point system has not been started: %s",p);
04270                     if ( !error ) error = 1;
04271                 }
04272                 else {
04273                     WORD *ow = AT.WorkPointer;
04274                     AT.WorkPointer = w;
04275                     c = *pp; c = 0;
04276                     ReadFloat((SBYTE *)p); /* Is now at AT.WorkPointer */
04277                     *pp = c;
04278                     AT.WorkPointer[0] = IFFLOATNUMBER;
04279                     w = AT.WorkPointer + AT.WorkPointer[1];
04280                     AT.WorkPointer = ow;
04281                     if ( level ) w[FUNHEAD+3] = -w[FUNHEAD+3];
04282                 }
04283                 goto DoneWithNumber;
04284             }
04285 /*
04286             Notation: Same as FLOATFUN but FLOATFUN replaced by IFFLOATNUMBER.
04287 */
04288 #endif
04289         u = w;
04290         *w++ = LONGNUMBER;
04291 /*
04292         Notation:
04293         LONGNUMBER,size,reducedsize*sign,numerator,denominator
04294         with the length of denominator and numerator equal to reducedsize
04295 */
04296         w += 2;
04297         if ( GetLong(p, (UWORD *)w,&ncoef) ) { ncoef = 1; error = 1; }
04298         w[-1] = ncoef;
04299         while ( FG.cTable[***p] == 1 );
04300         if ( *p == '/' ) {
04301             p++;
04302             if ( FG.cTable[*p] != 1 ) {
04303                 MesCerr("sequence",p); error = 1; goto OnlyNum;
04304             }
04305             if ( GetLong(p,CIsratC,&ncoef) ) {
04306                 ncoef = 1; error = 1;
04307             }
04308             while ( FG.cTable[***p] == 1 );
04309             if ( ncoef == 0 ) {
04310                 MesPrint("&Division by zero!");
04311                 error = 1;
04312             }
04313         }
04314         else {
04315             if ( w[-1] != 0 ) {
04316                 if ( Simplify(BHEAD (UWORD *)w,(WORD *) (w-1),
04317                     CIsratC,&ncoef) ) error = 1;
04318                 else {
04319                     i = w[-1];
04320                     if ( i >= ncoef ) {
04321                         i = w[-1];
04322                         w += i;
04323                         i -= ncoef;
04324                         s = (WORD *)CIsratC;
04325                         NCOPY(w,s,ncoef);
04326                         while ( --i >= 0 ) *w++ = 0;
04327                     }
04328                     else {
04329                         w += i;
04330                         i = ncoef - i;
04331                         while ( --i >= 0 ) *w++ = 0;
04332                         s = (WORD *)CIsratC;
04333                         NCOPY(w,s,ncoef);
04334                     }
04335                 }
04336             }
04337         }
04338     }

```

```

04339         else {
04340 OnlyNum:
04341             w += ncoef;
04342             if ( ncoef > 0 ) {
04343                 ncoef--; *w++ = 1;
04344                 while ( --ncoef >= 0 ) *w++ = 0;
04345             }
04346         }
04347         u[1] = WORDDIF(w,u);
04348         u[2] = (u[1] - 3)/2;
04349         if ( level ) u[2] = -u[2];
04350 #ifdef WITHFLOAT
04351 DoneWithNumber:
04352 #endif
04353         gotexp = 1;
04354     }
04355     else if ( *p == '+' ) { p++; goto ReDo; }
04356     else if ( *p == '-' ) { level ^= 1; p++; goto ReDo; }
04357     else if ( *p == 'c' || *p == 'C' ) { /* Count or Coefficient */
04358         if ( gotexp == 1 ) { MesCerr("position for ",p); error = 1; }
04359         while ( FG.cTable[++p] == 0 );
04360         c = *p; *p = 0;
04361         if ( !StrICmp(inp, (UBYTE *) "count" ) ) {
04362             *p = c;
04363             if ( c != '(' ) {
04364                 MesPrint("&no ( after count");
04365                 error = 1;
04366                 goto endofif;
04367             }
04368             inp = p;
04369             SKIPBRA4(p);
04370             c = *p; *p = 0; *inp = ',';
04371             w = CountComp(inp,w);
04372             *p = c; *inp = '(';
04373             if ( w == 0 ) { error = 1; goto endofif; }
04374             gotexp = 1;
04375         }
04376         else if ( ConWord(inp, (UBYTE *) "coefficient" ) && ( p - inp ) > 3 ) {
04377             *w++ = COEFFI;
04378             *w++ = 2;
04379             *p = c;
04380             gotexp = 1;
04381         }
04382         else goto NoGood;
04383         inp = p;
04384     }
04385     else if ( *p == 'm' || *p == 'M' ) { /* match */
04386         if ( gotexp == 1 ) { MesCerr("position for ",p); error = 1; }
04387         while ( !FG.cTable[++p] );
04388         c = *p; *p = 0;
04389         if ( !StrICmp(inp, (UBYTE *) "match" ) ) {
04390             *p = c;
04391             if ( c != '(' ) {
04392                 MesPrint("&no ( after match");
04393                 error = 1;
04394                 goto endofif;
04395             }
04396             p++; inp = p;
04397             SKIPBRA4(p);
04398             *p = '=';
04399         /*
04400          Now we can call the reading of the lhs of an id statement.
04401          This has to be modified in the future.
04402         */
04403         AT.WorkSpace = AT.WorkPointer = w;
04404         ppp = inp;
04405         while ( FG.cTable[*ppp] == 0 && ppp < p ) ppp++;
04406         if ( *ppp == ',' ) AC.idoption = 0;
04407         else AC.idoption = SUBMULTI;
04408         level = CoIdExpression(inp,TYPEEIF);
04409         AT.WorkSpace = OldSpace;
04410         AT.WorkPointer = OldWork;
04411         if ( level != 0 ) {
04412             if ( level < 0 ) { error = -1; goto endofif; }
04413             error = 1;
04414         }
04415         /*
04416          If we pop numlhs we are in good shape
04417         */
04418         s = u = C->lhs[C->numlhs];
04419         while ( u < C->Pointer ) *w++ = *u++;
04420         C->numlhs--; C->Pointer = s;
04421         *p++ = ')';
04422         inp = p;
04423         gotexp = 1;
04424     }
04425     else if ( !StrICmp(inp, (UBYTE *) "multipleof" ) ) {

```

```

04426         if ( gotexp == 1 ) { MesCerr("position for )",p); error = 1; }
04427         *p = c;
04428         if ( c != '(' ) {
04429             MesPrint("&no ( after multipleof");
04430             error = 1; goto endofif;
04431         }
04432         p++;
04433         if ( FG.cTable[*p] != 1 ) {
04434             MesPrint("&multipleof needs a short positive integer argument");
04435             error = 1; goto endofif;
04436         }
04437         ParseNumber(x,p)
04438         if ( *p != ')' || x <= 0 || x > MAXPOSITIVE ) goto Nomulof;
04439         p++;
04440         *w++ = MULTIPLEOF; *w++ = 3; *w++ = (WORD)x;
04441         inp = p;
04442         gotexp = 1;
04443     }
04444     else {
04445         MesPrint("&Unrecognized word: %s",inp);
04446         *p = c;
04447         error = 1;
04448         level = 0;
04449         if ( c == '(' ) SKIPBRA4(p)
04450         inp = ++p;
04451         gotexp = 1;
04452     }
04453 }
04454 else if ( *p == 'f' || *p == 'F' ) { /* FindLoop */
04455     if ( gotexp == 1 ) { MesCerr("position for )",p); error = 1; }
04456     while ( FG.cTable[++p] == 0 );
04457     c = *p; *p = 0;
04458     if ( !StrICmp(inp, (UBYTE *) "findloop") ) {
04459         *p = c;
04460         if ( c != '(' ) {
04461             MesPrint("&no ( after findloop");
04462             error = 1;
04463             goto endofif;
04464         }
04465         inp = p;
04466         SKIPBRA4(p);
04467         c = ++p; *p = 0; *inp = ',';
04468         if ( CoFindLoop(inp) ) { error = 1; goto endofif; }
04469         s = u = C->lhs[C->numlhs];
04470         while ( u < C->Pointer ) *w++ = *u++;
04471         C->numlhs--; C->Pointer = s;
04472         *p = c; *inp = '(';
04473         if ( w == 0 ) { error = 1; goto endofif; }
04474         gotexp = 1;
04475     }
04476     else if ( !StrICmp(inp, (UBYTE *) "flag") ) {
04477         UBYTE cc = c, *pppp;
04478         *p = cc;
04479         if ( cc != '(' ) {
04480             MesPrint("&no ( after flag");
04481             error = 1;
04482             goto endofif;
04483         }
04484         inp = p;
04485         SKIPBRA4(p);
04486         cc = ++p; *p = 0; *inp = ','; pppp = p;
04487         ww = w;
04488         *w++ = IFUSERFLAG; *w++ = 0;
04489         while ( *inp ) {
04490             int x = 0;
04491             while ( *inp == ',' ) inp++;
04492             if ( *inp == 0 || *inp == ')' ) break;
04493             while ( *inp >= '0' && *inp <= '9' ) x = 10*x + (*inp++ - '0');
04494             if ( x < 1 || x > BITSINWORD ) {
04495                 MesPrint("&Flag number %d outside the permitted range 1-%d.",BITSINWORD);
04496                 error = 1;
04497             }
04498             *w++ = x-1;
04499         }
04500         ww[1] = w-ww;
04501         p = pppp; *p = cc; *inp = '(';
04502         gotexp = 1;
04503         if ( ww[1] <= 2 ) {
04504             MesPrint("&The userflag condition in the if statement needs arguments.");
04505             error = 1;
04506         }
04507         inp = p;
04508         gotexp = 1;
04509     }
04510     else goto NoGood;
04511 }
04512 else if ( *p == 'e' || *p == 'E' ) { /* Expression */

```

```

04513         if ( gotexp == 1 ) { MesCerr("position for )",p); error = 1; }
04514         while ( FG.cTable[***p] == 0 );
04515         c = *p; *p = 0;
04516         if ( !StrICmp(inp, (UBYTE *)"expression") ) {
04517             *p = c;
04518             if ( c != '(' ) {
04519                 MesPrint("&no ( after expression");
04520                 error = 1;
04521                 goto endofif;
04522             }
04523             p++; ww = w; *w++ = IFEXPRESSION; w++;
04524             while ( *p != ')' ) {
04525                 if ( *p == ',' ) { p++; continue; }
04526                 if ( *p == '[' || FG.cTable[*p] == 0 ) {
04527                     pp = p;
04528                     if ( ( p = SkipAName(p) ) == 0 ) {
04529                         MesPrint("&Improper name for an expression: '%s'",pp);
04530                         error = 1;
04531                         goto endofif;
04532                     }
04533                     c = *p; *p = 0;
04534                     if ( GetName(AC.exprnames,pp,&number,NOAUTO) == CEXPRESSION ) {
04535                         *w++ = number;
04536                     }
04537                     else if ( GetName(AC.varnames,pp,&number,NOAUTO) != NAMENOTFOUND ) {
04538                         MesPrint("&%s is not an expression",pp);
04539                         error = 1;
04540                         *w++ = number;
04541                     }
04542                     *p = c;
04543                 }
04544                 else {
04545                     MesPrint("&Illegal object in Expression in if-statement");
04546                     error = 1;
04547                     while ( *p && *p != ',' && *p != ')' ) p++;
04548                     if ( *p == 0 || *p == ')' ) break;
04549                 }
04550             }
04551             ww[1] = w - ww;
04552             p++;
04553             gotexp = 1;
04554         }
04555         else goto NoGood;
04556         inp = p;
04557     }
04558     else if ( *p == 'i' || *p == 'I' ) { /* IsFactorized */
04559         if ( gotexp == 1 ) { MesCerr("position for )",p); error = 1; }
04560         while ( FG.cTable[***p] == 0 );
04561         c = *p; *p = 0;
04562         if ( !StrICmp(inp, (UBYTE *)"isfactorized") ) {
04563             *p = c;
04564             if ( c != '(' ) { /* No expression means current expression */
04565                 ww = w; *w++ = IFISFACTORIZED; w++;
04566             }
04567             else {
04568                 p++; ww = w; *w++ = IFISFACTORIZED; w++;
04569                 while ( *p != ')' ) {
04570                     if ( *p == ',' ) { p++; continue; }
04571                     if ( *p == '[' || FG.cTable[*p] == 0 ) {
04572                         pp = p;
04573                         if ( ( p = SkipAName(p) ) == 0 ) {
04574                             MesPrint("&Improper name for an expression: '%s'",pp);
04575                             error = 1;
04576                             goto endofif;
04577                         }
04578                         c = *p; *p = 0;
04579                         if ( GetName(AC.exprnames,pp,&number,NOAUTO) == CEXPRESSION ) {
04580                             *w++ = number;
04581                         }
04582                         else if ( GetName(AC.varnames,pp,&number,NOAUTO) != NAMENOTFOUND ) {
04583                             MesPrint("&%s is not an expression",pp);
04584                             error = 1;
04585                             *w++ = number;
04586                         }
04587                         *p = c;
04588                     }
04589                     else {
04590                         MesPrint("&Illegal object in IsFactorized in if-statement");
04591                         error = 1;
04592                         while ( *p && *p != ',' && *p != ')' ) p++;
04593                         if ( *p == 0 || *p == ')' ) break;
04594                     }
04595                 }
04596                 p++;
04597             }
04598             ww[1] = w - ww;
04599             gotexp = 1;

```

```

04600     }
04601     else goto NoGood;
04602     inp = p;
04603 }
04604 else if ( *p == 'o' || *p == 'O' ) { /* Occurs */
04605 /*
04606     Tests whether variables occur inside a term.
04607     At the moment this is done one by one.
04608     If we want to do them in groups we should do the reading
04609     a bit different: each as a variable in a term, and then
04610     use Normalize to get the variables grouped and in order.
04611     That way FindVar (in if.c) can work more efficiently.
04612     Still to be done!!!
04613     TASK: Nice little task for someone to learn.
04614 */
04615     UBYTE cc;
04616     if ( gotexp == 1 ) { MesCerr("position for "),p); error = 1; }
04617     while ( FG.cTable[++p] == 0 );
04618     c = cc = *p; *p = 0;
04619     if ( !StrICmp(inp,(UBYTE *)"occurs") ) {
04620         WORD c1, c2, type;
04621         *p = cc;
04622         if ( cc != '(' ) {
04623             MesPrint("&no ( after occurs");
04624             error = 1;
04625             goto endofif;
04626         }
04627         inp = p;
04628         SKIPBRA4(p);
04629         cc = ++p; *p = 0; *inp = ','; pp = p;
04630         ww = w;
04631         *w++ = IFOCCURS; *w++ = 0;
04632         while ( *inp ) {
04633             while ( *inp == ',' ) inp++;
04634             if ( *inp == 0 || *inp == ')' ) break;
04635 /*
04636             Now read a list of names
04637             We can have symbols, vectors, dotproducts, indices, functions.
04638             There could also be dummy indices and/or extra symbols.
04639 */
04640             if ( *inp == '[' || FG.cTable[*inp] == 0 ) {
04641                 if ( ( p = SkipAName(inp) ) == 0 ) return(0);
04642                 c = *p; *p = 0;
04643                 type = GetName(AC.varnames,inp,&c1,WITHAUTO);
04644                 if ( c == '.' ) {
04645                     if ( type == CVECTOR || type == CDUBIOUS ) {
04646                         *p++ = c;
04647                         inp = p;
04648                         p = SkipAName(p);
04649                         if ( p == 0 ) return(0);
04650                         c = *p;
04651                         *p = 0;
04652                         type = GetName(AC.varnames,inp,&c2,WITHAUTO);
04653                         if ( type != CVECTOR && type != CDUBIOUS ) {
04654                             MesPrint("&Not a vector in dotproduct in if statement: %s",inp);
04655                             error = 1;
04656                         }
04657                         else type = CDOTPRODUCT;
04658                     }
04659                     else {
04660                         MesPrint("&Illegal use of . after %s in if statement",inp);
04661                         if ( type == NAMENOTFOUND )
04662                             MesPrint("&%s is not a properly declared variable",inp);
04663                         error = 1;
04664                         *p++ = c;
04665                         while ( *p && *p != ')' && *p != ',' ) p++;
04666                         if ( *p == ',' && FG.cTable[p[1]] == 1 ) {
04667                             p++;
04668                             while ( *p && *p != ')' && *p != ',' ) p++;
04669                         }
04670                         continue;
04671                     }
04672                 }
04673                 *p = c;
04674                 switch ( type ) {
04675                     case CSYMBOL: /* To worry about extra symbols */
04676                         *w++ = SYMBOL;
04677                         *w++ = c1;
04678                     break;
04679                     case CINDEX:
04680                         *w++ = INDEX;
04681                         *w++ = c1 + AM.OffsetIndex;
04682                     break;
04683                     case CVECTOR:
04684                         *w++ = VECTOR;
04685                         *w++ = c1 + AM.OffsetVector;
04686                     break;

```



```

04687             case CDOTPRODUCT:
04688                 *w++ = DOTPRODUCT;
04689                 *w++ = c1 + AM.OffsetVector;
04690                 *w++ = c2 + AM.OffsetVector;
04691             break;
04692             case CFUNCTION:
04693                 *w++ = FUNCTION;
04694                 *w++ = c1+FUNCTION;
04695             break;
04696             default:
04697                 MesPrint("&Illegal variable %s in occurs condition in if
statement",inp);
04698                 error = 1;
04699                 break;
04700             }
04701             inp = p;
04702         }
04703         else {
04704             MesPrint("&Illegal object %s in occurs condition in if statement",inp);
04705             error = 1;
04706             break;
04707         }
04708     }
04709     ww[1] = w-ww;
04710     p = pp; *p = cc; *inp = '(';
04711     gotexp = 1;
04712     if ( ww[1] <= 2 ) {
04713         MesPrint("&The occurs condition in the if statement needs arguments.");
04714         error = 1;
04715     }
04716 }
04717 else goto NoGood;
04718 inp = p;
04719 }
04720 else if ( *p == '$' ) {
04721     if ( gotexp == 1 ) { MesCerr("position for ",p); error = 1; }
04722     p++; inp = p;
04723     while ( FG.cTable[*p] == 0 || FG.cTable[*p] == 1 ) p++;
04724     c = *p; *p = 0;
04725     if ( ( i = GetDollar(inp) ) < 0 ) {
04726         MesPrint("&undefined dollar expression %s",inp);
04727         error = 1;
04728         i = AddDollar(inp,DOLUNDEFINED,0,0);
04729     }
04730     *p = c;
04731     *w++ = IFDOLLAR; *w++ = 3; *w++ = i;
04732 /*
04733     And then the IFDOLLAREXTRA pieces for [1] [$y] etc
04734 */
04735     if ( *p == '[' ) {
04736         p++;
04737         if ( ( w = GetIfDollarFactor(&p,w) ) == 0 ) {
04738             error = 1;
04739             goto endofif;
04740         }
04741         else if ( *p != ']' ) {
04742             error = 1;
04743             goto endofif;
04744         }
04745         p++;
04746     }
04747     inp = p;
04748     gotexp = 1;
04749 }
04750 #ifdef WITHFLOAT
04751     else if ( *p == '.' ) {
04752         pp = CheckFloat(p,&spec);
04753         if ( pp > p ) goto HaveFloat;
04754     }
04755 #endif
04756     else if ( *p == '(' ) {
04757         if ( gotexp ) {
04758             MesCerr("parenthesis",p);
04759             error = 1;
04760             goto endofif;
04761         }
04762         gotexp = 0;
04763         if ( ++lenlev >= AC.MaxIf ) DoubleIfBuffers();
04764         AC.IfCount[lenpp++] = w-OldWork;
04765         *w++ = SUBEXPR;
04766         w += 2;
04767         p++;
04768     }
04769     else if ( *p == ')' ) {
04770         if ( gotexp == 0 ) { MesCerr("position for ",p); error = 1; }
04771         gotexp = 1;
04772         u = AC.IfCount[--lenpp]+OldWork;

```

```

04773         lenlev--;
04774         u[1] = w - u;
04775         if ( lenlev <= 0 ) {          /* End if condition */
04776             AT.WorkSpace = OldSpace;
04777             AT.WorkPointer = OldWork;
04778             AddNtoL(OldWork[1],OldWork);
04779             p++;
04780             if ( *p == ')' ) {
04781                 MesPrint("&unmatched parenthesis in if/while ()");
04782                 error = 1;
04783                 while ( *++p == ')' );
04784             }
04785             if ( *p ) {
04786                 level = CompileStatement(p);
04787                 if ( level ) error = level;
04788                 while ( *p ) p++;
04789                 if ( CoEndIf(p) && error == 0 ) error = 1;
04790             }
04791             return(error);
04792         }
04793         p++;
04794     }
04795     else if ( *p == '>' ) {
04796         if ( gotexp == 0 ) goto NoExp;
04797         if ( p[1] == '=' ) { *w++ = GREATEREQUAL; *w++ = 2; p += 2; }
04798         else { *w++ = GREATER; *w++ = 2; p++; }
04799         gotexp = 0;
04800     }
04801     else if ( *p == '<' ) {
04802         if ( gotexp == 0 ) goto NoExp;
04803         if ( p[1] == '=' ) { *w++ = LESSEQUAL; *w++ = 2; p += 2; }
04804         else { *w++ = LESS; *w++ = 2; p++; }
04805         gotexp = 0;
04806     }
04807     else if ( *p == '=' ) {
04808         if ( gotexp == 0 ) goto NoExp;
04809         if ( p[1] == '=' ) p++;
04810         *w++ = EQUAL; *w++ = 2; p++;
04811         gotexp = 0;
04812     }
04813     else if ( *p == '!' && p[1] == '=' ) {
04814         if ( gotexp == 0 ) { p++; goto NoExp; }
04815         *w++ = NOTEQUAL; *w++ = 2; p += 2;
04816         gotexp = 0;
04817     }
04818     else if ( *p == '|' && p[1] == '|' ) {
04819         if ( gotexp == 0 ) { p++; goto NoExp; }
04820         *w++ = ORCOND; *w++ = 2; p += 2;
04821         gotexp = 0;
04822     }
04823     else if ( *p == '&' && p[1] == '&' ) {
04824         if ( gotexp == 0 ) {
04825             p++;
04826             p++;
04827             MesCerr("sequence",p);
04828             error = 1;
04829         }
04830         else {
04831             *w++ = ANDCOND; *w++ = 2; p += 2;
04832             gotexp = 0;
04833         }
04834     }
04835     else if ( *p == 0 ) {
04836         MesPrint("&Unmatched parentheses");
04837         error = 1;
04838         goto endofif;
04839     }
04840     else {
04841         if ( FG.cTable[*p] == 0 ) {
04842             WORD ij;
04843             inp = p;
04844             while ( ( ij = FG.cTable[*++p] ) == 0 || ij == 1 );
04845             c = *p; *p = 0;
04846             goto NoGood;
04847         }
04848         MesCerr("sequence",p);
04849         error = 1;
04850         p++;
04851     }
04852 }
04853 endofif;
04854 return(error);
04855 }
04856
04857 /*
04858 #] CoIf :
04859 #[ CoElse :

```

```

04860 */
04861
04862 int CoElse(UBYTE *p)
04863 {
04864     int error = 0;
04865     CBUF *C = cbuf+AC.cbunum;
04866     if ( *p != 0 ) {
04867         while ( *p == ',' ) p++;
04868         if ( tolower(*p) == 'i' && tolower(p[1]) == 'f' && p[2] == '(' )
04869             return(CoElseIf(p+2));
04870         MesPrint("&No extra text allowed as part of an else statement");
04871         error = 1;
04872     }
04873     if ( AC.IfLevel <= 0 ) { MesPrint("&else statement without if"); return(1); }
04874     if ( AC.IfSumCheck[AC.IfLevel-1] != NestingChecksum() - 1 ) {
04875         MesNesting();
04876         error = 1;
04877     }
04878     Add3Com(TYPEELSE,AC.IfLevel)
04879     C->Buffer[AC.IfStack[-1]] = C->numlhs;
04880     AC.IfStack[-1] = C->Pointer - C->Buffer - 1;
04881     return(error);
04882 }
04883
04884 /*
04885 #] CoElse :
04886 #[ CoElseIf :
04887 */
04888
04889 int CoElseIf(UBYTE *inp)
04890 {
04891     CBUF *C = cbuf+AC.cbunum;
04892     if ( AC.IfLevel <= 0 ) { MesPrint("&elseif statement without if"); return(1); }
04893     Add3Com(TYPEELSE,-AC.IfLevel)
04894     AC.IfLevel--;
04895     C->Buffer[*--AC.IfStack] = C->numlhs;
04896     return(CoIf(inp));
04897 }
04898
04899 /*
04900 #] CoElseIf :
04901 #[ CoEndIf :
04902
04903     It puts a RHS-level at the position indicated in the AC.IfStack.
04904     This corresponds to the label belonging to a forward goto.
04905     It is the goto that belongs either to the failing condition
04906     of the if (no else statement), or the completion of the
04907     success path (with else statement)
04908     The code is a jump to the next statement. It is there to prevent
04909     problems with
04910     if ( .. )
04911         if ( .. )
04912         endif;
04913     elseif ( .. )
04914 */
04915
04916 int CoEndIf(UBYTE *inp)
04917 {
04918     CBUF *C = cbuf+AC.cbunum;
04919     WORD i = C->numlhs, to, k = -AC.IfLevel;
04920     int error = 0;
04921     while ( *inp == ',' ) inp++;
04922     if ( *inp != 0 ) {
04923         error = 1;
04924         MesPrint("&No extra text allowed as part of an endif/elseif statement");
04925     }
04926     if ( AC.IfLevel <= 0 ) {
04927         MesPrint("&Endif statement without corresponding if"); return(1);
04928     }
04929     AC.IfLevel--;
04930     C->Buffer[*--AC.IfStack] = i+1;
04931     if ( AC.IfSumCheck[AC.IfLevel] != NestingChecksum() ) {
04932         MesNesting();
04933         error = 1;
04934     }
04935     Add3Com(TYPEENDIF,i+1)
04936
04937     /*
04938     Now the search for the TYPEELSE in front of the elseif statements
04939     */
04940     to = C->numlhs;
04941     while ( i > 0 ) {
04942         if ( C->lhs[i][0] == TYPEELSE && C->lhs[i][2] == to ) to = i;
04943         if ( C->lhs[i][0] == TYPEIF ) {
04944             if ( C->lhs[i][2] == to ) {
04945                 i--;
04946                 if ( i <= 0 || C->lhs[i][0] != TYPEELSE
04947                     || C->lhs[i][2] != k ) break;

```

```

04947         C->lhs[i][2] = C->numlhs;
04948         to = i;
04949     }
04950 }
04951     i--;
04952 }
04953     return(error);
04954 }
04955
04956 /*
04957     #] CoEndIf :
04958     #[ CoWhile :
04959 */
04960
04961 int CoWhile(UBYTE *inp)
04962 {
04963     CBUF *C = cbuf+AC.cbunum;
04964     WORD startnum = C->numlhs + 1;
04965     int error;
04966     AC.WhileLevel++;
04967     error = CoIf(inp);
04968     if ( C->numlhs > startnum && C->lhs[startnum][2] == C->numlhs
04969         && C->lhs[C->numlhs][0] == TYPEENDIF ) {
04970         C->lhs[C->numlhs][2] = startnum-1;
04971         AC.WhileLevel--;
04972     }
04973     else C->lhs[startnum][2] = startnum;
04974     return(error);
04975 }
04976
04977 /*
04978     #] CoWhile :
04979     #[ CoEndWhile :
04980 */
04981
04982 int CoEndWhile(UBYTE *inp)
04983 {
04984     int error = 0;
04985     WORD i;
04986     CBUF *C = cbuf+AC.cbunum;
04987     if ( AC.WhileLevel <= 0 ) {
04988         MesPrint("&EndWhile statement without corresponding While"); return(1);
04989     }
04990     AC.WhileLevel--;
04991     i = C->Buffer[AC.IfStack[-1]];
04992     error = CoEndIf(inp);
04993     C->lhs[C->numlhs][2] = i - 1;
04994     return(error);
04995 }
04996
04997 /*
04998     #] CoEndWhile :
04999     #[ DoFindLoop :
05000
05001     Function,arguments=number,loopsize=number,outfun=function,include=index;
05002 */
05003
05004 static char *messfind[] = {
05005     "Findloop(function,arguments=#,loopsize=#|<#)[,include=index]"
05006     ,"Replaceloop,function,arguments=#,loopsize=#|<#),outfun=function[,include=index]"
05007 };
05008 static WORD comfindloop[7] = { TYPEFINDLOOP,7,0,0,0,0,0 };
05009
05010 int DoFindLoop(UBYTE *inp, int mode)
05011 {
05012     UBYTE *s, c;
05013     WORD funnum, nargs = 0, nloop = 0, indexnum = 0, outfun = 0;
05014     int type, aflag, lflag, indflag, outflag, error = 0, sym;
05015     while ( *inp == ',' ) inp++;
05016     if ( ( s = SkipAName(inp) ) == 0 ) {
05017         syntax:
05018         MesPrint("&Proper syntax is:");
05019         MesPrint("%s",messfind[mode]);
05020         return(1);
05021     }
05022     c = *s; *s = 0;
05023     if ( ( ( type = GetName(AC.varnames,inp,&funnum,WITHAUTO) ) == NAMENOTFOUND )
05024         || type != CFUNCTION || ( ( sym = (functions[funnum].symmetric) & ~REVERSEORDER )
05025         != SYMMETRIC && sym != ANTISYMMETRIC ) ) {
05026         MesPrint("&%s should be a (anti)symmetric function or tensor",inp);
05027         error = 1;
05028     }
05029     funnum += FUNCTION;
05030     *s = c; inp = s;
05031     aflag = lflag = indflag = outflag = 0;
05032     while ( *inp == ',' ) {
05033         while ( *inp == ',' ) inp++;

```

```

05034     s = inp;
05035     if ( ( s = SkipAName(inp) ) == 0 ) goto syntax;
05036     c = *s; *s = 0;
05037     if ( StrICont(inp, (UBYTE *)"arguments") == 0 ) {
05038         if ( c != '=' ) goto syntax;
05039         *s++ = c;
05040         NeedNumber(nargs, s, syntax)
05041         aflag++;
05042         inp = s;
05043     }
05044     else if ( StrICont(inp, (UBYTE *)"loopsize") == 0 ) {
05045         if ( c != '=' && c != '<' ) goto syntax;
05046         *s++ = c;
05047         if ( FG.cTable[*s] == 1 ) {
05048             NeedNumber(nloop, s, syntax)
05049             if ( nloop < 2 ) {
05050                 MesPrint("&loopsize should be at least 2");
05051                 error = 1;
05052             }
05053             if ( c == '<' ) nloop = -nloop;
05054         }
05055         else if ( tolower(*s) == 'a' && tolower(s[1]) == 'l'
05056             && tolower(s[2]) == 'l' && FG.cTable[s[3]] > 1 ) {
05057             nloop = -1; s += 3;
05058             if ( c != '=' ) goto syntax;
05059         }
05060         inp = s;
05061         lflag++;
05062     }
05063     else if ( StrICont(inp, (UBYTE *)"include") == 0 ) {
05064         if ( c != '=' ) goto syntax;
05065         *s++ = c;
05066         if ( ( inp = SkipAName(s) ) == 0 ) goto syntax;
05067         c = *inp; *inp = 0;
05068         if ( ( type = GetName(AC.varnames, s, &indexnum, WITHAUTO) ) != CINDEX ) {
05069             MesPrint("&%s is not a proper index", s);
05070             error = 1;
05071         }
05072         else if ( indexnum < WILDOFFSET
05073             && indices[indexnum].dimension == 0 ) {
05074             MesPrint("&%s should be a summable index", s);
05075             error = 1;
05076         }
05077         indexnum += AM.OffsetIndex;
05078         *inp = c;
05079         indflag++;
05080     }
05081     else if ( StrICont(inp, (UBYTE *)"outfun") == 0 ) {
05082         if ( c != '=' ) goto syntax;
05083         *s++ = c;
05084         if ( ( inp = SkipAName(s) ) == 0 ) goto syntax;
05085         c = *inp; *inp = 0;
05086         if ( ( type = GetName(AC.varnames, s, &outfun, WITHAUTO) ) != CFUNCTION ) {
05087             MesPrint("&%s is not a proper function or tensor", s);
05088             error = 1;
05089         }
05090         outfun += FUNCTION;
05091         outflag++;
05092         *inp = c;
05093     }
05094     else {
05095         MesPrint("&Unrecognized option in FindLoop or ReplaceLoop: %s", inp);
05096         error = 1;
05097         *s = c; inp = s;
05098         while ( *inp && *inp != ',' ) inp++;
05099     }
05100 }
05101 if ( *inp != 0 && mode == REPLACELOOP ) goto syntax;
05102 if ( mode == FINDLOOP && outflag > 0 ) {
05103     MesPrint("&outflag option is illegal in FindLoop");
05104     error = 1;
05105 }
05106 if ( mode == REPLACELOOP && outflag == 0 ) goto syntax;
05107 if ( aflag == 0 || lflag == 0 ) goto syntax;
05108 comfindloop[3] = funnum;
05109 comfindloop[4] = nloop;
05110 comfindloop[5] = nargs;
05111 comfindloop[6] = outfun;
05112 comfindloop[1] = 7;
05113 if ( indflag ) {
05114     if ( mode == 0 ) comfindloop[2] = indexnum + 5;
05115     else comfindloop[2] = -indexnum - 5;
05116 }
05117 else comfindloop[2] = mode;
05118 AddNtoL(comfindloop[1], comfindloop);
05119 return(error);
05120 }

```

```

05121
05122 /*
05123     #] DoFindLoop :
05124     #[ CoFindLoop :
05125 */
05126
05127 int CoFindLoop(UBYTE *inp)
05128 { return(DoFindLoop(inp,FINDLOOP)); }
05129
05130 /*
05131     #] CoFindLoop :
05132     #[ CoReplaceLoop :
05133 */
05134
05135 int CoReplaceLoop(UBYTE *inp)
05136 {
05137     int error = DoFindLoop(inp,REPLACELOOP);
05138     if ( error ) {
05139         Terminate(-1);
05140     }
05141     return(error);
05142 }
05143
05144 /*
05145     #] CoReplaceLoop :
05146     #[ CoFunPowers :
05147 */
05148
05149 static UBYTE *FunPowOptions[] = {
05150     (UBYTE *) "nofunpowers"
05151     , (UBYTE *) "commutingonly"
05152     , (UBYTE *) "allfunpowers"
05153 };
05154
05155 int CoFunPowers(UBYTE *inp)
05156 {
05157     UBYTE *option, c;
05158     int i, maxoptions = sizeof(FunPowOptions)/sizeof(UBYTE *);
05159     while ( *inp == ',' ) inp++;
05160     option = inp;
05161     inp = SkipAName(inp); c = *inp; *inp = 0;
05162     for ( i = 0; i < maxoptions; i++ ) {
05163         if ( StrICont(option,FunPowOptions[i]) == 0 ) {
05164             if ( c ) {
05165                 *inp = c;
05166                 MesPrint("&Illegal FunPowers statement");
05167                 return(1);
05168             }
05169             AC.funpowers = i;
05170             return(0);
05171         }
05172     }
05173     MesPrint("&Illegal option in FunPowers statement: %s",option);
05174     return(1);
05175 }
05176
05177 /*
05178     #] CoFunPowers :
05179     #[ CoUnitTrace :
05180 */
05181
05182 int CoUnitTrace(UBYTE *s)
05183 {
05184     WORD num;
05185     if ( FG.cTable[*s] == 1 ) {
05186         ParseNumber(num,s)
05187         if ( *s != 0 ) {
05188             nogood: MesPrint("&Value of UnitTrace should be a (positive) number or a symbol");
05189             return(1);
05190         }
05191         AC.lUnitTrace[0] = SNUMBER;
05192         AC.lUnitTrace[2] = num;
05193     }
05194     else {
05195         if ( GetName(AC.varnames,s,&num,WITHAUTO) == CSYMBOL ) {
05196             AC.lUnitTrace[0] = SYMBOL;
05197             AC.lUnitTrace[2] = num;
05198             num = -num;
05199         }
05200         else goto nogood;
05201         s = SkipAName(s);
05202         if ( *s ) goto nogood;
05203     }
05204     AC.lUnitTrace = num;
05205     return(0);
05206 }
05207

```

```

05208 /*
05209  #] CoUnitTrace :
05210  #[ CoTerm :
05211
05212  Note: termstack holds the offset of the term statement in the compiler
05213  buffer. termsortstack holds the offset of the last sort statement
05214  (or the corresponding term statement)
05215 */
05216
05217 int CoTerm(UBYTE *s)
05218 {
05219     GETIDENTITY
05220     WORD *w = AT.WorkPointer;
05221     int error = 0;
05222     while ( *s == ',' ) s++;
05223     if ( *s ) {
05224         MesPrint("&Illegal syntax for Term statement");
05225         return(1);
05226     }
05227     if ( AC.termlevel+1 >= AC.maxtermlevel ) {
05228         if ( AC.maxtermlevel <= 0 ) {
05229             AC.maxtermlevel = 20;
05230             AC.termstack = (LONG *)Malloc1(AC.maxtermlevel*sizeof(LONG), "termstack");
05231             AC.termsortstack = (LONG *)Malloc1(AC.maxtermlevel*sizeof(LONG), "termsortstack");
05232             AC.termsumcheck = (WORD *)Malloc1(AC.maxtermlevel*sizeof(WORD), "termsumcheck");
05233         }
05234         else {
05235             DoubleBuffer((void **)AC.termstack, (void **)AC.termstack+AC.maxtermlevel,
05236                 sizeof(LONG), "doubling termstack");
05237             DoubleBuffer((void **)AC.termsortstack,
05238                 (void **)AC.termsortstack+AC.maxtermlevel,
05239                 sizeof(LONG), "doubling termsortstack");
05240             DoubleBuffer((void **)AC.termsumcheck,
05241                 (void **)AC.termsumcheck+AC.maxtermlevel,
05242                 sizeof(LONG), "doubling termsumcheck");
05243             AC.maxtermlevel *= 2;
05244         }
05245     }
05246     AC.termsumcheck[AC.termlevel] = NestingChecksum();
05247     AC.termstack[AC.termlevel] = cbuf[AC.cbunum].Pointer
05248         - cbuf[AC.cbunum].Buffer + 2;
05249     AC.termsortstack[AC.termlevel] = AC.termstack[AC.termlevel] + 1;
05250     AC.termlevel++;
05251     *w++ = TYPETERM;
05252     w++;
05253     *w++ = cbuf[AC.cbunum].numlhs;
05254     *w++ = cbuf[AC.cbunum].numlhs;
05255     AT.WorkPointer[1] = w - AT.WorkPointer;
05256     AddNtoL(AT.WorkPointer[1], AT.WorkPointer);
05257     return(error);
05258 }
05259
05260 /*
05261  #] CoTerm :
05262  #[ CoEndTerm :
05263 */
05264
05265 int CoEndTerm(UBYTE *s)
05266 {
05267     CBUF *C = cbuf+AC.cbunum;
05268     while ( *s == ',' ) s++;
05269     if ( *s ) {
05270         MesPrint("&Illegal syntax for EndTerm statement");
05271         return(1);
05272     }
05273     if ( AC.termlevel <= 0 ) {
05274         MesPrint("&EndTerm without corresponding Argument statement");
05275         return(1);
05276     }
05277     AC.termlevel--;
05278     cbuf[AC.cbunum].Buffer[AC.termstack[AC.termlevel]] = C->numlhs;
05279     cbuf[AC.cbunum].Buffer[AC.termsortstack[AC.termlevel]] = C->numlhs;
05280     if ( AC.termsumcheck[AC.termlevel] != NestingChecksum() ) {
05281         MesNesting();
05282         return(1);
05283     }
05284     return(0);
05285 }
05286
05287 /*
05288  #] CoEndTerm :
05289  #[ CoSort :
05290 */
05291
05292 int CoSort(UBYTE *s)
05293 {
05294     GETIDENTITY

```

```

05295     WORD *w = AT.WorkPointer;
05296     int error = 0;
05297     while ( *s == ',' ) s++;
05298     if ( *s ) {
05299         MesPrint("&Illegal syntax for Sort statement");
05300         error = 1;
05301     }
05302     if ( AC.termlevel <= 0 ) {
05303         MesPrint("&The Sort statement can only be used inside a term environment");
05304         error = 1;
05305     }
05306     if ( error ) return(error);
05307     *w++ = TYPESORT;
05308     w++;
05309     w++;
05310     cbuf[AC.cbunum].Buffer[AC.termstack[AC.termlevel-1]] =
05311         *w = cbuf[AC.cbunum].numlhs+1;
05312     w++;
05313     AC.termstack[AC.termlevel-1] = cbuf[AC.cbunum].Pointer
05314         - cbuf[AC.cbunum].Buffer + 3;
05315     if ( AC.termcheck[AC.termlevel-1] != NestingChecksum() - 1 ) {
05316         MesNesting();
05317         return(1);
05318     }
05319     AT.WorkPointer[1] = w - AT.WorkPointer;
05320     AddNtoL(AT.WorkPointer[1],AT.WorkPointer);
05321     return(error);
05322 }
05323
05324 /*
05325 #] CoSort :
05326 #[ CoPolyFun :
05327
05328 Collect,functionname
05329 */
05330
05331 int CoPolyFun(UBYTE *s)
05332 {
05333     GETIDENTITY
05334     WORD numfun;
05335     int type, error = 0;
05336     UBYTE *t;
05337     AR.PolyFun = AC.lPolyFun = 0;
05338     AR.PolyFunInv = AC.lPolyFunInv = 0;
05339     AR.PolyFunType = AC.lPolyFunType = 0;
05340     AR.PolyFunExp = AC.lPolyFunExp = 0;
05341     AR.PolyFunVar = AC.lPolyFunVar = 0;
05342     AR.PolyFunPow = AC.lPolyFunPow = 0;
05343     if ( *s == 0 ) { return(0); }
05344     t = SkipAName(s);
05345     if ( t == 0 || *t != 0 ) {
05346         MesPrint("&PolyFun statement needs a single commuting function for its argument");
05347         return(1);
05348     }
05349     if ( ( ( type = GetName(AC.varnames,s,&numfun,WITHAUTO) ) != CFUNCTION )
05350     || ( functions[numfun].spec != 0 ) || ( functions[numfun].commute != 0 ) ) {
05351         MesPrint("&%s should be a regular commuting function",s);
05352         if ( type < 0 ) {
05353             if ( GetName(AC.exprnames,s,&numfun,NOAUTO) == NAMENOTFOUND )
05354                 AddFunction(s,0,0,0,0,0,-1,-1);
05355         }
05356         error = 1;
05357     }
05358     else {
05359         AR.PolyFun = AC.lPolyFun = numfun+FUNCTION;
05360         AR.PolyFunType = AC.lPolyFunType = 1;
05361     }
05362     #ifdef WITHFLOAT
05363     if ( mpfaux_ != 0 ) {
05364         MesPrint("&Simultaneous use of PolyFun and float_ is not allowed.");
05365         error = 1;
05366     }
05367     #endif
05368     return(error);
05369 }
05370
05371 /*
05372 #] CoPolyFun :
05373 #[ CoPolyRatFun :
05374
05375 PolyRatFun [,functionname[,functionname](option)]
05376 */
05377
05378 int CoPolyRatFun(UBYTE *s)
05379 {
05380     GETIDENTITY
05381     WORD numfun;

```



```

05382     int type, error = 0;
05383     UBYTE *t, c;
05384     AR.PolyFun = AC.lPolyFun = 0;
05385     AR.PolyFunInv = AC.lPolyFunInv = 0;
05386     AR.PolyFunType = AC.lPolyFunType = 0;
05387     AR.PolyFunExp = AC.lPolyFunExp = 0;
05388     AR.PolyFunVar = AC.lPolyFunVar = 0;
05389     AR.PolyFunPow = AC.lPolyFunPow = 0;
05390     if ( *s == 0 ) return(error);
05391     t = SkipAName(s);
05392     if ( t == 0 ) goto NumErr;
05393     c = *t; *t = 0;
05394 #ifdef WITHFLOAT
05395     if ( mpfaux_ != 0 ) {
05396         MesPrint("&Simultaneous use of PolyFun and float_ is not allowed.");
05397         error = 1;
05398     }
05399 #endif
05400     if ( ( ( type = GetName(AC.varnames,s,&numfun,WITHAUTO) ) != CFUNCTION )
05401 || ( functions[numfun].spec != 0 ) || ( functions[numfun].commute != 0 ) ) {
05402         MesPrint("&%s should be a regular commuting function",s);
05403         if ( type < 0 ) {
05404             if ( GetName(AC.exprnames,s,&numfun,NOAUTO) == NAMENOTFOUND )
05405                 AddFunction(s,0,0,0,0,0,-1,-1);
05406         }
05407         return(1);
05408     }
05409     AR.PolyFun = AC.lPolyFun = numfun+FUNCTION;
05410     AR.PolyFunInv = AC.lPolyFunInv = 0;
05411     AR.PolyFunType = AC.lPolyFunType = 2;
05412     AC.PolyRatFunChanged = 1;
05413     if ( c == 0 ) return(error);
05414     *t = c;
05415     if ( *t == '-' ) { AC.PolyRatFunChanged = 0; t++; }
05416     while ( *t == ',' || *t == ' ' || *t == '\t' ) t++;
05417     if ( *t == 0 ) return(error);
05418     if ( *t != '(' ) {
05419         s = t;
05420         t = SkipAName(s);
05421         if ( t == 0 ) goto NumErr;
05422         c = *t; *t = 0;
05423         if ( ( type = GetName(AC.varnames,s,&numfun,WITHAUTO) ) != CFUNCTION )
05424         || ( functions[numfun].spec != 0 ) || ( functions[numfun].commute != 0 ) ) {
05425             MesPrint("&%s should be a regular commuting function",s);
05426             if ( type < 0 ) {
05427                 if ( GetName(AC.exprnames,s,&numfun,NOAUTO) == NAMENOTFOUND )
05428                     AddFunction(s,0,0,0,0,0,-1,-1);
05429             }
05430             return(1);
05431         }
05432         AR.PolyFunInv = AC.lPolyFunInv = numfun+FUNCTION;
05433         if ( c == 0 ) return(error);
05434         *t = c;
05435         if ( *t == '-' ) { AC.PolyRatFunChanged = 0; t++; }
05436         while ( *t == ',' || *t == ' ' || *t == '\t' ) t++;
05437         if ( *t == 0 ) return(error);
05438     }
05439     if ( *t == '(' ) {
05440         t++;
05441         while ( *t == ',' || *t == ' ' || *t == '\t' ) t++;
05442     /*
05443         Next we need a keyword like
05444         (divergence,ep)
05445         (expand,ep,maxpow)
05446     */
05447     s = t;
05448     t = SkipAName(s);
05449     if ( t == 0 ) goto NumErr;
05450     c = *t; *t = 0;
05451     if ( ( StrICmp(s,(UBYTE *)"divergence") == 0 )
05452 || ( StrICmp(s,(UBYTE *)"finddivergence") == 0 ) ) {
05453         if ( c != ',' ) {
05454             MesPrint("&Illegal option field in PolyRatFun statement.");
05455             return(1);
05456         }
05457         *t = c;
05458         while ( *t == ',' || *t == ' ' || *t == '\t' ) t++;
05459         s = t;
05460         t = SkipAName(s);
05461         if ( t == 0 ) goto NumErr;
05462         c = *t; *t = 0;
05463         if ( ( type = GetName(AC.varnames,s,&AC.lPolyFunVar,WITHAUTO) ) != CSYMBOL ) {
05464             MesPrint("&Illegal symbol %s in option field in PolyRatFun statement.",s);
05465             return(1);
05466         }
05467         *t = c;
05468         while ( *t == ',' || *t == ' ' || *t == '\t' ) t++;

```

```

05469         if ( *t != ')' ) {
05470             MesPrint("&Illegal termination of option in PolyRatFun statement.");
05471             return(1);
05472         }
05473         AR.PolyFunExp = AC.lPolyFunExp = 1;
05474         AR.PolyFunVar = AC.lPolyFunVar;
05475         symbols[AC.lPolyFunVar].minpower = -MAXPOWER;
05476         symbols[AC.lPolyFunVar].maxpower = MAXPOWER;
05477     }
05478     else if ( StrICmp(s, (UBYTE *)"expand") == 0 ) {
05479         WORD x = 0, etype = 2;
05480         if ( c != ',' ) {
05481             MesPrint("&Illegal option field in PolyRatFun statement.");
05482             return(1);
05483         }
05484         *t = c;
05485         while ( *t == ',' || *t == ' ' || *t == '\t' ) t++;
05486         s = t;
05487         t = SkipAName(s);
05488         if ( t == 0 ) goto NumErr;
05489         c = *t; *t = 0;
05490         if ( ( type = GetName(AC.varnames, s, &AC.lPolyFunVar, WITHAUTO) ) != CSYMBOL ) {
05491             MesPrint("&Illegal symbol %s in option field in PolyRatFun statement.", s);
05492             return(1);
05493         }
05494         *t = c;
05495         while ( *t == ',' || *t == ' ' || *t == '\t' ) t++;
05496         if ( *t > '9' || *t < '0' ) {
05497             MesPrint("&Illegal option field in PolyRatFun statement.");
05498             return(1);
05499         }
05500         while ( *t <= '9' && *t >= '0' ) x = 10*x + *t++ - '0';
05501         while ( *t == ',' || *t == ' ' || *t == '\t' ) t++;
05502         if ( *t != ')' ) {
05503             s = t;
05504             t = SkipAName(s);
05505             if ( t == 0 ) goto ParErr;
05506             c = *t; *t = 0;
05507             if ( StrICmp(s, (UBYTE *)"fixed") == 0 ) {
05508                 etype = 3;
05509             }
05510             else if ( StrICmp(s, (UBYTE *)"relative") == 0 ) {
05511                 etype = 2;
05512             }
05513             else {
05514                 MesPrint("&Illegal termination of option in PolyRatFun statement.");
05515                 return(1);
05516             }
05517             *t = c;
05518             while ( *t == ',' || *t == ' ' || *t == '\t' ) t++;
05519             if ( *t != ')' ) {
05520                 MesPrint("&Illegal termination of option in PolyRatFun statement.");
05521                 return(1);
05522             }
05523         }
05524         AR.PolyFunExp = AC.lPolyFunExp = etype;
05525         AR.PolyFunVar = AC.lPolyFunVar;
05526         AR.PolyFunPow = AC.lPolyFunPow = x;
05527         symbols[AC.lPolyFunVar].minpower = -MAXPOWER;
05528         symbols[AC.lPolyFunVar].maxpower = MAXPOWER;
05529     }
05530     else {
05531 ParErr: MesPrint("&Illegal option %s in PolyRatFun statement.", s);
05532         return(1);
05533     }
05534     t++;
05535     while ( *t == ',' || *t == ' ' || *t == '\t' ) t++;
05536     if ( *t == 0 ) return(error);
05537 }
05538 NumErr:
05539 MesPrint("&PolyRatFun statement needs one or two commuting function(s) for its argument(s)");
05540 return(1);
05541 }
05542
05543 /*
05544 #] CoPolyRatFun :
05545 #[ CoMerge :
05546 */
05547
05548 int CoMerge(UBYTE *inp)
05549 {
05550     UBYTE *s = inp;
05551     int type;
05552     WORD numfunc, option = 0;
05553     if ( tolower(s[0]) == 'o' && tolower(s[1]) == 'n' && tolower(s[2]) == 'c' &&
05554         tolower(s[3]) == 'e' && tolower(s[4]) == ',' ) {
05555         option = 1; s += 5;

```

```

05556     }
05557     else if ( tolower(s[0]) == 'a' && tolower(s[1]) == 'l' && tolower(s[2]) == 'l' &&
05558              tolower(s[3]) == ',' ) {
05559         option = 0; s += 4;
05560     }
05561     if ( *s == '$' ) {
05562         if ( ( type = GetName(AC.dollarnames,s+1,&numfunc,NOAUTO) ) == CDOLLAR )
05563             numfunc = -numfunc;
05564         else {
05565             MesPrint("&%s is undefined",s);
05566             numfunc = AddDollar(s+1,DOLINDEX,&one,1);
05567             return(1);
05568         }
05569     tests: s = SkipAName(s);
05570     if ( *s != 0 ) {
05571         MesPrint("&Merge/shuffle should have a single function or $variable for its argument");
05572         return(1);
05573     }
05574     }
05575     else if ( ( type = GetName(AC.varnames,s,&numfunc,WITHAUTO) ) == CFUNCTION ) {
05576         numfunc += FUNCTION;
05577         goto tests;
05578     }
05579     else if ( type != -1 ) {
05580         if ( type != CDUBIOUS ) {
05581             NameConflict(type,s);
05582             type = MakeDubious(AC.varnames,s,&numfunc);
05583         }
05584         return(1);
05585     }
05586     else {
05587         MesPrint("&%s is not a function",s);
05588         numfunc = AddFunction(s,0,0,0,0,0,-1,-1) + FUNCTION;
05589         return(1);
05590     }
05591     Add4Com(TYPEMERGE,numfunc,option);
05592     return(0);
05593 }
05594
05595 /*
05596  #] CoMerge :
05597  #[ CoStuffle :
05598
05599     Important for future options: The bit, given by 256 (bit 8) is reserved
05600     internally for keeping track of the sign in the number of Stuffle
05601     additions.
05602 */
05603
05604 int CoStuffle(UBYTE *inp)
05605 {
05606     UBYTE *s = inp, *ss, c;
05607     int type;
05608     WORD numfunc, option = 0;
05609     if ( tolower(s[0]) == 'o' && tolower(s[1]) == 'n' && tolower(s[2]) == 'c' &&
05610          tolower(s[3]) == 'e' && tolower(s[4]) == ',' ) {
05611         option = 1; s += 5;
05612     }
05613     else if ( tolower(s[0]) == 'a' && tolower(s[1]) == 'l' && tolower(s[2]) == 'l' &&
05614              tolower(s[3]) == ',' ) {
05615         option = 0; s += 4;
05616     }
05617     ss = SkipAName(s);
05618     c = *ss; *ss = 0;
05619     if ( *s == '$' ) {
05620         if ( ( type = GetName(AC.dollarnames,s+1,&numfunc,NOAUTO) ) == CDOLLAR )
05621             numfunc = -numfunc;
05622         else {
05623             MesPrint("&%s is undefined",s);
05624             numfunc = AddDollar(s+1,DOLINDEX,&one,1);
05625             return(1);
05626         }
05627     tests: *ss = c;
05628     if ( *ss != '+' && *ss != '-' && ss[1] != 0 ) {
05629         MesPrint("&Stuffle should have a single function or $variable for its argument, followed
05630 by either + or -");
05631         return(1);
05632     }
05633     if ( *ss == '-' ) option += 2;
05634     }
05635     else if ( ( type = GetName(AC.varnames,s,&numfunc,WITHAUTO) ) == CFUNCTION ) {
05636         numfunc += FUNCTION;
05637         goto tests;
05638     }
05639     else if ( type != -1 ) {
05640         if ( type != CDUBIOUS ) {
05641             NameConflict(type,s);
05642             type = MakeDubious(AC.varnames,s,&numfunc);

```

```

05642     }
05643     return(1);
05644 }
05645 else {
05646     MesPrint("%s is not a function",s);
05647     numfunc = AddFunction(s,0,0,0,0,0,-1,-1) + FUNCTION;
05648     return(1);
05649 }
05650 Add4Com(TYPESTUFFLE,numfunc,option);
05651 return(0);
05652 }
05653
05654 /*
05655 #] CoStuffle :
05656 #[ CoProcessBucket :
05657 */
05658
05659 int CoProcessBucket(UBYTE *s)
05660 {
05661     LONG x;
05662     while ( *s == ',' || *s == '=' ) s++;
05663     ParseNumber(x,s)
05664     if ( *s && *s != ' ' && *s != '\t' ) {
05665         MesPrint("&Numerical value expected for ProcessBucketSize");
05666         return(1);
05667     }
05668     AC.ProcessBucketSize = x;
05669     return(0);
05670 }
05671
05672 /*
05673 #] CoProcessBucket :
05674 #[ CoThreadBucket :
05675 */
05676
05677 int CoThreadBucket(UBYTE *s)
05678 {
05679     LONG x;
05680     while ( *s == ',' || *s == '=' ) s++;
05681     ParseNumber(x,s)
05682     if ( *s && *s != ' ' && *s != '\t' ) {
05683         MesPrint("&Numerical value expected for ThreadBucketSize");
05684         return(1);
05685     }
05686     if ( x <= 0 ) {
05687         Warning("Negative of zero value not allowed for ThreadBucketSize. Adjusted to 1.");
05688         x = 1;
05689     }
05690     AC.ThreadBucketSize = x;
05691 #ifdef WITHPTHREADS
05692     if ( AS.MultiThreaded ) MakeThreadBuckets(-1,1);
05693 #endif
05694     return(0);
05695 }
05696
05697 /*
05698 #] CoThreadBucket :
05699 #[ DoArgPlode :
05700
05701 Syntax: a list of functions.
05702 If the functions have an argument it must be a function.
05703 In the case f(g) we treat f(g(...)) with g any argument.
05704 (not yet implemented)
05705 */
05706
05707 int DoArgPlode(UBYTE *s, int par)
05708 {
05709     GETIDENTITY
05710     WORD numfunc, type, error = 0, *w, n;
05711     UBYTE *t,c;
05712     int i;
05713     w = AT.WorkPointer;
05714     *w++ = par;
05715     w++;
05716     while ( *s == ',' ) s++;
05717     while ( *s ) {
05718         if ( *s == '$' ) {
05719             MesPrint("&We don't do dollar variables yet in ArgImplode/ArgExplode");
05720             return(1);
05721         }
05722         t = s;
05723         if ( ( s = SkipAName(s) ) == 0 ) return(1);
05724         c = *s; *s = 0;
05725         if ( ( type = GetName(AC.varnames,t,&numfunc,WITHAUTO) ) == CFUNCTION ) {
05726             numfunc += FUNCTION;
05727         }
05728         else if ( type != -1 ) {

```

```

05729         if ( type != CDUBIOUS ) {
05730             NameConflict(type,t);
05731             type = MakeDubious(AC.varnames,t,&numfunc);
05732         }
05733         error = 1;
05734     }
05735     else {
05736         MesPrint("%s is not a function",t);
05737         numfunc = AddFunction(s,0,0,0,0,0,-1,-1) + FUNCTION;
05738         return(1);
05739     }
05740     *s = c;
05741     *w++ = numfunc;
05742     *w++ = FUNHEAD;
05743     #if FUNHEAD > 2
05744     for ( i = 2; i < FUNHEAD; i++ ) *w++ = 0;
05745     #endif
05746     if ( *s && *s != ',' ) {
05747         MesPrint("&Illegal character in ArgImplode/ArgExplode statement: %s",s);
05748         return(1);
05749     }
05750     while ( *s == ',' ) s++;
05751 }
05752 n = w - AT.WorkPointer;
05753 AT.WorkPointer[1] = n;
05754 AddNtoL(n,AT.WorkPointer);
05755 return(error);
05756 }
05757
05758 /*
05759  #] DoArgPlode :
05760  #[ CoArgExplode :
05761  */
05762
05763 int CoArgExplode(UBYTE *s) { return(DoArgPlode(s,TYPEARGEXPLODE)); }
05764
05765 /*
05766  #] CoArgExplode :
05767  #[ CoArgImplode :
05768  */
05769
05770 int CoArgImplode(UBYTE *s) { return(DoArgPlode(s,TYPEARGIMPLODE)); }
05771
05772 /*
05773  #] CoArgImplode :
05774  #[ CoClearTable :
05775  */
05776
05777 int CoClearTable(UBYTE *s)
05778 {
05779     UBYTE c, *t;
05780     int j, type, error = 0;
05781     WORD numfun;
05782     TABLES T, TT;
05783     if ( *s == 0 ) {
05784         MesPrint("&The ClearTable statement needs at least one (table) argument.");
05785         return(1);
05786     }
05787     while ( *s ) {
05788         t = s;
05789         s = SkipAName(s);
05790         c = *s; *s = 0;
05791         if ( ( ( type = GetName(AC.varnames,t,&numfun,WITHAUTO) ) != CFUNCTION )
05792             && type != CDUBIOUS ) {
05793             MesPrint("&%s is not a table",t);
05794             error = 4;
05795             if ( type < 0 ) numfun = AddFunction(t,0,0,0,0,0,-1,-1);
05796             *s = c;
05797             if ( *s == ',' ) s++;
05798             continue;
05799         }
05800     /*
05801         else if ( ( ( T = functions[numfun].tabl ) == 0 )
05802             || ( T->sparse == 0 ) ) goto nofunc;
05803     */
05804     else if ( ( T = functions[numfun].tabl ) == 0 ) goto nofunc;
05805     numfun += FUNCTION;
05806     *s = c;
05807     if ( *s == ',' ) s++;
05808     /*
05809     Now we clear the table.
05810     */
05811     if ( T->sparse ) {
05812         if ( T->boomlijst ) M_free(T->boomlijst,"TableTree");
05813         for ( j = 0; j < T->buffersfill; j++ ) { /* was <= */
05814             finishcbuf(T->buffers[j]);
05815         }

```

```

05816         if ( T->buffers ) M_free(T->buffers,"Table buffers");
05817         finishcbuf(T->bufnum);
05818
05819         T->boomlijst = 0;
05820         T->numtree = 0; T->rootnum = 0; T->MaxTreeSize = 0;
05821         T->boomlijst = 0;
05822         T->bufnum = inicbufs();
05823         T->bufferssize = 8;
05824         T->buffers = (WORD *)Malloc1(sizeof(WORD)*T->bufferssize,"Table buffers");
05825         T->buffersfill = 0;
05826         T->buffers[T->buffersfill++] = T->bufnum;
05827
05828         T->totind = 0;           /* At the moment there are this many */
05829         T->reserved = 0;
05830
05831         ClearTableTree(T);
05832
05833         if ( T->spare ) {
05834             if ( T->tablepointers ) M_free(T->tablepointers,"tablepointers");
05835             T->tablepointers = 0;
05836             TT = T->spare;
05837             if ( TT->tablepointers ) M_free(TT->tablepointers,"tablepointers");
05838             for ( j = 0; j < TT->buffersfill; j++ ) {
05839                 finishcbuf(TT->buffers[j]);
05840             }
05841             if ( TT->boomlijst ) M_free(TT->boomlijst,"TableTree");
05842             if ( TT->buffers ) M_free(TT->buffers,"Table buffers");
05843             if ( TT->mm ) M_free(TT->mm,"tableminmax");
05844             if ( TT->flags ) M_free(TT->flags,"tableflags");
05845             M_free(TT,"table");
05846             SpareTable(T);
05847         }
05848     }
05849     else EmptyTable(T);
05850 }
05851 return(error);
05852 }
05853
05854 /*
05855 #] CoClearTable :
05856 #[ CoDenominators :
05857 */
05858
05859 int CoDenominators(UBYTE *s)
05860 {
05861     WORD numfun;
05862     int type;
05863     UBYTE *t = SkipAName(s), *t1;
05864     if ( t == 0 ) goto syntaxerror;
05865     t1 = t; while ( *t1 == ',' || *t1 == ' ' || *t1 == '\t' ) t1++;
05866     if ( *t1 ) goto syntaxerror;
05867     *t = 0;
05868     if ( ( ( type = GetName(AC.varnames,s,&numfun,WITHAUTO) ) != CFUNCTION )
05869         || ( functions[numfun].spec != 0 ) ) {
05870         if ( type < 0 ) {
05871             if ( GetName(AC.exprnames,s,&numfun,NOAUTO) == NAMENOTFOUND )
05872                 AddFunction(s,0,0,0,0,0,-1,-1);
05873         }
05874         goto syntaxerror;
05875     }
05876     Add3Com(TYPEDENOMINATORS,numfun+FUNCTION);
05877     return(0);
05878 syntaxerror:
05879     MesPrint("&Denominators statement needs one regular function for its argument");
05880     return(1);
05881 }
05882
05883 /*
05884 #] CoDenominators :
05885 #[ CoDropCoefficient :
05886 */
05887
05888 int CoDropCoefficient(UBYTE *s)
05889 {
05890     if ( *s == 0 ) {
05891         Add2Com(TYPEDROPCOEFFICIENT);
05892         return(0);
05893     }
05894     MesPrint("&Illegal argument in DropCoefficient statement: '%s'",s);
05895     return(1);
05896 }
05897 /*
05898 #] CoDropCoefficient :
05899 #[ CoDropSymbols :
05900 */
05901
05902 int CoDropSymbols(UBYTE *s)

```

```

05903 {
05904     if ( *s == 0 ) {
05905         Add2Com(TYPEDROPSYMBOLS)
05906         return(0);
05907     }
05908     MesPrint("Illegal argument in DropSymbols statement: '%s'",s);
05909     return(1);
05910 }
05911 /*
05912 #] CoDropSymbols :
05913 #[ CoToPolynomial :
05914
05915 Converts the current term as much as possible to symbols.
05916 Keeps a list of all objects converted to symbols in AM.sbufnum.
05917 Note that this cannot be executed in parallel because we have only
05918 a single compiler buffer for this. Hence we switch on the noparallel
05919 module option.
05920
05921 Option(s):
05922     OnlyFunctions [,name1][,name2][,...,namem];
05923 */
05924
05925 int CoToPolynomial(UBYTE *inp)
05926 {
05927     int error = 0;
05928     while ( *inp == ' ' || *inp == ',' || *inp == '\t' ) inp++;
05929     if ( ( AC.topolynomialflag & ~TOPOLYNOMIALFLAG ) != 0 ) {
05930         MesPrint("&ToPolynomial statement and FactArg statement are not allowed in the same module");
05931         return(1);
05932     }
05933     if ( AO.OptimizeResult.code != NULL ) {
05934         MesPrint("&Using ToPolynomial statement when there are still optimization results active.");
05935         MesPrint("&Please use #ClearOptimize instruction first.");
05936         MesPrint("&This will loose the optimized expression.");
05937         return(1);
05938     }
05939     if ( *inp == 0 ) {
05940         Add3Com(TYPETOPOLYNOMIAL,DOALL)
05941     }
05942     else {
05943         int numargs = 0;
05944         WORD *funnums = 0, type, num;
05945         UBYTE *s, c;
05946         s = SkipAName(inp);
05947         if ( s == 0 ) return(1);
05948         c = *s; *s = 0;
05949         if ( StrICmp(inp,(UBYTE *)"onlyfunctions") ) {
05950             MesPrint("&Illegal option %s in ToPolynomial statement",inp);
05951             *s = c;
05952             return(1);
05953         }
05954         *s = c;
05955         inp = s;
05956         while ( *inp == ' ' || *inp == ',' || *inp == '\t' ) inp++;
05957         s = inp;
05958         while ( *s ) s++;
05959     /*
05960     Get definitely enough space for the numbers of the functions
05961     */
05962     funnums = (WORD *)Malloc1(((LONG)(s-inp)+3)*sizeof(WORD),"ToPolynomial");
05963     while ( *inp ) {
05964         s = SkipAName(inp);
05965         if ( s == 0 ) return(1);
05966         c = *s; *s = 0;
05967         type = GetName(AC.varnames,inp,&num,WITHAUTO);
05968         if ( type != CFUNCTION ) {
05969             MesPrint("&%s is not a function in ToPolynomial statement",inp);
05970             error = 1;
05971         }
05972         funnums[3+numargs++] = num+FUNCTION;
05973         *s = c;
05974         inp = s;
05975         while ( *inp == ' ' || *inp == ',' || *inp == '\t' ) inp++;
05976     }
05977     funnums[0] = TYPETOPOLYNOMIAL;
05978     funnums[1] = numargs+3;
05979     funnums[2] = ONLYFUNCTIONS;
05980
05981     AddNtoL(numargs+3,funnums);
05982     if ( funnums ) M_free(funnums,"ToPolynomial");
05983 }
05984 AC.topolynomialflag |= TOPOLYNOMIALFLAG;
05985 #ifdef WITHMPI
05986 /* In ParFORM, ToPolynomial has to be executed on the master. */
05987 AC.mparallelflag |= NOPARALLEL_CONVPOLY;
05988 #endif
05989 return(error);

```

```

05990 }
05991
05992 /*
05993  #] CoToPolynomial :
05994  #[ CoFromPolynomial :
05995
05996  Converts the current term as much as possible back from extra symbols
05997  to their original values. Does not look inside functions.
05998 */
05999
06000 int CoFromPolynomial(UBYTE *inp)
06001 {
06002     while ( *inp == ' ' || *inp == ',' || *inp == '\t' ) inp++;
06003     if ( *inp == 0 ) {
06004         if ( AO.OptimizeResult.code != NULL ) {
06005             MesPrint("&Using FromPolynomial statement when there are still optimization results
active.");
06006             MesPrint("&Please use #ClearOptimize instruction first.");
06007             MesPrint("&This will loose the optimized expression.");
06008             return(1);
06009         }
06010         Add2Com(TYPEFROMPOLYNOMIAL)
06011         return(0);
06012     }
06013     MesPrint("&Illegal argument in FromPolynomial statement: '%s'",inp);
06014     return(1);
06015 }
06016
06017 /*
06018  #] CoFromPolynomial :
06019  #[ CoArgToExtraSymbol :
06020
06021  Converts the specified function arguments into extra symbols.
06022
06023  Syntax: ArgToExtraSymbol [ToNumber] [<argument specifications>]
06024 */
06025
06026 int CoArgToExtraSymbol(UBYTE *s)
06027 {
06028     CBUF *C = cbuf + AC.cbunum;
06029     WORD *lhs;
06030
06031     /* TODO: resolve interference with rational arithmetic. (#138) */
06032     if ( ( AC.topolynomialflag & ~TOPOLYNOMIALFLAG ) != 0 ) {
06033         MesPrint("&ArgToExtraSymbol statement and FactArg statement are not allowed in the same
module");
06034         return(1);
06035     }
06036     if ( AO.OptimizeResult.code != NULL ) {
06037         MesPrint("&Using ArgToExtraSymbol statement when there are still optimization results
active.");
06038         MesPrint("&Please use #ClearOptimize instruction first.");
06039         MesPrint("&This will loose the optimized expression.");
06040         return(1);
06041     }
06042
06043     SkipSpaces(&s);
06044     int tonumber = ConsumeOption(&s, "tonumber");
06045
06046     int ret = DoArgument(s, TYPEARGTOEXTRASymbol);
06047     if ( ret ) return(ret);
06048
06049     /*
06050     * The "scale" parameter is unused. Instead, we put the "tonumber"
06051     * parameter.
06052     */
06053     lhs = C->lhs[C->numlhs];
06054     if ( lhs[4] != 1 ) {
06055         Warning("scale parameter (^n) is ignored in ArgToExtraSymbol");
06056     }
06057     lhs[4] = tonumber;
06058
06059     AC.topolynomialflag |= TOPOLYNOMIALFLAG; /* This flag is also used in ParFORM. */
06060 #ifdef WITHMPI
06061     /*
06062     * In ParFORM, the conversion to extra symbols has to be performed on
06063     * the master.
06064     */
06065     AC.mparallelfalg |= NOPARALLEL_CONVPOLY;
06066 #endif
06067     return(0);
06068 }
06069
06070
06071 /*
06072  #] CoArgToExtraSymbol :
06073  #[ CoExtraSymbols :

```



```

06074 */
06075
06076 int CoExtraSymbols(UBYTE *inp)
06077 {
06078     UBYTE *arg1, *arg2, c, *s;
06079     WORD i, j, type, number;
06080     while ( *inp == ' ' || *inp == ',' || *inp == '\t' ) inp++;
06081     if ( FG.cTable[*inp] != 0 ) {
06082         MesPrint("&Illegal argument in ExtraSymbols statement: '%s'",inp);
06083         return(1);
06084     }
06085     arg1 = inp;
06086     while ( FG.cTable[*inp] == 0 ) inp++;
06087     c = *inp; *inp = 0;
06088     if ( ( StrICmp(arg1, (UBYTE *) "array") == 0 )
06089         || ( StrICmp(arg1, (UBYTE *) "vector") == 0 ) ) {
06090         AC.extrasymbols = 1;
06091     }
06092     else if ( StrICmp(arg1, (UBYTE *) "underscore") == 0 ) {
06093         AC.extrasymbols = 0;
06094     }
06095     /*
06096     else if ( StrICmp(arg1, (UBYTE *) "nothing") == 0 ) {
06097         AC.extrasymbols = 2;
06098     }
06099     */
06100     else {
06101         MesPrint("&Illegal keyword in ExtraSymbols statement: '%s'",arg1);
06102         return(1);
06103     }
06104     *inp = c;
06105     while ( *inp == ' ' || *inp == ',' || *inp == '\t' ) inp++;
06106     if ( FG.cTable[*inp] != 0 ) {
06107         MesPrint("&Illegal argument in ExtraSymbols statement: '%s'",inp);
06108         return(1);
06109     }
06110     arg2 = inp;
06111     while ( FG.cTable[*inp] <= 1 ) inp++;
06112     if ( *inp != 0 ) {
06113         MesPrint("&Illegal end of ExtraSymbols statement: '%s'",inp);
06114         return(1);
06115     }
06116     /*
06117         Now check whether this object has been declared already.
06118         That would not be allowed.
06119     */
06120     if ( AC.extrasymbols == 1 ) {
06121         type = GetName(AC.varnames,arg2,&number,NOAUTO);
06122         if ( type != NAMENOTFOUND ) {
06123             MesPrint("&ExtraSymbols statement: '%s' has already been declared before",arg2);
06124             return(1);
06125         }
06126     }
06127     else if ( AC.extrasymbols == 0 ) {
06128         if ( *arg2 == 'N' ) {
06129             s = arg2+1;
06130             while ( FG.cTable[*s] == 1 ) s++;
06131             if ( *s == 0 ) {
06132                 MesPrint("&ExtraSymbols statement: '%s' creates conflicts with summed indices",arg2);
06133                 return(1);
06134             }
06135         }
06136     }
06137     if ( AC.extrasym ) { M_free(AC.extrasym,"extrasym"); AC.extrasym = 0; }
06138     i = inp - arg2 + 1;
06139     AC.extrasym = (UBYTE *) Malloc1(i*sizeof(UBYTE),"extrasym");
06140     for ( j = 0; j < i; j++ ) AC.extrasym[j] = arg2[j];
06141     return(0);
06142 }
06143
06144 /*
06145 #] CoExtraSymbols :
06146 #[ GetIfDollarFactor :
06147 */
06148
06149 WORD *GetIfDollarFactor(UBYTE **inp, WORD *w)
06150 {
06151     LONG x;
06152     WORD number;
06153     UBYTE *name, c, *s;
06154     s = *inp;
06155     if ( FG.cTable[*s] == 1 ) {
06156         x = 0;
06157         while ( FG.cTable[*s] == 1 ) {
06158             x = 10*x + *s++ - '0';
06159             if ( x >= MAXPOSITIVE ) {
06160                 MesPrint("&Value in dollar factor too large");

```

```

06161         while ( FG.cTable[*s] == 1 ) s++;
06162         *inp = s;
06163         return(0);
06164     }
06165 }
06166 *w++ = IFDOLLAREXTRA;
06167 *w++ = 3;
06168 *w++ = -x-1;
06169 *inp = s;
06170 return(w);
06171 }
06172 if ( *s != '$' ) {
06173     MesPrint("&Factor indicator for $-variable should be a number or a $-variable.");
06174     return(0);
06175 }
06176 s++; name = s;
06177 while ( FG.cTable[*s] < 2 ) s++;
06178 c = *s; *s = 0;
06179 if ( GetName(AC.dollarnames,name,&number,NOAUTO) == NAMENOTFOUND ) {
06180     MesPrint("&dollar in if statement should have been defined previously");
06181     return(0);
06182 }
06183 *s = c;
06184 *w++ = IFDOLLAREXTRA;
06185 *w++ = 3;
06186 *w++ = number;
06187 if ( c == '[' ) {
06188     s++;
06189     *inp = s;
06190     if ( ( w = GetIfDollarFactor(inp,w) ) == 0 ) return(0);
06191     s = *inp;
06192     if ( *s != ']' ) {
06193         MesPrint("&unmatched [] in $ in if statement");
06194         return(0);
06195     }
06196     s++;
06197     *inp = s;
06198 }
06199 return(w);
06200 }
06201
06202 /*
06203 #] GetIfDollarFactor :
06204 #[ GetDoParam :
06205 */
06206
06207 UBYTE *GetDoParam(UBYTE *inp, WORD **wp, int par)
06208 {
06209     LONG x;
06210     WORD number;
06211     UBYTE *name, c;
06212     if ( FG.cTable[*inp] == 1 ) {
06213         x = 0;
06214         while ( *inp >= '0' && *inp <= '9' ) {
06215             x = 10*x + *inp++ - '0';
06216             if ( x > MAXPOSITIVE ) {
06217                 if ( par == -1 ) {
06218                     MesPrint("&Value in dollar factor too large");
06219                 }
06220                 else {
06221                     MesPrint("&Value in do loop boundaries too large");
06222                 }
06223                 while ( FG.cTable[*inp] == 1 ) inp++;
06224                 return(0);
06225             }
06226         }
06227         if ( par > 0 ) {
06228             *(*wp)++ = SNUMBER;
06229             *(*wp)++ = (WORD)x;
06230         }
06231         else {
06232             *(*wp)++ = DOLLAREXPR2;
06233             *(*wp)++ = -((WORD)x)-1;
06234         }
06235         return(inp);
06236     }
06237     if ( *inp != '$' ) {
06238         return(0);
06239     }
06240     inp++; name = inp;
06241     while ( FG.cTable[*inp] < 2 ) inp++;
06242     c = *inp; *inp = 0;
06243     if ( GetName(AC.dollarnames,name,&number,NOAUTO) == NAMENOTFOUND ) {
06244         if ( par == -1 ) {
06245             MesPrint("&dollar in print statement should have been defined previously");
06246         }
06247         else {

```

```

06248         MesPrint("&dollar in do loop boundaries should have been defined previously");
06249     }
06250     return(0);
06251 }
06252 *inp = c;
06253 if ( par > 0 ) {
06254     *(*wp)++ = DOLLAREXPRESSION;
06255     *(*wp)++ = number;
06256 }
06257 else {
06258     *(*wp)++ = DOLLAREXPR2;
06259     *(*wp)++ = number;
06260 }
06261 if ( c == '[' ) {
06262     inp++;
06263     inp = GetDoParam(inp,wp,0);
06264     if ( inp == 0 ) return(0);
06265     if ( *inp != ']' ) {
06266         if ( par == -1 ) {
06267             MesPrint("&unmatched [] in $ in print statement");
06268         }
06269         else {
06270             MesPrint("&unmatched [] in do loop boundaries");
06271         }
06272         return(0);
06273     }
06274     inp++;
06275 }
06276 return(inp);
06277 }
06278
06279 /*
06280 #] GetDoParam :
06281 #[ CoDo :
06282 */
06283
06284 int CoDo(UBYTE *inp)
06285 {
06286     GETIDENTITY
06287     CBUF *C = cbuf+AC.cbunum;
06288     WORD *w, numparam;
06289     int error = 0, i;
06290     UBYTE *name, c;
06291     if ( AC.doloopstack == 0 ) {
06292         AC.doloopstacksize = 20;
06293         AC.doloopstack = (WORD *)Malloc1(AC.doloopstacksize*2*sizeof(WORD),"doloop stack");
06294         AC.doloopnest = AC.doloopstack + AC.doloopstacksize;
06295     }
06296     if ( AC.dolooplevel >= AC.doloopstacksize ) {
06297         WORD *newstack, *newnest, newsize;
06298         newsize = AC.doloopstacksize * 2;
06299         newstack = (WORD *)Malloc1(newsize*2*sizeof(WORD),"doloop stack");
06300         newnest = newstack + newsize;
06301         for ( i = 0; i < newsize; i++ ) {
06302             newstack[i] = AC.doloopstack[i];
06303             newnest[i] = AC.doloopnest[i];
06304         }
06305         M_free(AC.doloopstack,"doloop stack");
06306         AC.doloopstack = newstack;
06307         AC.doloopnest = newnest;
06308         AC.doloopstacksize = newsize;
06309     }
06310     AC.doloopnest[AC.dolooplevel] = NestingChecksum();
06311
06312     w = AT.WorkPointer;
06313     *w++ = TYPEDOLOOP;
06314     w++; /* Space for the length of the statement */
06315
06316     /* Now the $loopvariable
06317     */
06318     while ( *inp == ',' ) inp++;
06319     if ( *inp != '$' ) {
06320         error = 1;
06321         MesPrint("&do loop parameter should be a dollar variable");
06322     }
06323     else {
06324         inp++;
06325         name = inp;
06326         if ( FG.cTable[*inp] != 0 ) {
06327             error = 1;
06328             MesPrint("&illegal name for do loop parameter");
06329         }
06330         while ( FG.cTable[*inp] < 2 ) inp++;
06331         c = *inp; *inp = 0;
06332         if ( GetName(AC.dollarnames,name,&numparam,NOAUTO) == NAMENOTFOUND ) {
06333             numparam = AddDollar(name,DOLUNDEFINED,0,0);
06334         }
06335     }

```

```

06335         *w++ = numparam;
06336         *inp = c;
06337         AddPotModdollar(numparam);
06338     }
06339     w++; /* space for the level of the enddo statement */
06340     while ( *inp == ',' ) inp++;
06341     if ( *inp != '=' ) goto IllSyntax;
06342     inp++;
06343     while ( *inp == ',' ) inp++;
06344 /*
06345     The start value
06346 */
06347     inp = GetDoParam(inp,&w,1);
06348     if ( inp == 0 || *inp != ',' ) goto IllSyntax;
06349     while ( *inp == ',' ) inp++;
06350 /*
06351     The end value
06352 */
06353     inp = GetDoParam(inp,&w,1);
06354     if ( inp == 0 || ( *inp != 0 && *inp != ',' ) ) goto IllSyntax;
06355 /*
06356     The increment value
06357 */
06358     if ( *inp != ',' ) {
06359         if ( *inp == 0 ) { *w++ = SNUMBER; *w++ = 1; }
06360         else goto IllSyntax;
06361     }
06362     else {
06363         while ( *inp == ',' ) inp++;
06364         inp = GetDoParam(inp,&w,1);
06365     }
06366     if ( inp == 0 || *inp != 0 ) goto IllSyntax;
06367     *w = 0;
06368     AT.WorkPointer[1] = w - AT.WorkPointer;
06369 /*
06370     Put away and set information for placing enddo information.
06371 */
06372     AddNtoL(AT.WorkPointer[1],AT.WorkPointer);
06373     AC.doloopstack[AC.dolooplevel++] = C->numlhs;
06374
06375     return(error);
06376
06377 IllSyntax:
06378     MesPrint("&Illegal syntax for do statement");
06379     return(1);
06380 }
06381
06382 /*
06383 #] CoDo :
06384 #[ CoEndDo :
06385 */
06386
06387 int CoEndDo(UBYTE *inp)
06388 {
06389     CBUF *C = cbuf+AC.cbufnum;
06390     WORD scratch[3];
06391     while ( *inp == ',' ) inp++;
06392     if ( *inp ) {
06393         MesPrint("&Illegal syntax for EndDo statement");
06394         return(1);
06395     }
06396     if ( AC.dolooplevel <= 0 ) {
06397         MesPrint("&EndDo without corresponding Do statement");
06398         return(1);
06399     }
06400     AC.dolooplevel--;
06401     scratch[0] = TYPEENDDOLOOP;
06402     scratch[1] = 3;
06403     scratch[2] = AC.doloopstack[AC.dolooplevel];
06404     AddNtoL(3,scratch);
06405     cbuf[AC.cbufnum].lhs[AC.doloopstack[AC.dolooplevel]][3] = C->numlhs;
06406     if ( AC.doloopnest[AC.dolooplevel] != NestingChecksum() ) {
06407         MesNesting();
06408         return(1);
06409     }
06410     return(0);
06411 }
06412
06413 /*
06414 #] CoEndDo :
06415 #[ CoFactDollar :
06416 */
06417
06418 int CoFactDollar(UBYTE *inp)
06419 {
06420     WORD numdollar;
06421     if ( *inp == '$' ) {

```

```

06422         if ( GetName(AC.dollarnames,inp+1,&numdollar,NOAUTO) != CDOLLAR ) {
06423             MesPrint("%s is undefined",inp);
06424             numdollar = AddDollar(inp+1,DOLINDEX,&one,1);
06425             return(1);
06426         }
06427         inp = SkipAName(inp+1);
06428         if ( *inp != 0 ) {
06429             MesPrint("&FactDollar should have a single $variable for its argument");
06430             return(1);
06431         }
06432         AddPotModdollar(numdollar);
06433     }
06434     else {
06435         MesPrint("%s is not a $-variable",inp);
06436         return(1);
06437     }
06438     Add3Com(TYPEFACTOR,numdollar);
06439     return(0);
06440 }
06441
06442 /*
06443 #] CoFactDollar :
06444 #[ CoFactorize :
06445 */
06446
06447 int CoFactorize(UBYTE *s) { return(DoFactorize(s,1)); }
06448
06449 /*
06450 #] CoFactorize :
06451 #[ CoNFactorize :
06452 */
06453
06454 int CoNFactorize(UBYTE *s) { return(DoFactorize(s,0)); }
06455
06456 /*
06457 #] CoNFactorize :
06458 #[ CoUnFactorize :
06459 */
06460
06461 int CoUnFactorize(UBYTE *s) { return(DoFactorize(s,3)); }
06462
06463 /*
06464 #] CoUnFactorize :
06465 #[ CoNUnFactorize :
06466 */
06467
06468 int CoNUnFactorize(UBYTE *s) { return(DoFactorize(s,2)); }
06469
06470 /*
06471 #] CoNUnFactorize :
06472 #[ DoFactorize :
06473 */
06474
06475 int DoFactorize(UBYTE *s,int par)
06476 {
06477     EXPRESSIONS e;
06478     WORD i;
06479     WORD number;
06480     UBYTE *t, c;
06481     int error = 0, keepzeroflag = 0;
06482     if ( *s == '(' ) {
06483         s++;
06484         while ( *s != ')' && *s ) {
06485             if ( FG.cTable[*s] == 0 ) {
06486                 t = s; while ( FG.cTable[*s] == 0 ) s++;
06487                 c = *s; *s = 0;
06488                 if ( StrICmp((UBYTE *)"keepzero",t) == 0 ) {
06489                     keepzeroflag = 1;
06490                 }
06491                 else {
06492                     MesPrint("&Illegal option in [N][Un]Factorize statement: %s",t);
06493                     error = 1;
06494                 }
06495                 *s = c;
06496             }
06497             while ( *s == ',' ) s++;
06498             if ( *s && *s != ')' && FG.cTable[*s] != 0 ) {
06499                 MesPrint("&Illegal character in option field of [N][Un]Factorize statement");
06500                 error = 1;
06501                 return(error);
06502             }
06503         }
06504         if ( *s ) s++;
06505         while ( *s == ',' || *s == ' ' ) s++;
06506     }
06507     if ( *s == 0 ) {
06508         for ( i = NumExpressions-1; i >= 0; i-- ) {

```

```

06509         e = Expressions+i;
06510         if ( e->replace >= 0 ) {
06511             e = Expressions + e->replace;
06512         }
06513         if ( e->status == LOCALEXPRESSION || e->status == GLOBALEXPRESSION
06514             || e->status == UNHIDELEXPRESSION || e->status == UNHIDEGEXPRESSION
06515             || e->status == INTOHIDELEXPRESSION || e->status == INTOHIDEGEXPRESSION
06516         ) {
06517             switch ( par ) {
06518                 case 0:
06519                     e->vflags &= ~TOBEFACTORED;
06520                     break;
06521                 case 1:
06522                     e->vflags |= TOBEFACTORED;
06523                     e->vflags &= ~TOBEUNFACTORED;
06524                     break;
06525                 case 2:
06526                     e->vflags &= ~TOBEUNFACTORED;
06527                     break;
06528                 case 3:
06529                     e->vflags |= TOBEUNFACTORED;
06530                     e->vflags &= ~TOBEFACTORED;
06531                     break;
06532             }
06533         }
06534         if ( ( e->vflags & TOBEFACTORED ) != 0 ) {
06535             if ( keepzeroflag ) e->vflags |= KEEPZERO;
06536             else e->vflags &= ~KEEPZERO;
06537         }
06538         else e->vflags &= ~KEEPZERO;
06539     }
06540 }
06541 else {
06542     for(;;) { /* Look for a (comma separated) list of variables */
06543         while ( *s == ',' ) s++;
06544         if ( *s == 0 ) break;
06545         if ( *s == '[' || FG.cTable[*s] == 0 ) {
06546             t = s;
06547             if ( ( s = SkipAName(s) ) == 0 ) {
06548                 MesPrint("&Improper name for an expression: '%s'",t);
06549                 return(1);
06550             }
06551             c = *s; *s = 0;
06552             if ( GetName(AC.exprnames,t,&number,NOAUTO) == CEXPRESSION ) {
06553                 e = Expressions+number;
06554                 if ( e->replace >= 0 ) {
06555                     e = Expressions + e->replace;
06556                 }
06557                 if ( e->status == LOCALEXPRESSION || e->status == GLOBALEXPRESSION
06558                     || e->status == UNHIDELEXPRESSION || e->status == UNHIDEGEXPRESSION
06559                     || e->status == INTOHIDELEXPRESSION || e->status == INTOHIDEGEXPRESSION
06560                 ) {
06561                     switch ( par ) {
06562                         case 0:
06563                             e->vflags &= ~TOBEFACTORED;
06564                             break;
06565                         case 1:
06566                             e->vflags |= TOBEFACTORED;
06567                             e->vflags &= ~TOBEUNFACTORED;
06568                             break;
06569                         case 2:
06570                             e->vflags &= ~TOBEUNFACTORED;
06571                             break;
06572                         case 3:
06573                             e->vflags |= TOBEUNFACTORED;
06574                             e->vflags &= ~TOBEFACTORED;
06575                             break;
06576                     }
06577                 }
06578                 if ( ( e->vflags & TOBEFACTORED ) != 0 ) {
06579                     if ( keepzeroflag ) e->vflags |= KEEPZERO;
06580                     else e->vflags &= ~KEEPZERO;
06581                 }
06582                 else e->vflags &= ~KEEPZERO;
06583             }
06584             else if ( GetName(AC.varnames,t,&number,NOAUTO) != NAMENOTFOUND ) {
06585                 MesPrint("&%s is not an expression",t);
06586                 error = 1;
06587             }
06588             *s = c;
06589         }
06590     }
06591     else {
06592         MesPrint("&Illegal object in (N)Factorize statement");
06593         error = 1;
06594         while ( *s && *s != ',' ) s++;
06595         if ( *s == 0 ) break;
06596     }

```

```

06596     }
06597
06598     }
06599     return(error);
06600 }
06601
06602 /*
06603  #] DoFactorize :
06604  #[ CoOptimizeOption :
06605
06606 */
06607
06608 int CoOptimizeOption(UBYTE *s)
06609 {
06610     UBYTE *name, *t1, *t2, c1, c2, *value, *u;
06611     int error = 0, x;
06612     double d;
06613     while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
06614     while ( *s ) {
06615         name = s; while ( FG.cTable[*s] == 0 ) s++;
06616         t1 = s; c1 = *t1;
06617         while ( *s == ' ' || *s == '\t' ) s++;
06618         if ( *s != '=' ) {
06619             correctuse:
06620                 MesPrint("&Correct use in Format,Optimize statement is Optionname=value");
06621                 error = 1;
06622                 while ( *s == ' ' || *s == ',' || *s == '\t' || *s == '=' ) s++;
06623                 *t1 = c1;
06624                 continue;
06625         }
06626         *t1 = 0;
06627         s++;
06628         while ( *s == ' ' || *s == '\t' ) s++;
06629         if ( *s == 0 ) goto correctuse;
06630         value = s;
06631         while ( FG.cTable[*s] <= 1 || *s== '.' || *s=='*' || *s == '(' || *s == ')' ) {
06632             if ( *s == '(' ) { SKIPBRA4(s) }
06633             s++;
06634         }
06635         t2 = s; c2 = *t2;
06636         while ( *s == ' ' || *s == '\t' ) s++;
06637         if ( *s && *s != ',' ) goto correctuse;
06638         if ( *s ) {
06639             s++;
06640             while ( *s == ' ' || *s == '\t' ) s++;
06641         }
06642         *t2 = 0;
06643     /*
06644     Now we have name=value with name and value zero terminated strings.
06645     */
06646     if ( StrICmp(name,(UBYTE *)"horner") == 0 ) {
06647         if ( StrICmp(value,(UBYTE *)"occurrence") == 0 ) {
06648             AO.Optimize.horner = O_OCCURRENCE;
06649         }
06650         else if ( StrICmp(value,(UBYTE *)"mcts") == 0 ) {
06651             AO.Optimize.horner = O_MCTS;
06652         }
06653         else if ( StrICmp(value,(UBYTE *)"sa") == 0 ) {
06654             AO.Optimize.horner = O_SIMULATED_ANNEALING;
06655         }
06656         else {
06657             AO.Optimize.horner = -1;
06658             MesPrint("&Unrecognized option value in Format,Optimize statement: %s=%s",name,value);
06659             error = 1;
06660         }
06661     }
06662     else if ( StrICmp(name,(UBYTE *)"hornerdirection") == 0 ) {
06663         if ( StrICmp(value,(UBYTE *)"forward") == 0 ) {
06664             AO.Optimize.hornerdirection = O_FORWARD;
06665         }
06666         else if ( StrICmp(value,(UBYTE *)"backward") == 0 ) {
06667             AO.Optimize.hornerdirection = O_BACKWARD;
06668         }
06669         else if ( StrICmp(value,(UBYTE *)"forwardorbackward") == 0 ) {
06670             AO.Optimize.hornerdirection = O_FORWARDORBACKWARD;
06671         }
06672         else if ( StrICmp(value,(UBYTE *)"forwardandbackward") == 0 ) {
06673             AO.Optimize.hornerdirection = O_FORWARDANDBACKWARD;
06674         }
06675         else {
06676             AO.Optimize.method = -1;
06677             MesPrint("&Unrecognized option value in Format,Optimize statement: %s=%s",name,value);
06678             error = 1;
06679         }
06680     }
06681     else if ( StrICmp(name,(UBYTE *)"method") == 0 ) {
06682         if ( StrICmp(value,(UBYTE *)"none") == 0 ) {

```

```

06683         AO.Optimize.method = O_NONE;
06684     }
06685     else if ( StrICmp(value, (UBYTE *)"cse") == 0 ) {
06686         AO.Optimize.method = O_CSE;
06687     }
06688     else if ( StrICmp(value, (UBYTE *)"csegreedy") == 0 ) {
06689         AO.Optimize.method = O_CSEGREEDY;
06690     }
06691     else if ( StrICmp(value, (UBYTE *)"greedy") == 0 ) {
06692         AO.Optimize.method = O_GREEDY;
06693     }
06694     else {
06695         AO.Optimize.method = -1;
06696         MesPrint("&Unrecognized option value in Format,Optimize statement: %s=%s",name,value);
06697         error = 1;
06698     }
06699 }
06700 else if ( StrICmp(name, (UBYTE *)"timelimit") == 0 ) {
06701     x = 0;
06702     u = value; while ( *u >= '0' && *u <= '9' ) x = 10*x + *u++ - '0';
06703     if ( *u != 0 ) {
06704         MesPrint("&Option TimeLimit in Format,Optimize statement should be a positive number:
06705 %s",value);
06706         AO.Optimize.mctstimelimit = 0;
06707         AO.Optimize.greedytimelimit = 0;
06708         error = 1;
06709     }
06710     else {
06711         AO.Optimize.mctstimelimit = x/2;
06712         AO.Optimize.greedytimelimit = x/2;
06713     }
06714 }
06715 else if ( StrICmp(name, (UBYTE *)"mctstimelimit") == 0 ) {
06716     x = 0;
06717     u = value; while ( *u >= '0' && *u <= '9' ) x = 10*x + *u++ - '0';
06718     if ( *u != 0 ) {
06719         MesPrint("&Option MCTSTimeLimit in Format,Optimize statement should be a positive
06720 number: %s",value);
06721         AO.Optimize.mctstimelimit = 0;
06722         error = 1;
06723     }
06724     else {
06725         AO.Optimize.mctstimelimit = x;
06726     }
06727 }
06728 else if ( StrICmp(name, (UBYTE *)"mctsnmexpand") == 0 ) {
06729     int y;
06730     x = 0;
06731     u = value; while ( *u >= '0' && *u <= '9' ) x = 10*x + *u++ - '0';
06732     if ( *u == '*' || *u == 'x' || *u == 'X' ) {
06733         u++; y = x;
06734         x = 0;
06735         while ( *u >= '0' && *u <= '9' ) x = 10*x + *u++ - '0';
06736     }
06737     else { y = 1; }
06738     if ( *u != 0 ) {
06739         MesPrint("&Option MCTSNmExpand in Format,Optimize statement should be a positive
06740 number: %s",value);
06741         AO.Optimize.mctsnmexpand= 0;
06742         AO.Optimize.mctsnmrepeat= 1;
06743         error = 1;
06744     }
06745     else {
06746         AO.Optimize.mctsnmexpand= x;
06747         AO.Optimize.mctsnmrepeat= y;
06748     }
06749 }
06750 else if ( StrICmp(name, (UBYTE *)"mctsnmrepeat") == 0 ) {
06751     x = 0;
06752     u = value; while ( *u >= '0' && *u <= '9' ) x = 10*x + *u++ - '0';
06753     if ( *u != 0 ) {
06754         MesPrint("&Option MCTSNmExpand in Format,Optimize statement should be a positive
06755 number: %s",value);
06756         AO.Optimize.mctsnmrepeat= 1;
06757         error = 1;
06758     }
06759     else {
06760         AO.Optimize.mctsnmrepeat= x;
06761     }
06762 }
06763 else if ( StrICmp(name, (UBYTE *)"mctsnmkeep") == 0 ) {
06764     x = 0;
06765     u = value; while ( *u >= '0' && *u <= '9' ) x = 10*x + *u++ - '0';
06766     if ( *u != 0 ) {
06767         MesPrint("&Option MCTSNmKeep in Format,Optimize statement should be a positive
06768 number: %s",value);
06769         AO.Optimize.mctsnmkeep= 0;

```



```

06765         error = 1;
06766     }
06767     else {
06768         AO.Optimize.mctsnumkeep= x;
06769     }
06770 }
06771 else if ( StrICmp(name, (UBYTE *)"mctsconstant") == 0 ) {
06772     d = 0;
06773     if ( sscanf ((char*)value, "%lf", &d) != 1 ) {
06774         MesPrint("&Option MCTSConstant in Format,Optimize statement should be a positive
number: %s",value);
06775         AO.Optimize.mctsconstant.fval = 0;
06776         error = 1;
06777     }
06778     else {
06779         AO.Optimize.mctsconstant.fval = d;
06780     }
06781 }
06782 else if ( StrICmp(name, (UBYTE *)"greedytimelimit") == 0 ) {
06783     x = 0;
06784     u = value; while ( *u >= '0' && *u <= '9' ) x = 10*x + *u++ - '0';
06785     if ( *u != 0 ) {
06786         MesPrint("&Option GreedyTimeLimit in Format,Optimize statement should be a positive
number: %s",value);
06787         AO.Optimize.greedytimelimit = 0;
06788         error = 1;
06789     }
06790     else {
06791         AO.Optimize.greedytimelimit = x;
06792     }
06793 }
06794 else if ( StrICmp(name, (UBYTE *)"greedyminnum") == 0 ) {
06795     x = 0;
06796     u = value; while ( *u >= '0' && *u <= '9' ) x = 10*x + *u++ - '0';
06797     if ( *u != 0 ) {
06798         MesPrint("&Option GreedyMinNum in Format,Optimize statement should be a positive
number: %s",value);
06799         AO.Optimize.greedyminnum= 0;
06800         error = 1;
06801     }
06802     else {
06803         AO.Optimize.greedyminnum= x;
06804     }
06805 }
06806 else if ( StrICmp(name, (UBYTE *)"greedymaxperc") == 0 ) {
06807     x = 0;
06808     u = value; while ( *u >= '0' && *u <= '9' ) x = 10*x + *u++ - '0';
06809     if ( *u != 0 ) {
06810         MesPrint("&Option GreedyMaxPerc in Format,Optimize statement should be a positive
number: %s",value);
06811         AO.Optimize.greedymaxperc= 0;
06812         error = 1;
06813     }
06814     else {
06815         AO.Optimize.greedymaxperc= x;
06816     }
06817 }
06818 else if ( StrICmp(name, (UBYTE *)"stats") == 0 ) {
06819     if ( StrICmp(value, (UBYTE *)"on") == 0 ) {
06820         AO.Optimize.printstats = 1;
06821     }
06822     else if ( StrICmp(value, (UBYTE *)"off") == 0 ) {
06823         AO.Optimize.printstats = 0;
06824     }
06825     else {
06826         AO.Optimize.printstats = 0;
06827         MesPrint("&Unrecognized option value in Format,Optimize statement: %s=%s",name,value);
06828         error = 1;
06829     }
06830 }
06831 else if ( StrICmp(name, (UBYTE *)"printscheme") == 0 ) {
06832     if ( StrICmp(value, (UBYTE *)"on") == 0 ) {
06833         AO.Optimize.schemeflags |= 1;
06834     }
06835     else if ( StrICmp(value, (UBYTE *)"off") == 0 ) {
06836         AO.Optimize.schemeflags &= ~1;
06837     }
06838     else {
06839         AO.Optimize.schemeflags &= ~1;
06840         MesPrint("&Unrecognized option value in Format,Optimize statement: %s=%s",name,value);
06841         error = 1;
06842     }
06843 }
06844 else if ( StrICmp(name, (UBYTE *)"debugflag") == 0 ) {
06845     /*
06846         This option is for debugging purposes only. Not in the manual!
06847         0x1: Print statements in reverse order.

```

```

06848         0x2: Print the scheme of the variables.
06849  */
06850         x = 0;
06851         u = value;
06852         if ( FG.cTable[*u] == 1 ) {
06853             while ( *u >= '0' && *u <= '9' ) x = 10*x + *u++ - '0';
06854             if ( *u != 0 ) {
06855                 MesPrint("&Numerical value for DebugFlag in Format,Optimize statement should be a
nonnegative number: %s",value);
06856                 AO.Optimize.debugflags = 0;
06857                 error = 1;
06858             }
06859             else {
06860                 AO.Optimize.debugflags = x;
06861             }
06862         }
06863         else if ( StrICmp(value,(UBYTE *)"on") == 0 ) {
06864             AO.Optimize.debugflags = 1;
06865         }
06866         else if ( StrICmp(value,(UBYTE *)"off") == 0 ) {
06867             AO.Optimize.debugflags = 0;
06868         }
06869         else {
06870             AO.Optimize.debugflags = 0;
06871             MesPrint("&Unrecognized option value in Format,Optimize statement: %s=%s",name,value);
06872             error = 1;
06873         }
06874     }
06875     else if ( StrICmp(name,(UBYTE *)"scheme") == 0 ) {
06876         UBYTE *ss, *s1, c;
06877         WORD type, numsym;
06878         AO.schemenum = 0;
06879         u = value;
06880         if ( *u != '(' ) {
06881             noscheme:
06882                 MesPrint("&Option Scheme in Format,Optimize statement should be an array of names or
integers between (): %s",value);
06883                 error = 1;
06884                 break;
06885         }
06886         u++; ss = u;
06887         while ( *ss == ' ' || *ss == '\t' || *ss == ',' ) ss++;
06888         if ( FG.cTable[*ss] == 0 || *ss == '$' || *ss == '[' ) { /* Name */
06889             s1 = u; SKIPBRA3(s1)
06890             if ( *s1 != ')' ) goto noscheme;
06891             while ( ss < s1 ) { if ( *ss++ == ',' ) AO.schemenum++; }
06892             *ss++ = 0; while ( *ss == ' ' ) ss++;
06893             if ( *ss != 0 ) goto noscheme;
06894             ss = u;
06895             if ( AO.schemenum < 1 ) {
06896                 MesPrint("&Option Scheme in Format,Optimize statement should have at least one
name or number between ()");
06897                 error = 1;
06898                 break;
06899             }
06900             if ( AO.inscheme ) M_free(AO.inscheme,"Horner input scheme");
06901             AO.inscheme = (WORD *)Malloc1((AO.schemenum+1)*sizeof(WORD),"Horner input scheme");
06902             while ( *ss == ' ' || *ss == '\t' || *ss == ',' ) ss++;
06903             AO.schemenum = 0;
06904             for(;;) {
06905                 if ( *ss == 0 ) break;
06906                 s1 = ss; ss = SkipAName(s1); c = *ss; *ss = 0;
06907             }
06908             if ( ss[-1] == '_' ) {
06909                 /*
06910                 Now AC.extrasym followed by a number and _
06911                 */
06912                 UBYTE *u1, *u2;
06913                 u1 = s1; u2 = AC.extrasym;
06914                 while ( *u1 == *u2 ) { u1++; u2++; }
06915                 if ( *u2 == 0 ) { /* Good start */
06916                     numsym = 0;
06917                     while ( *u1 >= '0' && *u1 <= '9' ) numsym = 10*numsym + *u1++ - '0';
06918                     if ( u1 != ss-1 || numsym == 0 || AC.extrasymbols != 0 ) {
06919                         MesPrint("&Improper use of extra symbol in scheme format option");
06920                         goto noscheme;
06921                     }
06922                     numsym = MAXVARIABLES-numsym;
06923                     ss++;
06924                     goto GotTheNumber;
06925                 }
06926             }
06927             else if ( *s1 == '$' ) {
06928                 GETIDENTITY
06929                 int numdollar;
06930                 if ( ( numdollar = GetDollar(s1+1) ) < 0 ) {
06931                     MesPrint("&Undefined variable %s",s1);

```

```

06932         error = 5;
06933     }
06934     else if ( ( numsym = DolToSymbol(BHEAD numdollar) ) < 0 ) {
06935         MesPrint("&$$s does not evaluate to a symbol",s1);
06936         error = 5;
06937     }
06938     *ss = c;
06939     goto GotTheNumber;
06940 }
06941 else if ( c == '(' ) {
06942     if ( StrCmp(s1,AC.extrasym) == 0 ) {
06943         if ( (AC.extrasymbols&1) != 1 ) {
06944             MesPrint("&Improper use of extra symbol in scheme format option");
06945             goto noscheme;
06946         }
06947         *ss++ = c;
06948         numsym = 0;
06949         while ( *ss >= '0' && *ss <= '9' ) numsym = 10*numsym + *ss++ - '0';
06950         if ( *ss != ')' ) {
06951             MesPrint("&Extra symbol should have a number for its argument.");
06952             goto noscheme;
06953         }
06954         numsym = MAXVARIABLES-numsym;
06955         ss++;
06956         goto GotTheNumber;
06957     }
06958 }
06959 type = GetName(AC.varnames,s1,&numsym,WITHAUTO);
06960 if ( ( type != CSYMBOL ) && type != CDUBIOUS ) {
06961     MesPrint("&$$s is not a symbol",s1);
06962     error = 4;
06963     if ( type < 0 ) numsym = AddSymbol(s1,-MAXPOWER,MAXPOWER,0,0);
06964 }
06965 *ss = c;
06966 GotTheNumber:
06967     AO.inscheme[AO.schemenum++] = numsym;
06968     while ( *ss == ' ' || *ss == '\t' || *ss == ',' ) ss++;
06969 }
06970 }
06971 }
06972 else if ( StrICmp(name,(UBYTE *)"mctsdecaymode") == 0 ) {
06973     x = 0;
06974     u = value;
06975     if ( FG.cTable[*u] == 1 ) {
06976         while ( *u >= '0' && *u <= '9' ) x = 10*x + *u++ - '0';
06977         if ( *u != 0 ) {
06978             MesPrint("&Option MCTSDecayMode in Format,Optimize statement should be a
nonnegative integer: %s",value);
06979             AO.Optimize.mctsdecaymode = 0;
06980             error = 1;
06981         }
06982         else {
06983             AO.Optimize.mctsdecaymode = x;
06984         }
06985     }
06986     else {
06987         AO.Optimize.mctsdecaymode = 0;
06988         MesPrint("&Unrecognized option value in Format,Optimize statement: %s=%s",name,value);
06989         error = 1;
06990     }
06991 }
06992 else if ( StrICmp(name,(UBYTE *)"saiter") == 0 ) {
06993     x = 0;
06994     u = value; while ( *u >= '0' && *u <= '9' ) x = 10*x + *u++ - '0';
06995     if ( *u != 0 ) {
06996         MesPrint("&Option SAIter in Format,Optimize statement should be a positive integer:
%s",value);
06997         AO.Optimize.saIter = 0;
06998         error = 1;
06999     }
07000     else {
07001         AO.Optimize.saIter = x;
07002     }
07003 }
07004 else if ( StrICmp(name,(UBYTE *)"samaxt") == 0 ) {
07005     d = 0;
07006     if ( sscanf ((char*)value, "%lf", &d) != 1 ) {
07007         MesPrint("&Option SAMaxT in Format,Optimize statement should be a positive number:
%s",value);
07008         AO.Optimize.saMaxT.fval = 0;
07009         error = 1;
07010     }
07011     else {
07012         AO.Optimize.saMaxT.fval = d;
07013     }
07014 }
07015 else if ( StrICmp(name,(UBYTE *)"samint") == 0 ) {

```

```

07016         d = 0;
07017         if ( sscanf ((char*)value, "%lf", &d) != 1 ) {
07018             MesPrint("&Option SAmiT in Format,Optimize statement should be a positive number:
%s",value);
07019             AO.Optimize.saMinT.fval = 0;
07020             error = 1;
07021         }
07022         else {
07023             AO.Optimize.saMinT.fval = d;
07024         }
07025     }
07026     else {
07027         MesPrint("&Unrecognized option name in Format,Optimize statement: %s",name);
07028         error = 1;
07029     }
07030     *t1 = c1; *t2 = c2;
07031 }
07032 return(error);
07033 }
07034
07035 /*
07036 #] CoOptimizeOption :
07037 #[ DoPutInside :
07038
07039 Syntax:
07040 PutIn[side],functionname[,brackets] -> par = 1
07041 AntiPutIn[side],functionname,antibrackets -> par = -1
07042 */
07043
07044 int CoPutInside(UBYTE *inp) { return(DoPutInside(inp,1)); }
07045 int CoAntiPutInside(UBYTE *inp) { return(DoPutInside(inp,-1)); }
07046
07047 int DoPutInside(UBYTE *inp, int par)
07048 {
07049     GETIDENTITY
07050     UBYTE *p, c;
07051     WORD *to, type, c1,c2,funnum, *WorkSave;
07052     int error = 0;
07053     while ( *inp == ' ' || *inp == '\t' || *inp == ',' ) inp++;
07054 /*
07055     First we need the name of a function. (Not a tensor or table!)
07056 */
07057     p = SkipAName(inp);
07058     if ( p == 0 ) return(1);
07059     c = *p; *p = 0;
07060     type = GetName(AC.varnames,inp,&funnum,WITHAUTO);
07061     if ( type != CFUNCTION || functions[funnum].tabl != 0 || functions[funnum].spec ) {
07062         MesPrint("&PutInside/AntiPutInside expects a regular function for its first argument");
07063         MesPrint("&Argument is %s",inp);
07064         error = 1;
07065     }
07066     funnum += FUNCTION;
07067     *p = c;
07068     inp = p;
07069     while ( *inp == ' ' || *inp == '\t' || *inp == ',' ) inp++;
07070     if ( *inp == 0 ) {
07071         if ( par == 1 ) {
07072             WORD tocompiler[4];
07073             tocompiler[0] = TYPEPUTINSIDE;
07074             tocompiler[1] = 4;
07075             tocompiler[2] = 0;
07076             tocompiler[3] = funnum;
07077             AddNtoL(4,tocompiler);
07078         }
07079         else {
07080             MesPrint("&AntiPutInside needs inside information.");
07081             error = 1;
07082         }
07083         return(error);
07084     }
07085     WorkSave = to = AT.WorkPointer;
07086     *to++ = TYPEPUTINSIDE;
07087     *to++ = 4;
07088     *to++ = par;
07089     *to++ = funnum;
07090     to++;
07091     while ( *inp ) {
07092         while ( *inp == ' ' || *inp == '\t' || *inp == ',' ) inp++;
07093         if ( *inp == 0 ) break;
07094         p = SkipAName(inp);
07095         if ( p == 0 ) { error = 1; break; }
07096         c = *p; *p = 0;
07097         type = GetName(AC.varnames,inp,&c1,WITHAUTO);
07098         if ( c == '.' ) {
07099             if ( type == CVECTOR || type == CDUBIOUS ) {
07100                 *p++ = c;
07101                 inp = p;

```

```

07102         p = SkipAName(inp);
07103         if ( p == 0 ) return(1);
07104         c = *p; *p = 0;
07105         type = GetName(AC.varnames,inp,&c2,WITHAUTO);
07106         if ( type != CVECTOR && type != CDUBIOUS ) {
07107             MesPrint("&Not a vector in dotproduct in PutInside/AntiPutInside statement:
%s",inp);
07108             error = 1;
07109         }
07110         else type = CDOTPRODUCT;
07111     }
07112     else {
07113         MesPrint("&Illegal use of . after %s in PutInside/AntiPutInside statement",inp);
07114         error = 1;
07115         *p = c; inp = p;
07116         continue;
07117     }
07118 }
07119 switch ( type ) {
07120     case CSYMBOL :
07121         *to++ = SYMBOL; *to++ = 4; *to++ = c1; *to++ = 1; break;
07122     case CVECTOR :
07123         *to++ = INDEX; *to++ = 3; *to++ = AM.OffsetVector + c1; break;
07124     case CFUNCTION :
07125         *to++ = c1+FUNCTION; *to++ = FUNHEAD; *to++ = 0;
07126         FILLFUN3(to)
07127         break;
07128     case CDOTPRODUCT :
07129         *to++ = DOTPRODUCT; *to++ = 5; *to++ = c1 + AM.OffsetVector;
07130         *to++ = c2 + AM.OffsetVector; *to++ = 1; break;
07131     case CDELTA :
07132         *to++ = DELTA; *to++ = 4; *to++ = EMPTYINDEX; *to++ = EMPTYINDEX; break;
07133     default :
07134         MesPrint("&Illegal variable request for %s in PutInside/AntiPutInside statement",inp);
07135         error = 1; break;
07136 }
07137 *p = c;
07138 inp = p;
07139 }
07140 *to++ = 1; *to++ = 1; *to++ = 3;
07141 AT.WorkPointer[1] = to - AT.WorkPointer;
07142 AT.WorkPointer[4] = AT.WorkPointer[1]-4;
07143 AT.WorkPointer = to;
07144 AC.BraceNormalize = 1;
07145 if ( Normalize(BHEAD WorkSave+4) ) { error = 1; }
07146 else {
07147     WorkSave[1] = WorkSave[4]+4;
07148     to = WorkSave + WorkSave[1] - 1;
07149     c1 = ABS(*to);
07150     WorkSave[1] -= c1;
07151     WorkSave[4] -= c1;
07152     AddNtoL(WorkSave[1],WorkSave);
07153 }
07154 AC.BraceNormalize = 0;
07155 AT.WorkPointer = WorkSave;
07156 return(error);
07157 }
07158
07159 /*
07160 #] DoPutInside :
07161 #[ CoSwitch :
07162
07163 Syntax: Switch $var;
07164 Be careful with illegal nestings with repeat, if, while.
07165 */
07166
07167 int CoSwitch(UBYTE *s)
07168 {
07169     WORD numdollar;
07170     SWITCH *sw;
07171     if ( *s == '$' ) {
07172         if ( GetName(AC.dollarnames,s+1,&numdollar,NOAUTO) != CDOLLAR ) {
07173             MesPrint("&%s is undefined in switch statement",s);
07174             numdollar = AddDollar(s+1,DOLINDEX,&one,1);
07175             return(1);
07176         }
07177         s = SkipAName(s+1);
07178         if ( *s != 0 ) {
07179             MesPrint("&Switch should have a single $variable for its argument");
07180             return(1);
07181         }
07182         /* AddPotModdollar(numdollar); */
07183     }
07184     else {
07185         MesPrint("&%s is not a $-variable in switch statement",s);
07186         return(1);
07187     }

```

```

07188 /*
07189     Now create the switch table. We will add to it each time we run
07190     into a new case. It will all be sorted out the moment we run into
07191     the endswitch statement.
07192 */
07193     AC.SwitchLevel++;
07194     if ( AC.SwitchInArray >= AC.MaxSwitch ) DoubleSwitchBuffers();
07195     AC.SwitchHeap[AC.SwitchLevel] = AC.SwitchInArray;
07196     sw = AC.SwitchArray + AC.SwitchInArray;
07197
07198     sw->iflevel = AC.IfLevel;
07199     sw->whilelevel = AC.WhileLevel;
07200     sw->nestingsum = NestingChecksum();
07201
07202     Add4Com(TYPESWITCH,numdollar,AC.SwitchInArray);
07203
07204     AC.SwitchInArray++;
07205     return(0);
07206 }
07207
07208 /*
07209     #] CoSwitch :
07210     #[ CoCase :
07211 */
07212
07213 int CoCase(UBYTE *s)
07214 {
07215     SWITCH *sw = AC.SwitchArray + AC.SwitchHeap[AC.SwitchLevel];
07216     WORD x = 0, sign = 1;
07217     while ( *s == ',' ) s++;
07218     SKIPBLANKS(s);
07219     while ( *s == '-' || *s == '+' ) {
07220         if ( *s == '-' ) sign = -sign;
07221         s++;
07222     }
07223     while ( FG.cTable[*s] == 1 ) { x = 10*x + *s++ - '0'; }
07224     x = sign*x;
07225
07226     if ( sw->iflevel != AC.IfLevel || sw->whilelevel != AC.WhileLevel
07227         || sw->nestingsum != NestingChecksum() ) {
07228         MesPrint("%Illegal nesting of switch/case/default with
07229 if/while/repeat/loop/argument/term/...");
07230         return(-1);
07231     }
07232
07233     /*
07234     Now add a case to the table with the current 'address'.
07235     */
07236     if ( sw->numcases >= sw->tablesize ) {
07237         int i;
07238         SWITCHTABLE *newtable;
07239         WORD newsize;
07240         if ( sw->tablesize == 0 ) newsize = 10;
07241         else newsize = 2*sw->tablesize;
07242         newtable = (SWITCHTABLE *)Malloc1(newsize*sizeof(SWITCHTABLE),"Switch table");
07243         if ( sw->table ) {
07244             for ( i = 0; i < sw->tablesize; i++ ) newtable[i] = sw->table[i];
07245             M_free(sw->table,"Switch table");
07246         }
07247         sw->table = newtable;
07248         sw->tablesize = newsize;
07249     }
07250     if ( sw->numcases == 0 ) { sw->mincase = sw->maxcase = x; }
07251     else if ( x > sw->maxcase ) sw->maxcase = x;
07252     else if ( x < sw->mincase ) sw->mincase = x;
07253     sw->table[sw->numcases].ncase = x;
07254     sw->table[sw->numcases].value = cbuf[AC.cbunum].numlhs;
07255     sw->table[sw->numcases].compbuffer = AC.cbunum;
07256     sw->numcases++;
07257     return(0);
07258 }
07259
07260 /*
07261     #] CoCase :
07262     #[ CoBreak :
07263 */
07264
07265 int CoBreak(UBYTE *s)
07266 {
07267     /*
07268     This involves a 'postponed' jump to the end. This can be done
07269     in a special routine during execution.
07270     That routine should also pop the switch level.
07271     */
07272     SWITCH *sw = AC.SwitchArray + AC.SwitchHeap[AC.SwitchLevel];
07273     if ( sw->iflevel != AC.IfLevel || sw->whilelevel != AC.WhileLevel
07274         || sw->nestingsum != NestingChecksum() ) {
07275         MesPrint("%Illegal nesting of switch/case/default with

```

```

    if/while/repeat/loop/argument/term/...");
07274     return(-1);
07275 }
07276 if ( *s ) {
07277     MesPrint("&No parameters allowed in Break statement");
07278     return(-1);
07279 }
07280 Add3Com(TYPEENDSWITCH,AC.SwitchHeap[AC.SwitchLevel]);
07281 return(0);
07282 }
07283
07284 /*
07285  #] CoBreak :
07286  #[ CoDefault :
07287  */
07288
07289 int CoDefault (UBYTE *s)
07290 {
07291     /*
07292      A bit like case, except that the address gets stored directly in the
07293      SWITCH struct.
07294     */
07295     SWITCH *sw = AC.SwitchArray + AC.SwitchHeap[AC.SwitchLevel];
07296     if ( sw->iflevel != AC.IfLevel || sw->whilelevel != AC.WhileLevel
07297         || sw->nestingsum != NestingChecksum() ) {
07298         MesPrint("&Illegal nesting of switch/case/default with
07299 if/while/repeat/loop/argument/term/...");
07300         return(-1);
07301     }
07302     if ( *s ) {
07303         MesPrint("&No parameters allowed in Default statement");
07304         return(-1);
07305     }
07306     sw->defaultcase.ncase = 0;
07307     sw->defaultcase.value = cbuf[AC.cbunum].numlhs;
07308     sw->defaultcase.compbuffer = AC.cbunum;
07309     return(0);
07310 }
07311 /*
07312  #] CoDefault :
07313  #[ CoEndSwitch :
07314  */
07315
07316 int CoEndSwitch (UBYTE *s)
07317 {
07318     /*
07319      We store this address in the SWITCH struct.
07320      Next we sort the table by ncase.
07321      Then we decide whether the table is DENSE or SPARSE.
07322      If it is dense we change the allocation and spread the cases is necessary.
07323      Finally we pop levels.
07324     */
07325     SWITCH *sw = AC.SwitchArray + AC.SwitchHeap[AC.SwitchLevel];
07326     WORD i;
07327     WORD totcases = sw->maxcase-sw->mincase+1;
07328     while ( *s == ',' ) s++;
07329     SKIPBLANKS(s)
07330     if ( *s ) {
07331         MesPrint("&No parameters allowed in EndSwitch statement");
07332         return(-1);
07333     }
07334     if ( sw->iflevel != AC.IfLevel || sw->whilelevel != AC.WhileLevel
07335         || sw->nestingsum != NestingChecksum() ) {
07336         MesPrint("&Illegal nesting of switch/case/default with
07337 if/while/repeat/loop/argument/term/...");
07338         return(-1);
07339     }
07340     if ( sw->defaultcase.value == 0 ) CoDefault(s);
07341     if ( totcases > sw->numcases*AM.jumpratio ) { /* The factor is experimental */
07342         sw->caseoffset = 0;
07343         sw->typetable = SPARSETABLE;
07344     }
07345     /*
07346      Now we need to sort sw->table
07347     */
07348     SwitchSplitMerge(sw->table,sw->numcases);
07349 }
07350 else { /* DENSE */
07351     SWITCHTABLE *ntable;
07352     sw->caseoffset = sw->mincase;
07353     sw->typetable = DENSETABLE;
07354     ntable = (SWITCHTABLE *)Malloc1(totcases*sizeof(SWITCHTABLE),"Switch table");
07355     for ( i = 0; i < totcases; i++ ) {
07356         ntable[i].ncase = i+sw->caseoffset;
07357         ntable[i].value = sw->defaultcase.value;
07358         ntable[i].compbuffer = sw->defaultcase.compbuffer;
07359     }

```

```

07358         for ( i = 0; i < sw->numcases; i++ ) {
07359             ntable[sw->table[i].ncase-sw->caseoffset] = sw->table[i];
07360         }
07361         M_free(sw->table,"Switch table");
07362         sw->table = ntable;
07363         sw->numcases = totcases;
07364     }
07365     sw->endswitch.ncase = 0;
07366     sw->endswitch.value = cbuf[AC.cbufnum].numlhs;
07367     sw->endswitch.compbuffer = AC.cbufnum;
07368     if ( sw->defaultcase.value == 0 ) {
07369         sw->defaultcase = sw->endswitch;
07370     }
07371     Add3Com(TYPEENDSWITCH,AC.SwitchHeap[AC.SwitchLevel]);
07372     /*
07373     Now we need to pop.
07374     */
07375     AC.SwitchLevel--;
07376     return(0);
07377 }
07378
07379 /*
07380 #] CoEndSwitch :
07381 #[ CoSetUserFlag :
07382 */
07383
07384 int CoSetUserFlag(UBYTE *s)
07385 {
07386     int error = 0;
07387     while ( *s && ( *s == ',' || *s == ' ' || *s == '\t' ) ) s++;
07388     while ( *s && ( FG.cTable[*s] == 1 ) ) {
07389         int x = 0;
07390         while ( *s && ( FG.cTable[*s] == 1 ) ) x = 10*x+(s++ - '0');
07391         if ( x < 1 || x > BITSINWORD ) {
07392             MesPrint("&Flag number %d outside the permitted range 1-&d.",BITSINWORD);
07393             error = 1;
07394         }
07395         else {
07396             Add3Com(TYPESETUSERFLAG,x-1);
07397         }
07398         while ( *s && ( *s == ',' || *s == ' ' || *s == '\t' ) ) s++;
07399     }
07400     if ( *s ) {
07401         MesPrint("&Illegal character in SetUserFlag statement: %s",s);
07402         error = 1;
07403     }
07404     return(error);
07405 }
07406
07407 /*
07408 #] CoSetUserFlag :
07409 #[ CoClearUserFlag :
07410 */
07411
07412 int CoClearUserFlag(UBYTE *s)
07413 {
07414     int error = 0;
07415     while ( *s && ( *s == ',' || *s == ' ' || *s == '\t' ) ) s++;
07416     while ( *s && ( FG.cTable[*s] == 1 ) ) {
07417         int x = 0;
07418         while ( *s && ( FG.cTable[*s] == 1 ) ) x = 10*x+(s++ - '0');
07419         if ( x < 1 || x > BITSINWORD ) {
07420             MesPrint("&Flag number %d outside the permitted range 1-&d.",BITSINWORD);
07421             error = 1;
07422         }
07423         else {
07424             Add3Com(TYPECLEARUSERFLAG,x);
07425         }
07426         while ( *s && ( *s == ',' || *s == ' ' || *s == '\t' ) ) s++;
07427     }
07428     if ( *s ) {
07429         MesPrint("&Illegal character in SetUserFlag statement: %s",s);
07430         error = 1;
07431     }
07432     return(error);
07433 }
07434
07435 /*
07436 #] CoClearUserFlag :
07437 #[ CoCreateAllLoops :
07438
07439 Syntax:
07440     CoCreateAllLoops,in-function,out-function,type-argument,ifnolooop;
07441 Types allowed:
07442     vector, index, symbol, snumber
07443 ifnolooop:
07444     ifnolooop=0 or ifnolooop=1

```



```

07445     in-function can be a tensor. In that case type can be only vector or index.
07446     out-function can be a tensor. In that case type can be only vector or index.
07447 */
07448
07449 int CoCreateAllLoops(UBYTE *s)
07450 {
07451     GETIDENTITY
07452     UBYTE *inname, *outname, *stype, c;
07453     WORD infun, outfun, x, type, tensorflag, typenum;
07454     WORD *WorkSave, *to;
07455     while ( *s == ',' || *s == ' ' ) s++;
07456     inname = s; s = SkipAName(s);
07457     c = *s; *s = 0;
07458     if ( ( ( type = GetName(AC.varnames,inname,&infun,WITHAUTO) ) != CFUNCTION )
07459     || ( ( functions[infun].spec != 0 ) && ( functions[infun].spec != TENSORFUNCTION ) ) ) {
07460         MesPrint("%s should be a regular function or a tensor.",inname);
07461         if ( type < 0 ) {
07462             if ( GetName(AC.exprnames,s,&infun,NOAUTO) == NAMENOTFOUND )
07463                 AddFunction(s,0,0,0,0,0,-1,-1);
07464         }
07465         return(1);
07466     }
07467     infun += FUNCTION;
07468     *s++ = c;
07469     while ( *s == ',' || *s == ' ' ) s++;
07470     outname = s; s = SkipAName(s);
07471     c = *s; *s = 0;
07472     if ( ( ( type = GetName(AC.varnames,outname,&outfun,WITHAUTO) ) != CFUNCTION )
07473     || ( ( functions[outfun].spec != 0 ) && ( functions[outfun].spec != TENSORFUNCTION ) ) ) {
07474         MesPrint("%s should be a regular function or a tensor.",outname);
07475         if ( type < 0 ) {
07476             if ( GetName(AC.exprnames,s,&outfun,NOAUTO) == NAMENOTFOUND )
07477                 AddFunction(s,0,0,0,0,0,-1,-1);
07478         }
07479         return(1);
07480     }
07481     outfun += FUNCTION;
07482     *s++ = c;
07483     if ( functions[infun].spec == TENSORFUNCTION ||
07484         functions[outfun].spec == TENSORFUNCTION ) tensorflag = 1;
07485     else tensorflag = 0;
07486 /*
07487     Now the type: type=....
07488 */
07489     while ( *s == ',' || *s == ' ' ) s++;
07490     stype = s;
07491     while ( FG.cTable[*s] == 0 ) s++;
07492     c = *s; *s = 0;
07493     if ( StrICmp(stype,(UBYTE *)"type") != 0 || c != '=' ) {
07494         MesPrint("&In CreateAllLoops statement: expected type=vartype.");
07495         return(1);
07496     }
07497     *s++ = c;
07498     stype = s;
07499     while ( FG.cTable[*s] == 0 ) s++;
07500     c = *s; *s = 0;
07501     if ( StrICmp(stype,(UBYTE *)"vector") == 0 ) {
07502         typenum = -VECTOR;
07503     }
07504     else if ( StrICmp(stype,(UBYTE *)"index") == 0 ) {
07505         typenum = -INDEX;
07506     }
07507     else if ( StrICmp(stype,(UBYTE *)"symbol") == 0 ) {
07508         if ( tensorflag ) goto notintensor;
07509         typenum = -SYMBOL;
07510     }
07511     else if ( StrICmp(stype,(UBYTE *)"snumber") == 0 ) {
07512         if ( tensorflag ) goto notintensor;
07513         typenum = -SNUMBER;
07514     }
07515     else {
07516         MesPrint("&Unknown/not allowed variable type in CreateAllLoops: %s",stype);
07517         return(1);
07518     }
07519     *s = c;
07520     while ( *s == ',' || *s == ' ' ) s++;
07521
07522     stype = s;
07523     while ( FG.cTable[*s] == 0 ) s++;
07524     c = *s; *s = 0;
07525     if ( StrICmp(stype,(UBYTE *)"ifnolooop") != 0 || c != '=' ) {
07526         MesPrint("&Unrecognised option in CreateAllLoops statement: %s",stype);
07527         return(1);
07528     }
07529     *s++ = c;
07530     x = -1;
07531     if ( FG.cTable[*s] == 1 ) {

```

```

07532         x = 0;
07533         do { x = 10*x + (*s++-'0'); } while (FG.cTable[*s] == 1);
07534     }
07535     if ( x != 0 && x != 1 ) {
07536         MesPrint("&Only options allowed for ifnolooop are 0 or 1.");
07537         return(1);
07538     }
07539     WorkSave = to = AT.WorkPointer;
07540     *to++ = TYPEALLLOOPS;
07541     *to++ = 6;
07542     *to++ = infun;
07543     *to++ = outfun;
07544     *to++ = typenum;
07545     *to++ = x;
07546
07547     AddNtoL(WorkSave[1],WorkSave);
07548
07549     return(0);
07550 notintensor:
07551     MesPrint("&Variable type not allowed in tensors: %s",type);
07552     return(1);
07553 }
07554
07555 /*
07556 #] CoCreateAllLoops :
07557 #[ CoCreateAllPaths :
07558
07559 Syntax:
07560     CreateAllPaths,end-function,intermediate-function,out-function,type-argument,ifnopath;
07561 Types allowed:
07562     vector, index, symbol, snumber
07563 ifnolooop:
07564     ifnolooop=0 or ifnolooop=1
07565 in-function can be a tensor. In that case type can be only vector or index.
07566 out-function can be a tensor. In that case type can be only vector or index.
07567 */
07568
07569 int CoCreateAllPaths(UBYTE *s)
07570 {
07571     GETIDENTITY
07572     UBYTE *endname,*iname, *outname, *stype, c;
07573     WORD endfun, infun, outfun, x, type, tensorflag, typenum;
07574     WORD *WorkSave, *to;
07575     while ( *s == ',' || *s == ' ' ) s++;
07576     endname = s; s = SkipAName(s);
07577     c = *s; *s = 0;
07578     if ( ( ( type = GetName(AC.varnames,endname,&endfun,WITHAUTO) ) != CFUNCTION )
07579 || ( ( functions[endfun].spec != 0 ) && ( functions[endfun].spec != TENSORFUNCTION ) ) ) {
07580         MesPrint("&%s should be a regular function or a tensor.",endname);
07581         if ( type < 0 ) {
07582             if ( GetName(AC.exprnames,s,&endfun,NOAUTO) == NAMENOTFOUND )
07583                 AddFunction(s,0,0,0,0,0,-1,-1);
07584         }
07585         return(1);
07586     }
07587     endfun += FUNCTION;
07588     *s++ = c;
07589     while ( *s == ',' || *s == ' ' ) s++;
07590     inname = s; s = SkipAName(s);
07591     c = *s; *s = 0;
07592     if ( ( ( type = GetName(AC.varnames,inname,&infun,WITHAUTO) ) != CFUNCTION )
07593 || ( ( functions[infun].spec != 0 ) && ( functions[infun].spec != TENSORFUNCTION ) ) ) {
07594         MesPrint("&%s should be a regular function or a tensor.",iname);
07595         if ( type < 0 ) {
07596             if ( GetName(AC.exprnames,s,&infun,NOAUTO) == NAMENOTFOUND )
07597                 AddFunction(s,0,0,0,0,0,-1,-1);
07598         }
07599         return(1);
07600     }
07601     infun += FUNCTION;
07602     *s++ = c;
07603     while ( *s == ',' || *s == ' ' ) s++;
07604     outname = s; s = SkipAName(s);
07605     c = *s; *s = 0;
07606     if ( ( ( type = GetName(AC.varnames,outname,&outfun,WITHAUTO) ) != CFUNCTION )
07607 || ( ( functions[outfun].spec != 0 ) && ( functions[outfun].spec != TENSORFUNCTION ) ) ) {
07608         MesPrint("&%s should be a regular function or a tensor.",outname);
07609         if ( type < 0 ) {
07610             if ( GetName(AC.exprnames,s,&outfun,NOAUTO) == NAMENOTFOUND )
07611                 AddFunction(s,0,0,0,0,0,-1,-1);
07612         }
07613         return(1);
07614     }
07615     outfun += FUNCTION;
07616     *s++ = c;
07617     if ( functions[infun].spec == TENSORFUNCTION ||
07618         functions[outfun].spec == TENSORFUNCTION ) tensorflag = 1;

```

```

07619     else tensorflag = 0;
07620 /*
07621     Now the type: type=....
07622 */
07623     while ( *s == ',' || *s == ' ' ) s++;
07624     stype = s;
07625     while ( FG.cTable[*s] == 0 ) s++;
07626     c = *s; *s = 0;
07627     if ( StrICmp(stype, (UBYTE *)"type") != 0 || c != '=' ) {
07628         MesPrint("&In CreateAllPaths statement: expected type=vartype.");
07629         return(1);
07630     }
07631     *s++ = c;
07632     stype = s;
07633     while ( FG.cTable[*s] == 0 ) s++;
07634     c = *s; *s = 0;
07635     if ( StrICmp(stype, (UBYTE *)"vector") == 0 ) {
07636         typenum = -VECTOR;
07637     }
07638     else if ( StrICmp(stype, (UBYTE *)"index") == 0 ) {
07639         typenum = -INDEX;
07640     }
07641     else if ( StrICmp(stype, (UBYTE *)"symbol") == 0 ) {
07642         if ( tensorflag ) goto notintensor;
07643         typenum = -SYMBOL;
07644     }
07645     else if ( StrICmp(stype, (UBYTE *)"snumber") == 0 ) {
07646         if ( tensorflag ) goto notintensor;
07647         typenum = -SNUMBER;
07648     }
07649     else {
07650         MesPrint("&Unknown/not allowed variable type in CreateAllPaths: %s", stype);
07651         return(1);
07652     }
07653     *s = c;
07654     while ( *s == ',' || *s == ' ' ) s++;
07655
07656     stype = s;
07657     while ( FG.cTable[*s] == 0 ) s++;
07658     c = *s; *s = 0;
07659     if ( StrICmp(stype, (UBYTE *)"ifnopath") != 0 || c != '=' ) {
07660         MesPrint("&Unrecognised option in CreateAllPaths statement: %s", stype);
07661         return(1);
07662     }
07663     *s++ = c;
07664     x = -1;
07665     if ( FG.cTable[*s] == 1 ) {
07666         x = 0;
07667         do { x = 10*x + (*s++-'0'); } while (FG.cTable[*s] == 1);
07668     }
07669     if ( x != 0 && x != 1 ) {
07670         MesPrint("&Only options allowed for ifnopath are 0 or 1.");
07671         return(1);
07672     }
07673     WorkSave = to = AT.WorkPointer;
07674     *to++ = TYPEALLPATHS;
07675     *to++ = 7;
07676     *to++ = endfun;
07677     *to++ = infun;
07678     *to++ = outfun;
07679     *to++ = typenum;
07680     *to++ = x;
07681
07682     AddNtoL(WorkSave[1], WorkSave);
07683
07684     return(0);
07685 notintensor:
07686     MesPrint("&CreateAllPaths: Variable type not allowed in tensors: %s", stype);
07687     return(1);
07688 }
07689
07690 /*
07691     #] CoCreateAllPaths :
07692     #[ CoCreateAll :
07693
07694     Syntax: subkey, two or three functions, type, ifnone
07695 */
07696 int CoCreateAll(UBYTE *s)
07697 {
07698     UBYTE *subkey;
07699     while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
07700     subkey = s;
07701     while ( FG.cTable[*s] == 0 ) s++;
07702     if ( *s != ' ' && *s != ',' && *s != '\t' ) {
07703         MesPrint("&Illegal subkey in CoCreate statement.");
07704         return(1);
07705     }

```

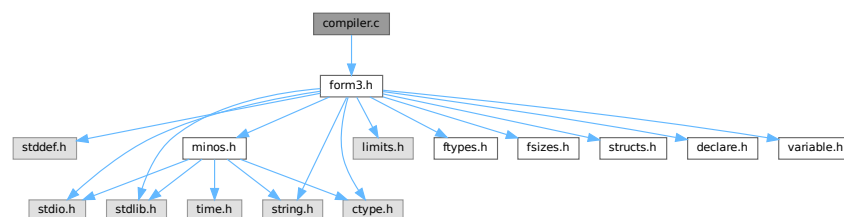
```

07706     }
07707     /* c = *s; */ *s++ = 0;
07708     while ( *s == '/' || *s == ',' || *s == '\\t' ) s++;
07709     if ( StrICmp(subkey, (UBYTE *) "loops" ) == 0 ) {
07710         return(CoCreateAllLoops(s));
07711     }
07712     else if ( StrICmp(subkey, (UBYTE *) "paths" ) == 0 ) {
07713         return(CoCreateAllPaths(s));
07714     }
07715     /*
07716     else if ( StrICmp(subkey, (UBYTE *) "motics" ) == 0 ) {
07717     }
07718     else if ( StrICmp(subkey, (UBYTE *) "onepi" ) == 0 ) {
07719     }
07720     else if ( StrICmp(subkey, (UBYTE *) "cuts" ) == 0 ) {
07721     }
07722     */
07723     else {
07724         MesPrint("%Illegal subkey in CoCreate statement: %s.", subkey);
07725         return(1);
07726     }
07727 }
07728
07729 /*
07730 #] CoCreateAll :
07731 */

```

4.11 compiler.c File Reference

#include "form3.h"
 Include dependency graph for compiler.c:



Data Structures

- struct [SuBbUf](#)

Macros

- #define [OPTION0](#) 1
- #define [OPTION1](#) 2
- #define [OPTION2](#) 3
- #define [REDUCESUBEXPBUFFERS](#)

Typedefs

- typedef struct [SuBbUf](#) SUBBUF

Functions

- void [inictable](#) (void)
- [KEYWORD](#) * [findcommand](#) (UBYTE *in)
- int [ParenthesesTest](#) (UBYTE *sin)
- UBYTE * [SkipAName](#) (UBYTE *s)
- UBYTE * [IsRHS](#) (UBYTE *s, UBYTE c)
- int [IsIdStatement](#) (UBYTE *s)
- int [CompileAlgebra](#) (UBYTE *s, int leftright, WORD *prototype)
- int [CompileStatement](#) (UBYTE *in)
- int [TestTables](#) (void)
- int [CompileSubExpressions](#) (SBYTE *tokens)
- int [CodeGenerator](#) (SBYTE *tokens)
- int [CompleteTerm](#) (WORD *term, UWORD *numer, UWORD *denom, WORD nnum, WORD nden, int sign)
- int [CodeFactors](#) (SBYTE *tokens)
- WORD [GenerateFactors](#) (WORD n, WORD inc)

Variables

- int [alfatable1](#) [27]
- [SUBBUF](#) * [subexpbuffers](#) = 0
- [SUBBUF](#) * [topsubexpbuffers](#) = 0
- LONG [insubexpbuffers](#) = 0

4.11.1 Detailed Description

The heart of the compiler. It contains the tables of statements. It finds the statements in the tables and calls the proper routines. For algebraic expressions it runs the compilation by first calling the tokenizer, splitting things into subexpressions and generating the code. There is a system for recognizing already existing subexpressions. This economizes on the length of the output.

Note: the compiler of FORM doesn't attempt to normalize the input. Hence $x+1$ and $1+x$ are different objects during compilation. Similarly $(a+b-b)$ will not be simplified to (a) .

Definition in file [compiler.c](#).

4.11.2 Macro Definition Documentation

4.11.2.1 OPTION0

```
#define OPTION0 1
```

Definition at line 253 of file [compiler.c](#).

4.11.2.2 OPTION1

```
#define OPTION1 2
```

Definition at line 254 of file [compiler.c](#).

4.11.2.3 OPTION2

```
#define OPTION2 3
```

Definition at line 255 of file [compiler.c](#).

4.11.2.4 REDUCESUBEXPBUFFERS

```
#define REDUCESUBEXPBUFFERS
```

Value:

```
{ if ( (topsubexpbuffers-subexpbuffers) > 256 ) {\
  M_free(subexpbuffers,"subexpbuffers");\
  subexpbuffers = (SUBBUF *)Malloc1(256*sizeof(SUBBUF),"subexpbuffers");\
  topsubexpbuffers = subexpbuffers+256; } insubexpbuffers = 0; }
```

Definition at line 266 of file [compiler.c](#).

4.11.3 Function Documentation

4.11.3.1 inictable()

```
void inictable (
    void )
```

Definition at line 310 of file [compiler.c](#).

4.11.3.2 findcommand()

```
KEYWORD * findcommand (
    UBYTE * in )
```

Definition at line 336 of file [compiler.c](#).

4.11.3.3 ParenthesesTest()

```
int ParenthesesTest (
    UBYTE * sin )
```

Definition at line 380 of file [compiler.c](#).

4.11.3.4 SkipAName()

```
UBYTE * SkipAName (
    UBYTE * s )
```

Definition at line 430 of file [compiler.c](#).

4.11.3.5 IsRHS()

```
UBYTE * IsRHS (
    UBYTE * s,
    UBYTE c )
```

Definition at line 458 of file [compiler.c](#).

4.11.3.6 IsIdStatement()

```
int IsIdStatement (
    UBYTE * s )
```

Definition at line 504 of file [compiler.c](#).

4.11.3.7 CompileAlgebra()

```
int CompileAlgebra (
    UBYTE * s,
    int leftright,
    WORD * prototype )
```

Definition at line 518 of file [compiler.c](#).

4.11.3.8 CompileStatement()

```
int CompileStatement (
    UBYTE * in )
```

Definition at line 554 of file [compiler.c](#).

4.11.3.9 TestTables()

```
int TestTables (
    void )
```

Definition at line 669 of file [compiler.c](#).

4.11.3.10 CompileSubExpressions()

```
int CompileSubExpressions (
    SBYTE * tokens )
```

Definition at line 713 of file [compiler.c](#).

4.11.3.11 CodeGenerator()

```
int CodeGenerator (
    SBYTE * tokens )
```

Definition at line [848](#) of file [compiler.c](#).

4.11.3.12 CompleteTerm()

```
int CompleteTerm (
    WORD * term,
    UWORD * numer,
    UWORD * denom,
    WORD nnum,
    WORD nden,
    int sign )
```

Definition at line [1911](#) of file [compiler.c](#).

4.11.3.13 CodeFactors()

```
int CodeFactors (
    SBYTE * tokens )
```

Definition at line [1948](#) of file [compiler.c](#).

4.11.3.14 GenerateFactors()

```
WORD GenerateFactors (
    WORD n,
    WORD inc )
```

Definition at line [2245](#) of file [compiler.c](#).

4.11.4 Variable Documentation

4.11.4.1 alfatable1

```
int alfatable1[27]
```

Definition at line [251](#) of file [compiler.c](#).

4.11.4.2 subexpbuffers

```
SUBBUF* subexpbuffers = 0
```

Definition at line [262](#) of file [compiler.c](#).

4.11.4.3 topsubexpbuffers

`SUBBUF* topsubexpbuffers = 0`

Definition at line 263 of file [compiler.c](#).

4.11.4.4 insubexpbuffers

`LONG insubexpbuffers = 0`

Definition at line 264 of file [compiler.c](#).

4.12 compiler.c

[Go to the documentation of this file.](#)

```
00001
00015 /* #[] License : */
00016 /*
00017 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00018 * When using this file you are requested to refer to the publication
00019 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00020 * This is considered a matter of courtesy as the development was paid
00021 * for by FOM the Dutch physics granting agency and we would like to
00022 * be able to track its scientific use to convince FOM of its value
00023 * for the community.
00024 *
00025 * This file is part of FORM.
00026 *
00027 * FORM is free software: you can redistribute it and/or modify it under the
00028 * terms of the GNU General Public License as published by the Free Software
00029 * Foundation, either version 3 of the License, or (at your option) any later
00030 * version.
00031 *
00032 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00033 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00034 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00035 * details.
00036 *
00037 * You should have received a copy of the GNU General Public License along
00038 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00039 */
00040 /* #[] License : */
00041 /*
00042 #[] includes :
00043 */
00044 #include "form3.h"
00045
00047 /*
00048 com1commands are the commands of which only part of the word has to
00049 be present. The order is rather important here.
00050 com2commands are the commands that must have their whole word match.
00051 here we can do a binary search.
00052 {{{
00053 */
00054
00055 static KEYWORD com1commands[] = {
00056 {"also", (TFUN)CoIdOld, STATEMENT, PARTEST}
00057 {"abackets", (TFUN)CoAntiBracket, TOOUTPUT, PARTEST}
00058 {"antisymmetrize", (TFUN)CoAntiSymmetrize, STATEMENT, PARTEST}
00059 {"antibrackets", (TFUN)CoAntiBracket, TOOUTPUT, PARTEST}
00060 {"brackets", (TFUN)CoBracket, TOOUTPUT, PARTEST}
00061 {"cfunctions", (TFUN)CoCFunction, DECLARATION, PARTEST|WITHAUTO}
00062 {"commuting", (TFUN)CoCFunction, DECLARATION, PARTEST|WITHAUTO}
00063 {"compress", (TFUN)CoCompress, DECLARATION, PARTEST}
00064 {"ctensors", (TFUN)CoCTensor, DECLARATION, PARTEST|WITHAUTO}
00065 {"cyclesymmetrize", (TFUN)CoCycleSymmetrize, STATEMENT, PARTEST}
00066 {"dimension", (TFUN)CoDimension, DECLARATION, PARTEST}
00067 {"discard", (TFUN)CoDiscard, STATEMENT, PARTEST}
00068 {"functions", (TFUN)CoFunction, DECLARATION, PARTEST|WITHAUTO}
00069 {"format", (TFUN)CoFormat, TOOUTPUT, PARTEST}
00070 {"fixindex", (TFUN)CoFixIndex, DECLARATION, PARTEST}
00071 {"global", (TFUN)CoGlobal, DEFINITION, PARTEST}
```

```

00072     ,{"gfactorized",      (TFUN)CoGlobalFactorized, DEFINITION, PARTEST}
00073     ,{"globalfactorized", (TFUN)CoGlobalFactorized, DEFINITION, PARTEST}
00074     ,{"goto",            (TFUN)CoGoTo,          STATEMENT, PARTEST}
00075     ,{"indexes",         (TFUN)CoIndex,         DECLARATION, PARTEST|WITHAUTO}
00076     ,{"indices",         (TFUN)CoIndex,         DECLARATION, PARTEST|WITHAUTO}
00077     ,{"identify",        (TFUN)CoId,           STATEMENT, PARTEST}
00078     ,{"idnew",           (TFUN)CoIdNew,         STATEMENT, PARTEST}
00079     ,{"idold",           (TFUN)CoIdOld,         STATEMENT, PARTEST}
00080     ,{"local",           (TFUN)CoLocal,         DEFINITION, PARTEST}
00081     ,{"lfactorized",     (TFUN)CoLocalFactorized, DEFINITION, PARTEST}
00082     ,{"localfactorized", (TFUN)CoLocalFactorized, DEFINITION, PARTEST}
00083     ,{"load",            (TFUN)CoLoad,          DECLARATION, PARTEST}
00084     ,{"label",           (TFUN)CoLabel,         STATEMENT, PARTEST}
00085     ,{"modulus",         (TFUN)CoModulus,       DECLARATION, PARTEST}
00086     ,{"multiply",        (TFUN)CoMultiply,      STATEMENT, PARTEST}
00087     ,{"nfunctions",     (TFUN)CoNFunction,     DECLARATION, PARTEST|WITHAUTO}
00088     ,{"nprint",          (TFUN)CoNPrint,        TOOUTPUT, PARTEST}
00089     ,{"ntensors",        (TFUN)CoNTensor,       DECLARATION, PARTEST|WITHAUTO}
00090     ,{"nwrite",          (TFUN)CoNWrite,        DECLARATION, PARTEST}
00091     ,{"print",           (TFUN)CoPrint,         MIXED, 0}
00092     ,{"redefine",        (TFUN)CoRedefine,      STATEMENT, 0}
00093     ,{"rcyclesymmetrize", (TFUN)CoRCycleSymmetrize, STATEMENT, PARTEST}
00094     ,{"symbols",         (TFUN)CoSymbol,        DECLARATION, PARTEST|WITHAUTO}
00095     ,{"save",            (TFUN)CoSave,          DECLARATION, PARTEST}
00096     ,{"symmetrize",      (TFUN)CoSymmetrize,    STATEMENT, PARTEST}
00097     ,{"tensors",        (TFUN)CoCTensor,       DECLARATION, PARTEST|WITHAUTO}
00098     ,{"unittrace",       (TFUN)CoUnitTrace,     DECLARATION, PARTEST}
00099     ,{"vectors",         (TFUN)CoVector,        DECLARATION, PARTEST|WITHAUTO}
00100     ,{"write",           (TFUN)CoWrite,         DECLARATION, PARTEST}
00101 };
00102
00103 static KEYWORD com2commands[] = {
00104     {"antiputinside",    (TFUN)CoAntiPutInside, STATEMENT, PARTEST}
00105     ,{"apply",           (TFUN)CoApply,          STATEMENT, PARTEST}
00106     ,{"aputinside",      (TFUN)CoAntiPutInside, STATEMENT, PARTEST}
00107     ,{"argexplode",      (TFUN)CoArgExplode,     STATEMENT, PARTEST}
00108     ,{"argimplode",      (TFUN)CoArgImplode,     STATEMENT, PARTEST}
00109     ,{"argtoextrasymbol", (TFUN)CoArgToExtraSymbol, STATEMENT, PARTEST}
00110     ,{"argument",        (TFUN)CoArgument,       STATEMENT, PARTEST}
00111     ,{"assign",          (TFUN)CoAssign,         STATEMENT, PARTEST}
00112     ,{"auto",            (TFUN)CoAuto,           DECLARATION, PARTEST}
00113     ,{"autodeclare",     (TFUN)CoAuto,           DECLARATION, PARTEST}
00114     ,{"break",           (TFUN)CoBreak,          STATEMENT, PARTEST}
00115     ,{"canonicalize",    (TFUN)CoCanonicalize,   STATEMENT, PARTEST}
00116     ,{"case",            (TFUN)CoCase,           STATEMENT, PARTEST}
00117     ,{"chainin",         (TFUN)CoChainin,        STATEMENT, PARTEST}
00118     ,{"chainout",        (TFUN)CoChainout,       STATEMENT, PARTEST}
00119     ,{"chisholm",        (TFUN)CoChisholm,       STATEMENT, PARTEST}
00120     ,{"cleartable",      (TFUN)CoClearTable,     DECLARATION, PARTEST}
00121     ,{"clearflag",       (TFUN)CoClearUserFlag,  STATEMENT, PARTEST}
00122     ,{"collect",         (TFUN)CoCollect,        SPECIFICATION, PARTEST}
00123     ,{"commuteinset",    (TFUN)CoCommuteInSet,   DECLARATION, PARTEST}
00124     ,{"contract",        (TFUN)CoContract,       STATEMENT, PARTEST}
00125     ,{"copspectator",    (TFUN)CoCopySpectator,  DEFINITION, PARTEST}
00126     ,{"createall",       (TFUN)CoCreateAll,      STATEMENT, PARTEST}
00127     ,{"createspectator", (TFUN)CoCreateSpectator,  DECLARATION, PARTEST}
00128     ,{"ctable",          (TFUN)CoCTable,         DECLARATION, PARTEST}
00129     ,{"deallocate",      (TFUN)CoDeallocateTable, DECLARATION, PARTEST}
00130     ,{"default",         (TFUN)CoDefault,        STATEMENT, PARTEST}
00131     ,{"delete",          (TFUN)CoDelete,         SPECIFICATION, PARTEST}
00132     ,{"denominators",    (TFUN)CoDenominators,   STATEMENT, PARTEST}
00133     ,{"disorder",        (TFUN)CoDisorder,       STATEMENT, PARTEST}
00134     ,{"do",              (TFUN)CoDo,             STATEMENT, PARTEST}
00135     ,{"drop",            (TFUN)CoDrop,           SPECIFICATION, PARTEST}
00136     ,{"dropcoefficient", (TFUN)CoDropCoefficient,  STATEMENT, PARTEST}
00137     ,{"dropsymbols",     (TFUN)CoDropSymbols,    STATEMENT, PARTEST}
00138     ,{"else",            (TFUN)CoElse,           STATEMENT, PARTEST}
00139     ,{"elseif",          (TFUN)CoElseIf,         STATEMENT, PARTEST}
00140     ,{"emptyspectator",  (TFUN)CoEmptySpectator,  SPECIFICATION, PARTEST}
00141     ,{"endargument",     (TFUN)CoEndArgument,   STATEMENT, PARTEST}
00142     ,{"enddo",           (TFUN)CoEndDo,          STATEMENT, PARTEST}
00143     ,{"endif",           (TFUN)CoEndIf,          STATEMENT, PARTEST}
00144     ,{"endinexpression", (TFUN)CoEndInExpression,  STATEMENT, PARTEST}
00145     ,{"endinside",       (TFUN)CoEndInside,     STATEMENT, PARTEST}
00146     ,{"endmodel",        (TFUN)CoEndModel,       DECLARATION, PARTEST}
00147     ,{"endrepeat",       (TFUN)CoEndRepeat,     STATEMENT, PARTEST}
00148     ,{"endswitch",       (TFUN)CoEndSwitch,     STATEMENT, PARTEST}
00149     ,{"endterm",         (TFUN)CoEndTerm,        STATEMENT, PARTEST}
00150     ,{"endwhile",        (TFUN)CoEndWhile,       STATEMENT, PARTEST}
00151 #ifdef WITHFLOAT
00152     ,{"evaluate",        (TFUN)CoEvaluate,       STATEMENT, PARTEST}
00153 #endif
00154     ,{"exit",            (TFUN)CoExit,           STATEMENT, PARTEST}
00155     ,{"extrasymbols",    (TFUN)CoExtraSymbols,   DECLARATION, PARTEST}
00156     ,{"factarg",         (TFUN)CoFactArg,        STATEMENT, PARTEST}
00157     ,{"factdollar",      (TFUN)CoFactDollar,    STATEMENT, PARTEST}
00158     ,{"factorize",       (TFUN)CoFactorize,      TOOUTPUT, PARTEST}

```

```

00159     ,{"fill", (TFUN) CoFill, DECLARATION, PARTEST}
00160     ,{"fillexpression", (TFUN) CoFillExpression, DECLARATION, PARTEST}
00161     ,{"frompolynomial", (TFUN) CoFromPolynomial, STATEMENT, PARTEST}
00162     ,{"funpowers", (TFUN) CoFunPowers, DECLARATION, PARTEST}
00163     ,{"hide", (TFUN) CoHide, SPECIFICATION, PARTEST}
00164     ,{"if", (TFUN) CoIf, STATEMENT, PARTEST}
00165     ,{"ifmatch", (TFUN) CoIfMatch, STATEMENT, PARTEST}
00166     ,{"ifnomatch", (TFUN) CoIfNoMatch, STATEMENT, PARTEST}
00167     ,{"ifnotmatch", (TFUN) CoIfNoMatch, STATEMENT, PARTEST}
00168     ,{"inexpression", (TFUN) CoInExpression, STATEMENT, PARTEST}
00169     ,{"inparallel", (TFUN) CoInParallel, SPECIFICATION, PARTEST}
00170     ,{"inside", (TFUN) CoInside, STATEMENT, PARTEST}
00171     ,{"insidefirst", (TFUN) CoInsideFirst, DECLARATION, PARTEST}
00172     ,{"intohide", (TFUN) CoIntoHide, SPECIFICATION, PARTEST}
00173     ,{"keep", (TFUN) CoKeep, SPECIFICATION, PARTEST}
00174     ,{"makeinteger", (TFUN) CoMakeInteger, STATEMENT, PARTEST}
00175     ,{"many", (TFUN) CoMany, STATEMENT, PARTEST}
00176     ,{"merge", (TFUN) CoMerge, STATEMENT, PARTEST}
00177     ,{"metric", (TFUN) CoMetric, DECLARATION, PARTEST}
00178     ,{"model", (TFUN) CoModel, DECLARATION, PARTEST}
00179     ,{"moduleoption", (TFUN) CoModuleOption, ATENDOFMODULE, PARTEST}
00180     ,{"multi", (TFUN) CoMulti, STATEMENT, PARTEST}
00181     ,{"multibracket", (TFUN) CoMultiBracket, STATEMENT, PARTEST}
00182     ,{"ndrop", (TFUN) CoNoDrop, SPECIFICATION, PARTEST}
00183     ,{"nfactorize", (TFUN) CoNFactorize, TOOUTPUT, PARTEST}
00184     ,{"nhide", (TFUN) CoNoHide, SPECIFICATION, PARTEST}
00185     ,{"nintohide", (TFUN) CoNoIntoHide, SPECIFICATION, PARTEST}
00186     ,{"normalize", (TFUN) CoNormalize, STATEMENT, PARTEST}
00187     ,{"notinparallel", (TFUN) CoNotInParallel, SPECIFICATION, PARTEST}
00188     ,{"nskip", (TFUN) CoNoSkip, SPECIFICATION, PARTEST}
00189     ,{"ntable", (TFUN) CoNTable, DECLARATION, PARTEST}
00190     ,{"nunfactorize", (TFUN) CoUnFactorize, TOOUTPUT, PARTEST}
00191     ,{"nunhide", (TFUN) CoNoUnHide, SPECIFICATION, PARTEST}
00192     ,{"off", (TFUN) CoOff, DECLARATION, PARTEST}
00193     ,{"on", (TFUN) CoOn, DECLARATION, PARTEST}
00194     ,{"once", (TFUN) CoOnce, STATEMENT, PARTEST}
00195     ,{"only", (TFUN) CoOnly, STATEMENT, PARTEST}
00196     ,{"particle", (TFUN) CoParticle, DECLARATION, PARTEST}
00197     ,{"polyfun", (TFUN) CoPolyFun, DECLARATION, PARTEST}
00198     ,{"polyratfun", (TFUN) CoPolyRatFun, DECLARATION, PARTEST}
00199     ,{"pophide", (TFUN) CoPopHide, SPECIFICATION, PARTEST}
00200     ,{"print[]", (TFUN) CoPrintB, TOOUTPUT, PARTEST}
00201     ,{"printtable", (TFUN) CoPrintTable, MIXED, PARTEST}
00202     ,{"processbucketsize", (TFUN) CoProcessBucket, DECLARATION, PARTEST}
00203     ,{"propercount", (TFUN) CoProperCount, DECLARATION, PARTEST}
00204     ,{"pushhide", (TFUN) CoPushHide, SPECIFICATION, PARTEST}
00205     ,{"putinside", (TFUN) CoPutInside, STATEMENT, PARTEST}
00206     ,{"ratio", (TFUN) CoRatio, STATEMENT, PARTEST}
00207     ,{"removespectator", (TFUN) CoRemoveSpectator, SPECIFICATION, PARTEST}
00208     ,{"renumber", (TFUN) CoRename, STATEMENT, PARTEST}
00209     ,{"repeat", (TFUN) CoRepeat, STATEMENT, PARTEST}
00210     ,{"replaceloop", (TFUN) CoReplaceLoop, STATEMENT, PARTEST}
00211     ,{"select", (TFUN) CoSelect, STATEMENT, PARTEST}
00212     ,{"set", (TFUN) CoSet, DECLARATION, PARTEST}
00213     ,{"setexitflag", (TFUN) CoSetExitFlag, STATEMENT, PARTEST}
00214     ,{"setflag", (TFUN) CoSetUserFlag, STATEMENT, PARTEST}
00215     ,{"shuffle", (TFUN) CoMerge, STATEMENT, PARTEST}
00216     ,{"skip", (TFUN) CoSkip, SPECIFICATION, PARTEST}
00217     ,{"sort", (TFUN) CoSort, STATEMENT, PARTEST}
00218     ,{"splitarg", (TFUN) CoSplitArg, STATEMENT, PARTEST}
00219     ,{"splitfirstarg", (TFUN) CoSplitFirstArg, STATEMENT, PARTEST}
00220     ,{"splitlastarg", (TFUN) CoSplitLastArg, STATEMENT, PARTEST}
00221     ,{"shuffle", (TFUN) CoShuffle, STATEMENT, PARTEST}
00222     ,{"sum", (TFUN) CoSum, STATEMENT, PARTEST}
00223     ,{"switch", (TFUN) CoSwitch, STATEMENT, PARTEST}
00224     ,{"table", (TFUN) CoTable, DECLARATION, PARTEST}
00225     ,{"tablebase", (TFUN) CoTableBase, DECLARATION, PARTEST}
00226     ,{"tb", (TFUN) CoTableBase, DECLARATION, PARTEST}
00227     ,{"term", (TFUN) CoTerm, STATEMENT, PARTEST}
00228     ,{"testuse", (TFUN) CoTestUse, STATEMENT, PARTEST}
00229     ,{"threadbucketsize", (TFUN) CoThreadBucket, DECLARATION, PARTEST}
00230 #ifdef WITHFLOAT
00231     ,{"tofloat", (TFUN) CoToFloat, STATEMENT, PARTEST}
00232 #endif
00233     ,{"topolynomial", (TFUN) CoToPolynomial, STATEMENT, PARTEST}
00234 #ifdef WITHFLOAT
00235     ,{"torat", (TFUN) CoToRat, STATEMENT, PARTEST}
00236     ,{"torational", (TFUN) CoToRat, STATEMENT, PARTEST}
00237 #endif
00238     ,{"tospectator", (TFUN) CoToSpectator, STATEMENT, PARTEST}
00239     ,{"totensor", (TFUN) CoToTensor, STATEMENT, PARTEST}
00240     ,{"tovector", (TFUN) CoToVector, STATEMENT, PARTEST}
00241     ,{"trace4", (TFUN) CoTrace4, STATEMENT, PARTEST}
00242     ,{"tracen", (TFUN) CoTraceN, STATEMENT, PARTEST}
00243     ,{"transform", (TFUN) CoTransform, STATEMENT, PARTEST}
00244     ,{"tryreplace", (TFUN) CoTryReplace, STATEMENT, PARTEST}
00245     ,{"unfactorize", (TFUN) CoUnFactorize, TOOUTPUT, PARTEST}

```

```

00246     ,{"unhide",          (TFUN)CoUnHide,          SPECIFICATION,PARTEST}
00247     ,{"vertex",         (TFUN)CoVertex,          DECLARATION,  PARTEST}
00248     ,{"while",          (TFUN)CoWhile,          STATEMENT,    PARTEST}
00249 };
00250
00251 int alfatable1[27];
00252
00253 #define OPTION0 1
00254 #define OPTION1 2
00255 #define OPTION2 3
00256
00257 typedef struct SuBbUf {
00258     WORD      subexpnum;
00259     WORD      buffernum;
00260 } SUBBUF;
00261
00262 SUBBUF *subexpbuffers = 0;
00263 SUBBUF *topsubexpbuffers = 0;
00264 LONG insubexpbuffers = 0;
00265
00266 #define REDUCESUBEXPBUFFERS { if ( (topsubexpbuffers-subexpbuffers) > 256 ) {\
00267     M_free(subexpbuffers,"subexpbuffers");\
00268     subexpbuffers = (SUBBUF *)Malloc1(256*sizeof(SUBBUF),"subexpbuffers");\
00269     topsubexpbuffers = subexpbuffers+256; } insubexpbuffers = 0; }
00270
00271 #if defined(ILP32)
00272
00273 #define PUTNUMBER128(t,n) { if ( n >= 16384 ) { \
00274     *t++ = n/(128*128); *t++ = (n/128)%128; *t++ = n%128; } \
00275     else if ( n >= 128 ) { *t++ = n/128; *t++ = n%128; } \
00276     else *t++ = n; }
00277 #define PUTNUMBER100(t,n) { if ( n >= 10000 ) { \
00278     *t++ = n/10000; *t++ = (n/100)%100; *t++ = n%100; } \
00279     else if ( n >= 100 ) { *t++ = n/100; *t++ = n%100; } \
00280     else *t++ = n; }
00281
00282 #elif ( defined(LLP64) || defined(LP64) )
00283
00284 #define PUTNUMBER128(t,n) { if ( n >= 2097152 ) { \
00285     *t++ = ((n/128)/128)/128; *t++ = ((n/128)/128)%128; *t++ = (n/128)%128; *t++ = n%128;
00286 } \
00287     else if ( n >= 16384 ) { \
00288     *t++ = n/(128*128); *t++ = (n/128)%128; *t++ = n%128; } \
00289     else if ( n >= 128 ) { *t++ = n/128; *t++ = n%128; } \
00290     else *t++ = n; }
00291 #define PUTNUMBER100(t,n) { if ( n >= 1000000 ) { \
00292     *t++ = ((n/100)/100)/100; *t++ = ((n/100)/100)%100; *t++ = (n/100)%100; *t++ = n%100;
00293 } \
00294     else if ( n >= 10000 ) { \
00295     *t++ = n/10000; *t++ = (n/100)%100; *t++ = n%100; } \
00296     else if ( n >= 100 ) { *t++ = n/100; *t++ = n%100; } \
00297     else *t++ = n; }
00298 #endif
00299 /*
00300     ]}]
00301     #] includes :
00302     #[ Compiler :
00303     #[ inictable :
00304
00305     Routine sets the table for 1-st characters that allow a faster
00306     start in the search in table 1 which should be sequential.
00307     Search in table 2 can be binary.
00308 */
00309
00310 void inictable(void)
00311 {
00312     KEYWORD *k = com1commands;
00313     int i, j, ksize;
00314     ksize = sizeof(com1commands)/sizeof(KEYWORD);
00315     j = 0;
00316     alfatable1[0] = 0;
00317     for ( i = 0; i < 26; i++ ) {
00318         while ( j < ksize && k[j].name[0] == 'a'+i ) j++;
00319         alfatable1[i+1] = j;
00320     }
00321 }
00322
00323 /*
00324     #] inictable :
00325     #[ findcommand :
00326
00327     Checks whether a command is in the command table.
00328     If so a pointer to the table element is returned.
00329     If not we return 0.
00330     Note that when a command is not in the table, we have

```

```

00331         to test whether it is an id command without id. It should
00332         then have the structure pattern = rhs. This should be done
00333         in the calling routine.
00334     */
00335
00336     KEYWORD *findcommand(UBYTE *in)
00337     {
00338         int hi, med, lo, i;
00339         UBYTE *s, c;
00340         s = in;
00341         while ( FG.cTable[*s] <= 1 ) s++;
00342         if ( s > in && *s == '[' && s[1] == ']' ) s += 2;
00343         if ( *s ) { c = *s; *s = 0; }
00344         else c = 0;
00345     /*
00346        First do a binary search in the second table
00347    */
00348        lo = 0;
00349        hi = sizeof(com2commands)/sizeof(KEYWORD)-1;
00350        do {
00351            med = ( hi + lo ) / 2;
00352            i = StrICmp(in, (UBYTE *)com2commands[med].name);
00353            if ( i == 0 ) { if ( c ) *s = c; return(com2commands+med); }
00354            if ( i < 0 ) hi = med-1;
00355            else lo = med+1;
00356        } while ( hi >= lo );
00357    /*
00358        Now do a 'hashed' search in the first table. It is sequential.
00359    */
00360        i = tolower(*in) - 'a';
00361        med = alfatable1[i];
00362        hi = alfatable1[i+1];
00363        while ( med < hi ) {
00364            if ( StrICont(in, (UBYTE *)com1commands[med].name) == 0 )
00365                { if ( c ) *s = c; return(com1commands+med); }
00366            med++;
00367        }
00368        if ( c ) *s = c;
00369    /*
00370        Unrecognized. Too bad!
00371    */
00372        return(0);
00373    }
00374
00375    /*
00376        #] findcommand :
00377        #[ ParenthesesTest :
00378    */
00379
00380    int ParenthesesTest(UBYTE *sin)
00381    {
00382        WORD L1 = 0, L2 = 0, L3 = 0;
00383        UBYTE *s = sin;
00384        while ( *s ) {
00385            if ( *s == '[' ) L1++;
00386            else if ( *s == ']' ) {
00387                L1--;
00388                if ( L1 < 0 ) { MesPrint("&Unmatched []"); return(1); }
00389            }
00390            s++;
00391        }
00392        if ( L1 > 0 ) { MesPrint("&Unmatched []"); return(1); }
00393        s = sin;
00394        while ( *s ) {
00395            if ( *s == '[' ) SKIPBRA1(s)
00396            else if ( *s == '(' ) { L2++; s++; }
00397            else if ( *s == ')' ) {
00398                L2--; s++;
00399                if ( L2 < 0 ) { MesPrint("&Unmatched ()"); return(1); }
00400            }
00401            else s++;
00402        }
00403        if ( L2 > 0 ) { MesPrint("&Unmatched ()"); return(1); }
00404        s = sin;
00405        while ( *s ) {
00406            if ( *s == '[' ) SKIPBRA1(s)
00407            else if ( *s == '[' ) SKIPBRA4(s)
00408            else if ( *s == '{' ) { L3++; s++; }
00409            else if ( *s == '}' ) {
00410                L3--; s++;
00411                if ( L3 < 0 ) { MesPrint("&Unmatched {}"); return(1); }
00412            }
00413            else s++;
00414        }
00415        if ( L3 > 0 ) { MesPrint("&Unmatched {}"); return(1); }
00416        return(0);
00417    }

```

```

00418
00419 /*
00420     #] ParenthesesTest :
00421     #[ SkipAName :
00422
00423     Skips a name and gives a pointer to the object after the name.
00424     If there is not a proper name, it returns a zero pointer.
00425     In principle the brackets match already, so the `if ( *s == 0 )'
00426     code is not really needed, but you never know how the program
00427     is extended later.
00428 */
00429
00430 UBYTE *SkipAName(UBYTE *s)
00431 {
00432     UBYTE *t = s;
00433     if ( *s == '[' ) {
00434         SKIPBRA1(s)
00435         if ( *s == 0 ) {
00436             MesPrint("&Illegal name: '%s'",t);
00437             return(0);
00438         }
00439         s++;
00440     }
00441     else if ( FG.cTable[*s] == 0 || *s == '_' || *s == '$' ) {
00442         if ( *s == '$' ) s++;
00443         while ( FG.cTable[*s] <= 1 ) s++;
00444         if ( *s == '_' ) s++;
00445     }
00446     else {
00447         MesPrint("&Illegal name: '%s'",t);
00448         return(0);
00449     }
00450     return(s);
00451 }
00452
00453 /*
00454     #] SkipAName :
00455     #[ IsRHS :
00456 */
00457
00458 UBYTE *IsRHS(UBYTE *s, UBYTE c)
00459 {
00460     while ( *s && *s != c ) {
00461         if ( *s == '[' ) {
00462             SKIPBRA1(s);
00463             if ( *s != ']' ) {
00464                 MesPrint("&Unmatched []");
00465                 return(0);
00466             }
00467         }
00468         else if ( *s == '{' ) {
00469             SKIPBRA2(s);
00470             if ( *s != '}' ) {
00471                 MesPrint("&Unmatched {}");
00472                 return(0);
00473             }
00474         }
00475         else if ( *s == '(' ) {
00476             SKIPBRA3(s);
00477             if ( *s != ')' ) {
00478                 MesPrint("&Unmatched ()");
00479                 return(0);
00480             }
00481         }
00482         else if ( *s == ')' ) {
00483             MesPrint("&Unmatched ()");
00484             return(0);
00485         }
00486         else if ( *s == '}' ) {
00487             MesPrint("&Unmatched {}");
00488             return(0);
00489         }
00490         else if ( *s == ']' ) {
00491             MesPrint("&Unmatched []");
00492             return(0);
00493         }
00494         s++;
00495     }
00496     return(s);
00497 }
00498
00499 /*
00500     #] IsRHS :
00501     #[ IsIdStatement :
00502 */
00503
00504 int IsIdStatement(UBYTE *s)

```

```

00505 {
00506     DUMMYUSE(s);
00507     return(0);
00508 }
00509
00510 /*
00511     #] IsIdStatement :
00512     #[ CompileAlgebra :
00513
00514     Returns either the number of the main level RHS (>= 0)
00515     or an error code (< 0)
00516 */
00517
00518 int CompileAlgebra(UBYTE *s, int leftright, WORD *prototype)
00519 {
00520     GETIDENTITY
00521     int error;
00522     WORD *oldproto = AC.ProtoType;
00523     AC.ProtoType = prototype;
00524     if ( AC.TokensWriteFlag ) {
00525         MesPrint("To tokenize: %s",s);
00526         error = tokenize(s,leftright);
00527         MesPrint(" The contents of the token buffer are:");
00528         WriteTokens(AC.tokens);
00529     }
00530     else error = tokenize(s,leftright);
00531     if ( error == 0 ) {
00532         AR.Eside = leftright;
00533         AC.CompileLevel = 0;
00534         if ( leftright == LHSIDE ) { AC.DumNum = AR.CurDum = 0; }
00535         error = CompileSubExpressions(AC.tokens);
00536         REDUCESUBEXPBUFFERS
00537     }
00538     else {
00539         AC.ProtoType = oldproto;
00540         return(-1);
00541     }
00542     AC.ProtoType = oldproto;
00543     if ( error < 0 ) return(-1);
00544     else if ( leftright == LHSIDE ) return(cbuf[AC.cbufnum].numlhs);
00545     else return(cbuf[AC.cbufnum].numrhs);
00546 }
00547
00548 /*
00549     #] CompileAlgebra :
00550     #[ CompileStatement :
00551
00552 */
00553
00554 int CompileStatement(UBYTE *in)
00555 {
00556     KEYWORD *k;
00557     UBYTE *s;
00558     int error1 = 0, error2;
00559     /* A.iStatement = */ s = in;
00560     if ( *s == 0 ) return(0);
00561     if ( *s == '$' ) {
00562         k = findcommand((UBYTE *) "assign");
00563     }
00564     else {
00565         if ( ( k = findcommand(s) ) == 0 && IsIdStatement(s) == 0 ) {
00566             MesPrint("&Unrecognized statement %s",s);
00567             return(1);
00568         }
00569         if ( k == 0 ) { /* Id statement without id. Note: id must be in table */
00570             k = com1commands + alfatable1['i'-'a'];
00571             while ( k->name[1] != 'd' || k->name[2] ) k++;
00572         }
00573         else {
00574             while ( FG.cTable[*s] <= 1 ) s++;
00575             if ( s > in && *s == '[' && s[1] == ']' ) s += 2;
00576         }
00577         /*
00578             The next statement is rather mysterious
00579             It is undone in DoPrint and CoMultiply, but it also causes effects
00580             in other (wrong) statements like dimension -4; or Trace4 -1;
00581             The code in pre.c (LoadStatement) has been changed 8-sep-2009
00582             to force a comma after the keyword. This means that the
00583             'mysterious' line is automatically inactive. Hence it is taken out.
00584
00585             if ( *s == '+' || *s == '-' ) s++;
00586         */
00587         if ( *s == ',' ) s++;
00588     }
00589     /*
00590         First the test on the order of the statements.
00591         This is relatively new (2.2c) and may cause some problems with old

```

```

00592     programs. Hence the first error message should explain!
00593  */
00594     if ( AP.PreAssignFlag == 0 && AM.OldOrderFlag == 0 ) {
00595         if ( AP.PreInsideLevel ) {
00596             if ( k->type != STATEMENT && k->type != MIXED ) {
00597                 MesPrint("%Only executable and print statements are allowed in an %#inside/%#endinside
construction");
00598                 return(-1);
00599             }
00600         }
00601         else {
00602             if ( ( AC.compiletype == DECLARATION || AC.compiletype == SPECIFICATION )
00603                 && ( k->type == STATEMENT || k->type == DEFINITION || k->type == TOOOUTPUT ) ) {
00604                 if ( AC.tablecheck == 0 ) {
00605                     AC.tablecheck = 1;
00606                     if ( TestTables() ) error1 = 1;
00607                 }
00608             }
00609             if ( k->type == MIXED ) {
00610                 if ( AC.compiletype <= DEFINITION ) {
00611                     AC.compiletype = STATEMENT;
00612                 }
00613             }
00614             else if ( k->type == MIXED2 ) {}
00615             else if ( k->type > AC.compiletype ) {
00616                 if ( StrCmp((UBYTE *) (k->name), (UBYTE *) "format") != 0 )
00617                     AC.compiletype = k->type;
00618             }
00619             else if ( k->type < AC.compiletype ) {
00620                 switch ( k->type ) {
00621                     case DECLARATION:
00622                         MesPrint("&Declaration out of order");
00623                         MesPrint("& %s",in);
00624                         break;
00625                     case DEFINITION:
00626                         MesPrint("&Definition out of order");
00627                         MesPrint("& %s",in);
00628                         break;
00629                     case SPECIFICATION:
00630                         MesPrint("&Specification out of order");
00631                         MesPrint("& %s",in);
00632                         break;
00633                     case STATEMENT:
00634                         MesPrint("&Statement out of order");
00635                         break;
00636                     case TOOOUTPUT:
00637                         MesPrint("&Output control statement out of order");
00638                         MesPrint("& %s",in);
00639                         break;
00640                 }
00641                 AC.compiletype = k->type;
00642                 if ( AC.firstctypemessage == 0 ) {
00643                     MesPrint("&Proper order inside a module is:");
00644                     MesPrint("Declarations, specifications, definitions, statements, output control
statements");
00645                     AC.firstctypemessage = 1;
00646                 }
00647                 error1 = 1;
00648             }
00649         }
00650     }
00651  /*
00652     Now we execute the tests that are prescribed by the flags.
00653  */
00654     if ( AC.AutoDeclareFlag && ( ( k->flags & WITHAUTO ) == 0 ) ) {
00655         MesPrint("&Illegal type of auto-declaration");
00656         return(1);
00657     }
00658     if ( ( ( k->flags & PARTEST ) != 0 ) && ParenthesesTest(s) ) return(1);
00659     error2 = (*k->func)(s);
00660     if ( error2 == 0 ) return(error1);
00661     return(error2);
00662 }
00663
00664  /*
00665     #] CompileStatement :
00666     #[ TestTables :
00667  */
00668
00669 int TestTables(void)
00670 {
00671     FUNCTIONS f = functions;
00672     TABLES t;
00673     WORD j;
00674     int error = 0, i;
00675     LONG x;
00676     i = NumFunctions + FUNCTION - MAXBUILTINFUNCTION - 1;

```



```

00677     f = f + MAXBUILTINFUNCTION - FUNCTION + 1;
00678     if ( AC.MustTestTable > 0 ) {
00679         while ( i > 0 ) {
00680             if ( ( t = f->tabl ) != 0 && t->strict > 0 && !t->sparse ) {
00681                 for ( x = 0, j = 0; x < t->totind; x++ ) {
00682                     if ( t->tablepointers[TABLEEXTENSION*x] < 0 ) j++;
00683                 }
00684                 if ( j > 0 ) {
00685                     if ( j > 1 ) {
00686                         MesPrint("&In table %s there are %d unfilled elements",
00687                             AC.varnames->namebuffer+f->name, j);
00688                     }
00689                     else {
00690                         MesPrint("&In table %s there is one unfilled element",
00691                             AC.varnames->namebuffer+f->name);
00692                     }
00693                     error = 1;
00694                 }
00695             }
00696             i--; f++;
00697         }
00698         AC.MustTestTable--;
00699     }
00700     return(error);
00701 }
00702
00703 /*
00704     #] TestTables :
00705     #[ CompileSubExpressions :
00706
00707     Now we attack the subexpressions from inside out.
00708     We try to see whether we had any of them already.
00709     We have to worry about adding the wildcard sum parameter
00710     to the prototype.
00711 */
00712
00713 int CompileSubExpressions(SBYTE *tokens)
00714 {
00715     GETIDENTITY
00716     SBYTE *fill = tokens, *s = tokens, *t;
00717     WORD number[MAXNUMSIZE], *oldwork, *w1, *w2;
00718     int level, num, i, sumlevel = 0, sumtype = SYMTOSYM;
00719     int retval, error = 0;
00720
00721     /* Eliminate all subexpressions. They are marked by LPARENTHESIS, RPARENTHESIS */
00722
00723     AC.CompileLevel++;
00724     while ( *s != TENDOFIT ) {
00725         if ( *s == TFUNOPEN ) {
00726             if ( fill < s ) *fill = TENDOFIT;
00727             t = fill - 1;
00728             while ( t >= tokens && t[0] >= 0 ) t--;
00729             if ( t >= tokens && *t == TFUNCTION ) {
00730                 t++; i = 0; while ( *t >= 0 ) i = 128*i + *t++;
00731                 if ( i == AM.sumnum || i == AM.sumpnum ) {
00732                     t = s + 1;
00733                     if ( *t == TSYMBOL || *t == TINDEX ) {
00734                         t++; i = 0; while ( *t >= 0 ) i = 128*i + *t++;
00735                         if ( s[1] == TINDEX ) {
00736                             i += AM.OffsetIndex;
00737                             sumtype = INDTOIND;
00738                         }
00739                         else sumtype = SYMTOSYM;
00740                         sumlevel = i;
00741                     }
00742                 }
00743             }
00744             *fill++ = *s++;
00745         }
00746         else if ( *s == TFUNCLOSE ) { sumlevel = 0; *fill++ = *s++; }
00747         else if ( *s == LPARENTHESIS ) {
00748             /*
00749                 We must make an exception here.
00750                 If the subexpression is just an integer, whatever its length,
00751                 we should try to keep it.
00752                 This is important when we have a function with an integer
00753                 argument. In particular this is relevant for the MZV program.
00754             */
00755             t = s; level = 0;
00756             while ( level >= 0 ) {
00757                 s++;
00758                 if ( *s == LPARENTHESIS ) level++;
00759                 else if ( *s == RPARENTHESIS ) level--;
00760                 else if ( *s == TENDOFIT ) {
00761                     MesPrint("&Unbalanced subexpression parentheses");
00762                     return(-1);
00763                 }

```

```

00764         }
00765         t++; *s = TENDOFIT;
00766         if ( sumlevel > 0 ) { /* Inside sum. Add wildcard to prototype */
00767             oldwork = w1 = AT.WorkPointer;
00768             w2 = AC.ProtoType;
00769             i = w2[1];
00770             while ( --i >= 0 ) *w1++ = *w2++;
00771             oldwork[1] += 4;
00772             *w1++ = sumtype; *w1++ = 4; *w1++ = sumlevel; *w1++ = sumlevel;
00773             w2 = AC.ProtoType; AT.WorkPointer = w1;
00774             AC.ProtoType = oldwork;
00775             num = CompileSubExpressions(t);
00776             AC.ProtoType = w2; AT.WorkPointer = oldwork;
00777         }
00778         else num = CompileSubExpressions(t);
00779         if ( num < 0 ) return(-1);
00780     /*
00781         Note that the subexpression code should always fit.
00782         We had two parentheses and at least two bytes contents.
00783         There cannot be more than 2^21 subexpressions or we get outside
00784         this minimum. Ignoring this might lead to really rare and
00785         hard to find errors, years from now.
00786     */
00787         if ( insubexpbuffers >= MAXSUBEXPRESSIONS ) {
00788             MesPrint("&More than %d subexpressions inside one
expression", (WORD)MAXSUBEXPRESSIONS);
00789             Terminate(-1);
00790         }
00791         if ( subexpbuffers+insubexpbuffers >= topsubexpbuffers ) {
00792             DoubleBuffer((void **)((void *)(&subexpbuffers))
00793                 , (void **)((void *)(&topsubexpbuffers)), sizeof(SUBBUF), "subexpbuffers");
00794         }
00795         subexpbuffers[insubexpbuffers].subexpnum = num;
00796         subexpbuffers[insubexpbuffers].buffernum = AC.cbufnum;
00797         num = insubexpbuffers++;
00798         *fill++ = TSUBEXP;
00799         i = 0;
00800         do { number[i++] = num & 0x7F; num >= 7; } while ( num );
00801         while ( --i >= 0 ) *fill++ = (SBYTE)(number[i]);
00802         s++;
00803     }
00804     else if ( *s == EMPTY ) s++;
00805     else *fill++ = *s++;
00806 }
00807 *fill = TENDOFIT;
00808 /*
00809     At this stage there are no more subexpressions.
00810     Hence we can do the basic compilation.
00811 */
00812     if ( AC.CompileLevel == 1 && AC.ToBeInFactors ) {
00813         error = CodeFactors(tokens);
00814     }
00815     AC.CompileLevel--;
00816     retval = CodeGenerator(tokens);
00817     if ( error < 0 ) return(error);
00818     return(retval);
00819 }
00820
00821 /*
00822     #] CompileSubExpressions :
00823     #[ CodeGenerator :
00824
00825     This routine does the real code generation.
00826     It returns the number of the rhs subexpression.
00827     At this point we do not have to worry about subexpressions,
00828     sets, setelements, simple vs complicated function arguments
00829     simple vs complicated powers etc.
00830
00831     The variable 'first' indicates whether we are starting a new term
00832
00833     The major complication are the set elements of type set[n].
00834     We have marked them as TSETNUM,n,Ttype,setnum
00835     They go into
00836     SETSET,size,subterm,relocation list
00837     in which the subterm should be ready to become a regular
00838     subterm in which the sets have been replaced by their element
00839     The relocation list consists of pairs of numbers:
00840     1: offset in the subterm, 2: the symbol n.
00841     Note that such a subterm can be a whole function with its arguments.
00842     We use the variable inset to indicate that we have something going.
00843     The relocation list is collected in the top of the Workspace.
00844 */
00845
00846 static UWORD *CGscrat7 = 0;
00847
00848 int CodeGenerator(SBYTE *tokens)
00849 {

```

```

00850     GETIDENTITY
00851     SBYTE *s = tokens, c;
00852     int i, sign = 1, first = 1, deno = 1, error = 0, minus, n, needarg, numexp, cc;
00853     int base, sumlevel = 0, sumtype = SYMTOSYM, firstsumarg, inset = 0;
00854     int funflag = 0, settype, x1, x2, mulflag = 0;
00855     WORD *t, *v, *r, *term, nnumerator, ndenominator, *oldwork, x3, y, nin;
00856     WORD *w1, *w2, *tsize = 0, *relo = 0;
00857     UWORD *numerator, *denominator, *innum;
00858     CBUF *C;
00859     POSITION position;
00860     WORD TmpProto[SUBEXPSIZE];
00861 /*
00862 #ifdef WITHPTHREADS
00863     RENUMBER renumber;
00864 #endif
00865 */
00866     RENUMBER renumber;
00867     if ( AC.TokensWriteFlag ) WriteTokens(tokens);
00868     if ( CGscrat7 == 0 )
00869         CGscrat7 = (UWORD *)Malloc1((AM.MaxTal+2)*sizeof(WORD), "CodeGenerator");
00870     AddrRHS(AC.cbufnum, 0);
00871     C = cbuf + AC.cbufnum;
00872     numexp = C->numrhs;
00873     C->NumTerms[numexp] = 0;
00874     C->numdum[numexp] = 0;
00875     oldwork = AT.WorkPointer;
00876     numerator = (UWORD *) (AT.WorkPointer);
00877     denominator = numerator + 2*AM.MaxTal;
00878     innum = denominator + 2*AM.MaxTal;
00879     term = (WORD *) (innum + 2*AM.MaxTal);
00880     AT.WorkPointer = term + AM.MaxTer/sizeof(WORD);
00881     if ( AT.WorkPointer > AT.WorkTop ) goto OverWork;
00882     cc = 0;
00883     t = term+1;
00884     numerator[0] = denominator[0] = 1;
00885     nnumerator = ndenominator = 1;
00886     while ( *s != TENDOFIT ) {
00887         if ( *s == TPLUS || *s == TMINUS ) {
00888             if ( first || mulflag ) { if ( *s == TMINUS ) sign = -sign; }
00889             else {
00890                 *term = t-term;
00891                 C->NumTerms[numexp]++;
00892                 if ( cc && sign ) C->CanCommu[numexp]++;
00893                 CompleteTerm(term, numerator, denominator, nnumerator, ndenominator, sign);
00894                 first = 1; cc = 0; t = term + 1; deno = 1;
00895                 numerator[0] = denominator[0] = 1;
00896                 nnumerator = ndenominator = 1;
00897                 if ( *s == TMINUS ) sign = -1;
00898                 else sign = 1;
00899             }
00900             s++;
00901         }
00902         else {
00903             mulflag = first = 0; c = *s++;
00904             switch ( c ) {
00905                 case TSYMBOL:
00906                     x1 = 0; while ( *s >= 0 ) { x1 = x1*128 + *s++; }
00907                     if ( *s == TWILDCARD ) { s++; x1 += 2*MAXPOWER; }
00908                     *t++ = SYMBOL; *t++ = 4; *t++ = x1;
00909                     if ( inset ) *relo = 2;
00910 TryPower:
00911                     if ( *s == TPOWER ) {
00912                         s++;
00913                         if ( *s == TMINUS ) { s++; deno = -deno; }
00914                         c = *s++;
00915                         base = ( c == TNUMBER ) ? 100: 128;
00916                         x2 = 0; while ( *s >= 0 ) { x2 = base*x2 + *s++; }
00917                         if ( c == TSYMBOL ) {
00918                             if ( *s == TWILDCARD ) s++;
00919                             x2 += 2*MAXPOWER;
00920                         }
00921                         *t++ = deno*x2;
00922                     }
00923                     else *t++ = deno;
00924                     deno = 1;
00925                     if ( inset ) {
00926                         while ( relo < AT.WorkTop ) *t++ = *relo++;
00927                         inset = 0; tsize[1] = t - tsize;
00928                     }
00929                     break;
00930                 case TINDEX:
00931                     x1 = 0; while ( *s >= 0 ) { x1 = x1*128 + *s++; }
00932                     *t++ = INDEX; *t++ = 3;
00933                     if ( *s == TWILDCARD ) { s++; x1 += WILDOFFSET; }
00934                     if ( inset ) { *t++ = x1; *relo = 2; }
00935                     else *t++ = x1 + AM.OffsetIndex;
00936                     if ( t[-1] > AM.IndDum ) {
00937                         x1 = t[-1] - AM.IndDum;

```

```

00937         if ( x1 > C->numdum[numexp] ) C->numdum[numexp] = x1;
00938     }
00939     goto fin;
00940 case TGENINDEX:
00941     *t++ = INDEX; *t++ = 3; *t++ = AC.DumNum+WILDOFFSET;
00942     deno = 1;
00943     break;
00944 case TVECTOR:
00945     x1 = 0; while ( *s >= 0 ) { x1 = x1*128 + *s++; }
00946 dovector:
00947     if ( inset == 0 ) x1 += AM.OffsetVector;
00948     if ( *s == TWILDCARD ) { s++; x1 += WILDOFFSET; }
00949     if ( inset ) *relo = 2;
00950     if ( *s == TDOT ) { /* DotProduct ? */
00951         s++;
00952         if ( *s == TSETNUM || *s == TSETDOL ) {
00953             settype = ( *s == TSETDOL );
00954             s++; x2 = 0; while ( *s >= 0 ) { x2 = x2*128 + *s++; }
00955             if ( settype ) x2 = -x2;
00956             if ( inset == 0 ) {
00957                 tsize = t; *t++ = SETSET; *t++ = 0;
00958                 relo = AT.WorkTop;
00959             }
00960             inset += 2;
00961             *--relo = x2; *--relo = 3;
00962         }
00963         if ( *s != TVECTOR && *s != TDUBIOUS ) {
00964             MesPrint("&Illegally formed dotproduct");
00965             error = 1;
00966         }
00967         s++; x2 = 0; while ( *s >= 0 ) { x2 = x2*128 + *s++; }
00968         if ( inset < 2 ) x2 += AM.OffsetVector;
00969         if ( *s == TWILDCARD ) { s++; x2 += WILDOFFSET; }
00970         *t++ = DOTPRODUCT; *t++ = 5; *t++ = x1; *t++ = x2;
00971         goto TryPower;
00972     }
00973     else if ( *s == TFUNOPEN ) {
00974         s++;
00975         if ( *s == TSETNUM || *s == TSETDOL ) {
00976             settype = ( *s == TSETDOL );
00977             s++; x2 = 0; while ( *s >= 0 ) { x2 = x2*128 + *s++; }
00978             if ( settype ) x2 = -x2;
00979             if ( inset == 0 ) {
00980                 tsize = t; *t++ = SETSET; *t++ = 0;
00981                 relo = AT.WorkTop;
00982             }
00983             inset += 2;
00984             *--relo = x2; *--relo = 3;
00985         }
00986         if ( *s == TINDEX || *s == TDUBIOUS ) {
00987             s++;
00988             x2 = 0; while ( *s >= 0 ) { x2 = x2*128 + *s++; }
00989             if ( inset < 2 ) x2 += AM.OffsetIndex;
00990             if ( *s == TWILDCARD ) { s++; x2 += WILDOFFSET; }
00991             *t++ = VECTOR; *t++ = 4; *t++ = x1; *t++ = x2;
00992             if ( t[-1] > AM.IndDum ) {
00993                 x2 = t[-1] - AM.IndDum;
00994                 if ( x2 > C->numdum[numexp] ) C->numdum[numexp] = x2;
00995             }
00996         }
00997     }
00998     else if ( *s == TGENINDEX ) {
00999         *t++ = VECTOR; *t++ = 4; *t++ = x1;
01000         *t++ = AC.DumNum + WILDOFFSET;
01001     }
01002     else if ( *s == TNUMBER || *s == TNUMBER1 ) {
01003         base = ( *s == TNUMBER ) ? 100: 128;
01004         s++;
01005         x2 = 0; while ( *s >= 0 ) { x2 = x2*base + *s++; }
01006         if ( x2 >= AM.OffsetIndex && inset < 2 ) {
01007             MesPrint("&Fixed index in vector greater than %d",
01008                 AM.OffsetIndex);
01009             return(-1);
01010         }
01011         *t++ = VECTOR; *t++ = 4; *t++ = x1; *t++ = x2;
01012     }
01013     else if ( *s == TVECTOR || ( *s == TMINUS && s[1] == TVECTOR ) ) {
01014         if ( *s == TMINUS ) { s++; sign = -sign; }
01015         s++;
01016         x2 = 0; while ( *s >= 0 ) { x2 = x2*128 + *s++; }
01017         if ( inset < 2 ) x2 += AM.OffsetVector;
01018         if ( *s == TWILDCARD ) { s++; x2 += WILDOFFSET; }
01019         *t++ = DOTPRODUCT; *t++ = 5; *t++ = x1; *t++ = x2; *t++ = deno;
01020     }
01021     else {
01022         MesPrint("&Illegal argument for vector");
01023         return(-1);
01024     }
01025     if ( *s != TFUNCLOSE ) {

```

```

01024         MesPrint("&Illegal argument for vector");
01025         return(-1);
01026     }
01027     s++;
01028 }
01029 else {
01030     if ( AC.DumNum ) {
01031         *t++ = VECTOR; *t++ = 4; *t++ = x1;
01032         *t++ = AC.DumNum + WILDOffset;
01033     }
01034     else {
01035         *t++ = INDEX; *t++ = 3; *t++ = x1;
01036     }
01037 }
01038 goto fin;
01039 case TDELTA:
01040     if ( *s != TFUNOPEN ) {
01041         MesPrint("&d_ needs two arguments");
01042         error = -1;
01043     }
01044     v = t; *t++ = DELTA; *t++ = 4;
01045     needarg = 2; x3 = x1 = -1;
01046     goto dotensor;
01047 case TFUNCTION:
01048     x1 = 0; while ( *s >= 0 ) { x1 = x1*128 + *s++; }
01049     if ( x1 == AM.sumnum || x1 == AM.sumpnum ) sumlevel = x1;
01050     x1 += FUNCTION;
01051     if ( x1 == FIRSTBRACKET ) {
01052         if ( s[0] == TFUNOPEN && s[1] == TEXPRESSION ) {
01053             doexpr:
01054                 s += 2;
01055                 *t++ = x1; *t++ = FUNHEAD+2; *t++ = 0;
01056                 if ( x1 == AR.PolyFun && AR.PolyFunType == 2 && AR.Eside != LHSIDE )
01057                     t[-1] |= MUSTCLEANPRF;
01058                 FILLFUN3(t)
01059                 x2 = 0; while ( *s >= 0 ) { x2 = x2*128 + *s++; }
01060                 *t++ = -EXPRESSION; *t++ = x2;
01061             /*
01062             The next code is added to facilitate parallel processing
01063             We need to call GetTable here to make sure all processors
01064             have the same numbering of all variables.
01065             */
01066             if ( Expressions[x2].status == STOREDEXPRESSION ) {
01067                 TMproto[0] = EXPRESSION;
01068                 TMproto[1] = SUBEXPSIZE;
01069                 TMproto[2] = x2;
01070                 TMproto[3] = 1;
01071                 { int ie; for ( ie = 4; ie < SUBEXPSIZE; ie++ ) TMproto[ie] = 0; }
01072                 AT.TMaddr = TMproto;
01073                 PUTZERO(position);
01074             }
01075             if ( (
01076                 #ifndef WITHPTHREADS
01077                 renumber =
01078                     GetTable(x2,&position,0) == 0 ) {
01079                 error = 1;
01080                 MesPrint("&Problems getting information about stored expression %s(1) "
01081                     ,EXPRNAME(x2));
01082             }
01083             #ifndef WITHPTHREADS
01084             M_free(renumber->symb.lo,"VarSpace");
01085             M_free(renumber,"Renummer");
01086             #endif
01087             /*
01088             if ( ( renumber = GetTable(x2,&position,0) == 0 ) {
01089                 error = 1;
01090                 MesPrint("&Problems getting information about stored expression %s(1) "
01091                     ,EXPRNAME(x2));
01092             }
01093             if ( renumber->symb.lo != AN.dummyrenumlist )
01094                 M_free(renumber->symb.lo,"VarSpace");
01095             M_free(renumber,"Renummer");
01096             AR.StoreData.dirtyflag = 1;
01097         }
01098     if ( *s != TFUNCLOSE ) {
01099         if ( x1 == FIRSTBRACKET )
01100             MesPrint("&Problems with argument of FirstBracket_");
01101         else if ( x1 == FIRSTTERM )
01102             MesPrint("&Problems with argument of FirstTerm_");
01103         else if ( x1 == CONTENTTERM )
01104             MesPrint("&Problems with argument of FirstTerm_");
01105         else if ( x1 == TERMSINEXPR )
01106             MesPrint("&Problems with argument of TermsIn_");
01107         else if ( x1 == SIZEOFFUNCTION )
01108             MesPrint("&Problems with argument of SizeOf_");
01109         else if ( x1 == NUMFACTORS )
01110             MesPrint("&Problems with argument of NumFactors_");

```

```

01111         else
01112             MesPrint("&Problems with argument of FactorIn_");
01113             error = 1;
01114             while ( *s != TENDOFIT && *s != TFUNCLOSE ) s++;
01115         }
01116         if ( *s == TFUNCLOSE ) s++;
01117         goto fin;
01118     }
01119 }
01120 else if ( x1 == TERMSINEXPR || x1 == SIZEOFFUNCTION || x1 == FACTORIN
01121 || x1 == NUMFACTORS || x1 == FIRSTTERM || x1 == CONTENTTERM ) {
01122     if ( s[0] == TFUNOPEN && s[1] == TEXPRESSION ) goto doexpr;
01123     if ( s[0] == TFUNOPEN && s[1] == TDOLLAR ) {
01124         s += 2;
01125         *t++ = x1; *t++ = FUNHEAD+2; *t++ = 0;
01126         FILLFUN3(t)
01127         x2 = 0; while ( *s >= 0 ) { x2 = x2*128 + *s++; }
01128         *t++ = -DOLLAREXPRESSION; *t++ = x2;
01129         if ( *s != TFUNCLOSE ) {
01130             if ( x1 == TERMSINEXPR )
01131                 MesPrint("&Problems with argument of TermsIn_");
01132             else if ( x1 == SIZEOFFUNCTION )
01133                 MesPrint("&Problems with argument of SizeOf_");
01134             else if ( x1 == NUMFACTORS )
01135                 MesPrint("&Problems with argument of NumFactors_");
01136             else
01137                 MesPrint("&Problems with argument of FactorIn_");
01138             error = 1;
01139             while ( *s != TENDOFIT && *s != TFUNCLOSE ) s++;
01140         }
01141         if ( *s == TFUNCLOSE ) s++;
01142         goto fin;
01143     }
01144 }
01145 x3 = x1;
01146 if ( inset && ( t-tsize == 2 ) ) x1 -= FUNCTION;
01147 if ( *s == TWILDCARD ) { x1 += WILDOFFSET; s++; }
01148 if ( functions[x3-FUNCTION].commute ) cc = 1;
01149 if ( *s != TFUNOPEN ) {
01150     *t++ = x1; *t++ = FUNHEAD; *t++ = 0;
01151     if ( x1 == AR.PolyFun && AR.PolyFunType == 2 && AR.Eside != LHSIDE )
01152         t[-1] |= MUSTCLEANPRF;
01153     FILLFUN3(t) sumlevel = 0; goto fin;
01154 }
01155 v = t; *t++ = x1; *t++ = FUNHEAD; *t++ = DIRTYFLAG;
01156 if ( x1 == AR.PolyFun && AR.PolyFunType == 2 && AR.Eside != LHSIDE )
01157     t[-1] |= MUSTCLEANPRF;
01158 FILLFUN3(t)
01159 needarg = -1;
01160 if ( !inset && functions[x3-FUNCTION].spec >= TENSORFUNCTION ) {
01161     dotensor:
01162     do {
01163         if ( needarg == 0 ) {
01164             if ( x1 >= 0 ) {
01165                 x3 = x1;
01166                 if ( x3 >= FUNCTION+WILDOFFSET ) x3 -= WILDOFFSET;
01167                 MesPrint("&Too many arguments in function %s",
01168                     VARNAME(functions, (x3-FUNCTION)) );
01169             }
01170             else
01171                 MesPrint("&d_ needs exactly two arguments");
01172             error = -1;
01173             needarg--;
01174         }
01175         else if ( needarg > 0 ) needarg--;
01176         s++;
01177         c = *s++;
01178         if ( c == TMINUS && *s == TVECTOR ) { sign = -sign; c = *s++; }
01179         base = ( c == TNUMBER ) ? 100: 128;
01180         x2 = 0; while ( *s >= 0 ) { x2 = base*x2 + *s++; }
01181         if ( *s == TWILDCARD && c != TNUMBER ) { x2 += WILDOFFSET; s++; }
01182         if ( c == TSETNUM || c == TSETDOL ) {
01183             if ( c == TSETDOL ) x2 = -x2;
01184             if ( inset == 0 ) {
01185                 w1 = t; t += 2; w2 = t;
01186                 while ( w1 > v ) *--w2 = *--w1;
01187                 tsize = v; relo = AT.WorkTop;
01188                 *v++ = SETSET; *v++ = 0;
01189             }
01190             inset = 2; *--relo = x2; *--relo = t - v;
01191             c = *s++;
01192             x2 = 0; while ( *s >= 0 ) x2 = 128*x2 + *s++;
01193             switch ( c ) {
01194                 case TINDEX:
01195                     *t++ = x2;
01196                     if ( t[-1]+AM.OffsetIndex > AM.IndDum ) {
01197                         x2 = t[-1]+AM.OffsetIndex - AM.IndDum;

```

```

01198             if ( x2 > C->numdum[numexp] ) C->numdum[numexp] = x2;
01199         }
01200         break;
01201     case TVECTOR:
01202         *t++ = x2; break;
01203     case TNUMBER1:
01204         if ( x2 >= 0 && x2 < AM.OffsetIndex ) {
01205             *t++ = x2; break;
01206         }
01207         /* fall through */
01208     default:
01209         MesPrint("&Illegal type of set inside tensor");
01210         error = 1;
01211         *t++ = x2;
01212         break;
01213     }
01214 }
01215 else { switch ( c ) {
01216     case TINDEX:
01217         if ( inset < 2 ) *t++ = x2 + AM.OffsetIndex;
01218         else *t++ = x2;
01219         if ( x2+AM.OffsetIndex > AM.IndDum ) {
01220             x2 = x2+AM.OffsetIndex - AM.IndDum;
01221             if ( x2 > C->numdum[numexp] ) C->numdum[numexp] = x2;
01222         }
01223         break;
01224     case TGENINDEX:
01225         *t++ = AC.DumNum + WILDOFFSET;
01226         break;
01227     case TVECTOR:
01228         if ( inset < 2 ) *t++ = x2 + AM.OffsetVector;
01229         else *t++ = x2;
01230         break;
01231     case TWILDARG:
01232         *t++ = FUNNYWILD; *t++ = x2;
01233         /* v[2] = 0; */
01234         break;
01235     case TDOLLAR:
01236         *t++ = FUNNYDOLLAR; *t++ = x2;
01237         break;
01238     case TDUBIOUS:
01239         if ( inset < 2 ) *t++ = x2 + AM.OffsetVector;
01240         else *t++ = x2;
01241         break;
01242     case TSGAMMA: /* Special gamma's */
01243         if ( x3 != GAMMA ) {
01244             MesPrint("&5_,6_,7_ can only be used inside g_");
01245             error = -1;
01246         }
01247         *t++ = -x2;
01248         break;
01249     case TNUMBER:
01250     case TNUMBER1:
01251         if ( x2 >= AM.OffsetIndex && inset < 2 ) {
01252             MesPrint("&Value of constant index in tensor too large");
01253             error = -1;
01254         }
01255         *t++ = x2;
01256         break;
01257     default:
01258         MesPrint("&Illegal object in tensor");
01259         error = -1;
01260         break;
01261     }}
01262     if ( inset >= 2 ) inset = 1;
01263 } while ( *s == TCOMMA );
01264 }
01265 else {
01266 dofunction: firstsumarg = 1;
01267 do {
01268     unsigned int ux2;
01269     s++;
01270     c = *s++;
01271     if ( c == TMINUS && ( *s == TVECTOR || *s == TNUMBER
01272 || *s == TNUMBER1 || *s == TSUBEXP ) ) {
01273         minus = 1; c = *s++;
01274     }
01275     else minus = 0;
01276     base = ( c == TNUMBER ) ? 100: 128;
01277     ux2 = 0; while ( *s >= 0 ) { ux2 = base+ux2 + *s++; }
01278     x2 = ux2; /* may cause an implementation-defined behaviour */
01279 } /*
01280 !!!!!!!! What if it does not fit?
01281 */
01282 if ( firstsumarg ) {
01283     firstsumarg = 0;
01284     if ( sumlevel > 0 ) {

```

```

01285         if ( c == TSYMBOL ) {
01286             sumlevel = x2; sumtype = SYMTOSYM;
01287         }
01288     else if ( c == TINDEX ) {
01289         sumlevel = x2+AM.OffsetIndex; sumtype = INDTOIND;
01290         if ( sumlevel > AM.IndDum ) {
01291             x2 = sumlevel - AM.IndDum;
01292             if ( x2 > C->numdum[numexp] ) C->numdum[numexp] = x2;
01293         }
01294     }
01295 }
01296 }
01297 if ( *s == TWILDCARD ) {
01298     if ( c == TSYMBOL ) x2 += 2*MAXPOWER;
01299     else if ( c != TNUMBER ) x2 += WILDOFFSET;
01300     s++;
01301 }
01302 switch ( c ) {
01303     case TSYMBOL:
01304         *t++ = -SYMBOL; *t++ = x2; break;
01305     case TDOLLAR:
01306         *t++ = -DOLLEXPRESSION; *t++ = x2; break;
01307     case TEXPRESS:
01308         *t++ = -EXPRESSION; *t++ = x2;
01309 /*
01310     The next code is added to facilitate parallel processing
01311     We need to call GetTable here to make sure all processors
01312     have the same numbering of all variables.
01313 */
01314     if ( Expressions[x2].status == STOREDEXPRESSION ) {
01315         TMproto[0] = EXPRESSION;
01316         TMproto[1] = SUBEXPSIZE;
01317         TMproto[2] = x2;
01318         TMproto[3] = 1;
01319         { int ie; for ( ie = 4; ie < SUBEXPSIZE; ie++ ) TMproto[ie] = 0; }
01320         AT.TMaddr = TMproto;
01321         PUTZERO(position);
01322 /*
01323         if ( (
01324             #ifndef WITHPTHREADS
01325             renumber =
01326             #endif
01327             GetTable(x2,&position,0) ) == 0 ) {
01328                 error = 1;
01329                 MesPrint("&Problems getting information about stored
01330                     ,EXPRNAME(x2));
01331             }
01332             #ifndef WITHPTHREADS
01333             M_free(renumber->symb.lo,"VarSpace");
01334             M_free(renumber,"Renumber");
01335             #endif
01336 */
01337         if ( ( renumber = GetTable(x2,&position,0) ) == 0 ) {
01338             error = 1;
01339             MesPrint("&Problems getting information about stored
01340                 ,EXPRNAME(x2));
01341         }
01342         if ( renumber->symb.lo != AN.dummyrenumlist )
01343             M_free(renumber->symb.lo,"VarSpace");
01344         M_free(renumber,"Renumber");
01345         AR.StoreData.dirtyflag = 1;
01346     }
01347     break;
01348 case TINDEX:
01349     *t++ = -INDEX; *t++ = x2 + AM.OffsetIndex;
01350     if ( t[-1] > AM.IndDum ) {
01351         x2 = t[-1] - AM.IndDum;
01352         if ( x2 > C->numdum[numexp] ) C->numdum[numexp] = x2;
01353     }
01354     break;
01355 case TGENINDEX:
01356     *t++ = -INDEX; *t++ = AC.DumNum + WILDOFFSET;
01357     break;
01358 case TVECTOR:
01359     if ( minus ) *t++ = -MINVECTOR;
01360     else *t++ = -VECTOR;
01361     *t++ = x2 + AM.OffsetVector;
01362     break;
01363 case TSGAMMA: /* Special gamma's */
01364     MesPrint("&5_,6_,7_ can only be used inside g_");
01365     error = -1;
01366     *t++ = -INDEX;
01367     *t++ = -x2;
01368     break;
01369 case TDUBIOUS:

```



```

01370         *t++ = -SYMBOL; *t++ = x2; break;
01371     case TFUNCTION:
01372         *t++ = -x2-FUNCTION;
01373         break;
01374     case TSET:
01375         *t++ = -SETSET;
01376         *t++ = x2;
01377         break;
01378     case TWILDARG:
01379         *t++ = -ARGWILD; *t++ = x2; break;
01380     case TSETDOL:
01381         x2 = -x2;
01382         /* fall through */
01383     case TSETNUM:
01384         if ( inset == 0 ) {
01385             w1 = t; t += 2; w2 = t;
01386             while ( w1 > v ) *--w2 = *--w1;
01387             tsize = v; relo = AT.WorkTop;
01388             *v++ = SETSET; *v++ = 0;
01389             inset = 1;
01390         }
01391         *--relo = x2; *--relo = t-v+1;
01392         c = *s++;
01393         x2 = 0; while ( *s >= 0 ) x2 = 128*x2 + *s++;
01394         switch ( c ) {
01395             case TFUNCTION:
01396                 (*relo)--; *t++ = -x2-1; break;
01397             case TSYMBOL:
01398                 *t++ = -SYMBOL; *t++ = x2; break;
01399             case TINDEX:
01400                 *t++ = -INDEX; *t++ = x2;
01401                 if ( x2+AM.OffsetIndex > AM.IndDum ) {
01402                     x2 = x2+AM.OffsetIndex - AM.IndDum;
01403                     if ( x2 > C->numdum[numexp] ) C->numdum[numexp] = x2;
01404                 }
01405                 break;
01406             case TVECTOR:
01407                 *t++ = -VECTOR; *t++ = x2; break;
01408             case TNUMBER1:
01409                 *t++ = -SNUMBER; *t++ = x2; break;
01410             default:
01411                 MesPrint("&Internal error 435");
01412                 error = 1;
01413                 *t++ = -SYMBOL; *t++ = x2; break;
01414         }
01415         break;
01416     case TSUBEXP:
01417         w2 = AC.ProtoType; i = w2[1];
01418         w1 = t;
01419         *t++ = i+ARGHEAD+4;
01420         *t++ = 1;
01421         FILLARG(t);
01422         *t++ = i + 4;
01423         while ( --i >= 0 ) *t++ = *w2++;
01424         w1[ARGHEAD+3] = subexpbuffers[x2].subexpnum;
01425         w1[ARGHEAD+5] = subexpbuffers[x2].buffernum;
01426         if ( sumlevel > 0 ) {
01427             w1[0] += 4;
01428             w1[ARGHEAD] += 4;
01429             w1[ARGHEAD+2] += 4;
01430             *t++ = sumtype; *t++ = 4;
01431             *t++ = sumlevel; *t++ = sumlevel;
01432         }
01433         *t++ = 1; *t++ = 1;
01434         if ( minus ) *t++ = -3;
01435         else *t++ = 3;
01436         break;
01437     case TNUMBER:
01438     case TNUMBER1:
01439         if ( minus ) x2 = UnsignedToInt(-IntAbs(x2));
01440         *t++ = -SNUMBER;
01441         *t++ = x2;
01442         break;
01443     default:
01444         MesPrint("&Illegal object in function");
01445         error = -1;
01446         break;
01447     }
01448     } while ( *s == TCOMMA );
01449 }
01450 if ( *s != TFUNCLOSE ) {
01451     MesPrint("&Illegal argument field for function. Expected ");
01452     return(-1);
01453 }
01454 s++; sumlevel = 0;
01455 v[1] = t-v;
01456 /*

```

```

01457         if ( *v == AM.termfunnum && ( v[1] != FUNHEAD+2 ||
01458         v[FUNHEAD] != -DOLLAREXPRESSION ) ) {
01459             MesPrint("&The function term_ can only have one argument with a single
    $-expression");
01460             error = 1;
01461         }
01462     */
01463     goto fin;
01464 case TDUBIOUS:
01465     x1 = 0; while ( *s >= 0 ) x1 = 128*x1 + *s++;
01466     if ( *s == TWILDCARD ) s++;
01467     if ( *s == TDOT ) goto dovector;
01468     if ( *s == TFUNOPEN ) {
01469         x1 += FUNCTION;
01470         cc = 1;
01471         v = t; *t++ = x1; *t++ = FUNHEAD; *t++ = DIRTYFLAG;
01472         if ( x1 == AR.PolyFun && AR.PolyFunType == 2 && AR.Eside != LHSIDE )
01473             t[-1] |= MUSTCLEANPRF;
01474         FILLFUN3(t)
01475         needarg = -1; goto dofunction;
01476     }
01477     *t++ = SYMBOL; *t++ = 4; *t++ = 0;
01478     if ( inset ) *relo = 2;
01479     goto TryPower;
01480 case TSUBEXP:
01481     x1 = 0; while ( *s >= 0 ) { x1 = x1*128 + *s++; }
01482     if ( *s == TPOWER ) {
01483         s++; c = *s++;
01484         base = ( c == TNUMBER ) ? 100: 128;
01485         x2 = 0; while ( *s >= 0 ) { x2 = base*x2 + *s++; }
01486         if ( *s == TWILDCARD ) { x2 += 2*MAXPOWER; s++; }
01487         else if ( c == TSYMBOL ) x2 += 2*MAXPOWER;
01488     }
01489     else x2 = 1;
01490     r = AC.ProtoType; n = r[1] - 5; r += 5;
01491     *t++ = SUBEXPRESSION; *t++ = r[-4];
01492     *t++ = subexpbuffers[x1].subexpnum;
01493     *t++ = x2*deno;
01494     *t++ = subexpbuffers[x1].buffernum;
01495     NCOPY(t,r,n);
01496     if ( cbuf[subexpbuffers[x1].buffernum].CanCommu[subexpbuffers[x1].subexpnum] ) cc = 1;
01497     deno = 1;
01498     break;
01499 case TMULTIPLY:
01500     mulflag = 1;
01501     break;
01502 case TDIVIDE:
01503     mulflag = 1;
01504     deno = -deno;
01505     break;
01506 case TEXPRESSION:
01507     cc = 1;
01508     x1 = 0; while ( *s >= 0 ) { x1 = x1*128 + *s++; }
01509     v = t;
01510     *t++ = EXPRESSION; *t++ = SUBEXPSIZE; *t++ = x1; *t++ = deno;
01511     *t++ = 0; FILLSUB(t)
01512 /*
01513     Here we had some erroneous code before. It should be after
01514     the reading of the parameters as it is now (after 15-jan-2007).
01515     Thomas Hahn noticed this and reported it.
01516 */
01517     if ( *s == TFUNOPEN ) {
01518         do {
01519             s++; c = *s++;
01520             base = ( c == TNUMBER ) ? 100: 128;
01521             x2 = 0; while ( *s >= 0 ) { x2 = base*x2 + *s++; }
01522             switch ( c ) {
01523                 case TSYMBOL:
01524                     *t++ = SYMBOL; *t++ = 4; *t++ = x2; *t++ = 1;
01525                     break;
01526                 case TINDEX:
01527                     *t++ = INDEX; *t++ = 3; *t++ = x2+AM.OffsetIndex;
01528                     if ( t[-1] > AM.IndDum ) {
01529                         x2 = t[-1] - AM.IndDum;
01530                         if ( x2 > C->numdum[numexp] ) C->numdum[numexp] = x2;
01531                     }
01532                     break;
01533                 case TVECTOR:
01534                     *t++ = INDEX; *t++ = 3; *t++ = x2+AM.OffsetVector;
01535                     break;
01536                 case TFUNCTION:
01537                     *t++ = x2+FUNCTION; *t++ = 2; break;
01538                 case TNUMBER:
01539                 case TNUMBER1:
01540                     if ( x2 >= AM.OffsetIndex || x2 < 0 ) {
01541                         MesPrint("&Index as argument of expression has illegal value");
01542                         error = -1;

```

```

01543     }
01544     *t++ = INDEX; *t++ = 3; *t++ = x2; break;
01545 case TSETDOL:
01546     x2 = -x2;
01547     /* fall through */
01548 case TSETNUM:
01549     if ( inset == 0 ) {
01550         w1 = t; t += 2; w2 = t;
01551         while ( w1 > v ) *--w2 = *--w1;
01552         tsize = v; relo = AT.WorkTop;
01553         *v++ = SETSET; *v++ = 0;
01554         inset = 1;
01555     }
01556     *--relo = x2; *--relo = t-v+2;
01557     c = *s++;
01558     x2 = 0; while ( *s >= 0 ) x2 = 128*x2 + *s++;
01559     switch ( c ) {
01560     case TFUNCTION:
01561         *relo -= 2; *t++ = -x2-1; break;
01562     case TSYMBOL:
01563         *t++ = SYMBOL; *t++ = 4; *t++ = x2; *t++ = 1; break;
01564     case TINDEX:
01565         *t++ = INDEX; *t++ = 3; *t++ = x2;
01566         if ( x2+AM.OffsetIndex > AM.IndDum ) {
01567             x2 = x2+AM.OffsetIndex - AM.IndDum;
01568             if ( x2 > C->numdum[numexp] ) C->numdum[numexp] = x2;
01569         }
01570         break;
01571     case TVECTOR:
01572         *t++ = VECTOR; *t++ = 3; *t++ = x2; break;
01573     case TNUMBER1:
01574         *t++ = SNUMBER; *t++ = 4; *t++ = x2; *t++ = 1; break;
01575     default:
01576         MesPrint("&Internal error 435");
01577         error = 1;
01578         *t++ = SYMBOL; *t++ = 4; *t++ = x2; *t++ = 1; break;
01579     }
01580     break;
01581 default:
01582     MesPrint("&Argument of expression can only be symbol, index, vector or
function");
01583     error = -1;
01584     break;
01585     }
01586 } while ( *s == TCOMMA );
01587 if ( *s != TFUNCLOSE ) {
01588     MesPrint("&Illegal object in argument field for expression");
01589     error = -1;
01590     while ( *s != TFUNCLOSE ) s++;
01591 }
01592 s++;
01593 }
01594 r = AC.ProtoType; n = r[1];
01595 if ( n > SUBEXPSIZE ) {
01596     *t++ = WILDCARDS; *t++ = n+2;
01597     NCOPY(t,r,n);
01598 }
01599 /*
01600 Code added for parallel processing.
01601 This is different from the other occurrences to test immediately
01602 for renumbering. Here we have to read the parameters first.
01603 */
01604 if ( Expressions[x1].status == STOREDEXPRESSION ) {
01605     v[1] = t-v;
01606     AT.TMaddr = v;
01607     PUTZERO(position);
01608 /*
01609     if ( (
01610 #ifdef WITHPTHREADS
01611         renumber =
01612 #endif
01613         GetTable(x1,&position,0) ) == 0 ) {
01614         error = 1;
01615         MesPrint("&Problems getting information about stored expression %s(3)"
01616             ,EXPRNAME(x1));
01617     }
01618 #ifdef WITHPTHREADS
01619     M_free(renumber->symb.lo,"VarSpace");
01620     M_free(renumber,"Renumber");
01621 #endif
01622 */
01623     if ( ( renumber = GetTable(x1,&position,0) ) == 0 ) {
01624         error = 1;
01625         MesPrint("&Problems getting information about stored expression %s(3)"
01626             ,EXPRNAME(x1));
01627     }
01628     if ( renumber->symb.lo != AN.dummyrenumlist )

```

```

01629         M_free(renumber->symb.lo,"VarSpace");
01630         M_free(renumber,"ReNUMBER");
01631         AR.StoreData.dirtyflag = 1;
01632     }
01633     if ( *s == LBRACE ) {
01634         /*
01635             This should be one term that should be inserted
01636             FROMBRAC size+2 ( term )
01637             Because this term should have been translated
01638             already we can copy it from the 'subexpression'
01639         */
01640         s++;
01641         if ( *s != TSUBEXP ) {
01642             MesPrint("&Internal error 23");
01643             Terminate(-1);
01644         }
01645         s++; x2 = 0; while ( *s >= 0 ) { x2 = 128*x2 + *s++; }
01646         r = cbuf[subexpbuffers[x2].buffernum].rhs[subexpbuffers[x2].subexpnum];
01647         *t++ = FROMBRAC; *t++ = *r+2;
01648         n = *r;
01649         NCOPY(t,r,n);
01650         if ( *r != 0 ) {
01651             MesPrint("&Object between [] in expression should be a single term");
01652             error = -1;
01653         }
01654         if ( *s != RBRACE ) {
01655             MesPrint("&Internal error 23b");
01656             Terminate(-1);
01657         }
01658         s++;
01659     }
01660     if ( *s == TPOWER ) {
01661         s++; c = *s++;
01662         base = ( c == TNUMBER ) ? 100: 128;
01663         x2 = 0; while ( *s >= 0 ) { x2 = base*x2 + *s++; }
01664         if ( *s == TWILDCARD || c == TSYMBOL ) { x2 += 2*MAXPOWER; s++; }
01665         v[3] = x2;
01666     }
01667     v[1] = t - v;
01668     deno = 1;
01669     break;
01670 case TNUMBER:
01671     if ( *s == 0 ) {
01672         s++;
01673         if ( *s == TPOWER ) {
01674             s++; if ( *s == TMINUS ) { s++; deno = -deno; }
01675             c = *s++; base = ( c == TNUMBER ) ? 100: 128;
01676             x2 = 0; while ( *s >= 0 ) { x2 = x2*base + *s++; }
01677             if ( x2 == 0 ) {
01678                 error = -1;
01679                 MesPrint("&Encountered 0^0 during compilation");
01680             }
01681             if ( deno < 0 ) {
01682                 error = -1;
01683                 MesPrint("&Division by zero during compilation (0 to the power negative
01684 number)");
01685             }
01686             else if ( deno < 0 ) {
01687                 error = -1;
01688                 MesPrint("&Division by zero during compilation");
01689             }
01690             sign = 0; break; /* term is zero */
01691         }
01692         y = *s++;
01693         if ( *s >= 0 ) { y = 100*y + *s++; }
01694         innum[0] = y; nin = 1;
01695         while ( *s >= 0 ) {
01696             y = *s++; x2 = 100;
01697             if ( *s >= 0 ) { y = 100*y + *s++; x2 = 10000; }
01698             Product(innum,&nin,(WORD)x2);
01699             if ( y ) AddLong(innum,nin,(UWORD *)(&y),(WORD)1,innum,&nin);
01700         }
01701     docoef:
01702     if ( *s == TPOWER ) {
01703         s++; if ( *s == TMINUS ) { s++; deno = -deno; }
01704         c = *s++; base = ( c == TNUMBER ) ? 100: 128;
01705         x2 = 0; while ( *s >= 0 ) { x2 = x2*base + *s++; }
01706         if ( x2 == 0 ) {
01707             innum[0] = 1; nin = 1;
01708         }
01709         else if ( RaisPow(BHEAD innum,&nin,x2) ) {
01710             error = -1; innum[0] = 1; nin = 1;
01711         }
01712     }
01713     if ( deno > 0 ) {
01714         Simplify(BHEAD innum,&nin,denominator,&denominator);

```

```

01715         for ( i = 0; i < nnumerator; i++ ) CGscrat7[i] = numerator[i];
01716         MulLong(innum,nin,CGscrat7,nnumerator,numerator,&nnumerator);
01717     }
01718     else if ( deno < 0 ) {
01719         Simplify(BHEAD innum,&nin,numerator,&nnumerator);
01720         for ( i = 0; i < ndenominator; i++ ) CGscrat7[i] = denominator[i];
01721         MulLong(innum,nin,CGscrat7,ndenominator,denominator,&ndenominator);
01722     }
01723     deno = 1;
01724     break;
01725 case TNUMBER1:
01726     if ( *s == 0 ) { s++; sign = 0; break; /* term is zero */ }
01727     y = *s++;
01728     if ( *s >= 0 ) { y = 128*y + *s++; }
01729     if ( inset == 0 ) {
01730         innum[0] = y; nin = 1;
01731         while ( *s >= 0 ) {
01732             y = *s++; x2 = 128;
01733             if ( *s >= 0 ) { y = 128*y + *s++; x2 = 16384; }
01734             Product(innum,&nin,(WORD)x2);
01735             if ( y ) AddLong(innum,nin,(UWORD *)&y,(WORD)1,innum,&nin);
01736         }
01737         goto docoef;
01738     }
01739     *relo = 2; *t++ = SNUMBER; *t++ = 4; *t++ = y;
01740     goto TryPower;
01741 #ifdef WITHFLOAT
01742 case TFLOAT:
01743     { WORD *w;
01744       s = ReadFloat(s);
01745       i = AT.WorkPointer[1];
01746       w = AT.WorkPointer;
01747       NCOPY(t,w,i);
01748     }
01749     /*
01750     Power?
01751     */
01752     break;
01753 #endif
01754 case TDOLLAR:
01755     {
01756         WORD *powplace;
01757         x1 = 0; while ( *s >= 0 ) { x1 = x1*128 + *s++; }
01758         if ( AR.Eside != LHSIDE ) {
01759             *t++ = SUBEXPRESSION; *t++ = SUBEXPSIZE; *t++ = x1;
01760         }
01761         else {
01762             *t++ = DOLLAREXPRESSION; *t++ = SUBEXPSIZE; *t++ = x1;
01763         }
01764         powplace = t; t++;
01765         *t++ = AM.dbufnum; FILLSUB(t)
01766     }
01767     /*
01768     Now we have to test for factors of dollars with [ ] and [ [ ] ]
01769     */
01770     if ( *s == LBRACE ) {
01771         int bracelevel = 1;
01772         s++;
01773         while ( bracelevel > 0 ) {
01774             if ( *s == RBRACE ) {
01775                 bracelevel--; s++;
01776             }
01777             else if ( *s == TNUMBER ) {
01778                 s++;
01779                 x2 = 0; while ( *s >= 0 ) { x2 = 100*x2 + *s++; }
01780                 *t++ = DOLLAREXPR2; *t++ = 3; *t++ = -x2-1;
01781             }
01782             while ( bracelevel > 0 ) {
01783                 if ( *s != RBRACE ) {
01784                     error = -1;
01785                     MesPrint("&Improper use of [] in $-variable.");
01786                     return(error);
01787                 }
01788                 else {
01789                     s++; bracelevel--;
01790                 }
01791             }
01792         }
01793     }
01794     else if ( *s == TDOLLAR ) {
01795         s++;
01796         x1 = 0; while ( *s >= 0 ) { x1 = x1*128 + *s++; }
01797         *t++ = DOLLAREXPR2; *t++ = 3; *t++ = x1;
01798         if ( *s == RBRACE ) goto CloseBraces;
01799         else if ( *s == LBRACE ) {
01800             s++; bracelevel++;
01801         }
01802     }

```

```

01802             else goto ErrorBraces;
01803         }
01804     }
01805     /*
01806         Finally we can continue with the power
01807     */
01808     if ( *s == TPOWER ) {
01809         s++;
01810         if ( *s == TMINUS ) { s++; deno = -deno; }
01811         c = *s++;
01812         base = ( c == TNUMBER ) ? 100: 128;
01813         x2 = 0; while ( *s >= 0 ) { x2 = base*x2 + *s++; }
01814         if ( c == TSYMBOL ) {
01815             if ( *s == TWILDCARD ) s++;
01816             x2 += 2*MAXPOWER;
01817         }
01818         *powplace = deno*x2;
01819     }
01820     else *powplace = deno;
01821     deno = 1;
01822     /*
01823         if ( inset ) {
01824             while ( relo < AT.WorkTop ) *t++ = *relo++;
01825             inset = 0; tsize[1] = t - tsize;
01826         }
01827     */
01828 }
01829     break;
01830 case TSETNUM:
01831     inset = 1; tsize = t; relo = AT.WorkTop;
01832     *t++ = SETSET; *t++ = 0;
01833     x1 = 0; while ( *s >= 0 ) x1 = x1*128 + *s++;
01834     *---relo = x1; *---relo = 0;
01835     break;
01836 case TSETDOL:
01837     inset = 1; tsize = t; relo = AT.WorkTop;
01838     *t++ = SETSET; *t++ = 0;
01839     x1 = 0; while ( *s >= 0 ) x1 = x1*128 + *s++;
01840     *---relo = -x1; *---relo = 0;
01841     break;
01842 case TFUNOPEN:
01843     MesPrint("&Illegal use of function arguments");
01844     error = -1;
01845     funflag = 1;
01846     deno = 1;
01847     break;
01848 case TFUNCLOSE:
01849     if ( funflag == 0 )
01850         MesPrint("&Illegal use of function arguments");
01851     error = -1;
01852     funflag = 0;
01853     deno = 1;
01854     break;
01855 case TSGAMMA:
01856     MesPrint("&Illegal use special gamma symbols 5_, 6_, 7_");
01857     error = -1;
01858     funflag = 0;
01859     deno = 1;
01860     break;
01861 case TCONJUGATE:
01862     MesPrint("&Complex conjugate operator (%#) is not implemented");
01863     error = -1;
01864     deno = 1;
01865     break;
01866 default:
01867     MesPrint("&Internal error in code generator. Unknown object: %d",c);
01868     error = -1;
01869     deno = 1;
01870     break;
01871 }
01872 }
01873 }
01874 if ( mulflag ) {
01875     MesPrint("&Irregular end of statement.");
01876     error = 1;
01877 }
01878 if ( !first && error == 0 ) {
01879     *term = t-term;
01880     C->NumTerms[numexp]++;
01881     if ( cc && sign ) C->CanCommu[numexp]++;
01882     error = CompleteTerm(term,numerator,denominator,nnumerator,ndenominator,sign);
01883 }
01884 AT.WorkPointer = oldwork;
01885 if ( error ) return(-1);
01886 AddToCB(C,0)
01887 if ( AC.CompileLevel > 0 && AR.Eside != LHSIDE ) {
01888     /* See whether we have this one already */

```

```

01889         error = InsTree(AC.cbufnum,C->numrhs);
01890         if ( error < (C->numrhs) ) {
01891             C->Pointer = C->rhs[C->numrhs--];
01892             return(error);
01893         }
01894     }
01895     return(C->numrhs);
01896 OverWork:
01897     MLOCK(ErrorMessageLock);
01898     MesWork();
01899     MUNLOCK(ErrorMessageLock);
01900     return(-1);
01901 }
01902
01903 /*
01904     #] CodeGenerator :
01905     #[ CompleteTerm :
01906
01907     Completes the term
01908     Puts it in the buffer
01909 */
01910
01911 int CompleteTerm(WORD *term, UWORD *number, UWORD *denom, WORD nnum, WORD nden, int sign)
01912 {
01913     int nsize, i;
01914     WORD *t;
01915     if ( sign == 0 ) return(0);      /* Term is zero */
01916     if ( nnum >= nden ) nsize = nnum;
01917     else nsize = nden;
01918     t = term + *term;
01919     for ( i = 0; i < nnum; i++ ) *t++ = number[i];
01920     for ( ; i < nsize; i++ ) *t++ = 0;
01921     for ( i = 0; i < nden; i++ ) *t++ = denom[i];
01922     for ( ; i < nsize; i++ ) *t++ = 0;
01923     *t++ = (2*nsize+1)*sign;
01924     *term = t - term;
01925     AddNtoC(AC.cbufnum,*term,term,7);
01926     return(0);
01927 }
01928
01929 /*
01930     #] CompleteTerm :
01931     #[ CodeFactors :
01932
01933     This routine does the part of reading in in terms of factors.
01934     If there is more than one term at this level we have only one
01935     factor. In that case any expression should first be unfactorized.
01936     Then the whole expression gets read as a new subexpression and finally
01937     we generate factor_*subexpression.
01938     If the whole has only multiplications we have factors. Then the
01939     nasty thing is powers of objects and in particular powers of
01940     factorized expressions or dollars.
01941     For a power we generate a new subexpression of the type
01942         1+factor_+...+factor_^(power-1)
01943     with which we multiply.
01944
01945     WE HAVE NOT YET WORRIED ABOUT SETS
01946 */
01947
01948 int CodeFactors(SBYTE *tokens)
01949 {
01950     GETIDENTITY
01951     EXPRESSIONS e = Expressions + AR.CurExpr;
01952     int nfactor = 1, nparenthesis, i, last = 0, error = 0;
01953     SBYTE *t, *startobject, *tt, *sl, *out, *outtokens;
01954     WORD nexpt, subexp = 0, power, pow, x2, powfactor, first;
01955     /*
01956         First scan the number of factors
01957     */
01958     t = tokens;
01959     while ( *t != TENDOFIT ) {
01960         if ( *t >= 0 ) { while ( *t >= 0 ) t++; continue; }
01961         if ( *t == LPARENTHESIS || *t == LBRACE || *t == TSETOPEN || *t == TFUNOPEN ) {
01962             nparenthesis = 0; t++;
01963             while ( nparenthesis >= 0 ) {
01964                 if ( *t == LPARENTHESIS || *t == LBRACE || *t == TSETOPEN || *t == TFUNOPEN )
01965                     nparenthesis++;
01966                 else if ( *t == RPARENTHESIS || *t == RBRACE || *t == TSETCLOSE || *t == TFUNCLOSE )
01967                     nparenthesis--;
01968                 t++;
01969             }
01970             continue;
01971         }
01972         else if ( ( *t == TPLUS || *t == TMINUS ) && ( t > tokens )
01973             && ( t[-1] != TPLUS && t[-1] != TMINUS ) ) {
01974             if ( t[-1] >= 0 || t[-1] == RPARENTHESIS || t[-1] == RBRACE
01975                 || t[-1] == TSETCLOSE || t[-1] == TFUNCLOSE ) {

```

```

01974         subexp = CodeGenerator(tokens);
01975         if ( subexp < 0 ) error = -1;
01976         if ( insubexpbuffers >= MAXSUBEXPRESSIONS ) {
01977             MesPrint("&More than %d subexpressions inside one
expression", (WORD)MAXSUBEXPRESSIONS);
01978             Terminate(-1);
01979         }
01980         if ( subexpbuffers+insubexpbuffers >= topsubexpbuffers ) {
01981             DoubleBuffer((void **)((void *)(&subexpbuffers))
01982                 , (void **)((void *)(&topsubexpbuffers)), sizeof(SUBBUF), "subexpbuffers");
01983         }
01984         subexpbuffers[insubexpbuffers].subexpnum = subexp;
01985         subexpbuffers[insubexpbuffers].buffernum = AC.cbufnum;
01986         subexp = insubexpbuffers++;
01987         t = tokens;
01988         *t++ = TSYMBOL; *t++ = FACTORSYMBOL;
01989         *t++ = TMULTIPLY; *t++ = TSUBEXP;
01990         PUTNUMBER128(t, subexp)
01991         *t++ = TENDOFIT;
01992         e->numfactors = 1;
01993         e->vflags |= ISFACTORIZED;
01994         return(subexp);
01995     }
01996 }
01997 else if ( ( *t == TMULTIPLY || *t == TDIVIDE ) && t > tokens ) {
01998     nfactor++;
01999 }
02000 else if ( *t == TEXPRESSION ) {
02001     t++;
02002     nexp = 0; while ( *t >= 0 ) { nexp = nexp*128 + *t++; }
02003     if ( *t == LBRACE ) continue;
02004     if ( ( AS.Oldvflags[nexp] & ISFACTORIZED ) != 0 ) {
02005         nfactor += AS.OldNumFactors[nexp];
02006     }
02007     else { nfactor++; }
02008     continue;
02009 }
02010 else if ( *t == TDOLLAR ) {
02011     t++;
02012     nexp = 0; while ( *t >= 0 ) { nexp = nexp*128 + *t++; }
02013     if ( *t == LBRACE ) continue;
02014     if ( Dollars[nexp].nfactors > 0 ) {
02015         nfactor += Dollars[nexp].nfactors;
02016     }
02017     else { nfactor++; }
02018     continue;
02019 }
02020 t++;
02021 }
02022 /*
02023 Now the real pass.
02024 nfactor is a not so reliable measure for the space we need.
02025 */
02026 outtokens = (SBYTE *)Malloc1((t-tokens)+(nfactor+2)*25)*sizeof(SBYTE), "CodeFactors";
02027 out = outtokens;
02028 t = tokens; first = 1; powfactor = 1;
02029 while ( *t == TPLUS || *t == TMINUS ) { if ( *t == TMINUS ) first = -first; t++; }
02030 if ( first < 0 ) {
02031     *out++ = TMINUS; *out++ = TSYMBOL; *out++ = FACTORSYMBOL;
02032     *out++ = TPOWER; *out++ = TNUMBER; PUTNUMBER100(out, powfactor)
02033     powfactor++;
02034 }
02035 startobject = t; power = 1;
02036 while ( *t != TENDOFIT ) {
02037     if ( *t >= 0 ) { while ( *t >= 0 ) t++; continue; }
02038     if ( *t == LPARENTHESIS || *t == LBRACE || *t == TSETOPEN || *t == TFUNOPEN ) {
02039         nparenthesis = 0; t++;
02040         while ( nparenthesis >= 0 ) {
02041             if ( *t == LPARENTHESIS || *t == LBRACE || *t == TSETOPEN || *t == TFUNOPEN )
nparenthesis++;
02042             else if ( *t == RPARENTHESIS || *t == RBRACE || *t == TSETCLOSE || *t == TFUNCLOSE )
nparenthesis--;
02043             t++;
02044         }
02045         continue;
02046     }
02047     else if ( ( *t == TMULTIPLY || *t == TDIVIDE ) && ( t > tokens ) ) {
02048         if ( t[-1] >= 0 || t[-1] == RPARENTHESIS || t[-1] == RBRACE
02049             || t[-1] == TSETCLOSE || t[-1] == TFUNCLOSE ) {
02050 dolast:
02051             if ( startobject ) { /* apparently power is 1 or -1 */
02052                 *out++ = TPLUS;
02053                 if ( power < 0 ) { *out++ = TNUMBER; *out++ = 1; *out++ = TDIVIDE; }
02054                 s1 = startobject;
02055                 while ( s1 < t ) *out++ = *s1++;
02056                 *out++ = TMULTIPLY; *out++ = TSYMBOL; *out++ = FACTORSYMBOL;
02057                 *out++ = TPOWER; *out++ = TNUMBER; PUTNUMBER100(out, powfactor)

```



```

02058         powfactor++;
02059     }
02060     if ( last ) { startobject = 0; break; }
02061     startobject = t+1;
02062     if ( *t == TDIVIDE ) power = -1;
02063     if ( *t == TMULTIPLY ) power = 1;
02064 }
02065 }
02066 else if ( *t == TPOWER ) {
02067     pow = 1;
02068     tt = t+1;
02069     while ( ( *tt == TMINUS ) || ( *tt == TPLUS ) ) {
02070         if ( *tt == TMINUS ) pow = -pow;
02071         tt++;
02072     }
02073     if ( *tt == TSymbol ) {
02074         tt++; while ( *tt >= 0 ) tt++;
02075         t = tt; continue;
02076     }
02077     tt++; x2 = 0; while ( *tt >= 0 ) { x2 = 100*x2 + *tt++; }
02078 /*
02079     We have an object in startobject till t. The power is
02080     power*pow*x2
02081 */
02082     power = power*pow*x2;
02083     if ( power < 0 ) { pow = -power; power = -1; }
02084     else if ( power == 0 ) { t = tt; startobject = tt; continue; }
02085     else { pow = power; power = 1; }
02086     *out++ = TPLUS;
02087     if ( pow > 1 ) {
02088         subexp = GenerateFactors(pow,1);
02089         if ( subexp < 0 ) { error = -1; subexp = 0; }
02090         *out++ = TSUBEXP; PUTNUMBER128(out,subexp);
02091     }
02092     *out++ = TSymbol; *out++ = FACTORSymbol;
02093     *out++ = TPOWER; *out++ = TNUMBER; PUTNUMBER100(out,powfactor)
02094     powfactor += pow;
02095     if ( power > 0 ) *out++ = TMULTIPLY;
02096     else *out++ = TDIVIDE;
02097     s1 = startobject; while ( s1 < t ) *out++ = *s1++;
02098     startobject = 0; t = tt; continue;
02099 }
02100 else if ( *t == TExpression ) {
02101     startobject = t;
02102     t++;
02103     nexp = 0; while ( *t >= 0 ) { nexp = nexp*128 + *t++; }
02104     if ( *t == LBRACE ) continue;
02105     if ( *t == LPARENTHESIS ) {
02106         nparenthesis = 0; t++;
02107         while ( nparenthesis >= 0 ) {
02108             if ( *t == LPARENTHESIS ) nparenthesis++;
02109             else if ( *t == RPARENTHESIS ) nparenthesis--;
02110             t++;
02111         }
02112     }
02113     if ( ( AS.Oldvflags[nexp] & ISFACTORIZED ) == 0 ) continue;
02114     if ( *t == TPOWER ) {
02115         pow = 1;
02116         tt = t+1;
02117         while ( ( *tt == TMINUS ) || ( *tt == TPLUS ) ) {
02118             if ( *tt == TMINUS ) pow = -pow;
02119             tt++;
02120         }
02121         if ( *tt != TNUMBER ) {
02122             MesPrint("Internal problems(1) in CodeFactors");
02123             return(-1);
02124         }
02125         tt++; x2 = 0; while ( *tt >= 0 ) { x2 = 100*x2 + *tt++; }
02126 /*
02127         We have an object in startobject till t. The power is
02128         power*pow*x2
02129 */
02130     dopower:
02131         power = power*pow*x2;
02132         if ( power < 0 ) { pow = -power; power = -1; }
02133         else if ( power == 0 ) { t = tt; startobject = tt; continue; }
02134         else { pow = power; power = 1; }
02135         *out++ = TPLUS;
02136         if ( pow > 1 ) {
02137             subexp = GenerateFactors(pow,AS.OldNumFactors[nexp]);
02138             if ( subexp < 0 ) { error = -1; subexp = 0; }
02139             *out++ = TSUBEXP; PUTNUMBER128(out,subexp)
02140             *out++ = TMULTIPLY;
02141         }
02142         i = powfactor-1;
02143         if ( i > 0 ) {
02144             *out++ = TSymbol; *out++ = FACTORSymbol;

```

```

02145         if ( i > 1 ) {
02146             *out++ = TPOWER; *out++ = TNUMBER; PUTNUMBER100(out,i)
02147         }
02148         *out++ = TMULTIPLY;
02149     }
02150     powfactor += AS.OldNumFactors[nexp]*pow;
02151     s1 = startobject;
02152     while ( s1 < t ) *out++ = *s1++;
02153     startobject = 0; t = tt; continue;
02154 }
02155 else {
02156     tt = t; pow = 1; x2 = 1; goto dopower;
02157 }
02158 }
02159 else if ( *t == TDOLLAR ) {
02160     startobject = t;
02161     t++;
02162     nexp = 0; while ( *t >= 0 ) { nexp = nexp*128 + *t++; }
02163     if ( *t == LBRACE ) continue;
02164     if ( Dollars[nexp].nfactors == 0 ) continue;
02165     if ( *t == TPOWER ) {
02166         pow = 1;
02167         tt = t+1;
02168         while ( ( *tt == TMINUS ) || ( *tt == TPLUS ) ) {
02169             if ( *tt == TMINUS ) pow = -pow;
02170             tt++;
02171         }
02172         if ( *tt != TNUMBER ) {
02173             MesPrint("Internal problems(2) in CodeFactors");
02174             return(-1);
02175         }
02176         tt++; x2 = 0; while ( *tt >= 0 ) { x2 = 100*x2 + *tt++; }
02177     /*
02178     We have an object in startobject till t. The power is
02179     power*pow*x2
02180     */
02181     dopower:
02182     power = power*pow*x2;
02183     if ( power < 0 ) { pow = -power; power = -1; }
02184     else if ( power == 0 ) { t = tt; startobject = tt; continue; }
02185     else { pow = power; power = 1; }
02186     if ( pow > 1 ) {
02187         subexp = GenerateFactors(pow,1);
02188         if ( subexp < 0 ) { error = -1; subexp = 0; }
02189     }
02190     for ( i = 1; i <= Dollars[nexp].nfactors; i++ ) {
02191         s1 = startobject; *out++ = TPLUS;
02192         while ( s1 < t ) *out++ = *s1++;
02193         *out++ = LBRACE; *out++ = TNUMBER; PUTNUMBER128(out,i)
02194         *out++ = RBRACE;
02195         *out++ = TMULTIPLY;
02196         *out++ = TSYMBOL; *out++ = FACTORSYMBOL;
02197         *out++ = TPOWER; *out++ = TNUMBER; PUTNUMBER100(out,powfactor)
02198         powfactor += pow;
02199         if ( pow > 1 ) {
02200             *out++ = TSUBEXP; PUTNUMBER128(out,subexp)
02201         }
02202     }
02203     startobject = 0; t = tt; continue;
02204 }
02205 else {
02206     tt = t; pow = 1; x2 = 1; goto dopower;
02207 }
02208 }
02209 t++;
02210 }
02211 if ( last == 0 ) { last = 1; goto dolast; }
02212 *out = TENDOFIT;
02213 e->numfactors = powfactor-1;
02214 e->vflags |= ISFACTORIZED;
02215 subexp = CodeGenerator(outtokens);
02216 if ( subexp < 0 ) error = -1;
02217 if ( insubexpbuffers >= MAXSUBEXPRESSIONS ) {
02218     MesPrint("&More than %d subexpressions inside one expression", (WORD)MAXSUBEXPRESSIONS);
02219     Terminate(-1);
02220 }
02221 if ( subexpbuffers+insubexpbuffers >= topsubexpbuffers ) {
02222     DoubleBuffer((void **)((void *)(&subexpbuffers))
02223     , (void **)((void *)(&topsubexpbuffers)), sizeof(SUBBUF), "subexpbuffers");
02224 }
02225 subexpbuffers[insubexpbuffers].subexpnum = subexp;
02226 subexpbuffers[insubexpbuffers].buffernum = AC.cbufnum;
02227 subexp = insubexpbuffers++;
02228 M_free(outtokens, "CodeFactors");
02229 s1 = tokens;
02230 *s1++ = TSUBEXP; PUTNUMBER128(s1,subexp); *s1++ = TENDOFIT;
02231 if ( error < 0 ) return(-1);

```

```

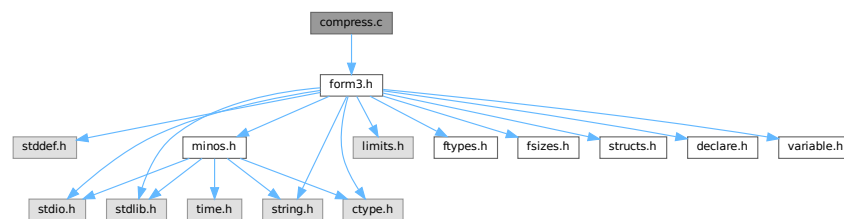
02232     else return(subexp);
02233 }
02234
02235 /*
02236     #] CodeFactors :
02237     #[ GenerateFactors :
02238
02239     Generates an expression of the type
02240     1+factor_+factor_^2+...+factor_^(n-1)
02241     (this is if inc=1)
02242     Returns the subexpression pointer of it.
02243 */
02244
02245 WORD GenerateFactors(WORD n,WORD inc)
02246 {
02247     WORD subexp;
02248     int i, error = 0;
02249     SBYTE *s;
02250     SBYTE *tokenbuffer = (SBYTE *)Malloc1(8*n*sizeof(SBYTE),"GenerateFactors");
02251     s = tokenbuffer;
02252     *s++ = TNUMBER; *s++ = 1;
02253     for ( i = inc; i < n*inc; i += inc ) {
02254         *s++ = TPLUS; *s++ = TSYMBOL; *s++ = FACTORSYMBOL;
02255         if ( i > 1 ) {
02256             *s++ = TPOWER; *s++ = TNUMBER;
02257             PUTNUMBER100(s,i)
02258         }
02259     }
02260     *s++ = TENDOFIT;
02261     subexp = CodeGenerator(tokenbuffer);
02262     if ( subexp < 0 ) error = -1;
02263     M_free(tokenbuffer,"GenerateFactors");
02264     if ( insubexpbuffers >= MAXSUBEXPRESSIONS ) {
02265         MesPrint("&More than %d subexpressions inside one expression", (WORD)MAXSUBEXPRESSIONS);
02266         Terminate(-1);
02267     }
02268     if ( subexpbuffers+insubexpbuffers >= topsubexpbuffers ) {
02269         DoubleBuffer((void **) ((void *)(&subexpbuffers))
02270             , (void **) ((void *)(&topsubexpbuffers)), sizeof(SUBBUF), "subexpbuffers");
02271     }
02272     subexpbuffers[insubexpbuffers].subexpnum = subexp;
02273     subexpbuffers[insubexpbuffers].buffernum = AC.cbufnum;
02274     subexp = insubexpbuffers++;
02275     if ( error < 0 ) return(error);
02276     return(subexp);
02277 }
02278
02279 /*
02280     #] GenerateFactors :
02281     #[ Compiler :
02282 */

```

4.13 compress.c File Reference

#include "form3.h"

Include dependency graph for compress.c:



4.13.1 Detailed Description

The routines for the use of gzip (de)compression of the information in the sort file.

Definition in file [compress.c](#).

4.14 compress.c

[Go to the documentation of this file.](#)

```

00001
00007 /* #[ License : */
00008 /*
00009  *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00010  *   When using this file you are requested to refer to the publication
00011  *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00012  *   This is considered a matter of courtesy as the development was paid
00013  *   for by FOM the Dutch physics granting agency and we would like to
00014  *   be able to track its scientific use to convince FOM of its value
00015  *   for the community.
00016  *
00017  *   This file is part of FORM.
00018  *
00019  *   FORM is free software: you can redistribute it and/or modify it under the
00020  *   terms of the GNU General Public License as published by the Free Software
00021  *   Foundation, either version 3 of the License, or (at your option) any later
00022  *   version.
00023  *
00024  *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00025  *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00026  *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00027  *   details.
00028  *
00029  *   You should have received a copy of the GNU General Public License along
00030  *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00031  */
00032 /* #[ License : */
00033
00034 #include "form3.h"
00035
00036 #ifdef WITHZLIB
00037 /*
00038 #define GZIPDEBUG
00039
00040     Low level routines for dealing with zlib during sorting and handling
00041     the scratch files. Work started 5-sep-2005.
00042     The .sor file handling was more or less completed on 8-sep-2005
00043     The handling of the scratch files still needs some thinking.
00044     Complications are:
00045         gzip compression should be per expression, not per buffer.
00046         No gzip compression for expressions with a bracket index.
00047         Separate decompression buffers for expressions in the rhs.
00048         This last one will involve more buffer work and organization.
00049         Information about compression should be stored for each expr.
00050         (including what method/program was used)
00051     Note: Be careful with compression. By far the most compact method
00052     is the original problem....
00053
00054     #[ Variables :
00055
00056     The following variables are to contain the intermediate buffers
00057     for the inflation of the various patches in the sort file.
00058     There can be up to MaxFpatches (FilePatches in the setup) and hence
00059     we can have that many streams simultaneously. We set this up once
00060     and only when needed.
00061     (in struct A.N or AB[threadnum].N)
00062     Bytef **AN.ziobufnum;
00063     Bytef *AN.ziobuffers;
00064 */
00065
00066 /*
00067     #[ Variables :
00068     #[ SetupOutputGZIP :
00069
00070     Routine prepares a gzip output stream for the given file.
00071 */
00072
00073 int SetupOutputGZIP(FILEHANDLE *f)
00074 {
00075     GETIDENTITY
00076
00077     if ( AT.SS != AT.S0 ) return(0);
00078     if ( AR.NoCompress == 1 ) return(0);
00079     if ( AR.gzipCompress <= 0 ) return(0);
00080
00081     if ( f->ziobuffer == 0 ) {
00082 /*
00083         1: Allocate a struct for the gzip stream:
00084 */
00085         f->zsp = Malloc1(sizeof(z_stream),"output zstream");
00086 /*
00087         2: Allocate the output buffer.

```

```

00088 */
00089     f->zlibbuffer =
00090         (Bytef *)Malloc1(f->ziosize*sizeof(char),"output zbuffer");
00091     if ( f->zsp == 0 || f->zlibbuffer == 0 ) {
00092         MLOCK(ErrorMessageLock);
00093         MesCall("SetupOutputGZIP");
00094         MUNLOCK(ErrorMessageLock);
00095         Terminate(-1);
00096     }
00097 }
00098 /*
00099 3: Set the default fields:
00100 */
00101 f->zsp->zalloc = Z_NULL;
00102 f->zsp->zfree  = Z_NULL;
00103 f->zsp->opaque = Z_NULL;
00104 /*
00105 4: Set the output space:
00106 */
00107 f->zsp->next_out = f->zlibbuffer;
00108 f->zsp->avail_out = f->ziosize;
00109 f->zsp->total_out = 0;
00110 /*
00111 5: Set the input space:
00112 */
00113 f->zsp->next_in = (Bytef *) (f->PObuffer);
00114 f->zsp->avail_in = (Bytef *) (f->POfill) - (Bytef *) (f->PObuffer);
00115 f->zsp->total_in = 0;
00116 /*
00117 6: Initiate the deflation
00118 */
00119 if ( deflateInit(f->zsp,AR.gzipCompress) != Z_OK ) {
00120     MLOCK(ErrorMessageLock);
00121     MesPrint("Error from zlib: %s",f->zsp->msg);
00122     MesCall("SetupOutputGZIP");
00123     MUNLOCK(ErrorMessageLock);
00124     Terminate(-1);
00125 }
00126
00127 return(0);
00128 }
00129
00130 /*
00131 #] SetupOutputGZIP :
00132 #[ PutOutputGZIP :
00133
00134 Routine is called when the PObuffer of f is full.
00135 The contents of it will be compressed and whenever the output buffer
00136 f->zlibbuffer is full it will be written and the output buffer
00137 will be reset.
00138 Upon exit the input buffer will be cleared.
00139 */
00140
00141 int PutOutputGZIP(FILEHANDLE *f)
00142 {
00143     GETIDENTITY
00144     int zerror;
00145     /*
00146     First set the number of bytes in the input
00147     */
00148     f->zsp->next_in = (Bytef *) (f->PObuffer);
00149     f->zsp->avail_in = (Bytef *) (f->POfill) - (Bytef *) (f->PObuffer);
00150     f->zsp->total_in = 0;
00151
00152     while ( ( zerror = deflate(f->zsp,Z_NO_FLUSH) ) == Z_OK ) {
00153         if ( f->zsp->avail_out == 0 ) {
00154             /*
00155             zibuffer is full. Write the output.
00156             */
00157             #ifdef GZIPDEBUG
00158             {
00159                 char *s = (char *) ((UBYTE *) (f->zlibbuffer)+f->ziosize);
00160                 MLOCK(ErrorMessageLock);
00161                 MesPrint("%wWriting %l bytes at %10p: %d %d %d %d %d"
00162                     ,f->ziosize,&(f->POposition),s[-5],s[-4],s[-3],s[-2],s[-1]);
00163                 MUNLOCK(ErrorMessageLock);
00164             }
00165             #endif
00166             #ifdef ALLLOCK
00167             LOCK(f->pthreadlock);
00168             #endif
00169             if ( f == AR.hidefile ) {
00170                 LOCK(AS.inputslock);
00171             }
00172             SeekFile(f->handle,&(f->POposition),SEEK_SET);
00173             if ( WriteFile(f->handle,(UBYTE *) (f->zlibbuffer),f->ziosize)
00174                 != f->ziosize ) {

```

```

00175         if ( f == AR.hidefile ) {
00176             UNLOCK(AS.inputslock);
00177         }
00178 #ifdef ALLLOCK
00179         UNLOCK(f->pthreadlock);
00180 #endif
00181         MLOCK(ErrorMessageLock);
00182         MesPrint("%wWrite error during compressed sort. Disk full?");
00183         MUNLOCK(ErrorMessageLock);
00184         return(-1);
00185     }
00186     if ( f == AR.hidefile ) {
00187         UNLOCK(AS.inputslock);
00188     }
00189 #ifdef ALLLOCK
00190         UNLOCK(f->pthreadlock);
00191 #endif
00192         ADDPOS(f->filesize,f->ziosize);
00193         ADDPOS(f->POposition,f->ziosize);
00194 #ifdef WITHPTHREADS
00195         if ( AS.MasterSort && AC.ThreadSortFileSynch ) {
00196             if ( f->handle >= 0 ) SynchFile(f->handle);
00197         }
00198 #endif
00199 /*
00200     Reset the output
00201 */
00202     f->zsp->next_out = f->ziobuffer;
00203     f->zsp->avail_out = f->ziosize;
00204     f->zsp->total_out = 0;
00205 }
00206 else if ( f->zsp->avail_in == 0 ) {
00207 /*
00208     We compressed everything and it sits in ziobuffer. Finish
00209 */
00210     return(0);
00211 }
00212 else {
00213     MLOCK(ErrorMessageLock);
00214     MesPrint("%w avail_in = %d, avail_out = %d.",f->zsp->avail_in,f->zsp->avail_out);
00215     MUNLOCK(ErrorMessageLock);
00216     break;
00217 }
00218 }
00219 MLOCK(ErrorMessageLock);
00220 MesPrint("%wError in gzip handling of output. zerror = %d",zerror);
00221 MUNLOCK(ErrorMessageLock);
00222 return(-1);
00223 }
00224 /*
00225 #] PutOutputGZIP :
00226 #[ FlushOutputGZIP :
00227
00228 Routine is called to flush a stream. The compression of the input buffer
00229 will be completed and the contents of f->ziobuffer will be written.
00230 Both buffers will be cleared.
00231 */
00232 int FlushOutputGZIP(FILEHANDLE *f)
00233 {
00234     GETIDENTITY
00235     int zerror;
00236 /*
00237     Set the proper parameters
00238 */
00239     f->zsp->next_in = (Bytef *) (f->PObuffer);
00240     f->zsp->avail_in = (Bytef *) (f->POfill) - (Bytef *) (f->PObuffer);
00241     f->zsp->total_in = 0;
00242     while ( ( zerror = deflate(f->zsp,Z_FINISH) ) == Z_OK ) {
00243         if ( f->zsp->avail_out == 0 ) {
00244             /*
00245                 Write the output
00246             */
00247             #ifdef GZIPDEBUG
00248                 MLOCK(ErrorMessageLock);
00249                 MesPrint("%wWriting %l bytes at %l0p",f->ziosize,&(f->POposition));
00250                 MUNLOCK(ErrorMessageLock);
00251             #endif
00252             #ifdef ALLLOCK
00253                 LOCK(f->pthreadlock);
00254             #endif
00255             if ( f == AR.hidefile ) {
00256                 UNLOCK(AS.inputslock);
00257             }
00258             SeekFile(f->handle,&(f->POposition),SEEK_SET);

```

```

00262         if ( WriteFile(f->handle, (UBYTE *) (f->ziobuffer), f->ziosize)
00263             != f->ziosize ) {
00264             if ( f == AR.hidefile ) {
00265                 UNLOCK(AS.inputslock);
00266             }
00267 #ifdef ALLLOCK
00268         UNLOCK(f->pthreadlock);
00269 #endif
00270         MLOCK(ErrorMessageLock);
00271         MesPrint("%wWrite error during compressed sort. Disk full?");
00272         MUNLOCK(ErrorMessageLock);
00273         return(-1);
00274     }
00275     if ( f == AR.hidefile ) {
00276         UNLOCK(AS.inputslock);
00277     }
00278 #ifdef ALLLOCK
00279     UNLOCK(f->pthreadlock);
00280 #endif
00281     ADDPOS(f->filesize, f->ziosize);
00282     ADDPOS(f->POposition, f->ziosize);
00283 #ifdef WITHPTHREADS
00284     if ( AS.MasterSort && AC.ThreadSortFileSynch ) {
00285         if ( f->handle >= 0 ) SynchFile(f->handle);
00286     }
00287 #endif
00288 /*
00289     Reset the output
00290 */
00291     f->zsp->next_out = f->ziobuffer;
00292     f->zsp->avail_out = f->ziosize;
00293     f->zsp->total_out = 0;
00294 }
00295 }
00296 if ( zerror == Z_STREAM_END ) {
00297 /*
00298     Write the output
00299 */
00300 #ifdef GZIPDEBUG
00301     MLOCK(ErrorMessageLock);
00302     MesPrint("%wWriting %l bytes at %l0p", (LONG) (f->zsp->avail_out), &(f->POposition));
00303     MUNLOCK(ErrorMessageLock);
00304 #endif
00305 #ifdef ALLLOCK
00306     LOCK(f->pthreadlock);
00307 #endif
00308     if ( f == AR.hidefile ) {
00309         LOCK(AS.inputslock);
00310     }
00311     SeekFile(f->handle, &(f->POposition), SEEK_SET);
00312     if ( WriteFile(f->handle, (UBYTE *) (f->ziobuffer), f->zsp->total_out)
00313         != (LONG) (f->zsp->total_out) ) {
00314         if ( f == AR.hidefile ) {
00315             UNLOCK(AS.inputslock);
00316         }
00317 #ifdef ALLLOCK
00318         UNLOCK(f->pthreadlock);
00319 #endif
00320         MLOCK(ErrorMessageLock);
00321         MesPrint("%wWrite error during compressed sort. Disk full?");
00322         MUNLOCK(ErrorMessageLock);
00323         return(-1);
00324     }
00325     if ( f == AR.hidefile ) {
00326         LOCK(AS.inputslock);
00327     }
00328 #ifdef ALLLOCK
00329     UNLOCK(f->pthreadlock);
00330 #endif
00331     ADDPOS(f->filesize, f->zsp->total_out);
00332     ADDPOS(f->POposition, f->zsp->total_out);
00333 #ifdef WITHPTHREADS
00334     if ( AS.MasterSort && AC.ThreadSortFileSynch ) {
00335         if ( f->handle >= 0 ) SynchFile(f->handle);
00336     }
00337 #endif
00338 #ifdef GZIPDEBUG
00339     MLOCK(ErrorMessageLock);
00340     { char *s = f->ziobuffer+f->zsp->total_out;
00341       MesPrint("%w    Last bytes written: %d %d %d %d %d", s[-5], s[-4], s[-3], s[-2], s[-1]);
00342     }
00343     MesPrint("%w    Perceived position in FlushOutputGZIP is %l0p", &(f->POposition));
00344     MUNLOCK(ErrorMessageLock);
00345 #endif
00346 /*
00347     Reset the output
00348 */

```

```

00349         f->zsp->next_out = f->ziobuffer;
00350         f->zsp->avail_out = f->ziosize;
00351         f->zsp->total_out = 0;
00352         if ( ( zerror = deflateEnd(f->zsp) ) == Z_OK ) return(0);
00353         MLOCK(ErrorMessageLock);
00354         if ( f->zsp->msg ) {
00355             MesPrint("%wError in finishing gzip handling of output: %s",f->zsp->msg);
00356         }
00357         else {
00358             MesPrint("%wError in finishing gzip handling of output.");
00359         }
00360         MUNLOCK(ErrorMessageLock);
00361     }
00362     else {
00363         MLOCK(ErrorMessageLock);
00364         MesPrint("%wError in gzip handling of output.");
00365         MUNLOCK(ErrorMessageLock);
00366     }
00367     return(-1);
00368 }
00369
00370 /*
00371  #] FlushOutputGZIP :
00372  #[ SetupAllInputGZIP :
00373
00374  Routine prepares all gzip input streams for a merge.
00375
00376  Problem (29-may-2008): If we never use GZIP compression, this routine
00377  will still allocate the array space. This is an enormous amount!
00378  It places an effective restriction on the value of SortIOSize
00379  */
00380
00381 int SetupAllInputGZIP(SORTING *S)
00382 {
00383     GETIDENTITY
00384     int i, NumberOpened = 0;
00385     z_stream zsp;
00386     /*
00387     This code was added 29-may-2008 by JV to prevent further processing if
00388     there is no compression at all (usually).
00389     */
00390     for ( i = 0; i < S->inNum; i++ ) {
00391         if ( S->fpincompressed[i] ) break;
00392     }
00393     if ( i >= S->inNum ) return(0);
00394
00395     if ( S->zsparray == 0 ) {
00396         S->zsparray = (z_stream*)Malloc1(sizeof(z_stream)*S->MaxFpatches,"input zstreams");
00397         if ( S->zsparray == 0 ) {
00398             MLOCK(ErrorMessageLock);
00399             MesCall("SetupAllInputGZIP");
00400             MUNLOCK(ErrorMessageLock);
00401             Terminate(-1);
00402         }
00403     }
00404     /*
00405     We add 128 bytes in the hope that if it can happen that it goes
00406     outside the buffer during decompression, it does not do damage.
00407     */
00408     AN.ziobuffers = (Bytef *)Malloc1(S->MaxFpatches*(S->file.ziosize+128)*sizeof(Bytef),"input raw
00409 buffers");
00410     /*
00411     This seems to be one of the really stupid errors:
00412     We allocate way too much space. Way way way too much.
00413     AN.ziobufnum = (Bytef **)Malloc1(S->MaxFpatches*S->file.ziosize*sizeof(Bytef *),"input raw
00414 pointers");
00415     */
00416     AN.ziobufnum = (Bytef **)Malloc1(S->MaxFpatches*sizeof(Bytef *),"input raw pointers");
00417     if ( AN.ziobuffers == 0 || AN.ziobufnum == 0 ) {
00418         MLOCK(ErrorMessageLock);
00419         MesCall("SetupAllInputGZIP");
00420         MUNLOCK(ErrorMessageLock);
00421         Terminate(-1);
00422     }
00423     for ( i = 0; i < S->MaxFpatches; i++ ) {
00424         AN.ziobufnum[i] = AN.ziobuffers + i * (S->file.ziosize+128);
00425     }
00426     for ( i = 0; i < S->inNum; i++ ) {
00427         #ifdef GZIPDEBUG
00428         MLOCK(ErrorMessageLock);
00429         MesPrint("%wPreparing z-stream %d with compression %d",i,S->fpincompressed[i]);
00430         MUNLOCK(ErrorMessageLock);
00431         #endif
00432         if ( S->fpincompressed[i] ) {
00433             zsp = &(S->zsparray[i]);
00434             /*
00435             1: Set the default fields:

```



```

00434 */
00435     zsp->zalloc = Z_NULL;
00436     zsp->zfree  = Z_NULL;
00437     zsp->opaque = Z_NULL;
00438 /*
00439     2: Set the output space:
00440 */
00441     zsp->next_out = Z_NULL;
00442     zsp->avail_out = 0;
00443     zsp->total_out = 0;
00444 /*
00445     3: Set the input space temporarily:
00446 */
00447     zsp->next_in = Z_NULL;
00448     zsp->avail_in = 0;
00449     zsp->total_in = 0;
00450 /*
00451     4: Initiate the inflation
00452 */
00453     if ( inflateInit(zsp) != Z_OK ) {
00454         MLOCK(ErrorMessageLock);
00455         if ( zsp->msg ) MesPrint("%wError from inflateInit: %s",zsp->msg);
00456         else           MesPrint("%wError from inflateInit");
00457         MesCall("SetupAllInputGZIP");
00458         MUNLOCK(ErrorMessageLock);
00459         Terminate(-1);
00460     }
00461     NumberOpened++;
00462 }
00463 }
00464 return (NumberOpened);
00465 }
00466
00467 /*
00468 #] SetupAllInputGZIP :
00469 #[ FillInputGZIP :
00470
00471 Routine is called when we need new input in the specified buffer.
00472 This buffer is used for the output and we keep reading and uncompressing
00473 input till either this buffer is full or the input stream is finished.
00474 The return value is the number of bytes in the buffer.
00475 */
00476
00477 LONG FillInputGZIP(FILEHANDLE *f, POSITION *position, UBYTE *buffer, LONG buffersize, int numstream)
00478 {
00479     GETIDENTITY
00480     int zerror;
00481     LONG readsize, toread = 0;
00482     SORTING *S = AT.SS;
00483     z_streamp zsp;
00484     POSITION pos;
00485     if ( S->fpincompressed[numstream] ) {
00486         zsp = &(S->zsparray[numstream]);
00487         zsp->next_out = (Bytef *)buffer;
00488         zsp->avail_out = buffersize;
00489         zsp->total_out = 0;
00490         if ( zsp->avail_in == 0 ) {
00491 /*
00492         First loading of the input
00493 */
00494         if ( ISGEPOSINC(S->fPatchesStop[numstream],*position,f->ziosize) ) {
00495             toread = f->ziosize;
00496         }
00497         else {
00498             DIFFPOS(pos,S->fPatchesStop[numstream],*position);
00499             toread = (LONG) (BASEPOSITION(pos));
00500         }
00501         if ( toread > 0 ) {
00502 #ifdef GZIPDEBUG
00503             MLOCK(ErrorMessageLock);
00504             MesPrint("%w-+Reading %l bytes in stream %d at position %l0p; stop at
00505 %l0p",toread,numstream,position,&(S->fPatchesStop[numstream]));
00506             MUNLOCK(ErrorMessageLock);
00507 #endif
00508 #ifdef ALLLOCK
00509             LOCK(f->pthreadlock);
00510 #endif
00511             SeekFile(f->handle,position,SEEK_SET);
00512             readsize = ReadFile(f->handle,(UBYTE *) (AN.ziobufnum[numstream]),toread);
00513             SeekFile(f->handle,position,SEEK_CUR);
00514 #ifdef ALLLOCK
00515             UNLOCK(f->pthreadlock);
00516 #endif
00517 #ifdef GZIPDEBUG
00518             MLOCK(ErrorMessageLock);
00519             { char *s = AN.ziobufnum[numstream]+readsize;
00520               MesPrint("%w read: %l +Last bytes read: %d %d %d %d %d in %s, newpos =

```

```

        %10p",readsize,s[-5],s[-4],s[-3],s[-2],s[-1],f->name,position);
00520     }
00521     MUNLOCK(ErrorMessageLock);
00522 #endif
00523     if ( readsize == 0 ) {
00524         zsp->next_in = AN.ziobufnum[numstream];
00525         zsp->avail_in = f->ziosize;
00526         zsp->total_in = 0;
00527         return(zsp->total_out);
00528     }
00529     if ( readsize < 0 ) {
00530         MLOCK(ErrorMessageLock);
00531         MesPrint("%wFillInputGZIP: Read error during compressed sort.");
00532         MUNLOCK(ErrorMessageLock);
00533         return(-1);
00534     }
00535     ADDPOS(f->filesize,readsize);
00536     ADDPOS(f->PPosition,readsize);
00537 /*
00538     Set the input
00539 */
00540     zsp->next_in = AN.ziobufnum[numstream];
00541     zsp->avail_in = readsize;
00542     zsp->total_in = 0;
00543 }
00544 }
00545 if ( toread > 0 || zsp->avail_in ) {
00546     while ( ( zerror = inflate(zsp,Z_NO_FLUSH) ) == Z_OK ) {
00547         if ( zsp->avail_out == 0 ) {
00548 /*
00549             Finish
00550 */
00551             return((LONG)(zsp->total_out));
00552         }
00553         if ( zsp->avail_in == 0 ) {
00554             if ( ISEQUALPOS(S->fPatchesStop[numstream],*position) ) {
00555 /*
00556                 We finished this stream. Try to terminate.
00557 */
00558                 zerror = Z_STREAM_END;
00559                 break;
00560             }
00561             if ( ( zerror = inflate(zsp,Z_SYNC_FLUSH) ) == Z_OK ) {
00562                 return((LONG)(zsp->total_out));
00563             }
00564             else {
00565                 break;
00566             }
00567 /*
00568 */
00569 #ifdef GZIPDEBUG
00570             MLOCK(ErrorMessageLock);
00571             MesPrint("%wClosing stream %d",numstream);
00572 #endif
00573             readsize = zsp->total_out;
00574 #ifdef GZIPDEBUG
00575             if ( readsize > 0 ) {
00576                 WORD *s = (WORD *) (buffer+zsp->total_out);
00577                 MesPrint("%w Last words: %d %d %d %d %d",s[-5],s[-4],s[-3],s[-2],s[-1]);
00578             }
00579             else {
00580                 MesPrint("%w No words");
00581             }
00582             MUNLOCK(ErrorMessageLock);
00583 #endif
00584             if ( ( zerror = inflateEnd(zsp) ) == Z_OK ) return(readsize);
00585             break;
00586 /*
00587             }
00588 */
00589             Read more input
00590 */
00591 #ifdef GZIPDEBUG
00592             if ( numstream == 0 ) {
00593                 MLOCK(ErrorMessageLock);
00594                 MesPrint("%wWant to read in stream 0 at position %10p",position);
00595                 MUNLOCK(ErrorMessageLock);
00596             }
00597 #endif
00598             if ( ISGEPOSINC(S->fPatchesStop[numstream],*position,f->ziosize) ) {
00599                 toread = f->ziosize;
00600             }
00601             else {
00602                 DIFPOS(pos,S->fPatchesStop[numstream],*position);
00603                 toread = (LONG)(BASEPOSITION(pos));
00604             }
00605 #ifdef GZIPDEBUG

```

```

00606             MLOCK(ErrorMessageLock);
00607             MesPrint("%w--Reading %l bytes in stream %d at position
%10p",toread,numstream,position);
00608             MUNLOCK(ErrorMessageLock);
00609 #endif
00610 #ifdef ALLLOCK
00611             LOCK(f->pthreadlock);
00612 #endif
00613             SeekFile(f->handle,position,SEEK_SET);
00614             readsize = ReadFile(f->handle,(UBYTE *) (AN.ziobufnum[numstream]),toread);
00615             SeekFile(f->handle,position,SEEK_CUR);
00616 #ifdef ALLLOCK
00617             UNLOCK(f->pthreadlock);
00618 #endif
00619 #ifdef GZIPDEBUG
00620             MLOCK(ErrorMessageLock);
00621             { char *s = AN.ziobufnum[numstream]+readsize;
00622               MesPrint("%w Last bytes read: %d %d %d %d %d",s[-5],s[-4],s[-3],s[-2],s[-1]);
00623             }
00624             MUNLOCK(ErrorMessageLock);
00625 #endif
00626             if ( readsize == 0 ) {
00627                 zsp->next_in = AN.ziobufnum[numstream];
00628                 zsp->avail_in = f->ziosize;
00629                 zsp->total_in = 0;
00630                 return(zsp->total_out);
00631             }
00632             if ( readsize < 0 ) {
00633                 MLOCK(ErrorMessageLock);
00634                 MesPrint("%wFillInputGZIP: Read error during compressed sort.");
00635                 MUNLOCK(ErrorMessageLock);
00636                 return(-1);
00637             }
00638             ADDPOS(f->filesize,readsize);
00639             ADDPOS(f->PPosition,readsize);
00640 /*
00641             Reset the input
00642 */
00643             zsp->next_in = AN.ziobufnum[numstream];
00644             zsp->avail_in = readsize;
00645             zsp->total_in = 0;
00646         }
00647         else {
00648             break;
00649         }
00650     }
00651 }
00652 else {
00653     zerror = Z_STREAM_END;
00654     zsp->total_out = 0;
00655 }
00656 #ifdef GZIPDEBUG
00657 MLOCK(ErrorMessageLock);
00658 MesPrint("%w zerror = %d in stream %d. At position %10p",zerror,numstream,position);
00659 MUNLOCK(ErrorMessageLock);
00660 #endif
00661 if ( zerror == Z_STREAM_END ) {
00662 /*
00663     Reset the input
00664 */
00665     zsp->next_in = Z_NULL;
00666     zsp->avail_in = 0;
00667     zsp->total_in = 0;
00668 /*
00669     Make the final call and finish
00670 */
00671 #ifdef GZIPDEBUG
00672 MLOCK(ErrorMessageLock);
00673 MesPrint("%wClosing stream %d",numstream);
00674 #endif
00675     readsize = zsp->total_out;
00676 #ifdef GZIPDEBUG
00677     if ( readsize > 0 ) {
00678         WORD *s = (WORD *) (buffer+zsp->total_out);
00679         MesPrint("%w -Last words: %d %d %d %d %d",s[-5],s[-4],s[-3],s[-2],s[-1]);
00680     }
00681     else {
00682         MesPrint("%w No words");
00683     }
00684     MUNLOCK(ErrorMessageLock);
00685 #endif
00686     if ( zsp->zalloc != Z_NULL ) {
00687         zerror = inflateEnd(zsp);
00688         zsp->zalloc = Z_NULL;
00689     }
00690     if ( zerror == Z_OK || zerror == Z_STREAM_END ) return(readsize);
00691 }

```

```

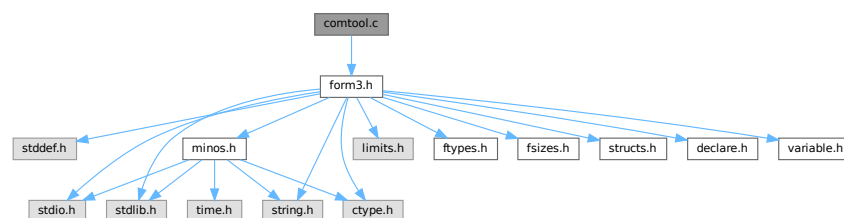
00692
00693     MLOCK(ErrorMessageLock);
00694     MesPrint("%wFillInputGZIP: Error in gzip handling of input. zerror = %d",zerror);
00695     MUNLOCK(ErrorMessageLock);
00696     return(-1);
00697 }
00698 else {
00699 #ifdef GZIPDEBUG
00700     MLOCK(ErrorMessageLock);
00701     MesPrint("%w++Reading %l bytes at position %l0p",bufferize,position);
00702     MUNLOCK(ErrorMessageLock);
00703 #endif
00704 #ifdef ALLLOCK
00705     LOCK(f->pthreadlock);
00706 #endif
00707     SeekFile(f->handle,position,SEEK_SET);
00708     readsize = ReadFile(f->handle,buffer,bufferize);
00709     SeekFile(f->handle,position,SEEK_CUR);
00710 #ifdef ALLLOCK
00711     UNLOCK(f->pthreadlock);
00712 #endif
00713     if ( readsize < 0 ) {
00714         MLOCK(ErrorMessageLock);
00715         MesPrint("%wFillInputGZIP: Read error during uncompressed sort.");
00716         MesPrint("%w++Reading %l bytes at position %l0p",bufferize,position);
00717         MUNLOCK(ErrorMessageLock);
00718     }
00719     return(readsize);
00720 }
00721 }
00722
00723 /*
00724 #] FillInputGZIP :
00725 #[ ClearSortGZIP :
00726 */
00727
00728 void ClearSortGZIP(FILEHANDLE *f)
00729 {
00730     if ( f->zibuffer ) {
00731         M_free(f->zibuffer,"output zbuffer");
00732         M_free(f->zsp,"output zstream");
00733         f->zibuffer = 0;
00734     }
00735 }
00736
00737 /*
00738 #] ClearSortGZIP :
00739 */
00740 #endif

```

4.15 comtool.c File Reference

```
#include "form3.h"
```

Include dependency graph for comtool.c:



Functions

- int [inicbufs](#) (void)
- void [finishcbuf](#) (WORD num)

- void [clearcbuf](#) (WORD num)
- WORD * [DoubleCbuffer](#) (int num, WORD *w, int par)
- WORD * [AddLHS](#) (int num)
- WORD * [AddRHS](#) (int num, int type)
- int [AddNtoL](#) (int n, WORD *array)
- int [AddNtoC](#) (int bufnum, int n, WORD *array, int par)
- int [InsTree](#) (int bufnum, int h)
- int [FindTree](#) (int bufnum, WORD *subexpr)
- void [RedoTree](#) (CBUF *C, int size)
- void [ClearTree](#) (int i)
- int [IniFbuffer](#) (WORD bufnum)
- LONG [numcommute](#) (WORD *terms, LONG *numterms)

4.15.1 Detailed Description

Utility routines for the compiler.

Definition in file [comtool.c](#).

4.15.2 Function Documentation

4.15.2.1 inicbufs()

```
int inicbufs (
    void )
```

Creates a new compiler buffer and returns its ID number.

Returns

The ID number for the new compiler buffer.

Definition at line [47](#) of file [comtool.c](#).

References [CbUf::boomlijst](#), [CbUf::Buffer](#), [CbUf::BufferSize](#), [CbUf::CanCommu](#), [CbUf::dimension](#), [CbUf::lhs](#), [CbUf::numdum](#), [CbUf::NumTerms](#), [CbUf::Pointer](#), [CbUf::rhs](#), and [CbUf::Top](#).

Referenced by [AddRHS\(\)](#), and [StartVariables\(\)](#).

4.15.2.2 finishcbuf()

```
void finishcbuf (
    WORD num )
```

Frees a compiler buffer.

Parameters

<i>num</i>	The ID number for the buffer to be freed.
------------	---

Definition at line 89 of file [comtool.c](#).

References [CbUf::boomlijst](#), [CbUf::Buffer](#), [CbUf::BufferSize](#), [CbUf::CanCommu](#), [CbUf::lhs](#), [CbUf::NumTerms](#), [CbUf::Pointer](#), [CbUf::rhs](#), and [CbUf::Top](#).

Referenced by [PF_BroadcastCBuf\(\)](#).

4.15.2.3 clearcbuf()

```
void clearcbuf (
    WORD num )
```

Clears contents in a compiler buffer.

Parameters

<i>num</i>	The ID number for the buffer to be cleared.
------------	---

Definition at line 116 of file [comtool.c](#).

References [CbUf::boomlijst](#), [CbUf::Buffer](#), [CbUf::Pointer](#), and [CbUf::rhs](#).

4.15.2.4 DoubleCbuffer()

```
WORD * DoubleCbuffer (
    int num,
    WORD * w,
    int par )
```

Doubles a compiler buffer.

Parameters

<i>num</i>	The ID number for the buffer to be doubled.
<i>w</i>	The pointer to the end (exclusive) of the current buffer. The contents in the range of [cbuff[num].Buffer,w) will be kept.

Definition at line 143 of file [comtool.c](#).

References [CbUf::Buffer](#), [CbUf::BufferSize](#), [CbUf::lhs](#), [CbUf::Pointer](#), [CbUf::rhs](#), and [CbUf::Top](#).

Referenced by [AddNtoC\(\)](#), and [AddNtoL\(\)](#).

4.15.2.5 AddLHS()

```
WORD * AddLHS (
    int num )
```

Adds an LHS to a compiler buffer and returns the pointer to a buffer for the new LHS.

Parameters

<i>num</i>	The ID number for the buffer to get another LHS.
------------	--

Definition at line 188 of file [comtool.c](#).

References [CbUf::lhs](#), and [CbUf::Pointer](#).

Referenced by [AddNtoL\(\)](#).

4.15.2.6 AddRHS()

```
WORD * AddRHS (
    int num,
    int type )
```

Adds an RHS to a compiler buffer and returns the pointer to a buffer for the new RHS.

Parameters

<i>num</i>	The ID number for the buffer to get another RHS.
<i>type</i>	If 0, the subexpression tree will be reallocated.

Definition at line 214 of file [comtool.c](#).

References [TaBIEs::buffers](#), [TaBIEs::buffersfill](#), [TaBIEs::bufferssize](#), [TaBIEs::bufnum](#), [CbUf::CanCommu](#), [CbUf::dimension](#), [inicbufs\(\)](#), [CbUf::numdum](#), [CbUf::NumTerms](#), [CbUf::Pointer](#), and [CbUf::rhs](#).

Referenced by [InsertArg\(\)](#), [StartVariables\(\)](#), and [TestMatch\(\)](#).

Here is the call graph for this function:

**4.15.2.7 AddNtoL()**

```
int AddNtoL (
    int n,
    WORD * array )
```

Adds an LHS with the given data to the current compiler buffer.

Parameters

<i>n</i>	The length of the data.
<i>array</i>	The data to be added.

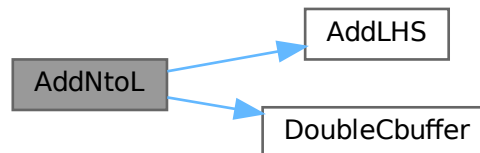
Returns

0 if succeeds.

Definition at line 288 of file [comtool.c](#).

References [AddLHS\(\)](#), [DoubleCbuffer\(\)](#), [CbUf::Pointer](#), and [CbUf::Top](#).

Here is the call graph for this function:

**4.15.2.8 AddNtoC()**

```

int AddNtoC (
    int bufnum,
    int n,
    WORD * array,
    int par )
  
```

Adds the given data to the last LHS/RHS in a compiler buffer.

Parameters

<i>bufnum</i>	The ID number for the buffer where the data will be added.
<i>n</i>	The length of the data.
<i>array</i>	The data to be added.

Returns

0 if succeeds.

Definition at line 317 of file [comtool.c](#).

References [DoubleCbuffer\(\)](#), [CbUf::Pointer](#), and [CbUf::Top](#).

Referenced by [InsertArg\(\)](#), and [StartVariables\(\)](#).

Here is the call graph for this function:



4.15.2.9 InsTree()

```
int InsTree (
    int bufnum,
    int h )
```

Definition at line [355](#) of file [comtool.c](#).

4.15.2.10 FindTree()

```
int FindTree (
    int bufnum,
    WORD * subexpr )
```

Definition at line [533](#) of file [comtool.c](#).

4.15.2.11 RedoTree()

```
void RedoTree (
    CBUF * C,
    int size )
```

Definition at line [569](#) of file [comtool.c](#).

4.15.2.12 ClearTree()

```
void ClearTree (
    int i )
```

Definition at line [588](#) of file [comtool.c](#).

4.15.2.13 IniFbuffer()

```
int IniFbuffer (
    WORD bufnum )
```

Initialize a factorization cache buffer. We set the size of the rhs and boomlijst buffers immediately to their final values.

Definition at line 614 of file [comtool.c](#).

References [tree::blnce](#), [CbUf::boomlijst](#), [CbUf::CanCommu](#), [CbUf::dimension](#), [tree::left](#), [CbUf::numdum](#), [CbUf::NumTerms](#), [tree::parent](#), [CbUf::rhs](#), [tree::right](#), [tree::usage](#), and [tree::value](#).

4.15.2.14 numcommute()

```
LONG numcommute (
    WORD * terms,
    LONG * numterms )
```

Definition at line 659 of file [comtool.c](#).

4.16 comtool.c

[Go to the documentation of this file.](#)

```
00001
00005 /* # [ License : */
00006 /*
00007 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00008 * When using this file you are requested to refer to the publication
00009 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00010 * This is considered a matter of courtesy as the development was paid
00011 * for by FOM the Dutch physics granting agency and we would like to
00012 * be able to track its scientific use to convince FOM of its value
00013 * for the community.
00014 *
00015 * This file is part of FORM.
00016 *
00017 * FORM is free software: you can redistribute it and/or modify it under the
00018 * terms of the GNU General Public License as published by the Free Software
00019 * Foundation, either version 3 of the License, or (at your option) any later
00020 * version.
00021 *
00022 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00023 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00024 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00025 * details.
00026 *
00027 * You should have received a copy of the GNU General Public License along
00028 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00029 */
00030 /* # [ License : */
00031 /*
00032 * # [ Includes :
00033 */
00034
00035 #include "form3.h"
00036
00037 /*
00038 * #] Includes :
00039 * # [ inicbufs :
00040 */
00041
00047 int inicbufs(void)
00048 {
00049     int i, num = AC.cbufList.num;
00050     CBUF *C = cbuf;
00051     for ( i = 0; i < num; i++, C++ ) {
00052         if ( C->Buffer == 0 ) break;
```

```

00053     }
00054     if ( i >= num ) C = (CBUF *)FromList(&AC.cbufList);
00055     else num = i;
00056     C->BufferSize = 2000;
00057     C->Buffer = (WORD *)Malloc1(C->BufferSize*sizeof(WORD),"compiler buffer-1");
00058     C->Pointer = C->Buffer;
00059     C->Top = C->Buffer + C->BufferSize;
00060     C->maxlhs = 10;
00061     C->lhs = (WORD **)Malloc1(C->maxlhs*sizeof(WORD *),"compiler buffer-2");
00062     C->numlhs = 0;
00063     C->mnumlhs = 0;
00064     C->maxrhs = 25;
00065     C->rhs = (WORD **)Malloc1(C->maxrhs*(sizeof(WORD *)+2*sizeof(LONG)+2*sizeof(WORD)),"compiler
buffer-3");
00066     C->CanCommu = (LONG *) (C->rhs+C->maxrhs);
00067     C->NumTerms = C->CanCommu+C->maxrhs;
00068     C->numdum = (WORD *) (C->NumTerms+C->maxrhs);
00069     C->dimension = C->numdum + C->maxrhs;
00070     C->numrhs = 0;
00071     C->mnumrhs = 0;
00072     C->rhs[0] = C->rhs[1] = C->Pointer;
00073     C->boomlijst = 0;
00074     RedoTree(C,C->maxrhs);
00075     ClearTree(num);
00076     return(num);
00077 }
00078
00079 /*
00080     #] inicbufs :
00081     #[ finishcbuf :
00082 */
00083
00089 void finishcbuf(WORD num)
00090 {
00091     CBUF *C = cbuf+num;
00092     if ( C->Buffer ) M_free(C->Buffer,"compiler buffer-1");
00093     if ( C->rhs ) M_free(C->rhs,"compiler buffer-3");
00094     if ( C->lhs ) M_free(C->lhs,"compiler buffer-2");
00095     if ( C->boomlijst ) M_free(C->boomlijst,"boomlijst");
00096     C->Top = C->Pointer = C->Buffer = 0;
00097     C->rhs = C->lhs = 0;
00098     C->CanCommu = 0;
00099     C->NumTerms = 0;
00100     C->BufferSize = 0;
00101     C->boomlijst = 0;
00102     C->numlhs = C->numrhs = C->maxlhs = C->maxrhs = C->mnumlhs =
00103     C->mnumrhs = C->numtree = C->rootnum = C->MaxTreeSize = 0;
00104 }
00105
00106 /*
00107     #] finishcbuf :
00108     #[ clearcbuf :
00109 */
00110
00116 void clearcbuf(WORD num)
00117 {
00118     CBUF *C = cbuf+num;
00119     if ( C->boomlijst ) M_free(C->boomlijst,"boomlijst");
00120     C->Pointer = C->Buffer;
00121     C->numrhs = C->numlhs = 0;
00122     C->mnumlhs = 0;
00123     C->boomlijst = 0;
00124     C->mnumrhs = 0;
00125     C->rhs[0] = C->rhs[1] = C->Pointer;
00126     C->numtree = C->rootnum = C->MaxTreeSize = 0;
00127     RedoTree(C,C->maxrhs);
00128     ClearTree(num);
00129 }
00130
00131 /*
00132     #] clearcbuf :
00133     #[ DoubleCbuffer :
00134 */
00135
00143 WORD *DoubleCbuffer(int num, WORD *w,int par)
00144 {
00145     CBUF *C = cbuf + num;
00146     LONG newsize = C->BufferSize*2;
00147     WORD *newbuffer = (WORD *)Malloc1(newsize*sizeof(WORD),"compiler buffer-4");
00148     WORD *wl, *w2;
00149     LONG offset, j, i;
00150     DUMMYUSE(par)
00151 /*
00152     MLOCK(ErrorMessageLock);
00153     MesPrint(" doubleCbuffer: par = %d",par);
00154     MUNLOCK(ErrorMessageLock);
00155 */

```

```

00156     w1 = C->Buffer; w2 = newbuffer;
00157     i = w - w1;
00158     j = i & 7;
00159     while ( --j >= 0 ) *w2++ = *w1++;
00160     i >= 3;
00161     while ( --i >= 0 ) {
00162         *w2++ = *w1++; *w2++ = *w1++; *w2++ = *w1++; *w2++ = *w1++;
00163         *w2++ = *w1++; *w2++ = *w1++; *w2++ = *w1++; *w2++ = *w1++;
00164     }
00165     offset = newbuffer - C->Buffer;
00166     for ( i = 0; i <= C->numlhs; i++ ) C->lhs[i] += offset;
00167     for ( i = 1; i <= C->numrhs; i++ ) C->rhs[i] += offset;
00168     w1 = C->Buffer;
00169     C->Pointer += offset;
00170     C->Top = newbuffer + newsize;
00171     C->BufferSize = newsize;
00172     C->Buffer = newbuffer;
00173     M_free(w1,"DoubleCbuffer");
00174     return(w2);
00175 }
00176
00177 /*
00178     #] DoubleCbuffer :
00179     #[ AddLHS :
00180 */
00181
00182 WORD *AddLHS(int num)
00183 {
00184     CBUF *C = cbuf + num;
00185     C->numlhs++;
00186     if ( C->numlhs >= (C->maxlhs-2) ) {
00187         WORD ***ppp = &(C->lhs); /* to avoid compiler warning */
00188         if ( DoubleList((void ***)ppp,&(C->maxlhs),sizeof(WORD *),
00189             "statement lists") ) Terminate(-1);
00190     }
00191     C->lhs[C->numlhs] = C->Pointer;
00192     C->lhs[C->numlhs+1] = 0;
00193     return(C->Pointer);
00194 }
00195
00196 /*
00197     #] AddLHS :
00198     #[ AddRHS :
00199 */
00200
00201 WORD *AddRHS(int num, int type)
00202 {
00203     LONG fullsize, *lold, newsize;
00204     int i;
00205     WORD *old, *wold;
00206     CBUF *C;
00207 restart:;
00208     C = cbuf + num;
00209     if ( C->numrhs >= (C->maxrhs-2) ) {
00210         if ( C->maxrhs == 0 ) newsize = 100;
00211         else newsize = C->maxrhs * 2;
00212         if ( newsize > MAXCOMBUFRHS ) newsize = MAXCOMBUFRHS;
00213         if ( newsize == C->maxrhs ) {
00214             if ( AC.tablefilling ) {
00215                 TABLES T = functions[AC.tablefilling].tabl;
00216                 /*
00217                     We add a compiler buffer, change a few settings and continue.
00218                 */
00219                 if ( T->buffersfill >= T->bufferssize ) {
00220                     int new1 = 2*T->bufferssize;
00221                     WORD *nbufs = (WORD *)Malloc1(new1*sizeof(WORD),"Table compile buffers");
00222                     for ( i = 0; i < T->buffersfill; i++ )
00223                         nbufs[i] = T->buffers[i];
00224                     for ( ; i < new1; i++ ) nbufs[i] = 0;
00225                     M_free(T->buffers,"Table compile buffers");
00226                     T->buffers = nbufs;
00227                     T->bufferssize = new1;
00228                 }
00229                 T->buffers[T->buffersfill++] = T->bufnum = inicbufs();
00230                 AC.cbufnum = num = T->bufnum;
00231                 goto restart;
00232             }
00233             else {
00234                 MesPrint("@Compiler buffer overflow. Try to make modules smaller");
00235                 Terminate(-1);
00236             }
00237         }
00238     }
00239     old = C->rhs;
00240     fullsize = newsize * (sizeof(WORD *) + 2*sizeof(LONG) + 2*sizeof(WORD));
00241     C->rhs = (WORD **)Malloc1(fullsize,"subexpression lists");
00242     for ( i = 0; i < C->maxrhs; i++ ) C->rhs[i] = old[i];
00243     lold = C->CanCommu; C->CanCommu = (LONG *) (C->rhs+newsize);

```

```

00256         for ( i = 0; i < C->maxrhs; i++ ) C->CanCommu[i] = lold[i];
00257         lold = C->NumTerms; C->NumTerms = (LONG *) (C->rhs+2*newsize);
00258         for ( i = 0; i < C->maxrhs; i++ ) C->NumTerms[i] = lold[i];
00259         wold = C->numdum; C->numdum = (WORD *) (C->NumTerms+newsize);
00260         for ( i = 0; i < C->maxrhs; i++ ) C->numdum[i] = wold[i];
00261         wold = C->dimension; C->dimension = (WORD *) (C->numdum+newsize);
00262         for ( i = 0; i < C->maxrhs; i++ ) C->dimension[i] = wold[i];
00263         if ( old ) M_free(old,"subexpression lists");
00264         C->maxrhs = newsize;
00265         if ( type == 0 ) RedoTree(C,C->maxrhs);
00266     }
00267     C->numrhs++;
00268     C->CanCommu[C->numrhs] = 0;
00269     C->NumTerms[C->numrhs] = 0;
00270     C->numdum[C->numrhs] = 0;
00271     C->dimension[C->numrhs] = 0;
00272     C->rhs[C->numrhs] = C->Pointer;
00273     return(C->Pointer);
00274 }
00275
00276 /*
00277  #] AddRHS :
00278  #[ AddNtoL :
00279  */
00280
00281 int AddNtoL(int n, WORD *array)
00282 {
00290     int i;
00291     CBUF *C = cbuf+AC.cbufnum;
00292     #ifdef COMPBUFFDEBUG
00293         MesPrint("LH: %a",n,array);
00294     #endif
00295     AddLHS(AC.cbufnum);
00296     while ( C->Pointer+n >= C->Top ) DoubleCbuffer(AC.cbufnum,C->Pointer,1);
00297     for ( i = 0; i < n; i++ ) *(C->Pointer)++ = *array++;
00298     return(0);
00299 }
00300
00301 /*
00302  #] AddNtoL :
00303  #[ AddNtoC :
00304
00305     Commentary: added the bufnum on 14-sep-2010 to make the whole a bit
00306     more flexible (JV). Still to do with AddNtoL.
00307  */
00308
00309 int AddNtoC(int bufnum, int n, WORD *array,int par)
00310 {
00319     int i;
00320     WORD *w;
00321     CBUF *C = cbuf+bufnum;
00322     #ifdef COMPBUFFDEBUG
00323         MesPrint("RH: %a",n,array);
00324     #endif
00325     while ( C->Pointer+n+1 >= C->Top ) DoubleCbuffer(bufnum,C->Pointer,50+par);
00326     w = C->Pointer;
00327     for ( i = 0; i < n; i++ ) *w++ = *array++;
00328     C->Pointer = w;
00329     return(0);
00330 }
00331
00332 /*
00333  #] AddNtoC :
00334  #[ InsTree :
00335
00336     Routines for balanced tree searching and insertion.
00337     Compared to Knuth we have a parent link. This minimizes the
00338     number of compares. That is better for anything that is more
00339     complicated than just single numbers.
00340     There are no provisions for removing elements from the tree.
00341     The routines are:
00342     void RedoTree(size) Re-allocates the tree space. There will
00343                         be MaxTreeSize = size elements.
00344     void ClearTree() Prunes the tree down to the root element.
00345     int InsTree(int,int) Searches for the requested element. If not found it
00346                         will allocate a new element, balance the tree if
00347                         necessary and return the called number.
00348                         If it was in the tree, it returns the tree 'value'.
00349
00350     Commentary: added the bufnum on 14-sep-2010 to make the whole a bit
00351     more flexible (JV).
00352  */
00353 static COMPTREE comptreezero = {0,0,0,0,0,0};
00354
00355 int InsTree(int bufnum, int h)
00356 {
00357     CBUF *C = cbuf + bufnum;

```

```

00358     COMPTREE *boomlijst = C->boomlijst, *q = boomlijst + C->rootnum, *p, *s;
00359     WORD *v1, *v2, *v3;
00360     int ip, iq, is;
00361
00362     if ( C->numtree + 1 >= C->MaxTreeSize ) {
00363         if ( C->MaxTreeSize == 0 ) {
00364             COMPTREE *root;
00365             C->MaxTreeSize = 125;
00366             C->boomlijst = (COMPTREE *)Malloc1((C->MaxTreeSize+1)*sizeof(COMPTREE),
00367                 "ClearInsTree");
00368             root = C->boomlijst;
00369             C->numtree = 0;
00370             C->rootnum = 0;
00371             root->left = -1;
00372             root->right = -1;
00373             root->parent = -1;
00374             root->blnce = 0;
00375             root->value = -1;
00376             root->usage = 0;
00377             for ( ip = 1; ip < C->MaxTreeSize; ip++ ) { C->boomlijst[ip] = comptreezero; }
00378         }
00379         else {
00380             is = C->MaxTreeSize * 2;
00381             s = (COMPTREE *)Malloc1((is+1)*sizeof(COMPTREE), "InsTree");
00382             for ( ip = 0; ip < C->MaxTreeSize; ip++ ) { s[ip] = C->boomlijst[ip]; }
00383             for ( ip = C->MaxTreeSize; ip <= is; ip++ ) { s[ip] = comptreezero; }
00384             if ( C->boomlijst ) M_free(C->boomlijst, "InsTree");
00385             C->boomlijst = s;
00386             C->MaxTreeSize = is;
00387         }
00388         boomlijst = C->boomlijst;
00389         q = boomlijst + C->rootnum;
00390     }
00391
00392     if ( q->right == -1 ) { /* First element */
00393         C->numtree++;
00394         s = boomlijst+C->numtree;
00395         q->right = C->numtree;
00396         s->parent = C->rootnum;
00397         s->left = s->right = -1;
00398         s->blnce = 0;
00399         s->value = h;
00400         s->usage = 1;
00401         return(h);
00402     }
00403     ip = q->right;
00404     while ( ip >= 0 ) {
00405         p = boomlijst + ip;
00406         v1 = C->rhs[p->value]; v2 = v3 = C->rhs[h];
00407         while ( *v3 ) v3 += *v3; /* find the 0 that indicates end-of-expr */
00408         while ( *v1 == *v2 && v2 < v3 ) { v1++; v2++; }
00409         if ( *v1 > *v2 ) {
00410             iq = p->right;
00411             if ( iq >= 0 ) { ip = iq; }
00412             else {
00413                 C->numtree++;
00414                 is = C->numtree;
00415                 p->right = is;
00416                 s = boomlijst + is;
00417                 s->parent = ip; s->left = s->right = -1;
00418                 s->blnce = 0; s->value = h; s->usage = 1;
00419                 p->blnce++;
00420                 if ( p->blnce == 0 ) return(h);
00421                 goto balance;
00422             }
00423         }
00424         else if ( *v1 < *v2 ) {
00425             iq = p->left;
00426             if ( iq >= 0 ) { ip = iq; }
00427             else {
00428                 C->numtree++;
00429                 is = C->numtree;
00430                 s = boomlijst+is;
00431                 p->left = is;
00432                 s->parent = ip; s->left = s->right = -1;
00433                 s->blnce = 0; s->value = h; s->usage = 1;
00434                 p->blnce--;
00435                 if ( p->blnce == 0 ) return(h);
00436                 goto balance;
00437             }
00438         }
00439         else {
00440             p->usage++;
00441             return(p->value);
00442         }
00443     }
00444     MesPrint("We vallen uit de boom!");

```

```

00445     Terminate(-1);
00446     return(h);
00447 balance;;
00448     for (;;) {
00449         p = boomlijst + ip;
00450         iq = p->parent;
00451         if ( iq == C->rootnum ) break;
00452         q = boomlijst + iq;
00453         if ( ip == q->left ) q->blnce--;
00454         else q->blnce++;
00455         if ( q->blnce == 0 ) break;
00456         if ( q->blnce == -2 ) {
00457             if ( p->blnce == -1 ) { /* single rotation */
00458                 q->left = p->right;
00459                 p->right = iq;
00460                 p->parent = q->parent;
00461                 q->parent = ip;
00462                 if ( boomlijst[p->parent].left == iq ) boomlijst[p->parent].left = ip;
00463                 else boomlijst[p->parent].right = ip;
00464                 if ( q->left >= 0 ) boomlijst[q->left].parent = iq;
00465                 q->blnce = p->blnce = 0;
00466             }
00467             else { /* double rotation */
00468                 s = boomlijst + is;
00469                 q->left = s->right;
00470                 p->right = s->left;
00471                 s->right = iq;
00472                 s->left = ip;
00473                 if ( p->right >= 0 ) boomlijst[p->right].parent = ip;
00474                 if ( q->left >= 0 ) boomlijst[q->left].parent = iq;
00475                 s->parent = q->parent;
00476                 q->parent = is;
00477                 p->parent = is;
00478                 if ( boomlijst[s->parent].left == iq )
00479                     boomlijst[s->parent].left = is;
00480                 else boomlijst[s->parent].right = is;
00481                 if ( s->blnce > 0 ) { q->blnce = s->blnce = 0; p->blnce = -1; }
00482                 else if ( s->blnce < 0 ) { p->blnce = s->blnce = 0; q->blnce = 1; }
00483                 else { p->blnce = s->blnce = q->blnce = 0; }
00484             }
00485             break;
00486         }
00487         else if ( q->blnce == 2 ) {
00488             if ( p->blnce == 1 ) { /* single rotation */
00489                 q->right = p->left;
00490                 p->left = iq;
00491                 p->parent = q->parent;
00492                 q->parent = ip;
00493                 if ( boomlijst[p->parent].left == iq ) boomlijst[p->parent].left = ip;
00494                 else boomlijst[p->parent].right = ip;
00495                 if ( q->right >= 0 ) boomlijst[q->right].parent = iq;
00496                 q->blnce = p->blnce = 0;
00497             }
00498             else { /* double rotation */
00499                 s = boomlijst + is;
00500                 q->right = s->left;
00501                 p->left = s->right;
00502                 s->left = iq;
00503                 s->right = ip;
00504                 if ( p->left >= 0 ) boomlijst[p->left].parent = ip;
00505                 if ( q->right >= 0 ) boomlijst[q->right].parent = iq;
00506                 s->parent = q->parent;
00507                 q->parent = is;
00508                 p->parent = is;
00509                 if ( boomlijst[s->parent].left == iq ) boomlijst[s->parent].left = is;
00510                 else boomlijst[s->parent].right = is;
00511                 if ( s->blnce < 0 ) { q->blnce = s->blnce = 0; p->blnce = 1; }
00512                 else if ( s->blnce > 0 ) { p->blnce = s->blnce = 0; q->blnce = -1; }
00513                 else { p->blnce = s->blnce = q->blnce = 0; }
00514             }
00515             break;
00516         }
00517         is = ip; ip = iq;
00518     }
00519     return(h);
00520 }
00521
00522 /*
00523  #] InsTree :
00524  #[ FindTree :
00525
00526  Routines for balanced tree searching.
00527  Is like InsTree but without the insertions.
00528  Returns -1 if the element is not in the tree.
00529  The advantage of this routine over InsTree is that this routine
00530  can be run in parallel.
00531 */

```

```

00532
00533 int FindTree(int bufnum, WORD *subexpr)
00534 {
00535     CBUF *C = cbuf + bufnum;
00536     COMPTREE *boomlijst = C->boomlijst, *q = boomlijst + C->rootnum, *p;
00537     WORD *v1, *v2, *v3;
00538     int ip, iq;
00539
00540     ip = q->right;
00541     while ( ip >= 0 ) {
00542         p = boomlijst + ip;
00543         v1 = C->rhs[p->value]; v2 = v3 = subexpr;
00544         while ( *v3 ) v3 += *v3; /* find the 0 that indicates end-of-expr */
00545         while ( *v1 == *v2 && v2 < v3 ) { v1++; v2++; }
00546         if ( *v1 > *v2 ) {
00547             iq = p->right;
00548             if ( iq >= 0 ) { ip = iq; }
00549             else { return(-1); }
00550         }
00551         else if ( *v1 < *v2 ) {
00552             iq = p->left;
00553             if ( iq >= 0 ) { ip = iq; }
00554             else { return(-1); }
00555         }
00556         else {
00557             p->usage++;
00558             return(p->value);
00559         }
00560     }
00561     return(-1);
00562 }
00563
00564 /*
00565 #] FindTree :
00566 #[ RedoTree :
00567 */
00568
00569 void RedoTree(CBUF *C, int size)
00570 {
00571     COMPTREE *newboomlijst;
00572     int i;
00573     newboomlijst = (COMPTREE *)Malloc1((size+1)*sizeof(COMPTREE), "newboomlijst");
00574     if ( C->boomlijst ) {
00575         if ( C->MaxTreeSize > size ) C->MaxTreeSize = size;
00576         for ( i = 0; i < C->MaxTreeSize; i++ ) newboomlijst[i] = C->boomlijst[i];
00577         M_free(C->boomlijst, "boomlijst");
00578     }
00579     C->boomlijst = newboomlijst;
00580     C->MaxTreeSize = size;
00581 }
00582
00583 /*
00584 #] RedoTree :
00585 #[ ClearTree :
00586 */
00587
00588 void ClearTree(int i)
00589 {
00590     CBUF *C = cbuf + i;
00591     COMPTREE *root = C->boomlijst;
00592     if ( root ) {
00593         C->numtree = 0;
00594         C->rootnum = 0;
00595         root->left = -1;
00596         root->right = -1;
00597         root->parent = -1;
00598         root->blnce = 0;
00599         root->value = -1;
00600         root->usage = 0;
00601     }
00602 }
00603
00604 /*
00605 #] ClearTree :
00606 #[ IniFbuffer :
00607 */
00614 int IniFbuffer(WORD bufnum)
00615 {
00616     CBUF *C = cbuf + bufnum;
00617     COMPTREE *root;
00618     int i;
00619     LONG fullsize;
00620     C->maxrhs = AM.fbuffersize;
00621     C->MaxTreeSize = AM.fbuffersize;
00622
00623     /*
00624     * Note that bufnum is a return value of inicbufs(). So C has been already

```



```

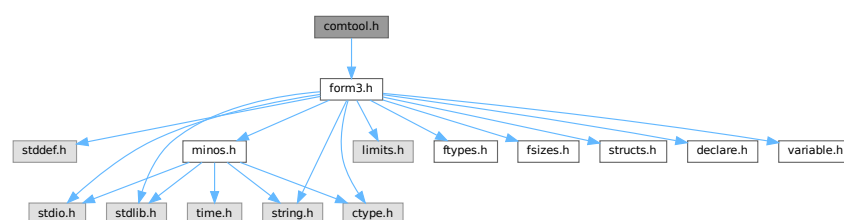
00625     * initialized. (TU 20 Dec 2011)
00626     */
00627     if ( C->boomlijst ) M_free(C->boomlijst, "IniFbuffer-tree");
00628     if ( C->rhs ) M_free(C->rhs, "IniFbuffer-rhs");
00629
00630     C->boomlijst = (COMPTREE *)Malloc1((C->MaxTreeSize+1)*sizeof(COMPTREE),"IniFbuffer-tree");
00631     root = C->boomlijst;
00632     C->numtree = 0;
00633     C->rootnum = 0;
00634     root->left = -1;
00635     root->right = -1;
00636     root->parent = -1;
00637     root->blnce = 0;
00638     root->value = -1;
00639     root->usage = 0;
00640     for ( i = 1; i < C->MaxTreeSize; i++ ) { C->boomlijst[i] = comptreezero; }
00641
00642     fullsize = (C->maxrhs+1) * (sizeof(WORD *) + 2*sizeof(LONG) + 2*sizeof(WORD));
00643     C->rhs = (WORD **)Malloc1(fullsize,"IniFbuffer-rhs");
00644     C->CanCommu = (LONG *) (C->rhs+C->maxrhs);
00645     C->NumTerms = (LONG *) (C->rhs+2*C->maxrhs);
00646     C->numdum = (WORD *) (C->NumTerms+C->maxrhs);
00647     C->dimension = (WORD *) (C->numdum+C->maxrhs);
00648
00649     return(0);
00650 }
00651
00652 /*
00653  #] IniFbuffer :
00654  #[ numcommute :
00655
00656  Returns the number of non-commuting terms in the expression
00657  */
00658
00659 LONG numcommute(WORD *terms, LONG *numterms)
00660 {
00661     LONG num = 0;
00662     WORD *t, *m;
00663     *numterms = 0;
00664     while ( *terms ) {
00665         *numterms += 1;
00666         t = terms + 1;
00667         GETSTOP(terms,m);
00668         while ( t < m ) {
00669             if ( *t >= FUNCTION ) {
00670                 if ( functions[*t-FUNCTION].commute ) { num++; break; }
00671             }
00672             t += t[1];
00673         }
00674         terms = terms + *terms;
00675     }
00676     return(num);
00677 }
00678
00679 /*
00680  #] numcommute :
00681  */

```

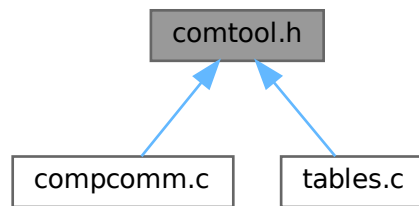
4.17 comtool.h File Reference

```
#include "form3.h"
```

Include dependency graph for comtool.h:



This graph shows which files directly or indirectly include this file:



4.17.1 Detailed Description

Utility routines for the compiler.

Definition in file [comtool.h](#).

4.18 comtool.h

[Go to the documentation of this file.](#)

```

00001
00005 /* #[] License : */
00006 /*
00007  * Copyright (C) 1984-2023 J.A.M. Vermaseren
00008  * When using this file you are requested to refer to the publication
00009  * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00010  * This is considered a matter of courtesy as the development was paid
00011  * for by FOM the Dutch physics granting agency and we would like to
00012  * be able to track its scientific use to convince FOM of its value
00013  * for the community.
00014  *
00015  * This file is part of FORM.
00016  *
00017  * FORM is free software: you can redistribute it and/or modify it under the
00018  * terms of the GNU General Public License as published by the Free Software
00019  * Foundation, either version 3 of the License, or (at your option) any later
00020  * version.
00021  *
00022  * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00023  * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00024  * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00025  * details.
00026  *
00027  * You should have received a copy of the GNU General Public License along
00028  * with FORM. If not, see <http://www.gnu.org/licenses/>.
00029  */
00030 /* #[] License : */
00031 #ifndef FORM_COMTOOL_H_
00032 #define FORM_COMTOOL_H_
00033 /*
00034  * #[] Includes :
00035  */
00036 #include "form3.h"
00037
00038 /*
00039  * #[] Includes :
00040  * #[] Inline functions :
00041  */
00042
00043 static inline void SkipSpaces(UBYTE **s)
00051 {
  
```

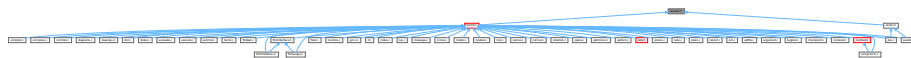
```

00052     const char *p = (const char *)s;
00053     while ( *p == ' ' || *p == ',' || *p == '\\t' ) p++;
00054     *s = (UBYTE *)p;
00055 }
00056
00066 static inline int ConsumeOption(UBYTE **s, const char *opt)
00067 {
00068     const char *p = (const char *)s;
00069     while ( *p && *opt && tolower(*p) == tolower(*opt) ) {
00070         p++;
00071         opt++;
00072     }
00073     /* Check if `opt` ended. */
00074     if ( !*opt ) {
00075         /* Check if `*p` is a word boundary. */
00076         if ( !*p || !(FG.cTable[(unsigned char)*p] == 0 ||
00077             FG.cTable[(unsigned char)*p] == 1 || *p == '_' ||
00078             *p == '$') ) {
00079             /* Consume the option. Skip the trailing spaces. */
00080             *s = (UBYTE *)p;
00081             SkipSpaces(s);
00082             return(1);
00083         }
00084     }
00085     return(0);
00086 }
00087
00088 /*
00089  #] Inline functions :
00090  */
00091 #endif /* FORM_COMTOOL_H_ */

```

4.19 declare.h File Reference

This graph shows which files directly or indirectly include this file:



Macros

- #define **MaX**(x, y) ((x) > (y) ? (x): (y))
- #define **MiN**(x, y) ((x) < (y) ? (x): (y))
- #define **ABS**(x) ((x) < 0 ? -(x): (x))
- #define **SGN**(x) ((x) > 0 ? 1 : (x) < 0 ? -1 : 0)
- #define **REDLENG**(x) (((x)<0)?((x)+1):((x)-1))/2)
- #define **INCLENG**(x) (((x)<0)?(((x)*2)-1):(((x)*2)+1))
- #define **GETCOEF**(x, y) x += *x; y = x[-1]; x -= ABS(y); y = REDLENG(y)
- #define **GETSTOP**(x, y) y = x+(*x)-1; y -= ABS(*y)-1
- #define **StuffAdd**(x, y) (((x)<0?-1:1)*y)+((y)<0?-1:1)*x)
- #define **EXCHN**(t1, t2, n) { WORD a,i; for(i=0;i<n;i++){a=t1[i];t1[i]=t2[i];t2[i]=a; } }
- #define **EXCH**(x, y) { WORD a = (x); (x) = (y); (y) = a; }
- #define **TOKENTOLINE**(x, y)
- #define **UngetFromStream**(stream, c) ((stream)->nextchar[(stream)->isnextchar++]=c)
- #define **AddLineFeed**(s, n) { (s)[(n)++] = LINEFEED; }
- #define **TryRecover**(x) Terminate(-1)
- #define **UngetChar**(c) { pushbackchar = c; }
- #define **ParseNumber**(x, s) {(x)=0;while(*s)>='0'&&*s<='9')(x)=10*(x)+*(s)++-'0';}
- #define **ParseSign**(sgn, s)
- #define **ParseSignedNumber**(x, s)
- #define **NCOPY**(s, t, n) { while ((n)-- > 0) { *s++ = *t++; } }
- #define **NCOPYI**(s, t, n) { while ((n)-- > 0) { *s++ = *t++; } }

- `#define NCOPYB(s, t, n) { while ( (n)-- > 0 ) { *s++ = *t++; } }`
- `#define NCOPYI32(s, t, n) { while ( (n)-- > 0 ) { *s++ = *t++; } }`
- `#define WCOPY(s, t, n) { int nn=n; WORD *ss=(WORD *)s, *tt=(WORD *)t; while ( (nn)-- > 0 ) { *ss++ = *tt++; } }`
- `#define NeedNumber(x, s, err)`
- `#define SKIPBLANKS(s) { while ( *(s) == ' ' || *(s) == '\t' ) (s)++; }`
- `#define FLUSHCONSOLE if ( AP.InOutBuf > 0 ) CharOut(LINEFEED)`
- `#define SKIPBRA1(s)`
- `#define SKIPBRA2(s)`
- `#define SKIPBRA3(s)`
- `#define SKIPBRA4(s)`
- `#define SKIPBRA5(s)`
- `#define CYCLE1(t, a, i) { t iX=*a; WORD jX; for(jX=1;jX<i;jX++)a[jX-1]=a[jX]; a[i-1]=iX; }`
- `#define AddToCB(c, wx)`
- `#define EXCHINOUT`
- `#define BACKINOUT`
- `#define CopyArg(to, from)`
- `#define FILLARG(w)`
- `#define COPYARG(w, t)`
- `#define ZEROARG(w)`
- `#define FILLFUN(w)`
- `#define COPYFUN(w, t)`
- `#define COPYFUN3(w, t)`
- `#define FILLFUN3(w)`
- `#define FILLSUB(w)`
- `#define COPYSUB(w, ww)`
- `#define FILLEXPR(w)`
- `#define NEXTARG(x) if(*x>0) x += *x; else if(*x <= -FUNCTION)x++; else x += 2;`
- `#define COPY1ARG(s1, t1)`
- `#define ZeroFillRange(w, begin, end)`
- `#define TABLESIZE(a, b) (((WORD)sizeof(a))/((WORD)sizeof(b)))`
- `#define WORDDIF(x, y) (WORD)(x-y)`
- `#define wsizeof(a) ((WORD)sizeof(a))`
- `#define VARNAME(type, num) (AC.varnames->namebuffer+type[num].name)`
- `#define DOLLARNAME(type, num) (AC.dollarnames->namebuffer+type[num].name)`
- `#define EXPRNAME(num) (AC.exprnames->namebuffer+Expressions[num].name)`
- `#define PREV(x) prevorder?prevorder:x`
- `#define Terminate(x) do { TerminateImpl(x, __FILE__, __LINE__, __FUNCTION__); } while(0)`
- `#define SETERROR(x) { Terminate(-1); return(-1); }`
- `#define DUMMYUSE(x) (void)(x);`
- `#define ADDPOS(pp, x) (pp).p1 = ((pp).p1+(LONG)(x))`
- `#define SETBASELENGTH(ss, x) (ss).p1 = (LONG)(x)`
- `#define SETBASEPOSITION(pp, x) (pp).p1 = (LONG)(x)`
- `#define ISEQUALPOSINC(pp1, pp2, x) ( (pp1).p1 == ((pp2).p1+(LONG)(x)) )`
- `#define ISGEPOSINC(pp1, pp2, x) ( (pp1).p1 >= ((pp2).p1+(LONG)(x)) )`
- `#define DIVPOS(pp, n) ( (pp).p1/(LONG)(n) )`
- `#define MULPOS(pp, n) (pp).p1 *= (LONG)(n)`
- `#define DIFPOS(ss, pp1, pp2) (ss).p1 = ((pp1).p1-(pp2).p1)`
- `#define DIFBASE(pp1, pp2) ((pp1).p1-(pp2).p1)`
- `#define ADD2POS(pp1, pp2) (pp1).p1 += (pp2).p1`
- `#define PUTZERO(pp) (pp).p1 = 0`
- `#define BASEPOSITION(pp) ((pp).p1)`
- `#define SETSTARTPOS(pp) (pp).p1 = -2`
- `#define NOTSTARTPOS(pp) ( (pp).p1 > -2 )`
- `#define ISMINPOS(pp) ( (pp).p1 == -1 )`

- #define [ISEQUALPOS](#)(pp1, pp2) ((pp1).p1 == (pp2).p1)
- #define [ISNOTEQUALPOS](#)(pp1, pp2) ((pp1).p1 != (pp2).p1)
- #define [ISLESSPOS](#)(pp1, pp2) ((pp1).p1 < (pp2).p1)
- #define [ISGEPOS](#)(pp1, pp2) ((pp1).p1 >= (pp2).p1)
- #define [ISNOTZEROPOS](#)(pp) ((pp).p1 != 0)
- #define [ISZEROPOS](#)(pp) ((pp).p1 == 0)
- #define [ISPOSP](#)(pp) ((pp).p1 > 0)
- #define [ISNEGPOS](#)(pp) ((pp).p1 < 0)
- #define [TOLONG](#)(x) ((LONG)(x))
- #define [Add2Com](#)(x) { WORD cod[2]; cod[0] = x; cod[1] = 2; [AddNtoL](#)(2,cod); }
- #define [Add3Com](#)(x1, x2) { WORD cod[3]; cod[0] = x1; cod[1] = 3; cod[2] = x2; [AddNtoL](#)(3,cod); }
- #define [Add4Com](#)(x1, x2, x3)
- #define [Add5Com](#)(x1, x2, x3, x4)
- #define [WantAddPointers](#)(x)
- #define [WantAddLongs](#)(x)
- #define [WantAddPositions](#)(x)
- #define [FORM_INLINE](#) inline
- #define [MEMORYMACROS](#)
- #define [TermMalloc](#)(x) ((AT.TermMemTop <= 0) ? TermMallocAddMemory(BHEAD0), AT.TermMemHeap[--AT.TermMemTop]: AT.TermMemHeap[--AT.TermMemTop])
- #define [NumberMalloc](#)(x) ((AT.NumberMemTop <= 0) ? NumberMallocAddMemory(BHEAD0), AT.NumberMemHeap[--AT.NumberMemTop]: AT.NumberMemHeap[--AT.NumberMemTop])
- #define [CacheNumberMalloc](#)(x) ((AT.CacheNumberMemTop <= 0) ? CacheNumberMallocAddMemory(BHEAD0), AT.CacheNumberMemHeap[--AT.CacheNumberMemTop]: AT.CacheNumberMemHeap[--AT.CacheNumberMemTop])
- #define [TermFree](#)(TermMem, x) AT.TermMemHeap[AT.TermMemTop++] = (WORD *) (TermMem)
- #define [NumberFree](#)(NumberMem, x) AT.NumberMemHeap[AT.NumberMemTop++] = (UWORD *) (NumberMem)
- #define [CacheNumberFree](#)(NumberMem, x) AT.CacheNumberMemHeap[AT.CacheNumberMemTop++] = (UWORD *) (NumberMem)
- #define [NestingChecksum](#)() (AC.IfLevel + AC.RepLevel + AC.arglevel + AC.insidelevel + AC.termlevel + AC.inexprlevel + AC.dolooplevel + AC.SwitchLevel)
- #define [MesNesting](#)() MesPrint("&Illegal nesting of if, repeat, argument, inside, term, inexpression and do")
- #define [MarkPolyRatFunDirty](#)(T)
- #define [PUSHPREASSIGNLEVEL](#)
- #define [POPPREASSIGNLEVEL](#)
- #define [EXTERNLOCK](#)(x)
- #define [INILOCK](#)(x)
- #define [LOCK](#)(x)
- #define [UNLOCK](#)(x)
- #define [EXTERNRWLOCK](#)(x)
- #define [INIRWLOCK](#)(x)
- #define [RWLOCKR](#)(x)
- #define [RWLOCKW](#)(x)
- #define [UNRWLOCK](#)(x)
- #define [MLOCK](#)(x)
- #define [MUNLOCK](#)(x)
- #define [GETIDENTITY](#)
- #define [GETBIDENTITY](#)
- #define [M_alloc](#)(x) malloc((size_t)(x))
- #define [CompareTerms](#) ((COMPARE)AR.CompareRoutine)
- #define [FiniShuffle](#) AN.SHvar.finishuf
- #define [DoShuffle](#) ((DO_UFFLE)AN.SHvar.do_uffle)

Typedefs

- typedef int(* [WRITEBUFTOEXTCHANNEL](#)) (char *, size_t)
- typedef int(* [GETCFROMEXTCHANNEL](#)) (void)
- typedef int(* [SETTERMINATORFOREXTCHANNEL](#)) (char *)
- typedef int(* [SETKILLMODEFOREXTCHANNEL](#)) (int, int)
- typedef LONG(* [WRITEFILE](#)) (int, UBYTE *, LONG)
- typedef WORD(* [GETTERM](#)) (PHEAD WORD *)

Functions

- void [TELLFILE](#) (int, [POSITION](#) *)
- void [StartVariables](#) (void)
- void [setSignalHandlers](#) (void)
- UBYTE * [CodeToLine](#) (WORD, UBYTE *)
- UBYTE * [AddArrayIndex](#) (WORD, UBYTE *)
- [INDEXENTRY](#) * [FindInIndex](#) (WORD, [FILEDATA](#) *, WORD, WORD)
- [INDEXENTRY](#) * [NextFileIndex](#) ([POSITION](#) *)
- WORD * [PasteTerm](#) (PHEAD WORD, WORD *, WORD *, WORD, WORD)
- UBYTE * [StrCopy](#) (UBYTE *, UBYTE *)
- UBYTE * [WrtPower](#) (UBYTE *, WORD)
- WORD [AccumGCD](#) (PHEAD UWORD *, WORD *, UWORD *, WORD)
- void [AddArgs](#) (PHEAD WORD *, WORD *, WORD *)
- WORD [AddCoef](#) (PHEAD WORD **, WORD **)
- WORD [AddLong](#) (UWORD *, WORD, UWORD *, WORD, UWORD *, WORD *)
- WORD [AddPLon](#) (UWORD *, WORD, UWORD *, WORD, UWORD *, WORD *)
- WORD [AddPoly](#) (PHEAD WORD **, WORD **)
- WORD [AddRat](#) (PHEAD UWORD *, WORD, UWORD *, WORD, UWORD *, WORD *)
- void [AddToLine](#) (UBYTE *)
- WORD [AddWild](#) (PHEAD WORD, WORD, WORD)
- WORD [BigLong](#) (UWORD *, WORD, UWORD *, WORD)
- WORD [BinomGen](#) (PHEAD WORD *, WORD, WORD **, WORD, WORD, WORD, WORD, WORD, UWORD *, WORD)
- WORD [CheckWild](#) (PHEAD WORD, WORD, WORD, WORD *)
- WORD [Chisholm](#) (PHEAD WORD *, WORD)
- WORD [CleanExpr](#) (WORD)
- void [CleanUp](#) (WORD)
- void [ClearWild](#) (PHEAD0)
- WORD [CompareFunctions](#) (WORD *, WORD *)
- WORD [Commute](#) (WORD *, WORD *)
- WORD [DetCommu](#) (WORD *)
- WORD [DoesCommu](#) (WORD *)
- int [CompArg](#) (WORD *, WORD *)
- WORD [CompCoef](#) (WORD *, WORD *)
- WORD [CompGroup](#) (PHEAD WORD, WORD **, WORD *, WORD *, WORD)
- WORD [Compare1](#) (PHEAD WORD *, WORD *, WORD)
- WORD [CountDo](#) (WORD *, WORD *)
- WORD [CountFun](#) (WORD *, WORD *)
- WORD [DimensionSubterm](#) (WORD *)
- WORD [DimensionTerm](#) (WORD *)
- WORD [DimensionExpression](#) (PHEAD WORD *)
- WORD [Deferred](#) (PHEAD WORD *, WORD)
- WORD [DeleteStore](#) (WORD)
- WORD [DetCurDum](#) (PHEAD WORD *)

- void [DetVars](#) (WORD *, WORD)
- WORD [Distribute](#) ([DISTRIBUTE](#) *, WORD)
- WORD [DivLong](#) (UWORD *, WORD, UWORD *, WORD, UWORD *, WORD *, UWORD *, WORD *)
- WORD [DivRat](#) (PHEAD UWORD *, WORD, UWORD *, WORD, UWORD *, WORD *)
- WORD [Divvy](#) (PHEAD UWORD *, WORD *, UWORD *, WORD)
- WORD [DoDelta](#) (WORD *)
- WORD [DoDelta3](#) (PHEAD WORD *, WORD)
- WORD [TestPartitions](#) (WORD *, [PARTI](#) *)
- WORD [DoPartitions](#) (PHEAD WORD *, WORD)
- int [CoCanonicalize](#) (UBYTE *)
- int [DoCanonicalize](#) (PHEAD WORD *, WORD *)
- WORD [GenTopologies](#) (PHEAD WORD *, WORD)
- WORD [GenDiagrams](#) (PHEAD WORD *, WORD)
- int [DoTopologyCanonicalize](#) (PHEAD WORD *, WORD, WORD, WORD *)
- int [DoShattering](#) (PHEAD WORD *, WORD *, WORD *, WORD)
- WORD [GenerateTopologies](#) (PHEAD WORD, WORD, WORD, WORD)
- WORD [DoTableExpansion](#) (WORD *, WORD)
- WORD [DoDistrib](#) (PHEAD WORD *, WORD)
- WORD [DoShuffle](#) (WORD *, WORD, WORD, WORD)
- WORD [DoPermutations](#) (PHEAD WORD *, WORD)
- int [Shuffle](#) (WORD *, WORD *, WORD *)
- int [FinishShuffle](#) (WORD *)
- WORD [DoStuff](#) (WORD *, WORD, WORD, WORD)
- int [Stuff](#) (WORD *, WORD *, WORD *)
- int [FinishStuff](#) (WORD *)
- WORD * [StuffRootAdd](#) (WORD *, WORD *, WORD *)
- WORD [TestUse](#) (WORD *, WORD)
- [DBASE](#) * [FindTB](#) (UBYTE *)
- int [CheckTableDeclarations](#) ([DBASE](#) *)
- WORD [Apply](#) (WORD *, WORD)
- int [ApplyExec](#) (WORD *, int, WORD)
- WORD [ApplyReset](#) (WORD)
- WORD [TableReset](#) (void)
- void [ReWorkT](#) (WORD *, WORD *, WORD)
- WORD [GetIfDollarNum](#) (WORD *, WORD *)
- int [FindVar](#) (WORD *, WORD *)
- WORD [DolfStatement](#) (PHEAD WORD *, WORD *)
- WORD [DoOnePow](#) (PHEAD WORD *, WORD, WORD, WORD *, WORD *, WORD, WORD *)
- void [DoRevert](#) (WORD *, WORD *)
- WORD [DoSumF1](#) (PHEAD WORD *, WORD *, WORD, WORD)
- WORD [DoSumF2](#) (PHEAD WORD *, WORD *, WORD, WORD)
- WORD [DoTheta](#) (PHEAD WORD *)
- LONG [EndSort](#) (PHEAD WORD *, int)
- WORD [EntVar](#) (WORD, UBYTE *, WORD, WORD, WORD, WORD)
- WORD [EpfCon](#) (PHEAD WORD *, WORD *, WORD, WORD)
- WORD [EpfFind](#) (PHEAD WORD *, WORD *)
- WORD [EpfGen](#) (WORD, WORD *, WORD *, WORD *, WORD)
- WORD [EqualArg](#) (WORD *, WORD, WORD)
- WORD [Evaluate](#) (UBYTE **)
- int [Factorial](#) (PHEAD WORD, UWORD *, WORD *)
- int [Bernoulli](#) (WORD, UWORD *, WORD *)
- int [FactorIn](#) (PHEAD WORD *, WORD)
- int [FactorInExpr](#) (PHEAD WORD *, WORD)
- WORD [FindAll](#) (PHEAD WORD *, WORD *, WORD, WORD *)
- WORD [FindMulti](#) (PHEAD WORD *, WORD *)

- WORD [FindOnce](#) (PHEAD WORD *, WORD *)
- WORD [FindOnly](#) (PHEAD WORD *, WORD *)
- WORD [FindRest](#) (PHEAD WORD *, WORD *)
- WORD **FindSpecial** (WORD *)
- WORD [FindrNumber](#) (WORD, [VARRENUM](#) *)
- void [FiniLine](#) (void)
- WORD [FiniTerm](#) (PHEAD WORD *, WORD *, WORD *, WORD, WORD)
- WORD [FlushOut](#) ([POSITION](#) *, [FILEHANDLE](#) *, int)
- void [FunLevel](#) (PHEAD WORD *)
- void [AdjustRenumScratch](#) (PHEAD0)
- void [GarbHand](#) (void)
- WORD [GcdLong](#) (PHEAD UWORD *, WORD, UWORD *, WORD, UWORD *, WORD *)
- WORD [LcmLong](#) (PHEAD UWORD *, WORD, UWORD *, WORD, UWORD *, WORD *)
- void **GCD** (UWORD *, WORD, UWORD *, WORD, UWORD *, WORD *)
- ULONG **GCD2** (ULONG, ULONG)
- WORD [Generator](#) (PHEAD WORD *, WORD)
- WORD [GetBinom](#) (UWORD *, WORD *, WORD, WORD)
- WORD [GetFromStore](#) (WORD *, [POSITION](#) *, [RENUMBER](#), WORD *, WORD)
- WORD [GetLong](#) (UBYTE *, UWORD *, WORD *)
- WORD [GetMoreTerms](#) (WORD *)
- WORD [GetMoreFromMem](#) (WORD *, WORD **)
- WORD [GetOneTerm](#) (PHEAD WORD *, [FILEHANDLE](#) *, [POSITION](#) *, int)
- [RENUMBER](#) [GetTable](#) (WORD, [POSITION](#) *, WORD)
- WORD [GetTerm](#) (PHEAD WORD *)
- WORD [Glue](#) (PHEAD WORD *, WORD *, WORD *, WORD)
- WORD [InFunction](#) (PHEAD WORD *, WORD *)
- void [IniLine](#) (WORD)
- WORD [IniVars](#) (void)
- void **Initialize** (void)
- WORD [InsertTerm](#) (PHEAD WORD *, WORD, WORD, WORD *, WORD *, WORD)
- void [LongToLine](#) (UWORD *, WORD)
- WORD [MakeDirty](#) (WORD *, WORD *, WORD)
- void [MarkDirty](#) (WORD *, WORD)
- void [PolyFunDirty](#) (PHEAD WORD *)
- void [PolyFunClean](#) (PHEAD WORD *)
- WORD [MakeModTable](#) (void)
- WORD [MatchE](#) (PHEAD WORD *, WORD *, WORD *, WORD)
- int [MatchCy](#) (PHEAD WORD *, WORD *, WORD *, WORD)
- int [FunMatchCy](#) (PHEAD WORD *, WORD *, WORD *, WORD)
- int [FunMatchSy](#) (PHEAD WORD *, WORD *, WORD *, WORD)
- int [MatchArgument](#) (PHEAD WORD *, WORD *)
- WORD [MatchFunction](#) (PHEAD WORD *, WORD *, WORD *)
- WORD [MergePatches](#) (WORD)
- WORD [MesCerr](#) (char *, UBYTE *)
- WORD [MesComp](#) (char *, UBYTE *, UBYTE *)
- WORD [Modulus](#) (WORD *)
- void [MoveDummies](#) (PHEAD WORD *, WORD)
- WORD [MulLong](#) (UWORD *, WORD, UWORD *, WORD, UWORD *, WORD *)
- WORD [MulRat](#) (PHEAD UWORD *, WORD, UWORD *, WORD, UWORD *, WORD *)
- WORD [Mully](#) (PHEAD UWORD *, WORD *, UWORD *, WORD)
- WORD [MultDo](#) (PHEAD WORD *, WORD *)
- WORD [NewSort](#) (PHEAD0)
- WORD [ExtraSymbol](#) (WORD, WORD, WORD, WORD *, WORD *)
- WORD [Normalize](#) (PHEAD WORD *)
- WORD [BracketNormalize](#) (PHEAD WORD *)

- void [DropCoefficient](#) (PHEAD WORD *)
- void [DropSymbols](#) (PHEAD WORD *)
- int [PutInside](#) (PHEAD WORD *, WORD *)
- WORD [OpenTemp](#) (void)
- void [Pack](#) (UWORD *, WORD *, UWORD *, WORD)
- LONG [PasteFile](#) (PHEAD WORD, WORD *, [POSITION](#) *, WORD **, [RENUMBER](#), WORD *, WORD)
- WORD [Permute](#) ([PERM](#) *, WORD)
- WORD [PermuteP](#) ([PERMP](#) *, WORD)
- WORD [PolyFunMul](#) (PHEAD WORD *)
- WORD [PopVariables](#) (void)
- WORD [PrepPoly](#) (PHEAD WORD *, WORD)
- WORD [Processor](#) (void)
- WORD [Product](#) (UWORD *, WORD *, WORD)
- void [PrtLong](#) (UWORD *, WORD, UBYTE *)
- void [PrtTerms](#) (void)
- void [PrintDeprecation](#) (const char *, const char *)
- void [PrintRunningTime](#) (void)
- LONG [GetRunningTime](#) (void)
- WORD [PutBracket](#) (PHEAD WORD *)
- LONG [PutIn](#) ([FILEHANDLE](#) *, [POSITION](#) *, WORD *, WORD **, int)
- WORD [PutInStore](#) ([INDEXENTRY](#) *, WORD)
- WORD [PutOut](#) (PHEAD WORD *, [POSITION](#) *, [FILEHANDLE](#) *, WORD)
- UWORD [Quotient](#) (UWORD *, WORD *, WORD)
- WORD [RaisPow](#) (PHEAD UWORD *, WORD *, UWORD)
- void [RaisPowCached](#) (PHEAD WORD, WORD, UWORD **, WORD *)
- WORD [RaisPowMod](#) (WORD, WORD, WORD)
- int [NormalModulus](#) (UWORD *, WORD *)
- int [MakeInverses](#) (void)
- int [GetModInverses](#) (WORD, WORD, WORD *, WORD *)
- int [GetLongModInverses](#) (PHEAD UWORD *, WORD, UWORD *, WORD, UWORD *, WORD *, UWORD *, WORD *)
- void [RatToLine](#) (UWORD *, WORD)
- WORD [RatioFind](#) (PHEAD WORD *, WORD *)
- WORD [RatioGen](#) (PHEAD WORD *, WORD *, WORD, WORD)
- WORD [ReNumber](#) (PHEAD WORD *)
- WORD [ReadSnum](#) (UBYTE **)
- WORD [Remain10](#) (UWORD *, WORD *)
- WORD [Remain4](#) (UWORD *, WORD *)
- WORD [ResetScratch](#) (void)
- WORD [ResolveSet](#) (PHEAD WORD *, WORD *, WORD *)
- WORD [RevertScratch](#) (void)
- WORD [ScanFunctions](#) (PHEAD WORD *, WORD *, WORD)
- void [SeekScratch](#) ([FILEHANDLE](#) *, [POSITION](#) *)
- void [SetEndScratch](#) ([FILEHANDLE](#) *, [POSITION](#) *)
- void [SetEndHScratch](#) ([FILEHANDLE](#) *, [POSITION](#) *)
- WORD [SetFileIndex](#) (void)
- WORD [Sflush](#) ([FILEHANDLE](#) *)
- WORD [Simplify](#) (PHEAD UWORD *, WORD *, UWORD *, WORD *)
- WORD [SortWild](#) (WORD *, WORD)
- FILE * [LocateBase](#) (char **, char **, char *)
- LONG [SplitMerge](#) (PHEAD WORD **, LONG)
- WORD [StoreTerm](#) (PHEAD WORD *)
- void [SubPLon](#) (UWORD *, WORD, UWORD *, WORD, UWORD *, WORD *)
- void [Substitute](#) (PHEAD WORD *, WORD *, WORD)
- WORD [SymFind](#) (PHEAD WORD *, WORD *)

- WORD [SymGen](#) (PHEAD WORD *, WORD *, WORD, WORD)
- WORD [Symmetrize](#) (PHEAD WORD *, WORD *, WORD, WORD, WORD)
- int [FullSymmetrize](#) (PHEAD WORD *, int)
- WORD [TakeModulus](#) (UWORD *, WORD *, UWORD *, WORD, WORD)
- WORD [TakeNormalModulus](#) (UWORD *, WORD *, UWORD *, WORD, WORD)
- void [TalToLine](#) (UWORD)
- WORD [TenVec](#) (PHEAD WORD *, WORD *, WORD, WORD)
- WORD [TenVecFind](#) (PHEAD WORD *, WORD *)
- WORD [TermRenum](#) (WORD *, [RENUMBER](#), WORD)
- void [TestDrop](#) (void)
- void [PutInVflags](#) (WORD)
- WORD [TestMatch](#) (PHEAD WORD *, WORD *)
- WORD [TestSub](#) (PHEAD WORD *, WORD)
- LONG [TimeCPU](#) (WORD)
- LONG [TimeChildren](#) (WORD)
- LONG [TimeWallClock](#) (WORD)
- LONG [Timer](#) (int)
- int [GetTimerInfo](#) (LONG **, LONG **)
- void [WriteTimerInfo](#) (LONG *, LONG *)
- LONG [GetWorkerTimes](#) (void)
- WORD [ToStorage](#) ([EXPRESSIONS](#), [POSITION](#) *)
- void [TokenToLine](#) (UBYTE *)
- WORD [Trace4](#) (PHEAD WORD *, WORD *, WORD, WORD)
- WORD [Trace4Gen](#) (PHEAD [TRACES](#) *, WORD)
- WORD [Trace4no](#) (WORD, WORD *, [TRACES](#) *)
- WORD [TraceFind](#) (PHEAD WORD *, WORD *)
- WORD [TraceN](#) (PHEAD WORD *, WORD *, WORD, WORD)
- WORD [TraceNgen](#) (PHEAD [TRACES](#) *, WORD)
- WORD [TraceNno](#) (WORD, WORD *, [TRACES](#) *)
- WORD [Traces](#) (PHEAD WORD *, WORD *, WORD, WORD)
- WORD [Trick](#) (WORD *, [TRACES](#) *)
- WORD [TryDo](#) (PHEAD WORD *, WORD *, WORD)
- void [UnPack](#) (UWORD *, WORD, WORD *, WORD *)
- WORD [VarStore](#) (UBYTE *, WORD, WORD, WORD)
- WORD [WildFill](#) (PHEAD WORD *, WORD *, WORD *)
- WORD [WriteAll](#) (void)
- WORD [WriteOne](#) (UBYTE *, int, int, WORD)
- void [WriteArgument](#) (WORD *)
- WORD [WriteExpression](#) (WORD *, LONG)
- WORD [WriteInnerTerm](#) (WORD *, WORD)
- void [WriteLists](#) (void)
- void [WriteSetup](#) (void)
- void [WriteStats](#) ([POSITION](#) *, WORD, WORD)
- WORD [WriteSubTerm](#) (WORD *, WORD)
- WORD [WriteTerm](#) (WORD *, WORD *, WORD, WORD, WORD)
- WORD [execarg](#) (PHEAD WORD *, WORD)
- WORD [execterm](#) (PHEAD WORD *, WORD)
- void [SpecialCleanup](#) (PHEAD0)
- void [SetMods](#) (void)
- void [UnSetMods](#) (void)
- WORD [DoExecute](#) (WORD, WORD)
- void [SetScratch](#) ([FILEHANDLE](#) *, [POSITION](#) *)
- void [Warning](#) (char *)
- void [HighWarning](#) (char *)
- int [SpareTable](#) ([TABLES](#))

- `UBYTE * strDup1 (UBYTE *, char *)`
- `void * Malloc (LONG)`
- `void * Malloc1 (LONG, const char *)`
- `int DoTail (int, UBYTE **)`
- `int OpenInput (void)`
- `int PutPreVar (UBYTE *, UBYTE *, UBYTE *, int)`
- `void Error0 (char *)`
- `void Error1 (char *, UBYTE *)`
- `void Error2 (char *, char *, UBYTE *)`
- `UBYTE ReadFromStream (STREAM *)`
- `UBYTE GetFromStream (STREAM *)`
- `UBYTE LookInStream (STREAM *)`
- `STREAM * OpenStream (UBYTE *, int, int, int)`
- `int LocateFile (UBYTE **, int)`
- `STREAM * CloseStream (STREAM *)`
- `void PositionStream (STREAM *, LONG)`
- `int ReverseStatements (STREAM *)`
- `int ProcessOption (UBYTE *, UBYTE *, int)`
- `int DoSetups (void)`
- `void TerminateImpl (int, const char *, int, const char *)`
- `NAMENODE * GetNode (NAMETREE *, UBYTE *)`
- `int AddName (NAMETREE *, UBYTE *, WORD, WORD, int *)`
- `int GetName (NAMETREE *, UBYTE *, WORD *, int)`
- `UBYTE * GetFunction (UBYTE *, WORD *)`
- `UBYTE * GetNumber (UBYTE *, WORD *)`
- `int GetLastExprName (UBYTE *, WORD *)`
- `int GetAutoName (UBYTE *, WORD *)`
- `int GetVar (UBYTE *, WORD *, WORD *, int, int)`
- `int MakeDubious (NAMETREE *, UBYTE *, WORD *)`
- `int GetOName (NAMETREE *, UBYTE *, WORD *, int)`
- `void DumpTree (NAMETREE *)`
- `void DumpNode (NAMETREE *, WORD, WORD)`
- `void LinkTree (NAMETREE *, WORD, WORD)`
- `void CopyTree (NAMETREE *, NAMETREE *, WORD, WORD)`
- `int CompactifyTree (NAMETREE *, WORD)`
- `NAMETREE * MakeNameTree (void)`
- `void FreeNameTree (NAMETREE *)`
- `int AddExpression (UBYTE *, int, int)`
- `int AddSymbol (UBYTE *, int, int, int, int)`
- `int AddDollar (UBYTE *, WORD, WORD *, LONG)`
- `int ReplaceDollar (WORD, WORD, WORD *, LONG)`
- `int DollarRaiseLow (UBYTE *, LONG)`
- `int AddVector (UBYTE *, int, int)`
- `int AddDubious (UBYTE *)`
- `int AddIndex (UBYTE *, int, int)`
- `UBYTE * DoDimension (UBYTE *, int *, int *)`
- `int AddFunction (UBYTE *, int, int, int, int, int, int, int)`
- `int CoCommuteInSet (UBYTE *)`
- `int CoFunction (UBYTE *, int, int)`
- `int TestName (UBYTE *)`
- `int AddSet (UBYTE *, WORD)`
- `int DoElements (UBYTE *, SETS, UBYTE *)`
- `int DoTempSet (UBYTE *, UBYTE *)`
- `int NameConflict (int, UBYTE *)`
- `int OpenFile (char *)`

- int [OpenAddFile](#) (char *)
- int [ReOpenFile](#) (char *)
- int [CreateFile](#) (char *)
- int [CreateLogFile](#) (char *)
- void [CloseFile](#) (int)
- int [CopyFile](#) (char *, char *)
- int [CreateHandle](#) (void)
- LONG [ReadFile](#) (int, UBYTE *, LONG)
- LONG [ReadPosFile](#) (PHEAD [FILEHANDLE](#) *, UBYTE *, LONG, [POSITION](#) *)
- LONG [WriteFileToFile](#) (int, UBYTE *, LONG)
- void [SeekFile](#) (int, [POSITION](#) *, int)
- LONG [TellFile](#) (int)
- void [FlushFile](#) (int)
- int [GetPosFile](#) (int, fpos_t *)
- int [SetPosFile](#) (int, fpos_t *)
- void [SynchFile](#) (int)
- void [TruncateFile](#) (int)
- int [GetChannel](#) (char *, int)
- int [GetAppendChannel](#) (char *)
- int [CloseChannel](#) (char *)
- void [inictable](#) (void)
- [KEYWORD](#) * [findcommand](#) (UBYTE *)
- int [inicbufs](#) (void)
- void [StartFiles](#) (void)
- UBYTE * [MakeDate](#) (void)
- void [PreProcessor](#) (void)
- void * [FromList](#) ([LIST](#) *)
- void * [From0List](#) ([LIST](#) *)
- void * [FromVarList](#) ([LIST](#) *)
- int [DoubleList](#) (void ***, int *, int, char *)
- int [DoubleLList](#) (void ***, LONG *, int, char *)
- void [DoubleBuffer](#) (void **, void **, int, char *)
- void [ExpandBuffer](#) (void **, LONG *, int)
- LONG [iexp](#) (LONG, int)
- int [IsLikeVector](#) (WORD *)
- int [AreArgsEqual](#) (WORD *, WORD *)
- int [CompareArgs](#) (WORD *, WORD *)
- UBYTE * [SkipField](#) (UBYTE *, int)
- int [StrCmp](#) (UBYTE *, UBYTE *)
- int [StrLCmp](#) (UBYTE *, UBYTE *)
- int [StrHICmp](#) (UBYTE *, UBYTE *)
- int [StrLCont](#) (UBYTE *, UBYTE *)
- int [CmpArray](#) (WORD *, WORD *, WORD)
- int [ConWord](#) (UBYTE *, UBYTE *)
- int [StrLen](#) (UBYTE *)
- UBYTE * [GetPreVar](#) (UBYTE *, int)
- void [ToGeneral](#) (WORD *, WORD *, WORD)
- WORD [ToPolyFunGeneral](#) (PHEAD WORD *)
- int [ToFast](#) (WORD *, WORD *)
- [SETUPPARAMETERS](#) * [GetSetupPar](#) (UBYTE *)
- int [RecalcSetups](#) (void)
- int [AllocSetups](#) (void)
- [SORTING](#) * [AllocSort](#) (LONG, LONG, LONG, LONG, int, int, LONG, int)
- void [AllocSortFileName](#) ([SORTING](#) *)
- UBYTE * [LoadInputFile](#) (UBYTE *, int)

- UBYTE [GetInput](#) (void)
- void [ClearPushback](#) (void)
- UBYTE [GetChar](#) (int)
- void [CharOut](#) (UBYTE)
- void [UnsetAllowDelay](#) (void)
- void [PopPreVars](#) (int)
- void [IniModule](#) (int)
- void [IniSpecialModule](#) (int)
- int [ModuleInstruction](#) (int *, int *)
- int [PreProInstruction](#) (void)
- int [LoadInstruction](#) (int)
- int [LoadStatement](#) (int)
- KEYWORD * [FindKeyWord](#) (UBYTE *, KEYWORD *, int)
- KEYWORD * [FindInKeyWord](#) (UBYTE *, KEYWORD *, int)
- int [DoDefine](#) (UBYTE *)
- int [DoRedefine](#) (UBYTE *)
- int [TheDefine](#) (UBYTE *, int)
- int [TheUndefine](#) (UBYTE *)
- int [ClearMacro](#) (UBYTE *)
- int [DoUndefine](#) (UBYTE *)
- int [DoInclude](#) (UBYTE *)
- int [DoReverseInclude](#) (UBYTE *)
- int [Include](#) (UBYTE *, int)
- int [DoExternal](#) (UBYTE *)
- int [DoToExternal](#) (UBYTE *)
- int [DoFromExternal](#) (UBYTE *)
- int [DoPrompt](#) (UBYTE *)
- int [DoSetExternal](#) (UBYTE *)
- int [DoSetExternalAttr](#) (UBYTE *)
- int [DoRmExternal](#) (UBYTE *)
- int [DoFactDollar](#) (UBYTE *)
- WORD [GetDollarNumber](#) (UBYTE **, DOLLARS)
- int [DoSetRandom](#) (UBYTE *)
- int [DoOptimize](#) (UBYTE *)
- int [DoClearOptimize](#) (UBYTE *)
- int [DoSkipExtraSymbols](#) (UBYTE *)
- int [DoTimeOutAfter](#) (UBYTE *)
- int [DoMessage](#) (UBYTE *)
- int [DoPreOut](#) (UBYTE *)
- int [DoPreAppend](#) (UBYTE *)
- int [DoPreCreate](#) (UBYTE *)
- int [DoPreAssign](#) (UBYTE *)
- int [DoPreBreak](#) (UBYTE *)
- int [DoPreDefault](#) (UBYTE *)
- int [DoPreSwitch](#) (UBYTE *)
- int [DoPreEndSwitch](#) (UBYTE *)
- int [DoPreCase](#) (UBYTE *)
- int [DoPreShow](#) (UBYTE *)
- int [DoPreExchange](#) (UBYTE *)
- int [DoSystem](#) (UBYTE *)
- int [DoPipe](#) (UBYTE *)
- void [StartPrepro](#) (void)
- int [Dolfddef](#) (UBYTE *, int)
- int [Dolfydef](#) (UBYTE *)
- int [Dolfndef](#) (UBYTE *)

- int [DoElse](#) (UBYTE *)
- int [DoElseif](#) (UBYTE *)
- int [DoEndif](#) (UBYTE *)
- int [DoTerminate](#) (UBYTE *)
- int [Dolf](#) (UBYTE *)
- int [DoCall](#) (UBYTE *)
- int [DoDebug](#) (UBYTE *)
- int [DoContinueDo](#) (UBYTE *)
- int [DoDo](#) (UBYTE *)
- int [DoBreakDo](#) (UBYTE *)
- int [DoEnddo](#) (UBYTE *)
- int [DoEndprocedure](#) (UBYTE *)
- int [DoInside](#) (UBYTE *)
- int [DoEndInside](#) (UBYTE *)
- int [DoProcedure](#) (UBYTE *)
- int [DoPrePrintTimes](#) (UBYTE *)
- int [DoPreWrite](#) (UBYTE *)
- int [DoPreClose](#) (UBYTE *)
- int [DoPreRemove](#) (UBYTE *)
- int [DoPreSortReallocate](#) (UBYTE *)
- int [DoCommentChar](#) (UBYTE *)
- int [DoProcExtension](#) (UBYTE *)
- int [DoPreReset](#) (UBYTE *)
- void [WriteString](#) (int, UBYTE *, int)
- void [WriteUnfinString](#) (int, UBYTE *, int)
- UBYTE * [AddToString](#) (UBYTE *, UBYTE *, int)
- UBYTE * [PreCalc](#) (void)
- UBYTE * [PreEval](#) (UBYTE *, LONG *)
- void [NumToStr](#) (UBYTE *, LONG)
- int [PreCmp](#) (int, int, UBYTE *, int, int, UBYTE *, int)
- int [PreEq](#) (int, int, UBYTE *, int, int, UBYTE *, int)
- UBYTE * [pParseObject](#) (UBYTE *, int *, LONG *)
- UBYTE * [PreIfEval](#) (UBYTE *, int *)
- int [EvalPreIf](#) (UBYTE *)
- int [PreLoad](#) ([PRELOAD](#) *, UBYTE *, UBYTE *, int, char *)
- int [PreSkip](#) (UBYTE *, UBYTE *, int)
- UBYTE * [EndOfToken](#) (UBYTE *)
- void [SetSpecialMode](#) (int, int)
- void [MakeGlobal](#) (void)
- int [ExecModule](#) (int)
- int [ExecStore](#) (void)
- void [FullCleanUp](#) (void)
- int [DoExecStatement](#) (void)
- int [DoPipeStatement](#) (void)
- int [DoPolyfun](#) (UBYTE *)
- int [DoPolyratfun](#) (UBYTE *)
- int [CompileStatement](#) (UBYTE *)
- UBYTE * [ToToken](#) (UBYTE *)
- int [GetDollar](#) (UBYTE *)
- int [MesWork](#) (void)
- int **MesPrint** (const char *,...)
- int [MesCall](#) (char *)
- UBYTE * [NumCopy](#) (WORD, UBYTE *)
- char * [LongCopy](#) (LONG, char *)
- char * [LongLongCopy](#) (off_t *, char *)

- void [ReserveTempFiles](#) (int)
- void [PrintTerm](#) (WORD *, char *)
- void [PrintTermC](#) (WORD *, char *)
- void [PrintSubTerm](#) (WORD *, char *)
- void [PrintWords](#) (WORD *, LONG)
- void [PrintSeq](#) (WORD *, char *)
- int [ExpandTripleDots](#) (int)
- LONG [ComPress](#) (WORD **, LONG *)
- void [StageSort](#) (FILEHANDLE *)
- void [M_free](#) (void *, const char *)
- void [ClearWildcardNames](#) (void)
- int [AddWildcardName](#) (UBYTE *)
- int [GetWildcardName](#) (UBYTE *)
- void [Globalize](#) (int)
- void [ResetVariables](#) (int)
- void [AddToPreTypes](#) (int)
- void [MessPreNesting](#) (int)
- LONG [GetStreamPosition](#) (STREAM *)
- WORD * [DoubleCbuffer](#) (int, WORD *, int)
- WORD * [AddLHS](#) (int)
- WORD * [AddRHS](#) (int, int)
- int [AddNtoL](#) (int, WORD *)
- int [AddNtoC](#) (int, int, WORD *, int)
- void [DoubleIfBuffers](#) (void)
- STREAM * [CreateStream](#) (UBYTE *)
- int [setonoff](#) (UBYTE *, int *, int, int)
- int [DoPrint](#) (UBYTE *, int)
- int [SetExpr](#) (UBYTE *, int, int)
- void [AddToCom](#) (int, WORD *)
- int [Add2ComStrings](#) (int, WORD *, UBYTE *, UBYTE *)
- int [DoSymmetrize](#) (UBYTE *, int)
- int [DoArgument](#) (UBYTE *, int)
- int [ArgFactorize](#) (PHEAD WORD *, WORD *)
- WORD * [TakeArgContent](#) (PHEAD WORD *, WORD *)
- WORD * [MakeInteger](#) (PHEAD WORD *, WORD *, WORD *)
- WORD * [MakeMod](#) (PHEAD WORD *, WORD *, WORD *)
- WORD [FindArg](#) (PHEAD WORD *)
- WORD [InsertArg](#) (PHEAD WORD *, WORD *, int)
- int [CleanupArgCache](#) (PHEAD WORD)
- int [ArgSymbolMerge](#) (WORD *, WORD *)
- int [ArgDotproductMerge](#) (WORD *, WORD *)
- void [SortWeights](#) (LONG *, LONG *, WORD)
- int [DoBrackets](#) (UBYTE *, int)
- int [DoPutInside](#) (UBYTE *, int)
- WORD * [CountComp](#) (UBYTE *, WORD *)
- int [CoAntiBracket](#) (UBYTE *)
- int [CoAntiSymmetrize](#) (UBYTE *)
- int [DoArgPlode](#) (UBYTE *, int)
- int [CoArgExplode](#) (UBYTE *)
- int [CoArgImplode](#) (UBYTE *)
- int [CoArgument](#) (UBYTE *)
- int [ColInside](#) (UBYTE *)
- int [ExecInside](#) (UBYTE *)
- int [ColnExpression](#) (UBYTE *)
- int [ColnParallel](#) (UBYTE *)

- int [CoNotInParallel](#) (UBYTE *)
- int [DoInParallel](#) (UBYTE *, int)
- int [CoEndInExpression](#) (UBYTE *)
- int [CoBracket](#) (UBYTE *)
- int [CoPutInside](#) (UBYTE *)
- int [CoAntiPutInside](#) (UBYTE *)
- int [CoMultiBracket](#) (UBYTE *)
- int [CoCFunction](#) (UBYTE *)
- int [CoCTensor](#) (UBYTE *)
- int [CoCollect](#) (UBYTE *)
- int [CoCompress](#) (UBYTE *)
- int [CoContract](#) (UBYTE *)
- int [CoCycleSymmetrize](#) (UBYTE *)
- int [CoDelete](#) (UBYTE *)
- int [CoTableBase](#) (UBYTE *)
- int [CoApply](#) (UBYTE *)
- int [CoDenominators](#) (UBYTE *)
- int [CoDimension](#) (UBYTE *)
- int [CoDiscard](#) (UBYTE *)
- int [CoDisorder](#) (UBYTE *)
- int [CoDrop](#) (UBYTE *)
- int [CoDropCoefficient](#) (UBYTE *)
- int [CoDropSymbols](#) (UBYTE *)
- int [CoElse](#) (UBYTE *)
- int [CoElseif](#) (UBYTE *)
- int [CoEndArgument](#) (UBYTE *)
- int [CoEndInside](#) (UBYTE *)
- int [CoEndIf](#) (UBYTE *)
- int [CoEndRepeat](#) (UBYTE *)
- int [CoEndTerm](#) (UBYTE *)
- int [CoEndWhile](#) (UBYTE *)
- int [CoExit](#) (UBYTE *)
- int [CoFactArg](#) (UBYTE *)
- int [CoFactDollar](#) (UBYTE *)
- int [CoFactorize](#) (UBYTE *)
- int [CoNFactorize](#) (UBYTE *)
- int [CoUnFactorize](#) (UBYTE *)
- int [CoNUnFactorize](#) (UBYTE *)
- int [DoFactorize](#) (UBYTE *, int)
- int [CoFill](#) (UBYTE *)
- int [CoFillExpression](#) (UBYTE *)
- int [CoFixIndex](#) (UBYTE *)
- int [CoFormat](#) (UBYTE *)
- int [CoGlobal](#) (UBYTE *)
- int [CoGlobalFactorized](#) (UBYTE *)
- int [CoGoTo](#) (UBYTE *)
- int [Cold](#) (UBYTE *)
- int [ColdNew](#) (UBYTE *)
- int [ColdOld](#) (UBYTE *)
- int [Colf](#) (UBYTE *)
- int [ColfMatch](#) (UBYTE *)
- int [ColfNoMatch](#) (UBYTE *)
- int [ColIndex](#) (UBYTE *)
- int [ColInsideFirst](#) (UBYTE *)
- int [CoKeep](#) (UBYTE *)

- int [CoLabel](#) (UBYTE *)
- int [CoLoad](#) (UBYTE *)
- int [CoLocal](#) (UBYTE *)
- int [CoLocalFactorized](#) (UBYTE *)
- int [CoMany](#) (UBYTE *)
- int [CoMerge](#) (UBYTE *)
- int [CoShuffle](#) (UBYTE *)
- int [CoMetric](#) (UBYTE *)
- int [CoModOption](#) (UBYTE *)
- int [CoModuleOption](#) (UBYTE *)
- int [CoModulus](#) (UBYTE *)
- int [CoMulti](#) (UBYTE *)
- int [CoMultiply](#) (UBYTE *)
- int [CoNFunction](#) (UBYTE *)
- int [CoNPrint](#) (UBYTE *)
- int [CoNTensor](#) (UBYTE *)
- int [CoNWrite](#) (UBYTE *)
- int [CoNoDrop](#) (UBYTE *)
- int [CoNoSkip](#) (UBYTE *)
- int [CoNormalize](#) (UBYTE *)
- int [CoMakeInteger](#) (UBYTE *)
- int [CoFlags](#) (UBYTE *, int)
- int [CoOff](#) (UBYTE *)
- int [CoOn](#) (UBYTE *)
- int [CoOnce](#) (UBYTE *)
- int [CoOnly](#) (UBYTE *)
- int [CoOptimizeOption](#) (UBYTE *)
- int [CoOptimize](#) (UBYTE *)
- int [CoPolyFun](#) (UBYTE *)
- int [CoPolyRatFun](#) (UBYTE *)
- int [CoPrint](#) (UBYTE *)
- int [CoPrintB](#) (UBYTE *)
- int [CoProperCount](#) (UBYTE *)
- int [CoUnitTrace](#) (UBYTE *)
- int [CoRCycleSymmetrize](#) (UBYTE *)
- int [CoRatio](#) (UBYTE *)
- int [CoRedefine](#) (UBYTE *)
- int [CoRenumber](#) (UBYTE *)
- int [CoRepeat](#) (UBYTE *)
- int [CoSave](#) (UBYTE *)
- int [CoSelect](#) (UBYTE *)
- int [CoSet](#) (UBYTE *)
- int [CoSetExitFlag](#) (UBYTE *)
- int [CoSkip](#) (UBYTE *)
- int [CoProcessBucket](#) (UBYTE *)
- int [CoPushHide](#) (UBYTE *)
- int [CoPopHide](#) (UBYTE *)
- int [CoHide](#) (UBYTE *)
- int [CoIntoHide](#) (UBYTE *)
- int [CoNoIntoHide](#) (UBYTE *)
- int [CoNoHide](#) (UBYTE *)
- int [CoUnHide](#) (UBYTE *)
- int [CoNoUnHide](#) (UBYTE *)
- int [CoSort](#) (UBYTE *)
- int [CoSplitArg](#) (UBYTE *)

- int [CoSplitFirstArg](#) (UBYTE *)
- int [CoSplitLastArg](#) (UBYTE *)
- int [CoSum](#) (UBYTE *)
- int [CoSymbol](#) (UBYTE *)
- int [CoSymmetrize](#) (UBYTE *)
- int [DoTable](#) (UBYTE *, int)
- int [CoTable](#) (UBYTE *)
- int [CoTerm](#) (UBYTE *)
- int [CoNTable](#) (UBYTE *)
- int [CoCTable](#) (UBYTE *)
- void [EmptyTable](#) (TABLES)
- int [CoToTensor](#) (UBYTE *)
- int [CoToVector](#) (UBYTE *)
- int [CoTrace4](#) (UBYTE *)
- int [CoTraceN](#) (UBYTE *)
- int [CoChisholm](#) (UBYTE *)
- int [CoTransform](#) (UBYTE *)
- int [CoClearTable](#) (UBYTE *)
- int [DoChain](#) (UBYTE *, int)
- int [CoChainin](#) (UBYTE *)
- int [CoChainout](#) (UBYTE *)
- int [CoTryReplace](#) (UBYTE *)
- int [CoVector](#) (UBYTE *)
- int [CoWhile](#) (UBYTE *)
- int [CoWrite](#) (UBYTE *)
- int [CoAuto](#) (UBYTE *)
- int [CoSwitch](#) (UBYTE *)
- int [CoCase](#) (UBYTE *)
- int [CoBreak](#) (UBYTE *)
- int [CoDefault](#) (UBYTE *)
- int [CoEndSwitch](#) (UBYTE *)
- int [CoTBaddto](#) (UBYTE *)
- int [CoTBaudit](#) (UBYTE *)
- int [CoTbcleanup](#) (UBYTE *)
- int [CoTbcreate](#) (UBYTE *)
- int [CoTBenter](#) (UBYTE *)
- int [CoTBhelp](#) (UBYTE *)
- int [CoTBload](#) (UBYTE *)
- int [CoTBoff](#) (UBYTE *)
- int [CoTBon](#) (UBYTE *)
- int [CoTBopen](#) (UBYTE *)
- int [CoTBreplace](#) (UBYTE *)
- int [CoTBuse](#) (UBYTE *)
- int [CoTestUse](#) (UBYTE *)
- int [CoThreadBucket](#) (UBYTE *)
- int [AddComString](#) (int, WORD *, UBYTE *, int)
- int [CompileAlgebra](#) (UBYTE *, int, WORD *)
- int [IsIdStatement](#) (UBYTE *)
- UBYTE * [IsRHS](#) (UBYTE *, UBYTE)
- int [ParenthesesTest](#) (UBYTE *)
- int [tokenize](#) (UBYTE *, WORD)
- void [WriteTokens](#) (SBYTE *)
- int [simp1token](#) (SBYTE *)
- int [simpwtoken](#) (SBYTE *)
- int [simp2token](#) (SBYTE *)

- int [simp3atoken](#) (SBYTE *, int)
- int [simp3btoken](#) (SBYTE *, int)
- int [simp4token](#) (SBYTE *)
- int [simp5token](#) (SBYTE *, int)
- int [simp6token](#) (SBYTE *, int)
- UBYTE * [SkipAName](#) (UBYTE *)
- int [TestTables](#) (void)
- int [GetLabel](#) (UBYTE *)
- int [ColdExpression](#) (UBYTE *, int)
- int [CoAssign](#) (UBYTE *)
- int [DoExpr](#) (UBYTE *, int, int)
- int [CompileSubExpressions](#) (SBYTE *)
- int [CodeGenerator](#) (SBYTE *)
- int [CompleteTerm](#) (WORD *, UWORD *, UWORD *, WORD, WORD, int)
- int [CodeFactors](#) (SBYTE *s)
- WORD [GenerateFactors](#) (WORD, WORD)
- int [InsTree](#) (int, int)
- int [FindTree](#) (int, WORD *)
- void [RedoTree](#) (CBUF *, int)
- void [ClearTree](#) (int)
- int [CatchDollar](#) (int)
- int [AssignDollar](#) (PHEAD WORD *, WORD)
- UBYTE * [WriteDollarToBuffer](#) (WORD, WORD)
- UBYTE * [WriteDollarFactorToBuffer](#) (WORD, WORD, WORD)
- void [AddToDollarBuffer](#) (UBYTE *)
- int [PutTermInDollar](#) (WORD *, WORD)
- void [TermAssign](#) (WORD *)
- void [WildDollars](#) (PHEAD WORD *)
- LONG [numcommute](#) (WORD *, LONG *)
- int [FullRenummer](#) (PHEAD WORD *, WORD)
- int [Lus](#) (WORD *, WORD, WORD, WORD, WORD, WORD)
- int [FindLus](#) (int, int, int)
- int [CoReplaceLoop](#) (UBYTE *)
- int [CoFindLoop](#) (UBYTE *)
- int [DoFindLoop](#) (UBYTE *, int)
- int [CoFunPowers](#) (UBYTE *)
- int [SortTheList](#) (int *, int)
- int [MatchIsPossible](#) (WORD *, WORD *)
- int [StudyPattern](#) (WORD *)
- WORD [DolToTensor](#) (PHEAD WORD)
- WORD [DolToFunction](#) (PHEAD WORD)
- WORD [DolToVector](#) (PHEAD WORD)
- WORD [DolToNumber](#) (PHEAD WORD)
- WORD [DolToSymbol](#) (PHEAD WORD)
- WORD [DolToIndex](#) (PHEAD WORD)
- LONG [DolToLong](#) (PHEAD WORD)
- int [DollarFactorize](#) (PHEAD WORD)
- int [CoPrintTable](#) (UBYTE *)
- int [CoDeallocateTable](#) (UBYTE *)
- void [CleanDollarFactors](#) (DOLLARS)
- WORD * [TakeDollarContent](#) (PHEAD WORD *, WORD **)
- WORD * [MakeDollarInteger](#) (PHEAD WORD *, WORD **)
- WORD * [MakeDollarMod](#) (PHEAD WORD *, WORD **)
- int [GetDolNum](#) (PHEAD WORD *, WORD *)
- void [AddPotModdollar](#) (WORD)

- int [Optimize](#) (WORD, int)
- int [ClearOptimize](#) (void)
- int **LoadOpti** (WORD)
- int **PutObject** (WORD *, int)
- void **CleanOptiBuffer** (void)
- int **PrintOptima** (WORD)
- int **FindScratchName** (void)
- WORD **MaxPowerOpti** (LONG)
- WORD **HuntNumFactor** (LONG, WORD *, int)
- WORD **HuntFactor** (LONG, WORD *, int)
- void **HuntPairs** (LONG, WORD)
- void **HuntBrackets** (LONG)
- int **AddToOpti** (WORD *, int)
- LONG **TestNewSca** (LONG, WORD *, WORD *)
- void **NormOpti** (WORD *)
- void **SortOpti** (LONG)
- void **SplitOpti** (WORD **, LONG)
- void **CombiOpti** (void)
- int [TakeLongRoot](#) (UWORD *, WORD *, WORD)
- int [TakeRatRoot](#) (UWORD *, WORD *, WORD)
- int [MakeRational](#) (WORD, WORD, WORD *, WORD *)
- int [MakeLongRational](#) (PHEAD UWORD *, WORD, UWORD *, WORD, UWORD *, WORD *)
- void **HuntPowers** (LONG, WORD)
- void **HuntNumBrackets** (LONG)
- void [ClearTableTree](#) ([TABLES](#))
- int [InsTableTree](#) ([TABLES](#), WORD *)
- void [RedoTableTree](#) ([TABLES](#), int)
- int [FindTableTree](#) ([TABLES](#), WORD *, int)
- void [finishcbuf](#) (WORD)
- void [clearcbuf](#) (WORD)
- void [CleanUpSort](#) (int)
- [FILEHANDLE](#) * [AllocFileHandle](#) (WORD, char *)
- void [DeAllocFileHandle](#) ([FILEHANDLE](#) *)
- void [LowerSortLevel](#) (void)
- WORD * [PolyRatFunSpecial](#) (PHEAD WORD *, WORD *)
- void [SimpleSplitMergeRec](#) (WORD *, WORD, WORD *)
- void [SimpleSplitMerge](#) (WORD *, WORD)
- WORD [BinarySearch](#) (WORD *, WORD, WORD)
- int [InsideDollar](#) (PHEAD WORD *, WORD)
- [DOLLARS](#) [DoToTerms](#) (PHEAD WORD)
- WORD [EvalDoLoopArg](#) (PHEAD WORD *, WORD)
- int [SetExprCases](#) (int, int, int)
- int [TestSelect](#) (WORD *, WORD *)
- void [SubsInAll](#) (PHEAD0)
- void [TransferBuffer](#) (int, int, int)
- int [TakeIDfunction](#) (PHEAD WORD *)
- int [MakeSetupAllocs](#) (void)
- int [TryFileSetups](#) (void)
- void [ExchangeExpressions](#) (int, int)
- void [ExchangeDollars](#) (int, int)
- int [GetFirstBracket](#) (WORD *, int)
- int [GetFirstTerm](#) (WORD *, int, int)
- int [GetContent](#) (WORD *, int)
- int [CleanupTerm](#) (WORD *)
- WORD [ContentMerge](#) (PHEAD WORD *, WORD *)

- UBYTE * [PrelfDollarEval](#) (UBYTE *, int *)
- LONG [TermsInDollar](#) (WORD)
- LONG [SizeOfDollar](#) (WORD)
- LONG [TermsInExpression](#) (WORD)
- LONG [SizeOfExpression](#) (WORD)
- WORD * [TranslateExpression](#) (UBYTE *)
- int [IsSetMember](#) (WORD *, WORD)
- int [IsMultipleOf](#) (WORD *, WORD *)
- int [TwoExprCompare](#) (WORD *, WORD *, int)
- void [UpdatePositions](#) (void)
- void [M_check](#) (void)
- void [M_print](#) (void)
- void [M_check1](#) (void)
- void [PrintTime](#) (UBYTE *)
- POSITION * [FindBracket](#) (WORD, WORD *)
- void [PutBracketInIndex](#) (PHEAD WORD *, POSITION *)
- void [ClearBracketIndex](#) (WORD)
- void [OpenBracketIndex](#) (WORD)
- int [DoNoParallel](#) (UBYTE *)
- int [DoParallel](#) (UBYTE *)
- int [DoModSum](#) (UBYTE *)
- int [DoModMax](#) (UBYTE *)
- int [DoModMin](#) (UBYTE *)
- int [DoModLocal](#) (UBYTE *)
- UBYTE * [DoModDollar](#) (UBYTE *, int)
- int [DoProcessBucket](#) (UBYTE *)
- int [DoinParallel](#) (UBYTE *)
- int [DonotinParallel](#) (UBYTE *)
- int [FlipTable](#) (FUNCTIONS, int)
- int [ChainIn](#) (PHEAD WORD *, WORD)
- int [ChainOut](#) (PHEAD WORD *, WORD)
- int [ArgumentImplode](#) (PHEAD WORD *, WORD *)
- int [ArgumentExplode](#) (PHEAD WORD *, WORD *)
- int [DenToFunction](#) (WORD *, WORD)
- WORD [HowMany](#) (PHEAD WORD *, WORD *)
- void [RemoveDollars](#) (void)
- LONG [CountTerms1](#) (PHEAD0)
- LONG [TermsInBracket](#) (PHEAD WORD *, WORD)
- int [Crash](#) (void)
- char * [str_dup](#) (char *)
- void [convertblock](#) (INDEXBLOCK *, INDEXBLOCK *, int)
- void [convertnamesblock](#) (NAMESBLOCK *, NAMESBLOCK *, int)
- void [convertiniinfo](#) (INIINFO *, INIINFO *, int)
- int [ReadIndex](#) (DBASE *)
- int [WriteIndexBlock](#) (DBASE *, MLONG)
- int [WriteNamesBlock](#) (DBASE *, MLONG)
- int [WriteIndex](#) (DBASE *)
- int [WriteIniInfo](#) (DBASE *)
- int [ReadIniInfo](#) (DBASE *)
- int [AddToIndex](#) (DBASE *, MLONG)
- DBASE * [GetDbase](#) (char *, MLONG)
- DBASE * [OpenDbase](#) (char *)
- char * [ReadObject](#) (DBASE *, MLONG, char *)
- char * [ReadijObject](#) (DBASE *, MLONG, MLONG, char *)
- int [ExistsObject](#) (DBASE *, MLONG, char *)

- int [DeleteObject](#) (DBASE *, MLONG, char *)
- int [WriteObject](#) (DBASE *, MLONG, char *, char *, MLONG)
- MLONG [AddObject](#) (DBASE *, MLONG, char *, char *)
- int **Cleanup** (DBASE *)
- DBASE * [NewDbase](#) (char *, MLONG)
- void [FreeTableBase](#) (DBASE *)
- int [ComposeTableNames](#) (DBASE *)
- int [PutTableNames](#) (DBASE *)
- MLONG [AddTableName](#) (DBASE *, char *, [TABLES](#))
- MLONG [GetTableName](#) (DBASE *, char *)
- MLONG [FindTableNumber](#) (DBASE *, char *)
- int [TryEnvironment](#) (void)
- int [CopyExpression](#) (FILEHANDLE *, FILEHANDLE *)
- int [set_in](#) (UBYTE, [set_of_char](#))
- one_byte [set_set](#) (UBYTE, [set_of_char](#))
- one_byte [set_del](#) (UBYTE, [set_of_char](#))
- one_byte [set_sub](#) ([set_of_char](#), [set_of_char](#), [set_of_char](#))
- int [DoPreAddSeparator](#) (UBYTE *)
- int [DoPreRmSeparator](#) (UBYTE *)
- int [openExternalChannel](#) (UBYTE *, int, UBYTE *, UBYTE *)
- int [initPresetExternalChannels](#) (UBYTE *, int)
- int [closeExternalChannel](#) (int)
- int [selectExternalChannel](#) (int)
- int [getCurrentExternalChannel](#) (void)
- void [closeAllExternalChannels](#) (void)
- UBYTE * [defineChannel](#) (UBYTE *, [HANDLERS](#) *)
- int [writeToChannel](#) (int, UBYTE *, [HANDLERS](#) *)
- int [writeBufToExtChannelOk](#) (char *, size_t)
- int [getcFromExtChannelOk](#) (void)
- int [setKillModeForExternalChannelOk](#) (int, int)
- int [setTerminatorForExternalChannelOk](#) (char *)
- int [getcFromExtChannelFailure](#) (void)
- int [setKillModeForExternalChannelFailure](#) (int, int)
- int [setTerminatorForExternalChannelFailure](#) (char *)
- int [writeBufToExtChannelFailure](#) (char *, size_t)
- int [ReleaseTB](#) (void)
- int [SymbolNormalize](#) (WORD *)
- int [TestFunFlag](#) (PHEAD WORD *)
- WORD [CompareSymbols](#) (PHEAD WORD *, WORD *, WORD)
- WORD [CompareHSymbols](#) (PHEAD WORD *, WORD *, WORD)
- WORD [NextPrime](#) (PHEAD WORD)
- WORD [Moebius](#) (PHEAD WORD)
- UWORD [wranf](#) (PHEAD0)
- UWORD [iranf](#) (PHEAD UWORD)
- void [iniwranf](#) (PHEAD0)
- UBYTE * [PreRandom](#) (UBYTE *)
- WORD * [PolyNormPoly](#) (PHEAD WORD)
- WORD * [EvaluateGcd](#) (PHEAD WORD *)
- int [TreatPolyRatFun](#) (PHEAD WORD *)
- WORD [ReadSaveHeader](#) (void)
- WORD [ReadSaveIndex](#) ([FILEINDEX](#) *)
- WORD [ReadSaveExpression](#) (UBYTE *, UBYTE *, LONG *, LONG *)
- UBYTE * [ReadSaveTerm32](#) (UBYTE *, UBYTE *, UBYTE **, UBYTE *, UBYTE *, int)
- WORD [ReadSaveVariables](#) (UBYTE *, UBYTE *, LONG *, LONG *, [INDEXENTRY](#) *, LONG *)
- WORD [WriteStoreHeader](#) (WORD)

- void [InitRecovery](#) (void)
- int [CheckRecoveryFile](#) (void)
- void [DeleteRecoveryFile](#) (void)
- char * [RecoveryFilename](#) (void)
- int [DoRecovery](#) (int *)
- void [DoCheckpoint](#) (int)
- void [NumberMallocAddMemory](#) (PHEAD0)
- void [CacheNumberMallocAddMemory](#) (PHEAD0)
- void [TermMallocAddMemory](#) (PHEAD0)
- void [ExprStatus](#) (EXPRESSIONS)
- void [iniTools](#) (void)
- int [TestTerm](#) (WORD *)
- WORD [RunTransform](#) (PHEAD WORD *term, WORD *params)
- WORD [RunEncode](#) (PHEAD WORD *fun, WORD *args, WORD *info)
- WORD [RunDecode](#) (PHEAD WORD *fun, WORD *args, WORD *info)
- WORD [RunReplace](#) (PHEAD WORD *fun, WORD *args, WORD *info)
- WORD [RunImplode](#) (WORD *fun, WORD *args)
- WORD [RunExplode](#) (PHEAD WORD *fun, WORD *args)
- int [TestArgNum](#) (int n, int totarg, WORD *args)
- WORD [PutArgInScratch](#) (WORD *arg, UWORD *scrat)
- UBYTE * [ReadRange](#) (UBYTE *s, WORD *out, int par)
- int [FindRange](#) (PHEAD WORD *, WORD *, WORD *, WORD)
- WORD [RunPermute](#) (PHEAD WORD *fun, WORD *args, WORD *info)
- WORD [RunReverse](#) (PHEAD WORD *fun, WORD *args)
- WORD [RunCycle](#) (PHEAD WORD *fun, WORD *args, WORD *info)
- WORD [RunAddArg](#) (PHEAD WORD *fun, WORD *args)
- WORD [RunMulArg](#) (PHEAD WORD *fun, WORD *args)
- WORD [RunIsLyndon](#) (PHEAD WORD *fun, WORD *args, int par)
- WORD [RunToLyndon](#) (PHEAD WORD *fun, WORD *args, int par)
- WORD [RunDropArg](#) (PHEAD WORD *fun, WORD *args)
- WORD [RunSelectArg](#) (PHEAD WORD *fun, WORD *args)
- WORD [RunDedup](#) (PHEAD WORD *fun, WORD *args)
- WORD [RunZtoHArg](#) (PHEAD WORD *fun, WORD *args)
- WORD [RunHtoZArg](#) (PHEAD WORD *fun, WORD *args)
- int [NormPolyTerm](#) (PHEAD WORD *)
- WORD [ComparePoly](#) (WORD *, WORD *, WORD)
- int [ConvertToPoly](#) (PHEAD WORD *, WORD *, WORD *, WORD)
- int [LocalConvertToPoly](#) (PHEAD WORD *, WORD *, WORD, WORD)
- int [ConvertFromPoly](#) (PHEAD WORD *, WORD *, WORD, WORD, WORD, WORD)
- WORD [FindSubterm](#) (WORD *)
- WORD [FindLocalSubterm](#) (PHEAD WORD *, WORD)
- void [PrintSubtermList](#) (int, int)
- void [PrintExtraSymbol](#) (int, WORD *, int)
- WORD [FindSubexpression](#) (WORD *)
- void [UpdateMaxSize](#) (void)
- int [CoToPolynomial](#) (UBYTE *)
- int [CoFromPolynomial](#) (UBYTE *)
- int [CoArgToExtraSymbol](#) (UBYTE *)
- int [CoExtraSymbols](#) (UBYTE *)
- UBYTE * [GetDoParam](#) (UBYTE *, WORD **, int)
- WORD * [GetIfDollarFactor](#) (UBYTE **, WORD *)
- int [CoDo](#) (UBYTE *)
- int [CoEndDo](#) (UBYTE *)
- int [ExtraSymFun](#) (PHEAD WORD *, WORD)
- int [PruneExtraSymbols](#) (WORD)

- int [IniFbuffer](#) (WORD)
- void [IniFbufs](#) (void)
- int [GCDfunction](#) (PHEAD WORD *, WORD)
- WORD * [GCDfunction3](#) (PHEAD WORD *, WORD *)
- WORD * [GCDfunction4](#) (PHEAD WORD *, WORD *)
- int [ReadPolyRatFun](#) (PHEAD WORD *)
- int [FromPolyRatFun](#) (PHEAD WORD *, WORD **, WORD **)
- void [PRFnormalize](#) (PHEAD WORD *)
- WORD * [PRFadd](#) (PHEAD WORD *, WORD *)
- WORD * [PolyDiv](#) (PHEAD WORD *, WORD *, char *)
- WORD * [PolyGCD](#) (PHEAD WORD *, WORD *)
- WORD * [PolyAdd](#) (PHEAD WORD *, WORD *)
- void [GCDclean](#) (PHEAD WORD *, WORD *)
- int [RatFunNormalize](#) (PHEAD WORD *)
- WORD * [TakeSymbolContent](#) (PHEAD WORD *, WORD *)
- int [GCDterms](#) (PHEAD WORD *, WORD *, WORD *)
- WORD * [PutExtraSymbols](#) (PHEAD WORD *, WORD, int *)
- WORD * [TakeExtraSymbols](#) (PHEAD WORD *, WORD)
- WORD * [MultiplyWithTerm](#) (PHEAD WORD *, WORD *, WORD)
- WORD * [TakeContent](#) (PHEAD WORD *, WORD *)
- int [MergeSymbolLists](#) (PHEAD WORD *, WORD *, int)
- int [MergeDotproductLists](#) (PHEAD WORD *, WORD *, int)
- WORD * [CreateExpression](#) (PHEAD WORD)
- int [DIVfunction](#) (PHEAD WORD *, WORD, int)
- WORD * [MULfunc](#) (PHEAD WORD *, WORD *)
- WORD * [ConvertArgument](#) (PHEAD WORD *, int *)
- int [ExpandRat](#) (PHEAD WORD *)
- int [InvPoly](#) (PHEAD WORD *, WORD, WORD)
- WORD [TestDoLoop](#) (PHEAD WORD *, WORD)
- WORD [TestEndDoLoop](#) (PHEAD WORD *, WORD)
- WORD * [poly_gcd](#) (PHEAD WORD *, WORD *, WORD)
- WORD * [poly_div](#) (PHEAD WORD *, WORD *, WORD)
- WORD * [poly_rem](#) (PHEAD WORD *, WORD *, WORD)
- WORD * [poly_inverse](#) (PHEAD WORD *, WORD *)
- WORD * [poly_mul](#) (PHEAD WORD *, WORD *)
- WORD * [poly_ratfun_add](#) (PHEAD WORD *, WORD *)
- int [poly_ratfun_normalize](#) (PHEAD WORD *)
- int [poly_factorize_argument](#) (PHEAD WORD *, WORD *)
- WORD * [poly_factorize_dollar](#) (PHEAD WORD *)
- int [poly_factorize_expression](#) (EXPRESSIONS)
- int [poly_unfactorize_expression](#) (EXPRESSIONS)
- void [poly_free_poly_vars](#) (PHEAD const char *)
- void [optimize_print_code](#) (int)
- int [DoPreAdd](#) (UBYTE *s)
- int [DoPreUseDictionary](#) (UBYTE *s)
- int [DoPreCloseDictionary](#) (UBYTE *s)
- int [DoPreOpenDictionary](#) (UBYTE *s)
- void [RemoveDictionary](#) (DICTIONARY *dict)
- void [UnSetDictionary](#) (void)
- int [SetDictionaryOptions](#) (UBYTE *options)
- int [SelectDictionary](#) (UBYTE *name, UBYTE *options)
- int [AddToDictionary](#) (DICTIONARY *dict, UBYTE *left, UBYTE *right)
- int [AddDictionary](#) (UBYTE *name)
- int [FindDictionary](#) (UBYTE *name)
- UBYTE * [IsExponentSign](#) (void)

- UBYTE * [IsMultiplySign](#) (void)
- void [TransformRational](#) (UWORD *a, WORD na)
- void [WriteDictionary](#) (DICTIONARY *)
- void [ShrinkDictionary](#) (DICTIONARY *)
- void [MultiplyToLine](#) (void)
- UBYTE * [FindSymbol](#) (WORD num)
- UBYTE * [FindVector](#) (WORD num)
- UBYTE * [FindIndex](#) (WORD num)
- UBYTE * [FindFunction](#) (WORD num)
- UBYTE * [FindFunWithArgs](#) (WORD *t)
- UBYTE * [FindExtraSymbol](#) (WORD num)
- LONG [DictToBytes](#) (DICTIONARY *dict, UBYTE *buf)
- DICTIONARY * [DictFromBytes](#) (UBYTE *buf)
- int [CoCreateSpectator](#) (UBYTE *inp)
- int [CoToSpectator](#) (UBYTE *inp)
- int [CoRemoveSpectator](#) (UBYTE *inp)
- int [CoEmptySpectator](#) (UBYTE *inp)
- int [CoCopySpectator](#) (UBYTE *inp)
- int [PutInSpectator](#) (WORD *, WORD)
- void [ClearSpectators](#) (WORD)
- WORD [GetFromSpectator](#) (WORD *, WORD)
- void [FlushSpectators](#) (void)
- WORD * [PreGCD](#) (PHEAD WORD *, WORD *, int)
- WORD * [FindCommonVariables](#) (PHEAD int, int)
- void [AddToSymbolList](#) (PHEAD WORD)
- int [AddToListPoly](#) (PHEAD0)
- int [ReadFromScratch](#) (FILEHANDLE *, POSITION *, UBYTE *, POSITION *)
- int [AddToScratch](#) (FILEHANDLE *, POSITION *, UBYTE *, POSITION *, int)
- int [DoPreAppendPath](#) (UBYTE *)
- int [DoPrePrependPath](#) (UBYTE *)
- int [DoSwitch](#) (PHEAD WORD *, WORD *)
- int [DoEndSwitch](#) (PHEAD WORD *, WORD *)
- SWITCHTABLE * [FindCase](#) (WORD, WORD)
- void [SwitchSplitMergeRec](#) (SWITCHTABLE *, WORD, SWITCHTABLE *)
- void [SwitchSplitMerge](#) (SWITCHTABLE *, WORD)
- int [DoubleSwitchBuffers](#) (void)
- int [DistrN](#) (int, int *, int, int *)
- int [DoNamespace](#) (UBYTE *)
- int [DoEndNamespace](#) (UBYTE *)
- int [DoUse](#) (UBYTE *)
- UBYTE * [SkipName](#) (UBYTE *)
- UBYTE * [ConstructName](#) (UBYTE *, UBYTE)
- int [DoSetUserFlag](#) (UBYTE *)
- int [DoClearUserFlag](#) (UBYTE *)
- int [DoUserFlag](#) (UBYTE *, int)
- VERTEX * [CreateVertex](#) (MODEL *)
- UBYTE * [ReadParticle](#) (UBYTE *, VERTEX *, MODEL *, int)
- int [CoModel](#) (UBYTE *)
- int [CoParticle](#) (UBYTE *)
- int [CoVertex](#) (UBYTE *)
- int [CoEndModel](#) (UBYTE *)
- int [LoadModel](#) (MODEL *)
- int [CoSetUserFlag](#) (UBYTE *)
- int [CoClearUserFlag](#) (UBYTE *)
- int [CoCreateAllLoops](#) (UBYTE *)

- int [CoCreateAllPaths](#) (UBYTE *)
- int [CoCreateAll](#) (UBYTE *)
- WORD [AllLoops](#) (PHEAD WORD *, WORD)
- LONG [StartLoops](#) (PHEAD WORD *, WORD, LONG, WORD, WORD *, WORD, WORD *, WORD)
- LONG [GenLoops](#) (PHEAD WORD *, WORD, LONG, WORD, WORD *, WORD, WORD *, WORD)
- void [LoopOutput](#) (PHEAD WORD *, WORD, WORD *, WORD)
- WORD [AllPaths](#) (PHEAD WORD *, WORD)
- LONG [GenPaths](#) (PHEAD WORD *, WORD, LONG, WORD, WORD *, WORD, WORD *, WORD)
- void [PathOutput](#) (PHEAD WORD *, WORD, WORD *, WORD)

4.19.1 Detailed Description

Contains macros and function declarations.

Definition in file [declare.h](#).

4.19.2 Macro Definition Documentation

4.19.2.1 MaX

```
#define MaX(
    x,
    y ) ((x) > (y) ? (x) : (y))
```

Definition at line 40 of file [declare.h](#).

4.19.2.2 MiN

```
#define MiN(
    x,
    y ) ((x) < (y) ? (x) : (y))
```

Definition at line 41 of file [declare.h](#).

4.19.2.3 ABS

```
#define ABS(
    x ) ((x) < 0 ? -(x) : (x))
```

Definition at line 42 of file [declare.h](#).

4.19.2.4 SGN

```
#define SGN(
    x ) ((x) > 0 ? 1 : (x) < 0 ? -1 : 0)
```

Definition at line 43 of file [declare.h](#).

4.19.2.5 REDLENG

```
#define REDLENG(  
    x )  ( ( (x)<0) ? ( (x)+1) : ( (x)-1) ) / 2)
```

Definition at line 44 of file [declare.h](#).

4.19.2.6 INCLENG

```
#define INCLENG(  
    x )  ( ( (x)<0) ? ( ( (x)*2)-1) : ( ( (x)*2)+1) )
```

Definition at line 45 of file [declare.h](#).

4.19.2.7 GETCOEF

```
#define GETCOEF(  
    x,  
    y )  x += *x; y = x[-1]; x -= ABS(y); y=REDLENG(y)
```

Definition at line 46 of file [declare.h](#).

4.19.2.8 GETSTOP

```
#define GETSTOP(  
    x,  
    y )  y=x+(*x)-1; y -= ABS(*y)-1
```

Definition at line 47 of file [declare.h](#).

4.19.2.9 StuffAdd

```
#define StuffAdd(  
    x,  
    y )  ( ( (x)<0?-1:1) * (y) + ( (y)<0?-1:1) * (x) )
```

Definition at line 48 of file [declare.h](#).

4.19.2.10 EXCHN

```
#define EXCHN(  
    t1,  
    t2,  
    n )  { WORD a,i; for(i=0;i<n;i++){a=t1[i];t1[i]=t2[i];t2[i]=a;} }
```

Definition at line 50 of file [declare.h](#).

4.19.2.11 EXCH

```
#define EXCH(  
    x,  
    y ) { WORD a = (x); (x) = (y); (y) = a; }
```

Definition at line 51 of file [declare.h](#).

4.19.2.12 TOKENTOLINE

```
#define TOKENTOLINE(  
    x,  
    y )
```

Value:

```
if ( AC.OutputSpaces == NOSPACEFORMAT ) { \  
TokenToLine((UBYTE *) (y)); } else { TokenToLine((UBYTE *) (x)); }
```

Definition at line 53 of file [declare.h](#).

4.19.2.13 UngetFromStream

```
#define UngetFromStream(  
    stream,  
    c ) ((stream)->nextchar[(stream)->isnextchar++]=c)
```

Definition at line 56 of file [declare.h](#).

4.19.2.14 AddLineFeed

```
#define AddLineFeed(  
    s,  
    n ) { (s)[(n)++] = LINEFEED; }
```

Definition at line 60 of file [declare.h](#).

4.19.2.15 TryRecover

```
#define TryRecover(  
    x ) Terminate(-1)
```

Definition at line 62 of file [declare.h](#).

4.19.2.16 UngetChar

```
#define UngetChar(  
    c ) { pushbackchar = c; }
```

Definition at line 63 of file [declare.h](#).

4.19.2.17 ParseNumber

```
#define ParseNumber(  
    x,  
    s ) { (x)=0;while(*(s)>='0'&&*(s)<='9') (x)=10*(x)+*(s)++ -'0'; }
```

Definition at line 64 of file [declare.h](#).

4.19.2.18 ParseSign

```
#define ParseSign(  
    sgn,  
    s )
```

Value:

```
{ (sgn)=0;while(*(s)=='-' || *(s)=='+') {\n  if ( *(s)++ == '-' ) sgn ^= 1;}}
```

Definition at line 65 of file [declare.h](#).

4.19.2.19 ParseSignedNumber

```
#define ParseSignedNumber(  
    x,  
    s )
```

Value:

```
{ int sgn; ParseSign(sgn,s)\n  ParseNumber(x,s) if ( sgn ) x = -x; }
```

Definition at line 67 of file [declare.h](#).

4.19.2.20 NCOPY

```
#define NCOPY(  
    s,  
    t,  
    n ) { while ( (n)-- > 0 ) { *s++ = *t++; } }
```

Definition at line 71 of file [declare.h](#).

4.19.2.21 NCOPYI

```
#define NCOPYI(  
    s,  
    t,  
    n ) { while ( (n)-- > 0 ) { *s++ = *t++; } }
```

Definition at line 73 of file [declare.h](#).

4.19.2.22 NCOPYB

```
#define NCOPYB(
    s,
    t,
    n ) { while ( (n)-- > 0 ) { *s++ = *t++; } }
```

Definition at line 74 of file [declare.h](#).

4.19.2.23 NCOPYI32

```
#define NCOPYI32(
    s,
    t,
    n ) { while ( (n)-- > 0 ) { *s++ = *t++; } }
```

Definition at line 75 of file [declare.h](#).

4.19.2.24 WCOPY

```
#define WCOPY(
    s,
    t,
    n ) { int nn=n; WORD *ss=(WORD *)s, *tt=(WORD *)t; while ( (nn)-- > 0 ) { *ss++ =
    *tt++; } }
```

Definition at line 76 of file [declare.h](#).

4.19.2.25 NeedNumber

```
#define NeedNumber(
    x,
    s,
    err )
```

Value:

```
{ int sgn = 1;
while ( *s == ' ' || *s == '\t' || *s == '-' || *s == '+' ) {
    if ( *s == '-' ) {sgn = -sgn;} s++; }
if ( chartype[*s] != 1 ) goto err;
ParseNumber(x,s)
if ( sgn < 0 ) { (x) = -(x); } while ( *s == ' ' || *s == '\t' ) s++;
}
```

Definition at line 78 of file [declare.h](#).

4.19.2.26 SKIPBLANKS

```
#define SKIPBLANKS(
    s ) { while ( *(s) == ' ' || *(s) == '\t' ) (s)++; }
```

Definition at line 85 of file [declare.h](#).

4.19.2.27 FLUSHCONSOLE

```
#define FLUSHCONSOLE if ( AP.InOutBuf > 0 ) CharOut(LINEFEED)
```

Definition at line 86 of file [declare.h](#).

4.19.2.28 SKIPBRA1

```
#define SKIPBRA1(  
    s )
```

Value:

```
{ int lev1=0; s++; while(*s) { if(*s=='['){lev1++;} \  
else {if(*s=='['&&--lev1<0){break;}} s++;} }
```

Definition at line 88 of file [declare.h](#).

4.19.2.29 SKIPBRA2

```
#define SKIPBRA2(  
    s )
```

Value:

```
{ int lev2=0; s++; while(*s) { if(*s=='{'){lev2++;} \  
else {if(*s=='{'&&--lev2<0){break;}} \  
else {if(*s=='{'){SKIPBRA1(s)}} s++;} }
```

Definition at line 90 of file [declare.h](#).

4.19.2.30 SKIPBRA3

```
#define SKIPBRA3(  
    s )
```

Value:

```
{ int lev3=0; s++; while(*s) { if(*s=='('){lev3++;} \  
else {if(*s=='('&&--lev3<0){break;}} \  
else {if(*s=='('){SKIPBRA2(s)} \  
else {if(*s=='('){SKIPBRA1(s)}}} s++;} }
```

Definition at line 93 of file [declare.h](#).

4.19.2.31 SKIPBRA4

```
#define SKIPBRA4(  
    s )
```

Value:

```
{ int lev4=0; s++; while(*s) { if(*s=='('){lev4++;} \  
else {if(*s=='('&&--lev4<0){break;}} \  
else {if(*s=='('){SKIPBRA1(s)}} s++;} }
```

Definition at line 97 of file [declare.h](#).

4.19.2.32 SKIPBRA5

```
#define SKIPBRA5(
    s )
```

Value:

```
{ int lev5=0; s++; while(*s) { if(*s=='{') {lev5++;} \
else {if(*s=='}') {&&--lev5<0} {break;} \
else {if(*s=='(') {SKIPBRA4(s)} \
else {if(*s=='{') {SKIPBRA1(s)}}} s++;} }
```

Definition at line 100 of file [declare.h](#).

4.19.2.33 CYCLE1

```
#define CYCLE1(
    t,
    a,
    i ) {t iX=*a; WORD jX; for(jX=1;jX<i;jX++)a[jX-1]=a[jX]; a[i-1]=iX;}
```

Definition at line 108 of file [declare.h](#).

4.19.2.34 AddToCB

```
#define AddToCB(
    c,
    wx )
```

Value:

```
if(c->Pointer>=c->Top) \
{DoubleCbuffer(c-cbuf,c->Pointer,21);} \
*(c->Pointer)++ = wx;
```

Definition at line 110 of file [declare.h](#).

4.19.2.35 EXCHINOUT

```
#define EXCHINOUT
```

Value:

```
{ FILEHANDLE *ffFi = AR.outfile; \
AR.outfile = AR.infile; AR.infile = ffFi; }
```

Definition at line 114 of file [declare.h](#).

4.19.2.36 BACKINOUT

```
#define BACKINOUT
```

Value:

```
{ FILEHANDLE *ffFi = AR.outfile; POSITION posi; \
AR.outfile = AR.infile; AR.infile = ffFi; \
SetEndScratch(AR.infile,&posi); }
```

Definition at line 116 of file [declare.h](#).

4.19.2.37 CopyArg

```
#define CopyArg(  
    to,  
    from )
```

Value:

```
{ if ( *from > 0 ) { int ica = *from; NCOPY(to,from,ica) } \  
else if ( *from <= -FUNCTION ) *to++ = *from++; \  
else { *to++ = *from++; *to++ = *from++; } }
```

Definition at line 120 of file [declare.h](#).

4.19.2.38 FILLARG

```
#define FILLARG(  
    w )
```

Definition at line 129 of file [declare.h](#).

4.19.2.39 COPYARG

```
#define COPYARG(  
    w,  
    t )
```

Definition at line 130 of file [declare.h](#).

4.19.2.40 ZEROARG

```
#define ZEROARG(  
    w )
```

Definition at line 131 of file [declare.h](#).

4.19.2.41 FILLFUN

```
#define FILLFUN(  
    w )
```

Definition at line 138 of file [declare.h](#).

4.19.2.42 COPYFUN

```
#define COPYFUN(  
    w,  
    t )
```

Definition at line 139 of file [declare.h](#).

4.19.2.43 COPYFUN3

```
#define COPYFUN3(  
    w,  
    t )
```

Definition at line 146 of file [declare.h](#).

4.19.2.44 FILLFUN3

```
#define FILLFUN3(  
    w )
```

Definition at line 147 of file [declare.h](#).

4.19.2.45 FILLSUB

```
#define FILLSUB(  
    w )
```

Definition at line 154 of file [declare.h](#).

4.19.2.46 COPYSUB

```
#define COPYSUB(  
    w,  
    ww )
```

Definition at line 155 of file [declare.h](#).

4.19.2.47 FILLEXP

```
#define FILLEXP(  
    w )
```

Definition at line 161 of file [declare.h](#).

4.19.2.48 NEXTARG

```
#define NEXTARG(  
    x ) if(*x>0) x += *x; else if(*x <= -FUNCTION)x++; else x += 2;
```

Definition at line 164 of file [declare.h](#).

4.19.2.49 COPY1ARG

```
#define COPY1ARG(  
    s1,  
    t1 )
```

Value:

```
{ int ica; if ( (ica==t1) > 0 ) { NCOPY(s1,t1,ica) } \  
else if(*t1<=-FUNCTION){*s1++=*t1++;} else{*s1++=*t1++;*s1++=*t1++;} }
```

Definition at line 165 of file [declare.h](#).

4.19.2.50 ZeroFillRange

```
#define ZeroFillRange(  
    w,  
    begin,  
    end )
```

Value:

```
do { \  
    int tmp_i; \  
    for ( tmp_i = begin; tmp_i < end; tmp_i++ ) { (w)[tmp_i] = 0; } \  
} while (0)
```

Fills a buffer by zero in the range [begin,end).

Parameters

<i>w</i>	The buffer.
<i>begin</i>	The index for the beginning of the range.
<i>end</i>	The index for the end of the range (exclusive).

Definition at line 175 of file [declare.h](#).

4.19.2.51 TABLESIZE

```
#define TABLESIZE(  
    a,  
    b ) ( (WORD) sizeof(a) / ( (WORD) sizeof(b) ) )
```

Definition at line 180 of file [declare.h](#).

4.19.2.52 WORDDIF

```
#define WORDDIF(  
    x,  
    y ) (WORD) (x-y)
```

Definition at line 181 of file [declare.h](#).

4.19.2.53 `wsizet`

```
#define wsizet(  
    a ) ((WORD)sizeof(a))
```

Definition at line 182 of file [declare.h](#).

4.19.2.54 `VARNAME`

```
#define VARNAME(  
    type,  
    num ) (AC.varnames->namebuffer+type[num].name)
```

Definition at line 183 of file [declare.h](#).

4.19.2.55 `DOLLARNAME`

```
#define DOLLARNAME(  
    type,  
    num ) (AC.dollarnames->namebuffer+type[num].name)
```

Definition at line 184 of file [declare.h](#).

4.19.2.56 `EXPRNAME`

```
#define EXPRNAME(  
    num ) (AC.exprnames->namebuffer+Expressions[num].name)
```

Definition at line 185 of file [declare.h](#).

4.19.2.57 `PREV`

```
#define PREV(  
    x ) prevorder?prevorder:x
```

Definition at line 187 of file [declare.h](#).

4.19.2.58 `Terminate`

```
#define Terminate(  
    x ) do { TerminateImpl(x, __FILE__, __LINE__, __FUNCTION__); } while(0)
```

Definition at line 189 of file [declare.h](#).

4.19.2.59 `SETERROR`

```
#define SETERROR(  
    x ) { Terminate(-1); return(-1); }
```

Definition at line 190 of file [declare.h](#).

4.19.2.60 DUMMYUSE

```
#define DUMMYUSE(  
    x ) (void) (x);
```

Definition at line 193 of file [declare.h](#).

4.19.2.61 ADDPOS

```
#define ADDPOS(  
    pp,  
    x ) (pp).p1 = ((pp).p1+(LONG)(x))
```

Definition at line 219 of file [declare.h](#).

4.19.2.62 SETBASELENGTH

```
#define SETBASELENGTH(  
    ss,  
    x ) (ss).p1 = (LONG)(x)
```

Definition at line 220 of file [declare.h](#).

4.19.2.63 SETBASEPOSITION

```
#define SETBASEPOSITION(  
    pp,  
    x ) (pp).p1 = (LONG)(x)
```

Definition at line 221 of file [declare.h](#).

4.19.2.64 ISEQUALPOSINC

```
#define ISEQUALPOSINC(  
    pp1,  
    pp2,  
    x ) ((pp1).p1 == ((pp2).p1+(LONG)(x)) )
```

Definition at line 222 of file [declare.h](#).

4.19.2.65 ISGEPOSINC

```
#define ISGEPOSINC(  
    pp1,  
    pp2,  
    x ) ((pp1).p1 >= ((pp2).p1+(LONG)(x)) )
```

Definition at line 223 of file [declare.h](#).

4.19.2.66 DIVPOS

```
#define DIVPOS(  
    pp,  
    n ) ( (pp).p1 / (LONG) (n) )
```

Definition at line 224 of file [declare.h](#).

4.19.2.67 MULPOS

```
#define MULPOS(  
    pp,  
    n ) (pp).p1 *= (LONG) (n)
```

Definition at line 225 of file [declare.h](#).

4.19.2.68 DIFPOS

```
#define DIFPOS(  
    ss,  
    pp1,  
    pp2 ) (ss).p1 = ((pp1).p1 - (pp2).p1)
```

Definition at line 228 of file [declare.h](#).

4.19.2.69 DIFBASE

```
#define DIFBASE(  
    pp1,  
    pp2 ) ((pp1).p1 - (pp2).p1)
```

Definition at line 229 of file [declare.h](#).

4.19.2.70 ADD2POS

```
#define ADD2POS(  
    pp1,  
    pp2 ) (pp1).p1 += (pp2).p1
```

Definition at line 230 of file [declare.h](#).

4.19.2.71 PUTZERO

```
#define PUTZERO(  
    pp ) (pp).p1 = 0
```

Definition at line 231 of file [declare.h](#).

4.19.2.72 BASEPOSITION

```
#define BASEPOSITION(  
    pp ) ( (pp).p1 )
```

Definition at line 232 of file [declare.h](#).

4.19.2.73 SETSTARTPOS

```
#define SETSTARTPOS(  
    pp ) (pp).p1 = -2
```

Definition at line 233 of file [declare.h](#).

4.19.2.74 NOTSTARTPOS

```
#define NOTSTARTPOS(  
    pp ) ( (pp).p1 > -2 )
```

Definition at line 234 of file [declare.h](#).

4.19.2.75 ISMINPOS

```
#define ISMINPOS(  
    pp ) ( (pp).p1 == -1 )
```

Definition at line 235 of file [declare.h](#).

4.19.2.76 ISEQUALPOS

```
#define ISEQUALPOS(  
    pp1,  
    pp2 ) ( (pp1).p1 == (pp2).p1 )
```

Definition at line 236 of file [declare.h](#).

4.19.2.77 ISNOTEQUALPOS

```
#define ISNOTEQUALPOS(  
    pp1,  
    pp2 ) ( (pp1).p1 != (pp2).p1 )
```

Definition at line 237 of file [declare.h](#).

4.19.2.78 ISLESSPOS

```
#define ISLESSPOS(  
    pp1,  
    pp2 ) ( (pp1).p1 < (pp2).p1 )
```

Definition at line 238 of file [declare.h](#).

4.19.2.79 ISGEPOS

```
#define ISGEPOS(  
    pp1,  
    pp2 ) ( (pp1).p1 >= (pp2).p1 )
```

Definition at line 239 of file [declare.h](#).

4.19.2.80 ISNOTZEROPOS

```
#define ISNOTZEROPOS(  
    pp ) ( (pp).p1 != 0 )
```

Definition at line 240 of file [declare.h](#).

4.19.2.81 ISZEROPOS

```
#define ISZEROPOS(  
    pp ) ( (pp).p1 == 0 )
```

Definition at line 241 of file [declare.h](#).

4.19.2.82 ISPOSPOS

```
#define ISPOSPOS(  
    pp ) ( (pp).p1 > 0 )
```

Definition at line 242 of file [declare.h](#).

4.19.2.83 ISNEGPOS

```
#define ISNEGPOS(  
    pp ) ( (pp).p1 < 0 )
```

Definition at line 243 of file [declare.h](#).

4.19.2.84 TOLONG

```
#define TOLONG(  
    x ) ( (LONG) (x) )
```

Definition at line 246 of file [declare.h](#).

4.19.2.85 Add2Com

```
#define Add2Com(  
    x ) { WORD cod[2]; cod[0] = x; cod[1] = 2; AddNtoL(2,cod); }
```

Definition at line 248 of file [declare.h](#).

4.19.2.86 Add3Com

```
#define Add3Com(  
    x1,  
    x2 ) { WORD cod[3]; cod[0] = x1; cod[1] = 3; cod[2] = x2; AddNtoL(3,cod); }
```

Definition at line 249 of file [declare.h](#).

4.19.2.87 Add4Com

```
#define Add4Com(  
    x1,  
    x2,  
    x3 )
```

Value:

```
{ WORD cod[4]; cod[0] = x1; cod[1] = 4; \  
cod[2] = x2; cod[3] = x3; AddNtoL(4,cod); }
```

Definition at line 250 of file [declare.h](#).

4.19.2.88 Add5Com

```
#define Add5Com(  
    x1,  
    x2,  
    x3,  
    x4 )
```

Value:

```
{ WORD cod[5]; cod[0] = x1; cod[1] = 5; \  
cod[2] = x2; cod[3] = x3; cod[4] = x4; AddNtoL(5,cod); }
```

Definition at line 252 of file [declare.h](#).

4.19.2.89 WantAddPointers

```
#define WantAddPointers(  
    x )
```

Value:

```
while((AT.pWorkPointer+(x))>AR.pWorkSize){WORD ***ppp=&AT.pWorkSpace;\  
ExpandBuffer((void **)ppp,&AR.pWorkSize,(int)(sizeof(WORD *)));}
```

Definition at line 258 of file [declare.h](#).

4.19.2.90 WantAddLongs

```
#define WantAddLongs(
    x )
```

Value:

```
while ( (AT.lWorkPointer+(x))>AR.lWorkSize) {LONG **ppp=&AT.lWorkSpace;\
ExpandBuffer( (void **)ppp, &AR.lWorkSize, sizeof(LONG) ); }
```

Definition at line 260 of file [declare.h](#).

4.19.2.91 WantAddPositions

```
#define WantAddPositions(
    x )
```

Value:

```
while ( (AT.posWorkPointer+(x))>AR.posWorkSize) {POSITION **ppp=&AT.posWorkSpace;\
ExpandBuffer( (void **)ppp, &AR.posWorkSize, sizeof(POSITION) ); }
```

Definition at line 262 of file [declare.h](#).

4.19.2.92 FORM_INLINE

```
#define FORM_INLINE inline
```

Definition at line 266 of file [declare.h](#).

4.19.2.93 MEMORYMACROS

```
#define MEMORYMACROS
```

Definition at line 274 of file [declare.h](#).

4.19.2.94 TermMalloc

```
#define TermMalloc(
    x ) ( (AT.TermMemTop <= 0 ) ? TermMallocAddMemory(BHEAD0), AT.TermMemHeap[--AT.TermMemTop]: AT.TermMemHeap[--AT.TermMemTop] )
```

Definition at line 278 of file [declare.h](#).

4.19.2.95 NumberMalloc

```
#define NumberMalloc(
    x ) ( (AT.NumberMemTop <= 0 ) ? NumberMallocAddMemory(BHEAD0), AT.NumberMemHeap[--AT.NumberMemTop]: AT.NumberMemHeap[--AT.NumberMemTop] )
```

Definition at line 279 of file [declare.h](#).

4.19.2.96 CacheNumberMalloc

```
#define CacheNumberMalloc(  
    x ) ( (AT.CacheNumberMemTop <= 0 ) ? CacheNumberMallocAddMemory(BHEAD0), AT.↵  
CacheNumberMemHeap[--AT.CacheNumberMemTop]: AT.CacheNumberMemHeap[--AT.CacheNumberMemTop] )
```

Definition at line 280 of file [declare.h](#).

4.19.2.97 TermFree

```
#define TermFree(  
    TermMem,  
    x ) AT.TermMemHeap[AT.TermMemTop++] = (WORD *) (TermMem)
```

Definition at line 281 of file [declare.h](#).

4.19.2.98 NumberFree

```
#define NumberFree(  
    NumberMem,  
    x ) AT.NumberMemHeap[AT.NumberMemTop++] = (UWORD *) (NumberMem)
```

Definition at line 282 of file [declare.h](#).

4.19.2.99 CacheNumberFree

```
#define CacheNumberFree(  
    NumberMem,  
    x ) AT.CacheNumberMemHeap[AT.CacheNumberMemTop++] = (UWORD *) (NumberMem)
```

Definition at line 283 of file [declare.h](#).

4.19.2.100 NestingChecksum

```
#define NestingChecksum( ) (AC.IfLevel + AC.RepLevel + AC.arglevel + AC.insidelevel + AC.↵  
termlevel + AC.inexprlevel + AC.dolooplevel + AC.SwitchLevel)
```

Definition at line 311 of file [declare.h](#).

4.19.2.101 MesNesting

```
#define MesNesting( ) MesPrint("&Illegal nesting of if, repeat, argument, inside, term, inexpression  
and do")
```

Definition at line 312 of file [declare.h](#).

4.19.2.102 MarkPolyRatFunDirty

```
#define MarkPolyRatFunDirty(
    T )
```

Value:

```
{if(*T&&AR.PolyFunType==2){WORD *TP,*TT;TT=T+*T;TT-=ABS(TT[-1]);\
TP=T+1;while(TP<TT){if(*TP==AR.PolyFun){TP[2]|=(DIRTYFLAG|MUSTCLEANPRF);}TP+=TP[1];}}}
```

Definition at line 314 of file [declare.h](#).

4.19.2.103 PUSHPREASSIGNLEVEL

```
#define PUSHPREASSIGNLEVEL
```

Value:

```
AP.PreAssignLevel++; { GETIDENTITY \
if ( AP.PreAssignLevel >= AP.MaxPreAssignLevel ) { int i; \
LONG *ap = (LONG *)Malloc1(2*AP.MaxPreAssignLevel*sizeof(LONG *),"PreAssignStack"); \
for ( i = 0; i < AP.MaxPreAssignLevel; i++ ) ap[i] = AP.PreAssignStack[i]; \
M_free(AP.PreAssignStack,"PreAssignStack"); \
AP.MaxPreAssignLevel *= 2; AP.PreAssignStack = ap; \
} \
*AT.WorkPointer++ = AP.PreContinuation; AP.PreContinuation = 0; \
AP.PreAssignStack[AP.PreAssignLevel] = AC.iPointer - AC.iBuffer; }
```

Definition at line 321 of file [declare.h](#).

4.19.2.104 POPPREASSIGNLEVEL

```
#define POPPREASSIGNLEVEL
```

Value:

```
if ( AP.PreAssignLevel > 0 ) { GETIDENTITY \
AC.iPointer = AC.iBuffer + AP.PreAssignStack[AP.PreAssignLevel--]; \
AP.PreContinuation = ---AT.WorkPointer; \
*AC.iPointer = 0; }
```

Definition at line 331 of file [declare.h](#).

4.19.2.105 EXTERNLOCK

```
#define EXTERNLOCK(
    x )
```

NOTE: We have replaced LOCK(ErrorMessageLock) and UNLOCK(ErrorMessageLock) by MLOCK(ErrorMessageLock) and MUNLOCK(ErrorMessageLock). They are used for the synchronised output in ParFORM. (TU 28 May 2011)

Definition at line 477 of file [declare.h](#).

4.19.2.106 INILOCK

```
#define INILOCK(
    x )
```

Definition at line 478 of file [declare.h](#).

4.19.2.107 LOCK

```
#define LOCK(  
    x )
```

Definition at line 479 of file [declare.h](#).

4.19.2.108 UNLOCK

```
#define UNLOCK(  
    x )
```

Definition at line 480 of file [declare.h](#).

4.19.2.109 EXTERNRWLOCK

```
#define EXTERNRWLOCK(  
    x )
```

Definition at line 481 of file [declare.h](#).

4.19.2.110 INIRWLOCK

```
#define INIRWLOCK(  
    x )
```

Definition at line 482 of file [declare.h](#).

4.19.2.111 RWLOCKR

```
#define RWLOCKR(  
    x )
```

Definition at line 483 of file [declare.h](#).

4.19.2.112 RWLOCKW

```
#define RWLOCKW(  
    x )
```

Definition at line 484 of file [declare.h](#).

4.19.2.113 UNRWLOCK

```
#define UNRWLOCK(  
    x )
```

Definition at line 485 of file [declare.h](#).

4.19.2.114 MLOCK

```
#define MLOCK(  
    x )
```

Definition at line 490 of file [declare.h](#).

4.19.2.115 MUNLOCK

```
#define MUNLOCK(  
    x )
```

Definition at line 491 of file [declare.h](#).

4.19.2.116 GETIDENTITY

```
#define GETIDENTITY
```

Definition at line 493 of file [declare.h](#).

4.19.2.117 GETBIDENTITY

```
#define GETBIDENTITY
```

Definition at line 494 of file [declare.h](#).

4.19.2.118 M_alloc

```
#define M_alloc(  
    x ) malloc((size_t)(x))
```

Definition at line 1009 of file [declare.h](#).

4.19.2.119 CompareTerms

```
#define CompareTerms ((COMPARE)AR.CompareRoutine)
```

Definition at line 1490 of file [declare.h](#).

4.19.2.120 FiniShuffle

```
#define FiniShuffle AN.SHvar.finishhuf
```

Definition at line 1491 of file [declare.h](#).

4.19.2.121 DoShuffle

```
#define DoShuffle ((DO_UFFLE)AN.SHvar.do_uffle)
```

Definition at line 1492 of file [declare.h](#).

4.19.3 Typedef Documentation

4.19.3.1 WRITEBUFTOEXTCHANNEL

```
typedef int(* WRITEBUFTOEXTCHANNEL) (char *, size_t)
```

Definition at line 1483 of file [declare.h](#).

4.19.3.2 GETCFROMEXTCHANNEL

```
typedef int(* GETCFROMEXTCHANNEL) (void)
```

Definition at line 1484 of file [declare.h](#).

4.19.3.3 SETTERMINATORFOREXTERNALCHANNEL

```
typedef int(* SETTERMINATORFOREXTERNALCHANNEL) (char *)
```

Definition at line 1485 of file [declare.h](#).

4.19.3.4 SETKILLMODEFOREXTERNALCHANNEL

```
typedef int(* SETKILLMODEFOREXTERNALCHANNEL) (int, int)
```

Definition at line 1486 of file [declare.h](#).

4.19.3.5 WRITEFILE

```
typedef LONG(* WRITEFILE) (int, UBYTE *, LONG)
```

Definition at line 1487 of file [declare.h](#).

4.19.3.6 GETTERM

```
typedef WORD(* GETTERM) (PHEAD WORD *)
```

Definition at line 1488 of file [declare.h](#).

4.19.4 Function Documentation

4.19.4.1 TELLFILE()

```
void TELLFILE (
    int handle,
    POSITION * position ) [extern]
```

Definition at line 1406 of file [tools.c](#).

4.19.4.2 StartVariables()

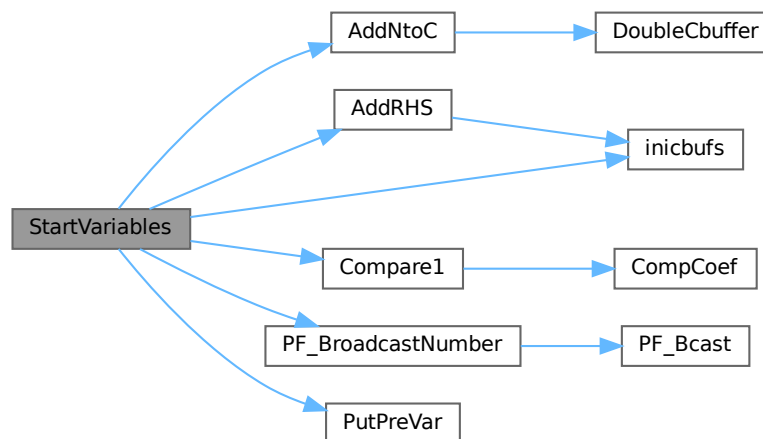
```
void StartVariables (
    void ) [extern]
```

All functions (well, nearly all) are declared here.

Definition at line 905 of file [startup.c](#).

References [AddNtoC\(\)](#), [AddRHS\(\)](#), [Compare1\(\)](#), [inicbufs\(\)](#), [FuNcTiOn::name](#), [PF_BroadcastNumber\(\)](#), [PutPreVar\(\)](#), and [FuNcTiOn::symmetric](#).

Here is the call graph for this function:



4.19.4.3 CodeToLine()

```
UBYTE * CodeToLine (
    WORD number,
    UBYTE * Out ) [extern]
```

Definition at line 658 of file [sch.c](#).

4.19.4.4 AddArrayIndex()

```

UBYTE * AddArrayIndex (
    WORD num,
    UBYTE * out ) [extern]

```

Definition at line 696 of file [sch.c](#).

4.19.4.5 FindInIndex()

```

INDEXENTRY * FindInIndex (
    WORD expr,
    FILEDATA * f,
    WORD par,
    WORD mode ) [extern]

```

Definition at line 2665 of file [store.c](#).

4.19.4.6 NextFileIndex()

```

INDEXENTRY * NextFileIndex (
    POSITION * indexpos ) [extern]

```

Definition at line 2258 of file [store.c](#).

4.19.4.7 PasteTerm()

```

WORD * PasteTerm (
    PHEAD WORD number,
    WORD * accum,
    WORD * position,
    WORD times,
    WORD divby ) [extern]

```

Puts the term at position in the accumulator accum at position 'number+1'. if times > 0 the coefficient of this term is multiplied by times/divby.

Parameters

<i>number</i>	The number of term fragments in accum that should be skipped
<i>accum</i>	The accumulator of term fragments
<i>position</i>	A position in (typically) a compiler buffer from where a (piece of a) term comes.
<i>times</i>	Multiply the result by this
<i>divby</i>	Divide the result by this.

This routine is typically used when we have to replace a (sub)expression pointer by a power of a (sub)expression. This uses mostly a binomial expansion and the new term is the old term multiplied one by one by terms of the new expression. The factors times and divby keep track of the binomial coefficient. Once this is complete, the routine FiniTerm will make the contents of the accumulator into a proper term that still needs to be normalized.

Definition at line [3002](#) of file [proces.c](#).

Referenced by [Generator\(\)](#).

4.19.4.8 StrCopy()

```
UBYTE * StrCopy (
    UBYTE * from,
    UBYTE * to ) [extern]
```

Definition at line [67](#) of file [sch.c](#).

4.19.4.9 WrtPower()

```
UBYTE * WrtPower (
    UBYTE * Out,
    WORD Power ) [extern]
```

Definition at line [738](#) of file [sch.c](#).

4.19.4.10 AccumGCD()

```
WORD AccumGCD (
    PHEAD UWORD * a,
    WORD * na,
    UWORD * b,
    WORD nb ) [extern]
```

Definition at line [647](#) of file [reken.c](#).

4.19.4.11 AddArgs()

```
void AddArgs (
    PHEAD WORD * s1,
    WORD * s2,
    WORD * m ) [extern]
```

Adds the arguments of two occurrences of the PolyFun.

Parameters

<i>s1</i>	Pointer to the first occurrence.
<i>s2</i>	Pointer to the second occurrence.
<i>m</i>	Pointer to where the answer should be.

Definition at line [2343](#) of file [sort.c](#).

Referenced by [AddPoly\(\)](#), and [MergePatches\(\)](#).

4.19.4.12 AddCoef()

```
WORD AddCoef (
    PHEAD WORD ** ps1,
    WORD ** ps2 ) [extern]
```

Adds the coefficients of the terms *ps1 and *ps2. The problem comes when there is not enough space for a new longer coefficient. First a local solution is tried. If this is not successful we need to move terms around. The possibility of a garbage collection should not be ignored, as avoiding this costs very much extra space which is nearly wasted otherwise.

If the return value is zero the terms cancelled.

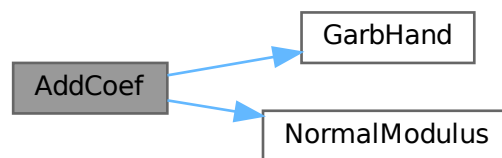
The resulting term is left in *ps1.

Definition at line 2042 of file [sort.c](#).

References [GarbHand\(\)](#), and [NormalModulus\(\)](#).

Referenced by [SplitMerge\(\)](#).

Here is the call graph for this function:



4.19.4.13 AddLong()

```
WORD AddLong (
    UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc ) [extern]
```

Definition at line 705 of file [reken.c](#).

4.19.4.14 AddPLon()

```
WORD AddPLon (
    UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc ) [extern]
```

Definition at line 750 of file [reken.c](#).

4.19.4.15 AddPoly()

```
WORD AddPoly (
    PHEAD WORD ** ps1,
    WORD ** ps2 ) [extern]
```

Routine should be called when $S \rightarrow \text{PolyWise} \neq 0$. It points then to the position of AR.PolyFun in both terms.

We add the contents of the arguments of the two polynomials. Special attention has to be given to special arguments. We have to reserve a space equal to the size of one term + the size of the argument of the other. The addition has to be done in this routine because not all objects are reentrant.

Newer addition (12-nov-2007). The PolyFun can have two arguments. In that case $S \rightarrow \text{PolyFlag}$ is 2 and we have to call the routine for adding rational polynomials. We have to be rather careful what happens with: The location of the output The order of the terms in the arguments At first we allow only univariate polynomials in the PolyFun. This restriction will be lifted a.s.a.p.

Parameters

<i>ps1</i>	A pointer to the position of the first term
<i>ps2</i>	A pointer to the position of the second term

Returns

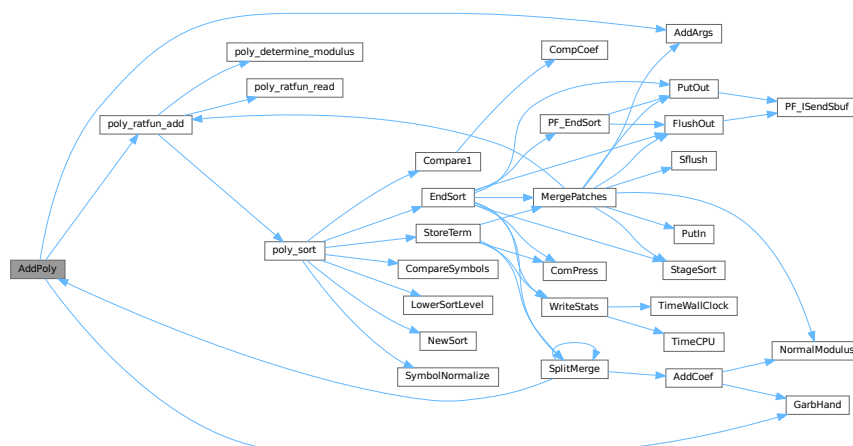
If zero the terms cancel. Otherwise the new term is in **ps1*.

Definition at line 2171 of file [sort.c](#).

References [AddArgs\(\)](#), [GarbHand\(\)](#), and [poly_ratfun_add\(\)](#).

Referenced by [SplitMerge\(\)](#).

Here is the call graph for this function:



4.19.4.16 AddRat()

```
WORD AddRat (
    PHEAD UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc ) [extern]
```

Definition at line 210 of file [reken.c](#).

4.19.4.17 AddToLine()

```
void AddToLine (
    UBYTE * s ) [extern]
```

Definition at line 82 of file [sch.c](#).

4.19.4.18 AddWild()

```
WORD AddWild (
    PHEAD WORD,
    WORD type,
    WORD newnumber ) [extern]
```

Definition at line 1544 of file [wildcard.c](#).

4.19.4.19 BigLong()

```
WORD BigLong (
    UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb ) [extern]
```

Definition at line 944 of file [reken.c](#).

4.19.4.20 BinomGen()

```
WORD BinomGen (
    PHEAD WORD * term,
    WORD level,
    WORD ** tstops,
    WORD x1,
    WORD x2,
    WORD pow1,
    WORD pow2,
    WORD sign,
    UWORD * coef,
    WORD ncoef ) [extern]
```

Definition at line 320 of file [ratio.c](#).

4.19.4.21 CheckWild()

```
WORD CheckWild (
    PHEAD WORD,
    WORD type,
    WORD newnumber,
    WORD * newval ) [extern]
```

Definition at line 1791 of file [wildcard.c](#).

4.19.4.22 Chisholm()

```
WORD Chisholm (
    PHEAD WORD * term,
    WORD level ) [extern]
```

Definition at line 1963 of file [opera.c](#).

4.19.4.23 CleanExpr()

```
WORD CleanExpr (
    WORD par ) [extern]
```

Definition at line 47 of file [execute.c](#).

4.19.4.24 CleanUp()

```
void CleanUp (
    WORD par ) [extern]
```

Definition at line 1761 of file [startup.c](#).

4.19.4.25 ClearWild()

```
void ClearWild (
    PHEAD0 ) [extern]
```

Definition at line 1518 of file [wildcard.c](#).

4.19.4.26 CompareFunctions()

```
WORD CompareFunctions (
    WORD * fleft,
    WORD * fright ) [extern]
```

Definition at line 54 of file [normal.c](#).

4.19.4.27 Commute()

```
WORD Commute (
    WORD * fleft,
    WORD * fright ) [extern]
```

Definition at line 118 of file [normal.c](#).

4.19.4.28 DetCommu()

```
WORD DetCommu (
    WORD * terms ) [extern]
```

Definition at line 4510 of file [normal.c](#).

4.19.4.29 DoesCommu()

```
WORD DoesCommu (
    WORD * term ) [extern]
```

Definition at line 4569 of file [normal.c](#).

4.19.4.30 CompArg()

```
int CompArg (
    WORD * s1,
    WORD * s2 ) [extern]
```

Definition at line 3307 of file [tools.c](#).

4.19.4.31 CompCoef()

```
WORD CompCoef (
    WORD * term1,
    WORD * term2 ) [extern]
```

Routine takes $a_1 \bmod m_1$ and $a_2 \bmod m_2$ and returns $a \bmod m_1*m_2$ with
 $a \bmod m_1 = a_1$ and $a \bmod m_2 = a_2$

Chinese remainder: $a \bmod (m_1*m_2) = q_1*m_1 + a_1$ $a \bmod (m_1*m_2) = q_2*m_2 + a_2$ Compute n_1 such that $(n_1*m_1)m_2$ is one
 Compute n_2 such that $(n_2*m_2)m_1$ is one Then $(a_1*n_2*m_2 + a_2*n_1*m_1) \bmod (m_1*m_2)$ is $a \bmod (m_1*m_2)$

Definition at line 3047 of file [reken.c](#).

Referenced by [Compare1\(\)](#).

4.19.4.32 CompGroup()

```
WORD CompGroup (
    PHEAD WORD,
    WORD ** args,
    WORD * a1,
    WORD * a2,
    WORD num ) [extern]
```

Definition at line 373 of file [function.c](#).

4.19.4.33 Compare1()

```
WORD Compare1 (
    PHEAD WORD * term1,
    WORD * term2,
    WORD level ) [extern]
```

Compares two terms. The answer is: 0 equal (with exception of the coefficient if level == 0.) >0 term1 comes first. <0 term2 comes first. Some special precautions may be needed to keep the CompCoef routine from generating overflows, although this is very unlikely in subterms. This routine should not return an error condition.

Originally this routine was called Compare. With the treatment of special polynomials with terms that contain only symbols and the need for extreme speed for the polynomial routines we made a special compare routine and now we store the address of the current compare routine in AR.CompareRoutine and have a macro Compare which makes all existing code work properly and we can just replace the routine on a thread by thread basis (each thread has its own AR struct).

Parameters

<i>term1</i>	First input term
<i>term2</i>	Second input term
<i>level</i>	The sorting level (may influence on the result)

Returns

0 equal (with exception of the coefficient if level == 0.) >0 term1 comes first. <0 term2 comes first.

When there are floating point numbers active (float_ = FLOATFUN) the presence of one or more float_ functions is returned in AT.SortFloatMode: 0: no float_ 1: float_ in term1 only 2: float_ in term2 only 3: float_ in both terms

Definition at line 2636 of file [sort.c](#).

References [CompCoef\(\)](#).

Referenced by [InFunction\(\)](#), [poly_sort\(\)](#), and [StartVariables\(\)](#).

Here is the call graph for this function:



4.19.4.34 CountDo()

```
WORD CountDo (
    WORD * term,
    WORD * instruct ) [extern]
```

Definition at line 552 of file [reshuf.c](#).

4.19.4.35 CountFun()

```
WORD CountFun (
    WORD * term,
    WORD * countfun ) [extern]
```

Definition at line 706 of file [reshuf.c](#).

4.19.4.36 DimensionSubterm()

```
WORD DimensionSubterm (
    WORD * subterm ) [extern]
```

Definition at line 867 of file [reshuf.c](#).

4.19.4.37 DimensionTerm()

```
WORD DimensionTerm (
    WORD * term ) [extern]
```

Definition at line 968 of file [reshuf.c](#).

4.19.4.38 DimensionExpression()

```
WORD DimensionExpression (
    PHEAD WORD * expr ) [extern]
```

Definition at line 1000 of file [reshuf.c](#).

4.19.4.39 Deferred()

```
WORD Deferred (
    PHEAD WORD * term,
    WORD level ) [extern]
```

Picks up the deferred brackets. These are the bracket contents of which we postpone the reading when we use the 'Keep Brackets' statement. These contents are multiplying the terms just before they are sent to the sorting system. Special attention goes to having it thread-safe We have to lock positioning the file and reading it in a thread specific buffer.

Parameters

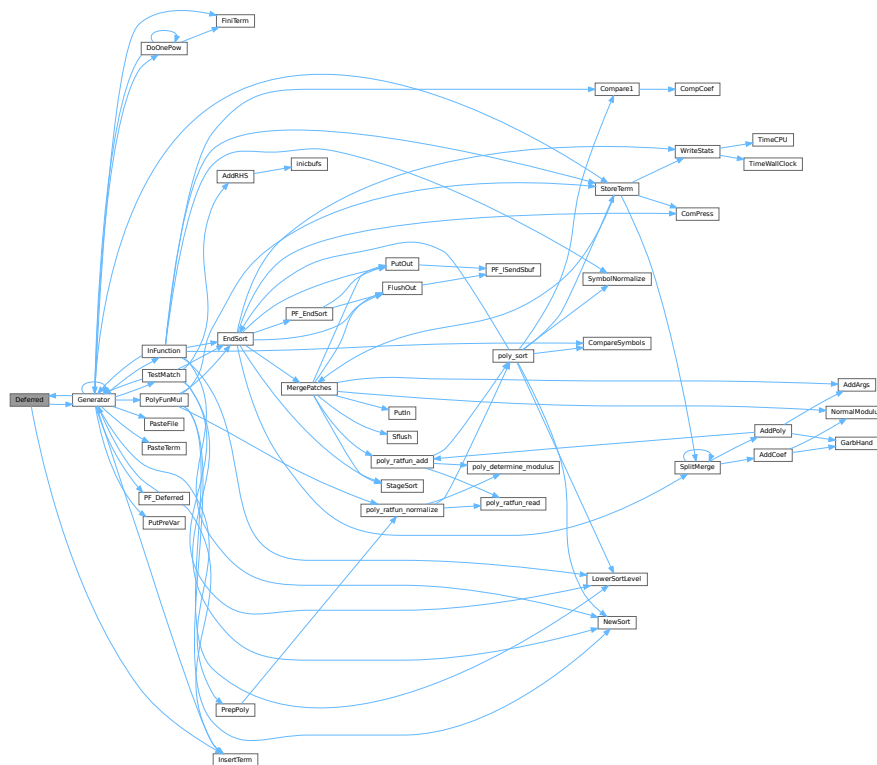
<i>term</i>	The term that must be multiplied by the contents of the current bracket
<i>level</i>	The compiler level. This is needed because after multiplying term by term we call Generator again.

Definition at line 4844 of file [proces.c](#).

References [Generator\(\)](#), and [InsertTerm\(\)](#).

Referenced by [Generator\(\)](#).

Here is the call graph for this function:



4.19.4.40 DeleteStore()

```
WORD DeleteStore (
    WORD par ) [extern]
```

Definition at line 773 of file [store.c](#).

4.19.4.41 DetCurDum()

```
WORD DetCurDum (
    PHEAD WORD * t ) [extern]
```

Definition at line 252 of file [reshuf.c](#).

4.19.4.42 DetVars()

```
void DetVars (
    WORD * term,
    WORD par ) [extern]
```

Definition at line 1830 of file [store.c](#).

4.19.4.43 Distribute()

```
WORD Distribute (
    DISTRIBUTE * d,
    WORD first ) [extern]
```

Definition at line 509 of file [symmetr.c](#).

4.19.4.44 DivLong()

```
WORD DivLong (
    UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc,
    UWORD * d,
    WORD * nd ) [extern]
```

Definition at line 972 of file [reken.c](#).

4.19.4.45 DivRat()

```
WORD DivRat (
    PHEAD UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc ) [extern]
```

Definition at line 500 of file [reken.c](#).

4.19.4.46 Divvy()

```
WORD Divvy (
    PHEAD UWORD * a,
    WORD * na,
    UWORD * b,
    WORD nb ) [extern]
```

Definition at line 172 of file [reken.c](#).

4.19.4.47 DoDelta()

```
WORD DoDelta (
    WORD * t ) [extern]
```

Definition at line [4355](#) of file [normal.c](#).

4.19.4.48 DoDelta3()

```
WORD DoDelta3 (
    PHEAD WORD * term,
    WORD level ) [extern]
```

Definition at line [1405](#) of file [reshuf.c](#).

4.19.4.49 TestPartitions()

```
WORD TestPartitions (
    WORD * tfun,
    PARTI * parti ) [extern]
```

Definition at line [1607](#) of file [reshuf.c](#).

4.19.4.50 DoPartitions()

```
WORD DoPartitions (
    PHEAD WORD * term,
    WORD level ) [extern]
```

Definition at line [1722](#) of file [reshuf.c](#).

4.19.4.51 CoCanonicalize()

```
int CoCanonicalize (
    UBYTE * s ) [extern]
```

Definition at line [64](#) of file [diagrams.c](#).

4.19.4.52 DoCanonicalize()

```
int DoCanonicalize (
    PHEAD WORD * term,
    WORD * params ) [extern]
```

Definition at line [185](#) of file [diagrams.c](#).

4.19.4.53 GenTopologies()

```
WORD GenTopologies (
    PHEAD WORD * term,
    WORD level ) [extern]
```

Definition at line 962 of file [diawrap.cc](#).

4.19.4.54 GenDiagrams()

```
WORD GenDiagrams (
    PHEAD WORD * term,
    WORD level ) [extern]
```

Definition at line 732 of file [diawrap.cc](#).

4.19.4.55 DoTopologyCanonicalize()

```
int DoTopologyCanonicalize (
    PHEAD WORD * term,
    WORD vert,
    WORD edge,
    WORD * args ) [extern]
```

Definition at line 417 of file [diagrams.c](#).

4.19.4.56 DoShattering()

```
int DoShattering (
    PHEAD WORD * connect,
    WORD * environ,
    WORD * partitions,
    WORD nvert ) [extern]
```

Definition at line 626 of file [diagrams.c](#).

4.19.4.57 GenerateTopologies()

```
WORD GenerateTopologies (
    PHEAD WORD,
    WORD nlegs,
    WORD setvert,
    WORD setmax ) [extern]
```

Definition at line 123 of file [topowrap.cc](#).

4.19.4.58 DoTableExpansion()

```
WORD DoTableExpansion (
    WORD * term,
    WORD level ) [extern]
```

Definition at line [334](#) of file [tables.c](#).

4.19.4.59 DoDistrib()

```
WORD DoDistrib (
    PHEAD WORD * term,
    WORD level ) [extern]
```

Definition at line [1119](#) of file [reshuf.c](#).

4.19.4.60 DoShuffle()

```
WORD DoShuffle (
    WORD * term,
    WORD level,
    WORD fun,
    WORD option ) [extern]
```

Definition at line [2095](#) of file [reshuf.c](#).

4.19.4.61 DoPermutations()

```
WORD DoPermutations (
    PHEAD WORD * term,
    WORD level ) [extern]
```

Definition at line [2007](#) of file [reshuf.c](#).

4.19.4.62 Shuffle()

```
int Shuffle (
    WORD * from1,
    WORD * from2,
    WORD * to ) [extern]
```

Definition at line [2229](#) of file [reshuf.c](#).

4.19.4.63 FinishShuffle()

```
int FinishShuffle (
    WORD * fini ) [extern]
```

Definition at line [2421](#) of file [reshuf.c](#).

4.19.4.64 DoStuffle()

```
WORD DoStuffle (
    WORD * term,
    WORD level,
    WORD fun,
    WORD option ) [extern]
```

Definition at line [2487](#) of file [reshuf.c](#).

4.19.4.65 Stuffle()

```
int Stuffle (
    WORD * from1,
    WORD * from2,
    WORD * to ) [extern]
```

Definition at line [2702](#) of file [reshuf.c](#).

4.19.4.66 FinishStuffle()

```
int FinishStuffle (
    WORD * fini ) [extern]
```

Definition at line [2829](#) of file [reshuf.c](#).

4.19.4.67 StuffRootAdd()

```
WORD * StuffRootAdd (
    WORD * t1,
    WORD * t2,
    WORD * to ) [extern]
```

Definition at line [2876](#) of file [reshuf.c](#).

4.19.4.68 TestUse()

```
WORD TestUse (
    WORD * term,
    WORD level ) [extern]
```

Definition at line [1414](#) of file [tables.c](#).

4.19.4.69 FindTB()

```
DBASE * FindTB (
    UBYTE * name ) [extern]
```

Definition at line [734](#) of file [tables.c](#).

4.19.4.70 CheckTableDeclarations()

```
int CheckTableDeclarations (
    DBASE * d ) [extern]
```

Definition at line 1178 of file [tables.c](#).

4.19.4.71 Apply()

```
WORD Apply (
    WORD * term,
    WORD level ) [extern]
```

Definition at line 1980 of file [tables.c](#).

4.19.4.72 ApplyExec()

```
int ApplyExec (
    WORD * term,
    int maxtogo,
    WORD level ) [extern]
```

Definition at line 2039 of file [tables.c](#).

4.19.4.73 ApplyReset()

```
WORD ApplyReset (
    WORD level ) [extern]
```

Definition at line 2256 of file [tables.c](#).

4.19.4.74 TableReset()

```
WORD TableReset (
    void ) [extern]
```

Definition at line 2292 of file [tables.c](#).

4.19.4.75 ReWorkT()

```
void ReWorkT (
    WORD * term,
    WORD * funs,
    WORD numfuns ) [extern]
```

Definition at line 1899 of file [tables.c](#).

4.19.4.76 GetIfDollarNum()

```
WORD GetIfDollarNum (
    WORD * ifp,
    WORD * ifstop ) [extern]
```

Definition at line 106 of file [if.c](#).

4.19.4.77 FindVar()

```
int FindVar (
    WORD * v,
    WORD * term ) [extern]
```

Definition at line 173 of file [if.c](#).

4.19.4.78 DolfStatement()

```
WORD DoIfStatement (
    PHEAD WORD * ifcode,
    WORD * term ) [extern]
```

Definition at line 280 of file [if.c](#).

4.19.4.79 DoOnePow()

```
WORD DoOnePow (
    PHEAD WORD * term,
    WORD power,
    WORD nexp,
    WORD * accum,
    WORD * aa,
    WORD level,
    WORD * freeze ) [extern]
```

Routine gets one power of an expression in the scratch system. If there are more powers needed there will be a recursion.

No attempt is made to use binomials because we have no information about commuting properties.

There is a searching for the contents of brackets if needed. This searching may be rather slow because of the single links.

Parameters

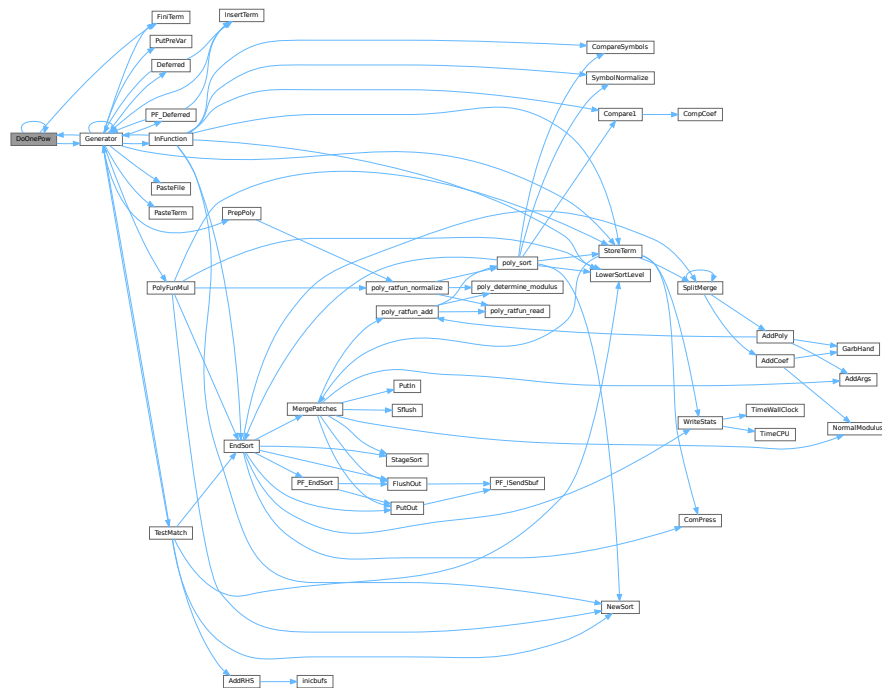
<i>term</i>	is the term we are adding to.
<i>power</i>	is the power of the expression that we need.
<i>nexp</i>	is the number of the expression.
<i>accum</i>	is the accumulator of terms. It accepts the termfragments that are made into a proper term in FiniTerm
<i>aa</i>	points to the start of the entire accumulator. In *aa we store the number of term fragments that are in the accumulator.
<i>level</i>	is the current depth in the tree of statements. It is needed to continue to the next
<i>freeze</i>	operation/substitution with each generated term
Generated by Doxygen	is the pointer to the bracket information that should be matched.

Definition at line 4623 of file [proces.c](#).

References [DoOnePow\(\)](#), [FiniTerm\(\)](#), [Generator\(\)](#), and [FiLe::handle](#).

Referenced by [DoOnePow\(\)](#), and [Generator\(\)](#).

Here is the call graph for this function:



4.19.4.80 DoRevert()

```

void DoRevert (
    WORD * fun,
    WORD * tmp ) [extern]
  
```

Definition at line 4422 of file [normal.c](#).

4.19.4.81 DoSumF1()

```

WORD DoSumF1 (
    PHEAD WORD * term,
    WORD * params,
    WORD * replac,
    WORD * level ) [extern]
  
```

Definition at line 395 of file [ratio.c](#).

4.19.4.82 DoSumF2()

```
WORD DoSumF2 (
    PHEAD WORD * term,
    WORD * params,
    WORD replac,
    WORD level ) [extern]
```

Definition at line 520 of file [ratio.c](#).

4.19.4.83 DoTheta()

```
WORD DoTheta (
    PHEAD WORD * t ) [extern]
```

Definition at line 4258 of file [normal.c](#).

4.19.4.84 EndSort()

```
LONG EndSort (
    PHEAD WORD * buffer,
    int par ) [extern]
```

Finishes a sort. At AR.sLevel == 0 the output is to the regular output stream. When AR.sLevel > 0, the parameter par determines the actual output. The AR.sLevel will be popped. All ongoing stages are finished and if the sortfile is open it is closed. The statistics are printed when AR.sLevel == 0 par == 0 [Output](#) to the buffer. par == 1 Sort for function arguments. The output will be copied into the buffer. It is assumed that this is in the Workspace. par == 2 Sort for \$-variable. We return the address of the buffer that contains the output in buffer (treated like WORD **). We first catch the output in a file (unless we can intercept things after the small buffer has been sorted) Then we read from the file into a buffer. Only when par == 0 data compression can be attempted at AT.SS==AT.S0.

Parameters

<i>buffer</i>	buffer for output when needed
<i>par</i>	See above

Returns

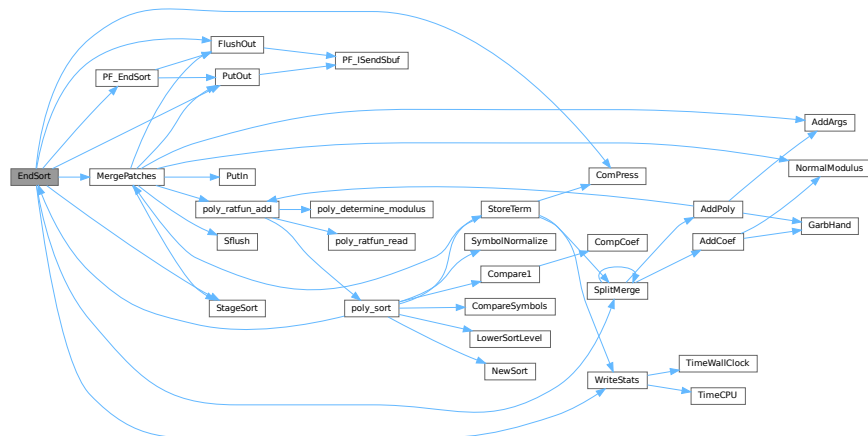
If negative: error. If positive: number of words in output.

Definition at line 733 of file [sort.c](#).

References [ComPress\(\)](#), [FlushOut\(\)](#), [MergePatches\(\)](#), [PF_EndSort\(\)](#), [PutOut\(\)](#), [SplitMerge\(\)](#), [StageSort\(\)](#), and [WriteStats\(\)](#).

Referenced by [generate_expression\(\)](#), [get_expression\(\)](#), [InFunction\(\)](#), [MakeDollarInteger\(\)](#), [MakeDollarMod\(\)](#), [PF_CollectModifiedDollars\(\)](#), [PF_Processor\(\)](#), [poly_factorize_expression\(\)](#), [poly_sort\(\)](#), [poly_unfactorize_expression\(\)](#), [PolyFunMul\(\)](#), [Processor\(\)](#), [TakeArgContent\(\)](#), and [TestMatch\(\)](#).

Here is the call graph for this function:



4.19.4.85 EntVar()

```

WORD EntVar (
    WORD type,
    UBYTE * name,
    WORD x,
    WORD y,
    WORD z,
    WORD d ) [extern]
  
```

Definition at line 557 of file [names.c](#).

4.19.4.86 EpfCon()

```

WORD EpfCon (
    PHEAD WORD * term,
    WORD * params,
    WORD num,
    WORD level ) [extern]
  
```

Definition at line 212 of file [opera.c](#).

4.19.4.87 EpfFind()

```

WORD EpfFind (
    PHEAD WORD * term,
    WORD * params ) [extern]
  
```

Definition at line 58 of file [opera.c](#).

4.19.4.88 EpfGen()

```
WORD EpfGen (
    WORD number,
    WORD * inlist,
    WORD * kron,
    WORD * perm,
    WORD sgn ) [extern]
```

Definition at line [282](#) of file [opera.c](#).

4.19.4.89 EqualArg()

```
WORD EqualArg (
    WORD * parms,
    WORD num1,
    WORD num2 ) [extern]
```

Definition at line [1379](#) of file [reshuf.c](#).

4.19.4.90 Factorial()

```
int Factorial (
    PHEAD WORD,
    UWORD * a,
    WORD * na ) [extern]
```

Definition at line [3449](#) of file [reken.c](#).

4.19.4.91 Bernoulli()

```
int Bernoulli (
    WORD n,
    UWORD * a,
    WORD * na ) [extern]
```

Definition at line [3529](#) of file [reken.c](#).

4.19.4.92 FactorIn()

```
int FactorIn (
    PHEAD WORD * term,
    WORD level ) [extern]
```

Definition at line [46](#) of file [factor.c](#).

4.19.4.93 FactorInExpr()

```
int FactorInExpr (
    PHEAD WORD * term,
    WORD level ) [extern]
```

Definition at line [421](#) of file [factor.c](#).

4.19.4.94 FindAll()

```
WORD FindAll (
    PHEAD WORD * term,
    WORD * pattern,
    WORD level,
    WORD * par ) [extern]
```

Definition at line [1197](#) of file [pattern.c](#).

4.19.4.95 FindMulti()

```
WORD FindMulti (
    PHEAD WORD * term,
    WORD * pattern ) [extern]
```

Definition at line [954](#) of file [findpat.c](#).

4.19.4.96 FindOnce()

```
WORD FindOnce (
    PHEAD WORD * term,
    WORD * pattern ) [extern]
```

Definition at line [419](#) of file [findpat.c](#).

4.19.4.97 FindOnly()

```
WORD FindOnly (
    PHEAD WORD * term,
    WORD * pattern ) [extern]
```

Definition at line [61](#) of file [findpat.c](#).

4.19.4.98 FindRest()

```
WORD FindRest (
    PHEAD WORD * term,
    WORD * pattern ) [extern]
```

Definition at line [1070](#) of file [findpat.c](#).

4.19.4.99 FindrNumber()

```
WORD FindrNumber (
    WORD n,
    VARENUM * v ) [extern]
```

Definition at line 2588 of file [store.c](#).

4.19.4.100 FiniLine()

```
void FiniLine (
    void ) [extern]
```

Definition at line 182 of file [sch.c](#).

4.19.4.101 FiniTerm()

```
WORD FiniTerm (
    PHEAD WORD * term,
    WORD * accum,
    WORD * termout,
    WORD number,
    WORD tepos ) [extern]
```

Concatenates the contents of the accumulator into a single legal term, which replaces the subexpression pointer

Parameters

<i>term</i>	the input term with the (sub)expression subterm
<i>accum</i>	the accumulator with the term fragments
<i>termout</i>	the location where the output should be written
<i>number</i>	the number of term fragments in the accumulator
<i>tepos</i>	the position of the subterm in term to be replaced

Definition at line 3067 of file [proces.c](#).

Referenced by [DoOnePow\(\)](#), and [Generator\(\)](#).

4.19.4.102 FlushOut()

```
WORD FlushOut (
    POSITION * position,
    FILEHANDLE * fi,
    int compr ) [extern]
```

Completes output to an output file and writes the trailing zero.

Parameters

<i>position</i>	The position in the file after writing
<i>fi</i>	The file (or its cache)
<i>compr</i>	Indicates whether there should be compression with gzip.

Returns

Regular conventions (OK -> 0).

Definition at line 1828 of file [sort.c](#).

References [FiLe::handle](#), [BrAcKeTiNfO::indexbuffer](#), and [PF_ISendSbuf\(\)](#).

Referenced by [EndSort\(\)](#), [MergePatches\(\)](#), [PF_EndSort\(\)](#), and [Processor\(\)](#).

Here is the call graph for this function:

**4.19.4.103 FunLevel()**

```
void FunLevel (
    PHEAD WORD * term ) [extern]
```

Definition at line 130 of file [reshuf.c](#).

4.19.4.104 AdjustRenumScratch()

```
void AdjustRenumScratch (
    PHEAD0 ) [extern]
```

Definition at line 506 of file [reshuf.c](#).

4.19.4.105 GarbHand()

```
void GarbHand (
    void ) [extern]
```

Garbage collection that takes place when the small extension is full and we need to place more terms there. When this is the case there are many holes in the small buffer and the whole can be compactified. The major complication is the buffer for SplitMerge. There are two options for temporary memory: 1: find some buffer that has enough space (maybe in the large buffer). 2: allocate a buffer. Give it back afterwards of course. If the small extension is properly dimensioned this routine should be called very rarely. Most of the time it will be called when the polyfun or polyratfun is active.

Definition at line 3642 of file [sort.c](#).

Referenced by [AddCoef\(\)](#), and [AddPoly\(\)](#).

4.19.4.106 GcdLong()

```
WORD GcdLong (
    PHEAD UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc ) [extern]
```

Definition at line 2276 of file [reken.c](#).

4.19.4.107 LcmLong()

```
WORD LcmLong (
    PHEAD UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc ) [extern]
```

Definition at line 2701 of file [reken.c](#).

4.19.4.108 Generator()

```
WORD Generator (
    PHEAD WORD * term,
    WORD level ) [extern]
```

The heart of the program. Here the expansion tree is set up in one giant recursion

Parameters

<i>term</i>	the input term. may be overwritten
<i>level</i>	the level in the compiler buffer (number of statement)

Returns

Normal conventions (OK = 0).

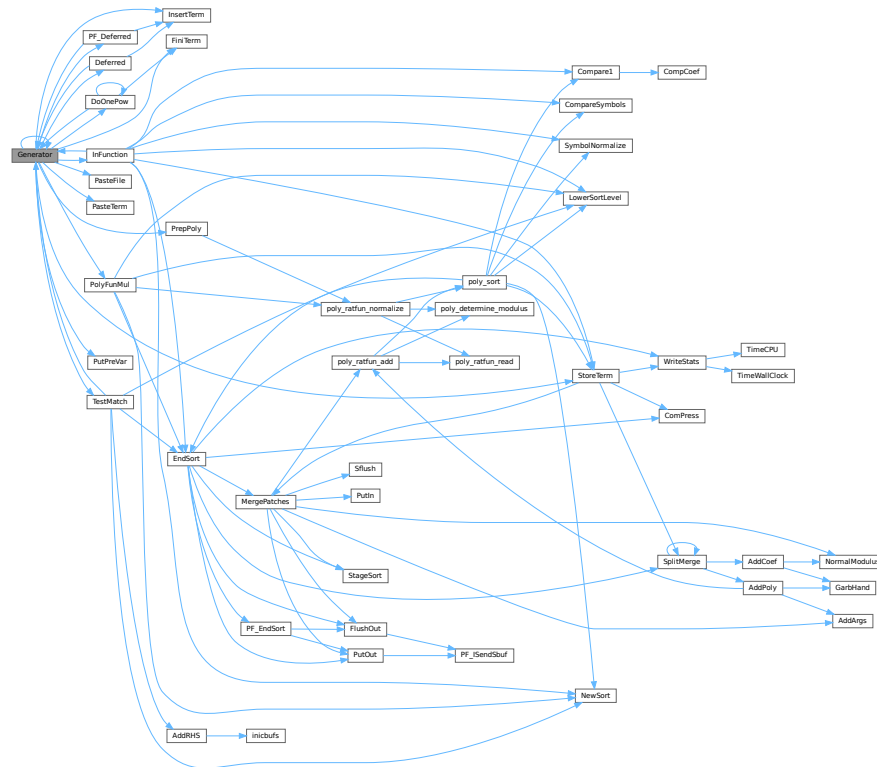
The routine looks first whether there are unsubstituted (sub)expressions. If so, one of them gets inserted term by term and the new term is used in a renewed call to Generator. If there are no (sub)expressions, the term is normalized, the compiler level is raised (next statement) and the program looks what type of statement this is. If this is a special statement it is either treated on the spot or the appropriate routine is called. If it is a substitution, the pattern matcher is called (TestMatch) which tells whether there was a match. If so we need to call TestSub again to test for (sub)expressions. If we run out of levels, the term receives a final treatment for modulus calculus and/or brackets and is then sent off to the sorting routines.

Definition at line 3266 of file [proces.c](#).

References [Deferred\(\)](#), [DoOnePow\(\)](#), [FiniTerm\(\)](#), [Generator\(\)](#), [InFunction\(\)](#), [InsertTerm\(\)](#), [CbUf::lhs](#), [VaRrEnUm::lo](#), [PasteFile\(\)](#), [PasteTerm\(\)](#), [PF_Deferred\(\)](#), [CbUf::Pointer](#), [PolyFunMul\(\)](#), [PrepPoly\(\)](#), [PutPreVar\(\)](#), [CbUf::rhs](#), [StoreTerm\(\)](#), [ReNuMbEr::symb](#), and [TestMatch\(\)](#).

Referenced by [Deferred\(\)](#), [DoOnePow\(\)](#), [generate_expression\(\)](#), [Generator\(\)](#), [InFunction\(\)](#), [MakeDollarInteger\(\)](#), [MakeDollarMod\(\)](#), [PF_CollectModifiedDollars\(\)](#), [PF_Deferred\(\)](#), [PF_Processor\(\)](#), [poly_factorize_expression\(\)](#), [poly_unfactorize_expression\(\)](#), [Processor\(\)](#), and [TestMatch\(\)](#).

Here is the call graph for this function:



4.19.4.109 GetBinom()

```
WORD GetBinom (
    UWORD * a,
    WORD * na,
    WORD i1,
    WORD i2 ) [extern]
```

Definition at line 2667 of file [reken.c](#).

4.19.4.110 GetFromStore()

```
WORD GetFromStore (
    WORD * to,
    POSITION * position,
    RENUMBER renumber,
    WORD * InCompState,
    WORD nexpr ) [extern]
```

Definition at line 1629 of file [store.c](#).

4.19.4.111 GetLong()

```
WORD GetLong (
    UBYTE * s,
    UWORD * a,
    WORD * na ) [extern]
```

Definition at line 1774 of file [reken.c](#).

4.19.4.112 GetMoreTerms()

```
WORD GetMoreTerms (
    WORD * term ) [extern]
```

Definition at line 1426 of file [store.c](#).

4.19.4.113 GetMoreFromMem()

```
WORD GetMoreFromMem (
    WORD * term,
    WORD ** tpoint ) [extern]
```

Definition at line 1522 of file [store.c](#).

4.19.4.114 GetOneTerm()

```
WORD GetOneTerm (
    PHEAD WORD * term,
    FILEHANDLE * fi,
    POSITION * pos,
    int par ) [extern]
```

Definition at line 1268 of file [store.c](#).

4.19.4.115 GetTable()

```
RENUMBER GetTable (
    WORD expr,
    POSITION * position,
    WORD mode ) [extern]
```

Definition at line 2826 of file [store.c](#).

4.19.4.116 GetTerm()

```
WORD GetTerm (
    PHEAD WORD * term ) [extern]
```

Definition at line 993 of file [store.c](#).

4.19.4.117 **Glue()**

```
WORD Glue (
    PHEAD WORD * term1,
    WORD * term2,
    WORD * sub,
    WORD insert ) [extern]
```

Definition at line [461](#) of file [ratio.c](#).

4.19.4.118 **InFunction()**

```
WORD InFunction (
    PHEAD WORD * term,
    WORD * termout ) [extern]
```

Makes the replacement of the subexpression with the number 'replac' in a function argument. Additional information is passed in some of the AR, AN, AT variables.

Parameters

<i>term</i>	The input term
<i>termout</i>	The output term

Returns

0: everything is fine, Negative: fatal, Positive: error.

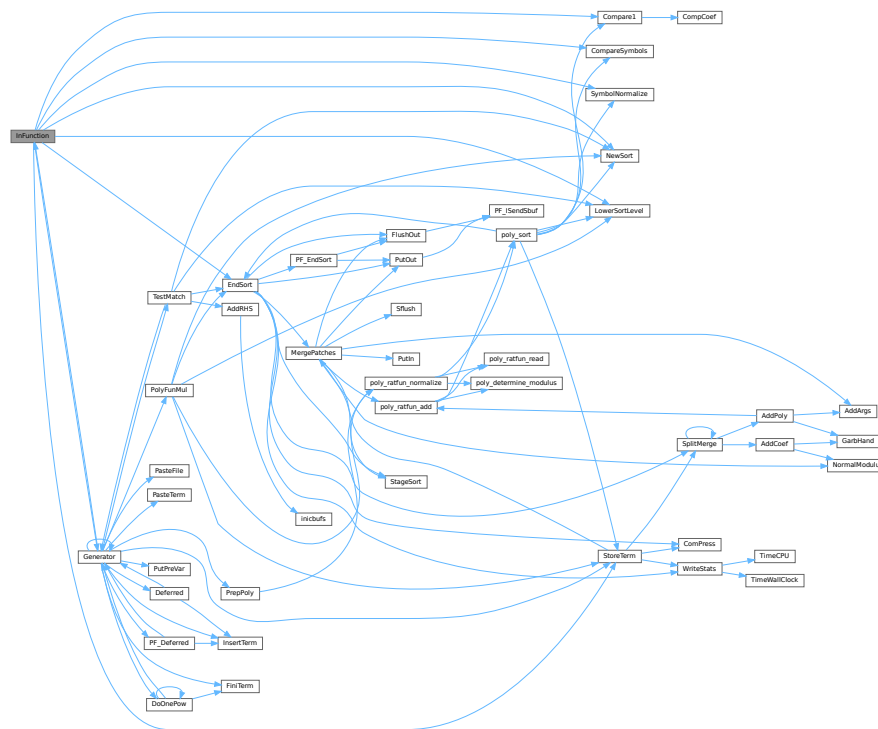
Special attention should be given to nested functions!

Definition at line [2174](#) of file [proces.c](#).

References [Compare1\(\)](#), [CompareSymbols\(\)](#), [EndSort\(\)](#), [Generator\(\)](#), [LowerSortLevel\(\)](#), [NewSort\(\)](#), [StoreTerm\(\)](#), and [SymbolNormalize\(\)](#).

Referenced by [Generator\(\)](#).

Here is the call graph for this function:



4.19.4.119 IniLine()

```
void IniLine (
    WORD extrablank ) [extern]
```

Definition at line 267 of file [sch.c](#).

4.19.4.120 IniVars()

```
WORD IniVars (
    void ) [extern]
```

Definition at line 1354 of file [startup.c](#).

4.19.4.121 InsertTerm()

```
WORD InsertTerm (
    PHEAD WORD * term,
    WORD replac,
    WORD extractbuff,
    WORD * position,
    WORD * termout,
    WORD tepos ) [extern]
```

Puts the terms 'term' and 'position' together into a single legal term in termout. replac is the number of the subexpression that should be replaced. It must be a positive term. When action is needed in the argument of a function all terms in that argument are dealt with recursively. The subexpression is sorted. Only one subexpression is done at a time this way.

Parameters

<i>term</i>	the input term
<i>replac</i>	number of the subexpression pointer to replace
<i>extractbuff</i>	number of the compiler buffer replac refers to
<i>position</i>	position from where to take the term in the compiler buffer
<i>termout</i>	the output term
<i>tepos</i>	offset in term where the subexpression is.

Returns

Normal conventions (OK = 0).

Definition at line [2744](#) of file [proces.c](#).

Referenced by [Deferred\(\)](#), [Generator\(\)](#), and [PF_Deferred\(\)](#).

4.19.4.122 LongToLine()

```
void LongToLine (
    UWORD * a,
    WORD na ) [extern]
```

Definition at line [303](#) of file [sch.c](#).

4.19.4.123 MakeDirty()

```
WORD MakeDirty (
    WORD * term,
    WORD * x,
    WORD par ) [extern]
```

Definition at line [51](#) of file [function.c](#).

4.19.4.124 MarkDirty()

```
void MarkDirty (
    WORD * term,
    WORD flags ) [extern]
```

Definition at line [97](#) of file [function.c](#).

4.19.4.125 PolyFunDirty()

```
void PolyFunDirty (
    PHEAD WORD * term ) [extern]
```

Definition at line [139](#) of file [function.c](#).

4.19.4.126 PolyFunClean()

```
void PolyFunClean (
    PHEAD WORD * term ) [extern]
```

Definition at line 174 of file [function.c](#).

4.19.4.127 MakeModTable()

```
WORD MakeModTable (
    void ) [extern]
```

Definition at line 3363 of file [reken.c](#).

4.19.4.128 MatchE()

```
WORD MatchE (
    PHEAD WORD * pattern,
    WORD * fun,
    WORD * inter,
    WORD par ) [extern]
```

Definition at line 56 of file [symmetr.c](#).

4.19.4.129 MatchCy()

```
int MatchCy (
    PHEAD WORD * pattern,
    WORD * fun,
    WORD * inter,
    WORD par ) [extern]
```

Definition at line 570 of file [symmetr.c](#).

4.19.4.130 FunMatchCy()

```
int FunMatchCy (
    PHEAD WORD * pattern,
    WORD * fun,
    WORD * inter,
    WORD par ) [extern]
```

Definition at line 1066 of file [symmetr.c](#).

4.19.4.131 FunMatchSy()

```
int FunMatchSy (
    PHEAD WORD * pattern,
    WORD * fun,
    WORD * inter,
    WORD par ) [extern]
```

Definition at line 1524 of file [symmetr.c](#).

4.19.4.132 MatchArgument()

```
int MatchArgument (
    PHEAD WORD * arg,
    WORD * pat ) [extern]
```

Definition at line [2051](#) of file [symmetr.c](#).

4.19.4.133 MatchFunction()

```
WORD MatchFunction (
    PHEAD WORD * pattern,
    WORD * interm,
    WORD * wilds ) [extern]
```

Definition at line [826](#) of file [function.c](#).

4.19.4.134 MergePatches()

```
WORD MergePatches (
    WORD par ) [extern]
```

The general merge routine. Can be used for the large buffer and the file merging. The array *S->Patches* tells where the patches start *S->pStop* tells where they end (has to be computed first). The end of a 'line to be merged' is indicated by a zero. If the end is reached without running into a zero or a term runs over the boundary of a patch it is a file merging operation and a new piece from the file is read in.

Parameters

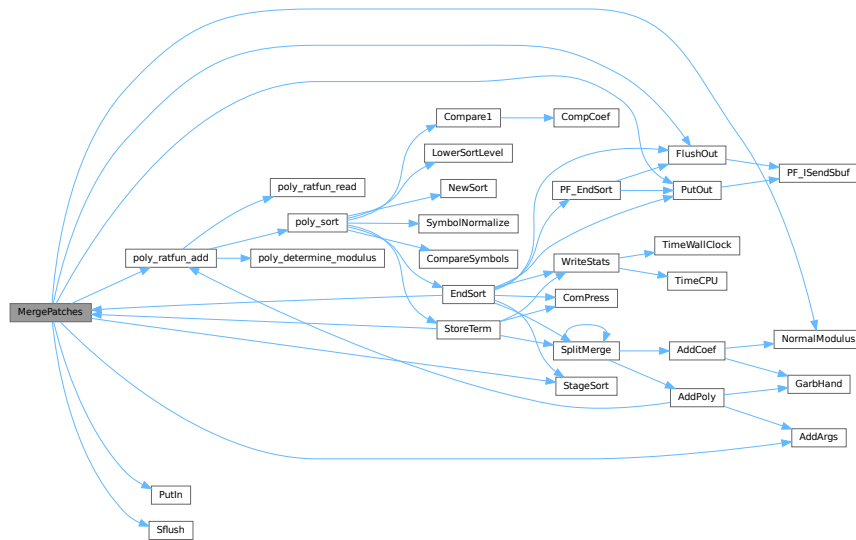
<i>par</i>	If <i>par</i> == 0 the sort is for file -> outputfile. If <i>par</i> == 1 the sort is for large buffer -> sortfile. If <i>par</i> == 2 the sort is for large buffer -> outputfile.
------------	--

Definition at line [3757](#) of file [sort.c](#).

References [AddArgs\(\)](#), [FlushOut\(\)](#), [FiLe::handle](#), [NormalModulus\(\)](#), [poly_ratfun_add\(\)](#), [PutIn\(\)](#), [PutOut\(\)](#), [Sflush\(\)](#), and [StageSort\(\)](#).

Referenced by [EndSort\(\)](#), and [StoreTerm\(\)](#).

Here is the call graph for this function:



4.19.4.135 MesCerr()

```
WORD MesCerr (
    char * s,
    UBYTE * t ) [extern]
```

Definition at line 824 of file [message.c](#).

4.19.4.136 MesComp()

```
WORD MesComp (
    char * s,
    UBYTE * p,
    UBYTE * q ) [extern]
```

Definition at line 843 of file [message.c](#).

4.19.4.137 Modulus()

```
WORD Modulus (
    WORD * term ) [extern]
```

Definition at line 3102 of file [reken.c](#).

4.19.4.138 MoveDummies()

```
void MoveDummies (
    PHEAD WORD * term,
    WORD shift ) [extern]
```

Definition at line 436 of file [reshuf.c](#).

4.19.4.139 MulLong()

```
WORD MulLong (
    UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc ) [extern]
```

Definition at line [841](#) of file [reken.c](#).

4.19.4.140 MulRat()

```
WORD MulRat (
    PHEAD UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc ) [extern]
```

Definition at line [378](#) of file [reken.c](#).

4.19.4.141 Mully()

```
WORD Mully (
    PHEAD UWORD * a,
    WORD * na,
    UWORD * b,
    WORD nb ) [extern]
```

Definition at line [126](#) of file [reken.c](#).

4.19.4.142 MultDo()

```
WORD MultDo (
    PHEAD WORD * term,
    WORD * pattern ) [extern]
```

Definition at line [1044](#) of file [reshuf.c](#).

4.19.4.143 NewSort()

```
WORD NewSort (
    PHEAD0 ) [extern]
```

Starts a new sort. At the lowest level this is a 'main sort' with the struct according to the parameters in S0. At higher levels this is a sort for functions, subroutines or dollars. We prepare the arrays and structs.

Returns

Regular convention (OK -> 0)

Definition at line [643](#) of file [sort.c](#).

Referenced by [generate_expression\(\)](#), [get_expression\(\)](#), [InFunction\(\)](#), [MakeDollarInteger\(\)](#), [MakeDollarMod\(\)](#), [PF_CollectModifiedDollars\(\)](#), [PF_Processor\(\)](#), [poly_factorize_expression\(\)](#), [poly_sort\(\)](#), [poly_unfactorize_expression\(\)](#), [PolyFunMul\(\)](#), [Processor\(\)](#), [TakeArgContent\(\)](#), and [TestMatch\(\)](#).

4.19.4.144 ExtraSymbol()

```
WORD ExtraSymbol (
    WORD sym,
    WORD pow,
    WORD nsym,
    WORD * ppsym,
    WORD * ncoef ) [extern]
```

Definition at line [4196](#) of file [normal.c](#).

4.19.4.145 Normalize()

```
WORD Normalize (
    PHEAD WORD * term ) [extern]
```

Definition at line [193](#) of file [normal.c](#).

4.19.4.146 BracketNormalize()

```
WORD BracketNormalize (
    PHEAD WORD * term ) [extern]
```

Definition at line [5271](#) of file [normal.c](#).

4.19.4.147 DropCoefficient()

```
void DropCoefficient (
    PHEAD WORD * term ) [extern]
```

Definition at line [5096](#) of file [normal.c](#).

4.19.4.148 DropSymbols()

```
void DropSymbols (
    PHEAD WORD * term ) [extern]
```

Definition at line [5114](#) of file [normal.c](#).

4.19.4.149 PutInside()

```
int PutInside (
    PHEAD WORD * term,
    WORD * code ) [extern]
```

Definition at line [609](#) of file [index.c](#).

4.19.4.150 OpenTemp()

```
WORD OpenTemp (
    void ) [extern]
```

Definition at line 48 of file [store.c](#).

4.19.4.151 Pack()

```
void Pack (
    UWORD * a,
    WORD * na,
    UWORD * b,
    WORD nb ) [extern]
```

Definition at line 62 of file [reken.c](#).

4.19.4.152 PasteFile()

```
LONG PasteFile (
    PHEAD WORD number,
    WORD * accum,
    POSITION * position,
    WORD ** accfill,
    RENUMBER renumber,
    WORD * freeze,
    WORD nexpr ) [extern]
```

Gets a term from stored expression *expr* and puts it in the accumulator at position *number*. It returns the length of the term that came from file.

Parameters

<i>number</i>	number of partial terms to skip in accum
<i>accum</i>	the accumulator
<i>position</i>	file position from where to get the stored term
<i>accfill</i>	returns tail position in accum
<i>renumber</i>	the renumber struct for the variables in the stored expression
<i>freeze</i>	information about if we need only the contents of a bracket
<i>nexpr</i>	the number of the stored expression

Returns

Normal conventions (OK = 0).

Definition at line 2880 of file [proces.c](#).

Referenced by [Generator\(\)](#).

4.19.4.153 Permute()

```
WORD Permute (
    PERM * perm,
    WORD first ) [extern]
```

Definition at line 443 of file [symmetr.c](#).

4.19.4.154 PermuteP()

```
WORD PermuteP (
    PERMP * perm,
    WORD first ) [extern]
```

Definition at line 477 of file [symmetr.c](#).

4.19.4.155 PolyFunMul()

```
WORD PolyFunMul (
    PHEAD WORD * term ) [extern]
```

Multiplies the arguments of multiple occurrences of the polyfun. In this routine we do the original PolyFun with one argument only. The PolyRatFun (PolyFunType = 2) is done in a dedicated routine in the file [polywrap.cc](#) The new result is written over the old result.

Parameters

<i>term</i>	It contains the input term and later the output.
-------------	--

Returns

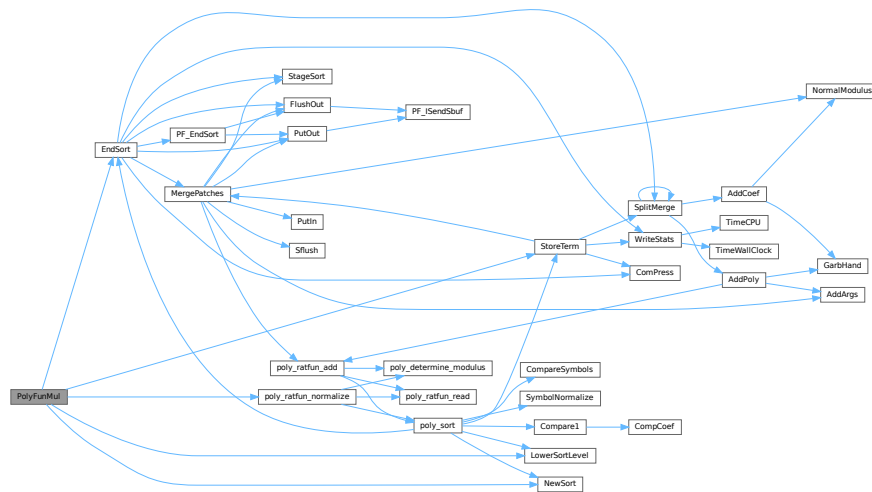
Normal conventions (OK = 0).

Definition at line 5360 of file [proces.c](#).

References [EndSort\(\)](#), [LowerSortLevel\(\)](#), [NewSort\(\)](#), [poly_ratfun_normalize\(\)](#), and [StoreTerm\(\)](#).

Referenced by [Generator\(\)](#).

Here is the call graph for this function:



4.19.4.156 PopVariables()

```
WORD PopVariables (
    void ) [extern]
```

Definition at line 211 of file [execute.c](#).

4.19.4.157 PrepPoly()

```
WORD PrepPoly (
    PHEAD WORD * term,
    WORD par ) [extern]
```

Routine checks whether the count of function AR.PolyFun is zero or one. If it is one and it has one scalarlike argument the coefficient of the term is pulled inside the argument. If the count is zero a new function is made with the coefficient as its only argument. The function should be placed at its proper position.

When this function is active it places the PolyFun as last object before the coefficient. This is needed because otherwise the compress algorithm has problems in MergePatches.

The bracket routine should also place the PolyFun at a comparable spot. The compression should then stop at the PolyFun. It doesn't really have to stop when writing the final result but this may be too complicated.

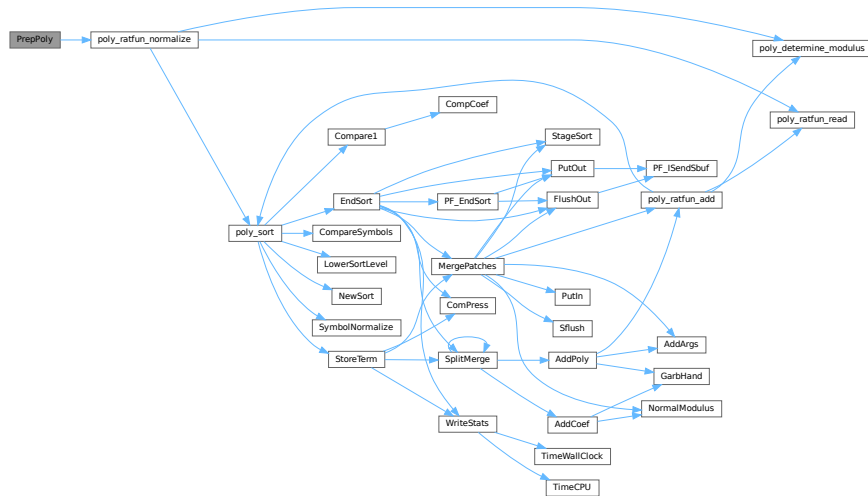
The parameter par tells whether we are at groundlevel or inside a function or dollar variable.

Definition at line 4972 of file [proces.c](#).

References [poly_ratfun_normalize\(\)](#).

Referenced by [Generator\(\)](#).

Here is the call graph for this function:



4.19.4.158 Processor()

```
WORD Processor (
    void ) [extern]
```

This is the central processor. It accepts a stream of Expressions which is accessed by calls to GetTerm. The expressions reside either in AR.infile or AR.hidefile. The definitions of an expression are seen as an id-statement, so the primary Expressions should be written to the system of scratch files as single terms with an expression pointer. Each expression is terminated with a zero and the whole is terminated by two zeroes.

The routine DoExecute should determine whether results are to be printed, should revert the scratch I/O directions etc. In principle it is DoExecute that calls Processor.

Returns

if everything OK: 0. Otherwise error. The preprocessor may continue with compilation though. Really fatal errors should return on the spot by calling Terminate.

Definition at line 64 of file [proces.c](#).

References [CbUf::Buffer](#), [EndSort\(\)](#), [FlushOut\(\)](#), [Generator\(\)](#), [CbUf::lhs](#), [LowerSortLevel\(\)](#), [NewSort\(\)](#), [PF_BroadcastRHS\(\)](#), [PF_InParallelProcessor\(\)](#), [PF_Processor\(\)](#), [CbUf::Pointer](#), [poly_factorize_expression\(\)](#), [poly_unfactorize_expression\(\)](#), [PutOut\(\)](#), [CbUf::rhs](#), and [StoreTerm\(\)](#).

The graph illustrates the dependencies between various modules in the 'poly' project. The nodes are represented as rectangles with labels. Some nodes have red borders, indicating they are part of the 'poly' project. The nodes are organized in a roughly hierarchical manner, with 'Process' at the top left and 'poly_determine_module' at the bottom right. The graph shows a high degree of interconnectivity, with many nodes having multiple incoming and outgoing dependencies.

Key nodes and their dependencies include:

- Process** (red border) depends on: *PF_BroadcastRHS*, *PF_BroadcastExpr*, *PF_LongingInterface*, *PF_LongingSend*, *PF_LongingUnpack*, *PF_PrepareLongingPush*, *PF_Processor*, *PF_LongingPush*, *PF_BroadcastInterfacePushers*, *PutPVar*, *PF_Deferred*, *Deferred*, *InsertTerm*, *FirstTerm*, *OnOnePVar*, *PastaTerm*, *PropPoly*, *EndSort*, *StageSort*, *SortMerge*, *Compress*, *SlowTerm*, *NewSort*, *LowerSortLevel*, *LocalConvertToPoly*, *poly_factorize_expression*, *poly_factorize_expression*, *poly_unfactorize_expression*, *PF_InfiniteProcessor*, *WriteStats*, *TimeCPU*, *TimeWallClock*, *PutOut*, *FlushOut*, *PF_SendBuf*, *PutIn*, *Silence*, *NormalModule*, *AddKeys*, *ASMCat*, *ASMPoly*, *poly_rattr_add*, *poly_determine_module*.
- PF_Processor** (red border) depends on: *PF_LongingInterface*, *PF_LongingSend*, *PF_LongingUnpack*, *PF_PrepareLongingPush*, *PF_LongingPush*, *PF_BroadcastInterfacePushers*, *PutPVar*, *PF_Deferred*, *Deferred*, *InsertTerm*, *FirstTerm*, *OnOnePVar*, *PastaTerm*, *PropPoly*, *EndSort*, *StageSort*, *SortMerge*, *Compress*, *SlowTerm*, *NewSort*, *LowerSortLevel*, *LocalConvertToPoly*, *poly_factorize_expression*, *poly_unfactorize_expression*, *PF_InfiniteProcessor*, *WriteStats*, *TimeCPU*, *TimeWallClock*, *PutOut*, *FlushOut*, *PF_SendBuf*, *PutIn*, *Silence*, *NormalModule*, *AddKeys*, *ASMCat*, *ASMPoly*, *poly_rattr_add*, *poly_determine_module*.
- EndSort** (red border) depends on: *PF_Deferred*, *Deferred*, *InsertTerm*, *FirstTerm*, *OnOnePVar*, *PastaTerm*, *PropPoly*, *StageSort*, *SortMerge*, *Compress*, *SlowTerm*, *NewSort*, *LowerSortLevel*, *LocalConvertToPoly*, *poly_factorize_expression*, *poly_unfactorize_expression*, *PF_InfiniteProcessor*, *WriteStats*, *TimeCPU*, *TimeWallClock*, *PutOut*, *FlushOut*, *PF_SendBuf*, *PutIn*, *Silence*, *NormalModule*, *AddKeys*, *ASMCat*, *ASMPoly*, *poly_rattr_add*, *poly_determine_module*.
- StageSort** (red border) depends on: *EndSort*, *SortMerge*, *Compress*, *SlowTerm*, *NewSort*, *LowerSortLevel*, *LocalConvertToPoly*, *poly_factorize_expression*, *poly_unfactorize_expression*, *PF_InfiniteProcessor*, *WriteStats*, *TimeCPU*, *TimeWallClock*, *PutOut*, *FlushOut*, *PF_SendBuf*, *PutIn*, *Silence*, *NormalModule*, *AddKeys*, *ASMCat*, *ASMPoly*, *poly_rattr_add*, *poly_determine_module*.
- SortMerge** (red border) depends on: *EndSort*, *StageSort*, *Compress*, *SlowTerm*, *NewSort*, *LowerSortLevel*, *LocalConvertToPoly*, *poly_factorize_expression*, *poly_unfactorize_expression*, *PF_InfiniteProcessor*, *WriteStats*, *TimeCPU*, *TimeWallClock*, *PutOut*, *FlushOut*, *PF_SendBuf*, *PutIn*, *Silence*, *NormalModule*, *AddKeys*, *ASMCat*, *ASMPoly*, *poly_rattr_add*, *poly_determine_module*.
- Compress** (red border) depends on: *EndSort*, *StageSort*, *SortMerge*, *SlowTerm*, *NewSort*, *LowerSortLevel*, *LocalConvertToPoly*, *poly_factorize_expression*, *poly_unfactorize_expression*, *PF_InfiniteProcessor*, *WriteStats*, *TimeCPU*, *TimeWallClock*, *PutOut*, *FlushOut*, *PF_SendBuf*, *PutIn*, *Silence*, *NormalModule*, *AddKeys*, *ASMCat*, *ASMPoly*, *poly_rattr_add*, *poly_determine_module*.
- SlowTerm** (red border) depends on: *EndSort*, *StageSort*, *SortMerge*, *Compress*, *NewSort*, *LowerSortLevel*, *LocalConvertToPoly*, *poly_factorize_expression*, *poly_unfactorize_expression*, *PF_InfiniteProcessor*, *WriteStats*, *TimeCPU*, *TimeWallClock*, *PutOut*, *FlushOut*, *PF_SendBuf*, *PutIn*, *Silence*, *NormalModule*, *AddKeys*, *ASMCat*, *ASMPoly*, *poly_rattr_add*, *poly_determine_module*.
- NewSort** (red border) depends on: *EndSort*, *StageSort*, *SortMerge*, *Compress*, *SlowTerm*, *LowerSortLevel*, *LocalConvertToPoly*, *poly_factorize_expression*, *poly_unfactorize_expression*, *PF_InfiniteProcessor*, *WriteStats*, *TimeCPU*, *TimeWallClock*, *PutOut*, *FlushOut*, *PF_SendBuf*, *PutIn*, *Silence*, *NormalModule*, *AddKeys*, *ASMCat*, *ASMPoly*, *poly_rattr_add*, *poly_determine_module*.
- LowerSortLevel** (red border) depends on: *EndSort*, *StageSort*, *SortMerge*, *Compress*, *SlowTerm*, *NewSort*, *LocalConvertToPoly*, *poly_factorize_expression*, *poly_unfactorize_expression*, *PF_InfiniteProcessor*, *WriteStats*, *TimeCPU*, *TimeWallClock*, *PutOut*, *FlushOut*, *PF_SendBuf*, *PutIn*, *Silence*, *NormalModule*, *AddKeys*, *ASMCat*, *ASMPoly*, *poly_rattr_add*, *poly_determine_module*.
- LocalConvertToPoly** (red border) depends on: *EndSort*, *StageSort*, *SortMerge*, *Compress*, *SlowTerm*, *NewSort*, *LowerSortLevel*, *poly_factorize_expression*, *poly_unfactorize_expression*, *PF_InfiniteProcessor*, *WriteStats*, *TimeCPU*, *TimeWallClock*, *PutOut*, *FlushOut*, *PF_SendBuf*, *PutIn*, *Silence*, *NormalModule*, *AddKeys*, *ASMCat*, *ASMPoly*, *poly_rattr_add*, *poly_determine_module*.
- poly_factorize_expression** (red border) depends on: *EndSort*, *StageSort*, *SortMerge*, *Compress*, *SlowTerm*, *NewSort*, *LowerSortLevel*, *LocalConvertToPoly*, *poly_unfactorize_expression*, *PF_InfiniteProcessor*, *WriteStats*, *TimeCPU*, *TimeWallClock*, *PutOut*, *FlushOut*, *PF_SendBuf*, *PutIn*, *Silence*, *NormalModule*, *AddKeys*, *ASMCat*, *ASMPoly*, *poly_rattr_add*, *poly_determine_module*.
- poly_unfactorize_expression** (red border) depends on: *EndSort*, *StageSort*, *SortMerge*, *Compress*, *SlowTerm*, *NewSort*, *LowerSortLevel*, *LocalConvertToPoly*, *poly_factorize_expression*, *PF_InfiniteProcessor*, *WriteStats*, *TimeCPU*, *TimeWallClock*, *PutOut*, *FlushOut*, *PF_SendBuf*, *PutIn*, *Silence*, *NormalModule*, *AddKeys*, *ASMCat*, *ASMPoly*, *poly_rattr_add*, *poly_determine_module*.
- PF_InfiniteProcessor** (red border) depends on: *EndSort*, *StageSort*, *SortMerge*, *Compress*, *SlowTerm*, *NewSort*, *LowerSortLevel*, *LocalConvertToPoly*, *poly_factorize_expression*, *poly_unfactorize_expression*, *WriteStats*, *TimeCPU*, *TimeWallClock*, *PutOut*, *FlushOut*, *PF_SendBuf*, *PutIn*, *Silence*, *NormalModule*, *AddKeys*, *ASMCat*, *ASMPoly*, *poly_rattr_add*, *poly_determine_module*.
- WriteStats** (red border) depends on: *EndSort*, *StageSort*, *SortMerge*, *Compress*, *SlowTerm*, *NewSort*, *LowerSortLevel*, *LocalConvertToPoly*, *poly_factorize_expression*, *poly_unfactorize_expression*, *PF_InfiniteProcessor*, *TimeCPU*, *TimeWallClock*, *PutOut*, *FlushOut*, *PF_SendBuf*, *PutIn*, *Silence*, *NormalModule*, *AddKeys*, *ASMCat*, *ASMPoly*, *poly_rattr_add*, *poly_determine_module*.
- TimeCPU** (red border) depends on: *EndSort*, *StageSort*, *SortMerge*, *Compress*, *SlowTerm*, *NewSort*, *LowerSortLevel*, *LocalConvertToPoly*, *poly_factorize_expression*, *poly_unfactorize_expression*, *PF_InfiniteProcessor*, *WriteStats*, *TimeWallClock*, *PutOut*, *FlushOut*, *PF_SendBuf*, *PutIn*, *Silence*, *NormalModule*, *AddKeys*, *ASMCat*, *ASMPoly*, *poly_rattr_add*, *poly_determine_module*.
- TimeWallClock** (red border) depends on: *EndSort*, *StageSort*, *SortMerge*, *Compress*, *SlowTerm*, *NewSort*, *LowerSortLevel*, *LocalConvertToPoly*, *poly_factor*

```
WORD Product (
    UWORD * a,
    WORD * na,
    WORD b ) [extern]
```

4.19.4.160 PrtLong()

Definition at line 1712 of file reken.c.

4.19.4.161 PrtTerms()

```
void PrtTerms (
    void ) [extern]
```

Definition at line 716 of file [sch.c](#).

4.19.4.162 PrintDeprecation()

```
void PrintDeprecation (
    const char * feature,
    const char * issue ) [extern]
```

Prints a deprecation warning for a given feature.

Parameters

<i>feature</i>	The name of the deprecated feature.
<i>issue</i>	The associated issue, e.g., "issues/700".

Definition at line 2063 of file [startup.c](#).

4.19.4.163 PrintRunningTime()

```
void PrintRunningTime (
    void ) [extern]
```

Definition at line 2087 of file [startup.c](#).

4.19.4.164 GetRunningTime()

```
LONG GetRunningTime (
    void ) [extern]
```

Definition at line 2128 of file [startup.c](#).

4.19.4.165 PutBracket()

```
WORD PutBracket (
    PHEAD WORD * termin ) [extern]
```

Definition at line 1111 of file [execute.c](#).

4.19.4.166 PutIn()

```
LONG PutIn (
    FILEHANDLE * file,
    POSITION * position,
    WORD * buffer,
    WORD ** take,
    int npat ) [extern]
```

Reads a new patch from position in file handle. It is put at buffer, anything after take is moved forward. This would be part of a term that hasn't been used yet. Because of this there should be some space before the start of the buffer

Parameters

<i>file</i>	The file system from which to read
<i>position</i>	The position from which to read
<i>buffer</i>	The buffer into which to read
<i>take</i>	The unused tail should be moved before the buffer
<i>npat</i>	The number of the patch. Is needed if the information was compressed with gzip, because each patch has its own independent gzip encoding.

Definition at line 1320 of file [sort.c](#).

References [FiLe::handle](#).

Referenced by [MergePatches\(\)](#).

4.19.4.167 PutInStore()

```
WORD PutInStore (
    INDEXENTRY * ind,
    WORD num ) [extern]
```

Definition at line 874 of file [store.c](#).

4.19.4.168 PutOut()

```
WORD PutOut (
    PHEAD WORD * term,
    POSITION * position,
    FILEHANDLE * fi,
    WORD ncomp ) [extern]
```

Routine writes one term to file handle at position. It returns the new value of the position.

NOTE: For 'final output' we may have to index the brackets. See the struct BRACKETINDEX. We should maintain:
 1: a list with brackets array with the brackets
 2: a list of objects of type BRACKETINDEX. It contains array with either pointers or offsets to the list of brackets. starting positions in the file. The index may be tied to a maximum size. In that case we may have to prune the list occasionally.

Parameters

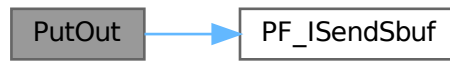
<i>term</i>	The term to be written
<i>position</i>	The position in the file. Afterwards it is updated
<i>fi</i>	The file (or its cache) to which should be written
<i>ncomp</i>	Information about what type of compression should be used

Definition at line 1466 of file [sort.c](#).

References [FiLe::handle](#), and [PF_IsSendSbuf\(\)](#).

Referenced by [EndSort\(\)](#), [generate_expression\(\)](#), [MergePatches\(\)](#), [PF_EndSort\(\)](#), [PF_Processor\(\)](#), and [Processor\(\)](#).

Here is the call graph for this function:



4.19.4.169 Quotient()

```
UWORD Quotient (  
    UWORD * a,  
    WORD * na,  
    WORD b ) [extern]
```

Definition at line 1620 of file [reken.c](#).

4.19.4.170 RaisPow()

```
WORD RaisPow (  
    PHEAD UWORD * a,  
    WORD * na,  
    UWORD b ) [extern]
```

Definition at line 1228 of file [reken.c](#).

4.19.4.171 RaisPowCached()

```
void RaisPowCached (  
    PHEAD WORD x,  
    WORD n,  
    UWORD ** c,  
    WORD * nc ) [extern]
```

Computes power x^n and caches the value

4.19.5 Description

Calculates the power x^n and stores the results for caching purposes. The pointer `c` (i.e., the pointer, and not what it points to) is overwritten. What it points to should not be overwritten in the calling function.

4.19.6 Notes

- Caching is done in `AT.small_power[]`. This array is extended if necessary.

Definition at line 1296 of file [reken.c](#).

Referenced by [poly::divmod_heap\(\)](#), [poly::divmod_univar\(\)](#), and [poly::mul_heap\(\)](#).

4.19.6.1 RaisPowMod()

```
WORD RaisPowMod (
    WORD x,
    WORD n,
    WORD m ) [extern]
```

Definition at line 1383 of file [reken.c](#).

4.19.6.2 NormalModulus()

```
int NormalModulus (
    UWORD * a,
    WORD * na ) [extern]
```

Brings a modular representation in the range $-p/2$ to $+p/2$ The return value tells whether anything was done. Routine made in the general modulus revamp of July 2008 (JV).

Definition at line 1403 of file [reken.c](#).

Referenced by [AddCoef\(\)](#), and [MergePatches\(\)](#).

4.19.6.3 MakeInverses()

```
int MakeInverses (
    void ) [extern]
```

Makes a table of inverses in modular calculus The modulus is in `AC.cmod` and `AC.ncmod` One should notice that the table of inverses can only be made if the modulus fits inside a single FORM word. Otherwise the table lookup becomes too difficult and the table too long.

Definition at line 1440 of file [reken.c](#).

References [GetModInverses\(\)](#).

Here is the call graph for this function:



4.19.6.4 GetModInverses()

```
int GetModInverses (
    WORD m1,
    WORD m2,
    WORD * im1,
    WORD * im2 ) [extern]
```

Input m1 and m2, which are relative prime. determines $a*m1+b*m2 = 1$ (and 1 is the gcd of m1 and m2) then $a*m1 = 1 \bmod m2$ and hence $im1 = a$. and $b*m2 = 1 \bmod m1$ and hence $im2 = b$. Set $m1 = 0*m1+1*m2 = a1*m1+b1*m2$ $m2 = 1*m1+0*m2 = a2*m1+b2*m2$ If everything is OK, the return value is zero

Definition at line [1476](#) of file [reken.c](#).

Referenced by [poly::divmod_heap\(\)](#), [poly::divmod_univar\(\)](#), [MakeDollarMod\(\)](#), [MakeInverses\(\)](#), [MakeMod\(\)](#), [TakeContent\(\)](#), and [TakeSymbolContent\(\)](#).

4.19.6.5 GetLongModInverses()

```
int GetLongModInverses (
    PHEAD UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * ia,
    WORD * nia,
    UWORD * ib,
    WORD * nib ) [extern]
```

Definition at line [1508](#) of file [reken.c](#).

4.19.6.6 RatToLine()

```
void RatToLine (
    UWORD * a,
    WORD na ) [extern]
```

Definition at line [337](#) of file [sch.c](#).

4.19.6.7 RatioFind()

```
WORD RatioFind (
    PHEAD WORD * term,
    WORD * params ) [extern]
```

Definition at line [62](#) of file [ratio.c](#).

4.19.6.8 RatioGen()

```
WORD RatioGen (
    PHEAD WORD * term,
    WORD * params,
    WORD num,
    WORD level ) [extern]
```

Definition at line 170 of file [ratio.c](#).

4.19.6.9 ReNumber()

```
WORD ReNumber (
    PHEAD WORD * term ) [extern]
```

Definition at line 80 of file [reshuf.c](#).

4.19.6.10 ReadSnum()

```
WORD ReadSnum (
    UBYTE ** p ) [extern]
```

Definition at line 1973 of file [tools.c](#).

4.19.6.11 Remain10()

```
WORD Remain10 (
    UWORD * a,
    WORD * na ) [extern]
```

Definition at line 1659 of file [reken.c](#).

4.19.6.12 Remain4()

```
WORD Remain4 (
    UWORD * a,
    WORD * na ) [extern]
```

Definition at line 1686 of file [reken.c](#).

4.19.6.13 ResetScratch()

```
WORD ResetScratch (
    void ) [extern]
```

Definition at line 235 of file [store.c](#).

4.19.6.14 ResolveSet()

```
WORD ResolveSet (
    PHEAD WORD * from,
    WORD * to,
    WORD * subs ) [extern]
```

Definition at line 1361 of file [wildcard.c](#).

4.19.6.15 RevertScratch()

```
WORD RevertScratch (
    void ) [extern]
```

Definition at line 190 of file [store.c](#).

4.19.6.16 ScanFunctions()

```
WORD ScanFunctions (
    PHEAD WORD * inpat,
    WORD * inter,
    WORD par ) [extern]
```

Definition at line 1616 of file [function.c](#).

4.19.6.17 SeekScratch()

```
void SeekScratch (
    FILEHANDLE * fi,
    POSITION * pos ) [extern]
```

Definition at line 64 of file [store.c](#).

4.19.6.18 SetEndScratch()

```
void SetEndScratch (
    FILEHANDLE * f,
    POSITION * position ) [extern]
```

Definition at line 75 of file [store.c](#).

4.19.6.19 SetEndHScratch()

```
void SetEndHScratch (
    FILEHANDLE * f,
    POSITION * position ) [extern]
```

Definition at line 89 of file [store.c](#).

4.19.6.20 SetFileIndex()

```
WORD SetFileIndex (
    void ) [extern]
```

Reads the next file index and puts it into AR.StoreData.Index. TODO

Returns

= 0 everything okay, != 0 an error occurred

Definition at line [2321](#) of file [store.c](#).

References [WriteStoreHeader\(\)](#).

Here is the call graph for this function:



4.19.6.21 Sflush()

```
WORD Sflush (
    FILEHANDLE * fi ) [extern]
```

Puts the contents of a buffer to output Only to be used when there is a single patch in the large buffer.

Parameters

<i>fi</i>	The filesystem (or its cache) to which the patch should be written
-----------	--

Definition at line [1380](#) of file [sort.c](#).

References [FiLe::handle](#).

Referenced by [MergePatches\(\)](#).

4.19.6.22 Simplify()

```
WORD Simplify (
    PHEAD UWORD * a,
    WORD * na,
    UWORD * b,
    WORD * nb ) [extern]
```

Definition at line [530](#) of file [reken.c](#).

4.19.6.23 SortWild()

```
WORD SortWild (
    WORD * w,
    WORD nw ) [extern]
```

Sorts the wildcard entries in the parameter w. Double entries are removed. Full space taken is nw words. Routine serves for the reading of wildcards in the compiler. The entries come in the format: (type,4,number,0) in which the zero is reserved for the future replacement of 'number'.

Parameters

<i>w</i>	buffer with wildcard entries.
<i>nw</i>	number of wildcard entries.

Returns

Normal conventions (OK -> 0)

Definition at line 4753 of file [sort.c](#).

4.19.6.24 LocateBase()

```
FILE * LocateBase (
    char ** name,
    char ** newname,
    char * iomode ) [extern]
```

Definition at line 225 of file [minos.c](#).

4.19.6.25 SplitMerge()

```
LONG SplitMerge (
    PHEAD WORD ** Pointer,
    LONG number ) [extern]
```

Algorithm by J.A.M.Vermaseren (31-7-1988)

Note that AN.SplitScratch and AN.InScratch are used also in GarbHand

Merge sort in memory. The input is an array of pointers. Sorting is done recursively by dividing the array in two equal parts and calling SplitMerge for each. When the parts are small enough we can do the compare and take the appropriate action. An addition is that we look for 'runs'. Sequences that are already ordered. This happens a lot when there is very little action in a module. This made FORM faster by a few percent.

Parameters

<i>Pointer</i>	The array of pointers to the terms to be sorted.
<i>number</i>	The number of pointers in Pointer.

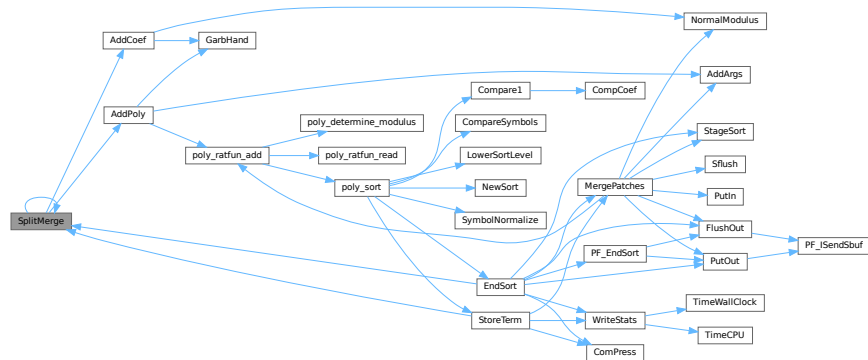
The terms are supposed to be sitting in the small buffer and there is supposed to be an extension to this buffer for when there are two terms that should be added and the result takes more space than each of the original terms. The notation guarantees that the result never needs more space than the sum of the spaces of the original terms.

Definition at line 3362 of file [sort.c](#).

References [AddCoef\(\)](#), [AddPoly\(\)](#), and [SplitMerge\(\)](#).

Referenced by [EndSort\(\)](#), [SplitMerge\(\)](#), and [StoreTerm\(\)](#).

Here is the call graph for this function:



4.19.6.26 StoreTerm()

```
WORD StoreTerm (
    PHEAD WORD * term ) [extern]
```

The central routine to accept terms, store them and keep things at least partially sorted. A call to EndSort will then complete storing and sorting.

Parameters

<i>term</i>	The term to be stored
-------------	-----------------------

Returns

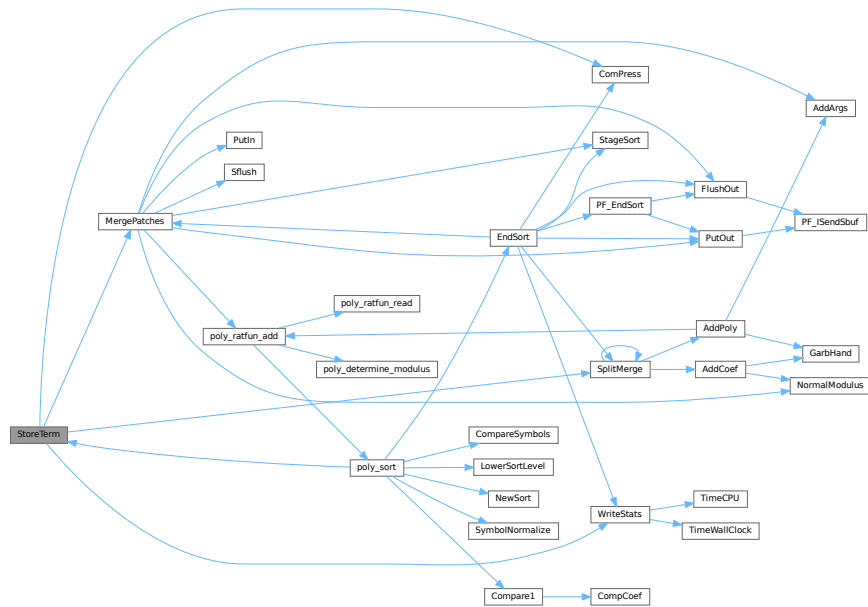
Regular return conventions (OK -> 0)

Definition at line 4540 of file [sort.c](#).

References [ComPress\(\)](#), [MergePatches\(\)](#), [SplitMerge\(\)](#), and [WriteStats\(\)](#).

Referenced by [Generator\(\)](#), [get_expression\(\)](#), [InFunction\(\)](#), [PF_Processor\(\)](#), [poly_sort\(\)](#), [PolyFunMul\(\)](#), [Processor\(\)](#), and [TakeArgContent\(\)](#).

Here is the call graph for this function:



4.19.6.27 SubPLon()

```

void SubPLon (
    UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc ) [extern]

```

Definition at line 803 of file [reken.c](#).

4.19.6.28 Substitute()

```

void Substitute (
    PHEAD WORD * term,
    WORD * pattern,
    WORD power ) [extern]

```

Definition at line 640 of file [pattern.c](#).

4.19.6.29 SymFind()

```

WORD SymFind (
    PHEAD WORD * term,
    WORD * params ) [extern]

```

Definition at line 633 of file [function.c](#).

4.19.6.30 SymGen()

```
WORD SymGen (
    PHEAD WORD * term,
    WORD * params,
    WORD num,
    WORD level ) [extern]
```

Definition at line [518](#) of file [function.c](#).

4.19.6.31 Symmetrize()

```
WORD Symmetrize (
    PHEAD WORD * func,
    WORD * Lijst,
    WORD ngroups,
    WORD gsize,
    WORD type ) [extern]
```

Definition at line [213](#) of file [function.c](#).

4.19.6.32 FullSymmetrize()

```
int FullSymmetrize (
    PHEAD WORD * fun,
    int type ) [extern]
```

Definition at line [473](#) of file [function.c](#).

4.19.6.33 TakeModulus()

```
WORD TakeModulus (
    UWORD * a,
    WORD * na,
    UWORD * cmodvec,
    WORD ncmod,
    WORD par ) [extern]
```

Definition at line [3149](#) of file [reken.c](#).

4.19.6.34 TakeNormalModulus()

```
WORD TakeNormalModulus (
    UWORD * a,
    WORD * na,
    UWORD * c,
    WORD nc,
    WORD par ) [extern]
```

Definition at line [3321](#) of file [reken.c](#).

4.19.6.35 TalToLine()

```
void TalToLine (
    UWORD x ) [extern]
```

Definition at line 522 of file [sch.c](#).

4.19.6.36 TenVec()

```
WORD TenVec (
    PHEAD WORD * term,
    WORD * params,
    WORD num,
    WORD level ) [extern]
```

Definition at line 2312 of file [opera.c](#).

4.19.6.37 TenVecFind()

```
WORD TenVecFind (
    PHEAD WORD * term,
    WORD * params ) [extern]
```

Definition at line 2178 of file [opera.c](#).

4.19.6.38 TermRenumber()

```
WORD TermRenumber (
    WORD * term,
    RENUMBER renumber,
    WORD nexpr ) [extern]
```

```
!! WORD *memterm=term; static LONG ctrap=0; !!!
```

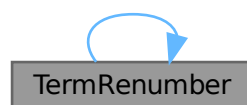
```
!! ctrap++; !!!
```

Definition at line 2428 of file [store.c](#).

References [ReNuMbEr::func](#), [ReNuMbEr::funnum](#), [ReNuMbEr::indi](#), [ReNuMbEr::indnum](#), [ReNuMbEr::symb](#), [ReNuMbEr::symnum](#), [TermRenumber\(\)](#), [ReNuMbEr::vecnum](#), and [ReNuMbEr::vect](#).

Referenced by [TermRenumber\(\)](#).

Here is the call graph for this function:



4.19.6.39 TestDrop()

```
void TestDrop (
    void ) [extern]
```

Definition at line 483 of file [execute.c](#).

4.19.6.40 PutInVflags()

```
void PutInVflags (
    WORD nexpr ) [extern]
```

Definition at line 561 of file [execute.c](#).

4.19.6.41 TestMatch()

```
WORD TestMatch (
    PHEAD WORD * term,
    WORD * level ) [extern]
```

This routine governs the pattern matching. If it decides that a substitution should be made, this can be either the insertion of a right hand side (C->rhs) or the automatic generation of terms as a result of an operation (like trace). The object to be replaced is removed from term and a subexpression pointer is inserted. If the substitution is made more than once there can be more subexpression pointers. Its number is positive as it corresponds to the level at which the C->rhs can be found in the compiler output. The subexpression pointer contains the wildcard substitution information. The power is found in *AT.TMout. For operations the subexpression pointer is negative and corresponds to an address in the array AT.TMout. In this array are then the instructions for the routine to be called and its number in the array 'Operations' The format is here: length,functionnumber,length-2 parameters

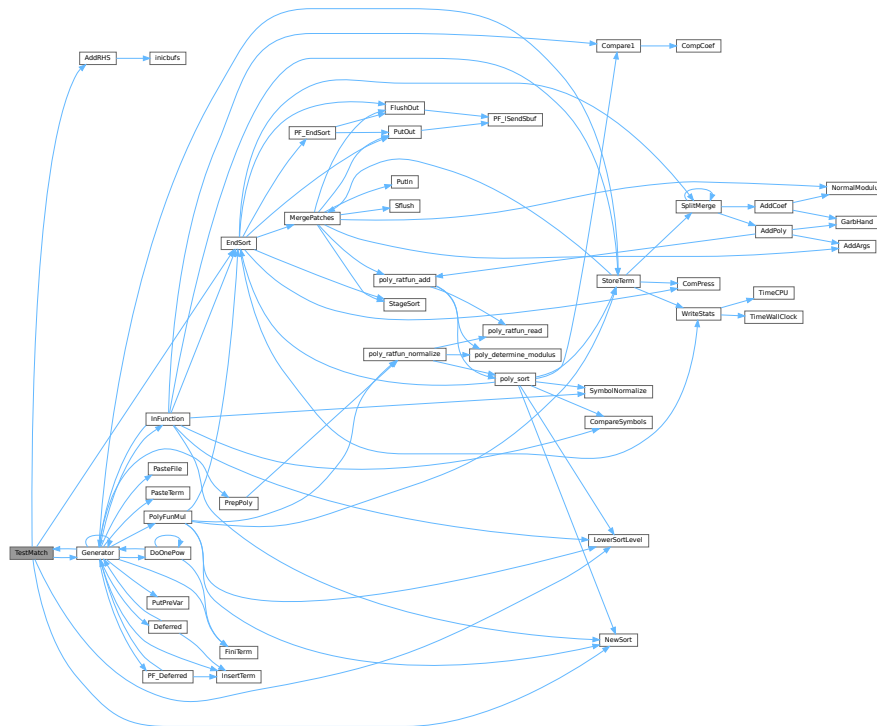
There is a certain complexity wrt repeat levels. Another complication is the poking of the wildcard values in the subexpression prototype in the compiler buffer. This was how things were done in the past with sequential FORM, but with the advent of TFORM this cannot be maintained. Now, for TFORM we make a copy of it. 7-may-2008 (JV): We cannot yet guarantee that this has been done 100% correctly. There are errors that occur in TFORM only and that may indicate problems.

Definition at line 97 of file [pattern.c](#).

References [AddRHS\(\)](#), [EndSort\(\)](#), [Generator\(\)](#), [CbUf::lhs](#), [LowerSortLevel\(\)](#), and [NewSort\(\)](#).

Referenced by [Generator\(\)](#).

Here is the call graph for this function:



4.19.6.42 TestSub()

```
WORD TestSub (
    PHEAD WORD * term,
    WORD level ) [extern]
```

Definition at line 684 of file [proces.c](#).

4.19.6.43 TimeCPU()

```
LONG TimeCPU (
    WORD par ) [extern]
```

Returns the CPU time.

Parameters

<i>par</i>	If zero, the CPU time will be reset to 0.
------------	---

Returns

The CPU time in milliseconds.

Definition at line 3551 of file [tools.c](#).

Referenced by [PF_GetSlaveTimes\(\)](#), [PF_Processor\(\)](#), and [WriteStats\(\)](#).

4.19.6.44 TimeChildren()

```
LONG TimeChildren (
    WORD par ) [extern]
```

Definition at line [3533](#) of file [tools.c](#).

4.19.6.45 TimeWallClock()

```
LONG TimeWallClock (
    WORD par ) [extern]
```

Returns the wall-clock time.

Parameters

<i>par</i>	If zero, the wall-clock time will be reset to 0.
------------	--

Returns

The wall-clock time in centiseconds.

Definition at line [3477](#) of file [tools.c](#).

Referenced by [DoCheckpoint\(\)](#), [DoRecovery\(\)](#), [find_Horner_MCTS\(\)](#), [optimize_expression_given_Horner\(\)](#), [optimize_greedy\(\)](#), and [WriteStats\(\)](#).

4.19.6.46 ToStorage()

```
WORD ToStorage (
    EXPRESSIONS e,
    POSITION * length ) [extern]
```

Definition at line [2017](#) of file [store.c](#).

4.19.6.47 TokenToLine()

```
void TokenToLine (
    UBYTE * s ) [extern]
```

Definition at line [551](#) of file [sch.c](#).

4.19.6.48 Trace4()

```
WORD Trace4 (
    PHEAD WORD * term,
    WORD * params,
    WORD num,
    WORD level ) [extern]
```

Definition at line [694](#) of file [opera.c](#).

4.19.6.49 Trace4Gen()

```
WORD Trace4Gen (
    PHEAD TRACES * t,
    WORD number ) [extern]
```

Definition at line 856 of file [opera.c](#).

4.19.6.50 Trace4no()

```
WORD Trace4no (
    WORD number,
    WORD * kron,
    TRACES * t ) [extern]
```

Definition at line 418 of file [opera.c](#).

4.19.6.51 TraceFind()

```
WORD TraceFind (
    PHEAD WORD * term,
    WORD * params ) [extern]
```

Definition at line 1853 of file [opera.c](#).

4.19.6.52 TraceN()

```
WORD TraceN (
    PHEAD WORD * term,
    WORD * params,
    WORD num,
    WORD level ) [extern]
```

Definition at line 1382 of file [opera.c](#).

4.19.6.53 TraceNgen()

```
WORD TraceNgen (
    PHEAD TRACES * t,
    WORD number ) [extern]
```

Definition at line 1446 of file [opera.c](#).

4.19.6.54 TraceNno()

```
WORD TraceNno (
    WORD number,
    WORD * kron,
    TRACES * t ) [extern]
```

Definition at line 1341 of file [opera.c](#).

4.19.6.55 Traces()

```
WORD Traces (
    PHEAD WORD * term,
    WORD * params,
    WORD num,
    WORD level ) [extern]
```

Definition at line [1831](#) of file [opera.c](#).

4.19.6.56 Trick()

```
WORD Trick (
    WORD * in,
    TRACES * t ) [extern]
```

Definition at line [331](#) of file [opera.c](#).

4.19.6.57 TryDo()

```
WORD TryDo (
    PHEAD WORD * term,
    WORD * pattern,
    WORD level ) [extern]
```

Definition at line [1072](#) of file [reshuf.c](#).

4.19.6.58 UnPack()

```
void UnPack (
    UWORD * a,
    WORD na,
    WORD * denom,
    WORD * numer ) [extern]
```

Definition at line [100](#) of file [reken.c](#).

4.19.6.59 VarStore()

```
WORD VarStore (
    UBYTE * s,
    WORD n,
    WORD name,
    WORD namesize ) [extern]
```

Definition at line [2370](#) of file [store.c](#).

4.19.6.60 WildFill()

```
WORD WildFill (
    PHEAD WORD * to,
    WORD * from,
    WORD * sub ) [extern]
```

Definition at line 65 of file [wildcard.c](#).

4.19.6.61 WriteAll()

```
WORD WriteAll (
    void ) [extern]
```

Definition at line 2501 of file [sch.c](#).

4.19.6.62 WriteOne()

```
WORD WriteOne (
    UBYTE * name,
    int alreadyinline,
    int noseimi,
    WORD plus ) [extern]
```

Definition at line 2727 of file [sch.c](#).

4.19.6.63 WriteArgument()

```
void WriteArgument (
    WORD * t ) [extern]
```

Definition at line 1424 of file [sch.c](#).

4.19.6.64 WriteExpression()

```
WORD WriteExpression (
    WORD * terms,
    LONG ltot ) [extern]
```

Definition at line 2470 of file [sch.c](#).

4.19.6.65 WriteInnerTerm()

```
WORD WriteInnerTerm (
    WORD * term,
    WORD first ) [extern]
```

Definition at line 1951 of file [sch.c](#).

4.19.6.66 WriteLists()

```
void WriteLists (
    void ) [extern]
```

Definition at line 810 of file [sch.c](#).

4.19.6.67 WriteSetup()

```
void WriteSetup (
    void ) [extern]
```

Definition at line 811 of file [setfile.c](#).

4.19.6.68 WriteStats()

```
void WriteStats (
    POSITION * plspace,
    WORD par,
    WORD checkLogType ) [extern]
```

Writes the statistics.

Parameters

<i>plspace</i>	The size in bytes currently occupied
<i>par</i>	par = 0 = STATSSPLITMERGE after a splitmerge. par = 1 = STATSMERGETOFILE after merge to sortfile. par = 2 = STATSPOSTSORT after the sort
<i>checkLogType</i>	== CHECKLOGTYPE: The output should not go to the log file if AM.LogType is 1.

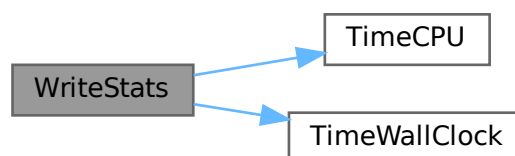
current expression is to be found in AR.CurExpr. terms are in S->TermsLeft. S->GenTerms.

Definition at line 121 of file [sort.c](#).

References [TimeCPU\(\)](#), and [TimeWallClock\(\)](#).

Referenced by [EndSort\(\)](#), [PF_Processor\(\)](#), and [StoreTerm\(\)](#).

Here is the call graph for this function:



4.19.6.69 WriteSubTerm()

```
WORD WriteSubTerm (
    WORD * sterm,
    WORD first ) [extern]
```

Definition at line 1542 of file [sch.c](#).

4.19.6.70 WriteTerm()

```
WORD WriteTerm (
    WORD * term,
    WORD * lbrac,
    WORD first,
    WORD prtf,
    WORD br ) [extern]
```

Definition at line 2148 of file [sch.c](#).

4.19.6.71 execarg()

```
WORD execarg (
    PHEAD WORD * term,
    WORD level ) [extern]
```

Definition at line 56 of file [argument.c](#).

4.19.6.72 execterm()

```
WORD execterm (
    PHEAD WORD * term,
    WORD level ) [extern]
```

Definition at line 1769 of file [argument.c](#).

4.19.6.73 SpecialCleanup()

```
void SpecialCleanup (
    PHEAD0 ) [extern]
```

Definition at line 1623 of file [execute.c](#).

4.19.6.74 SetMods()

```
void SetMods (
    void ) [extern]
```

Definition at line 1637 of file [execute.c](#).

4.19.6.75 UnSetMods()

```
void UnSetMods (
    void ) [extern]
```

Definition at line 1655 of file [execute.c](#).

4.19.6.76 DoExecute()

```
WORD DoExecute (
    WORD par,
    WORD skip ) [extern]
```

Definition at line 615 of file [execute.c](#).

4.19.6.77 SetScratch()

```
void SetScratch (
    FILEHANDLE * f,
    POSITION * position ) [extern]
```

Definition at line 115 of file [store.c](#).

4.19.6.78 Warning()

```
void Warning (
    char * s ) [extern]
```

Definition at line 790 of file [message.c](#).

4.19.6.79 HighWarning()

```
void HighWarning (
    char * s ) [extern]
```

Definition at line 802 of file [message.c](#).

4.19.6.80 SpareTable()

```
int SpareTable (
    TABLES TT ) [extern]
```

Definition at line 694 of file [tables.c](#).

4.19.6.81 strDup1()

```
UBYTE * strDup1 (
    UBYTE * instr,
    char * ifwrong ) [extern]
```

Definition at line 1904 of file [tools.c](#).

4.19.6.82 Malloc()

```
void * Malloc (
    LONG size ) [extern]
```

Definition at line 2221 of file [tools.c](#).

4.19.6.83 Malloc1()

```
void * Malloc1 (
    LONG size,
    const char * messageifwrong ) [extern]
```

Definition at line 2303 of file [tools.c](#).

4.19.6.84 DoTail()

```
int DoTail (
    int argc,
    UBYTE ** argv ) [extern]
```

Definition at line 237 of file [startup.c](#).

4.19.6.85 OpenInput()

```
int OpenInput (
    void ) [extern]
```

Definition at line 542 of file [startup.c](#).

4.19.6.86 PutPreVar()

```
int PutPreVar (
    UBYTE * name,
    UBYTE * value,
    UBYTE * args,
    int mode ) [extern]
```

Inserts/Updates a preprocessor variable in the name administration.

Parameters

<i>name</i>	Character string with the variable name.
<i>value</i>	Character string with a possible value. Special case: if this argument is zero, then we have no value. Note: This is different from having an empty argument! This should only occur when the name starts with a ?
<i>args</i>	Character string with possible arguments.
<i>mode</i>	=0: always create a new name entry, =1: try to do a redefinition if possible.

Returns

Index of used entry in name list.

Definition at line 718 of file [pre.c](#).

Referenced by [ClearOptimize\(\)](#), [Generator\(\)](#), [Optimize\(\)](#), [PF_BroadcastRedefinedPreVars\(\)](#), [StartVariables\(\)](#), and [TheDefine\(\)](#).

4.19.6.87 Error0()

```
void Error0 (
    char * s ) [extern]
```

Definition at line 56 of file [message.c](#).

4.19.6.88 Error1()

```
void Error1 (
    char * s,
    UBYTE * t ) [extern]
```

Definition at line 67 of file [message.c](#).

4.19.6.89 Error2()

```
void Error2 (
    char * s1,
    char * s2,
    UBYTE * t ) [extern]
```

Definition at line 78 of file [message.c](#).

4.19.6.90 ReadFromStream()

```
UBYTE ReadFromStream (
    STREAM * stream ) [extern]
```

Definition at line 151 of file [tools.c](#).

4.19.6.91 GetFromStream()

```
UBYTE GetFromStream (  
    STREAM * stream ) [extern]
```

Definition at line 286 of file [tools.c](#).

4.19.6.92 LookInStream()

```
UBYTE LookInStream (  
    STREAM * stream ) [extern]
```

Definition at line 309 of file [tools.c](#).

4.19.6.93 OpenStream()

```
STREAM * OpenStream (  
    UBYTE * name,  
    int type,  
    int prevarmode,  
    int raiselow ) [extern]
```

Definition at line 321 of file [tools.c](#).

4.19.6.94 LocateFile()

```
int LocateFile (  
    UBYTE ** name,  
    int type ) [extern]
```

Definition at line 576 of file [tools.c](#).

4.19.6.95 CloseStream()

```
STREAM * CloseStream (  
    STREAM * stream ) [extern]
```

Definition at line 663 of file [tools.c](#).

4.19.6.96 PositionStream()

```
void PositionStream (  
    STREAM * stream,  
    LONG position ) [extern]
```

Definition at line 823 of file [tools.c](#).

4.19.6.97 ReverseStatements()

```
int ReverseStatements (
    STREAM * stream ) [extern]
```

Definition at line 855 of file [tools.c](#).

4.19.6.98 ProcessOption()

```
int ProcessOption (
    UBYTE * s1,
    UBYTE * s2,
    int filetype ) [extern]
```

Definition at line 182 of file [setfile.c](#).

4.19.6.99 DoSetups()

```
int DoSetups (
    void ) [extern]
```

Definition at line 132 of file [setfile.c](#).

4.19.6.100 TerminateImpl()

```
void TerminateImpl (
    int errorcode,
    const char * file,
    int line,
    const char * function ) [extern]
```

Definition at line 1850 of file [startup.c](#).

4.19.6.101 GetNode()

```
NAMENODE * GetNode (
    NAMETREE * nametree,
    UBYTE * name ) [extern]
```

Definition at line 49 of file [names.c](#).

4.19.6.102 AddName()

```
int AddName (
    NAMETREE * nametree,
    UBYTE * name,
    WORD type,
    WORD number,
    int * nodenum ) [extern]
```

Definition at line 71 of file [names.c](#).

4.19.6.103 GetName()

```
int GetName (
    NAMETREE * nametree,
    UBYTE * namein,
    WORD * number,
    int par ) [extern]
```

Definition at line 252 of file [names.c](#).

4.19.6.104 GetFunction()

```
UBYTE * GetFunction (
    UBYTE * s,
    WORD * funnum ) [extern]
```

Definition at line 342 of file [names.c](#).

4.19.6.105 GetNumber()

```
UBYTE * GetNumber (
    UBYTE * s,
    WORD * num ) [extern]
```

Definition at line 388 of file [names.c](#).

4.19.6.106 GetLastExprName()

```
int GetLastExprName (
    UBYTE * name,
    WORD * number ) [extern]
```

Definition at line 438 of file [names.c](#).

4.19.6.107 GetAutoName()

```
int GetAutoName (
    UBYTE * name,
    WORD * number ) [extern]
```

Definition at line 479 of file [names.c](#).

4.19.6.108 GetVar()

```
int GetVar (
    UBYTE * name,
    WORD * type,
    WORD * number,
    int wantedtype,
    int par ) [extern]
```

Definition at line 524 of file [names.c](#).

4.19.6.109 MakeDubious()

```
int MakeDubious (
    NAMETREE * nametree,
    UBYTE * name,
    WORD * number ) [extern]
```

Definition at line 2693 of file [names.c](#).

4.19.6.110 GetOName()

```
int GetOName (
    NAMETREE * nametree,
    UBYTE * name,
    WORD * number,
    int par ) [extern]
```

Definition at line 460 of file [names.c](#).

4.19.6.111 DumpTree()

```
void DumpTree (
    NAMETREE * nametree ) [extern]
```

Definition at line 602 of file [names.c](#).

4.19.6.112 DumpNode()

```
void DumpNode (
    NAMETREE * nametree,
    WORD node,
    WORD depth ) [extern]
```

Definition at line 615 of file [names.c](#).

4.19.6.113 LinkTree()

```
void LinkTree (
    NAMETREE * tree,
    WORD offset,
    WORD numnodes ) [extern]
```

Definition at line 784 of file [names.c](#).

4.19.6.114 CopyTree()

```
void CopyTree (
    NAMETREE * newtree,
    NAMETREE * oldtree,
    WORD node,
    WORD par ) [extern]
```

Definition at line 696 of file [names.c](#).

4.19.6.115 CompactifyTree()

```
int CompactifyTree (
    NAMETREE * nametree,
    WORD par ) [extern]
```

Definition at line 634 of file [names.c](#).

4.19.6.116 MakeNameTree()

```
NAMETREE * MakeNameTree (
    void ) [extern]
```

Definition at line 815 of file [names.c](#).

4.19.6.117 FreeNameTree()

```
void FreeNameTree (
    NAMETREE * n ) [extern]
```

Definition at line 834 of file [names.c](#).

4.19.6.118 AddExpression()

```
int AddExpression (
    UBYTE * name,
    int x,
    int y ) [extern]
```

Definition at line 2744 of file [names.c](#).

4.19.6.119 AddSymbol()

```
int AddSymbol (
    UBYTE * name,
    int minpow,
    int maxpow,
    int cplx,
    int dim ) [extern]
```

Definition at line 920 of file [names.c](#).

4.19.6.120 AddDollar()

```
int AddDollar (
    UBYTE * name,
    WORD type,
    WORD * start,
    LONG size ) [extern]
```

Definition at line 2602 of file [names.c](#).

4.19.6.121 ReplaceDollar()

```
int ReplaceDollar (
    WORD number,
    WORD newtype,
    WORD * newstart,
    LONG newsize ) [extern]
```

Definition at line [2646](#) of file [names.c](#).

4.19.6.122 DollarRaiseLow()

```
int DollarRaiseLow (
    UBYTE * name,
    LONG value ) [extern]
```

Definition at line [2552](#) of file [dollar.c](#).

4.19.6.123 AddVector()

```
int AddVector (
    UBYTE * name,
    int cplx,
    int dim ) [extern]
```

Definition at line [1244](#) of file [names.c](#).

4.19.6.124 AddDubious()

```
int AddDubious (
    UBYTE * name ) [extern]
```

Definition at line [2679](#) of file [names.c](#).

4.19.6.125 AddIndex()

```
int AddIndex (
    UBYTE * name,
    int dim,
    int dim4 ) [extern]
```

Definition at line [1088](#) of file [names.c](#).

4.19.6.126 DoDimension()

```
UBYTE * DoDimension (
    UBYTE * s,
    int * dim,
    int * dim4 ) [extern]
```

Definition at line [1160](#) of file [names.c](#).

4.19.6.127 AddFunction()

```
int AddFunction (
    UBYTE * name,
    int comm,
    int istensor,
    int cplx,
    int symprop,
    int dim,
    int argmax,
    int argmin ) [extern]
```

Definition at line 1326 of file [names.c](#).

4.19.6.128 CoCommuteInSet()

```
int CoCommuteInSet (
    UBYTE * s ) [extern]
```

Definition at line 1356 of file [names.c](#).

4.19.6.129 CoFunction()

```
int CoFunction (
    UBYTE * s,
    int comm,
    int istensor ) [extern]
```

Definition at line 1446 of file [names.c](#).

4.19.6.130 TestName()

```
int TestName (
    UBYTE * name ) [extern]
```

Definition at line 3254 of file [names.c](#).

4.19.6.131 AddSet()

```
int AddSet (
    UBYTE * name,
    WORD dim ) [extern]
```

Definition at line 2122 of file [names.c](#).

4.19.6.132 DoElements()

```
int DoElements (
    UBYTE * s,
    SETS set,
    UBYTE * name ) [extern]
```

Definition at line 2155 of file [names.c](#).

4.19.6.133 DoTempSet()

```
int DoTempSet (
    UBYTE * from,
    UBYTE * to ) [extern]
```

Definition at line [2480](#) of file [names.c](#).

4.19.6.134 NameConflict()

```
int NameConflict (
    int type,
    UBYTE * name ) [extern]
```

Definition at line [2728](#) of file [names.c](#).

4.19.6.135 OpenFile()

```
int OpenFile (
    char * name ) [extern]
```

Definition at line [983](#) of file [tools.c](#).

4.19.6.136 OpenAddFile()

```
int OpenAddFile (
    char * name ) [extern]
```

Definition at line [1002](#) of file [tools.c](#).

4.19.6.137 ReOpenFile()

```
int ReOpenFile (
    char * name ) [extern]
```

Definition at line [1023](#) of file [tools.c](#).

4.19.6.138 CreateFile()

```
int CreateFile (
    char * name ) [extern]
```

Definition at line [1043](#) of file [tools.c](#).

4.19.6.139 CreateLogFile()

```
int CreateLogFile (
    char * name ) [extern]
```

Definition at line [1060](#) of file [tools.c](#).

4.19.6.140 CloseFile()

```
void CloseFile (
    int handle ) [extern]
```

Definition at line 1078 of file [tools.c](#).

4.19.6.141 CopyFile()

```
int CopyFile (
    char * source,
    char * dest ) [extern]
```

Copies a file with name *source to a file named *dest. The involved files must not be open. Returns non-zero if an error occurred. Uses if possible the combined large and small sorting buffers as cache.

Definition at line 1101 of file [tools.c](#).

Referenced by [DoRecovery\(\)](#).

4.19.6.142 CreateHandle()

```
int CreateHandle (
    void ) [extern]
```

Definition at line 1162 of file [tools.c](#).

4.19.6.143 ReadFile()

```
LONG ReadFile (
    int handle,
    UBYTE * buffer,
    LONG size ) [extern]
```

Definition at line 1217 of file [tools.c](#).

4.19.6.144 ReadPosFile()

```
LONG ReadPosFile (
    PHEAD FILEHANDLE * fi,
    UBYTE * buffer,
    LONG size,
    POSITION * pos ) [extern]
```

Definition at line 1267 of file [tools.c](#).

4.19.6.145 WriteFileToFile()

```
LONG WriteFileToFile (
    int handle,
    UBYTE * buffer,
    LONG size ) [extern]
```

Definition at line [1333](#) of file [tools.c](#).

4.19.6.146 SeekFile()

```
void SeekFile (
    int handle,
    POSITION * offset,
    int origin ) [extern]
```

Definition at line [1371](#) of file [tools.c](#).

4.19.6.147 TellFile()

```
LONG TellFile (
    int handle ) [extern]
```

Definition at line [1396](#) of file [tools.c](#).

4.19.6.148 FlushFile()

```
void FlushFile (
    int handle ) [extern]
```

Definition at line [1420](#) of file [tools.c](#).

4.19.6.149 GetPosFile()

```
int GetPosFile (
    int handle,
    fpos_t * pospointer ) [extern]
```

Definition at line [1434](#) of file [tools.c](#).

4.19.6.150 SetPosFile()

```
int SetPosFile (
    int handle,
    fpos_t * pospointer ) [extern]
```

Definition at line [1448](#) of file [tools.c](#).

4.19.6.151 SynchFile()

```
void SynchFile (
    int handle ) [extern]
```

Definition at line 1468 of file [tools.c](#).

4.19.6.152 TruncateFile()

```
void TruncateFile (
    int handle ) [extern]
```

Definition at line 1490 of file [tools.c](#).

4.19.6.153 GetChannel()

```
int GetChannel (
    char * name,
    int mode ) [extern]
```

Definition at line 1510 of file [tools.c](#).

4.19.6.154 GetAppendChannel()

```
int GetAppendChannel (
    char * name ) [extern]
```

Definition at line 1546 of file [tools.c](#).

4.19.6.155 CloseChannel()

```
int CloseChannel (
    char * name ) [extern]
```

Definition at line 1576 of file [tools.c](#).

4.19.6.156 inictable()

```
void inictable (
    void ) [extern]
```

Definition at line 310 of file [compiler.c](#).

4.19.6.157 findcommand()

```
KEYWORD * findcommand (
    UBYTE * in ) [extern]
```

Definition at line 336 of file [compiler.c](#).

4.19.6.158 inicbufs()

```
int inicbufs (
    void ) [extern]
```

Creates a new compiler buffer and returns its ID number.

Returns

The ID number for the new compiler buffer.

Definition at line 47 of file [comtool.c](#).

References [CbUf::boomlijst](#), [CbUf::Buffer](#), [CbUf::BufferSize](#), [CbUf::CanCommu](#), [CbUf::dimension](#), [CbUf::lhs](#), [CbUf::numdum](#), [CbUf::NumTerms](#), [CbUf::Pointer](#), [CbUf::rhs](#), and [CbUf::Top](#).

Referenced by [AddRHS\(\)](#), and [StartVariables\(\)](#).

4.19.6.159 StartFiles()

```
void StartFiles (
    void ) [extern]
```

Definition at line 956 of file [tools.c](#).

4.19.6.160 MakeDate()

```
UBYTE * MakeDate (
    void ) [extern]
```

Definition at line 2090 of file [tools.c](#).

4.19.6.161 PreProcessor()

```
void PreProcessor (
    void ) [extern]
```

Definition at line 947 of file [pre.c](#).

4.19.6.162 FromList()

```
void * FromList (
    LIST * L ) [extern]
```

Definition at line 2780 of file [tools.c](#).

4.19.6.163 From0List()

```
void * From0List (
    LIST * L ) [extern]
```

Definition at line 2806 of file [tools.c](#).

4.19.6.164 FromVarList()

```
void * FromVarList (
    LIST * L ) [extern]
```

Definition at line 2835 of file [tools.c](#).

4.19.6.165 DoubleList()

```
int DoubleList (
    void *** lijst,
    int * oldsize,
    int objectsize,
    char * nameoftype ) [extern]
```

Definition at line 2877 of file [tools.c](#).

4.19.6.166 DoubleLList()

```
int DoubleLList (
    void *** lijst,
    LONG * oldsize,
    int objectsize,
    char * nameoftype ) [extern]
```

Definition at line 2929 of file [tools.c](#).

4.19.6.167 DoubleBuffer()

```
void DoubleBuffer (
    void ** start,
    void ** stop,
    int size,
    char * text ) [extern]
```

Definition at line 2977 of file [tools.c](#).

4.19.6.168 ExpandBuffer()

```
void ExpandBuffer (
    void ** buffer,
    LONG * oldsize,
    int type ) [extern]
```

Definition at line 3002 of file [tools.c](#).

4.19.6.169 iexp()

```
LONG iexp (
    LONG x,
    int p ) [extern]
```

Definition at line [3026](#) of file [tools.c](#).

4.19.6.170 IsLikeVector()

```
int IsLikeVector (
    WORD * arg ) [extern]
```

Definition at line [3229](#) of file [tools.c](#).

4.19.6.171 AreArgsEqual()

```
int AreArgsEqual (
    WORD * arg1,
    WORD * arg2 ) [extern]
```

Definition at line [3257](#) of file [tools.c](#).

4.19.6.172 CompareArgs()

```
int CompareArgs (
    WORD * arg1,
    WORD * arg2 ) [extern]
```

Definition at line [3276](#) of file [tools.c](#).

4.19.6.173 SkipField()

```
UBYTE * SkipField (
    UBYTE * s,
    int level ) [extern]
```

Definition at line [1946](#) of file [tools.c](#).

4.19.6.174 StrCmp()

```
int StrCmp (
    UBYTE * s1,
    UBYTE * s2 ) [extern]
```

Definition at line [1700](#) of file [tools.c](#).

4.19.6.175 StrICmp()

```
int StrICmp (
    UBYTE * s1,
    UBYTE * s2 ) [extern]
```

Definition at line 1711 of file [tools.c](#).

4.19.6.176 StrHICmp()

```
int StrHICmp (
    UBYTE * s1,
    UBYTE * s2 ) [extern]
```

Definition at line 1722 of file [tools.c](#).

4.19.6.177 StrICont()

```
int StrICont (
    UBYTE * s1,
    UBYTE * s2 ) [extern]
```

Definition at line 1733 of file [tools.c](#).

4.19.6.178 CmpArray()

```
int CmpArray (
    WORD * t1,
    WORD * t2,
    WORD n ) [extern]
```

Definition at line 1745 of file [tools.c](#).

4.19.6.179 ConWord()

```
int ConWord (
    UBYTE * s1,
    UBYTE * s2 ) [extern]
```

Definition at line 1759 of file [tools.c](#).

4.19.6.180 StrLen()

```
int StrLen (
    UBYTE * s ) [extern]
```

Definition at line 1771 of file [tools.c](#).

4.19.6.181 GetPreVar()

```
UBYTE * GetPreVar (
    UBYTE * name,
    int flag ) [extern]
```

Definition at line [536](#) of file [pre.c](#).

4.19.6.182 ToGeneral()

```
void ToGeneral (
    WORD * r,
    WORD * m,
    WORD par ) [extern]
```

Definition at line [3057](#) of file [tools.c](#).

4.19.6.183 ToPolyFunGeneral()

```
WORD ToPolyFunGeneral (
    PHEAD WORD * term ) [extern]
```

Definition at line [3154](#) of file [tools.c](#).

4.19.6.184 ToFast()

```
int ToFast (
    WORD * r,
    WORD * m ) [extern]
```

Definition at line [3101](#) of file [tools.c](#).

4.19.6.185 GetSetupPar()

```
SETUPPARAMETERS * GetSetupPar (
    UBYTE * s ) [extern]
```

Definition at line [337](#) of file [setfile.c](#).

4.19.6.186 RecalcSetups()

```
int RecalcSetups (
    void ) [extern]
```

Definition at line [357](#) of file [setfile.c](#).

4.19.6.187 AllocSetups()

```
int AllocSetups (
    void ) [extern]
```

Definition at line [404](#) of file [setfile.c](#).

4.19.6.188 AllocSort()

```
SORTING * AllocSort (
    LONG inLargeSize,
    LONG inSmallSize,
    LONG inSmallEsize,
    LONG inTermsInSmall,
    int inMaxPatches,
    int inMaxFpatches,
    LONG inIOSize,
    int level ) [extern]
```

Definition at line [869](#) of file [setfile.c](#).

4.19.6.189 AllocSortFileName()

```
void AllocSortFileName (
    SORTING * sort ) [extern]
```

Definition at line [1019](#) of file [setfile.c](#).

4.19.6.190 LoadInputFile()

```
UBYTE * LoadInputFile (
    UBYTE * filename,
    int type ) [extern]
```

Definition at line [112](#) of file [tools.c](#).

4.19.6.191 GetInput()

```
UBYTE GetInput (
    void ) [extern]
```

Definition at line [132](#) of file [pre.c](#).

4.19.6.192 ClearPushback()

```
void ClearPushback (
    void ) [extern]
```

Definition at line [161](#) of file [pre.c](#).

4.19.6.193 GetChar()

```
UBYTE GetChar (
    int level ) [extern]
```

Definition at line 179 of file [pre.c](#).

4.19.6.194 CharOut()

```
void CharOut (
    UBYTE c ) [extern]
```

Definition at line 489 of file [pre.c](#).

4.19.6.195 UnsetAllowDelay()

```
void UnsetAllowDelay (
    void ) [extern]
```

Definition at line 510 of file [pre.c](#).

4.19.6.196 PopPreVars()

```
void PopPreVars (
    int tonumber ) [extern]
```

Definition at line 820 of file [pre.c](#).

4.19.6.197 IniModule()

```
void IniModule (
    int type ) [extern]
```

Definition at line 835 of file [pre.c](#).

4.19.6.198 IniSpecialModule()

```
void IniSpecialModule (
    int type ) [extern]
```

Definition at line 937 of file [pre.c](#).

4.19.6.199 ModuleInstruction()

```
int ModuleInstruction (
    int * moduletype,
    int * specialtype ) [extern]
```

Definition at line 76 of file [module.c](#).

4.19.6.200 PreProInstruction()

```
int PreProInstruction (
    void ) [extern]
```

Definition at line 1166 of file [pre.c](#).

4.19.6.201 LoadInstruction()

```
int LoadInstruction (
    int mode ) [extern]
```

Definition at line 1254 of file [pre.c](#).

4.19.6.202 LoadStatement()

```
int LoadStatement (
    int type ) [extern]
```

Definition at line 1457 of file [pre.c](#).

4.19.6.203 FindKeyWord()

```
KEYWORD * FindKeyWord (
    UBYTE * theword,
    KEYWORD * table,
    int size ) [extern]
```

Definition at line 1963 of file [pre.c](#).

4.19.6.204 FindInKeyWord()

```
KEYWORD * FindInKeyWord (
    UBYTE * theword,
    KEYWORD * table,
    int size ) [extern]
```

Definition at line 1994 of file [pre.c](#).

4.19.6.205 DoDefine()

```
int DoDefine (
    UBYTE * s ) [extern]
```

Definition at line 2159 of file [pre.c](#).

4.19.6.206 DoRedefine()

```
int DoRedefine (
    UBYTE * s ) [extern]
```

Definition at line 2169 of file [pre.c](#).

4.19.6.207 TheDefine()

```
int TheDefine (
    UBYTE * s,
    int mode ) [extern]
```

Preprocessor assignment. Possible arguments and values are treated and the new preprocessor variable is put into the name administration.

Parameters

<i>s</i>	Pointer to the character string following the preprocessor command.
<i>mode</i>	Bitmask. 0-bit clear: always create a new name entry, 0-bit set: try to redefine an existing name, 1-bit set: ignore preprocessor if/switch status.

Returns

zero: no errors, negative number: errors.

Definition at line 2024 of file [pre.c](#).

References [PutPreVar\(\)](#).

Here is the call graph for this function:



4.19.6.208 TheUndefine()

```
int TheUndefine (
    UBYTE * name ) [extern]
```

Definition at line 2209 of file [pre.c](#).

4.19.6.209 ClearMacro()

```
int ClearMacro (
    UBYTE * name ) [extern]
```

Definition at line 2181 of file [pre.c](#).

4.19.6.210 DoUndefine()

```
int DoUndefine (
    UBYTE * s ) [extern]
```

Definition at line 2263 of file [pre.c](#).

4.19.6.211 DoInclude()

```
int DoInclude (
    UBYTE * s ) [extern]
```

Definition at line 2315 of file [pre.c](#).

4.19.6.212 DoReverseInclude()

```
int DoReverseInclude (
    UBYTE * s ) [extern]
```

Definition at line 2322 of file [pre.c](#).

4.19.6.213 Include()

```
int Include (
    UBYTE * s,
    int type ) [extern]
```

Definition at line 2329 of file [pre.c](#).

4.19.6.214 DoExternal()

```
int DoExternal (
    UBYTE * s ) [extern]
```

Definition at line 5424 of file [pre.c](#).

4.19.6.215 DoToExternal()

```
int DoToExternal (
    UBYTE * s ) [extern]
```

Definition at line 5986 of file [pre.c](#).

4.19.6.216 DoFromExternal()

```
int DoFromExternal (
    UBYTE * s ) [extern]
```

Definition at line 5791 of file [pre.c](#).

4.19.6.217 DoPrompt()

```
int DoPrompt (
    UBYTE * s ) [extern]
```

Definition at line 5506 of file [pre.c](#).

4.19.6.218 DoSetExternal()

```
int DoSetExternal (
    UBYTE * s ) [extern]
```

Definition at line 5539 of file [pre.c](#).

4.19.6.219 DoSetExternalAttr()

```
int DoSetExternalAttr (
    UBYTE * s ) [extern]
```

Definition at line 5609 of file [pre.c](#).

4.19.6.220 DoRmExternal()

```
int DoRmExternal (
    UBYTE * s ) [extern]
```

Definition at line 5726 of file [pre.c](#).

4.19.6.221 DoFactDollar()

```
int DoFactDollar (
    UBYTE * s ) [extern]
```

Definition at line 6633 of file [pre.c](#).

4.19.6.222 GetDollarNumber()

```
WORD GetDollarNumber (
    UBYTE ** inp,
    DOLLARS d ) [extern]
```

Definition at line 6670 of file [pre.c](#).

4.19.6.223 DoSetRandom()

```
int DoSetRandom (
    UBYTE * s ) [extern]
```

Definition at line 6760 of file [pre.c](#).

4.19.6.224 DoOptimize()

```
int DoOptimize (
    UBYTE * s ) [extern]
```

Definition at line 6803 of file [pre.c](#).

4.19.6.225 DoClearOptimize()

```
int DoClearOptimize (
    UBYTE * s ) [extern]
```

Definition at line 6936 of file [pre.c](#).

4.19.6.226 DoSkipExtraSymbols()

```
int DoSkipExtraSymbols (
    UBYTE * s ) [extern]
```

Definition at line 6956 of file [pre.c](#).

4.19.6.227 DoTimeOutAfter()

```
int DoTimeOutAfter (
    UBYTE * s ) [extern]
```

Definition at line 7182 of file [pre.c](#).

4.19.6.228 DoMessage()

```
int DoMessage (
    UBYTE * s ) [extern]
```

Definition at line 3676 of file [pre.c](#).

4.19.6.229 DoPreOut()

```
int DoPreOut (
    UBYTE * s ) [extern]
```

Definition at line 3747 of file [pre.c](#).

4.19.6.230 DoPreAppend()

```
int DoPreAppend (  
    UBYTE * s ) [extern]
```

Definition at line [3804](#) of file [pre.c](#).

4.19.6.231 DoPreCreate()

```
int DoPreCreate (  
    UBYTE * s ) [extern]
```

Definition at line [3847](#) of file [pre.c](#).

4.19.6.232 DoPreAssign()

```
int DoPreAssign (  
    UBYTE * s ) [extern]
```

Definition at line [2128](#) of file [pre.c](#).

4.19.6.233 DoPreBreak()

```
int DoPreBreak (  
    UBYTE * s ) [extern]
```

Definition at line [4047](#) of file [pre.c](#).

4.19.6.234 DoPreDefault()

```
int DoPreDefault (  
    UBYTE * s ) [extern]
```

Definition at line [4110](#) of file [pre.c](#).

4.19.6.235 DoPreSwitch()

```
int DoPreSwitch (  
    UBYTE * s ) [extern]
```

Definition at line [4161](#) of file [pre.c](#).

4.19.6.236 DoPreEndSwitch()

```
int DoPreEndSwitch (  
    UBYTE * s ) [extern]
```

Definition at line [4134](#) of file [pre.c](#).

4.19.6.237 DoPreCase()

```
int DoPreCase (
    UBYTE * s ) [extern]
```

Definition at line 4071 of file [pre.c](#).

4.19.6.238 DoPreShow()

```
int DoPreShow (
    UBYTE * s ) [extern]
```

Definition at line 4215 of file [pre.c](#).

4.19.6.239 DoPreExchange()

```
int DoPreExchange (
    UBYTE * s ) [extern]
```

Definition at line 2490 of file [pre.c](#).

4.19.6.240 DoSystem()

```
int DoSystem (
    UBYTE * s ) [extern]
```

Definition at line 4254 of file [pre.c](#).

4.19.6.241 DoPipe()

```
int DoPipe (
    UBYTE * s ) [extern]
```

Definition at line 3690 of file [pre.c](#).

4.19.6.242 StartPrepro()

```
void StartPrepro (
    void ) [extern]
```

Definition at line 4432 of file [pre.c](#).

4.19.6.243 Dolfdef()

```
int DoIfdef (
    UBYTE * s,
    int par ) [extern]
```

Definition at line 3391 of file [pre.c](#).

4.19.6.244 DoIfydef()

```
int DoIfydef (
    UBYTE * s ) [extern]
```

Definition at line [3416](#) of file [pre.c](#).

4.19.6.245 DoIfndef()

```
int DoIfndef (
    UBYTE * s ) [extern]
```

Definition at line [3426](#) of file [pre.c](#).

4.19.6.246 DoElse()

```
int DoElse (
    UBYTE * s ) [extern]
```

Definition at line [3095](#) of file [pre.c](#).

4.19.6.247 DoElseif()

```
int DoElseif (
    UBYTE * s ) [extern]
```

Definition at line [3132](#) of file [pre.c](#).

4.19.6.248 DoEndif()

```
int DoEndif (
    UBYTE * s ) [extern]
```

Definition at line [3313](#) of file [pre.c](#).

4.19.6.249 DoTerminate()

```
int DoTerminate (
    UBYTE * s ) [extern]
```

Definition at line [2757](#) of file [pre.c](#).

4.19.6.250 DoIf()

```
int DoIf (
    UBYTE * s ) [extern]
```

Definition at line [3367](#) of file [pre.c](#).

4.19.6.251 DoCall()

```
int DoCall (
    UBYTE * s ) [extern]
```

Definition at line 2551 of file [pre.c](#).

4.19.6.252 DoDebug()

```
int DoDebug (
    UBYTE * s ) [extern]
```

Definition at line 2720 of file [pre.c](#).

4.19.6.253 DoContinueDo()

```
int DoContinueDo (
    UBYTE * s ) [extern]
```

Definition at line 2780 of file [pre.c](#).

4.19.6.254 DoDo()

```
int DoDo (
    UBYTE * s ) [extern]
```

Definition at line 2811 of file [pre.c](#).

4.19.6.255 DoBreakDo()

```
int DoBreakDo (
    UBYTE * s ) [extern]
```

Definition at line 3026 of file [pre.c](#).

4.19.6.256 DoEnddo()

```
int DoEnddo (
    UBYTE * s ) [extern]
```

Definition at line 3167 of file [pre.c](#).

4.19.6.257 DoEndprocedure()

```
int DoEndprocedure (
    UBYTE * s ) [extern]
```

Definition at line 3341 of file [pre.c](#).

4.19.6.258 DoInside()

```
int DoInside (
    UBYTE * s ) [extern]
```

Definition at line [3451](#) of file [pre.c](#).

4.19.6.259 DoEndInside()

```
int DoEndInside (
    UBYTE * s ) [extern]
```

Definition at line [3544](#) of file [pre.c](#).

4.19.6.260 DoProcedure()

```
int DoProcedure (
    UBYTE * s ) [extern]
```

Definition at line [4005](#) of file [pre.c](#).

4.19.6.261 DoPrePrintTimes()

```
int DoPrePrintTimes (
    UBYTE * s ) [extern]
```

Definition at line [3770](#) of file [pre.c](#).

4.19.6.262 DoPreWrite()

```
int DoPreWrite (
    UBYTE * s ) [extern]
```

Definition at line [3964](#) of file [pre.c](#).

4.19.6.263 DoPreClose()

```
int DoPreClose (
    UBYTE * s ) [extern]
```

Definition at line [3920](#) of file [pre.c](#).

4.19.6.264 DoPreRemove()

```
int DoPreRemove (
    UBYTE * s ) [extern]
```

Definition at line [3887](#) of file [pre.c](#).

4.19.6.265 DoPreSortReallocate()

```
int DoPreSortReallocate (
    UBYTE * s ) [extern]
```

Definition at line [3784](#) of file [pre.c](#).

4.19.6.266 DoCommentChar()

```
int DoCommentChar (
    UBYTE * s ) [extern]
```

Definition at line [2097](#) of file [pre.c](#).

4.19.6.267 DoPrcExtension()

```
int DoPrcExtension (
    UBYTE * s ) [extern]
```

Definition at line [3713](#) of file [pre.c](#).

4.19.6.268 DoPreReset()

```
int DoPreReset (
    UBYTE * s ) [extern]
```

Definition at line [6995](#) of file [pre.c](#).

4.19.6.269 WriteString()

```
void WriteString (
    int type,
    UBYTE * str,
    int num ) [extern]
```

Definition at line [1807](#) of file [tools.c](#).

4.19.6.270 WriteUnfinString()

```
void WriteUnfinString (
    int type,
    UBYTE * str,
    int num ) [extern]
```

Definition at line [1841](#) of file [tools.c](#).

4.19.6.271 AddToString()

```
UBYTE * AddToString (
    UBYTE * outstring,
    UBYTE * extrastring,
    int par ) [extern]
```

Definition at line [1866](#) of file [tools.c](#).

4.19.6.272 PreCalc()

```
UBYTE * PreCalc (
    void ) [extern]
```

Definition at line [5113](#) of file [pre.c](#).

4.19.6.273 PreEval()

```
UBYTE * PreEval (
    UBYTE * s,
    LONG * x ) [extern]
```

Definition at line [5191](#) of file [pre.c](#).

4.19.6.274 NumToStr()

```
void NumToStr (
    UBYTE * s,
    LONG x ) [extern]
```

Definition at line [1783](#) of file [tools.c](#).

4.19.6.275 PreCmp()

```
int PreCmp (
    int type,
    int val,
    UBYTE * t,
    int type2,
    int val2,
    UBYTE * t2,
    int cmpop ) [extern]
```

Definition at line [4624](#) of file [pre.c](#).

4.19.6.276 PreEq()

```
int PreEq (
    int type,
    int val,
    UBYTE * t,
    int type2,
    int val2,
    UBYTE * t2,
    int egop ) [extern]
```

Definition at line [4648](#) of file [pre.c](#).

4.19.6.277 pParseObject()

```
UBYTE * pParseObject (
    UBYTE * s,
    int * type,
    LONG * val2 ) [extern]
```

Definition at line [4677](#) of file [pre.c](#).

4.19.6.278 PreIfEval()

```
UBYTE * PreIfEval (
    UBYTE * s,
    int * value ) [extern]
```

Definition at line [4495](#) of file [pre.c](#).

4.19.6.279 EvalPreIf()

```
int EvalPreIf (
    UBYTE * s ) [extern]
```

Definition at line [4460](#) of file [pre.c](#).

4.19.6.280 PreLoad()

```
int PreLoad (
    PRELOAD * p,
    UBYTE * start,
    UBYTE * stop,
    int mode,
    char * message ) [extern]
```

Definition at line [4294](#) of file [pre.c](#).

4.19.6.281 PreSkip()

```
int PreSkip (
    UBYTE * start,
    UBYTE * stop,
    int mode ) [extern]
```

Definition at line [4377](#) of file [pre.c](#).

4.19.6.282 EndOfToken()

```
UBYTE * EndOfToken (
    UBYTE * s ) [extern]
```

Definition at line [1919](#) of file [tools.c](#).

4.19.6.283 SetSpecialMode()

```
void SetSpecialMode (
    int moduletype,
    int specialtype ) [extern]
```

Definition at line [257](#) of file [module.c](#).

4.19.6.284 MakeGlobal()

```
void MakeGlobal (
    void ) [extern]
```

Definition at line [382](#) of file [execute.c](#).

4.19.6.285 ExecModule()

```
int ExecModule (
    int moduletype ) [extern]
```

Definition at line [274](#) of file [module.c](#).

4.19.6.286 ExecStore()

```
int ExecStore (
    void ) [extern]
```

Definition at line [299](#) of file [module.c](#).

4.19.6.287 FullCleanUp()

```
void FullCleanUp (
    void ) [extern]
```

Definition at line 314 of file [module.c](#).

4.19.6.288 DoExecStatement()

```
int DoExecStatement (
    void ) [extern]
```

Definition at line 780 of file [module.c](#).

4.19.6.289 DoPipeStatement()

```
int DoPipeStatement (
    void ) [extern]
```

Definition at line 797 of file [module.c](#).

4.19.6.290 DoPolyfun()

```
int DoPolyfun (
    UBYTE * s ) [extern]
```

Definition at line 395 of file [module.c](#).

4.19.6.291 DoPolyratfun()

```
int DoPolyratfun (
    UBYTE * s ) [extern]
```

Definition at line 458 of file [module.c](#).

4.19.6.292 CompileStatement()

```
int CompileStatement (
    UBYTE * in ) [extern]
```

Definition at line 554 of file [compiler.c](#).

4.19.6.293 ToToken()

```
UBYTE * ToToken (
    UBYTE * s ) [extern]
```

Definition at line 1931 of file [tools.c](#).

4.19.6.294 GetDollar()

```
int GetDollar (
    UBYTE * name ) [extern]
```

Definition at line [590](#) of file [names.c](#).

4.19.6.295 MesWork()

```
int MesWork (
    void ) [extern]
```

Definition at line [89](#) of file [message.c](#).

4.19.6.296 MesCall()

```
int MesCall (
    char * s ) [extern]
```

Definition at line [814](#) of file [message.c](#).

4.19.6.297 NumCopy()

```
UBYTE * NumCopy (
    WORD y,
    UBYTE * to ) [extern]
```

Definition at line [1997](#) of file [tools.c](#).

4.19.6.298 LongCopy()

```
char * LongCopy (
    LONG y,
    char * to ) [extern]
```

Definition at line [2024](#) of file [tools.c](#).

4.19.6.299 LongLongCopy()

```
char * LongLongCopy (
    off_t * y,
    char * to ) [extern]
```

Definition at line [2051](#) of file [tools.c](#).

4.19.6.300 ReserveTempFiles()

```
void ReserveTempFiles (
    int par ) [extern]
```

Definition at line [666](#) of file [startup.c](#).

4.19.6.301 PrintTerm()

```
void PrintTerm (
    WORD * term,
    char * where ) [extern]
```

Definition at line [857](#) of file [message.c](#).

4.19.6.302 PrintTermC()

```
void PrintTermC (
    WORD * term,
    char * where ) [extern]
```

Definition at line [887](#) of file [message.c](#).

4.19.6.303 PrintSubTerm()

```
void PrintSubTerm (
    WORD * term,
    char * where ) [extern]
```

Definition at line [921](#) of file [message.c](#).

4.19.6.304 PrintWords()

```
void PrintWords (
    WORD * buffer,
    LONG number ) [extern]
```

Definition at line [943](#) of file [message.c](#).

4.19.6.305 PrintSeq()

```
void PrintSeq (
    WORD * a,
    char * text ) [extern]
```

Definition at line [961](#) of file [message.c](#).

4.19.6.306 ExpandTripleDots()

```
int ExpandTripleDots (
    int par ) [extern]
```

Definition at line 1595 of file [pre.c](#).

4.19.6.307 ComPress()

```
LONG ComPress (
    WORD ** ss,
    LONG * n ) [extern]
```

Gets a list of pointers to terms and compresses the terms. In *n* it collects the number of terms and the return value of the function is the space that is occupied.

We have to pay some special attention to the compression of terms with a PolyFun. This PolyFun should occur only straight before the coefficient, so we can use the same trick as for the coefficient to sabotage compression of this object (Replace in the history the function pointer by zero. This is safe, because terms that would be identical otherwise would have been added).

Parameters

<i>ss</i>	Array of pointers to terms to be compressed.
<i>n</i>	Number of pointers in <i>ss</i> .

Returns

Total number of words needed for the compressed result.

Definition at line 3196 of file [sort.c](#).

Referenced by [EndSort\(\)](#), and [StoreTerm\(\)](#).

4.19.6.308 StageSort()

```
void StageSort (
    FILEHANDLE * fout ) [extern]
```

Prepares a stage 4 or higher sort. Stage 4 sorts occur when the sort file contains more patches than can be merged in one pass.

Definition at line 4654 of file [sort.c](#).

References [FiLe::handle](#).

Referenced by [EndSort\(\)](#), and [MergePatches\(\)](#).

4.19.6.309 M_free()

```
void M_free (
    void * x,
    const char * where ) [extern]
```

Definition at line 2383 of file [tools.c](#).

4.19.6.310 ClearWildcardNames()

```
void ClearWildcardNames (
    void ) [extern]
```

Definition at line 849 of file [names.c](#).

4.19.6.311 AddWildcardName()

```
int AddWildcardName (
    UBYTE * name ) [extern]
```

Definition at line 854 of file [names.c](#).

4.19.6.312 GetWildcardName()

```
int GetWildcardName (
    UBYTE * name ) [extern]
```

Definition at line 898 of file [names.c](#).

4.19.6.313 Globalize()

```
void Globalize (
    int par ) [extern]
```

Definition at line 3155 of file [names.c](#).

4.19.6.314 ResetVariables()

```
void ResetVariables (
    int par ) [extern]
```

Definition at line 2832 of file [names.c](#).

4.19.6.315 AddToPreTypes()

```
void AddToPreTypes (
    int type ) [extern]
```

Definition at line 5341 of file [pre.c](#).

4.19.6.316 MessPreNesting()

```
void MessPreNesting (
    int par ) [extern]
```

Definition at line [5359](#) of file [pre.c](#).

4.19.6.317 GetStreamPosition()

```
LONG GetStreamPosition (
    STREAM * stream ) [extern]
```

Definition at line [813](#) of file [tools.c](#).

4.19.6.318 DoubleCbuffer()

```
WORD * DoubleCbuffer (
    int num,
    WORD * w,
    int par ) [extern]
```

Doubles a compiler buffer.

Parameters

<i>num</i>	The ID number for the buffer to be doubled.
<i>w</i>	The pointer to the end (exclusive) of the current buffer. The contents in the range of [cbuff[num].Buffer,w) will be kept.

Definition at line [143](#) of file [comtool.c](#).

References [CbUf::Buffer](#), [CbUf::BufferSize](#), [CbUf::lhs](#), [CbUf::Pointer](#), [CbUf::rhs](#), and [CbUf::Top](#).

Referenced by [AddNtoC\(\)](#), and [AddNtoL\(\)](#).

4.19.6.319 AddLHS()

```
WORD * AddLHS (
    int num ) [extern]
```

Adds an LHS to a compiler buffer and returns the pointer to a buffer for the new LHS.

Parameters

<i>num</i>	The ID number for the buffer to get another LHS.
------------	--

Definition at line [188](#) of file [comtool.c](#).

References [CbUf::lhs](#), and [CbUf::Pointer](#).

Referenced by [AddNtoL\(\)](#).

4.19.6.320 AddRHS()

```
WORD * AddRHS (
    int num,
    int type ) [extern]
```

Adds an RHS to a compiler buffer and returns the pointer to a buffer for the new RHS.

Parameters

<i>num</i>	The ID number for the buffer to get another RHS.
<i>type</i>	If 0, the subexpression tree will be reallocated.

Definition at line 214 of file [comtool.c](#).

References [TaBIEs::buffers](#), [TaBIEs::buffersfill](#), [TaBIEs::bufferssize](#), [TaBIEs::bufnum](#), [CbUf::CanCommu](#), [CbUf::dimension](#), [inicbufs\(\)](#), [CbUf::numdum](#), [CbUf::NumTerms](#), [CbUf::Pointer](#), and [CbUf::rhs](#).

Referenced by [InsertArg\(\)](#), [StartVariables\(\)](#), and [TestMatch\(\)](#).

Here is the call graph for this function:



4.19.6.321 AddNtoL()

```
int AddNtoL (
    int n,
    WORD * array ) [extern]
```

Adds an LHS with the given data to the current compiler buffer.

Parameters

<i>n</i>	The length of the data.
<i>array</i>	The data to be added.

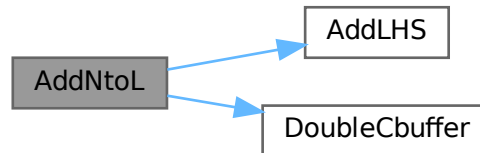
Returns

0 if succeeds.

Definition at line 288 of file [comtool.c](#).

References [AddLHS\(\)](#), [DoubleCbuffer\(\)](#), [CbUf::Pointer](#), and [CbUf::Top](#).

Here is the call graph for this function:



4.19.6.322 AddNtoC()

```

int AddNtoC (
    int bufnum,
    int n,
    WORD * array,
    int par ) [extern]
  
```

Adds the given data to the last LHS/RHS in a compiler buffer.

Parameters

<i>bufnum</i>	The ID number for the buffer where the data will be added.
<i>n</i>	The length of the data.
<i>array</i>	The data to be added.

Returns

0 if succeeds.

Definition at line 317 of file [comtool.c](#).

References [DoubleCbuffer\(\)](#), [CbUf::Pointer](#), and [CbUf::Top](#).

Referenced by [InsertArg\(\)](#), and [StartVariables\(\)](#).

Here is the call graph for this function:



4.19.6.323 DoubleIfBuffers()

```
void DoubleIfBuffers (
    void ) [extern]
```

Definition at line [1033](#) of file [if.c](#).

4.19.6.324 CreateStream()

```
STREAM * CreateStream (
    UBYTE * where ) [extern]
```

Definition at line [782](#) of file [tools.c](#).

4.19.6.325 setonoff()

```
int setonoff (
    UBYTE * s,
    int * flag,
    int onvalue,
    int offvalue ) [extern]
```

Definition at line [490](#) of file [compcomm.c](#).

4.19.6.326 DoPrint()

```
int DoPrint (
    UBYTE * s,
    int par ) [extern]
```

Definition at line [1041](#) of file [compcomm.c](#).

4.19.6.327 SetExpr()

```
int SetExpr (
    UBYTE * s,
    int setunset,
    int par ) [extern]
```

Definition at line [1512](#) of file [compcomm.c](#).

4.19.6.328 AddToCom()

```
void AddToCom (
    int n,
    WORD * array ) [extern]
```

Definition at line [1649](#) of file [compcomm.c](#).

4.19.6.329 Add2ComStrings()

```
int Add2ComStrings (
    int n,
    WORD * array,
    UBYTE * string1,
    UBYTE * string2 ) [extern]
```

Definition at line 1723 of file [compcomm.c](#).

4.19.6.330 DoSymmetrize()

```
int DoSymmetrize (
    UBYTE * s,
    int par ) [extern]
```

Definition at line 2217 of file [compcomm.c](#).

4.19.6.331 DoArgument()

```
int DoArgument (
    UBYTE * s,
    int par ) [extern]
```

Definition at line 1862 of file [compcomm.c](#).

4.19.6.332 ArgFactorize()

```
int ArgFactorize (
    PHEAD WORD * argin,
    WORD * argout ) [extern]
```

Definition at line 2060 of file [argument.c](#).

4.19.6.333 TakeArgContent()

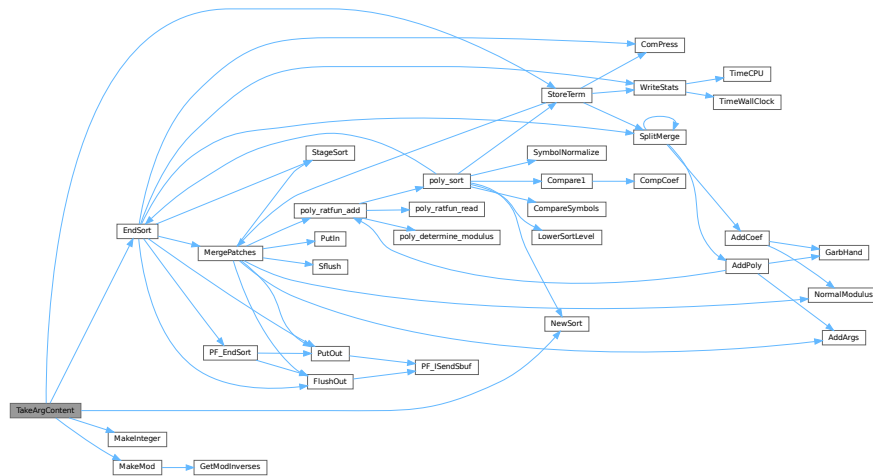
```
WORD * TakeArgContent (
    PHEAD WORD * argin,
    WORD * argout ) [extern]
```

Implements part of the old ExecArg in which we take common factors from arguments with more than one term. The common pieces are put in argout as a sequence of arguments. The part with the multiple terms that are now relative prime is put in argfree which is allocated via TermMalloc and is given as the return value. The difference with the old code is that negative powers are always removed. Hence it is as in MakeInteger in which only numerators will be left: now only zero or positive powers will be remaining.

Definition at line 2766 of file [argument.c](#).

References [EndSort\(\)](#), [MakeInteger\(\)](#), [MakeMod\(\)](#), [NewSort\(\)](#), and [StoreTerm\(\)](#).

Here is the call graph for this function:



4.19.6.334 MakeInteger()

```
WORD * MakeInteger (
    PHEAD WORD * argin,
    WORD * argout,
    WORD * argfree ) [extern]
```

For normalizing everything to integers we have to determine for all elements of this argument the LCM of the denominators and the GCD of the numerators. The input argument is in argin. The number that comes out should go to argout. The new pointer in the argout buffer is the return value. The normalized argument is in argfree.

Definition at line 3312 of file [argument.c](#).

Referenced by [TakeArgContent\(\)](#).

4.19.6.335 MakeMod()

```
WORD * MakeMod (
    PHEAD WORD * argin,
    WORD * argout,
    WORD * argfree ) [extern]
```

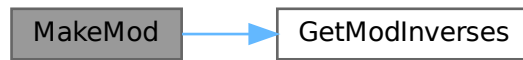
Similar to MakeInteger but now with modulus arithmetic using only a one WORD 'prime'. We make the coefficient of the first term in the argument equal to one. Already the coefficients are taken modulus AN.cmod and AN.ncmod == 1

Definition at line 3483 of file [argument.c](#).

References [GetModInverses\(\)](#).

Referenced by [TakeArgContent\(\)](#).

Here is the call graph for this function:



4.19.6.336 FindArg()

```
WORD FindArg (
    PHEAD WORD * a ) [extern]
```

Looks the argument up in the (workers) table. If it is found the number in the table is returned (plus one to make it positive). If it is not found we look in the compiler provided table. If it is found - the number in the table is returned (minus one to make it negative). If in neither table we return zero.

Definition at line 2513 of file [argument.c](#).

4.19.6.337 InsertArg()

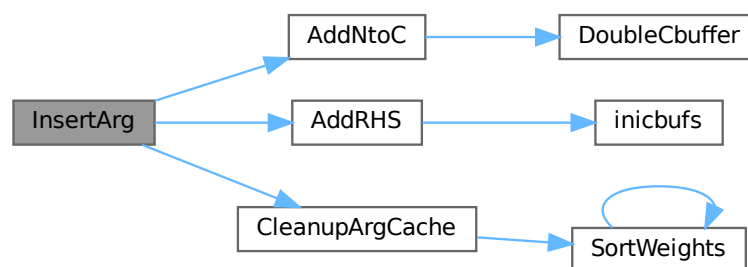
```
WORD InsertArg (
    PHEAD WORD * argin,
    WORD * argout,
    int par ) [extern]
```

Inserts the argument into the (workers) table. If the table is too full we eliminate half of it. The eliminated elements are the ones that have not been used most recently, weighted by their total use and age(?). If par == 0 it inserts in the regular factorization cache If par == 1 it inserts in the cache defined with the FactorCache statement

Definition at line 2537 of file [argument.c](#).

References [AddNtoC\(\)](#), [AddRHS\(\)](#), and [CleanupArgCache\(\)](#).

Here is the call graph for this function:



4.19.6.338 CleanupArgCache()

```
int CleanupArgCache (
    PHEAD WORD bufnum ) [extern]
```

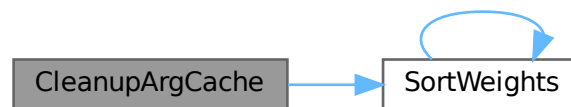
Cleans up the argument factorization cache. We throw half the elements. For a weight of what we want to keep we use the product of usage and the number in the buffer.

Definition at line 2572 of file [argument.c](#).

References [CbUf::boomlijst](#), [CbUf::Buffer](#), [CbUf::Pointer](#), [CbUf::rhs](#), [SortWeights\(\)](#), and [tree::usage](#).

Referenced by [InsertArg\(\)](#).

Here is the call graph for this function:



4.19.6.339 ArgSymbolMerge()

```
int ArgSymbolMerge (
    WORD * t1,
    WORD * t2 ) [extern]
```

Definition at line 2643 of file [argument.c](#).

4.19.6.340 ArgDotproductMerge()

```
int ArgDotproductMerge (
    WORD * t1,
    WORD * t2 ) [extern]
```

Definition at line 2698 of file [argument.c](#).

4.19.6.341 SortWeights()

```
void SortWeights (
    LONG * weights,
    LONG * extraspace,
    WORD number ) [extern]
```

Sorts an array of LONGS in the same way SplitMerge (in [sort.c](#)) works We use gradual division in two.

Definition at line [3528](#) of file [argument.c](#).

References [SortWeights\(\)](#).

Referenced by [CleanupArgCache\(\)](#), and [SortWeights\(\)](#).

Here is the call graph for this function:



4.19.6.342 DoBrackets()

```
int DoBrackets (
    UBYTE * inp,
    int par ) [extern]
```

Definition at line [3752](#) of file [compcomm.c](#).

4.19.6.343 DoPutInside()

```
int DoPutInside (
    UBYTE * inp,
    int par ) [extern]
```

Definition at line [7047](#) of file [compcomm.c](#).

4.19.6.344 CountComp()

```
WORD * CountComp (
    UBYTE * inp,
    WORD * to ) [extern]
```

Definition at line [4030](#) of file [compcomm.c](#).

4.19.6.345 CoAntiBracket()

```
int CoAntiBracket (
    UBYTE * inp ) [extern]
```

Definition at line 3887 of file [compcomm.c](#).

4.19.6.346 CoAntiSymmetrize()

```
int CoAntiSymmetrize (
    UBYTE * s ) [extern]
```

Definition at line 2344 of file [compcomm.c](#).

4.19.6.347 DoArgPlode()

```
int DoArgPlode (
    UBYTE * s,
    int par ) [extern]
```

Definition at line 5707 of file [compcomm.c](#).

4.19.6.348 CoArgExplode()

```
int CoArgExplode (
    UBYTE * s ) [extern]
```

Definition at line 5763 of file [compcomm.c](#).

4.19.6.349 CoArgImplode()

```
int CoArgImplode (
    UBYTE * s ) [extern]
```

Definition at line 5770 of file [compcomm.c](#).

4.19.6.350 CoArgument()

```
int CoArgument (
    UBYTE * s ) [extern]
```

Definition at line 2096 of file [compcomm.c](#).

4.19.6.351 CoInside()

```
int CoInside (
    UBYTE * s ) [extern]
```

Definition at line 2129 of file [compcomm.c](#).

4.19.6.352 ExecInside()

```
int ExecInside (
    UBYTE * s ) [extern]
```

Definition at line 1681 of file [dollar.c](#).

4.19.6.353 CoInExpression()

```
int CoInExpression (
    UBYTE * s ) [extern]
```

Definition at line 3357 of file [compcomm.c](#).

4.19.6.354 CoInParallel()

```
int CoInParallel (
    UBYTE * s ) [extern]
```

Definition at line 3263 of file [compcomm.c](#).

4.19.6.355 CoNotInParallel()

```
int CoNotInParallel (
    UBYTE * s ) [extern]
```

Definition at line 3273 of file [compcomm.c](#).

4.19.6.356 DoInParallel()

```
int DoInParallel (
    UBYTE * s,
    int par ) [extern]
```

Definition at line 3288 of file [compcomm.c](#).

4.19.6.357 CoEndInExpression()

```
int CoEndInExpression (
    UBYTE * s ) [extern]
```

Definition at line 3410 of file [compcomm.c](#).

4.19.6.358 CoBracket()

```
int CoBracket (
    UBYTE * inp ) [extern]
```

Definition at line 3879 of file [compcomm.c](#).

4.19.6.359 CoPutInside()

```
int CoPutInside (
    UBYTE * inp ) [extern]
```

Definition at line 7044 of file [compcomm.c](#).

4.19.6.360 CoAntiPutInside()

```
int CoAntiPutInside (
    UBYTE * inp ) [extern]
```

Definition at line 7045 of file [compcomm.c](#).

4.19.6.361 CoMultiBracket()

```
int CoMultiBracket (
    UBYTE * inp ) [extern]
```

Definition at line 3898 of file [compcomm.c](#).

4.19.6.362 CoCFunction()

```
int CoCFunction (
    UBYTE * s ) [extern]
```

Definition at line 1625 of file [names.c](#).

4.19.6.363 CoCTensor()

```
int CoCTensor (
    UBYTE * s ) [extern]
```

Definition at line 1627 of file [names.c](#).

4.19.6.364 CoCollect()

```
int CoCollect (
    UBYTE * s ) [extern]
```

Definition at line 428 of file [compcomm.c](#).

4.19.6.365 CoCompress()

```
int CoCompress (
    UBYTE * s ) [extern]
```

Definition at line 506 of file [compcomm.c](#).

4.19.6.366 CoContract()

```
int CoContract (
    UBYTE * s ) [extern]
```

Definition at line 1787 of file [compcomm.c](#).

4.19.6.367 CoCycleSymmetrize()

```
int CoCycleSymmetrize (
    UBYTE * s ) [extern]
```

Definition at line 2351 of file [compcomm.c](#).

4.19.6.368 CoDelete()

```
int CoDelete (
    UBYTE * s ) [extern]
```

Definition at line 942 of file [compcomm.c](#).

4.19.6.369 CoTableBase()

```
int CoTableBase (
    UBYTE * s ) [extern]
```

Definition at line 627 of file [tables.c](#).

4.19.6.370 CoApply()

```
int CoApply (
    UBYTE * s ) [extern]
```

Definition at line 1796 of file [tables.c](#).

4.19.6.371 CoDenominators()

```
int CoDenominators (
    UBYTE * s ) [extern]
```

Definition at line 5859 of file [compcomm.c](#).

4.19.6.372 CoDimension()

```
int CoDimension (
    UBYTE * s ) [extern]
```

Definition at line 1226 of file [names.c](#).

4.19.6.373 CoDiscard()

```
int CoDiscard (
    UBYTE * s ) [extern]
```

Definition at line 1764 of file [compcomm.c](#).

4.19.6.374 CoDisorder()

```
int CoDisorder (
    UBYTE * inp ) [extern]
```

Definition at line 417 of file [comexpr.c](#).

4.19.6.375 CoDrop()

```
int CoDrop (
    UBYTE * s ) [extern]
```

Definition at line 1557 of file [compcomm.c](#).

4.19.6.376 CoDropCoefficient()

```
int CoDropCoefficient (
    UBYTE * s ) [extern]
```

Definition at line 5888 of file [compcomm.c](#).

4.19.6.377 CoDropSymbols()

```
int CoDropSymbols (
    UBYTE * s ) [extern]
```

Definition at line 5902 of file [compcomm.c](#).

4.19.6.378 CoElse()

```
int CoElse (
    UBYTE * p ) [extern]
```

Definition at line 4862 of file [compcomm.c](#).

4.19.6.379 CoElseif()

```
int CoElseif (
    UBYTE * inp ) [extern]
```

Definition at line 4889 of file [compcomm.c](#).

4.19.6.380 CoEndArgument()

```
int CoEndArgument (
    UBYTE * s ) [extern]
```

Definition at line [2103](#) of file [compcomm.c](#).

4.19.6.381 CoEndInside()

```
int CoEndInside (
    UBYTE * s ) [extern]
```

Definition at line [2136](#) of file [compcomm.c](#).

4.19.6.382 CoEndIf()

```
int CoEndIf (
    UBYTE * inp ) [extern]
```

Definition at line [4916](#) of file [compcomm.c](#).

4.19.6.383 CoEndRepeat()

```
int CoEndRepeat (
    UBYTE * inp ) [extern]
```

Definition at line [3714](#) of file [compcomm.c](#).

4.19.6.384 CoEndTerm()

```
int CoEndTerm (
    UBYTE * s ) [extern]
```

Definition at line [5265](#) of file [compcomm.c](#).

4.19.6.385 CoEndWhile()

```
int CoEndWhile (
    UBYTE * inp ) [extern]
```

Definition at line [4982](#) of file [compcomm.c](#).

4.19.6.386 CoExit()

```
int CoExit (
    UBYTE * s ) [extern]
```

Definition at line [3236](#) of file [compcomm.c](#).

4.19.6.387 CoFactArg()

```
int CoFactArg (
    UBYTE * s ) [extern]
```

Definition at line 2197 of file [compcomm.c](#).

4.19.6.388 CoFactDollar()

```
int CoFactDollar (
    UBYTE * inp ) [extern]
```

Definition at line 6418 of file [compcomm.c](#).

4.19.6.389 CoFactorize()

```
int CoFactorize (
    UBYTE * s ) [extern]
```

Definition at line 6447 of file [compcomm.c](#).

4.19.6.390 CoNFactorize()

```
int CoNFactorize (
    UBYTE * s ) [extern]
```

Definition at line 6454 of file [compcomm.c](#).

4.19.6.391 CoUnFactorize()

```
int CoUnFactorize (
    UBYTE * s ) [extern]
```

Definition at line 6461 of file [compcomm.c](#).

4.19.6.392 CoNUnFactorize()

```
int CoNUnFactorize (
    UBYTE * s ) [extern]
```

Definition at line 6468 of file [compcomm.c](#).

4.19.6.393 DoFactorize()

```
int DoFactorize (
    UBYTE * s,
    int par ) [extern]
```

Definition at line 6475 of file [compcomm.c](#).

4.19.6.394 CoFill()

```
int CoFill (
    UBYTE * inp ) [extern]
```

Definition at line [1070](#) of file [comexpr.c](#).

4.19.6.395 CoFillExpression()

```
int CoFillExpression (
    UBYTE * inp ) [extern]
```

Definition at line [1356](#) of file [comexpr.c](#).

4.19.6.396 CoFixIndex()

```
int CoFixIndex (
    UBYTE * s ) [extern]
```

Definition at line [999](#) of file [compcomm.c](#).

4.19.6.397 CoFormat()

```
int CoFormat (
    UBYTE * s ) [extern]
```

Definition at line [153](#) of file [compcomm.c](#).

4.19.6.398 CoGlobal()

```
int CoGlobal (
    UBYTE * inp ) [extern]
```

Definition at line [73](#) of file [comexpr.c](#).

4.19.6.399 CoGlobalFactorized()

```
int CoGlobalFactorized (
    UBYTE * inp ) [extern]
```

Definition at line [87](#) of file [comexpr.c](#).

4.19.6.400 CoGoTo()

```
int CoGoTo (
    UBYTE * inp ) [extern]
```

Definition at line [1816](#) of file [compcomm.c](#).

4.19.6.401 Cold()

```
int CoId (
    UBYTE * inp ) [extern]
```

Definition at line 395 of file [comexpr.c](#).

4.19.6.402 ColdNew()

```
int CoIdNew (
    UBYTE * inp ) [extern]
```

Definition at line 406 of file [comexpr.c](#).

4.19.6.403 ColdOld()

```
int CoIdOld (
    UBYTE * inp ) [extern]
```

Definition at line 384 of file [comexpr.c](#).

4.19.6.404 Colf()

```
int CoIf (
    UBYTE * inp ) [extern]
```

Definition at line 4220 of file [compcomm.c](#).

4.19.6.405 ColfMatch()

```
int CoIfMatch (
    UBYTE * inp ) [extern]
```

Definition at line 450 of file [comexpr.c](#).

4.19.6.406 ColfNoMatch()

```
int CoIfNoMatch (
    UBYTE * inp ) [extern]
```

Definition at line 461 of file [comexpr.c](#).

4.19.6.407 ColIndex()

```
int CoIndex (
    UBYTE * s ) [extern]
```

Definition at line 1112 of file [names.c](#).

4.19.6.408 CoInsideFirst()

```
int CoInsideFirst (
    UBYTE * s ) [extern]
```

Definition at line 928 of file [compcomm.c](#).

4.19.6.409 CoKeep()

```
int CoKeep (
    UBYTE * s ) [extern]
```

Definition at line 987 of file [compcomm.c](#).

4.19.6.410 CoLabel()

```
int CoLabel (
    UBYTE * inp ) [extern]
```

Definition at line 1835 of file [compcomm.c](#).

4.19.6.411 CoLoad()

```
int CoLoad (
    UBYTE * inp ) [extern]
```

Definition at line 573 of file [store.c](#).

4.19.6.412 CoLocal()

```
int CoLocal (
    UBYTE * inp ) [extern]
```

Definition at line 66 of file [comexpr.c](#).

4.19.6.413 CoLocalFactorized()

```
int CoLocalFactorized (
    UBYTE * inp ) [extern]
```

Definition at line 80 of file [comexpr.c](#).

4.19.6.414 CoMany()

```
int CoMany (
    UBYTE * inp ) [extern]
```

Definition at line 428 of file [comexpr.c](#).

4.19.6.415 CoMerge()

```
int CoMerge (
    UBYTE * inp ) [extern]
```

Definition at line [5548](#) of file [compcomm.c](#).

4.19.6.416 CoStuffle()

```
int CoStuffle (
    UBYTE * inp ) [extern]
```

Definition at line [5604](#) of file [compcomm.c](#).

4.19.6.417 CoMetric()

```
int CoMetric (
    UBYTE * s ) [extern]
```

Definition at line [1033](#) of file [compcomm.c](#).

4.19.6.418 CoModOption()

```
int CoModOption (
    UBYTE * s ) [extern]
```

Definition at line [212](#) of file [module.c](#).

4.19.6.419 CoModuleOption()

```
int CoModuleOption (
    UBYTE * s ) [extern]
```

Definition at line [143](#) of file [module.c](#).

4.19.6.420 CoModulus()

```
int CoModulus (
    UBYTE * inp ) [extern]
```

Definition at line [3553](#) of file [compcomm.c](#).

4.19.6.421 CoMulti()

```
int CoMulti (
    UBYTE * inp ) [extern]
```

Definition at line [439](#) of file [comexpr.c](#).

4.19.6.422 CoMultiply()

```
int CoMultiply (
    UBYTE * inp ) [extern]
```

Definition at line 1031 of file [comexpr.c](#).

4.19.6.423 CoNFunction()

```
int CoNFunction (
    UBYTE * s ) [extern]
```

Definition at line 1624 of file [names.c](#).

4.19.6.424 CoNPrint()

```
int CoNPrint (
    UBYTE * s ) [extern]
```

Definition at line 1271 of file [compcomm.c](#).

4.19.6.425 CoNTensor()

```
int CoNTensor (
    UBYTE * s ) [extern]
```

Definition at line 1626 of file [names.c](#).

4.19.6.426 CoNWrite()

```
int CoNWrite (
    UBYTE * s ) [extern]
```

Definition at line 2390 of file [compcomm.c](#).

4.19.6.427 CoNoDrop()

```
int CoNoDrop (
    UBYTE * s ) [extern]
```

Definition at line 1564 of file [compcomm.c](#).

4.19.6.428 CoNoSkip()

```
int CoNoSkip (
    UBYTE * s ) [extern]
```

Definition at line 1578 of file [compcomm.c](#).

4.19.6.429 CoNormalize()

```
int CoNormalize (
    UBYTE * s ) [extern]
```

Definition at line [2162](#) of file [compcomm.c](#).

4.19.6.430 CoMakeInteger()

```
int CoMakeInteger (
    UBYTE * s ) [extern]
```

Definition at line [2169](#) of file [compcomm.c](#).

4.19.6.431 CoFlags()

```
int CoFlags (
    UBYTE * s,
    int value ) [extern]
```

Definition at line [555](#) of file [compcomm.c](#).

4.19.6.432 CoOff()

```
int CoOff (
    UBYTE * s ) [extern]
```

Definition at line [590](#) of file [compcomm.c](#).

4.19.6.433 CoOn()

```
int CoOn (
    UBYTE * s ) [extern]
```

Definition at line [656](#) of file [compcomm.c](#).

4.19.6.434 CoOnce()

```
int CoOnce (
    UBYTE * inp ) [extern]
```

Definition at line [472](#) of file [comexpr.c](#).

4.19.6.435 CoOnly()

```
int CoOnly (
    UBYTE * inp ) [extern]
```

Definition at line [483](#) of file [comexpr.c](#).

4.19.6.436 CoOptimizeOption()

```
int CoOptimizeOption (  
    UBYTE * s ) [extern]
```

Definition at line [6608](#) of file [compcomm.c](#).

4.19.6.437 CoPolyFun()

```
int CoPolyFun (  
    UBYTE * s ) [extern]
```

Definition at line [5331](#) of file [compcomm.c](#).

4.19.6.438 CoPolyRatFun()

```
int CoPolyRatFun (  
    UBYTE * s ) [extern]
```

Definition at line [5378](#) of file [compcomm.c](#).

4.19.6.439 CoPrint()

```
int CoPrint (  
    UBYTE * s ) [extern]
```

Definition at line [1257](#) of file [compcomm.c](#).

4.19.6.440 CoPrintB()

```
int CoPrintB (  
    UBYTE * s ) [extern]
```

Definition at line [1264](#) of file [compcomm.c](#).

4.19.6.441 CoProperCount()

```
int CoProperCount (  
    UBYTE * s ) [extern]
```

Definition at line [935](#) of file [compcomm.c](#).

4.19.6.442 CoUnitTrace()

```
int CoUnitTrace (  
    UBYTE * s ) [extern]
```

Definition at line [5182](#) of file [compcomm.c](#).

4.19.6.443 CoRCycleSymmetrize()

```
int CoRCycleSymmetrize (
    UBYTE * s ) [extern]
```

Definition at line [2358](#) of file [compcomm.c](#).

4.19.6.444 CoRatio()

```
int CoRatio (
    UBYTE * s ) [extern]
```

Definition at line [2417](#) of file [compcomm.c](#).

4.19.6.445 CoRedefine()

```
int CoRedefine (
    UBYTE * s ) [extern]
```

Definition at line [2457](#) of file [compcomm.c](#).

4.19.6.446 CoRenumber()

```
int CoRenumber (
    UBYTE * s ) [extern]
```

Definition at line [2562](#) of file [compcomm.c](#).

4.19.6.447 CoRepeat()

```
int CoRepeat (
    UBYTE * inp ) [extern]
```

Definition at line [3691](#) of file [compcomm.c](#).

4.19.6.448 CoSave()

```
int CoSave (
    UBYTE * inp ) [extern]
```

Definition at line [363](#) of file [store.c](#).

4.19.6.449 CoSelect()

```
int CoSelect (
    UBYTE * inp ) [extern]
```

Definition at line [494](#) of file [comexpr.c](#).

4.19.6.450 CoSet()

```
int CoSet (
    UBYTE * s ) [extern]
```

Definition at line [2355](#) of file [names.c](#).

4.19.6.451 CoSetExitFlag()

```
int CoSetExitFlag (
    UBYTE * s ) [extern]
```

Definition at line [3436](#) of file [compcomm.c](#).

4.19.6.452 CoSkip()

```
int CoSkip (
    UBYTE * s ) [extern]
```

Definition at line [1571](#) of file [compcomm.c](#).

4.19.6.453 CoProcessBucket()

```
int CoProcessBucket (
    UBYTE * s ) [extern]
```

Definition at line [5659](#) of file [compcomm.c](#).

4.19.6.454 CoPushHide()

```
int CoPushHide (
    UBYTE * s ) [extern]
```

Definition at line [1278](#) of file [compcomm.c](#).

4.19.6.455 CoPopHide()

```
int CoPopHide (
    UBYTE * s ) [extern]
```

Definition at line [1322](#) of file [compcomm.c](#).

4.19.6.456 CoHide()

```
int CoHide (
    UBYTE * inp ) [extern]
```

Definition at line [1585](#) of file [compcomm.c](#).

4.19.6.457 CoIntoHide()

```
int CoIntoHide (
    UBYTE * inp ) [extern]
```

Definition at line 1603 of file [compcomm.c](#).

4.19.6.458 CoNoIntoHide()

```
int CoNoIntoHide (
    UBYTE * inp ) [extern]
```

Definition at line 1621 of file [compcomm.c](#).

4.19.6.459 CoNoHide()

```
int CoNoHide (
    UBYTE * inp ) [extern]
```

Definition at line 1628 of file [compcomm.c](#).

4.19.6.460 CoUnHide()

```
int CoUnHide (
    UBYTE * inp ) [extern]
```

Definition at line 1635 of file [compcomm.c](#).

4.19.6.461 CoNoUnHide()

```
int CoNoUnHide (
    UBYTE * inp ) [extern]
```

Definition at line 1642 of file [compcomm.c](#).

4.19.6.462 CoSort()

```
int CoSort (
    UBYTE * s ) [extern]
```

Definition at line 5292 of file [compcomm.c](#).

4.19.6.463 CoSplitArg()

```
int CoSplitArg (
    UBYTE * s ) [extern]
```

Definition at line 2176 of file [compcomm.c](#).

4.19.6.464 CoSplitFirstArg()

```
int CoSplitFirstArg (  
    UBYTE * s )    [extern]
```

Definition at line [2183](#) of file [compcomm.c](#).

4.19.6.465 CoSplitLastArg()

```
int CoSplitLastArg (  
    UBYTE * s )    [extern]
```

Definition at line [2190](#) of file [compcomm.c](#).

4.19.6.466 CoSum()

```
int CoSum (  
    UBYTE * s )    [extern]
```

Definition at line [2583](#) of file [compcomm.c](#).

4.19.6.467 CoSymbol()

```
int CoSymbol (  
    UBYTE * s )    [extern]
```

Definition at line [946](#) of file [names.c](#).

4.19.6.468 CoSymmetrize()

```
int CoSymmetrize (  
    UBYTE * s )    [extern]
```

Definition at line [2337](#) of file [compcomm.c](#).

4.19.6.469 DoTable()

```
int DoTable (  
    UBYTE * s,  
    int par )    [extern]
```

Definition at line [1666](#) of file [names.c](#).

4.19.6.470 CoTable()

```
int CoTable (  
    UBYTE * s )    [extern]
```

Definition at line [2048](#) of file [names.c](#).

4.19.6.471 CoTerm()

```
int CoTerm (
    UBYTE * s ) [extern]
```

Definition at line 5217 of file [compcomm.c](#).

4.19.6.472 CoNTable()

```
int CoNTable (
    UBYTE * s ) [extern]
```

Definition at line 2058 of file [names.c](#).

4.19.6.473 CoCTable()

```
int CoCTable (
    UBYTE * s ) [extern]
```

Definition at line 2068 of file [names.c](#).

4.19.6.474 EmptyTable()

```
void EmptyTable (
    TABLES T ) [extern]
```

Definition at line 2078 of file [names.c](#).

4.19.6.475 CoToTensor()

```
int CoToTensor (
    UBYTE * s ) [extern]
```

Definition at line 2703 of file [compcomm.c](#).

4.19.6.476 CoToVector()

```
int CoToVector (
    UBYTE * s ) [extern]
```

Definition at line 2871 of file [compcomm.c](#).

4.19.6.477 CoTrace4()

```
int CoTrace4 (
    UBYTE * s ) [extern]
```

Definition at line 2941 of file [compcomm.c](#).

4.19.6.478 CoTraceN()

```
int CoTraceN (  
    UBYTE * s ) [extern]
```

Definition at line 3028 of file [compcomm.c](#).

4.19.6.479 CoChisholm()

```
int CoChisholm (  
    UBYTE * s ) [extern]
```

Definition at line 3088 of file [compcomm.c](#).

4.19.6.480 CoTransform()

```
int CoTransform (  
    UBYTE * in ) [extern]
```

Definition at line 87 of file [transform.c](#).

4.19.6.481 CoClearTable()

```
int CoClearTable (  
    UBYTE * s ) [extern]
```

Definition at line 5777 of file [compcomm.c](#).

4.19.6.482 DoChain()

```
int DoChain (  
    UBYTE * s,  
    int option ) [extern]
```

Definition at line 3170 of file [compcomm.c](#).

4.19.6.483 CoChainin()

```
int CoChainin (  
    UBYTE * s ) [extern]
```

Definition at line 3214 of file [compcomm.c](#).

4.19.6.484 CoChainout()

```
int CoChainout (  
    UBYTE * s ) [extern]
```

Definition at line 3226 of file [compcomm.c](#).

4.19.6.485 CoTryReplace()

```
int CoTryReplace (
    UBYTE * p ) [extern]
```

Definition at line [3450](#) of file [compcomm.c](#).

4.19.6.486 CoVector()

```
int CoVector (
    UBYTE * s ) [extern]
```

Definition at line [1267](#) of file [names.c](#).

4.19.6.487 CoWhile()

```
int CoWhile (
    UBYTE * inp ) [extern]
```

Definition at line [4961](#) of file [compcomm.c](#).

4.19.6.488 CoWrite()

```
int CoWrite (
    UBYTE * s ) [extern]
```

Definition at line [2365](#) of file [compcomm.c](#).

4.19.6.489 CoAuto()

```
int CoAuto (
    UBYTE * inp ) [extern]
```

Definition at line [2572](#) of file [names.c](#).

4.19.6.490 CoSwitch()

```
int CoSwitch (
    UBYTE * s ) [extern]
```

Definition at line [7167](#) of file [compcomm.c](#).

4.19.6.491 CoCase()

```
int CoCase (
    UBYTE * s ) [extern]
```

Definition at line [7213](#) of file [compcomm.c](#).

4.19.6.492 CoBreak()

```
int CoBreak (
    UBYTE * s ) [extern]
```

Definition at line [7263](#) of file [compcomm.c](#).

4.19.6.493 CoDefault()

```
int CoDefault (
    UBYTE * s ) [extern]
```

Definition at line [7289](#) of file [compcomm.c](#).

4.19.6.494 CoEndSwitch()

```
int CoEndSwitch (
    UBYTE * s ) [extern]
```

Definition at line [7316](#) of file [compcomm.c](#).

4.19.6.495 CoTBaddto()

```
int CoTBaddto (
    UBYTE * s ) [extern]
```

Definition at line [801](#) of file [tables.c](#).

4.19.6.496 CoTBaudit()

```
int CoTBaudit (
    UBYTE * s ) [extern]
```

Definition at line [1473](#) of file [tables.c](#).

4.19.6.497 CoTbcleanup()

```
int CoTbcleanup (
    UBYTE * s ) [extern]
```

Definition at line [1575](#) of file [tables.c](#).

4.19.6.498 CoTBcreate()

```
int CoTBcreate (
    UBYTE * s ) [extern]
```

Definition at line [756](#) of file [tables.c](#).

4.19.6.499 CoTBenter()

```
int CoTBenter (
    UBYTE * s ) [extern]
```

Definition at line 974 of file [tables.c](#).

4.19.6.500 CoTBhelp()

```
int CoTBhelp (
    UBYTE * s ) [extern]
```

Definition at line 1883 of file [tables.c](#).

4.19.6.501 CoTBload()

```
int CoTBload (
    UBYTE * ss ) [extern]
```

Definition at line 1252 of file [tables.c](#).

4.19.6.502 CoTBoff()

```
int CoTBoff (
    UBYTE * s ) [extern]
```

Definition at line 1544 of file [tables.c](#).

4.19.6.503 CoTBon()

```
int CoTBon (
    UBYTE * s ) [extern]
```

Definition at line 1513 of file [tables.c](#).

4.19.6.504 CoTBopen()

```
int CoTBopen (
    UBYTE * s ) [extern]
```

Definition at line 772 of file [tables.c](#).

4.19.6.505 CoTBreplace()

```
int CoTBreplace (
    UBYTE * s ) [extern]
```

Definition at line 1587 of file [tables.c](#).

4.19.6.506 CoTBuse()

```
int CoTBuse (
    UBYTE * s ) [extern]
```

Definition at line 1602 of file [tables.c](#).

4.19.6.507 CoTestUse()

```
int CoTestUse (
    UBYTE * s ) [extern]
```

Definition at line 1133 of file [tables.c](#).

4.19.6.508 CoThreadBucket()

```
int CoThreadBucket (
    UBYTE * s ) [extern]
```

Definition at line 5677 of file [compcomm.c](#).

4.19.6.509 AddComString()

```
int AddComString (
    int n,
    WORD * array,
    UBYTE * thestring,
    int par ) [extern]
```

Definition at line 1664 of file [compcomm.c](#).

4.19.6.510 CompileAlgebra()

```
int CompileAlgebra (
    UBYTE * s,
    int leftright,
    WORD * prototype ) [extern]
```

Definition at line 518 of file [compiler.c](#).

4.19.6.511 IsIdStatement()

```
int IsIdStatement (
    UBYTE * s ) [extern]
```

Definition at line 504 of file [compiler.c](#).

4.19.6.512 IsRHS()

```
UBYTE * IsRHS (
    UBYTE * s,
    UBYTE c ) [extern]
```

Definition at line 458 of file [compiler.c](#).

4.19.6.513 ParenthesesTest()

```
int ParenthesesTest (
    UBYTE * sin ) [extern]
```

Definition at line 380 of file [compiler.c](#).

4.19.6.514 tokenize()

```
int tokenize (
    UBYTE * in,
    WORD leftright ) [extern]
```

Definition at line 58 of file [token.c](#).

4.19.6.515 WriteTokens()

```
void WriteTokens (
    SBYTE * in ) [extern]
```

Definition at line 657 of file [token.c](#).

4.19.6.516 simp1token()

```
int simp1token (
    SBYTE * s ) [extern]
```

Definition at line 705 of file [token.c](#).

4.19.6.517 simpwtoken()

```
int simpwtoken (
    SBYTE * s ) [extern]
```

Definition at line 794 of file [token.c](#).

4.19.6.518 simp2token()

```
int simp2token (
    SBYTE * s ) [extern]
```

Definition at line 948 of file [token.c](#).

4.19.6.519 simp3atoken()

```
int simp3atoken (
    SBYTE * s,
    int mode ) [extern]
```

Definition at line 1196 of file [token.c](#).

4.19.6.520 simp3btoken()

```
int simp3btoken (
    SBYTE * s,
    int mode ) [extern]
```

Definition at line 1437 of file [token.c](#).

4.19.6.521 simp4token()

```
int simp4token (
    SBYTE * s ) [extern]
```

Definition at line 1822 of file [token.c](#).

4.19.6.522 simp5token()

```
int simp5token (
    SBYTE * s,
    int mode ) [extern]
```

Definition at line 1982 of file [token.c](#).

4.19.6.523 simp6token()

```
int simp6token (
    SBYTE * tokens,
    int mode ) [extern]
```

Definition at line 2028 of file [token.c](#).

4.19.6.524 SkipAName()

```
UBYTE * SkipAName (
    UBYTE * s ) [extern]
```

Definition at line 430 of file [compiler.c](#).

4.19.6.525 TestTables()

```
int TestTables (
    void ) [extern]
```

Definition at line 669 of file [compiler.c](#).

4.19.6.526 GetLabel()

```
int GetLabel (
    UBYTE * name ) [extern]
```

Definition at line 2787 of file [names.c](#).

4.19.6.527 ColdExpression()

```
int CoIdExpression (
    UBYTE * inp,
    int type ) [extern]
```

Definition at line 507 of file [comexpr.c](#).

4.19.6.528 CoAssign()

```
int CoAssign (
    UBYTE * inp ) [extern]
```

Definition at line 1924 of file [comexpr.c](#).

4.19.6.529 DoExpr()

```
int DoExpr (
    UBYTE * inp,
    int type,
    int par ) [extern]
```

Definition at line 96 of file [comexpr.c](#).

4.19.6.530 CompileSubExpressions()

```
int CompileSubExpressions (
    SBYTE * tokens ) [extern]
```

Definition at line 713 of file [compiler.c](#).

4.19.6.531 CodeGenerator()

```
int CodeGenerator (
    SBYTE * tokens ) [extern]
```

Definition at line 848 of file [compiler.c](#).

4.19.6.532 CompleteTerm()

```
int CompleteTerm (
    WORD * term,
    UWORD * numer,
    UWORD * denom,
    WORD nnum,
    WORD nden,
    int sign ) [extern]
```

Definition at line 1911 of file [compiler.c](#).

4.19.6.533 CodeFactors()

```
int CodeFactors (
    SBYTE * s ) [extern]
```

Definition at line 1948 of file [compiler.c](#).

4.19.6.534 GenerateFactors()

```
WORD GenerateFactors (
    WORD n,
    WORD inc ) [extern]
```

Definition at line 2245 of file [compiler.c](#).

4.19.6.535 InsTree()

```
int InsTree (
    int bufnum,
    int h ) [extern]
```

Definition at line 355 of file [comtool.c](#).

4.19.6.536 FindTree()

```
int FindTree (
    int bufnum,
    WORD * subexpr ) [extern]
```

Definition at line 533 of file [comtool.c](#).

4.19.6.537 RedoTree()

```
void RedoTree (
    CBUF * C,
    int size ) [extern]
```

Definition at line 569 of file [comtool.c](#).

4.19.6.538 ClearTree()

```
void ClearTree (
    int i ) [extern]
```

Definition at line 588 of file [comtool.c](#).

4.19.6.539 CatchDollar()

```
int CatchDollar (
    int par ) [extern]
```

Definition at line 54 of file [dollar.c](#).

4.19.6.540 AssignDollar()

```
int AssignDollar (
    PHEAD WORD * term,
    WORD level ) [extern]
```

Definition at line 209 of file [dollar.c](#).

4.19.6.541 WriteDollarToBuffer()

```
UBYTE * WriteDollarToBuffer (
    WORD numdollar,
    WORD par ) [extern]
```

Definition at line 596 of file [dollar.c](#).

4.19.6.542 WriteDollarFactorToBuffer()

```
UBYTE * WriteDollarFactorToBuffer (
    WORD numdollar,
    WORD numfac,
    WORD par ) [extern]
```

Definition at line 687 of file [dollar.c](#).

4.19.6.543 AddToDollarBuffer()

```
void AddToDollarBuffer (
    UBYTE * s ) [extern]
```

Definition at line 762 of file [dollar.c](#).

4.19.6.544 PutTermInDollar()

```
int PutTermInDollar (
    WORD * term,
    WORD numdollar ) [extern]
```

Definition at line 863 of file [dollar.c](#).

4.19.6.545 TermAssign()

```
void TermAssign (
    WORD * term ) [extern]
```

Definition at line 803 of file [dollar.c](#).

4.19.6.546 WildDollars()

```
void WildDollars (
    PHEAD WORD * term ) [extern]
```

Definition at line 891 of file [dollar.c](#).

4.19.6.547 numcommute()

```
LONG numcommute (
    WORD * terms,
    LONG * numterms ) [extern]
```

Definition at line 659 of file [comtool.c](#).

4.19.6.548 FullRenumber()

```
int FullRenumber (
    PHEAD WORD * term,
    WORD par ) [extern]
```

Definition at line 324 of file [reshuf.c](#).

4.19.6.549 Lus()

```
int Lus (
    WORD * term,
    WORD funnum,
    WORD loopsize,
    WORD numargs,
    WORD outfun,
    WORD mode ) [extern]
```

Definition at line 57 of file [lus.c](#).

4.19.6.550 FindLus()

```
int FindLus (
    int from,
    int level,
    int openindex ) [extern]
```

Definition at line 515 of file [lus.c](#).

4.19.6.551 CoReplaceLoop()

```
int CoReplaceLoop (
    UBYTE * inp ) [extern]
```

Definition at line 5135 of file [compcomm.c](#).

4.19.6.552 CoFindLoop()

```
int CoFindLoop (
    UBYTE * inp ) [extern]
```

Definition at line 5127 of file [compcomm.c](#).

4.19.6.553 DoFindLoop()

```
int DoFindLoop (
    UBYTE * inp,
    int mode ) [extern]
```

Definition at line 5010 of file [compcomm.c](#).

4.19.6.554 CoFunPowers()

```
int CoFunPowers (
    UBYTE * inp ) [extern]
```

Definition at line 5155 of file [compcomm.c](#).

4.19.6.555 SortTheList()

```
int SortTheList (
    int * slist,
    int num ) [extern]
```

Definition at line 578 of file [lus.c](#).

4.19.6.556 MatchIsPossible()

```
int MatchIsPossible (
    WORD * pattern,
    WORD * term ) [extern]
```

Definition at line 299 of file [smart.c](#).

4.19.6.557 StudyPattern()

```
int StudyPattern (
    WORD * lhs ) [extern]
```

Definition at line 62 of file [smart.c](#).

4.19.6.558 DoIToTensor()

```
WORD DoIToTensor (
    PHEAD WORD ) [extern]
```

Definition at line 1082 of file [dollar.c](#).

4.19.6.559 DoIToFunction()

```
WORD DoIToFunction (
    PHEAD WORD ) [extern]
```

Definition at line 1143 of file [dollar.c](#).

4.19.6.560 DoIToVector()

```
WORD DoIToVector (
    PHEAD WORD ) [extern]
```

Definition at line 1200 of file [dollar.c](#).

4.19.6.561 DoIToNumber()

```
WORD DoIToNumber (
    PHEAD WORD ) [extern]
```

Definition at line 1264 of file [dollar.c](#).

4.19.6.562 DoIToSymbol()

```
WORD DoIToSymbol (
    PHEAD WORD ) [extern]
```

Definition at line 1323 of file [dollar.c](#).

4.19.6.563 DoIToIndex()

```
WORD DoIToIndex (
    PHEAD WORD ) [extern]
```

Definition at line 1377 of file [dollar.c](#).

4.19.6.564 DoIToLong()

```
LONG DoIToLong (
    PHEAD WORD ) [extern]
```

Definition at line 1606 of file [dollar.c](#).

4.19.6.565 DollarFactorize()

```
int DollarFactorize (
    PHEAD WORD ) [extern]
```

Definition at line 2955 of file [dollar.c](#).

4.19.6.566 CoPrintTable()

```
int CoPrintTable (
    UBYTE * inp ) [extern]
```

Definition at line 1748 of file [comexpr.c](#).

4.19.6.567 CoDeallocateTable()

```
int CoDeallocateTable (
    UBYTE * inp ) [extern]
```

Definition at line 1982 of file [comexpr.c](#).

4.19.6.568 CleanDollarFactors()

```
void CleanDollarFactors (
    DOLLARS d ) [extern]
```

Definition at line 3554 of file [dollar.c](#).

4.19.6.571 MakeDollarMod()

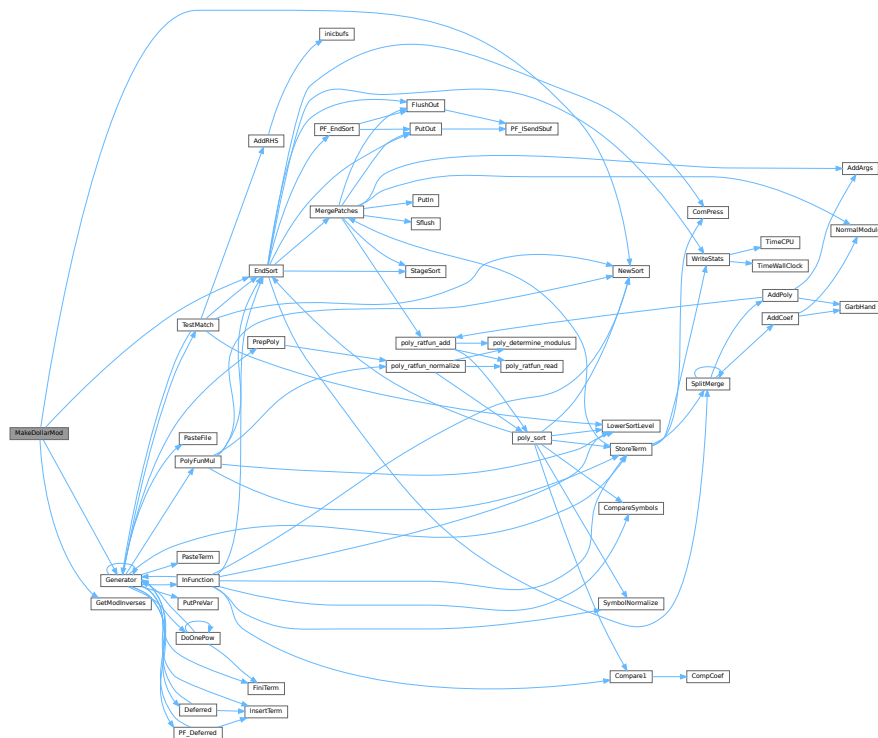
```
WORD * MakeDollarMod (
    PHEAD WORD * buffer,
    WORD ** bufout ) [extern]
```

Similar to `MakeDollarInteger` but now with modulus arithmetic using only a one WORD 'prime'. We make the coefficient of the first term in the argument equal to one. Already the coefficients are taken modulus `AN.cmod` and `AN.ncmod == 1`

Definition at line 3801 of file dollar.c.

References [EndSort\(\)](#), [Generator\(\)](#), [GetModInverses\(\)](#), and [NewSort\(\)](#).

Here is the call graph for this function:



4.19.6.572 GetDolNum()

```
int GetDolNum (
    PHEAD WORD * t,
    WORD * tstop ) [extern]
```

Definition at line 3846 of file dollar.c.

4.19.6.573 AddPotModdollar()

```
void AddPotModdollar (
    WORD numdollar ) [extern]
```

Adds a `$-variable` specified by *numdollar* to the list of potentially modified `$-variables` unless it has already been included in the list.

Parameters

<i>numdollar</i>	The index of the \$-variable to be added.
------------------	---

Definition at line 3959 of file [dollar.c](#).

4.19.6.574 Optimize()

```
int Optimize (
    WORD exprnr,
    int do_print ) [extern]
```

Optimization of expression

4.19.7 Description

This method takes an input expression and generates optimized code to calculate its value. The following methods are called to do so:

- (1) `get_expression` : to read to expression
- (2) `get_brackets` : find brackets for simultaneous optimization
- (3) `occurrence_order` or `find_Horner_MCTS` : to determine (the) Horner scheme(s) to use; this depends on `AO.optimize.horner`
- (4) `optimize_expression_given_Horner` : to do the optimizations for each Horner scheme; this method does either CSE or greedy optimizations dependings on `AO.optimize.method`
- (5) `generate_output` : to format the output in Form notation and store it in a buffer
- (6a) `optimize_print_code` : to print the expression (for "Print") or (6b) `generate_expression` : to modify the expression (for "#Optimize")

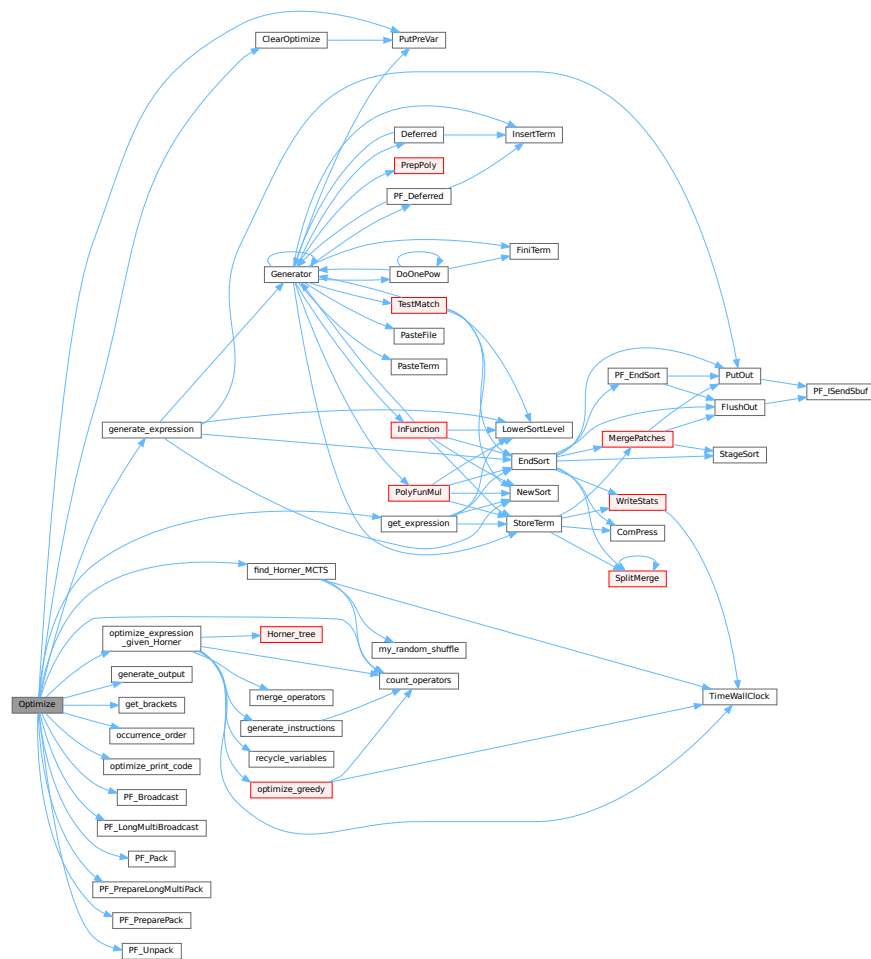
On ParFORM, all the processes must call this function at the same time. Then

- (1) Because only the master can access to the expression to be optimized, the master broadcast the expression to all the slaves after reading the expression (`PF_get_expression`).
- (2) `get_brackets` reads `optimize_expr` as the input and it works also on the slaves. We leave it although the bracket information is not needed on the slaves (used in (5) on the master).
- (3) and (4) `find_Horner_MCTS` and `optimize_expression_given_Horner` are parallelized.
- (5), (6a) and (6b) are needed only on the master.

Definition at line 4638 of file [optimize.cc](#).

References [ClearOptimize\(\)](#), [count_operators\(\)](#), [find_Horner_MCTS\(\)](#), [generate_expression\(\)](#), [generate_output\(\)](#), [get_brackets\(\)](#), [get_expression\(\)](#), [occurrence_order\(\)](#), [optimize_expression_given_Horner\(\)](#), [optimize_print_code\(\)](#), [PF_Broadcast\(\)](#), [PF_LongMultiBroadcast\(\)](#), [PF_Pack\(\)](#), [PF_PrepareLongMultiPack\(\)](#), [PF_PreparePack\(\)](#), [PF_Unpack\(\)](#), and [PutPreVar\(\)](#).

Here is the call graph for this function:



4.19.7.1 ClearOptimize()

```
int ClearOptimize (
    void ) [extern]
```

Optimization of expression

4.19.8 Description

Clears the buffers that were used for optimization output. Clears the expression from the buffers (marks it to be dropped). Note: we need to use the expression by its name, because numbers may change if we drop other expressions between when we do the optimizations and clear the results (in [execute.c](#)). Also this is not 100% safe, because we could overwrite the optimized expression. But that can be done only in a Local or Global statement and hence we only have to test there that we might have to call ClearOptimize first. (in file [comexpr.c](#))

Definition at line 4975 of file [optimize.cc](#).

References [PutPreVar\(\)](#).

Referenced by [Optimize\(\)](#).

Here is the call graph for this function:



4.19.8.1 TakeLongRoot()

```
int TakeLongRoot (
    UWORD * a,
    WORD * n,
    WORD power ) [extern]
```

Definition at line [2735](#) of file [reken.c](#).

4.19.8.2 TakeRatRoot()

```
int TakeRatRoot (
    UWORD * a,
    WORD * n,
    WORD power ) [extern]
```

Definition at line [680](#) of file [reken.c](#).

4.19.8.3 MakeRational()

```
int MakeRational (
    WORD a,
    WORD m,
    WORD * b,
    WORD * c ) [extern]
```

Definition at line [2855](#) of file [reken.c](#).

4.19.8.4 MakeLongRational()

```
int MakeLongRational (
    PHEAD UWORD * a,
    WORD na,
    UWORD * m,
    WORD nm,
    UWORD * b,
    WORD * nb ) [extern]
```

Definition at line [2903](#) of file [reken.c](#).

4.19.8.5 ClearTableTree()

```
void ClearTableTree (
    TABLES T ) [extern]
```

Definition at line 72 of file [tables.c](#).

4.19.8.6 InsTableTree()

```
int InsTableTree (
    TABLES T,
    WORD * tp ) [extern]
```

Definition at line 103 of file [tables.c](#).

4.19.8.7 RedoTableTree()

```
void RedoTableTree (
    TABLES T,
    int newsize ) [extern]
```

Definition at line 266 of file [tables.c](#).

4.19.8.8 FindTableTree()

```
int FindTableTree (
    TABLES T,
    WORD * tp,
    int inc ) [extern]
```

Definition at line 291 of file [tables.c](#).

4.19.8.9 finishcbuf()

```
void finishcbuf (
    WORD num ) [extern]
```

Frees a compiler buffer.

Parameters

<i>num</i>	The ID number for the buffer to be freed.
------------	---

Definition at line 89 of file [comtool.c](#).

References [CbUf::boomlijst](#), [CbUf::Buffer](#), [CbUf::BufferSize](#), [CbUf::CanCommu](#), [CbUf::lhs](#), [CbUf::NumTerms](#), [CbUf::Pointer](#), [CbUf::rhs](#), and [CbUf::Top](#).

Referenced by [PF_BroadcastCBuf\(\)](#).

4.19.8.10 clearcbuf()

```
void clearcbuf (
    WORD num ) [extern]
```

Clears contents in a compiler buffer.

Parameters

<i>num</i>	The ID number for the buffer to be cleared.
------------	---

Definition at line 116 of file [comtool.c](#).

References [CbUf::boomlijst](#), [CbUf::Buffer](#), [CbUf::Pointer](#), and [CbUf::rhs](#).

4.19.8.11 CleanUpSort()

```
void CleanUpSort (
    int num ) [extern]
```

Partially or completely frees function sort buffers.

Definition at line 4845 of file [sort.c](#).

4.19.8.12 AllocFileHandle()

```
FILEHANDLE * AllocFileHandle (
    WORD par,
    char * name ) [extern]
```

Definition at line 1044 of file [setfile.c](#).

4.19.8.13 DeAllocFileHandle()

```
void DeAllocFileHandle (
    FILEHANDLE * fh ) [extern]
```

Definition at line 1105 of file [setfile.c](#).

4.19.8.14 LowerSortLevel()

```
void LowerSortLevel (
    void ) [extern]
```

Lowers the level in the sort system.

Definition at line 4945 of file [sort.c](#).

Referenced by [generate_expression\(\)](#), [get_expression\(\)](#), [InFunction\(\)](#), [PF_CollectModifiedDollars\(\)](#), [PF_Processor\(\)](#), [poly_factorize_expression\(\)](#), [poly_sort\(\)](#), [poly_unfactorize_expression\(\)](#), [PolyFunMul\(\)](#), [Processor\(\)](#), and [TestMatch\(\)](#).

4.19.8.15 PolyRatFunSpecial()

```
WORD * PolyRatFunSpecial (
    PHEAD WORD * t1,
    WORD * t2 ) [extern]
```

Definition at line [4962](#) of file [sort.c](#).

4.19.8.16 SimpleSplitMergeRec()

```
void SimpleSplitMergeRec (
    WORD * array,
    WORD num,
    WORD * auxarray ) [extern]
```

Definition at line [5064](#) of file [sort.c](#).

4.19.8.17 SimpleSplitMerge()

```
void SimpleSplitMerge (
    WORD * array,
    WORD num ) [extern]
```

Definition at line [5092](#) of file [sort.c](#).

4.19.8.18 BinarySearch()

```
WORD BinarySearch (
    WORD * array,
    WORD num,
    WORD x ) [extern]
```

Definition at line [5110](#) of file [sort.c](#).

4.19.8.19 InsideDollar()

```
int InsideDollar (
    PHEAD WORD * ll,
    WORD level ) [extern]
```

Definition at line [1746](#) of file [dollar.c](#).

4.19.8.20 DolToTerms()

```
DOLLARS DolToTerms (
    PHEAD WORD ) [extern]
```

Definition at line [1458](#) of file [dollar.c](#).

4.19.8.21 EvalDoLoopArg()

```
WORD EvalDoLoopArg (
    PHEAD WORD * arg,
    WORD par ) [extern]
```

Evaluates one argument of a do loop. Such an argument is constructed from SNUMBERS DOLLAREXPRES-
SIONs and possibly DOLLAREXPR2s which indicate factors of the preceding dollar. Hence we have SNUM-
BER,num DOLLAREXPRESSSION,numdollar DOLLAREXPRESSSION,numdollar,DOLLAREXPR2,numfactor DOL-
LAREXPRESSSION,numdollar,DOLLAREXPR2,numfactor,DOLLAREXPR2,numfactor etc. Because we have a do-
loop at every stage we should have a number. The notation in DOLLAREXPR2 is that ≥ 0 is number of yet another
dollar and < 0 is $-n-1$ with n the array element or zero. The return value is the (short) number. The routine works its
way through the list in a recursive manner.

Definition at line 2651 of file [dollar.c](#).

References [EvalDoLoopArg\(\)](#).

Referenced by [EvalDoLoopArg\(\)](#).

Here is the call graph for this function:



4.19.8.22 SetExprCases()

```
int SetExprCases (
    int par,
    int setunset,
    int val ) [extern]
```

Definition at line 1357 of file [compcomm.c](#).

4.19.8.23 TestSelect()

```
int TestSelect (
    WORD * term,
    WORD * setp ) [extern]
```

Definition at line 1748 of file [pattern.c](#).

4.19.8.24 SubsInAll()

```
void SubsInAll (
    PHEAD0 ) [extern]
```

Definition at line 1982 of file [pattern.c](#).

4.19.8.25 TransferBuffer()

```
void TransferBuffer (
    int from,
    int to,
    int spectator ) [extern]
```

Definition at line 2143 of file [pattern.c](#).

4.19.8.26 TakeIDfunction()

```
int TakeIDfunction (
    PHEAD WORD * term ) [extern]
```

Definition at line 2213 of file [pattern.c](#).

4.19.8.27 MakeSetupAllocs()

```
int MakeSetupAllocs (
    void ) [extern]
```

Definition at line 1122 of file [setfile.c](#).

4.19.8.28 TryFileSetups()

```
int TryFileSetups (
    void ) [extern]
```

Definition at line 1142 of file [setfile.c](#).

4.19.8.29 ExchangeExpressions()

```
void ExchangeExpressions (
    int num1,
    int num2 ) [extern]
```

Definition at line 1670 of file [execute.c](#).

4.19.8.30 ExchangeDollars()

```
void ExchangeDollars (
    int num1,
    int num2 ) [extern]
```

Definition at line 1849 of file [dollar.c](#).

4.19.8.31 GetFirstBracket()

```
int GetFirstBracket (
    WORD * term,
    int num ) [extern]
```

Definition at line 1744 of file [execute.c](#).

4.19.8.32 GetFirstTerm()

```
int GetFirstTerm (
    WORD * term,
    int num,
    int pre ) [extern]
```

Gets the first term of an expression. Routine should be thread safe.

Parameters

out	<i>term</i>	Pointer to the buffer where the term should be written.
in	<i>num</i>	The expression number from which to get the term.
in	<i>pre</i>	Denotes a pre-processor call (1) or not (0). Used to determine the correct source file for the expression.

Returns

First WORD of term, a negative value denotes an error.

Definition at line 1863 of file [execute.c](#).

References [FiLe::handle](#), [VaRrEnUm::lo](#), and [ReNuMbEr::syMb](#).

4.19.8.33 GetContent()

```
int GetContent (
    WORD * content,
    int num ) [extern]
```

Definition at line 1971 of file [execute.c](#).

4.19.8.34 CleanupTerm()

```
int CleanupTerm (
    WORD * term ) [extern]
```

Definition at line 2088 of file [execute.c](#).

4.19.8.35 ContentMerge()

```
WORD ContentMerge (
    PHEAD WORD * content,
    WORD * term ) [extern]
```

Definition at line 2112 of file [execute.c](#).

4.19.8.36 PreIfDollarEval()

```
UBYTE * PreIfDollarEval (
    UBYTE * s,
    int * value ) [extern]
```

Definition at line 1978 of file [dollar.c](#).

4.19.8.37 TermsInDollar()

```
LONG TermsInDollar (
    WORD num ) [extern]
```

Definition at line 1867 of file [dollar.c](#).

4.19.8.38 SizeOfDollar()

```
LONG SizeOfDollar (
    WORD num ) [extern]
```

Definition at line 1916 of file [dollar.c](#).

4.19.8.39 TermsInExpression()

```
LONG TermsInExpression (
    WORD num ) [extern]
```

Definition at line 2376 of file [execute.c](#).

4.19.8.40 SizeOfExpression()

```
LONG SizeOfExpression (
    WORD num ) [extern]
```

Definition at line 2388 of file [execute.c](#).

4.19.8.41 TranslateExpression()

```
WORD * TranslateExpression (
    UBYTE * s ) [extern]
```

Definition at line 2160 of file [dollar.c](#).

4.19.8.42 IsSetMember()

```
int IsSetMember (
    WORD * buffer,
    WORD numset ) [extern]
```

Definition at line 2217 of file [dollar.c](#).

4.19.8.43 IsMultipleOf()

```
int IsMultipleOf (
    WORD * buf1,
    WORD * buf2 ) [extern]
```

Definition at line 2398 of file [dollar.c](#).

4.19.8.44 TwoExprCompare()

```
int TwoExprCompare (
    WORD * buf1,
    WORD * buf2,
    int oprtr ) [extern]
```

Definition at line 2473 of file [dollar.c](#).

4.19.8.45 UpdatePositions()

```
void UpdatePositions (
    void ) [extern]
```

Definition at line 2400 of file [execute.c](#).

4.19.8.46 M_print()

```
void M_print (
    void ) [extern]
```

Definition at line 2517 of file [tools.c](#).

4.19.8.47 M_check1()

```
void M_check1 (
    void ) [extern]
```

Definition at line 2516 of file [tools.c](#).

4.19.8.48 PrintTime()

```
void PrintTime (
    UBYTE * mess ) [extern]
```

Definition at line 784 of file [sch.c](#).

4.19.8.49 FindBracket()

```
POSITION * FindBracket (
    WORD nexp,
    WORD * bracket ) [extern]
```

Definition at line 65 of file [index.c](#).

4.19.8.50 PutBracketInIndex()

```
void PutBracketInIndex (
    PHEAD WORD * term,
    POSITION * newpos ) [extern]
```

Definition at line 331 of file [index.c](#).

4.19.8.51 ClearBracketIndex()

```
void ClearBracketIndex (
    WORD numexp ) [extern]
```

Definition at line 546 of file [index.c](#).

4.19.8.52 OpenBracketIndex()

```
void OpenBracketIndex (
    WORD nexpr ) [extern]
```

Definition at line 578 of file [index.c](#).

4.19.8.53 DoNoParallel()

```
int DoNoParallel (
    UBYTE * s ) [extern]
```

Definition at line 529 of file [module.c](#).

4.19.8.54 DoParallel()

```
int DoParallel (
    UBYTE * s ) [extern]
```

Definition at line 544 of file [module.c](#).

4.19.8.55 DoModSum()

```
int DoModSum (
    UBYTE * s ) [extern]
```

Definition at line 559 of file [module.c](#).

4.19.8.56 DoModMax()

```
int DoModMax (
    UBYTE * s ) [extern]
```

Definition at line 579 of file [module.c](#).

4.19.8.57 DoModMin()

```
int DoModMin (
    UBYTE * s ) [extern]
```

Definition at line 599 of file [module.c](#).

4.19.8.58 DoModLocal()

```
int DoModLocal (
    UBYTE * s ) [extern]
```

Definition at line 619 of file [module.c](#).

4.19.8.59 DoModDollar()

```
UBYTE * DoModDollar (
    UBYTE * s,
    int type ) [extern]
```

Definition at line 657 of file [module.c](#).

4.19.8.60 DoProcessBucket()

```
int DoProcessBucket (
    UBYTE * s ) [extern]
```

Definition at line 639 of file [module.c](#).

4.19.8.61 DoinParallel()

```
int DoinParallel (
    UBYTE * s ) [extern]
```

Definition at line 758 of file [module.c](#).

4.19.8.62 DonotinParallel()

```
int DonotinParallel (
    UBYTE * s ) [extern]
```

Definition at line 768 of file [module.c](#).

4.19.8.63 FlipTable()

```
int FlipTable (
    FUNCTIONS f,
    int type ) [extern]
```

Definition at line 671 of file [tables.c](#).

4.19.8.64 ChainIn()

```
int ChainIn (
    PHEAD WORD * term,
    WORD funnum ) [extern]
```

Definition at line 691 of file [function.c](#).

4.19.8.65 ChainOut()

```
int ChainOut (
    PHEAD WORD * term,
    WORD funnum ) [extern]
```

Definition at line 748 of file [function.c](#).

4.19.8.66 ArgumentImplode()

```
int ArgumentImplode (
    PHEAD WORD * term,
    WORD * thelist ) [extern]
```

Definition at line 1845 of file [argument.c](#).

4.19.8.67 ArgumentExplode()

```
int ArgumentExplode (
    PHEAD WORD * term,
    WORD * thelist ) [extern]
```

Definition at line [1949](#) of file [argument.c](#).

4.19.8.68 DenToFunction()

```
int DenToFunction (
    WORD * term,
    WORD numfun ) [extern]
```

Definition at line [2556](#) of file [wildcard.c](#).

4.19.8.69 HowMany()

```
WORD HowMany (
    PHEAD WORD * ifcode,
    WORD * term ) [extern]
```

Definition at line [840](#) of file [if.c](#).

4.19.8.70 RemoveDollars()

```
void RemoveDollars (
    void ) [extern]
```

Definition at line [3128](#) of file [names.c](#).

4.19.8.71 CountTerms1()

```
LONG CountTerms1 (
    PHEAD0 ) [extern]
```

Definition at line [2461](#) of file [execute.c](#).

4.19.8.72 TermsInBracket()

```
LONG TermsInBracket (
    PHEAD WORD * term,
    WORD level ) [extern]
```

Definition at line [2569](#) of file [execute.c](#).

4.19.8.73 Crash()

```
int Crash (
    void ) [extern]
```

Definition at line 3834 of file [tools.c](#).

4.19.8.74 str_dup()

```
char * str_dup (
    char * str ) [extern]
```

Definition at line 101 of file [minos.c](#).

4.19.8.75 convertblock()

```
void convertblock (
    INDEXBLOCK * in,
    INDEXBLOCK * out,
    int mode ) [extern]
```

Definition at line 120 of file [minos.c](#).

4.19.8.76 convertnamesblock()

```
void convertnamesblock (
    NAMESBLOCK * in,
    NAMESBLOCK * out,
    int mode ) [extern]
```

Definition at line 171 of file [minos.c](#).

4.19.8.77 convertiniinfo()

```
void convertiniinfo (
    INIINFO * in,
    INIINFO * out,
    int mode ) [extern]
```

Definition at line 197 of file [minos.c](#).

4.19.8.78 ReadIndex()

```
int ReadIndex (
    DBASE * d ) [extern]
```

Definition at line 288 of file [minos.c](#).

4.19.8.79 WriteIndexBlock()

```
int WriteIndexBlock (
    DBASE * d,
    MLONG num ) [extern]
```

Definition at line 401 of file [minos.c](#).

4.19.8.80 WriteNamesBlock()

```
int WriteNamesBlock (
    DBASE * d,
    MLONG num ) [extern]
```

Definition at line 422 of file [minos.c](#).

4.19.8.81 WriteIndex()

```
int WriteIndex (
    DBASE * d ) [extern]
```

Definition at line 445 of file [minos.c](#).

4.19.8.82 WriteIniInfo()

```
int WriteIniInfo (
    DBASE * d ) [extern]
```

Definition at line 508 of file [minos.c](#).

4.19.8.83 ReadIniInfo()

```
int ReadIniInfo (
    DBASE * d ) [extern]
```

Definition at line 525 of file [minos.c](#).

4.19.8.84 AddToIndex()

```
int AddToIndex (
    DBASE * d,
    MLONG number ) [extern]
```

Definition at line 1067 of file [minos.c](#).

4.19.8.85 GetDbase()

```
DBASE * GetDbase (
    char * filename,
    MLONG rwmode ) [extern]
```

Definition at line 548 of file [minos.c](#).

4.19.8.86 OpenDbase()

```
DBASE * OpenDbase (
    char * filename ) [extern]
```

Definition at line 822 of file [minos.c](#).

4.19.8.87 ReadObject()

```
char * ReadObject (
    DBASE * d,
    MLONG tablenumber,
    char * arguments ) [extern]
```

Definition at line 1296 of file [minos.c](#).

4.19.8.88 ReadijObject()

```
char * ReadijObject (
    DBASE * d,
    MLONG i,
    MLONG j,
    char * arguments ) [extern]
```

Definition at line 1377 of file [minos.c](#).

4.19.8.89 ExistsObject()

```
int ExistsObject (
    DBASE * d,
    MLONG tablenumber,
    char * arguments ) [extern]
```

Definition at line 1429 of file [minos.c](#).

4.19.8.90 DeleteObject()

```
int DeleteObject (
    DBASE * d,
    MLONG tablenumber,
    char * arguments ) [extern]
```

Definition at line 1461 of file [minos.c](#).

4.19.8.91 WriteObject()

```
int WriteObject (
    DBASE * d,
    MLONG tablename,
    char * arguments,
    char * rhs,
    MLONG number ) [extern]
```

Definition at line 1196 of file [minos.c](#).

4.19.8.92 AddObject()

```
MLONG AddObject (
    DBASE * d,
    MLONG tablename,
    char * arguments,
    char * rhs ) [extern]
```

Definition at line 1154 of file [minos.c](#).

4.19.8.93 NewDbase()

```
DBASE * NewDbase (
    char * name,
    MLONG number ) [extern]
```

Definition at line 604 of file [minos.c](#).

4.19.8.94 FreeTableBase()

```
void FreeTableBase (
    DBASE * d ) [extern]
```

Definition at line 741 of file [minos.c](#).

4.19.8.95 ComposeTableNames()

```
int ComposeTableNames (
    DBASE * d ) [extern]
```

Definition at line 773 of file [minos.c](#).

4.19.8.96 PutTableNames()

```
int PutTableNames (
    DBASE * d ) [extern]
```

Definition at line 971 of file [minos.c](#).

4.19.8.97 AddTableName()

```
MLONG AddTableName (
    DBASE * d,
    char * name,
    TABLES T ) [extern]
```

Definition at line 856 of file [minos.c](#).

4.19.8.98 GetTableName()

```
MLONG GetTableName (
    DBASE * d,
    char * name ) [extern]
```

Definition at line 939 of file [minos.c](#).

4.19.8.99 FindTableNumber()

```
MLONG FindTableNumber (
    DBASE * d,
    char * name ) [extern]
```

Definition at line 1168 of file [minos.c](#).

4.19.8.100 TryEnvironment()

```
int TryEnvironment (
    void ) [extern]
```

Definition at line 1228 of file [setfile.c](#).

4.19.8.101 CopyExpression()

```
int CopyExpression (
    FILEHANDLE * from,
    FILEHANDLE * to ) [extern]
```

Definition at line 3308 of file [store.c](#).

4.19.8.102 set_in()

```
int set_in (
    UBYTE ch,
    set_of_char set ) [extern]
```

Definition at line 2112 of file [tools.c](#).

4.19.8.103 set_set()

```
one_byte set_set (
    UBYTE ch,
    set_of_char set ) [extern]
```

Definition at line 2132 of file [tools.c](#).

4.19.8.104 set_del()

```
one_byte set_del (
    UBYTE ch,
    set_of_char set ) [extern]
```

Definition at line 2153 of file [tools.c](#).

4.19.8.105 set_sub()

```
one_byte set_sub (
    set_of_char set,
    set_of_char set1,
    set_of_char set2 ) [extern]
```

Definition at line 2175 of file [tools.c](#).

4.19.8.106 DoPreAddSeparator()

```
int DoPreAddSeparator (
    UBYTE * s ) [extern]
```

Definition at line 5382 of file [pre.c](#).

4.19.8.107 DoPreRmSeparator()

```
int DoPreRmSeparator (
    UBYTE * s ) [extern]
```

Definition at line 5407 of file [pre.c](#).

4.19.8.108 openExternalChannel()

```
int openExternalChannel (
    UBYTE * cmd,
    int daemonize,
    UBYTE * shellname,
    UBYTE * stderrname ) [extern]
```

Definition at line 230 of file [extcmd.c](#).

4.19.8.109 initPresetExternalChannels()

```
int initPresetExternalChannels (
    UBYTE * theline,
    int thetimeout ) [extern]
```

Definition at line [232](#) of file [extcmd.c](#).

4.19.8.110 closeExternalChannel()

```
int closeExternalChannel (
    int n ) [extern]
```

Definition at line [233](#) of file [extcmd.c](#).

4.19.8.111 selectExternalChannel()

```
int selectExternalChannel (
    int n ) [extern]
```

Definition at line [234](#) of file [extcmd.c](#).

4.19.8.112 getCurrentExternalChannel()

```
int getCurrentExternalChannel (
    void ) [extern]
```

Definition at line [235](#) of file [extcmd.c](#).

4.19.8.113 closeAllExternalChannels()

```
void closeAllExternalChannels (
    void ) [extern]
```

Definition at line [236](#) of file [extcmd.c](#).

4.19.8.114 defineChannel()

```
UBYTE * defineChannel (
    UBYTE * s,
    HANDLERS * h ) [extern]
```

Definition at line [6033](#) of file [pre.c](#).

4.19.8.115 writeToChannel()

```
int writeToChannel (
    int wtype,
    UBYTE * s,
    HANDLERS * h ) [extern]
```

Definition at line 6068 of file [pre.c](#).

4.19.8.116 ReleaseTB()

```
int ReleaseTB (
    void ) [extern]
```

Definition at line 2319 of file [tables.c](#).

4.19.8.117 SymbolNormalize()

```
int SymbolNormalize (
    WORD * term ) [extern]
```

Routine normalizes terms that contain only symbols. Regular minimum and maximum properties are ignored.

We check whether there are negative powers in the output. This is not allowed.

Definition at line 5144 of file [normal.c](#).

Referenced by [InFunction\(\)](#), and [poly_sort\(\)](#).

4.19.8.118 TestFunFlag()

```
int TestFunFlag (
    PHEAD WORD * tfun ) [extern]
```

Definition at line 5240 of file [normal.c](#).

4.19.8.119 CompareSymbols()

```
WORD CompareSymbols (
    PHEAD WORD * term1,
    WORD * term2,
    WORD par ) [extern]
```

Compares the terms, based on the value of AN.polysortflag. If $term1 < term2$ the return value is -1 If $term1 > term2$ the return value is 1 If $term1 = term2$ the return value is 0 The coefficients may differ. The terms contain only a single subterm of type SYMBOL. If AN.polysortflag = 0 it is a 'regular' compare. If AN.polysortflag = 1 the sum of the powers is more important par is a dummy parameter to make the parameter field identical to that of Compare1 which is the regular compare routine in [sort.c](#)

Definition at line 3100 of file [sort.c](#).

Referenced by [InFunction\(\)](#), and [poly_sort\(\)](#).

4.19.8.120 CompareHSymbols()

```
WORD CompareHSymbols (
    PHEAD WORD * term1,
    WORD * term2,
    WORD par ) [extern]
```

Compares terms that can have only SYMBOL and HAAKJE subterms. If $term1 < term2$ the return value is -1 If $term1 > term2$ the return value is 1 If $term1 = term2$ the return value is 0 par is a dummy parameter to make the parameter field identical to that of Compare1 which is the regular compare routine in [sort.c](#)

Definition at line [3143](#) of file [sort.c](#).

4.19.8.121 NextPrime()

```
WORD NextPrime (
    PHEAD WORD num ) [extern]
```

Gives the next prime number in the list of prime numbers.

If the list isn't long enough we expand it. For ease in ParForm and because these lists shouldn't be very big we let each worker keep its own list.

The list is cut off at MAXPOWER, because we don't want to get into trouble that the power of a variable gets larger than the prime number.

Definition at line [3664](#) of file [reken.c](#).

4.19.8.122 Moebius()

```
WORD Moebius (
    PHEAD WORD ) [extern]
```

Definition at line [3728](#) of file [reken.c](#).

4.19.8.123 wranf()

```
UWORD wranf (
    PHEAD0 ) [extern]
```

Definition at line [3868](#) of file [reken.c](#).

4.19.8.124 iranf()

```
UWORD iranf (
    PHEAD UWORD ) [extern]
```

Definition at line [3887](#) of file [reken.c](#).

4.19.8.125 iniwranf()

```
void iniwranf (
    PHEAD0 ) [extern]
```

Definition at line 3819 of file [reken.c](#).

4.19.8.126 PreRandom()

```
UBYTE * PreRandom (
    UBYTE * s ) [extern]
```

Definition at line 3912 of file [reken.c](#).

4.19.8.127 TreatPolyRatFun()

```
int TreatPolyRatFun (
    PHEAD WORD * prf ) [extern]
```

Definition at line 4960 of file [normal.c](#).

4.19.8.128 ReadSaveHeader()

```
WORD ReadSaveHeader (
    void ) [extern]
```

Reads the header in the save file and sets function pointers and flags according to the information found there. Must be called before any other ReadSave... function.

Currently works only for the exchange between 32bit and 64bit machines (WORD size must be 2 or 4 bytes)!

It is called by CoLoad().

Returns

= 0 everything okay, != 0 an error occurred

Definition at line 4065 of file [store.c](#).

4.19.8.129 ReadSaveIndex()

```
WORD ReadSaveIndex (
    FILEINDEX * fileind ) [extern]
```

Reads a FILEINDEX from the open save file specified by AO.SaveData.Handle. Translations for adjusting endianness and data sizes are done if necessary.

Depends on the assumption that sizeof(FILEINDEX) is the same everywhere. If FILEINDEX or INDEXENTRY change, then this functions has to be adjusted.

Called by CoLoad() and FindInIndex().

Parameters

<i>fileind</i>	contains the read FILEINDEX after successful return. must point to allocated, big enough memory.
----------------	--

Returns

= 0 everything okay, != 0 an error occurred

Definition at line 4183 of file [store.c](#).

References [INFILEINDEX](#), [FiLeInDeX::next](#), [FiLeInDeX::number](#), and [FuNcTiOn::number](#).

4.19.8.130 ReadSaveExpression()

```
WORD ReadSaveExpression (
    UBYTE * buffer,
    UBYTE * top,
    LONG * size,
    LONG * outsize ) [extern]
```

Reads an expression from the open file specified by AO.SaveData.Handle. The endianness flip and a resizing without renumbering is done in this function. Thereafter the buffer consists of chunks with a uniform maximal word size (32bit at the moment). The actual renumbering is then done by calling the function [ReadSaveTerm32\(\)](#). The result is returned in *buffer*.

If the translation at some point doesn't fit into the buffer anymore, the function returns and must be called again. In any case *size* returns the number of successfully read bytes, *outsize* returns the number of successfully written bytes, and the file will be positioned at the next byte after the successfully read data.

It is called by PutInStore().

Parameters

<i>buffer</i>	output buffer, holds the (translated) expression
<i>top</i>	end of buffer
<i>size</i>	number of read bytes
<i>outsize</i>	number of written bytes

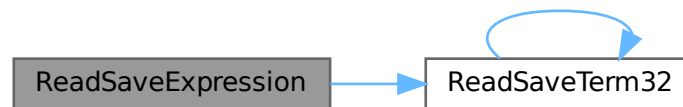
Returns

= 0 everything okay, != 0 an error occurred

Definition at line 5146 of file [store.c](#).

References [ReadSaveTerm32\(\)](#).

Here is the call graph for this function:



4.19.8.131 ReadSaveTerm32()

```

UBYTE * ReadSaveTerm32 (
    UBYTE * bin,
    UBYTE * binend,
    UBYTE ** bout,
    UBYTE * boutend,
    UBYTE * top,
    int terminbuf ) [extern]
  
```

Reads a single term from the given buffer at *bin* and write the translated term back to this buffer at *bout*.

[ReadSaveTerm32\(\)](#) is currently the only instantiation of a [ReadSaveTerm](#)-function. It only deals with data that already has the correct endianness and that is resized to 32bit words but without being renumbered or translated in any other way. It uses the compress buffer `AR.CompressBuffer`.

The function is reentrant in order to cope with nested function arguments. It is called by [ReadSaveExpression\(\)](#) and itself.

The *return value* indicates the position in the input buffer up to which the data has already been successfully processed. The parameter *bout* returns the corresponding position in the output buffer.

Parameters

<i>bin</i>	start of the input buffer
<i>binend</i>	end of the input buffer
<i>bout</i>	as input points to the beginning of the output buffer, as output points behind the already translated data in the output buffer
<i>boutend</i>	end of already decompressed data in output buffer
<i>top</i>	end of output buffer
<i>terminbuf</i>	flag whether decompressed data is already in the output buffer. used in recursive calls

Returns

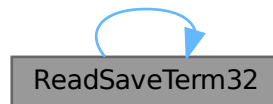
pointer to the next unprocessed data in the input buffer

Definition at line [4736](#) of file [store.c](#).

References [ReadSaveTerm32\(\)](#).

Referenced by [ReadSaveExpression\(\)](#), and [ReadSaveTerm32\(\)](#).

Here is the call graph for this function:



4.19.8.132 ReadSaveVariables()

```
WORD ReadSaveVariables (
    UBYTE * buffer,
    UBYTE * top,
    LONG * size,
    LONG * outsize,
    INDEXENTRY * ind,
    LONG * stage ) [extern]
```

Reads the variables from the open file specified by AO.SaveData.Handle. It reads the **size* bytes and writes them to the **buffer*. It is called by PutInStore().

If translation is necessary, the data might shrink or grow in size, then **size* is adjusted so that the reading and writing fits into the memory from the buffer to the top. The actual number of read bytes is returned in **size*, the number of written bytes is returned in **outsize*.

If the **size* is smaller than the actual size of the variables, this function will be called several times and needs to remember the current position in the variable structure. The parameter *stage* does this job. When [ReadSaveVariables\(\)](#) is called for the first time, this parameter should have the value -1.

The parameter *ind* is used to get the number of variables.

Parameters

<i>buffer</i>	read variables are written into this allocated memory
<i>top</i>	upper end of allocated memory
<i>size</i>	number of bytes to read. might return a smaller number of read bytes if translation was necessary
<i>outsize</i>	if translation has be done, outsize contains the number of written bytes
<i>ind</i>	pointer of INDEXENTRY for the current expression. read-only
<i>stage</i>	should be -1 for the first call, will be increased by ReadSaveVariables to memorize the position in the variable structure

Returns

= 0 everything okay, != 0 an error occurred

Definition at line [4369](#) of file [store.c](#).

References [InDeXeNtRy::nfunctions](#), [InDeXeNtRy::nindices](#), [InDeXeNtRy::nsymbols](#), [InDeXeNtRy::nvectors](#), and [FuNcTiOn::spec](#).

4.19.8.133 WriteStoreHeader()

```
WORD WriteStoreHeader (
    WORD handle ) [extern]
```

Writes header with information about system architecture and FORM revision to an open store file.

Called by [SetFileIndex\(\)](#).

Parameters

<i>handle</i>	specifies open file to which header will be written
---------------	---

Returns

= 0 everything okay, != 0 an error occurred

Definition at line [3972](#) of file [store.c](#).

References [STOREHEADER::endianness](#), [STOREHEADER::lenLONG](#), [STOREHEADER::lenPOINTER](#), [STOREHEADER::lenPOS](#), [STOREHEADER::lenWORD](#), [STOREHEADER::maxpower](#), [STOREHEADER::sFun](#), [STOREHEADER::sInd](#), [STOREHEADER::sSym](#), [STOREHEADER::sVec](#), and [STOREHEADER::wildoffset](#).

Referenced by [SetFileIndex\(\)](#).

4.19.8.134 InitRecovery()

```
void InitRecovery (
    void ) [extern]
```

Sets up the strings for the filenames of the recovery files. This functions should only be called once to avoid memory leaks and after AM.TempDir has been initialized.

Definition at line [399](#) of file [checkpoint.c](#).

4.19.8.135 CheckRecoveryFile()

```
int CheckRecoveryFile (
    void ) [extern]
```

Checks whether a snapshot/recovery file exists. Returns 1 if it exists, 0 otherwise.

Definition at line [278](#) of file [checkpoint.c](#).

References [RecoveryFilename\(\)](#).

Here is the call graph for this function:



4.19.8.136 DeleteRecoveryFile()

```
void DeleteRecoveryFile (  
    void ) [extern]
```

Deletes the recovery files. It is called by `CleanUp()` in the case of a successful completion.

Definition at line 333 of file [checkpoint.c](#).

4.19.8.137 RecoveryFilename()

```
char * RecoveryFilename (  
    void ) [extern]
```

Returns pointer to recovery filename.

Definition at line 364 of file [checkpoint.c](#).

Referenced by [CheckRecoveryFile\(\)](#), and [DoRecovery\(\)](#).

4.19.8.138 DoRecovery()

```
int DoRecovery (  
    int * moduletype ) [extern]
```

Reads from the recovery file and restores all necessary variables and states in FORM, so that the execution can recommence in preprocessor() as if no restart of FORM had occurred.

The recovery file is read into memory as a whole. The pointer `p` then points into this memory at the next non-processed data. The macros by which variables are restored, like `R_SET`, automatically increase `p` appropriately.

If something goes wrong, the function returns with a non-zero value.

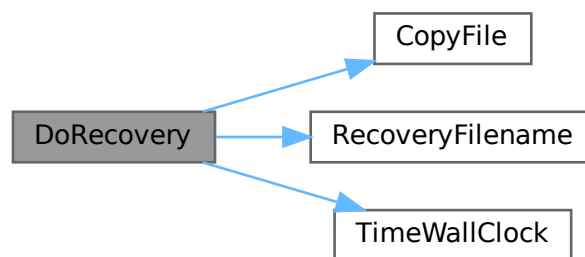
Allocated memory that would be lost when overwriting the global structs with data from the file is freed first. A major part of the code deals with the restoration of pointers. The idiom we use is to memorize the original pointer value (`org`), allocate new memory and copy the data from the file into this memory, calculate the offset between the old pointer value and the new allocated memory position (`ofs`), and then correct all affected pointers (`+=ofs`).

We rely on the fact that several variables (especially in AM) are already assigned the correct values by the startup functions. That means, in principle, that a change in the setup files between snapshot creation and recovery will be noticed.

Definition at line 1402 of file [checkpoint.c](#).

References [TaBlEs::argtail](#), [TaBlEs::boomlijst](#), [BrAcKeTiNfO::bracketbuffer](#), [TaBlEs::buffers](#), [TaBlEs::buffersize](#), [CopyFile\(\)](#), [TaBlEs::flags](#), [ReNuMbEr::func](#), [ReNuMbEr::funnum](#), [VaRrEnUm::hi](#), [BrAcKeTiNfO::indexbuffer](#), [ReNuMbEr::indi](#), [ReNuMbEr::indnum](#), [VaRrEnUm::lo](#), [TaBlEs::MaxTreeSize](#), [TaBlEs::mm](#), [TaBlEs::numind](#), [TaBlEs::pattern](#), [TaBlEs::prototype](#), [TaBlEs::prototypeSize](#), [RecoveryFilename\(\)](#), [ExPrEsSiOn::renumlists](#), [TaBlEs::reserved](#), [TaBlEs::spare](#), [TaBlEs::sparse](#), [VaRrEnUm::start](#), [ReNuMbEr::symb](#), [ReNuMbEr::symnum](#), [TaBlEs::tablepointers](#), [TimeWallClock\(\)](#), [TaBlEs::totind](#), [ReNuMbEr::vecnum](#), and [ReNuMbEr::vect](#).

Here is the call graph for this function:



4.19.8.139 DoCheckpoint()

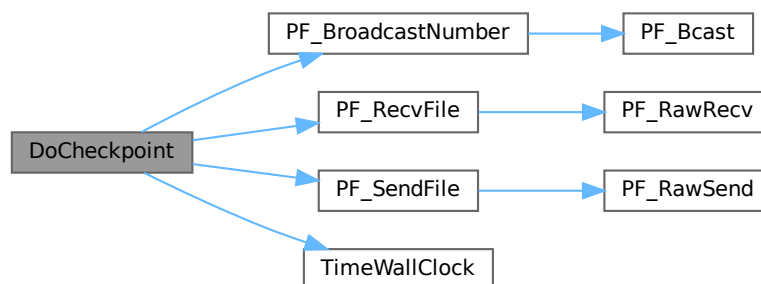
```
void DoCheckpoint (
    int moduletype ) [extern]
```

Checks whether a snapshot should be done. Calls `DoSnapshot()` to create the snapshot.

Definition at line 3111 of file [checkpoint.c](#).

References [PF_BroadcastNumber\(\)](#), [PF_RecvFile\(\)](#), [PF_SendFile\(\)](#), and [TimeWallClock\(\)](#).

Here is the call graph for this function:



4.19.8.140 NumberMallocAddMemory()

```
void NumberMallocAddMemory (
    PHEAD0 ) [extern]
```

Definition at line 2654 of file [tools.c](#).

4.19.8.141 CacheNumberMallocAddMemory()

```
void CacheNumberMallocAddMemory (
    PHEAD0 ) [extern]
```

Definition at line 2728 of file [tools.c](#).

4.19.8.142 TermMallocAddMemory()

```
void TermMallocAddMemory (
    PHEAD0 ) [extern]
```

Definition at line 2550 of file [tools.c](#).

4.19.8.143 ExprStatus()

```
void ExprStatus (
    EXPRESSIONS e ) [extern]
```

Definition at line 66 of file [bugtool.c](#).

4.19.8.144 iniTools()

```
void iniTools (
    void ) [extern]
```

Definition at line 2201 of file [tools.c](#).

4.19.8.145 TestTerm()

```
int TestTerm (
    WORD * term ) [extern]
```

Tests the consistency of the term. Returns 0 when the term is OK. Any nonzero value is trouble. In the current version the testing isn't 100% complete. For instance, we don't check the validity of the symbols nor do we check the range of their powers. Etc. This should be extended when the need is there.

Parameters

<i>term</i>	the term to be tested
-------------	-----------------------

Definition at line [3862](#) of file [tools.c](#).

References [TestTerm\(\)](#).

Referenced by [TestTerm\(\)](#).

Here is the call graph for this function:



4.19.8.146 RunTransform()

```
WORD RunTransform (
    PHEAD WORD * term,
    WORD * params ) [extern]
```

Definition at line [732](#) of file [transform.c](#).

4.19.8.147 RunEncode()

```
WORD RunEncode (
    PHEAD WORD * fun,
    WORD * args,
    WORD * info ) [extern]
```

Definition at line [982](#) of file [transform.c](#).

4.19.8.148 RunDecode()

```
WORD RunDecode (
    PHEAD WORD * fun,
    WORD * args,
    WORD * info ) [extern]
```

Definition at line [1169](#) of file [transform.c](#).

4.19.8.149 RunReplace()

```
WORD RunReplace (
    PHEAD WORD * fun,
    WORD * args,
    WORD * info ) [extern]
```

Definition at line [1339](#) of file [transform.c](#).

4.19.8.150 RunImplode()

```
WORD RunImplode (
    WORD * fun,
    WORD * args ) [extern]
```

Definition at line 1817 of file [transform.c](#).

4.19.8.151 RunExplode()

```
WORD RunExplode (
    PHEAD WORD * fun,
    WORD * args ) [extern]
```

Definition at line 2018 of file [transform.c](#).

4.19.8.152 TestArgNum()

```
int TestArgNum (
    int n,
    int totarg,
    WORD * args ) [extern]
```

Definition at line 3281 of file [transform.c](#).

4.19.8.153 PutArgInScratch()

```
WORD PutArgInScratch (
    WORD * arg,
    UWORD * scrat ) [extern]
```

Definition at line 3350 of file [transform.c](#).

4.19.8.154 ReadRange()

```
UBYTE * ReadRange (
    UBYTE * s,
    WORD * out,
    int par ) [extern]
```

Definition at line 3386 of file [transform.c](#).

4.19.8.155 FindRange()

```
int FindRange (
    PHEAD WORD * args,
    WORD * arg1,
    WORD * arg2,
    WORD totarg ) [extern]
```

Definition at line 3546 of file [transform.c](#).

4.19.8.156 RunPermute()

```
WORD RunPermute (
    PHEAD WORD * fun,
    WORD * args,
    WORD * info ) [extern]
```

Definition at line [2132](#) of file [transform.c](#).

4.19.8.157 RunReverse()

```
WORD RunReverse (
    PHEAD WORD * fun,
    WORD * args ) [extern]
```

Definition at line [2359](#) of file [transform.c](#).

4.19.8.158 RunCycle()

```
WORD RunCycle (
    PHEAD WORD * fun,
    WORD * args,
    WORD * info ) [extern]
```

Definition at line [2535](#) of file [transform.c](#).

4.19.8.159 RunAddArg()

```
WORD RunAddArg (
    PHEAD WORD * fun,
    WORD * args ) [extern]
```

Definition at line [2687](#) of file [transform.c](#).

4.19.8.160 RunMulArg()

```
WORD RunMulArg (
    PHEAD WORD * fun,
    WORD * args ) [extern]
```

Definition at line [2776](#) of file [transform.c](#).

4.19.8.161 RunIsLyndon()

```
WORD RunIsLyndon (
    PHEAD WORD * fun,
    WORD * args,
    int par ) [extern]
```

Definition at line [2917](#) of file [transform.c](#).

4.19.8.162 RunToLyndon()

```
WORD RunToLyndon (  
    PHEAD WORD * fun,  
    WORD * args,  
    int par ) [extern]
```

Definition at line 2995 of file [transform.c](#).

4.19.8.163 RunDropArg()

```
WORD RunDropArg (  
    PHEAD WORD * fun,  
    WORD * args ) [extern]
```

Definition at line 3107 of file [transform.c](#).

4.19.8.164 RunSelectArg()

```
WORD RunSelectArg (  
    PHEAD WORD * fun,  
    WORD * args ) [extern]
```

Definition at line 3133 of file [transform.c](#).

4.19.8.165 RunDedup()

```
WORD RunDedup (  
    PHEAD WORD * fun,  
    WORD * args ) [extern]
```

Definition at line 2446 of file [transform.c](#).

4.19.8.166 RunZtoHArg()

```
WORD RunZtoHArg (  
    PHEAD WORD * fun,  
    WORD * args ) [extern]
```

Definition at line 3161 of file [transform.c](#).

4.19.8.167 RunHtoZArg()

```
WORD RunHtoZArg (  
    PHEAD WORD * fun,  
    WORD * args ) [extern]
```

Definition at line 3216 of file [transform.c](#).

4.19.8.168 NormPolyTerm()

```
int NormPolyTerm (
    PHEAD WORD * term ) [extern]
```

Definition at line 53 of file [notation.c](#).

4.19.8.169 ConvertToPoly()

```
int ConvertToPoly (
    PHEAD WORD * term,
    WORD * outterm,
    WORD * comlist,
    WORD par ) [extern]
```

Definition at line 307 of file [notation.c](#).

4.19.8.170 LocalConvertToPoly()

```
int LocalConvertToPoly (
    PHEAD WORD * term,
    WORD * outterm,
    WORD startebuf,
    WORD par ) [extern]
```

Converts a generic term to polynomial notation in which there are only symbols and brackets. During conversion there will be only symbols. Brackets are stripped. Objects that need 'translation' are put inside a special compiler buffer and represented by a symbol. The numbering of the extra symbols is down from the maximum. In principle there can be a problem when running into the already assigned ones. This uses the FindTree for searching in the global tree and then looks further in the AT.ebufnum. This allows fully parallel processing. Hence we need no locks. Cannot be used in the same module as ConvertToPoly.

Definition at line 510 of file [notation.c](#).

Referenced by [poly_factorize_expression\(\)](#).

4.19.8.171 ConvertFromPoly()

```
int ConvertFromPoly (
    PHEAD WORD * term,
    WORD * outterm,
    WORD from,
    WORD to,
    WORD offset,
    WORD par ) [extern]
```

Definition at line 660 of file [notation.c](#).

4.19.8.172 FindSubterm()

```
WORD FindSubterm (
    WORD * subterm ) [extern]
```

Definition at line 773 of file [notation.c](#).

4.19.8.173 FindLocalSubterm()

```
WORD FindLocalSubterm (
    PHEAD WORD * subterm,
    WORD startebuf ) [extern]
```

Definition at line [842](#) of file [notation.c](#).

4.19.8.174 PrintSubtermList()

```
void PrintSubtermList (
    int from,
    int to ) [extern]
```

Definition at line [895](#) of file [notation.c](#).

4.19.8.175 PrintExtraSymbol()

```
void PrintExtraSymbol (
    int num,
    WORD * terms,
    int par ) [extern]
```

Definition at line [1007](#) of file [notation.c](#).

4.19.8.176 FindSubexpression()

```
WORD FindSubexpression (
    WORD * subexpr ) [extern]
```

Definition at line [1094](#) of file [notation.c](#).

4.19.8.177 UpdateMaxSize()

```
void UpdateMaxSize (
    void ) [extern]
```

Definition at line [1610](#) of file [tools.c](#).

4.19.8.178 CoToPolynomial()

```
int CoToPolynomial (
    UBYTE * inp ) [extern]
```

Definition at line [5925](#) of file [compcomm.c](#).

4.19.8.179 CoFromPolynomial()

```
int CoFromPolynomial (
    UBYTE * inp ) [extern]
```

Definition at line [6000](#) of file [compcomm.c](#).

4.19.8.180 CoArgToExtraSymbol()

```
int CoArgToExtraSymbol (
    UBYTE * s ) [extern]
```

Definition at line [6026](#) of file [compcomm.c](#).

4.19.8.181 CoExtraSymbols()

```
int CoExtraSymbols (
    UBYTE * inp ) [extern]
```

Definition at line [6076](#) of file [compcomm.c](#).

4.19.8.182 GetDoParam()

```
UBYTE * GetDoParam (
    UBYTE * inp,
    WORD ** wp,
    int par ) [extern]
```

Definition at line [6207](#) of file [compcomm.c](#).

4.19.8.183 GetIfDollarFactor()

```
WORD * GetIfDollarFactor (
    UBYTE ** inp,
    WORD * w ) [extern]
```

Definition at line [6149](#) of file [compcomm.c](#).

4.19.8.184 CoDo()

```
int CoDo (
    UBYTE * inp ) [extern]
```

Definition at line [6284](#) of file [compcomm.c](#).

4.19.8.185 CoEndDo()

```
int CoEndDo (
    UBYTE * inp ) [extern]
```

Definition at line [6387](#) of file [compcomm.c](#).

4.19.8.186 ExtraSymFun()

```
int ExtraSymFun (
    PHEAD WORD * term,
    WORD level ) [extern]
```

Definition at line [1146](#) of file [notation.c](#).

4.19.8.187 PruneExtraSymbols()

```
int PruneExtraSymbols (
    WORD downto ) [extern]
```

Definition at line [1214](#) of file [notation.c](#).

4.19.8.188 IniFbuffer()

```
int IniFbuffer (
    WORD bufnum ) [extern]
```

Initialize a factorization cache buffer. We set the size of the rhs and boomlijst buffers immediately to their final values.

Definition at line [614](#) of file [comtool.c](#).

References [tree::blnce](#), [CbUf::boomlijst](#), [CbUf::CanCommu](#), [CbUf::dimension](#), [tree::left](#), [CbUf::numdum](#), [CbUf::NumTerms](#), [tree::parent](#), [CbUf::rhs](#), [tree::right](#), [tree::usage](#), and [tree::value](#).

4.19.8.189 GCDfunction()

```
int GCDfunction (
    PHEAD WORD * term,
    WORD level ) [extern]
```

Definition at line [609](#) of file [ratio.c](#).

4.19.8.190 GCDfunction3()

```
WORD * GCDfunction3 (
    PHEAD WORD * in1,
    WORD * in2 ) [extern]
```

Definition at line [1141](#) of file [ratio.c](#).

4.19.8.191 ReadPolyRatFun()

```
int ReadPolyRatFun (
    PHEAD WORD * term ) [extern]
```

Definition at line 2264 of file [ratio.c](#).

4.19.8.192 FromPolyRatFun()

```
int FromPolyRatFun (
    PHEAD WORD * fun,
    WORD ** numout,
    WORD ** denout ) [extern]
```

Definition at line 2383 of file [ratio.c](#).

4.19.8.193 PolyDiv()

```
WORD * PolyDiv (
    PHEAD WORD * a,
    WORD * b,
    char * text ) [extern]
```

Definition at line 2741 of file [ratio.c](#).

4.19.8.194 GCDclean()

```
void GCDclean (
    PHEAD WORD * num,
    WORD * den ) [extern]
```

Definition at line 2644 of file [ratio.c](#).

4.19.8.195 TakeSymbolContent()

```
WORD * TakeSymbolContent (
    PHEAD WORD * in,
    WORD * term ) [extern]
```

Implements part of the old ExecArg in which we take common factors from arguments with more than one term. We allow only symbols as this code is used for the polyratfun only. We have a special routine, because the generic TakeContent does too much work and speed is at a premium here. Input: in is the input expression as a sequence of terms. **Output:** term: the content return value: the contentfree expression. it is in new allocation, made by TermMalloc. (should be in a TermMalloc space?)

Definition at line 2434 of file [ratio.c](#).

References [GetModInverses\(\)](#).

Here is the call graph for this function:



4.19.8.196 GCDterms()

```
int GCDterms (
    PHEAD WORD * term1,
    WORD * term2,
    WORD * termout ) [extern]
```

Definition at line [2076](#) of file [ratio.c](#).

4.19.8.197 PutExtraSymbols()

```
WORD * PutExtraSymbols (
    PHEAD WORD * in,
    WORD startebuf,
    int * actionflag ) [extern]
```

Definition at line [1244](#) of file [ratio.c](#).

4.19.8.198 TakeExtraSymbols()

```
WORD * TakeExtraSymbols (
    PHEAD WORD * in,
    WORD startebuf ) [extern]
```

Definition at line [1273](#) of file [ratio.c](#).

4.19.8.199 MultiplyWithTerm()

```
WORD * MultiplyWithTerm (
    PHEAD WORD * in,
    WORD * term,
    WORD par ) [extern]
```

Definition at line [1312](#) of file [ratio.c](#).

4.19.8.200 TakeContent()

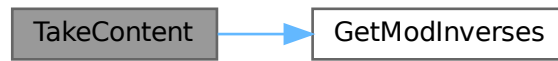
```
WORD * TakeContent (
    PHEAD WORD * in,
    WORD * term ) [extern]
```

Implements part of the old ExecArg in which we take common factors from arguments with more than one term. Here the input is a sequence of terms in 'in' and the answer is a content-free sequence of terms. This sequence has been allocated by the Malloc1 routine in a call to EndSort, unless the expression was already content-free. In that case the input pointer is returned. The content is returned in term. This is supposed to be a separate allocation, made by TermMalloc in the calling routine.

Definition at line [1376](#) of file [ratio.c](#).

References [GetModInverses\(\)](#).

Here is the call graph for this function:



4.19.8.201 MergeSymbolLists()

```
int MergeSymbolLists (
    PHEAD WORD * old,
    WORD * extra,
    int par ) [extern]
```

Definition at line 1808 of file [ratio.c](#).

4.19.8.202 MergeDotproductLists()

```
int MergeDotproductLists (
    PHEAD WORD * old,
    WORD * extra,
    int par ) [extern]
```

Definition at line 1927 of file [ratio.c](#).

4.19.8.203 CreateExpression()

```
WORD * CreateExpression (
    PHEAD WORD ) [extern]
```

Definition at line 2020 of file [ratio.c](#).

4.19.8.204 DIVfunction()

```
int DIVfunction (
    PHEAD WORD * term,
    WORD level,
    int par ) [extern]
```

Definition at line 2788 of file [ratio.c](#).

4.19.8.205 MULfunc()

```
WORD * MULfunc (
    PHEAD WORD * p1,
    WORD * p2 ) [extern]
```

Definition at line [2957](#) of file [ratio.c](#).

4.19.8.206 ConvertArgument()

```
WORD * ConvertArgument (
    PHEAD WORD * arg,
    int * type ) [extern]
```

Definition at line [3018](#) of file [ratio.c](#).

4.19.8.207 ExpandRat()

```
int ExpandRat (
    PHEAD WORD * fun ) [extern]
```

Definition at line [3113](#) of file [ratio.c](#).

4.19.8.208 InvPoly()

```
int InvPoly (
    PHEAD WORD * inpoly,
    WORD maxpow,
    WORD sym ) [extern]
```

Definition at line [3592](#) of file [ratio.c](#).

4.19.8.209 TestDoLoop()

```
WORD TestDoLoop (
    PHEAD WORD * lhsbuf,
    WORD level ) [extern]
```

Definition at line [2762](#) of file [dollar.c](#).

4.19.8.210 TestEndDoLoop()

```
WORD TestEndDoLoop (
    PHEAD WORD * lhsbuf,
    WORD level ) [extern]
```

Definition at line [2840](#) of file [dollar.c](#).

4.19.8.211 poly_gcd()

```
WORD * poly_gcd (  
    PHEAD WORD * a,  
    WORD * b,  
    WORD fit ) [extern]
```

Polynomial gcd

4.19.9 Description

This method calculates the greatest common divisor of two polynomials, given by two zero-terminated Form-style term lists.

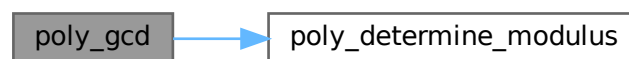
4.19.10 Notes

- The result is written at newly allocated memory
- Called from [ratio.c](#)
- Calls `polygcd::gcd`

Definition at line [124](#) of file [polywrap.cc](#).

References [poly_determine_modulus\(\)](#).

Here is the call graph for this function:



4.19.10.1 poly_div()

```
WORD * poly_div (  
    PHEAD WORD * a,  
    WORD * b,  
    WORD fit ) [extern]
```

Definition at line [435](#) of file [polywrap.cc](#).

4.19.10.2 poly_rem()

```
WORD * poly_rem (
    PHEAD WORD * a,
    WORD * b,
    WORD fit ) [extern]
```

Definition at line [463](#) of file [polywrap.cc](#).

4.19.10.3 poly_inverse()

```
WORD * poly_inverse (
    PHEAD WORD * arga,
    WORD * argb ) [extern]
```

Definition at line [1743](#) of file [polywrap.cc](#).

4.19.10.4 poly_mul()

```
WORD * poly_mul (
    PHEAD WORD * a,
    WORD * b ) [extern]
```

Definition at line [1948](#) of file [polywrap.cc](#).

4.19.10.5 poly_ratfun_add()

```
WORD * poly_ratfun_add (
    PHEAD WORD * t1,
    WORD * t2 ) [extern]
```

Addition of PolyRatFuns

4.19.11 Description

This method gets two pointers to polyratfuns with up to two arguments each and calculates the sum.

4.19.14 Notes

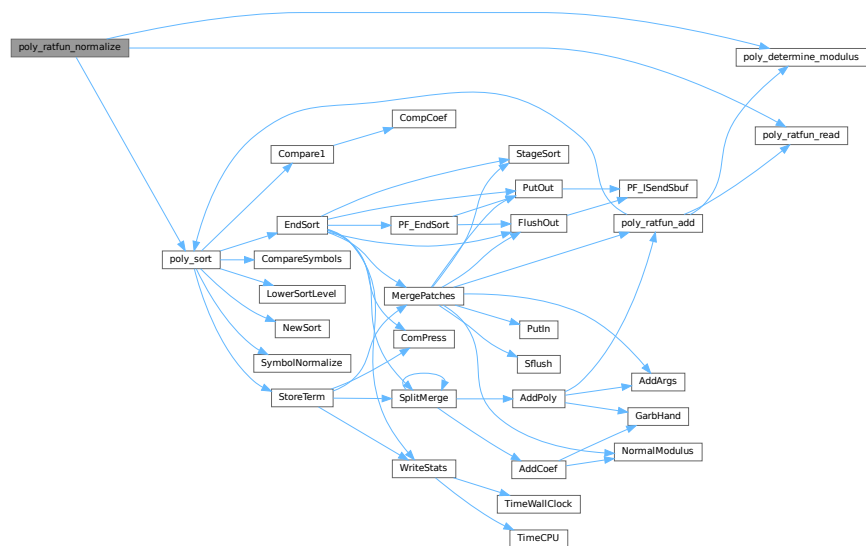
- The result overwrites the original term
- Called from [proces.c](#)
- Calls `poly::operators` and `polygcd::gcd`

Definition at line 769 of file [polywrap.cc](#).

References [poly_determine_modulus\(\)](#), [poly_ratfun_read\(\)](#), and [poly_sort\(\)](#).

Referenced by [PolyFunMul\(\)](#), and [PrepPoly\(\)](#).

Here is the call graph for this function:



4.19.14.1 poly_factorize_argument()

```

int poly_factorize_argument (
    PHEAD WORD * argin,
    WORD * argout ) [extern]
  
```

Factorization of function arguments

4.19.15 Description

This method factorizes the Form-style argument `argin`.

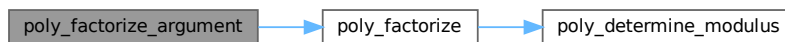
4.19.16 Notes

- The result is written at argout
- Called from [argument.c](#)
- Calls `poly_factorize`

Definition at line 1112 of file [polywrap.cc](#).

References [poly_factorize\(\)](#).

Here is the call graph for this function:



4.19.16.1 `poly_factorize_dollar()`

```
WORD * poly_factorize_dollar (
    PHEAD WORD * argin ) [extern]
```

Factorization of dollar variables

4.19.17 Description

This method factorizes a dollar variable.

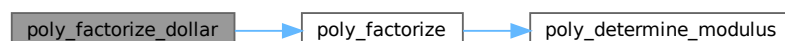
4.19.18 Notes

- The result is written at newly allocated memory.
- Called from [dollar.c](#)
- Calls `poly_factorize`

Definition at line 1146 of file [polywrap.cc](#).

References [poly_factorize\(\)](#).

Here is the call graph for this function:



4.19.18.1 poly_factorize_expression()

```
int poly_factorize_expression (
    EXPRESSIONS expr ) [extern]
```

Factorization of expressions

4.19.19 Description

This method factorizes an expression.

4.19.20 Notes

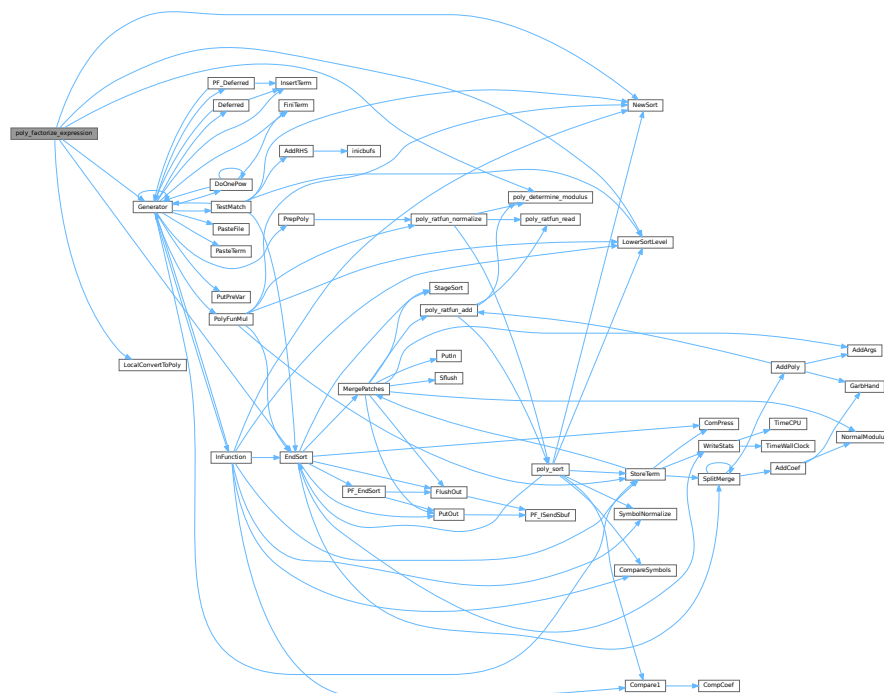
- The result overwrites the input expression
- Called from [proces.c](#)
- Calls `polyfact::factorize`

Definition at line 1178 of file [polywrap.cc](#).

References [EndSort\(\)](#), [Generator\(\)](#), [File::handle](#), [LocalConvertToPoly\(\)](#), [LowerSortLevel\(\)](#), [NewSort\(\)](#), and [poly_determine_modulus\(\)](#).

Referenced by [PF_Processor\(\)](#), and [Processor\(\)](#).

Here is the call graph for this function:



4.19.22.1 poly_free_poly_vars()

```
void poly_free_poly_vars (
    PHEAD const char * text ) [extern]
```

Definition at line 2014 of file [polywrap.cc](#).

4.19.22.2 optimize_print_code()

```
void optimize_print_code (
    int print_expr ) [extern]
```

Print optimized code

4.19.23 Description

This method prints the optimized code via "PrintExtraSymbol". Depending on the flag, the original expression is printed (for "Print") or not (for "#Optimize / #write "O").

Definition at line 4525 of file [optimize.cc](#).

Referenced by [Optimize\(\)](#).

4.19.23.1 DoPreAdd()

```
int DoPreAdd (
    UBYTE * s ) [extern]
```

Definition at line 1067 of file [dict.c](#).

4.19.23.2 DoPreUseDictionary()

```
int DoPreUseDictionary (
    UBYTE * s ) [extern]
```

Definition at line 1022 of file [dict.c](#).

4.19.23.3 DoPreCloseDictionary()

```
int DoPreCloseDictionary (
    UBYTE * s ) [extern]
```

Definition at line 998 of file [dict.c](#).

4.19.23.4 DoPreOpenDictionary()

```
int DoPreOpenDictionary (
    UBYTE * s ) [extern]
```

Definition at line 959 of file [dict.c](#).

4.19.23.5 RemoveDictionary()

```
void RemoveDictionary (
    DICTIONARY * dict ) [extern]
```

Definition at line 902 of file [dict.c](#).

4.19.23.6 UnSetDictionary()

```
void UnSetDictionary (
    void ) [extern]
```

Definition at line 883 of file [dict.c](#).

4.19.23.7 SetDictionaryOptions()

```
int SetDictionaryOptions (
    UBYTE * options ) [extern]
```

Definition at line 790 of file [dict.c](#).

4.19.23.8 AddToDictionary()

```
int AddToDictionary (
    DICTIONARY * dict,
    UBYTE * left,
    UBYTE * right ) [extern]
```

Definition at line 491 of file [dict.c](#).

4.19.23.9 AddDictionary()

```
int AddDictionary (
    UBYTE * name ) [extern]
```

Definition at line 449 of file [dict.c](#).

4.19.23.10 FindDictionary()

```
int FindDictionary (
    UBYTE * name ) [extern]
```

Definition at line 434 of file [dict.c](#).

4.19.23.11 IsExponentSign()

```
UBYTE * IsExponentSign (
    void ) [extern]
```

Definition at line 238 of file [dict.c](#).

4.19.23.12 IsMultiplySign()

```
UBYTE * IsMultiplySign (
    void ) [extern]
```

Definition at line 218 of file [dict.c](#).

4.19.23.13 TransformRational()

```
void TransformRational (
    UWORD * a,
    WORD na ) [extern]
```

Definition at line 71 of file [dict.c](#).

4.19.23.14 WriteDictionary()

```
void WriteDictionary (
    DICTIONARY * dict ) [extern]
```

Definition at line 1307 of file [sch.c](#).

4.19.23.15 ShrinkDictionary()

```
void ShrinkDictionary (
    DICTIONARY * dict ) [extern]
```

Definition at line 945 of file [dict.c](#).

4.19.23.16 MultiplyToLine()

```
void MultiplyToLine (
    void ) [extern]
```

Definition at line 671 of file [sch.c](#).

4.19.23.17 FindSymbol()

```
UBYTE * FindSymbol (
    WORD num ) [extern]
```

Definition at line 258 of file [dict.c](#).

4.19.23.18 FindVector()

```
UBYTE * FindVector (
    WORD num ) [extern]
```

Definition at line 279 of file [dict.c](#).

4.19.23.19 FindIndex()

```
UBYTE * FindIndex (
    WORD num ) [extern]
```

Definition at line 301 of file [dict.c](#).

4.19.23.20 FindFunction()

```
UBYTE * FindFunction (
    WORD num ) [extern]
```

Definition at line 323 of file [dict.c](#).

4.19.23.21 FindFunWithArgs()

```
UBYTE * FindFunWithArgs (
    WORD * t ) [extern]
```

Definition at line 345 of file [dict.c](#).

4.19.23.22 FindExtraSymbol()

```
UBYTE * FindExtraSymbol (
    WORD num ) [extern]
```

Definition at line 377 of file [dict.c](#).

4.19.23.23 DictToBytes()

```
LONG DictToBytes (
    DICTIONARY * dict,
    UBYTE * buf ) [extern]
```

Definition at line 1119 of file [dict.c](#).

4.19.23.24 DictFromBytes()

```
DICTIONARY * DictFromBytes (
    UBYTE * buf ) [extern]
```

Definition at line 1152 of file [dict.c](#).

4.19.23.25 CoCreateSpectator()

```
int CoCreateSpectator (
    UBYTE * inp ) [extern]
```

Definition at line 89 of file [spectator.c](#).

4.19.23.26 CoToSpectator()

```
int CoToSpectator (
    UBYTE * inp ) [extern]
```

Definition at line 181 of file [spectator.c](#).

4.19.23.27 CoRemoveSpectator()

```
int CoRemoveSpectator (
    UBYTE * inp ) [extern]
```

Definition at line 233 of file [spectator.c](#).

4.19.23.28 CoEmptySpectator()

```
int CoEmptySpectator (
    UBYTE * inp ) [extern]
```

Definition at line 293 of file [spectator.c](#).

4.19.23.29 CoCopySpectator()

```
int CoCopySpectator (
    UBYTE * inp ) [extern]
```

Definition at line 459 of file [spectator.c](#).

4.19.23.30 PutInSpectator()

```
int PutInSpectator (
    WORD * term,
    WORD specnum ) [extern]
```

Definition at line 345 of file [spectator.c](#).

4.19.23.31 ClearSpectators()

```
void ClearSpectators (
    WORD par ) [extern]
```

Definition at line 675 of file [spectator.c](#).

4.19.23.32 GetFromSpectator()

```
WORD GetFromSpectator (
    WORD * term,
    WORD specnum ) [extern]
```

Definition at line 601 of file [spectator.c](#).

4.19.23.33 FlushSpectators()

```
void FlushSpectators (
    void ) [extern]
```

Definition at line 402 of file [spectator.c](#).

4.19.23.34 ReadFromScratch()

```
int ReadFromScratch (
    FILEHANDLE * fi,
    POSITION * pos,
    UBYTE * buffer,
    POSITION * length ) [extern]
```

Definition at line 273 of file [store.c](#).

4.19.23.35 AddToScratch()

```
int AddToScratch (
    FILEHANDLE * fi,
    POSITION * pos,
    UBYTE * buffer,
    POSITION * length,
    int withflush ) [extern]
```

Definition at line 300 of file [store.c](#).

4.19.23.36 DoPreAppendPath()

```
int DoPreAppendPath (
    UBYTE * s ) [extern]
```

Appends the given path (absolute or relative to the current file directory) to the FORM path.

Syntax: `#appendpath <path>`

Definition at line 7153 of file [pre.c](#).

4.19.23.37 DoPrePrependPath()

```
int DoPrePrependPath (
    UBYTE * s ) [extern]
```

Prepends the given path (absolute or relative to the current file directory) to the FORM path.

Syntax: #prependpath <path>

Definition at line 7170 of file [pre.c](#).

4.19.23.38 DoSwitch()

```
int DoSwitch (
    PHEAD WORD * term,
    WORD * lhs ) [extern]
```

Definition at line 1070 of file [if.c](#).

4.19.23.39 DoEndSwitch()

```
int DoEndSwitch (
    PHEAD WORD * term,
    WORD * lhs ) [extern]
```

Definition at line 1086 of file [if.c](#).

4.19.23.40 FindCase()

```
SWITCHTABLE * FindCase (
    WORD nswitch,
    WORD ncase ) [extern]
```

Definition at line 1097 of file [if.c](#).

4.19.23.41 SwitchSplitMergeRec()

```
void SwitchSplitMergeRec (
    SWITCHTABLE * array,
    WORD num,
    SWITCHTABLE * auxarray ) [extern]
```

Definition at line 1184 of file [if.c](#).

4.19.23.42 SwitchSplitMerge()

```
void SwitchSplitMerge (
    SWITCHTABLE * array,
    WORD num ) [extern]
```

Definition at line 1213 of file [if.c](#).

4.19.23.43 DoubleSwitchBuffers()

```
int DoubleSwitchBuffers (
    void ) [extern]
```

Definition at line [1135](#) of file [if.c](#).

4.19.23.44 DistrN()

```
int DistrN (
    int n,
    int * cpl,
    int ncpl,
    int * scratch ) [extern]
```

Definition at line [4048](#) of file [tools.c](#).

4.19.23.45 DoNamespace()

```
int DoNamespace (
    UBYTE * s ) [extern]
```

Definition at line [7234](#) of file [pre.c](#).

4.19.23.46 DoEndNamespace()

```
int DoEndNamespace (
    UBYTE * s ) [extern]
```

Definition at line [7282](#) of file [pre.c](#).

4.19.23.47 DoUse()

```
int DoUse (
    UBYTE * s ) [extern]
```

Definition at line [7497](#) of file [pre.c](#).

4.19.23.48 SkipName()

```
UBYTE * SkipName (
    UBYTE * s ) [extern]
```

Definition at line [7305](#) of file [pre.c](#).

4.19.23.49 ConstructName()

```
UBYTE * ConstructName (
    UBYTE * s,
    UBYTE type ) [extern]
```

Definition at line 7405 of file [pre.c](#).

4.19.23.50 DoSetUserFlag()

```
int DoSetUserFlag (
    UBYTE * s ) [extern]
```

Definition at line 7662 of file [pre.c](#).

4.19.23.51 DoClearUserFlag()

```
int DoClearUserFlag (
    UBYTE * s ) [extern]
```

Definition at line 7652 of file [pre.c](#).

4.19.23.52 CreateVertex()

```
VERTEX * CreateVertex (
    MODEL * m )
```

Definition at line 46 of file [model.c](#).

4.19.23.53 ReadParticle()

```
UBYTE * ReadParticle (
    UBYTE * s,
    VERTEX * v,
    MODEL * m,
    int par )
```

Definition at line 76 of file [model.c](#).

4.19.23.54 CoModel()

```
int CoModel (
    UBYTE * s )
```

Definition at line 119 of file [model.c](#).

4.19.23.55 CoParticle()

```
int CoParticle (
    UBYTE * s )
```

Definition at line 194 of file [model.c](#).

4.19.23.56 CoVertex()

```
int CoVertex (
    UBYTE * s )
```

Definition at line 276 of file [model.c](#).

4.19.23.57 CoEndModel()

```
int CoEndModel (
    UBYTE * s )
```

Definition at line 379 of file [model.c](#).

4.19.23.58 LoadModel()

```
int LoadModel (
    MODEL * m )
```

Definition at line 27 of file [diawrap.cc](#).

4.19.23.59 CoSetUserFlag()

```
int CoSetUserFlag (
    UBYTE * s )
```

Definition at line 7384 of file [compcomm.c](#).

4.19.23.60 CoClearUserFlag()

```
int CoClearUserFlag (
    UBYTE * s )
```

Definition at line 7412 of file [compcomm.c](#).

4.19.23.61 CoCreateAllLoops()

```
int CoCreateAllLoops (
    UBYTE * s )
```

Definition at line 7449 of file [compcomm.c](#).

4.19.23.62 CoCreateAllPaths()

```
int CoCreateAllPaths (
    UBYTE * s )
```

Definition at line [7569](#) of file [compcomm.c](#).

4.19.23.63 CoCreateAll()

```
int CoCreateAll (
    UBYTE * s )
```

Definition at line [7697](#) of file [compcomm.c](#).

4.19.23.64 AllLoops()

```
WORD AllLoops (
    PHEAD WORD * term,
    WORD level )
```

Definition at line [650](#) of file [lus.c](#).

4.19.23.65 StartLoops()

```
LONG StartLoops (
    PHEAD WORD * term,
    WORD level,
    LONG vert,
    WORD nvert,
    WORD * arglist,
    WORD nargs,
    WORD * loop,
    WORD nloop )
```

Definition at line [894](#) of file [lus.c](#).

4.19.23.66 GenLoops()

```
LONG GenLoops (
    PHEAD WORD * term,
    WORD level,
    LONG vert,
    WORD nvert,
    WORD * arglist,
    WORD nargs,
    WORD * loop,
    WORD nloop )
```

Definition at line [980](#) of file [lus.c](#).

4.19.23.67 LoopOutput()

```
void LoopOutput (
    PHEAD WORD * term,
    WORD level,
    WORD * loop,
    WORD nloop )
```

Definition at line [1059](#) of file [lus.c](#).

4.19.23.68 AllPaths()

```
WORD AllPaths (
    PHEAD WORD * term,
    WORD level )
```

Definition at line [1123](#) of file [lus.c](#).

4.19.23.69 GenPaths()

```
LONG GenPaths (
    PHEAD WORD * term,
    WORD level,
    LONG vert,
    WORD nvert,
    WORD * arglist,
    WORD nargs,
    WORD * path,
    WORD npath )
```

Definition at line [1413](#) of file [lus.c](#).

4.19.23.70 PathOutput()

```
void PathOutput (
    PHEAD WORD * term,
    WORD level,
    WORD * path,
    WORD npath )
```

Definition at line [1486](#) of file [lus.c](#).

4.20 declare.h

[Go to the documentation of this file.](#)

```

00001 #ifndef __FDECLARE__
00002 #define __FDECLARE__
00003
00009 /* #[ License : */
00010 /*
00011 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00012 * When using this file you are requested to refer to the publication
00013 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00014 * This is considered a matter of courtesy as the development was paid
00015 * for by FOM the Dutch physics granting agency and we would like to
00016 * be able to track its scientific use to convince FOM of its value
00017 * for the community.
00018 *
00019 * This file is part of FORM.
00020 *
00021 * FORM is free software: you can redistribute it and/or modify it under the
00022 * terms of the GNU General Public License as published by the Free Software
00023 * Foundation, either version 3 of the License, or (at your option) any later
00024 * version.
00025 *
00026 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00027 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00028 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00029 * details.
00030 *
00031 * You should have received a copy of the GNU General Public License along
00032 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00033 */
00034 /* #] License : */
00035
00036 /*
00037 #[ Macro's :
00038 */
00039
00040 #define MaX(x,y) ((x) > (y) ? (x) : (y))
00041 #define MiN(x,y) ((x) < (y) ? (x) : (y))
00042 #define ABS(x) ( (x) < 0 ? -(x) : (x) )
00043 #define SGN(x) ( (x) > 0 ? 1 : (x) < 0 ? -1 : 0 )
00044 #define REDLENG(x) (((x)<0)?((x)+1):((x)-1))/2)
00045 #define INCLENG(x) (((x)<0)?((x)*2)-1:((x)*2)+1))
00046 #define GETCOEF(x,y) x += *x;y = x[-1];x -= ABS(y);y=REDLENG(y)
00047 #define GETSTOP(x,y) y=x+(*x)-1;y -= ABS(*y)-1
00048 #define StuffAdd(x,y) (((x)<0?-1:1)*(y)+((y)<0?-1:1)*(x))
00049
00050 #define EXCHN(t1,t2,n) { WORD a,i; for(i=0;i<n;i++){a=t1[i];t1[i]=t2[i];t2[i]=a;} }
00051 #define EXCH(x,y) { WORD a = (x); (x) = (y); (y) = a; }
00052
00053 #define TOKENTOline(x,y) if ( AC.OutputSpaces == NOSPACEFORMAT ) { \
00054 TokenToLine((UBYTE *) (y)); } else { TokenToLine((UBYTE *) (x)); }
00055
00056 #define UngetFromStream(stream,c) ((stream)->nextchar[(stream)->isnextchar++]=c)
00057 #ifdef WITHRETURN
00058 #define AddLineFeed(s,n) { (s)[(n)++] = CARRIAGERETURN; (s)[(n)++] = LINEFEED; }
00059 #else
00060 #define AddLineFeed(s,n) { (s)[(n)++] = LINEFEED; }
00061 #endif
00062 #define TryRecover(x) Terminate(-1)
00063 #define UngetChar(c) { pushbackchar = c; }
00064 #define ParseNumber(x,s) { (x)=0;while(*(s)>='0'&&*(s)<='9')(x)=10*(x)+*(s)++ -'0';}
00065 #define ParseSign(sgn,s) { (sgn)=0;while(*(s)=='-'||*(s)=='+'){ \
00066 if ( *(s)++ == '-' ) sgn ^= 1;}}
00067 #define ParseSignedNumber(x,s) { int sgn; ParseSign(sgn,s)\
00068 ParseNumber(x,s) if ( sgn ) x = -x; }
00069
00070 /* (n) is necessary here, since the macro is sometimes passed dereferenced pointers for n */
00071 #define NCOPY(s,t,n) { while ( (n)-- > 0 ) { *s++ = *t++; } }
00072 /*#define NCOPY(s,t,n) { memcpy(s,t,n*sizeof(WORD)); s+=n; t+=n; n = -1; }*/
00073 #define NCOPYI(s,t,n) { while ( (n)-- > 0 ) { *s++ = *t++; } }
00074 #define NCOPYB(s,t,n) { while ( (n)-- > 0 ) { *s++ = *t++; } }
00075 #define NCOPYI32(s,t,n) { while ( (n)-- > 0 ) { *s++ = *t++; } }
00076 #define WCOPY(s,t,n) { int nn=n; WORD *ss=(WORD *)s, *tt=(WORD *)t; while ( (nn)-- > 0 ) { *ss++ =
*tt++; } }
00077
00078 #define NeedNumber(x,s,err) { int sgn = 1;
00079 while ( *s == ' ' || *s == '\t' || *s == '-' || *s == '+' ) {
00080 if ( *s == '-' ) {sgn = -sgn;} s++; }
00081 if ( chartype[*s] != 1 ) goto err;
00082 ParseNumber(x,s)
00083 if ( sgn < 0 ) {(x) = -(x);} while ( *s == ' ' || *s == '\t' ) s++; \
00084 }
00085 #define SKIPBLANKS(s) { while ( *(s) == ' ' || *(s) == '\t' ) (s)++; }
00086 #define FLUSHCONSOLE if ( AP.InOutBuf > 0 ) CharOut(LINEFEED)

```

```

00087
00088 #define SKIPBRA1(s) { int lev1=0; s++; while(*s) { if(*s=='{') {lev1++;} \
00089 else {if(*s=='}' && --lev1<0) {break;} } s++; } }
00090 #define SKIPBRA2(s) { int lev2=0; s++; while(*s) { if(*s=='{') {lev2++;} \
00091 else {if(*s=='}' && --lev2<0) {break;} } \
00092 else {if(*s=='{') {SKIPBRA1(s);} } s++; } }
00093 #define SKIPBRA3(s) { int lev3=0; s++; while(*s) { if(*s=='(') {lev3++;} \
00094 else {if(*s==')' && --lev3<0) {break;} } \
00095 else {if(*s=='{') {SKIPBRA2(s);} } \
00096 else {if(*s=='[') {SKIPBRA1(s);} } } s++; } }
00097 #define SKIPBRA4(s) { int lev4=0; s++; while(*s) { if(*s=='(') {lev4++;} \
00098 else {if(*s==')' && --lev4<0) {break;} } \
00099 else {if(*s=='{') {SKIPBRA1(s);} } s++; } }
00100 #define SKIPBRA5(s) { int lev5=0; s++; while(*s) { if(*s=='{') {lev5++;} \
00101 else {if(*s=='}' && --lev5<0) {break;} } \
00102 else {if(*s=='(') {SKIPBRA4(s);} } \
00103 else {if(*s=='[') {SKIPBRA1(s);} } } s++; } }
00104
00105 /*
00106 #define CYCLE1(a,i) {WORD iX,jX; iX=*a; for(jX=1;jX<i;jX++)a[jX-1]=a[jX]; a[i-1]=iX;}
00107 */
00108 #define CYCLE1(t,a,i) {t iX=*a; WORD jX; for(jX=1;jX<i;jX++)a[jX-1]=a[jX]; a[i-1]=iX;}
00109
00110 #define AddToCB(c,wx) if(c->Pointer>=c->Top) \
00111 {DoubleCbuffer(c-cbuf,c->Pointer,21);} \
00112 *(c->Pointer)++ = wx;
00113
00114 #define EXCHINOUT { FILEHANDLE *ffFi = AR.outfile; \
00115 AR.outfile = AR.infile; AR.infile = ffFi; }
00116 #define BACKINOUT { FILEHANDLE *ffFi = AR.outfile; POSITION posi; \
00117 AR.outfile = AR.infile; AR.infile = ffFi; \
00118 SetEndScratch(AR.infile,&posi); }
00119
00120 #define CopyArg(to,from) { if ( *from > 0 ) { int ica = *from; NCOPY(to,from,ica) } \
00121 else if ( *from <= -FUNCTION ) *to++ = *from++; \
00122 else { *to++ = *from++; *to++ = *from++; } }
00123
00124 #if ARGHEAD > 2
00125 #define FILLARG(w) { int i = ARGHEAD-2; while ( --i >= 0 ) *w++ = 0; }
00126 #define COPYARG(w,t) { int i = ARGHEAD-2; while ( --i >= 0 ) *w++ = *t++; }
00127 #define ZEROARG(w) { int i; for ( i = 2; i < ARGHEAD; i++ ) w[i] = 0; }
00128 #else
00129 #define FILLARG(w)
00130 #define COPYARG(w,t)
00131 #define ZEROARG(w)
00132 #endif
00133
00134 #if FUNHEAD > 2
00135 #define FILLFUN(w) { *w++ = 0; FILLFUN3(w) }
00136 #define COPYFUN(w,t) { *w++ = *t++; COPYFUN3(w,t) }
00137 #else
00138 #define FILLFUN(w)
00139 #define COPYFUN(w,t)
00140 #endif
00141
00142 #if FUNHEAD > 3
00143 #define FILLFUN3(w) { int ie = FUNHEAD-3; while ( --ie >= 0 ) *w++ = 0; }
00144 #define COPYFUN3(w,t) { int ie = FUNHEAD-3; while ( --ie >= 0 ) *w++ = *t++; }
00145 #else
00146 #define COPYFUN3(w,t)
00147 #define FILLFUN3(w)
00148 #endif
00149
00150 #if SUBEXPSIZE > 5
00151 #define FILLSUB(w) { int ie = SUBEXPSIZE-5; while ( --ie >= 0 ) *w++ = 0; }
00152 #define COPYSUB(w,ww) { int ie = SUBEXPSIZE-5; while ( --ie >= 0 ) *w++ = *ww++; }
00153 #else
00154 #define FILLSUB(w)
00155 #define COPYSUB(w,ww)
00156 #endif
00157
00158 #if EXPRHEAD > 4
00159 #define FILLEXPR(w) { int ie = EXPRHEAD-4; while ( --ie >= 0 ) *w++ = 0; }
00160 #else
00161 #define FILLEXPR(w)
00162 #endif
00163
00164 #define NEXTARG(x) if(*x>0) x += *x; else if(*x <= -FUNCTION)x++; else x += 2;
00165 #define COPY1ARG(sl,t1) { int ica; if ( (ica==t1) > 0 ) { NCOPY(sl,t1,ica) } \
00166 else if(*t1<=-FUNCTION){*sl++=*t1++;} else{*sl++=*t1++;*sl++=*t1++;} }
00167
00175 #define ZeroFillRange(w,begin,end) do { \
00176 int tmp_i; \
00177 for ( tmp_i = begin; tmp_i < end; tmp_i++ ) { (w)[tmp_i] = 0; } \
00178 } while (0)
00179
00180 #define TABLESIZE(a,b) (((WORD)sizeof(a))/((WORD)sizeof(b)))

```

```

00181 #define WORDDIF(x,y) (WORD)(x-y)
00182 #define wsizeof(a) ((WORD)sizeof(a))
00183 #define VARNAME(type,num) (AC.varnames->namebuffer+type[num].name)
00184 #define DOLLARNAME(type,num) (AC.dollarnames->namebuffer+type[num].name)
00185 #define EXPNAME(num) (AC.exprnames->namebuffer+Expressions[num].name)
00186
00187 #define PREV(x) prevorder?prevorder:x
00188
00189 #define Terminate(x) do { TerminateImpl(x, __FILE__, __LINE__, __FUNCTION__); } while(0)
00190 #define SETERROR(x) { Terminate(-1); return(-1); }
00191
00192 /* use this macro to avoid the unused parameter warning */
00193 #define DUMMYUSE(x) (void)(x);
00194
00195 #ifdef _FILE_OFFSET_BITS
00196 #if _FILE_OFFSET_BITS==64
00197 /*:[19mar2004 mt]*/
00198
00199 #define ADDPOS(pp,x) (pp).p1 = ((pp).p1+(off_t)(x))
00200 #define SETBASELENGTH(ss,x) (ss).p1 = (off_t)(x)
00201 #define SETBASEPOSITION(pp,x) (pp).p1 = (off_t)(x)
00202 #define ISEQUALPOSINC(pp1,pp2,x) ( (pp1).p1 == ((pp2).p1+(off_t)(x)) )
00203 #define ISGEPOSINC(pp1,pp2,x) ( (pp1).p1 >= ((pp2).p1+(off_t)(x)) )
00204 #define DIVPOS(pp,n) ( (pp).p1/(off_t)(n) )
00205 #define MULPOS(pp,n) (pp).p1 *= (off_t)(n)
00206
00207 #else
00208
00209 #define ADDPOS(pp,x) (pp).p1 = ((pp).p1+(x))
00210 #define SETBASELENGTH(ss,x) (ss).p1 = (x)
00211 #define SETBASEPOSITION(pp,x) (pp).p1 = (x)
00212 #define ISEQUALPOSINC(pp1,pp2,x) ( (pp1).p1 == ((pp2).p1+(LONG)(x)) )
00213 #define ISGEPOSINC(pp1,pp2,x) ( (pp1).p1 >= ((pp2).p1+(LONG)(x)) )
00214 #define DIVPOS(pp,n) ( (pp).p1/(n) )
00215 #define MULPOS(pp,n) (pp).p1 *= (n)
00216 #endif
00217 #else
00218
00219 #define ADDPOS(pp,x) (pp).p1 = ((pp).p1+(LONG)(x))
00220 #define SETBASELENGTH(ss,x) (ss).p1 = (LONG)(x)
00221 #define SETBASEPOSITION(pp,x) (pp).p1 = (LONG)(x)
00222 #define ISEQUALPOSINC(pp1,pp2,x) ( (pp1).p1 == ((pp2).p1+(LONG)(x)) )
00223 #define ISGEPOSINC(pp1,pp2,x) ( (pp1).p1 >= ((pp2).p1+(LONG)(x)) )
00224 #define DIVPOS(pp,n) ( (pp).p1/(LONG)(n) )
00225 #define MULPOS(pp,n) (pp).p1 *= (LONG)(n)
00226
00227 #endif
00228 #define DIFPOS(ss,pp1,pp2) (ss).p1 = ((pp1).p1-(pp2).p1)
00229 #define DIFBASE(pp1,pp2) ((pp1).p1-(pp2).p1)
00230 #define ADD2POS(pp1,pp2) (pp1).p1 += (pp2).p1
00231 #define PUTZERO(pp) (pp).p1 = 0
00232 #define BASEPOSITION(pp) ((pp).p1)
00233 #define SETSTARTPOS(pp) (pp).p1 = -2
00234 #define NOTSTARTPOS(pp) ( (pp).p1 > -2 )
00235 #define ISMINPOS(pp) ( (pp).p1 == -1 )
00236 #define ISEQUALPOS(pp1,pp2) ( (pp1).p1 == (pp2).p1 )
00237 #define ISNOTEQUALPOS(pp1,pp2) ( (pp1).p1 != (pp2).p1 )
00238 #define ISLESSPOS(pp1,pp2) ( (pp1).p1 < (pp2).p1 )
00239 #define ISGEPOS(pp1,pp2) ( (pp1).p1 >= (pp2).p1 )
00240 #define ISNOTZEROPOS(pp) ( (pp).p1 != 0 )
00241 #define ISZEROPOS(pp) ( (pp).p1 == 0 )
00242 #define ISPOSPOS(pp) ( (pp).p1 > 0 )
00243 #define ISNEGPOS(pp) ( (pp).p1 < 0 )
00244 extern void TELLFILE(int,POSITION *);
00245
00246 #define TOLONG(x) ((LONG)(x))
00247
00248 #define Add2Com(x) { WORD cod[2]; cod[0] = x; cod[1] = 2; AddNtoL(2,cod); }
00249 #define Add3Com(x1,x2) { WORD cod[3]; cod[0] = x1; cod[1] = 3; cod[2] = x2; AddNtoL(3,cod); }
00250 #define Add4Com(x1,x2,x3) { WORD cod[4]; cod[0] = x1; cod[1] = 4; \
00251     cod[2] = x2; cod[3] = x3; AddNtoL(4,cod); }
00252 #define Add5Com(x1,x2,x3,x4) { WORD cod[5]; cod[0] = x1; cod[1] = 5; \
00253     cod[2] = x2; cod[3] = x3; cod[4] = x4; AddNtoL(5,cod); }
00254
00255 /*
00256     The temporary variable ppp is to avoid a compiler warning about strict aliasing
00257 */
00258 #define WantAddPointers(x) while((AT.pWorkPointer+(x))>AR.pWorkSize){WORD ***ppp=&AT.pWorkSpace;\
00259     ExpandBuffer((void **)ppp,&AR.pWorkSize,(int)(sizeof(WORD *)));}
00260 #define WantAddLongs(x) while((AT.lWorkPointer+(x))>AR.lWorkSize){LONG **ppp=&AT.lWorkSpace;\
00261     ExpandBuffer((void **)ppp,&AR.lWorkSize,sizeof(LONG));}
00262 #define WantAddPositions(x) while((AT.posWorkPointer+(x))>AR.posWorkSize){POSITION
00263     **ppp=&AT.posWorkSpace;\
00264     ExpandBuffer((void **)ppp,&AR.posWorkSize,sizeof(POSITION));}
00265
00266 /* inline in form3.h (or config.h). */
00267 #define FORM_INLINE inline

```

```

00267
00268 /*
00269     Macro's for memory management. This can be done by routines, but that
00270     would be slower. Inline routines could do this, but we don't want to
00271     leave this to the friendliness of the compiler(s).
00272     The routines can be found in the file tools.c
00273 */
00274 #define MEMORYMACROS
00275
00276 #ifndef MEMORYMACROS
00277
00278 #define TermMalloc(x) ( (AT.TermMemTop <= 0) ? TermMallocAddMemory(BHEAD0),
AT.TermMemHeap[--AT.TermMemTop]: AT.TermMemHeap[--AT.TermMemTop] )
00279 #define NumberMalloc(x) ( (AT.NumberMemTop <= 0) ? NumberMallocAddMemory(BHEAD0),
AT.NumberMemHeap[--AT.NumberMemTop]: AT.NumberMemHeap[--AT.NumberMemTop] )
00280 #define CacheNumberMalloc(x) ( (AT.CacheNumberMemTop <= 0) ? CacheNumberMallocAddMemory(BHEAD0),
AT.CacheNumberMemHeap[--AT.CacheNumberMemTop]: AT.CacheNumberMemHeap[--AT.CacheNumberMemTop] )
00281 #define TermFree(TermMem,x) AT.TermMemHeap[AT.TermMemTop++] = (WORD *) (TermMem)
00282 #define NumberFree(NumberMem,x) AT.NumberMemHeap[AT.NumberMemTop++] = (UWORD *) (NumberMem)
00283 #define CacheNumberFree(NumberMem,x) AT.CacheNumberMemHeap[AT.CacheNumberMemTop++] = (UWORD
*) (NumberMem)
00284
00285 #else
00286
00287 #define TermMalloc(x) TermMalloc2(BHEAD (char *) (x))
00288 #define NumberMalloc(x) NumberMalloc2(BHEAD (char *) (x))
00289 #define CacheNumberMalloc(x) CacheNumberMalloc2(BHEAD (char *) (x))
00290 #define TermFree(x,y) TermFree2(BHEAD (WORD *) (x), (char *) (y))
00291 #define NumberFree(x,y) NumberFree2(BHEAD (UWORD *) (x), (char *) (y))
00292 #define CacheNumberFree(x,y) CacheNumberFree2(BHEAD (UWORD *) (x), (char *) (y))
00293
00294 #endif
00295
00296 /*
00297 * Macros for checking nesting levels in the compiler, used as follows:
00298 *
00299 *   AC.IfSumCheck[AC.IfLevel] = NestingChecksum();
00300 *   AC.IfLevel++;
00301 *
00302 *   AC.IfLevel--;
00303 *   if ( AC.IfSumCheck[AC.IfLevel] != NestingChecksum() ) {
00304 *       MesNesting();
00305 *   }
00306 *
00307 * Note that NestingChecksum() also contains AC.IfLevel and so in this case
00308 * using increment/decrement operators on it in the left-hand side may be
00309 * confusing.
00310 */
00311 #define NestingChecksum() (AC.IfLevel + AC.RepLevel + AC.arglevel + AC.insidelevel + AC.termlevel +
AC.inexprlevel + AC.dolooplevel + AC.SwitchLevel)
00312 #define MesNesting() MesPrint("&Illegal nesting of if, repeat, argument, inside, term, inexpression
and do")
00313
00314 #define MarkPolyRatFunDirty(T) {if(*T&&AR.PolyFunType==2){WORD *TP,*TT;TT=T+*T;TT-=ABS(TT[-1]);\
TP=T+1;while(TP<TT){if(*TP==AR.PolyFun){TP[2]|=(DIRTYFLAG|MUSTCLEANPRF);}TP+=TP[1];}}
00315
00316
00317 /*
00318     Macros for nesting input levels for #name = ...; assign instructions.
00319     Note that the level should never go below zero.
00320 */
00321 #define PUSHPREASSIGNLEVEL AP.PreAssignLevel++; { GETIDENTITY \
00322     if ( AP.PreAssignLevel >= AP.MaxPreAssignLevel ) { int i; \
00323         LONG *ap = (LONG *) Malloc1(2*AP.MaxPreAssignLevel*sizeof(LONG *),"PreAssignStack"); \
00324         for ( i = 0; i < AP.MaxPreAssignLevel; i++ ) ap[i] = AP.PreAssignStack[i]; \
00325         M_free(AP.PreAssignStack,"PreAssignStack"); \
00326         AP.MaxPreAssignLevel *= 2; AP.PreAssignStack = ap; \
00327     } \
00328     *AT.WorkPointer++ = AP.PreContinuation; AP.PreContinuation = 0; \
00329     AP.PreAssignStack[AP.PreAssignLevel] = AC.iPointer - AC.iBuffer; }
00330
00331 #define POPPREASSIGNLEVEL if ( AP.PreAssignLevel > 0 ) { GETIDENTITY \
00332     AC.iPointer = AC.iBuffer + AP.PreAssignStack[AP.PreAssignLevel--]; \
00333     AP.PreContinuation = *--AT.WorkPointer; \
00334     *AC.iPointer = 0; }
00335
00336 #ifndef WITHFLOAT
00337 /*
00338     The following macro's are needed to avoid problems with the compilers
00339     and gmp.h. For the C++ files we need the Form include files to be inside
00340     and extern C {} environment, but then the structs.h needs gmp.h to
00341     recognise the mpf_t datatype. This causes no end of problems.
00342     Hence we collect the sensitive objects as (void *) and cast them to
00343     something usable with the macro's below. This way the gmp.h file
00344     needs to be included only in a very limited number of .c files.
00345 */
00346 #define mpftab1 ((mpf_t *) (AT.mpf_tab1))
00347 #define mpftab2 ((mpf_t *) (AT.mpf_tab2))

```

```

00348 #define mpfaux_ ((mpf_t *) (AT.aux_))
00349 #define aux1 ((mpf_t *) (AT.aux_)) [0]
00350 #define aux2 ((mpf_t *) (AT.aux_)) [1]
00351 #define aux3 ((mpf_t *) (AT.aux_)) [2]
00352 #define aux4 ((mpf_t *) (AT.aux_)) [3]
00353 #define aux5 ((mpf_t *) (AT.aux_)) [4]
00354 #define auxjm ((mpf_t *) (AT.aux_)) [5]
00355 #define auxjjm ((mpf_t *) (AT.aux_)) [6]
00356 #define auxsum ((mpf_t *) (AT.aux_)) [7]
00357
00358 #define mpfdelta1 ((mpf_t *) (AS.delta_1)) [0]
00359
00360 #endif
00361
00362
00363 /*
00364     MesPrint("P-level popped to %d with %d", AP.PreAssignLevel, (WORD) (AC.iPointer - AC.iBuffer));
00365
00366     #] Macro's :
00367     #[ Inline functions :
00368 */
00369
00370 /*
00371 * The following three functions give the unsigned absolute value of a signed
00372 * integer even for the most negative integer. This is beyond the scope of
00373 * the standard abs() function and its family, whose return-values are signed.
00374 * In short, we should not use the unary minus operator with signed numbers
00375 * unless we are sure that there are no integer overflows. Instead, we rely on
00376 * two well-defined operations: (i) signed-to-unsigned conversion and
00377 * (ii) unary minus of unsigned operands.
00378 *
00379 * See also:
00380 *   https://stackoverflow.com/a/4536188 (Unary minus and signed-to-unsigned conversion)
00381 *   https://stackoverflow.com/q/8026694 (C: unary minus operator behavior with unsigned operands)
00382 *   https://stackoverflow.com/q/1610947 (Why does stdlib.h's abs() family of functions return a
signed value?)
00383 *   https://blog.regehr.org/archives/226 (A Guide to Undefined Behavior in C and C++, Part 2)
00384 */
00385 static inline unsigned int IntAbs(int x)
00386 {
00387     if ( x >= 0 ) return x;
00388     return(-(unsigned int)x);
00389 }
00390
00391 static inline UWORD WordAbs(WORD x)
00392 {
00393     if ( x >= 0 ) return x;
00394     return(-(UWORD)x);
00395 }
00396
00397 static inline ULONG LongAbs(LONG x)
00398 {
00399     if ( x >= 0 ) return x;
00400     return(-(ULONG)x);
00401 }
00402
00403 /*
00404 * The following functions provide portable unsigned-to-signed conversions
00405 * (to avoid the implementation-defined behaviour), which is expected to be
00406 * optimized to a no-op.
00407 *
00408 * See also:
00409 *   https://stackoverflow.com/a/13208789 (Efficient unsigned-to-signed cast avoiding
implementation-defined behavior)
00410 */
00411 static inline int UnsignedToInt(unsigned int x)
00412 {
00413     extern void TerminateImpl(int, const char*, int, const char*);
00414     if ( x <= INT_MAX ) return(x);
00415     if ( x >= (unsigned int)INT_MIN )
00416         return((int)(x - (unsigned int)INT_MIN) + INT_MIN);
00417     Terminate(1);
00418     return(0);
00419 }
00420
00421 static inline WORD UWordToWord(UWORD x)
00422 {
00423     extern void TerminateImpl(int, const char*, int, const char*);
00424     if ( x <= WORD_MAX_VALUE ) return(x);
00425     if ( x >= (UWORD)WORD_MIN_VALUE )
00426         return((WORD)(x - (UWORD)WORD_MIN_VALUE) + WORD_MIN_VALUE);
00427     Terminate(1);
00428     return(0);
00429 }
00430
00431 static inline LONG ULongToLong(ULONG x)
00432 {

```

```

00433     extern void TerminateImpl(int, const char*, int, const char*);
00434     if ( x <= LONG_MAX_VALUE ) return(x);
00435     if ( x >= (ULONG)LONG_MIN_VALUE )
00436         return((LONG)(x - (ULONG)LONG_MIN_VALUE) + LONG_MIN_VALUE);
00437     Terminate(1);
00438     return(0);
00439 }
00440
00441 /*
00442     #] Inline functions :
00443     #[ Thread objects :
00444 */
00445
00452 #ifdef WITHPTHREADS
00453
00454 #define EXTERNLOCK(x) extern pthread_mutex_t x;
00455 #define INILOCK(x) pthread_mutex_t x = PTHREAD_MUTEX_INITIALIZER;
00456 #define EXTERNRWLOCK(x) extern pthread_rwlock_t x;
00457 #define INIRWLOCK(x) pthread_rwlock_t x = PTHREAD_RWLOCK_INITIALIZER;
00458 #ifndef DEBUGGINGLOCKS
00459 #include <asm/errno.h>
00460 #define LOCK(x) while ( pthread_mutex_trylock(&(x)) == EBUSY ) {}
00461 #define RWLOCKR(x) while ( pthread_rwlock_tryrdlock(&(x)) == EBUSY ) {}
00462 #define RWLOCKW(x) while ( pthread_rwlock_trywrlock(&(x)) == EBUSY ) {}
00463 #else
00464 #define LOCK(x) pthread_mutex_lock(&(x))
00465 #define RWLOCKR(x) pthread_rwlock_rdlock(&(x))
00466 #define RWLOCKW(x) pthread_rwlock_wrlock(&(x))
00467 #endif
00468 #define UNLOCK(x) pthread_mutex_unlock(&(x))
00469 #define UNRWLOCK(x) pthread_rwlock_unlock(&(x))
00470 #define MLOCK(x) LOCK(x)
00471 #define MUNLOCK(x) UNLOCK(x)
00472
00473 #define GETBIDENTITY
00474 #define GETIDENTITY int identity = WhoAmI(); ALLPRIVATES *B = AB[identity];
00475 #else
00476
00477 #define EXTERNLOCK(x)
00478 #define INILOCK(x)
00479 #define LOCK(x)
00480 #define UNLOCK(x)
00481 #define EXTERNRWLOCK(x)
00482 #define INIRWLOCK(x)
00483 #define RWLOCKR(x)
00484 #define RWLOCKW(x)
00485 #define UNRWLOCK(x)
00486 #ifndef WITHMPI
00487 #define MLOCK(x) do { if ( PF.me != MASTER ) PF_MLock(); } while (0)
00488 #define MUNLOCK(x) do { if ( PF.me != MASTER ) PF_MUnlock(); } while (0)
00489 #else
00490 #define MLOCK(x)
00491 #define MUNLOCK(x)
00492 #endif
00493 #define GETIDENTITY
00494 #define GETBIDENTITY
00495
00496 #endif
00497
00498 /*
00499     #] Thread objects :
00500     #[ Declarations :
00501 */
00502
00503 #ifdef TERMMALLOCDEBUG
00504 extern WORD **DebugHeap1, **DebugHeap2;
00505 #endif
00506
00511 extern void StartVariables(void);
00512 extern void setSignalHandlers(void);
00513 extern UBYTE *CodeToLine(WORD, UBYTE *);
00514 extern UBYTE *AddArrayIndex(WORD, UBYTE *);
00515 extern INDEXENTRY *FindInIndex(WORD, FILEDATA *, WORD, WORD);
00516 extern INDEXENTRY *NextFileIndex(POSITION *);
00517 extern WORD *PasteTerm(PHEAD WORD, WORD *, WORD *, WORD, WORD);
00518 extern UBYTE *StrCopy(UBYTE *, UBYTE *);
00519 extern UBYTE *WrtPower(UBYTE *, WORD);
00520 extern WORD AccumGCD(PHEAD UWORD *, WORD *, UWORD *, WORD);
00521 extern void AddArgs(PHEAD WORD *, WORD *, WORD *);
00522 extern WORD AddCoef(PHEAD WORD **, WORD **);
00523 extern WORD AddLong(UWORD *, WORD, UWORD *, WORD, UWORD *, WORD *);
00524 extern WORD AddPLon(UWORD *, WORD, UWORD *, WORD, UWORD *, WORD *);
00525 extern WORD AddPoly(PHEAD WORD **, WORD **);
00526 extern WORD AddRat(PHEAD UWORD *, WORD, UWORD *, WORD, UWORD *, WORD *);
00527 extern void AddToLine(UBYTE *);
00528 extern WORD AddWild(PHEAD WORD, WORD, WORD);
00529 extern WORD BigLong(UWORD *, WORD, UWORD *, WORD);

```

```

00530 extern WORD BinomGen(PHEAD WORD *, WORD, WORD **, WORD, WORD, WORD, WORD, WORD, UWORD *, WORD);
00531 extern WORD CheckWild(PHEAD WORD, WORD, WORD, WORD *);
00532 extern WORD Chisholm(PHEAD WORD *, WORD);
00533 extern WORD CleanExpr(WORD);
00534 extern void CleanUp(WORD);
00535 extern void ClearWild(PHEAD0);
00536 extern WORD CompareFunctions(WORD *, WORD *);
00537 extern WORD Commute(WORD *, WORD *);
00538 extern WORD DetCommu(WORD *);
00539 extern WORD DoesCommu(WORD *);
00540 extern int CompArg(WORD *, WORD *);
00541 extern WORD CompCoeef(WORD *, WORD *);
00542 extern WORD CompGroup(PHEAD WORD, WORD **, WORD *, WORD *, WORD);
00543 extern WORD Compare1(PHEAD WORD *, WORD *, WORD);
00544 extern WORD CountDo(WORD *, WORD *);
00545 extern WORD CountFun(WORD *, WORD *);
00546 extern WORD DimensionSubterm(WORD *);
00547 extern WORD DimensionTerm(WORD *);
00548 extern WORD DimensionExpression(PHEAD WORD *);
00549 extern WORD Deferred(PHEAD WORD *, WORD);
00550 extern WORD DeleteStore(WORD);
00551 extern WORD DetCurDum(PHEAD WORD *);
00552 extern void DetVars(WORD *, WORD);
00553 extern WORD Distribute(DISTRIBUTE *, WORD);
00554 extern WORD DivLong(UWORD *, WORD, UWORD *, WORD, UWORD *, WORD *, UWORD *, WORD *);
00555 extern WORD DivRat(PHEAD UWORD *, WORD, UWORD *, WORD, UWORD *, WORD *);
00556 extern WORD Divvy(PHEAD UWORD *, WORD *, UWORD *, WORD);
00557 extern WORD DoDelta(WORD *);
00558 extern WORD DoDelta3(PHEAD WORD *, WORD);
00559 extern WORD TestPartitions(WORD *, PARTI *);
00560 extern WORD DoPartitions(PHEAD WORD *, WORD);
00561 extern int CoCanonicalize(UBYTE *);
00562 extern int DoCanonicalize(PHEAD WORD *, WORD *);
00563 extern WORD GenTopologies(PHEAD WORD *, WORD);
00564 extern WORD GenDiagrams(PHEAD WORD *, WORD);
00565 extern int DoTopologyCanonicalize(PHEAD WORD *, WORD, WORD, WORD *);
00566 extern int DoShattering(PHEAD WORD *, WORD *, WORD *, WORD);
00567 extern WORD GenerateTopologies(PHEAD WORD, WORD, WORD, WORD);
00568 extern WORD DoTableExpansion(WORD *, WORD);
00569 extern WORD DoDistrib(PHEAD WORD *, WORD);
00570 extern WORD DoShuffle(WORD *, WORD, WORD, WORD);
00571 extern WORD DoPermutations(PHEAD WORD *, WORD);
00572 extern int Shuffle(WORD *, WORD *, WORD *);
00573 extern int FinishShuffle(WORD *);
00574 extern WORD DoStuffle(WORD *, WORD, WORD, WORD);
00575 extern int Stuffle(WORD *, WORD *, WORD *);
00576 extern int FinishStuffle(WORD *);
00577 extern WORD *StuffRootAdd(WORD *, WORD *, WORD *);
00578 extern WORD TestUse(WORD *, WORD);
00579 extern DBASE *FindTB(UBYTE *);
00580 extern int CheckTableDeclarations(DBASE *);
00581 extern WORD Apply(WORD *, WORD);
00582 extern int ApplyExec(WORD *, int, WORD);
00583 extern WORD ApplyReset(WORD);
00584 extern WORD TableReset(void);
00585 extern void ReWorkT(WORD *, WORD *, WORD);
00586 extern WORD GetIfDollarNum(WORD *, WORD *);
00587 extern int FindVar(WORD *, WORD *);
00588 extern WORD DoIfStatement(PHEAD WORD *, WORD *);
00589 extern WORD DoOnePow(PHEAD WORD *, WORD, WORD, WORD *, WORD *, WORD, WORD *);
00590 extern void DoRevert(WORD *, WORD *);
00591 extern WORD DoSumF1(PHEAD WORD *, WORD *, WORD, WORD);
00592 extern WORD DoSumF2(PHEAD WORD *, WORD *, WORD, WORD);
00593 extern WORD DoTheta(PHEAD WORD *);
00594 extern LONG EndSort(PHEAD WORD *, int);
00595 extern WORD EntVar(WORD, UBYTE *, WORD, WORD, WORD, WORD);
00596 extern WORD EpfCon(PHEAD WORD *, WORD *, WORD, WORD);
00597 extern WORD EpfFind(PHEAD WORD *, WORD *);
00598 extern WORD EpfGen(WORD, WORD *, WORD *, WORD *, WORD);
00599 extern WORD EqualArg(WORD *, WORD, WORD);
00600 extern WORD Evaluate(UBYTE **);
00601 extern int Factorial(PHEAD WORD, UWORD *, WORD *);
00602 extern int Bernoulli(WORD, UWORD *, WORD *);
00603 extern int FactorIn(PHEAD WORD *, WORD);
00604 extern int FactorInExpr(PHEAD WORD *, WORD);
00605 extern WORD FindAll(PHEAD WORD *, WORD *, WORD, WORD *);
00606 extern WORD FindMulti(PHEAD WORD *, WORD *);
00607 extern WORD FindOnce(PHEAD WORD *, WORD *);
00608 extern WORD FindOnly(PHEAD WORD *, WORD *);
00609 extern WORD FindRest(PHEAD WORD *, WORD *);
00610 extern WORD FindSpecial(WORD *);
00611 extern WORD FindrNumber(WORD, VARRENUM *);
00612 extern void FiniLine(void);
00613 extern WORD FiniTerm(PHEAD WORD *, WORD *, WORD *, WORD, WORD);
00614 extern WORD FlushOut(POSITION *, FILEHANDLE *, int);
00615 extern void FunLevel(PHEAD WORD *);
00616 extern void AdjustRenumScratch(PHEAD0);

```



```

00617 extern void GarbHand(void);
00618 extern WORD GcdLong(PHEAD UWORD *,WORD,UWORD *,WORD,UWORD *,WORD *);
00619 extern WORD LcmLong(PHEAD UWORD *,WORD,UWORD *,WORD,UWORD *,WORD *);
00620 extern void GCD(UWORD *,WORD,UWORD *,WORD,UWORD *,WORD *);
00621 extern ULONG GCD2(ULONG,ULONG);
00622 extern WORD Generator(PHEAD WORD *,WORD);
00623 extern WORD GetBinom(UWORD *,WORD *,WORD,WORD);
00624 extern WORD GetFromStore(WORD *,POSITION *,RENUMBER,WORD *,WORD);
00625 extern WORD GetLong(UBYTE *,UWORD *,WORD *);
00626 extern WORD GetMoreTerms(WORD *);
00627 extern WORD GetMoreFromMem(WORD *,WORD **);
00628 extern WORD GetOneTerm(PHEAD WORD *,FILEHANDLE *,POSITION *,int);
00629 extern RENUMBER GetTable(WORD,POSITION *,WORD);
00630 extern WORD GetTerm(PHEAD WORD *);
00631 extern WORD Glue(PHEAD WORD *,WORD *,WORD *,WORD);
00632 extern WORD InFunction(PHEAD WORD *,WORD *);
00633 extern void IniLine(WORD);
00634 extern WORD IniVars(void);
00635 extern void Initialize(void);
00636 extern WORD InsertTerm(PHEAD WORD *,WORD,WORD,WORD *,WORD *,WORD);
00637 extern void LongToLine(UWORD *,WORD);
00638 extern WORD MakeDirty(WORD *,WORD *,WORD);
00639 extern void MarkDirty(WORD *,WORD);
00640 extern void PolyFunDirty(PHEAD WORD *);
00641 extern void PolyFunClean(PHEAD WORD *);
00642 extern WORD MakeModTable(void);
00643 extern WORD MatchE(PHEAD WORD *,WORD *,WORD *,WORD);
00644 extern int MatchCy(PHEAD WORD *,WORD *,WORD *,WORD);
00645 extern int FunMatchCy(PHEAD WORD *,WORD *,WORD *,WORD);
00646 extern int FunMatchSy(PHEAD WORD *,WORD *,WORD *,WORD);
00647 extern int MatchArgument(PHEAD WORD *,WORD *);
00648 extern WORD MatchFunction(PHEAD WORD *,WORD *,WORD *);
00649 extern WORD MergePatches(WORD);
00650 extern WORD MesCerr(char *, UBYTE *);
00651 extern WORD MesComp(char *, UBYTE *, UBYTE *);
00652 extern WORD Modulus(WORD *);
00653 extern void MoveDummies(PHEAD WORD *,WORD);
00654 extern WORD MullLong(UWORD *,WORD,UWORD *,WORD,UWORD *,WORD *);
00655 extern WORD MulRat(PHEAD UWORD *,WORD,UWORD *,WORD,UWORD *,WORD *);
00656 extern WORD Mully(PHEAD UWORD *,WORD *,UWORD *,WORD);
00657 extern WORD MultDo(PHEAD WORD *,WORD *);
00658 extern WORD NewSort(PHEAD0);
00659 extern WORD ExtraSymbol(WORD,WORD,WORD,WORD *,WORD *);
00660 extern WORD Normalize(PHEAD WORD *);
00661 extern WORD BracketNormalize(PHEAD WORD *);
00662 extern void DropCoefficient(PHEAD WORD *);
00663 extern void DropSymbols(PHEAD WORD *);
00664 extern int PutInside(PHEAD WORD *, WORD *);
00665 extern WORD OpenTemp(void);
00666 extern void Pack(UWORD *,WORD *,UWORD *,WORD );
00667 extern LONG PasteFile(PHEAD WORD,WORD *,POSITION *,WORD **,RENUMBER,WORD *,WORD);
00668 extern WORD Permute(PERM *,WORD);
00669 extern WORD PermuteP(PERMP *,WORD);
00670 extern WORD PolyFunMul(PHEAD WORD *);
00671 extern WORD PopVariables(void);
00672 extern WORD PrepPoly(PHEAD WORD *,WORD);
00673 extern WORD Processor(void);
00674 extern WORD Product(UWORD *,WORD *,WORD);
00675 extern void PrtLong(UWORD *,WORD,UBYTE *);
00676 extern void PrtTerms(void);
00677 extern void PrintDeprecation(const char *,const char *);
00678 extern void PrintRunningTime(void);
00679 extern LONG GetRunningTime(void);
00680 extern WORD PutBracket(PHEAD WORD *);
00681 extern LONG PutIn(FILEHANDLE *,POSITION *,WORD *,WORD **,int);
00682 extern WORD PutInStore(INDEXENTRY *,WORD);
00683 extern WORD PutOut(PHEAD WORD *,POSITION *,FILEHANDLE *,WORD);
00684 extern UWORD Quotient(UWORD *,WORD *,WORD);
00685 extern WORD RaisPow(PHEAD UWORD *,WORD *,UWORD);
00686 extern void RaisPowCached(PHEAD WORD,WORD, UWORD **, WORD *);
00687 extern WORD RaisPowMod(WORD, WORD, WORD);
00688 extern int NormalModulus(UWORD *,WORD *);
00689 extern int MakeInverses(void);
00690 extern int GetModInverses(WORD,WORD,WORD *,WORD *);
00691 extern int GetLongModInverses(PHEAD UWORD *, WORD, UWORD *, WORD, UWORD *, WORD *, UWORD *, WORD
*);
00692 extern void RatToLine(UWORD *,WORD);
00693 extern WORD RatioFind(PHEAD WORD *,WORD *);
00694 extern WORD RatioGen(PHEAD WORD *,WORD *,WORD,WORD);
00695 extern WORD ReNumber(PHEAD WORD *);
00696 extern WORD ReadSnum(UBYTE **);
00697 extern WORD Remain10(UWORD *,WORD *);
00698 extern WORD Remain4(UWORD *,WORD *);
00699 extern WORD ResetScratch(void);
00700 extern WORD ResolveSet(PHEAD WORD *,WORD *,WORD *);
00701 extern WORD RevertScratch(void);
00702 extern WORD ScanFunctions(PHEAD WORD *,WORD *,WORD);

```



```

00703 extern void SeekScratch(FILEHANDLE *, POSITION *);
00704 extern void SetEndScratch(FILEHANDLE *, POSITION *);
00705 extern void SetEndHScratch(FILEHANDLE *, POSITION *);
00706 extern WORD SetFileIndex(void);
00707 extern WORD Sflush(FILEHANDLE *);
00708 extern WORD Simplify(PHEAD UWORD *, WORD *, UWORD *, WORD *);
00709 extern WORD SortWild(WORD *, WORD);
00710 extern FILE *LocateBase(char **, char **, char *);
00711 extern LONG SplitMerge(PHEAD WORD **, LONG);
00712 extern WORD StoreTerm(PHEAD WORD *);
00713 extern void SubPLon(UWORD *, WORD, UWORD *, WORD, UWORD *, WORD *);
00714 extern void Substitute(PHEAD WORD *, WORD *, WORD);
00715 extern WORD SymFind(PHEAD WORD *, WORD *);
00716 extern WORD SymGen(PHEAD WORD *, WORD *, WORD, WORD);
00717 extern WORD Symmetrize(PHEAD WORD *, WORD *, WORD, WORD, WORD);
00718 extern int FullSymmetrize(PHEAD WORD *, int);
00719 extern WORD TakeModulus(UWORD *, WORD *, UWORD *, WORD, WORD);
00720 extern WORD TakeNormalModulus(UWORD *, WORD *, UWORD *, WORD, WORD);
00721 extern void TalToLine(UWORD);
00722 extern WORD TenVec(PHEAD WORD *, WORD *, WORD, WORD);
00723 extern WORD TenVecFind(PHEAD WORD *, WORD *);
00724 extern WORD TermRenumber(WORD *, RENUMBER, WORD);
00725 extern void TestDrop(void);
00726 extern void PutInVflags(WORD);
00727 extern WORD TestMatch(PHEAD WORD *, WORD *);
00728 extern WORD TestSub(PHEAD WORD *, WORD);
00729 extern LONG TimeCPU(WORD);
00730 extern LONG TimeChildren(WORD);
00731 extern LONG TimeWallClock(WORD);
00732 extern LONG Timer(int);
00733 extern int GetTimerInfo(LONG **, LONG **);
00734 extern void WriteTimerInfo(LONG *, LONG *);
00735 extern LONG GetWorkerTimes(void);
00736 extern WORD ToStorage(EXPRESSIONS, POSITION *);
00737 extern void TokenToLine(UBYTE *);
00738 extern WORD Trace4(PHEAD WORD *, WORD *, WORD, WORD);
00739 extern WORD Trace4Gen(PHEAD TRACES *, WORD);
00740 extern WORD Trace4no(WORD, WORD *, TRACES *);
00741 extern WORD TraceFind(PHEAD WORD *, WORD *);
00742 extern WORD TraceN(PHEAD WORD *, WORD *, WORD, WORD);
00743 extern WORD TraceNgen(PHEAD TRACES *, WORD);
00744 extern WORD TraceNno(WORD, WORD *, TRACES *);
00745 extern WORD Traces(PHEAD WORD *, WORD *, WORD, WORD);
00746 extern WORD Trick(WORD *, TRACES *);
00747 extern WORD TryDo(PHEAD WORD *, WORD *, WORD);
00748 extern void UnPack(UWORD *, WORD, WORD *, WORD *);
00749 extern WORD VarStore(UBYTE *, WORD, WORD, WORD);
00750 extern WORD WildFill(PHEAD WORD *, WORD *, WORD *);
00751 extern WORD WriteAll(void);
00752 extern WORD WriteOne(UBYTE *, int, int, WORD);
00753 extern void WriteArgument(WORD *);
00754 extern WORD WriteExpression(WORD *, LONG);
00755 extern WORD WriteInnerTerm(WORD *, WORD);
00756 extern void WriteLists(void);
00757 extern void WriteSetup(void);
00758 extern void WriteStats(POSITION *, WORD, WORD);
00759 extern WORD WriteSubTerm(WORD *, WORD);
00760 extern WORD WriteTerm(WORD *, WORD *, WORD, WORD, WORD);
00761 extern WORD execarg(PHEAD WORD *, WORD);
00762 extern WORD execterm(PHEAD WORD *, WORD);
00763 extern void SpecialCleanup(PHEAD0);
00764 extern void SetMods(void);
00765 extern void UnSetMods(void);
00766
00767 /*-----*/
00768
00769 extern WORD DoExecute(WORD, WORD);
00770 extern void SetScratch(FILEHANDLE *, POSITION *);
00771 extern void Warning(char *);
00772 extern void HighWarning(char *);
00773 extern int SpareTable(TABLES);
00774
00775 extern UBYTE *strDup1(UBYTE *, char *);
00776 extern void *Malloc(LONG);
00777 extern void *Malloc1(LONG, const char *);
00778 extern int DoTail(int, UBYTE **);
00779 extern int OpenInput(void);
00780 extern int PutPreVar(UBYTE *, UBYTE *, UBYTE *, int);
00781 extern void Error0(char *);
00782 extern void Error1(char *, UBYTE *);
00783 extern void Error2(char *, char *, UBYTE *);
00784 extern UBYTE ReadFromStream(STREAM *);
00785 extern UBYTE GetFromStream(STREAM *);
00786 extern UBYTE LookInStream(STREAM *);
00787 extern STREAM *OpenStream(UBYTE *, int, int, int);
00788 extern int LocateFile(UBYTE **, int);
00789 extern STREAM *CloseStream(STREAM *);

```

```

00790 extern void      PositionStream(STREAM *,LONG);
00791 extern int        ReverseStatements(STREAM *);
00792 extern int        ProcessOption(UBYTE *,UBYTE *,int);
00793 extern int        DoSetups(void);
00794 extern void      TerminateImpl(int, const char *,int, const char *);
00795 extern NAMENODE *GetNode(NAMETREE *,UBYTE *);
00796 extern int        AddName(NAMETREE *,UBYTE *,WORD,WORD,int *);
00797 extern int        GetName(NAMETREE *,UBYTE *,WORD *,int);
00798 extern UBYTE      *GetFunction(UBYTE *,WORD *);
00799 extern UBYTE      *GetNumber(UBYTE *,WORD *);
00800 extern int        GetLastExprName(UBYTE *,WORD *);
00801 extern int        GetAutoName(UBYTE *,WORD *);
00802 extern int        GetVar(UBYTE *,WORD *,WORD *,int,int);
00803 extern int        MakeDubious(NAMETREE *,UBYTE *,WORD *);
00804 extern int        GetOName(NAMETREE *,UBYTE *,WORD *,int);
00805 extern void      DumpTree(NAMETREE *);
00806 extern void      DumpNode(NAMETREE *,WORD,WORD);
00807 extern void      LinkTree(NAMETREE *,WORD,WORD);
00808 extern void      CopyTree(NAMETREE *,NAMETREE *,WORD,WORD);
00809 extern int        CompactifyTree(NAMETREE *,WORD);
00810 extern NAMETREE *MakeNameTree(void);
00811 extern void      FreeNameTree(NAMETREE *);
00812 extern int        AddExpression(UBYTE *,int,int);
00813 extern int        AddSymbol(UBYTE *,int,int,int,int);
00814 extern int        AddDollar(UBYTE *,WORD,WORD *,LONG);
00815 extern int        ReplaceDollar(WORD,WORD,WORD *,LONG);
00816 extern int        DollarRaiseLow(UBYTE *,LONG);
00817 extern int        AddVector(UBYTE *,int,int);
00818 extern int        AddDubious(UBYTE *);
00819 extern int        AddIndex(UBYTE *,int,int);
00820 extern UBYTE      *DoDimension(UBYTE *,int *,int *);
00821 extern int        AddFunction(UBYTE *,int,int,int,int,int,int,int);
00822 extern int        CoCommuteInSet(UBYTE *);
00823 extern int        CoFunction(UBYTE *,int,int);
00824 extern int        TestName(UBYTE *);
00825 extern int        AddSet(UBYTE *,WORD);
00826 extern int        DoElements(UBYTE *,SETS,UBYTE *);
00827 extern int        DoTempSet(UBYTE *,UBYTE *);
00828 extern int        NameConflict(int,UBYTE *);
00829 extern int        OpenFile(char *);
00830 extern int        OpenAddFile(char *);
00831 extern int        ReOpenFile(char *);
00832 extern int        CreateFile(char *);
00833 extern int        CreateLogFile(char *);
00834 extern void      CloseFile(int);
00835 extern int        CopyFile(char *, char *);
00836 extern int        CreateHandle(void);
00837 extern LONG       ReadFile(int,UBYTE *,LONG);
00838 extern LONG       ReadPosFile(PHEAD FILEHANDLE *,UBYTE *,LONG,POSITION *);
00839 extern LONG       WriteFileToFile(int,UBYTE *,LONG);
00840 extern void      SeekFile(int,POSITION *,int);
00841 extern LONG       TellFile(int);
00842 extern void      FlushFile(int);
00843 extern int        GetPosFile(int,fpos_t *);
00844 extern int        SetPosFile(int,fpos_t *);
00845 extern void      SynchFile(int);
00846 extern void      TruncateFile(int);
00847 extern int        GetChannel(char *,int);
00848 extern int        GetAppendChannel(char *);
00849 extern int        CloseChannel(char *);
00850 extern void      inictable(void);
00851 extern KEYWORD    *findcommand(UBYTE *);
00852 extern int        inicbufs(void);
00853 extern void      StartFiles(void);
00854 extern UBYTE      *MakeDate(void);
00855 extern void      PreProcessor(void);
00856 extern void      *FromList(LIST *);
00857 extern void      *From0List(LIST *);
00858 extern void      *FromVarList(LIST *);
00859 extern int        DoubleList(void ***,int *,int,char *);
00860 extern int        DoubleLList(void ***,LONG *,int,char *);
00861 extern void      DoubleBuffer(void **,void **,int,char *);
00862 extern void      ExpandBuffer(void **,LONG *,int);
00863 extern LONG       iexp(LONG,int);
00864 extern int        IsLikeVector(WORD *);
00865 extern int        AreArgsEqual(WORD *,WORD *);
00866 extern int        CompareArgs(WORD *,WORD *);
00867 extern UBYTE      *SkipField(UBYTE *,int);
00868 extern int        StrCmp(UBYTE *,UBYTE *);
00869 extern int        StrICmp(UBYTE *,UBYTE *);
00870 extern int        StrHICmp(UBYTE *,UBYTE *);
00871 extern int        StrICont(UBYTE *,UBYTE *);
00872 extern int        CmpArray(WORD *,WORD *,WORD);
00873 extern int        ConWord(UBYTE *,UBYTE *);
00874 extern int        StrLen(UBYTE *);
00875 extern UBYTE      *GetPreVar(UBYTE *,int);
00876 extern void      ToGeneral(WORD *,WORD *,WORD);

```

```

00877 extern WORD    ToPolyFunGeneral (PHEAD WORD *);
00878 extern int      ToFast (WORD *,WORD *);
00879 extern SETUPPARAMETERS *GetSetupPar (UBYTE *);
00880 extern int      RecalcSetups (void);
00881 extern int      AllocSetups (void);
00882 extern SORTING *AllocSort (LONG, LONG, LONG, LONG, int, int, LONG, int);
00883 extern void     AllocSortFileName (SORTING *);
00884 extern UBYTE *LoadInputFile (UBYTE *,int);
00885 extern UBYTE    GetInput (void);
00886 extern void     ClearPushback (void);
00887 extern UBYTE    GetChar (int);
00888 extern void     CharOut (UBYTE);
00889 extern void     UnsetAllowDelay (void);
00890 extern void     PopPreVars (int);
00891 extern void     IniModule (int);
00892 extern void     IniSpecialModule (int);
00893 extern int      ModuleInstruction (int *,int *);
00894 extern int      PreProInstruction (void);
00895 extern int      LoadInstruction (int);
00896 extern int      LoadStatement (int);
00897 extern KEYWORD *FindKeyWord (UBYTE *,KEYWORD *,int);
00898 extern KEYWORD *FindInKeyWord (UBYTE *,KEYWORD *,int);
00899 extern int      DoDefine (UBYTE *);
00900 extern int      DoRedefine (UBYTE *);
00901 extern int      TheDefine (UBYTE *,int);
00902 extern int      TheUndefine (UBYTE *);
00903 extern int      ClearMacro (UBYTE *);
00904 extern int      DoUndefine (UBYTE *);
00905 extern int      DoInclude (UBYTE *);
00906 extern int      DoReverseInclude (UBYTE *);
00907 extern int      Include (UBYTE *,int);
00908 /*[14apr2004 mt]:*/
00909 extern int      DoExternal (UBYTE *);
00910 extern int      DoToExternal (UBYTE *);
00911 extern int      DoFromExternal (UBYTE *);
00912 extern int      DoPrompt (UBYTE *);
00913 extern int      DoSetExternal (UBYTE *);
00914 /*[10may2006 mt]:*/
00915 extern int      DoSetExternalAttr (UBYTE *);
00916 /*:[10may2006 mt]:*/
00917 extern int      DoRmExternal (UBYTE *);
00918 /*:[14apr2004 mt]:*/
00919 extern int      DoFactDollar (UBYTE *);
00920 extern WORD     GetDollarNumber (UBYTE **,DOLLARS);
00921 extern int      DoSetRandom (UBYTE *);
00922 extern int      DoOptimize (UBYTE *);
00923 extern int      DoClearOptimize (UBYTE *);
00924 extern int      DoSkipExtraSymbols (UBYTE *);
00925 extern int      DoTimeOutAfter (UBYTE *);
00926 extern int      DoMessage (UBYTE *);
00927 extern int      DoPreOut (UBYTE *);
00928 extern int      DoPreAppend (UBYTE *);
00929 extern int      DoPreCreate (UBYTE *);
00930 extern int      DoPreAssign (UBYTE *);
00931 extern int      DoPreBreak (UBYTE *);
00932 extern int      DoPreDefault (UBYTE *);
00933 extern int      DoPreSwitch (UBYTE *);
00934 extern int      DoPreEndSwitch (UBYTE *);
00935 extern int      DoPreCase (UBYTE *);
00936 extern int      DoPreShow (UBYTE *);
00937 extern int      DoPreExchange (UBYTE *);
00938 extern int      DoSystem (UBYTE *);
00939 extern int      DoPipe (UBYTE *);
00940 extern void     StartPrepro (void);
00941 extern int      DoIfdef (UBYTE *,int);
00942 extern int      DoIfydef (UBYTE *);
00943 extern int      DoIfndef (UBYTE *);
00944 extern int      DoElse (UBYTE *);
00945 extern int      DoElseif (UBYTE *);
00946 extern int      DoEndif (UBYTE *);
00947 extern int      DoTerminate (UBYTE *);
00948 extern int      DoIf (UBYTE *);
00949 extern int      DoCall (UBYTE *);
00950 extern int      DoDebug (UBYTE *);
00951 extern int      DoContinueDo (UBYTE *);
00952 extern int      DoDo (UBYTE *);
00953 extern int      DoBreakDo (UBYTE *);
00954 extern int      DoEnddo (UBYTE *);
00955 extern int      DoEndprocedure (UBYTE *);
00956 extern int      DoInside (UBYTE *);
00957 extern int      DoEndInside (UBYTE *);
00958 extern int      DoProcedure (UBYTE *);
00959 extern int      DoPrePrintTimes (UBYTE *);
00960 extern int      DoPreWrite (UBYTE *);
00961 extern int      DoPreClose (UBYTE *);
00962 extern int      DoPreRemove (UBYTE *);
00963 extern int      DoPreSortReallocate (UBYTE *);

```

```

00964 extern int      DoCommentChar (UBYTE *);
00965 extern int      DoPrCExtension (UBYTE *);
00966 extern int      DoPreReset (UBYTE *);
00967 extern void      WriteString (int, UBYTE *, int);
00968 extern void      WriteUnfinString (int, UBYTE *, int);
00969 extern UBYTE     *AddToString (UBYTE *, UBYTE *, int);
00970 extern UBYTE     *PreCalc (void);
00971 extern UBYTE     *PreEval (UBYTE *, LONG *);
00972 extern void      NumToStr (UBYTE *, LONG);
00973 extern int      PreCmp (int, int, UBYTE *, int, int, UBYTE *, int);
00974 extern int      PreEq (int, int, UBYTE *, int, int, UBYTE *, int);
00975 extern UBYTE     *pParseObject (UBYTE *, int *, LONG *);
00976 extern UBYTE     *PreIfEval (UBYTE *, int *);
00977 extern int      EvalPreIf (UBYTE *);
00978 extern int      PreLoad (PRELOAD *, UBYTE *, UBYTE *, int, char *);
00979 extern int      PreSkip (UBYTE *, UBYTE *, int);
00980 extern UBYTE     *EndOfToken (UBYTE *);
00981 extern void      SetSpecialMode (int, int);
00982 extern void      MakeGlobal (void);
00983 extern int      ExecModule (int);
00984 extern int      ExecStore (void);
00985 extern void      FullCleanup (void);
00986 extern int      DoExecStatement (void);
00987 extern int      DoPipeStatement (void);
00988 extern int      DoPolyfun (UBYTE *);
00989 extern int      DoPolyratfun (UBYTE *);
00990 extern int      CompileStatement (UBYTE *);
00991 extern UBYTE     *ToToken (UBYTE *);
00992 extern int      GetDollar (UBYTE *);
00993 extern int      MesWork (void);
00994 extern int      MesPrint (const char *, ...);
00995 extern int      MesCall (char *);
00996 extern UBYTE     *NumCopy (WORD, UBYTE *);
00997 extern char      *LongCopy (LONG, char *);
00998 extern char      *LongLongCopy (off_t *, char *);
00999 extern void      ReserveTempFiles (int);
01000 extern void      PrintTerm (WORD *, char *);
01001 extern void      PrintTermC (WORD *, char *);
01002 extern void      PrintSubTerm (WORD *, char *);
01003 extern void      PrintWords (WORD *, LONG);
01004 extern void      PrintSeq (WORD *, char *);
01005 extern int      ExpandTripleDots (int);
01006 extern LONG      ComPress (WORD **, LONG *);
01007 extern void      StageSort (FILEHANDLE *);
01008
01009 #define M_alloc(x)      malloc((size_t)(x))
01010
01011 extern void      M_free (void *, const char *);
01012 extern void      ClearWildcardNames (void);
01013 extern int      AddWildcardName (UBYTE *);
01014 extern int      GetWildcardName (UBYTE *);
01015 extern void      Globalize (int);
01016 extern void      ResetVariables (int);
01017 extern void      AddToPreTypes (int);
01018 extern void      MessPreNesting (int);
01019 extern LONG      GetStreamPosition (STREAM *);
01020 extern WORD      *DoubleCbuffer (int, WORD *, int);
01021 extern WORD      *AddLHS (int);
01022 extern WORD      *AddRHS (int, int);
01023 extern int      AddNtoL (int, WORD *);
01024 extern int      AddNtoC (int, int, WORD *, int);
01025 extern void      DoubleIfBuffers (void);
01026 extern STREAM    *CreateStream (UBYTE *);
01027
01028 extern int      setonoff (UBYTE *, int *, int, int);
01029 extern int      DoPrint (UBYTE *, int);
01030 extern int      SetExpr (UBYTE *, int, int);
01031 extern void      AddToCom (int, WORD *);
01032 extern int      Add2ComStrings (int, WORD *, UBYTE *, UBYTE *);
01033 extern int      DoSymmetrize (UBYTE *, int);
01034 extern int      DoArgument (UBYTE *, int);
01035 extern int      ArgFactorize (PHEAD WORD *, WORD *);
01036 extern WORD      *TakeArgContent (PHEAD WORD *, WORD *);
01037 extern WORD      *MakeInteger (PHEAD WORD *, WORD *, WORD *);
01038 extern WORD      *MakeMod (PHEAD WORD *, WORD *, WORD *);
01039 extern WORD      FindArg (PHEAD WORD *);
01040 extern WORD      InsertArg (PHEAD WORD *, WORD *, int);
01041 extern int      CleanupArgCache (PHEAD WORD);
01042 extern int      ArgSymbolMerge (WORD *, WORD *);
01043 extern int      ArgDotproductMerge (WORD *, WORD *);
01044 extern void      SortWeights (LONG *, LONG *, WORD);
01045 extern int      DoBrackets (UBYTE *, int);
01046 extern int      DoPutInside (UBYTE *, int);
01047 extern WORD      *CountComp (UBYTE *, WORD *);
01048 extern int      CoAntiBracket (UBYTE *);
01049 extern int      CoAntiSymmetrize (UBYTE *);
01050 extern int      DoArgPlode (UBYTE *, int);

```

```

01051 extern int      CoArgExplode(UBYTE *);
01052 extern int      CoArgImplode(UBYTE *);
01053 extern int      CoArgument(UBYTE *);
01054 extern int      CoInside(UBYTE *);
01055 extern int      ExecInside(UBYTE *);
01056 extern int      CoInExpression(UBYTE *);
01057 extern int      CoInParallel(UBYTE *);
01058 extern int      CoNotInParallel(UBYTE *);
01059 extern int      DoInParallel(UBYTE *,int);
01060 extern int      CoEndInExpression(UBYTE *);
01061 extern int      CoBracket(UBYTE *);
01062 extern int      CoPutInside(UBYTE *);
01063 extern int      CoAntiPutInside(UBYTE *);
01064 extern int      CoMultiBracket(UBYTE *);
01065 extern int      CoCFunction(UBYTE *);
01066 extern int      CoCTensor(UBYTE *);
01067 extern int      CoCollect(UBYTE *);
01068 extern int      CoCompress(UBYTE *);
01069 extern int      CoContract(UBYTE *);
01070 extern int      CoCycleSymmetrize(UBYTE *);
01071 extern int      CoDelete(UBYTE *);
01072 extern int      CoTableBase(UBYTE *);
01073 extern int      CoApply(UBYTE *);
01074 extern int      CoDenominators(UBYTE *);
01075 extern int      CoDimension(UBYTE *);
01076 extern int      CoDiscard(UBYTE *);
01077 extern int      CoDisorder(UBYTE *);
01078 extern int      CoDrop(UBYTE *);
01079 extern int      CoDropCoefficient(UBYTE *);
01080 extern int      CoDropSymbols(UBYTE *);
01081 extern int      CoElse(UBYTE *);
01082 extern int      CoElseIf(UBYTE *);
01083 extern int      CoEndArgument(UBYTE *);
01084 extern int      CoEndInside(UBYTE *);
01085 extern int      CoEndIf(UBYTE *);
01086 extern int      CoEndRepeat(UBYTE *);
01087 extern int      CoEndTerm(UBYTE *);
01088 extern int      CoEndWhile(UBYTE *);
01089 extern int      CoExit(UBYTE *);
01090 extern int      CoFactArg(UBYTE *);
01091 extern int      CoFactDollar(UBYTE *);
01092 extern int      CoFactorize(UBYTE *);
01093 extern int      CoNFactorize(UBYTE *);
01094 extern int      CoUnFactorize(UBYTE *);
01095 extern int      CoNUnFactorize(UBYTE *);
01096 extern int      DoFactorize(UBYTE *,int);
01097 extern int      CoFill(UBYTE *);
01098 extern int      CoFillExpression(UBYTE *);
01099 extern int      CoFixIndex(UBYTE *);
01100 extern int      CoFormat(UBYTE *);
01101 extern int      CoGlobal(UBYTE *);
01102 extern int      CoGlobalFactorized(UBYTE *);
01103 extern int      CoGoTo(UBYTE *);
01104 extern int      CoId(UBYTE *);
01105 extern int      CoIdNew(UBYTE *);
01106 extern int      CoIdOld(UBYTE *);
01107 extern int      CoIf(UBYTE *);
01108 extern int      CoIfMatch(UBYTE *);
01109 extern int      CoIfNoMatch(UBYTE *);
01110 extern int      CoIndex(UBYTE *);
01111 extern int      CoInsideFirst(UBYTE *);
01112 extern int      CoKeep(UBYTE *);
01113 extern int      CoLabel(UBYTE *);
01114 extern int      CoLoad(UBYTE *);
01115 extern int      CoLocal(UBYTE *);
01116 extern int      CoLocalFactorized(UBYTE *);
01117 extern int      CoMany(UBYTE *);
01118 extern int      CoMerge(UBYTE *);
01119 extern int      CoShuffle(UBYTE *);
01120 extern int      CoMetric(UBYTE *);
01121 extern int      CoModOption(UBYTE *);
01122 extern int      CoModuleOption(UBYTE *);
01123 extern int      CoModulus(UBYTE *);
01124 extern int      CoMulti(UBYTE *);
01125 extern int      CoMultiply(UBYTE *);
01126 extern int      CoNFunction(UBYTE *);
01127 extern int      CoNPrint(UBYTE *);
01128 extern int      CoNTensor(UBYTE *);
01129 extern int      CoNWrite(UBYTE *);
01130 extern int      CoNoDrop(UBYTE *);
01131 extern int      CoNoSkip(UBYTE *);
01132 extern int      CoNormalize(UBYTE *);
01133 extern int      CoMakeInteger(UBYTE *);
01134 extern int      CoFlags(UBYTE *,int);
01135 extern int      CoOff(UBYTE *);
01136 extern int      CoOn(UBYTE *);
01137 extern int      CoOnce(UBYTE *);

```

```

01138 extern int      CoOnly(UBYTE *);
01139 extern int      CoOptimizeOption(UBYTE *);
01140 extern int      CoOptimize(UBYTE *);
01141 extern int      CoPolyFun(UBYTE *);
01142 extern int      CoPolyRatFun(UBYTE *);
01143 extern int      CoPrint(UBYTE *);
01144 extern int      CoPrintB(UBYTE *);
01145 extern int      CoProperCount(UBYTE *);
01146 extern int      CoUnitTrace(UBYTE *);
01147 extern int      CoRCycleSymmetrize(UBYTE *);
01148 extern int      CoRatio(UBYTE *);
01149 extern int      CoRedefine(UBYTE *);
01150 extern int      CoRenumber(UBYTE *);
01151 extern int      CoRepeat(UBYTE *);
01152 extern int      CoSave(UBYTE *);
01153 extern int      CoSelect(UBYTE *);
01154 extern int      CoSet(UBYTE *);
01155 extern int      CoSetExitFlag(UBYTE *);
01156 extern int      CoSkip(UBYTE *);
01157 extern int      CoProcessBucket(UBYTE *);
01158 extern int      CoPushHide(UBYTE *);
01159 extern int      CoPopHide(UBYTE *);
01160 extern int      CoHide(UBYTE *);
01161 extern int      CoIntoHide(UBYTE *);
01162 extern int      CoNoIntoHide(UBYTE *);
01163 extern int      CoNoHide(UBYTE *);
01164 extern int      CoUnHide(UBYTE *);
01165 extern int      CoNoUnHide(UBYTE *);
01166 extern int      CoSort(UBYTE *);
01167 extern int      CoSplitArg(UBYTE *);
01168 extern int      CoSplitFirstArg(UBYTE *);
01169 extern int      CoSplitLastArg(UBYTE *);
01170 extern int      CoSum(UBYTE *);
01171 extern int      CoSymbol(UBYTE *);
01172 extern int      CoSymmetrize(UBYTE *);
01173 extern int      DoTable(UBYTE *,int);
01174 extern int      CoTable(UBYTE *);
01175 extern int      CoTerm(UBYTE *);
01176 extern int      CoNTable(UBYTE *);
01177 extern int      CoCTable(UBYTE *);
01178 extern void      EmptyTable(TABLES);
01179 extern int      CoToTensor(UBYTE *);
01180 extern int      CoToVector(UBYTE *);
01181 extern int      CoTrace4(UBYTE *);
01182 extern int      CoTraceN(UBYTE *);
01183 extern int      CoChisholm(UBYTE *);
01184 extern int      CoTransform(UBYTE *);
01185 extern int      CoClearTable(UBYTE *);
01186 extern int      DoChain(UBYTE *,int);
01187 extern int      CoChainin(UBYTE *);
01188 extern int      CoChainout(UBYTE *);
01189 extern int      CoTryReplace(UBYTE *);
01190 extern int      CoVector(UBYTE *);
01191 extern int      CoWhile(UBYTE *);
01192 extern int      CoWrite(UBYTE *);
01193 extern int      CoAuto(UBYTE *);
01194 extern int      CoSwitch(UBYTE *);
01195 extern int      CoCase(UBYTE *);
01196 extern int      CoBreak(UBYTE *);
01197 extern int      CoDefault(UBYTE *);
01198 extern int      CoEndSwitch(UBYTE *);
01199 extern int      CoTBaddto(UBYTE *);
01200 extern int      CoTBaudit(UBYTE *);
01201 extern int      CoTbcleanup(UBYTE *);
01202 extern int      CoTbcreate(UBYTE *);
01203 extern int      CoTBenter(UBYTE *);
01204 extern int      CoTBhelp(UBYTE *);
01205 extern int      CoTBload(UBYTE *);
01206 extern int      CoTBoff(UBYTE *);
01207 extern int      CoTBon(UBYTE *);
01208 extern int      CoTBopen(UBYTE *);
01209 extern int      CoTBreplace(UBYTE *);
01210 extern int      CoTBuse(UBYTE *);
01211 extern int      CoTestUse(UBYTE *);
01212 extern int      CoThreadBucket(UBYTE *);
01213 extern int      AddComString(int,WORD *,UBYTE *,int);
01214 extern int      CompileAlgebra(UBYTE *,int,WORD *);
01215 extern int      IsIdStatement(UBYTE *);
01216 extern UBYTE *  *IsRHS(UBYTE *,UBYTE *);
01217 extern int      ParenthesesTest(UBYTE *);
01218 extern int      tokenize(UBYTE *,WORD);
01219 extern void      WriteTokens(SBYTE *);
01220 extern int      simpltoken(SBYTE *);
01221 extern int      simpwtoken(SBYTE *);
01222 extern int      simp2token(SBYTE *);
01223 extern int      simp3atoken(SBYTE *,int);
01224 extern int      simp3btoken(SBYTE *,int);

```



```

01225 extern int      simp4token(SBYTE *);
01226 extern int      simp5token(SBYTE *,int);
01227 extern int      simp6token(SBYTE *,int);
01228 extern UBYTE *SkipAName(UBYTE *);
01229 extern int      TestTables(void);
01230 extern int      GetLabel(UBYTE *);
01231 extern int      CoIdExpression(UBYTE *,int);
01232 extern int      CoAssign(UBYTE *);
01233 extern int      DoExpr(UBYTE *,int,int);
01234 extern int      CompileSubExpressions(SBYTE *);
01235 extern int      CodeGenerator(SBYTE *);
01236 extern int      CompleteTerm(WORD *,UWORD *,WORD,WORD,int);
01237 extern int      CodeFactors(SBYTE *s);
01238 extern WORD     GenerateFactors(WORD,WORD);
01239 extern int      InsTree(int,int);
01240 extern int      FindTree(int,WORD *);
01241 extern void     RedoTree(CBUF *,int);
01242 extern void     ClearTree(int);
01243 extern int      CatchDollar(int);
01244 extern int      AssignDollar(PHEAD WORD *,WORD);
01245 extern UBYTE *WriteDollarToBuffer(WORD,WORD);
01246 extern UBYTE *WriteDollarFactorToBuffer(WORD,WORD,WORD);
01247 extern void     AddToDollarBuffer(UBYTE *);
01248 extern int      PutTermInDollar(WORD *,WORD);
01249 extern void     TermAssign(WORD *);
01250 extern void     WildDollars(PHEAD WORD *);
01251 extern LONG     numcommute(WORD *,LONG *);
01252 extern int      FullRenummer(PHEAD WORD *,WORD);
01253 extern int      Lus(WORD *,WORD,WORD,WORD,WORD,WORD);
01254 extern int      FindLus(int,int,int);
01255 extern int      CoReplaceLoop(UBYTE *);
01256 extern int      CoFindLoop(UBYTE *);
01257 extern int      DoFindLoop(UBYTE *,int);
01258 extern int      CoFunPowers(UBYTE *);
01259 extern int      SortTheList(int *,int);
01260 extern int      MatchIsPossible(WORD *,WORD *);
01261 extern int      StudyPattern(WORD *);
01262 extern WORD     DolToTensor(PHEAD WORD);
01263 extern WORD     DolToFunction(PHEAD WORD);
01264 extern WORD     DolToVector(PHEAD WORD);
01265 extern WORD     DolToNumber(PHEAD WORD);
01266 extern WORD     DolToSymbol(PHEAD WORD);
01267 extern WORD     DolToIndex(PHEAD WORD);
01268 extern LONG     DolToLong(PHEAD WORD);
01269 extern int      DollarFactorize(PHEAD WORD);
01270 extern int      CoPrintTable(UBYTE *);
01271 extern int      CoDeallocateTable(UBYTE *);
01272 extern void     CleanDollarFactors(DOLLARS);
01273 extern WORD     *TakeDollarContent(PHEAD WORD *,WORD **);
01274 extern WORD     *MakeDollarInteger(PHEAD WORD *,WORD **);
01275 extern WORD     *MakeDollarMod(PHEAD WORD *,WORD **);
01276 extern int      GetDolNum(PHEAD WORD *,WORD *);
01277 extern void     AddPotModdollar(WORD);
01278
01279 extern int      Optimize(WORD, int);
01280 extern int      ClearOptimize(void);
01281 extern int      LoadOpti(WORD);
01282 extern int      PutObject(WORD *,int);
01283 extern void     CleanOptiBuffer(void);
01284 extern int      PrintOptima(WORD);
01285 extern int      FindScratchName(void);
01286 extern WORD     MaxPowerOpti(LONG);
01287 extern WORD     HuntNumFactor(LONG,WORD *,int);
01288 extern WORD     HuntFactor(LONG,WORD *,int);
01289 extern void     HuntPairs(LONG,WORD);
01290 extern void     HuntBrackets(LONG);
01291 extern int      AddToOpti(WORD *,int);
01292 extern LONG     TestNewSca(LONG,WORD *,WORD *);
01293 extern void     NormOpti(WORD *);
01294 extern void     SortOpti(LONG);
01295 extern void     SplitOpti(WORD **,LONG);
01296 extern void     CombiOpti(void);
01297 extern int      TakeLongRoot(UWORD *,WORD *,WORD);
01298 extern int      TakeRatRoot(UWORD *,WORD *,WORD);
01299 extern int      MakeRational(WORD ,WORD ,WORD *,WORD *);
01300 extern int      MakeLongRational(PHEAD UWORD *,WORD ,UWORD *,WORD ,UWORD *,WORD *);
01301 extern void     HuntPowers(LONG,WORD);
01302 extern void     HuntNumBrackets(LONG);
01303 extern void     ClearTableTree(TABLES);
01304 extern int      InsTableTree(TABLES,WORD *);
01305 extern void     RedoTableTree(TABLES,int);
01306 extern int      FindTableTree(TABLES,WORD *,int);
01307 extern void     finishcbuf(WORD);
01308 extern void     clearcbuf(WORD);
01309 extern void     CleanUpSort(int);
01310 extern FILEHANDLE *AllocFileHandle(WORD,char *);
01311 extern void     DeAllocFileHandle(FILEHANDLE *);

```

```

01312 extern void      LowerSortLevel(void);
01313 extern WORD      *PolyRatFunSpecial(PHEAD WORD *, WORD *);
01314 extern void      SimpleSplitMergeRec(WORD *,WORD,WORD *);
01315 extern void      SimpleSplitMerge(WORD *,WORD);
01316 extern WORD      BinarySearch(WORD *,WORD,WORD);
01317 extern int       InsideDollar(PHEAD WORD *,WORD);
01318 extern DOLLARS   DolToTerms(PHEAD WORD);
01319 extern WORD      EvalDoLoopArg(PHEAD WORD *,WORD);
01320 extern int       SetExprCases(int,int,int);
01321 extern int       TestSelect(WORD *,WORD *);
01322 extern void      SubsInAll(PHEAD0);
01323 extern void      TransferBuffer(int,int,int);
01324 extern int       TakeIDfunction(PHEAD WORD *);
01325 extern int       MakeSetupAllocs(void);
01326 extern int       TryFileSetups(void);
01327 extern void      ExchangeExpressions(int,int);
01328 extern void      ExchangeDollars(int,int);
01329 extern int       GetFirstBracket(WORD *,int);
01330 extern int       GetFirstTerm(WORD *,int,int);
01331 extern int       GetContent(WORD *,int);
01332 extern int       CleanupTerm(WORD *);
01333 extern WORD      ContentMerge(PHEAD WORD *,WORD *);
01334 extern UBYTE     *PreIfDollarEval(UBYTE *,int *);
01335 extern LONG      TermsInDollar(WORD);
01336 extern LONG      SizeOfDollar(WORD);
01337 extern LONG      TermsInExpression(WORD);
01338 extern LONG      SizeOfExpression(WORD);
01339 extern WORD      *TranslateExpression(UBYTE *);
01340 extern int       IsSetMember(WORD *,WORD);
01341 extern int       IsMultipleOf(WORD *,WORD *);
01342 extern int       TwoExprCompare(WORD *,WORD *,int);
01343 extern void      UpdatePositions(void);
01344 extern void      M_check(void);
01345 extern void      M_print(void);
01346 extern void      M_check1(void);
01347 extern void      PrintTime(UBYTE *);
01348
01349 extern POSITION   *FindBracket(WORD,WORD *);
01350 extern void      PutBracketInIndex(PHEAD WORD *,POSITION *);
01351 extern void      ClearBracketIndex(WORD);
01352 extern void      OpenBracketIndex(WORD);
01353 extern int       DoNoParallel(UBYTE *);
01354 extern int       DoParallel(UBYTE *);
01355 extern int       DoModSum(UBYTE *);
01356 extern int       DoModMax(UBYTE *);
01357 extern int       DoModMin(UBYTE *);
01358 extern int       DoModLocal(UBYTE *);
01359 extern UBYTE     *DoModDollar(UBYTE *,int);
01360 extern int       DoProcessBucket(UBYTE *);
01361 extern int       DoinParallel(UBYTE *);
01362 extern int       DonotinParallel(UBYTE *);
01363
01364 extern int       FlipTable(FUNCTIONS,int);
01365 extern int       ChainIn(PHEAD WORD *,WORD);
01366 extern int       ChainOut(PHEAD WORD *,WORD);
01367 extern int       ArgumentImplode(PHEAD WORD *,WORD *);
01368 extern int       ArgumentExplode(PHEAD WORD *,WORD *);
01369 extern int       DenToFunction(WORD *,WORD);
01370
01371 extern WORD      HowMany(PHEAD WORD *,WORD *);
01372 extern void      RemoveDollars(void);
01373 extern LONG      CountTerms1(PHEAD0);
01374 extern LONG      TermsInBracket(PHEAD WORD *,WORD);
01375 extern int       Crash(void);
01376
01377 extern char      *str_dup(char *);
01378 extern void      convertblock(INDEXBLOCK *,INDEXBLOCK *,int);
01379 extern void      convertnamesblock(NAMESBLOCK *,NAMESBLOCK *,int);
01380 extern void      convertiniinfo(INIINFO *,INIINFO *,int);
01381 extern int       ReadIndex(DBASE *);
01382 extern int       WriteIndexBlock(DBASE *,MLONG);
01383 extern int       WriteNamesBlock(DBASE *,MLONG);
01384 extern int       WriteIndex(DBASE *);
01385 extern int       WriteIniInfo(DBASE *);
01386 extern int       ReadIniInfo(DBASE *);
01387 extern int       AddToIndex(DBASE *,MLONG);
01388 extern DBASE     *GetDbase(char *,MLONG);
01389 extern DBASE     *OpenDbase(char *);
01390 extern char      *ReadObject(DBASE *,MLONG,char *);
01391 extern char      *ReadiObject(DBASE *,MLONG,MLONG,char *);
01392 extern int       ExistsObject(DBASE *,MLONG,char *);
01393 extern int       DeleteObject(DBASE *,MLONG,char *);
01394 extern int       WriteObject(DBASE *,MLONG,char *,char *,MLONG);
01395 extern MLONG     AddObject(DBASE *,MLONG,char *,char *);
01396 extern int       Cleanup(DBASE *);
01397 extern DBASE     *NewDbase(char *,MLONG);
01398 extern void      FreeTableBase(DBASE *);

```



```

01399 extern int      ComposeTableNames (DBASE *);
01400 extern int      PutTableNames (DBASE *);
01401 extern MLONG    AddTableName (DBASE *, char *, TABLES);
01402 extern MLONG    GetTableName (DBASE *, char *);
01403 extern MLONG    FindTableNumber (DBASE *, char *);
01404 extern int      TryEnvironment (void);
01405
01406 #ifdef WITHZLIB
01407 extern int      SetupOutputGZIP (FILEHANDLE *);
01408 extern int      PutOutputGZIP (FILEHANDLE *);
01409 extern int      FlushOutputGZIP (FILEHANDLE *);
01410 extern int      SetupAllInputGZIP (SORTING *);
01411 extern LONG     FillInputGZIP (FILEHANDLE *, POSITION *, UBYTE *, LONG, int);
01412 extern void     ClearSortGZIP (FILEHANDLE *f);
01413 #endif
01414
01415 #ifdef WITHPTHREADS
01416 extern void     BeginIdentities (void);
01417 extern int      WhoAmI (void);
01418 extern int      StartAllThreads (int);
01419 extern void     StartHandleLock (void);
01420 extern void     TerminateAllThreads (void);
01421 extern int      GetAvailableThread (void);
01422 extern int      ConditionalGetAvailableThread (void);
01423 extern int      BalanceRunThread (PHEAD int, WORD *, WORD);
01424 extern void     WakeupThread (int, int);
01425 extern int      MasterWait (void);
01426 extern int      InParallelProcessor (void);
01427 extern int      ThreadsProcessor (EXPRESSIONS, WORD, WORD);
01428 extern int      MasterMerge (void);
01429 extern int      PutToMaster (PHEAD WORD *);
01430 extern void     SetWorkerFiles (void);
01431 extern int      MakeThreadBuckets (int, int);
01432 extern int      SendOneBucket (int);
01433 extern int      LoadOneThread (int, int, THREADBUCKET *, int);
01434 extern void     *RunSortBot (void *);
01435 extern void     MasterWaitAllSortBots (void);
01436 extern int      SortBotMerge (PHEAD0);
01437 extern int      SortBotOut (PHEAD WORD *);
01438 extern void     DefineSortBotTree (void);
01439 extern int      SortBotMasterMerge (void);
01440 extern int      SortBotWait (int);
01441 extern void     StartIdentity (void);
01442 extern void     FinishIdentity (void *);
01443 extern int      SetIdentity (int *);
01444 extern ALLPRIVATES *InitializeOneThread (int);
01445 extern void     FinalizeOneThread (int);
01446 extern void     ClearAllThreads (void);
01447 extern void     *RunThread (void *);
01448 extern void     IAmAvailable (int);
01449 extern int      ThreadWait (int);
01450 extern int      ThreadClaimedBlock (int);
01451 extern int      GetThread (int);
01452 extern int      UpdateOneThread (int);
01453 extern void     MasterWaitAll (void);
01454 extern void     MasterWaitAllBlocks (void);
01455 extern int      MasterWaitThread (int);
01456 extern void     WakeupMasterFromThread (int, int);
01457 extern int      LoadReadjusted (void);
01458 extern int      IniSortBlocks (int);
01459 extern int      UpdateSortBlocks (int);
01460 extern int      TreatIndexEntry (PHEAD LONG);
01461 extern WORD     GetTerm2 (PHEAD WORD *);
01462 extern void     SetHideFiles (void);
01463
01464 #endif
01465
01466 extern int      CopyExpression (FILEHANDLE *, FILEHANDLE *);
01467
01468 extern int      set_in (UBYTE, set_of_char);
01469 extern one_byte set_set (UBYTE, set_of_char);
01470 extern one_byte set_del (UBYTE, set_of_char);
01471 extern one_byte set_sub (set_of_char, set_of_char, set_of_char);
01472 extern int      DoPreAddSeparator (UBYTE *);
01473 extern int      DoPreRmSeparator (UBYTE *);
01474
01475 /*See the file extcmd.c*/
01476 extern int      openExternalChannel (UBYTE *, int, UBYTE *, UBYTE *);
01477 extern int      initPresetExternalChannels (UBYTE *, int);
01478 extern int      closeExternalChannel (int);
01479 extern int      selectExternalChannel (int);
01480 extern int      getCurrentExternalChannel (void);
01481 extern void     closeAllExternalChannels (void);
01482
01483 typedef int      (*WRITEBUFTOEXTCHANNEL) (char *, size_t);
01484 typedef int      (*GETCFROMEXTCHANNEL) (void);
01485 typedef int      (*SETTERMINATORFOREXTCHANNEL) (char *);

```

```

01486 typedef int (*SETKILLMODEFOREXTERNALCHANNEL)(int,int);
01487 typedef LONG (*WRITEFILE)(int,UBYTE *,LONG);
01488 typedef WORD (*GETTERM)(PHEAD WORD *);
01489
01490 #define CompareTerms ((COMPARE)AR.CompareRoutine)
01491 #define FiniShuffle AN.SHvar.finishuf
01492 #define DoShuffle ((DO_UFFLE)AN.SHvar.do_uffle)
01493
01494 extern UBYTE *defineChannel(UBYTE*, HANDLERS*);
01495 extern int writeToChannel(int,UBYTE *,HANDLERS*);
01496 #ifdef WITHEXTERNALCHANNEL
01497 extern LONG WriteToExternalChannel(int,UBYTE *,LONG);
01498 #endif
01499 extern int writeBufToExtChannelOk(char *,size_t);
01500 extern int getCFromExtChannelOk(void);
01501 extern int setKillModeForExternalChannelOk(int,int);
01502 extern int setTerminatorForExternalChannelOk(char *);
01503 extern int getCFromExtChannelFailure(void);
01504 extern int setKillModeForExternalChannelFailure(int,int);
01505 extern int setTerminatorForExternalChannelFailure(char *);
01506 extern int writeBufToExtChannelFailure(char *,size_t);
01507
01508 extern int ReleaseTB(void);
01509
01510 extern int SymbolNormalize(WORD *);
01511 extern int TestFunFlag(PHEAD WORD *);
01512 extern WORD CompareSymbols(PHEAD WORD *,WORD *,WORD);
01513 extern WORD CompareHSymbols(PHEAD WORD *,WORD *,WORD);
01514 extern WORD NextPrime(PHEAD WORD);
01515 extern WORD Moebius(PHEAD WORD);
01516 extern UWORD wranf(PHEAD0);
01517 extern UWORD iranf(PHEAD UWORD);
01518 extern void iniwranf(PHEAD0);
01519 extern UBYTE *PreRandom(UBYTE *);
01520
01521 extern WORD *PolyNormPoly (PHEAD WORD);
01522 extern WORD *EvaluateGcd(PHEAD WORD *);
01523 extern int TreatPolyRatFun(PHEAD WORD *);
01524
01525 extern WORD ReadSaveHeader(void);
01526 extern WORD ReadSaveIndex(FILEINDEX *);
01527 extern WORD ReadSaveExpression(UBYTE *,UBYTE *,LONG *,LONG *);
01528 extern UBYTE *ReadSaveTerm32(UBYTE *,UBYTE *,UBYTE **,UBYTE *,UBYTE *,int);
01529 extern WORD ReadSaveVariables(UBYTE *,UBYTE *,LONG *,LONG *,INDEXENTRY *,LONG *);
01530 extern WORD WriteStoreHeader(WORD);
01531
01532 extern void InitRecovery(void);
01533 extern int CheckRecoveryFile(void);
01534 extern void DeleteRecoveryFile(void);
01535 extern char *RecoveryFilename(void);
01536 extern int DoRecovery(int *);
01537 extern void DoCheckpoint(int);
01538
01539 extern void NumberMallocAddMemory(PHEAD0);
01540 extern void CacheNumberMallocAddMemory(PHEAD0);
01541 extern void TermMallocAddMemory(PHEAD0);
01542 #ifndef MEMORYMACROS
01543 extern WORD *TermMalloc2(PHEAD char *text);
01544 extern void TermFree2(PHEAD WORD *term,char *text);
01545 extern UWORD *NumberMalloc2(PHEAD char *text);
01546 extern UWORD *CacheNumberMalloc2(PHEAD char *text);
01547 extern void NumberFree2(PHEAD UWORD *NumberMem,char *text);
01548 extern void CacheNumberFree2(PHEAD UWORD *NumberMem,char *text);
01549 #endif
01550
01551 extern void ExprStatus(EXPRESSIONS);
01552 extern void iniTools(void);
01553 extern int TestTerm(WORD *);
01554
01555 extern WORD RunTransform(PHEAD WORD *term, WORD *params);
01556 extern WORD RunEncode(PHEAD WORD *fun, WORD *args, WORD *info);
01557 extern WORD RunDecode(PHEAD WORD *fun, WORD *args, WORD *info);
01558 extern WORD RunReplace(PHEAD WORD *fun, WORD *args, WORD *info);
01559 extern WORD RunImplode(WORD *fun, WORD *args);
01560 extern WORD RunExplode(PHEAD WORD *fun, WORD *args);
01561 extern int TestArgNum(int n, int totarg, WORD *args);
01562 extern WORD PutArgInScratch(WORD *arg,UWORD *scrat);
01563 extern UBYTE *ReadRange(UBYTE *s, WORD *out, int par);
01564 extern int FindRange(PHEAD WORD *,WORD *,WORD *,WORD);
01565 extern WORD RunPermute(PHEAD WORD *fun, WORD *args, WORD *info);
01566 extern WORD RunReverse(PHEAD WORD *fun, WORD *args);
01567 extern WORD RunCycle(PHEAD WORD *fun, WORD *args, WORD *info);
01568 extern WORD RunAddArg(PHEAD WORD *fun, WORD *args);
01569 extern WORD RunMulArg(PHEAD WORD *fun, WORD *args);
01570 extern WORD RunIsLyndon(PHEAD WORD *fun, WORD *args, int par);
01571 extern WORD RunToLyndon(PHEAD WORD *fun, WORD *args, int par);
01572 extern WORD RunDropArg(PHEAD WORD *fun, WORD *args);

```

```

01573 extern WORD RunSelectArg(PHEAD WORD *fun, WORD *args);
01574 extern WORD RunDedup(PHEAD WORD *fun, WORD *args);
01575 extern WORD RunZtoHArg(PHEAD WORD *fun, WORD *args);
01576 extern WORD RunHtoZArg(PHEAD WORD *fun, WORD *args);
01577
01578 extern int NormPolyTerm(PHEAD WORD *);
01579 extern WORD ComparePoly(WORD *, WORD *, WORD);
01580 extern int ConvertToPoly(PHEAD WORD *, WORD *, WORD *, WORD);
01581 extern int LocalConvertToPoly(PHEAD WORD *, WORD *, WORD, WORD);
01582 extern int ConvertFromPoly(PHEAD WORD *, WORD *, WORD, WORD, WORD, WORD);
01583 extern WORD FindSubterm(WORD *);
01584 extern WORD FindLocalSubterm(PHEAD WORD *, WORD);
01585 extern void PrintSubtermList(int, int);
01586 extern void PrintExtraSymbol(int, WORD *, int);
01587 extern WORD FindSubexpression(WORD *);
01588
01589 extern void UpdateMaxSize(void);
01590
01591 extern int CoToPolynomial(UBYTE *);
01592 extern int CoFromPolynomial(UBYTE *);
01593 extern int CoArgToExtraSymbol(UBYTE *);
01594 extern int CoExtraSymbols(UBYTE *);
01595 extern UBYTE *GetDoParam(UBYTE *, WORD **, int);
01596 extern WORD *GetIfDollarFactor(UBYTE **, WORD *);
01597 extern int CoDo(UBYTE *);
01598 extern int CoEndDo(UBYTE *);
01599 extern int ExtraSymFun(PHEAD WORD *, WORD);
01600 extern int PruneExtraSymbols(WORD);
01601 extern int IniFbuffer(WORD);
01602 extern void IniFbufs(void);
01603 extern int GCDfunction(PHEAD WORD *, WORD);
01604 extern WORD *GCDfunction3(PHEAD WORD *, WORD *);
01605 extern WORD *GCDfunction4(PHEAD WORD *, WORD *);
01606 extern int ReadPolyRatFun(PHEAD WORD *);
01607 extern int FromPolyRatFun(PHEAD WORD *, WORD **, WORD **);
01608 extern void PRFnormalize(PHEAD WORD *);
01609 extern WORD *PRFadd(PHEAD WORD *, WORD *);
01610 extern WORD *PolyDiv(PHEAD WORD *, WORD *, char *);
01611 extern WORD *PolyGCD(PHEAD WORD *, WORD *);
01612 extern WORD *PolyAdd(PHEAD WORD *, WORD *);
01613 extern void GCDclean(PHEAD WORD *, WORD *);
01614 extern int RatFunNormalize(PHEAD WORD *);
01615 extern WORD *TakeSymbolContent(PHEAD WORD *, WORD *);
01616 extern int GCDterms(PHEAD WORD *, WORD *, WORD *);
01617 extern WORD *PutExtraSymbols(PHEAD WORD *, WORD, int *);
01618 extern WORD *TakeExtraSymbols(PHEAD WORD *, WORD);
01619 extern WORD *MultiplyWithTerm(PHEAD WORD *, WORD *, WORD);
01620 extern WORD *TakeContent(PHEAD WORD *, WORD *);
01621 extern int MergeSymbolLists(PHEAD WORD *, WORD *, int);
01622 extern int MergeDotproductLists(PHEAD WORD *, WORD *, int);
01623 extern WORD *CreateExpression(PHEAD WORD);
01624 extern int DIVfunction(PHEAD WORD *, WORD, int);
01625 extern WORD *MULfunc(PHEAD WORD *, WORD *);
01626 extern WORD *ConvertArgument(PHEAD WORD *, int *);
01627 extern int ExpandRat(PHEAD WORD *);
01628 extern int InvPoly(PHEAD WORD *, WORD, WORD);
01629 extern WORD TestDoLoop(PHEAD WORD *, WORD);
01630 extern WORD TestEndDoLoop(PHEAD WORD *, WORD);
01631
01632 extern WORD *poly_gcd(PHEAD WORD *, WORD *, WORD);
01633 extern WORD *poly_div(PHEAD WORD *, WORD *, WORD);
01634 extern WORD *poly_rem(PHEAD WORD *, WORD *, WORD);
01635 extern WORD *poly_inverse(PHEAD WORD *, WORD *);
01636 extern WORD *poly_mul(PHEAD WORD *, WORD *);
01637 extern WORD *poly_ratfun_add(PHEAD WORD *, WORD *);
01638 extern int poly_ratfun_normalize(PHEAD WORD *);
01639 extern int poly_factorize_argument(PHEAD WORD *, WORD *);
01640 extern WORD *poly_factorize_dollar(PHEAD WORD *);
01641 extern int poly_factorize_expression(EXPRESSIONS);
01642 extern int poly_unfactorize_expression(EXPRESSIONS);
01643 extern void poly_free_poly_vars(PHEAD const char *);
01644
01645 #ifdef WITHFLINT
01646 extern void flint_final_cleanup_thread(void);
01647 extern void flint_final_cleanup_master(void);
01648 extern WORD* flint_div(PHEAD WORD *, WORD *, const WORD);
01649 extern int flint_factorize_argument(PHEAD WORD *, WORD *);
01650 extern WORD* flint_factorize_dollar(PHEAD WORD *);
01651 extern WORD* flint_gcd(PHEAD WORD *, WORD *, const WORD);
01652 extern WORD* flint_inverse(PHEAD WORD *, WORD *);
01653 extern WORD* flint_mul(PHEAD WORD *, WORD *);
01654 extern WORD* flint_ratfun_add(PHEAD WORD *, WORD *);
01655 extern int flint_ratfun_normalize(PHEAD WORD *);
01656 extern WORD* flint_rem(PHEAD WORD *, WORD *, const WORD);
01657 extern void flint_startup_init(void);
01658 #endif
01659

```

```

01660 extern void optimize_print_code (int);
01661
01662 #ifdef WITHPTHREADS
01663 extern void find_Horner_MCTS_expand_tree(void);
01664 extern void find_Horner_MCTS_expand_tree_threaded(void);
01665 extern void optimize_expression_given_Horner(void);
01666 extern void optimize_expression_given_Horner_threaded(void);
01667 #endif
01668
01669 extern int DoPreAdd(UBYTE *s);
01670 extern int DoPreUseDictionary(UBYTE *s);
01671 extern int DoPreCloseDictionary(UBYTE *s);
01672 extern int DoPreOpenDictionary(UBYTE *s);
01673 extern void RemoveDictionary(DICTIONARY *dict);
01674 extern void UnSetDictionary(void);
01675 extern int SetDictionaryOptions(UBYTE *options);
01676 extern int SelectDictionary(UBYTE *name,UBYTE *options);
01677 extern int AddToDictionary(DICTIONARY *dict,UBYTE *left,UBYTE *right);
01678 extern int AddDictionary(UBYTE *name);
01679 extern int FindDictionary(UBYTE *name);
01680 extern UBYTE *IsExponentSign(void);
01681 extern UBYTE *IsMultiplySign(void);
01682 extern void TransformRational(WORD *a, WORD na);
01683 extern void WriteDictionary(DICTIONARY *);
01684 extern void ShrinkDictionary(DICTIONARY *);
01685 extern void MultiplyToLine(void);
01686 extern UBYTE *FindSymbol(WORD num);
01687 extern UBYTE *FindVector(WORD num);
01688 extern UBYTE *FindIndex(WORD num);
01689 extern UBYTE *FindFunction(WORD num);
01690 extern UBYTE *FindFunWithArgs(WORD *t);
01691 extern UBYTE *FindExtraSymbol(WORD num);
01692 extern LONG DictToBytes(DICTIONARY *dict,UBYTE *buf);
01693 extern DICTIONARY *DictFromBytes(UBYTE *buf);
01694 extern int CoCreateSpectator(UBYTE *inp);
01695 extern int CoToSpectator(UBYTE *inp);
01696 extern int CoRemoveSpectator(UBYTE *inp);
01697 extern int CoEmptySpectator(UBYTE *inp);
01698 extern int CoCopySpectator(UBYTE *inp);
01699 extern int PutInSpectator(WORD *,WORD);
01700 extern void ClearSpectators(WORD);
01701 extern WORD GetFromSpectator(WORD *,WORD);
01702 extern void FlushSpectators(void);
01703
01704 extern WORD *PreGCD(PHEAD WORD *, WORD *,int);
01705 extern WORD *FindCommonVariables(PHEAD int,int);
01706 extern void AddToSymbolList(PHEAD WORD);
01707 extern int AddToListPoly(PHEAD0);
01708 extern int InvPoly(PHEAD WORD *,WORD,WORD);
01709
01710 extern int ReadFromScratch(FILEHANDLE *,POSITION *,UBYTE *,POSITION *);
01711 extern int AddToScratch(FILEHANDLE *,POSITION *,UBYTE *,POSITION *,int);
01712
01713 extern int DoPreAppendPath(UBYTE *);
01714 extern int DoPrePrependPath(UBYTE *);
01715
01716 extern int DoSwitch(PHEAD WORD *, WORD *);
01717 extern int DoEndSwitch(PHEAD WORD *, WORD *);
01718 extern SWITCHTABLE *FindCase(WORD, WORD);
01719 extern void SwitchSplitMergeRec(SWITCHTABLE *, WORD, SWITCHTABLE *);
01720 extern void SwitchSplitMerge(SWITCHTABLE *, WORD);
01721 extern int DoubleSwitchBuffers(void);
01722
01723 extern int DistrN(int, int *, int, int *);
01724
01725 extern int DoNamespace(UBYTE *);
01726 extern int DoEndNamespace(UBYTE *);
01727 extern int DoUse(UBYTE *);
01728 extern UBYTE *SkipName(UBYTE *);
01729 extern UBYTE *ConstructName(UBYTE *,UBYTE);
01730 extern int DoSetUserFlag(UBYTE *);
01731 extern int DoClearUserFlag(UBYTE *);
01732 extern int DoUserFlag(UBYTE *,int);
01733
01734 VERTEX *CreateVertex(MODEL *);
01735 UBYTE *ReadParticle(UBYTE *, VERTEX *,MODEL *, int);
01736 int CoModel(UBYTE *);
01737 int CoParticle(UBYTE *);
01738 int CoVertex(UBYTE *);
01739 int CoEndModel(UBYTE *);
01740 int LoadModel(MODEL *);
01741
01742 int CoSetUserFlag(UBYTE *);
01743 int CoClearUserFlag(UBYTE *);
01744 int CoCreateAllLoops(UBYTE *);
01745 int CoCreateAllPaths(UBYTE *);
01746 int CoCreateAll(UBYTE *);

```

```

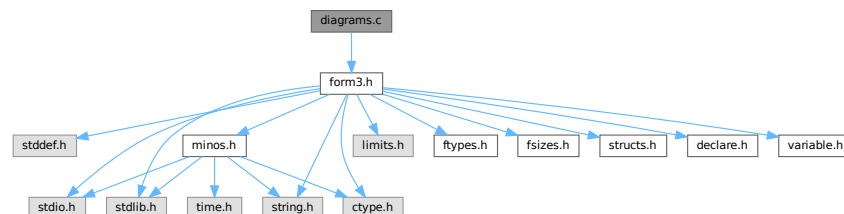
01747
01748 WORD AllLoops(PHEAD WORD *,WORD);
01749 LONG StartLoops(PHEAD WORD *,WORD,WORD,WORD *,WORD,WORD *,WORD);
01750 LONG GenLoops(PHEAD WORD *,WORD,WORD,WORD *,WORD,WORD *,WORD);
01751 void LoopOutput(PHEAD WORD *,WORD,WORD *,WORD);
01752 WORD AllPaths(PHEAD WORD *,WORD);
01753 LONG GenPaths(PHEAD WORD *,WORD,WORD,WORD *,WORD,WORD *,WORD);
01754 void PathOutput(PHEAD WORD *,WORD,WORD *,WORD);
01755
01756 #ifdef WITHFLOAT
01757 int DoStartFloat(UBYTE *);
01758 int DoEndFloat(UBYTE *);
01759 int SetFloatPrecision(WORD);
01760 void SetupMZVTables(void);
01761 void SetupMPFTables(void);
01762 void ClearMZVTables(void);
01763 int EvaluateEuler(PHEAD WORD *,WORD,WORD);
01764 int EvaluateMZVhalf(PHEAD WORD *,WORD,WORD);
01765 int EvaluateSqrt(PHEAD WORD *,WORD,WORD);
01766 int CoEvaluate(UBYTE *);
01767 int PrintFloat(WORD *fun,int numdigits);
01768 int CoToRat(UBYTE *);
01769 int CoToFloat(UBYTE *);
01770 int ToRat(PHEAD WORD *,WORD);
01771 int ToFloat(PHEAD WORD *,WORD);
01772 WORD FloatFunToRat(PHEAD UWORD *,WORD *);
01773 WORD AddWithFloat(PHEAD WORD **,WORD **);
01774 WORD MergeWithFloat(PHEAD WORD **,WORD **);
01775 void ClearFloat(void);
01776 int TestFloat(WORD *);
01777 SBYTE *ReadFloat(SBYTE *);
01778 UBYTE *CheckFloat(UBYTE *,int *);
01779 void SetFloatPrecision(LONG);
01780 int EvaluateFun(PHEAD WORD *,WORD,WORD *);
01781 #endif
01782
01783 /*
01784 #} Declarations :
01785 */
01786 #endif

```

4.21 diagrams.c File Reference

#include "form3.h"

Include dependency graph for diagrams.c:



Functions

- int [CoCanonicalize](#) (UBYTE *s)
- int [DoCanonicalize](#) (PHEAD WORD *term, WORD *params)
- int [DoTopologyCanonicalize](#) (PHEAD WORD *term, WORD vert, WORD edge, WORD *args)
- int [DoShattering](#) (PHEAD WORD *connect, WORD *environ, WORD *partitions, WORD nvert)

4.21.1 Detailed Description

Contains the wrapper routines for diagram manipulations.

Definition in file [diagrams.c](#).

4.21.2 Function Documentation

4.21.2.1 CoCanonicalize()

```
int CoCanonicalize (
    UBYTE * s )
```

Definition at line 64 of file [diagrams.c](#).

4.21.2.2 DoCanonicalize()

```
int DoCanonicalize (
    PHEAD WORD * term,
    WORD * params )
```

Definition at line 185 of file [diagrams.c](#).

4.21.2.3 DoTopologyCanonicalize()

```
int DoTopologyCanonicalize (
    PHEAD WORD * term,
    WORD vert,
    WORD edge,
    WORD * args )
```

Definition at line 417 of file [diagrams.c](#).

4.21.2.4 DoShattering()

```
int DoShattering (
    PHEAD WORD * connect,
    WORD * environ,
    WORD * partitions,
    WORD nvert )
```

Definition at line 626 of file [diagrams.c](#).

4.22 diagrams.c

[Go to the documentation of this file.](#)

```
00001
00005 /* # [ License : */
00006 /*
00007 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00008 * When using this file you are requested to refer to the publication
00009 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00010 * This is considered a matter of courtesy as the development was paid
00011 * for by FOM the Dutch physics granting agency and we would like to
00012 * be able to track its scientific use to convince FOM of its value
00013 * for the community.
00014 *
00015 * This file is part of FORM.
00016 *
00017 * FORM is free software: you can redistribute it and/or modify it under the
```

```

00018 *   terms of the GNU General Public License as published by the Free Software
00019 *   Foundation, either version 3 of the License, or (at your option) any later
00020 *   version.
00021 *
00022 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00023 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00024 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00025 *   details.
00026 *
00027 *   You should have received a copy of the GNU General Public License along
00028 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00029 */
00030 /* #] License : */
00031 /*
00032     #[ Includes : diagrams.c
00033 */
00034
00035 #include "form3.h"
00036
00037 static WORD one = 1;
00038
00039 /*
00040     #] Includes :
00041     #[ CoCanonicalize :
00042
00043     Syntax:
00044         Canonicalize,mainoption,...
00045     With the main options currently:
00046         Canonicalize,topology,vertexfunction,edgefunction,OutDollar,extraoptions;
00047         Canonicalize,polynomial,InDollar,set_or_setname_or_dollar,OutDollar,extraoptions;
00048     The vertex function needs to have the format (assume it is called vx):
00049         vx(p1,p2,-p3) or vx(-p1,p2,p3,-p4) etc.
00050     The external lines have a vertex with only one line.
00051     All momenta that form connections should be unique.
00052     In principle the - signs are not relevant for the topology, but they
00053     may exist already in the remaining part of the diagram. They are also
00054     part of the canonical form.
00055     The edge function can be used in different ways, depending on the options.
00056     The extraoption(s) should be nonnegative integers or $variables that evaluate
00057     into nonnegative integers (integers less than 2^31).
00058     The Indollar variable contains the polynomial to be canonicalized.
00059     The OutDollar variable should be the name of a $-variable (as in $out) which
00060     will be filled with a replace_ function. The canonicalization can then
00061     be executed in the whole term with the Multiply $out; command.
00062 */
00063
00064 int CoCanonicalize(UBYTE *s)
00065 {
00066     WORD args[10], *a, num;
00067     UBYTE *t, c;
00068     args[0] = TYPECANONICALIZE;
00069     a = args+2;
00070     while ( *s == ',' || *s == '\t' || *s == ' ' ) s++;
00071     t = s; while ( FG.cTable[*s] == 0 ) s++;
00072     c = *s; *s = 0;
00073     if ( StrICmp(t,(UBYTE *)("topology")) == 0 ) {
00074         *s = c; *a++ = 0;
00075         while ( *s == ',' || *s == '\t' || *s == ' ' ) s++;
00076         s = GetFunction(s,a);
00077         while ( *s == ',' || *s == '\t' || *s == ' ' ) s++;
00078         s = GetFunction(s,a+1);
00079         if ( *a == 0 || a[1] == 0 ) return(1);
00080         a += 2;
00081     }
00082     else if ( StrICmp(t,(UBYTE *)("polynomial")) == 0 ) {
00083         *s = c; *a++ = 1;
00084         while ( *s == ',' || *s == '\t' || *s == ' ' ) s++;
00085     /*
00086         Now get the name of the input $-variable
00087     */
00088         if ( *s != '$' ) {
00089             MesPrint("&Canonicalize statement needs a $-variable for its input.");
00090             return(1);
00091         }
00092         s++; t = s; while ( FG.cTable[*s] < 2 ) s++;
00093         c = *s; *s = 0;
00094         if ( GetName(AC.dollarnames,t,&num,NOAUTO) == CDOLLAR ) *a++ = num;
00095         else { *a++ = AddDollar(t,DOLINDEX,&one,1); }
00096         *s = c;
00097     /*
00098         Now the set
00099     */
00100         if ( *s == '{' ) {
00101             t = s+1; SKIPBRA2(s)
00102             c = *s; *s = 0;
00103             *a++ = DoTempSet(t,s);
00104             *s++ = c;

```

```

00105     }
00106     else if ( FG.cTable[*s] == 0 || *s == '[' ) {
00107         t = s;
00108         if ( ( s = SkipAName(s) ) == 0 ) {
00109             MesPrint("&Illegal name for set in Canonicalize statement: %s",t);
00110             return(1);
00111         }
00112         c = *s; *s = 0;
00113         if ( GetName(AC.varnames,t,a,WITHAUTO) == CSET ) {
00114             if ( Sets[*a].type != CSYMBOL ) {
00115                 MesPrint("&In Canonicalize: %s is not a set of symbols.",t);
00116                 return(1);
00117             }
00118         }
00119         else {
00120             MesPrint("&In Canonicalize: %s is not a set.",t);
00121             return(1);
00122         }
00123         *s = c; a++;
00124     }
00125     else if ( *s == '$' ) {
00126         s++; t = s; while ( FG.cTable[*s] < 2 ) s++;
00127         c = *s; *s = 0;
00128         if ( GetName(AC.dollarnames,t,&num,NOAUTO) == CDOLLAR ) *a++ = -num-2;
00129         else {
00130             MesPrint("&In Canonicalize: %s is undefined.",t-1);
00131             return(1);
00132         }
00133         *s = c;
00134     }
00135     else {
00136         MesPrint("&In Canonicalize: Illegal third(=set) argument.");
00137         return(1);
00138     }
00139 }
00140 else {
00141     MesPrint("&Unrecognized option in Canonicalize statement: %s",t);
00142     return(1);
00143 }
00144 /*
00145  Now get the name of the output $-variable
00146 */
00147 while ( *s == ',' || *s == '\t' || *s == ' ' ) s++;
00148 if ( *s != '$' ) {
00149     MesPrint("&Canonicalize statement needs a $-variable for its output.");
00150     return(1);
00151 }
00152 s++; t = s; while ( FG.cTable[*s] < 2 ) s++;
00153 c = *s; *s = 0;
00154 if ( GetName(AC.dollarnames,t,&num,NOAUTO) == CDOLLAR ) *a++ = num;
00155 else { *a++ = AddDollar(t,DOLINDEX,&one,1); }
00156 *s = c;
00157 /*
00158  Now the options. At the moment we just do one of them.
00159  (the first extra option is relevant to determine the use of the edge function)
00160  In the future we may have to be more flexible.
00161 */
00162 a[0] = 0; /* default value */
00163 while ( *s == ',' || *s == '\t' || *s == ' ' ) s++;
00164 if ( *s != 0 ) {
00165     s = GetNumber(s,a);
00166     if ( *a == -1 ) return(1);
00167     while ( *s == ',' || *s == '\t' || *s == ' ' ) s++;
00168     a++;
00169 }
00170 /*
00171  Now complete the args string and put it in the compiler buffer
00172 */
00173 args[1] = a-args;
00174 AddNtoL(args[1],args);
00175 return(0);
00176 }
00177
00178 /*
00179  #] CoCanonicalize :
00180  #[ DoCanonicalize :
00181
00182  Does the canonicalization. The output term overwrites the input term.
00183 */
00184
00185 int DoCanonicalize(PHEAD WORD *term, WORD *params)
00186 {
00187     WORD args[10];
00188     int i;
00189     /*
00190      First check whether we need to expand dollars;
00191      */

```



```

00192     for ( i = 0; i < params[1]; i++ ) args[i] = params[i];
00193     if ( args[2] == 0 ) { /* topology */
00194         for ( i = 3; i < 5; i++ ) {
00195             if ( args[i] < 0 ) { /* This is a dollar */
00196                 args[i] = DolToFunction(BHEAD -args[i]-2);
00197                 if ( args[i] == 0 ) {
00198                     MLOCK(ErrorMessageLock);
00199                     MesPrint("Value of $-variable in Canonicalize statement should be a function.");
00200                     MUNLOCK(ErrorMessageLock);
00201                     Terminate(-1);
00202                 }
00203             }
00204         }
00205         for ( i = 6; i < args[1]; i++ ) { /* Extra options */
00206             if ( args[i] < 0 ) { /* This is a dollar */
00207                 args[i] = DolToNumber(BHEAD -args[i]-2);
00208                 if ( args[i] < 0 ) {
00209                     MLOCK(ErrorMessageLock);
00210                     MesPrint("Value of $-variable in Canonicalize statement should be a nonnegative
number < %1.", (LONG)MAXPOSITIVE);
00211                     MUNLOCK(ErrorMessageLock);
00212                     Terminate(-1);
00213                 }
00214             }
00215         }
00216         switch ( args[6] ) {
00217             case 1: { /* pass the vertex and edge functions. */
00218                 WORD *tstop, *t, *tedge, *te;
00219                 tstop = term + *term; tstop -= ABS(tstop[-1]);
00220                 t = term+1;
00221                 tedge = AT.WorkPointer; te = tedge+1;
00222                 while ( t < tstop ) {
00223                     if ( *t != args[3] && *t != args[4] ) { t += t[1]; continue; }
00224                     for ( i = 0; i < t[1]; i++ ) te[i] = t[i];
00225                     te += t[1]; t += t[1];
00226                 }
00227                 *te++ = 1; *te++ = 1; *te++ = 3;
00228                 tedge[0] = te-tedge;
00229                 AT.WorkPointer = te;
00230             /*
00231              DoVertexCanonicalize(BHEAD term,tedge,args[3],args[4],args[5],args[6]);
00232             */
00233             AT.WorkPointer = tedge;
00234             } break;
00235             case 2: { /* pass the edge functions only */
00236                 WORD *tstop, *t, *tedge, *te;
00237                 tstop = term + *term; tstop -= ABS(tstop[-1]);
00238                 t = term+1;
00239                 tedge = AT.WorkPointer; te = tedge+1;
00240                 while ( t < tstop ) {
00241                     if ( *t != args[4] ) { t += t[1]; continue; }
00242                     for ( i = 0; i < t[1]; i++ ) te[i] = t[i];
00243                     te += t[1]; t += t[1];
00244                 }
00245                 *te++ = 1; *te++ = 1; *te++ = 3;
00246                 tedge[0] = te-tedge;
00247                 AT.WorkPointer = te;
00248             /*
00249              DoEdgeCanonicalize(BHEAD term,tedge,args[5]);
00250             */
00251             AT.WorkPointer = tedge;
00252             } break;
00253             default: {
00254                 DoTopologyCanonicalize(BHEAD term,args[3],args[4],args+5);
00255             } break;
00256         }
00257     /*
00258     Call here the topology canonicalization
00259     We will have the arguments:
00260     args[3]: The function used as vertex function
00261     args[4]: The function used as edge function
00262     args[5]: The number of the dollar to be used for the output
00263     args[6]: Potentially other options (like saying how to use args[4]).
00264     term:    The term in which the topology resides.
00265     */
00266     }
00267     else if ( args[2] == 1 ) { /* polynomial */
00268         WORD *symlist, nsymlist;
00269         for ( i = 6; i < args[1]; i++ ) { /* Extra options */
00270             if ( args[i] < 0 ) { /* This is a dollar */
00271                 args[i] = DolToNumber(BHEAD -args[i]-2);
00272                 if ( args[i] < 0 ) {
00273                     MLOCK(ErrorMessageLock);
00274                     MesPrint("Value of $-variable in Canonicalize statement should be a nonnegative
number < %1.", (LONG)MAXPOSITIVE);
00275                     MUNLOCK(ErrorMessageLock);
00276

```

```

00277             Terminate(-1);
00278         }
00279     }
00280 }
00281 /*
00282     Now we sort out the set. We create a pointer to the list of set
00283     elements, and we determine the number of elements in the set.
00284 */
00285     symlist = AT.WorkPointer;
00286     if ( args[4] < -1 ) { /* Dollar that should expand into a list of symbols */
00287         DOLLARS d = Dollars - args[4] - 2;
00288         WORD *ds, *insym;
00289         if ( d->type != DOLWILDARGS ) {
00290 NoWildArg:
00291             MLOCK(ErrorMessageLock);
00292             MesPrint("Value of $-variable in Canonicalize statement should be a argument wildcard
of symbol arguments.");
00293             MUNLOCK(ErrorMessageLock);
00294             Terminate(-1);
00295         }
00296         insym = symlist; ds = d->where+1;
00297         while ( *ds ) {
00298             if ( *ds != -SYMBOL ) goto NoWildArg;
00299             *insym++ = ds[1];
00300             ds += 2;
00301         }
00302         nsymlist = insym-symlist;
00303     }
00304     else { /*if ( args[4] >= 0 ) */
00305         WORD *ss, *sy, n;
00306         ss = (WORD *) (AC.SetElementList.lijst)+Sets[args[4]].first;
00307         nsymlist = n = Sets[args[4]].last-Sets[args[4]].first;
00308         sy = symlist = AT.WorkPointer;
00309         NCOPY(sy,ss,n);
00310     }
00311     AT.WorkPointer = symlist+nsymlist;
00312 /*
00313     Call here the polynomial canonicalization
00314     We will have the arguments:
00315     args[3]: The number of the dollar to be used for the input
00316     symlist: an array of symbols
00317     nsymlist: the number of symbols in symlist
00318     args[5]: The number of the dollar to be used for the output
00319     args[6]: Potentially other options.
00320 */
00321 /*
00322     DoPolynomialCanonicalize(BHEAD args[3],symlist,nsymlist,args[5],args[6]);
00323 */
00324     AT.WorkPointer = symlist;
00325 }
00326 return(0);
00327 }
00328
00329 /*
00330 #] DoCanonicalize :
00331 #[ GenTopologies :
00332
00333     This function has the syntax
00334     topologies_(nloops,      Number of loops
00335         nlegs,                Number of legs
00336         setvertexsizes,       A set which tells which vertices are allowed like {3,4}.
00337         set_extmomenta,       The name of a set with the external momenta
00338         set_intmomenta        The name of a set with the internal momenta
00339         [,options])
00340     The output will be using the built in functions vertex_ and edge_.
00341
00342     The test for whether this function can be evaluated is in TestSub (inside
00343     file proces.c) (search for the string TOPOLOGIES).
00344     This passes the code -15 in AN.TeInFun to Generator, which then calls
00345     the GenTopologies routine.
00346 */
00347
00348 #ifdef OLDTPOPO
00349
00350 WORD GenTopologies(PHEAD WORD *term,WORD level)
00351 {
00352     WORD *tl, *ttl, *tstop, *t, *tt;
00353     WORD *oldworkpointer = AT.WorkPointer;
00354     WORD option1 = 0, option2 = 0, setoption = -1;
00355     WORD retval;
00356 /*
00357     We have to go through the testing procedure again, because there could
00358     be more than one topologies_ function and not all have to be expandable.
00359 */
00360     tstop = term+*term; tstop -= ABS(tstop[-1]);
00361     tt = term+1;

```

```

00363     while ( tt < tstop ) {
00364         t = tt; tt = t+t[1];
00365         if ( *t != TOPOLOGIES ) continue;
00366         tt = t + t[1]; t1 = t + FUNHEAD;
00367         if ( t1+10 > tt || *t1 != -SNUMBER || t1[1] < 0 ||          /* loops */
00368             t1[2] != -SNUMBER || ( t1[3] < 0 && t1[3] != -2 ) || /* legs */
00369             t1[4] != -SETSET || Sets[t1[5]].type != CNUMBER ||    /* set vertices */
00370             t1[6] != -SETSET || Sets[t1[7]].type != CVECTOR ||   /* outvectors */
00371             t1[8] != -SETSET || Sets[t1[9]].type != CVECTOR ) continue;
00372         tt1 = t1 + 10;
00373         if ( tt1+2 <= tt && tt1[0] == -SETSET ) {
00374             if ( Sets[t1[5]].last-Sets[t1[5]].first !=
00375                 Sets[tt1[1]].last-Sets[tt1[1]].first ) continue;
00376             setoption = tt1[1]; tt1 += 2;
00377         }
00378         if ( tt1+2 <= tt && tt1[0] == -SNUMBER ) { option1 = tt1[1]; tt1 += 2; }
00379         if ( tt1+2 <= tt && tt1[0] == -SNUMBER ) { option2 = tt1[1]; tt1 += 2; }
00380         AT.setinterntopo = t1[9];
00381         AT.setexterntopo = t1[7];
00382         AT.TopologiesTerm = term;
00383         AT.TopologiesStart = t;
00384         AT.TopologiesLevel = level;
00385         AT.TopologiesOptions[0] = option1;
00386         AT.TopologiesOptions[1] = option2;
00387         retval = GenerateTopologies(BHEAD t1[1],t1[3],t1[5],setoption);
00388         AT.WorkPointer = oldworkpointer;
00389         return(retval);
00390     }
00391     MLOCK(ErrorMessageLock);
00392     MesPrint("Internal error: topologies_ function not encountered.");
00393     MUNLOCK(ErrorMessageLock);
00394     return(-1);
00395 }
00396 }
00397
00398 #endif
00399
00400 /*
00401  #] GenTopologies :
00402  #[ DoTopologyCanonicalize :
00403
00404  term: The term
00405  vert: the vertex function
00406  edge: the edge function
00407  args[0]: the number of the output dollar
00408  args[1]: options
00409  return value should be zero if all is correct.
00410
00411  The external lines connect to an 'external vertex' which has only one line.
00412  The vertices are of a type: vertex_(p1,p2,-p3)*vertex_(p3,p4,p5) etc.
00413  The edges indicate noninteger powers of the lines:
00414      edge_(p1,2) means 1/p1.p1^(2*ep)
00415  */
00416
00417 int DoTopologyCanonicalize(PHEAD WORD *term,WORD vert,WORD edge,WORD *args)
00418 {
00419     int nvert = 0, nvert2, i, ii, jj, flipnames = 0, nparts/*, level, num */;
00420     WORD *tstop, *t, *tt, *tend, *td;
00421     WORD *oldworkpointer = AT.WorkPointer;
00422     WORD *termcopy = TermMalloc("TopologyCanonize1");
00423     WORD *vet= TermMalloc("TopologyCanonize2");
00424     WORD *partition, *environ, *connect, *pparts, *p;
00425     /*
00426     WORD *pparts;
00427     */
00428     WORD momenta[150],flips[50],nmomenta = 0, nflips = 0;
00429     /*
00430     Step one: the vertices should get a number. We copy the term for this.
00431         We need a high number for the vertex function to make sure
00432         that it comes after the edge function in the sorting.
00433     */
00434     if ( args[0] < args[1] ) { flipnames = 1; }
00435     tend = term + *term; tend -= ABS(tend[-1]); t = term+1; tt = termcopy+1;
00436     while ( t < tend ) {
00437         if ( *t == vert ) {
00438             for ( i = FUNHEAD; i < t[1]; i += 2 ) {
00439                 if ( t[i] == -VECTOR || ( t[i] == -INDEX && t[i+1] < 0 ) ) {
00440                     momenta[nmomenta++] = -VECTOR;
00441                     momenta[nmomenta++] = t[i+1];
00442                 }
00443                 else if ( t[i] == -MINVECTOR ) {
00444                     momenta[nmomenta++] = -MINVECTOR;
00445                     momenta[nmomenta++] = t[i+1];
00446                 }
00447                 else goto notgoodvertex;
00448                 momenta[nmomenta++] = nvert;
00449             }

```

```

00450         ii = FUNHEAD; i = t[1]-FUNHEAD;
00451         NCOPY(tt,t,ii)
00452         if ( flipnames ) tt[-FUNHEAD] = edge;
00453         tt[-FUNHEAD+1] += 2;
00454         *tt++ = -CNUMBER; *tt++ = nvert++;
00455     }
00456     else if ( *t == edge && flipnames ) {
00457         i = t[1] - 1; *tt++ = vert; t++;
00458     }
00459     else {
00460 notgoodvertex:
00461         i = t[1];
00462     }
00463     NCOPY(tt,t,i)
00464 }
00465 while ( t < tend ) *tt++ = *t++;
00466 termcopy[0] = tt - termcopy;
00467 if ( flipnames ) EXCH(edge,vert)
00468 nvert2 = nvert*nvert;
00469 /*
00470 Sort the momenta. Keep the sign order.
00471 */
00472 for ( i = 0; i < nmomenta-3; i+=3 ) {
00473     jj = i;
00474     while ( jj >= 0 && momenta[jj+4] > momenta[jj+1] ) {
00475         EXCH(momenta[jj+5],momenta[jj+2])
00476         EXCH(momenta[jj+4],momenta[jj+1])
00477         EXCH(momenta[jj+3],momenta[jj])
00478         jj -= 3;
00479     }
00480 }
00481 /*
00482 Step two: make now the edge functions in the proper notation.
00483 */
00484 t = vet+1;
00485 for ( i = 0; i < nmomenta; i += 6 ) {
00486     if ( momenta[i] == -VECTOR && momenta[i+3] == -MINVECTOR
00487         && momenta[i+1] == momenta[i+4] ) {
00488     }
00489     else if ( momenta[i] == -MINVECTOR && momenta[i+3] == -VECTOR
00490         && momenta[i+1] == momenta[i+4] ) {
00491         flips[nflips++] = momenta[i+1];
00492         DUMMYUSE(flips[nflips-1]);
00493     }
00494     else { /* something wrong with the momenta */
00495         MLOCK(ErrorMessageLock);
00496         MesPrint("No momentum conservation or wrong momenta in Canonicalize statement");
00497         MUNLOCK(ErrorMessageLock);
00498         Terminate(-1);
00499     }
00500     *t++ = EDGE; *t++ = FUNHEAD+10; FILLFUN(t)
00501     *t++ = -SNUMBER; *t++ = momenta[i+2];
00502     *t++ = -SNUMBER; *t++ = momenta[i+5];
00503     *t++ = -VECTOR; *t++ = momenta[i+1];
00504     *t++ = -SNUMBER; *t++ = 0; /* provisional power/color, multiple of ep */
00505     *t++ = -SNUMBER; *t++ = 0; /* provisional power/color, integer */
00506 }
00507 tend = t;
00508 *t++ = 1; *t++ = 1; *t++ = 3; vet[0] = t-vet; *t = 0;
00509 /*
00510 Now the powers of the denominators
00511 */
00512 tstop = termcopy+*termcopy; tstop -= ABS(tstop[-1]); td = termcopy+1;
00513 while ( td < tstop ) {
00514     if ( *td == edge && td[1] == FUNHEAD+4 ) {
00515         if ( td[FUNHEAD+2] == -SNUMBER && ( td[FUNHEAD] == -VECTOR || td[FUNHEAD] == -INDEX
00516             || td[FUNHEAD] == -MINVECTOR ) ) {}
00517         else {
00518             MLOCK(ErrorMessageLock);
00519             MesPrint("Illegal argument in edge function in Canonicalize statement");
00520             MUNLOCK(ErrorMessageLock);
00521             Terminate(-1);
00522         }
00523         tt = vet+1;
00524         while ( tt < tend ) {
00525             if ( tt[FUNHEAD+5] == td[FUNHEAD+1] ) { tt[FUNHEAD+7] = td[FUNHEAD+3]; break; }
00526             tt += tt[1];
00527         }
00528     }
00529     else if ( *td == DOTPRODUCT ) break;
00530     td += td[1];
00531 }
00532 if ( td < tstop ) {
00533     tt = vet+1;
00534     while ( tt < tend ) {
00535 /*
00536         tt[FUNHEAD+5] is a vector. We look for dotproducts with twice tt[FUNHEAD+5]

```

```

00537 */
00538         for ( i = 2; i < td[1]; i += 3 ) {
00539             if ( td[i] == tt[FUNHEAD+5] && td[i+1] == tt[FUNHEAD+5] ) {
00540                 tt[FUNHEAD+9] = td[i+2];
00541                 break;
00542             }
00543         }
00544         tt += tt[1];
00545     }
00546 }
00547 Normalize(BHEAD vet);
00548 /*
00549 Now we have a term `vet` in the proper notation and we can start.
00550 To keep track of the shattering we use an array of 2*nvert.
00551 each entry is Number,marker
00552 When the marker is zero, the vertices are in the same partition.
00553 For the environment we need a matrix that is nvert x nvert
00554 At the same time we keep the connectivity matrix, because that will
00555 save much time later.
00556 The partitions are stored in a matrix as well. This allows them to be
00557 treated as a stack. The entries are separated by 0 if they belong to
00558 the same part, and by a 1 when they belong to different parts.
00559 */
00560 partition = AT.WorkPointer; AT.WorkPointer += 2*nvert2;
00561 for ( i = 0; i < nvert; i++ ) { partition[2*i] = i; partition[2*i+1] = 0; }
00562 partition[2*i-1] = -1; /* end of the first part which is currently all vertices */
00563 nparts = 1;
00564
00565 connect = AT.WorkPointer; AT.WorkPointer += nvert2;
00566 for ( i = 0; i < nvert2; i++ ) connect[i] = 0;
00567 tstop = vet+vet; tstop -= ABS(tstop[-1]); t = vet+1;
00568 while ( t < tstop ) {
00569     if ( *t == EDGE ) {
00570         connect[t[FUNHEAD+1]*nvert+t[FUNHEAD+3]]++;
00571         connect[t[FUNHEAD+3]*nvert+t[FUNHEAD+1]]++;
00572     }
00573     t += t[1];
00574 }
00575 for ( i = 0; i < nvert; i++ ) {
00576 MesPrint("connectivity: %d -- %a",i,nvert,connect+i*nvert);
00577 }
00578 /*
00579 Create the environment matrix and sort it.
00580 */
00581 environ = AT.WorkPointer; AT.WorkPointer += nvert2;
00582 /*
00583 And now the refinement process starts.
00584 */
00585 WantAddPointers(nvert+1);
00586 for ( i = 0; i < nvert2; i++ ) environ[i] = 0;
00587 /* level = 0; */
00588 pparts = partition;
00589 while ( nparts < nvert ) {
00590     nparts = DoShattering(BHEAD connect,environ,pparts,nvert);
00591     if ( nparts < nvert ) { /* raise level and make a copy and split a part */
00592         p = pparts + 2*nvert;
00593         /* level++; */
00594         for ( i = 0; i < 2*nvert; i++ ) p[i] = pparts[i];
00595         for ( ii = 0; ii < 2*nvert; ii += 2 ) {
00596             if ( p[ii+1] == 0 ) { /* found a part with more than one */
00597                 /* num = 2; */ i = ii+2;
00598                 while ( p[i+1] == 0 ) { /* num++; */ i += 2; }
00599                 p[ii+1] = -1; pparts = p;
00600                 break;
00601             }
00602         }
00603     }
00604 }
00605 }
00606 MesPrint("partition: %d -- %a",nparts,2*nvert,pparts);
00607
00608 }
00609 /*
00610 Just for now
00611 */
00612 PutTermInDollar(vet,args[0]);
00613
00614
00615 TermFree(vet,"TopologyCanonize2");
00616 TermFree(termcopy,"TopologyCanonize1");
00617 AT.WorkPointer = oldworkpointer;
00618 return(0);
00619 }
00620
00621 /*
00622 #] DoTopologyCanonicalize :
00623 #[ DoShattering :

```

```

00624 */
00625
00626 int DoShattering(PHEAD WORD *connect, WORD *environ, WORD *partitions, WORD nvert)
00627 {
00628     int nparts, i, j, ii, jj, iii, jjj, newmarker;
00629     WORD **p = AT.pWorkSpace + AT.pWorkPointer, *part, *endpart;
00630     WORD *poin1, *poin2;
00631     #ifdef SHATBUG
00632     MesPrint("Entering DoShattering. partitions = %a", 2*nvert, partitions);
00633     #endif
00634     restart:
00635     /*
00636         Determine the number of parts
00637         p will be an array with pointers to the parts.
00638         We made space for this array in the calling routine and because this
00639         routine is not calling any other routines we do not need to raise
00640         the pointer in this stack (AT.pWorkPointer).
00641     */
00642     nparts = 0; newmarker = 0;
00643     part = partitions; endpart = part + 2*nvert;
00644     p[0] = part;
00645     while ( part < endpart ) {
00646         if ( part[1] != 0 ) { p[++nparts] = part+2; }
00647         part += 2;
00648     }
00649     for ( i = 0; i < nparts; i++ )
00650         AT.WorkPointer[i] = (p[i+1]-p[i])/2;
00651     #ifdef SHATBUG
00652     MesPrint("DoShattering: calculated the pointers");
00653     MesPrint("DoShattering: sizes: %a", nparts, AT.WorkPointer);
00654     MesPrint("DoShattering: p[0]: %a, p[1]: %a", 6, p[0], 6, p[1]);
00655     #endif
00656     for ( i = 0; i < nparts; i++ ) {
00657         if ( AT.WorkPointer[i] > 1 ) {
00658             for ( j = 0; j < nparts; j++ ) {
00659                 /*
00660                     Shatter part i wrt part j.
00661                     if there is action, go to restart.
00662                 */
00663                 for ( ii = 0; ii < AT.WorkPointer[i]; ii++ ) {
00664                     for ( jj = 0; jj < AT.WorkPointer[j]; jj++ ) {
00665                         environ[ii*AT.WorkPointer[j]+jj] += connect[p[i][2*ii]*nvert+p[j][2*jj]];
00666                     }
00667                 }
00668                 #ifdef SHATBUG
00669                 for ( ii = 0; ii < AT.WorkPointer[i]; ii++ ) {
00670                     MesPrint("Environ(%d,%d): %a", i, j, AT.WorkPointer[j], environ+ii*AT.WorkPointer[j]);
00671                 }
00672                 #endif
00673                 /*
00674                     Sort the rows internally, then sort the rows wrt each other
00675                     and finally place new markers. If a new marker, we restart.
00676                     Don't forget to clean up the environ array.
00677                 */
00678                 for ( ii = 0; ii < nvert; ii++ ) {
00679                     poin1 = environ+ii*AT.WorkPointer[j];
00680                     for ( jj = 0; jj < AT.WorkPointer[j]-1; jj++ ) {
00681                         jjj = jj;
00682                         while ( jjj >= 0 && poin1[jjj+1] > poin1[jjj] ) {
00683                             EXCH(poin1[jjj+1], poin1[jjj]);
00684                             jjj--;
00685                         }
00686                     }
00687                 }
00688                 #ifdef SHATBUG
00689                 for ( ii = 0; ii < AT.WorkPointer[i]; ii++ ) {
00690                     MesPrint("environ(%d,%d): %a", i, j, AT.WorkPointer[j], environ+ii*AT.WorkPointer[j]);
00691                 }
00692                 #endif
00693                 for ( ii = 0; ii < AT.WorkPointer[i]-1; ii++ ) {
00694                     poin2 = environ+ii*AT.WorkPointer[j]; poin1 = poin2+AT.WorkPointer[j];
00695                     iii = ii;
00696                     while ( iii >= 0 && ( CmpArray(poin1, poin2, AT.WorkPointer[j]) < 0 ) ) {
00697                         EXCHN(poin2, poin1, AT.WorkPointer[j]);
00698                         EXCH(p[i][2*iii+2], p[i][2*iii]);
00699                         iii--; poin1 = poin2; poin2 = poin1-AT.WorkPointer[j];
00700                     }
00701                 }
00702                 #ifdef SHATBUG
00703                 for ( ii = 0; ii < AT.WorkPointer[i]; ii++ ) {
00704                     MesPrint("environ(%d,%d): %a", i, j, AT.WorkPointer[j], environ+ii*AT.WorkPointer[j]);
00705                 }
00706                 MesPrint("partitions = %a", 2*nvert, partitions);
00707                 #endif
00708                 for ( ii = 0; ii < AT.WorkPointer[i]-1; ii++ ) {
00709                     poin2 = environ+ii*AT.WorkPointer[j]; poin1 = poin2+AT.WorkPointer[j];
00710                     if ( CmpArray(poin1, poin2, AT.WorkPointer[j]) == 0 ) continue;

```

```

00711         p[i][2*ii+1] = -1; nparts++; newmarker++;
00712     }
00713 #ifdef SHATBUG
00714 MesPrint("partitions = %a",2*nvert,partitions);
00715 #endif
00716 /*
00717     Clear environ. This is probably faster than just clearing the whole array.
00718     Maybe in the future a test could be done on nvert to decide how to clear.
00719 */
00720     for ( ii = 0; ii < AT.WorkPointer[i]; ii++ ) {
00721         for ( jj = 0; jj < AT.WorkPointer[j]; jj++ ) {
00722             environ[ii*AT.WorkPointer[j]+jj] = 0;
00723         }
00724     }
00725     if ( newmarker ) { goto restart; }
00726 }
00727 }
00728 }
00729 return(nparts);
00730 }
00731
00732 /*
00733 #] DoShattering :
00734 */

```

4.23 diawrap.cc

```

00001 // #[ Includes : diawrap.cc
00002
00003 extern "C" {
00004 #include "form3.h"
00005 }
00006
00007 #include "grccparam.h"
00008 #include "grcc.h"
00009
00010 #define MAXPOINTS 120
00011
00012 typedef struct ToPoTyPe {
00013     WORD *vert;
00014     WORD *vertmax;
00015     Options *opt;
00016     int cldeg[MAXPOINTS], clnum[MAXPOINTS], clext[MAXPOINTS];
00017     int cmind[MAXLEGS+1],cmaxd[MAXLEGS+1];
00018     int ncl, nvert;
00019 } TOPOTYPE;
00020
00021 static void ProcessDiagram(EGraph *eg, void *ti);
00022 static int processVertex(TOPOTYPE *TopoInf, int pointsremaining, int level);
00023
00024 // #] Includes :
00025 // #[ LoadModel :
00026
00027 int LoadModel (MODEL *m)
00028 {
00029 //
00030 // First check whether there was a model already.
00031 // In the Kaneko setup there can be only one at the same time.
00032 // Hence if there was one we need to remove it first.
00033 // Unless of course, it was already the model we want.
00034 //
00035 if ( m->grccmodel != NULL ) return(0);
00036
00037 int i,j,k;
00038 //
00039 // First the information that goes into the Model struct.
00040 // Note that new Model takes over (does not copy) minp.cnamlist.
00041 //
00042 MInput minp;
00043 PInput pinp;
00044 IInput iinp;
00045 if ( m->ncouplings > GRCC_MAXNCPLG ) {
00046     MesPrint("Too many coupling constants in model. Current limit is %d.",(WORD)GRCC_MAXNCPLG);
00047     MesPrint("Suggestion: recompile Form with a larger value for GRCC_MAXNCPLG");
00048     return(-1);
00049 }
00050 minp.defpart = GRCC_DEFBYCODE;
00051 minp.name = (char *) (m->name);
00052 minp.ncouple = m->ncouplings;
00053 for ( i = 0; i < GRCC_MAXNCPLG; i++ ) minp.cnamlist[i] = NULL;
00054 for ( i = 0; i < minp.ncouple; i++ )
00055     minp.cnamlist[i] = (char *) (VARNAME(symbols,m->couplings[i]));
00056 Model *mdl = new Model(&minp);

```

```

00057 //
00058 // Now the particles
00059 //
00060 for ( i = 0; i < m->nparticles; i++ ) {
00061     if ( minp.defpart == GRCC_DEFBYCODE ) {
00062         pinp.name = NULL;
00063         pinp.aname = NULL;
00064         pinp.pcode = m->vertices[i]->particles[0].number;
00065         pinp.acode = m->vertices[i]->particles[1].number;
00066         switch ( m->vertices[i]->particles[0].spin ) {
00067             case 1:
00068                 pinp.ptypec = GRCC_PT_Scalar;
00069                 break;
00070             case -1:
00071                 pinp.ptypec = GRCC_PT_Ghost;
00072                 break;
00073             case 2:
00074                 if ( m->vertices[i]->particles[0].type == 0 )
00075                     pinp.ptypec = GRCC_PT_Majorana;
00076                 else
00077                     pinp.ptypec = GRCC_PT_Undef;
00078                 break;
00079             case -2:
00080                 if ( m->vertices[i]->particles[0].type == 0 )
00081                     pinp.ptypec = GRCC_PT_Majorana;
00082                 else
00083                     pinp.ptypec = GRCC_PT_Dirac;
00084                 break;
00085             case 3:
00086                 pinp.ptypec = GRCC_PT_Vector;
00087                 break;
00088             default:
00089                 pinp.ptypec = GRCC_PT_Undef;
00090                 break;
00091         }
00092     }
00093     else {
00094         pinp.name = (char *) (VARNAME(functions,m->vertices[i]->particles[0].number));
00095         pinp.aname = (char *) (VARNAME(functions,m->vertices[i]->particles[1].number));
00096         switch ( m->vertices[i]->particles[0].spin ) {
00097             case 1:
00098                 pinp.ptypen = "scalar";
00099                 break;
00100             case -1:
00101                 pinp.ptypen = "ghost";
00102                 break;
00103             case 2:
00104                 if ( m->vertices[i]->particles[0].type == 0 )
00105                     pinp.ptypen = "majorana";
00106                 else
00107                     pinp.ptypen = "undef";
00108                 break;
00109             case -2:
00110                 if ( m->vertices[i]->particles[0].type == 0 )
00111                     pinp.ptypen = "majorana";
00112                 else
00113                     pinp.ptypen = "dirac";
00114                 break;
00115             case 3:
00116                 pinp.ptypen = "vector";
00117                 break;
00118             default:
00119                 pinp.ptypen = "undef";
00120                 break;
00121         }
00122     }
00123     pinp.exonly = m->vertices[i]->externonly;
00124     mdl->addParticle(&pinp);
00125 }
00126 mdl->addParticleEnd();
00127 //
00128 // Now the vertices
00129 //
00130 for ( i = m->nparticles; i < m->invertices; i++ ) {
00131     VERTEX *v = m->vertices[i];
00132     if ( minp.defpart == GRCC_DEFBYCODE ) {
00133         iinp.icode = NODEFUNCTION+i;
00134         iinp.name = NULL;
00135     }
00136     else {
00137         iinp.name = "node_";
00138     }
00139     iinp.nplistn = v->nparticles;
00140     for ( j = 0; j < iinp.nplistn; j++ ) {
00141         if ( minp.defpart == GRCC_DEFBYCODE ) {
00142             iinp.plistc[j] = v->particles[j].number;
00143         }
00144     }

```



```

00144         else {
00145             iinp.plistn[j] = (char *) (VARNAME(functions,v->particles[j].number));
00146         }
00147     }
00148 //
00149 // We need a properly ordered list of coupling constants
00150 // The ordered list is in m->couplings.
00151 // For each vertex they are in v->couplings.
00152 //
00153 // iinp.cvallist = (int *)Malloca1(m->ncouplings*sizeof(int),"couplings");
00154
00155     for ( j = 0; j < m->ncouplings; j++ ) {
00156         iinp.cvallist[j] = 0;
00157         for ( k = 0; k < v->ncouplings; k += 2 ) {
00158             if ( v->couplings[k] == m->couplings[j] ) {
00159                 iinp.cvallist[j] = v->couplings[k+1];
00160                 break;
00161             }
00162         }
00163     }
00164     mdl->addInteraction(&iinp);
00165 }
00166 mdl->addInteractionEnd();
00167 m->grccmodel = (void *)mdl;
00168 return(0);
00169 }
00170
00171 // #] LoadModel :
00172 // #[ ConvertParticle :
00173
00174 int ConvertParticle(Model *model,int formnum)
00175 {
00176 //
00177 // Returns the grcc number of the particle, because grcc does not convert
00178 //
00179     int i;
00180     for ( i = 0; i < model->nParticles; i++ ) {
00181         if ( model->particles[i]->pcode == formnum ) { return(i); }
00182         else if ( model->particles[i]->acode == formnum ) { return(-i); }
00183     }
00184     MesPrint("Particle %d not found in model %s",formnum,model->name);
00185     Terminate(-1);
00186     return(0);
00187 }
00188
00189 // #] ConvertParticle :
00190 // #[ ReConvertParticle :
00191
00192 int ReConvertParticle(Model *model,int grccnum)
00193 {
00194 //
00195 // Returns the grcc number of the particle, because grcc does not convert
00196 //
00197     if ( grccnum < 0 ) { return(model->particles[-grccnum]->acode); }
00198     else { return(model->particles[grccnum]->pcode); }
00199 }
00200
00201 // #] ReConvertParticle :
00202 // #[ numParticle :
00203
00204 int numParticle(MODEL *m,WORD n)
00205 {
00206     int i;
00207     for ( i = 0; i < m->nparticles; i++ ) {
00208         if ( m->vertices[i]->particles[0].number == n ) return(i);
00209         if ( m->vertices[i]->particles[1].number == n ) return(i);
00210     }
00211     MesPrint("numParticle: particle %d not found in model",n);
00212     Terminate(-1);
00213     return(-1);
00214 }
00215
00216 // #] numParticle :
00217 // #[ ProcessDiagram :
00218
00219 void ProcessDiagram(EGraph *eg, void *ti)
00220 {
00221 //
00222 // This is the routine that gets a complete diagram and passes it on
00223 // to Form (Generator) for further algebraic manipulations.
00224 // The term is picked up from AT.diaterm and a new term is constructed
00225 // in the workspace.
00226 //
00227     GETIDENTITY
00228     TERMINFO *info = (TERMINFO *)ti;
00229
00230     if ( ( info->flags & TOPOLOGIESONLY ) == TOPOLOGIESONLY ) return;

```

```

00231
00232     WORD *term = info->term, *newterm, *oldworkpointer = AT.WorkPointer;
00233     WORD *tdia = term + info->diaoffset;
00234     WORD *tail = tdia + tdia[1];
00235     WORD *tend = term + *term;
00236     WORD *fill, *startfill, *cfill, *afill;
00237     int i, j, intr;
00238     Model *model = (Model *)info->currentModel;
00239     MODEL *m = (MODEL *)info->currentMODEL;
00240     int numlegs, vect, edge;
00241
00242     newterm = term + *term;
00243     for ( i = 1; i < info->diaoffset; i++ ) newterm[i] = term[i];
00244     fill = newterm + info->diaoffset;
00245     //
00246     // Now get the nodes
00247     //
00248     if ( ( info->flags & NONODES ) == 0 ) {
00249         for ( i = 0; i < eg->nNodes; i++ ) {
00250             //
00251             //     node_(number,coupling,particle_1(momentum_1),...,particle_n(momentum_n))
00252             //
00253             numlegs = eg->nodes[i]->deg;
00254             startfill = fill;
00255             *fill++ = NODEFUNCTION;
00256             *fill++ = 0;
00257             FILLFUN(fill)
00258             *fill++ = -SNUMBER; *fill++ = i+1;
00259             //
00260             // Now we put the coupling constants. This is done inside the
00261             // function for when we want to work with counterterms.
00262             //
00263             if ( !eg->isExternal(i) ) {
00264                 afill = fill; *fill++ = 0; *fill++ = 1; FILLARG(fill)
00265                 cfill = fill; *fill++ = 0;
00266                 intr = eg->nodes[i]->intrct;
00267                 for ( j = 0; j < model->interacts[intr]->nclist; j++ ) {
00268                     if ( model->interacts[intr]->clist[j] != 0 ) {
00269                         *fill++ = SYMBOL; *fill++ = 4;
00270                         *fill++ = m->couplings[j];
00271                         *fill++ = model->interacts[intr]->clist[j];
00272                     }
00273                 }
00274                 *fill++ = 1; *fill++ = 1; *fill++ = 3;
00275                 *cfill = fill - cfill;
00276                 *afill = fill - afill;
00277             }
00278             else {
00279                 *fill++ = -SNUMBER;
00280                 *fill++ = 1;
00281             }
00282             //
00283             // Now the particles and their momenta.
00284             //
00285             for ( j = 0; j < numlegs; j++ ) {
00286                 *fill++ = ARGHEAD+FUNHEAD+6;
00287                 *fill++ = 0;
00288                 FILLARG(fill)
00289                 edge = eg->nodes[i]->edges[j];
00290                 vect = ABS(edge)-1;
00291                 *fill++ = 6+FUNHEAD;
00292                 int a;
00293                 if ( edge < 0 ) { a = ReConvertParticle(model,-eg->edges[vect]->ptcl); }
00294                 else { a = ReConvertParticle(model,eg->edges[vect]->ptcl); }
00295                 *fill++ = a;
00296                 *fill++ = FUNHEAD+2;
00297                 FILLFUN(fill)
00298                 *fill++ = edge < 0 ? -MINVECTOR: -VECTOR;
00299                 if ( numlegs == 1 || vect < info->numextern ) { // Look up in set of external momenta
00300                     *fill++ = SetElements[Sets[info->externalset].first+vect];
00301                 }
00302                 else { // Look up in set of internal momenta set
00303                     *fill++ = SetElements[Sets[info->internalset].first+(vect-eg->nExtern)];
00304                 }
00305                 *fill++ = 1; *fill++ = 1; *fill++ = 3;
00306             }
00307             startfill[1] = fill-startfill;
00308         }
00309     }
00310     if ( ( info->flags & WITHEDGES ) == WITHEDGES ) {
00311         for ( i = 0; i < eg->nEdges; i++ ) {
00312             int n1 = eg->edges[i]->nodes[0];
00313             int n2 = eg->edges[i]->nodes[1];
00314             // int l1 = eg->edges[i]->nlegs[0];
00315             // int l2 = eg->edges[i]->nlegs[1];
00316
00317             startfill = fill;

```

```

00318         *fill++ = EDGE;
00319         *fill++ = 0;
00320         FILLFUN(fill)
00321         *fill++ = -SNUMBER; *fill++ = i+1; // number of the edge
00322 //
00323         *fill++ = ARGHEAD+FUNHEAD+6;
00324         *fill++ = 0;
00325         FILLARG(fill)
00326         *fill++ = 6+FUNHEAD;
00327         int a = ReConvertParticle(model, eg->edges[i]->ptcl);
00328         *fill++ = a;
00329         *fill++ = FUNHEAD+2;
00330         FILLFUN(fill)
00331         *fill++ = -VECTOR;
00332         // Look up in set of internal momenta set
00333         if ( i < info->numextern ) { // Look up in set of external momenta
00334             *fill++ = SetElements[Sets[info->externalset].first+i];
00335         }
00336         else { // Look up in set of internal momenta set
00337             *fill++ = SetElements[Sets[info->internalset].first+(i-eg->nExtern)];
00338         }
00339         *fill++ = 1; *fill++ = 1; *fill++ = 3;
00340 //
00341         *fill++ = -SNUMBER; *fill++ = n1+1; // number of the node from
00342         *fill++ = -SNUMBER; *fill++ = n2+1; // number of the node to
00343         startfill[1] = fill - startfill;
00344     }
00345 }
00346 if ( ( info->flags & WITHBLOCKS ) == WITHBLOCKS ) {
00347     for ( i = 0; i < eg->econn->nblocks; i++ ) {
00348         startfill = fill;
00349         *fill++ = BLOCK;
00350         *fill++ = 0;
00351         FILLFUN(fill);
00352         *fill++ = -SNUMBER;
00353         *fill++ = i+1;
00354         *fill++ = -SNUMBER;
00355         *fill++ = eg->econn->blocks[i].loop;
00356 //
00357 //         Now we have to make a list of all nodes inside this block
00358 //
00359         int bnodes[GRCC_MAXNODES], k;
00360         WORD *argfill = fill, *funfill;
00361         *fill++ = 0; *fill++ = 0; FILLARG(fill)
00362         *fill++ = 0;
00363         for ( k = 0; k < GRCC_MAXNODES; k++ ) bnodes[k] = 0;
00364         for ( k = 0; k < eg->econn->blocks[i].nmedges; k++ ) {
00365             bnodes[eg->econn->blocks[i].edges[k][0]] = 1;
00366             bnodes[eg->econn->blocks[i].edges[k][1]] = 1;
00367         }
00368         for ( k = 0; k < GRCC_MAXNODES; k++ ) {
00369             if ( bnodes[k] == 0 ) continue;
00370 //
00371 //             Now we put the node inside this argument.
00372 //
00373             funfill = fill;
00374             *fill++ = NODEFUNCTION;
00375             *fill++ = 0;
00376             FILLFUN(fill)
00377             *fill++ = -SNUMBER; *fill++ = k+1;
00378             numlegs = eg->nodes[k]->deg;
00379             for ( j = 0; j < numlegs; j++ ) {
00380                 edge = eg->nodes[k]->edges[j];
00381                 vect = ABS(edge)-1;
00382                 *fill++ = -VECTOR;
00383                 if ( numlegs == 1 || vect < info->numextern ) { // Look up in set of external
momenta
00384                     *fill++ = SetElements[Sets[info->externalset].first+vect];
00385                 }
00386                 else { // Look up in set of internal momenta set
00387                     *fill++ = SetElements[Sets[info->internalset].first+(vect-info->numextern)];
00388                 }
00389             }
00390             funfill[1] = fill-funfill;
00391         }
00392         *fill++ = 1; *fill++ = 1; *fill++ = 3;
00393         *argfill = fill - argfill;
00394         argfill[ARGHEAD] = argfill[0] - ARGHEAD;
00395         startfill[1] = fill-startfill;
00396     }
00397 }
00398 if ( ( info->flags & WITHONEPI ) == WITHONEPI ) {
00399     for ( i = 0; i < eg->econn->nopic; i++ ) {
00400         startfill = fill;
00401         *fill++ = ONEPI;
00402         *fill++ = 0;
00403         FILLFUN(fill);

```

```

00404         *fill++ = -SNUMBER;
00405         *fill++ = i+1;
00406         *fill++ = -ONEPI;
00407         for ( j = 0; j < eg->econn->opics[i].nnodes; j++ ) {
00408             *fill++ = -SNUMBER;
00409             *fill++ = eg->econn->opics[i].nodes[j]+1;
00410         }
00411         startfill[1] = fill-startfill;
00412     }
00413 }
00414 //
00415 // Topology counter. We have exaggerated a bit with the eye on the far future.
00416 //
00417 if ( info->numtopo-1 < MAXPOSITIVE ) {
00418     *fill++ = TOPO; *fill++ = FUNHEAD+2; FILLFUN(fill)
00419     *fill++ = -SNUMBER; *fill++ = (WORD)(info->numtopo-1);
00420 }
00421 else if ( info->numtopo-1 < FULLMAX-1 ) {
00422     *fill++ = TOPO; *fill++ = FUNHEAD+ARGHEAD+4; FILLFUN(fill)
00423     *fill++ = ARGHEAD+4; *fill++ = 0; FILLARG(fill)
00424     *fill++ = 4;
00425     *fill++ = (WORD)((info->numtopo-1) & WORDMASK);
00426     *fill++ = 1; *fill++ = 3;
00427 }
00428 else { // for now: science fiction
00429     *fill++ = TOPO; *fill++ = FUNHEAD+ARGHEAD+6; FILLFUN(fill)
00430     *fill++ = ARGHEAD+6; *fill++ = 0; FILLARG(fill)
00431     *fill++ = 6; *fill++ = (WORD)((info->numtopo-1) » BITSINWORD);
00432     *fill++ = (WORD)((info->numtopo-1) & WORDMASK);
00433     *fill++ = 0; *fill++ = 1; *fill++ = 5;
00434 }
00435 //
00436 // Symmetry factors. We let Normalize do the multiplication.
00437 //
00438 if ( eg->nsym != 1 ) {
00439     *fill++ = SNUMBER; *fill++ = 4; *fill++ = (WORD)eg->nsym; *fill++ = -1;
00440 }
00441 if ( eg->esym != 1 ) {
00442     *fill++ = SNUMBER; *fill++ = 4; *fill++ = (WORD)eg->esym; *fill++ = -1;
00443 }
00444 if ( eg->extperm != 1 ) {
00445     *fill++ = SNUMBER; *fill++ = 4; *fill++ = (WORD)eg->extperm; *fill++ = 1;
00446 }
00447 //
00448 // finish it off
00449 //
00450 while ( tail < tend ) *fill++ = *tail++;
00451 if ( eg->fsign < 0 ) fill[-1] = -fill[-1];
00452 *newterm = fill - newterm;
00453 AT.WorkPointer = fill;
00454
00455 // MesPrint("<> %a",newterm[0],newterm);
00456
00457 Generator(BHEAD newterm,info->level);
00458 AT.WorkPointer = oldworkpointer;
00459 }
00460
00461 // #] ProcessDiagram :
00462 // #[ fendMG :
00463
00464 Bool fendMG(EGraph *eg, void *ti)
00465 {
00466     DUMMYUSE(eg);
00467     DUMMYUSE(ti);
00468     return True;
00469 }
00470
00471 // #] fendMG :
00472 // #[ ProcessTopology :
00473
00474 Bool ProcessTopology(EGraph *eg, void *ti)
00475 {
00476     //
00477     // This routine is called for each new topology.
00478     // New convention: return True; generate the corresponding diagrams if needed
00479     // return False; skip diagram generation (when asked for).
00480     //
00481     TERMINFO *info = (TERMINFO *)ti;
00482 #ifndef WITHEARLYVETO
00483     if ( ( ( info->flags & CHECKEXTERN ) == CHECKEXTERN ) && info->currentMODEL != NULL ) {
00484         int i, j;
00485         int numlegs, vect, edge;
00486         for ( i = 0; i < eg->nNodes; i++ ) {
00487             if ( eg->isExternal(i) ) continue;
00488             numlegs = eg->nodes[i]->deg;
00489             for ( j = 0; j < numlegs; j++ ) {
00490                 edge = eg->nodes[i]->edges[j];

```

```

00491         vect = ABS(edge)-1;
00492         if ( vect < info->numextern && info->legcouple[vect][numlegs] == 0 ) {
00493
00494             // This cannot be.
00495
00496             info->numtopo++;
00497             return False;
00498         }
00499     }
00500 }
00501 }
00502 #endif
00503 if ( ( info->flags & TOPOLOGIESONLY ) == 0 ) {
00504     info->numtopo++;
00505     return True;
00506 }
00507 //
00508 // Now we are just generating topologies.
00509 //
00510 GETIDENTITY
00511 WORD *term = info->term, *newterm, *oldworkpointer = AT.WorkPointer;
00512 WORD *tdia = term + info->diaoffset;
00513 WORD *tail = tdia + tdia[1];
00514 WORD *tend = term + *term;
00515 WORD *fill, *startfill;
00516 Model *model = (Model *)info->currentModel;
00517 MODEL *m = (MODEL *)info->currentMODEL;
00518 int i, j;
00519 int numlegs, vect, edge;
00520
00521 newterm = term + *term;
00522 for ( i = 1; i < info->diaoffset; i++ ) newterm[i] = term[i];
00523 fill = newterm + info->diaoffset;
00524 //
00525 // Now get the nodes
00526 //
00527 for ( i = 0; i < eg->nNodes; i++ ) {
00528 //
00529 //     node_(number,momentum_1,...,momentum_n)
00530 //
00531     numlegs = eg->nodes[i]->deg;
00532     startfill = fill;
00533     *fill++ = NODEFUNCTION;
00534     *fill++ = 0;
00535     FILLFUN(fill)
00536     *fill++ = -SNUMBER; *fill++ = i+1;
00537     if ( model != NULL && m != NULL ) {
00538         if ( !eg->isExternal(i) ) {
00539             WORD *afill = fill; *fill++ = 0; *fill++ = 1; FILLARG(fill)
00540             WORD *cfill = fill; *fill++ = 0;
00541             int cpl = 2*eg->nodes[i]->extloop+eg->nodes[i]->deg-2;
00542             *fill++ = SYMBOL; *fill++ = 4;
00543             *fill++ = m->couplings[0];
00544             *fill++ = cpl;
00545             *fill++ = 1; *fill++ = 1; *fill++ = 3;
00546             *cfill = fill - cfill;
00547             *afill = fill - afill;
00548             if ( *afill == ARGHEAD+8 && afill[ARGHEAD+4] == 1 ) {
00549                 fill = afill; *fill++ = -SYMBOL; *fill++ = afill[ARGHEAD+3];
00550             }
00551         }
00552         else {
00553             *fill++ = -SNUMBER;
00554             *fill++ = 1;
00555         }
00556     }
00557 //
00558 // Now the momenta.
00559 //
00560     for ( j = 0; j < numlegs; j++ ) {
00561         edge = eg->nodes[i]->edges[j];
00562         vect = ABS(edge)-1;
00563         *fill++ = edge < 0 ? -MINVECTOR: -VECTOR;
00564         if ( numlegs == 1 || vect < info->numextern ) { // Look up in set of external momenta
00565             *fill++ = SetElements[Sets[info->externalset].first+vect];
00566         }
00567         else { // Look up in set of internal momenta set
00568             *fill++ = SetElements[Sets[info->internalset].first+(vect-info->numextern)];
00569         }
00570     }
00571     startfill[1] = fill-startfill;
00572 }
00573 if ( ( info->flags & WITHEDGES ) == WITHEDGES ) {
00574     for ( i = 0; i < eg->nEdges; i++ ) {
00575         int n1 = eg->edges[i]->nodes[0];
00576         int n2 = eg->edges[i]->nodes[1];
00577         int l1 = eg->edges[i]->nlegs[0];

```

```

00578 //      int l2 = eg->edges[i]->nlegs[1];
00579      startfill = fill;
00580      *fill++ = EDGE;
00581      *fill++ = 0;
00582      FILLFUN(fill)
00583      *fill++ = -SNUMBER; *fill++ = i+1; // number of the edge
00584 //
00585      *fill++ = -VECTOR;
00586      if ( i < info->numextern ) { // Look up in set of external momenta
00587          *fill++ = SetElements[Sets[info->externalset].first+i];
00588      }
00589      else { // Look up in set of internal momenta set
00590          *fill++ = SetElements[Sets[info->internalset].first+(i-eg->nExtern)];
00591      }
00592 //
00593      *fill++ = -SNUMBER; *fill++ = n1+1; // number of the node from
00594      *fill++ = -SNUMBER; *fill++ = n2+1; // number of the node to
00595      startfill[1] = fill - startfill;
00596  }
00597 }
00598 //
00599 // Block information
00600 //
00601 if ( ( info->flags & WITHBLOCKS ) == WITHBLOCKS ) {
00602     for ( i = 0; i < eg->econn->nblocks; i++ ) {
00603         startfill = fill;
00604         *fill++ = BLOCK;
00605         *fill++ = 0;
00606         FILLFUN(fill);
00607         *fill++ = -SNUMBER;
00608         *fill++ = i+1;
00609         *fill++ = -SNUMBER;
00610         *fill++ = eg->econn->blocks[i].loop;
00611 //
00612 //      Now we have to make a list of all nodes inside this block
00613 //
00614         int bnodes[GRCC_MAXNODES], k;
00615         WORD *argfill = fill, *funfill;
00616         *fill++ = 0; *fill++ = 0; FILLARG(fill)
00617         *fill++ = 0;
00618         for ( k = 0; k < GRCC_MAXNODES; k++ ) bnodes[k] = 0;
00619         for ( k = 0; k < eg->econn->blocks[i].nmedges; k++ ) {
00620             bnodes[eg->econn->blocks[i].edges[k][0]] = 1;
00621             bnodes[eg->econn->blocks[i].edges[k][1]] = 1;
00622         }
00623         for ( k = 0; k < GRCC_MAXNODES; k++ ) {
00624             if ( bnodes[k] == 0 ) continue;
00625 //
00626 //      Now we put the node inside this argument.
00627 //
00628             funfill = fill;
00629             *fill++ = NODEFUNCTION;
00630             *fill++ = 0;
00631             FILLFUN(fill)
00632             *fill++ = -SNUMBER; *fill++ = k+1;
00633             numlegs = eg->nodes[k]->deg;
00634             for ( j = 0; j < numlegs; j++ ) {
00635                 edge = eg->nodes[k]->edges[j];
00636                 vect = ABS(edge)-1;
00637                 *fill++ = -VECTOR;
00638                 if ( numlegs == 1 || vect < info->numextern ) { // Look up in set of external
momenta
00639                     *fill++ = SetElements[Sets[info->externalset].first+vect];
00640                 }
00641                 else { // Look up in set of internal momenta set
00642                     *fill++ = SetElements[Sets[info->internalset].first+(vect-info->numextern)];
00643                 }
00644             }
00645             funfill[1] = fill-funfill;
00646         }
00647         *fill++ = 1; *fill++ = 1; *fill++ = 3;
00648         *argfill = fill - argfill;
00649         argfill[ARGHEAD] = argfill[0] - ARGHEAD;
00650         startfill[1] = fill-startfill;
00651     }
00652 }
00653 //
00654 // if ( eg->econn->narticuls > 0 ) {
00655 //     startfill = fill;
00656 //     *fill++ = BLOCK;
00657 //     *fill++ = 0;
00658 //     FILLFUN(fill);
00659 //     for ( i = 0; i < eg->econn->snodes; i++ ) {
00660 //         if ( eg->econn->articuls[i] != 0 ) {
00661 //             *fill++ = -SNUMBER;
00662 //             *fill++ = i+1;
00663 //         }
00664 //     }
00665 // }

```

```

00664 //      startfill[1] = fill-startfill;
00665 //      }
00666 //  }
00667 if ( ( info->flags & WITHONEPI ) == WITHONEPI ) {
00668     for ( i = 0; i < eg->econn->nopic; i++ ) {
00669         startfill = fill;
00670         *fill++ = ONEPI;
00671         *fill++ = 0;
00672         FILLFUN(fill);
00673         *fill++ = -SNUMBER;
00674         *fill++ = i+1;
00675         *fill++ = -ONEPI;
00676         for ( j = 0; j < eg->econn->opics[i].nnodes; j++ ) {
00677             *fill++ = -SNUMBER;
00678             *fill++ = eg->econn->opics[i].nodes[j]+1;
00679         }
00680         startfill[1] = fill-startfill;
00681     }
00682 }
00683 //
00684 // Topology counter. We have exaggerated a bit with the eye on the far future.
00685 //
00686 if ( info->numtopo < MAXPOSITIVE ) {
00687     *fill++ = TOPO; *fill++ = FUNHEAD+2; FILLFUN(fill)
00688     *fill++ = -SNUMBER; *fill++ = (WORD)(info->numtopo);
00689 }
00690 else if ( info->numtopo < FULLMAX-1 ) {
00691     *fill++ = TOPO; *fill++ = FUNHEAD+ARGHEAD+4; FILLFUN(fill)
00692     *fill++ = ARGHEAD+4; *fill++ = 0; FILLARG(fill)
00693     *fill++ = 4;
00694     *fill++ = (WORD)(info->numtopo & WORDMASK);
00695     *fill++ = 1; *fill++ = 3;
00696 }
00697 else { // for now: science fiction
00698     *fill++ = TOPO; *fill++ = FUNHEAD+ARGHEAD+6; FILLFUN(fill)
00699     *fill++ = ARGHEAD+6; *fill++ = 0; FILLARG(fill)
00700     *fill++ = 6; *fill++ = (WORD)(info->numtopo » BITSINWORD);
00701     *fill++ = (WORD)(info->numtopo & WORDMASK);
00702     *fill++ = 0; *fill++ = 1; *fill++ = 5;
00703 }
00704 //
00705 // Symmetry factors. We let Normalize do the multiplication.
00706 //
00707 if ( eg->nsym != 1 ) {
00708     *fill++ = SNUMBER; *fill++ = 4; *fill++ = (WORD)eg->nsym; *fill++ = -1;
00709 }
00710 if ( eg->esym != 1 ) {
00711     *fill++ = SNUMBER; *fill++ = 4; *fill++ = (WORD)eg->esym; *fill++ = -1;
00712 }
00713 //
00714 // finish it off
00715 //
00716 while ( tail < tend ) *fill++ = *tail++;
00717 if ( eg->fsign < 0 ) fill[-1] = -fill[-1];
00718 *newterm = fill - newterm;
00719 AT.WorkPointer = fill;
00720
00721 //MesPrint("<> %a",*newterm,newterm);
00722
00723 Generator(BHEAD newterm,info->level);
00724 AT.WorkPointer = oldworkpointer;
00725 info->numtopo++;
00726 return False;
00727 }
00728
00729 // #] ProcessTopology :
00730 // #[ GenDiagrams :
00731
00732 WORD GenDiagrams(PHEAD WORD *term, WORD level)
00733 {
00734     Model *model;
00735     MODEL *m;
00736     Options *opt;
00737     Process *proc;
00738     int pid = 1, x;
00739     int babble = 0; // Later we may set this at the FORM code level
00740     TERMINFO info;
00741     WORD inset,outset,*coupl,setnum,optionnumber = 0;
00742     int i, j, cpl[GRCC_MAXNCPLG];
00743     int ninitl, initlPart[GRCC_MAXLEGS], nfinal, finalPart[GRCC_MAXLEGS];
00744     for ( i = 0; i < GRCC_MAXNCPLG; i++ ) cpl[i] = 0;
00745     //
00746     // Here we create an object of type Option and load it up.
00747     // Next we run the diagram generation on it.
00748     //
00749     info.term = term;
00750     info.level = level;

```

```

00751     info.diaoffset = AR.funoffset;
00752     info.externalset = term[info.diaoffset+FUNHEAD+7];
00753     info.internalset = term[info.diaoffset+FUNHEAD+9];
00754     info.flags = 0;
00755     inset = term[info.diaoffset+FUNHEAD+3];
00756     outset = term[info.diaoffset+FUNHEAD+5];
00757     coupl = term + info.diaoffset + FUNHEAD + 10;
00758     if ( *coupl < 0 ) {
00759         if ( term[info.diaoffset+1] > FUNHEAD + 12 ) {
00760             optionnumber = term[info.diaoffset+FUNHEAD+13];
00761         }
00762     }
00763     else {
00764         if ( term[info.diaoffset+1] > *coupl+FUNHEAD+10 )
00765             optionnumber = term[info.diaoffset+*coupl+FUNHEAD+11];
00766     }
00767     setnum = term[info.diaoffset+FUNHEAD+1];
00768
00769     m = AC.models[SetElements[Sets[setnum].first]];
00770     LoadModel(m);
00771     model = (Model *)m->grccmodel;
00772
00773     info.currentModel = (void *)model;
00774     info.currentMODEL = (void *)m;
00775     info.numdia = 0;
00776     info.numtopo = 1;
00777     info.flags = optionnumber;
00778
00779     opt = new Options();
00780
00781     opt->setOutAG(ProcessDiagram, &info);
00782     opt->setOutMG(ProcessTopology, &info);
00783     // opt->setEndMG(fendMG, &info);
00784
00785     opt->values[GRCC_OPT_1PI] = ( optionnumber & ONEPARTICLEIRREDUCIBLE ) == ONEPARTICLEIRREDUCIBLE;
00786     opt->values[GRCC_OPT_NoTadpole] = ( optionnumber & NOTADPOLES ) == NOTADPOLES;
00787     //
00788     // Next are snails:
00789     //
00790     opt->values[GRCC_OPT_No1PtBlock] = ( optionnumber & NOTADPOLES ) == NOTADPOLES;
00791     //
00792     if ( ( optionnumber & WITHINSERTIONS ) == WITHINSERTIONS ) {
00793         opt->values[GRCC_OPT_No2PtL1PI] = True;
00794         opt->values[GRCC_OPT_NoAdj2PtV] = True;
00795         opt->values[GRCC_OPT_No2PtL1PI] = True;
00796     }
00797     else {
00798         opt->values[GRCC_OPT_NoAdj2PtV] = True;
00799     }
00800     opt->values[GRCC_OPT_SymmInitial] = ( optionnumber & WITHSYMMETRIZE ) == WITHSYMMETRIZE;
00801     opt->values[GRCC_OPT_SymmFinal] = ( optionnumber & WITHSYMMETRIZE ) == WITHSYMMETRIZE;
00802
00803     // opt->values[GRCC_OPT_Block] = ( optionnumber & WITHBLOCKS ) == WITHBLOCKS;
00804
00805     opt->setOutputF(False, "");
00806     opt->setOutputP(False, "");
00807     opt->printLevel(babble);
00808
00809     // opt->values[GRCC_OPT_Step] = GRCC_AGraph;
00810
00811     // Load the various arrays.
00812
00813     ninitl = Sets[inset].last - Sets[inset].first;
00814     for ( i = 0; i < ninitl; i++ ) {
00815         x = SetElements[Sets[inset].first+i];
00816         initlPart[i] = ConvertParticle(model,x);
00817         info.legcouple[i] = m->vertices[numParticle(m,x)]->couplings;
00818     }
00819     nfinal = Sets[outset].last - Sets[outset].first;
00820     for ( i = 0; i < nfinal; i++ ) {
00821         x = SetElements[Sets[outset].first+i];
00822         finalPart[i] = ConvertParticle(model,x);
00823         info.legcouple[i+ninitl] = m->vertices[numParticle(m,x)]->couplings;
00824     }
00825     info.numextern = ninitl + nfinal;
00826     for ( i = 2; i <= MAXLEGS; i++ ) {
00827         if ( m->legcouple[i] == 1 ) {
00828             for ( j = 0; j < info.numextern; j++ ) {
00829                 if ( info.legcouple[j][i] == 0 ) { info.flags |= CHECKEXTERN; goto Go_on; }
00830             }
00831         }
00832     }
00833     Go_on;;
00834     //
00835     // Now we have to sort out the coupling constants.
00836     // The argument at coupl can be of type -SNUMBER, -SYMBOL or generic
00837     // It has however already be tested for syntax.

```



```

00838 // Note that one cannot have 1 for the coupling constants.
00839 // In that case one should select 0 loops or something equivalent.
00840 //
00841 if ( *coupl == -SNUMBER ) { // Number of loops
00842 //
00843 // This is the complicated case.
00844 // We have to compute the number of coupling constants and then
00845 // generate diagrams for all combinations with the proper power.
00846 //
00847 int nc = coupl[1]*2 + ninitl + nfinal - 2;
00848 int *scratch = (int *)Malloca(nc*sizeof(int),"DistrN");
00849 scratch[0] = -2; // indicating startup cq first call.
00850
00851 if ( ( info.flags & TOPOLOGIESONLY ) == 0 ) {
00852 while ( DistrN(nc,cpl,m->ncouplings,scratch) ) {
00853 proc = new Process(pid, model, opt,
00854 ninitl, initlPart, nfinal, finalPart, cpl);
00855 delete proc;
00856 info.numtopo = 1;
00857 }
00858 }
00859 else {
00860 cpl[0] = nc;
00861 proc = new Process(pid, model, opt,
00862 ninitl, initlPart, nfinal, finalPart, cpl);
00863 delete proc;
00864 }
00865 M_free(scratch,"DistrN");
00866 opt->end();
00867 delete opt;
00868 return(0);
00869 }
00870 else if ( *coupl == -SYMBOL ) { // Just a single power of one constant
00871 for ( i = 0; i < m->ncouplings; i++ ) {
00872 if ( m->couplings[i] == coupl[1] ) {
00873 cpl[i] = 1;
00874 break;
00875 }
00876 }
00877 }
00878 else { // One term with powers of coupling constants
00879 WORD *t, *tstop;
00880 t = coupl + ARGHEAD+3;
00881 tstop = coupl+coupl; tstop -= ABS(tstop[-1]);
00882 while ( t < tstop ) {
00883 for ( i = 0; i < m->ncouplings; i++ ) {
00884 if ( m->couplings[i] == *t ) {
00885 cpl[i] = t[1];
00886 break;
00887 }
00888 }
00889 t += 2;
00890 }
00891 }
00892 /*
00893 And now the generation:
00894 */
00895 proc = new Process(pid, model, opt,
00896 ninitl, initlPart, nfinal, finalPart, cpl);
00897 opt->end();
00898 delete proc;
00899 delete opt;
00900 return(0);
00901 }
00902
00903 // #] GenDiagrams :
00904 // #[ processVertex :
00905
00906 // Routine is to be used recursively to work its way through a list
00907 // of possible vertices. The array of vertices is in TopoInf->vert
00908 // with TopoInf->nvert the number of possible vertices.
00909 // Currently we allow in TopoInf->vert only vertices with 3 or more edges.
00910 //
00911 // We work with a point system. Each n-point vertex contributes n-2 points.
00912 // When all points are assigned, we can call mgraph->generate().
00913 //
00914 // The number of vertices of a given number of edges is stored in
00915 // TopoInf->cnum[...] but the loop that determines how many there are
00916 // may be limited by the corresponding element in TopoInf->vertmax[level]
00917
00918 int processVertex(TOPOTYPE *TopoInf, int pointsremaining, int level)
00919 {
00920 int i, j;
00921
00922 for ( i = pointsremaining, j = 0; i >= 0; i -= TopoInf->vert[level]-2, j++ ) {
00923 if ( TopoInf->vertmax && TopoInf->vertmax[level] >= 0
00924 && j > TopoInf->vertmax[level] ) break;

```

```

00925         if ( i == 0 ) { // We got one!
00926             TopoInf->cldeg[TopoInf->ncl] = TopoInf->vert[level];
00927             TopoInf->clnum[TopoInf->ncl] = j;
00928             TopoInf->clxt[TopoInf->ncl] = 0;
00929             TopoInf->ncl++;
00930
00931             MGraph *mgraph = new MGraph(1, TopoInf->ncl, TopoInf->cldeg,
00932                                         TopoInf->clnum, TopoInf->clxt,
00933                                         TopoInf->cmind, TopoInf->cmaxd, TopoInf->opt);
00934
00935             mgraph->generate();
00936
00937             delete mgraph;
00938
00939             TopoInf->ncl--;
00940
00941             break;
00942         }
00943         if ( level < TopoInf->nvert-1 ) {
00944             if ( j > 0 ) {
00945                 TopoInf->cldeg[TopoInf->ncl] = TopoInf->vert[level];
00946                 TopoInf->clnum[TopoInf->ncl] = j;
00947                 TopoInf->clxt[TopoInf->ncl] = 0;
00948                 TopoInf->ncl++;
00949             }
00950             if ( processVertex(TopoInf,i,level+1) < 0 ) return(-1);
00951             if ( j > 0 ) { TopoInf->ncl--; }
00952         }
00953     }
00954     return(0);
00955 }
00956
00957 // #] processVertex :
00958 // #[ GenTopologies :
00959
00960 #define TOPO_MAXVERT 10
00961
00962 WORD GenTopologies(PHEAD WORD *term, WORD level)
00963 {
00964     Options *opt = new Options;
00965     int nlegs, nloops, i, identical;
00966     TERMINFO info;
00967     WORD *t, *t1, *tstop;
00968     TOPOTYPE TopoInf;
00969     SETS s;
00970 //
00971     info.term = term;
00972     info.level = level;
00973     info.diaoffset = AR.funoffset;
00974     info.flags = 0;
00975
00976     t = term + info.diaoffset; // the function
00977     t1 = t + FUNHEAD;          // its arguments
00978     tstop = t + t[1];
00979
00980     info.externalset = t1[7];
00981     info.internalset = t1[9];
00982
00983     s = &(Sets[t1[5]]);
00984     TopoInf.nvert = s->last - s->first;
00985     TopoInf.vert = &(SetElements[s->first]);
00986
00987     nloops = t1[1];
00988     nlegs = t1[3];
00989
00990     info.numextern = nlegs;
00991
00992     for ( i = 0; i <= MAXLEGS; i++ ) { TopoInf.cmind[i] = TopoInf.cmaxd[i] = 0; }
00993
00994     t1 += 10;
00995     if ( t1 < tstop && t1[0] == -SETSET ) {
00996         TopoInf.vertmax = &(SetElements[Sets[t1[1]].first]);
00997         t1 += 2;
00998     }
00999     else TopoInf.vertmax = NULL;
01000
01001     info.flags |= TOPOLOGIESONLY; // this is the topologies_ function after all.
01002     if ( t1 < tstop && t1[0] == -SNUMBER ) {
01003         if ( ( t1[1] & NONODES ) == NONODES ) info.flags |= NONODES;
01004         if ( ( t1[1] & WITHEDGES ) == WITHEDGES ) info.flags |= WITHEDGES;
01005         if ( ( t1[1] & WITHBLOCKS ) == WITHBLOCKS ) info.flags |= WITHBLOCKS;
01006         if ( ( t1[1] & WITHONEPI ) == WITHONEPI ) info.flags |= WITHONEPI;
01007         opt->values[GRCC_OPT_1PI] = ( t1[1] & ONEPARTICLEIRREDUCIBLE ) == ONEPARTICLEIRREDUCIBLE;
01008         // opt->values[GRCC_OPT_NoTadpole] = ( t1[1] & NOTADPOLES ) == NOTADPOLES;
01009         opt->values[GRCC_OPT_NoTadpole] = ( t1[1] & NOSNAILS ) == NOSNAILS;
01010         opt->values[GRCC_OPT_No1PtBlock] = ( t1[1] & NOTADPOLES ) == NOTADPOLES;
01011         opt->values[GRCC_OPT_NoExtSelf] = ( t1[1] & NOEXTSELF ) == NOEXTSELF;

```

```

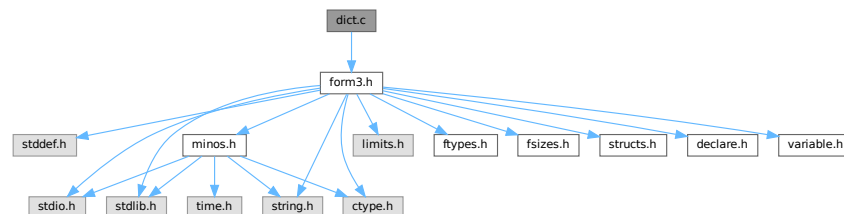
01012
01013     if ( ( t1[1] & WITHINSECTIONS ) == WITHINSECTIONS ) {
01014         opt->values[GRCC_OPT_No2PtL1PI] = True;
01015         opt->values[GRCC_OPT_NoAdj2PtV] = True;
01016         opt->values[GRCC_OPT_No2PtL1PI] = True;
01017     }
01018     opt->values[GRCC_OPT_SymmInitial] = ( t1[1] & WITHSYMMETRIZE ) == WITHSYMMETRIZE;
01019 }
01020
01021 info.numdia = 0;
01022 info.numtopo = 1;
01023
01024 opt->setOutAG(ProcessDiagram, &info);
01025 opt->setOutMG(ProcessTopology, &info);
01026 //
01027 // Now we should sum over all possible vertices and run MGraph for
01028 // each combination. This is done by recursion in the processVertex routine
01029 // First load up the relevant arrays.
01030 //
01031
01032 // First the external nodes.
01033
01034 if ( nlegs == -2 ) {
01035     nlegs = 2;
01036     identical = 1;
01037 }
01038 for ( i = 0; i < nlegs; i++ ) {
01039     TopoInf.cldeg[i] = 1; TopoInf.clnum[i] = 1; TopoInf.clex[i] = -1;
01040 }
01041 int points = 2*nloops-2+nlegs;
01042
01043 if ( identical == 1 ) { /* Only propagator topologies..... */
01044     nlegs = 1;
01045     TopoInf.clnum[0] = 2;
01046 }
01047 TopoInf.ncl = nlegs;
01048 TopoInf.opt = opt;
01049
01050 if ( points >= MAXPOINTS ) {
01051     MLOCK(ErrorMessageLock);
01052     MesPrint("GenTopologies: %d loops and %d legs considered excessive",nloops,nlegs);
01053     MUNLOCK(ErrorMessageLock);
01054     Terminate(-1);
01055 }
01056 if ( processVertex(&TopoInf,points,0) != 0 ) {
01057     MLOCK(ErrorMessageLock);
01058     MesPrint("Called from GenTopologies with %d loops and %d legs",nloops,nlegs);
01059     MUNLOCK(ErrorMessageLock);
01060     Terminate(-1);
01061 }
01062 delete opt;
01063 return(0);
01064 }
01065
01066 // #] GenTopologies :
01067

```

4.24 dict.c File Reference

```
#include "form3.h"
```

Include dependency graph for dict.c:



Functions

- void [TransformRational](#) (UWORD *a, WORD na)
- UBYTE * [IsMultiplySign](#) (void)
- UBYTE * [IsExponentSign](#) (void)
- UBYTE * [FindSymbol](#) (WORD num)
- UBYTE * [FindVector](#) (WORD num)
- UBYTE * [FindIndex](#) (WORD num)
- UBYTE * [FindFunction](#) (WORD num)
- UBYTE * [FindFunWithArgs](#) (WORD *t)
- UBYTE * [FindExtraSymbol](#) (WORD num)
- int [FindDictionary](#) (UBYTE *name)
- int [AddDictionary](#) (UBYTE *name)
- int [AddToDictionary](#) (DICTIONARY *dict, UBYTE *left, UBYTE *right)
- int [UseDictionary](#) (UBYTE *name, UBYTE *options)
- int [SetDictionaryOptions](#) (UBYTE *options)
- void [UnSetDictionary](#) (void)
- void [RemoveDictionary](#) (DICTIONARY *dict)
- void [ShrinkDictionary](#) (DICTIONARY *dict)
- int [DoPreOpenDictionary](#) (UBYTE *s)
- int [DoPreCloseDictionary](#) (UBYTE *s)
- int [DoPreUseDictionary](#) (UBYTE *s)
- int [DoPreAdd](#) (UBYTE *s)
- LONG [DictToBytes](#) (DICTIONARY *dict, UBYTE *buf)
- DICTIONARY * [DictFromBytes](#) (UBYTE *buf)

4.24.1 Detailed Description

Contains the code pertaining to dictionaries Commands are: #opendictionary name #closedictionary #selectdictionary name <options> There can be several dictionaries, but only one can be active. Defining elements is done with #add object: "replacement" Replacements are strings when a dictionary is for output translation. Objects can be 1: a number (rational) 2: a variable 3: * ^ 4: a function with arguments

Definition in file [dict.c](#).

4.24.2 Function Documentation

4.24.2.1 TransformRational()

```
void TransformRational (
    UWORD * a,
    WORD na )
```

Definition at line 71 of file [dict.c](#).

4.24.2.2 IsMultiplySign()

```
UBYTE * IsMultiplySign (
    void )
```

Definition at line 218 of file [dict.c](#).

4.24.2.3 IsExponentSign()

```
UBYTE * IsExponentSign (  
    void )
```

Definition at line 238 of file [dict.c](#).

4.24.2.4 FindSymbol()

```
UBYTE * FindSymbol (  
    WORD num )
```

Definition at line 258 of file [dict.c](#).

4.24.2.5 FindVector()

```
UBYTE * FindVector (  
    WORD num )
```

Definition at line 279 of file [dict.c](#).

4.24.2.6 FindIndex()

```
UBYTE * FindIndex (  
    WORD num )
```

Definition at line 301 of file [dict.c](#).

4.24.2.7 FindFunction()

```
UBYTE * FindFunction (  
    WORD num )
```

Definition at line 323 of file [dict.c](#).

4.24.2.8 FindFunWithArgs()

```
UBYTE * FindFunWithArgs (  
    WORD * t )
```

Definition at line 345 of file [dict.c](#).

4.24.2.9 FindExtraSymbol()

```
UBYTE * FindExtraSymbol (  
    WORD num )
```

Definition at line 377 of file [dict.c](#).

4.24.2.10 FindDictionary()

```
int FindDictionary (
    UBYTE * name )
```

Definition at line [434](#) of file [dict.c](#).

4.24.2.11 AddDictionary()

```
int AddDictionary (
    UBYTE * name )
```

Definition at line [449](#) of file [dict.c](#).

4.24.2.12 AddToDictionary()

```
int AddToDictionary (
    DICTIONARY * dict,
    UBYTE * left,
    UBYTE * right )
```

Definition at line [491](#) of file [dict.c](#).

4.24.2.13 UseDictionary()

```
int UseDictionary (
    UBYTE * name,
    UBYTE * options )
```

Definition at line [766](#) of file [dict.c](#).

4.24.2.14 SetDictionaryOptions()

```
int SetDictionaryOptions (
    UBYTE * options )
```

Definition at line [790](#) of file [dict.c](#).

4.24.2.15 UnSetDictionary()

```
void UnSetDictionary (
    void )
```

Definition at line [883](#) of file [dict.c](#).

4.24.2.16 RemoveDictionary()

```
void RemoveDictionary (
    DICTIONARY * dict )
```

Definition at line 902 of file [dict.c](#).

4.24.2.17 ShrinkDictionary()

```
void ShrinkDictionary (
    DICTIONARY * dict )
```

Definition at line 945 of file [dict.c](#).

4.24.2.18 DoPreOpenDictionary()

```
int DoPreOpenDictionary (
    UBYTE * s )
```

Definition at line 959 of file [dict.c](#).

4.24.2.19 DoPreCloseDictionary()

```
int DoPreCloseDictionary (
    UBYTE * s )
```

Definition at line 998 of file [dict.c](#).

4.24.2.20 DoPreUseDictionary()

```
int DoPreUseDictionary (
    UBYTE * s )
```

Definition at line 1022 of file [dict.c](#).

4.24.2.21 DoPreAdd()

```
int DoPreAdd (
    UBYTE * s )
```

Definition at line 1067 of file [dict.c](#).

4.24.2.22 DictToBytes()

```
LONG DictToBytes (
    DICTIONARY * dict,
    UBYTE * buf )
```

Definition at line 1119 of file [dict.c](#).

4.24.2.23 DictFromBytes()

```
DICTIONARY * DictFromBytes (
    UBYTE * buf )
```

Definition at line 1152 of file [dict.c](#).

4.25 dict.c

[Go to the documentation of this file.](#)

```
00001
00018 /* #[ License : */
00019 /*
00020 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00021 * When using this file you are requested to refer to the publication
00022 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00023 * This is considered a matter of courtesy as the development was paid
00024 * for by FOM the Dutch physics granting agency and we would like to
00025 * be able to track its scientific use to convince FOM of its value
00026 * for the community.
00027 *
00028 * This file is part of FORM.
00029 *
00030 * FORM is free software: you can redistribute it and/or modify it under the
00031 * terms of the GNU General Public License as published by the Free Software
00032 * Foundation, either version 3 of the License, or (at your option) any later
00033 * version.
00034 *
00035 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00036 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00037 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00038 * details.
00039 *
00040 * You should have received a copy of the GNU General Public License along
00041 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00042 */
00043 /* #] License : */
00044 /*
00045     #[ Includes : ratio.c
00046
00047     Data setup:
00048         AO.Dictionaries          Array of pointers to DICTIONARY
00049         AO.NumDictionaries
00050         AO.SizeDictionaries
00051         AO.CurrentDictionary
00052         AO.CurDictNumbers
00053         AO.CurDictVariables
00054         AO.CurDictSpecials
00055         AP.OpenDictionary
00056 */
00057
00058 #include "form3.h"
00059
00060 /*
00061     #] Includes :
00062     #[ TransformRational:
00063
00064     Tries to transform the rational number a according to the rules of
00065     the current dictionary. Whatever cannot be translated goes to the
00066     regular output.
00067     Options for AO.CurDictNumbers are:
00068         DICT_ALLNUMBERS, DICT_RATIONALONLY, DICT_INTEGERONLY, DICT_NONUMBERS
00069 */
00070
00071 void TransformRational(UWORD *a, WORD na)
00072 {
00073     DICTIONARY *dict;
00074     WORD i, j, nb, i1, i2; UWORD *b;
00075     if ( AO.CurrentDictionary <= 0 ) goto NoAction;
00076     dict = AO.Dictionaries[AO.CurrentDictionary-1];
00077     if ( na < 0 ) na = -na;
00078     switch ( AO.CurDictNumbers ) {
00079     case DICT_NONUMBERS:
00080         goto NoAction;
00081     case DICT_INTEGERONLY:
00082         if ( a[na] != 1 ) goto NoAction;
00083         if ( na > 1 ) {
00084             for ( i = 1; i < na; i++ ) {
```



```

00085         if ( a[na+i] != 0 ) goto NoAction;
00086     }
00087 }
00088 Numeratoronly:;
00089     for ( i = dict->numelements-1; i >= 0; i-- ) {
00090         if ( dict->elements[i]->type == DICT_INTEGERNUMBER ) {
00091             if ( dict->elements[i]->size == na ) {
00092                 for ( j = 0; j < na; j++ ) {
00093                     if ( (UWORD)(dict->elements[i]->lhs[j]) != a[j] ) break;
00094                 }
00095                 if ( j == na ) { /* Got it */
00096                     TokenToLine((UBYTE *) (dict->elements[i]->rhs));
00097                     return;
00098                 }
00099             }
00100         }
00101     }
00102     goto NotFound;
00103 case DICT_RATIONALONLY:
00104     nb = 2*na;
00105     for ( i = dict->numelements-1; i >= 0; i-- ) {
00106         if ( dict->elements[i]->type == DICT_RATIONALNUMBER ) {
00107             if ( dict->elements[i]->size == nb+2 ) {
00108                 for ( j = 0; j < nb; j++ ) {
00109                     if ( (UWORD)(dict->elements[i]->lhs[j+1]) != a[j] ) break;
00110                 }
00111                 if ( j == nb ) { /* Got it */
00112                     TokenToLine((UBYTE *) (dict->elements[i]->rhs));
00113                     return;
00114                 }
00115             }
00116         }
00117     }
00118     goto NotFound;
00119 case DICT_ALLNUMBERS:
00120 /*
00121     First fish for rationals
00122 */
00123     nb = 2*na;
00124     for ( i = dict->numelements-1; i >= 0; i-- ) {
00125         if ( dict->elements[i]->type == DICT_RATIONALNUMBER ) {
00126             if ( dict->elements[i]->size == nb+2 ) {
00127                 for ( j = 0; j < nb; j++ ) {
00128                     if ( (UWORD)(dict->elements[i]->lhs[j+1]) != a[j] ) break;
00129                 }
00130                 if ( j == nb ) { /* Got it */
00131                     TokenToLine((UBYTE *) (dict->elements[i]->rhs));
00132                     return;
00133                 }
00134             }
00135         }
00136     }
00137 /*
00138     Now look for element[j1]/element[j2]
00139 */
00140     nb = na; b = a+na;
00141     while ( b[nb-1] == 0 ) nb--;
00142     if ( nb == 1 && b[0] == 1 ) goto Numeratoronly;
00143     while ( a[na-1] == 0 ) na--;
00144     for ( i1 = dict->numelements-1; i1 >= 0; i1-- ) {
00145         if ( dict->elements[i1]->type == DICT_INTEGERNUMBER ) {
00146             if ( dict->elements[i1]->size == na ) {
00147                 for ( j = 0; j < na; j++ ) {
00148                     if ( (UWORD)(dict->elements[i1]->lhs[j]) != a[j] ) break;
00149                 }
00150                 if ( j == na ) break;
00151             }
00152         }
00153     }
00154     for ( i2 = dict->numelements-1; i2 >= 0; i2-- ) {
00155         if ( dict->elements[i2]->type == DICT_INTEGERNUMBER ) {
00156             if ( dict->elements[i2]->size == nb ) {
00157                 for ( j = 0; j < nb; j++ ) {
00158                     if ( (UWORD)(dict->elements[i2]->lhs[j]) != b[j] ) break;
00159                 }
00160                 if ( j == nb ) break;
00161             }
00162         }
00163     }
00164     if ( i1 < 0 ) {
00165         if ( i2 < 0 ) goto NotFound;
00166         else { /* number/replacement[i2] */
00167             LongToLine(a,na);
00168             if ( na > 1 || ( AO.DoubleFlag & 4 ) == 4 ) {
00169                 if ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE
00170                     || AC.OutputMode == CMODE ) {
00171                     if ( ( AO.DoubleFlag & 2 ) == 2 ) { AddToLine((UBYTE *) ".Q0/"); }

```

```

00172                 else if ( ( AO.DoubleFlag & 1 ) == 1 ) { AddToLine((UBYTE *)".D0/"); }
00173                 else { AddToLine((UBYTE *)"/"); }
00174             }
00175         }
00176         else AddToLine((UBYTE *)("/"));
00177         TokenToLine((UBYTE *) (dict->elements[i2]->rhs));
00178     }
00179 }
00180 else if ( i2 < 0 ) { /* replacement[i1]/number */
00181     TokenToLine((UBYTE *) (dict->elements[i1]->rhs));
00182     AddToLine((UBYTE *) ("/"));
00183     LongToLine((UWORD *) (b),nb);
00184     if ( nb > 1 || ( AO.DoubleFlag & 4 ) == 4 ) {
00185         if ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE
00186             || AC.OutputMode == CMODE ) {
00187             if ( ( AO.DoubleFlag & 2 ) == 2 ) { AddToLine((UBYTE *)".Q0"); }
00188             else if ( ( AO.DoubleFlag & 1 ) == 1 ) { AddToLine((UBYTE *)".D0"); }
00189         }
00190     }
00191 }
00192 else { /* replacement[i1]/replacement[i2] */
00193     TokenToLine((UBYTE *) (dict->elements[i1]->rhs));
00194     AddToLine((UBYTE *) ("/"));
00195     TokenToLine((UBYTE *) (dict->elements[i2]->rhs));
00196 }
00197 break;
00198 default:
00199     MesPrint("Illegal code in TransformRational: %d",AO.CurDictNumbers);
00200     Terminate(-1);
00201 }
00202 return;
00203 NotFound:
00204     if ( na != 1 || a[1] != 1 ) {
00205         if ( AO.CurDictNumberWarning ) {
00206             MesPrint("»»»»»Could not translate coefficient with dictionary %s«««««",dict->name);
00207         }
00208     }
00209 NoAction:
00210     RatToLine(a,na);
00211     return;
00212 }
00213 /*
00214 #] TransformRational:
00215 #[ IsMultiplySign:
00216 */
00217
00218 UBYTE *IsMultiplySign(void)
00219 {
00220     DICTIONARY *dict;
00221     int i;
00222     if ( AO.CurrentDictionary <= 0 ) return(0);
00223     dict = AO.Dictionaries[AO.CurrentDictionary-1];
00224     if ( dict->characters == 0 ) return(0);
00225     for ( i = dict->numelements-1; i >= 0; i-- ) {
00226         if ( ( dict->elements[i]->type == DICT_SPECIALCHARACTER )
00227             && ( dict->elements[i]->lhs[0] == (WORD)('^*') ) )
00228             return((UBYTE *) (dict->elements[i]->rhs));
00229     }
00230     return(0);
00231 }
00232
00233 /*
00234 #] IsMultiplySign:
00235 #[ IsExponentSign:
00236 */
00237
00238 UBYTE *IsExponentSign(void)
00239 {
00240     DICTIONARY *dict;
00241     int i;
00242     if ( AO.CurrentDictionary <= 0 ) return(0);
00243     dict = AO.Dictionaries[AO.CurrentDictionary-1];
00244     if ( dict->characters == 0 ) return(0);
00245     for ( i = dict->numelements-1; i >= 0; i-- ) {
00246         if ( ( dict->elements[i]->type == DICT_SPECIALCHARACTER )
00247             && ( dict->elements[i]->lhs[0] == (WORD)('^') ) )
00248             return((UBYTE *) (dict->elements[i]->rhs));
00249     }
00250     return(0);
00251 }
00252
00253 /*
00254 #] IsExponentSign:
00255 #[ FindSymbol :
00256 */
00257
00258 UBYTE *FindSymbol(WORD num)

```

```

00259 {
00260     if ( AO.CurrentDictionary > 0 ) {
00261         DICTIONARY *dict = AO.Dictionaries[AO.CurrentDictionary-1];
00262         int i;
00263         if ( dict->variables > 0 && AO.CurDictVariables == DICT_DOVARIABLES ) {
00264             for ( i = dict->numelements-1; i >= 0; i-- ) {
00265                 if ( dict->elements[i]->type == DICT_SYMBOL &&
00266                     dict->elements[i]->lhs[0] == num )
00267                     return((UBYTE *) (dict->elements[i]->rhs));
00268             }
00269         }
00270     }
00271     return(VARNAME(symbols,num));
00272 }
00273
00274 /*
00275 #] FindSymbol :
00276 #[ FindVector :
00277 */
00278
00279 UBYTE *FindVector(WORD num)
00280 {
00281     if ( AO.CurrentDictionary > 0 ) {
00282         DICTIONARY *dict = AO.Dictionaries[AO.CurrentDictionary-1];
00283         int i;
00284         if ( dict->variables > 0 && AO.CurDictVariables == DICT_DOVARIABLES ) {
00285             for ( i = dict->numelements-1; i >= 0; i-- ) {
00286                 if ( dict->elements[i]->type == DICT_VECTOR &&
00287                     dict->elements[i]->lhs[0] == num )
00288                     return((UBYTE *) (dict->elements[i]->rhs));
00289             }
00290         }
00291     }
00292     num -= AM.OffsetVector;
00293     return(VARNAME(vectors,num));
00294 }
00295
00296 /*
00297 #] FindVector :
00298 #[ FindIndex :
00299 */
00300
00301 UBYTE *FindIndex(WORD num)
00302 {
00303     if ( AO.CurrentDictionary > 0 ) {
00304         DICTIONARY *dict = AO.Dictionaries[AO.CurrentDictionary-1];
00305         int i;
00306         if ( dict->variables > 0 && AO.CurDictVariables == DICT_DOVARIABLES ) {
00307             for ( i = dict->numelements-1; i >= 0; i-- ) {
00308                 if ( dict->elements[i]->type == DICT_INDEX &&
00309                     dict->elements[i]->lhs[0] == num )
00310                     return((UBYTE *) (dict->elements[i]->rhs));
00311             }
00312         }
00313     }
00314     num -= AM.OffsetIndex;
00315     return(VARNAME(indices,num));
00316 }
00317
00318 /*
00319 #] FindIndex :
00320 #[ FindFunction :
00321 */
00322
00323 UBYTE *FindFunction(WORD num)
00324 {
00325     if ( AO.CurrentDictionary > 0 ) {
00326         DICTIONARY *dict = AO.Dictionaries[AO.CurrentDictionary-1];
00327         int i;
00328         if ( dict->variables > 0 && AO.CurDictVariables == DICT_DOVARIABLES ) {
00329             for ( i = dict->numelements-1; i >= 0; i-- ) {
00330                 if ( dict->elements[i]->type == DICT_FUNCTION &&
00331                     dict->elements[i]->lhs[0] == num )
00332                     return((UBYTE *) (dict->elements[i]->rhs));
00333             }
00334         }
00335     }
00336     num -= FUNCTION;
00337     return(VARNAME(functions,num));
00338 }
00339
00340 /*
00341 #] FindFunction :
00342 #[ FindFunWithArgs :
00343 */
00344
00345 UBYTE *FindFunWithArgs(WORD *t)

```

```

00346 {
00347     if ( AO.CurrentDictionary > 0 ) {
00348         DICTIONARY *dict = AO.Dictionaries[AO.CurrentDictionary-1];
00349         int i, j;
00350         if ( dict->funwith > 0
00351             && AO.CurDictFunWithArgs == DICT_DOFUNWITHARGS ) {
00352             for ( i = dict->numelements-1; i >= 0; i-- ) {
00353                 if ( dict->elements[i]->type == DICT_FUNCTION_WITH_ARGUMENTS &&
00354                     (WORD)(dict->elements[i]->lhs[0]) == t[0] &&
00355                     (WORD)(dict->elements[i]->lhs[1]) == t[1] ) {
00356                     for ( j = 2; j < t[1]; j++ ) {
00357                         if ( (WORD)(dict->elements[i]->lhs[j]) != t[j] ) break;
00358                     }
00359                     if ( j >= t[1] ) return((UBYTE *) (dict->elements[i]->rhs));
00360                 }
00361             }
00362         }
00363     }
00364     return(0);
00365 }
00366
00367 /*
00368 #] FindFunWithArgs :
00369 #[ FindExtraSymbol :
00370
00371     The extra symbol is constructed in the WorkSpace. This way we do not
00372     have to worry about Malloc and freeing the object later.
00373     The input value num is already the number of the extra symbol.
00374     We do NOT need num = MAXVARIABLES-num;
00375 */
00376
00377 UBYTE *FindExtraSymbol(WORD num)
00378 {
00379     GETIDENTITY;
00380     UBYTE *out = (UBYTE *) (AT.WorkPointer);
00381     *out = 0;
00382     if ( AO.CurrentDictionary > 0 ) {
00383         DICTIONARY *dict = AO.Dictionaries[AO.CurrentDictionary-1];
00384         int i;
00385         if ( dict->ranges > 0 && AO.CurDictVariables == DICT_DOVARIABLES ) {
00386             for ( i = dict->numelements-1; i >= 0; i-- ) {
00387                 if ( dict->elements[i]->type == DICT_RANGE
00388                     && num >= dict->elements[i]->lhs[0]
00389                     && num <= dict->elements[i]->lhs[1] ) {
00390
00391                     /*
00392                     Now we have to translate the rhs
00393                     %# gives the number
00394                     %@ gives the number as its position in the range
00395                     */
00396
00397                     UBYTE *r = (UBYTE *) (dict->elements[i]->rhs);
00398                     while ( *r ) {
00399                         if ( *r == (UBYTE)'%' && ( r[1] == (UBYTE)'#'
00400                             || r[1] == (UBYTE)'@' ) ) {
00401                             if ( r[1] == (UBYTE)'#' ) {
00402                                 out = NumCopy(num,out);
00403                             }
00404                             else {
00405                                 out = NumCopy(num-dict->elements[i]->lhs[0]+1,out);
00406                             }
00407                             r += 2;
00408                         }
00409                         else {
00410                             *out++ = *r++;
00411                         }
00412                     }
00413                     *out = 0;
00414                     return((UBYTE *) (AT.WorkPointer));
00415                 }
00416             }
00417         }
00418         out = StrCopy((UBYTE *) AC.extrasym,out);
00419         if ( AC.extrasymbols == 0 ) {
00420             out = NumCopy(num,out);
00421             out = StrCopy((UBYTE *) "_",out);
00422         }
00423         else if ( AC.extrasymbols == 1 ) {
00424             out = AddArrayIndex(num,out);
00425         }
00426         return((UBYTE *) (AT.WorkPointer));
00427     }
00428
00429 /*
00430 #] FindExtraSymbol :
00431 #[ FindDictionary :
00432 */

```

```

00433
00434 int FindDictionary(UBYTE *name)
00435 {
00436     int i;
00437     for ( i = 0; i < AO.NumDictionaries; i++ ) {
00438         if ( StrCmp(AO.Dictionaries[i]->name,name) == 0 )
00439             return(i+1);
00440     }
00441     return(0);
00442 }
00443
00444 /*
00445 #] FindDictionary :
00446 #[ AddDictionary :
00447 */
00448
00449 int AddDictionary(UBYTE *name)
00450 {
00451     DICTIONARY *dict;
00452     /*
00453      First make space for the pointer in the list.
00454     */
00455     if ( AO.NumDictionaries >= AO.SizeDictionaries-1 ) {
00456         DICTIONARY **d;
00457         int i;
00458         if ( AO.SizeDictionaries <= 0 ) AO.SizeDictionaries = 10;
00459         else AO.SizeDictionaries = 2*AO.SizeDictionaries;
00460         d = (DICTIONARY **)Malloc1(AO.SizeDictionaries*sizeof(DICTIONARY *),"Dictionaries");
00461         for ( i = 0; i < AO.NumDictionaries; i++ ) d[i] = AO.Dictionaries[i];
00462         if ( AO.Dictionaries != 0 ) M_free(AO.Dictionaries,"Dictionaries");
00463         AO.Dictionaries = d;
00464     }
00465     /*
00466      Now create an empty dictionary.
00467     */
00468     dict = (DICTIONARY *)Malloc1(sizeof(DICTIONARY),"Dictionary");
00469     AO.Dictionaries[AO.NumDictionaries++] = dict;
00470     dict->elements = 0;
00471     dict->name = strDup1(name,"DictionaryName");
00472     dict->sizeelements = 0;
00473     dict->numelements = 0;
00474     dict->numbers = 0;
00475     dict->variables = 0;
00476     dict->characters = 0;
00477     dict->funwith = 0;
00478     dict->gnumelements = 0;
00479     dict->ranges = 0;
00480
00481     return(AO.NumDictionaries);
00482 }
00483
00484 /*
00485 #] AddDictionary :
00486 #[ AddToDictionary :
00487
00488     To be called from #add left:right
00489 */
00490
00491 int AddToDictionary(DICTIONARY *dict,UBYTE *left,UBYTE *right)
00492 {
00493     GETIDENTITY
00494     CBUF *C = cbuf+AC.cbunum;
00495     WORD *w = AT.WorkPointer;
00496     WORD *OldWork = AT.WorkPointer;
00497     WORD *s, oldnumrhs = C->numrhs, oldnumlhs = C->numlhs;
00498     WORD *ow, *ww, *mm, oldEside, *where = 0, type, number, range[3];
00499     LONG oldcpointer;
00500     int error = 0, sizelhs, sizerhs, i, retcode;
00501     UBYTE *r;
00502     DICTIONARY_ELEMENT *new;
00503     WORD power = (WORD)('^'), times = (WORD)('*');
00504     if ( ( left[0] == '^' && left[1] == 0 )
00505         || ( left[0] == '*' && left[1] == '*' && left[2] == 0 ) ) {
00506         type = DICT_SPECIALCHARACTER;
00507         number = 1;
00508         where = &power;
00509         goto TestDouble;
00510     }
00511     else if ( left[0] == '*' && left[1] == 0 ) {
00512         type = DICT_SPECIALCHARACTER;
00513         number = 1;
00514         where = &times;
00515         goto TestDouble;
00516     }
00517     else if ( left[0] == '(' ) { /* range of extra symbols */
00518         WORD x1 = 0, x2 = 0;
00519         r = left+1;

```

```

00520         while ( FG.cTable[*r] == 1 ) x1 = 10*x1 + *r++ - '0';
00521         if ( *r == ',' ) {
00522             r++;
00523             while ( FG.cTable[*r] == 1 ) x2 = 10*x2 + *r++ - '0';
00524         }
00525         else x2 = x1;
00526         number = 2;
00527         if ( *r != ')' ) {
00528             MesPrint("&Illegal range specification in LHS of %#add instruction.");
00529             return(1);
00530         }
00531         type = DICT_RANGE;
00532         if ( x1 <= 0 || x2 <= 0 || x1 > x2 ) {
00533             MesPrint("&Illegal range in LHS of %#add instruction.");
00534             return(1);
00535         }
00536         range[0] = x1;
00537         range[1] = x2;
00538         range[2] = 0;
00539         where = range;
00540         goto TestDouble;
00541     }
00542 /*
00543     Translate the left part. Determine type.
00544     We follow the code in CoIdExpression and then veto what we do not like.
00545     Just make sure to pop what needs to be popped in the compiler buffer.
00546 */
00547     AC.ProtoType = w;
00548     *w++ = SUBEXPRESSION;
00549     *w++ = SUBEXPSIZE;
00550     *w++ = C->numrhs+1;
00551     *w++ = 1;
00552     *w++ = AC.cbufnum;
00553     FILLSUB(w)
00554     AC.WildC = w;
00555     AC.NwildC = 0;
00556     AT.WorkPointer = s = w + 4*AM.MaxWildcards + 8;
00557 /*
00558     Now read the LHS
00559 */
00560     oldcpointer = AddLHS(AC.cbufnum) - C->Buffer;
00561
00562     if ( ( retcode = CompileAlgebra(left,LHSIDE,AC.ProtoType) ) < 0 ) { error = 1; }
00563     else AC.ProtoType[2] = retcode;
00564     AT.WorkPointer = s;
00565     if ( AC.NwildC && SortWild(w,AC.NwildC) ) error = 1;
00566
00567     OldWork[1] = AC.WildC-OldWork;
00568     w = AC.WildC;
00569     AT.WorkPointer = w;
00570     s = C->rhs[C->numrhs];
00571 /*
00572     We have the expression in the compiler buffers.
00573     The main level is at lhs[numlhs]
00574     The partial lhs (including ProtoType) is in OldWork (in Workspace)
00575     We need to load the result at w after the prototype
00576     Because these sort routines don't use the Workspace
00577     there should not be a conflict
00578 */
00579     if ( !error && *s == 0 ) {
00580     IllLeft:MesPrint("&Illegal LHS in dictionary");
00581         AC.lhdollarflag = 0;
00582         return(1);
00583     }
00584     if ( !error && *(s+s) != 0 ) {
00585         MesPrint("&LHS in dictionary should be one term only");
00586         return(1);
00587     }
00588     if ( error == 0 ) {
00589         if ( NewSort(BHEAD0) || NewSort(BHEAD0) ) {
00590             if ( !error ) error = 1;
00591             return(error);
00592         }
00593         AN.RepPoint = AT.RepCount + 1;
00594         ow = (WORD *) ((UBYTE *) (AT.WorkPointer)) + AM.MaxTer;
00595         mm = s; ww = ow; i = *mm;
00596         while ( --i >= 0 ) { *ww++ = *mm++; } AT.WorkPointer = ww;
00597         AC.lhdollarflag = 0; oldEside = AR.Eside; AR.Eside = LHSIDE;
00598         AR.Cnumlhs = C->numlhs;
00599         if ( Generator(BHEAD ow,C->numlhs) ) {
00600             AR.Eside = oldEside;
00601             LowerSortLevel(); LowerSortLevel(); goto IllLeft;
00602         }
00603         AR.Eside = oldEside;
00604         AT.WorkPointer = w;
00605         if ( EndSort(BHEAD w,0) < 0 ) { LowerSortLevel(); goto IllLeft; }
00606         if ( *w == 0 || *(w+w) != 0 ) {

```

```

00607         MesPrint("&LHS must be one term");
00608         AC.lhdollarflag = 0;
00609         return(1);
00610     }
00611     LowerSortLevel();
00612 }
00613 AT.WorkPointer = w + *w;
00614 AC.DumNum = 0;
00615 /*
00616 Everything is now after OldWork. We can pop the compilerbuffer.
00617 Next test for illegal things like a coefficient
00618 At this point we have:
00619 w = the term of the LHS
00620 */
00621 C->Pointer = C->Buffer + oldcpointer;
00622 C->numrhs = oldnumrhs;
00623 C->numlhs = oldnumlhs;
00624 AC.lhdollarflag = 0;
00625 /*
00626 Test for undesirables.
00627 1: wildcards
00628 2: sign
00629 3: more than one term
00630 4: composite terms
00631 */
00632 if ( AC.ProtoType[1] != SUBEXPSIZE ) {
00633     MesPrint("& Currently no wildcards allowed in dictionaries.");
00634     return(1);
00635 }
00636 if ( w[w[0]-1] < 0 ) {
00637     MesPrint("& Currently no sign allowed in dictionaries.");
00638     return(1);
00639 }
00640 if ( w[w[0]] != 0 ) {
00641     MesPrint("& More than one term in dictionary element.");
00642     return(1);
00643 }
00644 if ( w[0] == w[w[0]-1]+1 ) { /* Only coefficient */
00645     WORD *number, *denom;
00646     WORD nsize, dsize;
00647     nsize = dsize = (w[w[0]-1]-1)/2;
00648     number = w+1;
00649     denom = number+nsize;
00650     while ( number[nsize-1] == 0 ) nsize--;
00651     while ( denom[dsize-1] == 0 ) dsize--;
00652     if ( dsize == 1 && denom[0] == 1 ) {
00653         type = DICT_INTEGERNUMBER;
00654         number = nsize;
00655         where = number;
00656     }
00657     else {
00658         type = DICT_RATIONALNUMBER;
00659         number = w[0];
00660         where = w;
00661     }
00662 }
00663 else {
00664     s = w + w[0]-1;
00665     if ( s[0] != 3 || s[-1] != 1 || s[-2] != 1 ) {
00666 Compositeness:;
00667         MesPrint("& Currently no composite objects allowed in dictionaries.");
00668         return(1);
00669     }
00670     if ( w[0] != w[2]+4 ) goto Compositeness;
00671     s = w+1;
00672     switch ( *s ) {
00673     case SYMBOL:
00674         if ( s[1] != 4 || s[3] != 1 ) goto Compositeness;
00675         type = DICT_SYMBOL;
00676         number = 1;
00677         where = s+2;
00678         break;
00679     case INDEX:
00680         if ( s[1] != 3 ) goto Compositeness;
00681         if ( s[2] < 0 ) type = DICT_VECTOR;
00682         else type = DICT_INDEX;
00683         number = 1;
00684         where = s+2;
00685         break;
00686     default:
00687         if ( *s < FUNCTION ) {
00688             MesPrint("& Illegal object in dictionary.");
00689             return(1);
00690         }
00691         if ( s[1] == FUNHEAD ) {
00692             type = DICT_FUNCTION;
00693             number = 1;

```

```

00694         where = s;
00695         break;
00696     }
00697     else {
00698         type = DICT_FUNCTION_WITH_ARGUMENTS;
00699         number = s[1];
00700         where = s;
00701     }
00702     break;
00703 }
00704 }
00705 TestDouble;;
00706 /*
00707  Create a new element
00708 */
00709 if ( dict->numelements >= dict->sizeelements ) {
00710     DICTIONARY_ELEMENT **d;
00711     if ( dict->sizeelements <= 0 ) dict->sizeelements = 10;
00712     else dict->sizeelements *= 2;
00713     d = (DICTIONARY_ELEMENT **)Malloc1(
00714         sizeof(DICTIONARY_ELEMENT *)*dict->sizeelements,"Dictionary elements");
00715     for ( i = 0; i < dict->numelements; i++ )
00716         d[i] = dict->elements[i];
00717     if ( dict->elements ) M_free(dict->elements,"Dictionary elements");
00718     dict->elements = d;
00719 }
00720 sizelhs = number+1;
00721 sizerhs = 1; r = right; while ( *r++ ) sizerhs++;
00722 sizerhs = (sizerhs+sizeof(WORD)-1)/sizeof(WORD)+1;
00723 new = (DICTIONARY_ELEMENT *)Malloc1(sizeof(DICTIONARY_ELEMENT)
00724     +sizeof(WORD)*(sizelhs+sizerhs),"Dictionary element");
00725 new->lhs = (WORD *) (new+1);
00726 new->rhs = new->lhs+sizelhs;
00727 new->type = type;
00728 new->size = number;
00729 for ( i = 0; i < number; i++ ) new->lhs[i] = where[i];
00730 new->lhs[i] = 0;
00731 r = (UBYTE *) (new->rhs);
00732 while ( *right ) {
00733     if ( *right == '\\ ' && ( right[1] == ' ' || right[1] == '\n' ) ) right++;
00734     *r++ = *right++;
00735 }
00736 *r = 0;
00737
00738 dict->elements[dict->numelements++] = new;
00739
00740 switch ( type ) {
00741     case DICT_INTEGERNUMBER:
00742     case DICT_RATIONALNUMBER:
00743         dict->numbers++; break;
00744     case DICT_SYMBOL:
00745     case DICT_VECTOR:
00746     case DICT_INDEX:
00747     case DICT_FUNCTION:
00748         dict->variables++; break;
00749     case DICT_FUNCTION_WITH_ARGUMENTS:
00750         dict->funwith++; break;
00751     case DICT_SPECIALCHARACTER:
00752         dict->characters++; break;
00753     case DICT_RANGE:
00754         dict->ranges++; break;
00755 }
00756
00757 AT.WorkPointer = OldWork;
00758 return(0);
00759 }
00760
00761 /*
00762 #] AddToDictionary :
00763 #[ UseDictionary :
00764 */
00765
00766 int UseDictionary(UBYTE *name,UBYTE *options)
00767 {
00768     int i;
00769     for ( i = 0; i < AO.NumDictionaries; i++ ) {
00770         if ( StrCmp(AO.Dictionaries[i]->name,name) == 0 ) {
00771             AO.CurrentDictionary = i+1;
00772             if ( SetDictionaryOptions(options) < 0 ) {
00773                 AO.CurrentDictionary = 0;
00774                 return(-1);
00775             }
00776             else { /* Now test whether what is requested is really there? */
00777                 return(0);
00778             }
00779         }
00780     }

```



```

00781     MesPrint("@There is no dictionary with the name %s",name);
00782     exit(-1);
00783 }
00784
00785 /*
00786  #] UseDictionary :
00787  #[ SetDictionaryOptions :
00788 */
00789
00790 int SetDictionaryOptions(UBYTE *options)
00791 {
00792     UBYTE *opt, *s, c;
00793     int retval = 0;
00794     s = options;
00795     AO.CurDictNumbers = DICT_ALLNUMBERS;
00796     AO.CurDictVariables = DICT_DOVARIABLES;
00797     AO.CurDictSpecials = DICT_DOSPECIALS;
00798     AO.CurDictFunWithArgs = DICT_DOFUNWITHARGS;
00799     AO.CurDictNumberWarning = 0;
00800     AO.CurDictNotInFunctions= 0;
00801     AO.CurDictInDollars = DICT_NOTINDOLLARS;
00802     while ( *s ) {
00803         opt = s;
00804         while ( *s && *s != ',' && *s != ' ' ) s++;
00805         c = *s; *s = 0;
00806         if ( opt[0] == '$' && opt[1] == 0 ) {
00807             AO.CurDictInDollars = DICT_INDOLLARS;
00808         }
00809         else if ( StrICmp(opt,(UBYTE *)"nonumbers") == 0 ) {
00810             AO.CurDictNumbers = DICT_NONUMBERS;
00811         }
00812         else if ( StrICmp(opt,(UBYTE *)"integeronly") == 0 ) {
00813             AO.CurDictNumbers = DICT_INTEGERONLY;
00814         }
00815         else if ( StrICmp(opt,(UBYTE *)"rationalonly") == 0 ) {
00816             AO.CurDictNumbers = DICT_RATIONALONLY;
00817         }
00818         else if ( StrICmp(opt,(UBYTE *)"allnumbers") == 0 ) {
00819             AO.CurDictNumbers = DICT_ALLNUMBERS;
00820         }
00821         else if ( StrICmp(opt,(UBYTE *)"novariables") == 0 ) {
00822             AO.CurDictVariables = DICT_NOVARIABLES;
00823         }
00824         else if ( StrICmp(opt,(UBYTE *)"numbersonly") == 0 ) {
00825             AO.CurDictNumbers = DICT_ALLNUMBERS;
00826             AO.CurDictVariables = DICT_NOVARIABLES;
00827             AO.CurDictSpecials = DICT_NOSPECIALS;
00828             AO.CurDictFunWithArgs = DICT_NOFUNWITHARGS;
00829         }
00830         else if ( StrICmp(opt,(UBYTE *)"variablesonly") == 0 ) {
00831             AO.CurDictNumbers = DICT_NONUMBERS;
00832             AO.CurDictVariables = DICT_DOVARIABLES;
00833             AO.CurDictSpecials = DICT_NOSPECIALS;
00834             AO.CurDictFunWithArgs = DICT_NOFUNWITHARGS;
00835         }
00836         else if ( StrICmp(opt,(UBYTE *)"nospecials") == 0 ) {
00837             AO.CurDictSpecials = DICT_NOSPECIALS;
00838         }
00839         else if ( StrICmp(opt,(UBYTE *)"specialonly") == 0 ) {
00840             AO.CurDictNumbers = DICT_NONUMBERS;
00841             AO.CurDictVariables = DICT_NOVARIABLES;
00842             AO.CurDictSpecials = DICT_DOSPECIALS;
00843             AO.CurDictFunWithArgs = DICT_NOFUNWITHARGS;
00844         }
00845         else if ( StrICmp(opt,(UBYTE *)"nofunwithargs") == 0 ) {
00846             AO.CurDictFunWithArgs = DICT_NOFUNWITHARGS;
00847         }
00848         else if ( StrICmp(opt,(UBYTE *)"funwithargsonly") == 0 ) {
00849             AO.CurDictNumbers = DICT_NONUMBERS;
00850             AO.CurDictVariables = DICT_NOVARIABLES;
00851             AO.CurDictSpecials = DICT_NOSPECIALS;
00852             AO.CurDictFunWithArgs = DICT_DOFUNWITHARGS;
00853         }
00854         else if ( StrICmp(opt,(UBYTE *)"warnings") == 0
00855             || StrICmp(opt,(UBYTE *)"warning") == 0 ) {
00856             AO.CurDictNumberWarning = 1;
00857         }
00858         else if ( StrICmp(opt,(UBYTE *)"nowarnings") == 0
00859             || StrICmp(opt,(UBYTE *)"nowarning") == 0 ) {
00860             AO.CurDictNumberWarning = 0;
00861         }
00862         else if ( StrICmp(opt,(UBYTE *)"infunctions") == 0 ) {
00863             AO.CurDictNotInFunctions= 0;
00864         }
00865         else if ( StrICmp(opt,(UBYTE *)"notinfunctions") == 0 ) {
00866             AO.CurDictNotInFunctions= 1;
00867         }
00868     }

```

```

00868         else {
00869             MesPrint("@ Unrecognized option in %#SetDictionary: %s",opt);
00870             retval = -1;
00871         }
00872         *s = c;
00873         if ( c == ',' ) s++;
00874     }
00875     return(retval);
00876 }
00877
00878 /*
00879  #] SetDictionaryOptions :
00880  #[ UnSetDictionary :
00881  */
00882
00883 void UnSetDictionary(void)
00884 {
00885     AO.CurrentDictionary = 0;
00886     AO.CurDictNumbers = -1;
00887     AO.CurDictVariables = -1;
00888     AO.CurDictSpecials = -1;
00889     AO.CurDictFunWithArgs = -1;
00890     AO.CurDictFunWithArgs = -1;
00891     AO.CurDictNumberWarning = -1;
00892     AO.CurDictNotInFunctions = -1;
00893 }
00894
00895 /*
00896  #] UnSetDictionary :
00897  #[ RemoveDictionary :
00898
00899     Mostly needed for .clear
00900  */
00901
00902 void RemoveDictionary(DICTIONARY *dict)
00903 {
00904     int i;
00905     if ( dict == 0 ) return;
00906     for ( i = 0; i < AO.NumDictionaries; i++ ) {
00907         if ( AO.Dictionaries[i] == dict ) {
00908             for ( i++; i < AO.NumDictionaries; i++ ) {
00909                 AO.Dictionaries[i-1] = AO.Dictionaries[i];
00910             }
00911             AO.NumDictionaries--;
00912             goto removeit;
00913         }
00914     }
00915     MesPrint("@ Dictionary not found in RemoveDictionary");
00916     exit(-1);
00917 removeit:;
00918     for ( i = 0; i < dict->numelements; i++ )
00919         M_free(dict->elements[i], "Dictionary element");
00920     for ( i = 0; i < dict->numelements; i++ ) dict->elements[i] = 0;
00921     if ( dict->elements ) M_free(dict->elements, "Dictionary elements");
00922     if ( dict->name ) {
00923         M_free(dict->name, "DictionaryName");
00924         dict->name = 0;
00925     }
00926     dict->sizeelements = 0;
00927     dict->numelements = 0;
00928     dict->numbers = 0;
00929     dict->variables = 0;
00930     dict->characters = 0;
00931     dict->funwith = 0;
00932     dict->gnumelements = 0;
00933     dict->ranges = 0;
00934 }
00935
00936 /*
00937  #] RemoveDictionary :
00938  #[ ShrinkDictionary :
00939
00940     To be called after a .store to restore the dictionary to the state
00941     it had at the last .global
00942     We do not make the elements array shorter.
00943  */
00944
00945 void ShrinkDictionary(DICTIONARY *dict)
00946 {
00947     while ( dict->numelements > dict->gnumelements ) {
00948         dict->numelements--;
00949         M_free(dict->elements[dict->numelements], "Dictionary element");
00950         dict->elements[dict->numelements] = 0;
00951     }
00952 }
00953
00954 /*

```

```

00955     #] ShrinkDictionary :
00956     #[ DoPreOpenDictionary :
00957  */
00958
00959 int DoPreOpenDictionary(UBYTE *s)
00960 {
00961     UBYTE *name;
00962     int dict;
00963     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
00964     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
00965     while ( *s == ' ' ) s++;
00966
00967     name = s; s = SkipAName(s);
00968     if ( *s != 0 && *s != ';' ) {
00969         MesPrint("@proper syntax is #opendictionary name");
00970         return(-1);
00971     }
00972     *s = 0;
00973
00974     if ( AP.OpenDictionary > 0 ) {
00975         MesPrint("@you cannot nest #opendictionary instructions");
00976         MesPrint("@dictionary %s is open already",
00977             AO.Dictionaries[AP.OpenDictionary-1]->name);
00978         return(-1);
00979     }
00980     if ( AO.CurrentDictionary > 0 ) {
00981         MesPrint("@before opening a dictionary you have to first close the selected dictionary");
00982         return(-1);
00983     }
00984     /*
00985     Do we have this dictionary already?
00986     */
00987     dict = FindDictionary(name);
00988     if ( dict == 0 ) dict = AddDictionary(name);
00989     AP.OpenDictionary = dict;
00990     return(0);
00991 }
00992
00993 /*
00994 #] DoPreOpenDictionary :
00995 #[ DoPreCloseDictionary :
00996 */
00997
00998 int DoPreCloseDictionary(UBYTE *s)
00999 {
01000     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
01001     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
01002     while ( *s == ' ' ) s++;
01003
01004     if ( AP.OpenDictionary == 0 && AO.CurrentDictionary == 0 ) {
01005         MesPrint("@you have neither an open, nor a selected dictionary");
01006         return(-1);
01007     }
01008
01009     AP.OpenDictionary = 0;
01010     AO.CurrentDictionary = 0;
01011
01012     AO.CurDictNotInFunctions = 0;
01013
01014     return(0);
01015 }
01016
01017 /*
01018 #] DoPreCloseDictionary :
01019 #[ DoPreUseDictionary :
01020 */
01021
01022 int DoPreUseDictionary(UBYTE *s)
01023 {
01024     UBYTE *options, c, *ss, *sss, *name;
01025     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
01026     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
01027     while ( *s == ' ' ) s++;
01028
01029     if ( AP.OpenDictionary > 0 ) {
01030         MesPrint("@before selecting a dictionary you have to first close the open dictionary");
01031         return(-1);
01032     }
01033
01034     name = s; s = SkipAName(s);
01035     ss = s; while ( *s && *s != '(' ) s++;
01036     c = *ss; *ss = 0;
01037     if ( c == 0 ) {
01038         options = ss;
01039     }
01040     else {
01041         options = s+1; SKIPBRA3(s)

```

```

01042         if ( *s != ')' ) {
01043             MesPrint("@Irregular end of %#UseDictionary instruction");
01044             return(-1);
01045         }
01046         sss = s;
01047         s++; while ( *s == ' ' || *s == '\t' || *s == ';' ) s++;
01048         *sss = 0;
01049         if ( *s ) {
01050             MesPrint("@Irregular end of %#UseDictionary instruction");
01051             return(-1);
01052         }
01053     }
01054     return(UseDictionary(name,options));
01055 }
01056
01057 /*
01058 #] DoPreUseDictionary :
01059 #[ DoPreAdd :
01060
01061 Syntax:
01062 #add left :right
01063 #add left : "right"
01064 Adds to the currently open dictionary
01065 */
01066
01067 int DoPreAdd(UBYTE *s)
01068 {
01069     UBYTE *left, *right;
01070
01071     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
01072     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
01073     while ( *s == ' ' ) s++;
01074
01075     if ( AP.OpenDictionary == 0 ) {
01076         MesPrint("@there is no open dictionary to add to");
01077         return(-1);
01078     }
01079     /*
01080     Scan to the : and mark the left and right parts.
01081     */
01082     left = s;
01083     while ( *s && *s != ';' ) {
01084         if ( *s == '[' ) { SKIPBRA1(s) s++; }
01085         else if ( *s == '{' ) { SKIPBRA2(s) s++; }
01086         else if ( *s == '(' ) { SKIPBRA3(s) s++; }
01087         else if ( *s == ']' || *s == '}' || *s == ')' ) {
01088             MesPrint("@unmatched brackets in #add instruction");
01089             return(-1);
01090         }
01091         else s++;
01092     }
01093     if ( *s == 0 ) {
01094         MesPrint("@Missing : in #add instruction");
01095         return(-1);
01096     }
01097     *s++ = 0;
01098     right = s;
01099     while ( *s == ' ' || *s == '\t' ) s++;
01100     if ( *s == '"' && s[1] ) {
01101         right = s+1;
01102         s = s+2;
01103         while ( *s ) s++;
01104         while ( s[-1] != '"' ) s--;
01105         if ( s <= right ) {
01106             MesPrint("@Irregular use of double quotes in #add instruction");
01107             return(-1);
01108         }
01109         s[-1] = 0;
01110     }
01111     return(AddToDictionary(AO.Dictionaries[AP.OpenDictionary-1],left,right));
01112 }
01113
01114 /*
01115 #] DoPreAdd :
01116 #[ DictToBytes :
01117 */
01118
01119 LONG DictToBytes(DICTIONARY *dict,UBYTE *buf)
01120 {
01121     int numelements = dict->numelements, sizeelement, i, j, x;
01122     UBYTE *s1, *s2 = buf;
01123     DICTIONARY_ELEMENT *e;
01124     /*
01125     First copy the struct
01126     */
01127     s1 = (UBYTE *)dict; j = sizeof(DICTIONARY);
01128     NCOPY(s2,s1,j)

```

```

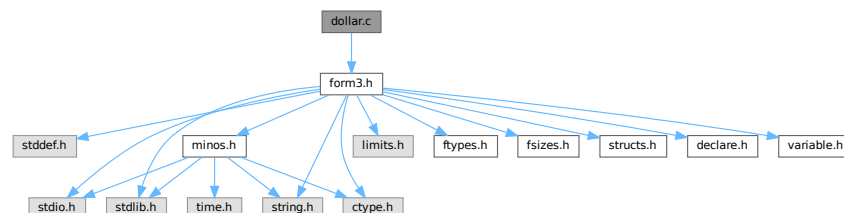
01129 /*
01130      Now the elements. Put a size indicator in front of each of them.
01131 */
01132 for ( i = 0; i < numelements; i++ ) {
01133     e = dict->elements[i];
01134     sizeelement = sizeof(DICTIONARY_ELEMENT)+(e->size+1)*sizeof(WORD);
01135     s1 = (UBYTE *)e->rhs; x = 0;
01136     while ( *s1 ) { s1++; x++; }
01137     x /= sizeof(WORD);
01138     sizeelement += (x+1) * sizeof(WORD);
01139     s1 = (UBYTE *)(&sizeelement); j = sizeof(WORD); NCOPY(s2,s1,j)
01140     s1 = (UBYTE *)e; j = sizeof(DICTIONARY_ELEMENT); NCOPY(s2,s1,j)
01141     s1 = (UBYTE *)e->lhs; j = (e->size+1)*(sizeof(WORD)); NCOPY(s2,s1,j)
01142     s1 = (UBYTE *)e->rhs; j = (x+1)*(sizeof(WORD)); NCOPY(s2,s1,j)
01143 }
01144 return(s2-buf);
01145 }
01146
01147 /*
01148      #] DictToBytes :
01149      #[ DictFromBytes :
01150 */
01151
01152 DICTIONARY *DictFromBytes(UBYTE *buf)
01153 {
01154     DICTIONARY *dict = Malloc1(sizeof(DICTIONARY),"Dictionary");
01155     UBYTE *s1, *s2;
01156     int i, j, sizeelement;
01157     DICTIONARY_ELEMENT *e;
01158     /*
01159      First read the dictionary itself
01160 */
01161     s1 = buf;
01162     s2 = (UBYTE *)dict; j = sizeof(DICTIONARY); NCOPY(s2,s1,j)
01163     /*
01164      Allocate the elements array:
01165 */
01166     dict->elements = (DICTIONARY_ELEMENT **)Malloc1(
01167         sizeof(DICTIONARY_ELEMENT *)*dict->sizeelements,"dictionary elements");
01168     for ( i = 0; i < dict->numelements; i++ ) {
01169         s2 = (UBYTE *)(&sizeelement); j = sizeof(WORD); NCOPY(s2,s1,j)
01170         e = (DICTIONARY_ELEMENT *)Malloc1(sizeelement*sizeof(UBYTE),"dictionary element");
01171         dict->elements[i] = e;
01172         j = sizeelement; s2 = (UBYTE *)e; NCOPY(s2,s1,j)
01173         e->lhs = (WORD *) (e+1);
01174         e->rhs = e->lhs + e->size+1;
01175     }
01176     return(dict);
01177 }
01178
01179 /*
01180      #] DictFromBytes :
01181 */

```

4.26 dollar.c File Reference

#include "form3.h"

Include dependency graph for dollar.c:



Macros

- #define [STEP2](#)

Functions

- int [CatchDollar](#) (int par)
- int [AssignDollar](#) (PHEAD WORD *term, WORD level)
- UBYTE * [WriteDollarToBuffer](#) (WORD numdollar, WORD par)
- UBYTE * [WriteDollarFactorToBuffer](#) (WORD numdollar, WORD numfac, WORD par)
- void [AddToDollarBuffer](#) (UBYTE *s)
- void [TermAssign](#) (WORD *term)
- int [PutTermInDollar](#) (WORD *term, WORD numdollar)
- void [WildDollars](#) (PHEAD WORD *term)
- WORD [DolToTensor](#) (PHEAD WORD numdollar)
- WORD [DolToFunction](#) (PHEAD WORD numdollar)
- WORD [DolToVector](#) (PHEAD WORD numdollar)
- WORD [DolToNumber](#) (PHEAD WORD numdollar)
- WORD [DolToSymbol](#) (PHEAD WORD numdollar)
- WORD [DolToIndex](#) (PHEAD WORD numdollar)
- [DOLLARS DolToTerms](#) (PHEAD WORD numdollar)
- LONG [DolToLong](#) (PHEAD WORD numdollar)
- int [ExecInside](#) (UBYTE *s)
- int [InsideDollar](#) (PHEAD WORD *ll, WORD level)
- void [ExchangeDollars](#) (int num1, int num2)
- LONG [TermsInDollar](#) (WORD num)
- LONG [SizeOfDollar](#) (WORD num)
- UBYTE * [PrelfDollarEval](#) (UBYTE *s, int *value)
- WORD * [TranslateExpression](#) (UBYTE *s)
- int [IsSetMember](#) (WORD *buffer, WORD numset)
- int [IsMultipleOf](#) (WORD *buf1, WORD *buf2)
- int [TwoExprCompare](#) (WORD *buf1, WORD *buf2, int oprtr)
- int [DollarRaiseLow](#) (UBYTE *name, LONG value)
- WORD [EvalDoLoopArg](#) (PHEAD WORD *arg, WORD par)
- WORD [TestDoLoop](#) (PHEAD WORD *lhsbuf, WORD level)
- WORD [TestEndDoLoop](#) (PHEAD WORD *lhsbuf, WORD level)
- int [DollarFactorize](#) (PHEAD WORD numdollar)
- void [CleanDollarFactors](#) ([DOLLARS](#) d)
- WORD * [TakeDollarContent](#) (PHEAD WORD *dollarbuffer, WORD **factor)
- WORD * [MakeDollarInteger](#) (PHEAD WORD *bufin, WORD **bufout)
- WORD * [MakeDollarMod](#) (PHEAD WORD *buffer, WORD **bufout)
- int [GetDolNum](#) (PHEAD WORD *t, WORD *tstop)
- void [AddPotModdollar](#) (WORD numdollar)

4.26.1 Detailed Description

The routines that deal with the dollar variables. The name administration is to be found in the file [names.c](#)

Definition in file [dollar.c](#).

4.26.2 Macro Definition Documentation

4.26.2.1 STEP2

```
#define STEP2
```

Factors a dollar expression. Notation: d->nfactors becomes nonzero. if the number of factors is one, we leave d->factors zero. Otherwise factors is an array of pointers to the factors. These are pointers of the type FAC-DOLLAR. fd->where pointer to contents in term notation fd->size size of the buffer fd->where points to fd->type DOLNUMBER or DOLTERMS fd->value value if type is DOLNUMBER and it fits in a WORD.

Definition at line 2953 of file [dollar.c](#).

4.26.3 Function Documentation

4.26.3.1 CatchDollar()

```
int CatchDollar (
    int par )
```

Definition at line 54 of file [dollar.c](#).

4.26.3.2 AssignDollar()

```
int AssignDollar (
    PHEAD WORD * term,
    WORD level )
```

Definition at line 209 of file [dollar.c](#).

4.26.3.3 WriteDollarToBuffer()

```
UBYTE * WriteDollarToBuffer (
    WORD numdollar,
    WORD par )
```

Definition at line 596 of file [dollar.c](#).

4.26.3.4 WriteDollarFactorToBuffer()

```
UBYTE * WriteDollarFactorToBuffer (
    WORD numdollar,
    WORD numfac,
    WORD par )
```

Definition at line 687 of file [dollar.c](#).

4.26.3.5 AddToDollarBuffer()

```
void AddToDollarBuffer (
    UBYTE * s )
```

Definition at line 762 of file [dollar.c](#).

4.26.3.6 TermAssign()

```
void TermAssign (
    WORD * term )
```

Definition at line 803 of file [dollar.c](#).

4.26.3.7 PutTermInDollar()

```
int PutTermInDollar (
    WORD * term,
    WORD numdollar )
```

Definition at line 863 of file [dollar.c](#).

4.26.3.8 WildDollars()

```
void WildDollars (
    PHEAD WORD * term )
```

Definition at line 891 of file [dollar.c](#).

4.26.3.9 DolToTensor()

```
WORD DolToTensor (
    PHEAD WORD numdollar )
```

Definition at line 1082 of file [dollar.c](#).

4.26.3.10 DolToFunction()

```
WORD DolToFunction (
    PHEAD WORD numdollar )
```

Definition at line 1143 of file [dollar.c](#).

4.26.3.11 DolToVector()

```
WORD DolToVector (
    PHEAD WORD numdollar )
```

Definition at line 1200 of file [dollar.c](#).

4.26.3.12 DolToNumber()

```
WORD DolToNumber (
    PHEAD WORD numdollar )
```

Definition at line 1264 of file [dollar.c](#).

4.26.3.13 DolToSymbol()

```
WORD DolToSymbol (
    PHEAD WORD numdollar )
```

Definition at line 1323 of file [dollar.c](#).

4.26.3.14 DolToIndex()

```
WORD DolToIndex (
    PHEAD WORD numdollar )
```

Definition at line 1377 of file [dollar.c](#).

4.26.3.15 DolToTerms()

```
DOLLARS DolToTerms (
    PHEAD WORD numdollar )
```

Definition at line 1458 of file [dollar.c](#).

4.26.3.16 DolToLong()

```
LONG DolToLong (
    PHEAD WORD numdollar )
```

Definition at line 1606 of file [dollar.c](#).

4.26.3.17 ExecInside()

```
int ExecInside (
    UBYTE * s )
```

Definition at line 1681 of file [dollar.c](#).

4.26.3.18 InsideDollar()

```
int InsideDollar (
    PHEAD WORD * ll,
    WORD level )
```

Definition at line 1746 of file [dollar.c](#).

4.26.3.19 ExchangeDollars()

```
void ExchangeDollars (
    int num1,
    int num2 )
```

Definition at line 1849 of file [dollar.c](#).

4.26.3.20 TermsInDollar()

```
LONG TermsInDollar (
    WORD num )
```

Definition at line 1867 of file [dollar.c](#).

4.26.3.21 SizeOfDollar()

```
LONG SizeOfDollar (
    WORD num )
```

Definition at line 1916 of file [dollar.c](#).

4.26.3.22 PreIfDollarEval()

```
UBYTE * PreIfDollarEval (
    UBYTE * s,
    int * value )
```

Definition at line 1978 of file [dollar.c](#).

4.26.3.23 TranslateExpression()

```
WORD * TranslateExpression (
    UBYTE * s )
```

Definition at line 2160 of file [dollar.c](#).

4.26.3.24 IsSetMember()

```
int IsSetMember (
    WORD * buffer,
    WORD numset )
```

Definition at line 2217 of file [dollar.c](#).

4.26.3.25 IsMultipleOf()

```
int IsMultipleOf (
    WORD * buf1,
    WORD * buf2 )
```

Definition at line 2398 of file [dollar.c](#).

4.26.3.26 TwoExprCompare()

```
int TwoExprCompare (
    WORD * buf1,
    WORD * buf2,
    int oprtr )
```

Definition at line 2473 of file [dollar.c](#).

4.26.3.27 DollarRaiseLow()

```
int DollarRaiseLow (
    UBYTE * name,
    LONG value )
```

Definition at line 2552 of file [dollar.c](#).

4.26.3.28 EvalDoLoopArg()

```
WORD EvalDoLoopArg (
    PHEAD WORD * arg,
    WORD par )
```

Evaluates one argument of a do loop. Such an argument is constructed from SNUMBERS DOLLAREXPRES-
SIONs and possibly DOLLAREXP2s which indicate factors of the preceding dollar. Hence we have SNUM-
BER,num DOLLAREXPRESSSION,numdollar DOLLAREXPRESSSION,numdollar,DOLLAREXP2,numfactor DOL-
LAREXPRESSSION,numdollar,DOLLAREXP2,numfactor,DOLLAREXP2,numfactor etc. Because we have a do-
loop at every stage we should have a number. The notation in DOLLAREXP2 is that ≥ 0 is number of yet another
dollar and < 0 is $-n-1$ with n the array element or zero. The return value is the (short) number. The routine works its
way through the list in a recursive manner.

Definition at line 2651 of file [dollar.c](#).

References [EvalDoLoopArg\(\)](#).

Referenced by [EvalDoLoopArg\(\)](#).

Here is the call graph for this function:



4.26.3.29 TestDoLoop()

```
WORD TestDoLoop (
    PHEAD WORD * lhsbuf,
    WORD level )
```

Definition at line 2762 of file [dollar.c](#).

4.26.3.30 TestEndDoLoop()

```
WORD TestEndDoLoop (
    PHEAD WORD * lhsbuf,
    WORD level )
```

Definition at line [2840](#) of file [dollar.c](#).

4.26.3.31 DollarFactorize()

```
int DollarFactorize (
    PHEAD WORD numdollar )
```

Definition at line [2955](#) of file [dollar.c](#).

4.26.3.32 CleanDollarFactors()

```
void CleanDollarFactors (
    DOLLARS d )
```

Definition at line [3554](#) of file [dollar.c](#).

4.26.3.33 TakeDollarContent()

```
WORD * TakeDollarContent (
    PHEAD WORD * dollarbuffer,
    WORD ** factor )
```

Definition at line [3575](#) of file [dollar.c](#).

4.26.3.34 MakeDollarInteger()

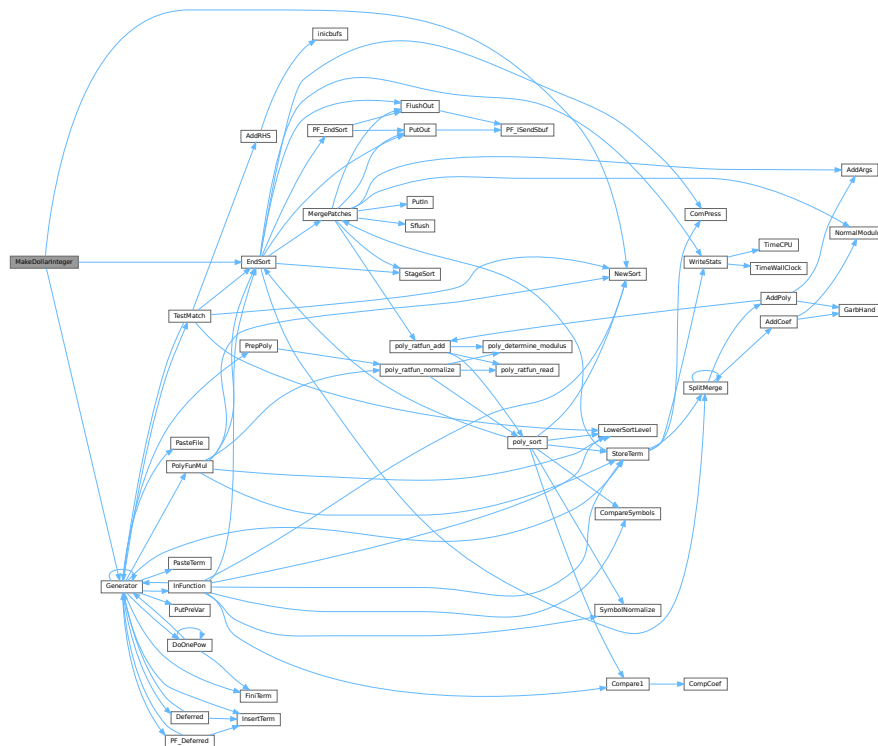
```
WORD * MakeDollarInteger (
    PHEAD WORD * bufin,
    WORD ** bufout )
```

For normalizing everything to integers we have to determine for all elements of this argument the LCM of the denominators and the GCD of the numerators. The input argument is in bufin. The number that comes out is the return value. The normalized argument is in bufout.

Definition at line [3627](#) of file [dollar.c](#).

References [EndSort\(\)](#), [Generator\(\)](#), and [NewSort\(\)](#).

Here is the call graph for this function:



4.26.3.35 MakeDollarMod()

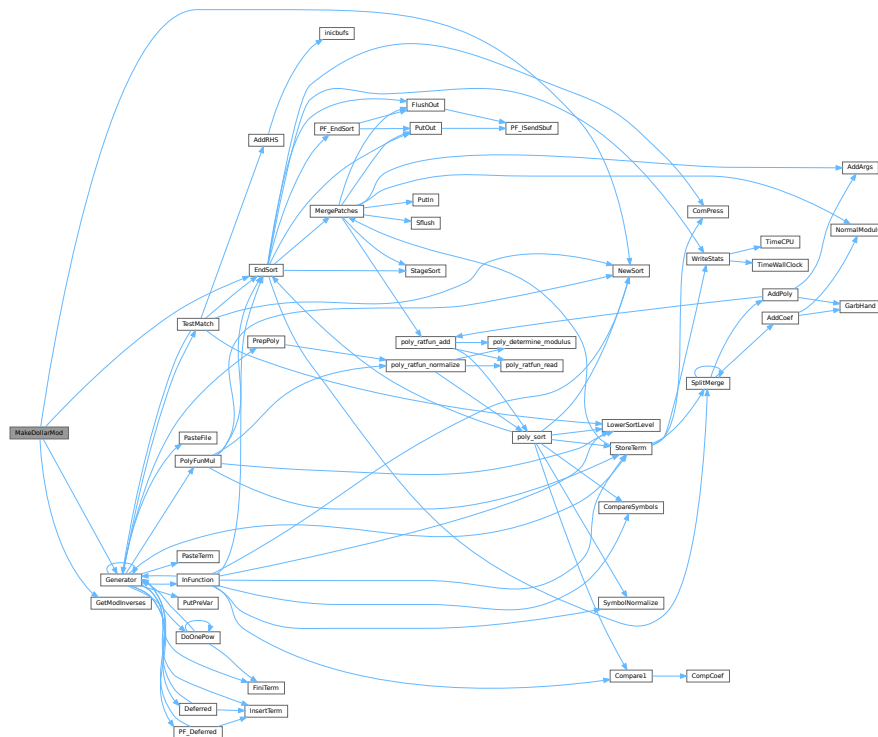
```
WORD * MakeDollarMod (
    PHEAD WORD * buffer,
    WORD ** bufout )
```

Similar to MakeDollarInteger but now with modulus arithmetic using only a one WORD 'prime'. We make the coefficient of the first term in the argument equal to one. Already the coefficients are taken modulus AN.cmod and AN.ncmod == 1

Definition at line 3801 of file dollar.c.

References [EndSort\(\)](#), [Generator\(\)](#), [GetModInverses\(\)](#), and [NewSort\(\)](#).

Here is the call graph for this function:



4.26.3.36 GetDolNum()

```
int GetDolNum (
    PHEAD WORD * t,
    WORD * tstop )
```

Definition at line 3846 of file [dollar.c](#).

4.26.3.37 AddPotModdollar()

```
void AddPotModdollar (
    WORD numdollar )
```

Adds a \$-variable specified by *numdollar* to the list of potentially modified \$-variables unless it has already been included in the list.

Parameters

<i>numdollar</i>	The index of the \$-variable to be added.
------------------	---

Definition at line 3959 of file [dollar.c](#).

4.27 dollar.c

[Go to the documentation of this file.](#)

```

00001
00006 /* #[] License : */
00007 /*
00008 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00009 * When using this file you are requested to refer to the publication
00010 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00011 * This is considered a matter of courtesy as the development was paid
00012 * for by FOM the Dutch physics granting agency and we would like to
00013 * be able to track its scientific use to convince FOM of its value
00014 * for the community.
00015 *
00016 * This file is part of FORM.
00017 *
00018 * FORM is free software: you can redistribute it and/or modify it under the
00019 * terms of the GNU General Public License as published by the Free Software
00020 * Foundation, either version 3 of the License, or (at your option) any later
00021 * version.
00022 *
00023 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00024 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00025 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00026 * details.
00027 *
00028 * You should have received a copy of the GNU General Public License along
00029 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00030 */
00031 /* #[] License : */
00032 /*
00033 #[] Includes :
00034 */
00035
00036 #include "form3.h"
00037
00038 /* EXTERNLOCK(dummylock) */
00039
00040 static UBYTE underscore[2] = {'_',0};
00041
00042 /*
00043 #[] Includes :
00044 #[] CatchDollar :
00045
00046 Works out a dollar expression during compile type.
00047 Steals it from the buffer and puts it in an assignment.
00048 At the moment we should keep this inside the small buffer.
00049 Later with more sort buffers we can do this better.
00050 Par == 0 : regular assignment
00051 par == -1: after error. Just make zero for now.
00052 */
00053
00054 int CatchDollar(int par)
00055 {
00056 GETIDENTITY
00057 CBUF *C = cbuf + AC.cbufnum;
00058 int error = 0, numterms = 0, numdollar, resetmods = 0;
00059 LONG newsize, retval;
00060 WORD *w, *t, n, nsize, *oldwork = AT.WorkPointer, *dbuffer;
00061 WORD oldncmod = AN.ncmod;
00062 DOLLARS d;
00063 if ( AN.ncmod && ( ( AC.modmode & ALSODOLLARS ) == 0 ) ) AN.ncmod = 0;
00064 if ( AN.ncmod && AN.cmod == 0 ) { SetMods(); resetmods = 1; }
00065
00066 numdollar = C->lhs[C->numlhs][2];
00067
00068 d = Dollars+numdollar;
00069 if ( par == -1 ) {
00070 d->type = DOLUNDEFINED;
00071 cbuf[AM.dbufnum].CanCommu[numdollar] = 0;
00072 cbuf[AM.dbufnum].NumTerms[numdollar] = 0;
00073 if ( d->where && d->where != &(AM.dollarzero) ) M_free(d->where,"$-buffer old");
00074 d->size = 0; d->where = &(AM.dollarzero);
00075 cbuf[AM.dbufnum].rhs[numdollar] = d->where;
00076 AN.ncmod = oldncmod;
00077 if ( resetmods ) UnSetMods();
00078 return(0);
00079 }
00080 #ifdef WITHMPI
00081 /*
00082 * The problem here is that only the master can make an assignment
00083 * like $a=g; where g is an expression: only the master has an access to
00084 * the expression. So, in cases where the RHS contains expression names,
00085 * only the master invokes Generator() and then broadcasts the result to
00086 * the all slaves.

```

```

00087      * Broadcasting must be performed immediately; one cannot postpone it
00088      * to the end of the module because the dollar variable is visible
00089      * in the current module. For the same reason, this should be done
00090      * regardless of on/off parallel status.
00091      * If the RHS does not contain any expression names, it can be processed
00092      * in each slave.
00093      */
00094      if ( PF.me == MASTER || !AC.RhsExprInModuleFlag ) {
00095 #endif
00096
00097      EXCHINOUT
00098
00099      if ( NewSort(BHEAD0) ) { if ( !error ) error = 1; goto onerror; }
00100      if ( NewSort(BHEAD0) ) {
00101          LowerSortLevel();
00102          if ( !error ) error = 1;
00103          goto onerror;
00104      }
00105      AN.RepPoint = AT.RepCount + 1;
00106      w = C->rhs[C->lhs[C->numlhs][5]];
00107      while ( *w ) {
00108          n = *w; t = oldwork;
00109          NCOPY(t,w,n);
00110          AT.WorkPointer = t;
00111          AR.Cnumlhs = C->numlhs;
00112          if ( Generator(BHEAD oldwork,C->numlhs) ) { error = 1; break; }
00113      }
00114      AT.WorkPointer = oldwork;
00115      AN.tryterm = 0; /* for now */
00116      dbuffer = 0;
00117      if ( ( retval = EndSort(BHEAD (WORD *)((void *)&dbuffer)),2) < 0 ) { error = 1; }
00118      LowerSortLevel();
00119      if ( retval <= 1 || dbuffer == 0 ) {
00120          d->type = DOLZERO;
00121          if ( d->where && d->where != &(AM.dollarzero) ) M_free(d->where,"$-buffer old");
00122          d->size = 0; d->where = &(AM.dollarzero);
00123          cbuf[AM.dbufnum].CanCommu[numdollar] = 0;
00124          cbuf[AM.dbufnum].NumTerms[numdollar] = 0;
00125          goto docopy2;
00126      }
00127      w = dbuffer;
00128      if ( error == 0 )
00129          while ( *w ) { w += *w; numterms++; }
00130      else
00131          goto onerror;
00132      newsize = (w-dbuffer)+1;
00133 #ifdef WITHMPI
00134 }
00135 if ( AC.RhsExprInModuleFlag )
00136     /* PF_BroadcastPreDollar allocates dbuffer for slaves! */
00137     if ( (error = PF_BroadcastPreDollar(&dbuffer, &newsize, &numterms)) != 0 )
00138         goto onerror;
00139 #endif
00140 if ( newsize < MINALLOC ) newsize = MINALLOC;
00141 newsize = (newsize+7)/8*8;
00142 if ( numterms == 0 ) {
00143     d->type = DOLZERO;
00144     goto docopy;
00145 }
00146 else if ( numterms == 1 ) {
00147     t = dbuffer;
00148     n = *t;
00149     nsize = t[n-1];
00150     if ( nsize < 0 ) { nsize = -nsize; }
00151     if ( nsize == (n-1) ) { /* numerical */
00152         nsize = (nsize-1)/2;
00153         w = t + 1 + nsize;
00154         if ( *w != 1 ) goto doterms;
00155         w++; while ( w < ( t + n - 1 ) ) { if ( *w ) break; w++; }
00156         if ( w < ( t + n - 1 ) ) goto doterms;
00157         d->type = DOLNUMBER;
00158         goto docopy;
00159     }
00160     else if ( n == 7 && t[6] == 3 && t[5] == 1 && t[4] == 1
00161         && t[1] == INDEX && t[2] == 3 ) {
00162         d->type = DOLINDEX;
00163         d->index = t[3];
00164         goto docopy;
00165     }
00166     else goto doterms;
00167 }
00168 else {
00169 doterms:;
00170     d->type = DOLTERMS;
00171     cbuf[AM.dbufnum].CanCommu[numdollar] = numcommute(dbuffer,
00172         &(cbuf[AM.dbufnum].NumTerms[numdollar]));
00173 docopy:;

```



```

00174         if ( d->where && d->where != &(AM.dollarzero) ) M_free(d->where,"$-buffer old");
00175         d->size = newsize; d->where = dbuffer;
00176 docopy2;;
00177         cbuf[AM.dbufnum].rhs[numdollar] = d->where;
00178     }
00179     if ( C->Pointer > C->rhs[C->numrhs] ) C->Pointer = C->rhs[C->numrhs];
00180     C->numlhs--; C->numrhs--;
00181 onerror:
00182 #ifdef WITHMPI
00183     if ( PF.me == MASTER || !AC.RhsExprInModuleFlag )
00184 #endif
00185     BACKINOUT
00186     AN.ncmod = oldncmod;
00187     if ( resetmods ) UnSetMods();
00188     return(error);
00189 }
00190
00191 /*
00192  #] CatchDollar :
00193  #[ AssignDollar :
00194
00195  To be called from Generator. Assigns an expression to a $ variable.
00196  This one is slightly different from CatchDollar.
00197  We have no easy buffer this time.
00198  We will have to hack our way using what we normally use for functions.
00199
00200  Note that in the threaded case we trust the user. That means that
00201  we are not going to recheck whether there is a maximum, minimum or sum.
00202  If the user says it is like that, we treat it like that.
00203  We only check that in this centralized version MODLOCAL isn't used.
00204
00205  In a later stage dtype could be used for actually checking MODMAX
00206  and MODMIN cases.
00207 */
00208
00209 int AssignDollar(PHEAD WORD *term, WORD level)
00210 {
00211     GETBIDENTITY
00212     CBUF *C = cbuf+AM.rbufnum;
00213     int numterms = 0, numdollar = C->lhs[level][2];
00214     LONG newsize;
00215     DOLLARS d = Dollars + numdollar;
00216     WORD *w, *t, n, nsize, *rh = cbuf[C->lhs[level][7]].rhs[C->lhs[level][5]];
00217     WORD *ss, *ww;
00218     WORD olddefer, oldcompress, oldncmod = AN.ncmod;
00219 #ifdef WITHPTHREADS
00220     int nummodopt, dtype = -1, dw;
00221     WORD numvalue;
00222     if ( AN.ncmod && ( ( AC.modmode & ALSODOLLARS ) == 0 ) ) AN.ncmod = 0;
00223     if ( AS.MultiThreaded && ( AC.mparallelflag == PARALLELFLAG ) ) {
00224 /*
00225         Here we come only when the module runs with more than one thread.
00226         This must be a variable with a special module option.
00227         For the multi-threaded version we only allow MODSUM, MODMAX and MODMIN.
00228 */
00229         for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
00230             if ( numdollar == ModOptdollars[nummodopt].number ) break;
00231         }
00232         if ( nummodopt >= NumModOptdollars ) {
00233             MLOCK(ErrorMessageLock);
00234             MesPrint("Illegal attempt to change $-variable in multi-threaded module %1",AC.CModule);
00235             MUNLOCK(ErrorMessageLock);
00236             Terminate(-1);
00237         }
00238         dtype = ModOptdollars[nummodopt].type;
00239         if ( dtype == MODLOCAL ) {
00240             d = ModOptdollars[nummodopt].dstruct+AT.identity;
00241         }
00242     }
00243 #endif
00244     DUMMYUSE(term);
00245     w = rh;
00246 /*
00247     First some shortcuts
00248 */
00249     if ( *w == 0 ) {
00250 /*
00251         #[ Thread version : Zero case
00252 */
00253 #ifdef WITHPTHREADS
00254         if ( dtype > 0 ) {
00255 /*             LOCK(d->pthreadlockwrite); */
00256             LOCK(d->pthreadlockread);
00257 NewValIsZero;;
00258             switch ( d->type ) {
00259                 case DOLZERO: goto NoChangeZero;
00260                 case DOLNUMBER:

```

```

00261         case DOLTERMS:
00262             if ( ( dw = d->where[0] ) > 0 && d->where[dw] != 0 ) {
00263                 break; /* was not a single number. Trust the user */
00264             }
00265             if ( dtype == MODMAX && d->where[dw-1] >= 0 ) goto NoChangeZero;
00266             if ( dtype == MODMIN && d->where[dw-1] <= 0 ) goto NoChangeZero;
00267             break;
00268         default:
00269             numvalue = DolToNumber(BHEAD numdollar);
00270             if ( AN.ErrorInDollar != 0 ) break;
00271             if ( dtype == MODMAX && numvalue >= 0 ) goto NoChangeZero;
00272             if ( dtype == MODMIN && numvalue <= 0 ) goto NoChangeZero;
00273             break;
00274     }
00275     d->type = DOLZERO;
00276     d->where[0] = 0;
00277     cbuf[AM.dbufnum].CanCommu[numdollar] = 0;
00278     cbuf[AM.dbufnum].NumTerms[numdollar] = 0;
00279 NoChangeZero:;
00280     CleanDollarFactors(d);
00281     /* UNLOCK(d->pthreadlockwrite); */
00282     UNLOCK(d->pthreadlockread);
00283     AN.ncmod = oldncmod;
00284     return(0);
00285 }
00286 #endif
00287 /*
00288     #] Thread version :
00289 */
00290     d->type = DOLZERO;
00291     d->where[0] = 0;
00292     cbuf[AM.dbufnum].CanCommu[numdollar] = 0;
00293     cbuf[AM.dbufnum].NumTerms[numdollar] = 0;
00294     CleanDollarFactors(d);
00295     AN.ncmod = oldncmod;
00296     return(0);
00297 }
00298 else if ( *w == 4 && w[4] == 0 && w[2] == 1 ) {
00299     /*
00300         #[ Thread version : New value is 'single precision'
00301     */
00302     #ifdef WITHPTHREADS
00303         if ( dtype > 0 ) {
00304             /* LOCK(d->pthreadlockwrite); */
00305             LOCK(d->pthreadlockread);
00306             if ( d->size < MINALLOC ) {
00307                 WORD oldsize, *oldwhere, i;
00308                 oldsize = d->size; oldwhere = d->where;
00309                 d->size = MINALLOC;
00310                 d->where = (WORD *)Malloc1(d->size*sizeof(WORD),"dollar contents");
00311                 cbuf[AM.dbufnum].rhs[numdollar] = d->where;
00312                 if ( oldsize > 0 ) {
00313                     for ( i = 0; i < oldsize; i++ ) d->where[i] = oldwhere[i];
00314                 }
00315                 else d->where[0] = 0;
00316                 if ( oldwhere && oldwhere != &(AM.dollarzero) ) M_free(oldwhere,"dollar contents");
00317             }
00318             switch ( d->type ) {
00319                 case DOLZERO:
00320 HandleDolZero:;
00321                     if ( dtype == MODMAX && w[3] <= 0 ) goto NoChangeOne;
00322                     if ( dtype == MODMIN && w[3] >= 0 ) goto NoChangeOne;
00323                     break;
00324                     case DOLNUMBER:
00325                     case DOLTERMS:
00326                         if ( ( dw = d->where[0] ) > 0 && d->where[dw] != 0 ) {
00327                             break; /* was not a single number. Trust the user */
00328                         }
00329                         if ( dtype == MODMAX && CompCoef(d->where,w) >= 0 ) goto NoChangeOne;
00330                         if ( dtype == MODMIN && CompCoef(d->where,w) <= 0 ) goto NoChangeOne;
00331                         break;
00332                     default:
00333                         {
00334                             /*
00335                                 Note that we convert the type for the next time around.
00336                             */
00337                             WORD extraterm[4];
00338                             numvalue = DolToNumber(BHEAD numdollar);
00339                             if ( AN.ErrorInDollar != 0 ) break;
00340                             if ( numvalue == 0 ) {
00341                                 d->type = DOLZERO;
00342                                 d->where[0] = 0;
00343                                 cbuf[AM.dbufnum].CanCommu[numdollar] = 0;
00344                                 cbuf[AM.dbufnum].NumTerms[numdollar] = 0;
00345                                 goto HandleDolZero;
00346                             }
00347                             d->where[0] = extraterm[0] = 4;

```

```

00348             d->where[1] = extraterm[1] = ABS(numvalue);
00349             d->where[2] = extraterm[2] = 1;
00350             d->where[3] = extraterm[3] = numvalue > 0 ? 3 : -3;
00351             d->where[4] = 0;
00352             d->type = DOLNUMBER;
00353             if ( dtype == MODMAX && CompCoef(extraterm,w) >= 0 ) goto NoChangeOne;
00354             if ( dtype == MODMIN && CompCoef(extraterm,w) <= 0 ) goto NoChangeOne;
00355             break;
00356         }
00357     }
00358     d->where[0] = w[0];
00359     d->where[1] = w[1];
00360     d->where[2] = w[2];
00361     d->where[3] = w[3];
00362     d->where[4] = 0;
00363     d->type = DOLNUMBER;
00364     cbuf[AM.dbufnum].CanCommu[numdollar] = 0;
00365     cbuf[AM.dbufnum].NumTerms[numdollar] = 1;
00366     NoChangeOne;
00367     CleanDollarFactors(d);
00368     /* UNLOCK(d->pthreadlockwrite); */
00369     UNLOCK(d->pthreadlockread);
00370     AN.ncmod = oldncmod;
00371     return(0);
00372 }
00373 #endif
00374 /*
00375     #] Thread version :
00376 */
00377     if ( d->size < MINALLOC ) {
00378         if ( d->where && d->where != &(AM.dollarzero) ) M_free(d->where,"dollar contents");
00379         d->size = MINALLOC;
00380         d->where = (WORD *)Malloc1(d->size*sizeof(WORD),"dollar contents");
00381         cbuf[AM.dbufnum].rhs[numdollar] = d->where;
00382     }
00383     d->where[0] = w[0];
00384     d->where[1] = w[1];
00385     d->where[2] = w[2];
00386     d->where[3] = w[3];
00387     d->where[4] = 0;
00388     d->type = DOLNUMBER;
00389     cbuf[AM.dbufnum].CanCommu[numdollar] = 0;
00390     cbuf[AM.dbufnum].NumTerms[numdollar] = 1;
00391     CleanDollarFactors(d);
00392     AN.ncmod = oldncmod;
00393     return(0);
00394 }
00395 /*
00396     Now the real evaluation.
00397     In the case of threads and MODSUM this requires an immediate lock.
00398     Otherwise the lock could be placed later.
00399 */
00400 #ifdef WITHPTHREADS
00401     if ( dtype == MODSUM ) {
00402         /* LOCK(d->pthreadlockwrite); */
00403         LOCK(d->pthreadlockread);
00404     }
00405 #endif
00406     CleanDollarFactors(d);
00407     /*
00408         The following case cannot occur. We treated it already
00409     */
00410     if ( *w == 0 ) {
00411         ss = 0; numterms = 0; newsize = 0;
00412         olddefer = AR.DeferFlag; AR.DeferFlag = 0;
00413         oldcompress = AR.NoCompress; AR.NoCompress = 1;
00414     }
00415     else
00416     /*
00417         {
00418         /*
00419             New value is an expression that has to be evaluated first
00420             This is all generic. It won't foliate due to the sort level
00421         */
00422         if ( NewSort(BHEAD0) ) {
00423             AN.ncmod = oldncmod;
00424             return(1);
00425         }
00426         olddefer = AR.DeferFlag; AR.DeferFlag = 0;
00427         oldcompress = AR.NoCompress; AR.NoCompress = 1;
00428         while ( *w ) {
00429             n = *w; t = ww = AT.WorkPointer;
00430             NCOPY(t,w,n);
00431             AT.WorkPointer = t;
00432             if ( Generator(BHEAD ww,AR.Cnumlhs) ) {
00433                 AT.WorkPointer = ww;
00434                 LowerSortLevel();

```

```

00435         AR.DeferFlag = olddefer;
00436         AN.ncmod = oldncmod;
00437         return(1);
00438     }
00439     AT.WorkPointer = ww;
00440 }
00441 AN.tryterm = 0; /* for now */
00442 if ( ( newsize = EndSort(BHEAD (WORD *) ((void *) (&ss)), 2) ) < 0 ) {
00443     AN.ncmod = oldncmod;
00444     return(1);
00445 }
00446 numterms = 0; t = ss; while ( *t ) { numterms++; t += *t; }
00447 }
00448 #ifdef WITHPTHREADS
00449     if ( dtype != MODSUM ) {
00450         /* LOCK(d->pthreadlockwrite); */
00451         LOCK(d->pthreadlockread);
00452     }
00453 #endif
00454     if ( numterms == 0 ) {
00455         /*
00456             the new value evaluates to zero
00457         */
00458         #ifdef WITHPTHREADS
00459             if ( dtype == MODMAX || dtype == MODMIN ) {
00460                 if ( ss ) { M_free(ss, "Sort of $"); ss = 0; }
00461                 AR.DeferFlag = olddefer; AR.NoCompress = oldcompress;
00462                 goto NewValIsZero;
00463             }
00464             else
00465         #endif
00466         {
00467             if ( d->where && d->where != &(AM.dollarzero) ) M_free(d->where, "dollar contents");
00468             d->where = &(AM.dollarzero);
00469             d->size = 0;
00470             cbuf[AM.dbufnum].rhs[numdollar] = 0;
00471             cbuf[AM.dbufnum].CanCommu[numdollar] = 0;
00472             cbuf[AM.dbufnum].NumTerms[numdollar] = 0;
00473             d->type = DOLZERO;
00474         }
00475         if ( ss ) { M_free(ss, "Sort of $"); ss = 0; }
00476     }
00477     else {
00478         /*
00479             #! Thread version :
00480         */
00481         #ifdef WITHPTHREADS
00482             if ( dtype == MODMAX || dtype == MODMIN ) {
00483                 if ( numterms == 1 && ( *ss-1 == ABS(ss[*ss-1]) ) ) { /* is number */
00484                     switch ( d->type ) {
00485                         case DOLZERO:
00486                             HandleDolZero1:
00487                                 if ( dtype == MODMAX && ss[*ss-1] > 0 ) break;
00488                                 if ( dtype == MODMIN && ss[*ss-1] < 0 ) break;
00489                                 if ( ss ) { M_free(ss, "Sort of $"); ss = 0; }
00490                                 AR.DeferFlag = olddefer; AR.NoCompress = oldcompress;
00491                                 goto NoChange;
00492                             case DOLTERMS:
00493                             case DOLNUMBER:
00494                                 if ( ( dw = d->where[0] ) > 0 && d->where[dw] != 0 ) break;
00495                                 if ( dtype == MODMAX && CompCoef(ss, d->where) > 0 ) break;
00496                                 if ( dtype == MODMIN && CompCoef(ss, d->where) < 0 ) break;
00497                                 if ( ss ) { M_free(ss, "Sort of $"); ss = 0; }
00498                                 AR.DeferFlag = olddefer; AR.NoCompress = oldcompress;
00499                                 goto NoChange;
00500                             default: {
00501                                 WORD extraterm[4];
00502                                 numvalue = DolToNumber(BHEAD numdollar);
00503                                 if ( AN.ErrorInDollar != 0 ) break;
00504                                 if ( numvalue == 0 ) {
00505                                     d->type = DOLZERO;
00506                                     d->where[0] = 0;
00507                                     cbuf[AM.dbufnum].CanCommu[numdollar] = 0;
00508                                     cbuf[AM.dbufnum].NumTerms[numdollar] = 0;
00509                                     goto HandleDolZero1;
00510                                 }
00511                                 d->where[0] = extraterm[0] = 4;
00512                                 d->where[1] = extraterm[1] = ABS(numvalue);
00513                                 d->where[2] = extraterm[2] = 1;
00514                                 d->where[3] = extraterm[3] = numvalue > 0 ? 3 : -3;
00515                                 d->where[4] = 0;
00516                                 d->type = DOLNUMBER;
00517                                 if ( dtype == MODMAX && CompCoef(ss, extraterm) > 0 ) break;
00518                                 if ( dtype == MODMIN && CompCoef(ss, extraterm) < 0 ) break;
00519                                 if ( ss ) { M_free(ss, "Sort of $"); ss = 0; }
00520                                 AR.DeferFlag = olddefer; AR.NoCompress = oldcompress;
00521                                 goto NoChange;

```

```

00522     }
00523     }
00524     }
00525     else {
00526         if ( ss ) { M_free(ss,"Sort of $"); ss = 0; }
00527         AR.DeferFlag = olddefer; AR.NoCompress = oldcompress;
00528         goto NoChange;
00529     }
00530 }
00531 #endif
00532 /*
00533 #] Thread version :
00534 */
00535     d->type = DOLTERMS;
00536     if ( d->where && d->where != &(AM.dollarzero) ) { M_free(d->where,"dollar contents"); d->where
= 0; }
00537     d->size = newsize + 1;
00538     d->where = ss;
00539     cbuf[AM.dbufnum].rhs[numdollar] = w = d->where;
00540 }
00541     AR.DeferFlag = olddefer; AR.NoCompress = oldcompress;
00542 /*
00543     Now find the special cases
00544 */
00545     if ( numterms == 0 ) {
00546         d->type = DOLZERO;
00547     }
00548     else if ( numterms == 1 ) {
00549         t = d->where;
00550         n = *t;
00551         nsize = t[n-1];
00552         if ( nsize < 0 ) { nsize = -nsize; }
00553         if ( nsize == (n-1) ) {
00554             nsize = (nsize-1)/2;
00555             w = t + 1 + nsize;
00556             if ( *w == 1 ) {
00557                 w++; while ( w < ( t + n - 1 ) ) { if ( *w ) break; w++; }
00558                 if ( w >= ( t + n - 1 ) ) d->type = DOLNUMBER;
00559             }
00560         }
00561         else if ( n == 7 && t[6] == 3 && t[5] == 1 && t[4] == 1
&& t[1] == INDEX && t[2] == 3 ) {
00562             d->type = DOLINDEX;
00563             d->index = t[3];
00564         }
00565     }
00566 }
00567     if ( d->type == DOLTERMS ) {
00568         cbuf[AM.dbufnum].CanCommu[numdollar] = numcommute(d->where,
&cbuf[AM.dbufnum].NumTerms[numdollar]);
00569     }
00570     else {
00571         cbuf[AM.dbufnum].CanCommu[numdollar] = 0;
00572         cbuf[AM.dbufnum].NumTerms[numdollar] = 1;
00573     }
00574 }
00575 #ifdef WITHPTHREADS
00576 NoChange;;
00577 /* UNLOCK(d->pthreadlockwrite); */
00578 UNLOCK(d->pthreadlockread);
00579 #endif
00580     AN.ncmod = oldncmod;
00581     return(0);
00582 }
00583
00584 /*
00585 #] AssignDollar :
00586 #[ WriteDollarToBuffer :
00587
00588     Takes the numbered dollar expression and writes it to output.
00589     We catch however the output in a buffer and return its address.
00590     This routine is needed when we need a text representation of
00591     a dollar expression like for the construction '$name' in the preprocessor.
00592     If par==0 we leave the current printing mode.
00593     If par==1 we insist on normal mode
00594 */
00595
00596 UBYTE *WriteDollarToBuffer(WORD numdollar, WORD par)
00597 {
00598     DOLLARS d = Dollars+numdollar;
00599     UBYTE *s, *oldcurbufwrt = AO.CurBufWrt;
00600     WORD *t, lbrac = 0, first = 0, arg[2], oldOutputMode = AC.OutputMode;
00601     WORD oldinfrack = AO.InFbrack;
00602     int error = 0;
00603     int dict = AO.CurrentDictionary;
00604
00605     AO.DollarOutSizeBuffer = 32;
00606     AO.DollarOutBuffer = (UBYTE *)Malloc1(AO.DollarOutSizeBuffer,"DollarOutBuffer");
00607     AO.DollarInOutBuffer = 1;

```

```

00608     AO.PrintType = 1;
00609     AO.InFbrack = 0;
00610     s = AO.DollarOutBuffer;
00611     *s = 0;
00612     if ( par > 0 && AO.CurDictInDollars == 0 ) {
00613         AC.OutputMode = NORMALFORMAT;
00614         AO.CurrentDictionary = 0;
00615     }
00616     else {
00617         AO.CurBufWrt = (UBYTE *)underscore;
00618     }
00619     AO.OutInBuffer = 1;
00620     switch ( d->type ) {
00621     case DOLARGUMENT:
00622         WriteArgument(d->where);
00623         break;
00624     case DOLSUBTERM:
00625         WriteSubTerm(d->where,1);
00626         break;
00627     case DOLNUMBER:
00628     case DOLTERMS:
00629         t = d->where;
00630         while ( *t ) {
00631             if ( WriteTerm(t,&lbrac,first,PRINTON,0) ) {
00632                 error = 1; break;
00633             }
00634             t += *t;
00635         }
00636         break;
00637     case DOLWILDARGS:
00638         t = d->where+1;
00639         while ( *t ) {
00640             WriteArgument(t);
00641             NEXTARG(t)
00642             if ( *t ) TokenToLine((UBYTE *)(","));
00643         }
00644         break;
00645     case DOLINDEX:
00646         arg[0] = -INDEX; arg[1] = d->index;
00647         WriteArgument(arg);
00648         break;
00649     case DOLZERO:
00650         *s++ = '0'; *s = 0;
00651         AO.DollarInOutBuffer = 1;
00652         break;
00653     case DOLUNDEFINED:
00654         *s = 0;
00655         AO.DollarInOutBuffer = 1;
00656         break;
00657     }
00658     AC.OutputMode = oldOutputMode;
00659     AO.OutInBuffer = 0;
00660     AO.InFbrack = oldinfbrack;
00661     AO.CurBufWrt = oldcurbufwrt;
00662     AO.CurrentDictionary = dict;
00663     if ( error ) {
00664         MLOCK(ErrorMessageLock);
00665         MesPrint("&Illegal dollar object for writing");
00666         MUNLOCK(ErrorMessageLock);
00667         M_free(AO.DollarOutBuffer,"DollarOutBuffer");
00668         AO.DollarOutBuffer = 0;
00669         AO.DollarOutSizeBuffer = 0;
00670         return(0);
00671     }
00672     return(AO.DollarOutBuffer);
00673 }
00674
00675 /*
00676 #] WriteDollarToBuffer :
00677 #[ WriteDollarFactorToBuffer :
00678
00679 Takes the numbered dollar expression and writes it to output.
00680 We catch however the output in a buffer and return its address.
00681 This routine is needed when we need a text representation of
00682 a dollar expression like for the construction '$name' in the preprocessor.
00683 If par==0 we leave the current printing mode.
00684 If par==1 we insist on normal mode
00685 */
00686
00687 UBYTE *WriteDollarFactorToBuffer(WORD numdollar, WORD numfac, WORD par)
00688 {
00689     DOLLARS d = Dollars+numdollar;
00690     UBYTE *s, *oldcurbufwrt = AO.CurBufWrt;
00691     WORD *t, lbrac = 0, first = 0, n[5], oldOutputMode = AC.OutputMode;
00692     WORD oldinfbrack = AO.InFbrack;
00693     int error = 0;
00694     int dict = AO.CurrentDictionary;

```

```

00695
00696     if ( numfac > d->nfactors || numfac < 0 ) {
00697         MLOCK(ErrorMessageLock);
00698         MesPrint("&Illegal factor number for this dollar variable: %d",numfac);
00699         MesPrint("&There are %d factors",d->nfactors);
00700         MUNLOCK(ErrorMessageLock);
00701         return(0);
00702     }
00703
00704     AO.DollarOutSizeBuffer = 32;
00705     AO.DollarOutBuffer = (UBYTE *)Malloc1(AO.DollarOutSizeBuffer,"DollarOutBuffer");
00706     AO.DollarInOutBuffer = 1;
00707     AO.PrintType = 1;
00708     AO.InFbrack = 0;
00709     s = AO.DollarOutBuffer;
00710     *s = 0;
00711     if ( par > 0 ) {
00712         AC.OutputMode = NORMALFORMAT;
00713         AO.CurrentDictionary = 0;
00714     }
00715     else {
00716         AO.CurBufWrt = (UBYTE *)underscore;
00717     }
00718     AO.OutInBuffer = 1;
00719     if ( numfac == 0 ) { /* write the number d->nfactors */
00720         n[0] = 4; n[1] = d->nfactors; n[2] = 1; n[3] = 3; n[4] = 0; t = n;
00721     }
00722     else if ( numfac == 1 && d->factors == 0 ) { /* Here d->factors is zero and d->where is fine */
00723         t = d->where;
00724     }
00725     else if ( d->factors[numfac-1].where == 0 ) { /* write the value */
00726         if ( d->factors[numfac-1].value < 0 ) {
00727             n[0] = 4; n[1] = -d->factors[numfac-1].value; n[2] = 1; n[3] = -3; n[4] = 0; t = n;
00728         }
00729         else {
00730             n[0] = 4; n[1] = d->factors[numfac-1].value; n[2] = 1; n[3] = 3; n[4] = 0; t = n;
00731         }
00732     }
00733     else { t = d->factors[numfac-1].where; }
00734     while ( *t ) {
00735         if ( WriteTerm(t,&lbrac,first,PRINTON,0) ) {
00736             error = 1; break;
00737         }
00738         t += *t;
00739     }
00740     AC.OutputMode = oldOutputMode;
00741     AO.OutInBuffer = 0;
00742     AO.InFbrack = oldinfbrack;
00743     AO.CurBufWrt = oldcurbufwrt;
00744     AO.CurrentDictionary = dict;
00745     if ( error ) {
00746         MLOCK(ErrorMessageLock);
00747         MesPrint("&Illegal dollar object for writing");
00748         MUNLOCK(ErrorMessageLock);
00749         M_free(AO.DollarOutBuffer,"DollarOutBuffer");
00750         AO.DollarOutBuffer = 0;
00751         AO.DollarOutSizeBuffer = 0;
00752         return(0);
00753     }
00754     return(AO.DollarOutBuffer);
00755 }
00756
00757 /*
00758 #] WriteDollarFactorToBuffer :
00759 #[ AddToDollarBuffer :
00760 */
00761
00762 void AddToDollarBuffer(UBYTE *s)
00763 {
00764     int i;
00765     UBYTE *t = s, *u, *newdob;
00766     LONG j;
00767     while ( *t ) { t++; }
00768     i = t - s;
00769     while ( i + AO.DollarInOutBuffer >= AO.DollarOutSizeBuffer ) {
00770         j = AO.DollarInOutBuffer;
00771         AO.DollarOutSizeBuffer -= 2;
00772         t = AO.DollarOutBuffer;
00773         newdob = (UBYTE *)Malloc1(AO.DollarOutSizeBuffer,"DollarOutBuffer");
00774         u = newdob;
00775         while ( --j >= 0 ) *u++ = *t++;
00776         M_free(AO.DollarOutBuffer,"DollarOutBuffer");
00777         AO.DollarOutBuffer = newdob;
00778     }
00779     t = AO.DollarOutBuffer + AO.DollarInOutBuffer-1;
00780     while ( t == AO.DollarOutBuffer && ( *s == '+' || *s == ' ' ) ) s++;
00781     i = 0;

```

```

00782     if ( AO.CurrentDictionary == 0 ) {
00783         while ( *s ) {
00784             if ( *s == ' ' ) { s++; continue; }
00785             *t++ = *s++; i++;
00786         }
00787     }
00788     else {
00789         while ( *s ) { *t++ = *s++; i++; }
00790     }
00791     *t = 0;
00792     AO.DollarInOutBuffer += i;
00793 }
00794
00795 /*
00796 #] AddToDollarBuffer :
00797 #[ TermAssign :
00798
00799 This routine is called from a piece of code in Normalize that has been
00800 commented out.
00801 */
00802
00803 void TermAssign(WORD *term)
00804 {
00805     DOLLARS d;
00806     WORD *t, *tstop, *astop, *w, *m;
00807     WORD i, newsize;
00808     for (;;) {
00809         astop = term + *term;
00810         tstop = astop - ABS(astop[-1]);
00811         t = term + 1;
00812         while ( t < tstop ) {
00813             if ( *t == AM.termfunnum && t[1] == FUNHEAD+2
00814                 && t[FUNHEAD] == -DOLLAREXPRESSION ) {
00815                 d = Dollars + t[FUNHEAD+1];
00816                 newsize = *term - FUNHEAD - 1;
00817                 if ( newsize < MINALLOC ) newsize = MINALLOC;
00818                 newsize = (newsize+7)/8*8;
00819                 if ( d->size > 2*newsize && d->size > 1000 ) {
00820                     if ( d->where && d->where != &(AM.dollarzero) ) M_free(d->where,"dollar
00821 contents");
00822                     d->size = 0;
00823                     d->where = &(AM.dollarzero);
00824                 }
00825                 if ( d->size < newsize ) {
00826                     if ( d->where && d->where != &(AM.dollarzero) ) M_free(d->where,"dollar
00827 contents");
00828                     d->size = newsize;
00829                     d->where = (WORD *)Malloc1(newsize*sizeof(WORD),"dollar contents");
00830                 }
00831                 cbuf[AM.dbufnum].rhs[t[FUNHEAD+1]] = w = d->where;
00832                 m = term;
00833                 while ( m < t ) *w++ = *m++;
00834                 m += t[1];
00835                 while ( m < tstop ) {
00836                     if ( *m == AM.termfunnum && m[1] == FUNHEAD+2
00837                         && m[FUNHEAD] == -DOLLAREXPRESSION ) { m += m[1]; }
00838                     else {
00839                         i = m[1];
00840                         while ( --i >= 0 ) *w++ = *m++;
00841                     }
00842                 }
00843                 while ( m < astop ) *w++ = *m++;
00844                 *(d->where) = w - d->where;
00845                 *w = 0;
00846                 d->type = DOLTERMS;
00847                 w = t; m = t + t[1];
00848                 while ( m < astop ) *w++ = *m++;
00849                 *term = w - term;
00850                 break;
00851             }
00852             t += t[1];
00853         }
00854         if ( t >= tstop ) return;
00855     }
00856 }
00857 /*
00858 #] TermAssign :
00859 #[ PutTermInDollar :
00860
00861 We assume here that the dollar is local.
00862 */
00863 int PutTermInDollar(WORD *term, WORD numdollar)
00864 {
00865     DOLLARS d = Dollars+numdollar;
00866     WORD i;

```



```

00867     if ( term == 0 || *term == 0 ) {
00868         d->type = DOLZERO;
00869         return(0);
00870     }
00871     if ( d->size < *term || d->size > 2*term[0] || d->where == 0 ) {
00872         if ( d->size > 0 && d->where ) {
00873             M_free(d->where,"dollar contents");
00874         }
00875         d->where = Malloc1((term[0]+1)*sizeof(WORD),"dollar contents");
00876         d->size = term[0]+1;
00877     }
00878     d->type = DOLTERMS;
00879     for ( i = 0; i < term[0]; i++ ) d->where[i] = term[i];
00880     d->where[i] = 0;
00881     return(0);
00882 }
00883
00884 /*
00885  #] PutTermInDollar :
00886  #[ WildDollars :
00887
00888      Note that we cannot upload wildcards into dollar variables when WITHPTHREADS.
00889  */
00890
00891 void WildDollars(PHEAD WORD *term)
00892 {
00893     GETBIDENTITY
00894     DOLLARS d;
00895     WORD *m, *t, *w, *ww, *orig = 0, *wildvalue, *wildstop;
00896     int numdollar;
00897     LONG weneed, i;
00898     #ifdef WITHPTHREADS
00899         int dtype = -1;
00900     #endif
00901     if ( term == 0 ) {
00902         m = wildvalue = AN.WildValue;
00903         wildstop = AN.WildStop;
00904     }
00905     else {
00906         ww = term + *term; ww -= ABS(ww[-1]); w = term+1;
00907         while ( w < ww && *w != SUBEXPRESSION ) w += w[1];
00908         if ( w >= ww ) return;
00909         wildstop = w + w[1];
00910         w += SUBEXPSIZE;
00911         wildvalue = m = w;
00912     }
00913     while ( m < wildstop ) {
00914         if ( *m != LOADDOLLAR ) { m += m[1]; continue; }
00915         t = m - 4;
00916         while ( *t == LOADDOLLAR || *t == FROMSET || *t == SETTONUM ) t -= 4;
00917         if ( t < wildvalue ) {
00918             MLOCK(ErrorMessageLock);
00919             MesPrint("&Serious bug in wildcard prototype. Found in WildDollars");
00920             MUNLOCK(ErrorMessageLock);
00921             Terminate(-1);
00922         }
00923         numdollar = m[2];
00924         d = Dollars + numdollar;
00925         #ifdef WITHPTHREADS
00926         {
00927             int nummodopt;
00928             dtype = -1;
00929             if ( AS.MultiThreaded && ( AC.mparallelflag == PARALLELFLAG ) ) {
00930                 for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
00931                     if ( numdollar == ModOptdollars[nummodopt].number ) break;
00932                 }
00933                 if ( nummodopt < NumModOptdollars ) {
00934                     dtype = ModOptdollars[nummodopt].type;
00935                     if ( dtype == MODLOCAL ) {
00936                         d = ModOptdollars[nummodopt].dstruct+AT.identity;
00937                     }
00938                     else {
00939                         MLOCK(ErrorMessageLock);
00940                         MesPrint("&Illegal attempt to use $-variable %s in module %1",
00941                             DOLLARNAME(Dollars,numdollar),AC.CModule);
00942                         MUNLOCK(ErrorMessageLock);
00943                         Terminate(-1);
00944                     }
00945                 }
00946             }
00947         }
00948         #endif
00949         /*
00950          The value of this wildcard goes into our $-variable
00951          First compute the space we need.
00952          */
00953         switch ( *t ) {

```

```

00954         case SYMTONUM:
00955             weneed = 5;
00956             break;
00957         case SYMTOSYM:
00958             weneed = 9;
00959             break;
00960         case SYMTOSUB:
00961         case VECTOSUB:
00962         case INDTOSUB:
00963             orig = cbuf[AT.ebufnum].rhs[t[3]];
00964             w = orig; while ( *w ) w += *w;
00965             weneed = w - orig + 1;
00966             break;
00967         case VECTOMIN:
00968         case VECTOVEC:
00969         case INDTOIND:
00970             weneed = 8;
00971             break;
00972         case FUNTOFUN:
00973             weneed = FUNHEAD+5;
00974             break;
00975         case ARGTOARG:
00976             orig = cbuf[AT.ebufnum].rhs[t[3]];
00977             if ( *orig > 0 ) weneed = *orig+2;
00978             else {
00979                 w = orig+1; while ( *w ) { NEXTARG(w) }
00980                 weneed = w - orig + 1;
00981             }
00982             break;
00983         default:
00984             weneed = MINALLOC;
00985             break;
00986     }
00987     if ( weneed < MINALLOC ) weneed = MINALLOC;
00988     weneed = ((weneed+7)/8)*8;
00989     if ( d->size > 2*weneed && d->size > 1000 ) {
00990         if ( d->where && d->where != &(AM.dollarzero) ) M_free(d->where,"dollarspace");
00991         d->where = &(AM.dollarzero);
00992         d->size = 0;
00993     }
00994     if ( d->size < weneed ) {
00995         if ( d->where && d->where != &(AM.dollarzero) ) M_free(d->where,"dollarspace");
00996         d->where = (WORD *)Malloc1(weneed*sizeof(WORD),"dollarspace");
00997         d->size = weneed;
00998     }
00999 /*
01000     It is not clear what the following code does for TFORM
01001
01002     if ( dtype != MODLOCAL ) {
01003 /*
01004         cbuf[AM.dbufnum].CanCommu[numdollar] = 0;
01005         cbuf[AM.dbufnum].NumTerms[numdollar] = 1;
01006 /*
01007         cbuf[AM.dbufnum].rhs[numdollar] = d->where; */
01008         cbuf[AM.dbufnum].rhs[numdollar] = (WORD *) (1);
01009     }
01010     Now load up the value of the wildcard in compiler buffer format
01011 */
01012     w = d->where;
01013     d->type = DOLTERMS;
01014     switch ( *t ) {
01015         case SYMTONUM:
01016             d->where[0] = 4; d->where[2] = 1;
01017             if ( t[3] >= 0 ) { d->where[1] = t[3]; d->where[3] = 3; }
01018             else { d->where[1] = -t[3]; d->where[3] = -3; }
01019             if ( t[3] == 0 ) { d->type = DOLZERO; d->where[0] = 0; }
01020             else { d->type = DOLNUMBER; d->where[4] = 0; }
01021             break;
01022         case SYMTOSYM:
01023             *w++ = 8;
01024             *w++ = SYMBOL;
01025             *w++ = 4;
01026             *w++ = t[3];
01027             *w++ = 1;
01028             *w++ = 1;
01029             *w++ = 1;
01030             *w++ = 3;
01031             *w = 0;
01032             break;
01033         case SYMTOSUB:
01034         case VECTOSUB:
01035         case INDTOSUB:
01036             while ( *orig ) {
01037                 i = *orig; while ( --i >= 0 ) *w++ = *orig++;
01038             }
01039             *w = 0;
01040 /*

```

```

01041             And then we have to fix up CanCommu
01042  */
01043             break;
01044         case VECTOMIN:
01045             *w++ = 7; *w++ = INDEX; *w++ = 3; *w++ = t[3];
01046             *w++ = 1; *w++ = 1; *w++ = -3; *w = 0;
01047             break;
01048         case VECTOVEC:
01049             *w++ = 7; *w++ = INDEX; *w++ = 3; *w++ = t[3];
01050             *w++ = 1; *w++ = 1; *w++ = 3; *w = 0;
01051             break;
01052         case INDTOIND:
01053             d->type = DOLINDEX; d->index = t[3]; *w = 0;
01054             break;
01055         case FUNTOFUN:
01056             *w++ = FUNHEAD+4; *w++ = t[3]; *w++ = FUNHEAD;
01057             FILLFUN(w)
01058             *w++ = 1; *w++ = 1; *w++ = 3; *w = 0;
01059             break;
01060         case ARGTOARG:
01061             if ( *orig > 0 ) ww = orig + *orig + 1;
01062             else {
01063                 ww = orig+1; while ( *ww ) { NEXTARG(ww) }
01064             }
01065             while ( orig < ww ) *w++ = *orig++;
01066             *w = 0;
01067             d->type = DOLWILDARGS;
01068             break;
01069         default:
01070             d->type = DOLUNDEFINED;
01071             break;
01072     }
01073     m += m[1];
01074 }
01075 }
01076
01077 /*
01078  #] WildDollars :
01079  #[ DolToTensor :      with LOCK
01080  */
01081
01082 WORD DolToTensor(PHEAD WORD numdollar)
01083 {
01084     GETBIDENTITY
01085     DOLLARS d = Dollars + numdollar;
01086     WORD retval;
01087     #ifdef WITHPTHREADS
01088     int nummodopt, dtype = -1;
01089     if ( AS.MultiThreaded && ( AC.mparallelflag == PARALLELFLAG ) ) {
01090         for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
01091             if ( numdollar == ModOptdollars[nummodopt].number ) break;
01092         }
01093         if ( nummodopt < NumModOptdollars ) {
01094             dtype = ModOptdollars[nummodopt].type;
01095             if ( dtype == MODLOCAL ) {
01096                 d = ModOptdollars[nummodopt].dstruct+AT.identity;
01097             }
01098             else {
01099                 LOCK(d->pthreadlockread);
01100             }
01101         }
01102     }
01103     #endif
01104     AN.ErrorInDollar = 0;
01105     if ( d->type == DOLTERMS && d->where[0] == FUNHEAD+4 &&
01106         d->where[FUNHEAD+4] == 0 && d->where[FUNHEAD+3] == 3 &&
01107         d->where[FUNHEAD+2] == 1 && d->where[FUNHEAD+1] == 1 &&
01108         d->where[1] >= FUNCTION && d->where[1] < FUNCTION+WILDOFFSET
01109         && functions[d->where[1]-FUNCTION].spec >= TENSORFUNCTION ) {
01110         retval = d->where[1];
01111     }
01112     else if ( d->type == DOLARGUMENT &&
01113         d->where[0] <= -FUNCTION && d->where[0] > -FUNCTION-WILDOFFSET
01114         && functions[-d->where[0]-FUNCTION].spec >= TENSORFUNCTION ) {
01115         retval = -d->where[0];
01116     }
01117     else if ( d->type == DOLWILDARGS && d->where[0] == 0
01118         && d->where[1] <= -FUNCTION && d->where[1] > -FUNCTION-WILDOFFSET
01119         && d->where[2] == 0
01120         && functions[-d->where[1]-FUNCTION].spec >= TENSORFUNCTION ) {
01121         retval = -d->where[1];
01122     }
01123     else if ( d->type == DOLSUBTERM &&
01124         d->where[0] >= FUNCTION && d->where[0] < FUNCTION+WILDOFFSET
01125         && functions[d->where[0]-FUNCTION].spec >= TENSORFUNCTION ) {
01126         retval = d->where[0];
01127     }

```

```

01128     else {
01129         AN.ErrorInDollar = 1;
01130         retval = 0;
01131     }
01132 #ifdef WITHPTHREADS
01133     if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
01134 #endif
01135     return(retval);
01136 }
01137
01138 /*
01139 #] DolToTensor :
01140 #[ DolToFunction : with LOCK
01141 */
01142
01143 WORD DolToFunction(PHEAD WORD numdollar)
01144 {
01145     GETBIDENTITY
01146     DOLLARS d = Dollars + numdollar;
01147     WORD retval;
01148 #ifdef WITHPTHREADS
01149     int nummodopt, dtype = -1;
01150     if ( AS.MultiThreaded && ( AC.mparallelflag == PARALLELFLAG ) ) {
01151         for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
01152             if ( numdollar == ModOptdollars[nummodopt].number ) break;
01153         }
01154         if ( nummodopt < NumModOptdollars ) {
01155             dtype = ModOptdollars[nummodopt].type;
01156             if ( dtype == MODLOCAL ) {
01157                 d = ModOptdollars[nummodopt].dstruct+AT.identity;
01158             }
01159             else {
01160                 LOCK(d->pthreadlockread);
01161             }
01162         }
01163     }
01164 #endif
01165     AN.ErrorInDollar = 0;
01166     if ( d->type == DOLTERMS && d->where[0] == FUNHEAD+4 &&
01167         d->where[FUNHEAD+4] == 0 && d->where[FUNHEAD+3] == 3 &&
01168         d->where[FUNHEAD+2] == 1 && d->where[FUNHEAD+1] == 1 &&
01169         d->where[1] >= FUNCTION && d->where[1] < FUNCTION+WILDOFFSET ) {
01170         retval = d->where[1];
01171     }
01172     else if ( d->type == DOLARGUMENT &&
01173         d->where[0] <= -FUNCTION && d->where[0] > -FUNCTION-WILDOFFSET ) {
01174         retval = -d->where[0];
01175     }
01176     else if ( d->type == DOLWILDARGS && d->where[0] == 0
01177         && d->where[1] <= -FUNCTION && d->where[1] > -FUNCTION-WILDOFFSET
01178         && d->where[2] == 0 ) {
01179         retval = -d->where[1];
01180     }
01181     else if ( d->type == DOLSUBTERM &&
01182         d->where[0] >= FUNCTION && d->where[0] < FUNCTION+WILDOFFSET ) {
01183         retval = d->where[0];
01184     }
01185     else {
01186         AN.ErrorInDollar = 1;
01187         retval = 0;
01188     }
01189 #ifdef WITHPTHREADS
01190     if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
01191 #endif
01192     return(retval);
01193 }
01194
01195 /*
01196 #] DolToFunction :
01197 #[ DolToVector : with LOCK
01198 */
01199
01200 WORD DolToVector(PHEAD WORD numdollar)
01201 {
01202     GETBIDENTITY
01203     DOLLARS d = Dollars + numdollar;
01204     WORD retval;
01205 #ifdef WITHPTHREADS
01206     int nummodopt, dtype = -1;
01207     if ( AS.MultiThreaded && ( AC.mparallelflag == PARALLELFLAG ) ) {
01208         for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
01209             if ( numdollar == ModOptdollars[nummodopt].number ) break;
01210         }
01211         if ( nummodopt < NumModOptdollars ) {
01212             dtype = ModOptdollars[nummodopt].type;
01213             if ( dtype == MODLOCAL ) {
01214                 d = ModOptdollars[nummodopt].dstruct+AT.identity;

```

```

01215         }
01216         else {
01217             LOCK(d->pthreadlockread);
01218         }
01219     }
01220 }
01221 #endif
01222 AN.ErrorInDollar = 0;
01223 if ( d->type == DOLINDEX && d->index < 0 ) {
01224     retval = d->index;
01225 }
01226 else if ( d->type == DOLARGUMENT && ( d->where[0] == -VECTOR
01227 || d->where[0] == -MINVECTOR ) ) {
01228     retval = d->where[1];
01229 }
01230 else if ( d->type == DOLSUBTERM && d->where[0] == INDEX
01231 && d->where[1] == 3 && d->where[2] < 0 ) {
01232     retval = d->where[2];
01233 }
01234 else if ( d->type == DOLTERMS && d->where[0] == 7 &&
01235 d->where[7] == 0 && d->where[6] == 3 &&
01236 d->where[5] == 1 && d->where[4] == 1 &&
01237 d->where[1] >= INDEX && d->where[3] < 0 ) {
01238     retval = d->where[3];
01239 }
01240 else if ( d->type == DOLWILDARGS && d->where[0] == 0
01241 && ( d->where[1] == -VECTOR || d->where[1] == -MINVECTOR )
01242 && d->where[3] == 0 ) {
01243     retval = d->where[2];
01244 }
01245 else if ( d->type == DOLWILDARGS && d->where[0] == 1
01246 && d->where[1] < 0 ) {
01247     retval = d->where[1];
01248 }
01249 else {
01250     AN.ErrorInDollar = 1;
01251     retval = 0;
01252 }
01253 #ifdef WITHPTHREADS
01254 if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
01255 #endif
01256 return(retval);
01257 }
01258
01259 /*
01260 #] DolToVector :
01261 #[ DolToNumber :
01262 */
01263
01264 WORD DolToNumber(PHEAD WORD numdollar)
01265 {
01266     GETBIDENTITY
01267     DOLLARS d = Dollars + numdollar;
01268 #ifdef WITHPTHREADS
01269     int nummodopt, dtype = -1;
01270     if ( AS.MultiThreaded && ( AC.mparallelfld == PARALLELFIELD ) ) {
01271         for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
01272             if ( numdollar == ModOptdollars[nummodopt].number ) break;
01273         }
01274         if ( nummodopt < NumModOptdollars ) {
01275             dtype = ModOptdollars[nummodopt].type;
01276             if ( dtype == MODLOCAL ) {
01277                 d = ModOptdollars[nummodopt].dstruct+AT.identity;
01278             }
01279         }
01280     }
01281 #endif
01282 AN.ErrorInDollar = 0;
01283 if ( ( d->type == DOLTERMS || d->type == DOLNUMBER )
01284 && d->where[0] == 4 &&
01285 d->where[4] == 0 && ( d->where[3] == 3 || d->where[3] == -3 )
01286 && d->where[2] == 1 && ( d->where[1] & TOPBITONLY ) == 0 ) {
01287     if ( d->where[3] > 0 ) return(d->where[1]);
01288     else return(-d->where[1]);
01289 }
01290 else if ( d->type == DOLARGUMENT && d->where[0] == -SNUMBER ) {
01291     return(d->where[1]);
01292 }
01293 else if ( d->type == DOLARGUMENT && d->where[0] == -INDEX
01294 && d->where[1] >= 0 && d->where[1] < AM.OffsetIndex ) {
01295     return(d->where[1]);
01296 }
01297 else if ( d->type == DOLZERO ) return(0);
01298 else if ( d->type == DOLWILDARGS && d->where[0] == 0
01299 && d->where[1] == -SNUMBER && d->where[3] == 0 ) {
01300     return(d->where[2]);
01301 }

```

```

01302     else if ( d->type == DOLINDEX && d->index >= 0 && d->index < AM.OffsetIndex ) {
01303         return(d->index);
01304     }
01305     else if ( d->type == DOLWILDARGS && d->where[0] == 1
01306     && d->where[1] >= 0 && d->where[1] < AM.OffsetIndex ) {
01307         return(d->where[1]);
01308     }
01309     else if ( d->type == DOLWILDARGS && d->where[0] == 0
01310     && d->where[1] == -INDEX && d->where[3] == 0 && d->where[2] >= 0
01311     && d->where[2] < AM.OffsetIndex ) {
01312         return(d->where[2]);
01313     }
01314     AN.ErrorInDollar = 1;
01315     return(0);
01316 }
01317
01318 /*
01319 #] DolToNumber :
01320 #[ DolToSymbol :      with LOCK
01321 */
01322
01323 WORD DolToSymbol(PHEAD WORD numdollar)
01324 {
01325     GETBIDENTITY
01326     DOLLARS d = Dollars + numdollar;
01327     WORD retval;
01328     #ifdef WITHPTHREADS
01329     int nummodopt, dtype = -1;
01330     if ( AS.MultiThreaded && ( AC.mparallelflag == PARALLELFLAG ) ) {
01331         for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
01332             if ( numdollar == ModOptdollars[nummodopt].number ) break;
01333         }
01334         if ( nummodopt < NumModOptdollars ) {
01335             dtype = ModOptdollars[nummodopt].type;
01336             if ( dtype == MODLOCAL ) {
01337                 d = ModOptdollars[nummodopt].dstruct+AT.identity;
01338             }
01339             else {
01340                 LOCK(d->pthreadlockread);
01341             }
01342         }
01343     }
01344     #endif
01345     AN.ErrorInDollar = 0;
01346     if ( d->type == DOLTERMS && d->where[0] == 8 &&
01347     d->where[8] == 0 && d->where[7] == 3 && d->where[6] == 1
01348     && d->where[5] == 1 && d->where[4] == 1 && d->where[1] == SYMBOL ) {
01349         retval = d->where[3];
01350     }
01351     else if ( d->type == DOLARGUMENT && d->where[0] == -SYMBOL ) {
01352         retval = d->where[1];
01353     }
01354     else if ( d->type == DOLSUBTERM && d->where[0] == SYMBOL
01355     && d->where[1] == 4 && d->where[3] == 1 ) {
01356         retval = d->where[2];
01357     }
01358     else if ( d->type == DOLWILDARGS && d->where[0] == 0
01359     && d->where[1] == -SYMBOL && d->where[3] == 0 ) {
01360         retval = d->where[2];
01361     }
01362     else {
01363         AN.ErrorInDollar = 1;
01364         retval = -1;
01365     }
01366     #ifdef WITHPTHREADS
01367     if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
01368     #endif
01369     return(retval);
01370 }
01371
01372 /*
01373 #] DolToSymbol :
01374 #[ DolToIndex :      with LOCK
01375 */
01376
01377 WORD DolToIndex(PHEAD WORD numdollar)
01378 {
01379     GETBIDENTITY
01380     DOLLARS d = Dollars + numdollar;
01381     WORD retval;
01382     #ifdef WITHPTHREADS
01383     int nummodopt, dtype = -1;
01384     if ( AS.MultiThreaded && ( AC.mparallelflag == PARALLELFLAG ) ) {
01385         for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
01386             if ( numdollar == ModOptdollars[nummodopt].number ) break;
01387         }
01388         if ( nummodopt < NumModOptdollars ) {

```

```

01389         dtype = ModOptdollars[nummodopt].type;
01390         if ( dtype == MODLOCAL ) {
01391             d = ModOptdollars[nummodopt].dstruct+AT.identity;
01392         }
01393         else {
01394             LOCK(d->pthreadlockread);
01395         }
01396     }
01397 }
01398 #endif
01399 AN.ErrorInDollar = 0;
01400 if ( d->type == DOLTERMS && d->where[0] == 7 &&
01401     d->where[7] == 0 && d->where[6] == 3 && d->where[5] == 1
01402     && d->where[4] == 1 && d->where[1] == INDEX && d->where[3] >= 0 ) {
01403     retval = d->where[3];
01404 }
01405 else if ( d->type == DOLARGUMENT && d->where[0] == -SNUMBER
01406     && d->where[1] >= 0 && d->where[1] < AM.OffsetIndex ) {
01407     retval = d->where[1];
01408 }
01409 else if ( d->type == DOLARGUMENT && d->where[0] == -INDEX
01410     && d->where[1] >= 0 ) {
01411     retval = d->where[1];
01412 }
01413 else if ( d->type == DOLZERO ) return(0);
01414 else if ( d->type == DOLWILDARGS && d->where[0] == 0
01415     && d->where[1] == -SNUMBER && d->where[3] == 0 && d->where[2] >= 0
01416     && d->where[2] < AM.OffsetIndex ) {
01417     retval = d->where[2];
01418 }
01419 else if ( d->type == DOLINDEX && d->index >= 0 ) {
01420     retval = d->index;
01421 }
01422 else if ( d->type == DOLNUMBER && d->where[0] == 4 && d->where[2] == 1
01423     && d->where[3] == 3 && d->where[4] == 0 && d->where[1] < AM.OffsetIndex ) {
01424     retval = d->where[1];
01425 }
01426 else if ( d->type == DOLWILDARGS && d->where[0] == 1
01427     && d->where[1] >= 0 ) {
01428     retval = d->where[1];
01429 }
01430 else if ( d->type == DOLSUBTERM && d->where[0] == INDEX
01431     && d->where[1] == 3 && d->where[2] >= 0 ) {
01432     retval = d->where[2];
01433 }
01434 else if ( d->type == DOLWILDARGS && d->where[0] == 0
01435     && d->where[1] == -INDEX && d->where[3] == 0 && d->where[2] >= 0 ) {
01436     retval = d->where[2];
01437 }
01438 else {
01439     AN.ErrorInDollar = 1;
01440     retval = 0;
01441 }
01442 #ifdef WITHPTHREADS
01443 if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
01444 #endif
01445 return(retval);
01446 }
01447
01448 /*
01449 #] DolToIndex :
01450 #[ DolToTerms :
01451
01452 Returns a struct of type DOLLARS which contains a copy of the
01453 original dollar variable, provided it can be expressed in terms of
01454 an expression (type = DOLTERMS). Otherwise it returns zero.
01455 The dollar is expressed in terms in the buffer "where"
01456 */
01457
01458 DOLLARS DolToTerms(PHEAD WORD numdollar)
01459 {
01460     GETBIDENTITY
01461     LONG size;
01462     DOLLARS d = Dollars + numdollar, newd;
01463     WORD *t, *w, i;
01464 #ifdef WITHPTHREADS
01465     int nummodopt, dtype = -1;
01466     if ( AS.MultiThreaded && ( AC.mparallelflag == PARALLELFLAG ) ) {
01467         for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
01468             if ( numdollar == ModOptdollars[nummodopt].number ) break;
01469         }
01470         if ( nummodopt < NumModOptdollars ) {
01471             dtype = ModOptdollars[nummodopt].type;
01472             if ( dtype == MODLOCAL ) {
01473                 d = ModOptdollars[nummodopt].dstruct+AT.identity;
01474             }
01475         }

```

```

01476     }
01477 #endif
01478     AN.ErrorInDollar = 0;
01479     switch ( d->type ) {
01480     case DOLARGUMENT:
01481         t = d->where;
01482         if ( t[0] < 0 ) {
01483 ShortArgument:
01484             w = AT.WorkPointer;
01485             if ( t[0] <= -FUNCTION ) {
01486                 *w++ = FUNHEAD+4; *w++ = -t[0];
01487                 *w++ = FUNHEAD; FILLFUN(w)
01488                 *w++ = 1; *w++ = 1; *w++ = 3;
01489             }
01490             else if ( t[0] == -SYMBOL ) {
01491                 *w++ = 8; *w++ = SYMBOL; *w++ = 4; *w++ = t[1];
01492                 *w++ = 1; *w++ = 1; *w++ = 1; *w++ = 3;
01493             }
01494             else if ( t[0] == -VECTOR || t[0] == -INDEX ) {
01495                 *w++ = 7; *w++ = INDEX; *w++ = 3; *w++ = t[1];
01496                 *w++ = 1; *w++ = 1; *w++ = 3;
01497             }
01498             else if ( t[0] == -MINVECTOR ) {
01499                 *w++ = 7; *w++ = INDEX; *w++ = 3; *w++ = t[1];
01500                 *w++ = 1; *w++ = 1; *w++ = -3;
01501             }
01502             else if ( t[0] == -SNUMBER ) {
01503                 *w++ = 4;
01504                 if ( t[1] < 0 ) {
01505                     *w++ = -t[1]; *w++ = 1; *w++ = -3;
01506                 }
01507                 else {
01508                     *w++ = t[1]; *w++ = 1; *w++ = 3;
01509                 }
01510             }
01511             *w = 0; size = w - AT.WorkPointer;
01512             w = AT.WorkPointer;
01513             break;
01514         }
01515         /* fall through */
01516     case DOLNUMBER:
01517     case DOLTERMS:
01518         t = d->where;
01519         while ( *t ) t += *t;
01520         size = t - d->where;
01521         w = d->where;
01522         break;
01523     case DOLSUBTERM:
01524         w = AT.WorkPointer;
01525         size = d->where[1];
01526         *w++ = size+4; t = d->where; NCOPY(w,t,size)
01527         *w++ = 1; *w++ = 1; *w++ = 3;
01528         w = AT.WorkPointer; size = d->where[1]+4;
01529         break;
01530     case DOLINDEX:
01531         w = AT.WorkPointer;
01532         *w++ = 7; *w++ = INDEX; *w++ = 3; *w++ = d->index;
01533         *w++ = 1; *w++ = 1; *w++ = 3; *w = 0;
01534         w = AT.WorkPointer; size = 7;
01535         break;
01536     case DOLWILDARGS:
01537     /*
01538         In some cases we can make a copy
01539     */
01540         t = d->where+1;
01541         if ( *t == 0 ) return(0);
01542         NEXTARG(t);
01543         if ( *t ) { /* More than one argument in here */
01544             MLOCK(ErrorMessageLock);
01545             MesPrint("Trying to convert a $ with an argument field into an expression");
01546             MUNLOCK(ErrorMessageLock);
01547             Terminate(-1);
01548         }
01549     /*
01550         Now we have a single argument
01551     */
01552         t = d->where+1;
01553         if ( *t < 0 ) goto ShortArgument;
01554         size = *t - ARGHEAD;
01555         w = t + ARGHEAD;
01556         break;
01557     case DOLUNDEFINED:
01558         MLOCK(ErrorMessageLock);
01559         MesPrint("Trying to use an undefined $ in an expression");
01560         MUNLOCK(ErrorMessageLock);
01561         Terminate(-1);
01562         /* fall through */

```



```

01563         case DOLZERO:
01564             if ( d->where ) { d->where[0] = 0; }
01565             else d->where = &(AM.dollarzero);
01566             size = 0;
01567             w = d->where;
01568             break;
01569         default:
01570             return(0);
01571     }
01572     newd = (DOLLARS)Malloc1(sizeof(struct DoLLaRS)+(size+1)*sizeof(WORD),
01573         "Copy of dollar variable");
01574     t = (WORD *) (newd+1);
01575     newd->where = t;
01576     newd->name = d->name;
01577     newd->node = d->node;
01578     newd->type = DOLTERMS;
01579     newd->size = size;
01580     newd->numdummies = d->numdummies;
01581 #ifdef WITHPTHREADS
01582     newd->pthreadlockread = dummylock;
01583     newd->pthreadlockwrite = dummylock;
01584 #endif
01585     size++;
01586     NCOPY(t,w,size);
01587     newd->nfactors = d->nfactors;
01588     if ( d->nfactors > 1 ) {
01589         newd->factors = (FACDOLLAR *)Malloc1(d->nfactors*sizeof(FACDOLLAR),"Dollar factors");
01590         for ( i = 0; i < d->nfactors; i++ ) {
01591             newd->factors[i].where = 0;
01592             newd->factors[i].size = 0;
01593             newd->factors[i].type = DOLUNDEFINED;
01594             newd->factors[i].value = d->factors[i].value;
01595         }
01596     }
01597     else { newd->factors = 0; }
01598     return(newd);
01599 }
01600
01601 /*
01602  #] DolToTerms :
01603  #[ DolToLong :
01604  */
01605
01606 LONG DolToLong(PHEAD WORD numdollar)
01607 {
01608     GETBIDENTITY
01609     DOLLARS d = Dollars + numdollar;
01610     LONG x;
01611 #ifdef WITHPTHREADS
01612     int nummodopt, dtype = -1;
01613     if ( AS.MultiThreaded && ( AC.mparallelflag == PARALLELFLAG ) ) {
01614         for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
01615             if ( numdollar == ModOptdollars[nummodopt].number ) break;
01616         }
01617         if ( nummodopt < NumModOptdollars ) {
01618             dtype = ModOptdollars[nummodopt].type;
01619             if ( dtype == MODLOCAL ) {
01620                 d = ModOptdollars[nummodopt].dstruct+AT.identity;
01621             }
01622         }
01623     }
01624 #endif
01625     AN.ErrorInDollar = 0;
01626     if ( ( d->type == DOLTERMS || d->type == DOLNUMBER )
01627         && d->where[0] == 4 &&
01628         d->where[4] == 0 && ( d->where[3] == 3 || d->where[3] == -3 )
01629         && d->where[2] == 1 && ( d->where[1] & TOPBITONLY ) == 0 ) {
01630         x = d->where[1];
01631         if ( d->where[3] > 0 ) return(x);
01632         else return(-x);
01633     }
01634     else if ( ( d->type == DOLTERMS || d->type == DOLNUMBER )
01635         && d->where[0] == 6 &&
01636         d->where[6] == 0 && ( d->where[5] == 5 || d->where[5] == -5 )
01637         && d->where[3] == 1 && d->where[4] == 1 && ( d->where[2] & TOPBITONLY ) == 0 ) {
01638         x = d->where[1] + ( (LONG) (d->where[2]) << BITSINWORD );
01639         if ( d->where[5] > 0 ) return(x);
01640         else return(-x);
01641     }
01642     else if ( d->type == DOLARGUMENT && d->where[0] == -SNUMBER ) {
01643         x = d->where[1];
01644         return(x);
01645     }
01646     else if ( d->type == DOLARGUMENT && d->where[0] == -INDEX
01647         && d->where[1] >= 0 && d->where[1] < AM.OffsetIndex ) {
01648         x = d->where[1];
01649         return(x);

```

```

01650     }
01651     else if ( d->type == DOLZERO ) return(0);
01652     else if ( d->type == DOLWILDARGS && d->where[0] == 0
01653     && d->where[1] == -SNUMBER && d->where[3] == 0 ) {
01654         x = d->where[2];
01655         return(x);
01656     }
01657     else if ( d->type == DOLINDEX && d->index >= 0 && d->index < AM.OffsetIndex ) {
01658         x = d->index;
01659         return(x);
01660     }
01661     else if ( d->type == DOLWILDARGS && d->where[0] == 1
01662     && d->where[1] >= 0 && d->where[1] < AM.OffsetIndex ) {
01663         x = d->where[1];
01664         return(x);
01665     }
01666     else if ( d->type == DOLWILDARGS && d->where[0] == 0
01667     && d->where[1] == -INDEX && d->where[3] == 0 && d->where[2] >= 0
01668     && d->where[2] < AM.OffsetIndex ) {
01669         x = d->where[2];
01670         return(x);
01671     }
01672     AN.ErrorInDollar = 1;
01673     return(0);
01674 }
01675
01676 /*
01677  #] DolToLong :
01678  #[ ExecInside :
01679  */
01680
01681 int ExecInside(UBYTE *s)
01682 {
01683     GETIDENTITY
01684     UBYTE *t, c;
01685     WORD *w, number;
01686     int error = 0;
01687     w = AT.WorkPointer;
01688     if ( AC.insidelevel >= MAXNEST ) {
01689         MLOCK(ErrorMessageLock);
01690         MesPrint("@Nesting of inside statements more than %d levels", (WORD)MAXNEST);
01691         MUNLOCK(ErrorMessageLock);
01692         return(-1);
01693     }
01694     AC.insidesumcheck[AC.insidelevel] = NestingChecksum();
01695     AC.insidestack[AC.insidelevel] = cbuf[AC.cbunum].Pointer
01696         - cbuf[AC.cbunum].Buffer + 2;
01697     AC.insidelevel++;
01698     *w++ = TYPEINSIDE;
01699     w++; w++;
01700     for(;;) { /* Look for a (comma separated) list of dollar variables */
01701         while ( *s == ',' ) s++;
01702         if ( *s == 0 ) break;
01703         if ( *s == '$' ) {
01704             s++; t = s;
01705             if ( FG.cTable[*s] != 0 ) {
01706                 MLOCK(ErrorMessageLock);
01707                 MesPrint("Illegal name for $ variable: %s", s-1);
01708                 MUNLOCK(ErrorMessageLock);
01709                 goto skipdol;
01710             }
01711             while ( FG.cTable[*s] == 0 || FG.cTable[*s] == 1 ) s++;
01712             c = *s; *s = 0;
01713             if ( ( number = GetDollar(t) ) < 0 ) {
01714                 number = AddDollar(t, 0, 0, 0);
01715             }
01716             *s = c;
01717             *w++ = number;
01718             AddPotModdollar(number);
01719         }
01720         else {
01721             MLOCK(ErrorMessageLock);
01722             MesPrint("&Illegal object in Inside statement");
01723             MUNLOCK(ErrorMessageLock);
01724             skipdol: error = 1;
01725             while ( *s && *s != ',' && s[1] != '$' ) s++;
01726             if ( *s == 0 ) break;
01727         }
01728     }
01729     AT.WorkPointer[1] = w - AT.WorkPointer;
01730     AddNtoL(AT.WorkPointer[1], AT.WorkPointer);
01731     return(error);
01732 }
01733
01734 /*
01735  #] ExecInside :
01736  #[ InsideDollar :

```

```

01737
01738     Execution part of Inside $a;
01739     We have to take the variables one by one and then
01740     convert them into proper terms and call Generator for the proper levels.
01741     The conversion copies the whole dollar into a new buffer, making us
01742     insensitive to redefinitions of $a inside the Inside.
01743     In the end we sort and redefine $a.
01744 */
01745
01746 int InsideDollar(PHEAD WORD *ll, WORD level)
01747 {
01748     GETBIDENTITY
01749     int numvar = (int)(ll[1]-3), j, error = 0;
01750     WORD numdol, *oldcterm, *oldwork = AT.WorkPointer, olddefer, *r, *m;
01751     WORD oldnumlhs, *dbuffer;
01752     DOLLARS d, newd;
01753     oldcterm = AN.cTerm; AN.cTerm = 0;
01754     oldnumlhs = AR.Cnumlhs; AR.Cnumlhs = ll[2];
01755     ll += 3;
01756     olddefer = AR.DeferFlag;
01757     AR.DeferFlag = 0;
01758     while ( --numvar >= 0 ) {
01759         numdol = *ll++;
01760         d = Dollars + numdol;
01761         {
01762 #ifdef WITHPTHREADS
01763             int nummodopt, dtype = -1;
01764             if ( AS.MultiThreaded && ( AC.mparallelfld == PARALLELFLAG ) ) {
01765                 for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
01766                     if ( numdol == ModOptdollars[nummodopt].number ) break;
01767                 }
01768                 if ( nummodopt < NumModOptdollars ) {
01769                     dtype = ModOptdollars[nummodopt].type;
01770                     if ( dtype == MODLOCAL ) {
01771                         d = ModOptdollars[nummodopt].dstruct+AT.identity;
01772                     }
01773                     else {
01774                         LOCK(d->pthreadlockwrite); */
01775                         LOCK(d->pthreadlockread);
01776                     }
01777                 }
01778             }
01779 #endif
01780             newd = DolToTerms(BHEAD numdol);
01781             if ( newd == 0 ) {
01782                 continue;
01783             }
01784             if ( newd->where[0] == 0 ) {
01785                 // DolToTerms potentially allocates memory. Free it.
01786                 // The free below is inside the while loop.
01787                 if ( newd->factors ) M_free(newd->factors, "Dollar factors");
01788                 M_free(newd, "Copy of dollar variable");
01789                 continue;
01790             }
01791             r = newd->where;
01792             NewSort(BHEAD0);
01793             while ( *r ) { /* Sum over the terms */
01794                 m = AT.WorkPointer;
01795                 j = *r;
01796                 while ( --j >= 0 ) *m++ = *r++;
01797                 AT.WorkPointer = m;
01798             }
01799             /*
01800             What to do with dummy indices?
01801             */
01802             if ( Generator(BHEAD oldwork, level) ) {
01803                 LowerSortLevel();
01804                 error = -1; goto idcall;
01805             }
01806             AT.WorkPointer = oldwork;
01807             AN.tryterm = 0; /* for now */
01808             if ( EndSort(BHEAD (WORD *)((void *)(&dbuffer)), 2) < 0 ) { error = 1; break; }
01809             if ( d->where && d->where != &(AM.dollarzero) ) M_free(d->where, "old buffer of dollar");
01810             d->where = dbuffer;
01811             if ( dbuffer == 0 || *dbuffer == 0 ) {
01812                 d->type = DOLZERO;
01813                 if ( dbuffer ) M_free(dbuffer, "buffer of dollar");
01814                 d->where = &(AM.dollarzero); d->size = 0;
01815             }
01816             else {
01817                 d->type = DOLTERMS;
01818                 r = d->where; while ( *r ) r += *r;
01819                 d->size = (r-d->where)+1;
01820             }
01821             /* cbuf[AM.dbufnum].rhs[numdol] = d->where; */
01822             cbuf[AM.dbufnum].rhs[numdol] = (WORD *)1;
01823             /*

```

```

01824         Now we have a little cleaning up to do
01825     */
01826 #ifdef WITHPTHREADS
01827     if ( dtype > 0 && dtype != MODLOCAL ) {
01828     /*         UNLOCK(d->pthreadlockwrite); */
01829         UNLOCK(d->pthreadlockread);
01830     }
01831 #endif
01832     if ( newd->factors ) M_free(newd->factors,"Dollar factors");
01833     M_free(newd,"Copy of dollar variable");
01834 }
01835 }
01836 idcall::
01837     AR.Cnumlhs = oldnumlhs;
01838     AR.DeferFlag = olddefer;
01839     AN.cTerm = oldcTerm;
01840     AT.WorkPointer = oldwork;
01841     return(error);
01842 }
01843
01844 /*
01845     #] InsideDollar :
01846     #[ ExchangeDollars :
01847 */
01848
01849 void ExchangeDollars(int num1, int num2)
01850 {
01851     DOLLARS d1, d2;
01852     WORD node1, node2;
01853     LONG nam;
01854     d1 = Dollars + num1; node1 = d1->node;
01855     d2 = Dollars + num2; node2 = d2->node;
01856     nam = d1->name; d1->name = d2->name; d2->name = nam;
01857     d1->node = node2; d2->node = node1;
01858     AC.dollarnames->namenode[node1].number = num2;
01859     AC.dollarnames->namenode[node2].number = num1;
01860 }
01861
01862 /*
01863     #] ExchangeDollars :
01864     #[ TermsInDollar :
01865 */
01866
01867 LONG TermsInDollar(WORD num)
01868 {
01869     GETIDENTITY
01870     DOLLARS d = Dollars + num;
01871     WORD *t;
01872     LONG n;
01873 #ifdef WITHPTHREADS
01874     int nummodopt, dtype = -1;
01875     if ( AS.MultiThreaded && ( AC.mparallelflag == PARALLELFLAG ) ) {
01876         for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
01877             if ( num == ModOptdollars[nummodopt].number ) break;
01878         }
01879         if ( nummodopt < NumModOptdollars ) {
01880             dtype = ModOptdollars[nummodopt].type;
01881             if ( dtype == MODLOCAL ) {
01882                 d = ModOptdollars[nummodopt].dstruct+AT.identity;
01883             }
01884             else {
01885                 LOCK(d->pthreadlockread);
01886             }
01887         }
01888     }
01889 #endif
01890     if ( d->type == DOLTERMS ) {
01891         n = 0;
01892         t = d->where;
01893         while ( *t ) { t += *t; n++; }
01894     }
01895     else if ( d->type == DOLWILDARGS ) {
01896         n = 0;
01897         if ( d->where[0] == 0 ) {
01898             t = d->where+1;
01899             while ( *t != 0 ) { NEXTARG(t); n++; }
01900         }
01901         else if ( d->where[0] == 1 ) n = 1;
01902     }
01903     else if ( d->type == DOLZERO ) n = 0;
01904     else n = 1;
01905 #ifdef WITHPTHREADS
01906     if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
01907 #endif
01908     return(n);
01909 }
01910

```

```

01911 /*
01912     #] TermsInDollar :
01913     #[ SizeOfDollar :
01914 */
01915
01916 LONG SizeOfDollar(WORD num)
01917 {
01918     GETIDENTITY
01919     DOLLARS d = Dollars + num;
01920     WORD *t;
01921     LONG n;
01922 #ifndef WITHPTHREADS
01923     int nummodopt, dtype = -1;
01924     if ( AS.MultiThreaded && ( AC.mparallelfld == PARALLELFLAG ) ) {
01925         for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
01926             if ( num == ModOptdollars[nummodopt].number ) break;
01927         }
01928         if ( nummodopt < NumModOptdollars ) {
01929             dtype = ModOptdollars[nummodopt].type;
01930             if ( dtype == MODLOCAL ) {
01931                 d = ModOptdollars[nummodopt].dstruct+AT.identity;
01932             }
01933             else {
01934                 LOCK(d->pthreadlockread);
01935             }
01936         }
01937     }
01938 #endif
01939     if ( d->type == DOLTERMS ) {
01940         t = d->where;
01941         while ( *t ) t += *t;
01942         t++;
01943         n = (LONG)(t - d->where);
01944     }
01945     else if ( d->type == DOLWILDARGS ) {
01946         n = 0;
01947         if ( d->where[0] == 0 ) {
01948             t = d->where+1;
01949             while ( *t != 0 ) { NEXTARG(t); n++; }
01950             t++;
01951             n = (LONG)(t - d->where);
01952         }
01953         else if ( d->where[0] == 1 ) n = 1;
01954     }
01955     else if ( d->type == DOLZERO ) n = 0;
01956     else n = 1;
01957 #ifdef WITHPTHREADS
01958     if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
01959 #endif
01960     return(n);
01961 }
01962
01963 /*
01964     #] SizeOfDollar :
01965     #[ PreIfDollarEval :
01966
01967     Routine is invoked in #if etc after $( is encountered.
01968     $(expr1 operator expr2) makes compares between expressions,
01969     $(expr1 operator _keyword) makes compares between expressions,
01970     interpreted as expressions. We are here mainly looking at $variables.
01971     First we look for the operator:
01972         >, <, ==, >=, <=, != : < means that it comes before.
01973     _keywords can be:
01974         _set(setname)    (does the expr belong to the set (only with == or !=))
01975         _productof(expr)
01976 */
01977
01978 UBYTE *PreIfDollarEval(UBYTE *s, int *value)
01979 {
01980     GETIDENTITY
01981     UBYTE *s1,*s2,*s3,*s4,*s5,*t,c,c1,c2,c3;
01982     int oprtr, type;
01983     WORD *buf1 = 0, *buf2 = 0, numset, *oldwork = AT.WorkPointer;
01984     EXCHINOUT
01985 /*
01986     Find the three composing objects (epxpression, operator, expression or keyw
01987 */
01988     while ( *s == ' ' || *s == '\t' || *s == '\n' || *s == '\r' ) s++;
01989     s1 = t = s;
01990     while ( *t != '=' && *t != '!' && *t != '>' && *t != '<' ) {
01991         if ( *t == '[' ) { SKIPBRA1(t) }
01992         else if ( *t == '{' ) { SKIPBRA2(t) }
01993         else if ( *t == '(' ) { SKIPBRA3(t) }
01994         else if ( *t == ']' || *t == '}' || *t == ')' ) {
01995             MLOCK(ErrorMessageLock);
01996             MesPrint("@Improper bracketting in #if");
01997             MUNLOCK(ErrorMessageLock);

```

```

01998         goto onerror;
01999     }
02000     t++;
02001 }
02002 s2 = t;
02003 while ( *t == '=' || *t == '!' || *t == '>' || *t == '<' ) t++;
02004 s3 = t;
02005 while ( *t && *t != ')' ) {
02006     if ( *t == '[' ) { SKIPBRA1(t) }
02007     else if ( *t == '{' ) { SKIPBRA2(t) }
02008     else if ( *t == '(' ) { SKIPBRA3(t) }
02009     else if ( *t == ']' || *t == '}' ) {
02010         MLOCK(ErrorMessageLock);
02011         MesPrint("@Improper brackets in #if");
02012         MUNLOCK(ErrorMessageLock);
02013         goto onerror;
02014     }
02015     t++;
02016 }
02017 if ( *t == 0 ) {
02018     MLOCK(ErrorMessageLock);
02019     MesPrint("@Missing ) to match $( in #if");
02020     MUNLOCK(ErrorMessageLock);
02021     goto onerror;
02022 }
02023 s4 = t; c2 = *s4; *s4 = 0;
02024 if ( s2+2 < s3 || s2 == s3 ) {
02025     IllOp;;
02026     MLOCK(ErrorMessageLock);
02027     MesPrint("@Illegal operator in $( option of #if");
02028     MUNLOCK(ErrorMessageLock);
02029     goto onerror;
02030 }
02031 if ( s2+1 == s3 ) {
02032     if ( *s2 == '=' ) oprtr = EQUAL;
02033     else if ( *s2 == '>' ) oprtr = GREATER;
02034     else if ( *s2 == '<' ) oprtr = LESS;
02035     else goto IllOp;
02036 }
02037 else if ( *s2 == '!' && s2[1] == '=' ) oprtr = NOTEQUAL;
02038 else if ( *s2 == '=' && s2[1] == '=' ) oprtr = EQUAL;
02039 else if ( *s2 == '<' && s2[1] == '=' ) oprtr = LESSEQUAL;
02040 else if ( *s2 == '>' && s2[1] == '=' ) oprtr = GREATEREQUAL;
02041 else goto IllOp;
02042 c1 = *s2; *s2 = 0;
02043 /*
02044     The two expressions are now zero terminated
02045     Look for the special keywords
02046 */
02047 while ( *s3 == ' ' || *s3 == '\t' || *s3 == '\n' || *s3 == '\r' ) s3++;
02048 t = s3;
02049 while ( chartype[*t] == 0 ) t++;
02050 if ( *t == '_' ) {
02051     t++; c = *t; *t = 0;
02052     if ( StrICmp(s3, (UBYTE *) "set_") == 0 ) {
02053         if ( oprtr != EQUAL && oprtr != NOTEQUAL ) {
02054             ImpOp;;
02055             MLOCK(ErrorMessageLock);
02056             MesPrint("@Improper operator for special keyword in $( ) option");
02057             MUNLOCK(ErrorMessageLock);
02058             goto onerror;
02059         }
02060         type = 1;
02061     }
02062     else if ( StrICmp(s3, (UBYTE *) "multipleof_") == 0 ) {
02063         if ( oprtr != EQUAL && oprtr != NOTEQUAL ) goto ImpOp;
02064         type = 2;
02065     }
02066     /*
02067     else if ( StrICmp(s3, (UBYTE *) "productof_") == 0 ) {
02068         if ( oprtr != EQUAL && oprtr != NOTEQUAL ) goto ImpOp;
02069         type = 3;
02070     }
02071 */
02072     else type = 0;
02073 }
02074 else { type = 0; c = *t; }
02075 if ( type > 0 ) {
02076     *t++ = c; s3 = t; s5 = s4-1;
02077     while ( *s5 != ')' ) {
02078         if ( *s5 == ' ' || *s5 == '\t' || *s5 == '\n' || *s5 == '\r' ) s5--;
02079         else {
02080             MLOCK(ErrorMessageLock);
02081             MesPrint("@Improper use of special keyword in $( ) option");
02082             MUNLOCK(ErrorMessageLock);
02083             goto onerror;
02084         }

```

```

02085     }
02086     c3 = *s5; *s5 = 0;
02087 }
02088 else { c3 = c2; s5 = s4; }
02089 /*
02090 Expand the first expression.
02091 */
02092 if ( ( buf1 = TranslateExpression(s1) ) == 0 ) {
02093     AT.WorkPointer = oldwork;
02094     goto onerror;
02095 }
02096 if ( type == 1 ) { /* determine the set */
02097     if ( *s3 == '{' ) {
02098         t = s3+1;
02099         SKIPBRA2(s3)
02100         numset = DoTempSet(t,s3);
02101         s3++;
02102         if ( numset < 0 ) {
02103 noset::
02104             MLOCK(ErrorMessageLock);
02105             MesPrint("@Argument of set_ is not a valid set");
02106             MUNLOCK(ErrorMessageLock);
02107             goto onerror;
02108         }
02109     }
02110     else {
02111         t = s3;
02112         while ( FG.cTable[*s3] == 0 || FG.cTable[*s3] == 1
02113             || *s3 == '_' ) s3++;
02114         c = *s3; *s3 = 0;
02115         if ( GetName(AC.varnames,t,&numset,NOAUTO) != CSET ) {
02116             *s3 = c; goto noset;
02117         }
02118         *s3 = c;
02119     }
02120     while ( *s3 == ' ' || *s3 == '\t' || *s3 == '\n' || *s3 == '\r' ) s3++;
02121     if ( s3 != s5 ) goto noset;
02122     *value = IsSetMember(buf1,numset);
02123     if ( oprtr == NOTEQUAL ) *value ^= 1;
02124 }
02125 else {
02126     if ( ( buf2 = TranslateExpression(s3) ) == 0 ) goto onerror;
02127 }
02128 if ( type == 0 ) {
02129     *value = TwoExprCompare(buf1,buf2,oprtr);
02130 }
02131 else if ( type == 2 ) {
02132     *value = IsMultipleOf(buf1,buf2);
02133     if ( oprtr == NOTEQUAL ) *value ^= 1;
02134 }
02135 /*
02136 else if ( type == 3 ) {
02137     *value = IsProductOf(buf1,buf2);
02138     if ( oprtr == NOTEQUAL ) *value ^= 1;
02139 }
02140 */
02141 if ( buf1 ) M_free(buf1,"Buffer in $()");
02142 if ( buf2 ) M_free(buf2,"Buffer in $()");
02143 *s5 = c3; *s4++ = c2; *s2 = c1;
02144 AT.WorkPointer = oldwork;
02145 BACKINOUT
02146 return(s4);
02147 onerror:
02148 if ( buf1 ) M_free(buf1,"Buffer in $()");
02149 if ( buf2 ) M_free(buf2,"Buffer in $()");
02150 AT.WorkPointer = oldwork;
02151 BACKINOUT
02152 return(0);
02153 }
02154
02155 /*
02156 #] PreIfDollarEval :
02157 #[ TranslateExpression :
02158 */
02159
02160 WORD *TranslateExpression(UBYTE *s)
02161 {
02162     GETIDENTITY
02163     CBUF *C = cbuf+AC.cbunum;
02164     WORD oldnumrhs = C->numrhs;
02165     LONG oldcpointer = C->Pointer - C->Buffer;
02166     WORD *w = AT.WorkPointer;
02167     WORD retcode, oldEside;
02168     WORD *outbuffer;
02169     *w++ = SUBEXPSize + 4;
02170     AC.ProtoType = w;
02171     *w++ = SUBEXPRESSION;

```

```

02172     *w++ = SUBEXPSIZE;
02173     *w++ = C->numrhs+1;
02174     *w++ = 1;
02175     *w++ = AC.cbufnum;
02176     FILLSUB(w)
02177     *w++ = 1; *w++ = 1; *w++ = 3; *w++ = 0;
02178     AT.WorkPointer = w;
02179     if ( ( retcode = CompileAlgebra(s,RHSIDE,AC.ProtoType) ) < 0 ) {
02180         MLOCK(ErrorMessageLock);
02181         MesPrint("@Error translating first expression in $( ) option");
02182         MUNLOCK(ErrorMessageLock);
02183         return(0);
02184     }
02185     else { AC.ProtoType[2] = retcode; }
02186 /*
02187     Evaluate this expression
02188 */
02189     if ( NewSort(BHEAD0) || NewSort(BHEAD0) ) { return(0); }
02190     AN.RepPoint = AT.RepCount + 1;
02191     oldEside = AR.Eside; AR.Eside = RHSIDE;
02192     AR.Cnumlhs = C->numlhs;
02193     if ( Generator(BHEAD AC.ProtoType-1,C->numlhs) ) {
02194         AR.Eside = oldEside;
02195         LowerSortLevel(); LowerSortLevel(); return(0);
02196     }
02197     AR.Eside = oldEside;
02198     AT.WorkPointer = w;
02199     AN.tryterm = 0; /* for now */
02200     if ( EndSort(BHEAD (WORD *)((void *)&outbuffer)),2) < 0 ) { LowerSortLevel(); return(0); }
02201     LowerSortLevel();
02202     C->Pointer = C->Buffer + oldcpointer;
02203     C->numrhs = oldnumrhs;
02204     AT.WorkPointer = AC.ProtoType - 1;
02205     return(outbuffer);
02206 }
02207
02208 /*
02209     #] TranslateExpression :
02210     #[ IsSetMember :
02211
02212     Checks whether the expression in the buffer can be seen as an element
02213     of the given set.
02214     For the special sets: if more than one term: no match!!!
02215 */
02216
02217 int IsSetMember(WORD *buffer, WORD numset)
02218 {
02219     WORD *t = buffer, *tt, num, csize, num1;
02220     WORD bufterm[4];
02221     int i, j, type;
02222     if ( numset < AM.NumFixedSets ) {
02223         if ( t[*t] != 0 ) return(0); /* More than one term */
02224         if ( *t == 0 ) {
02225             if ( numset == POS0_ || numset == NEG0_ || numset == EVEN_
02226                 || numset == Z_ || numset == Q_ ) return(1);
02227             else return(0);
02228         }
02229         if ( numset == SYMBOL_ ) {
02230             if ( *t == 8 && t[1] == SYMBOL && t[7] == 3 && t[6] == 1
02231                 && t[5] == 1 && t[4] == 1 ) return(1);
02232             else return(0);
02233         }
02234         if ( numset == INDEX_ ) {
02235             if ( *t == 7 && t[1] == INDEX && t[6] == 3 && t[5] == 1
02236                 && t[4] == 1 && t[3] > 0 ) return(1);
02237             if ( *t == 4 && t[3] == 3 && t[2] == 1 && t[1] < AM.OffsetIndex )
02238                 return(1);
02239             return(0);
02240         }
02241         if ( numset == FIXED_ ) {
02242             if ( *t == 7 && t[1] == INDEX && t[6] == 3 && t[5] == 1
02243                 && t[4] == 1 && t[3] > 0 && t[3] < AM.OffsetIndex ) return(1);
02244             if ( *t == 4 && t[3] == 3 && t[2] == 1 && t[1] < AM.OffsetIndex )
02245                 return(1);
02246             return(0);
02247         }
02248         if ( numset == DUMMYINDEX_ ) {
02249             if ( *t == 7 && t[1] == INDEX && t[6] == 3 && t[5] == 1
02250                 && t[4] == 1 && t[3] >= AM.IndDum && t[3] < AM.IndDum+MAXDUMMIES ) return(1);
02251             if ( *t == 4 && t[3] == 3 && t[2] == 1
02252                 && t[1] >= AM.IndDum && t[1] < AM.IndDum+MAXDUMMIES ) return(1);
02253             return(0);
02254         }
02255         if ( numset == VECTOR_ ) {
02256             if ( *t == 7 && t[1] == INDEX && t[6] == 3 && t[5] == 1
02257                 && t[4] == 1 && t[3] < (AM.OffsetVector+WILDOFFSET) && t[3] >= AM.OffsetVector )
02258                 return(1);

```



```

02258         return(0);
02259     }
02260     tt = t + *t - 1;
02261     if ( ABS(tt[0]) != *t-1 ) return(0);
02262     if ( numset == Q_ ) return(1);
02263     if ( numset == POS_ || numset == POS0_ ) return(tt[0]>0);
02264     else if ( numset == NEG_ || numset == NEG0_ ) return(tt[0]<0);
02265     i = (ABS(tt[0])-1)/2;
02266     tt -= i;
02267     if ( tt[0] != 1 ) return(0);
02268     for ( j = 1; j < i; j++ ) { if ( tt[j] != 0 ) return(0); }
02269     if ( numset == Z_ ) return(1);
02270     if ( numset == ODD_ ) return(t[1]&1);
02271     if ( numset == EVEN_ ) return(1-(t[1]&1));
02272     return(0);
02273 }
02274 if ( t[*t] != 0 ) return(0); /* More than one term */
02275 type = Sets[numset].type;
02276 switch ( type ) {
02277     case CSYMBOL:
02278         if ( t[0] == 8 && t[1] == SYMBOL && t[7] == 3 && t[6] == 1
02279             && t[5] == 1 && t[4] == 1 ) {
02280             num = t[3];
02281         }
02282         else if ( t[0] == 4 && t[2] == 1 && t[1] <= MAXPOWER ) {
02283             num = t[1];
02284             if ( t[3] < 0 ) num = -num;
02285             num += 2*MAXPOWER;
02286         }
02287         else return(0);
02288         break;
02289     case CVECTOR:
02290         if ( t[0] == 7 && t[1] == INDEX && t[6] == 3 && t[5] == 1
02291             && t[4] == 1 && t[3] < 0 ) {
02292             num = t[3];
02293         }
02294         else return(0);
02295         break;
02296     case CINDEX:
02297         if ( t[0] == 7 && t[1] == INDEX && t[6] == 3 && t[5] == 1
02298             && t[4] == 1 && t[3] > 0 ) {
02299             num = t[3];
02300         }
02301         else if ( t[0] == 4 && t[3] == 3 && t[2] == 1 && t[1] < AM.OffsetIndex ) {
02302             num = t[1];
02303         }
02304         else return(0);
02305         break;
02306     case CFUNCTION:
02307         if ( t[0] == 4+FUNHEAD && t[3+FUNHEAD] == 3 && t[2+FUNHEAD] == 1
02308             && t[1+FUNHEAD] == 1 && t[1] >= FUNCTION ) {
02309             num = t[1];
02310         }
02311         else return(0);
02312         break;
02313     case CNUMBER:
02314         if ( t[0] == 4 && t[2] == 1 && t[1] <= AM.OffsetIndex && t[3] == 3 ) {
02315             num = t[1];
02316         }
02317         else return(0);
02318         break;
02319     case CRANGE:
02320         csize = t[t[0]-1];
02321         csize = ABS(csize);
02322         if ( csize != t[0]-1 ) return(0);
02323         if ( Sets[numset].first < 3*MAXPOWER ) {
02324             num1 = num = Sets[numset].first;
02325             if ( num >= MAXPOWER ) num -= 2*MAXPOWER;
02326             if ( num == 0 ) {
02327                 if ( num1 < MAXPOWER ) {
02328                     if ( t[t[0]-1] >= 0 ) return(0);
02329                 }
02330                 else if ( t[t[0]-1] > 0 ) return(0);
02331             }
02332             else {
02333                 bufterm[0] = 4; bufterm[1] = ABS(num);
02334                 bufterm[2] = 1;
02335                 if ( num < 0 ) bufterm[3] = -3;
02336                 else bufterm[3] = 3;
02337                 num = CompCoef(t,bufterm);
02338                 if ( num1 < MAXPOWER ) {
02339                     if ( num >= 0 ) return(0);
02340                 }
02341                 else if ( num > 0 ) return(0);
02342             }
02343         }
02344         if ( Sets[numset].last > -3*MAXPOWER ) {

```

```

02345         num1 = num = Sets[numset].last;
02346         if ( num <= -MAXPOWER ) num += 2*MAXPOWER;
02347         if ( num == 0 ) {
02348             if ( num1 > -MAXPOWER ) {
02349                 if ( t[t[0]-1] <= 0 ) return(0);
02350             }
02351             else if ( t[t[0]-1] < 0 ) return(0);
02352         }
02353         else {
02354             bufterm[0] = 4; bufterm[1] = ABS(num);
02355             bufterm[2] = 1;
02356             if ( num < 0 ) bufterm[3] = -3;
02357             else bufterm[3] = 3;
02358             num = CompCoeef(t,bufterm);
02359             if ( num1 > -MAXPOWER ) {
02360                 if ( num <= 0 ) return(0);
02361             }
02362             else if ( num < 0 ) return(0);
02363         }
02364     }
02365     return(1);
02366     break;
02367     default: return(0);
02368 }
02369 t = SetElements + Sets[numset].first;
02370 tt = SetElements + Sets[numset].last;
02371 do {
02372     if ( num == *t ) return(1);
02373     t++;
02374 } while ( t < tt );
02375 return(0);
02376 }
02377
02378 /*
02379 #] IsSetMember :
02380 #[ IsProductOf :
02381
02382 Checks whether the expression in buf1 is a single term multiple of
02383 the expression in buf2.
02384
02385 int IsProductOf(WORD *buf1, WORD *buf2)
02386 {
02387     return(0);
02388 }
02389
02390
02391 #] IsProductOf :
02392 #[ IsMultipleOf :
02393
02394 Checks whether the expression in buf1 is a numerical multiple of
02395 the expression in buf2.
02396 */
02397
02398 int IsMultipleOf(WORD *buf1, WORD *buf2)
02399 {
02400     GETIDENTITY
02401     LONG num1, num2;
02402     WORD *t1, *t2, *m1, *m2, *r1, *r2, nc1, nc2, ni1, ni2;
02403     UWORD *IfScrat1, *IfScrat2;
02404     int i, j;
02405     if ( *buf1 == 0 && *buf2 == 0 ) return(1);
02406 /*
02407     First count terms
02408 */
02409     t1 = buf1; t2 = buf2; num1 = 0; num2 = 0;
02410     while ( *t1 ) { t1 += *t1; num1++; }
02411     while ( *t2 ) { t2 += *t2; num2++; }
02412     if ( num1 != num2 ) return(0);
02413 /*
02414     Test similarity of terms. Difference up to a number.
02415 */
02416     t1 = buf1; t2 = buf2;
02417     while ( *t1 ) {
02418         m1 = t1+1; m2 = t2+1; t1 += *t1; t2 += *t2;
02419         r1 = t1 - ABS(t1[-1]); r2 = t2 - ABS(t2[-1]);
02420         if ( r1-m1 != r2-m2 ) return(0);
02421         while ( m1 < r1 ) {
02422             if ( *m1 != *m2 ) return(0);
02423             m1++; m2++;
02424         }
02425     }
02426 /*
02427     Now we have to test the constant factor
02428 */
02429     IfScrat1 = (UWORD *) (TermMalloc("IsMultipleOf")); IfScrat2 = (UWORD
02430 *) (TermMalloc("IsMultipleOf"));
02431     t1 = buf1; t2 = buf2;

```

```

02431     t1 += *t1; t2 += *t2;
02432     if ( *t1 == 0 && *t2 == 0 ) return(1);
02433     r1 = t1 - ABS(t1[-1]); r2 = t2 - ABS(t2[-1]);
02434     nc1 = REDLENG(t1[-1]); nc2 = REDLENG(t2[-1]);
02435     if ( DivRat(BHEAD (UWORD *)r1,nc1,(UWORD *)r2,nc2,IfScrat1,&ni1) ) {
02436         MLOCK(ErrorMessageLock);
02437         MesPrint("@Called from MultipleOf in $( )");
02438         MUNLOCK(ErrorMessageLock);
02439         TermFree(IfScrat1,"IsMultipleOf"); TermFree(IfScrat2,"IsMultipleOf");
02440         Terminate(-1);
02441     }
02442     while ( *t1 ) {
02443         t1 += *t1; t2 += *t2;
02444         r1 = t1 - ABS(t1[-1]); r2 = t2 - ABS(t2[-1]);
02445         nc1 = REDLENG(t1[-1]); nc2 = REDLENG(t2[-1]);
02446         if ( DivRat(BHEAD (UWORD *)r1,nc1,(UWORD *)r2,nc2,IfScrat2,&ni2) ) {
02447             MLOCK(ErrorMessageLock);
02448             MesPrint("@Called from MultipleOf in $( )");
02449             MUNLOCK(ErrorMessageLock);
02450             TermFree(IfScrat1,"IsMultipleOf"); TermFree(IfScrat2,"IsMultipleOf");
02451             Terminate(-1);
02452         }
02453         if ( ni1 != ni2 ) return(0);
02454         i = 2*ABS(ni1);
02455         for ( j = 0; j < i; j++ ) {
02456             if ( IfScrat1[j] != IfScrat2[j] ) {
02457                 TermFree(IfScrat1,"IsMultipleOf"); TermFree(IfScrat2,"IsMultipleOf");
02458                 return(0);
02459             }
02460         }
02461     }
02462     TermFree(IfScrat1,"IsMultipleOf"); TermFree(IfScrat2,"IsMultipleOf");
02463     return(1);
02464 }
02465
02466 /*
02467 #] IsMultipleOf :
02468 #[ TwoExprCompare :
02469
02470 Compares the expressions in buf1 and buf2 according to oprtr
02471 */
02472
02473 int TwoExprCompare(WORD *buf1, WORD *buf2, int oprtr)
02474 {
02475     GETIDENTITY
02476     WORD *t1, *t2, cond;
02477     t1 = buf1; t2 = buf2;
02478     while ( *t1 && *t2 ) {
02479         cond = CompareTerms(BHEAD t1,t2,1);
02480         if ( cond != 0 ) {
02481             if ( cond > 0 ) { /* t1 comes first */
02482                 switch ( oprtr ) { /* t1 is less */
02483                     case EQUAL: return(0);
02484                     case NOTEQUAL: return(1);
02485                     case GREATEREQUAL: return(0);
02486                     case GREATER: return(0);
02487                     case LESS: return(1);
02488                     case LESSEQUAL: return(1);
02489                 }
02490             }
02491             else {
02492                 switch ( oprtr ) {
02493                     case EQUAL: return(0);
02494                     case NOTEQUAL: return(1);
02495                     case GREATEREQUAL: return(1);
02496                     case GREATER: return(1);
02497                     case LESS: return(0);
02498                     case LESSEQUAL: return(0);
02499                 }
02500             }
02501         }
02502         t1 += *t1; t2 += *t2;
02503     }
02504     if ( *t1 == *t2 ) { /* They are equal */
02505         switch ( oprtr ) {
02506             case EQUAL: return(1);
02507             case NOTEQUAL: return(0);
02508             case GREATEREQUAL: return(1);
02509             case GREATER: return(0);
02510             case LESS: return(0);
02511             case LESSEQUAL: return(1);
02512         }
02513     }
02514     else if ( *t1 ) { /* t1 is greater */
02515         switch ( oprtr ) {
02516             case EQUAL: return(0);
02517             case NOTEQUAL: return(1);

```

```

02518         case GREATEREQUAL: return(1);
02519         case GREATER: return(1);
02520         case LESS: return(0);
02521         case LESSEQUAL: return(0);
02522     }
02523 }
02524 else {
02525     switch ( oprtr ) { /* t1 is less */
02526         case EQUAL: return(0);
02527         case NOTEQUAL: return(1);
02528         case GREATEREQUAL: return(0);
02529         case GREATER: return(0);
02530         case LESS: return(1);
02531         case LESSEQUAL: return(1);
02532     }
02533 }
02534 MLOCK(ErrorMessageLock);
02535 MesPrint("@Internal problems with operator in $( )");
02536 MUNLOCK(ErrorMessageLock);
02537 Terminate(-1);
02538 return(0);
02539 }
02540
02541 /*
02542  #] TwoExprCompare :
02543  #[ DollarRaiseLow :
02544
02545  Raises or lowers the numerical value of a dollar variable
02546  Not to be used in parallel.
02547 */
02548
02549 static UWORD *dscrat = 0;
02550 static WORD ndscrat;
02551
02552 int DollarRaiseLow(UBYTE *name, LONG value)
02553 {
02554     GETIDENTITY
02555     int num;
02556     DOLLARS d;
02557     int sgn = 1;
02558     WORD lnum[4], nnum, *t1, *t2, i;
02559     UBYTE *s, c;
02560     s = name; while ( *s ) s++;
02561     if ( s[-1] == '-' && s[-2] == '-' && s > name+2 ) s -= 2;
02562     else if ( s[-1] == '+' && s[-2] == '+' && s > name+2 ) s -= 2;
02563     c = *s; *s = 0;
02564     num = GetDollar(name);
02565     *s = c;
02566     d = Dollars + num;
02567     if ( value < 0 ) { value = -value; sgn = -1; }
02568     if ( d->type == DOLZERO ) {
02569         if ( d->where ) M_free(d->where, "DollarRaiseLow");
02570         d->size = MINALLOC;
02571         d->where = (WORD *)Malloc1(d->size*sizeof(WORD), "DollarRaiseLow");
02572         if ( ( value & AWORDMASK ) != 0 ) {
02573             d->where[0] = 6; d->where[1] = value » BITSINWORD;
02574             d->where[2] = (WORD)value; d->where[3] = 1; d->where[4] = 0;
02575             d->where[5] = 5*sgn; d->where[6] = 0;
02576             d->type = DOLTERMS;
02577         }
02578     }
02579     else {
02580         d->where[0] = 4; d->where[1] = (WORD)value; d->where[2] = 1;
02581         d->where[3] = 3*sgn; d->where[4] = 0;
02582         d->type = DOLNUMBER;
02583     }
02584     else if ( d->type == DOLNUMBER || ( d->type == DOLTERMS
02585     && d->where[d->where[0]] == 0
02586     && d->where[0] == ABS(d->where[d->where[0]-1])+1 ) ) {
02587         if ( ( value & AWORDMASK ) != 0 ) {
02588             lnum[0] = value » BITSINWORD;
02589             lnum[1] = (WORD)value; lnum[2] = 1; lnum[3] = 0;
02590             nnum = 2*sgn;
02591         }
02592         else {
02593             lnum[0] = (WORD)value; lnum[1] = 1; nnum = sgn;
02594         }
02595         i = d->where[d->where[0]-1];
02596         i = REDLENG(i);
02597         if ( dscrat == 0 ) {
02598             dscrat = (UWORD *)Malloc1((AM.MaxTal+2)*sizeof(UWORD), "DollarRaiseLow");
02599         }
02600         if ( AddRat(BHEAD (UWORD *) (d->where+1), i,
02601         (UWORD *)lnum, nnum, dscrat, &ndscrat) ) {
02602             MLOCK(ErrorMessageLock);
02603             MesCall("DollarRaiseLow");
02604             MUNLOCK(ErrorMessageLock);

```

```

02605         Terminate(-1);
02606     }
02607     ndscrat = INCLENG(ndscrat);
02608     i = ABS(ndscrat);
02609     if ( i == 0 ) {
02610         M_free(d->where,"DollarRaiseLow");
02611         d->where = 0;
02612         d->type = DOLZERO;
02613         d->size = 0;
02614         return(0);
02615     }
02616     if ( i+2 > d->size ) {
02617         M_free(d->where,"DollarRaiseLow");
02618         d->size = i+2;
02619         if ( d->size < MINALLOC ) d->size = MINALLOC;
02620         d->size = ((d->size+7)/8)*8;
02621         d->where = (WORD *)Malloc1(d->size*sizeof(WORD),"DollarRaiseLow");
02622     }
02623     t1 = d->where; *t1++ = i+1; t2 = (WORD *)dscrat;
02624     while ( --i > 0 ) *t1++ = *t2++;
02625     *t1++ = ndscrat; *t1 = 0;
02626     d->type = DOLTERMS;
02627 }
02628 return(0);
02629 }
02630
02631 /*
02632  #] DollarRaiseLow :
02633  #[ EvalDoLoopArg :
02634  */
02651 WORD EvalDoLoopArg(PHEAD WORD *arg, WORD par)
02652 {
02653     WORD num, type, *td;
02654     DOLLARS d;
02655     if ( *arg == SNUMBER ) return(arg[1]);
02656     if ( *arg == DOLLAREXPR2 && arg[1] < 0 ) return(-arg[1]-1);
02657     d = Dollars + arg[1];
02658     #ifdef WITHPTHREADS
02659     {
02660         int nummodopt, dtype = -1;
02661         if ( AS.MultiThreaded && ( AC.mparallelfalg == PARALLELFLAG ) ) {
02662             for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
02663                 if ( arg[1] == ModOptdollars[nummodopt].number ) break;
02664             }
02665             if ( nummodopt < NumModOptdollars ) {
02666                 dtype = ModOptdollars[nummodopt].type;
02667                 if ( dtype == MODLOCAL ) {
02668                     d = ModOptdollars[nummodopt].dstruct+AT.identity;
02669                 }
02670             }
02671         }
02672     }
02673     #endif
02674     if ( *arg == DOLLAREXPRESSION ) {
02675         if ( arg[2] != DOLLAREXPR2 ) { /* end of chain */
02676             endofchain:
02677             type = d->type;
02678             if ( type == DOLZERO ) {}
02679             else if ( type == DOLNUMBER ) {
02680                 td = d->where;
02681                 if ( ( td[0] != 4 ) || ( (td[1]&SPECMASK) != 0 ) || ( td[2] != 1 ) ) {
02682                     MLOCK(ErrorMessageLock);
02683                     if ( par == -1 ) {
02684                         MesPrint("$-variable is not a short number in print statement");
02685                     }
02686                     else {
02687                         MesPrint("$-variable is not a short number in do loop");
02688                     }
02689                     MUNLOCK(ErrorMessageLock);
02690                     Terminate(-1);
02691                 }
02692                 return( td[3] > 0 ? td[1]: -td[1] );
02693             }
02694             else {
02695                 MLOCK(ErrorMessageLock);
02696                 if ( par == -1 ) {
02697                     MesPrint("$-variable is not a number in print statement");
02698                 }
02699                 else {
02700                     MesPrint("$-variable is not a number in do loop");
02701                 }
02702                 MUNLOCK(ErrorMessageLock);
02703                 Terminate(-1);
02704             }
02705             return(0);
02706         }
02707         num = EvalDoLoopArg(BHEAD arg+2,par);

```

```

02708     }
02709     else if ( *arg == DOLLAREXP2 ) {
02710         if ( arg[1] < 0 ) { num = -arg[1]-1; }
02711         else if ( arg[2] != DOLLAREXP2 && par == -1 ) {
02712             goto endofchain;
02713         }
02714         else { num = EvalDoLoopArg(BHEAD arg+2,par); }
02715     }
02716     else {
02717         MLOCK(ErrorMessageLock);
02718         if ( par == -1 ) {
02719             MesPrint("Invalid $-variable in print statement");
02720         }
02721         else {
02722             MesPrint("Invalid $-variable in do loop");
02723         }
02724         MUNLOCK(ErrorMessageLock);
02725         Terminate(-1);
02726         return(0);
02727     }
02728     if ( num == 0 ) return(d->nfactors);
02729     if ( num > d->nfactors || num < 1 ) {
02730         MLOCK(ErrorMessageLock);
02731         if ( par == -1 ) {
02732             MesPrint("Not a valid factor number for $-variable in print statement");
02733         }
02734         else {
02735             MesPrint("Not a valid factor number for $-variable in do loop");
02736         }
02737         MUNLOCK(ErrorMessageLock);
02738         Terminate(-1);
02739         return(0);
02740     }
02741     if ( d->factors[num].type == DOLNUMBER )
02742         return(d->factors[num].value);
02743     else { /* If correct, type can only be DOLNUMBER or DOLTERMS */
02744         MLOCK(ErrorMessageLock);
02745         if ( par == -1 ) {
02746             MesPrint("$-variable in print statement is not a number");
02747         }
02748         else {
02749             MesPrint("$-variable in do loop is not a number");
02750         }
02751         MUNLOCK(ErrorMessageLock);
02752         Terminate(-1);
02753         return(0);
02754     }
02755 }
02756
02757 /*
02758 #] EvalDoLoopArg :
02759 #[ TestDoLoop :
02760 */
02761
02762 WORD TestDoLoop(PHEAD WORD *lhsbuf, WORD level)
02763 {
02764     GETBIDENTITY
02765     WORD start,finish,incr;
02766     WORD *h;
02767     DOLLARS d;
02768     h = lhsbuf + 4; /* address of the start value */
02769     start = EvalDoLoopArg(BHEAD h,0);
02770     while ( ( *h == DOLLAREXP2 || *h == DOLLAREXP2 )
02771         && ( h[2] == DOLLAREXP2 ) ) h += 2;
02772     h += 2;
02773     finish = EvalDoLoopArg(BHEAD h,0);
02774     while ( ( *h == DOLLAREXP2 || *h == DOLLAREXP2 )
02775         && ( h[2] == DOLLAREXP2 ) ) h += 2;
02776     h += 2;
02777     incr = EvalDoLoopArg(BHEAD h,0);
02778
02779     if ( ( finish == start ) || ( finish > start && incr > 0 )
02780         || ( finish < start && incr < 0 ) ) {}
02781     else { level = lhsbuf[3]; } /* skips the loop */
02782 /*
02783 Put start in the dollar variable indicated by lhsbuf[2]
02784 */
02785     d = Dollars + lhsbuf[2];
02786 #ifdef WITHPTHREADS
02787     {
02788         int nummodopt, dtype = -1;
02789         if ( AS.MultiThreaded && ( AC.mparallelflag == PARALLELFLAG ) ) {
02790             for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
02791                 if ( lhsbuf[2] == ModOptdollars[nummodopt].number ) break;
02792             }
02793             if ( nummodopt < NumModOptdollars ) {
02794                 dtype = ModOptdollars[nummodopt].type;

```

```

02795             if ( dtype == MODLOCAL ) {
02796                 d = ModOptdollars[nummodopt].dstruct+AT.identity;
02797             }
02798         }
02799     }
02800 }
02801 #endif
02802
02803     if ( d->size < MINALLOC ) {
02804         if ( d->where && d->where != &(AM.dollarzero) ) M_free(d->where,"dollar contents");
02805         d->size = MINALLOC;
02806         d->where = (WORD *)Malloc1(d->size*sizeof(WORD),"dollar contents");
02807     }
02808     if ( start > 0 ) {
02809         d->where[0] = 4;
02810         d->where[1] = start;
02811         d->where[2] = 1;
02812         d->where[3] = 3;
02813         d->where[4] = 0;
02814         d->type = DOLNUMBER;
02815     }
02816     else if ( start < 0 ) {
02817         d->where[0] = 4;
02818         d->where[1] = -start;
02819         d->where[2] = 1;
02820         d->where[3] = -3;
02821         d->where[4] = 0;
02822         d->type = DOLNUMBER;
02823     }
02824     else
02825         d->type = DOLZERO;
02826
02827     if ( d == Dollars + lhsbuf[2] ) {
02828         cbuf[AM.dbufnum].CanCommu[lhsbuf[2]] = 0;
02829         cbuf[AM.dbufnum].NumTerms[lhsbuf[2]] = 1;
02830         cbuf[AM.dbufnum].rhs[lhsbuf[2]] = d->where;
02831     }
02832     return(level);
02833 }
02834
02835 /*
02836 #] TestDoLoop :
02837 #[ TestEndDoLoop :
02838 */
02839
02840 WORD TestEndDoLoop(PHEAD WORD *lhsbuf, WORD level)
02841 {
02842     GETBIDENTITY
02843     WORD start,finish,incr,value;
02844     WORD *h;
02845     DOLLARS d;
02846     h = lhsbuf + 4; /* address of the start value */
02847     start = EvalDoLoopArg(BHEAD h,0);
02848     while ( ( *h == DOLLAREXPRESSION || *h == DOLLAREXP2 )
02849         && ( h[2] == DOLLAREXP2 ) ) h += 2;
02850     h += 2;
02851     finish = EvalDoLoopArg(BHEAD h,0);
02852     while ( ( *h == DOLLAREXPRESSION || *h == DOLLAREXP2 )
02853         && ( h[2] == DOLLAREXP2 ) ) h += 2;
02854     h += 2;
02855     incr = EvalDoLoopArg(BHEAD h,0);
02856
02857     if ( ( finish == start ) || ( finish > start && incr > 0 )
02858         || ( finish < start && incr < 0 ) ) {}
02859     else { level = lhsbuf[3]; } /* skips the loop */
02860 /*
02861 Put start in the dollar variable indicated by lhsbuf[2]
02862 */
02863     d = Dollars + lhsbuf[2];
02864 #ifdef WITHPTHREADS
02865     {
02866         int nummodopt, dtype = -1;
02867         if ( AS.MultiThreaded && ( AC.mparallelfld == PARALLELFIELD ) ) {
02868             for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
02869                 if ( lhsbuf[2] == ModOptdollars[nummodopt].number ) break;
02870             }
02871             if ( nummodopt < NumModOptdollars ) {
02872                 dtype = ModOptdollars[nummodopt].type;
02873                 if ( dtype == MODLOCAL ) {
02874                     d = ModOptdollars[nummodopt].dstruct+AT.identity;
02875                 }
02876             }
02877         }
02878     }
02879 #endif
02880 /*
02881 Get the value

```

```

02882 */
02883     if ( d->type == DOLZERO ) {
02884         value = 0;
02885     }
02886     else if ( ( d->type == DOLNUMBER || d->type == DOLTERMS )
02887         && ( d->where[4] == 0 ) && ( d->where[0] == 4 )
02888         && ( d->where[1] > 0 ) && ( d->where[2] == 1 ) ) {
02889         value = ( d->where[3] < 0 ) ? -d->where[1]: d->where[1];
02890     }
02891     else {
02892         MLOCK(ErrorMessageLock);
02893         MesPrint("Wrong type of object in do loop parameter");
02894         MUNLOCK(ErrorMessageLock);
02895         Terminate(-1);
02896         return(level);
02897     }
02898     value += incr;
02899     if ( ( finish > start && value <= finish ) ||
02900         ( finish < start && value >= finish ) ||
02901         ( finish == start && value == finish ) ) {}
02902     else level = lhsbuf[3];
02903
02904     if ( d->size < MINALLOC ) {
02905         if ( d->where && d->where != &(AM.dollarzero) ) M_free(d->where,"dollar contents");
02906         d->size = MINALLOC;
02907         d->where = (WORD *)Malloc1(d->size*sizeof(WORD),"dollar contents");
02908     }
02909     if ( value > 0 ) {
02910         d->where[0] = 4;
02911         d->where[1] = value;
02912         d->where[2] = 1;
02913         d->where[3] = 3;
02914         d->where[4] = 0;
02915         d->type = DOLNUMBER;
02916     }
02917     else if ( start < 0 ) {
02918         d->where[0] = 4;
02919         d->where[1] = -value;
02920         d->where[2] = 1;
02921         d->where[3] = -3;
02922         d->where[4] = 0;
02923         d->type = DOLNUMBER;
02924     }
02925     else
02926         d->type = DOLZERO;
02927
02928     if ( d == Dollars + lhsbuf[2] ) {
02929         cbuf[AM.dbufnum].CanCommu[lhsbuf[2]] = 0;
02930         cbuf[AM.dbufnum].NumTerms[lhsbuf[2]] = 1;
02931         cbuf[AM.dbufnum].rhs[lhsbuf[2]] = d->where;
02932     }
02933     return(level);
02934 }
02935
02936 /*
02937     #] TestEndDoLoop :
02938     #[ DollarFactorize :
02939 */
02940 /* #define STEP2 */
02941 #define STEP2
02942
02943 int DollarFactorize(PHEAD WORD numdollar)
02944 {
02945     GETBIDENTITY
02946     DOLLARS d = Dollars + numdollar;
02947     CBUF *C, *CC;
02948     WORD *oldworkpointer;
02949     WORD *buf1, *t, *term, *buf1content, *buf2, *termextra;
02950     WORD *buf3, *argextra;
02951 #ifdef STEP2
02952     WORD *tstop, pow, *r;
02953 #endif
02954     int i, j, jj, action = 0, sign = 1;
02955     LONG insize, ii;
02956     WORD startebuf = cbuf[AT.ebufnum].numrhs;
02957     WORD nfactors, factorsincontent, extrafactor = 0;
02958     WORD oldsorttype = AR.SortType;
02959 #ifdef WITHPTHREADS
02960     int nummodopt, dtype;
02961     dtype = -1;
02962     if ( AS.MultiThreaded ) {
02963         for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
02964             if ( numdollar == ModOptdollars[nummodopt].number ) break;
02965         }
02966         if ( nummodopt < NumModOptdollars ) {
02967             dtype = ModOptdollars[nummodopt].type;
02968         }
02969     }
02970 #endif

```



```

02981         if ( dtype == MODLOCAL ) {
02982             d = ModOptdollars[nummodopt].dstruct+AT.identity;
02983         }
02984         else {
02985             LOCK(d->pthreadlockread);
02986         }
02987     }
02988 }
02989 #endif
02990 CleanDollarFactors(d);
02991 #ifdef WITHPTHREADS
02992     if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
02993 #endif
02994     if ( d->type != DOLTERMS ) { /* only one term */
02995         if ( d->type != DOLZERO ) d->nfactors = 1;
02996         return(0);
02997     }
02998     if ( d->where[d->where[0]] == 0 ) { /* only one term. easy */
02999     }
03000 /*
03001     Here should come the code for the factorization
03002     We copied the routine ArgFactorize in argument.c and changed the
03003     memory management completely. For the actual factorization it
03004     calls WORD *DoFactorizeDollar(PHEAD WORD *expr) which allocates
03005     space for the answer. Notation:
03006     term,...,term,0,term,...,term,0,term,...,term,0,0
03007
03008     #] Step 1: sort the terms properly and/or make copy --> bufl,insize
03009 */
03010     term = d->where;
03011     AR.SortType = SORTHIGHFIRST;
03012     if ( oldsorttype != AR.SortType ) {
03013         NewSort(BHEAD0);
03014         while ( *term ) {
03015             t = term + *term;
03016             if ( AN.ncmod != 0 ) {
03017                 if ( AN.ncmod != 1 || ( (WORD)AN.cmod[0] < 0 ) ) {
03018                     AR.SortType = oldsorttype;
03019                     MLOCK(ErrorMessageLock);
03020                     MesPrint("Factorization modulus a number, greater than a WORD not implemented.");
03021                     MUNLOCK(ErrorMessageLock);
03022                     Terminate(-1);
03023                 }
03024                 if ( Modulus(term) ) {
03025                     AR.SortType = oldsorttype;
03026                     MLOCK(ErrorMessageLock);
03027                     MesCall("DollarFactorize");
03028                     MUNLOCK(ErrorMessageLock);
03029                     Terminate(-1);
03030                 }
03031                 if ( !*term ) { term = t; continue; }
03032             }
03033             StoreTerm(BHEAD term);
03034             term = t;
03035         }
03036         AN.tryterm = 0; /* for now */
03037         EndSort(BHEAD (WORD *)((void *)(&bufl)),2);
03038         t = bufl; while ( *t ) t += *t;
03039         insize = t - bufl;
03040     }
03041     else {
03042         t = term; while ( *t ) t += *t;
03043         ii = insize = t - term;
03044         bufl = (WORD *)Malloc1((insize+1)*sizeof(WORD),"DollarFactorize-1");
03045         t = bufl;
03046         NCOPY(t,term,ii);
03047         *t++ = 0;
03048     }
03049 /*
03050     #] Step 1:
03051     #] Step 2: take out the 'content'.
03052 */
03053 #ifdef STEP2
03054     buflcontent = TermMalloc("DollarContent");
03055     AN.tryterm = -1;
03056     if ( ( bufl2 = TakeContent(BHEAD bufl,buflcontent) ) == 0 ) {
03057         AN.tryterm = 0;
03058         TermFree(buflcontent,"DollarContent");
03059         M_free(bufl,"DollarFactorize-1");
03060         AR.SortType = oldsorttype;
03061         MLOCK(ErrorMessageLock);
03062         MesCall("DollarFactorize");
03063         MUNLOCK(ErrorMessageLock);
03064         Terminate(-1);
03065         return(1);
03066     }
03067     else if ( ( buflcontent[0] == 4 ) && ( buflcontent[1] == 1 ) &&

```

```

03068         ( buf1content[2] == 1 ) && ( buf1content[3] == 3 ) ) { /* Nothing happened */
03069     AN.tryterm = 0;
03070     if ( buf2 != buf1 ) {
03071         M_free(buf2,"DollarFactorize-2");
03072         buf2 = buf1;
03073     }
03074     factorsincontent = 0;
03075 }
03076 else {
03077 /*
03078     The way we took out objects is rather brutish. We have to normalize
03079 */
03080     AN.tryterm = 0;
03081     if ( buf2 != buf1 ) M_free(buf1,"DollarFactorize-1");
03082     buf1 = buf2;
03083     t = buf1; while ( *t ) t += *t;
03084     insize = t - buf1;
03085 /*
03086     Now analyse how many factors there are in the content
03087 */
03088     factorsincontent = 0;
03089     term = buf1content;
03090     tstop = term + *term;
03091     if ( tstop[-1] < 0 ) factorsincontent++;
03092     if ( ABS(tstop[-1]) == 3 && tstop[-2] == 1 && tstop[-3] == 1 ) {
03093         tstop -= ABS(tstop[-1]);
03094     }
03095     else {
03096         factorsincontent++;
03097         tstop -= ABS(tstop[-1]);
03098     }
03099     term++;
03100     while ( term < tstop ) {
03101         switch ( *term ) {
03102             case SYMBOL:
03103                 t = term+2; i = (term[1]-2)/2;
03104                 while ( i > 0 ) {
03105                     factorsincontent += ABS(t[1]);
03106                     i--; t += 2;
03107                 }
03108                 break;
03109             case DOTPRODUCT:
03110                 t = term+2; i = (term[1]-2)/3;
03111                 while ( i > 0 ) {
03112                     factorsincontent += ABS(t[2]);
03113                     i--; t += 3;
03114                 }
03115                 break;
03116             case VECTOR:
03117             case DELTA:
03118                 factorsincontent += (term[1]-2)/2;
03119                 break;
03120             case INDEX:
03121                 factorsincontent += term[1]-2;
03122                 break;
03123             default:
03124                 if ( *term >= FUNCTION ) factorsincontent++;
03125                 break;
03126         }
03127         term += term[1];
03128     }
03129 }
03130 #else
03131     factorsincontent = 0;
03132     buf1content = 0;
03133 #endif
03134 /*
03135     #] Step 2: take out the 'content'.
03136     #[ Step 3: ConvertToPoly
03137         if there are objects that are not SYMBOLs,
03138         invoke ConvertToPoly
03139         We keep the original in buf1 in case there are no factors
03140 */
03141     t = buf1;
03142     while ( *t ) {
03143         if ( ( t[1] != SYMBOL ) && ( *t != (ABS(t[*t-1])+1) ) ) {
03144             action = 1; break;
03145         }
03146         t += *t;
03147     }
03148     if ( DetCommu(buf1) > 1 ) {
03149         MesPrint("Cannot factorize a $-expression with more than one noncommuting object");
03150         AR.SortType = oldsorttype;
03151         M_free(buf1,"DollarFactorize-2");
03152         if ( buf1content ) TermFree(buf1content,"DollarContent");
03153         MesCall("DollarFactorize");
03154         Terminate(-1);

```

```

03155         return(-1);
03156     }
03157     if ( action ) {
03158         t = buf1;
03159         termextra = AT.WorkPointer;
03160         NewSort(BHEAD0);
03161         NewSort(BHEAD0);
03162         while ( *t ) {
03163             if ( LocalConvertToPoly(BHEAD t,termextra,startebuf,0) < 0 ) {
03164 getout:
03165                 AR.SortType = oldsorttype;
03166                 M_free(buf1,"DollarFactorize-2");
03167                 if ( buf1content ) TermFree(buf1content,"DollarContent");
03168                 MesCall("DollarFactorize");
03169                 Terminate(-1);
03170                 return(-1);
03171             }
03172             StoreTerm(BHEAD termextra);
03173             t += *t;
03174         }
03175         AN.tryterm = 0; /* for now */
03176         if ( EndSort(BHEAD (WORD *)((void *)&buf2)),2) < 0 ) { goto getout; }
03177         LowerSortLevel();
03178         t = buf2; while ( *t > 0 ) t += *t;
03179     }
03180     else {
03181         buf2 = buf1;
03182     }
03183     /*
03184         #] Step 3: ConvertToPoly
03185         #[ Step 4: Now the hard work.
03186     */
03187     if ( ( buf3 = poly_factorize_dollar(BHEAD buf2) ) == 0 ) {
03188         MesCall("DollarFactorize");
03189         AR.SortType = oldsorttype;
03190         if ( buf2 != buf1 && buf2 ) M_free(buf2,"DollarFactorize-3");
03191         M_free(buf1,"DollarFactorize-3");
03192         if ( buf1content ) TermFree(buf1content,"DollarContent");
03193         Terminate(-1);
03194         return(-1);
03195     }
03196     if ( buf2 != buf1 && buf2 ) {
03197         M_free(buf2,"DollarFactorize-3");
03198         buf2 = 0;
03199     }
03200     term = buf3;
03201     AR.SortType = oldsorttype;
03202     /*
03203     Count the factors and strip a factor -1
03204     */
03205     nfactors = 0;
03206     while ( *term ) {
03207 #ifdef STEP2
03208         if ( *term == 4 && term[4] == 0 && term[3] == -3 && term[2] == 1
03209             && term[1] == 1 ) {
03210             WORD *tt1, *tt2, *ttstop;
03211             sign = -sign;
03212             tt1 = term; tt2 = term + *term + 1;
03213             ttstop = tt2;
03214             while ( *ttstop ) {
03215                 while ( *ttstop ) ttstop += *ttstop;
03216                 ttstop++;
03217             }
03218             while ( tt2 < ttstop ) *tt1++ = *tt2++;
03219             *tt1 = 0;
03220             factorsincontent++;
03221             extrafactor++;
03222         }
03223     else
03224 #endif
03225     {
03226         term += *term;
03227         while ( *term ) { term += *term; }
03228         nfactors++; term++;
03229     }
03230 }
03231 /*
03232     We have now:
03233     buf1: the original before ConvertToPoly for if only one factor
03234     buf3: the factored expression with nfactors factors
03235
03236     #] Step 4:
03237     #[ Step 5: ConvertFromPoly
03238         If ConvertToPoly was used, use now ConvertFromPoly
03239         Be careful: there should be more than one factor now.
03240     */
03241 #ifdef WITHPTHREADS

```

```

03242     if ( dtype > 0 && dtype != MODLOCAL ) { LOCK(d->pthreadlockread); }
03243 #endif
03244     if ( nfactors == 1 && extrafactor == 0 ) { /* we can use the buf1 contents */
03245         if ( factorsincontent == 0 ) {
03246             d->nfactors = 1;
03247 #ifdef WITHPTHREADS
03248             if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
03249 #endif
03250 /*
03251     We used here (before 3-sep-2015) the original and did not make
03252     provisions for having a factors struct, figuring that all info
03253     is identical to the full dollar. This makes things too
03254     complicated at later stages.
03255 */
03256     d->factors = (FACDOLLAR *)Malloc1(sizeof(FACDOLLAR),"factors in dollar");
03257     term = buf1; while ( *term ) term += *term;
03258     d->factors[0].size = i = term - buf1;
03259     d->factors[0].where = t = (WORD *)Malloc1(sizeof(WORD)*(i+1),"DollarFactorize-5");
03260     term = buf1; NCOPY(t,term,i); *t = 0;
03261     AR.SortType = oldsorttype;
03262     M_free(buf3,"DollarFactorize-4");
03263     if ( buf2 != buf1 && buf2 ) M_free(buf2,"DollarFactorize-4");
03264     M_free(buf1,"DollarFactorize-4");
03265     if ( buf1content ) TermFree(buf1content,"DollarContent");
03266     return(0);
03267 }
03268     else {
03269         d->factors = (FACDOLLAR *)Malloc1(sizeof(FACDOLLAR)*(nfactors+factorsincontent),"factors
in dollar");
03270         term = buf1; while ( *term ) term += *term;
03271         d->factors[0].size = i = term - buf1;
03272         d->factors[0].where = t = (WORD *)Malloc1(sizeof(WORD)*(i+1),"DollarFactorize-5");
03273         term = buf1; NCOPY(t,term,i); *t = 0;
03274         M_free(buf3,"DollarFactorize-4");
03275         buf3 = 0;
03276         if ( buf2 != buf1 && buf2 ) {
03277             M_free(buf2,"DollarFactorize-4");
03278             buf2 = 0;
03279         }
03280     }
03281 }
03282     else if ( action ) {
03283         C = cbuf+AC.cbunum;
03284         CC = cbuf+AT.ebunum;
03285         oldworkpointer = AT.WorkPointer;
03286         d->factors = (FACDOLLAR *)Malloc1(sizeof(FACDOLLAR)*(nfactors+factorsincontent),"factors in
dollar");
03287         term = buf3;
03288         for ( i = 0; i < nfactors; i++ ) {
03289             argextra = AT.WorkPointer;
03290             NewSort(BHEAD0);
03291             NewSort(BHEAD0);
03292             while ( *term ) {
03293                 if ( ConvertFromPoly(BHEAD term,argextra,numxsymbol,CC->numrhs-startebuf+numxsymbol
,startebuf-numxsymbol,1) <= 0 ) {
03294                     LowerSortLevel();
03295                     AR.SortType = oldsorttype;
03296 getout2:    M_free(d->factors,"factors in dollar");
03297                     d->factors = 0;
03298 #ifdef WITHPTHREADS
03299                     if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
03300 #endif
03301                     M_free(buf3,"DollarFactorize-4");
03302                     if ( buf2 != buf1 && buf2 ) M_free(buf2,"DollarFactorize-4");
03303                     M_free(buf1,"DollarFactorize-4");
03304                     if ( buf1content ) TermFree(buf1content,"DollarContent");
03305                     return(-3);
03306                 }
03307                 AT.WorkPointer = argextra + *argextra;
03308 /*
03309     ConvertFromPoly leaves terms with subexpressions. Hence:
03310 */
03311         if ( Generator(BHEAD argextra,C->numlhs+1) ) {
03312             goto getout2;
03313         }
03314         term += *term;
03315     }
03316     term++;
03317     AT.WorkPointer = oldworkpointer;
03318     AN.tryterm = 0; /* for now */
03319     EndSort(BHEAD (WORD *)((void *)(&(d->factors[i].where))),2);
03320     LowerSortLevel();
03321     d->factors[i].type = DOLTERMS;
03322     t = d->factors[i].where;
03323     while ( *t ) t += *t;
03324     d->factors[i].size = t - d->factors[i].where;
03325 }
03326

```

```

03327     CC->numrhs = startebuf;
03328 }
03329 else {
03330     C = cbuf+AC.cbufnum;
03331     oldworkpointer = AT.WorkPointer;
03332     d->factors = (FACDOLLAR *)Malloc1(sizeof(FACDOLLAR)*(nfactors+factorsincontent),"factors in
dollar");
03333     term = buf3;
03334     for ( i = 0; i < nfactors; i++ ) {
03335         NewSort(BHEAD0);
03336         while ( *term ) {
03337             argextra = oldworkpointer;
03338             j = *term;
03339             NCOPY(argextra,term,j)
03340             AT.WorkPointer = argextra;
03341             if ( Generator(BHEAD oldworkpointer,C->numlhs+1) ) {
03342                 goto getout2;
03343             }
03344         }
03345         term++;
03346         AT.WorkPointer = oldworkpointer;
03347         AN.tryterm = 0; /* for now */
03348         EndSort(BHEAD (WORD *)((void *)(&(d->factors[i].where))),2);
03349         d->factors[i].type = DOLTERMS;
03350         t = d->factors[i].where;
03351         while ( *t ) t += *t;
03352         d->factors[i].size = t - d->factors[i].where;
03353     }
03354 }
03355 d->nfactors = nfactors + factorsincontent;
03356 /*
03357     #] Step 5: ConvertFromPoly
03358     #[ Step 6: The factors of the content
03359 */
03360 if ( buf3 ) M_free(buf3,"DollarFactorize-5");
03361 if ( buf2 != buf1 && buf2 ) M_free(buf2,"DollarFactorize-5");
03362 M_free(buf1,"DollarFactorize-5");
03363 j = nfactors;
03364 #ifdef STEP2
03365 term = buf1content;
03366 tstop = term + *term;
03367 if ( tstop[-1] < 0 ) { tstop[-1] = -tstop[-1]; sign = -sign; }
03368 tstop -= tstop[-1];
03369 term++;
03370 while ( term < tstop ) {
03371     switch ( *term ) {
03372     case SYMBOL:
03373         t = term+2; i = (term[1]-2)/2;
03374         while ( i > 0 ) {
03375             if ( t[1] < 0 ) { t[1] = -t[1]; pow = -1; }
03376             else { pow = 1; }
03377             for ( jj = 0; jj < t[1]; jj++ ) {
03378                 r = d->factors[j].where = (WORD *)Malloc1(9*sizeof(WORD),"factor");
03379                 r[0] = 8; r[1] = SYMBOL; r[2] = 4; r[3] = *t; r[4] = pow;
03380                 r[5] = 1; r[6] = 1; r[7] = 3; r[8] = 0;
03381                 d->factors[j].type = DOLTERMS;
03382                 d->factors[j].size = 8;
03383                 j++;
03384             }
03385             i--; t += 2;
03386         }
03387         break;
03388     case DOTPRODUCT:
03389         t = term+2; i = (term[1]-2)/3;
03390         while ( i > 0 ) {
03391             if ( t[2] < 0 ) { t[2] = -t[2]; pow = -1; }
03392             else { pow = 1; }
03393             for ( jj = 0; jj < t[2]; jj++ ) {
03394                 r = d->factors[j].where = (WORD *)Malloc1(10*sizeof(WORD),"factor");
03395                 r[0] = 9; r[1] = DOTPRODUCT; r[2] = 5; r[3] = t[0]; r[4] = t[1];
03396                 r[5] = pow; r[6] = 1; r[7] = 1; r[8] = 3; r[9] = 0;
03397                 d->factors[j].type = DOLTERMS;
03398                 d->factors[j].size = 9;
03399                 j++;
03400             }
03401             i--; t += 3;
03402         }
03403         break;
03404     case VECTOR:
03405     case DELTA:
03406         t = term+2; i = (term[1]-2)/2;
03407         while ( i > 0 ) {
03408             for ( jj = 0; jj < t[1]; jj++ ) {
03409                 r = d->factors[j].where = (WORD *)Malloc1(9*sizeof(WORD),"factor");
03410                 r[0] = 8; r[1] = *term; r[2] = 4; r[3] = *t; r[4] = t[1];
03411                 r[5] = 1; r[6] = 1; r[7] = 3; r[8] = 0;
03412                 d->factors[j].type = DOLTERMS;

```

```

03413             d->factors[j].size = 8;
03414             j++;
03415         }
03416         i--; t += 2;
03417     }
03418     break;
03419 case INDEX:
03420     t = term+2; i = term[1]-2;
03421     while ( i > 0 ) {
03422         for ( jj = 0; jj < t[1]; jj++ ) {
03423             r = d->factors[j].where = (WORD *)Malloca(8*sizeof(WORD), "factor");
03424             r[0] = 7; r[1] = *term; r[2] = 3; r[3] = *t;
03425             r[4] = 1; r[5] = 1; r[6] = 3; r[7] = 0;
03426             d->factors[j].type = DOLTERMS;
03427             d->factors[j].size = 7;
03428             j++;
03429         }
03430         i--; t++;
03431     }
03432     break;
03433 default:
03434     if ( *term >= FUNCTION ) {
03435         r = d->factors[j].where = (WORD *)Malloca((term[1]+5)*sizeof(WORD), "factor");
03436         *r++ = d->factors[j].size = term[1]+4;
03437         for ( jj = 0; jj < t[1]; jj++ ) *r++ = term[jj];
03438         *r++ = 1; *r++ = 1; *r++ = 3; *r = 0;
03439         j++;
03440     }
03441     break;
03442 }
03443 term += term[1];
03444 }
03445 #endif
03446 /*
03447     #] Step 6:
03448     #[ Step 7: Numerical factors
03449 */
03450 #ifdef STEP2
03451     term = buf1content;
03452     tstop = term + *term;
03453     if ( tstop[-1] == 3 && tstop[-2] == 1 && tstop[-3] == 1 ) {}
03454     else if ( tstop[-1] == 3 && tstop[-2] == 1 && (UWORD)(tstop[-3]) <= MAXPOSITIVE ) {
03455         d->factors[j].where = 0;
03456         d->factors[j].size = 0;
03457         d->factors[j].type = DOLNUMBER;
03458         d->factors[j].value = sign*tstop[-3];
03459         sign = 1;
03460         j++;
03461     }
03462     else {
03463         d->factors[j].where = r = (WORD *)Malloca((tstop[-1]+2)*sizeof(WORD), "numfactor");
03464         d->factors[j].size = tstop[-1]+1;
03465         d->factors[j].type = DOLTERMS;
03466         d->factors[j].value = 0;
03467         i = tstop[-1];
03468         t = tstop - i;
03469         *r++ = tstop[-1]+1;
03470         NCOPY(r,t,i);
03471         *r = 0;
03472         if ( sign < 0 ) {
03473             r = d->factors[j].where;
03474             while ( *r ) {
03475                 r += *r; r[-1] = -r[-1];
03476             }
03477             sign = 1;
03478         }
03479         j++;
03480     }
03481 #endif
03482 if ( sign < 0 ) { /* Note that this guy should come first */
03483     for ( jj = j; jj > 0; jj-- ) {
03484         d->factors[jj] = d->factors[jj-1];
03485     }
03486     d->factors[0].where = 0;
03487     d->factors[0].size = 0;
03488     d->factors[0].type = DOLNUMBER;
03489     d->factors[0].value = -1;
03490     j++;
03491 }
03492 d->nfactors = j;
03493 if ( buf1content ) TermFree(buf1content, "DollarContent");
03494 /*
03495     #] Step 7:
03496     #[ Step 8: Sorting the factors
03497
03498     There are d->nfactors factors. Look which ones have a 'where'
03499     Sort them by bubble sort

```

```

03500 /*
03501     if ( d->nfactors > 1 ) {
03502         WORD ***fac, j1, j2, k, ret, *s1, *s2, *s3;
03503         LONG **facsize, x;
03504         facsize = (LONG **)Malloc1((sizeof(WORD **)+sizeof(LONG *))d->nfactors,"SortDollarFactors");
03505         fac = (WORD **)(facsize+d->nfactors);
03506         k = 0;
03507         for ( j = 0; j < d->nfactors; j++ ) {
03508             if ( d->factors[j].where ) {
03509                 fac[k] = &(d->factors[j].where);
03510                 facsize[k] = &(d->factors[j].size);
03511                 k++;
03512             }
03513         }
03514         if ( k > 1 ) {
03515             for ( j = 1; j < k; j++ ) { /* bubble sort */
03516                 j1 = j; j2 = j1-1;
03517 nextj1:;
03518                 s1 = *(fac[j1]); s2 = *(fac[j2]);
03519                 while ( *s1 && *s2 ) {
03520                     if ( ( ret = CompareTerms(BHEAD s2, s1, (WORD)2) ) == 0 ) {
03521                         s1 += *s1; s2 += *s2;
03522                     }
03523                     else if ( ret > 0 ) goto nextj;
03524                     else {
03525     exch:
03526                         s3 = *(fac[j1]); *(fac[j1]) = *(fac[j2]); *(fac[j2]) = s3;
03527                         x = *(facsize[j1]); *(facsize[j1]) = *(facsize[j2]); *(facsize[j2]) = x;
03528                         j1--; j2--;
03529                         if ( j1 > 0 ) goto nextj1;
03530                         goto nextj;
03531                     }
03532                 }
03533                 if ( *s1 ) goto nextj;
03534                 if ( *s2 ) goto exch;
03535 nextj:;
03536             }
03537         }
03538         M_free(facsize,"SortDollarFactors");
03539     }
03540     /*
03541         #] Step 8:
03542     */
03543     #ifdef WITHPTHREADS
03544     if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
03545     #endif
03546     return(0);
03547 }
03548
03549 /*
03550     #] DollarFactorize :
03551     #[ CleanDollarFactors :
03552 */
03553
03554 void CleanDollarFactors(DOLLARS d)
03555 {
03556     int i;
03557     if ( d->nfactors > 1 ) {
03558         for ( i = 0; i < d->nfactors; i++ ) {
03559             if ( d->factors[i].where )
03560                 M_free(d->factors[i].where,"dollar factors");
03561         }
03562     }
03563     if ( d->factors ) {
03564         M_free(d->factors,"dollar factors");
03565         d->factors = 0;
03566     }
03567     d->nfactors = 0;
03568 }
03569
03570 /*
03571     #] CleanDollarFactors :
03572     #[ TakeDollarContent :
03573 */
03574
03575 WORD *TakeDollarContent(PHEAD WORD *dollarbuffer, WORD **factor)
03576 {
03577     WORD *remain, *t;
03578     int pow;
03579     /*
03580     We force the sign of the first term to be positive.
03581     */
03582     t = dollarbuffer; pow = 1;
03583     t += *t;
03584     if ( t[-1] < 0 ) {
03585         pow = 0;
03586         t[-1] = -t[-1];

```

```

03587         while ( *t ) {
03588             t += *t; t[-1] = -t[-1];
03589         }
03590     }
03591     /*
03592     Now the GCD of the numerators and the LCM of the denominators:
03593     */
03594     if ( AN.cmod != 0 ) {
03595         if ( ( *factor = MakeDollarMod(BHEAD dollarbuffer,&remain) ) == 0 ) {
03596             Terminate(-1);
03597         }
03598         if ( pow == 0 ) {
03599             (*factor)[**factor-1] = -(*factor)[**factor-1];
03600             (*factor)[**factor-1] += AN.cmod[0];
03601         }
03602     }
03603     else {
03604         if ( ( *factor = MakeDollarInteger(BHEAD dollarbuffer,&remain) ) == 0 ) {
03605             Terminate(-1);
03606         }
03607         if ( pow == 0 ) {
03608             (*factor)[**factor-1] = -(*factor)[**factor-1];
03609         }
03610     }
03611     return(remain);
03612 }
03613
03614 /*
03615 #] TakeDollarContent :
03616 #[ MakeDollarInteger :
03617 */
03627 WORD *MakeDollarInteger(PHEAD WORD *bufin,WORD **bufout)
03628 {
03629     GETBIDENTITY
03630     UWORD *GCDBuffer, *GCDBuffer2, *LCMbuffer, *LCMb, *LCMc;
03631     WORD *r, *r1, *r2, *r3, *rnext, i, k, j, *oldworkpointer, *factor;
03632     WORD kGCD, kLCM, kGCD2, kkLCM, jLCM, jGCD;
03633     CBUF *C = cbuf+AC.cbunum;
03634
03635     GCDBuffer = NumberMalloc("MakeDollarInteger");
03636     GCDBuffer2 = NumberMalloc("MakeDollarInteger");
03637     LCMbuffer = NumberMalloc("MakeDollarInteger");
03638     LCMb = NumberMalloc("MakeDollarInteger");
03639     LCMc = NumberMalloc("MakeDollarInteger");
03640     r = bufin;
03641     /*
03642     First take the first term to load up the LCM and the GCD
03643     */
03644     r2 = r + *r;
03645     j = r2[-1];
03646     r3 = r2 - ABS(j);
03647     k = REDLENG(j);
03648     if ( k < 0 ) k = -k;
03649     while ( ( k > 1 ) && ( r3[k-1] == 0 ) ) k--;
03650     for ( kGCD = 0; kGCD < k; kGCD++ ) GCDBuffer[kGCD] = r3[kGCD];
03651     k = REDLENG(j);
03652     if ( k < 0 ) k = -k;
03653     r3 += k;
03654     while ( ( k > 1 ) && ( r3[k-1] == 0 ) ) k--;
03655     for ( kLCM = 0; kLCM < k; kLCM++ ) LCMbuffer[kLCM] = r3[kLCM];
03656     r1 = r2;
03657     /*
03658     Now go through the rest of the terms in this argument.
03659     */
03660     while ( *r1 ) {
03661         r2 = r1 + *r1;
03662         j = r2[-1];
03663         r3 = r2 - ABS(j);
03664         k = REDLENG(j);
03665         if ( k < 0 ) k = -k;
03666         while ( ( k > 1 ) && ( r3[k-1] == 0 ) ) k--;
03667         if ( ( ( GCDBuffer[0] == 1 ) && ( kGCD == 1 ) ) ) {
03668             /*
03669             GCD is already 1
03670             */
03671         }
03672         else if ( ( ( k != 1 ) || ( r3[0] != 1 ) ) ) {
03673             if ( GcdLong(BHEAD GCDBuffer,kGCD,(UWORD *)r3,k,GCDBuffer2,&kGCD2) ) {
03674                 goto MakeDollarIntegerErr;
03675             }
03676             kGCD = kGCD2;
03677             for ( i = 0; i < kGCD; i++ ) GCDBuffer[i] = GCDBuffer2[i];
03678         }
03679         else {
03680             kGCD = 1; GCDBuffer[0] = 1;
03681         }
03682         k = REDLENG(j);

```



```

03683         if ( k < 0 ) k = -k;
03684         r3 += k;
03685         while ( ( k > 1 ) && ( r3[k-1] == 0 ) ) k--;
03686         if ( ( ( LCMbuffer[0] == 1 ) && ( kLCM == 1 ) ) ) {
03687             for ( kLCM = 0; kLCM < k; kLCM++ )
03688                 LCMbuffer[kLCM] = r3[kLCM];
03689         }
03690         else if ( ( k != 1 ) || ( r3[0] != 1 ) ) {
03691             if ( GcdLong(BHEAD LCMbuffer,kLCM, (UWORD *)r3,k,LCMb,&kkLCM) ) {
03692                 goto MakeDollarIntegerErr;
03693             }
03694             DivLong( (UWORD *)r3,k,LCMb,kkLCM,LCMb,&kkLCM,LCMc,&jLCM);
03695             Mullong(LCMbuffer,kLCM,LCMb,kkLCM,LCMc,&jLCM);
03696             for ( kLCM = 0; kLCM < jLCM; kLCM++ )
03697                 LCMbuffer[kLCM] = LCMc[kLCM];
03698         }
03699         else {} /* LCM doesn't change */
03700         r1 = r2;
03701     }
03702     /*
03703     Now put the factor together: GCD/LCM
03704     */
03705     r3 = (WORD *) (GCDbuffer);
03706     if ( kGCD == kLCM ) {
03707         for ( jGCD = 0; jGCD < kGCD; jGCD++ )
03708             r3[jGCD+kGCD] = LCMbuffer[jGCD];
03709         k = kGCD;
03710     }
03711     else if ( kGCD > kLCM ) {
03712         for ( jGCD = 0; jGCD < kLCM; jGCD++ )
03713             r3[jGCD+kGCD] = LCMbuffer[jGCD];
03714         for ( jGCD = kLCM; jGCD < kGCD; jGCD++ )
03715             r3[jGCD+kGCD] = 0;
03716         k = kGCD;
03717     }
03718     else {
03719         for ( jGCD = kGCD; jGCD < kLCM; jGCD++ )
03720             r3[jGCD] = 0;
03721         for ( jGCD = 0; jGCD < kLCM; jGCD++ )
03722             r3[jGCD+kLCM] = LCMbuffer[jGCD];
03723         k = kLCM;
03724     }
03725     j = 2*k+1;
03726     /*
03727     Now we have to write this to factor
03728     */
03729     factor = r1 = (WORD *) Malloc1((j+2)*sizeof(WORD), "MakeDollarInteger");
03730     *r1++ = j+1; r2 = r3;
03731     for ( i = 0; i < k; i++ ) { *r1++ = *r2++; *r1++ = *r2++; }
03732     *r1++ = j;
03733     *r1 = 0;
03734     /*
03735     Next we have to take the factor out from the argument.
03736     This cannot be done in location, because the denominator stuff can make
03737     coefficients longer.
03738
03739     We do this via a sort because the things may be jumbled any way and we
03740     do not know in advance how much space we need.
03741     */
03742     NewSort(BHEAD0);
03743     r = bufin;
03744     oldworkpointer = AT.WorkPointer;
03745     while ( *r ) {
03746         rnext = r + *r;
03747         j = ABS(rnext[-1]);
03748         r3 = rnext - j;
03749         r2 = oldworkpointer;
03750         while ( r < r3 ) *r2++ = *r++;
03751         j = (j-1)/2; /* reduced length. Remember, k is the other red length */
03752         if ( DivRat(BHEAD (UWORD *)r3,j,GCDbuffer,k, (UWORD *)r2,&i) ) {
03753             goto MakeDollarIntegerErr;
03754         }
03755         i = 2*i+1;
03756         r2 = r2 + i;
03757         if ( rnext[-1] < 0 ) r2[-1] = -i;
03758         else r2[-1] = i;
03759         *oldworkpointer = r2-oldworkpointer;
03760         AT.WorkPointer = r2;
03761         if ( Generator(BHEAD oldworkpointer,C->numlhs) ) {
03762             goto MakeDollarIntegerErr;
03763         }
03764         r = rnext;
03765     }
03766     AT.WorkPointer = oldworkpointer;
03767     AN.tryterm = 0; /* for now */
03768     EndSort(BHEAD (WORD *)bufout,2);
03769     /*

```

```

03770 Cleanup
03771 */
03772     NumberFree(LCMC,"MakeDollarInteger");
03773     NumberFree(LCMb,"MakeDollarInteger");
03774     NumberFree(LCMbuffer,"MakeDollarInteger");
03775     NumberFree(GCDBuffer2,"MakeDollarInteger");
03776     NumberFree(GCDBuffer,"MakeDollarInteger");
03777     return(factor);
03778
03779 MakeDollarIntegerErr:
03780     NumberFree(LCMC,"MakeDollarInteger");
03781     NumberFree(LCMb,"MakeDollarInteger");
03782     NumberFree(LCMbuffer,"MakeDollarInteger");
03783     NumberFree(GCDBuffer2,"MakeDollarInteger");
03784     NumberFree(GCDBuffer,"MakeDollarInteger");
03785     MesCall("MakeDollarInteger");
03786     Terminate(-1);
03787     return(0);
03788 }
03789
03790 /*
03791 #] MakeDollarInteger :
03792 #[ MakeDollarMod :
03793 */
03801 WORD *MakeDollarMod(PHEAD WORD *buffer, WORD **bufout)
03802 {
03803     GETBIDENTITY
03804     WORD *r, *r1, x, xx, ix, ip;
03805     WORD *factor, *oldworkpointer;
03806     int i;
03807     CBUF *C = cbuf+AC.cbunum;
03808     r = buffer;
03809     x = r[*r-3];
03810     if ( r[*r-1] < 0 ) x += AN.cmod[0];
03811     if ( GetModInverses(x,(WORD)(AN.cmod[0]),&ix,&ip) ) {
03812         Terminate(-1);
03813     }
03814     factor = (WORD *)Malloca(5*sizeof(WORD),"MakeDollarMod");
03815     factor[0] = 4; factor[1] = x; factor[2] = 1; factor[3] = 3; factor[4] = 0;
03816 /*
03817     Now we have to multiply all coefficients by ix.
03818     This does not make things longer, but we should keep to the conventions
03819     of MakeDollarInteger.
03820 */
03821     NewSort(BHEAD0);
03822     r = buffer;
03823     oldworkpointer = AT.WorkPointer;
03824     while ( *r ) {
03825         r1 = oldworkpointer; i = *r;
03826         NCOPY(r1,r,i);
03827         xx = r1[-3]; if ( r1[-1] < 0 ) xx += AN.cmod[0];
03828         r1[-1] = (WORD)((LONG)xx*ix) % AN.cmod[0];
03829         *r1 = 0; AT.WorkPointer = r1;
03830         if ( Generator(BHEAD oldworkpointer,C->numlhs) ) {
03831             Terminate(-1);
03832         }
03833     }
03834     AT.WorkPointer = oldworkpointer;
03835     AN.tryterm = 0; /* for now */
03836     EndSort(BHEAD (WORD *)bufout,2);
03837     return(factor);
03838 }
03839 /*
03840 #] MakeDollarMod :
03841 #[ GetDolNum :
03842
03843     Evaluates a chain of DOLLAREXPR2 into a number
03844 */
03845
03846 int GetDolNum(PHEAD WORD *t, WORD *tstop)
03847 {
03848     DOLLARS d;
03849     WORD num, *w;
03850     if ( t+3 < tstop && t[3] == DOLLAREXPR2 ) {
03851         d = Dollars + t[2];
03852 #ifdef WITHPTHREADS
03853     {
03854         int nummodopt, dtype;
03855         dtype = -1;
03856         if ( AS.MultiThreaded && ( AC.mparallelflag == PARALLELFLAG ) ) {
03857             for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
03858                 if ( t[2] == ModOptdollars[nummodopt].number ) break;
03859             }
03860             if ( nummodopt < NumModOptdollars ) {
03861                 dtype = ModOptdollars[nummodopt].type;
03862                 if ( dtype == MODLOCAL ) {
03863                     d = ModOptdollars[nummodopt].dstruct+AT.identity;

```

```

03864         }
03865     else {
03866         MLOCK(ErrorMessageLock);
03867         MesPrint("&Illegal attempt to use $-variable %s in module %1",
03868             DOLLARNAME(Dollars,t[2]),AC.CModule);
03869         MUNLOCK(ErrorMessageLock);
03870         Terminate(-1);
03871     }
03872 }
03873 }
03874 }
03875 #endif
03876 if ( d->factors == 0 ) {
03877     MLOCK(ErrorMessageLock);
03878     MesPrint("Attempt to use a factor of an unfactored $-variable");
03879     MUNLOCK(ErrorMessageLock);
03880     Terminate(-1);
03881 }
03882 num = GetDolNum(BHEAD t+t[1],tstop);
03883 if ( num == 0 ) return(d->nfactors);
03884 if ( num > d->nfactors ) {
03885     MLOCK(ErrorMessageLock);
03886     MesPrint("Attempt to use an nonexistent factor %d of a $-variable",num);
03887     MUNLOCK(ErrorMessageLock);
03888     Terminate(-1);
03889 }
03890 w = d->factors[num-1].where;
03891 if ( w == 0 ) return(d->factors[num-1].value);
03892 if ( w[0] == 4 && w[4] == 0 && w[3] == 3 && w[2] == 1 && w[1] > 0
03893     && w[1] < MAXPOSITIVE ) return(w[1]);
03894 else {
03895     MLOCK(ErrorMessageLock);
03896     MesPrint("Illegal type of factor number of a $-variable");
03897     MUNLOCK(ErrorMessageLock);
03898     Terminate(-1);
03899 }
03900 }
03901 else if ( t[2] < 0 ) {
03902     return(-t[2]-1);
03903 }
03904 else {
03905     d = Dollars + t[2];
03906 #ifdef WITHPTHREADS
03907     {
03908         int nummodopt, dtype;
03909         dtype = -1;
03910         if ( AS.MultiThreaded && ( AC.mparallelfld == PARALLELFIELD ) ) {
03911             for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
03912                 if ( t[2] == ModOptdollars[nummodopt].number ) break;
03913             }
03914             if ( nummodopt < NumModOptdollars ) {
03915                 dtype = ModOptdollars[nummodopt].type;
03916                 if ( dtype == MODLOCAL ) {
03917                     d = ModOptdollars[nummodopt].dstruct+AT.identity;
03918                 }
03919                 else {
03920                     MLOCK(ErrorMessageLock);
03921                     MesPrint("&Illegal attempt to use $-variable %s in module %1",
03922                         DOLLARNAME(Dollars,t[2]),AC.CModule);
03923                     MUNLOCK(ErrorMessageLock);
03924                     Terminate(-1);
03925                 }
03926             }
03927         }
03928     }
03929 #endif
03930 if ( d->type == DOLZERO ) return(0);
03931 if ( d->type == DOLTERMS || d->type == DOLNUMBER ) {
03932     if ( d->where[0] == 4 && d->where[4] == 0 && d->where[3] == 3
03933         && d->where[2] == 1 && d->where[1] > 0
03934         && d->where[1] < MAXPOSITIVE ) return(d->where[1]);
03935     MLOCK(ErrorMessageLock);
03936     MesPrint("Attempt to use an nonexistent factor of a $-variable");
03937     MUNLOCK(ErrorMessageLock);
03938     Terminate(-1);
03939 }
03940 MLOCK(ErrorMessageLock);
03941 MesPrint("Illegal type of factor number of a $-variable");
03942 MUNLOCK(ErrorMessageLock);
03943 Terminate(-1);
03944 }
03945 return(0);
03946 }
03947 /*
03948 #] GetDolNum :
03949 #[ AddPotModdollar :

```

```

03951 */
03952
03959 void AddPotModdollar (WORD numdollar)
03960 {
03961     int i, n = NumPotModdollars;
03962     for ( i = 0; i < n; i++ ) {
03963         if ( numdollar == PotModdollars[i] ) break;
03964     }
03965     if ( i >= n ) {
03966         *(WORD *)FromList (&AC.PotModDolList) = numdollar;
03967     }
03968 }
03969
03970 /*
03971     #] AddPotModdollar :
03972 */

```

4.28 evaluate.c

```

00001 /*
00002     #[ includes :
00003 */
00004
00005 #include "form3.h"
00006 #include <stdarg.h>
00007 #include <gmp.h>
00008 #include <mpfr.h>
00009
00010 #define EXTRAPRECISION 4
00011
00012 #define RND MPFR_RNDN
00013
00014 #define auxr1 ((mpfr_t *) (AT.auxr_)) [0]
00015 #define auxr2 ((mpfr_t *) (AT.auxr_)) [1]
00016 #define auxr3 ((mpfr_t *) (AT.auxr_)) [2]
00017 #define auxr4 ((mpfr_t *) (AT.auxr_)) [3]
00018 #define auxr5 ((mpfr_t *) (AT.auxr_)) [4]
00019
00020 mpfr_t ln2;
00021
00022 int PackFloat (WORD *,mpfr_t);
00023 int UnpackFloat (mpfr_t,WORD *);
00024 void RatToFloat (mpfr_t, UWORD *, int);
00025 void FormtoZ (mpz_t,UWORD *,WORD);
00026
00027 /*
00028     #] includes :
00029     #[ mpfr_ :
00030         #[ IntegerToFloatr :
00031
00032             Converts a Form long integer to a mpfr_ float of default size.
00033             We assume that sizeof(unsigned long int) = 2*sizeof(UWORD).
00034 */
00035
00036 void IntegerToFloatr (mpfr_t result, UWORD *formlong, int longsize)
00037 {
00038     mpz_t z;
00039     mpz_init (z);
00040     FormtoZ (z,formlong,longsize);
00041     mpfr_set_z (result,z,RND);
00042     mpz_clear (z);
00043 }
00044
00045 /*
00046     #] IntegerToFloatr :
00047     #[ RatToFloatr :
00048
00049         Converts a Form rational to a gmp float of default size.
00050 */
00051
00052 void RatToFloatr (mpfr_t result, UWORD *formrat, int ratsize)
00053 {
00054     GETIDENTITY
00055     int nnum, nden;
00056     UWORD *num, *den;
00057     int sgn = 0;
00058     if ( ratsize < 0 ) { ratsize = -ratsize; sgn = 1; }
00059     nnum = nden = (ratsize-1)/2;
00060     num = formrat; den = formrat+nnum;
00061     while ( nnum > 1 && num[nnum-1] == 0 ) { nnum--; }
00062     while ( nden > 1 && den[nden-1] == 0 ) { nden--; }
00063     IntegerToFloatr (auxr4,num,nnum);
00064     IntegerToFloatr (auxr5,den,nden);

```

```

00065     mpfr_div(result,auxr4,auxr5,RND);
00066     if ( sgn > 0 ) mpfr_neg(result,result,RND);
00067 }
00068
00069 /*
00070     #] RatToFloat :
00071     #[ SetFloatPrecision :
00072
00073     The AC.DefaultPrecision is in bits.
00074     prec is in limbs.
00075     fprec is NOT in bits for mpfr_ which is slightly less than in mpf_
00076     We also set the auxr1,auxr2,auxr3,auxr4,auxr5 variables.
00077 */
00078
00079 void SetFloatPrecision(LONG prec)
00080 {
00081     /*
00082         mpfr_prec_t fprec = prec * mp_bits_per_limb - EXTRAPRECISION;
00083     */
00084     mpfr_prec_t fprec = prec + 1;
00085     mpfr_set_default_prec(fprec);
00086 #ifdef WITHPTHREADS
00087     int totnum = AM.totalnumberofthreads, id;
00088     mpfr_t *a;
00089 #ifdef WITHSORTBOTS
00090     totnum = MaX(2*AM.totalnumberofthreads-3,AM.totalnumberofthreads);
00091 #endif
00092     if ( AB[0]->T.auxr_ ) {
00093         for ( id = 0; id < totnum; id++ ) {
00094             a = (mpfr_t *)AB[id]->T.auxr_;
00095             mpfr_clears(a[0],a[1],a[2],a[3],a[4],(mpfr_ptr)0);
00096             M_free(AB[id]->T.auxr_,"AB[id]->T.auxr_");
00097             AB[id]->T.auxr_ = 0;
00098         }
00099     }
00100     for ( id = 0; id < totnum; id++ ) {
00101         AB[id]->T.auxr_ = (void *)Malloca(sizeof(mpfr_t)*5,"AB[id]->T.auxr_");
00102         a = (mpfr_t *)AB[id]->T.auxr_;
00103     /*
00104         We work here with a[0] etc because the aux1 etc contain B which
00105         in the current routine would be AB[0] only
00106     */
00107         mpfr_inits2(fprec,a[0],a[1],a[2],a[3],a[4],(mpfr_ptr)0);
00108     }
00109 #else
00110     if ( AT.auxr_ ) {
00111         mpfr_clears(auxr1,auxr2,auxr3,auxr4,auxr5,(mpfr_ptr)0);
00112         M_free(AT.auxr_,"AT.auxr_");
00113         AT.auxr_ = 0;
00114     }
00115     AT.auxr_ = (void *)Malloca(sizeof(mpfr_t)*5,"AT.auxr_");
00116     mpfr_inits2(fprec,auxr1,auxr2,auxr3,auxr4,auxr5,(mpfr_ptr)0);
00117 #endif
00118 }
00119
00120 /*
00121     #] SetFloatPrecision :
00122     #[ ClearFloat :
00123 */
00124
00125 void ClearFloat(void)
00126 {
00127 #ifdef WITHPTHREADS
00128     int totnum = AM.totalnumberofthreads, id;
00129     mpfr_t *a;
00130 #ifdef WITHSORTBOTS
00131     totnum = MaX(2*AM.totalnumberofthreads-3,AM.totalnumberofthreads);
00132 #endif
00133     if ( AB[0]->T.auxr_ ) {
00134         for ( id = 0; id < totnum; id++ ) {
00135             a = (mpfr_t *)AB[id]->T.auxr_;
00136             mpfr_clears(a[0],a[1],a[2],a[3],a[4],(mpfr_ptr)0);
00137             M_free(AB[id]->T.auxr_,"AB[id]->T.auxr_");
00138             AB[id]->T.auxr_ = 0;
00139         }
00140     /*
00141         M_free(AB[0]->T.auxr_,"AB[0]->T.auxr_");
00142         AB[0]->T.auxr_ = 0;
00143     */
00144     }
00145 #else
00146     if ( AT.auxr_ ) {
00147         mpfr_clears(auxr1,auxr2,auxr3,auxr4,auxr5,(mpfr_ptr)0);
00148         M_free(AT.auxr_,"AT.auxr_");
00149         AT.auxr_ = 0;
00150     }
00151 #endif

```

```

00152 /*
00153     if ( AS.delta_1 ) {
00154         mpf_clear(ln2);
00155         AS.delta_1 = 0;
00156     }
00157 */
00158 }
00159
00160 /*
00161     #] ClearfFloat :
00162     #[ GetFloatArgument :
00163
00164     Convert an argument to an mpfr float if possible.
00165     Return value: 0: was converted successfully.
00166                 -1: could not convert.
00167     Note: arguments with more than one term should be evaluated first
00168     inside an Argument environment. If there is still more than one
00169     term remaining we get the code: "could not convert".
00170     par tells which argument we need. If it is negative it should also
00171     be the last argument.
00172 */
00173
00174 int GetFloatArgument(PHEAD mpfr_t f_out,WORD *fun,int par)
00175 {
00176     WORD *term, *tn, *t, *tstop, first, ncoef, *arg, *argp, abspar;
00177     arg = fun+FUNHEAD;
00178     abspar = ABS(par);
00179     while ( arg < fun+fun[1] && abspar > 1 ) { abspar--; NEXTARG(arg) }
00180     if ( arg >= fun+fun[1] || abspar != 1 ) return(-1);
00181     if ( par < 0 ) {
00182         argp = arg; NEXTARG(argp); if ( argp != fun+fun[1] ) return(-1);
00183     }
00184     if ( *arg < 0 ) {
00185         if ( *arg == -SNUMBER ) {
00186             mpfr_set_si(f_out,(LONG)(arg[1]),RND);
00187         }
00188         else if ( *arg == -SYMBOL && ( arg[1] == PISYMBOL ) ) {
00189             mpfr_const_pi(f_out,RND);
00190         }
00191         else if ( *arg == -INDEX && arg[1] >= 0 && arg[1] < AM.OffsetIndex ) {
00192             mpfr_set_ui(f_out,(ULONG)(arg[1]),RND);
00193         }
00194         else { return(-1); }
00195         return(0);
00196     }
00197     else if ( arg[0] != arg[ARGHEAD]+ARGHEAD ) { /* more than one term */
00198         return(-1);
00199     }
00200     term = arg+ARGHEAD;
00201     tn = term+*term;
00202     tstop = tn-ABS(tn[-1]);
00203     t = term+1;
00204     first = 1;
00205     while ( t <= tstop ) {
00206         if ( t == tstop ) { /* Fraction */
00207             if ( first ) RatToFloatr(f_out,(UWORD *)tstop,tn[-1]);
00208             else {
00209                 RatToFloatr(auxr4,(UWORD *)tstop,tn[-1]);
00210                 mpfr_mul(f_out,f_out,auxr4,RND);
00211             }
00212             return(0);
00213         }
00214         else if ( *t == FLOATFUN ) {
00215             if ( t+t[1] != tstop ) return(-1);
00216             if ( UnpackFloat(aux5,t) < 0 ) return(-1);
00217             if ( first ) {
00218                 mpfr_set_f(f_out,aux5,RND);
00219                 first = 0;
00220             }
00221             else {
00222                 mpfr_set_f(auxr4,aux5,RND);
00223                 mpfr_mul(f_out,f_out,auxr4,RND);
00224             }
00225             first = 0;
00226             if ( tn[-1] < 0 ) { /* change sign */
00227                 ncoef = -tn[-1];
00228                 mpfr_neg(f_out,f_out,RND);
00229             }
00230             else ncoef = tn[-1];
00231             if ( ncoef == 3 && tn[-2] == 1 && tn[-3] == 1 ) return(0);
00232         }
00233         /*
00234         If the argument was properly normalized we are not supposed to come here.
00235         */
00236         MLOCK(ErrorMessageLock);
00237         MesPrint("Unnormalized argument in GetFloatArgument: %a",*term,term);
00238         MUNLOCK(ErrorMessageLock);
00239         Terminate(-1);

```

```

00239     }
00240     else if ( t[1] == SYMBOL && t[2] == 4 && t[3] == PISYMBOL && t[4] == 1 ) {
00241         if ( first ) {
00242             mpfr_const_pi(f_out,RND);
00243             first = 0;
00244         }
00245         else {
00246             mpfr_const_pi(auxr5,RND);
00247             mpfr_mul(f_out,f_out,auxr5,RND);
00248         }
00249     }
00250     else { /* This we do not / cannot do */
00251         return(-1);
00252     }
00253     t += t[1];
00254 }
00255 return(0);
00256 }
00257
00258 /*
00259     #] GetFloatArgument :
00260     #[ GetPiArgument :
00261
00262     Tests for sin,cos,tan whether the argument is a simple
00263     multiple of pi or can be reduced to such.
00264     Return value: -1: the answer is no.
00265     0-23: we have 0, 1/12*pi,...,23/12*pi_
00266     With success most values can be worked out by simple means.
00267 */
00268
00269 int GetPiArgument(PHEAD WORD *arg)
00270 {
00271     UWORD *coef, *co, *tco, twelve, *number, *denom, *c, *d;
00272     WORD i, ii, i2, iflip, nc, nd, *t, *tn, *tstop;
00273     int rem;
00274     /*
00275     One: determine whether there is a rational coefficient and a pi_:
00276     */
00277     if ( *arg == -SNUMBER && arg[1] == 0 ) return(0);
00278     if ( *arg == -SYMBOL && arg[1] == PISYMBOL ) return(12);
00279     if ( *arg < 0 ) return(-1);
00280     if ( arg[ARGHEAD] != *arg-ARGHEAD ) return(-1);
00281     t = arg+ARGHEAD+1;
00282     tn = arg+arg; tstop = tn - ABS(tn[-1]);
00283     if ( *t != SYMBOL || t[1] != 4 || t[2] != PISYMBOL || t[3] != 1
00284         || t+t[1] != tstop ) return(-1);
00285     /*
00286     The denominator must be a divisor of 12
00287     1: copy the coefficient
00288     */
00289     co = coef = NumberMalloc("GetPiArgument");
00290     tco = (UWORD *)tstop;
00291     i = tn[-1]; if ( i < 0 ) { i = -i; iflip = 1; } else iflip = 0;
00292     ii = (i-1)/2;
00293     twelve = 12;
00294     NCOPY(co,tco,i);
00295     co = coef;
00296     Mully(BHEAD co,&ii,&twelve,1);
00297     /*
00298     Now the denominator should be 1
00299     i2 = i = (ii-1)/2;
00300     */
00301     i2 = i = ii;
00302     number = co; denom = number + i;
00303     if ( i > 1 ) {
00304         while ( i2 > 0 ) {
00305             if ( denom[i2--] != 0 ) {
00306                 NumberFree(co,"GetPiArgument");
00307                 return(-1);
00308             }
00309         }
00310     }
00311     if ( *denom != 1 ) {
00312         NumberFree(co,"GetPiArgument");
00313         return(-1);
00314     }
00315     /*
00316     Now we need the numerator modulus 24.
00317     */
00318     if ( i == 1 ) {
00319         rem = *number % 24;
00320     }
00321     else {
00322         c = NumberMalloc("GetPiArgument");
00323         d = NumberMalloc("GetPiArgument");
00324         twelve *= 2;
00325         DivLong(number,i,&twelve,1,c,&nc,d,&nd);

```

```

00326         rem = *d % 24;
00327         NumberFree(d,"GetPiArgument");
00328         NumberFree(c,"GetPiArgument");
00329     }
00330     NumberFree(co,"GetPiArgument");
00331     if ( iflip && rem != 0 ) rem = 24-rem;
00332     return(rem);
00333 }
00334
00335 /*
00336     #] GetPiArgument :
00337     #[ EvaluateFun :
00338
00339     What we need to do is:
00340     1: look for a function to be treated.
00341     2: make sure its argument is treatable.
00342     3: call the proper mpfr_ function.
00343     4: accumulate the result.
00344     For some functions we have to insert 'smart' shortcuts as is
00345     the case with sin_(pi_) or sin_(pi_/6) of sqrt(4/9) etc.
00346     Otherwise we may have to insert a value for pi_ first.
00347     There are several types of arguments:
00348     a: (short) integers.
00349     b: rationals.
00350     c: floats.
00351     We accumulate the result(s) in auxr2. The argument comes in aux5
00352     and a result in auxr3 which then gets multiplied into auxr2.
00353     In the end we have to combine auxr2 with whatever coefficient
00354     existed already.
00355
00356     When the float system is started we need for aux only 3 variables
00357     per thread. auxr1-auxr3. This should be done separately.
00358     The main problem is the conversion of mpfr_t to float_ and/or mpf_t
00359     and back.
00360 */
00361
00362 int EvaluateFun(PHEAD WORD *term, WORD level, WORD *pars)
00363 {
00364     WORD *t, *tstop, *tt, *tnext, *newterm, i;
00365     WORD *oldworkpointer = AT.WorkPointer, nsize, nsgn;
00366     int retval = 0, first = 1, pimul;
00367
00368     tstop = term + *term; tstop -= ABS(tstop[-1]);
00369     if ( AT.WorkPointer < term+*term ) AT.WorkPointer = term + *term;
00370     t = term+1;
00371     mpfr_set_ui(auxr2,1L,RND);
00372     while ( t < tstop ) {
00373         if ( pars[2] == *t ) { /* have to do this one if possible */
00374             TestArgument:
00375             /*
00376                 There must be a single argument, except for the AGM function
00377             */
00378             tnext = t+t[1]; tt = t+FUNHEAD; NEXTARG(tt);
00379             if( *t == SYMBOL ) {
00380                 for ( WORD* ti = t+2; ti < t+t[1]; ti+=2 ) {
00381                     if( ( *ti == PISYMBOL || *ti == EESYMBOL || *ti == EMSYMBOL )
00382                         && ( pars[2] == ALLFUNCTIONS || pars[3] == *ti ) ) {
00383                         if ( *ti == PISYMBOL )
00384                             mpfr_const_pi(auxr3,RND);
00385                         else if ( *ti == EESYMBOL ) {
00386                             mpfr_set_ui(auxr3,1,RND);
00387                             mpfr_exp(auxr3,auxr3,RND);
00388                         }
00389                         else if ( *ti == EMSYMBOL )
00390                             mpfr_const_euler(auxr3,RND);
00391                         if ( ti[1] != 1 )
00392                             mpfr_pow_si(auxr3,auxr3,ti[1],RND);
00393                         mpfr_mul(auxr2,auxr2,auxr3,RND);
00394                         ti[1] = 0;
00395                     }
00396                 }
00397                 first = 0;
00398                 goto nextfun;
00399             }
00400             if ( tt != tnext && *t != AGMFUNCTION ) goto nextfun;
00401             if ( *t == SINFUNCTION ) {
00402                 pimul = GetPiArgument(BHEAD t+FUNHEAD);
00403                 if ( pimul >= 0 && pimul < 24 ) {
00404                     if ( pimul == 0 || pimul == 12 ) goto getout;
00405                     if ( pimul > 12 ) { pimul = 24-pimul; nsgn = -1; }
00406                     else nsgn = 1;
00407                     if ( pimul > 6 ) pimul = 12-pimul;
00408                     switch ( pimul ) {
00409                         case 1:
00410                             label1:
00411                             mpfr_sqrt_ui(auxr3,3L,RND);

```



```

00413             mpfr_sub_ui (auxr3, auxr3, 1L, RND);
00414             mpfr_sqrt_ui (auxr1, 2L, RND);
00415             mpfr_div_ui (auxr1, auxr1, 4L, RND);
00416             mpfr_mul (auxr3, auxr3, auxr1, RND);
00417             if ( nsgn < 0 ) mpfr_neg (auxr3, auxr3, RND);
00418             mpfr_mul (auxr2, auxr2, auxr3, RND);
00419             break;
00420         case 2:
00421 label2:
00422             mpfr_div_ui (auxr2, auxr2, 2L, RND);
00423             if ( nsgn < 0 ) mpfr_neg (auxr2, auxr2, RND);
00424             break;
00425         case 3:
00426 label3:
00427             mpfr_sqrt_ui (auxr3, 2L, RND);
00428             mpfr_div_ui (auxr3, auxr3, 2L, RND);
00429             if ( nsgn < 0 ) mpfr_neg (auxr3, auxr3, RND);
00430             mpfr_mul (auxr2, auxr2, auxr3, RND);
00431             break;
00432         case 4:
00433 label4:
00434             mpfr_sqrt_ui (auxr3, 3L, RND);
00435             mpfr_div_ui (auxr3, auxr3, 2L, RND);
00436             if ( nsgn < 0 ) mpfr_neg (auxr3, auxr3, RND);
00437             mpfr_mul (auxr2, auxr2, auxr3, RND);
00438             break;
00439         case 5:
00440 label5:
00441             mpfr_sqrt_ui (auxr3, 3L, RND);
00442             mpfr_add_ui (auxr3, auxr3, 1L, RND);
00443             mpfr_sqrt_ui (auxr1, 2L, RND);
00444             mpfr_div_ui (auxr1, auxr1, 4L, RND);
00445             mpfr_mul (auxr3, auxr3, auxr1, RND);
00446             if ( nsgn < 0 ) mpfr_neg (auxr3, auxr3, RND);
00447             mpfr_mul (auxr2, auxr2, auxr3, RND);
00448             break;
00449         case 6:
00450 label6:
00451             if ( nsgn < 0 ) mpfr_neg (auxr2, auxr2, RND);
00452             break;
00453     }
00454     *t = 0; first = 0;
00455     goto nextfun;
00456 }
00457 }
00458 else if ( *t == COSFUNCTION ) {
00459     pimul = GetPiArgument (BHEAD t+FUNHEAD);
00460     if ( pimul >= 0 && pimul < 24 ) {
00461         if ( pimul > 12 ) pimul = 24-12;
00462         if ( pimul > 6 ) { pimul = 12-pimul; nsgn = -1; }
00463         else nsgn = 1;
00464         if ( pimul == 6 ) goto getout;
00465         switch ( pimul ) {
00466             case 0: goto label6;
00467             case 1: goto label5;
00468             case 2: goto label4;
00469             case 3: goto label3;
00470             case 4: goto label2;
00471             case 5: goto label1;
00472         }
00473     }
00474 }
00475 else if ( *t == TANFUNCTION ) {
00476     pimul = GetPiArgument (BHEAD t+FUNHEAD);
00477     if ( pimul >= 0 && pimul < 24 ) {
00478         if ( pimul == 6 || pimul == 18 ) goto nextfun;
00479         if ( pimul == 0 || pimul == 12 ) goto getout;
00480         if ( pimul > 12 ) { pimul = 24-pimul; nsgn = -1; }
00481         else nsgn = 1;
00482         if ( pimul > 6 ) { pimul = 12-pimul; nsgn = -nsgn; }
00483         switch ( pimul ) {
00484             case 1:
00485                 mpfr_sqrt_ui (auxr3, 3L, RND);
00486                 mpfr_sub_ui (auxr3, auxr3, 2L, RND);
00487                 if ( nsgn > 0 ) mpfr_neg (auxr3, auxr3, RND);
00488                 mpfr_mul (auxr2, auxr2, auxr3, RND);
00489                 break;
00490             case 2:
00491                 mpfr_sqrt_ui (auxr3, 3L, RND);
00492                 mpfr_div_ui (auxr3, auxr3, 3L, RND);
00493                 if ( nsgn < 0 ) mpfr_neg (auxr3, auxr3, RND);
00494                 mpfr_mul (auxr2, auxr2, auxr3, RND);
00495                 break;
00496             case 3:
00497                 break;
00498             case 4:
00499                 mpfr_sqrt_ui (auxr3, 3L, RND);

```

```

00500             if ( nsgn < 0 ) mpfr_neg(auxr3,auxr3,RND);
00501             mpfr_mul(auxr2,auxr2,auxr3,RND);
00502             break;
00503         case 5:
00504             mpfr_sqrt_ui(auxr3,3L,RND);
00505             mpfr_add_ui(auxr3,auxr3,2L,RND);
00506             if ( nsgn < 0 ) mpfr_neg(auxr3,auxr3,RND);
00507             mpfr_mul(auxr2,auxr2,auxr3,RND);
00508             break;
00509     }
00510     *t = 0;
00511     goto nextfun;
00512 }
00513 }
00514
00515 if ( *t == AGMFUNCTION ) {
00516     if ( GetFloatArgument(BHEAD auxr1,t,1) < 0 ) goto nextfun;
00517     if ( GetFloatArgument(BHEAD auxr3,t,-2) < 0 ) goto nextfun;
00518 }
00519 else if ( GetFloatArgument(BHEAD auxr1,t,-1) < 0 ) goto nextfun;
00520 nsgn = mpfr_sgn(auxr1);
00521
00522 switch ( *t ) {
00523     case SQRTFUNCTION:
00524         if ( nsgn < 0 ) goto nextfun;
00525         if ( nsgn == 0 ) goto getout;
00526         else mpfr_sqrt(auxr3,auxr1,RND);
00527         mpfr_mul(auxr2,auxr2,auxr3,RND);
00528         break;
00529     case LNFUNCTION:
00530         if ( nsgn <= 0 ) goto nextfun;
00531         else mpfr_log(auxr3,auxr1,RND);
00532         mpfr_mul(auxr2,auxr2,auxr3,RND);
00533         break;
00534     case LI2FUNCTION: /* should be between -1 and +1 */
00535         mpfr_abs(auxr3,auxr1,RND);
00536         if ( mpfr_cmp_ui(auxr3,1L) > 0 ) goto nextfun;
00537         mpfr_li2(auxr3,auxr1,RND);
00538         mpfr_mul(auxr2,auxr2,auxr3,RND);
00539         break;
00540     case GAMMAFUN:
00541         /*
00542          We cannot do this when the argument is a non-positive integer
00543          */
00544         if ( t[FUNHEAD] == -SNUMBER && t[FUNHEAD+1] <= 0 ) goto nextfun;
00545         if ( t[FUNHEAD] == t[1]-FUNHEAD && ABS(t[t[1]-1]) ==
00546             t[FUNHEAD]-ARGHEAD-1 && t[t[1]-1] < 0 ) {
00547             nsize = (-t[t[1]-1]-1)/2;
00548             if ( t[t[1]-1-nsize] == 1 ) {
00549                 for ( i = 1; i < nsize; i++ ) {
00550                     if ( t[t[1]-1-nsize+i] != 0 ) break;
00551                 }
00552                 if ( i >= nsize ) goto nextfun;
00553             }
00554         }
00555         mpfr_gamma(auxr3,auxr1,RND);
00556         mpfr_mul(auxr2,auxr2,auxr3,RND);
00557         break;
00558     case AGMFUNCTION:
00559         mpfr_agm(auxr3,auxr1,auxr3,RND);
00560         mpfr_mul(auxr2,auxr2,auxr3,RND);
00561         break;
00562     case SINHFUNCTION:
00563         mpfr_sinh(auxr3,auxr1,RND);
00564         mpfr_mul(auxr2,auxr2,auxr3,RND);
00565         break;
00566     case COSHFUNCTION:
00567         mpfr_cosh(auxr3,auxr1,RND);
00568         mpfr_mul(auxr2,auxr2,auxr3,RND);
00569         break;
00570     case TANHFUNCTION:
00571         mpfr_tanh(auxr3,auxr1,RND);
00572         mpfr_mul(auxr2,auxr2,auxr3,RND);
00573         break;
00574     case ASINHFUNCTION:
00575         mpfr_asinh(auxr3,auxr1,RND);
00576         mpfr_mul(auxr2,auxr2,auxr3,RND);
00577         break;
00578     case ACOSHFUNCTION:
00579         mpfr_acosh(auxr3,auxr1,RND);
00580         mpfr_mul(auxr2,auxr2,auxr3,RND);
00581         break;
00582     case ATANHFUNCTION:
00583         if ( nsgn == 0 ) goto getout;
00584         mpfr_abs(auxr3,auxr1,RND);
00585         if ( mpfr_cmp_ui(auxr3,1L) > 0 ) goto nextfun;
00586         mpfr_atanh(auxr3,auxr1,RND);

```

```

00587         mpfr_mul(auxr2,auxr2,auxr3,RND);
00588     break;
00589     case ASINFUNCTION:
00590         if ( nsgn == 0 ) goto getout;
00591         mpfr_abs(auxr3,auxr1,RND);
00592         if ( mpfr_cmp_ui(auxr3,1L) > 0 ) goto nextfun;
00593         mpfr_asin(auxr3,auxr1,RND);
00594         mpfr_mul(auxr2,auxr2,auxr3,RND);
00595     break;
00596     case ACOSFUNCTION:
00597         if ( nsgn == 0 ) goto getout;
00598         mpfr_abs(auxr3,auxr1,RND);
00599         if ( mpfr_cmp_ui(auxr3,1L) > 0 ) goto nextfun;
00600         mpfr_acos(auxr3,auxr1,RND);
00601         mpfr_mul(auxr2,auxr2,auxr3,RND);
00602     break;
00603     case ATANFUNCTION:
00604         mpfr_atan(auxr3,auxr1,RND);
00605         mpfr_mul(auxr2,auxr2,auxr3,RND);
00606     break;
00607     case SINFUNCTION:
00608         mpfr_sin(auxr3,auxr1,RND);
00609         mpfr_mul(auxr2,auxr2,auxr3,RND);
00610     break;
00611     case COSFUNCTION:
00612         mpfr_cos(auxr3,auxr1,RND);
00613         mpfr_mul(auxr2,auxr2,auxr3,RND);
00614     break;
00615     case TANFUNCTION:
00616         mpfr_tan(auxr3,auxr1,RND);
00617         mpfr_mul(auxr2,auxr2,auxr3,RND);
00618     break;
00619     default:
00620         goto nextfun;
00621     break;
00622 }
00623 first = 0;
00624 *t = 0;
00625 goto nextfun;
00626 }
00627 else if ( pars[2] == ALLFUNCTIONS ) {
00628     /*
00629         Now we have to test whether this is one of our functions
00630     */
00631     if ( t[1] == FUNHEAD ) goto nextfun;
00632     switch ( *t ) {
00633     case SQRTFUNCTION:
00634     case LNFUNCTION:
00635     case LI2FUNCTION:
00636     case LINFUNCTION:
00637     case ASINFUNCTION:
00638     case ACOSFUNCTION:
00639     case ATANFUNCTION:
00640     case SINHFUNCTION:
00641     case COSHFUNCTION:
00642     case TANHFUNCTION:
00643     case ASINHFUNCTION:
00644     case ACOSHFUNCTION:
00645     case ATANHFUNCTION:
00646     case SINFUNCTION:
00647     case COSFUNCTION:
00648     case TANFUNCTION:
00649     case AGMFUNCTION:
00650     case SYMBOL:
00651         goto TestArgument;
00652     case MZV:
00653     case EULER:
00654     case MZVHALF:
00655         goto nextfun;
00656     default:
00657         MLOCK(ErrorMessageLock);
00658         MesPrint("Function in evaluate statement not yet implemented.");
00659         MUNLOCK(ErrorMessageLock);
00660         break;
00661     }
00662 }
00663 else goto nextfun;
00664 nextfun:
00665     t += t[1];
00666 }
00667 if ( first == 1 ) return(Generator(BHEAD term,level));
00668 mpfr_get_f(aux4,auxr2,RND);
00669 /*
00670     Step 3:
00671     Now the regular coefficient, if it is not 1/l.
00672     We have two cases: size +- 3, or bigger.
00673 */

```

```

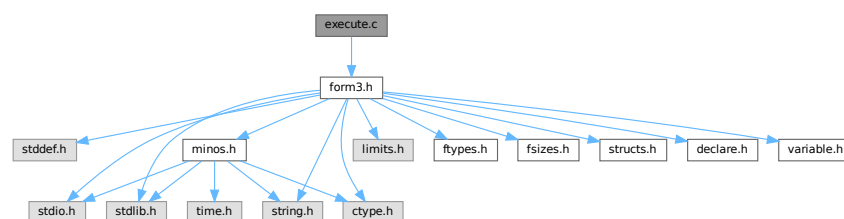
00674
00675     nsize = term[*term-1];
00676     if ( nsize < 0 ) { nsize = -nsize; nsgn = -1; }
00677     else nsgn = 1;
00678     if ( aux4->_mp_size < 0 ) {
00679         aux4->_mp_size = -aux4->_mp_size;
00680         nsgn = -nsgn;
00681     }
00682
00683     if ( nsize == 3 ) {
00684         if ( tstop[0] != 1 ) {
00685             mpf_mul_ui(aux4,aux4, (ULONG) ((UWORD)tstop[0]));
00686         }
00687         if ( tstop[1] != 1 ) {
00688             mpf_div_ui(aux4,aux4, (ULONG) ((UWORD)tstop[1]));
00689         }
00690     }
00691     else {
00692         RatToFloat(aux5, (UWORD *)tstop, nsize);
00693         mpf_mul(aux4,aux4,aux5);
00694     }
00695     /*
00696     Now we have to locate possible other float_ functions.
00697     Note possible incompatibilities between the mpf and mpfr formats.
00698     */
00699     t = term+1;
00700     while ( t < tstop ) {
00701         if ( *t == FLOATFUN ) {
00702             UnpackFloat(aux5,t);
00703             mpf_mul(aux4,aux4,aux5);
00704         }
00705         t += t[1];
00706     }
00707     /*
00708     Now we should compose the new term in the Workspace.
00709     */
00710     t = term+1;
00711     newterm = AT.WorkPointer;
00712     tt = newterm+1;
00713     while ( t < tstop ) {
00714         if ( *t == 0 || *t == FLOATFUN ) t += t[1];
00715         else {
00716             i = t[1]; NCOPY(tt,t,i);
00717         }
00718     }
00719     PackFloat(tt,aux4);
00720     tt += tt[1];
00721     *tt++ = 1; *tt++ = 1; *tt++ = 3*nsgn;
00722     *newterm = tt-newterm;
00723     AT.WorkPointer = tt;
00724     retval = Generator(BHEAD newterm,level);
00725 getout:
00726     AT.WorkPointer = oldworkpointer;
00727     return(retval);
00728 }
00729
00730 /*
00731     #] EvaluateFun :
00732     #] mpfr_ :
00733     */

```

4.29 execute.c File Reference

```
#include "form3.h"
```

Include dependency graph for execute.c:



Macros

- `#define CURRENTBRACKET 1`
- `#define BRACKETCURRENTEXPR 2`
- `#define BRACKETOTHEREXPR 3`
- `#define NOBRACKETACTIVE 4`

Functions

- WORD [CleanExpr](#) (WORD par)
- WORD [PopVariables](#) (void)
- void [MakeGlobal](#) (void)
- void [TestDrop](#) (void)
- void [PutInVflags](#) (WORD nexpr)
- WORD [DoExecute](#) (WORD par, WORD skip)
- WORD [PutBracket](#) (PHEAD WORD *termin)
- void [SpecialCleanup](#) (PHEAD0)
- void [SetMods](#) (void)
- void [UnSetMods](#) (void)
- void [ExchangeExpressions](#) (int num1, int num2)
- int [GetFirstBracket](#) (WORD *term, int num)
- int [GetFirstTerm](#) (WORD *term, int num, int pre)
- int [GetContent](#) (WORD *content, int num)
- int [CleanupTerm](#) (WORD *term)
- WORD [ContentMerge](#) (PHEAD WORD *content, WORD *term)
- LONG [TermsInExpression](#) (WORD num)
- LONG [SizeOfExpression](#) (WORD num)
- void [UpdatePositions](#) (void)
- LONG [CountTerms1](#) (PHEAD0)
- LONG [TermsInBracket](#) (PHEAD WORD *term, WORD level)

4.29.1 Detailed Description

The routines that start the execution phase of a module. It also contains the routines for placing the bracket subterm.

Definition in file [execute.c](#).

4.29.2 Macro Definition Documentation

4.29.2.1 CURRENTBRACKET

```
#define CURRENTBRACKET 1
```

Definition at line [2564](#) of file [execute.c](#).

4.29.2.2 BRACKETCURRENTEXPR

```
#define BRACKETCURRENTEXPR 2
```

Definition at line [2565](#) of file [execute.c](#).

4.29.2.3 BRACKETOTHEREXPR

```
#define BRACKETOTHEREXPR 3
```

Definition at line [2566](#) of file [execute.c](#).

4.29.2.4 NOBRACKETACTIVE

```
#define NOBRACKETACTIVE 4
```

Definition at line [2567](#) of file [execute.c](#).

4.29.3 Function Documentation

4.29.3.1 CleanExpr()

```
WORD CleanExpr (
    WORD par )
```

Definition at line [47](#) of file [execute.c](#).

4.29.3.2 PopVariables()

```
WORD PopVariables (
    void )
```

Definition at line [211](#) of file [execute.c](#).

4.29.3.3 MakeGlobal()

```
void MakeGlobal (
    void )
```

Definition at line [382](#) of file [execute.c](#).

4.29.3.4 TestDrop()

```
void TestDrop (
    void )
```

Definition at line [483](#) of file [execute.c](#).

4.29.3.5 PutInVflags()

```
void PutInVflags (
    WORD nexpr )
```

Definition at line [561](#) of file [execute.c](#).

4.29.3.6 DoExecute()

```
WORD DoExecute (
    WORD par,
    WORD skip )
```

Definition at line 615 of file [execute.c](#).

4.29.3.7 PutBracket()

```
WORD PutBracket (
    PHEAD WORD * termin )
```

Definition at line 1111 of file [execute.c](#).

4.29.3.8 SpecialCleanup()

```
void SpecialCleanup (
    PHEAD0 )
```

Definition at line 1623 of file [execute.c](#).

4.29.3.9 SetMods()

```
void SetMods (
    void )
```

Definition at line 1637 of file [execute.c](#).

4.29.3.10 UnSetMods()

```
void UnSetMods (
    void )
```

Definition at line 1655 of file [execute.c](#).

4.29.3.11 ExchangeExpressions()

```
void ExchangeExpressions (
    int num1,
    int num2 )
```

Definition at line 1670 of file [execute.c](#).

4.29.3.12 GetFirstBracket()

```
int GetFirstBracket (
    WORD * term,
    int num )
```

Definition at line [1744](#) of file [execute.c](#).

4.29.3.13 GetFirstTerm()

```
int GetFirstTerm (
    WORD * term,
    int num,
    int pre )
```

Gets the first term of an expression. Routine should be thread safe.

Parameters

out	<i>term</i>	Pointer to the buffer where the term should be written.
in	<i>num</i>	The expression number from which to get the term.
in	<i>pre</i>	Denotes a pre-processor call (1) or not (0). Used to determine the correct source file for the expression.

Returns

First WORD of term, a negative value denotes an error.

Definition at line 1863 of file [execute.c](#).

References [FiLe::handle](#), [VaRrEnUm::lo](#), and [ReNuMbEr::symb](#).

4.29.3.14 GetContent()

```
int GetContent (
    WORD * content,
    int num )
```

Definition at line 1971 of file [execute.c](#).

4.29.3.15 CleanupTerm()

```
int CleanupTerm (
    WORD * term )
```

Definition at line 2088 of file [execute.c](#).

4.29.3.16 ContentMerge()

```
WORD ContentMerge (
    PHEAD WORD * content,
    WORD * term )
```

Definition at line 2112 of file [execute.c](#).

4.29.3.17 TermsInExpression()

```
LONG TermsInExpression (
    WORD num )
```

Definition at line 2376 of file [execute.c](#).

4.29.3.18 SizeOfExpression()

```
LONG SizeOfExpression (
    WORD num )
```

Definition at line [2388](#) of file [execute.c](#).

4.29.3.19 UpdatePositions()

```
void UpdatePositions (
    void )
```

Definition at line [2400](#) of file [execute.c](#).

4.29.3.20 CountTerms1()

```
LONG CountTerms1 (
    PHEAD0 )
```

Definition at line [2461](#) of file [execute.c](#).

4.29.3.21 TermsInBracket()

```
LONG TermsInBracket (
    PHEAD WORD * term,
    WORD level )
```

Definition at line [2569](#) of file [execute.c](#).

4.30 execute.c

[Go to the documentation of this file.](#)

```
00001
00006 /* #[ License : */
00007 /*
00008 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00009 * When using this file you are requested to refer to the publication
00010 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00011 * This is considered a matter of courtesy as the development was paid
00012 * for by FOM the Dutch physics granting agency and we would like to
00013 * be able to track its scientific use to convince FOM of its value
00014 * for the community.
00015 *
00016 * This file is part of FORM.
00017 *
00018 * FORM is free software: you can redistribute it and/or modify it under the
00019 * terms of the GNU General Public License as published by the Free Software
00020 * Foundation, either version 3 of the License, or (at your option) any later
00021 * version.
00022 *
00023 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00024 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00025 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00026 * details.
00027 *
00028 * You should have received a copy of the GNU General Public License along
00029 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00030 */
00031 /* #[ License : */
```

```

00032 /*
00033     #[ Includes : execute.c
00034 */
00035
00036 #include "form3.h"
00037
00038 /*
00039     #] Includes :
00040     #[ DoExecute :
00041         #[ CleanExpr :
00042
00043         par == 1 after .store or .clear
00044         par == 0 after .sort
00045 */
00046
00047 WORD CleanExpr(WORD par)
00048 {
00049     GETIDENTITY
00050     WORD j, n, i;
00051     POSITION length;
00052     EXPRESSIONS e_in, e_out, e;
00053     int numhid = 0;
00054     NAMENODE *node;
00055     n = NumExpressions;
00056     j = 0;
00057     e_in = e_out = Expressions;
00058     if ( n > 0 ) { do {
00059         e_in->vflags &= ~( TOBEFACTORED | TOBEUNFACTORED );
00060         if ( par ) {
00061             if ( e_in->renumlists ) {
00062                 if ( e_in->renumlists != AN.dummyrenumlist )
00063                     M_free(e_in->renumlists, "Renumber-lists");
00064                 e_in->renumlists = 0;
00065             }
00066             if ( e_in->renum ) {
00067                 M_free(e_in->renum, "Renumber"); e_in->renum = 0;
00068             }
00069         }
00070         if ( e_in->status == HIDDENEXPRESSION
00071             || e_in->status == HIDDENEXPRESSION ) numhid++;
00072         switch ( e_in->status ) {
00073             case SPECTATOREXPRESSION:
00074             case LOCALEXPRESSION:
00075             case HIDDENEXPRESSION:
00076                 if ( par ) {
00077                     AC.exprnames->namenode[e_in->node].type = CDELETE;
00078                     AC.DidClean = 1;
00079                     if ( e_in->status != HIDDENEXPRESSION )
00080                         ClearBracketIndex(e_in->Expressions);
00081                     break;
00082                 }
00083                 /* fall through */
00084             case GLOBALEXPRESSION:
00085             case HIDDENEXPRESSION:
00086                 if ( par ) {
00087 #ifdef WITHMPI
00088                     /*
00089                      * Broadcast the global expression from the master to the all workers.
00090                      */
00091                     if ( PF_BroadcastExpr(e_in, e_in->status == HIDDENEXPRESSION ? AR.hidefile :
AR.outfile) ) return -1;
00092                     if ( PF.me == MASTER ) {
00093 #endif
00094                         e = e_in;
00095                         i = n-1;
00096                         while ( --i >= 0 ) {
00097                             e++;
00098                             if ( e_in->status == HIDDENEXPRESSION ) {
00099                                 if ( e->status == HIDDENEXPRESSION
00100                                     || e->status == HIDDENEXPRESSION ) break;
00101                             }
00102                             else {
00103                                 if ( e->status == GLOBALEXPRESSION
00104                                     || e->status == LOCALEXPRESSION ) break;
00105                             }
00106                         }
00107 #ifdef WITHMPI
00108                     }
00109                     else {
00110                         /*
00111                          * On the slaves, the broadcast expression is sitting at the end of the file.
00112                          */
00113                         e = e_in;
00114                         i = -1;
00115                     }
00116 #endif
00117                     if ( i >= 0 ) {

```

```

00118         DIFPOS(length,e->onfile,e_in->onfile);
00119     }
00120     else {
00121         FILEHANDLE *f = e_in->status == HIDDENEXPRESSION ? AR.hidefile : AR.outfile;
00122         if ( f->handle < 0 ) {
00123             SETBASELENGTH(length,TOLONG(f->POfull)
00124                 - TOLONG(f->PObuffer)
00125                 - BASEPOSITION(e_in->onfile));
00126         }
00127         else {
00128             SeekFile(f->handle,&(f->filesize),SEEK_SET);
00129             DIFPOS(length,f->filesize,e_in->onfile);
00130         }
00131     }
00132     if ( ToStorage(e_in,&length) ) {
00133         return(MesCall("CleanExpr"));
00134     }
00135     e_in->status = STOREDEXPRESSION;
00136     if ( e_in->status != HIDDENEXPRESSION )
00137         ClearBracketIndex(e_in->Expressions);
00138 }
00139 /* fall through */
00140 case SKIPLEXPRESSION:
00141 case DROPLEXPRESSION:
00142 case DROPHLEXPRESSION:
00143 case DROPGEXPRESSION:
00144 case DROPHGEXPRESSION:
00145 case STOREDEXPRESSION:
00146 case DROPSPECTATOREXPRESSION:
00147     if ( e_out != e_in ) {
00148         node = AC.exprnames->namenode + e_in->node;
00149         node->number = e_out - Expressions;
00150     }
00151     e_out->onfile = e_in->onfile;
00152     e_out->size = e_in->size;
00153     e_out->printflag = 0;
00154     if ( par ) e_out->status = STOREDEXPRESSION;
00155     else e_out->status = e_in->status;
00156     e_out->name = e_in->name;
00157     e_out->node = e_in->node;
00158     e_out->renum = e_in->renum;
00159     e_out->renumlists = e_in->renumlists;
00160     e_out->counter = e_in->counter;
00161     e_out->hidelevel = e_in->hidelevel;
00162     e_out->inmem = e_in->inmem;
00163     e_out->bracketinfo = e_in->bracketinfo;
00164     e_out->newbracketinfo = e_in->newbracketinfo;
00165     e_out->numdummies = e_in->numdummies;
00166     e_out->numfactors = e_in->numfactors;
00167     e_out->vflags = e_in->vflags;
00168     e_out->uflags = e_in->uflags;
00169     e_out->sizeprototype = e_in->sizeprototype;
00170 }
00171 #ifdef PARALLELCODE
00172     e_out->partodo = 0;
00173 #endif
00174     e_out++;
00175     j++;
00176     break;
00177 case DROPEDEXPRESSION:
00178     break;
00179 default:
00180     AC.exprnames->namenode[e_in->node].type = CDELETE;
00181     AC.DidClean = 1;
00182     break;
00183 }
00184     e_in++;
00185 } while ( --n > 0 ); }
00186 UpdateMaxSize();
00187 NumExpressions = j;
00188 if ( numhid == 0 && AR.hidefile->PObuffer ) {
00189     if ( AR.hidefile->handle >= 0 ) {
00190         CloseFile(AR.hidefile->handle);
00191         remove(AR.hidefile->name);
00192         AR.hidefile->handle = -1;
00193     }
00194     AR.hidefile->POfull =
00195     AR.hidefile->POfill = AR.hidefile->PObuffer;
00196     PUTZERO(AR.hidefile->POposition);
00197 }
00198 FlushSpectators();
00199 return(0);
00200 }
00201
00202 /*
00203     #] CleanExpr :
00204     #[ PopVariables :

```

```

00205
00206     Pops the local variables from the tables.
00207     The Expressions are reprocessed and their tables are compactified.
00208
00209 */
00210
00211 WORD PopVariables(void)
00212 {
00213     GETIDENTITY
00214     WORD i, j, retval;
00215     UBYTE *s;
00216
00217     retval = CleanExpr(1);
00218     ResetVariables(1);
00219
00220     if ( AC.DidClean ) CompactifyTree(AC.exprnames,EXPRNAMES);
00221
00222     AC.CodesFlag = AM.gCodesFlag;
00223     AC.NamesFlag = AM.gNamesFlag;
00224     AC.StatsFlag = AM.gStatsFlag;
00225 #ifdef WITHFLOAT
00226     AC.MaxWeight = AM.gMaxWeight;
00227     AC.DefaultPrecision = AM.gDefaultPrecision;
00228 #endif
00229     AC.OldFactArgFlag = AM.gOldFactArgFlag;
00230     AC.TokensWriteFlag = AM.gTokensWriteFlag;
00231     AC.extrasymbols = AM.gextrasymbols;
00232     if ( AC.extrasym ) { M_free(AC.extrasym,"extrasym"); AC.extrasym = 0; }
00233     i = 1; s = AM.gextrasym; while ( *s ) { s++; i++; }
00234     AC.extrasym = (UBYTE *)Malloc1(i*sizeof(UBYTE),"extrasym");
00235     for ( j = 0; j < i; j++ ) AC.extrasym[j] = AM.gextrasym[j];
00236     AO.NoSpacesInNumbers = AM.gNoSpacesInNumbers;
00237     AO.IndentSpace = AM.gIndentSpace;
00238     AC.lUnitTrace = AM.gUnitTrace;
00239     AC.lDefDim = AM.gDefDim;
00240     AC.lDefDim4 = AM.gDefDim4;
00241     if ( AC.halfmod ) {
00242         if ( AC.ncmod == AM.gncmod && AC.modmode == AM.gmodmode ) {
00243             j = ABS(AC.ncmod);
00244             while ( --j >= 0 ) {
00245                 if ( AC.cmod[j] != AM.gcmod[j] ) break;
00246             }
00247             if ( j >= 0 ) {
00248                 M_free(AC.halfmod,"halfmod");
00249                 AC.halfmod = 0; AC.nhalfmod = 0;
00250             }
00251         }
00252         else {
00253             M_free(AC.halfmod,"halfmod");
00254             AC.halfmod = 0; AC.nhalfmod = 0;
00255         }
00256     }
00257     if ( AC.modinverses ) {
00258         if ( AC.ncmod == AM.gncmod && AC.modmode == AM.gmodmode ) {
00259             j = ABS(AC.ncmod);
00260             while ( --j >= 0 ) {
00261                 if ( AC.cmod[j] != AM.gcmod[j] ) break;
00262             }
00263             if ( j >= 0 ) {
00264                 M_free(AC.modinverses,"modinverses");
00265                 AC.modinverses = 0;
00266             }
00267         }
00268         else {
00269             M_free(AC.modinverses,"modinverses");
00270             AC.modinverses = 0;
00271         }
00272     }
00273     AN.ncmod = AC.ncmod = AM.gncmod;
00274     AC.npowmod = AM.gnpowmod;
00275     AC.modmode = AM.gmodmode;
00276     if ( ( ( AC.modmode & INVERSETABLE ) != 0 ) && ( AC.modinverses == 0 ) )
00277         MakeInverses();
00278     AC.funpowers = AM.gfunpowers;
00279     AC.lPolyFun = AM.gPolyFun;
00280     AC.lPolyFunInv = AM.gPolyFunInv;
00281     AC.lPolyFunType = AM.gPolyFunType;
00282     AC.lPolyFunExp = AM.gPolyFunExp;
00283     AR.PolyFunVar = AC.lPolyFunVar = AM.gPolyFunVar;
00284     AC.lPolyFunPow = AM.gPolyFunPow;
00285     AC.parallelflag = AM.gparallelflag;
00286     AC.ProcessBucketSize = AC.mProcessBucketSize = AM.gProcessBucketSize;
00287     AC.properorderflag = AM.gproperorderflag;
00288     AC.ThreadBucketSize = AM.gThreadBucketSize;
00289     AC.ThreadStats = AM.gThreadStats;
00290     AC.FinalStats = AM.gFinalStats;
00291     AC.OldGCDflag = AM.gOldGCDflag;

```

```

00292     AC.WTimeStatsFlag = AM.gWTimeStatsFlag;
00293     AC.ThreadsFlag = AM.gThreadsFlag;
00294     AC.ThreadBalancing = AM.gThreadBalancing;
00295     AC.ThreadSortFileSynch = AM.gThreadSortFileSynch;
00296     AC.ProcessStats = AM.gProcessStats;
00297     AC.OldParallelStats = AM.gOldParallelStats;
00298     AC.IsFortran90 = AM.gIsFortran90;
00299     AC.SizeCommuteInSet = AM.gSizeCommuteInSet;
00300     PruneExtraSymbols(AM.gnumextrasyms);
00301
00302     if ( AC.Fortran90Kind ) {
00303         M_free(AC.Fortran90Kind,"Fortran90 Kind");
00304         AC.Fortran90Kind = 0;
00305     }
00306     if ( AM.gFortran90Kind ) {
00307         AC.Fortran90Kind = strdup(AM.gFortran90Kind,"Fortran90 Kind");
00308     }
00309     if ( AC.ThreadsFlag && AM.totalnumberofthreads > 1 ) AS.MultiThreaded = 1;
00310     {
00311         UWORD *p, *m;
00312         p = AM.gcmmod;
00313         m = AC.cmod;
00314         j = ABS(AC.ncmod);
00315         NCOPY(m,p,j);
00316         p = AM.gpowmod;
00317         m = AC.powmod;
00318         j = AC.npowmod;
00319         NCOPY(m,p,j);
00320         if ( AC.DirtPow ) {
00321             if ( MakeModTable() ) {
00322                 MesPrint("===No printing in powers of generator");
00323             }
00324             AC.DirtPow = 0;
00325         }
00326     }
00327     {
00328         WORD *p, *m;
00329         p = AM.gUniTrace;
00330         m = AC.lUniTrace;
00331         j = 4;
00332         NCOPY(m,p,j);
00333     }
00334     AC.Cnumpows = AM.gCnumpows;
00335     AC.OutputMode = AM.gOutputMode;
00336     AC.OutputSpaces = AM.gOutputSpaces;
00337     AC.OutNumberType = AM.gOutNumberType;
00338     AR.SortType = AC.SortType = AM.gSortType;
00339     AC.ShortStatsMax = AM.gShortStatsMax;
00340 /*
00341     Now we have to clean up the commutation properties
00342 */
00343     for ( i = 0; i < NumFunctions; i++ ) functions[i].flags &= ~COULDCOMMUTE;
00344     if ( AC.CommuteInSet ) {
00345         WORD *g, *gg;
00346         g = AC.CommuteInSet;
00347         while ( *g ) {
00348             gg = g+1; g += *g;
00349             while ( gg < g ) {
00350                 if ( *gg <= GAMMASEVEN && *gg >= GAMMA ) {
00351                     functions[GAMMA-FUNCTION].flags |= COULDCOMMUTE;
00352                     functions[GAMMAI-FUNCTION].flags |= COULDCOMMUTE;
00353                     functions[GAMMAFIVE-FUNCTION].flags |= COULDCOMMUTE;
00354                     functions[GAMMASIX-FUNCTION].flags |= COULDCOMMUTE;
00355                     functions[GAMMASEVEN-FUNCTION].flags |= COULDCOMMUTE;
00356                 }
00357                 else {
00358                     functions[*gg-FUNCTION].flags |= COULDCOMMUTE;
00359                 }
00360             }
00361         }
00362     }
00363 /*
00364     Clean up the dictionaries.
00365 */
00366     for ( i = AO.NumDictionaries-1; i >= AO.gNumDictionaries; i-- ) {
00367         RemoveDictionary(AO.Dictionaries[i]);
00368         M_free(AO.Dictionaries[i],"Dictionary");
00369     }
00370     for( ; i >= 0; i-- ) {
00371         ShrinkDictionary(AO.Dictionaries[i]);
00372     }
00373     AO.NumDictionaries = AO.gNumDictionaries;
00374     return(retval);
00375 }
00376
00377 /*
00378     #] PopVariables :

```

```

00379         #! MakeGlobal :
00380     */
00381
00382 void MakeGlobal(void)
00383 {
00384     WORD i, j, *pp, *mm;
00385     UWORD *p, *m;
00386     UBYTE *s;
00387     Globalize(0);
00388
00389     AM.gCodesFlag = AC.CodesFlag;
00390     AM.gNamesFlag = AC.NamesFlag;
00391     AM.gStatsFlag = AC.StatsFlag;
00392     #ifndef WITHFLOAT
00393     AM.gMaxWeight = AC.MaxWeight;
00394     AM.gDefaultPrecision = AC.DefaultPrecision;
00395     #endif
00396     AM.gOldFactArgFlag = AC.OldFactArgFlag;
00397     AM.gextrasyms = AC.extrasyms;
00398     if ( AM.gextrasym ) { M_free(AM.gextrasym,"extrasym"); AM.gextrasym = 0; }
00399     i = 1; s = AC.extrasym; while ( *s ) { s++; i++; }
00400     AM.gextrasym = (UBYTE *)Malloc1(i*sizeof(UBYTE),"extrasym");
00401     for ( j = 0; j < i; j++ ) AM.gextrasym[j] = AC.extrasym[j];
00402     AM.gTokensWriteFlag = AC.TokensWriteFlag;
00403     AM.gNoSpacesInNumbers = AO.NoSpacesInNumbers;
00404     AM.gIndentSpace = AO.IndentSpace;
00405     AM.gUnitTrace = AC.lUnitTrace;
00406     AM.gDefDim = AC.lDefDim;
00407     AM.gDefDim4 = AC.lDefDim4;
00408     AM.gncmod = AC.ncmod;
00409     AM.gnpowmod = AC.npowmod;
00410     AM.gmodmode = AC.modmode;
00411     AM.gCnumpows = AC.Cnumpows;
00412     AM.gOutputMode = AC.OutputMode;
00413     AM.gOutputSpaces = AC.OutputSpaces;
00414     AM.gOutNumberType = AC.OutNumberType;
00415     AM.gfunpowers = AC.funpowers;
00416     AM.gPolyFun = AC.lPolyFun;
00417     AM.gPolyFunInv = AC.lPolyFunInv;
00418     AM.gPolyFunType = AC.lPolyFunType;
00419     AM.gPolyFunExp = AC.lPolyFunExp;
00420     AM.gPolyFunVar = AC.lPolyFunVar;
00421     AM.gPolyFunPow = AC.lPolyFunPow;
00422     AM.gparallelflag = AC.parallelflag;
00423     AM.gProcessBucketSize = AC.ProcessBucketSize;
00424     AM.gproperorderflag = AC.properorderflag;
00425     AM.gThreadBucketSize = AC.ThreadBucketSize;
00426     AM.gThreadStats = AC.ThreadStats;
00427     AM.gFinalStats = AC.FinalStats;
00428     AM.gOldGCDflag = AC.OldGCDflag;
00429     AM.gWTimeStatsFlag = AC.WTimeStatsFlag;
00430     AM.gThreadsFlag = AC.ThreadsFlag;
00431     AM.gThreadBalancing = AC.ThreadBalancing;
00432     AM.gThreadSortFileSynch = AC.ThreadSortFileSynch;
00433     AM.gProcessStats = AC.ProcessStats;
00434     AM.gOldParallelStats = AC.OldParallelStats;
00435     AM.gIsFortran90 = AC.IsFortran90;
00436     AM.gSizeCommuteInSet = AC.SizeCommuteInSet;
00437     AM.gnumextrasym = (cbuf+AM.sbufnum)->numrhs;
00438     if ( AM.gFortran90Kind ) {
00439         M_free(AM.gFortran90Kind,"Fortran 90 Kind");
00440         AM.gFortran90Kind = 0;
00441     }
00442     if ( AC.Fortran90Kind ) {
00443         AM.gFortran90Kind = strDup1(AC.Fortran90Kind,"Fortran 90 Kind");
00444     }
00445     p = AM.gcmod;
00446     m = AC.cmod;
00447     i = ABS(AC.ncmod);
00448     NCOPY(p,m,i);
00449     p = AM.gpowmod;
00450     m = AC.powmod;
00451     i = AC.npowmod;
00452     NCOPY(p,m,i);
00453     pp = AM.gUnitTrace;
00454     mm = AC.lUnitTrace;
00455     i = 4;
00456     NCOPY(pp,mm,i);
00457     AM.gSortType = AC.SortType;
00458     AM.gShortStatsMax = AC.ShortStatsMax;
00459
00460     if ( AO.CurrentDictionary > 0 || AP.OpenDictionary > 0 ) {
00461         Warning("You cannot have an open or selected dictionary at a .global. Dictionary closed.");
00462         AP.OpenDictionary = 0;
00463         AO.CurrentDictionary = 0;
00464     }
00465

```

```

00466     AO.gNumDictionaries = AO.NumDictionaries;
00467     for ( i = 0; i < AO.NumDictionaries; i++ ) {
00468         AO.Dictionaries[i]->gnumelements = AO.Dictionaries[i]->numelements;
00469     }
00470     if ( AM.NumSpectatorFiles > 0 ) {
00471         for ( i = 0; i < AM.SizeForSpectatorFiles; i++ ) {
00472             if ( AM.SpectatorFiles[i].name != 0 )
00473                 AM.SpectatorFiles[i].flags |= GLOBALSPECTATORFLAG;
00474         }
00475     }
00476 }
00477
00478 /*
00479     #] MakeGlobal :
00480     #[ TestDrop :
00481 */
00482
00483 void TestDrop(void)
00484 {
00485     EXPRESSIONS e;
00486     WORD j;
00487     for ( j = 0, e = Expressions; j < NumExpressions; j++, e++ ) {
00488         switch ( e->status ) {
00489             case SKIPLEXPRESSION:
00490                 e->status = LOCALEXPRESSION;
00491                 break;
00492             case UNHIDELEXPRESSION:
00493                 e->status = LOCALEXPRESSION;
00494                 ClearBracketIndex(j);
00495                 e->bracketinfo = e->newbracketinfo; e->newbracketinfo = 0;
00496                 break;
00497             case HIDELEXPRESSION:
00498                 e->status = HIDDENLEXPRESSION;
00499                 break;
00500             case SKIPGEXPRESSION:
00501                 e->status = GLOBALEXPRESSION;
00502                 break;
00503             case UNHIDEGEXPRESSION:
00504                 e->status = GLOBALEXPRESSION;
00505                 ClearBracketIndex(j);
00506                 e->bracketinfo = e->newbracketinfo; e->newbracketinfo = 0;
00507                 break;
00508             case HIDEGEXPRESSION:
00509                 e->status = HIDDENGEXPRESSION;
00510                 break;
00511             case DROPLEXPRESSION:
00512             case DROPGEXPRESSION:
00513             case DROPHLEXPRESSION:
00514             case DROPHGEXPRESSION:
00515             case DROPSPECTATOREXPRESSION:
00516                 e->status = DROPEDEXPRESSION;
00517                 ClearBracketIndex(j);
00518                 e->bracketinfo = e->newbracketinfo; e->newbracketinfo = 0;
00519                 if ( e->replace >= 0 ) {
00520                     Expressions[e->replace].replace = REGULAREXPRESSION;
00521                     AC.exprnames->namenode[e->node].number = e->replace;
00522                     e->replace = REGULAREXPRESSION;
00523                 }
00524                 else {
00525                     AC.exprnames->namenode[e->node].type = CDELETE;
00526                     AC.DidClean = 1;
00527                 }
00528                 break;
00529             case LOCALEXPRESSION:
00530             case GLOBALEXPRESSION:
00531                 ClearBracketIndex(j);
00532                 e->bracketinfo = e->newbracketinfo; e->newbracketinfo = 0;
00533                 break;
00534             case HIDDENLEXPRESSION:
00535             case HIDDENGEXPRESSION:
00536                 break;
00537             case INTOHIDELEXPRESSION:
00538                 ClearBracketIndex(j);
00539                 e->bracketinfo = e->newbracketinfo; e->newbracketinfo = 0;
00540                 e->status = HIDDENLEXPRESSION;
00541                 break;
00542             case INTOHIDEGEXPRESSION:
00543                 ClearBracketIndex(j);
00544                 e->bracketinfo = e->newbracketinfo; e->newbracketinfo = 0;
00545                 e->status = HIDDENGEXPRESSION;
00546                 break;
00547             default:
00548                 ClearBracketIndex(j);
00549                 e->bracketinfo = 0;
00550                 break;
00551         }
00552         if ( e->replace == NEWLYDEFINEDEXPRESSION ) e->replace = REGULAREXPRESSION;

```



```

00553     }
00554 }
00555
00556 /*
00557     #] TestDrop :
00558     #[ PutInVflags :
00559 */
00560
00561 void PutInVflags(WORD nexpr)
00562 {
00563     EXPRESSIONS e = Expressions + nexpr;
00564     POSITION *old;
00565     WORD *oldw;
00566     int i;
00567 restart:;
00568     if ( AS.OldOnFile == 0 ) {
00569         AS.NumOldOnFile = 20;
00570         AS.OldOnFile = (POSITION *)Malloc1(AS.NumOldOnFile*sizeof(POSITION),"file pointers");
00571     }
00572     else if ( nexpr >= AS.NumOldOnFile ) {
00573         old = AS.OldOnFile;
00574         AS.OldOnFile = (POSITION *)Malloc1(2*AS.NumOldOnFile*sizeof(POSITION),"file pointers");
00575         for ( i = 0; i < AS.NumOldOnFile; i++ ) AS.OldOnFile[i] = old[i];
00576         AS.NumOldOnFile = 2*AS.NumOldOnFile;
00577         M_free(old,"process file pointers");
00578     }
00579     if ( AS.OldNumFactors == 0 ) {
00580         AS.NumOldNumFactors = 20;
00581         AS.OldNumFactors = (WORD *)Malloc1(AS.NumOldNumFactors*sizeof(WORD),"numfactors pointers");
00582         AS.Oldvflags = (WORD *)Malloc1(AS.NumOldNumFactors*sizeof(WORD),"vflags pointers");
00583         AS.Olduflags = (WORD *)Malloc1(AS.NumOldNumFactors*sizeof(WORD),"uflags pointers");
00584     }
00585     else if ( nexpr >= AS.NumOldNumFactors ) {
00586         oldw = AS.OldNumFactors;
00587         AS.OldNumFactors = (WORD *)Malloc1(2*AS.NumOldNumFactors*sizeof(WORD),"numfactors pointers");
00588         for ( i = 0; i < AS.NumOldNumFactors; i++ ) AS.OldNumFactors[i] = oldw[i];
00589         M_free(oldw,"numfactors pointers");
00590         oldw = AS.Oldvflags;
00591         AS.Oldvflags = (WORD *)Malloc1(2*AS.NumOldNumFactors*sizeof(WORD),"vflags pointers");
00592         for ( i = 0; i < AS.NumOldNumFactors; i++ ) AS.Oldvflags[i] = oldw[i];
00593         M_free(oldw,"vflags pointers");
00594         oldw = AS.Olduflags;
00595         AS.Olduflags = (WORD *)Malloc1(2*AS.NumOldNumFactors*sizeof(WORD),"uflags pointers");
00596         for ( i = 0; i < AS.NumOldNumFactors; i++ ) AS.Olduflags[i] = oldw[i];
00597         M_free(oldw,"uflags pointers");
00598         AS.NumOldNumFactors = 2*AS.NumOldNumFactors;
00599     }
00600 /*
00601     The next is needed when we Load a .sav file with lots of expressions.
00602 */
00603     if ( nexpr >= AS.NumOldOnFile || nexpr >= AS.NumOldNumFactors ) goto restart;
00604     AS.OldOnFile[nexpr] = e->onfile;
00605     AS.OldNumFactors[nexpr] = e->numfactors;
00606     AS.Oldvflags[nexpr] = e->vflags;
00607     AS.Olduflags[nexpr] = e->uflags;
00608 }
00609
00610 /*
00611     #] PutInVflags :
00612     #[ DoExecute :
00613 */
00614
00615 WORD DoExecute(WORD par, WORD skip)
00616 {
00617     GETIDENTITY
00618     WORD RetCode = 0;
00619     int i, oldmultithreaded = AS.MultiThreaded;
00620 #ifdef PARALLELLECODE
00621     int j;
00622 #endif
00623
00624     SpecialCleanup(BHEAD0);
00625     if ( skip ) goto skipexec;
00626     if ( AC.IfLevel > 0 ) {
00627         MesPrint(" %d endif statement(s) missing",AC.IfLevel);
00628         RetCode = 1;
00629     }
00630     if ( AC.WhileLevel > 0 ) {
00631         MesPrint(" %d endwhile statement(s) missing",AC.WhileLevel);
00632         RetCode = 1;
00633     }
00634     if ( AC.arglevel > 0 ) {
00635         MesPrint(" %d endargument statement(s) missing",AC.arglevel);
00636         RetCode = 1;
00637     }
00638     if ( AC.termlevel > 0 ) {
00639         MesPrint(" %d endterm statement(s) missing",AC.termlevel);

```

```

00640         RetCode = 1;
00641     }
00642     if ( AC.insidelevel > 0 ) {
00643         MesPrint(" %d endinside statement(s) missing",AC.insidelevel);
00644         RetCode = 1;
00645     }
00646     if ( AC.inexprlevel > 0 ) {
00647         MesPrint(" %d endinexpression statement(s) missing",AC.inexprlevel);
00648         RetCode = 1;
00649     }
00650     if ( AC.NumLabels > 0 ) {
00651         for ( i = 0; i < AC.NumLabels; i++ ) {
00652             if ( AC.Labels[i] < 0 ) {
00653                 MesPrint(" -->Label %s missing",AC.LabelNames[i]);
00654                 RetCode = 1;
00655             }
00656         }
00657     }
00658     if ( AC.SwitchLevel > 0 ) {
00659         MesPrint(" %d endswitch statement(s) missing",AC.SwitchLevel);
00660         RetCode = 1;
00661     }
00662     if ( AC.dolooplevel > 0 ) {
00663         MesPrint(" %d enddo statement(s) missing",AC.dolooplevel);
00664         RetCode = 1;
00665     }
00666     if ( AP.OpenDictionary > 0 ) {
00667         MesPrint(" Dictionary %s has not been closed.",
00668             AO.Dictionaries[AP.OpenDictionary-1]->name);
00669         AP.OpenDictionary = 0;
00670         RetCode = 1;
00671     }
00672     if ( RetCode ) return(RetCode);
00673     AR.Cnumlhs = cbuf[AM.rbufnum].numlhs;
00674
00675     if ( ( AS.ExecMode = par ) == GLOBALMODULE ) AS.ExecMode = 0;
00676 #ifdef PARALLELCODE
00677 /*
00678     Now check whether we have either the regular parallel flag or the
00679     mparallel flag set.
00680     Next check whether any of the expressions has partodo set.
00681     If any of the above we need to check what the dollar status is.
00682 */
00683     AC.partodoflag = -1;
00684     if ( NumPotModdollars >= 0 ) {
00685         for ( i = 0; i < NumExpressions; i++ ) {
00686             if ( Expressions[i].partodo ) { AC.partodoflag = 1; break; }
00687         }
00688     }
00689 #ifdef WITHMPI
00690     if ( AC.partodoflag > 0 && PF.numtasks < 3 ) {
00691         AC.partodoflag = 0;
00692     }
00693 #endif
00694     if ( AC.partodoflag > 0 || ( NumPotModdollars > 0 && AC.mparallelfld == PARALLELFLAG ) ) {
00695         if ( NumPotModdollars > NumModOptdollars ) {
00696             AC.mparallelfld |= NOPARALLEL_DOLLAR;
00697 #ifdef WITHPTHREADS
00698             AS.MultiThreaded = 0;
00699 #endif
00700             AC.partodoflag = 0;
00701         }
00702         else {
00703             for ( i = 0; i < NumPotModdollars; i++ ) {
00704                 for ( j = 0; j < NumModOptdollars; j++ ) {
00705                     if ( PotModdollars[i] == ModOptdollars[j].number ) break;
00706                     if ( j >= NumModOptdollars ) {
00707                         AC.mparallelfld |= NOPARALLEL_DOLLAR;
00708 #ifdef WITHPTHREADS
00709                         AS.MultiThreaded = 0;
00710 #endif
00711                         AC.partodoflag = 0;
00712                         break;
00713                     }
00714                     switch ( ModOptdollars[j].type ) {
00715                         case MODSUM:
00716                         case MODMAX:
00717                         case MODMIN:
00718                         case MODLOCAL:
00719                             break;
00720                         default:
00721                             AC.mparallelfld |= NOPARALLEL_DOLLAR;
00722                             AS.MultiThreaded = 0;
00723                             AC.partodoflag = 0;
00724                             break;
00725                     }
00726                 }

```

```

00727     }
00728     }
00729     else if ( ( AC.mparallelfldag & NOPARALLEL_USER ) != 0 ) {
00730 #ifdef WITHPTHREADS
00731     AS.MultiThreaded = 0;
00732 #endif
00733     AC.partodoflag = 0;
00734     }
00735     if ( AC.partodoflag == 0 ) {
00736     for ( i = 0; i < NumExpressions; i++ ) {
00737     Expressions[i].partodo = 0;
00738     }
00739     }
00740     else if ( AC.partodoflag == -1 ) {
00741     AC.partodoflag = 0;
00742     }
00743 #endif
00744 #ifdef WITHMPI
00745 /*
00746  * Check RHS expressions.
00747  */
00748 if ( AC.RhsExprInModuleFlag && (AC.mparallelfldag == PARALLELFLAG || AC.partodoflag) ) {
00749     if (PF.rhsInParallel) {
00750     PF.mkSlaveInfile=1;
00751     if(PF.me != MASTER){
00752     PF.slavebuf.PObuffer=(WORD *)Malloc1(AM.ScratSize*sizeof(WORD),"PF inbuf");
00753     PF.slavebuf.POsize=AM.ScratSize*sizeof(WORD);
00754     PF.slavebuf.POfull = PF.slavebuf.POfill = PF.slavebuf.PObuffer;
00755     PF.slavebuf.POstop= PF.slavebuf.PObuffer+AM.ScratSize;
00756     PUTZERO(PF.slavebuf.POposition);
00757     }/*if(PF.me != MASTER)*/
00758     }
00759     else {
00760     AC.mparallelfldag |= NOPARALLEL_RHS;
00761     AC.partodoflag = 0;
00762     for ( i = 0; i < NumExpressions; i++ ) {
00763     Expressions[i].partodo = 0;
00764     }
00765     }
00766     }
00767 /*
00768  * Set $-variables with MODSUM to zero on the slaves.
00769  */
00770 if ( (AC.mparallelfldag == PARALLELFLAG || AC.partodoflag) && PF.me != MASTER ) {
00771     for ( i = 0; i < NumModOptdollars; i++ ) {
00772     if ( ModOptdollars[i].type == MODSUM ) {
00773     DOLLARS d = Dollars + ModOptdollars[i].number;
00774     d->type = DOLZERO;
00775     if ( d->where && d->where != &AM.dollarzero ) M_free(d->where, "old content of
dollar");
00776     d->where = &AM.dollarzero;
00777     d->size = 0;
00778     CleanDollarFactors(d);
00779     }
00780     }
00781     }
00782 #endif
00783     AR.SortType = AC.SortType;
00784 #ifdef WITHMPI
00785     if ( PF.me == MASTER )
00786 #endif
00787     {
00788     if ( AC.SetupFlag ) WriteSetup();
00789     if ( AC.NamesFlag || AC.CodesFlag ) WriteLists();
00790     }
00791     if ( par == GLOBALMODULE ) MakeGlobal();
00792     if ( RevertScratch() ) return(-1);
00793     if ( AC.ncmod ) SetMods();
00794 /*
00795  * Warn if the module has to run in sequential mode due to some problems.
00796  */
00797 #ifdef WITHMPI
00798     if ( PF.me == MASTER )
00799 #endif
00800     {
00801     if ( !AC.ThreadsFlag || AC.mparallelfldag & NOPARALLEL_USER ) {
00802     /* The user switched off the parallel execution explicitly. */
00803     }
00804     else if ( AC.mparallelfldag & NOPARALLEL_DOLLAR ) {
00805     if ( AC.WarnFlag >= 2 ) { /* HighWarning */
00806     int i, j, k, n;
00807     UBYTE *s, *sl;
00808     s = strDup1((UBYTE *)"", "NOPARALLEL_DOLLAR s");
00809     n = 0;
00810     j = NumPotModdollars;
00811     for ( i = 0; i < j; i++ ) {
00812     for ( k = 0; k < NumModOptdollars; k++ )

```

```

00813         if ( ModOptdollars[k].number == PotModdollars[i] ) break;
00814     if ( k >= NumModOptdollars ) {
00815         /* global $-variable */
00816         if ( n > 0 )
00817             s = AddToString(s, (UBYTE *)", ", 0);
00818         s = AddToString(s, (UBYTE *)"$", 0);
00819         s = AddToString(s, DOLLARNAME(Dollars, PotModdollars[i]), 0);
00820         n++;
00821     }
00822 }
00823 s1 = strdup((UBYTE *)"This module is forced to run in sequential mode due to
$-variable", "NOPARALLEL_DOLLAR s1");
00824 if ( n != 1 )
00825     s1 = AddToString(s1, (UBYTE *)"s", 0);
00826 s1 = AddToString(s1, (UBYTE *)": ", 0);
00827 s1 = AddToString(s1, s, 0);
00828 HighWarning((char *)s1);
00829 M_free(s, "NOPARALLEL_DOLLAR s");
00830 M_free(s1, "NOPARALLEL_DOLLAR s1");
00831 }
00832 }
00833 else if ( AC.mparallelflag & NOPARALLEL_RHS ) {
00834     HighWarning("This module is forced to run in sequential mode due to RHS expression
names");
00835 }
00836 else if ( AC.mparallelflag & NOPARALLEL_CONVPOLY ) {
00837     HighWarning("This module is forced to run in sequential mode due to conversion to extra
symbols");
00838 }
00839 else if ( AC.mparallelflag & NOPARALLEL_SPECTATOR ) {
00840     HighWarning("This module is forced to run in sequential mode due to
tospectator/copspectator");
00841 }
00842 else if ( AC.mparallelflag & NOPARALLEL_TBLDOLLAR ) {
00843     HighWarning("This module is forced to run in sequential mode due to $-variable assignments
in tables");
00844 }
00845 else if ( AC.mparallelflag & NOPARALLEL_NPROC ) {
00846     HighWarning("This module is forced to run in sequential mode because there is only one
processor");
00847 }
00848 }
00849 /*
00850     Now the actual execution
00851 */
00852 #ifdef WITHMPI
00853 /*
00854  * Turn on AS.printflag to print runtime errors occurring on slaves.
00855  */
00856 AS.printflag = 1;
00857 #endif
00858 if ( AP.preError == 0 && ( Processor() || WriteAll() ) ) RetCode = -1;
00859 #ifdef WITHMPI
00860 AS.printflag = 0;
00861 #endif
00862 /*
00863     That was it. Next is cleanup.
00864 */
00865 if ( AC.ncmod ) UnSetMods();
00866 AS.MultiThreaded = oldmultithreaded;
00867 TableReset();
00868
00869 /*[28sep2005 mt]:*/
00870 #ifdef WITHMPI
00871 /* Combine and then broadcast modified dollar variables. */
00872 if ( NumPotModdollars > 0 ) {
00873     RetCode = PF_CollectModifiedDollars();
00874     if ( RetCode ) return RetCode;
00875     RetCode = PF_BroadcastModifiedDollars();
00876     if ( RetCode ) return RetCode;
00877 }
00878 /* Broadcast the list of objects converted to symbols in AM.sbufnum. */
00879 if ( AC.topolynomialflag & TOPOLYNOMIALFLAG ) {
00880     RetCode = PF_BroadcastCBuf(AM.sbufnum);
00881     if ( RetCode ) return RetCode;
00882 }
00883 /*
00884  * Broadcast AR.expflags, which may be used on the slaves in the next module
00885  * via ZERO_ or UNCHANGED_. It also broadcasts several flags of each expression.
00886  */
00887 RetCode = PF_BroadcastExpFlags();
00888 if ( RetCode ) return RetCode;
00889 /*
00890  * Clean the hide file on the slaves, which was used for RHS expressions
00891  * broadcast from the master at the beginning of the module.
00892  */
00893 if ( PF.me != MASTER && AR.hidefile->PObuffer ) {

```

```

00894         if ( AR.hidefile->handle >= 0 ) {
00895             CloseFile(AR.hidefile->handle);
00896             AR.hidefile->handle = -1;
00897             remove(AR.hidefile->name);
00898         }
00899         AR.hidefile->POfull = AR.hidefile->POfill = AR.hidefile->PObuffer;
00900         PUTZERO(AR.hidefile->POposition);
00901     }
00902 #endif
00903 #ifdef WITHPTHREADS
00904     for ( j = 0; j < NumModOptdollars; j++ ) {
00905         if ( ModOptdollars[j].dstruct ) {
00906             /*
00907                 First clean up dollar values.
00908             */
00909             for ( i = 0; i < AM.totalnumberofthreads; i++ ) {
00910                 if ( ModOptdollars[j].dstruct[i].size > 0 ) {
00911                     CleanDollarFactors(&(ModOptdollars[j].dstruct[i]));
00912                     M_free(ModOptdollars[j].dstruct[i].where,"Local dollar value");
00913                 }
00914             }
00915             /*
00916                 Now clean up the whole array.
00917             */
00918             M_free(ModOptdollars[j].dstruct,"Local DOLLARS");
00919             ModOptdollars[j].dstruct = 0;
00920         }
00921     }
00922 #endif
00923 /*:[28sep2005 mt]*/
00924
00925 /*
00926     @@@@@@@@@@@@@@@@@@
00927     Now follows the code to invalidate caches for all objects in the
00928     PotModdollars. There are NumPotModdollars of them and PotModdollars
00929     is an array of WORD.
00930 */
00931 /*
00932     Cleanup:
00933 */
00934 #ifdef JV_IS_WRONG
00935 /*
00936     Giving back this memory gives way too much activity with Malloc1
00937     Better to keep it and just put the number of used objects to zero (JV)
00938     If you put the lijst equal to NULL, please also make maxnum = 0
00939 */
00940     if ( ModOptdollars ) M_free(ModOptdollars, "ModOptdollars pointer");
00941     if ( PotModdollars ) M_free(PotModdollars, "PotModdollars pointer");
00942
00943     /* ModOptdollars changed to AC.ModOptDolList.lijst because AIX C compiler complained. MF
00944     30/07/2003. */
00945     AC.ModOptDolList.lijst = NULL;
00946     /* PotModdollars changed to AC.PotModDolList.lijst because AIX C compiler complained. MF
00947     30/07/2003. */
00948     AC.PotModDolList.lijst = NULL;
00949 #endif
00950     NumPotModdollars = 0;
00951     NumModOptdollars = 0;
00952 skipexec:
00953 /*
00954     Clean up the switch information.
00955     We keep the switch array and heap.
00956 */
00957 if ( AC.SwitchInArray > 0 ) {
00958     for ( i = 0; i < AC.SwitchInArray; i++ ) {
00959         SWITCH *sw = AC.SwitchArray + i;
00960         if ( sw->table ) M_free(sw->table,"Switch table");
00961         sw->table = 0;
00962         sw->defaultcase.ncase = 0;
00963         sw->defaultcase.value = 0;
00964         sw->defaultcase.compbuffer = 0;
00965         sw->endswitch.ncase = 0;
00966         sw->endswitch.value = 0;
00967         sw->endswitch.compbuffer = 0;
00968         sw->typetable = 0;
00969         sw->maxcase = 0;
00970         sw->mincase = 0;
00971         sw->numcases = 0;
00972         sw->tablesize = 0;
00973         sw->caseoffset = 0;
00974         sw->iflevel = 0;
00975         sw->whilelevel = 0;
00976         sw->nestingsum = 0;
00977     }
00978     AC.SwitchInArray = 0;
00979     AC.SwitchLevel = 0;

```

```

00979 }
00980 #ifdef PARALLELCODE
00981     AC.numpfirstnum = 0;
00982 #endif
00983     AC.DidClean = 0;
00984     AC.PolyRatFunChanged = 0;
00985     TestDrop();
00986     if ( par == STOREMODULE || par == CLEARMODULE ) {
00987         ClearOptimize();
00988         if ( par == STOREMODULE && PopVariables() ) RetCode = -1;
00989         if ( AR.infile->handle >= 0 ) {
00990             CloseFile(AR.infile->handle);
00991             remove(AR.infile->name);
00992             AR.infile->handle = -1;
00993         }
00994         AR.infile->POfill = AR.infile->PObuffer;
00995         PUTZERO(AR.infile->POposition);
00996         AR.infile->POfull = AR.infile->PObuffer;
00997         if ( AR.outfile->handle >= 0 ) {
00998             CloseFile(AR.outfile->handle);
00999             remove(AR.outfile->name);
01000             AR.outfile->handle = -1;
01001         }
01002         AR.outfile->POfull =
01003         AR.outfile->POfill = AR.outfile->PObuffer;
01004         PUTZERO(AR.outfile->POposition);
01005         if ( AR.hidefile->handle >= 0 ) {
01006             CloseFile(AR.hidefile->handle);
01007             remove(AR.hidefile->name);
01008             AR.hidefile->handle = -1;
01009         }
01010         AR.hidefile->POfull =
01011         AR.hidefile->POfill = AR.hidefile->PObuffer;
01012         PUTZERO(AR.hidefile->POposition);
01013         AC.HideLevel = 0;
01014         if ( par == CLEARMODULE ) {
01015             if ( DeleteStore(0) < 0 ) {
01016                 MesPrint("Cannot restart the storage file");
01017                 RetCode = -1;
01018             }
01019             else RetCode = 0;
01020             Cleanup(1);
01021             ResetVariables(2);
01022             AM.gProcessBucketSize = AM.hProcessBucketSize;
01023             AM.gparallelflag = PARALLELFLAG;
01024             AM.gnumextrasyms = AM.ggnumextrasyms;
01025             PruneExtraSymbols(AM.ggnumextrasyms);
01026             IniVars();
01027         }
01028         ClearSpectators(par);
01029     }
01030     else {
01031         if ( CleanExpr(0) ) RetCode = -1;
01032         if ( AC.DidClean ) CompactifyTree(AC.exprnames,EXPRNAMES);
01033         ResetVariables(0);
01034         CleanupSort(-1);
01035     }
01036     clearcbuf(AC.cbunum);
01037     if ( AC.MultiBracketBuf != 0 ) {
01038         for ( i = 0; i < MAXMULTIBRACKETLEVELS; i++ ) {
01039             if ( AC.MultiBracketBuf[i] ) {
01040                 M_free(AC.MultiBracketBuf[i],"bracket buffer i");
01041                 AC.MultiBracketBuf[i] = 0;
01042             }
01043         }
01044         AC.MultiBracketLevels = 0;
01045         M_free(AC.MultiBracketBuf,"multi bracket buffer");
01046         AC.MultiBracketBuf = 0;
01047     }
01048
01049     if ( AC.SortReallocateFlag ) {
01050         /* Reallocate the sort buffers to reduce resident set usage */
01051         /* AT.SS is the same as AT.S0 here */
01052         SORTING* S = AT.S0;
01053         M_free(S->lBuffer, "SortReallocate lBuffer+sBuffer");
01054         S->lBuffer = Malloc1(sizeof(* (S->lBuffer)) * (S->LargeSize+S->SmallSize), "SortReallocate
lBuffer+sBuffer");
01055         S->lTop = S->lBuffer+S->LargeSize;
01056         S->sBuffer = S->lTop;
01057         if ( S->LargeSize == 0 ) { S->lBuffer = 0; S->lTop = 0; }
01058         S->sTop = S->sBuffer + S->SmallSize;
01059         S->sTop2 = S->sBuffer + S->SmallSize;
01060         S->sHalf = S->sBuffer + (LONG)((S->SmallSize+S->SmallSize)>>1);
01061
01062 #ifdef WITHPTHREADS
01063         /* We have to re-set the pointers into master lBuffer in the SortBlocks */
01064         UpdateSortBlocks(AM.totalnumberofthreads-1);

```

```

01065
01066      /* The SortBots do not have a real sort buffer to reallocate. */
01067      /* AB[0] has been reallocated above already. */
01068      for ( i = 1; i < AM.totalnumberofthreads; i++ ) {
01069          SORTING* S = AB[i]->T.S0;
01070          M_free(S->lBuffer, "SortReallocate lBuffer+sBuffer");
01071          S->lBuffer = Malloc1(sizeof(*(S->lBuffer))*(S->LargeSize+S->SmallEsize), "SortReallocate
lBuffer+sBuffer");
01072          S->lTop = S->lBuffer+S->LargeSize;
01073          S->sBuffer = S->lTop;
01074          if ( S->LargeSize == 0 ) { S->lBuffer = 0; S->lTop = 0; }
01075          S->sTop = S->sBuffer + S->SmallSize;
01076          S->sTop2 = S->sBuffer + S->SmallEsize;
01077          S->sHalf = S->sBuffer + (LONG)((S->SmallSize+S->SmallEsize)>>1);
01078      }
01079  #endif
01080  }
01081  if ( AC.SortReallocateFlag == 2 ) {
01082      /* The Flag was set for a single module by the preprocessor #sortreallocate,
01083         so turn it off again. */
01084      AC.SortReallocateFlag = 0;
01085  }
01086
01087  return(RetCode);
01088 }
01089
01090 /*
01091      #] DoExecute :
01092      #[ PutBracket :
01093
01094  Routine uses the bracket info to split a term into two pieces:
01095  1: the part outside the bracket, and
01096  2: the part inside the bracket.
01097  These parts are separated by a subterm of type HAAKJE.
01098  This subterm looks like: HAAKJE,3,level
01099  The level is used for nestings of brackets. The print routines
01100  cannot handle this yet (31-Mar-1988).
01101
01102  The Bracket selector is in AT.BrackBuf in the form of a regular term,
01103  but without coefficient.
01104  When AR.BracketOn < 0 we have a so-called antibracket. The main effect
01105  is an exchange of the inner and outer part and where the coefficient goes.
01106
01107  Routine recoded to facilitate b p1,p2; etc for dotproducts and tensors
01108  15-oct-1991
01109 */
01110
01111 WORD PutBracket(PHEAD WORD *termin)
01112 {
01113     GETBIDENTITY
01114     WORD *t, *t1, *b, i, j, *lastfun;
01115     WORD *t2, *s1, *s2;
01116     WORD *bStop, *bb, *bf, *tStop;
01117     WORD *term1,*term2, *m1, *m2, *tStopa;
01118     WORD *bbb = 0, *bind, *binst = 0, *bwild = 0, *bss = 0, *bns = 0, *bset = 0;
01119     term1 = AT.WorkPointer+1;
01120     term2 = (WORD *)(((UBYTE *) (term1)) + AM.MaxTer);
01121     if ( ( (WORD *)(((UBYTE *) (term2)) + AM.MaxTer) ) > AT.WorkTop ) return(MesWork());
01122     if ( AR.BracketOn < 0 ) {
01123         t2 = term1; t1 = term2;      /* AntiBracket */
01124     }
01125     else {
01126         t1 = term1; t2 = term2;      /* Regular bracket */
01127     }
01128     b = AT.BrackBuf; bStop = b+*b; b++;
01129     while ( b < bStop ) {
01130         if ( *b == INDEX ) { bwild = 1; bbb = b+2; binst = b + b[1]; }
01131         if ( *b == SETSET ) { bset = 1; bss = b+2; bns = b + b[1]; }
01132         b += b[1];
01133     }
01134
01135     t = termin; tStopa = t + *t; i = *(t + *t -1); i = ABS(i);
01136     if ( AR.PolyFun && AT.PolyAct ) tStop = termin + AT.PolyAct;
01137     #ifdef WITHFLOAT
01138     else if ( AT.FloatPos ) tStop = termin + AT.FloatPos;
01139     #endif
01140     else tStop = tStopa - i;
01141     t++;
01142     if ( AR.BracketOn < 0 ) {
01143         lastfun = 0;
01144         while ( t < tStop && *t >= FUNCTION
&& functions[*t-FUNCTION].commute ) {
01145             b = AT.BrackBuf+1;
01146             while ( b < bStop ) {
01147                 if ( *b == *t ) {
01148                     lastfun = t;
01149                     while ( t < tStop && *t >= FUNCTION

```

```

01151         && functions[*t-FUNCTION].commute ) t += t[1];
01152         goto NextNcom1;
01153     }
01154     b += b[1];
01155 }
01156 if ( bset ) {
01157     b = bss;
01158     while ( b < bns ) {
01159         if ( b[1] == CFUNCTION ) { /* Set of functions */
01160             SETS set = Sets+b[0]; WORD i;
01161             for ( i = set->first; i < set->last; i++ ) {
01162                 if ( SetElements[i] == *t ) {
01163                     lastfun = t;
01164                     while ( t < tStop && *t >= FUNCTION
01165                         && functions[*t-FUNCTION].commute ) t += t[1];
01166                     goto NextNcom1;
01167                 }
01168             }
01169         }
01170         b += 2;
01171     }
01172 }
01173 if ( bwild && *t >= FUNCTION && functions[*t-FUNCTION].spec ) {
01174     s1 = t + t[1];
01175     s2 = t + FUNHEAD;
01176     while ( s2 < s1 ) {
01177         bind = bbb;
01178         while ( bind < binst ) {
01179             if ( *bind == *s2 ) {
01180                 lastfun = t;
01181                 while ( t < tStop && *t >= FUNCTION
01182                     && functions[*t-FUNCTION].commute ) t += t[1];
01183                 goto NextNcom1;
01184             }
01185             bind++;
01186         }
01187         s2++;
01188     }
01189 }
01190 t += t[1];
01191 }
01192 NextNcom1:
01193 s1 = termin + 1;
01194 if ( lastfun ) {
01195     while ( s1 < lastfun ) *t2++ = *s1++;
01196     while ( s1 < t ) *t1++ = *s1++;
01197 }
01198 else {
01199     while ( s1 < t ) *t2++ = *s1++;
01200 }
01201 }
01202 }
01203 else {
01204     lastfun = t;
01205     while ( t < tStop && *t >= FUNCTION
01206         && functions[*t-FUNCTION].commute ) {
01207         b = AT.BrackBuf+1;
01208         while ( b < bStop ) {
01209             if ( *b == *t ) { lastfun = t + t[1]; goto NextNcom; }
01210             b += b[1];
01211         }
01212         if ( bset ) {
01213             b = bss;
01214             while ( b < bns ) {
01215                 if ( b[1] == CFUNCTION ) { /* Set of functions */
01216                     SETS set = Sets+b[0]; WORD i;
01217                     for ( i = set->first; i < set->last; i++ ) {
01218                         if ( SetElements[i] == *t ) {
01219                             lastfun = t + t[1];
01220                             goto NextNcom;
01221                         }
01222                     }
01223                 }
01224                 b += 2;
01225             }
01226         }
01227         if ( bwild && *t >= FUNCTION && functions[*t-FUNCTION].spec ) {
01228             s1 = t + t[1];
01229             s2 = t + FUNHEAD;
01230             while ( s2 < s1 ) {
01231                 bind = bbb;
01232                 while ( bind < binst ) {
01233                     if ( *bind == *s2 ) { lastfun = t + t[1]; goto NextNcom; }
01234                     bind++;
01235                 }
01236                 s2++;
01237             }

```



```

01238         }
01239 NextNcom:
01240         t += t[1];
01241     }
01242     s1 = termin + 1;
01243     while ( s1 < lastfun ) *t1++ = *s1++;
01244     while ( s1 < t ) *t2++ = *s1++;
01245 }
01246 /*
01247 Now we have only commuting functions left. Move the b pointer to them.
01248 */
01249 b = AT.BrackBuf + 1;
01250 while ( b < bStop && *b >= FUNCTION
01251     && ( *b < FUNCTION || functions[*b-FUNCTION].commute ) ) {
01252     b += b[1];
01253 }
01254 bf = b;
01255
01256 while ( t < tStop && ( bf < bStop || bwild || bset ) ) {
01257     b = bf;
01258     while ( b < bStop && *b != *t ) { b += b[1]; }
01259     i = t[1];
01260     if ( *t >= FUNCTION ) { /* We are in function territory */
01261         if ( b < bStop && *b == *t ) goto FunBrac;
01262         if ( bset ) {
01263             b = bss;
01264             while ( b < bns ) {
01265                 if ( b[1] == CFUNCTION ) { /* Set of functions */
01266                     SETS set = Sets+b[0]; WORD i;
01267                     for ( i = set->first; i < set->last; i++ ) {
01268                         if ( SetElements[i] == *t ) goto FunBrac;
01269                     }
01270                 }
01271                 b += 2;
01272             }
01273         }
01274         if ( bwild && *t >= FUNCTION && functions[*t-FUNCTION].spec ) {
01275             s1 = t + t[1];
01276             s2 = t + FUNHEAD;
01277             while ( s2 < s1 ) {
01278                 bind = bbb;
01279                 while ( bind < binst ) {
01280                     if ( *bind == *s2 ) goto FunBrac;
01281                     bind++;
01282                 }
01283                 s2++;
01284             }
01285         }
01286         NCOPY(t2,t,i);
01287         continue;
01288 FunBrac:
01289         NCOPY(t1,t,i);
01290         continue;
01291     }
01292 /*
01293 We have left: DELTA, INDEX, VECTOR, DOTPRODUCT, SYMBOL
01294 */
01295     if ( *t == DELTA ) {
01296         if ( b < bStop && *b == DELTA ) {
01297             b += b[1];
01298             NCOPY(t1,t,i);
01299         }
01300     }
01301     else if ( *t == INDEX ) {
01302         if ( bwild ) {
01303             m1 = t1; m2 = t2;
01304             *t1++ = *t; t1++; *t2++ = *t; t2++;
01305             bind = bbb;
01306             j = t[1] - 2;
01307             t += 2;
01308             while ( --j >= 0 ) {
01309                 while ( *bind < *t && bind < binst ) bind++;
01310                 if ( *bind == *t && bind < binst ) {
01311                     *t1++ = *t++;
01312                 }
01313                 else if ( bset ) {
01314                     WORD *b3 = bss;
01315                     while ( b3 < bns ) {
01316                         if ( b3[1] == CVECTOR ) {
01317                             SETS set = Sets+b3[0]; WORD i;
01318                             for ( i = set->first; i < set->last; i++ ) {
01319                                 if ( SetElements[i] == *t ) {
01320                                     *t1++ = *t++;
01321                                     goto nextind;
01322                                 }
01323                             }
01324                         }

```

```

01325             b3 += 2;
01326         }
01327         *t2++ = *t++;
01328     }
01329     else *t2++ = *t++;
01330 nextind:;
01331     }
01332     m1[1] = WORDDIFF(t1,m1);
01333     if ( m1[1] == 2 ) t1 = m1;
01334     m2[1] = WORDDIFF(t2,m2);
01335     if ( m2[1] == 2 ) t2 = m2;
01336 }
01337 else if ( bset ) {
01338     m1 = t1; m2 = t2;
01339     *t1++ = *t; t1++; *t2++ = *t; t2++;
01340     j = t[1] - 2;
01341     t += 2;
01342     while ( --j >= 0 ) {
01343         WORD *b3 = bss;
01344         while ( b3 < bns ) {
01345             if ( b3[1] == CVECTOR ) {
01346                 SETS set = Sets+b3[0]; WORD i;
01347                 for ( i = set->first; i < set->last; i++ ) {
01348                     if ( SetElements[i] == *t ) {
01349                         *t1++ = *t++;
01350                         goto nextind2;
01351                     }
01352                 }
01353             }
01354             b3 += 2;
01355         }
01356         *t2++ = *t++;
01357 nextind2:;
01358     }
01359     m1[1] = WORDDIFF(t1,m1);
01360     if ( m1[1] == 2 ) t1 = m1;
01361     m2[1] = WORDDIFF(t2,m2);
01362     if ( m2[1] == 2 ) t2 = m2;
01363 }
01364 else {
01365     NCOPY(t2,t,i);
01366 }
01367 }
01368 else if ( *t == VECTOR ) {
01369     if ( ( b < bStop && *b == VECTOR ) || bwild ) {
01370         if ( b < bStop && *b == VECTOR ) {
01371             bb = b + b[1]; b += 2;
01372         }
01373         else bb = b;
01374         j = t[1] - 2;
01375         m1 = t1; m2 = t2; *t1++ = *t; *t2++ = *t; t1++; t2++; t += 2;
01376         while ( j > 0 ) {
01377             j -= 2;
01378             while ( b < bb && ( *b < *t ||
01379                 ( *b == *t && b[1] < t[1] ) ) ) b += 2;
01380             if ( b < bb && ( *t == *b && t[1] == b[1] ) ) {
01381                 *t1++ = *t++; *t1++ = *t++; goto nextvec;
01382             }
01383             else if ( bwild ) {
01384                 bind = bbb;
01385                 while ( bind < binst ) {
01386                     if ( *t == *bind || t[1] == *bind ) {
01387                         *t1++ = *t++; *t1++ = *t++;
01388                         goto nextvec;
01389                     }
01390                     bind++;
01391                 }
01392             }
01393             if ( bset ) {
01394                 WORD *b3 = bss;
01395                 while ( b3 < bns ) {
01396                     if ( b3[1] == CVECTOR ) {
01397                         SETS set = Sets+b3[0]; WORD i;
01398                         for ( i = set->first; i < set->last; i++ ) {
01399                             if ( SetElements[i] == *t ) {
01400                                 *t1++ = *t++; *t1++ = *t++;
01401                                 goto nextvec;
01402                             }
01403                         }
01404                     }
01405                     b3 += 2;
01406                 }
01407             }
01408             *t2++ = *t++; *t2++ = *t++;
01409 nextvec:;
01410         }
01411         m1[1] = WORDDIFF(t1,m1);

```

```

01412         if ( m1[1] == 2 ) t1 = m1;
01413         m2[1] = WORDDIF(t2,m2);
01414         if ( m2[1] == 2 ) t2 = m2;
01415     }
01416     else if ( bset ) {
01417         m1 = t1; *t1++ = *t; t1++;
01418         m2 = t2; *t2++ = *t; t2++;
01419         s2 = t + i; t += 2;
01420         while ( t < s2 ) {
01421             WORD *b3 = bss;
01422             while ( b3 < bns ) {
01423                 if ( b3[1] == CVECTOR ) {
01424                     SETS set = Sets+b3[0]; WORD i;
01425                     for ( i = set->first; i < set->last; i++ ) {
01426                         if ( SetElements[i] == *t ) {
01427                             *t1++ = *t++; *t1++ = *t++;
01428                             goto nextvec2;
01429                         }
01430                     }
01431                 }
01432                 b3 += 2;
01433             }
01434             *t2++ = *t++; *t2++ = *t++;
01435 nextvec2:;
01436         }
01437         m1[1] = WORDDIF(t1,m1);
01438         if ( m1[1] == 2 ) t1 = m1;
01439         m2[1] = WORDDIF(t2,m2);
01440         if ( m2[1] == 2 ) t2 = m2;
01441     }
01442     else {
01443         NCOPY(t2,t,i);
01444     }
01445 }
01446 else if ( *t == DOTPRODUCT ) {
01447     if ( ( b < bStop && *b == *t ) || bwild ) {
01448         m1 = t1; *t1++ = *t; t1++;
01449         m2 = t2; *t2++ = *t; t2++;
01450         if ( b >= bStop || *b != *t ) { bb = b; s1 = b; }
01451         else {
01452             s1 = b + b[1]; bb = b + 2;
01453         }
01454         s2 = t + i; t += 2;
01455         while ( t < s2 && ( bb < s1 || bwild || bset ) ) {
01456             while ( bb < s1 && ( *bb < *t ||
01457                 ( *bb == *t && bb[1] < t[1] ) ) ) bb += 3;
01458             if ( bb < s1 && *bb == *t && bb[1] == t[1] ) {
01459                 *t1++ = *t++; *t1++ = *t++; *t1++ = *t++; bb += 3;
01460                 goto nextdot;
01461             }
01462             else if ( bwild ) {
01463                 bind = bbb;
01464                 while ( bind < binst ) {
01465                     if ( *bind == *t || *bind == t[1] ) {
01466                         *t1++ = *t++; *t1++ = *t++; *t1++ = *t++;
01467                         goto nextdot;
01468                     }
01469                     bind++;
01470                 }
01471             }
01472             if ( bset ) {
01473                 WORD *b3 = bss;
01474                 while ( b3 < bns ) {
01475                     if ( b3[1] == CVECTOR ) {
01476                         SETS set = Sets+b3[0]; WORD i;
01477                         for ( i = set->first; i < set->last; i++ ) {
01478                             if ( SetElements[i] == *t || SetElements[i] == t[1] ) {
01479                                 *t1++ = *t++; *t1++ = *t++; *t1++ = *t++;
01480                                 goto nextdot;
01481                             }
01482                         }
01483                     }
01484                     b3 += 2;
01485                 }
01486             }
01487             *t2++ = *t++; *t2++ = *t++; *t2++ = *t++;
01488 nextdot:;
01489         }
01490         while ( t < s2 ) *t2++ = *t++;
01491         m1[1] = WORDDIF(t1,m1);
01492         if ( m1[1] == 2 ) t1 = m1;
01493         m2[1] = WORDDIF(t2,m2);
01494         if ( m2[1] == 2 ) t2 = m2;
01495     }
01496     else if ( bset ) {
01497         m1 = t1; *t1++ = *t; t1++;
01498         m2 = t2; *t2++ = *t; t2++;

```

```

01499         s2 = t + i; t += 2;
01500         while ( t < s2 ) {
01501             WORD *b3 = bss;
01502             while ( b3 < bns ) {
01503                 if ( b3[1] == CVECTOR ) {
01504                     SETS set = Sets+b3[0]; WORD i;
01505                     for ( i = set->first; i < set->last; i++ ) {
01506                         if ( SetElements[i] == *t || SetElements[i] == t[1] ) {
01507                             *t1++ = *t++; *t1++ = *t++; *t1++ = *t++;
01508                             goto nextdot2;
01509                         }
01510                     }
01511                 }
01512                 b3 += 2;
01513             }
01514             *t2++ = *t++; *t2++ = *t++; *t2++ = *t++;
01515 nextdot2:;
01516         }
01517         m1[1] = WORDDIF(t1,m1);
01518         if ( m1[1] == 2 ) t1 = m1;
01519         m2[1] = WORDDIF(t2,m2);
01520         if ( m2[1] == 2 ) t2 = m2;
01521     }
01522     else { NCOPY(t2,t,i); }
01523 }
01524 else if ( *t == SYMBOL ) {
01525     if ( b < bStop && *b == *t ) {
01526         m1 = t1; *t1++ = *t; t1++;
01527         m2 = t2; *t2++ = *t; t2++;
01528         s1 = b + b[1]; bb = b+2;
01529         s2 = t + i; t += 2;
01530         while ( bb < s1 && t < s2 ) {
01531             while ( bb < s1 && *bb < *t ) bb += 2;
01532             if ( bb >= s1 ) {
01533                 if ( bset ) goto TrySymbolSet;
01534                 break;
01535             }
01536             if ( *bb == *t ) { *t1++ = *t++; *t1++ = *t++; }
01537             else if ( bset ) {
01538                 WORD *bbb;
01539 TrySymbolSet:
01540                 bbb = bss;
01541                 while ( bbb < bns ) {
01542                     if ( bbb[1] == CSYMBOL ) { /* Set of symbols */
01543                         SETS set = Sets+bbb[0]; WORD i;
01544                         for ( i = set->first; i < set->last; i++ ) {
01545                             if ( SetElements[i] == *t ) {
01546                                 *t1++ = *t++; *t1++ = *t++;
01547                                 goto NextSymbol;
01548                             }
01549                         }
01550                     }
01551                     bbb += 2;
01552                 }
01553                 *t2++ = *t++; *t2++ = *t++;
01554             }
01555             else { *t2++ = *t++; *t2++ = *t++; }
01556 NextSymbol:;
01557         }
01558         while ( t < s2 ) *t2++ = *t++;
01559         m1[1] = WORDDIF(t1,m1);
01560         if ( m1[1] == 2 ) t1 = m1;
01561         m2[1] = WORDDIF(t2,m2);
01562         if ( m2[1] == 2 ) t2 = m2;
01563     }
01564     else if ( bset ) {
01565         WORD *bbb;
01566         m1 = t1; *t1++ = *t; t1++;
01567         m2 = t2; *t2++ = *t; t2++;
01568         s2 = t + i; t += 2;
01569         while ( t < s2 ) {
01570             bbb = bss;
01571             while ( bbb < bns ) {
01572                 if ( bbb[1] == CSYMBOL ) { /* Set of symbols */
01573                     SETS set = Sets+bbb[0]; WORD i;
01574                     for ( i = set->first; i < set->last; i++ ) {
01575                         if ( SetElements[i] == *t ) {
01576                             *t1++ = *t++; *t1++ = *t++;
01577                             goto NextSymbol2;
01578                         }
01579                     }
01580                 }
01581                 bbb += 2;
01582             }
01583             *t2++ = *t++; *t2++ = *t++;
01584 NextSymbol2:;
01585         }

```

```

01586         m1[1] = WORDDIF(t1,m1);
01587         if ( m1[1] == 2 ) t1 = m1;
01588         m2[1] = WORDDIF(t2,m2);
01589         if ( m2[1] == 2 ) t2 = m2;
01590     }
01591     else { NCOPY(t2,t,i); }
01592 }
01593 else {
01594     NCOPY(t2,t,i);
01595 }
01596 }
01597 if ( ( i = WORDDIF(tStop,t) ) > 0 ) NCOPY(t2,t,i);
01598 if ( AR.BracketOn < 0 ) {
01599     s1 = t1; t1 = t2; t2 = s1;
01600 }
01601 do { *t2++ = *t++; } while ( t < (WORD *)tStopa );
01602 t = AT.WorkPointer;
01603 i = WORDDIF(t1,term1);
01604 *t++ = 4 + i + WORDDIF(t2,term2);
01605 t += i;
01606 *t++ = HAAKJE;
01607 *t++ = 3;
01608 *t++ = 0; /* This feature won't be used for a while */
01609 i = WORDDIF(t2,term2);
01610 t1 = term2;
01611 if ( i > 0 ) NCOPY(t,t1,i);
01612
01613 AT.WorkPointer = t;
01614
01615 return(0);
01616 }
01617
01618 /*
01619     #] PutBracket :
01620     #[ SpecialCleanup :
01621 */
01622
01623 void SpecialCleanup(PHEAD0)
01624 {
01625     GETBIDENTITY
01626     if ( AT.previousEfactor ) M_free(AT.previousEfactor,"Efactor cache");
01627     AT.previousEfactor = 0;
01628 }
01629
01630 /*
01631     #] SpecialCleanup :
01632     #[ SetMods :
01633 */
01634
01635 #ifndef WITHPTHREADS
01636
01637 void SetMods(void)
01638 {
01639     int i, n;
01640     if ( AN.cmod != 0 ) M_free(AN.cmod,"AN.cmod");
01641     n = ABS(AN.ncmod);
01642     AN.cmod = (UWORD *)Malloc1(sizeof(WORD)*n,"AN.cmod");
01643     for ( i = 0; i < n; i++ ) AN.cmod[i] = AC.cmod[i];
01644 }
01645
01646 #endif
01647
01648 /*
01649     #] SetMods :
01650     #[ UnSetMods :
01651 */
01652
01653 #ifndef WITHPTHREADS
01654
01655 void UnSetMods(void)
01656 {
01657     if ( AN.cmod != 0 ) M_free(AN.cmod,"AN.cmod");
01658     AN.cmod = 0;
01659 }
01660
01661 #endif
01662
01663 /*
01664     #] UnSetMods :
01665     #] DoExecute :
01666     #[ Expressions :
01667     #[ ExchangeExpressions :
01668 */
01669
01670 void ExchangeExpressions(int num1, int num2)
01671 {
01672     GETIDENTITY

```

```

01673     WORD node1, node2, namesize, TMproto[SUBEXPSIZE];
01674     INDEXENTRY *ind;
01675     EXPRESSIONS e1, e2;
01676     LONG a;
01677     SBYTE *s1, *s2;
01678     int i;
01679     e1 = Expressions + num1;
01680     e2 = Expressions + num2;
01681     node1 = e1->node;
01682     node2 = e2->node;
01683     AC.exprnames->namenode[node1].number = num2;
01684     AC.exprnames->namenode[node2].number = num1;
01685     a = e1->name; e1->name = e2->name; e2->name = a;
01686     namesize = e1->namesize; e1->namesize = e2->namesize; e2->namesize = namesize;
01687     e1->node = node2;
01688     e2->node = node1;
01689     if ( e1->status == STOREDEXPRESSION ) {
01690 /*
01691         Find the name in the index and replace by the new name
01692 */
01693         TMproto[0] = EXPRESSION;
01694         TMproto[1] = SUBEXPSIZE;
01695         TMproto[2] = num1;
01696         TMproto[3] = 1;
01697         { int ie; for ( ie = 4; ie < SUBEXPSIZE; ie++ ) TMproto[ie] = 0; }
01698         AT.TMaddr = TMproto;
01699         ind = FindInIndex(num1,&AR.StoreData,0,0);
01700         s1 = (SBYTE *) (AC.exprnames->namebuffer+e1->name);
01701         i = e1->namesize;
01702         s2 = ind->name;
01703         NCOPY(s2,s1,i);
01704         *s2 = 0;
01705         SeekFile(AR.StoreData.Handle,&(e1->onfile),SEEK_SET);
01706         if ( WriteFile(AR.StoreData.Handle,(UBYTE *)ind,
01707             (LONG)(sizeof(INDEXENTRY)) != sizeof(INDEXENTRY) ) {
01708             MesPrint("File error while exchanging expressions");
01709             Terminate(-1);
01710         }
01711         FlushFile(AR.StoreData.Handle);
01712     }
01713     if ( e2->status == STOREDEXPRESSION ) {
01714 /*
01715         Find the name in the index and replace by the new name
01716 */
01717         TMproto[0] = EXPRESSION;
01718         TMproto[1] = SUBEXPSIZE;
01719         TMproto[2] = num2;
01720         TMproto[3] = 1;
01721         { int ie; for ( ie = 4; ie < SUBEXPSIZE; ie++ ) TMproto[ie] = 0; }
01722         AT.TMaddr = TMproto;
01723         ind = FindInIndex(num1,&AR.StoreData,0,0);
01724         s1 = (SBYTE *) (AC.exprnames->namebuffer+e2->name);
01725         i = e2->namesize;
01726         s2 = ind->name;
01727         NCOPY(s2,s1,i);
01728         *s2 = 0;
01729         SeekFile(AR.StoreData.Handle,&(e2->onfile),SEEK_SET);
01730         if ( WriteFile(AR.StoreData.Handle,(UBYTE *)ind,
01731             (LONG)(sizeof(INDEXENTRY)) != sizeof(INDEXENTRY) ) {
01732             MesPrint("File error while exchanging expressions");
01733             Terminate(-1);
01734         }
01735         FlushFile(AR.StoreData.Handle);
01736     }
01737 }
01738 /*
01739     #] ExchangeExpressions :
01740     #[ GetFirstBracket :
01741 */
01742 */
01743
01744 int GetFirstBracket(WORD *term, int num)
01745 {
01746 /*
01747     Gets the first bracket of the expression 'num'
01748     Puts it in term. If no brackets the answer is one.
01749     Routine should be thread-safe
01750 */
01751     GETIDENTITY
01752     POSITION position, oldposition;
01753     RENUMBER renumber;
01754     FILEHANDLE *fi;
01755     WORD type, *oldcomppointer, oldonefile, numword;
01756     WORD *t, *tstop;
01757
01758     oldcomppointer = AR.CompressPointer;
01759     type = Expressions[num].status;

```

```

01760     if ( type == STOREDEXPRESSION ) {
01761         WORD TMproto[SUBEXPSIZE];
01762         TMproto[0] = EXPRESSION;
01763         TMproto[1] = SUBEXPSIZE;
01764         TMproto[2] = num;
01765         TMproto[3] = 1;
01766         { int ie; for ( ie = 4; ie < SUBEXPSIZE; ie++ ) TMproto[ie] = 0; }
01767         AT.TMaddr = TMproto;
01768         PUTZERO(position);
01769         if ( ( renumber = GetTable(num,&position,0) ) == 0 ) {
01770             MesCall("GetFirstBracket");
01771             SETERROR(-1)
01772         }
01773         if ( GetFromStore(term,&position,renumber,&numword,num) < 0 ) {
01774             MesCall("GetFirstBracket");
01775             SETERROR(-1)
01776         }
01777         /*
01778         #ifdef WITHPTHREADS
01779         */
01780         if ( renumber->symb.lo != AN.dummyrenumlist )
01781             M_free(renumber->symb.lo,"VarSpace");
01782         M_free(renumber,"Renumber");
01783         /*
01784         #endif
01785         */
01786     }
01787     else { /* Active expression */
01788         oldonefile = AR.GetOneFile;
01789         if ( type == HIDDENLEXPRESSION || type == HIDDENEXPRESSION ) {
01790             AR.GetOneFile = 2; fi = AR.hidefile;
01791         }
01792         else {
01793             AR.GetOneFile = 0; fi = AR.infile;
01794         }
01795         if ( fi->handle >= 0 ) {
01796             PUTZERO(oldposition);
01797             /*
01798             SeekFile(fi->handle,&oldposition,SEEK_CUR);
01799             */
01800         }
01801         else {
01802             SETBASEPOSITION(oldposition,fi->POfill-fi->PObuffer);
01803         }
01804         position = AS.OldOnFile[num];
01805         if ( GetOneTerm(BHEAD term,fi,&position,1) < 0
01806             || ( GetOneTerm(BHEAD term,fi,&position,1) < 0 ) ) {
01807             MLOCK(ErrorMessageLock);
01808             MesCall("GetFirstBracket");
01809             MUNLOCK(ErrorMessageLock);
01810             SETERROR(-1)
01811         }
01812         if ( fi->handle >= 0 ) {
01813             /*
01814             SeekFile(fi->handle,&oldposition,SEEK_SET);
01815             if ( ISNEGPOS(oldposition) ) {
01816                 MLOCK(ErrorMessageLock);
01817                 MesPrint("File error");
01818                 MUNLOCK(ErrorMessageLock);
01819                 SETERROR(-1)
01820             }
01821             */
01822         }
01823         else {
01824             fi->POfill = fi->PObuffer+BASEPOSITION(oldposition);
01825         }
01826         AR.GetOneFile = oldonefile;
01827     }
01828     AR.CompressPointer = oldcomppointer;
01829     if ( *term ) {
01830         tstop = term + *term; tstop -= ABS(tstop[-1]);
01831         t = term + 1;
01832         while ( t < tstop ) {
01833             if ( *t == HAAKJE ) break;
01834             t += t[1];
01835         }
01836         if ( t >= tstop ) {
01837             term[0] = 4; term[1] = 1; term[2] = 1; term[3] = 3;
01838         }
01839         else {
01840             *t++ = 1; *t++ = 1; *t++ = 3; *term = t - term;
01841         }
01842     }
01843     else {
01844         term[0] = 4; term[1] = 1; term[2] = 1; term[3] = 3;
01845     }
01846     return(*term);

```

```

01847 }
01848
01849 /*
01850      #] GetFirstBracket :
01851      #[ GetFirstTerm :
01852 */
01853
01863 int GetFirstTerm(WORD *term, int num, int pre)
01864 {
01865     GETIDENTITY
01866     POSITION position, oldposition;
01867     RENUMBER renumber;
01868     FILEHANDLE *fi;
01869     WORD type, *oldcomppointer, olddonefile, numword;
01870
01871     oldcomppointer = AR.CompressPointer;
01872     type = Expressions[num].status;
01873     if ( type == STOREDEXPRESSION ) {
01874         WORD TMproto[SUBEXPSIZE];
01875         TMproto[0] = EXPRESSION;
01876         TMproto[1] = SUBEXPSIZE;
01877         TMproto[2] = num;
01878         TMproto[3] = 1;
01879         { int ie; for ( ie = 4; ie < SUBEXPSIZE; ie++ ) TMproto[ie] = 0; }
01880         AT.TMaddr = TMproto;
01881         PUTZERO(position);
01882         if ( ( renumber = GetTable(num,&position,0) ) == 0 ) {
01883             MesCall("GetFirstTerm");
01884             SETERROR(-1)
01885         }
01886         if ( GetFromStore(term,&position,renumber,&numword,num) < 0 ) {
01887             MesCall("GetFirstTerm");
01888             SETERROR(-1)
01889         }
01890     }
01891     #ifdef WITHPTHREADS
01892     /*
01893         if ( renumber->symb.lo != AN.dummyrenumlist )
01894             M_free(renumber->symb.lo,"VarSpace");
01895         M_free(renumber,"Renumber");
01896     */
01897     #endif
01898     /*
01899     }
01900     else {
01901         /* Active expression */
01902         olddonefile = AR.GetOneFile;
01903         if ( type == HIDDENLEXPRESSION || type == HIDDENGEXPRESSION ) {
01904             AR.GetOneFile = 2; fi = AR.hidefile;
01905         }
01906         else {
01907             AR.GetOneFile = 0;
01908             if ( Expressions[num].replace == NEWLYDEFINEDEXPRESSION ) {
01909                 /* During execution, if the expression has already been processed it
01910                  will be in the outfile. If it has not, the usage is illegal according
01911                  to the manual, though no error is given. */
01912                 if ( pre == 0 ) { fi = AR.outfile; }
01913                 /* During preprocessing, the expression certainly has not been processed
01914                  yet. Print an error and terminate. */
01915                 else {
01916                     MesPrint("&isnumerical: expression is not yet defined!");
01917                     SETERROR(-1);
01918                 }
01919             }
01920             else {
01921                 /* During execution, we should use the definition as stored at the end
01922                  of the previous module. This is in the infile. */
01923                 if ( pre == 0 ) { fi = AR.infile; }
01924                 /* During preprocessing, this function is called before the RevertScratch
01925                  at the beginning of this module's execution. Thus the expressions are
01926                  in the outfile of the previous module. */
01927                 else { fi = AR.outfile; }
01928             }
01929         }
01930         if ( fi->handle >= 0 ) {
01931             PUTZERO(oldposition);
01932             /*
01933             SeekFile(fi->handle,&oldposition,SEEK_CUR);
01934             */
01935         }
01936         else {
01937             SETBASEPOSITION(oldposition,fi->POfill-fi->PObuffer);
01938         }
01939         position = AS.OldOnFile[num];
01940         if ( GetOneTerm(BHEAD term,fi,&position,1) < 0
01941             || ( GetOneTerm(BHEAD term,fi,&position,1) < 0 ) ) {
01942             MLOCK(ErrorMessageLock);
01943             MesCall("GetFirstTerm");
01944         }
01945     }
01946 }

```



```

01943         MUNLOCK(ErrorMessageLock);
01944         SETERROR(-1)
01945     }
01946     if ( fi->handle >= 0 ) {
01947 /*
01948         SeekFile(fi->handle,&oldposition,SEEK_SET);
01949         if ( ISNEGPOS(oldposition) ) {
01950             MLOCK(ErrorMessageLock);
01951             MesPrint("File error");
01952             MUNLOCK(ErrorMessageLock);
01953             SETERROR(-1)
01954         }
01955 */
01956     }
01957     else {
01958         fi->POfill = fi->PObuffer+BASEPOSITION(oldposition);
01959     }
01960     AR.GetOneFile = oldonefile;
01961 }
01962 AR.CompressPointer = oldcomppointer;
01963 return(*term);
01964 }
01965 /*
01966 #] GetFirstTerm :
01967 #[ GetContent :
01968 */
01970
01971 int GetContent(WORD *content, int num)
01972 {
01973 /*
01974     Gets the content of the expression 'num'
01975     Puts it in content.
01976     Routine should be thread-safe
01977     The content is defined as the term that will make the expression 'num'
01978     with integer coefficients, no GCD and all common factors taken out,
01979     all negative powers removed when we divide the expression by this
01980     content.
01981 */
01982     GETIDENTITY
01983     POSITION position, oldposition;
01984     RENUMBER renumber;
01985     FILEHANDLE *fi;
01986     WORD type, *oldcomppointer, oldonefile, numword, *term, i;
01987     WORD *cbuffer = TermMalloc("GetContent");
01988     WORD *oldworkpointer = AT.WorkPointer;
01989
01990     oldcomppointer = AR.CompressPointer;
01991     type = Expressions[num].status;
01992     if ( type == STOREDEXPRESSION ) {
01993         WORD TMproto[SUBEXPSIZE];
01994         TMproto[0] = EXPRESSION;
01995         TMproto[1] = SUBEXPSIZE;
01996         TMproto[2] = num;
01997         TMproto[3] = 1;
01998         { int ie; for ( ie = 4; ie < SUBEXPSIZE; ie++ ) TMproto[ie] = 0; }
01999         AT.TMaddr = TMproto;
02000         PUTZERO(position);
02001         if ( ( renumber = GetTable(num,&position,0) ) == 0 ) goto CalledFrom;
02002         if ( GetFromStore(cbuffer,&position,renumber,&numword,num) < 0 ) goto CalledFrom;
02003         for(;;) {
02004             term = oldworkpointer;
02005             AR.CompressPointer = oldcomppointer;
02006             if ( GetFromStore(term,&position,renumber,&numword,num) < 0 ) goto CalledFrom;
02007             if ( *term == 0 ) break;
02008 /*
02009             'merge' the two terms
02010 */
02011             if ( ContentMerge(BHEAD cbuffer,term) < 0 ) goto CalledFrom;
02012         }
02013 /*
02014 #ifdef WITHPTHREADS
02015 */
02016         if ( renumber->symb.lo != AN.dummyrenumlist )
02017             M_free(renumber->symb.lo,"VarSpace");
02018         M_free(renumber,"Renummer");
02019 /*
02020 #endif
02021 */
02022     }
02023     else { /* Active expression */
02024         oldonefile = AR.GetOneFile;
02025         if ( type == HIDDENLEXPRESSION || type == HIDDENEXPRESSION ) {
02026             AR.GetOneFile = 2; fi = AR.hidefile;
02027         }
02028         else {
02029             AR.GetOneFile = 0;

```

```

02030         if ( Expressions[num].replace == NEWLYDEFINEDEXPRESSION )
02031             fi = AR.outfile;
02032         else fi = AR.infile;
02033     }
02034     if ( fi->handle >= 0 ) {
02035         PUTZERO(oldposition);
02036     /*
02037         SeekFile(fi->handle,&oldposition,SEEK_CUR);
02038     */
02039     }
02040     else {
02041         SETBASEPOSITION(oldposition,fi->POfill-fi->PObuffer);
02042     }
02043     position = AS.OldOnFile[num];
02044     if ( GetOneTerm(BHEAD cbuffer,fi,&position,1) < 0 ) goto CalledFrom;
02045     AR.CompressPointer = oldcomppointer;
02046     if ( GetOneTerm(BHEAD cbuffer,fi,&position,1) < 0 ) goto CalledFrom;
02047 /*
02048     Now go through the terms. For each term we have to test whether
02049     what is in cbuffer is also in that term. If not, we have to remove
02050     it from cbuffer. Additionally we have to accumulate the GCD of the
02051     numerators and the LCM of the denominators. This is all done in the
02052     routine ContentMerge.
02053 */
02054     for(;;) {
02055         term = oldworkpointer;
02056         AR.CompressPointer = oldcomppointer;
02057         if ( GetOneTerm(BHEAD term,fi,&position,1) < 0 ) goto CalledFrom;
02058         if ( *term == 0 ) break;
02059     /*
02060         'merge' the two terms
02061     */
02062         if ( ContentMerge(BHEAD cbuffer,term) < 0 ) goto CalledFrom;
02063     }
02064     if ( fi->handle < 0 ) {
02065         fi->POfill = fi->PObuffer+BASEPOSITION(oldposition);
02066     }
02067     AR.GetOneFile = oldonefile;
02068 }
02069 AR.CompressPointer = oldcomppointer;
02070 for ( i = 0; i < *cbuffer; i++ ) content[i] = cbuffer[i];
02071 TermFree(cbuffer,"GetContent");
02072 AT.WorkPointer = oldworkpointer;
02073 return(*content);
02074 CalledFrom:
02075 MLOCK(ErrorMessageLock);
02076 MesCall("GetContent");
02077 MUNLOCK(ErrorMessageLock);
02078 SETERROR(-1)
02079 }
02080
02081 /*
02082     #] GetContent :
02083     #[ CleanupTerm :
02084
02085     Removes noncommuting objects from the term
02086 */
02087
02088 int CleanupTerm(WORD *term)
02089 {
02090     WORD *tstop, *t, *tfill, *tt;
02091     GETSTOP(term,tstop);
02092     t = term+1;
02093     while ( t < tstop ) {
02094         if ( *t >= FUNCTION && ( functions[*t-FUNCTION].commute || *t == DENOMINATOR ) ) {
02095             tfill = t; tt = t + t[1]; tstop = term + *term;
02096             while ( tt < tstop ) *tfill++ = *tt++;
02097             *term = tfill - term;
02098             tstop -= ABS(tfill[-1]);
02099         }
02100         else {
02101             t += t[1];
02102         }
02103     }
02104     return(0);
02105 }
02106
02107 /*
02108     #] CleanupTerm :
02109     #[ ContentMerge :
02110 */
02111
02112 WORD ContentMerge(PHEAD WORD *content, WORD *term)
02113 {
02114     GETBIDENTITY
02115     WORD *cstop, csize, crsize, sign = 1, numsize, densize, i, tsize, tdsiz;
02116     UWORD *num, *den, *tnum, *tden;

```

```

02117     WORD *outfill, *outb = TermMalloc("ContentMerge"), *ct;
02118     WORD *t, *tstop, tsize, trsize, *told;
02119     WORD *t1, *t2, *c1, *c2, i1, i2, *outl;
02120     WORD didsymbol = 0, diddotp = 0, tfirst;
02121     cstop = content + *content;
02122     csize = cstop[-1];
02123     if ( csize < 0 ) { sign = -sign; csize = -csize; }
02124     cstop -= csize;
02125     numsize = densize = crsize = (csize-1)/2;
02126     num = NumberMalloc("ContentMerge");
02127     den = NumberMalloc("ContentMerge");
02128     for ( i = 0; i < numsize; i++ ) num[i] = (UWORD) (cstop[i]);
02129     for ( i = 0; i < densize; i++ ) den[i] = (UWORD) (cstop[i+crsize]);
02130     while ( num[numsize-1] == 0 ) numsize--;
02131     while ( den[densize-1] == 0 ) densize--;
02132 /*
02133     First we do the coefficient
02134 */
02135     tstop = term + *term;
02136     tsize = tstop[-1];
02137     if ( tsize < 0 ) tsize = -tsize;
02138 /* else { sign = 1; } */
02139     tstop = tstop - tsize;
02140     tnsz = tdsz = trsize = (tsize-1)/2;
02141     tnum = (UWORD *)tstop; tden = (UWORD *) (tstop + trsize);
02142     while ( tnum[tnsz-1] == 0 ) tnsz--;
02143     while ( tden[tdsz-1] == 0 ) tdsz--;
02144     GcdLong(BHEAD num, numsize, tnum, tnsz, num, &numsize);
02145     if ( LcmLong(BHEAD den, densize, tden, tdsz, den, &densize) ) goto CalledFrom;
02146     outfill = outb + 1;
02147     ct = content + 1;
02148     t = term + 1;
02149     while ( ct < cstop ) {
02150         switch ( *ct ) {
02151             case SYMBOL:
02152                 didsymbol = 1;
02153                 t = term+1;
02154                 while ( t < tstop && *t != *ct ) t += t[1];
02155                 if ( t >= tstop ) break;
02156                 t1 = t+2; t2 = t+t[1];
02157                 c1 = ct+2; c2 = ct+ct[1];
02158                 outl = outfill; *outfill++ = *ct; outfill++;
02159                 while ( c1 < c2 && t1 < t2 ) {
02160                     if ( *c1 == *t1 ) {
02161                         if ( t1[1] <= c1[1] ) {
02162                             *outfill++ = *t1++; *outfill++ = *t1++;
02163                             c1 += 2;
02164                         }
02165                         else {
02166                             *outfill++ = *c1++; *outfill++ = *c1++;
02167                             t1 += 2;
02168                         }
02169                     }
02170                     else if ( *c1 < *t1 ) {
02171                         if ( c1[1] < 0 ) {
02172                             *outfill++ = *c1++; *outfill++ = *c1++;
02173                         }
02174                         else { c1 += 2; }
02175                     }
02176                     else {
02177                         if ( t1[1] < 0 ) {
02178                             *outfill++ = *t1++; *outfill++ = *t1++;
02179                         }
02180                         else t1 += 2;
02181                     }
02182                 }
02183                 while ( c1 < c2 ) {
02184                     if ( c1[1] < 0 ) { *outfill++ = c1[0]; *outfill++ = c1[1]; }
02185                     c1 += 2;
02186                 }
02187                 while ( t1 < t2 ) {
02188                     if ( t1[1] < 0 ) { *outfill++ = t1[0]; *outfill++ = t1[1]; }
02189                     t1 += 2;
02190                 }
02191                 outl[1] = outfill - outl;
02192                 if ( outl[1] == 2 ) outfill = outl;
02193                 break;
02194             case DOTPRODUCT:
02195                 diddotp = 1;
02196                 t = term+1;
02197                 while ( t < tstop && *t != *ct ) t += t[1];
02198                 if ( t >= tstop ) break;
02199                 t1 = t+2; t2 = t+t[1];
02200                 c1 = ct+2; c2 = ct+ct[1];
02201                 outl = outfill; *outfill++ = *ct; outfill++;
02202                 while ( c1 < c2 && t1 < t2 ) {
02203                     if ( *c1 == *t1 && c1[1] == t1[1] ) {

```

```

02204         if ( t1[2] <= c1[2] ) {
02205             *outfill++ = *t1++; *outfill++ = *t1++; *outfill++ = *t1++;
02206             c1 += 3;
02207         }
02208         else {
02209             *outfill++ = *c1++; *outfill++ = *c1++; *outfill++ = *c1++;
02210             t1 += 3;
02211         }
02212     }
02213     else if ( *c1 < *t1 || ( *c1 == *t1 && c1[1] < t1[1] ) ) {
02214         if ( c1[2] < 0 ) {
02215             *outfill++ = *c1++; *outfill++ = *c1++; *outfill++ = *c1++;
02216         }
02217         else { c1 += 3; }
02218     }
02219     else {
02220         if ( t1[2] < 0 ) {
02221             *outfill++ = *t1++; *outfill++ = *t1++; *outfill++ = *t1++;
02222         }
02223         else t1 += 3;
02224     }
02225 }
02226 while ( c1 < c2 ) {
02227     if ( c1[2] < 0 ) { *outfill++ = c1[0]; *outfill++ = c1[1]; *outfill++ = c1[1]; }
02228     c1 += 3;
02229 }
02230 while ( t1 < t2 ) {
02231     if ( t1[2] < 0 ) { *outfill++ = t1[0]; *outfill++ = t1[1]; *outfill++ = t1[1]; }
02232     t1 += 3;
02233 }
02234 outl[1] = outfill - outl;
02235 if ( outl[1] == 2 ) outfill = outl;
02236 break;
02237 case INDEX:
02238     t = term+1;
02239     while ( t < tstop && *t != *ct ) t += t[1];
02240     if ( t >= tstop ) break;
02241     t1 = t+2; t2 = t+t[1];
02242     c1 = ct+2; c2 = ct+ct[1];
02243     outl = outfill; *outfill++ = *ct; outfill++;
02244     while ( c1 < c2 && t1 < t2 ) {
02245         if ( *c1 == *t1 ) {
02246             *outfill++ = *c1++;
02247             t1 += 1;
02248         }
02249         else if ( *c1 < *t1 ) { c1 += 1; }
02250         else { t1 += 1; }
02251     }
02252     outl[1] = outfill - outl;
02253     if ( outl[1] == 2 ) outfill = outl;
02254     break;
02255 case VECTOR:
02256 case DELTA:
02257     t = term+1;
02258     while ( t < tstop && *t != *ct ) t += t[1];
02259     if ( t >= tstop ) break;
02260     t1 = t+2; t2 = t+t[1];
02261     c1 = ct+2; c2 = ct+ct[1];
02262     outl = outfill; *outfill++ = *ct; outfill++;
02263     while ( c1 < c2 && t1 < t2 ) {
02264         if ( *c1 == *t1 && c1[1] && t1[1] ) {
02265             *outfill++ = *c1++; *outfill++ = *c1++;
02266             t1 += 2;
02267         }
02268         else if ( *c1 < *t1 || ( *c1 == *t1 && c1[1] < t1[1] ) ) {
02269             c1 += 2;
02270         }
02271         else {
02272             t1 += 2;
02273         }
02274     }
02275     outl[1] = outfill - outl;
02276     if ( outl[1] == 2 ) outfill = outl;
02277     break;
02278 case GAMMA:
02279 default: /* Functions */
02280     told = t;
02281     t = term+1;
02282     while ( t < tstop ) {
02283         if ( *t != *ct ) { t += t[1]; continue; }
02284         if ( ct[1] != t[1] ) { t += t[1]; continue; }
02285         if ( ct[2] != t[2] ) { t += t[1]; continue; }
02286         t1 = t; t2 = ct; i1 = t1[1]; i2 = t2[1];
02287         while ( i1 > 0 ) {
02288             if ( *t1 != *t2 ) break;
02289             t1++; t2++; i1--;
02290         }

```

```

02291             if ( i1 != 0 ) { t += t[1]; continue; }
02292             t1 = t;
02293             for ( i = 0; i < i2; i++ ) { *outfill++ = *t++; }
02294 /*
02295             Mark as 'used'. The flags must be different!
02296 */
02297             t1[2] |= SUBTERMUSED1;
02298             ct[2] |= SUBTERMUSED2;
02299             t = told;
02300             break;
02301         }
02302         break;
02303     }
02304     ct += ct[1];
02305 }
02306 if ( diddotp == 0 ) {
02307     t = term+1; while ( t < tstop && *t != DOTPRODUCT ) t += t[1];
02308     if ( t < tstop ) { /* now we need the negative powers */
02309         tfirst = 1; told = outfill;
02310         for ( i = 2; i < t[1]; i += 3 ) {
02311             if ( t[i+2] < 0 ) {
02312                 if ( tfirst ) { *outfill++ = DOTPRODUCT; *outfill++ = 0; tfirst = 0; }
02313                 *outfill++ = t[i]; *outfill++ = t[i+1]; *outfill++ = t[i+2];
02314             }
02315         }
02316         if ( outfill > told ) told[1] = outfill-told;
02317     }
02318 }
02319 if ( didsymbol == 0 ) {
02320     t = term+1; while ( t < tstop && *t != SYMBOL ) t += t[1];
02321     if ( t < tstop ) { /* now we need the negative powers */
02322         tfirst = 1; told = outfill;
02323         for ( i = 2; i < t[1]; i += 2 ) {
02324             if ( t[i+1] < 0 ) {
02325                 if ( tfirst ) { *outfill++ = SYMBOL; *outfill++ = 0; tfirst = 0; }
02326                 *outfill++ = t[i]; *outfill++ = t[i+1];
02327             }
02328         }
02329         if ( outfill > told ) told[1] = outfill-told;
02330     }
02331 }
02332 /*
02333 Now put the coefficient back.
02334 */
02335 if ( numsize < densize ) {
02336     for ( i = numsize; i < densize; i++ ) num[i] = 0;
02337     numsize = densize;
02338 }
02339 else if ( densize < numsize ) {
02340     for ( i = densize; i < numsize; i++ ) den[i] = 0;
02341     densize = numsize;
02342 }
02343 for ( i = 0; i < numsize; i++ ) *outfill++ = num[i];
02344 for ( i = 0; i < densize; i++ ) *outfill++ = den[i];
02345 csize = numsize+densize+1;
02346 if ( sign < 0 ) csize = -csize;
02347 *outfill++ = csize;
02348 *outb = outfill-outb;
02349 NumberFree(den,"ContentMerge");
02350 NumberFree(num,"ContentMerge");
02351 for ( i = 0; i < *outb; i++ ) content[i] = outb[i];
02352 TermFree(outb,"ContentMerge");
02353 /*
02354 Now we have to 'restore' the term to its original.
02355 We do not restore the content, because if anything was used the
02356 new content overwrites the old. 6-mar-2018 JV
02357 */
02358 t = term + 1;
02359 while ( t < tstop ) {
02360     if ( *t >= FUNCTION ) t[2] &= ~SUBTERMUSED1;
02361     t += t[1];
02362 }
02363 return(*content);
02364 CalledFrom:
02365 MLOCK(ErrorMessageLock);
02366 MesCall("GetContent");
02367 MUNLOCK(ErrorMessageLock);
02368 SETERROR(-1)
02369 }
02370
02371 /*
02372     #] ContentMerge :
02373     #[ TermsInExpression :
02374 */
02375
02376 LONG TermsInExpression(WORD num)
02377 {

```

```

02378     LONG x = Expressions[num].counter;
02379     if ( x >= 0 ) return(x);
02380     return(-1);
02381 }
02382
02383 /*
02384     #] TermsInExpression :
02385     #[ SizeOfExpression :
02386 */
02387
02388 LONG SizeOfExpression(WORD num)
02389 {
02390     LONG x = (LONG) (DIVPOS(Expressions[num].size, sizeof(WORD)));
02391     if ( x >= 0 ) return(x);
02392     return(-1);
02393 }
02394
02395 /*
02396     #] SizeOfExpression :
02397     #[ UpdatePositions :
02398 */
02399
02400 void UpdatePositions(void)
02401 {
02402     EXPRESSIONS e = Expressions;
02403     POSITION *old;
02404     WORD *oldw;
02405     int i;
02406     if ( NumExpressions > 0 &&
02407         ( AS.OldOnFile == 0 || AS.NumOldOnFile < NumExpressions ) ) {
02408         if ( AS.OldOnFile ) {
02409             old = AS.OldOnFile;
02410             AS.OldOnFile = (POSITION *)Malloca1(NumExpressions*sizeof(POSITION), "file pointers");
02411             for ( i = 0; i < AS.NumOldOnFile; i++ ) AS.OldOnFile[i] = old[i];
02412             AS.NumOldOnFile = NumExpressions;
02413             M_free(old, "process file pointers");
02414         }
02415         else {
02416             AS.OldOnFile = (POSITION *)Malloca1(NumExpressions*sizeof(POSITION), "file pointers");
02417             AS.NumOldOnFile = NumExpressions;
02418         }
02419     }
02420     if ( NumExpressions > 0 &&
02421         ( AS.OldNumFactors == 0 || AS.NumOldNumFactors < NumExpressions ) ) {
02422         if ( AS.OldNumFactors ) {
02423             oldw = AS.OldNumFactors;
02424             AS.OldNumFactors = (WORD *)Malloca1(NumExpressions*sizeof(WORD), "numfactors pointers");
02425             for ( i = 0; i < AS.NumOldNumFactors; i++ ) AS.OldNumFactors[i] = oldw[i];
02426             M_free(oldw, "numfactors pointers");
02427             oldw = AS.Oldvflags;
02428             AS.Oldvflags = (WORD *)Malloca1(NumExpressions*sizeof(WORD), "vflags pointers");
02429             for ( i = 0; i < AS.NumOldNumFactors; i++ ) AS.Oldvflags[i] = oldw[i];
02430             AS.NumOldNumFactors = NumExpressions;
02431             M_free(oldw, "vflags pointers");
02432             oldw = AS.Olduflags;
02433             AS.Olduflags = (WORD *)Malloca1(NumExpressions*sizeof(WORD), "uflags pointers");
02434             for ( i = 0; i < AS.NumOldNumFactors; i++ ) AS.Olduflags[i] = oldw[i];
02435             AS.NumOldNumFactors = NumExpressions;
02436             M_free(oldw, "uflags pointers");
02437         }
02438         else {
02439             AS.OldNumFactors = (WORD *)Malloca1(NumExpressions*sizeof(WORD), "numfactors pointers");
02440             AS.Oldvflags = (WORD *)Malloca1(NumExpressions*sizeof(WORD), "vflags pointers");
02441             AS.Olduflags = (WORD *)Malloca1(NumExpressions*sizeof(WORD), "uflags pointers");
02442             AS.NumOldNumFactors = NumExpressions;
02443         }
02444     }
02445     for ( i = 0; i < NumExpressions; i++ ) {
02446         AS.OldOnFile[i] = e[i].onfile;
02447         AS.OldNumFactors[i] = e[i].numfactors;
02448         AS.Oldvflags[i] = e[i].vflags;
02449         AS.Olduflags[i] = e[i].uflags;
02450     }
02451 }
02452
02453 /*
02454     #] UpdatePositions :
02455     #[ CountTerms1 :      LONG CountTerms1()
02456
02457     Counts the terms in the current deferred bracket
02458     Is mainly an adaptation of the routine Deferred in proces.c
02459 */
02460
02461 LONG CountTerms1(PHEAD0)
02462 {
02463     GETBIDENTITY
02464     POSITION oldposition, startposition;

```

```

02465     WORD *t, *m, *mstop, decr, i, *oldwork, retval;
02466     WORD *oldipointer = AR.CompressPointer;
02467     WORD oldGetOneFile = AR.GetOneFile, olddeferflag = AR.DeferFlag;
02468     LONG numterms = 0;
02469     AR.GetOneFile = 1;
02470     oldwork = AT.WorkPointer;
02471     AT.WorkPointer = (WORD *)(((UBYTE *) (AT.WorkPointer)) + AM.MaxTer);
02472     AR.DeferFlag = 0;
02473     startposition = AR.DefPosition;
02474     /*
02475         Store old position
02476     */
02477     if ( AR.infile->handle >= 0 ) {
02478         PUTZERO(oldposition);
02479     /*
02480         SeekFile(AR.infile->handle,&oldposition,SEEK_CUR);
02481     */
02482     }
02483     else {
02484         SETBASEPOSITION(oldposition,AR.infile->POfill-AR.infile->PObuffer);
02485         AR.infile->POfill = (WORD *)(((UBYTE *) (AR.infile->PObuffer)
02486             +BASEPOSITION(startposition)));
02487     }
02488     /*
02489         Look in the CompressBuffer where the bracket contents start
02490     */
02491     t = m = AR.CompressBuffer;
02492     t += *t;
02493     mstop = t - ABS(t[-1]);
02494     m++;
02495     while ( *m != HAAKJE && m < mstop ) m += m[1];
02496     if ( m >= mstop ) { /* No deferred action! */
02497         numterms = 1;
02498         AR.DeferFlag = olddeferflag;
02499         AT.WorkPointer = oldwork;
02500         AR.GetOneFile = oldGetOneFile;
02501         return(numterms);
02502     }
02503     mstop = m + m[1];
02504     decr = WORDDIF(mstop,AR.CompressBuffer)-1;
02505
02506     m = AR.CompressBuffer;
02507     t = AR.CompressPointer;
02508     i = *m;
02509     NCOPY(t,m,i);
02510     AR.TePos = 0;
02511     AN.TeSuOut = 0;
02512     /*
02513         Status:
02514         First bracket content starts at mstop.
02515         Next term starts at startposition.
02516         Decompression information is in AR.CompressPointer.
02517         The outside of the bracket runs from AR.CompressBuffer+1 to mstop.
02518     */
02519     AR.CompressPointer = oldipointer;
02520     for(;;) {
02521         numterms++;
02522         retval = GetOneTerm(BHEAD AT.WorkPointer,AR.infile,&startposition,0);
02523         if ( retval >= 0 ) AR.CompressPointer = oldipointer;
02524         if ( retval <= 0 ) break;
02525         t = AR.CompressPointer;
02526         if ( *t < (1 + decr + ABS(*(t+*t-1))) ) break;
02527         t++;
02528         m = AR.CompressBuffer+1;
02529         while ( m < mstop ) {
02530             if ( *m != *t ) goto Thatsit;
02531             m++; t++;
02532         }
02533     }
02534     Thatsit:;
02535     /*
02536         Finished. Reposition the file, restore information and return.
02537     */
02538     AT.WorkPointer = oldwork;
02539     if ( AR.infile->handle >= 0 ) {
02540     /*
02541         SeekFile(AR.infile->handle,&oldposition,SEEK_SET);
02542     */
02543     }
02544     else {
02545         AR.infile->POfill = AR.infile->PObuffer + BASEPOSITION(oldposition);
02546     }
02547     AR.DeferFlag = olddeferflag;
02548     AR.GetOneFile = oldGetOneFile;
02549     return(numterms);
02550 }
02551

```

```

02552 /*
02553     #] CountTerms1 :
02554     #[ TermsInBracket :    LONG TermsInBracket(term,level)
02555
02556     The function TermsInBracket_()
02557     Syntax:
02558         TermsInBracket_() : The current bracket in a Keep Brackets
02559         TermsInBracket_(bracket) : This bracket in the current expression
02560         TermsInBracket_(expression,bracket) : This bracket in the given expression
02561     All other specifications don't have any effect.
02562 */
02563
02564 #define CURRENTBRACKET 1
02565 #define BRACKETCURRENTEXPR 2
02566 #define BRACKETOTHEREXPR 3
02567 #define NOBRACKETACTIVE 4
02568
02569 LONG TermsInBracket(PHEAD WORD *term, WORD level)
02570 {
02571     WORD *t, *tstop, *b, *tt, *n1, *n2;
02572     int type = 0, i, num;
02573     LONG numterms = 0;
02574     WORD *bracketbuffer = AT.WorkPointer;
02575     t = term; GETSTOP(t,tstop);
02576     t++; b = bracketbuffer;
02577     while ( t < tstop ) {
02578         if ( *t != TERMSINBRACKET ) { t += t[1]; continue; }
02579         if ( t[1] == FUNHEAD || (
02580             t[1] == FUNHEAD+2
02581             && t[FUNHEAD] == -SNUMBER
02582             && t[FUNHEAD+1] == 0
02583         ) ) {
02584             if ( AC.ComDefer == 0 ) {
02585                 type = NOBRACKETACTIVE;
02586             }
02587             else {
02588                 type = CURRENTBRACKET;
02589             }
02590             *b = 0;
02591             break;
02592         }
02593         if ( t[FUNHEAD] == -EXPRESSION ) {
02594             if ( t[FUNHEAD+2] < 0 ) {
02595                 if ( ( t[FUNHEAD+2] <= -FUNCTION ) && ( t[1] == FUNHEAD+3 ) ) {
02596                     type = BRACKETOTHEREXPR;
02597                     *b++ = FUNHEAD+4; *b++ = -t[FUNHEAD+2]; *b++ = FUNHEAD;
02598                     for ( i = 2; i < FUNHEAD; i++ ) *b++ = 0;
02599                     *b++ = 1; *b++ = 1; *b++ = 3;
02600                     break;
02601                 }
02602                 else if ( ( t[FUNHEAD+2] > -FUNCTION ) && ( t[1] == FUNHEAD+4 ) ) {
02603                     type = BRACKETOTHEREXPR;
02604                     tt = t + FUNHEAD+2;
02605                     switch ( *tt ) {
02606                         case -SYMBOL:
02607                             *b++ = 8; *b++ = SYMBOL; *b++ = 4; *b++ = tt[1];
02608                             *b++ = 1; *b++ = 1; *b++ = 1; *b++ = 3;
02609                             break;
02610                         case -SNUMBER:
02611                             if ( tt[1] == 1 ) {
02612                                 *b++ = 4; *b++ = 1; *b++ = 1; *b++ = 3;
02613                             }
02614                             else goto IllBraReq;
02615                             break;
02616                         default:
02617                             goto IllBraReq;
02618                     }
02619                     break;
02620                 }
02621             }
02622             else if ( ( t[FUNHEAD+2] == (t[1]-FUNHEAD-2) ) &&
02623                 ( t[FUNHEAD+2+ARGHEAD] == (t[FUNHEAD+2]-ARGHEAD) ) ) {
02624                 type = BRACKETOTHEREXPR;
02625                 tt = t + FUNHEAD + ARGHEAD; num = *tt;
02626                 for ( i = 0; i < num; i++ ) *b++ = *tt++;
02627                 break;
02628             }
02629         }
02630         else {
02631             if ( t[FUNHEAD] < 0 ) {
02632                 if ( ( t[FUNHEAD] <= -FUNCTION ) && ( t[1] == FUNHEAD+1 ) ) {
02633                     type = BRACKETCURRENTEXPR;
02634                     *b++ = FUNHEAD+4; *b++ = -t[FUNHEAD+2]; *b++ = FUNHEAD;
02635                     for ( i = 2; i < FUNHEAD; i++ ) *b++ = 0;
02636                     *b++ = 1; *b++ = 1; *b++ = 3; *b = 0;
02637                     break;
02638                 }

```



```

02639         else if ( ( t[FUNHEAD] > -FUNCTION ) && ( t[1] == FUNHEAD+2 ) ) {
02640             type = BRACKETCURRENTEXPR;
02641             tt = t + FUNHEAD+2;
02642             switch ( *tt ) {
02643                 case -SYMBOL:
02644                     *b++ = 8; *b++ = SYMBOL; *b++ = 4; *b++ = tt[1];
02645                     *b++ = 1; *b++ = 1; *b++ = 1; *b++ = 3;
02646                     break;
02647                 case -SNUMBER:
02648                     if ( tt[1] == 1 ) {
02649                         *b++ = 4; *b++ = 1; *b++ = 1; *b++ = 3;
02650                     }
02651                     else goto IllBraReq;
02652                     break;
02653                 default:
02654                     goto IllBraReq;
02655             }
02656             break;
02657         }
02658     }
02659     else if ( ( t[FUNHEAD] == (t[1]-FUNHEAD) ) &&
02660         ( t[FUNHEAD+ARGHEAD] == (t[FUNHEAD]-ARGHEAD) ) ) {
02661         type = BRACKETCURRENTEXPR;
02662         tt = t + FUNHEAD + ARGHEAD; num = *tt;
02663         for ( i = 0; i < num; i++ ) *b++ = *tt++;
02664         break;
02665     }
02666     else {
02667     IllBraReq:
02668         MLOCK(ErrorMessageLock);
02669         MesPrint("Illegal bracket request in termsinbracket_ function.");
02670         MUNLOCK(ErrorMessageLock);
02671         Terminate(-1);
02672     }
02673 }
02674 t += t[1];
02675 }
02676 AT.WorkPointer = b;
02677 if ( AT.WorkPointer + *term + 4 > AT.WorkTop ) {
02678     MLOCK(ErrorMessageLock);
02679     MesWork();
02680     MesPrint("Called from termsinbracket_ function.");
02681     MUNLOCK(ErrorMessageLock);
02682     return(-1);
02683 }
02684 /*
02685 We are now in the position to look for the bracket
02686 */
02687 switch ( type ) {
02688     case CURRENTBRACKET:
02689     /*
02690         The code here should be rather similar to when we pick up
02691         the contents of the bracket. In our case we only count the
02692         terms though.
02693     */
02694         numterms = CountTerms1(BHEAD0);
02695         break;
02696     case BRACKETCURRENTEXPR:
02697     /*
02698         Not implemented yet.
02699     */
02700         MLOCK(ErrorMessageLock);
02701         MesPrint("termsinbracket_ function currently only handles Keep Brackets.");
02702         MUNLOCK(ErrorMessageLock);
02703         return(-1);
02704     case BRACKETTOTHEREXPR:
02705         MLOCK(ErrorMessageLock);
02706         MesPrint("termsinbracket_ function currently only handles Keep Brackets.");
02707         MUNLOCK(ErrorMessageLock);
02708         return(-1);
02709     case NOBRACKETACTIVE:
02710         numterms = 1;
02711         break;
02712 }
02713 /*
02714 Now we have the number in numterms. We replace the function by it.
02715 */
02716 n1 = term; n2 = AT.WorkPointer; tstop = n1 + *n1;
02717 while ( n1 < t ) *n2++ = *n1++;
02718 i = numterms » BITSINWORD;
02719 if ( i == 0 ) {
02720     *n2++ = LNUMBER; *n2++ = 4; *n2++ = 1; *n2++ = (WORD)(numterms & WORDMASK);
02721 }
02722 else {
02723     *n2++ = LNUMBER; *n2++ = 5; *n2++ = 2;
02724     *n2++ = (WORD)(numterms & WORDMASK); *n2++ = i;
02725 }

```

```

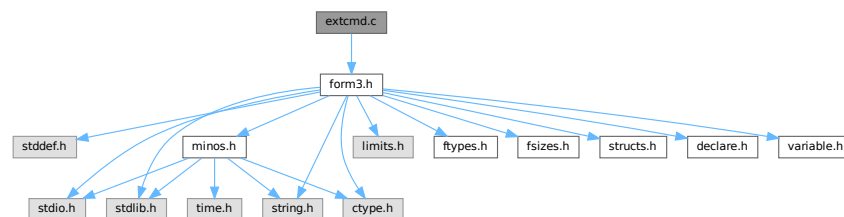
02726     nl += nl[1];
02727     while ( nl < tstop ) *n2++ = *nl++;
02728     AT.WorkPointer[0] = n2 - AT.WorkPointer;
02729     AT.WorkPointer = n2;
02730     if ( Generator(BHEAD nl,level) < 0 ) {
02731         AT.WorkPointer = bracketbuffer;
02732         MLOCK(ErrorMessageLock);
02733         MesPrint("Called from termsinbracket_ function.");
02734         MUNLOCK(ErrorMessageLock);
02735         return(-1);
02736     }
02737 /*
02738     Finished. Reset things and return.
02739 */
02740     AT.WorkPointer = bracketbuffer;
02741     return(numterms);
02742 }
02743 /*
02744     #] TermsInBracket :      LONG TermsInBracket(term,level)
02745     #] Expressions :
02746 */

```

4.31 extcmd.c File Reference

#include "form3.h"

Include dependency graph for extcmd.c:



Functions

- int [openExternalChannel](#) (UBYTE *cmd, int daemonize, UBYTE *shellname, UBYTE *stderrname)
- int [initPresetExternalChannels](#) (UBYTE *theline, int thetimeout)
- int [closeExternalChannel](#) (int n)
- int [selectExternalChannel](#) (int n)
- int [getCurrentExternalChannel](#) (void)
- void [closeAllExternalChannels](#) (void)

4.31.1 Detailed Description

The system that takes care of communication with external programs.

Definition in file [extcmd.c](#).

4.31.2 Function Documentation

4.31.2.1 openExternalChannel()

```
int openExternalChannel (
    UBYTE * cmd,
    int daemonize,
    UBYTE * shellname,
    UBYTE * stderrname )
```

Definition at line 230 of file [extcmd.c](#).

4.31.2.2 initPresetExternalChannels()

```
int initPresetExternalChannels (
    UBYTE * theline,
    int thetimeout )
```

Definition at line 232 of file [extcmd.c](#).

4.31.2.3 closeExternalChannel()

```
int closeExternalChannel (
    int n )
```

Definition at line 233 of file [extcmd.c](#).

4.31.2.4 selectExternalChannel()

```
int selectExternalChannel (
    int n )
```

Definition at line 234 of file [extcmd.c](#).

4.31.2.5 getCurrentExternalChannel()

```
int getCurrentExternalChannel (
    void )
```

Definition at line 235 of file [extcmd.c](#).

4.31.2.6 closeAllExternalChannels()

```
void closeAllExternalChannels (
    void )
```

Definition at line 236 of file [extcmd.c](#).

4.32 extcmd.c

[Go to the documentation of this file.](#)

```

00001
00005 /* #[ License : */
00006 /*
00007 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00008 *   When using this file you are requested to refer to the publication
00009 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00010 *   This is considered a matter of courtesy as the development was paid
00011 *   for by FOM the Dutch physics granting agency and we would like to
00012 *   be able to track its scientific use to convince FOM of its value
00013 *   for the community.
00014 *
00015 *   This file is part of FORM.
00016 *
00017 *   FORM is free software: you can redistribute it and/or modify it under the
00018 *   terms of the GNU General Public License as published by the Free Software
00019 *   Foundation, either version 3 of the License, or (at your option) any later
00020 *   version.
00021 *
00022 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00023 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00024 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00025 *   details.
00026 *
00027 *   You should have received a copy of the GNU General Public License along
00028 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00029 */
00030 /* #[ License : */
00031 /*
00032     #[ Documentation :
00033
00034     This module is written by M.Tentyukov as a part of implementation of
00035     interaction between FORM and external processes, first release
00036     09.04.2004. A part of this code is copied from the DIANA project, which was
00037     written by M. Tentyukov and published under the GPL version 2 as
00038     published by the Free Software Foundation.
00039
00040     This file is completely re-written by M.Tentyukov in May 2006.
00041     Since the interface was changed, the public function were changed,
00042     also. A new public functions were added: initPresetExternalChannels()
00043     (see comments just before this function in the present file) and
00044     setKillModeForExternalChannel (a pointer, not a function).
00045
00046     If a macro WITHEXTERNALCHANNEL is not defined, all public functions
00047     are stubs returning failure.
00048
00049     The idea is to start an external command swallowing
00050     its stdin and stdout. This can be done by means of the function
00051     int openExternalChannel(cmd,daemonize,shellname,stderrname), where
00052     cmd is a command to run,
00053     daemonize: if !=0 then start the command in the "daemon" mode,
00054     shellname: if !=NULL, execute the command in a subshell,
00055     stderrname: if != NULL, redirect stderr of the command to this file.
00056     The function returns some small positive integer number (the
00057     descriptor of a newly created external channel), or -1 on failure.
00058
00059     After the command is started, it becomes a _current_ opened external
00060     channel. The buffer can be sent to its stdin by a function
00061     int writeBufToExtChannel(buf, n)
00062     (here buf is a pointer to the buffer, n is the length in bytes; the
00063     function returns 0 in success, or -1 on failure),
00064     or one character can be read from its stdout by
00065     means of the function
00066     int getcFromExtChannel().
00067
00068     The latter returns the
00069     character casted to integer, or something <0. This can be -2 (if there
00070     is no current external channel) or EOF, if the external program closes
00071     its stdout, or if the external program outputs a string coinciding
00072     with a _terminator_.
00073
00074     By default, the terminator if an empty line. For the current external
00075     channel it can be set by means of the function
00076     int setTerminatorForExternalChannel(newterminator).
00077     The function returns 0 in success, or !=0 if something is wrong (no
00078     current channel, too long terminator).
00079
00080     After getcFromExtChannel() returns EOF, the current channel becomes
00081     undefined. Any further attempts to read information by
00082     getcFromExtChannel() result in -2. To set (re-set) a current channel,
00083     the function
00084     int selectExternalChannel(n)
00085     can be used. This function accepts the valid external channel

```

```

00086     descriptor (returned by openExternalChannel) and returns the
00087     descriptor of a previous current channel (0, if there was no current
00088     channel, or -1, if the external channel descriptor is invalid).
00089     If n == 0, the function undefine the current external channel.
00090
00091     The function
00092     int closeExternalChannel(n)
00093     destroys the opened external channel with the descriptor n. It returns
00094     0 in success, or -1 on failure. If the corresponding external channel
00095     was the current one, the current channel becomes undefined. If n==0,
00096     the function closes the current external channel.
00097
00098     The function
00099     int getCurrentExternalChannel(void)
00100     returns the descriptor if the current external channel, or 0 , if
00101     there is no current external channel.
00102
00103     The function
00104     void closeAllExternalChannels(void)
00105
00106     destroys all opened external channels.
00107
00108     List of all public functions:
00109     int openExternalChannel(UBYTE *cmd,int daemonize,UBYTE *shellname, UBYTE * stderrname);
00110     int initPresetExternalChannels(UBYTE *theline, int thetimeout);
00111     int setTerminatorForExternalChannel(char *newterminator);
00112     int setKillModeForExternalChannel(int signum, int sentToWholeGroup);
00113     int closeExternalChannel(int n);
00114     int selectExternalChannel(int n);
00115     int writeBufToExtChannel(char *buf,int n);
00116     int getcFromExtChannel(void);
00117     int getCurrentExternalChannel(void);
00118     void closeAllExternalChannels(void);
00119
00120     ATTENTION!
00121
00122     Four of them:
00123     1 setTerminatorForExternalChannel
00124     2 setKillModeForExternalChannel
00125     3 writeBufToExtChannel
00126     4 getcFromExtChannel
00127
00128     are NOT functions, but variables (pointers) of a corresponding type.
00129     They are initialised by proper values to avoid repeated error checking.
00130
00131     All public functions are independent of realization hidden in this module.
00132     All other functions may have a returned type/parameters type local w.r.t.
00133     this module; they are not declared outside of this file.
00134
00135     #] Documentation :
00136     #[ Selftest initializing:
00137 */
00138
00139 /*
00140 Uncomment to get a self-consistent program:
00141 #define SELFTEST 1
00142 */
00143
00144
00145 #ifndef SELFTEST
00146 #define WITHEXTERNALCHANNEL 1
00147 #ifndef _MSC_VER
00148 #define FORM_INLINE __inline
00149 #else
00150 #define FORM_INLINE inline
00151 #endif
00152
00153 /*
00154     From form3.h:
00155 */
00156 typedef unsigned char UBYTE;
00157
00158 /*The following variables should be defined in variable.h:*/
00159 extern int (*writeBufToExtChannel)(char *buffer, size_t n);
00160 extern int (*getcFromExtChannel)();
00161 extern int (*setTerminatorForExternalChannel)(char *buffer);
00162 extern int (*setKillModeForExternalChannel)(int signum, int sentToWholeGroup);
00163
00164 #else /*ifndef SELFTEST*/
00165 #include "form3.h"
00166 #endif /*ifndef SELFTEST ... else*/
00167 /*
00168 pid_t  getExternalChannelPid(void);
00169 */
00170 /*
00171     #] Selftest initializing:
00172     #[ Includes :

```

```

00173 */
00174 #ifdef WITHEXTERNALCHANNEL
00175 #include <stdio.h>
00176 #ifndef _MSC_VER
00177 #include <unistd.h>
00178 #endif
00179 #include <fcntl.h>
00180 #include <sys/types.h>
00181 #ifndef _MSC_VER
00182 #include <sys/time.h>
00183 #include <sys/wait.h>
00184 #endif
00185 #include <errno.h>
00186 #include <signal.h>
00187 #include <limits.h>
00188 /*
00189     #] Includes :
00190     #[ FailureFunctions:
00191 */
00192
00193 /*Non-initialized variant of public functions:*/
00194 int writeBufToExtChannelFailure(char *buf, size_t count)
00195 {
00196     DUMMYUSE(buf); DUMMYUSE(count);
00197     return(-1);
00198 }/*writeBufToExtChannelFailure*/
00199
00200 int setTerminatorForExternalChannelFailure(char *newTerminator)
00201 {
00202     DUMMYUSE(newTerminator);
00203     return(-1);
00204 }/*setTerminatorForExternalChannelFailure*/
00205
00206 int setKillModeForExternalChannelFailure(int signum, int sentToWholeGroup)
00207 {
00208     DUMMYUSE(signum); DUMMYUSE(sentToWholeGroup);
00209     return(-1);
00210 }/*setKillModeForExternalChannelFailure*/
00211
00212 int getcFromExtChannelFailure(void)
00213 {
00214     return(-2);
00215 }/*getcFromExtChannelFailure*/
00216
00217 int (*writeBufToExtChannel)(char *buffer, size_t n) = &writeBufToExtChannelFailure;
00218 int (*setTerminatorForExternalChannel)(char *buffer) =
00219     &setTerminatorForExternalChannelFailure;
00220 int (*setKillModeForExternalChannel)(int signum, int sentToWholeGroup) =
00221     &setKillModeForExternalChannelFailure;
00222 int (*getcFromExtChannel)(void) = &getcFromExtChannelFailure;
00223 #endif
00224 /*
00225     #] FailureFunctions:
00226     #[ Stubs :
00227 */
00228 #ifndef WITHEXTERNALCHANNEL
00229 /*Stubs for public functions:*/
00230 int openExternalChannel(UBYTE *cmd, int daemonize, UBYTE *shellname, UBYTE *stderrname)
00231 { DUMMYUSE(cmd); DUMMYUSE(daemonize); DUMMYUSE(shellname); DUMMYUSE(stderrname); return(-1); };
00232 int initPresetExternalChannels(UBYTE *theline, int thetimeout) { DUMMYUSE(theline);
    DUMMYUSE(thetimeout); return(-1); };
00233 int closeExternalChannel(int n) { DUMMYUSE(n); return(-1); };
00234 int selectExternalChannel(int n) { DUMMYUSE(n); return(-1); };
00235 int getCurrentExternalChannel(void) { return(0); };
00236 void closeAllExternalChannels(void) {};
00237 #else /*ifndef WITHEXTERNALCHANNEL*/
00238 /*
00239     #] Stubs :
00240     #[ Local types :
00241 */
00242 /*First argument for the function signal:*/
00243 #ifndef INTSIGHANDLER
00244 typedef void (*mysighandler_t)(int);
00245 #else
00246 /* Sometimes, this nonsense may occurs:*/
00247 /*typedef int (*mysighandler_t)(int);*/
00248 #endif
00249
00250 /*Input IO buffer size increment -- each time the buffer
00251 is expired it will be increased by this value (in bytes):*/
00252 #define DELTA_EXT_BUF 128
00253
00254 /*Re-allocatable array containing External Channel
00255 handlers increased each time by this value:*/
00256 #define DELTA_EXT_LIST 8
00257
00258 /*How many times I/O routines may attempt to continue their work

```

```

00259 in some failures:*/
00260 #define MAX_FAILS_IO 2
00261
00262 /*The external channel handler structure:*/
00263 typedef struct ExternalChannel {
00264     pid_t pid;          /*PID of the external process*/
00265     pid_t gpid;         /*process group ID of the external process.
00266                          If <=0, not used, if >0, the kill signals
00267                          is sent to the whole group */
00268     FILE *frec;         /*stdout of the external process*/
00269     char *INbuf;        /*External channel buffer*/
00270     char *IBfill;       /*Position in INbuf from which the next input character will be read*/
00271     char *IBfull;       /*End of read INbuf*/
00272     char *IBstop;       /*end of allocated space for INbuf*/
00273     char *terminator; /* Terminator - when extern. program outputs ONLY this string,
00274                          it is assumed that the answer is ready, and getcFromExtChannel
00275                          returns EOF. Should not be longer then the minimal buffer!*/
00276     /*Info fields, not changable after creating a channel:*/
00277     char *cmd;          /*the command*/
00278     char *shellname;
00279     char *stderrname; /*filename to redirect stderr, or NULL*/
00280     int fsend;         /*stdin of the external process*/
00281     int killSignal;    /*signal to kill*/
00282     int daemonize; /*0 --neither setsid nor daemonize, !=0 -- full daemonization*/
00283     PADPOINTER(0,3,0,0);
00284 } EXTHANDLE;
00285
00286
00287 static EXTHANDLE *externalChannelsList=0;
00288 /*Here integers are better than pointers: */
00289 static int externalChannelsListStop=0;
00290 static int externalChannelsListFill=0;
00291
00292 /*"current" external channel:*/
00293 static EXTHANDLE *externalChannelsListTop=0;
00294 /*
00295     #] Local types :
00296     #[ Selftest functions :
00297 */
00298 #ifdef SELFTEST
00299
00300 /*For malloc prototype:*/
00301 #include <stdlib.h>
00302
00303 /*StrLen, Malloc1, M_free and strDup1 are defined in tools.c -- here only emulation:*/
00304 int StrLen(char *pattern)
00305 {
00306     register char *p=(char*)pattern;
00307     while (*p)p++;
00308     return((int) ((p-(char*)pattern)) );
00309 }/*StrLen*/
00310 void *Malloc1(int l, char *c)
00311 {
00312     return(malloc(l));
00313 }
00314 void M_free(void *p,char *c)
00315 {
00316     return(free(p));
00317 }
00318
00319 char *strDup1(UBYTE *instring, char *ifwrong)
00320 {
00321     UBYTE *s = instring, *to;
00322     while ( *s ) s++;
00323     to = s = (UBYTE *)Malloc1((s-instring)+1,ifwrong);
00324     while ( *instring ) *to++ = *instring++;
00325     *to = 0;
00326     return(s);
00327 }
00328
00329 /*PutPreVar from pre.c -- just ths stub:*/
00330 int PutPreVar(UBYTE *a,UBYTE *b,UBYTE *c,int i)
00331 {
00332     return(0);
00333 }
00334
00335 #endif
00336 /*
00337     #] Selftest functions :
00338     #[ Local functions :
00339 */
00340
00341 /*Initialize one cell of handler:*/
00342 static FORM_INLINE void extHandlerInit(EXTHANDLE *h)
00343 {
00344     h->pid=-1;
00345     h->gpid=-1;

```

```

00346     h->fsend=0;
00347     h->killSignal=SIGKILL;
00348     h->daemonize=1;
00349     h->frec=NULL;
00350     h->INbuf=h->IBfill=h->IBfull=h->IBstop=
00351     h->terminator=h->cmd=h->shellname=h->stderrname=NULL;
00352 }/*extHandlerInit*/
00353
00354 /* Copies each field of handler:*/
00355 static FORM_INLINE void extHandlerSwallowCopy(EXTHANDLE *to, EXTHANDLE *from)
00356 {
00357     to->pid=from->pid;
00358     to->gpId=from->gpId;
00359     to->fsend=from->fsend;
00360     to->killSignal=from->killSignal;
00361     to->daemonize=from->daemonize;
00362     to->frec=from->frec;
00363     to->INbuf=from->INbuf;
00364     to->IBfill=from->IBfill;
00365     to->IBfull=from->IBfull;
00366     to->IBstop=from->IBstop;
00367     to->terminator=from->terminator;
00368     to->cmd=from->cmd;
00369     to->shellname=from->shellname;
00370     to->stderrname=from->stderrname;
00371 }/*extHandlerSwallow*/
00372
00373 /*Allocates memory for fields of handler which have no fixed
00374 storage size and initializes some fields:*/
00375 static FORM_INLINE void
00376 extHandlerAlloc(EXTHANDLE *h, char *cmd, char *shellname, char *stderrname)
00377 {
00378     h->IBfill=h->IBfull=h->INbuf=
00379         Malloc1(DELTA_EXT_BUF,"External channel buffer");
00380     h->IBstop=h->INbuf+DELTA_EXT_BUF;
00381     /*Initialize a terminator:*/
00382     *(h->terminator=Malloc1(DELTA_EXT_BUF,"External channel terminator"))='\n';
00383     (h->terminator)[1]='\0';/*By default the terminator is '\n'*/
00384     /*Deep copy all strings:*/
00385     if(cmd!=NULL)
00386         h->cmd=(char *)strDup1((UBYTE *)cmd,"External channel command");
00387     else/*cmd cannot be NULL! If this is NULL then force it to be something special*/
00388         h->cmd=(char *)strDup1((UBYTE *)"/","External channel command");
00389     if(shellname!=NULL)
00390         h->shellname=
00391             (char *)strDup1((UBYTE *)shellname,"External channel shell name");
00392     if(stderrname!=NULL)
00393         h->stderrname=
00394             (char *)strDup1((UBYTE *)stderrname,"External channel stderr name");
00395 }/*extHandlerAlloc*/
00396
00397 /*Deallocates dynamically allocated fields of a handler:*/
00398 static FORM_INLINE void extHandlerFree(EXTHANDLE *h)
00399 {
00400     if(h->stderrname) M_free(h->stderrname,"External channel stderr name");
00401     if(h->shellname) M_free(h->shellname,"External channel shell name");
00402     if(h->cmd) M_free(h->cmd,"External channel command");
00403     if(h->terminator)M_free(h->terminator,"External channel terminator");
00404     if(h->INbuf)M_free(h->INbuf,"External channel buffer");
00405     extHandlerInit(h);
00406 }/*extHandlerFree*/
00407 /* Closes all descriptors, kills the external process, frees all internal fields,
00408 BUT does NOT free the main container:*/
00409 static void destroyExternalChannel(EXTHANDLE *h)
00410 {
00411     /*Note, this function works in parallel mode correctly, see comments below.*/
00412
00413     /*Note, for slaves in a parallel mode h->pid == 0:*/
00414     if( (h->pid > 0) && (h->killSignal > 0) ){
00415         int chstatus;
00416         if( h->gpId > 0)
00417             chstatus=kill(-h->gpId,h->killSignal);
00418         else
00419             chstatus=kill(h->pid,h->killSignal);
00420         if(chstatus==0)
00421             /*If the process will not be killed by this signal, FORM hangs up here!*/
00422             waitpid(h->pid, &chstatus, 0);
00423     }/*if( (h->pid > 0) && (h->killSignal > 0) )*/
00424
00425     /*Note, for slaves in a parallel mode h->frec == h->fsend == 0:*/
00426     if(h->frec) fclose(h->frec);
00427     if( h->fsend > 0) close(h->fsend);
00428
00429     extHandlerFree(h);
00430     /*Does not do "free(h)"!*/
00431 }/*destroyExternalChannel*/
00432

```



```

00433 /*Wrapper to the read() syscall, to handle possible interrupts by unblocked signals:*/
00434 static FORM_INLINE ssize_t read2b(int fd, char *buf, size_t count)
00435 {
00436     ssize_t res;
00437
00438     if( (res=read(fd,buf,count)) <1 )/*EOF or read is interrupted by a signal?:*/
00439         while( (errno == EINTR)&&(res <1) )
00440             /*The call was interrupted by a signal before any data was read, try again:*/
00441             res=read(fd,buf,count);
00442     return (res);
00443 }/*read2b*/
00444
00445 /*Wrapper to the write() syscall, to handle possible interrupts by unblocked signals:*/
00446 static FORM_INLINE ssize_t writeFromb(int fd, char *buf, size_t count)
00447 {
00448     ssize_t res;
00449     if( (res=write(fd,buf,count)) <1 )/*Is write interrupted by a signal?:*/
00450         while( (errno == EINTR)&&(res <1) )
00451             /*The call was interrupted by a signal before any data was written, try again:*/
00452             res=write(fd,buf,count);
00453     return (res);
00454 }/*writeFromb*/
00455
00456 /* Read one (binary) PID from the file descriptor fd:*/
00457 static FORM_INLINE pid_t readpid(int fd)
00458 {
00459     pid_t tmp;
00460     if(read2b(fd, (char*)&tmp,sizeof(pid_t))!=sizeof(pid_t))
00461         return (pid_t)-1;
00462     return tmp;
00463 }/*readpid*/
00464
00465 /* Writeone (binary) PID to the file descriptor fd:*/
00466 static FORM_INLINE pid_t writepid(int fd, pid_t thepid)
00467 {
00468     if(writeFromb(fd, (char*)&thepid,sizeof(pid_t))!=sizeof(pid_t))
00469         return (pid_t)-1;
00470     return (pid_t)0;
00471 }/*writepid*/
00472
00473 /*Writes exactly count bytes from the buffer buf into the descriptor fd, independently on
00474 nonblocked signals and the MPU/buffer hits. Returns 0 or -1:
00475 */
00476 static FORM_INLINE int writexactly(int fd, char *buf, size_t count)
00477 {
00478     ssize_t i;
00479     int j=0,n=0;
00480
00481     for(;;){
00482         if( (i=writeFromb(fd, buf+j, count-j)) < 0 ) return(-1);
00483         j+=i;
00484         if ( ((size_t)j) == count ) break;
00485         if(i==0)n++;
00486         else n=0;
00487         if(n>MAX_FAILS_IO)return (-1);
00488     }/*for(;;)*/
00489     return (0);
00490 }/*writexactly*/
00491
00492 /* Set the FD_CLOEXEC flag of desc if value is nonzero,
00493 or clear the flag if value is 0.
00494 Return 0 on success, or -1 on error with errno set. */
00495 static int set_cloexec_flag(int desc, int value)
00496 {
00497     int oldflags = fcntl (desc, F_GETFD, 0);
00498     /* If reading the flags failed, return error indication now.*/
00499     if (oldflags < 0)
00500         return (oldflags);
00501     /* Set just the flag we want to set. */
00502     if (value != 0)
00503         oldflags |= FD_CLOEXEC;
00504     else
00505         oldflags &= ~FD_CLOEXEC;
00506     /* Store modified flag word in the descriptor. */
00507     return (fcntl(desc, F_SETFD, oldflags));
00508 }/*set_cloexec_flag*/
00509
00510 /* Adds the integer fd to the array fifo of length top+1 so that
00511 the array is ascendantly ordered. It is supposed that all 0 -- top-1
00512 elements in the array are already ordered:*/
00513 static void pushDescriptor(int *fifo, int top, int fd)
00514 {
00515     if ( top == 0 ) {
00516         fifo[top] = fd;
00517     } else {
00518         int ins=top-1;
00519         if( fifo[ins]<=fd )

```

```

00520         fifo[top]=fd;
00521     else{
00522         /*Find the position:*/
00523         while( (ins>=0)&&(fifo[ins]>fd) )ins--;
00524         /*Move all elements starting from the position to the right:*/
00525         for(ins++;top>ins; top--)
00526             fifo[top]=fifo[top-1];
00527         /*Put the element:*/
00528         fifo[ins]=fd;
00529     }
00530 }
00531 }/*pushDescriptor*/
00532
00533 /*Close all descriptors greater or equal than startFrom except those
00534 listed in the ascendantly ordered array usedFd of length top:*/
00535 static FORM_INLINE void closeAllDescriptors(int startFrom, int *usedFd, int top)
00536 {
00537     int n,maxfd;
00538     for(n=0;n<top; n++){
00539         maxfd=usedFd[n];
00540         for(;startFrom<maxfd;startFrom++)/*Close all less than maxfd*/
00541             close(startFrom);
00542         startFrom++;/*skip maxfd*/
00543     }/*for(;startFrom<maxfd;startFrom++)*/
00544     /*Close all the rest:*/
00545     maxfd=sysconf(_SC_OPEN_MAX);
00546     for(;startFrom<maxfd;startFrom++)
00547         close(startFrom);
00548 }/*closeAllDescriptors*/
00549
00550 typedef int L_APIPE[2];
00551 /*Closes both pipe descriptors if not -1:*/
00552 static void closepipe(L_APIPE *thepipe)
00553 {
00554     if( (*thepipe)[0] != -1) close ((*thepipe)[0]);
00555     if( (*thepipe)[1] != -1) close ((*thepipe)[1]);
00556 }/*closepipe*/
00557
00558 /*Parses the cmd line like "sh -c myprg", passes each option to the
00559 corresponding element of argv, ends agrv by NULL. Returns the
00560 number of stored argv elements, or -1 if fails:*/
00561 static FORM_INLINE int parseline(char **argv, char *cmd)
00562 {
00563     int n=0;
00564     while(*cmd != '\0'){
00565         for( ; (*cmd <= ' ') && (*cmd != '\0') ;cmd++);
00566         if(*cmd != '\0'){
00567             argv[n]=cmd;
00568             while(++cmd > ' ');
00569             if(*cmd != '\0')
00570                 *cmd++ = '\0';
00571             n++;
00572         }/*if(*cmd != '\0')*/
00573     }/*while(*cmd != '\0')*/
00574     argv[n]=NULL;
00575     if(n==0)return -1;
00576     return n;
00577 }/*parseline*/
00578
00579 /*Reads positive decimal number (not bigger than maxnum)
00580 from the string and returns it;
00581 the pointer *b is set to the next non-converted character:*/
00582 static LONG str2i(char *str, char **b, LONG maxnum)
00583 {
00584     LONG n=0;
00585     /*Eat trailing spaces:*/
00586     while(*str<=' ')if(*str++ == '\0')return(-1);
00587     (*b)=str;
00588     while (*str>='0'&&*str<='9')
00589         if( (n=10*n + *str++ - '0')>maxnum )
00590             return(-1);
00591     if((*b)==str)/*No single number!*/
00592         return(-1);
00593     (*b)=str;
00594     return(n);
00595 }
00596
00597 /*Converts long integer to a decimal representation.
00598 For portability reasons we cannot use LongCopy from tools.c
00599 since theoretically LONG may be smaller than pid_t:*/
00600 static char *l2s(LONG x, char *to)
00601 {
00602     char *s;
00603     int i = 0, j;
00604     s = to;
00605     do { *s++ = (x % 10)+'0'; i++; } while ( ( x /= 10 ) != 0 );
00606     *s-- = '\0';

```

```

00607     j = ( i - 1 ) >> 1;
00608     while ( j >= 0 ) {
00609         i = to[j]; to[j] = s[-j]; s[-j] = (char)i; j--;
00610     }
00611     return(s+1);
00612 }
00613
00614 /*like strcat() but returns the pointer to the end of the
00615 resulting string:*/
00616 static FORM_INLINE char *addStr(char *to, char *from)
00617 {
00618     while( ( *to++ = *from++ ) != '\0' );
00619     return(to-1);
00620 }/*addStr*/
00621
00622
00623 /*Try to write (atomically) short buffer (of length count) to fd.
00624 timeout is a timeout in millisecs. Returns number of written bytes or -1:*/
00625 static FORM_INLINE ssize_t writeSome(int fd, char *buf, size_t count, int timeout)
00626 {
00627     ssize_t res = 0;
00628     fd_set wfds;
00629     struct timeval tv;
00630     int nrep=5; /*five attempts it interrupted by a non-blocking signal*/
00631     int flags = fcntl(fd, F_GETFL, 0);
00632
00633     /*Add O_NONBLOCK:*/
00634     fcntl(fd, F_SETFL, flags | O_NONBLOCK);
00635     /* important -- in order to avoid blocking of short receiver buffer*/
00636
00637     do{
00638         FD_ZERO(&wfds);
00639         FD_SET(fd, &wfds);
00640         /* Wait up to timeout. */
00641
00642         tv.tv_sec = timeout / 1000;
00643         tv.tv_usec = (timeout % 1000) * 1000;
00644         nrep--;
00645
00646         switch(select(fd+1, NULL, &wfds, NULL, &tv)){
00647             case -1:
00648                 if((nrep == 0) || (errno != EINTR) ){
00649                     perror("select()");
00650                     res=-1;
00651                     nrep=0;
00652                 }/*else -- A non blocked signal was caught, just repeat*/
00653                 break;
00654             case 0: /*timeout*/
00655                 res=-1;
00656                 nrep=0;
00657                 break;
00658             default:
00659                 if( (res=write(fd, buf, count)) < 0 ) /*Signal?*/
00660                     while( (errno == EINTR) && (res < 0) )
00661                         res=write(fd, buf, count);
00662                 nrep=0;
00663             }/*switch*/
00664         }while(nrep);
00665
00666         /*restore the flags:*/
00667         fcntl(fd, F_SETFL, flags);
00668         return (res);
00669     }/*writeSome*/
00670
00671 /*Try to read short buffer (of length not more than count)
00672 from fd. timeout is a timeout in millisecs. Returns number
00673 of written bytes or -1: */
00674 static FORM_INLINE ssize_t readSome(int fd, char *buf, size_t count, int timeout)
00675 {
00676     ssize_t res = 0;
00677     fd_set rfds;
00678     struct timeval tv;
00679     int nrep=5; /*five attempts it interrupted by a non-blocking signal*/
00680
00681     do{
00682         FD_ZERO(&rfds);
00683         FD_SET(fd, &rfds);
00684         /* Wait up to timeout. */
00685
00686         tv.tv_sec = timeout / 1000;
00687         tv.tv_usec = (timeout % 1000) * 1000;
00688         nrep--;
00689
00690         switch(select(fd+1, &rfds, NULL, NULL, &tv)){
00691             case -1:
00692                 if((nrep == 0) || (errno != EINTR) ){
00693                     perror("select()");

```

```

00694         res=-1;
00695         nrep=0;
00696     }/*else -- A non blocked signal was caught, just repeat*/
00697     break;
00698     case 0:/*timeout*/
00699         res=-1;
00700         nrep=0;
00701         break;
00702     default:
00703         if( (res=read(fd,buf,count)) <0 )/*Signal?*/
00704             while( (errno == EINTR)&&(res <0) )
00705                 res=read(fd,buf,count);
00706         nrep=0;
00707     }/*switch*/
00708     }while(nrep);
00709     return (res);
00710 }/*readSome*/
00711
00712 /*
00713     #] Local functions :
00714     #[ Ok functions:
00715 */
00716
00717 /*Copies (deep copy) newTerminator to thehandler->terminator. Returns 0 if
00718 newTerminator fits to the buffer, or !0 if it does not fit. ATT! In the
00719 latter case thehandler->terminator is NOT '\0' terminated! */
00720 int setTerminatorForExternalChannelOk(char *newTerminator)
00721 {
00722     int i=DELTA_EXT_BUF;
00723     /*
00724         No problems with externalChannelsListTop are
00725         possible since this function may be invoked only
00726         when the current channel is defined and externalChannelsListTop
00727         is set properly
00728     */
00729     char *t=externalChannelsListTop->terminator;
00730
00731     for( ; i>1; i--)
00732         if( (*t++ = *newTerminator++)=='\0' )
00733             break;
00734     /*Add trailing '\n', if absent:*/
00735     if( (i == DELTA_EXT_BUF)/*newTerminator == '\0'*/
00736         || (*t-2)!='\n' )
00737     {
00738         *(t-1)='\n';*t='\0';
00739     }
00739     return(i==1);
00740 }/*setTerminatorForExternalChannelOk*/
00741
00742 /*Interface to change handler fields "killSignal" and "gpid"*/
00743 int setKillModeForExternalChannelOk(int signum, int sentToWholeGroup)
00744 {
00745     if(signum<0)
00746         return(-1);
00747     /*
00748         No problems with externalChannelsListTop are
00749         possible since this function may be invoked only
00750         when the current channel is defined and externalChannelsListTop
00751         is set properly
00752     */
00753     externalChannelsListTop->killSignal=signum;
00754     if(sentToWholeGroup){/*gpid must be >0*/
00755         if(externalChannelsListTop->gpid <= 0)
00756             externalChannelsListTop->gpid=-externalChannelsListTop->gpid;
00757     }else{/*gpid must be <=0*/
00758         if(externalChannelsListTop->gpid>0)
00759             externalChannelsListTop->gpid=-externalChannelsListTop->gpid;
00760     }
00761     return(0);
00762 }/*setKillModeForExternalChannelOk*/
00763
00764 /*
00765     #[ getcFromExtChannelOk
00766 */
00767 /*Returns one character from the external channel. If the input is expired,
00768 returns EOF. If the external process is finished completely, the function closes
00769 the channel (and returns EOF). If the external process was finished, the function
00770 returns EOF:*/
00771 int getcFromExtChannelOk(void)
00772 {
00773     mysighandler_t oldPIPE = 0;
00774     EXTHANDLE *h;
00775     int ret;
00776
00777     if (externalChannelsListTop->IBfill < externalChannelsListTop->IBfull)
00778         /*in buffer*/
00779         return( *(externalChannelsListTop->IBfill++) );
00780     /*else -- the buffer is empty*/

```

```

00781     ret=EOF;
00782     h= externalChannelsListTop;
00783 #ifdef WITHMPI
00784     if ( PF.me == MASTER ){
00785 #endif
00786         /* Temporary ignore this signal:*/
00787         /* if compiler fails here, try to change the definition of
00788            mysighandler_t on the beginning of this file
00789            (just define INTSIGHANDLER).*/
00790         oldPIPE=signal(SIGPIPE,SIG_IGN);
00791 #ifdef WITHMPI
00792         if( fgets(h->INbuf,h->IBstop - h->INbuf, h->frec) == 0 )/*Fail! EOF?*/
00793             *(h->INbuf)='\0';/*Empty line may not appear here!*/
00794 #else
00795         if( (fgets(h->INbuf,h->IBstop - h->INbuf, h->frec) == 0)/*Fail! EOF?*/
00796             || ( *(h->INbuf) == '\0')/*Empty line? This shouldn't be!*/
00797         ){
00798             closeExternalChannel(externalChannelsListTop-externalChannelsList+1);
00799             /*Note, this code is only for the sequential mode! */
00800             goto getcFromExtChannelReady;
00801             /*Here we assume that fgets is never interrupted by singals*/
00802         }/*if( fgets(h->INbuf,h->IBstop - h->INbuf, h->frec) == 0 )*/
00803 #endif
00804 #ifdef WITHMPI
00805         }/*if ( PF.me == MASTER */
00806
00807         /*Master broadcasts result to slaves, slaves read it from the master:*/
00808         if( PF_BroadcastString((UBYTE *)h->INbuf) )/*Fail!*/
00809             MesPrint("Fail broadcasting external channel results");
00810             Terminate(-1);
00811         }/*if( PF_BroadcastString((UBYTE *)h->INbuf) )*/
00812
00813         if( *(h->INbuf) == '\0')/*Empty line? This shouldn't be!*/
00814             closeExternalChannel(externalChannelsListTop-externalChannelsList+1);
00815             goto getcFromExtChannelReady;
00816         }/*if( *(h->INbuf) == '\0')*/
00817 #endif
00818
00819         {/*Block*/
00820             char *t=h->terminator;
00821             /*Move IBfull to the end of read line and compare the line with the terminator.
00822             Note, by construction the terminator fits to the first read line, see
00823             the function setTerminatorForExternalChannel.*/
00824             for(h->IBfull=h->INbuf; *(h->IBfull)!='\0'; (h->IBfull)++)
00825                 if( *t== *(h->IBfull) )
00826                     t++;
00827             else
00828                 break;/*not a terminator*/
00829             /*Continue moving IBfull to the end of read line:*/
00830             while(*(h->IBfull)!='\0') (h->IBfull)++;
00831
00832             if( (t-h->terminator) == (h->IBfull-h->INbuf) ){
00833                 /*Terminator!*/
00834                 /*Reset the channel*/
00835                 h->IBfull=h->IBfill=h->INbuf;
00836                 externalChannelsListTop=0;/*Undefine the current channel*/
00837                 writeBufToExtChannel=&writeBufToExtChannelFailure;
00838                 getcFromExtChannel=&getcFromExtChannelFailure;
00839                 setTerminatorForExternalChannel=&setTerminatorForExternalChannelFailure;
00840                 setKillModeForExternalChannel=&setKillModeForExternalChannelFailure;
00841                 goto getcFromExtChannelReady;
00842             }/*if(t == (h->IBfull-h->INbuf) )*/
00843         }/*Block*/
00844         /*Does the buffer have enough capacity?*/
00845         while( *(h->IBfull - 1) != '\n' )/*Buffer is not enough!*/
00846             /*Extend the buffer:*/
00847             int l= (h->IBstop - h->INbuf)+DELTA_EXT_BUF;
00848             char *newbuf=Malloc1(l,"External channel buffer");
00849             /*We wouldn't like to use realloc.*/
00850             /*Copy the buffer:*/
00851             char *n=newbuf,*o=h->INbuf;
00852             while( (*n++ = *o++)!='\0' );
00853             /*Att! The order of the following operators is important!*/
00854             h->IBfull= newbuf+(h->IBfull-h->INbuf);
00855             M_free(h->INbuf,"External channel buffer");
00856             h->INbuf = newbuf;
00857             h->IBstop = h->INbuf+l;
00858 #ifdef WITHMPI
00859             if ( PF.me == MASTER ){
00860                 (h->IBfull)[1]='\0';/*Will mark (h->IBfull)[1] as '!' for failure*/
00861                 if( fgets(h->IBfull,h->IBstop - h->IBfull, h->frec) == 0 ){
00862                     /*EOF! No trailing '\n'?*/
00863                     /*Mark:*/
00864                     (h->IBfull)[0]='\0';
00865                     (h->IBfull)[1]='!';
00866                     (h->IBfull)[2]='\0';
00867                     /*The string "\0!\0" is used as an image of NULL.*/

```

```

00868         }/*if( fgets(h->IBfull,h->IBstop - h->IBfull, h->frec) == 0 )*/
00869     }/*if ( PF.me == MASTER )*/
00870
00871     /*Master broadcasts results to slaves, slaves read it from the master:*/
00872     if( PF_BroadcastString((UBYTE *)h->IBfull) ){/*Fail!*/
00873         MesPrint("Fail broadcasting external channel results");
00874         Terminate(-1);
00875     }/*if( PF_BroadcastString(h->IBfull) )*/
00876
00877     /*The string "\0!\0" is used as the image of NULL.*/
00878     if(
00879         ( (h->IBfull)[0]=='\0' )
00880         && ( (h->IBfull)[1]=='!' )
00881         && ( (h->IBfull)[2]=='\0' )
00882     ){/*EOF! No trailing '\n'?*/
00883         break;
00884     }else
00885         if( fgets(h->IBfull,h->IBstop - h->IBfull, h->frec) == 0 )
00886             /*EOF! No trailing '\n'?*/
00887             break;
00888     }endif
00889     while( *(h->IBfull)!='\0' ) (h->IBfull)++;
00890 }/*while( *(h->IBfull - 1) != '\n' )*/
00891 /*In h->INbuf we have a fresh string.*/
00892 ret+=(h->IBfill=h->INbuf);
00893 h->IBfill++;/*Next time a new, isn't it?*/
00894
00895 getcFromExtChannelReady:
00896 #ifdef WITHMPI
00897     if ( PF.me == MASTER ){
00898 #endif
00899         signal(SIGPIPE,oldPIPE);
00900 #ifdef WITHMPI
00901     }/*if ( PF.me == MASTER )*/
00902 #endif
00903     return(ret);
00904 }/*getcFromExtChannelOk*/
00905 /*
00906     #] getcFromExtChannelOk
00907 */
00908 /*Writes exactly count bytes from the buffer buf to the external channel thehandler
00909 Returns 0 (on success) or -1:
00910 */
00911 int writeBufToExtChannelOk(char *buf, size_t count)
00912 {
00913     int ret;
00914     mysighandler_t oldPIPE;
00915 #ifdef WITHMPI
00916     /*Only master communicates with the external program:*/
00917     if ( PF.me == MASTER ){
00918 #endif
00919         /* Temporary ignore this signal:*/
00920         /* if compiler fails here, try to change the definition of
00921         mysighandler_t on the beginning of this file
00922         (just define INTSIGHANDLER)*/
00923         oldPIPE=signal(SIGPIPE,SIG_IGN);
00924         ret=writeexactly( externalChannelsListTop->fsend, buf, count);
00925         signal(SIGPIPE,oldPIPE);
00926 #ifdef WITHMPI
00927     }else{
00928         /*Do not wait the master status: this would be too slow!*/
00929         ret=0;
00930     }
00931 #endif
00932     return(ret);
00933 }/*writeBufToExtChannel*/
00934 /*
00935     #] Ok functions:
00936     #[ do_run_cmd :
00937 */
00938 /*The function returns PID of the started command*/
00939 static FORM_INLINE pid_t do_run_cmd(
00940     int *fdsend,
00941     int *fdreceive,
00942     int *gpId, /*returns group process ID*/
00943     int ttyMode,
00944     /*
00945         &8 - daemonizeing
00946         &16 - setsid()*/
00947     char *cmd,
00948     char *argv[],
00949     char *stderrname
00950 )
00951 {
00952     int fdin[2]={-1,-1}, fdout[2]={-1,-1}, fdsig[2]={-1,-1};
00953     /*initialised by -1 for possible rollback at failure, see closepipe() above*/

```

```

00955
00956 pid_t childpid, fatherchildpid = (pid_t)0;
00957 mysighandler_t oldPIPE=NULL;
00958
00959     if(
00960         (pipe(fdsig)!=0)/*This pipe will be used by a child to tell the father if fail.*/
00961         ||(pipe(fdin)!=0)
00962         ||(pipe(fdout)!=0)
00963     )goto fail_do_run_cmd;
00964
00965     if((childpid = fork()) == -1){
00966         perror("fork");
00967         goto fail_do_run_cmd;
00968     }/*if((childpid = fork()) == -1)*/
00969
00970     if(childpid == 0){/*Child.*/
00971         int fifo[3], top=0;
00972         /*
00973          * To be thread safely we can't rely on ascendant order of opened
00974          * file descriptors. So we put each of descriptor we have to
00975          * preserve into the array fifo.
00976          * Note, in _this_ process there are no any threads but descriptors
00977          * were created in frame of the parent process which may have
00978          * multiple threads.
00979          */
00980         /*Mark descriptors which will NOT be closed:*/
00981         pushDescriptor(fifo,top++,fdsig[1]);
00982         pushDescriptor(fifo,top++,fdin[0]);
00983         pushDescriptor(fifo,top++,fdout[1]);
00984         /*Close all except stdin, stdout, stderr and placed into fifo:*/
00985         closeAllDescriptors(3,fifo, top);
00986
00987         /*Now reopen stdin and stdout.*/
00988         /*thread-safety is not a problem here since there are no any threads up to now:*/
00989         if(
00990             (close(0) == -1 )||/* Use fdin as stdin :*/
00991             (dup(fdin[0]) == -1 )||
00992             (close(1)==-1)||/* Use fdout as stdout:*/
00993             (dup(fdout[1]) == -1 )
00994         )
00995         {/*Fail!*/
00996             /*Signal to parent:*/
00997             writepid(fdsig[1],(pid_t)-2);
00998             _exit(1);
00999         }
01000
01001         if(stderrname != NULL){
01002             if(
01003                 (close(2) != 0 )||
01004                 (open(stderrname,O_WRONLY)<0)
01005             )
01006             {/*Fail!*/
01007                 writepid(fdsig[1],(pid_t)-2);
01008                 _exit(1);
01009             }
01010         }/*if(stderrname != NULL)*/
01011
01012         if( ttymode & 16 )/* create a session and sets the process group ID */
01013             setsid();
01014
01015         /* */
01016         if(set_cloexec_flag (fdsig[1], 1)!=0){/*Error?*/
01017             /*Signal to parent:*/
01018             writepid(fdsig[1],(pid_t)-2);
01019             _exit(1);
01020         }/*if(set_cloexec_flag (fdsig[1], 1)!=0)*/
01021
01022         if( ttymode & 8 )/*Daemonize*/
01023             int fdsig2[2];/*To check exec() success*/
01024             if(
01025                 pipe(fdsig2)||
01026                 (set_cloexec_flag (fdsig2[1], 1)!=0)
01027             )
01028             {/*Error?*/
01029                 /*Signal to parent:*/
01030                 writepid(fdsig[1],(pid_t)-2);
01031                 _exit(1);
01032             }
01033             set_cloexec_flag (fdsig2[0], 1);
01034             switch(childpid=fork()){
01035                 case 0:/*grandchild*/
01036                     /*Execute external command:*/
01037                     execvp(cmd, argv);
01038                     /* Control can reach this point only on error!*/
01039                     writepid(fdsig2[1],(pid_t)-2);
01040                     break;
01041                 case -1:

```

```

01042         /* Control can reach this point only on error!*/
01043         /*Inform the father about the failure*/
01044         writepid(fdsig[1],(pid_t)-2);
01045         _exit(1);/*The child, just exit, not return*/
01046     default:/*Son of his father*/
01047         close(fdsig2[1]);
01048         /*Ignore SIGPIPE (up to the end of the process):*/
01049         signal(SIGPIPE,SIG_IGN);
01050
01051         /*Wait on read() while the grandchild close the pipe
01052         (on success) or send -2 (if exec() fails).*/
01053         /*There are two possibilities:
01054         -1 -- this is ok, the pipe was closed on exec,
01055         the program was successfully executed;
01056         -2 -- something is wrong, exec failed since the
01057         grandchild sends -2 after exec.
01058         */
01059         if( readpid(fdsig2[0]) != (pid_t)-1 )/*something is wrong*/
01060             writepid(fdsig[1],(pid_t)-1);
01061         else/*ok, send PID of the grandchild to the father:*/
01062             writepid(fdsig[1],childpid);
01063         /*Die and free the life space for the grandchild:*/
01064         _exit(0);/*The child, just exit, not return*/
01065     }/*switch(childpid=fork())*/
01066 }else{/*if( ttymode & 8 )*/
01067     execvp(cmd, argv);
01068     /* Control can reach this point only on error!*/
01069     writepid(fdsig[1],(pid_t)-2);
01070     _exit(2);/*The child, just exit, not return*/
01071 }/*if( ttymode & 8 )...else*/
01072 }else{/* The (grand)father*/
01073     close(fdsig[1]);
01074     /*To prevent closing fdsig in rollback:*/
01075     fdsig[1]=-1;
01076     close(fdin[0]);
01077     close(fdout[1]);
01078     *fdsend = fdin[1];
01079     *fdreceive = fdout[0];
01080
01081     /*Get the process group ID.*/
01082     /*Avoid to use getpgid() which is non-standard.*/
01083     if( ttymode & 16)/*setsid() was invoked, the child is a group leader:*/
01084         *gpid=childpid;
01085     else/*the child belongs to the same process group as the this process:*/
01086         *gpid=getpgrp();/*if compiler fails here, try getpgrp(0) instead!*/
01087     /*
01088     Rationale: getpgrp conform to POSIX.1 while 4.3BSD provides a
01089     getpgrp() function that returns the process group ID for a
01090     specified process.
01091     */
01092
01093     /* Temporary ignore this signal:*/
01094     /* if compiler fails here, try to change the definition of
01095     mysighandler_t on the beginning of this file
01096     (just define INTSIGHANDLER)*/
01097     oldPIPE=signal(SIGPIPE,SIG_IGN);
01098
01099     if( ttymode & 8 ){/*Daemonize*/
01100         /*Read the grandchild PID from the son.*/
01101         fatherchildpid=childpid;
01102         if( (childpid=readpid(fdsig[0]))<0 ){
01103             /*Daemonization process fails for some reasons!*/
01104             childpid=fatherchildpid;/*for rollback*/
01105             goto fail_do_run_cmd;
01106         }
01107     }else{
01108         /*fdsig[1] should be closed on exec and this read operation
01109         must fail on success:*/
01110         if( readpid(fdsig[0])!= (pid_t)-1 )
01111             goto fail_do_run_cmd;
01112     }/*if( ttymode & 8 ) ... else*/
01113 }/*if(childpid == 0)...else*/
01114
01115 /*Here can be ONLY the father*/
01116 close(fdsig[0]);
01117 /*To prevent closing fdsig in rollback after goto fail_flnk_do_runcmd:*/
01118 fdsig[0]=-1;
01119
01120 if( ttymode & 8 ){/*Daemonize*/
01121     int i;
01122     /*Wait while the father of a grandchild dies:*/
01123     waitpid(fatherchildpid,&i,0);
01124 }
01125
01126 /*Restore the signal:*/
01127 signal(SIGPIPE,oldPIPE);
01128

```



```

01129     return(childpid);
01130
01131     fail_do_run_cmd:
01132         closepipe(&fdout);
01133         closepipe(&fdin);
01134         closepipe(&fdsig);
01135         return((pid_t)-1);
01136 }/*do_run_cmd*/
01137 /*
01138  #] do_run_cmd :
01139  #[ run_cmd :
01140 */
01141 /*Starts the command cmd (directly, if shellpath is NULL, or in a subshell),
01142  swallowing its stdin and stdout;
01143  stderr will be re-directed to stderrname (if !=NULL). Returns PID of
01144  the started process. Stdin will be available as fdsend, and stdout
01145  will be available as fdreceive:*/
01146 static FORM_INLINE pid_t run_cmd(char *cmd,
01147                                  int *fdsend,
01148                                  int *fdreceive,
01149                                  int *gpipid,
01150                                  int daemonize,
01151                                  char *shellpath,
01152                                  char *stderrname )
01153 {
01154     char **argv;
01155     pid_t thepid;
01156
01157     cmd=(char*)strDupl((UBYTE*)cmd, "run_cmd: cmd");/*detouch cmd*/
01158
01159
01160     /* Prepare arguments for execvp:*/
01161     if(shellpath != NULL){/*Run in a subshell.*/
01162         int nopt;
01163         /*Allocate space which is definitely enough:*/
01164         argv=Malloc1(StrLen((UBYTE*)shellpath)*sizeof(char)+2,"run_cmd:argv");
01165         shellpath=(char*)strDupl((UBYTE*)shellpath, "run_cmd: shellpath");/*detouch shellpath*/
01166         /*Parse a shell (e.g., "/bin/sh -c"):*/
01167         nopt=parseline(argv, shellpath);
01168         /* and add the command as a shell argument:*/
01169         argv[nopt]=cmd;
01170         argv[nopt+1]=NULL;
01171     }else{/*Run the command directly:*/
01172         /*Allocate space which is definitely enough:*/
01173         argv=Malloc1(StrLen((UBYTE*)cmd)*sizeof(char)+1,"run_cmd:argv");
01174         parseline(argv, cmd);
01175     }
01176
01177     thepid=do_run_cmd(
01178         fdsend,
01179         fdreceive,
01180         gpipid,
01181         (daemonize)?(8|16):0,
01182         argv[0],
01183         argv,
01184         stderrname
01185     );
01186
01187     M_free(argv,"run_cmd:argv");
01188     if(shellpath)
01189         M_free(shellpath,"run_cmd:argv");
01190     M_free(cmd, "run_cmd: cmd");
01191
01192     return(thepid);
01193 }/*run_cmd*/
01194 /*
01195  #] run_cmd :
01196  #[ createExternalChannel :
01197 */
01198 /*The structure to pass parameters to createExternalChannel
01199  and openExternalChannel in case of preset channel (instead of
01200  shellname):*/
01201 typedef struct{
01202     int fdin;
01203     int fdout;
01204     pid_t theppid;
01205 }ECINFOSTRUCT;
01206
01207 /* Creates a new external channel starting the command cmd (if cmd !=NULL)
01208  or using information from (ECINFOSTRUCT *)shellname, if cmd ==NULL:*/
01209 static FORM_INLINE void *createExternalChannel(
01210     EXTHANDLE *h,
01211     char *cmd, /*Command to run or NULL*/
01212     /* --neither setsid nor daemonize, !=0 -- full daemonization:*/
01213     int daemonize,
01214     char *shellname,/* The shell (like "/bin/sh -c") or NULL*/
01215     char *stderrname/*filename to redirect stderr or NULL*/

```

```

01216                                     )
01217 {
01218     int fdreceive=0;
01219     int gpid = 0;
01220     ECINFOSTRUCT *psetInfo;
01221 #ifdef WITHMPI
01222     char statusbuf[2]={'\0','\0'}; /*'\0' if run_cmd returns ok, '!' otherwise.*/
01223 #endif
01224     extHandlerInit(h);
01225
01226     h->pid=0;
01227
01228     if( cmd==NULL ){ /*Instead of starting a new command, use preset channel:*/
01229         psetInfo=(ECINFOSTRUCT *)shellname;
01230         shellname=NULL;
01231         h->killSignal=0;
01232         h->daemonize=0;
01233     }
01234
01235     /*Create a channel:*/
01236 #ifdef WITHMPI
01237     if ( PF.me == MASTER ){
01238 #endif
01239         if(cmd!=NULL)
01240             h->pid=run_cmd (cmd, &(h->fsend),
01241                             &fdreceive,&gpid,daemonize,shellname,stderrname);
01242         else{
01243             gpid=-psetInfo->theppid;
01244             h->pid=psetInfo->theppid;
01245             h->fsend=psetInfo->fdout;
01246             fdreceive=psetInfo->fdin;
01247         }
01248 #ifdef WITHMPI
01249         if(h->pid<0)
01250             statusbuf[0]='!'; /*Broadcast fail to slaves*/
01251     }
01252     /*else: Keep h->pid = 0 and h->fsend = 0 for slaves in parallel mode!*/
01253
01254     /*Master broadcasts status to slaves, slaves read it from the master:*/
01255     if( PF_BroadcastString((UBYTE *)statusbuf) ){ /*Fail!*/
01256         h->pid=-1;
01257     }else if( statusbuf[0]!='!') /*Master fails*/
01258         h->pid=-1;
01259 #endif
01260
01261     if(h->pid<0)goto createExternalChannelFails;
01262 #ifdef WITHMPI
01263     if ( PF.me == MASTER ){
01264 #endif
01265         h->gpid=gpid;
01266         /*Open stdout of a newly created program as FILE* :*/
01267         if( (h->frec=fdopen(fdreceive,"r")) == 0 )goto createExternalChannelFails;
01268 #ifdef WITHMPI
01269     }
01270 #endif
01271 #endif
01272     /*Initialize buffers:*/
01273     extHandlerAlloc(h,cmd,shellname,stderrname);
01274     return(h);
01275     /*Something is wrong?*/
01276     createExternalChannelFails:
01277
01278     destroyExternalChannel(h);
01279     return(NULL);
01280 } /*createExternalChannel*/
01281
01282 /*
01283 #] createExternalChannel :
01284 #[ openExternalChannel :
01285 */
01286 int openExternalChannel(UBYTE *cmd, int daemonize, UBYTE *shellname, UBYTE *stderrname)
01287 {
01288     EXTHANDLE *h=externalChannelsListTop;
01289     int i=0;
01290
01291     for(externalChannelsListTop=0;i<externalChannelsListFill;i++)
01292         if(externalChannelsList[i].cmd==0){
01293             externalChannelsListTop=externalChannelsList+i;
01294             break;
01295         } /*if(externalChannelsList[i].cmd!=0)*/
01296
01297     if(externalChannelsListTop==0){
01298         /*No free cells, create the new one:*/
01299         if(externalChannelsListFill == externalChannelsListStop){
01300             /*The list is full, increase and reallocate it:*/
01301             EXTHANDLE *newbuf=Malloc1(
01302                 (externalChannelsListStop+=DELTA_EXT_LIST)*sizeof(EXTHANDLE),

```

```

01303                                     "External channel list");
01304         for(i=0;i<externalChannelsListFill;i++)
01305             extHandlerSwallowCopy(newbuf+i,externalChannelsList+i);
01306         if(externalChannelsList!=0)
01307             M_free(externalChannelsList,"External channel list");
01308         for(;i<externalChannelsListStop;i++)
01309             extHandlerInit(newbuf+i);
01310         externalChannelsList=newbuf;
01311     }/*if(externalChannelsListFill == externalChannelsListStop)*/
01312     externalChannelsListTop=externalChannelsList+externalChannelsListFill;
01313     externalChannelsListFill++;
01314 }/*if(externalChannelsListTop==0)*/
01315
01316     if(createExternalChannel(
01317                                     externalChannelsListTop,
01318                                     (char*) cmd,
01319                                     daemonize,
01320                                     (char*)shellname,
01321                                     (char*)stderrname
01322                                     )!=NULL){
01323         writeBufToExtChannel=&writeBufToExtChannelOk;
01324         getcFromExtChannel=&getcFromExtChannelOk;
01325         setTerminatorForExternalChannel=&setTerminatorForExternalChannelOk;
01326         setKillModeForExternalChannel=&setKillModeForExternalChannelOk;
01327         return(externalChannelsListTop-externalChannelsList+1);
01328     }
01329     /*else - failure!*/
01330     externalChannelsListTop=h;
01331     return(-1);
01332 }/*openExternalChannel*/
01333
01334 #] openExternalChannel :
01335 #[ initPresetExternalChannels :
01336 */
01337 /*Just simple wrapper to invoke openExternalChannel()
01338 from initPresetExternalChannels()*/
01339 static FORM_INLINE int openPresetExternalChannel(int fdin, int fdout, pid_t theppid)
01340 {
01341     ECINFOSTRUCT inf;
01342     inf.fdin=fdin; inf.fdout=fdout;inf.theppid=theppid;
01343     return( openExternalChannel(NULL,0,(UBYTE *)&inf,NULL) );
01344 }/*openPresetExternalChannel*/
01345
01346 #define PIDTXTSIZE 23
01347 #define BOTHPIDSIZE 45
01348 #ifndef LONG_MAX
01349 #define LONG_MAX 0x7FFFFFFFL
01350 #endif
01351
01352 /*There is a possibility to start FORM from another program providing
01353 one (or more) communication channels. These channels will be
01354 visible from a FORM program as ``pre-opened'' external channels existing
01355 after FORM starts.
01356 Before starting FORM, the parent application must create one or more
01357 pairs of pipes The read-only descriptor of the first pipe in the pair
01358 and the write-only descriptor of the second pipe must be passed to
01359 FORM as an argument of a command line option -pipe in ASCII decimal
01360 format. The argument of the option is a comma-separated list of pairs
01361 r#w# where r# is a read-only descriptor and w# is a write-only
01362 descriptor. Alternatively, the environment variable FORM_PIPES can be
01363 used.
01364 The following function expects as the first argument
01365 this comma-separated list of the descriptor pairs and tries to
01366 initialize each of channel during thetimeout milliseconds*/
01367
01368 int initPresetExternalChannels(UBYTE *theline, int thetimeout)
01369 {
01370     int i, nchannels = 0;
01371     int pidln; /*The length of FORM PID in pidtxt*/
01372     char pidtxt[PIDTXTSIZE], /*64/Log_2[10] = 19.3, this is enough for any integer*/
01373          chdescriptor[PIDTXTSIZE],
01374          bothpidtxt[BOTHPIDSIZE], /*"#,#\n\0"*/
01375          *c,*b = 0;
01376     int pip[2];
01377     pid_t ppid;
01378     if ( theline == NULL ) return(-1);
01379     /*Put into pidtxt PID\n:*/
01380     c = l2s((LONG)getpid(),pidtxt);
01381     *c++='\n';
01382     *c = '\0';
01383     pidln = c-pidtxt;
01384
01385     do {
01386         pip[0] = (int)str2i((char*)theline,&c,0xFFFF);
01387         if( ( pip[0] < 0 ) || ( *c != ',' ) ) goto presetFails;
01388
01389         theline = (UBYTE*)c + 1;

```

```

01390     pip[1] = (int)str2i((char*)theline,&c,0xFFFF);
01391     if ( (pip[1] < 0) || ( (*c != ',') && ( *c != '\0' ) ) ) goto presetFails;
01392     theline = (UBYTE *)c + 1;
01393     /*Now we have two descriptors.
01394     According to the protocol, FORM must send to external channel
01395     it's PID with added '\n' and read back two comma-separated
01396     decimals with added '\n'. The first must be repeated FORM PID,
01397     the second must be the parent PID
01398     */
01399     if ( writeSome(pip[1],pidtxt,pidln,thetimeout) != pidln ) goto presetFails;
01400     i = readSome(pip[0],bothpidtxt,BOTHPIDSIZE,thetimeout);
01401     if( ( i < 4 ) /*at least 1,2\n*/
01402         || ( i == BOTHPIDSIZE ) /*No space for trailing '\0'*/
01403     ) goto presetFails;
01404     /*read the FORM PID:*/
01405     ppid = (pid_t)str2i(bothpidtxt,&b,getpid());
01406     if( ( *b != ',' ) || ( ppid != getpid() ) ) goto presetFails;
01407     /*read the parent PID:*/
01408     /*The problem is that we do not know the the real type of pid_t.
01409     But long should be enough. On obsolete systems (when LONG_MAX
01410     is not defined) we assume pid_t is 32-bit integer.
01411     This can lead to problem with portability: */
01412     ppid = (pid_t)str2i(b+1,&b,LONG_MAX);
01413     if ( (*b != '\n') || (ppid<2) ) goto presetFails;
01414     i = openPresetExternalChannel(pip[0],pip[1],ppid);
01415     if ( i < 0 ) goto presetFails;
01416     nchannels++;
01417     /*Now use bothpidtxt as a buffer for preprovar, the space is enough:*/
01418     /*"PIPE#_" where # is ne order number of the channel:*/
01419     b = l2s(nchannels,addStr(bothpidtxt,"PIPE"));
01420     *b = '\0';
01421     b[1] = '\0';
01422     *l2s(i,chdescriptor) = '\0';
01423     PutPreVar((UBYTE*)bothpidtxt,(UBYTE*)chdescriptor,0,0);
01424 } while ( *c != '\0' );
01425 /*Export proprovar "PIPES_":*/
01426 *l2s(nchannels,chdescriptor)='\0';
01427 PutPreVar((UBYTE*)"PIPES_",(UBYTE*)chdescriptor,0,0);
01428
01429 /*success:*/
01430 return (nchannels);
01431
01432 presetFails:
01433     /*Here we assume the descriptors the beginning of the list!*/
01434     for(i=0; i<nchannels; i++)
01435         destroyExternalChannel(externalChannelsList+i);
01436     return(-1);
01437 } /*initPresetExternalChannels*/
01438 /*
01439     #] initPresetExternalChannels :
01440     #[ selectExternalChannel :
01441     */
01442 /*
01443     Accepts the valid external channel descriptor (returned by
01444     openExternalChannel) and returns the descriptor of a previous current
01445     channel (0, if there was no current channel, or -1, if the external
01446     channel descriptor is invalid). If n == 0, the function undefine the
01447     current external channel:
01448     */
01449 int selectExternalChannel(int n)
01450 {
01451     int ret=0;
01452     if(externalChannelsListTop!=0)
01453         ret=externalChannelsListTop-externalChannelsList+1;
01454
01455     if(--n<0){
01456         if(n!=-1)
01457             return(-1);
01458         externalChannelsListTop=0;
01459         writeBufToExtChannel=&writeBufToExtChannelFailure;
01460         getcFromExtChannel=&getcFromExtChannelFailure;
01461         setTerminatorForExternalChannel=&setTerminatorForExternalChannelFailure;
01462         setKillModeForExternalChannel=&setKillModeForExternalChannelFailure;
01463         return(ret);
01464     }
01465     if(
01466         (n>=externalChannelsListFill)||
01467         (externalChannelsList[n].cmd==0)
01468     )
01469         return(-1);
01470
01471     externalChannelsListTop=externalChannelsList+n;
01472     writeBufToExtChannel=&writeBufToExtChannelOk;
01473     getcFromExtChannel=&getcFromExtChannelOk;
01474     setTerminatorForExternalChannel=&setTerminatorForExternalChannelOk;
01475     setKillModeForExternalChannel=&setKillModeForExternalChannelOk;
01476     return(ret);

```

```

01477  */selectExternalChannel*/
01478  /*
01479      #] selectExternalChannel :
01480      #[ closeExternalChannel :
01481  */
01482
01483  /*
01484  Destroys the opened external channel with the descriptor n. It returns
01485  0 in success, or -1 on failure. If the corresponding external channel
01486  was the current one, the current channel becomes undefined. If n==0,
01487  the function closes the current external channel.
01488  */
01489  int closeExternalChannel(int n)
01490  {
01491      if(n==0)
01492          n=externalChannelsListTop-externalChannelsList;
01493      else
01494          n--;/*Count from 0*/
01495
01496      if(
01497          (n<0) ||
01498          (n>=externalChannelsListFill) ||
01499          (externalChannelsList[n].cmd==0)
01500      )/*No such a channel*/
01501          return(-1);
01502
01503      destroyExternalChannel(externalChannelsList+n);
01504      /*If the current external channel was destroyed, undefine current channel:*/
01505      if(externalChannelsListTop==externalChannelsList+n){
01506          externalChannelsListTop=NULL;
01507          writeBufToExtChannel=&writeBufToExtChannelFailure;
01508          getcFromExtChannel=&getcFromExtChannelFailure;
01509          setTerminatorForExternalChannel=&setTerminatorForExternalChannelFailure;
01510          setKillModeForExternalChannel=&setKillModeForExternalChannelFailure;
01511      }/*if(externalChannelsListTop==externalChannelsList+n)*/
01512      return(0);
01513  }/*closeExternalChannel*/
01514  /*
01515      #] closeExternalChannel :
01516      #[ closeAllExternalChannels :
01517  */
01518  void closeAllExternalChannels(void)
01519  {
01520      int i;
01521      for(i=0; i<externalChannelsListFill; i++)
01522          destroyExternalChannel(externalChannelsList+i);
01523      externalChannelsListFill=externalChannelsListStop=0;
01524      externalChannelsListTop=NULL;
01525
01526      writeBufToExtChannel=&writeBufToExtChannelFailure;
01527      getcFromExtChannel=&getcFromExtChannelFailure;
01528      setTerminatorForExternalChannel=&setTerminatorForExternalChannelFailure;
01529      setKillModeForExternalChannel=&setKillModeForExternalChannelFailure;
01530
01531      if(externalChannelsList!=NULL){
01532          M_free(externalChannelsList,"External channel list");
01533          externalChannelsList=NULL;
01534      }
01535  }/*closeAllExternalChannels*/
01536  /*
01537      #] closeAllExternalChannels :
01538      #[ getExternalChannelPid :
01539  */
01540  #ifdef SELFTEST
01541  pid_t getExternalChannelPid(void)
01542  {
01543      if(externalChannelsListTop!=0)
01544          return(externalChannelsListTop->pid);
01545      return(-1);
01546  }/*getExternalChannelPid*/
01547  #endif
01548  /*
01549      #] getExternalChannelPid :
01550      #[ getCurrentExternalChannel :
01551  */
01552
01553  int getCurrentExternalChannel(void)
01554  {
01555
01556      if ( externalChannelsListTop != 0 )
01557          return(externalChannelsListTop-externalChannelsList+1);
01558      return(0);
01559  }/*getCurrentExternalChannel*/
01560  /*
01561      #] getCurrentExternalChannel :
01562      #[ Selftest main :
01563  */

```

```

01564
01565 #ifdef SELFTEST
01566
01567 /*
01568      This is the example of how all these public functions may be used:
01569 */
01570
01571 char buf[1024];
01572 char buf2[1024];
01573
01574 void help(void)
01575 {
01576     printf("String started with a special letter is a command\n");
01577     printf("Known commands are:\n");
01578     printf("H or ? -- this help\n");
01579     printf("Nn<command> -- start a new command\n");
01580     printf("S<command> -- start a new command in a subshell,daemon,stderr>/dev/null\n");
01581     printf("C# -- destroy channel #\n");
01582     printf("R# -- set a new cahhel(number#) as a current one\n");
01583     printf("K#1 #2 -- set signal for kill and kill mode (0 or !=0)\n");
01584     printf("    ^d to quit\n");
01585 }/*help*/
01586
01587 int main (void)
01588 {
01589     int i, j, k,last;
01590     long long sum = 0;
01591
01592     /*openExternalChannel(UBYTE *cmd, int daemonize, UBYTE *shellname, UBYTE *stderrname)*/
01593
01594     help();
01595
01596     printf("Initial channel:%d\n",last=openExternalChannel((UBYTE*)"cat",0,NULL,NULL));
01597
01598     if( ( i = setTerminatorForExternalChannel("qu") ) != 0 ) return 1;
01599     printf("Terminator is 'qu'\n");
01600
01601     while ( fgets(buf, 1024, stdin) != NULL ) {
01602         if ( *buf == 'N' ) {
01603             printf("New channel:%d\n",j=openExternalChannel((UBYTE*)buf+1,0,NULL,NULL));
01604             continue;
01605         }
01606         else if ( *buf == 'C' ) {
01607             int n;
01608             sscanf(buf+1,"%d",&n);
01609             printf("Destroy last channel:%d\n",closeExternalChannel(n));
01610             continue;
01611         }
01612         else if ( *buf == 'R' ) {
01613             int n = 0;
01614             sscanf(buf+1,"%d",&n);
01615             printf("Reopen channel %d:%d\n",n,selectExternalChannel(n));
01616             continue;
01617         }else if( *buf == 'K' ) {
01618             int n=0,g = 0;
01619             sscanf(buf+1,"%d %d",&n,&g);
01620             printf("setKillMode %d\n",setKillModeForExternalChannel(n,g));
01621             continue;
01622         }else if( *buf == 'S' ) {
01623             printf("New channel with sh&d&stderr:%d\n",
01624                 j=openExternalChannel((UBYTE*)buf+1,1,(UBYTE*)"/bin/sh -c", (UBYTE*)"/dev/null"));
01625             continue;
01626         }
01627         else if( ( *buf == 'H' ) || ( *buf == '?' ) ){
01628             help();
01629             continue;
01630         }
01631
01632         writeBufToExtChannel(buf,k=StrLen(buf));
01633         sum += k;
01634         for ( j = 0; ( i = getcFromExtChannel() ) != '\n'; j++) {
01635             if ( i == EOF ) {
01636                 printf("EOF!\n");
01637                 break;
01638             }
01639             buf2[j] = i;
01640         }
01641         buf2[j] = '\0';
01642         printf("I got:'%s'; pid=%d\n",buf2,getExternalChannelPid());
01643     }
01644     printf("Total:%lld bytes\n",sum);
01645     closeAllExternalChannels();
01646     return 0;
01647 }
01648 #endif /*ifdef SELFTEST*/
01649 /*
01650     #] Selftest main :

```

```

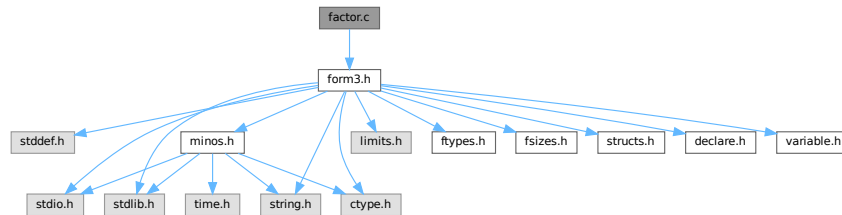
01651 */
01652
01653 #endif /*ifndef WITHEXTERNALCHANNEL ... else*/

```

4.33 factor.c File Reference

```
#include "form3.h"
```

Include dependency graph for factor.c:



Functions

- int [FactorIn](#) (PHEAD WORD *term, WORD level)
- int [FactorInExpr](#) (PHEAD WORD *term, WORD level)

4.33.1 Detailed Description

The routines for finding (one term) factors in dollars and expressions.

Definition in file [factor.c](#).

4.33.2 Function Documentation

4.33.2.1 FactorIn()

```

int FactorIn (
    PHEAD WORD * term,
    WORD level )

```

Definition at line [46](#) of file [factor.c](#).

4.33.2.2 FactorInExpr()

```

int FactorInExpr (
    PHEAD WORD * term,
    WORD level )

```

Definition at line [421](#) of file [factor.c](#).

4.34 factor.c

[Go to the documentation of this file.](#)

```

00001
00005 /* #[] License : */
00006 /*
00007 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00008 *   When using this file you are requested to refer to the publication
00009 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00010 *   This is considered a matter of courtesy as the development was paid
00011 *   for by FOM the Dutch physics granting agency and we would like to
00012 *   be able to track its scientific use to convince FOM of its value
00013 *   for the community.
00014 *
00015 *   This file is part of FORM.
00016 *
00017 *   FORM is free software: you can redistribute it and/or modify it under the
00018 *   terms of the GNU General Public License as published by the Free Software
00019 *   Foundation, either version 3 of the License, or (at your option) any later
00020 *   version.
00021 *
00022 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00023 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00024 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00025 *   details.
00026 *
00027 *   You should have received a copy of the GNU General Public License along
00028 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00029 */
00030 /* #[] License : */
00031 /*
00032 *   #[] Includes : factor.c
00033 */
00034
00035 #include "form3.h"
00036
00037 /*
00038 *   #[] Includes :
00039 *   #[] FactorIn :
00040
00041 *   This routine tests for a factor in a dollar expression.
00042
00043 *   Note that unlike with regular active or hidden expressions we cannot
00044 *   add memory as easily as dollars are rather volatile.
00045 */
00046 int FactorIn(PHEAD WORD *term, WORD level)
00047 {
00048     GETBIDENTITY
00049     WORD *t, *tstop, *m, *mm, *oldwork, *mstop, *n1, *n2, *n3, *n4, *n1stop, *n2stop;
00050     WORD *r1, *r2, *r3, *r4, j, k, kGCD, kGCD2, kLCM, jGCD, kkLCM, jLCM, size;
00051     UWORD *GCDBuffer, *GCDBuffer2, *LCMbuffer, *LCMb, *LCMc;
00052     int fromwhere = 0, i;
00053     DOLLARS d;
00054     t = term; GETSTOP(t,tstop); t++;
00055     while ( ( t < tstop ) && ( *t != FACTORIN || ( ( *t == FACTORIN )
00056         && ( t[FUNHEAD] != -DOLLAREXPRESSION || t[1] != FUNHEAD+2 ) ) ) ) t += t[1];
00057     if ( t >= tstop ) {
00058         MLOCK(ErrorMessageLock);
00059         MesPrint("Internal error. Could not find proper factorin_ function.");
00060         MUNLOCK(ErrorMessageLock);
00061         return(-1);
00062     }
00063     oldwork = AT.WorkPointer;
00064     d = Dollars + t[FUNHEAD+1];
00065 #ifdef WITHPTHREADS
00066     {
00067         int nummodopt, dtype = -1;
00068         if ( AS.MultiThreaded ) {
00069             for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
00070                 if ( t[FUNHEAD+1] == ModOptdollars[nummodopt].number ) break;
00071             }
00072             if ( nummodopt < NumModOptdollars ) {
00073                 dtype = ModOptdollars[nummodopt].type;
00074                 if ( dtype == MODLOCAL ) {
00075                     d = ModOptdollars[nummodopt].dstruct+AT.identity;
00076                 }
00077             }
00078         }
00079     }
00080 #endif
00081     if ( d->type == DOLTERMS ) {
00082         fromwhere = 1;
00083     }
00084     else if ( ( d = DolToTerms(BHEAD t[FUNHEAD+1]) ) == 0 ) {

```



```

00086 /*
00087     The variable cannot convert to an expression
00088     We replace the function by 1.
00089 */
00090     m = oldwork; n1 = term;
00091     while ( n1 < t ) *m++ = *n1++;
00092     n1 = t + t[1]; tstop = term + *term;
00093     while ( n1 < tstop ) *m++ = *n1++;
00094     *oldwork = m - oldwork;
00095     AT.WorkPointer = m;
00096     if ( Generator(BHEAD oldwork, level) ) return(-1);
00097     AT.WorkPointer = oldwork;
00098     return(0);
00099 }
00100 if ( d->where[0] == 0 ) {
00101     if ( fromwhere == 0 ) {
00102         if ( d->factors ) M_free(d->factors, "Dollar factors");
00103         M_free(d, "Dollar in FactorIn_");
00104     }
00105     return(0);
00106 }
00107 /*
00108     Now we have an expression in d->where. Find the symbolic factor that
00109     divides the expression and the numerical factor that makes all
00110     coefficients integer.
00111
00112     For the symbolic factor we make a copy of the first term, and then
00113     go through all terms, scratching in the copy the objects that do not
00114     occur in the terms.
00115 */
00116     m = oldwork;
00117     mm = d->where;
00118     k = *mm - ABS((mm[*mm-1]));
00119     for ( j = 0; j < k; j++ ) *m++ = *mm++;
00120     mstop = m;
00121     *oldwork = k;
00122 /*
00123     The copy is in place. Now search through the terms. Start at the second term
00124 */
00125     mm = d->where + d->where[0];
00126     while ( *mm ) {
00127         m = oldwork+1;
00128         r2 = mm+*mm;
00129         r2 -= ABS(r2[-1]);
00130         r1 = mm+1;
00131         while ( m < mstop ) {
00132             while ( r1 < r2 ) {
00133                 if ( *r1 != *m ) {
00134                     r1 += r1[1]; continue;
00135                 }
00136             }
00137             /*
00138                 Now the various cases
00139             */
00140             if ( *m == SYMBOL ) {
00141                 n1 = m+2; n1stop = m+m[1];
00142                 n2stop = r1+r1[1];
00143                 while ( n1 < n1stop ) {
00144                     n2 = r1+2;
00145                     while ( n2 < n2stop ) {
00146                         if ( *n1 != *n2 ) { n2 += 2; continue; }
00147                         if ( n1[1] > 0 ) {
00148                             if ( n2[1] < 0 ) { n2 += 2; continue; }
00149                             if ( n2[1] < n1[1] ) n1[1] = n2[1];
00150                         }
00151                         else {
00152                             if ( n2[1] > 0 ) { n2 += 2; continue; }
00153                             if ( n2[1] > n1[1] ) n1[1] = n2[1];
00154                         }
00155                         break;
00156                     }
00157                     if ( n2 >= n2stop ) { /* scratch symbol */
00158                         if ( m[1] == 4 ) goto scratch;
00159                         m[1] -= 2;
00160                         n3 = n1; n4 = n1+2;
00161                         while ( n4 < mstop ) *n3++ = *n4++;
00162                         *oldwork = n3 - oldwork;
00163                         mstop -= 2; n1stop -= 2;
00164                         continue;
00165                     }
00166                     n1 += 2;
00167                 }
00168                 break;
00169             }
00170             /*
00171                 #] SYMBOL :
00172                 #[ DOTPRODUCT :

```

```

00173 */
00174         else if ( *m == DOTPRODUCT ) {
00175             n1 = m+2; n1stop = m+m[1];
00176             n2stop = r1+r1[1];
00177             while ( n1 < n1stop ) {
00178                 n2 = r1+2;
00179                 while ( n2 < n2stop ) {
00180                     if ( *n1 != *n2 || n1[1] != n2[1] ) { n2 += 3; continue; }
00181                     if ( n1[2] > 0 ) {
00182                         if ( n2[2] < 0 ) { n2 += 3; continue; }
00183                         if ( n2[2] < n1[2] ) n1[2] = n2[2];
00184                     }
00185                     else {
00186                         if ( n2[2] > 0 ) { n2 += 3; continue; }
00187                         if ( n2[2] > n1[2] ) n1[2] = n2[2];
00188                     }
00189                     break;
00190                 }
00191                 if ( n2 >= n2stop ) { /* scratch symbol */
00192                     if ( m[1] == 5 ) goto scratch;
00193                     m[1] -= 3;
00194                     n3 = n1; n4 = n1+3;
00195                     while ( n4 < mstop ) *n3++ = *n4++;
00196                     *oldwork = n3 - oldwork;
00197                     mstop -= 3; n1stop -= 3;
00198                     continue;
00199                 }
00200                 n1 += 3;
00201             }
00202             break;
00203         }
00204 /*
00205 #] DOTPRODUCT :
00206 #[ VECTOR :
00207 */
00208         else if ( *m == VECTOR ) {
00209 /*
00210     Here we have to be careful if there is more than
00211     one of the same
00212 */
00213             n1 = m+2; n1stop = m+m[1];
00214             n2 = r1+2; n2stop = r1+r1[1];
00215             while ( n1 < n1stop ) {
00216                 while ( n2 < n2stop ) {
00217                     if ( *n1 == *n2 && n1[1] == n2[1] ) {
00218                         n2 += 2; goto nextn1;
00219                     }
00220                     n2 += 2;
00221                 }
00222                 if ( n2 >= n2stop ) { /* scratch symbol */
00223                     if ( m[1] == 4 ) goto scratch;
00224                     m[1] -= 2;
00225                     n3 = n1; n4 = n1+2;
00226                     while ( n4 < mstop ) *n3++ = *n4++;
00227                     *oldwork = n3 - oldwork;
00228                     mstop -= 2; n1stop -= 2;
00229                     continue;
00230                 }
00231                 n2 = r1+2;
00232 nextn1:             n1 += 2;
00233             }
00234             break;
00235         }
00236 /*
00237 #] VECTOR :
00238 #[ REMAINDER :
00239 */
00240         else {
00241 /*
00242     Again: watch for multiple occurrences of the same object
00243 */
00244             if ( m[1] != r1[1] ) { r1 += r1[1]; continue; }
00245             for ( j = 2; j < m[1]; j++ ) {
00246                 if ( m[j] != r1[j] ) break;
00247             }
00248             if ( j < m[1] ) { r1 += r1[1]; continue; }
00249             r1 += r1[1]; /* to restart at the next potential match */
00250             goto nextm; /* match */
00251         }
00252 /*
00253 #] REMAINDER :
00254 */
00255     }
00256     if ( r1 >= r2 ) { /* no factor! */
00257 scratch:;
00258         r3 = m + m[1]; r4 = m;
00259         while ( r3 < mstop ) *r4++ = *r3++;

```

```

00260             *oldwork = r4 - oldwork;
00261             if ( *oldwork == 1 ) goto nofactor;
00262             mstop = r4;
00263             r1 = mm + 1;
00264             continue;
00265         }
00266         r1 = mm + 1;
00267 nextm:      m += m[1];
00268     }
00269     mm = mm + *mm;
00270 }
00271
00272 nofactor;;
00273 /*
00274     For the coefficient we have to determine the LCM of the denominators
00275     and the GCD of the numerators.
00276 */
00277     GCDbuffer = NumberMalloc("FactorIn"); GCDbuffer2 = NumberMalloc("FactorIn");
00278     LCMbuffer = NumberMalloc("FactorIn"); LCMb = NumberMalloc("FactorIn"); LCMc =
00279     NumberMalloc("FactorIn");
00280     r1 = d->where;
00281 /*
00282     First take the first term to load up the LCM and the GCD
00283 */
00284     r2 = r1 + *r1;
00285     j = r2[-1];
00286     r3 = r2 - ABS(j);
00287     k = REDLENG(j);
00288     if ( k < 0 ) k = -k;
00289     while ( ( k > 1 ) && ( r3[k-1] == 0 ) ) k--;
00290     for ( kGCD = 0; kGCD < k; kGCD++ ) GCDbuffer[kGCD] = r3[kGCD];
00291     k = REDLENG(j);
00292     if ( k < 0 ) k = -k;
00293     r3 += k;
00294     while ( ( k > 1 ) && ( r3[k-1] == 0 ) ) k--;
00295     for ( kLCM = 0; kLCM < k; kLCM++ ) LCMbuffer[kLCM] = r3[kLCM];
00296     r1 = r2;
00297 /*
00298     Now go through the rest of the terms in this dollar buffer.
00299 */
00300     while ( *r1 ) {
00301         r2 = r1 + *r1;
00302         j = r2[-1];
00303         r3 = r2 - ABS(j);
00304         k = REDLENG(j);
00305         if ( k < 0 ) k = -k;
00306         while ( ( k > 1 ) && ( r3[k-1] == 0 ) ) k--;
00307         if ( ( ( GCDbuffer[0] == 1 ) && ( kGCD == 1 ) ) ) {
00308             /*
00309                 GCD is already 1
00310             */
00311             else if ( ( ( k != 1 ) || ( r3[0] != 1 ) ) ) {
00312                 if ( GcdLong(BHEAD GCDbuffer, kGCD, (UWORD *)r3, k, GCDbuffer2, &kGCD2) ) {
00313                     goto onerror;
00314                 }
00315                 kGCD = kGCD2;
00316                 for ( i = 0; i < kGCD; i++ ) GCDbuffer[i] = GCDbuffer2[i];
00317             }
00318             else {
00319                 kGCD = 1; GCDbuffer[0] = 1;
00320             }
00321             k = REDLENG(j);
00322             if ( k < 0 ) k = -k;
00323             r3 += k;
00324             while ( ( k > 1 ) && ( r3[k-1] == 0 ) ) k--;
00325             if ( ( ( LCMbuffer[0] == 1 ) && ( kLCM == 1 ) ) ) {
00326                 for ( kLCM = 0; kLCM < k; kLCM++ )
00327                     LCMbuffer[kLCM] = r3[kLCM];
00328             }
00329             else if ( ( k != 1 ) || ( r3[0] != 1 ) ) {
00330                 if ( GcdLong(BHEAD LCMbuffer, kLCM, (UWORD *)r3, k, LCMb, &kLCM) ) {
00331                     goto onerror;
00332                 }
00333                 DivLong((UWORD *)r3, k, LCMb, kLCM, LCMb, &kLCM, LCMc, &jLCM);
00334                 MulLong(LCMbuffer, kLCM, LCMb, kLCM, LCMc, &jLCM);
00335                 for ( kLCM = 0; kLCM < jLCM; kLCM++ )
00336                     LCMbuffer[kLCM] = LCMc[kLCM];
00337             }
00338             else {} /* LCM doesn't change */
00339             r1 = r2;
00340         }
00341     }
00342 /*
00343     Now put the factor together: GCD/LCM
00344 */
00345     r3 = (WORD *) (GCDbuffer);
00346     if ( kGCD == kLCM ) {

```

```

00346         for ( jGCD = 0; jGCD < kGCD; jGCD++ )
00347             r3[jGCD+kGCD] = LCMbuffer[jGCD];
00348         k = kGCD;
00349     }
00350     else if ( kGCD > kLCM ) {
00351         for ( jGCD = 0; jGCD < kLCM; jGCD++ )
00352             r3[jGCD+kGCD] = LCMbuffer[jGCD];
00353         for ( jGCD = kLCM; jGCD < kGCD; jGCD++ )
00354             r3[jGCD+kGCD] = 0;
00355         k = kGCD;
00356     }
00357     else {
00358         for ( jGCD = kGCD; jGCD < kLCM; jGCD++ )
00359             r3[jGCD] = 0;
00360         for ( jGCD = 0; jGCD < kLCM; jGCD++ )
00361             r3[jGCD+kLCM] = LCMbuffer[jGCD];
00362         k = kLCM;
00363     }
00364     j = 2*k+1;
00365     mm = m = oldwork + oldwork[0];
00366 /*
00367     Now compose the new term
00368 */
00369     n1 = term;
00370     while ( n1 < t ) *m++ = *n1++;
00371     n1 += n1[1];
00372     n2 = oldwork+1;
00373     while ( n2 < mm ) *m++ = *n2++;
00374     while ( n1 < tstop ) *m++ = *n1++;
00375 /*
00376     And the coefficient
00377 */
00378     size = term[*term-1];
00379     size = REDLENG(size);
00380     if ( MulRat(BHEAD (UWORD *)tstop,size,(UWORD *)r3,k,
00381                (UWORD *)m,&size) ) goto onerror;
00382     size = INCLENG(size);
00383     k = size < 0 ? -size: size;
00384     m[k-1] = size;
00385     m += k;
00386     *mm = (WORD)(m - mm);
00387     AT.WorkPointer = m;
00388     if ( Generator(BHEAD mm,level) ) goto onerror;
00389     AT.WorkPointer = oldwork;
00390     if ( fromwhere == 0 ) {
00391         if ( d->factors ) M_free(d->factors,"Dollar factors");
00392         M_free(d,"Dollar in FactorIn");
00393     }
00394     NumberFree(GCDbuffer,"FactorIn"); NumberFree(GCDbuffer2,"FactorIn");
00395     NumberFree(LCMbuffer,"FactorIn"); NumberFree(LCMb,"FactorIn"); NumberFree(LCMc,"FactorIn");
00396     return(0);
00397 onerror:
00398     AT.WorkPointer = oldwork;
00399     MLOCK(ErrorMessageLock);
00400     MesCall("FactorIn");
00401     MUNLOCK(ErrorMessageLock);
00402     NumberFree(GCDbuffer,"FactorIn"); NumberFree(GCDbuffer2,"FactorIn");
00403     NumberFree(LCMbuffer,"FactorIn"); NumberFree(LCMb,"FactorIn"); NumberFree(LCMc,"FactorIn");
00404     return(-1);
00405 }
00406
00407 /*
00408 #] FactorIn :
00409 #[ FactorInExpr :
00410
00411     This routine tests for a factor in an active or hidden expression.
00412
00413     The factor from the last call is stored in a cache.
00414     Main problem here is whether the cache is global or private to each thread.
00415     A global cache gives most likely the same traffic jam for the computation
00416     as a local cache. The local cache may stay valid longer.
00417     In the future we may make it such that threads can look at the cache
00418     of the others, or even whether a certain factor is under construction.
00419 */
00420
00421 int FactorInExpr(PHEAD WORD *term, WORD level)
00422 {
00423     GETBIDENTITY
00424     WORD *t, *tstop, *m, *oldwork, *mstop, *n1, *n2, *n3, *n4, *n1stop, *n2stop;
00425     WORD *r1, *r2, *r3, *r4, j, k, kGCD, kGCD2, kLCM, jGCD, kkLCM, jLCM, size, sign;
00426     WORD *newterm, expr = 0;
00427     WORD olddeferflag = AR.DeferFlag, oldgetfile = AR.GetFile, oldhold = AR.KeptInHold;
00428     WORD newgetfile, newhold;
00429     int i;
00430     EXPRESSIONS e;
00431     FILEHANDLE *file = 0;
00432     POSITION position, oldposition, startposition;

```

```

00433     WORD *oldcpointer = AR.CompressPointer;
00434     UWORD *GCDBuffer, *GCDBuffer2, *LCMbuffer, *LCMb, *LCMc;
00435     GCDBuffer = NumberMalloc("FactorInExpr"); GCDBuffer2 = NumberMalloc("FactorInExpr");
00436     LCMbuffer = NumberMalloc("FactorInExpr"); LCMb = NumberMalloc("FactorInExpr"); LCMc =
NumberMalloc("FactorInExpr");
00437     t = term; GETSTOP(t,tstop); t++;
00438     while ( t < tstop ) {
00439         if ( *t == FACTORIN && t[1] == FUNHEAD+2 && t[FUNHEAD] == -EXPRESSION ) {
00440             expr = t[FUNHEAD+1];
00441             break;
00442         }
00443         t += t[1];
00444     }
00445     if ( t >= tstop ) {
00446         MLOCK(ErrorMessageLock);
00447         MesPrint("Internal error. Could not find proper factorin_ function.");
00448         MUNLOCK(ErrorMessageLock);
00449         NumberFree(GCDBuffer, "FactorInExpr"); NumberFree(GCDBuffer2, "FactorInExpr");
00450         NumberFree(LCMbuffer, "FactorInExpr"); NumberFree(LCMb, "FactorInExpr");
NumberFree(LCMc, "FactorInExpr");
00451         return(-1);
00452     }
00453     oldwork = AT.WorkPointer;
00454     if ( AT.previousEfactor && ( expr == AT.previousEfactor[0] ) ) {
00455         /*
00456             We have a hit in the cache. Construct the new term.
00457             At the moment AT.previousEfactor[1] is reserved for future flags
00458         */
00459         goto PutTheFactor;
00460     }
00461     /*
00462     No hit. We have to do the work. We start with constructing the factor
00463     in the WorkSpace. Later we will move it to the cache.
00464     Finally we will jump to PutTheFactor.
00465     */
00466     e = Expressions + expr;
00467     switch ( e->status ) {
00468         case LOCALEXPRESSION:
00469         case SKIPLEXPRESSION:
00470         case DROPLEXPRESSION:
00471         case GLOBALEXPRESSION:
00472         case SKIPGEXPRESSION:
00473         case DROPGEXPRESSION:
00474     /*
00475         Expression is to be found in the input Scratch file.
00476         Set the file handle and the position.
00477         The rest is done by GetTerm.
00478     */
00479         newhold = 0;
00480         newgetfile = 1;
00481         file = AR.infile;
00482         break;
00483         case HIDDENLEXPRESSION:
00484         case HIDDENGEXPRESSION:
00485         case HIDELEXPRESSION:
00486         case HIDEGEXPRESSION:
00487         case DROPHLEXPRESSION:
00488         case DROPHGEXPRESSION:
00489         case UNHIDELEXPRESSION:
00490         case UNHIDEGEXPRESSION:
00491     /*
00492         Expression is to be found in the hide Scratch file.
00493         Set the file handle and the position.
00494         The rest is done by GetTerm.
00495     */
00496         newhold = 0;
00497         newgetfile = 2;
00498         file = AR.hidefile;
00499         break;
00500         case STOREDEXPRESSION:
00501     /*
00502         This is an 'illegal' case
00503     */
00504         MLOCK(ErrorMessageLock);
00505         MesPrint("Error: factorin_ cannot determine factors in stored expressions.");
00506         MUNLOCK(ErrorMessageLock);
00507         NumberFree(GCDBuffer, "FactorInExpr"); NumberFree(GCDBuffer2, "FactorInExpr");
00508         NumberFree(LCMbuffer, "FactorInExpr"); NumberFree(LCMb, "FactorInExpr");
NumberFree(LCMc, "FactorInExpr");
00509         return(-1);
00510         case DROPPEDEXPRESSION:
00511     /*
00512         We replace the function by 1.
00513     */
00514         m = oldwork; n1 = term;
00515         while ( n1 < t ) *m++ = *n1++;
00516         n1 = t + t[1]; tstop = term + *term;

```

```

00517         while ( nl < tstop ) *m++ = *nl++;
00518         *oldwork = m - oldwork;
00519         AT.WorkPointer = m;
00520         if ( Generator(BHEAD oldwork,level) ) {
00521             NumberFree(GCDbuffer,"FactorInExpr"); NumberFree(GCDbuffer2,"FactorInExpr");
00522             NumberFree(LCMbuffer,"FactorInExpr"); NumberFree(LCMb,"FactorInExpr");
00523             NumberFree(LCMc,"FactorInExpr");
00524             return(-1);
00525         }
00526         AT.WorkPointer = oldwork;
00527         NumberFree(GCDbuffer,"FactorInExpr"); NumberFree(GCDbuffer2,"FactorInExpr");
00528         NumberFree(LCMbuffer,"FactorInExpr"); NumberFree(LCMb,"FactorInExpr");
00529         NumberFree(LCMc,"FactorInExpr");
00530         return(0);
00531     default:
00532         MLOCK(ErrorMessageLock);
00533         MesPrint("Error: Illegal expression in factorinexpr.");
00534         MUNLOCK(ErrorMessageLock);
00535         NumberFree(GCDbuffer,"FactorInExpr"); NumberFree(GCDbuffer2,"FactorInExpr");
00536         NumberFree(LCMbuffer,"FactorInExpr"); NumberFree(LCMb,"FactorInExpr");
00537         NumberFree(LCMc,"FactorInExpr");
00538         return(-1);
00539     }
00540 /*
00541  Before we start with the file we set the buffers for the coefficient
00542  For the coefficient we have to determine the LCM of the denominators
00543  and the GCD of the numerators.
00544 */
00545 position = AS.OldOnFile[expr];
00546 AR.DeferFlag = 0; AR.KeptInHold = newhold; AR.GetFile = newgetfile;
00547 SeekScratch(file,&oldposition);
00548 SetScratch(file,&position);
00549 if ( GetTerm(BHEAD oldwork) <= 0 ) {
00550     MLOCK(ErrorMessageLock);
00551     MesPrint("(5) Expression %d has problems in scratchfile",expr);
00552     MUNLOCK(ErrorMessageLock);
00553     NumberFree(GCDbuffer,"FactorInExpr"); NumberFree(GCDbuffer2,"FactorInExpr");
00554     NumberFree(LCMbuffer,"FactorInExpr"); NumberFree(LCMb,"FactorInExpr");
00555     NumberFree(LCMc,"FactorInExpr");
00556     return(-1);
00557 }
00558 SeekScratch(file,&startposition);
00559 SeekScratch(file,&position);
00560 /*
00561  Load the first term in the workspace
00562 */
00563 if ( GetTerm(BHEAD oldwork) == 0 ) {
00564     SetScratch(file,&oldposition); /* We still need this until Processor is clean */
00565     AR.DeferFlag = olddeferflag;
00566     oldwork[0] = 4; oldwork[1] = 1; oldwork[2] = 1; oldwork[3] = 3;
00567     goto Complete;
00568 }
00569 SeekScratch(file,&position);
00570 AR.DeferFlag = olddeferflag; AR.KeptInHold = oldhold; AR.GetFile = oldgetfile;
00571 r2 = m = oldwork + *oldwork;
00572 j = m[-1];
00573 m -= ABS(j);
00574 *oldwork = (WORD) (m-oldwork);
00575 AT.WorkPointer = newterm = mstop = m;
00576 /*
00577  Now take the coefficient of the first term to load up the LCM and the GCD
00578 */
00579 r3 = m;
00580 k = REDLENG(j);
00581 if ( k < 0 ) { k = -k; sign = -1; }
00582 else { sign = 1; }
00583 while ( ( k > 1 ) && ( r3[k-1] == 0 ) ) k--;
00584 for ( kGCD = 0; kGCD < k; kGCD++ ) GCDbuffer[kGCD] = r3[kGCD];
00585 k = REDLENG(j);
00586 if ( k < 0 ) k = -k;
00587 r3 += k;
00588 while ( ( k > 1 ) && ( r3[k-1] == 0 ) ) k--;
00589 for ( kLCM = 0; kLCM < k; kLCM++ ) LCMbuffer[kLCM] = r3[kLCM];
00590 /*
00591  The copy and the coefficient are in place. Now search through the terms.
00592 */
00593 for ( ;; ) {
00594     AR.DeferFlag = 0; AR.KeptInHold = newhold; AR.GetFile = newgetfile;
00595     SetScratch(file,&position);
00596     size = GetTerm(BHEAD newterm);
00597     SeekScratch(file,&position);
00598     AR.DeferFlag = olddeferflag; AR.KeptInHold = oldhold; AR.GetFile = oldgetfile;
00599     if ( size == 0 ) break;
00600     m = oldwork+1;
00601     r2 = newterm + *newterm;
00602     r2 -= ABS(r2[-1]);

```

```

00600     r1 = newterm+1;
00601     while ( m < mstop ) {
00602         while ( r1 < r2 ) {
00603             if ( *r1 != *m ) {
00604                 r1 += r1[1]; continue;
00605             }
00606         /*
00607             Now the various cases
00608             #[ SYMBOL :
00609         */
00610         if ( *m == SYMBOL ) {
00611             n1 = m+2; n1stop = m+m[1];
00612             n2stop = r1+r1[1];
00613             while ( n1 < n1stop ) {
00614                 n2 = r1+2;
00615                 while ( n2 < n2stop ) {
00616                     if ( *n1 != *n2 ) { n2 += 2; continue; }
00617                     if ( n1[1] > 0 ) {
00618                         if ( n2[1] < 0 ) { n2 += 2; continue; }
00619                         if ( n2[1] < n1[1] ) n1[1] = n2[1];
00620                     }
00621                     else {
00622                         if ( n2[1] > 0 ) { n2 += 2; continue; }
00623                         if ( n2[1] > n1[1] ) n1[1] = n2[1];
00624                     }
00625                     break;
00626                 }
00627                 if ( n2 >= n2stop ) { /* scratch symbol */
00628                     if ( m[1] == 4 ) goto scratch;
00629                     m[1] -= 2;
00630                     n3 = n1; n4 = n1+2;
00631                     while ( n4 < mstop ) *n3++ = *n4++;
00632                     *oldwork = n3 - oldwork;
00633                     mstop -= 2; n1stop -= 2;
00634                     continue;
00635                 }
00636                 n1 += 2;
00637             }
00638             break;
00639         }
00640     /*
00641     #[ SYMBOL :
00642     #[ DOTPRODUCT :
00643     */
00644     else if ( *m == DOTPRODUCT ) {
00645         n1 = m+2; n1stop = m+m[1];
00646         n2stop = r1+r1[1];
00647         while ( n1 < n1stop ) {
00648             n2 = r1+2;
00649             while ( n2 < n2stop ) {
00650                 if ( *n1 != *n2 || n1[1] != n2[1] ) { n2 += 3; continue; }
00651                 if ( n1[2] > 0 ) {
00652                     if ( n2[2] < 0 ) { n2 += 3; continue; }
00653                     if ( n2[2] < n1[2] ) n1[2] = n2[2];
00654                 }
00655                 else {
00656                     if ( n2[2] > 0 ) { n2 += 3; continue; }
00657                     if ( n2[2] > n1[2] ) n1[2] = n2[2];
00658                 }
00659                 break;
00660             }
00661             if ( n2 >= n2stop ) { /* scratch dotproduct */
00662                 if ( m[1] == 5 ) goto scratch;
00663                 m[1] -= 3;
00664                 n3 = n1; n4 = n1+3;
00665                 while ( n4 < mstop ) *n3++ = *n4++;
00666                 *oldwork = n3 - oldwork;
00667                 mstop -= 3; n1stop -= 3;
00668                 continue;
00669             }
00670             n1 += 3;
00671         }
00672         break;
00673     }
00674     /*
00675     #[ DOTPRODUCT :
00676     #[ VECTOR :
00677     */
00678     else if ( *m == VECTOR ) {
00679     /*
00680         Here we have to be careful if there is more than
00681         one of the same
00682     */
00683         n1 = m+2; n1stop = m+m[1];
00684         n2 = r1+2; n2stop = r1+r1[1];
00685         while ( n1 < n1stop ) {
00686             while ( n2 < n2stop ) {

```

```

00687             if ( *n1 == *n2 && n1[1] == n2[1] ) {
00688                 n2 += 2; goto nextn1;
00689             }
00690             n2 += 2;
00691         }
00692         if ( n2 >= n2stop ) { /* scratch vector */
00693             if ( m[1] == 4 ) goto scratch;
00694             m[1] -= 2;
00695             n3 = n1; n4 = n1+2;
00696             while ( n4 < mstop ) *n3++ = *n4++;
00697             *oldwork = n3 - oldwork;
00698             mstop -= 2; n1stop -= 2;
00699             continue;
00700         }
00701         n2 = r1+2;
00702 nextn1:     n1 += 2;
00703     }
00704     break;
00705 }
00706 /*
00707 #] VECTOR :
00708 #[ REMAINDER :
00709 */
00710     else {
00711 /*
00712         Again: watch for multiple occurrences of the same object
00713 */
00714         if ( m[1] != r1[1] ) { r1 += r1[1]; continue; }
00715         for ( j = 2; j < m[1]; j++ ) {
00716             if ( m[j] != r1[j] ) break;
00717         }
00718         if ( j < m[1] ) { r1 += r1[1]; continue; }
00719         r1 += r1[1]; /* to restart at the next potential match */
00720         goto nextm; /* match */
00721     }
00722 /*
00723 #] REMAINDER :
00724 */
00725     }
00726     if ( r1 >= r2 ) { /* no factor! */
00727 scratch:;
00728         r3 = m + m[1]; r4 = m;
00729         while ( r3 < mstop ) *r4++ = *r3++;
00730         *oldwork = r4 - oldwork;
00731         if ( *oldwork == 1 ) goto nofactor;
00732         mstop = r4;
00733         r1 = newterm + 1;
00734         continue;
00735     }
00736     r1 = newterm + 1;
00737 nextm:     m += m[1];
00738 }
00739 nofactor:;
00740 /*
00741     Now the coefficient
00742 */
00743     r2 = newterm + *newterm;
00744     j = r2[-1];
00745     r3 = r2 - ABS(j);
00746     k = REDLENG(j);
00747     if ( k < 0 ) k = -k;
00748     while ( ( k > 1 ) && ( r3[k-1] == 0 ) ) k--;
00749     if ( ( ( GCDbuffer[0] == 1 ) && ( kGCD == 1 ) ) ) {
00750 /*
00751         GCD is already 1
00752 */
00753     }
00754     else if ( ( ( k != 1 ) || ( r3[0] != 1 ) ) ) {
00755         if ( GcdLong(BHEAD GCDbuffer,kGCD, (UWORD *)r3,k,GCDbuffer2,&kGCD2) ) {
00756             goto onerror;
00757         }
00758         kGCD = kGCD2;
00759         for ( i = 0; i < kGCD; i++ ) GCDbuffer[i] = GCDbuffer2[i];
00760     }
00761     else {
00762         kGCD = 1; GCDbuffer[0] = 1;
00763     }
00764     k = REDLENG(j);
00765     if ( k < 0 ) k = -k;
00766     r3 += k;
00767     while ( ( k > 1 ) && ( r3[k-1] == 0 ) ) k--;
00768     if ( ( ( LCMbuffer[0] == 1 ) && ( kLCM == 1 ) ) ) {
00769         for ( kLCM = 0; kLCM < k; kLCM++ )
00770             LCMbuffer[kLCM] = r3[kLCM];
00771     }
00772     else if ( ( k != 1 ) || ( r3[0] != 1 ) ) {
00773         if ( GcdLong(BHEAD LCMbuffer,kLCM, (UWORD *)r3,k,LCMb,&kLCM) ) {

```



```

00774         goto onerror;
00775     }
00776     DivLong( (UWORD *)r3,k,LCMb,kkLCM,LCMb,&kkLCM,LCMc,&jLCM);
00777     Mullong(LCMbuffer,kLCM,LCMb,kkLCM,LCMc,&jLCM);
00778     for ( kLCM = 0; kLCM < jLCM; kLCM++ )
00779         LCMbuffer[kLCM] = LCMc[kLCM];
00780 }
00781 else {} /* LCM doesn't change */
00782 }
00783 SetScratch(file,&oldposition); /* Needed until Processor is thread safe */
00784 AR.DeferFlag = olddeferflag;
00785 /*
00786 Now put the term together in oldwork: GCD/LCM
00787 We have already the algebraic contents there.
00788 */
00789 r3 = (WORD *) (GCDbuffer);
00790 r4 = (WORD *) (LCMbuffer);
00791 r1 = oldwork + *oldwork;
00792 if ( kGCD == kLCM ) {
00793     for ( jGCD = 0; jGCD < kGCD; jGCD++ ) *r1++ = *r3++;
00794     for ( jGCD = 0; jGCD < kGCD; jGCD++ ) *r1++ = *r4++;
00795     k = 2*kGCD+1;
00796 }
00797 else if ( kGCD > kLCM ) {
00798     for ( jGCD = 0; jGCD < kGCD; jGCD++ ) *r1++ = *r3++;
00799     for ( jGCD = 0; jGCD < kLCM; jGCD++ ) *r1++ = *r4++;
00800     for ( jGCD = kLCM; jGCD < kGCD; jGCD++ ) *r1++ = 0;
00801     k = 2*kGCD+1;
00802 }
00803 else {
00804     for ( jGCD = 0; jGCD < kGCD; jGCD++ ) *r1++ = *r3++;
00805     for ( jGCD = kGCD; jGCD < kLCM; jGCD++ ) *r1++ = 0;
00806     for ( jGCD = 0; jGCD < kLCM; jGCD++ ) *r1++ = *r4++;
00807     k = 2*kLCM+1;
00808 }
00809 if ( sign < 0 ) *r1++ = -k;
00810 else *r1++ = k;
00811 *oldwork = (WORD) (r1-oldwork);
00812 /*
00813 Now put the new term in the cache
00814 */
00815 Complete;;
00816 if ( AT.previousEfactor ) M_free(AT.previousEfactor,"Efactor cache");
00817 AT.previousEfactor = (WORD *) Malloc1(( *oldwork+2)*sizeof(WORD),"Efactor cache");
00818 AT.previousEfactor[0] = expr;
00819 r1 = oldwork; r2 = AT.previousEfactor + 2; k = *oldwork;
00820 NCOPY(r2,r1,k)
00821 AT.previousEfactor[1] = 0;
00822 /*
00823 Now we construct the new term in the workspace.
00824 */
00825 PutTheFactor;;
00826 if ( AT.WorkPointer + AT.previousEfactor[2] >= AT.WorkTop ) {
00827     MLOCK(ErrorMessageLock);
00828     MesWork();
00829     MesPrint("Called from factorin_");
00830     MUNLOCK(ErrorMessageLock);
00831     NumberFree(GCDbuffer,"FactorInExpr"); NumberFree(GCDbuffer2,"FactorInExpr");
00832     NumberFree(LCMbuffer,"FactorInExpr"); NumberFree(LCMb,"FactorInExpr");
00833     NumberFree(LCMc,"FactorInExpr");
00834     return(-1);
00835 }
00836 n1 = oldwork; n2 = term; while ( n2 < t ) *n1++ = *n2++;
00837 n2 = AT.previousEfactor+2; GETSTOP(n2,n2stop); n3 = n2 + *n2;
00838 n2++; while ( n2 < n2stop ) *n1++ = *n2++;
00839 n2 = t + t[1]; while ( n2 < tstop ) *n1++ = *n2++;
00840 size = term[*term-1];
00841 size = REDLENG(size);
00842 k = n3[-1]; k = REDLENG(k);
00843 if ( MulRat(BHEAD (UWORD *)tstop,size,(UWORD *)n2stop,k,
00844             (UWORD *)n1,&size) ) goto onerror;
00845 size = INCLENG(size);
00846 k = size < 0 ? -size: size;
00847 n1 += k; n1[-1] = size;
00848 *oldwork = n1 - oldwork;
00849 AT.WorkPointer = n1;
00850 if ( Generator(BHEAD oldwork,level) ) {
00851     NumberFree(GCDbuffer,"FactorInExpr"); NumberFree(GCDbuffer2,"FactorInExpr");
00852     NumberFree(LCMbuffer,"FactorInExpr"); NumberFree(LCMb,"FactorInExpr");
00853     NumberFree(LCMc,"FactorInExpr");
00854     return(-1);
00855 }
00856 AT.WorkPointer = oldwork;
00857 AR.CompressPointer = oldcpointer;
00858 NumberFree(GCDbuffer,"FactorInExpr"); NumberFree(GCDbuffer2,"FactorInExpr");
00859 NumberFree(LCMbuffer,"FactorInExpr"); NumberFree(LCMb,"FactorInExpr");
00860 NumberFree(LCMc,"FactorInExpr");

```

```

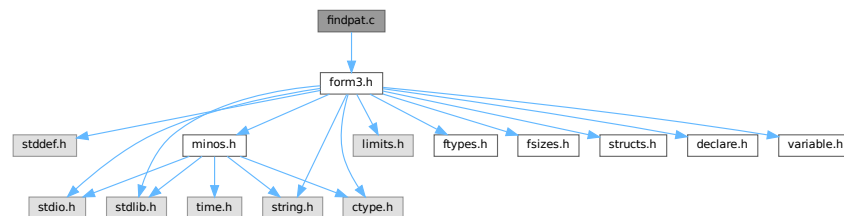
00858     return(0);
00859 onerror:
00860     AT.WorkPointer = oldwork;
00861     AR.CompressPointer = oldcpointer;
00862     MLOCK(ErrorMessageLock);
00863     MesCall("FactorInExpr");
00864     MUNLOCK(ErrorMessageLock);
00865     NumberFree(GCDbuffer,"FactorInExpr"); NumberFree(GCDbuffer2,"FactorInExpr");
00866     NumberFree(LCMbuffer,"FactorInExpr"); NumberFree(LCMB,"FactorInExpr");
00867     NumberFree(LCMc,"FactorInExpr");
00867     return(-1);
00868 }
00869
00870 /*
00871  #] FactorInExpr :
00872  */

```

4.35 findpat.c File Reference

```
#include "form3.h"
```

Include dependency graph for findpat.c:



Functions

- WORD [FindOnly](#) (PHEAD WORD *term, WORD *pattern)
- WORD [FindOnce](#) (PHEAD WORD *term, WORD *pattern)
- WORD [FindMulti](#) (PHEAD WORD *term, WORD *pattern)
- WORD [FindRest](#) (PHEAD WORD *term, WORD *pattern)

4.35.1 Detailed Description

Pattern matching of symbols and dotproducts. There are various routines because of the options in the id-statements like once, only, multi and many. These are among the oldest routines in FORM and that can be noticed, because the interplay with the function matching is not complete. When we match functions and halfway we fail we can backtrack properly. With the symbols, the dotproducts and the vectors (in [pattern.c](#)) there is no proper backtracking. Hence the routines here need still quite some work or may even have to be rewritten.

Definition in file [findpat.c](#).

4.35.2 Function Documentation

4.35.2.1 FindOnly()

```

WORD FindOnly (
    PHEAD WORD * term,
    WORD * pattern )

```

Definition at line 61 of file [findpat.c](#).

4.35.2.2 FindOnce()

```
WORD FindOnce (
    PHEAD WORD * term,
    WORD * pattern )
```

Definition at line [419](#) of file [findpat.c](#).

4.35.2.3 FindMulti()

```
WORD FindMulti (
    PHEAD WORD * term,
    WORD * pattern )
```

Definition at line [954](#) of file [findpat.c](#).

4.35.2.4 FindRest()

```
WORD FindRest (
    PHEAD WORD * term,
    WORD * pattern )
```

Definition at line [1070](#) of file [findpat.c](#).

4.36 findpat.c

[Go to the documentation of this file.](#)

```
00001
00013 /* #[] License : */
00014 /*
00015 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00016 * When using this file you are requested to refer to the publication
00017 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00018 * This is considered a matter of courtesy as the development was paid
00019 * for by FOM the Dutch physics granting agency and we would like to
00020 * be able to track its scientific use to convince FOM of its value
00021 * for the community.
00022 *
00023 * This file is part of FORM.
00024 *
00025 * FORM is free software: you can redistribute it and/or modify it under the
00026 * terms of the GNU General Public License as published by the Free Software
00027 * Foundation, either version 3 of the License, or (at your option) any later
00028 * version.
00029 *
00030 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00031 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00032 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00033 * details.
00034 *
00035 * You should have received a copy of the GNU General Public License along
00036 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00037 */
00038 /* #[] License : */
00039 /*
00040 #[] Includes : findpat.c
00041 */
00042
00043 #include "form3.h"
00044
00045 /*
00046 #[] Includes :
00047 #[] Patterns :
00048 #[] FindOnly : WORD FindOnly(term,pattern)
```

```

00049
00050     The current version doesn't scan function arguments yet. 10-Apr-1988
00051
00052     This routine searches for an exact match. This means in particular:
00053     1: x^# must match exactly.
00054     2: x^n? must have a single value for n that cannot be adapted.
00055
00056     When setp != 0 it points to a collection of sets
00057     A match can occur only if no object will be left that belongs
00058     to any of these sets.
00059  */
00060
00061 WORD FindOnly(PHEAD WORD *term, WORD *pattern)
00062 {
00063     GETBIDENTITY
00064     WORD *t, *m;
00065     WORD *tstop, *mstop;
00066     WORD *xstop, *ystop, *setp = AN.ForFindOnly;
00067     WORD n, nt, *p, nq;
00068     WORD older[NORMSIZE], *q, newval1, newval2, newval3;
00069     AN.UsedOtherFind = 0;
00070     m = pattern;
00071     mstop = m + *m;
00072     m++;
00073     t = term;
00074     t += *term - 1;
00075     tstop = t - ABS(*t) + 1;
00076     t = term;
00077     t++;
00078     while ( t < tstop && *t > DOTPRODUCT ) t += t[1];
00079     while ( m < mstop && *m > DOTPRODUCT ) m += m[1];
00080     if ( m < mstop ) { do {
00081  /*
00082         #[ SYMBOLS :
00083  */
00084         if ( *m == SYMBOL ) {
00085             ystop = m + m[1];
00086             m += 2;
00087             n = 0;
00088             p = older;
00089             if ( t < tstop ) while ( *t != SYMBOL ) {
00090                 t += t[1];
00091                 if ( t >= tstop ) {
00092 OnlyZer1:
00093                     do {
00094                         if ( *m >= 2*MAXPOWER ) return(0);
00095                         if ( m[1] >= 2*MAXPOWER ) nt = m[1];
00096                         else if ( m[1] <= -2*MAXPOWER ) nt = -m[1];
00097                         else return(0);
00098                         nt -= 2*MAXPOWER;
00099                         if ( CheckWild(BHEAD nt, SYMTONUM, 0, &newval3) ) return(0);
00100                         AddWild(BHEAD nt, SYMTONUM, 0);
00101                         m += 2;
00102                     } while ( m < ystop );
00103                     goto EndLoop;
00104                 }
00105             }
00106             else goto OnlyZer1;
00107             xstop = t + t[1];
00108             t += 2;
00109             do {
00110                 if ( *m == *t && t < xstop ) {
00111                     if ( m[1] == t[1] ) { m += 2; t += 2; }
00112                     else if ( m[1] >= 2*MAXPOWER ) {
00113                         nt = t[1];
00114                         nq = m[1];
00115                         goto OnlyL2;
00116                     }
00117                     else if ( m[1] <= -2*MAXPOWER ) {
00118                         nt = -t[1];
00119                         nq = -m[1];
00120                         nq -= 2*MAXPOWER;
00121                         if ( CheckWild(BHEAD nq, SYMTONUM, nt, &newval3) ) return(0);
00122                         AddWild(BHEAD nq, SYMTONUM, nt);
00123                         m += 2;
00124                         t += 2;
00125                     }
00126                     else {
00127                         *p++ = *t++; *p++ = *t++; n += 2;
00128                     }
00129                 }
00130                 else if ( *m >= 2*MAXPOWER ) {
00131                     while ( t < xstop ) { *p++ = *t++; *p++ = *t++; n += 2; }
00132                     nq = n;
00133                     p = older;
00134                     while ( nq > 0 ) {
00135                         if ( !CheckWild(BHEAD *m-2*MAXPOWER, SYMTOSYM, *p, &newval1) ) {

```

```

00136             if ( m[1] == p[1] ) {
00137                 AddWild(BHEAD *m-2*MAXPOWER, SYMTOSYM, newval1);
00138                 break;
00139             }
00140             else if ( m[1] >= 2*MAXPOWER && m[1] != *m ) {
00141                 if ( !CheckWild(BHEAD m[1]-2*MAXPOWER, SYMTONUM, p[1], &newval3) ) {
00142                     AddWild(BHEAD m[1]-2*MAXPOWER, SYMTONUM, p[1]);
00143                     AddWild(BHEAD *m-2*MAXPOWER, SYMTOSYM, newval1);
00144                     break;
00145                 }
00146             }
00147             else if ( m[1] <= -2*MAXPOWER && m[1] != -( *m ) ) {
00148                 if ( !CheckWild(BHEAD -m[1]-2*MAXPOWER, SYMTONUM, -p[1], &newval3) ) {
00149                     AddWild(BHEAD -m[1]-2*MAXPOWER, SYMTONUM, -p[1]);
00150                     AddWild(BHEAD *m-2*MAXPOWER, SYMTOSYM, newval1);
00151                     break;
00152                 }
00153             }
00154         }
00155         nq -= 2;
00156         p += 2;
00157     }
00158     if ( nq <= 0 ) return(0);
00159     nq -= 2;
00160     n -= 2;
00161     q = p + 2;
00162     while ( --nq >= 0 ) *p++ = *q++;
00163     m += 2;
00164 }
00165 else {
00166     if ( t >= xstop || *m < *t ) {
00167         if ( m[1] >= 2*MAXPOWER ) nt = m[1];
00168         else if ( m[1] <= -2*MAXPOWER ) nt = -m[1];
00169         else return(0);
00170         nt -= 2*MAXPOWER;
00171         if ( CheckWild(BHEAD nt, SYMTONUM, 0, &newval3) ) return(0);
00172         AddWild(BHEAD nt, SYMTONUM, 0);
00173         m += 2;
00174     }
00175     else {
00176         *p++ = *t++; *p++ = *t++; n += 2;
00177     }
00178 }
00179 } while ( m < ystop );
00180 if ( setp ) {
00181     while ( t < xstop ) { *p++ = *t++; *p++ = *t++; nt += 2; }
00182     p = older;
00183     while ( n > 0 ) {
00184         nq = setp[1] - 2;
00185         m = setp + 2;
00186         while ( --nq >= 0 ) {
00187             if ( Sets[*m].type != CSYMBOL ) { m++; continue; }
00188             t = SetElements + Sets[*m].first;
00189             tstop = SetElements + Sets[*m].last;
00190             while ( t < tstop ) {
00191                 if ( *t++ == *p ) return(0);
00192             }
00193             m++;
00194         }
00195         n -= 2;
00196         p += 2;
00197     }
00198 }
00199 return(1);
00200 }
00201 /*
00202     #] SYMBOLS :
00203     #[ DOTPRODUCTS :
00204 */
00205 else if ( *m == DOTPRODUCT ) {
00206     ystop = m + m[1];
00207     m += 2;
00208     n = 0;
00209     p = older;
00210     if ( t < tstop ) {
00211         if ( *t < DOTPRODUCT ) goto OnlyZer2;
00212         while ( *t > DOTPRODUCT ) {
00213             t += t[1];
00214             if ( t >= tstop || *t < DOTPRODUCT ) {
00215 OnlyZer2:
00216                 do {
00217                     if ( *m >= (AM.OffsetVector+WILDOFFSET)
00218                         || m[1] >= (AM.OffsetVector+WILDOFFSET) ) return(0);
00219                     if ( m[2] >= 2*MAXPOWER ) nq = m[2];
00220                     else if ( m[2] <= -2*MAXPOWER ) nq = -m[2];
00221                     else return(0);
00222                     nq -= 2*MAXPOWER;

```

```

00223         if ( CheckWild(BHEAD nq, SYMTONUM, 0, &newval3) ) return(0);
00224         AddWild(BHEAD nq, SYMTONUM, 0);
00225         m += 3;
00226     } while ( m < ystop );
00227     goto EndLoop;
00228 }
00229 }
00230 }
00231 else goto OnlyZer2;
00232 xstop = t + t[1];
00233 t += 2;
00234 do {
00235     if ( *m == *t && m[1] == t[1] && t < xstop ) {
00236         if ( t[2] != m[2] ) {
00237             if ( m[2] >= 2*MAXPOWER ) {
00238                 nq = m[2];
00239                 nt = t[2];
00240             }
00241             else if ( m[2] <= -2*MAXPOWER ) {
00242                 nq = -m[2];
00243                 nt = -t[2];
00244             }
00245             else return(0);
00246             nq -= 2*MAXPOWER;
00247             if ( CheckWild(BHEAD nq, SYMTONUM, nt, &newval3) ) return(0);
00248             AddWild(BHEAD nq, SYMTONUM, nt);
00249         }
00250         t += 3; m += 3;
00251     }
00252     else if ( *m >= (AM.OffsetVector+WILDOFFSET) ) {
00253         while ( t < xstop ) {
00254             *p++ = *t++; *p++ = *t++; *p++ = *t++; n += 3;
00255         }
00256         nq = n;
00257         p = older;
00258         while ( nq > 0 ) {
00259             if ( *m == m[1] ) {
00260                 if ( *p != p[1] ) goto NextInDot;
00261             }
00262             if ( !CheckWild(BHEAD *m-WILDOFFSET, VECTOVEC, *p, &newval1) &&
00263                 !CheckWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, p[1], &newval2) ) {
00264                 if ( p[2] == m[2] ) {
00265                     AddWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, newval2);
00266                     AddWild(BHEAD *m-WILDOFFSET, VECTOVEC, newval1);
00267                     break;
00268                 }
00269                 if ( m[2] >= 2*MAXPOWER ) {
00270                     if ( !CheckWild(BHEAD m[2]-2*MAXPOWER, SYMTONUM, p[2], &newval3) ) {
00271                         AddWild(BHEAD m[2]-2*MAXPOWER, SYMTONUM, newval3);
00272                         goto OnlyL9;
00273                     }
00274                 }
00275                 else if ( m[2] <= -2*MAXPOWER ) {
00276                     if ( !CheckWild(BHEAD -m[2]-2*MAXPOWER, SYMTONUM, -p[2], &newval3) ) {
00277                         AddWild(BHEAD -m[2]-2*MAXPOWER, SYMTONUM, -p[2]);
00278                         goto OnlyL9;
00279                     }
00280                 }
00281             }
00282             if ( !CheckWild(BHEAD *m-WILDOFFSET, VECTOVEC, p[1], &newval1) &&
00283                 !CheckWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, *p, &newval2) ) {
00284                 if ( p[2] == m[2] ) {
00285                     AddWild(BHEAD *m-WILDOFFSET, VECTOVEC, newval1);
00286                     AddWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, newval2);
00287                     break;
00288                 }
00289                 if ( m[2] >= 2*MAXPOWER ) {
00290                     if ( !CheckWild(BHEAD m[2]-2*MAXPOWER, SYMTONUM, p[2], &newval3) ) {
00291                         AddWild(BHEAD m[2]-2*MAXPOWER, SYMTONUM, p[2]);
00292                         goto OnlyL10;
00293                     }
00294                 }
00295                 else if ( m[2] <= -2*MAXPOWER ) {
00296                     if ( !CheckWild(BHEAD -m[2]-2*MAXPOWER, SYMTONUM, -p[2], &newval3) ) {
00297                         AddWild(BHEAD -m[2]-2*MAXPOWER, SYMTONUM, -p[2]);
00298                         goto OnlyL10;
00299                     }
00300                 }
00301             }
00302             NextInDot:
00303             p += 3; nq -= 3;
00304         }
00305         if ( nq <= 0 ) return(0);
00306         q = p+3;
00307         nq -= 3;
00308         n -= 3;
00309         while ( --nq >= 0 ) *p++ = *q++;

```

```

00310         m += 3;
00311     }
00312     else if ( m[1] >= (AM.OffsetVector+WILDOFFSET) ) {
00313         while ( *m >= *t && t < xstop ) {
00314             *p++ = *t++; *p++ = *t++; *p++ = *t++; n += 3;
00315         }
00316         nq = n;
00317         p = older;
00318         while ( nq > 0 ) {
00319             if ( *m == *p && !CheckWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, p[1], &newval1) ) {
00320                 if ( p[2] == m[2] ) {
00321                     AddWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, newval1);
00322                     break;
00323                 }
00324                 else if ( m[2] >= 2*MAXPOWER ) {
00325                     if ( !CheckWild(BHEAD m[2]-2*MAXPOWER, SYMTONUM, p[2], &newval3) ) {
00326                         AddWild(BHEAD m[2]-2*MAXPOWER, SYMTONUM, p[2]);
00327                         AddWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, newval1);
00328                         break;
00329                     }
00330                 }
00331                 else if ( m[2] <= -2*MAXPOWER ) {
00332                     if ( !CheckWild(BHEAD -m[2]-2*MAXPOWER, SYMTONUM, -p[2], &newval3) ) {
00333                         AddWild(BHEAD -m[2]-2*MAXPOWER, SYMTONUM, -p[2]);
00334                         AddWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, newval1);
00335                         break;
00336                     }
00337                 }
00338             }
00339             if ( *m == p[1] && !CheckWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, *p, &newval1) ) {
00340                 if ( p[2] == m[2] ) {
00341                     AddWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, newval1);
00342                     break;
00343                 }
00344                 if ( m[2] >= 2*MAXPOWER ) {
00345                     if ( !CheckWild(BHEAD m[2]-2*MAXPOWER, SYMTONUM, p[2], &newval3) ) {
00346                         AddWild(BHEAD m[2]-2*MAXPOWER, SYMTONUM, p[2]);
00347                         AddWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, newval1);
00348                         break;
00349                     }
00350                 }
00351                 else if ( m[2] <= -2*MAXPOWER ) {
00352                     if ( !CheckWild(BHEAD -m[2]-2*MAXPOWER, SYMTONUM, -p[2], &newval3) ) {
00353                         AddWild(BHEAD -m[2]-2*MAXPOWER, SYMTONUM, -p[2]);
00354                         AddWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, newval1);
00355                         break;
00356                     }
00357                 }
00358             }
00359             p += 3; nq -= 3;
00360         }
00361         if ( nq <= 0 ) return(0);
00362         q = p+3;
00363         nq -= 3;
00364         n -= 3;
00365         while ( --nq >= 0 ) *p++ = *q++;
00366         m += 3;
00367     }
00368     else {
00369         if ( t >= xstop || *m < *t || ( *m == *t && m[1] < t[1] ) ) {
00370             if ( m[2] > 2*MAXPOWER ) nt = m[2];
00371             else if ( m[2] <= -2*MAXPOWER ) nt = -m[2];
00372             else return(0);
00373             nt -= 2*MAXPOWER;
00374             if ( CheckWild(BHEAD nt, SYMTONUM, 0, &newval3) ) return(0);
00375             AddWild(BHEAD nt, SYMTONUM, 0);
00376             m += 3;
00377         }
00378         else {
00379             *p++ = *t++; *p++ = *t++; *p++ = *t++; n += 3;
00380         }
00381     }
00382 } while ( m < ystop );
00383 t = xstop;
00384 }
00385 /*
00386     #] DOTPRODUCTS :
00387 */
00388 else {
00389     MLOCK(ErrorMessageLock);
00390     MesPrint("Error in pattern(1)");
00391     MUNLOCK(ErrorMessageLock);
00392     Terminate(-1);
00393 }
00394 EndLoop;
00395 } while ( m < mstop ); }
00396 if ( setp ) {

```

```

00397 /*
00398     while ( t < tstop && *t > SYMBOL ) t += t[1];
00399     if ( t < tstop && setp[1] > 2 ) return(0);
00400 */
00401     /* There were nonempty sets */
00402     /* Empty sets are rejected by the compiler */
00403 }
00404 return(1);
00405 }
00406
00407 /*
00408     #] FindOnly :
00409     #[ FindOnce :          WORD FindOnce(term,pattern)
00410
00411     Searches for a single match in term. The difference with multi
00412     lies mainly in the fact that here functions may occur.
00413     The functions have not been implemented yet. (10-Apr-1988)
00414     Wildcard powers are adjustable. The value closer to zero is taken.
00415     Positive and negative gives (o surprise) zero.
00416
00417 */
00418
00419 WORD FindOnce(PHEAD WORD *term, WORD *pattern)
00420 {
00421     GETBIDENTITY
00422     WORD *t, *m;
00423     WORD *tstop, *mstop;
00424     WORD *xstop, *ystop;
00425     WORD n, nt, *p, nq, mt, ch;
00426     WORD older[2*NORMSIZE], *q, newval1, newval2, newval3;
00427     AN.UsedOtherFind = 0;
00428     m = pattern;
00429     mstop = m + *m;
00430     m++;
00431     t = term;
00432     t += *term - 1;
00433     tstop = t - ABS(*t) + 1;
00434     t = term;
00435     t++;
00436     while ( t < tstop && *t > DOTPRODUCT ) t += t[1];
00437     while ( m < mstop && *m > DOTPRODUCT ) m += m[1];
00438     if ( m < mstop ) { do {
00439 /*
00440         #[ SYMBOLS :
00441 */
00442         if ( *m == SYMBOL ) {
00443             ystop = m + m[1];
00444             m += 2;
00445             n = 0;
00446             p = older;
00447             if ( t < tstop ) while ( *t != SYMBOL ) {
00448                 t += t[1];
00449                 if ( t >= tstop ) {
00450 TryZero:
00451                     do {
00452                         if ( *m >= 2*MAXPOWER ) return(0);
00453                         if ( m[1] >= 2*MAXPOWER ) nt = m[1];
00454                         else if ( m[1] <= -2*MAXPOWER ) nt = -m[1];
00455                         else return(0);
00456                         nt -= 2*MAXPOWER;
00457                         if ( ( ch = CheckWild(BHEAD nt,SYMTONUM,0,&newval3) ) != 0 ) {
00458                             if ( ch > 1 ) return(0);
00459                             if ( AN.oldtype != SYMTONUM ) return(0);
00460                             if ( *AN.MaskPointer == 2 ) return(0);
00461                         }
00462                         AddWild(BHEAD nt,SYMTONUM,0);
00463                         m += 2;
00464                     } while ( m < ystop );
00465                     goto EndLoop;
00466                 }
00467             }
00468             else goto TryZero;
00469             xstop = t + t[1];
00470             t += 2;
00471             do {
00472                 if ( *m == *t && t < xstop ) {
00473                     nt = t[1];
00474                     mt = m[1];
00475                     if ( ( mt > 0 && mt <= nt ) ||
00476                         ( mt < 0 && mt >= nt ) ) { m += 2; t += 2; }
00477                     else if ( mt >= 2*MAXPOWER ) goto OnceL2;
00478                     else if ( mt <= -2*MAXPOWER ) {
00479                         nt = -nt;
00480                         mt = -mt;
00481                     }
00482                     OnceL2:
00483                     mt -= 2*MAXPOWER;
00484                     if ( ( ch = CheckWild(BHEAD mt,SYMTONUM,nt,&newval3) ) != 0 ) {
00485                         if ( ch > 1 ) return(0);

```



```

00484         if ( AN.oldtype != SYMTONUM ) return(0);
00485         if ( AN.oldvalue <= 0 ) {
00486             if ( nt < AN.oldvalue ) nt = AN.oldvalue;
00487             else {
00488                 if ( *AN.MaskPointer == 2 ) return(0);
00489                 if ( nt > 0 ) nt = 0;
00490             }
00491         }
00492         if ( AN.oldvalue >= 0 ) {
00493             if ( nt > AN.oldvalue ) nt = AN.oldvalue;
00494             else {
00495                 if ( *AN.MaskPointer == 2 ) return(0);
00496                 if ( nt < 0 ) nt = 0;
00497             }
00498         }
00499     }
00500     AddWild(BHEAD mt, SYMTONUM, nt);
00501     m += 2;
00502     t += 2;
00503 }
00504 else {
00505     *p++ = *t++; *p++ = *t++; n += 2;
00506 }
00507 }
00508 else if ( *m >= 2*MAXPOWER ) {
00509     while ( t < xstop ) { *p++ = *t++; *p++ = *t++; n += 2; }
00510     nq = n;
00511     p = older;
00512     while ( nq > 0 ) {
00513         nt = p[1];
00514         if ( !CheckWild(BHEAD *m-2*MAXPOWER, SYMTOSYM, *p, &newvall) ) {
00515             mt = m[1];
00516             if ( ( mt > 0 && mt <= nt ) ||
00517                 ( mt < 0 && mt >= nt ) ) {
00518                 AddWild(BHEAD *m-2*MAXPOWER, SYMTOSYM, newvall);
00519                 break;
00520             }
00521             else if ( mt >= 2*MAXPOWER && mt != *m ) {
00522                 mt -= 2*MAXPOWER;
00523                 if ( ( ch = CheckWild(BHEAD mt, SYMTONUM, nt, &newval3) ) != 0 ) {
00524                     if ( ch > 1 ) return(0);
00525                     if ( AN.oldtype == SYMTONUM ) {
00526                         if ( AN.oldvalue >= 0 ) {
00527                             if ( nt > AN.oldvalue ) nt = AN.oldvalue;
00528                             else {
00529                                 if ( *AN.MaskPointer == 2 ) return(0);
00530                                 if ( nt < 0 ) nt = 0;
00531                             }
00532                         }
00533                         else {
00534                             if ( nt < AN.oldvalue ) nt = AN.oldvalue;
00535                             else {
00536                                 if ( *AN.MaskPointer == 2 ) return(0);
00537                                 if ( nt > 0 ) nt = 0;
00538                             }
00539                         }
00540                     }
00541                     AddWild(BHEAD mt, SYMTONUM, nt);
00542                     AddWild(BHEAD *m-2*MAXPOWER, SYMTOSYM, newvall);
00543                     break;
00544                 }
00545             }
00546             else {
00547                 AddWild(BHEAD mt, SYMTONUM, nt);
00548                 AddWild(BHEAD *m-2*MAXPOWER, SYMTOSYM, newvall);
00549                 break;
00550             }
00551             }
00552             else if ( mt <= -2*MAXPOWER && mt != -( *m ) ) {
00553                 nt = -nt;
00554                 mt = -mt;
00555                 goto OnceL4a;
00556             }
00557             }
00558             nq -= 2;
00559             p += 2;
00560         }
00561         if ( nq <= 0 ) return(0);
00562         nq -= 2;
00563         n -= 2;
00564         q = p + 2;
00565         while ( --nq >= 0 ) *p++ = *q++;
00566         m += 2;
00567     }
00568     else {
00569         if ( t >= xstop || *m < *t ) {
00570             if ( m[1] >= 2*MAXPOWER ) nt = m[1];
00571             else if ( m[1] <= -2*MAXPOWER ) nt = -m[1];

```

```

00571         else return(0);
00572         nt -= 2*MAXPOWER;
00573         if ( ( ch = CheckWild(BHEAD nt,SYMTONUM,0,&newval3) ) != 0 ) {
00574             if ( ch > 1 ) return(0);
00575             if ( AN.oldtype != SYMTONUM ) return(0);
00576             if ( *AN.MaskPointer == 2 ) return(0);
00577         }
00578         AddWild(BHEAD nt,SYMTONUM,0);
00579         m += 2;
00580     }
00581     else {
00582         *p++ = *t++; *p++ = *t++; n += 2;
00583     }
00584 }
00585 } while ( m < ystop );
00586 }
00587 /*
00588     #] SYMBOLS :
00589     #[ DOTPRODUCTS :
00590 */
00591 else if ( *m == DOTPRODUCT ) {
00592     ystop = m + m[1];
00593     m += 2;
00594     n = 0;
00595     p = older;
00596     if ( t < tstop ) {
00597         if ( *t < DOTPRODUCT ) goto OnceOp;
00598         while ( *t > DOTPRODUCT ) {
00599             t += t[1];
00600             if ( t >= tstop || *t < DOTPRODUCT ) {
00601 OnceOp:
00602                 do {
00603                     if ( *m >= (AM.OffsetVector+WILDOFFSET)
00604                         || m[1] >= (AM.OffsetVector+WILDOFFSET) ) return(0);
00605                     if ( m[2] >= 2*MAXPOWER ) {
00606                         nq = m[2] - 2*MAXPOWER;
00607                     }
00608                     else if ( m[2] <= -2*MAXPOWER ) {
00609                         nq = -m[2] - 2*MAXPOWER;
00610                     }
00611                     else return(0);
00612                     if ( CheckWild(BHEAD nq,SYMTONUM,(WORD)0,&newval3) ) {
00613                         if ( AN.oldtype != SYMTONUM ) return(0);
00614                         if ( *AN.MaskPointer == 2 ) return(0);
00615                     }
00616                     AddWild(BHEAD nq,SYMTONUM,(WORD)0);
00617                     m += 3;
00618                 } while ( m < ystop );
00619                 goto EndLoop;
00620             }
00621         }
00622     }
00623     else goto OnceOp;
00624     xstop = t + t[1];
00625     t += 2;
00626     do {
00627         if ( *m == *t && m[1] == t[1] && t < xstop ) {
00628             nt = t[2];
00629             mt = m[2];
00630 /*
00631             if ( ( nt > 0 && nt < mt ) ||
00632                 ( nt < 0 && nt > mt ) ) {
00633                 if ( mt <= -2*MAXPOWER ) {
00634                     mt = -mt;
00635                     nt = -nt;
00636                 }
00637             else if ( mt < 2*MAXPOWER ) return(0);
00638             mt -= 2*MAXPOWER;
00639             if ( CheckWild(BHEAD mt,SYMTONUM,nt,&newval3) ) {
00640                 if ( AN.oldtype != SYMTONUM ) return(0);
00641                 if ( AN.oldvalue <= 0 ) {
00642                     if ( nt < AN.oldvalue ) nt = AN.oldvalue;
00643                     else {
00644                         if ( *AN.MaskPointer == 2 ) return(0);
00645                         if ( nt > 0 ) nt = 0;
00646                     }
00647                 }
00648                 if ( AN.oldvalue >= 0 ) {
00649                     if ( nt > AN.oldvalue ) nt = AN.oldvalue;
00650                     else {
00651                         if ( *AN.MaskPointer == 2 ) return(0);
00652                         if ( nt < 0 ) nt = 0;
00653                     }
00654                 }
00655             }
00656             AddWild(BHEAD mt,SYMTONUM,nt);
00657             m += 3; t += 3;

```

```

00658     }
00659     else if ( ( nt > 0 && nt >= mt && mt > -2*MAXPOWER )
00660     || ( nt < 0 && nt <= mt && mt < 2*MAXPOWER ) ) {
00661         m += 3; t += 3;
00662     }
00663     /*
00664     if ( ( mt > 0 && mt <= nt ) ||
00665         ( mt < 0 && mt >= nt ) ) { m += 3; t += 3; }
00666     else if ( mt >= 2*MAXPOWER ) goto OnceL7;
00667     else if ( mt <= -2*MAXPOWER ) {
00668         nt = -nt;
00669         mt = -mt;
00670     OnceL7:
00671         if ( CheckWild(BHEAD mt, SYMTONUM, nt, &newval3) ) {
00672             if ( AN.oldtype != SYMTONUM ) return(0);
00673             if ( AN.oldvalue <= 0 ) {
00674                 if ( nt < AN.oldvalue ) nt = AN.oldvalue;
00675                 else {
00676                     if ( *AN.MaskPointer == 2 ) return(0);
00677                     if ( nt > 0 ) nt = 0;
00678                 }
00679             }
00680             if ( AN.oldvalue >= 0 ) {
00681                 if ( nt > AN.oldvalue ) nt = AN.oldvalue;
00682                 else {
00683                     if ( *AN.MaskPointer == 2 ) return(0);
00684                     if ( nt < 0 ) nt = 0;
00685                 }
00686             }
00687         }
00688         AddWild(BHEAD mt, SYMTONUM, nt);
00689         m += 3;
00690         t += 3;
00691     }
00692     else {
00693         *p++ = *t++; *p++ = *t++; *p++ = *t++; n += 3;
00694     }
00695 }
00696 else if ( *m >= (AM.OffsetVector+WILDOFFSET) ) {
00697     while ( t < xstop ) {
00698         *p++ = *t++; *p++ = *t++; *p++ = *t++; n += 3;
00699     }
00700     nq = n;
00701     p = older;
00702     while ( nq > 0 ) {
00703         if ( *m == m[1] ) {
00704             if ( *p != p[1] ) goto NextInDot;
00705         }
00706         if ( !CheckWild(BHEAD *m-WILDOFFSET, VECTOVEC, *p, &newval1) &&
00707             !CheckWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, p[1], &newval2) ) {
00708             nt = p[2];
00709             mt = m[2];
00710             if ( ( mt > 0 && nt >= mt ) ||
00711                 ( mt < 0 && nt <= mt ) ) {
00712     OnceL9:
00713                 AddWild(BHEAD *m-WILDOFFSET, VECTOVEC, newval1);
00714                 AddWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, newval2);
00715                 break;
00716             }
00717     OnceL9a:
00718             if ( mt >= 2*MAXPOWER ) {
00719                 mt -= 2*MAXPOWER;
00720                 if ( CheckWild(BHEAD mt, SYMTONUM, nt, &newval3) ) {
00721                     if ( AN.oldtype == SYMTONUM ) {
00722                         if ( AN.oldvalue >= 0 ) {
00723                             if ( nt > AN.oldvalue ) nt = AN.oldvalue;
00724                             else {
00725                                 if ( *AN.MaskPointer == 2 ) return(0);
00726                                 if ( nt < 0 ) nt = 0;
00727                             }
00728                         }
00729                         else {
00730                             if ( nt < AN.oldvalue ) nt = AN.oldvalue;
00731                             else {
00732                                 if ( *AN.MaskPointer == 2 ) return(0);
00733                                 if ( nt > 0 ) nt = 0;
00734                             }
00735                         }
00736                     }
00737                     AddWild(BHEAD mt, SYMTONUM, nt);
00738                     goto OnceL9;
00739                 }
00740             }
00741             else {
00742                 AddWild(BHEAD mt, SYMTONUM, nt);
00743                 goto OnceL9;
00744             }
00745         }
00746     }
00747     else if ( mt <= -2*MAXPOWER ) {
00748         mt = -mt;

```

```

00745             nt = -nt;
00746             goto OnceL9a;
00747         }
00748     }
00749     if ( !CheckWild(BHEAD *m-WILDOFFSET, VECTOVEC, p[1], &newval1) &&
00750         !CheckWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, *p, &newval2) ) {
00751         nt = p[2];
00752         mt = m[2];
00753         if ( ( mt > 0 && nt >= mt ) ||
00754             ( mt < 0 && nt <= mt ) ) {
00755             OnceL10:
00756                 AddWild(BHEAD *m-WILDOFFSET, VECTOVEC, newval1);
00757                 AddWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, newval2);
00758                 break;
00759         }
00760         if ( mt >= 2*MAXPOWER ) {
00761             OnceL10a:
00762                 mt -= 2*MAXPOWER;
00763                 if ( CheckWild(BHEAD mt, SYMTONUM, nt, &newval3) ) {
00764                     if ( AN.oldtype == SYMTONUM ) {
00765                         if ( AN.oldvalue >= 0 ) {
00766                             if ( nt > AN.oldvalue ) nt = AN.oldvalue;
00767                             else {
00768                                 if ( *AN.MaskPointer == 2 ) return(0);
00769                                 if ( nt < 0 ) nt = 0;
00770                             }
00771                         }
00772                         else {
00773                             if ( nt < AN.oldvalue ) nt = AN.oldvalue;
00774                             else {
00775                                 if ( *AN.MaskPointer == 2 ) return(0);
00776                                 if ( nt > 0 ) nt = 0;
00777                             }
00778                         }
00779                     }
00780                     AddWild(BHEAD mt, SYMTONUM, nt);
00781                     goto OnceL10;
00782                 }
00783             }
00784             else {
00785                 AddWild(BHEAD mt, SYMTONUM, nt);
00786                 goto OnceL10;
00787             }
00788         }
00789         else if ( mt <= -2*MAXPOWER ) {
00790             mt = -mt;
00791             nt = -nt;
00792             goto OnceL10a;
00793         }
00794     }
00795     NextInDot:
00796     p += 3; nq -= 3;
00797     }
00798     if ( nq <= 0 ) return(0);
00799     else {
00800         q = p+3;
00801         nq -= 3;
00802         n -= 3;
00803         while ( --nq >= 0 ) *p++ = *q++;
00804     }
00805     m += 3;
00806 }
00807 else if ( m[1] >= (AM.OffsetVector+WILDOFFSET) ) {
00808     while ( *m >= *t && t < xstop ) {
00809         *p++ = *t++; *p++ = *t++; *p++ = *t++; n += 3;
00810     }
00811     nq = n;
00812     p = older;
00813     while ( nq > 0 ) {
00814         if ( *m == *p && !CheckWild(BHEAD m[1]-WILDOFFSET,
00815             VECTOVEC, p[1], &newval1) ) {
00816             nt = p[2];
00817             mt = m[2];
00818             if ( ( mt > 0 && nt >= mt ) ||
00819                 ( mt < 0 && nt <= mt ) ) {
00820                 AddWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, newval1);
00821                 break;
00822             }
00823         }
00824         else if ( mt >= 2*MAXPOWER ) {
00825             OnceL7a:
00826                 mt -= 2*MAXPOWER;
00827                 if ( CheckWild(BHEAD mt, SYMTONUM, nt, &newval3) ) {
00828                     if ( AN.oldtype == SYMTONUM ) {
00829                         if ( AN.oldvalue >= 0 ) {
00830                             if ( nt > AN.oldvalue ) nt = AN.oldvalue;
00831                             else {
00832                                 if ( *AN.MaskPointer == 2 ) return(0);
00833                                 if ( nt < 0 ) nt = 0;
00834                             }
00835                         }
00836                         else {
00837                             if ( nt < AN.oldvalue ) nt = AN.oldvalue;
00838                             else {
00839                                 if ( *AN.MaskPointer == 2 ) return(0);
00840                                 if ( nt > 0 ) nt = 0;
00841                             }
00842                         }
00843                     }
00844                     AddWild(BHEAD mt, SYMTONUM, nt);
00845                     goto OnceL10;
00846                 }
00847             }
00848             else {
00849                 AddWild(BHEAD mt, SYMTONUM, nt);
00850                 goto OnceL10;
00851             }
00852         }
00853     }
00854 }

```

```

00832             if ( nt < AN.oldvalue ) nt = AN.oldvalue;
00833         else {
00834             if ( *AN.MaskPointer == 2 ) return(0);
00835             if ( nt > 0 ) nt = 0;
00836         }
00837     }
00838     AddWild(BHEAD mt, SYMTONUM, nt);
00839     AddWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, newval1);
00840     break;
00841 }
00842 }
00843 else {
00844     AddWild(BHEAD mt, SYMTONUM, nt);
00845     AddWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, newval1);
00846     break;
00847 }
00848 }
00849 else if ( mt <= -2*MAXPOWER ) {
00850     mt = -mt;
00851     nt = -nt;
00852     goto OnceL7a;
00853 }
00854 }
00855 if ( *m == p[1] && !CheckWild(BHEAD m[1]-WILDOFFSET,
00856 VECTOVEC, *p, &newval1) ) {
00857     nt = p[2];
00858     mt = m[2];
00859     if ( ( mt > 0 && nt >= mt ) ||
00860         ( mt < 0 && nt <= mt ) ) {
00861         AddWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, newval1);
00862         break;
00863     }
00864     if ( mt >= 2*MAXPOWER ) {
00865         mt -= 2*MAXPOWER;
00866         OnceL8a:
00867         if ( CheckWild(BHEAD mt, SYMTONUM, nt, &newval3) ) {
00868             if ( AN.oldtype == SYMTONUM ) {
00869                 if ( AN.oldvalue >= 0 ) {
00870                     if ( nt > AN.oldvalue ) nt = AN.oldvalue;
00871                 }
00872                 else {
00873                     if ( *AN.MaskPointer == 2 ) return(0);
00874                     if ( nt < 0 ) nt = 0;
00875                 }
00876             }
00877             else {
00878                 if ( nt < AN.oldvalue ) nt = AN.oldvalue;
00879                 else {
00880                     if ( *AN.MaskPointer == 2 ) return(0);
00881                     if ( nt > 0 ) nt = 0;
00882                 }
00883             }
00884             AddWild(BHEAD mt, SYMTONUM, nt);
00885             AddWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, newval1);
00886             break;
00887         }
00888     }
00889     else {
00890         AddWild(BHEAD mt, SYMTONUM, nt);
00891         AddWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, newval1);
00892         break;
00893     }
00894 }
00895 else if ( mt < -2*MAXPOWER ) {
00896     mt = -mt;
00897     nt = -nt;
00898     goto OnceL8a;
00899 }
00900 }
00901 p += 3; nq -= 3;
00902 if ( nq <= 0 ) return(0);
00903 q = p+3;
00904 nq -= 3;
00905 n -= 3;
00906 while ( --nq >= 0 ) *p++ = *q++;
00907 m += 3;
00908 }
00909 else {
00910     if ( t >= xstop || *m < *t || ( *m == *t && m[1] < t[1] ) ) {
00911         if ( m[2] >= 2*MAXPOWER ) nt = m[2];
00912         else if ( m[2] <= -2*MAXPOWER ) nt = -m[2];
00913         else return(0);
00914         nt -= 2*MAXPOWER;
00915         if ( CheckWild(BHEAD nt, SYMTONUM, 0, &newval3) ) {
00916             if ( AN.oldtype != SYMTONUM ) return(0);
00917             if ( *AN.MaskPointer == 2 ) return(0);
00918         }
00919         AddWild(BHEAD nt, SYMTONUM, 0);

```

```

00919             m += 3;
00920         }
00921         else {
00922             *p++ = *t++; *p++ = *t++; *p++ = *t++; n += 3;
00923         }
00924     }
00925     } while ( m < ystop );
00926     t = xstop;
00927 }
00928 /*
00929     #] DOTPRODUCTS :
00930 */
00931     else {
00932         MLOCK(ErrorMessageLock);
00933         MesPrint("Error in pattern(2)");
00934         MUNLOCK(ErrorMessageLock);
00935         Terminate(-1);
00936     }
00937 EndLoop;;
00938     } while ( m < mstop ); }
00939     else {
00940         return(-1);
00941     }
00942     return(1);
00943 }
00944
00945 /*
00946     #] FindOnce :
00947     #[ FindMulti :      WORD FindMulti(term,pattern)
00948
00949     Note that multi cannot deal with wildcards. Those patterns revert
00950     to many which gives subsequent calls to once.
00951
00952 */
00953
00954 WORD FindMulti(PHEAD WORD *term, WORD *pattern)
00955 {
00956     GETBIDENTITY
00957     WORD *t, *m, *p;
00958     WORD *tstop, *mstop;
00959     WORD *xstop, *ystop;
00960     WORD mt, power, n, nq;
00961     WORD older[2*NORMSIZE], *q, newvall;
00962     AN.UsedOtherFind = 0;
00963     m = pattern;
00964     mstop = m + *m;
00965     m++;
00966     t = term;
00967     t += *term - 1;
00968     tstop = t - ABS(*t) + 1;
00969     t = term;
00970     t++;
00971     while ( t < tstop && *t > DOTPRODUCT ) t += t[1];
00972     while ( m < mstop && *m > DOTPRODUCT ) m += m[1];
00973     power = -1;          /* No power yet */
00974     if ( m < mstop ) { do {
00975 /*
00976         #] SYMBOLS :
00977 */
00978         if ( *m == SYMBOL ) {
00979             ystop = m + m[1];
00980             m += 2;
00981             if ( t >= tstop ) return(0);
00982             while ( *t != SYMBOL ) { t += t[1]; if ( t >= tstop ) return(0); }
00983             xstop = t + t[1];
00984             t += 2;
00985             p = older;
00986             n = 0;
00987             do {
00988                 if ( *m >= 2*MAXPOWER ) {
00989                     while ( t < xstop ) { *p++ = *t++; *p++ = *t++; n += 2; }
00990                     nq = n;
00991                     p = older;
00992                     while ( nq > 0 ) {
00993                         if ( !CheckWild(BHEAD *m-2*MAXPOWER, SYMTOSYM,*p,&newvall) ) {
00994                             mt = p[1]/m[1];
00995                             if ( mt > 0 ) {
00996                                 if ( power < 0 || mt < power ) power = mt;
00997                                 AddWild(BHEAD *m-2*MAXPOWER, SYMTOSYM,newvall);
00998                                 break;
00999                             }
01000                         }
01001                         nq -= 2;
01002                         p += 2;
01003                     }
01004                     if ( nq <= 0 ) return(0);
01005                     nq -= 2;

```

```

01006             n -= 2;
01007             q = p + 2;
01008             while ( --nq >= 0 ) *p++ = *q++;
01009             m += 2;
01010         }
01011         else if ( t >= xstop ) return(0);
01012         else if ( *m == *t ) {
01013             if ( ( mt = t[1]/m[1] ) <= 0 ) return(0);
01014             if ( power < 0 || mt < power ) power = mt;
01015             m += 2;
01016             t += 2;
01017         }
01018         else if ( *m < *t ) return(0);
01019         else { *p++ = *t++; *p++ = *t++; n += 2; }
01020     } while ( m < ystop );
01021 }
01022 /*
01023     #] SYMBOLS :
01024     #[ DOTPRODUCTS :
01025 */
01026     else if ( *m == DOTPRODUCT ) {
01027         ystop = m + m[1];
01028         m += 2;
01029         if ( t >= tstop ) return(0);
01030         while ( *t != DOTPRODUCT ) { t += t[1]; if ( t >= tstop ) return(0); }
01031         xstop = t + t[1];
01032         t += 2;
01033         do {
01034             if ( t >= xstop ) return(0);
01035             if ( *t == *m ) {
01036                 if ( t[1] == m[1] ) {
01037                     if ( ( mt = t[2]/m[2] ) <= 0 ) return(0);
01038                     if ( power < 0 || mt < power ) power = mt;
01039                     m += 3;
01040                 }
01041                 else if ( t[1] > m[1] ) return(0);
01042             }
01043             else if ( *t > *m ) return(0);
01044             t += 3;
01045         } while ( m < ystop );
01046         t = xstop;
01047     }
01048 /*
01049     #] DOTPRODUCTS :
01050 */
01051     else {
01052         MLOCK(ErrorMessageLock);
01053         MesPrint("Error in pattern(3)");
01054         MUNLOCK(ErrorMessageLock);
01055         Terminate(-1);
01056     }
01057 } while ( m < mstop ); }
01058 if ( power < 0 ) power = 0;
01059 return(power);
01060 }
01061
01062 /*
01063     #] FindMulti :
01064     #[ FindRest :          WORD FindRest(term,pattern)
01065
01066     This routine scans for anything but dotproducts and symbols.
01067
01068 */
01069
01070 WORD FindRest(PHEAD WORD *term, WORD *pattern)
01071 {
01072     GETBIDENTITY
01073     WORD *t, *m, *tt, wild, regular;
01074     WORD *tstop, *mstop;
01075     WORD *xstop, *ystop;
01076     WORD n, *p, nq;
01077     WORD older[NORMSIZE], *q, newvall, newval2;
01078     int i, ntw;
01079     AN.UsedOtherFind = 0;
01080     AN.findTerm = term; AN.findPattern = pattern;
01081     m = AN.WildValue;
01082     i = (m[-SUBEXPSIZE+1]-SUBEXPSIZE)/4;
01083     ntw = 0;
01084     while ( i > 0 ) {
01085         if ( m[0] == ARGTOARG ) ntw++;
01086         m += m[1];
01087         i--;
01088     }
01089     t = term;
01090     t += *term - 1;
01091     tstop = t - ABS(*t) + 1;
01092     t = term;

```

```

01093     t++; p = t;
01094     while ( t < tstop && *t > DOTPRODUCT ) t += t[1];
01095     tstop = t;
01096     t = p;
01097     m = pattern;
01098     mstop = m + *m;
01099     m++;
01100     p = m;
01101     while ( m < mstop && *m > DOTPRODUCT ) m += m[1];
01102     mstop = m;
01103     m = p;
01104     if ( m < mstop ) {
01105     do {
01106     /*
01107         #[ FUNCTIONS :
01108     */
01109         if ( *m >= FUNCTION ) {
01110             if ( *mstop > 5 && !MatchIsPossible(pattern,term) ) return(0);
01111             ystop = m;
01112             n = 0;
01113             do {
01114                 m += m[1]; n++;
01115             } while ( m < mstop && *m >= FUNCTION );
01116             AT.WorkPointer += n;
01117             while ( t < tstop && *t == SUBEXPRESSION ) t += t[1];
01118             tt = xstop = t;
01119             nq = 0;
01120             while ( t < tstop && ( *t >= FUNCTION || *t == SUBEXPRESSION ) ) {
01121                 if ( *t != SUBEXPRESSION ) {
01122                     nq++;
01123                     if ( functions[*t-FUNCTION].commute ) tt = t + t[1];
01124                 }
01125                 t += t[1];
01126             }
01127             if ( nq < n ) return(0);
01128             AN.terstart = term;
01129             AN.terstop = t;
01130             AN.terfirstcomm = tt;
01131             AN.patstop = m;
01132             AN.NumTotWildArgs = ntw;
01133             if ( !ScanFunctions(BHEAD ystop,xstop,0) ) return(0);
01134         }
01135     /*
01136         #[ FUNCTIONS :
01137         #[ VECTORS :
01138     */
01139     else if ( *m == VECTOR ) {
01140         while ( t < tstop && *t != VECTOR ) t += t[1];
01141         if ( t >= tstop ) return(0);
01142         xstop = t + t[1];
01143         ystop = m + m[1];
01144         t += 2;
01145         m += 2;
01146         n = 0;
01147         p = older;
01148         do {
01149             if ( *m == *t && m[1] == t[1] && t < xstop ) {
01150                 m += 2; t += 2;
01151             }
01152             else if ( *m >= (AM.OffsetVector+WILDOFFSET) ) {
01153                 if ( t < xstop ) {
01154                     p = older + n;
01155                     do { *p++ = *t++; n++; } while ( t < xstop );
01156                 }
01157                 p = older;
01158                 nq = n;
01159                 if ( ( m[1] < (AM.OffsetIndex+WILDOFFSET) )
01160                     || ( m[1] >= (AM.OffsetIndex+2*WILDOFFSET) ) ) {
01161                     while ( nq > 0 ) {
01162                         if ( m[1] == p[1] ) {
01163                             if ( !CheckWild(BHEAD *m-WILDOFFSET,VECTOVEC,*p,&newval1) ) {
01164 RestL11: AddWild(BHEAD *m-WILDOFFSET,VECTOVEC,newval1);
01165                             break;
01166                         }
01167                     }
01168                     p += 2;
01169                     nq -= 2;
01170                 }
01171             }
01172             else { /* Double wildcard */
01173                 while ( nq > 0 ) {
01174                     if ( !CheckWild(BHEAD *m-WILDOFFSET,VECTOVEC,*p,&newval1) &&
01175                         !CheckWild(BHEAD m[1]-WILDOFFSET,INDTOIND,p[1],&newval2) ) {
01176                         AddWild(BHEAD m[1]-WILDOFFSET,INDTOIND,newval2);
01177                         goto RestL11;
01178                     }
01179                     p += 2;

```



```

01180             nq -= 2;
01181         }
01182     }
01183     if ( nq > 0 ) {
01184         nq -= 2; q = p + 2; n -= 2;
01185         while ( --nq >= 0 ) *p++ = *q++;
01186     }
01187     else return(0);
01188     m += 2;
01189 }
01190 else if ( ( *m <= *t )
01191 && ( m[1] >= (AM.OffsetIndex + WILDOFFSET) )
01192 && ( m[1] < (AM.OffsetIndex + 2*WILDOFFSET) ) ) {
01193     if ( *m == *t && t < xstop ) {
01194         p = older;
01195         p += n;
01196         *p++ = *t++;
01197         *p++ = *t++;
01198         n += 2;
01199     }
01200     p = older;
01201     nq = n;
01202     while ( nq > 0 ) {
01203         if ( *m == *p ) {
01204             if ( !CheckWild(BHEAD m[1]-WILDOFFSET,INDTOIND,p[1],&newvall) ) {
01205                 AddWild(BHEAD m[1]-WILDOFFSET,INDTOIND,newvall);
01206                 break;
01207             }
01208         }
01209         p += 2;
01210         nq -= 2;
01211     }
01212     if ( nq > 0 ) {
01213         nq -= 2; q = p + 2; n -= 2;
01214         while ( --nq >= 0 ) *p++ = *q++;
01215     }
01216     else return(0);
01217     m += 2;
01218 }
01219 else {
01220     if ( t >= xstop ) return(0);
01221     *p++ = *t++; *p++ = *t++; n += 2;
01222 }
01223 } while ( m < ystop );
01224 }
01225 /*
01226 #] VECTORS :
01227 #[ INDICES :
01228 */
01229 else if ( *m == INDEX ) {
01230 /*
01231     This needs only to say that there is a match, after matching
01232     a 'wildcard'. This has to be prepared in TestMatch. The C->rhs
01233     should provide the replacement inside the prototype!
01234     Next question: id,p=q/2+r/2
01235 */
01236     while ( *t != INDEX ) { t += t[1]; if ( t >= tstop ) return(0); }
01237     xstop = t + t[1];
01238     ystop = m + m[1];
01239     t += 2;
01240     m += 2;
01241     n = 0;
01242     p = older;
01243     do {
01244         if ( *m == *t && t < xstop && m < ystop ) {
01245             t++; m++;
01246         }
01247         else if ( ( *m >= (AM.OffsetIndex+WILDOFFSET) )
01248 && ( *m < (AM.OffsetIndex+2*WILDOFFSET) ) ) {
01249             while ( t < xstop ) {
01250                 *p++ = *t++; n++;
01251             }
01252             if ( !n ) return(0);
01253             nq = n;
01254             q = older;
01255             do {
01256                 if ( !CheckWild(BHEAD *m-WILDOFFSET,INDTOIND,*q,&newvall) ) {
01257                     AddWild(BHEAD *m-WILDOFFSET,INDTOIND,newvall);
01258                     break;
01259                 }
01260                 q++;
01261                 nq--;
01262             } while ( nq > 0 );
01263             if ( nq <= 0 ) return (0);
01264             n--;
01265             nq--;
01266             p = q + 1;

```

```

01267         while ( nq > 0 ) { *q++ = *p++; nq--; }
01268         p--;
01269         m++;
01270     }
01271     else if ( ( *m >= (AM.OffsetVector+WILDOFFSET) )
01272     && ( *m < (AM.OffsetVector+2*WILDOFFSET) ) ) {
01273         while ( t < xstop ) {
01274             *p++ = *t++; n++;
01275         }
01276         if ( !n ) return(0);
01277         nq = n;
01278         q = older;
01279         do {
01280             if ( !CheckWild(BHEAD *m-WILDOFFSET, VECTOVEC, *q, &newval1) ) {
01281                 AddWild(BHEAD *m-WILDOFFSET, VECTOVEC, newval1);
01282                 break;
01283             }
01284             q++;
01285             nq--;
01286         } while ( nq > 0 );
01287         if ( nq <= 0 ) return (0);
01288         n--;
01289         nq--;
01290         p = q + 1;
01291         while ( nq > 0 ) { *q++ = *p++; nq--; }
01292         p--;
01293         m++;
01294     }
01295     else {
01296         if ( t >= xstop ) return(0);
01297         *p++ = *t++; n++;
01298     }
01299 } while ( m < ystop );
01300
01301 /*
01302     return(0);
01303 */
01304 }
01305 /*
01306     #] INDICES :
01307     #[ DELTAS :
01308 */
01309 else if ( *m == DELTA ) {
01310     while ( *t != DELTA ) { t += t[1]; if ( t >= tstop ) return(0); }
01311     xstop = t + t[1];
01312     ystop = m + m[1];
01313     t += 2;
01314     m += 2;
01315     n = 0;
01316     p = older;
01317     do {
01318         if ( *t == *m && t[1] == m[1] && t < xstop ) {
01319             m += 2;
01320             t += 2;
01321         }
01322         else if ( ( *m >= (AM.OffsetIndex+WILDOFFSET) )
01323         && ( *m < (AM.OffsetIndex+2*WILDOFFSET) )
01324         && ( m[1] >= (AM.OffsetIndex+WILDOFFSET) )
01325         && ( m[1] < (AM.OffsetIndex+2*WILDOFFSET) ) ) { /* Two dummies */
01326             while ( t < xstop ) {
01327                 *p++ = *t++; *p++ = *t++; n += 2;
01328             }
01329             if ( !n ) return(0);
01330             nq = n;
01331             q = older;
01332             do {
01333                 if ( !CheckWild(BHEAD *m-WILDOFFSET, INDTOIND, *q, &newval1) &&
01334                 !CheckWild(BHEAD m[1]-WILDOFFSET, INDTOIND, q[1], &newval2) ) {
01335                     AddWild(BHEAD *m-WILDOFFSET, INDTOIND, newval1);
01336                     AddWild(BHEAD m[1]-WILDOFFSET, INDTOIND, newval2);
01337                     break;
01338                 }
01339                 if ( !CheckWild(BHEAD *m-WILDOFFSET, INDTOIND, q[1], &newval1) &&
01340                 !CheckWild(BHEAD m[1]-WILDOFFSET, INDTOIND, *q, &newval2) ) {
01341                     AddWild(BHEAD *m-WILDOFFSET, INDTOIND, newval1);
01342                     AddWild(BHEAD m[1]-WILDOFFSET, INDTOIND, newval2);
01343                     break;
01344                 }
01345                 q += 2;
01346                 nq -= 2;
01347             } while ( nq > 0 );
01348             if ( nq <= 0 ) return(0);
01349             n -= 2;
01350             nq -= 2;
01351             p = q + 2;
01352             while ( nq > 0 ) { *q++ = *p++; nq--; }
01353             p -= 2;

```

```

01354         m += 2;
01355     }
01356     else if ( ( m[1] >= (AM.OffsetIndex+WILDOFFSET) )
01357     && ( m[1] < (AM.OffsetIndex+2*WILDOFFSET) ) ) {
01358         wild = m[1]; regular = *m;
01359     OneWild:
01360         while ( ( regular == *t || regular == t[1] ) && t < xstop ) {
01361             *p++ = *t++; *p++ = *t++; n += 2;
01362         }
01363         if ( !n ) return(0);
01364         nq = n;
01365         q = older;
01366         do {
01367             if ( regular == *q && !CheckWild(BHEAD wild-WILDOFFSET,INDTOIND,q[1],&newvall)
01368         ) {
01369                 AddWild(BHEAD wild-WILDOFFSET,INDTOIND,newvall);
01370                 break;
01371             }
01372             if ( regular == q[1] && !CheckWild(BHEAD wild-WILDOFFSET,INDTOIND,*q,&newvall)
01373         ) {
01374                 AddWild(BHEAD wild-WILDOFFSET,INDTOIND,newvall);
01375                 break;
01376             }
01377             q += 2;
01378             nq -= 2;
01379             } while ( nq > 0 );
01380             if ( nq <= 0 ) return(0);
01381             n -= 2;
01382             nq -= 2;
01383             p = q + 2;
01384             while ( nq > 0 ) { *q++ = *p++; nq--; }
01385             p -= 2;
01386             m += 2;
01387         }
01388         else if ( ( *m >= (AM.OffsetIndex+WILDOFFSET) )
01389         && ( *m < (AM.OffsetIndex+2*WILDOFFSET) ) ) {
01390             wild = *m; regular = m[1];
01391             goto OneWild;
01392         }
01393         else {
01394             if ( t >= tstop || *m < *t || ( *m == *t && m[1] < t[1] ) )
01395                 return(0);
01396             *p++ = *t++; *p++ = *t++; n += 2;
01397         }
01398     } while ( m < ystop );
01399 }
01400 /*
01401     #] DELTAS :
01402     */
01403     else {
01404         MLOCK(ErrorMessageLock);
01405         MesPrint("Pattern not yet implemented");
01406         MUNLOCK(ErrorMessageLock);
01407         Terminate(-1);
01408     }
01409 } while ( m < mstop );
01410 return(1);
01411 }
01412
01413 /*
01414     #] FindRest :
01415     #] Patterns :
01416     */

```

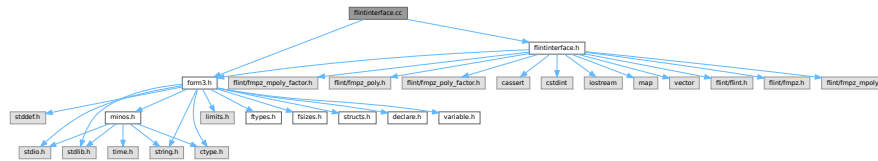
4.37 flintinterface.cc File Reference

```

#include "form3.h"
#include "flintinterface.h"

```

Include dependency graph for flintinterface.cc:



Macros

- `#define IFW(x) { if ( write ) {x;} }`

4.37.1 Detailed Description

Contains functions which interface with FLINT functions to perform arithmetic with uni- and multi-variate polynomials and perform factorization.

Definition in file [flintinterface.cc](#).

4.37.2 Macro Definition Documentation

4.37.2.1 IFW

```
#define IFW(
    x ) { if ( write ) {x;} }
```

Definition at line 1761 of file [flintinterface.cc](#).

4.38 flintinterface.cc

[Go to the documentation of this file.](#)

```

00001
00007 #ifdef HAVE_CONFIG_H
00008 #ifndef CONFIG_H_INCLUDED
00009 #define CONFIG_H_INCLUDED
00010 #include <config.h>
00011 #endif
00012 #endif
00013
00014 extern "C" {
00015 #include "form3.h"
00016 }
00017
00018 #include "flintinterface.h"
00019
00020 /*
00021  * [ Types
00022  * Use fixed-size integer types (int32_t, uint32_t, int64_t, uint64_t) except when we refer
00023  * specifically to FORM term data, when we use WORD. This code has only been tested on 64bit
00024  * systems where WORDs are int32_t, and some functions (fmpz_get_form, fmpz_set_form) require this
00025  * to be the case. Enforce this:
00026  */
00027 static_assert(sizeof(WORD) == sizeof(int32_t), "flint interface expects 32bit WORD");
00028 static_assert(sizeof(WORD) == sizeof(uint32_t), "flint interface expects 32bit WORD");
00029 static_assert(BITSINWORD == 32, "flint interface expects 32bit WORD");
00030 static_assert(sizeof(ulong) == sizeof(uint64_t), "flint interface expects ulong is uint64_t");

```

```

00031 static_assert(sizeof(slong) == sizeof(int64_t), "flint interface expects slong is int64_t");
00032
00033 /*
00034  * Flint functions take arguments or return values which may be "slong" or "ulong" in its
00035  * documentation, and these are int64_t and uint64_t respectively.
00036  *
00037  * #] Types
00038  */
00039
00040 /*
00041  * #[ flint::cleanup :
00042  */
00043 void flint::cleanup(void) {
00044     flint_cleanup();
00045 }
00046 /*
00047  * #] flint::cleanup :
00048  * #[ flint::cleanup_master :
00049  */
00050 void flint::cleanup_master(void) {
00051     flint_cleanup_master();
00052 }
00053 /*
00054  * #] flint::cleanup_master :
00055  * #[ flint::divmod_mpoly :
00056  */
00057 WORD* flint::divmod_mpoly(PHEAD const WORD *a, const WORD *b, const bool return_rem,
00058     const WORD must_fit_term, const var_map_t &var_map) {
00059
00060     flint::mpoly_ctx ctx(var_map.size());
00061     flint::mpoly pa(ctx.d), pb(ctx.d), denpa(ctx.d), denpb(ctx.d);
00062
00063     flint::from_argument_mpoly(pa.d, denpa.d, a, false, var_map, ctx.d);
00064     flint::from_argument_mpoly(pb.d, denpb.d, b, false, var_map, ctx.d);
00065
00066     // The input won't have any symbols with negative powers, but there may be rational
00067     // coefficients. Verify this:
00068     if ( fmpz_mpoly_is_fmpz(denpa.d, ctx.d) != 1 ) {
00069         MLOCK(ErrorMessageLock);
00070         MesPrint("flint::divmod_mpoly: error: denpa is non-constant");
00071         MUNLOCK(ErrorMessageLock);
00072         Terminate(-1);
00073     }
00074     if ( fmpz_mpoly_is_fmpz(denpb.d, ctx.d) != 1 ) {
00075         MLOCK(ErrorMessageLock);
00076         MesPrint("flint::divmod_mpoly: error: denpb is non-constant");
00077         MUNLOCK(ErrorMessageLock);
00078         Terminate(-1);
00079     }
00080
00081     flint::fmpz scale;
00082     flint::mpoly div(ctx.d), rem(ctx.d);
00083
00084     fmpz_mpoly_quasidivrem(scale.d, div.d, rem.d, pa.d, pb.d, ctx.d);
00085
00086     // The quotient must be multiplied by the denominator of the divisor
00087     fmpz_mpoly_mul(div.d, div.d, denpb.d, ctx.d);
00088
00089     // The overall denominator of both div and rem is given by scale*denpa. This we will pass to
00090     // to_argument_mpoly's "denscale" argument which performs the division in the result. We have
00091     // already checked denpa is just an fmpz.
00092     fmpz_mul(scale.d, scale.d, fmpz_mpoly_term_coeff_ref(denpa.d, 0, ctx.d));
00093
00094     // Determine the size of the output by passing write = false.
00095     const bool with_arghead = false;
00096     bool write = false;
00097     const uint64_t prev_size = 0;
00098     const uint64_t out_size = return_rem ?
00099         (uint64_t)flint::to_argument_mpoly(BHEAD NULL, with_arghead, must_fit_term, write, prev_size,
00100             rem.d, var_map, ctx.d, scale.d)
00101         :
00102         (uint64_t)flint::to_argument_mpoly(BHEAD NULL, with_arghead, must_fit_term, write, prev_size,
00103             div.d, var_map, ctx.d, scale.d)
00104     ;
00105     WORD* res = (WORD *)Malloc1(sizeof(WORD)*out_size, "flint::divrem_mpoly");
00106
00107     // Write out the result
00108     write = true;
00109     if ( return_rem ) {
00110         (uint64_t)flint::to_argument_mpoly(BHEAD res, with_arghead, must_fit_term, write, prev_size,
00111             rem.d, var_map, ctx.d, scale.d);
00112     }
00113     else {

```

```

00118         (uint64_t)flint::to_argument_mpoly(BHEAD res, with_arghead, must_fit_term, write, prev_size,
00119         div.d, var_map, ctx.d, scale.d);
00120     }
00121
00122     return res;
00123 }
00124 /*
00125  #] flint::divmod_mpoly :
00126  #[ flint::divmod_mpoly :
00127  */
00128 WORD* flint::divmod_poly(PHEAD const WORD *a, const WORD *b, const bool return_rem,
00129     const WORD must_fit_term, const var_map_t &var_map) {
00130
00131     flint::poly pa, pb, denpa, denpb;
00132
00133     flint::from_argument_poly(pa.d, denpa.d, a, false);
00134     flint::from_argument_poly(pb.d, denpb.d, b, false);
00135
00136     // The input won't have any symbols with negative powers, but there may be rational
00137     // coefficients. Verify this:
00138     if ( fmpz_poly_length(denpa.d) != 1 ) {
00139         MLOCK(ErrorMessageLock);
00140         MesPrint("flint::divmod_poly: error: denpa is non-constant");
00141         MUNLOCK(ErrorMessageLock);
00142         Terminate(-1);
00143     }
00144     if ( fmpz_poly_length(denpb.d) != 1 ) {
00145         MLOCK(ErrorMessageLock);
00146         MesPrint("flint::divmod_poly: error: denpb is non-constant");
00147         MUNLOCK(ErrorMessageLock);
00148         Terminate(-1);
00149     }
00150
00151     flint::fmpz scale;
00152     uint64_t scale_power = 0;
00153     flint::poly div, rem;
00154
00155     // Get the leading coefficient of pb. fmpz_poly_pseudo_divrem returns only the scaling power.
00156     fmpz_poly_get_coeff_fmpz(scale.d, pb.d, fmpz_poly_degree(pb.d));
00157
00158     fmpz_poly_pseudo_divrem(div.d, rem.d, (ulong*)&scale_power, pa.d, pb.d);
00159
00160     // The quotient must be multiplied by the denominator of the divisor
00161     fmpz_poly_mul(div.d, div.d, denpb.d);
00162
00163     // The overall denominator of both div and rem is given by scale^scale_power*denpa. This we will
00164     // pass to to_argument_poly's "denscale" argument which performs the division in the result. We
00165     // have already checked denpa is just an fmpz.
00166     fmpz_pow_ui(scale.d, scale.d, scale_power);
00167     fmpz_mul(scale.d, scale.d, fmpz_poly_get_coeff_ptr(denpa.d, 0));
00168
00169     // Determine the size of the output by passing write = false.
00170     const bool with_arghead = false;
00171     bool write = false;
00172     const uint64_t prev_size = 0;
00173     const uint64_t out_size = return_rem ?
00174         (uint64_t)flint::to_argument_poly(BHEAD NULL, with_arghead, must_fit_term, write, prev_size,
00175         rem.d, var_map, scale.d)
00176         :
00177         (uint64_t)flint::to_argument_poly(BHEAD NULL, with_arghead, must_fit_term, write, prev_size,
00178         div.d, var_map, scale.d)
00179     ;
00180     WORD* res = (WORD *)Malloc1(sizeof(WORD)*out_size, "flint::divrem_poly");
00181
00182     // Write out the result
00183     write = true;
00184     if ( return_rem ) {
00185         (uint64_t)flint::to_argument_poly(BHEAD res, with_arghead, must_fit_term, write, prev_size,
00186         rem.d, var_map, scale.d);
00187     }
00188     else {
00189         (uint64_t)flint::to_argument_poly(BHEAD res, with_arghead, must_fit_term, write, prev_size,
00190         div.d, var_map, scale.d);
00191     }
00192
00193     return res;
00194 }
00195 /*
00196  #] flint::divmod_mpoly :
00197  #[ flint::factorize_mpoly :
00198  */
00199 WORD* flint::factorize_mpoly(PHEAD const WORD *argin, WORD *argout, const bool with_arghead,
00200     const bool is_fun_arg, const var_map_t &var_map) {
00201
00202

```

```

00205     flint::mpoly_ctx ctx(var_map.size());
00206     flint::mpoly arg(ctx.d), den(ctx.d), base(ctx.d);
00207
00208     flint::from_argument_mpoly(arg.d, den.d, argin, with_arghead, var_map, ctx.d);
00209     // The denominator must be 1:
00210     if ( fmpz_mpoly_is_one(den.d, ctx.d) != 1 ) {
00211         MLOCK(ErrorMessageLock);
00212         MesPrint("flint::factorize_mpoly error: den != 1");
00213         MUNLOCK(ErrorMessageLock);
00214         Terminate(-1);
00215     }
00216
00217     // Now we can factor the mpoly:
00218     flint::mpoly_factor arg_fac(ctx.d);
00219     fmpz_mpoly_factor(arg_fac.d, arg.d, ctx.d);
00220     const int64_t num_factors = fmpz_mpoly_factor_length(arg_fac.d, ctx.d);
00221
00222     flint::fmpz overall_constant;
00223     fmpz_mpoly_factor_get_constant_fmpz(overall_constant.d, arg_fac.d, ctx.d);
00224     // FORM should always have taken the overall constant out in the content. Thus this overall
00225     // constant factor should be +- 1 here. Verify this:
00226     if ( ! ( fmpz_equal_si(overall_constant.d, 1) || fmpz_equal_si(overall_constant.d, -1) ) ) {
00227         MLOCK(ErrorMessageLock);
00228         MesPrint("flint::factorize_mpoly error: overall constant factor != +-1");
00229         MUNLOCK(ErrorMessageLock);
00230         Terminate(-1);
00231     }
00232
00233     // Construct the output. If argout is not NULL, we write the result there.
00234     // Otherwise, allocate memory.
00235     // The output is zero-terminated list of factors. If with_arghead, each has an arghead which
00236     // contains its size. Otherwise, the factors are zero separated.
00237
00238     // We only need to determine the size of the output if we are allocating memory, but we need to
00239     // loop through the factors to fix their signs anyway. Do both together in one loop:
00240
00241     // Initially 1, for the final trailing 0.
00242     uint64_t output_size = 1;
00243
00244     // For finding the highest symbol, in FORM's lexicographic ordering
00245     var_map_t var_map_inv;
00246     for ( auto x: var_map ) {
00247         var_map_inv[x.second] = x.first;
00248     }
00249
00250     // Store whether we should flip the factor sign in the output:
00251     vector<int32_t> base_sign(num_factors, 0);
00252
00253     for ( int64_t i = 0; i < num_factors; i++ ) {
00254         fmpz_mpoly_factor_get_base(base.d, arg_fac.d, i, ctx.d);
00255         const int64_t exponent = fmpz_mpoly_factor_get_exp_si(arg_fac.d, i, ctx.d);
00256
00257         // poly_factorize makes sure the highest power of the "highest symbol" (in FORM's
00258         // lexicographic ordering) has a positive coefficient. Check this, update overall_constant
00259         // of the factorization if necessary.
00260         // Store the sign per factor, so that we can flip the signs in the output without re-checking
00261         // the individual terms again.
00262         uint32_t max_var = 0; // FORM symbols start at 20, 0 is a good initial value.
00263         int32_t max_pow = -1;
00264         vector<int64_t> base_term_exponents(var_map.size(), 0);
00265
00266         for ( int64_t j = 0; j < fmpz_mpoly_length(base.d, ctx.d); j++ ) {
00267             fmpz_mpoly_get_term_exp_si((slong*)base_term_exponents.data(), base.d, (slong)j, ctx.d);
00268
00269             for ( size_t k = 0; k < var_map.size(); k++ ) {
00270                 if ( base_term_exponents[k] > 0 && ( var_map_inv.at(k) > max_var ||
00271                     ( var_map_inv.at(k) == max_var && base_term_exponents[k] > max_pow ) ) ) {
00272                     max_var = var_map_inv.at(k);
00273                     max_pow = base_term_exponents[k];
00274                     base_sign[i] = fmpz_sgn(fmpz_mpoly_term_coeff_ref(base.d, j, ctx.d));
00275                 }
00276             }
00277         }
00278
00279         // If this base's sign will be flipped an odd number of times, there is a contribution to
00280         // the overall sign of the whole factorization:
00281         if ( ( base_sign[i] == -1 ) && ( exponent % 2 == 1 ) ) {
00282             fmpz_neg(overall_constant.d, overall_constant.d);
00283         }
00284
00285         // Now determine the output size of the factor, if we are allocating the memory
00286         if ( argout == NULL ) {
00287             const bool write = false;
00288             for ( int64_t j = 0; j < exponent; j++ ) {
00289                 output_size += (uint64_t)flint::to_argument_mpoly(BHEAD NULL, with_arghead,
00290                     is_fun_arg, write, 0, base.d, var_map, ctx.d);

```

```

00292     }
00293     }
00294 }
00295 if ( fmpz_sgn(overall_constant.d) == -1 && argout == NULL ) {
00296     // Add space for a fast-notation number or a normal-notation number and zero separator
00297     output_size += with_arghead ? 2 : 4+1;
00298 }
00299
00300 // Now make the allocation if necessary:
00301 if ( argout == NULL ) {
00302     argout = (WORD*)Malloca(sizeof(WORD)*output_size, "flint::factorize_mpoly");
00303 }
00304
00305
00306 // And now comes the actual output:
00307 WORD* old_argout = argout;
00308
00309 // If the overall sign is negative, first write a full-notation -1. It will be absorbed into the
00310 // overall factor in the content by the caller.
00311 if ( fmpz_sgn(overall_constant.d) == -1 ) {
00312     if ( with_arghead ) {
00313         // poly writes in fast notation in this case. Fast notation is expected by the caller, to
00314         // properly merge it with the overall factor of the content.
00315         *argout++ = -SNUMBER;
00316         *argout++ = -1;
00317     }
00318     else {
00319         *argout++ = 4; // term size
00320         *argout++ = 1; // numerator
00321         *argout++ = 1; // denominator
00322         *argout++ = -3; // coeff size, negative number
00323         *argout++ = 0; // factor separator
00324     }
00325 }
00326
00327 for ( int64_t i = 0; i < num_factors; i++ ) {
00328     fmpz_mpoly_factor_get_base(base.d, arg_fac.d, i, ctx.d);
00329     const int64_t exponent = fmpz_mpoly_factor_get_exp_si(arg_fac.d, i, ctx.d);
00330
00331     if ( base_sign[i] == -1 ) {
00332         fmpz_mpoly_neg(base.d, base.d, ctx.d);
00333     }
00334
00335     const bool write = true;
00336     for ( int64_t j = 0; j < exponent; j++ ) {
00337         argout += flint::to_argument_mpoly(BHEAD argout, with_arghead, is_fun_arg, write,
00338             argout-old_argout, base.d, var_map, ctx.d);
00339     }
00340 }
00341 // Final trailing zero to denote the end of the factors.
00342 *argout++ = 0;
00343
00344 return old_argout;
00345 }
00346 /*
00347 #] flint::factorize_mpoly :
00348 #[ flint::factorize_poly :
00349 */
00350 WORD* flint::factorize_poly(PHEAD const WORD *argin, WORD *argout, const bool with_arghead,
00351     const bool is_fun_arg, const var_map_t &var_map) {
00352
00353     flint::poly arg, den;
00354
00355     flint::from_argument_poly(arg.d, den.d, argin, with_arghead);
00356     // The denominator must be 1:
00357     if ( fmpz_poly_is_one(den.d) != 1 ) {
00358         MLOCK(ErrorMessageLock);
00359         MesPrint("flint::factorize_poly error: den != 1");
00360         MUNLOCK(ErrorMessageLock);
00361         Terminate(-1);
00362     }
00363
00364
00365     // Now we can factor the poly:
00366     flint::poly_factor arg_fac;
00367     fmpz_poly_factor(arg_fac.d, arg.d);
00368     // fmpz_poly_factor_t lacks some convenience functions which fmpz_mpoly_factor_t has.
00369     // I have worked out how to get the factors by looking at how fmpz_poly_factor_print works.
00370     const long num_factors = (arg_fac.d)->num;
00371
00372
00373     // Construct the output. If argout is not NULL, we write the result there.
00374     // Otherwise, allocate memory.
00375     // The output is zero-terminated list of factors. If with_arghead, each has an arghead which
00376     // contains its size. Otherwise, the factors are zero separated.
00377     if ( argout == NULL ) {
00378         // First we need to determine the size of the output. This is the same procedure as the

```



```

00379         // loop below, but we don't write anything in to_argument_poly (write arg: false).
00380         // Initially 1, for the final trailing 0.
00381         uint64_t output_size = 1;
00382
00383         for ( long i = 0; i < num_factors; i++ ) {
00384             fmpz_poly_struct* base = (arg_fac.d)->p + i;
00385
00386             const long exponent = (arg_fac.d)->exp[i];
00387
00388             const bool write = false;
00389             for ( long j = 0; j < exponent; j++ ) {
00390                 output_size += (uint64_t)flint::to_argument_poly(BHEAD NULL, with_arghead,
00391                     is_fun_arg, write, 0, base, var_map);
00392             }
00393         }
00394
00395         argout = (WORD*)Malloca(sizeof(WORD)*output_size, "flint::factorize_poly");
00396     }
00397
00398     WORD* old_argout = argout;
00399
00400     for ( long i = 0; i < num_factors; i++ ) {
00401         fmpz_poly_struct* base = (arg_fac.d)->p + i;
00402
00403         const long exponent = (arg_fac.d)->exp[i];
00404
00405         const bool write = true;
00406         for ( long j = 0; j < exponent; j++ ) {
00407             argout += flint::to_argument_poly(BHEAD argout, with_arghead, is_fun_arg, write,
00408                 argout-old_argout, base, var_map);
00409         }
00410     }
00411     *argout = 0;
00412
00413     return old_argout;
00414 }
00415 /*
00416 #] flint::factorize_poly :
00417
00418 #[ flint::form_sort :
00419 */
00420 // Sort terms using form's sorting routines. Uses a custom (faster) compare routine, since here
00421 // only symbols can appear.
00422 // This is a modified poly_sort from polywrap.cc.
00423 void flint::form_sort(PHEAD WORD *terms) {
00424
00425     if ( terms[0] < 0 ) {
00426         // Fast notation, there is nothing to do
00427         return;
00428     }
00429
00430     const WORD oldsorttype = AR.SortType;
00431     AR.SortType = SORTHIGHFIRST;
00432
00433     const WORD in_size = terms[0] - ARGHEAD;
00434     WORD out_size;
00435
00436     if ( NewSort(BHEAD0) ) {
00437         Terminate(-1);
00438     }
00439     AR.CompareRoutine = (COMPAREDUMMY)(&CompareSymbols);
00440
00441     // Make sure the symbols are in the right order within the terms
00442     for ( WORD i = ARGHEAD; i < terms[0]; i += terms[i] ) {
00443         if ( SymbolNormalize(terms+i) < 0 || StoreTerm(BHEAD terms+i) ) {
00444             AR.SortType = oldsorttype;
00445             AR.CompareRoutine = (COMPAREDUMMY)(&Compare1);
00446             LowerSortLevel();
00447             Terminate(-1);
00448         }
00449     }
00450
00451     if ( ( out_size = EndSort(BHEAD terms+ARGHEAD, 1) ) < 0 ) {
00452         AR.SortType = oldsorttype;
00453         AR.CompareRoutine = (COMPAREDUMMY)(&Compare1);
00454         Terminate(-1);
00455     }
00456
00457     // Check the final size
00458     if ( in_size != out_size ) {
00459         MLOCK(ErrorMessageLock);
00460         MesPrint("flint::form_sort: error: unexpected sorted arg length change %d->%d", in_size,
00461             out_size);
00462         MUNLOCK(ErrorMessageLock);
00463         Terminate(-1);
00464     }
00465 }

```

```

00466     AR.SortType = oldsorttype;
00467     AR.CompareRoutine = (COMPAREDUMMY)(&Compare1);
00468     terms[1] = 0; // set dirty flag to zero
00469 }
00470 /*
00471     #] flint::form_sort :
00472
00473     #[ flint::from_argument_mpoly :
00474 */
00475 // Convert a FORM argument (or 0-terminated list of terms: with_arghead == false) to a
00476 // (multi-variate) fmpz_mpoly_t poly. The "denominator" is return in denpoly, and contains the
00477 // overall negative-power numeric and symbolic factor.
00478 uint64_t flint::from_argument_mpoly(fmpz_mpoly_t poly, fmpz_mpoly_t denpoly, const WORD *args,
00479     const bool with_arghead, const var_map_t &var_map, const fmpz_mpoly_ctx_t ctx) {
00480
00481     // Some callers re-use their poly, denpoly to avoid calling init/clear unnecessarily.
00482     // Make sure they are 0 to begin.
00483     fmpz_mpoly_set_si(poly, 0, ctx);
00484     fmpz_mpoly_set_si(denpoly, 0, ctx);
00485
00486     // First check for "fast notation" arguments:
00487     if ( *args == -SNUMBER ) {
00488         fmpz_mpoly_set_si(poly, *(args+1), ctx);
00489         fmpz_mpoly_set_si(denpoly, 1, ctx);
00490         return 2;
00491     }
00492
00493     if ( *args == -SYMBOL ) {
00494         // A "fast notation" SYMBOL has a power and coefficient of 1:
00495         vector<uint64_t> exponents(var_map.size(), 0);
00496         exponents[var_map.at(*(args+1))] = 1;
00497
00498         fmpz_mpoly_set_coeff_ui_ui(poly, (ulong)1, (ulong*)exponents.data(), ctx);
00499         fmpz_mpoly_set_ui(denpoly, (ulong)1, ctx);
00500         return 2;
00501     }
00502
00503
00504     // Now we can iterate through the terms of the argument. If we have
00505     // an ARGHEAD, we already know where to terminate. Otherwise we'll have
00506     // to loop until the terminating 0.
00507     const WORD* arg_stop = with_arghead ? args+args[0] : (WORD*)UINT64_MAX;
00508     uint64_t arg_size = 0;
00509     if ( with_arghead ) {
00510         arg_size = args[0];
00511         args += ARGHEAD;
00512     }
00513
00514
00515     // Search for numerical or symbol denominators to create "denpoly".
00516     flint::fmpz den_coeff, tmp;
00517     fmpz_set_si(den_coeff, 1);
00518     vector<uint64_t> neg_exponents(var_map.size(), 0);
00519
00520     for ( const WORD* term = args; term < arg_stop; term += term[0] ) {
00521         const WORD* term_stop = term+term[0];
00522         const WORD coeff_size = (term_stop)[-1];
00523         const WORD* symbol_stop = term_stop - ABS(coeff_size);
00524         const WORD* t = term;
00525
00526         t++;
00527         if ( t == symbol_stop ) {
00528             // Just a number, no symbols
00529         }
00530         else {
00531             t++; // this entry is SYMBOL
00532             t++; // this entry just has the size of the symbol array, but we can use symbol_stop
00533
00534             for ( const WORD* s = t; s < symbol_stop; s += 2 ) {
00535                 if ( *(s+1) < 0 ) {
00536                     neg_exponents[var_map.at(*s)] =
00537                         MaX(neg_exponents[var_map.at(*s)], (uint64_t)(-*(s+1)));
00538                 }
00539             }
00540         }
00541     }
00542
00543     // Now check for a denominator in the coefficient:
00544     if ( *(symbol_stop+ABS(coeff_size/2)) != 1 ) {
00545         flint::fmpz_set_form(tmp.d, (UWORD*)(symbol_stop+ABS(coeff_size/2)), ABS(coeff_size/2));
00546         // Record the LCM of the coefficient denominators:
00547         fmpz_lcm(den_coeff.d, den_coeff.d, tmp.d);
00548     }
00549
00550     if ( !with_arghead && *term_stop == 0 ) {
00551         // + 1 for the terminating 0
00552         arg_size = term_stop - args + 1;
00553         break;
00554     }

```

```

00553     }
00554
00555 }
00556 // Assemble denpoly.
00557 fmpz_mpoly_set_coeff_fmpz_ui(denpoly, den_coeff.d, (ulong*)neg_exponents.data(), ctx);
00558
00559 // For the term coefficients
00560 flint::fmpz coeff;
00561
00562 for ( const WORD* term = args; term < arg_stop; term += term[0] ) {
00563     const WORD* term_stop = term+term[0];
00564     const WORD coeff_size = (term_stop)[-1];
00565     const WORD* symbol_stop = term_stop - ABS(coeff_size);
00566     const WORD* t = term;
00567
00568     vector<uint64_t> exponents(var_map.size(), 0);
00569
00570     t++; // skip over the total size entry
00571     if ( t == symbol_stop ) {
00572         // Just a number, no symbols
00573     }
00574     else {
00575         t++; // this entry is SYMBOL
00576         t++; // this entry just has the size of the symbol array, but we can use symbol_stop
00577         for ( const WORD* s = t; s < symbol_stop; s += 2 ) {
00578             exponents[var_map.at(*s)] = *(s+1);
00579         }
00580     }
00581     // Now read the coefficient
00582     flint::fmpz_set_form(coeff.d, (UWORD*)symbol_stop, coeff_size/2);
00583
00584     // Multiply by denominator LCM
00585     fmpz_mul(coeff.d, coeff.d, den_coeff.d);
00586
00587     // Shift by neg_exponents
00588     for ( size_t i = 0; i < var_map.size(); i++ ) {
00589         exponents[i] += neg_exponents[i];
00590     }
00591
00592     // Read the denominator if there is one, and divide it out of the coefficient
00593     if ( *(symbol_stop+ABS(coeff_size/2)) != 1 ) {
00594         flint::fmpz_set_form(tmp.d, (UWORD*)(symbol_stop+ABS(coeff_size/2)), ABS(coeff_size/2));
00595         // By construction, this is an exact division
00596         fmpz_divexact(coeff.d, coeff.d, tmp.d);
00597     }
00598
00599     // Push the term to the mpoly, remember to sort when finished! This is much faster than using
00600     // fmpz_mpoly_set_coeff_fmpz_ui when the terms arrive in the "wrong order".
00601     fmpz_mpoly_push_term_fmpz_ui(poly, coeff.d, (ulong*)exponents.data(), ctx);
00602
00603     if ( !with_arghead && *term_stop == 0 ) {
00604         break;
00605     }
00606 }
00607
00608 // And now sort the mpoly
00609 fmpz_mpoly_sort_terms(poly, ctx);
00610
00611 return arg_size;
00612 }
00613
00614 /*
00615 #] flint::from_argument_mpoly :
00616 #[ flint::from_argument_poly :
00617 */
00618 // Convert a FORM argument (or 0-terminated list of terms: with_arghead == false) to a
00619 // (uni-variate) fmpz_poly_t poly. The "denominator" is return in denpoly, and contains the
00620 // overall negative-power numeric and symbolic factor.
00621 uint64_t flint::from_argument_poly(fmpz_poly_t poly, fmpz_poly_t denpoly, const WORD *args,
00622     const bool with_arghead) {
00623
00624     // Some callers re-use their poly, denpoly to avoid calling init/clear unnecessarily.
00625     // Make sure they are 0 to begin.
00626     fmpz_poly_set_si(poly, 0);
00627     fmpz_poly_set_si(denpoly, 0);
00628
00629     // First check for "fast notation" arguments:
00630     if ( *args == -SNUMBER ) {
00631         fmpz_poly_set_si(poly, *(args+1));
00632         fmpz_poly_set_si(denpoly, 1);
00633         return 2;
00634     }
00635
00636     if ( *args == -SYMBOL ) {
00637         // A "fast notation" SYMBOL has a power and coefficient of 1:
00638         fmpz_poly_set_coeff_si(poly, 1, 1);
00639         fmpz_poly_set_si(denpoly, 1);

```

```

00640         return 2;
00641     }
00642
00643     // Now we can iterate through the terms of the argument. If we have
00644     // an ARGHEAD, we already know where to terminate. Otherwise we'll have
00645     // to loop until the terminating 0.
00646     const WORD* arg_stop = with_arghead ? args+args[0] : (WORD*)UINT64_MAX;
00647     uint64_t arg_size = 0;
00648     if ( with_arghead ) {
00649         arg_size = args[0];
00650         args += ARGHEAD;
00651     }
00652
00653     // Search for numerical or symbol denominators to create "denpoly".
00654     flint::fmpz den_coeff, tmp;
00655     fmpz_set_si(den_coeff.d, 1);
00656     uint64_t neg_exponent = 0;
00657
00658     for ( const WORD* term = args; term < arg_stop; term += term[0] ) {
00659
00660         const WORD* term_stop = term+term[0];
00661         const WORD coeff_size = (term_stop)[-1];
00662         const WORD* symbol_stop = term_stop - ABS(coeff_size);
00663         const WORD* t = term;
00664
00665         t++; // skip over the total size entry
00666         if ( t == symbol_stop ) {
00667             // Just a number, no symbols
00668         }
00669         else {
00670             t++; // this entry is SYMBOL
00671             t++; // this entry is the size of the symbol array
00672             t++; // this is the first (and only) symbol code
00673             if ( *t < 0 ) {
00674                 neg_exponent = MaX(neg_exponent, (uint64_t)(-(*t)) );
00675             }
00676         }
00677
00678         // Now check for a denominator in the coefficient:
00679         if ( *(symbol_stop+ABS(coeff_size/2)) != 1 ) {
00680             flint::fmpz_set_form(tmp.d, (UWORD*)(symbol_stop+ABS(coeff_size/2)), ABS(coeff_size/2));
00681             // Record the LCM of the coefficient denominators:
00682             fmpz_lcm(den_coeff.d, den_coeff.d, tmp.d);
00683         }
00684
00685         if ( *term_stop == 0 ) {
00686             // + 1 for the terminating 0
00687             arg_size = term_stop - args + 1;
00688             break;
00689         }
00690     }
00691     // Assemble denpoly.
00692     fmpz_poly_set_coeff_fmpz(denpoly, neg_exponent, den_coeff.d);
00693
00694     // For the term coefficients
00695     flint::fmpz coeff;
00696
00697     for ( const WORD* term = args; term < arg_stop; term += term[0] ) {
00698
00699         const WORD* term_stop = term+term[0];
00700         const WORD coeff_size = (term_stop)[-1];
00701         const WORD* symbol_stop = term_stop - ABS(coeff_size);
00702         const WORD* t = term;
00703
00704         uint64_t exponent = 0;
00705
00706         t++; // skip over the total size entry
00707         if ( t == symbol_stop ) {
00708             // Just a number, no symbols
00709         }
00710         else {
00711             t++; // this entry is SYMBOL
00712             t++; // this entry is the size of the symbol array
00713             t++; // this is the first (and only) symbol code
00714             exponent = *t++;
00715         }
00716
00717         // Now read the coefficient
00718         flint::fmpz_set_form(coeff.d, (UWORD*)symbol_stop, coeff_size/2);
00719
00720         // Multiply by denominator LCM
00721         fmpz_mul(coeff.d, coeff.d, den_coeff.d);
00722
00723         // Shift by neg_exponent
00724         exponent += neg_exponent;
00725     }
00726

```

```

00727         // Read the denominator if there is one, and divide it out of the coefficient
00728         if ( *(symbol_stop+ABS(coeff_size/2)) != 1 ) {
00729             flint::fmpz_set_form(tmp.d, (UWORD*)(symbol_stop+ABS(coeff_size/2)), ABS(coeff_size/2));
00730             // By construction, this is an exact division
00731             fmpz_divexact(coeff.d, coeff.d, tmp.d);
00732         }
00733
00734         // Add the term to the poly
00735         fmpz_poly_set_coeff_fmpz(poly, exponent, coeff.d);
00736
00737         if ( *term_stop == 0 ) {
00738             break;
00739         }
00740     }
00741
00742     return arg_size;
00743 }
00744 /*
00745 #] flint::from_argument_poly :
00746
00747 #[ flint::fmpz_get_form :
00748 */
00749 // Write FORM's long integer representation of an fmpz at a, and put the number of WORDs at na.
00750 // na carries the sign of the integer.
00751 WORD flint::fmpz_get_form(fmpz_t z, WORD *a) {
00752     WORD na = 0;
00753     const int32_t sgn = fmpz_sgn(z);
00754     if ( sgn == -1 ) {
00755         fmpz_neg(z, z);
00756     }
00757     const int64_t nlimbs = fmpz_size(z);
00758
00759     // This works but is UB?
00760     //fmpz_get_ui_array(reinterpret_cast<uint64_t*>(a), nlimbs, z);
00761
00762     // Use fixed-size functions to get limb data where possible. These probably cover most real
00763     // cases.
00764     if ( nlimbs == 1 ) {
00765         const uint64_t limb = fmpz_get_ui(z);
00766         a[0] = (WORD)(limb & 0xFFFFFFFF);
00767         na++;
00768         a[1] = (WORD)(limb >> BITSINWORD);
00769         if ( a[1] != 0 ) {
00770             na++;
00771         }
00772     }
00773
00774     else if ( nlimbs == 2 ) {
00775         uint64_t limb_hi = 0, limb_lo = 0;
00776         fmpz_get_uiui((ulong*)&limb_hi, (ulong*)&limb_lo, z);
00777         a[0] = (WORD)(limb_lo & 0xFFFFFFFF);
00778         na++;
00779         a[1] = (WORD)(limb_lo >> BITSINWORD);
00780         na++;
00781         a[2] = (WORD)(limb_hi & 0xFFFFFFFF);
00782         na++;
00783         a[3] = (WORD)(limb_hi >> BITSINWORD);
00784         if ( a[3] != 0 ) {
00785             na++;
00786         }
00787     }
00788     else {
00789         vector<uint64_t> limb_data(nlimbs, 0);
00790         fmpz_get_ui_array((ulong*)limb_data.data(), (ulong)nlimbs, z);
00791         for ( long i = 0; i < nlimbs; i++ ) {
00792             a[2*i] = (WORD)(limb_data[i] & 0xFFFFFFFF);
00793             na++;
00794             a[2*i+1] = (WORD)(limb_data[i] >> BITSINWORD);
00795             if ( a[2*i+1] != 0 || i < (nlimbs-1) ) {
00796                 // The final limb might fit in a single 32bit WORD. Only
00797                 // increment na if the final WORD is non zero.
00798                 na++;
00799             }
00800         }
00801     }
00802
00803     // And now put the sign in the number of limbs
00804     if ( sgn == -1 ) {
00805         na = -na;
00806     }
00807
00808     return na;
00809 }
00810 /*
00811 #] flint::fmpz_get_form :
00812 #[ flint::fmpz_set_form :
00813 */

```

```

00814 // Create an fmpz directly from FORM's long integer representation. fmpz uses 64bit unsigned limbs,
00815 // but FORM uses 32bit UWORDS on 64bit architectures so we can't use fmpz_set_ui_array directly.
00816 void flint::fmpz_set_form(fmpz_t z, UWORD *a, WORD na) {
00817
00818     if ( na == 0 ) {
00819         fmpz_zero(z);
00820         return;
00821     }
00822
00823     // Negative na represents a negative number
00824     int32_t sgn = 1;
00825     if ( na < 0 ) {
00826         sgn = -1;
00827         na = -na;
00828     }
00829
00830     // Remove padding. FORM stores numerators and denominators with equal numbers of limbs but we
00831     // don't need to do this within the fmpz. It is not necessary to do this really, the fmpz
00832     // doesn't add zero limbs unnecessarily, but we might be able to avoid creating the limb_data
00833     // array below.
00834     while ( a[na-1] == 0 ) {
00835         na--;
00836     }
00837
00838     // If the number fits in fixed-size fmpz_set functions, we don't need to use additional memory
00839     // to convert to uint64_t. These probably cover most real cases.
00840     if ( na == 1 ) {
00841         fmpz_set_ui(z, (uint64_t)a[0]);
00842     }
00843     else if ( na == 2 ) {
00844         fmpz_set_ui(z, (((uint64_t)a[1])«BITSINWORD) + (uint64_t)a[0]);
00845     }
00846     else if ( na == 3 ) {
00847         fmpz_set_uiui(z, (uint64_t)a[2], (((uint64_t)a[1])«BITSINWORD) + (uint64_t)a[0]);
00848     }
00849     else if ( na == 4 ) {
00850         fmpz_set_uiui(z, (((uint64_t)a[3])«BITSINWORD) + (uint64_t)a[2],
00851             (((uint64_t)a[1])«BITSINWORD) + (uint64_t)a[0]);
00852     }
00853     else {
00854         const int32_t nlimbs = (na+1)/2;
00855         vector<uint64_t> limb_data(nlimbs, 0);
00856         for ( int32_t i = 0; i < nlimbs; i++ ) {
00857             if ( 2*i+1 <= na-1 ) {
00858                 limb_data[i] = (uint64_t)a[2*i] + (((uint64_t)a[2*i+1])«BITSINWORD);
00859             }
00860             else {
00861                 limb_data[i] = (uint64_t)a[2*i];
00862             }
00863         }
00864         fmpz_set_ui_array(z, (ulong*)limb_data.data(), nlimbs);
00865     }
00866
00867     // Finally set the sign.
00868     if ( sgn == -1 ) {
00869         fmpz_neg(z, z);
00870     }
00871
00872     return;
00873 }
00874 /*
00875 #] flint::fmpz_set_form :
00876
00877 #[ flint::gcd_mpoly :
00878 */
00879 // Return a pointer to a buffer containing the GCD of the 0-terminated term lists at a and b.
00880 // If must_fit_term, this should be a TermMalloc buffer. Otherwise Malloc1 the buffer.
00881 // For multi-variate cases.
00882 WORD* flint::gcd_mpoly(PHEAD const WORD *a, const WORD *b, const WORD must_fit_term,
00883     const var_map_t &var_map) {
00884
00885     flint::mpoly_ctx ctx(var_map.size());
00886     flint::mpoly pa(ctx.d, pb(ctx.d), denpa(ctx.d), denpb(ctx.d), gcd(ctx.d);
00887
00888     flint::from_argument_mpoly(pa.d, denpa.d, a, false, var_map, ctx.d);
00889     flint::from_argument_mpoly(pb.d, denpb.d, b, false, var_map, ctx.d);
00890
00891     // denpa, denpb must be 1:
00892     if ( fmpz_mpoly_is_one(denpa.d, ctx.d) != 1 ) {
00893         MLOCK(ErrorMessageLock);
00894         MesPrint("flint::gcd_mpoly: error: denpa != 1");
00895         MUNLOCK(ErrorMessageLock);
00896         Terminate(-1);
00897     }
00898     if ( fmpz_mpoly_is_one(denpb.d, ctx.d) != 1 ) {
00899         MLOCK(ErrorMessageLock);
00900         MesPrint("flint::gcd_mpoly: error: denpb != 1");

```

```

00901     MUNLOCK(ErrorMessageLock);
00902     Terminate(-1);
00903 }
00904
00905 // poly returns pa if pa == pb, regardless of the lcoeff sign
00906 if ( fmpz_mpoly_equal(pa.d, pb.d, ctx.d) ) {
00907     fmpz_mpoly_set(gcd.d, pa.d, ctx.d);
00908 }
00909 else {
00910     // We need some gymnastics to have the same sign conventions as the poly class. It takes the
00911     // integer or univar content out of a,b, with the convention that the content sign matches
00912     // the lcoeff sign. Since FORM has already taken out the content, we are left with +-1. In
00913     // Flint, the content always has a positive sign so here we should find +1. Check this:
00914     flint::mpoly tmp(ctx.d);
00915     fmpz_mpoly_term_content(tmp.d, pa.d, ctx.d);
00916     if ( fmpz_mpoly_is_one(tmp.d, ctx.d) != 1 ) {
00917         MLOCK(ErrorMessageLock);
00918         MesPrint("flint::gcd_mpoly: error: content of 1st arg != 1");
00919         MUNLOCK(ErrorMessageLock);
00920         Terminate(-1);
00921     }
00922     fmpz_mpoly_term_content(tmp.d, pb.d, ctx.d);
00923     if ( fmpz_mpoly_is_one(tmp.d, ctx.d) != 1 ) {
00924         MLOCK(ErrorMessageLock);
00925         MesPrint("flint::gcd_mpoly: error: content of 2nd arg != 1");
00926         MUNLOCK(ErrorMessageLock);
00927         Terminate(-1);
00928     }
00929
00930     // The poly class now divides the content out of a,b so that they have a positive lcoeff.
00931     // Then it multiplies the final gcd (which is given a positive lcoeff also) by
00932     // gcd(cont a, cont b). There it has gcd(1,1) = gcd(-1,1) = gcd(1,-1) = 1, and
00933     // gcd(-1,-1) = -1 (because of the pa==pb early return). So: if both input polys have a
00934     // negative lcoeff, we will flip the sign in the final result.
00935     bool flip_sign = 0;
00936     if ( ( fmpz_sgn(fmpz_mpoly_term_coeff_ref(pa.d, 0, ctx.d)) == -1 ) &&
00937         ( fmpz_sgn(fmpz_mpoly_term_coeff_ref(pb.d, 0, ctx.d)) == -1 ) ) {
00938         flip_sign = 1;
00939     }
00940
00941     fmpz_mpoly_gcd(gcd.d, pa.d, pb.d, ctx.d);
00942     if ( flip_sign ) {
00943         fmpz_mpoly_neg(gcd.d, gcd.d, ctx.d);
00944     }
00945 }
00946
00947 // This is freed by the caller
00948 WORD *res;
00949 if ( must_fit_term ) {
00950     res = TermMalloc("flint::gcd_mpoly");
00951 }
00952 else {
00953     // Determine the size of the GCD by passing write = false.
00954     const bool with_arghead = false;
00955     const bool write = false;
00956     const uint64_t prev_size = 0;
00957     const uint64_t gcd_size = (uint64_t)flint::to_argument_mpoly(BHEAD NULL,
00958         with_arghead, must_fit_term, write, prev_size, gcd.d, var_map, ctx.d);
00959
00960     res = (WORD *)Malloca(sizeof(WORD)*gcd_size, "flint::gcd_mpoly");
00961 }
00962
00963 const bool with_arghead = false;
00964 const bool write = true;
00965 const uint64_t prev_size = 0;
00966 flint::to_argument_mpoly(BHEAD res, with_arghead, must_fit_term, write, prev_size, gcd.d,
00967     var_map, ctx.d);
00968
00969 return res;
00970 }
00971 /*
00972 #] flint::gcd_mpoly :
00973 #[ flint::gcd_poly :
00974 */
00975 // Return a pointer to a buffer containing the GCD of the 0-terminated term lists at a and b.
00976 // If must_fit_term, this should be a TermMalloc buffer. Otherwise Malloca the buffer.
00977 // For uni-variate cases.
00978 WORD* flint::gcd_poly(PHEAD const WORD *a, const WORD *b, const WORD must_fit_term,
00979     const var_map_t &var_map) {
00980
00981     flint::poly pa, pb, denpa, denpb, gcd;
00982
00983     flint::from_argument_poly(pa.d, denpa.d, a, false);
00984     flint::from_argument_poly(pb.d, denpb.d, b, false);
00985
00986     // denpa, denpb must be 1:
00987     if ( fmpz_poly_is_one(denpa.d) != 1 ) {

```

```

00988     MLOCK(ErrorMessageLock);
00989     MesPrint("flint::gcd_poly: error: denpa != 1");
00990     MUNLOCK(ErrorMessageLock);
00991     Terminate(-1);
00992 }
00993 if ( fmpz_poly_is_one(denpb.d) != 1 ) {
00994     MLOCK(ErrorMessageLock);
00995     MesPrint("flint::gcd_poly: error: denpb != 1");
00996     MUNLOCK(ErrorMessageLock);
00997     Terminate(-1);
00998 }
00999
01000 // poly returns pa if pa == pb, regardless of the lcoeff sign
01001 if ( fmpz_poly_equal(pa.d, pb.d) ) {
01002     fmpz_poly_set(gcd.d, pa.d);
01003 }
01004 else {
01005     // Here, we don't have to make any sign flips like the mpoly case, because poly's
01006     // integer_gcd(1,1) = integer_gcd(-1,1) = integer_gcd(1,-1) = integer_gcd(-1,-1) = +1.
01007     // Still, verify that the content is 1:
01008     flint::fmpz tmp;
01009     fmpz_poly_content(tmp.d, pa.d);
01010     if ( fmpz_is_one(tmp.d) != 1 ) {
01011         MLOCK(ErrorMessageLock);
01012         MesPrint("flint::gcd_poly: error: content of 1st arg != 1");
01013         MUNLOCK(ErrorMessageLock);
01014         Terminate(-1);
01015     }
01016     fmpz_poly_content(tmp.d, pb.d);
01017     if ( fmpz_is_one(tmp.d) != 1 ) {
01018         MLOCK(ErrorMessageLock);
01019         MesPrint("flint::gcd_poly: error: content of 2nd arg != 1");
01020         MUNLOCK(ErrorMessageLock);
01021         Terminate(-1);
01022     }
01023 }
01024 fmpz_poly_gcd(gcd.d, pa.d, pb.d);
01025 }
01026
01027 // This is freed by the caller
01028 WORD *res;
01029 if ( must_fit_term ) {
01030     res = TermMalloc("flint::gcd_poly");
01031 }
01032 else {
01033     // Determine the size of the GCD by passing write = false.
01034     const bool with_arghead = false;
01035     const bool write = false;
01036     const uint64_t prev_size = 0;
01037     const uint64_t gcd_size = (uint64_t)flint::to_argument_poly(BHEAD NULL,
01038         with_arghead, must_fit_term, write, prev_size, gcd.d, var_map);
01039
01040     res = (WORD *)Malloca(sizeof(WORD)*gcd_size, "flint::gcd_poly");
01041 }
01042
01043 const bool with_arghead = false;
01044 const uint64_t prev_size = 0;
01045 const bool write = true;
01046 flint::to_argument_poly(BHEAD res, with_arghead, must_fit_term, write, prev_size, gcd.d,
01047     var_map);
01048
01049 return res;
01050 }
01051 /*
01052 #] flint::gcd_poly :
01053
01054 #[ flint::get_variables :
01055 */
01056 // Get the list of symbols which appear in the vector of expressions. These are polyratfun
01057 // numerators, denominators or expressions from calls to gcd_ etc. Return this list as a map
01058 // between indices and symbol codes.
01059 // TODO FACTORSYMBOL last?
01060 flint::var_map_t flint::get_variables(const vector<WORD*> &es, const bool with_arghead,
01061     const bool sort_vars) {
01062
01063     int32_t num_vars = 0;
01064     // To be used if we sort by highest degree, as the polu code does.
01065     vector<int32_t> degrees;
01066     var_map_t var_map;
01067
01068     // extract all variables
01069     for ( size_t ei = 0; ei < es.size(); ei++ ) {
01070         WORD *e = es[ei];
01071
01072         // fast notation
01073         if ( *e == -SNUMBER ) {
01074

```



```

01075         else if ( *e == -SYMBOL ) {
01076             if ( !var_map.count(e[1]) ) {
01077                 var_map[e[1]] = num_vars++;
01078                 degrees.push_back(1);
01079             }
01080         }
01081         // JD: Here we need to check for non-symbol/number terms in fast notation.
01082         else if ( *e < 0 ) {
01083             MLOCK(ErrorMessageLock);
01084             MesPrint("ERROR: polynomials and polyratfuns must contain symbols only");
01085             MUNLOCK(ErrorMessageLock);
01086             Terminate(1);
01087         }
01088         else {
01089             for ( WORD i = with_arghead ? ARGHEAD:0; with_arghead ? i < e[0]:e[i] != 0; i += e[i] ) {
01090                 if ( i+1 < i+e[i]-ABS(e[i+e[i]-1]) && e[i+1] != SYMBOL ) {
01091                     MLOCK(ErrorMessageLock);
01092                     MesPrint("ERROR: polynomials and polyratfuns must contain symbols only");
01093                     MUNLOCK(ErrorMessageLock);
01094                     Terminate(1);
01095                 }
01096
01097                 for ( WORD j = i+3; j<i+e[i]-ABS(e[i+e[i]-1]); j += 2 ) {
01098                     if ( !var_map.count(e[j]) ) {
01099                         var_map[e[j]] = num_vars++;
01100                         degrees.push_back(e[j+1]);
01101                     }
01102                     else {
01103                         degrees[var_map[e[j]]] = MaX(degrees[var_map[e[j]]], e[j+1]);
01104                     }
01105                 }
01106             }
01107         }
01108     }
01109
01110     if ( sort_vars ) {
01111         // bubble sort variables in decreasing order of degree
01112         // (this seems better for factorization)
01113         for ( size_t i = 0; i < var_map.size(); i++ ) {
01114             for ( size_t j = 0; j+1 < var_map.size(); j++ ) {
01115                 if ( degrees[j] < degrees[j+1] ) {
01116                     swap(degrees[j], degrees[j+1]);
01117
01118                     // Find the map keys associated with the values we want to swap
01119                     uint32_t j0 = 0;
01120                     uint32_t j1 = 0;
01121                     for ( auto x: var_map ) {
01122                         if ( x.second == j ) {
01123                             j0 = x.first;
01124                         }
01125                         else if ( x.second == j+1 ) {
01126                             j1 = x.first;
01127                         }
01128                     }
01129                     swap(var_map.at(j0), var_map.at(j1));
01130                 }
01131             }
01132         }
01133     }
01134     // Otherwise, sort lexicographically in FORM's ordering
01135     else {
01136         for ( size_t i = 0; i < var_map.size(); i++ ) {
01137             for ( size_t j = 0; j+1 < var_map.size(); j++ ) {
01138                 uint32_t j0 = 0;
01139                 uint32_t j1 = 0;
01140                 for ( auto x: var_map ) {
01141                     if ( x.second == j ) {
01142                         j0 = x.first;
01143                     }
01144                     else if ( x.second == j+1 ) {
01145                         j1 = x.first;
01146                     }
01147                 }
01148                 if ( j0 > j1 ) {
01149                     swap(var_map.at(j0), var_map.at(j1));
01150                 }
01151             }
01152         }
01153     }
01154
01155     return var_map;
01156 }
01157 /*
01158 #] flint::get_variables :
01159
01160 #[ flint::inverse_poly :
01161 */

```

```

01162 WORD* flint::inverse_poly(PHEAD const WORD *a, const WORD *b, const var_map_t &var_map) {
01163
01164     flint::poly pa, pb, denpa, denpb;
01165
01166     flint::from_argument_poly(pa.d, denpa.d, a, false);
01167     flint::from_argument_poly(pb.d, denpb.d, b, false);
01168
01169     // fmpz_poly_xgcd is undefined if the content of pa and pb are not 1. Take the content out.
01170     // fmpz_poly_content gives the non-negative content and fmpz_poly_primitive normalizes to a
01171     // non-negative lcoeff, so we need to add the sign to the content if the polys have a negative
01172     // lcoeff. We don't need to keep the content of pb, it is a numerical multiple of the modulus.
01173     flint::fmpz content_a, resultant;
01174     flint::poly inverse, tmp;
01175     fmpz_poly_content(content_a.d, pa.d);
01176     if ( fmpz_sgn(fmpz_poly_lead(pa.d)) == -1 ) {
01177         fmpz_neg(content_a.d, content_a.d);
01178     }
01179     fmpz_poly_primitive_part(pa.d, pa.d);
01180     fmpz_poly_primitive_part(pb.d, pb.d);
01181
01182     // Special cases:
01183     // Possibly strange that we give 1 for inverse_(x1,1) but here we take MMA's convention.
01184     if ( fmpz_poly_is_one(pa.d) && fmpz_poly_is_one(pb.d) ) {
01185         fmpz_poly_one(inverse.d);
01186         fmpz_one(resultant.d);
01187     }
01188     else if ( fmpz_poly_is_one(pb.d) ) {
01189         fmpz_poly_zero(inverse.d);
01190         fmpz_one(resultant.d);
01191     }
01192     else {
01193         // Now use the extended Euclidean algorithm to find inverse, resultant, tmp of the Bezout
01194         // identity: inverse*pa + tmp*pb = resultant. Then inverse/resultant is the multiplicative
01195         // inverse of pa mod pb. We'll divide by resultant in the denscale argument of
01196         to_argument_poly.
01197         fmpz_poly_xgcd(resultant.d, inverse.d, tmp.d, pa.d, pb.d);
01198
01199         // If the resultant is zero, the inverse does not exist:
01200         if ( fmpz_is_zero(resultant.d) ) {
01201             MLOCK(ErrorMessageLock);
01202             MesPrint("flint::inverse_poly error: inverse does not exist");
01203             MUNLOCK(ErrorMessageLock);
01204             Terminate(-1);
01205         }
01206
01207         // Multiply inverse by denpa. denpb is a numerical multiple of the modulus, so doesn't matter.
01208         // We also need to divide by content_a, which we do in the denscale argument of to_argument_poly
01209         // by multiplying resultant by content_a here.
01210         fmpz_poly_mul(inverse.d, inverse.d, denpa.d);
01211         fmpz_mul(resultant.d, resultant.d, content_a.d);
01212
01213         WORD* res;
01214         // First determine the result size, and malloc. The result should have no arghead. Here we use
01215         // the "scale" argument of to_argument_poly since resultant might not be 1.
01216         const bool with_arghead = false;
01217         bool write = false;
01218         const bool must_fit_term = false;
01219         const uint64_t prev_size = 0;
01220         const uint64_t res_size = (uint64_t)flint::to_argument_poly(BHEAD NULL,
01221             with_arghead, must_fit_term, write, prev_size, inverse.d, var_map, resultant.d);
01222         res = (WORD*)Malloc1(sizeof(WORD)*res_size, "flint::inverse_poly");
01223
01224         write = true;
01225         flint::to_argument_poly(BHEAD res, with_arghead, must_fit_term, write, prev_size, inverse.d,
01226             var_map, resultant.d);
01227
01228         return res;
01229     }
01230 }
01231 /*
01232 #] flint::inverse_poly :
01233
01234 #[ flint::mul_mpoly :
01235 */
01236 // Return a pointer to a buffer containing the product of the 0-terminated term lists at a and b.
01237 // For multi-variate cases.
01238 WORD* flint::mul_mpoly(PHEAD const WORD *a, const WORD *b, const var_map_t &var_map) {
01239
01240     flint::mpoly_ctx ctx(var_map.size());
01241     flint::mpoly pa(ctx.d), pb(ctx.d), denpa(ctx.d), denpb(ctx.d);
01242
01243     flint::from_argument_mpoly(pa.d, denpa.d, a, false, var_map, ctx.d);
01244     flint::from_argument_mpoly(pb.d, denpb.d, b, false, var_map, ctx.d);
01245
01246     // denpa, denpb must be integers. Negative symbol powers have been converted to extra symbols.
01247     if ( fmpz_mpoly_is_fmpz(denpa.d, ctx.d) != 1 ) {

```

```

01248     MLOCK(ErrorMessageLock);
01249     MesPrint("flint::mul_mpoly: error: denpa is non-constant");
01250     MUNLOCK(ErrorMessageLock);
01251     Terminate(-1);
01252 }
01253 if ( fmpz_mpoly_is_fmpz(denpb.d, ctx.d) != 1 ) {
01254     MLOCK(ErrorMessageLock);
01255     MesPrint("flint::mul_mpoly: error: denpb is non-constant");
01256     MUNLOCK(ErrorMessageLock);
01257     Terminate(-1);
01258 }
01259
01260 // Multiply numerators, store result in pa
01261 fmpz_mpoly_mul(pa.d, pa.d, pb.d, ctx.d);
01262 // Multiply denominators, store result in denpa, and convert to an fmpz:
01263 fmpz_mpoly_mul(denpa.d, denpa.d, denpb.d, ctx.d);
01264 flint::fmpz den;
01265 fmpz_mpoly_get_fmpz(den.d, denpa.d, ctx.d);
01266
01267 WORD* res;
01268 // First determine the result size, and malloc. The result should have no arghead. Here we use
01269 // the "scale" argument of to_argument_mpoly since den might not be 1.
01270 const bool with_arghead = false;
01271 bool write = false;
01272 const bool must_fit_term = false;
01273 const uint64_t prev_size = 0;
01274 const uint64_t mul_size = (uint64_t)flint::to_argument_mpoly(BHEAD NULL,
01275     with_arghead, must_fit_term, write, prev_size, pa.d, var_map, ctx.d, den.d);
01276 res = (WORD*)Malloca(sizeof(WORD)*mul_size, "flint::mul_mpoly");
01277
01278 write = true;
01279 flint::to_argument_mpoly(BHEAD res, with_arghead, must_fit_term, write, prev_size, pa.d,
01280     var_map, ctx.d, den.d);
01281
01282 return res;
01283 }
01284 /*
01285 #] flint::mul_mpoly :
01286 #[ flint::mul_poly :
01287 */
01288 // Return a pointer to a buffer containing the product of the 0-terminated term lists at a and b.
01289 // For uni-variate cases.
01290 WORD* flint::mul_poly(PHEAD const WORD *a, const WORD *b, const var_map_t &var_map) {
01291
01292     flint::poly pa, pb, denpa, denpb;
01293
01294     flint::from_argument_poly(pa.d, denpa.d, a, false);
01295     flint::from_argument_poly(pb.d, denpb.d, b, false);
01296
01297     // denpa, denpb must be integers. Negative symbol powers have been converted to extra symbols.
01298     if ( fmpz_poly_degree(denpa.d) != 0 ) {
01299         MLOCK(ErrorMessageLock);
01300         MesPrint("flint::mul_poly: error: denpa is non-constant");
01301         MUNLOCK(ErrorMessageLock);
01302         Terminate(-1);
01303     }
01304     if ( fmpz_poly_degree(denpb.d) != 0 ) {
01305         MLOCK(ErrorMessageLock);
01306         MesPrint("flint::mul_poly: error: denpb is non-constant");
01307         MUNLOCK(ErrorMessageLock);
01308         Terminate(-1);
01309     }
01310
01311     // Multiply numerators, store result in pa
01312     fmpz_poly_mul(pa.d, pa.d, pb.d);
01313     // Multiply denominators, store result in denpa, and convert to an fmpz:
01314     fmpz_poly_mul(denpa.d, denpa.d, denpb.d);
01315     flint::fmpz den;
01316     fmpz_poly_get_coeff_fmpz(den.d, denpa.d, 0);
01317
01318     WORD* res;
01319     // First determine the result size, and malloc. The result should have no arghead. Here we use
01320     // the "scale" argument of to_argument_poly since den might not be 1.
01321     const bool with_arghead = false;
01322     bool write = false;
01323     const bool must_fit_term = false;
01324     const uint64_t prev_size = 0;
01325     const uint64_t mul_size = (uint64_t)flint::to_argument_poly(BHEAD NULL,
01326         with_arghead, must_fit_term, write, prev_size, pa.d, var_map, den.d);
01327     res = (WORD*)Malloca(sizeof(WORD)*mul_size, "flint::mul_poly");
01328
01329     write = true;
01330     flint::to_argument_poly(BHEAD res, with_arghead, must_fit_term, write, prev_size, pa.d,
01331         var_map, den.d);
01332
01333     return res;
01334 }

```

```

01335 /*
01336 #] flint::mul_poly :
01337
01338 #[ flint::ratfun_add_mpoly :
01339 */
01340 // Add the multi-variate FORM rational polynomials at t1 and t2. The result is written at out.
01341 void flint::ratfun_add_mpoly(PHEAD const WORD *t1, const WORD *t2, WORD *out,
01342     const var_map_t &var_map) {
01343
01344     flint::mpoly_ctx ctx(var_map.size());
01345     flint::mpoly gcd(ctx.d), num1(ctx.d), den1(ctx.d), num2(ctx.d), den2(ctx.d);
01346
01347     flint::ratfun_read_mpoly(t1, num1.d, den1.d, var_map, ctx.d);
01348     flint::ratfun_read_mpoly(t2, num2.d, den2.d, var_map, ctx.d);
01349
01350     if ( fmpz_mpoly_cmp(den1.d, den2.d, ctx.d) != 0 ) {
01351         fmpz_mpoly_gcd_cofactors(gcd.d, den1.d, den2.d, den1.d, den2.d, ctx.d);
01352
01353         fmpz_mpoly_mul(num1.d, num1.d, den2.d, ctx.d);
01354         fmpz_mpoly_mul(num2.d, num2.d, den1.d, ctx.d);
01355
01356         fmpz_mpoly_add(num1.d, num1.d, num2.d, ctx.d);
01357         fmpz_mpoly_mul(den1.d, den1.d, den2.d, ctx.d);
01358         fmpz_mpoly_mul(den1.d, den1.d, gcd.d, ctx.d);
01359     }
01360     else {
01361         fmpz_mpoly_add(num1.d, num1.d, num2.d, ctx.d);
01362     }
01363
01364     // Finally divide out any common factors between the resulting num1, den1:
01365     fmpz_mpoly_gcd_cofactors(gcd.d, num1.d, den1.d, num1.d, den1.d, ctx.d);
01366
01367     flint::util::fix_sign_fmpz_mpoly_ratfun(num1.d, den1.d, ctx.d);
01368
01369     // Result in FORM notation:
01370     *out++ = AR.PolyFun;
01371     WORD* args_size = out++;
01372     WORD* args_flag = out++;
01373     *args_flag = 0; // clean prf
01374     FILLFUN3(out); // Remainder of funhead, if it is larger than 3
01375
01376     const bool with_arghead = true;
01377     const bool must_fit_term = true;
01378     const bool write = true;
01379     // prev_size + 4, to account for final term size and coeff of "1/1"
01380     out += flint::to_argument_mpoly(BHEAD out, with_arghead, must_fit_term, write, out-args_size+4,
01381         num1.d, var_map, ctx.d);
01382     out += flint::to_argument_mpoly(BHEAD out, with_arghead, must_fit_term, write, out-args_size+4,
01383         den1.d, var_map, ctx.d);
01384
01385     *args_size = out - args_size + 1; // The +1 is to include the function ID
01386     AT.WorkPointer = out;
01387 }
01388 /*
01389 #] flint::ratfun_add_poly :
01390 #[ flint::ratfun_add_poly :
01391 */
01392 // Add the uni-variate FORM rational polynomials at t1 and t2. The result is written at out.
01393 void flint::ratfun_add_poly(PHEAD const WORD *t1, const WORD *t2, WORD *out,
01394     const var_map_t &var_map) {
01395
01396     flint::poly gcd, num1, den1, num2, den2;
01397
01398     flint::ratfun_read_poly(t1, num1.d, den1.d);
01399     flint::ratfun_read_poly(t2, num2.d, den2.d);
01400
01401     if ( fmpz_poly_equal(den1.d, den2.d) == 0 ) {
01402         flint::util::simplify_fmpz_poly(den1.d, den2.d, gcd.d);
01403
01404         fmpz_poly_mul(num1.d, num1.d, den2.d);
01405         fmpz_poly_mul(num2.d, num2.d, den1.d);
01406
01407         fmpz_poly_add(num1.d, num1.d, num2.d);
01408         fmpz_poly_mul(den1.d, den1.d, den2.d);
01409         fmpz_poly_mul(den1.d, den1.d, gcd.d);
01410     }
01411     else {
01412         fmpz_poly_add(num1.d, num1.d, num2.d);
01413     }
01414
01415     // Finally divide out any common factors between the resulting num1, den1:
01416     flint::util::simplify_fmpz_poly(num1.d, den1.d, gcd.d);
01417
01418     flint::util::fix_sign_fmpz_poly_ratfun(num1.d, den1.d);
01419
01420     // Result in FORM notation:
01421     *out++ = AR.PolyFun;

```

```

01422     WORD* args_size = out++;
01423     WORD* args_flag = out++;
01424     *args_flag = 0; // clean prf
01425     FILLFUN3(out); // Remainder of funhead, if it is larger than 3
01426
01427     const bool with_arghead = true;
01428     const bool must_fit_term = true;
01429     const bool write = true;
01430     // prev_size + 4, to account for final term size and coeff of "1/1"
01431     out += flint::to_argument_poly(BHEAD out, with_arghead, must_fit_term, write, out-args_size+4,
01432         num1.d, var_map);
01433     out += flint::to_argument_poly(BHEAD out, with_arghead, must_fit_term, write, out-args_size+4,
01434         den1.d, var_map);
01435
01436     *args_size = out - args_size + 1; // The +1 is to include the function ID
01437     AT.WorkPointer = out;
01438 }
01439 /*
01440 #] flint::ratfun_add_poly :
01441
01442 #[ flint::ratfun_normalize_mpoly :
01443 */
01444 // Multiply and simplify occurrences of the multi-variate FORM rational polynomials found in term.
01445 // The final term is written in place, with the rational polynomial at the end.
01446 void flint::ratfun_normalize_mpoly(PHEAD WORD *term, const var_map_t &var_map) {
01447
01448     // The length of the coefficient
01449     const WORD ncoeff = (term + *term)[-1];
01450     // The end of the term data, before the coefficient:
01451     const WORD *tstop = term + *term - ABS(ncoeff);
01452
01453     flint::mpoly_ctx ctx(var_map.size());
01454     flint::mpoly num1(ctx.d), den1(ctx.d), num2(ctx.d), den2(ctx.d), gcd(ctx.d);
01455
01456     // Start with "trivial" polynomials, and multiply in the num and den of the prf which appear.
01457     flint::fmpz tmpNum, tmpDen;
01458     flint::fmpz_set_form(tmpNum.d, (UWORD*)tstop, ncoeff/2);
01459     flint::fmpz_set_form(tmpDen.d, (UWORD*)tstop+ABS(ncoeff/2), ABS(ncoeff/2));
01460     fmpz_mpoly_set_fmpz(num1.d, tmpNum.d, ctx.d);
01461     fmpz_mpoly_set_fmpz(den1.d, tmpDen.d, ctx.d);
01462
01463     // Loop over the occurrences of PolyFun in the term, and multiply in to num1, den1.
01464     // s tracks where we are writing the non-PolyFun term data. The final PolyFun will
01465     // go at the end.
01466     WORD* term_size = term;
01467     WORD* s = term + 1;
01468     for ( WORD *t = term + 1; t < tstop; ) {
01469         if ( *t == AR.PolyFun ) {
01470             flint::ratfun_read_mpoly(t, num2.d, den2.d, var_map, ctx.d);
01471
01472             // get gcd of num1,den2 and num2,den1 and then assemble
01473             fmpz_mpoly_gcd_cofactors(gcd.d, num1.d, den2.d, num1.d, den2.d, ctx.d);
01474             fmpz_mpoly_gcd_cofactors(gcd.d, num2.d, den1.d, num2.d, den1.d, ctx.d);
01475
01476             fmpz_mpoly_mul(num1.d, num1.d, num2.d, ctx.d);
01477             fmpz_mpoly_mul(den1.d, den1.d, den2.d, ctx.d);
01478
01479             t += t[1];
01480         }
01481
01482         else {
01483             // Not a PolyFun, just copy or skip over
01484             WORD i = t[1];
01485             if ( s != t ) { NCOPY(s,t,i); }
01486             else { t += i; s += i; }
01487         }
01488     }
01489
01490     flint::util::fix_sign_fmpz_mpoly_ratfun(num1.d, den1.d, ctx.d);
01491
01492     // Result in FORM notation:
01493     WORD* out = s;
01494     *out++ = AR.PolyFun;
01495     WORD* args_size = out++;
01496     WORD* args_flag = out++;
01497     *args_flag &= ~MUSTCLEANPRF;
01498
01499     const bool with_arghead = true;
01500     const bool must_fit_term = true;
01501     const bool write = true;
01502     out += flint::to_argument_mpoly(BHEAD out, with_arghead, must_fit_term, write, out-term_size,
01503         num1.d, var_map, ctx.d);
01504     out += flint::to_argument_mpoly(BHEAD out, with_arghead, must_fit_term, write, out-term_size,
01505         den1.d, var_map, ctx.d);
01506
01507     *args_size = out - args_size + 1; // The +1 is to include the function ID
01508

```

```

01509 // +3 for the coefficient of 1/1, which is added after the check
01510 if ( sizeof(WORD)*(out-term_size+3) > (size_t)AM.MaxTer ) {
01511     MLOCK(ErrorMessageLock);
01512     MesPrint("flint::ratfun_normalize: output exceeds MaxTermSize");
01513     MUNLOCK(ErrorMessageLock);
01514     Terminate(-1);
01515 }
01516
01517 *out++ = 1;
01518 *out++ = 1;
01519 *out++ = 3; // the term's coefficient is now 1/1
01520
01521 *term_size = out - term_size;
01522 }
01523 /*
01524 #] flint::ratfun_normalize_mpoly :
01525 #[ flint::ratfun_normalize_poly :
01526 */
01527 // Multiply and simplify occurrences of the uni-variate FORM rational polynomials found in term.
01528 // The final term is written in place, with the rational polynomial at the end.
01529 void flint::ratfun_normalize_poly(PHEAD WORD *term, const var_map_t &var_map) {
01530
01531     // The length of the coefficient
01532     const WORD ncoeff = (term + *term)[-1];
01533     // The end of the term data, before the coefficient:
01534     const WORD *tstop = term + *term - ABS(ncoeff);
01535
01536     flint::poly num1, den1, num2, den2, gcd;
01537
01538     // Start with "trivial" polynomials, and multiply in the num and den of the prf which appear.
01539     flint::fmpz tmpNum, tmpDen;
01540     flint::fmpz_set_form(tmpNum.d, (UWORD*)tstop, ncoeff/2);
01541     flint::fmpz_set_form(tmpDen.d, (UWORD*)tstop+ABS(ncoeff/2), ABS(ncoeff/2));
01542     fmpz_poly_set_fmpz(num1.d, tmpNum.d);
01543     fmpz_poly_set_fmpz(den1.d, tmpDen.d);
01544
01545     // Loop over the occurrences of PolyFun in the term, and multiply in to num1, den1.
01546     // s tracks where we are writing the non-PolyFun term data. The final PolyFun will
01547     // go at the end.
01548     WORD* term_size = term;
01549     WORD* s = term + 1;
01550     for ( WORD *t = term + 1; t < tstop; ) {
01551         if ( *t == AR.PolyFun ) {
01552             flint::ratfun_read_poly(t, num2.d, den2.d);
01553
01554             // get gcd of num1,den2 and num2,den1 and then assemble
01555             flint::util::simplify_fmpz_poly(num1.d, den2.d, gcd.d);
01556             flint::util::simplify_fmpz_poly(num2.d, den1.d, gcd.d);
01557
01558             fmpz_poly_mul(num1.d, num1.d, num2.d);
01559             fmpz_poly_mul(den1.d, den1.d, den2.d);
01560
01561             t += t[1];
01562         }
01563
01564         else {
01565             // Not a PolyFun, just copy or skip over
01566             WORD i = t[1];
01567             if ( s != t ) { NCOPY(s,t,i); }
01568             else { t += i; s += i; }
01569         }
01570     }
01571
01572     flint::util::fix_sign_fmpz_poly_ratfun(num1.d, den1.d);
01573
01574     // Result in FORM notation:
01575     WORD* out = s;
01576     *out++ = AR.PolyFun;
01577     WORD* args_size = out++;
01578     WORD* args_flag = out++;
01579     *args_flag &= ~MUSTCLEANPRF;
01580
01581     const bool with_arghead = true;
01582     const bool must_fit_term = true;
01583     const bool write = true;
01584     out += flint::to_argument_poly(BHEAD out, with_arghead, must_fit_term, write, out-term_size,
01585         num1.d, var_map);
01586     out += flint::to_argument_poly(BHEAD out, with_arghead, must_fit_term, write, out-term_size,
01587         den1.d, var_map);
01588
01589     *args_size = out - args_size + 1; // The +1 is to include the function ID
01590
01591     // +3 for the coefficient of 1/1, which is added after the check
01592     if ( sizeof(WORD)*(out-term_size+3) > (size_t)AM.MaxTer ) {
01593         MLOCK(ErrorMessageLock);
01594         MesPrint("flint::ratfun_normalize: output exceeds MaxTermSize");
01595         MUNLOCK(ErrorMessageLock);

```

```

01596     Terminate(-1);
01597 }
01598
01599 *out++ = 1;
01600 *out++ = 1;
01601 *out++ = 3; // the term's coefficient is now 1/1
01602
01603 *term_size = out - term_size;
01604 }
01605 /*
01606 #] flint::ratfun_normalize_poly :
01607
01608 #[ flint::ratfun_read_mpoly :
01609 */
01610 // Read the multi-variate FORM rational polynomial at a and create fmpz_mpoly_t numerator and
01611 // denominator.
01612 void flint::ratfun_read_mpoly(const WORD *a, fmpz_mpoly_t num, fmpz_mpoly_t den,
01613     const var_map_t &var_map, fmpz_mpoly_ctx_t ctx) {
01614
01615     // The end of the arguments:
01616     const WORD* arg_stop = a+a[1];
01617
01618     const bool must_normalize = (a[2] & MUSTCLEANPRF) != 0;
01619
01620     a += FUNHEAD;
01621     if ( a >= arg_stop ) {
01622         MLOCK(ErrorMessageLock);
01623         MesPrint("ERROR: PolyRatFun cannot have zero arguments");
01624         MUNLOCK(ErrorMessageLock);
01625         Terminate(-1);
01626     }
01627
01628     // Polys to collect the "den of the num" and "den of the den".
01629     // Input can arrive like this when enabling the PolyRatFun or moving things into it.
01630     flint::mpoly den_num(ctx), den_den(ctx);
01631
01632     // Read the numerator
01633     flint::from_argument_mpoly(num, den_num.d, a, true, var_map, ctx);
01634     NEXTARG(a);
01635
01636     if ( a < arg_stop ) {
01637         // Read the denominator
01638         flint::from_argument_mpoly(den, den_den.d, a, true, var_map, ctx);
01639         NEXTARG(a);
01640     }
01641     else {
01642         // The denominator is 1
01643         MLOCK(ErrorMessageLock);
01644         MesPrint("implement this");
01645         MUNLOCK(ErrorMessageLock);
01646         Terminate(-1);
01647     }
01648     if ( a < arg_stop ) {
01649         MLOCK(ErrorMessageLock);
01650         MesPrint("ERROR: PolyRatFun cannot have more than two arguments");
01651         MUNLOCK(ErrorMessageLock);
01652         Terminate(-1);
01653     }
01654
01655     // Multiply the num by den_den and den by den_num:
01656     fmpz_mpoly_mul(num, num, den_den.d, ctx);
01657     fmpz_mpoly_mul(den, den, den_num.d, ctx);
01658
01659     if ( must_normalize ) {
01660         flint::mpoly gcd(ctx);
01661         fmpz_mpoly_gcd_cofactors(gcd.d, num, den, num, den, ctx);
01662     }
01663 }
01664 /*
01665 #] flint::ratfun_read_mpoly :
01666 #[ flint::ratfun_read_poly :
01667 */
01668 // Read the uni-variate FORM rational polynomial at a and create fmpz_mpoly_t numerator and
01669 // denominator.
01670 void flint::ratfun_read_poly(const WORD *a, fmpz_poly_t num, fmpz_poly_t den) {
01671
01672     // The end of the arguments:
01673     const WORD* arg_stop = a+a[1];
01674
01675     const bool must_normalize = (a[2] & MUSTCLEANPRF) != 0;
01676
01677     a += FUNHEAD;
01678     if ( a >= arg_stop ) {
01679         MLOCK(ErrorMessageLock);
01680         MesPrint("ERROR: PolyRatFun cannot have zero arguments");
01681         MUNLOCK(ErrorMessageLock);
01682         Terminate(-1);

```

```

01683     }
01684
01685     // Polys to collect the "den of the num" and "den of the den".
01686     // Input can arrive like this when enabling the PolyRatFun or moving things into it.
01687     flint::poly den_num, den_den;
01688
01689     // Read the numerator
01690     flint::from_argument_poly(num, den_num.d, a, true);
01691     NEXTARG(a);
01692
01693     if ( a < arg_stop ) {
01694         // Read the denominator
01695         flint::from_argument_poly(den, den_den.d, a, true);
01696         NEXTARG(a);
01697     }
01698     else {
01699         // The denominator is 1
01700         MLOCK(ErrorMessageLock);
01701         MesPrint("implement this");
01702         MUNLOCK(ErrorMessageLock);
01703         Terminate(-1);
01704     }
01705     if ( a < arg_stop ) {
01706         MLOCK(ErrorMessageLock);
01707         MesPrint("ERROR: PolyRatFun cannot have more than two arguments");
01708         MUNLOCK(ErrorMessageLock);
01709         Terminate(-1);
01710     }
01711
01712     // Multiply the num by den_den and den by den_num:
01713     fmpz_poly_mul(num, num, den_den.d);
01714     fmpz_poly_mul(den, den, den_num.d);
01715
01716     if ( must_normalize ) {
01717         flint::poly gcd;
01718         flint::util::simplify_fmpz_poly(num, den, gcd.d);
01719     }
01720 }
01721 /*
01722 #] flint::ratfun_read_poly :
01723
01724 #[ flint::startup_init :
01725 */
01726 // The purpose of this function is it work around threading issues in flint, reported in
01727 // https://github.com/flintlib/flint/issues/1652 and fixed in
01728 // https://github.com/flintlib/flint/pull/1658 . This implies versions prior to 3.1.0,
01729 // but at least 3.0.0 (when gr was introduced).
01730 void flint::startup_init(void) {
01731
01732 #if __FLINT_RELEASE >= 30000
01733     // Here we initialize some gr contexts so that their method tables are populated. Crashes have
01734     // otherwise been observed due to these two contexts in particular.
01735     fmpz_t dummy_fmpz;
01736     fmpz_init(dummy_fmpz);
01737     fmpz_set_si(dummy_fmpz, 19);
01738
01739     gr_ctx_t dummy_gr_ctx_fmpz_mod;
01740     gr_ctx_init_fmpz_mod(dummy_gr_ctx_fmpz_mod, dummy_fmpz);
01741     gr_ctx_clear(dummy_gr_ctx_fmpz_mod);
01742
01743     gr_ctx_t dummy_gr_ctx_nmod;
01744     gr_ctx_init_nmod(dummy_gr_ctx_nmod, 19);
01745     gr_ctx_clear(dummy_gr_ctx_nmod);
01746
01747     fmpz_clear(dummy_fmpz);
01748 #endif
01749 }
01750 /*
01751 #] flint::startup_init :
01752
01753 #[ flint::to_argument_mpoly :
01754 */
01755 // Convert a fmpz_mpoly_t to a FORM argument (or 0-terminated list of terms: with_arghead==false).
01756 // If the caller is building an output term, prev_size contains the size of the term so far, to
01757 // check that the output fits in AM.MaxTer if must_fit_term.
01758 // All coefficients will be divided by denscale (which might just be 1).
01759 // If write is false, we never write to out but only track the total would-be size. This lets this
01760 // function be repurposed as a "size of FORM notation" function without duplicating the code.
01761 #define IFW(x) { if ( write ) {x;} }
01762 uint64_t flint::to_argument_mpoly(PHEAD WORD *out, const bool with_arghead,
01763     const bool must_fit_term, const bool write, const uint64_t prev_size, const fmpz_mpoly_t poly,
01764     const var_map_t &var_map, const fmpz_mpoly_ctx_t ctx, const fmpz_t denscale) {
01765
01766     // out is modified later, keep the pointer at entry
01767     const WORD* out_entry = out;
01768
01769     // Track the total size written. We could do this with pointer differences, but if

```



```

01770 // write == false we don't write to or move out to be able to find the size that way.
01771 uint64_t ws = 0;
01772
01773 // Check there is at least space for ARGHEAD WORDs (the arghead or fast-notation number/symbol)
01774 if ( write && must_fit_term && (sizeof(WORD)*(prev_size + ARGHEAD) > (size_t)AM.MaxTer) ) {
01775     MLOCK(ErrorMessageLock);
01776     MesPrint("flint::to_argument_mpoly: output exceeds MaxTermSize");
01777     MUNLOCK(ErrorMessageLock);
01778     Terminate(-1);
01779 }
01780
01781 // Create the inverse of var_map, so we don't have to search it for each symbol written
01782 var_map_t var_map_inv;
01783 for ( auto x: var_map ) {
01784     var_map_inv[x.second] = x.first;
01785 }
01786
01787 vector<int64_t> exponents(var_map.size());
01788 const int64_t n_terms = fmpz_mpoly_length(poly, ctx);
01789
01790 if ( n_terms == 0 ) {
01791     if ( with_arghead ) {
01792         IFW(*out++ = -SNUMBER); ws++;
01793         IFW(*out++ = 0); ws++;
01794         return ws;
01795     }
01796     else {
01797         IFW(*out++ = 0); ws++;
01798         return ws;
01799     }
01800 }
01801
01802 // For dividing out denscale
01803 flint::fmpz coeff, den, gcd;
01804
01805 // The mpoly might be constant or a single symbol with coeff 1. Use fast notation if possible.
01806 if ( with_arghead && n_terms == 1 ) {
01807
01808     if ( fmpz_mpoly_is_fmpz(poly, ctx) ) {
01809         // The mpoly is constant. Use fast notation if the number is an integer and small enough:
01810
01811         fmpz_mpoly_get_term_coeff_fmpz(coeff.d, poly, 0, ctx);
01812         fmpz_set(den.d, denscale);
01813         flint::util::simplify_fmpz(coeff.d, den.d, gcd.d);
01814
01815         if ( fmpz_is_one(den.d) && fmpz_fits_si(coeff.d) ) {
01816             const int64_t fast_coeff = fmpz_get_si(coeff.d);
01817             // While ">=", could work here, FORM does not use fast notation for INT_MIN
01818             if ( fast_coeff > INT32_MIN && fast_coeff <= INT32_MAX ) {
01819                 IFW(*out++ = -SNUMBER); ws++;
01820                 IFW(*out++ = (WORD)fast_coeff); ws++;
01821                 return ws;
01822             }
01823         }
01824     }
01825
01826     else {
01827         fmpz_mpoly_get_term_coeff_fmpz(coeff.d, poly, 0, ctx);
01828         fmpz_set(den.d, denscale);
01829         flint::util::simplify_fmpz(coeff.d, den.d, gcd.d);
01830
01831         if ( fmpz_is_one(coeff.d) && fmpz_is_one(den.d) ) {
01832             // The coefficient is one. Now check the symbol powers:
01833
01834             fmpz_mpoly_get_term_exp_si((slong*)exponents.data(), poly, 0, ctx);
01835             int64_t use_fast = 0;
01836             uint32_t fast_symbol = 0;
01837
01838             for ( size_t i = 0; i < var_map.size(); i++ ) {
01839                 if ( exponents[i] == 1 ) fast_symbol = var_map_inv[i];
01840                 use_fast += exponents[i];
01841             }
01842
01843             // use_fast has collected the total degree. If it is 1, then fast_symbol holds the
01844             code
01845             if ( use_fast == 1 ) {
01846                 IFW(*out++ = -SYMBOL); ws++;
01847                 IFW(*out++ = fast_symbol); ws++;
01848                 return ws;
01849             }
01850         }
01851     }
01852 }
01853
01854 WORD *tmp_coeff = (WORD *)NumberMalloc("flint::to_argument_mpoly");
01855 WORD *tmp_den = (WORD *)NumberMalloc("flint::to_argument_mpoly");

```

```

01856
01857 WORD* arg_size = 0;
01858 WORD* arg_flag = 0;
01859 if ( with_arghead ) {
01860     IFW(arg_size = out++); ws++; // total arg size
01861     IFW(arg_flag = out++); ws++;
01862     IFW(*arg_flag = 0); // clean argument
01863 }
01864
01865 for ( int64_t i = 0; i < n_terms; i++ ) {
01866
01867     fmpz_mpoly_get_term_exp_si((slong*)exponents.data(), poly, i, ctx);
01868
01869     fmpz_mpoly_get_term_coeff_fmpz(coeff.d, poly, i, ctx);
01870     fmpz_set(den.d, denscale);
01871     flint::util::simplify_fmpz(coeff.d, den.d, gcd.d);
01872
01873     uint32_t num_symbols = 0;
01874     for ( size_t j = 0; j < var_map.size(); j++ ) {
01875         if ( exponents[j] != 0 ) { num_symbols += 1; }
01876     }
01877
01878     // Convert the coefficient, write in temporary space
01879     const WORD num_size = flint::fmpz_get_form(coeff.d, tmp_coeff);
01880     const WORD den_size = flint::fmpz_get_form(den.d, tmp_den);
01881     const WORD coeff_size = [num_size, den_size] () -> WORD {
01882         WORD size = ABS(num_size) > ABS(den_size) ? ABS(num_size) : ABS(den_size);
01883         return size * SGN(num_size) * SGN(den_size);
01884     }();
01885
01886     // Now we have the number of symbols and the coeff size, we can determine the output size.
01887     // Check it fits if necessary: term size, num,den of "coeff_size", +1 for total coeff size
01888     uint64_t current_size = prev_size + ws + 1 + 2*ABS(coeff_size) + 1;
01889     if ( num_symbols ) {
01890         // symbols header, code,power of each symbol:
01891         current_size += 2 + 2*num_symbols;
01892     }
01893     if ( write && must_fit_term && (sizeof(WORD)*current_size > (size_t)AM.MaxTer) ) {
01894         MLOCK(ErrorMessageLock);
01895         MesPrint("flint::to_argument_mpoly: output exceeds MaxTermSize");
01896         MUNLOCK(ErrorMessageLock);
01897         Terminate(-1);
01898     }
01899
01900     WORD* term_size = 0;
01901     IFW(term_size = out++); ws++;
01902     if ( num_symbols ) {
01903         IFW(*out++ = SYMBOL); ws++;
01904         WORD* symbol_size = 0;
01905         IFW(symbol_size = out++); ws++;
01906         IFW(*symbol_size = 2);
01907
01908         for ( size_t j = 0; j < var_map.size(); j++ ) {
01909             if ( exponents[j] != 0 ) {
01910                 IFW(*out++ = var_map_inv[j]); ws++;
01911                 IFW(*out++ = exponents[j]); ws++;
01912                 IFW(*symbol_size += 2);
01913             }
01914         }
01915     }
01916
01917     // Copy numerator
01918     for ( WORD j = 0; j < ABS(num_size); j++ ) {
01919         IFW(*out++ = tmp_coeff[j]); ws++;
01920     }
01921     for ( WORD j = ABS(num_size); j < ABS(coeff_size); j++ ) {
01922         IFW(*out++ = 0); ws++;
01923     }
01924     // Copy denominator
01925     for ( WORD j = 0; j < ABS(den_size); j++ ) {
01926         IFW(*out++ = tmp_den[j]); ws++;
01927     }
01928     for ( WORD j = ABS(den_size); j < ABS(coeff_size); j++ ) {
01929         IFW(*out++ = 0); ws++;
01930     }
01931
01932     IFW(*out = 2*ABS(coeff_size) + 1); // the size of the coefficient
01933     IFW(if ( coeff_size < 0 ) { *out = -( *out ); });
01934     IFW(out++); ws++;
01935
01936     IFW(*term_size = out - term_size);
01937 }
01938
01939 if ( with_arghead ) {
01940     IFW(*arg_size = out - arg_size);
01941     if ( write ) {
01942         // Sort into form highfirst ordering

```

```

01943         flint::form_sort(BHEAD (WORD*) (out_entry));
01944     }
01945 }
01946 else {
01947     // with no arghead, we write a terminating zero
01948     IFW(*out++ = 0); ws++;
01949 }
01950
01951 NumberFree(tmp_coeff, "flint::to_argument_mpoly");
01952 NumberFree(tmp_den, "flint::to_argument_mpoly");
01953
01954 return ws;
01955 }
01956
01957 // If no denscale argument is supplied, just set it to 1 and call the usual function
01958 uint64_t flint::to_argument_mpoly(PHEAD WORD *out, const bool with_arghead,
01959     const bool must_fit_term, const bool write, const uint64_t prev_size, const fmpz_mpoly_t poly,
01960     const var_map_t &var_map, const fmpz_mpoly_ctx_t ctx) {
01961     flint::fmpz tmp;
01962     fmpz_set_ui(tmp.d, 1);
01963
01964     uint64_t ret = flint::to_argument_mpoly(BHEAD out, with_arghead, must_fit_term, write,
01965         prev_size, poly, var_map, ctx, tmp.d);
01966
01967     return ret;
01968 }
01969
01970 /*
01971 #] flint::to_argument_mpoly :
01972 #[ flint::to_argument_poly :
01973 */
01974 // Convert a fmpz_poly_t to a FORM argument (or 0-terminated list of terms: with_arghead==false).
01975 // If the caller is building an output term, prev_size contains the size of the term so far, to
01976 // check that the output fits in AM.MaxTer if must_fit_term.
01977 // All coefficients will be divided by denscale (which might just be 1).
01978 // If write is false, we never write to out but only track the total would-be size. This lets this
01979 // function be repurposed as a "size of FORM notation" function without duplicating the code.
01980 uint64_t flint::to_argument_poly(PHEAD WORD *out, const bool with_arghead,
01981     const bool must_fit_term, const bool write, const uint64_t prev_size, const fmpz_poly_t poly,
01982     const var_map_t &var_map, const fmpz_t denscale) {
01983     // Track the total size written. We could do this with pointer differences, but if
01984     // write == false we don't write to or move out to be able to find the size that way.
01985     uint64_t ws = 0;
01986
01987     // Check there is at least space for ARGHEAD WORDs (the arghead or fast-notation number/symbol)
01988     if ( write && must_fit_term && (sizeof(WORD)*(prev_size + ARGHEAD) > (size_t)AM.MaxTer) ) {
01989         MLOCK(ErrorMessageLock);
01990         MesPrint("flint::to_argument_poly: output exceeds MaxTermSize");
01991         MUNLOCK(ErrorMessageLock);
01992         Terminate(-1);
01993     }
01994
01995     // Create the inverse of var_map, so we don't have to search it for each symbol written
01996     var_map_t var_map_inv;
01997     for ( auto x: var_map ) {
01998         var_map_inv[x.second] = x.first;
01999     }
02000
02001     const int64_t n_terms = fmpz_poly_length(poly);
02002
02003     // The poly is zero
02004     if ( n_terms == 0 ) {
02005         if ( with_arghead ) {
02006             IFW(*out++ = -SNUMBER); ws++;
02007             IFW(*out++ = 0); ws++;
02008             return ws;
02009         }
02010         else {
02011             IFW(*out++ = 0); ws++;
02012             return ws;
02013         }
02014     }
02015
02016     // For dividing out denscale
02017     flint::fmpz coeff, den, gcd;
02018
02019     // The poly is constant, use fast notation if the coefficient is integer and small enough
02020     if ( with_arghead && n_terms == 1 ) {
02021         fmpz_poly_get_coeff_fmpz(coeff.d, poly, 0);
02022         fmpz_set(den.d, denscale);
02023         flint::util::simplify_fmpz(coeff.d, den.d, gcd.d);
02024
02025         if ( fmpz_is_one(den.d) && fmpz_fits_si(coeff.d) ) {
02026             const long fast_coeff = fmpz_get_si(coeff.d);
02027             // While ">=", could work here, FORM does not use fast notation for INT_MIN

```

```

02030         if ( fast_coeff > INT_MIN && fast_coeff <= INT_MAX ) {
02031             IFW(*out++ = -SNUMBER); ws++;
02032             IFW(*out++ = (WORD)fast_coeff); ws++;
02033             return ws;
02034         }
02035     }
02036 }
02037
02038 // The poly might be a single symbol with coeff 1, use fast notation if so.
02039 if ( with_arghead && n_terms == 2 ) {
02040     if ( fmpz_is_zero(fmpz_poly_get_coeff_ptr(poly, 0)) ) {
02041         // The constant term is zero
02042
02043         fmpz_poly_get_coeff_fmpz(coeff.d, poly, 1);
02044         fmpz_set(den.d, denscale);
02045         flint::util::simplify_fmpz(coeff.d, den.d, gcd.d);
02046
02047         if ( fmpz_is_one(coeff.d) && fmpz_is_one(den.d) ) {
02048             // Single symbol with coeff 1. Use fast notation:
02049             IFW(*out++ = -SYMBOL); ws++;
02050             IFW(*out++ = var_map_inv[0]); ws++;
02051             return ws;
02052         }
02053     }
02054 }
02055
02056 WORD *tmp_coeff = (WORD *)NumberMalloc("flint::to_argument_poly");
02057 WORD *tmp_den = (WORD *)NumberMalloc("flint::to_argument_mpoly");
02058
02059 WORD* arg_size = 0;
02060 WORD* arg_flag = 0;
02061 if ( with_arghead ) {
02062     IFW(arg_size = out++); ws++; // total arg size
02063     IFW(arg_flag = out++); ws++;
02064     IFW(*arg_flag = 0); // clean argument
02065 }
02066
02067 // In reverse, since we want a "highfirst" output
02068 for ( int64_t i = n_terms-1; i >= 0; i-- ) {
02069
02070     // fmpz_poly is dense, there might be many zero coefficients:
02071     if ( !fmpz_is_zero(fmpz_poly_get_coeff_ptr(poly, i)) ) {
02072
02073         fmpz_poly_get_coeff_fmpz(coeff.d, poly, i);
02074         fmpz_set(den.d, denscale);
02075         flint::util::simplify_fmpz(coeff.d, den.d, gcd.d);
02076
02077         // Convert the coefficient, write in temporary space
02078         const WORD num_size = flint::fmpz_get_form(coeff.d, tmp_coeff);
02079         const WORD den_size = flint::fmpz_get_form(den.d, tmp_den);
02080         const WORD coeff_size = [num_size, den_size] () -> WORD {
02081             WORD size = ABS(num_size) > ABS(den_size) ? ABS(num_size) : ABS(den_size);
02082             return size * SGN(num_size) * SGN(den_size);
02083         }();
02084
02085         // Now we have the coeff size, we can determine the output size
02086         // Check it fits if necessary: symbol code, power, num, den of "coeff_size",
02087         // +1 for total coeff size
02088         uint64_t current_size = prev_size + ws + 1 + 2*ABS(coeff_size) + 1;
02089         if ( i > 0 ) {
02090             // and also symbols header, code, power of the symbol
02091             current_size += 4;
02092         }
02093         if ( write && must_fit_term && (sizeof(WORD)*current_size > (size_t)AM.MaxTer) ) {
02094             MLOCK(ErrorMessageLock);
02095             MesPrint("flint::to_argument_poly: output exceeds MaxTermSize");
02096             MUNLOCK(ErrorMessageLock);
02097             Terminate(-1);
02098         }
02099
02100         WORD* term_size = 0;
02101         IFW(term_size = out++); ws++;
02102
02103         if ( i > 0 ) {
02104             IFW(*out++ = SYMBOL); ws++;
02105             IFW(*out++ = 4); ws++; // The symbol array size, it is univariate
02106             IFW(*out++ = var_map_inv[0]); ws++;
02107             IFW(*out++ = i); ws++;
02108         }
02109
02110         // Copy numerator
02111         for ( WORD j = 0; j < ABS(num_size); j++ ) {
02112             IFW(*out++ = tmp_coeff[j]); ws++;
02113         }
02114         for ( WORD j = ABS(num_size); j < ABS(coeff_size); j++ ) {
02115             IFW(*out++ = 0); ws++;
02116         }
02117     }

```

```

02117         // Copy denominator
02118         for ( WORD j = 0; j < ABS(den_size); j++ ) {
02119             IFW(*out++ = tmp_den[j]); ws++;
02120         }
02121         for ( WORD j = ABS(den_size); j < ABS(coeff_size); j++ ) {
02122             IFW(*out++ = 0); ws++;
02123         }
02124
02125         IFW(*out = 2*ABS(coeff_size) + 1); // the size of the coefficient
02126         IFW(if ( coeff_size < 0 ) { *out = -(*out); });
02127         IFW(out++); ws++;
02128
02129         IFW(*term_size = out - term_size);
02130     }
02131 }
02132 }
02133
02134 if ( with_arghead ) {
02135     IFW(*arg_size = out - arg_size);
02136 }
02137 else {
02138     // with no arghead, we write a terminating zero
02139     IFW(*out++ = 0); ws++;
02140 }
02141
02142 NumberFree(tmp_coeff, "flint::to_argument_poly");
02143 NumberFree(tmp_den, "flint::to_argument_poly");
02144
02145 return ws;
02146 }
02147
02148 // If no denscale argument is supplied, just set it to 1 and call the usual function
02149 uint64_t flint::to_argument_poly(PHEAD WORD *out, const bool with_arghead,
02150     const bool must_fit_term, const bool write, const uint64_t prev_size, const fmpz_poly_t poly,
02151     const var_map_t &var_map) {
02152
02153     fmpz_t tmp;
02154     fmpz_set_ui(tmp.d, 1);
02155
02156     uint64_t ret = flint::to_argument_poly(BHEAD out, with_arghead, must_fit_term, write, prev_size,
02157         poly, var_map, tmp.d);
02158
02159     return ret;
02160 }
02161 /*
02162 #] flint::to_argument_poly :
02163 */
02164
02165 // Utility functions
02166 /*
02167 #[ flint::util::simplify_fmpz :
02168 */
02169 // Divide the GCD out of num and den
02170 inline void flint::util::simplify_fmpz(fmpz_t num, fmpz_t den, fmpz_t gcd) {
02171     fmpz_gcd(gcd, num, den);
02172     if ( !fmpz_is_one(gcd) ) {
02173         fmpz_divexact(num, num, gcd);
02174         fmpz_divexact(den, den, gcd);
02175     }
02176 }
02177 /*
02178 #] flint::util::simplify_fmpz :
02179 #[ flint::util::simplify_fmpz_poly :
02180 */
02181 // Divide the GCD out of num and den
02182 inline void flint::util::simplify_fmpz_poly(fmpz_poly_t num, fmpz_poly_t den, fmpz_poly_t gcd) {
02183     fmpz_poly_gcd(gcd, num, den);
02184     if ( !fmpz_poly_is_one(gcd) ) {
02185 #if __FLINT_RELEASE >= 30100
02186         // This should be faster than fmpz_poly_div, see https://github.com/flintlib/flint/pull/1766
02187         fmpz_poly_divexact(num, num, gcd);
02188         fmpz_poly_divexact(den, den, gcd);
02189     #else
02190         fmpz_poly_div(num, num, gcd);
02191         fmpz_poly_div(den, den, gcd);
02192     #endif
02193     }
02194 }
02195 /*
02196 #] flint::util::simplify_fmpz_poly :
02197 #[ flint::util::fix_sign_fmpz_mpoly_ratfun :
02198 */
02199 inline void flint::util::fix_sign_fmpz_mpoly_ratfun(fmpz_mpoly_t num, fmpz_mpoly_t den,
02200     const fmpz_mpoly_ctx_t ctx) {
02201     // Fix sign to align with poly: the leading denominator term should have a positive coeff
02202     if ( fmpz_sgn(fmpz_mpoly_term_coeff_ref(den, 0, ctx)) == -1 ) {
02203         fmpz_mpoly_neg(num, num, ctx);

```

```

02204     fmpz_mpoly_neg(den, den, ctx);
02205 }
02206 }
02207 /*
02208 #] flint::util::fix_sign_fmpz_mpoly_ratfun :
02209 #[ flint::util::fix_sign_fmpz_poly_ratfun :
02210 */
02211 inline void flint::util::fix_sign_fmpz_poly_ratfun(fmpz_poly_t num, fmpz_poly_t den) {
02212     // Fix sign to align with poly: the leading denominator term should have a positive coeff
02213     if ( fmpz_sgn(fmpz_poly_get_coeff_ptr(den, fmpz_poly_degree(den))) == -1 ) {
02214         fmpz_poly_neg(num, num);
02215         fmpz_poly_neg(den, den);
02216     }
02217 }
02218 /*
02219 #] flint::util::fix_sign_fmpz_poly_ratfun :
02220 */

```

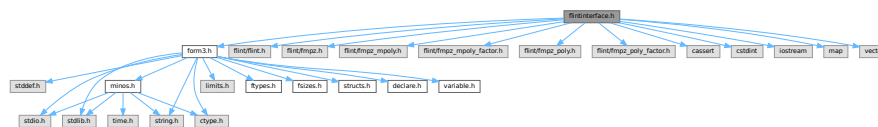
4.39 flintinterface.h File Reference

```

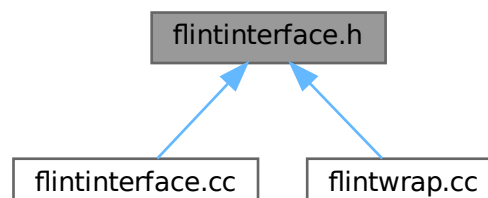
#include "form3.h"
#include <flint/flint.h>
#include <flint/fmpz.h>
#include <flint/fmpz_mpoly.h>
#include <flint/fmpz_mpoly_factor.h>
#include <flint/fmpz_poly.h>
#include <flint/fmpz_poly_factor.h>
#include <cassert>
#include <cstdint>
#include <iostream>
#include <map>
#include <vector>

```

Include dependency graph for flintinterface.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- class [flint::fmpz](#)
- class [flint::poly](#)
- class [flint::poly_factor](#)
- class [flint::mpoly](#)
- class [flint::mpoly_factor](#)
- class [flint::mpoly_ctx](#)

Typedefs

- typedef std::map< uint32_t, uint32_t > [flint::var_map_t](#)

Functions

- void [flint::cleanup](#) (void)
- void [flint::cleanup_master](#) (void)
- WORD * [flint::divmod_mpoly](#) (PHEAD const WORD *, const WORD *, const bool, const WORD, const var_map_t &)
- WORD * [flint::divmod_poly](#) (PHEAD const WORD *, const WORD *, const bool, const WORD, const var_map_t &)
- WORD * [flint::factorize_mpoly](#) (PHEAD const WORD *, WORD *, const bool, const bool, const var_map_t &)
- WORD * [flint::factorize_poly](#) (PHEAD const WORD *, WORD *, const bool, const bool, const var_map_t &)
- void [flint::form_sort](#) (PHEAD WORD *)
- uint64_t [flint::from_argument_mpoly](#) (fmpz_mpoly_t, fmpz_mpoly_t, const WORD *, const bool, const var_map_t &, const fmpz_mpoly_ctx_t)
- uint64_t [flint::from_argument_poly](#) (fmpz_poly_t, fmpz_poly_t, const WORD *, const bool)
- WORD [flint::fmpz_get_form](#) (fmpz_t, WORD *)
- void [flint::fmpz_set_form](#) (fmpz_t, UWORD *, WORD)
- WORD * [flint::gcd_mpoly](#) (PHEAD const WORD *, const WORD *, const WORD, const var_map_t &)
- WORD * [flint::gcd_poly](#) (PHEAD const WORD *, const WORD *, const WORD, const var_map_t &)
- var_map_t [flint::get_variables](#) (const vector< WORD * > &, const bool, const bool)
- WORD * [flint::inverse_poly](#) (PHEAD const WORD *, const WORD *, const var_map_t &)
- WORD * [flint::mul_mpoly](#) (PHEAD const WORD *, const WORD *, const var_map_t &)
- WORD * [flint::mul_poly](#) (PHEAD const WORD *, const WORD *, const var_map_t &)
- void [flint::ratfun_add_mpoly](#) (PHEAD const WORD *, const WORD *, WORD *, const var_map_t &)
- void [flint::ratfun_add_poly](#) (PHEAD const WORD *, const WORD *, WORD *, const var_map_t &)
- void [flint::ratfun_normalize_mpoly](#) (PHEAD WORD *, const var_map_t &)
- void [flint::ratfun_normalize_poly](#) (PHEAD WORD *, const var_map_t &)
- void [flint::ratfun_read_mpoly](#) (const WORD *, fmpz_mpoly_t, fmpz_mpoly_t, const var_map_t &, fmpz_mpoly_ctx_t)
- void [flint::ratfun_read_poly](#) (const WORD *, fmpz_poly_t, fmpz_poly_t)
- void [flint::startup_init](#) (void)
- uint64_t [flint::to_argument_mpoly](#) (PHEAD WORD *, const bool, const bool, const bool, const uint64_t, const fmpz_mpoly_t, const var_map_t &, const fmpz_mpoly_ctx_t)
- uint64_t [flint::to_argument_mpoly](#) (PHEAD WORD *, const bool, const bool, const bool, const uint64_t, const fmpz_mpoly_t, const var_map_t &, const fmpz_mpoly_ctx_t, const fmpz_t)
- uint64_t [flint::to_argument_poly](#) (PHEAD WORD *, const bool, const bool, const bool, const uint64_t, const fmpz_poly_t, const var_map_t &)
- uint64_t [flint::to_argument_poly](#) (PHEAD WORD *, const bool, const bool, const bool, const uint64_t, const fmpz_poly_t, const var_map_t &, const fmpz_t)
- void [flint::util::simplify_fmpz](#) (fmpz_t, fmpz_t, fmpz_t)
- void [flint::util::simplify_fmpz_poly](#) (fmpz_poly_t, fmpz_poly_t, fmpz_poly_t)
- void [flint::util::fix_sign_fmpz_mpoly_ratfun](#) (fmpz_mpoly_t, fmpz_mpoly_t, const fmpz_mpoly_ctx_t)
- void [flint::util::fix_sign_fmpz_poly_ratfun](#) (fmpz_poly_t, fmpz_poly_t)

4.39.1 Detailed Description

Prototypes for functions in [flintinterface.cc](#)

Definition in file [flintinterface.h](#).

4.39.2 Typedef Documentation

4.39.2.1 var_map_t

```
typedef std::map<uint32_t,uint32_t> flint::var_map_t
```

Definition at line 50 of file [flintinterface.h](#).

4.39.3 Function Documentation

4.39.3.1 cleanup()

```
void flint::cleanup (  
    void )
```

Definition at line 43 of file [flintinterface.cc](#).

4.39.3.2 cleanup_master()

```
void flint::cleanup_master (  
    void )
```

Definition at line 50 of file [flintinterface.cc](#).

4.39.3.3 divmod_mpoly()

```
WORD * flint::divmod_mpoly (  
    PHEAD const WORD * a,  
    const WORD * b,  
    const bool return_rem,  
    const WORD must_fit_term,  
    const var_map_t & var_map )
```

Definition at line 58 of file [flintinterface.cc](#).

4.39.3.4 divmod_poly()

```
WORD * flint::divmod_poly (  
    PHEAD const WORD * a,  
    const WORD * b,  
    const bool return_rem,  
    const WORD must_fit_term,  
    const var_map_t & var_map )
```

Definition at line 128 of file [flintinterface.cc](#).

4.39.3.5 factorize_mpoly()

```
WORD * flint::factorize_mpoly (
    PHEAD const WORD * argin,
    WORD * argout,
    const bool with_arghead,
    const bool is_fun_arg,
    const var_map_t & var_map )
```

Definition at line 202 of file [flintinterface.cc](#).

4.39.3.6 factorize_poly()

```
WORD * flint::factorize_poly (
    PHEAD const WORD * argin,
    WORD * argout,
    const bool with_arghead,
    const bool is_fun_arg,
    const var_map_t & var_map )
```

Definition at line 350 of file [flintinterface.cc](#).

4.39.3.7 form_sort()

```
void flint::form_sort (
    PHEAD WORD * terms )
```

Definition at line 423 of file [flintinterface.cc](#).

4.39.3.8 from_argument_mpoly()

```
uint64_t flint::from_argument_mpoly (
    fmpz_mpoly_t poly,
    fmpz_mpoly_t denpoly,
    const WORD * args,
    const bool with_arghead,
    const var_map_t & var_map,
    const fmpz_mpoly_ctx_t ctx )
```

Definition at line 478 of file [flintinterface.cc](#).

4.39.3.9 from_argument_poly()

```
uint64_t flint::from_argument_poly (
    fmpz_poly_t poly,
    fmpz_poly_t denpoly,
    const WORD * args,
    const bool with_arghead )
```

Definition at line 621 of file [flintinterface.cc](#).

4.39.3.10 `mpz_get_form()`

```
WORD flint::mpz_get_form (
    mpz_t z,
    WORD * a )
```

Definition at line 751 of file [flintinterface.cc](#).

4.39.3.11 `mpz_set_form()`

```
void flint::mpz_set_form (
    mpz_t z,
    UWORD * a,
    WORD na )
```

Definition at line 816 of file [flintinterface.cc](#).

4.39.3.12 `gcd_mpoly()`

```
WORD * flint::gcd_mpoly (
    PHEAD const WORD * a,
    const WORD * b,
    const WORD must_fit_term,
    const var_map_t & var_map )
```

Definition at line 882 of file [flintinterface.cc](#).

4.39.3.13 `gcd_poly()`

```
WORD * flint::gcd_poly (
    PHEAD const WORD * a,
    const WORD * b,
    const WORD must_fit_term,
    const var_map_t & var_map )
```

Definition at line 978 of file [flintinterface.cc](#).

4.39.3.14 `get_variables()`

```
flint::var_map_t flint::get_variables (
    const vector< WORD * > & es,
    const bool with_arghead,
    const bool sort_vars )
```

Definition at line 1060 of file [flintinterface.cc](#).

4.39.3.15 inverse_poly()

```
WORD * flint::inverse_poly (
    PHEAD const WORD * a,
    const WORD * b,
    const var_map_t & var_map )
```

Definition at line 1162 of file [flintinterface.cc](#).

4.39.3.16 mul_mpoly()

```
WORD * flint::mul_mpoly (
    PHEAD const WORD * a,
    const WORD * b,
    const var_map_t & var_map )
```

Definition at line 1238 of file [flintinterface.cc](#).

4.39.3.17 mul_poly()

```
WORD * flint::mul_poly (
    PHEAD const WORD * a,
    const WORD * b,
    const var_map_t & var_map )
```

Definition at line 1290 of file [flintinterface.cc](#).

4.39.3.18 ratfun_add_mpoly()

```
void flint::ratfun_add_mpoly (
    PHEAD const WORD * t1,
    const WORD * t2,
    WORD * out,
    const var_map_t & var_map )
```

Definition at line 1341 of file [flintinterface.cc](#).

4.39.3.19 ratfun_add_poly()

```
void flint::ratfun_add_poly (
    PHEAD const WORD * t1,
    const WORD * t2,
    WORD * out,
    const var_map_t & var_map )
```

Definition at line 1393 of file [flintinterface.cc](#).

4.39.3.20 `ratfun_normalize_mpoly()`

```
void flint::ratfun_normalize_mpoly (
    PHEAD WORD * term,
    const var_map_t & var_map )
```

Definition at line 1446 of file [flintinterface.cc](#).

4.39.3.21 `ratfun_normalize_poly()`

```
void flint::ratfun_normalize_poly (
    PHEAD WORD * term,
    const var_map_t & var_map )
```

Definition at line 1529 of file [flintinterface.cc](#).

4.39.3.22 `ratfun_read_mpoly()`

```
void flint::ratfun_read_mpoly (
    const WORD * a,
    fmpz_mpoly_t num,
    fmpz_mpoly_t den,
    const var_map_t & var_map,
    fmpz_mpoly_ctx_t ctx )
```

Definition at line 1612 of file [flintinterface.cc](#).

4.39.3.23 `ratfun_read_poly()`

```
void flint::ratfun_read_poly (
    const WORD * a,
    fmpz_poly_t num,
    fmpz_poly_t den )
```

Definition at line 1670 of file [flintinterface.cc](#).

4.39.3.24 `startup_init()`

```
void flint::startup_init (
    void )
```

Definition at line 1730 of file [flintinterface.cc](#).

4.39.3.25 to_argument_mpoly() [1/2]

```
uint64_t flint::to_argument_mpoly (
    PHEAD WORD * out,
    const bool with_arghead,
    const bool must_fit_term,
    const bool write,
    const uint64_t prev_size,
    const fmpz_mpoly_t poly,
    const var_map_t & var_map,
    const fmpz_mpoly_ctx_t ctx )
```

Definition at line 1958 of file [flintinterface.cc](#).

4.39.3.26 to_argument_mpoly() [2/2]

```
uint64_t flint::to_argument_mpoly (
    PHEAD WORD * out,
    const bool with_arghead,
    const bool must_fit_term,
    const bool write,
    const uint64_t prev_size,
    const fmpz_mpoly_t poly,
    const var_map_t & var_map,
    const fmpz_mpoly_ctx_t ctx,
    const fmpz_t denscale )
```

Definition at line 1762 of file [flintinterface.cc](#).

4.39.3.27 to_argument_poly() [1/2]

```
uint64_t flint::to_argument_poly (
    PHEAD WORD * out,
    const bool with_arghead,
    const bool must_fit_term,
    const bool write,
    const uint64_t prev_size,
    const fmpz_poly_t poly,
    const var_map_t & var_map )
```

Definition at line 2149 of file [flintinterface.cc](#).

4.39.3.28 to_argument_poly() [2/2]

```
uint64_t flint::to_argument_poly (
    PHEAD WORD * out,
    const bool with_arghead,
    const bool must_fit_term,
    const bool write,
    const uint64_t prev_size,
    const fmpz_poly_t poly,
    const var_map_t & var_map,
    const fmpz_t denscale )
```

Definition at line 1980 of file [flintinterface.cc](#).

4.39.3.29 `simplify_fmpz()`

```
void flint::util::simplify_fmpz (
    fmpz_t num,
    fmpz_t den,
    fmpz_t gcd ) [inline]
```

Definition at line 2170 of file [flintinterface.cc](#).

4.39.3.30 `simplify_fmpz_poly()`

```
void flint::util::simplify_fmpz_poly (
    fmpz_poly_t num,
    fmpz_poly_t den,
    fmpz_poly_t gcd ) [inline]
```

Definition at line 2182 of file [flintinterface.cc](#).

4.39.3.31 `fix_sign_fmpz_mpoly_ratfun()`

```
void flint::util::fix_sign_fmpz_mpoly_ratfun (
    fmpz_mpoly_t num,
    fmpz_mpoly_t den,
    const fmpz_mpoly_ctx_t ctx ) [inline]
```

Definition at line 2199 of file [flintinterface.cc](#).

4.39.3.32 `fix_sign_fmpz_poly_ratfun()`

```
void flint::util::fix_sign_fmpz_poly_ratfun (
    fmpz_poly_t num,
    fmpz_poly_t den ) [inline]
```

Definition at line 2211 of file [flintinterface.cc](#).

4.40 `flintinterface.h`

[Go to the documentation of this file.](#)

```
00001 #pragma once
00007 extern "C" {
00008 #include "form3.h"
00009 }
00010
00011 #if defined(WINDOWS)
00012 // flint.h defines WORD(xx), which conflicts with the one defined in form3.h.
00013 #undef WORD
00014 #endif
00015
00016 #include <flint/flint.h>
00017 #if __FLINT_RELEASE >= 30000
00018 #include <flint/gr.h>
00019 #endif
00020 #include <flint/fmpz.h>
00021 #include <flint/fmpz_mpoly.h>
00022 #include <flint/fmpz_mpoly_factor.h>
00023 #include <flint/fmpz_poly.h>
```

```

00024 #include <flint/fmpz_poly_factor.h>
00025
00026 #if defined(WINDOWS)
00027 // Redefine WORD here to match form3.h.
00028 #undef WORD
00029 #define WORD FORM_WORD
00030 #endif
00031
00032 #include <cassert>
00033 #include <cstdint>
00034 #include <iostream>
00035 #include <map>
00036 #include <vector>
00037
00038
00039 // The bits of std that are needed:
00040 using std::cout;
00041 using std::endl;
00042 using std::map;
00043 using std::swap;
00044 using std::string;
00045 using std::vector;
00046
00047
00048 namespace flint {
00049
00050     typedef std::map<uint32_t, uint32_t> var_map_t;
00051
00052     // Small wrappers around the flint structs to enable RAII init and clear. "d" represents the
00053     // data, and this member will be passed to flint functions.
00054     class fmpz {
00055     public:
00056         fmpz_t d;
00057         fmpz() { fmpz_init(d); }
00058         ~fmpz() { fmpz_clear(d); }
00059         void print(const string& text) { cout << text; fmpz_print(d); cout << endl; }
00060     };
00061     class poly {
00062     public:
00063         fmpz_poly_t d;
00064         poly() { fmpz_poly_init(d); }
00065         ~poly() { fmpz_poly_clear(d); }
00066         void print(const string& text) {
00067             cout << text; fmpz_poly_print_pretty(d, "x"); cout << endl;
00068         }
00069     };
00070     class poly_factor {
00071     public:
00072         fmpz_poly_factor_t d;
00073         poly_factor() { fmpz_poly_factor_init(d); }
00074         ~poly_factor() { fmpz_poly_factor_clear(d); }
00075     };
00076     class mpoly {
00077     private:
00078         fmpz_mpoly_ctx_struct *ctx; // We need to keep a copy of the context pointer for clearing.
00079     public:
00080         fmpz_mpoly_t d;
00081         explicit mpoly(fmpz_mpoly_ctx_struct *ctx_in) : ctx(ctx_in) { fmpz_mpoly_init(d, ctx); }
00082         ~mpoly() { fmpz_mpoly_clear(d, ctx); }
00083         void print(const string& text) {
00084             cout << text;
00085             fmpz_mpoly_print_pretty(d, 0, ctx);
00086             cout << endl;
00087         }
00088     };
00089     class mpoly_factor {
00090     private:
00091         fmpz_mpoly_ctx_struct *ctx; // We need to keep a copy of the context pointer for clearing.
00092     public:
00093         fmpz_mpoly_factor_t d;
00094         explicit mpoly_factor(fmpz_mpoly_ctx_struct *ctx_in) : ctx(ctx_in) {
00095             fmpz_mpoly_factor_init(d, ctx);
00096         }
00097         ~mpoly_factor() { fmpz_mpoly_factor_clear(d, ctx); }
00098     };
00099     class mpoly_ctx {
00100     public:
00101         fmpz_mpoly_ctx_t d;
00102         explicit mpoly_ctx(int64_t nvars) { fmpz_mpoly_ctx_init(d, nvars, ORD_LEX); }
00103         ~mpoly_ctx() { fmpz_mpoly_ctx_clear(d); }
00104     };
00105
00106     void cleanup(void);
00107     void cleanup_master(void);
00108
00109     WORD* divmod_mpoly(PHEAD const WORD *, const WORD *, const bool, const WORD, const var_map_t &);
00110     WORD* divmod_poly(PHEAD const WORD *, const WORD *, const bool, const WORD, const var_map_t &);

```

```

00111
00112 WORD* factorize_mpoly(PHEAD const WORD *, WORD *, const bool, const bool, const var_map_t &);
00113 WORD* factorize_poly(PHEAD const WORD *, WORD *, const bool, const bool, const var_map_t &);
00114
00115 void form_sort(PHEAD WORD *);
00116
00117 uint64_t from_argument_mpoly(fmpz_mpoly_t, fmpz_mpoly_t, const WORD *, const bool,
00118     const var_map_t &, const fmpz_mpoly_ctx_t);
00119 uint64_t from_argument_poly(fmpz_poly_t, fmpz_poly_t, const WORD *, const bool);
00120
00121 WORD fmpz_get_form(fmpz_t, WORD *);
00122 void fmpz_set_form(fmpz_t, UWORD *, WORD);
00123
00124 WORD* gcd_mpoly(PHEAD const WORD *, const WORD *, const WORD, const var_map_t &);
00125 WORD* gcd_poly(PHEAD const WORD *, const WORD *, const WORD, const var_map_t &);
00126
00127 var_map_t get_variables(const vector<WORD*> &, const bool, const bool);
00128
00129 WORD* inverse_poly(PHEAD const WORD *, const WORD *, const var_map_t &);
00130
00131 WORD* mul_mpoly(PHEAD const WORD *, const WORD *, const var_map_t &);
00132 WORD* mul_poly(PHEAD const WORD *, const WORD *, const var_map_t &);
00133
00134 void ratfun_add_mpoly(PHEAD const WORD *, const WORD *, WORD *, const var_map_t &);
00135 void ratfun_add_poly(PHEAD const WORD *, const WORD *, WORD *, const var_map_t &);
00136
00137 void ratfun_normalize_mpoly(PHEAD WORD *, const var_map_t &);
00138 void ratfun_normalize_poly(PHEAD WORD *, const var_map_t &);
00139
00140 void ratfun_read_mpoly(const WORD *, fmpz_mpoly_t, fmpz_mpoly_t, const var_map_t &,
00141     fmpz_mpoly_ctx_t);
00142 void ratfun_read_poly(const WORD *, fmpz_poly_t, fmpz_poly_t);
00143
00144 void startup_init(void);
00145
00146 uint64_t to_argument_mpoly(PHEAD WORD *, const bool, const bool, const bool, const uint64_t,
00147     const fmpz_mpoly_t, const var_map_t &, const fmpz_mpoly_ctx_t);
00148 uint64_t to_argument_mpoly(PHEAD WORD *, const bool, const bool, const bool, const uint64_t,
00149     const fmpz_mpoly_t, const var_map_t &, const fmpz_mpoly_ctx_t, const fmpz_t);
00150 uint64_t to_argument_poly(PHEAD WORD *, const bool, const bool, const bool, const uint64_t,
00151     const fmpz_poly_t, const var_map_t &);
00152 uint64_t to_argument_poly(PHEAD WORD *, const bool, const bool, const bool, const uint64_t,
00153     const fmpz_poly_t, const var_map_t &, const fmpz_t);
00154
00155 namespace util {
00156
00157     void simplify_fmpz(fmpz_t, fmpz_t, fmpz_t);
00158     void simplify_fmpz_poly(fmpz_poly_t, fmpz_poly_t, fmpz_poly_t);
00159
00160     void fix_sign_fmpz_mpoly_ratfun(fmpz_mpoly_t, fmpz_mpoly_t, const fmpz_mpoly_ctx_t);
00161     void fix_sign_fmpz_poly_ratfun(fmpz_poly_t, fmpz_poly_t);
00162
00163 }
00164
00165
00166 }

```

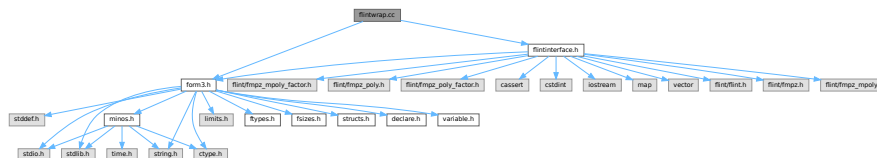
4.41 flintwrap.cc File Reference

```

#include "form3.h"
#include "flintinterface.h"

```

Include dependency graph for flintwrap.cc:



Functions

- void [flint_final_cleanup_thread](#) (void)

- void [flint_final_cleanup_master](#) (void)
- WORD * [flint_div](#) (PHEAD WORD *a, WORD *b, const WORD must_fit_term)
- int [flint_factorize_argument](#) (PHEAD WORD *argin, WORD *argout)
- WORD * [flint_factorize_dollar](#) (PHEAD WORD *argin)
- WORD * [flint_gcd](#) (PHEAD WORD *a, WORD *b, const WORD must_fit_term)
- WORD * [flint_inverse](#) (PHEAD WORD *a, WORD *b)
- WORD * [flint_mul](#) (PHEAD WORD *a, WORD *b)
- WORD * [flint_ratfun_add](#) (PHEAD WORD *t1, WORD *t2)
- int [flint_ratfun_normalize](#) (PHEAD WORD *term)
- WORD * [flint_rem](#) (PHEAD WORD *a, WORD *b, const WORD must_fit_term)
- void [flint_startup_init](#) (void)

4.41.1 Detailed Description

Contains functions to call FLINT interface from the rest of FORM.

Definition in file [flintwrap.cc](#).

4.41.2 Function Documentation

4.41.2.1 flint_final_cleanup_thread()

```
void flint_final_cleanup_thread (  
    void )
```

Definition at line 16 of file [flintwrap.cc](#).

4.41.2.2 flint_final_cleanup_master()

```
void flint_final_cleanup_master (  
    void )
```

Definition at line 23 of file [flintwrap.cc](#).

4.41.2.3 flint_div()

```
WORD * flint_div (  
    PHEAD WORD * a,  
    WORD * b,  
    const WORD must_fit_term )
```

Definition at line 30 of file [flintwrap.cc](#).

4.41.2.4 flint_factorize_argument()

```
int flint_factorize_argument (  
    PHEAD WORD * argin,  
    WORD * argout )
```

Definition at line 51 of file [flintwrap.cc](#).

4.41.2.5 flint_factorize_dollar()

```
WORD * flint_factorize_dollar (
    PHEAD WORD * argin )
```

Definition at line 72 of file [flintwrap.cc](#).

4.41.2.6 flint_gcd()

```
WORD * flint_gcd (
    PHEAD WORD * a,
    WORD * b,
    const WORD must_fit_term )
```

Definition at line 91 of file [flintwrap.cc](#).

4.41.2.7 flint_inverse()

```
WORD * flint_inverse (
    PHEAD WORD * a,
    WORD * b )
```

Definition at line 111 of file [flintwrap.cc](#).

4.41.2.8 flint_mul()

```
WORD * flint_mul (
    PHEAD WORD * a,
    WORD * b )
```

Definition at line 131 of file [flintwrap.cc](#).

4.41.2.9 flint_ratfun_add()

```
WORD * flint_ratfun_add (
    PHEAD WORD * t1,
    WORD * t2 )
```

Definition at line 149 of file [flintwrap.cc](#).

4.41.2.10 flint_ratfun_normalize()

```
int flint_ratfun_normalize (
    PHEAD WORD * term )
```

Definition at line 187 of file [flintwrap.cc](#).

4.41.2.11 flint_rem()

```
WORD * flint_rem (
    PHEAD WORD * a,
    WORD * b,
    const WORD must_fit_term )
```

Definition at line 259 of file [flintwrap.cc](#).

4.41.2.12 flint_startup_init()

```
void flint_startup_init (
    void )
```

Definition at line 280 of file [flintwrap.cc](#).

4.42 flintwrap.cc

[Go to the documentation of this file.](#)

```
00001
00006 extern "C" {
00007 #include "form3.h"
00008 }
00009
00010 #include "flintinterface.h"
00011
00012
00013 /*
00014  #[ flint_final_cleanup_thread :
00015  */
00016 void flint_final_cleanup_thread(void) {
00017     flint::cleanup();
00018 }
00019 /*
00020  #] flint_final_cleanup_thread :
00021  #[ flint_final_cleanup_master :
00022  */
00023 void flint_final_cleanup_master(void) {
00024     flint::cleanup_master();
00025 }
00026 /*
00027  #] flint_final_cleanup_master :
00028  #[ flint_div :
00029  */
00030 WORD* flint_div(PHEAD WORD *a, WORD *b, const WORD must_fit_term) {
00031     // Extract expressions
00032     vector<WORD*> e;
00033     e.push_back(a);
00034     e.push_back(b);
00035     const bool with_arghead = false;
00036     const bool sort_vars = false;
00037     const flint::var_map_t var_map = flint::get_variables(e, with_arghead, sort_vars);
00038
00039     const bool return_rem = false;
00040     if ( var_map.size() > 1 ) {
00041         return flint::divmod_mpoly(BHEAD a, b, return_rem, must_fit_term, var_map);
00042     }
00043     else {
00044         return flint::divmod_poly(BHEAD a, b, return_rem, must_fit_term, var_map);
00045     }
00046 }
00047 /*
00048  #] flint_div :
00049  #[ flint_factorize_argument :
00050  */
00051 int flint_factorize_argument(PHEAD WORD *argin, WORD *argout) {
00052
00053     const bool with_arghead = true;
00054     const bool sort_vars = true;
00055     const flint::var_map_t var_map = flint::get_variables(vector<WORD*>(1,argin), with_arghead,
00056         sort_vars);
```

```

00057
00058     const bool is_fun_arg = true;
00059     if ( var_map.size() > 1 ) {
00060         flint::factorize_mpoly(BHEAD argin, argout, with_arghead, is_fun_arg, var_map);
00061     }
00062     else {
00063         flint::factorize_poly(BHEAD argin, argout, with_arghead, is_fun_arg, var_map);
00064     }
00065
00066     return 0;
00067 }
00068 /*
00069  #] flint_factorize_argument :
00070  #[ flint_factorize_dollar :
00071  */
00072 WORD* flint_factorize_dollar(PHEAD WORD *argin) {
00073
00074     const bool with_arghead = false;
00075     const bool sort_vars = true;
00076     const flint::var_map_t var_map = flint::get_variables(vector<WORD*>(1,argin), with_arghead,
00077         sort_vars);
00078
00079     const bool is_fun_arg = false;
00080     if ( var_map.size() > 1 ) {
00081         return flint::factorize_mpoly(BHEAD argin, NULL, with_arghead, is_fun_arg, var_map);
00082     }
00083     else {
00084         return flint::factorize_poly(BHEAD argin, NULL, with_arghead, is_fun_arg, var_map);
00085     }
00086 }
00087 /*
00088  #] flint_factorize_dollar :
00089  #[ flint_gcd :
00090  */
00091 WORD* flint_gcd(PHEAD WORD *a, WORD *b, const WORD must_fit_term) {
00092     // Extract expressions
00093     vector<WORD *> e;
00094     e.push_back(a);
00095     e.push_back(b);
00096     const bool with_arghead = false;
00097     const bool sort_vars = true;
00098     const flint::var_map_t var_map = flint::get_variables(e, with_arghead, sort_vars);
00099
00100     if ( var_map.size() > 1 ) {
00101         return flint::gcd_mpoly(BHEAD a, b, must_fit_term, var_map);
00102     }
00103     else {
00104         return flint::gcd_poly(BHEAD a, b, must_fit_term, var_map);
00105     }
00106 }
00107 /*
00108  #] flint_gcd :
00109  #[ flint_inverse :
00110  */
00111 WORD* flint_inverse(PHEAD WORD *a, WORD *b) {
00112     // Extract expressions
00113     vector<WORD *> e;
00114     e.push_back(a);
00115     e.push_back(b);
00116     const flint::var_map_t var_map = flint::get_variables(e, false, false);
00117
00118     if ( var_map.size() > 1 ) {
00119         MLOCK(ErrorMessageLock);
00120         MesPrint("flint_inverse: error: only univariate polynomials are supported.");
00121         MUNLOCK(ErrorMessageLock);
00122         Terminate(-1);
00123     }
00124
00125     return flint::inverse_poly(BHEAD a, b, var_map);
00126 }
00127 /*
00128  #] flint_inverse :
00129  #[ flint_mul :
00130  */
00131 WORD* flint_mul(PHEAD WORD *a, WORD *b) {
00132     // Extract expressions
00133     vector<WORD *> e;
00134     e.push_back(a);
00135     e.push_back(b);
00136     const flint::var_map_t var_map = flint::get_variables(e, false, false);
00137
00138     if ( var_map.size() > 1 ) {
00139         return flint::mul_mpoly(BHEAD a, b, var_map);
00140     }
00141     else {
00142         return flint::mul_poly(BHEAD a, b, var_map);
00143     }

```

```

00144 }
00145 /*
00146     #] flint_mul :
00147     #[ flint_ratfun_add :
00148 */
00149 WORD* flint_ratfun_add(PHEAD WORD *t1, WORD *t2) {
00150
00151     if ( AR.PolyFunExp == 1 ) {
00152         MLOCK(ErrorMessageLock);
00153         MesPrint("flint_ratfun_add: PolyFunExp unimplemented.");
00154         MUNLOCK(ErrorMessageLock);
00155         Terminate(-1);
00156     }
00157
00158     WORD *oldworkpointer = AT.WorkPointer;
00159
00160     // Extract expressions: the num and den of both prf
00161     vector<WORD *> e;
00162     for (WORD *t=t1+FUNHEAD; t<t1+t1[1];) {
00163         e.push_back(t);
00164         NEXTARG(t);
00165     }
00166     for (WORD *t=t2+FUNHEAD; t<t2+t2[1];) {
00167         e.push_back(t);
00168         NEXTARG(t);
00169     }
00170     const bool with_arghead = true;
00171     const bool sort_vars = true;
00172     const flint::var_map_t var_map = flint::get_variables(e, with_arghead, sort_vars);
00173
00174     if ( var_map.size() > 1 ) {
00175         flint::ratfun_add_mpoly(BHEAD t1, t2, oldworkpointer, var_map);
00176     }
00177     else {
00178         flint::ratfun_add_poly(BHEAD t1, t2, oldworkpointer, var_map);
00179     }
00180
00181     return oldworkpointer;
00182 }
00183 /*
00184     #] flint_ratfun_add :
00185     #[ flint_ratfun_normalize :
00186 */
00187 int flint_ratfun_normalize(PHEAD WORD *term) {
00188
00189     // The length of the coefficient
00190     const WORD ncoeff = (term + *term)[-1];
00191     // The end of the term data, before the coefficient:
00192     const WORD *tstop = term + *term - ABS(ncoeff);
00193
00194     // Search the term for multiple PolyFun or one dirty one.
00195     unsigned num_polyratfun = 0;
00196     for (WORD *t = term+1; t < tstop; t += t[1]) {
00197         if (*t == AR.PolyFun) {
00198             // Found one!
00199             num_polyratfun++;
00200             if ((t[2] & MUSTCLEANPRF) != 0) {
00201                 // Dirty, increment again to force normalisation for single PolyFun
00202                 num_polyratfun++;
00203             }
00204             if (num_polyratfun > 1) {
00205                 // We're not counting all occurrences, just determining if there
00206                 // is something to be done.
00207                 break;
00208             }
00209         }
00210     }
00211     if (num_polyratfun <= 1) {
00212         // There is nothing to do, return early
00213         return 0;
00214     }
00215
00216     // When there are polyratfun with only one argument: rename them temporarily to TMPPOLYFUN.
00217     for (WORD *t = term+1; t < tstop; t += t[1]) {
00218         if (*t == AR.PolyFun && (t[1] == FUNHEAD+t[FUNHEAD] || t[1] == FUNHEAD+2 )) {
00219             *t = TMPPOLYFUN;
00220         }
00221     }
00222
00223
00224     // Extract all variables in the polyfuns
00225     // Collect pointers to each relevant argument
00226     vector<WORD *> e;
00227     for (WORD *t=term+1; t<tstop; t+=t[1]) {
00228         if (*t == AR.PolyFun) {
00229             for (WORD *t2 = t+FUNHEAD; t2<t+t[1];) {
00230                 e.push_back(t2);

```

```

00231         NEXTARG(t2);
00232     }
00233 }
00234 }
00235 const bool with_arghead = true;
00236 const bool sort_vars = true;
00237 const flint::var_map_t var_map = flint::get_variables(e, with_arghead, sort_vars);
00238
00239 if ( var_map.size() > 1 ) {
00240     flint::ratfun_normalize_mpoly(BHEAD term, var_map);
00241 }
00242 else {
00243     flint::ratfun_normalize_poly(BHEAD term, var_map);
00244 }
00245
00246 // Undo renaming of single-argument PolyFun
00247 const WORD *new_tstop = term + *term - ABS((term + *term)[-1]);
00248 for (WORD *t=term+1; t<new_tstop; t+=t[1]) {
00249     if (*t == TMPPOLYFUN ) *t = AR.PolyFun;
00250 }
00251
00252 return 0;
00253 }
00254 }
00255 /*
00256 #] flint_ratfun_normalize :
00257 #[ flint_rem :
00258 */
00259 WORD* flint_rem(PHEAD WORD *a, WORD *b, const WORD must_fit_term) {
00260     // Extract expressions
00261     vector<WORD *> e;
00262     e.push_back(a);
00263     e.push_back(b);
00264     const bool with_arghead = false;
00265     const bool sort_vars = false;
00266     const flint::var_map_t var_map = flint::get_variables(e, with_arghead, sort_vars);
00267
00268     const bool return_rem = true;
00269     if ( var_map.size() > 1 ) {
00270         return flint::divmod_mpoly(BHEAD a, b, return_rem, must_fit_term, var_map);
00271     }
00272     else {
00273         return flint::divmod_poly(BHEAD a, b, return_rem, must_fit_term, var_map);
00274     }
00275 }
00276 /*
00277 #] flint_rem :
00278 #[ flint_startup_init :
00279 */
00280 void flint_startup_init(void) {
00281     flint::startup_init();
00282 }
00283 /*
00284 #] flint_startup_init :
00285 */

```

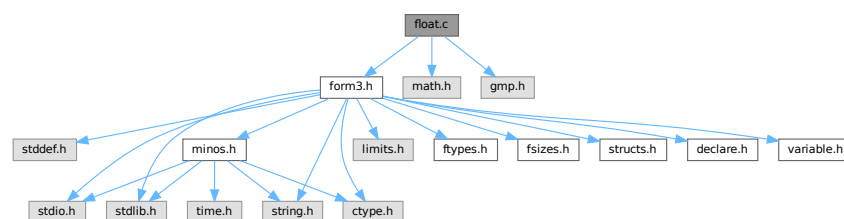
4.43 float.c File Reference

```

#include "form3.h"
#include <math.h>
#include <gmp.h>

```

Include dependency graph for float.c:



Macros

- `#define WITHCUTOFF`
- `#define GMPSPREAD (GMP_LIMB_BITS/BITSINWORD)`

Functions

- void `Form_mpf_init` (mpf_t t)
- void `Form_mpf_clear` (mpf_t t)
- void `Form_mpf_set_prec_raw` (mpf_t t, ULONG newprec)
- void `FormtoZ` (mpz_t z, UWORD *a, WORD na)
- void `ZtoForm` (UWORD *a, WORD *na, mpz_t z)
- long `FloatToInteger` (UWORD *out, mpf_t floatin, long *bitsused)
- void `IntegerToFloat` (mpf_t result, UWORD *formlong, int longsize)
- int `FloatToRat` (UWORD *ratout, WORD *nratout, mpf_t floatin)
- int `AddFloats` (PHEAD WORD *fun3, WORD *fun1, WORD *fun2)
- int `MulFloats` (PHEAD WORD *fun3, WORD *fun1, WORD *fun2)
- int `DivFloats` (PHEAD WORD *fun3, WORD *fun1, WORD *fun2)
- int `AddRatToFloat` (PHEAD WORD *outfun, WORD *infun, UWORD *formrat, WORD nrat)
- int `MulRatToFloat` (PHEAD WORD *outfun, WORD *infun, UWORD *formrat, WORD nrat)
- void `SimpleDelta` (mpf_t sum, int m)
- void `SimpleDeltaC` (mpf_t sum, int m)
- void `SingleTable` (mpf_t *tabl, int N, int m, int pow)
- void `DoubleTable` (mpf_t *tabout, mpf_t *tabin, int N, int m, int pow)
- void `EndTable` (mpf_t sum, mpf_t *tabin, int N, int m, int pow)
- void `deltaMZV` (mpf_t, int *, int)
- void `deltaEuler` (mpf_t, int *, int)
- void `deltaEulerC` (mpf_t, int *, int)
- void `CalculateMZVhalf` (mpf_t, int *, int)
- void `CalculateMZV` (mpf_t, int *, int)
- void `CalculateEuler` (mpf_t, int *, int)
- int `ExpandMZV` (WORD *term, WORD level)
- int `ExpandEuler` (WORD *term, WORD level)
- int `PackFloat` (WORD *, mpf_t)
- int `UnpackFloat` (mpf_t, WORD *)
- void `RatToFloat` (mpf_t result, UWORD *formrat, int ratsize)
- void `Form_mpf_empty` (mpf_t t)
- int `TestFloat` (WORD *fun)
- WORD `FloatFunToRat` (PHEAD UWORD *ratout, WORD *in)
- void `ZeroTable` (mpf_t *tab, int N)
- SBYTE * `ReadFloat` (SBYTE *s)
- UBYTE * `CheckFloat` (UBYTE *ss, int *spec)
- int `SetFloatPrecision` (WORD prec)
- int `PrintFloat` (WORD *fun, int numdigits)
- void `SetupMZVTables` (void)
- void `SetupMPFTables` (void)
- void `ClearMZVTables` (void)
- int `CoToFloat` (UBYTE *s)
- int `CoToRat` (UBYTE *s)
- int `ToFloat` (PHEAD WORD *term, WORD level)
- int `ToRat` (PHEAD WORD *term, WORD level)
- WORD `AddWithFloat` (PHEAD WORD **ps1, WORD **ps2)
- WORD `MergeWithFloat` (PHEAD WORD **interm1, WORD **interm2)
- int `EvaluateEuler` (PHEAD WORD *term, WORD level, WORD par)
- int `CoEvaluate` (UBYTE *s)

4.43.1 Detailed Description

This file contains numerical routines that deal with floating point numbers. We use the GMP for arbitrary precision floating point arithmetic. The functions of the type sin, cos, ln, sqrt etc are handled by the mpfr library. The reason that for MZV's and the general notation we use the GMP (mpf_) is because the contents of the structs have been frozen and can be used for storage in a Form function float_. With mpfr_ this is not safely possible. All mpfr_ related code is in the file [evaluate.c](#).

Definition in file [float.c](#).

4.43.2 Macro Definition Documentation

4.43.2.1 WITHCUTOFF

```
#define WITHCUTOFF
```

Definition at line 45 of file [float.c](#).

4.43.2.2 GMPSPREAD

```
#define GMPSPREAD (GMP_LIMB_BITS/BITSINWORD)
```

Definition at line 47 of file [float.c](#).

4.43.3 Function Documentation

4.43.3.1 Form_mpf_init()

```
void Form_mpf_init (  
    mpf_t t )
```

Definition at line 176 of file [float.c](#).

4.43.3.2 Form_mpf_clear()

```
void Form_mpf_clear (  
    mpf_t t )
```

Definition at line 200 of file [float.c](#).

4.43.3.3 Form_mpf_set_prec_raw()

```
void Form_mpf_set_prec_raw (  
    mpf_t t,  
    ULONG newprec )
```

Definition at line 226 of file [float.c](#).

4.43.3.4 FormtoZ()

```
void FormtoZ (
    mpz_t z,
    UWORD * a,
    WORD na )
```

Definition at line 511 of file [float.c](#).

4.43.3.5 ZtoForm()

```
void ZtoForm (
    UWORD * a,
    WORD * na,
    mpz_t z )
```

Definition at line 555 of file [float.c](#).

4.43.3.6 FloatToInteger()

```
long FloatToInteger (
    UWORD * out,
    mpf_t floatin,
    long * bitsused )
```

Definition at line 581 of file [float.c](#).

4.43.3.7 IntegerToFloat()

```
void IntegerToFloat (
    mpf_t result,
    UWORD * formlong,
    int longsize )
```

Definition at line 604 of file [float.c](#).

4.43.3.8 FloatToRat()

```
int FloatToRat (
    UWORD * ratout,
    WORD * nratout,
    mpf_t floatin )
```

Definition at line 668 of file [float.c](#).

4.43.3.9 AddFloats()

```
int AddFloats (
    PHEAD WORD * fun3,
    WORD * fun1,
    WORD * fun2 )
```

Definition at line 978 of file [float.c](#).

4.43.3.10 MulFloats()

```
int MulFloats (
    PHEAD WORD * fun3,
    WORD * fun1,
    WORD * fun2 )
```

Definition at line 994 of file [float.c](#).

4.43.3.11 DivFloats()

```
int DivFloats (
    PHEAD WORD * fun3,
    WORD * fun1,
    WORD * fun2 )
```

Definition at line 1010 of file [float.c](#).

4.43.3.12 AddRatToFloat()

```
int AddRatToFloat (
    PHEAD WORD * outfun,
    WORD * infun,
    UWORD * formrat,
    WORD nrat )
```

Definition at line 1028 of file [float.c](#).

4.43.3.13 MulRatToFloat()

```
int MulRatToFloat (
    PHEAD WORD * outfun,
    WORD * infun,
    UWORD * formrat,
    WORD nrat )
```

Definition at line 1047 of file [float.c](#).

4.43.3.14 SimpleDelta()

```
void SimpleDelta (
    mpf_t sum,
    int m )
```

Definition at line 1680 of file [float.c](#).

4.43.3.15 SimpleDeltaC()

```
void SimpleDeltaC (
    mpf_t sum,
    int m )
```

Definition at line 1727 of file [float.c](#).

4.43.3.16 SingleTable()

```
void SingleTable (
    mpf_t * tabl,
    int N,
    int m,
    int pow )
```

Definition at line 1774 of file [float.c](#).

4.43.3.17 DoubleTable()

```
void DoubleTable (
    mpf_t * tabout,
    mpf_t * tabin,
    int N,
    int m,
    int pow )
```

Definition at line 1826 of file [float.c](#).

4.43.3.18 EndTable()

```
void EndTable (
    mpf_t sum,
    mpf_t * tabin,
    int N,
    int m,
    int pow )
```

Definition at line 1877 of file [float.c](#).

4.43.3.19 deltaMZV()

```
void deltaMZV (
    mpf_t result,
    int * indexes,
    int depth )
```

Definition at line 1922 of file [float.c](#).

4.43.3.20 `deltaEuler()`

```
void deltaEuler (
    mpf_t result,
    int * indexes,
    int depth )
```

Definition at line 1989 of file `float.c`.

4.43.3.21 `deltaEulerC()`

```
void deltaEulerC (
    mpf_t result,
    int * indexes,
    int depth )
```

Definition at line 2045 of file `float.c`.

4.43.3.22 `CalculateMZVhalf()`

```
void CalculateMZVhalf (
    mpf_t result,
    int * indexes,
    int depth )
```

Definition at line 2118 of file `float.c`.

4.43.3.23 `CalculateMZV()`

```
void CalculateMZV (
    mpf_t result,
    int * indexes,
    int depth )
```

Definition at line 2139 of file `float.c`.

4.43.3.24 `CalculateEuler()`

```
void CalculateEuler (
    mpf_t result,
    int * Zindexes,
    int depth )
```

Definition at line 2204 of file `float.c`.

4.43.3.25 `ExpandMZV()`

```
int ExpandMZV (
    WORD * term,
    WORD level )
```

Definition at line 2280 of file `float.c`.

4.43.3.26 ExpandEuler()

```
int ExpandEuler (
    WORD * term,
    WORD level )
```

Definition at line [2292](#) of file [float.c](#).

4.43.3.27 PackFloat()

```
int PackFloat (
    WORD * fun,
    mpf_t infloat )
```

Definition at line [270](#) of file [float.c](#).

4.43.3.28 UnpackFloat()

```
int UnpackFloat (
    mpf_t outfloat,
    WORD * fun )
```

Definition at line [364](#) of file [float.c](#).

4.43.3.29 RatToFloat()

```
void RatToFloat (
    mpf_t result,
    UWORD * formrat,
    int ratsize )
```

Definition at line [620](#) of file [float.c](#).

4.43.3.30 Form_mpf_empty()

```
void Form_mpf_empty (
    mpf_t t )
```

Definition at line [213](#) of file [float.c](#).

4.43.3.31 TestFloat()

```
int TestFloat (
    WORD * fun )
```

Definition at line [452](#) of file [float.c](#).

4.43.3.32 FloatFunToRat()

```
WORD FloatFunToRat (
    PHEAD UWORD * ratout,
    WORD * in )
```

Definition at line [642](#) of file [float.c](#).

4.43.3.33 ZeroTable()

```
void ZeroTable (
    mpf_t * tab,
    int N )
```

Definition at line [804](#) of file [float.c](#).

4.43.3.34 ReadFloat()

```
SBYTE * ReadFloat (
    SBYTE * s )
```

Definition at line [822](#) of file [float.c](#).

4.43.3.35 CheckFloat()

```
UBYTE * CheckFloat (
    UBYTE * ss,
    int * spec )
```

Definition at line [848](#) of file [float.c](#).

4.43.3.36 SetFloatPrecision()

```
int SetFloatPrecision (
    WORD prec )
```

Definition at line [911](#) of file [float.c](#).

4.43.3.37 PrintFloat()

```
int PrintFloat (
    WORD * fun,
    int numdigits )
```

Definition at line [940](#) of file [float.c](#).

4.43.3.38 SetupMZVTables()

```
void SetupMZVTables (
    void )
```

Definition at line 1064 of file [float.c](#).

4.43.3.39 SetupMPFTables()

```
void SetupMPFTables (
    void )
```

Definition at line 1161 of file [float.c](#).

4.43.3.40 ClearMZVTables()

```
void ClearMZVTables (
    void )
```

Definition at line 1215 of file [float.c](#).

4.43.3.41 CoToFloat()

```
int CoToFloat (
    UBYTE * s )
```

Definition at line 1292 of file [float.c](#).

4.43.3.42 CoToRat()

```
int CoToRat (
    UBYTE * s )
```

Definition at line 1314 of file [float.c](#).

4.43.3.43 ToFloat()

```
int ToFloat (
    PHEAD WORD * term,
    WORD level )
```

Definition at line 1338 of file [float.c](#).

4.43.3.44 ToRat()

```
int ToRat (
    PHEAD WORD * term,
    WORD level )
```

Definition at line 1370 of file [float.c](#).

4.43.3.45 AddWithFloat()

```
WORD AddWithFloat (
    PHEAD WORD ** ps1,
    WORD ** ps2 )
```

Definition at line 1439 of file [float.c](#).

4.43.3.46 MergeWithFloat()

```
WORD MergeWithFloat (
    PHEAD WORD ** interm1,
    WORD ** interm2 )
```

Definition at line 1560 of file [float.c](#).

4.43.3.47 EvaluateEuler()

```
int EvaluateEuler (
    PHEAD WORD * term,
    WORD level,
    WORD par )
```

Definition at line 2320 of file [float.c](#).

4.43.3.48 CoEvaluate()

```
int CoEvaluate (
    UBYTE * s )
```

Definition at line 2436 of file [float.c](#).

4.44 float.c

[Go to the documentation of this file.](#)

```
00001
00011 /* # [ License : */
00012 /*
00013 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00014 * When using this file you are requested to refer to the publication
00015 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00016 * This is considered a matter of courtesy as the development was paid
00017 * for by FOM the Dutch physics granting agency and we would like to
00018 * be able to track its scientific use to convince FOM of its value
00019 * for the community.
00020 *
00021 * This file is part of FORM.
00022 *
00023 * FORM is free software: you can redistribute it and/or modify it under the
00024 * terms of the GNU General Public License as published by the Free Software
00025 * Foundation, either version 3 of the License, or (at your option) any later
00026 * version.
00027 *
00028 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00029 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00030 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00031 * details.
00032 *
```



```

00033  *   You should have received a copy of the GNU General Public License along
00034  *   with FORM.  If not, see <http://www.gnu.org/licenses/>.
00035  */
00036  /* #] License : */
00037  /*
00038      #[ Includes : float.c
00039  */
00040
00041  #include "form3.h"
00042  #include <math.h>
00043  #include <gmp.h>
00044
00045  #define WITHCUTOFF
00046
00047  #define GMPSPREAD (GMP_LIMB_BITS/BITSINWORD)
00048
00049  void Form_mpf_init(mpf_t t);
00050  void Form_mpf_clear(mpf_t t);
00051  void Form_mpf_set_prec_raw(mpf_t t, ULONG newprec);
00052  void FormtoZ(mpz_t z, UWORD *a, WORD na);
00053  void ZtoForm(UWORD *a, WORD *na, mpz_t z);
00054  long FloatToInteger(UWORD *out, mpf_t floatin, long *bitsused);
00055  void IntegerToFloat(mpf_t result, UWORD *formlong, int longsize);
00056  int FloatToRat(UWORD *ratout, WORD *nratout, mpf_t floatin);
00057  int AddFloats(PHEAD WORD *fun3, WORD *fun1, WORD *fun2);
00058  int MulFloats(PHEAD WORD *fun3, WORD *fun1, WORD *fun2);
00059  int DivFloats(PHEAD WORD *fun3, WORD *fun1, WORD *fun2);
00060  int AddRatToFloat(PHEAD WORD *outfun, WORD *infun, UWORD *formrat, WORD nrat);
00061  int MulRatToFloat(PHEAD WORD *outfun, WORD *infun, UWORD *formrat, WORD nrat);
00062  void SimpleDelta(mpf_t sum, int m);
00063  void SimpleDeltaC(mpf_t sum, int m);
00064  void SingleTable(mpf_t *tab1, int N, int m, int pow);
00065  void DoubleTable(mpf_t *tabout, mpf_t *tabin, int N, int m, int pow);
00066  void EndTable(mpf_t sum, mpf_t *tabin, int N, int m, int pow);
00067  void deltaMZV(mpf_t, int *, int);
00068  void deltaEuler(mpf_t, int *, int);
00069  void deltaEulerC(mpf_t, int *, int);
00070  void CalculateMZVhalf(mpf_t, int *, int);
00071  void CalculateMZV(mpf_t, int *, int);
00072  void CalculateEuler(mpf_t, int *, int);
00073  int ExpandMZV(WORD *term, WORD level);
00074  int ExpandEuler(WORD *term, WORD level);
00075  int PackFloat(WORD *, mpf_t);
00076  int UnpackFloat(mpf_t, WORD *);
00077  void RatToFloat(mpf_t result, UWORD *formrat, int ratsize);
00078
00079  /*
00080      #] Includes :
00081      #[ Some GMP float info :
00082
00083      From gmp.h:
00084
00085      typedef struct
00086      {
00087          int _mp_prec;      Max precision, in number of `mp_limb_t's.
00088                          Set by mpf_init and modified by
00089                          mpf_set_prec.  The area pointed to by the
00090                          _mp_d field contains `prec' + 1 limbs.
00091          int _mp_size;      abs(_mp_size) is the number of limbs the
00092                          last field points to.  If _mp_size is
00093                          negative this is a negative number.
00094          mp_exp_t _mp_exp;  Exponent, in the base of `mp_limb_t'.
00095          mp_limb_t *_mp_d;  Pointer to the limbs.
00096      } __mpf_struct;
00097
00098      typedef __mpf_struct mpf_t[1];
00099
00100      Currently:
00101          sizeof(int) = 4
00102          sizeof(mpf_t) = 24
00103          sizeof(mp_limb_t) = 8,
00104          sizeof(mp_exp_t) = 8,
00105          sizeof a pointer is 8.
00106      If any of this changes in the future we would have to change the
00107      PackFloat and UnpackFloat routines.
00108
00109      Our float_ function is packed with the 4 arguments:
00110          -SNUMBER _mp_prec
00111          -SNUMBER _mp_size
00112          exponent which can be -SNUMBER or a regular term
00113          the limbs as n/1 in regular term format, or just an -SNUMBER.
00114      During normalization the sign is taken away from the second argument
00115      and put as the sign of the complete term.  This is easier for the
00116      WriteInnerTerm routine in sch.c
00117
00118      Wildcarding of functions excludes matches with float_
00119      Float_ always ends up inside the bracket, just like poly(rat)fun.

```

```

00120
00121     From mpfr.h
00122
00123     The formats here are different. This has, among others, to do with
00124     the rounding. The result is that we need different aux variables
00125     when working with mpfr and we need a different conversion routine
00126     to float. It also means that we need to treat the mzv_ etc functions
00127     completely separately. For now we try to develop that in the file
00128     Evaluate.c. At a later stage we may merge the two files.
00129     Originally we had the sqrt in float.c but it seems better to put
00130     it in Evaluate.c
00131
00132     #] Some GMP float info :
00133     #[ Low Level :
00134
00135         In the low level routines we interact directly with the content
00136         of the GMP structs. This can be done safely, because their
00137         layout is documented. We pay particular attention to the init
00138         and clear routines, because they involve malloc/free calls.
00139
00140         #[ Explanations :
00141
00142         We need a number of facilities inside Form to deal with floating point
00143         numbers, taking into account that they are only meant for sporadic use.
00144         1: We need a special function float_ who's arguments contain a limb
00145            representation of the mpf_t of the gmp.
00146         2: Some functions may return a floating point number. This is then in
00147            the form of an occurrence of the function float_.
00148         3: We need a print routine to print this float_.
00149         4: We need routines for conversions:
00150            a: (long) int to float.
00151            b: rat to float.
00152            c: float to rat (if possible)
00153            d: float to integer (with rounding toward zero).
00154         5: We need calculational operations for add, sub, mul, div, exp.
00155         6: We need service routines to pack and unpack the function float_.
00156         7: The coefficient should be pulled inside float_.
00157         8: Float_ cannot occur inside PolyRatFun.
00158         9: We need to be able to treat float_ as the coefficient during sorting.
00159         10: For now, we need the functions mzv_ and eulerf_ for the evaluation
00160             of mzv's and euler sums.
00161         11: We need a setting for the default precision.
00162         12: We need a special flag for the dirty field of arguments to avoid
00163             the normalization of the argument with the limbs.
00164             Alternatively, the float_ should be not a regular function, but
00165             get a status < FUNCTION. Just like SNUMBER, LNUMBER etc.
00166
00167         Note that we can keep this thread-safe. The only routine that is not
00168         thread-safe is the print routine, but that is in accordance with all
00169         other print routines in Form. When called from MesPrint it needs to
00170         be protected by a lock, as is done standard inside Form.
00171
00172         #] Explanations :
00173         #[ Form_mpf_init :
00174     */
00175
00176 void Form_mpf_init(mpf_t t)
00177 {
00178     mp_limb_t *d;
00179     int i, prec;
00180     if ( t->_mp_d ) { M_free(t->_mp_d,"Form_mpf_init"); t->_mp_d = 0; }
00181     prec = (AC.DefaultPrecision + 8*sizeof(mp_limb_t)-1)/(8*sizeof(mp_limb_t)) + 1;
00182     t->_mp_prec = prec;
00183     t->_mp_size = 0;
00184     t->_mp_exp = 0;
00185     d = (mp_limb_t *)Malloc1(prec*sizeof(mp_limb_t),"Form_mpf_init");
00186     if ( d == 0 ) {
00187         MesPrint("Fatal error in Malloc1 call in Form_mpf_init. Trying to allocate %ld bytes.",
00188             prec*sizeof(mp_limb_t));
00189         Terminate(-1);
00190     }
00191     t->_mp_d = d;
00192     for ( i = 0; i < prec; i++ ) d[i] = 0;
00193 }
00194
00195 /*
00196     #] Form_mpf_init :
00197     #[ Form_mpf_clear :
00198 */
00199
00200 void Form_mpf_clear(mpf_t t)
00201 {
00202     if ( t->_mp_d ) { M_free(t->_mp_d,"Form_mpf_init"); t->_mp_d = 0; }
00203     t->_mp_prec = 0;
00204     t->_mp_size = 0;
00205     t->_mp_exp = 0;
00206 }

```

```

00207
00208 /*
00209     #] Form_mpf_clear :
00210     #[ Form_mpf_empty :
00211 */
00212
00213 void Form_mpf_empty(mpf_t t)
00214 {
00215     int i;
00216     for ( i = 0; i < t->_mp_prec; i++ ) t->_mp_d[i] = 0;
00217     t->_mp_size = 0;
00218     t->_mp_exp = 0;
00219 }
00220
00221 /*
00222     #] Form_mpf_empty :
00223     #[ Form_mpf_set_prec_raw :
00224 */
00225
00226 void Form_mpf_set_prec_raw(mpf_t t, ULONG newprec)
00227 {
00228     ULONG newpr = (newprec + 8*sizeof(mp_limb_t)-1)/(8*sizeof(mp_limb_t)) + 1;
00229     if ( newpr <= (ULONG)(t->_mp_prec) ) {
00230         int i, used = ABS(t->_mp_size) ;
00231         if ( newpr > (ULONG)used ) {
00232             for ( i = used; i < (int)newpr; i++ ) t->_mp_d[i] = 0;
00233         }
00234         if ( t->_mp_size < 0 ) newpr = -newpr;
00235         t->_mp_size = newpr;
00236     }
00237     else {
00238         /*
00239             We can choose here between "forget about it" or making a new malloc.
00240             If the program is well designed, we should never get here though.
00241             For now we choose the "forget it" option.
00242         */
00243         MesPrint("Trying too big a precision %ld in Form_mpf_set_prec_raw.", newprec);
00244         MesPrint("Old maximal precision is %ld.",
00245             (size_t)(t->_mp_prec-1)*sizeof(mp_limb_t)*8);
00246         Terminate(-1);
00247     }
00248 }
00249
00250 /*
00251     #] Form_mpf_set_prec_raw :
00252     #[ PackFloat :
00253
00254     Puts an object of type mpf_t inside a function float_
00255     float_(prec, nlimbs, exp, limbs);
00256     Complication: the limbs and the exponent are long's. That means
00257     two times the size of a Form (U)WORD.
00258     To save space we could split the limbs in two because Form stores
00259     coefficients as rationals with equal space for numerator and denominator.
00260     Hence for each limb we could put the most significant UWORD in the
00261     numerator and the least significant limb in the denominator.
00262     This gives a problem with the compilation and subsequent potential
00263     normalization of the 'fraction'. Hence for now we take the wasteful
00264     approach. At a later stage we can try to veto normalization of FLOATFUN.
00265     Caution: gmp/fmp is little endian and Form is big endian.
00266
00267     Zero is represented by zero limbs (and zero exponent).
00268 */
00269
00270 int PackFloat(WORD *fun, mpf_t infloat)
00271 {
00272     WORD *t, nlimbs, num, nnum;
00273     mp_limb_t *d = infloat->_mp_d; /* Pointer to the limbs. */
00274     int i;
00275     long e = infloat->_mp_exp;
00276
00277     t = fun;
00278     *t++ = FLOATFUN;
00279     t++;
00280     FILLFUN(t);
00281     *t++ = -SNUMBER;
00282     *t++ = infloat->_mp_prec;
00283     *t++ = -SNUMBER;
00284     *t++ = infloat->_mp_size;
00285     /*
00286     Now the exponent which is a signed long
00287     */
00288     if ( e < 0 ) {
00289         e = -e;
00290         if ( ( e >> (BITSINWORD-1) ) == 0 ) {
00291             *t++ = -SNUMBER; *t++ = -e;
00292         }
00293     }
    else {

```

```

00294         *t++ = ARGHEAD+6; *t++ = 0; FILLARG(t);
00295         *t++ = 6;
00296         *t++ = (UWORD)e;
00297         *t++ = (UWORD)(e » BITSINWORD);
00298         *t++ = 1; *t++ = 0; *t++ = -5;
00299     }
00300 }
00301 else {
00302     if ( ( e » (BITSINWORD-1) ) == 0 ) {
00303         *t++ = -SNUMBER; *t++ = e;
00304     }
00305     else {
00306         *t++ = ARGHEAD+6; *t++ = 0; FILLARG(t);
00307         *t++ = 6;
00308         *t++ = (UWORD)e;
00309         *t++ = (UWORD)(e » BITSINWORD);
00310         *t++ = 1; *t++ = 0; *t++ = 5;
00311     }
00312 }
00313 /*
00314 and now the limbs. Note that the number of active limbs could be less
00315 than prec+1 in which case we copy the smaller number.
00316 */
00317 nlimbs = infloat->_mp_size < 0 ? -infloat->_mp_size: infloat->_mp_size;
00318 if ( nlimbs == 0 ) {
00319 }
00320 else if ( nlimbs == 1 && (ULONG)(*d) < ((ULONG)1)«(BITSINWORD-1) ) {
00321     *t++ = -SNUMBER;
00322     *t++ = (UWORD)(*d);
00323 }
00324 else {
00325     if ( d[nlimbs-1] < ((ULONG)1)«BITSINWORD ) {
00326         num = 2*nlimbs-1; /* number of UWORDS needed */
00327     }
00328     else {
00329         num = 2*nlimbs; /* number of UWORDS needed */
00330     }
00331     nnum = num;
00332     *t++ = 2*num+2+ARGHEAD;
00333     *t++ = 0;
00334     FILLARG(t);
00335     *t++ = 2*num+2;
00336     while ( num > 1 ) {
00337         *t++ = (UWORD)(*d); *t++ = (UWORD)((ULONG)(*d)»BITSINWORD); d++;
00338         num -= 2;
00339     }
00340     if ( num == 1 ) { *t++ = (UWORD)(*d); }
00341     *t++ = 1;
00342     for ( i = 1; i < nnum; i++ ) *t++ = 0;
00343     *t++ = 2*nnum+1; /* the sign is hidden in infloat->_mp_size */
00344 }
00345 fun[1] = t-fun;
00346 return(fun[1]);
00347 }
00348
00349 /*
00350     #] PackFloat :
00351     #[ UnpackFloat :
00352
00353     Takes the arguments of a function float_ and puts it inside an mpf_t.
00354     For notation see commentary for PutInFloat.
00355
00356     We should make a new allocation for the limbs if the precision exceeds
00357     the present precision of outfloat, or when outfloat has not been
00358     initialized yet.
00359
00360     The return value is -1 if the contents of the FLOATFUN cannot be converted.
00361     Otherwise the return value is the number of limbs.
00362 */
00363
00364 int UnpackFloat(mpf_t outfloat,WORD *fun)
00365 {
00366     UWORD *t;
00367     WORD *f;
00368     int num, i;
00369     mp_limb_t *d;
00370 /*
00371     Very first step: check whether we can use float_:
00372 */
00373     GETIDENTITY
00374     if ( AT.aux_ == 0 ) {
00375         MesPrint("Illegal attempt at using a float_ function without proper startup.");
00376         MesPrint("Please use %#StartFloat <options> first.");
00377         Terminate(-1);
00378     }
00379 /*
00380     Check first the number and types of the arguments

```

```

00381     There should be 4. -SNUMBER,-SNUMBER,-SNUMBER or a long number.
00382     + the limbs, either -SNUMBER or Long number in the form of a Form rational.
00383 */
00384     if ( TestFloat(fun) == 0 ) goto Incorrect;
00385     f = fun + FUNHEAD + 2;
00386     if ( ABS(f[1]) > f[-1]+1 ) goto Incorrect;
00387     if ( f[1] > outfloat->_mp_prec+1
00388         || outfloat->_mp_d == 0 ) {
00389         /*
00390          We need to make a new allocation.
00391          Unfortunately we cannot use Malloc1 because that is not
00392          recognised by gmp and hence we cannot clear with mpf_clear.
00393          Maybe we can do something about it by making our own
00394          mpf_init and mpf_clear?
00395         */
00396         if ( outfloat->_mp_d != 0 ) free(outfloat->_mp_d);
00397         outfloat->_mp_d = (mp_limb_t *)malloc((f[1]+1)*sizeof(mp_limb_t));
00398     }
00399     num = f[1];
00400     outfloat->_mp_size = num;
00401     if ( num < 0 ) { num = -num; }
00402     f += 2;
00403     if ( *f == -SNUMBER ) {
00404         outfloat->_mp_exp = (mp_exp_t)(f[1]);
00405         f += 2;
00406     }
00407     else {
00408         f += ARGHEAD+6;
00409         if ( f[-1] == -5 ) {
00410             outfloat->_mp_exp =
00411                 -(mp_exp_t)((((ULONG)(f[-4]))«BITSINWORD)+f[-5]);
00412         }
00413         else if ( f[-1] == 5 ) {
00414             outfloat->_mp_exp =
00415                 (mp_exp_t)((((ULONG)(f[-4]))«BITSINWORD)+f[-5]);
00416         }
00417     }
00418     /*
00419     And now the limbs if needed
00420     */
00421     d = outfloat->_mp_d;
00422     if ( outfloat->_mp_size == 0 ) {
00423         for ( i = 0; i < outfloat->_mp_prec; i++ ) *d++ = 0;
00424         return(0);
00425     }
00426     else if ( *f == -SNUMBER ) {
00427         *d++ = (mp_limb_t)f[1];
00428         for ( i = 0; i < outfloat->_mp_prec; i++ ) *d++ = 0;
00429         return(0);
00430     }
00431     num = (*f-ARGHEAD-2)/2; /* 2*number of limbs in the argument */
00432     t = (UWORD *) (f+ARGHEAD+1);
00433     while ( num > 1 ) {
00434         *d++ = (mp_limb_t)((((ULONG)(t[1]))«BITSINWORD)+t[0]);
00435         t += 2; num -= 2;
00436     }
00437     if ( num == 1 ) *d++ = (mp_limb_t)((UWORD)(t));
00438     for ( i = d-outfloat->_mp_d; i <= outfloat->_mp_prec; i++ ) *d++ = 0;
00439     return(0);
00440 Incorrect:
00441     return(-1);
00442 }
00443
00444 /*
00445     #] UnpackFloat :
00446     #[ TestFloat :
00447
00448     Tells whether the function (with its arguments) is a legal float_.
00449     We assume, as in the whole float library that one limb is 2 WORDS.
00450 */
00451
00452 int TestFloat (WORD *fun)
00453 {
00454     WORD *fstop, *f, num, nnum, i;
00455     fstop = fun+fun[1];
00456     f = fun + FUNHEAD;
00457     num = 0;
00458     /*
00459     Count arguments
00460     */
00461     while ( f < fstop ) { num++; NEXTARG(f); }
00462     if ( num != 4 ) return(0);
00463     f = fun + FUNHEAD;
00464     if ( *f != -SNUMBER ) return(0);
00465     if ( f[1] < 0 ) return(0);
00466     f += 2;
00467     if ( *f != -SNUMBER ) return(0);

```

```

00468     num = ABS(f[1]); /* number of limbs */
00469     f += 2;
00470     if ( *f == -SNUMBER ) { f += 2; }
00471     else {
00472         if ( *f != ARGHEAD+6 ) return(0);
00473         if ( ABS(f[ARGHEAD+5]) != 5 ) return(0);
00474         if ( f[ARGHEAD+3] != 1 ) return(0);
00475         if ( f[ARGHEAD+4] != 0 ) return(0);
00476         f += *f;
00477     }
00478     if ( num == 0 ) return(1);
00479     if ( *f == -SNUMBER ) { if ( num != 1 ) return(0); }
00480     else {
00481         nnum = (ABS(f[*f-1])-1)/2;
00482         if ( ( *f != 4*num+2+ARGHEAD ) && ( *f != 4*num+ARGHEAD ) ) return(0);
00483         if ( ( nnum != 2*num ) && ( nnum != 2*num-1 ) ) return(0);
00484         f += ARGHEAD+nnum+1;
00485         if ( f[0] != 1 ) return(0);
00486         for ( i = 1; i < nnum; i++ ) {
00487             if ( f[i] != 0 ) return(0);
00488         }
00489     }
00490     return(1);
00491 }
00492
00493 /*
00494     #] TestFloat :
00495     #[ FormtoZ :
00496
00497     Converts a Form long integer to a GMP long integer.
00498
00499     typedef struct
00500     {
00501         int _mp_alloc;   Number of *limbs* allocated and pointed
00502                         to by the _mp_d field.
00503         int _mp_size;    abs(_mp_size) is the number of limbs the
00504                         last field points to. If _mp_size is
00505                         negative this is a negative number.
00506         mp_limb_t *_mp_d; Pointer to the limbs.
00507     } __mpz_struct;
00508     typedef __mpz_struct mpz_t[1];
00509 */
00510
00511 void FormtoZ(mpz_t z, UWORD *a, WORD na)
00512 {
00513     int nlimbs, sgn = 1;
00514     mp_limb_t *d;
00515     UWORD *b = a;
00516     WORD nb = na;
00517
00518     if ( nb == 0 ) {
00519         z->_mp_size = 0;
00520         z->_mp_d[0] = 0;
00521         return;
00522     }
00523     if ( nb < 0 ) { sgn = -1; nb = -nb; }
00524     nlimbs = (nb+1)/2;
00525     if ( mpz_cmp_si(z, 0L) <= 1 ) {
00526         mpz_set_ui(z, 10000000);
00527     }
00528     if ( z->_mp_alloc <= nlimbs ) {
00529         mpz_t zz;
00530         mpz_init(zz);
00531         while ( z->_mp_alloc <= nlimbs ) {
00532             mpz_mul(zz, z, z);
00533             mpz_set(z, zz);
00534         }
00535         mpz_clear(zz);
00536     }
00537     z->_mp_size = sgn*nlimbs;
00538     d = z->_mp_d;
00539     while ( nb > 1 ) {
00540         *d++ = (((mp_limb_t) (b[1])) << BITSINWORD) + (mp_limb_t) (b[0]);
00541         b += 2; nb -= 2;
00542     }
00543     if ( nb == 1 ) { *d = (mp_limb_t) (*b); }
00544 }
00545
00546 /*
00547     #] FormtoZ :
00548     #[ ZtoForm :
00549
00550     Converts a GMP long integer to a Form long integer.
00551     Mainly an exercise of going from little endian ULONGs.
00552     to big endian UWORDS
00553 */
00554

```

```

00555 void ZtoForm(UWORD *a,WORD *na,mpz_t z)
00556 {
00557     mp_limb_t *d = z->_mp_d;
00558     int nlimbs = ABS(z->_mp_size), i;
00559     UWORD j;
00560     if ( nlimbs == 0 ) { *na = 0; return; }
00561     *na = 0;
00562     for ( i = 0; i < nlimbs-1; i++ ) {
00563         *a++ = (UWORD)(*d);
00564         *a++ = (UWORD)((*d++)»BITSINWORD);
00565         *na += 2;
00566     }
00567     *na += 1;
00568     *a++ = (UWORD)(*d);
00569     j = (UWORD)((*d)»BITSINWORD);
00570     if ( j != 0 ) { *a = j; *na += 1; }
00571     if ( z->_mp_size < 0 ) *na = -*na;
00572 }
00573
00574 /*
00575     #] ZtoForm :
00576     #[ FloatToInteger :
00577
00578     Converts a GMP float to a long Form integer.
00579 */
00580
00581 long FloatToInteger(UWORD *out, mpf_t floatin, long *bitsused)
00582 {
00583     mpz_t z;
00584     WORD nout, nx;
00585     UWORD x;
00586     mpz_init(z);
00587     mpz_set_f(z,floatin);
00588     ZtoForm(out,&nout,z);
00589     mpz_clear(z);
00590     x = out[0]; nx = 0;
00591     while ( x ) { nx++; x »= 1; }
00592     *bitsused = (nout-1)*BITSINWORD + nx;
00593     return(nout);
00594 }
00595
00596 /*
00597     #] FloatToInteger :
00598     #[ IntegerToFloat :
00599
00600     Converts a Form long integer to a gmp float of default size.
00601     We assume that sizeof(unsigned long int) = 2*sizeof(UWORD).
00602 */
00603
00604 void IntegerToFloat(mpf_t result, UWORD *formlong, int longsize)
00605 {
00606     mpz_t z;
00607     mpz_init(z);
00608     FormtoZ(z,formlong,longsize);
00609     mpf_set_z(result,z);
00610     mpz_clear(z);
00611 }
00612
00613 /*
00614     #] IntegerToFloat :
00615     #[ RatToFloat :
00616
00617     Converts a Form rational to a gmp float of default size.
00618 */
00619
00620 void RatToFloat(mpf_t result, UWORD *formrat, int ratsize)
00621 {
00622     GETIDENTITY
00623     int nnum, nden;
00624     UWORD *num, *den;
00625     int sgn = 0;
00626     if ( ratsize < 0 ) { ratsize = -ratsize; sgn = 1; }
00627     nnum = nden = (ratsize-1)/2;
00628     num = formrat; den = formrat+nnum;
00629     while ( num[nnum-1] == 0 ) { nnum--; }
00630     while ( den[nden-1] == 0 ) { nden--; }
00631     IntegerToFloat(aux2,num,nnum);
00632     IntegerToFloat(aux3,den,nden);
00633     mpf_div(result,aux2,aux3);
00634     if ( sgn > 0 ) mpf_neg(result,result);
00635 }
00636
00637 /*
00638     #] RatToFloat :
00639     #[ FloatFunToRat :
00640 */
00641

```

```

00642 WORD FloatFunToRat(PHEAD UWORD *ratout,WORD *in)
00643 {
00644     WORD oldin = in[0], nratout;
00645     in[0] = FLOATFUN;
00646     UnpackFloat(aux4,in);
00647     FloatToRat(ratout,&nratout,aux4);
00648     in[0] = oldin;
00649     return(nratout);
00650 }
00651
00652 /*
00653     #] FloatFunToRat :
00654     #[ FloatToRat :
00655
00656     Converts a gmp float to a Form rational.
00657     The algorithm we use is 'repeated fractions' of the style
00658         n0+1/(n1+1/(n2+1/(n3+....)))
00659     The moment we get an n that is very large we should have the proper fraction
00660     Main problem: what if the sequence keeps going?
00661                 (there are always rounding errors)
00662     We need a cutoff with an error condition.
00663     Basically the product n0*n1*n2*... is an underlimit on the number of
00664     digits in the answer. We can put a limit on this.
00665     A return value of zero means that everything went fine.
00666 */
00667 int FloatToRat(UWORD *ratout, WORD *nratout, mpf_t floatin)
00668 {
00669     GETIDENTITY
00670     WORD *out = AT.WorkPointer;
00671     WORD s; /* the sign. */
00672     UWORD *a, *b, *c, *d, *mc;
00673     WORD na, nb, nc, nd, i;
00674     int nout;
00675     LONG oldpWorkPointer = AT.pWorkPointer;
00676     long bitsused = 0, totalbitsused = 0, totalbits = AC.DefaultPrecision;
00677     int retval = 0, startnul;
00678     WantAddPointers(AC.DefaultPrecision); /* Horrible overkill */
00679     AT.pWorkspace[AT.pWorkPointer++] = out;
00680     a = NumberMalloc("FloatToRat");
00681     b = NumberMalloc("FloatToRat");
00682
00683     s = mpf_sgn(floatin);
00684     if ( s < 0 ) mpf_neg(floatin,floatin);
00685
00686     Form_mpf_empty(aux1);
00687     Form_mpf_empty(aux2);
00688     Form_mpf_empty(aux3);
00689
00690     mpf_trunc(aux1,floatin); /* This should now be an integer */
00691     mpf_sub(aux2,floatin,aux1); /* and this >= 0 */
00692     if ( mpf_sgn(aux1) == 0 ) { *out++ = 0; startnul = 1; }
00693     else {
00694         nout = FloatToInteger((UWORD *)out,aux1,&totalbitsused);
00695         out += nout;
00696         startnul = 0;
00697     }
00698     AT.pWorkspace[AT.pWorkPointer++] = out;
00699     if ( mpf_sgn(aux2) ) {
00700         for(;;) {
00701             mpf_ui_div(aux3,1,aux2);
00702             mpf_trunc(aux1,aux3);
00703             mpf_sub(aux2,aux3,aux1);
00704             if ( mpf_sgn(aux1) == 0 ) { *out++ = 0; }
00705             else {
00706                 nout = FloatToInteger((UWORD *)out,aux1,&bitsused);
00707                 out += nout;
00708                 totalbitsused += bitsused;
00709             }
00710             if ( bitsused > (totalbits-totalbitsused)/2 ) { break; }
00711             if ( mpf_sgn(aux2) == 0 ) {
00712                 /*if ( startnul == 1 )*/ AT.pWorkspace[AT.pWorkPointer++] = out;
00713                 break;
00714             }
00715             AT.pWorkspace[AT.pWorkPointer++] = out;
00716         }
00717     }
00718     /*
00719     At this point we have the function with the repeated fraction.
00720     Now we should work out the fraction. Form code would be:
00721         repeat id dum_(?a,x1?,x2?) = dum_(?a,x1+1/x2);
00722         id dum_(x?) = x;
00723     We have to work backwards. This is why we made a list of pointers
00724     in AT.pWorkspace
00725
00726     Now we need the long integers a and b.
00727     Note that by construction there should never be a GCD!
00728 */

```



```

00729     out = (WORD *) (AT.pWorkspace[--AT.pWorkPointer]);
00730     na = 1; a[0] = 1;
00731     c = (UWORD *) (AT.pWorkspace[--AT.pWorkPointer]);
00732     nc = nb = ((UWORD *) out) - c;
00733     if ( nc > 10 ) {
00734         mc = c;
00735         c = (UWORD *) (AT.pWorkspace[--AT.pWorkPointer]);
00736         nc = nb = ((UWORD *) mc) - c;
00737     }
00738     for ( i = 0; i < nb; i++ ) b[i] = c[i];
00739     mc = c = NumberMalloc("FloatToRat");
00740     while ( AT.pWorkPointer > oldpWorkPointer ) {
00741         d = (UWORD *) (AT.pWorkspace[--AT.pWorkPointer]);
00742         nd = (UWORD *) (AT.pWorkspace[AT.pWorkPointer+1]) - d; /* This is the x1 in the formula */
00743         if ( nd == 1 && *d == 0 ) break;
00744         c = a; a = b; b = c; nc = na; na = nb; nb = nc; /* 1/x2 */
00745         MullLong(d, nd, a, na, mc, &nc);
00746         AddLong(mc, nc, b, nb, b, &nb);
00747     }
00748     NumberFree(mc, "FloatToRat");
00749     if ( startnul == 0 ) {
00750         c = a; a = b; b = c; nc = na; na = nb; nb = nc;
00751     }
00752 }
00753 else {
00754     c = (UWORD *) (AT.pWorkspace[oldpWorkPointer]);
00755     na = (UWORD *) out - c;
00756     for ( i = 0; i < na; i++ ) a[i] = c[i];
00757     b[0] = 1; nb = 1;
00758 }
00759 /*
00760 Now we have the fraction in a/b. Create the rational and add the sign.
00761 */
00762 d = ratout;
00763 if ( na == nb ) {
00764     *nratout = (2*na+1)*s;
00765     for ( i = 0; i < na; i++ ) *d++ = a[i];
00766     for ( i = 0; i < nb; i++ ) *d++ = b[i];
00767 }
00768 else if ( na < nb ) {
00769     *nratout = (2*nb+1)*s;
00770     for ( i = 0; i < na; i++ ) *d++ = a[i];
00771     for ( ; i < nb; i++ ) *d++ = 0;
00772     for ( i = 0; i < nb; i++ ) *d++ = b[i];
00773 }
00774 else {
00775     *nratout = (2*na+1)*s;
00776     for ( i = 0; i < na; i++ ) *d++ = a[i];
00777     for ( i = 0; i < nb; i++ ) *d++ = b[i];
00778     for ( ; i < na; i++ ) *d++ = 0;
00779 }
00780 *d = (UWORD) (*nratout);
00781 NumberFree(b, "FloatToRat");
00782 NumberFree(a, "FloatToRat");
00783 AT.pWorkPointer = oldpWorkPointer;
00784 /*
00785 Just for check we go back to float
00786 */
00787 if ( s < 0 ) mpf_neg(floatin, floatin);
00788 /*
00789 {
00790     WORD n = *ratout;
00791     RatToFloat(aux1, ratout, n);
00792     mpf_sub(aux2, floatin, aux1);
00793     gmp_printf("Diff is %.*Fe\n", 40, aux2);
00794 }
00795 */
00796 return(retval);
00797 }
00798
00799 /*
00800 #] FloatToRat :
00801 #[ ZeroTable :
00802 */
00803 void ZeroTable(mpf_t *tab, int N)
00804 {
00805     int i, j;
00806     for ( i = 0; i < N; i++ ) {
00807         for ( j = 0; j < tab[i]->_mp_prec; j++ ) tab[i]->_mp_d[j] = 0;
00808     }
00809 }
00810
00811 /*
00812 #] ZeroTable :
00813 #[ ReadFloat :
00814
00815

```

```

00816      Used by the compiler. It reads a floating point number and
00817      outputs it at AT.WorkPointer as if it were a float_ function
00818      with its arguments.
00819      The call enters with s[-1] == TFLOAT.
00820  */
00821
00822 SBYTE *ReadFloat(SBYTE *s)
00823 {
00824     GETIDENTITY
00825     SBYTE *ss, c;
00826     ss = s;
00827     while ( *ss > 0 ) ss++;
00828     c = *ss; *ss = 0;
00829     gmp_sscanf((char *)s,"%Ff",aux1);
00830     if ( AT.WorkPointer > AT.WorkTop ) {
00831         MLOCK(ErrorMessageLock);
00832         MesWork();
00833         MUNLOCK(ErrorMessageLock);
00834         Terminate(-1);
00835     }
00836     PackFloat(AT.WorkPointer,aux1);
00837     *ss = c;
00838     return(ss);
00839 }
00840
00841 /*
00842     #] ReadFloat :
00843     #[ CheckFloat :
00844
00845     For the tokenizer. Tests whether the input string can be a float.
00846  */
00847
00848 UBYTE *CheckFloat(UBYTE *ss, int *spec)
00849 {
00850     GETIDENTITY
00851     UBYTE *s = ss;
00852     int zero = 1, gotdot = 0;
00853     while ( FG.cTable[s[-1]] == 1 ) s--;
00854     *spec = 0;
00855     if ( FG.cTable[*s] == 1 ) {
00856         while ( *s == '0' ) s++;
00857         if ( FG.cTable[*s] == 1 ) {
00858             s++;
00859             while ( FG.cTable[*s] == 1 ) s++;
00860             zero = 0;
00861         }
00862         if ( *s == '.' ) { goto dot; }
00863     }
00864     else if ( *s == '.' ) {
00865 dot:
00866         gotdot = 1;
00867         s++;
00868         if ( FG.cTable[*s] != 1 && zero == 1 ) return(ss);
00869         while ( *s == '0' ) s++;
00870         if ( FG.cTable[*s] == 1 ) {
00871             s++;
00872             while ( FG.cTable[*s] == 1 ) s++;
00873             zero = 0;
00874         }
00875     }
00876     else return(ss);
00877 /*
00878     Now we have had the mantissa part, which may be zero.
00879     Check for an exponent.
00880  */
00881     if ( *s == 'e' || *s == 'E' ) {
00882         s++;
00883         if ( *s == '-' || *s == '+' ) s++;
00884         if ( FG.cTable[*s] == 1 ) {
00885             s++;
00886             while ( FG.cTable[*s] == 1 ) s++;
00887         }
00888         else { return(ss); }
00889     }
00890     else if ( gotdot == 0 ) return(ss);
00891     if ( AT.aux_ == 0 ) { /* no floating point system */
00892         *spec = -1;
00893         return(s);
00894     }
00895     if ( zero ) *spec = 1;
00896     return(s);
00897 }
00898
00899 /*
00900     #] CheckFloat :
00901     #] Low Level :
00902     #[ Float Routines :

```

```

00903     #] SetFloatPrecision :
00904
00905     We set the default precision of the floats and allocate
00906     space for an output string if we want to write the float.
00907     Space needed: exponent: up to 12 chars.
00908     mantissa 2+10*prec/33 + a little bit extra.
00909 */
00910
00911 int SetFloatPrecision(WORD prec)
00912 {
00913     if ( prec <= 0 ) {
00914         MesPrint("&Illegal value for number of bits for floating point constants: %d",prec);
00915         return(-1);
00916     }
00917     else {
00918         AC.DefaultPrecision = prec;
00919         if ( AO.floatspace != 0 ) M_free(AO.floatspace,"floatspace");
00920         AO.floatsize = ((10*prec)/33+20)*sizeof(char);
00921         AO.floatspace = (UBYTE *)Malloc1(AO.floatsize,"floatspace");
00922         mpf_set_default_prec(prec);
00923         return(0);
00924     }
00925 }
00926
00927 /*
00928     #] SetFloatPrecision :
00929     #] PrintFloat :
00930
00931     Print the float_ function with its arguments as a floating point
00932     number with numdigits digits.
00933     If however we use rawfloat as a format option it prints it as a
00934     regular Form function and it should never come here because the
00935     print routines in sch.c should intercept it.
00936     The buffer AO.floatspace is allocated when the precision of the
00937     floats is set in the routine SetupMZVTables.
00938 */
00939
00940 int PrintFloat(WORD *fun,int numdigits)
00941 {
00942     GETIDENTITY
00943     int n = 0;
00944     int prec = (AC.DefaultPrecision-AC.MaxWeight-1)*log10(2.0);
00945     if ( numdigits > prec || numdigits == 0 ) {
00946         numdigits = prec;
00947     }
00948 /*
00949     GMP's gmp_snprintf always prints a non-zero number before the decimal point, so we
00950     ask for one digit less.
00951 */
00952     if ( UnpackFloat(aux4,fun) == 0 )
00953         n = gmp_snprintf((char *) (AO.floatspace),AO.floatsize,"%.*Fe",numdigits-1,aux4);
00954     if ( numdigits == prec ) {
00955         UBYTE *s1, *s2;
00956         int n1, n2 = n;
00957         s1 = AO.floatspace+n;
00958         while ( s1 > AO.floatspace && s1[-1] != 'e'
00959             && s1[-1] != 'E' && s1[-1] != '.' ) { s1--; n2--; }
00960         if ( s1 > AO.floatspace && s1[-1] != '.' ) {
00961             s1--; n2--;
00962             s2 = s1; n1 = n2;
00963             while ( s1[-1] == '0' ) { s1--; n1--; }
00964             if ( s1[-1] == '.' ) { s1++; n1++; }
00965             while ( n2 < n ) { *s1++ = *s2++; n2++; }
00966             n -= (n2-n1);
00967             *s1 = 0;
00968         }
00969     }
00970     return(n);
00971 }
00972
00973 /*
00974     #] PrintFloat :
00975     #] AddFloats :
00976 */
00977
00978 int AddFloats(PHEAD WORD *fun3, WORD *fun1, WORD *fun2)
00979 {
00980     int retval = 0;
00981     if ( UnpackFloat(aux1,fun1) == 0 && UnpackFloat(aux2,fun2) == 0 ) {
00982         mpf_add(aux1,aux1,aux2);
00983         PackFloat(fun3,aux1);
00984     }
00985     else { retval = -1; }
00986     return(retval);
00987 }
00988
00989 /*

```

```

00990         #] AddFloats :
00991         #[ MulFloats :
00992     */
00993
00994 int MulFloats(PHEAD WORD *fun3, WORD *fun1, WORD *fun2)
00995 {
00996     int retval = 0;
00997     if ( UnpackFloat(aux1,fun1) == 0 && UnpackFloat(aux2,fun2) == 0 ) {
00998         mpf_mul(aux1,aux1,aux2);
00999         PackFloat(fun3,aux1);
01000     }
01001     else { retval = -1; }
01002     return(retval);
01003 }
01004
01005 /*
01006     #] MulFloats :
01007     #[ DivFloats :
01008 */
01009
01010 int DivFloats(PHEAD WORD *fun3, WORD *fun1, WORD *fun2)
01011 {
01012     int retval = 0;
01013     if ( UnpackFloat(aux1,fun1) == 0 && UnpackFloat(aux2,fun2) == 0 ) {
01014         mpf_div(aux1,aux1,aux2);
01015         PackFloat(fun3,aux1);
01016     }
01017     else { retval = -1; }
01018     return(retval);
01019 }
01020
01021 /*
01022     #] DivFloats :
01023     #[ AddRatToFloat :
01024
01025     Note: this can be optimized, because often the rat is rather simple.
01026 */
01027
01028 int AddRatToFloat(PHEAD WORD *outfun, WORD *infun, UWORD *formrat, WORD nrat)
01029 {
01030     int retval = 0;
01031     if ( UnpackFloat(aux2,infun) == 0 ) {
01032         RatToFloat(aux1,formrat,nrat);
01033         mpf_add(aux2,aux2,aux1);
01034         PackFloat(outfun,aux2);
01035     }
01036     else retval = -1;
01037     return(retval);
01038 }
01039
01040 /*
01041     #] AddRatToFloat :
01042     #[ MulRatToFloat :
01043
01044     Note: this can be optimized, because often the rat is rather simple.
01045 */
01046
01047 int MulRatToFloat(PHEAD WORD *outfun, WORD *infun, UWORD *formrat, WORD nrat)
01048 {
01049     int retval = 0;
01050     if ( UnpackFloat(aux2,infun) == 0 ) {
01051         RatToFloat(aux1,formrat,nrat);
01052         mpf_mul(aux2,aux2,aux1);
01053         PackFloat(outfun,aux2);
01054     }
01055     else retval = -1;
01056     return(retval);
01057 }
01058
01059 /*
01060     #] MulRatToFloat :
01061     #[ SetupMZVTables :
01062 */
01063
01064 void SetupMZVTables(void)
01065 {
01066     /*
01067     Sets up a table of N+1 mpf_t floats with variable precision.
01068     It assumes that each next element needs one bit less.
01069     Initializes all of them.
01070     We take some extra space, to have one limb overflow everywhere.
01071     Because the depth of the MZV's can get close to the weight
01072     and each deeper sum goes one higher, we make the tables size a bit bigger.
01073     This may not be needed if we fiddle with the sum boundaries.
01074     */
01075     #ifdef WITHPTHREADS
01076         int i, Nw, id, totnum;

```

```

01077     size_t N;
01078     mpf_t *a;
01079     Nw = AC.DefaultPrecision;
01080     N = (size_t)Nw;
01081     totnum = AM.totalnumberofthreads;
01082     for ( id = 0; id < totnum; id++ ) {
01083         if ( AB[id]->T.mpf_tab1 ) {
01084             a = (mpf_t *)AB[id]->T.mpf_tab1;
01085             for ( i = 0; i <=Nw; i++ ) {
01086                 mpf_clear(a[i]);
01087             }
01088             M_free(AB[id]->T.mpf_tab1,"mpftab1");
01089         }
01090         AB[id]->T.mpf_tab1 = (void *)Malloc1((N+2)*sizeof(mpf_t),"mpftab1");
01091         a = (mpf_t *)AB[id]->T.mpf_tab1;
01092         for ( i = 0; i <=Nw; i++ ) {
01093             /*
01094              As explained in the comment above, we could make this variable precision
01095              using mpf_init2.
01096             */
01097             mpf_init(a[i]);
01098         }
01099         if ( AB[id]->T.mpf_tab2 ) {
01100             a = (mpf_t *)AB[id]->T.mpf_tab2;
01101             for ( i = 0; i <=Nw; i++ ) {
01102                 mpf_clear(a[i]);
01103             }
01104             M_free(AB[id]->T.mpf_tab2,"mpftab2");
01105         }
01106         AB[id]->T.mpf_tab2 = (void *)Malloc1((N+2)*sizeof(mpf_t),"mpftab2");
01107         a = (mpf_t *)AB[id]->T.mpf_tab2;
01108         for ( i = 0; i <=Nw; i++ ) {
01109             mpf_init(a[i]);
01110         }
01111     }
01112 #else
01113     int i, Nw;
01114     size_t N;
01115     Nw = AC.DefaultPrecision;
01116     N = (size_t)Nw;
01117     if ( AT.mpf_tab1 ) {
01118         for ( i = 0; i <= Nw; i++ ) {
01119             mpf_clear(mpftab1[i]);
01120         }
01121         M_free(AT.mpf_tab1,"mpftab1");
01122     }
01123     AT.mpf_tab1 = (void *)Malloc1((N+2)*sizeof(mpf_t),"mpftab1");
01124     for ( i = 0; i <= Nw; i++ ) {
01125         /*
01126          As explained in the comment above, we could make this variable precision
01127          using mpf_init2.
01128         */
01129         mpf_init(mpftab1[i]);
01130     }
01131     if ( AT.mpf_tab2 ) {
01132         for ( i = 0; i <= Nw; i++ ) {
01133             mpf_clear(mpftab2[i]);
01134         }
01135         M_free(AT.mpf_tab2,"mpftab2");
01136     }
01137     AT.mpf_tab2 = (void *)Malloc1((N+2)*sizeof(mpf_t),"mpftab2");
01138     for ( i = 0; i <= Nw; i++ ) {
01139         mpf_init(mpftab2[i]);
01140     }
01141 #endif
01142     if ( AS.delta_1 ) {
01143         mpf_clear(mpfdelta1);
01144         M_free(AS.delta_1,"delta1");
01145     }
01146     AS.delta_1 = (void *)Malloc1(sizeof(mpf_t),"delta1");
01147     mpf_init(mpfdelta1);
01148     SimpleDelta(mpfdelta1,1); /* this can speed up things. delta1 = ln(2) */
01149     /*
01150     Finally the character buffer for printing
01151     if ( AO.floatspace ) M_free(AO.floatspace,"floatspace");
01152     AO.floatspace = (UBYTE *)Malloc1(((10*AC.DefaultPrecision)/33+40)*sizeof(UBYTE),"floatspace");
01153     */
01154 }
01155
01156 /*
01157     #] SetupMZVTables :
01158     #[ SetupMPFTables :
01159     */
01160
01161 void SetupMPFTables(void)
01162 {
01163 #ifdef WITHPTHREADS

```

```

01164     int id, totnum;
01165     mpf_t *a;
01166     /*
01167     Now the aux variables
01168     */
01169     #ifdef WITHSORTBOTS
01170     totnum = MaX(2*AM.totalnumberofthreads-3,AM.totalnumberofthreads);
01171     #endif
01172     for ( id = 0; id < totnum; id++ ) {
01173         if ( AB[id]->T.aux_ ) {
01174             /*
01175             We work here with a[0] etc because the aux1 etc contain B which
01176             in the current routine would be AB[0] only
01177             */
01178             a = (mpf_t *)AB[id]->T.aux_;
01179             mpf_clears(a[0],a[1],a[2],a[3],a[4],a[5],a[6],a[7],(mpf_ptr)0);
01180             M_free(AB[id]->T.aux_,"AB[id]->T.aux_");
01181         }
01182         AB[id]->T.aux_ = (void *)Malloca(sizeof(mpf_t)*8,"AB[id]->T.aux_");
01183         a = (mpf_t *)AB[id]->T.aux_;
01184         mpf_inits(a[0],a[1],a[2],a[3],a[4],a[5],a[6],a[7],(mpf_ptr)0);
01185         if ( AB[id]->T.indil ) M_free(AB[id]->T.indil,"indil");
01186         AB[id]->T.indil = (int *)Malloca(sizeof(int)*AC.MaxWeight*2,"indil");
01187         AB[id]->T.indi2 = AB[id]->T.indil + AC.MaxWeight;
01188     }
01189     #else
01190     /*
01191     Now the aux variables
01192     */
01193     if ( AT.aux_ ) {
01194         mpf_clears(aux1,aux2,aux3,aux4,aux5,auxjm,auxjjm,auxsum,(mpf_ptr)0);
01195         M_free(AT.aux_,"AT.aux_");
01196     }
01197     AT.aux_ = (void *)Malloca(sizeof(mpf_t)*8,"AT.aux_");
01198     mpf_inits(aux1,aux2,aux3,aux4,aux5,auxjm,auxjjm,auxsum,(mpf_ptr)0);
01199     if ( AT.indil ) M_free(AT.indil,"indil");
01200     AT.indil = (int *)Malloca(sizeof(int)*AC.MaxWeight*2,"indil");
01201     AT.indi2 = AT.indil + AC.MaxWeight;
01202     #endif
01203     /*
01204     Finally the character buffer for printing
01205     if ( AO.floatspace ) M_free(AO.floatspace,"floatspace");
01206     AO.floatspace = (UBYTE *)Malloca(((10*AC.DefaultPrecision)/33+40)*sizeof(UBYTE),"floatspace");
01207     */
01208 }
01209
01210 /*
01211     #] SetupMPFTables :
01212     #[ ClearMZVTables :
01213     */
01214
01215 void ClearMZVTables(void)
01216 {
01217     #ifdef WITHPTHREADS
01218     int i, id, totnum;
01219     mpf_t *a;
01220     totnum = AM.totalnumberofthreads;
01221     for ( id = 0; id < totnum; id++ ) {
01222         if ( AB[id]->T.mpf_tab1 ) {
01223             /*
01224             We work here with a[0] etc because the aux1, mpftab1 etc contain B
01225             which in the current routine would be AB[0] only
01226             */
01227             a = (mpf_t *)AB[id]->T.mpf_tab1;
01228             for ( i = 0; i <=AC.DefaultPrecision; i++ ) {
01229                 mpf_clear(a[i]);
01230             }
01231             M_free(AB[id]->T.mpf_tab1,"mpftab1");
01232             AB[id]->T.mpf_tab1 = 0;
01233         }
01234         if ( AB[id]->T.mpf_tab2 ) {
01235             a = (mpf_t *)AB[id]->T.mpf_tab2;
01236             for ( i = 0; i <=AC.DefaultPrecision; i++ ) {
01237                 mpf_clear(a[i]);
01238             }
01239             M_free(AB[id]->T.mpf_tab2,"mpftab2");
01240             AB[id]->T.mpf_tab2 = 0;
01241         }
01242     }
01243     #ifdef WITHSORTBOTS
01244     totnum = MaX(2*AM.totalnumberofthreads-3,AM.totalnumberofthreads);
01245     #endif
01246     for ( id = 0; id < totnum; id++ ) {
01247         if ( AB[id]->T.aux_ ) {
01248             a = (mpf_t *)AB[id]->T.aux_;
01249             mpf_clears(a[0],a[1],a[2],a[3],a[4],a[5],a[6],a[7],(mpf_ptr)0);
01250             M_free(AB[id]->T.aux_,"AB[id]->T.aux_");

```

```

01251         AB[id]->T.aux_ = 0;
01252     }
01253     if ( AB[id]->T.indil ) { M_free(AB[id]->T.indil,"indil"); AB[id]->T.indil = 0; }
01254 }
01255 #else
01256     int i;
01257     if ( AT.mpf_tab1 ) {
01258         for ( i = 0; i <= AC.DefaultPrecision; i++ ) {
01259             mpf_clear(mpftab1[i]);
01260         }
01261         M_free(AT.mpf_tab1,"mpftab1");
01262         AT.mpf_tab1 = 0;
01263     }
01264     if ( AT.mpf_tab2 ) {
01265         for ( i = 0; i <= AC.DefaultPrecision; i++ ) {
01266             mpf_clear(mpftab2[i]);
01267         }
01268         M_free(AT.mpf_tab2,"mpftab2");
01269         AT.mpf_tab2 = 0;
01270     }
01271     if ( AT.aux_ ) {
01272         mpf_clears(aux1,aux2,aux3,aux4,aux5,auxjm,auxjjm,auxsum,(mpf_ptr)0);
01273         M_free(AT.aux_,"AT.aux_");
01274         AT.aux_ = 0;
01275     }
01276     if ( AT.indil ) { M_free(AT.indil,"indil"); AT.indil = 0; }
01277 #endif
01278     if ( AO.floatspace ) { M_free(AO.floatspace,"floatspace"); AO.floatspace = 0; }
01279     AO.floatsize = 0; }
01280     if ( AS.delta_1 ) {
01281         mpf_clear(mpfdelta1);
01282         M_free(AS.delta_1,"delta1");
01283         AS.delta_1 = 0;
01284     }
01285 }
01286
01287 /*
01288     #] ClearMZVTables :
01289     #[ CoToFloat :
01290 */
01291
01292 int CoToFloat(UBYTE *s)
01293 {
01294     GETIDENTITY
01295     if ( AT.aux_ == 0 ) {
01296         MesPrint("&Illegal attempt to convert to float_ without activating floating point numbers.");
01297         MesPrint("&Forgotten %#startfloat instruction?");
01298         return(1);
01299     }
01300     while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
01301     if ( *s ) {
01302         MesPrint("&Illegal argument(s) in ToFloat statement: '%s'",s);
01303         return(1);
01304     }
01305     Add2Com(TYPETOFLOAT);
01306     return(0);
01307 }
01308
01309 /*
01310     #] CoToFloat :
01311     #[ CoToRat :
01312 */
01313
01314 int CoToRat(UBYTE *s)
01315 {
01316     GETIDENTITY
01317     if ( AT.aux_ == 0 ) {
01318         MesPrint("&Illegal attempt to convert from float_ without activating floating point
01319 numbers.");
01319         MesPrint("&Forgotten %#startfloat instruction?");
01320         return(1);
01321     }
01322     while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
01323     if ( *s ) {
01324         MesPrint("&Illegal argument(s) in ToRat statement: '%s'",s);
01325         return(1);
01326     }
01327     Add2Com(TYPETORAT);
01328     return(0);
01329 }
01330
01331 /*
01332     #] CoToRat :
01333     #[ ToFloat :
01334
01335     Converts the coefficient to floating point if it is still a rat.
01336 */

```

```

01337
01338 int ToFloat(PHEAD WORD *term, WORD level)
01339 {
01340     WORD *t, *tstop, nsize, nsign = 3;
01341     t = term+*term;
01342     nsize = ABS(t[-1]);
01343     tstop = t - nsize;
01344     if ( t[-1] < 0 ) { nsign = -nsign; }
01345     if ( nsize == 3 && t[-2] == 1 && t[-3] == 1 ) { /* Could be float */
01346         t = term + 1;
01347         while ( t < tstop ) {
01348             if ( (*t == FLOATFUN) && ( t+t[1] == tstop ) ) {
01349                 return(Generator(BHEAD term,level));
01350             }
01351             t += t[1];
01352         }
01353     }
01354     RatToFloat(aux4, (UWORD *)tstop,nsize);
01355     PackFloat(tstop,aux4);
01356     tstop += tstop[1];
01357     *tstop++ = 1; *tstop++ = 1; *tstop++ = nsign;
01358     *term = tstop - term;
01359     AT.WorkPointer = tstop;
01360     return(Generator(BHEAD term,level));
01361 }
01362
01363 /*
01364     #] ToFloat :
01365     #[ ToRat :
01366
01367     Tries to convert the coefficient to rational if it is still a float.
01368 */
01369
01370 int ToRat(PHEAD WORD *term, WORD level)
01371 {
01372     WORD *tstop, *t, nsize, nsign, ncoef;
01373     /*
01374     1: find the float which should be at the end.
01375     */
01376     tstop = term + *term; nsize = ABS(tstop[-1]);
01377     nsign = tstop[-1] < 0 ? -1: 1; tstop -= nsize;
01378     t = term+1;
01379     while ( t < tstop ) {
01380         if ( *t == FLOATFUN && t + t[1] == tstop && TestFloat(t) &&
01381             nsize == 3 && tstop[0] == 1 && tstop[1] == 1 ) break;
01382         t += t[1];
01383     }
01384     if ( t < tstop ) {
01385         /*
01386             Now t points at the float_ function and everything is correct.
01387             The result can go straight over the float_ function.
01388         */
01389         UnpackFloat(aux4,t);
01390         if ( FloatToRat((UWORD *)t,&ncoef,aux4) == 0 ) {
01391             t += ABS(ncoef);
01392             t[-1] = ncoef*nsign;
01393             *term = t - term;
01394         }
01395     }
01396     return(Generator(BHEAD term,level));
01397 }
01398
01399 /*
01400     #] ToRat :
01401     #] Float Routines :
01402     #[ Sorting :
01403
01404     We need a method to see whether two terms need addition that could
01405     involve floating point numbers.
01406     1: if PolyWise is active, we do not need anything, because
01407         Poly(Rat)Fun and floating point are mutually exclusive.
01408     2: This means that there should be only interference in AddCoef.
01409         and the AddRat parts in MergePatches, MasterMerge, SortBotMerge
01410         and PF_GetLoser.
01411     The compare routine(s) should recognise the float_ and give off
01412     a code (SortFloatMode) inside the AT struct:
01413     0: no float_
01414     1: float_ in term1 only
01415     2: float_ in term2 only
01416     3: float_ in both terms
01417     Be careful: we have several compare routines, because various codes
01418     use their own routines and we just set a variable with its address.
01419     Currently we have Compare1, CompareSymbols and CompareHSymbols.
01420     Only Compare1 should be active for this. The others should make sure
01421     that the proper variable is always zero.
01422     Remember: the sign of the coefficient is in the last word of the term.
01423

```



```

01424         We need two routines:
01425         1: AddWithFloat for SplitMerge which sorts by pointer.
01426         2: MergeWithFloat for MergePatches etc which keeps terms as much
01427            as possible in their location.
01428         The routines start out the same, because AT.SortFloatMode, set in
01429         Compare1, tells more or less what should be done.
01430         The difference is in where we leave the result.
01431
01432         In the future we may want to add a feature that makes the result
01433         zero if the sum comes within a certain distance of the numerical
01434         accuracy, like for instance 5% of the number of bits.
01435
01436         #[] AddWithFloat :
01437     */
01438
01439 WORD AddWithFloat(PHEAD WORD **ps1, WORD **ps2)
01440 {
01441     GETBIDENTITY
01442     SORTING *S = AT.SS;
01443     WORD *coef1, *coef2, size1, size2, *fun1, *fun2, *fun3;
01444     WORD *s1,*s2,sign3,j,jj, *t1, *t2, i;
01445     s1 = *ps1;
01446     s2 = *ps2;
01447     coef1 = s1+s1; size1 = coef1[-1]; coef1 -= ABS(coef1[-1]);
01448     coef2 = s2+s2; size2 = coef2[-1]; coef2 -= ABS(coef2[-1]);
01449     if ( AT.SortFloatMode == 3 ) {
01450         fun1 = s1+1; while ( fun1 < coef1 && fun1[0] != FLOATFUN ) fun1 += fun1[1];
01451         fun2 = s2+1; while ( fun2 < coef2 && fun2[0] != FLOATFUN ) fun2 += fun2[1];
01452         UnpackFloat(aux1,fun1);
01453         if ( size1 < 0 ) mpf_neg(aux1,aux1);
01454         UnpackFloat(aux2,fun2);
01455         if ( size2 < 0 ) mpf_neg(aux2,aux2);
01456     }
01457     else if ( AT.SortFloatMode == 1 ) {
01458         fun1 = s1+1; while ( fun1 < coef1 && fun1[0] != FLOATFUN ) fun1 += fun1[1];
01459         UnpackFloat(aux1,fun1);
01460         if ( size1 < 0 ) mpf_neg(aux1,aux1);
01461         fun2 = coef2;
01462         RatToFloat(aux2,(UWORD *)coef2,size2);
01463     }
01464     else if ( AT.SortFloatMode == 2 ) {
01465         fun2 = s2+1; while ( fun2 < coef2 && fun2[0] != FLOATFUN ) fun2 += fun2[1];
01466         fun1 = coef1;
01467         RatToFloat(aux1,(UWORD *)coef1,size1);
01468         UnpackFloat(aux2,fun2);
01469         if ( size2 < 0 ) mpf_neg(aux2,aux2);
01470     }
01471     else {
01472         MLOCK(ErrorMessageLock);
01473         MesPrint("Illegal value %d for AT.SortFloatMode in AddWithFloat.",AT.SortFloatMode);
01474         MUNLOCK(ErrorMessageLock);
01475         Terminate(-1);
01476         return(0);
01477     }
01478     mpf_add(aux3,aux1,aux2);
01479     sign3 = mpf_sgn(aux3);
01480     if ( sign3 == 0 ) { /* May be rare! */
01481         *ps1 = 0; *ps2 = 0; AT.SortFloatMode = 0; return(0);
01482     }
01483     if ( sign3 < 0 ) mpf_neg(aux3,aux3);
01484     fun3 = TermMalloc("AddWithFloat");
01485     PackFloat(fun3,aux3);
01486     /*
01487     Now we have to calculate where the sumterm fits.
01488     If we are sloppy, we can be faster, but run the risk to need the
01489     extension space, even when it is not needed. At slightly lower speed
01490     (ie first creating the result in scribble space) we are better off.
01491     This is why we use TermMalloc.
01492
01493     The new term will have a rational coefficient of 1,1,+3.
01494     The size will be (fun1 or fun2 - term) + fun3 +3;
01495     */
01496     if ( AT.SortFloatMode == 3 ) {
01497         if ( fun1[1] == fun3[1] ) {
01498 Over1:
01499             i = fun3[1]; t1 = fun1; t2 = fun3; NCOPY(t1,t2,i);
01500             *t1++ = 1; *t1++ = 1; *t1++ = sign3 < 0 ? -3: 3;
01501             *s1 = t1-s1; goto Finished;
01502         }
01503         else if ( fun2[1] == fun3[1] ) {
01504 Over2:
01505             i = fun3[1]; t1 = fun2; t2 = fun3; NCOPY(t1,t2,i);
01506             *t1++ = 1; *t1++ = 1; *t1++ = sign3 < 0 ? -3: 3;
01507             *s2 = t1-s2; *ps1 = s2; goto Finished;
01508         }
01509         else if ( fun1[1] >= fun3[1] ) goto Over1;
01510         else if ( fun2[1] >= fun3[1] ) goto Over2;

```

```

01511     }
01512     else if ( AT.SortFloatMode == 1 ) {
01513         if ( fun1[1] >= fun3[1] ) goto Over1;
01514         else if ( fun3[1]+3 <= ABS(size2) ) goto Over2;
01515     }
01516     else if ( AT.SortFloatMode == 2 ) {
01517         if ( fun2[1] >= fun3[1] ) goto Over2;
01518         else if ( fun3[1]+3 <= ABS(size1) ) goto Over1;
01519     }
01520     /*
01521     Does not fit. Go to extension space.
01522     */
01523     jj = fun1-s1;
01524     j = jj+fun3[1]+3; /* space needed */
01525     if ( (S->sFill + j) >= S->sTop2 ) {
01526         GarbHand();
01527         s1 = *ps1; /* new position of the term after the garbage collection */
01528         fun1 = s1+jj;
01529     }
01530     t1 = S->sFill;
01531     for ( i = 0; i < jj; i++ ) *t1++ = s1[i];
01532     i = fun3[1]; s1 = fun3; NCOPY(t1,s1,i);
01533     *t1++ = 1; *t1++ = 1; *t1++ = sign3 < 0 ? -3: 3;
01534     *ps1 = S->sFill;
01535     **ps1 = t1-*ps1;
01536     S->sFill = t1;
01537 Finished:
01538     *ps2 = 0;
01539     TermFree(fun3,"AddWithFloat");
01540     AT.SortFloatMode = 0;
01541     if ( **ps1 > AM.MaxTer/((LONG)(sizeof(WORD))) ) {
01542         MLOCK(ErrorMessageLock);
01543         MesPrint("Term too complex after addition in sort. MaxTermSize = %10l",
01544             AM.MaxTer/sizeof(WORD));
01545         MUNLOCK(ErrorMessageLock);
01546         Terminate(-1);
01547     }
01548     return(1);
01549 }
01550
01551 /*
01552     #] AddWithFloat :
01553     #[ MergeWithFloat :
01554
01555     Note that we always park the result on top of term1.
01556     This makes life easier, because the original AddRat in MergePatches
01557     does this as well.
01558 */
01559
01560 WORD MergeWithFloat(PHEAD WORD **interm1, WORD **interm2)
01561 {
01562     GETBIDENTITY
01563     WORD *coef1, *coef2, size1, size2, *fun1, *fun2, *fun3, *tt;
01564     WORD sign3,j,jj, *t1, *t2, i, *term1 = *interm1, *term2 = *interm2, retval = 0;
01565     coef1 = term1+*term1; size1 = coef1[-1]; coef1 -= ABS(size1);
01566     coef2 = term2+*term2; size2 = coef2[-1]; coef2 -= ABS(size2);
01567     if ( AT.SortFloatMode == 3 ) {
01568         fun1 = term1+1; while ( fun1 < coef1 && fun1[0] != FLOATFUN ) fun1 += fun1[1];
01569         fun2 = term2+1; while ( fun2 < coef2 && fun2[0] != FLOATFUN ) fun2 += fun2[1];
01570         UnpackFloat(aux1,fun1);
01571         if ( size1 < 0 ) mpf_neg(aux1,aux1);
01572         UnpackFloat(aux2,fun2);
01573         if ( size2 < 0 ) mpf_neg(aux2,aux2);
01574     }
01575     else if ( AT.SortFloatMode == 1 ) {
01576         fun1 = term1+1; while ( fun1 < coef1 && fun1[0] != FLOATFUN ) fun1 += fun1[1];
01577         UnpackFloat(aux1,fun1);
01578         if ( size1 < 0 ) mpf_neg(aux1,aux1);
01579         fun2 = coef2;
01580         RatToFloat(aux2,(UWORD *)coef2,size2);
01581     }
01582     else if ( AT.SortFloatMode == 2 ) {
01583         fun2 = term2+1; while ( fun2 < coef2 && fun2[0] != FLOATFUN ) fun2 += fun2[1];
01584         fun1 = coef1;
01585         RatToFloat(aux1,(UWORD *)coef1,size1);
01586         UnpackFloat(aux2,fun2);
01587         if ( size2 < 0 ) mpf_neg(aux2,aux2);
01588     }
01589     else {
01590         MLOCK(ErrorMessageLock);
01591         MesPrint("Illegal value %d for AT.SortFloatMode in MergeWithFloat.",AT.SortFloatMode);
01592         MUNLOCK(ErrorMessageLock);
01593         Terminate(-1);
01594         return(0);
01595     }
01596     mpf_add(aux3,aux1,aux2);
01597     sign3 = mpf_sgn(aux3);

```

```

01598     if ( sign3 == 0 ) { /* May be very rare! */
01599         AT.SortFloatMode = 0; return(0);
01600     }
01601     /*
01602     Now check whether we can park the result on top of one of the input terms.
01603     */
01604     if ( sign3 < 0 ) mpf_neg(aux3,aux3);
01605     fun3 = TermMalloc("MergeWithFloat");
01606     PackFloat(fun3,aux3);
01607     if ( AT.SortFloatMode == 3 ) {
01608         if ( fun1[1] + ABS(size1) == fun3[1] + 3 ) {
01609 OnTopOf1:   t1 = fun3; t2 = fun1;
01610             for ( i = 0; i < fun3[1]; i++ ) *t2++ = *t1++;
01611             *t2++ = 1; *t2++ = 1; *t2++ = sign3 < 0 ? -3: 3;
01612             retval = 1;
01613         }
01614         else if ( fun1[1] + ABS(size1) > fun3[1] + 3 ) {
01615 Shift1:     t2 = term1 + *term1; tt = t2;
01616             *--t2 = sign3 < 0 ? -3: 3; *--t2 = 1; *--t2 = 1;
01617             t1 = fun3 + fun3[1]; for ( i = 0; i < fun3[1]; i++ ) *--t2 = *--t1;
01618             t1 = fun1;
01619             while ( t1 > term1 ) *--t2 = *--t1;
01620             *t2 = tt-t2; term1 = t2;
01621             retval = 1;
01622         }
01623         else { /* Here we have to move term1 to the left to make room. */
01624 Over1:     jj = fun3[1]-fun1[1]+3-ABS(size1); /* This is positive */
01625             t2 = term1-jj; t1 = term1;
01626             while ( t1 < fun1 ) *t2++ = *t1++;
01627             term1 -= jj;
01628             *term1 += jj;
01629             for ( i = 0; i < fun3[1]; i++ ) *t2++ = fun3[i];
01630             *t2++ = 1; *t2++ = 1; *t2++ = sign3 < 0 ? -3: 3;
01631             retval = 1;
01632         }
01633     }
01634     else if ( AT.SortFloatMode == 1 ) {
01635         if ( fun1[1] + ABS(size1) == fun3[1] + 3 ) goto OnTopOf1;
01636         else if ( fun1[1] + ABS(size1) > fun3[1] + 3 ) goto Shift1;
01637         else goto Over1;
01638     }
01639     else { /* Can only be 2, based on previous tests */
01640         if ( fun3[1] + 3 == ABS(size1) ) {
01641             t2 = coef1; t1 = fun3;
01642             for ( i = 0; i < fun3[1]; i++ ) *t2++ = *t1++;
01643             *t2++ = 1; *t2++ = 1; *t2++ = sign3 < 0 ? -3: 3;
01644             retval = 1;
01645         }
01646         else if ( fun3[1] + 3 < ABS(size1) ) {
01647             j = ABS(size1) - fun3[1] - 3;
01648             t2 = term1 + *term1; tt = t2;
01649             *--t2 = sign3 < 0 ? -3: 3; *--t2 = 1; *--t2 = 1;
01650             t2 -= fun3[1]; t1 = t2-j;
01651             while ( t2 > term1 ) *--t2 = *--t1;
01652             *t2 = tt-t2; term1 = t2;
01653             retval = 1;
01654         }
01655         else goto Over1;
01656     }
01657     *interm1 = term1;
01658     TermFree(fun3,"MergeWithFloat");
01659     AT.SortFloatMode = 0;
01660     return(retval);
01661 }
01662
01663 /*
01664     #] MergeWithFloat :
01665     #] Sorting :
01666     #[ MZV :
01667
01668     The functions here allow the arbitrary precision calculation
01669     of MZV's and Euler sums.
01670     They are called when the functions mzv_ and/or euler_ are used
01671     and the evaluate statement is applied.
01672     The output is in a float_ function.
01673     The expand statement tries to express the functions in terms of a basis.
01674     The bases are the 'standard basis for the euler sums and the
01675     pushdown bases from the datamine, unless otherwise specified.
01676
01677     #[ SimpleDelta :
01678 */
01679
01680 void SimpleDelta(mpf_t sum, int m)
01681 {
01682     long s = 1;
01683     long prec = AC.DefaultPrecision;
01684     unsigned long xprec = (unsigned long)prec, x;

```

```

01685     int j, jmax, n;
01686     mpf_t jm, jjm;
01687     mpf_init(jm); mpf_init(jjm);
01688     if ( m < 0 ) { s = -1; m = -m; }
01689
01690 /*
01691  We will loop until 1/2^j/j^m is smaller than the default precision.
01692  Just running to prec is however overkill, specially when m is large.
01693  We try to estimate a better value.
01694  jmax = prec - ((log2(prec)-1)*m).
01695  Hence we need the leading bit of prec.
01696  We are still overshooting a bit.
01697 */
01698     n = 0; x = xprec;
01699     while ( x ) { x >>= 1; n++; }
01700 /*
01701  We have now n = floor(log2(x))+1.
01702 */
01703     n--;
01704     jmax = (int)((int)xprec - (n-1)*m);
01705     mpf_set_ui(sum,0);
01706     for ( j = 1; j <= jmax; j++ ) {
01707 #ifdef WITHCUTOFF
01708         xprec--;
01709         mpf_set_prec_raw(jm,xprec);
01710         mpf_set_prec_raw(jjm,xprec);
01711 #endif
01712         mpf_set_ui(jm,1L);
01713         mpf_div_ui(jjm,jm,(unsigned long)j);
01714         mpf_pow_ui(jm,jjm,(unsigned long)m);
01715         mpf_div_2exp(jjm,jm,(unsigned long)j);
01716         if ( s == -1 && j%2 == 1 ) mpf_sub(sum,sum,jjm);
01717         else mpf_add(sum,sum,jjm);
01718     }
01719     mpf_clear(jjm); mpf_clear(jm);
01720 }
01721
01722 /*
01723     #] SimpleDelta :
01724     #[ SimpleDeltaC :
01725 */
01726
01727 void SimpleDeltaC(mpf_t sum, int m)
01728 {
01729     long s = 1;
01730     long prec = AC.DefaultPrecision;
01731     unsigned long xprec = (unsigned long)prec, x;
01732     int j, jmax, n;
01733     mpf_t jm, jjm;
01734     mpf_init(jm); mpf_init(jjm);
01735     if ( m < 0 ) { s = -1; m = -m; }
01736 /*
01737  We will loop until 1/2^j/j^m is smaller than the default precision.
01738  Just running to prec is however overkill, specially when m is large.
01739  We try to estimate a better value.
01740  jmax = prec - ((log2(prec)-1)*m).
01741  Hence we need the leading bit of prec.
01742  We are still overshooting a bit.
01743 */
01744     n = 0; x = xprec;
01745     while ( x ) { x >>= 1; n++; }
01746 /*
01747  We have now n = floor(log2(x))+1.
01748 */
01749     n--;
01750     jmax = (int)((int)xprec - (n-1)*m);
01751     if ( s < 0 ) jmax /= 2;
01752     mpf_set_si(sum,0L);
01753     for ( j = 1; j <= jmax; j++ ) {
01754 #ifdef WITHCUTOFF
01755         xprec--;
01756         mpf_set_prec_raw(jm,xprec);
01757         mpf_set_prec_raw(jjm,xprec);
01758 #endif
01759         mpf_set_ui(jm,1L);
01760         mpf_div_ui(jjm,jm,(unsigned long)j);
01761         mpf_pow_ui(jm,jjm,(unsigned long)m);
01762         if ( s == -1 ) mpf_div_2exp(jjm,jm,2*(unsigned long)j);
01763         else mpf_div_2exp(jjm,jm,(unsigned long)j);
01764         mpf_add(sum,sum,jjm);
01765     }
01766     mpf_clear(jjm); mpf_clear(jm);
01767 }
01768
01769 /*
01770     #] SimpleDeltaC :
01771     #[ SingleTable :

```

```

01772 */
01773
01774 void SingleTable(mpf_t *tabl, int N, int m, int pow)
01775 {
01776     /*
01777         Creates a table T(1,...,N) with
01778         T(i) = sum_(j,i,N,[sign_(j)]/2^j/j^m)
01779         To make this table we have two options:
01780         1: run the sum backwards with the potential problem that the
01781            precision is difficult to manage.
01782         2: run the sum forwards. Take sum_(j,1,i-1,...) and later subtract.
01783         When doing Euler sums we may need also 1/4^j
01784         pow: 1 -> 1/2^j
01785             2 -> 1/4^j
01786     */
01787     GETIDENTITY
01788     long s = 1;
01789     int j;
01790 #ifdef WITHCUTOFF
01791     long prec = mpf_get_default_prec();
01792 #endif
01793     mpf_t jm, jjm;
01794     mpf_init(jm); mpf_init(jjm);
01795     if ( pow < 1 || pow > 2 ) {
01796         printf("Wrong parameter pow in SingleTable: %d\n",pow);
01797         exit(-1);
01798     }
01799     if ( m < 0 ) { m = -m; s = -1; }
01800     mpf_set_si(auxsum,0L);
01801     for ( j = N; j > 0; j-- ) {
01802 #ifdef WITHCUTOFF
01803         mpf_set_prec_raw(jm,prec-j);
01804         mpf_set_prec_raw(jjm,prec-j);
01805 #endif
01806         mpf_set_ui(jm,1L);
01807         mpf_div_ui(jjm,jm,(unsigned long)j);
01808         mpf_pow_ui(jm,jjm,(unsigned long)m);
01809         mpf_div_2exp(jjm,jm,pow*(unsigned long)j);
01810         if ( pow == 2 ) mpf_add(auxsum,auxsum,jjm);
01811         else if ( s == -1 && j%2 == 1 ) mpf_sub(auxsum,auxsum,jjm);
01812         else mpf_add(auxsum,auxsum,jjm);
01813     }
01814     /*
01815     And now copy auxsum to tablelement j
01816     */
01817     mpf_set(tabl[j],auxsum);
01818     mpf_clear(jjm); mpf_clear(jm);
01819 }
01820
01821 /*
01822 #] SingleTable :
01823 #[ DoubleTable :
01824 */
01825
01826 void DoubleTable(mpf_t *tabout, mpf_t *tabin, int N, int m, int pow)
01827 {
01828     GETIDENTITY
01829     long s = 1;
01830 #ifdef WITHCUTOFF
01831     long prec = mpf_get_default_prec();
01832 #endif
01833     int j;
01834     mpf_t jm, jjm;
01835     mpf_init(jm); mpf_init(jjm);
01836     if ( pow < -1 || pow > 2 ) {
01837         printf("Wrong parameter pow in SingleTable: %d\n",pow);
01838         exit(-1);
01839     }
01840     if ( m < 0 ) { m = -m; s = -1; }
01841     mpf_set_ui(auxsum,0L);
01842     for ( j = N; j > 0; j-- ) {
01843 #ifdef WITHCUTOFF
01844         mpf_set_prec_raw(jm,prec-j);
01845         mpf_set_prec_raw(jjm,prec-j);
01846 #endif
01847         mpf_set_ui(jm,1L);
01848         mpf_div_ui(jjm,jm,(unsigned long)j);
01849         mpf_pow_ui(jm,jjm,(unsigned long)m);
01850         if ( pow == -1 ) {
01851             mpf_mul_2exp(jjm,jm,(unsigned long)j);
01852             mpf_mul(jm,jjm,tabin[j+1]);
01853         }
01854         else if ( pow > 0 ) {
01855             mpf_div_2exp(jjm,jm,pow*(unsigned long)j);
01856             mpf_mul(jm,jjm,tabin[j+1]);
01857         }
01858         else {

```

```

01859         mpf_mul(jm, jm, tabin[j+1]);
01860     }
01861     if ( pow == 2 ) mpf_add(auxsum, auxsum, jm);
01862     else if ( s == -1 && j%2 == 1 ) mpf_sub(auxsum, auxsum, jm);
01863     else mpf_add(auxsum, auxsum, jm);
01864 /*
01865     And now copy auxsum to tablelement j
01866 */
01867     mpf_set(tabout[j], auxsum);
01868 }
01869 mpf_clear(jjm); mpf_clear(jm);
01870 }
01871
01872 /*
01873     #] DoubleTable :
01874     #[ EndTable :
01875 */
01876
01877 void EndTable(mpf_t sum, mpf_t *tabin, int N, int m, int pow)
01878 {
01879     long s = 1;
01880 #ifdef WITHCUTOFF
01881     long prec = mpf_get_default_prec();
01882 #endif
01883     int j;
01884     mpf_t jm, jjm;
01885     mpf_init(jm); mpf_init(jjm);
01886     if ( pow < -1 || pow > 2 ) {
01887         printf("Wrong parameter pow in SingleTable: %d\n", pow);
01888         exit(-1);
01889     }
01890     if ( m < 0 ) { m = -m; s = -1; }
01891     mpf_set_si(sum, 0L);
01892     for ( j = N; j > 0; j-- ) {
01893 #ifdef WITHCUTOFF
01894         mpf_set_prec_raw(jm, prec-j);
01895         mpf_set_prec_raw(jjm, prec-j);
01896 #endif
01897         mpf_set_ui(jm, 1L);
01898         mpf_div_ui(jjm, jm, (unsigned long)j);
01899         mpf_pow_ui(jm, jjm, (unsigned long)m);
01900         if ( pow == -1 ) {
01901             mpf_mul_2exp(jjm, jm, (unsigned long)j);
01902             mpf_mul(jm, jjm, tabin[j+1]);
01903         }
01904         else if ( pow > 0 ) {
01905             mpf_div_2exp(jjm, jm, pow*(unsigned long)j);
01906             mpf_mul(jm, jjm, tabin[j+1]);
01907         }
01908         else {
01909             mpf_mul(jm, jm, tabin[j+1]);
01910         }
01911         if ( s == -1 && j%2 == 1 ) mpf_sub(sum, sum, jm);
01912         else mpf_add(sum, sum, jm);
01913     }
01914     mpf_clear(jjm); mpf_clear(jm);
01915 }
01916
01917 /*
01918     #] EndTable :
01919     #[ deltaMZV :
01920 */
01921
01922 void deltaMZV(mpf_t result, int *indexes, int depth)
01923 {
01924     GETIDENTITY
01925 /*
01926     Because all sums go roughly like 1/2^j we need about depth steps.
01927 */
01928 /* MesPrint("deltaMZV(%a)", depth, indexes); */
01929     if ( depth == 1 ) {
01930         if ( indexes[0] == 1 ) {
01931             mpf_set(result, mpfdelta1);
01932             return;
01933         }
01934         SimpleDelta(result, indexes[0]);
01935     }
01936     else if ( depth == 2 ) {
01937         if ( indexes[0] == 1 && indexes[1] == 1 ) {
01938             mpf_pow_ui(result, mpfdelta1, 2);
01939             mpf_div_2exp(result, result, 1);
01940         }
01941         else {
01942             SingleTable(mpf_tabl, AC.DefaultPrecision-AC.MaxWeight+1, indexes[0], 1);
01943             EndTable(result, mpftabl, AC.DefaultPrecision-AC.MaxWeight, indexes[1], 0);
01944         }
01945     }

```

```

01946     else if ( depth > 2 ) {
01947         mpf_t *mpftab3;
01948         int d;
01949         /*
01950          Check first whether this is a power of delta1 = ln(2)
01951          */
01952         for ( d = 0; d < depth; d++ ) {
01953             if ( indexes[d] != 1 ) break;
01954         }
01955         if ( d == depth ) { /* divide by fac_(depth) */
01956             mpf_pow_ui(result,mpfdelta1,depth);
01957             for ( d = 2; d <= depth; d++ ) {
01958                 mpf_div_ui(result,result,(unsigned long)d);
01959             }
01960         }
01961         else {
01962             d = depth-1;
01963             SingleTable(mpftab1,AC.DefaultPrecision-AC.MaxWeight+d,*indexes,1);
01964             d--; indexes++;
01965             for(;;) {
01966                 DoubleTable(mpftab2,mpftab1,AC.DefaultPrecision-AC.MaxWeight+d,*indexes,0);
01967                 d--; indexes++;
01968                 if ( d == 0 ) {
01969                     EndTable(result,mpftab2,AC.DefaultPrecision-AC.MaxWeight,*indexes,0);
01970                     break;
01971                 }
01972                 mpftab3 = (mpf_t *)AT.mpf_tab1; AT.mpf_tab1 = AT.mpf_tab2;
01973                 AT.mpf_tab2 = (void *)mpftab3;
01974             }
01975         }
01976     }
01977     else if ( depth == 0 ) {
01978         mpf_set_ui(result,1L);
01979     }
01980 }
01981
01982 /*
01983     #] deltaMZV :
01984     #[ deltaEuler :
01985
01986     Regular Euler delta with - signs, but everywhere 1/2^j
01987 */
01988
01989 void deltaEuler(mpf_t result, int *indexes, int depth)
01990 {
01991     GETIDENTITY
01992     int m;
01993     #ifdef DEBUG
01994     int i;
01995     printf(" deltaEuler(");
01996     for ( i = 0; i < depth; i++ ) {
01997         if ( i != 0 ) printf(",");
01998         printf("%d",indexes[i]);
01999     }
02000     printf(") = ";
02001     #endif
02002     if ( depth == 1 ) {
02003         SimpleDelta(result,indexes[0]);
02004     }
02005     else if ( depth == 2 ) {
02006         SingleTable(mpftab1,AC.DefaultPrecision-AC.MaxWeight+1,indexes[0],1);
02007         m = indexes[1]; if ( indexes[0] < 0 ) m = -m;
02008         EndTable(result,mpftab1,AC.DefaultPrecision,m,0);
02009     }
02010     else if ( depth > 2 ) {
02011         int d;
02012         mpf_t *mpftab3;
02013         d = depth-1;
02014         SingleTable(mpftab1,AC.DefaultPrecision-AC.MaxWeight+d,*indexes,1);
02015         d--; indexes++;
02016         m = *indexes; if ( indexes[-1] < 0 ) m = -m;
02017         for(;;) {
02018             DoubleTable(mpftab2,mpftab1,AC.DefaultPrecision-AC.MaxWeight+d,m,0);
02019             d--; indexes++;
02020             m = *indexes; if ( indexes[-1] < 0 ) m = -m;
02021             if ( d == 0 ) {
02022                 EndTable(result,mpftab2,AC.DefaultPrecision-AC.MaxWeight,m,0);
02023                 break;
02024             }
02025             mpftab3 = (mpf_t *)AT.mpf_tab1; AT.mpf_tab1 = AT.mpf_tab2;
02026             AT.mpf_tab2 = (void *)mpftab3;
02027         }
02028     }
02029     else if ( depth == 0 ) {
02030         mpf_set_ui(result,1L);
02031     }
02032     #ifdef DEBUG

```

```

02033     gmp_printf("%.Fe\n",40,result);
02034 #endif
02035 }
02036
02037 /*
02038     #] deltaEuler :
02039     #[ deltaEulerC :
02040
02041     Conjugate Euler delta without - signs, but possibly 1/4^j
02042     When there is a - in the string we have 1/4.
02043 */
02044
02045 void deltaEulerC(mpf_t result, int *indexes, int depth)
02046 {
02047     GETIDENTITY
02048     int m;
02049 #ifdef DEBUG
02050     int i;
02051     printf(" deltaEulerC(");
02052     for ( i = 0; i < depth; i++ ) {
02053         if ( i != 0 ) printf(",");
02054         printf("%d",indexes[i]);
02055     }
02056     printf(") = ");
02057 #endif
02058     mpf_set_ui(result,0);
02059     if ( depth == 1 ) {
02060         if ( indexes[0] == 0 ) {
02061             printf("Illegal index in depth=1 deltaEulerC: %d\n",indexes[0]);
02062         }
02063         if ( indexes[0] < 0 ) SimpleDeltaC(result,indexes[0]);
02064         else SimpleDelta(result,indexes[0]);
02065     }
02066     else if ( depth == 2 ) {
02067         int par;
02068         m = indexes[0];
02069         if ( m < 0 ) SingleTable(mpftab1,AC.DefaultPrecision-AC.MaxWeight+depth+1,-m,2);
02070         else SingleTable(mpftab1,AC.DefaultPrecision-AC.MaxWeight+depth+1, m,1);
02071         m = indexes[1];
02072         if ( m < 0 ) { m = -m; par = indexes[0] < 0 ? 0: 1; }
02073         else { par = indexes[0] < 0 ? -1: 0; }
02074         EndTable(result,mpftab1,AC.DefaultPrecision-AC.MaxWeight,m,par);
02075     }
02076     else if ( depth > 2 ) {
02077         int d, par;
02078         mpf_t *mpftab3;
02079         d = depth-1;
02080         m = indexes[0];
02081         ZeroTable(mpftab1,AC.DefaultPrecision+1);
02082         if ( m < 0 ) SingleTable(mpftab1,AC.DefaultPrecision-AC.MaxWeight+d+1,-m,2);
02083         else SingleTable(mpftab1,AC.DefaultPrecision-AC.MaxWeight+d+1, m,1);
02084         d--; indexes++; m = indexes[0];
02085         if ( m < 0 ) { m = -m; par = indexes[-1] < 0 ? 0: 1; }
02086         else { par = indexes[-1] < 0 ? -1: 0; }
02087         for(;;) {
02088             ZeroTable(mpftab2,AC.DefaultPrecision-AC.MaxWeight+d);
02089             DoubleTable(mpftab2,mpftab1,AC.DefaultPrecision-AC.MaxWeight+d,m,par);
02090             d--; indexes++; m = indexes[0];
02091             if ( m < 0 ) { m = -m; par = indexes[-1] < 0 ? 0: 1; }
02092             else { par = indexes[-1] < 0 ? -1: 0; }
02093             if ( d == 0 ) {
02094                 mpf_set_ui(result,0);
02095                 EndTable(result,mpftab2,AC.DefaultPrecision-AC.MaxWeight,m,par);
02096                 break;
02097             }
02098             mpftab3 = (mpf_t *)AT.mpf_tab1; AT.mpf_tab1 = AT.mpf_tab2;
02099             AT.mpf_tab2 = (void *)mpftab3;
02100         }
02101     }
02102     else if ( depth == 0 ) {
02103         mpf_set_ui(result,1L);
02104     }
02105 #ifdef DEBUG
02106     gmp_printf("%.Fe\n",40,result);
02107 #endif
02108 }
02109
02110 /*
02111     #] deltaEulerC :
02112     #[ CalculateMZVhalf :
02113
02114     This routine is mainly for support when large numbers of
02115     MZV's have to be calculated at the same time.
02116 */
02117
02118 void CalculateMZVhalf(mpf_t result, int *indexes, int depth)
02119 {

```



```

02120     int i;
02121     if ( depth < 0 ) {
02122         printf("Illegal depth in CalculateMZVhalf: %d\n",depth);
02123         exit(-1);
02124     }
02125     for ( i = 0; i < depth; i++ ) {
02126         if ( indexes[i] <= 0 ) {
02127             printf("Illegal index[%d] in CalculateMZVhalf: %d\n",i,indexes[i]);
02128             exit(-1);
02129         }
02130     }
02131     deltaMZV(result,indexes,depth);
02132 }
02133
02134 /*
02135     #] CalculateMZVhalf :
02136     #[ CalculateMZV :
02137 */
02138
02139 void CalculateMZV(mpf_t result, int *indexes, int depth)
02140 {
02141     GETIDENTITY
02142     int num1, num2 = 0, i, j = 0;
02143     if ( depth < 0 ) {
02144         printf("Illegal depth in CalculateMZV: %d\n",depth);
02145         exit(-1);
02146     }
02147     if ( indexes[0] == 1 ) {
02148         printf("Divergent MZV in CalculateMZV\n");
02149         exit(-1);
02150     }
02151     /* MesPrint("calculateMZV(%a)",depth,indexes); */
02152     for ( i = 0; i < depth; i++ ) {
02153         if ( indexes[i] <= 0 ) {
02154             printf("Illegal index[%d] in CalculateMZV: %d\n",i,indexes[i]);
02155             exit(-1);
02156         }
02157         AT.indi1[i] = indexes[i];
02158     }
02159     num1 = depth; i = 0;
02160     deltaMZV(result,indexes,depth);
02161     for(;;) {
02162         if ( AT.indi1[0] > 1 ) {
02163             AT.indi1[0] -= 1;
02164             for ( j = num2; j > 0; j-- ) AT.indi2[j] = AT.indi2[j-1];
02165             AT.indi2[0] = 1; num2++;
02166         }
02167         else {
02168             AT.indi2[0] += 1;
02169             for ( j = 1; j < num1; j++ ) AT.indi1[j-1] = AT.indi1[j];
02170             num1--;
02171             if ( num1 == 0 ) break;
02172         }
02173         deltaMZV(aux1,AT.indi1,num1);
02174         deltaMZV(aux2,AT.indi2,num2);
02175         mpf_mul(aux3,aux1,aux2);
02176         mpf_add(result,result,aux3);
02177     }
02178     deltaMZV(aux3,AT.indi2,num2);
02179     mpf_add(result,result,aux3);
02180 }
02181
02182 /*
02183     #] CalculateMZV :
02184     #[ CalculateEuler :
02185
02186     There is a problem of notation here.
02187     Basically there are two notations.
02188     1:  $Z(m_1,m_2,m_3) = \sum_{j_3=1}^{\infty} \text{sign}_{-}(m_3) / j_3^{\text{abs}_{-}(m_3)} * \sum_{j_2=1}^{\infty} \text{sign}_{-}(m_2) / j_2^{\text{abs}_{-}(m_2)} * \sum_{j_1=1}^{\infty} \text{sign}_{-}(m_1) / j_1^{\text{abs}_{-}(m_1)}$ 
02189     See Broadhurst,1996
02190     2:  $L(m_1,m_2,m_3) = \sum_{j_1=1}^{\infty} \text{sign}_{-}(m_1) * \sum_{j_2=1}^{\infty} \text{sign}_{-}(m_2) * \sum_{j_3=1}^{\infty} \text{sign}_{-}(m_3) / j_3^{\text{abs}_{-}(m_3)} / (j_2+j_3)^{\text{abs}_{-}(m_2)} / (j_1+j_2+j_3)^{\text{abs}_{-}(m_1)}$ 
02191     See Borwein, Bradley, Broadhurst and Lisonek, 1999
02192     The L corresponds with the H of the datamine up to  $\text{sign}_{-}(m_1*m_2*m_3)$ 
02193     The algorithm follows 2, but the more common notation is 1.
02194     Hence we start with a conversion.
02195 */
02196
02197 void CalculateEuler(mpf_t result, int *Zindexes, int depth)
02198 {
02199     GETIDENTITY

```

```

02207     int s1, num1, num2, i, j;
02208
02209     int *indexes = (int *) (AT.WorkPointer);
02210     for ( i = 0; i < depth; i++ ) indexes[i] = Zindexes[i];
02211     for ( i = 0; i < depth-1; i++ ) {
02212         if ( Zindexes[i] < 0 ) {
02213             for ( j = i+1; j < depth; j++ ) indexes[j] = -indexes[j];
02214         }
02215     }
02216
02217     if ( depth < 0 ) {
02218         printf("Illegal depth in CalculateEuler: %d\n",depth);
02219         exit(-1);
02220     }
02221     if ( indexes[0] == 1 ) {
02222         printf("Divergent Euler sum in CalculateEuler\n");
02223         exit(-1);
02224     }
02225     for ( i = 0, j = 0; i < depth; i++ ) {
02226         if ( indexes[i] == 0 ) {
02227             printf("Illegal index[%d] in CalculateEuler: %d\n",i,indexes[i]);
02228             exit(-1);
02229         }
02230         if ( indexes[i] < 0 ) j = 1;
02231         AT.indi1[i] = indexes[i];
02232     }
02233     if ( j == 0 ) {
02234         CalculateMZV(result,indexes,depth);
02235         return;
02236     }
02237     num1 = depth; AT.indi2[0] = 0; num2 = 0;
02238     deltaEuler(result,AT.indi1,depth);
02239     s1 = 0;
02240     for(;;) {
02241         s1++;
02242         if ( AT.indi1[0] > 1 ) {
02243             AT.indi1[0] -= 1;
02244             for ( j = num2; j > 0; j-- ) AT.indi2[j] = AT.indi2[j-1];
02245             AT.indi2[0] = 1; num2++;
02246         }
02247         else if ( AT.indi1[0] < -1 ) {
02248             AT.indi1[0] += 1;
02249             for ( j = num2; j > 0; j-- ) AT.indi2[j] = AT.indi2[j-1];
02250             AT.indi2[0] = 1; num2++;
02251         }
02252         else if ( AT.indi1[0] == -1 ) {
02253             for ( j = num2; j > 0; j-- ) AT.indi2[j] = AT.indi2[j-1];
02254             AT.indi2[0] = -1; num2++;
02255             for ( j = 1; j < num1; j++ ) AT.indi1[j-1] = AT.indi1[j];
02256             num1--;
02257         }
02258         else {
02259             AT.indi2[0] = AT.indi2[0] < 0 ? AT.indi2[0]-1 : AT.indi2[0]+1;
02260             for ( j = 1; j < num1; j++ ) AT.indi1[j-1] = AT.indi1[j];
02261             num1--;
02262         }
02263         if ( num1 == 0 ) break;
02264         deltaEuler(aux1,AT.indi1,num1);
02265         deltaEulerC(aux2,AT.indi2,num2);
02266         mpf_mul(aux3,aux1,aux2);
02267         if ( (depth+num1+num2+s1)%2 == 0 ) mpf_add(result,result,aux3);
02268         else mpf_sub(result,result,aux3);
02269     }
02270     deltaEulerC(aux3,AT.indi2,num2);
02271     if ( (depth+num2+s1)%2 == 0 ) mpf_add(result,result,aux3);
02272     else mpf_sub(result,result,aux3);
02273 }
02274
02275 /*
02276     #] CalculateEuler :
02277     #[ ExpandMZV :
02278 */
02279
02280 int ExpandMZV(WORD *term, WORD level)
02281 {
02282     DUMMYUSE(term);
02283     DUMMYUSE(level);
02284     return(0);
02285 }
02286
02287 /*
02288     #] ExpandMZV :
02289     #[ ExpandEuler :
02290 */
02291
02292 int ExpandEuler(WORD *term, WORD level)
02293 {

```

```

02294     DUMMYUSE(term);
02295     DUMMYUSE(level);
02296     return(0);
02297 }
02298
02299 /*
02300     #] ExpandEuler :
02301     #[ EvaluateEuler :
02302
02303     There are various steps here:
02304     1: locate an Euler function.
02305     2: evaluate it into a float.
02306     3: convert the coefficient to a float and multiply.
02307     4: Put the new term together.
02308     5: Restart to see whether there are more Euler functions.
02309     The compiler should check that there is no conflict between
02310     the evaluate command and a potential polyfun or polyratfun.
02311     Yet, when reading in expressions there can be a conflict.
02312     Hence the Normalize routine should check as well.
02313
02314     par == MZV: MZV
02315     par == EULER: Euler
02316     par == MZVHALF: MZV sums in 1/2 rather than 1. -> deltaMZV.
02317     par == ALLMZVFUNCTIONS: all of the above.
02318 */
02319
02320 int EvaluateEuler(PHEAD WORD *term, WORD level, WORD par)
02321 {
02322     WORD *t, *tstop, *tt, *tnext, sumweight, depth, first = 1, *newterm, i;
02323     WORD *oldworkpointer = AT.WorkPointer, nsize, *indexes;
02324     int retval;
02325     tstop = term + *term; tstop -= ABS(tstop[-1]);
02326     if ( AT.WorkPointer < term+*term ) AT.WorkPointer = term + *term;
02327 /*
02328     Step 1. We also need to verify that the argument we find is legal
02329 */
02330     t = term+1;
02331     while ( t < tstop ) {
02332         if ( ( *t == par ) || ( ( par == ALLMZVFUNCTIONS ) && (
02333             *t == MZV || *t == EULER || *t == MZVHALF ) ) ) {
02334             /* all arguments should be small integers */
02335             indexes = AT.WorkPointer;
02336             sumweight = 0; depth = 0;
02337             tnext = t + t[1]; tt = t + FUNHEAD;
02338             while ( tt < tnext ) {
02339                 if ( *tt != -SNUMBER ) goto nextfun;
02340                 if ( tt[1] == 0 ) goto nextfun;
02341                 indexes[depth] = tt[1];
02342                 sumweight += ABS(tt[1]); depth++;
02343                 tt += 2;
02344             }
02345             if ( sumweight > AC.MaxWeight ) {
02346                 MesPrint("Error: Weight of Euler/MZV sum greater than %d",sumweight);
02347                 MesPrint("Please increase MaxWeight in form.set.");
02348                 Terminate(-1);
02349             }
02350 /*
02351             Step 2: evaluate:
02352 */
02353             AT.WorkPointer += depth;
02354             if ( first ) {
02355                 if ( *t == MZV ) CalculateMZV(aux4,indexes,depth);
02356                 else if ( *t == EULER ) CalculateEuler(aux4,indexes,depth);
02357                 else if ( *t == MZVHALF ) CalculateMZVhalf(aux4,indexes,depth);
02358                 first = 0;
02359             }
02360             else {
02361                 if ( *t == MZV ) CalculateMZV(aux5,indexes,depth);
02362                 else if ( *t == EULER ) CalculateEuler(aux5,indexes,depth);
02363                 else if ( *t == MZVHALF ) CalculateMZVhalf(aux5,indexes,depth);
02364                 mpf_mul(aux4,aux4,aux5);
02365             }
02366             *t = 0;
02367         }
02368     nextfun:
02369         t += t[1];
02370     }
02371     if ( first == 1 ) return(Generator(BHEAD term,level));
02372 /*
02373     Step 3:
02374     Now the regular coefficient, if it is not 1/1.
02375     We have two cases: size +- 3, or bigger.
02376 */
02377     nsize = term[*term-1];
02378     if ( nsize < 0 ) {
02379         mpf_neg(aux4,aux4);
02380         nsize = -nsize;

```

```

02381     }
02382     if ( nsize == 3 ) {
02383         if ( tstop[0] != 1 ) {
02384             mpf_mul_ui(aux4,aux4, (ULONG) ((UWORD)tstop[0]));
02385         }
02386         if ( tstop[1] != 1 ) {
02387             mpf_div_ui(aux4,aux4, (ULONG) ((UWORD)tstop[1]));
02388         }
02389     }
02390     else {
02391         RatToFloat(aux5, (UWORD *)tstop, nsize);
02392         mpf_mul(aux4,aux4,aux5);
02393     }
02394     /*
02395     Now we have to locate possible other float_ functions.
02396     */
02397     t = term+1;
02398     while ( t < tstop ) {
02399         if ( *t == FLOATFUN ) {
02400             UnpackFloat(aux5,t);
02401             mpf_mul(aux4,aux4,aux5);
02402         }
02403         t += t[1];
02404     }
02405     /*
02406     Now we should compose the new term in the Workspace.
02407     */
02408     t = term+1;
02409     newterm = AT.WorkPointer;
02410     tt = newterm+1;
02411     while ( t < tstop ) {
02412         if ( *t == 0 || *t == FLOATFUN ) t += t[1];
02413         else {
02414             i = t[1]; NCOPY(tt,t,i);
02415         }
02416     }
02417     PackFloat(tt,aux4);
02418     tt += tt[1];
02419     *tt++ = 1; *tt++ = 1; *tt++ = 3;
02420     *newterm = tt-newterm;
02421     AT.WorkPointer = tt;
02422     retval = Generator(BHEAD newterm,level);
02423     AT.WorkPointer = oldworkpointer;
02424     return(retval);
02425 }
02426
02427 /*
02428     #] EvaluateEuler :
02429     #] MZV :
02430     #[ Functions :
02431     #[ CoEvaluate :
02432
02433     Current subkeys: mzv_, euler_ and sqrt_.
02434     */
02435
02436 int CoEvaluate(UBYTE *s)
02437 {
02438     UBYTE *subkey, c;
02439     WORD numfun, type;
02440     int error = 0;
02441     while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
02442     if ( *s == 0 ) {
02443         /*
02444         No subkey, which means all functions.
02445         */
02446         Add3Com(TYPEEVALUATE,ALLFUNCTIONS);
02447         /*
02448         The MZV, EULER and MZVHALF are done separately
02449         */
02450         Add3Com(TYPEEVALUATE,ALLMZVFUNCTIONS);
02451         return(0);
02452     }
02453     while ( *s ) {
02454         subkey = s;
02455         while ( FG.cTable[*s] == 0 ) s++;
02456         if ( *s == '_' ) s++;
02457         c = *s; *s++ = 0;
02458         /*
02459         We still need provisions for pi_ and possibly other constants.
02460         */
02461         if ( ( ( type = GetName(AC.varnames,subkey,&numfun,NOAUTO) ) != CFUNCTION )
02462             || ( functions[numfun].spec != 0 ) ) {
02463
02464             if ( type == CSYMBOL ) {
02465                 Add4Com(TYPEEVALUATE,SYMBOL,numfun);
02466                 break;
02467             }

```

```

02468 /*
02469         This cannot work.
02470 */
02471     MesPrint("%s should be a built in function that can be evaluated numerically.",s);
02472     error = 1;
02473 }
02474 else {
02475     switch ( numfun+FUNCTION ) {
02476         case MZV:
02477         case EULER:
02478         case MZVHALF:
02479         case SQRTFUNCTION:
02480     /*
02481         The following functions are treated in evaluate.c
02482
02483         case LNFUNCTION:
02484         case SINFUNCTION:
02485         case COSFUNCTION:
02486         case TANFUNCTION:
02487         case ASINFUNCTION:
02488         case ACOSFUNCTION:
02489         case ATANFUNCTION:
02490         case ATAN2FUNCTION:
02491         case SINHFUNCTION:
02492         case COSHFUNCTION:
02493         case TANHFUNCTION:
02494         case ASINHFUNCTION:
02495         case ACOSHFUNCTION:
02496         case ATANHFUNCTION:
02497         case LI2HFUNCTION:
02498         case LINHFUNCTION:
02499         case AGMFUNCTION:
02500         case GAMMAFUN:
02501
02502         At a later stage we can add more functions from mpfr here
02503         mpfr_(number,arg(s))
02504     */
02505         Add3Com(TYPEEVALUATE,numfun+FUNCTION);
02506         break;
02507     default:
02508         MesPrint("%s should be a built in function that can be evaluated numerically.",s);
02509         error = 1;
02510         break;
02511     }
02512 }
02513 *s = c;
02514 while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
02515 }
02516 return(error);
02517 }
02518
02519 /*
02520     #] CoEvaluate :
02521     #] Functions :
02522 */

```

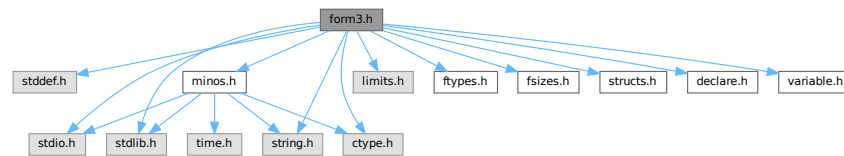
4.45 form3.h File Reference

```

#include <stddef.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <limits.h>
#include "ftypes.h"
#include "fsizes.h"
#include "minos.h"
#include "structs.h"
#include "declare.h"
#include "variable.h"

```

Include dependency graph for form3.h:



Macros

- #define **MAJORVERSION** 5
- #define **MINORVERSION** 0
- #define **PRODUCTIONDATE** "8-nov-2022"
- #define **BETAVERSION**
- #define **inline**
- #define **NDEBUG**
- #define **STATIC_ASSERT**(condition) STATIC_ASSERT__1(condition, __LINE__)
- #define **STATIC_ASSERT__1**(X, L) STATIC_ASSERT__2(X, L)
- #define **STATIC_ASSERT__2**(X, L) STATIC_ASSERT__3(X, L)
- #define **STATIC_ASSERT__3**(X, L) typedef char static_assertion_failed_##L[(!!(X))*2-1]
- #define **TOPBITONLY** ((ULONG)1 << (BITSINWORD - 1)) /* 0x00008000UL */
- #define **TOPLONGBITONLY** ((ULONG)1 << (BITSINLONG - 1)) /* 0x80000000UL */
- #define **SPECMASK** ((UWORD)1 << (BITSINWORD - 1)) /* 0x8000U */
- #define **WILDMASK** ((UWORD)1 << (BITSINWORD - 2)) /* 0x4000U */
- #define **WORDMASK** ((ULONG)FULLMAX - 1) /* 0x0000FFFFUL */
- #define **AWORDMASK** (WORDMASK << BITSINWORD) /* 0xFFFF0000UL */
- #define **FULLMAX** ((LONG)1 << BITSINWORD) /* 0x00010000L */
- #define **MAXPOSITIVE** ((LONG)(TOPBITONLY - 1)) /* 0x00007FFFL */
- #define **MAXLONG** ((LONG)(TOPLONGBITONLY - 1)) /* 0x7FFFFFFFL */
- #define **MAXPOSITIVE2** (MAXPOSITIVE / 2) /* 0x00003FFFL */
- #define **MAXPOSITIVE4** (MAXPOSITIVE / 4) /* 0x00001FFFL */
- #define **form_alignof**(type) offsetof(struct { char c_; type x_; }, x_)
- #define **PADDUMMY**(type, size) UBYTE d_u_m_m_y[form_alignof(type) - ((size) & (form_alignof(type) - 1))]
- #define **PADPOSITION**(ptr_, long_, int_, word_, byte_)
- #define **PADPOINTER**(long_, int_, word_, byte_)
- #define **PADLONG**(int_, word_, byte_)
- #define **PADINT**(word_, byte_)
- #define **PADWORD**(byte_)
- #define **WITHSORTBOTS**
- #define **FILES** FILE
- #define **Uopen**(x, y) fopen(x, y)
- #define **Uflush**(x) fflush(x)
- #define **Uclose**(x) fclose(x)
- #define **Uread**(x, y, z, u) fread(x, y, z, u)
- #define **Uwrite**(x, y, z, u) fwrite(x, y, z, u)
- #define **Usetbuf**(x, y) setbuf(x, y)
- #define **Useek**(x, y, z) fseek(x, y, z)
- #define **Utell**(x) ftell(x)
- #define **Ugetpos**(x, y) fgetpos(x, y)
- #define **Usetpos**(x, y) fsetpos(x, y)
- #define **Usync**(x) fflush(x)

- `#define Utruncate(x) _chsize(_fileno(x),0)`
- `#define Ustdout stdout`
- `#define MAX_OPEN_FILES FOPEN_MAX`
- `#define bzero(b, len) (memset((b), 0, (len)), (void)0)`
- `#define GetPID() ((LONG)GetCurrentProcessId())`

Typedefs

- `typedef signed char SBYTE`
- `typedef unsigned char UBYTE`
- `typedef unsigned int UINT`
- `typedef ULONG RLONG`
- `typedef INT64 MLONG`
- `typedef char BOOL`

Functions

- `STATIC_ASSERT (sizeof(WORD) *8==BITSINWORD)`
- `STATIC_ASSERT (sizeof(LONG) *8==BITSINLONG)`
- `STATIC_ASSERT (sizeof(LONG) >=sizeof(int *))`
- `STATIC_ASSERT (sizeof(INT16)==2)`
- `STATIC_ASSERT (sizeof(INT32)==4)`
- `STATIC_ASSERT (sizeof(INT64)==8)`

4.45.1 Detailed Description

Contains critical defines for the compilation process Also contains the inclusion of all necessary header files. There are also some system dependencies concerning file functions.

Definition in file [form3.h](#).

4.45.2 Macro Definition Documentation

4.45.2.1 MAJORVERSION

```
#define MAJORVERSION 5
```

Definition at line 47 of file [form3.h](#).

4.45.2.2 MINORVERSION

```
#define MINORVERSION 0
```

Definition at line 48 of file [form3.h](#).

4.45.2.3 PRODUCTIONDATE

```
#define PRODUCTIONDATE "8-nov-2022"
```

Definition at line 53 of file [form3.h](#).

4.45.2.4 BETAVERSION

```
#define BETAVERSION
```

Definition at line 57 of file [form3.h](#).

4.45.2.5 inline

```
#define inline
```

Definition at line 134 of file [form3.h](#).

4.45.2.6 NDEBUG

```
#define NDEBUG
```

Definition at line 161 of file [form3.h](#).

4.45.2.7 STATIC_ASSERT

```
#define STATIC_ASSERT(  
    condition ) STATIC_ASSERT__1(condition, __LINE__)
```

Definition at line 172 of file [form3.h](#).

4.45.2.8 STATIC_ASSERT__1

```
#define STATIC_ASSERT__1(  
    X,  
    L ) STATIC_ASSERT__2(X, L)
```

Definition at line 173 of file [form3.h](#).

4.45.2.9 STATIC_ASSERT__2

```
#define STATIC_ASSERT__2(  
    X,  
    L ) STATIC_ASSERT__3(X, L)
```

Definition at line 174 of file [form3.h](#).

4.45.2.10 STATIC_ASSERT__3

```
#define STATIC_ASSERT__3(  
    X,  
    L )   typedef char static_assertion_failed_##L[ (!! (X)) *2-1]
```

Definition at line 175 of file [form3.h](#).

4.45.2.11 TOPBITONLY

```
#define TOPBITONLY ((ULONG)1 << (BITSINWORD - 1)) /* 0x00008000UL */
```

Definition at line 310 of file [form3.h](#).

4.45.2.12 TOPLONGBITONLY

```
#define TOPLONGBITONLY ((ULONG)1 << (BITSINLONG - 1)) /* 0x80000000UL */
```

Definition at line 311 of file [form3.h](#).

4.45.2.13 SPECMASK

```
#define SPECMASK ((UWORD)1 << (BITSINWORD - 1)) /* 0x8000U */
```

Definition at line 312 of file [form3.h](#).

4.45.2.14 WILDMASK

```
#define WILDMASK ((UWORD)1 << (BITSINWORD - 2)) /* 0x4000U */
```

Definition at line 313 of file [form3.h](#).

4.45.2.15 WORDMASK

```
#define WORDMASK ((ULONG)FULLMAX - 1) /* 0x0000FFFFUL */
```

Definition at line 314 of file [form3.h](#).

4.45.2.16 AWORDMASK

```
#define AWORDMASK (WORDMASK << BITSINWORD) /* 0xFFFF0000UL */
```

Definition at line 315 of file [form3.h](#).

4.45.2.17 FULLMAX

```
#define FULLMAX ((LONG)1 << BITSINWORD) /* 0x00010000L */
```

Definition at line 316 of file [form3.h](#).

4.45.2.18 MAXPOSITIVE

```
#define MAXPOSITIVE ((LONG)(TOPBITONLY - 1)) /* 0x00007FFFL */
```

Definition at line 317 of file [form3.h](#).

4.45.2.19 MAXLONG

```
#define MAXLONG ((LONG)(TOPLONGBITONLY - 1)) /* 0x7FFFFFFFL */
```

Definition at line 318 of file [form3.h](#).

4.45.2.20 MAXPOSITIVE2

```
#define MAXPOSITIVE2 (MAXPOSITIVE / 2) /* 0x00003FFFL */
```

Definition at line 319 of file [form3.h](#).

4.45.2.21 MAXPOSITIVE4

```
#define MAXPOSITIVE4 (MAXPOSITIVE / 4) /* 0x00001FFFL */
```

Definition at line 320 of file [form3.h](#).

4.45.2.22 form_alignof

```
#define form_alignof(  
    type ) offsetof(struct { char c_; type x_; }, x_)
```

Definition at line 336 of file [form3.h](#).

4.45.2.23 PADDUMMY

```
#define PADDUMMY(  
    type,  
    size ) UBYTE d_u_m_m_y[form_alignof(type) - ((size) & (form_alignof(type) -  
1)]]
```

Definition at line 391 of file [form3.h](#).

4.45.2.24 PADPOSITION

```
#define PADPOSITION(  
    ptr_,  
    long_,  
    int_,  
    word_,  
    byte_ )
```

Value:

```
PADDUMMY(off_t, \  
+ sizeof(int *) * (ptr_) \  
+ sizeof(LONG) * (long_) \  
+ sizeof(int) * (int_) \  
+ sizeof(WORD) * (word_) \  
+ sizeof(UBYTE) * (byte_) \  
)
```

Definition at line 393 of file [form3.h](#).

4.45.2.25 PADPOINTER

```
#define PADPOINTER(  
    long_,  
    int_,  
    word_,  
    byte_ )
```

Value:

```
PADDUMMY(int *, \  
+ sizeof(LONG) * (long_) \  
+ sizeof(int) * (int_) \  
+ sizeof(WORD) * (word_) \  
+ sizeof(UBYTE) * (byte_) \  
)
```

Definition at line 401 of file [form3.h](#).

4.45.2.26 PADLONG

```
#define PADLONG(  
    int_,  
    word_,  
    byte_ )
```

Value:

```
PADDUMMY(LONG, \  
+ sizeof(int) * (int_) \  
+ sizeof(WORD) * (word_) \  
+ sizeof(UBYTE) * (byte_) \  
)
```

Definition at line 408 of file [form3.h](#).

4.45.2.27 PADINT

```
#define PADINT(  
    word_,  
    byte_ )
```

Value:

```
PADDUMMY(int, \  
    + sizeof(WORD) * (word_) \  
    + sizeof(UBYTE) * (byte_) \  
    )
```

Definition at line 414 of file [form3.h](#).

4.45.2.28 PADWORD

```
#define PADWORD(  
    byte_ )
```

Value:

```
PADDUMMY(WORD, \  
    + sizeof(UBYTE) * (byte_) \  
    )
```

Definition at line 419 of file [form3.h](#).

4.45.2.29 WITHSORTBOTS

```
#define WITHSORTBOTS
```

Definition at line 429 of file [form3.h](#).

4.45.2.30 FILES

```
#define FILES FILE
```

Definition at line 511 of file [form3.h](#).

4.45.2.31 Uopen

```
#define Uopen(  
    x,  
    y ) fopen(x,y)
```

Definition at line 512 of file [form3.h](#).

4.45.2.32 Uflush

```
#define Uflush(  
    x ) fflush(x)
```

Definition at line 513 of file [form3.h](#).

4.45.2.33 Uclose

```
#define Uclose(  
    x ) fclose(x)
```

Definition at line 514 of file [form3.h](#).

4.45.2.34 Uread

```
#define Uread(  
    x,  
    y,  
    z,  
    u ) fread(x,y,z,u)
```

Definition at line 515 of file [form3.h](#).

4.45.2.35 Uwrite

```
#define Uwrite(  
    x,  
    y,  
    z,  
    u ) fwrite(x,y,z,u)
```

Definition at line 516 of file [form3.h](#).

4.45.2.36 Usetbuf

```
#define Usetbuf(  
    x,  
    y ) setbuf(x,y)
```

Definition at line 517 of file [form3.h](#).

4.45.2.37 Useek

```
#define Useek(  
    x,  
    y,  
    z ) fseek(x,y,z)
```

Definition at line 518 of file [form3.h](#).

4.45.2.38 Utell

```
#define Utell(  
    x ) ftell(x)
```

Definition at line 519 of file [form3.h](#).

4.45.2.39 Ugetpos

```
#define Ugetpos(  
    x,  
    y ) fgetpos(x,y)
```

Definition at line 520 of file [form3.h](#).

4.45.2.40 Usetpos

```
#define Usetpos(  
    x,  
    y ) fsetpos(x,y)
```

Definition at line 521 of file [form3.h](#).

4.45.2.41 Usync

```
#define Usync(  
    x ) fflush(x)
```

Definition at line 522 of file [form3.h](#).

4.45.2.42 Utruncate

```
#define Utruncate(  
    x ) _chsize(_fileno(x),0)
```

Definition at line 523 of file [form3.h](#).

4.45.2.43 Ustdout

```
#define Ustdout stdout
```

Definition at line 524 of file [form3.h](#).

4.45.2.44 MAX_OPEN_FILES

```
#define MAX_OPEN_FILES FOPEN_MAX
```

Definition at line 525 of file [form3.h](#).

4.45.2.45 bzero

```
#define bzero(  
    b,  
    len ) (memset((b), 0, (len)), (void)0)
```

Definition at line 526 of file [form3.h](#).

4.45.2.46 GetPID

```
#define GetPID( ) ((LONG)GetCurrentProcessId())
```

Definition at line 527 of file [form3.h](#).

4.45.3 Typedef Documentation

4.45.3.1 SBYTE

```
typedef signed char SBYTE
```

Definition at line 299 of file [form3.h](#).

4.45.3.2 UBYTE

```
typedef unsigned char UBYTE
```

Definition at line 300 of file [form3.h](#).

4.45.3.3 UINT

```
typedef unsigned int UINT
```

Definition at line 301 of file [form3.h](#).

4.45.3.4 RLONG

```
typedef ULONG RLONG
```

Definition at line 302 of file [form3.h](#).

4.45.3.5 MLONG

```
typedef INT64 MLONG
```

Definition at line 303 of file [form3.h](#).

4.45.3.6 BOOL

```
typedef char BOOL
```

Definition at line 308 of file [form3.h](#).

4.46 form3.h

[Go to the documentation of this file.](#)

```

00001
00008 /* #[ License : */
00009 /*
00010 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00011 *   When using this file you are requested to refer to the publication
00012 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00013 *   This is considered a matter of courtesy as the development was paid
00014 *   for by FOM the Dutch physics granting agency and we would like to
00015 *   be able to track its scientific use to convince FOM of its value
00016 *   for the community.
00017 *
00018 *   This file is part of FORM.
00019 *
00020 *   FORM is free software: you can redistribute it and/or modify it under the
00021 *   terms of the GNU General Public License as published by the Free Software
00022 *   Foundation, either version 3 of the License, or (at your option) any later
00023 *   version.
00024 *
00025 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00026 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00027 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00028 *   details.
00029 *
00030 *   You should have received a copy of the GNU General Public License along
00031 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00032 */
00033 /* #] License : */
00034
00035 #ifndef __FORM3H__
00036 #define __FORM3H__
00037
00038 #ifdef HAVE_CONFIG_H
00039
00040 #ifndef CONFIG_H_INCLUDED
00041 #define CONFIG_H_INCLUDED
00042 #include <config.h>
00043 #endif
00044
00045 #else /* HAVE_CONFIG_H */
00046
00047 #define MAJORVERSION 5
00048 #define MINORVERSION 0
00049
00050 #ifdef __DATE__
00051 #define PRODUCTIONDATE __DATE__
00052 #else
00053 #define PRODUCTIONDATE "8-nov-2022"
00054 #endif
00055
00056 /*#undef BETAVERSION */
00057 #define BETAVERSION
00058
00059 #ifdef LINUX32
00060 #define UNIX
00061 #define LINUX
00062 #define LLP32
00063 #define SIZEOF_LONG_LONG 8
00064 #define _FILE_OFFSET_BITS 64
00065 #define WITHZLIB
00066 #define WITHGMP
00067 #define WITHPOSIXCLOCK
00068 #endif
00069
00070 #ifdef LINUX64
00071 #define UNIX
00072 #define LINUX
00073 #define LP64
00074 #define WITHZLIB
00075 #define WITHGMP
00076 #define WITHPOSIXCLOCK
00077 #define WITHFLOAT
00078 #endif
00079
00080 #ifdef APPLE32
00081 #define UNIX
00082 #define LLP32
00083 #define SIZEOF_LONG_LONG 8
00084 #define _FILE_OFFSET_BITS 64
00085 #define WITHZLIB
00086 #endif
00087
00088 #ifdef APPLE64

```



```

00089 #define UNIX
00090 #define LP64
00091 #define WITHZLIB
00092 #define WITHGMP
00093 #define WITHPOSIXCLOCK
00094 #define WITHFLOAT
00095 #define HAVE_UNORDERED_MAP
00096 #define HAVE_UNORDERED_SET
00097 #endif
00098
00099 #ifdef CYGWIN32
00100 #define UNIX
00101 #define ILP32
00102 #define SIZEOF_LONG_LONG 8
00103 #endif
00104
00105 #ifdef _MSC_VER
00106 #define WINDOWS
00107 #define _CRT_SECURE_NO_WARNINGS
00108 #if defined(_WIN64)
00109 #define LLP64
00110 #elif defined(_WIN32)
00111 #define ILP32
00112 #define SIZEOF_LONG_LONG 8
00113 #endif
00114 #endif
00115
00116 /*
00117  * We must not define WITHPOSIXCLOCK in compiling the sequential FORM or ParFORM.
00118  */
00119 #if !defined(WITHPTHREADS) && defined(WITHPOSIXCLOCK)
00120 #undef WITHPOSIXCLOCK
00121 #endif
00122
00123 #if !defined(__cplusplus) && !defined(inline)
00124 #if (defined(__STDC_VERSION__) && __STDC_VERSION__ >= 199901L) || (defined(__GNUC__) &&
    !defined(__STRICT_ANSI__))
00125 /* "inline" is available. */
00126 #elif defined(__GNUC__)
00127 /* GNU C compiler has "__inline__". */
00128 #define inline __inline__
00129 #elif defined(_MSC_VER)
00130 /* Microsoft C compiler has "__inline". */
00131 #define inline __inline
00132 #else
00133 /* Inline functions may be not supported. Define "inline" to be empty. */
00134 #define inline
00135 #endif
00136 #endif
00137
00138 #endif /* HAVE_CONFIG_H */
00139
00140 /* Workaround for MSVC. */
00141 #if defined(_MSC_VER)
00142 /*
00143  * Old versions of MSVC didn't support C99 function `snprintf`, which is used
00144  * in poly.cc. On the other hand, macroizing `snprintf` gives a fatal error
00145  * with MSVC >= 2015.
00146  */
00147 #if _MSC_VER < 1900
00148 #define snprintf _snprintf
00149 #endif
00150 #endif
00151
00152 /*
00153  * Translate our dialect "DEBUGGING" to the standard "NDEBUG".
00154  */
00155 #ifdef DEBUGGING
00156 #ifdef NDEBUG
00157 #undef NDEBUG
00158 #endif
00159 #else
00160 #ifndef NDEBUG
00161 #define NDEBUG
00162 #endif
00163 #endif
00164
00165 /*
00166  * STATIC_ASSERT(condition) will fail to be compiled if the given
00167  * condition is false.
00168  */
00169 /*
00170 #define STATIC_ASSERT(condition)
00171 */
00172 #define STATIC_ASSERT(condition) STATIC_ASSERT__1(condition, __LINE__)
00173 #define STATIC_ASSERT__1(X, L) STATIC_ASSERT__2(X, L)
00174 #define STATIC_ASSERT__2(X, L) STATIC_ASSERT__3(X, L)

```

```

00175 #define STATIC_ASSERT__3(X,L) \
00176     typedef char static_assertion_failed_##L[(!!(X))*2-1]
00177
00178 /*
00179  * UNIX or WINDOWS must be defined.
00180  */
00181 #if defined(UNIX)
00182 #define mBSD
00183 #define ANSI
00184 #elif defined(WINDOWS)
00185 #define ANSI
00186 #define WIN32_LEAN_AND_MEAN
00187 #include <windows.h>
00188 #include <io.h>
00189 #include <fcntl.h>
00190 /* Undefine/rename conflicted symbols. */
00191 #undef MAXLONG /* WinNT.h */
00192 #define WORD FORM_WORD /* WinDef.h */
00193 #define LONG FORM_LONG /* WinNT.h */
00194 #define ULONG FORM_ULONG /* WinDef.h */
00195 #define BOOL FORM_BOOL /* WinDef.h */
00196 #undef CreateFile /* WinBase.h */
00197 #undef CopyFile /* WinBase.h */
00198 #define OpenFile FORM_OpenFile /* WinBase.h */
00199 #define ReOpenFile FORM_ReOpenFile /* WinBase.h */
00200 #define ReadFile FORM_ReadFile /* WinBase.h */
00201 #define WriteFile FORM_WriteFile /* WinBase.h */
00202 #define DeleteObject FORM_DeleteObject /* WinGDI.h */
00203 #else
00204 #error UNIX or WINDOWS must be defined!
00205 #endif
00206
00207 /*
00208  * Data model. ILP32 or LLP64 or LP64 must be defined.
00209  *
00210  * Here we define basic types WORD, LONG and their unsigned versions
00211  * UWORD and ULONG. LONG must be double size of WORD. Their actual types
00212  * are system-dependent. BITSINWORD and BITSINLONG are also defined.
00213  * INT16, INT32 (also INT64 and INT128 if available) are used for
00214  * system independent saved expressions (store.c).
00215  */
00216 #if defined(ILP32)
00217
00218 typedef short WORD;
00219 typedef long LONG;
00220 typedef unsigned short UWORD;
00221 typedef unsigned long ULONG;
00222 #define BITSINWORD 16
00223 #define BITSINLONG 32
00224 #define INT16 short
00225 #define INT32 int
00226 #undef INT64
00227 #undef INT128
00228
00229 #ifdef SIZEOF_LONG_LONG
00230 #if SIZEOF_LONG_LONG == 8
00231 #define INT64 long long
00232 #endif
00233 #endif
00234
00235 #ifndef INT64
00236 #error INT64 is not available!
00237 #endif
00238
00239 #define WORD_MIN_VALUE SHRT_MIN
00240 #define WORD_MAX_VALUE SHRT_MAX
00241 #define LONG_MIN_VALUE LONG_MIN
00242 #define LONG_MAX_VALUE LONG_MAX
00243
00244 #elif defined(LLP64)
00245
00246 typedef int WORD;
00247 typedef long long LONG;
00248 typedef unsigned int UWORD;
00249 typedef unsigned long long ULONG;
00250 #define BITSINWORD 32
00251 #define BITSINLONG 64
00252 #define INT16 short
00253 #define INT32 int
00254 #define INT64 long long
00255 #undef INT128
00256
00257 #define WORD_MIN_VALUE INT_MIN
00258 #define WORD_MAX_VALUE INT_MAX
00259 #define LONG_MIN_VALUE LLONG_MIN
00260 #define LONG_MAX_VALUE LLONG_MAX
00261

```

```

00262 #elif defined(LP64)
00263
00264 typedef int WORD;
00265 typedef long LONG;
00266 typedef unsigned int UWORD;
00267 typedef unsigned long ULONG;
00268 #define BITSINWORD 32
00269 #define BITSINLONG 64
00270 #define INT16 short
00271 #define INT32 int
00272 #define INT64 long
00273 #undef INT128
00274
00275 #define WORD_MIN_VALUE INT_MIN
00276 #define WORD_MAX_VALUE INT_MAX
00277 #define LONG_MIN_VALUE LONG_MIN
00278 #define LONG_MAX_VALUE LONG_MAX
00279
00280 #else
00281 #error ILP32 or LLP64 or LP64 must be defined!
00282 #endif
00283
00284 STATIC_ASSERT(sizeof(WORD) * 8 == BITSINWORD);
00285 STATIC_ASSERT(sizeof(LONG) * 8 == BITSINLONG);
00286 STATIC_ASSERT(sizeof(WORD) * 2 == sizeof(LONG));
00287 STATIC_ASSERT(sizeof(LONG) >= sizeof(int *));
00288 STATIC_ASSERT(sizeof(INT16) == 2);
00289 STATIC_ASSERT(sizeof(INT32) == 4);
00290 STATIC_ASSERT(sizeof(INT64) == 8);
00291 #ifdef INT128
00292 STATIC_ASSERT(sizeof(INT128) == 16);
00293 #endif
00294
00295 #if BITSINWORD == 32
00296 #define WORDSIZE32 1
00297 #endif
00298
00299 typedef signed char SBYTE;
00300 typedef unsigned char UBYTE;
00301 typedef unsigned int UINT;
00302 typedef ULONG RLONG; /* Used in reken.c. */
00303 typedef INT64 MLONG; /* See commentary in minos.h. */
00304 /*
00305  * NOTE: we don't use the standard _Bool (or C++ bool) because its size is
00306  * implementation-dependent and messes up the traditional PADXXX macros.
00307  */
00308 typedef char BOOL;
00309
00310 #define TOPBITONLY ((ULONG)1 < (BITSINWORD - 1)) /* E.g. in 32-bits */
00311 #define TOPLONGBITONLY ((ULONG)1 < (BITSINLONG - 1)) /* 0x00008000UL */
00312 #define SPECMASK ((UWORD)1 < (BITSINWORD - 1)) /* 0x80000000UL */
00313 #define WILDMASK ((UWORD)1 < (BITSINWORD - 2)) /* 0x4000U */
00314 #define WORDMASK ((ULONG)FULLMAX - 1) /* 0x0000FFFFUL */
00315 #define AWORDMASK (WORDMASK < BITSINWORD) /* 0xFFFF0000UL */
00316 #define FULLMAX ((LONG)1 < BITSINWORD) /* 0x00010000L */
00317 #define MAXPOSITIVE ((LONG)(TOPBITONLY - 1)) /* 0x00007FFF */
00318 #define MAXLONG ((LONG)(TOPLONGBITONLY - 1)) /* 0x7FFFFFFFL */
00319 #define MAXPOSITIVE2 (MAXPOSITIVE / 2) /* 0x00003FFF */
00320 #define MAXPOSITIVE4 (MAXPOSITIVE / 4) /* 0x00001FFF */
00321
00322 /*
00323  * form_alignof(type) returns the number of bytes used in the alignment of
00324  * the type.
00325  */
00326 #if !defined(form_alignof)
00327 #if defined(__GNUC__)
00328 /* GNU C compiler has "__alignof__". */
00329 #define form_alignof(type) __alignof__(type)
00330 #elif defined(_MSC_VER)
00331 /* Microsoft C compiler has "__alignof". */
00332 #define form_alignof(type) __alignof__(type)
00333 #elif !defined(__cplusplus)
00334 /* Generic case in C. */
00335 #include <stddef.h>
00336 #define form_alignof(type) offsetof(struct { char c_; type x_; }, x_)
00337 #else
00338 /* Generic case in C++, at least works with a POD struct. */
00339 #include <cstddef>
00340 namespace alignof_impl_ {
00341 template<typename T> struct calc {
00342     struct X { char c_; T x_; };
00343     enum { value = offsetof(X, x_) };
00344 };
00345 }
00346 #define form_alignof(type) alignof_impl_::calc<type>::value
00347 #endif
00348 #endif

```

```

00349
00350 /*
00351  * Macros to be inserted at the end of a structure to align the whole structure.
00352  *
00353  * In the currently available systems,
00354  *   sizeof(POSITION) >= sizeof(pointers) == sizeof(LONG) >= sizeof(int)
00355  *   >= sizeof(WORD) >= sizeof(UBYTE) = 1.
00356  * (POSITION is defined in struct.h and contains only an off_t variable.)
00357  * Thus, if we put members of a structure in this order and use those macros,
00358  * then we can align the data without relying on extra paddings added by
00359  * the compiler. For example,
00360  *   typedef struct {
00361  *       int *a;
00362  *       LONG b;
00363  *       WORD c[2];
00364  *       UBYTE d;
00365  *       PADPOINTER(1,0,2,1);
00366  *   } A;
00367  *   typedef struct {
00368  *       POSITION p;
00369  *       A a; // aligned same as pointers
00370  *       int *b;
00371  *       LONG c;
00372  *       UBYTE d;
00373  *       PADPOSITION(1,1,0,0,1+sizeof(A));
00374  *   } B;
00375  * The cost for the use of those PADXXX macros is a padding (>= 1 byte) will
00376  * be always inserted even in the case that no padding is actually needed.
00377  *
00378  * Numbers for the arguments have to be calculated manually and so very
00379  * error-prone. Be careful!
00380  *
00381  * Note that there is a 32-bit system in which off_t is aligned on 8-byte
00382  * boundary, (e.g., Cygwin with large file support), but still the above
00383  * inequalities are satisfied.
00384  *
00385  * The legendary story of these macros--they fixed some problems in ancient
00386  * times when compilers were unreliable and didn't know how to correctly compute
00387  * structure paddings--has been handed down, though nowadays there are only
00388  * disadvantages for them in practice (ancient compilers most likely can't
00389  * compile C99 and C++98+TR1 sources anyway).
00390  */
00391 #define PADDUMMY(type, size) \
00392     UBYTE d_u_m_m_y[form_alignof(type) - ((size) & (form_alignof(type) - 1))]
00393 #define PADPOSITION(ptr_, long_, int_, word_, byte_) \
00394     PADDUMMY(off_t, \
00395         + sizeof(int *) * (ptr_) \
00396         + sizeof(LONG) * (long_) \
00397         + sizeof(int) * (int_) \
00398         + sizeof(WORD) * (word_) \
00399         + sizeof(UBYTE) * (byte_) \
00400     )
00401 #define PADPOINTER(long_, int_, word_, byte_) \
00402     PADDUMMY(int *, \
00403         + sizeof(LONG) * (long_) \
00404         + sizeof(int) * (int_) \
00405         + sizeof(WORD) * (word_) \
00406         + sizeof(UBYTE) * (byte_) \
00407     )
00408 #define PADLONG(int_, word_, byte_) \
00409     PADDUMMY(LONG, \
00410         + sizeof(int) * (int_) \
00411         + sizeof(WORD) * (word_) \
00412         + sizeof(UBYTE) * (byte_) \
00413     )
00414 #define PADINT(word_, byte_) \
00415     PADDUMMY(int, \
00416         + sizeof(WORD) * (word_) \
00417         + sizeof(UBYTE) * (byte_) \
00418     )
00419 #define PADWORD(byte_) \
00420     PADDUMMY(WORD, \
00421         + sizeof(UBYTE) * (byte_) \
00422     )
00423 /*
00424  *
00425  * #define WITHPCOUNTER
00426  * #define DEBUGGINGLOCKS
00427  * #define WITHSTATS
00428  */
00429 #define WITHSORTBOTS
00430
00431 #include <stdio.h>
00432 #include <stdlib.h>
00433 #include <string.h>
00434 #include <ctype.h>
00435 #include <limits.h>

```

```

00436 #ifdef ANSI
00437 #include <stdarg.h>
00438 #include <time.h>
00439 #endif
00440 #ifdef WINDOWS
00441 #include "fwin.h"
00442 #endif
00443 #ifdef UNIX
00444 #include <unistd.h>
00445 #include <time.h>
00446 #include <fcntl.h>
00447 #include <sys/file.h>
00448 #include "unix.h"
00449 #endif
00450 #ifdef WITHZLIB
00451 #ifdef WITHZSTD
00452 #include <zstd_zlibwrapper.h>
00453 #else
00454 #include <zlib.h>
00455 #endif
00456 #endif
00457 #ifdef WITHPTHREADS
00458 #include <pthread.h>
00459 #endif
00460
00461 /*
00462     PARALLELCODE indicates code that is common for TFORM and ParFORM but
00463     should not be there for sequential FORM.
00464 */
00465 #if defined(WITHMPI) || defined(WITHPTHREADS)
00466 #define PARALLELCODE
00467 #endif
00468
00469 #include "ftypes.h"
00470 #include "sizes.h"
00471 #include "minos.h"
00472 #include "structs.h"
00473 #include "declare.h"
00474 #include "variable.h"
00475
00476 /*
00477  * The interface to file routines for UNIX or non-UNIX (Windows).
00478  */
00479 #ifdef UNIX
00480
00481 #define UFILES
00482 typedef struct FILEs {
00483     int descriptor;
00484 } FILEs;
00485 extern FILEs *Uopen(char *, char *);
00486 extern int Uclose(FILEs *);
00487 extern size_t Uread(char *, size_t, size_t, FILEs *);
00488 extern size_t Uwrite(char *, size_t, size_t, FILEs *);
00489 extern int Useek(FILEs *, off_t, int);
00490 extern off_t Utell(FILEs *);
00491 extern void Uflush(FILEs *);
00492 extern int Ugetpos(FILEs *, fpos_t *);
00493 extern int Usetpos(FILEs *, fpos_t *);
00494 extern void Usetbuf(FILEs *, char *);
00495 #define Usync(f) fsync(f->descriptor)
00496 #define Utruncate(f) { \
00497     if ( ftruncate(f->descriptor, 0) ) { \
00498         MLOCK(ErrorMessageLock); \
00499         MesPrint("Utruncate failed"); \
00500         MUNLOCK(ErrorMessageLock); \
00501         /* Calling Terminate() here may cause an infinite loop due to CleanupSort(). */ \
00502         /* Terminate(-1); */ \
00503     } \
00504 }
00505 extern FILEs *Ustdout;
00506 #define MAX_OPEN_FILES getdtablesize()
00507 #define GetPID() ((LONG)getpid())
00508
00509 #else /* UNIX */
00510
00511 #define FILES FILE
00512 #define Uopen(x,y) fopen(x,y)
00513 #define Uflush(x) fflush(x)
00514 #define Uclose(x) fclose(x)
00515 #define Uread(x,y,z,u) fread(x,y,z,u)
00516 #define Uwrite(x,y,z,u) fwrite(x,y,z,u)
00517 #define Usetbuf(x,y) setbuf(x,y)
00518 #define Useek(x,y,z) fseek(x,y,z)
00519 #define Utell(x) ftell(x)
00520 #define Ugetpos(x,y) fgetpos(x,y)
00521 #define Usetpos(x,y) fsetpos(x,y)
00522 #define Usync(x) fflush(x)

```

```

00523 #define Utruncate(x) _chsize(_fileno(x),0)
00524 #define Ustdout stdout
00525 #define MAX_OPEN_FILES FOPEN_MAX
00526 #define bzero(b,len) (memset((b), 0, (len)), (void)0)
00527 #define GetPID() ((LONG)GetCurrentProcessId())
00528
00529 #endif /* UNIX */
00530
00531 /*
00532  * Some system may implement the POSIX "environ" variable as a macro, e.g.,
00533  * https://github.com/mingw-w64/mingw-w64/blob/v10.0.0/mingw-w64-headers/crt/stdlib.h#L704
00534  * which breaks the definition of DoShattering() in diagrams.c that uses
00535  * "environ" as a formal parameter. Because FORM doesn't use the POSIX "environ"
00536  * or its variant anyway, we can just undefine it.
00537  */
00538 #ifdef environ
00539 #undef environ
00540 #endif
00541
00542 #ifdef WITHMPI
00543 #include "parallel.h"
00544 #endif
00545
00546 #endif /* __FORM3H__ */

```

4.47 fsizes.h File Reference

This graph shows which files directly or indirectly include this file:



Macros

- #define [MAXPRENAMESIZE](#) 128
- #define [MAXPOWER](#) 10000
- #define [MAXVARIABLES](#) 8050
- #define [MAXDOLLARVARIABLES](#) 32000
- #define [WILDOFFSET](#) 6100
- #define [MAXINNAMETREE](#) 32768
- #define [MAXDUMMIES](#) 1000
- #define [WORKBUFFER](#) 10000000
- #define [MAXTER](#) 10000
- #define [HALFMAX](#) 0x100
- #define [MAXSUBEXPRESSIONS](#) 0x3FFF
- #define [MAXFILESTREAMSIZE](#) 1048576
- #define [MAXENAME](#) 16
- #define [MAXSAVEFUNCTION](#) 16384
- #define [MAXPARLEVEL](#) 100
- #define [MAXNUMBERSIZE](#) 200
- #define [MAXREPEAT](#) 100
- #define [NORMSIZE](#) 1000
- #define [INITNODESIZE](#) 10
- #define [INITNAMESIZE](#) 100
- #define [NUMFIXED](#) 128
- #define [MAXNEST](#) 100
- #define [MAXMATCH](#) 30
- #define [MAXIF](#) 20
- #define [SIZEFACS](#) 640L

- #define NUMFACS 50
- #define MAXLOOPS 30
- #define MAXLABELS 20
- #define COMMERCIALSIZE 24
- #define MAXFLAGS 16
- #define COMPRESSBUFFER 90000
- #define FORTRANCONTINUATIONLINES 15
- #define MAXLEVELS 2000
- #define MAXLHS 400
- #define MAXWILDC 100
- #define NUMTABLEENTRIES 1000
- #define COMPILERBUFFER 20000
- #define SMALLBUFFER 10000000L
- #define SMALLOVERFLOW 20000000L
- #define TERMSSMALL 100000L
- #define LARGEBUFFER 50000000L
- #define SCRATCHSIZE 50000000L
- #define MAXPATCHES 256
- #define MAXFPATCHES 256
- #define SORTIOSIZE 200000L
- #define SSMALLBUFFER 2560016L
- #define SSMALLOVERFLOW 3840032L
- #define STERMSSMALL 10000L
- #define SLARGEBUFFER 26880512L
- #define SMAXPATCHES 64
- #define SMAXFPATCHES 64
- #define SSORTIOSIZE 32768L
- #define SPECTATORSIZE 1048576L
- #define MAXFLEVELS 30
- #define COMPINC 2
- #define MAXNUMSIZE 10
- #define MAXBRACKETBUFFERSIZE 200000
- #define SFHSIZE 40
- #define DEFAULTPROCESSBUCKETSIZE 1000
- #define SHMWINSIZE 65536L
- #define TABLEEXTENSION 6
- #define GZIPDEFAULT 6
- #define DEFAULTTTHREADS 0
- #define DEFAULTTTHREADBUCKETSIZE 500
- #define DEFAULTTTHREADLOADBALANCING 1
- #define THREADSCRATCHSIZE 100000L
- #define THREADSCRATCHOUTSIZE 2500000L
- #define MAXTABLECOMBUF 1000000L
- #define MAXCOMBUFRHS 32500L
- #define NUMSTORECACHES 4
- #define SIZESTORECACHE 32768
- #define INDENTSPACE 3
- #define MULTIINDENTSPACE 1
- #define MAXMULTIBRACKETLEVELS 25
- #define FBUFFERSIZE 1026
- #define NPAIR1 38
- #define NPAIR2 89
- #define MAXLINELENGTH 256
- #define MINALLOC 32
- #define JUMPRATIO 4

- `#define` [MAXPARTICLES](#) 20
- `#define` [MAXCOUPLINGS](#) 20
- `#define` [NUMOPTIONS](#) 20
- `#define` [MAXLEGS](#) 20

4.47.1 Detailed Description

The definition the default values for certain buffer sizes etc.

Definition in file [fsizes.h](#).

4.47.2 Macro Definition Documentation

4.47.2.1 MAXPRENAMESIZE

```
#define MAXPRENAMESIZE 128
```

Definition at line [36](#) of file [fsizes.h](#).

4.47.2.2 MAXPOWER

```
#define MAXPOWER 10000
```

Definition at line [58](#) of file [fsizes.h](#).

4.47.2.3 MAXVARIABLES

```
#define MAXVARIABLES 8050
```

Definition at line [59](#) of file [fsizes.h](#).

4.47.2.4 MAXDOLLARVARIABLES

```
#define MAXDOLLARVARIABLES 32000
```

Definition at line [60](#) of file [fsizes.h](#).

4.47.2.5 WILDOFFSET

```
#define WILDOFFSET 6100
```

Definition at line [61](#) of file [fsizes.h](#).

4.47.2.6 MAXINNAMETREE

```
#define MAXINNAMETREE 32768
```

Definition at line 62 of file [fsizes.h](#).

4.47.2.7 MAXDUMMIES

```
#define MAXDUMMIES 1000
```

Definition at line 63 of file [fsizes.h](#).

4.47.2.8 WORKBUFFER

```
#define WORKBUFFER 10000000
```

Definition at line 64 of file [fsizes.h](#).

4.47.2.9 MAXTER

```
#define MAXTER 10000
```

Definition at line 65 of file [fsizes.h](#).

4.47.2.10 HALFMAX

```
#define HALFMAX 0x100
```

Definition at line 66 of file [fsizes.h](#).

4.47.2.11 MAXSUBEXPRESSIONS

```
#define MAXSUBEXPRESSIONS 0x3FFF
```

Definition at line 67 of file [fsizes.h](#).

4.47.2.12 MAXFILESTREAMSIZE

```
#define MAXFILESTREAMSIZE 1048576
```

Definition at line 68 of file [fsizes.h](#).

4.47.2.13 MAXENAME

```
#define MAXENAME 16
```

Definition at line 70 of file [fsizes.h](#).

4.47.2.14 MAXSAVEFUNCTION

```
#define MAXSAVEFUNCTION 16384
```

Definition at line 71 of file [fsizes.h](#).

4.47.2.15 MAXPARLEVEL

```
#define MAXPARLEVEL 100
```

Definition at line 73 of file [fsizes.h](#).

4.47.2.16 MAXNUMBERSIZE

```
#define MAXNUMBERSIZE 200
```

Definition at line 74 of file [fsizes.h](#).

4.47.2.17 MAXREPEAT

```
#define MAXREPEAT 100
```

Definition at line 76 of file [fsizes.h](#).

4.47.2.18 NORMSIZE

```
#define NORMSIZE 1000
```

Definition at line 77 of file [fsizes.h](#).

4.47.2.19 INITNODESIZE

```
#define INITNODESIZE 10
```

Definition at line 79 of file [fsizes.h](#).

4.47.2.20 INITNAMESIZE

```
#define INITNAMESIZE 100
```

Definition at line 80 of file [fsizes.h](#).

4.47.2.21 NUMFIXED

```
#define NUMFIXED 128
```

Definition at line 82 of file [fsizes.h](#).

4.47.2.22 MAXNEST

```
#define MAXNEST 100
```

Definition at line 83 of file [fsizes.h](#).

4.47.2.23 MAXMATCH

```
#define MAXMATCH 30
```

Definition at line 84 of file [fsizes.h](#).

4.47.2.24 MAXIF

```
#define MAXIF 20
```

Definition at line 85 of file [fsizes.h](#).

4.47.2.25 SIZEFACS

```
#define SIZEFACS 640L
```

Definition at line 86 of file [fsizes.h](#).

4.47.2.26 NUMFACS

```
#define NUMFACS 50
```

Definition at line 87 of file [fsizes.h](#).

4.47.2.27 MAXLOOPS

```
#define MAXLOOPS 30
```

Definition at line 88 of file [fsizes.h](#).

4.47.2.28 MAXLABELS

```
#define MAXLABELS 20
```

Definition at line 89 of file [fsizes.h](#).

4.47.2.29 COMMERCIALSIZE

```
#define COMMERCIALSIZE 24
```

Definition at line 90 of file [fsizes.h](#).

4.47.2.30 MAXFLAGS

```
#define MAXFLAGS 16
```

Definition at line 91 of file [fsizes.h](#).

4.47.2.31 COMPRESSBUFFER

```
#define COMPRESSBUFFER 90000
```

Definition at line 96 of file [fsizes.h](#).

4.47.2.32 FORTRANCONTINUATIONLINES

```
#define FORTRANCONTINUATIONLINES 15
```

Definition at line 97 of file [fsizes.h](#).

4.47.2.33 MAXLEVELS

```
#define MAXLEVELS 2000
```

Definition at line 98 of file [fsizes.h](#).

4.47.2.34 MAXLHS

```
#define MAXLHS 400
```

Definition at line 99 of file [fsizes.h](#).

4.47.2.35 MAXWILDC

```
#define MAXWILDC 100
```

Definition at line 100 of file [fsizes.h](#).

4.47.2.36 NUMTABLEENTRIES

```
#define NUMTABLEENTRIES 1000
```

Definition at line 101 of file [fsizes.h](#).

4.47.2.37 COMPILERBUFFER

```
#define COMPILERBUFFER 20000
```

Definition at line 102 of file [fsizes.h](#).

4.47.2.38 SMALLBUFFER

```
#define SMALLBUFFER 10000000L
```

Definition at line 119 of file [fsizes.h](#).

4.47.2.39 SMALLOVERFLOW

```
#define SMALLOVERFLOW 20000000L
```

Definition at line 120 of file [fsizes.h](#).

4.47.2.40 TERMSSMALL

```
#define TERMSSMALL 100000L
```

Definition at line 121 of file [fsizes.h](#).

4.47.2.41 LARGEBUFFER

```
#define LARGEBUFFER 50000000L
```

Definition at line 122 of file [fsizes.h](#).

4.47.2.42 SCRATCHSIZE

```
#define SCRATCHSIZE 50000000L
```

Definition at line 123 of file [fsizes.h](#).

4.47.2.43 MAXPATCHES

```
#define MAXPATCHES 256
```

Definition at line 125 of file [fsizes.h](#).

4.47.2.44 MAXFPATCHES

```
#define MAXFPATCHES 256
```

Definition at line 126 of file [fsizes.h](#).

4.47.2.45 SORTIOSIZE

```
#define SORTIOSIZE 200000L
```

Definition at line 127 of file [fsizes.h](#).

4.47.2.46 SSMALLBUFFER

```
#define SSMALLBUFFER 2560016L
```

Definition at line 129 of file [fsizes.h](#).

4.47.2.47 SSALLOVERFLOW

```
#define SSALLOVERFLOW 3840032L
```

Definition at line 130 of file [fsizes.h](#).

4.47.2.48 STERMSSMALL

```
#define STERMSSMALL 10000L
```

Definition at line 131 of file [fsizes.h](#).

4.47.2.49 SLARGEBUFFER

```
#define SLARGEBUFFER 26880512L
```

Definition at line 132 of file [fsizes.h](#).

4.47.2.50 SMAXPATCHES

```
#define SMAXPATCHES 64
```

Definition at line 133 of file [fsizes.h](#).

4.47.2.51 SMAXFPATCHES

```
#define SMAXFPATCHES 64
```

Definition at line 134 of file [fsizes.h](#).

4.47.2.52 SSORTIOSIZE

```
#define SSORTIOSIZE 32768L
```

Definition at line 135 of file [fsizes.h](#).

4.47.2.53 SPECTATORSIZE

```
#define SPECTATORSIZE 1048576L
```

Definition at line 137 of file [fsizes.h](#).

4.47.2.54 MAXFLEVELS

```
#define MAXFLEVELS 30
```

Definition at line 139 of file [fsizes.h](#).

4.47.2.55 COMPINC

```
#define COMPINC 2
```

Definition at line 141 of file [fsizes.h](#).

4.47.2.56 MAXNUMSIZE

```
#define MAXNUMSIZE 10
```

Definition at line 143 of file [fsizes.h](#).

4.47.2.57 MAXBRACKETBUFFERSIZE

```
#define MAXBRACKETBUFFERSIZE 200000
```

Definition at line 145 of file [fsizes.h](#).

4.47.2.58 SFHSIZE

```
#define SFHSIZE 40
```

Definition at line 147 of file [fsizes.h](#).

4.47.2.59 DEFAULTPROCESSBUCKETSIZE

```
#define DEFAULTPROCESSBUCKETSIZE 1000
```

Definition at line 149 of file [fsizes.h](#).

4.47.2.60 SHMWINSIZE

```
#define SHMWINSIZE 65536L
```

Definition at line 150 of file [fsizes.h](#).

4.47.2.61 TABLEEXTENSION

```
#define TABLEEXTENSION 6
```

Definition at line 152 of file [fsizes.h](#).

4.47.2.62 GZIPDEFAULT

```
#define GZIPDEFAULT 6
```

Definition at line 154 of file [fsizes.h](#).

4.47.2.63 DEFAULTTHREADS

```
#define DEFAULTTHREADS 0
```

Definition at line 155 of file [fsizes.h](#).

4.47.2.64 DEFAULTTHREADBUCKETSIZE

```
#define DEFAULTTHREADBUCKETSIZE 500
```

Definition at line 156 of file [fsizes.h](#).

4.47.2.65 DEFAULTTHREADLOADBALANCING

```
#define DEFAULTTHREADLOADBALANCING 1
```

Definition at line 157 of file [fsizes.h](#).

4.47.2.66 THREADSCRATCHSIZE

```
#define THREADSCRATCHSIZE 100000L
```

Definition at line 158 of file [fsizes.h](#).

4.47.2.67 THREADSCRATCHOUTSIZE

```
#define THREADSCRATCHOUTSIZE 2500000L
```

Definition at line 159 of file [fsizes.h](#).

4.47.2.68 MAXTABLECOMBUF

```
#define MAXTABLECOMBUF 1000000L
```

Definition at line 165 of file [fsizes.h](#).

4.47.2.69 MAXCOMBUFRHS

```
#define MAXCOMBUFRHS 32500L
```

Definition at line 166 of file [fsizes.h](#).

4.47.2.70 NUMSTORECACHES

```
#define NUMSTORECACHES 4
```

Definition at line 169 of file [fsizes.h](#).

4.47.2.71 SIZESTORECACHE

```
#define SIZESTORECACHE 32768
```

Definition at line 170 of file [fsizes.h](#).

4.47.2.72 INDENTSPACE

```
#define INDENTSPACE 3
```

Definition at line 172 of file [fsizes.h](#).

4.47.2.73 MULTIINDENTSPACE

```
#define MULTIINDENTSPACE 1
```

Definition at line 174 of file [fsizes.h](#).

4.47.2.74 MAXMULTIBRACKETLEVELS

```
#define MAXMULTIBRACKETLEVELS 25
```

Definition at line 175 of file [fsizes.h](#).

4.47.2.75 FBUFFERSIZE

```
#define FBUFFERSIZE 1026
```

Definition at line 177 of file [fsizes.h](#).

4.47.2.76 NPAIR1

```
#define NPAIR1 38
```

Definition at line 181 of file [fsizes.h](#).

4.47.2.77 NPAIR2

```
#define NPAIR2 89
```

Definition at line 182 of file [fsizes.h](#).

4.47.2.78 MAXLINELENGTH

```
#define MAXLINELENGTH 256
```

Definition at line 184 of file [fsizes.h](#).

4.47.2.79 MINALLOC

```
#define MINALLOC 32
```

Definition at line 185 of file [fsizes.h](#).

4.47.2.80 JUMPRATIO

```
#define JUMPRATIO 4
```

Definition at line 187 of file [fsizes.h](#).

4.47.2.81 MAXPARTICLES

```
#define MAXPARTICLES 20
```

Definition at line 191 of file [fsizes.h](#).

4.47.2.82 MAXCOUPLINGS

```
#define MAXCOUPLINGS 20
```

Definition at line 192 of file [fsizes.h](#).

4.47.2.83 NUMOPTIONS

```
#define NUMOPTIONS 20
```

Definition at line 193 of file [fsizes.h](#).

4.47.2.84 MAXLEGS

```
#define MAXLEGS 20
```

Definition at line 194 of file [fsizes.h](#).

4.48 fsizes.h

[Go to the documentation of this file.](#)

```

00001
00006 /* #[ License : */
00007 /*
00008 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00009 *   When using this file you are requested to refer to the publication
00010 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00011 *   This is considered a matter of courtesy as the development was paid
00012 *   for by FOM the Dutch physics granting agency and we would like to
00013 *   be able to track its scientific use to convince FOM of its value
00014 *   for the community.
00015 *
00016 *   This file is part of FORM.
00017 *
00018 *   FORM is free software: you can redistribute it and/or modify it under the
00019 *   terms of the GNU General Public License as published by the Free Software
00020 *   Foundation, either version 3 of the License, or (at your option) any later
00021 *   version.
00022 *
00023 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00024 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00025 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00026 *   details.
00027 *
00028 *   You should have received a copy of the GNU General Public License along
00029 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00030 */
00031 /* #] License : */
00032
00033 /*
00034     First the fixed variables
00035 */
00036 #define MAXPRENAMESIZE 128
00037 /*
00038     The following variables are default sizes. They can be changed
00039     into values read from the setup file
00040
00041     Remark (21-dec-2008 JV): WILDOFFSET*3 should be larger than WILDMASK!!!
00042     old value was WILDOFFSET 200000100
00043     be careful with old .sav files!!!
00044 */
00045 #ifdef WORDSIZE32
00046 #define MAXPOWER 500000000
00047 #define MAXVARIABLES 200000050
00048 #define MAXDOLLARVARIABLES 1000000000L
00049 #define WILDOFFSET 400000100
00050 #define MAXINNAMETREE 2000000000
00051 #define MAXDUMMIES 100000000
00052 #define WORKBUFFER 40000000
00053 #define MAXTER 40000
00054 #define HALFMAX 0x10000
00055 #define MAXSUBEXPRESSIONS 0x1FFFFFFF
00056 #define MAXFILESTREAMSIZE 1024
00057 #else
00058 #define MAXPOWER 10000
00059 #define MAXVARIABLES 8050
00060 #define MAXDOLLARVARIABLES 32000
00061 #define WILDOFFSET 6100
00062 #define MAXINNAMETREE 32768
00063 #define MAXDUMMIES 1000
00064 #define WORKBUFFER 10000000
00065 #define MAXTER 10000
00066 #define HALFMAX 0x100
00067 #define MAXSUBEXPRESSIONS 0x3FFF
00068 #define MAXFILESTREAMSIZE 1048576
00069 #endif
00070 #define MAXNAME 16
00071 #define MAXSAVEFUNCTION 16384
00072
00073 #define MAXPARLEVEL 100
00074 #define MAXNUMBERSIZE 200
00075
00076 #define MAXREPEAT 100
00077 #define NORMSIZE 1000
00078
00079 #define INITNODESIZE 10
00080 #define INITNAMESIZE 100
00081
00082 #define NUMFIXED 128
00083 #define MAXNEST 100
00084 #define MAXMATCH 30
00085 #define MAXIF 20
00086 #define SIZEFACS 640L

```

```
00087 #define NUMFACS 50
00088 #define MAXLOOPS 30
00089 #define MAXLABELS 20
00090 #define COMMERCIALSIZE 24
00091 #define MAXFLAGS 16
00092 /*
00093     The next quantities should still be eliminated from the program
00094     This should be together with changes in setfile!
00095 */
00096 #define COMPRESSBUFFER 90000
00097 #define FORTRANCONTINUATIONLINES 15
00098 #define MAXLEVELS 2000
00099 #define MAXLHS 400
00100 #define MAXWILDC 100
00101 #define NUMTABLEENTRIES 1000
00102 #define COMPILERBUFFER 20000
00103
00104 #ifdef WORDSIZE32
00105 #ifdef WITHPTHREADS
00106 #define SMALLBUFFER 300000000L
00107 #define SMALLOVERFLOW 600000000L
00108 #define TERMSSMALL 3000000L
00109 #define LARGEBUFFER 1500000000L
00110 #define SCRATCHSIZE 500000000L
00111 #else
00112 #define SMALLBUFFER 150000000L
00113 #define SMALLOVERFLOW 300000000L
00114 #define TERMSSMALL 2000000L
00115 #define LARGEBUFFER 800000000L
00116 #define SCRATCHSIZE 500000000L
00117 #endif
00118 #else
00119 #define SMALLBUFFER 100000000L
00120 #define SMALLOVERFLOW 20000000L
00121 #define TERMSSMALL 100000L
00122 #define LARGEBUFFER 50000000L
00123 #define SCRATCHSIZE 50000000L
00124 #endif
00125 #define MAXPATCHES 256
00126 #define MAXFPATCHES 256
00127 #define SORTIOSIZE 200000L
00128
00129 #define SSMALLBUFFER 2560016L
00130 #define SSMALLOVERFLOW 3840032L
00131 #define STERMSSMALL 10000L
00132 #define SLARGEBUFFER 26880512L
00133 #define SMAXPATCHES 64
00134 #define SMAXFPATCHES 64
00135 #define SSORTIOSIZE 32768L
00136
00137 #define SPECTATORSIZE 1048576L
00138
00139 #define MAXFLEVELS 30
00140
00141 #define COMPINC 2
00142
00143 #define MAXNUMSIZE 10
00144
00145 #define MAXBRACKETBUFFERSIZE 200000
00146
00147 #define SFHSIZE 40
00148
00149 #define DEFAULTPROCESSBUCKETSIZE 1000
00150 #define SHMWINSIZE 65536L
00151
00152 #define TABLEEXTENSION 6
00153
00154 #define GZIPDEFAULT 6
00155 #define DEFAULTTHREADS 0
00156 #define DEFAULTTHREADBUCKETSIZE 500
00157 #define DEFAULTTHREADLOADBALANCING 1
00158 #define THREADSCRATCHSIZE 100000L
00159 #define THREADSCRATCHOUTSIZE 2500000L
00160
00161 #ifdef WORDSIZE32
00162 #define MAXTABLECOMBUF 100000000000L
00163 #define MAXCOMBUFRHS 1000000000L
00164 #else
00165 #define MAXTABLECOMBUF 1000000L
00166 #define MAXCOMBUFRHS 32500L
00167 #endif
00168
00169 #define NUMSTORECACHES 4
00170 #define SIZESTORECACHE 32768
00171
00172 #define INDENTSPACE 3
00173
```

```

00174 #define MULTIINDENTSPACE 1
00175 #define MAXMULTIBRACKETLEVELS 25
00176
00177 #define FBUFFERSIZE 1026
00178 /*
00179      For the random number generator (see commentary there)
00180 */
00181 #define NPAIR1 38
00182 #define NPAIR2 89
00183
00184 #define MAXLINELENGTH 256
00185 #define MINALLOC 32
00186
00187 #define JUMPRATIO 4
00188 /*
00189      Note: MAXCOUPLINGS should be at least MAXPARTICLES/2+1
00190 */
00191 #define MAXPARTICLES 20
00192 #define MAXCOUPLINGS 20
00193 #define NUMOPTIONS 20
00194 #define MAXLEGS 20
00195
00196 #ifdef WITHFLOAT
00197 #define MAXWEIGHT 0
00198 #define DEFAULTPRECISION 1000
00199 #endif

```

4.49 ftypes.h File Reference

This graph shows which files directly or indirectly include this file:



Macros

- #define [WITHOUTERROR](#) 0
- #define [WITHERROR](#) 1
- #define [FILESTREAM](#) 0
- #define [PREVARSTREAM](#) 1
- #define [PREREADSTREAM](#) 2
- #define [PIPESTREAM](#) 3
- #define [PRECALCSTREAM](#) 4
- #define [DOLLARSTREAM](#) 5
- #define [PREREADSTREAM2](#) 6
- #define [EXTERNALCHANNELSTREAM](#) 7
- #define [PREREADSTREAM3](#) 8
- #define [REVERSEFILESTREAM](#) 9
- #define [INPUTSTREAM](#) 10
- #define [ENDOFSTREAM](#) 0xFF
- #define [ENDOFINPUT](#) 0xFF
- #define [SUBROUTINEFILE](#) 0
- #define [PROCEDUREFILE](#) 1
- #define [HEADERFILE](#) 2
- #define [SETUPFILE](#) 3
- #define [TABLEBASEFILE](#) 4
- #define [FIRSTMODULE](#) -1
- #define [GLOBALMODULE](#) 0
- #define [SORTMODULE](#) 1
- #define [STOREMODULE](#) 2

- #define [CLEARMODULE](#) 3
- #define [ENDMODULE](#) 4
- #define [POLYFUN](#) 0
- #define [NOPARALLEL_DOLLAR](#) 0x0001
- #define [NOPARALLEL_RHS](#) 0x0002
- #define [NOPARALLEL_CONVPOLY](#) 0x0004
- #define [NOPARALLEL_SPECTATOR](#) 0x0008
- #define [NOPARALLEL_USER](#) 0x0010
- #define [NOPARALLEL_TBLDOLLAR](#) 0x0100
- #define [NOPARALLEL_NPROC](#) 0x0200
- #define [PARALLELFLAG](#) 0x0000
- #define [PRENOACTION](#) 0
- #define [PRERAISEAFTER](#) 1
- #define [PRELOWERAFTER](#) 2
- #define [WITHSEMICOLON](#) 0
- #define [WITHOUTSEMICOLON](#) 1
- #define [MODULEINSTACK](#) 8
- #define [EXECUTINGIF](#) 0
- #define [LOOKINGFORELSE](#) 1
- #define [LOOKINGFORENDIF](#) 2
- #define [NEWSTATEMENT](#) 1
- #define [OLDSTATEMENT](#) 0
- #define [EXECUTINGPRESWITCH](#) 0
- #define [SEARCHINGPRECASE](#) 1
- #define [SEARCHINGPREENDSWITCH](#) 2
- #define [PREPROONLY](#) 1
- #define [DUMPTOCOMPILER](#) 2
- #define [DUMPOUTTERMS](#) 4
- #define [DUMPINTERMS](#) 8
- #define [DUMPTOSORT](#) 16
- #define [DUMPTOPARALLEL](#) 32
- #define [THREADSDEBUG](#) 64
- #define [ERROROUT](#) 0
- #define [INPUTOUT](#) 1
- #define [STATSOUT](#) 2
- #define [EXPRROUT](#) 3
- #define [WRITEOUT](#) 4
- #define [EXTERNALCHANNELOUT](#) 5
- #define [NUMERICLOOP](#) 0
- #define [LISTEDLOOP](#) 1
- #define [ONEEXPRESSION](#) 2
- #define [PRETYPENONE](#) 0
- #define [PRETYPEIF](#) 1
- #define [PRETYPEDO](#) 2
- #define [PRETYPEPROCEDURE](#) 3
- #define [PRETYPESWITCH](#) 4
- #define [PRETYPEINSIDE](#) 5
- #define [DECLARATION](#) 1
- #define [SPECIFICATION](#) 2
- #define [DEFINITION](#) 3
- #define [STATEMENT](#) 4
- #define [TOOOUTPUT](#) 5
- #define [ATENDOFMODULE](#) 6
- #define [MIXED2](#) 8
- #define [MIXED](#) 9

- #define NOAUTO 0
- #define PARTEST 1
- #define WITHAUTO 2
- #define ALLVARIABLES -1
- #define SYMBOLONLY 1
- #define INDEXONLY 2
- #define VECTORONLY 4
- #define FUNCTIONONLY 8
- #define SETONLY 16
- #define EXPRESSIONONLY 32

Defines: compiler types

Type of variable found by the compiler.

- #define CDELETE -1
- #define ANYTYPE -1
- #define CSYMBOL 0
- #define CINDEX 1
- #define CVECTOR 2
- #define CFUNCTION 3
- #define CSET 4
- #define CEXPRESSION 5
- #define CDOTPRODUCT 6
- #define CNUMBER 7
- #define CSUBEXP 8
- #define CDELTA 9
- #define CDOLLAR 10
- #define CDUBIOUS 11
- #define CRANGE 12
- #define CMODEL 13
- #define CVECTOR1 21
- #define CDOUBLEDOT 22
- #define TSYMBOL -1
- #define TINDEX -2
- #define TVECTOR -3
- #define TFUNCTION -4
- #define TSET -5
- #define TEXPRESSSION -6
- #define TDOTPRODUCT -7
- #define TNUMBER -8
- #define TSUBEXP -9
- #define TDELTA -10
- #define TDOLLAR -11
- #define TDUBIOUS -12
- #define LPARENTHESIS -13
- #define RPARENTHESIS -14
- #define TWILDCARD -15
- #define TWILDARG -16
- #define TDOT -17
- #define LBRACE -18
- #define RBRACE -19
- #define TCOMMA -20
- #define TFUNOPEN -21
- #define TFUNCLOSE -22
- #define TMULTIPLY -23
- #define TDIVIDE -24
- #define TPOWER -25
- #define TPLUS -26
- #define TMINUS -27
- #define TNOT -28
- #define TENDOFIT -29
- #define TSETOPEN -30

- #define TSETCLOSE -31
- #define TGENINDEX -32
- #define TCONJUGATE -33
- #define LRPARENTHESIS -34
- #define TNUMBER1 -35
- #define TPOWER1 -36
- #define TEMPTY -37
- #define TSETNUM -38
- #define TSGAMMA -39
- #define TSETDOL -40
- #define TFLOAT -41
- #define TYPEISFUN 0
- #define TYPEISSUB 1
- #define TYPEISMYSTERY -1
- #define LHSIDEX 2
- #define LHSIDE 1
- #define RHSIDE 0
- #define FORTRANMODE 1
- #define REDUCEMODE 2
- #define MAPLEMODE 3
- #define MATHEMATICAMODE 4
- #define CMODE 5
- #define VORTRANMODE 6
- #define PFORTRANMODE 7
- #define DOUBLEFORTRANMODE 33
- #define DOUBLEPRECISIONFLAG 32
- #define NODOUBLEMASK 31
- #define QUADRUPLEFORTRANMODE 65
- #define QUADRUPLEPRECISIONFLAG 64
- #define ALLINTEGERDOUBLE 128
- #define NOQUADMASK 63
- #define NORMALFORMAT 0
- #define NOSPACEFORMAT 1
- #define ISNOTFORTRAN90 0
- #define ISFORTRAN90 1
- #define ALSOREVERSE 1
- #define CHISHOLM 2
- #define NOTRICK 16
- #define SORTLOWFIRST 0
- #define SORTHIGHFIRST 1
- #define SORTPOWERFIRST 2
- #define SORTANTIPOWER 3
- #define STATSSPLITMERGE 0
- #define STATSMERGETOFILE 1
- #define STATSPOSTSORT 2
- #define NOCHECKLOGTYPE 0
- #define CHECKLOGTYPE 1
- #define NMIN4SHIFT 4
- #define SYMBOL 1
- #define DOTPRODUCT 2
- #define VECTOR 3
- #define INDEX 4
- #define EXPRESSION 5
- #define SUBEXPRESSION 6
- #define DOLLAREXPRESSION 7
- #define SETSET 8
- #define ARGWILD 9
- #define MINVECTOR 10
- #define SETEXP 11
- #define DOLLAREXP2 12
- #define HAAKJE0 9
- #define FUNCTION 20
- #define TMPPOLYFUN 14
- #define ARGFIELD 15

- #define [SNUMBER](#) 16
- #define [LNUMBER](#) 17
- #define [HAAKJE](#) 18
- #define [DELTA](#) 19
- #define [EXPONENT](#) 20
- #define [DENOMINATOR](#) 21
- #define [SETFUNCTION](#) 22
- #define [GAMMA](#) 23
- #define [GAMMAI](#) 24
- #define [GAMMAFIVE](#) 25
- #define [GAMMASIX](#) 26
- #define [GAMMASEVEN](#) 27
- #define [SUMF1](#) 28
- #define [SUMF2](#) 29
- #define [DUMFUN](#) 30
- #define [REPLACEMENT](#) 31
- #define [REVERSEFUNCTION](#) 32
- #define [DISTRIBUTION](#) 33
- #define [DELTA3](#) 34
- #define [DUMMYFUN](#) 35
- #define [DUMMYTEN](#) 36
- #define [LEVICIVITA](#) 37
- #define [FACTORIAL](#) 38
- #define [INVERSEFACTORIAL](#) 39
- #define [BINOMIAL](#) 40
- #define [NUMARGSFUN](#) 41
- #define [SIGNFUN](#) 42
- #define [MODFUNCTION](#) 43
- #define [MOD2FUNCTION](#) 44
- #define [MINFUNCTION](#) 45
- #define [MAXFUNCTION](#) 46
- #define [ABSFUNCTION](#) 47
- #define [SIGFUNCTION](#) 48
- #define [INTFUNCTION](#) 49
- #define [THETA](#) 50
- #define [THETA2](#) 51
- #define [DELTA2](#) 52
- #define [DELTAP](#) 53
- #define [BERNOULLIFUNCTION](#) 54
- #define [COUNTFUNCTION](#) 55
- #define [MATCHFUNCTION](#) 56
- #define [PATTERNFUNCTION](#) 57
- #define [TERMFUNCTION](#) 58
- #define [CONJUGATION](#) 59
- #define [ROOTFUNCTION](#) 60
- #define [TABLEFUNCTION](#) 61
- #define [FIRSTBRACKET](#) 62
- #define [TERMSINEXPR](#) 63
- #define [NUMTERMSFUN](#) 64
- #define [GCDFUNCTION](#) 65
- #define [DIVFUNCTION](#) 66
- #define [REMFUNCTION](#) 67
- #define [MAXPOWEROF](#) 68
- #define [MINPOWEROF](#) 69
- #define [TABLESTUB](#) 70
- #define [FACTORIN](#) 71
- #define [TERMSINBRACKET](#) 72
- #define [WILDARGFUN](#) 73
- #define [SQRTFUNCTION](#) 74
- #define [LNFUNTION](#) 75
- #define [SINFUNTION](#) 76
- #define [COSFUNCTION](#) 77
- #define [TANFUNCTION](#) 78
- #define [ASINFUNTION](#) 79

- #define [ACOSFUNCTION](#) 80
- #define [ATANFUNCTION](#) 81
- #define [ATAN2FUNCTION](#) 82
- #define [SINHFUNCTION](#) 83
- #define [COSHFUNCTION](#) 84
- #define [TANHFUNCTION](#) 85
- #define [ASINHFUNCTION](#) 86
- #define [ACOSHFUNCTION](#) 87
- #define [ATANHFUNCTION](#) 88
- #define [LI2FUNCTION](#) 89
- #define [LINFUNCTION](#) 90
- #define [EXTRASYMFUN](#) 91
- #define [RANDOMFUNCTION](#) 92
- #define [RANPERM](#) 93
- #define [NUMFACTORS](#) 94
- #define [FIRSTTERM](#) 95
- #define [CONTENTTERM](#) 96
- #define [PRIMENUMBER](#) 97
- #define [EXTEUCLIDEAN](#) 98
- #define [MAKERATIONAL](#) 99
- #define [INVERSEFUNCTION](#) 100
- #define [IDFUNCTION](#) 101
- #define [PUTFIRST](#) 102
- #define [PERMUTATIONS](#) 103
- #define [PARTITIONS](#) 104
- #define [MULFUNCTION](#) 105
- #define [MOEBIUS](#) 106
- #define [TOPOLOGIES](#) 107
- #define [DIAGRAMS](#) 108
- #define [TOPO](#) 109
- #define [NODEFUNCTION](#) 110
- #define [EDGE](#) 111
- #define [SIZEOFFUNCTION](#) 112
- #define [BLOCK](#) 113
- #define [ONEPI](#) 114
- #define [PHI](#) 115
- #define [MAXBUILTINFUNCTION](#) 115
- #define [FIRSTUSERFUNCTION](#) 150
- #define [ALLFUNCTIONS](#) -1
- #define [ALLMZVFUNCTIONS](#) -2
- #define [ISYMBOL](#) 0
- #define [PISYMBOL](#) 1
- #define [COEFFSYMBOL](#) 2
- #define [NUMERATORSYMBOL](#) 3
- #define [DENOMINATORSYMBOL](#) 4
- #define [WILDARGSYMBOL](#) 5
- #define [DIMENSIONSYMBOL](#) 6
- #define [FACTORSYMBOL](#) 7
- #define [SEPARATESYMBOL](#) 8
- #define [EESYMBOL](#) 9
- #define [EMSYMBOL](#) 10
- #define [BUILTINSYMBOLS](#) 11
- #define [FIRSTUSERSYMBOL](#) 20
- #define [BUILTININDICES](#) 1
- #define [BUILTINVECTORS](#) 1
- #define [BUILTINDOLLARS](#) 1
- #define [WILDARGVECTOR](#) 0
- #define [WILDARGINDEX](#) 0
- #define [TYPEEXPRESSION](#) 0
- #define [TYPEIDNEW](#) 1
- #define [TYPEIDOLD](#) 2
- #define [TYPEOPERATION](#) 3
- #define [TYPEREPEAT](#) 4
- #define [TYPEENDREPEAT](#) 5

- #define [TYPECOUNT](#) 20
- #define [TYPEMULT](#) 21
- #define [TYPEGOTO](#) 22
- #define [TYPEDISCARD](#) 23
- #define [TYPEIF](#) 24
- #define [TYPEELSE](#) 25
- #define [TYPEELIF](#) 26
- #define [TYPEENDIF](#) 27
- #define [TYPESUM](#) 28
- #define [TYPECHISHOLM](#) 29
- #define [TYPEREVERSE](#) 30
- #define [YPEARG](#) 31
- #define [TYPENORM](#) 32
- #define [TYPENORM2](#) 33
- #define [TYPENORM3](#) 34
- #define [TYPEEXIT](#) 35
- #define [TYPESETEXIT](#) 36
- #define [TYPEPRINT](#) 37
- #define [TYPEFPRINT](#) 38
- #define [TYPEREDEFPRE](#) 39
- #define [TYPESPLITARG](#) 40
- #define [TYPESPLITARG2](#) 41
- #define [TYPEFACTARG](#) 42
- #define [TYPEFACTARG2](#) 43
- #define [TYPETRY](#) 44
- #define [TYPEASSIGN](#) 45
- #define [TYPERENUMBER](#) 46
- #define [TYPESUMFIX](#) 47
- #define [TYPEFINDLOOP](#) 48
- #define [TYPEUNRAVEL](#) 49
- #define [TYPEADJUSTBOUNDS](#) 50
- #define [TYPEINSIDE](#) 51
- #define [TYPETERM](#) 52
- #define [TYPESORT](#) 53
- #define [TYPEDETCURDUM](#) 54
- #define [TYPEINEXPRESSION](#) 55
- #define [TYPESPLITFIRSTARG](#) 56
- #define [TYPESPLITLASTARG](#) 57
- #define [TYPEMERGE](#) 58
- #define [TYPETESTUSE](#) 59
- #define [TYPEAPPLY](#) 60
- #define [TYPEAPPLYRESET](#) 61
- #define [TYPECHAININ](#) 62
- #define [TYPECHAINOUT](#) 63
- #define [TYPENORM4](#) 64
- #define [TYPEFACTOR](#) 65
- #define [YPEARGIMPLODE](#) 66
- #define [TYPEARGEXPLODE](#) 67
- #define [TYPEDENOMINATORS](#) 68
- #define [TYPESTUFFLE](#) 69
- #define [TYPEDROPCOEFFICIENT](#) 70
- #define [TYPETRANSFORM](#) 71
- #define [TYPETOPOLYNOMIAL](#) 72
- #define [TYPEFROMPOLYNOMIAL](#) 73
- #define [TYPEDOLOOP](#) 74
- #define [TYPEENDDOLOOP](#) 75
- #define [TYPEDROPSYMBOLS](#) 76
- #define [TYPEPUTINSIDE](#) 77
- #define [TYPETOSPECTATOR](#) 78
- #define [YPEARGTOEXTRASymbol](#) 79
- #define [TYPECANONICALIZE](#) 80
- #define [TYPESWITCH](#) 81
- #define [TYPEENDSWITCH](#) 82
- #define [TYPESETUSERFLAG](#) 83

- #define [TYPECLEARUSERFLAG](#) 84
- #define [TYPEALLLOOPS](#) 85
- #define [TYPEALLPATHS](#) 86
- #define [TAKETRACE](#) 1
- #define [CONTRACT](#) 2
- #define [RATIO](#) 3
- #define [SYMMETRIZE](#) 4
- #define [TENVEC](#) 5
- #define [SUMNUM1](#) 6
- #define [SUMNUM2](#) 7
- #define [WILDDUMMY](#) 0
- #define [SYMTONUM](#) 1
- #define [SYMTOSYM](#) 2
- #define [SYMTOSUB](#) 3
- #define [VECTOMIN](#) 4
- #define [VECTOVEC](#) 5
- #define [VECTOSUB](#) 6
- #define [INDTOIND](#) 7
- #define [INDTOSUB](#) 8
- #define [FUNTOFUN](#) 9
- #define [ARGTOARG](#) 10
- #define [ARLTOARL](#) 11
- #define [EXPTOEXP](#) 12
- #define [FROMBRAC](#) 13
- #define [FROMSET](#) 14
- #define [SETTONUM](#) 15
- #define [WILDCARDS](#) 16
- #define [SETNUMBER](#) 17
- #define [LOADDOLLAR](#) 18
- #define [NUMTONUM](#) 20
- #define [NUMTOSYM](#) 21
- #define [NUMTOIND](#) 22
- #define [NUMTOSUB](#) 23
- #define [EATTENSOR](#) 0x2000
- #define [CLEANFLAG](#) 0
- #define [DIRTYFLAG](#) 1
- #define [DIRTYSYMFLAG](#) 2
- #define [MUSTCLEANPRF](#) 4
- #define [SUBTERMUSED1](#) 8
- #define [SUBTERMUSED2](#) 16
- #define [ALLDIRTY](#) (DIRTYFLAG|DIRTYSYMFLAG)
- #define [ARGHEAD](#) 2
- #define [FUNHEAD](#) 3
- #define [SUBEXPSIZE](#) 5
- #define [EXPRHEAD](#) 5
- #define [TYPEARGHEADSIZE](#) 6
- #define [SKIP](#) 1
- #define [DROP](#) 2
- #define [HIDE](#) 3
- #define [UNHIDE](#) 4
- #define [INTOHIDE](#) 5
- #define [LOCALEXPRESSION](#) 0
- #define [SKIPLEXPRESSION](#) 1
- #define [DROPLEXPRESSION](#) 2
- #define [DROPPEDEXPRESSION](#) 3
- #define [GLOBALEXPRESSION](#) 4
- #define [SKIPGEXPRESSION](#) 5
- #define [DROPGEXPRESSION](#) 6
- #define [STOREDEXPRESSION](#) 8
- #define [HIDDENLEXPRESSION](#) 9
- #define [HIDDENGEXPRESSION](#) 13
- #define [INCEXPRESSION](#) 9
- #define [HIDELEXPRESSION](#) 10
- #define [HIDEGEXPRESSION](#) 14

- #define [DROPHLEXPRESSION](#) 11
- #define [DROPHGEXPRESSION](#) 15
- #define [UNHIDELEXPRESSION](#) 12
- #define [UNHIDEGEXPRESSION](#) 16
- #define [INTOHIDELEXPRESSION](#) 17
- #define [INTOHIDEGEXPRESSION](#) 18
- #define [SPECTATOREXPRESSION](#) 19
- #define [DROPSPECTATOREXPRESSION](#) 20
- #define [SKIPUNHIDELEXPRESSION](#) 21
- #define [SKIPUNHIDEGEXPRESSION](#) 22
- #define [PRINTOFF](#) 0
- #define [PRINTON](#) 1
- #define [PRINTCONTENTS](#) 2
- #define [PRINTCONTENT](#) 3
- #define [PRINTLFILE](#) 4
- #define [PRINTONETERM](#) 8
- #define [PRINTONEFUNCTION](#) 16
- #define [PRINTALL](#) 32
- #define [REGULAREXPRESSION](#) -1
- #define [REDEFINEDEXPRESSION](#) -2
- #define [NEWLYDEFINEDEXPRESSION](#) -3

Defines: function specs

Function specifications.

- #define [VERTEXFUNCTION](#) -1
- #define [GENERALFUNCTION](#) 0
- #define [TENSORFUNCTION](#) 2
- #define [GAMMAFUNCTION](#) 4
- #define [POS_](#) 0 /* integer > 0 */
- #define [POS0_](#) 1 /* integer >= 0 */
- #define [NEG_](#) 2 /* integer < 0 */
- #define [NEG0_](#) 3 /* integer <= 0 */
- #define [EVEN_](#) 4 /* integer (even) */
- #define [ODD_](#) 5 /* integer (odd) */
- #define [Z_](#) 6 /* integer */
- #define [SYMBOL_](#) 7 /* symbol only */
- #define [FIXED_](#) 8 /* fixed index */
- #define [INDEX_](#) 9 /* index only */
- #define [Q_](#) 10 /* rational */
- #define [DUMMYINDEX_](#) 11 /* dummy index only */
- #define [VECTOR_](#) 12 /* vector only */
- #define [GAMMA1](#) 0
- #define [GAMMA5](#) -1
- #define [GAMMA6](#) -2
- #define [GAMMA7](#) -3
- #define [FUNNYVEC](#) -4
- #define [FUNNYWILD](#) -5
- #define [SUMMEDIND](#) -6
- #define [NOINDEX](#) -7
- #define [FUNNYDOLLAR](#) -8
- #define [EMPTYINDEX](#) -9
- #define [MINSPEC](#) -10
- #define [USEDFLAG](#) 2
- #define [DUMMYFLAG](#) 1
- #define [MAINSORT](#) 0
- #define [FUNCTIONSORT](#) 1
- #define [SUBSORT](#) 2
- #define [FLOATMODE](#) 1
- #define [RATIONALMODE](#) 0
- #define [NUMSPECSETS](#) 10
- #define [ISZERO](#) 1
- #define [ISUNMODIFIED](#) 2

- #define ISCOMPRESSED 4
- #define ISINRHS 8
- #define ISFACTORIZED 16
- #define TOBEFACTORED 32
- #define TOBEUNFACTORED 64
- #define KEEPZERO 128
- #define VARTYPENONE 0
- #define VARTYPECOMPLEX 1
- #define VARTYPEIMAGINARY 2
- #define VARTYPEROOTOFUNITY 4
- #define VARTYPEMINUS 8
- #define CYCLESYMMETRIC 1
- #define RCYCLESYMMETRIC 2
- #define SYMMETRIC 3
- #define ANTISYMMETRIC 4
- #define REVERSEORDER 256
- #define SUBMULTI 1
- #define SUBONCE 2
- #define SUBONLY 3
- #define SUBMANY 4
- #define SUBVECTOR 5
- #define SUBSELECT 6
- #define SUBALL 7
- #define SUBMASK 15
- #define SUBDISORDER 16
- #define SUBAFTER 32
- #define SUBAFTERNOT 64
- #define IDHEAD 6
- #define DOLLARFLAG 1
- #define NORMALIZEFLAG 2
- #define GIDENT 1
- #define GFIVE 4
- #define GPLUS 3
- #define GMINUS 2
- #define LONGNUMBER 1
- #define MATCH 2
- #define COEFFI 3
- #define SUBEXPR 4
- #define MULTIPLEOF 5
- #define IFDOLLAR 6
- #define IFEXPRESSION 7
- #define IFDOLLAREXTRA 8
- #define IFISFACTORIZED 9
- #define IFOCCURS 10
- #define IFUSERFLAG 11
- #define IFFLOATNUMBER 12
- #define GREATER 0
- #define GREATEREQUAL 1
- #define LESS 2
- #define LESSEQUAL 3
- #define EQUAL 4
- #define NOTEQUAL 5
- #define ORCOND 6
- #define ANDCOND 7
- #define DUMMY 1
- #define SORT 1
- #define STORE 2
- #define END 3
- #define GLOBAL 4
- #define CLEAR 5
- #define VECTBIT 1
- #define DOTPBIT 2
- #define FUNBIT 4
- #define SETBIT 8

- #define [EXTRAPARAMETER](#) 0x4000
- #define [GENCOMMUTE](#) 0
- #define [GENNONCOMMUTE](#) 0x2000
- #define [NAMENOTFOUND](#) -9
- #define [DOLUNDEFINED](#) 0
- #define [DOLNUMBER](#) 1
- #define [DOLARGUMENT](#) 2
- #define [DOLSUBTERM](#) 3
- #define [DOLTERMS](#) 4
- #define [DOLWILDARGS](#) 5
- #define [DOLINDEX](#) 6
- #define [DOLZERO](#) 7
- #define [FINDLOOP](#) 0
- #define [REPLACELOOP](#) 1
- #define [NOFUNPOWERS](#) 0
- #define [COMFUNPOWERS](#) 1
- #define [ALLFUNPOWERS](#) 2
- #define [PROPERORDERFLAG](#) 0
- #define [REGULAR](#) 0
- #define [FINISH](#) 1
- #define [POLYADD](#) 1
- #define [POLYSUB](#) 2
- #define [POLYMUL](#) 3
- #define [POLYDIV](#) 4
- #define [POLYREM](#) 5
- #define [POLYGCD](#) 6
- #define [POLYINTFAC](#) 7
- #define [POLYNORM](#) 8
- #define [MODNONE](#) 0
- #define [MODSUM](#) 1
- #define [MODMAX](#) 2
- #define [MODMIN](#) 3
- #define [MODLOCAL](#) 4
- #define [ELEMENTUSED](#) 1
- #define [ELEMENTLOADED](#) 2
- #define [POSNEG](#) 0x1
- #define [INVERSETABLE](#) 0x2
- #define [COEFFICIENTSONLY](#) 0x4
- #define [ALSOPOWERS](#) 0x8
- #define [ALSOFUNARGS](#) 0x10
- #define [ALSODOLLARS](#) 0x20
- #define [NOINVERSES](#) 0x40
- #define [POSITIVEONLY](#) 0
- #define [UNPACK](#) 0x80
- #define [NOUNPACK](#) 0
- #define [FROMFUNCTION](#) 0x100
- #define [VARNAMES](#) 0
- #define [AUTONAMES](#) 1
- #define [EXPRNAMES](#) 2
- #define [DOLLARNAMES](#) 3
- #define [DUMMYBUFFER](#) 1
- #define [ALLARGS](#) 1
- #define [NUMARG](#) 2
- #define [ARGRANGE](#) 3
- #define [MAKEARGS](#) 4
- #define [MAXRANGEINDICATOR](#) 4
- #define [REPLACEARG](#) 5
- #define [ENCODEARG](#) 6
- #define [DECODEARG](#) 7
- #define [IMPLODEARG](#) 8
- #define [EXPLODEARG](#) 9
- #define [PERMUTEARG](#) 10
- #define [REVERSEARG](#) 11
- #define [CYCLEARG](#) 12

- #define ISLYNDON 13
- #define ISLYNDONR 14
- #define TOLYNDON 15
- #define TOLYNDONR 16
- #define ADDARG 17
- #define MULTIPLYARG 18
- #define DROPARG 19
- #define SELECTARG 20
- #define DEDUPARG 21
- #define ZTOHARG 22
- #define HTOZARG 23
- #define BASECODE 1
- #define YESLYNDON 1
- #define NOLYNDON 2
- #define TOPOLYNOMIALFLAG 1
- #define FACTARGFLAG 2
- #define OLDFACTARG 1
- #define NEWFACTARG 0
- #define FROMMODULEOPTION 0
- #define FROMPOINTINSTRUCTION 1
- #define EXTRASYMBOL 0
- #define REGULARSYMBOL 1
- #define EXPRESSIONNUMBER 2
- #define O_NONE 0
- #define O_CSE 1
- #define O_CSEGREEDY 2
- #define O_GREEDY 3
- #define O_OCCURRENCE 0
- #define O_MCTS 1
- #define O_SIMULATED_ANNEALING 2
- #define O_FORWARD 0
- #define O_BACKWARD 1
- #define O_FORWARDORBACKWARD 2
- #define O_FORWARDANDBACKWARD 3
- #define OPTHEAD 3
- #define DOALL 1
- #define ONLYFUNCTIONS 2
- #define INUSE 1
- #define COULDCOMMUTE 2
- #define DOESNOTCOMMUTE 4
- #define DICT_NONUMBERS 0
- #define DICT_INTEGERONLY 1
- #define DICT_RATIONALONLY 2
- #define DICT_ALLNUMBERS 3
- #define DICT_NOVARIABLES 0
- #define DICT_DOVARIABLES 1
- #define DICT_NOSPECIALS 0
- #define DICT_DOSPECIALS 1
- #define DICT_NOFUNWITHARGS 0
- #define DICT_DOFUNWITHARGS 1
- #define DICT_NOTINDOLLARS 0
- #define DICT_INDOLLARS 1
- #define DICT_INTEGERNUMBER 1
- #define DICT_RATIONALNUMBER 2
- #define DICT_SYMBOL 3
- #define DICT_VECTOR 4
- #define DICT_INDEX 5
- #define DICT_FUNCTION 6
- #define DICT_FUNCTION_WITH_ARGUMENTS 7
- #define DICT_SPECIALCHARACTER 8
- #define DICT_RANGE 9
- #define READSPECTATORFLAG 3
- #define GLOBALSPECTATORFLAG 1
- #define ORDEREDSET 1

- `#define DENSETABLE 1`
- `#define SPARSETABLE 0`
- `#define ONEPARTICLEIRREDUCIBLE 1`
- `#define WITHINSERTIONS 2`
- `#define NOTADPOLES 4`
- `#define WITHSYMMETRIZE 8`
- `#define TOPOLOGIESONLY 16`
- `#define NONODES 32`
- `#define WITHEDGES 64`
- `#define CHECKEXTERN 128`
- `#define WITHBLOCKS 256`
- `#define WITHONEPI 512`
- `#define NOSNAILS 1024`
- `#define NOEXTSELF 2048`

Typedefs

- `typedef void(* PVFUNWP) ()`
- `typedef void(* PVFUNV) ()`
- `typedef int(* CFUN) ()`
- `typedef int(* TFUN) ()`
- `typedef int(* TFUN1) ()`

4.49.1 Detailed Description

Contains the definitions of many internal codes Rather than using numbers directly we do this by defines, making it much easier to change things. Changing things is sometimes also a good way of testing the code.

Definition in file [ftypes.h](#).

4.49.2 Macro Definition Documentation

4.49.2.1 WITHOUTERROR

```
#define WITHOUTERROR 0
```

The next macros were introduced when TFORM was programmed. In the case of workers, each worker may need some private data. These can in principle be accessed by some posix calls but that is unnecessarily slow. The passing of a pointer to the complete data struct with private data will be much faster. And anyway, there would have to be a macro that either makes the posix call (TFORM) or doesn't (FORM). The solution by having macro's that either pass the pointer (TFORM) or don't pass it (FORM) is seen as the best solution.

In the declarations and the calling of the functions we have to use the PHEAD or the BHEAD macro, respectively, if the pointer is to be passed. These macro's contain the comma as well. Hence we need special macro's if there are no other arguments. These are called PHEAD0 and BHEAD0.

Definition at line 51 of file [ftypes.h](#).

4.49.2.2 WITHERROR

```
#define WITHERROR 1
```

Definition at line 52 of file [ftypes.h](#).

4.49.2.3 FILESTREAM

```
#define FILESTREAM 0
```

Definition at line 58 of file [ftypes.h](#).

4.49.2.4 PREVARSTREAM

```
#define PREVARSTREAM 1
```

Definition at line 59 of file [ftypes.h](#).

4.49.2.5 PREREADSTREAM

```
#define PREREADSTREAM 2
```

Definition at line 60 of file [ftypes.h](#).

4.49.2.6 PIPESTREAM

```
#define PIPESTREAM 3
```

Definition at line 61 of file [ftypes.h](#).

4.49.2.7 PRECALCSTREAM

```
#define PRECALCSTREAM 4
```

Definition at line 62 of file [ftypes.h](#).

4.49.2.8 DOLLARSTREAM

```
#define DOLLARSTREAM 5
```

Definition at line 63 of file [ftypes.h](#).

4.49.2.9 PREREADSTREAM2

```
#define PREREADSTREAM2 6
```

Definition at line 64 of file [ftypes.h](#).

4.49.2.10 EXTERNALCHANNELSTREAM

```
#define EXTERNALCHANNELSTREAM 7
```

Definition at line 65 of file [ftypes.h](#).

4.49.2.11 PREREADSTREAM3

```
#define PREREADSTREAM3 8
```

Definition at line 66 of file [ftypes.h](#).

4.49.2.12 REVERSEFILESTREAM

```
#define REVERSEFILESTREAM 9
```

Definition at line 67 of file [ftypes.h](#).

4.49.2.13 INPUTSTREAM

```
#define INPUTSTREAM 10
```

Definition at line 68 of file [ftypes.h](#).

4.49.2.14 ENDOFSTREAM

```
#define ENDOFSTREAM 0xFF
```

Definition at line 70 of file [ftypes.h](#).

4.49.2.15 ENDOFINPUT

```
#define ENDOFINPUT 0xFF
```

Definition at line 71 of file [ftypes.h](#).

4.49.2.16 SUBROUTINEFILE

```
#define SUBROUTINEFILE 0
```

Definition at line 77 of file [ftypes.h](#).

4.49.2.17 PROCEDUREFILE

```
#define PROCEDUREFILE 1
```

Definition at line 78 of file [ftypes.h](#).

4.49.2.18 HEADERFILE

```
#define HEADERFILE 2
```

Definition at line 79 of file [ftypes.h](#).

4.49.2.19 SETUPFILE

```
#define SETUPFILE 3
```

Definition at line 80 of file [ftypes.h](#).

4.49.2.20 TABLEBASEFILE

```
#define TABLEBASEFILE 4
```

Definition at line 81 of file [ftypes.h](#).

4.49.2.21 FIRSTMODULE

```
#define FIRSTMODULE -1
```

Definition at line 87 of file [ftypes.h](#).

4.49.2.22 GLOBALMODULE

```
#define GLOBALMODULE 0
```

Definition at line 88 of file [ftypes.h](#).

4.49.2.23 SORTMODULE

```
#define SORTMODULE 1
```

Definition at line 89 of file [ftypes.h](#).

4.49.2.24 STOREMODULE

```
#define STOREMODULE 2
```

Definition at line 90 of file [ftypes.h](#).

4.49.2.25 CLEARMODULE

```
#define CLEARMODULE 3
```

Definition at line 91 of file [ftypes.h](#).

4.49.2.26 ENDMODULE

```
#define ENDMODULE 4
```

Definition at line 92 of file [ftypes.h](#).

4.49.2.27 POLYFUN

```
#define POLYFUN 0
```

Definition at line 94 of file [ftypes.h](#).

4.49.2.28 NOPARALLEL_DOLLAR

```
#define NOPARALLEL_DOLLAR 0x0001
```

Definition at line 96 of file [ftypes.h](#).

4.49.2.29 NOPARALLEL_RHS

```
#define NOPARALLEL_RHS 0x0002
```

Definition at line 97 of file [ftypes.h](#).

4.49.2.30 NOPARALLEL_CONVPOLY

```
#define NOPARALLEL_CONVPOLY 0x0004
```

Definition at line 98 of file [ftypes.h](#).

4.49.2.31 NOPARALLEL_SPECTATOR

```
#define NOPARALLEL_SPECTATOR 0x0008
```

Definition at line 99 of file [ftypes.h](#).

4.49.2.32 NOPARALLEL_USER

```
#define NOPARALLEL_USER 0x0010
```

Definition at line 100 of file [ftypes.h](#).

4.49.2.33 NOPARALLEL_TBLDOLLAR

```
#define NOPARALLEL_TBLDOLLAR 0x0100
```

Definition at line 101 of file [ftypes.h](#).

4.49.2.34 NOPARALLEL_NPROC

```
#define NOPARALLEL_NPROC 0x0200
```

Definition at line 102 of file [ftypes.h](#).

4.49.2.35 PARALLELFLAG

```
#define PARALLELFLAG 0x0000
```

Definition at line 103 of file [ftypes.h](#).

4.49.2.36 PRENOACTION

```
#define PRENOACTION 0
```

Definition at line 105 of file [ftypes.h](#).

4.49.2.37 PRERAISEAFTER

```
#define PRERAISEAFTER 1
```

Definition at line 106 of file [ftypes.h](#).

4.49.2.38 PRELOWERAFTER

```
#define PRELOWERAFTER 2
```

Definition at line 107 of file [ftypes.h](#).

4.49.2.39 WITHSEMICOLON

```
#define WITHSEMICOLON 0
```

Definition at line 114 of file [ftypes.h](#).

4.49.2.40 WITHOUTSEMICOLON

```
#define WITHOUTSEMICOLON 1
```

Definition at line 115 of file [ftypes.h](#).

4.49.2.41 MODULEINSTACK

```
#define MODULEINSTACK 8
```

Definition at line 116 of file [ftypes.h](#).

4.49.2.42 EXECUTINGIF

```
#define EXECUTINGIF 0
```

Definition at line 117 of file [ftypes.h](#).

4.49.2.43 LOOKINGFORELSE

```
#define LOOKINGFORELSE 1
```

Definition at line 118 of file [ftypes.h](#).

4.49.2.44 LOOKINGFORENDIF

```
#define LOOKINGFORENDIF 2
```

Definition at line 119 of file [ftypes.h](#).

4.49.2.45 NEWSTATEMENT

```
#define NEWSTATEMENT 1
```

Definition at line 120 of file [ftypes.h](#).

4.49.2.46 OLDSTATEMENT

```
#define OLDSTATEMENT 0
```

Definition at line 121 of file [ftypes.h](#).

4.49.2.47 EXECUTINGPRESWITCH

```
#define EXECUTINGPRESWITCH 0
```

Definition at line 123 of file [ftypes.h](#).

4.49.2.48 SEARCHINGPRECASE

```
#define SEARCHINGPRECASE 1
```

Definition at line 124 of file [ftypes.h](#).

4.49.2.49 SEARCHINGPREENDSWITCH

```
#define SEARCHINGPREENDSWITCH 2
```

Definition at line 125 of file [ftypes.h](#).

4.49.2.50 PREPROONLY

```
#define PREPROONLY 1
```

Definition at line 127 of file [ftypes.h](#).

4.49.2.51 DUMPTOCOMPILER

```
#define DUMPTOCOMPILER 2
```

Definition at line 128 of file [ftypes.h](#).

4.49.2.52 DUMPOUTTERMS

```
#define DUMPOUTTERMS 4
```

Definition at line 129 of file [ftypes.h](#).

4.49.2.53 DUMPINTERMS

```
#define DUMPINTERMS 8
```

Definition at line 130 of file [ftypes.h](#).

4.49.2.54 DUMPTOSORT

```
#define DUMPTOSORT 16
```

Definition at line 131 of file [ftypes.h](#).

4.49.2.55 DUMPTOPARALLEL

```
#define DUMPTOPARALLEL 32
```

Definition at line 132 of file [ftypes.h](#).

4.49.2.56 THREADSDEBUG

```
#define THREADSDEBUG 64
```

Definition at line 133 of file [ftypes.h](#).

4.49.2.57 ERROROUT

```
#define ERROROUT 0
```

Definition at line 135 of file [ftypes.h](#).

4.49.2.58 INPUTOUT

```
#define INPUTOUT 1
```

Definition at line 136 of file [ftypes.h](#).

4.49.2.59 STATSOUT

```
#define STATSOUT 2
```

Definition at line 137 of file [ftypes.h](#).

4.49.2.60 EXPRSOUT

```
#define EXPRSOUT 3
```

Definition at line 138 of file [ftypes.h](#).

4.49.2.61 WRITEOUT

```
#define WRITEOUT 4
```

Definition at line 139 of file [ftypes.h](#).

4.49.2.62 EXTERNALCHANNELOUT

```
#define EXTERNALCHANNELOUT 5
```

Definition at line 141 of file [ftypes.h](#).

4.49.2.63 NUMERICALLOOP

```
#define NUMERICALLOOP 0
```

Definition at line 143 of file [ftypes.h](#).

4.49.2.64 LISTEDLOOP

```
#define LISTEDLOOP 1
```

Definition at line 144 of file [ftypes.h](#).

4.49.2.65 ONEEXPRESSION

```
#define ONEEXPRESSION 2
```

Definition at line 145 of file [ftypes.h](#).

4.49.2.66 PRETYPENONE

```
#define PRETYPENONE 0
```

Definition at line 147 of file [ftypes.h](#).

4.49.2.67 PRETYPEIF

```
#define PRETYPEIF 1
```

Definition at line 148 of file [ftypes.h](#).

4.49.2.68 PRETYPEDO

```
#define PRETYPEDO 2
```

Definition at line 149 of file [ftypes.h](#).

4.49.2.69 PRETYPEPROCEDURE

```
#define PRETYPEPROCEDURE 3
```

Definition at line 150 of file [ftypes.h](#).

4.49.2.70 PRETYPESWITCH

```
#define PRETYPESWITCH 4
```

Definition at line 151 of file [ftypes.h](#).

4.49.2.71 PRETYPEINSIDE

```
#define PRETYPEINSIDE 5
```

Definition at line 152 of file [ftypes.h](#).

4.49.2.72 DECLARATION

```
#define DECLARATION 1
```

Definition at line 158 of file [ftypes.h](#).

4.49.2.73 SPECIFICATION

```
#define SPECIFICATION 2
```

Definition at line 159 of file [ftypes.h](#).

4.49.2.74 DEFINITION

```
#define DEFINITION 3
```

Definition at line 160 of file [ftypes.h](#).

4.49.2.75 STATEMENT

```
#define STATEMENT 4
```

Definition at line 161 of file [ftypes.h](#).

4.49.2.76 TOOOUTPUT

```
#define TOOOUTPUT 5
```

Definition at line 162 of file [ftypes.h](#).

4.49.2.77 ATENDOFMODULE

```
#define ATENDOFMODULE 6
```

Definition at line 163 of file [ftypes.h](#).

4.49.2.78 MIXED2

```
#define MIXED2 8
```

Definition at line 164 of file [ftypes.h](#).

4.49.2.79 MIXED

```
#define MIXED 9
```

Definition at line 165 of file [ftypes.h](#).

4.49.2.80 NOAUTO

```
#define NOAUTO 0
```

Definition at line 191 of file [ftypes.h](#).

4.49.2.81 PARTEST

```
#define PARTEST 1
```

Definition at line 192 of file [ftypes.h](#).

4.49.2.82 WITHAUTO

```
#define WITHAUTO 2
```

Definition at line 193 of file [ftypes.h](#).

4.49.2.83 ALLVARIABLES

```
#define ALLVARIABLES -1
```

Definition at line 195 of file [ftypes.h](#).

4.49.2.84 SYMBOLONLY

```
#define SYMBOLONLY 1
```

Definition at line 196 of file [ftypes.h](#).

4.49.2.85 INDEXONLY

```
#define INDEXONLY 2
```

Definition at line 197 of file [ftypes.h](#).

4.49.2.86 VECTORONLY

```
#define VECTORONLY 4
```

Definition at line 198 of file [ftypes.h](#).

4.49.2.87 FUNCTIONONLY

```
#define FUNCTIONONLY 8
```

Definition at line 199 of file [ftypes.h](#).

4.49.2.88 SETONLY

```
#define SETONLY 16
```

Definition at line 200 of file [ftypes.h](#).

4.49.2.89 EXPRESSIONONLY

```
#define EXPRESSIONONLY 32
```

Definition at line 201 of file [ftypes.h](#).

4.49.2.90 CDELETE

```
#define CDELETE -1
```

Definition at line 212 of file [ftypes.h](#).

4.49.2.91 ANYTYPE

```
#define ANYTYPE -1
```

Definition at line 213 of file [ftypes.h](#).

4.49.2.92 CSYMBOL

```
#define CSYMBOL 0
```

Definition at line 214 of file [ftypes.h](#).

4.49.2.93 CINDEX

```
#define CINDEX 1
```

Definition at line 215 of file [ftypes.h](#).

4.49.2.94 CVECTOR

```
#define CVECTOR 2
```

Definition at line 216 of file [ftypes.h](#).

4.49.2.95 CFUNCTION

```
#define CFUNCTION 3
```

Definition at line 217 of file [ftypes.h](#).

4.49.2.96 CSET

```
#define CSET 4
```

Definition at line 218 of file [ftypes.h](#).

4.49.2.97 CEXPRESSION

```
#define CEXPRESSION 5
```

Definition at line 219 of file [ftypes.h](#).

4.49.2.98 CDOTPRODUCT

```
#define CDOTPRODUCT 6
```

Definition at line 220 of file [ftypes.h](#).

4.49.2.99 CNUMBER

```
#define CNUMBER 7
```

Definition at line 221 of file [ftypes.h](#).

4.49.2.100 CSUBEXP

```
#define CSUBEXP 8
```

Definition at line 222 of file [ftypes.h](#).

4.49.2.101 CDELTA

```
#define CDELTA 9
```

Definition at line 223 of file [ftypes.h](#).

4.49.2.102 CDOLLAR

```
#define CDOLLAR 10
```

Definition at line 224 of file [ftypes.h](#).

4.49.2.103 CDUBIOUS

```
#define CDUBIOUS 11
```

Definition at line 225 of file [ftypes.h](#).

4.49.2.104 CRANGE

```
#define CRANGE 12
```

Definition at line 226 of file [ftypes.h](#).

4.49.2.105 CMODEL

```
#define CMODEL 13
```

Definition at line 227 of file [ftypes.h](#).

4.49.2.106 CVECTOR1

```
#define CVECTOR1 21
```

Definition at line 228 of file [ftypes.h](#).

4.49.2.107 CDOUBLEDOT

```
#define CDOUBLEDOT 22
```

Definition at line 229 of file [ftypes.h](#).

4.49.2.108 TSYMBOL

```
#define TSYMBOL -1
```

Definition at line 237 of file [ftypes.h](#).

4.49.2.109 TINDEX

```
#define TINDEX -2
```

Definition at line 238 of file [ftypes.h](#).

4.49.2.110 TVECTOR

```
#define TVECTOR -3
```

Definition at line 239 of file [ftypes.h](#).

4.49.2.111 TFUNCTION

```
#define TFUNCTION -4
```

Definition at line 240 of file [ftypes.h](#).

4.49.2.112 TSET

```
#define TSET -5
```

Definition at line 241 of file [ftypes.h](#).

4.49.2.113 TEXPRESSSION

```
#define TEXPRESSSION -6
```

Definition at line 242 of file [ftypes.h](#).

4.49.2.114 TDOTPRODUCT

```
#define TDOTPRODUCT -7
```

Definition at line 243 of file [ftypes.h](#).

4.49.2.115 TNUMBER

```
#define TNUMBER -8
```

Definition at line 244 of file [ftypes.h](#).

4.49.2.116 TSUBEXP

```
#define TSUBEXP -9
```

Definition at line 245 of file [ftypes.h](#).

4.49.2.117 TDELTA

```
#define TDELTA -10
```

Definition at line 246 of file [ftypes.h](#).

4.49.2.118 TDOLLAR

```
#define TDOLLAR -11
```

Definition at line 247 of file [ftypes.h](#).

4.49.2.119 TDUBIOUS

```
#define TDUBIOUS -12
```

Definition at line 248 of file [ftypes.h](#).

4.49.2.120 LPARENTHESIS

```
#define LPARENTHESIS -13
```

Definition at line 249 of file [ftypes.h](#).

4.49.2.121 RPARENTHESIS

```
#define RPARENTHESIS -14
```

Definition at line 250 of file [ftypes.h](#).

4.49.2.122 TWILDCARD

```
#define TWILDCARD -15
```

Definition at line 251 of file [ftypes.h](#).

4.49.2.123 TWILDARG

```
#define TWILDARG -16
```

Definition at line 252 of file [ftypes.h](#).

4.49.2.124 TDOT

```
#define TDOT -17
```

Definition at line 253 of file [ftypes.h](#).

4.49.2.125 LBRACE

```
#define LBRACE -18
```

Definition at line 254 of file [ftypes.h](#).

4.49.2.126 RBRACE

```
#define RBRACE -19
```

Definition at line 255 of file [ftypes.h](#).

4.49.2.127 TCOMMA

```
#define TCOMMA -20
```

Definition at line 256 of file [ftypes.h](#).

4.49.2.128 TFUNOPEN

```
#define TFUNOPEN -21
```

Definition at line 257 of file [ftypes.h](#).

4.49.2.129 TFUNCLOSE

```
#define TFUNCLOSE -22
```

Definition at line 258 of file [ftypes.h](#).

4.49.2.130 TMULTIPLY

```
#define TMULTIPLY -23
```

Definition at line 259 of file [ftypes.h](#).

4.49.2.131 TDIVIDE

```
#define TDIVIDE -24
```

Definition at line 260 of file [ftypes.h](#).

4.49.2.132 TPOWER

```
#define TPOWER -25
```

Definition at line 261 of file [ftypes.h](#).

4.49.2.133 TPLUS

```
#define TPLUS -26
```

Definition at line 262 of file [ftypes.h](#).

4.49.2.134 TMINUS

```
#define TMINUS -27
```

Definition at line 263 of file [ftypes.h](#).

4.49.2.135 TNOT

```
#define TNOT -28
```

Definition at line 264 of file [ftypes.h](#).

4.49.2.136 TENDOFIT

```
#define TENDOFIT -29
```

Definition at line 265 of file [ftypes.h](#).

4.49.2.137 TSETOPEN

```
#define TSETOPEN -30
```

Definition at line 266 of file [ftypes.h](#).

4.49.2.138 TSETCLOSE

```
#define TSETCLOSE -31
```

Definition at line 267 of file [ftypes.h](#).

4.49.2.139 TGENINDEX

```
#define TGENINDEX -32
```

Definition at line 268 of file [ftypes.h](#).

4.49.2.140 TCONJUGATE

```
#define TCONJUGATE -33
```

Definition at line 269 of file [ftypes.h](#).

4.49.2.141 LRPARENTHESSES

```
#define LRPARENTHESSES -34
```

Definition at line 270 of file [ftypes.h](#).

4.49.2.142 TNUMBER1

```
#define TNUMBER1 -35
```

Definition at line 271 of file [ftypes.h](#).

4.49.2.143 TPOWER1

```
#define TPOWER1 -36
```

Definition at line 272 of file [ftypes.h](#).

4.49.2.144 TEMPTY

```
#define TEMPTY -37
```

Definition at line 273 of file [ftypes.h](#).

4.49.2.145 TSETNUM

```
#define TSETNUM -38
```

Definition at line 274 of file [ftypes.h](#).

4.49.2.146 TSGAMMA

```
#define TSGAMMA -39
```

Definition at line 275 of file [ftypes.h](#).

4.49.2.147 TSETDOL

```
#define TSETDOL -40
```

Definition at line 276 of file [ftypes.h](#).

4.49.2.148 TFLOAT

```
#define TFLOAT -41
```

Definition at line 277 of file [ftypes.h](#).

4.49.2.149 TYPEISFUN

```
#define TYPEISFUN 0
```

Definition at line 279 of file [ftypes.h](#).

4.49.2.150 TYPEISSUB

```
#define TYPEISSUB 1
```

Definition at line 280 of file [ftypes.h](#).

4.49.2.151 TYPEISMYSTERY

```
#define TYPEISMYSTERY -1
```

Definition at line 281 of file [ftypes.h](#).

4.49.2.152 LHSIDEX

```
#define LHSIDEX 2
```

Definition at line 283 of file [ftypes.h](#).

4.49.2.153 LHSIDE

```
#define LHSIDE 1
```

Definition at line 284 of file [ftypes.h](#).

4.49.2.154 RHSIDE

```
#define RHSIDE 0
```

Definition at line 285 of file [ftypes.h](#).

4.49.2.155 FORTRANMODE

```
#define FORTRANMODE 1
```

Definition at line 291 of file [ftypes.h](#).

4.49.2.156 REDUCEMODE

```
#define REDUCEMODE 2
```

Definition at line 292 of file [ftypes.h](#).

4.49.2.157 MAPLEMODE

```
#define MAPLEMODE 3
```

Definition at line 293 of file [ftypes.h](#).

4.49.2.158 MATHEMATICAMODE

```
#define MATHEMATICAMODE 4
```

Definition at line 294 of file [ftypes.h](#).

4.49.2.159 CMODE

```
#define CMODE 5
```

Definition at line 295 of file [ftypes.h](#).

4.49.2.160 VORTRANMODE

```
#define VORTRANMODE 6
```

Definition at line 296 of file [ftypes.h](#).

4.49.2.161 PFORTRANMODE

```
#define PFORTRANMODE 7
```

Definition at line 297 of file [ftypes.h](#).

4.49.2.162 DOUBLEFORTRANMODE

```
#define DOUBLEFORTRANMODE 33
```

Definition at line 298 of file [ftypes.h](#).

4.49.2.163 DOUBLEPRECISIONFLAG

```
#define DOUBLEPRECISIONFLAG 32
```

Definition at line 299 of file [ftypes.h](#).

4.49.2.164 NODOUBLEMASK

```
#define NODOUBLEMASK 31
```

Definition at line 300 of file [ftypes.h](#).

4.49.2.165 QUADRUPLEFORTRANMODE

```
#define QUADRUPLEFORTRANMODE 65
```

Definition at line 301 of file [ftypes.h](#).

4.49.2.166 QUADRUPLEPRECISIONFLAG

```
#define QUADRUPLEPRECISIONFLAG 64
```

Definition at line 302 of file [ftypes.h](#).

4.49.2.167 ALLINTEGERDOUBLE

```
#define ALLINTEGERDOUBLE 128
```

Definition at line 303 of file [ftypes.h](#).

4.49.2.168 NOQUADMASK

```
#define NOQUADMASK 63
```

Definition at line 304 of file [ftypes.h](#).

4.49.2.169 NORMALFORMAT

```
#define NORMALFORMAT 0
```

Definition at line 305 of file [ftypes.h](#).

4.49.2.170 NOSPACEFORMAT

```
#define NOSPACEFORMAT 1
```

Definition at line 306 of file [ftypes.h](#).

4.49.2.171 ISNOTFORTRAN90

```
#define ISNOTFORTRAN90 0
```

Definition at line 308 of file [ftypes.h](#).

4.49.2.172 ISFORTRAN90

```
#define ISFORTRAN90 1
```

Definition at line 309 of file [ftypes.h](#).

4.49.2.173 ALSOREVERSE

```
#define ALSOREVERSE 1
```

Definition at line 311 of file [ftypes.h](#).

4.49.2.174 CHISHOLM

```
#define CHISHOLM 2
```

Definition at line 312 of file [ftypes.h](#).

4.49.2.175 NOTRICK

```
#define NOTRICK 16
```

Definition at line 313 of file [ftypes.h](#).

4.49.2.176 SORTLOWFIRST

```
#define SORTLOWFIRST 0
```

Definition at line 315 of file [ftypes.h](#).

4.49.2.177 SORTHIGHFIRST

```
#define SORTHIGHFIRST 1
```

Definition at line 316 of file [ftypes.h](#).

4.49.2.178 SORTPOWERFIRST

```
#define SORTPOWERFIRST 2
```

Definition at line 317 of file [ftypes.h](#).

4.49.2.179 SORTANTIPOWER

```
#define SORTANTIPOWER 3
```

Definition at line 318 of file [ftypes.h](#).

4.49.2.180 STATSSPLITMERGE

```
#define STATSSPLITMERGE 0
```

Definition at line 324 of file [ftypes.h](#).

4.49.2.181 STATSMERGETOFILE

```
#define STATSMERGETOFILE 1
```

Definition at line 325 of file [ftypes.h](#).

4.49.2.182 STATSPOSTSORT

```
#define STATSPOSTSORT 2
```

Definition at line 326 of file [ftypes.h](#).

4.49.2.183 NOCHECKLOGTYPE

```
#define NOCHECKLOGTYPE 0
```

Definition at line 329 of file [ftypes.h](#).

4.49.2.184 CHECKLOGTYPE

```
#define CHECKLOGTYPE 1
```

Definition at line 330 of file [ftypes.h](#).

4.49.2.185 NMIN4SHIFT

```
#define NMIN4SHIFT 4
```

Definition at line 332 of file [ftypes.h](#).

4.49.2.186 SYMBOL

```
#define SYMBOL 1
```

Definition at line 346 of file [ftypes.h](#).

4.49.2.187 DOTPRODUCT

```
#define DOTPRODUCT 2
```

Definition at line 347 of file [ftypes.h](#).

4.49.2.188 VECTOR

```
#define VECTOR 3
```

Definition at line 348 of file [ftypes.h](#).

4.49.2.189 INDEX

```
#define INDEX 4
```

Definition at line 349 of file [ftypes.h](#).

4.49.2.190 EXPRESSION

```
#define EXPRESSION 5
```

Definition at line 350 of file [ftypes.h](#).

4.49.2.191 SUBEXPRESSION

```
#define SUBEXPRESSION 6
```

Definition at line 351 of file [ftypes.h](#).

4.49.2.192 DOLLAREXPRESSION

```
#define DOLLAREXPRESSION 7
```

Definition at line 352 of file [ftypes.h](#).

4.49.2.193 SETSET

```
#define SETSET 8
```

Definition at line 353 of file [ftypes.h](#).

4.49.2.194 ARGWILD

```
#define ARGWILD 9
```

Definition at line 354 of file [ftypes.h](#).

4.49.2.195 MINVECTOR

```
#define MINVECTOR 10
```

Definition at line 355 of file [ftypes.h](#).

4.49.2.196 SETEXP

```
#define SETEXP 11
```

Definition at line 356 of file [ftypes.h](#).

4.49.2.197 DOLLAREXP2

```
#define DOLLAREXP2 12
```

Definition at line 357 of file [ftypes.h](#).

4.49.2.198 HAAKJE0

```
#define HAAKJE0 9
```

Definition at line 358 of file [ftypes.h](#).

4.49.2.199 FUNCTION

```
#define FUNCTION 20
```

Definition at line 359 of file [ftypes.h](#).

4.49.2.200 TMPPOLYFUN

```
#define TMPPOLYFUN 14
```

Definition at line 361 of file [ftypes.h](#).

4.49.2.201 ARGFIELD

```
#define ARGFIELD 15
```

Definition at line 362 of file [ftypes.h](#).

4.49.2.202 SNUMBER

```
#define SNUMBER 16
```

Definition at line 363 of file [ftypes.h](#).

4.49.2.203 LNUMBER

```
#define LNUMBER 17
```

Definition at line 364 of file [ftypes.h](#).

4.49.2.204 HAAKJE

```
#define HAAKJE 18
```

Definition at line 365 of file [ftypes.h](#).

4.49.2.205 DELTA

```
#define DELTA 19
```

Definition at line 366 of file [ftypes.h](#).

4.49.2.206 EXPONENT

```
#define EXPONENT 20
```

Definition at line 367 of file [ftypes.h](#).

4.49.2.207 DENOMINATOR

```
#define DENOMINATOR 21
```

Definition at line 368 of file [ftypes.h](#).

4.49.2.208 SETFUNCTION

```
#define SETFUNCTION 22
```

Definition at line 369 of file [ftypes.h](#).

4.49.2.209 GAMMA

```
#define GAMMA 23
```

Definition at line 370 of file [ftypes.h](#).

4.49.2.210 GAMMAI

```
#define GAMMAI 24
```

Definition at line 371 of file [ftypes.h](#).

4.49.2.211 GAMMAFIVE

```
#define GAMMAFIVE 25
```

Definition at line 372 of file [ftypes.h](#).

4.49.2.212 GAMMASIX

```
#define GAMMASIX 26
```

Definition at line 373 of file [ftypes.h](#).

4.49.2.213 GAMMASEVEN

```
#define GAMMASEVEN 27
```

Definition at line 374 of file [ftypes.h](#).

4.49.2.214 SUMF1

```
#define SUMF1 28
```

Definition at line 375 of file [ftypes.h](#).

4.49.2.215 SUMF2

```
#define SUMF2 29
```

Definition at line 376 of file [ftypes.h](#).

4.49.2.216 DUMFUN

```
#define DUMFUN 30
```

Definition at line 377 of file [ftypes.h](#).

4.49.2.217 REPLACEMENT

```
#define REPLACEMENT 31
```

Definition at line 378 of file [ftypes.h](#).

4.49.2.218 REVERSEFUNCTION

```
#define REVERSEFUNCTION 32
```

Definition at line 379 of file [ftypes.h](#).

4.49.2.219 DISTRIBUTION

```
#define DISTRIBUTION 33
```

Definition at line 380 of file [ftypes.h](#).

4.49.2.220 DELTA3

```
#define DELTA3 34
```

Definition at line 381 of file [ftypes.h](#).

4.49.2.221 DUMMYFUN

```
#define DUMMYFUN 35
```

Definition at line 382 of file [ftypes.h](#).

4.49.2.222 DUMMYTEN

```
#define DUMMYTEN 36
```

Definition at line 383 of file [ftypes.h](#).

4.49.2.223 LEVICIVITA

```
#define LEVICIVITA 37
```

Definition at line 384 of file [ftypes.h](#).

4.49.2.224 FACTORIAL

```
#define FACTORIAL 38
```

Definition at line 385 of file [ftypes.h](#).

4.49.2.225 INVERSEFACTORIAL

```
#define INVERSEFACTORIAL 39
```

Definition at line 386 of file [ftypes.h](#).

4.49.2.226 BINOMIAL

```
#define BINOMIAL 40
```

Definition at line 387 of file [ftypes.h](#).

4.49.2.227 NUMARGSFUN

```
#define NUMARGSFUN 41
```

Definition at line 388 of file [ftypes.h](#).

4.49.2.228 SIGNFUN

```
#define SIGNFUN 42
```

Definition at line 389 of file [ftypes.h](#).

4.49.2.229 MODFUNCTION

```
#define MODFUNCTION 43
```

Definition at line 390 of file [ftypes.h](#).

4.49.2.230 MOD2FUNCTION

```
#define MOD2FUNCTION 44
```

Definition at line 391 of file [ftypes.h](#).

4.49.2.231 MINFUNCTION

```
#define MINFUNCTION 45
```

Definition at line 392 of file [ftypes.h](#).

4.49.2.232 MAXFUNCTION

```
#define MAXFUNCTION 46
```

Definition at line 393 of file [ftypes.h](#).

4.49.2.233 ABSFUNCTION

```
#define ABSFUNCTION 47
```

Definition at line 394 of file [ftypes.h](#).

4.49.2.234 SIGFUNCTION

```
#define SIGFUNCTION 48
```

Definition at line 395 of file [ftypes.h](#).

4.49.2.235 INTFUNCTION

```
#define INTFUNCTION 49
```

Definition at line 396 of file [ftypes.h](#).

4.49.2.236 THETA

```
#define THETA 50
```

Definition at line 397 of file [ftypes.h](#).

4.49.2.237 THETA2

```
#define THETA2 51
```

Definition at line 398 of file [ftypes.h](#).

4.49.2.238 DELTA2

```
#define DELTA2 52
```

Definition at line 399 of file [ftypes.h](#).

4.49.2.239 DELTAP

```
#define DELTAP 53
```

Definition at line 400 of file [ftypes.h](#).

4.49.2.240 BERNOULLIFUNCTION

```
#define BERNOULLIFUNCTION 54
```

Definition at line 401 of file [ftypes.h](#).

4.49.2.241 COUNTFUNCTION

```
#define COUNTFUNCTION 55
```

Definition at line 402 of file [ftypes.h](#).

4.49.2.242 MATCHFUNCTION

```
#define MATCHFUNCTION 56
```

Definition at line 403 of file [ftypes.h](#).

4.49.2.243 PATTERNFUNCTION

```
#define PATTERNFUNCTION 57
```

Definition at line 404 of file [ftypes.h](#).

4.49.2.244 TERMFUNCTION

```
#define TERMFUNCTION 58
```

Definition at line 405 of file [ftypes.h](#).

4.49.2.245 CONJUGATION

```
#define CONJUGATION 59
```

Definition at line 406 of file [ftypes.h](#).

4.49.2.246 ROOTFUNCTION

```
#define ROOTFUNCTION 60
```

Definition at line 407 of file [ftypes.h](#).

4.49.2.247 TABLEFUNCTION

```
#define TABLEFUNCTION 61
```

Definition at line 408 of file [ftypes.h](#).

4.49.2.248 FIRSTBRACKET

```
#define FIRSTBRACKET 62
```

Definition at line 409 of file [ftypes.h](#).

4.49.2.249 TERMSINEXPR

```
#define TERMSINEXPR 63
```

Definition at line 410 of file [ftypes.h](#).

4.49.2.250 NUMTERMSFUN

```
#define NUMTERMSFUN 64
```

Definition at line 411 of file [ftypes.h](#).

4.49.2.251 GCDFUNCTION

```
#define GCDFUNCTION 65
```

Definition at line 412 of file [ftypes.h](#).

4.49.2.252 DIVFUNCTION

```
#define DIVFUNCTION 66
```

Definition at line 413 of file [ftypes.h](#).

4.49.2.253 REMFUNCTION

```
#define REMFUNCTION 67
```

Definition at line 414 of file [ftypes.h](#).

4.49.2.254 MAXPOWEROF

```
#define MAXPOWEROF 68
```

Definition at line 415 of file [ftypes.h](#).

4.49.2.255 MINPOWEROF

```
#define MINPOWEROF 69
```

Definition at line 416 of file [ftypes.h](#).

4.49.2.256 TABLESTUB

```
#define TABLESTUB 70
```

Definition at line 417 of file [ftypes.h](#).

4.49.2.257 FACTORIN

```
#define FACTORIN 71
```

Definition at line 418 of file [ftypes.h](#).

4.49.2.258 TERMSINBRACKET

```
#define TERMSINBRACKET 72
```

Definition at line 419 of file [ftypes.h](#).

4.49.2.259 WILDARGFUN

```
#define WILDARGFUN 73
```

Definition at line [420](#) of file [ftypes.h](#).

4.49.2.260 SQRTFUNCTION

```
#define SQRTFUNCTION 74
```

Definition at line [428](#) of file [ftypes.h](#).

4.49.2.261 LNFUNCTION

```
#define LNFUNCTION 75
```

Definition at line [429](#) of file [ftypes.h](#).

4.49.2.262 SINFUNCTION

```
#define SINFUNCTION 76
```

Definition at line [430](#) of file [ftypes.h](#).

4.49.2.263 COSFUNCTION

```
#define COSFUNCTION 77
```

Definition at line [431](#) of file [ftypes.h](#).

4.49.2.264 TANFUNCTION

```
#define TANFUNCTION 78
```

Definition at line [432](#) of file [ftypes.h](#).

4.49.2.265 ASINFUNCTION

```
#define ASINFUNCTION 79
```

Definition at line [433](#) of file [ftypes.h](#).

4.49.2.266 ACOSFUNCTION

```
#define ACOSFUNCTION 80
```

Definition at line [434](#) of file [ftypes.h](#).

4.49.2.267 ATANFUNCTION

```
#define ATANFUNCTION 81
```

Definition at line 435 of file [ftypes.h](#).

4.49.2.268 ATAN2FUNCTION

```
#define ATAN2FUNCTION 82
```

Definition at line 436 of file [ftypes.h](#).

4.49.2.269 SINHFUNCTION

```
#define SINHFUNCTION 83
```

Definition at line 437 of file [ftypes.h](#).

4.49.2.270 COSHFUNCTION

```
#define COSHFUNCTION 84
```

Definition at line 438 of file [ftypes.h](#).

4.49.2.271 TANHFUNCTION

```
#define TANHFUNCTION 85
```

Definition at line 439 of file [ftypes.h](#).

4.49.2.272 ASINHFUNCTION

```
#define ASINHFUNCTION 86
```

Definition at line 440 of file [ftypes.h](#).

4.49.2.273 ACOSHFUNCTION

```
#define ACOSHFUNCTION 87
```

Definition at line 441 of file [ftypes.h](#).

4.49.2.274 ATANHFUNCTION

```
#define ATANHFUNCTION 88
```

Definition at line 442 of file [ftypes.h](#).

4.49.2.275 LI2FUNCTION

```
#define LI2FUNCTION 89
```

Definition at line 443 of file [ftypes.h](#).

4.49.2.276 LINFUNCTION

```
#define LINFUNCTION 90
```

Definition at line 444 of file [ftypes.h](#).

4.49.2.277 EXTRASYMFUN

```
#define EXTRASYMFUN 91
```

Definition at line 446 of file [ftypes.h](#).

4.49.2.278 RANDOMFUNCTION

```
#define RANDOMFUNCTION 92
```

Definition at line 447 of file [ftypes.h](#).

4.49.2.279 RANPERM

```
#define RANPERM 93
```

Definition at line 448 of file [ftypes.h](#).

4.49.2.280 NUMFACTORS

```
#define NUMFACTORS 94
```

Definition at line 449 of file [ftypes.h](#).

4.49.2.281 FIRSTTERM

```
#define FIRSTTERM 95
```

Definition at line 450 of file [ftypes.h](#).

4.49.2.282 CONTENTTERM

```
#define CONTENTTERM 96
```

Definition at line 451 of file [ftypes.h](#).

4.49.2.283 PRIMENUMBER

```
#define PRIMENUMBER 97
```

Definition at line 452 of file [ftypes.h](#).

4.49.2.284 EXTEUCLIDEAN

```
#define EXTEUCLIDEAN 98
```

Definition at line 453 of file [ftypes.h](#).

4.49.2.285 MAKERATIONAL

```
#define MAKERATIONAL 99
```

Definition at line 454 of file [ftypes.h](#).

4.49.2.286 INVERSEFUNCTION

```
#define INVERSEFUNCTION 100
```

Definition at line 455 of file [ftypes.h](#).

4.49.2.287 IDFUNCTION

```
#define IDFUNCTION 101
```

Definition at line 456 of file [ftypes.h](#).

4.49.2.288 PUTFIRST

```
#define PUTFIRST 102
```

Definition at line 457 of file [ftypes.h](#).

4.49.2.289 PERMUTATIONS

```
#define PERMUTATIONS 103
```

Definition at line 458 of file [ftypes.h](#).

4.49.2.290 PARTITIONS

```
#define PARTITIONS 104
```

Definition at line 459 of file [ftypes.h](#).

4.49.2.291 MULFUNCTION

```
#define MULFUNCTION 105
```

Definition at line 460 of file [ftypes.h](#).

4.49.2.292 MOEBIUS

```
#define MOEBIUS 106
```

Definition at line 461 of file [ftypes.h](#).

4.49.2.293 TOPOLOGIES

```
#define TOPOLOGIES 107
```

Definition at line 462 of file [ftypes.h](#).

4.49.2.294 DIAGRAMS

```
#define DIAGRAMS 108
```

Definition at line 463 of file [ftypes.h](#).

4.49.2.295 TOPO

```
#define TOPO 109
```

Definition at line 464 of file [ftypes.h](#).

4.49.2.296 NODEFUNCTION

```
#define NODEFUNCTION 110
```

Definition at line 465 of file [ftypes.h](#).

4.49.2.297 EDGE

```
#define EDGE 111
```

Definition at line 466 of file [ftypes.h](#).

4.49.2.298 SIZEOFFUNCTION

```
#define SIZEOFFUNCTION 112
```

Definition at line 467 of file [ftypes.h](#).

4.49.2.299 BLOCK

```
#define BLOCK 113
```

Definition at line 468 of file [ftypes.h](#).

4.49.2.300 ONEPI

```
#define ONEPI 114
```

Definition at line 469 of file [ftypes.h](#).

4.49.2.301 PHI

```
#define PHI 115
```

Definition at line 470 of file [ftypes.h](#).

4.49.2.302 MAXBUILTINFUNCTION

```
#define MAXBUILTINFUNCTION 115
```

Definition at line 482 of file [ftypes.h](#).

4.49.2.303 FIRSTUSERFUNCTION

```
#define FIRSTUSERFUNCTION 150
```

Definition at line 485 of file [ftypes.h](#).

4.49.2.304 ALLFUNCTIONS

```
#define ALLFUNCTIONS -1
```

Definition at line 487 of file [ftypes.h](#).

4.49.2.305 ALLMZVFUNCTIONS

```
#define ALLMZVFUNCTIONS -2
```

Definition at line 488 of file [ftypes.h](#).

4.49.2.306 ISYMBOL

```
#define ISYMBOL 0
```

Definition at line 496 of file [ftypes.h](#).

4.49.2.307 PISYMBOL

```
#define PISYMBOL 1
```

Definition at line 497 of file [ftypes.h](#).

4.49.2.308 COEFFSYMBOL

```
#define COEFFSYMBOL 2
```

Definition at line 498 of file [ftypes.h](#).

4.49.2.309 NUMERATORSYMBOL

```
#define NUMERATORSYMBOL 3
```

Definition at line 499 of file [ftypes.h](#).

4.49.2.310 DENOMINATORSYMBOL

```
#define DENOMINATORSYMBOL 4
```

Definition at line 500 of file [ftypes.h](#).

4.49.2.311 WILDARGSYMBOL

```
#define WILDARGSYMBOL 5
```

Definition at line 501 of file [ftypes.h](#).

4.49.2.312 DIMENSIONSYPMBOL

```
#define DIMENSIONSYPMBOL 6
```

Definition at line 502 of file [ftypes.h](#).

4.49.2.313 FACTORSYMBOL

```
#define FACTORSYMBOL 7
```

Definition at line 503 of file [ftypes.h](#).

4.49.2.314 SEPARATESYMBOL

```
#define SEPARATESYMBOL 8
```

Definition at line 504 of file [ftypes.h](#).

4.49.2.315 EESYMBOL

```
#define EESYMBOL 9
```

Definition at line 505 of file [ftypes.h](#).

4.49.2.316 EMSYMBOL

```
#define EMSYMBOL 10
```

Definition at line 506 of file [ftypes.h](#).

4.49.2.317 BUILTINSYMBOLS

```
#define BUILTINSYMBOLS 11
```

Definition at line 508 of file [ftypes.h](#).

4.49.2.318 FIRSTUSERSYMBOL

```
#define FIRSTUSERSYMBOL 20
```

Definition at line 509 of file [ftypes.h](#).

4.49.2.319 BUILTININDICES

```
#define BUILTININDICES 1
```

Definition at line 511 of file [ftypes.h](#).

4.49.2.320 BUILTINVECTORS

```
#define BUILTINVECTORS 1
```

Definition at line 512 of file [ftypes.h](#).

4.49.2.321 BUILTINDOLLARS

```
#define BUILTINDOLLARS 1
```

Definition at line 513 of file [ftypes.h](#).

4.49.2.322 WILDARGVECTOR

```
#define WILDARGVECTOR 0
```

Definition at line 515 of file [ftypes.h](#).

4.49.2.323 WILDARGINDEX

```
#define WILDARGINDEX 0
```

Definition at line 516 of file [ftypes.h](#).

4.49.2.324 TYPEEXPRESSION

```
#define TYPEEXPRESSION 0
```

Definition at line 527 of file [ftypes.h](#).

4.49.2.325 TYPEIDNEW

```
#define TYPEIDNEW 1
```

Definition at line 528 of file [ftypes.h](#).

4.49.2.326 TYPEIDOLD

```
#define TYPEIDOLD 2
```

Definition at line 529 of file [ftypes.h](#).

4.49.2.327 TYPEOPERATION

```
#define TYPEOPERATION 3
```

Definition at line 530 of file [ftypes.h](#).

4.49.2.328 TYPEEREPEAT

```
#define TYPEEREPEAT 4
```

Definition at line 531 of file [ftypes.h](#).

4.49.2.329 TYPEENDREPEAT

```
#define TYPEENDREPEAT 5
```

Definition at line 532 of file [ftypes.h](#).

4.49.2.330 TYPECOUNT

```
#define TYPECOUNT 20
```

Definition at line 536 of file [ftypes.h](#).

4.49.2.331 TYPEMULT

```
#define TYPEMULT 21
```

Definition at line 537 of file [ftypes.h](#).

4.49.2.332 TYPEGOTO

```
#define TYPEGOTO 22
```

Definition at line 538 of file [ftypes.h](#).

4.49.2.333 TYPEDISCARD

```
#define TYPEDISCARD 23
```

Definition at line 539 of file [ftypes.h](#).

4.49.2.334 TYPEIF

```
#define TYPEIF 24
```

Definition at line 540 of file [ftypes.h](#).

4.49.2.335 TYPEELSE

```
#define TYPEELSE 25
```

Definition at line 541 of file [ftypes.h](#).

4.49.2.336 TYPEELIF

```
#define TYPEELIF 26
```

Definition at line 542 of file [ftypes.h](#).

4.49.2.337 TYPEENDIF

```
#define TYPEENDIF 27
```

Definition at line 543 of file [ftypes.h](#).

4.49.2.338 TYPESUM

```
#define TYPESUM 28
```

Definition at line 544 of file [ftypes.h](#).

4.49.2.339 TYPECHISHOLM

```
#define TYPECHISHOLM 29
```

Definition at line 545 of file [ftypes.h](#).

4.49.2.340 TYPEREVERSE

```
#define TYPEREVERSE 30
```

Definition at line 546 of file [ftypes.h](#).

4.49.2.341 TYPEARG

```
#define TYPEARG 31
```

Definition at line 547 of file [ftypes.h](#).

4.49.2.342 TYPENORM

```
#define TYPENORM 32
```

Definition at line 548 of file [ftypes.h](#).

4.49.2.343 TYPENORM2

```
#define TYPENORM2 33
```

Definition at line 549 of file [ftypes.h](#).

4.49.2.344 TYPENORM3

```
#define TYPENORM3 34
```

Definition at line 550 of file [ftypes.h](#).

4.49.2.345 TYPEEXIT

```
#define TYPEEXIT 35
```

Definition at line 551 of file [ftypes.h](#).

4.49.2.346 TYPESETEXIT

```
#define TYPESETEXIT 36
```

Definition at line 552 of file [ftypes.h](#).

4.49.2.347 TYPEPRINT

```
#define TYPEPRINT 37
```

Definition at line 553 of file [ftypes.h](#).

4.49.2.348 TYPEFPRINT

```
#define TYPEFPRINT 38
```

Definition at line 554 of file [ftypes.h](#).

4.49.2.349 TYPEREDEFPRE

```
#define TYPEREDEFPRE 39
```

Definition at line 555 of file [ftypes.h](#).

4.49.2.350 TYPESPLITARG

```
#define TYPESPLITARG 40
```

Definition at line 556 of file [ftypes.h](#).

4.49.2.351 TYPESPLITARG2

```
#define TYPESPLITARG2 41
```

Definition at line 557 of file [ftypes.h](#).

4.49.2.352 TYPEFACTARG

```
#define TYPEFACTARG 42
```

Definition at line 558 of file [ftypes.h](#).

4.49.2.353 TYPEFACTARG2

```
#define TYPEFACTARG2 43
```

Definition at line 559 of file [ftypes.h](#).

4.49.2.354 TYPETRY

```
#define TYPETRY 44
```

Definition at line 560 of file [ftypes.h](#).

4.49.2.355 TYPEASSIGN

```
#define TYPEASSIGN 45
```

Definition at line [561](#) of file [ftypes.h](#).

4.49.2.356 TYPERENUMBER

```
#define TYPERENUMBER 46
```

Definition at line [562](#) of file [ftypes.h](#).

4.49.2.357 TYPESUMFIX

```
#define TYPESUMFIX 47
```

Definition at line [563](#) of file [ftypes.h](#).

4.49.2.358 TYPEFINDLOOP

```
#define TYPEFINDLOOP 48
```

Definition at line [564](#) of file [ftypes.h](#).

4.49.2.359 TYPEUNRAVEL

```
#define TYPEUNRAVEL 49
```

Definition at line [565](#) of file [ftypes.h](#).

4.49.2.360 TYPEADJUSTBOUNDS

```
#define TYPEADJUSTBOUNDS 50
```

Definition at line [566](#) of file [ftypes.h](#).

4.49.2.361 TYPEINSIDE

```
#define TYPEINSIDE 51
```

Definition at line [567](#) of file [ftypes.h](#).

4.49.2.362 TYPETERM

```
#define TYPETERM 52
```

Definition at line [568](#) of file [ftypes.h](#).

4.49.2.363 TYPESORT

```
#define TYPESORT 53
```

Definition at line 569 of file [ftypes.h](#).

4.49.2.364 TYPEDETCURDUM

```
#define TYPEDETCURDUM 54
```

Definition at line 570 of file [ftypes.h](#).

4.49.2.365 TYPEINEXPRESSION

```
#define TYPEINEXPRESSION 55
```

Definition at line 571 of file [ftypes.h](#).

4.49.2.366 TYPESPLITFIRSTARG

```
#define TYPESPLITFIRSTARG 56
```

Definition at line 572 of file [ftypes.h](#).

4.49.2.367 TYPESPLITLASTARG

```
#define TYPESPLITLASTARG 57
```

Definition at line 573 of file [ftypes.h](#).

4.49.2.368 TYPEMERGE

```
#define TYPEMERGE 58
```

Definition at line 574 of file [ftypes.h](#).

4.49.2.369 TYPETESTUSE

```
#define TYPETESTUSE 59
```

Definition at line 575 of file [ftypes.h](#).

4.49.2.370 TYPEAPPLY

```
#define TYPEAPPLY 60
```

Definition at line 576 of file [ftypes.h](#).

4.49.2.371 TYPEAPPLYRESET

```
#define TYPEAPPLYRESET 61
```

Definition at line 577 of file [ftypes.h](#).

4.49.2.372 TYPECHAININ

```
#define TYPECHAININ 62
```

Definition at line 578 of file [ftypes.h](#).

4.49.2.373 TYPECHAINOUT

```
#define TYPECHAINOUT 63
```

Definition at line 579 of file [ftypes.h](#).

4.49.2.374 TYPENORM4

```
#define TYPENORM4 64
```

Definition at line 580 of file [ftypes.h](#).

4.49.2.375 TYPEFACTOR

```
#define TYPEFACTOR 65
```

Definition at line 581 of file [ftypes.h](#).

4.49.2.376 TYPEARGIMPLODE

```
#define TYPEARGIMPLODE 66
```

Definition at line 582 of file [ftypes.h](#).

4.49.2.377 TYPEARGEXPLODE

```
#define TYPEARGEXPLODE 67
```

Definition at line 583 of file [ftypes.h](#).

4.49.2.378 TYPEDENOMINATORS

```
#define TYPEDENOMINATORS 68
```

Definition at line 584 of file [ftypes.h](#).

4.49.2.379 TYPESTUFFLE

```
#define TYPESTUFFLE 69
```

Definition at line 585 of file [ftypes.h](#).

4.49.2.380 TYPEDROPCOEFFICIENT

```
#define TYPEDROPCOEFFICIENT 70
```

Definition at line 586 of file [ftypes.h](#).

4.49.2.381 TYPETRANSFORM

```
#define TYPETRANSFORM 71
```

Definition at line 587 of file [ftypes.h](#).

4.49.2.382 TYPETOPOLYNOMIAL

```
#define TYPETOPOLYNOMIAL 72
```

Definition at line 588 of file [ftypes.h](#).

4.49.2.383 TYPEFROMPOLYNOMIAL

```
#define TYPEFROMPOLYNOMIAL 73
```

Definition at line 589 of file [ftypes.h](#).

4.49.2.384 TYPEDOLOOP

```
#define TYPEDOLOOP 74
```

Definition at line 590 of file [ftypes.h](#).

4.49.2.385 TYPEENDDOLOOP

```
#define TYPEENDDOLOOP 75
```

Definition at line 591 of file [ftypes.h](#).

4.49.2.386 TYPEDROPSYMBOLS

```
#define TYPEDROPSYMBOLS 76
```

Definition at line 592 of file [ftypes.h](#).

4.49.2.387 TYPEPUTINSIDE

```
#define TYPEPUTINSIDE 77
```

Definition at line 593 of file [ftypes.h](#).

4.49.2.388 TYPETOSPECTATOR

```
#define TYPETOSPECTATOR 78
```

Definition at line 594 of file [ftypes.h](#).

4.49.2.389 TYPEARGTOEXTRASymbol

```
#define TYPEARGTOEXTRASymbol 79
```

Definition at line 595 of file [ftypes.h](#).

4.49.2.390 TYPECANONICALIZE

```
#define TYPECANONICALIZE 80
```

Definition at line 596 of file [ftypes.h](#).

4.49.2.391 TYPESWITCH

```
#define TYPESWITCH 81
```

Definition at line 597 of file [ftypes.h](#).

4.49.2.392 TYPEENDSWITCH

```
#define TYPEENDSWITCH 82
```

Definition at line 598 of file [ftypes.h](#).

4.49.2.393 TYPESETUSERFLAG

```
#define TYPESETUSERFLAG 83
```

Definition at line 599 of file [ftypes.h](#).

4.49.2.394 TYPECLEARUSERFLAG

```
#define TYPECLEARUSERFLAG 84
```

Definition at line 600 of file [ftypes.h](#).

4.49.2.395 TYPEALLLOOPS

```
#define TYPEALLLOOPS 85
```

Definition at line 601 of file [ftypes.h](#).

4.49.2.396 TYPEALLPATHS

```
#define TYPEALLPATHS 86
```

Definition at line 602 of file [ftypes.h](#).

4.49.2.397 TAKETRACE

```
#define TAKETRACE 1
```

Definition at line 614 of file [ftypes.h](#).

4.49.2.398 CONTRACT

```
#define CONTRACT 2
```

Definition at line 615 of file [ftypes.h](#).

4.49.2.399 RATIO

```
#define RATIO 3
```

Definition at line 616 of file [ftypes.h](#).

4.49.2.400 SYMMETRIZE

```
#define SYMMETRIZE 4
```

Definition at line 617 of file [ftypes.h](#).

4.49.2.401 TENVEC

```
#define TENVEC 5
```

Definition at line 618 of file [ftypes.h](#).

4.49.2.402 SUMNUM1

```
#define SUMNUM1 6
```

Definition at line 619 of file [ftypes.h](#).

4.49.2.403 SUMNUM2

```
#define SUMNUM2 7
```

Definition at line [620](#) of file [ftypes.h](#).

4.49.2.404 WILDDUMMY

```
#define WILDDUMMY 0
```

Definition at line [626](#) of file [ftypes.h](#).

4.49.2.405 SYMTONUM

```
#define SYMTONUM 1
```

Definition at line [627](#) of file [ftypes.h](#).

4.49.2.406 SYMTOSYM

```
#define SYMTOSYM 2
```

Definition at line [628](#) of file [ftypes.h](#).

4.49.2.407 SYMTOSUB

```
#define SYMTOSUB 3
```

Definition at line [629](#) of file [ftypes.h](#).

4.49.2.408 VECTOMIN

```
#define VECTOMIN 4
```

Definition at line [630](#) of file [ftypes.h](#).

4.49.2.409 VECTOVEC

```
#define VECTOVEC 5
```

Definition at line [631](#) of file [ftypes.h](#).

4.49.2.410 VECTOSUB

```
#define VECTOSUB 6
```

Definition at line [632](#) of file [ftypes.h](#).

4.49.2.411 INDTOIND

```
#define INDTOIND 7
```

Definition at line 633 of file [ftypes.h](#).

4.49.2.412 INDTOSUB

```
#define INDTOSUB 8
```

Definition at line 634 of file [ftypes.h](#).

4.49.2.413 FUNTOFUN

```
#define FUNTOFUN 9
```

Definition at line 635 of file [ftypes.h](#).

4.49.2.414 ARGTOARG

```
#define ARGTOARG 10
```

Definition at line 636 of file [ftypes.h](#).

4.49.2.415 ARLTOARL

```
#define ARLTOARL 11
```

Definition at line 637 of file [ftypes.h](#).

4.49.2.416 EXPTOEXP

```
#define EXPTOEXP 12
```

Definition at line 638 of file [ftypes.h](#).

4.49.2.417 FROMBRAC

```
#define FROMBRAC 13
```

Definition at line 639 of file [ftypes.h](#).

4.49.2.418 FROMSET

```
#define FROMSET 14
```

Definition at line 640 of file [ftypes.h](#).

4.49.2.419 SETTONUM

```
#define SETTONUM 15
```

Definition at line 641 of file [ftypes.h](#).

4.49.2.420 WILDCARDS

```
#define WILDCARDS 16
```

Definition at line 642 of file [ftypes.h](#).

4.49.2.421 SETNUMBER

```
#define SETNUMBER 17
```

Definition at line 643 of file [ftypes.h](#).

4.49.2.422 LOADDOLLAR

```
#define LOADDOLLAR 18
```

Definition at line 644 of file [ftypes.h](#).

4.49.2.423 NUMTONUM

```
#define NUMTONUM 20
```

Definition at line 648 of file [ftypes.h](#).

4.49.2.424 NUMTOSYM

```
#define NUMTOSYM 21
```

Definition at line 649 of file [ftypes.h](#).

4.49.2.425 NUMTOIND

```
#define NUMTOIND 22
```

Definition at line 650 of file [ftypes.h](#).

4.49.2.426 NUMTOSUB

```
#define NUMTOSUB 23
```

Definition at line 651 of file [ftypes.h](#).

4.49.2.427 EATTENSOR

```
#define EATTENSOR 0x2000
```

Definition at line 655 of file [ftypes.h](#).

4.49.2.428 CLEANFLAG

```
#define CLEANFLAG 0
```

Definition at line 661 of file [ftypes.h](#).

4.49.2.429 DIRTYFLAG

```
#define DIRTYFLAG 1
```

Definition at line 662 of file [ftypes.h](#).

4.49.2.430 DIRTSYMFLAG

```
#define DIRTSYMFLAG 2
```

Definition at line 663 of file [ftypes.h](#).

4.49.2.431 MUSTCLEANPRF

```
#define MUSTCLEANPRF 4
```

Definition at line 664 of file [ftypes.h](#).

4.49.2.432 SUBTERMUSED1

```
#define SUBTERMUSED1 8
```

Definition at line 665 of file [ftypes.h](#).

4.49.2.433 SUBTERMUSED2

```
#define SUBTERMUSED2 16
```

Definition at line 666 of file [ftypes.h](#).

4.49.2.434 ALLDIRTY

```
#define ALLDIRTY (DIRTYFLAG|DIRTSYMFLAG)
```

Definition at line 667 of file [ftypes.h](#).

4.49.2.435 ARGHEAD

```
#define ARGHEAD 2
```

Definition at line 669 of file [ftypes.h](#).

4.49.2.436 FUNHEAD

```
#define FUNHEAD 3
```

Definition at line 670 of file [ftypes.h](#).

4.49.2.437 SUBEXPSIZE

```
#define SUBEXPSIZE 5
```

Definition at line 671 of file [ftypes.h](#).

4.49.2.438 EXPRHEAD

```
#define EXPRHEAD 5
```

Definition at line 672 of file [ftypes.h](#).

4.49.2.439 TYPEARGHEADSIZE

```
#define TYPEARGHEADSIZE 6
```

Definition at line 673 of file [ftypes.h](#).

4.49.2.440 SKIP

```
#define SKIP 1
```

Definition at line 680 of file [ftypes.h](#).

4.49.2.441 DROP

```
#define DROP 2
```

Definition at line 681 of file [ftypes.h](#).

4.49.2.442 HIDE

```
#define HIDE 3
```

Definition at line 682 of file [ftypes.h](#).

4.49.2.443 UNHIDE

```
#define UNHIDE 4
```

Definition at line 683 of file [ftypes.h](#).

4.49.2.444 INTOHIDE

```
#define INTOHIDE 5
```

Definition at line 684 of file [ftypes.h](#).

4.49.2.445 LOCALEXPRESSION

```
#define LOCALEXPRESSION 0
```

Definition at line 690 of file [ftypes.h](#).

4.49.2.446 SKIPLEXPRESSION

```
#define SKIPLEXPRESSION 1
```

Definition at line 691 of file [ftypes.h](#).

4.49.2.447 DROPLEXPRESSION

```
#define DROPLEXPRESSION 2
```

Definition at line 692 of file [ftypes.h](#).

4.49.2.448 DROPEDEXPRESSION

```
#define DROPEDEXPRESSION 3
```

Definition at line 693 of file [ftypes.h](#).

4.49.2.449 GLOBALEXPRESSION

```
#define GLOBALEXPRESSION 4
```

Definition at line 694 of file [ftypes.h](#).

4.49.2.450 SKIPGEXPRESSION

```
#define SKIPGEXPRESSION 5
```

Definition at line 695 of file [ftypes.h](#).

4.49.2.451 DROPGEXPRESSION

```
#define DROPGEXPRESSION 6
```

Definition at line 696 of file [ftypes.h](#).

4.49.2.452 STOREDEXPRESSION

```
#define STOREDEXPRESSION 8
```

Definition at line 697 of file [ftypes.h](#).

4.49.2.453 HIDDENLEXPRESSION

```
#define HIDDENLEXPRESSION 9
```

Definition at line 698 of file [ftypes.h](#).

4.49.2.454 HIDDENGEXPRESSION

```
#define HIDDENGEXPRESSION 13
```

Definition at line 699 of file [ftypes.h](#).

4.49.2.455 INCEXPRESSION

```
#define INCEXPRESSION 9
```

Definition at line 700 of file [ftypes.h](#).

4.49.2.456 HIDELEXPRESSION

```
#define HIDELEXPRESSION 10
```

Definition at line 701 of file [ftypes.h](#).

4.49.2.457 HIDEGEXPRESSION

```
#define HIDEGEXPRESSION 14
```

Definition at line 702 of file [ftypes.h](#).

4.49.2.458 DROPHLEXPRESSION

```
#define DROPHLEXPRESSION 11
```

Definition at line 703 of file [ftypes.h](#).

4.49.2.459 DROPHGEXPRESSION

```
#define DROPHGEXPRESSION 15
```

Definition at line 704 of file [ftypes.h](#).

4.49.2.460 UNHIDELEXPRESSION

```
#define UNHIDELEXPRESSION 12
```

Definition at line 705 of file [ftypes.h](#).

4.49.2.461 UNHIDEGEXPRESSION

```
#define UNHIDEGEXPRESSION 16
```

Definition at line 706 of file [ftypes.h](#).

4.49.2.462 INTOHIDELEXPRESSION

```
#define INTOHIDELEXPRESSION 17
```

Definition at line 707 of file [ftypes.h](#).

4.49.2.463 INTOHIDEGEXPRESSION

```
#define INTOHIDEGEXPRESSION 18
```

Definition at line 708 of file [ftypes.h](#).

4.49.2.464 SPECTATOREXPRESSION

```
#define SPECTATOREXPRESSION 19
```

Definition at line 709 of file [ftypes.h](#).

4.49.2.465 DROPSPECTATOREXPRESSION

```
#define DROPSPECTATOREXPRESSION 20
```

Definition at line 710 of file [ftypes.h](#).

4.49.2.466 SKIPUNHIDELEXPRESSION

```
#define SKIPUNHIDELEXPRESSION 21
```

Definition at line 711 of file [ftypes.h](#).

4.49.2.467 SKIPUNHIDEGEXPRESSION

```
#define SKIPUNHIDEGEXPRESSION 22
```

Definition at line 712 of file [ftypes.h](#).

4.49.2.468 PRINTOFF

```
#define PRINTOFF 0
```

Definition at line 714 of file [ftypes.h](#).

4.49.2.469 PRINTON

```
#define PRINTON 1
```

Definition at line 715 of file [ftypes.h](#).

4.49.2.470 PRINTCONTENTS

```
#define PRINTCONTENTS 2
```

Definition at line 716 of file [ftypes.h](#).

4.49.2.471 PRINTCONTENT

```
#define PRINTCONTENT 3
```

Definition at line 717 of file [ftypes.h](#).

4.49.2.472 PRINTLFILE

```
#define PRINTLFILE 4
```

Definition at line 718 of file [ftypes.h](#).

4.49.2.473 PRINTONETERM

```
#define PRINTONETERM 8
```

Definition at line 719 of file [ftypes.h](#).

4.49.2.474 PRINTONEFUNCTION

```
#define PRINTONEFUNCTION 16
```

Definition at line 720 of file [ftypes.h](#).

4.49.2.475 PRINTALL

```
#define PRINTALL 32
```

Definition at line 721 of file [ftypes.h](#).

4.49.2.476 REGULAREXPRESSION

```
#define REGULAREXPRESSION -1
```

Definition at line 727 of file [ftypes.h](#).

4.49.2.477 REDEFINEDEXPRESSION

```
#define REDEFINEDEXPRESSION -2
```

Definition at line 728 of file [ftypes.h](#).

4.49.2.478 NEWLYDEFINEDEXPRESSION

```
#define NEWLYDEFINEDEXPRESSION -3
```

Definition at line 729 of file [ftypes.h](#).

4.49.2.479 VERTEXFUNCTION

```
#define VERTEXFUNCTION -1
```

Definition at line 738 of file [ftypes.h](#).

4.49.2.480 GENERALFUNCTION

```
#define GENERALFUNCTION 0
```

Definition at line 739 of file [ftypes.h](#).

4.49.2.481 TENSORFUNCTION

```
#define TENSORFUNCTION 2
```

Definition at line 740 of file [ftypes.h](#).

4.49.2.482 GAMMAFUNCTION

```
#define GAMMAFUNCTION 4
```

Definition at line 741 of file [ftypes.h](#).

4.49.2.483 POS_

```
#define POS_ 0 /* integer > 0 */
```

Definition at line 748 of file [ftypes.h](#).

4.49.2.484 POS0_

```
#define POS0_ 1 /* integer >= 0 */
```

Definition at line 749 of file [ftypes.h](#).

4.49.2.485 NEG_

```
#define NEG_ 2 /* integer < 0 */
```

Definition at line 750 of file [ftypes.h](#).

4.49.2.486 NEG0_

```
#define NEG0_ 3 /* integer <= 0 */
```

Definition at line 751 of file [ftypes.h](#).

4.49.2.487 EVEN_

```
#define EVEN_ 4 /* integer (even) */
```

Definition at line 752 of file [ftypes.h](#).

4.49.2.488 ODD_

```
#define ODD_ 5 /* integer (odd) */
```

Definition at line 753 of file [ftypes.h](#).

4.49.2.489 Z_

```
#define Z_ 6 /* integer */
```

Definition at line 754 of file [ftypes.h](#).

4.49.2.490 SYMBOL_

```
#define SYMBOL_ 7 /* symbol only */
```

Definition at line 755 of file [ftypes.h](#).

4.49.2.491 FIXED_

```
#define FIXED_ 8 /* fixed index */
```

Definition at line 756 of file [ftypes.h](#).

4.49.2.492 INDEX_

```
#define INDEX_ 9 /* index only */
```

Definition at line 757 of file [ftypes.h](#).

4.49.2.493 Q_

```
#define Q_ 10 /* rational */
```

Definition at line 758 of file [ftypes.h](#).

4.49.2.494 DUMMYINDEX_

```
#define DUMMYINDEX_ 11 /* dummy index only */
```

Definition at line 759 of file [ftypes.h](#).

4.49.2.495 VECTOR_

```
#define VECTOR_ 12 /* vector only */
```

Definition at line 760 of file [ftypes.h](#).

4.49.2.496 GAMMA1

```
#define GAMMA1 0
```

Definition at line 766 of file [ftypes.h](#).

4.49.2.497 GAMMA5

```
#define GAMMA5 -1
```

Definition at line 767 of file [ftypes.h](#).

4.49.2.498 GAMMA6

```
#define GAMMA6 -2
```

Definition at line 768 of file [ftypes.h](#).

4.49.2.499 GAMMA7

```
#define GAMMA7 -3
```

Definition at line 769 of file [ftypes.h](#).

4.49.2.500 FUNNYVEC

```
#define FUNNYVEC -4
```

Definition at line 770 of file [ftypes.h](#).

4.49.2.501 FUNNYWILD

```
#define FUNNYWILD -5
```

Definition at line 771 of file [ftypes.h](#).

4.49.2.502 SUMMEDIND

```
#define SUMMEDIND -6
```

Definition at line 772 of file [ftypes.h](#).

4.49.2.503 NOINDEX

```
#define NOINDEX -7
```

Definition at line 773 of file [ftypes.h](#).

4.49.2.504 FUNNYDOLLAR

```
#define FUNNYDOLLAR -8
```

Definition at line 774 of file [ftypes.h](#).

4.49.2.505 EMPTYINDEX

```
#define EMPTYINDEX -9
```

Definition at line 775 of file [ftypes.h](#).

4.49.2.506 MINSPEC

```
#define MINSPEC -10
```

Definition at line 781 of file [ftypes.h](#).

4.49.2.507 USEDFLAG

```
#define USEDFLAG 2
```

Definition at line 783 of file [ftypes.h](#).

4.49.2.508 DUMMYFLAG

```
#define DUMMYFLAG 1
```

Definition at line 784 of file [ftypes.h](#).

4.49.2.509 MAINSORT

```
#define MAINSORT 0
```

Definition at line 786 of file [ftypes.h](#).

4.49.2.510 FUNCTIONSORT

```
#define FUNCTIONSORT 1
```

Definition at line 787 of file [ftypes.h](#).

4.49.2.511 SUBSORT

```
#define SUBSORT 2
```

Definition at line 788 of file [ftypes.h](#).

4.49.2.512 FLOATMODE

```
#define FLOATMODE 1
```

Definition at line 790 of file [ftypes.h](#).

4.49.2.513 RATIONALMODE

```
#define RATIONALMODE 0
```

Definition at line 791 of file [ftypes.h](#).

4.49.2.514 NUMSPECSETS

```
#define NUMSPECSETS 10
```

Definition at line 793 of file [ftypes.h](#).

4.49.2.515 ISZERO

```
#define ISZERO 1
```

Definition at line 795 of file [ftypes.h](#).

4.49.2.516 ISUNMODIFIED

```
#define ISUNMODIFIED 2
```

Definition at line 796 of file [ftypes.h](#).

4.49.2.517 ISCOMPRESSED

```
#define ISCOMPRESSED 4
```

Definition at line 797 of file [ftypes.h](#).

4.49.2.518 ISINRHS

```
#define ISINRHS 8
```

Definition at line 798 of file [ftypes.h](#).

4.49.2.519 ISFACTORIZED

```
#define ISFACTORIZED 16
```

Definition at line 799 of file [ftypes.h](#).

4.49.2.520 TOBEFACTORED

```
#define TOBEFACTORED 32
```

Definition at line 800 of file [ftypes.h](#).

4.49.2.521 TOBEUNFACTORED

```
#define TOBEUNFACTORED 64
```

Definition at line 801 of file [ftypes.h](#).

4.49.2.522 KEEPZERO

```
#define KEEPZERO 128
```

Definition at line 802 of file [ftypes.h](#).

4.49.2.523 VARTYPENONE

```
#define VARTYPENONE 0
```

Definition at line 804 of file [ftypes.h](#).

4.49.2.524 VARTYPECOMPLEX

```
#define VARTYPECOMPLEX 1
```

Definition at line 805 of file [ftypes.h](#).

4.49.2.525 VARTYPEIMAGINARY

```
#define VARTYPEIMAGINARY 2
```

Definition at line 806 of file [ftypes.h](#).

4.49.2.526 VARTYPEROOTOFUNITY

```
#define VARTYPEROOTOFUNITY 4
```

Definition at line 807 of file [ftypes.h](#).

4.49.2.527 VARTYPEMINUS

```
#define VARTYPEMINUS 8
```

Definition at line 808 of file [ftypes.h](#).

4.49.2.528 CYCLESYMMETRIC

```
#define CYCLESYMMETRIC 1
```

Definition at line 809 of file [ftypes.h](#).

4.49.2.529 RCYCLESYMMETRIC

```
#define RCYCLESYMMETRIC 2
```

Definition at line 810 of file [ftypes.h](#).

4.49.2.530 SYMMETRIC

```
#define SYMMETRIC 3
```

Definition at line 811 of file [ftypes.h](#).

4.49.2.531 ANTISYMMETRIC

```
#define ANTISYMMETRIC 4
```

Definition at line 812 of file [ftypes.h](#).

4.49.2.532 REVERSEORDER

```
#define REVERSEORDER 256
```

Definition at line 813 of file [ftypes.h](#).

4.49.2.533 SUBMULTI

```
#define SUBMULTI 1
```

Definition at line 819 of file [ftypes.h](#).

4.49.2.534 SUBONCE

```
#define SUBONCE 2
```

Definition at line 820 of file [ftypes.h](#).

4.49.2.535 SUBONLY

```
#define SUBONLY 3
```

Definition at line 821 of file [ftypes.h](#).

4.49.2.536 SUBMANY

```
#define SUBMANY 4
```

Definition at line 822 of file [ftypes.h](#).

4.49.2.537 SUBVECTOR

```
#define SUBVECTOR 5
```

Definition at line 823 of file [ftypes.h](#).

4.49.2.538 SUBSELECT

```
#define SUBSELECT 6
```

Definition at line 824 of file [ftypes.h](#).

4.49.2.539 SUBALL

```
#define SUBALL 7
```

Definition at line 825 of file [ftypes.h](#).

4.49.2.540 SUBMASK

```
#define SUBMASK 15
```

Definition at line 826 of file [ftypes.h](#).

4.49.2.541 SUBDISORDER

```
#define SUBDISORDER 16
```

Definition at line 827 of file [ftypes.h](#).

4.49.2.542 SUBAFTER

```
#define SUBAFTER 32
```

Definition at line 828 of file [ftypes.h](#).

4.49.2.543 SUBAFTERNOT

```
#define SUBAFTERNOT 64
```

Definition at line 829 of file [ftypes.h](#).

4.49.2.544 IDHEAD

```
#define IDHEAD 6
```

Definition at line 831 of file [ftypes.h](#).

4.49.2.545 DOLLARFLAG

```
#define DOLLARFLAG 1
```

Definition at line 833 of file [ftypes.h](#).

4.49.2.546 NORMALIZEFLAG

```
#define NORMALIZEFLAG 2
```

Definition at line 834 of file [ftypes.h](#).

4.49.2.547 GIDENT

```
#define GIDENT 1
```

Definition at line 836 of file [ftypes.h](#).

4.49.2.548 GFIVE

```
#define GFIVE 4
```

Definition at line 837 of file [ftypes.h](#).

4.49.2.549 GPLUS

```
#define GPLUS 3
```

Definition at line 838 of file [ftypes.h](#).

4.49.2.550 GMINUS

```
#define GMINUS 2
```

Definition at line 839 of file [ftypes.h](#).

4.49.2.551 LONGNUMBER

```
#define LONGNUMBER 1
```

Definition at line 845 of file [ftypes.h](#).

4.49.2.552 MATCH

```
#define MATCH 2
```

Definition at line 846 of file [ftypes.h](#).

4.49.2.553 COEFFI

```
#define COEFFI 3
```

Definition at line 847 of file [ftypes.h](#).

4.49.2.554 SUBEXPR

```
#define SUBEXPR 4
```

Definition at line 848 of file [ftypes.h](#).

4.49.2.555 MULTIPLEOF

```
#define MULTIPLEOF 5
```

Definition at line 849 of file [ftypes.h](#).

4.49.2.556 IFDOLLAR

```
#define IFDOLLAR 6
```

Definition at line 850 of file [ftypes.h](#).

4.49.2.557 IFEXPRESSION

```
#define IFEXPRESSION 7
```

Definition at line 851 of file [ftypes.h](#).

4.49.2.558 IFDOLLAREXTRA

```
#define IFDOLLAREXTRA 8
```

Definition at line 852 of file [ftypes.h](#).

4.49.2.559 IFISFACTORIZED

```
#define IFISFACTORIZED 9
```

Definition at line 853 of file [ftypes.h](#).

4.49.2.560 IFOCCURS

```
#define IFOCCURS 10
```

Definition at line 854 of file [ftypes.h](#).

4.49.2.561 IFUSERFLAG

```
#define IFUSERFLAG 11
```

Definition at line 855 of file [ftypes.h](#).

4.49.2.562 IFFLOATNUMBER

```
#define IFFLOATNUMBER 12
```

Definition at line 856 of file [ftypes.h](#).

4.49.2.563 GREATER

```
#define GREATER 0
```

Definition at line 857 of file [ftypes.h](#).

4.49.2.564 GREATEREQUAL

```
#define GREATEREQUAL 1
```

Definition at line 858 of file [ftypes.h](#).

4.49.2.565 LESS

```
#define LESS 2
```

Definition at line 859 of file [ftypes.h](#).

4.49.2.566 LESSEQUAL

```
#define LESSEQUAL 3
```

Definition at line 860 of file [ftypes.h](#).

4.49.2.567 EQUAL

```
#define EQUAL 4
```

Definition at line 861 of file [ftypes.h](#).

4.49.2.568 NOTEQUAL

```
#define NOTEQUAL 5
```

Definition at line 862 of file [ftypes.h](#).

4.49.2.569 ORCOND

```
#define ORCOND 6
```

Definition at line 863 of file [ftypes.h](#).

4.49.2.570 ANDCOND

```
#define ANDCOND 7
```

Definition at line 864 of file [ftypes.h](#).

4.49.2.571 DUMMY

```
#define DUMMY 1
```

Definition at line 865 of file [ftypes.h](#).

4.49.2.572 SORT

```
#define SORT 1
```

Definition at line 866 of file [ftypes.h](#).

4.49.2.573 STORE

```
#define STORE 2
```

Definition at line 867 of file [ftypes.h](#).

4.49.2.574 END

```
#define END 3
```

Definition at line 868 of file [ftypes.h](#).

4.49.2.575 GLOBAL

```
#define GLOBAL 4
```

Definition at line 869 of file [ftypes.h](#).

4.49.2.576 CLEAR

```
#define CLEAR 5
```

Definition at line 870 of file [ftypes.h](#).

4.49.2.577 VECTBIT

```
#define VECTBIT 1
```

Definition at line 872 of file [ftypes.h](#).

4.49.2.578 DOTPBIT

```
#define DOTPBIT 2
```

Definition at line 873 of file [ftypes.h](#).

4.49.2.579 FUNBIT

```
#define FUNBIT 4
```

Definition at line 874 of file [ftypes.h](#).

4.49.2.580 SETBIT

```
#define SETBIT 8
```

Definition at line 875 of file [ftypes.h](#).

4.49.2.581 EXTRAPARAMETER

```
#define EXTRAPARAMETER 0x4000
```

Definition at line 877 of file [ftypes.h](#).

4.49.2.582 GENCOMMUTE

```
#define GENCOMMUTE 0
```

Definition at line 878 of file [ftypes.h](#).

4.49.2.583 GENNONCOMMUTE

```
#define GENNONCOMMUTE 0x2000
```

Definition at line 879 of file [ftypes.h](#).

4.49.2.584 NAMENOTFOUND

```
#define NAMENOTFOUND -9
```

Definition at line 881 of file [ftypes.h](#).

4.49.2.585 DOLUNDEFINED

```
#define DOLUNDEFINED 0
```

Definition at line 887 of file [ftypes.h](#).

4.49.2.586 DOLNUMBER

```
#define DOLNUMBER 1
```

Definition at line 888 of file [ftypes.h](#).

4.49.2.587 DOLARGUMENT

```
#define DOLARGUMENT 2
```

Definition at line 889 of file [ftypes.h](#).

4.49.2.588 DOLSUBTERM

```
#define DOLSUBTERM 3
```

Definition at line 890 of file [ftypes.h](#).

4.49.2.589 DOLTERMS

```
#define DOLTERMS 4
```

Definition at line 891 of file [ftypes.h](#).

4.49.2.590 DOLWILDARGS

```
#define DOLWILDARGS 5
```

Definition at line 892 of file [ftypes.h](#).

4.49.2.591 DOLINDEX

```
#define DOLINDEX 6
```

Definition at line 893 of file [ftypes.h](#).

4.49.2.592 DOLZERO

```
#define DOLZERO 7
```

Definition at line 894 of file [ftypes.h](#).

4.49.2.593 FINDLOOP

```
#define FINDLOOP 0
```

Definition at line 896 of file [ftypes.h](#).

4.49.2.594 REPLACELOOP

```
#define REPLACELOOP 1
```

Definition at line 897 of file [ftypes.h](#).

4.49.2.595 NOFUNPOWERS

```
#define NOFUNPOWERS 0
```

Definition at line 899 of file [ftypes.h](#).

4.49.2.596 COMFUNPOWERS

```
#define COMFUNPOWERS 1
```

Definition at line 900 of file [ftypes.h](#).

4.49.2.597 ALLFUNPOWERS

```
#define ALLFUNPOWERS 2
```

Definition at line 901 of file [ftypes.h](#).

4.49.2.598 PROPERORDERFLAG

```
#define PROPERORDERFLAG 0
```

Definition at line 903 of file [ftypes.h](#).

4.49.2.599 REGULAR

```
#define REGULAR 0
```

Definition at line 905 of file [ftypes.h](#).

4.49.2.600 FINISH

```
#define FINISH 1
```

Definition at line 906 of file [ftypes.h](#).

4.49.2.601 POLYADD

```
#define POLYADD 1
```

Definition at line 908 of file [ftypes.h](#).

4.49.2.602 POLYSUB

```
#define POLYSUB 2
```

Definition at line 909 of file [ftypes.h](#).

4.49.2.603 POLYMUL

```
#define POLYMUL 3
```

Definition at line 910 of file [ftypes.h](#).

4.49.2.604 POLYDIV

```
#define POLYDIV 4
```

Definition at line 911 of file [ftypes.h](#).

4.49.2.605 POLYREM

```
#define POLYREM 5
```

Definition at line 912 of file [ftypes.h](#).

4.49.2.606 POLYGCD

```
#define POLYGCD 6
```

Definition at line 913 of file [ftypes.h](#).

4.49.2.607 POLYINTFAC

```
#define POLYINTFAC 7
```

Definition at line 914 of file [ftypes.h](#).

4.49.2.608 POLYNORM

```
#define POLYNORM 8
```

Definition at line 915 of file [ftypes.h](#).

4.49.2.609 MODNONE

```
#define MODNONE 0
```

Definition at line 917 of file [ftypes.h](#).

4.49.2.610 MODSUM

```
#define MODSUM 1
```

Definition at line 918 of file [ftypes.h](#).

4.49.2.611 MODMAX

```
#define MODMAX 2
```

Definition at line 919 of file [ftypes.h](#).

4.49.2.612 MODMIN

```
#define MODMIN 3
```

Definition at line 920 of file [ftypes.h](#).

4.49.2.613 MODLOCAL

```
#define MODLOCAL 4
```

Definition at line 921 of file [ftypes.h](#).

4.49.2.614 ELEMENTUSED

```
#define ELEMENTUSED 1
```

Definition at line 923 of file [ftypes.h](#).

4.49.2.615 ELEMENTLOADED

```
#define ELEMENTLOADED 2
```

Definition at line 924 of file [ftypes.h](#).

4.49.2.616 POSNEG

```
#define POSNEG 0x1
```

Definition at line 929 of file [ftypes.h](#).

4.49.2.617 INVERSETABLE

```
#define INVERSETABLE 0x2
```

Definition at line 930 of file [ftypes.h](#).

4.49.2.618 COEFFICIENTSONLY

```
#define COEFFICIENTSONLY 0x4
```

Definition at line 931 of file [ftypes.h](#).

4.49.2.619 ALSOPOWERS

```
#define ALSOPOWERS 0x8
```

Definition at line 932 of file [ftypes.h](#).

4.49.2.620 ALSOFUNARGS

```
#define ALSOFUNARGS 0x10
```

Definition at line 933 of file [ftypes.h](#).

4.49.2.621 ALSODOLLARS

```
#define ALSODOLLARS 0x20
```

Definition at line 934 of file [ftypes.h](#).

4.49.2.622 NOINVERSES

```
#define NOINVERSES 0x40
```

Definition at line 935 of file [ftypes.h](#).

4.49.2.623 POSITIVEONLY

```
#define POSITIVEONLY 0
```

Definition at line 937 of file [ftypes.h](#).

4.49.2.624 UNPACK

```
#define UNPACK 0x80
```

Definition at line 938 of file [ftypes.h](#).

4.49.2.625 NOUNPACK

```
#define NOUNPACK 0
```

Definition at line 939 of file [ftypes.h](#).

4.49.2.626 FROMFUNCTION

```
#define FROMFUNCTION 0x100
```

Definition at line 940 of file [ftypes.h](#).

4.49.2.627 VARNAMES

```
#define VARNAMES 0
```

Definition at line 942 of file [ftypes.h](#).

4.49.2.628 AUTONAMES

```
#define AUTONAMES 1
```

Definition at line 943 of file [ftypes.h](#).

4.49.2.629 EXPRNAMES

```
#define EXPRNAMES 2
```

Definition at line 944 of file [ftypes.h](#).

4.49.2.630 DOLLARNAMES

```
#define DOLLARNAMES 3
```

Definition at line 945 of file [ftypes.h](#).

4.49.2.631 DUMMYBUFFER

```
#define DUMMYBUFFER 1
```

Definition at line 1000 of file [ftypes.h](#).

4.49.2.632 ALLARGS

```
#define ALLARGS 1
```

Definition at line 1002 of file [ftypes.h](#).

4.49.2.633 NUMARG

```
#define NUMARG 2
```

Definition at line 1003 of file [ftypes.h](#).

4.49.2.634 ARGRANGE

```
#define ARGRANGE 3
```

Definition at line 1004 of file [ftypes.h](#).

4.49.2.635 MAKEARGS

```
#define MAKEARGS 4
```

Definition at line 1005 of file [ftypes.h](#).

4.49.2.636 MAXRANGEINDICATOR

```
#define MAXRANGEINDICATOR 4
```

Definition at line 1006 of file [ftypes.h](#).

4.49.2.637 REPLACEARG

```
#define REPLACEARG 5
```

Definition at line 1007 of file [ftypes.h](#).

4.49.2.638 ENCODEARG

```
#define ENCODEARG 6
```

Definition at line 1008 of file [ftypes.h](#).

4.49.2.639 DECODEARG

```
#define DECODEARG 7
```

Definition at line 1009 of file [ftypes.h](#).

4.49.2.640 IMplodeARG

```
#define IMplodeARG 8
```

Definition at line 1010 of file [ftypes.h](#).

4.49.2.641 EXPLODEARG

```
#define EXPLODEARG 9
```

Definition at line 1011 of file [ftypes.h](#).

4.49.2.642 PERMUTEARG

```
#define PERMUTEARG 10
```

Definition at line 1012 of file [ftypes.h](#).

4.49.2.643 REVERSEARG

```
#define REVERSEARG 11
```

Definition at line 1013 of file [ftypes.h](#).

4.49.2.644 CYCLEARG

```
#define CYCLEARG 12
```

Definition at line 1014 of file [ftypes.h](#).

4.49.2.645 ISLYNDON

```
#define ISLYNDON 13
```

Definition at line 1015 of file [ftypes.h](#).

4.49.2.646 ISLYNDONR

```
#define ISLYNDONR 14
```

Definition at line 1016 of file [ftypes.h](#).

4.49.2.647 TOLYNDON

```
#define TOLYNDON 15
```

Definition at line 1017 of file [ftypes.h](#).

4.49.2.648 TOLYNDONR

```
#define TOLYNDONR 16
```

Definition at line 1018 of file [ftypes.h](#).

4.49.2.649 ADDARG

```
#define ADDARG 17
```

Definition at line 1019 of file [ftypes.h](#).

4.49.2.650 MULTIPLYARG

```
#define MULTIPLYARG 18
```

Definition at line 1020 of file [ftypes.h](#).

4.49.2.651 DROPARG

```
#define DROPARG 19
```

Definition at line 1021 of file [ftypes.h](#).

4.49.2.652 SELECTARG

```
#define SELECTARG 20
```

Definition at line 1022 of file [ftypes.h](#).

4.49.2.653 DEDUPARG

```
#define DEDUPARG 21
```

Definition at line 1023 of file [ftypes.h](#).

4.49.2.654 ZTOHARG

```
#define ZTOHARG 22
```

Definition at line 1024 of file [ftypes.h](#).

4.49.2.655 HTOZARG

```
#define HTOZARG 23
```

Definition at line 1025 of file [ftypes.h](#).

4.49.2.656 BASECODE

```
#define BASECODE 1
```

Definition at line 1033 of file [ftypes.h](#).

4.49.2.657 YESLYNDON

```
#define YESLYNDON 1
```

Definition at line 1034 of file [ftypes.h](#).

4.49.2.658 NOLYNDON

```
#define NOLYNDON 2
```

Definition at line 1035 of file [ftypes.h](#).

4.49.2.659 TOPOLYNOMIALFLAG

```
#define TOPOLYNOMIALFLAG 1
```

Definition at line 1037 of file [ftypes.h](#).

4.49.2.660 FACTARGFLAG

```
#define FACTARGFLAG 2
```

Definition at line 1038 of file [ftypes.h](#).

4.49.2.661 OLDFACTARG

```
#define OLDFACTARG 1
```

Definition at line 1040 of file [ftypes.h](#).

4.49.2.662 NEWFACTARG

```
#define NEWFACTARG 0
```

Definition at line 1041 of file [ftypes.h](#).

4.49.2.663 FROMMODULEOPTION

```
#define FROMMODULEOPTION 0
```

Definition at line 1043 of file [ftypes.h](#).

4.49.2.664 FROMPOINTINSTRUCTION

```
#define FROMPOINTINSTRUCTION 1
```

Definition at line 1044 of file [ftypes.h](#).

4.49.2.665 EXTRASYMBOL

```
#define EXTRASYMBOL 0
```

Definition at line 1046 of file [ftypes.h](#).

4.49.2.666 REGULARSYMBOL

```
#define REGULARSYMBOL 1
```

Definition at line 1047 of file [ftypes.h](#).

4.49.2.667 EXPRESSIONNUMBER

```
#define EXPRESSIONNUMBER 2
```

Definition at line 1048 of file [ftypes.h](#).

4.49.2.668 O_NONE

```
#define O_NONE 0
```

Definition at line 1050 of file [ftypes.h](#).

4.49.2.669 O_CSE

```
#define O_CSE 1
```

Definition at line 1051 of file [ftypes.h](#).

4.49.2.670 O_CSEGREEDY

```
#define O_CSEGREEDY 2
```

Definition at line 1052 of file [ftypes.h](#).

4.49.2.671 O_GREEDY

```
#define O_GREEDY 3
```

Definition at line 1053 of file [ftypes.h](#).

4.49.2.672 O_OCCURRENCE

```
#define O_OCCURRENCE 0
```

Definition at line 1055 of file [ftypes.h](#).

4.49.2.673 O_MCTS

```
#define O_MCTS 1
```

Definition at line 1056 of file [ftypes.h](#).

4.49.2.674 O_SIMULATED_ANNEALING

```
#define O_SIMULATED_ANNEALING 2
```

Definition at line 1057 of file [ftypes.h](#).

4.49.2.675 O_FORWARD

```
#define O_FORWARD 0
```

Definition at line 1059 of file [ftypes.h](#).

4.49.2.676 O_BACKWARD

```
#define O_BACKWARD 1
```

Definition at line 1060 of file [ftypes.h](#).

4.49.2.677 O_FORWARDORBACKWARD

```
#define O_FORWARDORBACKWARD 2
```

Definition at line 1061 of file [ftypes.h](#).

4.49.2.678 O_FORWARDANDBACKWARD

```
#define O_FORWARDANDBACKWARD 3
```

Definition at line 1062 of file [ftypes.h](#).

4.49.2.679 OPTHEAD

```
#define OPTHEAD 3
```

Definition at line 1064 of file [ftypes.h](#).

4.49.2.680 DOALL

```
#define DOALL 1
```

Definition at line 1065 of file [ftypes.h](#).

4.49.2.681 ONLYFUNCTIONS

```
#define ONLYFUNCTIONS 2
```

Definition at line 1066 of file [ftypes.h](#).

4.49.2.682 INUSE

```
#define INUSE 1
```

Definition at line 1068 of file [ftypes.h](#).

4.49.2.683 COULDCOMMUTE

```
#define COULDCOMMUTE 2
```

Definition at line 1069 of file [ftypes.h](#).

4.49.2.684 DOESNOTCOMMUTE

```
#define DOESNOTCOMMUTE 4
```

Definition at line 1070 of file [ftypes.h](#).

4.49.2.685 DICT_NONUMBERS

```
#define DICT_NONUMBERS 0
```

Definition at line 1072 of file [ftypes.h](#).

4.49.2.686 DICT_INTEGERONLY

```
#define DICT_INTEGERONLY 1
```

Definition at line 1073 of file [ftypes.h](#).

4.49.2.687 DICT_RATIONALONLY

```
#define DICT_RATIONALONLY 2
```

Definition at line 1074 of file [ftypes.h](#).

4.49.2.688 DICT_ALLNUMBERS

```
#define DICT_ALLNUMBERS 3
```

Definition at line 1075 of file [ftypes.h](#).

4.49.2.689 DICT_NOVARIABLES

```
#define DICT_NOVARIABLES 0
```

Definition at line 1076 of file [ftypes.h](#).

4.49.2.690 DICT_DOVARIABLES

```
#define DICT_DOVARIABLES 1
```

Definition at line 1077 of file [ftypes.h](#).

4.49.2.691 DICT_NOSPECIALS

```
#define DICT_NOSPECIALS 0
```

Definition at line 1078 of file [ftypes.h](#).

4.49.2.692 DICT_DOSPECIALS

```
#define DICT_DOSPECIALS 1
```

Definition at line 1079 of file [ftypes.h](#).

4.49.2.693 DICT_NOFUNWITHARGS

```
#define DICT_NOFUNWITHARGS 0
```

Definition at line 1080 of file [ftypes.h](#).

4.49.2.694 DICT_DOFUNWITHARGS

```
#define DICT_DOFUNWITHARGS 1
```

Definition at line 1081 of file [ftypes.h](#).

4.49.2.695 DICT_NOTINDOLLARS

```
#define DICT_NOTINDOLLARS 0
```

Definition at line 1082 of file [ftypes.h](#).

4.49.2.696 DICT_INDOLLARS

```
#define DICT_INDOLLARS 1
```

Definition at line 1083 of file [ftypes.h](#).

4.49.2.697 DICT_INTEGERNUMBER

```
#define DICT_INTEGERNUMBER 1
```

Definition at line 1085 of file [ftypes.h](#).

4.49.2.698 DICT_RATIONALNUMBER

```
#define DICT_RATIONALNUMBER 2
```

Definition at line 1086 of file [ftypes.h](#).

4.49.2.699 DICT_SYMBOL

```
#define DICT_SYMBOL 3
```

Definition at line 1087 of file [ftypes.h](#).

4.49.2.700 DICT_VECTOR

```
#define DICT_VECTOR 4
```

Definition at line 1088 of file [ftypes.h](#).

4.49.2.701 DICT_INDEX

```
#define DICT_INDEX 5
```

Definition at line 1089 of file [ftypes.h](#).

4.49.2.702 DICT_FUNCTION

```
#define DICT_FUNCTION 6
```

Definition at line 1090 of file [ftypes.h](#).

4.49.2.703 DICT_FUNCTION_WITH_ARGUMENTS

```
#define DICT_FUNCTION_WITH_ARGUMENTS 7
```

Definition at line 1091 of file [ftypes.h](#).

4.49.2.704 DICT_SPECIALCHARACTER

```
#define DICT_SPECIALCHARACTER 8
```

Definition at line 1092 of file [ftypes.h](#).

4.49.2.705 DICT_RANGE

```
#define DICT_RANGE 9
```

Definition at line 1093 of file [ftypes.h](#).

4.49.2.706 READSPECTATORFLAG

```
#define READSPECTATORFLAG 3
```

Definition at line 1095 of file [ftypes.h](#).

4.49.2.707 GLOBALSPECTATORFLAG

```
#define GLOBALSPECTATORFLAG 1
```

Definition at line 1096 of file [ftypes.h](#).

4.49.2.708 ORDEREDSET

```
#define ORDEREDSET 1
```

Definition at line 1098 of file [ftypes.h](#).

4.49.2.709 DENSETABLE

```
#define DENSETABLE 1
```

Definition at line 1100 of file [ftypes.h](#).

4.49.2.710 SPARSETABLE

```
#define SPARSETABLE 0
```

Definition at line 1101 of file [ftypes.h](#).

4.49.2.711 ONEPARTICLEIRREDUCIBLE

```
#define ONEPARTICLEIRREDUCIBLE 1
```

Definition at line 1103 of file [ftypes.h](#).

4.49.2.712 WITHINSERTIONS

```
#define WITHINSERTIONS 2
```

Definition at line 1104 of file [ftypes.h](#).

4.49.2.713 NOTADPOLES

```
#define NOTADPOLES 4
```

Definition at line 1105 of file [ftypes.h](#).

4.49.2.714 WITHSYMMETRIZE

```
#define WITHSYMMETRIZE 8
```

Definition at line 1106 of file [ftypes.h](#).

4.49.2.715 TOPOLOGIESONLY

```
#define TOPOLOGIESONLY 16
```

Definition at line 1107 of file [ftypes.h](#).

4.49.2.716 NONODES

```
#define NONODES 32
```

Definition at line 1108 of file [ftypes.h](#).

4.49.2.717 WITHEDGES

```
#define WITHEDGES 64
```

Definition at line 1109 of file [ftypes.h](#).

4.49.2.718 CHECKEXTERN

```
#define CHECKEXTERN 128
```

Definition at line 1110 of file [ftypes.h](#).

4.49.2.719 WITHBLOCKS

```
#define WITHBLOCKS 256
```

Definition at line 1111 of file [ftypes.h](#).

4.49.2.720 WITHONEPI

```
#define WITHONEPI 512
```

Definition at line 1112 of file [ftypes.h](#).

4.49.2.721 NOSNAILS

```
#define NOSNAILS 1024
```

Definition at line 1113 of file [ftypes.h](#).

4.49.2.722 NOEXTSELF

```
#define NOEXTSELF 2048
```

Definition at line 1114 of file [ftypes.h](#).

4.49.3 Typedef Documentation

4.49.3.1 PVFUNWP

```
typedef void(* PVFUNWP) ()
```

Definition at line 183 of file [ftypes.h](#).

4.49.3.2 PVFUNV

```
typedef void(* PVFUNV) ()
```

Definition at line 184 of file [ftypes.h](#).

4.49.3.3 CFUN

```
typedef int(* CFUN) ()
```

Definition at line 185 of file [ftypes.h](#).

4.49.3.4 TFUN

```
typedef int(* TFUN) ()
```

Definition at line 186 of file [ftypes.h](#).

4.49.3.5 TFUN1

```
typedef int(* TFUN1) ()
```

Definition at line 187 of file [ftypes.h](#).

4.50 ftypes.h

[Go to the documentation of this file.](#)

```

00001
00009 /* #[ License : */
00010 /*
00011 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00012 *   When using this file you are requested to refer to the publication
00013 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00014 *   This is considered a matter of courtesy as the development was paid
00015 *   for by FOM the Dutch physics granting agency and we would like to
00016 *   be able to track its scientific use to convince FOM of its value
00017 *   for the community.
00018 *
00019 *   This file is part of FORM.
00020 *
00021 *   FORM is free software: you can redistribute it and/or modify it under the
00022 *   terms of the GNU General Public License as published by the Free Software
00023 *   Foundation, either version 3 of the License, or (at your option) any later
00024 *   version.
00025 *
00026 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00027 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00028 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00029 *   details.
00030 *
00031 *   You should have received a copy of the GNU General Public License along
00032 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00033 */
00034 /* #] License : */
00035
00051 #define WITHOUTERROR 0
00052 #define WITHERROR 1
00053
00054 /*
00055     The various streams. (look also in tools.c)
00056 */
00057
00058 #define FILESTREAM 0
00059 #define PREVARSTREAM 1
00060 #define PREREADSTREAM 2
00061 #define PIPESTREAM 3
00062 #define PRECALCSTREAM 4
00063 #define DOLLARSTREAM 5
00064 #define PREREADSTREAM2 6
00065 #define EXTERNALCHANNELSTREAM 7
00066 #define PREREADSTREAM3 8
00067 #define REVERSEFILESTREAM 9
00068 #define INPUTSTREAM 10
00069
00070 #define ENDOFSTREAM 0xFF
00071 #define ENDOFINPUT 0xFF
00072
00073 /*
00074     Types of files
00075 */
00076
00077 #define SUBROUTINEFILE 0
00078 #define PROCEDUREFILE 1
00079 #define HEADERFILE 2
00080 #define SETUPFILE 3
00081 #define TABLEBASEFILE 4
00082
00083 /*
00084     Types of modules
00085 */
00086
00087 #define FIRSTMODULE -1
00088 #define GLOBALMODULE 0
00089 #define SORTMODULE 1
00090 #define STOREMODULE 2
00091 #define CLEARMODULE 3
00092 #define ENDMODULE 4
00093
00094 #define POLYFUN 0
00095
00096 #define NOPARALLEL_DOLLAR 0x0001
00097 #define NOPARALLEL_RHS 0x0002
00098 #define NOPARALLEL_CONVPOLY 0x0004
00099 #define NOPARALLEL_SPECTATOR 0x0008
00100 #define NOPARALLEL_USER 0x0010
00101 #define NOPARALLEL_TBLDOLLAR 0x0100
00102 #define NOPARALLEL_NPROC 0x0200
00103 #define PARALLELFLAG 0x0000
00104

```

```
00105 #define PRENOACTION 0
00106 #define PRERAISEAFTER 1
00107 #define PRELOWERAFTER 2
00108 /*
00109 #define ELIUMOD 1
00110 #define ELIZMOD 2
00111 #define SKIUMOD 3
00112 #define SKIZMOD 4
00113 */
00114 #define WITHSEMICOLON 0
00115 #define WITHOUTSEMICOLON 1
00116 #define MODULEINSTACK 8
00117 #define EXECUTINGIF 0
00118 #define LOOKINGFORELSE 1
00119 #define LOOKINGFORENDIF 2
00120 #define NEWSTATEMENT 1
00121 #define OLDSTATEMENT 0
00122
00123 #define EXECUTINGPRESWITCH 0
00124 #define SEARCHINGPRECASE 1
00125 #define SEARCHINGPREENDSWITCH 2
00126
00127 #define PREPROONLY 1
00128 #define DUMPTOCOMPILER 2
00129 #define DUMPOUTTERMS 4
00130 #define DUMPINTERMS 8
00131 #define DUMPTOSORT 16
00132 #define DUMPTOPARALLEL 32
00133 #define THREADSDEBUG 64
00134
00135 #define ERROROUT 0
00136 #define INPUTOUT 1
00137 #define STATSOUT 2
00138 #define EXPRSOUT 3
00139 #define WRITEOUT 4
00140
00141 #define EXTERNALCHANNELOUT 5
00142
00143 #define NUMERICALLOOP 0
00144 #define LISTEDLOOP 1
00145 #define ONEEXPRESSION 2
00146
00147 #define PRETYPENONE 0
00148 #define PRETYPEIF 1
00149 #define PRETYPEDO 2
00150 #define PRETYPEPROCEDURE 3
00151 #define PRETYPESWITCH 4
00152 #define PRETYPEINSIDE 5
00153
00154 /*
00155     Type of statement. Used to make sure that the statements are in proper order
00156 */
00157
00158 #define DECLARATION 1
00159 #define SPECIFICATION 2
00160 #define DEFINITION 3
00161 #define STATEMENT 4
00162 #define TOOOUTPUT 5
00163 #define ATENDOFMODULE 6
00164 #define MIXED2 8
00165 #define MIXED 9
00166
00167 /*
00168     The typedefs are to allow the compilers to do better error checking.
00169 */
00170
00171 #ifdef ANSI
00172     typedef void (*PVFUNWP) (WORD *);
00173 #ifdef INTELCOMPILER
00174     typedef void (*PVFUNV) ();
00175     typedef int (*CFUN) ();
00176 #else
00177     typedef void (*PVFUNV) (void);
00178     typedef int (*CFUN) (void);
00179 #endif
00180     typedef int (*TFUN) (UBYTE *);
00181     typedef int (*TFUN1) (UBYTE *,int);
00182 #else
00183     typedef void (*PVFUNWP) ();
00184     typedef void (*PVFUNV) ();
00185     typedef int (*CFUN) ();
00186     typedef int (*TFUN) ();
00187     typedef int (*TFUN1) ();
00188 #endif
00189
00190
00191 #define NOAUTO 0
```

```
00192 #define PARTEST 1
00193 #define WITHAUTO 2
00194
00195 #define ALLVARIABLES -1
00196 #define SYMBOLONLY 1
00197 #define INDEXONLY 2
00198 #define VECTORONLY 4
00199 #define FUNCTIONONLY 8
00200 #define SETONLY 16
00201 #define EXPRESSIONONLY 32
00202
00203
00211
00212 #define CDELETE -1
00213 #define ANYTYPE -1
00214 #define CSYMBOL 0
00215 #define CINDEX 1
00216 #define CVECTOR 2
00217 #define CFUNCTION 3
00218 #define CSET 4
00219 #define CEXPRESSION 5
00220 #define CDOTPRODUCT 6
00221 #define CNUMBER 7
00222 #define CSUBEXP 8
00223 #define CDELTA 9
00224 #define CDOLLAR 10
00225 #define CDUBIOUS 11
00226 #define CRANGE 12
00227 #define CMODEL 13
00228 #define CVECTOR1 21
00229 #define CDOUBLEDOT 22
00230
00233 /*
00234     Types of tokens in the tokenizer.
00235 */
00236
00237 #define TSYMBOL -1
00238 #define TINDEX -2
00239 #define TVECTOR -3
00240 #define TFUNCTION -4
00241 #define TSET -5
00242 #define TEXPRESSON -6
00243 #define TDOTPRODUCT -7
00244 #define TNUMBER -8
00245 #define TSUBEXP -9
00246 #define TDELTA -10
00247 #define TDOLLAR -11
00248 #define TDUBIOUS -12
00249 #define LPARENTHESIS -13
00250 #define RPARENTHESIS -14
00251 #define TWILDCARD -15
00252 #define TWILDARG -16
00253 #define TDOT -17
00254 #define LBRACE -18
00255 #define RBRACE -19
00256 #define TCOMMA -20
00257 #define TFUNOPEN -21
00258 #define TFUNCLOSE -22
00259 #define TMULTIPLY -23
00260 #define TDIVIDE -24
00261 #define TPOWER -25
00262 #define TPLUS -26
00263 #define TMINUS -27
00264 #define TNOT -28
00265 #define TENDOFIT -29
00266 #define TSETOPEN -30
00267 #define TSETCLOSE -31
00268 #define TGENINDEX -32
00269 #define TCONJUGATE -33
00270 #define LRPARENTHESSES -34
00271 #define TNUMBER1 -35
00272 #define TPOWER1 -36
00273 #define TEMPTY -37
00274 #define TSETNUM -38
00275 #define TSGAMMA -39
00276 #define TSETDOL -40
00277 #define TFLOAT -41
00278
00279 #define TYPEISFUN 0
00280 #define TYPEISSUB 1
00281 #define TYPEISMYSTERY -1
00282
00283 #define LHSIDEX 2
00284 #define LHSIDE 1
00285 #define RHSIDE 0
00286
00287 /*
```

```

00288      Output modes
00289      */
00290
00291 #define FORTRANMODE 1
00292 #define REDUCEMODE 2
00293 #define MAPLEMODE 3
00294 #define MATHEMATICAMODE 4
00295 #define CMODE 5
00296 #define VORTRANMODE 6
00297 #define PFORTRANMODE 7
00298 #define DOUBLEFORTRANMODE 33
00299 #define DOUBLEPRECISIONFLAG 32
00300 #define NODOUBLEMASK 31
00301 #define QUADRUPLEFORTRANMODE 65
00302 #define QUADRUPLEPRECISIONFLAG 64
00303 #define ALLINTEGERDOUBLE 128
00304 #define NOQUADMASK 63
00305 #define NORMALFORMAT 0
00306 #define NOSPACEFORMAT 1
00307
00308 #define ISNOTFORTRAN90 0
00309 #define ISFORTRAN90 1
00310
00311 #define ALSOREVERSE 1
00312 #define CHISHOLM 2
00313 #define NOTRICK 16
00314
00315 #define SORTLOWFIRST 0
00316 #define SORTHIGHFIRST 1
00317 #define SORTPOWERFIRST 2
00318 #define SORTANTIPOWER 3
00319
00320 /* These control the output of WriteStats. The different sort
00321  * types have differently formatted stats. These variables should
00322  * not have arbitrary values since they are also used as array
00323  * indices by WriteStats. */
00324 #define STATSSPLITMERGE 0
00325 #define STATSMERGETOFILE 1
00326 #define STATSPOSTSORT 2
00327 /* CHECKLOGTYPE implies that statistics will not be printed in
00328  * the log file if AM.LogType = 1 (when running with -ll or -F). */
00329 #define NOCHECKLOGTYPE 0
00330 #define CHECKLOGTYPE 1
00331
00332 #define NMIN4SHIFT 4
00333 /*
00334      The next are the main codes.
00335      Note: SETSET is not allowed to be 4*n+1
00336      We use those codes in CoIdExpression for function information
00337      after the pattern. Because SETSET also stands there we have to
00338      be careful!!
00339      Don't forget MAXBUILTINFUNCTION when adding codes!
00340      The object FUNCTION is at the start of the functions that are in regular
00341      notation. Anything below it is in special notation.
00342
00343      Remark: HAAKJEO is for compression purposes and should only occur
00344      at moments that ARGWILD cannot occur.
00345 */
00346 #define SYMBOL 1
00347 #define DOTPRODUCT 2
00348 #define VECTOR 3
00349 #define INDEX 4
00350 #define EXPRESSION 5
00351 #define SUBEXPRESSION 6
00352 #define DOLLAREXPRESSION 7
00353 #define SETSET 8
00354 #define ARGWILD 9
00355 #define MINVECTOR 10
00356 #define SETEXP 11
00357 #define DOLLAREXPR2 12
00358 #define HAAKJEO 9
00359 #define FUNCTION 20
00360
00361 #define TMPPOLYFUN 14
00362 #define ARGFIELD 15
00363 #define SNUMBER 16
00364 #define LNUMBER 17
00365 #define HAAKJE 18
00366 #define DELTA 19
00367 #define EXPONENT 20
00368 #define DENOMINATOR 21
00369 #define SETFUNCTION 22
00370 #define GAMMA 23
00371 #define GAMMAI 24
00372 #define GAMMAFIVE 25
00373 #define GAMMASIX 26
00374 #define GAMMASEVEN 27

```



```
00375 #define SUMF1 28
00376 #define SUMF2 29
00377 #define DUMFUN 30
00378 #define REPLACEMENT 31
00379 #define REVERSEFUNCTION 32
00380 #define DISTRIBUTION 33
00381 #define DELTA3 34
00382 #define DUMMYFUN 35
00383 #define DUMMYTEN 36
00384 #define LEVICIVITA 37
00385 #define FACTORIAL 38
00386 #define INVERSEFACTORIAL 39
00387 #define BINOMIAL 40
00388 #define NUMARGSFUN 41
00389 #define SIGNFUN 42
00390 #define MODFUNCTION 43
00391 #define MOD2FUNCTION 44
00392 #define MINFUNCTION 45
00393 #define MAXFUNCTION 46
00394 #define ABSFUNCTION 47
00395 #define SIGFUNCTION 48
00396 #define INTFUNCTION 49
00397 #define THETA 50
00398 #define THETA2 51
00399 #define DELTA2 52
00400 #define DELTAP 53
00401 #define BERNOULLIFUNCTION 54
00402 #define COUNTFUNCTION 55
00403 #define MATCHFUNCTION 56
00404 #define PATTERNFUNCTION 57
00405 #define TERMFUNCTION 58
00406 #define CONJUGATION 59
00407 #define ROOTFUNCTION 60
00408 #define TABLEFUNCTION 61
00409 #define FIRSTBRACKET 62
00410 #define TERMSINEXPR 63
00411 #define NUMTERMSFUN 64
00412 #define GCDFUNCTION 65
00413 #define DIVFUNCTION 66
00414 #define REMFUNCTION 67
00415 #define MAXPOWEROF 68
00416 #define MINPOWEROF 69
00417 #define TABLESTUB 70
00418 #define FACTORIN 71
00419 #define TERMSINBRACKET 72
00420 #define WILDARGFUN 73
00421 /*
00422      In the past we would add new functions here and raise the numbers
00423      on the special reserved names. This is impractical in the light of
00424      the .sav files. The .sav files need a mechanism that contains the
00425      value of MAXBUILTINFUNCTION at the moment of writing. This allows
00426      form corrections if this value has changed in the mean time.
00427 */
00428 #define SQRTFUNCTION 74
00429 #define LNFUNCTION 75
00430 #define SINFUNCTION 76
00431 #define COSFUNCTION 77
00432 #define TANFUNCTION 78
00433 #define ASINFUNCTION 79
00434 #define ACOSFUNCTION 80
00435 #define ATANFUNCTION 81
00436 #define ATAN2FUNCTION 82
00437 #define SINHFUNTION 83
00438 #define COSHFUNTION 84
00439 #define TANHFUNCTION 85
00440 #define ASINHFUNTION 86
00441 #define ACOSHFUNTION 87
00442 #define ATANHFUNTION 88
00443 #define LI2FUNCTION 89
00444 #define LINFUNTION 90
00445
00446 #define EXTRASYMFUN 91
00447 #define RANDOMFUNCTION 92
00448 #define RANPERM 93
00449 #define NUMFACTORS 94
00450 #define FIRSTTERM 95
00451 #define CONTENTTERM 96
00452 #define PRIMENUMBER 97
00453 #define EXTEUCLIDEAN 98
00454 #define MAKERATIONAL 99
00455 #define INVERSEFUNCTION 100
00456 #define IDFUNCTION 101
00457 #define PUTFIRST 102
00458 #define PERMUTATIONS 103
00459 #define PARTITIONS 104
00460 #define MULFUNCTION 105
00461 #define MOEBIUS 106
```

```
00462 #define TOPOLOGIES 107
00463 #define DIAGRAMS 108
00464 #define TOPO 109
00465 #define NODEFUNCTION 110
00466 #define EDGE 111
00467 #define SIZEOFFUNCTION 112
00468 #define BLOCK 113
00469 #define ONEPI 114
00470 #define PHI 115
00471 #ifdef WITHFLOAT
00472 #define FLOATFUN 116
00473 #define TOFLOAT 117
00474 #define TORAT 118
00475 #define MZV 119
00476 #define EULER 120
00477 #define MZVHALF 121
00478 #define AGMFUN 122
00479 #define GAMMAFUN 123
00480 #define MAXBUILTINFUNCTION 123
00481 #else
00482 #define MAXBUILTINFUNCTION 115
00483 #endif
00484
00485 #define FIRSTUSERFUNCTION 150
00486
00487 #define ALLFUNCTIONS -1
00488 #define ALLMZVFUNCTIONS -2
00489
00490 /*
00491     Note: if we add a builtin table we have to look also inside names.c
00492     in the routine Globalize because there we assume there does not exist
00493     such an object
00494 */
00495
00496 #define ISYMBOL 0
00497 #define PISYMBOL 1
00498 #define COEFFSYMBOL 2
00499 #define NUMERATORSYMBOL 3
00500 #define DENOMINATORSYMBOL 4
00501 #define WILDARGSYMBOL 5
00502 #define DIMENSIONSYMBOL 6
00503 #define FACTORSYMBOL 7
00504 #define SEPARATESYMBOL 8
00505 #define EESYMBOL 9
00506 #define EMSYMBOL 10
00507
00508 #define BUILTINSYMBOLS 11
00509 #define FIRSTUSERSYMBOL 20
00510
00511 #define BUILTININDICES 1
00512 #define BUILTINVECTORS 1
00513 #define BUILTINDOLLARS 1
00514
00515 #define WILDARGVECTOR 0
00516 #define WILDARGINDEX 0
00517
00518 /*
00519     The objects that have a name that starts with TYPE are codes of statements
00520     made by the compiler. Each statement starts with such a code, followed by
00521     its size. For how most of these statements are used can be seen in the
00522     Generator function in the file proces.c
00523     TYPEOPERATION is an anachronism that remains used only for the statements
00524     that are executed in the file opera.c (like traces and contractions).
00525 */
00526
00527 #define TYPEEXPRESSION 0
00528 #define TYPEIDNEW 1
00529 #define TYPEIDOLD 2
00530 #define TYPEOPERATION 3
00531 #define TYPEPERPEAT 4
00532 #define TYPEENDREPEAT 5
00533 /*
00534     The next counts must be higher than the ones before
00535 */
00536 #define TYPECOUNT 20
00537 #define TYPEMULT 21
00538 #define TYPEGOTO 22
00539 #define TYPEDISCARD 23
00540 #define TYPEIF 24
00541 #define TYPEELSE 25
00542 #define TYPEELIF 26
00543 #define TYPEENDIF 27
00544 #define TYPESUM 28
00545 #define TYPECHISHOLM 29
00546 #define TYPEPERVERSE 30
00547 #define TYPEARG 31
00548 #define TYPENORM 32
```

```
00549 #define TYPENORM2 33
00550 #define TYPENORM3 34
00551 #define TYPEEXIT 35
00552 #define TYPESETEXIT 36
00553 #define TYPEPRINT 37
00554 #define TYPEFPRINT 38
00555 #define TYPEPERDEFPRE 39
00556 #define TYPESPLITARG 40
00557 #define TYPESPLITARG2 41
00558 #define TYPEFACTARG 42
00559 #define TYPEFACTARG2 43
00560 #define TYPETRY 44
00561 #define TYPEASSIGN 45
00562 #define TYPERENUMBER 46
00563 #define TYPESUMFIX 47
00564 #define TYPEFINDLOOP 48
00565 #define TYPEUNRAVEL 49
00566 #define TYPEADJUSTBOUNDS 50
00567 #define TYPEINSIDE 51
00568 #define TYPETERM 52
00569 #define TYPESORT 53
00570 #define TYPEDETCURDUM 54
00571 #define TYPEINEXPRESSION 55
00572 #define TYPESPLITFIRSTARG 56
00573 #define TYPESPLITLASTARG 57
00574 #define TYPEMERGE 58
00575 #define TYPETESTUSE 59
00576 #define TYPEAPPLY 60
00577 #define TYPEAPPLYRESET 61
00578 #define TYPECHAININ 62
00579 #define TYPECHAINOUT 63
00580 #define TYPENORM4 64
00581 #define TYPEFACTOR 65
00582 #define TYPEARGIMPLODE 66
00583 #define TYPEARGEXPLODE 67
00584 #define TYPEDENOMINATORS 68
00585 #define TYPESTUFFLE 69
00586 #define TYPEDROPCOEFFICIENT 70
00587 #define TYPETRANSFORM 71
00588 #define TYPETOPOLYNOMIAL 72
00589 #define TYPEFROMPOLYNOMIAL 73
00590 #define TYPEDOLOOP 74
00591 #define TYPEENDDOLOOP 75
00592 #define TYPEDROPSYMBOLS 76
00593 #define TYPEPUTINSIDE 77
00594 #define TYPETOSPECTATOR 78
00595 #define TYPEARGTOEXTRASymbol 79
00596 #define TYPECANONICALIZE 80
00597 #define TYPESWITCH 81
00598 #define TYPEENDSWITCH 82
00599 #define TYPESETUSERFLAG 83
00600 #define TYPECLEARUSERFLAG 84
00601 #define TYPEALLLOOPS 85
00602 #define TYPEALLPATHS 86
00603 #ifdef WITHFLOAT
00604 #define TYPEEVALUATE 87
00605 #define TYPEEXPAND 88
00606 #define TYPETOFloat 89
00607 #define TYPETORAT 90
00608 #endif
00609
00610 /*
00611     The codes for the 'operations' that are part of TYPEOPERATION.
00612 */
00613
00614 #define TAKETRACE 1
00615 #define CONTRACT 2
00616 #define RATIO 3
00617 #define SYMMETRIZE 4
00618 #define TENVEC 5
00619 #define SUMNUM1 6
00620 #define SUMNUM2 7
00621
00622 /*
00623     The various types of wildcards.
00624 */
00625
00626 #define WILDDUMMY 0
00627 #define SYMTONUM 1
00628 #define SYMTOSYM 2
00629 #define SYMTOSUB 3
00630 #define VECTOMIN 4
00631 #define VECTOVEC 5
00632 #define VECTOSUB 6
00633 #define INDTOIND 7
00634 #define INDTOSUB 8
00635 #define FUNTOFUN 9
```

```
00636 #define ARGTOARG 10
00637 #define ARLTOARL 11
00638 #define EXPTOEXP 12
00639 #define FROMBRAC 13
00640 #define FROMSET 14
00641 #define SETTONUM 15
00642 #define WILDCARDS 16
00643 #define SETNUMBER 17
00644 #define LOADDOLLAR 18
00645 /*
00646     Some new types of wildcards that hold only for function arguments.
00647 */
00648 #define NUMTONUM 20
00649 #define NUMTOSYM 21
00650 #define NUMTOIND 22
00651 #define NUMTOSUB 23
00652 /*
00653     Flag or-ed with ARGTOARG to give the number of arguments.
00654 */
00655 #define EATTENSOR 0x2000
00656
00657 /*
00658     Dirty flags (introduced when functions got a field with a dirty flag)
00659 */
00660
00661 #define CLEANFLAG 0
00662 #define DIRTYFLAG 1
00663 #define DIRTYSYMFLAG 2
00664 #define MUSTCLEANPRF 4
00665 #define SUBTERMUSED1 8
00666 #define SUBTERMUSED2 16
00667 #define ALLDIRTY (DIRTYFLAG|DIRTYSYMFLAG)
00668
00669 #define ARGHEAD 2
00670 #define FUNHEAD 3
00671 #define SUBEXPSIZE 5
00672 #define EXPRHEAD 5
00673 #define TYPEARGHEADSIZE 6
00674
00675 /*
00676     Actions to be taken with expressions. They are marked with these objects
00677     during compilation.
00678 */
00679
00680 #define SKIP 1
00681 #define DROP 2
00682 #define HIDE 3
00683 #define UNHIDE 4
00684 #define INTOHIDE 5
00685
00686 /*
00687     Types of expressions
00688 */
00689
00690 #define LOCALEXPRESSION 0
00691 #define SKIPLEXPRESSION 1
00692 #define DROPLEXPRESSION 2
00693 #define DROPPEDLEXPRESSION 3
00694 #define GLOBALEXPRESSION 4
00695 #define SKIPGEXPRESSION 5
00696 #define DROPGEXPRESSION 6
00697 #define STOREDEXPRESSION 8
00698 #define HIDDENLEXPRESSION 9
00699 #define HIDDENGEXPRESSION 13
00700 #define INCEXPRESSION 9
00701 #define HIDELEXPRESSION 10
00702 #define HIDEGEXPRESSION 14
00703 #define DROPHLEXPRESSION 11
00704 #define DROPHGEXPRESSION 15
00705 #define UNHIDELEXPRESSION 12
00706 #define UNHIDEGEXPRESSION 16
00707 #define INTOHIDELEXPRESSION 17
00708 #define INTOHIDEGEXPRESSION 18
00709 #define SPECTATOREXPRESSION 19
00710 #define DROPSPECTATOREXPRESSION 20
00711 #define SKIPUNHIDELEXPRESSION 21
00712 #define SKIPUNHIDEGEXPRESSION 22
00713
00714 #define PRINTOFF 0
00715 #define PRINTON 1
00716 #define PRINTCONTENTS 2
00717 #define PRINTCONTENT 3
00718 #define PRINTLFILE 4
00719 #define PRINTONETERM 8
00720 #define PRINTONEFUNCTION 16
00721 #define PRINTALL 32
00722
```

```

00723 /*
00724     Special codes for the replace variable in the EXPRESSIONS struct
00725 */
00726
00727 #define REGULAREXPRESSION -1
00728 #define REDEFINEDEXPRESSION -2
00729 #define NEWLYDEFINEDEXPRESSION -3
00730
00738 #define VERTEXFUNCTION -1
00739 #define GENERALFUNCTION 0
00740 #define TENSORFUNCTION 2
00741 #define GAMMAFUNCTION 4
00744 /*
00745     Special sets
00746 */
00747
00748 #define POS_          0 /* integer > 0 */
00749 #define POS0_         1 /* integer >= 0 */
00750 #define NEG_          2 /* integer < 0 */
00751 #define NEG0_         3 /* integer <= 0 */
00752 #define EVEN_         4 /* integer (even) */
00753 #define ODD_          5 /* integer (odd) */
00754 #define Z_            6 /* integer */
00755 #define SYMBOL_       7 /* symbol only */
00756 #define FIXED_        8 /* fixed index */
00757 #define INDEX_        9 /* index only */
00758 #define Q_            10 /* rational */
00759 #define DUMMYINDEX_   11 /* dummy index only */
00760 #define VECTOR_       12 /* vector only */
00761
00762 /*
00763     Special indices.
00764 */
00765
00766 #define GAMMA1 0
00767 #define GAMMA5 -1
00768 #define GAMMA6 -2
00769 #define GAMMA7 -3
00770 #define FUNNYVEC -4
00771 #define FUNNYWILD -5
00772 #define SUMMEDIND -6
00773 #define NOINDEX -7
00774 #define FUNNYDOLLAR -8
00775 #define EMPTYINDEX -9
00776
00777 /*
00778     The next one should be less than all of the above special indices.
00779 */
00780
00781 #define MINSPEC -10
00782
00783 #define USEDFLAG 2
00784 #define DUMMYFLAG 1
00785
00786 #define MAINSORT 0
00787 #define FUNCTIONSORT 1
00788 #define SUBSORT 2
00789
00790 #define FLOATMODE 1
00791 #define RATIONALMODE 0
00792
00793 #define NUMSPECSETS 10
00794
00795 #define ISZERO 1
00796 #define ISUNMODIFIED 2
00797 #define ISCOMPRESSED 4
00798 #define ISINRHS 8
00799 #define ISFACTORIZED 16
00800 #define TOBEFACTORED 32
00801 #define TOBEUNFACTORED 64
00802 #define KEEPZERO 128
00803
00804 #define VARTYPENONE 0
00805 #define VARTYPECOMPLEX 1
00806 #define VARTYPEIMAGINARY 2
00807 #define VARTYPEROOTOFUNITY 4
00808 #define VARTYPEMINUS 8
00809 #define CYCLESYMMETRIC 1
00810 #define RCYCLESYMMETRIC 2
00811 #define SYMMETRIC 3
00812 #define ANTISYMMETRIC 4
00813 #define REVERSEORDER 256
00814
00815 /*
00816     Types of id statements (substitutions)
00817 */
00818

```

```
00819 #define SUBMULTI 1
00820 #define SUBONCE 2
00821 #define SUBONLY 3
00822 #define SUBMANY 4
00823 #define SUBVECTOR 5
00824 #define SUBSELECT 6
00825 #define SUBALL 7
00826 #define SUBMASK 15
00827 #define SUBDISORDER 16
00828 #define SUBAFTER 32
00829 #define SUBAFTERNOT 64
00830
00831 #define IDHEAD 6
00832
00833 #define DOLLARFLAG 1
00834 #define NORMALIZEFLAG 2
00835
00836 #define GIDENT 1
00837 #define GFIVE 4
00838 #define GPLUS 3
00839 #define GMINUS 2
00840
00841 /*
00842     Types of objects inside an if clause.
00843 */
00844
00845 #define LONGNUMBER 1
00846 #define MATCH 2
00847 #define COEFFI 3
00848 #define SUBEXPR 4
00849 #define MULTIPLEOF 5
00850 #define IFDOLLAR 6
00851 #define IFEXPRESSION 7
00852 #define IFDOLLAREXTRA 8
00853 #define IFISFACTORIZED 9
00854 #define IFOCCURS 10
00855 #define IFUSERFLAG 11
00856 #define IFFLOATNUMBER 12
00857 #define GREATER 0
00858 #define GREATEREQUAL 1
00859 #define LESS 2
00860 #define LESSEQUAL 3
00861 #define EQUAL 4
00862 #define NOTEQUAL 5
00863 #define ORCOND 6
00864 #define ANDCOND 7
00865 #define DUMMY 1
00866 #define SORT 1
00867 #define STORE 2
00868 #define END 3
00869 #define GLOBAL 4
00870 #define CLEAR 5
00871
00872 #define VECTBIT 1
00873 #define DOTPBIT 2
00874 #define FUNBIT 4
00875 #define SETBIT 8
00876
00877 #define EXTRAPARAMETER 0x4000
00878 #define GENCOMMUTE 0
00879 #define GENNONCOMMUTE 0x2000
00880
00881 #define NAMENOTFOUND -9
00882
00883 /*
00884     Types of dollar expressions.
00885 */
00886
00887 #define DOLUNDEFINED 0
00888 #define DOLNUMBER 1
00889 #define DOLARGUMENT 2
00890 #define DOLSUBTERM 3
00891 #define DOLTERMS 4
00892 #define DOLWILDARGS 5
00893 #define DOLINDEX 6
00894 #define DOLZERO 7
00895
00896 #define FINDLOOP 0
00897 #define REPLACELOOP 1
00898
00899 #define NOFUNPOWERS 0
00900 #define COMFUNPOWERS 1
00901 #define ALLFUNPOWERS 2
00902
00903 #define PROPERORDERFLAG 0
00904
00905 #define REGULAR 0
```

```
00906 #define FINISH 1
00907
00908 #define POLYADD 1
00909 #define POLYSUB 2
00910 #define POLYMUL 3
00911 #define POLYDIV 4
00912 #define POLYREM 5
00913 #define POLYGCD 6
00914 #define POLYINTFAC 7
00915 #define POLYNORM 8
00916
00917 #define MODNONE 0
00918 #define MODSUM 1
00919 #define MODMAX 2
00920 #define MODMIN 3
00921 #define MODLOCAL 4
00922
00923 #define ELEMENTUSED 1
00924 #define ELEMENTLOADED 2
00925 /*
00926     Variables for the modulus statement, flags in AC.modmode
00927     For explanation, see CoModulus
00928 */
00929 #define POSNEG 0x1
00930 #define INVERSETABLE 0x2
00931 #define COEFFICIENTSONLY 0x4
00932 #define ALSOPOWERS 0x8
00933 #define ALSOFUNARGS 0x10
00934 #define ALSODOLLARS 0x20
00935 #define NOINVERSES 0x40
00936
00937 #define POSITIVEONLY 0
00938 #define UNPACK 0x80
00939 #define NOUNPACK 0
00940 #define FROMFUNCTION 0x100
00941
00942 #define VARNAMES 0
00943 #define AUTONAMES 1
00944 #define EXPRNAMES 2
00945 #define DOLLARNAMES 3
00946
00947 #ifndef WITHPTHREADS
00948 /*
00949     Signals that the workers have to react to
00950 */
00951
00952 #define TERMINATETHREAD -1
00953 #define STARTNEWEXPRESSION 1
00954 #define LOWESTLEVELGENERATION 2
00955 #define FINISHEXPRESSION 3
00956 #define CLEANUPEXPRESSION 4
00957 #define HIGHERLEVELGENERATION 5
00958 #define STARTNEWMODULE 6
00959 #define CLAIMOUTPUT 7
00960 #define FINISHEXPRESSION2 8
00961 #define INISORTBOT 7
00962 #define RUNSORTBOT 8
00963 #define DOONEEXPRESSION 9
00964 #define DOBRACKETS 10
00965 #define CLEARCLOCK 11
00966 #define MCTSEXPANDTREE 12
00967 #define OPTIMIZEEXPRESSION 13
00968
00969 #define MASTERBUFFERISFULL 1
00970
00971 /*
00972     Bucket states
00973 */
00974
00975 #define BUCKETFREE 1
00976 #define BUCKETINUSE 0
00977 #define BUCKETCOMINGFREE 2
00978 #define BUCKETFILLED -1
00979 #define BUCKETATEND -2
00980 #define BUCKETTERMINATED 3
00981 #define BUCKETRELEASED 4
00982
00983 #define NUMBEROFBLOCKSINSORT 10
00984 #define MINIMUMNUMBEROFTERMS 10
00985
00986 #define BUCKETDOINGTERM 1
00987 #define BUCKETASSIGNED -1
00988 #define BUCKETTOBERELEASED -2
00989 #define BUCKETPREPARINGTERM 0
00990
00991 #define BUCKETDOINGTERMS 0
00992 #define BUCKETDOINGBRACKET 1
```

```

00993 #endif
00994
00995 /*
00996     The next variable is because there is some use of cbufnum that is
00997     probably irrelevant. We use here DUMMYBUFNUM instead of AC.cbufnum
00998     just in case we run into trouble later.
00999 */
01000 #define DUMMYBUFFER 1
01001
01002 #define ALLARGS      1
01003 #define NUMARG       2
01004 #define ARGRANGE     3
01005 #define MAKEARGS     4
01006 #define MAXRANGEINDICATOR 4
01007 #define REPLACEARG   5
01008 #define ENCODEARG    6
01009 #define DECODEARG    7
01010 #define IMplodeARG   8
01011 #define EXPLODEARG   9
01012 #define PERMUTEARG   10
01013 #define REVERSEARG   11
01014 #define CYCLEARG     12
01015 #define ISLYNDON     13
01016 #define ISLYNDONR    14
01017 #define TOLYNDON     15
01018 #define TOLYNDONR    16
01019 #define ADDARG       17
01020 #define MULTIPLYARG  18
01021 #define DROPARG      19
01022 #define SELECTARG    20
01023 #define DEDUPARG     21
01024 #define ZTOHARG      22
01025 #define HTOZARG      23
01026 /*
01027 #define ZTOSARG      24
01028 #define STOZARG      25
01029 #define HTOSARG      26
01030 #define STOARG       27
01031 */
01032
01033 #define BASECODE 1
01034 #define YESLYNDON 1
01035 #define NOLYNDON 2
01036
01037 #define TOPOLYNOMIALFLAG 1
01038 #define FACTARGFLAG 2
01039
01040 #define OLDFACTARG 1
01041 #define NEWFACTARG 0
01042
01043 #define FROMMODULEOPTION 0
01044 #define FROMPOINTINSTRUCTION 1
01045
01046 #define EXTRASYMBOL 0
01047 #define REGULARSYMBOL 1
01048 #define EXPRESSIONNUMBER 2
01049
01050 #define O_NONE 0
01051 #define O_CSE 1
01052 #define O_CSEGREEDY 2
01053 #define O_GREEDY 3
01054
01055 #define O_OCCURRENCE 0
01056 #define O_MCTS 1
01057 #define O_SIMULATED_ANNEALING 2
01058
01059 #define O_FORWARD 0
01060 #define O_BACKWARD 1
01061 #define O_FORWARDORBACKWARD 2
01062 #define O_FORWARDANDBACKWARD 3
01063
01064 #define OPTHEAD 3
01065 #define DOALL 1
01066 #define ONLYFUNCTIONS 2
01067
01068 #define INUSE 1
01069 #define COULDCOMMUTE 2
01070 #define DOESNOTCOMMUTE 4
01071
01072 #define DICT_NONUMBERS 0
01073 #define DICT_INTEGERONLY 1
01074 #define DICT_RATIONALONLY 2
01075 #define DICT_ALLNUMBERS 3
01076 #define DICT_NOVARIABLES 0
01077 #define DICT_DOVARIABLES 1
01078 #define DICT_NOSPECIALS 0
01079 #define DICT_DOSPECIALS 1

```



```

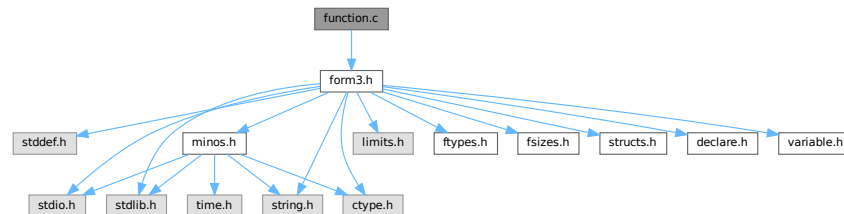
01080 #define DICT_NOFUNWITHARGS 0
01081 #define DICT_DOFUNWITHARGS 1
01082 #define DICT_NOTINDOLLARS 0
01083 #define DICT_INDOLLARS 1
01084
01085 #define DICT_INTEGERNUMBER 1
01086 #define DICT_RATIONALNUMBER 2
01087 #define DICT_SYMBOL 3
01088 #define DICT_VECTOR 4
01089 #define DICT_INDEX 5
01090 #define DICT_FUNCTION 6
01091 #define DICT_FUNCTION_WITH_ARGUMENTS 7
01092 #define DICT_SPECIALCHARACTER 8
01093 #define DICT_RANGE 9
01094
01095 #define READSPECTATORFLAG 3
01096 #define GLOBALSPECTATORFLAG 1
01097
01098 #define ORDEREDSET 1
01099
01100 #define DENSETABLE 1
01101 #define SPARSETABLE 0
01102
01103 #define ONEPARTICLEIRREDUCIBLE 1
01104 #define WITHINSERTIONS 2
01105 #define NOTADPOLES 4
01106 #define WITHSYMMETRIZE 8
01107 #define TOPOLOGIESONLY 16
01108 #define NONODES 32
01109 #define WITHEDGES 64
01110 #define CHECKEXTERN 128
01111 #define WITHBLOCKS 256
01112 #define WITHONEPI 512
01113 #define NOSNAILS 1024
01114 #define NOEXTSELF 2048
01115

```

4.51 function.c File Reference

```
#include "form3.h"
```

Include dependency graph for function.c:



Functions

- WORD [MakeDirty](#) (WORD *term, WORD *x, WORD par)
- void [MarkDirty](#) (WORD *term, WORD flags)
- void [PolyFunDirty](#) (PHEAD WORD *term)
- void [PolyFunClean](#) (PHEAD WORD *term)
- WORD [Symmetrize](#) (PHEAD WORD *func, WORD *Lijst, WORD ngroups, WORD gsize, WORD type)
- WORD [CompGroup](#) (PHEAD WORD type, WORD **args, WORD *a1, WORD *a2, WORD num)
- int [FullSymmetrize](#) (PHEAD WORD *fun, int type)
- WORD [SymGen](#) (PHEAD WORD *term, WORD *params, WORD num, WORD level)
- WORD [SymFind](#) (PHEAD WORD *term, WORD *params)
- int [ChainIn](#) (PHEAD WORD *term, WORD funnum)
- int [ChainOut](#) (PHEAD WORD *term, WORD funnum)
- WORD [MatchFunction](#) (PHEAD WORD *pattern, WORD *interm, WORD *wilds)
- WORD [ScanFunctions](#) (PHEAD WORD *inpat, WORD *inter, WORD par)

4.51.1 Detailed Description

The file with the central routines for the pattern matching of functions and their arguments. The file also contains the routines for the execution of the Symmetrize statement and its variations (like antisymmetrize etc).

Definition in file [function.c](#).

4.51.2 Function Documentation

4.51.2.1 MakeDirty()

```
WORD MakeDirty (
    WORD * term,
    WORD * x,
    WORD par )
```

Definition at line 51 of file [function.c](#).

4.51.2.2 MarkDirty()

```
void MarkDirty (
    WORD * term,
    WORD flags )
```

Definition at line 97 of file [function.c](#).

4.51.2.3 PolyFunDirty()

```
void PolyFunDirty (
    PHEAD WORD * term )
```

Definition at line 139 of file [function.c](#).

4.51.2.4 PolyFunClean()

```
void PolyFunClean (
    PHEAD WORD * term )
```

Definition at line 174 of file [function.c](#).

4.51.2.5 Symmetrize()

```
WORD Symmetrize (
    PHEAD WORD * func,
    WORD * Lijst,
    WORD ngroups,
    WORD gsize,
    WORD type )
```

Definition at line 213 of file [function.c](#).

4.51.2.6 CompGroup()

```
WORD CompGroup (
    PHEAD WORD type,
    WORD ** args,
    WORD * a1,
    WORD * a2,
    WORD num )
```

Definition at line 373 of file [function.c](#).

4.51.2.7 FullSymmetrize()

```
int FullSymmetrize (
    PHEAD WORD * fun,
    int type )
```

Definition at line 473 of file [function.c](#).

4.51.2.8 SymGen()

```
WORD SymGen (
    PHEAD WORD * term,
    WORD * params,
    WORD num,
    WORD level )
```

Definition at line 518 of file [function.c](#).

4.51.2.9 SymFind()

```
WORD SymFind (
    PHEAD WORD * term,
    WORD * params )
```

Definition at line 633 of file [function.c](#).

4.51.2.10 ChainIn()

```
int ChainIn (
    PHEAD WORD * term,
    WORD funnum )
```

Definition at line 691 of file [function.c](#).

4.51.2.11 ChainOut()

```
int ChainOut (
    PHEAD WORD * term,
    WORD funnum )
```

Definition at line 748 of file [function.c](#).

4.51.2.12 MatchFunction()

```
WORD MatchFunction (
    PHEAD WORD * pattern,
    WORD * interm,
    WORD * wilds )
```

Definition at line 826 of file [function.c](#).

4.51.2.13 ScanFunctions()

```
WORD ScanFunctions (
    PHEAD WORD * inpat,
    WORD * inter,
    WORD par )
```

Definition at line 1616 of file [function.c](#).

4.52 function.c

[Go to the documentation of this file.](#)

```
00001
00008 /* # License : */
00009 /*
00010 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00011 * When using this file you are requested to refer to the publication
00012 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00013 * This is considered a matter of courtesy as the development was paid
00014 * for by FOM the Dutch physics granting agency and we would like to
00015 * be able to track its scientific use to convince FOM of its value
00016 * for the community.
00017 *
00018 * This file is part of FORM.
00019 *
00020 * FORM is free software: you can redistribute it and/or modify it under the
00021 * terms of the GNU General Public License as published by the Free Software
00022 * Foundation, either version 3 of the License, or (at your option) any later
00023 * version.
00024 *
00025 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00026 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00027 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00028 * details.
00029 *
00030 * You should have received a copy of the GNU General Public License along
00031 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00032 */
00033 /* # License : */
00034 /*
00035 * # Includes : function.c
00036 */
00037
00038 #include "form3.h"
00039
00040 /*
00041 * # Includes :
00042 * # Utilities :
00043 * # MakeDirty :
00044 *
00045 * Routine finds the function with the address x in it
00046 * and mark all arguments that contain x as dirty.
00047 * if par == 0 term is a full term, else term is the start of a
00048 * function
00049 */
00050
00051 WORD MakeDirty(WORD *term, WORD *x, WORD par)
00052 {
00053     WORD *next, *n;
00054     if ( !par ) {
00055         next = term; next += *term;
```

```

00056     next -= ABS(next[-1]);
00057     term++;
00058     if ( x < term ) return(0);
00059     if ( x >= next ) return(0);
00060     while ( term < next ) {
00061         n = term + term[1];
00062         if ( x < n ) break;
00063         term = n;
00064     }
00065     /* next = n; */
00066 }
00067 else {
00068     next = term + term[1];
00069     if ( x < term || x >= next ) return(0);
00070 }
00071 if ( *term < FUNCTION ) return(0);
00072 if ( functions[*term-FUNCTION].spec >= TENSORFUNCTION ) return(0);
00073 term += FUNHEAD;
00074 if ( x < term ) return(0);
00075 next = term; NEXTARG(next)
00076 while ( x >= next ) { term = next; NEXTARG(next) }
00077 if ( *term < 0 ) return(0);
00078 term[1] = 1;
00079 term += ARGHEAD;
00080 if ( x < term ) return(1);
00081 next = term + *term;
00082 while ( x >= next ) { term = next; next += *next; }
00083 MakeDirty(term,x,0);
00084 return(1);
00085 }
00086
00087 /*
00088     #] MakeDirty :
00089     #[ MarkDirty :
00090
00091     Routine marks all functions dirty with the given flags.
00092     Is to be used when there is a possibility that symmetrization
00093     properties of functions may have changed. In that case we play
00094     it safe.
00095 */
00096 void MarkDirty(WORD *term, WORD flags)
00097 {
00098     WORD *t, *r, *m, *tstop;
00099     GETSTOP(term,tstop);
00100     t = term+1;
00101     while ( t < tstop ) {
00102         if ( *t < FUNCTION ) { t += t[1]; continue; }
00103         t[2] |= flags;
00104         if ( *t < FUNCTION+WILDOFFSET && functions[*t-FUNCTION].spec > 0 ) {
00105             t += t[1]; continue;
00106         }
00107         if ( *t >= FUNCTION+WILDOFFSET && functions[*t-FUNCTION-WILDOFFSET].spec > 0 ) {
00108             t += t[1]; continue;
00109         }
00110     }
00111     r = t + FUNHEAD;
00112     t += t[1];
00113     while ( r < t ) {
00114         if ( *r <= 0 ) {
00115             if ( *r <= -FUNCTION ) r++;
00116             else r += 2;
00117             continue;
00118         }
00119         r[1] |= DIRTYFLAG;
00120         m = r + ARGHEAD;
00121         r += *r;
00122         while ( m < r ) {
00123             MarkDirty(m,flags);
00124             m += *m;
00125         }
00126     }
00127 }
00128 }
00129
00130 /*
00131     #] MarkDirty :
00132     #[ PolyFunDirty :
00133
00134     Routine marks the PolyFun or the PolyRatFun dirty.
00135     This is used when there is modular calculus and the modulus
00136     has changed for the current module.
00137 */
00138 void PolyFunDirty(PHEAD WORD *term)
00139 {
00140     GETBIDENTITY
00141     WORD *t, *tstop, *endarg;

```

```

00143     tstop = term + *term;
00144     tstop -= ABS(tstop[-1]);
00145     t = term+1;
00146     while ( t < tstop ) {
00147         if ( *t == AR.PolyFun ) {
00148             if ( AR.PolyFunType == 2 ) t[2] |= MUSTCLEANPRF;
00149             endarg = t + t[1];
00150             t[2] |= DIRTYFLAG;
00151             t += FUNHEAD;
00152             while ( t < endarg ) {
00153                 if ( *t > 0 ) {
00154                     t[1] |= DIRTYFLAG;
00155                 }
00156                 NEXTARG(t);
00157             }
00158         }
00159         else {
00160             t += t[1];
00161         }
00162     }
00163 }
00164
00165 /*
00166     #] PolyFunDirty :
00167     #[ PolyFunClean :
00168
00169     Routine marks the PolyFun or the PolyRatFun clean.
00170     This is used when there is modular calculus and the modulus
00171     has changed for the current module.
00172 */
00173
00174 void PolyFunClean(PHEAD WORD *term)
00175 {
00176     GETBIDENTITY
00177     WORD *t, *tstop;
00178     tstop = term + *term;
00179     tstop -= ABS(tstop[-1]);
00180     t = term+1;
00181     while ( t < tstop ) {
00182         if ( *t == AR.PolyFun ) {
00183             t[2] &= ~MUSTCLEANPRF;
00184         }
00185         t += t[1];
00186     }
00187 }
00188
00189 /*
00190     #] PolyFunClean :
00191     #[ Symmetrize :
00192
00193     (Anti)Symmetrizes the arguments of a function.
00194     Nlist tells of how many arguments are involved.
00195     Nlist == 0      All arguments must be sorted.
00196     Nlist > 0      Arguments mentioned are to be sorted, rest skipped.
00197     type = SYMMETRIC      Full symmetrization
00198     type = ANTISYMMETRIC: Full symmetrization
00199     type = CYCLESYMMETRIC: Cyclic
00200     type = RCYCLESYMMETRIC: Cyclic or reverse
00201     Return value: OR of:
00202         0 even, 1 odd
00203         2 equal groups
00204         4 there was a permutation.
00205
00206     The information in Lijst tells what grouping is to be applied.
00207     The information is:
00208     ngroups number of groups
00209     gsize size of groups
00210     Lijst[0].... The groups.
00211 */
00212
00213 WORD Symmetrize(PHEAD WORD *func, WORD *Lijst, WORD ngroups, WORD gsize,
00214     WORD type)
00215 {
00216     GETBIDENTITY
00217     WORD **args,**arg,nargs;
00218     WORD *to, *r, *fstop;
00219     WORD i, j, k, ff, exch, nexch, neq;
00220     WORD *a1, *a2, *a3;
00221     WORD reverseorder;
00222     if ( ( type & REVERSEORDER ) != 0 ) reverseorder = -1;
00223     else reverseorder = 1;
00224     type &= ~REVERSEORDER;
00225
00226     ff = ( *func > FUNCTION ) ? functions[*func-FUNCTION].spec: 0;
00227
00228     if ( 2*func[1] > AN.arglistsize ) {
00229         if ( AN.arglist ) M_free(AN.arglist,"Symmetrize");

```

```

00230     AN.arglistsize = 2*func[l] + 8;
00231     AN.arglist = (WORD **)Malloc1(AN.arglistsize*sizeof(WORD *),"Symmetrize");
00232 }
00233 arg = args = AN.arglist;
00234 to = AT.WorkPointer;
00235 r = func;
00236 fstop = r + r[l];
00237 r += FUNHEAD;
00238 nargs = 0;
00239 while ( r < fstop ) { /* Make list of arguments */
00240     *arg++ = r;
00241     nargs++;
00242     if ( ff ) {
00243         if ( *r == FUNNYWILD ) r++;
00244         r++;
00245     }
00246     else { NEXTARG(r); }
00247 }
00248 exch = 0;
00249 nexch = 0;
00250 neq = 0;
00251 al = Lijst;
00252 if ( type == SYMMETRIC || type == ANTISYMMETRIC ) {
00253     for ( i = 1; i < ngroups; i++ ) {
00254         a3 = a2 = al + gsize;
00255         k = reverseorder*CompGroup(BHEAD ff,args,al,a2,gsiz);
00256         if ( k < 0 ) {
00257             j = i-1;
00258             for (;) {
00259                 for ( k = 0; k < gsize; k++ ) {
00260                     r = args[al[k]]; args[al[k]] = args[a2[k]]; args[a2[k]] = r;
00261                 }
00262                 exch ^= 1;
00263                 nexch = 4;
00264                 if ( j <= 0 ) break;
00265                 al -= gsize;
00266                 a2 -= gsize;
00267                 k = reverseorder*CompGroup(BHEAD ff,args,al,a2,gsiz);
00268                 if ( k == 0 ) neq = 2;
00269                 if ( k >= 0 ) break;
00270                 j--;
00271             }
00272         }
00273         else if ( k == 0 ) neq = 2;
00274         al = a3;
00275     }
00276 }
00277 else if ( type == CYCLESYMMETRIC || type == RCYCLESYMMETRIC ) {
00278     WORD rev = 0, jmin = 0, ii, iimin;
00279 recycle:
00280     for ( j = 1; j < ngroups; j++ ) {
00281         for ( i = 0; i < ngroups; i++ ) {
00282             iimin = jmin + i;
00283             if ( iimin >= ngroups ) iimin -= ngroups;
00284             ii = j + i;
00285             if ( ii >= ngroups ) ii -= ngroups;
00286             k = reverseorder*CompGroup(BHEAD ff,args,Lijst+gsiz*iimin,Lijst+gsiz*ii,gsiz);
00287             if ( k > 0 ) break;
00288             if ( k < 0 ) { jmin = j; nexch = 4; break; }
00289         }
00290     }
00291     if ( type == RCYCLESYMMETRIC && rev == 0 && ngroups > 1 ) {
00292         for ( j = 0; j < ngroups; j++ ) {
00293             for ( i = 0; i < ngroups; i++ ) {
00294                 iimin = jmin + i;
00295                 if ( iimin >= ngroups ) iimin -= ngroups;
00296                 ii = j - i;
00297                 if ( ii < 0 ) ii += ngroups;
00298                 k = reverseorder*CompGroup(BHEAD ff,args,Lijst+gsiz*iimin,Lijst+gsiz*ii,gsiz);
00299                 if ( k > 0 ) break;
00300                 if ( k < 0 ) {
00301                     nexch = 4;
00302                     jmin = 0;
00303                     al = Lijst;
00304                     a2 = Lijst + gsize * (ngroups-1);
00305                     while ( a2 > al ) {
00306                         for ( k = 0; k < gsize; k++ ) {
00307                             r = args[al[k]];
00308                             args[al[k]] = args[a2[k]];
00309                             args[a2[k]] = r;
00310                         }
00311                         al += gsize; a2 -= gsize;
00312                     }
00313                     rev = 1;
00314                     goto recycle;
00315                 }
00316             }
00317         }
00318     }

```

```

00317     }
00318 }
00319 if ( jmin != 0 ) {
00320     arg = AN.arglist + func[1];
00321     a1 = Lijst + gsize * jmin;
00322     k = gsize * ngroups;
00323     a2 = Lijst + k;
00324     for ( i = 0; i < k; i++ ) {
00325         if ( a1 >= a2 ) a1 = Lijst;
00326         *arg++ = args[*a1++];
00327     }
00328     arg = AN.arglist + func[1];
00329     a1 = Lijst;
00330     for ( i = 0; i < k; i++ ) args[*a1++] = *arg++;
00331 }
00332 }
00333 r = func;
00334 i = FUNHEAD;
00335 NCOPY(to,r,i);
00336 for ( i = 0; i < nargs; i++ ) {
00337     if ( ff ) {
00338         if ( *(args[i]) == FUNNYWILD ) {
00339             *to++ = *(args[i]);
00340             *to++ = args[i][1];
00341         }
00342         else *to++ = *(args[i]);
00343     }
00344     else if ( ( j = *args[i] ) < 0 ) {
00345         *to++ = j;
00346         if ( j > -FUNCTION ) *to++ = args[i][1];
00347     }
00348     else {
00349         r = args[i];
00350         NCOPY(to,r,j);
00351     }
00352 }
00353 i = func[1];
00354 to = func;
00355 r = AT.WorkPointer;
00356 NCOPY(to,r,i);
00357 return ( exch | nexch | neq );
00358 }
00359
00360 /*
00361     #] Symmetrize :
00362     #[ CompGroup :
00363
00364     Routine compares two groups of arguments
00365     The arguments are in args[a1[i]] and args[a2[i]]
00366     for i = 0 to num
00367     type indicates the type of function.
00368     return value: -1 if there should be an exchange
00369     0 if they are equal
00370     1 if they are OK.
00371 */
00372
00373 WORD CompGroup(PHEAD WORD type, WORD **args, WORD *a1, WORD *a2, WORD num)
00374 {
00375     GETBIDENTITY
00376     WORD *t1, *t2, i1, i2, n, k;
00377
00378     for ( n = 0; n < num; n++ ) {
00379         t1 = args[a1[n]]; t2 = args[a2[n]];
00380         if ( type >= TENSORFUNCTION ) {
00381             if ( AR.Eside == LHSIDE || AR.Eside == LHSIDEX ) {
00382                 if ( *t1 == FUNNYWILD ) {
00383                     if ( *t2 == FUNNYWILD ) {
00384                         if ( t1[1] < t2[1] ) return(1);
00385                         if ( t1[1] > t2[1] ) return(-1);
00386                     }
00387                     return(-1);
00388                 }
00389                 else if ( *t2 == FUNNYWILD ) {
00390                     return(1);
00391                 }
00392                 else {
00393                     if ( *t1 < *t2 ) return(1);
00394                     if ( *t1 > *t2 ) return(-1);
00395                 }
00396             }
00397             else {
00398                 if ( *t1 < *t2 ) return(1);
00399                 if ( *t1 > *t2 ) return(-1);
00400             }
00401         }
00402         else if ( type == 0 ) {
00403             if ( AC.properorderflag ) {

```



```

00404         k = CompArg(t1,t2);
00405         if ( k < 0 ) return(1);
00406         if ( k > 0 ) return(-1);
00407         NEXTARG(t1)
00408         NEXTARG(t2)
00409     }
00410     else {
00411         if ( *t1 > 0 ) {
00412             i1 = *t1 - ARGHEAD - 1;
00413             t1 += ARGHEAD + 1;
00414             if ( *t2 > 0 ) {
00415                 i2 = *t2 - ARGHEAD - 1;
00416                 t2 += ARGHEAD + 1;
00417                 while ( i1 > 0 && i2 > 0 ) {
00418                     if ( *t1 > *t2 ) return(-1);
00419                     else if ( *t1 < *t2 ) return(1);
00420                     i1--; i2--; t1++; t2++;
00421                 }
00422                 if ( i1 > 0 ) return(-1);
00423                 else if ( i2 > 0 ) return(1);
00424             }
00425             /*
00426              This seems to be a bug. Reported by Aneesh Monahar, 28-sep-2005
00427              else return(1);
00428             */
00429             else return(-1);
00430         }
00431         else if ( *t2 > 0 ) return(1);
00432         else {
00433             if ( *t1 != *t2 ) {
00434                 if ( *t1 <= -FUNCTION && *t2 <= -FUNCTION ) {
00435                     if ( *t1 < *t2 ) return(-1);
00436                     return(1);
00437                 }
00438                 else {
00439                     if ( *t1 < *t2 ) return(1);
00440                     return(-1);
00441                 }
00442             }
00443             if ( *t1 > -FUNCTION ) {
00444                 if ( t1[1] != t2[1] ) {
00445                     if ( t1[1] < t2[1] ) return(1);
00446                     return(-1);
00447                 }
00448             }
00449         }
00450     }
00451 }
00452 }
00453 return(0);
00454 }
00455
00456 /*
00457     #] CompGroup :
00458     #[ FullSymmetrize :
00459
00460     Relay function for Normalize to execute a full symmetrization
00461     of a function fun. It hooks into Symmetrize according to the
00462     calling conventions for it.
00463     type = 0: Symmetrize
00464     type = 1: AntiSymmetrize
00465     type = 2: CycleSymmetrize
00466     type = 3: RCycleSymmetrize
00467     Return values:
00468     bit 0: odd permutation
00469     bit 1: identical arguments
00470     bit 2: there was a permutation.
00471 */
00472
00473 int FullSymmetrize(PHEAD WORD *fun, int type)
00474 {
00475     GETBIDENTITY
00476     WORD *Lijst, count = 0;
00477     WORD *t, *funstop, i;
00478     int retval;
00479
00480     if ( functions[*fun-FUNCTION].spec > 0 ) {
00481         count = fun[1] - FUNHEAD;
00482         for ( i = fun[1]-1; i >= FUNHEAD; i-- ) {
00483             if ( fun[i] == FUNNYWILD ) count--;
00484         }
00485     }
00486     else {
00487         funstop = fun + fun[1];
00488         t = fun + FUNHEAD;
00489         while ( t < funstop ) { count++; NEXTARG(t) }
00490     }

```

```

00491     if ( count < 2 ) {
00492         fun[2] &= ~DIRTYSYMFLAG;
00493         return(0);
00494     }
00495     Lijst = AT.WorkPointer;
00496     for ( i = 0; i < count; i++ ) Lijst[i] = i;
00497     AT.WorkPointer += count;
00498     retval = Symmetrize(BHEAD fun,Lijst,count,1,type);
00499     fun[2] &= ~DIRTYSYMFLAG;
00500     AT.WorkPointer = Lijst;
00501     return(retval);
00502 }
00503
00504 /*
00505     #] FullSymmetrize :
00506     #[ SymGen :
00507
00508     Routine does the outer work in the symmetrization.
00509     It locates the function(s) and loads up the parameters.
00510     It also studies the result.
00511
00512     if params[4] = -1 and no extra -> all
00513         extra -> strip groups with elements too large
00514         0 -> if group with element too large: nofun
00515         >0 -> must have right number of arguments
00516 */
00517
00518 WORD SymGen(PHEAD WORD *term, WORD *params, WORD num, WORD level)
00519 {
00520     GETBIDENTITY
00521     WORD *t, *r, *m;
00522     WORD i, j, k, c1, c2, ngroup;
00523     WORD *rstop, Nlist, *inLijst, *Lijst, sign = 1, sumch = 0, count;
00524     DUMMYUSE(num);
00525     c1 = params[3]; /* function number */
00526     c2 = FUNCTION + WILDOFFSET;
00527     Nlist = params[4];
00528     if ( Nlist < 0 ) Nlist = 0;
00529     else Nlist = params[0] - 7;
00530     t = term;
00531     m = t + *t;
00532     m -= ABS(m[-1]);
00533     t++;
00534     while ( t < m ) {
00535         if ( *t == c1 || c1 > c2 ) { /* Candidate function */
00536             if ( *t >= FUNCTION && functions[*t-FUNCTION].spec
00537                 >= TENSORFUNCTION ) {
00538                 count = t[1] - FUNHEAD;
00539             }
00540             else {
00541                 count = 0;
00542                 r = t;
00543                 rstop = t + t[1];
00544                 r += FUNHEAD;
00545                 while ( r < rstop ) { count++; NEXTARG(r) }
00546             }
00547             if ( ( j = params[4] ) > 0 && j != count ) goto NextFun;
00548             if ( j == 0 ) {
00549                 inLijst = params+7;
00550                 for ( i = 0; i < Nlist; i++ )
00551                     if ( inLijst[i] > count-1 ) goto NextFun;
00552             }
00553
00554             if ( Nlist > (params[0] - 7) ) Nlist = params[0] - 7;
00555             Lijst = AT.WorkPointer;
00556             inLijst = params + 7;
00557             ngroup = params[5];
00558             if ( Nlist > 0 && j < 0 ) {
00559                 k = 0;
00560                 for ( i = 0; i < ngroup; i++ ) {
00561                     for ( j = 0; j < params[6]; j++ ) {
00562                         if ( inLijst[j] > count+1 ) {
00563                             inLijst += params[6];
00564                             goto NextGroup;
00565                         }
00566                     }
00567                     j = params[6];
00568                     NCOPY(Lijst,inLijst,j);
00569                     k++;
00570 NextGroup:;
00571                 }
00572                 if ( k <= 1 ) goto NextFun;
00573                 ngroup = k;
00574                 inLijst = AT.WorkPointer;
00575                 AT.WorkPointer = Lijst;
00576                 Lijst = inLijst;
00577             }

```

```

00578         else if ( Nlist == 0 ) {
00579             for ( i = 0; i < count; i++ ) Lijst[i] = i;
00580             AT.WorkPointer += count;
00581             ngroup = count;
00582         }
00583         else {
00584             for ( i = 0; i < Nlist; i++ ) Lijst[i] = inLijst[i];
00585             AT.WorkPointer += Nlist;
00586         }
00587         j = Symmetrize(BHEAD t,Lijst,ngroup,params[6],params[2]);
00588         AT.WorkPointer = Lijst;
00589         if ( params[2] == 4 ) { /* antisymmetric */
00590             if ( ( j & 1 ) != 0 ) sign = -sign;
00591             if ( ( j & 2 ) != 0 ) return(0); /* equal arguments */
00592         }
00593         if ( ( j & 4 ) != 0 ) sumch++;
00594         t[2] &= ~DIRTYSYMFLAG;
00595     }
00596 NextFun:
00597     t += t[1];
00598 }
00599 if ( sign < 0 ) {
00600     t = term;
00601     t += *t - 1;
00602     *t = -*t;
00603 }
00604 if ( sumch ) {
00605     if ( Normalize(BHEAD term) ) {
00606         MLOCK(ErrorMessageLock);
00607         MesCall("SymGen");
00608         MUNLOCK(ErrorMessageLock);
00609         return(-1);
00610     }
00611     if ( !*term ) return(0);
00612     *AN.RepPoint = 1;
00613     AR.expchanged = 1;
00614     if ( AR.CurDum > AM.IndDum && AR.sLevel <= 0 ) ReNumber(BHEAD term);
00615 }
00616 return(Generator(BHEAD term,level));
00617 }
00618
00619 /*
00620 #] SymGen :
00621 #[ SymFind :
00622
00623 There is a certain amount of double work here, as this routine
00624 finds the function to be treated, while the SymGen routine has
00625 to find it again. Note however that this way things remain
00626 uniform and simple. Moreover this avoids problems with actions
00627 on more than one function simultaneously.
00628 Output in AT.TMout:
00629 Number,sym/anti,fun,lenpar,ngroups,gsizes,fields
00630
00631 */
00632
00633 WORD SymFind(PHEAD WORD *term, WORD *params)
00634 {
00635     GETBIDENTITY
00636     WORD *t, *r, *m;
00637     WORD j, c1, c2, count;
00638     WORD *rstop;
00639     c1 = params[4]; /* function number */
00640     c2 = FUNCTION + WILDOFFSET;
00641     t = term;
00642     m = t + *t;
00643     m -= ABS(m[-1]);
00644     t++;
00645     while ( t < m ) {
00646         if ( *t == c1 || c1 > c2 ) { /* Candidate function */
00647             if ( *t >= FUNCTION && functions[*t-FUNCTION].spec
00648                 >= TENSORFUNCTION ) { count = t[1] - FUNHEAD; }
00649             else {
00650                 count = 0;
00651                 r = t;
00652                 rstop = t + t[1];
00653                 r += FUNHEAD;
00654                 while ( r < rstop ) { count++; NEXTARG(r) }
00655             }
00656             if ( ( j = params[5] ) > 0 && j != count ) goto NextFun;
00657             if ( j == 0 ) {
00658                 r = params + 8;
00659                 rstop = params + params[1];
00660                 while ( r < rstop ) {
00661                     if ( *r > count + 1 ) goto NextFun;
00662                     r++;
00663                 }
00664             }

```

```

00665
00666         t = AT.TMout;
00667         r = params;
00668         j = r[1] - 1;
00669         *t++ = j;
00670         *t++ = SYMMETRIZE;
00671         r += 3;
00672         j--;
00673         NCOPY(t,r,j);
00674         return(1);
00675     }
00676 NextFun:
00677     t += t[1];
00678 }
00679 return(0);
00680 }
00681
00682 /*
00683     #] SymFind :
00684     #[ ChainIn :
00685
00686     Equivalent to repeat id f(?a)*f(?b) = f(?a,?b);
00687
00688     This one always takes less space.
00689 */
00690
00691 int ChainIn(PHEAD WORD *term, WORD funnum)
00692 {
00693     GETBIDENTITY
00694     WORD *t, *tend, *m, *tt, *ts;
00695     int action, normFlag = 0;
00696     if ( funnum < 0 ) { /* Dollar to be expanded */
00697         funnum = DolToFunction(BHEAD -funnum);
00698         if ( AN.ErrorInDollar || funnum <= 0 ) {
00699             MLOCK(ErrorMessageLock);
00700             MesPrint("Dollar variable does not evaluate to function in ChainIn statement");
00701             MUNLOCK(ErrorMessageLock);
00702             return(-1);
00703         }
00704     }
00705     do {
00706         action = 0;
00707         tend = term+*term;
00708         tend -= ABS(tend[-1]);
00709         t = term+1;
00710         while ( t < tend ) {
00711             if ( *t != funnum ) { t += t[1]; continue; }
00712             m = t;
00713             t += t[1];
00714             tt = t;
00715             if ( t >= tend || *t != funnum ) continue;
00716             action = 1;
00717             normFlag = 1;
00718             while ( t < tend && *t == funnum ) {
00719                 ts = t + t[1];
00720                 t += FUNHEAD;
00721                 while ( t < ts ) *tt++ = *t++;
00722             }
00723             m[1] = tt - m;
00724             ts = term + *term;
00725             while ( t < ts ) *tt++ = *t++;
00726             *term = tt - term;
00727             break;
00728         }
00729     } while ( action );
00730
00731     if ( normFlag ) {
00732         /* We need to check the newly-constructed arguments w.r.t symmetry properties */
00733         MarkDirty(term, DIRTSYMFLAG);
00734         AT.WorkPointer = term + *term;
00735         Normalize(BHEAD term);
00736     }
00737     return(0);
00738 }
00739 }
00740
00741 /*
00742     #] ChainIn :
00743     #[ ChainOut :
00744
00745     Equivalent to repeat id f(x1?,x2?,?a) = f(x1)*f(x2,?a);
00746 */
00747
00748 int ChainOut(PHEAD WORD *term, WORD funnum)
00749 {
00750     GETBIDENTITY
00751     WORD *t, *tend, *tt, *ts, *w, *ws;

```

```

00752     int flag = 0, i;
00753     if ( funnum < 0 ) { /* Dollar to be expanded */
00754         funnum = DolToFunction(BHEAD -funnum);
00755         if ( AN.ErrorInDollar || funnum <= 0 ) {
00756             MLOCK(ErrorMessageLock);
00757             MesPrint("Dollar variable does not evaluate to function in ChainOut statement");
00758             MUNLOCK(ErrorMessageLock);
00759             return(-1);
00760         }
00761     }
00762     tend = term+*term;
00763     if ( AT.WorkPointer < tend ) AT.WorkPointer = tend;
00764     tend -= ABS(tend[-1]);
00765     t = term+1; tt = term; w = AT.WorkPointer;
00766     while ( t < tend ) {
00767         if ( *t != funnum || t[1] == FUNHEAD ) { t += t[1]; continue; }
00768         flag = 1;
00769         while ( tt < t ) *w++ = *tt++;
00770         ts = t + t[1];
00771         t += FUNHEAD;
00772         while ( t < ts ) {
00773             ws = w;
00774             for ( i = 0; i < FUNHEAD; i++ ) *w++ = tt[i];
00775             if ( functions[*tt-FUNCTION].spec >= TENSORFUNCTION ) {
00776                 *w++ = *tt++;
00777             }
00778             else if ( *t < 0 ) {
00779                 if ( *t <= -FUNCTION ) *w++ = *tt++;
00780                 else { *w++ = *t++; *w++ = *t++; }
00781             }
00782             else {
00783                 i = *t; NCOPY(w,t,i);
00784             }
00785             ws[1] = w - ws;
00786         }
00787         tt = t;
00788     }
00789     if ( flag == 1 ) {
00790         ts = term + *term;
00791         while ( tt < ts ) *w++ = *tt++;
00792         *AT.WorkPointer = w - AT.WorkPointer;
00793         t = term; w = AT.WorkPointer; i = *w;
00794         NCOPY(t,w,i)
00795         AT.WorkPointer = term + *term;
00796         Normalize(BHEAD term);
00797     }
00798     return(0);
00799 }
00800
00801 /*
00802     #] ChainOut :
00803     #] Utilities :
00804     #[ Patterns :
00805         #[ MatchFunction :           WORD MatchFunction(pattern,interm,wilds)
00806
00807         The routine assumes that the function numbers are the same.
00808         The contents are compared and a possible wildcard assignment
00809         is made. Note that it may be necessary to use a wildcard
00810         assignment stack to do things right.
00811         The routine can become arbitrarily complicated as there is
00812         no end to the possible wilddarding.
00813         Examples:
00814         - a: No wilddarding -> straight match
00815         - b: Individual arguments (object -> object)
00816         - c: whole arguments (object to subexpression)
00817         - d: any argumentlist
00818         - e: part of an argument (object inside subexpression)
00819
00820         The ones with a minus sign in front have been implemented.
00821
00822         Note: the argument wilds allows backtracking when multiple
00823         ?a,?b give a match that later turns out to be useless.
00824 */
00825
00826 WORD MatchFunction(PHEAD WORD *pattern, WORD *interm, WORD *wilds)
00827 {
00828     GETBIDENTITY
00829     WORD *m, *t, *r, i;
00830     WORD *mstop = 0, *tstop = 0;
00831     WORD *argmstop, *argtstop;
00832     WORD *mtrmstop, *ttrmstop;
00833     WORD *msubstop, *mnextsub;
00834     WORD msizcoef, mcount, tcount, newvalue, j;
00835     WORD *oldm, *oldt;
00836     WORD *OldWork, numofwildarg;
00837     WORD nwstore, tobeeaten, reservevalue = 0, resernum = 0, withwild;
00838     WORD *wildargtaken;

```

```

00839     CBUF *C = cbuf+AT.ebufnum;
00840     int ntw = AN.NumTotWildArgs;
00841     LONG oldcpointer = C->Pointer - C->Buffer;
00842     /*
00843     Test first for a straight match
00844     */
00845     AN.RepFunList[AN.RepFunNum+1] = 0;
00846     if ( *wilds == 0 ) {
00847         m = pattern; t = interm;
00848
00849         if ( *m != *t ) {
00850             if ( *m < (FUNCTION + WILDOFFSET) ) return(0);
00851             if ( *t < FUNCTION ) return(0);
00852             if ( functions[*t-FUNCTION].spec !=
00853                 functions[*m-FUNCTION-WILDOFFSET].spec ) return(0);
00854         }
00855         i = m[1];
00856         if ( *m >= (FUNCTION + WILDOFFSET) ) { i--; m++; t++; }
00857         do { if ( *m++ != *t++ ) break; } while ( --i > 0 );
00858         if ( i <= 0 ) { /* Arguments match */
00859             if ( AN.SignCheck && AN.ExpectedSign ) return(0);
00860             i = *pattern - WILDOFFSET;
00861             if ( i >= FUNCTION ) {
00862                 if ( *interm != GAMMA
00863 #ifdef WITHFLOAT
00864                     && ( *interm != FLOATFUN )
00865 #endif
00866
00867                     && !CheckWild(BHEAD i,FUNTOFUN,*interm,&newvalue) ) {
00868                         AddWild(BHEAD i,FUNTOFUN,newvalue);
00869                         return(1);
00870                     }
00871                     return(0);
00872                 }
00873                 else return(1);
00874             }
00875         }
00876     /*
00877     Store the current Wildcard assignments
00878     */
00879     t = wildargtaken = OldWork = AT.WorkPointer;
00880     t += ntw;
00881     m = AN.WildValue;
00882     nwstore = i = (m[-SUBEXPSIZE+1]-SUBEXPSIZE)/4;
00883     if ( i > 0 ) {
00884         r = AT.WildMask;
00885         do {
00886             *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *r++;
00887         } while ( --i > 0 );
00888         *t++ = C->numrhs;
00889     }
00890     if ( t >= AT.WorkTop ) {
00891         MLOCK(ErrorMessageLock);
00892         MesWork();
00893         MUNLOCK(ErrorMessageLock);
00894         Terminate(-1);
00895     }
00896     AT.WorkPointer = t;
00897
00898     if ( *wilds ) {
00899         if ( *wilds == 1 ) goto endloop;
00900         else goto enloop; /* tensors = 2 */
00901     }
00902     m = pattern; t = interm;
00903     /*
00904     Single out the specials
00905     */
00906     if ( *t == GAMMA ) {
00907     /*
00908         #[ GAMMA :
00909
00910         For the gamma's we need to do two things:
00911         a: Find that there is a match
00912         b: Find where the match occurs in the string
00913         This last thing cannot be stored in the current conventions,
00914         but once the wildcard assignments have been made it is much
00915         easier to find it back.
00916         Alternative: replace the function number in the term temporarily
00917         by the offset inside the string. This makes things maybe easier.
00918     */
00919         if ( *m != GAMMA ) goto NoCaseB;
00920         i = t[1] - m[1];
00921         if ( m[1] == FUNHEAD+1 ) {
00922             if ( i ) goto NoCaseB;
00923             if ( m[FUNHEAD] < (AM.OffsetIndex+WILDOFFSET) ||
00924                 t[FUNHEAD] >= (AM.OffsetIndex+WILDOFFSET) ) goto NoCaseB;
00925

```

```

00926         if ( CheckWild(BHEAD m[FUNHEAD]-WILDOFFSET,INDTOIND,t[FUNHEAD],&newvalue) ) goto NoCaseB;
00927         AddWild(BHEAD m[FUNHEAD]-WILDOFFSET,INDTOIND,newvalue);
00928
00929         AT.WorkPointer = OldWork;
00930         if ( AN.SignCheck && AN.ExpectedSign ) return(0);
00931         return(1);          /* m was eaten. we have a match! */
00932     }
00933     if ( i < 0 ) goto NoCaseB; /* Pattern longer than target */
00934     mstop = m + m[1];
00935     tstop = t + t[1];
00936     m += FUNHEAD; t += FUNHEAD;
00937     if ( *m >= (AM.OffsetIndex+WILDOFFSET) && *t < (AM.OffsetIndex+WILDOFFSET) ) {
00938         if ( CheckWild(BHEAD *m-WILDOFFSET,INDTOIND,*t,&newvalue) ) goto NoCaseB;
00939         reservevalue = newvalue;
00940         withwild = 1;
00941         resernum = *m-WILDOFFSET;
00942         AddWild(BHEAD *m-WILDOFFSET,INDTOIND,newvalue);
00943     }
00944     else if ( *m != *t ) goto NoCaseB;
00945     else withwild = 0;
00946     m++; t++;
00947     oldm = m; argtstop = oldt = t;
00948     j = 0;          /* No wildcard assignments yet */
00949     while ( i >= 0 ) {
00950         if ( *m == *t ) {
00951             WithGamma: m++; t++;
00952             if ( m >= mstop ) {
00953                 if ( t < tstop && mstop < AN.patstop ) {
00954                     WORD k;
00955                     mnextsub = pattern + pattern[1];
00956                     k = *mnextsub;
00957                     while ( k == GAMMA && mnextsub[FUNHEAD]
00958                         != pattern[FUNHEAD] ) {
00959                         mnextsub += mnextsub[1];
00960                         if ( mnextsub >= AN.patstop ) goto FullOK;
00961                         k = *mnextsub;
00962                     }
00963                     if ( k >= FUNCTION ) {
00964                         if ( k > (FUNCTION + WILDOFFSET) ) k -= WILDOFFSET;
00965                         if ( functions[k-FUNCTION].commute ) goto NoGamma;
00966                     }
00967                 }
00968                 FullOK: if ( AN.SignCheck && AN.ExpectedSign ) goto NoGamma;
00969                 AN.RepFunList[AN.RepFunNum+1] = WORDDIF(oldt, argtstop);
00970                 return(1);
00971             }
00972             if ( t >= tstop ) goto NoCaseB;
00973         }
00974         else if ( *m >= (AM.OffsetIndex+WILDOFFSET)
00975             && *m < (AM.OffsetIndex + (WILDOFFSET<1)) && ( *t >= 0 ||
00976             *t < MINSPEC ) ) { /* Wildcard index */
00977             if ( !CheckWild(BHEAD *m-WILDOFFSET,INDTOIND,*t,&newvalue) ) {
00978                 AddWild(BHEAD *m-WILDOFFSET,INDTOIND,newvalue);
00979                 j = 1;
00980                 goto WithGamma;
00981             }
00982             else goto NoGamma;
00983         }
00984         else if ( *m < MINSPEC && *m >= (AM.OffsetVector+WILDOFFSET)
00985             && *t < MINSPEC ) { /* Wildcard vector */
00986             if ( !CheckWild(BHEAD *m-WILDOFFSET,VECTOVEC,*t,&newvalue) ) {
00987                 AddWild(BHEAD *m-WILDOFFSET,VECTOVEC,newvalue);
00988                 j = 1;
00989                 goto WithGamma;
00990             }
00991             else goto NoGamma;
00992         }
00993         else {
00994             NoGamma:
00995             if ( j ) { /* Undo wildcards */
00996                 m = AN.WildValue;
00997                 t = OldWork + AN.NumTotWildArgs; r = AT.WildMask; j = nwstore;
00998                 if ( j > 0 ) {
00999                     do {
01000                         *m++ = *t++; *m++ = *t++;
01001                         *m++ = *t++; *m++ = *t++; *r++ = *t++;
01002                     } while ( --j > 0 );
01003                     C->numrhs = *t++;
01004                     C->Pointer = C->Buffer + oldcpointer;
01005                 }
01006                 j = 0;
01007             }
01008             m = oldm; t = ++oldt; i--;
01009             if ( withwild ) {
01010                 AddWild(BHEAD resernum,INDTOIND,reservevalue);
01011             }
01012         }

```

```

01013     }
01014     goto NoCaseB;
01015 /*
01016     #] GAMMA :
01017     #[ Tensors :
01018 */
01019 }
01020 else if ( *t >= FUNCTION && functions[*t-FUNCTION].spec >= TENSORFUNCTION ) {
01021     mstop = m + m[1];
01022     tstop = t + t[1];
01023     mcount = 0;
01024     m += FUNHEAD;
01025     t += FUNHEAD;
01026     AN.WildArgs = 0;
01027     tcount = WORDDIFF(tstop,t);
01028     while ( m < mstop ) {
01029         if ( *m == FUNNYWILD ) { m++; AN.WildArgs++; }
01030         m++; mcount++;
01031     }
01032     tobeeaten = tcount - mcount + AN.WildArgs;
01033     if ( tobeeaten ) {
01034         if ( tobeeaten < 0 || AN.WildArgs == 0 ) {
01035             AT.WorkPointer = OldWork;
01036             return(0); /* Cannot match */
01037         }
01038     }
01039     AT.WildArgTaken[0] = AN.WildEat = tobeeaten;
01040     for ( i = 1; i < AN.WildArgs; i++ ) AT.WildArgTaken[i] = 0;
01041 toploop:
01042     numofwildarg = 0;
01043
01044     m = pattern; t = interm;
01045     mstop = m + m[1];
01046     if ( *m != *t ) {
01047         i = *m - WILDOFFSET;
01048         if ( CheckWild(BHEAD i,FUNTOFUN,*t,&newvalue) ) goto NoCaseB;
01049         AddWild(BHEAD i,FUNTOFUN,newvalue);
01050     }
01051     m += FUNHEAD;
01052     t += FUNHEAD;
01053     while ( m < mstop ) {
01054 /*
01055         First test for an exact match
01056 */
01057         if ( *m == *t ) { m++; t++; continue; }
01058 /*
01059         No exact match. Try ARGWILD
01060 */
01061         AN.argaddress = t;
01062         if ( *m == FUNNYWILD ) {
01063             tobeeaten = AT.WildArgTaken[numofwildarg++];
01064             if ( CheckWild(BHEAD m[1],ARGTOARG|EATTENSOR,tobeeaten,t) ) goto endloop;
01065             AddWild(BHEAD m[1],ARGTOARG|EATTENSOR,tobeeaten);
01066             m += 2;
01067             t += tobeeaten;
01068             continue;
01069         }
01070 /*
01071         Now the various cases:
01072 */
01073         i = *m;
01074         if ( i < MINSPEC ) {
01075             if ( *t != i ) {
01076                 if ( *t >= MINSPEC ) goto endloop;
01077                 i -= WILDOFFSET;
01078                 if ( i < AM.OffsetVector ) goto endloop;
01079                 if ( CheckWild(BHEAD i,VECTOVEC,*t,&newvalue) )
01080                     goto endloop;
01081                 AddWild(BHEAD i,VECTOVEC,newvalue);
01082             }
01083         }
01084         else if ( i >= AM.OffsetIndex ) { /* Index */
01085             if ( i < ( AM.OffsetIndex + WILDOFFSET ) ) goto endloop;
01086             if ( i >= ( AM.OffsetIndex + (WILDOFFSET«1) ) ) {
01087                 /* Summed over index */
01088                 goto endloop; /* For the moment */
01089             }
01090             i -= WILDOFFSET;
01091             if ( CheckWild(BHEAD i,INDTOIND,*t,&newvalue) )
01092                 goto endloop; /* Assignment not allowed */
01093             AddWild(BHEAD i,INDTOIND,newvalue);
01094         }
01095         else goto endloop;
01096         m++; t++;
01097     }
01098     if ( AN.SignCheck && AN.ExpectedSign ) goto endloop;
01099     AT.WorkPointer = OldWork;

```



```

01100         if ( AN.WildArgs > 1 ) *wilds = 2;
01101         return(1);          /* m was eaten. we have a match! */
01102
01103     endloop;;
01104     /*
01105         restore the current Wildcard assignments
01106     */
01107     i = nwstore;
01108     if ( i > 0 ) {
01109         m = AN.WildValue;
01110         t = OldWork + ntw; r = AT.WildMask;
01111         do {
01112             *m++ = *t++; *m++ = *t++; *m++ = *t++; *m++ = *t++; *r++ = *t++;
01113         } while ( --i > 0 );
01114         C->numrhs = *t++;
01115         C->Pointer = C->Buffer + oldcpointer;
01116     }
01117     enloop;;
01118     i = AN.WildArgs - 1;
01119     if ( i <= 0 ) {
01120         AT.WorkPointer = OldWork;
01121         return(0);
01122     }
01123     while ( --i >= 0 ) {
01124         if ( AT.WildArgTaken[i] == 0 ) {
01125             if ( i == 0 ) {
01126                 AT.WorkPointer = OldWork;
01127                 *wilds = 0;
01128                 return(0);
01129             }
01130         }
01131         else {
01132             (AT.WildArgTaken[i])--;
01133             numofwildarg = 0;
01134             for ( j = 0; j <= i; j++ ) {
01135                 numofwildarg += AT.WildArgTaken[j];
01136             }
01137             AT.WildArgTaken[j] = AN.WildEat-numofwildarg;
01138             for ( j++; j < AN.WildArgs; j++ ) AT.WildArgTaken[j] = 0;
01139             break;
01140         }
01141     }
01142     goto toploop;
01143     /*
01144         #] Tensors :
01145     */
01146 }
01147 /*
01148     Count the number of arguments. Either equal or an argument wildcard.
01149 */
01150 mstop = m + m[1];
01151 tstop = t + t[1];
01152 mcount = 0; tcount = 0;
01153 m += FUNHEAD; t += FUNHEAD;
01154 while ( t < tstop ) { tcount++; NEXTARG(t) }
01155 AN.WildArgs = 0;
01156 while ( m < mstop ) {
01157     mcount++;
01158     if ( *m == -ARGWILD ) AN.WildArgs++;
01159     NEXTARG(m)
01160 }
01161 tobeeaten = tcount - mcount + AN.WildArgs;
01162 if ( tobeeaten ) {
01163     if ( tobeeaten < 0 || AN.WildArgs == 0 ) {
01164         AT.WorkPointer = OldWork;
01165         return(0); /* Cannot match */
01166     }
01167 }
01168 /*
01169     Set up the array AT.WildArgTaken for the number of arguments that each
01170     wildarg eats.
01171 */
01172 AT.WildArgTaken[0] = AN.WildEat = tobeeaten;
01173 for ( i = 1; i < AN.WildArgs; i++ ) AT.WildArgTaken[i] = 0;
01174 topofloop:
01175 numofwildarg = 0;
01176 /*
01177     Test for single wildcard object/argument
01178 */
01179 m = pattern; t = interm;
01180 if ( *m != *t ) {
01181     i = *m - WILD_OFFSET;
01182     if ( CheckWild(BHEAD i,FUNTOFUN,*t,&newvalue) ) goto NoCaseB;
01183     AddWild(BHEAD i,FUNTOFUN,newvalue);
01184 }
01185 mstop = m + m[1];
01186 /* tstop = t + t[1]; */

```

```

01187     m += FUNHEAD;
01188     t += FUNHEAD;
01189     while ( m < mstop ) {
01190         argmstop = oldm = m;
01191         argtstop = oldt = t;
01192         NEXTARG(argmstop)
01193         NEXTARG(argtstop)
01194         if ( t == tstop ) { /* This concerns a very rare bug */
01195             if ( *m == -ARGWILD ) goto ArgAll;
01196             goto endofloop;
01197         }
01198         if ( *m < 0 && *t < 0 ) {
01199             if ( *t <= -FUNCTION ) {
01200                 if ( *t == *m ) {}
01201                 else if ( *m <= -FUNCTION-WILDOFFSET
01202                     && functions[-*t-FUNCTION].spec
01203                     == functions[-*m-FUNCTION-WILDOFFSET].spec ) {
01204                     i = -*m - WILDOFFSET;
01205                     if ( CheckWild(BHEAD i,FUNTOFUN,-*t,&newvalue) ) goto endofloop;
01206                     AddWild(BHEAD i,FUNTOFUN,newvalue);
01207                 }
01208                 else if ( *m == -SYMBOL && m[1] >= 2*MAXPOWER ) {
01209                     i = m[1] - 2*MAXPOWER;
01210                     AN.argaddress = AT.FunArg;
01211                     AT.FunArg[ARGHEAD+1] = -*t;
01212                     if ( CheckWild(BHEAD i,SYMTOSUB,1,AN.argaddress) ) goto endofloop;
01213                     AddWild(BHEAD i,SYMTOSUB,0);
01214                 }
01215                 else if ( *m == -ARGWILD ) {
01216 ArgAll:
01217                     i = AT.WildArgTaken[numofwildarg++];
01218                     AN.argaddress = t;
01219                     if ( CheckWild(BHEAD m[1],ARGTOARG,i,t) ) goto endofloop;
01220                     AddWild(BHEAD m[1],ARGTOARG,i);
01221                     m += 2; /*
01222                     while ( --i >= 0 ) { NEXTARG(t) }
01223                     argtstop = t;
01224                 }
01225                 else goto endofloop;
01226             }
01227             else if ( *t == *m ) {
01228                 if ( t[1] == m[1] ) {}
01229                 else if ( *t == -SYMBOL ) {
01230                     j = SYMTOSYM;
01231 SymAll:
01232                     if ( ( i = m[1] - 2*MAXPOWER ) < 0 ) goto endofloop;
01233                     if ( CheckWild(BHEAD i,j,t[1],&newvalue) ) goto endofloop;
01234                     AddWild(BHEAD i,j,newvalue);
01235                 }
01236                 else if ( *t == -INDEX ) {
01237 IndAll:
01238                     i = m[1] - WILDOFFSET;
01239                     if ( i < AM.OffsetIndex || i >= WILDOFFSET+AM.OffsetIndex )
01240                         goto endofloop;
01241                     /* We kill the summed over indices here */
01242                     if ( CheckWild(BHEAD i,INDTOIND,t[1],&newvalue) ) goto endofloop;
01243                     AddWild(BHEAD i,INDTOIND,newvalue);
01244                 }
01245                 else if ( *t == -VECTOR || *t == -MINVECTOR ) {
01246                     i = m[1] - WILDOFFSET;
01247                     if ( i < AM.OffsetVector ) goto endofloop;
01248                     if ( CheckWild(BHEAD i,VECTOVEC,t[1],&newvalue) ) goto endofloop;
01249                     AddWild(BHEAD i,VECTOVEC,newvalue);
01250                 }
01251                 else goto endofloop;
01252             }
01253             else if ( *m == -ARGWILD ) goto ArgAll;
01254             else if ( *m == -INDEX && m[1] >= AM.OffsetIndex+WILDOFFSET
01255                 && m[1] < AM.OffsetIndex+(WILDOFFSET<1) ) {
01256                 if ( *t == -VECTOR ) goto IndAll;
01257                 if ( *t == -SNUMBER && t[1] >= 0 && t[1] < AM.OffsetIndex ) goto IndAll;
01258                 if ( *t == -MINVECTOR ) {
01259                     i = m[1] - WILDOFFSET;
01260                     AN.argaddress = AT.MinVecArg;
01261                     AT.MinVecArg[ARGHEAD+3] = t[1];
01262                     if ( CheckWild(BHEAD i,INDTOSUB,1,AN.argaddress) ) goto endofloop;
01263                     AddWild(BHEAD i,INDTOSUB,(WORD)0);
01264                 }
01265                 else goto endofloop;
01266             }
01267             else if ( *m == -SYMBOL && m[1] >= 2*MAXPOWER && *t == -SNUMBER ) {
01268                 j = SYMTONUM;
01269                 goto SymAll;
01270             }
01271             else if ( *m == -VECTOR && *t == -MINVECTOR &&
01272                 ( i = m[1] - WILDOFFSET ) >= AM.OffsetVector ) {
01273 /*
01274 =====
01275 AN.argaddress = AT.MinVecArg;

```

```

01274         AT.MinVecArg[ARGHEAD+3] = t[1];
01275         if ( CheckWild(BHEAD i, VECTOSUB, 1, AN.argaddress) ) goto endofloop;
01276         AddWild(BHEAD i, VECTOSUB, (WORD)0);
01277 =====
01278 */
01279         if ( CheckWild(BHEAD i, VECTOMIN, t[1], &newvalue) ) goto endofloop;
01280         AddWild(BHEAD i, VECTOMIN, newvalue);
01281     }
01282 }
01283 else if ( *m == -MINVECTOR && *t == -VECTOR &&
01284 ( i = m[1] - WILDOFFSET ) >= AM.OffsetVector ) {
01285 /*
01286 =====
01287         AN.argaddress = AT.MinVecArg;
01288         AT.MinVecArg[ARGHEAD+3] = t[1];
01289         if ( CheckWild(BHEAD i, VECTOSUB, 1, AN.argaddress) ) goto endofloop;
01290         AddWild(BHEAD i, VECTOSUB, (WORD)0);
01291 =====
01292 */
01293         if ( CheckWild(BHEAD i, VECTOMIN, t[1], &newvalue) ) goto endofloop;
01294         AddWild(BHEAD i, VECTOMIN, newvalue);
01295     }
01296     else goto endofloop;
01297 }
01298 else if ( *t <= -FUNCTION && *m > 0 ) {
01299     if ( ( m[ARGHEAD]+ARGHEAD == *m ) && m[*m-1] == 3
01300 && m[*m-2] == 1 && m[*m-3] == 1 && m[ARGHEAD+1] >= FUNCTION
01301 && m[ARGHEAD+2] == *m-ARGHEAD-4 ) { /* Check for f(?a) etc */
01302         WORD *mmmst, *mmm;
01303         if ( m[ARGHEAD+1] >= FUNCTION+WILDOFFSET ) {
01304             /* i = *m - WILDOFFSET; */
01305             i = m[ARGHEAD+1] - WILDOFFSET;
01306             if ( CheckWild(BHEAD i, FUNTOFUN, -*t, &newvalue) ) goto endofloop;
01307             AddWild(BHEAD i, FUNTOFUN, newvalue);
01308         }
01309         else if ( m[ARGHEAD+1] != -*t ) goto endofloop;
01310     /*
01311         Only arguments allowed are ?a etc.
01312     */
01313     mmmst = m+*m-3;
01314     mmm = m + ARGHEAD + FUNHEAD + 1;
01315     while ( mmm < mmmst ) {
01316         if ( *mmm != -ARGWILD ) goto endofloop;
01317         i = 0;
01318         AN.argaddress = t;
01319         if ( CheckWild(BHEAD mmm[1], ARGTOARG, i, t) ) goto endofloop;
01320         AddWild(BHEAD mmm[1], ARGTOARG, i);
01321         mmm += 2;
01322     }
01323 }
01324     else goto endofloop;
01325 }
01326 else if ( *m < 0 && *t > 0 ) {
01327     if ( *m == -SYMBOL ) { /* SYMTOSUB */
01328         if ( m[1] < 2*MAXPOWER ) goto endofloop;
01329         i = m[1] - 2*MAXPOWER;
01330         AN.argaddress = t;
01331         if ( CheckWild(BHEAD i, SYMTOSUB, 1, AN.argaddress) ) goto endofloop;
01332         AddWild(BHEAD i, SYMTOSUB, 0);
01333     }
01334     else if ( *m == -VECTOR ) {
01335         if ( ( i = m[1] - WILDOFFSET ) < AM.OffsetVector )
01336             goto endofloop;
01337         AN.argaddress = t;
01338         if ( CheckWild(BHEAD i, VECTOSUB, 1, t) ) goto endofloop;
01339         AddWild(BHEAD i, VECTOSUB, (WORD)0);
01340     }
01341     else if ( *m == -INDEX ) {
01342         if ( ( i = m[1] - WILDOFFSET ) < AM.OffsetIndex ) goto endofloop;
01343         if ( i >= AM.OffsetIndex + WILDOFFSET ) goto endofloop;
01344         AN.argaddress = t;
01345         if ( CheckWild(BHEAD i, INDTO SUB, 1, AN.argaddress) ) goto endofloop;
01346         AddWild(BHEAD i, INDTO SUB, (WORD)0);
01347     }
01348     else if ( *m == -ARGWILD ) goto ArgAll;
01349     else goto endofloop;
01350 }
01351 else if ( *m > 0 && *t > 0 ) {
01352     WORD ii = *t-*m;
01353     i = *m;
01354     do { if ( *m++ != *t++ ) break; } while ( --i > 0 );
01355     if ( i == 1 && ii == 0 ) { /* sign difference */
01356         goto endofloop;
01357     }
01358     else if ( i > 0 ) {
01359         WORD *cto, *cfrom, *csav, ci;
01360         WORD oRepFunNum;

```

```

01361      WORD *oRepFunList;
01362      WORD *oterstart,*oterstop,*opatstop;
01363      WORD oExpectedSign;
01364      WORD wildargs, wildeat;
01365  /*
01366      Not an exact match here.
01367      We have to hope that the pattern contains a composite wildcard.
01368  */
01369      m = oldm; t = oldt;
01370      m += ARGHEAD; t += ARGHEAD;          /* Point at (first?) term */
01371      mtrmstop = m + *m;
01372      ttrmstop = t + *t;
01373      if ( mtrmstop < argmstop ) goto endofloop; /* More than one term */
01374      msizcoef = mtrmstop[-1];
01375      if ( msizcoef < 0 ) msizcoef = -msizcoef;
01376      msubstop = mtrmstop - msizcoef;
01377      m++;
01378      if ( m >= msubstop ) goto endofloop;    /* Only coefficient */
01379  /*
01380      Here we have a composite term. It can match provided it
01381      matches the entire argument. This argument must be a
01382      single term also and the coefficients should match
01383      (more or less).
01384      The matching takes:
01385      1: Match the functions etc. Nothing can be left.
01386      2: Match dotproducts and symbols. ONLY must match
01387         and nothing may be left.
01388      For safety it is best to take the term out and put it
01389      in workspace.
01390  */
01391
01392      if ( argtstop > ttrmstop ) goto endofloop;
01393      m--;
01394      oterstart = AN.terstart;
01395      oterstop = AN.terstop;
01396      opatstop = AN.patstop;
01397      oRepFunList = AN.RepFunList;
01398      oRepFunNum = AN.RepFunNum;
01399      AN.RepFunNum = 0;
01400      AN.RepFunList = AT.WorkPointer;
01401      AT.WorkPointer = (WORD *)(((UBYTE *) (AT.WorkPointer)) + AM.MaxTer);
01402      if ( AT.WorkPointer+*t+5 > AT.WorkTop ) {
01403          MLOCK(ErrorMessageLock);
01404          MesWork();
01405          MUNLOCK(ErrorMessageLock);
01406          return(-1);
01407      }
01408      csav = cto = AT.WorkPointer;
01409      cfrom = t;
01410      ci = *t;
01411      while ( --ci >= 0 ) *cto++ = *cfrom++;
01412      AT.WorkPointer = cto;
01413      ci = msizcoef;
01414      cfrom = mtrmstop;
01415      --ci;
01416      if ( abs(*--cfrom) != abs(*--cto) ) {
01417          AT.WorkPointer = csav;
01418          AN.RepFunList = oRepFunList;
01419          AN.RepFunNum = oRepFunNum;
01420          AN.terstart = oterstart;
01421          AN.terstop = oterstop;
01422          AN.patstop = opatstop;
01423          goto endofloop;
01424      }
01425      i = (*cfrom != *cto) ? 1 : 0; /* buffer AN.ExpectedSign until we are beyond the goto
01426  */
01427      while ( --ci >= 0 ) {
01428          if ( *--cfrom != *--cto ) {
01429              AT.WorkPointer = csav;
01430              AN.RepFunList = oRepFunList;
01431              AN.RepFunNum = oRepFunNum;
01432              AN.terstart = oterstart;
01433              AN.terstop = oterstop;
01434              AN.patstop = opatstop;
01435              goto endofloop;
01436          }
01437      }
01438      oExpectedSign = AN.ExpectedSign; /* buffer AN.ExpectedSign until we are beyond
FindRest/FindOnly */
01439      AN.ExpectedSign = i;
01440      *m -= msizcoef;
01441      wildargs = AN.WildArgs;
01442      wildeat = AN.WildEat;
01443      for ( i = 0; i < wildargs; i++ ) wildargtaken[i] = AT.WildArgTaken[i];
01444      AN.ForFindOnly = 0; AN.UseFindOnly = 1;
01445      AN.nogroundlevel++;
01446      if ( FindRest(BHEAD csav,m) && ( AN.UsedOtherFind || FindOnly(BHEAD csav,m) ) ) {}

```

```

01446         else {
01447 nomatch:
01448             *m += msizcoef;
01449             AT.WorkPointer = csav;
01450             AN.RepFunList = oRepFunList;
01451             AN.RepFunNum = oRepFunNum;
01452             AN.terstart = oterstart;
01453             AN.terstop = oterstop;
01454             AN.patstop = opatstop;
01455             AN.WildArgs = wildargs;
01456             AN.WildEat = wildeat;
01457             AN.ExpectedSign = oExpectedSign;
01458             AN.nogroundlevel--;
01459             for ( i = 0; i < wildargs; i++ ) AT.WildArgTaken[i] = wildargtaken[i];
01460             goto endofloop;
01461         }
01462 /*
01463 */
01463         if ( *m == 1 || m[1] < FUNCTION || functions[m[1]-FUNCTION].spec >= TENSORFUNCTION ) {
01464             if ( *m == 1 || m[1] < FUNCTION ) {
01465                 if ( AN.ExpectedSign ) goto nomatch;
01466             }
01467             else {
01468                 if ( m[1] > FUNCTION + WILDOFFSET ) {
01469                     if ( functions[m[1]-FUNCTION-WILDOFFSET].spec >= TENSORFUNCTION ) {
01470                         if ( AN.ExpectedSign != AN.RepFunList[AN.RepFunNum-1] ) goto nomatch;
01471                     }
01472                 }
01473                 else {
01474                     if ( AN.ExpectedSign != AN.RepFunList[AN.RepFunNum-1] ) goto nomatch;
01475                 }
01476                 if ( functions[m[1]-FUNCTION].spec >= TENSORFUNCTION ) {
01477                     if ( AN.ExpectedSign != AN.RepFunList[AN.RepFunNum-1] ) goto nomatch;
01478                 }
01479             }
01480         }
01481         AN.nogroundlevel--;
01482         AN.ExpectedSign = oExpectedSign;
01483         AN.WildArgs = wildargs;
01484         AN.WildEat = wildeat;
01485         for ( i = 0; i < wildargs; i++ ) AT.WildArgTaken[i] = wildargtaken[i];
01486         Substitute(BHEAD csav,m,1);
01487         cto = csav;
01488         cfrom = cto + *cto - msizcoef;
01489         cto++;
01490         *m += msizcoef;
01491         AT.WorkPointer = csav;
01492         AN.RepFunList = oRepFunList;
01493         AN.RepFunNum = oRepFunNum;
01494         AN.terstart = oterstart;
01495         AN.terstop = oterstop;
01496         AN.patstop = opatstop;
01497         if ( *cto != SUBEXPRESSION ) goto endofloop;
01498         cto += cto[1];
01499         if ( cto < cfrom ) goto endofloop;
01500     }
01501 }
01502 else goto endofloop;
01503
01504 t = argtstop; /* Next argument */
01505 m = argmstop;
01506 }
01507 if ( AN.SignCheck && AN.ExpectedSign ) goto endofloop;
01508 AT.WorkPointer = OldWork;
01509 if ( AN.WildArgs > 1 ) *wilds = 1;
01510 if ( AN.SignCheck && AN.ExpectedSign ) return(0);
01511 return(1); /* m was eaten. we have a match! */
01512
01513 endofloop;;
01514 /*
01515     restore the current Wildcard assignments
01516 */
01517 i = nwstore;
01518 if ( i > 0 ) {
01519     m = AN.WildValue;
01520     t = OldWork + ntw; r = AT.WildMask;
01521     do {
01522         *m++ = *t++; *m++ = *t++; *m++ = *t++; *m++ = *t++; *r++ = *t++;
01523     } while ( --i > 0 );
01524     C->numrhs = *t++;
01525     C->Pointer = C->Buffer + oldcpointer;
01526 }
01527
01528 endoloop;;
01529 i = AN.WildArgs-1;
01530 if ( i <= 0 ) {
01531     AT.WorkPointer = OldWork;

```

```

01532     return(0);
01533 }
01534 while ( --i >= 0 ) {
01535     if ( AT.WildArgTaken[i] == 0 ) {
01536         if ( i == 0 ) {
01537             AT.WorkPointer = OldWork;
01538             return(0);
01539         }
01540     }
01541     else {
01542         (AT.WildArgTaken[i])--;
01543         numofwildarg = 0;
01544         for ( j = 0; j <= i; j++ ) {
01545             numofwildarg += AT.WildArgTaken[j];
01546         }
01547         AT.WildArgTaken[j] = AN.WildEat-numofwildarg;
01548 /* ----> bug to be replaced in other source code */
01549         for ( j++; j < AN.WildArgs; j++ ) AT.WildArgTaken[j] = 0;
01550         break;
01551     }
01552 }
01553 goto topofloop;
01554 NoCaseB:
01555 /*
01556 Restore the old Wildcard assignments
01557 */
01558 i = nwstore;
01559 if ( i > 0 ) {
01560     m = AN.WildValue;
01561     t = OldWork + ntwa; r = AT.WildMask;
01562     do {
01563         *m++ = *t++; *m++ = *t++; *m++ = *t++; *m++ = *t++; *r++ = *t++;
01564     } while ( --i > 0 );
01565     C->numrhs = *t++;
01566     C->Pointer = C->Buffer + oldcpointer;
01567 }
01568 AT.WorkPointer = OldWork;
01569 return(0); /* no match */
01570 }
01571
01572 /*
01573 #] MatchFunction :
01574 #[ ScanFunctions :          WORD ScanFunctions(inpat,inter,par)
01575
01576 Finds in which functions to look for a match.
01577 inpat is the start of the pattern still to be matched.
01578 inter is the start of the term still to be matched.
01579 par gives information about commutativity.
01580     par = 0: nothing special
01581     par = 1: regular noncommuting function
01582     par = 2: GAMMA function
01583
01584 AN.patstop: end of the functions field in the search pattern
01585 AN.terstop: end of the functions field in the target pattern
01586 AN.terstart: address of entire term;
01587
01588 The actual matching of the functions and their arguments is done
01589 in a number of different routines. Mainly MatchFunction when there
01590 are no symmetry properties.
01591 Also: MatchE
01592       MatchCy
01593       FunMatchSy
01594       FunMatchCy
01595
01596 The main problem here is backtracking, ie continuing with wildcard
01597 possibilities when a first assignment doesn't work.
01598 Important note: this was completely forgotten in the symmetric
01599 functions till 6-jan-2009. As of the moment this still has to
01600 be fixed.  ?????21-mar-2023????? Is this still unfixed?????
01601
01602 Functions inside functions can cause problems when antisymmetric
01603 functions are involved. The sign of the term may be at stake.
01604 At the lowest level this is no problem but in f(-fas(n2,n1)) this
01605 plays a role. Next is when we have a product of functions inside
01606 an argument. The strategy must be that we test the sign only at the
01607 last function. Hence, when inpat+inpat[1] >= AN.patstop.
01608 We might relax that to the last antisymmetric function at a later stage.
01609
01610 New scheme to be implemented for non-commuting objects:
01611 When we are matching a second (or higher) function, any match can only
01612 be directly after the last matched non-commuting function or a commuting
01613 function. This will take care of whatever happens in MatchE etc.
01614 */
01615
01616 WORD ScanFunctions(PHEAD WORD *inpat, WORD *inter, WORD par)
01617 {
01618     GETBIDENTITY

```

```

01619     WORD i, *m, *t, *r, sym, psym;
01620     WORD *newpat, *newter, *instart, *oinpat = 0, *ointer = 0;
01621     WORD nwstore, offset, *OldWork, SetStop = 0, oRepFunNum = AN.RepFunNum;
01622     WORD wilds, wildargs = 0, wildeat = 0, *wildargtaken;
01623     WORD *Oterfirstcomm = AN.terfirstcomm;
01624     CBUF *C = cbuf+AT.ebufnum;
01625     int ntw = AN.NumTotWildArgs;
01626     LONG oldcpointer = C->Pointer - C->Buffer;
01627     WORD oldSignCheck = AN.SignCheck;
01628     instart = inter;
01629 /*
01630     Only active for the last function in the pattern.
01631     The actual test on the sign is in MatchFunction or the symmetric functions
01632 */
01633     if ( AN.nogroundlevel ) {
01634         AN.SignCheck = ( inpat + inpat[1] >= AN.patstop ) ? 1 : 0;
01635     }
01636     else {
01637         AN.SignCheck = 0;
01638     }
01639 /*
01640         Store the current Wildcard assignments
01641 */
01642     t = wildargtaken = OldWork = AT.WorkPointer;
01643     t += ntw;
01644     m = AN.WildValue;
01645     nwstore = i = (m[-SUBEXPSIZE+1]-SUBEXPSIZE)/4;
01646     if ( i > 0 ) {
01647         r = AT.WildMask;
01648         do {
01649             *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *r++;
01650         } while ( --i > 0 );
01651         *t++ = C->numrhs;
01652     }
01653     if ( t >= AT.WorkTop ) {
01654         MLOCK(ErrorMessageLock);
01655         MesWork();
01656         MUNLOCK(ErrorMessageLock);
01657         Terminate(-1);
01658     }
01659     AT.WorkPointer = t;
01660     do {
01661 #ifndef NEWCOMMUTE
01662 /*
01663         Find an eligible unsubstituted function
01664 */
01665         if ( AN.RepFunNum > 0 ) {
01666 /*
01667             First try a non-commuting function, just after the last
01668             substituted non-commuting function.
01669 */
01670             if ( *inter >= FUNCTION && functions[*inter-FUNCTION].commute ) {
01671                 do {
01672                     offset = WORDDIF(inter,AN.terstart);
01673                     for ( i = 0; i < AN.RepFunNum; i += 2 ) {
01674                         if ( AN.RepFunList[i] >= offset ) break;
01675                     }
01676                     if ( i >= AN.RepFunNum ) break;
01677                     inter += inter[1];
01678                 } while ( inter < AN.terfirstcomm );
01679                 if ( inter < AN.terfirstcomm ) { /* Check that it is directly after */
01680                     for ( i = 0; i < AN.RepFunNum; i += 2 ) {
01681                         if ( functions[AN.terstart[AN.RepFunList[i]]-FUNCTION].commute
01682                             && AN.RepFunList[i]+AN.terstart[AN.RepFunList[i]+1] == offset ) break;
01683                     }
01684                     if ( i < AN.RepFunNum ) goto trythis;
01685                 }
01686                 inter = AN.terfirstcomm;
01687             }
01688 /*
01689             Now try one of the commuting functions
01690 */
01691             while ( inter < AN.terstop ) {
01692                 offset = WORDDIF(inter,AN.terstart);
01693                 for ( i = 0; i < AN.RepFunNum; i += 2 ) {
01694                     if ( AN.RepFunList[i] == offset ) break;
01695                 }
01696                 if ( i >= AN.RepFunNum ) break;
01697                 inter += inter[1];
01698             }
01699             if ( inter >= AN.terstop ) goto Failure;
01700 trythis:;
01701         }
01702         else {
01703 /*
01704             The first function can be anywhere. We have no problems.
01705 */

```

```

01706         offset = WORDDIF(inter,AN.terstart);
01707     }
01708 #else
01709     /* first find an unsubstituted function */
01710     do {
01711         offset = WORDDIF(inter,AN.terstart);
01712         for ( i = 0; i < AN.RepFunNum; i += 2 ) {
01713             if ( AN.RepFunList[i] == offset ) break;
01714         }
01715         if ( i >= AN.RepFunNum ) break;
01716         inter += inter[1];
01717     } while ( inter < AN.terstop );
01718     if ( inter >= AN.terstop ) goto Failure;
01719 #endif
01720     wilds = 0;
01721     /* We found one */
01722     if ( *inter >= FUNCTION && *inpat >= FUNCTION ) {
01723         if ( *inpat == *inter || *inpat >= FUNCTION + WILDOFFSET ) {
01724             /*
01725                 if ( inter[1] == FUNHEAD ) goto rewild;
01726             */
01727             if ( functions[*inter-FUNCTION].spec >= TENSORFUNCTION
01728                 && ( *inter == *inpat ||
01729                     functions[*inpat-FUNCTION-WILDOFFSET].spec >= TENSORFUNCTION ) ) {
01730                 sym = functions[*inter-FUNCTION].symmetric & ~REVERSEORDER;
01731                 if ( *inpat == *inter ) psym = sym;
01732                 else psym = functions[*inpat-FUNCTION-WILDOFFSET].symmetric & ~REVERSEORDER;
01733                 if ( sym == ANTISYMMETRIC || sym == SYMMETRIC
01734                     || psym == SYMMETRIC || psym == ANTISYMMETRIC ) {
01735                     if ( sym == ANTISYMMETRIC && psym == SYMMETRIC ) goto rewild;
01736                     if ( sym == SYMMETRIC && psym == ANTISYMMETRIC ) goto rewild;
01737                 /*
01738                     Special function call for (anti)symmetric tensors
01739                 */
01740                 if ( MatchE(BHEAD inpat,inter,instart,par) ) goto OnSuccess;
01741             }
01742             else if ( sym == CYCLESYMMETRIC || sym == RCYCLESYMMETRIC
01743                 || psym == CYCLESYMMETRIC || psym == RCYCLESYMMETRIC ) {
01744                 /*
01745                     Special function call for (r)cyclic tensors
01746                 */
01747                 if ( MatchCy(BHEAD inpat,inter,instart,par) ) goto OnSuccess;
01748             }
01749             else goto rewild;
01750         }
01751         else if ( functions[*inter-FUNCTION].spec <= 0
01752             && ( *inter == *inpat ||
01753                 functions[*inpat-FUNCTION-WILDOFFSET].spec <= 0 ) ) {
01754             sym = functions[*inter-FUNCTION].symmetric & ~REVERSEORDER;
01755             if ( *inpat == *inter ) psym = sym;
01756             else psym = functions[*inpat-FUNCTION-WILDOFFSET].symmetric & ~REVERSEORDER;
01757             if ( psym == SYMMETRIC || sym == SYMMETRIC
01758                 /*
01759                     The next statement was commented out. Why????
01760                     Werkt nog niet. Tekan wordt nog niet bijgehouden.
01761                     5-nov-2001
01762                 */
01763                 || psym == ANTISYMMETRIC || sym == ANTISYMMETRIC
01764             ) {
01765                 if ( sym == ANTISYMMETRIC && psym == SYMMETRIC ) goto rewild;
01766                 if ( sym == SYMMETRIC && psym == ANTISYMMETRIC ) goto rewild;
01767                 if ( FunMatchSy(BHEAD inpat,inter,instart,par) ) goto OnSuccess;
01768             }
01769             else
01770                 if ( sym == CYCLESYMMETRIC || sym == RCYCLESYMMETRIC
01771                     || psym == CYCLESYMMETRIC || psym == RCYCLESYMMETRIC ) {
01772                 if ( FunMatchCy(BHEAD inpat,inter,instart,par) ) goto OnSuccess;
01773             }
01774             else goto rewild;
01775         }
01776         else goto rewild;
01777         AN.terfirstcomm = Oterfirstcomm;
01778     }
01779     else if ( par > 0 ) { SetStop = 1; goto maybenext; }
01780 }
01781 else {
01782 rewild:
01783     AN.terfirstcomm = Oterfirstcomm;
01784     if ( *inter != SUBEXPRESSION && MatchFunction(BHEAD inpat,inter,&wilds) ) {
01785         AN.terfirstcomm = Oterfirstcomm;
01786         if ( wilds ) {
01787             /*
01788                 Store wildcards to continue in MatchFunction if the current
01789                 wildcards do not work out.
01790             */
01791             wildargs = AN.WildArgs;
01792             wildeat = AN.WildEat;

```



```

01793         for ( i = 0; i < wildargs; i++ ) wildargtaken[i] = AT.WildArgTaken[i];
01794         oinpat = inpat; ointer = inter;
01795     }
01796     if ( par && *inter == GAMMA && AN.RepFunList[AN.RepFunNum+1] ) {
01797         SetStop = 1; goto NoMat;
01798     }
01799     if ( par == 2 ) {
01800         if ( *inter < FUNCTION || functions[*inter-FUNCTION].commute ) {
01801             goto NoMat;
01802         }
01803         par = 1;
01804     }
01805     AN.RepFunList[AN.RepFunNum] = offset;
01806     AN.RepFunNum += 2;
01807     newpat = inpat + inpat[1];
01808     if ( newpat >= AN.patstop ) {
01809         if ( AN.UseFindOnly == 0 ) {
01810             if ( FindOnce(BHEAD AN.findTerm,AN.findPattern) ) {
01811                 AN.UsedOtherFind = 1;
01812                 goto OnSuccess;
01813             }
01814             AN.RepFunNum -= 2;
01815             goto NoMat;
01816         }
01817         goto OnSuccess;
01818     }
01819     if ( *inter < FUNCTION || functions[*inter-FUNCTION].commute ) {
01820         newter = inter + inter[1];
01821         if ( newter >= AN.terstop ) goto Failure;
01822         if ( *inter == GAMMA && inpat[1] <
01823             inter[1] - AN.RepFunList[AN.RepFunNum-1] ) {
01824             if ( ScanFunctions(BHEAD newpat,newter,2) ) goto OnSuccess;
01825             AN.terfirstcomm = Oterfirstcomm;
01826         }
01827         else if ( *newter == SUBEXPRESSION ) {}
01828         else if ( functions[*inter-FUNCTION].commute ) {
01829             if ( ScanFunctions(BHEAD newpat,newter,1) ) goto OnSuccess;
01830             AN.terfirstcomm = Oterfirstcomm;
01831             if ( ( *newpat < (FUNCTION+WILDOFFSET)
01832                 && ( functions[*newpat-FUNCTION].commute == 0 ) ) ||
01833                 ( *newpat >= (FUNCTION+WILDOFFSET)
01834                 && ( functions[*newpat-FUNCTION-WILDOFFSET].commute == 0 ) ) ) {
01835                 newter = AN.terfirstcomm;
01836                 if ( newter < AN.terstop && ScanFunctions(BHEAD newpat,newter,1) ) goto
OnSuccess;
01837             }
01838         }
01839         else {
01840             if ( ScanFunctions(BHEAD newpat,instart,1) ) goto OnSuccess;
01841             AN.terfirstcomm = Oterfirstcomm;
01842         }
01843         SetStop = par;
01844     }
01845     else {
01846         /*
01847             Shouldn't this be newpat instead of inpat????
01848         */
01849         if ( par && inter > instart && ( ( *newpat < (FUNCTION+WILDOFFSET)
01850             && functions[*newpat-FUNCTION].commute ) ||
01851             ( *newpat >= (FUNCTION+WILDOFFSET)
01852             && functions[*newpat-FUNCTION-WILDOFFSET].commute ) ) ) {
01853             SetStop = 1;
01854         }
01855         else {
01856             newter = instart;
01857             if ( ScanFunctions(BHEAD newpat,newter,par) ) goto OnSuccess;
01858             AN.terfirstcomm = Oterfirstcomm;
01859         }
01860     }
01861     /*
01862         Restore the old Wildcard assignments
01863     */
01864     NoMat:
01865     i = nwstore;
01866     if ( i > 0 ) {
01867         m = AN.WildValue;
01868         t = OldWork + ntw; r = AT.WildMask;
01869         do {
01870             *m++ = *t++; *m++ = *t++; *m++ = *t++; *m++ = *t++; *r++ = *t++;
01871         } while ( --i > 0 );
01872         C->numrhs = *t++;
01873         C->Pointer = C->Buffer + oldcpointer;
01874     }
01875     /*
01876         AN.RepFunNum -= 2; */
01877     AN.RepFunNum = oRepFunNum;
01878     if ( wilds ) {
01879         inter = ointer; inpat = oinpat;

```

```

01879         AN.WildArgs = wildargs;
01880         AN.WildEat = wildeat;
01881         for ( i = 0; i < wildargs; i++ ) AT.WildArgTaken[i] = wildargtaken[i];
01882         goto rewild;
01883     }
01884     if ( SetStop ) break;
01885 }
01886 else if ( par ) {
01887 maybenext:
01888     if ( *inpat < (FUNCTION+WILDOFFSET) ) {
01889         if ( *inpat < FUNCTION ||
01890             functions[*inpat-FUNCTION].commute ) break;
01891     }
01892     else {
01893         if ( functions[*inpat-FUNCTION-WILDOFFSET].commute ) break;
01894     }
01895 }
01896 inter += inter[1];
01897 } while ( inter < AN.terstop );
01898 Failure:
01899     AN.SignCheck = oldSignCheck;
01900     AT.WorkPointer = OldWork;
01901     return(0);
01902 OnSuccess:
01903     if ( AT.idallflag && AN.nogroundlevel <= 0 ) {
01904         if ( AT.idallmaxnum > 0 && AT.idallnum >= AT.idallmaxnum ) {
01905             AN.terfirstcomm = Oterfirstcomm;
01906             AN.SignCheck = oldSignCheck;
01907             AT.WorkPointer = OldWork;
01908             return(0);
01909         }
01910         SubsInAll(BHEAD0);
01911         AT.idallnum++;
01912         if ( AT.idallmaxnum == 0 || AT.idallnum < AT.idallmaxnum ) goto NoMat;
01913     }
01914     AN.terfirstcomm = Oterfirstcomm;
01915     AN.SignCheck = oldSignCheck;
01916 /*
01917 Now the disorder test
01918 */
01919     if ( AN.DisOrderFlag && AN.RepFunNum >= 4 ) {
01920         WORD k, kk;
01921         for ( i = 2; i < AN.RepFunNum; i += 2 ) {
01922 /*
01923 -----> We still have to copy the code from Normalize wrt properorderflag
01924 */
01925             m = AN.terstart + AN.RepFunList[i-2];
01926             t = AN.terstart + AN.RepFunList[i];
01927             if ( *m != *t ) {
01928                 if ( *m > *t ) continue;
01929                 goto doesmatch;
01930             }
01931             if ( *m >= FUNCTION && functions[*m-FUNCTION].spec >=
01932                 TENSORFUNCTION ) {
01933                 k = m[1] - FUNHEAD;
01934                 kk = t[1] - FUNHEAD;
01935                 m += FUNHEAD;
01936                 t += FUNHEAD;
01937             }
01938             else {
01939                 k = m[1] - FUNHEAD;
01940                 kk = t[1] - FUNHEAD;
01941                 m += FUNHEAD;
01942                 t += FUNHEAD;
01943             }
01944             while ( k > 0 && kk > 0 ) {
01945                 if ( *m < *t ) goto NextFor;
01946                 else if ( *m++ > *t++ ) goto doesmatch;
01947                 k--; kk--;
01948             }
01949             if ( k > 0 ) goto doesmatch;
01950 NextFor:;
01951         }
01952         SetStop = 1;
01953         goto NoMat;
01954     }
01955 doesmatch:
01956     AT.WorkPointer = OldWork;
01957     return(1);
01958 }
01959
01960 /*
01961 #] ScanFunctions :
01962 #] Patterns :
01963 */

```

4.53 fwin.h File Reference

Macros

- `#define` [LINEFEED](#) `'\n'`
- `#define` [CARRIAGERETURN](#) `0x0D`
- `#define` [WITHRETURN](#)
- `#define` [WITHSYSTEM](#)
- `#define` [P_term](#)(code) `exit((int)(code<0?-code:code))`
- `#define` [SEPARATOR](#) `'\'`
- `#define` [ALTSEPARATOR](#) `'/'`
- `#define` [PATHSEPARATOR](#) `','`
- `#define` [WITH_ENV](#)

4.53.1 Detailed Description

Settings for Windows computers.

Definition in file [fwin.h](#).

4.53.2 Macro Definition Documentation

4.53.2.1 LINEFEED

```
#define LINEFEED '\n'
```

Definition at line [33](#) of file [fwin.h](#).

4.53.2.2 CARRIAGERETURN

```
#define CARRIAGERETURN 0x0D
```

Definition at line [34](#) of file [fwin.h](#).

4.53.2.3 WITHRETURN

```
#define WITHRETURN
```

Definition at line [35](#) of file [fwin.h](#).

4.53.2.4 WITHSYSTEM

```
#define WITHSYSTEM
```

Definition at line [37](#) of file [fwin.h](#).

4.53.2.5 P_term

```
#define P_term(  
    code ) exit((int) (code<0?-code:code))
```

Definition at line 39 of file [fwin.h](#).

4.53.2.6 SEPARATOR

```
#define SEPARATOR '\\'
```

Definition at line 41 of file [fwin.h](#).

4.53.2.7 ALTSEPARATOR

```
#define ALTSEPARATOR '/'
```

Definition at line 42 of file [fwin.h](#).

4.53.2.8 PATHSEPARATOR

```
#define PATHSEPARATOR ';' 
```

Definition at line 43 of file [fwin.h](#).

4.53.2.9 WITH_ENV

```
#define WITH_ENV
```

Definition at line 44 of file [fwin.h](#).

4.54 fwin.h

[Go to the documentation of this file.](#)

```
00001  
00006 /* #[ License : */  
00007 /*  
00008 * Copyright (C) 1984-2023 J.A.M. Vermaseren  
00009 * When using this file you are requested to refer to the publication  
00010 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025  
00011 * This is considered a matter of courtesy as the development was paid  
00012 * for by FOM the Dutch physics granting agency and we would like to  
00013 * be able to track its scientific use to convince FOM of its value  
00014 * for the community.  
00015 *  
00016 * This file is part of FORM.  
00017 *  
00018 * FORM is free software: you can redistribute it and/or modify it under the  
00019 * terms of the GNU General Public License as published by the Free Software  
00020 * Foundation, either version 3 of the License, or (at your option) any later  
00021 * version.  
00022 *  
00023 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY  
00024 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS  
00025 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more  
00026 * details.
```

```

00027  *
00028  *   You should have received a copy of the GNU General Public License along
00029  *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00030  */
00031  /* #] License : */
00032
00033 #define LINEFEED '\n'
00034 #define CARRIAGERETURN 0x0D
00035 #define WITHRETURN
00036
00037 #define WITHSYSTEM
00038
00039 #define P_term(code)    exit((int)(code<0?-code:code))
00040
00041 #define SEPARATOR '\\\
00042 #define ALTSEPARATOR '/'
00043 #define PATHSEPARATOR ';'
00044 #define WITH_ENV

```

4.55 gentopo.cc

```

00001 // { ( [
00002
00003 #ifdef HAVE_CONFIG_H
00004 #ifndef CONFIG_H_INCLUDED
00005 #define CONFIG_H_INCLUDED
00006 #include <config.h>
00007 #endif
00008 #endif
00009
00010 #include <iostream>
00011 #include <iomanip>
00012 #include <sstream>
00013 #include <cstdio>
00014 #include <stdlib.h>
00015
00016 #include "grcc.h"
00017 #include "gentopo.h"
00018
00019 #define DUMMYUSE(x) (void)(x)
00020
00021 using namespace std;
00022
00023 // The next limitation is imposed by the fact that the latest compilers
00024 // can give warnings on declarations like "int dtc11[nNodes]"
00025 // With a bit of overkill there should be no real problems.
00026 #define MAXNODES 100
00027 #define MAXNCLASSES 100
00028
00029 // Generate scalar connected Feynman graphs.
00030
00031 //=====
00032
00033 typedef int Bool;
00034 const int T_True = 1;
00035 const int T_False = 0;
00036
00037 // compile options
00038 const int CHECK = T_True;
00039 //const int MONITOR = T_True;
00040 const int MONITOR = T_False;
00041
00042 const int OPTPRINT = T_False;
00043
00044 const int DEBUG0 = T_False;
00045 //const int DEBUG1 = T_False;
00046 const int DEBUG = T_False;
00047
00048 // for debugging memory use
00049 #define DEBUGM T_False
00050
00051 //=====
00052 class T_MGraph;
00053 class T_EGraph;
00054
00055 #define Extern
00056 #define Global
00057 #define Local
00058
00059 // External functions
00060 Extern void toForm(T_EGraph *egraph);
00061 Extern int countPhiCon(int ex, int lp, int v4);
00062

```

```

00063 // Temporal definition: they should be replaced by FROM functions.
00064 Local BigInt factorial(int n);
00065 Local BigInt ipow(int n, int p);
00066
00067 // Local functions
00068 Local int nextPerm(int nelem, int nclass, int *cl, int *r, int *q, int *p, int count);
00069
00070 Local void erEnd(const char *msg);
00071 Local int *newArray(int size, int val);
00072 Local void deletArray(int *a);
00073 Local void copyArray(int size, int *a0, int *a1);
00074 Local int **newMat(int n0, int n1, int val);
00075 Local void deleteMat(int **m, int n0, int n1);
00076 Local void printArray(int n, int *p);
00077 Local void printMat(int n0, int n1, int **m);
00078
00079 #if DEBUGM
00080 static int countNMC = 0;
00081 static int countEG = 0;
00082 static int countMG = 0;
00083 #endif
00084
00085 //=====
00086 // class T_EGraph
00087
00088 T_EGraph::T_EGraph(int nnodes, int nedges, int mxdeg)
00089 {
00090     int j;
00091
00092     nNodes = nnodes;
00093     nEdges = nedges;
00094     maxdeg = mxdeg; // maximum value of degree of nodes
00095     nExtern = 0;
00096
00097     nodes = new T_ENode[nNodes];
00098     edges = new T_EEdge[nEdges+1];
00099     for (j = 0; j < nNodes; j++) {
00100         nodes[j].deg = 0;
00101         nodes[j].edges = new int[maxdeg];
00102     }
00103 #if DEBUGM
00104     printf("+++ new T_EGraph %d\n", ++countEG);
00105     if(countEG > 100) { exit(1); }
00106 #endif
00107 }
00108
00109 T_EGraph::~T_EGraph()
00110 {
00111     int j;
00112
00113     for (j = 0; j < nNodes; j++) {
00114         delete[] nodes[j].edges;
00115     }
00116     delete[] nodes;
00117     delete[] edges;
00118 #if DEBUGM
00119     printf("+++ delete T_EGraph %d\n", countEG--);
00120 #endif
00121 }
00122
00123 // construct T_EGraph from adjacency matrix
00124 void T_EGraph::init(int pid, long gid, int **adjmat, Bool sopi, BigInt nsm, BigInt esm)
00125 {
00126     int n0, n1, ed, e, eext, eint;
00127     // Bool ok;
00128
00129     pId = pid;
00130     gId = gid;
00131     opi = sopi;
00132     nsym = nsm;
00133     esym = esm;
00134
00135     for (n0 = 0; n0 < nNodes; n0++) {
00136         nodes[n0].deg = 0;
00137     }
00138     ed = 1;
00139     for (n0 = 0; n0 < nNodes; n0++) {
00140         for (e = 0; e < adjmat[n0][n0]/2; e++, ed++) {
00141             edges[ed].nodes[0] = n0;
00142             edges[ed].nodes[1] = n0;
00143             nodes[n0].edges[nodes[n0].deg++] = - ed;
00144             nodes[n0].edges[nodes[n0].deg++] = ed;
00145             edges[ed].ext = nodes[n0].ext;
00146         }
00147         for (n1 = n0+1; n1 < nNodes; n1++) {
00148             for (e = 0; e < adjmat[n0][n1]; e++, ed++) {
00149                 edges[ed].nodes[0] = n0;

```

```

00150         edges[ed].nodes[1] = n1;
00151         nodes[n0].edges[nodes[n0].deg++] = - ed;
00152         nodes[n1].edges[nodes[n1].deg++] =  ed;
00153         edges[ed].ext = (nodes[n0].ext || nodes[n1].ext);
00154     }
00155 }
00156 }
00157 if (CHECK) {
00158     if (ed != nEdges+1) {
00159         printf("*** T_EGraph::init: ed=%d != nEdges=%d\n", ed, nEdges);
00160         erEnd("*** T_EGraph::init: illegal connection\n");
00161     }
00162 }
00163
00164 // name of momenta
00165 eext = 1;
00166 eint = 1;
00167 for (ed = 1; ed <= nEdges; ed++) {
00168     if (edges[ed].ext) {
00169         edges[ed].momn = eext++;
00170         edges[ed].momc[0] = 'Q';
00171         edges[ed].momc[1] = ((char)0);
00172     } else {
00173         edges[ed].momn = eint++;
00174         edges[ed].momc[0] = 'P';
00175         edges[ed].momc[1] = ((char)0);
00176     }
00177 }
00178 }
00179
00180 // set external particle to node 'nd'
00181 void T_EGraph::setExtern(int nd, Bool val)
00182 {
00183     nodes[nd].ext = val;
00184 }
00185
00186 // end of calling 'setExtern'
00187 void T_EGraph::endSetExtern(void)
00188 {
00189     int n;
00190
00191     nExtern = 0;
00192     for (n = 0; n < nNodes; n++) {
00193         if (nodes[n].ext) {
00194             nExtern++;
00195         }
00196     }
00197 }
00198
00199 // print the T_EGraph
00200 void T_EGraph::print()
00201 {
00202     int nd, lg, ed, nlp;
00203
00204     nlp = nEdges - nNodes + 1;
00205     printf("\n");
00206     printf("T_EGraph: pId=%d gId=%d nExtern=%d nLoops=%d nNodes=%d nEdges=%d\n",
00207         pId, gId, nExtern, nlp, nNodes, nEdges);
00208     printf("      sym = (%ld * %ld) maxdeg=%d\n", nsym, esym, maxdeg);
00209     printf("      Nodes\n");
00210     for (nd = 0; nd < nNodes; nd++) {
00211         if (nodes[nd].ext) {
00212             printf("      %2d Extern ", nd);
00213         } else {
00214             printf("      %2d Vertex ", nd);
00215         }
00216         printf("deg=%d [", nodes[nd].deg);
00217         for (lg = 0; lg < nodes[nd].deg; lg++) {
00218             printf(" %3d", nodes[nd].edges[lg]);
00219         }
00220         printf("]\n");
00221     }
00222     printf("      Edges\n");
00223     for (ed = 1; ed <= nEdges; ed++) {
00224         if (edges[ed].ext) {
00225             printf("      %2d Extern ", ed);
00226         } else {
00227             printf("      %2d Intern ", ed);
00228         }
00229         printf("%s%d", (edges[ed].ext)? "Q": "p", edges[ed].momn);
00230         printf(" [%3d %3d]\n", edges[ed].nodes[1], edges[ed].nodes[0]);
00231     }
00232 }
00233
00234 //=====
00235 // class T_MNode : nodes in T_MGraph
00236

```

```

00237 T_MNode::T_MNode(int vid, int vdeg, Bool vext, int vclss)
00238 {
00239     id      = vid;    // id of the node
00240     deg     = vdeg;   // degree of the node
00241     freelg  = vdeg;   // number of free legs
00242     clss    = vclss;  // class to which the node belongs
00243     ext     = vext;   // external node or not
00244 }
00245
00246 //=====
00247 // class of node-classes for T_MGraph
00248 //
00249 class T_MNodeClass {
00250 public:
00251     int    nNodes;           // the number of nodes
00252     int    nClasses;        // the number of classes
00253     int    *clist;          // the number of nodes in each class
00254     int    *ndcl;           // node --> class
00255     int    **clmat;         // matrix used for classification
00256     int    *flist;          // the first node in each class
00257     int    maxdeg;          // maximal value of degree(node)
00258     int    forallignment;
00259
00260     T_MNodeClass(int nnodes, int ncl);
00261     ~T_MNodeClass();
00262     void init(int *cl, int mxdeg, int **adjmat);
00263     void copy(T_MNodeClass* mnc);
00264     int  clCmp(int nd0, int nd1, int cn);
00265     void printMat(void);
00266
00267     void mkFlist(void);
00268     void mkNdCl(void);
00269     void mkClMat(int **adjmat);
00270     void incMat(int nd, int td, int val);
00271     Bool chkOrd(int nd, int ndc, T_MNodeClass *cl, int *dtcl);
00272     int  cmpArray(int *a0, int *a1, int ma);
00273 };
00274
00275 T_MNodeClass::T_MNodeClass(int nnodes, int ncl)
00276 {
00277     nNodes    = nnodes;
00278     nClasses   = ncl;
00279     clist      = new int[nClasses];
00280     ndcl       = new int[nNodes];
00281     clmat      = newMat(nNodes, nClasses, 0);
00282     flist      = new int[nClasses+1];
00283
00284     #if DEBUGM
00285         printf("+++ new    T_MNodeClass %d\n", ++countNMC);
00286         if(countNMC > 100) { exit(1); }
00287     #endif
00288 }
00289
00290 T_MNodeClass::~T_MNodeClass()
00291 {
00292     delete[] clist;
00293     delete[] ndcl;
00294     deleteMat(clmat, nNodes, nClasses);
00295     delete[] flist;
00296
00297     #if DEBUGM
00298         printf("+++ delete T_MNodeClass %d\n", countNMC--);
00299     #endif
00300 }
00301
00302 void T_MNodeClass::init(int *cl, int mxdeg, int **adjmat)
00303 {
00304     int j;
00305
00306     for (j = 0; j < nClasses; j++) {
00307         clist[j] = cl[j];
00308     }
00309     maxdeg     = mxdeg;
00310     mkNdCl();
00311     mkClMat(adjmat);
00312     mkFlist();
00313 }
00314
00315 void T_MNodeClass::copy(T_MNodeClass* mnc)
00316 {
00317     int j, k;
00318
00319     for (k = 0; k < nClasses; k++) {
00320         clist[k] = mnc->clist[k];
00321     }
00322     maxdeg     = mnc->maxdeg;
00323     for (j = 0; j < nNodes; j++) {

```



```

00324     ndcl[j] = mnc->ndcl[j];
00325     for (k = 0; k < nClasses; k++) {
00326         clmat[j][k] = mnc->clmat[j][k];
00327     }
00328 }
00329 for (k = 0; k < nClasses+1; k++) {
00330     flist[k] = mnc->flist[k];
00331 }
00332 }
00333
00334 // Construct flist
00335 // The set of nodes in class 'cl' is [flist[cl],...,flist[cl+1]-1]
00336 void T_MNodeClass::mkFlist(void)
00337 {
00338     int j, f;
00339
00340     f = 0;
00341     for (j = 0; j < nClasses; j++) {
00342         flist[j] = f;
00343         f += clist[j];
00344     }
00345     flist[nClasses] = f;
00346 }
00347
00348 // Construct ndcl
00349 // ndcl[nd] = (the class id in which node 'nd' belongs)
00350 void T_MNodeClass::mkNdCl(void)
00351 {
00352     int c, k;
00353     int nd = 0;
00354
00355     for (c = 0; c < nClasses; c++) {
00356         for (k = 0; k < clist[c]; k++) {
00357             ndcl[nd++] = c;
00358         }
00359     }
00360 }
00361
00362 // Comparison of two nodes 'nd0' and 'nd1'
00363 // Ordering is lexicographic (class, connection configuration)
00364 int T_MNodeClass::clCmp(int nd0, int nd1, int cn)
00365 {
00366
00367     // Whether two nodes are in a same class or not.
00368     int cmp = ndcl[nd0] - ndcl[nd1];
00369     if (cmp != 0) {
00370         return cmp;
00371     }
00372
00373     // Sign '--' signifies the reverse ordering
00374     cmp = - cmpArray(clmat[nd0], clmat[nd1], cn);
00375     if (cmp != 0) {
00376         return cmp;
00377     }
00378     // for particles ???
00379     return cmp;
00380 }
00381
00382 // Construct a matrix 'clmat[nd][tc]' which is the number
00383 // of edges connecting 'nd' and all nodes in class 'tc'.
00384 void T_MNodeClass::mkClMat(int **adjmat)
00385 {
00386     int nd, td, tc;
00387
00388     for (nd = 0; nd < nNodes; nd++) {
00389         for (td = 0; td < nNodes; td++) {
00390             tc = ndcl[td]; // another node
00391             clmat[nd][tc] += adjmat[nd][td]; // the number of edges 'nd'--'td'
00392         }
00393     }
00394 }
00395 }
00396
00397 // Increase the number of edges by 'val' between 'nd' and 'td'.
00398 void T_MNodeClass::incMat(int nd, int td, int val)
00399 {
00400     int tdc = ndcl[td]; // class of 'td'
00401     clmat[nd][tdc] += val; // modify matrix 'clmat'.
00402 }
00403
00404 // Check whether the configuration satisfies the ordering condition or not.
00405 Bool T_MNodeClass::chkOrd(int nd, int ndc, T_MNodeClass *cl, int *dtcl)
00406 {
00407     Bool tcl[MAXNCCLASSES];
00408     // Bool tcl[cl->nClasses];
00409     int tn, tc, mxn, cmp, n;
00410     DUMMYUSE(nd);

```

```

00411
00412     for (tc = 0; tc < cl->nClasses; tc++) {
00413         tcl[tc] = T_False;
00414     }
00415     for (tn = 0; tn < nNodes; tn++) {
00416         if (dtcl[tn] == 0) {
00417             tcl[cl->ndcl[tn]] = T_False;
00418         }
00419     }
00420     for (tc = 0; tc < cl->nClasses; tc++) {
00421         if (tcl[tc] && ndc != tc) {
00422             mxn = flist[tc+1];
00423             for (n = flist[tc]+1; n < mxn; n++) {
00424                 cmp = - clmat[n-1][tc] + clmat[n][tc];
00425                 if (cmp > 0) {
00426                     return T_False;
00427                 }
00428             }
00429         }
00430     }
00431     return T_True;
00432 }
00433
00434 // Print configuration matrix.
00435 void T_MNodeClass::printMat(void)
00436 {
00437     int j1, j2;
00438
00439     cout << endl;
00440
00441     // the first line
00442     cout << setw(2) << "nd" << ": " << setw(2) << "cl: ";
00443     for (j2 = 0; j2 < nClasses; j2++) {
00444         cout << setw(2) << j2 << " ";
00445     }
00446     cout << endl;
00447
00448     // print raw
00449     for (j1 = 0; j1 < nNodes; j1++) {
00450         cout << setw(2) << j1 << ": " << setw(2) << ndcl[j1] << ": [";
00451         for (j2 = 0; j2 < nClasses; j2++) {
00452             cout << " " << setw(2) << clmat[j1][j2];
00453         }
00454         cout << "]" << endl;
00455     }
00456 }
00457
00458 int T_MNodeClass::cmpArray(int *a0, int *a1, int ma)
00459 {
00460     for (int j = 0; j < ma; j++) {
00461         if (a0[j] < a1[j]) {
00462             return -1;
00463         } else if (a0[j] > a1[j]) {
00464             return 1;
00465         }
00466     }
00467     return 0;
00468 }
00469
00470 //=====
00471 // class T_MGraph : scalar graph expressed by matrix form
00472
00473 T_MGraph::T_MGraph(int pid, int ncl, int *cldeg, int *clnum, int *clex, Bool sopi)
00474 {
00475     int nn, ne, j, k;
00476
00477     // initial conditions
00478     nClasses = ncl;
00479     clist = new int[ncl];
00480
00481     mindeg = -1;
00482     maxdeg = -1;
00483     ne = 0;
00484     nn = 0;
00485     for (j = 0; j < nClasses; j++) {
00486         clist[j] = clnum[j];
00487         nn += clnum[j];
00488         ne += cldeg[j]*clnum[j];
00489         if (mindeg < 0) {
00490             mindeg = cldeg[j];
00491         } else {
00492             mindeg = min(mindeg, cldeg[j]);
00493         }
00494         if (maxdeg < 0) {
00495             maxdeg = cldeg[j];
00496         } else {
00497             maxdeg = max(maxdeg, cldeg[j]);

```

```

00498     }
00499 }
00500 if (ne % 2 != 0) {
00501     printf("Sum of degrees are not even\n");
00502     for (j = 0; j < nClasses; j++) {
00503         printf("class %2d: %2d %2d %2d\n",
00504             j, cldeg[j], clnum[j], clext[j]);
00505     }
00506     erEnd("illegal degrees of nodes");
00507 }
00508 pid = pid;
00509 nNodes = nn;
00510 nodes = new T_MNode*[nNodes];
00511
00512 selOPI = sopi;
00513
00514 nEdges = ne / 2;
00515 nLoops = nEdges - nNodes + 1;
00516
00517 egraph = new T_EGraph(nNodes, nEdges, maxdeg);
00518 nn = 0;
00519 for (j = 0; j < nClasses; j++) {
00520     for (k = 0; k < clist[j]; k++, nn++) {
00521         nodes[nn] = new T_MNode(nn, cldeg[j], clext[j], j);
00522         egraph->setExtern(nn, clext[j]);
00523     }
00524 }
00525 egraph->endSetExtern();
00526
00527 // generated set of graphs
00528 ndiag = 0;
00529 nlPI = 0;
00530
00531 // the current graph
00532 adjMat = newMat(nNodes, nNodes, 0);
00533 nsym = ToBigInt(0);
00534 esym = ToBigInt(0);
00535 clPI = 0;
00536 wsum = ToFraction(0, 1);
00537 wsopi = ToFraction(0, 1);
00538
00539 // current node classification
00540 curcl = new T_MNodeClass(nNodes, nClasses);
00541
00542 // measures of efficiency
00543 ngen = 0;
00544 ngconn = 0;
00545
00546 if (MONITOR) {
00547     nCallRefine = 0;
00548     discardOrd = 0;
00549     discardRefine = 0;
00550     discardDisc = 0;
00551     discardIso = 0;
00552 }
00553
00554 // work space for isIsomorphic
00555 modmat = newMat(nNodes, nNodes, 0);
00556 permp = newArray(nNodes, 0);
00557 permq = newArray(nNodes, 0);
00558 permr = newArray(nNodes, 0);
00559
00560 // work space for bisearchM
00561 bidef = newArray(nNodes, 0);
00562 bilow = newArray(nNodes, 0);
00563 bicount = 0;
00564 DUMMYUSE(padding);
00565
00566 #if DEBUGM
00567     printf("+++ new T_MGraph %d\n", ++countMG);
00568     if(countEG > 100) { exit(1); }
00569 #endif
00570 }
00571
00572 T_MGraph::~T_MGraph()
00573 {
00574     int j;
00575
00576     deletArray(bilow);
00577     deletArray(bidef);
00578     deletArray(permr);
00579     deletArray(permq);
00580     deletArray(permp);
00581
00582     deleteMat(modmat, nNodes, nNodes);
00583     delete curcl;
00584     deleteMat(adjMat, nNodes, nNodes);

```

```

00585     delete egraph;
00586     for (j = 0; j < nNodes; j++) {
00587         delete nodes[j];
00588     }
00589     delete[] nodes;
00590     delete[] clist;
00591
00592     #if DEBUGM
00593         printf("+++ delete T_MGraph %d\n", countMG++);
00594     #endif
00595 }
00596
00597 void T_MGraph::printAdjMat(T_MNodeClass *cl)
00598 {
00599     int j1, j2;
00600
00601     cout << "      ";
00602     for (j2 = 0; j2 < nNodes; j2++) {
00603         cout << " " << setw(2) << j2;
00604     }
00605     cout << endl;
00606     for (j1 = 0; j1 < nNodes; j1++) {
00607         cout << setw(2) << j1 << ": [";
00608         for (j2 = 0; j2 < nNodes; j2++) {
00609             cout << " " << setw(2) << adjMat[j1][j2];
00610         }
00611         cout << "]" << cl->ndcl[j1] << endl;
00612     }
00613 }
00614 }
00615
00616 //-----
00617 // Check graph can be a connected one.
00618 // If a connected component without free leg is not the whole graph then
00619 // return T_False, otherwise return T_True.
00620 Bool T_MGraph::isConnected(void)
00621 {
00622     int j, n, nv;
00623
00624     for (j = 0; j < nNodes; j++) {
00625         nodes[j]->visited = -1;
00626     }
00627     if (visit(0)) {
00628         return T_True;
00629     }
00630     nv = 0;
00631     for (n = 0; n < nNodes; n++) {
00632         if (nodes[n]->visited >= 0) {
00633             nv++;
00634         }
00635     }
00636     return (nv == nNodes);
00637 }
00638
00639 // Visiting connected node used for 'isConnected'
00640 // If child nodes has free legs, then this function returns T_True.
00641 // otherwise it returns T_False.
00642 Bool T_MGraph::visit(int nd)
00643 {
00644     int td;
00645
00646     // This node has free legs.
00647     if (nodes[nd]->freelg > 0) {
00648         return T_True;
00649     }
00650     nodes[nd]->visited = 0;
00651     for (td = 0; td < nNodes; td++) {
00652         if ((adjMat[nd][td] > 0) and (nodes[td]->visited < 0)) {
00653             if (visit(td)) {
00654                 return T_True;
00655             }
00656         }
00657     }
00658     // all the child nodes has no free legs.
00659     return T_False;
00660 }
00661
00662 // Check whether the current graph is the Representative of a isomorphic class.
00663 // nsym = symmetry factor by the permutation of nodes.
00664 // esym = symmetry factor by the permutation of edge.
00665 // If this graph is not a representative, then returns T_False.
00666 Bool T_MGraph::isIsomorphic(T_MNodeClass *cl)
00667 {
00668     int j1, j2, cmp, count, nself;
00669
00670     nsym = ToBigInt(0);
00671     esym = ToBigInt(1);

```

```

00672
00673     count = 0;
00674     while (T_True) {
00675         count = nextPerm(nNodes, cl->nClasses, cl->clist,
00676             permr, permq, permp, count);
00677
00678         if (count < 0) {
00679             // calculate permutations of edges
00680             esym = ToBigInt(1);
00681             for (j1 = 0; j1 < nNodes; j1++) {
00682                 if (adjMat[j1][j1] > 0) {
00683                     nself = adjMat[j1][j1]/2;
00684                     esym *= factorial(nself);
00685                     esym *= ipow(2, nself);
00686                 }
00687                 for (j2 = j1+1; j2 < nNodes; j2++) {
00688                     if (adjMat[j1][j2] > 0) {
00689                         esym *= factorial(adjMat[j1][j2]);
00690                     }
00691                 }
00692             }
00693             return T_True;
00694         }
00695
00696         permMat(nNodes, permp, adjMat, modmat);
00697         cmp = compMat(nNodes, adjMat, modmat);
00698         if (cmp < 0) {
00699             return T_False;
00700         } else if (cmp == 0) {
00701             // save permutation ???
00702             nsym = nsym + ToBigInt(1);
00703         }
00704     }
00705 }
00706
00707 // apply permutation to matrix 'mat0' and obtain 'mat1'
00708 void T_MGraph::permMat(int size, int *perm, int **mat0, int **mat1)
00709 {
00710     int j1, j2;
00711
00712     for (j1 = 0; j1 < size; j1++) {
00713         for (j2 = 0; j2 < size; j2++) {
00714             mat1[j1][j2] = mat0[perm[j1]][perm[j2]];
00715         }
00716     }
00717 }
00718
00719 // comparison of matrix
00720 int T_MGraph::compMat(int size, int **mat0, int **mat1)
00721 {
00722     int j1, j2, cmp;
00723
00724     for (j1 = 0; j1 < size; j1++) {
00725         for (j2 = 0; j2 < size; j2++) {
00726             cmp = mat0[j1][j2] - mat1[j1][j2];
00727             if (cmp != 0) {
00728                 return cmp;
00729             }
00730         }
00731     }
00732     return 0;
00733 }
00734
00735
00736 // Refine the classification
00737 // cl : the current 'T_MNodeClass' object
00738 // cn : the class number
00739 // Returns (the new class number corresponds to 'cn') if OK, or
00740 // 'None' if ordering condition is not satisfied
00741 T_MNodeClass *T_MGraph::refineClass(T_MNodeClass *cl, int cn)
00742 {
00743     T_MNodeClass *ccl = cl;
00744     int ccn = cn;
00745     T_MNodeClass *ncl = NULL;
00746     int ucl[MAXNODES];
00747     int nucl, nce;
00748     int td, cmp;
00749
00750     nucl = 0;
00751     while (ccl->nClasses != nucl) { // repeat refinement.
00752         nce = 0;
00753         nucl = 0;
00754         for (td = 1; td < nNodes; td++) {
00755             // 'td' is the next node and the current node is 'td-1'.
00756             // Count up the number of the elements in the current class
00757             // corresponding to the current node
00758             nce++;

```

```

00759         cmp = ccl->clCmp(td-1, td, ccn);
00760
00761         // the ordering condition is not satisfied.
00762         if (cmp > 0) {
00763             if (DEBUG) {
00764                 cout << "refine: cls = " << cl->clist << endl;
00765                 cl->printMat();
00766                 cout << "clmat" << endl;
00767                 ccl->printMat();
00768                 cout << "refine: discard: cls = " << ccl->clist
00769                     << "ucl = " << ucl << endl;
00770             }
00771             if (ccl != cl) {
00772                 delete ccl;
00773             }
00774             return NULL;
00775
00776         } else if (cmp < 0) {
00777             // 'td' is in the next class to the current node.
00778             ucl[nucl++] = nce;          // close the current class
00779
00780             // start new class
00781             nce = 0;
00782         }
00783         // nothing to do for the case of 'cmp == 0'.
00784     }
00785
00786     // close array 'ucl'.
00787     ucl[nucl++] = nce + 1;
00788
00789     // class is not modified
00790     if (nucl == ccl->nClasses) {
00791         ncl = ccl;
00792         break;
00793     }
00794
00795     // inconsistent
00796     } else if (nucl < ccl->nClasses) {
00797         erEnd("refineClasses : smaller number of classes");
00798     }
00799
00800     // preparation of the next repetition.
00801     } else {
00802         if (cn == ccl->nClasses) {
00803             td = nNodes;
00804         } else {
00805             td = ccl->flist[cn+1]-1;
00806         }
00807         if (ccl != cl) {
00808             delete ccl;
00809         }
00810         ccl = new T_MNodeClass(nNodes, nucl);
00811         ccl->init(ucl, maxdeg, adjMat);
00812         if (td == nNodes) {
00813             ccn = ccl->nClasses;
00814         } else {
00815             ccn = ccl->ndcl[td];
00816         }
00817         nucl = 0;
00818     }
00819
00820     if (DEBUG) {
00821         cout << "refine: ucl = " << ucl << endl;
00822         cout << "refine: ncl = " << ncl->clist << endl;
00823         ncl->printMat();
00824     }
00825
00826     return ncl;
00827 }
00828
00829 // Search biconnected component
00830 // visit : pd --> nd --> td
00831 // ne : the number of edges between pd and nd.
00832 void T_MGraph::bisearchM(int nd, int pd, int ne)
00833 {
00834     int td;
00835
00836     bidef[nd] = bicount;
00837     bilow[nd] = bicount;
00838     bicount++;
00839
00840     for (td = 0; td < nNodes; td++) {
00841         if (nodes[td]->ext) { // ignore external node
00842             continue;
00843         } else if (td == nd) { // pd --> nd --> nd
00844             continue;
00845         }

```

```

00846
00847     } else if (td == pd) {          // pd --> nd --> pd
00848         if (ne > 1) {
00849             bilow[nd] = min(bilow[nd], bidef[pd]);
00850         }
00851
00852     } else if (adjMat[td][nd] < 1) { // td is not adjacent to nd
00853         continue;
00854
00855     } else if (bidef[td] >= 0) { // back edge
00856         bilow[nd] = min(bilow[nd], bidef[td]);
00857
00858     // new node
00859     } else {
00860
00861         bisearchM(td, nd, adjMat[td][nd]);
00862
00863         // ordinary case
00864         if (bilow[td] > bidef[nd]) {
00865             nBridges++;
00866         }
00867         bilow[nd] = min(bilow[nd], bilow[td]);
00868     }
00869 }
00870
00871 // nd is the starting point and not an external line
00872 // if (pd < 0 && (!nodes[nd]->ext || nodes[nd]->deg > 1)) {
00873 // if (pd < 0 && !nodes[nd]->ext && nodes[nd]->deg == 1) {
00874 if (pd < 0 && nodes[nd]->deg != 1) {
00875     // nBridges += nodes[nd]->deg;
00876     nBridges++;
00877 }
00878 }
00879
00880 // Count the number of lPI components.
00881 // The algorithm is described in
00882 // A.V. Aho, J.E. Hopcroft and J.D. Ullman
00883 // 'The Design and Analysis of Computer Algorithms', Chap. 5
00884 // 1974, Addison-Wesley.
00885
00886 int T_MGraph::countlPI(void)
00887 {
00888     int j;
00889
00890     if (nLoops < 0) {
00891         return 1;
00892     }
00893
00894     // initialization
00895     bicount = 0;
00896     nBridges = 0;
00897     for (j = 0; j < nNodes; j++) {
00898         bidef[j] = -1;
00899         bilow[j] = -1;
00900     }
00901
00902     bisearchM(0, -1, 0);
00903
00904     return nBridges;
00905 }
00906
00907 //-----
00908 // Generate graphs
00909 // the generation process starts from 'connectClass'.
00910
00911 long T_MGraph::generate(void)
00912 {
00913     T_MNodeClass *cl;
00914     int dscl[MAXNODES];
00915     int n;
00916
00917     for (n = 0; n < nNodes; n++) {
00918         dscl[n] = T_False;
00919     }
00920
00921     // Initial classification of nodes.
00922     cl = new T_MNodeClass(nNodes, nClasses);
00923     cl->init(clist, maxdeg, adjMat);
00924     connectClass(cl, dscl);
00925
00926     // Print the result.
00927
00928     if (MONITOR) {
00929         cout << endl;
00930         cout << endl;
00931         cout << "* Total " << ndiag << " Graphs.";
00932         cout << "(" << n1PI << " lPI)";

```

```

00933 // cout << " wsum = " << wsum << "(" << wsopi << "lPI)" << endl;
00934 cout << endl;
00935 }
00936
00937 if (MONITOR) {
00938     cout << "* refine: " << nCallRefine << endl;
00939     cout << "* discard for ordering: " << discardOrd << endl;
00940     cout << "* discard for refinement: " << discardRefine << endl;
00941     cout << "* discard for disconnected: " << discardDisc << endl;
00942     cout << "* discard for duplication: " << discardIso << endl;
00943 }
00944 delete cl;
00945
00946 return ndiag;
00947 }
00948
00949 int T_MGraph::findNextCl(T_MNodeClass *cl, int *dscl)
00950 {
00951     int mine, cr, c, n, me;
00952
00953     mine = -1;
00954     cr = -1;
00955     for (c = 0; c < cl->nClasses; c++) {
00956         n = cl->flist[c];
00957         if (!dscl[n]) {
00958             me = nodes[n]->freelg;
00959             if (me > 0) {
00960                 if (nodes[n]->freelg < nodes[n]->deg) {
00961                     return c;
00962                 }
00963                 if (mine < 0 || mine > me) {
00964                     mine = me;
00965                     cr = c;
00966                 }
00967             }
00968         }
00969     }
00970     return cr;
00971 }
00972
00973 int T_MGraph::findNextTCl(T_MNodeClass *cl, int *dtcl)
00974 {
00975     int c, n, mine, cr, me;
00976
00977     mine = -1;
00978     cr = -1;
00979     for (c = 0; c < cl->nClasses; c++) {
00980         for (n = cl->flist[c]; n < cl->flist[c+1]; n++) {
00981             if (!dtcl[n]) {
00982                 me = nodes[n]->freelg;
00983                 if (me > 0) {
00984                     if (nodes[n]->freelg < nodes[n]->deg) {
00985                         return c;
00986                     }
00987                     if (mine < 0 || mine > me) {
00988                         mine = me;
00989                         cr = c;
00990                     }
00991                 }
00992             }
00993         }
00994     }
00995     return cr;
00996 }
00997
00998 // Connect nodes in a class to others
00999 void T_MGraph::connectClass(T_MNodeClass *cl, int *dscl)
01000 {
01001     int sc, sn;
01002     T_MNodeClass *xcl;
01003
01004     if (DEBUG0) {
01005         printf("connectClass:begin:");
01006         printf(" dscl="); printArray(nNodes, dscl);
01007         printf("\n");
01008     }
01009     xcl = refineClass(cl, cl->nClasses);
01010
01011     if (xcl == NULL) {
01012         if (MONITOR) {
01013             discardRefine++;
01014         }
01015     } else {
01016         sc = findNextCl(xcl, dscl);
01017         if (sc < 0) {
01018             newGraph(cl);
01019         } else {

```



```

01020         sn = xcl->flist[sc];
01021         connectNode(sc, sn, xcl, dscl);
01022     }
01023 }
01024 if (xcl != cl && xcl != NULL) {
01025     delete xcl;
01026 }
01027 if (DEBUG0) {
01028     printf("connectClass:end\n");
01029 }
01030 }
01031
01032 //-----
01033 void T_MGraph::connectNode(int sc, int ss, T_MNodeClass *cl, int *dscl)
01034 {
01035     int sn;
01036     int dtcl[MAXNODES];
01037
01038     if (DEBUG0) {
01039         printf("connectNode:begin:(%d,%d)", sc, ss);
01040         printf(" dscl="); printArray(nNodes, dscl);
01041         printf("\n");
01042     }
01043     if (ss >= cl->flist[sc+1]) {
01044         connectClass(cl, dscl);
01045         if (DEBUG0) {
01046             printf("connectNode:end1\n");
01047         }
01048         return;
01049     }
01050
01051     copyArray(nNodes, dscl, dtcl);
01052     for (sn = ss; sn < cl->flist[sc+1]; sn++) {
01053         if (!dscl[sn]) {
01054             connectLeg(sc, sn, sc, sn, cl, dscl, dtcl);
01055             if (DEBUG0) {
01056                 printf("connectNode:end2\n");
01057             }
01058             return;
01059         }
01060     }
01061     if (DEBUG0) {
01062         printf("connectNode:end3\n");
01063     }
01064 }
01065
01066 // Add one connection between two legs.
01067 // 1. select another node to connect
01068 // 2. determine multiplicity of the connection
01069 //
01070 // Arguments
01071 //   cn : the current class
01072 //   nd : the current node to be connected.
01073 //   nextnd : {nextnd, ...} is the possible target node of the connection.
01074 //   cl   : the current node class
01075
01076 void T_MGraph::connectLeg(int sc, int sn, int tc, int ts, T_MNodeClass *cl, int *dscl, int* dtcl)
01077 {
01078     int tn, maxself, nc2, nc, maxcon, ts1, wc, ncm;
01079     int dtcl1[MAXNODES];
01080
01081     if (DEBUG0) {
01082         printf("connectLeg:begin:(%d,%d,%d,%d)", sc, sn, tc, ts);
01083         printf(" dscl="); printArray(nNodes, dscl);
01084         printf(" dtcl="); printArray(nNodes, dtcl);
01085         printf("\n");
01086         printAdjMat(cl);
01087     }
01088
01089     if (sn >= cl->flist[sc+1]) {
01090         erEnd("*** connectLeg : illegal control");
01091         return;
01092     }
01093
01094     // There remains no free legs in the node 'sn' : move to next node.
01095     } else if (nodes[sn]->freelg < 1) {
01096
01097         if (!isConnected()) {
01098             if (DEBUG0) {
01099                 printf("connectLeg:disconnected\n");
01100             }
01101             if (MONITOR) {
01102                 discardDisc++;
01103             }
01104         } else {
01105             if (DEBUG0) {
01106                 printf("connectLeg: call conNode\n");
01107             }
01108         }
01109     }
01110 }

```

```

01107         dscl[sn] = T_True;
01108
01109         // next node in the current class.
01110         connectNode(sc, sn+1, cl, dscl);
01111
01112         dscl[sn] = T_False;
01113     }
01114
01115     // connect a free leg of the current node 'sn'.
01116 } else {
01117     copyArray(nNodes, dtcl, dtcl1);
01118
01119     ts1 = ts;
01120     for (wc = 0; wc < nNodes; wc++) {
01121         if (ts1 >= cl->flist[tc+1]) {
01122             tc = findNextTcl(cl, dtcl1);
01123             if (DEBUG0) {
01124                 printf("connectLeg:1:tc=%d\n", tc);
01125             }
01126             if (tc < 0) {
01127                 if (DEBUG0) {
01128                     printf("connectLeg:end1:(%d,%d,%d,%d)\n", sc, sn, tc, ts);
01129                 }
01130                 return;
01131             }
01132             if (tc != sc && !cl->chkOrd(sn, sc, cl, dtcl)) {
01133                 if (MONITOR) {
01134                     discardOrd++;
01135                 }
01136                 if (DEBUG0) {
01137                     printf("connectLeg:end2:(%d,%d,%d,%d)\n", sc, sn, tc, ts);
01138                 }
01139                 return;
01140             }
01141         }
01142     }
01143     ts1 = -1;
01144
01145     // repeat for all possible target nodes
01146     // for all tn in tc
01147     if (DEBUG0) {
01148         printf("connectLeg:2:tc=%d, fl:%d-->%d\n", tc, cl->flist[tc], cl->flist[tc+1]);
01149     }
01150     for (tn = cl->flist[tc]; tn < cl->flist[tc+1]; tn++) {
01151         if (DEBUG0) {
01152             printf("connectLeg:2:%d=>try %d\n", sn, tn);
01153         }
01154     }
01155     if (dtcl1[tn]) {
01156         if (DEBUG0) {
01157             printf("connectLeg:3\n");
01158         }
01159         continue;
01160     }
01161     dtcl1[tn] = T_True;
01162
01163     if (sc == tc && sn > tn) {
01164         if (DEBUG0) {
01165             printf("connectLeg:4\n");
01166         }
01167         continue;
01168     }
01169     // self-loop
01170 } else if (sn == tn) {
01171     if (nNodes > 1) {
01172         // there are two or more nodes in the graph :
01173         // avoid disconnected graph
01174         maxxself = min((nodes[sn]->freelg)/2, (nodes[sn]->deg-1)/2);
01175     } else {
01176         // there is only one node the graph.
01177         maxxself = nodes[sn]->freelg/2;
01178     }
01179
01180     // If we can assume no tadpole, the following line can be used.
01181     if (selOPI and nNodes > 2) {
01182         maxxself = min((nodes[sn]->deg-2)/2, maxxself);
01183     }
01184
01185     // vary the number of connection.
01186     for (nc2 = maxxself; nc2 > 0; nc2--) {
01187         // if nNodes > 1 and ndg == nc2:
01188         // // disconnected
01189         // continue
01190         nc = 2*nc2;
01191         if (DEBUG0) {
01192             printf("connectLeg: call conLeg: (same) %d=>%d(%d)\n", sn, tn, nc);
01193         }
01194     }

```

```

01194         ncm = 5*nc;
01195         adjMat[sn][sn] = nc;
01196         nodes[sn]->freelg -= nc;
01197         cl->incMat(sn, tn, ncm);
01198         ts1 = tn + 1;
01199
01200         // next connection
01201         connectLeg(sc, sn, tc, ts1, cl, dscl, dtcl1);
01202
01203         // restore the configuration
01204         cl->incMat(tn, sn, - ncm);
01205         adjMat[sn][sn] = 0;
01206         nodes[sn]->freelg += nc;
01207         if (DEBUG0) {
01208             printf("connectLeg: ret conLeg: (same) %d=>%d(%d)\n", sn, tn,nc);
01209         }
01210     }
01211
01212     // connections between different nodes.
01213 } else {
01214     // maximum possible connection number
01215     maxcon = min(nodes[sn]->freelg, nodes[tn]->freelg);
01216
01217     // avoid disconnected graphs.
01218     if (nNodes > 2 && nodes[sn]->deg == nodes[tn]->deg) {
01219         maxcon = min(maxcon, nodes[sn]->deg-1);
01220     }
01221
01222     if (CHECK) {
01223         if ((adjMat[sn][tn] != 0) || (adjMat[sn][tn] != adjMat[tn][sn])) {
01224             printf("*** inconsistent connection: sn=%d, tn=%d", sn, tn);
01225             printf(" dscl="); printArray(nNodes, dscl);
01226             printf(" dtcl="); printArray(nNodes, dtcl);
01227             printf("\n");
01228             printAdjMat(cl);
01229             erEnd("*** inconsistent connection ");
01230         }
01231     }
01232
01233     // vary number of connections
01234     for (nc = maxcon; nc > 0; nc--) {
01235         if (DEBUG0) {
01236             printf("connectLeg: call conLeg: (diff) %d=>%d(%d)", sn, tn,nc);
01237             printf(" dtcl="); printArray(nNodes, dtcl);
01238             printf("\n");
01239         }
01240         adjMat[sn][tn] = nc;
01241         adjMat[tn][sn] = nc;
01242         nodes[sn]->freelg -= nc;
01243         nodes[tn]->freelg -= nc;
01244         cl->incMat(sn, tn, nc);
01245         cl->incMat(tn, sn, nc);
01246         ts1 = tn + 1;
01247
01248         // next connection
01249         connectLeg(sc, sn, tc, ts1, cl, dscl, dtcl1);
01250
01251         // restore configuration
01252         cl->incMat(sn, tn, - nc);
01253         cl->incMat(tn, sn, - nc);
01254         adjMat[sn][tn] = 0;
01255         adjMat[tn][sn] = 0;
01256         nodes[sn]->freelg += nc;
01257         nodes[tn]->freelg += nc;
01258         if (DEBUG0) {
01259             printf("connectLeg: ret conLeg: (diff) %d=>%d(%d)\n", sn, tn,nc);
01260             printf(" dtcl="); printArray(nNodes, dtcl);
01261             printAdjMat(cl);
01262             printf("\n");
01263         }
01264     }
01265 }
01266 }
01267 if (ts1 < 0) {
01268     ts1 = cl->flist[tc+1];
01269 }
01270 } // for wc
01271 }
01272 if (DEBUG0) {
01273     printf("connectLeg:end3:(%d,%d,%d,%d)\n", sc, sn, tc, ts);
01274 }
01275 }
01276
01277 // A new candidate diagram is obtained.
01278 // It may be a disconnected graph
01279
01280 void T_MGraph::newGraph(T_MNodeClass *cl)

```

```

01281 {
01282     int connected;
01283     T_MNodeClass *xcl;
01284     Bool sopi;
01285
01286     ngen++;
01287     // printf("newGraph: %d\n", ngen);
01288
01289     // refine class and check ordering condition
01290     xcl = refineClass(cl, cl->nClasses);
01291     if (xcl == NULL) {
01292         // printf("newGraph: fail refine\n");
01293         if (MONITOR) {
01294             discardRefine++;
01295         }
01296
01297         // check whether this is a connected graph or not.
01298     } else {
01299         connected = isConnected();
01300         if (!connected) {
01301             // printf("newGraph: not connected\n");
01302             if (DEBUG0) {
01303                 cout << "+++ disconnected graph" << ngen << endl;
01304                 xcl->printMat();
01305             }
01306
01307             if (MONITOR) {
01308                 discardDisc++;
01309             }
01310
01311             // connected graph : check isomorphism of the graph
01312         } else {
01313             ngconn++;
01314             if (!isIsomorphic(xcl)) {
01315                 // printf("newGraph: not isomorphic\n");
01316                 if (DEBUG) {
01317                     cout << "+++ duplicated graph" << ngen << ngconn << endl;
01318                     xcl->printMat();
01319                 }
01320
01321                 if (MONITOR) {
01322                     discardIso++;
01323                 }
01324
01325                 // We got a new connected and a unique representation of a class.
01326             } else {
01327
01328                 clPI = count1PI();
01329                 if (!selOPI || clPI == 1) {
01330                     wsum = wsum + ToFraction(1, nsym*esym);
01331                     if (clPI == 1) {
01332                         wsopi = wsopi + ToFraction(1, nsym*esym);
01333                     }
01334                     curcl->copy(xcl);
01335                     ndiag++;
01336                     sopi = (clPI == 1);
01337                     if (sopi) {
01338                         n1PI++;
01339                     }
01340
01341                     if (OPTPRINT) {
01342                         cout << endl;
01343                         cout << "Graph : " << ndiag
01344                             << "(" << ngen << ")"
01345                             << " 1PI comp. = " << clPI
01346                             << " sym. factor = (" << nsym << "*" << esym << ")"
01347                             << endl;
01348                         printAdjMat(cl);
01349                         // cl->printMat();
01350                         if (MONITOR) {
01351                             cout << "refine: " << nCallRefine << endl;
01352                             cout << "discard for ordering: " << discardOrd << endl;
01353                             cout << "discard for refinement: " << discardRefine << endl;
01354                             cout << "discard for disconnected: " << discardDisc << endl;
01355                             cout << "discard for duplication: " << discardIso << endl;
01356                         }
01357                     }
01358
01359                     // go to next step
01360                     egraph->init(pId, ndiag, adjMat, sopi, nsym, esym);
01361                     toForm(egraph);
01362                 }
01363             }
01364         }
01365     }
01366     // printf("in newGraph cl=%p, xcl=%p\n", cl, xcl);
01367     if (xcl != cl && xcl != NULL) {

```

```

01368         delete xcl;
01369     }
01370 }
01371
01372
01373 // go to next step
01374 // 1. convert data format which treat the edges as objects.
01375 // 2. definition of loop momenta to the edges.
01376 // 3. analysis of loop structure in a graph.
01377 // 4. assignment of particles to edges and vertices to nodes
01378 // 5. recalculate symmetry factor
01379
01380 // Permutation
01381 Local int nextPerm(int nelem, int nclass, int *cl, int *r, int *q, int *p, int count)
01382 {
01383     int j, k, n, e, t;
01384     Bool b;
01385
01386     for (j = 0; j < nelem; j++) {
01387         p[j] = j;
01388     }
01389     if (count < 1) {
01390         for (j = 0; j < nelem; j++) {
01391             q[j] = 0;
01392             r[j] = 0;
01393         }
01394         j = 0;
01395         for (k = 0; k < nclass; k++) {
01396             n = cl[k];
01397             for (e = 0; e < n; e++) {
01398                 r[j] = n - e - 1;
01399                 j++;
01400             }
01401         }
01402         if (j != nelem) {
01403             erEnd("*** inconsistent # elements");
01404         }
01405         return 1;
01406     }
01407     b = T_False;
01408     for (j = nelem-1; j >= 0; j--) {
01409         if (q[j] < r[j]) {
01410             for (k = j+1; k < nelem; k++) {
01411                 q[k] = 0;
01412             }
01413             q[j]++;
01414             b = T_True;
01415             break;
01416         }
01417     }
01418     if (!b) {
01419         return (-count);
01420     }
01421
01422     for (j = 0; j < nelem; j++) {
01423         k = j + q[j];
01424         t = p[j];
01425         p[j] = p[k];
01426         p[k] = t;
01427     }
01428     return count + 1;
01429 }
01430
01431 Local BigInt factorial(int n)
01432 /* return (n < 1) ? 1 : n!
01433 */
01434 {
01435     int r, j;
01436
01437     r = 1;
01438     for (j = 2; j <= n; j++) {
01439         r *= j;
01440     }
01441     return r;
01442 }
01443
01444 Local BigInt ipow(int n, int p)
01445 {
01446     int r, j;
01447     DUMMYUSE(p);
01448
01449     r = 1;
01450     for (j = 2; j <= n; j++) {
01451         r *= n;
01452     }
01453     return r;
01454 }

```

```

01455
01456
01457 Local void erEnd(const char *msg)
01458 {
01459     printf("*** Error : %s\n", msg);
01460     exit(1);
01461 }
01462
01463 /* memory allocation */
01464 Local int  *newArray(int size, int val)
01465 {
01466     int *a, j;
01467
01468     a = new int[size];
01469     for (j = 0; j < size; j++) {
01470         a[j] = val;
01471     }
01472     return a;
01473 }
01474
01475 Local void deletArray(int *a)
01476 {
01477     delete[] a;
01478 }
01479
01480 Local void copyArray(int size, int *a0, int *a1)
01481 {
01482     int j;
01483     for (j = 0; j < size; j++) {
01484         a1[j] = a0[j];
01485     }
01486 }
01487
01488 Local int  **newMat(int n0, int n1, int val)
01489 {
01490     int **m, j;
01491
01492     m = new int*[n0];
01493     for (j = 0; j < n0; j++) {
01494         m[j] = newArray(n1, val);
01495     }
01496     return m;
01497 }
01498
01499 Local void  deleteMat(int **m, int n0, int n1)
01500 {
01501     int j;
01502     DUMMYUSE(n1);
01503
01504     for (j = 0; j < n0; j++) {
01505         deletArray(m[j]);
01506     }
01507     delete[] m;
01508 }
01509
01510 //=====
01511 // Functions for testing the program.
01512 //
01513 //-----
01514 // Testing function nextPerm
01515 //
01516 Local void printArray(int n, int *p)
01517 {
01518     int j;
01519
01520     printf("[");
01521     for (j = 0; j < n; j++) {
01522         printf(" %2d", p[j]);
01523     }
01524     printf("]");
01525 }
01526
01527 Local void printMat(int n0, int n1, int **m)
01528 {
01529     int j;
01530
01531     for (j = 0; j < n0; j++) {
01532         printArray(n1, m[j]);
01533         printf("\n");
01534     }
01535 }
01536
01537 Global void testPerm()
01538 {
01539     int nelem, nperm, nclist, n, count;
01540     int *p, *q, *r;
01541     int clist[] = {1, 2, 2, 3};

```

```

01542
01543     nclist = sizeof(clist)/sizeof(int);
01544     nelem = 0;
01545     nperm = 1;
01546     for (n = 0; n < nclist; n++) {
01547         nelem += clist[n];
01548         nperm *= factorial(clist[n]);
01549     }
01550
01551     printf("+++ clist = (%d) ", nclist);
01552     printArray(nclist, clist);
01553     printf("\n");
01554     printf("+++ nelem = %d, nperm = %d\n", nelem, nperm);
01555
01556     p = new int[nelem];
01557     q = new int[nelem];
01558     r = new int[nelem];
01559     count = 0;
01560     while (T_True) {
01561         count = nextPerm(nelem, nclist, clist, r, q, p, count);
01562         if (count < 0) {
01563             count = -count;
01564             break;
01565         }
01566         printf("%4d:", count);
01567         printArray(nelem, p);
01568         printf("\n");
01569
01570         if (count > nperm) {
01571             break;
01572         }
01573     }
01574     if (count != nperm) {
01575         printf("*** %d != %d\n", count, nperm);
01576     }
01577     delete[] p;
01578     delete[] q;
01579     delete[] r;
01580 }
01581
01582 // } } ]
01583

```

4.56 gentopo.h

```

00001 #pragma once
00002
00003 // Temporal definition of big integer
00004 #define BigInt long
00005 #define ToBigInt(x) ((BigInt) (x))
00006 //
00007 // Temporal definition of ratio of two big integers
00008 #define Fraction double
00009 #define ToFraction(x, y) (((double) (x))/((double) (y)))
00010
00011 //=====
00012 // Output to FROM
00013
00014 //=====
00015 // class of nodes for T_EGraph
00016
00017 class T_ENode {
00018 public:
00019     int deg;
00020     int ext;
00021     int *edges;
00022 };
00023
00024 class T_EEdge {
00025 public:
00026     int nodes[2];
00027     int ext;
00028     int momn;
00029     char momc[2]; // no more used
00030     // momentum is printed like: ("%s%d", (enode.ext)?"Q":"p", enode.momn)
00031     char padding[6];
00032 };
00033
00034 class T_EGraph {
00035 public:
00036     long gId;
00037     int pId;
00038     int nNodes;

```

```

00039     int nEdges;
00040     int maxdeg;
00041     int nExtern;
00042
00043     int opi;
00044     BigInt nsym, esym;
00045
00046     T_ENode *nodes;
00047     T_EEdge *edges;
00048
00049     T_EGraph(int nnodes, int nedges, int mxdeg);
00050     ~T_EGraph();
00051     void print(void);
00052     void init(int pid, long gid, int **adjmat, int sopi, BigInt nsym, BigInt esym);
00053
00054     void setExtern(int nd, int val);
00055     void endSetExtern(void);
00056 };
00057
00058 //=====
00059 // class of nodes for T_MGraph
00060
00061 class T_MNode {
00062 public:
00063     int id;           // node id
00064     int deg;          // degree(node) = the number of legs
00065     int cls;          // initial class number in which the node belongs
00066     int ext;
00067
00068     int freelg;       // the number of free legs
00069     int visited;
00070
00071     T_MNode(int id, int deg, int ext, int cls);
00072 };
00073
00074 //=====
00075 // class of node-classes for T_MGraph
00076 class T_MNodeClass;
00077
00078 //=====
00079 // class of scalar graph expressed by matrix form
00080 //
00081 // Input  : the classified set of nodes.
00082 // Output  : control passed to 'T_EGraph(self)'
00083
00084 class T_MGraph {
00085 public:
00086
00087     // initial conditions
00088     int pId;           // process/subprocess ID
00089     int nNodes;        // the number of nodes
00090     int nEdges;        // the number of edges
00091     int nLoops;        // the number of loops
00092     T_MNode **nodes;   // table of T_MNode object
00093     int *clist;        // list of initial classes
00094     int nClasses;      // the number of initial classes
00095     // int ndcl;        // node --> initial class number
00096     int mindeg;        // minimum value of degree of nodes
00097     int maxdeg;        // maximum value of degree of nodes
00098
00099     int selOPI;        // flag to select 1PI graphs
00100
00101     // generated set of graphs
00102     long ndiag;         // the total number of generated graphs
00103     long n1PI;         // the total number of 1PI graphs
00104     int nBridges;      // the number of bridges
00105
00106     // the current graph
00107     int clPI;          // the number of 1PI components
00108     int **adjMat;      // adjacency matrix
00109     BigInt nsym;       // symmetry factor from nodes
00110     BigInt esym;       // symmetry factor from edges
00111     Fraction wsum;     // weighted sum of graphs
00112     Fraction wsopi;    // weighted sum of 1PI graphs
00113     T_MNodeClass *curcl; // the current 'T_MNodeClass' object
00114     T_EGraph *egraph;
00115
00116     // measures of efficiency
00117     long ngen;         // generated graph before check
00118     long ngconn;       // generated connected graph before check
00119
00120     long nCallRefine;
00121     long discardOrd;
00122     long discardRefine;
00123     long discardDisc;
00124     long discardIso;

```



```

00126
00127 // functions */
00128 T_MGraph(int pid, int ncl, int *cldeg, int *clnum, int *clex, int sopi);
00129 ~T_MGraph();
00130 long generate(void);
00131
00132 private:
00133 // work space for isomorphism
00134 int **modmat; // permuted adjacency matrix
00135 int *permp;
00136 int *permq;
00137 int *permr;
00138
00139 // work space for biconnected component
00140 int *bidef;
00141 int *bilow;
00142 int bicount;
00143 int padding;
00144
00145 void printAdjMat(T_MNodeClass *cl);
00146 int isConnected(void);
00147 int visit(int nd);
00148 int isIsomorphic(T_MNodeClass *cl);
00149 void permMat(int size, int *perm, int **mat0, int **mat1);
00150 int compMat(int size, int **mat0, int **mat1);
00151 T_MNodeClass *refineClass(T_MNodeClass *cl, int cn);
00152 void bisearchM(int nd, int pd, int ne);
00153 int countLPI(void);
00154
00155 int findNextCl(T_MNodeClass *cl, int *dscl);
00156 int findNextTCl(T_MNodeClass *cl, int *dcl);
00157 void connectClass(T_MNodeClass *cl, int *dscl);
00158 void connectNode(int sc, int ss, T_MNodeClass *cl, int *dscl);
00159 void connectLeg(int sc, int sn, int tc, int ts, T_MNodeClass *cl, int *dscl, int* dtcl);
00160 void newGraph(T_MNodeClass *cl);
00161
00162 };

```

4.57 grcc.cc

```

00001 // { ( [
00002 //*****
00003 // grcc.cc
00004
00005 #ifndef NOFORM
00006 extern "C" {
00007 #include "form3.h"
00008 }
00009 #endif
00010
00011 #include <iostream>
00012 #include <iomanip>
00013 #include <sstream>
00014 #include <cstdio>
00015 #include <stdlib.h>
00016 #include <string.h>
00017 #include <time.h>
00018
00019 // #define DEBUG9
00020
00021 // optimization : best combination depends on process by process
00022 #define SIMPSEL
00023
00024 // optimization : lower and upper bound of deg of target node of connection.
00025 #define MINMAXLEG
00026
00027 // optimization : optimization with respect to 'extonly' particles
00028 #define OPTEXTONLY
00029
00030 // check UNIQUE interaction by name or by code
00031 #define UNIQINTR
00032
00033 // possible extensions of the program
00034 // #define ORBITS
00035
00036 // reverse direction of an edge of agraph when anti-particle flows on it.
00037 // In this case the direction of edges will be different
00038 // from ones of original mgraph.
00039 // #define EDGEORDER
00040
00041 //-----
00042 #include "grcc.h"
00043

```

```

00044 #ifdef GRCC_NAMESPACE
00045 using namespace Grcc;
00046 #endif
00047
00048 //-----
00049 // Macro functions
00050 #define CLWIGHTD(x)    (5*(x))
00051 #define CLWIGHTO(x)    (3*(x)-2)
00052
00053 //-----
00054 // Static variables
00055
00056 static OptDef optDef[] = {
00057 {"Step", "Generate particle assigned graphs", GRCC_AGraph, 0},
00058 {"Outgrf", "Output to file (out.grf)", False, 0},
00059 {"Outgrp", "Output to file (out.grp)", False, 0},
00060 {"lPI", "Only lPI graphs", True, 0},
00061 {"NoSelfLoop", "Exclude graphs with loops consist of 1 edge", True, 0},
00062 {"NoTadpole", "Exclude graphs with tadpoles (2 edge connected)", True, 0},
00063 {"No1PtBlock", "Exclude graphs with tadpole blocks (2 node connected)", False, 0},
00064 {"No2PtLlPI", "Exclude graphs with 2-point subgraphs", False, 0},
00065 {"NoExtSelf", "Exclude graphs with 2-pt subgraphs at ext. particles", False, 0},
00066 {"NoAdj2PtV", "Exclude graphs with an edge connecting 2-pt vertices", False, 0},
00067 {"Block", "Exclude graphs with more than one block", False, 0},
00068 {"SymmInitial", "Symmetrize initial particles", False, 0},
00069 {"SymmFinal", "Symmetrize final particles", False, 0},
00070 };
00071
00072 static int nOptDef = sizeof(optDef)/sizeof(OptDef);
00073
00074 static int prlevel = 2;
00075 static ErExit *erExit = NULL;
00076 static void *erExitArg = NULL;
00077
00078 //*****
00079 // Utility functions
00080 //=====
00081
00082 static void erEnd(const char *msg);
00083 static Bool nextPart(int nelem, int nclist, int *clist, int *nl, int *r);
00084 static void prlist(int n, const int *a, const char *msg);
00085 static int *newArray(int size, int val);
00086 static int *deleteArray(int *a);
00087 static int **newMat(int n0, int n1, int val);
00088 static int **deleteMat(int **m, int n0);
00089 static void prIntArray(int n, int *p, const char *msg);
00090 static void prIntArrayErr(int n, int *p, const char *msg);
00091 static int *intdup(int n, int *v);
00092 static int *delintdup(int *v);
00093 static void bsort(int n, int *ord, int *a);
00094 static void sorti(int n, int *a);
00095 static int sortb(int n, int *a);
00096 static int toSList(int n, int *a);
00097 static int intSetAdd(int n, int *a, int v, const int size);
00098 static int intSListAdd(int n, int *a, int v, const int size);
00099 static int cmpArray(int na, int *a, int nb, int *b);
00100 static Bool leqArray(int n, int *a, int *b);
00101 static void prMomStr(int mom, const char *ms, int mn);
00102 static void listDiff(int na, int *a, int nb, int *b, int *np, int *p, int *nq, int *q, int *nr, int
*r);
00103 static Bool isSubSList(int na, int *a, int nb, int *b);
00104 static int subtrSet(int na, int *a, int nb, int *b, int *c, int size);
00105 static int nextPerm(int nelem, int nclass, int *cl, int *r, int *q, int *p, int count);
00106 static BigInt ipow(int n, int p);
00107 static BigInt factorial(int n);
00108
00109 // for debugging
00110 #ifdef CHECK
00111 static Bool isIn(int n, int *a, int v);
00112 #endif
00113
00114 //=====
00115 // class Options
00116 //-----
00117 Options::Options(void)
00118 {
00119     if (nOptDef != GRCC_OPT_Size) {
00120         fprintf(GRCC_Stderr, "*** Options: inconsistent default values\n");
00121         exit(1);
00122     }
00123     model = NULL;
00124     proc = NULL;
00125     sproc = NULL;
00126
00127     outmg = NULL;
00128     endmg = NULL;
00129     outag = NULL;

```

```

00130
00131     argmg = NULL;
00132     argemg = NULL;
00133     argag = NULL;
00134
00135     setDefaultValue();
00136
00137     // for output
00138     out = new Output(this);
00139
00140     // subrpocess
00141     sproc = NULL;
00142
00143     // measuring time
00144     time0 = 0.0;
00145     time1 = 0.0;
00146 }
00147
00148 //-----
00149 Options::~Options(void)
00150 {
00151     outmg = NULL;
00152     outag = NULL;
00153
00154     argmg = NULL;
00155     argag = NULL;
00156
00157     delete out;
00158 }
00159
00160 //-----
00161 void Options::setDefaultValue(void)
00162 {
00163     int j;
00164
00165     // default values
00166     for (j = 0; j < GRCC_OPT_Size; j++) {
00167         values[j] = optDef[j].defaultv;
00168     }
00169
00170     values[GRCC_OPT_Step] = GRCC_AGraph;
00171
00172     // print level
00173     prlevel = 1;
00174 }
00175
00176 //-----
00177 void Options::setOutMG(OutEGB *omg, void *pt)
00178 {
00179     outmg = omg;
00180     argmg = pt;
00181 }
00182
00183 //-----
00184 void Options::setEndMG(OutEGB *emg, void *pt)
00185 {
00186     endmg = emg;
00187     argemg = pt;
00188 }
00189
00190 //-----
00191 void Options::setOutAG(OutEG *oag, void *pt)
00192 {
00193     outag = oag;
00194     argag = pt;
00195 }
00196
00197 //-----
00198 void Options::setErExit(ErExit *ere, void *erearg)
00199 {
00200     erExit = ere;
00201     erExitArg = erearg;
00202 }
00203
00204 //-----
00205 void Options::printLevel(int l)
00206 {
00207     prlevel = l;
00208 }
00209
00210 //-----
00211 void Options::setValue(int ind, int val)
00212 {
00213     if (0 <= ind && ind < GRCC_OPT_Size) {
00214         values[ind] = val;
00215     } else {
00216

```

```

00217         if (prlevel > 0) {
00218             fprintf(GRCC_Stderr, "*** Options::setValue : invalid index=%d ",
00219                 ind);
00220             fprintf(GRCC_Stderr, "(val=%d)\n", val);
00221         }
00222         erEnd("Options::setValue : invalid index");
00223     }
00224 }
00225
00226 //-----
00227 int Options::getValue(int ind)
00228 {
00229     if (0 <= ind && ind < GRCC_OPT_Size) {
00230         return values[ind];
00231     } else {
00232         if (prlevel > 0) {
00233             fprintf(GRCC_Stderr, "*** Options::getValue : invalid index=%d\n", ind);
00234         }
00235         erEnd("Options::getValue : invalid index");
00236     }
00237     return -1;
00238 }
00239
00240 //-----
00241 void Options::print(void)
00242 {
00243     int j;
00244
00245     printf("Options\n");
00246     printf("+++ GRCC_OPT_Size=%d, print level=%d: ",
00247         GRCC_OPT_Size, prlevel);
00248     printf("symbol = value (default)\n");
00249     for (j=0; j < GRCC_OPT_Size; j++) {
00250         printf("    %4d GRCC_OPT_%-15s = %2d (%2d)\n",
00251             j, optDef[j].name, values[j], optDef[j].defaultv);
00252     }
00253     printf("    outgrf=%s, outgrp=%s\n", out->outgrf, out->outgrp);
00254 }
00255
00256 //-----
00257 const OptDef *Options::getDef(void)
00258 {
00259     return optDef;
00260 }
00261
00262 //-----
00263 void Options::setOutputF(Bool outf, const char *fname)
00264 {
00265     values[GRCC_OPT_Outgrf] = outf;
00266     out->setOutgrf(fname);
00267 }
00268
00269 //-----
00270 void Options::setOutputP(Bool outp, const char *fname)
00271 {
00272     values[GRCC_OPT_Outgrp] = outp;
00273     out->setOutgrp(fname);
00274 }
00275
00276 //-----
00277 void Options::printModel(void)
00278 {
00279     if (prlevel > 0) {
00280         if (model != NULL) {
00281             model->prModel();
00282         } else {
00283             printf("*** model is not defined\n");
00284         }
00285     }
00286 }
00287
00288 //-----
00289 void Options::outModel(void)
00290 {
00291     if (out != NULL && model != NULL) {
00292         out->outModelF();
00293         out->outModelP();
00294     }
00295 }
00296
00297 //-----
00298 void Options::begin(Model *mdl)
00299 {
00300     model = mdl;
00301     if (out != NULL) {
00302         if (values[GRCC_OPT_Outgrf]) {

```

```

00304         out->outBeginF(model, values[GRCC_OPT_Outgrf]);
00305         if (model != NULL) {
00306             out->outModelF();
00307         }
00308     }
00309     if (values[GRCC_OPT_Outgrf]) {
00310         out->outBeginP(model, values[GRCC_OPT_Outgrf]);
00311         if (model != NULL) {
00312             out->outModelP();
00313         }
00314     }
00315 }
00316 }
00317
00318 //-----
00319 void Options::end(void)
00320 {
00321     if (prlevel > 0) {
00322         printf("Optimization: ");
00323 #ifdef SIMPSEL
00324         printf("SIMPSEL=1 ");
00325 #else
00326         printf("SIMPSEL=0 ");
00327 #endif
00328 #ifdef MINMAXLEG
00329         printf("MINMAXLEG=1 ");
00330 #else
00331         printf("MINMAXLEG=0 ");
00332 #endif
00333 #ifdef OPTEXTONLY
00334         printf("OPTEXTONLY=1 ");
00335 #else
00336         printf("OPTEXTONLY=0 ");
00337 #endif
00338         printf("\n");
00339     }
00340
00341     if (out != NULL && values[GRCC_OPT_Outgrf]) {
00342         out->outEndF();
00343     }
00344     if (out != NULL && values[GRCC_OPT_Outgrp]) {
00345         out->outEndP();
00346     }
00347     model = NULL;
00348 }
00349 //
00350 //-----
00351 void Options::beginProc(Process *prc)
00352 {
00353     prc = prc;
00354     if (out != NULL && values[GRCC_OPT_Outgrf]) {
00355         out->outProcBeginF(prc);
00356     }
00357     if (out != NULL && values[GRCC_OPT_Outgrp]) {
00358         out->outProcBeginP(prc);
00359     }
00360     if (prc != NULL) {
00361         prc->mgrcount = 0;
00362         prc->agrcount = 0;
00363     }
00364 }
00365
00366 //-----
00367 void Options::endProc(void)
00368 {
00369     int k;
00370
00371     if (prc == NULL) {
00372         if (out != NULL && values[GRCC_OPT_Outgrf]) {
00373             out->outProcEndF();
00374         }
00375         if (out != NULL && values[GRCC_OPT_Outgrp]) {
00376             out->outProcEndP();
00377         }
00378         return;
00379     }
00380     if (prlevel > 0) {
00381         printf("\n");
00382         printf("+++ Proc %d: ext=%d, loop=%d, ",
00383             prc->id, prc->nExtern, prc->loop);
00384         if (model != NULL) {
00385             printf("order=");
00386             prIntArray(model->ncouple, prc->clist, ": ");
00387             model->prParticleArray(prc->ninitl, prc->initlPart, "-->");
00388             model->prParticleArray(prc->nfinal, prc->finalPart, "");
00389         }
00390         printf(" (%8.2f sec)\n", prc->sec);

```

```

00391
00392
00393     printf("    Proc    %d: Total M-Graphs=%ld, M-Graphs=",
00394            proc->id, proc->nMGraphs);
00395     proc->wMGraphs.print(" (Conn)\n");
00396
00397     printf("    Proc    %d: Total M-Graphs=%ld, M-Graphs=",
00398            proc->id, proc->nMOPI);
00399     proc->wMOPI.print(" (lPI)\n");
00400
00401     printf("    Proc    %d: Total A-Graphs=%ld, A-Graphs=",
00402            proc->id, proc->nAGraphs);
00403     proc->wAGraphs.print(" (Conn)\n");
00404
00405     printf("    Proc    %d: Total A-Graphs=%ld, A-Graphs=",
00406            proc->id, proc->nAOPI);
00407     proc->wAOPI.print(" (lPI)\n");
00408
00409     printf("# { %d,{", proc->ninitl);
00410     for (k = 0; k < proc->ninitl; k++) {
00411         if (k != 0) {
00412             printf(", ");
00413         }
00414         if (proc->model != NULL) {
00415             printf("\'%s\'", proc->model->particleName(proc->initlPart[k]));
00416         } else {
00417             printf("%d", proc->initlPart[k]);
00418         }
00419     }
00420     printf("}, %d,{", proc->nfinal);
00421     for (k = 0; k < proc->nfinal; k++) {
00422         if (k != 0) {
00423             printf(", ");
00424         }
00425         if (proc->model != NULL) {
00426             printf("\'%s\'", proc->model->particleName(proc->finalPart[k]));
00427         } else {
00428             printf("%d", proc->initlPart[k]);
00429         }
00430     }
00431     printf("}, ");
00432     if (model != NULL) {
00433         printf("{");
00434         for (k = 0; k < model->ncouple; k++) {
00435             if (k != 0) {
00436                 printf(", ");
00437             }
00438             printf("%d", proc->clist[k]);
00439         }
00440         printf("},");
00441     }
00442     printf("},%6ldL,%6ldL,%3ldL, -1.0, %4.2f),\n",
00443            proc->nAOPI, proc->wAOPI.num, proc->wAOPI.den, proc->sec);
00444 }
00445
00446 if (out != NULL && values[GRCC_OPT_Outgrf]) {
00447     out->outProcEndF();
00448 }
00449 if (out != NULL && values[GRCC_OPT_Outgrp]) {
00450     out->outProcEndP();
00451 }
00452 proc = NULL;
00453 }
00454
00455 //-----
00456 void Options::beginSubProc(SProcess *sprc)
00457 {
00458     sprc = sprc;
00459
00460     // 'beginProc' is called before
00461     if (out != NULL && values[GRCC_OPT_Outgrf]) {
00462         out->outSProcBeginF(sprc);
00463     }
00464     if (out != NULL && values[GRCC_OPT_Outgrp]) {
00465         out->outSProcBeginP(sprc);
00466     }
00467     if (proc != NULL) {
00468         return;
00469     }
00470     if (sprc != NULL) {
00471         sprc->mgrcount = 0;
00472         sprc->agrcount = 0;
00473     }
00474 }
00475
00476 //-----
00477 void Options::endSubProc(void)

```

```

00478 {
00479     MGraph *mgraph;
00480
00481     if (sproc == NULL) {
00482         if (out != NULL && values[GRCC_OPT_Outgrf]) {
00483             out->outProcEndF();
00484         }
00485         if (out != NULL && values[GRCC_OPT_Outgrp]) {
00486             out->outProcEndP();
00487         }
00488         return;
00489     }
00490     mgraph = sproc->mgraph;
00491     if (sproc != NULL) {
00492         sproc->resultMGraph(mgraph->cDiag, mgraph->wscon,
00493                             mgraph->c1PI, mgraph->wsopi);
00494     }
00495
00496     if (prlevel > 1) {
00497         printf("\n");
00498         printf("+++ Subproc %d: ext=%d, loop=%d, nodes=%d, edges=%d\n",
00499                sproc->id, sproc->nExtern, sproc->loop,
00500                sproc->nNodes, sproc->nEdges);
00501
00502         printf("    Subproc %d: Total M-Graphs=%ld, M-Wsum=",
00503                sproc->id, sproc->nMGraphs);
00504         sproc->wMGraphs.print(" (Conn)\n");
00505
00506         printf("    Subproc %d: Total M-Graphs=%ld, M-Wsum=",
00507                sproc->id, sproc->nMOPI);
00508         sproc->wMOPI.print(" (1PI)\n");
00509
00510         printf("    Subproc %d: Total A-Graphs=%ld, A-Wsum=",
00511                sproc->id, sproc->nAGraphs);
00512         sproc->wAGraphs.print(" (Conn)\n");
00513
00514         printf("    Subproc %d: Total A-Graphs=%ld, A-Wsum=",
00515                sproc->id, sproc->nAOPI);
00516         sproc->wAOPI.print(" (1PI)\n");
00517     }
00518     if (prlevel > 0) {
00519         printf("\n");
00520         printf("* Total %ld MGraphs; %ld 1PI", mgraph->cDiag, mgraph->c1PI);
00521         printf(" wscon = ");
00522         mgraph->wscon.print(" ( ");
00523         mgraph->wsopi.print(" 1PI)\n");
00524
00525     #ifdef MONITOR
00526         printf("* refine: %ld\n", mgraph->nCallRefine);
00527         printf("* discarded for refinement: %ld\n", mgraph->discardRefine);
00528         printf("* discarded for disconnected: %ld\n", mgraph->discardDisc);
00529         printf("* discarded for duplication: %ld\n", mgraph->discardIso);
00530         printf("* discarded by outMG: %ld\n", mgraph->discardMG);
00531     #endif
00532     }
00533
00534     if (proc != NULL) {
00535         return;
00536     }
00537     if (out != NULL && values[GRCC_OPT_Outgrf]) {
00538         out->outProcEndF();
00539     }
00540     if (out != NULL && values[GRCC_OPT_Outgrp]) {
00541         out->outProcEndP();
00542     }
00543 }
00544
00545 //-----
00546 void Options::newMGraph(MGraph *mgr)
00547 {
00548     MGraph *mgraph = mgr;
00549
00550     #ifdef DEBUG
00551     if (proc != NULL) {
00552         printf("+++ New EGraph (MG): %ld\n", proc->mgrcount);
00553     } else if (sproc != NULL) {
00554         printf("+++ New EGraph (MG): %ld\n", sproc->mgrcount);
00555     }
00556     #endif
00557     if (proc != NULL) {
00558         proc->mgrcount++;
00559         mgr->egraph->mId = proc->mgrcount;
00560     } else if (sproc != NULL) {
00561         sproc->mgrcount++;
00562         mgr->egraph->mId = sproc->mgrcount;
00563     }
00564 }

```

```

00565     if (out != NULL
00566         && values[GRCC_OPT_Step] == GRCC_MGraph) {
00567         if (values[GRCC_OPT_Outgrf]) {
00568             out->outEGraphF(mgraph->egraph);
00569         }
00570         if (values[GRCC_OPT_Outgrp]) {
00571             out->outEGraphP(mgraph->egraph);
00572         }
00573     }
00574     if (sproc != NULL) {
00575         sproc->endMGraph(mgr);
00576     }
00577     if (endmg != NULL) {
00578         (*endmg)(mgr->egraph, argemg);
00579     }
00580 }
00581
00582 //-----
00583 void Options::newAGraph(EGraph *egraph)
00584 {
00585     Fraction sf, zr;
00586
00587 #ifdef DEBUG
00588     if (proc != NULL) {
00589         printf("+++ New EGraph (AG): %ld (%ld)\n",
00590             proc->agrcount, proc->mgrcount);
00591     } else if (sproc != NULL) {
00592         printf("+++ New EGraph (AG): %ld (%ld)\n",
00593             sproc->agrcount, sproc->mgrcount);
00594     }
00595 #endif
00596     if (proc != NULL) {
00597         proc->agrcount++;
00598         egraph->sId = proc->agrcount;
00599     } else if (sproc != NULL) {
00600         sproc->agrcount++;
00601         egraph->sId = sproc->agrcount;
00602     }
00603
00604     if (sproc != NULL) {
00605         zr = Fraction(0, 1);
00606         if ((egraph->nsym)*(egraph->esym) != 0) {
00607             sf = Fraction(egraph->extperm, egraph->nsym*egraph->esym);
00608         } else {
00609             sf = zr;
00610         }
00611         if (egraph->mgraph->opi) {
00612             sproc->resultAGraph(1, sf, 1, sf);
00613         } else {
00614             sproc->resultAGraph(1, sf, 0, zr);
00615         }
00616     }
00617
00618     if (out != NULL && values[GRCC_OPT_Outgrf]) {
00619         out->outEGraphF(egraph);
00620     }
00621     if (out != NULL && values[GRCC_OPT_Outgrp]) {
00622         out->outEGraphP(egraph);
00623     }
00624     if (outag != NULL) {
00625 #ifdef DEBUG1
00626         printf("call outag\n");
00627         egraph->model->prModel();
00628         egraph->print();
00629 #endif
00630         (*outag)(egraph, argag);
00631     }
00632
00633     sproc->endAGraph(egraph);
00634 }
00635
00636 //=====
00637 Output::Output(Options *optn)
00638 {
00639     opt      = optn;
00640     model    = NULL;
00641     proc     = NULL;
00642     sproc    = NULL;
00643     outgrfp  = NULL;
00644     outgrpp  = NULL;
00645     procId   = -1;
00646     outgrf   = NULL;
00647     outgrp   = NULL;
00648     outproc  = False;
00649 }
00650 }
00651

```



```

00652 //-----
00653 Output::~Output(void)
00654 {
00655     if (outgrfp != NULL) {
00656         if (prlevel > 0) {
00657             fprintf(GRCC_Stderr, "*** file has not been closed : \"%s\\n\"",
00658                 outgrf);
00659         }
00660         fclose(outgrfp);
00661         outgrfp = NULL;
00662         erEnd("file has not been closed");
00663     }
00664     if (outgrf != NULL) {
00665         free(outgrf);
00666         outgrf = NULL;
00667     }
00668     if (outgrpp != NULL) {
00669         if (prlevel > 0) {
00670             fprintf(GRCC_Stderr, "*** file has not been closed : \"%s\\n\"",
00671                 outgrp);
00672         }
00673         fclose(outgrpp);
00674         outgrpp = NULL;
00675         erEnd("file has not been closed");
00676     }
00677     if (outgrp != NULL) {
00678         free(outgrp);
00679         outgrp = NULL;
00680     }
00681 }
00682
00683 //-----
00684 void Output::setOutgrf(const char *fname)
00685 {
00686     if (outgrf != NULL && strcmp(outgrf, fname) == 0) {
00687         return;
00688     }
00689     if (outgrf != NULL) {
00690         free(outgrf);
00691     }
00692     if (fname == NULL || strlen(fname) < 1) {
00693         outgrf = NULL;
00694     } else {
00695         outgrf = strdup(fname);
00696     }
00697 }
00698
00699 //-----
00700 void Output::setOutgrp(const char *fname)
00701 {
00702     if (outgrp != NULL && strcmp(outgrp, fname) == 0) {
00703         return;
00704     }
00705     if (outgrp != NULL) {
00706         free(outgrp);
00707     }
00708     if (fname == NULL || strlen(fname) < 1) {
00709         outgrp = NULL;
00710     } else {
00711         outgrp = strdup(fname);
00712     }
00713 }
00714
00715 //-----
00716 Bool Output::outBeginF(Model *mdl, Bool pr)
00717 {
00718     model = mdl;
00719     if (outgrf == NULL || strlen(outgrf) < 1) {
00720         outgrfp = NULL;
00721         return True;
00722     } else if (outgrfp != NULL) {
00723         if (prlevel > 0) {
00724             fprintf(GRCC_Stderr, "*** outBegin: \"%s\\n\" is already opened\\n",
00725                 outgrf);
00726         }
00727         erEnd("outBegin: file is already opened\\n");
00728     }
00729     if (!pr) {
00730         return True;
00731     }
00732     if ((outgrfp = fopen(outgrf, "w")) == NULL) {
00733         if (prlevel > 0) {
00734             fprintf(GRCC_Stderr, "*** outBegin: cannot open %s\\n", outgrf);

```

```

00739     }
00740     return False;
00741 }
00742 fprintf(outgrfp, "%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%\n");
00743 fprintf(outgrfp, "% Generated by 'grcc'\n");
00744 fprintf(outgrfp, "Version={2,2,0,0};\n");
00745 if (model != NULL) {
00746     fprintf(outgrfp, "Model=\"./%s.mdl\";\n", model->name);
00747 } else {
00748     fprintf(outgrfp, "Model=\"./Phi34.mdl\";\n");
00749 }
00750
00751
00752 return True;
00753 }
00754
00755 //-----
00756 Bool Output::outBeginP(Model *mdl, Bool pr)
00757 {
00758     model = mdl;
00759
00760     if (outgrp == NULL || strlen(outgrp) < 1) {
00761         outgrpp = NULL;
00762         return True;
00763     } else if (outgrpp != NULL) {
00764         if (prlevel > 0) {
00765             fprintf(GRCC_Stderr, "*** outBegin: \"%s\" is already opened\n",
00766                 outgrp);
00767         }
00768         erEnd("outBegin: file is already opened\n");
00769     }
00770
00771     if (!pr) {
00772         return True;
00773     }
00774     if ((outgrpp = fopen(outgrp, "w")) == NULL) {
00775         if (prlevel > 0) {
00776             fprintf(GRCC_Stderr, "*** outBegin: cannot open %s\n", outgrp);
00777         }
00778         return False;
00779     }
00780     fprintf(outgrpp, "#####\n");
00781     fprintf(outgrpp, "# Generated by 'grcc'\n");
00782
00783     return True;
00784 }
00785
00786 //-----
00787 void Output::outEndF(void)
00788 {
00789     if (outgrfp == NULL) {
00790         return;
00791     }
00792     fprintf(outgrfp, "%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%\n");
00793     fprintf(outgrfp, "End=1;\n");
00794     fprintf(outgrfp, "%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%\n");
00795     fclose(outgrfp);
00796     outgrfp = NULL;
00797
00798     free(outgrf);
00799     outgrf = NULL;
00800 }
00801
00802 //-----
00803 void Output::outEndP(void)
00804 {
00805     if (outgrpp == NULL) {
00806         return;
00807     }
00808     fprintf(outgrpp, "#####\n");
00809     fprintf(outgrpp, "# End\n");
00810     fprintf(outgrpp, "#####\n");
00811     fclose(outgrpp);
00812     outgrpp = NULL;
00813
00814     free(outgrp);
00815     outgrp = NULL;
00816 }
00817
00818 //-----
00819 void Output::outProcBeginF(Process *prc)
00820 {
00821     int k, ex;
00822
00823     if (prc == NULL) {
00824         return;
00825     }

```

```

00826
00827     proc    = prc;
00828     sproc   = NULL;
00829     procId  = proc->id;
00830
00831     if (outgrfp == NULL) {
00832         return;
00833     }
00834     if (outproc) {
00835         return;
00836     }
00837     outproc = True;
00838
00839     fprintf(outgrfp, "%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%\n");
00840     fprintf(outgrfp, "Process=%d;\n", proc->id);
00841     fprintf(outgrfp, "External=%d;\n", proc->ninitl+proc->nfinal);
00842     for (k = 0, ex = 0; k < proc->ninitl; k++, ex++) {
00843         fprintf(outgrfp, "%4d= initial %s;\n", ex,
00844             proc->model->particleName(proc->initlPart[k]));
00845     }
00846     for (k = 0; k < proc->nfinal; k++, ex++) {
00847         fprintf(outgrfp, "%4d= final %s;\n", ex,
00848             proc->model->particleName(proc->finalPart[k]));
00849     }
00850     fprintf(outgrfp, "End;\n");
00851     for (k = 0; k < proc->model->ncouple; k++) {
00852         fprintf(outgrfp, "%s=%d; ", proc->model->cnlist[k], proc->clist[k]);
00853     }
00854     fprintf(outgrfp, "Loop=%d;\n", proc->loop);
00855     fprintf(outgrfp, "OPI=%s;\n", (opt->values[GRCC_OPT_1PI] > 0? "Yes" : "No"));
00856     fprintf(outgrfp, "Assign=%s;\n", (opt->values[GRCC_OPT_Step]==GRCC_AGraph ? "Yes" : "No"));
00857     fprintf(outgrfp, "%% Options : dummy values\n");
00858     fprintf(outgrfp, "Expand=Yes; ExpMin=0; ExpMax=-1;\n");
00859     fprintf(outgrfp, "Block=No; Tadpole=Yes; Extself=Yes;\n");
00860     fprintf(outgrfp, "Selfe=Yes; Countert=No; AnyCT=No; Undefp=No;\n");
00861 }
00862
00863 //-----
00864 void Output::outProcBeginP(Process *prc)
00865 {
00866     if (prc == NULL) {
00867         return;
00868     }
00869
00870     proc    = prc;
00871     sproc   = NULL;
00872     procId  = proc->id;
00873
00874     if (outgrpp == NULL) {
00875         return;
00876     }
00877     if (outproc) {
00878         return;
00879     }
00880     outproc = True;
00881
00882     fprintf(outgrfp, "# Process=%d;\n", proc->id);
00883 }
00884
00885 //-----
00886 void Output::outProcBegin0(int next, int couple, int loop)
00887 {
00888     int k, ex;
00889
00890     procId = 0;
00891
00892     if (outgrfp == NULL) {
00893         return;
00894     }
00895     if (outproc) {
00896         return;
00897     }
00898     outproc = True;
00899
00900     fprintf(outgrfp, "%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%\n");
00901     fprintf(outgrfp, "Process=%d;\n", procId);
00902     fprintf(outgrfp, "External=%d;\n", next);
00903     for (k = 0, ex = 0; k < next; k++, ex++) {
00904         fprintf(outgrfp, "%4d= initial undef;\n", ex);
00905     }
00906     fprintf(outgrfp, "End;\n");
00907     fprintf(outgrfp, "GRCC_PHI=%d; ", couple);
00908     fprintf(outgrfp, "Loop=%d;\n", loop);
00909     fprintf(outgrfp, "OPI=%s;\n", (opt->values[GRCC_OPT_1PI] > 0? "Yes" : "No"));
00910     fprintf(outgrfp, "Assign=No;\n");
00911     fprintf(outgrfp, "%% Options : dummy values\n");
00912     fprintf(outgrfp, "Expand=Yes; ExpMin=0; ExpMax=-1;\n");

```

```

00913     fprintf(outgrfp, "Block=No; Tadpole=Yes; Extself=Yes;\n");
00914     fprintf(outgrfp, "Selfe=Yes; Countert=No; AnyCT=No; Undefp=No;\n");
00915 }
00916
00917 //-----
00918 void Output::outSProcBeginF(SProcess *sprc)
00919 {
00920     int j, k, ex;
00921     PNodeClass *pnc;
00922
00923     if (sprc == NULL) {
00924         return;
00925     }
00926
00927     sprc = sprc;
00928     procId = sprc->id;
00929     pnc = sprc->pnclass;
00930
00931     if (outgrfp == NULL) {
00932         return;
00933     }
00934     if (outproc) {
00935         return;
00936     }
00937     outproc = True;
00938     fprintf(outgrfp, "%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%\n");
00939     fprintf(outgrfp, "Process=%d;\n", sprc->id);
00940     fprintf(outgrfp, "External=%d;\n", sprc->nExtern);
00941
00942     ex = 0;
00943     for (j = 0; j < pnc->nclass; j++) {
00944         if (pnc->type[j] == GRCC_AT_Initial) {
00945             for (k = 0; k < pnc->count[j]; k++) {
00946                 fprintf(outgrfp, "%4d= initial %s;\n", ex,
00947                     model->particleName(pnc->particle[j]));
00948                 ex++;
00949             }
00950         } else if (pnc->type[j] == GRCC_AT_Final) {
00951             for (k = 0; k < pnc->count[j]; k++) {
00952                 fprintf(outgrfp, "%4d= final %s;\n", ex,
00953                     model->particleName(-pnc->particle[j]));
00954                 ex++;
00955             }
00956         }
00957     }
00958
00959     fprintf(outgrfp, "Eend;\n");
00960     for (k = 0; k < sprc->model->ncouple; k++) {
00961         fprintf(outgrfp, "%s=%d; ", sprc->model->cnlist[k], sprc->clist[k]);
00962     }
00963     fprintf(outgrfp, "Loop=%d;\n", sprc->loop);
00964     fprintf(outgrfp, "OPI=%s;\n", (opt->values[GRCC_OPT_1PI] > 0 ? "Yes" : "No"));
00965     fprintf(outgrfp, "Assign=%s;\n", (opt->values[GRCC_OPT_Step]==GRCC_AGraph ? "Yes" : "No"));
00966     fprintf(outgrfp, "%% Options : dummy values\n");
00967     fprintf(outgrfp, "Expand=Yes; ExpMin=0; ExpMax=-1;\n");
00968     fprintf(outgrfp, "Block=No; Tadpole=Yes; Extself=Yes;\n");
00969     fprintf(outgrfp, "Selfe=Yes; Countert=No; AnyCT=No; Undefp=No;\n");
00970 }
00971
00972 //-----
00973 void Output::outSProcBeginP(SProcess *sprc)
00974 {
00975     if (sprc == NULL) {
00976         return;
00977     }
00978
00979     sprc = sprc;
00980     procId = sprc->id;
00981
00982     if (outgrpp == NULL) {
00983         return;
00984     }
00985     if (outproc) {
00986         return;
00987     }
00988     outproc = True;
00989     fprintf(outgrpp, "# SProcess=%d;\n", sprc->id);
00990 }
00991
00992 //-----
00993 void Output::outProcEndF(void)
00994 {
00995     if (outgrfp == NULL) {
00996         return;
00997     }
00998     if (proc != NULL) {
00999         fprintf(outgrfp, "%%-----\n");

```

```

01000         fprintf(outgrfp, "Pend=%d;\n", proc->id);
01001     } else if (sproc != NULL) {
01002         fprintf(outgrfp, "%%-----\n");
01003         fprintf(outgrfp, "Pend=%d;\n", sproc->id);
01004     } else {
01005         fprintf(outgrfp, "%%-----\n");
01006         fprintf(outgrfp, "Pend=1;\n");
01007     }
01008     fflush(outgrfp);
01009 }
01010
01011 //-----
01012 void Output::outProcEndP(void)
01013 {
01014     if (outgrpp == NULL) {
01015         return;
01016     }
01017     if (proc != NULL) {
01018         fprintf(outgrpp, "# Pend=%d;\n", proc->id);
01019     }
01020     fflush(outgrpp);
01021 }
01022
01023 //-----
01024 void Output::outEGraphF(EGraph *egraph)
01025 {
01026     int nd, lg, ptcl, ed, intr, j, k, loop;
01027     Model *mdl = egraph->model;
01028     Bool popt;
01029     EFLine *fl;
01030
01031     #ifdef DEBUG9
01032     if (egraph->mgraph != NULL) {
01033         printf("outEGraph:sId=%ld\n", egraph->mId);
01034         // egraph->mgraph->print();
01035         egraph->mgraph->mconn->print();
01036     }
01037     #endif
01038     if (outgrfp == NULL) {
01039         return;
01040     }
01041     fprintf(outgrfp, "%%-----\n");
01042     if (egraph->assigned) {
01043         fprintf(outgrfp, "Graph=%ld;\n", egraph->sId);
01044         fprintf(outgrfp, "%% AGraph=%ld;\n", egraph->aId);
01045     } else {
01046         fprintf(outgrfp, "Graph=%ld;\n", egraph->mId);
01047     }
01048     fprintf(outgrfp, "Gtype=%ld;\n", egraph->mId);
01049     if (egraph->assigned) {
01050         fprintf(outgrfp, "Sfactor=%ld;\n",
01051             egraph->nsym * egraph->esym * egraph->fsign);
01052     } else {
01053         fprintf(outgrfp, "Sfactor=%ld;\n", egraph->nsym * egraph->esym);
01054     }
01055     fprintf(outgrfp, "%% ExtPerm=%ld; NSym=%ld; ESym=%ld; NSym1=%ld;"
01056         " Multp=%ld;\n",
01057         egraph->extperm, egraph->nsym, egraph->esym, egraph->nsym1,
01058         egraph->multp);
01059
01060     fprintf(outgrfp, "Vertex=%d;\n", egraph->nNodes - egraph->nExtern);
01061
01062     for (nd = 0; nd < egraph->nNodes; nd++) {
01063         fprintf(outgrfp, "%4d", nd);
01064         if (mdl != NULL && egraph->assigned && !egraph->isExternal(nd)) {
01065             intr = egraph->nodes[nd]->intrct;
01066             loop = mdl->interacts[intr]->loop;
01067             popt = False;
01068
01069             // not tree vertex
01070             if (loop > 0) {
01071                 fprintf(outgrfp, "[loop=%d", loop);
01072                 popt = True;
01073             }
01074
01075             // multiple coupling constants
01076             if (mdl->ncouple > 1) {
01077                 if (popt) {
01078                     fprintf(outgrfp, ";order={");
01079                 } else {
01080                     fprintf(outgrfp, "[order={");
01081                     popt = True;
01082                 }
01083                 for (j = 0; j < mdl->interacts[intr]->nclist; j++) {
01084                     if (j != 0) {
01085                         fprintf(outgrfp, ",");
01086                     }

```

```

01087         fprintf(outgrfp, "%d", mdl->interacts[intr]->clist[j]);
01088     }
01089     fprintf(outgrfp, " ");
01090 }
01091 if (popt) {
01092     fprintf(outgrfp, " ");
01093 }
01094 } else if (!egraph->isExternal(nd)) {
01095     loop = egraph->nodes[nd]->extloop;
01096     // not tree vertex
01097     if (loop > 0) {
01098         fprintf(outgrfp, "[loop=%d]", loop);
01099     }
01100 }
01101 fprintf(outgrfp, "={");
01102
01103 // list of legs
01104 for (lg = 0; lg < egraph->nodes[nd]->deg; lg++) {
01105     if (lg != 0) {
01106         fprintf(outgrfp, ", ");
01107     }
01108     ed = V2Iedge(egraph->nodes[nd]->edges[lg]);
01109     fprintf(outgrfp, "%4d", egraph->nodes[nd]->edges[lg]);
01110     ptcl = egraph->edges[ed]->ptcl;
01111     if (mdl != NULL && ptcl != 0 && egraph->assigned) {
01112         if (egraph->nodes[nd]->edges[lg] < 0) {
01113             ptcl = mdl->normalParticle(-ptcl);
01114         } else {
01115             ptcl = mdl->normalParticle(ptcl);
01116         }
01117         fprintf(outgrfp, "[%s]", mdl->particleName(ptcl));
01118     } else {
01119         fprintf(outgrfp, "[undef]");
01120     }
01121 }
01122 fprintf(outgrfp, "];\n");
01123 }
01124 // print Fermion line as comment line
01125 if (egraph->nflines >= 0) {
01126     fprintf(outgrfp, "%% FLines=%d; FSign=%d; sId=%ld;\n",
01127         egraph->nflines, egraph->fsign, egraph->sId);
01128 }
01129 for (j = 0; j < egraph->nflines; j++) {
01130     fl = egraph->flines[j];
01131     fprintf(outgrfp, "%% %4d", j);
01132     if (fl->ftype == FL_Open) {
01133         fprintf(outgrfp, "[Open]=[");
01134     } else if (fl->ftype == FL_Closed) {
01135         fprintf(outgrfp, "[Loop]=[");
01136     } else {
01137         fprintf(outgrfp, " ?%d", fl->ftype);
01138     }
01139     for (k = 0; k < fl->nlist; k++) {
01140         if (k != 0) {
01141             fprintf(outgrfp, ", ");
01142         }
01143         fprintf(outgrfp, "%d", Abs(fl->elist[k]));
01144     }
01145     fprintf(outgrfp, "];\n");
01146 }
01147 fprintf(outgrfp, "Vend;\n");
01148 fprintf(outgrfp, "Gend;\n");
01149 }
01150
01151 //-----
01152 void Output::outEGraphP(EGraph *eg)
01153 {
01154     int j, nd, lg, ed, nin, nfi;
01155     char nl = '\n';
01156     ENode *node;
01157     static char undef[] = "Undef";
01158     char *s;
01159
01160     // model
01161     fprintf(outgrpp, "egraph[%ld] = {%c", eg->sId-1, nl);
01162     if (model==NULL) {
01163         fprintf(outgrpp, " \\"Model\\": [\\"Undef\\", 1],%c", nl);
01164     } else {
01165         fprintf(outgrpp, " \\"Model\\": [\\"%s\\", %d],%c",
01166             model->name, model->ncouple, nl);
01167     }
01168
01169     // process
01170     nin = nfi = 0;
01171     for (j = 0; j < eg->nNodes; j++) {
01172         if (eg->isExternal(j)) {
01173             if (eg->nodes[j]->extloop == GRCC_AT_Final) {

```

```

01174         nfi++;
01175     } else {
01176         nin++;
01177     }
01178 }
01179 }
01180 fprintf(outgrpp, "  \"Process\": [%d, %d], %d, [",
01181         nin, nfi, eg->nLoops);
01182 if (proc != NULL && model != NULL) {
01183     for (j = 0; j < model->ncouple; j++) {
01184         if (j != 0) {
01185             fprintf(outgrpp, ", ");
01186         }
01187         fprintf(outgrpp, "%d", proc->clist[j]);
01188     }
01189 }
01190 fprintf(outgrpp, "],%c", nl);
01191
01192 // option of the process
01193 fprintf(outgrpp, "  \"Opt\": [");
01194 if (eg->opt != NULL) {
01195     for (j = 0; j < GRCC_OPT_Size; j++) {
01196         if (j != 0) {
01197             fprintf(outgrpp, ", ");
01198         }
01199         fprintf(outgrpp, "%d", eg->opt->values[j]);
01200     }
01201 }
01202 fprintf(outgrpp, "],%c", nl);
01203
01204 // graph
01205 fprintf(outgrpp, "  \"Id\": [%ld, %ld, %ld],%c",
01206         eg->mId, eg->aId, eg->sId, nl);
01207 fprintf(outgrpp, "  \"GStat\": [%d, %d, %d],%c",
01208         eg->assigned, eg->nNodes, eg->nEdges, nl);
01209 fprintf(outgrpp, "  \"Sym\": [%d, %ld, %ld, %ld, %ld],%c",
01210         eg->fsign, eg->nsym, eg->esym, eg->nsym1, eg->multp, nl);
01211
01212 // nodes
01213 fprintf(outgrpp, "  \"Nodes\": [%c", nl);
01214 for (nd = 0; nd < eg->nNodes; nd++) {
01215     node = eg->nodes[nd];
01216     if (model==NULL || !eg->assigned) {
01217         s = undef;
01218     } else if (eg->isExternal(nd)) {
01219         s = model->particleName(node->intrct);
01220     } else {
01221         s = model->interacts[node->intrct]->name;
01222     }
01223     fprintf(outgrpp, "    [%d, \"%s\", %d, [",
01224             nd, s, node->extloop);
01225     for (lg = 0; lg < eg->nodes[nd]->deg; lg++) {
01226         if (lg != 0) {
01227             fprintf(outgrpp, ", ");
01228         }
01229         fprintf(outgrpp, "%d", eg->nodes[nd]->edges[lg]);
01230     }
01231     fprintf(outgrpp, "],%c", nl);
01232 }
01233 fprintf(outgrpp, " ],%c", nl);
01234
01235 // edges
01236 fprintf(outgrpp, "  \"Edges\": [%c", nl);
01237 for (ed = 0; ed < eg->nEdges; ed++) {
01238     if (model != NULL) {
01239         s = model->particleName(eg->edges[ed]->ptcl);
01240         fprintf(outgrpp, "    [%d, \"%s\", [%d, %d]],%c",
01241                 ed+1, s, eg->edges[ed]->nodes[0], eg->edges[ed]->nodes[1], nl);
01242     } else {
01243         fprintf(outgrpp, "    [%d, \"Undef\", [%d, %d]],%c",
01244                 ed+1, eg->edges[ed]->nodes[0], eg->edges[ed]->nodes[1], nl);
01245     }
01246 }
01247
01248 fprintf(outgrpp, " ],%c", nl);
01249
01250 fprintf(outgrpp, "};\n");
01251 }
01252
01253 //-----
01254 void Output::outModelF(void)
01255 {
01256     char      *mdlfn;
01257     FILE      *mdlfp;
01258     Particle   *pt;
01259     Interaction *vt;
01260     int       j, k, l, ln;

```

```

01261     const char      *ptn;
01262
01263     if (model == NULL) {
01264         if (prlevel > 0) {
01265             fprintf(GRCC_Stderr, "*** Output::outModel : model is not defined\n");
01266         }
01267         return;
01268     }
01269     ln = strlen(model->name)+5;
01270     mdlfn = new char[ln];
01271     snprintf(mdlfn, ln, "%s.mdl", model->name);
01272
01273     if ((mdlfp = fopen(mdlfn, "w")) == NULL) {
01274         if (prlevel > 0) {
01275             fprintf(GRCC_Stderr, "*** Output::outModel : cannot open \"%s\\n\"",
01276                 mdlfn);
01277         }
01278         return;
01279     }
01280
01281     for (l = 0; l < 60; l++) { putc('% ', mdlfp); } putc('\n', mdlfp);
01282     fprintf(mdlfp, "%s File \"%s\\n\"", mdlfn);
01283     fprintf(mdlfp, "%s Generated by graph generator\n");
01284     fprintf(mdlfp, " Version={2,2,0};\n");
01285
01286     // coupling constants
01287     fprintf(mdlfp, " Order={");
01288     for (j = 0; j < model->ncouple; j++) {
01289         if (j != 0) {
01290             fprintf(mdlfp, ", ");
01291         }
01292         fprintf(mdlfp, "%s", model->cnlist[j]);
01293     }
01294     fprintf(mdlfp, "};\n");
01295
01296     for (l = 0; l < 60; l++) { putc('% ', mdlfp); } putc('\n', mdlfp);
01297     fprintf(mdlfp, "%s particles\n");
01298     for (l = 0; l < 60; l++) { putc('% ', mdlfp); } putc('\n', mdlfp);
01299     for (j = 1; j < model->nParticles; j++) {
01300         pt = model->particles[j];
01301         fprintf(mdlfp, " Particle=%s; Antiparticle=%s;\n",
01302             pt->name, pt->aname);
01303         ptn = pt->typeGName();
01304         fprintf(mdlfp, " PType=%s; Charge=0; Color=1; Mass=0; Width=0;\n", ptn);
01305         fprintf(mdlfp, " PCode=%d; Massless; \n", pt->pcode);
01306         fprintf(mdlfp, " Pend;\n");
01307         fprintf(mdlfp, "%s\n");
01308     }
01309     for (l = 0; l < 60; l++) { putc('% ', mdlfp); } putc('\n', mdlfp);
01310     fprintf(mdlfp, "%s interactions\n");
01311     for (l = 0; l < 60; l++) { putc('% ', mdlfp); } putc('\n', mdlfp);
01312     for (j = 0; j < model->nInteracts; j++) {
01313         vt = model->interacts[j];
01314         fprintf(mdlfp, " Vertex={");
01315         for (k = 0; k < vt->nplist; k++) {
01316             if (k != 0) {
01317                 fprintf(mdlfp, ", ");
01318             }
01319             fprintf(mdlfp, "%s", model->particleName(vt->plist[k]));
01320         }
01321         fprintf(mdlfp, "}; ");
01322         for (k = 0; k < model->ncouple; k++) {
01323             fprintf(mdlfp, "%s=%d; ", model->cnlist[k], vt->clist[k]);
01324         }
01325         fprintf(mdlfp, "FName=%s; Vend;\n", vt->name);
01326     }
01327     for (l = 0; l < 60; l++) { putc('% ', mdlfp); } putc('\n', mdlfp);
01328     fprintf(mdlfp, " Mend;\n");
01329     for (l = 0; l < 60; l++) { putc('% ', mdlfp); } putc('\n', mdlfp);
01330     fclose(mdlfp);
01331     delete[] mdlfn;
01332 }
01333
01334 //-----
01335 void Output::outModelP(void)
01336 {
01337     char      *mdlfn;
01338     FILE      *mdlfp;
01339     Particle   *pt;
01340     Interaction *vt;
01341     int        j, k, l, ln;
01342     const char *ptn;
01343
01344     if (model == NULL) {
01345         if (prlevel > 0) {
01346             fprintf(GRCC_Stderr, "*** Output::outModel : ");
01347             fprintf(GRCC_Stderr, "model is not defined\n");

```



```

01348     }
01349     return;
01350 }
01351 ln = strlen(model->name)+5;
01352 mdlfn = new char[ln];
01353 snprintf(mdlfn, ln, "%s.mdp", model->name);
01354
01355 if ((mdlfp = fopen(mdlfn, "w")) == NULL) {
01356     if (prlevel > 0) {
01357         fprintf(GRCC_Stderr, "*** Output::outModel : cannot open \"%s\\n\",",
01358             mdlfn);
01359     }
01360     return;
01361 }
01362
01363 for (l = 0; l < 60; l++) { putc('#', mdlfp); } putc('\n', mdlfp);
01364 fprintf(mdlfp, "# File \"%s\\n\", mdlfn);
01365 fprintf(mdlfp, "# Generated by graph generator grcc\\n");
01366
01367 // coupling constants
01368 fprintf(mdlfp, "Model={\\n");
01369 fprintf(mdlfp, "  \\\"Model\\\": {");
01370 fprintf(mdlfp, "\\\"%s\\\", %d, [", model->name, model->ncouple);
01371 for (j = 0; j < model->ncouple; j++) {
01372     if (j != 0) {
01373         fprintf(mdlfp, ", ");
01374     }
01375     fprintf(mdlfp, "%s", model->cnlist[j]);
01376 }
01377 fprintf(mdlfp, "],\\n");
01378
01379 fprintf(mdlfp, "  \\\"Particles\\\": {\\n");
01380 for (j = 1; j < model->nParticles; j++) {
01381     pt = model->particles[j];
01382     ptn = pt->typeGName();
01383     fprintf(mdlfp, "    [\\\"%s\\\", \\\"%s\\\", \\\"%s\\\",\\n",
01384         pt->name, pt->aname, ptn);
01385 }
01386 fprintf(mdlfp, "  \\\"Interactions\\\": {\\n");
01387 for (j = 0; j < model->nInteracts; j++) {
01388     vt = model->interacts[j];
01389     fprintf(mdlfp, "    [\\\"%s\\\", %d, [", vt->name, vt->nplist);
01390     for (k = 0; k < vt->nplist; k++) {
01391         if (k != 0) {
01392             fprintf(mdlfp, ", ");
01393         }
01394         fprintf(mdlfp, "%s", model->particleName(vt->plist[k]));
01395     }
01396     fprintf(mdlfp, "], [");
01397     for (k = 0; k < model->ncouple; k++) {
01398         if (k != 0) {
01399             fprintf(mdlfp, ", ");
01400         }
01401         fprintf(mdlfp, "%d", vt->clist[k]);
01402     }
01403     fprintf(mdlfp, "],\\n");
01404 }
01405 fprintf(mdlfp, " ],\\n");
01406 fprintf(mdlfp, "],\\n");
01407 fclose(mdlfp);
01408 delete[] mdlfn;
01409 }
01410
01411 //*****
01412 // model.cc
01413
01414 //=====
01415
01416 static const char *ptypenames[] = GRCC_PT_Table;
01417 static const char *pgtypenames[] = GRCC_PT_GTable;
01418
01419 //=====
01420 // class Particle
01421 //-----
01422 Particle::Particle(Model *mdl, int pid, PInput *pinp)
01423 {
01424     static char buff[100];
01425     static int nbuff=100;
01426     int j;
01427
01428     mdl = mdl; // the model
01429     id = pid; // id of this particle
01430     if (pinp->name == NULL || strlen(pinp->name) < 1) {
01431         if (pinp->pcode != 0) {
01432             snprintf(buff, nbuff, "pa%d", pinp->pcode);
01433             name = strdup(buff);
01434         } else {

```

```

01435         snprintf(buff, nbuff, "pi%d", pid);
01436         name = strdup(buff);
01437     }
01438 } else {
01439     name = strdup(pinp->name); // the name of the particle
01440 }
01441 if (pinp->aname == NULL) {
01442     if (pinp->acode != 0) {
01443         snprintf(buff, nbuff, "ap%d", Abs(pinp->acode));
01444         aname = strdup(buff);
01445     } else {
01446         snprintf(buff, nbuff, "ai%d", pid);
01447         aname = strdup(buff);
01448     }
01449 } else {
01450     aname = strdup(pinp->aname); // the name of the anti-particle
01451 }
01452 pcode = pinp->pcode;
01453 acode = pinp->acode;
01454 if (Abs(mdl->defpart) == GRCC_DEFBYCODE) {
01455     neutral = (pcode == acode);
01456 } else if (Abs(mdl->defpart) == GRCC_DEFBYNAME) {
01457     neutral = ((strcmp(name, aname)==0) ? True : False);
01458 }
01459
01460 // convert ptype string to its code.
01461 if (Abs(mdl->defpart) == GRCC_DEFBYCODE) {
01462     if (pinp->ptypespec < GRCC_PT_Undef || pinp->ptypespec > GRCC_PT_Size) {
01463         if (prlevel > 0) {
01464             fprintf(GRCC_Stderr, "*** particle type is not defined: %d\n",
01465                 pinp->ptypespec);
01466         }
01467         erEnd("particle type is not defined (illegal code)");
01468     }
01469     ptype = pinp->ptypespec;
01470 } else if (Abs(mdl->defpart) == GRCC_DEFBYNAME) {
01471     if (pinp->ptypen == NULL) {
01472         erEnd("particle type is not defined (NULL)");
01473     }
01474     ptype = -1;
01475     for (j = 0; j < GRCC_PT_Size; j++) {
01476         if (strcmp(pinp->ptypen, ptypenames[j]) == 0) {
01477             ptype = j;
01478             break;
01479         }
01480     }
01481     if (ptype < 0) {
01482         if (prlevel > 0) {
01483             fprintf(GRCC_Stderr, "*** particle type \"%s\" is not defined\n",
01484                 pinp->ptypen);
01485         }
01486         erEnd("particle type is not defined (name)");
01487     }
01488 }
01489 if (ptype == GRCC_PT_Undef && ptype != 0) {
01490     erEnd("illegal ptype");
01491 }
01492
01493 extonly = pinp->extonly;
01494 cmindeg = 0;
01495 cmaxdeg = 0;
01496 }
01497
01498 //-----
01499 Particle::~Particle(void)
01500 {
01501     free(aname);
01502     free(name);
01503 }
01504
01505 //-----
01506 char *Particle::particleName(int p)
01507 {
01508     if (p < 0) {
01509         return aname;
01510     } else {
01511         return name;
01512     }
01513 }
01514
01515 //-----
01516 int Particle::particleCode(int p)
01517 {
01518     if (p < 0) {
01519         return acode;
01520     } else {
01521         return pcode;

```

```

01522     }
01523 }
01524
01525 //-----
01526 int Particle::isNeutral(void)
01527 {
01528     return neutral;
01529 }
01530
01531 //-----
01532 const char *Particle::typeName(void)
01533 {
01534     return ptypenames[ptype];
01535 }
01536
01537 //-----
01538 const char *Particle::typeGName(void)
01539 {
01540     return pgtypenames[ptype];
01541 }
01542
01543 //-----
01544 char *Particle::aparticle(void)
01545 {
01546     static char prtcl[] = "Particle";
01547
01548     if (isNeutral()) {
01549         return prtcl;
01550     } else {
01551         return aname;
01552     }
01553 }
01554
01555 //-----
01556 void Particle::prParticle(void)
01557 {
01558     if (isNeutral()) {
01559         printf("pid=%d, name=\"%s\", real_field, ", id, name);
01560     } else {
01561         printf("pid=%d, name=\"%s\", aname=\"%s\", ", id, name, aname);
01562     }
01563     printf("ptype=%s, pcode=%d, acode=%d, extonly=%d, cdeg=(%d,%d)\n",
01564           ptypenames[ptype], pcode, acode, extonly, cmindeg, cmaxdeg);
01565 }
01566
01567 //=====
01568 // class Interaction
01569 //-----
01570 Interaction::Interaction(Model *mdl, int iid, const char *nam, int icd, int *cpl, int nlgs, int
01571 *plst, int csm, int lp)
01572 {
01573     static char buff[100];
01574     static int nbuff=100;
01575     Particle *p;
01576     int j, ndir, sdir, jdir, nmaj, jmaj, ngho, sgho, jgho, ptcl;
01577     Bool ok;
01578
01579     mdl = mdl; // the model
01580     id = iid; // id of this interaction
01581     icode = icd; // id of this interaction
01582     csum = csm; // the total orders of coupling constants
01583     nlegs = nlgs; // the size of plist[]
01584     loop = lp; // the number of loops
01585
01586     if (nam == NULL || strlen(nam) < 1) {
01587         snprintf(buff, nbuff, "vt%d", icd);
01588         name = strdup(buff);
01589     } else {
01590         name = strdup(nam); // the name of the interaction
01591     }
01592     nclist = mdl->ncouple;
01593     clist = new int[nclist];
01594     icode = icd;
01595     for (j = 0; j < mdl->ncouple; j++) {
01596         clist[j] = cpl[j];
01597     }
01598     plist = new int[nlegs];
01599     nplist = nlegs;
01600     for (j = 0; j < nlegs; j++) {
01601         plist[j] = plst[j];
01602     }
01603     nslist = 0;
01604     slist = NULL;
01605
01606     // check fermion number conservation
01607     ndir = 0;
01608     sdir = 0;

```

```

01608     jdir = 0;
01609     nmaj = 0;
01610     jmaj = 0;
01611     ngho = 0;
01612     sgho = 0;
01613     jgho = 0;
01614     ok = True;
01615     for (j = 0; j < nplist; j++) {
01616         if (plist[j] > 0) {
01617             p = mdl->particles[plist[j]];
01618             ptcl = 1;
01619         } else {
01620             p = mdl->particles[-plist[j]];
01621             ptcl = -1;
01622         }
01623
01624         if (p->ptype == GRCC_PT_Dirac) {
01625             ndir++;
01626             if (ptcl > 0) {
01627                 sdir++;
01628             } else {
01629                 sdir--;
01630             }
01631             if (Abs(sdir) == 1) {
01632                 jdir = j;
01633             } else if ((Abs(sdir) == 0 && j != jdir + 1) || Abs(sdir) > 1) {
01634                 fprintf(GRCC_Stderr, "*** Interaction: Dirac and anti-Dirac should "
01635                     "be arranged in pairs in an interaction.\n");
01636                 ok = False;
01637             }
01638         } else if (p->ptype == GRCC_PT_Ghost) {
01639             ngho++;
01640             if (ptcl > 0) {
01641                 sgho++;
01642             } else {
01643                 sgho--;
01644             }
01645             if (Abs(sgho) == 1) {
01646                 jgho = j;
01647             } else if ((Abs(sgho) == 0 && j != jgho + 1) || Abs(sgho) > 1) {
01648                 fprintf(GRCC_Stderr, "*** Interaction: Ghost and anti-Ghost should "
01649                     "be arranged in pairs in an interaction.\n");
01650                 ok = False;
01651             }
01652         } else if (p->ptype == GRCC_PT_Majorana) {
01653             nmaj++;
01654             if (nmaj % 2 == 1) {
01655                 jmaj = j;
01656             } else if (j != jmaj + 1) {
01657                 fprintf(GRCC_Stderr, "*** Interaction: two Majoranas should "
01658                     "be arranged in pairs in an interaction.\n");
01659                 ok = False;
01660             }
01661         }
01662     }
01663     if (sdir != 0) {
01664         fprintf(GRCC_Stderr, "*** Interaction: Dirac number is not conserved\n");
01665         ok = False;
01666     }
01667     if (sgho != 0) {
01668         fprintf(GRCC_Stderr, "*** Interaction: Ghost number is not conserved\n");
01669         ok = False;
01670     }
01671     if (nmaj % 2 != 0) {
01672         fprintf(GRCC_Stderr, "*** Interaction: odd number of Majorana particles\n");
01673         ok = False;
01674     }
01675     if (ndir + nmaj + ngho > 2) {
01676         fprintf(GRCC_Stderr, "+++ Interaction: more than 2 Dirac/Majorana/Ghost "
01677             "particles are interacting.\n");
01678         fprintf(GRCC_Stderr, "    Interaction has %d Dirac, %d Ghost and %d Majorana "
01679             "particles.\n", ndir, nmaj, ngho);
01680         fprintf(GRCC_Stderr, "    Sign factors related to fermions will not "
01681             "be calculated correctly.\n");
01682         mdl->skipFLine = True;
01683     }
01684     if (!ok) {
01685         prInteraction();
01686         erEnd("illegal Dirac/Majorana/Ghost interaction");
01687     }
01688 }
01689
01690 //-----
01691 Interaction::~Interaction(void)
01692 {
01693     if (slist != NULL) {
01694         delete[] slist;

```

```

01695     }
01696     if (plist != NULL) {
01697         delete[] plist;
01698     }
01699     if (clist != NULL) {
01700         delete[] clist;
01701     }
01702     free(name);
01703 }
01704
01705 //-----
01706 void Interaction::prInteraction(void)
01707 {
01708     int j;
01709
01710     printf("vid=%d, icode=%d, name=\"%s\", loop=%d, csum=%d, cpl=",
01711           id, icode, name, loop, csum);
01712     prIntArray(mdl->ncouple, clist, "", legs="");
01713     for (j = 0; j < nplist; j++) {
01714         if (j != 0) {
01715             printf(", ");
01716         }
01717         printf("%s", mdl->particleName(plist[j]));
01718     }
01719     printf("];\n");
01720 }
01721 }
01722
01723 //=====
01724 // class Model
01725 //-----
01726 Model::Model(MInput *minp)
01727 {
01728     static PInput pundef = {"undef", "undef", 0, 0, "undef", 0, 0};
01729     int j;
01730
01731     if (minp->name == NULL || strlen(minp->name) < 1) {
01732         erEnd("model should have a name");
01733     }
01734     name = strdup(minp->name);
01735     ncouple = minp->ncouple;
01736     if (ncouple >= GRCC_MAXNCPLG) {
01737         erEnd("too many coupling constants in the model (GRCC_MAXNCPLG)");
01738     }
01739     // cnlist = minp->cnamlist;
01740     cnlist = new char*[ncouple];
01741     for (j = 0; j < ncouple; j++) {
01742         cnlist[j] = strdup(minp->cnamlist[j]);
01743     }
01744
01745     nParticles = 0;
01746     particles = new Particle*[GRCC_MAXMPARTICLES];
01747     for (j = 0; j < GRCC_MAXMPARTICLES; j++) {
01748         particles[j] = NULL;
01749     }
01750     pdef = False;
01751
01752     nInteracts = 0;
01753     interacts = new Interaction*[GRCC_MAXMINTERACT];
01754     for (j = 0; j < GRCC_MAXMINTERACT; j++) {
01755         interacts[j] = NULL;
01756     }
01757     vdef = False;
01758
01759     if (particles == NULL || interacts == NULL) {
01760         erEnd("memory allocation failed");
01761     }
01762
01763 #ifdef UNIQINTR
01764     if (minp->defpart != GRCC_DEFBYNAME
01765         && minp->defpart != GRCC_DEFBYCODE) {
01766         erEnd("illegal value of defpart");
01767     }
01768 #else
01769     if (Abs(minp->defpart) != GRCC_DEFBYNAME
01770         && Abs(minp->defpart) != GRCC_DEFBYCODE) {
01771         erEnd("illegal value of defpart");
01772     }
01773 #endif
01774
01775     defpart = minp->defpart;
01776
01777     maxnlegs = 0;
01778     maxcpl = 0;
01779     maxloop = 0;
01780     ncplgcp = 0;
01781     skipFLine = False;

```

```

01782
01783     addParticle(&pundef);
01784 }
01785
01786 //-----
01787 Model::~Model(void)
01788 {
01789     int j;
01790
01791     for (j = ncplgcp-1; j >= 0; j--) {
01792         cplgvl[j] = delintdup(cplgvl[j]);
01793     }
01794     delete[] cplgvl;
01795     cplgnvl = delintdup(cplgnvl);
01796     cplglg = delintdup(cplglg);
01797     cplgcp = delintdup(cplgcp);
01798
01799     for (j = GRCC_MAXMINTERACT-1; j >= 0; j--) {
01800         if (interacts[j] != NULL) {
01801             interacts[j]->slist = delintdup(interacts[j]->slist);
01802             delete interacts[j];
01803             interacts[j] = NULL;
01804         }
01805     }
01806     delete[] interacts;
01807     interacts = NULL;
01808
01809     for (j = GRCC_MAXMPARTICLES-1; j >= 0; j--) {
01810         if (particles[j] != NULL) {
01811             delete particles[j];
01812             particles[j] = NULL;
01813         }
01814     }
01815     delete[] particles;
01816     particles = NULL;
01817
01818     for (j=ncouple-1; j>=0; j--) {
01819         free(cnlist[j]);
01820     }
01821     delete[] cnlist;
01822
01823     free(name);
01824 }
01825
01826 //-----
01827 void Model::prModel(void)
01828 {
01829     static char hdr[] = "#=====\n";
01830     static char hdl[] = "#-----\n";
01831     int j;
01832
01833     printf("%s", hdr);
01834     printf("Model=\"%s\\", " ", name);
01835     printf("coupling=");
01836     for (j = 0; j < ncouple; j++) {
01837         if (j != 0) {
01838             printf(", ");
01839         }
01840         printf("%s\\", cnlist[j]);
01841     }
01842     printf("];\n");
01843     printf("%s", hdl);
01844     printf("Particles=%d;\n", nParticles);
01845     for (j = 1; j < nParticles; j++) {
01846         particles[j]->prParticle();
01847     }
01848     printf("EndParticle;\n");
01849     printf("allPart (%d) = [", nallPart);
01850     for (j = 0; j < nallPart; j++) {
01851         if (j != 0) {
01852             printf(", ");
01853         }
01854         printf("%d", allPart[j]);
01855     }
01856     printf("]\n");
01857
01858     printf("%s", hdl);
01859     printf("Interactions=%d;\n", nInteracts);
01860     for (j = 0; j < nInteracts; j++) {
01861         interacts[j]->prInteraction();
01862     }
01863     printf("EndInteraction;\n");
01864     printf("%s", hdl);
01865     printf("InteractionTable;\n");
01866     printf("# %d class in (total coupling, degree)\n", ncplgcp);
01867     for (j = 0; j < ncplgcp; j++) {
01868         printf(" class=%d: cp=%d, lg=%d, vl=", j, cplgcp[j], cplglg[j]);

```

```

01869         prIntArray(cplgnvl[j], cplgvl[j], "\n");
01870     }
01871     printf("EndInteractionTable;\n");
01872     printf("%s", hdr);
01873 }
01874
01875 //-----
01876 void Model::addParticle(PInput *pinp)
01877 {
01878     int nid, aid, pid, pcd, acd;
01879
01880     if (nParticles >= GRCC_MAXMPARTICLES) {
01881         erEnd("particle table overflow (GRCC_MAXMPARTICLES)");
01882     }
01883
01884     // uniqueness check
01885     if (Abs(defpart) == GRCC_DEFBYCODE) {
01886         pcd = findParticleCode(pinp->pcode);
01887         acd = findParticleCode(pinp->acode);
01888         if (pcd > 0 || acd > 0) {
01889             if (prlevel > 0) {
01890                 fprintf(GRCC_Stderr, "*** particle code [%d, %d] ",
01891                     pinp->pcode, pinp->acode);
01892                 fprintf(GRCC_Stderr, "is already used.\n");
01893             }
01894             erEnd("particle code is already defined");
01895         }
01896     } else if (Abs(defpart) == GRCC_DEFBYNAME) {
01897         if (pinp->name == NULL || pinp->aname == NULL ||
01898             strlen(pinp->name) < 1 || strlen(pinp->aname) < 1) {
01899             erEnd("no name of particle or anti-particle.");
01900         }
01901         nid = findParticleName(pinp->name);
01902         aid = findParticleName(pinp->aname);
01903         if (nid > 0 || aid > 0) {
01904             if (prlevel > 0) {
01905                 fprintf(GRCC_Stderr, "*** particle name [%s, %s] ",
01906                     pinp->name, pinp->aname);
01907                 fprintf(GRCC_Stderr, "is already used.\n");
01908             }
01909             erEnd("particle name is already defined.");
01910         }
01911     }
01912
01913     pid = nParticles++;
01914     particles[pid] = new Particle(this, pid, pinp);
01915 }
01916
01917 //-----
01918 void Model::addParticleEnd(void)
01919 {
01920     int p, ap;
01921
01922     pdef = True;
01923
01924     #ifdef OPTEXTONLY
01925         nallPart = 0;
01926         for (p = nParticles-1; p >= 1; p--) {
01927             if (!particles[p]->extonly) {
01928                 ap = antiParticle(p);
01929                 if (ap != p) {
01930                     allPart[nallPart++] = ap;
01931                 }
01932             }
01933         }
01934         for (p = 1; p < nParticles; p++) {
01935             if (!particles[p]->extonly) {
01936                 allPart[nallPart++] = p;
01937             }
01938         }
01939         sorti(nallPart, allPart);
01940     #else
01941         nallPart = 0;
01942         for (p = nParticles-1; p >= 1; p--) {
01943             ap = antiParticle(p);
01944             if (ap != p) {
01945                 allPart[nallPart++] = ap;
01946             }
01947         }
01948         for (p = 1; p < nParticles; p++) {
01949             allPart[nallPart++] = p;
01950         }
01951         sorti(nallPart, allPart);
01952     #endif
01953 }
01954
01955 //-----

```

```

01956 void Model::addInteraction(IInput *iinp)
01957 {
01958     int vid, nlegs, j, k, c;
01959     int csum, lp2, lp;
01960     int plist[GRCC_MAXLEGS];
01961
01962     if (!pdef) {
01963         erEnd("call 'addParticleEnd' before 'addInteraction'");
01964     }
01965     if (nInteracts >= GRCC_MAXMINTERACT) {
01966         erEnd("too many interactions in the model (GRCC_MAXMINTERACT)");
01967     }
01968     nlegs = iinp->nplistn;
01969     if (nlegs >= GRCC_MAXLEGS) {
01970         erEnd("too many legs in an interaction (GRCC_MAXLEGS)");
01971     }
01972
01973     // check identifier
01974     if (defpart == GRCC_DEFBYCODE) {
01975         if (iinp->icode < 0) {
01976             if (prlevel > 0) {
01977                 fprintf(GRCC_Stderr, "*** vertex code %d should be positive\n",
01978                     iinp->icode);
01979             }
01980             erEnd("vertex code should be positive");
01981         } else {
01982             vid = findInteractionCode(iinp->icode);
01983             if (vid >= 0) {
01984                 if (prlevel > 0) {
01985                     fprintf(GRCC_Stderr, "*** vertex code %d is already used\n",
01986                         iinp->icode);
01987                 }
01988                 erEnd("vertex code is already used");
01989             }
01990         }
01991     } else if (defpart == GRCC_DEFBYNAME) {
01992         if (iinp->name == NULL || strlen(iinp->name) < 1) {
01993             erEnd("no name of vertex\n");
01994         }
01995         vid = findInteractionName(iinp->name);
01996         if (vid >= 0) {
01997             if (prlevel > 0) {
01998                 fprintf(GRCC_Stderr, "*** vertex name %s is already used\n",
01999                     iinp->name);
02000                 erEnd("vertex name is already used");
02001             }
02002         }
02003     } else {
02004 #ifdef UNIQINTR
02005         // defpart ==< 0
02006         erEnd("illegal value of defpart");
02007 #else
02008         // don't care about duplicated name/code of interaction
02009         ;
02010 #endif
02011     }
02012     vid = nInteracts++;
02013
02014     for (j = 0; j < nlegs; j++) {
02015         if (Abs(defpart) == GRCC_DEFBYCODE) {
02016             plist[j] = findParticleCode(iinp->plistc[j]);
02017             if (plist[j] == 0) {
02018                 if (prlevel > 0) {
02019                     fprintf(GRCC_Stderr, "*** particle code %d ",
02020                         iinp->plistc[j]);
02021                     fprintf(GRCC_Stderr, "is not defined\n");
02022                 }
02023                 erEnd("particle is not defined (code)");
02024             }
02025         } else if (Abs(defpart) == GRCC_DEFBYNAME) {
02026             plist[j] = findParticleName(iinp->plistn[j]);
02027             if (plist[j] == 0) {
02028                 if (prlevel > 0) {
02029                     fprintf(GRCC_Stderr, "*** particle name %s ",
02030                         iinp->plistn[j]);
02031                     fprintf(GRCC_Stderr, "is not defined\n");
02032                 }
02033                 erEnd("particle is not defined (name)");
02034             }
02035         }
02036     }
02037
02038     csum = 0;
02039     for (j = 0; j < ncouple; j++) {
02040         c = iinp->cvalist[j];
02041         if (c < 0) {
02042             if (prlevel > 0) {

```



```

02043         fprintf(GRCC_Stderr, "*** illegal value of c-constants \n");
02044         for (k = 0; k < ncouple; k++) {
02045             fprintf(GRCC_Stderr, "      %s = %d\n",
02046                     cnlist[k], iinp->cvallist[k]);
02047         }
02048     }
02049     erEnd("illegal value of c-constants");
02050 }
02051 csum += c;
02052 }
02053 if (csum < 0) {
02054     if (prlevel > 0) {
02055         fprintf(GRCC_Stderr, "*** illegal total coupling-constants %d\n",
02056                 csum);
02057         for (k = 0; k < ncouple; k++) {
02058             fprintf(GRCC_Stderr, "      %s = %d\n",
02059                     cnlist[k], iinp->cvallist[k]);
02060         }
02061     }
02062     erEnd("illegal value of c-constants");
02063 }
02064
02065 // assume : csum = nlegs - 2 + 2*loop
02066 lp2 = csum - nlegs + 2;
02067 if (lp2 % 2 != 0 || lp2 < 0) {
02068     if (prlevel > 0) {
02069         fprintf(GRCC_Stderr, "*** illegal coupling const : ");
02070         fprintf(GRCC_Stderr, "nlegs - 2 + 2*loop: 2*loop = %d\n", lp2);
02071     }
02072 }
02073 lp = lp2/2;
02074
02075 maxnlegs = Max(maxnlegs, nlegs);
02076 maxloop = Max(maxloop, lp);
02077 maxcpl = Max(maxcpl, csum);
02078
02079 interacts[vid] = new Interaction(this, vid, iinp->name, iinp->icode,
02080                                 iinp->cvallist, nlegs, plist, csum, lp);
02081 }
02082
02083 //-----
02084 void Model::addInteractionEnd(void)
02085 {
02086     int lst[GRCC_MAXMINTERACT];
02087     int tcp[GRCC_MAXMINTERACT], tlg[GRCC_MAXMINTERACT];
02088     int tnvl[GRCC_MAXMINTERACT], *tv1[GRCC_MAXMINTERACT];
02089     Interaction *vt;
02090     int cp, lg, j, k, p;
02091
02092     vdef = True;
02093     ncplgcp = 0;
02094     for (cp = 0; cp <= maxcpl; cp++) {
02095         for (lg = 0; lg <= maxnlegs; lg++) {
02096             k = 0;
02097             for (j = 0; j < nInteracts; j++) {
02098                 vt = interacts[j];
02099                 if (vt->csum == cp && vt->nlegs == lg) {
02100                     lst[k++] = j;
02101                 }
02102             }
02103         }
02104         if (k > 0) {
02105             tcp[ncplgcp] = cp;
02106             tlg[ncplgcp] = lg;
02107             tnvl[ncplgcp] = k;
02108             tv1[ncplgcp] = intdup(k, lst);
02109             ncplgcp++;
02110         }
02111     }
02112 }
02113 cplgcp = intdup(ncplgcp, tcp);
02114 cplglg = intdup(ncplgcp, tlg);
02115 cplgnvl = intdup(ncplgcp, tnvl);
02116 cplgvl = new int*[ncplgcp];
02117 for (j = 0; j < ncplgcp; j++) {
02118     if (cplgnvl[j] > 0) {
02119         cplgvl[j] = tv1[j];
02120     } else {
02121         cplgvl[j] = NULL;
02122     }
02123 }
02124
02125 for (j = 0; j < nInteracts; j++) {
02126     vt = interacts[j];
02127     if (vt->slist != NULL) {
02128         erEnd("addInteractionEnd: vt->nslist != NULL");
02129     }

```

```

02130         vt->slist = intdup(vt->nplist, vt->plist);
02131         vt->nslist = toSList(vt->nplist, vt->slist);
02132     }
02133
02134     for (p = 0; p < nParticles; p++) {
02135         particles[p]->cmindeg = 0;
02136         particles[p]->cmaxdeg = 0;
02137     }
02138     for (j = 0; j < nInteracts; j++) {
02139         vt = interacts[j];
02140         for (k = 0; k < vt->nplist; k++) {
02141             p = abs(vt->plist[k]);
02142             if (particles[p]->cmindeg == 0) {
02143                 particles[p]->cmindeg = vt->nlegs;
02144                 particles[p]->cmaxdeg = vt->nlegs;
02145             } else {
02146                 particles[p]->cmindeg = Min(particles[p]->cmindeg, vt->nlegs);
02147                 particles[p]->cmaxdeg = Max(particles[p]->cmaxdeg, vt->nlegs);
02148             }
02149         }
02150     }
02151 #ifdef DEBUG
02152     prModel();
02153 #endif
02154 }
02155
02156 //-----
02157 int Model::findParticleName(const char *name)
02158 {
02159     int j;
02160
02161     for (j = 0; j < nParticles; j++) {
02162         if (strcmp(name, particles[j]->name) == 0) {
02163             return j;
02164         }
02165     }
02166     for (j = 0; j < nParticles; j++) {
02167         if (strcmp(name, particles[j]->aname) == 0) {
02168             return -j;
02169         }
02170     }
02171     return 0;
02172 }
02173
02174 //-----
02175 int Model::findParticleCode(int pcd)
02176 {
02177     int j;
02178
02179     for (j = 0; j < nParticles; j++) {
02180         if (pcd == particles[j]->pcode) {
02181             return j;
02182         } else if (pcd == particles[j]->acode) {
02183             return -j;
02184         }
02185     }
02186     return 0;
02187 }
02188
02189 //-----
02190 char *Model::particleName(int p)
02191 {
02192     int q;
02193
02194     if (Abs(p) >= nParticles) {
02195         if (prlevel > 0) {
02196             fprintf(GRCC_Stderr, "\n*** Model::particleName: ");
02197             fprintf(GRCC_Stderr, "illegal particle id=%d\n", p);
02198         }
02199         erEnd("Model::particleName: illegal particle");
02200     }
02201     if (p < 0) {
02202         q = -p;
02203     } else {
02204         q = p;
02205     }
02206     return particles[q]->particleName(p);
02207 }
02208
02209 //-----
02210 int Model::particleCode(int p)
02211 {
02212     int q;
02213
02214     if (Abs(p) >= nParticles) {
02215         fprintf(GRCC_Stderr, "\n*** Model::particleCode: illegal particle id=%d\n",
02216             p);

```

```

02217         erEnd("Model::particleCode: illegal particle");
02218     }
02219     if (p < 0) {
02220         q = - p;
02221     } else {
02222         q = p;
02223     }
02224     return particles[q]->particleCode(p);
02225 }
02226
02227 //-----
02228 void Model::prParticleArray(int n, int *a, const char *msg)
02229 {
02230     int j;
02231
02232     printf("[");
02233     for (j = 0; j < n; j++) {
02234         if (j != 0) {
02235             printf(", ");
02236         }
02237         printf("%s", particleName(a[j]));
02238     }
02239     printf("] %s", msg);
02240 }
02241
02242 //-----
02243 int Model::findMClass(const int cpl, const int dgr)
02244 {
02245     int j;
02246
02247     for (j = 0; j < ncplgcp; j++) {
02248         if (cplgcp[j] == cpl && cplglg[j] == dgr) {
02249             return j;
02250         }
02251     }
02252     return -1;
02253 }
02254
02255 //-----
02256 int Model::normalParticle(int pt)
02257 {
02258     int ptc = Abs(pt);
02259     Particle *p;
02260
02261     if (ptc >= nParticles) {
02262         return 0;
02263     }
02264     p = particles[ptc];
02265     if (p->isNeutral()) {
02266         return ptc;
02267     } else {
02268         return pt;
02269     }
02270 }
02271
02272 //-----
02273 int Model::antiParticle(int pt)
02274 {
02275     int ptc = Abs(pt);
02276     Particle *p;
02277
02278     p = particles[ptc];
02279     if (p->isNeutral()) {
02280         return ptc;
02281     } else {
02282         return -pt;
02283     }
02284 }
02285
02286 //-----
02287 int *Model::allParticles(int *len)
02288 {
02289     *len = nallPart;
02290     return allPart;
02291 }
02292
02293 //-----
02294 int Model::findInteractionName(const char *name)
02295 {
02296     int j;
02297
02298     for (j = 0; j < nInteracts; j++) {
02299         if (strcmp(name, interacts[j]->name) == 0) {
02300             return j;
02301         }
02302     }
02303     return -1;

```

```

02304 }
02305
02306 //-----
02307 int Model::findInteractionCode(int icd)
02308 {
02309     int j;
02310
02311     for (j = 0; j < nInteracts; j++) {
02312         if (icd == interacts[j]->icode) {
02313             return j;
02314         }
02315     }
02316     return -1;
02317 }
02318
02319 //*****
02320 // proc.cc
02321
02322 //-----
02323 // class PNodeClass
02324 //-----
02325 PNodeClass::PNodeClass(SProcess *spc, int nnods, int nclss, NCInput *cls)
02326 {
02327     // Create a class of nodes
02328     // nnodes : # nodes
02329     // nclss : # classes
02330     // cls : table of class inputs
02331     //
02332     int j, k, nn, lp2;
02333     Bool ok = True;
02334
02335     sproc = spc;
02336     nnodes = nnods;
02337     nclass = nclss;
02338     deg = new int[nclass];
02339     type = new int[nclass];
02340     particle = new int[nclass];
02341     count = new int[nclass];
02342     couple = new int[nclass];
02343     cmindeg = new int[nclass];
02344     cmaxdeg = new int[nclass];
02345     for (j = 0; j < nclass; j++) {
02346         deg[j] = cls[j].cldeg;
02347         count[j] = cls[j].clnum;
02348         type[j] = cls[j].cltyp;
02349         cmindeg[j] = cls[j].cmindeg;
02350         cmaxdeg[j] = cls[j].cmaxdeg;
02351         particle[j] = cls[j].ptcl;
02352         couple[j] = cls[j].cpl;
02353         lp2 = (couple[j]-deg[j]+2);
02354         if (!isATEXternal(type[j]) && (lp2 % 2 != 0 || lp2 < 0)) {
02355             if (prlevel > 0) {
02356                 fprintf(GRCC_Stderr, "*** PNodeClass: illegal loop: "
02357                     ": 2*loop=cpl[%d] (%d)-deg[%d] (%d)+2 =2*loop = %d\n",
02358                     j, couple[j], j, deg[j], lp2);
02359                 for (k = 0; k < nclss; k++) {
02360                     fprintf(GRCC_Stderr, "k=%d, deg=%d, typ=%d, ptcl=%d, ",
02361                         k, deg[k], type[k], particle[k]);
02362                     fprintf(GRCC_Stderr, "cpl=%d, cnt=%d\n",
02363                         couple[k], count[k]);
02364                 }
02365             }
02366             ok = False;
02367         }
02368     }
02369     if (!ok) {
02370         erEnd("PNodeClass: illegal loop");
02371     }
02372     cl2nd = new int[nclass+1];
02373     nd2cl = new int[nnodes];
02374     cl2mcl = new int[nnodes];
02375
02376     nn = 0;
02377     for (j = 0; j < nclass; j++) {
02378         cl2nd[j] = nn;
02379         for (k = 0; k < count[j]; k++, nn++) {
02380             nd2cl[nn] = j;
02381         }
02382         if (isATEXternal(type[j])) {
02383             cl2mcl[j] = -1;
02384         } else {
02385             cl2mcl[j] = spc->model->findMClass(couple[j], deg[j]);
02386             if (cl2mcl[j] < 0) {
02387                 if (prlevel > 0) {
02388                     fprintf(GRCC_Stderr, "*** PNodeClass : no vertex : ");
02389                     fprintf(GRCC_Stderr, "coupling=%d, degree=%d\n",
02390                         couple[j], deg[j]);

```

```

02391         }
02392         erEnd("PNodeClass : no vertex");
02393     }
02394 }
02395 }
02396 cl2nd[nclass] = nnodes;
02397 }
02398 //-----
02399 PNodeClass::PNodeClass(SProcess *spc, int nnods, int nclass, int *dgs, int *typ, int *ptcl, int *cpl,
int *cnt, int *cmind, int *cmaxd)
02400 {
02401     // Create a class of nodes
02402     // nnodes : # nodes
02403     // nclass : # classes
02404     // dgs : degree of a node, which is common to the vertices in a class
02405     // typ : initial/final/vertex
02406     // ptcl : particle code or list of possible interactions
02407     // cpl : values of coupling constants.
02408     // cnt : the number of nodes in this class
02409     //
02410     int j, k, nn, lp2;
02411     Bool ok = True;
02412
02413     sproc = spc;
02414     nnodes = nnods;
02415     nclass = nclass;
02416     deg = new int[nclass];
02417     type = new int[nclass];
02418     particle = new int[nclass];
02419     count = new int[nclass];
02420     couple = new int[nclass];
02421     cmindeg = new int[nclass];
02422     cmaxdeg = new int[nclass];
02423     for (j = 0; j < nclass; j++) {
02424         deg[j] = dgs[j];
02425         type[j] = typ[j];
02426         particle[j] = ptcl[j];
02427         count[j] = cnt[j];
02428         couple[j] = cpl[j];
02429         cmindeg[j] = cmind[j];
02430         cmaxdeg[j] = cmaxd[j];
02431         lp2 = (cpl[j]-dgs[j]+2);
02432         if (!isATEXternal(type[j]) && (lp2 % 2 != 0 || lp2 < 0)) {
02433             if (prlevel > 0) {
02434                 fprintf(GRCC_Stderr, "*** PNodeClass: illegal loop: "
02435                     ": 2*loop=cpl[%d] (%d)-deg[%d] (%d)+2 =2*loop = %d\n",
02436                     j, cpl[j], j, dgs[j], lp2);
02437                 for (k = 0; k < nclass; k++) {
02438                     fprintf(GRCC_Stderr, "k=%d, dgs=%d, typ=%d, ptcl=%d, ",
02439                         k, dgs[k], typ[k], ptcl[k]);
02440                     fprintf(GRCC_Stderr, "cpl=%d, cnt=%d\n", cpl[k], cnt[k]);
02441                 }
02442             }
02443             ok = False;
02444         }
02445     }
02446     if (!ok) {
02447         erEnd("PNodeClass: illegal loop");
02448     }
02449     cl2nd = new int[nclass+1];
02450     nd2cl = new int[nnodes];
02451     cl2mcl = new int[nnodes];
02452
02453     nn = 0;
02454     for (j = 0; j < nclass; j++) {
02455         cl2nd[j] = nn;
02456         for (k = 0; k < count[j]; k++, nn++) {
02457             nd2cl[nn] = j;
02458         }
02459         if (isATEXternal(type[j])) {
02460             cl2mcl[j] = -1;
02461         } else {
02462             cl2mcl[j] = spc->model->findMClass(couple[j], deg[j]);
02463             if (cl2mcl[j] < 0) {
02464                 if (prlevel > 0) {
02465                     fprintf(GRCC_Stderr, "*** PNodeClass : no vertex : ");
02466                     fprintf(GRCC_Stderr, "coupling=%d, degree=%d\n",
02467                         couple[j], deg[j]);
02468                 }
02469                 erEnd("PNodeClass : no vertex");
02470             }
02471         }
02472     }
02473     cl2nd[nclass] = nnodes;
02474 }
02475
02476 //-----

```

```

02477 PNodeClass::~PNodeClass(void)
02478 {
02479     delete[] cl2mcl;
02480     delete[] nd2cl;
02481     delete[] cl2nd;
02482     delete[] cmaxdeg;
02483     delete[] cmindeg;
02484     delete[] couple;
02485     delete[] count;
02486     delete[] particle;
02487     delete[] type;
02488     delete[] deg;
02489 }
02490
02491 //-----
02492 void PNodeClass::prPNodeClass(void)
02493 {
02494     int j;
02495
02496     printf("+++ PNodeClass: nclass=%d, nnodes=%d\n", nclass, nnodes);
02497     for (j = 0; j < nclass; j++) {
02498         prElem(j);
02499     }
02500     printf("\n");
02501 }
02502
02503 //-----
02504 void PNodeClass::prElem(int j)
02505 {
02506     int tp;
02507
02508     printf("%3d: %-7s(%2d), ", j, GRCC_AT_NdStr(type[j]), type[j]);
02509     printf("deg=%d, count=%d, nodes[%d--%d], couple=%d, cmindeg=%d, cmaxdeg=%d",
02510           deg[j], count[j], cl2nd[j], cl2nd[j+1], couple[j], cmindeg[j], cmaxdeg[j]);
02511     tp = type[j];
02512     if (isATEXternal(tp)) {
02513         if (sproc->model != NULL) {
02514             printf(", ptcl=%s ", sproc->model->particleName(particle[j]));
02515         } else {
02516             printf(", ptcl=%d ", particle[j]);
02517         }
02518     }
02519     printf("\n");
02520 }
02521 }
02522
02523 //=====
02524 // class SProcess
02525 //-----
02526 SProcess::SProcess(Model *mdl, Process *prc, Options *opts, int sid, int *clst, int ncls, int *cdeg,
02527                   int *ctyp, int *ptcl, int *cpl, int *cnum, int *cmind, int *cmaxd)
02528 {
02529     // Construct a Subprocess object
02530     //   mdl          : model
02531     //   prc          : process (may be NULL)
02532     //   opts         : options
02533     //   sid          : id of the sprocess
02534     //   ncls         : the number of classes
02535     //   cdeg[ncls]   : the table of degrees of nodes.
02536     //   ctyp[ncls]   : the table of nodes in the classes
02537     //   ptcl[ncls]   : particle(External)/interaction code(Internal)
02538     //   cpl[ncls]    : the table of total order of coupling constants.
02539     //   cnum[ncls]   : the table of nodes in the classes
02540     //   cmind[ncls]  : the table of min(deg of connectabl vertex)
02541     //   cmaxd[ncls]  : the table of max(deg of connectabl vertex)
02542
02543     int j, cp, lp2, ndeg, nvrt, tcp10;
02544     bool ok;
02545
02546     if (ncls < 1) {
02547         fprintf(GRCC_Stderr, "*** SProcess::SProcess: no class %d\n", ncls);
02548         erEnd("SProcess::SProcess: no Class");
02549     }
02550     id = sid;
02551     model = mdl;
02552     proc = prc;
02553     opt = opts;
02554     nclass = ncls;
02555
02556     nNodes = 0;
02557     nEdges = 0;
02558     nExtern = 0;
02559     tCouple = 0;
02560     loop = 0;
02561     mgraph = NULL;
02562     egraph = NULL;
02563     astack = NULL;

```

```

02563
02564     mgrcount = 0;
02565     agrcount = 0;
02566     extperm = 1;
02567
02568     // the results of the graph generation
02569     nMGraphs = 0; // the number of generated M-graphs
02570     nMOPI = 0; // the number of lPI M-graphs
02571     wMGraphs.setValue(0,1); // the weighted sum of M-graphs
02572     wMOPI.setValue(0,1); // the weighted sum of lPI M-graphs
02573
02574     nAGraphs = 0; // the number of generated A-graphs
02575     nAOPI = 0; // the number of lPI A-graphs
02576     wAGraphs.setValue(0,1); // the weighted sum of A-graphs
02577     wAOPI.setValue(0,1); // the weighted sum of lPI A-graphs
02578
02579     // check input
02580     nNodes = 0;
02581     nExtern = 0;
02582
02583     ndeg = 0;
02584     nvrt = 0;
02585     ok = True;
02586
02587     tcpl0 = 0;
02588     for (j = 0; j < model->ncouple; j++) {
02589         clist[j] = clst[j];
02590         tcpl0 += clist[j];
02591     }
02592     for (j = 0; j < nclass; j++) {
02593         cp = cpl[j];
02594         if (cp > 0) {
02595             lp2 = cpl[j] - cdeg[j] + 2;
02596             if (lp2 % 2 != 0 || lp2 < 0) {
02597                 if (prlevel > 0) {
02598                     fprintf(GRCC_Stderr, "*** SProcess::SProcess: illegal loop:"
02599                             "class %d, cp=%d, deg=%d, lp2=%d\n",
02600                             j, cp, cdeg[j], lp2);
02601                 }
02602                 ok = False;
02603             }
02604         }
02605         tCouple += cp*cnum[j];
02606         ndeg += cdeg[j]*cnum[j];
02607
02608         if (!isATEXternal(ctyp[j])) {
02609             nvrt += cnum[j];
02610         } else if (isATEXternal(ctyp[j])) {
02611             if (model->normalParticle(ptcl[j]) == 0) {
02612                 if (prlevel > 0) {
02613                     fprintf(GRCC_Stderr, "*** SProcess::SProcess: ");
02614                     fprintf(GRCC_Stderr, "illegal particle code: ");
02615                     fprintf(GRCC_Stderr, "code: class %d, ptcl=%d\n", j, ptcl[j]);
02616                 }
02617                 ok = False;
02618             } else {
02619                 nExtern += cnum[j];
02620             }
02621             extperm *= factorial(cnum[j]);
02622         } else {
02623             if (prlevel > 0) {
02624                 fprintf(GRCC_Stderr, "*** SProcess::SProcess: illegal type: ");
02625                 fprintf(GRCC_Stderr, "class %d, type=%d\n", j, ctyp[j]);
02626             }
02627             ok = False;
02628         }
02629     }
02630
02631     if (tcpl0 != tCouple) {
02632         if (prlevel > 0) {
02633             fprintf(GRCC_Stderr, "*** SProcess::SProcess: ");
02634             fprintf(GRCC_Stderr, "illegal coupling constants:");
02635             fprintf(GRCC_Stderr, " %d != %d\n", tcpl0, tCouple);
02636         }
02637         ok = False;
02638     }
02639     if (ndeg == nExtern) {
02640         if (prlevel > 0) {
02641             fprintf(GRCC_Stderr, "*** SProcess::SProcess: ");
02642             fprintf(GRCC_Stderr, "no vertices: ");
02643             fprintf(GRCC_Stderr, "nExtern=%d, ndeg=%d\n", nExtern, ndeg);
02644         }
02645         ok = False;
02646     }
02647     if (ndeg % 2 == 0) {
02648         nEdges = ndeg/2;
02649     } else {

```

```

02650         if (prlevel > 0) {
02651             fprintf(GRCC_Stderr, "*** SProcess::SProcess: ");
02652             fprintf(GRCC_Stderr, "illegal total deg = %d (not even)\n", ndeg);
02653         }
02654         ok = False;
02655     }
02656     nNodes = nvrt + nExtern;
02657     if (nNodes >= GRCC_MAXNODES) {
02658         if (prlevel > 0) {
02659             fprintf(GRCC_Stderr, "*** SProcess::SProcess: ");
02660             fprintf(GRCC_Stderr, "too many nodes = %d\n", nNodes);
02661             fprintf(GRCC_Stderr, "    nExtern=%d, nvrt=%d (GRCC_MAXNODES)\n",
02662                 nExtern, nvrt);
02663         }
02664         ok = False;
02665     }
02666     if (!ok) {
02667         prSProcess();
02668         erEnd("SProcess::SProcess: illegal input");
02669     }
02670     if (proc == NULL) {
02671         opt->beginProc(NULL);
02672     }
02673
02674     lp2 = tCouple - nExtern + 2;
02675     if (lp2 % 2 != 0 || lp2 < 0) {
02676         if (prlevel > 0) {
02677             fprintf(GRCC_Stderr, "*** SProcess::SProcess: illegal loop: "
02678                 "tCouple=%d, nExtern=%d, 2*loop=%d\n",
02679                 tCouple, nExtern, lp2);
02680         }
02681         ok = False;
02682     }
02683     loop = lp2/2;
02684
02685     // stack for assignment
02686     if (opt->values[GRCC_OPT_Step] == GRCC_AGraph) {
02687         astack = new AStack(GRCC_MAXNSTACK, GRCC_MAXESTACK);
02688     }
02689
02690     // save to PNodeClass object
02691     pnclass = new PNodeClass(this, nNodes, nclass, cdeg, ctyp, ptcl, cpl, cnum, cmind, cmaxd);
02692 }
02693
02694 //-----
02695 SProcess::SProcess(Model *mdl, Process *prc, Options *opts, int sid, int *clst, int ncls, NCInput
    *cls)
02696 {
02697     // Construct a Subprocess object
02698     //   mdl          : model
02699     //   prc          : process (may be NULL)
02700     //   opts         : options
02701     //   sid          : id of the sprocess
02702     //   ncls         : the number of classes
02703     //   cls          : input data of classes
02704
02705     int j, cp, lp2, ndeg, nvrt, tcpl0;
02706     bool ok;
02707
02708     if (ncls < 1) {
02709         fprintf(GRCC_Stderr, "*** SProcess::SProcess: no class %d\n", ncls);
02710         erEnd("SProcess::SProcess: no Class");
02711     }
02712     id      = sid;
02713     model   = mdl;
02714     proc    = prc;
02715     opt     = opts;
02716     nclass  = ncls;
02717
02718     nNodes  = 0;
02719     nEdges  = 0;
02720     nExtern = 0;
02721     tCouple = 0;
02722     loop    = 0;
02723     mgraph  = NULL;
02724     egraph  = NULL;
02725     astack  = NULL;
02726
02727     mgrcount = 0;
02728     agrcount = 0;
02729     extperm  = 1;
02730
02731     // the results of the graph generation
02732     nMGraphs = 0; // the number of generated M-graphs
02733     nMOPI    = 0; // the number of lPI M-graphs
02734     wMGraphs.setValue(0,1); // the weighted sum of M-graphs
02735     wMOPI.setValue(0,1); // the weighted sum of lPI M-graphs

```



```

02736
02737     nAGraphs = 0; // the number of generated A-graphs
02738     nAOPI = 0; // the number of 1PI A-graphs
02739     wAGraphs.setValue(0,1); // the weighted sum of A-graphs
02740     wAOPI.setValue(0,1); // the weighted sum of 1PI A-graphs
02741
02742     // check input
02743     nNodes = 0;
02744     nExtern = 0;
02745
02746     ndeg = 0;
02747     nvrt = 0;
02748     ok = True;
02749
02750     tcpl0 = 0;
02751     for (j = 0; j < model->ncouple; j++) {
02752         clist[j] = clst[j];
02753         tcpl0 += clist[j];
02754     }
02755     for (j = 0; j < nclass; j++) {
02756         cp = cls[j].cple;
02757         if (cp > 0) {
02758             lp2 = cls[j].cple - cls[j].cldeg + 2;
02759             if (lp2 % 2 != 0 || lp2 < 0) {
02760                 if (prlevel > 0) {
02761                     fprintf(GRCC_Stderr, "*** SProcess::SProcess: illegal loop: "
02762                             "class %d, cp=%d, deg=%d, lp2=%d\n",
02763                             j, cp, cls[j].cldeg, lp2);
02764                 }
02765                 ok = False;
02766             }
02767         }
02768         tCouple += cp*cls[j].clnum;
02769         ndeg += cls[j].cldeg*cls[j].clnum;
02770
02771         if (!isATEXternal(cls[j].cltyp)) {
02772             nvrt += cls[j].clnum;
02773         } else if (isATEXternal(cls[j].cltyp)) {
02774             if (model->normalParticle(cls[j].ptcl) == 0) {
02775                 if (prlevel > 0) {
02776                     fprintf(GRCC_Stderr, "*** SProcess::SProcess: ");
02777                     fprintf(GRCC_Stderr, "illegal particle code: ");
02778                     fprintf(GRCC_Stderr, "code: class %d, ptcl=%d\n", j, cls[j].ptcl);
02779                 }
02780                 ok = False;
02781             } else {
02782                 nExtern += cls[j].clnum;
02783             }
02784             extperm *= factorial(cls[j].clnum);
02785         } else {
02786             if (prlevel > 0) {
02787                 fprintf(GRCC_Stderr, "*** SProcess::SProcess: illegal type: ");
02788                 fprintf(GRCC_Stderr, "class %d, type=%d\n", j, cls[j].cltyp);
02789             }
02790             ok = False;
02791         }
02792     }
02793 }
02794 if (tcpl0 != tCouple) {
02795     if (prlevel > 0) {
02796         fprintf(GRCC_Stderr, "*** SProcess::SProcess: ");
02797         fprintf(GRCC_Stderr, "illegal coupling constants:");
02798         fprintf(GRCC_Stderr, " %d != %d\n", tcpl0, tCouple);
02799     }
02800     ok = False;
02801 }
02802 if (ndeg == nExtern) {
02803     if (prlevel > 0) {
02804         fprintf(GRCC_Stderr, "*** SProcess::SProcess: ");
02805         fprintf(GRCC_Stderr, "no vertices: ");
02806         fprintf(GRCC_Stderr, "nExtern=%d, ndeg=%d\n", nExtern, ndeg);
02807     }
02808     ok = False;
02809 }
02810 if (ndeg % 2 == 0) {
02811     nEdges = ndeg/2;
02812 } else {
02813     if (prlevel > 0) {
02814         fprintf(GRCC_Stderr, "*** SProcess::SProcess: ");
02815         fprintf(GRCC_Stderr, "illegal total deg = %d (not even)\n", ndeg);
02816     }
02817     ok = False;
02818 }
02819 nNodes = nvrt + nExtern;
02820 if (nNodes >= GRCC_MAXNODES) {
02821     if (prlevel > 0) {
02822         fprintf(GRCC_Stderr, "*** SProcess::SProcess: ");

```

```

02823         fprintf(GRCC_Stderr, "too many nodes = %d\n", nNodes);
02824         fprintf(GRCC_Stderr, "      nExtern=%d, nvert=%d (GRCC_MAXNODES)\n",
02825             nExtern, nvert);
02826     }
02827     ok = False;
02828 }
02829 if (!ok) {
02830     erEnd("SProcess::SProcess: illegal input");
02831 }
02832 if (proc == NULL) {
02833     opt->beginProc(NULL);
02834 }
02835
02836 lp2 = tCouple - nExtern + 2;
02837 if (lp2 % 2 != 0 || lp2 < 0) {
02838     if (prlevel > 0) {
02839         fprintf(GRCC_Stderr, "*** SProcess::SProcess: illegal loop: "
02840             "tCouple=%d, nExtern=%d, 2*loop=%d\n",
02841             tCouple, nExtern, lp2);
02842     }
02843     ok = False;
02844 }
02845 loop = lp2/2;
02846
02847 // stack for assignment
02848 if (opt->values[GRCC_OPT_Step] == GRCC_AGraph) {
02849     astack = new AStack(GRCC_MAXNSTACK, GRCC_MAXESTACK);
02850 }
02851
02852 // save to PNodeClass object
02853 pnclass = new PNodeClass(this, nNodes, nclass, cls);
02854 }
02855
02856 //-----
02857 SProcess::~SProcess(void)
02858 {
02859     delete pnclass;
02860     delete astack;
02861     delete mgraph;
02862 }
02863
02864 //-----
02865 void SProcess::prSProcess(void)
02866 {
02867     printf("\n");
02868     printf("+++ Subprocess %d, class=%d\n", id, nclass);
02869     if (pnclass != NULL) {
02870         pnclass->prPNodeClass();
02871     } else {
02872         printf(" pnclass = NULL\n");
02873     }
02874 }
02875
02876 //-----
02877 BigInt SProcess::generate(void)
02878 {
02879     // Entry point of Feynman graph generation
02880     int cldeg[GRCC_MAXNODES], clnum[GRCC_MAXNODES], cltyp[GRCC_MAXNODES];
02881     int cmind[GRCC_MAXNODES], cmamd[GRCC_MAXNODES];
02882     int ncl;
02883     BigInt ng;
02884
02885     ncl = toMNodeClass(cltyp, cldeg, clnum, cmind, cmamd);
02886
02887     opt->beginSubProc(this);
02888
02889     mgraph = new MGraph(id, ncl, cldeg, clnum, cltyp, cmind, cmamd, opt);
02890
02891     ng = mgraph->generate();
02892
02893     opt->endSubProc();
02894
02895     delete mgraph;
02896     mgraph = NULL;
02897
02898     return ng;
02899 }
02900
02901 //-----
02902 int SProcess::toMNodeClass(int *cltyp, int *cldeg, int *clnum, int *cmind, int *cmamd)
02903 {
02904     int j;
02905
02906     for (j = 0; j < nclass; j++) {
02907         if (isATEXternal(pnclass->type[j])) {
02908             cltyp[j] = -1;
02909         } else {

```

```

02910         cltyp[j] = (pnclass->couple[j]-pnclass->deg[j]+2)/2;
02911     }
02912     cldeg[j] = pnclass->deg[j];
02913     clnum[j] = pnclass->count[j];
02914     cmind[j] = pnclass->cminddeg[j];
02915     cmaxd[j] = pnclass->cmaxdeg[j];
02916 }
02917 return nclass;
02918 }
02919
02920 //-----
02921 void SProcess::assign(MGraph *mgr)
02922 {
02923     PNodeClass *pnc;
02924
02925     pnc = match(mgr);
02926
02927     agraph = new Assign(this, mgr, pnc);
02928
02929 #ifdef CHECK
02930     agraph->checkAG("SProcess::assign");
02931 #endif
02932     delete pnc;
02933     delete agraph;
02934     agraph = NULL;
02935 }
02936
02937 //-----
02938 PNodeClass *SProcess::match(MGraph *mgr)
02939 {
02940     PNodeClass *pnc = NULL;
02941
02942     int typ[GRCC_MAXNODES], dgs[GRCC_MAXNODES];
02943     int cnt[GRCC_MAXNODES], ptcl[GRCC_MAXNODES];
02944     int cpl[GRCC_MAXNODES];
02945     int n2m[GRCC_MAXNODES];
02946     int cmind[GRCC_MAXNODES], cmaxd[GRCC_MAXNODES];
02947     int j, k, l, nd, mc, mcl, mclss;
02948
02949     if (mgr->nNodes != nNodes) {
02950         if (prlevel > 0) {
02951             fprintf(GRCC_Stderr, "*** SProcess::match: different nNodes=%d != %d\n",
02952                 nNodes, mgr->nNodes);
02953         }
02954         erEnd("SProcess::match: different nNodes");
02955     }
02956
02957     mcl = toMNodeClass(typ, dgs, cnt, cmind, cmaxd);
02958
02959     nd = 0;
02960     for (j = 0; j < mcl; j++) {
02961         for (k = 0; k < cnt[j]; k++, nd++) {
02962             n2m[nd] = j;
02963         }
02964     }
02965
02966     nd = 0;
02967     mclss = mgr->curcl->nClasses;
02968     for (j = 0; j < mclss; j++) {
02969         for (k = 0; k < mgr->curcl->clist[j]; k++, nd++) {
02970             mc = mgr->nodes[nd]->clss;
02971             if (mc != n2m[nd]) {
02972                 if (prlevel > 0) {
02973                     fprintf(GRCC_Stderr, "*** SProcess::match: ");
02974                     fprintf(GRCC_Stderr, "inconsistent class\n");
02975                     for (l = 0; l < nNodes; l++) {
02976                         fprintf(GRCC_Stderr, "    %d: %d, %d\n",
02977                             j, mgr->nodes[l]->clss, n2m[l]);
02978                     }
02979                 }
02980                 erEnd("SProcess::match: inconsistent class");
02981             }
02982         }
02983     }
02984     for (j = 0; j < mcl; j++) {
02985         dgs[j] = pnclass->deg[j];
02986         typ[j] = pnclass->type[j];
02987         ptcl[j] = pnclass->particle[j];
02988         cnt[j] = pnclass->count[j];
02989         cpl[j] = pnclass->couple[j];
02990         cmind[j] = pnclass->cminddeg[j];
02991         cmaxd[j] = pnclass->cmaxdeg[j];
02992     }
02993     pnc = new PNodeClass(this, nNodes, mcl, dgs, typ, ptcl, cpl, cnt, cmind, cmaxd);
02994
02995     return pnc;
02996 }

```

```

02997
02998 //-----
02999 void SProcess::resultMGraph(BigInt nmgraphs, Fraction mwsum, BigInt nmopi, Fraction mwopi)
03000 {
03001     nMGraphs += nmgraphs;
03002     nMOPI    += nmopi;
03003     wMGraphs.add(mwsum);
03004     wMOPI.add(mwopi);
03005 }
03006
03007 //-----
03008 void SProcess::resultAGraph(BigInt nmgraphs, Fraction mwsum, BigInt nmopi, Fraction mwopi)
03009 {
03010     nAGraphs += nmgraphs;
03011     nAOPI    += nmopi;
03012     wAGraphs.add(mwsum);
03013     wAOPI.add(mwopi);
03014 }
03015
03016 //-----
03017 void SProcess::endMGraph(MGraph *mgr)
03018 {
03019     egraph = mgr->egraph;
03020
03021     if (opt->values[GRCC_OPT_Step] != GRCC_MGraph) {
03022         assign(mgr);
03023     }
03024 }
03025
03026 //-----
03027 void SProcess::endAGraph(EGraph *egr)
03028 {
03029     egraph = egr;
03030 }
03031
03032
03033 //=====
03034 // class Process
03035 //-----
03036 Process::Process(int pid, Model *modl, Options *optn, int nini, int *intlPrt, int nfin, int *finlPrt,
03037 int *coupling)
03038 {
03039     // Define a process and construct a set of sprocesses
03040     // model      : Model object
03041     // opt        : Option object
03042     // intlPart   : list of particle-id of intlPart particles
03043     // finalPart  : list of particle-id of final particles
03044     // coupling   : list of coupling constants
03045
03046     int j, lp2;
03047     Bool ok;
03048
03049     id      = pid;
03050     model   = modl;
03051     opt     = optn;
03052     ninitl  = nini;
03053     nfinal  = nfin;
03054     initlPart = intdup(ninitl, intlPrt);
03055     finalPart = intdup(nfinal, finlPrt);
03056
03057     // count total number of coupling constants.
03058     cttotal = 0;
03059     for (j = 0; j < GRCC_MAXNCPLG; j++) {
03060         if (j < model->ncouple) {
03061             clist[j] = coupling[j];
03062             cttotal += coupling[j];
03063         } else {
03064             clist[j] = 0;
03065         }
03066     }
03067     if (cttotal < 1) {
03068         erEnd("Process: coupling constant = 0");
03069     }
03070
03071     nExtern = 0;
03072     loop    = -1;
03073     maxnlegs = 0;
03074     mgrcount = 0;
03075     agrcount = 0;
03076
03077     // table of sprocesses
03078     nSubproc = 0;
03079     sproc    = NULL;
03080
03081     // the results of the graph generation
03082     nMGraphs = 0; // the number of generated M-graphs
03083     nMOPI    = 0; // the number of 1PI M-graphs

```

```

03083     wMGraphs.setValue(0,1);           // the weighted sum of M-graphs
03084     wMOPI.setValue(0,1);              // the weighted sum of 1PI M-graphs
03085
03086     nAGraphs = 0;                      // the number of generated A-graphs
03087     nAOPI    = 0;                      // the number of 1PI A-graphs
03088     wAGraphs.setValue(0,1);           // the weighted sum of A-graphs
03089     wAOPI.setValue(0,1);              // the weighted sum of 1PI A-graphs
03090
03091     ok = True;
03092
03093     // numbers
03094     nExtern = ninitl + nfinal;
03095     maxnlegs = Max(nExtern, model->maxnlegs);
03096
03097     // count loops
03098     lp2 = ctotat - nExtern + 2;
03099     if (lp2 % 2 != 0) {
03100         if (prlevel > 0) {
03101             fprintf(GRCC_Stderr, "*** cannot generate : 2*loop is odd : "
03102                     "2*loop = %d, ctotat=%d, nExtern=%d\n",
03103                     lp2, ctotat, nExtern);
03104         }
03105         ok = False;
03106     } else if (lp2 < 0) {
03107         if (prlevel > 0) {
03108             fprintf(GRCC_Stderr, "*** cannot make a connected graph : "
03109                     "2*loop=%d, ctotat=%d, nExtern=%d\n",
03110                     lp2, ctotat, nExtern);
03111         }
03112         ok = False;
03113     }
03114
03115     loop = lp2 / 2;
03116
03117     if (!ok) {
03118         if (prlevel > 0) {
03119             fprintf(GRCC_Stderr, "*** Process: illegal input: lp2 = %d\n", lp2);
03120         }
03121         erEnd("Process: illegal input");
03122     }
03123
03124 #ifdef DEBUG
03125     // print message
03126     prProcess();
03127 #endif
03128
03129     // construct sprocesses
03130     mkSProcess();
03131 }
03132
03133 //-----
03134 Process::~Process(void)
03135 {
03136     initlPart = delintdup(initlPart);
03137     finalPart = delintdup(finalPart);
03138     delete sproc;
03139 }
03140
03141 //-----
03142 void Process::prProcess(void)
03143 {
03144     printf("+++ process options : OPI = %d, (Step) = %d, coupling=%d: ",
03145           opt->values[GRCC_OPT_1PI], opt->values[GRCC_OPT_Step], ctotat);
03146     prIntArray(model->ncouple, clist, "\n");
03147 }
03148
03149 //-----
03150 void Process::mkSProcess(void)
03151 {
03152     // Construct sprocesses
03153     // Each sprocess is a set of nodes whose degree and order of
03154     // coupling constants are determined.
03155     // Only the total coupling constants are considered here even if
03156     // two or more coupling constants are defined in the model
03157
03158     NCInput cls[GRCC_MAXMINTERACT];
03159     int r, j, k, lin2, nvtx, nleg, nc, n0;
03160     int nl[GRCC_MAXMINTERACT];
03161     double proct0 = 0, proct1 = 0;
03162
03163     // output file to start process
03164     opt->beginProc(this);
03165
03166     // starting time
03167     Second(&proct0);
03168
03169     // generate all possible number of (nl[i]),

```

```

03170 // \sum_i nl[i]*cplgcp[i] = (total number of coupling constants)
03171
03172 r = -1;
03173 nSubproc = 0;
03174 ngraphs = 0;
03175 nopi = 0;
03176 wgraphs = 0;
03177 wopi = 0;
03178
03179 // for partitions of (leg, order)
03180
03181 while (nextPart(ctotal, model->ncplgcp, model->cplgcp, nl, &r)) {
03182 #ifdef DEBUG
03183     printf("nextPart:ctotal=%d, nc=%d, c=", ctotal, model->ncplgcp);
03184     prIntArray(model->ncplgcp, model->cplgcp, "", nl="");
03185     prIntArray(model->ncplgcp, nl, "");
03186     printf("\n", r);
03187 #endif
03188     // count the total number of vertices and legs.
03189     nvtx = 0;
03190     nleg = 0;
03191     for (j = 0; j < model->ncplgcp; j++) {
03192         nvtx += nl[j];
03193         nleg += nl[j]*model->cplglg[j];
03194     }
03195
03196     // count loops
03197     lin2 = nExtern + nleg;
03198     if (lin2 % 2 != 0) {
03199         continue;
03200     }
03201     loop = lin2/2 - nExtern - nvtx + 1;
03202     if (loop < 0) {
03203         continue;
03204     }
03205
03206     nc = 0;
03207     if (opt->values[GRCC_OPT_SymmInitial]) {
03208         n0 = nc;
03209         for (j = 0; j < ninitl; j++) {
03210             for (k = n0; k < nc; k++) {
03211                 if (cls[k].ptcl == initlPart[j]) {
03212                     cls[k].clnum++;
03213                     break;
03214                 }
03215             }
03216             if (k >= nc) {
03217                 cls[nc].ptcl = initlPart[j];
03218                 cls[nc].clnum = 1;
03219                 cls[nc].cple = 0;
03220                 cls[nc].clnum = 1;
03221                 cls[nc].cldeg = 1;
03222                 cls[nc].cltyp = GRCC_AT_Initial;
03223                 cls[nc].ptcl = initlPart[j];
03224                 cls[nc].cmind
03225                     = model->particles[Abs(cls[nc].ptcl)]->cminddeg;
03226                 cls[nc].cmaxd
03227                     = model->particles[Abs(cls[nc].ptcl)]->cmaxdeg;
03228                 if (opt->values[GRCC_OPT_NoExtSelf] > 0 && nExtern > 2) {
03229                     cls[nc].cmind = Max(cls[nc].cmind, 3);
03230                 }
03231                 nc++;
03232             }
03233         }
03234     } else {
03235         for (j = 0; j < ninitl; j++) {
03236             cls[nc].ptcl = initlPart[j];
03237             cls[nc].clnum = 1;
03238             cls[nc].cple = 0;
03239             cls[nc].cldeg = 1;
03240             cls[nc].cltyp = GRCC_AT_Initial;
03241             cls[nc].cmind = model->particles[Abs(cls[nc].ptcl)]->cminddeg;
03242             cls[nc].cmaxd = model->particles[Abs(cls[nc].ptcl)]->cmaxdeg;
03243             if (opt->values[GRCC_OPT_NoExtSelf] > 0 && nExtern > 2) {
03244                 cls[nc].cmind = Max(cls[nc].cmind, 3);
03245             }
03246             nc++;
03247         }
03248     }
03249     if (opt->values[GRCC_OPT_SymmFinal]) {
03250         n0 = nc;
03251         for (j = 0; j < nfinal; j++) {
03252             for (k = n0; k < nc; k++) {
03253                 if (cls[k].ptcl == model->antiParticle(finalPart[j])) {
03254                     cls[k].clnum++;
03255                     break;
03256                 }

```

```

03257         }
03258         if (k >= nc) {
03259             cls[nc].ptcl = model->antiParticle(finalPart[j]);
03260             cls[nc].clnum = 1;
03261             cls[nc].cple = 0;
03262             cls[nc].cldeg = 1;
03263             cls[nc].cltyp = GRCC_AT_Final;
03264             cls[nc].cmind
03265                 = model->particles[Abs(cls[nc].ptcl)]->cminddeg;
03266             cls[nc].cmaxd
03267                 = model->particles[Abs(cls[nc].ptcl)]->cmaxdeg;
03268             if (opt->values[GRCC_OPT_NoExtSelf] > 0 && nExtern > 2) {
03269                 cls[nc].cmind = Max(cls[nc].cmind, 3);
03270             }
03271             nc++;
03272         }
03273     }
03274 } else {
03275     for (j = 0; j < nfinal; j++) {
03276         cls[nc].ptcl = model->antiParticle(finalPart[j]);
03277         cls[nc].cldeg = 1;
03278         cls[nc].cple = 0;
03279         cls[nc].clnum = 1;
03280         cls[nc].cltyp = GRCC_AT_Final;
03281         cls[nc].cmind = model->particles[Abs(cls[nc].ptcl)]->cminddeg;
03282         cls[nc].cmaxd = model->particles[Abs(cls[nc].ptcl)]->cmaxdeg;
03283         if (opt->values[GRCC_OPT_NoExtSelf] > 0 && nExtern > 2) {
03284             cls[nc].cmind = Max(cls[nc].cmind, 3);
03285         }
03286         nc++;
03287     }
03288 }
03289 for (j = 0; j < model->ncplgcp; j++) {
03290     if (nl[j] > 0) {
03291         cls[nc].ptcl = 0;
03292         cls[nc].cple = model->cplgcp[j];
03293         cls[nc].cldeg = model->cplglg[j];
03294         cls[nc].clnum = nl[j];
03295         // number of loops
03296         cls[nc].cltyp = (model->cplgcp[j] - model->cplglg[j] + 2)/2;
03297         cls[nc].cmind = 0;
03298         cls[nc].cmaxd = 0;
03299         nc++;
03300     }
03301 }
03302 #ifdef DEBUG1
03303     for (j = 0; j < nc; j++) {
03304         printf("%d/%d: deg=%d, typ=%d, ptcl=%d, cple=%d, num=%d\n",
03305             j, nc, cls[j].cldeg, cls[j].cltyp, cls[j].ptcl,
03306             cls[j].cple, cls[j].clnum);
03307     }
03308 #endif
03309 // create a sprocess ??? to be rewritten
03310 sproc = new SProcess(model, this, opt, nSubproc, clist, nc, cls);
03311
03312 // list of sprocesses
03313 if (nSubproc >= GRCC_MAXSUBPROCS) {
03314     erEnd("Subclass: too many sprocesses (GRCC_MAXSUBPROCS)");
03315 }
03316 sptbl[nSubproc++] = sproc;
03317
03318 #ifdef DEBUG
03319     // print sprocesses
03320     sproc->prSProcess();
03321 #endif
03322
03323 // generate M-graphs
03324 sproc->generate();
03325
03326 // summation of the numbers and weighted numbers of graphs
03327 nMGraphs += sproc->nMGraphs;
03328 nMOPI += sproc->nMOPI;
03329 wMGraphs.add(sproc->wMGraphs);
03330 wMOPI.add(sproc->wMOPI);
03331
03332 nAGraphs += sproc->nAGraphs;
03333 nAOPI += sproc->nAOPI;
03334 wAGraphs.add(sproc->wAGraphs);
03335 wAOPI.add(sproc->wAOPI);
03336
03337 if (nAGraphs > 0) {
03338     ngraphs = nAGraphs;
03339 } else {
03340     ngraphs = nAGraphs;
03341 }
03342 delete sproc;
03343 sproc = NULL;

```

```

03344     }
03345
03346     // ending time
03347     Second(&proct1);
03348     sec = proct1-proct0;
03349
03350     // output file to finish process
03351     opt->endProc();
03352 }
03353
03354 //*****
03355 // mgraph.cc
03356 //=====
03357 //-----
03358 MNodeClass::MNodeClass(int nnodes, int nclasses)
03359 {
03360     // Input
03361     // nnodes : possible maximal number of nodes
03362     // nclasses : possible maximal number of classes
03363
03364     int j, k;
03365
03366     nNodes = nnodes;
03367     nClasses = nclasses;
03368     maxdeg = 0;
03369     for (j = 0; j < nNodes; j++) {
03370         for (k = 0; k < nNodes; k++) {
03371             clmat[j][k] = 0;
03372         }
03373         clist[j] = 0;
03374         ndcl[j] = 0;
03375         flist[j] = 0;
03376         clord[j] = 0;
03377         cmindeg[j] = 0;
03378         cmaxdeg[j] = 0;
03379     }
03380     flist[nNodes] = 0;
03381     flg0 = 0;
03382     flg1 = 0;
03383     flg2 = 0;
03384 }
03385
03386 //-----
03387 MNodeClass::~MNodeClass(void)
03388 {
03389 }
03390
03391 //=====
03392 // class MNodeClass
03393 //-----
03394 void MNodeClass::init(int *cl, int mxdeg, int **adjmat)
03395 {
03396     // initialize the object
03397     // Argument
03398     // cl[j] : list of number of nodes in the j-th class
03399     // mxdeg : maximum degree to nodes
03400     // adjmat[j][k] : adjacency matrix
03401
03402     int j;
03403
03404     for (j = 0; j < nClasses; j++) {
03405         clist[j] = cl[j];
03406         clord[j] = j;
03407     }
03408     maxdeg = mxdeg;
03409     mkNdCl();
03410     mkClMat(adjmat);
03411     mkFlist();
03412     flg0 = 0;
03413     flg1 = 0;
03414     flg2 = 0;
03415 }
03416
03417 //-----
03418 void MNodeClass::copy(MNodeClass* mnc)
03419 {
03420     // copy MNodeClass 'mnc' to this object
03421
03422     int j, k;
03423
03424     nClasses = mnc->nClasses;
03425     for (k = 0; k < nClasses; k++) {
03426         clist[k] = mnc->clist[k];
03427     }
03428     maxdeg = mnc->maxdeg;
03429     for (j = 0; j < nNodes; j++) {
03430         ndcl[j] = mnc->ndcl[j];

```



```

03431         clord[j] = mnc->clord[j];
03432         for (k = 0; k < nClasses; k++) {
03433             clmat[j][k] = mnc->clmat[j][k];
03434         }
03435     }
03436     for (k = 0; k < nClasses+1; k++) {
03437         flist[k] = mnc->flist[k];
03438     }
03439 }
03440
03441 //-----
03442 void MNodeClass::mkFlist(void)
03443 {
03444     // Construct flist
03445     // The set of nodes in class 'c' is [flist[c],...,flist[c+1]-1]
03446
03447     int j, f;
03448
03449     f = 0;
03450     for (j = 0; j < nClasses; j++) {
03451         flist[j] = f;
03452         f += clist[j];
03453     }
03454     flist[nClasses] = f;
03455 }
03456
03457 //-----
03458 void MNodeClass::mkNdCl(void)
03459 {
03460     // Construct ndcl
03461     // ndcl[nd] = (the class id in which node 'nd' belongs)
03462
03463     int c, k;
03464     int nd = 0;
03465
03466     for (c = 0; c < nClasses; c++) {
03467         for (k = 0; k < clist[c]; k++) {
03468             ndcl[nd++] = c;
03469         }
03470     }
03471 }
03472
03473 //-----
03474 int MNodeClass::clCmp(int nd0, int nd1, int cn)
03475 {
03476     // Comparison of two nodes 'nd0' and 'nd1'
03477     // in lexicographic ordering of [class, connection configuration]
03478
03479     int cmp;
03480     // Whether two nodes are in a same class or not.
03481
03482     cmp = ndcl[nd0] - ndcl[nd1];
03483     if (cmp != 0) {
03484         return cmp;
03485     }
03486
03487     // Sign '-' signifies the reverse ordering
03488     cmp = - cmpMNCArray(clmat[nd0], clmat[nd1], cn);
03489     if (cmp != 0) {
03490         return cmp;
03491     }
03492     return cmp;
03493 }
03494
03495 //-----
03496 void MNodeClass::mkClMat(int **adjmat)
03497 {
03498     // Construct a matrix 'clmat[nd][tc]' which is the number
03499     // of edges connecting 'nd' and all nodes in class 'tc'.
03500
03501     int nd, td, tc;
03502
03503     for (nd = 0; nd < nNodes; nd++) {
03504         for (td = 0; td < nNodes; td++) {
03505             tc = ndcl[td]; // another node
03506             if (nd == tc) {
03507                 clmat[nd][tc] += CLWIGHTD(adjmat[nd][td]);
03508             } else {
03509                 clmat[nd][tc] += CLWIGHTO(adjmat[nd][td]);
03510             }
03511         }
03512     }
03513 }
03514
03515 //-----
03516 void MNodeClass::incMat(int nd, int td, int val)
03517 {

```

```

03518 // Increase the number of edges by 'val' between 'nd' and 'td'.
03519
03520 int tdc = ndcl[td]; // class of 'td'
03521 clmat[nd][tdc] += val; // modify matrix 'clmat'.
03522 }
03523
03524 //-----
03525 void MNodeClass::printMat(void)
03526 {
03527 // Print configuration matrix.
03528
03529 int j1, j2;
03530
03531 printf("\n");
03532
03533 printf("nClasses=%d", nClasses);
03534 printf(" clord="); printArray(nClasses, clord, "");
03535 printf(" flist="); printArray(nClasses, flist, "\n");
03536 printf("flg = (%d, %d, %d)\n", flg0, flg1, flg2);
03537
03538 // the first line
03539 printf("nd: cl: ");
03540 for (j2 = 0; j2 < nClasses; j2++) {
03541 printf("%2d ", j2);
03542 }
03543 printf("\n");
03544
03545 // print raw
03546 for (j1 = 0; j1 < nNodes; j1++) {
03547 printf("%2d; %2d: [", j1, ndcl[j1]);
03548 for (j2 = 0; j2 < nClasses; j2++) {
03549 printf(" %2d", clmat[j1][j2]);
03550 }
03551 printf("]\n");
03552 }
03553 }
03554
03555 //-----
03556 int MNodeClass::cmpMNCArray(int *a0, int *a1, int ma)
03557 {
03558 int j, k;
03559
03560 for (k = 0; k < ma; k++) {
03561 j = clord[k];
03562 if (a0[j] < a1[j]) {
03563 return -1;
03564 } else if (a0[j] > a1[j]) {
03565 return 1;
03566 }
03567 }
03568 return 0;
03569 }
03570
03571 //-----
03572 void MNodeClass::reorder(MGraph *mg)
03573 {
03574 int flg[GRCC_MAXNODES];
03575 int co, cn;
03576 int f0, f1, f2;
03577
03578 f0 = 0;
03579 f1 = 0;
03580 f2 = 0;
03581 for (co = 0; co < nClasses; co++) {
03582 cn = flist[co];
03583 if (mg->nodes[cn]->freelg < 1) {
03584 flg[co] = 0;
03585 f0++;
03586 } else if (mg->nodes[cn]->freelg < mg->nodes[cn]->deg) {
03587 flg[co] = mg->nodes[cn]->deg + maxdeg*clist[co];
03588 f1++;
03589 } else {
03590 flg[co] = maxdeg*(mg->nodes[cn]->deg + maxdeg*clist[co]);
03591 f2++;
03592 }
03593 }
03594 if (f0 < nClasses) {
03595 bsort(nClasses, clord, flg);
03596 }
03597 flg0 = f0;
03598 flg1 = f0 + f1;
03599 flg2 = f0 + f1 + f2;
03600 }
03601
03602 //=====
03603 // class MNode : nodes in MGraph
03604 //-----

```

```

03605 MNode::MNode(int vid, int vclss, NCInput *mgi)
03606 {
03607     // Arguments
03608     //   vid   : identifier of the vertex
03609     //   vdeg   : degree of the node
03610     //   vextlp : external node (-1) or looped vertex
03611     //   vclss  : class of the node
03612
03613     id       = vid;           // id of the node
03614     clss     = vclss;         // class to which the node belongs
03615     deg      = mgi->cldeg;     // degree of the node
03616     freelg   = mgi->cldeg;     // number of free legs
03617     extloop  = mgi->cltyp;     // external node or not
03618     cmindeg  = mgi->cmindeg;    // min(deg of connectable vertex)
03619     cmaxdeg  = mgi->cmaxdeg;    // max(deg of connectable vertex)
03620 #ifdef DEBUG3
03621     printf("MNode:id=%d, freelg=%d, cmindeg=%d, cmaxdeg=%d\n",
03622           id, freelg, cmindeg, cmaxdeg);
03623 #endif
03624 }
03625 //-----
03626 MNode::MNode(int vid, int vdeg, int vextlp, int vclss, int cmin, int cmax)
03627 {
03628     // Arguments
03629     //   vid   : identifier of the vertex
03630     //   vdeg   : degree of the node
03631     //   vextlp : external node (-1) or looped vertex
03632     //   vclss  : class of the node
03633
03634     id       = vid;           // id of the node
03635     deg      = vdeg;           // degree of the node
03636     freelg   = vdeg;           // number of free legs
03637     clss     = vclss;         // class to which the node belongs
03638     extloop  = vextlp;        // external node or not
03639     cmindeg  = cmin;           // min(deg of connectable vertex)
03640     cmaxdeg  = cmax;           // max(deg of connectable vertex)
03641 #ifdef DEBUG3
03642     printf("MNode:id=%d, freelg=%d, cmindeg=%d, cmaxdeg=%d\n", id, freelg, cmindeg, cmaxdeg);
03643 #endif
03644 }
03645
03646 //=====
03647 // class MGraph : scalar graph expressed by matrix form
03648 //-----
03649 MGraph::MGraph(int pid, int ncl, int *cldeg, int *clnum, int *cltyp, int *cmindeg, int *cmaxdeg, Options
*opts)
03650 {
03651     int nn, ne, j, k;
03652
03653 #ifdef DEBUGM
03654     printf("MGraph::MGraph(pid=%d, ncl=%d)\n", pid, ncl);
03655     for (j = 0; j < ncl; j++){
03656         printf("%d: deg=%d, num=%d, typ=%d, mind=%d, maxd=%d\n",
03657               j, cldeg[j], clnum[j], cltyp[j], cmindeg[j], cmaxdeg[j]);
03658     }
03659     opts->print();
03660 #endif
03661
03662     // initial conditions
03663     nClasses = ncl;
03664     clist     = new int[ncl];
03665     opt       = opts;
03666     mId       = -1;
03667
03668     mindeg    = -1;
03669     maxdeg    = -1;
03670     ne = 0;
03671     nn = 0;
03672     for (j = 0; j < nClasses; j++) {
03673         clist[j] = clnum[j];
03674         nn += clnum[j];
03675         ne += cldeg[j]*clnum[j];
03676         if (mindeg < 0) {
03677             mindeg = cldeg[j];
03678         } else {
03679             mindeg = Min(mindeg, cldeg[j]);
03680         }
03681         if (maxdeg < 0) {
03682             maxdeg = cldeg[j];
03683         } else {
03684             maxdeg = Max(maxdeg, cldeg[j]);
03685         }
03686     }
03687     if (ne % 2 != 0) {
03688         if (prlevel > 0) {
03689             fprintf(stderr, "Sum of degrees are not even\n");
03690             for (j = 0; j < nClasses; j++) {

```

```

03691             fprintf(GRCC_Stderr, "class %2d: %2d %2d %2d\n",
03692                       j, cldeg[j], clnum[j], cltyp[j]);
03693         }
03694     }
03695     erEnd("illegal degrees of nodes");
03696 }
03697 pId = pid;
03698 nNodes = nn;
03699 nodes = new MNode*[nNodes];
03700 group = new SGroup();
03701 #ifndef ORBITS
03702     orbits = new MORbits();
03703 #endif
03704
03705     nEdges = ne / 2;
03706     nLoops = nEdges - nNodes + 1;
03707
03708     egraph = new EGraph(nNodes, nEdges, maxdeg);
03709     nn = 0;
03710     nExtern = 0;
03711 #ifdef DEBUG3
03712     printf("MGraph::MGraph: nClasses=%d\n", nClasses);
03713 #endif
03714     for (j = 0; j < nClasses; j++) {
03715 #ifdef DEBUG3
03716         printf("cmind[%d]=%d, cmaxd[%d]=%d\n", j, cmind[j], j, cmaxd[j]);
03717 #endif
03718         for (k = 0; k < clist[j]; k++, nn++) {
03719             nodes[nn] = new MNode(nn, cldeg[j], cltyp[j], j, cmind[j], cmaxd[j]);
03720             egraph->setExtLoop(nn, cltyp[j]);
03721             if (cltyp[j] < 0) {
03722                 nExtern++;
03723             }
03724         }
03725     }
03726     egraph->endSetExtLoop();
03727
03728     init();
03729 }
03730
03731 //-----
03732 MGraph::MGraph(int pid, int ncl, NCInput *mgi, Options *opts)
03733 {
03734     int nn, ne, j, k;
03735
03736 #ifdef DEBUGM
03737     printf("MGraph::MGraph(pid=%d, ncl=%d)\n", pid, ncl);
03738     for (j = 0; j < ncl; j++){
03739         printf("%d: deg=%d, num=%d, typ=%d, mind=%d, maxd=%d\n",
03740               j, mgi[j].cldeg, mgi[j].clnum, mgi[j].cltyp,
03741               mgi[j].cmind, mgi[j].cmaxd);
03742     }
03743     opts->print();
03744 #endif
03745
03746     // initial conditions
03747     nClasses = ncl;
03748     clist = new int[ncl];
03749     opt = opts;
03750     mId = -1;
03751
03752     mindeg = -1;
03753     maxdeg = -1;
03754     ne = 0;
03755     nn = 0;
03756     for (j = 0; j < nClasses; j++) {
03757         clist[j] = mgi[j].clnum;
03758         nn += mgi[j].clnum;
03759         ne += mgi[j].cldeg*mgi[j].clnum;
03760         if (mindeg < 0) {
03761             mindeg = mgi[j].cldeg;
03762         } else {
03763             mindeg = Min(mindeg, mgi[j].cldeg);
03764         }
03765         if (maxdeg < 0) {
03766             maxdeg = mgi[j].cldeg;
03767         } else {
03768             maxdeg = Max(maxdeg, mgi[j].cldeg);
03769         }
03770     }
03771     if (ne % 2 != 0) {
03772         if (prlevel > 0) {
03773             fprintf(GRCC_Stderr, "Sum of degrees are not even\n");
03774             for (j = 0; j < nClasses; j++) {
03775                 fprintf(GRCC_Stderr, "class %2d: %2d %2d %2d\n",
03776                       j, mgi[j].cldeg, mgi[j].clnum, mgi[j].cltyp);
03777             }

```

```

03778     }
03779     erEnd("illegal degrees of nodes");
03780 }
03781 pId    = pid;
03782 nNodes = nn;
03783 nodes  = new MNode*[nNodes];
03784 group  = new SGroup();
03785 #ifdef ORBITS
03786 orbits = new MOrbits();
03787 #endif
03788
03789 nEdges = ne / 2;
03790 nLoops = nEdges - nNodes + 1;
03791
03792 egraph = new EGraph(nNodes, nEdges, maxdeg);
03793 nn = 0;
03794 nExtern = 0;
03795 #ifdef DEBUG3
03796 printf("MGraph::MGraph: nClasses=%d\n", nClasses);
03797 #endif
03798 for (j = 0; j < nClasses; j++) {
03799 #ifdef DEBUG3
03800     printf("cmind[%d]=%d, cmaxd[%d]=%d\n",
03801           j, mgi[j].cmind, j, mgi[j].cmaxd);
03802 #endif
03803     for (k = 0; k < clist[j]; k++, nn++) {
03804         nodes[nn] = new MNode(nn, j, mgi[j]);
03805         egraph->setExtLoop(nn, mgi[j].cltyp);
03806         if (mgi[j].cltyp < 0) {
03807             nExtern++;
03808         }
03809     }
03810 }
03811 egraph->endSetExtLoop();
03812
03813 init();
03814 }
03815
03816 //-----
03817 void MGraph::init(void)
03818 {
03819     // generated set of graphs
03820     cDiag      = 0;
03821     c1PI       = 0;
03822     cNoTadpole = 0;
03823     cNoTadBlock = 0;
03824     c1PINoTadBlock = 0;
03825
03826     // the current graph
03827     adjMat      = newMat(nNodes, nNodes, 0);
03828     nsym        = ToBigInt(0);
03829     esym        = ToBigInt(0);
03830     wscon       = Fraction(0, 1);
03831     wsopi       = Fraction(0, 1);
03832
03833     // current node classification
03834     curcl       = new MNodeClass(nNodes, nClasses);
03835
03836     // table of n-edge connected components
03837     mconn       = new MConn(nNodes, nEdges);
03838
03839     // measures of efficiency
03840     ngen        = 0;
03841     ngconn      = 0;
03842
03843     #ifdef MONITOR
03844         nCallRefine = 0;
03845         discardRefine = 0;
03846         discardDisc = 0;
03847         discardIso = 0;
03848     #endif
03849
03850     // work space for isIsomorphic
03851     modmat = newMat(nNodes, nNodes, 0);
03852
03853     // work space for bisearchM
03854     bidef = newArray(nNodes, 0);
03855     bilow = newArray(nNodes, 0);
03856     bicount = 0;
03857 }
03858
03859 //-----
03860 MGraph::~MGraph(void)
03861 {
03862     int j;
03863
03864     // group->delGroup();

```

```

03865
03866     bilow = deleteArray(bilow);
03867     bidef = deleteArray(bidef);
03868
03869
03870     modmat = deleteMat(modmat, nNodes);
03871     delete mconn;
03872     delete curcl;
03873     adjMat = deleteMat(adjMat, nNodes);
03874 #ifdef ORBITS
03875     delete orbits;
03876 #endif
03877     delete egraph;
03878     delete group;
03879     for (j = 0; j < nNodes; j++) {
03880         delete nodes[j];
03881         nodes[j] = NULL;
03882     }
03883     delete[] nodes;
03884     delete[] clist;
03885
03886 }
03887
03888 //-----
03889 void MGraph::printAdjMat(MNodeClass *cl)
03890 {
03891     int j1, j2;
03892
03893     printf(" ");
03894     for (j2 = 0; j2 < nNodes; j2++) {
03895         printf(" %2d", j2);
03896     }
03897     printf("\n");
03898     for (j1 = 0; j1 < nNodes; j1++) {
03899         printf("%2d : [", j1);
03900         for (j2 = 0; j2 < nNodes; j2++) {
03901             printf(" %2d", adjMat[j1][j2]);
03902         }
03903         printf("] %2d\n", cl->ndcl[j1]);
03904     }
03905 }
03906
03907 //-----
03908 void MGraph::print(void)
03909 {
03910     int j;
03911
03912     printf("MGraph: pId=%d, cDiag=%ld, cPI=%ld\n", pId, cDiag, cPI);
03913     printf("      nNodes=%d, nEdges=%d, nExtern=%d, nLoops=%d "
03914            "mindeg=%d, maxdeg=%d, sym=(%ld, %ld)\n",
03915            nNodes, nEdges, nExtern, nLoops, mindeg, maxdeg, nsym, esym);
03916     printf("      Nodes=%d\n", nNodes);
03917     for (j = 0; j < nNodes; j++) {
03918         printf("      %2d: id=%d, deg=%d, clss=%d, extloop=%d, ",
03919                j, nodes[j]->id, nodes[j]->deg, nodes[j]->clss,
03920                nodes[j]->extloop);
03921         printf("mind=%d, maxd=%d, freelg=%d\n",
03922                nodes[j]->cmindeg, nodes[j]->cmxdeg, nodes[j]->freelg);
03923     }
03924
03925     curcl->printMat();
03926     printf("\n");
03927
03928     printAdjMat(curcl);
03929 }
03930
03931 //-----
03932 Bool MGraph::isConnected(void)
03933 {
03934     // Check graph can be a connected one.
03935     // If a connected component without free leg is not the whole graph then
03936     // return False, otherwise return True.
03937
03938     int j, n, nv;
03939
03940     for (j = 0; j < nNodes; j++) {
03941         nodes[j]->visited = -1;
03942     }
03943     if (visit(0)) {
03944         return True;
03945     }
03946     nv = 0;
03947     for (n = 0; n < nNodes; n++) {
03948         if (nodes[n]->visited >= 0) {
03949             nv++;
03950         }
03951     }

```

```

03952     return (nv == nNodes);
03953 }
03954
03955 //-----
03956 Bool MGraph::visit(int nd)
03957 {
03958     // Visiting connected node used for 'isConnected'
03959     // If child nodes has free legs, then this function returns True.
03960     // otherwise it returns False.
03961
03962     int td;
03963
03964     // This node has free legs.
03965     if (nodes[nd]->freelg > 0) {
03966         return True;
03967     }
03968     nodes[nd]->visited = 0;
03969     for (td = 0; td < nNodes; td++) {
03970         if ((adjMat[nd][td] > 0) && (nodes[td]->visited < 0)) {
03971             if (visit(td)) {
03972                 return True;
03973             }
03974         }
03975     }
03976     // all the child nodes has no free legs.
03977     return False;
03978 }
03979
03980 //-----
03981 Bool MGraph::isIsomorphic(MNodeClass *cl)
03982 {
03983     // Check whether the current graph is the Representative
03984     // of a isomorphic class.
03985     // nsym = symmetry factor by the permutation of nodes.
03986     // esym = symmetry factor by the permutation of edge.
03987     // If this graph is not a representative, then returns False.
03988
03989     int j1, j2, cmp, nself;
03990     int *perm;
03991
03992     nsym = ToBigInt(0);
03993     esym = ToBigInt(1);
03994
03995     group->newGroup(nNodes, cl->nClasses, cl->clist);
03996
03997 #ifdef ORBITS
03998     orbits->initPerm(nNodes);
03999 #endif
04000
04001     while (True) {
04002         perm = group->genNext();
04003
04004         if (perm == NULL) {
04005             // calculate permutations of edges
04006             esym = ToBigInt(1);
04007             for (j1 = 0; j1 < nNodes; j1++) {
04008                 if (adjMat[j1][j1] > 0) {
04009                     nself = adjMat[j1][j1]/2;
04010                     esym *= factorial(nself);
04011                     esym *= ipow(2, nself);
04012                 }
04013                 for (j2 = j1+1; j2 < nNodes; j2++) {
04014                     if (adjMat[j1][j2] > 0) {
04015                         esym *= factorial(adjMat[j1][j2]);
04016                     }
04017                 }
04018             }
04019             return True;
04020         }
04021
04022         permMat(nNodes, perm, adjMat, modmat);
04023         cmp = compMat(nNodes, adjMat, modmat);
04024         if (cmp < 0) {
04025             return False;
04026         } else if (cmp == 0) {
04027             // if ngroup < 0, symmetry group is $S_{-group}$.
04028             group->addGroup(perm);
04029
04030             nsym = nsym + ToBigInt(1);
04031 #ifdef ORBITS
04032             orbits->fromPerm(perm);
04033 #endif
04034         }
04035     }
04036 }
04037
04038 //-----

```

```

04039 void MGraph::permMat(int size, int *perm, int **mat0, int **mat1)
04040 {
04041     // apply permutation to matrix 'mat0' and obtain 'mat1'
04042
04043     int j1, j2;
04044
04045     for (j1 = 0; j1 < size; j1++) {
04046         for (j2 = 0; j2 < size; j2++) {
04047             mat1[j1][j2] = mat0[perm[j1]][perm[j2]];
04048         }
04049     }
04050 }
04051
04052 //-----
04053 int MGraph::compMat(int size, int **mat0, int **mat1)
04054 {
04055     // comparison of matrix
04056
04057     int j1, j2, cmp;
04058
04059     for (j1 = 0; j1 < size; j1++) {
04060         for (j2 = 0; j2 < size; j2++) {
04061             cmp = mat0[j1][j2] - mat1[j1][j2];
04062             if (cmp != 0) {
04063                 return cmp;
04064             }
04065         }
04066     }
04067     return 0;
04068 }
04069
04070 //-----
04071 MNodeClass *MGraph::refineClass(MNodeClass *cl)
04072 {
04073     // Refine the classification
04074     // cl : the current 'MNodeClass' object
04075     // cn : the class number
04076     // Returns (the new class number corresponds to 'cn') if OK, or
04077     // 'None' if ordering condition is not satisfied
04078
04079     MNodeClass *ccl = cl;
04080     MNodeClass *xcl = NULL;
04081     MNodeClass *ncl = NULL;
04082     int ucl[GRCC_MAXNODES];
04083     int ccn = cl->nClasses;
04084     int nucl, nce;
04085     int td, cmp;
04086     int nccl;
04087
04088 #ifdef MONITOR
04089     nCallRefine++;
04090 #endif
04091     nucl = 0;
04092     while (ccl->nClasses != nucl) { // repeat refinement.
04093         nce = 0;
04094         nucl = 0;
04095         for (td = 1; td < nNodes; td++) {
04096             // 'td' is the next node and the current node is 'td-1'.
04097             // Count up the number of the elements in the current class
04098             // corresponding to the current node
04099             nce++;
04100             cmp = ccl->clCmp(td-1, td, ccn);
04101
04102             // the ordering condition is not satisfied.
04103             if (cmp > 0) {
04104                 if (ccl != cl) {
04105                     delete ccl;
04106                 }
04107                 return NULL;
04108             }
04109             else if (cmp < 0) {
04110                 // 'td' is in the next class to the current node.
04111                 ucl[nucl++] = nce; // close the current class
04112
04113                 // start new class
04114                 nce = 0;
04115             }
04116             // nothing to do for the case of 'cmp == 0'.
04117         }
04118     }
04119
04120     // close array 'ucl'.
04121     ucl[nucl++] = nce + 1;
04122
04123 #ifdef CHECK
04124     // inconsistent
04125     if (nucl < ccl->nClasses) {

```



```

04126         erEnd("refineClasses : smaller number of classes");
04127     }
04128 #endif
04129
04130     // preparation of the next repetition.
04131     xcl = new MNodeClass(nNodes, nucl);
04132     xcl->init(ucl, maxdeg, adjMat);
04133     ccn = xcl->nClasses;
04134
04135     nccl = ccl->nClasses;
04136
04137     if (ccl != cl) {
04138         delete ccl;
04139     }
04140     ccl = xcl;
04141     if (ccl->nClasses == nccl) {
04142         ccl->reorder(this);
04143         return ccl;
04144     }
04145
04146     nucl = 0;
04147 }
04148
04149 return ncl;
04150 }
04151
04152 //-----
04153 void MGraph::biconnME(void)
04154 {
04155     // Count the number of lPI components.
04156
04157     int j, root, vr, next, nart;
04158     MCopi mopi;
04159     MCBlock mblk;
04160
04161     // initialization
04162     bicount = 0;
04163
04164     mconn->init();
04165
04166     for (j = 0; j < nNodes; j++) {
04167         bidef[j] = -1;
04168         bilow[j] = -1;
04169     }
04170
04171     // find a root for the root of bisearch
04172     // root must be an external node
04173     // or an vertex when there are not external nodes.
04174     root = -1;
04175     vr = -1;
04176     for (j = 0; j < nNodes; j++) {
04177         if (bidef[j] < 0) {
04178             if (isExternal(j)) {
04179                 root = j;
04180                 break;
04181             } else if (vr < 0) {
04182                 vr = j;
04183             }
04184         }
04185     }
04186     // case (2)
04187     if (root < 0) {
04188         root = vr;
04189     }
04190
04191     if (root >= 0) {
04192         bisearchME(root, -1, 0, &mopi, &mblk, &next, &nart);
04193
04194         mconn->nlpopic = 0;
04195         mconn->nctopic = 0;
04196         for (j = 0; j < mconn->nopic; j++) {
04197             if (mconn->opics[j].loop > 0) {
04198                 mconn->nlpopic++;
04199             } else if (mconn->opics[j].ctloop > 0) {
04200                 mconn->nctopic++;
04201             }
04202         }
04203
04204         mconn->ne0bridges = 0;
04205         mconn->nelbridges = 0;
04206         for (j = 0; j < mconn->nbridges; j++) {
04207             if (mconn->bridges[j].next == 0) {
04208                 mconn->ne0bridges++;
04209             }
04210             if (mconn->bridges[j].next == 1) {
04211                 mconn->nelbridges++;
04212             }
04213         }
04214     }
04215 }

```

```

04213     }
04214
04215     mconn->nselfloops = 0;
04216     for (j = 0; j < nNodes; j++) {
04217         mconn->nselfloops += adjMat[j][j]/2;
04218     }
04219
04220     mconn->nalblocks = 0;
04221     for (j = 0; j < mconn->nblocks; j++) {
04222         if (mconn->blocks[j].nartps == 1) {
04223             mconn->nalblocks++;
04224         }
04225     }
04226     mconn->neblocks = mconn->nblocks + mconn->nctopic;
04227
04228     // no vertex.
04229 } else {
04230     // no vertices
04231     // Connected ==> only when ex=2, loop=0
04232     mconn->nopic = 0;
04233     mconn->ne0bridges = 0;
04234     mconn->nelbridges = 0;
04235 }
04236 }
04237
04238 //-----
04239 void MGraph::bisearchME(int nd, int pd, int ned, MCOpi *mopi, MCBlock *mblk, int *next, int *nart)
04240 {
04241     // Search biconnected component
04242     // visit : pd --> nd --> td
04243     // ned : the number of edges between pd and nd.
04244     // Output
04245     //
04246     //
04247     //      |
04248     //      x-----+
04249     //      |       |
04250     //      ----x pd |
04251     //      |       |
04252     //      x-----x nd | n art. pints.
04253     //      | / \ |
04254     //      | / \ |
04255     // m art p. x-x | x-x
04256     //          n1 x n3
04257     //          n2
04258     // output of bisearchME(n1, nd, ...) ==> npart = m
04259     // output of bisearchME(n2, nd, ...) ==> npart = 1
04260     // output of bisearchME(n3, nd, ...) ==> npart = n
04261     //
04262     // output of bisearchME(nd, pd, ...) ==> npart = n+1
04263     // n1 => block of (m+1) articulation points
04264     // n2 => block of 2 articulation points
04265     // n3 => no block
04266     //
04267     Bool newv;
04268     int td, td0, lp, conn, next1, nart1, nart2, nvisit, ndart;
04269     int opit, opit0, blkt;
04270     MCOpi mopil;
04271     MCBlock mblk1;
04272
04273     mopi->init();
04274     mblk->init();
04275
04276     *next = 0; // # external below 'nd' inclusive
04277     *nart = 0; // # articulation points 'nd' inclusive
04278     nart1 = 0; // # articulation points below 'nd'.
04279     nart2 = 0; // # 'nd' is articulation point then 1
04280     ndart = 0; // # the number of blocks attached to 'nd'
04281
04282     newv = (bidef[nd] < 0);
04283
04284     bidef[nd] = bicount;
04285     bilow[nd] = bicount;
04286     bicount++;
04287
04288     opit0 = mconn->opistkpctr;
04289
04290     if (pd < 0) {
04291         mconn->pushNode(nd);
04292     }
04293
04294     // external node and not the root
04295     if (isExternal(nd)) {
04296         if (pd >= 0) {
04297             mopi->next = 1;
04298             mopi->nlegs = 1;
04299             mblk->nartps = 1;

```

```

04300         *next          = 1;
04301         *nart           = 1;
04302         return;
04303     }
04304 }
04305
04306 nvisit = 0;
04307 td0    = -1;
04308 for (td = 0; td < nNodes; td++) {
04309     conn = adjMat[nd][td];
04310
04311     // td is not adjacent to nd
04312     if (conn < 1) {
04313         continue;
04314     }
04315     nvisit++;
04316     td0 = td;
04317
04318     // external node
04319     if (isExternal(td) && bidef[td] < 0) {
04320         mopi->nlegs++;
04321         mopi->next++;
04322         (*next)++;
04323         ndart++;
04324         mconn->addArtic(nd, 1);
04325         mblk->nartps++;
04326         nart2 = 1;
04327         if (newv) {
04328             mconn->pushNode(td);
04329         }
04330
04331         // self-loop : pd --> nd --> nd
04332     } else if (td == nd) {
04333         lp = conn/2;
04334         mopi->loop += lp;
04335         mopi->nedges += lp;
04336         ndart += lp;
04337         mconn->addArtic(nd, lp);
04338         mconn->addBlockSelf(nd, lp);
04339         mblk->nartps++;
04340         nart2 = 1;
04341
04342
04343         // back to the parent : pd --> nd --> pd, pd is a vertex
04344         // Back edges when the connection is multi-edges (ned > 1)
04345     } else if (td == pd) {
04346         if (ned > 1) {
04347             bilow[nd] = Min(bilow[nd], bidef[pd]);
04348             mopi->loop += ned - 1;
04349             mblk->loop += ned - 1;
04350         } else {
04351             // cancel this visit
04352             nvisit--;
04353         }
04354
04355         // back edge :td is already visited
04356     } else if (bidef[td] >= 0) {
04357         bilow[nd] = Min(bilow[nd], bidef[td]);
04358         if (bidef[nd] >= bidef[td]) {
04359             mopi->loop += conn;
04360             mopi->nedges += conn;
04361             mblk->loop += conn;
04362             mconn->pushEdge(td, nd);
04363         }
04364
04365         // new node
04366     } else if (bidef[td] < 0) {
04367
04368         // stack pointer
04369         if (pd < 0 && isExternal(nd)) {
04370             opit = opit0;
04371         } else {
04372             opit = mconn->opistkptr;
04373         }
04374         blkst = mconn->blkstkptr;
04375         mblk1.init();
04376
04377         mconn->pushEdge(nd, td);
04378
04379         // visit child
04380         bisearchME(td, nd, adjMat[td][nd], &mopil, &mblk1, &next1, &nart1);
04381
04382         // articulation point of bridge
04383         if (bilow[td] > bidef[nd]) {
04384
04385             // new OPI component (not including 'td')
04386             mopil.nlegs++;

```

```

04387
04388         mconn->addOPic(&mopil, opit);
04389         mopil.init();
04390         mopil.nlegs = 1;
04391         if (pd >= 0 || !isExternal(nd)) {
04392             // opi
04393             mconn->addBridge(nd, td, next1, nExtern);
04394
04395             // block
04396             ndart++;
04397             mconn->addArtic(nd, 1);
04398             // discard nparts from nodes below td
04399             mblk1.nartps = 2;
04400             mconn->addBlock(&mblk1, blkt);
04401             mblk1.init();
04402             mblk1.nartps = 1;
04403
04404             nart1 = 0;
04405             nart2 = 1; // nd is an articulation point
04406         }
04407     } else if (bilow[td] == bidef[nd]) {
04408         // articulation point of a block
04409         mopil.nedges += conn;
04410         if (pd >= 0 || !isExternal(nd)) {
04411             ndart++;
04412             mconn->addArtic(nd, 1);
04413             mblk1.nartps++;
04414             mconn->addBlock(&mblk1, blkt);
04415             mblk1.init();
04416             mblk1.nartps = 1;
04417             nart1 = 0;
04418             nart2 = 1; // nd is an articulation point
04419         }
04420     }
04421 } else {
04422     // inside a block
04423     mopil.nedges += conn;
04424 }
04425
04426 bilow[nd] = Min(bilow[nd], bilow[td]);
04427 mopil->nlegs += mopil.nlegs;
04428 mopil->next += mopil.next;
04429 mopil->loop += mopil.loop;
04430 mopil->nedges += mopil.nedges;
04431 mopil->ctloop += mopil.ctloop;
04432
04433 mblk->nartps += mblk1.nartps;
04434 mblk->loop += mblk1.loop;
04435
04436 *next += next1;
04437 *nart += nart1;
04438 }
04439
04440 } // end of for
04441
04442 // no connection except for 'pd'==>'nd'. Then 'nd' is an art. point
04443 if (nvisit == 0) {
04444     nart2 = 1;
04445 }
04446 *nart += nart2;
04447
04448 // add # loop inside counter terms
04449 if (newv) {
04450     if (pd >= 0) {
04451         mconn->pushNode(nd);
04452     }
04453     mopil->ctloop += Max(0, nodes[nd]->extloop);
04454 }
04455
04456 // 'nd' is the root node
04457 if (pd < 0) {
04458     if (isExternal(nd)) {
04459         if (td0 >= 0) {
04460             mconn->addArtic(td0, 1);
04461         }
04462         if (mconn->nopic > 0) {
04463             (mconn->opics[mconn->nopic-1].next)++;
04464         }
04465     } else {
04466         mconn->addOPic(mopi, opit0);
04467         if (ndart == 1) {
04468             // 'nd' has only 1 child
04469             // the root is not really an art. point
04470             (*nart)--;
04471             mconn->addArtic(nd, -1);
04472             (mconn->blocks[mconn->nblocks-1].nartps)--;
04473         }

```

```

04474     }
04475     }
04476 }
04477
04478 //-----
04479 BigInt MGraph::generate(void)
04480 {
04481     // Generate graphs
04482     // the generation process starts from 'connectClass'.
04483     MNodeClass *cl;
04484
04485 #ifdef DEBUGM
04486     printf("MGraph::generate:\n");
04487     // opt->printLevel(2);
04488 #endif
04489     // Initial classification of nodes.
04490     cl = new MNodeClass(nNodes, nClasses);
04491     cl->init(cList, maxdeg, adjMat);
04492     cl->reorder(this);
04493     connectClass(cl);
04494
04495     delete cl;
04496
04497 #ifdef DEBUGM
04498     printf("MGraph::generate:end:%ld\n", cDiag);
04499 #endif
04500     return cDiag;
04501 }
04502
04503 //-----
04504 void MGraph::connectClass(MNodeClass *cl)
04505 {
04506     // Connect nodes in a class to others
04507
04508     int sc, sn;
04509     MNodeClass *xcl;
04510
04511     xcl = refineClass(cl);
04512
04513 #ifdef DEBUG3
04514     printf("connectClass\n");
04515     print();
04516 #endif
04517     if (xcl == NULL) {
04518 #ifdef MONITOR
04519         discardRefine++;
04520 #endif
04521     } else {
04522         if (xcl->flg0 >= xcl->nClasses) {
04523             newGraph(xcl);
04524         } else {
04525             sc = xcl->clord[xcl->flg0];
04526             sn = xcl->flist[sc];
04527             connectNode(xcl->flg0, sn, xcl);
04528         }
04529     }
04530
04531     if (xcl != cl && xcl != NULL) {
04532         delete xcl;
04533     }
04534 }
04535
04536 //-----
04537 void MGraph::connectNode(int so, int ss, MNodeClass *cl)
04538 {
04539     int sc, sn;
04540
04541     sc = cl->clord[so];
04542     if (ss >= cl->flist[sc+1]) {
04543         connectClass(cl);
04544         return;
04545     }
04546
04547 #ifdef DEBUG3
04548     printf("connectNode\n");
04549     print();
04550 #endif
04551     for (sn = ss; sn < cl->flist[sc+1]; sn++) {
04552         connectLeg(so, sn, so, sn, cl);
04553         return;
04554     }
04555 }
04556
04557 //-----
04558 void MGraph::connectLeg(int so, int sn, int to, int ts, MNodeClass *cl)
04559 {
04560     // Add one connection between two legs.

```

```

04561 // 1. select another node to connect
04562 // 2. determine multiplicity of the connection
04563
04564 int sc, tc, tn, maxself, nc2, nc, maxcon, tsl, ncm, tol;
04565
04566 sc = cl->clord[so];
04567
04568 #ifdef CHECK
04569 if (sn >= cl->flist[sc+1]) {
04570     erEnd("connectLeg : illegal control");
04571     return;
04572 }
04573 #endif
04574 #ifdef DEBUG3
04575 printf("connectLeg:0\n");
04576 print();
04577 #endif
04578
04579 // There remains no free legs in the node 'sn' : move to next node.
04580 if (nodes[sn]->freelg < 1) {
04581
04582     if (!isConnected()) {
04583 #ifdef MONITOR
04584         discardDisc++;
04585 #endif
04586     } else {
04587 #ifdef DEBUG3
04588         printf("call connectNode:1\n");
04589 #endif
04590         // next node in the current class.
04591         connectNode(so, sn+1, cl);
04592     }
04593     return;
04594 }
04595
04596 #ifdef DEBUG3
04597 printf("connectLeg:2\n");
04598 print();
04599 #endif
04600 // connect a free leg of the current node 'sn'.
04601 for (tol = to; tol < cl->nClasses; tol++) {
04602     tc = cl->clord[tol];
04603     if (tol == to) {
04604         tsl = ts;
04605     } else {
04606         tsl = cl->flist[tc];
04607     }
04608 #ifdef MINMAXLEG
04609     if (tsl >= nNodes) {
04610         continue;
04611     }
04612 # ifdef DEBUG3
04613     printf("connectLeg:0: sn=%d, tsl=%d, ?(%d <= %d <= %d)\n",
04614         sn, tsl, nodes[sn]->cmindeg, nodes[tsl]->deg, nodes[sn]->cmaxdeg);
04615     printf("connectLeg:0: tsl=%d, sn=%d, ?(%d <= %d <= %d)\n",
04616         tsl, sn, nodes[tsl]->cmindeg, nodes[sn]->deg, nodes[tsl]->cmaxdeg);
04617 # endif
04618     if ((nodes[sn]->cmindeg > 0 && nodes[tsl]->deg < nodes[sn]->cmindeg)
04619         || (nodes[sn]->cmaxdeg > 0 && nodes[tsl]->deg > nodes[sn]->cmaxdeg)
04620         || (nodes[tsl]->cmindeg > 0 && nodes[sn]->deg < nodes[tsl]->cmindeg)
04621         || (nodes[tsl]->cmaxdeg > 0 && nodes[sn]->deg > nodes[tsl]->cmaxdeg)) {
04622 # ifdef DEBUG3
04623         printf("connectLeg:1: sn=%d, tsl=%d, !(%d <= %d <= %d)\n",
04624             sn, tsl, nodes[sn]->cmindeg, nodes[tsl]->deg, nodes[sn]->cmaxdeg);
04625         printf("connectLeg:2: tsl=%d, sn=%d, !(%d <= %d <= %d)\n",
04626             tsl, sn, nodes[tsl]->cmindeg, nodes[sn]->deg, nodes[tsl]->cmaxdeg);
04627 # endif
04628         continue;
04629     }
04630 #ifdef DEBUG3
04631     printf("connectLeg:3: sn=%d, tsl=%d\n", sn, tsl);
04632 #endif
04633 #endif
04634     for (tn = tsl; tn < cl->flist[tc+1]; tn++) {
04635         if (sc == tc && sn > tn) {
04636             continue;
04637         }
04638         if (nodes[tn]->freelg < 1) {
04639             continue;
04640         }
04641         // self-loop
04642     } else if (sn == tn) {
04643         if (opt->values[GRCC_OPT_NoSelfLoop] > 0) {
04644             continue;
04645         }
04646         if (nNodes > 1) {
04647             // there are two or more nodes in the graph :

```

```

04648         // avoid disconnected graph
04649         maxself = Min((nodes[sn]->freelg)/2, (nodes[sn]->deg-1)/2);
04650     } else {
04651         // there is only one node the graph.
04652         maxself = nodes[sn]->freelg/2;
04653     }
04654
04655     // If we can assume no tadpole, the following line can be used.
04656     if ((opt->values[GRCC_OPT_1PI] > 0
04657         || opt->values[GRCC_OPT_NoTadpole] > 0) && nExtern > 2) {
04658         maxself = Min((nodes[sn]->deg-2)/2, maxself);
04659     }
04660
04661     // for the number of connection.
04662     for (nc2 = maxself; nc2 > 0; nc2--) {
04663         nc = 2*nc2;
04664         ncm = CLWIGHTD(nc);
04665         adjMat[sn][sn] = nc;
04666         nodes[sn]->freelg -= nc;
04667         cl->incMat(sn, tn, ncm);
04668
04669         // next connection
04670 #ifdef DEBUG3
04671         printf("call connectLeg:1: %d--%d\n", sn, sn);
04672 #endif
04673         connectLeg(so, sn, tol, tn+1, cl);
04674
04675         // restore the configuration
04676         cl->incMat(tn, sn, - ncm);
04677         adjMat[sn][sn] = 0;
04678         nodes[sn]->freelg += nc;
04679     }
04680
04681     // connections between different nodes.
04682 } else {
04683     // maximum possible connection number
04684     maxcon = Min(nodes[sn]->freelg, nodes[tn]->freelg);
04685
04686     // avoid disconnected graphs.
04687     if (nNodes > 2 && nodes[sn]->deg == nodes[tn]->deg) {
04688         maxcon = Min(maxcon, nodes[sn]->deg-1);
04689     }
04690
04691 #ifdef CHECK
04692     if ((adjMat[sn][tn] != 0) ||
04693         (adjMat[sn][tn] != adjMat[tn][sn])) {
04694         printf("*** inconsistent connection: sn=%d, tn=%d",
04695             sn, tn);
04696         printAdjMat(cl);
04697         erEnd("inconsistent connection ");
04698     }
04699 #endif
04700
04701     // vary number of connections
04702     for (nc = maxcon; nc > 0; nc--) {
04703         ncm = CLWIGHTO(nc);
04704         adjMat[sn][tn] = nc;
04705         adjMat[tn][sn] = nc;
04706         nodes[sn]->freelg -= nc;
04707         nodes[tn]->freelg -= nc;
04708         cl->incMat(sn, tn, ncm);
04709         cl->incMat(tn, sn, ncm);
04710
04711         // next connection
04712 #ifdef DEBUG3
04713         printf("call connectLeg:2: %d--%d\n", sn, tn);
04714 #endif
04715         connectLeg(so, sn, tol, tn+1, cl);
04716
04717         // restore configuration
04718         cl->incMat(sn, tn, - ncm);
04719         cl->incMat(tn, sn, - ncm);
04720         adjMat[sn][tn] = 0;
04721         adjMat[tn][sn] = 0;
04722         nodes[sn]->freelg += nc;
04723         nodes[tn]->freelg += nc;
04724     }
04725 }
04726 }
04727 }
04728 }
04729
04730 //-----
04731 Bool MGraph::isOptM(void)
04732 {
04733     Bool ok = True;
04734 }

```

```

04735     opi      = (mconn->nopic == 1);
04736     opiloop   = (mconn->nlpopic <= 1);
04737     tadpole   = (mconn->ne0bridges >= ((nExtern == 1) ? 0 : 1));
04738     selfloop   = (mconn->nselfloops > 0);
04739     tadblock   = (mconn->nalblocks > 0);
04740     block      = (mconn->neblocks == 1);
04741     extself    = (mconn->nelbridges > 0);
04742
04743     if (opt->values[GRCC_OPT_1PI] > 0) {
04744         ok = ok && opi;
04745 #ifdef DEBUGM
04746         printf("isOptM: 1PI:%d nopic=%d\n", ok, mconn->nopic);
04747 #endif
04748     } else if (opt->values[GRCC_OPT_1PI] < 0) {
04749         ok = ok && !opi;
04750 #ifdef DEBUGM
04751         printf("isOptM: -1PI:%d nopic=%d\n", ok, mconn->nopic);
04752 #endif
04753     }
04754     if (opt->values[GRCC_OPT_NoExtSelf] > 0) {
04755         ok = ok && !extself;
04756 #ifdef DEBUGM
04757         printf("isOptM: NoExtSelf:%d, extself=%d\n", ok, extself);
04758 #endif
04759     } else if (opt->values[GRCC_OPT_NoExtSelf] < 0) {
04760         ok = ok && extself;
04761 #ifdef DEBUGM
04762         printf("isOptM: -NoExtSelf:%d, extself=%d\n", ok, extself);
04763 #endif
04764     }
04765     if (opt->values[GRCC_OPT_NoTadpole] > 0) {
04766         ok = ok && !tadpole;
04767 #ifdef DEBUGM
04768         printf("isOptM: NoTadPole:%d, tadpole=%d\n", ok, tadpole);
04769 #endif
04770     } else if (opt->values[GRCC_OPT_NoTadpole] < 0) {
04771         ok = ok && tadpole;
04772 #ifdef DEBUGM
04773         printf("isOptM: -NoTadPole:%d\n", ok);
04774 #endif
04775     }
04776     if (opt->values[GRCC_OPT_NoSelfLoop] > 0) {
04777 #ifdef DEBUGM
04778         printf("isOptM: NoSelfLoop:%d\n", ok);
04779 #endif
04780     } else if (opt->values[GRCC_OPT_NoSelfLoop] < 0) {
04781         ok = ok && selfloop;
04782 #ifdef DEBUGM
04783         printf("isOptM: -NoSelfLoop:%d\n", ok);
04784 #endif
04785     }
04786     if (opt->values[GRCC_OPT_No1PtBlock] > 0) {
04787         ok = ok && !tadblock;
04788 #ifdef DEBUGM
04789         printf("isOptM: No1PtBlock:%d\n", ok);
04790 #endif
04791     } else if (opt->values[GRCC_OPT_No1PtBlock] < 0) {
04792         ok = ok && tadblock;
04793 #ifdef DEBUGM
04794         printf("isOptM: -NoSelfLoop:%d\n", ok);
04795 #endif
04796     }
04797     if (opt->values[GRCC_OPT_Block] > 0) {
04798         ok = ok && block;
04799 #ifdef DEBUGM
04800         printf("isOptM: Block:%d\n", ok);
04801 #endif
04802     } else if (opt->values[GRCC_OPT_Block] < 0) {
04803         ok = ok && !block;
04804 #ifdef DEBUGM
04805         printf("isOptM: -Block:%d\n", ok);
04806 #endif
04807     }
04808 #ifdef DEBUGM
04809     printf("isOptM:%d\n", ok);
04810 #endif
04811     return ok;
04812 }
04813
04814 //-----
04815 void MGraph::newGraph(MNodeClass *cl)
04816 {
04817     // A new candidate diagram is obtained.
04818     // It may be a disconnected graph
04819     //
04820     // Things to be done in the steps followed by this function.
04821     // 1. convert data format which treat the edges as objects.

```



```

04822 // 2. definition of loop momenta to the edges.
04823 // 3. analysis of loop structure in a graph.
04824 // 4. assignment of particles to edges and vertices to nodes
04825 // 5. recalculate symmetry factor
04826
04827 int connected;
04828 MNodeClass *xcl;
04829 Bool ok;
04830
04831 ngen++;
04832
04833 // refine class and check ordering condition
04834 xcl = refineClass(cl);
04835 if (xcl == NULL) {
04836 #ifdef MONITOR
04837     discardRefine++;
04838 #endif
04839
04840 // check whether this is a connected graph or not.
04841 } else {
04842     connected = isConnected();
04843     if (!connected) {
04844 #ifdef MONITOR
04845         discardDisc++;
04846 #endif
04847
04848 // connected graph : check isomorphism of the graph
04849 } else {
04850     ngconn++;
04851     if (!isIsomorphic(xcl)) {
04852 #ifdef MONITOR
04853         discardIso++;
04854 #endif
04855
04856 // We got a new connected and a unique representation of a class.
04857 } else {
04858     biconnME();
04859     if (isOptM()) {
04860         egraph->fromMGraph(this);
04861         if (egraph->isOptE()) {
04862             curcl->copy(xcl);
04863 #ifdef ORBITS
04864             orbits->toOrbits();
04865 #endif
04866             ok = True;
04867             if (opt->outmg != NULL) {
04868                 if (opt->proc != NULL) {
04869                     egraph->mId = opt->proc->mgrcount + 1;
04870                     egraph->sId = opt->proc->agrcount + 1;
04871                 } else if (opt->sproc != NULL) {
04872                     egraph->mId = opt->sproc->mgrcount + 1;
04873                     egraph->sId = opt->sproc->agrcount + 1;
04874                 } else {
04875                     egraph->mId = cDiag;
04876                 }
04877 #ifdef DEBUG1
04878                 printf("call outmg\n");
04879                 egraph->model->prModel();
04880                 opt->print();
04881                 egraph->print();
04882 #endif
04883                 ok = (*(opt->outmg))(egraph, opt->argmg);
04884 #ifdef DEBUG
04885                 printf("ok=%d\n", ok);
04886 #endif
04887             }
04888
04889             if (ok) {
04890                 // # generated graphs
04891                 cDiag++;
04892                 wscon.add(1, nsym*esym);
04893
04894                 // # generated lPI graphs
04895                 if (opi) {
04896                     wsopi.add(1, nsym*esym);
04897                     c1PI++;
04898                 }
04899
04900                 // # no tadpoles
04901                 if (!tadpole) {
04902                     cNoTadpole++;
04903                 }
04904                 // # no tadblocks
04905                 if (!tadblock) {
04906                     cNoTadBlock++;
04907                     if (opi) {
04908                         c1PINoTadBlock++;

```

```

04909         }
04910     }
04911
04912 #ifdef MONITOR
04913     printf("\n");
04914     printf("Graph : %ld (%ld) lPIComp=%d lPILoop=%d",
04915           cDiag, ngen, nPIComps, nPILoop);
04916     printf(" sym. factor = (%ld*%ld)\n", nsym, esym);
04917     printAdjMat(c1);
04918     // c1->printMat();
04919 #   ifdef ORBITS
04920     orbits->print();
04921 #   endif
04922 #   ifdef DEBUGM
04923     printf("refine:                %ld\n",
04924           nCallRefine);
04925     printf("discarded for refinement:  %ld\n",
04926           discardRefine);
04927     printf("discarded for disconnected: %ld\n",
04928           discardDisc);
04929     printf("discarded for duplication: %ld\n",
04930           discardIso);
04931 #   endif
04932 #endif
04933     // go to next step
04934 #ifdef DEBUGM
04935     printf("newGraph:%ld:accepted\n", ngen);
04936 #endif
04937     opt->newMGraph(this);
04938 } else {
04939 #ifdef DEBUGM
04940     printf("deleted by setOutMG-function\n");
04941 #endif
04942 }
04943 } else {
04944 #ifdef DEBUGM
04945     printf("newGraph:%ld:discardOptE\n", ngen);
04946 #endif
04947 }
04948 } else {
04949 #ifdef DEBUGM
04950     printf("newGraph:%ld:discardOptM\n", ngen);
04951 #endif
04952 }
04953 }
04954 }
04955 }
04956 if (xcl != c1 && xcl != NULL) {
04957     delete xcl;
04958 }
04959 }
04960
04961 //=====
04962 // class MORbits: orbits of nodes
04963 //-----
04964 MORbits::MORbits(void)
04965 {
04966     nOrbits = 0;
04967     nNodes  = 0;
04968 }
04969
04970 //-----
04971 MORbits::~MORbits(void)
04972 {
04973 }
04974
04975 //-----
04976 void MORbits::print(void)
04977 {
04978     int c;
04979
04980     printf("Orbits : nOrbits=%d: nd2or=", nOrbits);
04981     prIntArray(nNodes, nd2or, "\n");
04982     for (c = 0; c < nOrbits; c++) {
04983         printf("    %2d: (%2d -- %2d) :", c, flist[c], flist[c+1]-1);
04984         prIntArray(flist[c+1]-flist[c], or2nd+flist[c], "\n");
04985     }
04986 }
04987
04988 //-----
04989 void MORbits::initPerm(int nnodes)
04990 {
04991     int j;
04992
04993     nNodes = nnodes;
04994     for (j = 0; j < nNodes; j++) {
04995         nd2or[j] = j;

```

```

04996     }
04997 }
04998
04999 //-----
05000 void MORbits::fromPerm(int *perm)
05001 {
05002     // update the orbits of nodes by the permutation
05003
05004     int j, j1, k, l;
05005
05006     for (j = 0; j < nNodes; j++) {
05007         if (nd2or[j] != j) {
05008             // not the first element of an old orbit
05009             continue;
05010         }
05011         // merge orbits to orbit 'j'
05012         k = j;
05013         for (j1 = 0; j1 < nNodes && (k = perm[k]) != j; j1++) {
05014             if (nd2or[k] == j) {
05015                 // 'k' is already in 'j'
05016                 ;
05017             } else {
05018                 // old orbit 'k' had several elements: merge to orbit 'j'
05019                 for (l = 0; l < nNodes; l++) {
05020                     if (nd2or[l] == k) {
05021                         nd2or[l] = j;
05022                     }
05023                 }
05024             }
05025         }
05026 #ifdef CHECK
05027         if (j1 >= nNodes) {
05028             printf("*** fromPerm: illegal control: j=%d, k=%d\n", j, k);
05029             printf("perm=");
05030             prIntArray(nNodes, perm, " nd2or=");
05031             prIntArray(nNodes, nd2or, "\n");
05032             erEnd("fromPerm: illegal control");
05033         }
05034 #endif
05035     }
05036 }
05037
05038 //-----
05039 void MORbits::toOrbits(void)
05040 {
05041     // fill elements from 'nd2or'
05042     int nd, nel, k, lor;
05043
05044     lor = -1;
05045     nel = 0;
05046     nOrbits = 0;
05047     for (nd = 0; nd < nNodes; nd++) {
05048         if (nd2or[nd] > lor) {
05049             // lowest element of a new orbit
05050             lor = nd2or[nd];
05051             flist[nOrbits++] = nel;
05052             for (k = nd; k < nNodes; k++) {
05053                 if (nd2or[k] == lor) {
05054                     or2nd[nel++] = k;
05055                 }
05056             }
05057         }
05058     }
05059     flist[nOrbits] = nNodes;
05060 }
05061
05062 //*****
05063 // n-edge connected components
05064 //=====
05065 // class MCopi: one n-edge connected component
05066 //-----
05067 MCopi::MCopi(void)
05068 {
05069     init();
05070 }
05071
05072 //-----
05073 MCopi::~MCopi(void)
05074 {
05075     nodes = NULL;
05076 }
05077
05078 //-----
05079 void MCopi::init(void)
05080 {
05081     nlegs = 0;
05082     nedges = 0;

```

```

05083     nnodes = 0;
05084     next   = 0;
05085     loop   = 0;
05086     ctloop = 0;
05087     nnodes = 0;
05088     nodes  = NULL;
05089 }
05090
05091 //=====
05092 // class MCBridge
05093 //-----
05094 MCBridge::MCBridge(void)
05095 {
05096 }
05097
05098 //-----
05099 MCBridge::~MCBridge(void)
05100 {
05101 }
05102
05103 //=====
05104 // class MCBlock
05105 //-----
05106 MCBlock::MCBlock(void)
05107 {
05108     init();
05109 }
05110
05111 //-----
05112 MCBlock::~MCBlock(void)
05113 {
05114 }
05115
05116 //-----
05117 void MCBlock::init(void)
05118 {
05119     edges   = NULL;
05120     nedges  = 0;
05121     nartps  = 0;
05122     loop    = 0;
05123 }
05124
05125 //=====
05126 // class MConn: table of n-edge connected components
05127 //-----
05128 MConn::MConn(int nnod, int nedg)
05129 {
05130     snodes = nnod;
05131     sedges = nedg;
05132
05133     opics   = new MCOp[snodes];
05134     bridges = new MCBridge[sedges];
05135     blocks  = new MCBlock[sedges];
05136     articuls = new int[snodes];
05137
05138     // workspace
05139     opisp   = new int[snodes];
05140     opistk  = new int[snodes];
05141     blksp   = new Edge2n[sedges];
05142     blkstk  = new Edge2n[sedges];
05143     init();
05144 }
05145
05146 //-----
05147 MConn::~MConn(void)
05148 {
05149     delete[] blkstk;
05150     delete[] blksp;
05151     delete[] opistk;
05152     delete[] opisp;
05153
05154     delete[] articuls;
05155     delete[] blocks;
05156     delete[] bridges;
05157     delete[] opics;
05158 }
05159
05160 //-----
05161 void MConn::init(void)
05162 {
05163     int j;
05164
05165     nopic      = 0;
05166     nlpopic    = 0;
05167     nctopic    = 0;
05168     nbridges   = 0;
05169     ne0bridges = 0;

```

```

05170     nelbridges = 0;
05171     nselfloops = 0;
05172
05173     nblocks    = 0;
05174     nalblocks  = 0;
05175     narticuls  = 0;
05176     neblocks   = 0;
05177
05178     opistkpctr = 0;
05179     nopisp     = 0;
05180     blkstkpctr = 0;
05181     nblksp     = 0;
05182
05183     for (j = 0; j < snodes; j++) {
05184         articuls[j] = 0;
05185     }
05186 }
05187
05188 //-----
05189 void MConn::pushNode(int nd)
05190 {
05191     if (opistkpctr >= snodes) {
05192         erEnd("MConn::pushNode: opistkpctr >= snodes");
05193     }
05194     opistk[opistkpctr++] = nd;
05195 }
05196
05197 //-----
05198 void MConn::pushEdge(int n0, int n1)
05199 {
05200     if (blkstkpctr >= sedges) {
05201         erEnd("MConn::pushEdge: blkstkpctr >= sedges");
05202     }
05203     if (n0 <= n1) {
05204         blkstk[blkstkpctr][0] = n0;
05205         blkstk[blkstkpctr][1] = n1;
05206     } else {
05207         blkstk[blkstkpctr][0] = n1;
05208         blkstk[blkstkpctr][1] = n0;
05209     }
05210     blkstkpctr++;
05211 }
05212
05213 //-----
05214 void MConn::addOPic(MCOpI *mopi, int stp)
05215 {
05216     int j, nn;
05217
05218     if (nopic >= snodes) {
05219         erEnd("MConn::addOPic: table overflow");
05220     }
05221
05222     nn = opistkpctr - stp;
05223     for (j = 0; j < nn; j++) {
05224         opisp[nopisp+j] = opistk[stp+j];
05225     }
05226     opics[nopic].nnodes = nn;
05227     opics[nopic].nlegs  = mopi->nlegs;
05228     opics[nopic].nedges = mopi->nedges;
05229     opics[nopic].next   = mopi->next;
05230     opics[nopic].loop   = mopi->loop;
05231     opics[nopic].ctloop = mopi->ctloop;
05232     opics[nopic].nodes  = opisp + nopisp;
05233     nopisp += nn;
05234     opistkpctr = stp;
05235
05236     nopic++;
05237 }
05238
05239 //-----
05240 void MConn::addBridge(int n0, int n1, int nex, int nextot)
05241 {
05242     if (nbridges >= sedges) {
05243         erEnd("MConn::addBridge: table overflow");
05244     }
05245     if (n0 <= n1) {
05246         bridges[nbridges].nodes[0] = n0;
05247         bridges[nbridges].nodes[1] = n1;
05248     } else {
05249         bridges[nbridges].nodes[0] = n1;
05250         bridges[nbridges].nodes[1] = n0;
05251     }
05252     bridges[nbridges].next = Min(nex, nextot-nex);
05253     nbridges++;
05254 }
05255
05256 //-----

```

```

05257 void MConn::addArtic(int nd, int mul)
05258 {
05259     articuls[nd] += mul;
05260     if (articuls[nd] == mul) {
05261         narticuls++;
05262     } else if (articuls[nd] == 0) {
05263         narticuls--;
05264     }
05265 }
05266
05267 //-----
05268 void MConn::addBlock(MCBlock *mblk, int stp)
05269 {
05270     int j, nn;
05271
05272     if (nopic >= snodes) {
05273         erEnd("MConn::addOPIC: table overflow");
05274     }
05275
05276     nn = blkstkptr - stp;
05277     for (j = 0; j < nn; j++) {
05278         blksp[nblksp+j][0] = blkstk[stp+j][0];
05279         blksp[nblksp+j][1] = blkstk[stp+j][1];
05280     }
05281     blocks[nblocks].edges = blksp + nblksp;
05282     blocks[nblocks].nmedges = nn;
05283     blocks[nblocks].nartps = mblk->nartps;
05284     blocks[nblocks].loop = mblk->loop;
05285     nblksp += nn;
05286     blkstkptr = stp;
05287
05288     nblocks++;
05289 }
05290
05291 //-----
05292 void MConn::addBlockSelf(int nd, int mult)
05293 {
05294     MCBlock mblk;
05295     int j, stp;
05296
05297     mblk.edges = NULL;
05298     mblk.nmedges = 0;
05299     mblk.nartps = 1;
05300     mblk.loop = 1;
05301
05302     for (j = 0; j < mult; j++) {
05303         stp = blkstkptr;
05304         pushEdge(nd, nd);
05305         addBlock(&mblk, stp);
05306     }
05307 }
05308
05309 //-----
05310 void MConn::print(void)
05311 {
05312     int j, k;
05313
05314     printf("+++ MConn object: snodes=%d, sedges=%d\n", snodes, sedges);
05315     printf("    nopic=%d, nlpopic=%d, nctopic=%d, nbridges=%d, "
05316           "ne0bridges=%d, nelbridges=%d\n",
05317           nopic, nlpopic, nctopic, nbridges, ne0bridges, nelbridges);
05318     printf("    nblocks=%d, neblocks=%d, nalblocks=%d, narticuls=%d, nselfloop=%d\n",
05319           nblocks, neblocks, nalblocks, narticuls, nselfloops);
05320
05321     printf("    lPI components (%d)\n", nopic);
05322     for (j = 0; j < nopic; j++) {
05323         printf("        %d: nleg=%d, nnodes=%d, nedge=%d, ",
05324               j, opics[j].nlegs, opics[j].nnodes, opics[j].nedges);
05325         printf("next=%d, loop=%d, ctlp=%d: [",
05326               opics[j].next, opics[j].loop, opics[j].ctlp);
05327         for (k = 0; k < opics[j].nnodes; k++) {
05328             printf(" %d", opics[j].nodes[k]);
05329         }
05330         printf("]\n");
05331     }
05332
05333     printf("    bridges (%d)\n", nbridges);
05334     if (nbridges > 0) {
05335         printf("        ");
05336         for (j = 0; j < nbridges; j++) {
05337             printf("(%d,%d)[ex=%d] ",
05338                   bridges[j].nodes[0], bridges[j].nodes[1], bridges[j].next);
05339         }
05340         printf("\n");
05341     }
05342
05343     printf("    blocks (%d)\n", nblocks);

```

```

05344     for (j = 0; j < nblocks; j++) {
05345         printf("      %d: nmedges=%d, nartps=%d, loop=%d: [",
05346             j, blocks[j].nmedges, blocks[j].nartps, blocks[j].loop);
05347         for (k = 0; k < blocks[j].nmedges; k++) {
05348             printf(" (%d,%d)", blocks[j].edges[k][0], blocks[j].edges[k][1]);
05349         }
05350         printf("]\n");
05351     }
05352
05353     printf(" articulation points (%d)\n", narticuls);
05354     if (narticuls > 0) {
05355         printf(" ");
05356         for (j = 0; j < snodes; j++) {
05357             if (articuls[j] != 0) {
05358 #if 0
05359                 printf("%d(%d) ", j, articuls[j]);
05360 #else
05361                 printf("%d ", j);
05362 #endif
05363             }
05364         }
05365         printf("\n");
05366     }
05367 }
05368
05369 //*****
05370 // sgroup.cc
05371 //=====
05372 // compile options
05373 //=====
05374 // table of group elements
05375 //-----
05376 SGroup::SGroup(void)
05377 {
05378     // should be called before graph generation
05379     nnodes = -1;
05380     nelem = 0;
05381     elem = NULL;
05382 }
05383
05384 //-----
05385 SGroup::~SGroup(void)
05386 {
05387     int j;
05388
05389     if (elem != NULL) {
05390         for (j = GRCC_MAXGROUP-1; j >= 0; j--) {
05391             if (elem[j] != NULL) {
05392                 delete[] elem[j];
05393                 elem[j] = NULL;
05394             }
05395         }
05396         delete[] elem;
05397         elem = NULL;
05398     }
05399 }
05400
05401 //-----
05402 void SGroup::print(void)
05403 {
05404     int j;
05405
05406     printf("SGroup : nnodes = %d, nelem=%ld, cgen=%d, csav=%d\n",
05407         nnodes, nelem, cgen, csav);
05408     printf("  eclass =");
05409     prIntArray(necclass, eclass, "\n");
05410     for (j = 0; j < nelem; j++) {
05411         printf("      %4d: ", j);
05412         prIntArray(nnodes, elem[j], "\n");
05413     }
05414 }
05415
05416 //-----
05417 void SGroup::newGroup(int nnd, int nclss, int *clss)
05418 {
05419     int j;
05420
05421     clearGroup();
05422     nnodes = nnd;
05423     necclass = nclss;
05424
05425     if (necclass > GRCC_MAXNODES) {
05426         erEnd("newGroup : too many classes (GRCC_MAXNODES)");
05427     }
05428     for (j = 0; j < necclass; j++) {
05429         eclass[j] = clss[j];
05430     }

```

```

05431
05432     nelem = 0;
05433     if (elem == NULL) {
05434         elem = new int*[GRCC_MAXGROUP];
05435         for (j = 0; j < GRCC_MAXGROUP; j++) {
05436             elem[j] = NULL;
05437         }
05438     }
05439 }
05440
05441 //-----
05442 void SGroup::clearGroup(void)
05443 {
05444     nelem = 0;
05445     csav = -1;
05446     cgen = -1;
05447 }
05448
05449 //-----
05450 void SGroup::delGroup(void)
05451 {
05452     int j;
05453
05454     for (j = 0; j < GRCC_MAXGROUP; j++) {
05455         if (elem[j] != NULL) {
05456             delete[] elem[j];
05457             elem[j] = NULL;
05458         }
05459     }
05460     delete[] elem;
05461     elem = NULL;
05462     clearGroup();
05463 }
05464
05465 //-----
05466 int *SGroup::genNext(void)
05467 {
05468     cgen = nextPerm(nnodes, necclass, eclass, pgr, pgq, permg, cgen);
05469     if (cgen < 0) {
05470         return NULL;
05471     }
05472     return permg;
05473 }
05474
05475 //-----
05476 void SGroup::addGroup(int *p)
05477 {
05478     int j;
05479
05480     if (nelem >= GRCC_MAXGROUP) {
05481         erEnd("addGroup : too many elements (GRCC_MAXGROUP)");
05482     }
05483     if (elem[nelem] == NULL) {
05484         elem[nelem] = new int[GRCC_MAXNODES];
05485     }
05486     for (j = 0; j < nnodes; j++) {
05487         elem[nelem][j] = p[j];
05488     }
05489     nelem++;
05490 }
05491
05492 //-----
05493 BigInt SGroup::nElem(void)
05494 {
05495     return nelem;
05496 }
05497
05498 //-----
05499 int *SGroup::nextElem(void)
05500 {
05501     if (nelem < 0) {
05502         cgen = nextPerm(nelem, necclass, eclass,
05503             psr, psq, perms, cgen);
05504         return perms;
05505     }
05506     if (cgen < 0) {
05507         cgen = 0;
05508     }
05509     if (cgen >= nelem) {
05510         return NULL;
05511     } else {
05512         return elem[cgen++];
05513     }
05514 }
05515
05516 //*****
05517 // egraph.cc

```



```

05518 // Generate scalar connected Feynman graphs.
05519 //=====
05520 static const char *GRCC_ND_names[] = GRCC_ND_NAMES;
05521 static const char *GRCC_ED_names[] = GRCC_ED_NAMES;
05522
05523 //-----
05524 // class ENode
05525 //-----
05526 ENode::ENode(void)
05527 {
05528     id      = -1;
05529     egraph  = NULL;
05530     edges   = NULL;
05531     maxdeg  = 0;
05532     deg     = 0;
05533
05534     klow    = NULL;
05535     extloop = 0;
05536     ndtype  = GRCC_ND_Undef;
05537
05538     intrct  = -1;
05539 }
05540
05541 //-----
05542 ENode::ENode(EGraph *egrph, int loops, int sdeg)
05543 {
05544     {
05545         id      = -1;
05546         egraph  = egrph;
05547         edges   = NULL;
05548         maxdeg  = sdeg;
05549         deg     = 0;
05550
05551         klow    = NULL;
05552         extloop = 0;
05553         ndtype  = GRCC_ND_Undef;
05554
05555         intrct  = -1;
05556
05557         edges = new int[sdeg];
05558         klow  = new int[loops+1];
05559     }
05560
05561 //-----
05562 ENode::~ENode(void)
05563 {
05564     if (klow != NULL) {
05565         delete[] klow;
05566         delete[] edges;
05567         klow = NULL;
05568         edges = NULL;
05569     }
05570 }
05571
05572 //-----
05573 void ENode::copy(ENode *en)
05574 {
05575     int d;
05576
05577     deg = en->deg;
05578     extloop = en->extloop;
05579     ndtype = en->ndtype;
05580     for (d = 0; d < deg; d++) {
05581         edges[d] = en->edges[d];
05582     }
05583 }
05584
05585 //-----
05586 void ENode::initAss(EGraph *egrph, int nid, int sdg)
05587 {
05588     id      = nid;
05589     egraph  = egrph;
05590     maxdeg  = sdg;
05591 }
05592
05593 //-----
05594 void ENode::setId(EGraph *egrph, const int nid)
05595 {
05596     id      = nid;
05597     egraph  = egrph;
05598 }
05599
06000 //-----
06001 void ENode::setExtern(int typ, int pt)
06002 {
06003     ndtype = typ;
06004     intrct = pt;

```

```

05605 }
05606
05607 //-----
05608 void ENode::setType(int typ)
05609 {
05610     #ifdef CHECK
05611         if (ndtype != GRCC_ND_Undef && ndtype != typ) {
05612             fprintf(GRCC_Stderr, "*** ndtype is already defined : old=%d, new = %d\n",
05613                 ndtype, typ);
05614             erEnd("ndtype is already defined");
05615         }
05616     #endif
05617     ndtype = typ;
05618 }
05619
05620 //-----
05621 void ENode::print(void)
05622 {
05623     int j;
05624
05625     printf("Enode %d deg=%d, extl=%2d, intr=%d ",
05626         id, deg, extloop, intrct);
05627     if (egraph->bicount > 0) {
05628         printf("(%-8s) ", GRCC_ND_names[ndtype]);
05629     }
05630     printf("edge=[");
05631     for (j = 0; j < deg; j++) {
05632         printf(" %2d", edges[j]);
05633     }
05634     printf("]\n");
05635 }
05636
05637 //=====
05638 // class EEdge
05639 //-----
05640 EEdge::EEdge(void)
05641 {
05642     id = -1;
05643     egraph = NULL;
05644     nodes[0] = -1;
05645     nodes[1] = -1;
05646     // nlegs[0] = -1;
05647     // nlegs[1] = -1;
05648     ptcl = GRCC_PT_Undef;
05649     deleted = False;
05650
05651     edtype = GRCC_ED_Undef;
05652     cut = False;
05653
05654     emom = NULL;
05655     lmom = NULL;
05656 }
05657
05658 //-----
05659 EEdge::EEdge(EGraph *egrph, int nedges, int nloops)
05660 {
05661     id = -1;
05662     egraph = egrph;
05663     nodes[0] = -1;
05664     nodes[1] = -1;
05665     nlegs[0] = -1;
05666     nlegs[1] = -1;
05667     ptcl = GRCC_PT_Undef;
05668     deleted = False;
05669
05670     edtype = GRCC_ED_Undef;
05671     cut = False;
05672
05673     emom = new int[nedges];
05674     lmom = new int[nloops];
05675 }
05676
05677 //-----
05678 EEdge::~EEdge(void)
05679 {
05680     if (lmom != NULL) {
05681         delete[] lmom;
05682     }
05683     if (emom != NULL) {
05684         delete[] emom;
05685     }
05686 }
05687
05688 //-----
05689 void EEdge::copy(EEdge *ee)
05690 {
05691     nodes[0] = ee->nodes[0];

```

```

05692     nodes[1] = ee->nodes[1];
05693     id       = ee->id;
05694     ext      = ee->ext;
05695     dir      = ee->dir;
05696     edtype   = ee->edtype;
05697     cut      = ee->cut;
05698 }
05699
05700 //-----
05701 void EEdge::setId(EGraph *egrph, const int eid)
05702 {
05703     id       = eid;
05704     egraph   = egrph;
05705 }
05706
05707 //-----
05708 void EEdge::setType(int typ)
05709 {
05710     #ifdef CHECK
05711         if (edtype != GRCC_ED_Undef) {
05712             erEnd("edtype is already defined");
05713         }
05714     #endif
05715     edtype = typ;
05716 }
05717
05718 //-----
05719 void EEdge::setEMom(int nedges, int *em, int dir)
05720 {
05721     int e;
05722
05723     for (e = 0; e < nedges; e++) {
05724         if (em[e] != 0) {
05725             emom[e] = dir;
05726         }
05727     }
05728 }
05729
05730 //-----
05731 void EEdge::setLMom(int k, int dir)
05732 {
05733     lmom[k] = dir;
05734 }
05735
05736 //-----
05737 void EEdge::print(void)
05738 {
05739     int zero, n, k;
05740
05741     printf("Edge %2d ext=%d ptcl=%2d ", id, ext, ptcl);
05742     if (egrph == NULL || !egrph->assigned) {
05743         printf("[%d, %d]", nodes[0], nodes[1]);
05744     } else {
05745         printf("[(%d,%d), (%d,%d)]", nodes[0], nlegs[0], nodes[1], nlegs[1]);
05746     }
05747
05748     if (egrph != NULL && egraph->bicount > 0) {
05749         if (cut) {
05750             printf(" %-9s ", "Cut");
05751         } else {
05752             printf(" %-9s ", GRCC_ED_names[edtype]);
05753         }
05754         printf("c%2d: ", opicomp);
05755         printf("%s%d =", (ext)?"Q":"p", id);
05756         zero = True;
05757         for (n = 0; n < egraph->nEdges; n++) {
05758             if (emom[n] != 0) {
05759                 prMomStr(emom[n], "Q", n);
05760                 zero = False;
05761             }
05762         }
05763         for (k = 0; k < egraph->nLoops; k++) {
05764             if (lmom[k] != 0) {
05765                 prMomStr(lmom[k], "k", k);
05766                 zero = False;
05767             }
05768         }
05769         if (zero) {
05770             printf(" 0");
05771         }
05772     }
05773     printf("\n");
05774 }
05775
05776 //=====
05777 // class EFLine
05778 //-----

```

```

05779 EFLine::EFLine(void)
05780 {
05781     ftype = FL_Open;
05782     fkind = 0;
05783     nlist = 0;
05784 }
05785
05786 //-----
05787 void EFLine::print(const char *msg)
05788 {
05789     if (ftype == FL_Open) {
05790         printf(" Open");
05791     } else if (ftype == FL_Closed) {
05792         printf(" Loop");
05793     } else {
05794         printf(" ?%d", ftype);
05795     }
05796     if (fkind == GRCC_PT_Dirac) {
05797         printf(" Dirac");
05798     } else if (fkind == GRCC_PT_Majorana) {
05799         printf(" Major");
05800     } else if (fkind == GRCC_PT_Ghost) {
05801         printf(" Ghost");
05802     } else {
05803         printf(" ?%d", fkind);
05804     }
05805     printf(" len=%d ", nlist);
05806     prIntArray(nlist, elist, msg);
05807 }
05808
05809 //=====
05810 // class EGraph
05811 //-----
05812 EGraph::EGraph(int nnodes, int nedges, int mxdeg)
05813 {
05814     // Arguments
05815     //   nnodes : the number of nodes
05816     //   nedges : the number of edges
05817     //   mxdeg  : the maximum number of degrees of nodes
05818
05819     int j;
05820
05821     opt      = NULL;
05822     mgraph   = NULL;
05823     econn    = NULL;
05824
05825     sNodes   = nnodes;
05826     sEdges   = nedges;
05827     sMaxdeg  = mxdeg;
05828     sLoops   = sEdges - sNodes + 1; // assume connected graph
05829
05830     pId      = -1;
05831     mId      = -1;
05832     aId      = -1;
05833     sId      = 0;
05834     gSubId   = -1;
05835     assigned = False;
05836
05837     nNodes   = sNodes;
05838     nEdges   = sEdges;
05839     nLoops   = 0;
05840     nExtern  = 0;
05841
05842     nsym     = 1;
05843     esym     = 1;
05844     extperm  = 1;
05845     nsym1    = 1;
05846     multp    = 1;
05847     totalc   = 0;
05848
05849     nodes = new ENode*[sNodes];
05850     for (j = 0; j < sNodes; j++) {
05851         nodes[j] = new ENode(this, sNodes, sMaxdeg);
05852     }
05853     edges = new EEdge*[sEdges];
05854     for (j = 0; j < sEdges; j++) {
05855         edges[j] = new EEdge(this, sEdges, sLoops);
05856     }
05857
05858     bidef    = new int[sNodes];
05859     bilow    = new int[sNodes];
05860     bicount  = -1;
05861
05862     extMom   = new int[sEdges+1];
05863
05864     fsign    = 1;
05865     nflines  = -1;

```

```

05866     for (j = 0; j < GRCC_MAXFLINES; j++) {
05867         flines[j] = NULL;
05868     }
05869 }
05870 }
05871
05872 //-----
05873 EGraph::~EGraph(void)
05874 {
05875     int j;
05876
05877     for (j = GRCC_MAXFLINES-1; j >= 0; j--) {
05878         if (flines[j] != NULL) {
05879             delete flines[j];
05880             flines[j] = NULL;
05881         }
05882     }
05883
05884     delete[] extMom;
05885
05886     delete[] bilow;
05887     delete[] bidef;
05888
05889     for (j = 0; j < sEdges; j++) {
05890         delete edges[j];
05891         edges[j] = NULL;
05892     }
05893     delete[] edges;
05894
05895     for (j = 0; j < sNodes; j++) {
05896         delete nodes[j];
05897         nodes[j] = NULL;
05898     }
05899     delete[] nodes;
05900 }
05901 }
05902
05903 //-----
05904 void EGraph::copy(EGraph *eg)
05905 {
05906     int nd, ed;
05907
05908 #ifndef CHECK
05909     if (sNodes < eg->nNodes || sEdges < eg->nEdges ||
05910         sMaxdeg < eg->sMaxdeg || sLoops < eg->nLoops) {
05911         erEnd("EGraph::copy: sizes are too small");
05912     }
05913 #endif
05914     model = eg->model;
05915     proc = eg->proc;
05916     sproc = eg->sproc;
05917     mgraph = eg->mgraph;
05918     opt = mgraph->opt;
05919     econn = mgraph->mconn;
05920
05921     pId = eg->pId;
05922     mId = eg->mId;
05923     gSubId = eg->gSubId;
05924     nNodes = eg->nNodes;
05925     nEdges = eg->nEdges;
05926     nExtern = eg->nExtern;
05927     nLoops = eg->nLoops;
05928
05929     for (nd = 0; nd < nNodes; nd++) {
05930         nodes[nd]->copy(eg->nodes[nd]);
05931     }
05932     for (ed = 0; ed < nEdges; ed++) {
05933         edges[ed]->copy(eg->edges[ed]);
05934     }
05935
05936     nsym = eg->nsym;
05937     esym = eg->esym;
05938     extperm = eg->extperm;
05939     nsym1 = eg->nsym1;
05940     multp = eg->multp;
05941
05942     bicount = -1;
05943 }
05944
05945 //-----
05946 void EGraph::setExtLoop(int nd, int val)
05947 {
05948     // set the node 'nd' being an external node (-1) or a looped vertex
05949
05950     nodes[nd]->extloop = val;
05951 }
05952

```

```

05953 //-----
05954 void EGraph::endSetExtLoop(void)
05955 {
05956     // end of calling 'setExtLoop'.
05957
05958     int n;
05959
05960     nExtern = 0;
05961     for (n = 0; n < nNodes; n++) {
05962         if (isExternal(n)) {
05963             nExtern++;
05964         }
05965     }
05966 }
05967
05968 //-----
05969 void EGraph::fromDGraph(DGraph *dg)
05970 {
05971     int deg[GRCC_MAXNODES];
05972     int maxdeg, nextern, nconn, tc;
05973     int n0, n1, e;
05974
05975     if (dg->nnodes > GRCC_MAXNODES) {
05976         fprintf(GRCC_Stderr, "*** too many nodes\n");
05977         exit(1);
05978     }
05979     if (dg->nedges > GRCC_MAXEDGES) {
05980         fprintf(GRCC_Stderr, "*** too many edges\n");
05981         exit(1);
05982     }
05983
05984     for (e = 0; e < dg->nedges; e++) {
05985         if (dg->edges[e][0] >= dg->nnodes || dg->edges[e][1] >= dg->nnodes) {
05986             fprintf(GRCC_Stderr, "*** undefined node:");
05987             fprintf(GRCC_Stderr, "edge[%d] = {%d, %d}\n",
05988                 e, dg->edges[e][0], dg->edges[e][1]);
05989             exit(1);
05990         }
05991     }
05992
05993     for (n0 = 0; n0 < dg->nnodes; n0++) {
05994         deg[n0] = 0;
05995     }
05996     for (e = 0; e < dg->nedges; e++) {
05997         deg[dg->edges[e][0]]++;
05998         deg[dg->edges[e][1]]++;
05999     }
06000     maxdeg = 0;
06001     nextern = 0;
06002     for (n0 = 0; n0 < dg->nnodes; n0++) {
06003         maxdeg = Max(maxdeg, deg[n0]);
06004         if (deg[n0] == 1) {
06005             nextern++;
06006         } if (deg[n0] < 0) {
06007             fprintf(GRCC_Stderr, "+++ node %d is isolated\n", n0);
06008         }
06009     }
06010
06011     mgraph = NULL;
06012     model = NULL;
06013     proc = NULL;
06014     sproc = NULL;
06015     opt = NULL;
06016
06017     nNodes = dg->nnodes;
06018     nEdges = dg->nedges;
06019     nLoops = 0;
06020     nExtern = nextern;
06021
06022     pId = 0;
06023     mId = 0;
06024     aId = 0;
06025     gSubId = 0;
06026     nsym = 1;
06027     nsym1 = 1;
06028     esym = 1;
06029     extperm = 1;
06030     multp = 1;
06031     // maxdeg = maxdeg;
06032
06033     for (n0 = 0; n0 < dg->nnodes; n0++) {
06034         nodes[n0]->deg = 0;
06035         nodes[n0]->extloop = dg->nodes[n0].extloop;
06036     }
06037     for (e = 0; e < dg->nedges; e++) {
06038         n0 = dg->edges[e][0];
06039         n1 = dg->edges[e][1];

```

```

06040         edges[e]->nodes[0] = n0;
06041         edges[e]->nodes[1] = n1;
06042         edges[e]->ext = (isExternal(n0) || isExternal(n1));
06043
06044         nodes[n0]->edges[nodes[n0]->deg++] = I2Vedge(e, -1);
06045         nodes[n1]->edges[nodes[n1]->deg++] = I2Vedge(e, +1);
06046     }
06047     for (n0 = 0; n0 < dg->nnodes; n0++) {
06048         nodes[n0]->setId(this, n0);
06049         nodes[n0]->intrct = dg->nodes[n0].intrct;
06050     }
06051
06052     for (e = 0; e < nEdges; e++) {
06053         edges[e]->setId(this, e);
06054     }
06055     tc = 0;
06056     for (n0 = 0; n0 < nNodes; n0++) {
06057         if (isExternal(n0)) {
06058             setExtern(n0, 1, GRCC_ND_Initial);
06059         } else {
06060             tc += 2*nodes[n0]->extloop + nodes[n0]->deg - 2;
06061         }
06062     }
06063     totalc = tc;
06064
06065     // get the number of connected components
06066     nconn = connComp();
06067     nLoops = nEdges - nNodes + nconn;
06068
06069     bicount = -1;
06070 }
06071
06072
06073 //-----
06074 void EGraph::fromMGraph(MGraph *mg)
06075 {
06076     // construct EGraph from adjacency matrix.
06077     // This function should be called after
06078     //   EGraph(), setExtLoop() and endSetExtLoop().
06079
06080     int n0, n1, ed, e;
06081     int j, ni, nf;
06082     PNodeClass *pnc;
06083     int k;
06084
06085 #ifndef CHECK
06086     if (sNodes < mg->nNodes || sEdges < mg->nEdges || sMaxdeg < mg->maxdeg) {
06087         erEnd("EGraph::fromMGraph: sizes are too small");
06088     }
06089     if (sLoops < mg->nLoops) {
06090         printf("too small sLoops : %d < %d\n", sLoops, mg->nLoops);
06091         erEnd("too small sLoops");
06092     }
06093 #endif
06094     mgraph = mg;
06095     sproc = mgraph->opt->sproc;
06096     if (sproc == NULL) {
06097         proc = NULL;
06098         model = NULL;
06099     } else {
06100         proc = sproc->proc;
06101         model = sproc->model;
06102     }
06103     opt = mgraph->opt;
06104     econn = mgraph->mconn;
06105
06106     nNodes = mg->nNodes;
06107     nEdges = mg->nEdges;
06108     nLoops = mg->nLoops;
06109     nExtern = mg->nExtern;
06110     pId = mg->pId;
06111     mId = mg->cDiag;
06112     aId = -1;
06113     if (opt->proc != NULL) {
06114         mId = mg->opt->proc->mgrcount;
06115         sId = mg->opt->proc->agrcount;
06116     } else if (mg->opt->sproc != NULL) {
06117         mId = mg->opt->sproc->mgrcount;
06118         sId = mg->opt->sproc->agrcount;
06119     } else {
06120         mId = mg->cDiag;
06121         sId = mg->cDiag;
06122     }
06123     gSubId = 0;
06124     nsym = mg->nsym;
06125     esym = mg->esym;
06126     extperm = 1;

```

```

06127     nsym1    = nsym;
06128     multp    = 1;
06129     maxdeg   = mg->maxdeg;
06130
06131     totalc   = 0;
06132     if (proc != NULL) {
06133         for (j = 0; j < proc->model->ncouple; j++) {
06134             totalc += proc->clist[j];
06135         }
06136     }
06137     for (ed = 0; ed < nEdges; ed++) {
06138         edges[ed]->edtype = GRCC_ED_Undef;
06139     }
06140
06141     ed = 0;
06142     for (n0 = 0; n0 < nNodes; n0++) {
06143         nodes[n0]->deg = 0;
06144     }
06145     for (n0 = 0; n0 < nNodes; n0++) {
06146         for (e = 0; e < mg->adjMat[n0][n0]/2; e++, ed++) {
06147             edges[ed]->nodes[0] = n0;
06148             edges[ed]->nodes[1] = n0;
06149             nodes[n0]->edges[nodes[n0]->deg++] = I2Vedge(ed, -1);
06150             nodes[n0]->edges[nodes[n0]->deg++] = I2Vedge(ed, +1);
06151             edges[ed]->ext = isExternal(n0);
06152         }
06153         for (n1 = n0+1; n1 < nNodes; n1++) {
06154             for (e = 0; e < mg->adjMat[n0][n1]; e++, ed++) {
06155                 edges[ed]->nodes[0] = n0;
06156                 edges[ed]->nodes[1] = n1;
06157                 nodes[n0]->edges[nodes[n0]->deg++] = I2Vedge(ed, -1);
06158                 nodes[n1]->edges[nodes[n1]->deg++] = I2Vedge(ed, +1);
06159                 edges[ed]->ext = (isExternal(n0) || isExternal(n1));
06160             }
06161         }
06162     }
06163 #ifndef CHECK
06164     if (ed != nEdges) {
06165         printf("*** EGraph::init: ed=%d != nEdges=%d\n",
06166             ed, nEdges+1);
06167         erEnd("EGraph::init: illegal connection");
06168     }
06169 #endif
06170
06171     for (j = 0; j < nNodes; j++) {
06172         nodes[j]->setId(this, j);
06173         nodes[j]->intrct = mg->nodes[j]->extloop;
06174     }
06175
06176     for (j = 0; j < nEdges; j++) {
06177         edges[j]->setId(this, j);
06178     }
06179
06180     ni = 0;
06181     nf = 0;
06182     if (proc != NULL) {
06183         ni = proc->ninitl;
06184         for (j = 0; j < ni; j++) {
06185             setExtern(j, proc->initlPart[j], GRCC_ND_Initial);
06186         }
06187
06188         nf = proc->nfinal;
06189         for (j = 0; j < nf; j++) {
06190             setExtern(j+ni, proc->finalPart[j], GRCC_ND_Final);
06191         }
06192         nExtern = ni + nf;
06193     } else if (sproc != NULL) {
06194         pnc = sproc->pnclass;
06195         for (j = 0; j < sproc->pnclass->nclass; j++) {
06196             if (pnc->type[j] == GRCC_AT_Initial || pnc->type[j] == GRCC_AT_External) {
06197                 for (k = pnc->cl2nd[j]; k < pnc->cl2nd[j+1]; k++) {
06198                     setExtern(j, k, GRCC_ND_Initial);
06199                     ni++;
06200                 }
06201             } else if (sproc->pnclass->type[j] == GRCC_AT_Final) {
06202                 for (k = pnc->cl2nd[j]; k < pnc->cl2nd[j+1]; k++) {
06203                     setExtern(j, k, GRCC_ND_Final);
06204                     nf++;
06205                 }
06206             }
06207         }
06208     }
06209     nExtern = ni + nf;
06210 }
06211 totalc = 0;
06212 for (j = 0; j < nNodes; j++) {
06213     if (!isExternal(j)) {

```



```

06214         totalc += 2*nodes[j]->extloop + nodes[j]->deg - 2;
06215     }
06216 }
06217
06218     bicount = -1;
06219 }
06220
06221 //-----
06222 ENode *EGraph::setExtern(int n0, int pt, int ndtyp)
06223 {
06224     ENode *nd;
06225     int npt, ept, eg;
06226
06227     nd = nodes[n0];
06228     if (nd->deg != 1) {
06229         if (prlevel > 0) {
06230             fprintf(GRCC_Stderr, "*** illegal external particle %d, %d, %d\n",
06231                 n0, pt, ndtyp);
06232         }
06233         erEnd("illegal external particle");
06234     }
06235
06236     // particle comes into the node
06237     if (ndtyp == GRCC_ND_Initial) {
06238         npt = pt;
06239     } else if (ndtyp == GRCC_ND_Final) {
06240         npt = -pt;
06241     } else {
06242         npt = 0;
06243         if (prlevel > 0) {
06244             fprintf(GRCC_Stderr, "*** illegal type of an external particle : "
06245                 "node = %d, particle = %d, type = %d", n0, pt, ndtyp);
06246         }
06247         erEnd("illegal type of an external particle");
06248     }
06249
06250     // edge attached to the external node
06251     eg = n0;
06252
06253     // particle flows on e from leg=0 to 1.
06254     if (model != NULL) {
06255         npt = model->normalParticle(npt);
06256         ept = model->normalParticle(-npt);
06257     } else {
06258         npt = 0;
06259         ept = 0;
06260     }
06261
06262     nd->setExtern(ndtyp, npt);
06263     edges[eg]->ptcl = ept;
06264
06265     return nd;
06266 }
06267
06268 //-----
06269 void EGraph::print(void)
06270 {
06271     // print the EGraph
06272
06273     int nd, ed, nlp;
06274
06275     nlp = nEdges - nNodes + 1;
06276     printf("\nEGraph\n");
06277     printf("    pId=%d, gSubId=%ld, mId=%ld, aId=%ld, sId=%ld\n",
06278         pId, gSubId, mId, aId, sId);
06279     printf("    sNodes=%d, sEdges=%d, sMaxdeg=%d, sLoops=%d\n",
06280         sNodes, sEdges, sMaxdeg, sLoops);
06281     printf("    nNodes=%d, nEdges=%d, nExtern=%d, nLoops=%d, totalc=%d\n",
06282         nNodes, nEdges, nExtern, nlp, totalc);
06283     printf("    ");
06284     if (model == NULL) {
06285         printf("model=NULL,");
06286     } else {
06287         printf("model=\"%s\",", model->name);
06288     }
06289     if (proc == NULL) {
06290         printf("proc=NULL,");
06291     } else {
06292         printf("proc=%d,", proc->id);
06293     }
06294     if (sproc == NULL) {
06295         printf("sproc=NULL,");
06296     } else {
06297         printf("sproc=%d,", sproc->id);
06298     }
06299     printf("\n");
06300     printf("    assigned=%d, sym = (%ld * %ld) ",

```

```

06301         assigned, nsym, esym);
06302     printf("extperm=%ld, nsym1=%ld, multp=%ld\n",
06303           extperm, nsym1, multp);
06304
06305     printf("  Nodes\n");
06306     for (nd = 0; nd < nNodes; nd++) {
06307         if (isExternal(nd)) {
06308             printf("    %2d Extern ", nd);
06309         } else {
06310             printf("    %2d Vertex ", nd);
06311         }
06312         nodes[nd]->print();
06313     }
06314     printf("  Edges\n");
06315     for (ed = 0; ed < nEdges; ed++) {
06316         if (edges[ed]->ext) {
06317             printf("    %2d Extern ", ed);
06318         } else {
06319             printf("    %2d Intern ", ed);
06320         }
06321         edges[ed]->print();
06322     }
06323     if (bicount > 0) {
06324         printf("  Biconn: nopicomp=%d, nopi2p=%d, opi2plp=%d, nadj2ptv=%d\n",
06325             nopicomp, nopi2p, opi2plp, nadj2ptv);
06326     }
06327     printf("\n");
06328
06329     if (nflines > 0) {
06330         prFLines();
06331     }
06332 }
06333 //-----
06334 void EGraph::prFLines(void)
06335 {
06336     int j;
06337
06338     printf("  Fermion lines %d, sign=%d (mId=%ld, aId=%ld)\n",
06339           nflines, fsign, mId, aId);
06340     for (j = 0; j < nflines; j++) {
06341         printf("%4d ", j);
06342         flines[j]->print("\n");
06343     }
06344 }
06345
06346 //-----
06347 int EGraph::dirEdge(int n, int e)
06348 {
06349     if (nodes[n]->edges[e] > 0) {
06350         return 1;
06351     } else {
06352         return -1;
06353     }
06354 }
06355
06356 //-----
06357 int EGraph::cmpMom(int *lm0, int *em0, int *lm1, int *em1)
06358 {
06359     int lp, ed, cmp, sgn = 0;
06360
06361     for (lp = 0; lp < nLoops; lp++) {
06362         if (lm0[lp] != 0) {
06363             if (lm1[lp] == 0) {
06364                 return 1;
06365             } else if (sgn == 0) {
06366                 sgn = lm0[lp]*lm1[lp]; // cancel first non-zero elements
06367 #ifdef CHECK
06368                 if (Abs(sgn) != 1) {
06369                     erEnd("cmpMom : illegal element of lmom");
06370                 }
06371 #endif
06372             } else {
06373                 cmp = lm0[lp] - sgn*lm1[lp];
06374                 if (cmp != 0) {
06375                     return cmp;
06376                 }
06377             }
06378         } else if (lm1[lp] != 0) { // lm0[lp] == 0
06379             return -1;
06380         }
06381     }
06382     for (ed = 0; ed < nEdges; ed++) {
06383         if (em0[ed] != 0) {
06384             if (em1[ed] == 0) {
06385                 return 1;
06386             } else if (sgn == 0) {
06387                 sgn = em0[ed]*em1[ed]; // cancel first non-zero elements

```

```

06388 #ifdef CHECK
06389         if (Abs(sgn) != 1) {
06390             erEnd("cmpMom : illegal element of emom");
06391         }
06392 #endif
06393     } else {
06394         cmp = em0[ed] - sgn*em1[ed];
06395         if (cmp != 0) {
06396             return cmp;
06397         }
06398     }
06399     } else if (em1[ed] != 0) { // em0[ed] == 0
06400         return -1;
06401     }
06402 }
06403 return 0;
06404 }
06405
06406 //-----
06407 int EGraph::groupLMom(int *grp, int *ed2gr)
06408 {
06409     // grp[g] : the number of elements in the group g (in [0,...,(ngrp-1)])
06410     // ed2gr[ed] : group number of edge ed.
06411
06412     int ed0, ed1, cmp, ngrp = -1;
06413
06414     for (ed0 = 0; ed0 < nEdges; ed0++) {
06415         ed2gr[ed0] = -1;
06416         grp[ed0] = 0;
06417     }
06418
06419     for (ed0 = 0; ed0 < nEdges; ed0++) {
06420         if (ed2gr[ed0] < 0) {
06421             ngrp++;
06422             ed2gr[ed0] = ngrp;
06423             grp[ngrp]++;
06424             for (ed1 = ed0+1; ed1 < nEdges; ed1++) {
06425                 cmp = cmpMom(edges[ed0]->lmom, edges[ed0]->emom,
06426                             edges[ed1]->lmom, edges[ed1]->emom);
06427                 if (cmp == 0) {
06428                     ed2gr[ed1] = ngrp;
06429                     grp[ngrp]++;
06430                 }
06431             }
06432         }
06433     }
06434     ngrp++;
06435
06436     return ngrp;
06437 }
06438
06439 //-----
06440 Bool EGraph::isOptE(void)
06441 {
06442     EGraph edupv = EGraph(sNodes, sEdges, sMaxdeg);
06443     EGraph *edup = &edupv;
06444     int grp[GRCC_MAXEDGES], ed2gr[GRCC_MAXEDGES];
06445     int g, ed, ngrp;
06446     Bool ok;
06447     int minopi2p;
06448
06449     if (opt->values[GRCC_OPT_No2PtL1PI] == 0
06450         && opt->values[GRCC_OPT_NoAdj2PtV] == 0) {
06451         return True;
06452     }
06453
06454     for (ed = 0; ed < nEdges; ed++) {
06455         edges[ed]->cut = False;
06456     }
06457
06458     biconne();
06459
06460     if (opt->values[GRCC_OPT_NoAdj2PtV] > 0) {
06461         if (nadj2ptv > 0) {
06462             return False;
06463         }
06464     } else if (opt->values[GRCC_OPT_NoAdj2PtV] < 0) {
06465         if (nadj2ptv < 1) {
06466             return False;
06467         }
06468     }
06469     if (opt->values[GRCC_OPT_No2PtL1PI] == 0) {
06470         return True;
06471     }
06472
06473     if (nExtern == 2 && nopicomp == 1) {
06474         minopi2p = 2;

```

```

06475     } else {
06476         minopi2p = 1;
06477     }
06478
06479     if (opi2plp > 0 && nopi2p >= minopi2p) {
06480         if (opt->values[GRCC_OPT_No2PtL1PI] > 0) {
06481             return False;
06482         } else if (opt->values[GRCC_OPT_No2PtL1PI] < 0) {
06483             return True;
06484         }
06485     }
06486
06487     edup->copy(this);
06488     ngrp = groupLMom(grp, ed2gr);
06489
06490     for (g = 0; g < ngrp; g++) {
06491         ok = True;
06492         if (grp[g] < 2) {
06493             continue;
06494         }
06495         for (ed = 0; ed < nEdges; ed++) {
06496             if (ed2gr[ed] == g) {
06497                 if (edges[ed]->ext) {
06498                     ok = False;
06499                     break;
06500                 }
06501                 edup->edges[ed]->cut = True;
06502             } else {
06503                 edup->edges[ed]->cut = False;
06504             }
06505         }
06506         if (ok) {
06507             edup->gSubId++;
06508             edup->biconnE();
06509
06510             if (edup->nExtern == 2 && edup->nopicomp == 1) {
06511                 minopi2p = 2;
06512             } else {
06513                 minopi2p = 1;
06514             }
06515
06516             if (edup->opi2plp > 0 && edup->nopi2p >= minopi2p) {
06517                 if (opt->values[GRCC_OPT_No2PtL1PI] > 0) {
06518                     return False;
06519                 } else if (opt->values[GRCC_OPT_No2PtL1PI] < 0) {
06520                     return True;
06521                 }
06522             }
06523         }
06524     }
06525
06526     if (opt->values[GRCC_OPT_No2PtL1PI] > 0) {
06527         return True;
06528     } else if (opt->values[GRCC_OPT_No2PtL1PI] < 0) {
06529         return False;
06530     } else {
06531         erEnd("isOptE: illegal control");
06532         return False;
06533     }
06534 }
06535
06536 //-----
06537 int EGraph::findRoot(void)
06538 {
06539     int root, vr, er, e, nd0, nd1;
06540
06541     root = -1;
06542     vr = -1;
06543     er = -1;
06544     for (e = 0; e < nEdges; e++) {
06545         if (edges[e]->cut || edges[e]->edtype == GRCC_ED_Deleted) {
06546             continue;
06547         }
06548         if (edges[e]->visited) {
06549             continue;
06550         }
06551         if (root < 0) {
06552             if (edges[e]->ext) {
06553                 nd0 = edges[e]->nodes[0];
06554                 nd1 = edges[e]->nodes[1];
06555                 if (isExternal(nd0) && !isExternal(nd1)) {
06556                     root = nd1;
06557                     break;
06558                 } else if (!isExternal(nd0) && isExternal(nd1)) {
06559                     root = nd0;
06560                     break;
06561                 }

```

```

06562     }
06563     }
06564     if (er < 0) {
06565         er = edges[e]->nodes[0];
06566     }
06567     if (vr < 0) {
06568         vr = edges[e]->nodes[0];
06569     }
06570 }
06571 // if root >= 0 : vertex adjacent to an external node
06572 if (root < 0) {
06573     // if vr >= 0 : a vertex
06574     if (vr >= 0) {
06575         root = vr;
06576     }
06577     // no vertex, only external nodes;
06578 } else {
06579     root = er;
06580 }
06581 }
06582 return root;
06583 }
06584
06585 //-----
06586 int EGraph::connComp(void)
06587 {
06588     // Count the number of connected component
06589     // If a connected component without free leg is not the whole graph then
06590     // return False, otherwise return True.
06591
06592     int j, ncc, nelelem;
06593
06594     for (j = 0; j < nNodes; j++) {
06595         nodes[j]->visited = -1;
06596     }
06597     ncc = 0; // # connected components
06598     nelelem = 0; // # visited nodes
06599     while (nelelem < nNodes) {
06600         for (j = 0; j < nNodes; j++) {
06601             if (nodes[j]->visited < 0) {
06602                 break;
06603             }
06604         }
06605         if (j >= nNodes) {
06606             break;
06607         }
06608         nelelem += connVisit(j, ncc);
06609         ncc++;
06610     }
06611     if (nelelem != nNodes) {
06612         erEnd("EGraph::connComp: illegal control");
06613     }
06614     return ncc;
06615 }
06616
06617 //-----
06618 int EGraph::connVisit(int nd, int ncc)
06619 {
06620     // Visiting connected node used for 'connComp'
06621
06622     int e, en, nn, j, nelelem;
06623
06624     nelelem = 1;
06625     nodes[nd]->visited = ncc;
06626     for (e = 0; e < nodes[nd]->deg; e++) {
06627         en = V2Iedge(nodes[nd]->edges[e]);
06628         for (j = 0; j < 2; j++) {
06629             nn = edges[en]->nodes[j];
06630             if (nodes[nn]->visited < 0) {
06631                 nelelem += connVisit(nn, ncc);
06632             }
06633         }
06634     }
06635     return nelelem;
06636 }
06637
06638 //-----
06639 void EGraph::biconnE(void)
06640 {
06641     int e, ie, root;
06642     int extlst[GRCC_MAXEDGES], intlst[GRCC_MAXEDGES];
06643     int opiext, opiloop;
06644
06645     nadj2ptv = 0;
06646     for (e = 0; e < nEdges; e++) {
06647         if (nodes[edges[e]->nodes[0]]->deg == 2 &&
06648             nodes[edges[e]->nodes[1]]->deg == 2 &&

```

```

06649         edges[e]->nodes[0] != edges[e]->nodes[1]) {
06650             nadj2ptv++;
06651         }
06652     }
06653
06654     biinitE();          // initialization
06655     bconn = 0;          // the number of connected components
06656
06657     opiext = 0;
06658     opiloop = 0;
06659
06660     for (e = 0; e < nEdges; e++) {
06661         // root of a spanning tree to be searched.
06662         root = findRoot();
06663         if (root < 0) {
06664             // no root : end of search
06665             break;
06666         }
06667
06668         // recursive search
06669         bisearchE(root, extlst, intlst, &opiext, &opiloop);
06670
06671         // opi2plp = Max(opi2plp, opiloop);      // ???
06672
06673         // adjust the #external for the root
06674         if (isExternal(root)) {
06675             opiext++;
06676         }
06677
06678         // 2 point function
06679         if (opiext == 2) {
06680             opi2plp = Max(opi2plp, opiloop);
06681             nopi2p++;
06682         }
06683         for (ie = 0; ie < nEdges; ie++) {
06684             if (intlst[ie]) {
06685                 edges[ie]->opicomp = nopicomp;
06686             }
06687         }
06688         nopicomp++;
06689
06690         bconn++;        // the number of connected component
06691     }
06692
06693     // momentum conservation of external particles
06694     extMomConsv();
06695
06696     #ifdef CHECK
06697         chkMomConsv();
06698     #endif
06699     #ifdef DEBUG
06700         printf("biconnE:opiext=%d, opiloop=%d, opi2plp=%d, nopi2p=%d, nopicomp=%d, bconn=%d\n",
06701             opiext, opiloop, opi2plp, nopi2p, nopicomp, bconn);
06702     #endif
06703     return;
06704 }
06705
06706
06707 //-----
06708 void EGraph::biinitE(void)
06709 {
06710     int n, j, e;
06711
06712     bicount = 0;          // counter of visiting node
06713     loopm = 0;           // # of independent loop momentum
06714
06715     nopicomp = 0;
06716     opi2plp = 0;
06717     nopi2p = 0;
06718
06719     for (n = 0; n < nNodes; n++) {
06720         bidef[n] = -1;
06721         bilow[n] = -1;
06722         nodes[n]->ndtype = GRCC_ND_Undef;
06723         for (j = 0; j < nLoops; j++) {
06724             nodes[n]->klow[j] = -1;
06725         }
06726     }
06727
06728     for (e = 0; e < nEdges; e++) {
06729         // edges[e]->cut is defined before
06730         edges[e]->visited = False;
06731         edges[e]->conid = -1;          // connected component
06732         edges[e]->edtype = GRCC_ED_Undef;
06733         edges[e]->opicomp = -1;
06734         for (j = 0; j < nEdges; j++) {
06735             edges[e]->emom[j] = 0;    // coefficients of ext. mom.

```

```

06736     }
06737     for (j = 0; j < nLoops; j++) {
06738         edges[e]->lmmom[j] = 0;           // coefficients of loop mom.
06739     }
06740 }
06741 }
06742
06743 //-----
06744 void EGraph::bisearchE(int nd, int *extlst, int *intlst, int *opiext, int *opiloop)
06745 {
06746     // Search in the spanning tree below 'nd'.
06747     // Arguments:
06748     //   nd       : input   : node to be visited
06749     //   extlst    : output  : set of external lines in the current lPI comp.
06750     //   intlst    : output  : set of internal lines in the current lPI comp.
06751     //   opiext    : output  : no. external lines of the current lPI component
06752     //   opiloop   : output  : no. loops of the current lPI component
06753     //
06754     // EGraph variables
06755     //   bicount   : counter of visiting node
06756     //   loopm     : no. of independent loop momentum
06757     //   nopicomp  : no. of OPI components
06758     //   opi2plp   : maximum number loops among 2-point looped OPI components
06759     //   nopi2p    : no. of 2-point OPI components
06760     //   edges[ie]->opicomp : OPI component no. of the edge
06761     //
06762
06763     int k, e, ie, ed, td, dir, j;
06764     int opiextl, opiloopl;
06765     int extlstl[GRCC_MAXEDGES]; // set of external nodes below
06766     int intlstl[GRCC_MAXEDGES]; // set of vertices in the lPI component
06767
06768     (*opiext) = 0;
06769     (*opiloop) = 0;
06770
06771     // visit node 'nd'
06772     bidef[nd] = bicount;
06773     bilow[nd] = bicount;
06774     for (k = 0; k < nLoops; k++) {
06775         nodes[nd]->klow[k] = bicount;
06776     }
06777     bicount++;
06778     for (j = 0; j < nEdges; j++) {
06779         extlst[j] = False;
06780         intlst[j] = False;
06781     }
06782
06783     // go to children : nd --> ed --> td
06784     for (e = 0; e < nodes[nd]->deg; e++) {
06785         ed = V2Iedge(nodes[nd]->edges[e]);
06786
06787         // check this edge is already visited
06788         if (edges[ed]->edtype == GRCC_ED_Deleted) {
06789             // permanently deleted edge
06790             continue;
06791         } else if (edges[ed]->visited) {
06792             // already visited (backward visit)
06793             continue;
06794         } else if (edges[ed]->cut) {
06795             // cut edge : treat as nd is an external
06796             (*opiext)++;
06797             nodes[nd]->setType(GRCC_ND_CPoint);
06798             continue;
06799         } else if (!isExternal(nd) && edges[ed]->ext) {
06800             // external
06801             edges[ed]->setType(GRCC_ED_Extern);
06802             edges[ed]->visited = True;
06803             edges[ed]->emom[ed] = 1;
06804             extlst[ed] = True;
06805             (*opiext)++;
06806             nodes[nd]->setType(GRCC_ND_CPoint);
06807             continue;
06808         }
06809
06810         // mark visited
06811         edges[ed]->visited = True;
06812
06813         // momentum is assigned on the backward move. (nd) 1 --> 0 (td)
06814         td = edges[ed]->nodes[0];
06815         dir = 1;
06816         if (td == nd) {
06817             td = edges[ed]->nodes[1];
06818             dir = -1;
06819         }
06820
06821         // biconnected component
06822         if (bidef[td] >= 0) {

```

```

06823         // a back edge is found. Already visited. Don't go further
06824         bilow[nd] = Min(bilow[nd], bilow[td]);
06825         edges[ed]->setType(GRCC_ED_Back);
06826         intlst[ed] = True;
06827         (*opiloop)++;
06828
06829         // create new loop momentum
06830         k = loopm++;
06831         nodes[nd]->klow[k] = Min(nodes[nd]->klow[k], nodes[td]->klow[k]);
06832         edges[ed]->setLMom(k, dir);
06833
06834         // self-loop
06835         if (td == nd) {
06836             nodes[nd]->setType(GRCC_ND_CPoint);
06837         }
06838     } else {
06839         // not a back edge
06840         // go down further
06841         bisearchE(td, extlst1, intlst1, &opiext1, &opiloop1);
06842
06843         // the set of external particles
06844         for (j = 0; j < nEdges; j++) {
06845             extlst[j] = extlst[j] || extlst1[j];
06846         }
06847
06848         // loop momenta
06849         for (k = 0; k < nLoops; k++) {
06850             if (nodes[td]->klow[k] <= nodes[nd]->klow[k]) {
06851                 // inside loop 'k'
06852                 edges[ed]->setLMom(k, dir);
06853             }
06854             nodes[nd]->klow[k] = Min(nodes[nd]->klow[k], nodes[td]->klow[k]);
06855         }
06856
06857         // cut point ?
06858         if (bilow[td] >= bidef[nd]) {
06859             // a cut point is found
06860             if (nodes[nd]->ndtype == GRCC_ND_Undef) {
06861                 nodes[nd]->setType(GRCC_ND_CPoint);
06862             }
06863             #ifndef CHECK
06864             } else if (nodes[nd]->ndtype != GRCC_ND_CPoint) {
06865                 if (prlevel > 0) {
06866                     fprintf(GRCC_Stderr, "bisearch: node %d is a cut point ",
06867                             nd);
06868                     fprintf(GRCC_Stderr, "(not undef %d)\n",
06869                             nodes[nd]->ndtype);
06870                 }
06871             }
06872             #endif
06873         } // cut point
06874
06875         // bridge ?
06876         if (bilow[td] > bidef[nd]) {
06877             // a bridge is found
06878             if (edges[ed]->edtype == GRCC_ED_Undef) {
06879                 edges[ed]->setType(GRCC_ED_Bridge);
06880                 nodes[td]->setType(GRCC_ND_CPoint);
06881             }
06882             #ifndef CHECK
06883             } else if (edges[ed]->edtype != GRCC_ED_Bridge) {
06884                 if (prlevel > 0) {
06885                     fprintf(GRCC_Stderr, "bisearch: edges %d is a bridge ", ed);
06886                     fprintf(GRCC_Stderr, "(not undef %d)\n", edges[ed]->edtype);
06887                 }
06888             }
06889             #endif
06890
06891             if (!edges[ed]->ext) {
06892                 // internal bridge
06893                 opiext1++; // from bridge
06894                 for (ie = 0; ie < nEdges; ie++) {
06895                     if (intlst1[ie]) {
06896                         // OPI component No.
06897                         edges[ie]->opicomp = nopicomp;
06898                         intlst1[ie] = False;
06899                     }
06900                 }
06901                 // 2pt OPI component
06902                 if (opiext1 == 2) {
06903                     opi2plp = Max(opi2plp, opiloop1);
06904
06905                     // the number of 2pt looped OPI components
06906                     nopi2p++;
06907                 }
06908                 // the number of OPI components
06909                 nopicomp++;
06910             }
06911         }

```



```

06910         opiext1 = 1;    // from bridge
06911         opiloop1 = 0;
06912
06913     } else {
06914         // not a bridge
06915
06916         bilow[nd] = Min(bilow[nd], bilow[td]);
06917
06918         if (edges[ed]->edtype == GRCC_ED_Undef) {
06919             edges[ed]->setType(GRCC_ED_Inloop);
06920 #ifndef CHECK
06921             } else if (edges[ed]->edtype != GRCC_ED_Inloop) {
06922                 if (prlevel > 0) {
06923                     fprintf(GRCC_Stderr, "bisearch: ");
06924                     fprintf(GRCC_Stderr, "edges %d is not undef (%d)\n",
06925                         ed, edges[ed]->edtype);
06926                 }
06927 #endif
06928             }
06929             intlst1[ed] = True;
06930         }
06931
06932         // merge to the current variables
06933         for (ie = 0; ie < nEdges; ie++) {
06934             intlst[ie] = intlst[ie] || intlst1[ie];
06935         }
06936         (*opiext) += opiext1;
06937         (*opiloop) += opiloop1;
06938
06939         // linear combination of external momenta
06940         edges[ed]->setEMom(nEdges, extlst1, dir);
06941     } // end of a child
06942 } // for (ed)
06943
06944 return;
06945 }
06946 }
06947
06948 //-----
06949 void EGraph::extMomConsv(void)
06950 {
06951     int e, ex, lex, rs;
06952
06953     lex = -1;
06954     for (e = 0; e < nEdges; e++) {
06955         if (edges[e]->ext) {
06956             lex = e;
06957             extMom[e] = edges[e]->emom[e];
06958         } else {
06959             extMom[e] = 0;
06960         }
06961     }
06962     // eliminate momentum of the last external particle
06963     if (lex < 0) {
06964         return;
06965     }
06966     for (e = 0; e < nEdges; e++) {
06967         rs = edges[e]->emom[lex];
06968         if (rs != 0) {
06969             for (ex = 0; ex < nEdges; ex++) {
06970                 edges[e]->emom[ex] -= rs*extMom[ex];
06971             }
06972         }
06973     }
06974 }
06975
06976 //-----
06977 void EGraph::chkMomConsv(void)
06978 {
06979     // check momentum conservation
06980
06981     int esum[GRCC_MAXEDGES];
06982     int lsum[GRCC_MAXNODES];
06983     Bool ok, okn;
06984     int n, ex, lk, ej, e, dir;
06985
06986     if (bicount < 1) {
06987         return;
06988     }
06989     ok = True;
06990     for (n = 0; n < nNodes; n++) {
06991         if (isExternal(n)) {
06992             continue;
06993         }
06994         for (ex = 0; ex < nEdges; ex++) {
06995             esum[ex] = 0;
06996         }

```

```

06997         for (lk = 0; lk < nLoops; lk++) {
06998             lsum[lk] = 0;
06999         }
07000         for (ej = 0; ej < nodes[n]->deg; ej++) {
07001             e = V2Iedge(nodes[n]->edges[ej]);
07002             if (edges[e]->nodes[0] == edges[e]->nodes[0]) {
07003                 continue;
07004             }
07005             dir = dirEdge(n, ej);
07006             for (ex = 0; ex < nEdges; ex++) {
07007                 if (edges[e]->ext) {
07008                     esum[ex] += dir*edges[e]->emom[ex];
07009                 }
07010             }
07011             for (lk = 0; lk < nLoops; lk++) {
07012                 lsum[lk] += dir*edges[e]->lmom[lk];
07013             }
07014         }
07015
07016         okn = True;
07017         for (ex = 0; ex < nEdges; ex++) {
07018             if (esum[ex] != 0) {
07019                 okn = False;
07020                 fprintf(GRCC_Stderr, "chkMomConsv:n=%d, esum[%d]=%d\n",
07021                     n, ex, esum[ex]);
07022             }
07023         }
07024
07025         for (lk = 0; lk < nLoops; lk++) {
07026             if (lsum[lk] != 0) {
07027                 okn = False;
07028                 fprintf(GRCC_Stderr, "chkMomConsv:n=%d, lsum[%d]=%d\n",
07029                     n, lk, lsum[lk]);
07030             }
07031         }
07032     }
07033
07034     if (!okn) {
07035         ok = False;
07036         fprintf(GRCC_Stderr, "*** Violation of momentum conservation ");
07037         fprintf(GRCC_Stderr, "at node =%d\n",n);
07038     }
07039 }
07040
07041 if (!ok) {
07042     print();
07043     erEnd("inconsistent momentum");
07044 }
07045 }
07046
07047 //-----
07048 int EGraph::legParticle(int ed, int el)
07049 {
07050     // outgoing particle from (edge, leg) = (ed, el)
07051
07052     return model->normalParticle((2*el-1)*edges[ed]->ptcl);
07053 }
07054
07055 //-----
07056 int EGraph::isFermion(int ed)
07057 {
07058     // return if particle on the edge ed follows Fermi statistics or not.
07059
07060     int ptcl, ptype;
07061
07062     ptcl = edges[ed]->ptcl;
07063     ptype = model->particles[Abs(ptcl)]->ptype;
07064
07065     return (ptype == GRCC_PT_Dirac || ptype == GRCC_PT_Majorana || ptype == GRCC_PT_Ghost);
07066 }
07067
07068 //-----
07069 int EGraph::fltrace(int fk, int nd0, int *fl)
07070 {
07071     int nfl, k, i, nd, nl, e, ed, el, fgcnt, fkind, lk;
07072
07073     nfl = 1;
07074     for (k = 0; k < nEdges; k++) {
07075         if (k >= nfl) {
07076             printf("*** fltrace:illegal contorl: k=%d, nEdges=%d\n", k, nEdges);
07077             break;
07078         }
07079         e = fl[k];
07080         ed = V2Iedge(e);
07081         el = V2Ileg(e);
07082 #ifdef CHECK
07083         if (ed > nEdges) {

```

```

07084         fprintf(GRCC_Stderr, "*** fltrace: ed=%d > nEdges=%d, fl=",
07085                 ed, nEdges);
07086         prIntArray(nNodes, fl, "\n");
07087         erEnd("fltrace: illegal fl");
07088     }
07089 #endif
07090     nd = edges[ed]->nodes[el];
07091     nl = edges[ed]->nlegs[el];
07092
07093     // end of the fline
07094     if (isExternal(nd) || nd == nd0) {
07095         fgcnt = 1;
07096         break;
07097     }
07098     fgcnt = 0;
07099     ed = 0;
07100     lk = 0;
07101     for (i = 0; i < nodes[nd]->deg; i++) {
07102         e = nodes[nd]->edges[i];
07103         ed = V2Iedge(e);
07104         fkind = model->particles[Abs(edges[ed]->ptcl)]->ptype;
07105         if (fkind == fk) {
07106             if (lk == 1) {
07107                 lk = -1;
07108             } else {
07109                 lk = 1;
07110             }
07111         } else {
07112             lk = 0;
07113         }
07114         if (!edges[ed]->visited) {
07115             if (!isFermion(ed)) {
07116                 edges[ed]->visited = True;
07117             } else {
07118                 if (fkind == fk) {
07119                     // i should be the neighbor of nl
07120                     // (nl, i) or (i, nl) should be a pair of fkind
07121                     if ((lk == 1 && nl == i+1) || (lk == -1 && nl == i-1)) {
07122                         edges[ed]->visited = True;
07123                         fgcnt++;
07124                         if (fgcnt == 1) {
07125                             fl[nfl++] = - e;
07126                             break;
07127                         }
07128                     }
07129                 }
07130             }
07131         }
07132     }
07133     if (fgcnt == 0) {
07134         printf("*** fline: Fermion number is not conserved\n");
07135         printf("    nd=%d, e=%d, ed=%d, fgcnt=%d\n",
07136                nd, e, ed, fgcnt);
07137         // erEnd("fline: Fermion number is not conserved");
07138     } else if (fgcnt > 1) {
07139         printf("+++ fline: more than two fermions: check fsign\n");
07140         printf("    nd=%d, e=%d, ed=%d, fgcnt=%d\n",
07141                nd, e, ed, fgcnt);
07142     }
07143 }
07144 if (k >= nNodes || k >= nfl) {
07145     printf("*** fline: illegal control\n");
07146     printf("    nfl=%d, ", nfl);
07147     prIntArray(nfl, fl, "\n");
07148     erEnd("fline: illegal control");
07149 }
07150 return nfl;
07151 }
07152
07153 //-----
07154 void EGraph::getFLines(void)
07155 {
07156     int fl[GRCC_MAXNODES];
07157     int nextn, exto[GRCC_MAXNODES];
07158     int e, ed, nd, floop, nswap, nfl, fkind, el, ptcl;
07159
07160     nflines = 0;
07161     for (ed = 0; ed < nEdges; ed++) {
07162         edges[ed]->visited = False;
07163     }
07164
07165     nextn = 0;
07166     for (ed = 0; ed < nEdges; ed++) {
07167         if (edges[ed]->visited) {
07168             continue;
07169         }
07170         if (!edges[ed]->ext) {

```

```

07171         continue;
07172     }
07173     el = 0;
07174     nd = edges[ed]->nodes[el];
07175     if (!isExternal(nd)) {
07176         el = 1;
07177         nd = edges[ed]->nodes[el];
07178         if (!isExternal(nd)) {
07179             erEnd("*** illegal control");
07180         }
07181     }
07182     if (!isFermion(ed)) {
07183         continue;
07184     }
07185     ptcl = legParticle(ed, 1-el);
07186     if (ptcl < 0) {
07187         continue;
07188     }
07189
07190     e = I2Vedge(ed, el);
07191     fl[0] = - e;
07192     nfl = 1;
07193     fkind = model->particles[Abs(edges[ed]->ptcl)]->ptype;
07194     edges[ed]->visited = True;
07195
07196     nfl = fltrace(fkind, nd, fl);
07197     addFLine(FL_Open, fkind, nfl, fl);
07198     e = fl[nfl-1];
07199     exto[nextn++] = edges[V2Iedge(e)]->nodes[V2Ileg(e)];
07200 }
07201
07202 // closed Fermion line
07203 floop = 0;
07204 for (ed = 0; ed < nEdges; ed++) {
07205     if (edges[ed]->visited) {
07206         continue;
07207     }
07208     if (!isFermion(ed)) {
07209         continue;
07210     }
07211     el = 0;
07212     ptcl = legParticle(ed, 1-el);
07213     if (ptcl < 0) {
07214         el = 1;
07215     }
07216     nd = edges[ed]->nodes[el];
07217     e = I2Vedge(ed, 1-el);
07218     fl[0] = e;
07219     nfl = 1;
07220     edges[ed]->visited = True;
07221
07222     fkind = model->particles[Abs(edges[ed]->ptcl)]->ptype;
07223     nfl = fltrace(fkind, nd, fl);
07224     addFLine(FL_Closed, fkind, nfl, fl);
07225     floop++;
07226 }
07227 if (nextn > 0) {
07228     nswap = sortb(nextn, exto);
07229 } else {
07230     nswap = 0;
07231 }
07232 if ((floop+nswap) % 2 == 0) {
07233     fsign = 1;
07234 } else {
07235     fsign = -1;
07236 }
07237 }
07238
07239 //-----
07240 void EGraph::addFLine(const FLType ft, int fk, int nfl, int *fl)
07241 {
07242     int j;
07243
07244     if (nflines >= GRCC_MAXFLINES) {
07245         erEnd("too many Fermion lines (GRCC_MAXEDGES)");
07246     }
07247     if (flines[nflines] == NULL) {
07248         flines[nflines] = new EFLine();
07249     }
07250     flines[nflines]->ftype = ft;
07251     flines[nflines]->fkind = fk;
07252     flines[nflines]->nlist = nfl;
07253     for (j = 0; j < nfl; j++) {
07254         flines[nflines]->elist[j] = fl[j];
07255     }
07256     nflines++;
07257 }

```

```

07258
07259 //*****
07260 // assign.cc
07261 //=====
07262 // Assign particles to the edges and interactions to nodes.
07263 // Completed graph data is saved in the form of EGraph.
07264
07265 // method : selection of assignable node
07266
07267 #ifdef DEBUGM
07268 static int nordleg = 0;
07269 static int nopleg = 0;
07270 static int niso = 0;
07271 static int nisol = 0;
07272 static int nisol1 = 0;
07273 static int nisol11 = 0;
07274 static int nisol12 = 0;
07275 static int nisol2 = 0;
07276 static int nisol3 = 0;
07277 static int nisol4 = 0;
07278 static int niso2 = 0;
07279 static int nivord = 0;
07280 static int nextonly = 0;
07281 #endif
07282
07283 //=====
07284 // class NCand
07285 NCand::NCand(const NCandSt sta, const int dega, const int nilst, int *ilst)
07286 {
07287     // candidate list of interactions attached to a vertex.
07288
07289     int j;
07290
07291     deg = dega;
07292     st = sta;
07293     nilist = nilst;
07294     for (j = 0; j < nilist; j++) {
07295         ilist[j] = ilst[j];
07296     }
07297
07298 #ifdef CHECK
07299     if (st == AS_Assigned) {
07300         if (nilist != 1) {
07301             erEnd("illegal assignment");
07302         }
07303     } else if (st == AS_AssExt) {
07304         if (nilist != 1) {
07305             erEnd("illegal assignment");
07306         }
07307     }
07308 #endif
07309 }
07310
07311 //-----
07312 NCand::~NCand(void)
07313 {
07314 }
07315
07316 //-----
07317 void NCand::prNCand(const char* msg)
07318 {
07319     printf("%d %d ", st, deg);
07320     prIntArray(nilist, ilist, msg);
07321 }
07322
07323 //=====
07324 // class ECand
07325 ECand::ECand(int dt, int nplst, int *plst)
07326 {
07327     int j;
07328
07329     det = dt;
07330     nplist = nplst;
07331     for (j = 0; j < nplist; j++) {
07332         plist[j] = plst[j];
07333     }
07334
07335     if (det && nplist != 1) {
07336         if (prlevel > 0) {
07337             fprintf(GRCC_Stderr, "*** ECand : len(plist) != 1 : det=%d ", det);
07338             prIntArrayErr(nplist, plist, "\n");
07339         }
07340         erEnd("ECand : len(plist) != 1");
07341     }
07342 }
07343
07344 //-----

```

```

07345 ECand::~ECand(void)
07346 {
07347 }
07348
07349 //-----
07350 void ECand::prECand(const char *msg)
07351 {
07352     prIntArray(nplist, plist, "");
07353     printf(" (det=%d)%s", det, msg);
07354 }
07355
07356 //=====
07357 // class ANode
07358 ANode::ANode(int dg)
07359 {
07360     int j;
07361
07362     deg = dg;
07363     nlegs = 0;
07364     anodes = new int[deg];
07365     aedges = new int[deg];
07366     aelegs = new int[deg];
07367     cand = NULL;
07368     for (j = 0; j < deg; j++) {
07369         anodes[j] = -1;
07370         aedges[j] = -1;
07371         aelegs[j] = -1;
07372     }
07373 }
07374
07375 //-----
07376 ANode::~ANode(void)
07377 {
07378     if (cand != NULL) {
07379         delete cand;
07380         cand = NULL;
07381     }
07382     if (aelegs != NULL) {
07383         delete[] aelegs;
07384         delete[] aedges;
07385         delete[] anodes;
07386         aelegs = NULL;
07387         aedges = NULL;
07388         anodes = NULL;
07389     }
07390 }
07391
07392 //-----
07393 int ANode::newleg(void)
07394 {
07395     // add a slot for a leg
07396
07397     int lg = nlegs;
07398
07399     nlegs++;
07400 #ifdef CHECK
07401     if (nlegs > deg) {
07402         fprintf(GRCC_Stderr, "*** ANode::newleg : nlegs = %d > deg = %d\n",
07403             nlegs, deg);
07404         erEnd("ANode::newleg : nlegs > deg");
07405     }
07406 #endif
07407     return lg;
07408 }
07409
07410 //=====
07411 // class AEdge
07412 AEdge::AEdge(int n0, int l0, int n1, int l1)
07413 {
07414     nodes[0] = n0; // node of 0 side of the edge
07415     nodes[1] = n1; // node of 1 side of the edge
07416     nlegs[0] = l0; // leg of the node of 0 side of the edge
07417     nlegs[1] = l1; // leg of the node of 1 side of the edge
07418     ptcl = GRCC_PT_Undef; // particle defined in the model
07419     cand = NULL; // candidate
07420 }
07421
07422 //-----
07423 AEdge::~AEdge(void)
07424 {
07425     if (cand != NULL) {
07426         delete cand;
07427     }
07428 }
07429
07430 //=====
07431

```

```

07432 // class Assign
07433 Assign::Assign(SProcess *sproc, MGraph *mgr, PNodeClass *pnc)
07434 {
07435     Bool ok;
07436     int j;
07437
07438     if (sproc == NULL) {
07439         erEnd("Assign: sproc == NULL");
07440     }
07441
07442     // pointers to related objects
07443     sproc      = sproc;
07444     model      = sproc->model;
07445     opt        = sproc->opt;
07446     mgrgraph   = mgr;
07447     pnclass    = pnc;
07448
07449     egraph = mgrgraph->egraph;
07450     if (egraph == NULL) {
07451         erEnd("Assign: egraph == NULL");
07452     }
07453
07454     astack     = sproc->astack;
07455     if (astack == NULL) {
07456         erEnd("Assign: astack == NULL");
07457     }
07458
07459 #ifdef DEBUG
07460     if (pnclass == NULL) {
07461         printf("+++ Assign: Assign : pnclass = NULL\n");
07462     } else {
07463         printf("+++ Assign: Assign : pnclass : \n");
07464         pnclass->prPNodeClass();
07465     }
07466 #endif
07467
07468     nNodes     = mgrgraph->nNodes;
07469     nEdges     = mgrgraph->nEdges;
07470     nExtern    = sproc->nExtern;
07471     nETotal    = 0; // total number of edges
07472
07473     // counters of graphs
07474     nAGraphs   = 0;
07475     wAGraphs   = Fraction(0,1);
07476     nAOPI      = 0;
07477     wAOPI      = Fraction(0,1);
07478
07479     nodes      = new ANode*[nNodes];
07480     edges      = new AEdge*[nEdges];
07481
07482     for (j = 0; j < nNodes; j++) {
07483         nodes[j] = NULL;
07484     }
07485     for (j = 0; j < nEdges; j++) {
07486         edges[j] = NULL;
07487     }
07488     for (j = 0; j < model->ncouple; j++) {
07489         cplleft[j] = sproc->clist[j];
07490     }
07491
07492     // initialize astack
07493     astack->setAGraph(this);
07494     astack->checkPoint(checkpoint0);
07495
07496     // import from mgrgraph
07497     ok = fromMGraph();
07498 #ifdef CHECK
07499     checkAG("Assign:Assign");
07500 #endif
07501
07502     if (ok) {
07503         // for debugging
07504 #ifdef DEBUG0
07505         printf("+++ End of initialization : candidate:\n");
07506         prCand("init");
07507         printf("\n");
07508 #endif
07509
07510         // start assignment
07511         assignAllVertices();
07512 #ifdef DEBUGM
07513         printf("mId=%ld, sId=%ld, ordleg=%d, ivord=%d, extonly=%d ",
07514             egraph->mId, egraph->sId, nordleg, nivord, nextonly);
07515         printf("niso=%d [%d(%d { %d, %d }, %d, %d, %d) %d]\n",
07516             niso, nisol, nisol1, nisol11, nisol12, nisol2, nisol3, nisol4,
07517             niso2);
07518 #endif

```

```

07519
07520     } else {
07521         // cannot assign
07522     }
07523
07524     // restore from the stack
07525 #ifdef CHECK
07526     astack->restoreMsg(checkpoint0, "Assign");
07527 #else
07528     astack->restore(checkpoint0);
07529 #endif
07530 }
07531
07532 //=====
07533 Assign::~Assign(void)
07534 {
07535     int j;
07536
07537     for (j = 0; j < nEdges; j++) {
07538         delete edges[j];
07539         edges[j] = NULL;
07540     }
07541     delete[] edges;
07542     edges = NULL;
07543
07544     for (j = 0; j < nNodes; j++) {
07545         delete nodes[j];
07546         nodes[j] = NULL;
07547     }
07548     delete[] nodes;
07549     nodes = NULL;
07550 }
07551
07552 //-----
07553 void Assign::prCand(const char *msg)
07554 {
07555     // Print candidate table
07556     int n, e, ne;
07557     AEdge *ed;
07558
07559     printf("\n");
07560     printf("+++ Candidate list: %s\n", msg);
07561     printf(" Nodes %d\n", nNodes);
07562     for (n = 0; n < nNodes; n++) {
07563         printf("%d: edges=", n);
07564         prIntArray(nodes[n]->deg, nodes[n]->aedges, ": cand=");
07565         if (nodes[n]->cand == NULL) {
07566             printf("NULL\n");
07567         } else {
07568             nodes[n]->cand->prNCand("\n");
07569         }
07570     }
07571     ne = Min(nEdges, nETotal);
07572     printf(" Edges %d\n", ne);
07573     for (e = 0; e < ne; e++) {
07574         ed = edges[e];
07575         if (ed == NULL) {
07576             printf("NULL_Edge\n");
07577         } else {
07578             printf("%d: %d->%d: cand=", e, ed->nodes[0], ed->nodes[1]);
07579             if (edges[e]->cand == NULL) {
07580                 printf("NULL\n");
07581             } else {
07582                 edges[e]->cand->prECand("\n");
07583             }
07584         }
07585     }
07586 }
07587
07588 //=====
07589 void Assign::checkAG(const char *msg)
07590 {
07591     int n, lg;
07592     Bool ok = True;
07593
07594     for (n = 0; n < nNodes; n++) {
07595         for (lg = 0; lg < nodes[n]->deg; lg++) {
07596             if (nodes[n]->aedges[lg] < 0) {
07597                 printf("*** checkAG:%s: n=%d, lg=%d, aedges=%d\n",
07598                     msg, n, lg, nodes[n]->aedges[lg]);
07599                 ok = False;
07600             }
07601         }
07602     }
07603     if (!ok) {
07604         prCand("checkAG");
07605         erEnd("checkAG: failed");
07606     }
07607 }

```



```

07606     }
07607 }
07608
07609 //=====
07610 // control of the process of particle/interaction assignment
07611
07612 //-----
07613 Bool Assign::assignAllVertices(void)
07614 {
07615     // Entry point of the assignment procedure
07616     Bool ok;
07617
07618 #ifdef CHECK
07619     checkCand("assignAllVertices:1");
07620 #endif
07621
07622 #ifdef DEBUG0
07623     printf("+++ particle assignment for '%ld'\n", mgraph->mId);
07624 #endif
07625
07626     // start main part
07627 #ifdef SIMPSEL
07628     ok = selectVertexSimp(-1);
07629 #else
07630     ok = selectVertex();
07631 #endif
07632
07633 #ifdef DEBUG0
07634     printf("\n");
07635     printf("+++ Total %ld assigned graphs for '%ld'\n",
07636           nAGraphs, mgraph->mId);
07637     printf("result: %ld ", nAGraphs);
07638     wAGraphs.print(" ");
07639     printf("%ld ", nAOPI);
07640     wAOPI.print("\n");
07641 #endif
07642
07643 #ifdef CHECK
07644     checkCand("assignAllVertices:2");
07645 #endif
07646
07647     return ok;
07648 }
07649
07650 //-----
07651 Bool Assign::selectVertexSimp(int lastv)
07652 {
07653     // Select a vertex for assignment of particles to legs
07654     //
07655     // Argument
07656     // lastv : previously assigned vertex; Nothing when lastv<0.
07657
07658     Bool ok;
07659     int v;
07660
07661 #ifdef CHECK
07662     checkCand("selectVertexSimp");
07663 #endif
07664
07665     // find a vertex to which interaction is assigned
07666     v = selUnAssVertexSimp(lastv);
07667
07668     // no more vertex
07669     if (v < 0) {
07670
07671         ok = allAssigned();
07672         if (ok) {
07673             opt->newAGraph(egraph);
07674         }
07675
07676         return ok;
07677     }
07678
07679     // select a leg and assign a particle to it
07680     ok = selectLeg(v, -1);
07681
07682     return ok;
07683 }
07684
07685 //-----
07686 Bool Assign::selectVertex(void)
07687 {
07688     // Select a vertex for assignment of particles to legs
07689     //
07690     // Argument
07691     // lastv : previously assigned vertex; Nothing when lastv<0.
07692

```

```

07693     Bool ok;
07694     int v;
07695
07696 #ifdef CHECK
07697     checkCand("selectVertex");
07698 #endif
07699
07700     // find a vertex to which interaction is assigned
07701     v = selUnAssVertex();
07702
07703     // no more vertex
07704     if (v < 0) {
07705
07706         ok = allAssigned();
07707         if (ok) {
07708             opt->newAGraph(egraph);
07709         }
07710
07711         return ok;
07712     }
07713
07714     // select a leg and assign a particle to it
07715     ok = selectLeg(v, -1);
07716
07717     return ok;
07718 }
07719
07720 //-----
07721 Bool Assign::selectLeg(int v, int lastlg)
07722 {
07723     // Select a leg of vertex 'v' for assignment of particle,
07724     // where the last assigned leg was 'lastlg'.
07725
07726     int pt, ln, j;
07727     Bool ok;
07728     CheckPt sav, sav0;
07729     int nplist, plist[GRCC_MAXMPARTICLES2];
07730
07731 #ifdef CHECK
07732     checkNode(v, "selectLeg:0");
07733     checkCand("selectLeg");
07734 #endif
07735
07736     // find a leg to be assigned
07737     ln = selUnAssLeg(v, lastlg);
07738
07739     // no more assignable leg for the current node
07740     if (ln < 0) {
07741         // move to next vertex
07742         ok = assignVertex(v);
07743
07744         return ok;
07745     }
07746
07747     // make the list of possible particles, incoming to the vertex
07748     astack->checkPoint(sav0);
07749     nplist = candPart(v, ln, plist, GRCC_MAXMPARTICLES2);
07750
07751     // no candidate
07752     if (nplist < 1) {
07753         return False;
07754     }
07755
07756     // assign each of possible particle to the leg 'lg'.
07757     astack->checkPoint(sav);
07758     for (j = 0; j < nplist; j++) {
07759         pt = plist[j];
07760
07761         // try to assign 'pt' to (v, ln)
07762         if(assignPLeg(v, ln, pt)) {
07763             // move to next leg
07764             selectLeg(v, ln);
07765         }
07766
07767 #ifdef CHECK
07768         astack->restoreMsg(sav, "selectLeg2");
07769 #else
07770         astack->restore(sav);
07771 #endif
07772     }
07773
07774 #ifdef CHECK
07775     astack->restoreMsg(sav0, "selectLeg2");
07776 #else
07777     astack->restore(sav0);
07778 #endif
07779     return True;

```

```

07780 }
07781
07782 //-----
07783 void Assign::saveCouple(int *sav)
07784 {
07785     int j;
07786     if (model->ncouple > 1) {
07787         for (j = 0; j < model->ncouple; j++) {
07788             sav[j] = cplleft[j];
07789         }
07790     }
07791 }
07792 }
07793
07794 //-----
07795 void Assign::restoreCouple(int *sav)
07796 {
07797     int j;
07798     if (model->ncouple > 1) {
07799         for (j = 0; j < model->ncouple; j++) {
07800             cplleft[j] = sav[j];
07801         }
07802     }
07803 }
07804 }
07805
07806 //-----
07807 Bool Assign::subCouple(int *cpl)
07808 {
07809     int j;
07810     if (model->ncouple > 1) {
07811         for (j = 0; j < model->ncouple; j++) {
07812             cplleft[j] -= cpl[j];
07813             if (cplleft[j] < 0) {
07814                 return False;
07815             }
07816         }
07817     }
07818 }
07819 return True;
07820 }
07821
07822 //-----
07823 Bool Assign::assignVertex(int v)
07824 {
07825     // Assign interactions to vertex 'v'
07826
07827     Bool done, ok;
07828     CheckPt sav, sav1;
07829     NCand *nc;
07830     int ia, n, j, cl;
07831     NCandSt vst;
07832     Bool ok1;
07833     int savc0[GRCC_MAXNCPLG], savc1[GRCC_MAXNCPLG];
07834
07835 #ifdef CHECK
07836     checkCand("assignVertex");
07837 #endif
07838
07839 // save the configuration
07840 astack->checkPoint(sav);
07841 saveCouple(savc0);
07842
07843 // assign to all vertices with only one candidate
07844 done = False;
07845 for (n = 0; n < nNodes; n++) {
07846     // check if vertex
07847     cl = pnclass->nd2cl[n];
07848     if (!isATEExternal(pnclass->type[cl])) {
07849         nc = nodes[n]->cand;
07850         if (nc->st == AS_UnAssLegs && nc->nilist == 1) {
07851             // bind interaction to the vertex
07852             ia = nc->ilist[0];
07853             vst = assignIVertex(n, ia);
07854             if (vst == AS_Impossible) {
07855                 // discard the current configuration
07856 #ifdef CHECK
07857                 astack->restoreMsg(sav, "assignVertex");
07858 #else
07859                 astack->restore(sav);
07860 #endif
07861                 restoreCouple(savc0);
07862                 return False;
07863             }
07864         }
07865     }
07866 }

```

```

07867         } else if (vst == AS_Assigned) {
07868             if (!subCouple(model->interacts[ia]->clist)) {
07869 #ifdef CHECK
07870                 astack->restoreMsg(sav, "assignVertex");
07871 #else
07872                 astack->restore(sav);
07873 #endif
07874                 restoreCouple(savc0);
07875                 return False;
07876             }
07877         }
07878         if (n == v) {
07879             done = (vst == AS_Assigned);
07880         }
07881     }
07882 }
07883 }
07884
07885 // uniquely determined
07886 ok = True;
07887 if (done) {
07888     // move to the next vertex
07889 #ifdef SIMPSEL
07890     ok = selectVertexSimp(v);
07891 #else
07892     ok = selectVertex();
07893 #endif
07894
07895 #ifdef CHECK
07896     astack->restoreMsg(sav, "assignVertex");
07897 #else
07898     astack->restore(sav);
07899 #endif
07900     restoreCouple(savc0);
07901
07902     return ok;
07903 }
07904
07905 // there are still several possibilities.
07906
07907 astack->checkPoint(sav1);
07908 saveCouple(savc1);
07909 for (j = 0; j < nodes[v]->cand->nilist; j++) {
07910     ia = nodes[v]->cand->ilist[j];
07911
07912     // bind interaction to the vertex
07913     ok1 = True;
07914     vst = assignIVertex(v, ia);
07915     if (vst == AS_Assigned) {
07916         ok1 = subCouple(model->interacts[ia]->clist);
07917     }
07918     if (ok1 && (vst == AS_Assigned || vst == AS_Assigned0)) {
07919         // move to the next vertex
07920 #ifdef SIMPSEL
07921         ok = selectVertexSimp(v);
07922 #else
07923         ok = selectVertex();
07924 #endif
07925     }
07926
07927 #ifdef CHECK
07928     astack->restoreMsg(sav1, "assignVertex");
07929 #else
07930     astack->restore(sav1);
07931 #endif
07932     restoreCouple(savc1);
07933 }
07934
07935 #ifdef CHECK
07936     astack->restoreMsg(sav, "assignVertex");
07937 #else
07938     astack->restore(sav);
07939 #endif
07940     restoreCouple(savc0);
07941
07942     return ok;
07943 }
07944
07945 //-----
07946 Bool Assign::allAssigned(void)
07947 {
07948     // Assignment of particles and interactions is finished.
07949     // Check isomorphism etc. and count symmetry factor
07950     // The sign from Fermi statistics is calculated in fillEGraph
07951
07952     BigInt nsym, esym, nsym1;
07953     MNodeClass *cl;

```

```

07954     Bool ok = True;
07955 #ifdef OPTEXTONLY
07956 #else
07957     Bool ext;
07958     int e, p;
07959 #endif
07960
07961 #ifdef CHECK
07962     checkCand("allAssigned");
07963 #endif
07964
07965     // check order of coupling constants
07966 #ifdef CHECK
07967     if (!checkOrderCpl()) {
07968         erEnd("allAssigned: checkOrderCpl = False");
07969     }
07970 #endif
07971
07972     // check duplication by violating ordering condition
07973     if (!isOrdLegs()) {
07974 #ifdef DEBUGM
07975         nordleg++;
07976 #endif
07977         return False;
07978     }
07979
07980     // classification of nodes
07981     cl = mgraph->curcl;
07982
07983     // check isomorphism and calculate sfactor
07984     nsym = 0;
07985     esym = 0;
07986
07987     ok = isIsomorphic(cl, &nsym, &esym, &nsym1);
07988     if (!ok || nsym < 1 || esym < 1) {
07989 #ifdef DEBUGM
07990         niso++;
07991 #endif
07992         return False;
07993     }
07994
07995     //-----
07996     // Now we got a new agraph.
07997
07998     // Update counters
07999     nAGraphs++;
08000     wAGraphs.add(1, nsym*esym);
08001     if (mgraph->opi) {
08002         nAOPI++;
08003         wAOPI.add(1, nsym*esym);
08004     }
08005
08006 #ifdef DEBUGl
08007     for (int j = 0; j < model->ncouple; j++) {
08008         if (cplleft[j] != 0) {
08009             printf("nAGraphs=%ld: 0 != cplleft =", nAGraphs);
08010             prIntArray(model->ncouple, cplleft, "\n");
08011             break;
08012         }
08013     }
08014 #endif
08015
08016 #ifdef CHECK
08017     checkAG("allAssigned");
08018 #endif
08019     // fill information to resulting Egraph
08020     fillEGraph(nAGraphs, nsym, esym, nsym1);
08021
08022 #ifdef OPTEXTONLY
08023 #else
08024     // check extonly particles
08025     for (e = 0; e < nEdges; e++) {
08026         p = Abs(egraph->edges[e]->ptcl);
08027         ext = egraph->edges[e]->ext;
08028         if (!ext) {
08029             if (model->particles[p]->extonly) {
08030 #ifdef DEBUGM
08031                 nextonly++;
08032 #endif
08033                 return False;
08034             }
08035         }
08036     }
08037 #endif
08038
08039 #ifdef DEBUG0
08040     printf("Assigned graph = %ld, sym = (%ld, %ld) ", nAGraphs, nsym, esym);

```

```

08041     prCand("allAssigned ");
08042 #endif
08043
08044     return True;
08045 }
08046
08047 //=====
08048 // Interface to MGraph and EGraph
08049 //-----
08050 Bool Assign::fromMGraph(void)
08051 {
08052     // Import from MGraph
08053
08054     int     npall, *pall;
08055     int     np[1];
08056     Bool    ok;
08057     int     j, k, n, nn, nl, deg, nc, ptcl, cl, typ, mcl, cll, typ1;
08058     int     lcn, ia;
08059     int     nilist, ilist[GRCC_MAXINTERACT];
08060     NCandSt st;
08061
08062     // create ANodes
08063     for (j = 0; j < nNodes; j++) {
08064         nodes[j] = new ANode(mgraph->nodes[j]->deg);
08065     }
08066
08067     // initialization of edge table
08068     nETotal = 0;
08069
08070     // List of all particles
08071     pall = model->allParticles(&npall);
08072
08073     // add nodes
08074     for (n = 0; n < mgraph->nNodes; n++) {
08075         cl = pnclass->nd2cl[n];
08076         typ = pnclass->type[cl];
08077
08078         // external particle
08079         if (isATExternal(typ)) {
08080
08081 #ifdef CHECK
08082             if (mgraph->nodes[n]->deg != 1) {
08083                 printf("*** assign:fromMGraph : "
08084                     "external but deg[%d] = %d != 1, type=%d\n",
08085                     n, mgraph->nodes[n]->deg, typ);
08086                 mgraph->print();
08087                 erEnd("assign:fromMGraph : illegal external particle");
08088             }
08089 #endif
08090             np[0] = pnclass->particle[cl];
08091             nodes[n]->cand = new NCand(AS_AssExt, 1, 1, np);
08092             for (nl = 0; nl < mgraph->nNodes; nl++) {
08093                 nc = mgraph->adjMat[n][nl];
08094                 for (k = 0; k < nc; k++) {
08095                     addEdge(n, nl, 1, np);
08096                 }
08097             }
08098
08099             // vertex
08100         } else {
08101
08102             // candidates of vertices
08103             deg = mgraph->nodes[n]->deg;
08104             st = AS_UnAssLegs;
08105             cl = sproc->pnclass->nd2cl[n]; // class in the process
08106             mcl = sproc->pnclass->cl2mcl[cl]; // class in the model
08107
08108             if (nodes[n]->cand != NULL) {
08109                 delete nodes[n]->cand;
08110             }
08111             lcn = model->ncouple;
08112
08113             // multiple coupling constants are defined in the model.
08114             if (lcn > 1) {
08115                 nilist = 0;
08116                 for (j = 0; j < model->cplgnvl[mcl]; j++) {
08117                     ia = model->cplgv1[mcl][j];
08118
08119                     // select by coupling constants
08120                     if (legArray(lcn, model->interacts[ia]->clist, cplleft)) {
08121                         ilist[nilist++] = ia;
08122                     }
08123                 }
08124                 nodes[n]->cand = new NCand(st, deg, nilist, ilist);
08125
08126             // there is only one coupling constant
08127             } else {

```

```

08128         nodes[n]->cand =
08129             new NCand(st, deg, model->cplgnvl[mcl], model->cplgvl[mcl]);
08130     }
08131
08132     // vertices
08133     for (n1 = n; n1 < mgraph->nNodes; n1++) {
08134         c1l = pnclass->nd2cl[n1];
08135         typ1 = pnclass->type[c1l];
08136         if (!isATEternal(typ1)) {
08137             nc = mgraph->adjMat[n][n1];
08138             if (n == n1) {
08139                 nc = nc/2;
08140             }
08141             for (k = 0; k < nc; k++) {
08142                 addEdge(n, n1, npall, pall);
08143             }
08144         }
08145     }
08146 }
08147 }
08148 #ifdef CHECK
08149 if (nETotal != nEdges) {
08150     printf("*** Assign::fromMGraph nETotal=%d != nEdges=%d\n",
08151         nETotal, nEdges);
08152     erEnd("Assign::fromMGraph nETotal= != nEdges");
08153 }
08154 #endif
08155
08156 // assign particle of an edge next to an external particle
08157 for (n = 0; n < mgraph->nNodes; n++) {
08158     cl = pnclass->nd2cl[n];
08159     typ = pnclass->type[cl];
08160     if (isATEternal(typ)) {
08161
08162         cl = pnclass->nd2cl[n];
08163         ptcl = pnclass->particle[cl];
08164
08165         // particle ptcl flows in to the node and
08166         // particle (- ptcl) flows in from the edge.
08167
08168 #ifdef CHECK
08169         if (ptcl == 0) {
08170             erEnd("fromMGraph: ptcl=0");
08171         }
08172 #endif
08173         ok = assignPLeg(n, 0, - ptcl);
08174         if (!ok) {
08175             // impossible config
08176             return False;
08177         }
08178         nn = nodes[n]->anodes[0];
08179         ok = updateCandNode(nn);
08180         if (!ok) {
08181             // impossible config
08182             return False;
08183         }
08184     }
08185 }
08186
08187 #ifdef CHECK
08188 checkCand("fromMGraph");
08189 checkAG("fromMGraph");
08190 #endif
08191
08192 return True;
08193 }
08194
08195 //-----
08196 void Assign::addEdge(int n0, int n1, int nplist, int *plist)
08197 {
08198     // add an edge and set connection information
08199     // n0 -- (new edge) -- n1, with candidates 'plist'
08200
08201     int lg0, lg1, eid;
08202     AEdge *aed;
08203
08204 #ifdef CHECK
08205     if (n0 >= nNodes || n1 >= nNodes) {
08206         printf("*** Assign::addEdge : undefined nodes %d: [%d, %d]",
08207             nETotal, n0, n1);
08208         erEnd("Assign::addEdge : undefined nodes");
08209     }
08210 #endif
08211
08212     // legs to be connected
08213     lg0 = nodes[n0]->newleg();
08214     lg1 = nodes[n1]->newleg();

```

```

08215
08216 // create edge
08217 #ifndef CHECK
08218     if (nETotal >= nEdges) {
08219         erEnd("too many edges");
08220     }
08221 #endif
08222
08223     aed = new AEdge(n0, lg0, n1, lg1);
08224     aed->cand = new ECand(False, nplist, plist);
08225
08226 #ifndef CHECK
08227     if (edges[nETotal] != NULL) {
08228         erEnd("edges[nETotal] != NULL");
08229     }
08230 #endif
08231
08232 // register to the edge table
08233 eid = nETotal;
08234 edges[nETotal++] = aed;
08235
08236 // connect edge and nodes
08237 connect(n0, lg0, eid, 0, n1, lg1);
08238 }
08239
08240 //-----
08241 void Assign::connect(int n0, int l0, int eg, int el, int n1, int l1)
08242 {
08243     // Connect (n0, l0) -- (eg, el) -- (n1)
08244
08245     ANode *nd0, *nd1;
08246     AEdge *ed;
08247     int eo;
08248
08249     nd0 = nodes[n0];
08250     nd1 = nodes[n1];
08251     ed = edges[eg];
08252     eo = 1 - el;
08253
08254     nd0->anodes[l0] = n1;
08255     nd0->aedges[l0] = eg;
08256     nd0->aelegs[l0] = el;
08257
08258     nd1->anodes[l1] = n0;
08259     nd1->aedges[l1] = eg;
08260     nd1->aelegs[l1] = eo;
08261
08262     ed->nodes[el] = n0;
08263     ed->nlegs[el] = l0;
08264
08265     ed->nodes[eo] = n1;
08266     ed->nlegs[eo] = l1;
08267 }
08268
08269 //-----
08270 Bool Assign::fillEGraph(int aid, BigInt nsym, BigInt esym, BigInt nsym1)
08271 {
08272     // Set variables of egraph with the result of particle assignment
08273     // Adjust the ordering of legs of vertices in a consistent way
08274     // with the interaction defined in the model.
08275
08276     ANode *an;
08277     ENode *en;
08278     int work[3][GRCC_MAXLEGS];
08279     int n, lr, e;
08280     int *elist;
08281     int lg, ed, eg;
08282 #ifndef CHECK
08283     int cl, ok = True;
08284 #endif
08285
08286     egraph->assigned = True;
08287
08288     egraph->aId = aid;
08289     egraph->nsym = nsym;
08290     egraph->esym = esym;
08291     egraph->bicount = -1;
08292     if (sproc != NULL) {
08293         egraph->extperm = sproc->extperm;
08294     } else {
08295         egraph->extperm = 1;
08296     }
08297     egraph->nsym1 = nsym1;
08298     egraph->multp = (egraph->extperm * egraph->nsym1) / egraph->nsym;
08299
08300 #ifndef CHECK
08301     checkAG("fillEGraph:0");

```



```

08302 #endif
08303     // external node
08304     for (n = 0; n < nNodes; n++) {
08305         // vertices
08306         #ifdef CHECK
08307             cl = pnclass->nd2cl[n];
08308             if (isATEXternal(pnclass->type[cl])) {
08309                 ;
08310             } else if (nodes[n]->cand->st != AS_Assigned) {
08311                 printf("*** fillEGraph : node %d is not assigned", n);
08312                 prCand("fillEGraph: node");
08313                 erEnd("fillEGraph : node is not assigned");
08314             }
08315         #endif
08316
08317         an = nodes[n];
08318         en = egraph->nodes[n];
08319
08320         en->initAss(egraph, n, an->deg);
08321
08322         en->intrct = an->cand->ilist[0];
08323         elist = reordLeg(n, work[0], work[1], work[2]);
08324         for (lr = 0; lr < nodes[n]->deg; lr++) {
08325             if (elist == NULL) {
08326                 lg = lr;
08327             } else {
08328                 lg = elist[lr];
08329             }
08330         #ifdef CHECK
08331             if (lg < 0 || lg >= nodes[n]->deg) {
08332                 printf("*** fillEGraph: n=%d, lr=%d: 0 <= lg=%d < %d\n",
08333                     n, lr, lg, nodes[n]->deg);
08334                 erEnd("fillEGraph: illegal reordering");
08335             }
08336         #endif
08337         ed = an->aedges[lg];
08338         eg = an->aelegs[lg];
08339
08340         en->edges[lr] = I2Vedge(ed, 2*eg-1);
08341
08342         egraph->edges[ed]->nodes[eg] = n;
08343         egraph->edges[ed]->nlegs[eg] = lr;
08344     }
08345 }
08346 // edges
08347 for (e = 0; e < nEdges; e++) {
08348     #ifdef CHECK
08349     if (edges[e]->cand->nplist != 1) {
08350         printf("*** fillEGraph : edge %d is not assigned", e);
08351         prCand("fillEGraph: edge");
08352         erEnd("fillEGraph : edge is not assigned");
08353     }
08354     #endif
08355     egraph->edges[e]->ptcl = edges[e]->cand->plist[0];
08356 }
08357 #ifdef EDGEORDER
08358 for (e = 0; e < nEdges; e++) {
08359     if (egraph->edges[e]->ptcl < 0) {
08360         egraph->edges[e]->ptcl = - edges[e]->cand->plist[0];
08361         n = egraph->edges[e]->nodes[0];
08362         lr = egraph->edges[e]->nlegs[0];
08363         egraph->edges[e]->nodes[0] = egraph->edges[e]->nodes[1];
08364         egraph->edges[e]->nlegs[0] = egraph->edges[e]->nlegs[1];
08365         egraph->edges[e]->nodes[1] = n;
08366         egraph->edges[e]->nlegs[1] = lr;
08367
08368         egraph->nodes[n]->edges[lr] = - egraph->nodes[n]->edges[lr];
08369         n = egraph->edges[e]->nodes[0];
08370         lr = egraph->edges[e]->nlegs[0];
08371         egraph->nodes[n]->edges[lr] = - egraph->nodes[n]->edges[lr];
08372     }
08373 }
08374 #endif
08375 #ifdef CHECK
08376 for (ed = 0; ed < nEdges; ed++) {
08377     n = egraph->edges[ed]->nodes[0];
08378     lr = egraph->edges[ed]->nlegs[0];
08379     if (egraph->nodes[n]->edges[lr] != -ed-1) {
08380         fprintf(GRCC_Stderr, "+++ node[%d][%d]=%d != - (edge[%d][0] + 1) = %d\n",
08381             n, lr, egraph->nodes[n]->edges[lr], e, -ed-1);
08382         ok = False;
08383     }
08384     n = egraph->edges[ed]->nodes[1];
08385     lr = egraph->edges[ed]->nlegs[1];
08386     if (egraph->nodes[n]->edges[lr] != ed+1) {
08387         fprintf(GRCC_Stderr, "+++ node[%d][%d]=%d != + (edge[%d][0] + 1) = %d\n",
08388             n, lr, egraph->nodes[n]->edges[lr], e, ed+1);

```

```

08389         ok = False;
08390     }
08391 }
08392 if (!ok) {
08393     egraph->print();
08394     erEnd("fillEGraph: illegal connection");
08395 }
08396 #endif
08397
08398 // analyse fermion line and determine Fermi statistical sign factor
08399 if (!model->skipFLine) {
08400     egraph->getFLines();
08401 } else {
08402     if (prlevel > 0) {
08403         fprintf(GRCC_Stderr, "+++ Sign factors related to "
08404             "Dirac/Majorana/Ghost particles are not calculated.\n");
08405     }
08406 }
08407 }
08408
08409 #ifdef CHECK
08410     checkAG("fillEGraph:0");
08411 #endif
08412     return True;
08413 }
08414 }
08415
08416 //-----
08417 int *Assign::reordLeg(int n, int *reord, int *plist, int *used)
08418 {
08419     // Reorder legs of node 'n' in accordance with the definition
08420     // of the interaction in the model
08421     //
08422     // plist[reord[j]] = model->interacts[ia]->plist[j]
08423
08424     int lg, ia, lr, deg, pt, ed;
08425     int *ilegs;
08426 #ifdef CHECK
08427     Bool found;
08428 #endif
08429
08430     // external node
08431     if (nodes[n]->cand->st == AS_AssExt) {
08432         reord[0] = 0;
08433         return 0;
08434     }
08435
08436     // degree of the node
08437     deg = nodes[n]->deg;
08438
08439     if (deg <= 1) {
08440         reord[0] = 0;
08441         return 0;
08442     }
08443
08444     // list of particles at the legs of the node 'n'
08445     for (lg = 0; lg < deg; lg++) {
08446         ed = nodes[n]->aedges[lg];
08447         pt = edges[ed]->cand->plist[0];
08448         plist[lg] = legEdgeParticle(n, lg, pt);
08449         used[lg] = 0;
08450     }
08451
08452     // list of legs in the interaction
08453     ia = nodes[n]->cand->ilist[0];
08454     ilegs = model->interacts[ia]->plist;
08455
08456     // reorder
08457 #ifdef CHECK
08458     found = False;
08459 #endif
08460     for (lr = 0; lr < deg; lr++) {
08461         for (lg = 0; lg < deg; lg++) {
08462             if (used[lg] == 0 && plist[lg] == ilegs[lr]) {
08463                 reord[lr] = lg;
08464                 used[lg] = 1;
08465 #ifdef CHECK
08466                 found = True;
08467 #endif
08468                 break;
08469             }
08470         }
08471     }
08472 #ifdef CHECK
08473     if (!found) {
08474         printf("*** reordLeg: illegal list of particles:"
08475             "interaction %d ", ia);

```

```

08476         prIntArray(deg, ilegs, " ");
08477         printf("vertex %d ", n);
08478         prIntArray(deg, plist, "\n");
08479         prCand("reordLeg");
08480         erEnd("reordLeg: illegal list of particles");
08481     }
08482 #endif
08483 }
08484
08485     return reord;
08486 }
08487
08488 //=====
08489 // Adjust the direction of a particle on an edge and on a leg of
08490 // node.
08491 //-----
08492 int Assign::getLegParticle(int n, int ln)
08493 {
08494     // Get particle code of (n, ln) when particle 'pt' runs
08495     // in the direction of the edge at (n, ln).
08496     //
08497     // Get particle code in the direction the edge at (n, ln)
08498     // when particle 'pt' is at leg (n, ln).
08499     //
08500     // These two cases are realized by the same function
08501
08502     ANode *nd;
08503     int elg, ed, pt;
08504
08505     nd = nodes[n];
08506     elg = nd->aelegs[ln];
08507     ed = nd->aedges[ln];
08508     pt = edges[ed]->cand->plist[0];
08509
08510     // normalization of sign for neutral particle
08511     if (elg == 0) {
08512         return model->normalParticle(-pt);
08513     } else {
08514         return model->normalParticle(pt);
08515     }
08516 }
08517
08518 //-----
08519 int Assign::legEdgeParticle(int n, int ln, int pt)
08520 {
08521     // Get particle code of (n, ln) when particle 'pt' runs
08522     // in the direction of the edge at (n, ln).
08523     //
08524     // Get particle code in the direction the edge at (n, ln)
08525     // when particle 'pt' is at leg (n, ln).
08526     //
08527     // These two cases are realized by the same function
08528
08529     // normalization of sign for neutral particle
08530
08531     if (nodes[n]->aelegs[ln] == 0) {
08532         return model->normalParticle(-pt);
08533     } else {
08534         return model->normalParticle(pt);
08535     }
08536 }
08537
08538 //-----
08539 int Assign::legPart(int v, int lg, int nplist, int *plist, int *rlist, const int size)
08540 {
08541     // Convert list of candidate particles in 'plist' at (v, lg) ==> edge
08542     // or edge ==> (v, lg)
08543
08544     int j, nrlist;
08545
08546     nrlist = 0;
08547     for (j = 0; j < nplist; j++) {
08548         nrlist = intSetAdd(nrlist, rlist,
08549                             legEdgeParticle(v, lg, plist[j]), size);
08550     }
08551     return nrlist;
08552 }
08553
08554 //-----
08555 int Assign::candPart(int v, int ln, int *plist, const int size)
08556 {
08557     // update candidate particles incoming to (v, ln)
08558
08559     int en;
08560     int ntplist;
08561     ECand *ec;
08562

```

```

08563     en = nodes[v]->aedges[ln];
08564
08565 #ifdef CHECK
08566     if (edges[en]->cand->det) {
08567         printf("*** candPart : particle of leg (%d, %d) "
08568             "is assigned to %d\n",
08569             v, ln, edges[en]->cand->plist[0]);
08570         checkCand("candPart");
08571         mgraph->printAdjMat(mgraph->curcl);
08572         prCand("candPart");
08573         erEnd("candPart : particle of leg is assigned");
08574     }
08575 #endif
08576
08577     ec = edges[en]->cand;
08578
08579     ntplist = legPart(v, ln, ec->nplist, ec->plist, plist, size);
08580
08581     return ntplist;
08582 }
08583
08584 //=====
08585 // tools for assignment
08586
08587 //-----
08588 int Assign::selUnAssVertexSimp(int lastv)
08589 {
08590     // Select a vertex for assignment to its leg.
08591     // We take one with minimum number of candidates
08592     //
08593     // There may be better method.
08594
08595     int v0, v;
08596
08597     // select vertex one by one in the sequential order of node number
08598     v0 = Max(lastv+1, nExtern);
08599     for (v = v0; v < nNodes; v++) {
08600         if (nodes[v]->cand->st == AS_UnAssLegs) {
08601             return v;
08602         }
08603     }
08604     return -1;
08605 }
08606
08607 //-----
08608 int Assign::selUnAssVertex(void)
08609 {
08610     // Select a vertex for assignment to its leg.
08611     // We take one with minimum number of candidates
08612     //
08613     // There may be better method.
08614
08615     int v0, v, nl;
08616
08617     // select vertex one by one in the sequential order of node number
08618     v0 = -1;
08619     nl = 0;
08620     for (v = 0; v < nNodes; v++) {
08621         if (nodes[v]->cand->st == AS_UnAssLegs) {
08622             if (nodes[v]->cand->nilist == 1) {
08623                 return v;
08624             } else if (nodes[v]->cand->nilist > 1) {
08625                 // select node with fewer candidates
08626                 if (v0 < 0 || nodes[v]->cand->nilist < nl) {
08627                     v0 = v;
08628                     nl = nodes[v]->cand->nilist;
08629                 }
08630             }
08631         }
08632     }
08633     return v0;
08634 }
08635
08636 //-----
08637 int Assign::selUnAssLeg(int v, int lastlg)
08638 {
08639     // Select a leg of vertex 'v' for the assignment.
08640     // The lastly selected leg was 'lastlg'.
08641
08642     int lg0, lg, e;
08643 #ifdef CHECK
08644     int n0, n1;
08645 #endif
08646
08647     // lg0 = Max(lastlg, lastlg + 1);
08648     lg0 = lastlg + 1;
08649

```

```

08650     for (lg = lg0; lg < nodes[v]->deg; lg++) {
08651
08652 #ifdef CHECK
08653     if (lastlg >= 0) {
08654         n0 = nodes[v]->anodes[lastlg];
08655     } else {
08656         n0 = -1;
08657     }
08658     n1 = nodes[v]->anodes[lg];
08659     if (n0 > n1) {
08660         printf("*** selUnAssLeg: n0=%d > n1=%d\n", n0, n1);
08661         printf("*** illegal connection\n");
08662         erEnd("selUnAssLeg: n0 > n1");
08663     }
08664 #endif
08665
08666     e = nodes[v]->aedges[lg];
08667     if (!edges[e]->cand->det) {
08668         return lg;
08669     }
08670
08671 }
08672 return -1;
08673 }
08674
08675 //-----
08676 NCandSt Assign::assignIVertex(int v, int ia)
08677 {
08678     // Try to assign interaction 'ia' to 'v'
08679
08680     int lplist[GRCC_MAXLEGS], iplist[GRCC_MAXLEGS];
08681     int lg, e, pt, deg;
08682
08683
08684     if (nodes[v]->cand->st == AS_Assigned || nodes[v]->cand->st == AS_AssExt) {
08685         return AS_Assigned0;
08686     }
08687
08688     deg = nodes[v]->deg;
08689     if (deg != model->interacts[ia]->nlegs) {
08690         return AS_Impossible;
08691     }
08692
08693     // list of particles at the legs of the vertex
08694     for (lg = 0; lg < deg; lg++) {
08695         e = nodes[v]->aedges[lg];
08696         if (edges[e]->cand->nplist != 1) {
08697             return AS_UnAssLegs;
08698         } else {
08699             pt = edges[e]->cand->plist[0];
08700             lplist[lg] = legEdgeParticle(v, lg, pt);
08701         }
08702         iplist[lg] = model->interacts[ia]->plist[lg];
08703     }
08704
08705     sorti(deg, lplist);
08706     sorti(deg, iplist);
08707
08708     // from interaction
08709     if (cmpArray(deg, lplist, deg, iplist) == 0) {
08710         // orders of coupling constants should be checked ???
08711         astack->pushNode(v);
08712         nodes[v]->cand->st = AS_Assigned;
08713         nodes[v]->cand->nlist = 1;
08714         nodes[v]->cand->ilist[0] = ia;
08715         return AS_Assigned;
08716     }
08717
08718     return AS_Impossible;
08719 }
08720
08721 //-----
08722 Bool Assign::assignPLeg(int n, int ln, int pt)
08723 {
08724     // A particle 'pt' is assigned to leg 'ln' of node 'n'
08725
08726     ANode *nd;
08727     int e, ept;
08728     Bool ok;
08729
08730 #ifdef CHECK
08731     checkNode(n, "assignPLeg0");
08732     checkCand("assignPLeg");
08733 #endif
08734
08735     // nodes and the edge
08736     nd = nodes[n];

```

```

08737     e   = nd->aedges[ln];
08738
08739     // already determined
08740     if (edges[e]->cand->det) {
08741         return True;
08742     }
08743
08744     if (!isOrdPLeg(n, ln, pt)) {
08745 #ifdef DEBUGM
08746         nopleg++;
08747 #endif
08748         return False;
08749     }
08750
08751     // particle on the edge
08752     ept = legEdgeParticle(n, ln, pt);
08753
08754 #ifdef CHECK
08755     if (!isIn(edges[e]->cand->nplist, edges[e]->cand->pclist, ept)) {
08756         printf("*** assignPLeg: particle %d is not in the cand. of e=%d",
08757             ept, e);
08758         edges[e]->cand->prECand("\n");
08759         prCand("assignPLeg");
08760         erEnd("assignPLeg: particle is not in the cand.");
08761     }
08762 #endif
08763
08764     // save the current configuration
08765     astack->pushEdge(e);
08766
08767     // determine the particle on the edge
08768     edges[e]->cand->nplist = 1;
08769     edges[e]->cand->pclist[0] = ept;
08770
08771     ok = detEdge(e);
08772
08773     return ok;
08774 }
08775
08776 //-----
08777 Bool Assign::detEdge(int e)
08778 {
08779     Bool ok0, ok1;
08780
08781     if (edges[e]->cand->nplist != 1) {
08782         return False;
08783     }
08784     edges[e]->cand->det = True;
08785
08786     // update candidate list
08787     ok0 = updateCandNode(edges[e]->nodes[0]);
08788     if (ok0) {
08789         ok1 = updateCandNode(edges[e]->nodes[1]);
08790     } else {
08791         ok1 = False;
08792     }
08793
08794     return (ok0 && ok1);
08795 }
08796
08797 //-----
08798 Bool Assign::isOrdPLeg(int n, int ln, int pt)
08799 {
08800     int e0, e1, ln0, ep0, pt0, nn, n0, n1, ln1, pt1, e1, ep1;
08801
08802     if (nodes[n]->deg < 2) {
08803         return True;
08804     }
08805     nn = nodes[n]->anodes[ln];
08806     if (n <= nn) {
08807         n0 = n;
08808         n1 = nn;
08809         ln0 = ln;
08810         pt0 = pt;
08811     } else {
08812         n0 = nn;
08813         n1 = n;
08814         e0 = nodes[n]->aedges[ln];
08815         e1 = nodes[n]->aelegs[ln];
08816         ln0 = edges[e0]->nlegs[1-e1];
08817         ep0 = legEdgeParticle(n, ln, pt);
08818         pt0 = legEdgeParticle(n0, ln0, ep0);
08819     }
08820
08821     for (ln1 = 0; ln1 < nodes[n0]->deg; ln1++) {
08822         if (ln1 == ln0 || n1 != nodes[n0]->anodes[ln1]) {
08823             continue;

```

```

08824     }
08825     e1 = nodes[n0]->aedges[ln1];
08826     if (!edges[e1]->cand->det) {
08827         continue;
08828     }
08829     ep1 = edges[e1]->cand->plist[0];
08830     pt1 = legEdgeParticle(n0, ln1, ep1);
08831     if ((ln0 < ln1 && pt0 > pt1) ||
08832         (ln0 > ln1 && pt0 < pt1)) {
08833         return False;
08834     }
08835 }
08836 return True;
08837 }
08838
08839 //=====
08840 // Operations of candidate
08841 //-----
08842 Bool Assign::candPartClassify(int v, int *npdass, int *pdass, int *npuass, int *puass, const int size)
08843 {
08844     // Construct the classified lists of particles possible to assign to
08845     // the legs of node v
08846     //
08847     // pdass = (duplicated set of particles where edges has a unique
08848     // candidate)
08849     // puass = (duplicated set of other candidate particles)
08850
08851     int lg, e, npl, ass, jd, ju, j, pt, pts;
08852
08853     jd = 0;
08854     ju = 0;
08855     for (lg = 0; lg < nodes[v]->deg; lg++) {
08856         e = nodes[v]->aedges[lg];
08857         npl = edges[e]->cand->nplist;
08858         if (npl < 1) {
08859             return False;
08860         }
08861         ass = (npl == 1);
08862         for (j = 0; j < edges[e]->cand->nplist; j++) {
08863             pt = edges[e]->cand->plist[j];
08864             pts = legEdgeParticle(v, lg, pt);
08865             if (ass) {
08866                 jd = intSListAdd(jd, pdass, pts, size);
08867             } else {
08868                 ju = intSListAdd(ju, puass, pts, size);
08869             }
08870         }
08871     }
08872     *npdass = jd;
08873     *npuass = ju;
08874
08875     return True;
08876 }
08877
08878 //-----
08879 Bool Assign::updateCandNode(int v)
08880 {
08881     // Update the configuration
08882     // - nodes[v]->cand->ilist
08883     // - edges[e]->cand->plist for the adjacent edge
08884     // - nodes[n]->cand->unass for the adjacent node 'n' of 'e'
08885
08886     int npdass, pdass[GRCC_MAXPSLIST];
08887     int npuass, puass[GRCC_MAXPSLIST];
08888     int nsub0, sub0[GRCC_MAXPSLIST];
08889     int niplist, iplist[GRCC_MAXPSLIST];
08890     int nd0, d0[GRCC_MAXMPARTICLES2];
08891     int nd1, d1[GRCC_MAXMPARTICLES2];
08892     int ns0, s0[GRCC_MAXMPARTICLES2];
08893     int neplist, eplist[GRCC_MAXMPARTICLES2];
08894     int nitlist, itlist[GRCC_MAXMINTERACT];
08895     int e, it, j, k, i, lg;
08896
08897     // external node
08898     if (nodes[v]->cand->st == AS_AssExt) {
08899 #ifdef CHECK
08900         e = nodes[v]->aedges[0];
08901         if (e < 0 || edges[e]->cand->nplist != 1) {
08902             printf("*** illegal external node: v=%d e=%d :", v, e);
08903             prCand("updateCandNode");
08904             erEnd("illegal external node");
08905         }
08906 #endif
08907
08908         return True;
08909     }
08910 }

```

```

08911 // impossible configuration.
08912 if (nodes[v]->cand->nilist < 1) {
08913     return False;
08914 }
08915
08916 // list of possible particles on the edges of the vertex.
08917 // pdass = (set of assigned particles)
08918 // puass = (list of particles of edges with two of more candidates)
08919
08920 niplist = 0;
08921 nitlist = 0;
08922 if (!candPartClassify(v, &npdass, pdass, &npuass, puass, GRCC_MAXPSLIST)) {
08923     return False;
08924 }
08925
08926 if (npdass < 1) {
08927     return False;
08928 }
08929
08930 // from interaction : slist
08931 // Conditions :
08932 // 1. pdass \subset slist
08933 // 2. slist-pdass \subset puass
08934 // from interaction : slist
08935 // conditions : pdass \subset slist \subset (pdass + puass)
08936 // sub0 = slist - pdass,
08937 nitlist = 0;
08938 for (j = 0; j < nodes[v]->cand->nilist; j++) {
08939     it = nodes[v]->cand->ilist[j];
08940     // conditions:
08941     // 1. pdass \subset slist <==> pdass-slist = \emptyset
08942     // 2. slist-pdass \subset puass <==> (slist-pdass)-puass = \emptyset
08943     if (isSubSList(npdass, pdass,
08944         model->interacts[it]->nslist,
08945         model->interacts[it]->slist) ) {
08946         nsub0 = subtrSet(model->interacts[it]->nslist,
08947             model->interacts[it]->slist,
08948             npdass, pdass, sub0, GRCC_MAXPSLIST);
08949         if (nsub0 < 1) {
08950             // slist == pdass : no additional candidate particle
08951             nitlist = intSetAdd(nitlist, itlist, it, GRCC_MAXMINTERACT);
08952         } else {
08953             if (isSubSList(nsub0, sub0, npuass, puass)) {
08954                 nitlist = intSetAdd(nitlist, itlist, it, GRCC_MAXMINTERACT);
08955                 for (k = 0; k < nsub0; k++) {
08956                     niplist = intSetAdd(niplist, iplist, sub0[k], GRCC_MAXPSLIST);
08957                 }
08958             }
08959         }
08960     }
08961 }
08962 }
08963
08964 if (nitlist < 1) {
08965     return False;
08966 }
08967
08968 if (nitlist != nodes[v]->cand->nilist) {
08969     astack->pushNode(v);
08970     nodes[v]->cand->nilist = nitlist;
08971     for (i = 0; i < nitlist; i++) {
08972         nodes[v]->cand->ilist[i] = itlist[i];
08973     }
08974 #ifdef CHECK
08975 } else if (nitlist > nodes[v]->cand->nilist) {
08976     erEnd("larger ncand");
08977 #endif
08978 }
08979
08980 // edge
08981 for (lg = 0; lg < nodes[v]->deg; lg++) {
08982     e = nodes[v]->aedges[lg];
08983     if (edges[e]->cand->nplist > 1) {
08984         // elist = {candidate particles in the direction of the edge}
08985         // s0 = plist \cap elist
08986         neplist = legPart(v, lg, niplist, iplist, eplist, GRCC_MAXMPARTICLES2);
08987         listDiff(edges[e]->cand->nplist, edges[e]->cand->plist,
08988             neplist, eplist,
08989             &nd0, d0, &ns0, s0, &nd1, d1);
08990
08991         if (ns0 < 1) {
08992             return False;
08993         }
08994         if (ns0 < edges[e]->cand->nplist) {
08995             astack->pushEdge(e);
08996             edges[e]->cand->nplist = ns0;
08997             for (i = 0; i < ns0; i++) {

```



```

08998         edges[e]->cand->plist[i] = s0[i];
08999     }
09000 }
09001 }
09002 }
09003 for (lg = 0; lg < nodes[v]->deg; lg++) {
09004     e = nodes[v]->aedges[lg];
09005     if (edges[e]->cand->nplist == 1 && !edges[e]->cand->det) {
09006         if(!detEdge(e)) {
09007             return False;
09008         }
09009     }
09010 }
09011 }
09012 return True;
09013 }
09014
09015 //=====
09016 // Check order of coupling constants, duplication and isomorphism
09017 //-----
09018 Bool Assign::checkOrderCpl(void)
09019 {
09020     // Check order of coupling constants
09021
09022     int ord[GRCC_MAXNCPLG];
09023     int lcn, n, ia, j, cl;
09024
09025     // list of coupling constants
09026     lcn = model->ncouple;
09027
09028     if (lcn == 1) {
09029         return True;
09030     }
09031
09032     // sum up for each coupling constants
09033     for (j = 0; j < GRCC_MAXNCPLG; j++) {
09034         ord[j] = 0;
09035     }
09036     for (n = 0; n < nNodes; n++) {
09037         cl = pnclass->nd2cl[n];
09038         if (!isATEExternal(pnclass->type[cl])) {
09039             ia = nodes[n]->cand->ilist[0];
09040             for (j = 0; j < lcn; j++) {
09041                 ord[j] += model->interacts[ia]->clist[j];
09042             }
09043         }
09044     }
09045
09046     // comparison
09047     for (j = 0; j < lcn; j++) {
09048         if (sproc->clist[j] != ord[j]) {
09049             return False;
09050         }
09051     }
09052     return True;
09053 }
09054
09055 //-----
09056 Bool Assign::isOrdLegs(void)
09057 {
09058     // Whether graph is duplicated because of the violation
09059     // of ordering in the case of multiple connections
09060     // v0 <-- v --> v1
09061
09062     int v, l0, v0, l1, v1, e0, e1, q0, q1, p0, p1, cl;
09063
09064     for (v = 0; v < nNodes; v++) {
09065         cl = pnclass->nd2cl[v];
09066         if (!isATEExternal(pnclass->type[cl])) {
09067             for (l0 = 0; l0 < nodes[v]->deg; l0++) {
09068                 v0 = nodes[v]->anodes[l0];
09069                 for (l1 = l0+1; l1 < nodes[v]->deg; l1++) {
09070                     v1 = nodes[v]->anodes[l1];
09071
09072                     if (v0 == v1 && v <= v0) {
09073                         // multiple connections (v, l0) and (v, l1)
09074                         e0 = nodes[v]->aedges[l0];
09075                         e1 = nodes[v]->aedges[l1];
09076                         q0 = edges[e0]->cand->plist[0];
09077                         q1 = edges[e1]->cand->plist[0];
09078                         p0 = legEdgeParticle(v, l0, q0);
09079                         p1 = legEdgeParticle(v, l1, q1);
09080                         if (p0 > p1) {
09081                             // violation of ordering
09082                             return False;
09083                         }
09084                     }

```

```

09085     }
09086     }
09087     }
09088     }
09089
09090     return True;
09091 }
09092
09093 //-----
09094 Bool Assign::isIsomorphic(MNodeClass *cl, BigInt *nsym, BigInt *esym, BigInt *nsym1)
09095 {
09096     // Check whether the current graph is the representative of
09097     // a isomorphic class.
09098     // Returns (nsym, esym)
09099     // nsym = symmetry factor by the permutation of nodes.
09100     // esym = symmetry factor by the permutation of edge.
09101     // If this graph is not a representative, then returns (0,0).
09102
09103     int j, cmp, n, cln;
09104     BigInt ngelem;
09105     int *p;
09106     Bool invext;
09107
09108     ngelem = mgraph->group->nElem();
09109 #ifdef CHECK
09110     if (mgraph->nsym > 1 && ngelem <= 1) {
09111         printf("*** isIsomorphic: illegal group: "
09112             "ngelem=%ld, mgraph->sym=(%ld, %ld)\n",
09113             ngelem, mgraph->nsym, mgraph->esym);
09114         erEnd("Assign::isIsomorphic: illegal group");
09115     }
09116 #endif
09117     if (ngelem == 0) {
09118         *nsym = 1;
09119         *nsym1 = 1;
09120     } else {
09121         *nsym = 0;
09122         *nsym1 = 0;
09123         for (j = 0; j < ngelem; j++) {
09124             // check the graph is the representative and count 'nsym'.
09125
09126             //p = mgraph->group->nextElem();
09127             p = mgraph->group->elem[j];
09128
09129             cmp = cmpPermGraph(p, cl);
09130
09131             if (cmp < 0) { // duplicated graph
09132 #ifdef DEBUGM
09133                 niso1++;
09134 #endif
09135                 return False;
09136             } else if (cmp == 0) { // not duplicated
09137                 (*nsym)++;
09138
09139                 if (opt->values[GRCC_OPT_SymmInitial] || opt->values[GRCC_OPT_SymmFinal]) {
09140                     invext = True;
09141                     for (n = 0; n < nNodes; n++) {
09142                         cln = pnclass->nd2cl[n];
09143                         if (!isATExternal(pnclass->type[cln])) {
09144                             continue;
09145                         }
09146                         if (p[n] != n) {
09147                             invext = False;
09148                         }
09149                     }
09150                     if (invext) {
09151                         (*nsym1)++;
09152                     }
09153                 } else {
09154                     (*nsym1)++;
09155                 }
09156             }
09157         }
09158     }
09159     if (*nsym1 == 0) {
09160         *nsym1 = *nsym;
09161     }
09162
09163     // calculate permutations of edges
09164     *esym = edgeSym();
09165     if (*esym < 1) {
09166 #ifdef DEBUGM
09167         niso2++;
09168 #endif
09169         return False;
09170     }
09171 }

```

```

09172     return True;
09173 }
09174
09175 //-----
09176 int Assign::cmpPermGraph(int *p, MNodeClass *cl)
09177 {
09178     // compare the graph with one permuted by 'p'.
09179
09180     int n, cmp, n1, n2, p1, p2, j;
09181     ANode *nd1, *np1;
09182     // ANode *nd2, *np2;
09183     int njn, jn[GRCC_MAXLEGS];
09184     int njp, jp[GRCC_MAXLEGS];
09185
09186 #ifdef CHECK
09187     if (p == NULL) {
09188         erEnd("Assign::cmpPermGraph: p=NULL");
09189     }
09190 #endif
09191 #ifdef DEBUG1
09192     printf("cmpPermGraph:0: p=");
09193     prIntArray(nNodes, p, "\n");
09194 #endif
09195     for (n = 0; n < nNodes; n++) {
09196         if (!isATEXternal(pnclass->type[pnclass->nd2cl[n]])) {
09197             cmp = cmpNodes(n, p[n], cl);
09198             if (cmp != 0) {
09199 #ifdef DEBUGM
09200                 if (cmp < 0) { nisol1++; }
09201 #endif
09202                 return cmp;
09203             }
09204         }
09205     }
09206
09207     njn = 0;
09208     njp = 0;
09209     for (n1 = 0; n1 < nNodes; n1++) {
09210         p1 = p[n1];
09211         nd1 = nodes[n1];
09212         np1 = nodes[p1];
09213         for (n2 = n1; n2 < nNodes; n2++) {
09214             if (mgraph->adjMat[n1][n2] == 0) {
09215                 continue;
09216             }
09217
09218             p2 = p[n2];
09219             if (p1 == n1 && p2 == n2) {
09220                 continue;
09221             }
09222             cmp = mgraph->adjMat[n1][n2] - mgraph->adjMat[p1][p2];
09223             if (cmp != 0) {
09224 #ifdef DEBUGM
09225                 if (cmp < 0) { nisol2++; }
09226 #endif
09227                 return cmp;
09228             }
09229
09230             njn = 0;
09231             njp = 0;
09232             for (j = 0; j < nd1->deg; j++) {
09233                 if (nd1->anodes[j] == n2) {
09234                     jn[njn++] = getLegParticle(n1, j);
09235                 }
09236             }
09237
09238             for (j = 0; j < np1->deg; j++) {
09239                 if (np1->anodes[j] == p2) {
09240                     jp[njp++] = getLegParticle(p1, j);
09241                 }
09242             }
09243
09244             cmp = njn - njp;
09245             if (cmp != 0) {
09246 #ifdef DEBUGM
09247                 if (cmp < 0) { nisol3++; }
09248 #endif
09249                 return cmp;
09250             }
09251
09252             // ignore ordering for multiple connections
09253             if (mgraph->adjMat[n1][n2] >= 2) {
09254                 sorti(njn, jn);
09255                 sorti(njp, jp);
09256             }
09257
09258             for (j = 0; j < njn; j++) {

```

```

09259             cmp = jn[j] - jp[j];
09260             if (cmp != 0) {
09261 #ifdef DEBUGM
09262                 if (cmp < 0) { niso14++; }
09263 #endif
09264                 return cmp;
09265             }
09266         }
09267     }
09268 }
09269
09270 return 0;
09271 }
09272
09273 //-----
09274 int Assign::cmpNodes(int nd0, int nd1, MNodeClass *cn)
09275 {
09276     // Comparison of two nodes 'nd0' and 'nd1'
09277     // Ordering is lexicographical (class, connection configuration)
09278     //
09279
09280     int cmp;
09281
09282     // Wether two nodes are in a same class or not.
09283     cmp = cn->ndcl[nd0] - cn->ndcl[nd1];
09284     if (cmp != 0) {
09285 #ifdef DEBUGM
09286         if (cmp < 0) { niso111++; }
09287 #endif
09288         return cmp;
09289     }
09290
09291     // interaction
09292     cmp = nodes[nd0]->cand->ilist[0] - nodes[nd1]->cand->ilist[0];
09293 #ifdef DEBUGM
09294     if (cmp < 0) { niso112++; }
09295 #endif
09296     return cmp;
09297 }
09298
09299 //-----
09300 BigInt Assign::edgeSym(void)
09301 {
09302     // calculate permutations of edges
09303
09304     int lg[GRCC_MAXLEGS], lt[GRCC_MAXLEGS], lc;
09305     ANode *nd1;
09306     int n1, n2, j, k, mult;
09307     BigInt esym;
09308
09309     esym = 1;
09310     for (n1 = 0; n1 < nNodes; n1++) {
09311         nd1 = nodes[n1];
09312         for (n2 = n1; n2 < nNodes; n2++) {
09313             if (mgraph->adjMat[n1][n2] > 1) {
09314                 lc = 0;
09315                 for (j = 0; j < nd1->deg; j++) {
09316                     if (nd1->anodes[j] == n2) {
09317                         lg[lc] = 1;
09318                         lt[lc] = getLegParticle(n1, j);
09319                         lc++;
09320                     }
09321                 }
09322                 for (j = 0; j < lc-1; j++) {
09323                     if (lg[j] > 0) {
09324                         for (k = lc-1; k > j; k--) {
09325                             if (lt[j] == lt[k]) {
09326                                 lg[j] += lg[k];
09327                                 lg[k] = 0;
09328                             }
09329                         }
09330                     }
09331                 }
09332
09333                 // calculate symmetry factor
09334                 for (j = 0; j < lc; j++) {
09335                     if (lg[j] < 1) {
09336                         continue;
09337                     }
09338                     mult = lg[j];
09339                     if (mult > 1) {
09340                         if (n1 == n2) {
09341                             // self-loop
09342                             esym *= ipow(2, mult/2) * factorial(mult/2);
09343                         } else {
09344                             esym *= factorial(mult);
09345                         }
09346                     }
09347                 }
09348             }
09349         }
09350     }

```

```

09346         }
09347     }
09348     }
09349 }
09350 }
09351
09352     return esym;
09353 }
09354
09355 #ifdef CHECK
09356 //=====
09357 // check
09358 //-----
09359 Bool Assign::checkCand(const char *msg)
09360 {
09361     // Check the consistency of Candidate data
09362
09363     Bool ok;
09364     ANode *na;
09365     NCand *nc;
09366     ECand *ec;
09367     int n, lg, e;
09368
09369     // check 'nodes->cand'
09370     ok = True;
09371     for (n = 0; n < nNodes; n++) {
09372         na = nodes[n];
09373         nc = na->cand;
09374
09375         // check unassigned vertex
09376         if (nc->st == AS_UnAssLegs) {
09377
09378             // check assigned vertex
09379             } else if (nc->st == AS_Assigned) {
09380                 if (nc->nilist < 1) {
09381                     printf("*** checkCand:7:%s:status (%d) of node %d says"
09382                            " interaction is assigned to %d but ilist=",
09383                            msg, nc->st, n, nc->st);
09384                     prIntArray(nc->nilist, nc->ilist, "\n");
09385                     ok = False;
09386                 }
09387
09388                 for (lg = 0; lg < nc->deg; lg++) {
09389                     e = na->aedges[lg];
09390                     ec = edges[e]->cand;
09391                     if (ec->nplist != 1) {
09392                         printf("*** checkCand:8:%s:status (%d) of node %d says"
09393                                " interaction is assigned "
09394                                " but unassigned edge %d is found\n",
09395                                msg, nc->st, n, e);
09396                         ok = False;
09397                     }
09398                 }
09399
09400                 // external particle
09401             } else if (nc->st == AS_AssExt) {
09402                 // pt = nc->ilist[0];
09403                 // *** pte = legEdgeParticle(n, 0, - pt);
09404             } else {
09405                 printf("*** checkCand:10:%s:illegal status of node %d : %d",
09406                        msg, n, nc->st);
09407                 ok = False;
09408             }
09409         }
09410
09411         // check edges
09412
09413         for (e = 0; e < nEdges; e++) {
09414             ec = edges[e]->cand;
09415             if (ec != NULL && ec->nplist < 1) {
09416                 printf("*** checkCand:12:%s:illegal edge %d\n", msg, e);
09417                 ok = False;
09418             }
09419         }
09420
09421         if (!ok) {
09422             printf("*** checkCand:15:%s:illegal configuration\n", msg);
09423             prCand("checkCand");
09424             printf("*** checkCand:16:illegal configuration\n");
09425             erEnd("checkCand:16:illegal configuration");
09426         }
09427         return ok;
09428     }
09429
09430 //-----
09431 void Assign::checkNode(int n, const char *msg)
09432 {

```

```

09433     int j, it;
09434
09435     for (j = 0; j < nodes[n]->cand->nilist; j++) {
09436         it = nodes[n]->cand->ilist[j];
09437         if (Abs(it) >= GRCC_MAXMINTERACT) {
09438             printf("*** %s: n=%d, j=%d, it=%d\n", msg, n, j, it);
09439             nodes[n]->cand->prNCand(msg);
09440             printf("\n");
09441             erEnd("checkNode:illegal it");
09442         }
09443     }
09444 }
09445 #endif // CHECK
09446
09447 //*****
09448 // astack.cc
09449 //=====
09450 // Assign particles to the edges and interactions to nodes.
09451 // Completed graph data is saved in the form of EGraph.
09452
09453 //=====
09454 // stack operations
09455 //-----
09456 void NStack::print(const char *msg)
09457 {
09458     printf(" node=%d, deg=%d, st=%d, ilist=", noden, deg, st);
09459     prillist(nilist, ilist, msg);
09460 }
09461
09462 //-----
09463 void EStack::print(const char *msg)
09464 {
09465     printf(" edge=%d, det=%d, plist=", edgen, det);
09466     prillist(nplist, plist, msg);
09467 }
09468
09469 //-----
09470 AStack::AStack(int nsize, int esize)
09471 {
09472     int j;
09473
09474     agraph = NULL;
09475
09476     nSize = nsize;
09477     eSize = esize;
09478     if (nSize > 0) {
09479         nStack = new NStack*[nSize];
09480     } else {
09481         nStack = NULL;
09482     }
09483     if (eSize > 0) {
09484         eStack = new EStack*[eSize];
09485     } else {
09486         eStack = NULL;
09487     }
09488
09489     nStackP = 0;
09490     eStackP = 0;
09491
09492     for (j = 0; j < nSize; j++) {
09493         nStack[j] = new NStack();
09494     }
09495
09496     for (j = 0; j < eSize; j++) {
09497         eStack[j] = new EStack();
09498     }
09499 }
09500
09501 //-----
09502 AStack::~AStack(void)
09503 {
09504     int j;
09505
09506     if (eStack != NULL) {
09507         for (j = eSize-1; j >= 0; j--) {
09508             if (eStack[j] != NULL) {
09509                 delete eStack[j];
09510                 eStack[j] = NULL;
09511             }
09512         }
09513         delete[] eStack;
09514         eStack = NULL;
09515     }
09516
09517     if (nStack != NULL) {
09518         for (j = nSize-1; j >= 0; j--) {
09519             if (nStack[j] != NULL) {

```

```

09520             delete nStack[j];
09521             nStack[j] = NULL;
09522         }
09523     }
09524     delete[] nStack;
09525     nStack = NULL;
09526 }
09527 }
09528
09529 //-----
09530 void AStack::setAGraph(Assign *ag)
09531 {
09532     agraph = ag;
09533 }
09534
09535 //-----
09536 void AStack::pushNode(int n)
09537 {
09538     NCand *nc;
09539     NStack *ns;
09540     int j;
09541
09542     #ifdef CHECK
09543     if (agraph == NULL) {
09544         erEnd("pushNode: agraph is not defined");
09545     }
09546     #endif
09547     if (nStackP >= GRCC_MAXNSTACK) {
09548         erEnd("N-stack overflow (GRCC_MAXNSTACK)");
09549     }
09550     nc = agraph->nodes[n]->cand;
09551     if (nc->nlist >= GRCC_MAXMINTERACT) {
09552         erEnd("N-stack: too long list (GRCC_MAXMINTERACT)");
09553     }
09554     ns = nStack[nStackP];
09555     ns->noden = n;
09556     ns->deg = nc->deg;
09557     ns->st = nc->st;
09558     ns->nlist = nc->nlist;
09559     for (j = 0; j < nc->nlist; j++) {
09560         ns->ilist[j] = nc->ilist[j];
09561     }
09562     nStackP++;
09563 }
09564
09565 //-----
09566 void AStack::pushEdge(int e)
09567 {
09568     ECand *ec;
09569     EStack *es;
09570     int j;
09571
09572     #ifdef CHECK
09573     if (agraph == NULL) {
09574         erEnd("pushEdge: agraph is not defined");
09575     }
09576     #endif
09577     if (eStackP >= GRCC_MAXESTACK) {
09578         erEnd("E-stack overflow (GRCC_MAXESTACK)");
09579     }
09580     ec = agraph->edges[e]->cand;
09581     if (ec->nplist >= GRCC_MAXMPARTICLES2) {
09582         erEnd("E-stack: Too long list (GRCC_MAXMPARTICLES)");
09583     }
09584     es = eStack[eStackP];
09585     es->edgen = e;
09586     es->det = ec->det;
09587     es->nplist = ec->nplist;
09588     for (j = 0; j < ec->nplist; j++) {
09589         es->plist[j] = ec->plist[j];
09590     }
09591     eStackP++;
09592 }
09593
09594 //-----
09595 void AStack::checkPoint(CheckPt sav)
09596 {
09597     sav[0] = nStackP;
09598     sav[1] = eStackP;
09599 }
09600
09601 //-----
09602 void AStack::restoreNode(int spr)
09603 {
09604     NStack *stc;
09605     int sp, n, j;
09606 }

```

```

09607     for (sp = nStackP-1; sp >= spr; sp--) {
09608         stc = nStack[sp];
09609         n = stc->noden;
09610         agraph->nodes[n]->cand->deg      = stc->deg;
09611         agraph->nodes[n]->cand->st      = stc->st;
09612         agraph->nodes[n]->cand->nilist  = stc->nilist;
09613         for (j = 0; j < stc->nilist; j++) {
09614             agraph->nodes[stc->noden]->cand->ilist[j] = stc->ilist[j];
09615         }
09616     }
09617     nStackP = spr;
09618 }
09619
09620 //-----
09621 void AStack::restoreEdge(int spr)
09622 {
09623     EStack *stc;
09624     int     sp, e, j;
09625
09626     for (sp = eStackP-1; sp >= spr; sp--) {
09627         stc = eStack[sp];
09628         e   = stc->edgen;
09629         agraph->edges[e]->cand->det = stc->det;
09630         agraph->edges[e]->cand->nplist = stc->nplist;
09631         for (j = 0; j < stc->nplist; j++) {
09632             agraph->edges[e]->cand->plist[j] = stc->plist[j];
09633         }
09634     }
09635     eStackP = spr;
09636 }
09637
09638 //-----
09639 void AStack::restore(CheckPt sav)
09640 {
09641     restoreNode(sav[0]);
09642     restoreEdge(sav[1]);
09643 }
09644
09645 #ifdef CHECK
09646 //-----
09647 void AStack::restoreMsg(CheckPt sav, const char *msg)
09648 {
09649     if (agraph == NULL) {
09650         erEnd("restore: agraph is not defined");
09651     }
09652
09653     restore(sav);
09654
09655     if (!agraph->checkCand("restore")) {
09656         printf("restore is called from %s\n", msg);
09657     }
09658 }
09659 #endif
09660
09661 //-----
09662 void AStack::prStack(void)
09663 {
09664     int j;
09665
09666     printf("+++ prStack : (%d, %d)", nStackP, eStackP);
09667     for (j = 0; j < nStackP; j++) {
09668         printf("N:%4d ", j);
09669         nStack[j]->print("\n");
09670     }
09671     for (j = 0; j < eStackP; j++) {
09672         printf("E:%4d ", j);
09673         eStack[j]->print("\n");
09674     }
09675 }
09676
09677 //*****
09678 // class Fraction
09679 //=====
09680 Fraction::Fraction(BigInt n, BigInt d)
09681 {
09682     BigInt g;
09683
09684     g = gcd(n, d);
09685     num = n/g;
09686     den = d/g;
09687     ratio = Real(n)/Real(d);
09688 }
09689
09690 //-----
09691 void Fraction::print(const char *msg)
09692 {
09693     double err = Abs(Real(num)/Real(den) - ratio);

```



```

09694
09695     if (err > GRCC_FRACERROR) {
09696         printf("%ld/%ld(%g) (overflow)%s", num, den, ratio, msg);
09697     } else {
09698         printf("%ld/%ld(%g)%s", num, den, ratio, msg);
09699     }
09700 }
09701
09702 //-----
09703 void Fraction::setValue(BigInt n, BigInt d)
09704 {
09705     num = n;
09706     den = d;
09707     ratio = Real(n)/Real(d);
09708 }
09709
09710 //-----
09711 void Fraction::setValue(Fraction &f)
09712 {
09713     num = f.num;
09714     den = f.den;
09715     ratio = f.ratio;
09716 }
09717
09718 //-----
09719 BigInt Fraction::gcd(BigInt n0, BigInt n1)
09720 {
09721     BigInt nn[2], r;
09722     int nc;
09723
09724     nn[0] = Abs(n0);
09725     nn[1] = Abs(n1);
09726     nc = 0;
09727
09728     while (1) {
09729         r = nn[nc] % nn[1-nc];
09730         if (r == 0) {
09731             return nn[1-nc];
09732         }
09733         nn[nc] = r;
09734         nc = 1 - nc;
09735     }
09736 }
09737
09738 //-----
09739 void Fraction::normal(void)
09740 {
09741     BigInt g;
09742
09743     if (den == 0) {
09744         erEnd("Fraction: den==0");
09745     }
09746     g = gcd(num, den);
09747     num /= g;
09748     den /= g;
09749     if (den < 0) {
09750         num = - num;
09751         den = - den;
09752     }
09753 }
09754
09755 //-----
09756 void Fraction::add(BigInt n, BigInt d)
09757 {
09758     BigInt g, dl;
09759
09760     if (d < 0) {
09761         n = -n;
09762         d = -d;
09763     }
09764     if (den < 0) {
09765         num = - num;
09766         den = - den;
09767     }
09768     g = gcd(den, d);
09769     dl = d/g;
09770     num = num * dl + n * (den/g);
09771     den = dl*den;
09772     g = gcd(num, den);
09773     num /= g;
09774     den /= g;
09775     ratio += Real(n)/Real(d);
09776 }
09777
09778 //-----
09779 void Fraction::add(Fraction f)
09780 {

```

```

09781     double r = ratio + f.ratio;
09782
09783     add(f.num, f.den);
09784     ratio = r;
09785 }
09786
09787 //-----
09788 void Fraction::sub(Fraction f)
09789 {
09790     double r = ratio - f.ratio;
09791
09792     add(-f.num, f.den);
09793     ratio = r;
09794 }
09795
09796 //-----
09797 Bool Fraction::isEq(Fraction f)
09798 {
09799     return (num == f.num && den == f.den);
09800 }
09801
09802 //*****
09803 // common.cc
09804 //=====
09805 static void erEnd(const char *msg)
09806 {
09807     if (erExit != NULL) {
09808         (*erExit)(msg, erExitArg);
09809     }
09810     fprintf(GRCC_Stderr, "*** Error : %s\n", msg);
09811     GRCC_ABORT();
09812 }
09813
09814 //-----
09815 static Bool nextPart(int nelem, int nclist, int *clist, int *nl, int *r)
09816 {
09817     int rem, pn, j;
09818
09819     if (*r < 0) {
09820         *r = 0;
09821         rem = nelem;
09822         for (j = 0; j < nclist; j++) {
09823             nl[j] = rem/clist[j];
09824             rem -= nl[j]*clist[j];
09825         }
09826         if (rem == 0) {
09827             return True;
09828         }
09829     } else {
09830         rem = 0;
09831     }
09832     for (int c = 0; c < 100; c++) {
09833         rem += nl[nclist-1]*clist[nclist-1];
09834         for (pn = nclist-2; pn >= 0 && nl[pn] == 0; pn--) {
09835             ;
09836         }
09837         if (pn < 0) {
09838             return False;
09839         }
09840         rem += clist[pn];
09841         nl[pn]--;
09842         for (j = pn+1; j < nclist; j++) {
09843             nl[j] = rem/clist[j];
09844             rem -= nl[j]*clist[j];
09845         }
09846         if (rem == 0) {
09847             return True;
09848         }
09849     }
09850     printf("*** nextPart : too many repetition\n");
09851     return False;
09852 }
09853
09854 //-----
09855 static int *intdup(int n, int *a)
09856 {
09857     int *r, j;
09858
09859     r = new int[n];
09860     for (j = 0; j < n; j++) {
09861         r[j] = a[j];
09862     }
09863     return r;
09864 }
09865
09866 //-----
09867 static int *delintdup(int *a)

```

```

09868 {
09869     if (a != NULL) {
09870         delete[] a;
09871     }
09872     return NULL;
09873 }
09874
09875 //-----
09876 static void prillist(int n, const int *a, const char *msg)
09877 {
09878     printf("[");
09879     for (int j = 0; j < n; j++) {
09880         if (j!=0) printf(", ");
09881         printf("%d", a[j]);
09882     }
09883     printf("] %s", msg);
09884 }
09885
09886 //-----
09887 static int nextPerm(int nelem, int nclass, int *cl, int *r, int *q, int *p, int count)
09888 {
09889     // Sequential generation of all permutations.
09890
09891     int j, k, n, e, t;
09892     Bool b;
09893
09894     for (j = 0; j < nelem; j++) {
09895         p[j] = j;
09896     }
09897     if (count < 1) {
09898         for (j = 0; j < nelem; j++) {
09899             q[j] = 0;
09900             r[j] = 0;
09901         }
09902         j = 0;
09903         for (k = 0; k < nclass; k++) {
09904             n = cl[k];
09905             for (e = 0; e < n; e++) {
09906                 r[j] = n - e - 1;
09907                 j++;
09908             }
09909         }
09910 #ifdef CHECK
09911         if (j != nelem) {
09912             erEnd("inconsistent # elements");
09913         }
09914 #endif
09915         return 1;
09916     }
09917     b = False;
09918     for (j = nelem-1; j >= 0; j--) {
09919         if (q[j] < r[j]) {
09920             for (k = j+1; k < nelem; k++) {
09921                 q[k] = 0;
09922             }
09923             q[j]++;
09924             b = True;
09925             break;
09926         }
09927     }
09928     if (!b) {
09929         return (-count);
09930     }
09931
09932     for (j = 0; j < nelem; j++) {
09933         k = j + q[j];
09934         t = p[j];
09935         p[j] = p[k];
09936         p[k] = t;
09937     }
09938     return count + 1;
09939 }
09940
09941 //-----
09942 static BigInt factorial(int n)
09943 {
09944     // returns 1 for n < 1
09945
09946     int r, j;
09947
09948     r = 1;
09949     for (j = 2; j <= n; j++) {
09950         r *= j;
09951     }
09952     return r;
09953 }
09954

```

```

09955 //-----
09956 static BigInt ipow(int n, int p)
09957 {
09958     int r, j;
09959
09960     r = 1;
09961     for (j = 0; j < p; j++) {
09962         r *= n;
09963     }
09964     return r;
09965 }
09966
09967 //-----
09968 static int *newArray(int size, int val)
09969 {
09970     // memory allocation of an array
09971
09972     int *a, j;
09973
09974     a = new int[size];
09975     for (j = 0; j < size; j++) {
09976         a[j] = val;
09977     }
09978     return a;
09979 }
09980
09981 //-----
09982 static int *deleteArray(int *a)
09983 {
09984     // memory allocation of an array
09985
09986     delete[] a;
09987     return NULL;
09988 }
09989
09990 //-----
09991 static int **newMat(int n0, int n1, int val)
09992 {
09993     int **m, j, k;
09994
09995     m = new int*[n0];
09996     for (j = 0; j < n0; j++) {
09997         m[j] = new int[n1];
09998         for (k = 0; k < n1; k++) {
09999             m[j][k] = val;
10000         }
10001     }
10002     return m;
10003 }
10004
10005 //-----
10006 static int **deleteMat(int **m, int n0)
10007 {
10008     int j;
10009
10010     for (j = n0-1; j >= 0; j--) {
10011         delete[] m[j];
10012     }
10013     delete[] m;
10014
10015     return NULL;
10016 }
10017
10018 //-----
10019 static void bsort(int n, int *ord, int *a)
10020 {
10021     int i, j, t;
10022
10023     for (j = n-1; j > 0; j--) {
10024         for (i = 0; i < j; i++) {
10025             if (a[ord[i+1]] < a[ord[i]]) {
10026                 t = ord[i];
10027                 ord[i] = ord[i+1];
10028                 ord[i+1] = t;
10029             }
10030         }
10031     }
10032 }
10033
10034 //-----
10035 static void prMomStr(int mom, const char *ms, int mn)
10036 {
10037     if (mom == 0) {
10038         return;
10039     } else if (mom == 1) {
10040         printf(" + %s%d", ms, mn);
10041     } else if (mom > 0) {

```

```

10042         printf(" + %d*%s%d", mom, ms, mn);
10043     } else if (mom == -1) {
10044         printf(" - %s%d", ms, mn);
10045     } else {
10046         printf(" - %d*%s%d", -mom, ms, mn);
10047     }
10048 }
10049
10050 //-----
10051 static void prIntArray(int n, int *p, const char *msg)
10052 {
10053     int j;
10054     printf("[");
10055     for (j = 0; j < n; j++) {
10056         if (j!=0) printf(", ");
10057         printf("%2d", p[j]);
10058     }
10059     printf("]%s", msg);
10060 }
10061
10062 //-----
10063 static void prIntArrayErr(int n, int *p, const char *msg)
10064 {
10065     int j;
10066     fprintf(GRCC_Stderr, "[");
10067     for (j = 0; j < n; j++) {
10068         if (j!=0) fprintf(GRCC_Stderr, ", ");
10069         fprintf(GRCC_Stderr, "%2d", p[j]);
10070     }
10071     fprintf(GRCC_Stderr, "]%s", msg);
10072 }
10073
10074 //-----
10075 static void sortrec(int l, int r, int *a)
10076 {
10077     // quick sort : taken from N. Wirth
10078     int i, j, x, w;
10079     i = l;
10080     j = r;
10081     x = a[(l+r)/2];
10082     do {
10083         while(a[i] < x) i++;
10084         while(x < a[j]) j--;
10085         if (i <= j) {
10086             w = a[i]; a[i] = a[j]; a[j] = w;
10087             i++; j--;
10088         }
10089     } while (i < j);
10090     if (l < j) sortrec(l, j, a);
10091     if (i < r) sortrec(i, r, a);
10092 }
10093
10094 //-----
10095 static void sorti(int size, int *a)
10096 {
10097     sortrec(0, size-1, a);
10098 }
10099
10100 //-----
10101 static int sortb(int n, int *a)
10102 {
10103     int i, j, t, nswap;
10104     nswap = 0;
10105     for (j = n-1; j > 0; j--) {
10106         for (i = 0; i < j; i++) {
10107             if(a[i+1] < a[i]) {
10108                 t = a[i];
10109                 a[i] = a[i+1];
10110                 a[i+1] = t;
10111                 nswap++;
10112             }
10113         }
10114     }
10115     // the sign of the permutation is (-1)^(nswap)
10116     return nswap;
10117 }
10118
10119 #ifdef CHECK
10120 //-----
10121 static Bool isIn(int n, int *a, int v)

```

```

10129 {
10130     int j;
10131
10132     for (j = 0; j < n; j++) {
10133         if (a[j] == v) {
10134             return True;
10135         }
10136     }
10137     return False;
10138 }
10139 #endif
10140
10141 //-----
10142 static int intSetAdd(int n, int *a, int v, const int size)
10143 {
10144     // Add 'v' into 'a'.
10145     // 'n' is the current # of element.
10146     // 'a' is assumed sorted without duplicated elements.
10147     // Size of 'a' should be large enough.
10148
10149     int j, k;
10150
10151     if (n >= size) {
10152         fprintf(stderr, "*** intSetAdd : array out of range (>%d)\n", size);
10153         erEnd("intSetAdd : array out of range (GCC_MAXPSLIST)");
10154     }
10155     for (j = 0; j < n; j++) {
10156         if (a[j] > v) {
10157             break;
10158         } else if (a[j] == v) {
10159             return n;
10160         }
10161     }
10162
10163     // 'j' can be equal to 'n'
10164     for (k = n-1; k >= j; k--) {
10165         a[k+1] = a[k];
10166     }
10167     a[j] = v;
10168     return n+1;
10169 }
10170
10171 //-----
10172 static int intSListAdd(int n, int *a, int v, const int size)
10173 {
10174     // Add 'v' into 'a'.
10175     // 'n' is the current # of element.
10176     // 'a' is assumed sorted without duplicated elements.
10177     // Size of 'a' should be large enough.
10178
10179     int j, k;
10180
10181     if (n >= size) {
10182         fprintf(stderr, "*** intSListAdd : array out of range (>%d)\n", size);
10183         erEnd("intSListAdd : array out of range");
10184     }
10185     for (j = 0; j < n; j++) {
10186         if (a[j] >= v) {
10187             break;
10188         }
10189     }
10190
10191     // 'j' can be equal to 'n'
10192     for (k = n-1; k >= j; k--) {
10193         a[k+1] = a[k];
10194     }
10195     a[j] = v;
10196     return n+1;
10197 }
10198
10199 //-----
10200 static int cmpArray(int na, int *a, int nb, int *b)
10201 {
10202     int j, cmp;
10203
10204     cmp = na - nb;
10205     if (cmp != 0) {
10206         return cmp;
10207     }
10208     for (j = 0; j < na; j++) {
10209         cmp = a[j] - b[j];
10210         if (cmp != 0) {
10211             return cmp;
10212         }
10213     }
10214     return 0;
10215 }

```

```

10216
10217 //-----
10218 static Bool  leqArray(int n, int *a, int *b)
10219 {
10220     int j;
10221
10222     for (j = 0; j < n; j++) {
10223         if (a[j] > b[j]) {
10224             return False;
10225         }
10226     }
10227     return True;
10228 }
10229
10230 //-----
10231 // convert list to sorted list
10232 static int  toSList(int n, int *a)
10233 {
10234     sorti(n, a);
10235     return n;
10236 }
10237
10238 //-----
10239 // as set operation assuming a and b are sorted with duplication
10240 //   p := a \ b;  q := a \cap b;  r := b \ a;
10241 // p, q, r are sorted without duplication
10242 static void  listDiff(int na, int *a, int nb, int *b, int *np, int *p, int *nq, int *q, int *nr, int
    *r)
10243 {
10244     int ja, jb;
10245
10246     *np = *nq = *nr = 0;
10247     ja = jb = 0;
10248     while (ja < na && jb < nb) {
10249         while (ja < na && a[ja] < b[jb]) {
10250             p[( *np )++] = a[ja++];
10251         }
10252         while (jb < nb && b[jb] < a[ja]) {
10253             r[( *nr )++] = b[jb++];
10254         }
10255         if (ja < na && jb < nb && a[ja] == b[jb]) {
10256             q[( *nq )++] = a[ja++];
10257             jb++;
10258         }
10259     }
10260     for (; ja < na; ja++) {
10261         p[( *np )++] = a[ja];
10262     }
10263     for (; jb < nb; jb++) {
10264         r[( *nr )++] = b[jb];
10265     }
10266 }
10267
10268 //-----
10269 static int  isSubSList(int na, int *a, int nb, int *b)
10270 {
10271     int ja, jb;
10272
10273     ja = jb = 0;
10274     while (ja < na && jb < nb) {
10275         for (; jb < nb && b[jb] < a[ja]; jb++) {
10276             ;
10277         }
10278         if (jb >= nb || b[jb] > a[ja]) {
10279             return False;
10280         }
10281         for (; jb < nb && ja < na && b[jb] == a[ja]; jb++, ja++) {
10282             ;
10283         }
10284     }
10285     return (ja >= na);
10286 }
10287
10288 //-----
10289 static int  subtrSet(int na, int *a, int nb, int *b, int *c, int size)
10290 {
10291     // set subtraction 'c' := 'a' - 'b'.
10292     // 'a' and 'b' are sorted lists (not always unique)
10293     // 'c' is the set (sorted and unique elements)
10294
10295     int ja, jb, jc;
10296
10297     jc = 0;
10298     ja = jb = 0;
10299     while (ja < na && jb < nb) {
10300         while (ja < na && a[ja] < b[jb]) {
10301             if (jc == 0 || c[jc-1] != a[ja]) {

```

```

10302         if (jc >= size) {
10303             erEnd("subsutSet: too small size of array (GRCC_MAXPSLIST)");
10304         }
10305         c[jc++] = a[ja];
10306     }
10307     ja++;
10308 }
10309 while (jb < nb && b[jb] < a[ja]) {
10310     ;
10311 }
10312 if (ja < na && jb < nb && a[ja] == b[jb]) {
10313     ja++;
10314     jb++;
10315 }
10316 }
10317 for (; ja < na; ja++) {
10318     if (jc == 0 || c[jc-1] != a[ja]) {
10319         c[jc++] = a[ja];
10320     }
10321 }
10322 return jc;
10323 }
10324
10325 // } ]

```

4.58 grcc.h

```

00001 #pragma once
00002
00003 #include <stdio.h>
00004 #include <stdlib.h>
00005 #include <string.h>
00006 #include <time.h>
00007
00008 //=====
00009 extern "C" {
00010 #include "grccparam.h"
00011 }
00012
00013 //=====
00014 // Name space of this library
00015 // See 'grccparam.h' for #define
00016 #ifndef GRCC_NAMESPACE
00017 namespace Grcc {
00018 #endif
00019
00020 //=====
00021 // For debugging
00022 //
00023 // #define MONITOR      // in graph elimination
00024 // #define CHECK        // check consistency
00025 #define DEBUGABORT
00026 // #define DEBUG
00027 // #define DEBUG0
00028 // #define DEBUG1
00029 // #define DEBUGM
00030
00031 //=====
00032 // Common types
00033 //
00034 typedef int Bool;
00035 const int True = 1;
00036 const int False = 0;
00037
00038
00039 //=====
00040 // Big integer (may be replaced by suitable one)
00041
00042 #define BigInt      long
00043 #define ToBigInt(x) ((BigInt) (x))
00044
00045 //=====
00046 // Macro functions
00047 #define Real(x)      ((double) (x))
00048 #define Abs(x)       ((x) >= 0 ? (x) : (-x))
00049 #define Sign(x)      ((x) >= 0 ? (1) : (-1))
00050 #define Max(x, y)    ((x) > (y) ? (x) : (y))
00051 #define Min(x, y)    ((x) < (y) ? (x) : (y))
00052 #define Second(t)    ((double) (*t) = (double) (clock() / ((double)CLOCKS_PER_SEC)))
00053 #define isATEternal(x) ((x) == GRCC_AT_Initial || (x) == GRCC_AT_Final || (x) == GRCC_AT_External)
00054
00055 //=====
00056 // types and classes

```



```

00057 typedef int   Edge2n[2];    // pair of nodes expressing an edge
00058 class Assign;
00059 class AStack;
00060 class EEdge;
00061 class EGraph;
00062 class ENode;
00063 class Interaction;
00064 class MGraph;
00065 class MNodeClass;
00066 class MCOpi;
00067 class MCBridge;
00068 class MCBlock;
00069 class MConn;
00070 class Model;
00071 class MOrbits;
00072 class Options;
00073 class Output;
00074 class Particle;
00075 class PNodeClass;
00076 class Process;
00077 class SGroup;
00078 class SProcess;
00079 class DCGraph;
00080
00081 //=====
00082 // type of output functions
00083 typedef Bool OutEGB(EGraph *, void *);
00084 typedef void OutEG(EGraph *, void *);
00085 typedef void ErExit(const char *msg, void *);
00086
00087 //=====
00088 class Options {
00089 public:
00090
00091     Model      *model;
00092     Process     *proc;
00093     SProcess    *sproc;
00094     Output      *out;           // for output
00095     OutEGB      *outmg;         // call back function of mgraph
00096     OutEGB      *endmg;         // call back function of end of mgraph
00097     OutEG        *outag;         // call back function of agraph
00098     void        *argmg;         // additional argument to outmg
00099     void        *argemg;        // additional argument to endmg
00100     void        *argag;         // additional argument to outag
00101
00102     //switches for graph generation
00103     int values[GRCC_OPT_Size];   // array of options
00104
00105     int          DUMMYPADDING;
00106
00107     // measuring time
00108     double time0;
00109     double time1;
00110
00111     //-----
00112     Options(void);
00113     ~Options(void);
00114     void setDefaultValue(void);
00115
00116     void setValue(int ind, int val);
00117     int  getValue(int ind);
00118
00119     void print(void);
00120
00121     void setOutputF(Bool outgrf, const char *fname);
00122     void setOutputP(Bool outgrp, const char *fname);
00123     void printLevel(int l);
00124
00125     void printModel(void);
00126     void setOutMG(OutEGB *omg, void *pt);
00127     void setOutAG(OutEG *oag, void *pt);
00128     void setEndMG(OutEGB *omg, void *pt);
00129     void setErExit(ErExit *ere, void *pt);
00130     const OptDef *getDef(void);
00131
00132     void begin(Model *mdl);
00133     void end(void);
00134     void beginProc(Process *prc);
00135     void endProc(void);
00136     void beginSubProc(SProcess *sprc);
00137     void endSubProc(void);
00138     void newMGraph(MGraph *mgr);
00139     void newAGraph(EGraph *egr);
00140     void outModel(void);
00141 };
00142
00143 //=====

```

```

00144 class Output {
00145     public:
00146
00147     Options    *opt;
00148     Model      *model;
00149     Process    *proc;
00150     SProcess    *sproc;
00151     char        *outgrf;
00152     FILE        *outgrfp;
00153     char        *outgrp;
00154     FILE        *outgrpp;
00155     int         procId;
00156     Bool        outproc;
00157
00158     Output(Options *optn);
00159     ~Output(void);
00160
00161     void setOutgrf(const char *fname);
00162     void setOutgrp(const char *fname);
00163     Bool outBeginF(Model *mdl, Bool pr);
00164     Bool outBeginP(Model *mdl, Bool pr);
00165     void outEndF(void);
00166     void outEndP(void);
00167     void outProcBeginF(Process *prc);
00168     void outProcBeginP(Process *prc);
00169     void outProcBegin0(int next, int couple, int loop);
00170     void outSProcBeginF(SProcess *sprc);
00171     void outSProcBeginP(SProcess *sprc);
00172     void outProcEndF(void);
00173     void outProcEndP(void);
00174     void outEGraphF(EGraph *egraph);
00175     void outEGraphP(EGraph *egraph);
00176     void outModelF(void);
00177     void outModelP(void);
00178
00179 };
00180
00181 //=====
00182 class Fraction {
00183     public:
00184     BigInt num, den;
00185     double ratio;
00186
00187     Fraction() { num = 0; den = 1; };
00188     Fraction(BigInt n, BigInt d);
00189
00190     void print(const char *msg);
00191     void setValue(BigInt n, BigInt d);
00192     void setValue(Fraction &f);
00193     void add(BigInt n, BigInt d);
00194     void add(Fraction f);
00195     void sub(Fraction f);
00196     BigInt gcd(BigInt n0, BigInt n1);
00197     void normal(void);
00198     Bool isEq(Fraction f);
00199 };
00200
00201 //*****
00202 // Model
00203 //=====
00204 //-----
00205 class Particle {
00206     public:
00207     Model *mdl;           // the model
00208     char *name;           // the name of the particle
00209     char *aname;          // the name of the anti-particle
00210     int id;               // id of this particle
00211     int ptype;            // the type of particle : GRCC_PT_Scalar, etc.
00212     int neutral;          // True if particle==anti-particle
00213     int pcode;            // Particle code
00214     int acode;            // Anti-particle code
00215     int cmindeg;          // min(deg of connectable interactions)
00216     int cmaxdeg;          // max(deg of connectable interactions)
00217
00218     int extonly;          // can appear only as external particle
00219
00220     //-----
00221     Particle(Model *mdl, int pid, PInput *pinp);
00222     ~Particle(void);
00223
00224     char *particleName(int p);
00225     int particleCode(int p);
00226     char *interactionName(int p);
00227     char *aparticle(void);
00228     void prParticle(void);
00229     int isNeutral(void);
00230     const char *typeName(void);

```

```

00231     const char *typeGName(void);
00232
00233 };
00234
00235 //-----
00236 class Interaction {
00237 public:
00238     Model *mdl;           // the model
00239     char *name;           // the name of the interaction
00240     int *plist;           // the list of particle codes
00241     int *clist;           // the orders of coupling constants
00242     int *slist;           // the sorted list of particle codes
00243
00244     int id;               // id of this interaction
00245     int csum;             // the total orders of coupling constants
00246     int nlegs;            // the number of legs
00247     int loop;             // the number of loops
00248     int icode;            // user defined interaction code
00249
00250     int nplist;           // the list of particle codes
00251     int nclist;           // the orders of coupling constants
00252     int nslist;           // the sorted list of particle codes
00253
00254     //-----
00255     Interaction(Model *mdl, int iid, const char* nam, int icode, int *cpl, int nlgs, int *plst, int
csm, int lp);
00256     ~Interaction(void);
00257
00258     void prInteraction(void);
00259 };
00260
00261 //-----
00262 class Model {
00263 public:
00264     char *name;           // the name of this model
00265     char **cnlist;        // the list of coupling constant names
00266     int ncouple;          // the number of coupling consants.
00267
00268     int nParticles;       // the number of particles
00269     Particle **particles; // the list of particles
00270     int pdef;             // the def. of partcls is ended
00271
00272     // list of particles and anti-p. without Undef.
00273     int nallPart;
00274     int allPart[GRCC_MAXMPARTICLES2];
00275
00276     // list of particles and anti-p. without ones of extloop=True
00277     int nintPart;
00278     int intPart[GRCC_MAXMPARTICLES2];
00279
00280     int nInteracts;       // the number of interactions.
00281     Interaction **interacts; // the list of interactions
00282     int vdef;             // the definition of interaction is ended
00283
00284     int maxnlegs;         // maximum number of legs of an interaction
00285     int maxcpl;           // maximum number of coupling const.
00286     int maxloop;          // maximum number of loops inside an intr.
00287
00288     // classification of interactions
00289     int *cplgcp;          // coupling constants
00290     int *cplglg;          // degree
00291     int *cplgnvl;         // number of vertices
00292     int **cplgv1;         // list of vertices
00293     int ncplgcp;          // the number of classes
00294
00295     int defpart;          // GRCC_DEFBYNAME or GRCC_DEFBYCODE
00296     int skipFLine;        // skip analysis fermion line
00297
00298     int DUMMYPADDING;
00299
00300     // methods
00301     Model(MInput *minp);
00302     ~Model(void);
00303
00304     void prModel(void);
00305     void addParticle(PInput *pinp);
00306     void addParticleEnd(void);
00307     void addInteraction(IInput *iinp);
00308     void addInteractionEnd(void);
00309     int findParticleName(const char *name);
00310     int findParticleCode(int pcd);
00311     int findInteractionName(const char *name);
00312     int findInteractionCode(int icd);
00313     char *particleName(int p);
00314     int particleCode(int p);
00315     int normalParticle(int pt);
00316     int antiParticle(int pt);

```

```

00317     int  *allParticles(int *len);
00318     int   findMClass(const int cpl, const int dgr);
00319     void  prParticleArray(int n, int *a, const char *msg);
00320 };
00321
00322 //*****
00323 // Process
00324 //=====
00325 class PNodeClass {
00326 public:
00327     SProcess *sproc;
00328     int      *deg;      // The degree of each node
00329     int      *type;     // The type of the class : GRCC_AT_XXX
00330     int      *particle; // The particle code of external node.
00331     int      *couple;   // The orders of coupling constants
00332     int      *cmindeg;  // min(deg of connectable vertex)
00333     int      *cmaxdeg;  // max(deg of connectable vertex)
00334     int      *count;    // The number of nodes in the class
00335     int      *cl2nd;    // cl2nd[class] <= nd < cl2nd[class+1] = set of nodes
00336     int      *nd2cl;    // nd2cl[node] = class
00337     int      *cl2mcl;   // cl2mcl[class] = class defined in the model.
00338     int      nnodes;
00339     int      nclass;
00340
00341     PNodeClass(SProcess *sproc, int nnodes, int nclass, NCInput *cls);
00342     PNodeClass(SProcess *sproc, int nnodes, int nclass, int *dgs, int *typ, int *ptcl, int *cpl, int
    *cnt, int *cmindeg, int *cmaxdeg);
00343     ~PNodeClass(void);
00344
00345     void prPNodeClass(void);
00346     void prElem(int e);
00347 };
00348
00349 //=====
00350 class SProcess {
00351 // In sprocess the number of nodes and edges are fixed.
00352 // A node is considered as
00353 // (degree, total order of coupling constants),
00354 // which pair of data defines a class of nodes.
00355 public:
00356     Model      *model;      // model
00357     Process    *proc;      // mother process
00358     Options    *opt;       // options
00359     PNodeClass *pnclass;    // list of classes (cpl and deg is determined)
00360     AStack     *astack;
00361
00362     MGraph     *mgraph;
00363     EGraph     *egraph;
00364     Assign     *agraph;
00365
00366     int      *cl2nd;      // (code of nodes in the class[c])
00367                        // = [cl2nd[c], ..., cl2nd[c+1]-1]
00368     int      *nd2cl;      // node nd is in the class pnclass[nd2cl[nd]]
00369     int      nclass;      // the number of classes
00370
00371     int      ninitl;      // the number of initial particles
00372     int      nfinal;      // the number of final particles
00373     int      nvert;       // the number of vertices
00374
00375     int      clist[GRCC_MAXNCPLG]; // coupling constants of the process
00376     int      id;          // sprocess id
00377     int      loop;        // the number of loops
00378     int      nNodes;      // the number of nodes
00379     int      nEdges;      // the number of edges
00380     int      nExtern;     // the number of external particles
00381     int      ncouple;     // the number of coupling constants
00382     int      tCouple;     // the total coupling constants
00383
00384     int      DUMMYPADDING;
00385
00386     BigInt    mgrcount;    // count generated mgraph
00387     BigInt    agrcount;    // count generated agraph
00388     BigInt    extperm;     // count generated agraph
00389
00390     // the results of the graph generation
00391     BigInt    nMGraphs;    // the number of generated M-graphs
00392     BigInt    nMOPI;       // the number of 1PI M-graphs
00393     Fraction  wMGraphs;    // the weighted sum of M-graphs
00394     Fraction  wMOPI;       // the weighted sum of 1PI M-graphs
00395
00396     BigInt    nAGraphs;    // the number of generated A-graphs
00397     BigInt    nAOPI;       // the number of 1PI A-graphs
00398     Fraction  wAGraphs;    // the weighted sum of A-graphs
00399     Fraction  wAOPI;       // the weighted sum of 1PI A-graphs
00400
00401     // methods
00402

```

```

00403     SProcess(Model *mdl, Process *prc, Options *opts, int sid, int *clst, int ncls, NCInput *cls);
00404     SProcess(Model *mdl, Process *prc, Options *opts, int sid, int *clst, int ncls, int *cdeg, int
*ctyp, int *ptcl, int *cpl, int *cnum, int *cmind, int *cmaxd);
00405
00406     ~SProcess(void);
00407
00408     void prSProcess(void);
00409     BigInt generate(void);
00410     void assign(MGraph *mgr);
00411
00412     int toMNodeClass(int *ctyp, int *cldeg, int *clnum, int *cmind, int *cmaxd);
00413     PNodeClass *match(MGraph *mgr);
00414
00415     void endMGraph(MGraph *mgr);
00416     void endAGraph(EGraph *egr);
00417
00418     void resultMGraph(BigInt nmgraphs, Fraction mwsum, BigInt nmopi, Fraction mwopi);
00419     void resultAGraph(BigInt nagraphs, Fraction awsum, BigInt naopi, Fraction awopi);
00420
00421 };
00422
00423 //=====
00424 class Process {
00425 public:
00426     Model      *model;          // model used for this process
00427     Options    *opt;            // options
00428     BigInt     mgrcount;        // count generated mgraph
00429     BigInt     agrcount;        // count generated agraph
00430     int        *initlPart;      // list of ids of initial particles
00431     int        *finalPart;      // list of ids of final particles
00432
00433     int        id;              // process id
00434     int        ninitl;          // the number of initial particles
00435     int        nfinal;          // the number of final particles
00436     int        ctotall;         // the total number of coupling constants
00437     int        nExtern;         // the number of external particles
00438     int        loop;            // the number of external particles
00439     int        maxnlegs;        // the maximum possible degree of node
00440     int        clist[GRCC_MAXNCPLG]; // coupling constans of the process
00441
00442     // table of sprocesses
00443     int        nSubproc;        // the number of sprocesses
00444     SProcess   *sptbl[GRCC_MAXSUBPROCS]; // the table of sprocesses
00445     SProcess   *sproc;          // the current process
00446
00447     // stack for the assignment;
00448     AStack     *astack;
00449
00450     // the number of graphs
00451     BigInt     ngraphs;
00452     BigInt     nopi;
00453     BigInt     wgraphs;
00454     BigInt     wopi;
00455
00456     // the results of the graph generation
00457     BigInt     nMGraphs;        // the number of generated M-graphs
00458     BigInt     nMOPI;           // the number of 1PI M-graphs
00459     Fraction   wMGraphs;        // the weighted sum of M-graphs
00460     Fraction   wMOPI;           // the weighted sum of 1PI M-graphs
00461
00462     BigInt     nAGraphs;        // the number of generated A-graphs
00463     BigInt     nAOPI;           // the number of 1PI A-graphs
00464     Fraction   wAGraphs;        // the weighted sum of A-graphs
00465     Fraction   wAOPI;           // the weighted sum of 1PI A-graphs
00466
00467     double     sec;
00468
00469     Process(int pid, Model *model, Options *opt, int nin, int *initlPart, int nfin, int *finalPart,
int *coupling);
00470     Process(int pid, Model *model, Options *opt, FGInput *fgi);
00471     ~Process(void);
00472
00473     void prProcess(void);
00474     void mkSProcess(void);
00475
00476 };
00477
00478 //*****
00479 // classes for EGraph
00480 //=====
00481 // Constants
00482
00483 #define GRCC_ED_Undef  0
00484 #define GRCC_ED_Deleted 1
00485 #define GRCC_ED_Extern 2
00486 #define GRCC_ED_Back  3

```

```

00488 #define GRCC_ED_Bridge 4
00489 #define GRCC_ED_Inloop 5
00490 #define GRCC_ED_Size 6
00491 #define GRCC_ED_NAMES {"Undef", "Deleted", "Extern", "Back_Edge", "Bridge", "In_Loop"}
00492
00493 #define GRCC_ND_Undef 0
00494 #define GRCC_ND_Deleted 1
00495 #define GRCC_ND_Initial 2
00496 #define GRCC_ND_Final 3
00497 #define GRCC_ND_CPoint 4
00498 #define GRCC_ND_VBlock 5
00499 #define GRCC_ND_Inblock 6
00500 #define GRCC_ND_Size 7
00501 #define GRCC_ND_NAMES {"Undef", "Deleted", "Init", "Final", "CPoint", "VBlock", "In_Block"}
00502
00503 //-----
00504 class ENode {
00505 public:
00506     EGraph *egraph;
00507     int *edges; // list of edges
00508                // the value is \pm [(edge index)+1]
00509
00510     int id; // id of the enode
00511     int maxdeg; // maximum degree
00512     int deg; // degree
00513     int extloop; // loop inside this node
00514     int ndtype; // type of the node
00515     int intrct; // assigned interaction/particle (ext.)
00516
00517     // used in EGraph::biconn()
00518     int *klow;
00519     int visited;
00520
00521     int DUMMYPADDING;
00522
00523     //-----
00524     // functions
00525     ENode(void);
00526     ENode(EGraph *egrph, int loops, int sdeg);
00527     ~ENode(void);
00528     void initAss(EGraph *egrph, int nid, int sdg);
00529     void setId(EGraph *egrph, const int nid);
00530     void copy(ENode *en);
00531     void setExtern(int typ, int pt);
00532     void setType(int typ);
00533     void print(void);
00534 };
00535
00536
00537 //-----
00538 class EEdge {
00539     // momentum is printed like: ("%s%d", (enode.ext)?"Q":"p", enode.momn)
00540 public:
00541     EGraph *egraph; // egraph
00542
00543     int id; // id
00544     int ext;
00545     int ptcl; // assigned particle (agraph)
00546     int deleted; // deleted edge, if true
00547     int nodes[2]; // nodes of bothsides
00548     int nlegs[2]; // nodes of both side (agraph)
00549
00550     // for biconn
00551     int *emom; // external momenta
00552     int *lmom; // loop momenta
00553     int *extMom; // set of external momenta.
00554
00555     Bool cut;
00556     int visited;
00557     int conid; // connected component
00558     int edtype; // type
00559     int opicomp; // id of lPI component
00560     int dir; // direction of momentum
00561
00562     //-----
00563     // functions
00564     EEdge(void);
00565     EEdge(EGraph *egrph, int nedges, int nloops);
00566     ~EEdge(void);
00567     void copy(EEdge *ee);
00568     void setId(EGraph *egrph, const int eid);
00569     void print(void);
00570
00571     void setType(int typ);
00572     void setLMom(int k, int dir);
00573     void setEMom(int nedges, int *extn, int dir);
00574

```

```

00575 };
00576
00577 //-----
00578 // type of fermion lines
00579
00580 typedef enum {FL_Open, FL_Closed} FLType;
00581
00582 //-----
00583 class EFLine {
00584 public:
00585     int     elist[GRCC_MAXNODES];    // list of (\pm [(edge index)+1])
00586     FLType  ftype;
00587     int     fkind;
00588     int     nlist;
00589
00590     EFLine(void);
00591     void print(const char *msg);
00592 };
00593
00594 //-----
00595 class EGraph {
00596 public:
00597     Options *opt;           // table of options
00598     Model *model;
00599     Process *proc;
00600     SProcess *sproc;
00601     MGraph *mgraph;
00602     MConn *econn;
00603
00604     ENode **nodes;
00605     EEdge **edges;         // edges[nEdges+1]: index starts from 1
00606
00607     BigInt mId;            // id of mgraph
00608     BigInt aId;            // id of agraph in the same mgraph
00609     BigInt sId;            // sequential no. of agraph
00610     BigInt gSubId;         // ???
00611     Bool assigned;         // mgraph (False) or agraph (True)
00612
00613     int     fsign;
00614     BigInt nsym, esym;     // symmetry factor with symm. ext.
00615     BigInt nsym1;          // symmetry factor without symm. ext.
00616     BigInt extperm;        // the order of group of symm. ext.
00617     BigInt multp;          // multiplicity of graph in symm. ext.
00618
00619     int     pId;
00620
00621     int     sNodes;        // memory size
00622     int     sEdges;        // memory size
00623     int     sMaxdeg;       // memory size
00624     int     sLoops;        // memory size
00625
00626     int     nNodes;
00627     int     nEdges;
00628     int     nExtern;
00629     int     maxdeg;        // maximum value of degree of nodes
00630     int     nLoops;
00631     int     totalc;        // total order of coupling constants
00632
00633     // biconnect
00634     int     nopicomp;
00635     int     opi2plp;
00636     int     nopi2p;
00637     int     nadj2ptv;      // the no. of edges connecting 2point vertices
00638
00639     int     DUMMYPADDING;
00640
00641     int     *bidef;
00642     int     *bilow;
00643     int     *extMom;
00644     int     bconn;
00645     int     bcount;
00646     int     loopm;
00647     int     opiCount;
00648
00649     // Fermion lines
00650     EFLine *flines[GRCC_MAXFLINES];
00651     int     nflines;
00652
00653     int     DUMMYPADDING1;
00654
00655     //-----
00656     // functions
00657     EGraph(int nnodes, int nedges, int mxdeg);
00658     ~EGraph(void);
00659
00660     void copy(EGraph *eg);
00661     void print(void);

```

```

00662 void fromDGraph(DGraph *dg);
00663 void fromMGraph(MGraph *mgraph);
00664 Bool isOptE(void);
00665
00666 ENode *setExtern(int n0, int pt, int ndtp);
00667 Bool isExternal(int nd) { return (nodes[nd]->extloop < 0); };
00668 Bool isFermion(int nd);
00669
00670 void setExtLoop(int nd, int val);
00671 void endSetExtLoop(void);
00672
00673 int connComp(void);
00674 int connVisit(int nd, int ncc);
00675
00676 void biconnE(void);
00677 void biinitE(void);
00678 void bisearchE(int nd, int *extlst, int *intlst, int *opiext, int *opiloop);
00679
00680 int findRoot(void);
00681 int dirEdge(int n, int e);
00682 void extMomConsv(void);
00683 int cmpMom(int *lm0, int *em0, int *lm1, int *em1);
00684 int groupLMom(int *grp, int *ed2gr);
00685
00686 void chkMomConsv(void);
00687
00688 void prFLines(void);
00689 void getFLines(void);
00690 int fltrace(int fk, int nd0, int *fl);
00691 void addFLine(const FLType ft, int fk, int nfl, int *fl);
00692
00693 int legParticle(int ed, int lg);
00694
00695 };
00696
00697 //*****
00698 // Symmetry group of graphs
00699 //=====
00700 class SGroup {
00701 public:
00702     BigInt size;           // # of allocated elements
00703     BigInt nelelem;        // # of saved elements
00704     int **elem;            // saved elements
00705     int nnodes;            // # of nodes.
00706     int neclass;           // # of classes
00707     int eclass[GRCC_MAXNODES]; // table of classes
00708     int cgen;              // counter for the generation
00709     int csav;              // counter for the saved elements
00710     int permg[GRCC_MAXNODES]; // resulting permutation
00711     int perms[GRCC_MAXNODES]; // curr. elem of saved ones.
00712
00713     int pgr[GRCC_MAXNODES]; // work
00714     int pgq[GRCC_MAXNODES]; // work
00715     int psr[GRCC_MAXNODES]; // work
00716     int psq[GRCC_MAXNODES]; // work
00717
00718     //-----
00719     // functions
00720
00721     SGroup(void);
00722     ~SGroup(void);
00723
00724     void print(void);
00725     void newGroup(int nelm, int nclss, int *clss);
00726     void clearGroup(void);
00727     void delGroup(void);
00728     int *genNext(void);
00729     void addGroup(int *p);
00730     BigInt nElem(void);
00731     int *nextElem(void);
00732 };
00733
00734 //*****
00735 // MGraph
00736 //=====
00737 // class of nodes for MGraph
00738
00739 class MNode {
00740 public:
00741     int id;                // node id
00742     int deg;               // degree(node) = the number of legs
00743     int clss;              // initial class number in which the node belongs
00744     int extloop;
00745     int cmindeg;
00746     int cmaxdeg;
00747
00748     int freeleg;           // the number of free legs

```



```

00749     int visited;
00750
00751     MNode(int id, int cls, NCInput *mgi);
00752     MNode(int id, int deg, int extloop, int cls, int cmind, int cmaxd);
00753 };
00754
00755 //=====
00756 // class of scalar graph expressed by matrix form
00757 //
00758 // Input : the classified set of nodes.
00759 // Output : control passed to 'EGraph(self)'
00760
00761 class MGraph {
00762 public:
00763     // initial conditions
00764     Options *opt;          // options
00765
00766     // symmetry group
00767     SGroup *group;         // symmetry group
00768     MOrbits *orbits;       // Orbits of nodes with respect to symmetry group
00769
00770     MNode **nodes;         // table of MNode object
00771     BigInt mId;            // process/sprocess ID
00772     int *clist;            // list of initial classes
00773     int pId;               // process/sprocess ID
00774     int nNodes;            // the number of nodes
00775     int nEdges;            // the number of edges
00776     int nLoops;            // the number of loops
00777     int nExtern;           // the number of external nodes
00778     int nClasses;          // the number of initial classes
00779     int mindeg;            // minimum value of degree of nodes
00780     int maxdeg;            // maximum value of degree of nodes
00781
00782     // the current graph
00783     int **adjMat;          // adjacency matrix
00784     MNodeClass *curcl;     // the current 'MNodeClass' object
00785     EGraph *egraph;
00786     BigInt nsym;           // symmetry factor from nodes
00787     BigInt esym;           // symmetry factor from edges
00788
00789     // generated set of graphs
00790     BigInt cDiag;           // the total number of generated graphs
00791     BigInt c1PI;           // the total number of 1PI graphs
00792     BigInt cNoTadpole;     // the total number of graphs without tadpoles
00793     BigInt cNoTadBlock;    // the total number of graphs without tad-blocks
00794     BigInt c1PINoTadBlock; // the total number of 1PI graphs without tad-blocks
00795     Fraction wscon;        // weighted sum of graphs
00796     Fraction wsopi;        // weighted sum of 1PI graphs
00797
00798     // measures of efficiency
00799     BigInt ngen;           // generated graph before check
00800     BigInt ngconn;        // generated connected graph before check
00801
00802     BigInt nCallRefine;
00803     BigInt discardRefine;
00804     BigInt discardDisc;
00805     BigInt discardIso;
00806
00807     // for options
00808     Bool opi;
00809     Bool opiloop;
00810     Bool extself;
00811     Bool selfloop;
00812     Bool tadpole;
00813     Bool tadblock;
00814     Bool block;
00815
00816     int DUMMYPADDING1;
00817
00818     // table of n edge-connected components
00819     MConn *mconn;
00820
00821     // work space for isomorphism
00822     int **modmat;          // permuted adjacency matrix
00823
00824     // work space for biconnected component
00825     int *bidef;
00826     int *bilow;
00827     int bicount;
00828
00829     int DUMMYPADDING2;
00830
00831     //-----
00832     // functions
00833     MGraph(int pid, int ncl, NCInput *mgi, Options *opt);

```

```

00836     MGraph(int pid, int ncl, int *cldeg, int *clnum, int *cllexl, int *cmind, int *cmaxd, Options
*opt);
00837     ~MGraph(void);
00838
00839     void    init(void);
00840     BigInt  generate(void);
00841     Bool    isExternal(int nd)  { return (nodes[nd]->extloop < 0); };
00842     void    printAdjMat(MNodeClass *cl);
00843     void    print(void);
00844
00845     Bool    isConnected(void);
00846     Bool    visit(int nd);
00847     Bool    isIsomorphic(MNodeClass *cl);
00848     void    permMat(int size, int *perm, int **mat0, int **mat1);
00849     int     compMat(int size, int **mat0, int **mat1);
00850     MNodeClass *refineClass(MNodeClass *cl);
00851     void    bisearchME(int nd, int pd, int ned, MCOpi *mopi, MCBBlock *mblk, int *next, int *nart);
00852     void    biconnME(void);
00853     Bool    isOptM(void);
00854
00855     void    connectClass(MNodeClass *cl);
00856     void    connectNode(int sc, int ss, MNodeClass *cl);
00857     void    connectLeg(int sc, int sn, int tc, int ts, MNodeClass *cl);
00858     void    newGraph(MNodeClass *cl);
00859
00860 };
00861
00862 //=====
00863 // class of node-classes for MGraph
00864 //-----
00865 class MNodeClass {
00866 public:
00867     int     clmat[GRCC_MAXNODES][GRCC_MAXNODES]; // matrix used for classification
00868     int     clist[GRCC_MAXNODES]; // the number of nodes in each class
00869     int     ndcl[GRCC_MAXNODES]; // node --> class
00870     int     flist[GRCC_MAXNODES+1]; // the first node in each class
00871     int     clord[GRCC_MAXNODES]; // ordering of classes
00872     int     cmindeg[GRCC_MAXNODES]; // min(deg of connectable node)
00873     int     cmaxdeg[GRCC_MAXNODES]; // max(deg of connectable node)
00874     int     nNodes; // the number of nodes
00875     int     nClasses; // the number of classes
00876     int     maxdeg; // maximal value of degree(node)
00877
00878     int     flg0;
00879     int     flg1;
00880     int     flg2;
00881
00882     MNodeClass(int nnodes, int nclasses);
00883     ~MNodeClass(void);
00884
00885     void    init(int *cl, int mxdeg, int **adjmat);
00886     void    copy(MNodeClass* mnc);
00887     int     clCmp(int nd0, int nd1, int cn);
00888     void    printMat(void);
00889
00890     void    mkFlist(void);
00891     void    mkNdCl(void);
00892     void    mkClMat(int **adjmat);
00893     void    incMat(int nd, int td, int val);
00894     int     cmpMNCArray(int *a0, int *a1, int ma);
00895     void    reorder(MGraph *mg);
00896 };
00897
00898 //=====
00899 // class of lPI component
00900 //-----
00901 class MCOpi {
00902 public:
00903     int *nodes; // array of nodes in
00904     int nnodes; // # nodes in the lPI component
00905     int nlegs; // # leg of the lPI component
00906     int next; // # external particles of the lPI comp.
00907     int nedges; // # edges in the lPI comp.
00908     int loop; // # loops in the lPI comp.
00909     int ctloop; // # loops in the counter terms in the OP comp.
00910
00911     MCOpi(void);
00912     ~MCOpi(void);
00913
00914     void init(void);
00915 };
00916
00917 //=====
00918 // class of bridge
00919 //-----
00920 class MCBridge {

```

```

00922 public:
00923     Edge2n nodes; // nodes at the both size of the bridge
00924     int next; // # momenta of ext. particles flowing on the bridge
00925
00926     MCBridge(void);
00927     ~MCBridge(void);
00928 };
00929
00930 //=====
00931 // class of block
00932 //-----
00933 class MCBlock {
00934 public:
00935     Edge2n *edges; // array of edges in the block
00936     int nedges; // # edges in the block
00937     int nartps; // # articulation points of the block
00938     int loop; // # loop in the block
00939
00940     int DUMMYPADDING;
00941
00942     MCBlock(void);
00943     ~MCBlock(void);
00944     void init(void);
00945 };
00946
00947 //=====
00948 // class of table of MConn
00949 //-----
00950 class MConn {
00951 public:
00952     // 2-edge connected components
00953     MCOpI *opics; // table of n-edge-connected components
00954     MCBridge *bridges; // table of bridges
00955     MCBlock *blocks; // table of blocks
00956     int *articuls; // buffer for nodes for articulation points
00957
00958     int *opisp; // buffer for nodes in lPI components
00959     int *opistk; // stack of nodes for lPI components.
00960     Edge2n *blksp; // buffer for edges in blocks
00961     Edge2n *blkstk; // stack of edges for blocks
00962     int snodes; // # nodes
00963     int sedges; // # edges
00964
00965     // opi components (edge-connected)
00966     int nopic; // # lPI components (n1PIComps)
00967     int nlpopic; // # looped lPI components (n1PIComps)
00968     int nctopic; // # lPI components of one counter term.
00969     int nbridges; // # bridges
00970     int ne0bridges; // # bridges whose next=0
00971     int nelbridges; // # bridges whose next=1
00972     int nselfloops;
00973
00974     // blocks (node-connected)
00975     int nblocks; // # blocks
00976     int nalblocks; // # bridges whose next=0
00977     int narticuls; // # articulation points
00978     int neblocks; // # effective looped blocks
00979
00980     // indices to work spaces
00981     int nopisp; // # used in opisp
00982     int opistkptr; // # stack pointer of opistk
00983     int nblksp; // # used in blksp
00984     int blkstkptr; // # stack pointer of tlkstk
00985
00986     int DUMMYPADDING;
00987
00988     MConn(int nnod, int nedg);
00989     ~MConn(void);
00990
00991     void init(void);
00992     void pushNode(int nd);
00993     void pushEdge(int n0, int n1);
00994     void addOpIc(MCOpI *mopi, int stp);
00995     void addBridge(int n0, int n1, int nex, int nextot);
00996     void addArtic(int nd, int mul);
00997     void addBlock(MCBBlock *eblk, int stp);
00998     void addBlockSelf(int nd, int mul);
00999     void print(void);
01000 };
01001
01002 //=====
01003 // class of orbits of nodes
01004 //-----
01005 class MORbits {
01006     // Usage
01007     // 1. node ==> orbit
01008     // (class) = nd2or[(node)]

```

```

01009 //
01010 // 2. class ==> nodes
01011 // for (c = 0; c < nClass; c++) {
01012 //     for (j = flist[c]; j < flist[c+1]; j++) {
01013 //         (node) = or2nd[c];
01014 //     }
01015 // }
01016 public:
01017     int nOrbits;
01018     int nNodes;
01019     int nd2or[GRCC_MAXNODES];
01020     int or2nd[GRCC_MAXNODES];
01021     int flist[GRCC_MAXNODES+1];
01022
01023     MOrbits(void);
01024     ~MOrbits(void);
01025
01026     void print(void);
01027
01028     // construction of orbits
01029     void initPerm(int nnodes);
01030     void fromPerm(int *perm);
01031     void toOrbits(void);
01032 };
01033
01034 //*****
01035 // Particle assignment
01036 //=====
01037 typedef int CheckPt[2];
01038
01039 typedef enum {
01040     AS_UnAssLegs, AS_Assigned, AS_Assigned0, AS_AssExt, AS_Impossible
01041 } NCandSt;
01042
01043 //=====
01044 class NCand {
01045     // List of candidats for the assignment of interactions to a vertex
01046
01047 public:
01048     int deg; // degree of the node
01049     NCandSt st; // status
01050     int nilist; // length of the list
01051     int ilist[GRCC_MAXMINTERACT]; // list of candidates
01052
01053     //=====
01054     NCand(const NCandSt sta, const int dega, const int nilst, int *ilst);
01055     ~NCand(void);
01056
01057     // print
01058     void prNCand(const char* msg);
01059 };
01060
01061 //=====
01062 class ECand {
01063     // List of candidats for the assignment of particles to an edge
01064
01065 public:
01066     Bool det; // determined or not
01067     int nplist; // size of the list of candidates
01068     int plist[GRCC_MAXMPARTICLES2]; // list of candidates
01069
01070     ECand(int dt, int nplist, int *plst);
01071     ~ECand(void);
01072
01073     // print
01074     void prECand(const char *msg);
01075 };
01076
01077 //=====
01078 class ANode {
01079     // Connection information of a node to others
01080     //
01081     // (this node) -- (adjacent edge) -- (next node)
01082     // (n0, j) e nl
01083     //
01084     // e = (aedges[j], aelegs[j])
01085     // nl = anodes[j]
01086
01087 public:
01088     int deg; // degree of the node
01089     int nlegs; // the number of legs already assigned
01090     int *anodes; // anodes[j] = (next node of leg j)
01091     int *aedges; // aedges[j] = (next edge of leg j)
01092     int *aelegs; // aelegs[j] = (leg of the next edge)
01093     NCand *cand; // candidate list
01094
01095     ANode(int dg);

```

```

01096     ~ANode(void);
01097
01098     int    newleg(void);          // get a new leg
01099 };
01100
01101 //=====
01102 class AEdge {
01103 // Connection information of an edge
01104 // (self) ==> nodes [(nodes[0], nlegs[0]), (nodes[1], nlegs[1])]
01105 // particle 'ptcl' flows from leg=0 to 1.
01106
01107 public:
01108
01109     ECand *cand;                  // candidate
01110     int    nodes[2];              // nodes of both sides of the edge
01111     int    nlegs[2];              // nodes of both sides of the edge
01112     int    ptcl;                  // particle defined in the model
01113
01114     int    DUMMYPADDING;
01115
01116     AEdge(int n0, int l0, int n1, int l1);
01117     ~AEdge(void);
01118 };
01119
01120 //=====
01121 class Assign {
01122 // Class for particle/interaction assignment.
01123
01124 public:
01125
01126     EGraph    *egraph;            // EGraph object
01127     MGraph    *mgraph;            // MGraph object
01128     SProcess   *sproc;            // Sub-process object
01129     Process    *proc;             // Process object
01130     Model      *model;            // Model object
01131     Options    *opt;              // Option object
01132     AStack     *astack;           // Stack for saving candidates
01133     PNodeClass *pnclass;          // class of nodes
01134     MOrbits    *orbits;           // orbits of nodes by symmetry group
01135
01136     int         nNodes;           // the number of nodes
01137     int         nEdges;           // the number of edges
01138     int         nExtern;          // the number of external particles.
01139     int         nETotal;          // the total number of edges
01140
01141     BigInt      nAGraphs;         // the number of assigned graphs
01142     Fraction    wAGraphs;         // the weighted sum of graphs
01143     BigInt      nAOPI;            // the number of assigned lPI
01144     Fraction    wAOPI;           // the weighted sum of lPI
01145
01146     ANode       **nodes;          // table of nodes
01147     AEdge       **edges;          // table of edges
01148
01149     CheckPt     checkpoint0;      // save stack pointers
01150
01151     int         cplleft[GRCC_MAXNCPLG]; // coupling constants left
01152
01153 //=====
01154     Assign(SProcess *sproc, MGraph *mgr, PNodeClass *pnc);
01155     ~Assign(void);
01156
01157 //=====
01158     // print lists of candidates
01159     void    prCand(const char *msg);
01160
01161     // check
01162     void    checkAG(const char *msg);
01163
01164 //=====
01165     // control of assignment
01166
01167     // entry point of assignment
01168     Bool    assignAllVertices(void);
01169
01170     // select a source node for assignment
01171     Bool    selectVertex(void);
01172     Bool    selectVertexSimp(int lastv);
01173
01174     // select a source leg of the node for assignment
01175     Bool    selectLeg(int v, int lastlg);
01176
01177     // assign a vertex a interaction
01178     Bool    assignVertex(int v);
01179
01180     // assignment procedure is finished. Needs check.
01181     Bool    allAssigned(void);
01182

```

```

01183 //=====
01184 // Input and output
01185
01186 // Input from mgraph
01187 Bool    fromMGraph(void);
01188
01189 // add an edge
01190 void    addEdge(int n0, int n1, int nplist, int *plist);
01191
01192 // connect nodes and edges.
01193 void    connect(int n0, int l0, int eg, int el, int n1, int l1);
01194
01195 // Output to egraph
01196 Bool    fillEGraph(int aid, BigInt nsym, BigInt esym, BigInt nsym1);
01197
01198 // reorder legs in accordance with the interaction definition
01199 int     *reordLeg(int n, int *reord, int *plist, int *used);
01200
01201 //=====
01202 // direction of particle on an edge and at (node, leg)
01203
01204 // convert particle code
01205 //   at node-leg ('n', 'ln') <==> edge at ('n', 'ln')
01206 int     getLegParticle(int n, int ln);
01207
01208 // convert particle 'pt' on the edge to ('n', 'ln')
01209 int     legEdgeParticle(int n, int ln, int pt);
01210
01211 // convert candidate list at ('v', 'lg') into in-coming direction
01212 int     legPart(int v, int lg, int nplst, int *plst, int *rlist, const int size);
01213
01214 // convert candidate list at ('v', 'lg') into in-coming direction
01215 int     candPart(int v, int ln, int *plist, const int size);
01216
01217 //=====
01218 // assignment
01219
01220 // find a vertex to be assigned
01221 int     selUnAssVertex(void);
01222 int     selUnAssVertexSimp(int lastv);
01223
01224 // find a leg of the vertex to be assigned
01225 int     selUnAssLeg(int v, int lastlg);
01226
01227 // assign the interaction 'ia' to the vertex 'v'.
01228 NCandSt assignIVertex(int v, int ia);
01229
01230 // assign particle 'pt' the the leg 'ln' of the node 'n'.
01231 Bool    assignPLeg(int n, int ln, int pt);
01232 Bool    isOrdPLeg(int n, int ln, int pt);
01233 Bool    detEdge(int e);
01234
01235 //=====
01236 // candidates
01237
01238 // construct lists of assigned / unassigned particles at a node
01239 Bool    candPartClassify(int v, int *npdass, int *pdass, int *npuass, int *puass, const int size);
01240
01241 // update candidate list for a vertex 'v'.
01242 Bool    updateCandNode(int v);
01243
01244 //=====
01245 // filter
01246
01247 Bool    checkOrderCpl(void);
01248 Bool    isOrdLegs(void);
01249 Bool    isIsomorphic(MNodeClass *cl, BigInt *nsym, BigInt *esym, BigInt *nsym1);
01250 int     cmpPermGraph(int *p, MNodeClass *cl);
01251 int     cmpNodes(int nd0, int nd1, MNodeClass *cn);
01252 BigInt  edgeSym(void);
01253
01254 void    saveCouple(int *sav);
01255 void    restoreCouple(int *sav);
01256 Bool    subCouple(int *cpl);
01257
01258 #ifdef CHECK
01259 //=====
01260 // check
01261 Bool    checkCand(const char *msg);
01262
01263 void    checkNode(int n, const char *msg);
01264 #endif
01265 };
01266
01267 //*****
01268 // Stack of candidate lists
01269 //=====

```

```

01270 class NStack {
01271 // Stack for backtracking method for a node.
01272
01273 public:
01274     int     noden;
01275     int     deg;
01276     NCandSt st;
01277     int     nilist;
01278     int     ilist[GRCC_MAXMINTERACT];
01279
01280     NStack() { };
01281     ~NStack() { };
01282
01283     void print(const char *msg);
01284
01285 };
01286
01287 //=====
01288 class EStack {
01289 // Stack for backtracking method for an edge.
01290
01291 public:
01292     int     edgen;
01293     int     det;
01294     int     nplist;
01295     int     plist[GRCC_MAXMPARTICLES2];
01296
01297     EStack() { };
01298     ~EStack() { };
01299
01300     void print(const char *msg);
01301 };
01302
01303
01304 //=====
01305 class AStack {
01306 // Stack for backtracking method for an edge.
01307
01308 public:
01309     Assign   *agraph;
01310
01311     NStack   **nStack;      // stack for nodes
01312     int       nStackP;      // stack pointer for nodes
01313     int       nSize;        // stack size
01314
01315     EStack   **eStack;      // stack for edges
01316     int       eStackP;      // stack pointer for edges
01317     int       eSize;        // stack size
01318
01319     AStack(int nSize, int eSize);
01320     ~AStack(void);
01321
01322     void setAGraph(Assign *ag);
01323
01324     void checkPoint(CheckPt sav);
01325     void restore(CheckPt sav);
01326     void restoreMsg(CheckPt sav, const char *msg);
01327     void prStack(void);
01328
01329     void pushNode(int n);
01330     void pushEdge(int e);
01331
01332 private:
01333     void restoreNode(int spr);
01334     void restoreEdge(int spr);
01335 };
01336
01337 //=====
01338 // end of namespace Grcc
01339 #ifndef GRCC_NAMESPACE
01340 }
01341 #endif

```

4.59 grccparam.h

```

00001 #ifndef GRCC_PARAM_H
00002 #else
00003 #define GRCC_PARAM_H
00004
00005 /*=====
00006  * Parameters
00007  */
00008 /*

```

```

00009 #define GRCC_NAMESPACE
00010 */
00011
00012 #define GRCC_MAXNCPLG      4
00013 #define GRCC_MAXLEGS      10
00014 #define GRCC_MAXMPARTICLES 50
00015 #define GRCC_MAXMINTERACT 200
00016 #define GRCC_MAXSUBPROCS  500
00017 #define GRCC_MAXNODES     20
00018 #define GRCC_MAXEDGES     100
00019 #define GRCC_MAXNSTACK    50
00020 #define GRCC_MAXESTACK    50
00021 #define GRCC_MAXGROUP     1000
00022 #define GRCC_MAXPSLIST    500
00023
00024 #define GRCC_MAXMPARTICLES2 (2*GRCC_MAXMPARTICLES-1)
00025 #define GRCC_MAXFLINES     GRCC_MAXEDGES
00026
00027 #define GRCC_FRACERROR     (1.0e-10)
00028
00029 /* error message */
00030 /*
00031 #define GRCC_Stderr        stderr
00032 */
00033 #define GRCC_Stderr        stdout
00034 /*
00035 #define GRCC_ABORT()      exit(1)
00036 */
00037 #define GRCC_ABORT()      abort()
00038
00039 /*-----
00040 * Constants
00041 */
00042
00043 /* model definition */
00044 #define GRCC_DEFBYNAME    1 /* define interactions by particle name */
00045 #define GRCC_DEFBYCODE    2 /* define interactions by particle code */
00046
00047 /* types of particles */
00048 #define GRCC_PT_Undef     0
00049 #define GRCC_PT_Scalar    1
00050 #define GRCC_PT_Dirac     2
00051 #define GRCC_PT_Majorana  3
00052 #define GRCC_PT_Vector    4
00053 #define GRCC_PT_Ghost     5
00054 #define GRCC_PT_Size     6
00055
00056 #define GRCC_PTS_Undef    "undef"
00057 #define GRCC_PTS_Scalar   "scalar"
00058 #define GRCC_PTS_Dirac    "dirac"
00059 #define GRCC_PTS_Majorana "majorana"
00060 #define GRCC_PTS_Vector   "vector"
00061 #define GRCC_PTS_Ghost    "ghost"
00062
00063 #define GRCC_PTO_General  0
00064 #define GRCC_PTO_ExtOnly  1
00065
00066 #define GRCC_PT_Table { GRCC_PTS_Undef, GRCC_PTS_Scalar, GRCC_PTS_Dirac, GRCC_PTS_Majorana,
GRCC_PTS_Vector, GRCC_PTS_Ghost}
00067
00068 #define GRCC_PT_GTable { "Undef", "Scalar", "Fermion", "Majorana", "Vector", "Ghost"}
00069
00070 /* for mgraph generation */
00071 #define GRCC_AT_Vertex    0
00072 #define GRCC_AT_External -1
00073 #define GRCC_AT_Initial  -2
00074 #define GRCC_AT_Final    -3
00075
00076 #define GRCC_AT_NdStr(x) ((x)>=GRCC_AT_Vertex ?"Vertex":      \
00077 ((x)==GRCC_AT_External ?"External":      \
00078 ((x)==GRCC_AT_Final ?"Fianl":           \
00079 ((x)==GRCC_AT_Initial ?"Initial":"Undef"))))
00080
00081 /* graph generation */
00082 #define GRCC_MGraph      0 /* mgraph (topology) generation */
00083 #define GRCC_AGraph      1 /* agraph generation */
00084
00085 /* options for graph greneration */
00086 #define GRCC_OPT_Step      0
00087 #define GRCC_OPT_Outgrf    1
00088 #define GRCC_OPT_Outgrp    2
00089 #define GRCC_OPT_lPI       3
00090 #define GRCC_OPT_NoSelfLoop 4
00091 #define GRCC_OPT_NoTadpole  5
00092 #define GRCC_OPT_No1PtBlock 6
00093 #define GRCC_OPT_No2PtL1PI  7
00094 #define GRCC_OPT_NoExtSelf  8

```



```

00095 #define GRCC_OPT_NoAdj2PtV      9
00096 #define GRCC_OPT_Block          10
00097 #define GRCC_OPT_SymmInitial    11
00098 #define GRCC_OPT_SymmFinal      12
00099 #define GRCC_OPT_Size          13
00100
00101 typedef struct {
00102     const char *name;
00103     const char *mean;
00104     int         defaultv;
00105     int         DUMMYPADDING;
00106 } OptDef;
00107
00108 /*-----
00109  * Conversion of
00110  *   eind : index (ed) of EGraph.edges[ed]
00111  *   sind : value (v)  of EGraph.nodes[nd]->edges[j])
00112  */
00113 #define I2Vedge(eind, sign)  (((sign<=0)?(-1):(1))*((eind)+1))
00114 #define V2Iedge(sind)       (abs(sind)-1)
00115 #define V2Ileg(sind)        ((sind)<0 ? 0 : 1)
00116 /*-----
00117  * data types for model definition
00118  */
00119 typedef struct {
00120     const char *name;           /* name of the model */
00121     int         defpart;        /* particle is defined by name or code */
00122                                 /* GRCC_DEFBYNAME or GRCC_DEFBYCODE */
00123     int         ncouple;        /* No. of coupling constants */
00124     const char *cnamlist[GRCC_MAXNCPLG]; /* Names of coupling constants */
00125 } MInput;
00126
00127 typedef struct {
00128     const char *name;           /* name of particle */
00129     const char *aname;          /* name of anti-particle */
00130     int         pcode;          /* code of particle */
00131     int         acode;          /* code of anti-particle */
00132     const char *ptypen;         /* type : 'GRCC_PTS_Scalar' etc */
00133     int         ptypec;         /* type : 'GRCC_PT_Scalar' etc */
00134     int         extonly;        /* only as external particle */
00135 } PInput;
00136
00137 typedef struct {
00138     const char *name;           /* name of interaction */
00139     int         icode;          /* code of interaction */
00140     int         nplstn;         /* the number of legs */
00141     const char *plistn[GRCC_MAXLEGS]; /* list of particle names */
00142     int         plstc[GRCC_MAXLEGS]; /* list of particle codes */
00143     int         cvallist[GRCC_MAXNCPLG]; /* coupling constants */
00144 } IInput;
00145
00146 /*-----
00147  * data type for node class input
00148  */
00149 typedef struct {
00150     /* used as input to MGraph() and SProcess() */
00151     int         cldeg;
00152     int         clnum;
00153     int         cltyp;
00154     int         cmind;
00155     int         cmaxd;
00156
00157     /* used as input to SProcess() */
00158     int         ptcl;
00159     int         cple;
00160 } NCInput;
00161
00162 /*-----
00163  * data types for process definition
00164  */
00165 typedef struct {
00166     long         ninitl;        /* the number of initial particles */
00167     const char *initln[GRCC_MAXNODES]; /* list of initial particles (name) */
00168     int         initlc[GRCC_MAXNODES]; /* list of final particles (code) */
00169     long         nfinal;        /* the number of final particles */
00170     const char *finaln[GRCC_MAXNODES]; /* list of final particles (name) */
00171     int         finalc[GRCC_MAXNODES]; /* list of final particles (code) */
00172     int         coupl[GRCC_MAXNCPLG]; /* list of orders of c. consts */
00173 } FGInput;
00174
00175 /* class of node : input for SProcess */
00176 typedef struct {
00177     int         cdeg;           /* degree of each node */
00178     int         ctyp;           /* typde : GRCC_AT_XXX */
00179     int         cnum;           /* the number of nodes in the class */
00180     int         cple;           /* total order of c.c. of each node */
00181                                 /* = 0 for external particle */

```

```

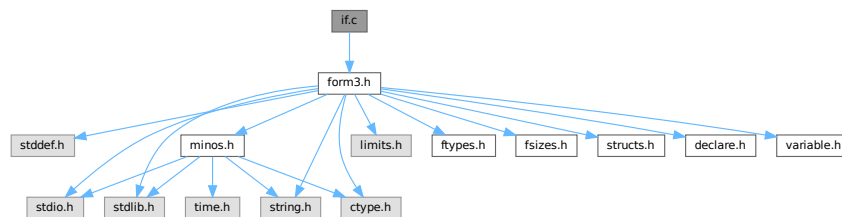
00182     const char *pname;                /* particle name defined in the model */
00183                                         /* = NULL for vertex */
00184     long      pcode;                  /* particle code defined in the model */
00185                                         /* = 0 for vertex */
00186                                         /* long = int + padding */
00187 } PGInput;
00188
00189 /*-----
00190  * minimum input data to EGraph
00191  */
00192 typedef struct {
00193     int      extloop;                /* external/loop */
00194     int      intret;                 /* particl/interaction */
00195 } DNode;
00196
00197 typedef int  DEdge[2]; /* {node0, node1} */
00198
00199 typedef struct {
00200     int      nnodes;
00201     int      nedges;
00202     DNode    *nodes;
00203     DEdge    *edges;
00204 } DGraph;
00205
00206 /*-----
00207  */
00208 #endif

```

4.60 if.c File Reference

```
#include "form3.h"
```

Include dependency graph for if.c:



Functions

- WORD [GetIfDollarNum](#) (WORD *ifp, WORD *ifstop)
- int [FindVar](#) (WORD *v, WORD *term)
- WORD [DolfStatement](#) (PHEAD WORD *ifcode, WORD *term)
- WORD [HowMany](#) (PHEAD WORD *ifcode, WORD *term)
- void [DoubleIfBuffers](#) (void)
- int [DoSwitch](#) (PHEAD WORD *term, WORD *lhs)
- int [DoEndSwitch](#) (PHEAD WORD *term, WORD *lhs)
- SWITCHTABLE * [FindCase](#) (WORD nswitch, WORD ncase)
- int [DoubleSwitchBuffers](#) (void)
- void [SwitchSplitMergeRec](#) (SWITCHTABLE *array, WORD num, SWITCHTABLE *auxarray)
- void [SwitchSplitMerge](#) (SWITCHTABLE *array, WORD num)

4.60.1 Detailed Description

Routines for the dealing with if statements.

Definition in file [if.c](#).

4.60.2 Function Documentation

4.60.2.1 GetIfDollarNum()

```
WORD GetIfDollarNum (
    WORD * ifp,
    WORD * ifstop )
```

Definition at line 106 of file [if.c](#).

4.60.2.2 FindVar()

```
int FindVar (
    WORD * v,
    WORD * term )
```

Definition at line 173 of file [if.c](#).

4.60.2.3 DoIfStatement()

```
WORD DoIfStatement (
    PHEAD WORD * ifcode,
    WORD * term )
```

Definition at line 280 of file [if.c](#).

4.60.2.4 HowMany()

```
WORD HowMany (
    PHEAD WORD * ifcode,
    WORD * term )
```

Definition at line 840 of file [if.c](#).

4.60.2.5 DoubleIfBuffers()

```
void DoubleIfBuffers (
    void )
```

Definition at line 1033 of file [if.c](#).

4.60.2.6 DoSwitch()

```
int DoSwitch (
    PHEAD WORD * term,
    WORD * lhs )
```

Definition at line 1070 of file [if.c](#).

4.60.2.7 DoEndSwitch()

```
int DoEndSwitch (
    PHEAD WORD * term,
    WORD * lhs )
```

Definition at line 1086 of file [if.c](#).

4.60.2.8 FindCase()

```
SWITCHTABLE * FindCase (
    WORD nswitch,
    WORD ncase )
```

Definition at line 1097 of file [if.c](#).

4.60.2.9 DoubleSwitchBuffers()

```
int DoubleSwitchBuffers (
    void )
```

Definition at line 1135 of file [if.c](#).

4.60.2.10 SwitchSplitMergeRec()

```
void SwitchSplitMergeRec (
    SWITCHTABLE * array,
    WORD num,
    SWITCHTABLE * auxarray )
```

Definition at line 1184 of file [if.c](#).

4.60.2.11 SwitchSplitMerge()

```
void SwitchSplitMerge (
    SWITCHTABLE * array,
    WORD num )
```

Definition at line 1213 of file [if.c](#).

4.61 if.c

[Go to the documentation of this file.](#)

```

00001
00005 /* #[ License : */
00006 /*
00007 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00008 *   When using this file you are requested to refer to the publication
00009 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00010 *   This is considered a matter of courtesy as the development was paid
00011 *   for by FOM the Dutch physics granting agency and we would like to
00012 *   be able to track its scientific use to convince FOM of its value
00013 *   for the community.
00014 *
00015 *   This file is part of FORM.
00016 *
00017 *   FORM is free software: you can redistribute it and/or modify it under the
00018 *   terms of the GNU General Public License as published by the Free Software
00019 *   Foundation, either version 3 of the License, or (at your option) any later
00020 *   version.
00021 *
00022 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00023 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00024 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00025 *   details.
00026 *
00027 *   You should have received a copy of the GNU General Public License along
00028 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00029 */
00030 /* #[ License : */
00031 /*
00032     #[ Includes : if.c
00033 */
00034
00035 #include "form3.h"
00036
00037 /*
00038     #[ Includes :
00039     #[ If statement :
00040         #[ Syntax :
00041
00042         The `if' is a conglomerate of statements: if,else,endif
00043
00044         The if consists in principle of:
00045
00046         if ( number );
00047             statements
00048         else;
00049             statements
00050         endif;
00051
00052         The first set is taken when number != 0.
00053         The else is not mandatory.
00054         TRUE = 1 and FALSE = 0
00055
00056         The number can be built up via a logical expression:
00057
00058             expr1 condition expr2
00059
00060         each expression can be a subexpression again. It has to be
00061         enclosed in parentheses in that case.
00062         Conditions are:
00063             >, >=, <, <=, ==, !=, ||, &&
00064
00065         When Expressions are chained evaluation is from left to right,
00066         independent of whether this indicates nonsense.
00067         if ( a || b || c || d ); is a perfectly normal statement.
00068         if ( a >= b || c == d ); would be messed up. This should be:
00069         if ( ( a >= b ) || ( c == d ) );
00070
00071         The building blocks of the Expressions are:
00072
00073             Match(option,pattern)   The number of times pattern fits in term_
00074             Count(...)              The count value of term_
00075             Coeff[icient]           The coefficient of term_
00076             FindLoop(options)       Are there loops (as in ReplaceLoop).
00077
00078         Implementation for internal notation:
00079
00080         TYPEIF,length,gotolevel(if fail),EXPRTYPE,length,.....
00081
00082         EXPRTYPE can be:
00083             SHORTNUMBER             ->,4,sign,size
00084             LONGNUMBER              ->,{ncoef+2|,ncoef,numer,denom
00085             MATCH                   ->,patternsiz+3,keyword,pattern

```

```

00086      MULTIPLEOF      ->,3,thenumber
00087      COUNT           ->,countsiz+2,countinfo
00088      TYPEFINDLOOP    ->,7 (findloop info)
00089      COEFFICIENT      ->,2
00090      IFDOLLAR         ->,3,dollarnumber
00091      SUBEXPR          ->,size,dummy,size1,EXPRTYPE,length,...
00092                      ,2,condition1,size2,...
00093                      This is like functions.
00094
00095      Note that there must be a restriction to the number of nestings
00096      of parentheses in an if statement. It has been set to 10.
00097
00098      The syntax of match corresponds to the syntax of the left side
00099      of an id statement. The only difference is the keyword
00100      MATCH vs TYPEIDNEW.
00101
00102      #] Syntax :
00103      #[ GetIfDollarNum :
00104  */
00105
00106  WORD GetIfDollarNum(WORD *ifp, WORD *ifstop)
00107  {
00108      DOLLARS d;
00109      WORD num, *w;
00110      if ( ifp[2] < 0 ) { return(-ifp[2]-1); }
00111      d = Dollars+ifp[2];
00112      if ( ifp+3 < ifstop && ifp[3] == IFDOLLAREXTRA ) {
00113          if ( d->nfactors == 0 ) {
00114              MLOCK(ErrorMessageLock);
00115              MesPrint("Attempt to use a factor of an unfactored $-variable");
00116              MUNLOCK(ErrorMessageLock);
00117              Terminate(-1);
00118          }
00119          num = GetIfDollarNum(ifp+3,ifstop);
00120          if ( num > d->nfactors ) {
00121              MLOCK(ErrorMessageLock);
00122              MesPrint("Dollar factor number %s out of range",num);
00123              MUNLOCK(ErrorMessageLock);
00124              Terminate(-1);
00125          }
00126          if ( num == 0 ) {
00127              return(d->nfactors);
00128          }
00129          w = d->factors[num-1].where;
00130          if ( w == 0 ) return(d->factors[num].value);
00131      getnumber:;
00132          if ( *w == 0 ) return(0);
00133          if ( *w == 4 && w[3] == 3 && w[2] == 1 && w[1] < MAXPOSITIVE && w[4] == 0 ) {
00134              return(w[1]);
00135          }
00136          if ( ( w[w[0]] != 0 ) || ( ABS(w[w[0]-1]) != w[0]-1 ) ) {
00137              MLOCK(ErrorMessageLock);
00138              MesPrint("Dollar factor number expected but found expression");
00139              MUNLOCK(ErrorMessageLock);
00140              Terminate(-1);
00141          }
00142          else {
00143              MLOCK(ErrorMessageLock);
00144              MesPrint("Dollar factor number out of range");
00145              MUNLOCK(ErrorMessageLock);
00146              Terminate(-1);
00147          }
00148          return(0);
00149      }
00150  /*
00151      Now we have just a dollar and should evaluate that into a short number
00152  */
00153      if ( d->type == DOLZERO ) {
00154          return(0);
00155      }
00156      else if ( d->type == DOLNUMBER || d->type == DOLTERMS ) {
00157          w = d->where; goto getnumber;
00158      }
00159      else {
00160          MLOCK(ErrorMessageLock);
00161          MesPrint("Dollar factor number is wrong type");
00162          MUNLOCK(ErrorMessageLock);
00163          Terminate(-1);
00164          return(0);
00165      }
00166  }
00167
00168  /*
00169      #] GetIfDollarNum :
00170      #[ FindVar :
00171  */
00172

```

```

00173 int FindVar(WORD *v, WORD *term)
00174 {
00175     WORD *t, *tstop, *m, *mstop, *f, *fstop, *a, *astop;
00176     GETSTOP(term,tstop);
00177     t = term+1;
00178     while ( t < tstop ) {
00179         if ( *v == *t && *v < FUNCTION ) { /* VECTOR, INDEX, SYMBOL, DOTPRODUCT */
00180             switch ( *v ) {
00181                 case SYMBOL:
00182                     m = t+2; mstop = t+t[1];
00183                     while ( m < mstop ) {
00184                         if ( *m == v[1] ) return(1);
00185                         m += 2;
00186                     }
00187                     break;
00188                 case INDEX:
00189                 case VECTOR:
00190 InVe:
00191                     m = t+2; mstop = t+t[1];
00192                     while ( m < mstop ) {
00193                         if ( *m == v[1] ) return(1);
00194                         m++;
00195                     }
00196                     break;
00197                 case DOTPRODUCT:
00198                     m = t+2; mstop = t+t[1];
00199                     while ( m < mstop ) {
00200                         if ( *m == v[1] && m[1] == v[2] ) return(1);
00201                         if ( *m == v[2] && m[1] == v[1] ) return(1);
00202                         m += 3;
00203                     }
00204                     break;
00205             }
00206         }
00207         else if ( *v == VECTOR && *t == INDEX ) goto InVe;
00208         else if ( *v == INDEX && *t == VECTOR ) goto InVe;
00209         else if ( ( *v == VECTOR || *v == INDEX ) && *t == DOTPRODUCT ) {
00210             m = t+2; mstop = t+t[1];
00211             while ( m < mstop ) {
00212                 if ( v[1] == m[0] || v[1] == m[1] ) return(1);
00213                 m += 3;
00214             }
00215         }
00216         else if ( *t >= FUNCTION ) {
00217             if ( *v == FUNCTION && v[1] == *t ) return(1);
00218             if ( functions[*t-FUNCTION].spec > 0 ) {
00219                 if ( *v == VECTOR || *v == INDEX ) { /* we need to check arguments */
00220                     int i;
00221                     for ( i = FUNHEAD; i < t[1]; i++ ) {
00222                         if ( v[1] == t[i] ) return(1);
00223                     }
00224                 }
00225             }
00226         }
00227         else {
00228             fstop = t + t[1]; f = t + FUNHEAD;
00229             while ( f < fstop ) { /* Do the arguments one by one */
00230                 if ( *f <= 0 ) {
00231                     switch ( *f ) {
00232                         case -SYMBOL:
00233                             if ( *v == SYMBOL && v[1] == f[1] ) return(1);
00234                             f += 2;
00235                             break;
00236                         case -VECTOR:
00237                         case -MINVECTOR:
00238                         case -INDEX:
00239                             if ( ( *v == VECTOR || *v == INDEX )
00240                                 && ( v[1] == f[1] ) ) return(1);
00241                             f += 2;
00242                             break;
00243                         case -SNUMBER:
00244                             f += 2;
00245                             break;
00246                         default:
00247                             if ( *v == FUNCTION && v[1] == -*f && *f <= -FUNCTION ) return(1);
00248                             if ( *f <= -FUNCTION ) f++;
00249                             else f += 2;
00250                             break;
00251                     }
00252                 }
00253                 else {
00254                     a = f + ARGHEAD; astop = f + *f;
00255                     while ( a < astop ) {
00256                         if ( FindVar(v,a) == 1 ) return(1);
00257                         a += *a;
00258                     }
00259                     f = astop;
00260                 }
00261             }
00262         }
00263     }
00264 }

```

```

00260         }
00261     }
00262 }
00263     t += t[1];
00264 }
00265     return(0);
00266 }
00267
00268 /*
00269     #] FindVar :
00270     #[ DoIfStatement :           WORD DoIfStatement(PHEAD ifcode,term)
00271
00272     The execution time part of the if-statement.
00273     The arguments are a pointer to the TYPEIF and a pointer to the term.
00274     The answer is either 1 (success) or 0 (fail).
00275     The calling routine can figure out where to go in case of failure
00276     by picking up gotolevel.
00277     Note that the whole setup asks for recursions.
00278 */
00279
00280 WORD DoIfStatement(PHEAD WORD *ifcode, WORD *term)
00281 {
00282     GETBIDENTITY
00283     WORD *ifstop, *ifp;
00284     UWORD *coef1 = 0, *coef2, *coef3, *cc;
00285     WORD ncoef1, ncoef2, ncoef3, i = 0, first, *r, acoef, ismull, ismul2, j;
00286     UWORD *Spac1, *Spac2;
00287     ifstop = ifcode + ifcode[1];
00288     ifp = ifcode + 3;
00289     if ( ifp >= ifstop ) return(1);
00290     if ( ( ifp + ifp[1] ) >= ifstop ) {
00291         switch ( *ifp ) {
00292             case LONGNUMBER:
00293                 if ( ifp[2] ) return(1);
00294                 else return(0);
00295             case MATCH:
00296             case TYPEIF:
00297                 if ( HowMany(BHEAD ifp,term) ) return(1);
00298                 else return(0);
00299             case TYPEFINDLOOP:
00300                 if ( Lus(term,ifp[3],ifp[4],ifp[5],ifp[6],ifp[2]) ) return(1);
00301                 else return(0);
00302             case TYPECOUNT:
00303                 if ( CountDo(term,ifp) ) return(1);
00304                 else return(0);
00305             case COEFFI:
00306             case MULTIPLEOF:
00307                 return(1);
00308             case IFDOLLAR:
00309                 {
00310                     DOLLARS d = Dollars + ifp[2];
00311 #ifndef WITHPTHREADS
00312                     int nummodopt, dtype = -1;
00313                     if ( AS.MultiThreaded ) {
00314                         for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
00315                             if ( ifp[2] == ModOptdollars[nummodopt].number ) break;
00316                         }
00317                         if ( nummodopt < NumModOptdollars ) {
00318                             dtype = ModOptdollars[nummodopt].type;
00319                             if ( dtype == MODLOCAL ) {
00320                                 d = ModOptdollars[nummodopt].dstruct+AT.identity;
00321                             }
00322                         }
00323                     }
00324                     dtype = d->type;
00325 #else
00326                     int dtype = d->type; /* We use dtype to make the operation atomic */
00327 #endif
00328                     if ( dtype == DOLZERO ) return(0);
00329                     if ( dtype == DOLUNDEFINED ) {
00330                         if ( AC.UnsureDollarMode == 0 ) {
00331                             MesPrint("%s is undefined",AC.dollarnames->namebuffer+d->name);
00332                             Terminate(-1);
00333                         }
00334                     }
00335                 }
00336                 return(1);
00337             case IFEXPRESSION:
00338                 r = ifp+2; j = ifp[1] - 2;
00339                 while ( --j >= 0 ) {
00340                     if ( *r == AR.CurExpr ) return(1);
00341                     r++;
00342                 }
00343                 return(0);
00344             case IFISFACTORIZED:
00345                 r = ifp+2; j = ifp[1] - 2;
00346                 if ( j == 0 ) {

```



```

00347         if ( ( Expressions[AR.CurExpr].vflags & ISFACTORIZED ) != 0 )
00348             return(1);
00349         else
00350             return(0);
00351     }
00352     while ( --j >= 0 ) {
00353         if ( ( Expressions[*r].vflags & ISFACTORIZED ) == 0 ) return(0);
00354         r++;
00355     }
00356     return(1);
00357 case IFOCCURS:
00358     {
00359         WORD *OccStop = ifp + ifp[1];
00360         ifp += 2;
00361         while ( ifp < OccStop ) {
00362             if ( FindVar(ifp,term) == 1 ) return(1);
00363             if ( *ifp == DOTPRODUCT ) ifp += 3;
00364             else ifp += 2;
00365         }
00366     }
00367     return(0);
00368 case IFUSERFLAG:
00369     if ( ( Expressions[AR.CurExpr].uflags & (1 << ifp[2]) ) != 0 )
00370         return(1);
00371     return(0);
00372 default:
00373     /*
00374         Now we have a subexpression. Test first for one with a single item.
00375     */
00376     if ( ifp[3] == ( ifp[1] + 3 ) ) return(DoIfStatement(BHEAD ifp,term));
00377     ifstop = ifp + ifp[1];
00378     ifp += 3;
00379     break;
00380 }
00381 }
00382 /*
00383 Here is the composite condition.
00384 */
00385 coef3 = NumberMalloc("DoIfStatement");
00386 Spac1 = NumberMalloc("DoIfStatement");
00387 Spac2 = (UWORD *) (TermMalloc("DoIfStatement"));
00388 ncoef1 = 0; first = 1; ismull = 0;
00389 do {
00390     if ( !first ) {
00391         ifp += 2;
00392         if ( ifp[-2] == ORCOND && ncoef1 ) {
00393             coef1 = Spac1;
00394             ncoef1 = 1; coef1[0] = coef1[1] = 1;
00395             goto SkipCond;
00396         }
00397         if ( ifp[-2] == ANDCOND && !ncoef1 ) goto SkipCond;
00398     }
00399     coef2 = Spac2;
00400     ncoef2 = 1;
00401     ismul2 = 0;
00402     switch ( *ifp ) {
00403     case LONGNUMBER:
00404         ncoef2 = ifp[2];
00405         j = 2*(ABS(ncoef2));
00406         cc = (UWORD *) (ifp + 3);
00407         for ( i = 0; i < j; i++ ) coef2[i] = cc[i];
00408         break;
00409 #ifdef WITHFLOAT
00410     case IFFLOATNUMBER:
00411         /*
00412             The sloppy solution is: Convert to rational.
00413             This way we can write it over coef2,ncoef2
00414         */
00415         ncoef2 = FloatFunToRat(BHEAD coef2,ifp);
00416         break;
00417 #endif
00418     case MATCH:
00419     case TYPEIF:
00420         coef2[0] = HowMany(BHEAD ifp,term);
00421         coef2[1] = 1;
00422         if ( coef2[0] == 0 ) ncoef2 = 0;
00423         break;
00424     case TYPECOUNT:
00425         acoef = CountDo(term,ifp);
00426         coef2[0] = ABS(acoef);
00427         coef2[1] = 1;
00428         if ( acoef == 0 ) ncoef2 = 0;
00429         else if ( acoef < 0 ) ncoef2 = -1;
00430         break;
00431     case TYPEFINDLOOP:
00432         acoef = Lus(term,ifp[3],ifp[4],ifp[5],ifp[6],ifp[2]);
00433         coef2[0] = ABS(acoef);

```

```

00434         coef2[1] = 1;
00435         if ( acoef == 0 ) ncoef2 = 0;
00436         else if ( acoef < 0 ) ncoef2 = -1;
00437         break;
00438     case COEFFI:
00439         r = term + *term;
00440         ncoef2 = r[-1];
00441         i = ABS(ncoef2);
00442         cc = (UWORD *) (r - i);
00443         if ( ncoef2 < 0 ) ncoef2 = (ncoef2+1)»1;
00444         else ncoef2 = (ncoef2-1)»1;
00445         i--; for ( j = 0; j < i; j++ ) coef2[j] = cc[j];
00446         break;
00447     case SUBEXPR:
00448         ncoef2 = coef2[0] = DoIfStatement(BHEAD ifp,term);
00449         coef2[1] = 1;
00450         break;
00451     case MULTIPLEOF:
00452         ncoef2 = 1;
00453         coef2[0] = ifp[2];
00454         coef2[1] = 1;
00455         ismul2 = 1;
00456         break;
00457     case IFDOLLAREXTRA:
00458         break;
00459     case IFDOLLAR:
00460         {
00461         /*
00462             We need to abstract a long rational in coef2
00463             with length ncoef2. What if that cannot be done?
00464         */
00465             DOLLARS d = Dollars + ifp[2];
00466 #ifdef WITHPTHREADS
00467             int nummodopt, dtype = -1;
00468             if ( AS.MultiThreaded ) {
00469                 for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
00470                     if ( ifp[2] == ModOptdollars[nummodopt].number ) break;
00471                 }
00472                 if ( nummodopt < NumModOptdollars ) {
00473                     dtype = ModOptdollars[nummodopt].type;
00474                     if ( dtype == MODLOCAL ) {
00475                         d = ModOptdollars[nummodopt].dstruct+AT.identity;
00476                     }
00477                     else {
00478                         LOCK(d->pthreadlockread);
00479                     }
00480                 }
00481             }
00482 #endif
00483         /*
00484             We have to pick up the IFDOLLAREXTRA pieces for [1], [$y] etc.
00485         */
00486         if ( ifp+3 < ifstop && ifp[3] == IFDOLLAREXTRA ) {
00487             if ( d->nfactors == 0 ) {
00488                 MLOCK(ErrorMessageLock);
00489                 MesPrint("Attempt to use a factor of an unfactored $-variable");
00490                 MUNLOCK(ErrorMessageLock);
00491                 Terminate(-1);
00492             } {
00493                 WORD num = GetIfDollarNum(ifp+3,ifstop);
00494                 WORD *w;
00495                 while ( ifp+3 < ifstop && ifp[3] == IFDOLLAREXTRA ) ifp += 3;
00496                 if ( num > d->nfactors ) {
00497                     MLOCK(ErrorMessageLock);
00498                     MesPrint("Dollar factor number %s out of range",num);
00499                     MUNLOCK(ErrorMessageLock);
00500                     Terminate(-1);
00501                 }
00502                 if ( num == 0 ) {
00503                     ncoef2 = 1; coef2[0] = d->nfactors; coef2[1] = 1;
00504                     break;
00505                 }
00506                 w = d->factors[num-1].where;
00507                 if ( w == 0 ) {
00508                     if ( d->factors[num-1].value < 0 ) {
00509                         ncoef2 = -1; coef2[0] = -d->factors[num-1].value; coef2[1] = 1;
00510                     }
00511                     else {
00512                         ncoef2 = 1; coef2[0] = d->factors[num-1].value; coef2[1] = 1;
00513                     }
00514                     break;
00515                 }
00516                 if ( w[*w] == 0 ) {
00517                     r = w + *w - 1;
00518                     i = ABS(*r);
00519                     if ( i == ( *w-1 ) ) {
00520                         ncoef2 = (i-1)/2;

```

```

00521             if ( *r < 0 ) ncoef2 = -ncoef2;
00522             i--; cc = coef2; r = w + 1;
00523             while ( --i >= 0 ) *cc++ = (UWORD) (*r++);
00524             break;
00525         }
00526     }
00527     goto generic;
00528 }
00529 }
00530 else {
00531     switch ( d->type ) {
00532     case DOLUNDEFINED:
00533         if ( AC.UnsureDollarMode == 0 ) {
00534             #ifdef WITHPTHREADS
00535                 if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
00536             #endif
00537             MLOCK(ErrorMessageLock);
00538             MesPrint("%s is undefined", AC.dollarnames->namebuffer+d->name);
00539             MUNLOCK(ErrorMessageLock);
00540             Terminate(-1);
00541         }
00542         ncoef2 = 0; coef2[0] = 0; coef2[1] = 1;
00543         break;
00544     case DOLZERO:
00545         ncoef2 = coef2[0] = 0; coef2[1] = 1;
00546         break;
00547     case DOLSUBTERM:
00548         if ( d->where[0] != INDEX || d->where[1] != 3
00549             || d->where[2] < 0 || d->where[2] >= AM.OffsetIndex ) {
00550             if ( AC.UnsureDollarMode == 0 ) {
00551                 #ifdef WITHPTHREADS
00552                     if ( dtype > 0 && dtype != MODLOCAL ) {
00553                         UNLOCK(d->pthreadlockread); }
00554                 #endif
00555                 MLOCK(ErrorMessageLock);
00556                 MesPrint("%s is of wrong
00557 type", AC.dollarnames->namebuffer+d->name);
00558                 MUNLOCK(ErrorMessageLock);
00559                 Terminate(-1);
00560             }
00561             ncoef2 = 0; coef2[0] = 0; coef2[1] = 1;
00562             break;
00563             /* fall through */
00564         case DOLINDEX:
00565             if ( d->index == 0 ) {
00566                 ncoef2 = coef2[0] = 0; coef2[1] = 1;
00567             }
00568             else if ( d->index > 0 && d->index < AM.OffsetIndex ) {
00569                 ncoef2 = 1; coef2[0] = d->index; coef2[1] = 1;
00570             }
00571             else if ( AC.UnsureDollarMode == 0 ) {
00572                 #ifdef WITHPTHREADS
00573                     if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
00574                 #endif
00575                 MLOCK(ErrorMessageLock);
00576                 MesPrint("%s is of wrong type", AC.dollarnames->namebuffer+d->name);
00577                 MUNLOCK(ErrorMessageLock);
00578                 Terminate(-1);
00579             }
00580             ncoef2 = coef2[0] = 0; coef2[1] = 1;
00581             break;
00582         case DOLWILDARGS:
00583             if ( d->where[0] <= -FUNCTION ||
00584                 ( d->where[0] < 0 && d->where[2] != 0 )
00585                 || ( d->where[0] > 0 && d->where[d->where[0]] != 0 )
00586             ) {
00587                 if ( AC.UnsureDollarMode == 0 ) {
00588                     #ifdef WITHPTHREADS
00589                         if ( dtype > 0 && dtype != MODLOCAL ) {
00590                             UNLOCK(d->pthreadlockread); }
00591                     #endif
00592                     MLOCK(ErrorMessageLock);
00593                     MesPrint("%s is of wrong
00594 type", AC.dollarnames->namebuffer+d->name);
00595                     MUNLOCK(ErrorMessageLock);
00596                     Terminate(-1);
00597                 }
00598                 ncoef2 = coef2[0] = 0; coef2[1] = 1;
00599                 break;
00600             }
00601             /* fall through */
00602         case DOLARGUMENT:
00603             if ( d->where[0] == -SNUMBER ) {
00604                 if ( d->where[1] == 0 ) {
00605                     ncoef2 = coef2[0] = 0;

```

```

00604         }
00605         else if ( d->where[1] < 0 ) {
00606             ncoef2 = -1;
00607             coef2[0] = -d->where[1];
00608         }
00609         else {
00610             ncoef2 = 1;
00611             coef2[0] = d->where[1];
00612         }
00613         coef2[1] = 1;
00614     }
00615     else if ( d->where[0] == -INDEX
00616     && d->where[1] >= 0 && d->where[1] < AM.OffsetIndex ) {
00617         if ( d->where[1] == 0 ) {
00618             ncoef2 = coef2[0] = 0; coef2[1] = 1;
00619         }
00620         else {
00621             ncoef2 = 1; coef2[0] = d->where[1];
00622             coef2[1] = 1;
00623         }
00624     }
00625     else if ( d->where[0] > 0
00626     && d->where[ARGHEAD] == (d->where[0]-ARGHEAD)
00627     && ABS(d->where[d->where[0]-1]) ==
00628         (d->where[0] - ARGHEAD-1) ) {
00629         i = d->where[d->where[0]-1];
00630         ncoef2 = (ABS(i)-1)/2;
00631         if ( i < 0 ) { ncoef2 = -ncoef2; i = -i; }
00632         i--; cc = coef2; r = d->where + ARGHEAD+1;
00633         while ( --i >= 0 ) *cc++ = (UWORD)(*r++);
00634     }
00635     else {
00636         if ( AC.UnsureDollarMode == 0 ) {
00637             #ifdef WITHPTHREADS
00638                 if ( dtype > 0 && dtype != MODLOCAL ) {
00639                     UNLOCK(d->pthreadlockread); }
00640             #endif
00641             MLOCK(ErrorMessageLock);
00642             MesPrint("%s is of wrong
type",AC.dollarnames->namebuffer+d->name);
00643             MUNLOCK(ErrorMessageLock);
00644             Terminate(-1);
00645             ncoef2 = 0; coef2[0] = 0; coef2[1] = 1;
00646         }
00647         break;
00648     case DOLNUMBER:
00649     case DOLTERMS:
00650         if ( d->where[d->where[0]] == 0 ) {
00651             r = d->where + d->where[0]-1;
00652             i = ABS(*r);
00653             if ( i == ( d->where[0]-1 ) ) {
00654                 ncoef2 = (i-1)/2;
00655                 if ( *r < 0 ) ncoef2 = -ncoef2;
00656                 i--; cc = coef2; r = d->where + 1;
00657                 while ( --i >= 0 ) *cc++ = (UWORD)(*r++);
00658                 break;
00659             }
00660         }
00661         generic;;
00662         if ( AC.UnsureDollarMode == 0 ) {
00663             #ifdef WITHPTHREADS
00664                 if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
00665             #endif
00666             MLOCK(ErrorMessageLock);
00667             MesPrint("%s is of wrong type",AC.dollarnames->namebuffer+d->name);
00668             MUNLOCK(ErrorMessageLock);
00669             Terminate(-1);
00670             ncoef2 = 0; coef2[0] = 0; coef2[1] = 1;
00671             break;
00672         }
00673     }
00674     }
00675     #ifdef WITHPTHREADS
00676     if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
00677     #endif
00678     }
00679     break;
00680 case IFEXPRESSION:
00681     r = ifp+2; j = ifp[1] - 2; ncoef2 = 0;
00682     while ( --j >= 0 ) {
00683         if ( *r == AR.CurExpr ) { ncoef2 = 1; break; }
00684         r++;
00685     }
00686     coef2[0] = ncoef2;
00687     coef2[1] = 1;
00688     break;

```

```

00689         case IFISFACTORIZED:
00690             r = ifp+2; j = ifp[1] - 2;
00691             if ( j == 0 ) {
00692                 ncoef2 = 0;
00693                 if ( ( Expressions[AR.CurExpr].vflags & ISFACTORIZED ) != 0 ) {
00694                     ncoef2 = 1;
00695                 }
00696             }
00697             else {
00698                 ncoef2 = 1;
00699                 while ( --j >= 0 ) {
00700                     if ( ( Expressions[*r].vflags & ISFACTORIZED ) == 0 ) {
00701                         ncoef2 = 0;
00702                         break;
00703                     }
00704                     r++;
00705                 }
00706             }
00707             coef2[0] = ncoef2;
00708             coef2[1] = 1;
00709             break;
00710         case IFOCCURS:
00711             {
00712                 WORD *OccStop = ifp + ifp[1], *ifpp = ifp+2;
00713                 ncoef2 = 0;
00714                 while ( ifpp < OccStop ) {
00715                     if ( FindVar(ifpp,term) == 1 ) {
00716                         ncoef2 = 1; break;
00717                     }
00718                     if ( *ifpp == DOTPRODUCT ) ifp += 3;
00719                     else ifpp += 2;
00720                 }
00721                 coef2[0] = ncoef2;
00722                 coef2[1] = 1;
00723             }
00724             break;
00725         case IFUSERFLAG:
00726             {
00727                 ncoef2 = 0;
00728                 if ( ( Expressions[AR.CurExpr].uflags & (1 « ifp[2]) ) != 0 )
00729                     ncoef2 = 1;
00730                 coef2[0] = ncoef2;
00731                 coef2[1] = 1;
00732             }
00733             break;
00734         default:
00735             break;
00736     }
00737     if ( !first ) {
00738         if ( ifp[-2] != ORCOND && ifp[-2] != ANDCOND ) {
00739             if ( ( ifp[-2] == EQUAL || ifp[-2] == NOTEQUAL ) &&
00740                 ( ismul2 || ismul1 ) ) {
00741                 if ( ismul1 && ismul2 ) {
00742                     if ( coef1[0] == coef2[0] ) i = 1;
00743                     else i = 0;
00744                 }
00745                 else {
00746                     if ( ismul1 ) {
00747                         if ( ncoef2 )
00748                             Divvy(BHEAD coef2,&ncoef2,coef1,ncoef1);
00749                         cc = coef2; ncoef3 = ncoef2;
00750                     }
00751                     else {
00752                         if ( ncoef1 )
00753                             Divvy(BHEAD coef1,&ncoef1,coef2,ncoef2);
00754                         cc = coef1; ncoef3 = ncoef1;
00755                     }
00756                     if ( ncoef3 < 0 ) ncoef3 = -ncoef3;
00757                     if ( ncoef3 == 0 ) {
00758                         if ( ifp[-2] == EQUAL ) i = 1;
00759                         else i = 0;
00760                     }
00761                     else if ( cc[ncoef3] != 1 ) {
00762                         if ( ifp[-2] == EQUAL ) i = 0;
00763                         else i = 1;
00764                     }
00765                     else {
00766                         for ( j = 1; j < ncoef3; j++ ) {
00767                             if ( cc[ncoef3+j] != 0 ) break;
00768                         }
00769                         if ( j < ncoef3 ) {
00770                             if ( ifp[-2] == EQUAL ) i = 0;
00771                             else i = 1;
00772                         }
00773                         else if ( ifp[-2] == EQUAL ) i = 1;
00774                         else i = 0;
00775                     }
00776                 }
00777             }

```

```

00776         }
00777         goto donemul;
00778     }
00779     else if ( AddRat(BHEAD coef1,ncoef1,coef2,-ncoef2,coef3,&ncoef3) ) {
00780         NumberFree(coef3,"DoIfStatement"); NumberFree(Spac1,"DoIfStatement");
TermFree(Spac2,"DoIfStatement");
00781         MesCall("DoIfStatement"); return(-1);
00782     }
00783     switch ( ifp[-2] ) {
00784     case GREATER:
00785         if ( ncoef3 > 0 ) i = 1;
00786         else i = 0;
00787         break;
00788     case GREATEREQUAL:
00789         if ( ncoef3 >= 0 ) i = 1;
00790         else i = 0;
00791         break;
00792     case LESS:
00793         if ( ncoef3 < 0 ) i = 1;
00794         else i = 0;
00795         break;
00796     case LESSEQUAL:
00797         if ( ncoef3 <= 0 ) i = 1;
00798         else i = 0;
00799         break;
00800     case EQUAL:
00801         if ( ncoef3 == 0 ) i = 1;
00802         else i = 0;
00803         break;
00804     case NOTEQUAL:
00805         if ( ncoef3 != 0 ) i = 1;
00806         else i = 0;
00807         break;
00808     }
00809 donemul:    if ( i ) { ncoef2 = 1; coef2 = Spac2; coef2[0] = coef2[1] = 1; }
00810             else ncoef2 = 0;
00811             ismul1 = ismul2 = 0;
00812         }
00813     }
00814     else {
00815         first = 0;
00816     }
00817     coef1 = Spac1;
00818     i = 2*ABS(ncoef2);
00819     for ( j = 0; j < i; j++ ) coef1[j] = coef2[j];
00820     ncoef1 = ncoef2;
00821 SkipCond:
00822     ifp += ifp[1];
00823     } while ( ifp < ifstop );
00824
00825     NumberFree(coef3,"DoIfStatement"); NumberFree(Spac1,"DoIfStatement");
TermFree(Spac2,"DoIfStatement");
00826     if ( ncoef1 ) return(1);
00827     else return(0);
00828 }
00829
00830 /*
00831     #] DoIfStatement :
00832     #[ HowMany :                WORD HowMany(ifcode,term)
00833
00834     Returns the number of times that the pattern in ifcode
00835     can be taken out from term. There is a subkey in ifcode[2];
00836     The notation is identical to the lhs of an id statement.
00837     Most of the code comes from TestMatch.
00838 */
00839
00840 WORD HowMany(PHEAD WORD *ifcode, WORD *term)
00841 {
00842     GETBIDENTITY
00843     WORD *m, *t, *r, *w, power, RetVal, i, topje, *newterm;
00844     WORD *OldWork, *ww, *mm;
00845     int *RepSto, RepVal;
00846     int numdollars = 0;
00847     m = ifcode + IDHEAD;
00848     AN.FullProto = m;
00849     AN.WildValue = w = m + SUBEXPSIZE;
00850     m += m[1];
00851     AN.WildStop = m;
00852     OldWork = AT.WorkPointer;
00853     if ( ( ifcode[4] & 1 ) != 0 ) { /* We have at least one dollar in the pattern */
00854         AR.Eside = LHSIDEX;
00855         ww = AT.WorkPointer; i = m[0]; mm = m;
00856         NCOPY(ww,mm,i);
00857         *OldWork += 3;
00858         *ww++ = 1; *ww++ = 1; *ww++ = 3;
00859         AT.WorkPointer = ww;
00860         RepSto = AN.RepPoint;

```

```

00861     RepVal = *RepSto;
00862     NewSort(BHEAD0);
00863     if ( Generator(BHEAD OldWork,AR.Cnumlhs) ) {
00864         LowerSortLevel();
00865         *RepSto = RepVal;
00866         AN.RepPoint = RepSto;
00867         AT.WorkPointer = OldWork;
00868         return(-1);
00869     }
00870     AT.WorkPointer = ww;
00871     if ( EndSort(BHEAD ww,0) < 0 ) {}
00872     *RepSto = RepVal;
00873     AN.RepPoint = RepSto;
00874     if ( *ww == 0 || *(ww+ww) != 0 ) {
00875         if ( AP.lhdollarerror == 0 ) {
00876             MLOCK(ErrorMessageLock);
00877             MesPrint("&LHS must be one term");
00878             MUNLOCK(ErrorMessageLock);
00879             AP.lhdollarerror = 1;
00880         }
00881         AT.WorkPointer = OldWork;
00882         return(-1);
00883     }
00884     m = ww; AT.WorkPointer = ww = m + *m;
00885     if ( m[*m-1] < 0 ) { m[*m-1] = -m[*m-1]; }
00886     *m -= m[*m-1];
00887     AR.Eside = RHSIDE;
00888 }
00889 else {
00890     ww = term + *term;
00891     if ( AT.WorkPointer < ww ) AT.WorkPointer = ww;
00892 }
00893 ClearWild(BHEAD0);
00894 while ( w < AN.WildStop ) {
00895     if ( *w == LOADDOLLAR ) numdollars++;
00896     w += w[1];
00897 }
00898 AN.RepFunNum = 0;
00899 AN.RepFunList = AT.WorkPointer;
00900 AT.WorkPointer = (WORD *)((UBYTE *) (AT.WorkPointer)) + AM.MaxTer);
00901 topje = cbuf[AT.ebufnum].numrhs;
00902 if ( AT.WorkPointer >= AT.WorkTop ) {
00903     MLOCK(ErrorMessageLock);
00904     MesWork();
00905     MUNLOCK(ErrorMessageLock);
00906     return(-1);
00907 }
00908 AN.DisOrderFlag = ifcode[2] & SUBDISORDER;
00909 switch ( ifcode[2] & (~SUBDISORDER) ) {
00910     case SUBONLY :
00911         /* Must be an exact match */
00912         AN.UseFindOnly = 1; AN.ForFindOnly = 0;
00913     /*
00914     Copy the term first to scratchterm. This is needed
00915     because of the Substitute.
00916     */
00917     i = *term;
00918     t = term; newterm = r = AT.WorkPointer;
00919     NCOPY(r,t,i); AT.WorkPointer = r;
00920     RetVal = 0;
00921     if ( FindRest(BHEAD newterm,m) && ( AN.UsedOtherFind ||
00922         FindOnly(BHEAD newterm,m) ) ) {
00923         Substitute(BHEAD newterm,m,1);
00924         if ( numdollars ) {
00925             WildDollars(BHEAD (WORD *)0);
00926             numdollars = 0;
00927         }
00928         ClearWild(BHEAD0);
00929         RetVal = 1;
00930     }
00931     else RetVal = 0;
00932     break;
00933     case SUBMANY :
00934     /*
00935     Copy the term first to scratchterm. This is needed
00936     because of the Substitute.
00937     */
00938     i = *term;
00939     t = term; newterm = r = AT.WorkPointer;
00940     NCOPY(r,t,i); AT.WorkPointer = r;
00941     RetVal = 0;
00942     AN.UseFindOnly = 0;
00943     if ( ( power = FindRest(BHEAD newterm,m) ) > 0 ) {
00944         if ( ( power = FindOnce(BHEAD newterm,m) ) > 0 ) {
00945             AN.UseFindOnly = 0;
00946             do {
00947                 Substitute(BHEAD newterm,m,1);

```

```

00948         if ( numdollars ) {
00949             WildDollars(BHEAD (WORD *)0);
00950             numdollars = 0;
00951         }
00952         ClearWild(BHEAD0);
00953         RetVal++;
00954     } while ( FindRest(BHEAD newterm,m) && (
00955         AN.UsedOtherFind || FindOnce(BHEAD newterm,m) ) );
00956     }
00957     else if ( power < 0 ) {
00958         do {
00959             Substitute(BHEAD newterm,m,1);
00960             if ( numdollars ) {
00961                 WildDollars(BHEAD (WORD *)0);
00962                 numdollars = 0;
00963             }
00964             ClearWild(BHEAD0);
00965             RetVal++;
00966         } while ( FindRest(BHEAD newterm,m) );
00967     }
00968 }
00969 else if ( power < 0 ) {
00970     if ( FindOnce(BHEAD newterm,m) ) {
00971         do {
00972             Substitute(BHEAD newterm,m,1);
00973             if ( numdollars ) {
00974                 WildDollars(BHEAD (WORD *)0);
00975                 numdollars = 0;
00976             }
00977             ClearWild(BHEAD0);
00978         } while ( FindOnce(BHEAD newterm,m) );
00979         RetVal = 1;
00980     }
00981 }
00982 break;
00983 case SUBONCE :
00984 /*
00985     Copy the term first to scratchterm. This is needed
00986     because of the Substitute.
00987 */
00988     i = *term;
00989     t = term; newterm = r = AT.WorkPointer;
00990     NCOPY(r,t,i); AT.WorkPointer = r;
00991     RetVal = 0;
00992     AN.UseFindOnly = 0;
00993     if ( FindRest(BHEAD newterm,m) && ( AN.UsedOtherFind || FindOnce(BHEAD newterm,m) ) ) {
00994         Substitute(BHEAD newterm,m,1);
00995         if ( numdollars ) {
00996             WildDollars(BHEAD (WORD *)0);
00997             numdollars = 0;
00998         }
00999         ClearWild(BHEAD0);
01000         RetVal = 1;
01001     }
01002     else RetVal = 0;
01003     break;
01004 case SUBMULTI :
01005     RetVal = FindMulti(BHEAD term,m);
01006     break;
01007 case SUBVECTOR :
01008     RetVal = 0;
01009     for ( i = 0; i < *term; i++ ) ww[i] = term[i];
01010     while ( ( power = FindAll(BHEAD ww,m,AR.Cnumlhs,ifcode) ) != 0 ) { RetVal += power; }
01011     break;
01012 case SUBSELECT :
01013     ifcode += IDHEAD; ifcode += ifcode[1]; ifcode += *ifcode;
01014     AN.UseFindOnly = 1; AN.ForFindOnly = ifcode;
01015     if ( FindRest(BHEAD term,m) && ( AN.UsedOtherFind ||
01016         FindOnly(BHEAD term,m) ) ) RetVal = 1;
01017     else RetVal = 0;
01018     break;
01019 default :
01020     RetVal = 0;
01021     break;
01022 }
01023 AT.WorkPointer = AN.RepFunList;
01024 cbuf[AT.ebufnum].numrhs = topje;
01025 return(RetVal);
01026 }
01027
01028 /*
01029     #] HowMany :
01030     #[ DoubleIfBuffers :
01031 */
01032
01033 void DoubleIfBuffers(void)
01034 {

```



```

01035     int newmax, i;
01036     WORD *newsumcheck;
01037     LONG *newheap, *newifcount;
01038     if ( AC.MaxIf == 0 ) newmax = 10;
01039     else newmax = 2*AC.MaxIf;
01040     newheap = (LONG *)Malloc1(sizeof(LONG)*(newmax+1),"IfHeap");
01041     newsumcheck = (WORD *)Malloc1(sizeof(WORD)*(newmax+1),"IfSumCheck");
01042     newifcount = (LONG *)Malloc1(sizeof(LONG)*(newmax+1),"IfCount");
01043     if ( AC.MaxIf ) {
01044         for ( i = 0; i < AC.MaxIf; i++ ) {
01045             newheap[i] = AC.IfHeap[i];
01046             newsumcheck[i] = AC.IfSumCheck[i];
01047             newifcount[i] = AC.IfCount[i];
01048         }
01049         AC.IfStack = (AC.IfStack-AC.IfHeap) + newheap;
01050         M_free(AC.IfHeap,"AC.IfHeap");
01051         M_free(AC.IfCount,"AC.IfCount");
01052         M_free(AC.IfSumCheck,"AC.IfSumCheck");
01053     }
01054     else {
01055         AC.IfStack = newheap;
01056     }
01057     AC.IfHeap = newheap;
01058     AC.IfSumCheck = newsumcheck;
01059     AC.IfCount = newifcount;
01060     AC.MaxIf = newmax;
01061 }
01062
01063 /*
01064     #] DoubleIfBuffers :
01065     #] If statement :
01066     #[ Switch statement :
01067     #[ DoSwitch :
01068 */
01069 int DoSwitch(PHEAD WORD *term, WORD *lhs)
01070 {
01071     /*
01072     For the moment we ignore the compiler buffer problems.
01073     */
01074     WORD numdollar = lhs[2];
01075     WORD ncase = DolToNumber(BHEAD numdollar);
01076     SWITCHTABLE *swtab = FindCase(lhs[3],ncase);
01077     return(Generator(BHEAD term,swtab->value));
01078 }
01079
01080 /*
01081     #] DoSwitch :
01082     #[ DoEndSwitch :
01083     */
01084 int DoEndSwitch(PHEAD WORD *term, WORD *lhs)
01085 {
01086     SWITCH *sw = AC.SwitchArray+lhs[2];
01087     return(Generator(BHEAD term,sw->endswitch.value+1));
01088 }
01089
01090 /*
01091     #] DoEndSwitch :
01092     #[ FindCase :
01093     */
01094 SWITCHTABLE *FindCase(WORD nswitch, WORD ncase)
01095 {
01096     /*
01097     First find the switch table and determine how we have to search.
01098     */
01099     SWITCH *sw = AC.SwitchArray+nswitch;
01100     WORD hi, lo, med;
01101     if ( sw->typetable == DENSETABLE ) {
01102         med = ncase - sw->caseoffset;
01103         if ( med >= sw->numcases || med < 0 ) return(&sw->defaultcase);
01104     }
01105     else {
01106         /*
01107         We need a binary search in the table.
01108         */
01109         if ( ncase > sw->maxcase || ncase < sw->mincase ) return(&sw->defaultcase);
01110         hi = sw->numcases-1; lo = 0;
01111         for(;;) {
01112             med = (hi+lo)/2;
01113             if ( ncase == sw->table[med].ncase ) break;
01114             else if ( ncase > sw->table[med].ncase ) {
01115                 lo = med+1;
01116             }
01117             else if ( lo > hi ) return(&sw->defaultcase);
01118         }
01119         else {

```

```

01122             hi = med-1;
01123             if ( hi < lo ) return(&sw->defaultcase);
01124         }
01125     }
01126 }
01127 return(&sw->table[med]);
01128 }
01129
01130 /*
01131     #] FindCase :
01132     #[ DoubleSwitchBuffers :
01133 */
01134
01135 int DoubleSwitchBuffers(void)
01136 {
01137     int newmax, i;
01138     SWITCH *newarray;
01139     WORD *newheap;
01140     if ( AC.MaxSwitch == 0 ) newmax = 10;
01141     else newmax = 2*AC.MaxSwitch;
01142     newarray = (SWITCH *)Malloca(sizeof(SWITCH)*(newmax+1),"SwitchArray");
01143     newheap = (WORD *)Malloca(sizeof(WORD)*(newmax+1),"SwitchHeap");
01144     if ( AC.MaxSwitch ) {
01145         for ( i = 0; i < AC.MaxSwitch; i++ ) {
01146             newarray[i] = AC.SwitchArray[i];
01147             newheap[i] = AC.SwitchHeap[i];
01148         }
01149         M_free(AC.SwitchHeap,"AC.SwitchHeap");
01150         M_free(AC.SwitchArray,"AC.SwitchArray");
01151     }
01152     for ( i = AC.MaxSwitch; i <= newmax; i++ ) {
01153         newarray[i].table = 0;
01154         newarray[i].tablesize = 0;
01155         newarray[i].defaultcase.ncase = 0;
01156         newarray[i].defaultcase.value = 0;
01157         newarray[i].defaultcase.compbuffer = 0;
01158         newarray[i].endswitch.ncase = 0;
01159         newarray[i].endswitch.value = 0;
01160         newarray[i].endswitch.compbuffer = 0;
01161         newarray[i].typetable = 0;
01162         newarray[i].mincase = 0;
01163         newarray[i].maxcase = 0;
01164         newarray[i].numcases = 0;
01165         newarray[i].caseoffset = 0;
01166         newarray[i].iflevel = 0;
01167         newarray[i].whilelevel = 0;
01168         newarray[i].nestingsum = 0;
01169         newheap[i] = 0;
01170     }
01171     AC.SwitchArray = newarray;
01172     AC.SwitchHeap = newheap;
01173     AC.MaxSwitch = newmax;
01174     return(0);
01175 }
01176
01177 /*
01178     #] DoubleSwitchBuffers :
01179     #[ SwitchSplitMerge :
01180
01181     Sorts an array of WORDs. No adding of equal objects.
01182 */
01183
01184 void SwitchSplitMergeRec(SWITCHTABLE *array,WORD num,SWITCHTABLE *auxarray)
01185 {
01186     WORD n1,n2,i,j,k;
01187     SWITCHTABLE *t1,*t2, t;
01188     if ( num < 2 ) return;
01189     if ( num == 2 ) {
01190         if ( array[0].ncase > array[1].ncase ) {
01191             t = array[0]; array[0] = array[1]; array[1] = t;
01192         }
01193         return;
01194     }
01195     n1 = num/2;
01196     n2 = num - n1;
01197     SwitchSplitMergeRec(array,n1,auxarray);
01198     SwitchSplitMergeRec(array+n1,n2,auxarray);
01199     if ( array[n1-1].ncase <= array[n1].ncase ) return;
01200
01201     t1 = array; t2 = auxarray; i = n1; NCOPY(t2,t1,i);
01202     i = 0; j = n1; k = 0;
01203     while ( i < n1 && j < num ) {
01204         if ( auxarray[i].ncase <= array[j].ncase ) { array[k++] = auxarray[i++]; }
01205         else { array[k++] = array[j++]; }
01206     }
01207     while ( i < n1 ) array[k++] = auxarray[i++];
01208 /*

```

```

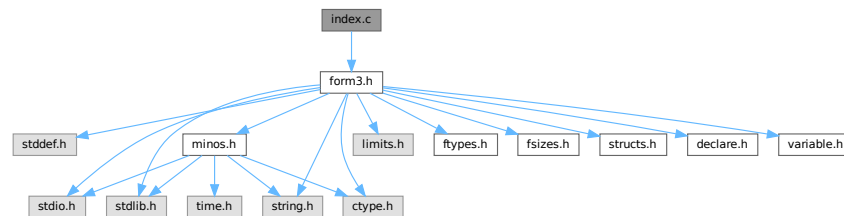
01209     Remember: remnants of j are still in place!
01210 */
01211 }
01212
01213 void SwitchSplitMerge(SWITCHTABLE *array, WORD num)
01214 {
01215     SWITCHTABLE *auxarray = (SWITCHTABLE *)Malloc1(sizeof(SWITCHTABLE)*num/2, "SwitchSplitMerge");
01216     SwitchSplitMergeRec(array, num, auxarray);
01217     M_free(auxarray, "SwitchSplitMerge");
01218 }
01219
01220 /*
01221     #] SwitchSplitMerge :
01222     #] Switch statement :
01223 */

```

4.62 index.c File Reference

```
#include "form3.h"
```

Include dependency graph for index.c:



Functions

- [POSITION](#) * [FindBracket](#) (WORD nexp, WORD *bracket)
- void [PutBracketInIndex](#) (PHEAD WORD *term, [POSITION](#) *newpos)
- void [ClearBracketIndex](#) (WORD numexp)
- void [OpenBracketIndex](#) (WORD nexpr)
- int [PutInside](#) (PHEAD WORD *term, WORD *code)

4.62.1 Detailed Description

The routines that deal with bracket indexing It creates the bracket index and it can find the brackets using the index.

Definition in file [index.c](#).

4.62.2 Function Documentation

4.62.2.1 FindBracket()

```

POSITION * FindBracket (
    WORD nexp,
    WORD * bracket )

```

Definition at line 65 of file [index.c](#).

4.62.2.2 PutBracketInIndex()

```
void PutBracketInIndex (
    PHEAD WORD * term,
    POSITION * newpos )
```

Definition at line 331 of file [index.c](#).

4.62.2.3 ClearBracketIndex()

```
void ClearBracketIndex (
    WORD numexp )
```

Definition at line 546 of file [index.c](#).

4.62.2.4 OpenBracketIndex()

```
void OpenBracketIndex (
    WORD nexpr )
```

Definition at line 578 of file [index.c](#).

4.62.2.5 PutInside()

```
int PutInside (
    PHEAD WORD * term,
    WORD * code )
```

Definition at line 609 of file [index.c](#).

4.63 index.c

[Go to the documentation of this file.](#)

```
00001
00008 /* # [ License : */
00009 /*
00010 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00011 * When using this file you are requested to refer to the publication
00012 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00013 * This is considered a matter of courtesy as the development was paid
00014 * for by FOM the Dutch physics granting agency and we would like to
00015 * be able to track its scientific use to convince FOM of its value
00016 * for the community.
00017 *
00018 * This file is part of FORM.
00019 *
00020 * FORM is free software: you can redistribute it and/or modify it under the
00021 * terms of the GNU General Public License as published by the Free Software
00022 * Foundation, either version 3 of the License, or (at your option) any later
00023 * version.
00024 *
00025 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00026 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00027 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00028 * details.
00029 *
00030 * You should have received a copy of the GNU General Public License along
00031 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00032 */
```

```

00033 /* #] License : */
00034
00035 /*
00036  #[ Includes : index.c
00037 */
00038
00039 #include "form3.h"
00040
00041 /*
00042  #] Includes :
00043  #[ syntax and use :
00044
00045  The indexing of brackets is not automatic! It should only be used
00046  when one intends to use the contents of individual brackets.
00047  This is done with the addition of a + sign to the bracket statement:
00048  B+ a,b,c; or AB+ a,b,c;
00049  It does require resources! The index is kept in memory and is removed
00050  once the expression is treated and passed on to the output with different
00051  or no brackets.
00052  The index is limited to a given amount of space. Hence if there are too
00053  many brackets we will skip some in the index. Skipping goes by space
00054  occupied by the contents. We take the two adjacent bracket(s) with the
00055  least space together and represent them by the first one only. This gives
00056  a new spot.
00057  The expression struct has two pointers:
00058  bracketinfo      for using.
00059  newbracketinfo   for making new index.
00060
00061  #] syntax and use :
00062  #[ FindBracket :
00063 */
00064
00065 POSITION *FindBracket(WORD nexp, WORD *bracket)
00066 {
00067     GETIDENTITY
00068     BRACKETINDEX *bi;
00069     BRACKETINFO *bracketinfo;
00070     EXPRESSIONS e = &(Expressions[nexp]);
00071     LONG hi, low, med;
00072     int i;
00073     WORD oldsorttype = AR.SortType, *t1, *t2, j, bsize, *term, *p, *pstop, *pp;
00074     WORD *tstop, *cp, a[4], *bracketh;
00075     FILEHANDLE *fi;
00076     POSITION auxpos, toppos;
00077     switch ( e->status ) {
00078         case UNHIDELEXPRESSION:
00079         case UNHIDEGEXPRESSION:
00080         case DROPHLEXPRESSION:
00081         case DROPHGEXPRESSION:
00082         case HIDDENLEXPRESSION:
00083         case HIDDENGEXPRESSION:
00084             fi = AR.hidefile;
00085             break;
00086         default:
00087             fi = AR.infile;
00088             break;
00089     }
00090     if ( AT.bracketinfo ) bracketinfo = AT.bracketinfo;
00091     else bracketinfo = e->bracketinfo;
00092     hi = bracketinfo->indexfill; low = 0;
00093     if ( hi <= 0 ) return(0);
00094 /*
00095  The next code is needed for a problem with sorting when there are
00096  only functions outside the bracket. This gives ordinarily the wrong
00097  sorting. We solve that by taking HAAKJE along with the outside while
00098  running the Compare. But this means that we need to copy bracket.
00099 */
00100     bracketh = TermMalloc("FindBracket");
00101     i = *bracket; p = bracket; pp = bracketh; NCOPY(pp,p,i)
00102     pp -= 3; *pp++ = HAAKJE; *pp++ = 3; *pp++ = 0; *pp++ = 1; *pp++ = 1; *pp++ = 3;
00103     *bracketh += 3;
00104
00105     AT.fromindex = 1;
00106     AR.SortType = bracketinfo->SortType;
00107     bi = bracketinfo->indexbuffer + hi - 1;
00108     if ( *bracketh == 7 ) {
00109         if ( bracketinfo->bracketbuffer[bi->bracket] == 7 ) i = 0;
00110         else i = -1;
00111     }
00112     else if ( bracketinfo->bracketbuffer[bi->bracket] == 7 ) i = 1;
00113     else i = CompareTerms(BHEAD bracketh,bracketinfo->bracketbuffer+bi->bracket,0);
00114     if ( i < 0 ) {
00115         AR.SortType = oldsorttype;
00116         AT.fromindex = 0;
00117         TermFree(bracketh,"FindBracket");
00118         return(0);
00119     }

```

```

00120     else if ( i == 0 ) med = hi-1;
00121     else {
00122         for (;;) {
00123             med = (hi+low)/2;
00124             bi = bracketinfo->indexbuffer + med;
00125             if ( *bracketh == 7 ) {
00126                 if ( bracketinfo->bracketbuffer[bi->bracket] == 7 ) i = 0;
00127                 else i = -1;
00128             }
00129             else if ( bracketinfo->bracketbuffer[bi->bracket] == 7 ) i = 1;
00130             else i = CompareTerms(BHEAD bracketh,bracketinfo->bracketbuffer+bi->bracket,0);
00131             if ( i == 0 ) { break; }
00132             if ( i > 0 ) {
00133                 if ( low == med ) { /* no occurrence */
00134                     AR.SortType = oldsorttype;
00135                     AT.fromindex = 0;
00136                     TermFree(bracketh,"FindBracket");
00137                     return(0);
00138                 }
00139                 hi = med;
00140             }
00141             else if ( i < 0 ) {
00142                 if ( low == med ) break;
00143                 low = med;
00144             }
00145         }}
00146     /*
00147     The bracket is now either bi itself or between bi and the next one
00148     or it is not present at all.
00149     */
00150     AN.theposition = AS.OldOnFile[nexp];
00151     ADD2POS(AN.theposition,bi->start);
00152     /*
00153     The seek will have to move closer to the actual read so that we
00154     can place a lock around the two.
00155     if ( fi->handle >= 0 ) SeekFile(fi->handle,&AN.theposition,SEEK_SET);
00156     else SetScratch(fi,&AN.theposition);
00157     */
00158     /*
00159     Put the bracket in the compress buffer as if it were the last term read.
00160     Have a look at AR.CompressPointer. (set it right)
00161     */
00162     term = AT.WorkPointer;
00163     t1 = bracketinfo->bracketbuffer+bi->bracket;
00164     j = *t1;
00165     /*
00166     Note that in the bracketbuffer, the bracket sits with HAAKJE.
00167
00168     The next is (hopefully) a bug fix. Originally the code read bsize = j
00169     but that overcounts one. We have the part outside the bracket and the
00170     coefficient which is 1,1,3. But we also have the length indicator.
00171     Where we use the variable bsize we do not include the length indicator,
00172     and we have the part outside plus 7,3,0 which is also three words.
00173     */
00174     bsize = j-1;
00175     t2 = AR.CompressPointer;
00176     NCOPY(t2,t1,j)
00177     if ( i == 0 ) { /* We found the proper bracket already */
00178         AR.SortType = oldsorttype;
00179         AT.fromindex = 0;
00180         TermFree(bracketh,"FindBracket");
00181         return(&AN.theposition);
00182     }
00183     /*
00184     Here we have to skip to the bracket if it exists (!)
00185     Let us first look whether the expression is in memory.
00186     If not we have to make a buffer to increase speed..
00187     */
00188     if ( fi->handle < 0 ) {
00189         p = (WORD *) ((UBYTE *) (fi->PObuffer)
00190             + BASEPOSITION(AS.OldOnFile[nexp])
00191             + BASEPOSITION(bi->start));
00192         pstop = (WORD *) ((UBYTE *) (fi->PObuffer)
00193             + BASEPOSITION(AS.OldOnFile[nexp])
00194             + BASEPOSITION(bi->next));
00195         while ( p < pstop ) {
00196             /*
00197             Check now: if size or part from previous term < size of bracket
00198             we have to setup the bracket again and test.
00199             Otherwise, skip immediately to the next term.
00200             */
00201             if ( *p <= -bsize ) { /* no change of bracket */
00202                 p++; p += *p + 1;
00203             }
00204             else if ( *p < 0 ) { /* changes bracket */
00205                 pp = p;
00206                 t2 = AR.CompressPointer;

```

```

00207         t1 = t2 - *p++ + 1;
00208         j = *p++;
00209         NCOPY(t1,p,j)
00210         t2++; while ( *t2 != HAAKJE ) t2 += t2[1];
00211         t2 += t2[1];
00212         a[1] = t2[0]; a[2] = t2[1]; a[3] = t2[2];
00213         *t2++ = 1; *t2++ = 1; *t2++ = 3;
00214         *AR.CompressPointer = t2 - AR.CompressPointer;
00215         bsize = *AR.CompressPointer - 1;
00216         if ( *bracketh == 7 ) {
00217             if ( AR.CompressPointer[0] == 7 ) i = 0;
00218             else i = -1;
00219         }
00220         else if ( AR.CompressPointer[0] == 7 ) i = 1;
00221         else i = CompareTerms(BHEAD bracketh,AR.CompressPointer,0);
00222         t2[-3] = a[1]; t2[-2] = a[2]; t2[-1] = a[3];
00223         if ( i == 0 ) {
00224             SETBASEPOSITION(AN.theposition,(pp-fi->PObuffer)*sizeof(WORD));
00225             fi->POfill = pp;
00226             goto found;
00227         }
00228         if ( i > 0 ) break; /* passed what was possible */
00229     }
00230     else { /* no compression. We have to check! */
00231         WORD *oldworkpointer = AT.WorkPointer, *t3, *t4;
00232         t2 = p + 1; while ( *t2 != HAAKJE ) t2 += t2[1];
00233         t2 += t2[1];
00234         /*
00235          Here we need to copy the term. Modifying has proven to
00236          be NOT threadsafe.
00237          */
00238         t3 = oldworkpointer; t4 = p;
00239         while ( t4 < t2 ) *t3++ = *t4++;
00240         *t3++ = 1; *t3++ = 1; *t3++ = 3;
00241         *oldworkpointer = t3 - oldworkpointer;
00242         bsize = *oldworkpointer - 1;
00243         AT.WorkPointer = t3;
00244         t3 = oldworkpointer;
00245         if ( *bracketh == 7 ) {
00246             if ( t3[0] == 7 ) i = 0;
00247             else i = -1;
00248         }
00249         else if ( t3[0] == 7 ) i = 1;
00250         else {
00251             i = CompareTerms(BHEAD bracketh,t3,0);
00252         }
00253         AT.WorkPointer = oldworkpointer;
00254         if ( i == 0 ) {
00255             SETBASEPOSITION(AN.theposition,(p-fi->PObuffer)*sizeof(WORD));
00256             fi->POfill = p;
00257             goto found;
00258         }
00259         if ( i > 0 ) break; /* passed what was possible */
00260         p += *p;
00261     }
00262 }
00263 AR.SortType = oldsorttype;
00264 AT.fromindex = 0;
00265 TermFree(bracketh,"FindBracket");
00266 return(0); /* Bracket does not exist */
00267 }
00268 else {
00269     /*
00270      In this case we can work with the old representation without HAAKJE.
00271      We stop searching when we reach toppos and we do not call Compare.
00272      */
00273     toppos = AS.OldOnFile[nexp];
00274     ADD2POS(toppos,bi->next);
00275     cp = AR.CompressPointer;
00276     for(;;) {
00277         auxpos = AN.theposition;
00278         GetOneTerm(BHEAD term,fi,&auxpos,0);
00279         if ( *term == 0 ) {
00280             AR.SortType = oldsorttype;
00281             AT.fromindex = 0;
00282             return(0); /* Bracket does not exist */
00283         }
00284         tstop = term + *term;
00285         tstop -= ABS(tstop[-1]);
00286         t1 = term + 1;
00287         while ( *t1 != HAAKJE && t1 < tstop ) t1 += t1[1];
00288         i = *bracket-4;
00289         if ( t1-term == *bracket-3 ) {
00290             t1 = term + 1; t2 = bracket+1;
00291             while ( i > 0 && *t1 == *t2 ) { t1++; t2++; i--; }
00292             if ( i <= 0 ) {
00293                 AR.CompressPointer = cp;

```

```

00294             goto found;
00295         }
00296     }
00297     AR.CompressPointer = cp;
00298     AN.theposition = auxpos;
00299 /*
00300     Now check whether we passed the 'point'
00301 */
00302     if ( ISGEPOS(AN.theposition,toppos) ) {
00303         AR.SortType = oldsorttype;
00304         AR.CompressPointer = cp;
00305         AT.fromindex = 0;
00306         TermFree(bracketh,"FindBracket");
00307         return(0); /* Bracket does not exist */
00308     }
00309 }
00310 }
00311 found:
00312     AR.SortType = oldsorttype;
00313     AT.fromindex = 0;
00314     TermFree(bracketh,"FindBracket");
00315     return(&AN.theposition);
00316 }
00317
00318 /*
00319     #] FindBracket :
00320     #[ PutBracketInIndex :
00321
00322     Call via
00323     if ( AR.BracketOn ) PutBracketInIndex(BHEAD term);
00324
00325     This means that there should be a bracket somewhere
00326     Note that the brackets come in in proper order.
00327
00328     DON'T forget AR.SortType to be put into e->bracketinfo->SortType
00329 */
00330
00331 void PutBracketInIndex(PHEAD WORD *term, POSITION *newpos)
00332 {
00333     GETBIDENTITY
00334     BRACKETINDEX *bi, *b1, *b2, *b3;
00335     BRACKETINFO *b;
00336     POSITION thepos;
00337     EXPRESSIONS e = Expressions + AR.CurExpr;
00338     LONG hi, i, average;
00339     WORD *t, *tstop, *t1, *t2, *oldt, oldsize, a[4];
00340     if ( ( b = e->newbracketinfo ) == 0 ) return;
00341     DIFPOS(thepos,*newpos,e->onfile);
00342     tstop = term + *term;
00343     tstop -= ABS(tstop[-1]);
00344     t = term+1;
00345     while ( *t != HAAKJE && t < tstop ) t += t[1];
00346     if ( t >= tstop ) return; /* no ticket, no laundry */
00347     t += t[1]; /* include HAAKJE for the sorting */
00348     a[0] = t[0]; a[1] = t[1]; a[2] = t[2];
00349     oldt = t; oldsize = *term; *t++ = 1; *t++ = 1; *t++ = 3;
00350     *term = t - term;
00351     AT.fromindex = 1;
00352 /*
00353     Check now with the last bracket in the buffer.
00354     If it is the same we can abort.
00355 */
00356     hi = b->indexfill;
00357     if ( hi > 0 ) {
00358         bi = b->indexbuffer + hi - 1;
00359         bi->next = thepos;
00360         if ( *term == 7 ) {
00361             if ( b->bracketbuffer[bi->bracket] == 7 ) i = 0;
00362             else i = -1;
00363         }
00364         else if ( b->bracketbuffer[bi->bracket] == 7 ) i = 1;
00365         else i = CompareTerms(BHEAD term,b->bracketbuffer+bi->bracket,0);
00366         if ( i == 0 ) { /* still the same bracket */
00367             bi->termsinbracket++;
00368             goto bracketdone;
00369         }
00370         if ( i > 0 ) { /* We have a problem */
00371 /*
00372         There is a special case in which we have only functions and
00373         term is contained completely in the bracket
00374 */
00375         /*
00376         t = term + 1;
00377         tstop = term + *term - 3;
00378         while ( t < tstop && *t > HAAKJE ) t += t[1];
00379         if ( t < tstop ) goto problems;
00380 */

```



```

00381         for ( i = 1; i < *term - 3; i++ ) {
00382             if ( term[i] != b->bracketbuffer[bi->bracket+i] ) break;
00383         }
00384         if ( i < *term - 3 ) {
00385             /*
00386             problems;;
00387             */
00388             *term = oldsize; oldt[0] = a[0]; oldt[1] = a[1]; oldt[2] = a[2];
00389             MLOCK(ErrorMessageLock);
00390             MesPrint("Error!!!! Illegal bracket sequence detected in PutBracketInIndex");
00391             #ifdef WITHPTHREADS
00392             MesPrint("Worker = %w");
00393             #endif
00394             PrintTerm(term,"term into index");
00395             PrintTerm(b->bracketbuffer+bi->bracket,"Last in index");
00396             MUNLOCK(ErrorMessageLock);
00397             AT.fromindex = 0;
00398             Terminate(-1);
00399         }
00400         i = -1;
00401     }
00402 }
00403 /*
00404 If there is room for more brackets, we add this one.
00405 */
00406 if ( b->bracketfill+*term >= b->bracketbuffersize
00407     && ( b->bracketbuffersize < AM.MaxBracketBufferSize
00408         || ( e->vflags & ISFACTORIZED ) != 0 ) ) {
00409     /*
00410     Enlarge bracket buffer
00411     */
00412     WORD *oldbracketbuffer = b->bracketbuffer;
00413     i = Max(b->bracketbuffersize * 2, b->bracketfill+*term+1);
00414     if ( i > AM.MaxBracketBufferSize && ( e->vflags & ISFACTORIZED ) == 0 )
00415         i = AM.MaxBracketBufferSize;
00416     if ( i > b->bracketfill+*term ) {
00417         b->bracketbuffersize = i;
00418         b->bracketbuffer = (WORD *)Malloca1(b->bracketbuffersize*sizeof(WORD),
00419             "new bracket buffer");
00420         t1 = b->bracketbuffer; t2 = oldbracketbuffer;
00421         i = b->bracketfill;
00422         NCOPY(t1,t2,i)
00423         if ( oldbracketbuffer ) M_free(oldbracketbuffer,"old bracket buffer");
00424     }
00425 }
00426 if ( b->bracketfill+*term < b->bracketbuffersize ) {
00427     if ( b->indexfill >= b->indexbuffersize ) {
00428         /*
00429         Enlarge index
00430         */
00431         BRACKETINDEX *oldindexbuffer = b->indexbuffer;
00432         b->indexbuffersize *= 2;
00433         b->indexbuffer = (BRACKETINDEX *)
00434             Malloca1(b->indexbuffersize*sizeof(BRACKETINDEX),"new bracket index");
00435         b1 = b->indexbuffer; b2 = oldindexbuffer;
00436         i = b->indexfill;
00437         NCOPY(b1,b2,i)
00438         if ( oldindexbuffer ) M_free(oldindexbuffer,"old bracket index");
00439     }
00440 }
00441 else {
00442     /*
00443     We have too many brackets in the buffer. Try to improve.
00444     This is the interesting algorithm. We try to eliminate about 1/4 to
00445     1/2 of the brackets from the index. This should be done by size of
00446     the bracket contents to make the searching as fast as possible.
00447     But! Do not touch the last bracket.
00448     Note that we are always filling from the back.
00449     Algorithm: Throw away every second bracket, unless b1+b2 is much longer
00450     than average. How much is something we can tune.
00451     */
00452     average = DIVPOS(thepos,b->indexfill+1);
00453     if ( average <= 0 ) {
00454         MLOCK(ErrorMessageLock);
00455         MesPrint("Problems with bracket buffer. Increase MaxBracketBufferSize in form.set");
00456         MesPrint("Current size is %l",AM.MaxBracketBufferSize*sizeof(WORD));
00457         MUNLOCK(ErrorMessageLock);
00458         Terminate(-1);
00459     }
00460     average *= 4; /* 2*2: one 2 for much longer, one 2 because we have pairs */
00461     t2 = b->bracketbuffer;
00462     b3 = b1 = b->indexbuffer;
00463     bi = b->indexbuffer + b->indexfill;
00464     b2 = b1+1;
00465     while ( b2+2 < bi ) {
00466         if ( DIFBASE(b2->next,b1->start) > average ) {
00467             t1 = b->bracketbuffer + b1->bracket;

```

```

00468         b1->bracket = t2 - b->bracketbuffer;
00469         i = *t1; NCOPY(t2,t1,i)
00470         *b3++ = *b1;
00471         t1 = b->bracketbuffer + b2->bracket;
00472         b2->bracket = t2 - b->bracketbuffer;
00473         i = *t1; NCOPY(t2,t1,i)
00474         *b3++ = *b2;
00475         if ( b3 <= b1 ) {
00476             PUTZERO(b1->start);
00477             PUTZERO(b1->next);
00478             b1->bracket = 0;
00479             b1->termsinbracket = 0;
00480         }
00481         if ( b3 <= b2 ) {
00482             PUTZERO(b2->start);
00483             PUTZERO(b2->next);
00484             b2->bracket = 0;
00485             b2->termsinbracket = 0;
00486         }
00487     }
00488     else {
00489         t1 = b->bracketbuffer + b1->bracket;
00490         b1->bracket = t2 - b->bracketbuffer;
00491         i = *t1; NCOPY(t2,t1,i)
00492         b1->next = b2->next;
00493         b1->termsinbracket += b2->termsinbracket;
00494         *b3++ = *b1;
00495         if ( b3 <= b1 ) {
00496             PUTZERO(b1->start);
00497             PUTZERO(b1->next);
00498             b1->bracket = 0;
00499             b1->termsinbracket = 0;
00500         }
00501         PUTZERO(b2->start);
00502         PUTZERO(b2->next);
00503         b2->bracket = 0;
00504         b2->termsinbracket = 0;
00505     }
00506     b1 += 2; b2 += 2;
00507 }
00508 while ( b1 < bi ) {
00509     t1 = b->bracketbuffer + b1->bracket;
00510     b1->bracket = t2 - b->bracketbuffer;
00511     i = *t1; NCOPY(t2,t1,i)
00512     *b3++ = *b1;
00513     if ( b3 <= b1 ) {
00514         PUTZERO(b1->start);
00515         PUTZERO(b1->next);
00516         b1->bracket = 0;
00517         b1->termsinbracket = 0;
00518     }
00519     b1++;
00520 }
00521 b->indexfill = b3 - b->indexbuffer;
00522 b->bracketfill = t2 - b->bracketbuffer;
00523 }
00524 bi = b->indexbuffer + b->indexfill;
00525 b->indexfill++;
00526 bi->bracket = b->bracketfill;
00527 bi->start = thepos;
00528 bi->next = thepos;
00529 bi->termsinbracket = 1;
00530 /*
00531 Copy the bracket into the buffer
00532 */
00533 t1 = term; t2 = b->bracketbuffer + bi->bracket; i = *t1;
00534 b->bracketfill += i;
00535 NCOPY(t2,t1,i)
00536 bracketdone:
00537 *term = oldsize; oldt[0] = a[0]; oldt[1] = a[1]; oldt[2] = a[2];
00538 AT.fromindex = 0;
00539 }
00540
00541 /*
00542 #] PutBracketInIndex :
00543 #[ ClearBracketIndex :
00544 */
00545
00546 void ClearBracketIndex(WORD numexp)
00547 {
00548     BRACKETINFO *b;
00549     if ( numexp >= 0 ) {
00550         b = Expressions[numexp].bracketinfo;
00551         Expressions[numexp].bracketinfo = 0;
00552     }
00553     else if ( numexp == -1 ) {
00554         GETIDENTITY

```

```

00555         b = AT.bracketinfo;
00556         AT.bracketinfo = 0;
00557     }
00558     else {
00559         numexp = -numexp-2;
00560         b = Expressions[numexp].newbracketinfo;
00561         Expressions[numexp].newbracketinfo = 0;
00562     }
00563     if ( b == 0 ) return;
00564     b->indexfill = b->indexbuffersize = 0;
00565     b->bracketfill = b->bracketbuffersize = 0;
00566     M_free(b->bracketbuffer, "ClearBracketBuffer");
00567     M_free(b->indexbuffer, "ClearIndexBuffer");
00568     M_free(b, "BracketInfo");
00569 }
00570
00571 /*
00572 #] ClearBracketIndex :
00573 #[ OpenBracketIndex :
00574
00575     Note: This routine is thread-safe
00576 */
00577 void OpenBracketIndex(WORD nexpr)
00578 {
00579     EXPRESSIONS e = Expressions + nexpr;
00580     BRACKETINFO *bi;
00581     LONG i;
00582     bi = (BRACKETINFO *)Malloc1(sizeof(BRACKETINFO), "BracketInfo");
00583     e->newbracketinfo = bi;
00584     /*
00585     i = 20*AM.MaxTer/sizeof(WORD);
00586     if ( i < 1000 ) i = 1000;
00587     */
00588     i = 2000;
00589     bi->bracketbuffer = (WORD *)Malloc1(i*sizeof(WORD), "Bracket Buffer");
00590     bi->bracketbuffersize = i;
00591     bi->bracketfill = 0;
00592     i = 50;
00593     bi->indexbuffer = (BRACKETINDEX *)Malloc1(i*sizeof(BRACKETINDEX), "Bracket Index");
00594     bi->indexbuffersize = i;
00595     bi->indexfill = 0;
00596     bi->SortType = AC.SortType;
00597 }
00598
00599
00600 /*
00601 #] OpenBracketIndex :
00602 #[ PutInside :
00603
00604     Puts a term, or a bracket determined part of a term inside a function.
00605
00606     AT.WorkPointer points at term+*term
00607 */
00608
00609 int PutInside(PHEAD WORD *term, WORD *code)
00610 {
00611     WORD *from, *to, *oldbuf, *tStop, *t, *tt, oldon, oldact, inc, argsize, *termout;
00612     int i, ii, error;
00613
00614     if ( code[1] == 4 && ( code[2] == 0 || code[2] == 1 ) ) {
00615         /*
00616         Put all inside. Move the term by 1+FUNHEAD+ARGHEAD
00617         */
00618         from = term+*term; to = from+1+ARGHEAD+FUNHEAD; i = ii = *term;
00619         to[0] = 1; to[1] = 1; to[2] = 3;
00620         while ( --i >= 0 ) *--to = *--from;
00621         to = term;
00622         *to++ = term[0]+4+ARGHEAD+FUNHEAD;
00623         *to++ = code[3];
00624         *to++ = ii+FUNHEAD+ARGHEAD;
00625         *to++ = 1; /* set dirty flags, because there could be a fast notation */
00626         FILLFUN3(to)
00627         *to++ = ii+ARGHEAD;
00628         *to++ = 1;
00629         FILLARG(to)
00630         return(0);
00631     }
00632     /*
00633     First we save the old bracket variables. Then we set variables to
00634     influence the PutBracket routine and call it.
00635     After that we set the values back and sort out the results by placing the
00636     inside of the bracket inside the function.
00637     */
00638     termout = AT.WorkPointer;
00639     oldbuf = AT.BrackBuf;
00640     oldon = AR.BracketOn;
00641     oldact = AT.PolyAct;

```

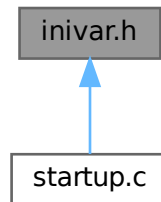
```

00642     AR.BracketOn = -code[2];
00643     AT.BrackBuf  = code+4;
00644     AT.PolyAct = 0;
00645     error = PutBracket(BHEAD term);
00646     AT.PolyAct = oldact;
00647     AT.BrackBuf = oldbuf;
00648     AR.BracketOn = oldon;
00649     if ( error ) return(error);
00650     i = *termout; from = termout; to = term;
00651     NCOPY(to,from,i);
00652     tStop = term +*term; tStop -= tStop[-1];
00653     t = term+1;
00654     while ( t < tStop && *t != HAAKJE ) t += t[1];
00655     from = term + *term;
00656     inc = FUNHEAD+ARGHEAD-t[1]+1;
00657     tt = t + t[1];
00658     argsize = from-tt+1;
00659     to = from + inc;
00660     to[0] = 1;
00661     to[1] = 1;
00662     to[2] = 3;
00663     while ( from > tt ) *--to = *--from;
00664     *--to = argsize;
00665     *t++ = code[3];
00666     *t++ = argsize+FUNHEAD+ARGHEAD;
00667     *t++ = 1;
00668     FILLFUN3(t);
00669     *t++ = argsize+ARGHEAD;
00670     *t++ = 1;
00671     FILLARG(t);
00672     *term += inc+3;
00673     AT.WorkPointer = term+*term;
00674     if ( Normalize(BHEAD term) ) error = 1;
00675     return(error);
00676 }
00677
00678 /*
00679 #] PutInside :
00680 */
00681
00682 /*
00683 The next routines are for indexing the local output files in a parallel
00684 sort. This indexing is needed to get a fast determination of the
00685 splitting terms needed to divide the terms evenly over the processors.
00686 Actually this method works well for ParFORM, but may not work well
00687 for TFORM.
00688
00689 #[ PutTermInIndex :
00690
00691 Puts a term in the term index.
00692 Action:
00693     if the index hasn't reached its full size
00694         if there is room, put the term
00695         if there is no room: extend the buffer, put the term
00696     else
00697         check if the last term has a number of the type skip*m+1
00698         if no, overwrite the last term
00699         if yes, check whether there is room for one more term
00700             yes: add the term
00701             no: drop all even terms, compress the list,
00702                 multiply skip by 2, and add this term.
00703
00704 int PutTermInIndex(WORD *term,POSITION *position)
00705 {
00706     return(0);
00707 }
00708
00709 #] PutTermInIndex :
00710 */

```

4.64 inivar.h File Reference

This graph shows which files directly or indirectly include this file:



Data Structures

- struct **fixedfun**

Variables

- [FIXEDGLOBALS FG](#)
- [ALLGLOBALS A](#)
- [FIXEDSET fixedsets \[\]](#)
- UBYTE [BufferForOutput](#) [MAXLINELENGTH+14]
- char * [setupfilename](#) = "form.set"

4.64.1 Detailed Description

Contains the initialization of a number of structs at compile time This file should only be included in the file [startup.c](#) !!!

Definition in file [inivar.h](#).

4.64.2 Variable Documentation

4.64.2.1 FG

[FIXEDGLOBALS FG](#)

Definition at line [34](#) of file [inivar.h](#).

4.64.2.2 A

[ALLGLOBALS A](#)

Definition at line [133](#) of file [inivar.h](#).

4.64.2.3 fixedsets

```
FIXEDSET fixedsets[]
```

Initial value:

```
= {
  {"pos_", "integers > 0", CSYMBOL, 0}
, {"pos0_", "integers >= 0", CSYMBOL, 0}
, {"neg_", "integers < 0", CSYMBOL, 0}
, {"neg0_", "integers <= 0", CSYMBOL, 0}
, {"even_", "even integers", CSYMBOL, 0}
, {"odd_", "odd integers", CSYMBOL, 0}
, {"int_", "all integers", CSYMBOL, 0}
, {"symbol_", "only symbols", CSYMBOL, MAXPOSITIVE}
, {"fixed_", "fixed indices", CINDEX, 0}
, {"index_", "all indices", CINDEX, 0}
, {"number_", "all rationals", CSYMBOL, 0}
, {"dummyindices_", "dummy indices", CINDEX, 0}
, {"vector_", "only vectors", CVECTOR, 0}
}
```

Definition at line 258 of file [inivar.h](#).

4.64.2.4 BufferForOutput

```
UBYTE BufferForOutput[MAXLINELENGTH+14]
```

Definition at line 274 of file [inivar.h](#).

4.64.2.5 setupfilename

```
char* setupfilename = "form.set"
```

Definition at line 276 of file [inivar.h](#).

4.65 inivar.h

[Go to the documentation of this file.](#)

```
00001
00007 /* #[] License : */
00008 /*
00009 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00010 * When using this file you are requested to refer to the publication
00011 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00012 * This is considered a matter of courtesy as the development was paid
00013 * for by FOM the Dutch physics granting agency and we would like to
00014 * be able to track its scientific use to convince FOM of its value
00015 * for the community.
00016 *
00017 * This file is part of FORM.
00018 *
00019 * FORM is free software: you can redistribute it and/or modify it under the
00020 * terms of the GNU General Public License as published by the Free Software
00021 * Foundation, either version 3 of the License, or (at your option) any later
00022 * version.
00023 *
00024 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00025 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00026 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00027 * details.
00028 *
00029 * You should have received a copy of the GNU General Public License along
00030 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00031 */
00032 /* #[] License : */
00033
```

```

00034 FIXEDGLOBALS FG = {
00035     {Traces                                     /* Dummy as zeroeth argument */
00036     ,Traces
00037     ,EpFCon
00038     ,RatioGen
00039     ,SymGen
00040     ,TenVec
00041     ,DoSumF1
00042     ,DoSumF2}
00043
00044     ,{TraceFind                                 /* Dummy as zeroeth argument */
00045     ,TraceFind
00046     ,EpFFind
00047     ,RatioFind
00048     ,SymFind
00049     ,TenVecFind}
00050
00051     ,{"symbol"
00052     , "index"
00053     , "vector"
00054     , "function"
00055     , "set"
00056     , "expression"
00057     , "dotproduct"
00058     , "number"
00059     , "subexp"
00060     , "delta"}
00061
00062     ,{"(local)"
00063     , "(skip/local)"
00064     , "(drop/local)"
00065     , "(dropped)"
00066     , "(global)"
00067     , "(skip/global)"
00068     , "(drop/global)"
00069     , "(dropped)"
00070     , "(stored)"
00071     , "(local-hidden)"
00072     , "(local-to be hidden)"
00073     , "(local-hidden-dropped)"
00074     , "(local-to be unhidden)"
00075     , "(global-hidden)"
00076     , "(global-to be hidden)"
00077     , "(global-hidden-dropped)"
00078     , "(global-to be unhidden)"
00079     , "(into-hide-local)"
00080     , "(into-hide-global)"
00081     , "(spectator)"
00082     , "(drop/spectator)"}
00083
00084     ,{" Functions"
00085     , " Commuting Functions"}
00086
00087     ,{"left      "
00088     , "active    "
00089     , "in output"}
00090
00091     ,(char *)0
00092     ,(char *)0
00093     ,(UBYTE *) "1"
00094     ,(WORD)0
00095     ,(WORD)0
00096     ,(WORD)0
00097     ,(WORD)0
00098
00099 /* ASCII table of character types. Note that on some computers this
00100 table may be different from the ASCII table.
00101
00102 -1 Illegal character
00103 0 Alphabetic character
00104 1 Digit
00105 2 . $ _ ? # or '
00106 3 [,]
00107 4 ( ) = ; or ,
00108 5 + - * % / ^ :
00109 6 blank, tab, linefeed
00110 7 {,|,}
00111 8 ! & < >
00112 9 "
00113 10 The ultimate end.
00114 */
00115 ,{
00116     10,255,255,255,255,255,255,255,255,255, 6, 6,255,255, 6,255,255,
00117     255,255,255,255,255,255,255,255,255,255,10,255,255,255,255,255,
00118     6, 8, 9, 2, 2, 5, 8, 2, 4, 4, 5, 5, 4, 5, 2, 5,
00119     1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 5, 4, 8, 4, 8, 2,
00120     255, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
00121     0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3,255, 3, 5, 2,

```

```

00121         255, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
00122         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 7, 7, 7, 255, 255,
00123         255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255,
00124         255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255,
00125         255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255,
00126         255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255,
00127         255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255,
00128         255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255,
00129         255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255,
00130         255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255 }
00131     };
00132
00133     ALLGLOBALS A;
00134     #ifdef WITHPTHREADS
00135     ALLPRIVATES **AB;
00136     #endif
00137
00138     static struct fixedfun {
00139         char *name;
00140         int        commu , tensor        , complx        , symmetric;
00141     } fixedfunctions[] = {
00142         {"exp_" , 1, 0, 0, 0} /* EXPONENT */
00143         , {"denom_" , 1, 0, 0, 0} /* DENOMINATOR */
00144         , {"setfun_" , 1, 0, 0, 0} /* SETFUNCTION */
00145         , {"g_" , 1, GAMMAFUNCTION, VARTYPEIMAGINARY, 0} /* GAMMA */
00146         , {"gi_" , 1, GAMMAFUNCTION, VARTYPEIMAGINARY, 0} /* GAMMAI */
00147         , {"g5_" , 1, GAMMAFUNCTION, VARTYPEIMAGINARY, 0} /* GAMMAFIVE */
00148         , {"g6_" , 1, GAMMAFUNCTION, VARTYPEIMAGINARY, 0} /* GAMMASIX */
00149         , {"g7_" , 1, GAMMAFUNCTION, VARTYPEIMAGINARY, 0} /* GAMMASEVEN */
00150         , {"sum_" , 1, 0, 0, 0} /* SUMF1 */
00151         , {"sump_" , 1, 0, 0, 0} /* SUMF2 */
00152         , {"dum_" , 1, 0, 0, 0} /* DUMFUN */
00153         , {"replace_" , 0, 0, 0, 0} /* REPLACEMENT */
00154         , {"reverse_" , 1, 0, 0, 0} /* REVERSEFUNCTION */
00155         , {"distrib_" , 1, 0, 0, 0} /* DISTRIBUTION */
00156         , {"dd_" , 0, TENSORFUNCTION, 0, 0} /* DELTA3 */
00157         , {"dummy_" , 0, 0, 0, 0} /* DUMMYFUN */
00158         , {"dummyten_" , 0, TENSORFUNCTION, 0, 0} /* DUMMYTEN */
00159         , {"e_" , 0, TENSORFUNCTION|1, VARTYPEIMAGINARY, ANTISYMMETRIC}
00160         /* LEVICIVITA */
00161         , {"fac_" , 0, 0, 0, 0} /* FACTORIAL */
00162         , {"invfac_" , 0, 0, 0, 0} /* INVERSEFACTORIAL */
00163         , {"binom_" , 0, 0, 0, 0} /* BINOMIAL */
00164         , {"nargs_" , 0, 0, 0, 0} /* NUMARGSFUN */
00165         , {"sign_" , 0, 0, 0, 0} /* SIGNFUN */
00166         , {"mod_" , 0, 0, 0, 0} /* MODFUNCTION */
00167         , {"mod2_" , 0, 0, 0, 0} /* MOD2FUNCTION */
00168         , {"min_" , 0, 0, 0, 0} /* MINFUNCTION */
00169         , {"max_" , 0, 0, 0, 0} /* MAXFUNCTION */
00170         , {"abs_" , 0, 0, 0, 0} /* ABSFUNCTION */
00171         , {"sig_" , 0, 0, 0, 0} /* SIGFUNCTION */
00172         , {"integer_" , 0, 0, 0, 0} /* INTFUNCTION */
00173         , {"theta_" , 0, 0, 0, 0} /* THETA */
00174         , {"thetap_" , 0, 0, 0, 0} /* THETA2 */
00175         , {"delta_" , 0, 0, 0, 0} /* DELTA2 */
00176         , {"deltap_" , 0, 0, 0, 0} /* DELTAP */
00177         , {"bernoulli_" , 0, 0, 0, 0} /* BERNOULLIFUNCTION */
00178         , {"count_" , 0, 0, 0, 0} /* COUNTFUNCTION */
00179         , {"match_" , 0, 0, 0, 0} /* MATCHFUNCTION */
00180         , {"pattern_" , 0, 0, VARTYPECOMPLEX, 0} /* PATTERNFUNCTION */
00181         , {"term_" , 1, 0, 0, 0} /* TERMFUNTION */
00182         , {"conjg_" , 1, 0, VARTYPECOMPLEX, 0} /* CONJUGATEFUNCTION */
00183         , {"root_" , 0, 0, 0, 0} /* ROOTFUNCTION */
00184         , {"table_" , 1, 0, 0, 0} /* TABLEFUNCTION */
00185         , {"firstbracket_" , 0, 0, 0, 0} /* FIRSTBRACKET */
00186         , {"termsin_" , 0, 0, 0, 0} /* TERMSINEXPR */
00187         , {"nterms_" , 0, 0, 0, 0} /* NUMTERMSFUN */
00188         , {"gcd_" , 0, 0, 0, 0} /* GCDFUNCTION */
00189         , {"div_" , 0, 0, 0, 0} /* DIVFUNCTION */
00190         , {"rem_" , 0, 0, 0, 0} /* REMFUNCTION */
00191         , {"maxpowerof_" , 0, 0, 0, 0} /* MAXPOWEROF */
00192         , {"minpowerof_" , 0, 0, 0, 0} /* MINPOWEROF */
00193         , {"tbl_" , 0, 0, 0, 0} /* TABLESTUB */
00194         , {"factorin_" , 0, 0, 0, 0} /* FACTORIN */
00195         , {"termsinbracket_" , 0, 0, 0, 0} /* TERMSINBRACKET */
00196         , {"farg_" , 0, 0, 0, 0} /* WILDARGFUN */
00197     /*
00198         The following names are reserved for the floating point library.
00199         As long as we have no floating point numbers they do not do anything.
00200     */
00201     , {"sqrt_" , 0, 0, 0, 0} /* SQRTFUNCTION */
00202     , {"ln_" , 0, 0, 0, 0} /* LNFUNTION */
00203     , {"sin_" , 0, 0, 0, 0} /* SINFUNTION */
00204     , {"cos_" , 0, 0, 0, 0} /* COSFUNCTION */
00205     , {"tan_" , 0, 0, 0, 0} /* TANFUNCTION */
00206     , {"asin_" , 0, 0, 0, 0} /* ASINFUNTION */
00207     , {"acos_" , 0, 0, 0, 0} /* ACOSFUNCTION */

```



```

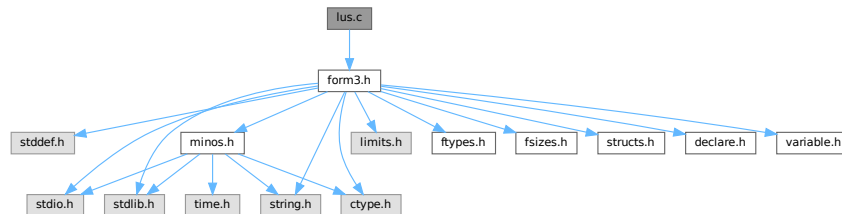
00208     ,{"atan_"      ,0,0      ,0      ,0} /* ATANFUNCTION */
00209     ,{"atan2_"     ,0,0      ,0      ,0} /* ATAN2FUNCTION */
00210     ,{"sinh_"      ,0,0      ,0      ,0} /* SINHFUNCTION */
00211     ,{"cosh_"      ,0,0      ,0      ,0} /* COSHFUNCTION */
00212     ,{"tanh_"      ,0,0      ,0      ,0} /* TANHFUNCTION */
00213     ,{"asinh_"     ,0,0      ,0      ,0} /* ASINHFUNCTION */
00214     ,{"acosh_"     ,0,0      ,0      ,0} /* ACOSHFUNCTION */
00215     ,{"atanh_"     ,0,0      ,0      ,0} /* ATANHFUNCTION */
00216     ,{"li2_"       ,0,0      ,0      ,0} /* LI2FUNCTION */
00217     ,{"lin_"       ,0,0      ,0      ,0} /* LINFUNCTION */
00218 /*
00219     From here on we continue with new functions (26-sep-2010)
00220 */
00221     ,{"extrasymbol_" ,0,0      ,0      ,0} /* EXTRASYMFUN */
00222     ,{"random_"      ,0,0      ,0      ,0} /* RANDOMFUNCTION */
00223     ,{"ranperm_"     ,1,0      ,0      ,0} /* RANPERM */
00224     ,{"numfactors_"  ,0,0      ,0      ,0} /* NUMFACTORS */
00225     ,{"firstterm_"   ,0,0      ,0      ,0} /* FIRSTTERM */
00226     ,{"content_"     ,0,0      ,0      ,0} /* CONTENTTERM */
00227     ,{"prime_"       ,0,0      ,0      ,0} /* PRIMENUMBER */
00228     ,{"exteuclidean_" ,0,0      ,0      ,0} /* EXTEUCLIDEAN */
00229     ,{"makerational_" ,0,0      ,0      ,0} /* MAKERATIONAL */
00230     ,{"inverse_"     ,0,0      ,0      ,0} /* INVERSEFUNCTION */
00231     ,{"id_"          ,1,0      ,0      ,0} /* IDFUNCTION */
00232     ,{"putfirst_"    ,1,0      ,0      ,0} /* PUTFIRST */
00233     ,{"perm_"        ,1,0      ,0      ,0} /* PERMUTATIONS */
00234     ,{"partitions_"  ,1,0      ,0      ,0} /* PARTITIONS */
00235     ,{"mul_"         ,0,0      ,0      ,0} /* MULFUNCTION */
00236     ,{"moebius_"     ,0,0      ,0      ,0} /* MOEBIUS */
00237     ,{"topologies_"  ,0,0      ,0      ,0} /* TOPOLOGIES */
00238     ,{"diagrams_"    ,0,0      ,0      ,0} /* DIAGRAMS */
00239     ,{"topo_"        ,0,0      ,0      ,0} /* TOPO */
00240     ,{"node_"        ,0,0      ,0      ,0} /* VERTEX */
00241     ,{"edge_"        ,0,0      ,0      ,0} /* EDGE */
00242     ,{"sizeof_"      ,0,0      ,0      ,0} /* SIZEOFFUNCTION */
00243     ,{"block_"       ,0,0      ,0      ,0} /* BLOCK */
00244     ,{"onepi_"       ,0,0      ,0      ,0} /* ONEPI */
00245     ,{"phi_"         ,0, VERTEXFUNCTION,0} /* PHI */
00246 #ifdef WITHFLOAT
00247     ,{"float_"       ,0,0      ,0      ,0} /* FLOATFUN */
00248     ,{"tofloat_"     ,0,0      ,0      ,0} /* TOFLOAT */
00249     ,{"torat_"       ,0,0      ,0      ,0} /* TORAT */
00250     ,{"mzv_"        ,0,0      ,0      ,0} /* MZV */
00251     ,{"euler_"       ,0,0      ,0      ,0} /* EULER */
00252     ,{"mzvhalf_"     ,0,0      ,0      ,0} /* MZVHALF */
00253     ,{"agm_"         ,0,0      ,0      ,0} /* AGMFUNCTION */
00254     ,{"gamma_"       ,0,0      ,0      ,0} /* GAMMAFUN */
00255 #endif
00256 };
00257
00258 FIXEDSET fixedsets[] = {
00259     {"pos_" , "integers > 0", CSYMBOL, 0} /* POS_ 0 */
00260     ,{"pos0_" , "integers >= 0", CSYMBOL, 0} /* POS0_ 1 */
00261     ,{"neg_" , "integers < 0", CSYMBOL, 0} /* NEG_ 2 */
00262     ,{"neg0_" , "integers <= 0", CSYMBOL, 0} /* NEG0_ 3 */
00263     ,{"even_" , "even integers", CSYMBOL, 0} /* EVEN_ 4 */
00264     ,{"odd_" , "odd integers", CSYMBOL, 0} /* ODD_ 5 */
00265     ,{"int_" , "all integers", CSYMBOL, 0} /* Z_ 6 */
00266     ,{"symbol_" , "only symbols", CSYMBOL, MAXPOSITIVE} /* SYMBOL_ 7 */
00267     ,{"fixed_" , "fixed indices", CINDEX, 0} /* FIXED_ 8 */
00268     ,{"index_" , "all indices", CINDEX, 0} /* INDEX_ 9 */
00269     ,{"number_" , "all rationals", CSYMBOL, 0} /* Q_ 10 */
00270     ,{"dummyindices_" , "dummy indices", CINDEX, 0} /* DUMMYINDEX_ 11 */
00271     ,{"vector_" , "only vectors", CVECTOR, 0} /* VECTOR_ 12 */
00272 };
00273
00274 UBYTE BufferForOutput[MAXLINELENGTH+14];
00275
00276 char *setupfilename = "form.set";
00277
00278 #ifdef WITHPTHREADS
00279 INILOCK(ErrorMessageLock)
00280 INILOCK(FileReadLock)
00281 #endif

```

4.66 lus.c File Reference

```
#include "form3.h"
```

Include dependency graph for lus.c:



Functions

- int [Lus](#) (WORD *term, WORD funnum, WORD loopsize, WORD numargs, WORD outfun, WORD mode)
- int [FindLus](#) (int from, int level, int openindex)
- int [SortTheList](#) (int *slist, int num)
- WORD [AllLoops](#) (PHEAD WORD *term, WORD level)
- LONG [StartLoops](#) (PHEAD WORD *term, WORD level, LONG vert, WORD nvert, WORD *arglist, WORD nargs, WORD *loop, WORD nloop)
- LONG [GenLoops](#) (PHEAD WORD *term, WORD level, LONG vert, WORD nvert, WORD *arglist, WORD nargs, WORD *loop, WORD nloop)
- void [LoopOutput](#) (PHEAD WORD *term, WORD level, WORD *loop, WORD nloop)
- WORD [AllPaths](#) (PHEAD WORD *term, WORD level)
- LONG [GenPaths](#) (PHEAD WORD *term, WORD level, LONG vert, WORD nvert, WORD *arglist, WORD nargs, WORD *path, WORD npath)
- void [PathOutput](#) (PHEAD WORD *term, WORD level, WORD *path, WORD npath)
- WORD [AllOnePI](#) (WORD *term, WORD level)
- int [RemoveBridges](#) (void)
- int [TakeOneLine](#) (WORD *term, WORD level)
- int [OutputOnePI](#) (PHEAD WORD *term, WORD level)

4.66.1 Detailed Description

Routines to find loops in index contractions. These routines allow for a category of topological statements. They were originally developed for the color library.

Definition in file [lus.c](#).

4.66.2 Function Documentation

4.66.2.1 Lus()

```
int Lus (
    WORD * term,
    WORD funnum,
    WORD loopsize,
    WORD numargs,
    WORD outfun,
    WORD mode )
```

Definition at line 57 of file [lus.c](#).

4.66.2.2 FindLus()

```
int FindLus (
    int from,
    int level,
    int openindex )
```

Definition at line 515 of file [lus.c](#).

4.66.2.3 SortTheList()

```
int SortTheList (
    int * slist,
    int num )
```

Definition at line 578 of file [lus.c](#).

4.66.2.4 AllLoops()

```
WORD AllLoops (
    PHEAD WORD * term,
    WORD level )
```

Definition at line 650 of file [lus.c](#).

4.66.2.5 StartLoops()

```
LONG StartLoops (
    PHEAD WORD * term,
    WORD level,
    LONG vert,
    WORD nvert,
    WORD * arglist,
    WORD nargs,
    WORD * loop,
    WORD nloop )
```

Definition at line 894 of file [lus.c](#).

4.66.2.6 GenLoops()

```
LONG GenLoops (
    PHEAD WORD * term,
    WORD level,
    LONG vert,
    WORD nvert,
    WORD * arglist,
    WORD nargs,
    WORD * loop,
    WORD nloop )
```

Definition at line 980 of file [lus.c](#).

4.66.2.7 LoopOutput()

```
void LoopOutput (
    PHEAD WORD * term,
    WORD level,
    WORD * loop,
    WORD nloop )
```

Definition at line 1059 of file [lus.c](#).

4.66.2.8 AllPaths()

```
WORD AllPaths (
    PHEAD WORD * term,
    WORD level )
```

Definition at line 1123 of file [lus.c](#).

4.66.2.9 GenPaths()

```
LONG GenPaths (
    PHEAD WORD * term,
    WORD level,
    LONG vert,
    WORD nvert,
    WORD * arglist,
    WORD nargs,
    WORD * path,
    WORD npath )
```

Definition at line 1413 of file [lus.c](#).

4.66.2.10 PathOutput()

```
void PathOutput (
    PHEAD WORD * term,
    WORD level,
    WORD * path,
    WORD npath )
```

Definition at line 1486 of file [lus.c](#).

4.66.2.11 AllOnePI()

```
WORD AllOnePI (
    WORD * term,
    WORD level )
```

Definition at line 1566 of file [lus.c](#).

4.66.2.12 RemoveBridges()

```
int RemoveBridges (
    void )
```

Definition at line [1586](#) of file [lus.c](#).

4.66.2.13 TakeOneLine()

```
int TakeOneLine (
    WORD * term,
    WORD level )
```

Definition at line [1596](#) of file [lus.c](#).

4.66.2.14 OutputOnePI()

```
int OutputOnePI (
    PHEAD WORD * term,
    WORD level )
```

Definition at line [1608](#) of file [lus.c](#).

4.67 lus.c

[Go to the documentation of this file.](#)

```
00001
00007 /* #[] License : */
00008 /*
00009 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00010 * When using this file you are requested to refer to the publication
00011 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00012 * This is considered a matter of courtesy as the development was paid
00013 * for by FOM the Dutch physics granting agency and we would like to
00014 * be able to track its scientific use to convince FOM of its value
00015 * for the community.
00016 *
00017 * This file is part of FORM.
00018 *
00019 * FORM is free software: you can redistribute it and/or modify it under the
00020 * terms of the GNU General Public License as published by the Free Software
00021 * Foundation, either version 3 of the License, or (at your option) any later
00022 * version.
00023 *
00024 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00025 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00026 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00027 * details.
00028 *
00029 * You should have received a copy of the GNU General Public License along
00030 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00031 */
00032 /* #[] License : */
00033 /*
00034 #[] Includes : lus.c
00035 */
00036
00037 #include "form3.h"
00038
00039 /*
00040 #[] Includes :
00041 #[] Lus :
00042
00043 Routine to find loops.
00044 Mode: 0: Just tell whether there is such a loop.
```

```

00045         1: Take out the functions and replace by outfun with the
00046             remaining arguments of the loop function
00047         > AM.OffsetIndex: This index must be included in the loop.
00048         < -AM.OffsetIndex: This index must be included in the loop. Replace.
00049     Return value: 0: no loop. 1: there is/was such a loop.
00050     funnum: the function(s) in which we look for a loop.
00051     numargs: the number of arguments admissible in the function.
00052     outfun: the output function in case of a substitution.
00053     loopsize: the size of the loop we are looking for.
00054             if < 0 we look for all loops.
00055 */
00056
00057 int Lus(WORD *term, WORD funnum, WORD loopsize, WORD numargs, WORD outfun, WORD mode)
00058 {
00059     GETIDENTITY
00060     WORD *w, *t, *tt, *m, *r, **loc, *tstop, minloopsize;
00061     int nfun, i, j, jj, k, n, sign = 0, action = 0, L, ten, ten2, totnum,
00062     sign2, *alist, *wi, mini, maxi, medi = 0;
00063     if ( numargs <= 1 ) return(0);
00064     GETSTOP(term,tstop);
00065 /*
00066     First count the number of functions with the proper number of arguments.
00067 */
00068     t = term+1; nfun = 0;
00069     if ( ( ten = functions[funnum-FUNCTION].spec ) >= TENSORFUNCTION ) {
00070         while ( t < tstop ) {
00071             if ( *t == funnum && t[1] == FUNHEAD+numargs ) { nfun++; }
00072             t += t[1];
00073         }
00074     }
00075     else {
00076         while ( t < tstop ) {
00077             if ( *t == funnum ) {
00078                 i = 0; m = t+FUNHEAD; t += t[1];
00079                 while ( m < t ) { i++; NEXTARG(m) }
00080                 if ( i == numargs ) nfun++;
00081             }
00082             else t += t[1];
00083         }
00084     }
00085     if ( loopsize < 0 ) minloopsize = 2;
00086     else minloopsize = loopsize;
00087     if ( funnum < minloopsize ) return(0); /* quick abort */
00088     if ( ((functions[funnum-FUNCTION].symmetric) & ~REVERSEORDER) == ANTISYMMETRIC ) sign = 1;
00089     if ( mode == 1 || mode < 0 ) {
00090         ten2 = functions[outfun-FUNCTION].spec >= TENSORFUNCTION;
00091     }
00092     else ten2 = -1;
00093 /*
00094     Allocations:
00095 */
00096     if ( AN.numflocs < funnum ) {
00097         if ( AN.funlocs ) M_free(AN.funlocs,"Lus: AN.funlocs");
00098         AN.numflocs = funnum;
00099         AN.funlocs = (WORD **)Malloc1(sizeof(WORD *)*AN.numflocs,"Lus: AN.funlocs");
00100     }
00101     if ( AN.numfargs < funnum*numargs ) {
00102         if ( AN.funargs ) M_free(AN.funargs,"Lus: AN.funargs");
00103         AN.numfargs = funnum*numargs;
00104         AN.funargs = (int *)Malloc1(sizeof(int *)*AN.numfargs,"Lus: AN.funargs");
00105     }
00106 /*
00107     Make a list of relevant indices
00108 */
00109     alist = AN.funargs; loc = AN.funlocs;
00110     t = term+1;
00111     if ( ten >= TENSORFUNCTION ) {
00112         while ( t < tstop ) {
00113             if ( *t == funnum && t[1] == FUNHEAD+numargs ) {
00114                 *loc++ = t;
00115                 t += FUNHEAD;
00116                 j = i = numargs; while ( --i >= 0 ) {
00117                     if ( *t >= AM.OffsetIndex &&
00118                         ( *t >= AM.OffsetIndex+WILDOFFSET ||
00119                           indices[*t-AM.OffsetIndex].dimension != 0 ) ) {
00119                         *alist++ = *t++; j--;
00120                     }
00121                     else t++;
00122                 }
00123                 while ( --j >= 0 ) *alist++ = -1;
00124             }
00125             else t += t[1];
00126         }
00127     }
00128     else {
00129         nfun = 0;
00130         while ( t < tstop ) {

```

```

00132         if ( *t == funnum ) {
00133             w = t;
00134             i = 0; m = t+FUNHEAD; t += t[1];
00135             while ( m < t ) { i++; NEXTARG(m) }
00136             if ( i == numargs ) {
00137                 m = w + FUNHEAD;
00138                 while ( m < t ) {
00139                     if ( *m == -INDEX && m[1] >= AM.OffsetIndex &&
00140                         ( m[1] >= AM.OffsetIndex+WILDOFFSET ||
00141                         indices[m[1]-AM.OffsetIndex].dimension != 0 ) ) {
00142                         *alist++ = m[1]; m += 2; i--;
00143                     }
00144                     else if ( ten2 >= TENSORFUNCTION && *m != -INDEX
00145                         && *m != -VECTOR && *m != -MINVECTOR &&
00146                         ( *m != -SNUMBER || *m < 0 || *m >= AM.OffsetIndex ) ) {
00147                         i = numargs; break;
00148                     }
00149                     else { NEXTARG(m) }
00150                 }
00151                 if ( i < numargs ) {
00152                     *loc++ = w;
00153                     nfun++;
00154                     while ( --i >= 0 ) *alist++ = -1;
00155                 }
00156             }
00157             else t += t[1];
00158         }
00159     }
00160     if ( nfun < minloopsize ) return(0);
00161 }
00162 /*
00163 We have now nfun objects. Not all indices may be usable though.
00164 If the list is not long, we use a quadratic algorithm to remove
00165 indices and vertices that cannot be used. If it becomes large we
00166 sort the list of available indices (and their multiplicity) and
00167 work with binary searches.
00168 */
00169 alist = AN.funargs; totnum = numargs*nfun;
00170 if ( nfun > 7 ) {
00171     if ( AN.funisize < totnum ) {
00172         if ( AN.funinds ) M_free(AN.funinds,"AN.funinds");
00173         AN.funisize = (totnum*3)/2;
00174         AN.funinds = (int *)Malloc1(AN.funisize*2*sizeof(int),"AN.funinds");
00175     }
00176     i = totnum; n = 0; wi = AN.funinds;
00177     while ( --i >= 0 ) {
00178         if ( *alist >= 0 ) { n++; *wi++ = *alist; *wi++ = 1; }
00179         alist++;
00180     }
00181     n = SortTheList(AN.funinds,n);
00182     do {
00183         action = 0;
00184         for ( i = 0; i < nfun; i++ ) {
00185             alist = AN.funargs + i*numargs;
00186             jj = numargs;
00187             for ( j = 0; j < jj; j++ ) {
00188                 if ( alist[j] < 0 ) break;
00189                 mini = 0; maxi = n-1;
00190                 while ( mini <= maxi ) {
00191                     medi = (mini + maxi) / 2; k = AN.funinds[2*medi];
00192                     if ( alist[j] > k ) mini = medi + 1;
00193                     else if ( alist[j] < k ) maxi = medi - 1;
00194                     else break;
00195                 }
00196                 if ( AN.funinds[2*medi+1] <= 1 ) {
00197                     (AN.funinds[2*medi+1])--;
00198                     jj--; k = j; while ( k < jj ) { alist[k] = alist[k+1]; k++; }
00199                     alist[jj] = -1; j--;
00200                 }
00201             }
00202             if ( jj < 2 ) {
00203                 if ( jj == 1 ) {
00204                     mini = 0; maxi = n-1;
00205                     while ( mini <= maxi ) {
00206                         medi = (mini + maxi) / 2; k = AN.funinds[2*medi];
00207                         if ( alist[0] > k ) mini = medi + 1;
00208                         else if ( alist[0] < k ) maxi = medi - 1;
00209                         else break;
00210                     }
00211                     (AN.funinds[2*medi+1])--;
00212                     if ( AN.funinds[2*medi+1] == 1 ) action++;
00213                 }
00214                 nfun--; totnum -= numargs; AN.funlocs[i] = AN.funlocs[nfun];
00215                 wi = AN.funargs + nfun*numargs;
00216                 for ( j = 0; j < numargs; j++ ) alist[j] = *wi++;
00217                 i--;
00218             }

```

```

00219         }
00220     } while ( action );
00221 }
00222 else {
00223     for ( i = 0; i < totnum; i++ ) {
00224         if ( alist[i] == -1 ) continue;
00225         for ( j = 0; j < totnum; j++ ) {
00226             if ( alist[j] == alist[i] && j != i ) break;
00227         }
00228         if ( j >= totnum ) alist[i] = -1;
00229     }
00230     do {
00231         action = 0;
00232         for ( i = 0; i < nfun; i++ ) {
00233             alist = AN.funargs + i*numargs;
00234             n = numargs;
00235             for ( k = 0; k < n; k++ ) {
00236                 if ( alist[k] < 0 ) { alist[k--] = alist[--n]; alist[n] = -1; }
00237             }
00238             if ( n <= 1 ) {
00239                 if ( n == 1 ) { j = alist[0]; }
00240                 else j = -1;
00241                 nfun--; totnum -= numargs; AN.funlocs[i] = AN.funlocs[nfun];
00242                 wi = AN.funargs + nfun * numargs;
00243                 for ( k = 0; k < numargs; k++ ) alist[k] = wi[k];
00244                 i--;
00245                 if ( j >= 0 ) {
00246                     for ( k = 0, jj = 0, wi = AN.funargs; k < totnum; k++, wi++ ) {
00247                         if ( *wi == j ) { jj++; if ( jj > 1 ) break; }
00248                     }
00249                     if ( jj <= 1 ) {
00250                         for ( k = 0, wi = AN.funargs; k < totnum; k++, wi++ ) {
00251                             if ( *wi == j ) { *wi = -1; action = 1; }
00252                         }
00253                     }
00254                 }
00255             }
00256         }
00257     } while ( action );
00258 }
00259 if ( nfun < minloopsz ) return(0);
00260 /*
00261 Now we have nfun objects, each with at least 2 indices, each of which
00262 occurs at least twice in our list. There will be a loop!
00263 */
00264 if ( mode != 0 && mode != 1 ) {
00265     if ( mode > 0 ) AN.tohunt = mode - 5;
00266     else AN.tohunt = -mode - 5;
00267     AN.nargs = numargs; AN.numoffuns = nfun;
00268     i = 0;
00269     if ( loopsize < 0 ) {
00270         if ( loopsize == -1 ) k = nfun;
00271         else { k = -loopsize-1; if ( k > nfun ) k = nfun; }
00272         for ( L = 2; L <= k; L++ ) {
00273             if ( FindLus(0,L,AN.tohunt) ) goto Success;
00274         }
00275     }
00276     else if ( FindLus(0,loopsize,AN.tohunt) ) { L = loopsize; goto Success; }
00277 }
00278 else {
00279     AN.nargs = numargs; AN.numoffuns = nfun;
00280     if ( loopsize < 0 ) {
00281         jj = 2; k = nfun;
00282         if ( loopsize < -1 ) { k = -loopsize-1; if ( k > nfun ) k = nfun; }
00283     }
00284     else { jj = k = loopsize; }
00285     for ( L = jj; L <= k; L++ ) {
00286         for ( i = 0; i <= nfun-L; i++ ) {
00287             alist = AN.funargs + i * numargs;
00288             for ( jj = 0; jj < numargs; jj++ ) {
00289                 if ( alist[jj] < 0 ) continue;
00290                 AN.tohunt = alist[jj];
00291                 for ( j = jj+1; j < numargs; j++ ) {
00292                     if ( alist[j] < 0 ) continue;
00293                     if ( FindLus(i+1,L-1,alist[j]) ) {
00294                         alist[0] = alist[jj];
00295                         alist[1] = alist[j];
00296                         goto Success;
00297                     }
00298                 }
00299             }
00300         }
00301     }
00302 }
00303 return(0);
00304 Success:;
00305 if ( mode == 0 || mode > 1 ) return(1);

```



```

00306 /*
00307     Now we have to make the replacement and fix the potential sign
00308 */
00309     sign2 = 1;
00310     wi = AN.funargs + i*numargs; loc = AN.funlocs + i;
00311     for ( k = 0; k < L; k++ ) *(loc[k]) = -1;
00312     if ( AT.WorkPointer < term + *term ) AT.WorkPointer = term + *term;
00313     w = AT.WorkPointer + 1;
00314     m = t = term + 1;
00315     while ( t < tstop ) {
00316         if ( *t == -1 ) break;
00317         t += t[1];
00318     }
00319     while ( m < t ) *w++ = *m++;
00320     r = w;
00321     *w++ = outfun;
00322     w++;
00323     *w++ = DIRTYFLAG;
00324     FILLFUN3(w)
00325     if ( functions[outfun-FUNCTION].spec >= TENSORFUNCTION ) {
00326         if ( ten >= TENSORFUNCTION ) {
00327             for ( i = 0; i < L; i++ ) {
00328                 alist = wi + i*numargs;
00329                 m = loc[i] + FUNHEAD;
00330                 for ( k = 0; k < numargs; k++ ) {
00331                     if ( m[k] == alist[0] ) {
00332                         if ( k != 0 ) {
00333                             jj = m[k]; m[k] = m[0]; m[0] = jj;
00334                             sign = -sign;
00335                         }
00336                         break;
00337                     }
00338                 }
00339                 for ( k = 1; k < numargs; k++ ) {
00340                     if ( m[k] == alist[1] ) {
00341                         if ( k != 1 ) {
00342                             jj = m[k]; m[k] = m[1]; m[1] = jj;
00343                             sign = -sign;
00344                         }
00345                         break;
00346                     }
00347                 }
00348                 m += 2;
00349                 for ( k = 2; k < numargs; k++ ) *w++ = *m++;
00350             }
00351         }
00352         else {
00353             WORD *t1, *t2, *t3;
00354             for ( i = 0; i < L; i++ ) {
00355                 alist = wi + i*numargs;
00356                 tt = loc[i];
00357                 m = tt + FUNHEAD;
00358                 for ( k = 0; k < numargs; k++ ) {
00359                     if ( *m == -INDEX && m[1] == alist[0] ) {
00360                         if ( k != 0 ) {
00361                             if ( ( k & 1 ) != 0 ) sign = -sign;
00362                             /*
00363                                 now move to position 0
00364                             */
00365                             t2 = m+2; t1 = m; t3 = tt+FUNHEAD;
00366                             while ( t1 > t3 ) { *--t2 = *--t1; }
00367                             t3[0] = -INDEX; t3[1] = alist[0];
00368                         }
00369                         break;
00370                     }
00371                 }
00372                 NEXTARG(m)
00373                 m = tt + FUNHEAD + 2;
00374                 for ( k = 1; k < numargs; k++ ) {
00375                     if ( *m == -INDEX && m[1] == alist[1] ) {
00376                         if ( k != 1 ) {
00377                             if ( ( k & 1 ) == 0 ) sign = -sign;
00378                             /*
00379                                 now move to position 1
00380                             */
00381                             t2 = m+2; t1 = m; t3 = tt+FUNHEAD+2;
00382                             while ( t1 > t3 ) { *--t2 = *--t1; }
00383                             t3[0] = -INDEX; t3[1] = alist[1];
00384                         }
00385                         break;
00386                     }
00387                 }
00388                 NEXTARG(m)
00389             }
00390             /*
00391                 now copy the remaining arguments to w
00392                 keep in mind that the output function is a tensor!
00393             */

```

```

00393         t1 = tt + FUNHEAD + 4;
00394         t2 = tt + tt[1];
00395         while ( t1 < t2 ) {
00396             if ( *t1 == -INDEX || *t1 == -VECTOR ) {
00397                 *w++ = t1[1]; t1 += 2;
00398             }
00399             else if ( *t1 == -MINVECTOR ) {
00400                 *w++ = t1[1]; t1 += 2; sign2 = -sign2;
00401             }
00402             else if ( ( *t1 == -SNUMBER ) && ( t1[1] >= 0 ) && ( t1[1] < AM.OffsetIndex ) ) {
00403                 *w++ = t1[1]; t1 += 2; sign2 = -sign2;
00404             }
00405             else {
00406                 MLOCK(ErrorMessageLock);
00407                 MesPrint("Illegal attempt to use a non-index-like argument in a tensor in
ReplaceLoop statement");
00408                 MUNLOCK(ErrorMessageLock);
00409                 Terminate(-1);
00410             }
00411         }
00412     }
00413 }
00414 }
00415 else {
00416     if ( ten >= TENSORFUNCTION ) {
00417         for ( i = 0; i < L; i++ ) {
00418             alist = wi + i*numargs;
00419             m = loc[i] + FUNHEAD;
00420             for ( k = 0; k < numargs; k++ ) {
00421                 if ( m[k] == alist[0] ) {
00422                     if ( k != 0 ) {
00423                         jj = m[k]; m[k] = m[0]; m[0] = jj;
00424                         sign = -sign;
00425                         break;
00426                     }
00427                 }
00428             }
00429             for ( k = 1; k < numargs; k++ ) {
00430                 if ( m[k] == alist[1] ) {
00431                     if ( k != 1 ) {
00432                         jj = m[k]; m[k] = m[1]; m[1] = jj;
00433                         sign = -sign;
00434                         break;
00435                     }
00436                 }
00437             }
00438             m += 2;
00439             for ( k = 2; k < numargs; k++ ) {
00440                 if ( *m >= AM.OffsetIndex ) { *w++ = -INDEX; }
00441                 else if ( *m < 0 ) { *w++ = -VECTOR; }
00442                 else { *w = -SNUMBER; }
00443                 *w++ = *m++;
00444             }
00445         }
00446     }
00447     else {
00448         WORD *t1, *t2, *t3;
00449         for ( i = 0; i < L; i++ ) {
00450             alist = wi + i*numargs;
00451             tt = loc[i];
00452             m = tt + FUNHEAD;
00453             for ( k = 0; k < numargs; k++ ) {
00454                 if ( *m == -INDEX && m[1] == alist[0] ) {
00455                     if ( k != 0 ) {
00456                         if ( ( k & 1 ) != 0 ) sign = -sign;
00457                     /*
00458                         now move to position 0
00459                     */
00460                     t2 = m+2; t1 = m; t3 = tt+FUNHEAD;
00461                     while ( t1 > t3 ) { *--t2 = *--t1; }
00462                     t3[0] = -INDEX; t3[1] = alist[0];
00463                 }
00464                 break;
00465             }
00466             NEXTARG(m)
00467         }
00468         m = tt + FUNHEAD + 2;
00469         for ( k = 1; k < numargs; k++ ) {
00470             if ( *m == -INDEX && m[1] == alist[1] ) {
00471                 if ( k != 1 ) {
00472                     if ( ( k & 1 ) == 0 ) sign = -sign;
00473                 /*
00474                     now move to position 1
00475                 */
00476                 t2 = m+2; t1 = m; t3 = tt+FUNHEAD+2;
00477                 while ( t1 > t3 ) { *--t2 = *--t1; }
00478                 t3[0] = -INDEX; t3[1] = alist[1];

```

```

00479             }
00480             break;
00481         }
00482         NEXTARG(m)
00483     }
00484     /*
00485         now copy the remaining arguments to w
00486     */
00487     t1 = tt + FUNHEAD + 4;
00488     t2 = tt + tt[1];
00489     while ( t1 < t2 ) *w++ = *t1++;
00490 }
00491 }
00492 }
00493 r[1] = w-r;
00494 while ( t < tstop ) {
00495     if ( *t == -1 ) { t += t[1]; continue; }
00496     i = t[1];
00497     NCOPY(w,t,i)
00498 }
00499 tstop = term + *term;
00500 while ( t < tstop ) *w++ = *t++;
00501 if ( sign < 0 ) w[-1] = -w[-1];
00502 i = w - AT.WorkPointer;
00503 *AT.WorkPointer = i;
00504 t = term; w = AT.WorkPointer;
00505 NCOPY(t,w,i)
00506 *AN.RepPoint = 1; /* For Repeat */
00507 return(1);
00508 }
00509
00510 /*
00511 #] Lus :
00512 #[ FindLus :
00513 */
00514
00515 int FindLus(int from, int level, int openindex)
00516 {
00517     GETIDENTITY
00518     int i, j, k, jj, *alist, *blist, *w, *m, partner;
00519     WORD **loc = AN.funlocs, *wor;
00520     if ( level == 1 ) {
00521         for ( i = from; i < AN.numoffuns; i++ ) {
00522             alist = AN.funargs + i*AN.nargs;
00523             for ( j = 0; j < AN.nargs; j++ ) {
00524                 if ( alist[j] == openindex ) {
00525                     for ( k = 0; k < AN.nargs; k++ ) {
00526                         if ( k == j ) continue;
00527                         if ( alist[k] == AN.tohunt ) {
00528                             loc[from] = loc[i];
00529                             alist = AN.funargs + from*AN.nargs;
00530                             alist[0] = openindex; alist[1] = AN.tohunt;
00531                             return(1);
00532                         }
00533                     }
00534                 }
00535             }
00536         }
00537     }
00538     else {
00539         for ( i = from; i < AN.numoffuns; i++ ) {
00540             alist = AN.funargs + i*AN.nargs;
00541             for ( j = 0; j < AN.nargs; j++ ) {
00542                 if ( alist[j] == openindex ) {
00543                     if ( from != i ) {
00544                         wor = loc[i]; loc[i] = loc[from]; loc[from] = wor;
00545                         blist = w = AN.funargs + from*AN.nargs;
00546                         m = alist;
00547                         k = AN.nargs;
00548                         while ( --k >= 0 ) { jj = *m; *m++ = *w; *w++ = jj; }
00549                     }
00550                     else blist = alist;
00551                     for ( k = 0; k < AN.nargs; k++ ) {
00552                         if ( k == j || blist[k] < 0 ) continue;
00553                         partner = blist[k];
00554                         if ( FindLus(from+1, level-1, partner) ) {
00555                             blist[0] = openindex; blist[1] = partner;
00556                             return(1);
00557                         }
00558                     }
00559                     if ( from != i ) {
00560                         wor = loc[i]; loc[i] = loc[from]; loc[from] = wor;
00561                         w = AN.funargs + from*AN.nargs;
00562                         m = alist;
00563                         k = AN.nargs;
00564                         while ( --k >= 0 ) { jj = *m; *m++ = *w; *w++ = jj; }
00565                     }
00566                 }
00567             }
00568         }
00569     }
00570 }

```

```

00566     }
00567     }
00568     }
00569 }
00570 return(0);
00571 }
00572
00573 /*
00574 #] FindLus :
00575 #[ SortTheList :
00576 */
00577
00578 int SortTheList(int *slist, int num)
00579 {
00580     GETIDENTITY
00581     int i, nleft, nright, *t1, *t2, *t3, *rlist;
00582     if ( num <= 2 ) {
00583         if ( num <= 1 ) return(num);
00584         if ( slist[0] < slist[2] ) return(2);
00585         if ( slist[0] > slist[2] ) {
00586             i = slist[0]; slist[0] = slist[2]; slist[2] = i;
00587             i = slist[1]; slist[1] = slist[3]; slist[3] = i;
00588             return(2);
00589         }
00590         slist[1] += slist[3];
00591         return(1);
00592     }
00593     else {
00594         nleft = num/2; rlist = slist + 2*nleft;
00595         nright = SortTheList(rlist,num-nleft);
00596         nleft = SortTheList(slist,nleft);
00597         if ( AN.tlistsize < nleft ) {
00598             if ( AN.tlistbuf ) M_free(AN.tlistbuf,"AN.tlistbuf");
00599             AN.tlistsize = (nleft*3)/2;
00600             AN.tlistbuf = (int *)Malloc1(AN.tlistsize*2*sizeof(int),"AN.tlistbuf");
00601         }
00602         i = nleft; t1 = slist; t2 = AN.tlistbuf;
00603         while ( --i >= 0 ) { *t2++ = *t1++; *t2++ = *t1++; }
00604         i = nleft+nright; t1 = AN.tlistbuf; t2 = rlist; t3 = slist;
00605         while ( nleft > 0 && nright > 0 ) {
00606             if ( *t1 < *t2 ) {
00607                 *t3++ = *t1++; *t3++ = *t1++; nleft--;
00608             }
00609             else if ( *t1 > *t2 ) {
00610                 *t3++ = *t2++; *t3++ = *t2++; nright--;
00611             }
00612             else {
00613                 *t3++ = *t1++; t2++; *t3++ = (*t1++) + (*t2++); i--;
00614                 nleft--; nright--;
00615             }
00616         }
00617         while ( --nleft >= 0 ) { *t3++ = *t1++; *t3++ = *t1++; }
00618         while ( --nright >= 0 ) { *t3++ = *t2++; *t3++ = *t2++; }
00619         return(i);
00620     }
00621 }
00622
00623 /*
00624 #] SortTheList :
00625 #[ AllLoops :
00626
00627 Routine finds all possible loops that can be made in the
00628 arguments of the symmetric commuting vertex function v and creates
00629 for each loop a new term which is the original term times the loop.
00630 The occurrences of v can have different numbers of arguments.
00631 Each argument that occurs twice (and in different instances of v)
00632 in total will be considered.
00633
00634 The input parameters are in C->lhs[level] and are
00635 C->lhs[level][0] TYPEALLLOOPS
00636 C->lhs[level][1] 6
00637 C->lhs[level][2] Number of function v.
00638 C->lhs[level][3] Number of the output function loop.
00639 C->lhs[level][4] option1: the type of argument.
00640 C->lhs[level][5] option2: 0,1: what to do if no loop.
00641 Eligible arguments must be of the type SYMBOL, VECTOR, INDEX or SNUMBER.
00642 They must all be of the same type, indicated by option1.
00643 Extra restriction: In a loop, each v can be visited at most once.
00644 If there is no loop at all, option2 determines whether the term
00645 remains unmodified, or is replaced by zero.
00646 Function v can be either a regular function or a tensor.
00647 To facilitate this we copy the relevant arguments into the workspace.
00648 */
00649
00650 WORD AllLoops(PHEAD WORD *term,WORD level)
00651 {
00652     CBUF *C = cbuf+AM.rbufnum;

```

```

00653     WORD vcode = C->lhs[level][2];      /* The input function */
00654     WORD option1 = C->lhs[level][4];      /* type of argument */
00655     WORD option2 = C->lhs[level][5];      /* what to do when no loop */
00656     WORD *tstop, *t, *tend, *tstart, *tfrom;
00657     WORD i, j, jj;
00658     WORD *arglist, nargs, *loop, nloop;
00659     WORD *oldworkpointer = AT.WorkPointer;
00660     LONG oldpworkpointer = AT.pWorkPointer;
00661     LONG numgenerated = 0, vert;
00662     WORD *a, *a1, *a2, *a3, *v, *vv, nvert, *to, *from, *tos, action;
00663 /*
00664     Search for the first occurrence of vcode.
00665 */
00666     tstop = term+term; tstop -= ABS(tstop[-1]);
00667     t = term + 1;
00668     while ( t < tstop && *t != vcode ) t += t[1];
00669     if ( t == tstop ) {
00670         if ( option2 == 0 ) return(0);
00671         else return(Generator(BHEAD term, level));
00672     }
00673     tstart = t;
00674     nvert = 0;
00675     do {
00676         t += t[1]; nvert++;
00677     } while ( t < tstop && *t == vcode );
00678     tend = t;
00679 /*
00680     Make room for 2*nvert pointers
00681 */
00682     WantAddPointers(2*nvert);
00683     vert = AT.pWorkPointer;
00684     AT.pWorkPointer += 2*nvert;
00685     nvert = 0;
00686 /*
00687     Next we copy these functions into the workspace, but only the arguments
00688     that agree with option1. Because of the difference between tensors and
00689     regular functions in the copy we strip the type of the argument.
00690 */
00691     to = AT.WorkPointer;
00692     from = tstart;
00693     if ( functions[vcode-FUNCTION].spec == TENSORFUNCTION ) {
00694         from = tstart;
00695         while ( from < tend ) {
00696             tos = to++;
00697             tfrom = from+from[1];
00698             from += FUNHEAD;
00699             while ( from < tfrom ) {
00700                 if ( option1 == -INDEX && *from >= 0 ) {
00701                     *to++ = *from++;
00702                 }
00703                 else if ( option1 == -VECTOR && *from < 0 ) {
00704                     *to++ = *from++ - AM.OffsetVector;
00705                 }
00706                 else {
00707                     from++;
00708                 }
00709             }
00710             *tos = to-from;
00711             if ( *tos < 3 ) to = tos;
00712             else AT.pWorkspace[vert+nvert++] = tos;
00713         }
00714     }
00715     else if ( functions[vcode-FUNCTION].spec == 0 ) {
00716         from = tstart;
00717         while ( from < tend ) {
00718             tos = to++;
00719             tfrom = from + from[1];
00720             from += FUNHEAD;
00721             while ( from < tfrom ) {
00722                 if ( option1 == -VECTOR
00723                     && ( from[0] == -INDEX || from[0] == -VECTOR )
00724                     && from[1] < 0 ) {
00725                     from++;
00726                     *to++ = *from++ - AM.OffsetVector;
00727                 }
00728                 else if ( option1 == *from ) {
00729                     from++; *to++ = *from++;
00730                 }
00731                 else {
00732                     NEXTARG(from);
00733                 }
00734             }
00735             *tos = to-tos;
00736             if ( *tos < 3 ) to = tos;
00737             else AT.pWorkspace[vert+nvert++] = tos;
00738         }
00739     }

```

```

00740     else {
00741         AT.WorkPointer = oldworkpointer;
00742         AT.pWorkPointer = oldpworkpointer;
00743         if ( option2 == 0 ) return(0);
00744         return(Generator(BHEAD term,level));
00745     }
00746     AT.WorkPointer = to;
00747     /*
00748     Now make a list of all relevant arguments.
00749     */
00750     a = arglist = AT.WorkPointer;
00751     for ( i = 0; i < nvert; i++ ) {
00752         v = AT.pWorkSpace[vert+i];
00753         j = *v-1; v++;
00754         NCOPY(a,v,j);
00755     }
00756     nargs = a-arglist;
00757     AT.WorkPointer = a;
00758     /*
00759     Now sort the list. Bubble type sort.
00760     */
00761     a1 = arglist; a2 = a1+1;
00762     while ( a2 < a ) {
00763         a3 = a2;
00764         while ( a3 > a1 && a3[-1] > a3[0] ) {
00765             EXCH(*a3,a3[-1]);
00766             a3--;
00767         }
00768         a2++;
00769     }
00770     /*
00771     Now remove the elements from the list that do not occur exactly twice.
00772     */
00773     a1 = arglist; a2 = a1; a3 = a1+nargs;
00774     while ( a2 < a3 ) {
00775         if ( a2+1 == a3 ) { break; }
00776         else if ( a2+2 == a3 ) {
00777             if ( a2[0] == a2[1] ) { *a1++ = a2[0]; }
00778             break;
00779         }
00780         else {
00781             if ( a2[0] != a2[1] ) { a2++; }
00782             else if ( a2[0] != a2[2] ) {
00783                 *a1++ = a2[0]; a2 += 2;
00784             }
00785             else {
00786                 a2++; while ( a2 < a3 && a2[-1] == a2[0] ) a2++;
00787             }
00788         }
00789     }
00790     nargs = a1-arglist;
00791     /*
00792     Now we need to redo the list of vertices and remove the elements
00793     that are not in our arglist.
00794     */
00795     do {
00796         action = 0;
00797         for ( i = 0; i < nvert; i++ ) {
00798             vv = v = AT.pWorkSpace[vert+i];
00799             v++;
00800             for ( j = 1; j < vv[0]; j++ ) {
00801                 for ( jj = 0; jj < nargs; jj++ ) {
00802                     if ( *v == arglist[jj] ) break;
00803                 }
00804                 if ( jj >= nargs ) { /* was not in the list */
00805                     vv[0] = vv[0]-1;
00806                     for ( jj = j; jj < vv[0]; jj++ ) vv[jj] = vv[jj+1];
00807                 }
00808                 v++;
00809             }
00810         }
00811     } /*
00812     Next we need to remove vertices that have only one object remaining.
00813     Also, the object can be removed from arglist. After that we go back
00814     to clean up the list.
00815     */
00816     for ( i = 0; i < nvert; i++ ) {
00817         vv = AT.pWorkSpace[vert+i];
00818         if ( vv[0] == 1 ) {
00819             AT.pWorkSpace[vert+i] = AT.pWorkSpace[vert+nvert-1];
00820             nvert--; i--; continue;
00821         }
00822         else if ( vv[0] == 2 ) {
00823             for ( j = 0; j < nargs; j++ ) {
00824                 if ( arglist[j] == vv[1] ) break;
00825             }
00826             while ( j < nargs-1 ) arglist[j] = arglist[j+1];

```

```

00827         nargs--;
00828         AT.pWorkSpace[vert+i] = AT.pWorkSpace[vert+nvert-1];
00829         nvert--; i--;
00830         action = 1;
00831     }
00832 }
00833 } while ( action );
00834 /*
00835 Because the user can remove tadpoles rather easily as in
00836 id v(?a,i?,?b,i?,?c) = v(?a,?b,?c)
00837 there is no need to avoid them as potential loops.
00838
00839 At this point we are ready to look for loops.
00840 We have
00841     arglist,nargs    list of eligible objects.
00842     vert,nvert       position in pWorkSpace for vertices and their number.
00843                     The vertices themselves are in the WorkSpace.
00844     loop,nloop       The buildup of the loop.
00845 */
00846 loop = AT.WorkPointer;
00847 AT.WorkPointer += nargs;
00848 nloop = 0;
00849 numgenerated += StartLoops(BHEAD term,level,vert,nvert,arglist,nargs,loop,nloop);
00850 AT.WorkPointer = oldworkpointer;
00851 AT.pWorkPointer = oldpworkpointer;
00852
00853 if ( numgenerated == 0 && option2 != 0 ) return(Generator(BHEAD term,level));
00854 return(0);
00855 }
00856
00857 /*
00858 #] AllLoops :
00859 #[ StartLoops :
00860
00861 Algorithm.
00862 1: have a list of vertices and objects that can be part of the loops.
00863 2: starting with the first element of arglist, look for the two
00864    vertices that contain this object. The first is vstart.
00865 3: At the second, look at all possible objects to continue from here.
00866 4: For each, find the vertex with its partner.
00867 5: if the partner vertex is vstart, we have a loop. --> 9.
00868 6: The partner vertex is not allowed to be a vertex that we passed
00869    in the current build-up of the loop.
00870 7: Put the object in loop and increase nloop.
00871 8: Now sum over all allowed objects in the partner vertex. --> 4.
00872
00873 9: Create the function loop with the arguments from loop,nloop.
00874 10: If a reverse cyclic permutation gives a smaller result, drop this
00875     solution and continue with the last summing over objects at a vertex.
00876 11: If not, output the result by adding the function loop
00877     to the term and call Generator(term,level)
00878
00879 12: when all possibilities with the first element of arglist have been
00880     exhausted, raise arglist and lower nargs by one. --> 2
00881 13: when nargs == 1, we can stop.
00882
00883 The inclusion of a new object when we sum over the possibilities at a
00884 vertex can best be done in a recursion, because we have no idea how
00885 many nested loops we would otherwise need. Of course the recursion can
00886 be emulated, but that makes the algorithm rather messy.
00887
00888 We have two routines: StartLoops and GenLoops.
00889 StartLoops takes care of the first argument (and the summing over who
00890 is the first argument), while GenLoops treats an additional vertex until
00891 a loop is closed. GenLoops also sends completed loops off to Generator.
00892 */
00893
00894 LONG StartLoops(PHEAD WORD *term,WORD level,LONG vert,WORD nvert,
00895                 WORD *arglist,WORD nargs,WORD *loop,WORD nloop)
00896 {
00897     LONG numgenerated = 0;
00898     WORD *v, *vv, *vstart, istart, *vpartner, ipartner, j;
00899     while ( nargs > 1 ) {
00900 /*
00901     Look for a vertex with arglist[0]. This is our starting vertex.
00902     Next we look for its partner and we put arglist[0] in loop.
00903 */
00904         nloop = 0;
00905         loop[nloop++] = arglist[0];
00906         for ( istart = 0; istart < nvert; istart++ ) {
00907             vstart = AT.pWorkSpace[vert+istart];
00908             v = vstart+1; vv = vstart + *vstart;
00909             do {
00910                 if ( *v == arglist[0] ) goto havestart;
00911                 v++;
00912             } while ( v < vv );
00913         }

```

```

00914 /*
00915     If we come here, we have a problem.
00916 */
00917     MesPrint("Internal error in StartLoops. Object not found.");
00918     Terminate(-1);
00919     return(-1);
00920 havestart:
00921     AT.pWorkSpace[vert+nvert] = vstart;
00922 /*
00923     Check for tadpole.
00924 */
00925     v++;
00926     while ( v < vv ) {
00927         if ( *v == arglist[0] ) { /* tadpole */
00928             LoopOutput(BHEAD term,level,loop,nloop);
00929             numgenerated++;
00930             goto nextarg;
00931         }
00932         v++;
00933     }
00934 /*
00935     Now the partner vertex.
00936 */
00937     for ( ipartner = istart+1; ipartner < nvert; ipartner++ ) {
00938         vpartner = AT.pWorkSpace[vert+ipartner];
00939         vv = vpartner+vpartner; v = vpartner+1;
00940         do {
00941             if ( *v == arglist[0] ) goto havepartner;
00942             v++;
00943         } while ( v < vv );
00944     }
00945     return(numgenerated);
00946 havepartner:
00947     AT.pWorkSpace[vert+nvert+1] = vpartner;
00948 /*
00949     Now we run through all other possibilities at vpartner.
00950     vv is still OK.
00951 */
00952     v = vpartner+1;
00953     while ( v < vv ) {
00954         if ( *v != arglist[0] ) {
00955             for ( j = 1; j < nargs; j++ ) {
00956                 if ( *v == arglist[j] ) {
00957                     loop[nloop++] = *v;
00958                     numgenerated += GenLoops(BHEAD term,level,vert,nvert,
00959                                             arglist,nargs,loop,nloop);
00960                     nloop--;
00961                     break;
00962                 }
00963             }
00964         }
00965         v++;
00966     }
00967 nextarg:
00968     arglist++; nargs--;
00969 }
00970 return(numgenerated);
00971 }
00972
00973 /*
00974 #] StartLoops :
00975 #[ GenLoops :
00976
00977 We enter with an open line in loop[nloop-1]
00978 */
00979
00980 LONG GenLoops(PHEAD WORD *term,WORD level,WORD vert,WORD nvert,
00981               WORD *arglist,WORD nargs,WORD *loop,WORD nloop)
00982 {
00983     LONG numgenerated = 0;
00984     WORD *vstart, *v, *vv, i, j, *vpartner;
00985 /*
00986     Start with checking whether the partner is in vstart (=vert[nvert])
00987 */
00988     vstart = AT.pWorkSpace[nvert];
00989     vv = vstart + *vstart; v = vstart+1;
00990     while ( v < vv ) {
00991         if ( *v == loop[nloop-1] ) {
00992 /*
00993             This closes the loop.
00994             Now we can output it.
00995 */
00996             LoopOutput(BHEAD term,level,loop,nloop);
00997             numgenerated++;
00998             return(numgenerated);
00999         }
01000         v++;

```



```

01001     }
01002  /*
01003     Start with finding the partner.
01004  */
01005     for ( i = 0; i < nvert; i++ ) {
01006         vpartner = AT.pWorkSpace[vert+i];
01007         if ( vpartner == vstart ) continue;
01008         for ( j = 0; j < nloop; j++ ) {
01009             if ( vpartner == AT.pWorkSpace[vert+nvert+j] ) break;
01010         }
01011         if ( j < nloop ) continue;
01012         v = vpartner+1; vv = vpartner + *vpartner;
01013         while ( v < vv ) {
01014             if ( *v == loop[nloop-1] ) {
01015  /*
01016                 Found the partner.
01017                 Now we can sum over the remaining permitted arguments of this vertex.
01018  */
01019                 v = vpartner + 1;
01020                 while ( v < vv ) {
01021  /*
01022                     *v should be in arglist.
01023  */
01024                     for ( j = 0; j < nargs; j++ ) {
01025                         if ( *v == arglist[j] ) break;
01026                     }
01027                     if ( j >= nargs ) { v++; continue; }
01028  /*
01029                     *v should not be in loops.
01030  */
01031                     for ( j = 0; j < nloop; j++ ) {
01032                         if ( *v == loop[j] ) break;
01033                     }
01034                     if ( j >= nloop ) {
01035                         AT.pWorkSpace[vert+nvert+nloop] = vpartner;
01036                         loop[nloop++] = *v;
01037                         numgenerated += GenLoops(BHEAD term,level,vert,nvert,
01038                                                 arglist,nargs,loop,nloop);
01039                         nloop--;
01040                     }
01041                     v++;
01042                 }
01043                 return(numgenerated);
01044             }
01045             v++;
01046         }
01047     }
01048  /*
01049     The partner is in a vertex we have finished before.
01050  */
01051     return(numgenerated);
01052 }
01053
01054  /*
01055     #] GenLoops :
01056     #[ LoopOutput :
01057  */
01058
01059 void LoopOutput(PHEAD WORD *term, WORD level, WORD *loop, WORD nloop)
01060 {
01061     CBUF *C = cbuf+AM.rbufnum;
01062     WORD loopfun = C->lhs[level][3]; /* The output function */
01063     WORD option1 = C->lhs[level][4]; /* type of argument */
01064     WORD *tstop, *tstop1, *t, *tt;
01065     WORD *outterm, *loop1;
01066     WORD i;
01067     tstop1 = term+*term; tstop = tstop1 - ABS(tstop1[-1]);
01068  /*
01069     Construct the rcycle symmetrized version of loop.
01070  */
01071     if ( nloop > 2 ) {
01072         loop1 = AT.WorkPointer;
01073         loop1[0] = loop[0];
01074         for ( i = 1; i < nloop; i++ ) { loop1[i] = loop[nloop-i]; }
01075         if ( loop1[1] < loop[1] ) {
01076             AT.WorkPointer += nloop;
01077             loop = loop1;
01078         }
01079     }
01080     outterm = AT.WorkPointer;
01081     tt = outterm; t = term;
01082     while ( t < tstop ) *tt++ = *t++;
01083     *tt++ = loopfun;
01084     if ( functions[loopfun-FUNCTION].spec == TENSORFUNCTION ) {
01085         *tt++ = FUNHEAD+nloop;
01086         FILLFUN(tt)
01087         if ( option1 == -VECTOR ) {

```

```

01088         for ( i = 0; i < nloop; i++ ) *tt++ = loop[i]+AM.OffsetVector;
01089     }
01090     else {
01091         for ( i = 0; i < nloop; i++ ) *tt++ = loop[i];
01092     }
01093 }
01094 else {
01095     *tt++ = FUNHEAD+nloop*2;
01096     FILLFUN(tt)
01097     for ( i = 0; i < nloop; i++ ) {
01098         *tt++ = option1;
01099         if ( option1 == -VECTOR ) *tt++ = loop[i] + AM.OffsetVector;
01100         else *tt++ = loop[i];
01101     }
01102 }
01103 while ( t < tstop1 ) *tt++ = *t++;
01104 *outterm = tt - outterm;
01105 AT.WorkPointer = tt;
01106 if ( Generator(BHEAD outterm,level) ) {
01107     MesCall("LoopOutput");
01108     Terminate(-1);
01109 }
01110 AT.WorkPointer = outterm;
01111 }
01112
01113 /*
01114 #] LoopOutput :
01115 #[ AllPaths :
01116
01117 This routine has many similarities with AllLoops.
01118 In AllLoops we have startingpoint and endpoint at the same vertex.
01119 In AllPaths the startingpoint and the endpoints are different and
01120 given in advance. This endfun can occur only twice.
01121 */
01122
01123 WORD AllPaths(PHEAD WORD *term,WORD level)
01124 {
01125     CBUF *C = cbuf+AM.rbufnum;
01126     WORD endcode = C->lhs[level][2]; /* The endpoint function */
01127     WORD vcode = C->lhs[level][3]; /* The intermediate function */
01128     WORD option1 = C->lhs[level][5]; /* type of argument */
01129     WORD option2 = C->lhs[level][6]; /* what to do when no loop */
01130     WORD *t, *tstop, *tendl, *tend2, *tstart, *tend, numend, nvert, npass;
01131     WORD *tfrom, *to, *tos, *from;
01132     WORD *arglist, nargs, *path, npath, *a, *a1, *a2, *a3;
01133     WORD i, j, jj, *v, *vv, action;
01134     LONG vert,vert1; /* ,vert2; */
01135     WORD numgenerated = 0;
01136     WORD *oldworkpointer = AT.WorkPointer;
01137     LONG oldpworkpointer = AT.pWorkPointer;
01138 /*
01139 Search for the two occurrences of endcode.
01140 */
01141 tstop = term+*term; tstop -= ABS(tstop[-1]);
01142 t = term + 1;
01143 while ( t < tstop && *t != endcode ) t += t[1];
01144 if ( t == tstop ) { /* no endpoints */
01145     if ( option2 == 0 ) return(0);
01146     else return(Generator(BHEAD term,level));
01147 }
01148 tendl = t;
01149 numend = 0;
01150 while ( t < tstop && *t == endcode ) { tend2 = t; t += t[1]; numend++; }
01151 if ( numend != 2 ) { /* not 2 endpoints -> no path */
01152     if ( option2 == 0 ) return(0);
01153     else return(Generator(BHEAD term,level));
01154 }
01155 /*
01156 Search for the intermediate functions.
01157 */
01158 t = term + 1;
01159 nvert = 0;
01160 while ( t < tstop && *t != vcode ) t += t[1];
01161 tstart = t;
01162 while ( t < tstop && *t == vcode ) {
01163     t += t[1]; nvert++;
01164 }
01165 tend = t;
01166 /*
01167 Next we copy the relevant content of the various functions into the
01168 Workspace. We keep pointers to the functions in pWorkspace.
01169 First the two endpoints and then the intermediate ones.
01170 */
01171 WantAddPointers((2*nvert+8));
01172 vert = AT.pWorkPointer+2;
01173 vert1 = AT.pWorkPointer;
01174 /* vert2 = AT.pWorkPointer+1; */

```

```

01175     AT.pWorkPointer += 2*nvert+8;
01176     nvert = 0;
01177  /*
01178     Copy the endpoints.
01179  */
01180     to = AT.WorkPointer;
01181
01182     if ( functions[endcode-FUNCTION].spec == TENSORFUNCTION ) {
01183         from = tend1; npass = 0;
01184 redo1:
01185         tos = to++;
01186         tfrom = from+from[1];
01187         from += FUNHEAD;
01188         while ( from < tfrom ) {
01189             if ( option1 == -INDEX && *from >= 0 ) {
01190                 *to++ = *from++;
01191             }
01192             else if ( option1 == -VECTOR && *from < 0 ) {
01193                 *to++ = *from++ - AM.OffsetVector;
01194             }
01195             else {
01196                 from++;
01197             }
01198         }
01199         *tos = to-tos;
01200         if ( *tos < 2 ) to = tos;
01201         else AT.pWorkSpace[vert1+npass] = tos;
01202         npass++;
01203         if ( from == tend2 ) goto redo1;
01204     }
01205     else if ( functions[endcode-FUNCTION].spec == 0 ) {
01206         from = tend1;
01207         npass = 0;
01208 redo2:
01209         tos = to++;
01210         tfrom = from + from[1];
01211         from += FUNHEAD;
01212         while ( from < tfrom ) {
01213             if ( option1 == -VECTOR
01214                 && ( from[0] == -INDEX || from[0] == -VECTOR )
01215                 && from[1] < 0 ) {
01216                 from++;
01217                 *to++ = *from++ - AM.OffsetVector;
01218             }
01219             else if ( option1 == *from ) {
01220                 from++; *to++ = *from++;
01221             }
01222             else {
01223                 NEXTARG(from);
01224             }
01225         }
01226         *tos = to-tos;
01227         if ( *tos < 2 ) to = tos;
01228         else AT.pWorkSpace[vert1+npass] = tos;
01229         npass++;
01230         if ( from == tend2 ) goto redo2;
01231     }
01232     else {
01233         AT.WorkPointer = oldworkpointer;
01234         AT.pWorkPointer = oldpworkpointer;
01235         if ( option2 == 0 ) return(0);
01236         return(Generator(BHEAD term,level));
01237     }
01238  /*
01239     And now the intermediate functions
01240  */
01241     from = tstart;
01242     if ( functions[vcode-FUNCTION].spec == TENSORFUNCTION ) {
01243         from = tstart;
01244         while ( from < tend ) {
01245             tos = to++;
01246             tfrom = from+from[1];
01247             from += FUNHEAD;
01248             while ( from < tfrom ) {
01249                 if ( option1 == -INDEX && *from >= 0 ) {
01250                     *to++ = *from++;
01251                 }
01252                 else if ( option1 == -VECTOR && *from < 0 ) {
01253                     *to++ = *from++ - AM.OffsetVector;
01254                 }
01255                 else {
01256                     from++;
01257                 }
01258             }
01259             *tos = to-tos;
01260             if ( *tos < 3 ) to = tos;
01261             else AT.pWorkSpace[vert+nvert++] = tos;

```

```

01262     }
01263 }
01264 else if ( functions[vcode-FUNCTION].spec == 0 ) {
01265     from = tstart;
01266     while ( from < tend ) {
01267         tos = to++;
01268         tfrom = from + from[1];
01269         from += FUNHEAD;
01270         while ( from < tfrom ) {
01271             if ( option1 == -VECTOR
01272                 && ( from[0] == -INDEX || from[0] == -VECTOR )
01273                 && from[1] < 0 ) {
01274                 from++;
01275                 *to++ = *from++ - AM.OffsetVector;
01276             }
01277             else if ( option1 == *from ) {
01278                 from++; *to++ = *from++;
01279             }
01280             else {
01281                 NEXTARG(from);
01282             }
01283         }
01284         *tos = to-tos;
01285         if ( *tos < 3 ) to = tos;
01286         else AT.pWorkSpace[vert+nvert++] = tos;
01287     }
01288 }
01289 else {
01290     AT.WorkPointer = oldworkpointer;
01291     AT.pWorkPointer = oldpworkpointer;
01292     if ( option2 == 0 ) return(0);
01293     return(Generator(BHEAD term,level));
01294 }
01295 AT.WorkPointer = to;
01296 /*
01297 Now make a list of all relevant arguments.
01298 */
01299 a = arglist = AT.WorkPointer;
01300 for ( i = -2; i < nvert; i++ ) {
01301     v = AT.pWorkSpace[vert+i];
01302     j = *v-1; v++;
01303     NCOPY(a,v,j);
01304 }
01305 nargs = a-arglist;
01306 AT.WorkPointer = a;
01307 /*
01308 Now sort the list. Bubble type sort.
01309 */
01310 a1 = arglist; a2 = a1+1;
01311 while ( a2 < a ) {
01312     a3 = a2;
01313     while ( a3 > a1 && a3[-1] > a3[0] ) {
01314         EXCH(*a3,a3[-1]);
01315         a3--;
01316     }
01317     a2++;
01318 }
01319 /*
01320 Now remove the elements from the list that do not occur exactly twice.
01321 */
01322 a1 = arglist; a2 = a1; a3 = a1+nargs;
01323 while ( a2 < a3 ) {
01324     if ( a2+1 == a3 ) { break; }
01325     else if ( a2+2 == a3 ) {
01326         if ( a2[0] == a2[1] ) { *a1++ = a2[0]; }
01327         break;
01328     }
01329     else {
01330         if ( a2[0] != a2[1] ) { a2++; }
01331         else if ( a2[0] != a2[2] ) {
01332             *a1++ = a2[0]; a2 += 2;
01333         }
01334         else {
01335             a2++; while ( a2 < a3 && a2[-1] == a2[0] ) a2++;
01336         }
01337     }
01338 }
01339 nargs = a1-arglist;
01340 /*
01341 Now we need to redo the list of vertices and remove the elements
01342 that are not in our arglist.
01343 */
01344 do {
01345     action = 0;
01346     for ( i = -2; i < nvert; i++ ) {
01347         vv = v = AT.pWorkSpace[vert+i];
01348         v++;

```

```

01349     for ( j = 1; j < vv[0]; j++ ) {
01350         for ( jj = 0; jj < nargs; jj++ ) {
01351             if ( *v == arglist[jj] ) break;
01352         }
01353         if ( jj >= nargs ) { /* was not in the list */
01354             vv[0] = vv[0]-1;
01355             for ( jj = j; jj < vv[0]; jj++ ) vv[jj] = vv[jj+1];
01356         }
01357         v++;
01358     }
01359 }
01360 /*
01361 Next we need to remove vertices that have only one object remaining.
01362 Also, the object can be removed from arglist. After that we go back
01363 to clean up the list.
01364 */
01365 for ( i = -2; i < nvert; i++ ) {
01366     vv = AT.pWorkSpace[vert+i];
01367     if ( vv[0] == 1 ) {
01368         AT.pWorkSpace[vert+i] = AT.pWorkSpace[vert+nvert-1];
01369         nvert--; i--; continue;
01370     }
01371     else if ( vv[0] == 2 && i >= 0 ) {
01372         for ( j = 0; j < nargs; j++ ) {
01373             if ( arglist[j] == vv[1] ) break;
01374         }
01375         while ( j < nargs-1 ) arglist[j] = arglist[j+1];
01376         nargs--;
01377         AT.pWorkSpace[vert+i] = AT.pWorkSpace[vert+nvert-1];
01378         nvert--; i--;
01379         action = 1;
01380     }
01381 }
01382 } while ( action );
01383 /*
01384 Now we have a clean list of connections and we can start building
01385 the path. We start with summing over all objects in vert1.
01386 */
01387 path = AT.WorkPointer; npath = 0;
01388 AT.WorkPointer += nvert+8;
01389
01390 t = AT.pWorkSpace[vert1];
01391 if ( *t >= 2 ) {
01392     for ( i = 1; i < *t; i++ ) {
01393         AT.pWorkSpace[vert+nvert] = t;
01394         path[npath++] = t[i];
01395         numgenerated += GenPaths(BHEAD term, level, vert, nvert, arglist, nargs, path, npath);
01396         npath--;
01397     }
01398 }
01399 AT.WorkPointer = oldworkpointer;
01400 AT.pWorkPointer = oldpworkpointer;
01401 if ( numgenerated == 0 && option2 != 0 ) return(Generator(BHEAD term, level));
01402 return(0);
01403 }
01404
01405 /*
01406 #] AllPaths :
01407 #[ GenPaths :
01408
01409 We enter with an open line in path[npath-1]
01410 We traverse though the vertices until we reach vert-1, the endpoint.
01411 */
01412
01413 LONG GenPaths(PHEAD WORD *term, WORD level, LONG vert, WORD nvert,
01414 WORD *arglist, WORD nargs, WORD *path, WORD npath)
01415 {
01416     LONG numgenerated = 0;
01417     WORD *t, *vpartner, *v, *vv;
01418     WORD i, j;
01419 /*
01420 Check whether path[npath-1] is part of the endpoint.
01421 */
01422 t = AT.pWorkSpace[vert-1];
01423 for ( i = 1; i < *t; i++ ) {
01424     if ( t[i] == path[npath-1] ) { /* Got a path! */
01425         PathOutput(BHEAD term, level, path, npath);
01426         numgenerated++;
01427         return(numgenerated);
01428     }
01429 }
01430 /*
01431 Start with finding the partner.
01432 */
01433 for ( i = 0; i < nvert; i++ ) {
01434     vpartner = AT.pWorkSpace[vert+i];
01435     for ( j = 0; j < npath; j++ ) {

```

```

01436         if ( vpartner == AT.pWorkSpace[vert+nvert+j] ) break;
01437     }
01438     if ( j < npath ) continue;
01439     v = vpartner+1; vv = vpartner + *vpartner;
01440     while ( v < vv ) {
01441         if ( *v == path[npath-1] ) {
01442             /*
01443                 Found the partner.
01444                 Now we can sum over the remaining permitted arguments of this vertex.
01445             */
01446             v = vpartner + 1;
01447             while ( v < vv ) {
01448                 /*
01449                     *v should be in arglist.
01450                 */
01451                 for ( j = 0; j < nargs; j++ ) {
01452                     if ( *v == arglist[j] ) break;
01453                 }
01454                 if ( j >= nargs ) { v++; continue; }
01455             /*
01456                 *v should not be in path.
01457             */
01458             for ( j = 0; j < npath; j++ ) {
01459                 if ( *v == path[j] ) break;
01460             }
01461             if ( j >= npath ) {
01462                 AT.pWorkSpace[vert+nvert+npath] = vpartner;
01463                 path[npath++] = *v;
01464                 numgenerated += GenPaths(BHEAD term, level, vert, nvert,
01465                                         arglist, nargs, path, npath);
01466                 npath--;
01467             }
01468             v++;
01469         }
01470         return(numgenerated);
01471     }
01472     v++;
01473 }
01474 }
01475 /*
01476 The partner is in a vertex we have finished before.
01477 */
01478 return(numgenerated);
01479 }
01480
01481 /*
01482 #] GenPaths :
01483 #[ PathOutput :
01484 */
01485
01486 void PathOutput(PHEAD WORD *term, WORD level, WORD *path, WORD npath)
01487 {
01488     CBUF *C = cbuf+AM.rbufnum;
01489     WORD pathfun = C->lhs[level][4]; /* The output function */
01490     WORD option1 = C->lhs[level][5]; /* type of argument */
01491     WORD *tstop, *tstop1, *t, *tt;
01492     WORD *outterm;
01493     WORD i;
01494     tstop1 = term+*term; tstop = tstop1 - ABS(tstop1[-1]);
01495     outterm = AT.WorkPointer;
01496     tt = outterm; t = term;
01497     while ( t < tstop ) *tt++ = *t++;
01498     *tt++ = pathfun;
01499     if ( functions[pathfun-FUNCTION].spec == TENSORFUNCTION ) {
01500         *tt++ = FUNHEAD+npath;
01501         FILLFUN(tt)
01502         if ( option1 == -VECTOR ) {
01503             for ( i = 0; i < npath; i++ ) *tt++ = path[i]+AM.OffsetVector;
01504         }
01505         else {
01506             for ( i = 0; i < npath; i++ ) *tt++ = path[i];
01507         }
01508     }
01509     else {
01510         *tt++ = FUNHEAD+npath*2;
01511         FILLFUN(tt)
01512         for ( i = 0; i < npath; i++ ) {
01513             *tt++ = option1;
01514             if ( option1 == -VECTOR ) *tt++ = path[i] + AM.OffsetVector;
01515             else *tt++ = path[i];
01516         }
01517     }
01518     while ( t < tstop1 ) *tt++ = *t++;
01519     *outterm = tt - outterm;
01520     AT.WorkPointer = tt;
01521     if ( Generator(BHEAD outterm, level) ) {
01522         MesCall("PathOutput");

```

```

01523         Terminate(-1);
01524     }
01525     AT.WorkPointer = outterm;
01526 }
01527
01528
01529
01530 /*
01531 #] PathOutput :
01532 #[ AllOnePI :
01533
01534 We assume a graph that has loops and is OnePI.
01535 This routine initializes the recursion that creates all onePI subgraphs.
01536
01537 Algorithm:
01538     a: have a routine that removes all bridges: RemoveBridges
01539     b: output an empty diagram.
01540     c: output the diagram itself
01541     d: cut a line, followed by removing all bridges.
01542     e: if the diagram still has lines, go to c, else go to the next line in d.
01543 In order to avoid factorial blowup, we need to prune the tree as fast as possible.
01544
01545 1: list of lines to be cut.
01546 2: once we have tried a line and removed its bridges, we do not have to
01547    try this line deeper in the tree, neither its bridges.
01548
01549
01550 1 2 3
01551     cut 1.                x
01552         cut 2 -> bridge 3    x
01553     cut 2.                x
01554         cut 3 -> bridge 1    ???? mag niet
01555     cut 3.                x
01556 Hence 1,2,3,4,5,6,7
01557     1 still to go 2,3,4,5,6,7
01558     2 still to go 3,4,5,6,7
01559     3 still to go 4,5,6,7
01560     etc.
01561 bridges: if x is bridge, we take x from the list_to_go.
01562 If we have a bridge that is a number lower than the list_to_go we skip this possibility.
01563 We reach the end when the list_to_go is empty.
01564 */
01565
01566 WORD AllOnePI(WORD *term,WORD level)
01567 {
01568     CBUF *C = cbuf+AM.rbufnum;
01569     WORD vcode = C->lhs[level][2];    /* The vertex function */
01570     WORD option1 = C->lhs[level][4];    /* type of argument */
01571 /*
01572     First we have to collect all relevant information about the diagram.
01573     We should start with removing bridges to make the real starting point onePI.
01574 */
01575     DUMMYUSE(term)
01576     DUMMYUSE(vcode)
01577     DUMMYUSE(option1)
01578     return(0);
01579 }
01580
01581 /*
01582 #] AllOnePI :
01583 #[ RemoveBridges :
01584 */
01585
01586 int RemoveBridges(void)
01587 {
01588     return(0);
01589 }
01590
01591 /*
01592 #] RemoveBridges :
01593 #[ TakeOneLine :
01594 */
01595
01596 int TakeOneLine(WORD*term,WORD level)
01597 {
01598     DUMMYUSE(term)
01599     DUMMYUSE(level)
01600     return(0);
01601 }
01602
01603 /*
01604 #] TakeOneLine :
01605 #[ OutputOnePI :
01606 */
01607
01608 int OutputOnePI(PHEAD WORD *term,WORD level)
01609 {

```

```

01610     return(Generator(BHEAD term,level));
01611 }
01612
01613 /*
01614    #] OutputOnePI :
01615 */
01616

```

4.68 mallocprotect.h

```

00001 #ifndef MALLOCPROTECT_H
00002 #define MALLOCPROTECT_H
00003
00010 /* #] License : */
00011 /*
00012  * Copyright (C) 1984-2023 J.A.M. Vermaseren
00013  * When using this file you are requested to refer to the publication
00014  * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00015  * This is considered a matter of courtesy as the development was paid
00016  * for by FOM the Dutch physics granting agency and we would like to
00017  * be able to track its scientific use to convince FOM of its value
00018  * for the community.
00019  *
00020  * This file is part of FORM.
00021  *
00022  * FORM is free software: you can redistribute it and/or modify it under the
00023  * terms of the GNU General Public License as published by the Free Software
00024  * Foundation, either version 3 of the License, or (at your option) any later
00025  * version.
00026  *
00027  * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00028  * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00029  * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00030  * details.
00031  *
00032  * You should have received a copy of the GNU General Public License along
00033  * with FORM. If not, see <http://www.gnu.org/licenses/>.
00034 */
00035 /* #] License : */
00036
00037 /*
00038    #] Documentation :
00039
00040 The enhanced malloc debugger. It intercepts access (both write AND
00041 read) to the memory outside the allocated chunk in the following
00042 manner: it prints out (to the stderr) the information about the event
00043 and goes to the spillock. The user is able to attach the debugger to
00044 the running process, investigate the problem and continue evaluation.
00045
00046 In case of error, the debugger will print to the stderr the following
00047 information:
00048 "***** PID: ###, Addr: ###, signal ### (code ###)"
00049 "Attach gdb -p ### and set var loopForever=0 in a corresponding frame to continue"
00050 or for TFORM:
00051 "Attach gdb -p ### and set var loopForever=0 in a corresponding frame
00052   of a thread with LPW id ### to continue"
00053
00054 and go to the spillock. The user may attach gdb by the command
00055 gdb -p ### and type "where" (in case of TFORM, the user should first
00056 switch to the proper thread). Then investigation of the corresponding
00057 frames should clarify the situation. In order to continue the program,
00058 the user might set (in the corresponding frame of the corresponding
00059 thread) the variable loopForever to 0. The debugger will try to remove
00060 local problem lead to the exception.
00061
00062 To activate the debugger, the macro MALLOCPROTECT should be
00063 defined. There are three possible values:
00064 #define MALLOCPROTECT -1 (any integer <0)
00065 #define MALLOCPROTECT 0
00066 #define MALLOCPROTECT 1 (any integer>0)
00067
00068 If MALLOCPROTECT < 0, the debugger will intercept any access to the
00069 memory before the allocated chunk, even one byte.
00070
00071 If MALLOCPROTECT == 0, the debugger will intercept any access to the
00072 memory before the allocated chunk, even one byte, and access to the
00073 memory page next to the allocated one.
00074
00075 If MALLOCPROTECT > 0, the debugger will intercept any access to the
00076 memory after the allocated chunk, even one byte, and access to the
00077 memory page before the allocated one.
00078
00079 The original FORM malloc debugger is able to catch errors when the

```



```

00080 allocated memory is freed improper, or if some small portion OUT of
00081 the allocated chunk is written. This debugger is a complementary
00082 one. It permits to catch situation when some small portion of memory
00083 before or after the allocated chunk is written or even just read.
00084
00085 The idea is to protect the beginning or the end of the allocated chunk
00086 for any kind of access and install the SIGSEGV handler in order to
00087 intercept the access to this memory. Moreover, the allocated memory is
00088 always immediately returned to the system so if the user tries to
00089 access the memory after it is freed then the handler is triggered,
00090 also.
00091
00092 The problem here is that we are able to allocate / protect only the
00093 whole page. So the user has to run the debugger twice: one time with
00094 the left alignment (#define MALLOCPROTECT <0), and then with the right
00095 alignment (#define MALLOCPROTECT >0). During the first run the possible
00096 errors like x[-1] will be caught, and the second run will manifest
00097 reading over the allocated ares.
00098
00099 The leftmost extra page is always allocated and protected. The size
00100 of the allocated chunk is stored in the beginning of this page.
00101
00102     #] Documentation :
00103     #[ Includes :
00104 */
00105
00106 #include <stdio.h>
00107 #include <stdlib.h>
00108 #include <unistd.h>
00109 #include <signal.h>
00110 #include <sys/mman.h>
00111
00112 #ifdef WITHPTHREADS
00113 #ifdef LINUX
00114 #include <sys/syscall.h>
00115 #endif
00116 #endif
00117
00118 /*
00119     #] Includes :
00120 */
00121
00122 static int pageSize=4096; /*the default value*/
00123
00124
00125 /*
00126     #[ segv_handler :
00127 */
00128
00129 /*The handler will be invoked on some signal (usually SIGSEGV). It
00130 will print some diagnostics to stderr and hands up in infinite loop
00131 waiting the user for debugging:*/
00132 static void segv_handler(int sig, siginfo_t *sip, void *xxx) {
00133     char *vadr;
00134     char *errStr;
00135     int actionBeforeExit=0; /* < 0 - unprotect, > 0 - map */
00136     switch(sip->si_signo) { /* All POSIX signals for which the field siginfo_t.si_addr is defined:*/
00137         case SIGILL:
00138             switch(sip->si_code) {
00139                 case ILL_ILLOPC:
00140                     errStr="SIGILL: Illegal opcode";
00141                     break;
00142                 case ILL_ILLOPN:
00143                     errStr="SIGILL: Illegal operand";
00144                     break;
00145                 case ILL_ILLADR:
00146                     errStr="SIGILL: Illegal addressing mode";
00147                     break;
00148                 case ILL_ILLTRP:
00149                     errStr="SIGILL: Illegal trap";
00150                     break;
00151                 case ILL_PRVOPC:
00152                     errStr="SIGILL: Privileged opcode";
00153                     break;
00154                 case ILL_PRVREG:
00155                     errStr="SIGILL: Privileged register";
00156                     break;
00157                 case ILL_COPROC:
00158                     errStr="SIGILL: Coprocessor error";
00159                     break;
00160                 case ILL_BADSTK:
00161                     errStr="SIGILL: Internal stack error";
00162                     break;
00163                 default:
00164                     errStr="SIGILL: Unknown signal code";
00165             } /*case SIGILL:*/
00166             break;

```

```

00167         case SIGFPE:
00168             switch(sip->si_code){
00169                 case FPE_INTDIV:
00170                     errStr="SIGFPE: Integer divide-by-zero";
00171                     break;
00172                 case FPE_INTOVF:
00173                     errStr="SIGFPE: Integer overflow";
00174                     break;
00175                 case FPE_FLTDIV:
00176                     errStr="SIGFPE: Floating point divide-by-zero";
00177                     break;
00178                 case FPE_FLTOVF:
00179                     errStr="SIGFPE: Floating point overflow";
00180                     break;
00181                 case FPE_FLTUND:
00182                     errStr="SIGFPE: Floating point underflow";
00183                     break;
00184                 case FPE_FLTRES:
00185                     errStr="SIGFPE: Floating point inexact result";
00186                     break;
00187                 case FPE_FLTINV:
00188                     errStr="SIGFPE: Invalid floating point operation";
00189                     break;
00190                 case FPE_FLTSUB:
00191                     errStr="SIGFPE: Subscript out of range";
00192                     break;
00193                 default:
00194                     errStr="SIGFPE: Unknown signal code";
00195             }/*switch(sip->si_code)*/
00196             break;
00197         case SIGSEGV:
00198             switch(sip->si_code){
00199                 case SEGV_MAPERR:
00200                     errStr="SIGSEGV: Address not mapped";
00201                     actionBeforeExit = 1;
00202                     break;
00203                 case SEGV_ACCERR:
00204                     errStr="SIGSEGV: Invalid permissions";
00205                     actionBeforeExit = -1;
00206                     break;
00207                 default:
00208                     errStr="SIGSEGV: Unknown signal code";
00209             }/*switch(sip->si_code)*/
00210             break;
00211         case SIGBUS:
00212             switch(sip->si_code){
00213                 case BUS_ADRALN:
00214                     errStr="SIGBUS: Invalid address alignment";
00215                     break;
00216                 case BUS_ADRERR:
00217                     errStr="SIGBUS: Non-existent physical address";
00218                     break;
00219                 case BUS_OBJERR:
00220                     errStr="SIGBUS: Object-specific hardware error";
00221                     break;
00222                 default:
00223                     errStr="SIGBUS: Unknown signal code";
00224             }/*switch(sip->si_code)*/
00225             break;
00226         default:
00227             errStr="Unknown signal";
00228     }/*switch(sip->si_signo)*/
00229
00230     vadr = (caddr_t)sip->si_addr;
00231     fprintf(stderr, "\n***** PID: %ld, Addr: %p, signal %s (code %d)\n",
00232         (long) getpid(), vadr, errStr, sip->si_code);
00233
00234     /*Block*/
00235     /*The process hangs up at this block.
00236     Attach gdb to investigate and continue:
00237     */
00238     volatile int loopForever=1;
00239     size_t alignedAdr=((size_t)vadr) & (~ (pageSize-1));
00240     fprintf(stderr, "    Attach gdb -p %ld and set var loopForever=0 in a corresponding frame",
00241         (long) getpid());
00242     #ifdef CORRECTCODE
00243     #ifdef WITHPTHREADS
00244     #ifdef LINUX
00245         fprintf(stderr, "\n    of a thread with LPW id %ld", (long) syscall(SYS_gettid));
00246     #else
00247         /*If the compiler fails here, just remove the next line:*/
00248         fprintf(stderr, "\n    of thread with LPW id %ld", (long) gettid());
00249     #endif
00250     #endif
00251     #endif
00252     fprintf(stderr, " to continue\n");
00253

```

```

00254
00255     while(loopForever)
00256         sleep(1);
00257
00258     if(actionBeforeExit<0)/*After changing loopForever=0, unprotect the page to continue:*/
00259         mprotect((char*)alignedAdr, pageSize, PROT_READ | PROT_WRITE);
00260     if(actionBeforeExit>0)/*After changing loopForever=0, map the page to continue:*/
00261 /*         mmap((void*)alignedAdr,pageSize,PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, 0, 0); */
00262         mmap((void*)alignedAdr,pageSize,PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANON, 0, 0);
00263     }/*Block*/
00264 }/*segv_handler*/
00265
00266 /*
00267  #] segv_handler :
00268  */
00269
00270 /*
00271  #[ mprotectInit :
00272  */
00273
00274 static FORM_INLINE int mprotectInit(void)
00275 {
00276     struct sigaction sa;
00277     pageSize = getpagesize();
00278     sa.sa_sigaction = &segv_handler;
00279
00280     sigemptyset(&sa.sa_mask);
00281     sigaddset(&sa.sa_mask, SIGIO);
00282     sigaddset(&sa.sa_mask, SIGALRM);
00283
00284     sa.sa_flags = SA_SIGINFO;
00285
00286     if (sigaction(SIGILL, &sa, NULL)) {
00287         fprintf(stderr,"Error on assigning %s.\n","SIGILL");
00288         return SIGILL;
00289     }
00290     if (sigaction(SIGFPE, &sa, NULL)) {
00291         fprintf(stderr,"Error on assigning %s.\n","SIGFPE");
00292         return SIGFPE;
00293     }
00294     if (sigaction(SIGSEGV, &sa, NULL)) {
00295         fprintf(stderr,"Error on assigning %s.\n","SIGSEGV");
00296         return SIGSEGV;
00297     }
00298     if (sigaction(SIGBUS, &sa, NULL)) {
00299         fprintf(stderr,"Error on assigning %s.\n","SIGBUS");
00300         return SIGBUS;
00301     }
00302     return 0;
00303 }/*mprotectInit*/
00304
00305 /*
00306  #] mprotectInit :
00307  */
00308
00309
00310 /*
00311  #[ mprotectMalloc :
00312  */
00313
00314 static void *mprotectMalloc(size_t theSize)
00315 {
00316     #if MALLOCPROTECT < 0
00317         /*Only one side is protected*/
00318         size_t nPages=1;
00319     #else
00320         /*Both sides are protected*/
00321         size_t nPages=2;
00322     #if MALLOCPROTECT > 0
00323         /*will need the original theSize*/
00324         size_t requestedSize=theSize;
00325     #endif
00326     #endif
00327
00328     char *ret=NULL;
00329     size_t *ptr;
00330
00331
00332
00333     /*Align required size to the pagesize:*/
00334     if(theSize % pageSize)
00335         nPages++;
00336     theSize= (theSize/pageSize+nPages)*pageSize;
00337
00338 /*     ret=(char*)mmap(0,theSize,PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, 0, 0); */
00339     ret=(char*)mmap(0,theSize,PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANON, 0, 0);
00340     if(ret == MAP_FAILED)

```

```

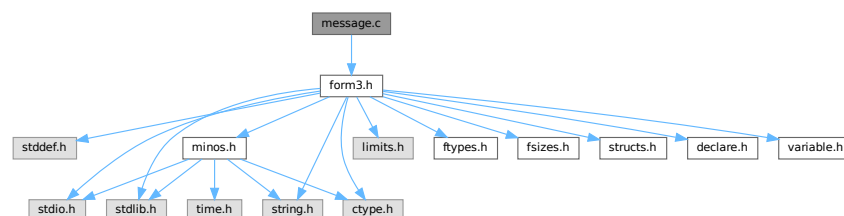
00341     return NULL;
00342     ptr=(size_t *)ret;
00343     *ptr=theSize;
00344     if(mprotect(ret, pageSize, PROT_NONE))return NULL;
00345     #if MALLOCPROTECT < 0
00346     return ret+pageSize;
00347     #else
00348     if(mprotect(ret+(theSize-pageSize), pageSize, PROT_NONE))return NULL;
00349     #if MALLOCPROTECT ==0
00350     return ret+pageSize;
00351     #endif
00352     #endif
00353     /*MALLOCPROTECT > 0, but we need conditional compilation
00354     since the variable requestedSize:*/
00355     #if MALLOCPROTECT > 0
00356
00357     /*Potential problems with alignment if the requested size is not
00358     a multiple of items. But no problems on x86-64:*/
00359     return ret+ (theSize-pageSize-requestedSize);
00360     #endif
00361 }/*mprotectMalloc*/
00362 /*
00363  #] mprotectMalloc :
00364  */
00365 /*
00366  #[ mprotectFree :
00367  */
00368 */
00369 static void mprotectFree(char *ptr)
00370 {
00371     size_t theSize;
00372     if(ptr==NULL) return;
00373     #if MALLOCPROTECT > 0
00374     /*The memory block was moved to the right, find the left page boundary:*/
00375     { /*Block*/
00376         size_t alignedPtr=((size_t)ptr) & ~(pageSize-1);
00377         ptr=(char*)alignedPtr;
00378     } /*Block*/
00379     #endif
00380     #endif
00381     ptr-=pageSize;
00382     mprotect(ptr, pageSize, PROT_READ);
00383     theSize=((size_t*)ptr);
00384     munmap(ptr,theSize);
00385 }/*mprotectFree*/
00386 /*
00387  #] mprotectFree :
00388  */
00389 */
00390 */
00391 #endif
00392 #endif

```

4.69 message.c File Reference

#include "form3.h"

Include dependency graph for message.c:



Functions

- void [Error0](#) (char *s)

- void [Error1](#) (char *s, UBYTE *t)
- void [Error2](#) (char *s1, char *s2, UBYTE *t)
- int [MesWork](#) (void)
- int [MesPrint](#) (va_alist)
- void [Warning](#) (char *s)
- void [HighWarning](#) (char *s)
- int [MesCall](#) (char *s)
- WORD [MesCerr](#) (char *s, UBYTE *t)
- WORD [MesComp](#) (char *s, UBYTE *p, UBYTE *q)
- void [PrintTerm](#) (WORD *term, char *where)
- void [PrintTermC](#) (WORD *term, char *where)
- void [PrintSubTerm](#) (WORD *term, char *where)
- void [PrintWords](#) (WORD *buffer, LONG number)
- void [PrintSeq](#) (WORD *a, char *text)

4.69.1 Detailed Description

Contains the routines that write messages. This includes the very important routine MesPrint which is the FORM equivalent of printf but then with escape sequences that are relevant for symbolic manipulation. The FORM statement Print "..." is passed almost literally to MesPrint.

Definition in file [message.c](#).

4.69.2 Function Documentation

4.69.2.1 Error0()

```
void Error0 (  
    char * s )
```

Definition at line 56 of file [message.c](#).

4.69.2.2 Error1()

```
void Error1 (  
    char * s,  
    UBYTE * t )
```

Definition at line 67 of file [message.c](#).

4.69.2.3 Error2()

```
void Error2 (  
    char * s1,  
    char * s2,  
    UBYTE * t )
```

Definition at line 78 of file [message.c](#).

4.69.2.4 MesWork()

```
int MesWork (
    void )
```

Definition at line 89 of file [message.c](#).

4.69.2.5 MesPrint()

```
int MesPrint (
    va_alist )
```

Definition at line 140 of file [message.c](#).

4.69.2.6 Warning()

```
void Warning (
    char * s )
```

Definition at line 790 of file [message.c](#).

4.69.2.7 HighWarning()

```
void HighWarning (
    char * s )
```

Definition at line 802 of file [message.c](#).

4.69.2.8 MesCall()

```
int MesCall (
    char * s )
```

Definition at line 814 of file [message.c](#).

4.69.2.9 MesCerr()

```
WORD MesCerr (
    char * s,
    UBYTE * t )
```

Definition at line 824 of file [message.c](#).

4.69.2.10 MesComp()

```
WORD MesComp (
    char * s,
    UBYTE * p,
    UBYTE * q )
```

Definition at line 843 of file [message.c](#).

4.69.2.11 PrintTerm()

```
void PrintTerm (
    WORD * term,
    char * where )
```

Definition at line [857](#) of file [message.c](#).

4.69.2.12 PrintTermC()

```
void PrintTermC (
    WORD * term,
    char * where )
```

Definition at line [887](#) of file [message.c](#).

4.69.2.13 PrintSubTerm()

```
void PrintSubTerm (
    WORD * term,
    char * where )
```

Definition at line [921](#) of file [message.c](#).

4.69.2.14 PrintWords()

```
void PrintWords (
    WORD * buffer,
    LONG number )
```

Definition at line [943](#) of file [message.c](#).

4.69.2.15 PrintSeq()

```
void PrintSeq (
    WORD * a,
    char * text )
```

Definition at line [961](#) of file [message.c](#).

4.70 message.c

[Go to the documentation of this file.](#)

```

00001
00009 /* #[] License : */
00010 /*
00011 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00012 *   When using this file you are requested to refer to the publication
00013 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00014 *   This is considered a matter of courtesy as the development was paid
00015 *   for by FOM the Dutch physics granting agency and we would like to
00016 *   be able to track its scientific use to convince FOM of its value
00017 *   for the community.
00018 *
00019 *   This file is part of FORM.
00020 *
00021 *   FORM is free software: you can redistribute it and/or modify it under the
00022 *   terms of the GNU General Public License as published by the Free Software
00023 *   Foundation, either version 3 of the License, or (at your option) any later
00024 *   version.
00025 *
00026 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00027 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00028 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00029 *   details.
00030 *
00031 *   You should have received a copy of the GNU General Public License along
00032 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00033 */
00034 /* #[] License : */
00035 /*
00036 *   #[] Includes :
00037 *
00038 *   The static variables for the messages can remain as such also for
00039 *   the parallel version as messages are to be locked to avoid problems
00040 *   with simultaneous messages.
00041 */
00042
00043 #include "form3.h"
00044
00045 static int iswarning = 0;
00046
00047 static char hex[] = {'0','1','2','3','4','5','6','7','8','9',
00048                     'A','B','C','D','E','F'};
00049
00050 /*
00051 *   #[] Includes :
00052 *   #[] exit :
00053 *   #[] Error0 :
00054 */
00055
00056 void Error0(char *s)
00057 {
00058     MesPrint("=== %s",s);
00059     Terminate(-1);
00060 }
00061
00062 /*
00063 *   #[] Error0 :
00064 *   #[] Error1 :
00065 */
00066
00067 void Error1(char *s, UBYTE *t)
00068 {
00069     MesPrint("@%s %s",s,t);
00070     Terminate(-1);
00071 }
00072
00073 /*
00074 *   #[] Error1 :
00075 *   #[] Error2 :
00076 */
00077
00078 void Error2(char *s1, char *s2, UBYTE *t)
00079 {
00080     MesPrint("@%s%s %s",s1,s2,t);
00081     Terminate(-1);
00082 }
00083
00084 /*
00085 *   #[] Error2 :
00086 *   #[] MesWork :
00087 */
00088
00089 int MesWork(void)

```



```

00090 {
00091     MesPrint("=== Workspace overflow. %l bytes is not enough.",AM.WorkSize);
00092     MesPrint("=== Change parameter WorkSpace in %s",setupfilename);
00093     Terminate(-1);
00094     return(-1);
00095 }
00096
00097 /*
00098     #] MesWork :
00099     #[ MesPrint :
00100
00101     Kind of a printf function for simple messages.
00102     The main concern is getting the arguments in a portable way.
00103     Note: many compilers have errors when sizeof(WORD) < sizeof(int)
00104     %a array of size n WORDs (two parameters, first is int, second WORD *)
00105     %b array of size n UBYTES (two parameters, first is int, second UBYTE *)
00106     %C array of size n chars (two parameters, first is int, second char *)
00107     %d word;
00108     %l long;
00109     %L long long *;
00110     %s string;
00111     %#i unsigned word filled
00112     %#d word positioned
00113     %#l long word positioned.
00114     %#L long long word * positioned.
00115     %#s string positioned.
00116     %#p position in file.
00117     %r The current term in raw format (internal representation)
00118     %t The current term (AN.currentTerm)
00119     %T The current term (AN.currentTerm) with its sign
00120     %w Number of the thread(worker)
00121     %$ The next $ in AN.listinprint
00122     %x hexadecimal. Takes 8 places. Mainly for debugging.
00123     %% %
00124     %# #
00125     # " ==> "
00126     @ " ==> " Preprocessor error
00127     & ' --> ' Regular compiler error
00128     Each call is terminated with a new line.
00129     Put a % at the end of the string to suppress the new line.
00130
00131     New feature (7-dec-2011): The & will only work when we do not block it
00132     from the execution of the print statement because we need the & also for
00133     the tabulator in the print "" statement.
00134 */
00135
00136 int
00137 #ifdef ANSI
00138 MesPrint(const char *fmt, ... )
00139 #else
00140 MesPrint(va_alist)
00141 va_dcl
00142 #endif
00143 {
00144     GETIDENTITY
00145     char Out[MAXLINELENGTH+14], *stopper, *t, *s, *u, c, *carray;
00146     UBYTE extrabuffer[MAXLINELENGTH+14];
00147     int w, x, i, specialerror = 0;
00148     LONG num, y;
00149     WORD *array;
00150     UBYTE *oldoutfill = AO.OutputLine, *barray;
00151     /*[19apr2004 mt]:*/
00152     LONG (*OldWrite)(int handle, UBYTE *buffer, LONG size) = WriteFile;
00153     /*:[19apr2004 mt]*/
00154     va_list ap;
00155     #ifdef ANSI
00156         va_start(ap,fmt);
00157         s = (char *)fmt;
00158     #else
00159         va_start(ap);
00160         s = va_arg(ap,char *);
00161     #endif
00162     #ifdef WITHMPI
00163         /*
00164          * On slaves, if AS.printflag is
00165          * = 0 : print nothing.
00166          * > 0 : synchronized output. All text will be sent to the master
00167          * in the next MUNLOCK().
00168          * < 0 : normal output.
00169          */
00170         if ( PF.me != MASTER && AS.printflag == 0 ) return(0);
00171         if ( PF.me == MASTER || AS.printflag < 0 )
00172     #endif
00173         FLUSHCONSOLE;
00174         /*
00175          * MesPrints() never prints a message to an external channel even if
00176          * WriteFile is set to &WriteToExternalChannel.

```

```

00177      */
00178 #ifndef WITHMPI
00179     WriteFile = PF.me == MASTER || AS.printflag > 0 ? &PF_WriteFileToFile : &WriteFileToFile;
00180 #else
00181     WriteFile = &WriteFileToFile;
00182 #endif
00183     AO.OutputLine = extrabuffer;
00184     t = Out;
00185     stopper = Out + AC.LineLength;
00186     while ( *s ) {
00187         if ( ( ( *s == '&' && AO.ErrorBlock == 0 ) || *s == '@' || *s == '#' ) && AC.CurrentStream !=
0 ) {
00188             u = (char *)AC.CurrentStream->name;
00189             while ( *u ) {
00190                 *t++ = *u++;
00191                 if ( t >= stopper ) {
00192                     num = t - Out;
00193                     WriteString(ERROROUT, (UBYTE *)Out, num);
00194                     num = 0; t = Out;
00195                 }
00196             }
00197             *t++ = ' ';
00198             if ( t+20 >= stopper ) {
00199                 num = t - Out;
00200                 WriteString(ERROROUT, (UBYTE *)Out, num);
00201                 num = 0; t = Out;
00202             }
00203             *t++ = 'L'; *t++ = 'i'; *t++ = 'n'; *t++ = 'e'; *t++ = ' ';
00204             if ( *s == '&' ) y = AC.CurrentStream->prevline;
00205             else y = AC.CurrentStream->linenumber;
00206             t = LongCopy(y, t);
00207             if ( !iswarning && ( *s == '&' || *s == '@' ) ) {
00208                 for ( i = 0; i < NumDoLoops; i++ ) DoLoops[i].errorsinloop = 1;
00209             }
00210         }
00211         if ( ( *s == '&' && AO.ErrorBlock == 0 ) ) {
00212             *t++ = ' '; *t++ = '-'; *t++ = '-'; *t++ = '>'; *t++ = ' '; s++;
00213         }
00214         else if ( *s == '@' || *s == '#' ) {
00215             *t++ = ' '; *t++ = '='; *t++ = '='; *t++ = '>'; *t++ = ' '; s++;
00216         }
00217         /*
00218         else if ( *s == '&' && AO.ErrorBlock == 1 ) {
00219         }
00220         */
00221         /*
00222         else if ( *s != '%' ) {
00223             *t++ = *s++;
00224             if ( t >= stopper ) {
00225                 num = t - Out;
00226                 WriteString(ERROROUT, (UBYTE *)Out, num);
00227                 num = 0; t = Out;
00228             }
00229         }
00230         else {
00231             s++;
00232             if ( *s == 'd' ) {
00233                 if ( ( w = va_arg(ap, int) ) < 0 ) { *t++ = '-'; w = -w; }
00234                 t = (char *)NumCopy(w, (UBYTE *)t);
00235             }
00236             else if ( *s == 'l' ) {
00237                 if ( ( y = va_arg(ap, LONG) ) < 0 ) { *t++ = '-'; y = -y; }
00238                 t = LongCopy(y, t);
00239             }
00240             /* #ifdef __GLIBC_HAVE_LONG_LONG */
00241             else if ( *s == 'p' ) {
00242                 POSITION *pp;
00243                 off_t ly;
00244                 pp = va_arg(ap, POSITION *);
00245                 ly = BASEPOSITION(*pp);
00246                 if ( ly < 0 ) { *t++ = '-'; ly = -ly; }
00247                 /*----change 10-feb-2003 did not have & */
00248                 t = LongLongCopy(&(ly), t);
00249             }
00250             /* #endif */
00251             else if ( *s == 'c' ) {
00252                 c = (char)(va_arg(ap, int));
00253                 *t++ = c; *t = 0;
00254             }
00255             else if ( *s == 'a' ) {
00256                 w = va_arg(ap, int);
00257                 array = va_arg(ap, WORD *);
00258                 while ( w > 0 ) {
00259                     t = (char *)NumCopy(*array, (UBYTE *)t);
00260                     if ( t >= stopper ) {
00261                         num = t - Out;
00262                         WriteString(ERROROUT, (UBYTE *)Out, num);

```

```

00263         t = Out;
00264         *t++ = ' ';
00265     }
00266     *t++ = ' ';
00267     w--; array++;
00268 }
00269 }
00270 else if ( *s == 'b' ) {
00271     w = va_arg(ap, int);
00272     barray = va_arg(ap, UBYTE *);
00273     while ( w > 0 ) {
00274         *t++ = hex[ ((barray)>>4)&0xF];
00275         *t++ = hex[ (barray)&0xF];
00276         *t = 0;
00277         if ( t >= stopper ) {
00278             num = t - Out;
00279             WriteString(ERROROUT, (UBYTE *)Out, num);
00280             t = Out;
00281             *t++ = ' ';
00282         }
00283         *t++ = ' ';
00284         w--; barray++;
00285     }
00286 }
00287 else if ( *s == 'C' ) {
00288     w = va_arg(ap, int);
00289     carray = va_arg(ap, char *);
00290     while ( w > 0 ) {
00291         if ( *carray < 32 ) *t++ = '^';
00292         else *t++ = *carray;
00293         *t = 0;
00294         if ( t >= stopper ) {
00295             num = t - Out;
00296             WriteString(ERROROUT, (UBYTE *)Out, num);
00297             t = Out;
00298             *t++ = ' ';
00299         }
00300         w--; carray++;
00301     }
00302 }
00303 else if ( *s == 'I' ) {
00304     int *iarray;
00305     w = va_arg(ap, int);
00306     iarray = va_arg(ap, int *);
00307     while ( w > 0 ) {
00308         t = (char *)LongCopy((LONG)(*iarray), (char *)t);
00309         if ( t >= stopper ) {
00310             num = t - Out;
00311             WriteString(ERROROUT, (UBYTE *)Out, num);
00312             t = Out;
00313             *t++ = ' ';
00314         }
00315         *t++ = ' ';
00316         w--; array++;
00317     }
00318 }
00319 else if ( *s == 'E' ) {
00320     LONG *larray;
00321     w = va_arg(ap, int);
00322     larray = va_arg(ap, LONG *);
00323     while ( w > 0 ) {
00324         t = (char *)LongCopy(*larray, (char *)t);
00325         if ( t >= stopper ) {
00326             num = t - Out;
00327             WriteString(ERROROUT, (UBYTE *)Out, num);
00328             t = Out;
00329             *t++ = ' ';
00330         }
00331         *t++ = ' ';
00332         w--; array++;
00333     }
00334 }
00335 else if ( *s == 's' ) {
00336     u = va_arg(ap, char *);
00337     while ( *u ) {
00338         if ( t >= stopper ) {
00339             num = t - Out;
00340             WriteString(ERROROUT, (UBYTE *)Out, num);
00341             t = Out;
00342         }
00343         *t++ = *u++;
00344     }
00345     *t = 0;
00346 }
00347 else if ( *s == 't' || *s == 'T' ) {
00348     WORD oldskip = AO.OutSkip, noleadsign;
00349     WORD oldmode = AC.OutputMode;

```

```

00350         WORD oldbracket = AO.IsBracket;
00351         WORD oldlength = AC.LineLength;
00352         UBYTE *oldStop = AO.OutStop;
00353         if ( AN.currentTerm ) {
00354             if ( AC.LineLength > MAXLINELENGTH ) AC.LineLength = MAXLINELENGTH;
00355             AO.IsBracket = 0;
00356             AO.OutSkip = 1;
00357             AC.OutputMode = 0;
00358             AO.OutFill = AO.OutputLine;
00359             AO.OutStop = AO.OutputLine + AC.LineLength;
00360             *t = 0;
00361             AddToLine( (UBYTE *)Out);
00362             if ( *s == 'T' ) noleadsign = 1;
00363             else noleadsign = 0;
00364             if ( WriteInnerTerm(AN.currentTerm,noleadsign) ) Terminate(-1);
00365             t = Out;
00366             u = (char *)AO.OutputLine;
00367             *(AO.OutFill) = 0;
00368             while ( u < (char *) (AO.OutFill) ) *t++ = *u++;
00369             *t = 0;
00370             AO.OutSkip = oldskip;
00371             AC.OutputMode = oldmode;
00372             AO.IsBracket = oldbracket;
00373             AC.LineLength = oldlength;
00374             AO.OutStop = oldStop;
00375         }
00376     }
00377     else if ( *s == 'r' ) {
00378         WORD oldskip = AO.OutSkip;
00379         WORD oldmode = AC.OutputMode;
00380         WORD oldbracket = AO.IsBracket;
00381         WORD oldlength = AC.LineLength;
00382         UBYTE *oldStop = AO.OutStop;
00383         if ( AN.currentTerm ) {
00384             WORD *tt = AN.currentTerm;
00385             if ( AC.LineLength > MAXLINELENGTH ) AC.LineLength = MAXLINELENGTH;
00386             AO.IsBracket = 0;
00387             AO.OutSkip = 1;
00388             AC.OutputMode = 0;
00389             AO.OutFill = AO.OutputLine;
00390             AO.OutStop = AO.OutputLine + AC.LineLength;
00391             *t = 0;
00392             i = *tt;
00393             while ( --i >= 0 ) {
00394                 t = (char *)NumCopy(*tt, (UBYTE *)t);
00395                 tt++;
00396                 if ( t >= stopper ) {
00397                     num = t - Out;
00398                     WriteString(ERROROUT, (UBYTE *)Out, num);
00399                     num = 0; t = Out;
00400                 }
00401                 *t++ = ' '; *t++ = ' ';
00402             }
00403             *t = 0;
00404             AO.OutSkip = oldskip;
00405             AC.OutputMode = oldmode;
00406             AO.IsBracket = oldbracket;
00407             AC.LineLength = oldlength;
00408             AO.OutStop = oldStop;
00409         }
00410     }
00411     else if ( *s == '$' ) {
00412         /*
00413         #[ dollars :
00414         */
00415         WORD oldskip = AO.OutSkip;
00416         WORD oldmode = AC.OutputMode;
00417         WORD oldbracket = AO.IsBracket;
00418         WORD oldlength = AC.LineLength;
00419         UBYTE *oldStop = AO.OutStop;
00420         WORD *term, indsubterm[3], *tt;
00421         WORD value[5], first, num;
00422         if ( *AN.listinprint != DOLLAREXPRESSION ) {
00423             specialerror = 1;
00424         }
00425         else {
00426             DOLLARS d = Dollars + AN.listinprint[1];
00427             #ifdef WITHPTHREADS
00428             int nummodopt, dtype;
00429             dtype = -1;
00430             if ( AS.MultiThreaded ) {
00431                 for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
00432                     if ( AN.listinprint[1] == ModOptdollars[nummodopt].number ) break;
00433                 }
00434                 if ( nummodopt < NumModOptdollars ) {
00435                     dtype = ModOptdollars[nummodopt].type;
00436                     if ( dtype == MODLOCAL ) {

```

```

00437             d = ModOptdollars[nummodopt].dstruct+AT.identity;
00438         }
00439         else {
00440             LOCK(d->pthreadlockread);
00441         }
00442     }
00443 }
00444 #endif
00445 AO.IsBracket = 0;
00446 AO.OutSkip = 0;
00447 AC.OutputMode = 0;
00448 AO.OutFill = AO.OutputLine;
00449 AO.OutStop = AO.OutputLine + AC.LineLength;
00450 *t = 0;
00451 AddToLine((UBYTE *)Out);
00452 if ( d->nfactors >= 1 && AN.listinprint[2] == DOLLAREXP2 ) {
00453     if ( d->type == 0 ||
00454         ( d->factors == 0 && d->nfactors != 1 ) ) goto dollarzero;
00455     num = EvalDoLoopArg(BHEAD AN.listinprint+2,-1);
00456     if ( num == 0 ) {
00457         value[0] = 4; value[1] = d->nfactors; value[2] = 1; value[3] = 3; value[4]
= 0;
00458         term = value; goto printterms;
00459     }
00460     if ( num == 1 && d->nfactors == 1 ) {
00461         term = d->where;
00462         if ( *term == 0 ) goto dollarzero;
00463         goto printterms;
00464     }
00465     if ( num > d->nfactors ) {
00466         MesPrint("\nFactor number for dollar is too large.");
00467         Terminate(-1);
00468     }
00469     term = d->factors[num-1].where;
00470     if ( term == 0 ) {
00471         if ( d->factors[num-1].value < 0 ) {
00472             value[0] = 4; value[1] = -d->factors[num-1].value; value[2] = 1;
value[3] = -3; value[4] = 0;
00473         }
00474         else {
00475             value[0] = 4; value[1] = d->factors[num-1].value; value[2] = 1;
value[3] = 3; value[4] = 0;
00476         }
00477         term = value;
00478     }
00479     goto printterms;
00480 }
00481 if ( d->type == DOLTERMS || d->type == DOLNUMBER ) {
00482     term = d->where;
00483     first = 1;
00484     do {
00485         if ( AC.LineLength > MAXLINELENGTH ) AC.LineLength = MAXLINELENGTH;
00486         AO.IsBracket = 0;
00487         AO.OutSkip = 1;
00488         AC.OutputMode = 0;
00489         AO.OutFill = AO.OutputLine;
00490         AO.OutStop = AO.OutputLine + AC.LineLength;
00491         *t = 0;
00492         AddToLine((UBYTE *)Out);
00493         if ( WriteInnerTerm(term,first) ) {
00494             #ifdef WITHPTHREADS
00495                 if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
00496             #endif
00497             Terminate(-1);
00498         }
00499         first = 0;
00500         t = Out;
00501         u = (char *)AO.OutputLine;
00502         *(AO.OutFill) = 0;
00503         while ( u < (char *) (AO.OutFill) ) *t++ = *u++;
00504         *t = 0;
00505         AO.OutSkip = oldskip;
00506         AC.OutputMode = oldmode;
00507         AO.IsBracket = oldbracket;
00508         AC.LineLength = oldlength;
00509         AO.OutStop = oldStop;
00510         term += *term;
00511     } while ( *term );
00512     AO.OutSkip = oldskip;
00513 }
00514 else if ( d->type == DOLSUBTERM ) {
00515     tt = d->where;
00516     dosubterm:
00517     if ( AC.LineLength > MAXLINELENGTH ) AC.LineLength = MAXLINELENGTH;
00518     AO.IsBracket = 0;
00519     AO.OutSkip = 1;
00520     AC.OutputMode = 0;
    AO.OutFill = AO.OutputLine;

```

```

00521         AO.OutStop = AO.OutputLine + AC.LineLength;
00522         *t = 0;
00523         AddToLine((UBYTE *)Out);
00524         if ( WriteSubTerm(tt,1) ) {
00525             #ifdef WITHPTHREADS
00526                 if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
00527             #endif
00528                 Terminate(-1);
00529         }
00530         t = Out;
00531         u = (char *)AO.OutputLine;
00532         *(AO.OutFill) = 0;
00533         while ( u < (char *) (AO.OutFill) ) *t++ = *u++;
00534         *t = 0;
00535         AO.OutSkip = oldskip;
00536         AC.OutputMode = oldmode;
00537         AO.IsBracket = oldbracket;
00538         AC.LineLength = oldlength;
00539         AO.OutStop = oldStop;
00540     }
00541     else if ( d->type == DOLUNDEFINED ) {
00542         *t++ = '*'; *t++ = '*'; *t++ = '*'; *t = 0;
00543     }
00544     else if ( d->type == DOLZERO ) {
00545         *t++ = '0'; *t = 0;
00546     }
00547     else if ( d->type == DOLINDEX ) {
00548         tt = indsubterm; *tt = INDEX;
00549         tt[1] = 3; tt[2] = d->index;
00550         goto dosubterm;
00551     }
00552     else if ( d->type == DOLARGUMENT ) {
00553         if ( AC.LineLength > MAXLINELENGTH ) AC.LineLength = MAXLINELENGTH;
00554         AO.IsBracket = 0;
00555         AO.OutSkip = 1;
00556         AC.OutputMode = 0;
00557         AO.OutFill = AO.OutputLine;
00558         AO.OutStop = AO.OutputLine + AC.LineLength;
00559         *t = 0;
00560         AddToLine((UBYTE *)Out);
00561         WriteArgument(d->where);
00562         t = Out;
00563         u = (char *)AO.OutputLine;
00564         *(AO.OutFill) = 0;
00565         while ( u < (char *) (AO.OutFill) ) *t++ = *u++;
00566         *t = 0;
00567         AO.OutSkip = oldskip;
00568         AC.OutputMode = oldmode;
00569         AO.IsBracket = oldbracket;
00570         AC.LineLength = oldlength;
00571         AO.OutStop = oldStop;
00572     }
00573     else if ( d->type == DOLWILDARGS ) {
00574         tt = d->where;
00575         if ( *tt == 0 ) { tt++;
00576             while ( *tt ) {
00577                 if ( AC.LineLength > MAXLINELENGTH ) AC.LineLength = MAXLINELENGTH;
00578                 AO.IsBracket = 0;
00579                 AO.OutSkip = 1;
00580                 AC.OutputMode = 0;
00581                 AO.OutFill = AO.OutputLine;
00582                 AO.OutStop = AO.OutputLine + AC.LineLength;
00583                 *t = 0;
00584                 AddToLine((UBYTE *)Out);
00585                 WriteArgument(tt);
00586                 NEXTARG(tt);
00587                 if ( *tt ) TokenToLine((UBYTE *)",");
00588                 t = Out;
00589                 u = (char *)AO.OutputLine;
00590                 *(AO.OutFill) = 0;
00591                 while ( u < (char *) (AO.OutFill) ) *t++ = *u++;
00592                 *t = 0;
00593                 AO.OutSkip = oldskip;
00594                 AC.OutputMode = oldmode;
00595                 AO.IsBracket = oldbracket;
00596                 AC.LineLength = oldlength;
00597                 AO.OutStop = oldStop;
00598             }
00599         }
00600     }
00601     else if ( *tt > 0 ) { /* Tensor arguments */
00602         i = *tt++;
00603         while ( --i >= 0 ) {
00604             indsubterm[0] = INDEX;
00605             indsubterm[1] = 3;
00606             indsubterm[2] = *tt++;
00607             if ( AC.LineLength > MAXLINELENGTH ) AC.LineLength = MAXLINELENGTH;
00608             AO.IsBracket = 0;

```

```

00608             AO.OutSkip = 1;
00609             AC.OutputMode = 0;
00610             AO.OutFill = AO.OutputLine;
00611             AO.OutStop = AO.OutputLine + AC.LineLength;
00612             *t = 0;
00613             AddToLine((UBYTE *)Out);
00614             if ( WriteSubTerm(indsubterm,1) ) Terminate(-1);
00615             if ( i > 0 ) TokenToLine((UBYTE *)",");
00616             t = Out;
00617             u = (char *)AO.OutputLine;
00618             *(AO.OutFill) = 0;
00619             while ( u < (char *) (AO.OutFill) ) *t++ = *u++;
00620             *t = 0;
00621             AO.OutSkip = oldskip;
00622             AC.OutputMode = oldmode;
00623             AO.IsBracket = oldbracket;
00624             AC.LineLength = oldlength;
00625             AO.OutStop = oldStop;
00626         }
00627     }
00628 }
00629 #ifdef WITHPTHREADS
00630     if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
00631 #endif
00632     AN.listinprint += 2;
00633     while ( AN.listinprint[0] == DOLLAREXPR2 ) AN.listinprint += 2;
00634 }
00635 /*
00636     #] dollars :
00637 */
00638 }
00639 #ifdef WITHPTHREADS
00640     else if ( *s == 'W' ) { /* number of the thread with time */
00641         LONG millitime;
00642         WORD timepart;
00643         t = (char *)NumCopy(identity, (UBYTE *)t);
00644         millitime = TimeCPU(1);
00645         timepart = (WORD)(millitime%1000);
00646         millitime /= 1000;
00647         timepart /= 10;
00648         *t++ = '('; *t = 0;
00649         t = (char *)LongCopy(millitime, (char *)t);
00650         *t++ = '.'; *t = 0;
00651         t = (char *)NumCopy(timepart, (UBYTE *)t);
00652         *t++ = ')'; *t = 0;
00653         if ( t >= stopper ) {
00654             num = t - Out;
00655             WriteString(ERROROUT, (UBYTE *)Out, num);
00656             num = 0; t = Out;
00657         }
00658     }
00659     else if ( *s == 'w' ) { /* number of the thread */
00660         t = (char *)NumCopy(identity, (UBYTE *)t);
00661     }
00662 #elif defined(WITHMPI)
00663     else if ( *s == 'W' ) { /* number of the thread with time */
00664         LONG millitime;
00665         WORD timepart;
00666         t = (char *)NumCopy(PF.me, (UBYTE *)t);
00667         millitime = TimeCPU(1);
00668         timepart = (WORD)(millitime%1000);
00669         millitime /= 1000;
00670         timepart /= 10;
00671         *t++ = '('; *t = 0;
00672         t = (char *)LongCopy(millitime, (char *)t);
00673         *t++ = '.'; *t = 0;
00674         t = (char *)NumCopy(timepart, (UBYTE *)t);
00675         *t++ = ')'; *t = 0;
00676         if ( t >= stopper ) {
00677             num = t - Out;
00678             WriteString(ERROROUT, (UBYTE *)Out, num);
00679             num = 0; t = Out;
00680         }
00681     }
00682     else if ( *s == 'w' ) { /* number of the thread */
00683         t = (char *)NumCopy(PF.me, (UBYTE *)t);
00684     }
00685 #else
00686     else if ( *s == 'w' ) { }
00687     else if ( *s == 'W' ) { }
00688 #endif
00689     else if ( FG.cTable[(int)*s] == 1 ) {
00690         x = *s++ - '0';
00691         while ( FG.cTable[(int)*s] == 1 )
00692             x = 10 * x + *s++ - '0';
00693     }
00694     if ( *s == 'l' || *s == 'd' ) {

```

```

00695         if ( *s == 'l' ) { y = va_arg(ap, LONG); }
00696     else { y = va_arg(ap, int); }
00697     if ( y < 0 ) { y = -y; w = 1; }
00698     else w = 0;
00699     u = t + x;
00700     do { *--u = y%10+'0'; y /= 10; } while ( y && u > t );
00701     if ( w && u > t ) *--u = '-';
00702     while ( --u >= t ) *u = ' ';
00703     t += x;
00704 }
00705 else if ( *s == 's' ) {
00706     u = va_arg(ap, char *);
00707     i = 0;
00708     while ( *u ) { i++; u++; }
00709     if ( i > x ) i = x;
00710     while ( x > i ) { *t++ = ' '; x--; }
00711     t += x;
00712     while ( --i >= 0 ) { *--t = *--u; }
00713     t += x;
00714 }
00715 else if ( *s == 'p' ) {
00716     POSITION *pp;
00717     /*#ifndef __GLIBC_HAVE_LONG_LONG */
00718     off_t ly;
00719     /*
00720     #else
00721         LONG ly;
00722     #endif
00723     */
00724     pp = va_arg(ap, POSITION *);
00725     ly = BASEPOSITION(*pp);
00726     u = t + x;
00727     do { *--u = ly%10+'0'; ly /= 10; } while ( ly && u > t );
00728     while ( --u >= t ) *u = ' ';
00729     t += x;
00730 }
00731 else if ( *s == 'i' ) {
00732     w = va_arg(ap, int);
00733     u = t + x;
00734     do { *--u = (char)(w%10+'0'); w /= 10; } while ( u > t );
00735     t += x;
00736 }
00737 else {
00738     w = va_arg(ap, int);
00739     u = t + x;
00740     do { *--u = (char)(w%10+'0'); w /= 10; } while ( w && u > t );
00741     while ( --u >= t ) *u = ' ';
00742     t += x;
00743 }
00744 }
00745 else if ( *s == 'x' ) {
00746     char ccc;
00747     y = va_arg(ap, LONG);
00748     i = 2*sizeof(LONG);
00749     while ( --i > 0 ) {
00750         ccc = ( y >> (i*4) ) & 0xF;
00751         if ( ccc ) break;
00752     }
00753     do {
00754         ccc = ( y >> (i*4) ) & 0xF;
00755         *t++ = hex[(int)ccc];
00756     } while ( --i >= 0 );
00757 }
00758 else if ( *s == '#' ) *t++ = *s;
00759 else if ( *s == '%' ) *t++ = *s;
00760 else if ( *s == 0 ) { *t++ = 0; break; }
00761 else if ( *s == '&' ) {
00762     *t++ = *s;
00763 }
00764 else {
00765     *t++ = '%';
00766     s--;
00767 }
00768 s++;
00769 }
00770 }
00771 num = t - Out;
00772 WriteString(ERROROUT, (UBYTE *)Out, num);
00773 va_end(ap);
00774 if ( specialerror == 1 ) {
00775     MesPrint("!!!Wrong object in Print statement!!!");
00776     MesPrint("!!!Object encountered is of a different type as in the format specifier");
00777 }
00778 AO.OutputLine = oldoutfill;
00779 /*[19apr2004 mt]:*/
00780 WriteFile=OldWrite;
00781 /*:[19apr2004 mt]*/

```



```

00782     return(-1);
00783 }
00784
00785 /*
00786     #] MesPrint :
00787     #[ Warning :
00788 */
00789
00790 void Warning(char *s)
00791 {
00792     iswarning = 1;
00793     if ( AC.WarnFlag ) MesPrint("&Warning: %s",s);
00794     iswarning = 0;
00795 }
00796
00797 /*
00798     #] Warning :
00799     #[ HighWarning :
00800 */
00801
00802 void HighWarning(char *s)
00803 {
00804     iswarning = 1;
00805     if ( AC.WarnFlag >= 2 ) MesPrint("&Warning: %s",s);
00806     iswarning = 0;
00807 }
00808
00809 /*
00810     #] HighWarning :
00811     #[ MesCall :
00812 */
00813
00814 int MesCall(char *s)
00815 {
00816     return(MesPrint((char *)"Called from %s",s));
00817 }
00818
00819 /*
00820     #] MesCall :
00821     #[ MesCerr :
00822 */
00823
00824 WORD MesCerr(char *s, UBYTE *t)
00825 {
00826     UBYTE *u, c;
00827     WORD i = 11;
00828     u = t;
00829     while ( *u && --i >= 0 ) u--;
00830     u++;
00831     c = **++t;
00832     *t = 0;
00833     MesPrint("&Illegal %s: %s",s,u);
00834     *t = c;
00835     return(-1);
00836 }
00837
00838 /*
00839     #] MesCerr :
00840     #[ MesComp :
00841 */
00842
00843 WORD MesComp(char *s, UBYTE *p, UBYTE *q)
00844 {
00845     UBYTE c;
00846     c = **++q; *q = 0;
00847     MesPrint("&%s: %s",s,p);
00848     *q = c;
00849     return(-1);
00850 }
00851
00852 /*
00853     #] MesComp :
00854     #[ PrintTerm :
00855 */
00856
00857 void PrintTerm(WORD *term, char *where)
00858 {
00859     UBYTE OutBuf[140];
00860     WORD *t, x;
00861     int i;
00862     AO.OutFill = AO.OutputLine = OutBuf;
00863     t = term;
00864     AO.OutSkip = 3;
00865     FiniLine();
00866     TokenToLine((UBYTE *)where);
00867     TokenToLine((UBYTE *)": ");
00868     i = *t;

```

```

00869     while ( --i >= 0 ) {
00870         x = *t++;
00871         if ( x < 0 ) {
00872             x = -x;
00873             TokenToLine((UBYTE *)"-");
00874         }
00875         TalToLine((UWORD)(x));
00876         TokenToLine((UBYTE *)" ");
00877     }
00878     AO.OutSkip = 0;
00879     FiniLine();
00880 }
00881
00882 /*
00883     #] PrintTerm :
00884     #[ PrintTermC :
00885 */
00886
00887 void PrintTermC(WORD *term, char *where)
00888 {
00889     UBYTE OutBuf[140];
00890     WORD *t, x;
00891     int i;
00892     if ( *term >= 0 ) {
00893         PrintTerm(term,where);
00894         return;
00895     }
00896     AO.OutFill = AO.OutputLine = OutBuf;
00897     t = term;
00898     AO.OutSkip = 3;
00899     FiniLine();
00900     TokenToLine((UBYTE *)where);
00901     TokenToLine((UBYTE *)": ");
00902     i = t[1]+2;
00903     while ( --i >= 0 ) {
00904         x = *t++;
00905         if ( x < 0 ) {
00906             x = -x;
00907             TokenToLine((UBYTE *)"-");
00908         }
00909         TalToLine((UWORD)(x));
00910         TokenToLine((UBYTE *)" ");
00911     }
00912     AO.OutSkip = 0;
00913     FiniLine();
00914 }
00915
00916 /*
00917     #] PrintTermC :
00918     #[ PrintSubTerm :
00919 */
00920
00921 void PrintSubTerm(WORD *term, char *where)
00922 {
00923     UBYTE OutBuf[140];
00924     WORD *t;
00925     int i;
00926     AO.OutFill = AO.OutputLine = OutBuf;
00927     t = term;
00928     AO.OutSkip = 3;
00929     FiniLine();
00930     TokenToLine((UBYTE *)where);
00931     TokenToLine((UBYTE *)": ");
00932     i = t[1];
00933     while ( --i >= 0 ) { TalToLine((UWORD)(*t++)); TokenToLine((UBYTE *)" "); }
00934     AO.OutSkip = 0;
00935     FiniLine();
00936 }
00937
00938 /*
00939     #] PrintSubTerm :
00940     #[ PrintWords :
00941 */
00942
00943 void PrintWords(WORD *buffer, LONG number)
00944 {
00945     UBYTE OutBuf[140];
00946     WORD *t;
00947     AO.OutFill = AO.OutputLine = OutBuf;
00948     t = buffer;
00949     AO.OutSkip = 3;
00950     FiniLine();
00951     while ( --number >= 0 ) { TalToLine((UWORD)(*t++)); TokenToLine((UBYTE *)" "); }
00952     AO.OutSkip = 0;
00953     FiniLine();
00954 }
00955

```

```

00956 /*
00957     #] PrintWords :
00958     #[ PrintSeq :
00959 */
00960
00961 void PrintSeq(WORD *a,char *text)
00962 {
00963     MesPrint(" %s:",text);
00964     while ( *a ) {
00965         MesPrint("      %a",a[0],a);
00966         a += *a;
00967     }
00968 }
00969
00970 /*
00971     #] PrintSeq :
00972     #] exit :
00973 */

```

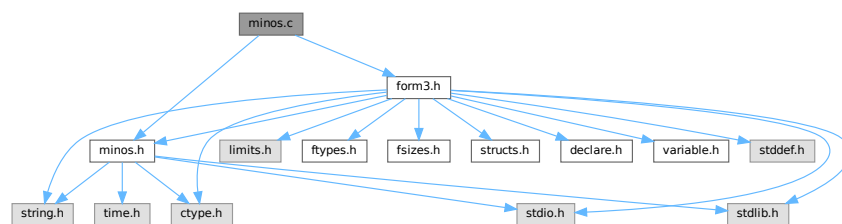
4.71 minos.c File Reference

```

#include "form3.h"
#include "minos.h"

```

Include dependency graph for minos.c:



Macros

- `#define` [CFD](#)(y, s, type, x, j)
- `#define` [CTD](#)(y, s, type, x, j)

Functions

- `int` [minosread](#) (FILE *f, char *buffer, MLONG size)
- `int` [minoswrite](#) (FILE *f, char *buffer, MLONG size)
- `char *` [str_dup](#) (char *str)
- `void` [convertblock](#) (INDEXBLOCK *in, INDEXBLOCK *out, int mode)
- `void` [convertnamesblock](#) (NAMESBLOCK *in, NAMESBLOCK *out, int mode)
- `void` [convertiniinfo](#) (INIINFO *in, INIINFO *out, int mode)
- `FILE *` [LocateBase](#) (char **name, char **newname, char *iomode)
- `int` [ReadIndex](#) (DBASE *d)
- `int` [WriteIndexBlock](#) (DBASE *d, MLONG num)
- `int` [WriteNamesBlock](#) (DBASE *d, MLONG num)
- `int` [WriteIndex](#) (DBASE *d)
- `int` [WriteIniInfo](#) (DBASE *d)
- `int` [ReadIniInfo](#) (DBASE *d)
- `DBASE *` [GetDbase](#) (char *filename, MLONG rmode)

- `DBASE * NewDbase` (`char *name`, `MLONG number`)
- `void FreeTableBase` (`DBASE *d`)
- `int ComposeTableNames` (`DBASE *d`)
- `DBASE * OpenDbase` (`char *filename`)
- `MLONG AddTableName` (`DBASE *d`, `char *name`, `TABLES T`)
- `MLONG GetTableName` (`DBASE *d`, `char *name`)
- `int PutTableNames` (`DBASE *d`)
- `int AddToIndex` (`DBASE *d`, `MLONG number`)
- `MLONG AddObject` (`DBASE *d`, `MLONG tablename`, `char *arguments`, `char *rhs`)
- `MLONG FindTableNumber` (`DBASE *d`, `char *name`)
- `int WriteObject` (`DBASE *d`, `MLONG tablename`, `char *arguments`, `char *rhs`, `MLONG number`)
- `char * ReadObject` (`DBASE *d`, `MLONG tablename`, `char *arguments`)
- `char * ReadijObject` (`DBASE *d`, `MLONG i`, `MLONG j`, `char *arguments`)
- `int ExistsObject` (`DBASE *d`, `MLONG tablename`, `char *arguments`)
- `int DeleteObject` (`DBASE *d`, `MLONG tablename`, `char *arguments`)

Variables

- `int withoutflush = 0`

4.71.1 Detailed Description

These are the low level functions for the database part of the tablebases. These routines have been copied (and then adapted) from the minos database program. This file goes together with [minos.h](#)

Definition in file [minos.c](#).

4.71.2 Macro Definition Documentation

4.71.2.1 CFD

```
#define CFD(
    y,
    s,
    type,
    x,
    j )
```

Value:

```
for(x=0, j=0; j<((int) sizeof(type)); j++) \
{ x=(x<<8)+((s++)&0xFF); } y=x;
```

Definition at line 55 of file [minos.c](#).

4.71.2.2 CTD

```
#define CTD(
    y,
    s,
    type,
    x,
    j )
```

Value:

```
x=y; for(j=sizeof(type)-1; j>=0; j--) { s[j]=x&0xFF; \
x>>=8; } s += sizeof(type);
```

Definition at line 57 of file [minos.c](#).

4.71.3 Function Documentation

4.71.3.1 minosread()

```
int minosread (
    FILE * f,
    char * buffer,
    MLONG size )
```

Definition at line 66 of file [minos.c](#).

4.71.3.2 minoswrite()

```
int minoswrite (
    FILE * f,
    char * buffer,
    MLONG size )
```

Definition at line 83 of file [minos.c](#).

4.71.3.3 str_dup()

```
char * str_dup (
    char * str )
```

Definition at line 101 of file [minos.c](#).

4.71.3.4 convertblock()

```
void convertblock (
    INDEXBLOCK * in,
    INDEXBLOCK * out,
    int mode )
```

Definition at line 120 of file [minos.c](#).

4.71.3.5 convertnamesblock()

```
void convertnamesblock (
    NAMESBLOCK * in,
    NAMESBLOCK * out,
    int mode )
```

Definition at line 171 of file [minos.c](#).

4.71.3.6 convertiniinfo()

```
void convertiniinfo (
    INIINFO * in,
    INIINFO * out,
    int mode )
```

Definition at line 197 of file [minos.c](#).

4.71.3.7 LocateBase()

```
FILE * LocateBase (
    char ** name,
    char ** newname,
    char * iomode )
```

Definition at line 225 of file [minos.c](#).

4.71.3.8 ReadIndex()

```
int ReadIndex (
    DBASE * d )
```

Definition at line 288 of file [minos.c](#).

4.71.3.9 WriteIndexBlock()

```
int WriteIndexBlock (
    DBASE * d,
    MLONG num )
```

Definition at line 401 of file [minos.c](#).

4.71.3.10 WriteNamesBlock()

```
int WriteNamesBlock (
    DBASE * d,
    MLONG num )
```

Definition at line 422 of file [minos.c](#).

4.71.3.11 WriteIndex()

```
int WriteIndex (
    DBASE * d )
```

Definition at line 445 of file [minos.c](#).

4.71.3.12 WriteIniInfo()

```
int WriteIniInfo (
    DBASE * d )
```

Definition at line 508 of file [minos.c](#).

4.71.3.13 ReadIniInfo()

```
int ReadIniInfo (
    DBASE * d )
```

Definition at line 525 of file [minos.c](#).

4.71.3.14 GetDbase()

```
DBASE * GetDbase (
    char * filename,
    MLONG rwmode )
```

Definition at line 548 of file [minos.c](#).

4.71.3.15 NewDbase()

```
DBASE * NewDbase (
    char * name,
    MLONG number )
```

Definition at line 604 of file [minos.c](#).

4.71.3.16 FreeTableBase()

```
void FreeTableBase (
    DBASE * d )
```

Definition at line 741 of file [minos.c](#).

4.71.3.17 ComposeTableNames()

```
int ComposeTableNames (
    DBASE * d )
```

Definition at line 773 of file [minos.c](#).

4.71.3.18 OpenDbase()

```
DBASE * OpenDbase (
    char * filename )
```

Definition at line 822 of file [minos.c](#).

4.71.3.19 AddTableName()

```
MLONG AddTableName (
    DBASE * d,
    char * name,
    TABLES T )
```

Definition at line 856 of file [minos.c](#).

4.71.3.20 GetTableName()

```
MLONG GetTableName (
    DBASE * d,
    char * name )
```

Definition at line 939 of file [minos.c](#).

4.71.3.21 PutTableNames()

```
int PutTableNames (
    DBASE * d )
```

Definition at line 971 of file [minos.c](#).

4.71.3.22 AddToIndex()

```
int AddToIndex (
    DBASE * d,
    MLONG number )
```

Definition at line 1067 of file [minos.c](#).

4.71.3.23 AddObject()

```
MLONG AddObject (
    DBASE * d,
    MLONG tablename,
    char * arguments,
    char * rhs )
```

Definition at line 1154 of file [minos.c](#).

4.71.3.24 FindTableNumber()

```
MLONG FindTableNumber (
    DBASE * d,
    char * name )
```

Definition at line 1168 of file [minos.c](#).

4.71.3.25 WriteObject()

```
int WriteObject (
    DBASE * d,
    MLONG tablenumber,
    char * arguments,
    char * rhs,
    MLONG number )
```

Definition at line 1196 of file [minos.c](#).

4.71.3.26 ReadObject()

```
char * ReadObject (
    DBASE * d,
    MLONG tablenumber,
    char * arguments )
```

Definition at line 1296 of file [minos.c](#).

4.71.3.27 ReadijObject()

```
char * ReadijObject (
    DBASE * d,
    MLONG i,
    MLONG j,
    char * arguments )
```

Definition at line 1377 of file [minos.c](#).

4.71.3.28 ExistsObject()

```
int ExistsObject (
    DBASE * d,
    MLONG tablenumber,
    char * arguments )
```

Definition at line 1429 of file [minos.c](#).

4.71.3.29 DeleteObject()

```
int DeleteObject (
    DBASE * d,
    MLONG tablenumber,
    char * arguments )
```

Definition at line 1461 of file [minos.c](#).

4.71.4 Variable Documentation

4.71.4.1 withoutflush

```
int withoutflush = 0
```

Definition at line 45 of file [minos.c](#).

4.72 minos.c

[Go to the documentation of this file.](#)

```
00001
00007 /* #[ License : */
00008 /*
00009 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00010 *   When using this file you are requested to refer to the publication
00011 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00012 *   This is considered a matter of courtesy as the development was paid
00013 *   for by FOM the Dutch physics granting agency and we would like to
00014 *   be able to track its scientific use to convince FOM of its value
00015 *   for the community.
00016 *
00017 *   This file is part of FORM.
00018 *
00019 *   FORM is free software: you can redistribute it and/or modify it under the
00020 *   terms of the GNU General Public License as published by the Free Software
00021 *   Foundation, either version 3 of the License, or (at your option) any later
00022 *   version.
00023 *
00024 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00025 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00026 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00027 *   details.
00028 *
00029 *   You should have received a copy of the GNU General Public License along
00030 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00031 */
00032 /* #[ License : */
00033 /*
00034 *   #[ Includes :
00035 *
00036 *   File contains the lowlevel routines for the management of primitive
00037 *   database-like files. The structures are explained in the file manage.h
00038 *   Original file for minos made by J.Vermaseren, april-1994.
00039 *
00040 *   Note: the minos primitives for writing are never invoked in parallel.
00041 */
00042
00043 #include "form3.h"
00044 #include "minos.h"
00045 int withoutflush = 0;
00046
00047 /*
00048 *   #[ Includes :
00049 *   #[ Variables :
00050 */
00051
00052 static INDEXBLOCK scratchblock;
00053 static NAMESBLOCK scratchnamesblock;
00054
00055 #define CFD(y,s,type,x,j) for(x=0,j=0;j<((int)sizeof(type));j++) \
00056     {x=(x<<8)+((*s++)&0x00FF);} y=x;
00057 #define CTD(y,s,type,x,j) x=y;for(j=sizeof(type)-1;j>=0;j--){s[j]=x&0xFF; \
00058     x>>8;} s += sizeof(type);
00059
00060 /*
00061 *   #[ Variables :
00062 *   #[ Utilities :
00063 *   #[ minosread :
00064 */
00065
00066 int minosread(FILE *f,char *buffer,MLONG size)
00067 {
00068     MLONG x;
00069     while ( size > 0 ) {
00070         x = fread(buffer,sizeof(char),size,f);
00071         if ( x <= 0 ) return(-1);
```

```

00072         buffer += x;
00073         size -= x;
00074     }
00075     return(0);
00076 }
00077
00078 /*
00079     #] minosread :
00080     #[ minoswrite :
00081 */
00082
00083 int minoswrite(FILE *f, char *buffer, MLONG size)
00084 {
00085     MLONG x;
00086     while ( size > 0 ) {
00087         x = fwrite(buffer, sizeof(char), size, f);
00088         if ( x <= 0 ) return(-1);
00089         buffer += x;
00090         size -= x;
00091     }
00092     if ( withoutflush == 0 ) fflush(f);
00093     return(0);
00094 }
00095
00096 /*
00097     #] minoswrite :
00098     #[ str_dup :
00099 */
00100 char *str_dup(char *str)
00101 {
00102     char *s, *t;
00103     int i;
00104     s = str;
00105     while ( *s ) s++;
00106     i = s - str + 1;
00107     if ( ( s = (char *)Malloc1((size_t)i, "a string copy") ) == 0 ) return(0);
00108     t = s;
00109     while ( *str ) *t++ = *str++;
00110     *t = 0;
00111     return(s);
00112 }
00113
00114 /*
00115     #] str_dup :
00116     #[ convertblock :
00117 */
00118 void convertblock(INDEXBLOCK *in, INDEXBLOCK *out, int mode)
00119 {
00120     char *s, *t;
00121     MLONG i, x;
00122     int j;
00123     OBJECTS *obj;
00124     switch ( mode ) {
00125     case TODISK:
00126         s = (char *)out;
00127         CTD(in->flags, s, MLONG, x, j)
00128         CTD(in->previousblock, s, MLONG, x, j)
00129         CTD(in->position, s, MLONG, x, j)
00130         for ( i = 0, obj = in->objects; i < NUMOBJECTS; i++, obj++ ) {
00131             CTD(obj->position, s, MLONG, x, j)
00132             CTD(obj->size, s, MLONG, x, j)
00133             CTD(obj->date, s, MLONG, x, j)
00134             CTD(obj->tablename, s, MLONG, x, j)
00135             CTD(obj->uncompressed, s, MLONG, x, j)
00136             CTD(obj->spare1, s, MLONG, x, j)
00137             CTD(obj->spare2, s, MLONG, x, j)
00138             CTD(obj->spare3, s, MLONG, x, j)
00139             t = obj->element;
00140             for ( j = 0; j < ELEMENTSIZE; j++ ) *s++ = *t++;
00141         }
00142         break;
00143     case FROMDISK:
00144         s = (char *)in;
00145         CFD(out->flags, s, MLONG, x, j)
00146         CFD(out->previousblock, s, MLONG, x, j)
00147         CFD(out->position, s, MLONG, x, j)
00148         for ( i = 0, obj = out->objects; i < NUMOBJECTS; i++, obj++ ) {
00149             CFD(obj->position, s, MLONG, x, j)
00150             CFD(obj->size, s, MLONG, x, j)
00151             CFD(obj->date, s, MLONG, x, j)
00152             CFD(obj->tablename, s, MLONG, x, j)
00153             CFD(obj->uncompressed, s, MLONG, x, j)
00154             CFD(obj->spare1, s, MLONG, x, j)
00155             CFD(obj->spare2, s, MLONG, x, j)
00156             CFD(obj->spare3, s, MLONG, x, j)
00157         }
00158     }

```

```

00159         t = obj->element;
00160         for ( j = 0; j < ELEMENTSIZE; j++ ) *t++ = *s++;
00161     }
00162     break;
00163 }
00164 }
00165
00166 /*
00167     #] convertblock :
00168     #[ convertnamesblock :
00169 */
00170
00171 void convertnamesblock(NAMESBLOCK *in,NAMESBLOCK *out,int mode)
00172 {
00173     char *s;
00174     MLONG x;
00175     int j;
00176     switch ( mode ) {
00177     case TODISK:
00178         s = (char *)out;
00179         CTD(in->previousblock,s,MLONG,x,j)
00180         CTD(in->position,s,MLONG,x,j)
00181         for ( j = 0; j < NAMETABLESIZE; j++ ) out->names[j] = in->names[j];
00182         break;
00183     case FROMDISK:
00184         s = (char *)in;
00185         CFD(out->previousblock,s,MLONG,x,j)
00186         CFD(out->position,s,MLONG,x,j)
00187         for ( j = 0; j < NAMETABLESIZE; j++ ) out->names[j] = in->names[j];
00188         break;
00189     }
00190 }
00191
00192 /*
00193     #] convertnamesblock :
00194     #[ convertiniinfo :
00195 */
00196
00197 void convertiniinfo(INIINFO *in,INIINFO *out,int mode)
00198 {
00199     char *s;
00200     MLONG i, x, *y;
00201     int j;
00202     switch ( mode ) {
00203     case TODISK:
00204         s = (char *)out; y = (MLONG *)in;
00205         for ( i = sizeof(INIINFO)/sizeof(MLONG); i > 0; i-- ) {
00206             CTD(*y,s,MLONG,x,j)
00207             y++;
00208         }
00209         break;
00210     case FROMDISK:
00211         s = (char *)in; y = (MLONG *)out;
00212         for ( i = sizeof(INIINFO)/sizeof(MLONG); i > 0; i-- ) {
00213             CFD(*y,s,MLONG,x,j)
00214             y++;
00215         }
00216         break;
00217     }
00218 }
00219
00220 /*
00221     #] convertiniinfo :
00222     #[ LocateBase :
00223 */
00224
00225 FILE *LocateBase(char **name, char **newname, char *iomode)
00226 {
00227     FILE *handle;
00228     int namesize, i;
00229     UBYTE *s, *to, *u1, *u2, *indir;
00230
00231     if ( ( handle = fopen(*name,iomode) ) != 0 ) {
00232         *newname = (char *)strDup1((UBYTE *)(*name),"LocateBase");
00233         return(handle);
00234     }
00235     namesize = 2; s = (UBYTE *)(*name);
00236     while ( *s ) { s++; namesize++; }
00237     indir = AM.IncDir;
00238     if ( indir ) {
00239         s = indir; i = 0;
00240         while ( *s ) { s++; i++; }
00241         *newname = (char *)Malloc1(namesize+i,"LocateBase");
00242         s = indir; to = (UBYTE *)(*newname);
00243         while ( *s ) *to++ = *s++;
00244         if ( to > (UBYTE *)(*newname) && to[-1] != SEPARATOR ) *to++ = SEPARATOR;
00245         s = (UBYTE *)(*name);

```

```

00246         while ( *s ) *to++ = *s++;
00247         *to = 0;
00248         if ( ( handle = fopen(*newname,iomode) ) != 0 ) {
00249             return(handle);
00250         }
00251         M_free(*newname,"LocateBase, incdir/file");
00252     }
00253     if ( AM.Path ) {
00254         ul = AM.Path;
00255         while ( *ul ) {
00256             u2 = ul; i = 0;
00257             while ( *ul && *ul != ':' ) {
00258                 if ( *ul == '\\\\' ) ul++;
00259                 ul++; i++;
00260             }
00261             *newname = (char *)Malloca1(namesize+i,"LocateBase");
00262             s = u2; to = (UBYTE *)(*newname);
00263             while ( s < ul ) {
00264                 if ( *s == '\\\\' ) s++;
00265                 *to++ = *s++;
00266             }
00267             if ( to > (UBYTE *)(*newname) && to[-1] != SEPARATOR ) *to++ = SEPARATOR;
00268             s = (UBYTE *)(*name);
00269             while ( *s ) *to++ = *s++;
00270             *to = 0;
00271             if ( ( handle = fopen(*newname,iomode) ) != 0 ) {
00272                 return(handle);
00273             }
00274             M_free(*newname,"LocateBase Path/file");
00275             if ( *ul ) ul++;
00276         }
00277     }
00278     /* Error1("LocateBase: Cannot find file",*name); */
00279     return(0);
00280 }
00281
00282 /*
00283     #] LocateBase :
00284     #] Utilities :
00285     #[ ReadIndex :
00286 */
00287
00288 int ReadIndex(DBASE *d)
00289 {
00290     MLONG i;
00291     INDEXBLOCK **ib;
00292     NAMESBLOCK **ina;
00293     MLONG position, size;
00294     /*
00295         Allocate the pieces one by one (makes it easier to give it back)
00296     */
00297     if ( d->info.numberofindexblocks <= 0 ) return(0);
00298     #ifndef WORDSIZE32
00299     if ( sizeof(INDEXBLOCK)*d->info.numberofindexblocks > MAXINDEXSIZE ) {
00300         MesPrint("We need more than %ld bytes for the index.\n",MAXINDEXSIZE);
00301         MesPrint("The file %s may not be a proper database\n",d->name);
00302         return(-1);
00303     }
00304     #endif
00305     size = sizeof(INDEXBLOCK)*d->info.numberofindexblocks;
00306     if ( ( ib = (INDEXBLOCK **)Malloca1(size,"tb,index") ) == 0 ) return(-1);
00307     for ( i = 0; i < d->info.numberofindexblocks; i++ ) {
00308         if ( ( ib[i] = (INDEXBLOCK *)Malloca1(sizeof(INDEXBLOCK),"index block") ) == 0 ) {
00309             for ( --i; i >= 0; i-- ) M_free(ib[i],"tb,indexblock");
00310             M_free(ib,"tb,index");
00311             return(-1);
00312         }
00313     }
00314     size = sizeof(NAMESBLOCK)*d->info.numberofnamesblocks;
00315     if ( ( ina = (NAMESBLOCK **)Malloca1(size,"tb,indexnames") ) == 0 ) return(-1);
00316     for ( i = 0; i < d->info.numberofnamesblocks; i++ ) {
00317         if ( ( ina[i] = (NAMESBLOCK *)Malloca1(sizeof(NAMESBLOCK),"index names block") ) == 0 ) {
00318             for ( --i; i >= 0; i-- ) M_free(ina[i],"index names block");
00319             M_free(ina,"tb,indexnames");
00320             for ( i = 0; i < d->info.numberofindexblocks; i++ ) M_free(ib[i],"tb,indexblock");
00321             M_free(ib,"tb,index");
00322             return(-1);
00323         }
00324     }
00325     /*
00326         Read the index blocks, from the back to the front. The links are only
00327         reliable that way.
00328     */
00329     position = d->info.lastindexblock;
00330     for ( i = d->info.numberofindexblocks - 1; i >= 0; i-- ) {
00331         fseek(d->handle,position,SEEK_SET);
00332         if ( minosread(d->handle,(char *)(&scratchblock),sizeof(INDEXBLOCK)) ) {

```

```

00333         MesPrint("Error while reading file %s\n",d->name);
00334 thisiswrong:
00335         for ( i = 0; i < d->info.numberofnamesblocks; i++ ) M_free(ina[i],"index names block");
00336         M_free(ina,"tb,indexnames");
00337         for ( i = 0; i < d->info.numberofindexblocks; i++ ) M_free(ib[i],"tb,indexblock");
00338         M_free(ib,"tb,index");
00339         return(-1);
00340     }
00341     convertblock(&scratchblock,ib[i],FROMDISK);
00342     if ( ib[i]->position != position ||
00343         ( ib[i]->previousblock <= 0 && i > 0 ) ) {
00344         MesPrint("File %s has inconsistent contents\n",d->name);
00345         goto thisiswrong;
00346     }
00347     position = ib[i]->previousblock;
00348 }
00349 d->info.firstindexblock = ib[0]->position;
00350 for ( i = 0; i < d->info.numberofindexblocks; i++ ) {
00351     ib[i]->flags &= MCLEANFLAG;
00352 }
00353 /*
00354 Read the names blocks, from the back to the front. The links are only
00355 reliable that way.
00356 */
00357 position = d->info.lastnameblock;
00358 for ( i = d->info.numberofnamesblocks - 1; i >= 0; i-- ) {
00359     fseek(d->handle,position,SEEK_SET);
00360     if ( minosread(d->handle,(char *)(&scratchnamesblock),sizeof(NAMESBLOCK)) ) {
00361         MesPrint("Error while reading file %s\n",d->name);
00362         goto thisiswrong;
00363     }
00364     convertnamesblock(&scratchnamesblock,ina[i],FROMDISK);
00365     if ( ina[i]->position != position ||
00366         ( ina[i]->previousblock <= 0 && i > 0 ) ) {
00367         MesPrint("File %s has inconsistent contents\n",d->name);
00368         goto thisiswrong;
00369     }
00370     position = ina[i]->previousblock;
00371 }
00372 d->info.firstnameblock = ina[0]->position;
00373 /*
00374 Give the old info back to the system.
00375 */
00376 if ( d->iblocks ) {
00377     for ( i = 0; i < d->info.numberofindexblocks; i++ ) {
00378         if ( d->iblocks[i] ) M_free(d->iblocks[i],"d->iblocks[i]");
00379     }
00380     M_free(d->iblocks,"d->iblocks");
00381 }
00382 if ( d->nblocks ) {
00383     for ( i = 0; i < d->info.numberofnamesblocks; i++ ) {
00384         if ( d->nblocks[i] ) M_free(d->nblocks[i],"d->nblocks[i]");
00385     }
00386     M_free(d->nblocks,"d->nblocks");
00387 }
00388 /*
00389 And substitute the new blocks
00390 */
00391 d->iblocks = ib;
00392 d->nblocks = ina;
00393 return(0);
00394 }
00395
00396 /*
00397 #] ReadIndex :
00398 #[ WriteIndexBlock :
00399 */
00400
00401 int WriteIndexBlock(DBASE *d,MLONG num)
00402 {
00403     if ( num >= d->info.numberofindexblocks ) {
00404         MesPrint("Illegal number specified for number of index blocks\n");
00405         return(-1);
00406     }
00407     fseek(d->handle,d->iblocks[num]->position,SEEK_SET);
00408     convertblock(d->iblocks[num],&scratchblock,TODISK);
00409     if ( minoswrite(d->handle,(char *)(&scratchblock),sizeof(INDEXBLOCK)) ) {
00410         MesPrint("Error while writing an index block in file %s\n",d->name);
00411         MesPrint("File may be unreliable now\n");
00412         return(-1);
00413     }
00414     return(0);
00415 }
00416
00417 /*
00418 #] WriteIndexBlock :
00419 #[ WriteNamesBlock :

```

```

00420 */
00421
00422 int WriteNamesBlock(DBASE *d,MLONG num)
00423 {
00424     if ( num >= d->info.numberofnamesblocks ) {
00425         MesPrint("Illegal number specified for number of names blocks\n");
00426         return(-1);
00427     }
00428     fseek(d->handle,d->nblocks[num]->position,SEEK_SET);
00429     convertnamesblock(d->nblocks[num],&scratchnamesblock,TODISK);
00430     if ( minoswrite(d->handle,(char *)(&scratchnamesblock),sizeof(NAMESBLOCK)) ) {
00431         MesPrint("Error while writing a names block in file %s\n",d->name);
00432         MesPrint("File may be unreliable now\n");
00433         return(-1);
00434     }
00435     return(0);
00436 }
00437
00438 /*
00439 #] WriteNamesBlock :
00440 #[ WriteIndex :
00441
00442 Problem here is to get the links right.
00443 */
00444
00445 int WriteIndex(DBASE *d)
00446 {
00447     MLONG i, position;
00448     if ( d->iblocks == 0 ) return(0);
00449     if ( d->nblocks == 0 ) return(0);
00450     for ( i = 0; i < d->info.numberofindexblocks; i++ ) {
00451         if ( d->iblocks[i] == 0 ) {
00452             MesPrint("Error: unassigned index blocks. Cannot write\n");
00453             return(-1);
00454         }
00455     }
00456     for ( i = 0; i < d->info.numberofnamesblocks; i++ ) {
00457         if ( d->nblocks[i] == 0 ) {
00458             MesPrint("Error: unassigned names blocks. Cannot write\n");
00459             return(-1);
00460         }
00461     }
00462     d->info.lastindexblock = -1;
00463     for ( i = 0; i < d->info.numberofindexblocks; i++ ) {
00464         position = d->iblocks[i]->position;
00465         if ( position <= 0 ) {
00466             fseek(d->handle,0,SEEK_END);
00467             position = ftell(d->handle);
00468             d->iblocks[i]->position = position;
00469             if ( i <= 0 ) d->iblocks[i]->previousblock = -1;
00470             else d->iblocks[i]->previousblock = d->iblocks[i-1]->position;
00471         }
00472         else fseek(d->handle,position,SEEK_SET);
00473         convertblock(d->iblocks[i],&scratchblock,TODISK);
00474         if ( minoswrite(d->handle,(char *)(&scratchblock),sizeof(INDEXBLOCK)) ) {
00475             MesPrint("Error while writing index of file %s",d->name);
00476             d->iblocks[i]->position = -1;
00477             return(-1);
00478         }
00479         d->info.lastindexblock = position;
00480     }
00481     d->info.lastnameblock = -1;
00482     for ( i = 0; i < d->info.numberofnamesblocks; i++ ) {
00483         position = d->nblocks[i]->position;
00484         if ( position <= 0 ) {
00485             fseek(d->handle,0,SEEK_END);
00486             position = ftell(d->handle);
00487             d->nblocks[i]->position = position;
00488             if ( i <= 0 ) d->nblocks[i]->previousblock = -1;
00489             else d->nblocks[i]->previousblock = d->nblocks[i-1]->position;
00490         }
00491         else fseek(d->handle,position,SEEK_SET);
00492         convertnamesblock(d->nblocks[i],&scratchnamesblock,TODISK);
00493         if ( minoswrite(d->handle,(char *)(&scratchnamesblock),sizeof(NAMESBLOCK)) ) {
00494             MesPrint("Error while writing index of file %s",d->name);
00495             d->nblocks[i]->position = -1;
00496             return(-1);
00497         }
00498         d->info.lastnameblock = position;
00499     }
00500     return(0);
00501 }
00502
00503 /*
00504 #] WriteIndex :
00505 #[ WriteIniInfo :
00506 */

```

```

00507
00508 int WriteIniInfo(DBASE *d)
00509 {
00510     INIINFO inf;
00511     fseek(d->handle,0,SEEK_SET);
00512     convertiniinfo(&(d->info),&inf,TODISK);
00513     if ( minoswrite(d->handle,(char *)(&inf),sizeof(INIINFO)) ) {
00514         MesPrint("Error while writing masterindex of file %s",d->name);
00515         return(-1);
00516     }
00517     return(0);
00518 }
00519
00520 /*
00521     #] WriteIniInfo :
00522     #[ ReadIniInfo :
00523 */
00524
00525 int ReadIniInfo(DBASE *d)
00526 {
00527     INIINFO inf;
00528     fseek(d->handle,0,SEEK_SET);
00529     if ( minosread(d->handle,(char *)(&inf),sizeof(INIINFO)) ) {
00530         MesPrint("Error while reading masterindex of file %s",d->name);
00531         return(-1);
00532     }
00533     convertiniinfo(&inf,&(d->info),FROMDISK);
00534     if ( d->info.entriesinindex < 0
00535         || d->info.numberofindexblocks < 0
00536         || d->info.lastindexblock < 0 ) {
00537         MesPrint("The file %s is not a proper database\n",d->name);
00538         return(-1);
00539     }
00540     return(0);
00541 }
00542
00543 /*
00544     #] ReadIniInfo :
00545     #[ GetDbase :
00546 */
00547
00548 DBASE *GetDbase(char *filename, MLONG rwmode)
00549 {
00550     FILE *f;
00551     DBASE *d;
00552     char *newname;
00553     if ( rwmode == 0 ) {
00554         if ( ( f = LocateBase(&filename,&newname,"rb") ) == 0 ) {
00555
00556             MesPrint("&Trying to open non-existent TableBase in readonly mode: %s", filename);
00557             Terminate(-1);
00558         }
00559     } else {
00560         if ( ( f = LocateBase(&filename,&newname,"r+b") ) == 0 ) {
00561
00562             return(NewDbase(filename,0));
00563         }
00564     }
00565
00566     /* setbuf(f,0); */
00567     d = (DBASE *)FromOList(&(AC.TableBaseList));
00568     d->mode = 0;
00569     d->tablenamessize = 0;
00570     d->topnumber = 0;
00571     d->tablenameefill = 0;
00572     d->iblocks = 0;
00573     d->nblocks = 0;
00574     d->tablenames = 0;
00575     d->rwmode = rwmode;
00576
00577     d->info.entriesinindex = 0;
00578     d->info.numberofindexblocks = 0;
00579     d->info.firstindexblock = 0;
00580     d->info.lastindexblock = 0;
00581     d->info.numberoftables = 0;
00582     d->info.numberofnamesblocks = 0;
00583     d->info.firstnameblock = 0;
00584     d->info.lastnameblock = 0;
00585
00586     d->name = str_dup(filename); /* For the moment just for the error messages */
00587     d->handle = f;
00588     if ( ReadIniInfo(d) || ReadIndex(d) ) { M_free(d,"index-d"); fclose(f); return(0); }
00589     if ( ComposeTableNames(d) < 0 ) { FreeTableBase(d); fclose(f); return(0); }
00590     // free allocation from previous str_dup
00591     M_free(d->name, "from str_dup");
00592     d->name = str_dup(filename);
00593     d->fullname = newname;

```



```

00594     return(d);
00595 }
00596
00597 /*
00598  #] GetDbase :
00599  #[ NewDbase :
00600
00601  Creates a new database with 'number' entries in the index.
00602 */
00603
00604 DBASE *NewDbase(char *name,MLONG number)
00605 {
00606     FILE *f;
00607     DBASE *d;
00608     MLONG numblocks, numnameblocks, i;
00609     char *s;
00610     /*-----change 10-feb-2003 */
00611     int j, jj;
00612     MLONG t = (MLONG)(time(0));
00613     /*-----*/
00614     if ( number < 0 ) number = 0;
00615     if ( ( f = fopen(name,"w+b") ) == 0 ) {
00616         MesPrint("Could not create a new file with name %s\n",name);
00617         return(0);
00618     }
00619     numblocks = (number+NUMOBJECTS-1)/NUMOBJECTS;
00620     numnameblocks = 1;
00621     if ( numblocks <= 0 ) numblocks = 1;
00622     if ( numnameblocks <= 0 ) numnameblocks = 1;
00623     d = (DBASE *)FromOList(&(AC.TableBaseList));
00624     if ( ( d->iblocks = (INDEXBLOCK **)Malloca1(numblocks*sizeof(INDEXBLOCK *),
00625 "new database") ) == 0 ) {
00626         NumTableBases--;
00627         return(0);
00628     }
00629     d->tablenames = 0;
00630     d->tablenamessize = 0;
00631     d->topnumber = 0;
00632     d->tablenameefill = 0;
00633     d->rwmode = 1;
00634
00635     d->mode = 0;
00636     if ( ( d->nblocks = (NAMESBLOCK **)Malloca1(sizeof(NAMESBLOCK *)*numnameblocks,
00637 "new database") ) == 0 ) {
00638         M_free(d->iblocks,"new database");
00639         NumTableBases--;
00640         return(0);
00641     }
00642     if ( ( f = fopen(name,"w+b") ) == 0 ) {
00643         MesPrint("Could not create new file %s\n",name);
00644         NumTableBases--;
00645         return(0);
00646     }
00647     /* setbuf(f,0); */
00648     d->name = str_dup(name);
00649     d->fullname = str_dup(name);
00650     d->handle = f;
00651
00652     d->info.entriesinindex = number;
00653     d->info.numberofindexblocks = numblocks;
00654     d->info.numberofnamesblocks = numnameblocks;
00655     d->info.firstindexblock = 0;
00656     d->info.lastindexblock = 0;
00657     d->info.numberoftables = 0;
00658     d->info.firstnameblock = 0;
00659     d->info.lastnameblock = 0;
00660
00661     if ( WriteIniInfo(d) ) {
00662 getout:
00663         fclose(f);
00664         remove(d->fullname);
00665         if ( d->name ) { M_free(d->name,"name tablebase"); d->name = 0; }
00666         if ( d->fullname ) { M_free(d->fullname,"fullname tablebase"); d->fullname = 0; }
00667         M_free(d->nblocks,"new database");
00668         M_free(d->iblocks,"new database");
00669         NumTableBases--;
00670         return(0);
00671     }
00672     for ( i = 0; i < numblocks; i++ ) {
00673         if ( ( d->iblocks[i] = (INDEXBLOCK *)Malloca1(sizeof(INDEXBLOCK),
00674 "index blocks of new database") ) == 0 ) {
00675             while ( --i >= 0 ) M_free(d->iblocks[i],"index blocks of new database");
00676             goto getout;
00677         }
00678         if ( i > 0 ) d->iblocks[i]->previousblock = d->iblocks[i-1]->position;
00679         else d->iblocks[i]->previousblock = -1;
00680         d->iblocks[i]->position = ftell(f);

```

```

00681         // Initialise, to keep valgrind happy
00682         d->iblocks[i]->flags = -1;
00683         /*-----change 10-feb-2003 */
00684         /*
00685             Zero things properly. We don't want garbage in the file.
00686         */
00687         for ( j = 0; j < NUMOBJECTS; j++ ) {
00688             d->iblocks[i]->objects[j].date = t;
00689             d->iblocks[i]->objects[j].size = 0;
00690             d->iblocks[i]->objects[j].position = -1;
00691             d->iblocks[i]->objects[j].tablename = 0;
00692             d->iblocks[i]->objects[j].uncompressed = 0;
00693             d->iblocks[i]->objects[j].spare1 = 0;
00694             d->iblocks[i]->objects[j].spare2 = 0;
00695             d->iblocks[i]->objects[j].spare3 = 0;
00696             for ( jj = 0; jj < ELEMENTSIZE; jj++ ) d->iblocks[i]->objects[j].element[jj] = 0;
00697         }
00698         convertblock(d->iblocks[i], &scratchblock, TODISK);
00699         if ( minoswrite(d->handle, (char *)(&scratchblock), sizeof(INDEXBLOCK)) ) {
00700             MesPrint("Error while writing new index blocks\n");
00701             goto getout;
00702         }
00703     }
00704     for ( i = 0; i < numnameblocks; i++ ) {
00705         if ( ( d->nblocks[i] = (NAMESBLOCK *)Malloc1(sizeof(NAMESBLOCK),
00706             "names blocks of new database") ) == 0 ) {
00707             while ( --i >= 0 ) { M_free(d->nblocks[i], "names blocks of new database"); }
00708             for ( i = 0; i < numblocks; i++ ) M_free(d->iblocks[i], "index blocks of new database");
00709             goto getout;
00710         }
00711         if ( i > 0 ) d->nblocks[i]->previousblock = d->nblocks[i-1]->position;
00712         else d->nblocks[i]->previousblock = -1;
00713         d->nblocks[i]->position = ftell(f);
00714         s = d->nblocks[i]->names;
00715         for ( j = 0; j < NAMETABLESIZE; j++ ) *s++ = 0;
00716         convertnamesblock(d->nblocks[i], &scratchnamesblock, TODISK);
00717         if ( minoswrite(d->handle, (char *)(&scratchnamesblock), sizeof(NAMESBLOCK)) ) {
00718             MesPrint("Error while writing new names blocks\n");
00719             for ( i = 0; i < numnameblocks; i++ ) M_free(d->nblocks[i], "names blocks of new
database");
00720             for ( i = 0; i < numblocks; i++ ) M_free(d->iblocks[i], "index blocks of new database");
00721             goto getout;
00722         }
00723     }
00724     d->info.firstindexblock = d->iblocks[0]->position;
00725     d->info.lastindexblock = d->iblocks[numblocks-1]->position;
00726     d->info.firstnameblock = d->nblocks[0]->position;
00727     d->info.lastnameblock = d->nblocks[numnameblocks-1]->position;
00728     if ( WriteIniInfo(d) ) {
00729         for ( i = 0; i < numnameblocks; i++ ) M_free(d->nblocks[i], "names blocks of new database");
00730         for ( i = 0; i < numblocks; i++ ) M_free(d->iblocks[i], "index blocks of new database");
00731         goto getout;
00732     }
00733     return(d);
00734 }
00735
00736 /*
00737     #] NewDbase :
00738     #[ FreeTableBase :
00739 */
00740
00741 void FreeTableBase(DBASE *d)
00742 {
00743     int i, j, *old, *newL;
00744     LIST *L;
00745     for ( i = 0; i < d->info.numberofnamesblocks; i++ ) M_free(d->nblocks[i], "nblocks[i]");
00746     for ( i = 0; i < d->info.numberofindexblocks; i++ ) M_free(d->iblocks[i], "iblocks[i]");
00747     M_free(d->nblocks, "nblocks");
00748     M_free(d->iblocks, "iblocks");
00749     if ( d->tablenames ) M_free(d->tablenames, "d->tablenames");
00750     if ( d->name ) M_free(d->name, "d->name");
00751     if ( d->fullname ) M_free(d->fullname, "d->fullname");
00752     i = d - tablebases;
00753     if ( i < ( NumTableBases - 1 ) ) {
00754         L = &(AC.TableBaseList);
00755         j = ( ( NumTableBases - i - 1 ) * L->size ) / sizeof(int);
00756         old = (int *)d; newL = (int *)d+1;
00757         while ( --j >= 0 ) *newL++ = *old++;
00758         j = L->size / sizeof(int);
00759         while ( --j >= 0 ) *newL++ = 0;
00760     }
00761     NumTableBases--;
00762     M_free(d, "tb, d");
00763 }
00764
00765 /*
00766     #] FreeTableBase :

```

```

00767     #[ ComposeTableNames :
00768
00769         The nameblocks are supposed to be in memory.
00770         Hence we have to go through them
00771     */
00772
00773 int ComposeTableNames(DBASE *d)
00774 {
00775     MLONG nsize = 0;
00776     int i, j, k;
00777     char *s, *t, *ss;
00778     d->topnumber = 0;
00779     i = 0; s = d->nblocks[i]->names; j = NAMETABLESIZE;
00780     while ( *s ) {
00781         if ( *s ) d->topnumber++;
00782         for ( k = 0; k < 2; k++ ) { /* name and argtail */
00783             while ( *s ) {
00784                 j--;
00785                 if ( j <= 0 ) {
00786                     i++; if ( i >= d->info.numberofnamesblocks ) goto gotall;
00787                     s = d->nblocks[i]->names; j = NAMETABLESIZE;
00788                 }
00789                 else s++;
00790             }
00791             j--;
00792             if ( j <= 0 ) {
00793                 i++; if ( i >= d->info.numberofnamesblocks ) goto gotall;
00794                 s = d->nblocks[i]->names; j = NAMETABLESIZE;
00795             }
00796             else s++;
00797         }
00798     }
00799 gotall:;
00800     nsize = (d->info.numberofnamesblocks-1)*NAMETABLESIZE +
00801             (s-d->nblocks[i]->names)+1;
00802     if ( ( d->tablenames = (char *)Malloca((2*nsize+30)*sizeof(char),"tablenames") )
00803         == 0 ) { return(-1); }
00804     t = d->tablenames;
00805     d->tablenamessize = 2*nsize+30;
00806     d->tablenamefill = nsize-1;
00807     for ( k = 0; k < i; k++ ) {
00808         ss = d->nblocks[k]->names;
00809         for ( j = 0; j < NAMETABLESIZE; j++ ) *t++ = *ss++;
00810     }
00811     ss = d->nblocks[i]->names;
00812     while ( ss < s ) *t++ = *ss++;
00813     *t = 0;
00814     return(0);
00815 }
00816
00817 /*
00818     #[ ComposeTableNames :
00819     #[ OpenDbase :
00820     */
00821
00822 DBASE *OpenDbase(char *filename)
00823 {
00824     FILE *f;
00825     DBASE *d;
00826     char *newname;
00827     if ( ( f = LocateBase(&filename,&newname,"r+b") ) == 0 ) {
00828         MesPrint("Cannot open file %s\n",filename);
00829         return(0);
00830     }
00831     /* setbuf(f,0); */
00832     d = (DBASE *)From0List(&(AC.TableBaseList));
00833     d->name = filename; /* For the moment just for the error messages */
00834     d->handle = f;
00835     if ( ReadIniInfo(d) || ReadIndex(d) ) { M_free(d,"OpenDbase"); fclose(f); return(0); }
00836     if ( ComposeTableNames(d) ) {
00837         FreeTableBase(d);
00838         fclose(f);
00839         return(0);
00840     }
00841     d->name = str_dup(filename);
00842     d->fullname = newname;
00843     return(d);
00844 }
00845
00846 /*
00847     #[ OpenDbase :
00848     #[ AddTableName :
00849
00850     Adds a name of a table. Writes the namelist to disk.
00851     Returns the number of this tablename in the database.
00852     If the name was already in the table we return its value in negative.
00853     Zero is an error!

```

```

00854 */
00855
00856 MLONG AddTableName(DBASE *d,char *name, TABLES T)
00857 {
00858     char *s, *t, *tt;
00859     int namesize, tailsize;
00860     MLONG newsize, i, num;
00861     /*
00862     First search for the name in what we have already
00863     */
00864     if ( d->tablenames ) {
00865         num = 0;
00866         s = d->tablenames;
00867         while ( *s ) {
00868             num++;
00869             t = name;
00870             while ( ( *s == *t ) && *t ) { s++; t++; }
00871             if ( *s == *t ) { return(-num); }
00872             while ( *s ) s++;
00873             s++;
00874             while ( *s ) s++;
00875             s++;
00876         }
00877     }
00878     /*
00879     This name has to be added
00880     */
00881     MesPrint("We add the name %s\n",name);
00882     t = name;
00883     while ( *t ) { t++; }
00884     namesize = t-name;
00885     if ( ( t = (char *) (T->argtail) ) != 0 ) {
00886         while ( *t ) { t++; }
00887         tailsize = t - (char *) (T->argtail);
00888     }
00889     else { tailsize = 0; }
00890     if ( d->tablenames == 0 ) {
00891         if ( ComposeTableNames(d) ) {
00892             FreeTableBase(d);
00893             M_free(d,"AddTableName");
00894             return(0);
00895         }
00896     }
00897     d->info.numberoftables++;
00898     while ( ( d->tablenameefill+namesize+tailsize+3 > d->tablenamessize )
00899         || ( d->tablenames == 0 ) ) {
00900         newsize = 2*d->tablenamessize + 2*namesize + 2*tailsize + 6;
00901         if ( ( t = (char *) Malloc1(newsize*sizeof(char),"AddTableName") ) == 0 )
00902             return(0);
00903         tt = t;
00904         if ( d->tablenames ) {
00905             s = d->tablenames;
00906             for ( i = 0; i < d->tablenameefill; i++ ) *t++ = *s++;
00907             *t = 0;
00908             M_free(d->tablenames,"d->tablenames");
00909         }
00910         d->tablenames = tt;
00911         d->tablenamessize = newsize;
00912     }
00913     s = d->tablenames + d->tablenameefill;
00914     t = name;
00915     while ( *t ) *s++ = *t++;
00916     *s++ = 0;
00917     t = (char *) (T->argtail);
00918     while ( *t ) *s++ = *t++;
00919     *s++ = 0;
00920     *s = 0;
00921     d->tablenameefill = s - d->tablenames;
00922     d->topnumber++;
00923     /*
00924     Now we have to synchronize
00925     */
00926     if ( PutTableNames(d) ) return(0);
00927     return(d->topnumber);
00928 }
00929
00930 /*
00931 #] AddTableName :
00932 #[ GetTableName :
00933
00934 Gets a name of a table.
00935 Returns the number of this tablename in the database.
00936 Zero -> error
00937 */
00938
00939 MLONG GetTableName(DBASE *d,char *name)
00940 {

```

```

00941     char *s, *t;
00942     MLONG num;
00943     /*
00944     search for the name in what we have
00945     */
00946     if ( d->tablenames ) {
00947         num = 0;
00948         s = d->tablenames;
00949         while ( *s ) {
00950             num++;
00951             t = name;
00952             while ( ( *s == *t ) && *t ) { s++; t++; }
00953             if ( *s == *t ) { return(num); }
00954             while ( *s ) s++;
00955             s++;
00956             while ( *s ) s++;
00957             s++;
00958         }
00959     }
00960     return(0);
00961 }
00962
00963 /*
00964 #] GetTableName :
00965 #[ PutTableNames :
00966
00967 Takes the names string in d->tablenames and puts it in the nblocks
00968 pieces. Writes what has been changed to disk.
00969 */
00970
00971 int PutTableNames(DBASE *d)
00972 {
00973     NAMESBLOCK **nnew;
00974     int i, j, firstdif;
00975     MLONG m;
00976     char *s, *t;
00977     /*
00978     Determine how many blocks are needed.
00979     */
00980     MLONG numblocks = d->tablenamefill/NAMETABLESIZE + 1;
00981     if ( d->info.numberofnamesblocks < numblocks ) {
00982         /*
00983         We need more blocks. First make sure of the space for nblocks.
00984         */
00985         if ( ( nnew = (NAMESBLOCK **)Malloc1(sizeof(NAMESBLOCK *)*numblocks,
00986             "new names block" ) ) == 0 ) {
00987             return(-1);
00988         }
00989         for ( i = 0; i < d->info.numberofnamesblocks; i++ ) {
00990             nnew[i] = d->nblocks[i];
00991         }
00992         free(d->nblocks);
00993         d->nblocks = nnew;
00994         for ( ; i < numblocks; i++ ) {
00995             if ( ( d->nblocks[i] = (NAMESBLOCK *)Malloc1(sizeof(NAMESBLOCK),
00996                 "additional names blocks " ) ) == 0 ) {
00997                 FreeTableBase(d);
00998                 return(-1);
00999             }
01000             d->nblocks[i]->previousblock = -1;
01001             d->nblocks[i]->position = -1;
01002             s = d->nblocks[i]->names;
01003             for ( j = 0; j < NAMETABLESIZE; j++ ) *s++ = 0;
01004         }
01005         d->info.numberofnamesblocks = numblocks;
01006     }
01007     /*
01008     Now look till where the new contents agree with the old.
01009     */
01010     firstdif = 0;
01011     i = 0; t = d->nblocks[i]->names; j = 0; s = d->tablenames;
01012     for ( m = 0; m < d->tablenamefill; m++ ) {
01013         if ( *s == *t ) {
01014             s++; t++; j++;
01015             if ( j >= NAMETABLESIZE ) {
01016                 i++;
01017                 t = d->nblocks[i]->names;
01018                 j = 0;
01019             }
01020         }
01021         else {
01022             firstdif = i;
01023             for ( ; m < d->tablenamefill; m++ ) {
01024                 *t++ = *s++; j++;
01025                 if ( j >= NAMETABLESIZE ) {
01026                     i++;
01027                     t = d->nblocks[i]->names;

```

```

01028         j = 0;
01029     }
01030 }
01031 *t = 0;
01032 break;
01033 }
01034 }
01035 for ( i = 0; i < d->info.numberofnamesblocks; i++ ) {
01036     if ( i == firstdif ) break;
01037     if ( d->nblocks[i]->position < 0 ) { firstdif = i; break; }
01038 }
01039 /*
01040 Now we have to (re)write the blocks, starting at firstdif.
01041 */
01042 for ( i = firstdif; i < d->info.numberofnamesblocks; i++ ) {
01043     if ( i > 0 ) d->nblocks[i]->previousblock = d->nblocks[i-1]->position;
01044     else d->nblocks[i]->previousblock = -1;
01045     if ( d->nblocks[i]->position < 0 ) {
01046         fseek(d->handle,0,SEEK_END);
01047         d->nblocks[i]->position = ftell(d->handle);
01048     }
01049     else fseek(d->handle,d->nblocks[i]->position,SEEK_SET);
01050     convertnamesblock(d->nblocks[i],&scratchnamesblock,TODISK);
01051     if ( minoswrite(d->handle,(char *)(&scratchnamesblock),sizeof(NAMESBLOCK)) ) {
01052         MesPrint("Error while writing names blocks\n");
01053         FreeTableBase(d);
01054         return(-1);
01055     }
01056 }
01057 d->info.lastnameblock = d->nblocks[d->info.numberofnamesblocks-1]->position;
01058 d->info.firstnameblock = d->nblocks[0]->position;
01059 return(WriteIniInfo(d));
01060 }
01061
01062 /*
01063 #] PutTableNames :
01064 #[ AddToIndex :
01065 */
01066
01067 int AddToIndex(DBASE *d,MLONG number)
01068 {
01069     MLONG i, oldnumofindexblocks = d->info.numberofindexblocks;
01070     MLONG j, newnumofindexblocks, jj;
01071     INDEXBLOCK **ib;
01072     MLONG t = (MLONG)(time(0));
01073     if ( number == 0 ) return(0);
01074     else if ( number < 0 ) {
01075         if ( d->info.entriesinindex < -number ) {
01076             MesPrint("There are only %ld entries in the index of file %s\n",
01077                 d->info.entriesinindex,d->name);
01078             return(-1);
01079         }
01080         d->info.entriesinindex += number;
01081     dowrite:
01082         if ( WriteIniInfo(d) ) {
01083             d->info.entriesinindex -= number;
01084             MesPrint("File may be corrupted\n");
01085             return(-1);
01086         }
01087     }
01088     else if ( d->info.entriesinindex+number <=
01089 NUMOBJECTS*d->info.numberofindexblocks ) {
01090         d->info.entriesinindex += number;
01091         goto dowrite;
01092     }
01093     else {
01094         d->info.entriesinindex += number;
01095         newnumofindexblocks = d->info.numberofindexblocks + ((number -
01096 (NUMOBJECTS*d->info.numberofindexblocks - d->info.entriesinindex))
01097 +NUMOBJECTS-1)/NUMOBJECTS;
01098         if ( ( ib = (INDEXBLOCK **)Mallocl(sizeof(INDEXBLOCK *)*newnumofindexblocks,
01099 "index") ) == 0 ) return(-1);
01100         for ( i = 0; i < d->info.numberofindexblocks; i++ ) {
01101             ib[i] = d->iblocks[i];
01102         }
01103         for ( i = d->info.numberofindexblocks; i < newnumofindexblocks; i++ ) {
01104             if ( ( ib[i] = (INDEXBLOCK *)Mallocl(sizeof(INDEXBLOCK),"index block") ) == 0 ) {
01105                 FreeTableBase(d);
01106                 return(-1);
01107             }
01108             if ( i > 0 ) ib[i]->previousblock = ib[i-1]->position;
01109             else ib[i]->previousblock = -1;
01110         }
01111         Zero things properly. We don't want garbage in the file.
01112     */
01113     for ( j = 0; j < NUMOBJECTS; j++ ) {
01114         ib[i]->objects[j].date = t;

```

```

01115         ib[i]->objects[j].size = 0;
01116         ib[i]->objects[j].position = -1;
01117         ib[i]->objects[j].tablenumber = 0;
01118         ib[i]->objects[j].uncompressed = 0;
01119         ib[i]->objects[j].spare1 = 0;
01120         ib[i]->objects[j].spare2 = 0;
01121         ib[i]->objects[j].spare3 = 0;
01122         for ( jj = 0; jj < ELEMENTSIZE; jj++ ) ib[i]->objects[j].element[jj] = 0;
01123     }
01124     fseek(d->handle,0,SEEK_END);
01125     ib[i]->position = ftell(d->handle);
01126     convertblock(ib[i],&scratchblock,TODISK);
01127     if ( minoswrite(d->handle,(char *)(&scratchblock),sizeof(INDEXBLOCK)) ) {
01128         MesPrint("Error while writing new index of file %s",d->name);
01129         FreeTableBase(d);
01130         return(-1);
01131     }
01132 }
01133 d->info.lastindexblock = ib[newnumofindexblocks-1]->position;
01134 d->info.firstindexblock = ib[0]->position;
01135 d->info.numberofindexblocks = newnumofindexblocks;
01136 if ( WriteIniInfo(d) ) {
01137     d->info.numberofindexblocks = oldnumofindexblocks;
01138     d->info.entriesinindex -= number;
01139     MesPrint("File may be corrupted\n");
01140     FreeTableBase(d);
01141     return(-1);
01142 }
01143 M_free(d->iblocks,"AddToIndex");
01144 d->iblocks = ib;
01145 }
01146 return(0);
01147 }
01148
01149 /*
01150 #] AddToIndex :
01151 #[ AddObject :
01152 */
01153
01154 MLONG AddObject(DBASE *d,MLONG tablenumber,char *arguments,char *rhs)
01155 {
01156     MLONG number;
01157     number = d->info.entriesinindex;
01158     if ( AddToIndex(d,1) ) return(-1);
01159     if ( WriteObject(d,tablenumber,arguments,rhs,number) ) return(-1);
01160     return(number);
01161 }
01162
01163 /*
01164 #] AddObject :
01165 #[ FindTableNumber :
01166 */
01167
01168 MLONG FindTableNumber(DBASE *d,char *name)
01169 {
01170     char *s = d->tablenames, *t, *ss;
01171     MLONG num = 0;
01172     ss = d->tablenames + d->tablenamefill;
01173     while ( s < ss ) {
01174         num++;
01175         t = name;
01176         while ( *s == *t && *t ) {
01177             s++; t++;
01178         }
01179         if ( *s == 0 && *t == 0 ) return(num);
01180         while ( *s ) s++;
01181         s++;
01182     }
01183     /* Skip also the argument tail
01184     */
01185     while ( *s ) s++;
01186     s++;
01187 }
01188 return(-1); /* Name not found */
01189 }
01190
01191 /*
01192 #] FindTableNumber :
01193 #[ WriteObject :
01194 */
01195
01196 int WriteObject(DBASE *d,MLONG tablenumber,char *arguments,char *rhs,MLONG number)
01197 {
01198     char *s, *a;
01199 #ifdef WITHZLIB
01200     char *buffer = 0;
01201     uLongf newsize = 0, oldsize = 0;

```

```

01202     uLong ssize;
01203     int error = 0;
01204 #endif
01205     MLONG i, j, position, size, n;
01206     OBJECTS *obj;
01207     if ( ( d->mode & INPUTONLY ) == INPUTONLY ) {
01208         MesPrint("Not allowed to write to input\n");
01209         return(-1);
01210     }
01211     if ( number >= d->info.entriesinindex ) {
01212         MesPrint("Reference to non-existing object number %ld\n",number+1);
01213         return(0);
01214     }
01215     j = number/NUMOBJECTS;
01216     i = number%NUMOBJECTS;
01217     obj = &(d->iblocks[j]->objects[i]);
01218     a = arguments;
01219     while ( *a ) a++;
01220     a++; n = a - arguments;
01221     if ( n > ELEMENTSIZE ) {
01222         MesPrint("Table element %s has more than %ld characters.\n",arguments,
01223             (MLONG)ELEMENTSIZE);
01224         return(-1);
01225     }
01226     s = obj->element;
01227     a = arguments;
01228     while ( *a ) *s++ = *a++;
01229     *s++ = 0;
01230     while ( n < ELEMENTSIZE ) { *s++ = 0; n++; }
01231     obj->spare1 = obj->spare2 = obj->spare3 = 0;
01232
01233     fseek(d->handle,0,SEEK_END);
01234     position = ftell(d->handle);
01235     s = rhs;
01236     while ( *s ) s++;
01237     s++;
01238     size = s - rhs;
01239 #ifdef WITHZLIB
01240     if ( ( d->mode & NOCOMPRESS ) == 0 ) {
01241         newsize = size + size/1000 + 20;
01242         if ( ( buffer = (char *)Malloca1(newsize*sizeof(char),"compress buffer") )
01243             == 0 ) {
01244             MesPrint("No compress used for element %s in file %s\n",arguments,d->name);
01245         }
01246     }
01247     else buffer = 0;
01248     if ( buffer ) {
01249         ssize = size;
01250 #ifdef WITHZSTD
01251         // Force the use of zlib for compressed Tablebase entries, so that tablebases created
01252         // with zstd-supported FORM builds can be used by zstd-unsupported FORM builds.
01253         const int old_isUsingZSTDcompression = ZWRAP_isUsingZSTDcompression();
01254         ZWRAP_useZSTDcompression(0);
01255 #endif
01256         if ( ( error = compress((Bytef *)buffer,&newsize,(Bytef *)rhs,ssize) ) != Z_OK ) {
01257             MesPrint("Error = %d\n",error);
01258             MesPrint("Due to error no compress used for element %s in file %s\n",arguments,d->name);
01259             M_free(buffer,"tb,WriteObject");
01260             buffer = 0;
01261         }
01262 #ifdef WITHZSTD
01263         ZWRAP_useZSTDcompression(old_isUsingZSTDcompression);
01264 #endif
01265     }
01266     if ( buffer ) {
01267         rhs = buffer;
01268         oldsize = size;
01269         size = newsize;
01270     }
01271 #endif
01272     if ( minoswrite(d->handle,rhs,size) ) {
01273         MesPrint("Error while writing rhs\n");
01274         return(-1);
01275     }
01276     obj->position = position;
01277     obj->size = size;
01278     obj->date = (MLONG)(time(0));
01279     obj->tablenumber = tablenumber;
01280 #ifdef WITHZLIB
01281     obj->uncompressed = oldsize;
01282     if ( buffer ) M_free(buffer,"tb,WriteObject");
01283 #else
01284     obj->uncompressed = 0;
01285 #endif
01286     return(WriteIndexBlock(d,j));
01287 }
01288

```



```

01289 /*
01290     #] WriteObject :
01291     #[ ReadObject :
01292
01293     Returns a pointer to the proper rhs
01294 */
01295
01296 char *ReadObject(DBASE *d,MLONG tablenumber,char *arguments)
01297 {
01298     OBJECTS *obj;
01299     MLONG i, j;
01300     char *buffer1, *s, *t;
01301 #ifdef WITHZLIB
01302     char *buffer2 = 0;
01303     uLongf finallength = 0;
01304 #endif
01305     if ( tablenumber > d->topnumber ) {
01306         MesPrint("Reference to non-existing table number in tablebase %s: %ld\n",
01307             d->name,tablenumber);
01308         return(0);
01309     }
01310 /*
01311     Start looking for the object
01312 */
01313     for ( i = 0; i < d->info.numberofindexblocks; i++ ) {
01314         for ( j = 0; j < NUMOBJECTS; j++ ) {
01315             if ( d->iblocks[i]->objects[j].tablenumber != tablenumber ) continue;
01316             s = arguments; t = d->iblocks[i]->objects[j].element;
01317             while ( *s == *t && *s ) { s++; t++; }
01318             if ( *t == 0 && *s == 0 ) goto foundelement;
01319         }
01320     }
01321     s = d->tablenames; i = 1;
01322     while ( *s ) {
01323         if ( i == tablenumber ) break;
01324         while ( *s ) s++;
01325         s++;
01326         while ( *s ) s++;
01327         s++;
01328         i++;
01329     }
01330     MesPrint("%s(%s) not found in tablebase %s\n",s,arguments,d->name);
01331     return(0);
01332 foundelement:;
01333     obj = &(d->iblocks[i]->objects[j]);
01334     fseek(d->handle,obj->position,SEEK_SET);
01335     if ( ( buffer1 = (char *)Malloc1(obj->size,"reading rhs buffer1") ) == 0 ) {
01336         return(0);
01337     }
01338 #ifdef WITHZLIB
01339     if ( obj->uncompressed > 0 ) {
01340         if ( ( buffer2 = (char *)Malloc1(obj->uncompressed,"reading rhs buffer2") ) == 0 ) {
01341             return(0);
01342         }
01343     }
01344     else buffer2 = 0;
01345 #endif
01346     if ( minosread(d->handle,buffer1,obj->size) ) {
01347         MesPrint("Could not read rhs %s in file %s\n",arguments,d->name);
01348         M_free(buffer1,"tb,ReadObject");
01349 #ifdef WITHZLIB
01350         if ( buffer2 ) M_free(buffer2,"tb,ReadObject");
01351 #endif
01352         return(0);
01353     }
01354 #ifdef WITHZLIB
01355     if ( buffer2 == 0 ) return(buffer1);
01356     finallength = obj->uncompressed;
01357     if ( uncompress((Bytef *)buffer2,&finallength,(Bytef *)buffer1,obj->size) != Z_OK ) {
01358         MesPrint("Cannot uncompress element %s in file %s\n",arguments,d->name);
01359         M_free(buffer1,"tb,ReadObject"); M_free(buffer2,"tb,ReadObject");
01360         return(0);
01361     }
01362     M_free(buffer1,"tb,ReadObject");
01363     return(buffer2);
01364 #else
01365     return(buffer1);
01366 #endif
01367 }
01368 /*
01369     #] ReadObject :
01370     #[ ReadijObject :
01371
01372     Returns a pointer to the proper rhs
01373 */

```

```

01376
01377 char *ReadijObject(DBASE *d,MLONG i,MLONG j,char *arguments)
01378 {
01379     OBJECTS *obj;
01380     char *buffer1;
01381     #ifdef WITHZLIB
01382     char *buffer2 = 0;
01383     uLongf finallength = 0;
01384     #endif
01385     obj = &(d->iblocks[i]->objects[j]);
01386     fseek(d->handle,obj->position,SEEK_SET);
01387     if ( ( buffer1 = (char *)Malloc1(obj->size,"reading rhs buffer1") ) == 0 ) {
01388         return(0);
01389     }
01390     #ifdef WITHZLIB
01391     if ( obj->uncompressed > 0 ) {
01392         if ( ( buffer2 = (char *)Malloc1(obj->uncompressed,"reading rhs buffer2") ) == 0 ) {
01393             return(0);
01394         }
01395     }
01396     else buffer2 = 0;
01397     #endif
01398     if ( minosread(d->handle,buffer1,obj->size) ) {
01399         MesPrint("Could not read rhs %s in file %s\n",arguments,d->name);
01400         if ( buffer1 ) M_free(buffer1,"rhs buffer1");
01401         #ifdef WITHZLIB
01402         if ( buffer2 ) M_free(buffer2,"rhs buffer2");
01403         #endif
01404         return(0);
01405     }
01406     #ifdef WITHZLIB
01407     if ( buffer2 == 0 ) return(buffer1);
01408     finallength = obj->uncompressed;
01409     if ( uncompress((Bytef *)buffer2,&finallength,(Bytef *)buffer1,obj->size) != Z_OK ) {
01410         MesPrint("Cannot uncompress element %s in file %s\n",arguments,d->name);
01411         if ( buffer1 ) M_free(buffer1,"rhs buffer1");
01412         if ( buffer2 ) M_free(buffer2,"rhs buffer2");
01413         return(0);
01414     }
01415     M_free(buffer1,"rhs buffer1");
01416     return(buffer2);
01417     #else
01418     return(buffer1);
01419     #endif
01420 }
01421
01422 /*
01423 #] ReadijObject :
01424 #[ ExistsObject :
01425
01426 Returns 1 if Object exists
01427 */
01428
01429 int ExistsObject(DBASE *d,MLONG tablenumber,char *arguments)
01430 {
01431     MLONG i, j;
01432     char *s, *t;
01433     if ( tablenumber > d->topnumber ) {
01434         MesPrint("Reference to non-existing table number in tablebase %s: %ld\n",
01435             d->name,tablenumber);
01436         return(0);
01437     }
01438     /*
01439     Start looking for the object
01440     */
01441     for ( i = 0; i < d->info.numberofindexblocks; i++ ) {
01442         for ( j = 0; j < NUMOBJECTS; j++ ) {
01443             if ( d->iblocks[i]->objects[j].tablenumber != tablenumber ) continue;
01444             s = arguments; t = d->iblocks[i]->objects[j].element;
01445             while ( *s == *t && *s ) { s++; t++; }
01446             if ( *t == 0 && *s == 0 ) return(1);
01447         }
01448     }
01449     return(0);
01450 }
01451
01452 /*
01453 #] ExistsObject :
01454 #[ DeleteObject :
01455
01456 Returns 1 if Object has been deleted.
01457 We leave a hole. Actually the object is still there but has been
01458 inactivated. It can be reactivated by calling this routine again.
01459 */
01460
01461 int DeleteObject(DBASE *d,MLONG tablenumber,char *arguments)
01462 {

```

```

01463     MLONG i, j;
01464     char *s, *t;
01465     if ( tablenumber > d->topnumber ) {
01466         MesPrint("Reference to non-existing table number in tablebase %s: %ld\n",
01467             d->name, tablenumber);
01468         return(0);
01469     }
01470     /*
01471     Start looking for the object
01472     */
01473     for ( i = 0; i < d->info.numberofindexblocks; i++ ) {
01474         for ( j = 0; j < NUMOBJECTS; j++ ) {
01475             if ( d->iblocks[i]->objects[j].tablenumber != tablenumber ) continue;
01476             s = arguments; t = d->iblocks[i]->objects[j].element;
01477             while ( *s == *t && *s ) { s++; t++; }
01478             if ( *t == 0 && *s == 0 ) {
01479                 d->iblocks[i]->objects[j].tablenumber =
01480                     -d->iblocks[i]->objects[j].tablenumber - 1;
01481                 return(1);
01482             }
01483         }
01484     }
01485     return(0);
01486 }
01487
01488 /*
01489 #] DeleteObject :
01490 */

```

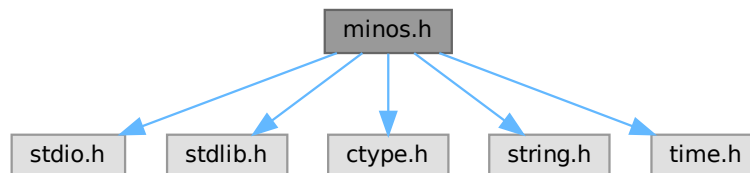
4.73 minos.h File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>
#include <time.h>

```

Include dependency graph for minos.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [iniinfo](#)
- struct [objects](#)
- struct [indexblock](#)
- struct [nameblock](#)
- struct [dbase](#)

Macros

- `#define` [MAXBASES](#) 16
- `#define` [NUMOBJECTS](#) 100
- `#define` [MAXINDEXSIZE](#) 33000000L
- `#define` [NAMETABLESIZE](#) 1008
- `#define` [ELEMENTSIZE](#) 128
- `#define` [TODISK](#) 0
- `#define` [FROMDISK](#) 1
- `#define` [MDIRTYFLAG](#) 1
- `#define` [MCLEANFLAG](#) (~MDIRTYFLAG)
- `#define` [INANDOUT](#) 0
- `#define` [INPUTONLY](#) 1
- `#define` [OUTPUTONLY](#) 2
- `#define` [NOCOMPRESS](#) 4

Typedefs

- `typedef struct` [iniinfo](#) **INIINFO**
- `typedef struct` [objects](#) **OBJECTS**
- `typedef struct` [indexblock](#) **INDEXBLOCK**
- `typedef struct` [nameblock](#) **NAMESBLOCK**
- `typedef struct` [dbase](#) **DBASE**

Functions

- `int` [minosread](#) (FILE *f, char *buffer, MLONG size)
- `int` [minoswrite](#) (FILE *f, char *buffer, MLONG size)

Variables

- `int` [withoutflush](#)

4.73.1 Detailed Description

Contains all needed declarations and definitions for the tablebase low level file routines. These have been taken from the minos database system and modified somewhat.

!!!CAUTION!!! Changes in this file will most likely have consequences for the recovery mechanism (see [checkpoint.c](#)). You need to care for the code in [checkpoint.c](#) as well and modify the code there accordingly!

Definition in file [minos.h](#).

4.73.2 Macro Definition Documentation

4.73.2.1 MAXBASES

```
#define MAXBASES 16
```

Definition at line 62 of file [minos.h](#).

4.73.2.2 NUMOBJECTS

```
#define NUMOBJECTS 100
```

Definition at line 69 of file [minos.h](#).

4.73.2.3 MAXINDEXSIZE

```
#define MAXINDEXSIZE 330000000L
```

Definition at line 70 of file [minos.h](#).

4.73.2.4 NAMETABLESIZE

```
#define NAMETABLESIZE 1008
```

Definition at line 71 of file [minos.h](#).

4.73.2.5 ELEMENTSIZE

```
#define ELEMENTSIZE 128
```

Definition at line 72 of file [minos.h](#).

4.73.2.6 TODISK

```
#define TODISK 0
```

Definition at line 143 of file [minos.h](#).

4.73.2.7 FROMDISK

```
#define FROMDISK 1
```

Definition at line 144 of file [minos.h](#).

4.73.2.8 MDIRTYFLAG

```
#define MDIRTYFLAG 1
```

Definition at line 145 of file [minos.h](#).

4.73.2.9 MCLEANFLAG

```
#define MCLEANFLAG (~MDIRTYFLAG)
```

Definition at line 146 of file [minos.h](#).

4.73.2.10 INANDOUT

```
#define INANDOUT 0
```

Definition at line 147 of file [minos.h](#).

4.73.2.11 INPUTONLY

```
#define INPUTONLY 1
```

Definition at line 148 of file [minos.h](#).

4.73.2.12 OUTPUTONLY

```
#define OUTPUTONLY 2
```

Definition at line 149 of file [minos.h](#).

4.73.2.13 NOCOMPRESS

```
#define NOCOMPRESS 4
```

Definition at line 150 of file [minos.h](#).

4.73.3 Function Documentation

4.73.3.1 minosread()

```
int minosread (
    FILE * f,
    char * buffer,
    MLONG size )
```

Definition at line 66 of file [minos.c](#).

4.73.3.2 minoswrite()

```
int minoswrite (
    FILE * f,
    char * buffer,
    MLONG size )
```

Definition at line 83 of file [minos.c](#).

4.73.4 Variable Documentation

4.73.4.1 withoutflush

int withoutflush [extern]

Definition at line 45 of file [minos.c](#).

4.74 minos.h

[Go to the documentation of this file.](#)

```

00001 #ifndef __MANAGE_H__
00002
00003 #define __MANAGE_H__
00004
00017 /* #[] License : */
00018 /*
00019  * Copyright (C) 1984-2023 J.A.M. Vermaseren
00020  * When using this file you are requested to refer to the publication
00021  * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00022  * This is considered a matter of courtesy as the development was paid
00023  * for by FOM the Dutch physics granting agency and we would like to
00024  * be able to track its scientific use to convince FOM of its value
00025  * for the community.
00026  *
00027  * This file is part of FORM.
00028  *
00029  * FORM is free software: you can redistribute it and/or modify it under the
00030  * terms of the GNU General Public License as published by the Free Software
00031  * Foundation, either version 3 of the License, or (at your option) any later
00032  * version.
00033  *
00034  * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00035  * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00036  * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00037  * details.
00038  *
00039  * You should have received a copy of the GNU General Public License along
00040  * with FORM. If not, see <http://www.gnu.org/licenses/>.
00041 */
00042 /* #[] License : */
00043
00044 #include <stdio.h>
00045 #include <stdlib.h>
00046 #include <ctype.h>
00047 #include <string.h>
00048 #include <time.h>
00049
00050 /*
00051  * The following typedef has been moved to form3.h where all the sizes
00052  * are defined for the various memory models.
00053  * We want MLONG to have a more or less fixed size.
00054  * In form3.h we try to fix it at 8 bytes.
00055  * This should make files exchangeable
00056  * between various 32-bits and 64-bits systems. At 4 bytes it might have
00057  * problems with files of more than 2 Gbytes.
00058
00059 typedef long MLONG;
00060 */
00061
00062 #define MAXBASES 16
00063 #ifdef WORDSIZE32
00064 #define NUMOBJECTS 1024
00065 #define MAXINDEXSIZE 1000000000L
00066 #define NAMETABLESIZE 1008
00067 #define ELEMENTSIZE 256
00068 #else
00069 #define NUMOBJECTS 100
00070 #define MAXINDEXSIZE 330000000L
00071 #define NAMETABLESIZE 1008
00072 #define ELEMENTSIZE 128
00073 #endif
00074
00075 int minosread(FILE *f,char *buffer,MLONG size);
00076 int minoswrite(FILE *f,char *buffer,MLONG size);
00077
00078 /*

```

```

00079     ELEMENTSIZE should make a nice number of sizeof(OBJECTS)
00080     Usually this will be much too much, but there are cases.....
00081 */
00082
00083 typedef struct iniinfo {
00084 /* should contains only MLONG variables or convertiniinfo should be modified */
00085     MLONG     entriesinindex;
00086     MLONG     numberofindexblocks;
00087     MLONG     firstindexblock;
00088     MLONG     lastindexblock;
00089     MLONG     numberoftables;
00090     MLONG     numberofnamesblocks;
00091     MLONG     firstnameblock;
00092     MLONG     lastnameblock;
00093 } INIINFO;
00094
00095 typedef struct objects {
00096 /* if any changes, convertblock should be adapted too!!!! */
00097     MLONG position;                /* position of RHS= */
00098     MLONG size;                    /* size on disk (could be compressed) */
00099     MLONG date;                    /* Time stamp */
00100     MLONG tablenumber;             /* Number of table. Refers to name in special index */
00101     MLONG uncompressed;           /* uncompressed size if compressed. If not: 0 */
00102     MLONG spare1;
00103     MLONG spare2;
00104     MLONG spare3;
00105     char element[ELEMENTSIZE];    /* table element in character form */
00106 } OBJECTS;
00107
00108 typedef struct indexblock {
00109     MLONG     flags;
00110     MLONG     previousblock;
00111     MLONG     position;
00112     OBJECTS   objects[NUMOBJECTS];
00113 } INDEXBLOCK;
00114
00115 typedef struct nameblock {
00116     MLONG     previousblock;
00117     MLONG     position;
00118     char      names[NAMETABLESIZE];
00119 } NAMESBLOCK;
00120
00121 typedef struct dbase {
00122     INIINFO    info;
00123     MLONG      mode;
00124     MLONG      tablenamessize;
00125     MLONG      topnumber;
00126     MLONG      tablenamefill;
00127     MLONG      rwmode;
00128     INDEXBLOCK **iblocks;
00129     NAMESBLOCK **nbblocks;
00130     FILE        *handle;
00131     char        *name;
00132     char        *fullname;
00133     char        *tablenames;
00134 } DBASE;
00135
00136 /*
00137 typedef int (*SFUN)(char *);
00138 typedef struct compile {
00139     char *keyword;
00140     SFUN func;
00141 } MCFUNCTION;
00142 */
00143 #define TODISK 0
00144 #define FROMDISK 1
00145 #define MDIRTYFLAG 1
00146 #define MCLEANFLAG (~MDIRTYFLAG)
00147 #define INANDOUT 0
00148 #define INPUTONLY 1
00149 #define OUTPUTONLY 2
00150 #define NOCOMPRESS 4
00151
00152 extern int withoutflush;
00153
00154 #endif

```

4.75 model.c

```

00001 /*
00002     #[ Explanations :
00003
00004     This is a second generation much simplified version of model.c

```



```

00005     Syntax:
00006         Model QED;
00007             Particle electron,positron,-2;
00008             Particle photon,+3;
00009             Vertex electron,positron,photon: g;
00010         EndModel;
00011     or:
00012         Model QCD;
00013             Particle quark,antiquark,-2;
00014             Particle ghost,antighost,-1;
00015             Particle gluon,+3;
00016             Vertex quark,antiquark,gluon: g;
00017             Vertex qghost,antighost,gluon: g;
00018             Vertex gluon,gluon,gluon: g;
00019             Vertex gluon,gluon,gluon,gluon: g^2;
00020         EndModel;
00021     Remarks:
00022     1: if no second particle is given, a particle and its antiparticle are the same
00023     2: Spin statistics is by dimension of SU(2) representation with a sign.
00024     3: In a vertex we need to mention the coupling constants.
00025     4: Internally a particle gives a vertex with only two lines.
00026     5: All particles must have been declared before any vertex.
00027
00028     The diagrams should be provided as a collection of nodes and edges
00029     with momenta with a direction substituted. The other parameters (like
00030     Lorentz indices, color indices etc.) can then be provided in a procedure
00031     that should go with this mode definition.
00032     In principle the edges are not needed if there are no 'insertions', but
00033     because we would like to have the insertions as well we need the edges.
00034
00035     #] Explanations :
00036     #[ Includes : model.c
00037 */
00038
00039 #include "form3.h"
00040
00041 /*
00042     #] Includes :
00043     #[ CreateVertex :
00044 */
00045
00046 VERTEX *CreateVertex(MODEL *m)
00047 {
00048     VERTEX *v, **new;
00049     WORD newsize;
00050     int i;
00051     if ( m->invertices >= m->sizevertices ) {
00052         if ( m->sizevertices == 0 ) newsize = 20;
00053         else newsize = 2*m->sizevertices;
00054         new = (VERTEX **)Malloc1(newsize*sizeof(VERTEX *),"m->vertices");
00055         for ( i = 0; i < m->sizevertices; i++ ) new[i] = m->vertices[i];
00056         if ( m->sizevertices > 0 ) M_free(m->vertices,"m->vertices");
00057         m->vertices = new;
00058         m->sizevertices = newsize;
00059     }
00060     v = (VERTEX *)Malloc1(sizeof(VERTEX),"VERTEX");
00061     m->vertices[m->invertices++] = v;
00062     v->nparticles = 0;
00063     v->ncouplings = 0;
00064     v->type = 0;
00065     v->error = 0;
00066     v->externonly = 0;
00067     v->spare = 0;
00068     return(v);
00069 }
00070
00071 /*
00072     #] CreateVertex :
00073     #[ ReadParticle :
00074 */
00075
00076 UBYTE *ReadParticle(UBYTE *s, VERTEX *v, MODEL *m, int par)
00077 {
00078     UBYTE *name, c;
00079     PARTICLE *p = v->particles + v->nparticles++;
00080     WORD type, funnum;
00081     int i, j;
00082     SKIPBLANKS(s)
00083     name = s; s = SkipName(s); c = *s; *s = 0;
00084     if ( GetVar(name,&type,&funnum,CFUNCTION,NOAUTO) == NAMENOTFOUND ) {
00085         p->number = AddFunction(name,0,VERTEXFUNCTION,0,0,0,-1,-1) + FUNCTION;
00086     }
00087     else if ( par == 1 && type == CFUNCTION && functions[funnum].spec == VERTEXFUNCTION ) {
00088         p->number = funnum+FUNCTION;
00089     }
00090     else if ( par == 0 && type == CFUNCTION && functions[funnum].spec == VERTEXFUNCTION ) {
00091         /*

```

```

00092         We should check whether this particle exists already in this model.
00093         If so, this will be an error.
00094     */
00095     for ( i = 0; i < m->nparticles-1; i++ ) {
00096         for ( j = 0; j < m->vertices[i]->nparticles; j++ ) {
00097             if ( m->vertices[i]->particles[j].number == funnum ) {
00098                 MesPrint("&Illegal attempt to redefine a particle in the same model: %s",name);
00099                 v->error = 1;
00100             }
00101         }
00102     }
00103     p->number = funnum+FUNCTION;
00104 }
00105 else {
00106     MesPrint("&Name of particle previously declared as another variable: %s",name);
00107     v->error = 1;
00108 }
00109 *s = c;
00110 while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
00111 return(s);
00112 }
00113
00114 /*
00115 #] ReadParticle :
00116 #[ CoModel :
00117 */
00118
00119 int CoModel(UBYTE *s)
00120 {
00121     UBYTE *name, c;
00122     MODEL *m = (MODEL *)Malloc1(sizeof(MODEL), "Model");
00123     WORD numberofset, *e;
00124     SETS set;
00125     SKIPBLANKS(s)
00126     AC.ModelLevel++;
00127     int i;
00128     /*
00129     We now have to add the model to the list of models.
00130     */
00131     if ( AC.modelspace == 0 || AC.nummodels >= AC.modelspace ) {
00132         int newspace = 2*AC.modelspace+2, i;
00133         MODEL **models = (MODEL **)Malloc1((sizeof(MODEL *))newspace, "AC.models");
00134         for ( i = 0; i < AC.modelspace; i++ ) models[i] = AC.models[i];
00135         if ( AC.models ) M_free(AC.models, "AC.models");
00136         AC.modelspace = newspace;
00137         AC.models = models;
00138     }
00139
00140     AC.models[AC.nummodels++] = m;
00141
00142     m->vertices = 0;
00143     m->couplings = 0;
00144     m->nparticles = m->nvertices = m->sizevertices = m->invertices = 0;
00145     m->sizecouplings = 0;
00146     m->error = 0;
00147     m->grccmodel = NULL;
00148
00149     name = s;
00150     if ( FG.cTable[*s] == 0 ) {
00151         while ( FG.cTable[*s] <= 1 ) s++;
00152         c = *s; *s = 0;
00153         m->name = strDup1(name, "Model name");
00154         *s = c;
00155         SKIPBLANKS(s)
00156         if ( *s != 0 ) {
00157             MesPrint("&Illegal option in model statement: %s",s);
00158             return(1);
00159         }
00160     }
00161     else {
00162         m->name = strDup1((UBYTE *) "---", "Model name");
00163         m->error = 1;
00164         MesPrint("&Illegal name for model: %s",name);
00165         return(1);
00166     }
00167     /*
00168     Now make it a set with one element. The type of the set is rather special
00169     */
00170     if ( GetName(AC.varnames, m->name, &numberofset, NOAUTO) != NAMENOTFOUND ) {
00171         MesPrint("&Name conflict with name %s of model", m->name);
00172         return(1);
00173     }
00174     numberofset = AddSet(name, 0);
00175     set = Sets + numberofset;
00176     set->type = CMODEL;
00177     e = (WORD *)FromVarList(&AC.SetElementList);
00178     *e = AC.nummodels-1;

```

```

00179     set->last++;
00180     AC.SetList.numtemp = AC.SetList.num;
00181     AC.SetElementList.numtemp = AC.SetElementList.num;
00182
00183     for ( i = 0; i <= MAXLEGS; i++ ) m->legcouple[i] = 0;
00184     m->legcouple[2] = 1;
00185
00186     return(0);
00187 }
00188
00189 /*
00190 #] CoModel :
00191 #[ CoParticle :
00192 */
00193
00194 int CoParticle(UBYTE *s)
00195 {
00196     MODEL *m;
00197     VERTEX *v;
00198     int i;
00199     if ( AC.nummodels <= 0 ) {
00200         MesPrint("&No open model for particle statement");
00201         return(1);
00202     }
00203     m = AC.models[AC.nummodels-1];
00204     if ( m->nvertices > 0 ) {
00205         MesPrint("&In a model description Particle statements should be before Vertex statements.");
00206         m->error = 1;
00207         return(1);
00208     }
00209     v = CreateVertex(m);
00210     m->nparticles++;
00211     s = ReadParticle(s,v,m,0);
00212     v->particles[0].type = 1;
00213     while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
00214     if ( v->error == 0 ) {
00215         if ( FG.cTable[*s] == 0 ) {
00216             s = ReadParticle(s,v,m,0);
00217             v->particles[1].type = -1;
00218             if ( v->particles[0].number == v->particles[1].number ) {
00219                 v->particles[0].type = 0;
00220                 v->particles[1].type = 0;
00221             }
00222         }
00223         else { /* Particle = antiparticle */
00224             v->particles[1] = v->particles[0];
00225             v->nparticles++;
00226             v->particles[0].type = 0;
00227             v->particles[1].type = 0;
00228         }
00229     }
00230     if ( v->error == 0 && ( *s == '+' || *s == '-' ) ) {
00231         int x = 0, sign = ( *s == '-' ) ? -1: 1;
00232         s++;
00233         while ( FG.cTable[*s] == 1 ) { x = 10*x + *s++ - '0'; }
00234         if ( x == 0 ) {
00235             v->error = 1;
00236             MesPrint("&Spin goes by dimension of SU(2) representation. Zero is not allowed.");
00237         }
00238         else { v->particles[0].spin = v->particles[1].spin = sign*x; }
00239         SKIPBLANKS(s)
00240     }
00241     else if ( v->error == 0 ) {
00242         v->particles[0].spin = v->particles[1].spin = 1;
00243         v->particles[0].type = 0;
00244         v->particles[1].type = 0;
00245     }
00246 /*
00247 Here will come future options
00248 */
00249 while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
00250 while ( v->error == 0 && *s != 0 && FG.cTable[*s] == 0 ) {
00251     UBYTE *opt = s, c;
00252     while ( FG.cTable[*s] == 0 ) s++;
00253     c = *s; *s = 0;
00254     if ( StrICont(opt,(UBYTE *)"external") == 0 ) { v->externonly = 1; }
00255     else {
00256         MesPrint("&Unrecognized option %s in particle statement.",opt);
00257         v->error = 1;
00258     }
00259     *s = c;
00260     while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
00261 }
00262 /*
00263 The couplings array will be used for registering in what kind of vertices the
00264 particle is involved. (example: in QCD the quarks only in 2 and 3-point vertices)
00265 */

```

```

00266     for ( i = 0; i < MAXPARTICLES; i++ ) v->couplings[i] = 0;
00267     v->couplings[2] = 1;
00268     return(v->error);
00269 }
00270
00271 /*
00272 #] CoParticle :
00273 #[ CoVertex :
00274 */
00275
00276 int CoVertex(UBYTE *s)
00277 {
00278     UBYTE *ss, *name, c;
00279     WORD type;
00280     MODEL *m;
00281     VERTEX *v;
00282     if ( AC.ModelLevel <= 0 ) {
00283         MesPrint("&No open model for vertex statement");
00284         return(1);
00285     }
00286     m = AC.models[AC.nummodels-1];
00287     /*
00288     Get an object of type VERTEX
00289     */
00290     v = CreateVertex(m);
00291     m->nvertices++;
00292     SKIPBLANKS(s);
00293     while ( *s && *s != ':' ) {
00294         if ( v->error == 0 ) {
00295             s = ReadParticle(s,v,m,1);
00296             if ( v->error ) m->error = 1;
00297         }
00298         else {
00299             while ( *s && *s != ':' ) {
00300                 if ( *s == '[' ) { SKIPBRA1(s) }
00301                 else if ( *s == '(' ) { SKIPBRA3(s) }
00302                 else if ( *s == 0 ) {
00303                     v->error = 1;
00304                     MesPrint("&No coupling constant in vertex statement.");
00305                     break;
00306                 }
00307             }
00308             break;
00309         }
00310     }
00311     if ( v->error == 0 ) {
00312         if ( *s == ':' ) { /* read symbols and powers */
00313             s++; SKIPBLANKS(s);
00314             while ( *s ) {
00315                 if ( v->ncouplings >= 2*MAXCOUPLINGS ) {
00316                     MesPrint("&More than %d coupling constants in vertex.",(WORD)MAXCOUPLINGS);
00317                     MesPrint("    Recompile with a larger value for MAXCOUPLINGS.");
00318                     return(1);
00319                 }
00320                 ss = s; s = SkipName(s); c = *s; *s = 0; name = ConstructName(ss,0);
00321                 if ( GetVar(name,&type,&v->couplings[v->ncouplings],CSYMBOL,WITHAUTO)
00322                     == NAMENOTFOUND ) {
00323                     WORD minpow = -MAXPOWER;
00324                     WORD maxpow = MAXPOWER;
00325                     WORD cplx = 0, dim = 0;
00326                     v->couplings[v->ncouplings] = AddSymbol(name,minpow,maxpow,cplx,dim);
00327                 }
00328                 *s = c;
00329                 v->ncouplings++;
00330             } /*
00331             Now possibly a power
00332             */
00333             if ( *s == '^' ) { /* we need an integer */
00334                 WORD x = 0; WORD sign = 1;
00335                 s++;
00336                 while ( *s == '-' || *s == '+' ) {
00337                     if ( *s == '-' ) sign = -sign;
00338                     s++;
00339                 }
00340                 while ( FG.cTable[*s] == 1 ) x = 10*x + *s++ - '0';
00341                 v->couplings[v->ncouplings++] = x;
00342             }
00343             else {
00344                 v->couplings[v->ncouplings++] = 1;
00345             }
00346             SKIPBLANKS(s);
00347             if ( *s == '*' ) { s++; continue; }
00348             if ( *s != ',' ) break;
00349             s++;
00350             SKIPBLANKS(s);
00351         }
00352     }

```

```

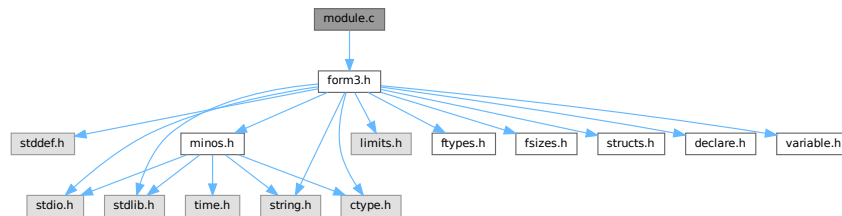
00353     else {
00354         v->error = 1;
00355         MesPrint("%A vertex statement needs at least one coupling constant.");
00356     }
00357 }
00358 if ( v->error == 0 ) {
00359 /*
00360     Register which types of vertices are possible for each particle.
00361 */
00362     int i, j;
00363     for ( i = 0; i < v->nparticles; i++ ) {
00364         for ( j = 0; j < m->nparticles; j++ ) {
00365             if ( m->vertices[j]->particles[0].number == v->particles[i].number ) break;
00366             if ( m->vertices[j]->particles[1].number == v->particles[i].number ) break;
00367         }
00368         m->vertices[j]->couplings[v->nparticles] = 1;
00369     }
00370 }
00371 return(v->error);
00372 }
00373
00374 /*
00375 #] CoVertex :
00376 #[ CoEndModel :
00377 */
00378
00379 int CoEndModel(UBYTE *s)
00380 {
00381     int i, j, k, kk;
00382     WORD csize = 0, *newcouplings, newsize;
00383     MODEL *m;
00384     VERTEX *v;
00385     if ( AC.ModelLevel <= 0 ) {
00386         MesPrint("%EndModel statement without matching Model statement");
00387         return(1);
00388     }
00389     m = AC.models[AC.nummodels-1];
00390 /*
00391     Now create a list of all coupling constants.
00392     Note that we do not expect an astronomical number of them and hence
00393     we use a simple insertion algorithm.
00394 */
00395     m->ncouplings = 0;
00396     for ( i = 0; i < m->nvertices; i++ ) {
00397         v = m->vertices[i+m->nparticles];
00398         m->legcouple[v->nparticles] = 1;
00399         if ( m->ncouplings + v->ncouplings > csize ) {
00400             if ( csize == 0 ) newsize = v->ncouplings + 10;
00401             else newsize = 2*csize;
00402             newcouplings = (WORD *)Malloc1(newsize*sizeof(WORD), "m->couplings");
00403             for ( k = 0; k < m->ncouplings; k++ ) newcouplings[k] = m->couplings[k];
00404             if ( csize > 0 ) M_free(m->couplings, "m->couplings");
00405             m->couplings = newcouplings;
00406             csize = newsize;
00407         }
00408         for ( j = 0; j < v->ncouplings; j += 2 ) {
00409             WORD sym = v->couplings[j];
00410             for ( k = 0; k < m->ncouplings; k++ ) {
00411                 if ( sym == m->couplings[k] ) break;
00412                 if ( sym < m->couplings[k] ) {
00413                     for ( kk = m->ncouplings; kk > k; k-- )
00414                         m->couplings[kk] = m->couplings[kk-1];
00415                     m->couplings[k] = sym;
00416                     m->ncouplings++;
00417                     break;
00418                 }
00419             }
00420             if ( k >= m->ncouplings ) m->couplings[m->ncouplings++] = sym;
00421         }
00422     }
00423     AC.ModelLevel--;
00424     DUMMYUSE(s)
00425     return(LoadModel(m));
00426 }
00427
00428 /*
00429 #] CoEndModel :
00430 */

```

4.76 module.c File Reference

```
#include "form3.h"
```

Include dependency graph for module.c:



Functions

- int [ModuleInstruction](#) (int *moduletype, int *specialtype)
- int [CoModuleOption](#) (UBYTE *s)
- int [CoModOption](#) (UBYTE *s)
- void [SetSpecialMode](#) (int moduletype, int specialtype)
- int [ExecModule](#) (int moduletype)
- int [ExecStore](#) (void)
- void [FullCleanUp](#) (void)
- int [DoPolyfun](#) (UBYTE *s)
- int [DoPolyratfun](#) (UBYTE *s)
- int [DoNoParallel](#) (UBYTE *s)
- int [DoParallel](#) (UBYTE *s)
- int [DoModSum](#) (UBYTE *s)
- int [DoModMax](#) (UBYTE *s)
- int [DoModMin](#) (UBYTE *s)
- int [DoModLocal](#) (UBYTE *s)
- int [DoProcessBucket](#) (UBYTE *s)
- UBYTE * [DoModDollar](#) (UBYTE *s, int type)
- int [DoinParallel](#) (UBYTE *s)
- int [DonotinParallel](#) (UBYTE *s)
- int [DoExecStatement](#) (void)
- int [DoPipeStatement](#) (void)

4.76.1 Detailed Description

A number of routines that deal with the moduleoption statement and the execution of modules. Additionally there are the execution of the exec and pipe instructions.

Definition in file [module.c](#).

4.76.2 Function Documentation

4.76.2.1 ModuleInstruction()

```
int ModuleInstruction (
    int * moduletype,
    int * specialtype )
```

Definition at line 76 of file [module.c](#).

4.76.2.2 CoModuleOption()

```
int CoModuleOption (
    UBYTE * s )
```

Definition at line 143 of file [module.c](#).

4.76.2.3 CoModOption()

```
int CoModOption (
    UBYTE * s )
```

Definition at line 212 of file [module.c](#).

4.76.2.4 SetSpecialMode()

```
void SetSpecialMode (
    int moduletype,
    int specialtype )
```

Definition at line 257 of file [module.c](#).

4.76.2.5 ExecModule()

```
int ExecModule (
    int moduletype )
```

Definition at line 274 of file [module.c](#).

4.76.2.6 ExecStore()

```
int ExecStore (
    void )
```

Definition at line 299 of file [module.c](#).

4.76.2.7 FullCleanUp()

```
void FullCleanUp (
    void )
```

Definition at line [314](#) of file [module.c](#).

4.76.2.8 DoPolyfun()

```
int DoPolyfun (
    UBYTE * s )
```

Definition at line [395](#) of file [module.c](#).

4.76.2.9 DoPolyratfun()

```
int DoPolyratfun (
    UBYTE * s )
```

Definition at line [458](#) of file [module.c](#).

4.76.2.10 DoNoParallel()

```
int DoNoParallel (
    UBYTE * s )
```

Definition at line [529](#) of file [module.c](#).

4.76.2.11 DoParallel()

```
int DoParallel (
    UBYTE * s )
```

Definition at line [544](#) of file [module.c](#).

4.76.2.12 DoModSum()

```
int DoModSum (
    UBYTE * s )
```

Definition at line [559](#) of file [module.c](#).

4.76.2.13 DoModMax()

```
int DoModMax (
    UBYTE * s )
```

Definition at line [579](#) of file [module.c](#).

4.76.2.14 DoModMin()

```
int DoModMin (  
    UBYTE * s )
```

Definition at line 599 of file [module.c](#).

4.76.2.15 DoModLocal()

```
int DoModLocal (  
    UBYTE * s )
```

Definition at line 619 of file [module.c](#).

4.76.2.16 DoProcessBucket()

```
int DoProcessBucket (  
    UBYTE * s )
```

Definition at line 639 of file [module.c](#).

4.76.2.17 DoModDollar()

```
UBYTE * DoModDollar (  
    UBYTE * s,  
    int type )
```

Definition at line 657 of file [module.c](#).

4.76.2.18 DoinParallel()

```
int DoinParallel (  
    UBYTE * s )
```

Definition at line 758 of file [module.c](#).

4.76.2.19 DonotinParallel()

```
int DonotinParallel (  
    UBYTE * s )
```

Definition at line 768 of file [module.c](#).

4.76.2.20 DoExecStatement()

```
int DoExecStatement (  
    void )
```

Definition at line 780 of file [module.c](#).

4.76.2.21 DoPipeStatement()

```
int DoPipeStatement (
    void )
```

Definition at line 797 of file [module.c](#).

4.77 module.c

[Go to the documentation of this file.](#)

```
00001
00007 /* #[] License : */
00008 /*
00009 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00010 * When using this file you are requested to refer to the publication
00011 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00012 * This is considered a matter of courtesy as the development was paid
00013 * for by FOM the Dutch physics granting agency and we would like to
00014 * be able to track its scientific use to convince FOM of its value
00015 * for the community.
00016 *
00017 * This file is part of FORM.
00018 *
00019 * FORM is free software: you can redistribute it and/or modify it under the
00020 * terms of the GNU General Public License as published by the Free Software
00021 * Foundation, either version 3 of the License, or (at your option) any later
00022 * version.
00023 *
00024 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00025 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00026 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00027 * details.
00028 *
00029 * You should have received a copy of the GNU General Public License along
00030 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00031 */
00032 /* #[] License : */
00033 /*
00034 * #[] Includes :
00035 */
00036
00037 #include "form3.h"
00038
00039 /*
00040 * #[] Includes :
00041 * #[] Modules :
00042 * #[] ModuleInstruction :
00043 *
00044 * Enters after the . of .sort etc
00045 * We have word[[(options)][commentary];]
00046 * Word is one of 'clear end global sort store'
00047 * Options are 'polyfun, endloopifunchanged, endloopifzero'
00048 * Additions for solving equations can be added to 'word'
00049 *
00050 * The flag in the moduleoptions is for telling whether the current
00051 * option can be followed by others. If it is > 0 it cannot.
00052 */
00053
00054 static KEYWORD ModuleWords[] = {
00055     {"clear", (TFUN)0, CLEARMODULE, 0}
00056     , {"end", (TFUN)0, ENDMODULE, 0}
00057     , {"global", (TFUN)0, GLOBALMODULE, 0}
00058     , {"sort", (TFUN)0, SORTMODULE, 0}
00059     , {"store", (TFUN)0, STOREMODULE, 0}
00060 };
00061
00062 static KEYWORD ModuleOptions[] = {
00063     {"inparallel", DoinParallel, 1, 1}
00064     , {"local", DoModLocal, MODLOCAL, 0}
00065     , {"maximum", DoModMax, MODMAX, 0}
00066     , {"minimum", DoModMin, MODMIN, 0}
00067     , {"noparallel", DoNoParallel, NOPARALLEL_USER, 0}
00068     , {"notinparallel", DonotinParallel, 0, 1}
00069     , {"parallel", DoParallel, PARALLELFLAG, 0}
00070     , {"polyfun", DoPolyfun, POLYFUN, 0}
00071     , {"polyratfun", DoPolyratfun, POLYFUN, 0}
00072     , {"processbucketsize", DoProcessBucket, 0, 0}
00073     , {"sum", DoModSum, MODSUM, 0}
```

```

00074 };
00075
00076 int ModuleInstruction(int *moduletype, int *specialtype)
00077 {
00078     UBYTE *t, *s, *u, c;
00079     KEYWORD *key;
00080     int addit = 0, error = 0, i, j;
00081     DUMMYUSE(specialtype);
00082     LoadInstruction(0);
00083     AC.firstctypemessage = 0;
00084     s = AP.preStart; SKIPBLANKS(s)
00085     t = EndOfToken(s); c = *t; *t = 0;
00086     AC.origin = FROMPOINTINSTRUCTION;
00087     key = FindKeyWord(AP.preStart, ModuleWords, sizeof(ModuleWords)/sizeof(KEYWORD));
00088     if ( key == 0 ) {
00089         MesPrint("@Unrecognized module terminator: %s",s);
00090         error = 1;
00091         key = ModuleWords;
00092         while ( StrCmp((UBYTE *)key->name, (UBYTE *)"end") ) key++;
00093     }
00094     *t = c;
00095     *moduletype = key->type;
00096     SKIPBLANKS(t);
00097     while ( *t == '(' ) { /* There are options */
00098         s = t+1; SKIPBRA3(t)
00099         if ( *t == 0 ) {
00100             MesPrint("@Improper options field in . instruction");
00101             error = 1;
00102         }
00103         else {
00104             *t = 0;
00105             if ( CoModOption(s) ) error = 1;
00106             *t++ = ')';
00107         }
00108     }
00109     if ( *t == ':' ) { /* There is an 'advertisement' */
00110         t++;
00111         SKIPBLANKS(t)
00112         s = t; i = 0;
00113         while ( *t && *t != ';' ) {
00114             if ( *t == '\\\ ' ) t++;
00115             t++; i++;
00116         }
00117         u = t;
00118         while ( u > s && u[-1] == ' ' ) { u--; i--; }
00119         if ( *u == '\\\ ' ) { u++; i++; }
00120         for ( j = COMMERCIALSIZE-1; j >= 0; j-- ) {
00121             if ( i <= 0 ) break;
00122             AC.Commercial[j] = *--u; i--;
00123             if ( u > s && u[-1] == '\\\ ' ) u--;
00124         }
00125         for ( ; j >= 0; j-- ) AC.Commercial[j] = ' ';
00126         AC.Commercial[COMMERCIALSIZE] = 0;
00127         addit += 2;
00128     }
00129     if ( addit && *t != ';' ) {
00130         MesPrint("@Improper ending of . instruction");
00131         error = -1;
00132     }
00133     return(error);
00134 }
00135
00136 /*
00137     #] ModuleInstruction :
00138     #[ CoModuleOption :
00139
00140     ModuleOption, options;
00141 */
00142
00143 int CoModuleOption(UBYTE *s)
00144 {
00145     UBYTE *t, *ttt, c;
00146     KEYWORD *option;
00147     int error = 0, polyflag = 0;
00148     AC.origin = FROMMODULEOPTION;
00149     if ( *s ) do {
00150         s = ToToken(s);
00151         t = EndOfToken(s);
00152         c = *t; *t = 0;
00153         option = FindKeyWord(s, ModuleOptions,
00154             sizeof(ModuleOptions)/sizeof(KEYWORD));
00155         if ( option == 0 ) {
00156             if ( polyflag ) {
00157                 *t = c; t++; s = SkipAName(t);
00158                 polyflag = 0;
00159                 continue;
00160             }

```

```

00161         else {
00162             MesPrint("@Unrecognized module option: %s",s);
00163             error = 1;
00164             polyflag = 0;
00165             *t = c;
00166         }
00167     }
00168     else {
00169         *t = c;
00170         SKIPBLANKS(t)
00171         if ( (option->func)(t) ) error = 1;
00172     }
00173     if ( StrCmp((UBYTE *) (option->name), (UBYTE *) ("polyfun")) == 0
00174         || StrCmp((UBYTE *) (option->name), (UBYTE *) ("polyratfun")) == 0 ) {
00175         polyflag = 1;
00176         t++;
00177         t = SkipAName(t);
00178     }
00179     else polyflag = 0;
00180     if ( option->flags > 0 ) return(error);
00181     while ( *t ) {
00182         if ( *t == ',' ) {
00183             tt = t+1;
00184             while ( *tt == ',' ) tt++;
00185             if ( *tt != '$' ) break;
00186             t = tt+1;
00187         }
00188         if ( *t == ')' ) break;
00189         if ( *t == '(' ) SKIPBRA3(t)
00190         else if ( *t == '{' ) SKIPBRA2(t)
00191         else if ( *t == '[' ) SKIPBRA1(t)
00192         t++;
00193     }
00194     s = t;
00195 } while ( *s == ',' );
00196 if ( *s ) {
00197     MesPrint("@Unrecognized module option: %s",s);
00198     error = 1;
00199 }
00200 return(error);
00201 }
00202
00203 /*
00204     #] CoModuleOption :
00205     #[ CoModOption :
00206
00207     To be called from a .instruction.
00208     Only recognizes polyfun. The newer ones should be via the
00209     ModuleOption statement.
00210 */
00211
00212 int CoModOption(UBYTE *s)
00213 {
00214     UBYTE *t,c;
00215     int error = 0;
00216     AC.origin = FROMPOINTINSTRUCTION;
00217     if ( *s ) do {
00218         s = ToToken(s);
00219         t = EndOfToken(s);
00220         c = *t; *t = 0;
00221         if ( StrICmp(s, (UBYTE *) "polyfun") == 0 ) {
00222             *t = c;
00223             SKIPBLANKS(t)
00224             if ( DoPolyfun(t) ) error = 1;
00225         }
00226         else if ( StrICmp(s, (UBYTE *) "polyratfun") == 0 ) {
00227             *t = c;
00228             SKIPBLANKS(t)
00229             if ( DoPolyratfun(t) ) error = 1;
00230         }
00231         else {
00232             MesPrint("@Unrecognized module option in .instruction: %s",s);
00233             error = 1;
00234             *t = c;
00235         }
00236         while ( *t ) {
00237             if ( *t == ',' || *t == ')' ) break;
00238             if ( *t == '(' ) SKIPBRA3(t)
00239             else if ( *t == '{' ) SKIPBRA2(t)
00240             else if ( *t == '[' ) SKIPBRA1(t)
00241             t++;
00242         }
00243         s = t;
00244     } while ( *s == ',' );
00245     if ( *s ) {
00246         MesPrint("@Unrecognized module option in .instruction: %s",s);
00247         error = 1;

```

```

00248     }
00249     return(error);
00250 }
00251
00252 /*
00253     #] CoModOption :
00254     #[ SetSpecialMode :
00255 */
00256
00257 void SetSpecialMode(int moduletype, int specialtype)
00258 {
00259     DUMMYUSE(moduletype); DUMMYUSE(specialtype);
00260 }
00261
00262 /*
00263     #] SetSpecialMode :
00264     #[ MakeGlobal :
00265 */
00266 void MakeGlobal()
00267 {
00268 }
00269
00270     #] MakeGlobal :
00271     #[ ExecModule :
00272 */
00273
00274 int ExecModule(int moduletype)
00275 {
00276     /*
00277     Here we check module options and other things that should
00278     be done at the start of a module.
00279     */
00280     #ifndef WITHFLOAT
00281         if ( ( AC.tMaxWeight != 0 && AC.tMaxWeight != AC.MaxWeight )
00282             || ( AC.tDefaultPrecision != 0 && AC.tDefaultPrecision != AC.DefaultPrecision ) ) {
00283             AC.DefaultPrecision = AC.tDefaultPrecision;
00284             AC.tDefaultPrecision = 0;
00285             AC.MaxWeight = AC.tMaxWeight;
00286             AC.tMaxWeight = 0;
00287             ClearMZVTables();
00288             SetupMZVTables();
00289         }
00290     #endif
00291     return(DoExecute(moduletype,0));
00292 }
00293
00294 /*
00295     #] ExecModule :
00296     #[ ExecStore :
00297 */
00298
00299 int ExecStore(void)
00300 {
00301     return(0);
00302 }
00303
00304 /*
00305     #] ExecStore :
00306     #[ FullCleanUp :
00307 */
00308     Remark 27-oct-2005 by JV
00309     This routine (and CleanUp in startup.c) may still need some work:
00310         What to do with preprocessor variables
00311         What to do with files we write to
00312 */
00313
00314 void FullCleanUp(void)
00315 {
00316     int j;
00317
00318     while ( AC.CurrentStream->previous >= 0 )
00319         AC.CurrentStream = CloseStream(AC.CurrentStream);
00320     AP.PreSwitchLevel = AP.PreIfLevel = 0;
00321
00322     for ( j = NumProcedures-1; j >= 0; j-- ) {
00323         if ( Procedures[j].name ) M_free(Procedures[j].name,"name of procedure");
00324         if ( Procedures[j].p.buffer ) M_free(Procedures[j].p.buffer,"buffer of procedure");
00325     }
00326     NumProcedures = 0;
00327
00328     while ( NumPre > AP.gNumPre ) {
00329         NumPre--;
00330         M_free(PreVar[NumPre].name,"PreVar[NumPre].name");
00331         PreVar[NumPre].name = PreVar[NumPre].value = 0;
00332     }
00333
00334     AC.DidClean = 0;

```

```

00335     for ( j = 0; j < NumExpressions; j++ ) {
00336         AC.exprnames->namenode[Expressions[j].node].type = CDELETE;
00337         AC.DidClean = 1;
00338     }
00339
00340     CompactifyTree(AC.exprnames,EXPRNAMES);
00341
00342     for ( j = AO.NumDictionaries-1; j >= 0; j-- ) {
00343         RemoveDictionary(AO.Dictionaries[j]);
00344         M_free(AO.Dictionaries[j],"Dictionary");
00345     }
00346     AO.NumDictionaries = AO.gNumDictionaries = 0;
00347     M_free(AO.Dictionaries,"Dictionaries");
00348     AO.Dictionaries = 0;
00349     AO.SizeDictionaries = 0;
00350     AP.OpenDictionary = 0;
00351     AO.CurrentDictionary = 0;
00352
00353     AP.ComChar = AP.cComChar;
00354     if ( AP.procedureExtension ) M_free(AP.procedureExtension,"procedureextension");
00355     AP.procedureExtension = strdup(AP.cprocedureExtension,"procedureextension");
00356
00357     AC.StatsFlag = AM.gStatsFlag = AM.ggStatsFlag;
00358     AC.extrasymbols = AM.gextrasymbols = AM.ggextrasymbols;
00359     AC.extrasym[0] = AM.gextrasym[0] = AM.ggextrasym[0] = 'Z';
00360     AC.extrasym[1] = AM.gextrasym[1] = AM.ggextrasym[1] = 0;
00361     AC.NoSpacesInNumbers = AM.gNoSpacesInNumbers = AM.ggNoSpacesInNumbers;
00362     AO.IndentSpace = AM.gIndentSpace = AM.ggIndentSpace;
00363     AC.ThreadStats = AM.gThreadStats = AM.ggThreadStats;
00364     AC.OldFactArgFlag = AM.gOldFactArgFlag = AM.ggOldFactArgFlag;
00365     AC.FinalStats = AM.gFinalStats = AM.ggFinalStats;
00366     AC.OldGCDflag = AM.gOldGCDflag = AM.ggOldGCDflag;
00367     AC.ThreadsFlag = AM.gThreadsFlag = AM.ggThreadsFlag;
00368     if ( AC.ThreadsFlag && AM.totalnumberofthreads > 1 ) AS.MultiThreaded = 1;
00369     AC.ThreadBucketSize = AM.gThreadBucketSize = AM.ggThreadBucketSize;
00370     AC.ThreadBalancing = AM.gThreadBalancing = AM.ggThreadBalancing;
00371     AC.ThreadSortFileSynch = AM.gThreadSortFileSynch = AM.ggThreadSortFileSynch;
00372     AC.ShortStatsMax = AM.gShortStatsMax = AM.ggShortStatsMax;
00373     AC.SizeCommuteInSet = AM.gSizeCommuteInSet = 0;
00374 #ifdef WITHFLOAT
00375     AC.MaxWeight = AM.gMaxWeight = AM.ggMaxWeight;
00376     AC.DefaultPrecision = AM.gDefaultPrecision = AM.ggDefaultPrecision;
00377 #endif
00378
00379     NumExpressions = 0;
00380     if ( DeleteStore(0) < 0 ) {
00381         MesPrint("@Cannot restart the storage file");
00382         Terminate(-1);
00383     }
00384     RemoveDollars();
00385     CleanUp(1);
00386     ResetVariables(2);
00387     IniVars();
00388 }
00389
00390 /*
00391     #] FullCleanUp :
00392     #[ DoPolyfun :
00393 */
00394
00395 int DoPolyfun(UBYTE *s)
00396 {
00397     GETIDENTITY
00398     UBYTE *t, c;
00399     WORD funnum, eqsign = 0;
00400     if ( AC.origin == FROMPOINTINSTRUCTION ) {
00401         if ( *s == 0 || *s == ',' || *s == ')' ) {
00402             AR.PolyFun = 0; AR.PolyFunType = 0;
00403             return(0);
00404         }
00405         if ( *s != '=' ) {
00406             MesPrint("@Proper use in point instructions is: PolyFun[=functionname]");
00407             return(-1);
00408         }
00409         eqsign = 1;
00410     }
00411     else {
00412         if ( *s == 0 ) {
00413             AR.PolyFun = 0; AR.PolyFunType = 0;
00414             return(0);
00415         }
00416         if ( *s != '=' && *s != ',' ) {
00417             MesPrint("@Proper use is: PolyFun[{ ,= }functionname]");
00418             return(-1);
00419         }
00420         if ( *s == '=' ) eqsign = 1;
00421     }

```

```

00422     s++;
00423     SKIPBLANKS(s)
00424     t = EndOfToken(s);
00425     c = *t; *t = 0;
00426
00427     if ( GetName(AC.varnames,s,&funnum,WITHAUTO) != CFUNCTION ) {
00428         if ( AC.origin != FROMPOINTINSTRUCTION && eqsign == 0 ) {
00429             AR.PolyFun = 0; AR.PolyFunType = 0;
00430             return(0);
00431         }
00432         MesPrint("@ %s is not a properly declared function",s);
00433         *t = c;
00434         return(-1);
00435     }
00436     if ( functions[funnum].spec > 0 || functions[funnum].commute != 0 ) {
00437         MesPrint("@The PolyFun must be a regular commuting function!");
00438         *t = c;
00439         return(-1);
00440     }
00441     AR.PolyFun = funnum+FUNCTION; AR.PolyFunType = 1;
00442     *t = c;
00443     SKIPBLANKS(t)
00444     if ( *t && *t != ',' && *t != ')' ) {
00445         t++; c = *t; *t = 0;
00446         MesPrint("@Improper ending of end-of-module instruction: %s",s);
00447         *t = c;
00448         return(-1);
00449     }
00450     return(0);
00451 }
00452
00453 /*
00454     #] DoPolyfun :
00455     #[ DoPolyratfun :
00456 */
00457
00458 int DoPolyratfun(UBYTE *s)
00459 {
00460     GETIDENTITY
00461     UBYTE *t, c;
00462     WORD funnum;
00463     if ( AC.origin == FROMPOINTINSTRUCTION ) {
00464         if ( *s == 0 || *s == ',' || *s == ')' ) {
00465             AR.PolyFun = 0; AR.PolyFunType = 0; AR.PolyFunInv = 0; AR.PolyFunExp = 0;
00466             return(0);
00467         }
00468         if ( *s != '=' ) {
00469             MesPrint("@Proper use in point instructions is:
PolyRatFun[=functionname[+functionname]]");
00470             return(-1);
00471         }
00472     }
00473     else {
00474         if ( *s == 0 ) {
00475             AR.PolyFun = 0; AR.PolyFunType = 0; AR.PolyFunInv = 0; AR.PolyFunExp = 0;
00476             return(0);
00477         }
00478         if ( *s != '=' && *s != ',' ) {
00479             MesPrint("@Proper use is: PolyRatFun[{ ,= }functionname[+functionname]]");
00480             return(-1);
00481         }
00482     }
00483     s++;
00484     SKIPBLANKS(s)
00485     t = EndOfToken(s);
00486     c = *t; *t = 0;
00487
00488     if ( GetName(AC.varnames,s,&funnum,WITHAUTO) != CFUNCTION ) {
00489 Error1:;
00490         MesPrint("@ %s is not a properly declared function",s);
00491         *t = c;
00492         return(-1);
00493     }
00494     if ( functions[funnum].spec > 0 || functions[funnum].commute != 0 ) {
00495 Error2:;
00496         MesPrint("@The PolyRatFun must be a regular commuting function!");
00497         *t = c;
00498         return(-1);
00499     }
00500     AR.PolyFun = funnum+FUNCTION; AR.PolyFunType = 2;
00501     AR.PolyFunInv = 0;
00502     AR.PolyFunExp = 0;
00503     AC.PolyRatFunChanged = 1;
00504     *t = c;
00505     if ( *t == '+' ) {
00506         t++; s = t;
00507         t = EndOfToken(s);

```

```

00508         c = *t; *t = 0;
00509         if ( GetName(AC.varnames,s,&funnum,WITHAUTO) != CFUNCTION ) goto Error1;
00510         if ( functions[funnum].spec > 0 || functions[funnum].commute != 0 ) goto Error2;
00511         AR.PolyFunInv = funnum+FUNCTION;
00512         *t = c;
00513     }
00514     SKIPBLANKS(t)
00515     if ( *t && *t != ',' && *t != ')' ) {
00516         t++; c = *t; *t = 0;
00517         MesPrint("@Improper ending of end-of-module instruction: %s",s);
00518         *t = c;
00519         return(-1);
00520     }
00521     return(0);
00522 }
00523
00524 /*
00525     #] DoPolyratfun :
00526     #[ DoNoParallel :
00527 */
00528
00529 int DoNoParallel(UBYTE *s)
00530 {
00531     if ( *s == 0 || *s == ',' || *s == ')' ) {
00532         AC.mparallelfldflag |= NOPARALLEL_USER;
00533         return(0);
00534     }
00535     MesPrint("@NoParallel should not have extra parameters");
00536     return(-1);
00537 }
00538
00539 /*
00540     #] DoNoParallel :
00541     #[ DoParallel :
00542 */
00543
00544 int DoParallel(UBYTE *s)
00545 {
00546     if ( *s == 0 || *s == ',' || *s == ')' ) {
00547         AC.mparallelfldflag &= ~NOPARALLEL_USER;
00548         return(0);
00549     }
00550     MesPrint("@Parallel should not have extra parameters");
00551     return(-1);
00552 }
00553
00554 /*
00555     #] DoParallel :
00556     #[ DoModSum :
00557 */
00558
00559 int DoModSum(UBYTE *s)
00560 {
00561     while ( *s == ',' ) s++;
00562     if ( *s != '$' ) {
00563         MesPrint("@Module Sum should mention which $-variables");
00564         return(-1);
00565     }
00566     s = DoModDollar(s,MODSUM);
00567     if ( s && *s != 0 && *s != ')' ) {
00568         MesPrint("@Irregular end of Sum option of Module statement");
00569         return(-1);
00570     }
00571     return(0);
00572 }
00573
00574 /*
00575     #] DoModSum :
00576     #[ DoModMax :
00577 */
00578
00579 int DoModMax(UBYTE *s)
00580 {
00581     while ( *s == ',' ) s++;
00582     if ( *s != '$' ) {
00583         MesPrint("@Module Maximum should mention which $-variables");
00584         return(-1);
00585     }
00586     s = DoModDollar(s,MODMAX);
00587     if ( s && *s != 0 ) {
00588         MesPrint("@Irregular end of Maximum option of Module statement");
00589         return(-1);
00590     }
00591     return(0);
00592 }
00593
00594 /*

```



```

00595         #] DoModMax :
00596         #[ DoModMin :
00597     */
00598
00599 int DoModMin(UBYTE *s)
00600 {
00601     while ( *s == ',' ) s++;
00602     if ( *s != '$' ) {
00603         MesPrint("@Module Minimum should mention which $-variables");
00604         return(-1);
00605     }
00606     s = DoModDollar(s,MODMIN);
00607     if ( s && *s != 0 ) {
00608         MesPrint("@Irregular end of Minimum option of Module statement");
00609         return(-1);
00610     }
00611     return(0);
00612 }
00613
00614 /*
00615         #] DoModMin :
00616         #[ DoModLocal :
00617     */
00618
00619 int DoModLocal(UBYTE *s)
00620 {
00621     while ( *s == ',' ) s++;
00622     if ( *s != '$' ) {
00623         MesPrint("@ModuleOption Local should mention which $-variables");
00624         return(-1);
00625     }
00626     s = DoModDollar(s,MODLOCAL);
00627     if ( s && *s != 0 ) {
00628         MesPrint("@Irregular end of Local option of ModuleOption statement");
00629         return(-1);
00630     }
00631     return(0);
00632 }
00633
00634 /*
00635         #] DoModLocal :
00636         #[ DoProcessBucket :
00637     */
00638
00639 int DoProcessBucket(UBYTE *s)
00640 {
00641     LONG x;
00642     while ( *s == ',' || *s == '=' ) s++;
00643     ParseNumber(x,s)
00644     if ( *s && *s != ' ' && *s != '\t' ) {
00645         MesPrint("&Numerical value expected for ProcessBucketSize");
00646         return(1);
00647     }
00648     AC.mProcessBucketSize = x;
00649     return(0);
00650 }
00651
00652 /*
00653         #] DoProcessBucket :
00654         #[ DoModDollar :
00655     */
00656
00657 UBYTE * DoModDollar(UBYTE *s, int type)
00658 {
00659     UBYTE *name, c;
00660     WORD number;
00661     MODOPTDOLLAR *md;
00662     while ( *s == '$' ) {
00663     /*
00664         Read the name of the dollar
00665         Mark the type
00666     */
00667         s++;
00668         name = s;
00669         if ( FG.cTable[*s] == 0 ) {
00670             while ( FG.cTable[*s] == 0 || FG.cTable[*s] == 1 ) s++;
00671             c = *s; *s = 0;
00672             number = GetDollar(name);
00673             if ( number < 0 ) {
00674                 number = AddDollar(s,0,0,0);
00675                 Warning("&Undefined $-variable in module statement");
00676             }
00677             md = (MODOPTDOLLAR *)FromList(&AC.ModOptDolList);
00678             md->number = number;
00679             md->type = type;
00680 #ifdef WITHPTHREADS
00681             if ( type == MODLOCAL ) {

```

```

00682         int j, i;
00683         DOLLARS dglobal, dlocal;
00684         md->dstruct = (DOLLARS)Malloc1(
00685             sizeof(struct DoLLaRs)*AM.totalnumberofthreads,"Local DOLLARS");
00686     /*
00687         Now copy the global dollar into the local copies.
00688         This can be nontrivial if the value needs an allocation.
00689         We don't really need the locks.
00690     */
00691     dglobal = Dollars + number;
00692     for ( j = 0; j < AM.totalnumberofthreads; j++ ) {
00693         dlocal = md->dstruct + j;
00694         dlocal->index = dglobal->index;
00695         dlocal->node = dglobal->node;
00696         dlocal->type = dglobal->type;
00697         dlocal->name = dglobal->name;
00698         dlocal->size = dglobal->size;
00699         dlocal->where = dglobal->where;
00700         if ( dlocal->size > 0 ) {
00701             dlocal->where = (WORD *)Malloc1((dlocal->size+1)*sizeof(WORD),"Local dollar
value");
00702             for ( i = 0; i < dlocal->size; i++ )
00703                 dlocal->where[i] = dglobal->where[i];
00704             dlocal->where[dlocal->size] = 0;
00705         }
00706         dlocal->pthreadlockread = dummylock;
00707         dlocal->pthreadlockwrite = dummylock;
00708         dlocal->nfactors = dglobal->nfactors;
00709         if ( dglobal->nfactors > 1 ) {
00710             int nsize;
00711             WORD *t, *m;
00712             dlocal->factors = (FACDOLLAR
*)Malloc1(dglobal->nfactors*sizeof(FACDOLLAR),"Dollar factors");
00713             for ( i = 0; i < dglobal->nfactors; i++ ) {
00714                 nsize = dglobal->factors[i].size;
00715                 dlocal->factors[i].type = DOLUNDEFINED;
00716                 dlocal->factors[i].value = dglobal->factors[i].value;
00717                 if ( ( dlocal->factors[i].size = nsize ) > 0 ) {
00718                     dlocal->factors[i].where = t = (WORD
*)Malloc1(sizeof(WORD)*(nsize+1),"DollarCopyFactor");
00719                     m = dglobal->factors[i].where;
00720                     NCOPY(t,m,nsize);
00721                     *t = 0;
00722                 }
00723                 else {
00724                     dlocal->factors[i].where = 0;
00725                 }
00726             }
00727         }
00728         else { dlocal->factors = 0; }
00729     }
00730 }
00731 else {
00732     md->dstruct = 0;
00733 }
00734 #endif
00735     *s = c;
00736 }
00737 else {
00738     MesPrint("&Illegal name for $-variable in module option");
00739     while ( *s != ',' && *s != 0 && *s != ')' ) s++;
00740 }
00741 while ( *s == ',' ) s++;
00742 }
00743 return(s);
00744 }
00745
00746 /*
00747     #] DoModDollar :
00748     #[ DoinParallel :
00749
00750     The idea is that we should have the commands
00751     ModuleOption,InParallel;
00752     ModuleOption,InParallel,name1,name2,...,namen;
00753     ModuleOption,NotInParallel,name1,name2,...,namen;
00754     The advantage over the InParallel statement is that this statement
00755     comes after the definition of the expressions.
00756 */
00757
00758 int DoinParallel(UBYTE *s)
00759 {
00760     return(DoInParallel(s,1));
00761 }
00762
00763 /*
00764     #] DoinParallel :
00765     #[ DonotinParallel :

```

```

00766 */
00767
00768 int DonotinParallel(UBYTE *s)
00769 {
00770     return(DoInParallel(s,0));
00771 }
00772
00773 /*
00774     #] DonotinParallel :
00775     #] Modules :
00776     #[ External :
00777     #[ DoExecStatement :
00778 */
00779
00780 int DoExecStatement(void)
00781 {
00782     #ifdef WITHSYSTEM
00783         FLUSHCONSOLE;
00784         if ( system((char *) (AP.preStart)) ) return(-1);
00785         return(0);
00786     #else
00787         Error0("External programs not implemented on this computer/system");
00788         return(-1);
00789     #endif
00790 }
00791
00792 /*
00793     #] DoExecStatement :
00794     #[ DoPipeStatement :
00795 */
00796
00797 int DoPipeStatement(void)
00798 {
00799     #ifdef WITHPIPE
00800         FLUSHCONSOLE;
00801         if ( OpenStream(AP.preStart,PIPESTREAM,0,PREENOACTION) == 0 ) return(-1);
00802         return(0);
00803     #else
00804         Error0("Pipes not implemented on this computer/system");
00805         return(-1);
00806     #endif
00807 }
00808
00809 /*
00810     #] DoPipeStatement :
00811     #] External :
00812 */

```

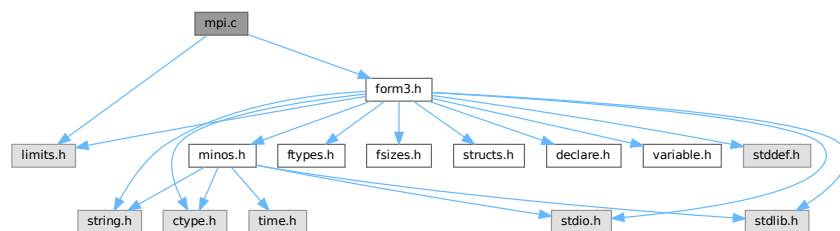
4.78 mpi.c File Reference

```

#include <limits.h>
#include "form3.h"

```

Include dependency graph for mpi.c:



Data Structures

- struct [longMultiStruct](#)

Macros

- `#define PF_PACKSIZE 1600`
- `#define MPI_ERRCODE_CHECK(err)`

Typedefs

- `typedef struct longMultiStruct PF_LONGMULTI`

Functions

- `LONG PF_RealTime (int i)`
- `int PF_LibInit (int *argcp, char ***argvp)`
- `int PF_LibTerminate (int error)`
- `int PF_Probe (int *src)`
- `int PF_ISendSbuf (int to, int tag)`
- `int PF_RecvWbuf (WORD *b, LONG *s, int *src)`
- `int PF_IRecvRbuf (PF_BUFFER *r, int bn, int from)`
- `int PF_WaitRbuf (PF_BUFFER *r, int bn, LONG *size)`
- `int PF_Bcast (void *buffer, int count)`
- `int PF_RawSend (int dest, void *buf, LONG l, int tag)`
- `LONG PF_RawRecv (int *src, void *buf, LONG thesize, int *tag)`
- `int PF_RawProbe (int *src, int *tag, int *bytesize)`
- `int PF_PrintPackBuf (char *s, int size)`
- `int PF_PreparePack (void)`
- `int PF_Pack (const void *buffer, size_t count, MPI_Datatype type)`
- `int PF_Unpack (void *buffer, size_t count, MPI_Datatype type)`
- `int PF_PackString (const UBYTE *str)`
- `int PF_UnpackString (UBYTE *str)`
- `int PF_Send (int to, int tag)`
- `int PF_Receive (int src, int tag, int *psrc, int *ptag)`
- `int PF_Broadcast (void)`
- `int PF_PrepareLongSinglePack (void)`
- `int PF_LongSinglePack (const void *buffer, size_t count, MPI_Datatype type)`
- `int PF_LongSingleUnpack (void *buffer, size_t count, MPI_Datatype type)`
- `int PF_LongSingleSend (int to, int tag)`
- `int PF_LongSingleReceive (int src, int tag, int *psrc, int *ptag)`
- `int PF_PrepareLongMultiPack (void)`
- `int PF_LongMultiPackImpl (const void *buffer, size_t count, size_t eSize, MPI_Datatype type)`
- `int PF_LongMultiUnpackImpl (void *buffer, size_t count, size_t eSize, MPI_Datatype type)`
- `int PF_LongMultiBroadcast (void)`

Variables

- `LONG PF_maxDollarChunkSize = 0`

4.78.1 Detailed Description

MPI dependent functions of parform

This file contains all the functions for the parallel version of form3 that explicitly need to call mpi routines. This is the only file that really needs to be linked to the mpi-library!

Definition in file [mpi.c](#).

4.78.2 Macro Definition Documentation

4.78.2.1 PF_PACKSIZE

```
#define PF_PACKSIZE 1600
```

Definition at line 58 of file [mpi.c](#).

4.78.2.2 MPI_ERRCODE_CHECK

```
#define MPI_ERRCODE_CHECK(  
    err )
```

Value:

```
do { \
    int _tmp_err = (err); \
    if ( _tmp_err != MPI_SUCCESS ) return _tmp_err != 0 ? _tmp_err : -1; \
} while (0)
```

A macro which exits the caller with a non-zero return value if *err* is not MPI_SUCCESS.

Parameters

<i>err</i>	The return value of a MPI function to be checked.
------------	---

Remarks

The MPI standard defines MPI_SUCCESS == 0. Then (_tmp_err == 0) appears twice and we can expect the second evaluation will be eliminated by the compiler optimization.

Definition at line 84 of file [mpi.c](#).

4.78.3 Function Documentation

4.78.3.1 PF_RealTime()

```
LONG PF_RealTime (  
    int i )
```

Returns the realtime in 1/100 sec. as a LONG.

Parameters

<i>i</i>	the timer will be reset if i == 0.
----------	------------------------------------

Returns

the real elapsed time in 1/100 second.

Definition at line 101 of file [mpi.c](#).

Referenced by [PF_Init\(\)](#).

4.78.3.2 PF_LibInit()

```
int PF_LibInit (
    int * argcp,
    char *** argvp )
```

Performs all library dependent initializations.

Parameters

<i>argcp</i>	pointer to the number of arguments.
<i>argvp</i>	pointer to the arguments.

Returns

0 if OK, nonzero on error.

Definition at line 123 of file [mpi.c](#).

Referenced by [PF_Init\(\)](#).

4.78.3.3 PF_LibTerminate()

```
int PF_LibTerminate (
    int error )
```

Exits mpi, when there is an error either indicated or happening, returnvalue is 1, else returnvalue is 0.

Parameters

<i>error</i>	an error code.
--------------	----------------

Returns

0 if OK, nonzero on error.

Definition at line 209 of file [mpi.c](#).

Referenced by [PF_Terminate\(\)](#).

4.78.3.4 PF_Probe()

```
int PF_Probe (
    int * src )
```

Probes the next incoming message. If *src* == PF_ANY_SOURCE this operation is blocking, otherwise nonblocking.

Parameters

<i>in, out</i>	<i>src</i>	the source process number. In output, the process number of actual found source.
----------------	------------	--

Returns

the tag value of the next incoming message if found, 0 if a nonblocking probe (input *src* != PF_ANY_SOURCE) did not find any messages. A negative returned value indicates an error.

Definition at line 230 of file [mpi.c](#).

4.78.3.5 PF_ISendSbuf()

```
int PF_ISendSbuf (
    int to,
    int tag )
```

Nonblocking send operation. It sends everything from *buff* to *fill* of the active buffer. Depending on *tag* it also can do waiting for other sends to finish or set the active buffer to the next one.

Parameters

<i>to</i>	the destination process number.
<i>tag</i>	the message tag.

Returns

0 if OK, nonzero on error.

Definition at line 261 of file [mpi.c](#).

Referenced by [FlushOut\(\)](#), [PF_Processor\(\)](#), and [PutOut\(\)](#).

4.78.3.6 PF_RecvWbuf()

```
int PF_RecvWbuf (
    WORD * b,
    LONG * s,
    int * src )
```

Blocking receive of a WORD buffer.

Parameters

<i>out</i>	<i>b</i>	the buffer to store the received data.
<i>in, out</i>	<i>s</i>	the size of the buffer. The output value is the actual size of the received data.
<i>in, out</i>	<i>src</i>	the source process number. The output value is the process number of actual source.

Returns

the received message tag. A negative value indicates an error.

Definition at line 337 of file [mpi.c](#).

4.78.3.7 PF_IRecvRbuf()

```
int PF_IRecvRbuf (
    PF_BUFFER * r,
    int bn,
    int from )
```

Posts nonblocking receive for the active receive buffer. The buffer is filled from `full` to `stop`.

Parameters

<i>r</i>	the PF_BUFFER struct for the nonblocking receive.
<i>bn</i>	the index of the cyclic buffer.
<i>from</i>	the source process number.

Returns

0 if OK, nonzero on error.

Definition at line 366 of file [mpi.c](#).

4.78.3.8 PF_WaitRbuf()

```
int PF_WaitRbuf (
    PF_BUFFER * r,
    int bn,
    LONG * size )
```

Waits for the buffer *bn* to finish a pending nonblocking receive. It returns the received tag and in **size* the number of field received. If the receive is already finished, just return the flag and size, else wait for it to finish, but also check for other pending receives.

Parameters

	<i>r</i>	the PF_BUFFER struct for the pending nonblocking receive.
	<i>bn</i>	the index of the cyclic buffer.
out	<i>size</i>	the actual size of received data.

Returns

the received message tag. A negative value indicates an error.

Definition at line 400 of file [mpi.c](#).

4.78.3.9 PF_Bcast()

```
int PF_Bcast (
    void * buffer,
    int count )
```

Broadcasts a message from the master to slaves.

Parameters

<i>in, out</i>	<i>buffer</i>	the starting address of buffer. The contents in this buffer on the master will be transferred to those on the slaves.
	<i>count</i>	the length of the buffer in bytes.

Returns

0 if OK, nonzero on error.

Definition at line 440 of file [mpi.c](#).

Referenced by [PF_BroadcastBuffer\(\)](#), and [PF_BroadcastNumber\(\)](#).

4.78.3.10 PF_RawSend()

```
int PF_RawSend (
    int dest,
    void * buf,
    LONG l,
    int tag )
```

Sends *l* bytes from *buf* to *dest*. Returns 0 on success, or -1.

Parameters

<i>dest</i>	the destination process number.
<i>buf</i>	the send buffer.
<i>l</i>	the size of data in the send buffer in bytes.
<i>tag</i>	the message tag.

Returns

0 if OK, nonzero on error.

Definition at line 463 of file [mpi.c](#).

Referenced by [PF_MUnlock\(\)](#), and [PF_SendFile\(\)](#).

4.78.3.11 PF_RawRecv()

```
LONG PF_RawRecv (
    int * src,
```

```

void * buf,
LONG thesize,
int * tag )

```

Receives not more than *thesize* bytes from *src*, returns the actual number of received bytes, or -1 on failure.

Parameters

in, out	<i>src</i>	the source process number. In output, that of the actual received message.
out	<i>buf</i>	the receive buffer.
	<i>thesize</i>	the size of the receive buffer in bytes.
out	<i>tag</i>	the message tag of the actual received message.

Returns

the actual sizeof received data in bytes, or -1 on failure.

Definition at line 484 of file [mpi.c](#).

Referenced by [PF_RecvFile\(\)](#).

4.78.3.12 PF_RawProbe()

```

int PF_RawProbe (
    int * src,
    int * tag,
    int * bytesize )

```

Probes an incoming message.

Parameters

in, out	<i>src</i>	the source process number. In output, that of the actual received message.
in, out	<i>tag</i>	the message tag. In output, that of the actual received message.
out	<i>bytesize</i>	the size of incoming data in bytes.

Returns

0 if OK, nonzero on error.

Definition at line 508 of file [mpi.c](#).

4.78.3.13 PF_PrintPackBuf()

```

int PF_PrintPackBuf (
    char * s,
    int size )

```

Prints the contents in the pack buffer.

Parameters

<i>s</i>	a message to be shown.
<i>size</i>	the length of the buffer to be shown.

Returns

0 if OK, nonzero on error.

Definition at line 594 of file [mpi.c](#).

4.78.3.14 PF_PreparePack()

```
int PF_PreparePack (
    void )
```

Prepares for the next pack operations on the sender.

Returns

0 if OK, nonzero on error.

Definition at line 624 of file [mpi.c](#).

Referenced by [Optimize\(\)](#), [PF_BroadcastPreDollar\(\)](#), [PF_BroadcastString\(\)](#), and [PF_Init\(\)](#).

4.78.3.15 PF_Pack()

```
int PF_Pack (
    const void * buffer,
    size_t count,
    MPI_Datatype type )
```

Adds data into the pack buffer.

Parameters

<i>buffer</i>	the pointer to the buffer storing the data to be packed.
<i>count</i>	the number of elements in the buffer.
<i>type</i>	the data type of elements in the buffer.

Returns

0 if OK, nonzero on error.

Definition at line 642 of file [mpi.c](#).

References [MPI_ERRCODE_CHECK](#).

Referenced by [Optimize\(\)](#), [PF_BroadcastPreDollar\(\)](#), and [PF_Init\(\)](#).

4.78.3.16 PF_Unpack()

```
int PF_Unpack (
    void * buffer,
    size_t count,
    MPI_Datatype type )
```

Retrieves the next data in the pack buffer.

Parameters

out	<i>buffer</i>	the pointer to the buffer to store the unpacked data.
	<i>count</i>	the number of elements of data to be received.
	<i>type</i>	the data type of elements of data to be received.

Returns

0 if OK, nonzero on error.

Definition at line 671 of file [mpi.c](#).

References [MPI_ERRCODE_CHECK](#).

Referenced by [Optimize\(\)](#), [PF_BroadcastPreDollar\(\)](#), and [PF_Init\(\)](#).

4.78.3.17 PF_PackString()

```
int PF_PackString (
    const UBYTE * str )
```

Packs a string *str* into the packed buffer `PF_packbuf`, including the trailing zero.

The first element (`PF_INT`) is the length of the packed portion of the string. If the string does not fit to the buffer `PF_packbuf`, the function packs only the initial portion. It returns the number of packed bytes, so if `(str[length-1]!='\0')` then the whole string fits to the buffer, if not, then the rest (`str+length`) must be packed and send again. On error, the function returns the negative error code.

One exception: the string `"\0\0"` is used as an image of the NULL, so all 3 characters will be packed.

Parameters

<i>str</i>	a string to be packed.
------------	------------------------

Returns

the number of packed bytes, or a negative value on failure.

Definition at line 706 of file [mpi.c](#).

Referenced by [PF_BroadcastString\(\)](#).

4.78.3.18 PF_UnpackString()

```
int PF_UnpackString (
    UBYTE * str )
```

Unpacks a string to *str* from the packed buffer PF_packbuf, including the trailing zero.

It returns the number of unpacked bytes, so if (str[length-1]=='\0') then the whole string was unpacked, if not, then the rest must be appended to (str+length). On error, the function returns the negative error code.

Parameters

out	<i>str</i>	the buffer to store the unpacked string
-----	------------	---

Returns

the number of unpacked bytes, or a negative value on failure.

Definition at line 774 of file [mpi.c](#).

Referenced by [PF_BroadcastString\(\)](#).

4.78.3.19 PF_Send()

```
int PF_Send (
    int to,
    int tag )
```

Sends the contents in the pack buffer to the process specified by *to*.

Example:

```
if ( PF.me == SRC ) {
    PF_PreparePack();
    // Packing operations here...
    PF_Send(DEST, TAG);
}
else if ( PF.me == DEST ) {
    PF_Receive(SRC, TAG, &actual_src, &actual_tag);
    // Unpacking operations here...
}
```

Parameters

<i>to</i>	the destination process number.
<i>tag</i>	the message tag.

Returns

0 if OK, nonzero on error.

Definition at line 822 of file [mpi.c](#).

References [MPI_ERRCODE_CHECK](#).

4.78.3.20 PF_Receive()

```
int PF_Receive (
    int src,
    int tag,
    int * psrc,
    int * ptag )
```

Receives data into the pack buffer from the process specified by *src*. This function allows &*src* == *psrc* or &*tag* == *ptag*. Either *psrc* or *ptag* can be NULL.

See the example of [PF_Send\(\)](#).

Parameters

	<i>src</i>	the source process number (can be PF_ANY_SOURCE).
	<i>tag</i>	the source message tag (can be PF_ANY_TAG).
out	<i>psrc</i>	the actual source process number of received message.
out	<i>ptag</i>	the received message tag.

Returns

0 if OK, nonzero on error.

Definition at line 848 of file [mpi.c](#).

References [MPI_ERRCODE_CHECK](#).

4.78.3.21 PF_Broadcast()

```
int PF_Broadcast (
    void )
```

Broadcasts the contents in the pack buffer on the master to those on the slaves.

Example:

```
if ( PF.me == MASTER ) {
    PF_PrepPack();
    // Packing operations here...
}
PF_Broadcast();
if ( PF.me != MASTER ) {
    // Unpacking operations here...
}
```

Returns

0 if OK, nonzero on error.

Definition at line 883 of file [mpi.c](#).

References [MPI_ERRCODE_CHECK](#).

Referenced by [Optimize\(\)](#), [PF_BroadcastPreDollar\(\)](#), [PF_BroadcastString\(\)](#), and [PF_Init\(\)](#).

4.78.3.22 PF_PrepareLongSinglePack()

```
int PF_PrepareLongSinglePack (
    void )
```

Prepares for the next long-single-pack operations on the sender.

Returns

0 if OK, nonzero on error.

Definition at line 1451 of file [mpi.c](#).

Referenced by [PF_CollectModifiedDollars\(\)](#), and [PF_Processor\(\)](#).

4.78.3.23 PF_LongSinglePack()

```
int PF_LongSinglePack (
    const void * buffer,
    size_t count,
    MPI_Datatype type )
```

Adds data into the "long single" pack buffer.

Parameters

<i>buffer</i>	the pointer to the buffer storing the data to be packed.
<i>count</i>	the number of elements in the buffer.
<i>type</i>	the data type of elements in the buffer.

Returns

0 if OK, nonzero on error.

Definition at line 1469 of file [mpi.c](#).

Referenced by [PF_CollectModifiedDollars\(\)](#), and [PF_Processor\(\)](#).

4.78.3.24 PF_LongSingleUnpack()

```
int PF_LongSingleUnpack (
    void * buffer,
    size_t count,
    MPI_Datatype type )
```

Retrieves the next data in the "long single" pack buffer.

Parameters

out	<i>buffer</i>	the pointer to the buffer to store the unpacked data.
	<i>count</i>	the number of elements of data to be received.
	<i>type</i>	the data type of elements of data to be received.

Returns

0 if OK, nonzero on error.

Definition at line 1503 of file [mpi.c](#).

Referenced by [PF_CollectModifiedDollars\(\)](#), and [PF_Processor\(\)](#).

4.78.3.25 PF_LongSingleSend()

```
int PF_LongSingleSend (
    int to,
    int tag )
```

Sends the contents in the "long single" pack buffer to the process specified by *to*.

Example:

```
if ( PF.me == SRC ) {
    PF_PrepareLongSinglePack();
    // Packing operations here...
    PF_LongSingleSend(DEST, TAG);
}
else if ( PF.me == DEST ) {
    PF_LongSingleReceive(SRC, TAG, &actual_src, &actual_tag);
    // Unpacking operations here...
}
```

Parameters

<i>to</i>	the destination process number.
<i>tag</i>	the message tag.

Returns

0 if OK, nonzero on error.

Definition at line 1540 of file [mpi.c](#).

Referenced by [PF_CollectModifiedDollars\(\)](#), and [PF_Processor\(\)](#).

4.78.3.26 PF_LongSingleReceive()

```
int PF_LongSingleReceive (
    int src,
    int tag,
    int * psrc,
    int * ptag )
```

Receives data into the "long single" pack buffer from the process specified by *src*. This function allows *&src == psrc* or *&tag == ptag*. Either *psrc* or *ptag* can be NULL.

See the example of [PF_LongSingleSend\(\)](#).

Parameters

	<i>src</i>	the source process number (can be PF_ANY_SOURCE).
	<i>tag</i>	the source message tag (can be PF_ANY_TAG).
out	<i>psrc</i>	the actual source process number of received message.
out	<i>ptag</i>	the received message tag.

Returns

0 if OK, nonzero on error.

Definition at line 1583 of file [mpi.c](#).

Referenced by [PF_CollectModifiedDollars\(\)](#), and [PF_Processor\(\)](#).

4.78.3.27 PF_PrepareLongMultiPack()

```
int PF_PrepareLongMultiPack (
    void )
```

Prepares for the next long-multi-pack operations on the sender.

Returns

0 if OK, nonzero on error.

Definition at line 1643 of file [mpi.c](#).

Referenced by [Optimize\(\)](#), [PF_BroadcastCBuf\(\)](#), [PF_BroadcastExpFlags\(\)](#), [PF_BroadcastModifiedDollars\(\)](#), and [PF_BroadcastRedefinedPreVars\(\)](#).

4.78.3.28 PF_LongMultiPackImpl()

```
int PF_LongMultiPackImpl (
    const void * buffer,
    size_t count,
    size_t eSize,
    MPI_Datatype type )
```

Adds data into the "long multi" pack buffer.

Parameters

<i>buffer</i>	the pointer to the buffer storing the data to be packed.
<i>count</i>	the number of elements in the buffer.
<i>eSize</i>	the byte size of each element of data.
<i>type</i>	the data type of elements in the buffer.

Returns

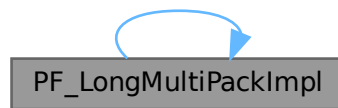
0 if OK, nonzero on error.

Definition at line 1662 of file [mpi.c](#).

References [PF_LongMultiPackImpl\(\)](#).

Referenced by [PF_LongMultiPackImpl\(\)](#).

Here is the call graph for this function:

**4.78.3.29 PF_LongMultiUnpackImpl()**

```

int PF_LongMultiUnpackImpl (
    void * buffer,
    size_t count,
    size_t eSize,
    MPI_Datatype type )
  
```

Retrieves the next data in the "long multi" pack buffer.

Parameters

out	<i>buffer</i>	the pointer to the buffer to store the unpacked data.
	<i>count</i>	the number of elements of data to be received.
	<i>eSize</i>	the byte size of each element of data.
	<i>type</i>	the data type of elements of data to be received.

Returns

0 if OK, nonzero on error.

Definition at line 1721 of file [mpi.c](#).

References [PF_LongMultiUnpackImpl\(\)](#).

Referenced by [PF_LongMultiUnpackImpl\(\)](#).

Here is the call graph for this function:



4.78.3.30 PF_LongMultiBroadcast()

```
int PF_LongMultiBroadcast (
    void )
```

Broadcasts the contents in the "long multi" pack buffer on the master to those on the slaves.

Example:

```
if ( PF.me == MASTER ) {
    PF_PrepareLongMultiPack();
    // Packing operations here...
}
PF_LongMultiBroadcast();
if ( PF.me != MASTER ) {
    // Unpacking operations here...
}
```

Returns

0 if OK, nonzero on error.

Definition at line 1807 of file [mpi.c](#).

Referenced by [Optimize\(\)](#), [PF_BroadcastCBuf\(\)](#), [PF_BroadcastExpFlags\(\)](#), [PF_BroadcastModifiedDollars\(\)](#), and [PF_BroadcastRedefinedPreVars\(\)](#).

4.78.4 Variable Documentation

4.78.4.1 PF_maxDollarChunkSize

```
LONG PF_maxDollarChunkSize = 0
```

Definition at line 69 of file [mpi.c](#).

4.79 mpi.c

[Go to the documentation of this file.](#)

```

00001
00009 /* #[] License : */
00010 /*
00011 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00012 *   When using this file you are requested to refer to the publication
00013 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00014 *   This is considered a matter of courtesy as the development was paid
00015 *   for by FOM the Dutch physics granting agency and we would like to
00016 *   be able to track its scientific use to convince FOM of its value
00017 *   for the community.
00018 *
00019 *   This file is part of FORM.
00020 *
00021 *   FORM is free software: you can redistribute it and/or modify it under the
00022 *   terms of the GNU General Public License as published by the Free Software
00023 *   Foundation, either version 3 of the License, or (at your option) any later
00024 *   version.
00025 *
00026 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00027 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00028 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00029 *   details.
00030 *
00031 *   You should have received a copy of the GNU General Public License along
00032 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00033 */
00034 /* #] License : */
00035 /*
00036     #[] Includes and variables :
00037 */
00038
00039 #include <limits.h>
00040 #include "form3.h"
00041
00042 #ifdef MPICH_PROFILING
00043 # include "mpe.h"
00044 #endif
00045
00046 #ifdef MPIDEBUGGING
00047 #include "mpidbg.h"
00048 #endif
00049
00050 /*[12oct2005 mt]:*/
00051 /*
00052     Today there was some cleanup, some stuff is moved into another place
00053     in this file, and PF.packsize is removed and PF_packsize is used
00054     instead. It is rather difficult to proper comment it, so not all these
00055     changing are marked by "[12oct2005 mt]"
00056 */
00057
00058 #define PF_PACKSIZE 1600
00059
00060 /*
00061     Size in bytes, will be initialized soon as
00062     PF_packsize=PF_PACKSIZE/sizeof(int)*sizeof(int); for possible
00063     future developing we prefer to do this initialization not here,
00064     but in PF_LibInit:
00065 */
00066
00067 static int PF_packsize = 0;
00068 static MPI_Status PF_status;
00069 LONG PF_maxDollarChunkSize = 0; /*:[04oct2005 mt]*/
00070
00071 static int PF_ShortPackInit(void);
00072 static int PF_longPackInit(void); /*:[12oct2005 mt]*/
00073
00084 #define MPI_ERRCODE_CHECK(err) \
00085     do { \
00086         int _tmp_err = (err); \
00087         if ( _tmp_err != MPI_SUCCESS ) return _tmp_err != 0 ? _tmp_err : -1; \
00088     } while (0)
00089
00090 /*
00091     #] Includes and variables :
00092     #[] PF_RealTime :
00093 */
00094
00101 LONG PF_RealTime(int i)
00102 {
00103     static double starttime;
00104     if ( i == PF_RESET ) {
00105         starttime = MPI_Wtime();

```

```

00106         return((LONG)0);
00107     }
00108     return((LONG)( 100. * (MPI_Wtime() - starttime) ));
00109 }
00110
00111 /*
00112 #] PF_RealTime :
00113 #[ PF_LibInit :
00114 */
00115
00123 int PF_LibInit(int *argcp, char ***argvp)
00124 {
00125     int ret;
00126     ret = MPI_Init(argcp,argvp);
00127     if ( ret != MPI_SUCCESS ) return(ret);
00128     ret = MPI_Comm_rank(PF_COMM,&PF.me);
00129     if ( ret != MPI_SUCCESS ) return(ret);
00130     ret = MPI_Comm_size(PF_COMM,&PF.numtasks);
00131     if ( ret != MPI_SUCCESS ) return(ret);
00132
00133     /* Initialization of packed communications. */
00134     PF_packsize = PF_PACKSIZE/sizeof(int)*sizeof(int);
00135     if ( PF_ShortPackInit() ) return -1;
00136     if ( PF_longPackInit() ) return -1;
00137
00138     /*Block*/
00139     int bytes, totalbytes=0;
00140 /*
00141     There is one problem with maximal possible packing: there is no API to
00142     convert bytes to the record number. So, here we calculate the buffer
00143     size needed for storing dollarvars:
00144
00145     LONG PF_maxDollarChunkSize is the size for the portion of the dollar
00146     variable buffer suitable for broadcasting. This variable should be
00147     visible from parallel.c
00148
00149     Evaluate PF_Pack(numterms,1,PF_INT):
00150 */
00151     if ( ( ret = MPI_Pack_size(1,PF_INT,PF_COMM,&bytes) )!=MPI_SUCCESS )
00152         return(ret);
00153     totalbytes+=bytes;
00154 /*
00155     Evaluate PF_Pack( newsize,1,PF_LONG):
00156 */
00157     if ( ( ret = MPI_Pack_size(1,PF_LONG,PF_COMM,&bytes) )!=MPI_SUCCESS )
00158         return(ret);
00159     totalbytes += bytes;
00160
00161     Now available room is PF_packsize-totalbytes
00162 */
00163     totalbytes = PF_packsize-totalbytes;
00164 /*
00165     Now totalbytes is the size of chunk in bytes.
00166     Evaluate this size in number of records:
00167
00168     Rough estimate:
00169 */
00170     PF_maxDollarChunkSize=totalbytes/sizeof(WORD);
00171 /*
00172     Go to the up limit:
00173 */
00174     do {
00175         if ( ( ret = MPI_Pack_size(
00176             ++PF_maxDollarChunkSize,PF_WORD,PF_COMM,&bytes) )!=MPI_SUCCESS )
00177             return(ret);
00178     } while ( bytes<totalbytes );
00179 /*
00180     Now the chunk size is too large
00181     And now evaluate the exact value:
00182 */
00183     do {
00184         if ( ( ret = MPI_Pack_size(
00185             --PF_maxDollarChunkSize,PF_WORD,PF_COMM,&bytes) )!=MPI_SUCCESS )
00186             return(ret);
00187     } while ( bytes>totalbytes );
00188 /*
00189     Now PF_maxDollarChunkSize is the size of chunk of PF_WORD fitting the
00190     buffer <= (PF_packsize-PF_INT-PF_LONG)
00191 */
00192     }/*Block*/
00193     return(0);
00194 }
00195
00196 #] PF_LibInit :
00197 #[ PF_LibTerminate :

```

```

00200 */
00201
00209 int PF_LibTerminate(int error)
00210 {
00211     DUMMYUSE(error);
00212     return(MPI_Finalize());
00213 }
00214
00215 /*
00216 #] PF_LibTerminate :
00217 #[ PF_Probe :
00218 */
00219
00230 int PF_Probe(int *src)
00231 {
00232     int ret, flag;
00233     if ( *src == PF_ANY_SOURCE ) { /*Blocking call*/
00234         ret = MPI_Probe(*src,MPI_ANY_TAG,PF_COMM,&PF_status);
00235         flag = 1;
00236     }
00237     else { /*Non-blocking call*/
00238         ret = MPI_Iprobe(*src,MPI_ANY_TAG,PF_COMM,&flag,&PF_status);
00239     }
00240     *src = PF_status.MPI_SOURCE;
00241     if ( ret != MPI_SUCCESS ) { if ( ret > 0 ) ret *= -1; return(ret); }
00242     if ( !flag ) return(0);
00243     return(PF_status.MPI_TAG);
00244 }
00245
00246 /*
00247 #] PF_Probe :
00248 #[ PF_IsendSbuf :
00249 */
00250
00261 int PF_IsendSbuf(int to, int tag)
00262 {
00263     PF_BUFFER *s = PF.sbuf;
00264     int a = s->active;
00265     int size = s->fill[a] - s->buff[a];
00266     int r = 0;
00267
00268     static int finished;
00269
00270     s->fill[a] = s->buff[a];
00271     if ( s->numbufs == 1 ) {
00272         r = MPI_Ssend(s->buff[a],size,PF_WORD,MASTER,tag,PF_COMM);
00273         if ( r != MPI_SUCCESS ) {
00274             fprintf(stderr,"%d|&d] PF_IsendSbuf: MPI_Ssend returns: %d \n",
00275                 PF.me,(int)AC.CModule,r);
00276             fflush(stderr);
00277             return(r);
00278         }
00279         return(0);
00280     }
00281
00282     switch ( tag ) { /* things to do before sending */
00283     case PF_TERM_MSGTAG:
00284         if ( PF.sbuf->request[to] != MPI_REQUEST_NULL)
00285             r = MPI_Wait(&PF.sbuf->request[to],&PF.sbuf->retstat[to]);
00286         if ( r != MPI_SUCCESS ) return(r);
00287         break;
00288     default:
00289         break;
00290     }
00291
00292     r = MPI_Isend(s->buff[a],size,PF_WORD,to,tag,PF_COMM,&s->request[a]);
00293
00294     if ( r != MPI_SUCCESS ) return(r);
00295
00296     switch ( tag ) { /* things to do after initialising sending */
00297     case PF_TERM_MSGTAG:
00298         finished = 0;
00299         break;
00300     case PF_ENDSORT_MSGTAG:
00301         if ( ++finished == PF.numtasks - 1 )
00302             r = MPI_Waitall(s->numbufs,s->request,s->status);
00303         if ( r != MPI_SUCCESS ) return(r);
00304         break;
00305     case PF_BUFFER_MSGTAG:
00306         if ( ++s->active >= s->numbufs ) s->active = 0;
00307         while ( s->request[s->active] != MPI_REQUEST_NULL ) {
00308             r = MPI_Waitsome(s->numbufs,s->request,&size,s->index,s->retstat);
00309             if ( r != MPI_SUCCESS ) return(r);
00310         }
00311         break;
00312     case PF_ENDBUFFER_MSGTAG:
00313         if ( ++s->active >= s->numbufs ) s->active = 0;

```

```

00314         r = MPI_Waitall(s->numbufs,s->request,s->status);
00315         if ( r != MPI_SUCCESS ) return(r);
00316         break;
00317     default:
00318         return(-99);
00319         break;
00320     }
00321     return(0);
00322 }
00323
00324 /*
00325     #] PF_IsendSbuf :
00326     #[ PF_RecvWbuf :
00327 */
00328
00337 int PF_RecvWbuf(WORD *b, LONG *s, int *src)
00338 {
00339     int i, r = 0;
00340
00341     r = MPI_Recv(b, (int)*s, PF_WORD, *src, PF_ANY_MSGTAG, PF_COMM, &PF_status);
00342     if ( r != MPI_SUCCESS ) { if ( r > 0 ) r *= -1; return(r); }
00343
00344     r = MPI_Get_count(&PF_status, PF_WORD, &i);
00345     if ( r != MPI_SUCCESS ) { if ( r > 0 ) r *= -1; return(r); }
00346
00347     *s = (LONG)i;
00348     *src = PF_status.MPI_SOURCE;
00349     return(PF_status.MPI_TAG);
00350 }
00351
00352 /*
00353     #] PF_RecvWbuf :
00354     #[ PF_IRecvRbuf :
00355 */
00356
00366 int PF_IRecvRbuf(PF_BUFFER *r, int bn, int from)
00367 {
00368     int ret;
00369     r->type[bn] = PF_WORD;
00370
00371     if ( r->numbufs == 1 ) {
00372         r->tag[bn] = MPI_ANY_TAG;
00373         r->from[bn] = from;
00374     }
00375     else {
00376         ret = MPI_Irecv(r->full[bn], (int)(r->stop[bn] - r->full[bn]), PF_WORD, from,
00377             MPI_ANY_TAG, PF_COMM, &r->request[bn]);
00378         if (ret != MPI_SUCCESS) { if (ret > 0) ret *= -1; return(ret); }
00379     }
00380     return(0);
00381 }
00382
00383 /*
00384     #] PF_IRecvRbuf :
00385     #[ PF_WaitRbuf :
00386 */
00387
00400 int PF_WaitRbuf(PF_BUFFER *r, int bn, LONG *size)
00401 {
00402     int ret, rsize;
00403
00404     if ( r->numbufs == 1 ) {
00405         *size = r->stop[bn] - r->full[bn];
00406         ret = MPI_Recv(r->full[bn], (int)*size, r->type[bn], r->from[bn], r->tag[bn],
00407             PF_COMM, &(r->status[bn]));
00408         if ( ret != MPI_SUCCESS ) { if ( ret > 0 ) ret *= -1; return(ret); }
00409         ret = MPI_Get_count(&(r->status[bn]), r->type[bn], &rsize);
00410         if ( ret != MPI_SUCCESS ) { if ( ret > 0 ) ret *= -1; return(ret); }
00411         if ( rsize > *size ) return(-99);
00412         *size = (LONG)rsize;
00413     }
00414     else {
00415         while ( r->request[bn] != MPI_REQUEST_NULL ) {
00416             ret = MPI_Waitsome(r->numbufs, r->request, &rsize, r->index, r->retstat);
00417             if ( ret != MPI_SUCCESS ) { if ( ret > 0 ) ret *= -1; return(ret); }
00418             while ( --rsize >= 0 ) r->status[r->index[rsize]] = r->retstat[rsize];
00419         }
00420         ret = MPI_Get_count(&(r->status[bn]), r->type[bn], &rsize);
00421         if ( ret != MPI_SUCCESS ) { if ( ret > 0 ) ret *= -1; return(ret); }
00422         *size = (LONG)rsize;
00423     }
00424     return(r->status[bn].MPI_TAG);
00425 }
00426
00427 /*
00428     #] PF_WaitRbuf :
00429     #[ PF_Bcast :

```

```

00430 */
00431
00440 int PF_Bcast(void *buffer, int count)
00441 {
00442     if ( MPI_Bcast(buffer, count, MPI_BYTE, MASTER, PF_COMM) != MPI_SUCCESS )
00443         return(-1);
00444     return(0);
00445 }
00446
00447 /*
00448     #] PF_Bcast :
00449     #[ PF_RawSend :
00450 */
00451
00463 int PF_RawSend(int dest, void *buf, LONG l, int tag)
00464 {
00465     int ret=MPI_Ssend(buf, (int)l, MPI_BYTE, dest, tag, PF_COMM);
00466     if ( ret != MPI_SUCCESS ) return(-1);
00467     return(0);
00468 }
00469 /*
00470     #] PF_RawSend :
00471     #[ PF_RawRecv :
00472 */
00473
00484 LONG PF_RawRecv(int *src, void *buf, LONG thesize, int *tag)
00485 {
00486     MPI_Status stat;
00487     int ret=MPI_Recv(buf, (int)thesize, MPI_BYTE, *src, MPI_ANY_TAG, PF_COMM, &stat);
00488     if ( ret != MPI_SUCCESS ) return(-1);
00489     if ( MPI_Get_count(&stat, MPI_BYTE, &ret) != MPI_SUCCESS ) return(-1);
00490     *tag = stat.MPI_TAG;
00491     *src = stat.MPI_SOURCE;
00492     return(ret);
00493 }
00494
00495 /*
00496     #] PF_RawRecv :
00497     #[ PF_RawProbe :
00498 */
00499
00508 int PF_RawProbe(int *src, int *tag, int *bytesize)
00509 {
00510     MPI_Status stat;
00511     int srcval = src != NULL ? *src : PF_ANY_SOURCE;
00512     int tagval = tag != NULL ? *tag : PF_ANY_MSGTAG;
00513     int ret = MPI_Probe(srcval, tagval, PF_COMM, &stat);
00514     if ( ret != MPI_SUCCESS ) return -1;
00515     if ( src != NULL ) *src = stat.MPI_SOURCE;
00516     if ( tag != NULL ) *tag = stat.MPI_TAG;
00517     if ( bytesize != NULL ) {
00518         ret = MPI_Get_count(&stat, MPI_BYTE, bytesize);
00519         if ( ret != MPI_SUCCESS ) return -1;
00520     }
00521     return 0;
00522 }
00523
00524 /*
00525     #] PF_RawProbe :
00526     #[ The pack buffer :
00527         #[ Variables :
00528 */
00529
00530 /*
00531     * The pack buffer with the fixed size (= PF_packsize).
00532 */
00533 static UBYTE *PF_packbuf = NULL;
00534 static UBYTE *PF_packstop = NULL;
00535 static int PF_packpos = 0;
00536
00537 /*
00538     #] Variables :
00539     #[ PF_ShortPackInit :
00540 */
00541
00548 static int PF_ShortPackInit(void)
00549 {
00550     PF_packbuf = (UBYTE *)Malloc1(sizeof(UBYTE) * PF_packsize, "PF_ShortPackInit");
00551     if ( PF_packbuf == NULL ) return -1;
00552     PF_packstop = PF_packbuf + PF_packsize;
00553     return 0;
00554 }
00555
00556 /*
00557     #] PF_ShortPackInit :
00558     #[ PF_InitPackBuf :
00559 */

```



```

00560
00566 static inline int PF_InitPackBuf(void)
00567 {
00568 /*
00569      This is definitely not the best place for allocating the
00570      buffer! Moved to PF_LibInit():
00571
00572      if ( PF_packbuf == 0 ) {
00573          PF_packbuf = (UBYTE *)Malloc1(sizeof(UBYTE)*PF.packsize,"PF_InitPackBuf");
00574          if ( PF_packbuf == 0 ) return(-1);
00575          PF_packstop = PF_packbuf + PF.packsize;
00576      }
00577 */
00578     PF_packpos = 0;
00579     return(0);
00580 }
00581
00582 /*
00583      #] PF_InitPackBuf :
00584      #[ PF_PrintPackBuf :
00585 */
00586
00594 int PF_PrintPackBuf(char *s, int size)
00595 {
00596 #ifdef NOMESPRINTYET
00597 /*
00598      The use of printf should be discouraged. The results are flushed to
00599      the output at unpredictable moments. We should use printf only
00600      during startup when MesPrint doesn't have its buffers and output
00601      channels initialized.
00602 */
00603     int i;
00604     printf("[%d] %s: ",PF.me,s);
00605     for(i=0;i<size;i++) printf("%d ",PF_packbuf[i]);
00606     printf("\n");
00607 #else
00608     MesPrint("[%d] %s: %a",PF.me,s,size,(WORD *) (PF_packbuf));
00609 #endif
00610     return(0);
00611 }
00612
00613 /*
00614      #] PF_PrintPackBuf :
00615      #[ PF_PreparePack :
00616 */
00617
00624 int PF_PreparePack(void)
00625 {
00626     return PF_InitPackBuf();
00627 }
00628
00629 /*
00630      #] PF_PreparePack :
00631      #[ PF_Pack :
00632 */
00633
00642 int PF_Pack(const void *buffer, size_t count, MPI_Datatype type)
00643 {
00644     int err, bytes;
00645
00646     if ( count > INT_MAX ) return -99;
00647
00648     err = MPI_Pack_size((int)count, type, PF_COMM, &bytes);
00649     MPI_ERRCODE_CHECK(err);
00650     if ( PF_packpos + bytes > PF_packstop - PF_packbuf ) return -99;
00651
00652     err = MPI_Pack((void *)buffer, (int)count, type, PF_packbuf, PF_packsize, &PF_packpos, PF_COMM);
00653     MPI_ERRCODE_CHECK(err);
00654
00655     return 0;
00656 }
00657
00658 /*
00659      #] PF_Pack :
00660      #[ PF_Unpack :
00661 */
00662
00671 int PF_Unpack(void *buffer, size_t count, MPI_Datatype type)
00672 {
00673     int err;
00674
00675     if ( count > INT_MAX ) return -99;
00676
00677     err = MPI_Unpack(PF_packbuf, PF_packsize, &PF_packpos, buffer, (int)count, type, PF_COMM);
00678     MPI_ERRCODE_CHECK(err);
00679
00680     return 0;

```

```

00681 }
00682
00683 /*
00684     #] PF_Unpack :
00685     #[ PF_PackString :
00686 */
00687
00706 int PF_PackString(const UBYTE *str)
00707 {
00708     int ret,buflength,bytes,length;
00709     /*
00710         length will be packed in the beginning.
00711         Decrement buffer size by the length of the field "length":
00712     */
00713     if ( ( ret = MPI_Pack_size(1,PF_INT,PF_COMM,&bytes) ) != MPI_SUCCESS )
00714         return(ret);
00715     buflength = PF_packsize - bytes;
00716     /*
00717         Calculate the string length (INCLUDING the trailing zero!):
00718     */
00719     for ( length = 0; length < buflength; length++ ) {
00720         if ( str[length] == '\0' ) {
00721             length++; /* since the trailing zero must be accounted */
00722             break;
00723         }
00724     }
00725     /*
00726         The string "\0!\0" is used as an image of the NULL.
00727     */
00728     if ( ( str[0] == '\0' ) /* empty string */
00729         && ( str[1] == '!' ) /* Special case? */
00730         && ( str[2] == '\0' ) /* Yes, pass 3 initial symbols */
00731         ) length += 2; /* all 3 characters will be packed */
00732     length++; /* Will be decremented in the following loop */
00733     /*
00734         The problem: packed size of byte may be not equal 1! So first, suppose
00735         it is 1, and if this is not the case decrease the length of the string
00736         until it fits the buffer:
00737     */
00738     do {
00739         if ( ( ret = MPI_Pack_size(--length,PF_BYTE,PF_COMM,&bytes) )
00740             != MPI_SUCCESS ) return(ret);
00741     } while ( bytes > buflength );
00742     /*
00743         Note, now if str[length-1] == '\0' then the string fits to the buffer
00744         (INCLUDING the trailing zero!);if not, the rest must be packed further!
00745
00746         Pack the length to PF_packbuf:
00747     */
00748     if ( ( ret = MPI_Pack(&length,1,PF_INT,PF_packbuf,PF_packsize,
00749         &PF_packpos,PF_COMM) ) != MPI_SUCCESS ) return(ret);
00750     /*
00751         Pack the string to PF_packbuf:
00752     */
00753     if ( ( ret = MPI_Pack((UBYTE *)str,length,PF_BYTE,PF_packbuf,PF_packsize,
00754         &PF_packpos,PF_COMM) ) != MPI_SUCCESS ) return(ret);
00755     return(length);
00756 }
00757
00758 /*
00759     #] PF_PackString :
00760     #[ PF_UnpackString :
00761 */
00762
00774 int PF_UnpackString(UBYTE *str)
00775 {
00776     int ret,length;
00777     /*
00778         Unpack the length:
00779     */
00780     if( (ret = MPI_Unpack(PF_packbuf,PF_packsize,&PF_packpos,
00781         &length,1,PF_INT,PF_COMM)) != MPI_SUCCESS )
00782         return(ret);
00783     /*
00784         Unpack the string:
00785     */
00786     if ( ( ret = MPI_Unpack(PF_packbuf,PF_packsize,&PF_packpos,
00787         str,length,PF_BYTE,PF_COMM) ) != MPI_SUCCESS ) return(ret);
00788     /*
00789         Now if str[length-1]=='\0' then the whole string
00790         (INCLUDING the trailing zero!) was unpacked ;if not, the rest
00791         must be unpacked to str+length.
00792     */
00793     return(length);
00794 }
00795
00796 /*

```

```

00797         #] PF_UnpackString :
00798         #[ PF_Send :
00799     */
00800
00822 int PF_Send(int to, int tag)
00823 {
00824     int err;
00825     err = MPI_Ssend(PF_packbuf, PF_packpos, MPI_PACKED, to, tag, PF_COMM);
00826     MPI_ERRCODE_CHECK(err);
00827     return 0;
00828 }
00829
00830 /*
00831     #] PF_Send :
00832     #[ PF_Receive :
00833 */
00834
00848 int PF_Receive(int src, int tag, int *psrc, int *ptag)
00849 {
00850     int err;
00851     MPI_Status status;
00852     PF_InitPackBuf();
00853     err = MPI_Recv(PF_packbuf, PF_packsize, MPI_PACKED, src, tag, PF_COMM, &status);
00854     MPI_ERRCODE_CHECK(err);
00855     if ( psrc ) *psrc = status.MPI_SOURCE;
00856     if ( ptag ) *ptag = status.MPI_TAG;
00857     return 0;
00858 }
00859
00860 /*
00861     #] PF_Receive :
00862     #[ PF_Broadcast :
00863 */
00864
00883 int PF_Broadcast(void)
00884 {
00885     int err;
00886     /*
00887     * If PF_SHORTBROADCAST is defined, then the broadcasting will be performed in
00888     * 2 steps. First, the size of the buffer will be broadcast, then the buffer of
00889     * exactly used size. This should be faster with slow connections, but slower on
00890     * SMP shmem MPI because of the latency.
00891     */
00892     #ifdef PF_SHORTBROADCAST
00893         int pos = PF_packpos;
00894     #endif
00895     if ( PF.me != MASTER ) {
00896         err = PF_InitPackBuf();
00897         if ( err ) return err;
00898     }
00899     #ifdef PF_SHORTBROADCAST
00900         err = MPI_Bcast(&pos, 1, MPI_INT, MASTER, PF_COMM);
00901         MPI_ERRCODE_CHECK(err);
00902         err = MPI_Bcast(PF_packbuf, pos, MPI_PACKED, MASTER, PF_COMM);
00903     #else
00904         err = MPI_Bcast(PF_packbuf, PF_packsize, MPI_PACKED, MASTER, PF_COMM);
00905     #endif
00906     MPI_ERRCODE_CHECK(err);
00907     return 0;
00908 }
00909
00910 /*
00911     #] PF_Broadcast :
00912     #] The pack buffer :
00913     #[ Long pack stuff :
00914     #[ Explanations :
00915
00916     The problems here are:
00917     1. We need to send/receive long dollar variables. For
00918     preprocessor-defined dollarvars we used multiply
00919     packing/broadcasting (see parallel.c:PF_BroadcastPreDollar())
00920     since each variable must be broadcast immediately. For run-time
00921     the changed dollar variables, collecting and broadcasting are
00922     performed at the end of the module and all modified dollarvars
00923     are transferred "at once", that is why the size of packed and
00924     transferred buffers may be really very large.
00925     2. There is some strange feature of MPI_Bcast() on Altix MPI
00926     implementation, namely, sometimes it silently fails with big
00927     buffers. For better performance, it would be useful to send one
00928     big buffer instead of several small ones (since the latency is more
00929     important than the bandwidth). That is why we need two different
00930     sets of routines: for long point-to-point communication we collect
00931     big re-allocatable buffer, the corresponding routines have the
00932     prefix PF_longSingle, and for broadcasting we pack data into
00933     several smaller buffers, the corresponding routines have the
00934     prefix PF_longMulti.
00935     Note, from portability reasons we cannot split large packed

```

```

00936     buffer into small chunks, send them and collect back on the other
00937     side, see "Advice to users" on page 180 MPI--The Complete Reference
00938     Volume1, second edition.
00939     OPTIMIZING:
00940     We assume, for most communications, the single buffer of size
00941     PF_packsize is enough.
00942
00943     How does it work:
00944     For point-to-point, there is one big re-allocatable
00945     buffer PF_longPackBuf with two integer positions: PF_longPackPos
00946     and PF_longPackTop (due to re-allocatable character of the buffer,
00947     it is better to use integers rather than pointers).
00948     Each time of re-allocation, the size of the buffer
00949     PF_longPackBuf is incremented by the same size of a "standard" chunk
00950     PF_packsize.
00951     For broadcasting there is one linked list (PF_longMultiRoot),
00952     which contains either positions of a chunk of PF_longPackBuf, or
00953     it's own buffer. This is done for better memory utilisation:
00954     longSingle and longMulti are never used simultaneously.
00955     When a new cell is needed for longMulti packing, we increment
00956     the counter PF_longPackN and just follow the list. If it is not
00957     possible, we allocate the cell's own buffer and link it to the end
00958     of the list PF_longMultiRoot.
00959     When PF_longPackPos is reallocated, we link new chunks into
00960     existing PF_longMultiRoot list before the first longMulti allocated
00961     cell's own buffer. The pointer PF_longMultiLastChunk points to the last
00962     cell of PF_longMultiRoot containing the pointer to the chunk of
00963     PF_longPackBuf.
00964     Initialization PF_longPackBuf is made by the function
00965     PF_longSingleReset(). In the begin of the PF_longPackBuf it packs
00966     the size of the last sent buffer. Upon sending, the program checks,
00967     whether there was at list one re-allocation (PF_longPackN>1) .
00968     If so, the sender first packs and sends small buffer
00969     (PF_longPackSmallBuf) containing one integer number -- the
00970     _negative_ new size of the send buffer. Getting the buffer, a
00971     receiver unpacks one integer and checks whether it is <0 . If so,
00972     the receiver will repeat receiving, but first it checks whether
00973     it has enough buffer and increase it, if necessary.
00974     Initialization PF_longMultiRoot is made by the function
00975     PF_longMultiReset(). In the begin of the first chunk it packs
00976     one integer -- the number 1. Upon sending, the program checks,
00977     how many cells were packed (PF_longPackN). If more than 1, the
00978     sender packs to the next cell the integer PF_longPackN, than
00979     packs PF_longPackN pairs of integers -- the information about how many
00980     times chunk on each cell was accessed by the packing procedure,
00981     this information is contained by the nPacks field of the cell
00982     structure, and how many non-complete items was at the end of this
00983     chunk the structure field lastLen. Then the sender sends first
00984     this auxiliary chunk.
00985     The receiver unpacks the integer from obtained chunk and, if this
00986     integer is more than 1, it gets more chunks, unpacking information
00987     from the first auxiliary chunk into the corresponding nPacks
00988     fields. Unpacking information from multiple chunks, the receiver
00989     knows, when the chunk is expired and it must switch to the next cell,
00990     successively decrementing corresponding nPacks field.
00991
00992     XXX: There are still some flaws:
00993     PF_LongSingleSend/PF_LongSingleReceive may fail, for example, for data
00994     transfers from the master to many slaves. Suppose that the master sends big
00995     data to slaves, which needs an increase of the buffer of the receivers. For
00996     the first data transfer, the master sends the new buffer size as the first
00997     message, and then sends the data as the second message, because
00998     PF_LongSinglePack records the increase of the buffer size on the master. For
00999     the next time, however, the master sends the data without sending the new
01000     buffer size, and then MPI_Recv fails due to the data overflow.
01001     In parallel.c, they are used for the communication from slaves to the
01002     master. In this case, this problem does not occur because the master always
01003     has enough buffer.
01004     The maximum size that PF_LongMultiBroadcast can broadcast is limited to
01005     around 320kB because the current implementation tries to pack all
01006     information of chained buffers into one buffer, whose size is PF_packsize
01007     = 1600B.
01008
01009     #] Explanations :
01010     #[ Variables :
01011 */
01012
01013 typedef struct longMultiStruct {
01014     UBYTE *buffer; /* NULL if */
01015     int bufpos; /* if >=0, PF_longPackBuf+bufpos is the chunk start */
01016     int packpos; /* the current position */
01017     int nPacks; /* How many times PF_longPack operates on this cell */
01018     int lastLen; /* if > 0, the last packing didn't fit completely to this
01019                  chunk, only lastLen items was packed, the rest is in
01020                  the next cell. */
01021     struct longMultiStruct *next; /* next linked cell, or NULL */
01022 } PF_LONGMULTI;

```

```

01023
01024 static UBYTE *PF_longPackBuf = NULL;
01025 static void *PF_longPackSmallBuf = NULL;
01026 static int PF_longPackPos = 0;
01027 static int PF_longPackTop = 0;
01028 static PF_LONGMULTI *PF_longMultiRoot = NULL;
01029 static PF_LONGMULTI *PF_longMultiTop = NULL;
01030 static PF_LONGMULTI *PF_longMultiLastChunk = NULL;
01031 static int PF_longPackN = 0;
01032
01033 /*
01034     #] Variables :
01035     #[ Long pack private functions :
01036     #[ PF_longMultiNewCell :
01037 */
01038
01039 static inline int PF_longMultiNewCell(void)
01040 {
01041     /*
01042         Allocate a new cell:
01043     */
01044     PF_longMultiTop->next = (PF_LONGMULTI *)
01045         Malloc1(sizeof(PF_LONGMULTI), "PF_longMultiCell");
01046     if ( PF_longMultiTop->next == NULL ) return(-1);
01047     /*
01048         Allocate a private buffer:
01049     */
01050     PF_longMultiTop->next->buffer=(UBYTE*)
01051         Malloc1(sizeof(UBYTE)*PF_packsize, "PF_longMultiChunk");
01052     if ( PF_longMultiTop->next->buffer == NULL ) return(-1);
01053     /*
01054         For the private buffer position is -1:
01055     */
01056     PF_longMultiTop->next->bufpos = -1;
01057     /*
01058         This is the last cell in the chain:
01059     */
01060     PF_longMultiTop->next->next = NULL;
01061     /*
01062         packpos and nPacks are not initialized!
01063     */
01064     return(0);
01065 }
01066
01067 /*
01068     #] PF_longMultiNewCell :
01069     #[ PF_longMultiPack2NextCell :
01070 */
01071 static inline int PF_longMultiPack2NextCell(void)
01072 {
01073     /*
01074         Is there a free cell in the chain?
01075     */
01076     if ( PF_longMultiTop->next == NULL ) {
01077         /*
01078             No, allocate the new cell with a private buffer:
01079         */
01080         if ( PF_longMultiNewCell() ) return(-1);
01081     }
01082     /*
01083         Move to the next cell in the chain:
01084     */
01085     PF_longMultiTop = PF_longMultiTop->next;
01086     /*
01087         if >=0, the cell buffer is the chunk of PF_longPackBuf, initialize it:
01088     */
01089     if ( PF_longMultiTop->bufpos > -1 )
01090         PF_longMultiTop->buffer = PF_longPackBuf+PF_longMultiTop->bufpos;
01091     /*
01092         else -- the cell has it's own private buffer.
01093         Initialize the cell fields:
01094     */
01095     PF_longMultiTop->nPacks = 0;
01096     PF_longMultiTop->lastLen = 0;
01097     PF_longMultiTop->packpos = 0;
01098     return(0);
01099 }
01100
01101 /*
01102     #] PF_longMultiPack2NextCell :
01103     #[ PF_longMultiNewChunkAdded :
01104 */
01105
01106 static inline int PF_longMultiNewChunkAdded(int n)
01107 {
01108     /*
01109         Store the list tail:

```

```

01110 */
01111 PF_LONGMULTI *MemCell = PF_longMultiLastChunk->next;
01112 int pos = PF_longPackTop;
01113
01114 while ( n-- > 0 ) {
01115 /*
01116     Allocate a new cell:
01117 */
01118     PF_longMultiLastChunk->next = (PF_LONGMULTI *)
01119         Malloc1(sizeof(PF_LONGMULTI), "PF_longMultiCell");
01120     if ( PF_longMultiLastChunk->next == NULL ) return(-1);
01121 /*
01122     Update the Last Chunk Pointer:
01123 */
01124     PF_longMultiLastChunk = PF_longMultiLastChunk->next;
01125 /*
01126     Initialize the new cell:
01127 */
01128     PF_longMultiLastChunk->bufpos = pos;
01129     pos += PF_packsize;
01130     PF_longMultiLastChunk->buffer = NULL;
01131     PF_longMultiLastChunk->packpos = 0;
01132     PF_longMultiLastChunk->nPacks = 0;
01133     PF_longMultiLastChunk->lastLen = 0;
01134 }
01135 /*
01136     Hitch the tail:
01137 */
01138     PF_longMultiLastChunk->next = MemCell;
01139     return(0);
01140 }
01141
01142 /*
01143     #] PF_longMultiNewChunkAdded :
01144     #[ PF_longCopyChunk :
01145 */
01146
01147 static inline void PF_longCopyChunk(int *to, int *from, int n)
01148 {
01149     NCOPYI(to, from, n)
01150 /* for ( ; n > 0; n-- ) *to++ = *from++; */
01151 }
01152
01153 /*
01154     #] PF_longCopyChunk :
01155     #[ PF_longAddChunk :
01156
01157     The chunk must be increased by n*PF_packsize.
01158 */
01159
01160 static int PF_longAddChunk(int n, int mustRealloc)
01161 {
01162     UBYTE *newbuf;
01163     if ( ( newbuf = (UBYTE*)Malloc1(sizeof(UBYTE)*(PF_longPackTop+n*PF_packsize),
01164         "PF_longPackBuf") ) == NULL ) return(-1);
01165 /*
01166     Allocate and chain a new cell for longMulti:
01167 */
01168     if ( PF_longMultiNewChunkAdded(n) ) return(-1);
01169 /*
01170     Copy the content to the new buffer:
01171 */
01172     if ( mustRealloc ) {
01173         PF_longCopyChunk((int*)newbuf, (int*)PF_longPackBuf, PF_longPackTop/sizeof(int));
01174     }
01175 /*
01176     Note, PF_packsize is multiple by sizeof(int) by construction!
01177 */
01178     PF_longPackTop += (n*PF_packsize);
01179 /*
01180     Free the old buffer and store the new one:
01181 */
01182     M_free(PF_longPackBuf, "PF_longPackBuf");
01183     PF_longPackBuf = newbuf;
01184 /*
01185     Count number of re-allocs:
01186 */
01187     PF_longPackN += n;
01188     return(0);
01189 }
01190
01191 /*
01192     #] PF_longAddChunk :
01193     #[ PF_longMultiHowSplit :
01194
01195     "count" of "type" elements in an input buffer occupy "bytes" bytes.
01196     We know from the algorithm, that it is too many. How to split

```

```

01197     the buffer so that the head fits to rest of a storage buffer?*/
01198 static inline int PF_longMultiHowSplit(int count, MPI_Datatype type, int bytes)
01199 {
01200     int ret, items, totalbytes;
01201
01202     if ( count < 2 ) return(0); /* Nothing to split */
01203 /*
01204     A rest of a storage buffer:
01205 */
01206     totalbytes = PF_packsize - PF_longMultiTop->packpos;
01207 /*
01208     Rough estimate:
01209 */
01210     items = (int)((double)totalbytes*count/bytes);
01211 /*
01212     Go to the up limit:
01213 */
01214     do {
01215         if ( ( ret = MPI_Pack_size(++items,type,PF_COMM,&bytes) )
01216             !=MPI_SUCCESS ) return(ret);
01217     } while ( bytes < totalbytes );
01218 /*
01219     Now the value of "items" is too large
01220     And now evaluate the exact value:
01221 */
01222     do {
01223         if ( ( ret = MPI_Pack_size(--items,type,PF_COMM,&bytes) )
01224             !=MPI_SUCCESS ) return(ret);
01225         if ( items == 0 ) /* Nothing about MPI_Pack_size(0) == 0 in standards! */
01226             return(0);
01227     } while ( bytes > totalbytes );
01228     return(items);
01229 }
01230 /*
01231     #] PF_longMultiHowSplit :
01232     #[ PF_longPackInit :
01233 */
01234
01235 static int PF_longPackInit(void)
01236 {
01237     int ret;
01238     PF_longPackBuf = (UBYTE*)Malloc1(sizeof(UBYTE)*PF_packsize,"PF_longPackBuf");
01239     if ( PF_longPackBuf == NULL ) return(-1);
01240 /*
01241     PF_longPackTop is not initialized yet, use in as a return value:
01242 */
01243     ret = MPI_Pack_size(1,MPI_INT,PF_COMM,&PF_longPackTop);
01244     if ( ret != MPI_SUCCESS ) return(ret);
01245
01246     PF_longPackSmallBuf =
01247         (void*)Malloc1(sizeof(UBYTE)*PF_longPackTop,"PF_longPackSmallBuf");
01248
01249     PF_longPackTop = PF_packsize;
01250     PF_longMultiRoot =
01251         (PF_LONGMULTI *)Malloc1(sizeof(PF_LONGMULTI),"PF_longMultiRoot");
01252     if ( PF_longMultiRoot == NULL ) return(-1);
01253     PF_longMultiRoot->bufpos = 0;
01254     PF_longMultiRoot->buffer = NULL;
01255     PF_longMultiRoot->next = NULL;
01256     PF_longMultiLastChunk = PF_longMultiRoot;
01257
01258     PF_longPackPos = 0;
01259     PF_longMultiRoot->packpos = 0;
01260     PF_longMultiTop = PF_longMultiRoot;
01261     PF_longPackN = 1;
01262     return(0);
01263 }
01264
01265 /*
01266     #] PF_longPackInit :
01267     #[ PF_longMultiPreparePrefix :
01268 */
01269
01270 static inline int PF_longMultiPreparePrefix(void)
01271 {
01272     int ret;
01273     PF_LONGMULTI *thePrefix;
01274     int i = PF_longPackN;
01275 /*
01276     Here we have PF_longPackN>1!
01277     New cell (at the list end) to create the auxiliary chunk:
01278 */
01279     if ( PF_longMultiPack2NextCell() ) return(-1);
01280 /*
01281     Store the pointer to the chunk we will proceed:
01282 */
01283     thePrefix = PF_longMultiTop;

```

```

01284 /*
01285     Pack PF_longPackN:
01286 */
01287 ret = MPI_Pack(&(PF_longPackN),
01288               1,
01289               MPI_INT,
01290               thePrefix->buffer,
01291               PF_packsize,
01292               &(thePrefix->packpos),
01293               PF_COMM);
01294 if ( ret != MPI_SUCCESS ) return(ret);
01295 /*
01296     And start from the beginning:
01297 */
01298 for ( PF_longMultiTop = PF_longMultiRoot; i > 0; i-- ) {
01299 /*
01300     Pack number of Pack hits:
01301 */
01302 ret = MPI_Pack(&(PF_longMultiTop->nPacks),
01303               1,
01304               MPI_INT,
01305               thePrefix->buffer,
01306               PF_packsize,
01307               &(thePrefix->packpos),
01308               PF_COMM);
01309 /*
01310     Pack the length of the last fit portion:
01311 */
01312 ret |= MPI_Pack(&(PF_longMultiTop->lastLen),
01313               1,
01314               MPI_INT,
01315               thePrefix->buffer,
01316               PF_packsize,
01317               &(thePrefix->packpos),
01318               PF_COMM);
01319 /*
01320     Check the size -- not necessary, MPI_Pack did it.
01321 */
01322 if ( ret != MPI_SUCCESS ) return(ret);
01323 /*
01324     Go to the next cell:
01325 */
01326 PF_longMultiTop = PF_longMultiTop->next;
01327 }
01328
01329 PF_longMultiTop = thePrefix;
01330 /*
01331     PF_longMultiTop is ready!
01332 */
01333 return(0);
01334 }
01335
01336 /*
01337     #] PF_longMultiPreparePrefix :
01338     #[ PF_longMultiProcessPrefix :
01339 */
01340
01341 static inline int PF_longMultiProcessPrefix(void)
01342 {
01343     int ret,i;
01344 /*
01345     We have PF_longPackN records packed in PF_longMultiRoot->buffer,
01346     pairs nPacks and lastLen. Loop through PF_longPackN cells,
01347     unpacking these integers into proper fields:
01348 */
01349 for ( PF_longMultiTop = PF_longMultiRoot, i = 0; i < PF_longPackN; i++ ) {
01350 /*
01351     Go to th next cell, allocating, when necessary:
01352 */
01353 if ( PF_longMultiPack2NextCell() ) return(-1);
01354 /*
01355     Unpack the number of Pack hits:
01356 */
01357 ret = MPI_Unpack(PF_longMultiRoot->buffer,
01358                 PF_packsize,
01359                 &( PF_longMultiRoot->packpos),
01360                 &(PF_longMultiTop->nPacks),
01361                 1,
01362                 MPI_INT,
01363                 PF_COMM);
01364 if ( ret != MPI_SUCCESS ) return(ret);
01365 /*
01366     Unpack the length of the last fit portion:
01367 */
01368 ret = MPI_Unpack(PF_longMultiRoot->buffer,
01369                 PF_packsize,
01370                 &( PF_longMultiRoot->packpos),

```



```

01371             &(PF_longMultiTop->lastLen),
01372             1,
01373             MPI_INT,
01374             PF_COMM);
01375         if ( ret != MPI_SUCCESS ) return(ret);
01376     }
01377     return(0);
01378 }
01379
01380 /*
01381     #] PF_longMultiProcessPrefix :
01382     #[ PF_longSingleReset :
01383 */
01384
01392 static inline int PF_longSingleReset(int is_sender)
01393 {
01394     int ret;
01395     PF_longPackPos=0;
01396     if ( is_sender ) {
01397         ret = MPI_Pack(&PF_longPackTop,1,MPI_INT,
01398             PF_longPackBuf,PF_longPackTop,&PF_longPackPos,PF_COMM);
01399         if ( ret != MPI_SUCCESS ) return(ret);
01400         PF_longPackN = 1;
01401     }
01402     else {
01403         PF_longPackN=0;
01404     }
01405     return(0);
01406 }
01407
01408 /*
01409     #] PF_longSingleReset :
01410     #[ PF_longMultiReset :
01411 */
01412
01420 static inline int PF_longMultiReset(int is_sender)
01421 {
01422     int ret = 0, theone = 1;
01423     PF_longMultiRoot->packpos = 0;
01424     if ( is_sender ) {
01425         ret = MPI_Pack(&theone,1,MPI_INT,
01426             PF_longPackBuf,PF_longPackTop,&(PF_longMultiRoot->packpos),PF_COMM);
01427         PF_longPackN = 1;
01428     }
01429     else {
01430         PF_longPackN = 0;
01431     }
01432     PF_longMultiRoot->nPacks = 0; /* The auxiliary field is not counted */
01433     PF_longMultiRoot->lastLen = 0;
01434     PF_longMultiTop = PF_longMultiRoot;
01435     PF_longMultiRoot->buffer = PF_longPackBuf;
01436     return ret;
01437 }
01438
01439 /*
01440     #] PF_longMultiReset :
01441     #] Long pack private functions :
01442     #[ PF_PrepareLongSinglePack :
01443 */
01444
01451 int PF_PrepareLongSinglePack(void)
01452 {
01453     return PF_longSingleReset(1);
01454 }
01455
01456 /*
01457     #] PF_PrepareLongSinglePack :
01458     #[ PF_LongSinglePack :
01459 */
01460
01469 int PF_LongSinglePack(const void *buffer, size_t count, MPI_Datatype type)
01470 {
01471     int ret, bytes;
01472     /* XXX: Limited by int size. */
01473     if ( count > INT_MAX ) return -99;
01474     ret = MPI_Pack_size((int)count,type,PF_COMM,&bytes);
01475     if ( ret != MPI_SUCCESS ) return(ret);
01476
01477     while ( PF_longPackPos+bytes > PF_longPackTop ) {
01478         if ( PF_longAddChunk(1, 1) ) return(-1);
01479     }
01480     /*
01481         PF_longAddChunk(1, 1) means, the chunk must
01482         be increased by 1 and re-allocated
01483     */
01484     ret = MPI_Pack((void *)buffer,(int)count,type,
01485         PF_longPackBuf,PF_longPackTop,&PF_longPackPos,PF_COMM);

```

```

01486     if ( ret != MPI_SUCCESS ) return(ret);
01487     return(0);
01488 }
01489
01490 /*
01491     #] PF_LongSinglePack :
01492     #[ PF_LongSingleUnpack :
01493 */
01494
01503 int PF_LongSingleUnpack(void *buffer, size_t count, MPI_Datatype type)
01504 {
01505     int ret;
01506     /* XXX: Limited by int size. */
01507     if ( count > INT_MAX ) return -99;
01508     ret = MPI_Unpack(PF_longPackBuf, PF_longPackTop, &PF_longPackPos,
01509                     buffer, (int)count, type, PF_COMM);
01510     if ( ret != MPI_SUCCESS ) return(ret);
01511     return(0);
01512 }
01513
01514 /*
01515     #] PF_LongSingleUnpack :
01516     #[ PF_LongSingleSend :
01517 */
01518
01540 int PF_LongSingleSend(int to, int tag)
01541 {
01542     int ret, pos = 0;
01543     /*
01544         Note, here we assume that this function couldn't be used
01545         with to == PF_ANY_SOURCE!
01546     */
01547     if ( PF_longPackN > 1 ) {
01548         /* The buffer was incremented, pack send the new size first: */
01549         int tmp = -PF_longPackTop;
01550         /*
01551             Negative value means there will be the second buffer
01552         */
01553         ret = MPI_Pack(&tmp, 1, PF_INT,
01554                       PF_longPackSmallBuf, PF_longPackTop, &pos, PF_COMM);
01555         if ( ret != MPI_SUCCESS ) return(ret);
01556         ret = MPI_Ssend(PF_longPackSmallBuf, pos, MPI_PACKED, to, tag, PF_COMM);
01557         if ( ret != MPI_SUCCESS ) return(ret);
01558     }
01559     ret = MPI_Ssend(PF_longPackBuf, PF_longPackPos, MPI_PACKED, to, tag, PF_COMM);
01560     if ( ret != MPI_SUCCESS ) return(ret);
01561     return(0);
01562 }
01563
01564 /*
01565     #] PF_LongSingleSend :
01566     #[ PF_LongSingleReceive :
01567 */
01568
01583 int PF_LongSingleReceive(int src, int tag, int *psrc, int *ptag)
01584 {
01585     int ret, missed, oncemore;
01586     MPI_Status status;
01587     PF_longSingleReset(0);
01588     do {
01589         ret = MPI_Recv(PF_longPackBuf, PF_longPackTop, MPI_PACKED, src, tag,
01590                       PF_COMM, &status);
01591         if ( ret != MPI_SUCCESS ) return(ret);
01592     /*
01593         The source and tag must be specified here for the case if
01594         MPI_Recv is performed more than once:
01595     */
01596         src = status.MPI_SOURCE;
01597         tag = status.MPI_TAG;
01598         if ( psrc ) *psrc = status.MPI_SOURCE;
01599         if ( ptag ) *ptag = status.MPI_TAG;
01600     /*
01601         Now we got either small buffer with the new PF_longPackTop,
01602         or just a regular chunk.
01603     */
01604         ret = MPI_Unpack(PF_longPackBuf, PF_longPackTop, &PF_longPackPos,
01605                           &missed, 1, MPI_INT, PF_COMM);
01606         if ( ret != MPI_SUCCESS ) return(ret);
01607     /*
01608         if ( missed < 0 ) { /* The small buffer was received. */
01609             oncemore = 1; /* repeat receiving afterwards */
01610             /* Reallocate the buffer and get the data */
01611             missed = -missed;
01612         */
01613         restore after unpacking small from buffer:
01614     */
01615         PF_longPackPos = 0;

```

```

01616     }
01617     else {
01618         oncemore = 0; /* That's all, no repetition */
01619     }
01620     if ( missed > PF_longPackTop ) {
01621         /*
01622          * The room must be increased. We need a re-allocation for the
01623          * case that there is no repetition.
01624          */
01625         if ( PF_longAddChunk( (missed-PF_longPackTop)/PF_packsize, !oncemore ) )
01626             return(-1);
01627     }
01628     } while ( oncemore );
01629     return(0);
01630 }
01631
01632 /*
01633  #] PF_LongSingleReceive :
01634  #[ PF_PrepareLongMultiPack :
01635  */
01636
01643 int PF_PrepareLongMultiPack(void)
01644 {
01645     return PF_longMultiReset(1);
01646 }
01647
01648 /*
01649  #] PF_PrepareLongMultiPack :
01650  #[ PF_LongMultiPackImpl :
01651  */
01652
01662 int PF_LongMultiPackImpl(const void*buffer, size_t count, size_t eSize, MPI_Datatype type)
01663 {
01664     int ret, items;
01665
01666     /* XXX: Limited by int size. */
01667     if ( count > INT_MAX ) return -99;
01668
01669     ret = MPI_Pack_size((int)count,type,PF_COMM,&items);
01670     if ( ret != MPI_SUCCESS ) return(ret);
01671
01672     if ( PF_longMultiTop->packpos + items <= PF_packsize ) {
01673         ret = MPI_Pack((void *)buffer,(int)count,type,PF_longMultiTop->buffer,
01674             PF_packsize,&(PF_longMultiTop->packpos),PF_COMM);
01675         if ( ret != MPI_SUCCESS ) return(ret);
01676         PF_longMultiTop->nPacks++;
01677         return(0);
01678     }
01679     /*
01680      The data do not fit to the rest of the buffer.
01681      There are two possibilities here: go to the next cell
01682      immediately, or first try to pack some portion. The function
01683      PF_longMultiHowSplit() returns the number of items could be
01684      packed in the end of the current cell:
01685     */
01686     if ( ( items = PF_longMultiHowSplit((int)count,type,items) ) < 0 ) return(items);
01687
01688     if ( items > 0 ) { /* store the head */
01689         ret = MPI_Pack((void *)buffer,items,type,PF_longMultiTop->buffer,
01690             PF_packsize,&(PF_longMultiTop->packpos),PF_COMM);
01691         if ( ret != MPI_SUCCESS ) return(ret);
01692         PF_longMultiTop->nPacks++;
01693         PF_longMultiTop->lastLen = items;
01694     }
01695     /*
01696      Now the rest should be packed to the new cell.
01697      Slide to the new cell:
01698     */
01699     if ( PF_longMultiPack2NextCell() ) return(-1);
01700     PF_longPackN++;
01701     /*
01702      Pack the rest to the next cell:
01703     */
01704     return(PF_LongMultiPackImpl((char *)buffer+items*eSize,count-items,eSize,type));
01705 }
01706
01707 /*
01708  #] PF_LongMultiPackImpl :
01709  #[ PF_LongMultiUnpackImpl :
01710  */
01711
01721 int PF_LongMultiUnpackImpl(void *buffer, size_t count, size_t eSize, MPI_Datatype type)
01722 {
01723     int ret;
01724
01725     /* XXX: Limited by int size. */
01726     if ( count > INT_MAX ) return -99;

```

```

01727
01728     if ( PF_longPackN < 2 ) { /* Just unpack the buffer from the single cell */
01729         ret = MPI_Unpack(
01730             PF_longMultiTop->buffer,
01731             PF_packsize,
01732             &(PF_longMultiTop->packpos),
01733             buffer,
01734             count,type,PF_COMM);
01735         if ( ret != MPI_SUCCESS ) return(ret);
01736         return(0);
01737     }
01738     /*
01739         More than one cell is in use.
01740     */
01741     if ( ( PF_longMultiTop->nPacks > 1 ) /* the cell is not expired */
01742         || /* The last cell contains exactly required portion: */
01743         ( ( PF_longMultiTop->nPacks == 1 ) && ( PF_longMultiTop->lastLen == 0 ) )
01744     ) { /* Just unpack the buffer from the current cell */
01745         ret = MPI_Unpack(
01746             PF_longMultiTop->buffer,
01747             PF_packsize,
01748             &(PF_longMultiTop->packpos),
01749             buffer,
01750             count,type,PF_COMM);
01751         if ( ret != MPI_SUCCESS ) return(ret);
01752         (PF_longMultiTop->nPacks)--;
01753         return(0);
01754     }
01755     if ( ( PF_longMultiTop->nPacks == 1 ) && ( PF_longMultiTop->lastLen != 0 ) ) {
01756         /*
01757             Unpack the head:
01758         */
01759         ret = MPI_Unpack(
01760             PF_longMultiTop->buffer,
01761             PF_packsize,
01762             &(PF_longMultiTop->packpos),
01763             buffer,
01764             PF_longMultiTop->lastLen,type,PF_COMM);
01765         if ( ret != MPI_SUCCESS ) return(ret);
01766         /*
01767             Decrement the counter by read items:
01768         */
01769         count -= PF_longMultiTop->lastLen;
01770         if ( count <= 0 ) return(-1); /*Something is wrong! */
01771         /*
01772             Shift the output buffer position:
01773         */
01774         buffer = (char *)buffer + PF_longMultiTop->lastLen * eSize;
01775         (PF_longMultiTop->nPacks)--;
01776     }
01777     /*
01778         Here PF_longMultiTop->nPacks == 0
01779     */
01780     if ( ( PF_longMultiTop = PF_longMultiTop->next ) == NULL ) return(-1);
01781     return(PF_LongMultiUnpackImpl(buffer,count,eSize,type));
01782 }
01783
01784 /*
01785     #] PF_LongMultiUnpackImpl :
01786     #[ PF_LongMultiBroadcast :
01787 */
01788
01807 int PF_LongMultiBroadcast(void)
01808 {
01809     int ret, i;
01810
01811     if ( PF.me == MASTER ) {
01812         /*
01813             PF_longPackN is the number of packed chunks. If it is more
01814             than 1, we have to pack a new one and send it first
01815         */
01816         if ( PF_longPackN > 1 ) {
01817             if ( PF_longMultiPreparePrefix() ) return(-1);
01818             ret = MPI_Bcast((void*)PF_longMultiTop->buffer,
01819                             PF_packsize,MPI_PACKED,MASTER,PF_COMM);
01820             if ( ret != MPI_SUCCESS ) return(ret);
01821         /*
01822             PF_longPackN was not incremented by PF_longMultiPreparePrefix()!
01823         */
01824     }
01825     /*
01826         Now we start from the beginning:
01827     */
01828     PF_longMultiTop = PF_longMultiRoot;
01829     /*
01830         Just broadcast all the chunks:
01831     */

```

```

01832         for ( i = 0; i < PF_longPackN; i++ ) {
01833             ret = MPI_Bcast((void*)PF_longMultiTop->buffer,
01834                             PF_packsize,MPI_PACKED,MASTER,PF_COMM);
01835             if ( ret != MPI_SUCCESS ) return(ret);
01836             PF_longMultiTop = PF_longMultiTop->next;
01837         }
01838         return(0);
01839     }
01840     /*
01841         else - the slave
01842     */
01843     PF_longMultiReset(0);
01844     /*
01845         Get the first chunk; it can be either the only data chunk, or
01846         an auxiliary chunk, if the data do not fit the single chunk:
01847     */
01848     ret = MPI_Bcast((void*)PF_longMultiRoot->buffer,
01849                     PF_packsize,MPI_PACKED,MASTER,PF_COMM);
01850     if ( ret != MPI_SUCCESS ) return(ret);
01851
01852     ret = MPI_Unpack((void*)PF_longMultiRoot->buffer,
01853                     PF_packsize,
01854                     &(PF_longMultiRoot->packpos),
01855                     &PF_longPackN,1,MPI_INT,PF_COMM);
01856     if ( ret != MPI_SUCCESS ) return(ret);
01857     /*
01858         Now in PF_longPackN we have the number of cells used
01859         for broadcasting. If it is >1, then we have to allocate
01860         enough cells, initialize them and receive all the chunks.
01861     */
01862     if ( PF_longPackN < 2 ) /* That's all, the single chunk is received. */
01863         return(0);
01864     /*
01865         Here we have to get PF_longPackN chunks. But, first,
01866         initialize cells by info from the received auxiliary chunk.
01867     */
01868     if ( PF_longMultiProcessPrefix() ) return(-1);
01869     /*
01870         Now we have free PF_longPackN cells, starting
01871         from PF_longMultiRoot->next, with properly initialized
01872         nPacks and lastLen fields. Get chunks:
01873     */
01874     for ( PF_longMultiTop = PF_longMultiRoot->next, i = 0; i < PF_longPackN; i++ ) {
01875         ret = MPI_Bcast((void*)PF_longMultiTop->buffer,
01876                         PF_packsize,MPI_PACKED,MASTER,PF_COMM);
01877         if ( ret != MPI_SUCCESS ) return(ret);
01878         if ( i == 0 ) { /* The first chunk, it contains extra "1". */
01879             int tmp;
01880             /*
01881                 Extract this 1 into tmp and forget about it.
01882             */
01883             ret = MPI_Unpack((void*)PF_longMultiTop->buffer,
01884                             PF_packsize,
01885                             &(PF_longMultiTop->packpos),
01886                             &tmp,1,MPI_INT,PF_COMM);
01887             if ( ret != MPI_SUCCESS ) return(ret);
01888         }
01889         PF_longMultiTop = PF_longMultiTop->next;
01890     }
01891     /*
01892         multiUnPack starts with PF_longMultiTop, skip auxiliary chunk in
01893         PF_longMultiRoot:
01894     */
01895     PF_longMultiTop = PF_longMultiRoot->next;
01896     return(0);
01897 }
01898
01899 /*
01900     #] PF_LongMultiBroadcast :
01901     #] Long pack stuff :
01902 */

```

4.80 mpidbg.h File Reference

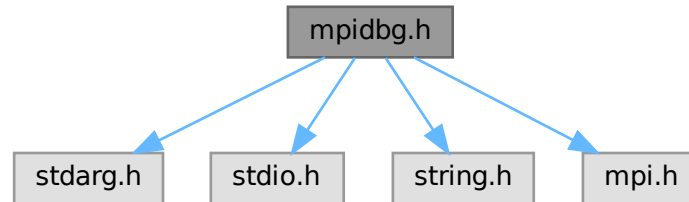
```

#include <stdarg.h>
#include <stdio.h>
#include <string.h>

```

```
#include <mpi.h>
```

Include dependency graph for mpidbg.h:



Macros

- #define [MPIDBG_RANK](#) MPIDBG_Get_rank()
- #define [MPIDBG_Out\(...\)](#) MPIDBG_Out(file, line, func, __VA_ARGS__)
- #define [MPIDBG_EXTARG](#) const char *file, int line, const char *func
- #define [MPI_Send\(...\)](#) MPIDBG_Send(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Recv\(...\)](#) MPIDBG_Recv(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Bsend\(...\)](#) MPIDBG_Bsend(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Ssend\(...\)](#) MPIDBG_Ssend(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Rsend\(...\)](#) MPIDBG_Rsend(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Isend\(...\)](#) MPIDBG_Isend(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Ibsend\(...\)](#) MPIDBG_Ibsend(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Issend\(...\)](#) MPIDBG_Issend(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Irsend\(...\)](#) MPIDBG_Irsend(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Irecv\(...\)](#) MPIDBG_Irecv(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Wait\(...\)](#) MPIDBG_Wait(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Test\(...\)](#) MPIDBG_Test(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Waitany\(...\)](#) MPIDBG_Waitany(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Testany\(...\)](#) MPIDBG_Testany(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Waitall\(...\)](#) MPIDBG_Waitall(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Testall\(...\)](#) MPIDBG_Testall(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Waitsome\(...\)](#) MPIDBG_Waitsome(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Testsome\(...\)](#) MPIDBG_Testsome(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Iprobe\(...\)](#) MPIDBG_Iprobe(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Probe\(...\)](#) MPIDBG_Probe(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Cancel\(...\)](#) MPIDBG_Cancel(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Test_cancelled\(...\)](#) MPIDBG_Test_cancelled(__VA_ARGS__, __FILE__, __LINE__, __func__,
↵
__)
- #define [MPI_Barrier\(...\)](#) MPIDBG_Barrier(__VA_ARGS__, __FILE__, __LINE__, __func__)
- #define [MPI_Bcast\(...\)](#) MPIDBG_Bcast(__VA_ARGS__, __FILE__, __LINE__, __func__)

4.80.1 Detailed Description

MPI APIs with the logging feature. NOTE: This file needs C99.

Definition in file [mpidbg.h](#).

4.80.2 Macro Definition Documentation

4.80.2.1 MPIDBG_RANK

```
#define MPIDBG_RANK MPIDBG_Get_rank()
```

Definition at line 55 of file [mpidbg.h](#).

4.80.2.2 MPIDBG_Out

```
#define MPIDBG_Out(  
    ... ) MPIDBG_Out(file, line, func, __VA_ARGS__)
```

Definition at line 72 of file [mpidbg.h](#).

4.80.2.3 MPIDBG_EXTARG

```
#define MPIDBG_EXTARG const char *file, int line, const char *func
```

Definition at line 139 of file [mpidbg.h](#).

4.80.2.4 MPI_Send

```
#define MPI_Send(  
    ... ) MPIDBG_Send(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 157 of file [mpidbg.h](#).

4.80.2.5 MPI_Recv

```
#define MPI_Recv(  
    ... ) MPIDBG_Recv(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 178 of file [mpidbg.h](#).

4.80.2.6 MPI_Bsend

```
#define MPI_Bsend(  
    ... ) MPIDBG_Bsend(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 197 of file [mpidbg.h](#).

4.80.2.7 MPI_Ssend

```
#define MPI_Ssend(  
    ... ) MPIDBG_Ssend(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 216 of file [mpidbg.h](#).

4.80.2.8 MPI_Rsend

```
#define MPI_Rsend(  
    ... ) MPIDBG_Rsend(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 235 of file [mpidbg.h](#).

4.80.2.9 MPI_Isend

```
#define MPI_Isend(  
    ... ) MPIDBG_Isend(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 254 of file [mpidbg.h](#).

4.80.2.10 MPI_Ibsend

```
#define MPI_Ibsend(  
    ... ) MPIDBG_Ibsend(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 273 of file [mpidbg.h](#).

4.80.2.11 MPI_Issend

```
#define MPI_Issend(  
    ... ) MPIDBG_Issend(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 292 of file [mpidbg.h](#).

4.80.2.12 MPI_Irsend

```
#define MPI_Irsend(  
    ... ) MPIDBG_Irsend(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 311 of file [mpidbg.h](#).

4.80.2.13 MPI_Irecv

```
#define MPI_Irecv(  
    ... ) MPIDBG_Irecv(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 330 of file [mpidbg.h](#).

4.80.2.14 MPI_Wait

```
#define MPI_Wait(  
    ... ) MPIDBG_Wait(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 353 of file [mpidbg.h](#).

4.80.2.15 MPI_Test

```
#define MPI_Test(  
    ... ) MPIDBG_Test(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 381 of file [mpidbg.h](#).

4.80.2.16 MPI_Waitany

```
#define MPI_Waitany(  
    ... ) MPIDBG_Waitany(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 407 of file [mpidbg.h](#).

4.80.2.17 MPI_Testany

```
#define MPI_Testany(  
    ... ) MPIDBG_Testany(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 438 of file [mpidbg.h](#).

4.80.2.18 MPI_Waitall

```
#define MPI_Waitall(  
    ... ) MPIDBG_Waitall(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 462 of file [mpidbg.h](#).

4.80.2.19 MPI_Testall

```
#define MPI_Testall(  
    ... ) MPIDBG_Testall(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 491 of file [mpidbg.h](#).

4.80.2.20 MPI_Waitsome

```
#define MPI_Waitsome(  
    ... ) MPIDBG_Waitsome(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 515 of file [mpidbg.h](#).

4.80.2.21 MPI_Testsome

```
#define MPI_Testsome(  
    ... ) MPIDBG_Testsome(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 539 of file [mpidbg.h](#).

4.80.2.22 MPI_Iprobe

```
#define MPI_Iprobe(  
    ... ) MPIDBG_Iprobe(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 565 of file [mpidbg.h](#).

4.80.2.23 MPI_Probe

```
#define MPI_Probe(  
    ... ) MPIDBG_Probe(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 586 of file [mpidbg.h](#).

4.80.2.24 MPI_Cancel

```
#define MPI_Cancel(  
    ... ) MPIDBG_Cancel(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 605 of file [mpidbg.h](#).

4.80.2.25 MPI_Test_cancelled

```
#define MPI_Test_cancelled(  
    ... ) MPIDBG_Test_cancelled(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 629 of file [mpidbg.h](#).

4.80.2.26 MPI_Barrier

```
#define MPI_Barrier(  
    ... ) MPIDBG_Barrier(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 648 of file [mpidbg.h](#).

4.80.2.27 MPI_Bcast

```
#define MPI_Bcast(  
    ... ) MPIDBG_Bcast(__VA_ARGS__, __FILE__, __LINE__, __func__)
```

Definition at line 667 of file [mpidbg.h](#).

4.81 mpidbg.h

[Go to the documentation of this file.](#)

```

00001 #ifndef MPIDBG_H
00002 #define MPIDBG_H
00003
00010 /* #[] License : */
00011 /*
00012  * Copyright (C) 1984-2023 J.A.M. Vermaseren
00013  * When using this file you are requested to refer to the publication
00014  * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00015  * This is considered a matter of courtesy as the development was paid
00016  * for by FOM the Dutch physics granting agency and we would like to
00017  * be able to track its scientific use to convince FOM of its value
00018  * for the community.
00019  *
00020  * This file is part of FORM.
00021  *
00022  * FORM is free software: you can redistribute it and/or modify it under the
00023  * terms of the GNU General Public License as published by the Free Software
00024  * Foundation, either version 3 of the License, or (at your option) any later
00025  * version.
00026  *
00027  * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00028  * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00029  * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00030  * details.
00031  *
00032  * You should have received a copy of the GNU General Public License along
00033  * with FORM. If not, see <http://www.gnu.org/licenses/>.
00034 */
00035 /* #[] License : */
00036
00037 /*
00038  #[] Includes :
00039 */
00040
00041 #include <stdarg.h>
00042 #include <stdio.h>
00043 #include <string.h>
00044 #include <mpi.h>
00045
00046 /*
00047  #[] Includes :
00048  #[] Utilities :
00049  #[] MPIDBG_RANK :
00050 */
00051
00052 static inline int MPIDBG_Get_rank(void) {
00053     return PF.me; /* Assume we are working with ParFORM. */
00054 }
00055 #define MPIDBG_RANK MPIDBG_Get_rank()
00056
00057 /*
00058  #[] MPIDBG_RANK :
00059  #[] MPIDBG_Out :
00060 */
00061
00062 static inline void MPIDBG_Out(const char *file, int line, const char *func, const char *fmt, ...) {
00063     char buf[1024]; /* Enough. */
00064     va_list ap;
00065     va_start(ap, fmt);
00066     snprintf(buf, 1024, "*** [%d] %10s %4d @ %-16s: ", MPIDBG_RANK, file, line, func);
00067     vsnprintf(buf + strlen(buf), 1024 - strlen(buf), fmt, ap);
00068     va_end(ap);
00069     /* Assume fprintf with a line will work well even in multi-processes. */
00070     fprintf(stderr, "%s\n", buf);
00071 }
00072 #define MPIDBG_Out(...) MPIDBG_Out(file, line, func, __VA_ARGS__)
00073
00074 /*
00075  #[] MPIDBG_Out :
00076  #[] MPIDBG_sprint_requests :
00077 */
00078
00079 static inline void MPIDBG_sprint_requests(char *buf, int count, const MPI_Request *requests)
00080 {
00081     /* Assume sprintf never fail and returns >= 0... */
00082     buf += sprintf(buf, "(");
00083     int i, first = 1;
00084     for (i = 0; i < count; i++) {
00085         if (first) {
00086             first = 0;
00087         }
00088         else {

```

```

00089         buf += sprintf(buf, ",");
00090     }
00091     if ( requests[i] != MPI_REQUEST_NULL ) {
00092         buf += sprintf(buf, "%d", i);
00093     }
00094     else {
00095         buf += sprintf(buf, "*");
00096     }
00097 }
00098 buf += sprintf(buf, " ");
00099 }
00100
00101 /*
00102     #] MPIDBG_sprint_requests :
00103     #[ MPIDBG_sprint_statuses :
00104 */
00105
00106 static inline void MPIDBG_sprint_statuses(char *buf, int count, const MPI_Request *old_requests, const
MPI_Request *new_requests, const MPI_Status *statuses)
00107 {
00108     /* Assume sprintf never fail and returns >= 0... */
00109     buf += sprintf(buf, "(");
00110     int i, first = 1;
00111     for ( i = 0; i < count; i++ ) {
00112         if ( first ) {
00113             first = 0;
00114         }
00115         else {
00116             buf += sprintf(buf, ",");
00117         }
00118         if ( old_requests[i] != MPI_REQUEST_NULL && new_requests[i] == MPI_REQUEST_NULL ) {
00119             int ret_size = 0;
00120             MPI_Get_count((MPI_Status *)&statuses[i], MPI_BYTE, &ret_size);
00121             buf += sprintf(buf, "(source=%d,tag=%d,size=%d)", statuses[i].MPI_SOURCE,
statuses[i].MPI_TAG, ret_size);
00122         } else {
00123             buf += sprintf(buf, "*");
00124         }
00125     }
00126     buf += sprintf(buf, " ");
00127 }
00128
00129 /*
00130     #] MPIDBG_sprint_statuses :
00131     #] Utilities :
00132     #[ MPI APIs :
00133 */
00134
00135 /*
00136 * The followings are the MPI APIs with the logging.
00137 */
00138
00139 #define MPIDBG_EXTARG const char *file, int line, const char *func
00140
00141 /*
00142     #[ MPI_Send :
00143 */
00144
00145 static inline int MPIDBG_Send(void *buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm
comm, MPIDBG_EXTARG)
00146 {
00147     MPIDBG_Out("MPI_Send: src=%d dest=%d tag=%d count=%d", MPIDBG_RANK, dest, tag, count);
00148     int ret = MPI_Send(buf, count, datatype, dest, tag, comm);
00149     if ( ret == MPI_SUCCESS ) {
00150         MPIDBG_Out("MPI_Send: OK");
00151     }
00152     else {
00153         MPIDBG_Out("MPI_Send: Failed");
00154     }
00155     return ret;
00156 }
00157 #define MPI_Send(...) MPIDBG_Send(__VA_ARGS__, __FILE__, __LINE__, __func__)
00158
00159 /*
00160     #] MPI_Send :
00161     #[ MPI_Recv :
00162 */
00163
00164 static inline int MPIDBG_Recv(void* buf, int count, MPI_Datatype datatype, int source, int tag,
MPI_Comm comm, MPI_Status *status, MPIDBG_EXTARG)
00165 {
00166     MPIDBG_Out("MPI_Recv: src=%d dest=%d tag=%d", source, MPIDBG_RANK, tag);
00167     int ret = MPI_Recv(buf, count, datatype, source, tag, comm, status);
00168     if ( ret == MPI_SUCCESS ) {
00169         int ret_count = 0;
00170         MPI_Get_count(status, datatype, &ret_count);
00171         MPIDBG_Out("MPI_Recv: OK src=%d dest=%d tag=%d count=%d", status->MPI_SOURCE, MPIDBG_RANK,

```

```

        status->MPI_TAG, ret_count);
00172     }
00173     else {
00174         MPIDBG_Out("MPI_Recv: Failed");
00175     }
00176     return ret;
00177 }
00178 #define MPI_Recv(...) MPIDBG_Recv(__VA_ARGS__, __FILE__, __LINE__, __func__)
00179
00180 /*
00181     #] MPI_Recv :
00182     #[ MPI_Bsend :
00183 */
00184
00185 static inline int MPIDBG_Bsend(void* buf, int count, MPI_Datatype datatype, int dest, int tag,
MPI_Comm comm, MPIDBG_EXTARG)
00186 {
00187     MPIDBG_Out("MPI_Bsend: src=%d dest=%d tag=%d count=%d", MPIDBG_RANK, dest, tag, count);
00188     int ret = MPI_Bsend(buf, count, datatype, dest, tag, comm);
00189     if ( ret == MPI_SUCCESS ) {
00190         MPIDBG_Out("MPI_Bsend: OK");
00191     }
00192     else {
00193         MPIDBG_Out("MPI_Bsend: Failed");
00194     }
00195     return ret;
00196 }
00197 #define MPI_Bsend(...) MPIDBG_Bsend(__VA_ARGS__, __FILE__, __LINE__, __func__)
00198
00199 /*
00200     #] MPI_Bsend :
00201     #[ MPI_Ssend :
00202 */
00203
00204 static inline int MPIDBG_Ssend(void* buf, int count, MPI_Datatype datatype, int dest, int tag,
MPI_Comm comm, MPIDBG_EXTARG)
00205 {
00206     MPIDBG_Out("MPI_Ssend: src=%d dest=%d tag=%d count=%d", MPIDBG_RANK, dest, tag, count);
00207     int ret = MPI_Ssend(buf, count, datatype, dest, tag, comm);
00208     if ( ret == MPI_SUCCESS ) {
00209         MPIDBG_Out("MPI_Ssend: OK");
00210     }
00211     else {
00212         MPIDBG_Out("MPI_Ssend: Failed");
00213     }
00214     return ret;
00215 }
00216 #define MPI_Ssend(...) MPIDBG_Ssend(__VA_ARGS__, __FILE__, __LINE__, __func__)
00217
00218 /*
00219     #] MPI_Ssend :
00220     #[ MPI_Rsend :
00221 */
00222
00223 static inline int MPIDBG_Rsend(void* buf, int count, MPI_Datatype datatype, int dest, int tag,
MPI_Comm comm, MPIDBG_EXTARG)
00224 {
00225     MPIDBG_Out("MPI_Rsend: src=%d dest=%d tag=%d count=%d", MPIDBG_RANK, dest, tag, count);
00226     int ret = MPI_Rsend(buf, count, datatype, dest, tag, comm);
00227     if ( ret == MPI_SUCCESS ) {
00228         MPIDBG_Out("MPI_Rsend: OK");
00229     }
00230     else {
00231         MPIDBG_Out("MPI_Rsend: Failed");
00232     }
00233     return ret;
00234 }
00235 #define MPI_Rsend(...) MPIDBG_Rsend(__VA_ARGS__, __FILE__, __LINE__, __func__)
00236
00237 /*
00238     #] MPI_Rsend :
00239     #[ MPI_Isend :
00240 */
00241
00242 static inline int MPIDBG_Isend(void* buf, int count, MPI_Datatype datatype, int dest, int tag,
MPI_Comm comm, MPI_Request *request, MPIDBG_EXTARG)
00243 {
00244     MPIDBG_Out("MPI_Isend: src=%d dest=%d tag=%d count=%d", MPIDBG_RANK, dest, tag, count);
00245     int ret = MPI_Isend(buf, count, datatype, dest, tag, comm, request);
00246     if ( ret == MPI_SUCCESS ) {
00247         MPIDBG_Out("MPI_Isend: OK");
00248     }
00249     else {
00250         MPIDBG_Out("MPI_Isend: Failed");
00251     }
00252     return ret;
00253 }

```

```

00254 #define MPI_Isend(...) MPIDBG_Isend(__VA_ARGS__, __FILE__, __LINE__, __func__)
00255
00256 /*
00257     #] MPI_Isend :
00258     #[ MPI_Ibsend :
00259 */
00260
00261 static inline int MPIDBG_Ibsend(void* buf, int count, MPI_Datatype datatype, int dest, int tag,
    MPI_Comm comm, MPI_Request *request, MPIDBG_EXTARG)
00262 {
00263     MPIDBG_Out("MPI_Ibsend: src=%d dest=%d tag=%d count=%d", MPIDBG_RANK, dest, tag, count);
00264     int ret = MPI_Ibsend(buf, count, datatype, dest, tag, comm, request);
00265     if ( ret == MPI_SUCCESS ) {
00266         MPIDBG_Out("MPI_Ibsend: OK");
00267     }
00268     else {
00269         MPIDBG_Out("MPI_Ibsend: Failed");
00270     }
00271     return ret;
00272 }
00273 #define MPI_Ibsend(...) MPIDBG_Ibsend(__VA_ARGS__, __FILE__, __LINE__, __func__)
00274
00275 /*
00276     #] MPI_Ibsend :
00277     #[ MPI_Issend :
00278 */
00279
00280 static inline int MPIDBG_Issend(void* buf, int count, MPI_Datatype datatype, int dest, int tag,
    MPI_Comm comm, MPI_Request *request, MPIDBG_EXTARG)
00281 {
00282     MPIDBG_Out("MPI_Issend: src=%d dest=%d tag=%d count=%d", MPIDBG_RANK, dest, tag, count);
00283     int ret = MPI_Issend(buf, count, datatype, dest, tag, comm, request);
00284     if ( ret == MPI_SUCCESS ) {
00285         MPIDBG_Out("MPI_Issend: OK");
00286     }
00287     else {
00288         MPIDBG_Out("MPI_Issend: Failed");
00289     }
00290     return ret;
00291 }
00292 #define MPI_Issend(...) MPIDBG_Issend(__VA_ARGS__, __FILE__, __LINE__, __func__)
00293
00294 /*
00295     #] MPI_Issend :
00296     #[ MPI_Irsend :
00297 */
00298
00299 static inline int MPIDBG_Irsend(void* buf, int count, MPI_Datatype datatype, int dest, int tag,
    MPI_Comm comm, MPI_Request *request, MPIDBG_EXTARG)
00300 {
00301     MPIDBG_Out("MPI_Irsend: src=%d dest=%d tag=%d count=%d", MPIDBG_RANK, dest, tag, count);
00302     int ret = MPI_Irsend(buf, count, datatype, dest, tag, comm, request);
00303     if ( ret == MPI_SUCCESS ) {
00304         MPIDBG_Out("MPI_Irsend: OK");
00305     }
00306     else {
00307         MPIDBG_Out("MPI_Irsend: Failed");
00308     }
00309     return ret;
00310 }
00311 #define MPI_Irsend(...) MPIDBG_Irsend(__VA_ARGS__, __FILE__, __LINE__, __func__)
00312
00313 /*
00314     #] MPI_Irsend :
00315     #[ MPI_Irecv :
00316 */
00317
00318 static inline int MPIDBG_Irecv(void* buf, int count, MPI_Datatype datatype, int source, int tag,
    MPI_Comm comm, MPI_Request *request, MPIDBG_EXTARG)
00319 {
00320     MPIDBG_Out("MPI_Irecv: src=%d dest=%d tag=%d", source, MPIDBG_RANK, tag);
00321     int ret = MPI_Irecv(buf, count, datatype, source, tag, comm, request);
00322     if ( ret == MPI_SUCCESS ) {
00323         MPIDBG_Out("MPI_Irecv: OK dest=%d", MPIDBG_RANK);
00324     }
00325     else {
00326         MPIDBG_Out("MPI_Irecv: Failed");
00327     }
00328     return ret;
00329 }
00330 #define MPI_Irecv(...) MPIDBG_Irecv(__VA_ARGS__, __FILE__, __LINE__, __func__)
00331
00332 /*
00333     #] MPI_Irecv :
00334     #[ MPI_Wait :
00335 */
00336

```

```

00337 static inline int MPIDBG_Wait(MPI_Request *request, MPI_Status *status, MPIDBG_EXTARG)
00338 {
00339     char buf[256 * 1]; /* Enough. */
00340     MPI_Request old_request = *request;
00341     MPIDBG_sprint_requests(buf, 1, request);
00342     MPIDBG_Out("MPI_Wait: rank=%d request=%s", MPIDBG_RANK, buf);
00343     int ret = MPI_Wait(request, status);
00344     if ( ret == MPI_SUCCESS ) {
00345         MPIDBG_sprint_statuses(buf, 1, request, &old_request, status);
00346         MPIDBG_Out("MPI_Wait: OK rank=%d result=%s", MPIDBG_RANK, buf);
00347     }
00348     else {
00349         MPIDBG_Out("MPI_Wait: Failed");
00350     }
00351     return ret;
00352 }
00353 #define MPI_Wait(...) MPIDBG_Wait(__VA_ARGS__, __FILE__, __LINE__, __func__)
00354
00355 /*
00356     #] MPI_Wait :
00357     #[ MPI_Test :
00358 */
00359
00360 static inline int MPIDBG_Test(MPI_Request *request, int *flag, MPI_Status *status, MPIDBG_EXTARG)
00361 {
00362     char buf[256 * 1]; /* Enough. */
00363     MPI_Request old_request = *request;
00364     MPIDBG_sprint_requests(buf, 1, request);
00365     MPIDBG_Out("MPI_Test: rank=%d request=%s", MPIDBG_RANK, buf);
00366     int ret = MPI_Test(request, flag, status);
00367     if ( ret == MPI_SUCCESS ) {
00368         if ( *flag ) {
00369             MPIDBG_sprint_statuses(buf, 1, request, &old_request, status);
00370             MPIDBG_Out("MPI_Test: OK rank=%d result=%s", MPIDBG_RANK, buf);
00371         }
00372         else {
00373             MPIDBG_Out("MPI_Test: OK flag=false");
00374         }
00375     }
00376     else {
00377         MPIDBG_Out("MPI_Test: Failed");
00378     }
00379     return ret;
00380 }
00381 #define MPI_Test(...) MPIDBG_Test(__VA_ARGS__, __FILE__, __LINE__, __func__)
00382
00383 /*
00384     #] MPI_Test :
00385     #[ MPI_Waitany :
00386 */
00387
00388 static inline int MPIDBG_Waitany(int count, MPI_Request *array_of_requests, int *index, MPI_Status
*status, MPIDBG_EXTARG)
00389 {
00390     char buf[256]; /* Enough. */
00391     MPI_Request old_requests[count];
00392     memcpy(old_requests, array_of_requests, sizeof(MPI_Request) * count);
00393     MPIDBG_sprint_requests(buf, count, array_of_requests);
00394     MPIDBG_Out("MPI_Waitany: rank=%d request=%s", MPIDBG_RANK, buf);
00395     int ret = MPI_Waitany(count, array_of_requests, index, status);
00396     if ( ret == MPI_SUCCESS ) {
00397         MPI_Status statuses[count];
00398         statuses[*index] = *status;
00399         MPIDBG_sprint_statuses(buf, count, old_requests, array_of_requests, statuses);
00400         MPIDBG_Out("MPI_Waitany: OK rank=%d result=%s", MPIDBG_RANK, buf);
00401     }
00402     else {
00403         MPIDBG_Out("MPI_Waitany: Failed");
00404     }
00405     return ret;
00406 }
00407 #define MPI_Waitany(...) MPIDBG_Waitany(__VA_ARGS__, __FILE__, __LINE__, __func__)
00408
00409 /*
00410     #] MPI_Waitany :
00411     #[ MPI_Testany :
00412 */
00413
00414 static inline int MPIDBG_Testany(int count, MPI_Request *array_of_requests, int *index, int *flag,
MPI_Status *status, MPIDBG_EXTARG)
00415 {
00416     char buf[256]; /* Enough. */
00417     MPI_Request old_requests[count];
00418     memcpy(old_requests, array_of_requests, sizeof(MPI_Request) * count);
00419     MPIDBG_sprint_requests(buf, count, array_of_requests);
00420     MPIDBG_Out("MPI_Testany: rank=%d request=%s", MPIDBG_RANK, buf);
00421     int ret = MPI_Testany(count, array_of_requests, index, flag, status);

```

```

00422     if ( ret == MPI_SUCCESS ) {
00423         if ( *flag ) {
00424             MPI_Status statuses[count];
00425             statuses[*index] = *status;
00426             MPIDBG_sprint_statuses(buf, count, old_requests, array_of_requests, statuses);
00427             MPIDBG_Out("MPI_Testany: OK rank=%d result=%s", MPIDBG_RANK, buf);
00428         }
00429         else {
00430             MPIDBG_Out("MPI_Testany: OK flag=false");
00431         }
00432     }
00433     else {
00434         MPIDBG_Out("MPI_Testany: Failed");
00435     }
00436     return ret;
00437 }
00438 #define MPI_Testany(...) MPIDBG_Testany(__VA_ARGS__, __FILE__, __LINE__, __func__)
00439
00440 /*
00441     #] MPI_Testany :
00442     #[ MPI_Waitall :
00443 */
00444
00445 static inline int MPIDBG_Waitall(int count, MPI_Request *array_of_requests, MPI_Status
*array_of_statuses, MPIDBG_EXTARG)
00446 {
00447     char buf[256 * count]; /* Enough. */
00448     MPI_Request old_requests[count];
00449     memcpy(old_requests, array_of_requests, sizeof(MPI_Request) * count);
00450     MPIDBG_sprint_requests(buf, count, array_of_requests);
00451     MPIDBG_Out("MPI_Waitall: rank=%d request=%s", MPIDBG_RANK, buf);
00452     int ret = MPI_Waitall(count, array_of_requests, array_of_statuses);
00453     if ( ret == MPI_SUCCESS ) {
00454         MPIDBG_sprint_statuses(buf, count, old_requests, array_of_requests, array_of_statuses);
00455         MPIDBG_Out("MPI_Waitall: OK rank=%d result=%s", MPIDBG_RANK, buf);
00456     }
00457     else {
00458         MPIDBG_Out("MPI_Waitall: Failed");
00459     }
00460     return ret;
00461 }
00462 #define MPI_Waitall(...) MPIDBG_Waitall(__VA_ARGS__, __FILE__, __LINE__, __func__)
00463
00464 /*
00465     #] MPI_Waitall :
00466     #[ MPI_Testall :
00467 */
00468
00469 static inline int MPIDBG_Testall(int count, MPI_Request *array_of_requests, int *flag, MPI_Status
*array_of_statuses, MPIDBG_EXTARG)
00470 {
00471     char buf[256 * count]; /* Enough. */
00472     MPI_Request old_requests[count];
00473     memcpy(old_requests, array_of_requests, sizeof(MPI_Request) * count);
00474     MPIDBG_sprint_requests(buf, count, array_of_requests);
00475     MPIDBG_Out("MPI_Testall: rank=%d request=%s", MPIDBG_RANK, buf);
00476     int ret = MPI_Testall(count, array_of_requests, flag, array_of_statuses);
00477     if ( ret == MPI_SUCCESS ) {
00478         if ( *flag ) {
00479             MPIDBG_sprint_statuses(buf, count, old_requests, array_of_requests, array_of_statuses);
00480             MPIDBG_Out("MPI_Testall: OK rank=%d result=%s", MPIDBG_RANK, buf);
00481         }
00482         else {
00483             MPIDBG_Out("MPI_Testall: OK flag=false");
00484         }
00485     }
00486     else {
00487         MPIDBG_Out("MPI_Testall: Failed");
00488     }
00489     return ret;
00490 }
00491 #define MPI_Testall(...) MPIDBG_Testall(__VA_ARGS__, __FILE__, __LINE__, __func__)
00492
00493 /*
00494     #] MPI_Testall :
00495     #[ MPI_Waitsome :
00496 */
00497
00498 static inline int MPIDBG_Waitsome(int incount, MPI_Request *array_of_requests, int *outcount, int
*array_of_indices, MPI_Status *array_of_statuses, MPIDBG_EXTARG)
00499 {
00500     char buf[256 * incount]; /* Enough. */
00501     MPI_Request old_requests[incount];
00502     memcpy(old_requests, array_of_requests, sizeof(MPI_Request) * incount);
00503     MPIDBG_sprint_requests(buf, incount, array_of_requests);
00504     MPIDBG_Out("MPI_Waitsome: rank=%d request=%s", MPIDBG_RANK, buf);
00505     int ret = MPI_Waitsome(incount, array_of_requests, outcount, array_of_indices, array_of_statuses);

```



```

00506     if ( ret == MPI_SUCCESS ) {
00507         MPIDBG_sprint_statuses(buf, incount, old_requests, array_of_requests, array_of_statuses);
00508         MPIDBG_Out("MPI_Waitosome: OK rank=%d result=%s", MPIDBG_RANK, buf);
00509     }
00510     else {
00511         MPIDBG_Out("MPI_Waitosome: Failed");
00512     }
00513     return ret;
00514 }
00515 #define MPI_Waitosome(...) MPIDBG_Waitosome(__VA_ARGS__, __FILE__, __LINE__, __func__)
00516
00517 /*
00518     #] MPI_Waitosome :
00519     #[ MPI_Testsome :
00520 */
00521
00522 static inline int MPIDBG_Testsome(int incount, MPI_Request *array_of_requests, int *outcount, int
    *array_of_indices, MPI_Status *array_of_statuses, MPIDBG_EXTARG)
00523 {
00524     char buf[256 * incount]; /* Enough. */
00525     MPI_Request old_requests[incount];
00526     memcpy(old_requests, array_of_requests, sizeof(MPI_Request) * incount);
00527     MPIDBG_sprint_requests(buf, incount, array_of_requests);
00528     MPIDBG_Out("MPI_Testsome: rank=%d request=%s", MPIDBG_RANK, buf);
00529     int ret = MPI_Testsome(incount, array_of_requests, outcount, array_of_indices, array_of_statuses);
00530     if ( ret == MPI_SUCCESS ) {
00531         MPIDBG_sprint_statuses(buf, incount, old_requests, array_of_requests, array_of_statuses);
00532         MPIDBG_Out("MPI_Testsome: OK rank=%d result=%s", MPIDBG_RANK, buf);
00533     }
00534     else {
00535         MPIDBG_Out("MPI_Testsome: Failed");
00536     }
00537     return ret;
00538 }
00539 #define MPI_Testsome(...) MPIDBG_Testsome(__VA_ARGS__, __FILE__, __LINE__, __func__)
00540
00541 /*
00542     #] MPI_Testsome :
00543     #[ MPI_Iprobe :
00544 */
00545
00546 static inline int MPIDBG_Iprobe(int source, int tag, MPI_Comm comm, int *flag, MPI_Status *status,
    MPIDBG_EXTARG)
00547 {
00548     MPIDBG_Out("MPI_Iprobe: src=%d dest=%d tag=%d", source, MPIDBG_RANK, tag);
00549     int ret = MPI_Iprobe(source, tag, comm, flag, status);
00550     if ( ret == MPI_SUCCESS ) {
00551         if ( *flag ) {
00552             int ret_size = 0;
00553             MPI_Get_count(status, MPI_BYTE, &ret_size);
00554             MPIDBG_Out("MPI_Iprobe: OK src=%d dest=%d tag=%d size=%d", status->MPI_SOURCE,
    MPIDBG_RANK, status->MPI_TAG, ret_size);
00555         }
00556         else {
00557             MPIDBG_Out("MPI_Iprobe: OK flag=false");
00558         }
00559     }
00560     else {
00561         MPIDBG_Out("MPI_Iprobe: Failed");
00562     }
00563     return ret;
00564 }
00565 #define MPI_Iprobe(...) MPIDBG_Iprobe(__VA_ARGS__, __FILE__, __LINE__, __func__)
00566
00567 /*
00568     #] MPI_Iprobe :
00569     #[ MPI_Probe :
00570 */
00571
00572 static inline int MPIDBG_Probe(int source, int tag, MPI_Comm comm, MPI_Status *status, MPIDBG_EXTARG)
00573 {
00574     MPIDBG_Out("MPI_Probe: src=%d dest=%d tag=%d", source, MPIDBG_RANK, tag);
00575     int ret = MPI_Probe(source, tag, comm, status);
00576     if ( ret == MPI_SUCCESS ) {
00577         int ret_size = 0;
00578         MPI_Get_count(status, MPI_BYTE, &ret_size);
00579         MPIDBG_Out("MPI_Probe: OK src=%d dest=%d tag=%d size=%d", status->MPI_SOURCE, MPIDBG_RANK,
    status->MPI_TAG, ret_size);
00580     }
00581     else {
00582         MPIDBG_Out("MPI_Probe: Failed");
00583     }
00584     return ret;
00585 }
00586 #define MPI_Probe(...) MPIDBG_Probe(__VA_ARGS__, __FILE__, __LINE__, __func__)
00587
00588 /*

```

```

00589         #] MPI_Probe :
00590         #[ MPI_Cancel :
00591     */
00592
00593     static inline int MPIDBG_Cancel(MPI_Request *request, MPIDBG_EXTARG)
00594     {
00595         MPIDBG_Out("MPI_Cancel: rank=%d", MPIDBG_RANK);
00596         int ret = MPI_Cancel(request);
00597         if ( ret == MPI_SUCCESS ) {
00598             MPIDBG_Out("MPI_Cancel: OK");
00599         }
00600         else {
00601             MPIDBG_Out("MPI_Cancel: Failed");
00602         }
00603         return ret;
00604     }
00605     #define MPI_Cancel(...) MPIDBG_Cancel(__VA_ARGS__, __FILE__, __LINE__, __func__)
00606
00607     /*
00608         #] MPI_Cancel :
00609         #[ MPI_Test_cancelled :
00610     */
00611
00612     static inline int MPIDBG_Test_cancelled(MPI_Status *status, int *flag, MPIDBG_EXTARG)
00613     {
00614         MPIDBG_Out("MPI_Test_cancelled: rank=%d", MPIDBG_RANK);
00615         int ret = MPI_Test_cancelled(status, flag);
00616         if ( ret == MPI_SUCCESS ) {
00617             if ( *flag ) {
00618                 MPIDBG_Out("MPI_Test_cancelled: OK flag=true");
00619             }
00620             else {
00621                 MPIDBG_Out("MPI_Test_cancelled: OK flag=false");
00622             }
00623         }
00624         else {
00625             MPIDBG_Out("MPI_Test_cancelled: Failed");
00626         }
00627         return ret;
00628     }
00629     #define MPI_Test_cancelled(...) MPIDBG_Test_cancelled(__VA_ARGS__, __FILE__, __LINE__, __func__)
00630
00631     /*
00632         #] MPI_Test_cancelled :
00633         #[ MPI_Barrier :
00634     */
00635
00636     static inline int MPIDBG_Barrier(MPI_Comm comm, MPIDBG_EXTARG)
00637     {
00638         MPIDBG_Out("MPI_Barrier: rank=%d", MPIDBG_RANK);
00639         int ret = MPI_Barrier(comm);
00640         if ( ret == MPI_SUCCESS ) {
00641             MPIDBG_Out("MPI_Barrier: OK");
00642         }
00643         else {
00644             MPIDBG_Out("MPI_Barrier: Failed");
00645         }
00646         return ret;
00647     }
00648     #define MPI_Barrier(...) MPIDBG_Barrier(__VA_ARGS__, __FILE__, __LINE__, __func__)
00649
00650     /*
00651         #] MPI_Barrier :
00652         #[ MPI_Bcast :
00653     */
00654
00655     static inline int MPIDBG_Bcast(void* buffer, int count, MPI_Datatype datatype, int root, MPI_Comm
comm, MPIDBG_EXTARG)
00656     {
00657         MPIDBG_Out("MPI_Bcast: root=%d count=%d", root, count);
00658         int ret = MPI_Bcast(buffer, count, datatype, root, comm);
00659         if ( ret == MPI_SUCCESS ) {
00660             MPIDBG_Out("MPI_Bcast: OK");
00661         }
00662         else {
00663             MPIDBG_Out("MPI_Bcast: Failed");
00664         }
00665         return ret;
00666     }
00667     #define MPI_Bcast(...) MPIDBG_Bcast(__VA_ARGS__, __FILE__, __LINE__, __func__)
00668
00669     /*
00670         #] MPI_Bcast :
00671         #] MPI APIs :
00672     */
00673
00674     #endif /* MPIDBG_H */

```

4.82 mytime.cc

```

00001 #ifdef HAVE_CONFIG_H
00002 #include <config.h>
00003 #endif
00004
00005 // A timing routine for debugging. Only on Unix (where sys/time.h is available).
00006 #ifdef UNIX
00007
00008 #include <sys/time.h>
00009 #include <cstdlib>
00010 #include <cstdio>
00011 #include <string>
00012
00013 #ifndef timersub
00014 /* timersub is not in POSIX, but presents on most BSD derivatives.
00015    This implementation is borrowed from glibc. (TU 23 Oct 2011) */
00016 #define timersub(a, b, result) \
00017     do { \
00018         (result)->tv_sec = (a)->tv_sec - (b)->tv_sec; \
00019         (result)->tv_usec = (a)->tv_usec - (b)->tv_usec; \
00020         if ((result)->tv_usec < 0) { \
00021             --(result)->tv_sec; \
00022             (result)->tv_usec += 1000000; \
00023         } \
00024     } while (0)
00025 #endif
00026
00027 bool starttime_set = false;
00028 timeval starttime;
00029
00030 double thetime () {
00031     if (!starttime_set) {
00032         gettimeofday(&starttime, NULL);
00033         starttime_set=true;
00034     }
00035
00036     timeval now,diff;
00037     gettimeofday(&now, NULL);
00038     timersub(&now,&starttime,&diff);
00039     return diff.tv_sec+diff.tv_usec/1000000.0;
00040 }
00041
00042 std::string thetime_str() {
00043     char res[10];
00044     snprintf (res,10,"%4.1f", thetime());
00045     return res;
00046 }
00047
00048 #endif // UNIX

```

4.83 mytime.h

```

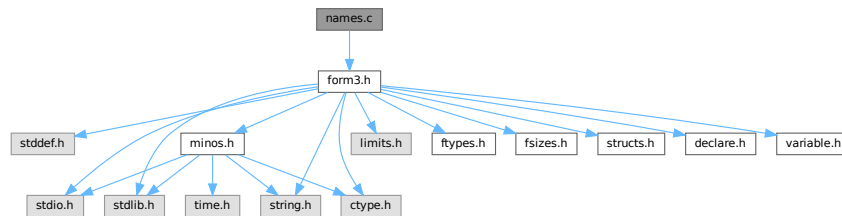
00001 #pragma once
00002 /* #[] License : */
00003 /*
00004  * Copyright (C) 1984-2023 J.A.M. Vermaseren
00005  * When using this file you are requested to refer to the publication
00006  * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00007  * This is considered a matter of courtesy as the development was paid
00008  * for by FOM the Dutch physics granting agency and we would like to
00009  * be able to track its scientific use to convince FOM of its value
00010  * for the community.
00011  *
00012  * This file is part of FORM.
00013  *
00014  * FORM is free software: you can redistribute it and/or modify it under the
00015  * terms of the GNU General Public License as published by the Free Software
00016  * Foundation, either version 3 of the License, or (at your option) any later
00017  * version.
00018  *
00019  * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00020  * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00021  * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00022  * details.
00023  *
00024  * You should have received a copy of the GNU General Public License along
00025  * with FORM. If not, see <http://www.gnu.org/licenses/>.
00026  */
00027 /* #[] License : */
00028
00029 double thetime();
00030 std::string thetime_str();

```

4.84 names.c File Reference

```
#include "form3.h"
```

Include dependency graph for names.c:



Functions

- [NAMENODE](#) * [GetNode](#) ([NAMETREE](#) *nametree, UBYTE *name)
- int [AddName](#) ([NAMETREE](#) *nametree, UBYTE *name, WORD type, WORD number, int *nodenum)
- int [GetName](#) ([NAMETREE](#) *nametree, UBYTE *namein, WORD *number, int par)
- UBYTE * [GetFunction](#) (UBYTE *s, WORD *funnum)
- UBYTE * [GetNumber](#) (UBYTE *s, WORD *num)
- int [GetLastExprName](#) (UBYTE *name, WORD *number)
- int [GetOName](#) ([NAMETREE](#) *nametree, UBYTE *name, WORD *number, int par)
- int [GetAutoName](#) (UBYTE *name, WORD *number)
- int [GetVar](#) (UBYTE *name, WORD *type, WORD *number, int wantedtype, int par)
- WORD [EntVar](#) (WORD type, UBYTE *name, WORD x, WORD y, WORD z, WORD d)
- int [GetDollar](#) (UBYTE *name)
- void [DumpTree](#) ([NAMETREE](#) *nametree)
- void [DumpNode](#) ([NAMETREE](#) *nametree, WORD node, WORD depth)
- int [CompactifyTree](#) ([NAMETREE](#) *nametree, WORD par)
- void [CopyTree](#) ([NAMETREE](#) *newtree, [NAMETREE](#) *oldtree, WORD node, WORD par)
- void [LinkTree](#) ([NAMETREE](#) *tree, WORD offset, WORD numnodes)
- [NAMETREE](#) * [MakeNameTree](#) (void)
- void [FreeNameTree](#) ([NAMETREE](#) *n)
- void [ClearWildcardNames](#) (void)
- int [AddWildcardName](#) (UBYTE *name)
- int [GetWildcardName](#) (UBYTE *name)
- int [AddSymbol](#) (UBYTE *name, int minpow, int maxpow, int cplx, int dim)
- int [CoSymbol](#) (UBYTE *s)
- int [AddIndex](#) (UBYTE *name, int dim, int dim4)
- int [CoIndex](#) (UBYTE *s)
- UBYTE * [DoDimension](#) (UBYTE *s, int *dim, int *dim4)
- int [CoDimension](#) (UBYTE *s)
- int [AddVector](#) (UBYTE *name, int cplx, int dim)
- int [CoVector](#) (UBYTE *s)
- int [AddFunction](#) (UBYTE *name, int comm, int istensor, int cplx, int symprop, int dim, int argmax, int argmin)
- int [CoCommutelnSet](#) (UBYTE *s)
- int [CoFunction](#) (UBYTE *s, int comm, int istensor)
- int [CoNFunction](#) (UBYTE *s)
- int [CoCFunction](#) (UBYTE *s)
- int [CoNTensor](#) (UBYTE *s)

- int [CoCTensor](#) (UBYTE *s)
- int [DoTable](#) (UBYTE *s, int par)
- int [CoTable](#) (UBYTE *s)
- int [CoNTable](#) (UBYTE *s)
- int [CoCTable](#) (UBYTE *s)
- void [EmptyTable](#) ([TABLES](#) T)
- int [AddSet](#) (UBYTE *name, WORD dim)
- int [DoElements](#) (UBYTE *s, [SETS](#) set, UBYTE *name)
- int [CoSet](#) (UBYTE *s)
- int [DoTempSet](#) (UBYTE *from, UBYTE *to)
- int [CoAuto](#) (UBYTE *inp)
- int [AddDollar](#) (UBYTE *name, WORD type, WORD *start, LONG size)
- int [ReplaceDollar](#) (WORD number, WORD newtype, WORD *newstart, LONG newsize)
- int [AddDubious](#) (UBYTE *name)
- int [MakeDubious](#) ([NAMETREE](#) *nametree, UBYTE *name, WORD *number)
- int [NameConflict](#) (int type, UBYTE *name)
- int [AddExpression](#) (UBYTE *name, int x, int y)
- int [GetLabel](#) (UBYTE *name)
- void [ResetVariables](#) (int par)
- void [RemoveDollars](#) (void)
- void [Globalize](#) (int par)
- int [TestName](#) (UBYTE *name)

4.84.1 Detailed Description

The complete names administration. All variables with a name have to pass here to be properly registered, have structs of the proper type assigned to them etc. The file also contains the utility routines for maintaining the balanced trees that make searching for names rather fast.

Definition in file [names.c](#).

4.84.2 Function Documentation

4.84.2.1 GetNode()

```
NAMENODE * GetNode (
    NAMETREE * nametree,
    UBYTE * name )
```

Definition at line 49 of file [names.c](#).

4.84.2.2 AddName()

```
int AddName (
    NAMETREE * nametree,
    UBYTE * name,
    WORD type,
    WORD number,
    int * nodenum )
```

Definition at line 71 of file [names.c](#).

4.84.2.3 GetName()

```
int GetName (
    NAMETREE * nametree,
    UBYTE * namein,
    WORD * number,
    int par )
```

Definition at line 252 of file [names.c](#).

4.84.2.4 GetFunction()

```
UBYTE * GetFunction (
    UBYTE * s,
    WORD * funnum )
```

Definition at line 342 of file [names.c](#).

4.84.2.5 GetNumber()

```
UBYTE * GetNumber (
    UBYTE * s,
    WORD * num )
```

Definition at line 388 of file [names.c](#).

4.84.2.6 GetLastExprName()

```
int GetLastExprName (
    UBYTE * name,
    WORD * number )
```

Definition at line 438 of file [names.c](#).

4.84.2.7 GetOName()

```
int GetOName (
    NAMETREE * nametree,
    UBYTE * name,
    WORD * number,
    int par )
```

Definition at line 460 of file [names.c](#).

4.84.2.8 GetAutoName()

```
int GetAutoName (
    UBYTE * name,
    WORD * number )
```

Definition at line 479 of file [names.c](#).

4.84.2.9 GetVar()

```
int GetVar (
    UBYTE * name,
    WORD * type,
    WORD * number,
    int wantedtype,
    int par )
```

Definition at line [524](#) of file [names.c](#).

4.84.2.10 EntVar()

```
WORD EntVar (
    WORD type,
    UBYTE * name,
    WORD x,
    WORD y,
    WORD z,
    WORD d )
```

Definition at line [557](#) of file [names.c](#).

4.84.2.11 GetDollar()

```
int GetDollar (
    UBYTE * name )
```

Definition at line [590](#) of file [names.c](#).

4.84.2.12 DumpTree()

```
void DumpTree (
    NAMETREE * nametree )
```

Definition at line [602](#) of file [names.c](#).

4.84.2.13 DumpNode()

```
void DumpNode (
    NAMETREE * nametree,
    WORD node,
    WORD depth )
```

Definition at line [615](#) of file [names.c](#).

4.84.2.14 CompactifyTree()

```
int CompactifyTree (
    NAMETREE * nametree,
    WORD par )
```

Definition at line 634 of file [names.c](#).

4.84.2.15 CopyTree()

```
void CopyTree (
    NAMETREE * newtree,
    NAMETREE * oldtree,
    WORD node,
    WORD par )
```

Definition at line 696 of file [names.c](#).

4.84.2.16 LinkTree()

```
void LinkTree (
    NAMETREE * tree,
    WORD offset,
    WORD numnodes )
```

Definition at line 784 of file [names.c](#).

4.84.2.17 MakeNameTree()

```
NAMETREE * MakeNameTree (
    void )
```

Definition at line 815 of file [names.c](#).

4.84.2.18 FreeNameTree()

```
void FreeNameTree (
    NAMETREE * n )
```

Definition at line 834 of file [names.c](#).

4.84.2.19 ClearWildcardNames()

```
void ClearWildcardNames (
    void )
```

Definition at line 849 of file [names.c](#).

4.84.2.20 AddWildcardName()

```
int AddWildcardName (
    UBYTE * name )
```

Definition at line [854](#) of file [names.c](#).

4.84.2.21 GetWildcardName()

```
int GetWildcardName (
    UBYTE * name )
```

Definition at line [898](#) of file [names.c](#).

4.84.2.22 AddSymbol()

```
int AddSymbol (
    UBYTE * name,
    int minpow,
    int maxpow,
    int cplx,
    int dim )
```

Definition at line [920](#) of file [names.c](#).

4.84.2.23 CoSymbol()

```
int CoSymbol (
    UBYTE * s )
```

Definition at line [946](#) of file [names.c](#).

4.84.2.24 AddIndex()

```
int AddIndex (
    UBYTE * name,
    int dim,
    int dim4 )
```

Definition at line [1088](#) of file [names.c](#).

4.84.2.25 CoIndex()

```
int CoIndex (
    UBYTE * s )
```

Definition at line [1112](#) of file [names.c](#).

4.84.2.26 DoDimension()

```
UBYTE * DoDimension (
    UBYTE * s,
    int * dim,
    int * dim4 )
```

Definition at line 1160 of file [names.c](#).

4.84.2.27 CoDimension()

```
int CoDimension (
    UBYTE * s )
```

Definition at line 1226 of file [names.c](#).

4.84.2.28 AddVector()

```
int AddVector (
    UBYTE * name,
    int cplx,
    int dim )
```

Definition at line 1244 of file [names.c](#).

4.84.2.29 CoVector()

```
int CoVector (
    UBYTE * s )
```

Definition at line 1267 of file [names.c](#).

4.84.2.30 AddFunction()

```
int AddFunction (
    UBYTE * name,
    int comm,
    int istensor,
    int cplx,
    int symprop,
    int dim,
    int argmax,
    int argmin )
```

Definition at line 1326 of file [names.c](#).

4.84.2.31 CoCommutateInSet()

```
int CoCommutateInSet (
    UBYTE * s )
```

Definition at line 1356 of file [names.c](#).

4.84.2.32 CoFunction()

```
int CoFunction (
    UBYTE * s,
    int comm,
    int istensor )
```

Definition at line 1446 of file [names.c](#).

4.84.2.33 CoNFunction()

```
int CoNFunction (
    UBYTE * s )
```

Definition at line 1624 of file [names.c](#).

4.84.2.34 CoCFunction()

```
int CoCFunction (
    UBYTE * s )
```

Definition at line 1625 of file [names.c](#).

4.84.2.35 CoNTensor()

```
int CoNTensor (
    UBYTE * s )
```

Definition at line 1626 of file [names.c](#).

4.84.2.36 CoCTensor()

```
int CoCTensor (
    UBYTE * s )
```

Definition at line 1627 of file [names.c](#).

4.84.2.37 DoTable()

```
int DoTable (
    UBYTE * s,
    int par )
```

Definition at line 1666 of file [names.c](#).

4.84.2.38 CoTable()

```
int CoTable (
    UBYTE * s )
```

Definition at line 2048 of file [names.c](#).

4.84.2.39 CoNTable()

```
int CoNTable (
    UBYTE * s )
```

Definition at line 2058 of file [names.c](#).

4.84.2.40 CoCTable()

```
int CoCTable (
    UBYTE * s )
```

Definition at line 2068 of file [names.c](#).

4.84.2.41 EmptyTable()

```
void EmptyTable (
    TABLES T )
```

Definition at line 2078 of file [names.c](#).

4.84.2.42 AddSet()

```
int AddSet (
    UBYTE * name,
    WORD dim )
```

Definition at line 2122 of file [names.c](#).

4.84.2.43 DoElements()

```
int DoElements (
    UBYTE * s,
    SETS set,
    UBYTE * name )
```

Definition at line 2155 of file [names.c](#).

4.84.2.44 CoSet()

```
int CoSet (
    UBYTE * s )
```

Definition at line 2355 of file [names.c](#).

4.84.2.45 DoTempSet()

```
int DoTempSet (
    UBYTE * from,
    UBYTE * to )
```

Definition at line 2480 of file [names.c](#).

4.84.2.46 CoAuto()

```
int CoAuto (
    UBYTE * inp )
```

Definition at line 2572 of file [names.c](#).

4.84.2.47 AddDollar()

```
int AddDollar (
    UBYTE * name,
    WORD type,
    WORD * start,
    LONG size )
```

Definition at line 2602 of file [names.c](#).

4.84.2.48 ReplaceDollar()

```
int ReplaceDollar (
    WORD number,
    WORD newtype,
    WORD * newstart,
    LONG newsize )
```

Definition at line 2646 of file [names.c](#).

4.84.2.49 AddDubious()

```
int AddDubious (
    UBYTE * name )
```

Definition at line 2679 of file [names.c](#).

4.84.2.50 MakeDubious()

```
int MakeDubious (
    NAMETREE * nametree,
    UBYTE * name,
    WORD * number )
```

Definition at line [2693](#) of file [names.c](#).

4.84.2.51 NameConflict()

```
int NameConflict (
    int type,
    UBYTE * name )
```

Definition at line [2728](#) of file [names.c](#).

4.84.2.52 AddExpression()

```
int AddExpression (
    UBYTE * name,
    int x,
    int y )
```

Definition at line [2744](#) of file [names.c](#).

4.84.2.53 GetLabel()

```
int GetLabel (
    UBYTE * name )
```

Definition at line [2787](#) of file [names.c](#).

4.84.2.54 ResetVariables()

```
void ResetVariables (
    int par )
```

Definition at line [2832](#) of file [names.c](#).

4.84.2.55 RemoveDollars()

```
void RemoveDollars (
    void )
```

Definition at line [3128](#) of file [names.c](#).

4.84.2.56 Globalize()

```
void Globalize (
    int par )
```

Definition at line [3155](#) of file [names.c](#).

4.84.2.57 TestName()

```
int TestName (
    UBYTE * name )
```

Definition at line [3254](#) of file [names.c](#).

4.85 names.c

[Go to the documentation of this file.](#)

```
00001
00009 /* #[] License : */
00010 /*
00011 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00012 * When using this file you are requested to refer to the publication
00013 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00014 * This is considered a matter of courtesy as the development was paid
00015 * for by FOM the Dutch physics granting agency and we would like to
00016 * be able to track its scientific use to convince FOM of its value
00017 * for the community.
00018 *
00019 * This file is part of FORM.
00020 *
00021 * FORM is free software: you can redistribute it and/or modify it under the
00022 * terms of the GNU General Public License as published by the Free Software
00023 * Foundation, either version 3 of the License, or (at your option) any later
00024 * version.
00025 *
00026 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00027 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00028 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00029 * details.
00030 *
00031 * You should have received a copy of the GNU General Public License along
00032 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00033 */
00034 /* #[] License : */
00035 /*
00036 #[] Includes :
00037 */
00038
00039 #include "form3.h"
00040
00041 /* EXTERNLOCK(dummylock) */
00042
00043 /*
00044 #[] Includes :
00045
00046 #[] GetNode :
00047 */
00048
00049 NAMENODE *GetNode(NAMETREE *nametree, UBYTE *name)
00050 {
00051     NAMENODE *n;
00052     int node, newnode, i;
00053     if ( nametree->namenode == 0 ) return(0);
00054     newnode = nametree->headnode;
00055     do {
00056         node = newnode;
00057         n = nametree->namenode+node;
00058         if ( ( i = StrCmp(name,nametree->namebuffer+n->name) ) < 0 )
00059             newnode = n->left;
00060         else if ( i > 0 ) newnode = n->right;
00061         else { return(n); }
00062     } while ( newnode >= 0 );
```

```

00063     return(0);
00064 }
00065
00066 /*
00067 #] GetNode :
00068 #[ AddName :
00069 */
00070
00071 int AddName(NAMETREE *nametree, UBYTE *name, WORD type, WORD number, int *nodenum)
00072 {
00073     NAMENODE *n, *nn, *nnn;
00074     UBYTE *s, *ss, *sss;
00075     LONG *c1, *c2, j, newsize;
00076     int node, newnode, node3, r, rr = 0, i, retval = 0;
00077     if ( nametree->namenode == 0 ) {
00078         s = name; i = 1; while ( *s ) { i++; s++; }
00079         j = INITNAMESIZE;
00080         if ( i > j ) j = i;
00081         nametree->namenode = (NAMENODE *)Mallocl(INITNODESIZE*sizeof(NAMENODE),
00082             "new nametree in AddName");
00083         nametree->namebuffer = (UBYTE *)Mallocl(j,
00084             "new namebuffer in AddName");
00085         nametree->nodesize = INITNODESIZE;
00086         nametree->namesize = j;
00087         nametree->namefill = i;
00088         nametree->nodefill = 1;
00089         nametree->headnode = 0;
00090         n = nametree->namenode;
00091         n->parent = n->left = n->right = -1;
00092         n->balance = 0;
00093         n->type = type;
00094         n->number = number;
00095         n->name = 0;
00096         s = name;
00097         ss = nametree->namebuffer;
00098         while ( *s ) *ss++ = *s++;
00099         *ss = 0;
00100         *nodenum = 0;
00101         return(retval);
00102     }
00103     newnode = nametree->headnode;
00104     do {
00105         node = newnode;
00106         n = nametree->namenode+node;
00107         if ( StrCmp(name, nametree->namebuffer+n->name) < 0 ) {
00108             newnode = n->left; r = -1;
00109         }
00110         else {
00111             newnode = n->right; r = 1;
00112         }
00113     } while ( newnode >= 0 );
00114     /*
00115     We are at the insertion point. Add the node.
00116     */
00117     if ( nametree->nodefill >= nametree->nodesize ) { /* Double allocation */
00118         newsize = nametree->nodesize * 2;
00119         if ( newsize > MAXINNAMETREE ) newsize = MAXINNAMETREE;
00120         if ( nametree->nodefill >= MAXINNAMETREE ) {
00121             MesPrint("!!!More than %l names in one object", (LONG)MAXINNAMETREE);
00122             Terminate(-1);
00123         }
00124         nnn = (NAMENODE *)Mallocl(2*((LONG)newsize*sizeof(NAMENODE)),
00125             "extra names in AddName");
00126         c1 = (LONG *)nnn; c2 = (LONG *)nametree->namenode;
00127         i = (nametree->nodefill * sizeof(NAMENODE))/sizeof(LONG);
00128         while ( --i >= 0 ) *c1++ = *c2++;
00129         M_free(nametree->namenode, "nametree->namenode");
00130         nametree->namenode = nnn;
00131         nametree->nodesize = newsize;
00132         n = nametree->namenode+node;
00133     }
00134     *nodenum = newnode = nametree->nodefill++;
00135     nn = nametree->namenode+newnode;
00136     nn->parent = node;
00137     if ( r < 0 ) n->left = newnode; else n->right = newnode;
00138     nn->left = nn->right = -1;
00139     nn->type = type;
00140     nn->number = number;
00141     nn->balance = 0;
00142     i = 1; s = name; while ( *s ) { i++; s++; }
00143     while ( nametree->namefill + i >= nametree->namesize ) { /* Double alloc */
00144         sss = (UBYTE *)Mallocl(2*nametree->namesize,
00145             "extra names in AddName");
00146         s = sss; ss = nametree->namebuffer; j = nametree->namefill;
00147         while ( --j >= 0 ) *s++ = *ss++;
00148         M_free(nametree->namebuffer, "nametree->namebuffer");
00149         nametree->namebuffer = sss;

```



```

00150     nametree->namesize *= 2;
00151 }
00152 s = nametree->namebuffer+nametree->namefill;
00153 nn->name = nametree->namefill;
00154 retval = nametree->namefill;
00155 nametree->namefill += i;
00156 while ( *name ) *s++ = *name++;
00157 *s = 0;
00158 /*
00159 Adjust the balance factors
00160 */
00161 while ( node >= 0 ) {
00162     n = nametree->namenode + node;
00163     if ( newnode == n->left ) rr = -1;
00164     else rr = 1;
00165     if ( n->balance == -rr ) { n->balance = 0; return(retval); }
00166     else if ( n->balance == rr ) break;
00167     n->balance = rr;
00168     newnode = node;
00169     node = n->parent;
00170 }
00171 if ( node < 0 ) return(retval);
00172 /*
00173 We have to rebalance the tree. There are two basic operations.
00174 n/node is the unbalanced node. newnode is its child.
00175 rr is the old balance of n/node.
00176 */
00177 nn = nametree->namenode + newnode;
00178 if ( nn->balance == -rr ) { /* The difficult case */
00179     if ( rr > 0 ) {
00180         node3 = nn->left;
00181         nnn = nametree->namenode + node3;
00182         nnn->parent = n->parent;
00183         n->parent = nn->parent = node3;
00184         if ( nnn->right >= 0 ) nametree->namenode[nnn->right].parent = newnode;
00185         if ( nnn->left >= 0 ) nametree->namenode[nnn->left].parent = node;
00186         n->right = nnn->left; nnn->left = node;
00187         nn->left = nnn->right; nnn->right = newnode;
00188         if ( nnn->balance > 0 ) { n->balance = -1; nn->balance = 0; }
00189         else if ( nnn->balance == 0 ) { n->balance = nn->balance = 0; }
00190         else { nn->balance = 1; n->balance = 0; }
00191     }
00192     else {
00193         node3 = nn->right;
00194         nnn = nametree->namenode + node3;
00195         nnn->parent = n->parent;
00196         n->parent = nn->parent = node3;
00197         if ( nnn->right >= 0 ) nametree->namenode[nnn->right].parent = node;
00198         if ( nnn->left >= 0 ) nametree->namenode[nnn->left].parent = newnode;
00199         n->left = nnn->right; nnn->right = node;
00200         nn->right = nnn->left; nnn->left = newnode;
00201         if ( nnn->balance < 0 ) { n->balance = 1; nn->balance = 0; }
00202         else if ( nnn->balance == 0 ) { n->balance = nn->balance = 0; }
00203         else { nn->balance = -1; n->balance = 0; }
00204     }
00205     nnn->balance = 0;
00206     if ( nnn->parent >= 0 ) {
00207         nn = nametree->namenode + nnn->parent;
00208         if ( node == nn->left ) nn->left = node3;
00209         else nn->right = node3;
00210     }
00211     if ( node == nametree->headnode ) nametree->headnode = node3;
00212 }
00213 else if ( nn->balance == rr ) { /* The easy case */
00214     nn->parent = n->parent; n->parent = newnode;
00215     if ( rr > 0 ) {
00216         if ( nn->left >= 0 ) nametree->namenode[nn->left].parent = node;
00217         n->right = nn->left; nn->left = node;
00218     }
00219     else {
00220         if ( nn->right >= 0 ) nametree->namenode[nn->right].parent = node;
00221         n->left = nn->right; nn->right = node;
00222     }
00223     if ( nn->parent >= 0 ) {
00224         nnn = nametree->namenode + nn->parent;
00225         if ( node == nnn->left ) nnn->left = newnode;
00226         else nnn->right = newnode;
00227     }
00228     nn->balance = n->balance = 0;
00229     if ( node == nametree->headnode ) nametree->headnode = newnode;
00230 }
00231 #ifdef DEBUGON
00232 else { /* Cannot be. Code here for debugging only */
00233     MesPrint("We ran into an impossible case in AddName\n");
00234     DumpTree(nametree);
00235     Terminate(-1);
00236 }

```

```

00237 #endif
00238     return(retval);
00239 }
00240
00241 /*
00242  #] AddName :
00243  #[ GetName :
00244
00245      When AutoDeclare is an active statement.
00246      If par == WITHAUTO and the variable is not found we have to check:
00247      1: that nametree != AC.exprnames && nametree != AC.dollarnames
00248      2: check that the variable is not in AC.exprnames after all.
00249      3: call GetAutoName and return its values.
00250 */
00251
00252 int GetName(NAMETREE *nametree, UBYTE *namein, WORD *number, int par)
00253 {
00254     NAMENODE *n;
00255     int node, newnode, i;
00256     UBYTE *s, *t, *u, *name;
00257     /* name = ConstructName(namein,0); */
00258     name = namein;
00259     if ( nametree->namenode == 0 || nametree->namefill == 0 ) goto NotFound;
00260     newnode = nametree->headnode;
00261     do {
00262         node = newnode;
00263         n = nametree->namenode+node;
00264         if ( ( i = StrCmp(name,nametree->namebuffer+n->name) ) < 0 )
00265             newnode = n->left;
00266         else if ( i > 0 ) newnode = n->right;
00267         else {
00268             *number = n->number;
00269             return(n->type);
00270         }
00271     } while ( newnode >= 0 );
00272     s = name;
00273     while ( *s ) s++;
00274     if ( s > name && s[-1] == '_' && nametree == AC.varnames ) {
00275     /*
00276         The Kronecker delta d_ is very special. It is not really a function.
00277     */
00278         if ( s == name+2 && ( *name == 'd' || *name == 'D' ) ) {
00279             *number = DELTA-FUNCTION;
00280             return(CDELTA);
00281         }
00282     /*
00283         Test for N#_? type variables (summed indices)
00284     */
00285         if ( s > name+2 && *name == 'N' ) {
00286             t = name+1; i = 0;
00287             while ( FG.cTable[*t] == 1 ) i = 10*i + *t++ - '0';
00288             if ( s == t+1 ) {
00289                 *number = i + AM.IndDum - AM.OffsetIndex;
00290                 return(CINDEX);
00291             }
00292         }
00293     /*
00294         Now test for any built in object
00295     */
00296         newnode = nametree->headnode;
00297         do {
00298             node = newnode;
00299             n = nametree->namenode+node;
00300             if ( ( i = StrHICmp(name,nametree->namebuffer+n->name) ) < 0 )
00301                 newnode = n->left;
00302             else if ( i > 0 ) newnode = n->right;
00303             else {
00304                 *number = n->number; return(n->type);
00305             }
00306         } while ( newnode >= 0 );
00307     /*
00308         Now we test for the extra symbols of the type STR###_
00309         The string sits in AC.extrasym and is followed by digits.
00310         The name is only legal if the number is in the
00311         range 1,...,cbuf[AM.sbufnum].numrhs
00312     */
00313         t = name; u = AC.extrasym;
00314         while ( *t == *u ) { t++; u++; }
00315         if ( *u == 0 && *t != 0 ) { /* potential hit */
00316             WORD x = 0;
00317             while ( FG.cTable[*t] == 1 ) {
00318                 x = 10*x + (*t++ - '0');
00319             }
00320             if ( *t == '_' && x > 0 && x <= cbuf[AM.sbufnum].numrhs ) { /* Hit */
00321                 *number = MAXVARIABLES-x;
00322                 return(CSYMBOL);
00323             }

```

```

00324     }
00325 }
00326 NotFound;;
00327 if ( par != WITHAUTO || nametree == AC.autonames ) return(NAMENOTFOUND);
00328 return(GetAutoName(name,number));
00329 }
00330
00331 /*
00332 #] GetName :
00333 #[ GetFunction :
00334
00335 Gets either a function or a $ that should expand into a function
00336 during runtime. In the case of the $ the value in funnum is -dolnum-1.
00337 The return value is the position after the name of the function or the $.
00338 */
00339
00340 static WORD one = 1;
00341
00342 UBYTE *GetFunction(UBYTE *s,WORD *funnum)
00343 {
00344     int type;
00345     WORD numfun;
00346     UBYTE *t1, c;
00347     if ( *s == '$' ) {
00348         t1 = s+1; while ( FG.cTable[*t1] < 2 ) t1++;
00349         c = *t1; *t1 = 0;
00350         if ( ( type = GetName(AC.dollarnames,s+1,&numfun,NOAUTO) ) == CDOLLAR ) {
00351             *funnum = -numfun-2;
00352         }
00353         else {
00354             MesPrint("%s is undefined",s);
00355             numfun = AddDollar(s+1,DOLINDEX,&one,1);
00356             *funnum = 0;
00357         }
00358     }
00359     else {
00360         t1 = SkipAName(s);
00361         c = *t1; *t1 = 0;
00362         if ( ( ( type = GetName(AC.varnames,s,&numfun,WITHAUTO) ) != CFUNCTION )
00363             || ( functions[numfun].spec > 0 ) ) {
00364             MesPrint("%s should be a regular function",s);
00365             *funnum = 0;
00366             if ( type < 0 ) {
00367                 if ( GetName(AC.exprnames,s,&numfun,NOAUTO) == NAMENOTFOUND )
00368                     AddFunction(s,0,0,0,0,0,-1,-1);
00369             }
00370             *t1 = c;
00371             return(t1);
00372         }
00373         *funnum = numfun+FUNCTION;
00374     }
00375     *t1 = c;
00376     return(t1);
00377 }
00378
00379 /*
00380 #] GetFunction :
00381 #[ GetNumber :
00382
00383 Gets either a number or a $ that should expand into a number
00384 during runtime. In the case of the $ the value in num is -dolnum-2.
00385 The return value is the position after the number or the $.
00386 */
00387
00388 UBYTE *GetNumber(UBYTE *s,WORD *num)
00389 {
00390     int type;
00391     WORD numfun;
00392     UBYTE *t1, c;
00393     while ( *s == '+' ) s++;
00394     if ( *s == '$' ) {
00395         t1 = s+1; while ( FG.cTable[*t1] < 2 ) t1++;
00396         c = *t1; *t1 = 0;
00397         if ( ( type = GetName(AC.dollarnames,s+1,&numfun,NOAUTO) ) == CDOLLAR ) {
00398             *num = -numfun-2;
00399         }
00400         else {
00401             MesPrint("%s is undefined",s);
00402             numfun = AddDollar(s+1,DOLINDEX,&one,1);
00403             *num = -1;
00404         }
00405     }
00406     else if ( *s >= '0' && *s <= '9' ) {
00407         ULONG x = *s++ - '0';
00408         while ( *s >= '0' && *s <= '9' ) { x = 10*x + (*s++-'0'); }
00409         t1 = s;
00410         if ( x >= MAXPOSITIVE ) goto illegal;

```

```

00411         *num = (WORD)x;
00412         return(t1);
00413     }
00414     else {
00415         if ( *s == '-' ) { s++; }
00416         if ( *s >= '0' && *s <= '9' ) { while ( *s >= '0' && *s <= '9' ) s++; t1 = s; }
00417         else { t1 = SkipAName(s); }
00418     illegal:
00419         *num = -1;
00420         MesPrint("%Illegal option in Canonicalize statement. Should be a nonnegative number or $
variable.");
00421         return(t1);
00422     }
00423     *t1 = c;
00424     return(t1);
00425 }
00426
00427 /*
00428 #] GetNumber :
00429 #[ GetLastExprName :
00430
00431 When AutoDeclare is an active statement.
00432 If par == WITHAUTO and the variable is not found we have to check:
00433 1: that nametree != AC.exprnames && nametree != AC.dollarnames
00434 2: check that the variable is not in AC.exprnames after all.
00435 3: call GetAutoName and return its values.
00436 */
00437
00438 int GetLastExprName(UBYTE *name, WORD *number)
00439 {
00440     int i;
00441     EXPRESSIONS e;
00442     for ( i = NumExpressions; i > 0; i-- ) {
00443         e = Expressions+i-1;
00444         if ( StrCmp(AC.exprnames->namebuffer+e->name,name) == 0 ) {
00445             *number = i-1;
00446             return(1);
00447         }
00448     }
00449     return(0);
00450 }
00451
00452 /*
00453 #] GetLastExprName :
00454 #[ GetOName :
00455
00456 Adds the proper offsets, so we do not have to do that in the calling
00457 routine.
00458 */
00459
00460 int GetOName(NAMETREE *nametree, UBYTE *name, WORD *number, int par)
00461 {
00462     int retval = GetName(nametree,name,number,par);
00463     switch ( retval ) {
00464         case CVECTOR: *number += AM.OffsetVector; break;
00465         case CINDEX: *number += AM.OffsetIndex; break;
00466         case CFUNCTION: *number += FUNCTION; break;
00467         default: break;
00468     }
00469     return(retval);
00470 }
00471
00472 /*
00473 #] GetOName :
00474 #[ GetAutoName :
00475
00476 This routine gets the automatic declarations
00477 */
00478
00479 int GetAutoName(UBYTE *name, WORD *number)
00480 {
00481     UBYTE *s, c;
00482     int type;
00483     if ( GetName(AC.exprnames,name,number,NOAUTO) != NAMENOTFOUND )
00484         return(NAMENOTFOUND);
00485     s = name;
00486     while ( *s ) { s++; }
00487     if ( s[-1] == '-' ) {
00488         return(NAMENOTFOUND);
00489     }
00490     while ( s > name ) {
00491         c = *s; *s = 0;
00492         type = GetName(AC.autonames,name,number,NOAUTO);
00493         *s = c;
00494         switch(type) {
00495             case CSYMBOL: {
00496                 SYMBOLS sym = ((SYMBOLS) (AC.AutoSymbolList.lijst)) + *number;

```

```

00497         *number = AddSymbol(name,sym->minpower,sym->maxpower,sym->complex,sym->dimension);
00498         return(type); }
00499     case CVECTOR: {
00500         VECTORS vec = ((VECTORS) (AC.AutoVectorList.lijst)) + *number;
00501         *number = AddVector(name,vec->complex,vec->dimension);
00502         return(type); }
00503     case CINDEX: {
00504         INDICES ind = ((INDICES) (AC.AutoIndexList.lijst)) + *number;
00505         *number = AddIndex(name,ind->dimension,ind->nmin4);
00506         return(type); }
00507     case CFUNCTION: {
00508         FUNCTIONS fun = ((FUNCTIONS) (AC.AutoFunctionList.lijst)) + *number;
00509         *number =
00510         AddFunction(name,fun->commute,fun->spec,fun->complex,fun->symmetric,fun->dimension,fun->maxnumargs,fun->minnumargs);
00511         return(type); }
00512         break;
00513     }
00514     s--;
00515 }
00516 return(NAMENOTFOUND);
00517 }
00518
00519 /*
00520 #] GetAutoName :
00521 #[ GetVar :
00522 */
00523
00524 int GetVar(UBYTE *name, WORD *type, WORD *number, int wantedtype, int par)
00525 {
00526     WORD funnum;
00527     int typ;
00528     if ( ( typ = GetName(AC.varnames,name,number,par) ) != wantedtype ) {
00529         if ( typ != NAMENOTFOUND ) {
00530             if ( wantedtype == -1 ) {
00531                 *type = typ;
00532                 return(1);
00533             }
00534             NameConflict(typ,name);
00535             MakeDubious(AC.varnames,name,&funnum);
00536             return(-1);
00537         }
00538         if ( ( typ = GetName(AC.exprnames,name,&funnum,par) ) != NAMENOTFOUND ) {
00539             if ( typ == wantedtype || wantedtype == -1 ) {
00540                 *number = funnum; *type = typ; return(1);
00541             }
00542             NameConflict(typ,name);
00543             return(-1);
00544         }
00545         return(NAMENOTFOUND);
00546     }
00547     if ( typ == -1 ) { return(0); }
00548     *type = typ;
00549     return(1);
00550 }
00551
00552 /*
00553 #] GetVar :
00554 #[ EntVar :
00555 */
00556
00557 WORD EntVar(WORD type, UBYTE *name, WORD x, WORD y, WORD z, WORD d)
00558 {
00559     switch ( type ) {
00560         case CSYMBOL:
00561             return(AddSymbol(name,y,z,x,d));
00562             break;
00563         case CINDEX:
00564             return(AddIndex(name,x,z));
00565             break;
00566         case CVECTOR:
00567             return(AddVector(name,x,d));
00568             break;
00569         case CFUNCTION:
00570             return(AddFunction(name,y,z,x,0,d,-1,-1));
00571             break;
00572         case CSET:
00573             AC.SetList.numtemp++;
00574             return(AddSet(name,d));
00575             break;
00576         case CEXPRESSION:
00577             return(AddExpression(name,x,y));
00578             break;
00579         default:
00580             break;
00581     }
00582     return(-1);

```

```

00583 }
00584
00585 /*
00586  #] EntVar :
00587  #[ GetDollar :
00588 */
00589
00590 int GetDollar(UBYTE *name)
00591 {
00592     WORD number;
00593     if ( GetName(AC.dollarnames,name,&number,NOAUTO) == NAMENOTFOUND ) return(-1);
00594     return((int)number);
00595 }
00596
00597 /*
00598  #] GetDollar :
00599  #[ DumpTree :
00600 */
00601
00602 void DumpTree(NAMETREE *nametree)
00603 {
00604     if ( nametree->headnode >= 0
00605         && nametree->namebuffer && nametree->namenode ) {
00606         DumpNode(nametree,nametree->headnode,0);
00607     }
00608 }
00609
00610 /*
00611  #] DumpTree :
00612  #[ DumpNode :
00613 */
00614
00615 void DumpNode(NAMETREE *nametree, WORD node, WORD depth)
00616 {
00617     NAMENODE *n;
00618     int i;
00619     char *name;
00620     n = nametree->namenode + node;
00621     if ( n->left >= 0 ) DumpNode(nametree,n->left,depth+1);
00622     for ( i = 0; i < depth; i++ ) printf(" ");
00623     name = (char *) (nametree->namebuffer+n->name);
00624     printf("%s(%d): { %d } ( %d ) ( %d ) [ %d ] \n",
00625         name,node,n->parent,n->left,n->right,n->balance);
00626     if ( n->right >= 0 ) DumpNode(nametree,n->right,depth+1);
00627 }
00628
00629 /*
00630  #] DumpNode :
00631  #[ CompactifyTree :
00632 */
00633
00634 int CompactifyTree(NAMETREE *nametree,WORD par)
00635 {
00636     NAMETREE newtree;
00637     NAMENODE *n;
00638     LONG i, j, ns, k;
00639     UBYTE *s;
00640
00641     for ( i = 0, j = 0, k = 0, n = nametree->namenode, ns = 0;
00642         i < nametree->nodefill; i++, n++ ) {
00643         if ( n->type != CDELETE ) {
00644             s = nametree->namebuffer+n->name;
00645             while ( *s ) { s++; ns++; }
00646             j++;
00647         }
00648         else k++;
00649     }
00650     if ( k == 0 ) return(0);
00651     if ( j == 0 ) {
00652         if ( nametree->namebuffer ) M_free(nametree->namebuffer,"nametree->namebuffer");
00653         if ( nametree->namenode ) M_free(nametree->namenode,"nametree->namenode");
00654         nametree->namebuffer = 0;
00655         nametree->namenode = 0;
00656         nametree->namesize = nametree->namefill =
00657         nametree->nodesize = nametree->nodefill =
00658         nametree->oldnamefill = nametree->oldnodefill = 0;
00659         nametree->globalnamefill = nametree->globalnodefill =
00660         nametree->clearnamefill = nametree->clearnodefill = 0;
00661         nametree->headnode = -1;
00662         return(0);
00663     }
00664     ns += j;
00665     if ( j < 10 ) j = 10;
00666     if ( ns < 100 ) ns = 100;
00667     newtree.namenode = (NAMENODE *)Malloc1(2*j*sizeof(NAMENODE),"compactify nametree");
00668     newtree.nodefill = 0; newtree.nodesize = 2*j;
00669     newtree.namebuffer = (UBYTE *)Malloc1(2*ns,"compactify nametree");

```

```

00670     newtree.namefill = 0; newtree.namesize = 2*ns;
00671     CopyTree(&newtree,nametree,nametree->headnode,par);
00672     newtree.namenode[newtree.nodefill»1].parent = -1;
00673     LinkTree(&newtree,(WORD)0,newtree.nodefill);
00674     newtree.headnode = newtree.nodefill » 1;
00675     M_free(nametree->namebuffer,"nametree->namebuffer");
00676     M_free(nametree->namenode,"nametree->namenode");
00677     nametree->namebuffer = newtree.namebuffer;
00678     nametree->namenode = newtree.namenode;
00679     nametree->namesize = newtree.namesize;
00680     nametree->namefill = newtree.namefill;
00681     nametree->nodesize = newtree.nodesize;
00682     nametree->nodefill = newtree.nodefill;
00683     nametree->oldnamefill = newtree.namefill;
00684     nametree->oldnodefill = newtree.nodefill;
00685     nametree->headnode = newtree.headnode;
00686
00687     /* DumpTree(nametree); */
00688     return(0);
00689 }
00690
00691 /*
00692  #] CompactifyTree :
00693  #[ CopyTree :
00694  */
00695
00696 void CopyTree(NAMETREE *newtree, NAMETREE *oldtree, WORD node, WORD par)
00697 {
00698     NAMENODE *n, *m;
00699     UBYTE *s, *t;
00700     n = oldtree->namenode+node;
00701     if ( n->left >= 0 ) CopyTree(newtree,oldtree,n->left,par);
00702     if ( n->type != CDELETE ) {
00703         m = newtree->namenode+newtree->nodefill;
00704         m->type = n->type;
00705         m->number = n->number;
00706         m->name = newtree->namefill;
00707         m->left = m->right = -1;
00708         m->balance = 0;
00709         switch ( n->type ) {
00710             case CSYMBOL:
00711                 if ( par == AUTONAMES ) {
00712                     autosymbols[n->number].name = newtree->namefill;
00713                     autosymbols[n->number].node = newtree->nodefill;
00714                 }
00715                 else {
00716                     symbols[n->number].name = newtree->namefill;
00717                     symbols[n->number].node = newtree->nodefill;
00718                 }
00719                 break;
00720             case CINDEX :
00721                 if ( par == AUTONAMES ) {
00722                     autoindices[n->number].name = newtree->namefill;
00723                     autoindices[n->number].node = newtree->nodefill;
00724                 }
00725                 else {
00726                     indices[n->number].name = newtree->namefill;
00727                     indices[n->number].node = newtree->nodefill;
00728                 }
00729                 break;
00730             case CVECTOR:
00731                 if ( par == AUTONAMES ) {
00732                     autovectors[n->number].name = newtree->namefill;
00733                     autovectors[n->number].node = newtree->nodefill;
00734                 }
00735                 else {
00736                     vectors[n->number].name = newtree->namefill;
00737                     vectors[n->number].node = newtree->nodefill;
00738                 }
00739                 break;
00740             case CFUNCTION:
00741                 if ( par == AUTONAMES ) {
00742                     autofunctions[n->number].name = newtree->namefill;
00743                     autofunctions[n->number].node = newtree->nodefill;
00744                 }
00745                 else {
00746                     functions[n->number].name = newtree->namefill;
00747                     functions[n->number].node = newtree->nodefill;
00748                 }
00749                 break;
00750             case CSET:
00751                 Sets[n->number].name = newtree->namefill;
00752                 Sets[n->number].node = newtree->nodefill;
00753                 break;
00754             case CEXPRESSION:
00755                 Expressions[n->number].name = newtree->namefill;
00756                 Expressions[n->number].node = newtree->nodefill;

```

```

00757         break;
00758     case CDUBIOUS:
00759         Dubious[n->number].name = newtree->namefill;
00760         Dubious[n->number].node = newtree->nodefill;
00761         break;
00762     case CDOLLAR:
00763         Dollars[n->number].name = newtree->namefill;
00764         Dollars[n->number].node = newtree->nodefill;
00765         break;
00766     default:
00767         MesPrint("Illegal variable type in CopyTree: %d",n->type);
00768         break;
00769     }
00770     newtree->nodefill++;
00771     s = newtree->namebuffer + newtree->namefill;
00772     t = oldtree->namebuffer + n->name;
00773     while ( *t ) { *s++ = *t++; newtree->namefill++; }
00774     *s = 0; newtree->namefill++;
00775 }
00776 if ( n->right >= 0 ) CopyTree(newtree,oldtree,n->right,par);
00777 }
00778
00779 /*
00780 #] CopyTree :
00781 #[ LinkTree :
00782 */
00783
00784 void LinkTree(NAMETREE *tree, WORD offset, WORD numnodes)
00785 {
00786     /*
00787     Makes the tree into a binary tree
00788     */
00789     int med,numleft,numright,medleft,medright;
00790     med = numnodes » 1;
00791     numleft = med;
00792     numright = numnodes - med - 1;
00793     medleft = numleft » 1;
00794     medright = ( numright » 1 ) + med + 1;
00795     if ( numleft > 0 ) {
00796         tree->namenode[offset+med].left = offset+medleft;
00797         tree->namenode[offset+medleft].parent = offset+med;
00798     }
00799     if ( numright > 0 ) {
00800         tree->namenode[offset+med].right = offset+medright;
00801         tree->namenode[offset+medright].parent = offset+med;
00802     }
00803     if ( numleft > 0 ) LinkTree(tree,offset,numleft);
00804     if ( numright > 0 ) LinkTree(tree,offset+med+1,numright);
00805     while ( numleft && numright ) { numleft »= 1; numright »= 1; }
00806     if ( numleft ) tree->namenode[offset+med].balance = -1;
00807     else if ( numright ) tree->namenode[offset+med].balance = 1;
00808 }
00809
00810 /*
00811 #] LinkTree :
00812 #[ MakeNameTree :
00813 */
00814
00815 NAMETREE *MakeNameTree(void)
00816 {
00817     NAMETREE *n;
00818     n = (NAMETREE *)Malloc1(sizeof(NAMETREE),"new nametree");
00819     n->namebuffer = 0;
00820     n->namenode = 0;
00821     n->namesize = n->namefill = n->nodesize = n->nodefill =
00822     n->oldnamefill = n->oldnodefill = 0;
00823     n->globalnamefill = n->globalnodefill =
00824     n->clearnamefill = n->clearnodefill = 0;
00825     n->headnode = -1;
00826     return(n);
00827 }
00828
00829 /*
00830 #] MakeNameTree :
00831 #[ FreeNameTree :
00832 */
00833
00834 void FreeNameTree(NAMETREE *n)
00835 {
00836     if ( n ) {
00837         if ( n->namebuffer ) M_free(n->namebuffer,"nametree->namebuffer");
00838         if ( n->namenode ) M_free(n->namenode,"nametree->namenode");
00839         M_free(n,"nametree");
00840     }
00841 }
00842
00843 /*

```



```

00844     #] FreeNameTree :
00845
00846     #[ WildcardNames :
00847 /*
00848
00849 void ClearWildcardNames(void)
00850 {
00851     AC.NumWildcardNames = 0;
00852 }
00853
00854 int AddWildcardName(UBYTE *name)
00855 {
00856     GETIDENTITY
00857     int size = 0, tocopy, i;
00858     UBYTE *s = name, *t, *newbuffer;
00859     while ( *s ) { s++; size++; }
00860     for ( i = 0, t = AC.WildcardNames; i < AC.NumWildcardNames; i++ ) {
00861         s = name;
00862         while ( ( *s == *t ) && *s ) { s++; t++; }
00863         if ( *s == 0 && *t == 0 ) return(i+1);
00864         while ( *t ) t++;
00865         t++;
00866     }
00867     tocopy = t - AC.WildcardNames;
00868     if ( tocopy + size + 1 > AC.WildcardBufferSize ) {
00869         if ( AC.WildcardBufferSize == 0 ) {
00870             AC.WildcardBufferSize = size+1;
00871             if ( AC.WildcardBufferSize < 100 ) AC.WildcardBufferSize = 100;
00872         }
00873         else if ( size+1 >= AC.WildcardBufferSize ) {
00874             AC.WildcardBufferSize += size+1;
00875         }
00876         else {
00877             AC.WildcardBufferSize *= 2;
00878         }
00879         newbuffer = (UBYTE *)Malloc1((LONG)AC.WildcardBufferSize,"argument list names");
00880         t = newbuffer;
00881         if ( AC.WildcardNames ) {
00882             s = AC.WildcardNames;
00883             while ( tocopy > 0 ) { *t++ = *s++; tocopy--; }
00884             M_free(AC.WildcardNames,"AC.WildcardNames");
00885         }
00886         AC.WildcardNames = newbuffer;
00887         M_free(AT.WildArgTaken,"AT.WildArgTaken");
00888         AT.WildArgTaken = (WORD *)Malloc1((LONG)AC.WildcardBufferSize*sizeof(WORD)/2
00889             ,"argument list names");
00890     }
00891     s = name;
00892     while ( *s ) *t++ = *s++;
00893     *t = 0;
00894     AC.NumWildcardNames++;
00895     return(AC.NumWildcardNames);
00896 }
00897
00898 int GetWildcardName(UBYTE *name)
00899 {
00900     UBYTE *s, *t;
00901     int i;
00902     for ( i = 0, t = AC.WildcardNames; i < AC.NumWildcardNames; i++ ) {
00903         s = name;
00904         while ( ( *s == *t ) && *s ) { s++; t++; }
00905         if ( *s == 0 && *t == 0 ) return(i+1);
00906         while ( *t ) t++;
00907         t++;
00908     }
00909     return(0);
00910 }
00911
00912 /*
00913     #] WildcardNames :
00914
00915     #[ AddSymbol :
00916
00917     The actual addition. Special routine for additions 'on the fly'
00918 */
00919
00920 int AddSymbol(UBYTE *name, int minpow, int maxpow, int cplx, int dim)
00921 {
00922     int nodenum, numsymbol = AC.Symbols->num;
00923     UBYTE *s = name;
00924     SYMBOLS sym = (SYMBOLS)FromVarList(AC.Symbols);
00925     bzero(sym,sizeof(struct SyMb01));
00926     sym->name = AddName(*AC.activenames,name,CSYMBOL,numsymbol,&nodenum);
00927     sym->minpower = minpow;
00928     sym->maxpower = maxpow;
00929     sym->complex = cplx;
00930     sym->flags = 0;

```

```

00931     sym->node      = nodenum;
00932     sym->dimension= dim;
00933     while ( *s ) s++;
00934     sym->namesize = (s-name)+1;
00935     return(numsymbol);
00936 }
00937
00938 /*
00939 #] AddSymbol :
00940 #[ CoSymbol :
00941
00942     Symbol declarations.  name[#{R|I|C}][[([min]:[max])]]
00943     Note that we know already that the parentheses match properly
00944 */
00945
00946 int CoSymbol(UBYTE *s)
00947 {
00948     int type, error = 0, minpow, maxpow, cplx, sgn, dim;
00949     WORD numsymbol;
00950     UBYTE *name, *oldc, c, cc;
00951     do {
00952         minpow = -MAXPOWER;
00953         maxpow =  MAXPOWER;
00954         cplx = 0;
00955         dim = 0;
00956         name = s;
00957         if ( ( s = SkipAName(s) ) == 0 ) {
00958             IllForm:    MesPrint("&Illegally formed name in symbol statement");
00959             error = 1;
00960             s = SkipField(name,0);
00961             goto eol;
00962         }
00963         oldc = s; cc = c = *s; *s = 0;
00964         if ( TestName(name) ) { *s = c; goto IllForm; }
00965         if ( cc == '#' ) {
00966             s++;
00967             if ( tolower(*s) == 'r' ) cplx = VARTYPENONE;
00968             else if ( tolower(*s) == 'c' ) cplx = VARTYPECOMPLEX;
00969             else if ( tolower(*s) == 'i' ) cplx = VARTYPEIMAGINARY;
00970             else if ( ( ( *s == '-' || *s == '+' || *s == '=' )
00971                 && ( s[1] >= '0' && s[1] <= '9' ) )
00972                 || ( *s >= '0' && *s <= '9' ) ) {
00973                 LONG x;
00974                 sgn = 0;
00975                 if ( *s == '-' ) { sgn = VARTYPEMINUS; s++; }
00976                 else if ( *s == '+' || *s == '=' ) { sgn = 0; s++; }
00977                 x = *s - '0';
00978                 while ( s[1] >= '0' && s[1] <= '9' ) {
00979                     x = 10*x + (s[1] - '0'); s++;
00980                 }
00981                 if ( x >= MAXPOWER || x <= 1 ) {
00982                     MesPrint("&Illegal value for root of unity %s",name);
00983                     error = 1;
00984                 }
00985                 else {
00986                     maxpow = x;
00987                 }
00988                 cplx = VARTYPEROOTOFUNITY | sgn;
00989             }
00990             else {
00991                 MesPrint("&Illegal specification for complexity of symbol %s",name);
00992                 *oldc = c;
00993                 error = 1;
00994                 s = SkipField(s,0);
00995                 goto eol;
00996             }
00997             s++; cc = *s;
00998         }
00999         if ( cc == '{' ) {
01000             s++;
01001             if ( ( *s == 'd' || *s == 'D' ) && s[1] == '=' ) {
01002                 s += 2;
01003                 if ( *s == '-' || *s == '+' || FG.cTable[*s] == 1 ) {
01004                     ParseSignedNumber(dim,s)
01005                     if ( dim < -HALFMAX || dim > HALFMAX ) {
01006                         MesPrint("&Warning: dimension of %s (%d) out of range"
01007                             ,name,dim);
01008                     }
01009                 }
01010                 if ( *s != '}' ) goto IllDim;
01011                 else s++;
01012             }
01013             else {
01014                 IllDim:    MesPrint("&Error: Illegal dimension field for variable %s",name);
01015                 error = 1;
01016                 s = SkipField(s,0);
01017                 goto eol;

```

```

01018         }
01019         cc = *s;
01020     }
01021     if ( cc == '(' ) {
01022         if ( ( cplx & VARTYPEROOTOFUNITY ) == VARTYPEROOTOFUNITY ) {
01023             MesPrint("&Root of unity property for %s cannot be combined with power
restrictions",name);
01024             error = 1;
01025         }
01026         s++;
01027         if ( *s == '-' || *s == '+' || FG.cTable[*s] == 1 ) {
01028             ParseSignedNumber(minpow,s)
01029             if ( minpow < -MAXPOWER ) {
01030                 minpow = -MAXPOWER;
01031                 if ( AC.WarnFlag )
01032                     MesPrint("&Warning: minimum power of %s corrected to %d"
,name,-MAXPOWER);
01033             }
01034         }
01035     }
01036     if ( *s != ':' ) {
01037 skippar: error = 1;
01038         s = SkipField(s,1);
01039         goto eol;
01040     }
01041     else s++;
01042     if ( *s == '-' || *s == '+' || FG.cTable[*s] == 1 ) {
01043         ParseSignedNumber(maxpow,s)
01044         if ( maxpow > MAXPOWER ) {
01045             maxpow = MAXPOWER;
01046             if ( AC.WarnFlag )
01047                 MesPrint("&Warning: maximum power of %s corrected to %d"
,name,MAXPOWER);
01048         }
01049     }
01050 }
01051 if ( *s != ')' ) goto skippar;
01052 s++;
01053 }
01054 if ( ( AC.AutoDeclareFlag == 0 &&
01055     ( ( type = GetName(AC.exprnames,name,&numsymbol,NOAUTO) )
01056       != NAMENOTFOUND ) )
01057 || ( ( type = GetName(*(AC.activenames),name,&numsymbol,NOAUTO) ) != NAMENOTFOUND ) ) {
01058     if ( type != CSYMBOL ) error = NameConflict(type,name);
01059     else {
01060         SYMBOLS sym = (SYMBOLS)(AC.Symbols->lijst) + numsymbol;
01061         if ( ( numsymbol == AC.lPolyFunVar ) && ( AC.lPolyFunType > 0 )
01062             && ( AC.lPolyFun != 0 ) && ( minpow > -MAXPOWER || maxpow < MAXPOWER ) ) {
01063             MesPrint("&The symbol %s is used by power expansions in the PolyRatFun!",name);
01064             error = 1;
01065         }
01066         sym->complex = cplx;
01067         sym->minpower = minpow;
01068         sym->maxpower = maxpow;
01069         sym->dimension= dim;
01070     }
01071 }
01072 else {
01073     AddSymbol(name,minpow,maxpow,cplx,dim);
01074 }
01075 *oldc = c;
01076 eol: while ( *s == ',' ) s++;
01077 } while ( *s );
01078 return(error);
01079 }
01080
01081 /*
01082 #] CoSymbol :
01083 #[ AddIndex :
01084
01085 The actual addition. Special routine for additions 'on the fly'
01086 */
01087
01088 int AddIndex(UBYTE *name, int dim, int dim4)
01089 {
01090     int nodenum, numindex = AC.Indices->num;
01091     INDICES ind = (INDICES)FromVarList(AC.Indices);
01092     UBYTE *s = name;
01093     bzero(ind,sizeof(struct InDeX));
01094     ind->name = AddName(*AC.activenames,name,CINDEX,numindex,&nodenum);
01095     ind->type = 0;
01096     ind->dimension = dim;
01097     ind->flags = 0;
01098     ind->nmin4 = dim4;
01099     ind->node = nodenum;
01100     while ( *s ) s++;
01101     ind->namesize = (s-name)+1;
01102     return(numindex);
01103 }

```

```

01104
01105 /*
01106 #] AddIndex :
01107 #[ CoIndex :
01108
01109 Index declarations. name[]={number|symbol[:othersymbol]}}
01110 */
01111
01112 int CoIndex(UBYTE *s)
01113 {
01114     int type, error = 0, dim, dim4;
01115     WORD numindex;
01116     UBYTE *name, *oldc, c;
01117     do {
01118         dim = AC.lDefDim;
01119         dim4 = AC.lDefDim4;
01120         name = s;
01121         if ( ( s = SkipAName(s) ) == 0 ) {
01122             IllForm: MesPrint("&Illegally formed name in index statement");
01123                 error = 1;
01124                 s = SkipField(name,0);
01125                 goto eol;
01126         }
01127         oldc = s; c = *s; *s = 0;
01128         if ( TestName(name) ) { *s = c; goto IllForm; }
01129         if ( c == '=' ) {
01130             s++;
01131             if ( ( s = DoDimension(s,&dim,&dim4) ) == 0 ) {
01132                 *oldc = c;
01133                 error = 1;
01134                 s = SkipField(name,0);
01135                 goto eol;
01136             }
01137         }
01138         if ( ( AC.AutoDeclareFlag == 0 &&
01139             ( ( type = GetName(AC.exprnames,name,&numindex,NOAUTO) )
01140             != NAMENOTFOUND ) )
01141         || ( ( type = GetName(*AC.activenames),name,&numindex,NOAUTO) ) != NAMENOTFOUND ) ) {
01142             if ( type != CINDEX ) error = NameConflict(type,name);
01143             else { /* reset the dimensions */
01144                 indices[numindex].dimension = dim;
01145                 indices[numindex].nmin4 = dim4;
01146             }
01147         }
01148         else AddIndex(name,dim,dim4);
01149         *oldc = c;
01150     eol: while ( *s == ',' ) s++;
01151     } while ( *s );
01152     return(error);
01153 }
01154
01155 /*
01156 #] CoIndex :
01157 #[ DoDimension :
01158 */
01159
01160 UBYTE *DoDimension(UBYTE *s, int *dim, int *dim4)
01161 {
01162     UBYTE c, *t = s;
01163     int type, error = 0;
01164     WORD numsymbol;
01165     NAMETREE **oldtree = AC.activenames;
01166     LIST* oldsymbols = AC.Symbols;
01167     *dim4 = -NMIN4SHIFT;
01168     if ( FG.cTable[*s] == 1 ) {
01169     retry:
01170         ParseNumber(*dim,s)
01171         #if ( BITSINWORD/8 < 4 )
01172         if ( *dim >= (1 << (BITSINWORD-1)) ) goto illeg;
01173         #endif
01174         *dim4 = *dim - 4;
01175         return(s);
01176     }
01177     else if ( ( (FG.cTable[*s] == 0) || ( *s == '[' ) )
01178         && ( s = SkipAName(s) ) != 0 ) {
01179         AC.activenames = &(AC.varnames);
01180         AC.Symbols = &(AC.SymbolList);
01181         c = *s; *s = 0;
01182         if ( ( ( type = GetName(AC.exprnames,t,&numsymbol,NOAUTO) ) != NAMENOTFOUND )
01183         || ( ( type = GetName(AC.varnames,t,&numsymbol,WITHAUTO) ) != NAMENOTFOUND ) ) {
01184             if ( type != CSYMBOL ) error = NameConflict(type,t);
01185         }
01186         else {
01187             numsymbol = AddSymbol(t,-MAXPOWER,MAXPOWER,0,0);
01188             if ( AC.WarnFlag )
01189                 MesPrint("&Warning: Implicit declaration of %s as a symbol",t);
01190         }

```

```

01191     *dim = -numsymbol;
01192     if ( ( *s = c ) == ':' ) {
01193         s++;
01194         t = s;
01195         if ( ( s = SkipAName(s) ) == 0 ) goto illeg;
01196         if ( ( ( type = GetName(AC.exprnames,t,&numsymbol,NOAUTO) ) != NAMENOTFOUND )
01197             || ( ( type = GetName(AC.varnames,t,&numsymbol,WITHAUTO) ) != NAMENOTFOUND ) ) {
01198             if ( type != CSYMBOL ) error = NameConflict(type,t);
01199         }
01200         else {
01201             numsymbol = AddSymbol(t,-MAXPOWER,MAXPOWER,0,0);
01202             if ( AC.WarnFlag )
01203                 MesPrint("&Warning: Implicit declaration of %s as a symbol",t);
01204         }
01205         *dim4 = -numsymbol-NMIN4SHIFT;
01206     }
01207 }
01208 else if ( *s == '+' && FG.cTable[s[1]] == 1 ) {
01209     s++; goto retry;
01210 }
01211 else {
01212 illeg: MesPrint("&Illegal dimension specification. Should be number >= 0, symbol or symbol:symbol");
01213     return(0);
01214 }
01215 AC.Symbols = oldsymbols;
01216 AC.activenames = oldtree;
01217 if ( error ) return(0);
01218 return(s);
01219 }
01220
01221 /*
01222 #] DoDimension :
01223 #[ CoDimension :
01224 */
01225
01226 int CoDimension(UBYTE *s)
01227 {
01228     s = DoDimension(s,&AC.lDefDim,&AC.lDefDim4);
01229     if ( s == 0 ) return(1);
01230     if ( *s != 0 ) {
01231         MesPrint("&Argument of dimension statement should be number >= 0, symbol or symbol:symbol");
01232         return(1);
01233     }
01234     return(0);
01235 }
01236
01237 /*
01238 #] CoDimension :
01239 #[ AddVector :
01240
01241 The actual addition. Special routine for additions 'on the fly'
01242 */
01243
01244 int AddVector(UBYTE *name, int cplx, int dim)
01245 {
01246     int nodenum, numvector = AC.Vectors->num;
01247     VECTORS v = (VECTORS)FromVarList(AC.Vectors);
01248     UBYTE *s = name;
01249     bzero(v,sizeof(struct VeCtOr));
01250     v->name = AddName(*AC.activenames,name,CVECTOR,numvector,&nodenum);
01251     v->complex = cplx;
01252     v->node = nodenum;
01253     v->dimension = dim;
01254     v->flags = 0;
01255     while ( *s ) s++;
01256     v->namesize = (s-name)+1;
01257     return(numvector);
01258 }
01259
01260 /*
01261 #] AddVector :
01262 #[ CoVector :
01263
01264 Vector declarations. The descriptor string is "(,%n)"
01265 */
01266
01267 int CoVector(UBYTE *s)
01268 {
01269     int type, error = 0, dim;
01270     WORD numvector;
01271     UBYTE *name, c, *endname;
01272     do {
01273         name = s;
01274         dim = 0;
01275         if ( ( s = SkipAName(s) ) == 0 ) {
01276 IllForm: MesPrint("&Illegally formed name in vector statement");
01277             error = 1;

```

```

01278         s = SkipField(s,0);
01279     }
01280     else {
01281         c = *s; *s = 0, endname = s;
01282         if ( TestName(name) ) { *s = c; goto IllForm; }
01283         if ( c == '{' ) {
01284             s++;
01285             if ( ( *s == 'd' || *s == 'D' ) && s[1] == '=' ) {
01286                 s += 2;
01287                 if ( *s == '-' || *s == '+' || FG.cTable[*s] == 1 ) {
01288                     ParseSignedNumber(dim,s)
01289                     if ( dim < -HALFMAX || dim > HALFMAX ) {
01290                         MesPrint("&Warning: dimension of %s (%d) out of range"
01291                             ,name,dim);
01292                     }
01293                 }
01294                 if ( *s != '}' ) goto IllDim;
01295                 else s++;
01296             }
01297             else {
01298 IllDim:         MesPrint("&Error: Illegal dimension field for variable %s",name);
01299                 error = 1;
01300                 s = SkipField(s,0);
01301                 while ( *s == ',' ) s++;
01302                 continue;
01303             }
01304         }
01305         if ( ( AC.AutoDeclareFlag == 0 &&
01306             ( ( type = GetName(AC.exprnames,name,&numvector,NOAUTO) )
01307               != NAMENOTFOUND ) )
01308             || ( ( type = GetName(*AC.activenames,name,&numvector,NOAUTO) ) != NAMENOTFOUND ) ) {
01309             if ( type != CVECTOR ) error = NameConflict(type,name);
01310         }
01311         else AddVector(name,0,dim);
01312         *endname = c;
01313     }
01314     while ( *s == ',' ) s++;
01315 } while ( *s );
01316 return(error);
01317 }
01318
01319 /*
01320 #] CoVector :
01321 #[ AddFunction :
01322
01323     The actual addition. Special routine for additions 'on the fly'
01324 */
01325
01326 int AddFunction(UBYTE *name, int comm, int istensor, int cplx, int symprop, int dim, int argmax, int
    argmin)
01327 {
01328     int nodenum, numfunction = AC.Functions->num;
01329     FUNCTIONS fun = (FUNCTIONS)FromVarList(AC.Functions);
01330     UBYTE *s = name;
01331     bzero(fun,sizeof(struct FuNcTiOn));
01332     fun->name = AddName(*AC.activenames,name,CFUNCTION,numfunction,&nodenum);
01333     fun->commute = comm;
01334     fun->spec = istensor;
01335     fun->complex = cplx;
01336     fun->tabl = 0;
01337     fun->flags = 0;
01338     fun->node = nodenum;
01339     fun->symminfo = 0;
01340     fun->symmetric = symprop;
01341     fun->dimension = dim;
01342     fun->maxnumargs = argmax;
01343     fun->minnumargs = argmin;
01344     while ( *s ) s++;
01345     fun->namesize = (s-name)+1;
01346     return(numfunction);
01347 }
01348
01349 /*
01350 #] AddFunction :
01351 #[ CoCommuteInSet :
01352
01353     Commuting, f1, ..., fn;
01354 */
01355
01356 int CoCommuteInSet(UBYTE *s)
01357 {
01358     UBYTE *name, *ss, c, *start = s;
01359     WORD number, type, *g, *gg;
01360     int error = 0, i, len = StrLen(s), len2 = 0;
01361     if ( AC.CommuteInSet != 0 ) {
01362         g = AC.CommuteInSet;
01363         while ( *g ) g += *g;

```

```

01364     len2 = g - AC.CommuteInSet;
01365     if ( len2+len+3 > AC.SizeCommuteInSet ) {
01366         gg = (WORD *)Malloc1((len2+len+3)*sizeof(WORD),"CommuteInSet");
01367         for ( i = 0; i < len2; i++ ) gg[i] = AC.CommuteInSet[i];
01368         gg[len2] = 0;
01369         M_free(AC.CommuteInSet,"CommuteInSet");
01370         AC.CommuteInSet = gg;
01371         AC.SizeCommuteInSet = len+len2+3;
01372         g = AC.CommuteInSet+len2;
01373     }
01374 }
01375 else {
01376     AC.SizeCommuteInSet = len+2;
01377     g = AC.CommuteInSet = (WORD *)Malloc1((len+3)*sizeof(WORD),"CommuteInSet");
01378     *g = 0;
01379 }
01380 gg = g++;
01381 ss = s-1;
01382 for(;;) {
01383     while ( *s == ' ' || *s == ' ' || *s == '\t' ) s++;
01384     if ( *s == 0 ) {
01385         if ( s - start >= len ) break;
01386         *s = ' '; s++;
01387         *g = 0;
01388         *gg = g-gg;
01389         if ( *gg < 2 ) {
01390             MesPrint("&There should be at least two noncommuting functions or tensors in a
commuting statement.");
01391             error = 1;
01392         }
01393         else if ( *gg == 2 ) {
01394             gg[2] = gg[1]; gg[3] = 0; gg[0] = 3;
01395         }
01396         gg = g++;
01397         continue;
01398     }
01399     if ( s > ss ) {
01400         if ( *s != '{' ) {
01401             MesPrint("&The CommuteInSet statement should have sets enclosed in {}.");
01402             error = 1;
01403             break;
01404         }
01405         ss = s;
01406         SKIPBRA2(ss) /* Note that parentheses were tested before */
01407         *ss = 0;
01408         s++;
01409     }
01410     name = s;
01411     s = SkipAName(s);
01412     c = *s; *s = 0;
01413     if ( ( type = GetName(AC.varnames,name,&number,NOAUTO) ) != CFUNCTION ) {
01414         MesPrint("&%s is not a function or tensor",name);
01415         error = 1;
01416     }
01417     else if ( functions[number].commute == 0 ){
01418         MesPrint("&%s is not a noncommuting function or tensor",name);
01419         error = 1;
01420     }
01421     else {
01422         *g++ = number+FUNCTION;
01423         functions[number].flags |= COULDCOMMUTE;
01424         if ( number+FUNCTION >= GAMMA && number+FUNCTION <= GAMMASEVEN ) {
01425             functions[GAMMA-FUNCTION].flags |= COULDCOMMUTE;
01426             functions[GAMMAI-FUNCTION].flags |= COULDCOMMUTE;
01427             functions[GAMMAFIVE-FUNCTION].flags |= COULDCOMMUTE;
01428             functions[GAMMASIX-FUNCTION].flags |= COULDCOMMUTE;
01429             functions[GAMMASEVEN-FUNCTION].flags |= COULDCOMMUTE;
01430         }
01431     }
01432     *s = c;
01433 }
01434 return(error);
01435 }
01436
01437 /*
01438 #] CoCommuteInSet :
01439 #[ CoFunction + ...:
01440
01441 Function declarations.
01442 The second parameter indicates commutation properties.
01443 The third parameter tells whether we have a tensor.
01444 */
01445
01446 int CoFunction(UBYTE *s, int comm, int istensor)
01447 {
01448     int type, error = 0, cplx, symtype, dim, argmax, argmin;
01449     WORD numfunction, reverseorder = 0, addone;

```

```

01450     UBYTE *name, *oldc, *par, c, cc;
01451     do {
01452         symtype = cplx = 0, argmin = argmax = -1;
01453         dim = 0;
01454         name = s;
01455         if ( ( s = SkipAName(s) ) == 0 ) {
01456 IllForm:     MesPrint("&Illegally formed function/tensor name");
01457                 error = 1;
01458                 s = SkipField(name,0);
01459                 goto eol;
01460         }
01461         oldc = s; cc = c = *s; *s = 0;
01462         if ( TestName(name) ) { *s = c; goto IllForm; }
01463         if ( c == '#' ) {
01464             s++;
01465             if ( tolower(*s) == 'r' ) cplx = VARTYPENONE;
01466             else if ( tolower(*s) == 'c' ) cplx = VARTYPECOMPLEX;
01467             else if ( tolower(*s) == 'i' ) cplx = VARTYPEIMAGINARY;
01468             else {
01469                 MesPrint("&Illegal specification for complexity of %s",name);
01470                 *oldc = c;
01471                 error = 1;
01472                 s = SkipField(s,0);
01473                 goto eol;
01474             }
01475             s++; cc = *s;
01476         }
01477         if ( cc == '{' ) {
01478             s++;
01479             if ( ( *s == 'd' || *s == 'D' ) && s[1] == '=' ) {
01480                 s += 2;
01481                 if ( *s == '-' || *s == '+' || FG.cTable[*s] == 1 ) {
01482                     ParseSignedNumber(dim,s)
01483                     if ( dim < -HALFMAX || dim > HALFMAX ) {
01484                         MesPrint("&Warning: dimension of %s (%d) out of range"
01485                                 ,name,dim);
01486                     }
01487                 }
01488                 if ( *s != '}' ) goto IllDim;
01489                 else s++;
01490             }
01491             else {
01492 IllDim:     MesPrint("&Error: Illegal dimension field for variable %s",name);
01493                 error = 1;
01494                 s = SkipField(s,0);
01495                 goto eol;
01496             }
01497             cc = *s;
01498         }
01499         if ( cc == '(' ) {
01500             s++;
01501             if ( *s == '-' ) {
01502                 reverseorder = REVERSEORDER;
01503                 s++;
01504             }
01505             else {
01506                 reverseorder = 0;
01507             }
01508             par = s;
01509             while ( FG.cTable[*s] == 0 ) s++;
01510             cc = *s; *s = 0;
01511             if ( s <= par ) {
01512 illegsym:   *s = cc;
01513                 MesPrint("&Illegal specification for symmetry of %s",name);
01514                 *oldc = c;
01515                 error = 1;
01516                 s = SkipField(s,1);
01517                 goto eol;
01518             }
01519             if ( StrICont(par,(UBYTE *)"symmetric") == 0 ) symtype = SYMMETRIC;
01520             else if ( StrICont(par,(UBYTE *)"antisymmetric") == 0 ) symtype = ANTISYMMETRIC;
01521             else if ( ( StrICont(par,(UBYTE *)"cyclesymmetric") == 0 )
01522                 || ( StrICont(par,(UBYTE *)"cyclic") == 0 ) ) symtype = CYCLESYMMETRIC;
01523             else if ( ( StrICont(par,(UBYTE *)"rcyclesymmetric") == 0 )
01524                 || ( StrICont(par,(UBYTE *)"rcyclic") == 0 )
01525                 || ( StrICont(par,(UBYTE *)"reversecyclic") == 0 ) ) symtype = RCYCLESYMMETRIC;
01526             else goto illegsym;
01527             *s = cc;
01528             if ( *s != ')' || ( s[1] && s[1] != ',' && s[1] != '<' ) ) {
01529                 Warning("&Excess information in symmetric properties currently ignored");
01530                 s = SkipField(s,1);
01531             }
01532             else s++;
01533             symtype |= reverseorder;
01534             cc = *s;
01535         }
01536 retry;;

```



```

01537         if ( cc == '<' ) {
01538             s++; addone = 0;
01539             if ( *s == '=' ) { addone++; s++; }
01540             argmax = 0;
01541             while ( FG.cTable[*s] == 1 ) { argmax = 10*argmax + *s++ - '0'; }
01542             argmax += addone;
01543             par = s;
01544             while ( FG.cTable[*s] == 0 ) s++;
01545             if ( s > par ) {
01546                 cc = *s; *s = 0;
01547                 if ( ( StrICont(par,(UBYTE *)"arguments") == 0 )
01548                     || ( StrICont(par,(UBYTE *)"args") == 0 ) ) {}
01549                 else {
01550                     Warning("&Illegal information in number of arguments properties currently
01551 ignored");
01552                     error = 1;
01553                     }
01554                     *s = cc;
01555                     }
01556                     if ( argmax <= 0 ) {
01557                         MesPrint("&Error: Cannot have fewer than 0 arguments for variable %s",name);
01558                         error = 1;
01559                     }
01560                     cc = *s;
01561             }
01562             if ( cc == '>' ) {
01563                 s++; addone = 1;
01564                 if ( *s == '=' ) { addone = 0; s++; }
01565                 argmin = 0;
01566                 while ( FG.cTable[*s] == 1 ) { argmin = 10*argmin + *s++ - '0'; }
01567                 argmin += addone;
01568                 par = s;
01569                 while ( FG.cTable[*s] == 0 ) s++;
01570                 if ( s > par ) {
01571                     cc = *s; *s = 0;
01572                     if ( ( StrICont(par,(UBYTE *)"arguments") == 0 )
01573                         || ( StrICont(par,(UBYTE *)"args") == 0 ) ) {}
01574                     else {
01575                         Warning("&Illegal information in number of arguments properties currently
01576 ignored");
01577                         error = 1;
01578                     }
01579                     *s = cc;
01580                     }
01581                     cc = *s;
01582             }
01583             if ( cc == '<' ) goto retry;
01584             if ( ( AC.AutoDeclareFlag == 0 &&
01585                 ( ( type = GetName(AC.exprnames,name,&numfunction,NOAUTO) )
01586                   != NAMENOTFOUND ) )
01587                 || ( ( type = GetName(*AC.activenames),name,&numfunction,NOAUTO) ) != NAMENOTFOUND ) ) {
01588                 if ( type != CFUNCTION ) error = NameConflict(type,name);
01589                 else {
01590                     /* FUNCTIONS fun = (FUNCTIONS) (AC.Functions->lijst) + numfunction-FUNCTION; */
01591                     FUNCTIONS fun = (FUNCTIONS) (AC.Functions->lijst) + numfunction;
01592                     if ( fun->tabl != 0 ) {
01593                         MesPrint("&Illegal attempt to change table into function");
01594                         error = 1;
01595                     }
01596                     fun->complex = cplx;
01597                     fun->commute = comm;
01598                     if ( istensor && fun->spec == 0 ) {
01599                         MesPrint("&Function %s changed to tensor",name);
01600                         error = 1;
01601                     }
01602                     else if ( istensor == 0 && fun->spec > 0 ) {
01603                         MesPrint("&Tensor %s changed to function",name);
01604                         error = 1;
01605                     }
01606                     fun->spec = istensor;
01607                     if ( fun->symmetric != symtype ) {
01608                         fun->symmetric = symtype;
01609                         AC.SymChangeFlag = 1;
01610                     }
01611                     fun->maxnumargs = argmax;
01612                     fun->minnumargs = argmin;
01613                 }
01614             }
01615             else {
01616                 AddFunction(name,comm,istensor,cplx,symtype,dim,argmax,argmin);
01617             }
01618             *oldc = c;
01619             eol: while ( *s == ',' ) s++;
01620             } while ( *s );
01621             return(error);

```

```

01622 }
01623
01624 int CoNFunction(UBYTE *s) { return(CoFunction(s,1,0)); }
01625 int CoCFunction(UBYTE *s) { return(CoFunction(s,0,0)); }
01626 int CoNTensor(UBYTE *s) { return(CoFunction(s,1,2)); }
01627 int CoCTensor(UBYTE *s) { return(CoFunction(s,0,2)); }
01628
01629 /*
01630      #] CoFunction + ....
01631      #[ DoTable :
01632
01633      Syntax:
01634      Table [check] [strict|relax] [zerofill] name(:1:2,...,regular arguments);
01635      name must be the name of a regular function.
01636      the table indices must be the first arguments.
01637      The parenthesis indicates 'name' as opposed to the options.
01638
01639      We leave behind:
01640      a struct tabl in the FUNCTION struct
01641      Regular table:
01642          an array tablepointers for the pointers to elements of rhs
01643          in the compiler struct cbuf[T->bufnum]
01644          an array MINMAX T->mm with the minima and maxima
01645          a prototype array
01646          an offset in the compiler buffer for the pattern to be matched
01647      Sparse table:
01648          Just the number of dimensions
01649          We will keep track of the number of defined elements in totind
01650          and in tablepointers we will have numind+1 positions for each
01651          element. The first numind elements for the indices and the
01652          last one for the element in cbuf[T->bufnum].rhs
01653
01654          If the number of dimensions is *<number>, there is not a fixed
01655          number of dimensions. Just a maximum. In that case the first
01656          index should be the number of other dimensions. This first index
01657          does not count in <number>.
01658
01659      Complication: to preserve speed we need a prototype and a pattern
01660      for each thread when we use WITHPTHREADS. This is because we write
01661      into those when looking for the pattern.
01662 */
01663
01664 static int nwarntab = 1;
01665
01666 int DoTable(UBYTE *s, int par)
01667 {
01668     GETIDENTITY
01669     UBYTE *name, *p, *inp, c;
01670     int i, j, k, sparseflag = 0, rflag = 0, checkflag = 0;
01671     int error = 0, ret, oldcbufnum, oldEside;
01672     WORD funnum, type, *OldWork, *w, *ww, *t, *tt, *flags1, oldnumrhs,oldnumlhs;
01673     LONG oldcpointer;
01674     MINMAX *mm, *mml;
01675     LONG x, y;
01676     TABLES T;
01677     CBUF *C;
01678
01679     while ( *s == ',' ) s++;
01680     do {
01681         name = s;
01682         if ( ( s = SkipAName(s) ) == 0 ) {
01683             IllForm: MesPrint("&Illegal name or option in table declaration");
01684             return(1);
01685         }
01686         c = *s; *s = 0;
01687         if ( TestName(name) ) { *s = c; goto IllForm; }
01688         *s = c;
01689         if ( *s == '(' ) break;
01690         if ( *s != ',' ) {
01691             MesPrint("&Illegal definition of table");
01692             return(1);
01693         }
01694         *s = 0;
01695     } while ( *s == ',' );
01696
01697     /*
01698     Secondary options
01699
01700     if ( StrICmp(name, (UBYTE *)("check" )) == 0 ) checkflag = 1;
01701     else if ( StrICmp(name, (UBYTE *)("zero" )) == 0 ) checkflag = 2;
01702     else if ( StrICmp(name, (UBYTE *)("one" )) == 0 ) checkflag = 3;
01703     else if ( StrICmp(name, (UBYTE *)("strict")) == 0 ) rflag = 1;
01704     else if ( StrICmp(name, (UBYTE *)("relax" )) == 0 ) rflag = -1;
01705     else if ( StrICmp(name, (UBYTE *)("zerofill" )) == 0 ) { rflag = -2; checkflag = 2; }
01706     else if ( StrICmp(name, (UBYTE *)("onefill" )) == 0 ) { rflag = -3; checkflag = 3; }
01707     else if ( StrICmp(name, (UBYTE *)("sparse")) == 0 ) sparseflag |= 1;
01708     else if ( StrICmp(name, (UBYTE *)("base")) == 0 ) sparseflag |= 3;
01709     else if ( StrICmp(name, (UBYTE *)("tablebase")) == 0 ) sparseflag |= 3;
01710     else {

```

```

01709         MesPrint("&Illegal option in table definition: '%s'",name);
01710         error = 1;
01711     }
01712     *s++ = ',';
01713     while ( *s == ',' ) s++;
01714 } while ( *s );
01715 if ( name == s || *s == 0 ) {
01716     MesPrint("&Illegal name or option in table declaration");
01717     return(1);
01718 }
01719 *s = 0;          /* *s could only have been a parenthesis */
01720 if ( sparseflag ) {
01721     if ( checkflag == 1 ) rflag = 0;
01722     else if ( checkflag == 2 ) rflag = -2;
01723     else if ( checkflag == 3 ) rflag = -3;
01724     else rflag = -1;
01725 }
01726 if ( ( ret = GetVar(name,&type,&funnum,CFUNCTION,NOAUTO) ) ==
01727     NAMENOTFOUND ) {
01728     if ( par == 0 ) {
01729         funnum = EntVar(CFUNCTION,name,0,1,0,0);
01730     }
01731     else if ( par == 1 || par == 2 ) {
01732         funnum = EntVar(CFUNCTION,name,0,0,0,0);
01733     }
01734 }
01735 else if ( ret <= 0 ) {
01736     funnum = EntVar(CFUNCTION,name,0,0,0,0);
01737     error = 1;
01738 }
01739 else {
01740     if ( par == 2 ) {
01741         if ( nwarntab ) {
01742             Warning("Table now declares its (commuting) function.");
01743             Warning("Earlier definition in Function statement obsolete. Please remove.");
01744             nwarntab = 0;
01745         }
01746     }
01747     else {
01748         error = 1;
01749         MesPrint("&(N)(C)Tables should not be declared previously");
01750     }
01751 }
01752 if ( functions[funnum].spec > 0 ) {
01753     MesPrint("&Tensors cannot become tables");
01754     return(1);
01755 }
01756 if ( functions[funnum].symmetric > 0 ) {
01757     MesPrint("&Functions with nontrivial symmetrization properties cannot become tables");
01758     return(1);
01759 }
01760 if ( functions[funnum].tabl ) {
01761     MesPrint("&Redefinition of an existing table is not allowed.");
01762     return(1);
01763 }
01764 functions[funnum].tabl = T = (TABLES)MALLOC1(sizeof(struct TableS),"table");
01765 /*
01766 Next we find the size of the table (if it is not sparse)
01767 */
01768 T->defined = T->mdefined = 0; T->sparse = sparseflag; T->mm = 0; T->flags = 0;
01769 T->numtree = 0; T->rootnum = 0; T->MaxTreeSize = 0;
01770 T->boomlijst = 0;
01771 T->strict = rflag;
01772 T->bounds = checkflag;
01773 T->bufnum = inicbufs();
01774 T->argtail = 0;
01775 T->sparse = 0;
01776 T->buffersize = 8;
01777 T->buffers = (WORD *)MALLOC1(sizeof(WORD)*T->buffersize,"Table buffers");
01778 T->buffersfill = 0;
01779 T->buffers[T->buffersfill++] = T->bufnum;
01780 T->mode = 0;
01781 T->numdummies = 0;
01782 mm = T->mm;
01783 T->numind = 0;
01784 if ( rflag > 0 ) AC.MustTestTable++;
01785 T->totind = 0;          /* Table hasn't been checked */
01786
01787 p = s; *s = '(';
01788 if ( sparseflag ) {
01789     /*
01790     First copy the tail, just in case we will construct a tablebase
01791     Note that we keep the ( to indicate a tail
01792     The actual arguments can be found after the comma. Before we have
01793     the dimension which the tablebase will need for consistency checking.
01794     */
01795     inp = p+1;

```

```

01796     SKIPBRA3(inp)
01797     c = *inp; *inp = 0;
01798     T->argtail = strDupl(p,"argtail");
01799     *inp = c;
01800 /*
01801     Now the regular compilation
01802 */
01803     inp = p++;
01804     if ( *p == '<' ) {
01805         WORD inc = 1;
01806         p++;
01807         if ( *p == '=' ) { inc++; p++; }
01808 /*
01809         We will use one extra number for telling how many there really are.
01810 */
01811         x = 0;
01812         while ( *p <= '9' && *p >= '0' ) x = 10*x + (*p++-'0');
01813         x = -x-inc;
01814         if ( x == -1 ) {
01815             MesPrint("&Maximum number of dimensions in *-table should be at least one.");
01816             error = 1;
01817             goto FinishUp2;
01818         }
01819     }
01820     else {
01821         ParseNumber(x,p)
01822         if ( FG.cTable[p[-1]] != 1 || ( *p != ',' && *p != ')' ) ) {
01823             p = inp;
01824             MesPrint("&First argument in a sparse table must be a number of dimensions");
01825             error = 1;
01826             x = 1;
01827         }
01828     }
01829     T->numind = x;
01830     T->mm = (MINMAX *)Malloc1(ABS(x)*sizeof(MINMAX),"table dimensions");
01831     T->flags = (WORD *)Malloc1(ABS(x)*sizeof(WORD),"table flags");
01832     mm = T->mm;
01833     inp = p;
01834     if ( *inp != ')' ) inp++;
01835     T->totind = 0; /* At the moment there are this many */
01836     T->tablepointers = 0;
01837     T->reserved = 0;
01838 }
01839 else {
01840     T->numind = 0;
01841     T->totind = 1;
01842     for(;;) { /* Read the dimensions as far as they can be recognized */
01843         inp = ++p;
01844         if ( FG.cTable[*p] != 1 && *p != '+' && *p != '-' ) break;
01845         ParseSignedNumber(x,p)
01846         if ( FG.cTable[p[-1]] != 1 || *p != ':' ) break;
01847         p++;
01848         ParseSignedNumber(y,p)
01849         if ( FG.cTable[p[-1]] != 1 || ( *p != ',' && *p != ')' ) ) {
01850             MesPrint("&Illegal dimension field in table declaration");
01851             return(1);
01852         }
01853         mml = (MINMAX *)Malloc1((T->numind+1)*sizeof(MINMAX),"table dimensions");
01854         flags1 = (WORD *)Malloc1((T->numind+1)*sizeof(WORD),"table flags");
01855         for ( i = 0; i < T->numind; i++ ) { mml[i] = T->mm[i]; flags1[i] = T->flags[i]; }
01856         if ( T->mm ) M_free(T->mm,"table dimensions");
01857         if ( T->flags ) M_free(T->flags,"table flags");
01858         T->mm = mml;
01859         T->flags = flags1;
01860         mm = T->mm + T->numind;
01861         mm->mini = x; mm->maxi = y;
01862         T->totind += mm->maxi-mm->mini+1;
01863         T->numind++;
01864         if ( *p == ')' ) { inp = p; break; }
01865     }
01866     w = T->tablepointers
01867     = (WORD *)Malloc1(TABLEEXTENSION*sizeof(WORD)*(T->totind),"table pointers");
01868     i = T->totind;
01869     for ( i = TABLEEXTENSION*T->totind; i > 0; i-- ) *w++ = -1; /* means: undefined */
01870     for ( i = T->numind-1, x = 1; i >= 0; i-- ) {
01871         T->mm[i].size = x; /* Defines increment in this dimension */
01872         x *= T->mm[i].maxi - T->mm[i].mini + 1;
01873     }
01874 }
01875 /*
01876     Now we redo the 'function part' and send it to the compiler.
01877     The prototype has to be picked up properly.
01878 */
01879     AT.WorkPointer++; /* We need one extra word later */
01880     OldWork = AT.WorkPointer;
01881     oldcbufnum = AC.cbufnum;
01882     AC.cbufnum = T->bnum;

```

```

01883     C = cbuf+AC.cbunum;
01884     oldcpointer = C->Pointer - C->Buffer;
01885     oldnumlhs = C->numlhs;
01886     oldnumrhs = C->numrhs;
01887     AddLHS(AC.cbunum);
01888     while ( s >= name ) *--inp = *s--;
01889     w = AT.WorkPointer;
01890     AC.ProtoType = w;
01891     *w++ = SUBEXPRESSION;
01892     *w++ = SUBEXPSIZE;
01893     *w++ = 0;
01894     *w++ = 1;
01895     *w++ = AC.cbunum;
01896     FILLSUB(w);
01897     AC.WildC = w;
01898     AC.NwildC = 0;
01899     AT.WorkPointer = w + 4*AM.MaxWildcards;
01900     if ( ( ret = CompileAlgebra(inp,LHSIDE,AC.ProtoType) ) < 0 ) {
01901         error = 1; goto FinishUp;
01902     }
01903     if ( AC.NwildC && SortWild(w,AC.NwildC) ) error = 1;
01904     w += AC.NwildC;
01905     i = w-OldWork;
01906     OldWork[1] = i;
01907 /*
01908     Basically we have to pull this pattern through Generator in case
01909     there are functions inside functions, or parentheses.
01910     We have to temporarily disable the .tabl to avoid problems with
01911     TestSub.
01912     Essential: we need to start NewSort twice to avoid the PutOut routines.
01913     The ground pattern is sitting in C->numrhs, but it could be that it
01914     has subexpressions in it. Hence it has to be worked out as the lhs in
01915     id statements (in comexpr.c).
01916 */
01917     OldWork[2] = C->numrhs;
01918     *w++ = 1; *w++ = 1; *w++ = 3;
01919     OldWork[-1] = w-OldWork+1;
01920     AT.WorkPointer = w;
01921     ww = C->rhs[C->numrhs];
01922     for ( j = 0; j < *ww; j++ ) w[j] = ww[j];
01923     AT.WorkPointer = w+*w;
01924     if ( *ww == 0 || ww[*ww] != 0 ) {
01925         MesPrint("&Illegal table pattern definition");
01926         AC.lhdollarflag = 0;
01927         error = 1;
01928     }
01929     if ( error ) goto FinishUp;
01930
01931     if ( NewSort(BHEAD0) || NewSort(BHEAD0) ) { error = 1; goto FinishUp; }
01932     AN.RepPoint = AT.RepCount + 1;
01933     AC.lhdollarflag = 0; oldEside = AR.Eside; AR.Eside = LHSIDE;
01934     AR.Cnumlhs = C->numlhs;
01935     functions[funnum].tabl = 0;
01936     if ( Generator(BHEAD w,C->numlhs) ) {
01937         functions[funnum].tabl = T;
01938         AR.Eside = oldEside;
01939         LowerSortLevel(); LowerSortLevel(); goto FinishUp;
01940     }
01941     functions[funnum].tabl = T;
01942     AR.Eside = oldEside;
01943     AT.WorkPointer = w;
01944     if ( EndSort(BHEAD w,0) < 0 ) { LowerSortLevel(); goto FinishUp; }
01945     if ( *w == 0 || *(w+*w) != 0 ) {
01946         MesPrint("&Irregular pattern in table definition");
01947         error = 1;
01948         goto FinishUp;
01949     }
01950     LowerSortLevel();
01951     if ( AC.lhdollarflag ) {
01952         MesPrint("&Unexpanded dollar variables are not allowed in table definition");
01953         error = 1;
01954         goto FinishUp;
01955     }
01956     AT.WorkPointer = ww = w + *w;
01957     if ( ww[-1] != 3 || ww[-2] != 1 || ww[-3] != 1 ) {
01958         MesPrint("&Coefficient of pattern in table definition should be 1.");
01959         error = 1;
01960         goto FinishUp;
01961     }
01962     AC.DumNum = 0;
01963 /*
01964     Now we have to allocate space for prototype+pattern
01965     In the case of TFORM we need extra pointers, because each worker has its own
01966 */
01967     j = *w + ABS(T->numind)*2-3;
01968 #ifdef WITHPTHREADS
01969     { int n;

```

```

01970     T->prototypeSize = ((i+j)*sizeof(WORD)+2*sizeof(WORD *)) * AM.totalnumberofthreads;
01971     T->prototype = (WORD **)Malloca1(T->prototypeSize,"table prototype");
01972     T->pattern = T->prototype + AM.totalnumberofthreads;
01973     t = (WORD *) (T->pattern + AM.totalnumberofthreads);
01974     for ( n = 0; n < AM.totalnumberofthreads; n++ ) {
01975         T->prototype[n] = t;
01976         for ( k = 0; k < i; k++ ) *t++ = OldWork[k];
01977     }
01978     T->pattern[0] = t;
01979     j--; w++;
01980     w[1] += ABS(T->numind)*2;
01981     for ( k = 0; k < FUNHEAD; k++ ) *t++ = *w++;
01982     j -= FUNHEAD;
01983     for ( k = 0; k < ABS(T->numind); k++ ) { *t++ = -SNUMBER; *t++ = 0; j -= 2; }
01984     for ( k = 0; k < j; k++ ) *t++ = *w++;
01985     if ( sparseflag ) T->pattern[0][1] = t - T->pattern[0];
01986     k = t - T->pattern[0];
01987     for ( n = 1; n < AM.totalnumberofthreads; n++ ) {
01988         T->pattern[n] = t; tt = T->pattern[0];
01989         for ( i = 0; i < k; i++ ) *t++ = *tt++;
01990     }
01991 }
01992 #else
01993     T->prototypeSize = (i+j)*sizeof(WORD);
01994     T->prototype = (WORD *)Malloca1(T->prototypeSize, "table prototype");
01995     T->pattern = T->prototype + i;
01996     for ( k = 0; k < i; k++ ) T->prototype[k] = OldWork[k];
01997     t = T->pattern;
01998     j--; w++;
01999     w[1] += ABS(T->numind)*2;
02000     for ( k = 0; k < FUNHEAD; k++ ) *t++ = *w++;
02001     j -= FUNHEAD;
02002     for ( k = 0; k < ABS(T->numind); k++ ) { *t++ = -SNUMBER; *t++ = 0; j -= 2; }
02003     for ( k = 0; k < j; k++ ) *t++ = *w++;
02004     if ( sparseflag ) T->pattern[1] = t - T->pattern;
02005 #endif
02006 /*
02007     At this point we can pop the compilerbuffer.
02008 */
02009     C->Pointer = C->Buffer + oldcpointer;
02010     C->numrhs = oldnumrhs;
02011     C->numlhs = oldnumlhs;
02012 /*
02013     Now check whether wildcards get converted to dollars (for PARALLEL)
02014     We give a warning!
02015 */
02016 #ifdef WITHPTHREADS
02017     t = T->prototype[0];
02018 #else
02019     t = T->prototype;
02020 #endif
02021     tt = t + t[1]; t += SUBEXPSIZE;
02022     while ( t < tt ) {
02023         if ( *t == LOADDOLLAR ) {
02024             Warning("The use of $-variable assignments in tables disables parallel\
02025 execution for the whole program.");
02026             AM.hparallelflag |= NOPARALLEL_TBLDOLLAR;
02027             AC.mparallelflag |= NOPARALLEL_TBLDOLLAR;
02028             AddPotModdollar(t[2]);
02029         }
02030         t += t[1];
02031     }
02032 FinishUp:;
02033     AT.WorkPointer = OldWork - 1;
02034     AC.cbufnum = oldcbufnum;
02035 FinishUp2:;
02036     if ( T->sparse ) ClearTableTree(T);
02037     if ( ( sparseflag & 2 ) != 0 ) {
02038         if ( T->sparse == 0 ) { SpareTable(T); }
02039     }
02040     return(error);
02041 }
02042
02043 /*
02044     #] DoTable :
02045     #[ CoTable :
02046 */
02047
02048 int CoTable(UBYTE *s)
02049 {
02050     return(DoTable(s,2));
02051 }
02052
02053 /*
02054     #] CoTable :
02055     #[ CoNTable :
02056 */

```

```

02057
02058 int CoNTable(UBYTE *s)
02059 {
02060     return(DoTable(s,0));
02061 }
02062
02063 /*
02064     #] CoNTable :
02065     #[ CoCTable :
02066 */
02067
02068 int CoCTable(UBYTE *s)
02069 {
02070     return(DoTable(s,1));
02071 }
02072
02073 /*
02074     #] CoCTable :
02075     #[ EmptyTable :
02076 */
02077
02078 void EmptyTable(TABLES T)
02079 {
02080     int j;
02081     if ( T->sparse ) ClearTableTree(T);
02082     if ( T->boomlijst ) M_free(T->boomlijst,"TableTree");
02083     T->boomlijst = 0;
02084     for ( j = 0; j < T->buffersfill; j++ ) { /* was <= */
02085         finishcbuf(T->buffers[j]);
02086     }
02087     if ( T->buffers ) M_free(T->buffers,"Table buffers");
02088     finishcbuf(T->bufnum);
02089     T->bufnum = inicbufs();
02090     T->bufferssize = 8;
02091     T->buffers = (WORD *)Malloc1(sizeof(WORD)*T->bufferssize,"Table buffers");
02092     T->buffersfill = 0;
02093     T->buffers[T->buffersfill++] = T->bufnum;
02094     T->defined = T->mdefined = 0; T->flags = 0;
02095     T->numtree = 0; T->rootnum = 0; T->MaxTreeSize = 0;
02096     T->sparse = 0; T->reserved = 0;
02097     if ( T->spare ) {
02098         TABLES TT = T->spare;
02099         if ( TT->mm ) M_free(TT->mm,"tableminmax");
02100         if ( TT->flags ) M_free(TT->flags,"tableflags");
02101         if ( TT->tablepointers ) M_free(TT->tablepointers,"tablepointers");
02102         for ( j = 0; j < TT->buffersfill; j++ ) {
02103             finishcbuf(TT->buffers[j]);
02104         }
02105         if ( TT->boomlijst ) M_free(TT->boomlijst,"TableTree");
02106         if ( TT->buffers ) M_free(TT->buffers,"Table buffers");
02107         M_free(TT,"table");
02108         SpareTable(T);
02109     }
02110     else {
02111         WORD *w = T->tablepointers;
02112         j = T->totind;
02113         for ( j = TABLEEXTENSION*T->totind; j > 0; j-- ) *w++ = -1; /* means: undefined */
02114     }
02115 }
02116
02117 /*
02118     #] EmptyTable :
02119     #[ AddSet :
02120 */
02121
02122 int AddSet(UBYTE *name, WORD dim)
02123 {
02124     int nodenum, numset = AC.SetList.num;
02125     SETS set = (SETS)FromVarList(&AC.SetList);
02126     UBYTE *s;
02127     if ( name ) {
02128         set->name = AddName(AC.varnames,name,CSET,numset,&nodenum);
02129         s = name;
02130         while ( *s ) s++;
02131         set->namesize = (s-name)+1;
02132         set->node = nodenum;
02133     }
02134     else {
02135         set->name = 0;
02136         set->namesize = 0;
02137         set->node = -1;
02138     }
02139     set->first =
02140     set->last = AC.SetElementList.num; /* set has no elements yet */
02141     set->type = -1; /* undefined as of yet */
02142     set->dimension = dim;
02143     set->flags = 0;

```

```

02144     return(numset);
02145 }
02146
02147 /*
02148 #] AddSet :
02149 #[ DoElements :
02150
02151 Remark (25-mar-2011): If the dimension has been set (dim != MAXPOSITIVE)
02152 we want to test dimensions. Numbers count as dimension zero?
02153 */
02154
02155 int DoElements(UBYTE *s, SETS set, UBYTE *cname)
02156 {
02157     int type, error = 0, x, sgn, i;
02158     WORD numset, *e;
02159     UBYTE c, *cname;
02160     while ( *s ) {
02161         if ( *s == ',' ) { s++; continue; }
02162         sgn = 0;
02163         while ( *s == '-' || *s == '+' ) {
02164             if ( *s == '-' ) sgn ^= 1;
02165             s++;
02166         }
02167         cname = s;
02168         if ( FG.cTable[*s] == 0 || *s == '_' || *s == '[' ) {
02169             if ( ( s = SkipAName(s) ) == 0 ) {
02170                 MesPrint("&Illegal name in set definition");
02171                 return(1);
02172             }
02173             c = *s; *s = 0;
02174             if ( ( ( type = GetName(AC.exprnames,cname,&numset,NOAUTO) ) == NAMENOTFOUND )
02175                 && ( ( type = GetOName(AC.varnames,cname,&numset,WITHAUTO) ) == NAMENOTFOUND ) ) {
02176                 DUBIOUSV dv;
02177                 int nodenum;
02178                 MesPrint("&%s has not been declared",cname);
02179             }
02180             /*
02181             We enter a 'dubious' declaration to cut down on errors
02182             */
02183             numset = AC.DubiousList.num;
02184             dv = (DUBIOUSV)FromVarList(&AC.DubiousList);
02185             dv->name = AddName(AC.varnames,cname,CDUBIOUS,numset,&nodenum);
02186             dv->node = nodenum;
02187             set->type = type = CDUBIOUS;
02188             set->dimension = 0;
02189             error = 1;
02190         }
02191         if ( set->type == -1 ) {
02192             if ( type == CSYMBOL ) {
02193                 for ( i = set->first; i < set->last; i++ ) {
02194                     SetElements[i] += 2*MAXPOWER;
02195                 }
02196                 set->type = type;
02197             }
02198             if ( set->type != type && set->type != CDUBIOUS
02199                 && type != CDUBIOUS ) {
02200                 if ( set->type != CNUMBER || ( type != CSYMBOL
02201                     && type != CINDEXT ) ) {
02202                     MesPrint(
02203                         "&%s has not the same type as the other members of the set"
02204                         ,cname);
02205                     error = 1;
02206                     set->type = CDUBIOUS;
02207                 }
02208                 else {
02209                     if ( type == CSYMBOL ) {
02210                         for ( i = set->first; i < set->last; i++ ) {
02211                             SetElements[i] += 2*MAXPOWER;
02212                         }
02213                     }
02214                     set->type = type;
02215                 }
02216             }
02217             if ( set->dimension != MAXPOSITIVE ) { /* Dimension check */
02218                 switch ( set->type ) {
02219                     case CSYMBOL:
02220                         if ( symbols[numset].dimension != set->dimension ) {
02221                             MesPrint("&Dimension check failed in set %s, symbol %s",
02222                                 VARNAME(Sets,(set-Sets)),
02223                                 VARNAME(symbols,numset));
02224                             error = 1;
02225                             set->dimension = MAXPOSITIVE;
02226                         }
02227                         break;
02228                     case CVECTOR:
02229                         if ( vectors[numset-AM.OffsetVector].dimension != set->dimension ) {
02230                             MesPrint("&Dimension check failed in set %s, vector %s",

```



```

02231             VARNAME(Sets, (set-Sets)),
02232             VARNAME(vectors, (numset-AM.OffsetVector)));
02233         error = 1;
02234         set->dimension = MAXPOSITIVE;
02235     }
02236     break;
02237     case CFUNCTION:
02238         if ( functions[numset-FUNCTION].dimension != set->dimension ) {
02239             MesPrint("&Dimension check failed in set %s, function %s",
02240                     VARNAME(Sets, (set-Sets)),
02241                     VARNAME(functions, (numset-FUNCTION)));
02242             error = 1;
02243         }
02244         break;
02245         set->dimension = MAXPOSITIVE;
02246     }
02247 }
02248 if ( sgn ) {
02249     if ( type != CVECTOR ) {
02250         MesPrint("&Illegal use of - sign in set. Can use only with vector or number");
02251         error = 1;
02252     }
02253 }
02254 /*
02255     numset = AM.OffsetVector - numset;
02256     numset |= SPECMASK;
02257     numset = AM.OffsetVector - numset;
02258 */
02259     numset -= WILDMASK;
02260 }
02261 *s = c;
02262 if ( name == 0 && *s == '?' ) {
02263     s++;
02264     switch ( set->type ) {
02265         case CSYMBOL:
02266             numset = -numset; break;
02267         case CVECTOR:
02268             numset += WILD OFFSET; break;
02269         case CINDEX:
02270             numset |= WILDMASK; break;
02271         case CFUNCTION:
02272             numset |= WILDMASK; break;
02273     }
02274     AC.wildflag = 1;
02275 }
02276 /*
02277     Now add the element to the set.
02278 */
02279 e = (WORD *)FromVarList(&AC.SetElementList);
02280 *e = numset;
02281 (set->last)++;
02282 }
02283 else if ( FG.cTable[*s] == 1 ) {
02284     ParseNumber(x,s)
02285     if ( sgn ) x = -x;
02286     if ( x >= MAXPOWER || x <= -MAXPOWER ||
02287         ( set->type == CINDEX && ( x < 0 || x >= AM.OffsetIndex ) ) ) {
02288         MesPrint("&Illegal value for set element: %d",x);
02289         if ( AC.firstconstindex ) {
02290             MesPrint("&%0 <= Fixed indices < ConstIndex(which is %d)",
02291                     AM.OffsetIndex-1);
02292             MesPrint("&For setting ConstIndex, read the chapter on the setup file");
02293             AC.firstconstindex = 0;
02294         }
02295         error = 1;
02296         x = 0;
02297     }
02298 }
02299 /*
02300     Check what is allowed with the type.
02301 */
02302 if ( set->type == -1 ) {
02303     if ( x < 0 || x >= AM.OffsetIndex ) {
02304         for ( i = set->first; i < set->last; i++ ) {
02305             SetElements[i] += 2*MAXPOWER;
02306         }
02307         set->type = CSYMBOL;
02308     }
02309     else set->type = CNUMBER;
02310 }
02311 else if ( set->type == CDUBIOUS ) {}
02312 else if ( set->type == CNUMBER && x < 0 ) {
02313     for ( i = set->first; i < set->last; i++ ) {
02314         SetElements[i] += 2*MAXPOWER;
02315     }
02316     set->type = CSYMBOL;
02317 }
02318 else if ( set->type != CSYMBOL && ( x < 0 ||
02319     ( set->type != CINDEX && set->type != CNUMBER ) ) ) {

```

```

02318         MesPrint("&Illegal mixture of element types in set");
02319         error = 1;
02320         set->type = CDUBIOUS;
02321     }
02322     /*
02323         Allocate an element
02324     */
02325     e = (WORD *)FromVarList(&AC.SetElementList);
02326     (set->last)++;
02327     if ( set->type == CSYMBOL ) *e = x + 2*MAXPOWER;
02328     /* else if ( set->type == CINDEX ) *e = x; */
02329     else *e = x;
02330 }
02331 else {
02332     MesPrint("&Illegal object in list of set elements");
02333     return(1);
02334 }
02335 }
02336 if ( error == 0 && ( ( set->flags & ORDEREDSET ) == ORDEREDSET ) ) {
02337     /*
02338         The set->last-set->first list of numbers must be sorted.
02339         Because we plan here potentially thousands of elements we use
02340         a simple version of splitmerge. In ordered sets we can search
02341         later with a binary search.
02342     */
02343     SimpleSplitMerge(SetElements+set->first,set->last-set->first);
02344 }
02345 return(error);
02346 }
02347
02348 /*
02349     #] DoElements :
02350     #[ CoSet :
02351
02352     Set declarations.
02353 */
02354
02355 int CoSet(UBYTE *s)
02356 {
02357     int type, error = 0, ordered = 0;
02358     UBYTE *name = s, c, *ss;
02359     SETS set;
02360     WORD numberofset, dim = MAXPOSITIVE;
02361     #ifdef WITHFLOAT
02362     /*-----*/
02363     {
02364         WORD numeq = 0;
02365         LONG x;
02366         ss = s;
02367         while ( *ss && *ss != ':' ) { if ( *ss == '=' ) numeq++; ss++; }
02368         if ( *ss == 0 && numeq == 1 ) { /* We have the Set var = value; variety */
02369             while ( FG.cTable[*s] == 0 ) s++;
02370             ss = s; c = *s; *s = 0;
02371             if ( c != '=' ) {
02372 Proper:
02373                 MesPrint("&Proper syntax for value-set is `Set name = value'");
02374                 return(1);
02375             }
02376             x = 0; s++;
02377             while ( *s >= '0' && *s <= '9' ) x = 10*x + (*s++-'0');
02378             if ( *s ) goto Proper;
02379             if ( StrICmp(name,(UBYTE *)"maxweight") == 0 ) {
02380                 AC.tMaxWeight = x; /* Temporary. Made permanent later */
02381             }
02382             else if ( StrICmp(name,(UBYTE *)"defaultprecision") == 0 ) {
02383                 AC.tDefaultPrecision = x; /* Temporary. Made permanent later */
02384             }
02385             else {
02386                 MesPrint("&Illegal subkey in value set: %s",name);
02387                 return(1);
02388             }
02389         }
02390     }
02391     /*-----*/
02392     #endif
02393     if ( ( s = SkipAName(s) ) == 0 ) {
02394 IllForm:MesPrint("&Illegal name for set");
02395         return(1);
02396     }
02397     c = *s; *s = 0;
02398     if ( TestName(name) ) goto IllForm;
02399     if ( ( ( type = GetName(AC.exprnames,name,&numberofset,NOAUTO) ) != NAMENOTFOUND )
02400 || ( ( type = GetName(AC.varnames,name,&numberofset,NOAUTO) ) != NAMENOTFOUND ) ) {
02401         if ( type != CSET ) NameConflict(type,name);
02402         else {
02403             MesPrint("&There is already a set with the name %s",name);
02404         }

```

```

02405     return(1);
02406 }
02407 if ( c == 0 ) {
02408     numberofset = AddSet(name,0);
02409     set = Sets + numberofset;
02410     return(0); /* empty set */
02411 }
02412 *s = c; ss = s; /* ss marks the end of the name */
02413 if ( *s == '(' ) {
02414     UBYTE *sss, cc;
02415     s++; sss = s; /* Beginning of option */
02416     while ( *s != ',' && *s != ')' && *s ) s++;
02417     cc = *s; *s = 0;
02418     if ( StrICont(sss,(UBYTE *)"ordered") == 0 ) {
02419         ordered = ORDEREDSET;
02420     }
02421     else {
02422         MesPrint("&Error: Illegal option in set definition: %s",sss);
02423         error = 1;
02424     }
02425     *s = cc;
02426     if ( *s != ')' ) {
02427         MesPrint("&Error: Currently only one option allowed in set definition.");
02428         error = 1;
02429         while ( *s && *s != ')' ) s++;
02430     }
02431     s++;
02432 }
02433 if ( *s == '{' ) {
02434     s++;
02435     if ( ( *s == 'd' || *s == 'D' ) && s[1] == '=' ) {
02436         s += 2;
02437         if ( *s == '-' || *s == '+' || FG.cTable[*s] == 1 ) {
02438             ParseSignedNumber(dim,s)
02439             if ( dim < -HALFMAX || dim > HALFMAX ) {
02440                 MesPrint("&Warning: dimension of %s (%d) out of range"
02441                     ,name,dim);
02442             }
02443         }
02444         if ( *s != '}' ) goto IllDim;
02445         else s++;
02446     }
02447     else {
02448 IllDim: MesPrint("&Error: Illegal dimension field for set %s",name);
02449         error = 1;
02450         s = SkipField(s,0);
02451     }
02452     while ( *s == ',' ) s++;
02453 }
02454 c = *ss; *ss = 0;
02455 numberofset = AddSet(name,dim);
02456 *ss = c;
02457 set = Sets + numberofset;
02458 set->flags |= ordered;
02459 if ( *s != ':' ) {
02460     MesPrint("&Proper syntax is `Set name:elements'");
02461     return(1);
02462 }
02463 s++;
02464 error = DoElements(s,set,name);
02465 AC.SetList.numtemp = AC.SetList.num;
02466 AC.SetElementList.numtemp = AC.SetElementList.num;
02467 return(error);
02468 }
02469
02470 /*
02471 #] CoSet :
02472 #[ DoTempSet :
02473
02474     Gets a {} set definition and returns a set number if the set is
02475     properly structured. This number refers either to an already
02476     existing set, or to a set that is defined here.
02477     From and to refer to the contents. They exclude the {}.
02478 */
02479 int DoTempSet(UBYTE *from, UBYTE *to)
02480 {
02481     int i, num, j, sgn;
02482     WORD *e, *ep;
02483     UBYTE c;
02484     int setnum = AddSet(0,MAXPOSITIVE);
02485     SETS set = Sets + setnum, setp;
02486     set->name = -1;
02487     set->type = -1;
02488     c = *to; *to = 0;
02489     AC.wildflag = 0;
02490     while ( *from == ',' ) from++;

```

```

02492     if ( *from == '<' || *from == '>' ) {
02493         set->type = CRANGE;
02494         set->first = 3*MAXPOWER;
02495         set->last = -3*MAXPOWER;
02496         while ( *from == '<' || *from == '>' ) {
02497             if ( *from == '<' ) {
02498                 j = 1; from++;
02499                 if ( *from == '=' ) { from++; j++; }
02500             }
02501             else {
02502                 j = -1; from++;
02503                 if ( *from == '=' ) { from++; j--; }
02504             }
02505             sgn = 1;
02506             while ( *from == '-' || *from == '+' ) {
02507                 if ( *from == '-' ) sgn = -sgn;
02508                 from++;
02509             }
02510             ParseNumber(num,from)
02511             if ( *from && *from != ',' ) {
02512                 MesPrint("&Illegal number in ranged set definition");
02513                 return(-1);
02514             }
02515             if ( sgn < 0 ) num = -num;
02516             if ( num >= MAXPOWER || num <= -MAXPOWER ) {
02517                 Warning("Value in ranged set too big. Adjusted to infinity.");
02518                 if ( num > 0 ) num = 3*MAXPOWER;
02519                 else num = -3*MAXPOWER;
02520             }
02521             else if ( j == 2 ) num += 2*MAXPOWER;
02522             else if ( j == -2 ) num -= 2*MAXPOWER;
02523             if ( j > 0 ) set->first = num;
02524             else set->last = num;
02525             while ( *from == ',' ) from++;
02526         }
02527         if ( *from ) {
02528             MesPrint("&Definition of ranged set contains illegal objects");
02529             return(-1);
02530         }
02531     }
02532     else if ( DoElements(from,set,(UBYTE *)0) != 0 ) {
02533         AC.SetElementList.num = set->first;
02534         AC.SetList.num--; *to = c;
02535         return(-1);
02536     }
02537     *to = c;
02538     /*
02539     Now we have to test whether this set exists already.
02540     */
02541     num = set->last - set->first;
02542     for ( setp = Sets, i = 0; i < AC.SetList.num-1; i++, setp++ ) {
02543         if ( num != setp->last - setp->first ) continue;
02544         if ( set->type != setp->type ) continue;
02545         if ( set->type == CRANGE ) {
02546             if ( set->first == setp->first ) return(setp-Sets);
02547         }
02548         else {
02549             e = SetElements + set->first;
02550             ep = SetElements + setp->first;
02551             j = num;
02552             while ( --j >= 0 ) if ( *e++ != *ep++ ) break;
02553             if ( j < 0 ) {
02554                 AC.SetElementList.num = set->first;
02555                 AC.SetList.num--;
02556                 return(setp - Sets);
02557             }
02558         }
02559     }
02560     return(setnum);
02561 }
02562
02563 /*
02564 #] DoTempSet :
02565 #[ CoAuto :
02566
02567 To prepare first:
02568     Use of the proper pointers in the various declaration routines
02569     Proper action in .store and .clear
02570 */
02571
02572 int CoAuto(UBYTE *inp)
02573 {
02574     int retval;
02575
02576     AC.Symbols = &(AC.AutoSymbolList);
02577     AC.Vectors = &(AC.AutoVectorList);
02578     AC.Indices = &(AC.AutoIndexList);

```

```

02579     AC.Functions = &(AC.AutoFunctionList);
02580     AC.activenames = &(AC.autonames);
02581     AC.AutoDeclareFlag = WITHAUTO;
02582
02583     while ( *inp == ',' ) inp++;
02584     retval = CompileStatement(inp);
02585
02586     AC.AutoDeclareFlag = 0;
02587     AC.Symbols = &(AC.SymbolList);
02588     AC.Vectors = &(AC.VectorList);
02589     AC.Indices = &(AC.IndexList);
02590     AC.Functions = &(AC.FunctionList);
02591     AC.activenames = &(AC.varnames);
02592     return(retval);
02593 }
02594
02595 /*
02596  #] CoAuto :
02597  #[ AddDollar :
02598
02599     The actual addition. Special routine for additions 'on the fly'
02600 */
02601
02602 int AddDollar(UBYTE *name, WORD type, WORD *start, LONG size)
02603 {
02604     int nodenum, numdollar = AP.DollarList.num;
02605     WORD *s, *t;
02606     DOLLARS dol = (DOLLARS)FromVarList(&AP.DollarList);
02607     dol->name = AddName(AC.dollarnames, name, CDOLLAR, numdollar, &nodenum);
02608     dol->type = type;
02609     dol->node = nodenum;
02610     dol->zero = 0;
02611     dol->numdummies = 0;
02612 #ifdef WITHPTHREADS
02613     dol->pthreadlockread = dummylock;
02614     dol->pthreadlockwrite = dummylock;
02615 #endif
02616     dol->nfactors = 0;
02617     dol->factors = 0;
02618     AddRHS(AM.dbufnum, 1);
02619     AddLHS(AM.dbufnum);
02620     if ( start && size > 0 ) {
02621         dol->size = size;
02622         dol->where =
02623             s = (WORD *)Malloc1((size+1)*sizeof(WORD), "$-variable contents");
02624         t = start;
02625         while ( --size >= 0 ) *s++ = *t++;
02626         *s = 0;
02627     }
02628     else { dol->where = &(AM.dollarzero); dol->size = 0; }
02629     cbuf[AM.dbufnum].rhs[numdollar] = dol->where;
02630     cbuf[AM.dbufnum].CanCommu[numdollar] = 0;
02631     cbuf[AM.dbufnum].NumTerms[numdollar] = 0;
02632
02633     return(numdollar);
02634 }
02635
02636 /*
02637  #] AddDollar :
02638  #[ ReplaceDollar :
02639
02640     Replacements of dollar variables can happen at any time.
02641     For debugging purposes we should have a tracing facility.
02642
02643     Not in use???
02644 */
02645
02646 int ReplaceDollar(WORD number, WORD newtype, WORD *newstart, LONG newsize)
02647 {
02648     int error = 0;
02649     DOLLARS dol = Dollars + number;
02650     WORD *s, *t;
02651     LONG i;
02652     dol->type = newtype;
02653     if ( dol->size == newsize && newsize > 0 && newstart ) {
02654         s = dol->where; t = newstart; i = newsize;
02655         while ( --i >= 0 ) { if ( *s++ != *t++ ) break; }
02656         if ( i < 0 ) return(0);
02657     }
02658     if ( dol->where && dol->where != &(dol->zero) ) {
02659         M_free(dol->where, "dollar->where"); dol->where = &(dol->zero); dol->size = 0;
02660     }
02661     if ( newstart && newsize > 0 ) {
02662         dol->size = newsize;
02663         dol->where =
02664             s = (WORD *)Malloc1((newsize+1)*sizeof(WORD), "$-variable contents");
02665         t = newstart; i = newsize;

```

```

02666         while ( --i >= 0 ) *s++ = *t++;
02667         *s = 0;
02668     }
02669     return(error);
02670 }
02671
02672 /*
02673  #] ReplaceDollar :
02674  #[ AddDubious :
02675
02676     This adds a variable of which we do not know the proper type.
02677 */
02678
02679 int AddDubious(UBYTE *name)
02680 {
02681     int nodenum, numdubious = AC.DubiousList.num;
02682     DUBIOUSV dub = (DUBIOUSV)FromVarList (&AC.DubiousList);
02683     dub->name = AddName(AC.varnames,name,CDUBIOUS,numdubious,&nodenum);
02684     dub->node = nodenum;
02685     return(numdubious);
02686 }
02687
02688 /*
02689  #] AddDubious :
02690  #[ MakeDubious :
02691 */
02692
02693 int MakeDubious(NAMETREE *nametree, UBYTE *name, WORD *number)
02694 {
02695     NAMENODE *n;
02696     int node, newnode, i;
02697     if ( nametree->namenode == 0 ) return(-1);
02698     newnode = nametree->headnode;
02699     do {
02700         node = newnode;
02701         n = nametree->namenode+node;
02702         if ( ( i = StrCmp(name,nametree->namebuffer+n->name) ) < 0 )
02703             newnode = n->left;
02704         else if ( i > 0 ) newnode = n->right;
02705         else {
02706             if ( n->type != CDUBIOUS ) {
02707                 int numdubious = AC.DubiousList.num;
02708                 FUNCTIONS dub = (FUNCTIONS)FromVarList (&AC.DubiousList);
02709                 dub->name = n->name;
02710                 n->number = numdubious;
02711             }
02712             *number = n->number;
02713             return(CDUBIOUS);
02714         }
02715     } while ( newnode >= 0 );
02716     return(-1);
02717 }
02718
02719 /*
02720  #] MakeDubious :
02721  #[ NameConflict :
02722 */
02723
02724 static char *nametype[] = { "symbol", "index", "vector", "function",
02725                             "set", "expression" };
02726 static char *plural[] = { "", "n", "", "", "", "n" };
02727
02728 int NameConflict(int type, UBYTE *name)
02729 {
02730     if ( type == NAMENOTFOUND ) {
02731         MesPrint("%s has not been declared",name);
02732     }
02733     else if ( type != CDUBIOUS )
02734         MesPrint("%s has been declared as a%s %s already",
02735                 name,plural[type],nametype[type]);
02736     return(1);
02737 }
02738
02739 /*
02740  #] NameConflict :
02741  #[ AddExpression :
02742 */
02743
02744 int AddExpression(UBYTE *name, int x, int y)
02745 {
02746     int nodenum, numexpr = AC.ExpressionList.num;
02747     EXPRESSIONS expr = (EXPRESSIONS)FromVarList (&AC.ExpressionList);
02748     UBYTE *s;
02749     expr->status = x;
02750     expr->printflag = y;
02751     PUTZERO(expr->onfile);
02752     PUTZERO(expr->size);

```

```

02753     expr->renum = 0;
02754     expr->renumlists = 0;
02755     expr->hidelevel = 0;
02756     expr->inmem = 0;
02757     expr->bracketinfo = expr->newbracketinfo = 0;
02758     if ( name ) {
02759         expr->name = AddName(AC.exprnames, name, CEXPRESSION, numexpr, &nodenum);
02760         expr->node = nodenum;
02761         expr->replace = NEWLYDEFINEDEXPRESSION ;
02762         s = name;
02763         while ( *s ) s++;
02764         expr->namesize = (s-name)+1;
02765     }
02766     else {
02767         expr->replace = REDEFINEDEXPRESSION;
02768         expr->name = AC.TransEname;
02769         expr->node = -1;
02770         expr->namesize = 0;
02771     }
02772     expr->vflags = 0;
02773     expr->numdummies = 0;
02774     expr->numfactors = 0;
02775     #ifdef PARALLELCODE
02776     expr->partodo = 0;
02777 #endif
02778     expr->uflags = 0;
02779     return(numexpr);
02780 }
02781
02782 /*
02783  #] AddExpression :
02784  #[ GetLabel :
02785  */
02786
02787 int GetLabel(UBYTE *name)
02788 {
02789     int i;
02790     LONG newnum;
02791     UBYTE **NewLabelNames;
02792     int *NewLabel;
02793     for ( i = 0; i < AC.NumLabels; i++ ) {
02794         if ( StrCmp(name, AC.LabelNames[i]) == 0 ) return(i);
02795     }
02796     if ( AC.NumLabels >= AC.MaxLabels ) {
02797         newnum = 2*AC.MaxLabels;
02798         if ( newnum == 0 ) newnum = 10;
02799         if ( newnum > 32765 ) newnum = 32765;
02800         if ( newnum == AC.MaxLabels ) {
02801             MesPrint("&More than 32765 labels in one module. Please simplify.");
02802             Terminate(-1);
02803         }
02804         NewLabelNames = (UBYTE **)Malloc1((sizeof(UBYTE *)+sizeof(int))
02805             *newnum, "Labels");
02806         NewLabel = (int *) (NewLabelNames+newnum);
02807         for ( i = 0; i < AC.MaxLabels; i++ ) {
02808             NewLabelNames[i] = AC.LabelNames[i];
02809             NewLabel[i] = AC.Labels[i];
02810         }
02811         if ( AC.LabelNames ) M_free(AC.LabelNames, "Labels");
02812         AC.LabelNames = NewLabelNames;
02813         AC.Labels = NewLabel;
02814         AC.MaxLabels = newnum;
02815     }
02816     i = AC.NumLabels++;
02817     AC.LabelNames[i] = strDup1(name, "Labels");
02818     AC.Labels[i] = -1;
02819     return(i);
02820 }
02821
02822 /*
02823  #] GetLabel :
02824  #[ ResetVariables :
02825
02826     Resets the variables.
02827     par = 0   The list of temporary sets (after each .sort)
02828     par = 1   The list of local variables (after each .store)
02829     par = 2   All variables (after each .clear)
02830  */
02831
02832 void ResetVariables(int par)
02833 {
02834     int i, j;
02835     TABLES T;
02836     switch ( par ) {
02837     case 0 : /* Only the sets without a name */
02838         AC.SetList.num = AC.SetList.numtemp;
02839         AC.SetElementList.num = AC.SetElementList.numtemp;

```

```

02840         break;
02841     case 2 :
02842         for ( i = AC.SymbolList.numclear; i < AC.SymbolList.num; i++ )
02843             AC.varnames->namenode[symbols[i].node].type = CDELETE;
02844         AC.SymbolList.num = AC.SymbolList.numglobal = AC.SymbolList.numclear;
02845         for ( i = AC.VectorList.numclear; i < AC.VectorList.num; i++ )
02846             AC.varnames->namenode[vectors[i].node].type = CDELETE;
02847         AC.VectorList.num = AC.VectorList.numglobal = AC.VectorList.numclear;
02848         for ( i = AC.IndexList.numclear; i < AC.IndexList.num; i++ )
02849             AC.varnames->namenode[indices[i].node].type = CDELETE;
02850         AC.IndexList.num = AC.IndexList.numglobal = AC.IndexList.numclear;
02851         for ( i = AC.FunctionList.numclear; i < AC.FunctionList.num; i++ ) {
02852             AC.varnames->namenode[functions[i].node].type = CDELETE;
02853             if ( ( T = functions[i].tabl ) != 0 ) {
02854                 if ( T->tablepointers ) M_free(T->tablepointers,"tablepointers");
02855                 if ( T->prototype ) M_free(T->prototype,"tableprototype");
02856                 if ( T->mm ) M_free(T->mm,"tableminmax");
02857                 if ( T->flags ) M_free(T->flags,"tableflags");
02858                 if ( T->argtail ) M_free(T->argtail,"table arguments");
02859                 if ( T->boomlijst ) M_free(T->boomlijst,"TableTree");
02860                 for ( j = 0; j < T->buffersfill; j++ ) { /* was <= */
02861                     finishcbuf(T->buffers[j]);
02862                 }
02863                 /*[07apr2004 mt]:*/ /*memory leak*/
02864                 if ( T->buffers ) M_free(T->buffers,"Table buffers");
02865                 /*:[07apr2004 mt]*/
02866                 finishcbuf(T->bufnum);
02867                 if ( T->spare ) {
02868                     TABLES TT = T->spare;
02869                     if ( TT->mm ) M_free(TT->mm,"tableminmax");
02870                     if ( TT->flags ) M_free(TT->flags,"tableflags");
02871                     if ( TT->tablepointers ) M_free(TT->tablepointers,"tablepointers");
02872                     for ( j = 0; j < TT->buffersfill; j++ ) { /* was <= */
02873                         finishcbuf(TT->buffers[j]);
02874                     }
02875                     if ( TT->boomlijst ) M_free(TT->boomlijst,"TableTree");
02876                     /*[07apr2004 mt]:*/ /*memory leak*/
02877                     if ( TT->buffers ) M_free(TT->buffers,"Table buffers");
02878                     /*:[07apr2004 mt]*/
02879                     M_free(TT,"table");
02880                 }
02881                 M_free(T,"table");
02882             }
02883         }
02884         AC.FunctionList.num = AC.FunctionList.numglobal = AC.FunctionList.numclear;
02885         for ( i = AC.SetList.numclear; i < AC.SetList.num; i++ ) {
02886             if ( Sets[i].node >= 0 )
02887                 AC.varnames->namenode[Sets[i].node].type = CDELETE;
02888         }
02889         AC.SetList.numtemp = AC.SetList.num = AC.SetList.numglobal = AC.SetList.numclear;
02890         for ( i = AC.DubiousList.numclear; i < AC.DubiousList.num; i++ )
02891             AC.varnames->namenode[Dubious[i].node].type = CDELETE;
02892         AC.DubiousList.num = AC.DubiousList.numglobal = AC.DubiousList.numclear;
02893         AC.SetElementList.numtemp = AC.SetElementList.num =
02894             AC.SetElementList.numglobal = AC.SetElementList.numclear;
02895         CompactifyTree(AC.varnames,VARNAMES);
02896         AC.varnames->namefill = AC.varnames->globalnamefill = AC.varnames->clearnamefill;
02897         AC.varnames->nodefill = AC.varnames->globalnodefill = AC.varnames->clearnodefill;
02898
02899         for ( i = AC.AutoSymbolList.numclear; i < AC.AutoSymbolList.num; i++ )
02900             AC.autonames->namenode[
02901                 ((SYMBOLS)(AC.AutoSymbolList.lijst))[i].node].type = CDELETE;
02902         AC.AutoSymbolList.num = AC.AutoSymbolList.numglobal
02903             = AC.AutoSymbolList.numclear;
02904         for ( i = AC.AutoVectorList.numclear; i < AC.AutoVectorList.num; i++ )
02905             AC.autonames->namenode[
02906                 ((VECTORS)(AC.AutoVectorList.lijst))[i].node].type = CDELETE;
02907         AC.AutoVectorList.num = AC.AutoVectorList.numglobal
02908             = AC.AutoVectorList.numclear;
02909         for ( i = AC.AutoIndexList.numclear; i < AC.AutoIndexList.num; i++ )
02910             AC.autonames->namenode[
02911                 ((INDICES)(AC.AutoIndexList.lijst))[i].node].type = CDELETE;
02912         AC.AutoIndexList.num = AC.AutoIndexList.numglobal
02913             = AC.AutoIndexList.numclear;
02914         for ( i = AC.AutoFunctionList.numclear; i < AC.AutoFunctionList.num; i++ ) {
02915             AC.autonames->namenode[
02916                 ((FUNCTIONS)(AC.AutoFunctionList.lijst))[i].node].type = CDELETE;
02917             if ( ( T = ((FUNCTIONS)(AC.AutoFunctionList.lijst))[i].tabl ) != 0 ) {
02918                 if ( T->tablepointers ) M_free(T->tablepointers,"tablepointers");
02919                 if ( T->prototype ) M_free(T->prototype,"tableprototype");
02920                 if ( T->mm ) M_free(T->mm,"tableminmax");
02921                 if ( T->flags ) M_free(T->flags,"tableflags");
02922                 if ( T->argtail ) M_free(T->argtail,"table arguments");
02923                 if ( T->boomlijst ) M_free(T->boomlijst,"TableTree");
02924                 for ( j = 0; j < T->buffersfill; j++ ) { /* was <= */
02925                     finishcbuf(T->buffers[j]);
02926                 }
02927             }
02928         }

```



```

02927         if ( T->spare ) {
02928             TABLES TT = T->spare;
02929             if ( TT->mm ) M_free(TT->mm,"tableminmax");
02930             if ( TT->flags ) M_free(TT->flags,"tableflags");
02931             if ( TT->tablepointers ) M_free(TT->tablepointers,"tablepointers");
02932             for ( j = 0; j < TT->buffersfill; j++ ) { /* was <= */
02933                 finishcbuf(TT->buffers[j]);
02934             }
02935             if ( TT->boomlijst ) M_free(TT->boomlijst,"TableTree");
02936             M_free(TT,"table");
02937         }
02938         M_free(T,"table");
02939     }
02940 }
02941 AC.AutoFunctionList.num = AC.AutoFunctionList.numglobal
02942                        = AC.AutoFunctionList.numclear;
02943 CompactifyTree(AC.autonames,AUTONAMES);
02944 AC.autonames->namefill = AC.autonames->globalnamefill
02945                        = AC.autonames->clearnamefill;
02946 AC.autonames->nodefill = AC.autonames->globalnodefill
02947                        = AC.autonames->clearnodefill;
02948 ReleaseTB();
02949 break;
02950 case 1 :
02951     for ( i = AC.SymbolList.numglobal; i < AC.SymbolList.num; i++ )
02952         AC.varnames->namenode[symbols[i].node].type = CDELETE;
02953     AC.SymbolList.num = AC.SymbolList.numglobal;
02954     for ( i = AC.VectorList.numglobal; i < AC.VectorList.num; i++ )
02955         AC.varnames->namenode[vectors[i].node].type = CDELETE;
02956     AC.VectorList.num = AC.VectorList.numglobal;
02957     for ( i = AC.IndexList.numglobal; i < AC.IndexList.num; i++ )
02958         AC.varnames->namenode[indices[i].node].type = CDELETE;
02959     AC.IndexList.num = AC.IndexList.numglobal;
02960     for ( i = AC.FunctionList.numglobal; i < AC.FunctionList.num; i++ ) {
02961         AC.varnames->namenode[functions[i].node].type = CDELETE;
02962         if ( ( T = functions[i].tabl ) != 0 ) {
02963             if ( T->tablepointers ) M_free(T->tablepointers,"tablepointers");
02964             if ( T->prototype ) M_free(T->prototype,"tableprototype");
02965             if ( T->mm ) M_free(T->mm,"tableminmax");
02966             if ( T->flags ) M_free(T->flags,"tableflags");
02967             if ( T->argtail ) M_free(T->argtail,"table arguments");
02968             if ( T->boomlijst ) M_free(T->boomlijst,"TableTree");
02969             for ( j = 0; j < T->buffersfill; j++ ) { /* was <= */
02970                 finishcbuf(T->buffers[j]);
02971             }
02972             /*[07apr2004 mt]:*/ /*memory leak*/
02973             if ( T->buffers ) M_free(T->buffers,"Table buffers");
02974             /*:[07apr2004 mt]*/
02975             finishcbuf(T->bufnum);
02976             if ( T->spare ) {
02977                 TABLES TT = T->spare;
02978                 if ( TT->mm ) M_free(TT->mm,"tableminmax");
02979                 if ( TT->flags ) M_free(TT->flags,"tableflags");
02980                 if ( TT->tablepointers ) M_free(TT->tablepointers,"tablepointers");
02981                 for ( j = 0; j < TT->buffersfill; j++ ) { /* was <= */
02982                     finishcbuf(TT->buffers[j]);
02983                 }
02984                 if ( TT->boomlijst ) M_free(TT->boomlijst,"TableTree");
02985                 /*[07apr2004 mt]:*/ /*memory leak*/
02986                 if ( TT->buffers ) M_free(TT->buffers,"Table buffers");
02987                 /*:[07apr2004 mt]*/
02988                 M_free(TT,"table");
02989             }
02990             M_free(T,"table");
02991         }
02992     }
02993 #ifdef TABLECLEANUP
02994 {
02995     int j;
02996     WORD *tp;
02997     for ( i = 0; i < AC.FunctionList.numglobal; i++ ) {
02998         /*
02999         Now, if the table definition is from after the .global
03000         while the function is from before, there is a problem.
03001         This could be resolved by defining CTable (=Table), Ntable
03002         and do away with the previous function definition.
03003         */
03004         if ( ( T = functions[i].tabl ) != 0 ) {
03005             /*
03006             First restore overwritten definitions.
03007             */
03008             if ( T->sparse ) {
03009                 T->totind = T->mdefined;
03010                 for ( j = 0, tp = T->tablepointers; j < T->totind; j++ ) {
03011                     tp += ABS(T->numind);
03012 #if TABLEEXTENSION == 2
03013                     tp[0] = tp[1];

```

```

03014 #else
03015         tp[0] = tp[2];
03016         tp[1] = tp[3];
03017         tp[4] = tp[5];
03018 #endif
03019         tp += TABLEEXTENSION;
03020     }
03021     RedoTableTree(T,T->totind);
03022     if ( T->spare ) {
03023         TABLES TT = T->spare;
03024         TT->totind = TT->mdefined;
03025         for ( j = 0, tp = TT->tablepointers; j < TT->totind; j++ ) {
03026             tp += ABS(TT->numind);
03027 #if TABLEEXTENSION == 2
03028             tp[0] = tp[1];
03029 #else
03030             tp[0] = tp[2];
03031             tp[1] = tp[3];
03032             tp[4] = tp[5];
03033 #endif
03034             tp += TABLEEXTENSION;
03035         }
03036         RedoTableTree(TT,TT->totind);
03037         cbuf[TT->bufnum].numlhs = cbuf[TT->bufnum].mnumlhs;
03038         cbuf[TT->bufnum].numrhs = cbuf[TT->bufnum].mnumrhs;
03039     }
03040 }
03041 else {
03042     for ( j = 0, tp = T->tablepointers; j < T->totind; j++ ) {
03043 #if TABLEEXTENSION == 2
03044         tp[0] = tp[1];
03045 #else
03046         tp[0] = tp[2];
03047         tp[1] = tp[3];
03048         tp[4] = tp[5];
03049 #endif
03050     }
03051     T->defined = T->mdefined;
03052 }
03053 cbuf[T->bufnum].numlhs = cbuf[T->bufnum].mnumlhs;
03054 cbuf[T->bufnum].numrhs = cbuf[T->bufnum].mnumrhs;
03055 }
03056 }
03057 }
03058 #endif
03059 AC.FunctionList.num = AC.FunctionList.numglobal;
03060 for ( i = AC.SetList.numglobal; i < AC.SetList.num; i++ ) {
03061     if ( Sets[i].node >= 0 )
03062         AC.varnames->namenode[Sets[i].node].type = CDELETE;
03063 }
03064 AC.SetList.numtemp = AC.SetList.num = AC.SetList.numglobal;
03065 for ( i = AC.DubiousList.numglobal; i < AC.DubiousList.num; i++ )
03066     AC.varnames->namenode[Dubious[i].node].type = CDELETE;
03067 AC.DubiousList.num = AC.DubiousList.numglobal;
03068 AC.SetElementList.numtemp = AC.SetElementList.num =
03069     AC.SetElementList.numglobal;
03070 CompactifyTree(AC.varnames,VARNAMES);
03071 AC.varnames->namefill = AC.varnames->globalnamefill;
03072 AC.varnames->nodefill = AC.varnames->globalnodefill;
03073
03074 for ( i = AC.AutoSymbolList.numglobal; i < AC.AutoSymbolList.num; i++ )
03075     AC.autonames->namenode[
03076         ((SYMBOLS)(AC.AutoSymbolList.lijst))[i].node].type = CDELETE;
03077 AC.AutoSymbolList.num = AC.AutoSymbolList.numglobal;
03078 for ( i = AC.AutoVectorList.numglobal; i < AC.AutoVectorList.num; i++ )
03079     AC.autonames->namenode[
03080         ((VECTORS)(AC.AutoVectorList.lijst))[i].node].type = CDELETE;
03081 AC.AutoVectorList.num = AC.AutoVectorList.numglobal;
03082 for ( i = AC.AutoIndexList.numglobal; i < AC.AutoIndexList.num; i++ )
03083     AC.autonames->namenode[
03084         ((INDICES)(AC.AutoIndexList.lijst))[i].node].type = CDELETE;
03085 AC.AutoIndexList.num = AC.AutoIndexList.numglobal;
03086 for ( i = AC.AutoFunctionList.numglobal; i < AC.AutoFunctionList.num; i++ ) {
03087     AC.autonames->namenode[
03088         ((FUNCTIONS)(AC.AutoFunctionList.lijst))[i].node].type = CDELETE;
03089     if ( ( T = ((FUNCTIONS)(AC.AutoFunctionList.lijst))[i].tabl ) != 0 ) {
03090         if ( T->tablepointers ) M_free(T->tablepointers,"tablepointers");
03091         if ( T->prototype ) M_free(T->prototype,"tableprototype");
03092         if ( T->mm ) M_free(T->mm,"tableminmax");
03093         if ( T->flags ) M_free(T->flags,"tableflags");
03094         if ( T->argtail ) M_free(T->argtail,"table arguments");
03095         if ( T->boomlijst ) M_free(T->boomlijst,"TableTree");
03096         for ( j = 0; j < T->buffersfill; j++ ) { /* was <= */
03097             finishcbuf(T->buffers[j]);
03098         }
03099         if ( T->spare ) {
03100             TABLES TT = T->spare;

```

```

03101         if ( TT->mm ) M_free(TT->mm,"tableminmax");
03102         if ( TT->flags ) M_free(TT->flags,"tableflags");
03103         if ( TT->tablepointers ) M_free(TT->tablepointers,"tablepointers");
03104         for ( j = 0; j < TT->buffersfill; j++ ) { /* was <= */
03105             finishcbuf(TT->buffers[j]);
03106         }
03107         if ( TT->boomlijst ) M_free(TT->boomlijst,"TableTree");
03108         M_free(TT,"table");
03109     }
03110     M_free(T,"table");
03111 }
03112 }
03113 AC.AutoFunctionList.num = AC.AutoFunctionList.numglobal;
03114
03115 CompactifyTree(AC.autonames,AUTONAMES);
03116
03117 AC.autonames->namefill = AC.autonames->globalnamefill;
03118 AC.autonames->nodefill = AC.autonames->globalnodefill;
03119 break;
03120 }
03121 }
03122
03123 /*
03124 #] ResetVariables :
03125 #[ RemoveDollars :
03126 */
03127
03128 void RemoveDollars(void)
03129 {
03130     DOLLARS d;
03131     CBUF *C = cbuf + AM.dbufnum;
03132     int numdollar = AP.DollarList.num;
03133     if ( numdollar > 0 ) {
03134         while ( numdollar > AM.gcNumDollars ) {
03135             numdollar--;
03136             d = Dollars + numdollar;
03137             if ( d->where && d->where != &(d->zero) && d->where != &(AM.dollarzero) ) {
03138                 M_free(d->where,"dollar->where"); d->where = &(d->zero); d->size = 0;
03139             }
03140             AC.dollarnames->namenode[d->node].type = CDELETE;
03141         }
03142         AP.DollarList.num = AM.gcNumDollars;
03143         CompactifyTree(AC.dollarnames,DOLLARNAMES);
03144
03145         C->numrhs = C->mnumrhs;
03146         C->numlhs = C->mnumlhs;
03147     }
03148 }
03149
03150 /*
03151 #] RemoveDollars :
03152 #[ Globalize :
03153 */
03154
03155 void Globalize(int par)
03156 {
03157     int i, j;
03158     WORD *tp;
03159     if ( par == 1 ) {
03160         AC.SymbolList.numclear = AC.SymbolList.num;
03161         AC.VectorList.numclear = AC.VectorList.num;
03162         AC.IndexList.numclear = AC.IndexList.num;
03163         AC.FunctionList.numclear = AC.FunctionList.num;
03164         AC.SetList.numclear = AC.SetList.num;
03165         AC.DubiousList.numclear = AC.DubiousList.num;
03166         AC.SetElementList.numclear = AC.SetElementList.num;
03167         AC.varnames->clearnamefill = AC.varnames->namefill;
03168         AC.varnames->clearnodefill = AC.varnames->nodefill;
03169
03170         AC.AutoSymbolList.numclear = AC.AutoSymbolList.num;
03171         AC.AutoVectorList.numclear = AC.AutoVectorList.num;
03172         AC.AutoIndexList.numclear = AC.AutoIndexList.num;
03173         AC.AutoFunctionList.numclear = AC.AutoFunctionList.num;
03174         AC.autonames->clearnamefill = AC.autonames->namefill;
03175         AC.autonames->clearnodefill = AC.autonames->nodefill;
03176     }
03177     /* for ( i = AC.FunctionList.numglobal; i < AC.FunctionList.num; i++ ) { */
03178     for ( i = MAXBUILTINFUNCTION-FUNCTION; i < AC.FunctionList.num; i++ ) {
03179         /*
03180             We need here not only the not-yet-global functions. The already
03181             global ones may have obtained extra elements.
03182         */
03183         if ( functions[i].tabl ) {
03184             TABLES T = functions[i].tabl;
03185             if ( T->sparse ) {
03186                 T->mdefined = T->totind;
03187                 for ( j = 0, tp = T->tablepointers; j < T->totind; j++ ) {

```

```

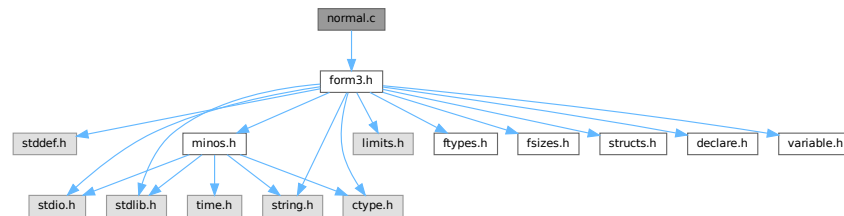
03188         tp += ABS(T->numind);
03189 #if TABLEEXTENSION == 2
03190     tp[1] = tp[0];
03191 #else
03192     tp[2] = tp[0]; tp[3] = tp[1]; tp[5] = tp[4] & (~ELEMENTUSED);
03193 #endif
03194     tp += TABLEEXTENSION;
03195 }
03196 if ( T->spare ) {
03197     TABLES TT = T->spare;
03198     TT->mdefined = TT->totind;
03199     for ( j = 0, tp = TT->tablepointers; j < TT->totind; j++ ) {
03200         tp += ABS(TT->numind);
03201 #if TABLEEXTENSION == 2
03202         tp[1] = tp[0];
03203 #else
03204         tp[2] = tp[0]; tp[3] = tp[1]; tp[5] = tp[4] & (~ELEMENTUSED);
03205 #endif
03206         tp += TABLEEXTENSION;
03207     }
03208     cbuf[TT->bufnum].mnumlhs = cbuf[TT->bufnum].numlhs;
03209     cbuf[TT->bufnum].mnumrhs = cbuf[TT->bufnum].numrhs;
03210 }
03211 }
03212 else {
03213     T->mdefined = T->defined;
03214     for ( j = 0, tp = T->tablepointers; j < T->totind; j++ ) {
03215 #if TABLEEXTENSION == 2
03216         tp[1] = tp[0];
03217 #else
03218         tp[2] = tp[0]; tp[3] = tp[1]; tp[5] = tp[4] & (~ELEMENTUSED);
03219 #endif
03220     }
03221 }
03222 cbuf[T->bufnum].mnumlhs = cbuf[T->bufnum].numlhs;
03223 cbuf[T->bufnum].mnumrhs = cbuf[T->bufnum].numrhs;
03224 }
03225 }
03226 for ( i = AC.AutoFunctionList.numglobal; i < AC.AutoFunctionList.num; i++ ) {
03227     if ( ((FUNCTIONS) (AC.AutoFunctionList.lijst))[i].tabl )
03228         ((FUNCTIONS) (AC.AutoFunctionList.lijst))[i].tabl->mdefined =
03229             ((FUNCTIONS) (AC.AutoFunctionList.lijst))[i].tabl->defined;
03230 }
03231 AC.SymbolList.numglobal = AC.SymbolList.num;
03232 AC.VectorList.numglobal = AC.VectorList.num;
03233 AC.IndexList.numglobal = AC.IndexList.num;
03234 AC.FunctionList.numglobal = AC.FunctionList.num;
03235 AC.SetList.numglobal = AC.SetList.num;
03236 AC.DubiousList.numglobal = AC.DubiousList.num;
03237 AC.SetElementList.numglobal = AC.SetElementList.num;
03238 AC.varnames->globalnamefill = AC.varnames->namefill;
03239 AC.varnames->globalnodefill = AC.varnames->nodefill;
03240
03241 AC.AutoSymbolList.numglobal = AC.AutoSymbolList.num;
03242 AC.AutoVectorList.numglobal = AC.AutoVectorList.num;
03243 AC.AutoIndexList.numglobal = AC.AutoIndexList.num;
03244 AC.AutoFunctionList.numglobal = AC.AutoFunctionList.num;
03245 AC.autonames->globalnamefill = AC.autonames->namefill;
03246 AC.autonames->globalnodefill = AC.autonames->nodefill;
03247 }
03248
03249 /*
03250 #] Globalize :
03251 #[ TestName :
03252 */
03253
03254 int TestName(UBYTE *name)
03255 {
03256     if ( *name == '[' ) {
03257         while ( *name ) name++;
03258         if ( name[-1] == ']' ) return(0);
03259         return(-1);
03260     }
03261     while ( *name ) {
03262         if ( *name == '_' ) return(-1);
03263         name++;
03264     }
03265     return(0);
03266 }
03267
03268 /*
03269 #] TestName :
03270 */

```

4.86 normal.c File Reference

```
#include "form3.h"
```

Include dependency graph for normal.c:



Macros

- `#define` [MAXNUMBEROFNONCOMTERMS](#) 2

Functions

- WORD [CompareFunctions](#) (WORD *fleft, WORD *fright)
- WORD [Commute](#) (WORD *fleft, WORD *fright)
- WORD [Normalize](#) (PHEAD WORD *term)
- WORD [ExtraSymbol](#) (WORD sym, WORD pow, WORD nsym, WORD *ppsym, WORD *ncoef)
- WORD [DoTheta](#) (PHEAD WORD *t)
- WORD [DoDelta](#) (WORD *t)
- void [DoRevert](#) (WORD *fun, WORD *tmp)
- WORD [DetCommu](#) (WORD *terms)
- WORD [DoesCommu](#) (WORD *term)
- int [TreatPolyRatFun](#) (PHEAD WORD *prf)
- void [DropCoefficient](#) (PHEAD WORD *term)
- void [DropSymbols](#) (PHEAD WORD *term)
- int [SymbolNormalize](#) (WORD *term)
- int [TestFunFlag](#) (PHEAD WORD *tfun)
- WORD [BracketNormalize](#) (PHEAD WORD *term)

4.86.1 Detailed Description

Mainly the routine `Normalize`. This routine brings terms to standard FORM. Currently it has one serious drawback. Its buffers are all in the stack. This means these buffers have a fixed size (`NORMSIZE`). In the past this has caused problems and `NORMSIZE` had to be increased.

It is not clear whether `Normalize` can be called recursively.

Definition in file [normal.c](#).

4.86.2 Macro Definition Documentation

4.86.2.1 MAXNUMBEROFNONCOMTERMS

```
#define MAXNUMBEROFNONCOMTERMS 2
```

Definition at line [4508](#) of file [normal.c](#).

4.86.3 Function Documentation

4.86.3.1 CompareFunctions()

```
WORD CompareFunctions (  
    WORD * fleft,  
    WORD * fright )
```

Definition at line [54](#) of file [normal.c](#).

4.86.3.2 Commute()

```
WORD Commute (  
    WORD * fleft,  
    WORD * fright )
```

Definition at line [118](#) of file [normal.c](#).

4.86.3.3 Normalize()

```
WORD Normalize (  
    PHEAD WORD * term )
```

Definition at line [193](#) of file [normal.c](#).

4.86.3.4 ExtraSymbol()

```
WORD ExtraSymbol (  
    WORD sym,  
    WORD pow,  
    WORD nsym,  
    WORD * ppsym,  
    WORD * ncoef )
```

Definition at line [4196](#) of file [normal.c](#).

4.86.3.5 DoTheta()

```
WORD DoTheta (  
    PHEAD WORD * t )
```

Definition at line [4258](#) of file [normal.c](#).

4.86.3.6 DoDelta()

```
WORD DoDelta (
    WORD * t )
```

Definition at line 4355 of file [normal.c](#).

4.86.3.7 DoRevert()

```
void DoRevert (
    WORD * fun,
    WORD * tmp )
```

Definition at line 4422 of file [normal.c](#).

4.86.3.8 DetCommu()

```
WORD DetCommu (
    WORD * terms )
```

Definition at line 4510 of file [normal.c](#).

4.86.3.9 DoesCommu()

```
WORD DoesCommu (
    WORD * term )
```

Definition at line 4569 of file [normal.c](#).

4.86.3.10 TreatPolyRatFun()

```
int TreatPolyRatFun (
    PHEAD WORD * prf )
```

Definition at line 4960 of file [normal.c](#).

4.86.3.11 DropCoefficient()

```
void DropCoefficient (
    PHEAD WORD * term )
```

Definition at line 5096 of file [normal.c](#).

4.86.3.12 DropSymbols()

```
void DropSymbols (
    PHEAD WORD * term )
```

Definition at line 5114 of file [normal.c](#).

4.86.3.13 SymbolNormalize()

```
int SymbolNormalize (
    WORD * term )
```

Routine normalizes terms that contain only symbols. Regular minimum and maximum properties are ignored.

We check whether there are negative powers in the output. This is not allowed.

Definition at line 5144 of file [normal.c](#).

Referenced by [InFunction\(\)](#), and [poly_sort\(\)](#).

4.86.3.14 TestFunFlag()

```
int TestFunFlag (
    PHEAD WORD * tfun )
```

Definition at line 5240 of file [normal.c](#).

4.86.3.15 BracketNormalize()

```
WORD BracketNormalize (
    PHEAD WORD * term )
```

Definition at line 5271 of file [normal.c](#).

4.87 normal.c

[Go to the documentation of this file.](#)

```
00001
00010 /* #[] License : */
00011 /*
00012 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00013 * When using this file you are requested to refer to the publication
00014 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00015 * This is considered a matter of courtesy as the development was paid
00016 * for by FOM the Dutch physics granting agency and we would like to
00017 * be able to track its scientific use to convince FOM of its value
00018 * for the community.
00019 *
00020 * This file is part of FORM.
00021 *
00022 * FORM is free software: you can redistribute it and/or modify it under the
00023 * terms of the GNU General Public License as published by the Free Software
00024 * Foundation, either version 3 of the License, or (at your option) any later
00025 * version.
00026 *
00027 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00028 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00029 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00030 * details.
00031 *
00032 * You should have received a copy of the GNU General Public License along
00033 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00034 */
00035 /* #[] License : */
00036 /*
00037 #[] Includes : normal.c
00038 */
00039
00040 #include "form3.h"
00041 #ifdef WITHFLOAT
```



```

00042 #include <gmp.h>
00043
00044 int PackFloat(WORD *,mpf_t);
00045 int UnpackFloat(mpf_t, WORD *);
00046 void RatToFloat(mpf_t result, UWORD *format, int ratsize);
00047 #endif
00048 /*
00049     #] Includes :
00050     #[ Normalize :
00051         #[ CompareFunctions :
00052 */
00053
00054 WORD CompareFunctions(WORD *fleft,WORD *fright)
00055 {
00056     WORD k, kk;
00057     if ( AC.properorderflag ) {
00058         if ( ( *fleft >= (FUNCTION+WILDOFFSET)
00059             && functions[*fleft-FUNCTION-WILDOFFSET].spec >= TENSORFUNCTION )
00060             || ( *fleft >= FUNCTION && *fleft < (FUNCTION + WILDOFFSET)
00061             && functions[*fleft-FUNCTION].spec >= TENSORFUNCTION ) ) {}
00062         else {
00063             WORD *s1, *s2, *ss1, *ss2;
00064             s1 = fleft+FUNHEAD; s2 = fright+FUNHEAD;
00065             ss1 = fleft + fleft[1]; ss2 = fright + fright[1];
00066             while ( s1 < ss1 && s2 < ss2 ) {
00067                 k = CompArg(s1,s2);
00068                 if ( k > 0 ) return(1);
00069                 if ( k < 0 ) return(0);
00070                 NEXTARG(s1)
00071                 NEXTARG(s2)
00072             }
00073             if ( s1 < ss1 ) return(1);
00074             return(0);
00075         }
00076         k = fleft[1] - FUNHEAD;
00077         kk = fright[1] - FUNHEAD;
00078         fleft += FUNHEAD;
00079         fright += FUNHEAD;
00080         while ( k > 0 && kk > 0 ) {
00081             if ( *fleft < *fright ) return(0);
00082             else if ( *fleft++ > *fright++ ) return(1);
00083             k--; kk--;
00084         }
00085         if ( k > 0 ) return(1);
00086         return(0);
00087     }
00088     else {
00089         k = fleft[1] - FUNHEAD;
00090         kk = fright[1] - FUNHEAD;
00091         fleft += FUNHEAD;
00092         fright += FUNHEAD;
00093         while ( k > 0 && kk > 0 ) {
00094             if ( *fleft < *fright ) return(0);
00095             else if ( *fleft++ > *fright++ ) return(1);
00096             k--; kk--;
00097         }
00098         if ( k > 0 ) return(1);
00099         return(0);
00100     }
00101 }
00102
00103 /*
00104     #] CompareFunctions :
00105     #[ Commute :
00106
00107     This function gets two adjacent function pointers and decides
00108     whether these two functions should be exchanged to obtain a
00109     natural ordering.
00110
00111     Currently there is only an ordering of gamma matrices belonging
00112     to different spin lines.
00113
00114     Note that we skip for now the cases of (F)^(3/2) or 1/F and a few more
00115     of such funny functions.
00116 */
00117
00118 WORD Commute(WORD *fleft, WORD *fright)
00119 {
00120     WORD fun1, fun2;
00121     if ( *fleft == DOLLAREXPRESSION || *fright == DOLLAREXPRESSION ) return(0);
00122     fun1 = ABS(*fleft); fun2 = ABS(*fright);
00123     if ( *fleft >= GAMMA && *fleft <= GAMMASEVEN
00124         && *fright >= GAMMA && *fright <= GAMMASEVEN ) {
00125         if ( fleft[FUNHEAD] < AM.OffsetIndex && fleft[FUNHEAD] > fright[FUNHEAD] )
00126             return(1);
00127         return(0);
00128     }

```

```

00129     if ( fun1 >= WILDOFFSET ) fun1 -= WILDOFFSET;
00130     if ( fun2 >= WILDOFFSET ) fun2 -= WILDOFFSET;
00131     if ( ( ( functions[fun1-FUNCTION].flags & COULDCOMMUTE ) == 0 )
00132         || ( ( functions[fun2-FUNCTION].flags & COULDCOMMUTE ) == 0 ) ) return(0);
00133 /*
00134     if other conditions will come here, keep in mind that if *fleft < 0
00135     or *fright < 0 they are arguments in the exponent function as in f^(3/2)
00136 */
00137     if ( AC.CommuteInSet == 0 ) return(0);
00138 /*
00139     The code for CompareFunctions can be stolen from the commuting case.
00140
00141     We need the syntax:
00142         Commute Fun1,Fun2,...,Fun'n';
00143     For this Fun1,...,Fun'n' need to be noncommuting functions.
00144     These functions will commute with all members of the group.
00145     In the AC.paircommute buffer the representation is
00146         'n'+1,element1,...,element'n','m'+1,element1,...,element'm',0
00147     A function can belong to more than one group.
00148     If a function commutes with itself, it is most efficient to make a separate
00149     group of two elements for it as in
00150         Commute T,T;    -> 3,T,T
00151 */
00152     if ( fun1 >= fun2 ) {
00153         WORD *group = AC.CommuteInSet, *g1, *g2, *g3;
00154         while ( *group > 0 ) {
00155             g3 = group + *group;
00156             g1 = group+1;
00157             while ( g1 < g3 ) {
00158                 if ( *g1 == fun1 || ( fun1 <= GAMMASEVEN && fun1 >= GAMMA
00159                     && *g1 <= GAMMASEVEN && *g1 >= GAMMA ) ) {
00160                     g2 = group+1;
00161                     while ( g2 < g3 ) {
00162                         if ( g1 != g2 && ( *g2 == fun2 ||
00163                             ( fun2 <= GAMMASEVEN && fun2 >= GAMMA
00164                             && *g2 <= GAMMASEVEN && *g2 >= GAMMA ) ) ) {
00165                             if ( fun1 != fun2 ) return(1);
00166                             if ( *fleft < 0 ) return(0);
00167                             if ( *fright < 0 ) return(1);
00168                             return(CompareFunctions(fleft,fright));
00169                         }
00170                         g2++;
00171                     }
00172                     break;
00173                 }
00174                 g1++;
00175             }
00176             group = g3;
00177         }
00178     }
00179     return(0);
00180 }
00181
00182 /*
00183     #] Commute :
00184     #[ Normalize :
00185
00186     This is the big normalization routine. It has a great need
00187     to be economical.
00188     There is a fixed limit to the number of objects coming in.
00189     Something should be done about it.
00190
00191 */
00192
00193 WORD Normalize(PHEAD WORD *term)
00194 {
00195 /*
00196     #[ Declarations :
00197 */
00198     GETBIDENTITY
00199     WORD *t, *m, *r, i, j, k, l, nsym, *ss, *tt, *u;
00200     WORD shortnum, stype;
00201     WORD *stop, *to = 0, *from = 0;
00202 /*
00203     The next variables could be better off in the AT.WorkSpace (?)
00204     Now they make stackallocations rather bothersome.
00205 */
00206     WORD psym[7*NORMSIZE],*ppsym;
00207     WORD pvec[NORMSIZE],*ppvec,nvec;
00208     WORD pdot[3*NORMSIZE],*ppdot,ndot;
00209     WORD pdel[2*NORMSIZE],*ppdel,ndel;
00210     WORD pind[NORMSIZE],nind;
00211     WORD *peps[NORMSIZE/3],neps;
00212     WORD *pden[NORMSIZE/3],nden;
00213     WORD *pcom[NORMSIZE],ncom;
00214     WORD *pnco[NORMSIZE],nnco;
00215     WORD *pcon[2*NORMSIZE],ncon;          /* Pointer to contractable indices */

```

```

00216     WORD *n_coef, ncoef;                      /* Accumulator for the coefficient */
00217     WORD *n_llnum, *lnum, nnum;
00218     WORD *termout, oldtoprhs = 0, subtype;
00219     WORD ReplaceType, ReplaceVeto = 0, didcontr, regval = 0;
00220     WORD *ReplaceSub;
00221     WORD *fillsetexp;
00222     CBUF *C = cbuf+AT.ebufnum;
00223     WORD *ANsc = 0, *ANsm = 0, *ANsr = 0, PolyFunMode;
00224 #ifdef WITHFLOAT
00225     WORD withfloat = 0;
00226     WORD *firstfloat = 0;
00227 #endif
00228     LONG oldcpointer = 0, x;
00229     n_coef = TermMalloc("NormCoef");
00230     n_llnum = TermMalloc("n_llnum");
00231     lnum = n_llnum+1;
00232 /*
00233     int termflag;
00234 */
00235 /*
00236     #] Declarations :
00237     #[ Setup :
00238     PrintTerm(term, "Normalize");
00239 */
00240
00241 Restart:
00242     didcontr = 0;
00243     ReplaceType = -1;
00244     t = term;
00245     if ( !*t ) { TermFree(n_coef, "NormCoef"); TermFree(n_llnum, "n_llnum"); return(regval); }
00246     r = t + *t;
00247     ncoef = r[-1];
00248     i = ABS(ncoef);
00249     r -= i;
00250     m = r;
00251     t = n_coef;
00252     NCOPY(t, r, i);
00253     termout = AT.WorkPointer;
00254     AT.WorkPointer = (WORD *)(((UBYTE *) (AT.WorkPointer)) + AM.MaxTer);
00255     fillsetexp = termout+1;
00256     AN.PolyNormFlag = 0; PolyFunMode = AN.PolyFunTodo;
00257 /*
00258     termflag = 0;
00259 */
00260 /*
00261     #] Setup :
00262     #[ First scan :
00263 */
00264     nsym = nvec = ndot = ndel = neps = nden =
00265     nind = ncom = nnco = ncon = 0;
00266     ppsym = psym;
00267     ppvec = pvec;
00268     ppdot = pdot;
00269     ppdel = pdel;
00270     t = term + 1;
00271 conscan:;
00272     if ( t < m ) do {
00273         r = t + t[1];
00274         switch ( *t ) {
00275             case SYMBOL :
00276                 t += 2;
00277                 from = m;
00278                 do {
00279                     if ( t[1] == 0 ) {
00280 /*
00281                         if ( *t == 0 || *t == MAXPOWER ) goto NormZZ; */
00282                         t += 2;
00283                         goto NextSymbol;
00284                     }
00285 /*
00286                     if ( *t <= DENOMINATORSYMBOL && *t >= COEFFSYMBOL ) {
00287                         if ( AN.NoScrat2 == 0 ) {
00288                             AN.NoScrat2 = (UWORD *)Mallco1((AM.MaxTal+2)*sizeof(UWORD), "Normalize");
00289                         }
00290                         if ( AN.cTerm ) m = AN.cTerm;
00291                         else m = term;
00292                         m += *m;
00293                         ncoef = REDLENG(ncoef);
00294                         if ( *t == COEFFSYMBOL ) {
00295                             i = t[1];
00296                             nnum = REDLENG(m[-1]);
00297                             m -= ABS(m[-1]);
00298                             if ( i > 0 ) {
00299                                 while ( i > 0 ) {
00300                                     if ( MulRat(BHEAD (UWORD *)n_coef, ncoef, (UWORD *)m, nnum,
00301                                         (UWORD *)n_coef, &ncoef) ) goto FromNorm;
00302                                     i--;

```

```

00303     }
00304     }
00305     else if ( i < 0 ) {
00306         while ( i < 0 ) {
00307             if ( DivRat (BHEAD (UWORD *)n_coef,ncoef, (UWORD *)m,nnum,
00308 (UWORD *)n_coef,&ncoef) ) goto FromNorm;
00309             i++;
00310         }
00311     }
00312 }
00313 else {
00314     i = m[-1];
00315     nnum = (ABS(i)-1)/2;
00316     if ( *t == NUMERATORSYMBOL ) { m -= nnum + 1; }
00317     else { m--; }
00318     while ( *m == 0 && nnum > 1 ) { m--; nnum--; }
00319     m -= nnum;
00320     if ( i < 0 && *t == NUMERATORSYMBOL ) nnum = -nnum;
00321     i = t[1];
00322     if ( i > 0 ) {
00323         while ( i > 0 ) {
00324             if ( Mully(BHEAD (UWORD *)n_coef,&ncoef, (UWORD *)m,nnum) )
00325                 goto FromNorm;
00326             i--;
00327         }
00328     }
00329     else if ( i < 0 ) {
00330         while ( i < 0 ) {
00331             if ( Divvy(BHEAD (UWORD *)n_coef,&ncoef, (UWORD *)m,nnum) )
00332                 goto FromNorm;
00333             i++;
00334         }
00335     }
00336 }
00337 ncoef = INCLENG(ncoef);
00338 t += 2;
00339 goto NextSymbol;
00340 }
00341 else if ( *t == DIMENSIONSYMBOL ) {
00342     if ( AN.cTerm ) m = AN.cTerm;
00343     else m = term;
00344     k = DimensionTerm(m);
00345     if ( k == 0 ) goto NormZero;
00346     if ( k == MAXPOSITIVE ) {
00347         MLOCK(ErrorMessageLock);
00348         MesPrint("Dimension_ is undefined in term %t");
00349         MUNLOCK(ErrorMessageLock);
00350         goto NormMin;
00351     }
00352     if ( k == -MAXPOSITIVE ) {
00353         MLOCK(ErrorMessageLock);
00354         MesPrint("Dimension_ out of range in term %t");
00355         MUNLOCK(ErrorMessageLock);
00356         goto NormMin;
00357     }
00358     if ( k > 0 ) { *((UWORD *)lnum) = k; nnum = 1; }
00359     else { *((UWORD *)lnum) = -k; nnum = -1; }
00360     ncoef = REDLENG(ncoef);
00361     if ( Mully(BHEAD (UWORD *)n_coef,&ncoef, (UWORD *)lnum,nnum) ) goto FromNorm;
00362     ncoef = INCLENG(ncoef);
00363     t += 2;
00364     goto NextSymbol;
00365 }
00366 if ( ( *t >= MAXPOWER && *t < 2*MAXPOWER )
00367 || ( *t < -MAXPOWER && *t > -2*MAXPOWER ) ) {
00368 /*
00369 #[ TO SNUMBER :
00370 */
00371     if ( *t < 0 ) {
00372         *t += MAXPOWER;
00373         *t = -*t;
00374         if ( t[1] & 1 ) ncoef = -ncoef;
00375     }
00376     else if ( *t == MAXPOWER ) {
00377         if ( t[1] > 0 ) goto NormZero;
00378         goto NormInf;
00379     }
00380     else {
00381         *t -= MAXPOWER;
00382     }
00383     lnum[0] = *t;
00384     nnum = 1;
00385     if ( t[1] && RaisPow(BHEAD (UWORD *)lnum,&nnum, (UWORD) (ABS(t[1])) ) )
00386         goto FromNorm;
00387     ncoef = REDLENG(ncoef);
00388     if ( t[1] < 0 ) {
00389         if ( Divvy(BHEAD (UWORD *)n_coef,&ncoef, (UWORD *)lnum,nnum) )

```

```

00390             goto FromNorm;
00391         }
00392     else if ( t[1] > 0 ) {
00393         if ( Mully(BHEAD (UWORD *)n_coef,&ncoef, (UWORD *)lnum,nnum) )
00394             goto FromNorm;
00395     }
00396     ncoef = INCLENG(ncoef);
00397 /*
00398     #] TO SNUMBER :
00399 */
00400     t += 2;
00401     goto NextSymbol;
00402 }
00403 if ( ( *t <= NumSymbols && *t > -MAXPOWER )
00404     && ( symbols[*t].complex & VARTYPEROOTOFUNITY ) == VARTYPEROOTOFUNITY ) {
00405     if ( t[1] <= 2*MAXPOWER && t[1] >= -2*MAXPOWER ) {
00406         t[1] %= symbols[*t].maxpower;
00407         if ( t[1] < 0 ) t[1] += symbols[*t].maxpower;
00408         if ( ( symbols[*t].complex & VARTYPEMINUS ) == VARTYPEMINUS ) {
00409             if ( ( ( symbols[*t].maxpower & 1 ) == 0 ) &&
00410                 ( t[1] >= symbols[*t].maxpower/2 ) ) {
00411                 t[1] -= symbols[*t].maxpower/2; ncoef = -ncoef;
00412             }
00413         }
00414         if ( t[1] == 0 ) { t += 2; goto NextSymbol; }
00415     }
00416 }
00417 i = nsym;
00418 m = ppsym;
00419 if ( i > 0 ) do {
00420     m -= 2;
00421     if ( *t == *m ) {
00422         t++; m++;
00423         if ( *t > 2*MAXPOWER || *t < -2*MAXPOWER
00424             || *m > 2*MAXPOWER || *m < -2*MAXPOWER ) {
00425             MLOCK(ErrorMessageLock);
00426             MesPrint("Illegal wildcard power combination.");
00427             MUNLOCK(ErrorMessageLock);
00428             goto NormMin;
00429         }
00430         *m += *t;
00431         if ( ( t[-1] <= NumSymbols && t[-1] > -MAXPOWER )
00432             && ( symbols[t[-1]].complex & VARTYPEROOTOFUNITY ) == VARTYPEROOTOFUNITY
00433         ) {
00434             *m %= symbols[t[-1]].maxpower;
00435             if ( *m < 0 ) *m += symbols[t[-1]].maxpower;
00436             if ( ( symbols[t[-1]].complex & VARTYPEMINUS ) == VARTYPEMINUS ) {
00437                 if ( ( ( symbols[t[-1]].maxpower & 1 ) == 0 ) &&
00438                     ( *m >= symbols[t[-1]].maxpower/2 ) ) {
00439                     *m -= symbols[t[-1]].maxpower/2; ncoef = -ncoef;
00440                 }
00441             }
00442             if ( *m >= 2*MAXPOWER || *m <= -2*MAXPOWER ) {
00443                 MLOCK(ErrorMessageLock);
00444                 MesPrint("Power overflow during normalization");
00445                 MUNLOCK(ErrorMessageLock);
00446                 goto NormMin;
00447             }
00448             if ( !*m ) {
00449                 m--;
00450                 while ( i < nsym )
00451                     { *m = m[2]; m++; *m = m[2]; m++; i++; }
00452                 ppsym -= 2;
00453                 nsym--;
00454             }
00455             t++;
00456             goto NextSymbol;
00457         }
00458     } while ( *t < *m && --i > 0 );
00459     m = ppsym;
00460     while ( i < nsym )
00461         { m--; m[2] = *m; m--; m[2] = *m; i++; }
00462     *m++ = *t++;
00463     *m = *t++;
00464     ppsym += 2;
00465     nsym++;
00466 NextSymbol:;
00467     } while ( t < r );
00468     m = from;
00469     break;
00470 case VECTOR :
00471     t += 2;
00472     do {
00473         if ( t[0] == AM.vectorzero ) goto NormZero;
00474         if ( t[1] == FUNNYVEC ) {
00475             pind[nind++] = *t;

```

```

00476         t += 2;
00477     }
00478     else if ( t[1] < 0 ) {
00479         if ( *t == NOINDEX && t[1] == NOINDEX ) t += 2;
00480         else {
00481             if ( t[1] == AM.vectorzero ) goto NormZero;
00482             *ppdot++ = *t++; *ppdot++ = *t++; *ppdot++ = 1; ndot++;
00483         }
00484     }
00485     else { *ppvec++ = *t++; *ppvec++ = *t++; nvec += 2; }
00486     } while ( t < r );
00487     break;
00488 case DOTPRODUCT :
00489     t += 2;
00490     do {
00491         if ( t[2] == 0 ) t += 3;
00492         else if ( ndot > 0 && t[0] == ppdot[-3]
00493             && t[1] == ppdot[-2] ) {
00494             ppdot[-1] += t[2];
00495             t += 3;
00496             if ( ppdot[-1] == 0 ) { ppdot -= 3; ndot--; }
00497         }
00498         else if ( t[0] == AM.vectorzero || t[1] == AM.vectorzero ) {
00499             if ( t[2] > 0 ) goto NormZero;
00500             goto NormInf;
00501         }
00502         else {
00503             *ppdot++ = *t++; *ppdot++ = *t++;
00504             *ppdot++ = *t++; ndot++;
00505         }
00506     } while ( t < r );
00507     break;
00508 case HAAKJE :
00509     break;
00510 case SETSET:
00511     if ( WildFill(BHEAD termout,term,AT.dummysubexp) < 0 ) goto FromNorm;
00512     i = *termout;
00513     t = termout; m = term;
00514     NCOPY(m,t,i);
00515     goto Restart;
00516 case DOLLAREXPRESSION :
00517 /*
00518     We have DOLLAREXPRESSION,4,number,power
00519     Replace by SUBEXPRESSION and exit elegantly to let
00520     TestSub pick it up. Of course look for special cases first.
00521     Note that we have a special compiler buffer for the values.
00522 */
00523     if ( AR.Eside != LHSIDE ) {
00524         DOLLARS d = Dollars + t[2];
00525 #ifdef WITHPTHREADS
00526         int nummodopt, ptype = -1;
00527         if ( AS.MultiThreaded ) {
00528             for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
00529                 if ( t[2] == ModOptdollars[nummodopt].number ) break;
00530             }
00531             if ( nummodopt < NumModOptdollars ) {
00532                 ptype = ModOptdollars[nummodopt].type;
00533                 if ( ptype == MODLOCAL ) {
00534                     d = ModOptdollars[nummodopt].dstruct+AT.identity;
00535                 }
00536                 else {
00537                     LOCK(d->pthreadlockread);
00538                 }
00539             }
00540         }
00541 #endif
00542         if ( d->type == DOLZERO ) {
00543 #ifdef WITHPTHREADS
00544             if ( ptype > 0 && ptype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
00545 #endif
00546             if ( t[3] == 0 ) goto NormZZ;
00547             if ( t[3] < 0 ) goto NormInf;
00548             goto NormZero;
00549         }
00550         else if ( d->type == DOLNUMBER ) {
00551             nnum = d->where[0];
00552             if ( nnum > 0 ) {
00553                 nnum = d->where[nnum-1];
00554                 if ( nnum < 0 ) { ncoef = -ncoef; nnum = -nnum; }
00555                 nnum = (nnum-1)/2;
00556                 for ( i = 1; i <= nnum; i++ ) lnum[i-1] = d->where[i];
00557             }
00558             if ( nnum == 0 || ( nnum == 1 && lnum[0] == 0 ) ) {
00559 #ifdef WITHPTHREADS
00560                 if ( ptype > 0 && ptype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
00561 #endif
00562                 if ( t[3] < 0 ) goto NormInf;

```

```

00563         else if ( t[3] == 0 ) goto NormZZ;
00564         goto NormZero;
00565     }
00566     if ( t[3] && RaisPow(BHEAD (UWORD *)lnum,&nnum, (UWORD) (ABS(t[3]))) ) goto
FromNorm;
00567     ncoef = REDLENG(ncoef);
00568     if ( t[3] < 0 ) {
00569         if ( Divvy(BHEAD (UWORD *)n_coef,&ncoef, (UWORD *)lnum,nnum) ) {
00570 #ifdef WITHPTHREADS
00571             if ( ptype > 0 && ptype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
00572 #endif
00573             goto FromNorm;
00574         }
00575     }
00576     else if ( t[3] > 0 ) {
00577         if ( Mully(BHEAD (UWORD *)n_coef,&ncoef, (UWORD *)lnum,nnum) ) {
00578 #ifdef WITHPTHREADS
00579             if ( ptype > 0 && ptype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
00580 #endif
00581             goto FromNorm;
00582         }
00583     }
00584     ncoef = INCLENG(ncoef);
00585 }
00586 else if ( d->type == DOLINDEX ) {
00587     if ( d->index == 0 ) {
00588 #ifdef WITHPTHREADS
00589         if ( ptype > 0 && ptype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
00590 #endif
00591         goto NormZero;
00592     }
00593     if ( d->index != NOINDEX ) pind[nind++] = d->index;
00594 }
00595 else if ( d->type == DOLTERMS ) {
00596     if ( t[3] >= MAXPOWER || t[3] <= -MAXPOWER ) {
00597         if ( d->where[0] == 0 ) goto NormZero;
00598         if ( d->where[d->where[0]] != 0 ) {
00599 I11DollarExp:
00600             MLOCK(ErrorMessageLock);
00601             MesPrint("!!!Illegal $ expansion with wildcard power!!!");
00602             MUNLOCK(ErrorMessageLock);
00603             goto FromNorm;
00604         }
00605     /*
00606     At this point we should only admit symbols and dotproducts
00607     We expand the dollar directly and do not send it back.
00608     */
00609     {
00610         WORD *td, *tdstop, dj;
00611         td = d->where+1;
00612         tdstop = d->where+d->where[0];
00613         if ( tdstop[-1] != 3 || tdstop[-2] != 1
00614             || tdstop[-3] != 1 ) goto I11DollarExp;
00615         tdstop -= 3;
00616         if ( td >= tdstop ) goto I11DollarExp;
00617         while ( td < tdstop ) {
00618             if ( *td == SYMBOL ) {
00619                 for ( dj = 2; dj < td[1]; dj += 2 ) {
00620                     if ( td[dj+1] == 1 ) {
00621                         *ppsym++ = td[dj];
00622                         *ppsym++ = t[3];
00623                         nsym++;
00624                     }
00625                     else if ( td[dj+1] == -1 ) {
00626                         *ppsym++ = td[dj];
00627                         *ppsym++ = -t[3];
00628                         nsym++;
00629                     }
00630                     else goto I11DollarExp;
00631                 }
00632             }
00633             else if ( *td == DOTPRODUCT ) {
00634                 for ( dj = 2; dj < td[1]; dj += 3 ) {
00635                     if ( td[dj+2] == 1 ) {
00636                         *ppdot++ = td[dj];
00637                         *ppdot++ = td[dj+1];
00638                         *ppdot++ = t[3];
00639                         ndot++;
00640                     }
00641                     else if ( td[dj+2] == -1 ) {
00642                         *ppdot++ = td[dj];
00643                         *ppdot++ = td[dj+1];
00644                         *ppdot++ = -t[3];
00645                         ndot++;
00646                     }
00647                     else goto I11DollarExp;
00648                 }
00649             }
00650         }
00651     }

```

```

00649                 else goto IllDollarExp;
00650                 td += td[1];
00651             }
00652             regval = 2;
00653             break;
00654         }
00655     }
00656     t[0] = SUBEXPRESSION;
00657     t[4] = AM.dbufnum;
00658     if ( t[3] == 0 ) {
00659 #ifdef WITHPTHREADS
00660         if ( ptype > 0 && ptype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
00661 #endif
00662         break;
00663     }
00664     regval = 2;
00665     t = r;
00666     while ( t < m ) {
00667         if ( *t == DOLLAREXPRESSION ) {
00668 #ifdef WITHPTHREADS
00669             if ( ptype > 0 && ptype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
00670 #endif
00671             d = Dollars + t[2];
00672 #ifdef WITHPTHREADS
00673             if ( AS.MultiThreaded ) {
00674                 for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
00675                     if ( t[2] == ModOptdollars[nummodopt].number ) break;
00676                 }
00677                 if ( nummodopt < NumModOptdollars ) {
00678                     ptype = ModOptdollars[nummodopt].type;
00679                     if ( ptype == MODLOCAL ) {
00680                         d = ModOptdollars[nummodopt].dstruct+AT.identity;
00681                     }
00682                     else {
00683                         LOCK(d->pthreadlockread);
00684                     }
00685                 }
00686             }
00687 #endif
00688             if ( d->type == DOLTERMS ) {
00689                 t[0] = SUBEXPRESSION;
00690                 t[4] = AM.dbufnum;
00691             }
00692             t += t[1];
00693         }
00694     }
00695 #ifdef WITHPTHREADS
00696     if ( ptype > 0 && ptype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
00697 #endif
00698     goto RegEnd;
00699 }
00700 else {
00701 #ifdef WITHPTHREADS
00702     if ( ptype > 0 && ptype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
00703 #endif
00704     MLOCK(ErrorMessageLock);
00705     MesPrint("!!!This $ variation has not been implemented yet!!!");
00706     MUNLOCK(ErrorMessageLock);
00707     goto NormMin;
00708 }
00709 #ifdef WITHPTHREADS
00710     if ( ptype > 0 && ptype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
00711 #endif
00712 }
00713 else {
00714     pnco[nnco++] = t;
00715     /*
00716     The next statement should be safe as the value is used
00717     only by the compiler (ie the master).
00718     */
00719     AC.lhdollarflag = 1;
00720 }
00721 break;
00722 case DELTA :
00723     t += 2;
00724     do {
00725         if ( *t < 0 ) {
00726             if ( *t == SUMMEDIND ) {
00727                 if ( t[1] < -NMIN4SHIFT ) {
00728                     k = -t[1]-NMIN4SHIFT;
00729                     k = ExtraSymbol(k,1,nsym,ppsym,&ncoef);
00730                     nsym += k;
00731                     ppsym += (k * 2);
00732                 }
00733                 else if ( t[1] == 0 ) goto NormZero;
00734                 else {
00735                     if ( t[1] < 0 ) {

```



```

00736         lnum[0] = -t[1];
00737         nnum = -1;
00738     }
00739     else {
00740         lnum[0] = t[1];
00741         nnum = 1;
00742     }
00743     ncoef = REDLENG(ncoef);
00744     if ( Mully(BHEAD (UWORD *)n_coef,&ncoef, (UWORD *)lnum,nnum) )
00745         goto FromNorm;
00746     ncoef = INCLENG(ncoef);
00747 }
00748 t += 2;
00749 }
00750 else if ( *t == NOINDEX && t[1] == NOINDEX ) t += 2;
00751 else if ( *t == EMPTYINDEX && t[1] == EMPTYINDEX ) {
00752     *ppdel++ = *t++; *ppdel++ = *t++; ndel += 2;
00753 }
00754 else
00755     if ( t[1] < 0 ) {
00756         *ppdot++ = *t++; *ppdot++ = *t++; *ppdot++ = 1; ndot++;
00757     }
00758     else {
00759         *ppvec++ = *t++; *ppvec++ = *t++; nvec += 2;
00760     }
00761 }
00762 else {
00763     if ( t[1] < 0 ) {
00764         *ppvec++ = t[1]; *ppvec++ = *t; t+=2; nvec += 2;
00765     }
00766     else { *ppdel++ = *t++; *ppdel++ = *t++; ndel += 2; }
00767 }
00768 } while ( t < r );
00769 break;
00770 case FACTORIAL :
00771 /*
00772     (FACTORIAL,FUNHEAD+2,...,-SNUMBER,number)
00773 */
00774 if ( t[FUNHEAD] == -SNUMBER && t[1] == FUNHEAD+2
00775     && t[FUNHEAD+1] >= 0 ) {
00776     if ( Factorial(BHEAD t[FUNHEAD+1], (UWORD *)lnum,&nnum) )
00777         goto FromNorm;
00778 MulIn:
00779     ncoef = REDLENG(ncoef);
00780     if ( Mully(BHEAD (UWORD *)n_coef,&ncoef, (UWORD *)lnum,nnum) ) goto FromNorm;
00781     ncoef = INCLENG(ncoef);
00782 }
00783 else pcom[ncom++] = t;
00784 break;
00785 case BERNOULLIFUNCTION :
00786 /*
00787     (BERNOULLIFUNCTION,FUNHEAD+2,...,-SNUMBER,number)
00788 */
00789 if ( ( t[FUNHEAD] == -SNUMBER && t[FUNHEAD+1] >= 0 )
00790     && ( t[1] == FUNHEAD+2 || ( t[1] == FUNHEAD+4 &&
00791     t[FUNHEAD+2] == -SNUMBER && ABS(t[FUNHEAD+3]) == 1 ) ) ) {
00792     WORD inum, mnum;
00793     if ( Bernoulli(t[FUNHEAD+1], (UWORD *)lnum,&nnum) )
00794         goto FromNorm;
00795     if ( nnum == 0 ) goto NormZero;
00796     inum = nnum; if ( inum < 0 ) inum = -inum;
00797     inum--; inum /= 2;
00798     mnum = inum;
00799     while ( lnum[mnum-1] == 0 ) mnum--;
00800     if ( nnum < 0 ) mnum = -mnum;
00801     ncoef = REDLENG(ncoef);
00802     if ( Mully(BHEAD (UWORD *)n_coef,&ncoef, (UWORD *)lnum,mnum) ) goto FromNorm;
00803     mnum = inum;
00804     while ( lnum[inum+mnum-1] == 0 ) mnum--;
00805     if ( Divvy(BHEAD (UWORD *)n_coef,&ncoef, (UWORD *) (lnum+inum),mnum) ) goto
FromNorm;
00806     ncoef = INCLENG(ncoef);
00807     if ( t[1] == FUNHEAD+4 && t[FUNHEAD+1] == 1
00808         && t[FUNHEAD+3] == -1 ) ncoef = -ncoef;
00809 }
00810 else pcom[ncom++] = t;
00811 break;
00812 case NUMARGSFUN:
00813 /*
00814     Numerical function giving the number of arguments.
00815 */
00816 k = 0;
00817 t += FUNHEAD;
00818 while ( t < r ) {
00819     k++;
00820     NEXTARG(t);
00821 }

```

```

00822         if ( k == 0 ) goto NormZero;
00823         *((UWORD *)lnum) = k;
00824         nnum = 1;
00825         goto MulIn;
00826     case NUMFACTORS:
00827 /*
00828         Numerical function giving the number of factors in an expression.
00829 */
00830         t += FUNHEAD;
00831         if ( *t == -EXPRESSION ) {
00832             k = AS.OldNumFactors[t[1]];
00833         }
00834         else if ( *t == -DOLLAREXPRESSION ) {
00835             k = Dollars[t[1]].nfactors;
00836         }
00837         else {
00838             pcom[ncom++] = t;
00839             break;
00840         }
00841         if ( k == 0 ) goto NormZero;
00842         *((UWORD *)lnum) = k;
00843         nnum = 1;
00844         goto MulIn;
00845     case NUMTERMSFUN:
00846 /*
00847         Numerical function giving the number of terms in the single argument.
00848 */
00849         if ( t[FUNHEAD] < 0 ) {
00850             if ( t[FUNHEAD] <= -FUNCTION && t[1] == FUNHEAD+1 ) break;
00851             if ( t[FUNHEAD] > -FUNCTION && t[1] == FUNHEAD+2 ) {
00852                 if ( t[FUNHEAD] == -SNUMBER && t[FUNHEAD+1] == 0 ) goto NormZero;
00853                 break;
00854             }
00855             pcom[ncom++] = t;
00856             break;
00857         }
00858         if ( t[FUNHEAD] > 0 && t[FUNHEAD] == t[1]-FUNHEAD ) {
00859             k = 0;
00860             t += FUNHEAD+ARGHEAD;
00861             while ( t < r ) {
00862                 k++;
00863                 t += *t;
00864             }
00865             if ( k == 0 ) goto NormZero;
00866             *((UWORD *)lnum) = k;
00867             nnum = 1;
00868             goto MulIn;
00869         }
00870         else pcom[ncom++] = t;
00871         break;
00872     case COUNTFUNCTION:
00873         if ( AN.cTerm ) {
00874             k = CountFun(AN.cTerm,t);
00875         }
00876         else {
00877             k = CountFun(term,t);
00878         }
00879         if ( k == 0 ) goto NormZero;
00880         if ( k > 0 ) { *((UWORD *)lnum) = k; nnum = 1; }
00881         else { *((UWORD *)lnum) = -k; nnum = -1; }
00882         goto MulIn;
00883         break;
00884     case MAKERATIONAL:
00885         if ( t[FUNHEAD] == -SNUMBER && t[FUNHEAD+2] == -SNUMBER
00886             && t[1] == FUNHEAD+4 && t[FUNHEAD+3] > 1 ) {
00887             WORD x1[2], sgn;
00888             if ( t[FUNHEAD+1] == 0 ) goto NormZero;
00889             if ( t[FUNHEAD+1] < 0 ) { t[FUNHEAD+1] = -t[FUNHEAD+1]; sgn = -1; }
00890             else sgn = 1;
00891             if ( MakeRational(t[FUNHEAD+1],t[FUNHEAD+3],x1,x1+1) ) {
00892                 static int warnflag = 1;
00893                 if ( warnflag ) {
00894                     MesPrint("%w Warning: fraction could not be reconstructed in
00895 MakeRational_");
00896                     warnflag = 0;
00897                 }
00898                 x1[0] = t[FUNHEAD+1]; x1[1] = 1;
00899             }
00900             if ( sgn < 0 ) { t[FUNHEAD+1] = -t[FUNHEAD+1]; x1[0] = -x1[0]; }
00901             if ( x1[0] < 0 ) { sgn = -1; x1[0] = -x1[0]; }
00902             else sgn = 1;
00903             ncoef = REDLENG(ncoef);
00904             if ( MulRat(BHEAD (UWORD *)n_coef,ncoef,(UWORD *)x1,sgn,
00905 (UWORD *)n_coef,&ncoef) ) goto FromNorm;
00906             ncoef = INCLENG(ncoef);
00907         }
00908         else {

```

```

00908     WORD narg = 0, *tt, *ttstop, *arg1 = 0, *arg2 = 0;
00909     UWORD *x1, *x2, **xx;
00910     WORD nx1, nx2, nxx;
00911     ttstop = t + t[1]; tt = t+FUNHEAD;
00912     while ( tt < ttstop ) {
00913         narg++;
00914         if ( narg == 1 ) arg1 = tt;
00915         else arg2 = tt;
00916         NEXTARG(tt);
00917     }
00918     if ( narg != 2 ) goto defaultcase;
00919     if ( *arg2 == -SNUMBER && arg2[1] <= 1 ) goto defaultcase;
00920     else if ( *arg2 > 0 && ttstop[-1] < 0 ) goto defaultcase;
00921     x1 = NumberMalloc("Norm-MakeRational");
00922     if ( *arg1 == -SNUMBER ) {
00923         if ( arg1[1] == 0 ) {
00924             NumberFree(x1, "Norm-MakeRational");
00925             goto NormZero;
00926         }
00927         if ( arg1[1] < 0 ) { x1[0] = -arg1[1]; nx1 = -1; }
00928         else { x1[0] = arg1[1]; nx1 = 1; }
00929     }
00930     else if ( *arg1 > 0 ) {
00931         WORD *tc;
00932         nx1 = (ABS(arg2[-1])-1)/2;
00933         tc = arg1+ARGHEAD+1+nx1;
00934         if ( tc[0] != 1 ) {
00935             NumberFree(x1, "Norm-MakeRational");
00936             goto defaultcase;
00937         }
00938         for ( i = 1; i < nx1; i++ ) if ( tc[i] != 0 ) {
00939             NumberFree(x1, "Norm-MakeRational");
00940             goto defaultcase;
00941         }
00942         tc = arg1+ARGHEAD+1;
00943         for ( i = 0; i < nx1; i++ ) x1[i] = tc[i];
00944         if ( arg2[-1] < 0 ) nx1 = -nx1;
00945     }
00946     else {
00947         NumberFree(x1, "Norm-MakeRational");
00948         goto defaultcase;
00949     }
00950     x2 = NumberMalloc("Norm-MakeRational");
00951     if ( *arg2 == -SNUMBER ) {
00952         if ( arg2[1] <= 1 ) {
00953             NumberFree(x2, "Norm-MakeRational");
00954             NumberFree(x1, "Norm-MakeRational");
00955             goto defaultcase;
00956         }
00957         else { x2[0] = arg2[1]; nx2 = 1; }
00958     }
00959     else if ( *arg2 > 0 ) {
00960         WORD *tc;
00961         nx2 = (ttstop[-1]-1)/2;
00962         tc = arg2+ARGHEAD+1+nx2;
00963         if ( tc[0] != 1 ) {
00964             NumberFree(x2, "Norm-MakeRational");
00965             NumberFree(x1, "Norm-MakeRational");
00966             goto defaultcase;
00967         }
00968         for ( i = 1; i < nx2; i++ ) if ( tc[i] != 0 ) {
00969             NumberFree(x2, "Norm-MakeRational");
00970             NumberFree(x1, "Norm-MakeRational");
00971             goto defaultcase;
00972         }
00973         tc = arg2+ARGHEAD+1;
00974         for ( i = 0; i < nx2; i++ ) x2[i] = tc[i];
00975     }
00976     else {
00977         NumberFree(x2, "Norm-MakeRational");
00978         NumberFree(x1, "Norm-MakeRational");
00979         goto defaultcase;
00980     }
00981     if ( BigLong(x1, ABS(nx1), x2, nx2) >= 0 ) {
00982         UWORD *x3 = NumberMalloc("Norm-MakeRational");
00983         UWORD *x4 = NumberMalloc("Norm-MakeRational");
00984         WORD nx3, nx4;
00985         DivLong(x1, nx1, x2, nx2, x3, &nx3, x4, &nx4);
00986         for ( i = 0; i < ABS(nx4); i++ ) x1[i] = x4[i];
00987         nx1 = nx4;
00988         NumberFree(x4, "Norm-MakeRational");
00989         NumberFree(x3, "Norm-MakeRational");
00990     }
00991     xx = (UWORD *) (TermMalloc("Norm-MakeRational"));
00992     if ( MakeLongRational(BHEAD x1, nx1, x2, nx2, xx, &nxx) ) {
00993         static int warnflag = 1;
00994         if ( warnflag ) {

```

```

00995             MesPrint("%w Warning: fraction could not be reconstructed in
MakeRational_");
00996             warnflag = 0;
00997             }
00998             ncoef = REDLENG(ncoef);
00999             if ( Mully(BHEAD (UWORD *)n_coef,&ncoef,x1,nx1) )
01000                 goto FromNorm;
01001             }
01002             else {
01003                 ncoef = REDLENG(ncoef);
01004                 if ( MulRat(BHEAD (UWORD *)n_coef,ncoef,xx,nxx,
01005                     (UWORD *)n_coef,&ncoef) ) goto FromNorm;
01006             }
01007             ncoef = INCLENG(ncoef);
01008             TermFree(xx,"Norm-MakeRational");
01009             NumberFree(x2,"Norm-MakeRational");
01010             NumberFree(x1,"Norm-MakeRational");
01011             }
01012             break;
01013         case TERMFUNCTION:
01014             if ( t[1] == FUNHEAD && AN.cTerm ) {
01015                 ANsr = r; ANsm = m; ANsc = AN.cTerm;
01016                 AN.cTerm = 0;
01017                 t = ANsc + 1;
01018                 m = ANsc + *ANsc;
01019                 ncoef = REDLENG(ncoef);
01020                 nnum = REDLENG(m[-1]);
01021                 m -= ABS(m[-1]);
01022                 if ( MulRat(BHEAD (UWORD *)n_coef,ncoef,(UWORD *)m,nnum,
01023                     (UWORD *)n_coef,&ncoef) ) goto FromNorm;
01024                 ncoef = INCLENG(ncoef);
01025                 r = t;
01026             }
01027             break;
01028         case FIRSTBRACKET:
01029             if ( ( t[1] == FUNHEAD+2 ) && t[FUNHEAD] == -EXPRESSION ) {
01030                 if ( GetFirstBracket(termout,t[FUNHEAD+1]) < 0 ) goto FromNorm;
01031                 if ( *termout == 0 ) goto NormZero;
01032                 if ( *termout > 4 ) {
01033                     WORD *r1, *r2, *r3;
01034                     while ( r < m ) *t++ = *r++;
01035                     r1 = term + *term;
01036                     r2 = termout + *termout; r2 -= ABS(r2[-1]);
01037                     while ( r < r1 ) *r2++ = *r++;
01038                     r3 = termout + 1;
01039                     while ( r3 < r2 ) *t++ = *r3++;
01040                     *term = t - term;
01041                     if ( AT.WorkPointer > term && AT.WorkPointer < t )
01042                         AT.WorkPointer = t;
01043                     goto Restart;
01044                 }
01045             }
01046             break;
01047         case FIRSTTERM:
01048         case CONTENTTERM:
01049             if ( ( t[1] == FUNHEAD+2 ) && t[FUNHEAD] == -EXPRESSION ) {
01050                 {
01051                     EXPRESSIONS e = Expressions+t[FUNHEAD+1];
01052                     POSITION oldondisk = AS.OldOnFile[t[FUNHEAD+1]];
01053                     if ( e->replace == NEWLYDEFINEDEXPRESSION ) {
01054                         AS.OldOnFile[t[FUNHEAD+1]] = e->onfile;
01055                     }
01056                     if ( *t == FIRSTTERM ) {
01057                         if ( GetFirstTerm(termout,t[FUNHEAD+1],0) < 0 ) goto FromNorm;
01058                     }
01059                     else if ( *t == CONTENTTERM ) {
01060                         if ( GetContent(termout,t[FUNHEAD+1]) < 0 ) goto FromNorm;
01061                     }
01062                     AS.OldOnFile[t[FUNHEAD+1]] = oldondisk;
01063                     if ( *termout == 0 ) goto NormZero;
01064                 }
01065             }
01066             PasteIn;;
01067             {
01068                 WORD *r1, *r2, *r3, *r4, *r5, nr1, *rterm;
01069                 r2 = termout + *termout; lnum = r2 - ABS(r2[-1]);
01070                 nnum = REDLENG(r2[-1]);
01071
01072                 r1 = term + *term; r3 = r1 - ABS(r1[-1]);
01073                 nr1 = REDLENG(r1[-1]);
01074                 if ( Mully(BHEAD (UWORD *)lnum,&nnum,(UWORD *)r3,nr1) ) goto FromNorm;
01075                 nnum = INCLENG(nnum); nr1 = ABS(nnum); lnum[nr1-1] = nnum;
01076                 rterm = TermMalloc("FirstTerm/ContentTerm");
01077                 r4 = rterm+1; r5 = term+1; while ( r5 < t ) *r4++ = *r5++;
01078                 r5 = termout+1; while ( r5 < lnum ) *r4++ = *r5++;
01079                 r5 = r; while ( r5 < r3 ) *r4++ = *r5++;
01080                 r5 = lnum; NCOPY(r4,r5,nr1);
01081                 *rterm = r4-rterm;

```

```

01081         nrl = *rterm; r1 = term; r2 = rterm; NCOPY(r1,r2,nrl);
01082         TermFree(rterm,"FirstTerm/ContentTerm");
01083         if ( AT.WorkPointer > term && AT.WorkPointer < r1 )
01084             AT.WorkPointer = r1;
01085         goto Restart;
01086     }
01087 }
01088 else if ( ( t[1] == FUNHEAD+2 ) && t[FUNHEAD] == -DOLLAREXPRESSION ) {
01089     DOLLARS d = Dollars + t[FUNHEAD+1], newd = 0;
01090     int idol, ido;
01091 #ifdef WITHPTHREADS
01092     int nummodopt, dtype = -1;
01093     if ( AS.MultiThreaded && ( AC.mparallelfld == PARALLELFLAG ) ) {
01094         for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
01095             if ( t[FUNHEAD+1] == ModOptdollars[nummodopt].number ) break;
01096         }
01097         if ( nummodopt < NumModOptdollars ) {
01098             dtype = ModOptdollars[nummodopt].type;
01099             if ( dtype == MODLOCAL ) {
01100                 d = ModOptdollars[nummodopt].dstruct+AT.identity;
01101             }
01102         }
01103     }
01104 #endif
01105     if ( d->where && ( d->type == DOLTERMS || d->type == DOLNUMBER ) ) {
01106         newd = d;
01107     }
01108     else {
01109         if ( ( newd = DolToTerms(BHEAD t[FUNHEAD+1]) ) == 0 )
01110             goto NormZero;
01111     }
01112     if ( newd->where[0] == 0 ) {
01113         M_free(newd,"Copy of dollar variable");
01114         goto NormZero;
01115     }
01116     if ( *t == FIRSTTERM ) {
01117         idol = newd->where[0];
01118         for ( ido = 0; ido < idol; ido++ ) termout[ido] = newd->where[ido];
01119     }
01120     else if ( *t == CONTENTTERM ) {
01121         WORD *tterm;
01122         tterm = newd->where;
01123         idol = tterm[0];
01124         for ( ido = 0; ido < idol; ido++ ) termout[ido] = tterm[ido];
01125         tterm += tterm;
01126         while ( *tterm ) {
01127             if ( ContentMerge(BHEAD termout,tterm) < 0 ) goto FromNorm;
01128             tterm += tterm;
01129         }
01130     }
01131     if ( newd != d ) {
01132         if ( newd->factors ) M_free(newd->factors,"Dollar factors");
01133         M_free(newd,"Copy of dollar variable");
01134         newd = 0;
01135     }
01136     goto PasteIn;
01137 }
01138 break;
01139 case TERMSINEXPR:
01140 {
01141     if ( ( t[1] == FUNHEAD+2 ) && t[FUNHEAD] == -EXPRESSION ) {
01142         x = TermsInExpression(t[FUNHEAD+1]);
01143         if ( x == 0 ) goto NormZero;
01144         if ( x < 0 ) {
01145             x = -x;
01146             if ( x > (LONG)WORDMASK ) { lnum[0] = x & WORDMASK;
01147                 lnum[1] = x » BITSINWORD; nnum = -2;
01148             }
01149             else { lnum[0] = x; nnum = -1; }
01150         }
01151         else if ( x > (LONG)WORDMASK ) {
01152             lnum[0] = x & WORDMASK;
01153             lnum[1] = x » BITSINWORD;
01154             nnum = 2;
01155         }
01156         else { lnum[0] = x; nnum = 1; }
01157         ncoef = REDLENG(ncoef);
01158         if ( Mully(BHEAD (UWORD *)n_coef,&ncoef, (UWORD *)lnum,nnum) )
01159             goto FromNorm;
01160         ncoef = INCLENG(ncoef);
01161     }
01162     else if ( ( t[1] == FUNHEAD+2 ) && t[FUNHEAD] == -DOLLAREXPRESSION ) {
01163         x = TermsInDollar(t[FUNHEAD+1]);
01164         goto multermnum;
01165     }
01166     else { pcom[ncom++] = t; }
01167 }

```

```

01168         break;
01169     case SIZEOFFUNCTION:
01170     {
01171         if ( ( t[1] == FUNHEAD+2 ) && t[FUNHEAD] == -EXPRESSION ) {
01172             x = SizeOfExpression(t[FUNHEAD+1]);
01173             goto multermnum;
01174         }
01175         else if ( ( t[1] == FUNHEAD+2 ) && t[FUNHEAD] == -DOLLAREXPRESSION ) {
01176             x = SizeOfDollar(t[FUNHEAD+1]);
01177             goto multermnum;
01178         }
01179         else { pcom[ncom++] = t; }
01180     }
01181     break;
01182 case MATCHFUNCTION:
01183 case PATTERNFUNCTION:
01184     break;
01185 case BINOMIAL:
01186 /*
01187     Binomial function for internal use for the moment.
01188     The routine in reken.c should be more efficient.
01189 */
01190     if ( t[1] == FUNHEAD+4 && t[FUNHEAD] == -SNUMBER
01191         && t[FUNHEAD+1] >= 0 && t[FUNHEAD+2] == -SNUMBER
01192         && t[FUNHEAD+3] >= 0 && t[FUNHEAD+1] >= t[FUNHEAD+3] ) {
01193         if ( t[FUNHEAD+1] > t[FUNHEAD+3] ) {
01194             if ( GetBinom((UWORD *)lnum,&nnum,
01195                 t[FUNHEAD+1],t[FUNHEAD+3]) ) goto FromNorm;
01196             if ( nnum == 0 ) goto NormZero;
01197             goto MulIn;
01198         }
01199     }
01200     else pcom[ncom++] = t;
01201     break;
01202 case SIGNFUN:
01203 /*
01204     Numerical function giving (-1)^arg
01205 */
01206     if ( t[1] == FUNHEAD+2 && t[FUNHEAD] == -SNUMBER ) {
01207         if ( ( t[FUNHEAD+1] & 1 ) != 0 ) ncoef = -ncoef;
01208     }
01209     else if ( ( t[FUNHEAD] > 0 ) && ( t[1] == FUNHEAD+t[FUNHEAD] )
01210         && ( t[FUNHEAD] == ARGHEAD+1+abs(t[t[1]-1]) ) ) {
01211         UWORD *numer1,*denom1;
01212         WORD nsize = abs(t[t[1]-1]), nnsz, isize;
01213         nnsz = (nsize-1)/2;
01214         numer1 = (UWORD *) (t + FUNHEAD+ARGHEAD+1);
01215         denom1 = numer1 + nnsz;
01216         for ( isize = 1; isize < nnsz; isize++ ) {
01217             if ( denom1[isize] ) break;
01218         }
01219         if ( ( denom1[0] != 1 ) || isize < nnsz ) {
01220             pcom[ncom++] = t;
01221         }
01222         else {
01223             if ( ( numer1[0] & 1 ) != 0 ) ncoef = -ncoef;
01224         }
01225     }
01226     else {
01227         goto doflags;
01228         pcom[ncom++] = t; /*
01229     */
01230     break;
01231 case SIGFUNCTION:
01232 /*
01233     Numerical function giving the sign of the numerical argument
01234     The sign of zero is 1.
01235     If there are roots of unity they are part of the sign.
01236 */
01237     if ( t[1] == FUNHEAD+2 && t[FUNHEAD] == -SNUMBER ) {
01238         if ( t[FUNHEAD+1] < 0 ) ncoef = -ncoef;
01239     }
01240     else if ( ( t[1] == FUNHEAD+2 ) && ( t[FUNHEAD] == -SYMBOL )
01241         && ( t[FUNHEAD+1] <= NumSymbols && t[FUNHEAD+1] > -MAXPOWER )
01242         && ( symbols[t[FUNHEAD+1]].complex & VARTYPEROOTOFUNITY ) == VARTYPEROOTOFUNITY ) {
01243         k = t[FUNHEAD+1];
01244         from = m;
01245         i = nsym;
01246         m = ppsym;
01247         if ( i > 0 ) do {
01248             m -= 2;
01249             if ( k == *m ) {
01250                 m++;
01251                 *m = *m + 1;
01252                 *m %= symbols[k].maxpower;
01253                 if ( ( symbols[k].complex & VARTYPEMINUS ) == VARTYPEMINUS ) {

```

```

01254             if ( ( ( symbols[k].maxpower & 1 ) == 0 ) &&
01255             ( *m >= symbols[k].maxpower/2 ) ) {
01256                 *m -= symbols[k].maxpower/2; ncoef = -ncoef;
01257             }
01258         }
01259         if ( !*m ) {
01260             m--;
01261             while ( i < nsym )
01262                 { *m = m[2]; m++; *m = m[2]; m++; i++; }
01263             ppsym -= 2;
01264             nsym--;
01265         }
01266         goto sigDoneSymbol;
01267     }
01268     } while ( k < *m && --i > 0 );
01269     m = ppsym;
01270     while ( i < nsym )
01271         { m--; m[2] = *m; m--; m[2] = *m; i++; }
01272     *m++ = k;
01273     *m = 1;
01274     ppsym += 2;
01275     nsym++;
01276 sigDoneSymbol:;
01277     m = from;
01278 }
01279 else if ( ( t[FUNHEAD] > 0 ) && ( t[1] == FUNHEAD+t[FUNHEAD] ) ) {
01280     if ( t[FUNHEAD] == ARGHEAD+1+abs(t[t[1]-1]) ) {
01281         if ( t[t[1]-1] < 0 ) ncoef = -ncoef;
01282     }
01283     /*
01284     Now we should fish out the roots of unity
01285     */
01286     else if ( ( t[FUNHEAD+ARGHEAD]+FUNHEAD+ARGHEAD == t[1] )
01287     && ( t[FUNHEAD+ARGHEAD+1] == SYMBOL ) ) {
01288         WORD *ts = t + FUNHEAD+ARGHEAD+3;
01289         WORD its = ts[-1]-2;
01290         while ( its > 0 ) {
01291             if ( ( *ts != 0 ) && ( ( *ts > NumSymbols || *ts <= -MAXPOWER )
01292             || ( symbols[*ts].complex & VARTYPEROOTOFUNITY ) != VARTYPEROOTOFUNITY ) ) {
01293                 goto signogood;
01294             }
01295             ts += 2; its -= 2;
01296         }
01297         /*
01298         Now we have only roots of unity which should be
01299         registered in the list of symbols.
01300         */
01301         if ( t[t[1]-1] < 0 ) ncoef = -ncoef;
01302         ts = t + FUNHEAD+ARGHEAD+3;
01303         its = ts[-1]-2;
01304         from = m;
01305         while ( its > 0 ) {
01306             i = nsym;
01307             m = ppsym;
01308             if ( i > 0 ) do {
01309                 m -= 2;
01310                 if ( *ts == *m ) {
01311                     ts++; m++;
01312                     *m += *ts;
01313                     if ( ( ts[-1] <= NumSymbols && ts[-1] > -MAXPOWER ) &&
01314                     ( symbols[ts[-1]].complex & VARTYPEROOTOFUNITY ) ==
01315                     VARTYPEROOTOFUNITY ) {
01316                         *m %= symbols[ts[-1]].maxpower;
01317                         if ( *m < 0 ) *m += symbols[ts[-1]].maxpower;
01318                         if ( ( symbols[ts[-1]].complex & VARTYPEMINUS ) ==
01319                         VARTYPEMINUS ) {
01320                             if ( ( ( symbols[ts[-1]].maxpower & 1 ) == 0 ) &&
01321                             ( *m >= symbols[ts[-1]].maxpower/2 ) ) {
01322                                 *m -= symbols[ts[-1]].maxpower/2; ncoef = -ncoef;
01323                             }
01324                         }
01325                     }
01326                     if ( !*m ) {
01327                         m--;
01328                         while ( i < nsym )
01329                             { *m = m[2]; m++; *m = m[2]; m++; i++; }
01330                         ppsym -= 2;
01331                         nsym--;
01332                     }
01333                     ts++; its -= 2;
01334                     goto sigNextSymbol;
01335                 }
01336             } while ( *ts < *m && --i > 0 );
01337             m = ppsym;
01338             while ( i < nsym )
01339                 { m--; m[2] = *m; m--; m[2] = *m; i++; }

```

```

01338             *m++ = *ts++;
01339             *m = *ts++;
01340             ppsym += 2;
01341             nsym++; its -= 2;
01342 sigNextSymbol:
01343             }
01344             m = from;
01345             }
01346             else {
01347 signogood:
01348                 pcom[ncom++] = t;
01349             }
01350             else pcom[ncom++] = t;
01351             break;
01352 case ABSFUNCTION:
01353 /*
01354     Numerical function giving the absolute value of the
01355     numerical argument. Or roots of unity.
01356 */
01357     if ( t[1] == FUNHEAD+2 && t[FUNHEAD] == -SNUMBER ) {
01358         k = t[FUNHEAD+1];
01359         if ( k < 0 ) k = -k;
01360         if ( k == 0 ) goto NormZero;
01361         *((UWORD *)lnum) = k; nnum = 1;
01362         goto MulIn;
01363     }
01364     else if ( t[1] == FUNHEAD+2 && t[FUNHEAD] == -SYMBOL ) {
01365         k = t[FUNHEAD+1];
01366         if ( ( k > NumSymbols || k <= -MAXPOWER )
01367             || ( symbols[k].complex & VARTYPEROOTOFUNITY ) != VARTYPEROOTOFUNITY )
01368             goto absnogood;
01369     }
01370     else if ( ( t[FUNHEAD] > 0 ) && ( t[1] == FUNHEAD+t[FUNHEAD] )
01371         && ( t[1] == FUNHEAD+ARGHEAD+t[FUNHEAD+ARGHEAD] ) ) {
01372         if ( t[FUNHEAD] == ARGHEAD+1+abs(t[t[1]-1]) ) {
01373             WORD *ts;
01374             ts = t + t[1] - 1;
01375 absnosymbols:
01376             ncoef = REDLENG(ncoef);
01377             nnum = REDLENG(*ts);
01378             if ( nnum < 0 ) nnum = -nnum;
01379             if ( MulRat(BHEAD (UWORD *)n_coef,ncoef,
01380                 (UWORD *) (ts-ABS(*ts)+1),nnum,
01381                 (UWORD *)n_coef,&ncoef) ) goto FromNorm;
01382             ncoef = INCLENG(ncoef);
01383         }
01384 /*
01385     Now get rid of the roots of unity. This includes i_
01386 */
01387     else if ( t[FUNHEAD+ARGHEAD+1] == SYMBOL ) {
01388         WORD *ts = t+FUNHEAD+ARGHEAD+1;
01389         WORD its = ts[1] - 2;
01390         ts += 2;
01391         while ( its > 0 ) {
01392             if ( *ts == 0 ) { }
01393             else if ( ( *ts > NumSymbols || *ts <= -MAXPOWER )
01394                 || ( symbols[*ts].complex & VARTYPEROOTOFUNITY )
01395                     != VARTYPEROOTOFUNITY ) goto absnogood;
01396             its -= 2; ts += 2;
01397         }
01398         goto absnosymbols;
01399     }
01400     else {
01401 absnogood:
01402         pcom[ncom++] = t;
01403     }
01404     else pcom[ncom++] = t;
01405     break;
01406 case MODFUNCTION:
01407 case MOD2FUNCTION:
01408 /*
01409     Mod function. Does work if two arguments and the
01410     second argument is a positive short number
01411 */
01412     if ( t[1] == FUNHEAD+4 && t[FUNHEAD] == -SNUMBER
01413         && t[FUNHEAD+2] == -SNUMBER && t[FUNHEAD+3] > 1 ) {
01414         WORD tmod;
01415         tmod = (t[FUNHEAD+1]%t[FUNHEAD+3]);
01416         if ( tmod < 0 ) tmod += t[FUNHEAD+3];
01417         if ( *t == MOD2FUNCTION && tmod > t[FUNHEAD+3]/2 )
01418             tmod -= t[FUNHEAD+3];
01419         if ( tmod < 0 ) {
01420             *((UWORD *)lnum) = -tmod;
01421             nnum = -1;
01422         }
01423         else if ( tmod > 0 ) {
01424             *((UWORD *)lnum) = tmod;

```



```

01425         nnum = 1;
01426     }
01427     else goto NormZero;
01428     goto MulIn;
01429 }
01430 else if ( t[1] > t[FUNHEAD+2] && t[FUNHEAD] > 0
01431 && t[FUNHEAD+t[FUNHEAD]] == -SNUMBER
01432 && t[FUNHEAD+t[FUNHEAD]+1] > 1
01433 && t[1] == FUNHEAD+2+t[FUNHEAD] ) {
01434     WORD *tntt = t+FUNHEAD, iii;
01435     iii = tntt[*tntt-1];
01436     if ( *tntt == tntt[ARGHEAD]+ARGHEAD &&
01437         tntt[ARGHEAD] == ABS(iii)+1 ) {
01438         WORD ncmmod = 1;
01439         WORD cmod = tntt[*tntt+1];
01440         iii = REDLENG(iii);
01441         if ( *t == MODFUNCTION ) {
01442             if ( TakeModulus((UWORD *) (tntt+ARGHEAD+1)
01443                 , &iii, (UWORD *) (&cmod), ncmmod, UNPACK|NOINVERSES) )
01444                 goto FromNorm;
01445         }
01446         else {
01447             if ( TakeModulus((UWORD *) (tntt+ARGHEAD+1)
01448                 , &iii, (UWORD *) (&cmod), ncmmod, UNPACK|POSNEG|NOINVERSES) )
01449                 goto FromNorm;
01450         }
01451         if ( *t == MOD2FUNCTION && tntt[ARGHEAD+1] > cmod/2 && iii > 0 ) {
01452             tntt[ARGHEAD+1] -= cmod;
01453         }
01454         if ( tntt[ARGHEAD+1] < 0 ) {
01455             *((UWORD *) lnum) = -tntt[ARGHEAD+1];
01456             nnum = -1;
01457         }
01458         else if ( tntt[ARGHEAD+1] > 0 ) {
01459             *((UWORD *) lnum) = tntt[ARGHEAD+1];
01460             nnum = 1;
01461         }
01462         else goto NormZero;
01463         goto MulIn;
01464     }
01465 }
01466 else if ( t[1] == FUNHEAD+2 && t[FUNHEAD] == -SNUMBER ) {
01467     *((UWORD *) lnum) = t[FUNHEAD+1];
01468     if ( *lnum == 0 ) goto NormZero;
01469     nnum = 1;
01470     goto MulIn;
01471 }
01472 else if ( ( ( t[FUNHEAD] < 0 ) && ( t[FUNHEAD] == -SNUMBER )
01473 && ( t[1] >= ( FUNHEAD+6+ARGHEAD ) )
01474 && ( t[FUNHEAD+2] >= 4+ARGHEAD )
01475 && ( t[t[1]-1] == t[FUNHEAD+2+ARGHEAD]-1 ) ) ||
01476 ( ( t[FUNHEAD] > 0 )
01477 && ( t[FUNHEAD]-ARGHEAD-1 == ABS(t[FUNHEAD+t[FUNHEAD]-1]) )
01478 && ( t[FUNHEAD+t[FUNHEAD]-ARGHEAD-1 == t[t[1]-1] ) ) ) ) {
01479 /*
01480     Check that the last (long) number is integer
01481 */
01482     WORD *tntt = t + t[1], iii, iiil;
01483     UWORD coefbuf[2], *coef2, ncoef2;
01484     iii = (tntt[-1]-1)/2;
01485     tntt -= iii;
01486     if ( tntt[-1] != 1 ) {
01487 exitfromhere:
01488         pcom[ncom++] = t;
01489         break;
01490     }
01491     iiil--;
01492     for ( iiil = 0; iiil < iii; iiil++ ) {
01493         if ( tntt[iiil] != 0 ) goto exitfromhere;
01494     }
01495 /*
01496     Now we have a hit!
01497     The first argument will be put in lnum.
01498     It will be a rational.
01499     The second argument will be a long integer in coef2.
01500 */
01501     tntt = t + FUNHEAD;
01502     if ( *tntt < 0 ) {
01503         if ( tntt[1] < 0 ) {
01504             nnum = -1; lnum[0] = -tntt[1]; lnum[1] = 1;
01505         }
01506         else {
01507             nnum = 1; lnum[0] = tntt[1]; lnum[1] = 1;
01508         }
01509     }
01510     else {
01511         nnum = ABS(tntt[tntt[0]-1] - 1);

```

```

01512         for ( iii = 0; iii < nnum; iii++ ) {
01513             lnum[iii] = ttt[ARGHEAD+1+iii];
01514         }
01515         nnum = nnum/2;
01516         if ( ttt[ttt[0]-1] < 0 ) nnum = -nnum;
01517     }
01518     NEXTARG(ttt);
01519     if ( *ttt < 0 ) {
01520         coef2 = coefbuf;
01521         ncoef2 = 3; *coef2 = (UWORD) (ttt[1]);
01522         coef2[1] = 1;
01523     }
01524     else {
01525         coef2 = (UWORD *) (ttt+ARGHEAD+1);
01526         ncoef2 = (ttt[ttt[0]-1]-1)/2;
01527     }
01528     if ( TakeModulus( (UWORD *) lnum, &nnum, (UWORD *) coef2, ncoef2,
01529                     UNPACK|NOINVERSES|FROMFUNCTION ) ) {
01530         goto FromNorm;
01531     }
01532     if ( *t == MOD2FUNCTION && nnum > 0 ) {
01533         UWORD *coef3 = NumberMalloc("Mod2Function"), two = 2;
01534         WORD ncoef3;
01535         if ( MulLong( (UWORD *) lnum, nnum, &two, 1, coef3, &ncoef3 ) )
01536             goto FromNorm;
01537         if ( BigLong(coef3, ncoef3, (UWORD *) coef2, ncoef2) > 0 ) {
01538             nnum = -nnum;
01539             AddLong( (UWORD *) lnum, nnum, (UWORD *) coef2, ncoef2
01540                     , (UWORD *) lnum, &nnum);
01541             nnum = -nnum;
01542         }
01543         NumberFree(coef3, "Mod2Function");
01544     }
01545     /*
01546     Do we have to pack? No, because the answer is not a fraction
01547     */
01548     ncoef = REDLENG(ncoef);
01549     if ( Mully(BHEAD (UWORD *) n_coef, &ncoef, (UWORD *) lnum, nnum) )
01550         goto FromNorm;
01551     ncoef = INCLENG(ncoef);
01552 }
01553 else pcom[ncom++] = t;
01554 break;
01555 case EXTEUCLIDEAN:
01556 {
01557     WORD argcount = 0, *tc, *ts, xc, xs, *tcc;
01558     UWORD *Num1, *Num2, *Num3, *Num4;
01559     WORD size1, size2, size3, size4, space;
01560     tc = t+FUNHEAD; ts = t + t[1];
01561     while ( argcount < 3 && tc < ts ) { NEXTARG(tc); argcount++; }
01562     if ( argcount != 2 ) goto defaultcase;
01563     if ( t[FUNHEAD] == -SNUMBER ) {
01564         if ( t[FUNHEAD+1] <= 1 ) goto defaultcase;
01565         if ( t[FUNHEAD+2] == -SNUMBER ) {
01566             if ( t[FUNHEAD+3] <= 1 ) goto defaultcase;
01567             Num2 = NumberMalloc("modinverses");
01568             *Num2 = t[FUNHEAD+3]; size2 = 1;
01569         }
01570         else {
01571             if ( ts[-1] < 0 ) goto defaultcase;
01572             if ( ts[-1] != t[FUNHEAD+2]-ARGHEAD-1 ) goto defaultcase;
01573             xs = (ts[-1]-1)/2;
01574             tcc = ts-xs-1;
01575             if ( *tcc != 1 ) goto defaultcase;
01576             for ( i = 1; i < xs; i++ ) {
01577                 if ( tcc[i] != 0 ) goto defaultcase;
01578             }
01579             Num2 = NumberMalloc("modinverses");
01580             size2 = xs;
01581             for ( i = 0; i < xs; i++ ) Num2[i] = t[FUNHEAD+ARGHEAD+3+i];
01582         }
01583         Num1 = NumberMalloc("modinverses");
01584         *Num1 = t[FUNHEAD+1]; size1 = 1;
01585     }
01586     else {
01587         tc = t + FUNHEAD + t[FUNHEAD];
01588         if ( tc[-1] < 0 ) goto defaultcase;
01589         if ( tc[-1] != t[FUNHEAD]-ARGHEAD-1 ) goto defaultcase;
01590         xc = (tc[-1]-1)/2;
01591         tcc = tc-xc-1;
01592         if ( *tcc != 1 ) goto defaultcase;
01593         for ( i = 1; i < xc; i++ ) {
01594             if ( tcc[i] != 0 ) goto defaultcase;
01595         }
01596         if ( *tc == -SNUMBER ) {
01597             if ( tc[1] <= 1 ) goto defaultcase;
01598             Num2 = NumberMalloc("modinverses");

```

```

01599         *Num2 = tc[1]; size2 = 1;
01600     }
01601     else {
01602         if ( ts[-1] < 0 ) goto defaultcase;
01603         if ( ts[-1] != t[FUNHEAD+2]-ARGHEAD-1 ) goto defaultcase;
01604         xs = (ts[-1]-1)/2;
01605         tcc = ts-xs-1;
01606         if ( *tcc != 1 ) goto defaultcase;
01607         for ( i = 1; i < xs; i++ ) {
01608             if ( tcc[i] != 0 ) goto defaultcase;
01609         }
01610         Num2 = NumberMalloc("modinverses");
01611         size2 = xs;
01612         for ( i = 0; i < xs; i++ ) Num2[i] = tc[ARGHEAD+1+i];
01613     }
01614     Num1 = NumberMalloc("modinverses");
01615     size1 = xc;
01616     for ( i = 0; i < xc; i++ ) Num1[i] = t[FUNHEAD+ARGHEAD+1+i];
01617 }
01618 Num3 = NumberMalloc("modinverses");
01619 Num4 = NumberMalloc("modinverses");
01620 GetLongModInverses(BHEAD Num1,size1,Num2,size2
01621     ,Num3,&size3,Num4,&size4);
01622 /*
01623     Now we have to compose the answer. This needs more space
01624     and hence we have to put this inside the term.
01625     Compute first how much extra space we need.
01626     Then move the trailing part of the term upwards.
01627     Do not forget relevant pointers!!! (r, m, termout, AT.WorkPointer)
01628 */
01629     space = 0;
01630     if ( ( size3 == 1 || size3 == -1 ) && (*Num3&TOPBITONLY) == 0 ) space += 2;
01631     else space += ARGHEAD + 2*ABS(size3) + 2;
01632     if ( ( size4 == 1 || size4 == -1 ) && (*Num4&TOPBITONLY) == 0 ) space += 2;
01633     else space += ARGHEAD + 2*ABS(size4) + 2;
01634     tt = term + *term; u = tt + space;
01635     while ( tt >= ts ) *--u = *--tt;
01636     m += space; r += space;
01637     *term += space;
01638     t[1] += space;
01639     if ( ( size3 == 1 || size3 == -1 ) && (*Num3&TOPBITONLY) == 0 ) {
01640         *ts++ = -SNUMBER; *ts = (WORD)(*Num3);
01641         if ( size3 < 0 ) *ts = -*ts;
01642         ts++;
01643     }
01644     else {
01645         *ts++ = 2*ABS(size3)+ARGHEAD+2;
01646         *ts++ = 0; FILLARG(ts)
01647         *ts++ = 2*ABS(size3)+1;
01648         for ( i = 0; i < ABS(size3); i++ ) *ts++ = Num3[i];
01649         *ts++ = 1;
01650         for ( i = 1; i < ABS(size3); i++ ) *ts++ = 0;
01651         if ( size3 < 0 ) *ts++ = 2*size3-1;
01652         else *ts++ = 2*size3+1;
01653     }
01654     if ( ( size4 == 1 || size4 == -1 ) && (*Num4&TOPBITONLY) == 0 ) {
01655         *ts++ = -SNUMBER; *ts = *Num4;
01656         if ( size4 < 0 ) *ts = -*ts;
01657         ts++;
01658     }
01659     else {
01660         *ts++ = 2*ABS(size4)+ARGHEAD+2;
01661         *ts++ = 0; FILLARG(ts)
01662         *ts++ = 2*ABS(size4)+2;
01663         for ( i = 0; i < ABS(size4); i++ ) *ts++ = Num4[i];
01664         *ts++ = 1;
01665         for ( i = 1; i < ABS(size4); i++ ) *ts++ = 0;
01666         if ( size4 < 0 ) *ts++ = 2*size4-1;
01667         else *ts++ = 2*size4+1;
01668     }
01669     NumberFree(Num4,"modinverses");
01670     NumberFree(Num3,"modinverses");
01671     NumberFree(Num1,"modinverses");
01672     NumberFree(Num2,"modinverses");
01673     t[2] = 0; /* mark function as clean. */
01674     goto Restart;
01675 }
01676 break;
01677 case GCDFUNCTION:
01678 #ifdef EVALUATEGCD
01679 #ifdef NEWGCDFUNCTION
01680     {
01681 /*
01682         Has two integer arguments
01683         Four cases: S,S, S,L, L,S, L,L
01684 */
01685         WORD *num1, *num2, size1, size2, stor1, stor2, *ttt, ti;

```

```

01686         if ( t[1] == FUNHEAD+4 && t[FUNHEAD] == -SNUMBER
01687         && t[FUNHEAD+2] == -SNUMBER && t[FUNHEAD+1] != 0
01688         && t[FUNHEAD+3] != 0 ) { /* Short,Short */
01689             stor1 = t[FUNHEAD+1];
01690             stor2 = t[FUNHEAD+3];
01691             if ( stor1 < 0 ) stor1 = -stor1;
01692             if ( stor2 < 0 ) stor2 = -stor2;
01693             num1 = &stor1; num2 = &stor2;
01694             size1 = size2 = 1;
01695             goto gcdcalc;
01696         }
01697     else if ( t[1] > FUNHEAD+4 ) {
01698         if ( t[FUNHEAD] == -SNUMBER && t[FUNHEAD+1] != 0
01699         && t[FUNHEAD+2] == t[1]-FUNHEAD-2 &&
01700         ABS(t[t[1]-1]) == t[FUNHEAD+2]-1-ARGHEAD ) { /* Short,Long */
01701             num2 = t + t[1];
01702             size2 = ABS(num2[-1]);
01703             ttt = num2-1;
01704             num2 -= size2;
01705             size2 = (size2-1)/2;
01706             ti = size2;
01707             while ( ti > 1 && ttt[-1] == 0 ) { ttt--; ti--; }
01708             if ( ti == 1 && ttt[-1] == 1 ) {
01709                 stor1 = t[FUNHEAD+1];
01710                 if ( stor1 < 0 ) stor1 = -stor1;
01711                 num1 = &stor1;
01712                 size1 = 1;
01713                 goto gcdcalc;
01714             }
01715             else pcom[ncom++] = t;
01716         }
01717     else if ( t[FUNHEAD] > 0 &&
01718     t[FUNHEAD]-1-ARGHEAD == ABS(t[t[FUNHEAD]+FUNHEAD-1]) ) {
01719         num1 = t + FUNHEAD + t[FUNHEAD];
01720         size1 = ABS(num1[-1]);
01721         ttt = num1-1;
01722         num1 -= size1;
01723         size1 = (size1-1)/2;
01724         ti = size1;
01725         while ( ti > 1 && ttt[-1] == 0 ) { ttt--; ti--; }
01726         if ( ti == 1 && ttt[-1] == 1 ) {
01727             if ( t[1]-FUNHEAD == t[FUNHEAD]+2 && t[t[1]-2] == -SNUMBER
01728             && t[t[1]-1] != 0 ) { /* Long,Short */
01729                 stor2 = t[t[1]-1];
01730                 if ( stor2 < 0 ) stor2 = -stor2;
01731                 num2 = &stor2;
01732                 size2 = 1;
01733                 goto gcdcalc;
01734             }
01735             else if ( t[1]-FUNHEAD == t[FUNHEAD]+t[FUNHEAD+t[FUNHEAD]]
01736             && ABS(t[t[1]-1]) == t[FUNHEAD+t[FUNHEAD]] - ARGHEAD-1 ) {
01737                 num2 = t + t[1];
01738                 size2 = ABS(num2[-1]);
01739                 ttt = num2-1;
01740                 num2 -= size2;
01741                 size2 = (size2-1)/2;
01742                 ti = size2;
01743                 while ( ti > 1 && ttt[-1] == 0 ) { ttt--; ti--; }
01744                 if ( ti == 1 && ttt[-1] == 1 ) {
01745                     if ( GcdLong(BHEAD (UWORD *)num1,size1,(UWORD *)num2,size2
01746                     , (UWORD *)lnum,&nnum) ) goto FromNorm;
01747                     goto MulIn;
01748                 }
01749                 else pcom[ncom++] = t;
01750             }
01751             else pcom[ncom++] = t;
01752         }
01753         else pcom[ncom++] = t;
01754     }
01755     else pcom[ncom++] = t;
01756 }
01757 else pcom[ncom++] = t;
01758 }
01759 #else
01760 {
01761     WORD *gcd = AT.WorkPointer;
01762     if ( ( gcd = EvaluateGcd(BHEAD t) ) == 0 ) goto FromNorm;
01763     if ( *gcd == 4 && gcd[1] == 1 && gcd[2] == 1 && gcd[4] == 0 ) {
01764         AT.WorkPointer = gcd;
01765     }
01766     else if ( gcd[*gcd] == 0 ) {
01767         WORD *t1, iii, change, *num, *den, numsize, densize;
01768         if ( gcd[*gcd-1] < *gcd-1 ) {
01769             t1 = gcd+1;
01770             for ( iii = 2; iii < t1[1]; iii += 2 ) {
01771                 change = ExtraSymbol(t1[iii],t1[iii+1],nsym,ppsym,&ncoef);
01772                 nsym += change;

```

```

01773             ppsym += change * 2;
01774         }
01775     }
01776     t1 = gcd + *gcd;
01777     iii = t1[-1]; num = t1-iii; numsize = (iii-1)/2;
01778     den = num + numsize; densize = numsize;
01779     while ( numsize > 1 && num[numsize-1] == 0 ) numsize--;
01780     while ( densize > 1 && den[densize-1] == 0 ) densize--;
01781     if ( numsize > 1 || num[0] != 1 ) {
01782         ncoef = REDLENG(ncoef);
01783         if ( Mully(BHEAD (UWORD *)n_coef,&ncoef, (UWORD *)num,numsize) ) goto
FromNorm;
01784         ncoef = INCLENG(ncoef);
01785     }
01786     if ( densize > 1 || den[0] != 1 ) {
01787         ncoef = REDLENG(ncoef);
01788         if ( Divvy(BHEAD (UWORD *)n_coef,&ncoef, (UWORD *)den,densize) ) goto
FromNorm;
01789         ncoef = INCLENG(ncoef);
01790     }
01791     AT.WorkPointer = gcd;
01792 }
01793 else { /* a whole expression */
01794 /*
01795     Action: Put the expression in a compiler buffer.
01796     Insert a SUBEXPRESSION subterm
01797     Set the return value of the routine such that in
01798     Generator the term gets sent again to TestSub.
01799
01800     1: put in C (ebufnum)
01801     2: after that the Workspace is free again.
01802     3: insert the SUBEXPRESSION
01803     4: copy the top part of the term down
01804 */
01805     LONG size = AT.WorkPointer - gcd;
01806
01807     ss = AddRHS(AT.ebufnum,1);
01808     while ( (ss + size + 10) > C->Top ) ss = DoubleCbuffer(AT.ebufnum,ss,13);
01809     tt = gcd;
01810     NCOPY(ss,tt,size);
01811     C->rhs[C->numrhs+1] = ss;
01812     C->Pointer = ss;
01813
01814     t[0] = SUBEXPRESSION;
01815     t[1] = SUBEXPSIZE;
01816     t[2] = C->numrhs;
01817     t[3] = 1;
01818     t[4] = AT.ebufnum;
01819     t += 5;
01820     tt = term + *term;
01821     while ( r < tt ) *t++ = *r++;
01822     *term = t - term;
01823
01824     regval = 1;
01825     goto RegEnd;
01826 }
01827 }
01828 #endif
01829 #else
01830 MesPrint(" Unexpected call to EvaluateGCD");
01831 Terminate(-1);
01832 #endif
01833 break;
01834 case MINFUNCTION:
01835 case MAXFUNCTION:
01836     if ( t[1] == FUNHEAD ) break;
01837     {
01838         WORD *ttt = t + FUNHEAD;
01839         WORD *tttstop = t + t[1];
01840         WORD tterm[4], iii;
01841         while ( ttt < tttstop ) {
01842             if ( *ttt > 0 ) {
01843                 if ( ttt[ARGHEAD]-1 > ABS(ttt[*ttt-1]) ) goto nospec;
01844                 ttt += *ttt;
01845             }
01846             else {
01847                 if ( *ttt != -SNUMBER ) goto nospec;
01848                 ttt += 2;
01849             }
01850         }
01851     }
01852 /*
01853     Function has only numerical arguments
01854     Pick up the first argument.
01855 */
01856     ttt = t + FUNHEAD;
01857     if ( *ttt > 0 ) {
01858 loadnew1:

```

```

01858         for ( iii = 0; iii < ttt[ARGHEAD]; iii++ )
01859             n_llnum[iii] = ttt[ARGHEAD+iii];
01860         ttt += *ttt;
01861     }
01862     else {
01863 loadnew2:
01864         if ( ttt[1] == 0 ) {
01865             n_llnum[0] = n_llnum[1] = n_llnum[2] = n_llnum[3] = 0;
01866         }
01867         else {
01868             n_llnum[0] = 4;
01869             if ( ttt[1] > 0 ) { n_llnum[1] = ttt[1]; n_llnum[3] = 3; }
01870             else { n_llnum[1] = -ttt[1]; n_llnum[3] = -3; }
01871             n_llnum[2] = 1;
01872         }
01873         ttt += 2;
01874     }
01875 /*
01876     Now loop over the other arguments
01877 */
01878     while ( ttt < tttstop ) {
01879         if ( *ttt > 0 ) {
01880             if ( n_llnum[0] == 0 ) {
01881                 if ( ( *t == MINFUNCTION && ttt[*ttt-1] < 0 )
01882                     || ( *t == MAXFUNCTION && ttt[*ttt-1] > 0 ) )
01883                     goto loadnew1;
01884             }
01885             else {
01886                 ttt += ARGHEAD;
01887                 iii = CompCoef(n_llnum,ttt);
01888                 if ( ( iii > 0 && *t == MINFUNCTION )
01889                     || ( iii < 0 && *t == MAXFUNCTION ) ) {
01890                     for ( iii = 0; iii < ttt[0]; iii++ )
01891                         n_llnum[iii] = ttt[iii];
01892                 }
01893             }
01894             ttt += *ttt;
01895         }
01896         else {
01897             if ( n_llnum[0] == 0 ) {
01898                 if ( ( *t == MINFUNCTION && ttt[1] < 0 )
01899                     || ( *t == MAXFUNCTION && ttt[1] > 0 ) )
01900                     goto loadnew2;
01901             }
01902             else if ( ttt[1] == 0 ) {
01903                 if ( ( *t == MINFUNCTION && n_llnum[*n_llnum-1] > 0 )
01904                     || ( *t == MAXFUNCTION && n_llnum[*n_llnum-1] < 0 ) ) {
01905                     n_llnum[0] = 0;
01906                 }
01907             }
01908             else {
01909                 tterm[0] = 4; tterm[2] = 1;
01910                 if ( ttt[1] < 0 ) { tterm[1] = -ttt[1]; tterm[3] = -3; }
01911                 else { tterm[1] = ttt[1]; tterm[3] = 3; }
01912                 iii = CompCoef(n_llnum,tterm);
01913                 if ( ( iii > 0 && *t == MINFUNCTION )
01914                     || ( iii < 0 && *t == MAXFUNCTION ) ) {
01915                     for ( iii = 0; iii < 4; iii++ )
01916                         n_llnum[iii] = tterm[iii];
01917                 }
01918             }
01919             ttt += 2;
01920         }
01921     }
01922     if ( n_llnum[0] == 0 ) goto NormZero;
01923     ncoef = REDLENG(ncoef);
01924     nnum = REDLENG(n_llnum[*n_llnum-1]);
01925     if ( MulRat(BHEAD (UWORD *)n_coef,ncoef, (UWORD *)lnum,nnum,
01926         (UWORD *)n_coef,&ncoef) ) goto FromNorm;
01927     ncoef = INCLENG(ncoef);
01928 }
01929 break;
01930 case INVERSEFACTORIAL:
01931     if ( t[FUNHEAD] == -SNUMBER && t[FUNHEAD+1] >= 0 ) {
01932         if ( Factorial(BHEAD t[FUNHEAD+1], (UWORD *)lnum,&nnum) )
01933             goto FromNorm;
01934         ncoef = REDLENG(ncoef);
01935         if ( Divvy(BHEAD (UWORD *)n_coef,&ncoef, (UWORD *)lnum,nnum) ) goto FromNorm;
01936         ncoef = INCLENG(ncoef);
01937     }
01938     else {
01939 nospec:
01940         pcom[ncom++] = t;
01941     }
01942     break;
01943 case MAXPOWEROF:
01944     if ( ( t[FUNHEAD] == -SYMBOL )
01945         && ( t[FUNHEAD+1] > 0 ) && ( t[1] == FUNHEAD+2 ) ) {

```

```

01945         *((UWORD *)lnum) = symbols[t[FUNHEAD+1]].maxpower;
01946         nnum = 1;
01947         goto MulIn;
01948     }
01949     else { pcom[ncom++] = t; }
01950     break;
01951 case MINPOWEROF:
01952     if ( ( t[FUNHEAD] == -SYMBOL )
01953         && ( t[FUNHEAD] > 0 ) && ( t[1] == FUNHEAD+2 ) ) {
01954         *((UWORD *)lnum) = symbols[t[FUNHEAD+1]].minpower;
01955         nnum = 1;
01956         goto MulIn;
01957     }
01958     else { pcom[ncom++] = t; }
01959     break;
01960 case PRIMENUMBER :
01961     if ( t[1] == FUNHEAD+2 && t[FUNHEAD] == -SNUMBER
01962         && t[FUNHEAD+1] > 0 ) {
01963         UWORD xp = (UWORD) (NextPrime(BHEAD t[FUNHEAD+1]));
01964         ncoef = REDLENG(ncoef);
01965         if ( Mully(BHEAD (UWORD *)n_coef,&ncoef,&xp,1) ) goto FromNorm;
01966         ncoef = INCLENG(ncoef);
01967     }
01968     else goto defaultcase;
01969     break;
01970 case MOEBIUS:
01971 /*
01972     Numerical function giving -1,0,1 or no evaluation
01973 */
01974     if ( t[1] == FUNHEAD+2 && t[FUNHEAD] == -SNUMBER
01975         && t[FUNHEAD+1] > 0 ) {
01976         WORD val = Moebius(BHEAD t[FUNHEAD+1]);
01977         if ( val == 0 ) goto NormZero;
01978         if ( val < 0 ) ncoef = -ncoef;
01979     }
01980     else {
01981         pcom[ncom++] = t;
01982     }
01983     break;
01984 case LNUMBER :
01985     ncoef = REDLENG(ncoef);
01986     if ( Mully(BHEAD (UWORD *)n_coef,&ncoef,(UWORD *) (t+3),t[2]) ) goto FromNorm;
01987     ncoef = INCLENG(ncoef);
01988     break;
01989 case SNUMBER :
01990     if ( t[2] < 0 ) {
01991         t[2] = -t[2];
01992         if ( t[3] & 1 ) ncoef = -ncoef;
01993     }
01994     else if ( t[2] == 0 ) {
01995         if ( t[3] < 0 ) goto NormInf;
01996         goto NormZero;
01997     }
01998     lnum[0] = t[2];
01999     nnum = 1;
02000     if ( t[3] && RaisPow(BHEAD (UWORD *)lnum,&nnum,(UWORD) (ABS(t[3]))) ) goto FromNorm;
02001     ncoef = REDLENG(ncoef);
02002     if ( t[3] < 0 ) {
02003         if ( Divvy(BHEAD (UWORD *)n_coef,&ncoef,(UWORD *)lnum,nnum) )
02004             goto FromNorm;
02005     }
02006     else if ( t[3] > 0 ) {
02007         if ( Mully(BHEAD (UWORD *)n_coef,&ncoef,(UWORD *)lnum,nnum) )
02008             goto FromNorm;
02009     }
02010     ncoef = INCLENG(ncoef);
02011     break;
02012 case GAMMA :
02013 case GAMMAI :
02014 case GAMMAFIVE :
02015 case GAMMASIX :
02016 case GAMMASEVEN :
02017     if ( t[1] == FUNHEAD ) {
02018         MLOCK(ErrorMessageLock);
02019         MesPrint("Gamma matrix without spin line encountered.");
02020         MUNLOCK(ErrorMessageLock);
02021         goto NormMin;
02022     }
02023     pnco[nnco++] = t;
02024     t += FUNHEAD+1;
02025     goto ScanCont;
02026 case LEVICIVITA :
02027     peps[neps++] = t;
02028     if ( ( t[2] & DIRTYFLAG ) == DIRTYFLAG ) {
02029         t[2] &= ~DIRTYFLAG;
02030         t[2] |= DIRTYSYMLFLAG;
02031     }

```

```

02032         t += FUNHEAD;
02033 ScanCont:   while ( t < r ) {
02034             if ( *t >= AM.OffsetIndex &&
02035                 ( *t >= AM.DumInd || ( *t < AM.WilInd &&
02036                     indices[*t-AM.OffsetIndex].dimension ) ) )
02037                 pcon[ncon++] = t;
02038             t++;
02039         }
02040         break;
02041 case EXPONENT :
02042     {
02043         WORD *rr;
02044         k = 1;
02045         rr = t + FUNHEAD;
02046         if ( *rr == ARGHEAD || ( *rr == -SNUMBER && rr[1] == 0 ) )
02047             k = 0;
02048         if ( *rr == -SNUMBER && rr[1] == 1 ) break;
02049         if ( *rr <= -FUNCTION ) k = *rr;
02050         NEXTARG(rr)
02051         if ( *rr == ARGHEAD || ( *rr == -SNUMBER && rr[1] == 0 ) ) {
02052             if ( k == 0 ) goto NormZZ;
02053             break;
02054         }
02055         if ( *rr == -SNUMBER && rr[1] > 0 && rr[1] < MAXPOWER && k < 0 ) {
02056             k = -k;
02057             if ( functions[k-FUNCTION].commute ) {
02058                 for ( i = 0; i < rr[1]; i++ ) pncnco[nncnco++] = rr-1;
02059             }
02060             else {
02061                 for ( i = 0; i < rr[1]; i++ ) pcom[ncom++] = rr-1;
02062             }
02063             break;
02064         }
02065         if ( k == 0 ) goto NormZero;
02066         if ( t[FUNHEAD] == -SYMBOL && *rr == -SNUMBER && t[1] == FUNHEAD+4 ) {
02067             if ( rr[1] < MAXPOWER ) {
02068                 t[FUNHEAD+2] = t[FUNHEAD+1]; t += FUNHEAD+2;
02069                 from = m;
02070                 goto NextSymbol;
02071             }
02072         }
02073 /*
02074         if ( ( t[FUNHEAD] > 0 && t[FUNHEAD+1] != 0 )
02075             || ( *rr > 0 && rr[1] != 0 ) ) {}
02076         else
02077 */
02078             t[2] &= ~DIRTYSYMFLAG;
02079
02080             pncnco[nncnco++] = t;
02081         }
02082         break;
02083 case DENOMINATOR :
02084     t[2] &= ~DIRTYSYMFLAG;
02085     pden[nden++] = t;
02086     pncnco[nncnco++] = t;
02087     break;
02088 case INDEX :
02089     t += 2;
02090     do {
02091         if ( *t == 0 || *t == AM.vectorzero ) goto NormZero;
02092         if ( *t > 0 && *t < AM.OffsetIndex ) {
02093             lnum[0] = *t++;
02094             nnum = 1;
02095             ncoef = REDLENG(ncoef);
02096             if ( Mully(BHEAD( UWORD *)n_coef, &ncoef, (UWORD *)lnum, nnum) )
02097                 goto FromNorm;
02098             ncoef = INCLENG(ncoef);
02099         }
02100         else if ( *t == NOINDEX ) t++;
02101         else pind[nind++] = *t++;
02102     } while ( t < r );
02103     break;
02104 case SUBEXPRESSION :
02105     if ( t[3] == 0 ) break;
02106 case EXPRESSION :
02107     goto RegEnd;
02108 case ROOTFUNCTION :
02109 /*
02110     Tries to take the n-th root inside the rationals
02111     If this is not possible, it clears all flags and
02112     hence tries no more.
02113     Notation:
02114     root_(power(=integer),(rational)number)
02115 */
02116     { WORD nc;
02117         if ( t[2] == 0 ) goto defaultcase;
02118         if ( t[FUNHEAD] != -SNUMBER || t[FUNHEAD+1] < 0 ) goto defaultcase;

```



```

02119         if ( t[FUNHEAD+2] == -SNUMBER ) {
02120             if ( t[FUNHEAD+1] == 0 && t[FUNHEAD+3] == 0 ) goto NormZZ;
02121             if ( t[FUNHEAD+1] == 0 ) break;
02122             if ( t[FUNHEAD+3] < 0 ) {
02123                 AT.WorkPointer[0] = -t[FUNHEAD+3];
02124                 nc = -1;
02125             }
02126             else {
02127                 AT.WorkPointer[0] = t[FUNHEAD+3];
02128                 nc = 1;
02129             }
02130             AT.WorkPointer[1] = 1;
02131         }
02132         else if ( t[FUNHEAD+2] == t[1]-FUNHEAD-2
02133             && t[FUNHEAD+2] == t[FUNHEAD+2+ARGHEAD]+ARGHEAD
02134             && ABS(t[t[1]-1]) == t[FUNHEAD+2+ARGHEAD] - 1 ) {
02135             WORD *r1, *r2;
02136             if ( t[FUNHEAD+1] == 0 ) break;
02137             i = t[t[1]-1]; r1 = t + FUNHEAD+ARGHEAD+3;
02138             nc = REDLENG(i);
02139             i = ABS(i) - 1;
02140             r2 = AT.WorkPointer;
02141             while ( --i >= 0 ) *r2++ = *r1++;
02142         }
02143         else goto defaultcase;
02144         if ( TakeRatRoot((UWORD *)AT.WorkPointer,&nc,t[FUNHEAD+1]) ) {
02145             t[2] = 0;
02146             goto defaultcase;
02147         }
02148         ncoef = REDLENG(ncoef);
02149         if ( Mully(BHEAD (UWORD *)n_coef,&ncoef,(UWORD *)AT.WorkPointer,nc) )
02150             goto FromNorm;
02151         if ( nc < 0 ) nc = -nc;
02152         if ( Divvy(BHEAD (UWORD *)n_coef,&ncoef,(UWORD *) (AT.WorkPointer+nc),nc) )
02153             goto FromNorm;
02154         ncoef = INCLENG(ncoef);
02155     }
02156     break;
02157 case RANDOMFUNCTION :
02158     {
02159         WORD nnc, nc, nca, nr;
02160         UWORD xx;
02161         /*
02162         Needs one positive integer argument.
02163         returns (wranf()%argument)+1.
02164         We may call wranf several times to paste UWORDS together
02165         when we need long numbers.
02166         We make little errors when taking the % operator
02167         (not 100% uniform). We correct for that by redoing the
02168         calculation in the (unlikely) case that we are in leftover area
02169         */
02170         if ( t[1] == FUNHEAD ) goto defaultcase;
02171         if ( t[1] == FUNHEAD+2 && t[FUNHEAD] == -SNUMBER &&
02172             t[FUNHEAD+1] > 0 ) {
02173             if ( t[FUNHEAD+1] == 1 ) break;
02174             redoshort:
02175             ((UWORD *)AT.WorkPointer)[0] = wranf(BHEAD0);
02176             ((UWORD *)AT.WorkPointer)[1] = wranf(BHEAD0);
02177             nr = 2;
02178             if ( ((UWORD *)AT.WorkPointer)[1] == 0 ) {
02179                 nr = 1;
02180                 if ( ((UWORD *)AT.WorkPointer)[0] == 0 ) {
02181                     nr = 0;
02182                 }
02183             }
02184             xx = (UWORD)(t[FUNHEAD+1]);
02185             if ( nr ) {
02186                 DivLong((UWORD *)AT.WorkPointer,nr
02187                     ,&xx,1
02188                     ,((UWORD *)AT.WorkPointer)+4,&nnc
02189                     ,((UWORD *)AT.WorkPointer)+2,&nc);
02190                 ((UWORD *)AT.WorkPointer)[4] = 0;
02191                 ((UWORD *)AT.WorkPointer)[5] = 0;
02192                 ((UWORD *)AT.WorkPointer)[6] = 1;
02193                 DivLong((UWORD *)AT.WorkPointer+4,3
02194                     ,&xx,1
02195                     ,((UWORD *)AT.WorkPointer)+9,&nnc
02196                     ,((UWORD *)AT.WorkPointer)+7,&nca);
02197                 AddLong((UWORD *)AT.WorkPointer+4,3
02198                     ,((UWORD *)AT.WorkPointer)+7,-nca
02199                     ,((UWORD *)AT.WorkPointer)+9,&nnc);
02200                 if ( BigLong((UWORD *)AT.WorkPointer,nr
02201                     ,((UWORD *)AT.WorkPointer)+9,nnc) >= 0 ) goto redoshort;
02202             }
02203             else nc = 0;
02204             if ( nc == 0 ) {
02205                 AT.WorkPointer[2] = (WORD)xx;

```

```

02206         nc = 1;
02207     }
02208     ncoef = REDLENG(ncoef);
02209     if ( Mully(BHEAD (UWORD *)n_coef,&ncoef, ((UWORD *) (AT.WorkPointer))+2,nc) )
02210         goto FromNorm;
02211     ncoef = INCLENG(ncoef);
02212 }
02213 else if ( t[FUNHEAD] > 0 && t[1] == t[FUNHEAD]+FUNHEAD
02214 && ABS(t[t[1]-1]) == t[FUNHEAD]-1-ARGHEAD && t[t[1]-1] > 0 ) {
02215     WORD nna, nnb, nni, nnb2, nnb2a;
02216     UWORD *nnt;
02217     nna = t[t[1]-1];
02218     nnb2 = nna-1;
02219     nnb = nnb2/2;
02220     nnt = (UWORD *) (t+t[1]-1-nnb); /* start of denominator */
02221     if ( *nnt != 1 ) goto defaultcase;
02222     for ( nni = 1; nni < nnb; nni++ ) {
02223         if ( nnt[nni] != 0 ) goto defaultcase;
02224     }
02225     nnt = (UWORD *) (t + FUNHEAD + ARGHEAD + 1);
02226
02227     for ( nni = 0; nni < nnb2; nni++ ) {
02228         ((UWORD *)AT.WorkPointer)[nni] = wranf(BHEAD0);
02229     }
02230     nnb2a = nnb2;
02231     while ( nnb2a > 0 && ((UWORD *)AT.WorkPointer)[nnb2a-1] == 0 ) nnb2a--;
02232     if ( nnb2a > 0 ) {
02233         DivLong((UWORD *)AT.WorkPointer,nnb2a
02234             ,nnt,nnb
02235             ,((UWORD *)AT.WorkPointer)+2*nnb2,&nnc
02236             ,((UWORD *)AT.WorkPointer)+nnb2,&nc);
02237         for ( nni = 0; nni < nnb2; nni++ ) {
02238             ((UWORD *)AT.WorkPointer)[nni+2*nnb2] = 0;
02239         }
02240         ((UWORD *)AT.WorkPointer)[3*nnb2] = 1;
02241         DivLong((UWORD *)AT.WorkPointer+2*nnb2,nnb2+1
02242             ,nnt,nnb
02243             ,((UWORD *)AT.WorkPointer)+4*nnb2+1,&nnc
02244             ,((UWORD *)AT.WorkPointer)+3*nnb2+1,&nca);
02245         AddLong((UWORD *)AT.WorkPointer+2*nnb2,nnb2+1
02246             ,((UWORD *)AT.WorkPointer)+3*nnb2+1,-nca
02247             ,((UWORD *)AT.WorkPointer)+4*nnb2+1,&nnc);
02248         if ( BigLong((UWORD *)AT.WorkPointer,nnb2a
02249             ,((UWORD *)AT.WorkPointer)+4*nnb2+1,nnc) >= 0 ) goto redoshort;
02250     }
02251     else nc = 0;
02252     if ( nc == 0 ) {
02253         for ( nni = 0; nni < nnb; nni++ ) {
02254             ((UWORD *)AT.WorkPointer)[nnb2+nni] = nnt[nni];
02255         }
02256         nc = nnb;
02257     }
02258     ncoef = REDLENG(ncoef);
02259     if ( Mully(BHEAD (UWORD *)n_coef,&ncoef, ((UWORD *) (AT.WorkPointer))+nnb2,nc) )
02260         goto FromNorm;
02261     ncoef = INCLENG(ncoef);
02262 }
02263 else goto defaultcase;
02264 }
02265 break;
02266 case RANPERM :
02267     if ( *t == RANPERM && t[1] > FUNHEAD && t[FUNHEAD] <= -FUNCTION ) {
02268         WORD **pwork;
02269         WORD *mm, *ww, *ow = AT.WorkPointer;
02270         WORD *Array, *targ, *argstop, narg = 0, itot;
02271         int ie;
02272         argstop = t+t[1];
02273         targ = t+FUNHEAD+1;
02274         while ( targ < argstop ) {
02275             narg++; NEXTARG(targ);
02276         }
02277         WantAddPointers(narg);
02278         pwork = AT.pWorkSpace + AT.pWorkPointer;
02279         targ = t+FUNHEAD+1; narg = 0;
02280         while ( targ < argstop ) {
02281             pwork[narg++] = targ;
02282             NEXTARG(targ);
02283         }
02284         /*
02285         Make a random permutation of the numbers 0,...,narg-1
02286         The following code works also for narg == 0 and narg == 1
02287         */
02288         ow = AT.WorkPointer;
02289         Array = AT.WorkPointer;
02290         AT.WorkPointer += narg;
02291         for ( i = 0; i < narg; i++ ) Array[i] = i;
02292         for ( i = 2; i <= narg; i++ ) {

```

```

02293         itot = (WORD)(iranf(BHEAD i));
02294         for ( j = 0; j < itot; j++ ) CYCLE1(WORD,Array,i)
02295     }
02296     mm = AT.WorkPointer;
02297     *mm++ = -t[FUNHEAD];
02298     *mm++ = t[1] - 1;
02299     for ( ie = 2; ie < FUNHEAD; ie++ ) *mm++ = t[ie];
02300     for ( i = 0; i < narg; i++ ) {
02301         ww = pwork[Array[i]];
02302         CopyArg(mm,ww);
02303     }
02304     mm = AT.WorkPointer; t++; ww = t;
02305     i = mm[1]; NCOPY(ww,mm,i)
02306     AT.WorkPointer = ow;
02307     goto TryAgain;
02308 }
02309 pnco[nnco++] = t;
02310 break;
02311 case PUTFIRST : /* First argument should be a function, second a number */
02312     if ( ( t[2] & DIRTYFLAG ) != 0 && t[FUNHEAD] <= -FUNCTION
02313         && t[FUNHEAD+1] == -SNUMBER && t[FUNHEAD+2] > 0 ) {
02314         WORD *rr = t+t[1], *mm = t+FUNHEAD+3, *tt, *ttl, *tt2, num = 0;
02315         /*
02316             now count the arguments. If not enough: no action.
02317         */
02318         while ( mm < rr ) { num++; NEXTARG(mm); }
02319         if ( num < t[FUNHEAD+2] ) { pnco[nnco++] = t; break; }
02320         /* Replace "putfirst_" with the resulting function code. mm goes to arg start. */
02321         *t = -t[FUNHEAD]; mm = t+FUNHEAD+3;
02322         /* Set i to the arg number we are putting first, then move mm to its start. */
02323         i = t[FUNHEAD+2];
02324         while ( --i > 0 ) { NEXTARG(mm); }
02325         /* Keep a pointer to the arguments trailing the selected: */
02326         WORD *argTail = mm;
02327         NEXTARG(argTail);
02328         tt = TermMalloc("Select_"); /* Move selected out of the way into tmp space */
02329         ttl = tt;
02330         /* Normal argument: */
02331         if ( *mm > 0 ) {
02332             for ( i = 0; i < *mm; i++ ) *ttl++ = mm[i];
02333         }
02334         /* Fast-notation function: single word */
02335         else if ( *mm <= -FUNCTION ) { *ttl++ = *mm; }
02336         /* Fast-notation symbol, number etc: two words */
02337         else { *ttl++ = mm[0]; *ttl++ = mm[1]; }
02338         /* Put tt2 at the start of the original arguments */
02339         tt2 = t+FUNHEAD+3;
02340         /* Copy leading original arguments after the new first, in the tmp space. */
02341         while ( tt2 < mm ) *ttl++ = *tt2++;
02342         /* i contains the size so far. ttl goes to the start of the tmp space.
02343            tt2 to the final argument location (we overwrite "putfirst_") */
02344         i = ttl-tt; ttl = tt; tt2 = t+FUNHEAD;
02345         /* Copy everything so far to its final place */
02346         NCOPY(tt2,ttl,i);
02347         /* We are finished with the tmp space */
02348         TermFree(tt,"Select_");
02349         /* Now copy the trailing args. Use the stored pointer, *mm has been edited
02350            during the copy to tt2 above! NEXTARG(mm) would produce nonsense. */
02351         while ( argTail < rr ) *tt2++ = *argTail++;
02352         /* Set the size of all function args */
02353         t[1] = tt2 - t;
02354         /* Now copy the rest of the term, and set the final term size */
02355         rr = term + *term;
02356         while ( argTail < rr ) *tt2++ = *argTail++;
02357         *term = tt2-term;
02358         goto Restart;
02359     }
02360     else pnco[nnco++] = t;
02361     break;
02362 #ifdef WITHFLOAT
02363     case FLOATFUN :
02364         /*
02365             If it is a proper float_ we give it special treatment.
02366             If it is not proper, we treat it as a regular commuting function.
02367         */
02368         if ( withfloat == 0 ) {
02369             if ( TestFloat(t) == 0 ) goto defaultcase;
02370             firstfloat = t;
02371             withfloat = 1;
02372         }
02373         else { /* There are now at least two float_'s. Time for action. */
02374             if ( TestFloat(t) == 0 ) goto defaultcase;
02375             if ( withfloat == 1 ) UnpackFloat(aux4,firstfloat);
02376             withfloat++;
02377             UnpackFloat(aux5,t);
02378             mpf_mul(aux4,aux4,aux5);
02379         }

```

```

02380             break;
02381 #endif
02382         case INTFUNCTION :
02383 /*
02384         Can be resolved if the first argument is a number
02385         and the second argument either doesn't exist or has
02386         the value +1, 0, -1
02387         +1 : rounding up
02388         0  : rounding towards zero
02389         -1 : rounding down (same as no argument)
02390 */
02391         if ( t[1] <= FUNHEAD ) break;
02392         {
02393             WORD *rr, den, num;
02394             to = t + FUNHEAD;
02395             if ( *to > 0 ) {
02396                 if ( *to == ARGHEAD ) goto NormZero;
02397                 rr = to + *to;
02398                 i = rr[-1];
02399                 j = ABS(i);
02400                 if ( to[ARGHEAD] != j+1 ) goto NoInteg;
02401                 if ( rr >= r ) k = -1;
02402                 else if ( *rr == ARGHEAD ) { k = 0; rr += ARGHEAD; }
02403                 else if ( *rr == -SNUMBER ) { k = rr[1]; rr += 2; }
02404                 else goto NoInteg;
02405                 if ( rr != r ) goto NoInteg;
02406                 if ( k > 1 || k < -1 ) goto NoInteg;
02407                 to += ARGHEAD+1;
02408                 j = (j-1) >> 1;
02409                 i = ( i < 0 ) ? -j: j;
02410                 UnPack((UWORD *)to,i,&den,&num);
02411 /*
02412                 Potentially the use of NoScrat2 is unsafe.
02413                 It makes the routine not reentrant, but because it is
02414                 used only locally and because we only call the
02415                 low level routines DivLong and AddLong which never
02416                 make calls involving Normalize, things are OK after all
02417 */
02418                 if ( AN.NoScrat2 == 0 ) {
02419                     AN.NoScrat2 = (UWORD *)Malloc1((AM.MaxTal+2)*sizeof(UWORD),"Normalize");
02420                 }
02421                 if ( DivLong((UWORD *)to,num,(UWORD *) (to+j),den
02422                     ,(UWORD *)AT.WorkPointer,&num,AN.NoScrat2,&den) ) goto FromNorm;
02423                 if ( k < 0 && den < 0 ) {
02424                     *AN.NoScrat2 = 1;
02425                     den = -1;
02426                     if ( AddLong((UWORD *)AT.WorkPointer,num
02427                         ,AN.NoScrat2,den,(UWORD *)AT.WorkPointer,&num) )
02428                         goto FromNorm;
02429                 }
02430                 else if ( k > 0 && den > 0 ) {
02431                     *AN.NoScrat2 = 1;
02432                     den = 1;
02433                     if ( AddLong((UWORD *)AT.WorkPointer,num,
02434                         AN.NoScrat2,den,(UWORD *)AT.WorkPointer,&num) )
02435                         goto FromNorm;
02436                 }
02437             }
02438             else if ( *to == -SNUMBER ) { /* No rounding needed */
02439                 if ( to[1] < 0 ) { *AT.WorkPointer = -to[1]; num = -1; }
02440                 else if ( to[1] == 0 ) goto NormZero;
02441                 else { *AT.WorkPointer = to[1]; num = 1; }
02442             }
02443             else goto NoInteg;
02444             if ( num == 0 ) goto NormZero;
02445             ncoef = REDLENG(ncoef);
02446             if ( Mully(BHEAD (UWORD *)n_coef,&ncoef,(UWORD *)AT.WorkPointer,num) )
02447                 goto FromNorm;
02448             ncoef = INCLENG(ncoef);
02449             break;
02450         }
02451     }
02452 NoInteg;;
02453 /*
02454     Fall through if it cannot be resolved
02455 */
02456     default :
02457 defaultcase:;
02458     if ( *t < FUNCTION ) {
02459         MLOCK(ErrorMessageLock);
02460         MesPrint("Illegal code in Norm");
02461 #ifdef DEBUGON
02462     {
02463         UBYTE OutBuf[140];
02464         AO.OutFill = AO.OutputLine = OutBuf;
02465         t = term;
02466         AO.OutSkip = 3;

```

```

02467         FiniLine();
02468         i = *t;
02469         while ( --i >= 0 ) {
02470             TalToLine((UWORD) (*t++));
02471             TokenToLine((UBYTE *) " ");
02472         }
02473         AO.OutSkip = 0;
02474         FiniLine();
02475     }
02476 #endif
02477     MUNLOCK(ErrorMessageLock);
02478     goto NormMin;
02479 }
02480 if ( *t == REPLACEMENT ) {
02481     if ( AR.Eside != LHSIDE ) ReplaceVeto--;
02482     pcom[ncom++] = t;
02483     break;
02484 }
02485 /*
02486 if ( *t == AM.termfunnum && t[1] == FUNHEAD+2
02487 && t[FUNHEAD] == -DOLLAREXPRESSION ) termflag++;
02488 */
02489 if ( *t == DUMMYFUN || *t == DUMMYTEN ) {}
02490 else {
02491     if ( *t < (FUNCTION + WILDOFFSET) ) {
02492         if ( ( ( functions[*t-FUNCTION].maxnumargs > 0 )
02493             || ( functions[*t-FUNCTION].minnumargs > 0 ) )
02494             && ( t[2] & DIRTYFLAG ) != 0 ) ) {
02495             /*
02496                 Number of arguments is bounded. And we have not checked.
02497             */
02498             WORD *ta = t + FUNHEAD, *tb = t + t[1];
02499             int numarg = 0;
02500             while ( ta < tb ) { numarg++; NEXTARG(ta) }
02501             if ( ( functions[*t-FUNCTION].maxnumargs > 0 )
02502                 && ( numarg >= functions[*t-FUNCTION].maxnumargs ) )
02503                 goto NormZero;
02504             if ( ( functions[*t-FUNCTION].minnumargs > 0 )
02505                 && ( numarg < functions[*t-FUNCTION].minnumargs ) )
02506                 goto NormZero;
02507         }
02508         doflags:
02509         if ( ( t[2] & DIRTYFLAG ) != 0 ) && ( functions[*t-FUNCTION].tabl == 0 ) ) {
02510             t[2] &= ~DIRTYFLAG;
02511             t[2] |= DIRTYSYMLFLAG;
02512         }
02513         if ( functions[*t-FUNCTION].commute ) { pnco[nnco++] = t; }
02514         else { pcom[ncom++] = t; }
02515     }
02516     else {
02517         if ( ( t[2] & DIRTYFLAG ) != 0 ) && ( functions[*t-FUNCTION-WILDOFFSET].tabl
02518             == 0 ) ) {
02519             t[2] &= ~DIRTYFLAG;
02520             t[2] |= DIRTYSYMLFLAG;
02521         }
02522         if ( functions[*t-FUNCTION-WILDOFFSET].commute ) {
02523             pnco[nnco++] = t;
02524         }
02525         else { pcom[ncom++] = t; }
02526     }
02527 }
02528 /* Now hunt for contractible indices */
02529
02530 if ( ( *t < (FUNCTION + WILDOFFSET)
02531 && functions[*t-FUNCTION].spec >= TENSORFUNCTION ) || (
02532 *t >= (FUNCTION + WILDOFFSET)
02533 && functions[*t-FUNCTION-WILDOFFSET].spec >= TENSORFUNCTION ) ) {
02534     if ( *t >= GAMMA && *t <= GAMMASEVEN ) t++;
02535     t += FUNHEAD;
02536     while ( t < r ) {
02537         if ( *t == AM.vectorzero ) goto NormZero;
02538         if ( *t >= AM.OffsetIndex && ( *t >= AM.DumInd
02539             || ( *t < AM.WilInd && indices[*t-AM.OffsetIndex].dimension ) ) ) {
02540             pcon[ncon++] = t;
02541         }
02542         else if ( *t == FUNNYWILD ) { t++; }
02543         t++;
02544     }
02545 }
02546 else {
02547     t += FUNHEAD;
02548     while ( t < r ) {
02549         if ( *t > 0 ) {
02550             /*
02551                 Here we should worry about a recursion
02552                 A problem is the possibility of a construct

```

```

02553             like f(mu+nu)
02554  */
02555             t += *t;
02556         }
02557         else if ( *t <= -FUNCTION ) t++;
02558         else if ( *t == -INDEX ) {
02559             if ( t[1] >= AM.OffsetIndex &&
02560                 ( t[1] >= AM.DumInd || ( t[1] < AM.WilInd
02561                     && indices[t[1]-AM.OffsetIndex].dimension ) ) )
02562                 pcon[ncon++] = t+1;
02563             t += 2;
02564         }
02565         else if ( *t == -SYMBOL ) {
02566             if ( t[1] >= MAXPOWER && t[1] < 2*MAXPOWER ) {
02567                 *t = -SNUMBER;
02568                 t[1] -= MAXPOWER;
02569             }
02570             else if ( t[1] < -MAXPOWER && t[1] > -2*MAXPOWER ) {
02571                 *t = -SNUMBER;
02572                 t[1] += MAXPOWER;
02573             }
02574             else t += 2;
02575         }
02576         else t += 2;
02577     }
02578     }
02579     break;
02580 }
02581 t = r;
02582 TryAgain:;
02583 } while ( t < m );
02584 if ( ANsc ) {
02585     AN.cTerm = ANsc;
02586     r = t = ANsr; m = ANsm;
02587     ANsc = ANsm = ANsr = 0;
02588     goto conscan;
02589 }
02590 /*
02591 #] First scan :
02592 #[ Easy denominators :
02593
02594 Easy denominators are denominators that can be replaced by
02595 negative powers of individual subterms. This may add to all
02596 our sublists.
02597
02598 */
02599 if ( nden ) {
02600     for ( k = 0, i = 0; i < nden; i++ ) {
02601         t = pden[i];
02602         if ( ( t[2] & DIRTYFLAG ) == 0 ) continue;
02603         r = t + t[1]; m = t + FUNHEAD;
02604         if ( m >= r ) {
02605             for ( j = i+1; j < nden; j++ ) pden[j-1] = pden[j];
02606             nden--;
02607             for ( j = 0; j < nnco; j++ ) if ( pnco[j] == t ) break;
02608             for ( j++; j < nnco; j++ ) pnco[j-1] = pnco[j];
02609             nnco--;
02610             i--;
02611         }
02612         else {
02613             NEXTARG(m);
02614             if ( m >= r ) continue;
02615         }
02616         /*
02617         We have more than one argument. Split the function.
02618         */
02619         if ( k == 0 ) {
02620             k = 1; to = termout; from = term;
02621         }
02622         while ( from < t ) *to++ = *from++;
02623         m = t + FUNHEAD;
02624         while ( m < r ) {
02625             stop = to;
02626             *to++ = DENOMINATOR;
02627             for ( j = 1; j < FUNHEAD; j++ ) *to++ = 0;
02628             if ( *m < -FUNCTION ) *to++ = *m++;
02629             else if ( *m < 0 ) { *to++ = *m++; *to++ = *m++; }
02630             else {
02631                 j = *m; while ( --j >= 0 ) *to++ = *m++;
02632             }
02633             stop[1] = WORDDIF(to,stop);
02634         }
02635         from = r;
02636         if ( i == nden - 1 ) {
02637             stop = term + *term;
02638             while ( from < stop ) *to++ = *from++;
02639             i = *termout = WORDDIF(to,termout);
02640             to = term; from = termout;
02641         }
02642     }

```

```

02640         while ( --i >= 0 ) *to++ = *from++;
02641         goto Restart;
02642     }
02643 }
02644 }
02645 for ( i = 0; i < nden; i++ ) {
02646     t = pden[i];
02647     if ( ( t[2] & DIRTYFLAG ) == 0 ) continue;
02648     t[2] = 0;
02649     if ( t[FUNHEAD] == -SYMBOL ) {
02650         WORD change;
02651         t += FUNHEAD+1;
02652         change = ExtraSymbol(*t,-1,nsym,ppsym,&ncoef);
02653         nsym += change;
02654         ppsym += change * 2;
02655         goto DropDen;
02656     }
02657     else if ( t[FUNHEAD] == -SNUMBER ) {
02658         t += FUNHEAD+1;
02659         if ( *t == 0 ) goto NormInf;
02660         if ( *t < 0 ) { *AT.WorkPointer = -*t; j = -1; }
02661         else { *AT.WorkPointer = *t; j = 1; }
02662         ncoef = REDLENG(ncoef);
02663         if ( Divvy(BHEAD (UWORD *)n_coef,&ncoef, (UWORD *)AT.WorkPointer,j) )
02664             goto FromNorm;
02665         ncoef = INCLENG(ncoef);
02666         goto DropDen;
02667     }
02668     else if ( t[FUNHEAD] == ARGHEAD ) goto NormInf;
02669     else if ( t[FUNHEAD] > 0 && t[FUNHEAD+ARGHEAD] ==
02670         t[FUNHEAD]-ARGHEAD ) {
02671         /* Only one term */
02672         r = t + t[1] - 1;
02673         t += FUNHEAD + ARGHEAD + 1;
02674         j = *r;
02675         m = r - ABS(*r) + 1;
02676         if ( j != 3 || ( (*m != 1) || ( m[1] != 1 ) ) ) {
02677             ncoef = REDLENG(ncoef);
02678             if ( DivRat(BHEAD (UWORD *)n_coef,ncoef, (UWORD *)m,REDLENG(j), (UWORD
02679 *)n_coef,&ncoef) ) goto FromNorm;
02680             ncoef = INCLENG(ncoef);
02681             j = ABS(j) - 3;
02682             t[-FUNHEAD-ARGHEAD] -= j;
02683             t[-ARGHEAD-1] -= j;
02684             t[-1] -= j;
02685             m[0] = m[1] = 1;
02686             m[2] = 3;
02687         }
02688         while ( t < m ) {
02689             r = t + t[1];
02690             if ( *t == SYMBOL || *t == DOTPRODUCT ) {
02691                 k = t[1];
02692                 pden[i][1] -= k;
02693                 pden[i][FUNHEAD] -= k;
02694                 pden[i][FUNHEAD+ARGHEAD] -= k;
02695                 m -= k;
02696                 stop = m + 3;
02697                 tt = to = t;
02698                 from = r;
02699                 if ( *t == SYMBOL ) {
02700                     t += 2;
02701                     while ( t < r ) {
02702                         WORD change;
02703                         change = ExtraSymbol(*t,-t[1],nsym,ppsym,&ncoef);
02704                         nsym += change;
02705                         ppsym += change * 2;
02706                         t += 2;
02707                     }
02708                 }
02709                 else {
02710                     t += 2;
02711                     while ( t < r ) {
02712                         *ppdot++ = *t++;
02713                         *ppdot++ = *t++;
02714                         *ppdot++ = -*t++;
02715                         ndot++;
02716                     }
02717                 }
02718                 while ( to < stop ) *to++ = *from++;
02719                 r = tt;
02720             }
02721             #ifdef WITHFLOAT
02722             else if ( *t == FLOATFUN && TestFloat(t) ) {
02723                 k = t[1];
02724                 pden[i][1] -= k;
02725                 pden[i][FUNHEAD] -= k;
02726                 pden[i][FUNHEAD+ARGHEAD] -= k;

```

```

02726         UnpackFloat(aux5,t);
02727         if ( withfloat == 0 ) {
02728             mpf_ui_div(aux4,1,aux5);
02729             withfloat = 2;
02730         }
02731         else {
02732             if ( withfloat == 1 ) UnpackFloat(aux4,firstfloat);
02733             mpf_div(aux4,aux4,aux5);
02734             withfloat++;
02735         }
02736         tt = to = t;
02737         while ( r < m ) *to++ = *r++;
02738         *to++ = 1; *to++ = 1; *to++ = 3;
02739         if ( ncoef < 0 ) t[-1] = -t[-1];
02740         m -= k;
02741         stop = m + 3;
02742         r = tt;
02743     }
02744 #endif
02745     t = r;
02746 }
02747 if ( pden[i][1] == 4+FUNHEAD+ARGHEAD ) {
02748 DropDen:
02749     for ( j = 0; j < nnco; j++ ) {
02750         if ( pden[i] == pnco[j] ) {
02751             --nnco;
02752             while ( j < nnco ) {
02753                 pnco[j] = pnco[j+1];
02754                 j++;
02755             }
02756             break;
02757         }
02758     }
02759     pden[i--] = pden[--nden];
02760 }
02761 }
02762 }
02763 }
02764 /*
02765 #] Easy denominators :
02766 #[ Index Contractions :
02767 */
02768 if ( ndel ) {
02769     t = pdel;
02770     for ( i = 0; i < ndel; i += 2 ) {
02771         if ( t[0] == t[1] ) {
02772             if ( t[0] == EMPTYINDEX ) {}
02773             else if ( *t < AM.OffsetIndex ) {
02774                 k = AC.FixIndices[*t];
02775                 if ( k < 0 ) { j = -1; k = -k; }
02776                 else if ( k > 0 ) j = 1;
02777                 else goto NormZero;
02778                 goto WithFix;
02779             }
02780             else if ( *t >= AM.DumInd ) {
02781                 k = AC.lDefDim;
02782                 if ( k ) goto docontract;
02783             }
02784             else if ( *t >= AM.WilInd ) {
02785                 k = indices[*t-AM.OffsetIndex-WILDOFFSET].dimension;
02786                 if ( k ) goto docontract;
02787             }
02788             else if ( ( k = indices[*t-AM.OffsetIndex].dimension ) != 0 ) {
02789 docontract:
02790                 if ( k > 0 ) {
02791                     j = 1;
02792                     shortnum = k;
02793                     ncoef = REDLENG(ncoef);
02794                     if ( Mully(BHEAD (UWORD *)n_coef,&ncoef, (UWORD *)(&shortnum),j) )
02795                         goto FromNorm;
02796                     ncoef = INCLENG(ncoef);
02797                 }
02798                 else {
02799                     WORD change;
02800                     change = ExtraSymbol((WORD) (-k), (WORD) 1, nsym, ppsym, &ncoef);
02801                     nsym += change;
02802                     ppsym += change * 2;
02803                 }
02804                 t[1] = pdel[ndel-1];
02805                 t[0] = pdel[ndel-2];
02806 HaveCon:
02807                 ndel -= 2;
02808                 i -= 2;
02809             }
02810         }
02811         else {
02812             if ( *t < AM.OffsetIndex && t[1] < AM.OffsetIndex ) goto NormZero;

```



```

02813         j = *t - AM.OffsetIndex;
02814         if ( j >= 0 && ( ( *t >= AM.DumInd && AC.lDefDim )
02815             || ( *t < AM.WilInd && indices[j].dimension ) ) ) {
02816             for ( j = i + 2, m = pdel+j; j < ndel; j += 2, m += 2 ) {
02817                 if ( *t == *m ) {
02818                     *t = m[1];
02819                     *m++ = pdel[ndel-2];
02820                     *m = pdel[ndel-1];
02821                     goto HaveCon;
02822                 }
02823                 else if ( *t == m[1] ) {
02824                     *t = *m;
02825                     *m++ = pdel[ndel-2];
02826                     *m = pdel[ndel-1];
02827                     goto HaveCon;
02828                 }
02829             }
02830         }
02831         j = t[1]-AM.OffsetIndex;
02832         if ( j >= 0 && ( ( t[1] >= AM.DumInd && AC.lDefDim )
02833             || ( t[1] < AM.WilInd && indices[j].dimension ) ) ) {
02834             for ( j = i + 2, m = pdel+j; j < ndel; j += 2, m += 2 ) {
02835                 if ( t[1] == *m ) {
02836                     t[1] = m[1];
02837                     *m++ = pdel[ndel-2];
02838                     *m = pdel[ndel-1];
02839                     goto HaveCon;
02840                 }
02841                 else if ( t[1] == m[1] ) {
02842                     t[1] = *m;
02843                     *m++ = pdel[ndel-2];
02844                     *m = pdel[ndel-1];
02845                     goto HaveCon;
02846                 }
02847             }
02848         }
02849         t += 2;
02850     }
02851 }
02852 if ( ndel > 0 ) {
02853     if ( nvec ) {
02854         t = pdel;
02855         for ( i = 0; i < ndel; i++ ) {
02856             if ( *t >= AM.OffsetIndex && ( ( *t >= AM.DumInd && AC.lDefDim ) ||
02857                 ( *t < AM.WilInd && indices[*t-AM.OffsetIndex].dimension ) ) ) {
02858                 r = pvec + 1;
02859                 for ( j = 1; j < nvec; j += 2 ) {
02860                     if ( *r == *t ) {
02861                         if ( i & 1 ) {
02862                             *r = t[-1];
02863                             *t-- = pdel[--ndel];
02864                             i -= 2;
02865                         }
02866                         else {
02867                             *r = t[1];
02868                             t[1] = pdel[--ndel];
02869                             i--;
02870                         }
02871                         *t-- = pdel[--ndel];
02872                         break;
02873                     }
02874                     r += 2;
02875                 }
02876             }
02877             t++;
02878         }
02879     }
02880     if ( ndel > 0 && ncon ) {
02881         t = pdel;
02882         for ( i = 0; i < ndel; i++ ) {
02883             if ( *t >= AM.OffsetIndex && ( ( *t >= AM.DumInd && AC.lDefDim ) ||
02884                 ( *t < AM.WilInd && indices[*t-AM.OffsetIndex].dimension ) ) ) {
02885                 for ( j = 0; j < ncon; j++ ) {
02886                     if ( *pcon[j] == *t ) {
02887                         if ( i & 1 ) {
02888                             *pcon[j] = t[-1];
02889                             *t-- = pdel[--ndel];
02890                             i -= 2;
02891                         }
02892                         else {
02893                             *pcon[j] = t[1];
02894                             t[1] = pdel[--ndel];
02895                             i--;
02896                         }
02897                         *t-- = pdel[--ndel];
02898                         didcontr++;
02899                         r = pcon[j];

```

```

02900         for ( j = 0; j < nnco; j++ ) {
02901             m = pnco[j];
02902             if ( r > m && r < m+m[1] ) {
02903                 m[2] |= DIRTYSYMFLAG;
02904                 break;
02905             }
02906         }
02907         for ( j = 0; j < ncom; j++ ) {
02908             m = pcom[j];
02909             if ( r > m && r < m+m[1] ) {
02910                 m[2] |= DIRTYSYMFLAG;
02911                 break;
02912             }
02913         }
02914         for ( j = 0; j < neps; j++ ) {
02915             m = peps[j];
02916             if ( r > m && r < m+m[1] ) {
02917                 m[2] |= DIRTYSYMFLAG;
02918                 break;
02919             }
02920         }
02921         break;
02922     }
02923 }
02924 }
02925     t++;
02926 }
02927 }
02928 }
02929 }
02930 if ( nvec ) {
02931     t = pvec + 1;
02932     for ( i = 3; i < nvec; i += 2 ) {
02933         k = *t - AM.OffsetIndex;
02934         if ( k >= 0 && ( ( *t > AM.DumInd && AC.lDefDim )
02935             || ( *t < AM.WilInd && indices[k].dimension ) ) ) {
02936             r = t + 2;
02937             for ( j = i; j < nvec; j += 2 ) {
02938                 if ( *r == *t ) { /* Another dotproduct */
02939                     *ppdot++ = t[-1];
02940                     *ppdot++ = r[-1];
02941                     *ppdot++ = 1;
02942                     ndot++;
02943                     *r-- = pvec[--nvec];
02944                     *r = pvec[--nvec];
02945                     *t-- = pvec[--nvec];
02946                     *t = pvec[--nvec];
02947                     i -= 2;
02948                     break;
02949                 }
02950                 r += 2;
02951             }
02952         }
02953         t += 2;
02954     }
02955     if ( nvec > 0 && ncon ) {
02956         t = pvec + 1;
02957         for ( i = 1; i < nvec; i += 2 ) {
02958             k = *t - AM.OffsetIndex;
02959             if ( k >= 0 && ( ( *t >= AM.DumInd && AC.lDefDim )
02960                 || ( *t < AM.WilInd && indices[k].dimension ) ) ) {
02961                 for ( j = 0; j < ncon; j++ ) {
02962                     if ( *pcon[j] == *t ) {
02963                         *pcon[j] = t[-1];
02964                         *t-- = pvec[--nvec];
02965                         *t = pvec[--nvec];
02966                         r = pcon[j];
02967                         pcon[j] = pcon[--ncon];
02968                         i -= 2;
02969                         for ( j = 0; j < nnco; j++ ) {
02970                             m = pnco[j];
02971                             if ( r > m && r < m+m[1] ) {
02972                                 m[2] |= DIRTYSYMFLAG;
02973                                 break;
02974                             }
02975                         }
02976                         for ( j = 0; j < ncom; j++ ) {
02977                             m = pcom[j];
02978                             if ( r > m && r < m+m[1] ) {
02979                                 m[2] |= DIRTYSYMFLAG;
02980                                 break;
02981                             }
02982                         }
02983                         for ( j = 0; j < neps; j++ ) {
02984                             m = peps[j];
02985                             if ( r > m && r < m+m[1] ) {
02986                                 m[2] |= DIRTYSYMFLAG;

```

```

02987                                     break;
02988                                     }
02989                                     }
02990                                     break;
02991                                     }
02992                                     }
02993                                     }
02994                                     t += 2;
02995                                     }
02996                                     }
02997                                     }
02998 /*
02999 #] Index Contractions :
03000 #[ NonCommuting Functions :
03001 */
03002 m = fillsetexp;
03003 if ( nnco ) {
03004     for ( i = 0; i < nnco; i++ ) {
03005         t = pnco[i];
03006         if ( ( *t >= (FUNCTION+WILDOFFSET)
03007             && functions[*t-FUNCTION-WILDOFFSET].spec <= 0 )
03008             || ( *t >= FUNCTION && *t < (FUNCTION + WILDOFFSET)
03009             && functions[*t-FUNCTION].spec <= 0 ) ) {
03010             DoRevert(t,m);
03011             if ( didcontr ) {
03012                 r = t + FUNHEAD;
03013                 t += t[1];
03014                 while ( r < t ) {
03015                     if ( *r == -INDEX && r[1] >= 0 && r[1] < AM.OffsetIndex ) {
03016                         *r = -SNUMBER;
03017                         didcontr--;
03018                         pnco[i][2] |= DIRTYSYMFLAG;
03019                     }
03020                     NEXTARG(r)
03021                 }
03022             }
03023         }
03024     }
03025
03026     /* First should come the code for function properties. */
03027
03028     /* First we test for symmetric properties and the DIRTYSYMFLAG */
03029
03030     for ( i = 0; i < nnco; i++ ) {
03031         t = pnco[i];
03032         if ( *t > 0 && ( t[2] & DIRTYSYMFLAG ) && *t != DOLLAREXPRESSION ) {
03033             l = 0; /* to make the compiler happy */
03034             if ( ( *t >= (FUNCTION+WILDOFFSET)
03035                 && ( l = functions[*t-FUNCTION-WILDOFFSET].symmetric ) > 0 )
03036                 || ( *t >= FUNCTION && *t < (FUNCTION + WILDOFFSET)
03037                 && ( l = functions[*t-FUNCTION].symmetric ) > 0 ) ) {
03038                 if ( *t >= (FUNCTION+WILDOFFSET) ) {
03039                     *t -= WILDOFFSET;
03040                     j = FullSymmetrize(BHEAD t,l);
03041                     *t += WILDOFFSET;
03042                 }
03043                 else j = FullSymmetrize(BHEAD t,l);
03044                 if ( (l & ~REVERSEORDER) == ANTISYMMETRIC ) {
03045                     if ( ( j & 2 ) != 0 ) goto NormZero;
03046                     if ( ( j & 1 ) != 0 ) ncoef = -ncoef;
03047                 }
03048             }
03049             else t[2] &= ~DIRTYSYMFLAG;
03050         }
03051     }
03052
03053     /* Non commuting functions are then tested for commutation
03054        rules. If needed their order is exchanged. */
03055
03056     k = nnco - 1;
03057     for ( i = 0; i < k; i++ ) {
03058         j = i;
03059         while ( Commute(pnco[j],pnco[j+1]) ) {
03060             t = pnco[j]; pnco[j] = pnco[j+1]; pnco[j+1] = t;
03061             l = j-1;
03062             while ( l >= 0 && Commute(pnco[l],pnco[l+1]) ) {
03063                 t = pnco[l]; pnco[l] = pnco[l+1]; pnco[l+1] = t;
03064                 l--;
03065             }
03066             if ( ++j >= k ) break;
03067         }
03068     }
03069
03070     /* Finally they are written to output. gamma matrices
03071        are bundled if possible */
03072
03073     for ( i = 0; i < nnco; i++ ) {

```

```

03074         t = pnco[i];
03075         if ( *t == IDFUNCTION ) AN.idfunctionflag = 1;
03076         if ( *t >= GAMMA && *t <= GAMMASEVEN ) {
03077             WORD gtype;
03078             to = m;
03079             *m++ = GAMMA;
03080             m++;
03081             FILLFUN(m)
03082             *m++ = stype = t[FUNHEAD];          /* type of string */
03083             j = 0;
03084             nnum = 0;
03085             do {
03086                 r = t + t[1];
03087                 if ( *t == GAMMAFIVE ) {
03088                     gtype = GAMMA5; t += FUNHEAD; goto onegammamatrix; }
03089                 else if ( *t == GAMMASIX ) {
03090                     gtype = GAMMA6; t += FUNHEAD; goto onegammamatrix; }
03091                 else if ( *t == GAMMASEVEN ) {
03092                     gtype = GAMMA7; t += FUNHEAD; goto onegammamatrix; }
03093                 t += FUNHEAD+1;
03094                 while ( t < r ) {
03095                     gtype = *t;
03096 onegammamatrix:
03097                     if ( gtype == GAMMA5 ) {
03098                         if ( j == GAMMA1 ) j = GAMMA5;
03099                         else if ( j == GAMMA5 ) j = GAMMA1;
03100                         else if ( j == GAMMA7 ) ncoef = -ncoef;
03101                         if ( nnum & 1 ) ncoef = -ncoef;
03102                     }
03103                     else if ( gtype == GAMMA6 || gtype == GAMMA7 ) {
03104                         if ( nnum & 1 ) {
03105                             if ( gtype == GAMMA6 ) gtype = GAMMA7;
03106                             else gtype = GAMMA6;
03107                         }
03108                         if ( j == GAMMA1 ) j = gtype;
03109                         else if ( j == GAMMA5 ) {
03110                             j = gtype;
03111                             if ( j == GAMMA7 ) ncoef = -ncoef;
03112                         }
03113                         else if ( j != gtype ) goto NormZero;
03114                         else {
03115                             shortnum = 2;
03116                             ncoef = REDLENG(ncoef);
03117                             if ( Mully(BHEAD (UWORD *)n_coef,&ncoef,(UWORD *)&shortnum,1) ) goto
FromNorm;
03118                             ncoef = INCLENG(ncoef);
03119                         }
03120                     }
03121                     else {
03122                         *m++ = gtype; nnum++;
03123                     }
03124                     t++;
03125                 }
03126             } while ( ( ++i < nnc ) && ( *(t = pnco[i]) >= GAMMA
&& *t <= GAMMASEVEN ) && ( t[FUNHEAD] == stype ) );
03127             i--;
03128             if ( j ) {
03129                 k = WORDDIF(m,to) - FUNHEAD-1;
03130                 r = m;
03131                 from = m++;
03132                 while ( --k >= 0 ) *from-- = *--r;
03133                 *from = j;
03134             }
03135             to[1] = WORDDIF(m,to);
03136         }
03137     }
03138     else if ( *t < 0 ) {
03139         *m++ = -*t; *m++ = FUNHEAD; *m++ = 0;
03140         FILLFUN3(m)
03141     }
03142     else {
03143         if ( ( t[2] & DIRTYFLAG ) == DIRTYFLAG
&& *t != REPLACEMENT && *t != DOLLAREXPRESSION
&& TestFunFlag(BHEAD t) ) ReplaceVeto = 1;
03144         k = t[1];
03145         NCOPY(m,t,k);
03146     }
03147 }
03148 }
03149 }
03150 }
03151 }
03152 }
03153 /*
03154 #] NonCommuting Functions :
03155 #[ Commuting Functions :
03156 */
03157 if ( ncom ) {
03158     for ( i = 0; i < ncom; i++ ) {
03159         t = pcom[i];

```

```

03160         if ( ( *t >= (FUNCTION+WILDOFFSET)
03161         && functions[*t-FUNCTION-WILDOFFSET].spec <= 0 )
03162         || ( *t >= FUNCTION && *t < (FUNCTION + WILDOFFSET)
03163         && functions[*t-FUNCTION].spec <= 0 ) ) {
03164             DoRevert(t,m);
03165             if ( didcontr ) {
03166                 r = t + FUNHEAD;
03167                 t += t[1];
03168                 while ( r < t ) {
03169                     if ( *r == -INDEX && r[1] >= 0 && r[1] < AM.OffsetIndex ) {
03170                         *r = -SNUMBER;
03171                         didcontr--;
03172                         pcom[i][2] |= DIRTYSYMFLAG;
03173                     }
03174                     NEXTARG(r)
03175                 }
03176             }
03177         }
03178     }
03179
03180     /* Now we test for symmetric properties and the DIRTYSYMFLAG */
03181
03182     for ( i = 0; i < ncom; i++ ) {
03183         t = pcom[i];
03184         if ( *t > 0 && ( t[2] & DIRTYSYMFLAG ) ) {
03185             l = 0; /* to make the compiler happy */
03186             if ( ( *t >= (FUNCTION+WILDOFFSET)
03187             && ( l = functions[*t-FUNCTION-WILDOFFSET].symmetric ) > 0 )
03188             || ( *t >= FUNCTION && *t < (FUNCTION + WILDOFFSET)
03189             && ( l = functions[*t-FUNCTION].symmetric ) > 0 ) ) {
03190                 if ( *t >= (FUNCTION+WILDOFFSET) ) {
03191                     *t -= WILDOFFSET;
03192                     j = FullSymmetrize(BHEAD t,l);
03193                     *t += WILDOFFSET;
03194                 }
03195                 else j = FullSymmetrize(BHEAD t,l);
03196                 if ( (l & ~REVERSEORDER) == ANTISYMMETRIC ) {
03197                     if ( ( j & 2 ) != 0 ) goto NormZero;
03198                     if ( ( j & 1 ) != 0 ) ncoef = -ncoef;
03199                 }
03200             }
03201             else t[2] &= ~DIRTYSYMFLAG;
03202         }
03203     }
03204     /*
03205     Sort the functions
03206     From a purists point of view this can be improved.
03207     There are slow and fast arguments and no conversions are
03208     taken into account here.
03209     */
03210     for ( i = 1; i < ncom; i++ ) {
03211         for ( j = i; j > 0; j-- ) {
03212             WORD jj,kk;
03213             jj = j-1;
03214             t = pcom[jj];
03215             r = pcom[j];
03216             if ( *t < 0 ) {
03217                 if ( *r < 0 ) { if ( *t >= *r ) goto NextI; }
03218                 else { if ( -*t <= *r ) goto NextI; }
03219                 goto jexch;
03220             }
03221             else if ( *r < 0 ) {
03222                 if ( *t < -*r ) goto NextI;
03223                 goto jexch;
03224             }
03225             else if ( *t != *r ) {
03226                 if ( *t < *r ) goto NextI;
03227                 t = pcom[j]; pcom[j] = pcom[jj]; pcom[jj] = t;
03228                 continue;
03229             }
03230             if ( AC.properorderflag ) {
03231                 if ( ( *t >= (FUNCTION+WILDOFFSET)
03232                 && functions[*t-FUNCTION-WILDOFFSET].spec >= TENSORFUNCTION )
03233                 || ( *t >= FUNCTION && *t < (FUNCTION + WILDOFFSET)
03234                 && functions[*t-FUNCTION].spec >= TENSORFUNCTION ) ) {}
03235                 else {
03236                     WORD *s1, *s2, *ss1, *ss2;
03237                     s1 = t+FUNHEAD; s2 = r+FUNHEAD;
03238                     ss1 = t + t[1]; ss2 = r + r[1];
03239                     while ( s1 < ss1 && s2 < ss2 ) {
03240                         k = CompArg(s1,s2);
03241                         if ( k > 0 ) goto jexch;
03242                         if ( k < 0 ) goto NextI;
03243                         NEXTARG(s1)
03244                         NEXTARG(s2)
03245                     }
03246                     if ( s1 < ss1 ) goto jexch;

```

```

03247         goto NextI;
03248     }
03249     k = t[1] - FUNHEAD;
03250     kk = r[1] - FUNHEAD;
03251     t += FUNHEAD;
03252     r += FUNHEAD;
03253     while ( k > 0 && kk > 0 ) {
03254         if ( *t < *r ) goto NextI;
03255         else if ( *t++ > *r++ ) goto jexch;
03256         k--; kk--;
03257     }
03258     if ( k > 0 ) goto jexch;
03259     goto NextI;
03260 }
03261 else
03262 {
03263     k = t[1] - FUNHEAD;
03264     kk = r[1] - FUNHEAD;
03265     t += FUNHEAD;
03266     r += FUNHEAD;
03267     while ( k > 0 && kk > 0 ) {
03268         if ( *t < *r ) goto NextI;
03269         else if ( *t++ > *r++ ) goto jexch;
03270         k--; kk--;
03271     }
03272     if ( k > 0 ) goto jexch;
03273     goto NextI;
03274 }
03275 }
03276 NextI;;
03277 }
03278 for ( i = 0; i < ncom; i++ ) {
03279     t = pcom[i];
03280     if ( *t == THETA || *t == THETA2 ) {
03281         if ( ( k = DoTheta(BHEAD t) ) == 0 ) goto NormZero;
03282         else if ( k < 0 ) {
03283             k = t[1];
03284             NCOPY(m,t,k);
03285         }
03286     }
03287     else if ( *t == DELTA2 || *t == DELTAP ) {
03288         if ( ( k = DoDelta(t) ) == 0 ) goto NormZero;
03289         else if ( k < 0 ) {
03290             k = t[1];
03291             NCOPY(m,t,k);
03292         }
03293     }
03294     else if ( *t == AR.PolyFunInv && AR.PolyFunType == 2 ) {
03295         /*
03296             If there are two arguments, exchange them, change the
03297             name of the function and go to dealing with PolyRatFun.
03298         */
03299         WORD *mm, *tt = t, numt = 0;
03300         tt += FUNHEAD;
03301         while ( tt < t+t[1] ) { numt++; NEXTARG(tt) }
03302         if ( numt == 2 ) {
03303             tt = t; mm = m; k = t[1];
03304             NCOPY(mm,tt,k)
03305             mm = m+FUNHEAD;
03306             NEXTARG(mm);
03307             tt = t+FUNHEAD;
03308             if ( *mm < 0 ) {
03309                 if ( *mm <= -FUNCTION ) { *tt++ = *mm++; }
03310                 else { *tt++ = *mm++; *tt++ = *mm++; }
03311             }
03312             else {
03313                 k = *mm; NCOPY(tt,mm,k)
03314             }
03315             mm = m+FUNHEAD;
03316             if ( *mm < 0 ) {
03317                 if ( *mm <= -FUNCTION ) { *tt++ = *mm++; }
03318                 else { *tt++ = *mm++; *tt++ = *mm++; }
03319             }
03320             else {
03321                 k = *mm; NCOPY(tt,mm,k)
03322             }
03323             *t = AR.PolyFun;
03324             t[2] |= MUSTCLEANPRF;
03325             goto regularratfun;
03326         }
03327     }
03328     else if ( *t == AR.PolyFun ) {
03329         if ( AR.PolyFunType == 1 ) { /* Regular PolyFun with one argument */
03330             if ( t[FUNHEAD+1] == 0 && AR.Eside != LHSIDE &&
03331                 t[1] == FUNHEAD + 2 && t[FUNHEAD] == -SNUMBER ) goto NormZero;
03332             if ( i > 0 && pcom[i-1][0] == AR.PolyFun ) {
03333                 if ( AN.PolyNormFlag == 0 ) {

```

```

03334             AN.PolyNormFlag = 1;
03335             AN.PolyFunTodo = 0;
03336         }
03337     }
03338     k = t[1];
03339     NCOPY(m,t,k);
03340 }
03341 else if ( AR.PolyFunType == 2 ) {
03342 /*
03343     PolyRatFun.
03344     Regular type: Two arguments
03345     Power expanded: One argument. Here to be treated as
03346     AR.PolyFunType == 1, but with power cutoff.
03347 */
03348 regulararratfun;;
03349 /*
03350     First check for zeroes.
03351 */
03352     if ( t[FUNHEAD+1] == 0 && AR.Eside != LHSIDE &&
03353         t[1] > FUNHEAD + 2 && t[FUNHEAD] == -SNUMBER ) {
03354         u = t + FUNHEAD + 2;
03355         if ( *u < 0 ) {
03356             if ( *u <= -FUNCTION ) {}
03357             else if ( t[1] == FUNHEAD+4 && t[FUNHEAD+2] == -SNUMBER
03358                 && t[FUNHEAD+3] == 0 ) goto NormPRF;
03359             else if ( t[1] == FUNHEAD+4 ) goto NormZero;
03360         }
03361         else if ( t[1] == *u+FUNHEAD+2 ) goto NormZero;
03362     }
03363     else {
03364         u = t+FUNHEAD; NEXTARG(u);
03365         if ( *u == -SNUMBER && u[1] == 0 ) goto NormInf;
03366     }
03367     if ( i > 0 && pcom[i-1][0] == AR.PolyFun ) AN.PolyNormFlag = 1;
03368     else if ( i < ncom-1 && pcom[i+1][0] == AR.PolyFun ) AN.PolyNormFlag = 1;
03369     k = t[1];
03370     if ( AN.PolyNormFlag ) {
03371         if ( AR.PolyFunExp == 0 ) {
03372             AN.PolyFunTodo = 0;
03373             NCOPY(m,t,k);
03374         }
03375         else if ( AR.PolyFunExp == 1 ) { /* get highest divergence */
03376             if ( PolyFunMode == 0 ) {
03377                 NCOPY(m,t,k);
03378                 AN.PolyFunTodo = 1;
03379             }
03380             else {
03381                 WORD *mmm = m;
03382                 NCOPY(m,t,k);
03383                 if ( TreatPolyRatFun(BHEAD mmm) != 0 )
03384                     goto FromNorm;
03385                 m = mmm+mmm[1];
03386             }
03387         }
03388         else {
03389             if ( PolyFunMode == 0 ) {
03390                 NCOPY(m,t,k);
03391                 AN.PolyFunTodo = 1;
03392             }
03393             else {
03394                 WORD *mmm = m;
03395                 NCOPY(m,t,k);
03396                 if ( ExpandRat(BHEAD mmm) != 0 )
03397                     goto FromNorm;
03398                 m = mmm+mmm[1];
03399             }
03400         }
03401     }
03402     else {
03403         if ( AR.PolyFunExp == 0 ) {
03404             AN.PolyFunTodo = 0;
03405             NCOPY(m,t,k);
03406         }
03407         else if ( AR.PolyFunExp == 1 ) { /* get highest divergence */
03408             WORD *mmm = m;
03409             NCOPY(m,t,k);
03410             if ( TreatPolyRatFun(BHEAD mmm) != 0 )
03411                 goto FromNorm;
03412             m = mmm+mmm[1];
03413         }
03414         else {
03415             WORD *mmm = m;
03416             NCOPY(m,t,k);
03417             if ( ExpandRat(BHEAD mmm) != 0 )
03418                 goto FromNorm;
03419             m = mmm+mmm[1];
03420         }
03421     }

```

```

03421         }
03422     }
03423 }
03424 else if ( *t > 0 ) {
03425     if ( ( t[2] & DIRTYFLAG ) == DIRTYFLAG
03426         && *t != REPLACEMENT && TestFunFlag(BHEAD t) ) ReplaceVeto = 1;
03427     k = t[1];
03428     NCOPY(m,t,k);
03429 }
03430 else {
03431     *m++ = -*t; *m++ = FUNHEAD; *m++ = 0;
03432     FILLFUN3(m)
03433 }
03434 }
03435 }
03436 /*
03437 #] Commuting Functions :
03438 #[ Track Replace_ :
03439 */
03440 if ( ReplaceVeto < 0 ) {
03441 /*
03442     We found one (or more) replace_ functions and all other
03443     functions are 'clean' (no dirty flag).
03444     Now we check whether one of these functions can be used.
03445     Thus far the functions go from fillsetexp to m.
03446     Somewhere in there there are -ReplaceVeto occurrences of REPLACEMENT.
03447     Hunt for the first one that fits the bill.
03448     Note that replace_ is a commuting function.
03449 */
03450     WORD *ma = fillsetexp, *mb, *mc;
03451     while ( ma < m ) {
03452         mb = ma + ma[1];
03453         if ( *ma != REPLACEMENT ) {
03454             ma = mb;
03455             continue;
03456         }
03457         if ( *ma == REPLACEMENT && ReplaceType == -1 ) {
03458             mc = ma;
03459             ReplaceType = 0;
03460             if ( AN.RSsize < 2*ma[1]+SUBEXPSIZE ) {
03461                 if ( AN.ReplaceScrat ) M_free(AN.ReplaceScrat,"AN.ReplaceScrat");
03462                 AN.RSsize = 2*ma[1]+SUBEXPSIZE+40;
03463                 AN.ReplaceScrat = (WORD *)Malloc1((AN.RSsize+1)*sizeof(WORD),"AN.ReplaceScrat");
03464             }
03465             ma += FUNHEAD;
03466             ReplaceSub = AN.ReplaceScrat;
03467             ReplaceSub += SUBEXPSIZE;
03468             while ( ma < mb ) {
03469                 if ( *ma > 0 ) goto NoRep;
03470                 if ( *ma <= -FUNCTION ) {
03471                     *ReplaceSub++ = FUNTOFUN;
03472                     *ReplaceSub++ = 4;
03473                     *ReplaceSub++ = -*ma++;
03474                     if ( *ma > -FUNCTION ) goto NoRep;
03475                     *ReplaceSub++ = -*ma++;
03476                 }
03477                 else if ( ma+4 > mb ) goto NoRep;
03478                 else {
03479                     if ( *ma == -SYMBOL ) {
03480                         if ( ma[2] == -SYMBOL && ma+4 <= mb )
03481                             *ReplaceSub++ = SYMTOSYM;
03482                     }
03483                     else if ( ma[2] == -SNUMBER && ma+4 <= mb ) {
03484                         *ReplaceSub++ = SYMTONUM;
03485                         if ( ReplaceType == 0 ) {
03486                             oldtoprhs = C->numrhs;
03487                             oldtopinter = C->Pointer - C->Buffer;
03488                         }
03489                         ReplaceType = 1;
03490                     }
03491                     else if ( ma[2] == ARGHEAD && ma+2+ARGHEAD <= mb ) {
03492                         *ReplaceSub++ = SYMTONUM;
03493                         *ReplaceSub++ = 4;
03494                         *ReplaceSub++ = ma[1];
03495                         *ReplaceSub++ = 0;
03496                         ma += 2+ARGHEAD;
03497                         continue;
03498                     }
03499                 }
03500             }
03501             /*
03502             Next is the subexpression. We have to test that
03503             it isn't vector-like or index-like
03504             */
03505             else if ( ma[2] > 0 ) {
03506                 WORD *sstop, *ttstop, n;
03507                 ss = ma+2;
03508                 sstop = ss + *ss;
03509                 ss += ARGHEAD;
03510                 while ( ss < sstop ) {

```



```

03508         tt = ss + *ss;
03509         ttstop = tt - ABS(tt[-1]);
03510         ss++;
03511         while ( ss < ttstop ) {
03512             if ( *ss == INDEX ) goto NoRep;
03513             ss += ss[1];
03514         }
03515         ss = tt;
03516     }
03517     subtype = SYMTOSUB;
03518     if ( ReplaceType == 0 ) {
03519         oldtoprhs = C->numrhs;
03520         oldcpointer = C->Pointer - C->Buffer;
03521     }
03522     ReplaceType = 1;
03523     ss = AddRHS(AT.ebufnum,1);
03524     tt = ma+2;
03525     n = *tt - ARGHEAD;
03526     tt += ARGHEAD;
03527     while ( (ss + n + 10) > C->Top ) ss = DoubleCbuffer(AT.ebufnum,ss,14);
03528     while ( --n >= 0 ) *ss++ = *tt++;
03529     *ss++ = 0;
03530     C->rhs[C->numrhs+1] = ss;
03531     C->Pointer = ss;
03532     *ReplaceSub++ = subtype;
03533     *ReplaceSub++ = 4;
03534     *ReplaceSub++ = ma[1];
03535     *ReplaceSub++ = C->numrhs;
03536     ma += 2 + ma[2];
03537     continue;
03538 }
03539 else goto NoRep;
03540 }
03541 else if ( ( *ma == -VECTOR || *ma == -MINVECTOR ) && ma+4 <= mb ) {
03542     if ( ma[2] == -VECTOR ) {
03543         if ( *ma == -VECTOR ) *ReplaceSub++ = VECTOVEC;
03544         else *ReplaceSub++ = VECTOMIN;
03545     }
03546     else if ( ma[2] == -MINVECTOR ) {
03547         if ( *ma == -VECTOR ) *ReplaceSub++ = VECTOMIN;
03548         else *ReplaceSub++ = VECTOVEC;
03549     }
03550 }
03551 /*
03552 Next is a vector-like subexpression
03553 Search for vector nature first
03554 */
03555 else if ( ma[2] > 0 ) {
03556     WORD *sstop, *ttstop, *w, *mm, n, count;
03557     WORD *v1, *v2 = 0;
03558     if ( *ma == -MINVECTOR ) {
03559         ss = ma+2;
03560         sstop = ss + *ss;
03561         ss += ARGHEAD;
03562         while ( ss < sstop ) {
03563             ss += *ss;
03564             ss[-1] = -ss[-1];
03565         }
03566         *ma = -VECTOR;
03567     }
03568     ss = ma+2;
03569     sstop = ss + *ss;
03570     ss += ARGHEAD;
03571     while ( ss < sstop ) {
03572         tt = ss + *ss;
03573         ttstop = tt - ABS(tt[-1]);
03574         ss++;
03575         count = 0;
03576         while ( ss < ttstop ) {
03577             if ( *ss == INDEX ) {
03578                 n = ss[1] - 2; ss += 2;
03579                 while ( --n >= 0 ) {
03580                     if ( *ss < MINSPEC ) count++;
03581                     ss++;
03582                 }
03583             }
03584             else ss += ss[1];
03585         }
03586         if ( count != 1 ) goto NoRep;
03587         ss = tt;
03588     }
03589     subtype = VECTOSUB;
03590     if ( ReplaceType == 0 ) {
03591         oldtoprhs = C->numrhs;
03592         oldcpointer = C->Pointer - C->Buffer;
03593     }
03594     ReplaceType = 1;
03595     mm = AddRHS(AT.ebufnum,1);

```

```

03595         *ReplaceSub++ = subtype;
03596         *ReplaceSub++ = 4;
03597         *ReplaceSub++ = ma[1];
03598         *ReplaceSub++ = C->numrhs;
03599         w = ma+2;
03600         n = *w - ARGHEAD;
03601         w += ARGHEAD;
03602         while ( (mm + n + 10) > C->Top )
03603             mm = DoubleCbuffer(AT.ebufnum,mm,15);
03604         while ( --n >= 0 ) *mm++ = *w++;
03605         *mm++ = 0;
03606         C->rhs[C->numrhs+1] = mm;
03607         C->Pointer = mm;
03608         mm = AddRHS(AT.ebufnum,1);
03609         w = ma+2;
03610         n = *w - ARGHEAD;
03611         w += ARGHEAD;
03612         while ( (mm + n + 13) > C->Top )
03613             mm = DoubleCbuffer(AT.ebufnum,mm,16);
03614         sstop = w + n;
03615         while ( w < sstop ) {
03616             tt = w + *w; ttstop = tt - ABS(tt[-1]);
03617             ss = mm; mm++; w++;
03618             while ( w < ttstop ) {          /* Subterms */
03619                 if ( *w != INDEX ) {
03620                     n = w[1];
03621                     NCOPY(mm,w,n);
03622                 }
03623                 else {
03624                     v1 = mm;
03625                     *mm++ = *w++;
03626                     *mm++ = n = *w++;
03627                     n -= 2;
03628                     while ( --n >= 0 ) {
03629                         if ( *w >= MINSPEC ) *mm++ = *w++;
03630                         else v2 = w++;
03631                     }
03632                     n = WORDDIF(mm,v1);
03633                     if ( n != v1[1] ) {
03634                         if ( n <= 2 ) mm -= 2;
03635                         else v1[1] = n;
03636                         *mm++ = VECTOR;
03637                         *mm++ = 4;
03638                         *mm++ = *v2;
03639                         *mm++ = FUNNYVEC;
03640                     }
03641                 }
03642             }
03643             while ( w < tt ) *mm++ = *w++;
03644             *ss = WORDDIF(mm,ss);
03645         }
03646         *mm++ = 0;
03647         C->rhs[C->numrhs+1] = mm;
03648         C->Pointer = mm;
03649         if ( mm > C->Top ) {
03650             MLOCK(ErrorMessageLock);
03651             MesPrint("Internal error in Normalize with extra compiler
buffer");
03652             MUNLOCK(ErrorMessageLock);
03653             Terminate(-1);
03654         }
03655         ma += 2 + ma[2];
03656         continue;
03657     }
03658     else goto NoRep;
03659 }
03660 else if ( *ma == -INDEX ) {
03661     if ( ( ma[2] == -INDEX || ma[2] == -VECTOR )
03662         && ma+4 <= mb )
03663         *ReplaceSub++ = INDTOIND;
03664     else if ( ma[1] >= AM.OffsetIndex ) {
03665         if ( ma[2] == -SNUMBER && ma+4 <= mb
03666             && ma[3] >= 0 && ma[3] < AM.OffsetIndex )
03667             *ReplaceSub++ = INDTOIND;
03668         else if ( ma[2] == ARGHEAD && ma+2+ARGHEAD <= mb ) {
03669             *ReplaceSub++ = INDTOIND;
03670             *ReplaceSub++ = 4;
03671             *ReplaceSub++ = ma[1];
03672             *ReplaceSub++ = 0;
03673             ma += 2+ARGHEAD;
03674             continue;
03675         }
03676         else goto NoRep;
03677     }
03678     else goto NoRep;
03679 }
03680 else goto NoRep;

```

```

03681             *ReplaceSub++ = 4;
03682             *ReplaceSub++ = ma[1];
03683             *ReplaceSub++ = ma[3];
03684             ma += 4;
03685         }
03686     }
03687 }
03688 AN.ReplaceScrat[1] = ReplaceSub-AN.ReplaceScrat;
03689 /*
03690     Success. This means that we have to remove the replace_
03691     from the functions. It starts at mc and end at mb.
03692 */
03693 while ( mb < m ) *mc++ = *mb++;
03694 m = mc;
03695 break;
03696 NoRep:
03697     if ( ReplaceType > 0 ) {
03698         C->numrhs = oldtoprhs;
03699         C->Pointer = C->Buffer + oldcpointer;
03700     }
03701     ReplaceType = -1;
03702     if ( ++ReplaceVeto >= 0 ) break;
03703 }
03704 ma = mb;
03705 }
03706 }
03707 /*
03708     #] Track Replace_ :
03709     #[ LeviCivita tensors :
03710 */
03711 if ( neps ) {
03712     to = m;
03713     for ( i = 0; i < neps; i++ ) { /* Put the indices in order */
03714         t = peps[i];
03715         if ( ( t[2] & DIRTYSYMFLAG ) != DIRTYSYMFLAG ) continue;
03716         t[2] &= ~DIRTYSYMFLAG;
03717         if ( AR.Eside == LHSIDE || AR.Eside == LHSIDEX ) {
03718             /* Potential problems with FUNNYWILD */
03719         }
03720     }
03721     /*
03722         First make sure all FUNNIES are at the end.
03723         Then sort separately
03724     */
03725     r = t + FUNHEAD;
03726     m = tt = t + t[1];
03727     while ( r < m ) {
03728         if ( *r != FUNNYWILD ) { r++; continue; }
03729         k = r[1]; u = r + 2;
03730         while ( u < tt ) {
03731             u[-2] = *u;
03732             if ( *u != FUNNYWILD ) ncoef = -ncoef;
03733             u++;
03734         }
03735         tt[-2] = FUNNYWILD; tt[-1] = k; m -= 2;
03736     }
03737     t += FUNHEAD;
03738     do {
03739         for ( r = t + 1; r < m; r++ ) {
03740             if ( *r < *t ) { k = *r; *r = *t; *t = k; ncoef = -ncoef; }
03741             else if ( *r == *t ) goto NormZero;
03742         }
03743         t++;
03744     } while ( t < m );
03745     do {
03746         for ( r = t + 2; r < tt; r += 2 ) {
03747             if ( r[1] < t[1] ) {
03748                 k = r[1]; r[1] = t[1]; t[1] = k; ncoef = -ncoef; }
03749             else if ( r[1] == t[1] ) goto NormZero;
03750         }
03751         t += 2;
03752     } while ( t < tt );
03753 }
03754 else {
03755     m = t + t[1];
03756     t += FUNHEAD;
03757     do {
03758         for ( r = t + 1; r < m; r++ ) {
03759             if ( *r < *t ) { k = *r; *r = *t; *t = k; ncoef = -ncoef; }
03760             else if ( *r == *t ) goto NormZero;
03761         }
03762         t++;
03763     } while ( t < m );
03764 }
03765 }
03766 /* Sort the tensors */
03767 for ( i = 0; i < (neps-1); i++ ) {

```

```

03768         t = peps[i];
03769         for ( j = i+1; j < neps; j++ ) {
03770             r = peps[j];
03771             if ( t[1] > r[1] ) {
03772                 peps[i] = m = r; peps[j] = r = t; t = m;
03773             }
03774             else if ( t[1] == r[1] ) {
03775                 k = t[1] - FUNHEAD;
03776                 m = t + FUNHEAD;
03777                 r += FUNHEAD;
03778                 do {
03779                     if ( *r < *m ) {
03780                         m = peps[j]; peps[j] = t; peps[i] = t = m;
03781                         break;
03782                     }
03783                     else if ( *r++ > *m++ ) break;
03784                 } while ( --k > 0 );
03785             }
03786         }
03787     }
03788     m = to;
03789     for ( i = 0; i < neps; i++ ) {
03790         t = peps[i];
03791         k = t[1];
03792         NCOPY(m,t,k);
03793     }
03794 }
03795 /*
03796 #] LeviCivita tensors :
03797 #[ Delta :
03798 */
03799 if ( ndel ) {
03800     r = t = pdel;
03801     for ( i = 0; i < ndel; i += 2, r += 2 ) {
03802         if ( r[1] < r[0] ) { k = *r; *r = r[1]; r[1] = k; }
03803     }
03804     for ( i = 2; i < ndel; i += 2, t += 2 ) {
03805         r = t + 2;
03806         for ( j = i; j < ndel; j += 2 ) {
03807             if ( *r > *t ) { r += 2; }
03808             else if ( *r < *t ) {
03809                 k = *r; *r++ = *t; *t++ = k;
03810                 k = *r; *r++ = *t; *t-- = k;
03811             }
03812             else {
03813                 if ( *++r < t[1] ) {
03814                     k = *r; *r = t[1]; t[1] = k;
03815                 }
03816                 r++;
03817             }
03818         }
03819     }
03820     t = pdel;
03821     *m++ = DELTA;
03822     *m++ = ndel + 2;
03823     i = ndel;
03824     NCOPY(m,t,i);
03825 }
03826 /*
03827 #] Delta :
03828 #[ Loose Vectors/Indices :
03829 */
03830 if ( nind ) {
03831     t = pind;
03832     for ( i = 0; i < nind; i++ ) {
03833         r = t + 1;
03834         for ( j = i+1; j < nind; j++ ) {
03835             if ( *r < *t ) {
03836                 k = *r; *r = *t; *t = k;
03837             }
03838             r++;
03839         }
03840         t++;
03841     }
03842     t = pind;
03843     *m++ = INDEX;
03844     *m++ = nind + 2;
03845     i = nind;
03846     NCOPY(m,t,i);
03847 }
03848 /*
03849 #] Loose Vectors/Indices :
03850 #[ Vectors :
03851 */
03852 if ( nvec ) {
03853     t = pvec;
03854     for ( i = 2; i < nvec; i += 2 ) {

```

```

03855         r = t + 2;
03856         for ( j = i; j < nvec; j += 2 ) {
03857             if ( *r == *t ) {
03858                 if ( *++r < t[1] ) {
03859                     k = *r; *r = t[1]; t[1] = k;
03860                 }
03861                 r++;
03862             }
03863             else if ( *r < *t ) {
03864                 k = *r; *r++ = *t; *t++ = k;
03865                 k = *r; *r++ = *t; *t-- = k;
03866             }
03867             else { r += 2; }
03868         }
03869         t += 2;
03870     }
03871     t = pvec;
03872     *m++ = VECTOR;
03873     *m++ = nvec + 2;
03874     i = nvec;
03875     NCOPY(m,t,i);
03876 }
03877 /*
03878 #] Vectors :
03879 #[ Dotproducts :
03880 */
03881 if ( ndot ) {
03882     to = m;
03883     m = t = pdot;
03884     i = ndot;
03885     while ( --i >= 0 ) {
03886         if ( *t > t[1] ) { j = *t; *t = t[1]; t[1] = j; }
03887         t += 3;
03888     }
03889     t = m;
03890     ndot *= 3;
03891     m += ndot;
03892     while ( t < (m-3) ) {
03893         r = t + 3;
03894         do {
03895             if ( *r == *t ) {
03896                 if ( *++r == *++t ) {
03897                     r++;
03898                     if ( ( *r < MAXPOWER && t[1] < MAXPOWER )
03899                         || ( *r > -MAXPOWER && t[1] > -MAXPOWER ) ) {
03900                         t++;
03901                         *t += *r;
03902                         if ( *t > MAXPOWER || *t < -MAXPOWER ) {
03903                             MLOCK(ErrorMessageLock);
03904                             MesPrint("Exponent of dotproduct out of range: %d",*t);
03905                             MUNLOCK(ErrorMessageLock);
03906                             goto NormMin;
03907                         }
03908                         ndot -= 3;
03909                         *r-- = *--m;
03910                         *r-- = *--m;
03911                         *r = *--m;
03912                         if ( !*t ) {
03913                             ndot -= 3;
03914                             *t-- = *--m;
03915                             *t-- = *--m;
03916                             *t = *--m;
03917                             t -= 3;
03918                             break;
03919                         }
03920                     }
03921                     else if ( *r < *++t ) {
03922                         k = *r; *r++ = *t; *t = k;
03923                     }
03924                     else r++;
03925                     t -= 2;
03926                 }
03927             else if ( *r < *t ) {
03928                 k = *r; *r++ = *t; *t++ = k;
03929                 k = *r; *r++ = *t; *t = k;
03930                 t -= 2;
03931             }
03932             else { r += 2; t--; }
03933         }
03934         else if ( *r < *t ) {
03935             k = *r; *r++ = *t; *t++ = k;
03936             k = *r; *r++ = *t; *t++ = k;
03937             k = *r; *r++ = *t; *t = k;
03938             t -= 2;
03939         }
03940         else { r += 3; }
03941     } while ( r < m );

```

```

03942         t += 3;
03943     }
03944     m = to;
03945     t = pdot;
03946     if ( ( i = ndot ) > 0 ) {
03947         *m++ = DOTPRODUCT;
03948         *m++ = i + 2;
03949         NCOPY(m,t,i);
03950     }
03951 }
03952 /*
03953 #] Dotproducts :
03954 #[ Symbols :
03955 */
03956 if ( nsym ) {
03957     nsym <= 1;
03958     t = psym;
03959     *m++ = SYMBOL;
03960     r = m;
03961     *m++ = ( i = nsym ) + 2;
03962     if ( i ) { do {
03963         if ( !*t ) {
03964             if ( t[1] < (2*MAXPOWER) ) { /* powers of i */
03965                 if ( t[1] & 1 ) { *m++ = 0; *m++ = 1; }
03966                 else *r -= 2;
03967                 if ( **t & 2 ) ncoef = -ncoef;
03968                 t++;
03969             }
03970         }
03971         else if ( *t <= NumSymbols && *t > -2*MAXPOWER ) { /* Put powers in range */
03972             if ( ( ( t[1] > symbols[*t].maxpower ) && ( symbols[*t].maxpower < MAXPOWER ) ) ||
03973                 ( t[1] < symbols[*t].minpower ) && ( symbols[*t].minpower > -MAXPOWER ) ) )
03974                 &&
03975                 ( t[1] < 2*MAXPOWER ) && ( t[1] > -2*MAXPOWER ) ) {
03976                 if ( i <= 2 || t[2] != *t ) goto NormZero;
03977             }
03978             if ( AN.ncmod == 1 && ( AC.modmode & ALSOPOWERS ) != 0 ) {
03979                 if ( AC.cmod[0] == 1 ) t[1] = 0;
03980                 else if ( t[1] >= 0 ) t[1] = 1 + (t[1]-1)%(AC.cmod[0]-1);
03981                 else {
03982                     t[1] = -1 - (-t[1]-1)%(AC.cmod[0]-1);
03983                     if ( t[1] < 0 ) t[1] += (AC.cmod[0]-1);
03984                 }
03985             }
03986             if ( ( t[1] < (2*MAXPOWER) && t[1] >= MAXPOWER )
03987                 || ( t[1] > -(2*MAXPOWER) && t[1] <= -MAXPOWER ) ) {
03988                 MLOCK(ErrorMessageLock);
03989                 MesPrint("Exponent out of range: %d",t[1]);
03990                 MUNLOCK(ErrorMessageLock);
03991                 goto NormMin;
03992             }
03993             if ( AT.TrimPower && AR.PolyFunVar == *t && t[1] > AR.PolyFunPow ) {
03994                 goto NormZero;
03995             }
03996             else if ( t[1] ) {
03997                 *m++ = *t++;
03998                 *m++ = *t++;
03999             }
04000             else { *r -= 2; t += 2; }
04001         }
04002         else {
04003             *m++ = *t++; *m++ = *t++;
04004         }
04005     } while ( (i--)> 0 ); }
04006     if ( *r <= 2 ) m = r-1;
04007 }
04008 /*
04009 #] Symbols :
04010 #[ float_ :
04011
04012 Here we treat float_ functions and combined them with the regular
04013 coefficient.
04014 */
04015 #ifdef WITHFLOAT
04016 if ( withfloat ) {
04017     WORD floatsign = 3;
04018 /*
04019 First check whether the coefficient is already 1/1
04020 */
04021 if ( ABS(ncoef) == 3 && n_coef[0] == 1 && n_coef[1] == 1 ) {
04022     if ( withfloat == 1 ) {
04023         t = firstfloat;
04024 /*
04025 We only interfere by making the sign positive.
04026 The float_ function has already been tested.
04027 */

```

```

04028         if ( t[FUNHEAD+3] < 0 ) {
04029             t[FUNHEAD+3] = -t[FUNHEAD+3];
04030             floatsign = -floatsign;
04031         }
04032         if ( ncoef < 0 ) floatsign = -floatsign;
04033 /*
04034         Copy to output;
04035 */
04036         AT.FloatPos = m-termout;
04037         i = t[1]; NCOPY(m,t,i)
04038     }
04039     else {
04040         PackFloat(m,aux4);
04041         if ( m[FUNHEAD+3] < 0 ) {
04042             m[FUNHEAD+3] = -m[FUNHEAD+3];
04043             floatsign = -floatsign;
04044         }
04045         if ( ncoef < 0 ) floatsign = -floatsign;
04046         AT.FloatPos = m-termout;
04047         m += m[1];
04048     }
04049 }
04050 else {
04051     if ( withfloat == 1 ) UnpackFloat(aux4,firstfloat);
04052     RatToFloat(aux5,(UWORD *)n_coef,ncoef);
04053     mpf_mul(aux4,aux4,aux5);
04054     PackFloat(m,aux4);
04055     AT.FloatPos = m-termout;
04056     if ( m[FUNHEAD+3] < 0 ) {
04057         m[FUNHEAD+3] = -m[FUNHEAD+3];
04058         floatsign = -floatsign;
04059     }
04060     m += m[1];
04061 }
04062 n_coef[0] = 1; n_coef[1] = 1; ncoef = floatsign;
04063 }
04064 else AT.FloatPos = 0;
04065 #endif
04066 /*
04067 #] float_ :
04068 #[ Do Replace_ :
04069 */
04070 stop = (WORD *)(((UBYTE *) (termout)) + AM.MaxTer);
04071 i = ABS(ncoef);
04072 if ( ( m + i ) > stop ) {
04073     MLOCK(ErrorMessageLock);
04074     MesPrint("Term too complex during normalization");
04075     MUNLOCK(ErrorMessageLock);
04076     goto NormMin;
04077 }
04078 if ( ReplaceType >= 0 ) {
04079     t = n_coef;
04080     i--;
04081     NCOPY(m,t,i);
04082     *m++ = ncoef;
04083     t = termout;
04084     *t = WORDDIF(m,t);
04085     if ( ReplaceType == 0 ) {
04086         AT.WorkPointer = termout+*termout;
04087         WildFill(BHEAD term,termout,AN.ReplaceScrat);
04088         termout = term + *term;
04089     }
04090     else {
04091         AT.WorkPointer = r = termout + *termout;
04092         WildFill(BHEAD r,termout,AN.ReplaceScrat);
04093         i = *r; m = term;
04094         NCOPY(m,r,i);
04095         termout = m;
04096
04097         r = m = term;
04098         r += *term; r -= ABS(r[-1]);
04099         m++;
04100         while ( m < r ) {
04101             if ( *m >= FUNCTION && m[1] > FUNHEAD &&
04102                 functions[*m-FUNCTION].spec != TENSORFUNCTION )
04103                 m[2] |= DIRTYFLAG;
04104             m += m[1];
04105         }
04106     }
04107 }
04108 /*
04109 The next 'reset' cannot be done. We still need the expression
04110 in the buffer. Note though that this may cause a runaway pointer
04111 if we are not very careful.
04112
04113 C->numrhs = oldtoprhs;
04114

```

```

04115         C->Pointer = C->Buffer + oldcpointer;
04116     */
04117     TermFree(n_llnum, "n_llnum");
04118     TermFree(n_coef, "NormCoef");
04119     return(1);
04120 }
04121 else {
04122     t = termout;
04123     k = WORDDIF(m,t);
04124     *t = k + i;
04125     m = term;
04126     NCOPY(m,t,k);
04127     i--;
04128     t = n_coef;
04129     NCOPY(m,t,i);
04130     *m++ = ncoef;
04131 }
04132 /*
04133 #] Do Replace_ :
04134 #[ Errors and Finish :
04135 */
04136 RegEnd:
04137     if ( termout < term + *term && termout >= term ) AT.WorkPointer = term + *term;
04138     else AT.WorkPointer = termout;
04139 /*
04140     if ( termflag ) { We have to assign the term to $variable(s)
04141         TermAssign(term);
04142     }
04143 */
04144     TermFree(n_llnum, "n_llnum");
04145     TermFree(n_coef, "NormCoef");
04146     return(regval);
04147
04148 NormInf:
04149     MLOCK(ErrorMessageLock);
04150     MesPrint("Division by zero during normalization");
04151     MUNLOCK(ErrorMessageLock);
04152     Terminate(-1);
04153
04154 NormZZ:
04155     MLOCK(ErrorMessageLock);
04156     MesPrint("0^0 during normalization of term");
04157     MUNLOCK(ErrorMessageLock);
04158     Terminate(-1);
04159
04160 NormPRF:
04161     MLOCK(ErrorMessageLock);
04162     MesPrint("0/0 in polyratfun during normalization of term");
04163     MUNLOCK(ErrorMessageLock);
04164     Terminate(-1);
04165
04166 NormZero:
04167     *term = 0;
04168     AT.WorkPointer = termout;
04169     TermFree(n_llnum, "n_llnum");
04170     TermFree(n_coef, "NormCoef");
04171     return(regval);
04172
04173 NormMin:
04174     TermFree(n_llnum, "n_llnum");
04175     TermFree(n_coef, "NormCoef");
04176     return(-1);
04177
04178 FromNorm:
04179     MLOCK(ErrorMessageLock);
04180     MesCall("Norm");
04181     MUNLOCK(ErrorMessageLock);
04182     TermFree(n_llnum, "n_llnum");
04183     TermFree(n_coef, "NormCoef");
04184     return(-1);
04185
04186 /*
04187 #] Errors and Finish :
04188 */
04189 }
04190
04191 /*
04192 #] Normalize :
04193 #[ ExtraSymbol :
04194 */
04195
04196 WORD ExtraSymbol(WORD sym, WORD pow, WORD nsym, WORD *ppsym, WORD *ncoef)
04197 {
04198     WORD *m, i;
04199     i = nsym;
04200     m = ppsym - 2;
04201     while ( i > 0 ) {

```



```

04202         if ( sym == *m ) {
04203             m++;
04204             if ( pow > 2*MAXPOWER || pow < -2*MAXPOWER
04205                 || *m > 2*MAXPOWER || *m < -2*MAXPOWER ) {
04206                 MLOCK(ErrorMessageLock);
04207                 MesPrint("Illegal wildcard power combination.");
04208                 MUNLOCK(ErrorMessageLock);
04209                 Terminate(-1);
04210             }
04211             *m += pow;
04212
04213             if ( ( sym <= NumSymbols && sym > -MAXPOWER )
04214                 && ( symbols[sym].complex & VARTYPEROOTOFUNITY ) == VARTYPEROOTOFUNITY ) {
04215                 *m %= symbols[sym].maxpower;
04216                 if ( *m < 0 ) *m += symbols[sym].maxpower;
04217                 if ( ( symbols[sym].complex & VARTYPEMINUS ) == VARTYPEMINUS ) {
04218                     if ( ( ( symbols[sym].maxpower & 1 ) == 0 ) &&
04219                         ( *m >= symbols[sym].maxpower/2 ) ) {
04220                         *m -= symbols[sym].maxpower/2; *ncoef = -*ncoef;
04221                     }
04222                 }
04223             }
04224
04225             if ( *m >= 2*MAXPOWER || *m <= -2*MAXPOWER ) {
04226                 MLOCK(ErrorMessageLock);
04227                 MesPrint("Power overflow during normalization");
04228                 MUNLOCK(ErrorMessageLock);
04229                 return(-1);
04230             }
04231             if ( !*m ) {
04232                 m--;
04233                 while ( i < nsym )
04234                     { *m = m[2]; m++; *m = m[2]; m++; i++; }
04235                 return(-1);
04236             }
04237             return(0);
04238         }
04239         else if ( sym < *m ) {
04240             m -= 2;
04241             i--;
04242         }
04243         else break;
04244     }
04245     m = ppsym;
04246     while ( i < nsym )
04247         { m--; m[2] = *m; m--; m[2] = *m; i++; }
04248     *m++ = sym;
04249     *m = pow;
04250     return(1);
04251 }
04252
04253 /*
04254     #] ExtraSymbol :
04255     #[ DoTheta :
04256 */
04257
04258 WORD DoTheta(PHEAD WORD *t)
04259 {
04260     GETBIDENTITY
04261     WORD k, *r1, *r2, *tstop, type;
04262     WORD ia, *ta, *tb, *stopa, *stopb;
04263     if ( AC.BracKetNormalize ) return(-1);
04264     type = *t;
04265     k = t[1];
04266     tstop = t + k;
04267     t += FUNHEAD;
04268     if ( k <= FUNHEAD ) return(1);
04269     r1 = t;
04270     NEXTARG(r1)
04271     if ( r1 == tstop ) {
04272         /*
04273             One argument
04274         */
04275         if ( *t == ARGHEAD ) {
04276             if ( type == THETA ) return(1);
04277             else return(0); /* THETA2 */
04278         }
04279         if ( *t < 0 ) {
04280             if ( *t == -SNUMBER ) {
04281                 if ( t[1] < 0 ) return(0);
04282                 else {
04283                     if ( type == THETA2 && t[1] == 0 ) return(0);
04284                     else return(1);
04285                 }
04286             }
04287             return(-1);
04288         }

```

```

04289         k = t[*t-1];
04290         if ( *t == ABS(k)+1+ARGHEAD ) {
04291             if ( k > 0 ) return(1);
04292             else return(0);
04293         }
04294         return(-1);
04295     }
04296     /*
04297     At least two arguments
04298     */
04299     r2 = r1;
04300     NEXTARG(r2)
04301     if ( r2 < tstop ) return(-1);    /* More than 2 arguments */
04302     /*
04303     Note now that zero has to be treated specially
04304     We take the criteria from the symmetrize routine
04305     */
04306     if ( *t == -SNUMBER && *r1 == -SNUMBER ) {
04307         if ( t[1] > r1[1] ) return(0);
04308         else if ( t[1] < r1[1] ) {
04309             return(1);
04310         }
04311         else if ( type == THETA ) return(1);
04312         else return(0);    /* THETA2 */
04313     }
04314     else if ( t[1] == 0 && *t == -SNUMBER ) {
04315         if ( *r1 > 0 ) { }
04316         else if ( *t < *r1 ) return(1);
04317         else if ( *t > *r1 ) return(0);
04318     }
04319     else if ( r1[1] == 0 && *r1 == -SNUMBER ) {
04320         if ( *t > 0 ) { }
04321         else if ( *t < *r1 ) return(1);
04322         else if ( *t > *r1 ) return(0);
04323     }
04324     r2 = AT.WorkPointer;
04325     if ( *t < 0 ) {
04326         ta = r2;
04327         ToGeneral(t,ta,0);
04328         r2 += *r2;
04329     }
04330     else ta = t;
04331     if ( *r1 < 0 ) {
04332         tb = r2;
04333         ToGeneral(r1,tb,0);
04334     }
04335     else tb = r1;
04336     stopa = ta + *ta;
04337     stopb = tb + *tb;
04338     ta += ARGHEAD; tb += ARGHEAD;
04339     while ( ta < stopa ) {
04340         if ( tb >= stopb ) return(0);
04341         if ( ( ia = CompareTerms(BHEAD ta,tb,(WORD)1) ) < 0 ) return(0);
04342         if ( ia > 0 ) return(1);
04343         ta += *ta;
04344         tb += *tb;
04345     }
04346     if ( type == THETA ) return(1);
04347     else return(0);    /* THETA2 */
04348 }
04349
04350 /*
04351     #] DoTheta :
04352     #[ DoDelta :
04353     */
04354
04355 WORD DoDelta(WORD *t)
04356 {
04357     WORD k, *r1, *r2, *tstop, isnum, isnum2, type = *t;
04358     if ( AC.BracketNormalize ) return(-1);
04359     k = t[1];
04360     if ( k <= FUNHEAD ) goto argzero;
04361     if ( k == FUNHEAD+ARGHEAD && t[FUNHEAD] == ARGHEAD ) goto argzero;
04362     tstop = t + k;
04363     t += FUNHEAD;
04364     r1 = t;
04365     NEXTARG(r1)
04366     if ( *t < 0 ) {
04367         k = 1;
04368         if ( *t == -SNUMBER ) { isnum = 1; k = t[1]; }
04369         else isnum = 0;
04370     }
04371     else {
04372         k = t[*t-1];
04373         k = ABS(k);
04374         if ( k == *t-ARGHEAD-1 ) isnum = 1;
04375         else isnum = 0;

```

```

04376     k = 1;
04377 }
04378 if ( r1 >= tstop ) {          /* Single argument */
04379     if ( !isnum ) return(-1);
04380     if ( k == 0 ) goto argzero;
04381     goto argnonzero;
04382 }
04383 r2 = r1;
04384 NEXTARG(r2)
04385 if ( r2 < tstop ) return(-1);
04386 if ( *r1 < 0 ) {
04387     if ( *r1 == -SNUMBER ) { isnum2 = 1; }
04388     else isnum2 = 0;
04389 }
04390 else {
04391     k = r1[*r1-1];
04392     k = ABS(k);
04393     if ( k == *r1-ARGHEAD-1 ) isnum2 = 1;
04394     else isnum2 = 0;
04395 }
04396 if ( isnum != isnum2 ) return(-1);
04397 tstop = r1;
04398 while ( t < tstop && r1 < r2 ) {
04399     if ( *t != *r1 ) {
04400         if ( !isnum ) return(-1);
04401         goto argnonzero;
04402     }
04403     t++; r1++;
04404 }
04405 if ( t != tstop || r1 != r2 ) {
04406     if ( !isnum ) return(-1);
04407     goto argnonzero;
04408 }
04409 argzero:
04410     if ( type == DELTA2 ) return(1);
04411     else return(0);
04412 argnonzero:
04413     if ( type == DELTA2 ) return(0);
04414     else return(1);
04415 }
04416
04417 /*
04418     #] DoDelta :
04419     #[ DoRevert :
04420 */
04421
04422 void DoRevert(WORD *fun, WORD *tmp)
04423 {
04424     WORD *t, *r, *m, *to, *tt, *mm, i, j;
04425     to = fun + fun[1];
04426     r = fun + FUNHEAD;
04427     while ( r < to ) {
04428         if ( *r <= 0 ) {
04429             if ( *r == -REVERSEFUNCTION ) {
04430                 m = r; mm = m+1;
04431                 while ( mm < to ) *m++ = *mm++;
04432                 to--;
04433                 (fun[1])--;
04434                 fun[2] |= DIRTYSYMFLAG;
04435             }
04436             else if ( *r <= -FUNCTION ) r++;
04437             else {
04438                 if ( *r == -INDEX && r[1] < MINSPEC ) *r = -VECTOR;
04439                 r += 2;
04440             }
04441         }
04442         else {
04443             if ( ( *r > ARGHEAD )
04444                 && ( r[ARGHEAD+1] == REVERSEFUNCTION )
04445                 && ( *r == (r[ARGHEAD]+ARGHEAD) )
04446                 && ( r[ARGHEAD] == (r[ARGHEAD+2]+4) )
04447                 && ( *(r+r-3) == 1 )
04448                 && ( *(r+r-2) == 1 )
04449                 && ( *(r+r-1) == 3 ) ) {
04450                 mm = r;
04451                 r += ARGHEAD + 1;
04452                 tt = r + r[1];
04453                 r += FUNHEAD;
04454                 m = tmp;
04455                 t = r;
04456                 j = 0;
04457                 while ( t < tt ) {
04458                     NEXTARG(t)
04459                     j++;
04460                 }
04461                 while ( --j >= 0 ) {
04462                     i = j;

```

```

04463         t = r;
04464         while ( --i >= 0 ) {
04465             NEXTARG(t)
04466         }
04467         if ( *t > 0 ) {
04468             i = *t;
04469             NCOPY(m,t,i);
04470         }
04471         else if ( *t <= -FUNCTION ) *m++ = *t++;
04472         else { *m++ = *t++; *m++ = *t++; }
04473     }
04474     i = WORDDIF(m,tmp);
04475     m = tmp;
04476     t = mm;
04477     r = t + *t;
04478     NCOPY(t,m,i);
04479     m = r;
04480     r = t;
04481     i = WORDDIF(to,m);
04482     NCOPY(t,m,i);
04483     fun[1] = WORDDIF(t,fun);
04484     to = t;
04485     fun[2] |= DIRTSYMLAG;
04486 }
04487 else r += *r;
04488 }
04489 }
04490 }
04491
04492 /*
04493     #] DoRevert :
04494     #] Normalize :
04495     #[ DetCommu :
04496
04497     Determines the number of terms in an expression that contain
04498     noncommuting objects. This can be used to see whether products of
04499     this expression can be evaluated with binomial coefficients.
04500
04501     We don't try to be fancy. If a term contains noncommuting objects
04502     we are not looking whether they can commute with complete other
04503     terms.
04504
04505     If the number gets too large we cut it off.
04506 */
04507
04508 #define MAXNUMBEROFNONCOMTERMS 2
04509
04510 WORD DetCommu(WORD *terms)
04511 {
04512     WORD *t, *tnext, *tstop;
04513     WORD num = 0;
04514     if ( *terms == 0 ) return(0);
04515     if ( terms[*terms] == 0 ) return(0);
04516     t = terms;
04517     while ( *t ) {
04518         tnext = t + *t;
04519         tstop = tnext - ABS(tnext[-1]);
04520         t++;
04521         while ( t < tstop ) {
04522             if ( *t >= FUNCTION ) {
04523                 if ( functions[*t-FUNCTION].commute ) {
04524                     num++;
04525                     if ( num >= MAXNUMBEROFNONCOMTERMS ) return(num);
04526                     break;
04527                 }
04528             }
04529             else if ( *t == SUBEXPRESSION ) {
04530                 if ( cbuf[t[4]].CanCommu[t[2]] ) {
04531                     num++;
04532                     if ( num >= MAXNUMBEROFNONCOMTERMS ) return(num);
04533                     break;
04534                 }
04535             }
04536             else if ( *t == EXPRESSION ) {
04537                 num++;
04538                 if ( num >= MAXNUMBEROFNONCOMTERMS ) return(num);
04539                 break;
04540             }
04541             else if ( *t == DOLLAREXPRESSION ) {
04542
04543                 /*
04544                 Technically this is not correct. We have to test first
04545                 whether this is MODLOCAL (in TFORM) and if so, use the
04546                 local version. Anyway, this should be rare to never
04547                 occurring because dollars should be replaced.
04548
04549                 if ( cbuf[AM.dbufnum].CanCommu[t[2]] ) {
04550                     num++;

```

```

04550             if ( num >= MAXNUMBEROFNONCOMTERMS ) return(num);
04551             break;
04552         }
04553     }
04554     t += t[1];
04555 }
04556 t = tnext;
04557 }
04558 return(num);
04559 }
04560
04561 /*
04562 #] DetCommu :
04563 #[ DoesCommu :
04564
04565     Determines the number of noncommuting objects in a term.
04566     If the number gets too large we cut it off.
04567 */
04568
04569 WORD DoesCommu(WORD *term)
04570 {
04571     WORD *tstop;
04572     WORD num = 0;
04573     if ( *term == 0 ) return(0);
04574     tstop = term + *term;
04575     tstop = tstop - ABS(tstop[-1]);
04576     term++;
04577     while ( term < tstop ) {
04578         if ( *term >= FUNCTION ) && ( functions[*term-FUNCTION].commute ) ) {
04579             num++;
04580             if ( num >= MAXNUMBEROFNONCOMTERMS ) return(num);
04581         }
04582         term += term[1];
04583     }
04584     return(num);
04585 }
04586
04587 /*
04588 #] DoesCommu :
04589 #[ PolyNormPoly :
04590
04591     Normalizes a polynomial
04592 */
04593
04594 #ifdef EVALUATEGCD
04595 WORD *PolyNormPoly (PHEAD WORD *Poly) {
04596
04597     GETBIDENTITY;
04598     WORD *buffer = AT.WorkPointer;
04599     WORD *p;
04600     if ( NewSort(BHEAD0) ) { Terminate(-1); }
04601     AR.CompareRoutine = (COMPAREDUMMY)(&CompareSymbols);
04602     while ( *Poly ) {
04603         p = Poly + *Poly;
04604         if ( SymbolNormalize(Poly) < 0 ) return(0);
04605         if ( StoreTerm(BHEAD Poly) ) {
04606             AR.CompareRoutine = (COMPAREDUMMY)(&Compare1);
04607             LowerSortLevel();
04608             Terminate(-1);
04609         }
04610         Poly = p;
04611     }
04612     if ( EndSort(BHEAD buffer,1) < 0 ) {
04613         AR.CompareRoutine = (COMPAREDUMMY)(&Compare1);
04614         Terminate(-1);
04615     }
04616     p = buffer;
04617     while ( *p ) p += *p;
04618     AR.CompareRoutine = (COMPAREDUMMY)(&Compare1);
04619     AT.WorkPointer = p + 1;
04620     return(buffer);
04621 }
04622 #endif
04623
04624 /*
04625 #] PolyNormPoly :
04626 #[ EvaluateGcd :
04627
04628     Try to evaluate the GCDFUNCTION gcd_.
04629     This function can have a number of arguments which can be numbers
04630     and/or polynomials. If there are objects that aren't SYMBOLS or numbers
04631     it cannot work currently.
04632
04633     To make this work properly we have to intervene in proces.c
04634     proces.c:             if ( Normalize(BHEAD m) ) {
04635 1060 proces.c:             if ( Normalize(BHEAD r) ) {
04636 1126?proces.c:             if ( Normalize(BHEAD term) ) {

```

```

04637     proces.c:             if ( Normalize(BHEAD AT.WorkPointer) ) goto PasErr;
04638 2308!proces.c:         if ( ( retnorm = Normalize(BHEAD term) ) != 0 ) {
04639     proces.c:             ReNumber(BHEAD term); Normalize(BHEAD term);
04640     proces.c:             if ( Normalize(BHEAD v) ) Terminate(-1);
04641     proces.c:             if ( Normalize(BHEAD w) ) { LowerSortLevel(); goto PolyCall; }
04642     proces.c:             if ( Normalize(BHEAD term) ) goto PolyCall;
04643 /*
04644 #ifdef EVALUATEGCD
04645
04646 WORD *EvaluateGcd(PHEAD WORD *subterm)
04647 {
04648     GETBIDENTITY
04649     WORD *oldworkpointer = AT.WorkPointer, *work1, *work2, *work3;
04650     WORD *t, *tt, *ttt, *t1, *t2, *t3, *t4, *tstop;
04651     WORD ct, nnum;
04652     UWORD gcdnum, stor;
04653     WORD *lnum=n_llnum+1;
04654     WORD *num1, *num2, *num3, *den1, *den2, *den3;
04655     WORD sizenum1, sizenum2, sizenum3, sizeden1, sizeden2, sizeden3;
04656     int i, isnumeric = 0, numarg = 0 /*, sizearg */;
04657     LONG size;
04658 /*
04659     Step 1: Look for -SNUMBER or -SYMBOL arguments.
04660             If encountered, treat everybody with it.
04661 */
04662     tt = subterm + subterm[1]; t = subterm + FUNHEAD;
04663
04664     while ( t < tt ) {
04665         numarg++;
04666         if ( *t == -SNUMBER ) {
04667             if ( t[1] == 0 ) {
04668 gcdzero:;
04669                 MLOCK(ErrorMessageLock);
04670                 MesPrint("Trying to take the GCD involving a zero term.");
04671                 MUNLOCK(ErrorMessageLock);
04672                 return(0);
04673             }
04674             gcdnum = ABS(t[1]);
04675             t1 = subterm + FUNHEAD;
04676             while ( gcdnum > 1 && t1 < tt ) {
04677                 if ( *t1 == -SNUMBER ) {
04678                     stor = ABS(t1[1]);
04679                     if ( stor == 0 ) goto gcdzero;
04680                     if ( GcdLong(BHEAD (UWORD *)&stor,1,(UWORD *)&gcdnum,1,
04681                             (UWORD *)lnum,&nnum) ) goto FromGCD;
04682                     gcdnum = lnum[0];
04683                     t1 += 2;
04684                     continue;
04685                 }
04686                 else if ( *t1 == -SYMBOL ) goto gcdisone;
04687                 else if ( *t1 < 0 ) goto gcdillegal;
04688 /*
04689                 Now we have to go through all the terms in the argument.
04690                 This includes long numbers.
04691 */
04692                 ttt = t1 + *t1;
04693                 ct = *ttt; *ttt = 0;
04694                 if ( t1[1] != 0 ) { /* First normalize the argument */
04695                     t1 = PolyNormPoly(BHEAD t1+ARGHEAD);
04696                 }
04697                 else t1 += ARGHEAD;
04698                 while ( *t1 ) {
04699                     t1 += *t1;
04700                     i = ABS(t1[-1]);
04701                     t2 = t1 - i;
04702                     i = (i-1)/2;
04703                     t3 = t2+i-1;
04704                     while ( t3 > t2 && *t3 == 0 ) { t3--; i--; }
04705                     if ( GcdLong(BHEAD (UWORD *)t2,(WORD)i,(UWORD *)&gcdnum,1,
04706                             (UWORD *)lnum,&nnum) ) {
04707                         *ttt = ct;
04708                         goto FromGCD;
04709                     }
04710                     gcdnum = lnum[0];
04711                     if ( gcdnum == 1 ) {
04712                         *ttt = ct;
04713                         goto gcdisone;
04714                     }
04715                 }
04716                 *ttt = ct;
04717                 t1 = ttt;
04718                 AT.WorkPointer = oldworkpointer;
04719             }
04720             if ( gcdnum == 1 ) goto gcdisone;
04721             oldworkpointer[0] = 4;
04722             oldworkpointer[1] = gcdnum;
04723             oldworkpointer[2] = 1;

```

```

04724         oldworkpointer[3] = 3;
04725         oldworkpointer[4] = 0;
04726         AT.WorkPointer = oldworkpointer + 5;
04727         return(oldworkpointer);
04728     }
04729     else if ( *t == -SYMBOL ) {
04730         t1 = subterm + FUNHEAD;
04731         i = t[1];
04732         while ( t1 < tt ) {
04733             if ( *t1 == -SNUMBER ) goto gcdisone;
04734             if ( *t1 == -SYMBOL ) {
04735                 if ( t1[1] != i ) goto gcdisone;
04736                 t1 += 2; continue;
04737             }
04738             if ( *t1 < 0 ) goto gcdillegal;
04739             ttt = t1 + *t1;
04740             ct = *ttt; *ttt = 0;
04741             if ( t1[1] != 0 ) { /* First normalize the argument */
04742                 t2 = PolyNormPoly(BHEAD t1+ARGHEAD);
04743             }
04744             else t2 = t1 + ARGHEAD;
04745             while ( *t2 ) {
04746                 t3 = t2+1;
04747                 t2 = t2 + *t2;
04748                 tstop = t2 - ABS(t2[-1]);
04749                 while ( t3 < tstop ) {
04750                     if ( *t3 != SYMBOL ) {
04751                         *ttt = ct;
04752                         goto gcdillegal;
04753                     }
04754                     t4 = t3 + 2;
04755                     t3 += t3[1];
04756                     while ( t4 < t3 ) {
04757                         if ( *t4 == i && t4[1] > 0 ) goto nextterminarg;
04758                         t4 += 2;
04759                     }
04760                 }
04761                 *ttt = ct;
04762                 goto gcdisone;
04763 nextterminarg:;
04764             }
04765             *ttt = ct;
04766             t1 = ttt;
04767             AT.WorkPointer = oldworkpointer;
04768         }
04769         oldworkpointer[0] = 8;
04770         oldworkpointer[1] = SYMBOL;
04771         oldworkpointer[2] = 4;
04772         oldworkpointer[3] = t[1];
04773         oldworkpointer[4] = 1;
04774         oldworkpointer[5] = 1;
04775         oldworkpointer[6] = 1;
04776         oldworkpointer[7] = 3;
04777         oldworkpointer[8] = 0;
04778         AT.WorkPointer = oldworkpointer+9;
04779         return(oldworkpointer);
04780     }
04781     else if ( *t < 0 ) {
04782 gcdillegal:;
04783         MLOCK(ErrorMessageLock);
04784         MesPrint("Illegal object in gcd_ function. Object not a number or a symbol.");
04785         MUNLOCK(ErrorMessageLock);
04786         goto FromGCD;
04787     }
04788     else if ( ABS(t[*t-1]) == *t-ARGHEAD-1 ) isnumeric = numarg;
04789     else if ( t[1] != 0 ) {
04790         ttt = t + *t; ct = *ttt; *ttt = 0;
04791         t = PolyNormPoly(BHEAD t+ARGHEAD);
04792         *ttt = ct;
04793         if ( t[*t] == 0 && ABS(t[*t-1]) == *t-ARGHEAD-1 ) isnumeric = numarg;
04794         AT.WorkPointer = oldworkpointer;
04795         t = ttt;
04796     }
04797     t += *t;
04798 }
04799 /*
04800     At this point there are only generic arguments.
04801     There are however still two cases:
04802     1: There is an argument that is purely numerical
04803        In that case we have to take the gcd of all coefficients
04804     2: All arguments are nontrivial polynomials.
04805        Here we don't worry so much about the factor. (???)
04806     We know whether case 1 occurs when isnumeric > 0.
04807     We can look up numarg to get a good starting value.
04808 */
04809     AT.WorkPointer = oldworkpointer;
04810     if ( isnumeric ) {

```

```

04811     t = subterm + FUNHEAD;
04812     for ( i = 1; i < isnumeric; i++ ) {
04813         NEXTARG(t);
04814     }
04815     if ( t[1] != 0 ) { /* First normalize the argument */
04816         ttt = t + *t; ct = *ttt; *ttt = 0;
04817         t = PolyNormPoly(BHEAD t+ARGHEAD);
04818         *ttt = ct;
04819     }
04820     t += *t;
04821     i = (ABS(t[-1])-1)/2;
04822     den1 = t - 1 - i;
04823     num1 = den1 - i;
04824     sizenum1 = sizeden1 = i;
04825     while ( sizenum1 > 1 && num1[sizenum1-1] == 0 ) sizenum1--;
04826     while ( sizeden1 > 1 && den1[sizeden1-1] == 0 ) sizeden1--;
04827     work1 = AT.WorkPointer+1; work2 = work1+sizenum1;
04828     for ( i = 0; i < sizenum1; i++ ) work1[i] = num1[i];
04829     for ( i = 0; i < sizeden1; i++ ) work2[i] = den1[i];
04830     num1 = work1; den1 = work2;
04831     AT.WorkPointer = work2 = work2 + sizeden1;
04832     t = subterm + FUNHEAD;
04833     while ( t < tt ) {
04834         ttt = t + *t; ct = *ttt; *ttt = 0;
04835         if ( t[1] != 0 ) {
04836             t = PolyNormPoly(BHEAD t+ARGHEAD);
04837         }
04838         else t += ARGHEAD;
04839         while ( *t ) {
04840             t += *t;
04841             i = (ABS(t[-1])-1)/2;
04842             den2 = t - 1 - i;
04843             num2 = den2 - i;
04844             sizenum2 = sizeden2 = i;
04845             while ( sizenum2 > 1 && num2[sizenum2-1] == 0 ) sizenum2--;
04846             while ( sizeden2 > 1 && den2[sizeden2-1] == 0 ) sizeden2--;
04847             num3 = AT.WorkPointer;
04848             if ( GcdLong(BHEAD (UWORD *)num2,sizenum2,(UWORD *)num1,sizenum1,
04849                 (UWORD *)num3,&sizenum3) ) goto FromGCD;
04850             sizenum1 = sizenum3;
04851             for ( i = 0; i < sizenum1; i++ ) num1[i] = num3[i];
04852             den3 = AT.WorkPointer;
04853             if ( GcdLong(BHEAD (UWORD *)den2,sizeden2,(UWORD *)den1,sizeden1,
04854                 (UWORD *)den3,&sizeden3) ) goto FromGCD;
04855             sizeden1 = sizeden3;
04856             for ( i = 0; i < sizeden1; i++ ) den1[i] = den3[i];
04857             if ( sizenum1 == 1 && num1[0] == 1 && sizeden1 == 1 && den1[1] == 1 )
04858                 goto gcdisone;
04859         }
04860         *ttt = ct;
04861         t = ttt;
04862         AT.WorkPointer = work2;
04863     }
04864     AT.WorkPointer = oldworkpointer;
04865     /*
04866     Now copy the GCD to the 'output'
04867     */
04868     if ( sizenum1 > sizeden1 ) {
04869         while ( sizenum1 > sizeden1 ) den1[sizeden1++] = 0;
04870     }
04871     else if ( sizenum1 < sizeden1 ) {
04872         while ( sizenum1 < sizeden1 ) num1[sizenum1++] = 0;
04873     }
04874     t = oldworkpointer;
04875     i = 2*sizenum1+1;
04876     *t++ = i+1;
04877     if ( num1 != t ) { NCOPY(t,num1,sizenum1); }
04878     else t += sizenum1;
04879     if ( den1 != t ) { NCOPY(t,den1,sizeden1); }
04880     else t += sizeden1;
04881     *t++ = i;
04882     *t++ = 0;
04883     AT.WorkPointer = t;
04884     return(oldworkpointer);
04885 }
04886 /*
04887 Now the real stuff with only polynomials.
04888 Pick up the shortest term to start.
04889 We are a bit brutish about this.
04890 */
04891 t = subterm + FUNHEAD;
04892 AT.WorkPointer += AM.MaxTer/sizeof(WORD);
04893 work2 = AT.WorkPointer;
04894 /*
04895 sizearg = subterm[1];
04896 */
04897 i = 0; work3 = 0;

```



```

04898     while ( t < tt ) {
04899         i++;
04900         work1 = AT.WorkPointer;
04901         ttt = t + *t; ct = *ttt; *ttt = 0;
04902         t = PolyNormPoly(BHEAD t+ARGHEAD);
04903         if ( *work1 < AT.WorkPointer-work1 ) {
04904             /*
04905                 sizearg = AT.WorkPointer-work1;
04906             */
04907                 numarg = i;
04908                 work3 = work1;
04909             }
04910             *ttt = ct; t = ttt;
04911         }
04912         *AT.WorkPointer++ = 0;
04913     /*
04914         We have properly normalized arguments and the shortest is indicated in work3
04915     */
04916     work1 = work3;
04917     while ( *work2 ) {
04918         if ( work2 != work3 ) {
04919             work1 = PolyGCD2(BHEAD work1,work2);
04920         }
04921         while ( *work2 ) work2 += *work2;
04922         work2++;
04923     }
04924     work2 = work1;
04925     while ( *work2 ) work2 += *work2;
04926     size = work2 - work1 + 1;
04927     t = oldworkpointer;
04928     NCOPY(t,work1,size);
04929     AT.WorkPointer = t;
04930     return(oldworkpointer);
04931
04932 gcdisone;;
04933     oldworkpointer[0] = 4;
04934     oldworkpointer[1] = 1;
04935     oldworkpointer[2] = 1;
04936     oldworkpointer[3] = 3;
04937     oldworkpointer[4] = 0;
04938     AT.WorkPointer = oldworkpointer+5;
04939     return(oldworkpointer);
04940 FromGCD:
04941     MLOCK(ErrorMessageLock);
04942     MesCall("EvaluateGcd");
04943     MUNLOCK(ErrorMessageLock);
04944     return(0);
04945 }
04946
04947 #endif
04948
04949 /*
04950     #] EvaluateGcd :
04951     #[ TreatPolyRatFun :
04952
04953     if ( AR.PolyFunExp == 1 ) we have to trim the contents of the polyratfun
04954     down to its most divergent term and give it coefficient +1. This is done
04955     by taking the terms with the least power in the variable in the numerator
04956     and in the denominator and then combine them.
04957     Answer is either PolyRatFun(ep^n,1) or PolyRatFun(1,1) or PolyRatFun(1,ep^n)
04958 */
04959
04960 int TreatPolyRatFun(PHEAD WORD *prf)
04961 {
04962     WORD *t, *tstop, *r, *rstop, *m, *mstop;
04963     WORD expl = MAXPOWER, exp2 = MAXPOWER;
04964     t = prf+FUNHEAD;
04965     if ( *t < 0 ) {
04966         if ( *t == -SYMBOL && t[1] == AR.PolyFunVar ) {
04967             if ( expl > 1 ) expl = 1;
04968             t += 2;
04969         }
04970         else {
04971             if ( expl > 0 ) expl = 0;
04972             NEXTARG(t)
04973         }
04974     }
04975     else {
04976         tstop = t + *t;
04977         t += ARGHEAD;
04978         while ( t < tstop ) {
04979             /*
04980                 Now look for the minimum power of AR.PolyFunVar
04981             */
04982                 r = t+1;
04983                 t += *t;
04984                 rstop = t - ABS(t[-1]);

```

```

04985         while ( r < rstop ) {
04986             if ( *r != SYMBOL ) { r += r[1]; continue; }
04987             m = r;
04988             mstop = m + m[1];
04989             m += 2;
04990             while ( m < mstop ) {
04991                 if ( *m == AR.PolyFunVar ) {
04992                     if ( m[1] < exp1 ) exp1 = m[1];
04993                     break;
04994                 }
04995                 m += 2;
04996             }
04997             if ( m == mstop ) {
04998                 if ( exp1 > 0 ) exp1 = 0;
04999             }
05000             break;
05001         }
05002         if ( r == rstop ) {
05003             if ( exp1 > 0 ) exp1 = 0;
05004         }
05005     }
05006     t = tstop;
05007 }
05008 if ( *t < 0 ) {
05009     if ( *t == -SYMBOL && t[1] == AR.PolyFunVar ) {
05010         if ( exp2 > 1 ) exp2 = 1;
05011     }
05012     else {
05013         if ( exp2 > 0 ) exp2 = 0;
05014     }
05015 }
05016 else {
05017     tstop = t + *t;
05018     t += ARGHEAD;
05019     while ( t < tstop ) {
05020         /*
05021             Now look for the minimum power of AR.PolyFunVar
05022         */
05023         r = t+1;
05024         t += *t;
05025         rstop = t - ABS(t[-1]);
05026         while ( r < rstop ) {
05027             if ( *r != SYMBOL ) { r += r[1]; continue; }
05028             m = r;
05029             mstop = m + m[1];
05030             m += 2;
05031             while ( m < mstop ) {
05032                 if ( *m == AR.PolyFunVar ) {
05033                     if ( m[1] < exp2 ) exp2 = m[1];
05034                     break;
05035                 }
05036                 m += 2;
05037             }
05038             if ( m == mstop ) {
05039                 if ( exp2 > 0 ) exp2 = 0;
05040             }
05041             break;
05042         }
05043         if ( r == rstop ) {
05044             if ( exp2 > 0 ) exp2 = 0;
05045         }
05046     }
05047 }
05048 /*
05049     Now we can compose the output.
05050     Notice that the output can never be longer than the input provided
05051     we never can have arguments that consist of just a function.
05052 */
05053 exp1 = exp1-exp2;
05054 /* if ( exp1 > 0 ) exp1 = 0; */
05055 t = prf+FUNHEAD;
05056 if ( exp1 == 0 ) {
05057     *t++ = -SNUMBER; *t++ = 1;
05058     *t++ = -SNUMBER; *t++ = 1;
05059 }
05060 else if ( exp1 > 0 ) {
05061     if ( exp1 == 1 ) {
05062         *t++ = -SYMBOL; *t++ = AR.PolyFunVar;
05063     }
05064     else {
05065         *t++ = 8+ARGHEAD;
05066         *t++ = 0;
05067         FILLARG(t);
05068         *t++ = 8; *t++ = SYMBOL; *t++ = 4; *t++ = AR.PolyFunVar;
05069         *t++ = exp1; *t++ = 1; *t++ = 1; *t++ = 3;
05070     }
05071     *t++ = -SNUMBER; *t++ = 1;

```

```

05072     }
05073     else {
05074         *t++ = -SNUMBER; *t++ = 1;
05075         if ( expl == -1 ) {
05076             *t++ = -SYMBOL; *t++ = AR.PolyFunVar;
05077         }
05078         else {
05079             *t++ = 8+ARGHEAD;
05080             *t++ = 0;
05081             FILLARG(t);
05082             *t++ = 8; *t++ = SYMBOL; *t++ = 4; *t++ = AR.PolyFunVar;
05083             *t++ = -expl; *t++ = 1; *t++ = 1; *t++ = 3;
05084         }
05085     }
05086     prf[2] = 0; /* Clean */
05087     prf[1] = t - prf;
05088     return(0);
05089 }
05090
05091 /*
05092 #] TreatPolyRatFun :
05093 #[ DropCoefficient :
05094 */
05095
05096 void DropCoefficient(PHEAD WORD *term)
05097 {
05098     GETBIDENTITY
05099     WORD *t = term + *term;
05100     WORD n, na;
05101     n = t[-1]; na = ABS(n);
05102     t -= na;
05103     if ( n == 3 && t[0] == 1 && t[1] == 1 ) return;
05104     *AN.RepPoint = 1;
05105     t[0] = 1; t[1] = 1; t[2] = 3;
05106     *term -= (na-3);
05107 }
05108
05109 /*
05110 #] DropCoefficient :
05111 #[ DropSymbols :
05112 */
05113
05114 void DropSymbols(PHEAD WORD *term)
05115 {
05116     GETBIDENTITY
05117     WORD *tend = term + *term, *t1, *t2, *tstop;
05118     tstop = tend - ABS(tend[-1]);
05119     t1 = term+1;
05120     while ( t1 < tstop ) {
05121         if ( *t1 == SYMBOL ) {
05122             *AN.RepPoint = 1;
05123             t2 = t1+t1[1];
05124             while ( t2 < tend ) *t1++ = *t2++;
05125             *term = t1 - term;
05126             break;
05127         }
05128         t1 += t1[1];
05129     }
05130 }
05131
05132 /*
05133 #] DropSymbols :
05134 #[ SymbolNormalize :
05135 */
05136
05137 int SymbolNormalize(WORD *term)
05138 {
05139     GETIDENTITY
05140     WORD buffer[7*NORMSIZE], *t, *b, *bb, *tt, *m, *tstop;
05141     int i;
05142     b = buffer;
05143     *b++ = SYMBOL; *b++ = 2;
05144     t = term + *term;
05145     tstop = t - ABS(t[-1]);
05146     t = term + 1;
05147     while ( t < tstop ) { /* Step 1: collect symbols */
05148         if ( *t == SYMBOL && t < tstop ) {
05149             for ( i = 2; i < t[1]; i += 2 ) {
05150                 bb = buffer+2;
05151                 while ( bb < b ) {
05152                     if ( bb[0] == t[i] ) { /* add powers */
05153                         bb[1] += t[i+1];
05154                         if ( bb[1] > MAXPOWER || bb[1] < -MAXPOWER ) {
05155                             MLOCK(ErrorMessageLock);
05156                             MesPrint("Power in SymbolNormalize out of range");
05157                             MUNLOCK(ErrorMessageLock);
05158                             return(-1);
05159                         }
05160                     }
05161                 }
05162             }
05163         }
05164     }

```

```

05167         if ( bb[1] == 0 ) {
05168             b -= 2;
05169             while ( bb < b ) {
05170                 bb[0] = bb[2]; bb[1] = bb[3]; bb += 2;
05171             }
05172         }
05173         goto Nexti;
05174     }
05175     else if ( bb[0] > t[i] ) { /* insert it */
05176         m = b;
05177         while ( m > bb ) { m[1] = m[-1]; m[0] = m[-2]; m -= 2; }
05178         b += 2;
05179         bb[0] = t[i];
05180         bb[1] = t[i+1];
05181         goto Nexti;
05182     }
05183     bb += 2;
05184 }
05185 if ( bb >= b ) { /* add it to the end */
05186     *b++ = t[i]; *b++ = t[i+1];
05187 }
05188 Nexti;;
05189 }
05190 }
05191 else {
05192     MLOCK(ErrorMessageLock);
05193     MesPrint("Illegal term in SymbolNormalize");
05194     MUNLOCK(ErrorMessageLock);
05195     return(-1);
05196 }
05197 t += t[1];
05198 }
05199 buffer[1] = b - buffer;
05200 /*
05201 Veto negative powers
05202 */
05203 if ( AT.LeaveNegative == 0 ) {
05204     b = buffer; bb = b + b[1]; b += 3;
05205     while ( b < bb ) {
05206         if ( *b < 0 ) {
05207             MLOCK(ErrorMessageLock);
05208             MesPrint("Negative power in SymbolNormalize");
05209             MUNLOCK(ErrorMessageLock);
05210             return(-1);
05211         }
05212         b += 2;
05213     }
05214 }
05215 /*
05216 Now we use the fact that the new term will not be longer than the old one
05217 Actually it should be shorter when there is more than one subterm!
05218 Copy back.
05219 */
05220 i = buffer[1];
05221 b = buffer; tt = term + 1;
05222 if ( i > 2 ) { NCOPY(tt,b,i) }
05223 if ( tt < tstop ) {
05224     i = term[*term-1];
05225     if ( i < 0 ) i = -i;
05226     *term -= (tstop-tt);
05227     NCOPY(tt,tstop,i)
05228 }
05229 return(0);
05230 }
05231
05232 /*
05233 #] SymbolNormalize :
05234 #[ TestFunFlag :
05235
05236 Tests whether a function still has unsubstituted subexpressions
05237 This function has its dirtyflag on!
05238 */
05239
05240 int TestFunFlag(PHEAD WORD *tfun)
05241 {
05242     WORD *t, *tstop, *r, *rstop, *m, *mstop;
05243     if ( functions[*tfun-FUNCTION].spec <= 0 ) return(0);
05244     tstop = tfun + tfun[1];
05245     t = tfun + FUNHEAD;
05246     while ( t < tstop ) {
05247         if ( *t < 0 ) { NEXTARG(t); continue; }
05248         rstop = t + *t;
05249         if ( t[1] == 0 ) { t = rstop; continue; }
05250         r = t + ARGHEAD;
05251         while ( r < rstop ) { /* Here we loop over terms */
05252             m = r+1; mstop = r+r; mstop -= ABS(mstop[-1]);
05253             while ( m < mstop ) { /* Loop over the subterms */

```

```

05254         if ( *m == SUBEXPRESSION || *m == EXPRESSION || *m == DOLLAREXPRESSION ) return(1);
05255         if ( ( *m >= FUNCTION ) && ( ( m[2] & DIRTYFLAG ) == DIRTYFLAG )
05256             && ( *m != REPLACEMENT ) && TestFunFlag(BHEAD m) ) return(1);
05257         m += m[1];
05258     }
05259     r += *r;
05260 }
05261 t += *t;
05262 }
05263 return(0);
05264 }
05265
05266 /*
05267 #] TestFunFlag :
05268 #[ BracketNormalize :
05269 */
05270
05271 WORD BracketNormalize(PHEAD WORD *term)
05272 {
05273     WORD *stop = term+*term-3, *t, *tt, *tstart, *r;
05274     WORD *oldwork = AT.WorkPointer;
05275     WORD *termout;
05276     WORD i, ii, j;
05277     termout = AT.WorkPointer = term+*term;
05278 /*
05279 First collect all functions and sort them
05280 */
05281 tt = termout+1; t = term+1;
05282 while ( t < stop ) {
05283     if ( *t >= FUNCTION ) { i = t[1]; NCOPY(tt,t,i); }
05284     else t += t[1];
05285 }
05286 if ( tt > termout+1 && tt-termout-1 > termout[2] ) { /* sorting */
05287     r = termout+1; ii = tt-r;
05288     for ( i = 0; i < ii-FUNHEAD; i += FUNHEAD ) { /* Bubble sort */
05289         for ( j = i+FUNHEAD; j > 0; j -= FUNHEAD ) {
05290             if ( functions[r[j-FUNHEAD]-FUNCTION].commute
05291                 && functions[r[j]-FUNCTION].commute == 0 ) break;
05292             if ( r[j-FUNHEAD] > r[j] ) EXCH(r[j-FUNHEAD],r[j])
05293             else break;
05294         }
05295     }
05296 }
05297 tstart = tt; t = term + 1; *tt++ = DELTA; *tt++ = 2;
05298 while ( t < stop ) {
05299     if ( *t == DELTA ) { i = t[1]-2; t += 2; tstart[1] += i; NCOPY(tt,t,i); }
05300     else t += t[1];
05301 }
05302 if ( tstart[1] > 2 ) {
05303     for ( r = tstart+2; r < tstart+tstart[1]; r += 2 ) {
05304         if ( r[0] > r[1] ) EXCH(r[0],r[1])
05305     }
05306 }
05307 if ( tstart[1] > 4 ) { /* sorting */
05308     r = tstart+2; ii = tstart[1]-2;
05309     for ( i = 0; i < ii-2; i += 2 ) { /* Bubble sort */
05310         for ( j = i+2; j > 0; j -= 2 ) {
05311             if ( r[j-2] > r[j] ) {
05312                 EXCH(r[j-2],r[j])
05313                 EXCH(r[j-1],r[j+1])
05314             }
05315             else if ( r[j-2] < r[j] ) break;
05316             else {
05317                 if ( r[j-1] > r[j+1] ) EXCH(r[j-1],r[j+1])
05318                 else break;
05319             }
05320         }
05321     }
05322 }
05323 tt = tstart+tstart[1];
05324 }
05325 else if ( tstart[1] == 2 ) { tt = tstart; }
05326 else tt = tstart+4;
05327
05328 tstart = tt; t = term + 1; *tt++ = INDEX; *tt++ = 2;
05329 while ( t < stop ) {
05330     if ( *t == INDEX ) { i = t[1]-2; t += 2; tstart[1] += i; NCOPY(tt,t,i); }
05331     else t += t[1];
05332 }
05333 if ( tstart[1] >= 4 ) { /* sorting */
05334     r = tstart+2; ii = tstart[1]-2;
05335     for ( i = 0; i < ii-1; i += 1 ) { /* Bubble sort */
05336         for ( j = i+1; j > 0; j -= 1 ) {
05337             if ( r[j-1] > r[j] ) EXCH(r[j-1],r[j])
05338             else break;
05339         }
05340     }

```

```

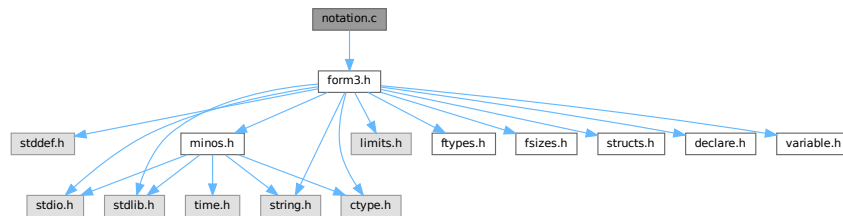
05341         tt = tstart+tstart[1];
05342     }
05343     else if ( tstart[1] == 2 ) { tt = tstart; }
05344     else tt = tstart+3;
05345
05346     tstart = tt; t = term + 1; *tt++ = DOTPRODUCT; *tt++ = 2;
05347     while ( t < stop ) {
05348         if ( *t == DOTPRODUCT ) { i = t[1]-2; t += 2; tstart[1] += i; NCOPY(tt,t,i); }
05349         else t += t[1];
05350     }
05351     if ( tstart[1] > 5 ) { /* sorting */
05352         r = tstart+2; ii = tstart[1]-2;
05353         for ( i = 0; i < ii; i += 3 ) {
05354             if ( r[i] > r[i+1] ) EXCH(r[i],r[i+1])
05355         }
05356         for ( i = 0; i < ii-3; i += 3 ) { /* Bubble sort */
05357             for ( j = i+3; j > 0; j -= 3 ) {
05358                 if ( r[j-3] < r[j] ) break;
05359                 if ( r[j-3] > r[j] ) {
05360                     EXCH(r[j-3],r[j])
05361                     EXCH(r[j-2],r[j+1])
05362                 }
05363                 else {
05364                     if ( r[j-2] > r[j+1] ) EXCH(r[j-2],r[j+1])
05365                     else break;
05366                 }
05367             }
05368         }
05369         tt = tstart+tstart[1];
05370     }
05371     else if ( tstart[1] == 2 ) { tt = tstart; }
05372     else {
05373         if ( tstart[2] > tstart[3] ) EXCH(tstart[2],tstart[3])
05374         tt = tstart+5;
05375     }
05376
05377     tstart = tt; t = term + 1; *tt++ = SYMBOL; *tt++ = 2;
05378     while ( t < stop ) {
05379         if ( *t == SYMBOL ) { i = t[1]-2; t += 2; tstart[1] += i; NCOPY(tt,t,i); }
05380         else t += t[1];
05381     }
05382     if ( tstart[1] > 4 ) { /* sorting */
05383         r = tstart+2; ii = tstart[1]-2;
05384         for ( i = 0; i < ii-2; i += 2 ) { /* Bubble sort */
05385             for ( j = i+2; j > 0; j -= 2 ) {
05386                 if ( r[j-2] > r[j] ) EXCH(r[j-2],r[j])
05387                 else break;
05388             }
05389         }
05390         tt = tstart+tstart[1];
05391     }
05392     else if ( tstart[1] == 2 ) { tt = tstart; }
05393     else tt = tstart+4;
05394
05395     tstart = tt; t = term + 1; *tt++ = SETSET; *tt++ = 2;
05396     while ( t < stop ) {
05397         if ( *t == SETSET ) { i = t[1]-2; t += 2; tstart[1] += i; NCOPY(tt,t,i); }
05398         else t += t[1];
05399     }
05400     if ( tstart[1] > 4 ) { /* sorting */
05401         r = tstart+2; ii = tstart[1]-2;
05402         for ( i = 0; i < ii-2; i += 2 ) { /* Bubble sort */
05403             for ( j = i+2; j > 0; j -= 2 ) {
05404                 if ( r[j-2] > r[j] ) {
05405                     EXCH(r[j-2],r[j])
05406                     EXCH(r[j-1],r[j+1])
05407                 }
05408                 else break;
05409             }
05410         }
05411         tt = tstart+tstart[1];
05412     }
05413     else if ( tstart[1] == 2 ) { tt = tstart; }
05414     else tt = tstart+4;
05415     *tt++ = 1; *tt++ = 1; *tt++ = 3;
05416     t = term; i = *termout = tt - termout; tt = termout;
05417     NCOPY(t,tt,i);
05418     AT.WorkPointer = oldwork;
05419     return(0);
05420 }
05421
05422 /*
05423 #] BracketNormalize :
05424 */

```

4.88 notation.c File Reference

```
#include "form3.h"
```

Include dependency graph for notation.c:



Functions

- int [NormPolyTerm](#) (PHEAD WORD *term)
- int [ConvertToPoly](#) (PHEAD WORD *term, WORD *outterm, WORD *comlist, WORD par)
- int [LocalConvertToPoly](#) (PHEAD WORD *term, WORD *outterm, WORD startbuf, WORD par)
- int [ConvertFromPoly](#) (PHEAD WORD *term, WORD *outterm, WORD from, WORD to, WORD offset, WORD par)
- WORD [FindSubterm](#) (WORD *subterm)
- WORD [FindLocalSubterm](#) (PHEAD WORD *subterm, WORD startbuf)
- void [PrintSubtermList](#) (int from, int to)
- void [PrintExtraSymbol](#) (int num, WORD *terms, int par)
- WORD [FindSubexpression](#) (WORD *subexpr)
- int [ExtraSymFun](#) (PHEAD WORD *term, WORD level)
- int [PruneExtraSymbols](#) (WORD downto)

4.88.1 Detailed Description

Contains the functions that deal with the rewriting and manipulation of expressions/terms in polynomial representation.

Definition in file [notation.c](#).

4.88.2 Function Documentation

4.88.2.1 NormPolyTerm()

```
int NormPolyTerm (
    PHEAD WORD * term )
```

Definition at line 53 of file [notation.c](#).

4.88.2.2 ConvertToPoly()

```
int ConvertToPoly (
    PHEAD WORD * term,
    WORD * outterm,
    WORD * comlist,
    WORD par )
```

Definition at line 307 of file [notation.c](#).

4.88.2.3 LocalConvertToPoly()

```
int LocalConvertToPoly (
    PHEAD WORD * term,
    WORD * outterm,
    WORD startebuf,
    WORD par )
```

Converts a generic term to polynomial notation in which there are only symbols and brackets. During conversion there will be only symbols. Brackets are stripped. Objects that need 'translation' are put inside a special compiler buffer and represented by a symbol. The numbering of the extra symbols is down from the maximum. In principle there can be a problem when running into the already assigned ones. This uses the FindTree for searching in the global tree and then looks further in the AT.ebufnum. This allows fully parallel processing. Hence we need no locks. Cannot be used in the same module as ConvertToPoly.

Definition at line 510 of file [notation.c](#).

Referenced by [poly_factorize_expression\(\)](#).

4.88.2.4 ConvertFromPoly()

```
int ConvertFromPoly (
    PHEAD WORD * term,
    WORD * outterm,
    WORD from,
    WORD to,
    WORD offset,
    WORD par )
```

Definition at line 660 of file [notation.c](#).

4.88.2.5 FindSubterm()

```
WORD FindSubterm (
    WORD * subterm )
```

Definition at line 773 of file [notation.c](#).

4.88.2.6 FindLocalSubterm()

```
WORD FindLocalSubterm (
    PHEAD WORD * subterm,
    WORD startebuf )
```

Definition at line [842](#) of file [notation.c](#).

4.88.2.7 PrintSubtermList()

```
void PrintSubtermList (
    int from,
    int to )
```

Definition at line [895](#) of file [notation.c](#).

4.88.2.8 PrintExtraSymbol()

```
void PrintExtraSymbol (
    int num,
    WORD * terms,
    int par )
```

Definition at line [1007](#) of file [notation.c](#).

4.88.2.9 FindSubexpression()

```
WORD FindSubexpression (
    WORD * subexpr )
```

Definition at line [1094](#) of file [notation.c](#).

4.88.2.10 ExtraSymFun()

```
int ExtraSymFun (
    PHEAD WORD * term,
    WORD level )
```

Definition at line [1146](#) of file [notation.c](#).

4.88.2.11 PruneExtraSymbols()

```
int PruneExtraSymbols (
    WORD downto )
```

Definition at line [1214](#) of file [notation.c](#).

4.89 notation.c

[Go to the documentation of this file.](#)

```

00001
00006 /* #[ License : */
00007 /*
00008 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00009 *   When using this file you are requested to refer to the publication
00010 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00011 *   This is considered a matter of courtesy as the development was paid
00012 *   for by FOM the Dutch physics granting agency and we would like to
00013 *   be able to track its scientific use to convince FOM of its value
00014 *   for the community.
00015 *
00016 *   This file is part of FORM.
00017 *
00018 *   FORM is free software: you can redistribute it and/or modify it under the
00019 *   terms of the GNU General Public License as published by the Free Software
00020 *   Foundation, either version 3 of the License, or (at your option) any later
00021 *   version.
00022 *
00023 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00024 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00025 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00026 *   details.
00027 *
00028 *   You should have received a copy of the GNU General Public License along
00029 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00030 */
00031 /* #[ License : */
00032 /*
00033 *   #[ Includes :
00034 */
00035
00036 #include "form3.h"
00037
00038 /*
00039 *   #[ Includes :
00040 *   #[ NormPolyTerm :
00041
00042 *   Brings a term to normal form.
00043
00044 *   This routine knows objects of the following types:
00045 *   SYMBOL
00046 *   HAAKJE
00047 *   SNUMBER
00048 *   LNUMBER
00049 *   The SNUMBER and LNUMBER are worked into the coefficient.
00050 *   One of the essences here is that everything can be done in place.
00051 */
00052
00053 int NormPolyTerm(PHEAD WORD *term)
00054 {
00055     WORD *tcoef, ncoef, *tstop, *tfill, *t, *tt;
00056     int equal, i;
00057     WORD *r1, *r2, *r3, *r4, *r5, *rfirst, rv;
00058     WORD *lnum, nnum; /* Scratch, originally for factorials */
00059 /*
00060 *   One: find the coefficient
00061 */
00062     tcoef = term+*term;
00063     ncoef = tcoef[-1];
00064     tstop = tcoef - ABS(tcoef[-1]);
00065     tfill = t = term + 1;
00066     rfirst = 0;
00067     if ( t >= tstop ) { return(*term); }
00068     while ( t < tstop ) {
00069         switch ( *t ) {
00070             case SYMBOL:
00071                 if ( rfirst == 0 ) {
00072 /*
00073 *   Here we only need to sort
00074 *   1: assume no equals. Bubble.
00075 */
00076                     rfirst = t;
00077                     r2 = rfirst+4; tt = r3 = t + t[1]; equal = 0;
00078                     while ( r2 < r3 ) {
00079                         r1 = r2 - 2;
00080                         if ( *r2 > *r1 ) { r2 += 2; continue; }
00081                         if ( *r2 == *r1 ) { r2 += 2; equal = 1; continue; }
00082                         rv = *r1; *r1 = *r2; *r2 = rv;
00083                         r1 -= 2; r2 -= 2; r4 = r2 + 2;
00084                         while ( r1 > t ) {
00085                             if ( *r2 >= *r1 ) { r2 = r4; break; }
00086                             rv = *r1; *r1 = *r2; *r2 = rv;

```

```

00087         r1 -= 2; r2 -= 2;
00088     }
00089 }
00090 /*
00091     2: hunt down the equal objects
00092     postpone eliminating zero powers.
00093 */
00094     if ( equal ) {
00095         r1 = t+2; r2 = r1+2;
00096         while ( r2 < r3 ) {
00097             if ( *r1 == *r2 ) {
00098                 r1[1] += r2[1];
00099                 r4 = r2+2;
00100                 while ( r4 < r3 ) *r2++ = *r4++;
00101                 t[1] -= 2;
00102                 r2 = r1 + 2; r3 -= 2;
00103             }
00104         }
00105     }
00106 }
00107 else {
00108     /*
00109         Here we only need to insert
00110     */
00111     r1 = t + 2; tt = r3 = t + t[1];
00112     while ( r1 < r3 ) {
00113         r2 = rfirst+2; r4 = rfirst + rfirst[1];
00114         while ( r2 < r4 ) {
00115             if ( *r1 == *r2 ) {
00116                 r2[1] += r1[1];
00117                 break;
00118             }
00119             else if ( *r2 > *r1 ) {
00120                 r5 = r4;
00121                 while ( r5 > r2 ) { r5[1] = r5[-1]; r5[0] = r5[-2]; r5 -= 2; }
00122                 rfirst[1] += 2;
00123                 *r2 = *r1; r2[1] = r1[1];
00124                 break;
00125             }
00126             r2 += 2;
00127         }
00128         if ( r2 == r4 ) {
00129             rfirst[1] += 2;
00130             *r2++ = *r1++; *r2++ = *r1++;
00131         }
00132         else r1 += 2;
00133     }
00134 }
00135 t = tt;
00136 break;
00137 case HAAKJE: /* Here we skip brackets */
00138     t += t[1];
00139     break;
00140 case SNUMBER:
00141     if ( t[2] < 0 ) {
00142         t[2] = -t[2];
00143         if ( t[3] & 1 ) ncoef = -ncoef;
00144     }
00145     else if ( t[2] == 0 ) {
00146         if ( t[3] < 0 ) goto NormInf;
00147         goto NormZero;
00148     }
00149     lnum = TermMalloc("lnum");
00150     lnum[0] = t[2];
00151     nnum = 1;
00152     if ( t[3] && RaisPow(BHEAD (UWORD *)lnum,&nnum, (UWORD) (ABS(t[3])) ) ) goto FromNorm;
00153     ncoef = REDLENG(ncoef);
00154     if ( t[3] < 0 ) {
00155         if ( Divvy(BHEAD (UWORD *)tstop,&ncoef, (UWORD *)lnum,nnum) )
00156             goto FromNorm;
00157     }
00158     else if ( t[3] > 0 ) {
00159         if ( Mully(BHEAD (UWORD *)tstop,&ncoef, (UWORD *)lnum,nnum) )
00160             goto FromNorm;
00161     }
00162     ncoef = INCLENG(ncoef);
00163     t += t[1];
00164     TermFree(lnum,"lnum");
00165     break;
00166 case LNUMBER:
00167     ncoef = REDLENG(ncoef);
00168     if ( Mully(BHEAD (UWORD *)tstop,&ncoef, (UWORD *) (t+3),t[2]) ) goto FromNorm;
00169     ncoef = INCLENG(ncoef);
00170     t += t[1];
00171     break;
00172 default:
00173     MLOCK(ErrorMessageLock);

```

```

00174         MesPrint("Illegal code in NormPolyTerm");
00175         MUNLOCK(ErrorMessageLock);
00176         Terminate(-1);
00177         break;
00178     }
00179 }
00180 /*
00181 Now we try to eliminate objects to the power zero.
00182 */
00183 if ( rfirst ) {
00184     r2 = rfirst+2;
00185     r3 = rfirst + rfirst[1];
00186     while ( r2 < r3 ) {
00187         if ( r2[1] == 0 ) {
00188             r1 = r2 + 2;
00189             while ( r1 < r3 ) { r1[-2] = r1[0]; r1[-1] = r1[1]; r1 += 2; }
00190             r3 -= 2;
00191             rfirst[1] -= 2;
00192         }
00193         else { r2 += 2; }
00194     }
00195     if ( rfirst[1] < 4 ) rfirst = 0;
00196 }
00197 /*
00198 Finally we put the term together
00199 */
00200 if ( rfirst ) {
00201     i = rfirst[1];
00202     NCOPY(tfill,rfirst,i)
00203 }
00204 i = ABS(ncoef)-1;
00205 NCOPY(tfill,tstop,i)
00206 *tfill++ = ncoef;
00207 *term = tfill - term;
00208 return(*term);
00209 NormZero:
00210 *term = 0;
00211 return(0);
00212 NormInf:
00213 MLOCK(ErrorMessageLock);
00214 MesPrint("0^0 in NormPolyTerm");
00215 MUNLOCK(ErrorMessageLock);
00216 Terminate(-1);
00217 return(-1);
00218 FromNorm:
00219 MLOCK(ErrorMessageLock);
00220 MesCall("NormPolyTerm");
00221 MUNLOCK(ErrorMessageLock);
00222 Terminate(-1);
00223 return(-1);
00224 }
00225
00226 /*
00227 #] NormPolyTerm :
00228 #[ ComparePoly :
00229 */
00251 #ifdef WITHCOMPAREPOLY
00252
00253 WORD ComparePoly(WORD *term1, WORD *term2, WORD level)
00254 {
00255     WORD *t1, *t2, *t3, *t4, *tstop1, *tstop2;
00256     tstop1 = term1 + *term1;
00257     tstop1 -= ABS(tstop1[-1]);
00258     tstop2 = term2 + *term2;
00259     tstop2 -= ABS(tstop2[-1]);
00260     t1 = term1+1;
00261     t2 = term2+1;
00262     while ( t1 < tstop1 && t2 < tstop2 ) {
00263         if ( *t1 == *t2 ) {
00264             if ( *t1 == HAAKJE ) {
00265                 if ( t1[2] != t2[2] ) return(t2[2]-t1[2]);
00266                 t1 += t1[1]; t2 += t2[1];
00267             }
00268             else { /* must be type SYMBOL */
00269                 t3 = t1 + t1[1]; t4 = t2 + t2[1];
00270                 t1 += 2; t2 += 2;
00271                 while ( t1 < t3 && t2 < t4 ) {
00272                     if ( *t1 != *t2 ) return(*t2-*t1);
00273                     if ( t1[1] != t2[1] ) return(t2[1]-t1[1]);
00274                     t1 += 2; t2 += 2;
00275                 }
00276                 if ( t1 < t3 ) return(-1);
00277                 if ( t2 < t4 ) return(1);
00278             }
00279         }
00280         else return(*t2-*t1);
00281     }

```

```

00282     if ( t1 < tstop1 ) return(-1);
00283     if ( t2 < tstop2 ) return(1);
00284     return(0);
00285 }
00286
00287 #endif
00288
00289 /*
00290     #] ComparePoly :
00291     #[ ConvertToPoly :
00292 */
00305 static int FirstWarnConvertToPoly = 1;
00306
00307 int ConvertToPoly(PHEAD WORD *term, WORD *outterm, WORD *comlist, WORD par)
00308 {
00309     WORD *tout, *tstop, ncoef, *t, *r, *tt, *ttwo = 0;
00310     int i, action = 0;
00311     tt = term + *term;
00312     ncoef = ABS(tt[-1]);
00313     tstop = tt - ncoef;
00314     tout = outterm+1;
00315     t = term + 1;
00316     if ( comlist[2] == DOALL ) {
00317         while ( t < tstop ) {
00318             if ( *t == SYMBOL ) {
00319                 r = t+2;
00320                 t += t[1];
00321                 while ( r < t ) {
00322                     if ( r[1] > 0 ) {
00323                         *tout++ = SYMBOL;
00324                         *tout++ = 4;
00325                         *tout++ = r[0];
00326                         *tout++ = r[1];
00327                     }
00328                     else {
00329                         tout[1] = SYMBOL;
00330                         tout[2] = 4;
00331                         tout[3] = r[0];
00332                         tout[4] = -1;
00333                         i = FindSubterm(tout+1);
00334                         *tout++ = SYMBOL;
00335                         *tout++ = 4;
00336                         *tout++ = MAXVARIABLES-i;
00337                         *tout++ = -r[1];
00338                         action = 1;
00339                     }
00340                     r += 2;
00341                 }
00342             }
00343             else if ( *t == DOTPRODUCT ) {
00344                 r = t + 2;
00345                 t += t[1];
00346                 while ( r < t ) {
00347                     tout[1] = DOTPRODUCT;
00348                     tout[2] = 5;
00349                     tout[3] = r[0];
00350                     tout[4] = r[1];
00351                     if ( r[2] < 0 ) {
00352                         tout[5] = -1;
00353                     }
00354                     else {
00355                         tout[5] = 1;
00356                     }
00357                     i = FindSubterm(tout+1);
00358                     *tout++ = SYMBOL;
00359                     *tout++ = 4;
00360                     *tout++ = MAXVARIABLES-i;
00361                     *tout++ = ABS(r[2]);
00362                     r += 3;
00363                     action = 1;
00364                 }
00365             }
00366             else if ( *t == VECTOR ) {
00367                 r = t + 2;
00368                 t += t[1];
00369                 while ( r < t ) {
00370                     tout[1] = VECTOR;
00371                     tout[2] = 4;
00372                     tout[3] = r[0];
00373                     tout[4] = r[1];
00374                     i = FindSubterm(tout+1);
00375                     *tout++ = SYMBOL;
00376                     *tout++ = 4;
00377                     *tout++ = MAXVARIABLES-i;
00378                     *tout++ = 1;
00379                     r += 2;
00380                     action = 1;

```

```

00381     }
00382 }
00383 else if ( *t == INDEX ) {
00384     r = t + 2;
00385     t += t[1];
00386     while ( r < t ) {
00387         tout[1] = INDEX;
00388         tout[2] = 3;
00389         tout[3] = r[0];
00390         i = FindSubterm(tout+1);
00391         *tout++ = SYMBOL;
00392         *tout++ = 4;
00393         *tout++ = MAXVARIABLES-i;
00394         *tout++ = 1;
00395         r++;
00396         action = 1;
00397     }
00398 }
00399 else if ( *t == HAAKJE ) {
00400     if ( par ) {
00401         tout[0] = 1; tout[1] = 1; tout[2] = 3;
00402         *outterm = (tout+3)-outterm;
00403         if ( NormPolyTerm(BHEAD outterm) < 0 ) return(-1);
00404         tout = outterm + *outterm;
00405         tout -= 3;
00406         i = t[1]; NCOPY(tout,t,i);
00407         ttwo = tout-1;
00408     }
00409     else { t += t[1]; }
00410 }
00411 else if ( *t >= FUNCTION ) {
00412     i = FindSubterm(t);
00413     t += t[1];
00414     *tout++ = SYMBOL;
00415     *tout++ = 4;
00416     *tout++ = MAXVARIABLES-i;
00417     *tout++ = 1;
00418     action = 1;
00419 }
00420 else {
00421     if ( FirstWarnConvertToPoly ) {
00422         MLOCK(ErrorMessageLock);
00423         MesPrint("Illegal object in conversion to polynomial notation");
00424         MUNLOCK(ErrorMessageLock);
00425         FirstWarnConvertToPoly = 0;
00426     }
00427     return(-1);
00428 }
00429 }
00430 NCOPY(tout,tstop,ncoef)
00431 if ( ttwo ) {
00432     WORD hh = *ttwo;
00433     *ttwo = tout-ttwo;
00434     if ( ( i = NormPolyTerm(BHEAD ttwo) ) >= 0 ) i = action;
00435     tout = ttwo + *ttwo;
00436     *ttwo = hh;
00437     *outterm = tout - outterm;
00438 }
00439 else {
00440     *outterm = tout-outterm;
00441     if ( ( i = NormPolyTerm(BHEAD outterm) ) >= 0 ) i = action;
00442 }
00443 }
00444 else if ( comlist[2] == ONLYFUNCTIONS ) {
00445     while ( t < tstop ) {
00446         if ( *t >= FUNCTION ) {
00447             if ( comlist[1] == 3 ) {
00448                 i = FindSubterm(t);
00449                 t += t[1];
00450                 *tout++ = SYMBOL;
00451                 *tout++ = 4;
00452                 *tout++ = MAXVARIABLES-i;
00453                 *tout++ = 1;
00454                 action = 1;
00455             }
00456             else {
00457                 for ( i = 3; i < comlist[1]; i++ ) {
00458                     if ( *t == comlist[i] ) break;
00459                 }
00460                 if ( i < comlist[1] ) {
00461                     i = FindSubterm(t);
00462                     t += t[1];
00463                     *tout++ = SYMBOL;
00464                     *tout++ = 4;
00465                     *tout++ = MAXVARIABLES-i;
00466                     *tout++ = 1;
00467                     action = 1;

```

```

00468         }
00469         else {
00470             i = t[1]; NCOPY(tout,t,i);
00471         }
00472     }
00473 }
00474 else {
00475     i = t[1]; NCOPY(tout,t,i);
00476 }
00477 }
00478 NCOPY(tout,tstop,ncoef)
00479 *outterm = tout-outterm;
00480 Normalize(BHEAD outterm);
00481 i = action;
00482 }
00483 else {
00484     MLOCK(ErrorMessageLock);
00485     MesPrint("Illegal internal code in conversion to polynomial notation");
00486     MUNLOCK(ErrorMessageLock);
00487     i = -1;
00488 }
00489 return(i);
00490 }
00491
00492 /*
00493     #] ConvertToPoly :
00494     #[ LocalConvertToPoly :
00495 */
00510 int LocalConvertToPoly(PHEAD WORD *term, WORD *outterm, WORD startebuf, WORD par)
00511 {
00512     WORD *tout, *tstop, ncoef, *t, *r, *tt, *ttwo = 0;
00513     int i, action = 0;
00514     tt = term + *term;
00515     ncoef = ABS(tt[-1]);
00516     tstop = tt - ncoef;
00517     tout = outterm+1;
00518     t = term + 1;
00519     while ( t < tstop ) {
00520         if ( *t == SYMBOL ) {
00521             r = t+2;
00522             t += t[1];
00523             while ( r < t ) {
00524                 if ( r[1] > 0 ) {
00525                     *tout++ = SYMBOL;
00526                     *tout++ = 4;
00527                     *tout++ = r[0];
00528                     *tout++ = r[1];
00529                 }
00530                 else {
00531                     tout[1] = SYMBOL;
00532                     tout[2] = 4;
00533                     tout[3] = r[0];
00534                     tout[4] = -1;
00535                     i = FindLocalSubterm(BHEAD tout+1,startebuf);
00536                     *tout++ = SYMBOL;
00537                     *tout++ = 4;
00538                     *tout++ = MAXVARIABLES-i;
00539                     *tout++ = -r[1];
00540                     action = 1;
00541                 }
00542                 r += 2;
00543             }
00544         }
00545         else if ( *t == DOTPRODUCT ) {
00546             r = t + 2;
00547             t += t[1];
00548             while ( r < t ) {
00549                 tout[1] = DOTPRODUCT;
00550                 tout[2] = 5;
00551                 tout[3] = r[0];
00552                 tout[4] = r[1];
00553                 if ( r[2] < 0 ) {
00554                     tout[5] = -1;
00555                 }
00556                 else {
00557                     tout[5] = 1;
00558                 }
00559                 i = FindLocalSubterm(BHEAD tout+1,startebuf);
00560                 *tout++ = SYMBOL;
00561                 *tout++ = 4;
00562                 *tout++ = MAXVARIABLES-i;
00563                 *tout++ = ABS(r[2]);
00564                 r += 3;
00565                 action = 1;
00566             }
00567         }
00568         else if ( *t == VECTOR ) {

```

```

00569         r = t + 2;
00570         t += t[1];
00571         while ( r < t ) {
00572             tout[1] = VECTOR;
00573             tout[2] = 4;
00574             tout[3] = r[0];
00575             tout[4] = r[1];
00576             i = FindLocalSubterm(BHEAD tout+1,startebuf);
00577             *tout++ = SYMBOL;
00578             *tout++ = 4;
00579             *tout++ = MAXVARIABLES-i;
00580             *tout++ = 1;
00581             r += 2;
00582             action = 1;
00583         }
00584     }
00585     else if ( *t == INDEX ) {
00586         r = t + 2;
00587         t += t[1];
00588         while ( r < t ) {
00589             tout[1] = INDEX;
00590             tout[2] = 3;
00591             tout[3] = r[0];
00592             i = FindLocalSubterm(BHEAD tout+1,startebuf);
00593             *tout++ = SYMBOL;
00594             *tout++ = 4;
00595             *tout++ = MAXVARIABLES-i;
00596             *tout++ = 1;
00597             r++;
00598             action = 1;
00599         }
00600     }
00601     else if ( *t == HAAKJE ) {
00602         if ( par ) {
00603             tout[0] = 1; tout[1] = 1; tout[2] = 3;
00604             *outterm = (tout+3)-outterm;
00605             if ( NormPolyTerm(BHEAD outterm) < 0 ) return(-1);
00606             tout = outterm + *outterm;
00607             tout -= 3;
00608             i = t[1]; NCOPY(tout,t,i);
00609             ttwo = tout-1;
00610         }
00611         else { t += t[1]; }
00612     }
00613     else if ( *t >= FUNCTION ) {
00614         i = FindLocalSubterm(BHEAD t,startebuf);
00615         t += t[1];
00616         *tout++ = SYMBOL;
00617         *tout++ = 4;
00618         *tout++ = MAXVARIABLES-i;
00619         *tout++ = 1;
00620         action = 1;
00621     }
00622     else {
00623         if ( FirstWarnConvertToPoly ) {
00624             MLOCK(ErrorMessageLock);
00625             MesPrint("Illegal object in conversion to polynomial notation");
00626             MUNLOCK(ErrorMessageLock);
00627             FirstWarnConvertToPoly = 0;
00628         }
00629         return(-1);
00630     }
00631 }
00632 NCOPY(tout,tstop,ncoef)
00633 if ( ttwo ) {
00634     WORD hh = *ttwo;
00635     *ttwo = tout-ttwo;
00636     if ( ( i = NormPolyTerm(BHEAD ttwo) ) >= 0 ) i = action;
00637     tout = ttwo + *ttwo;
00638     *ttwo = hh;
00639     *outterm = tout - outterm;
00640 }
00641 else {
00642     *outterm = tout-outterm;
00643     if ( ( i = NormPolyTerm(BHEAD outterm) ) >= 0 ) i = action;
00644 }
00645 return(i);
00646 }
00647
00648 /*
00649 #] LocalConvertToPoly :
00650 #[ ConvertFromPoly :
00651
00652 Converts a generic term from polynomial notation to the original
00653 in which the extra symbols have been replaced by their values.
00654 The output is in outterm.
00655 We only deal with the extra symbols in the range from < i <= to

```



```

00656         The output has to be sent to TestSub because it may contain
00657         subexpressions when extra symbols have been replaced.
00658     */
00659
00660 int ConvertFromPoly(PHEAD WORD *term, WORD *outterm, WORD from, WORD to, WORD offset, WORD par)
00661 {
00662     WORD *tout, *tstop, *tstop1, ncoef, *t, *r, *tt;
00663     int i;
00664     /* first = 1; */
00665     tt = term + *term;
00666     tout = outterm+1;
00667     ncoef = ABS(tt[-1]);
00668     tstop = tt - ncoef;
00669     /*
00670         r = t = term + 1;
00671         while ( t < tstop ) {
00672             if ( *t == SYMBOL ) {
00673                 tstop1 = t + t[1];
00674                 tt = t + 2;
00675                 while ( tt < tstop1 ) {
00676                     if ( ( *tt < MAXVARIABLES - to )
00677                         || ( *tt >= MAXVARIABLES - from ) ) {
00678                         tt += 2;
00679                     }
00680                     else break;
00681                 }
00682                 if ( tt >= tstop1 ) { t = tstop1; continue; }
00683                 while ( r < t ) *tout++ = *r++;
00684                 t += 2;
00685                 first = 0;
00686                 while ( t < tstop1 ) {
00687                     if ( ( *t < MAXVARIABLES - to )
00688                         || ( *t >= MAXVARIABLES - from ) ) {
00689                         *tout++ = SYMBOL;
00690                         *tout++ = 4;
00691                         *tout++ = *t++;
00692                         *tout++ = *t++;
00693                     }
00694                     else {
00695                         *tout++ = SUBEXPRESSION;
00696                         *tout++ = SUBEXPSIZE;
00697                         *tout++ = MAXVARIABLES - *t++ + offset;
00698                         *tout++ = *t++;
00699                         if ( par ) *tout++ = AT.ebufnum;
00700                         else *tout++ = AM.sbufnum;
00701                         FILLSUB(tout)
00702                     }
00703                 }
00704                 r = t;
00705             }
00706             else {
00707                 t += t[1];
00708             }
00709         }
00710         if ( first ) {
00711             i = *term; t = term;
00712             NCOPY(outterm,t,i);
00713             return(*term);
00714         }
00715         while ( r < t ) *tout++ = *r++;
00716         NCOPY(tout,tstop,ncoef)
00717         *outterm = tout-outterm;
00718     */
00719     t = term + 1;
00720     while ( t < tstop ) {
00721         if ( *t == SYMBOL ) {
00722             tstop1 = t + t[1];
00723             tt = t + 2;
00724             while ( tt < tstop1 ) {
00725                 if ( ( *tt < MAXVARIABLES - to )
00726                     || ( *tt >= MAXVARIABLES - from ) ) {
00727                     tt += 2;
00728                 }
00729                 else {
00730                     *tout++ = SUBEXPRESSION;
00731                     *tout++ = SUBEXPSIZE;
00732                     *tout++ = MAXVARIABLES - *tt++ + offset;
00733                     *tout++ = *tt++;
00734                     if ( par ) *tout++ = AT.ebufnum;
00735                     else *tout++ = AM.sbufnum;
00736                     FILLSUB(tout)
00737                 }
00738             }
00739             r = tout; t += 2;
00740             *tout++ = SYMBOL; *tout++ = 0;
00741             while ( t < tstop1 ) {
00742                 if ( ( *t < MAXVARIABLES - to )

```

```

00743         || ( *t >= MAXVARIABLES - from ) ) {
00744             *tout++ = *t++;
00745             *tout++ = *t++;
00746         }
00747         else { t += 2; }
00748     }
00749     r[1] = tout - r;
00750     if ( r[1] <= 2 ) tout = r;
00751 }
00752 else {
00753     i = t[1]; NCOPY(tout,t,i)
00754 }
00755 }
00756 NCOPY(tout,tstop,ncoef)
00757 *outterm = tout-outterm;
00758 return(*outterm);
00759 }
00760
00761 /*
00762     #] ConvertFromPoly :
00763     #[ FindSubterm :
00764
00765     In this routine we look up a variable.
00766     If we don't find it we will enter it in the subterm compiler buffer
00767     Searching is by tree structure.
00768     Adding changes the tree.
00769
00770     Notice that in TFORM we should be in sequential mode.
00771 */
00772
00773 WORD FindSubterm(WORD *subterm)
00774 {
00775     WORD old[5], *ss, *term, number;
00776     CBUF *C = cbuf + AM.sbufnum;
00777     LONG oldCpointer;
00778     term = subterm-1;
00779     ss = subterm+subterm[1];
00780 /*
00781     Convert to proper term
00782 */
00783     old[0] = *term; old[1] = ss[0]; old[2] = ss[1]; old[3] = ss[2]; old[4] = ss[3];
00784     ss[0] = 1; ss[1] = 1; ss[2] = 3; ss[3] = 0; *term = subterm[1]+4;
00785 /*
00786     We may have to add the term to the compiler
00787     buffer and then to the tree. This cannot be done in parallel and
00788     hence we have to set a lock.
00789 */
00790     LOCK(AM.sbuflock);
00791
00792     oldCpointer = C->Pointer-C->Buffer; /* Offset of course !!!!!*/
00793     AddRHS(AM.sbufnum,1);
00794     AddNtoC(AM.sbufnum,*term,term,8);
00795     AddToCB(C,0)
00796 /*
00797     See whether we have this one already. If not, insert it in the tree.
00798 */
00799     number = InsTree(AM.sbufnum,C->numrhs);
00800 /*
00801     Restore old values and return what is needed.
00802 */
00803     if ( number < (C->numrhs) ) { /* It existed already */
00804         C->Pointer = oldCpointer + C->Buffer;
00805         C->numrhs--;
00806     }
00807     else {
00808         GETIDENTITY
00809         WORD dim = DimensionSubterm(subterm);
00810
00811         if ( dim == -MAXPOSITIVE ) { /* Give error message but continue */
00812             WORD *old = AN.currentTerm;
00813             AN.currentTerm = term;
00814             MLOCK(ErrorMessageLock);
00815             MesPrint("Dimension out of range in %t");
00816             MUNLOCK(ErrorMessageLock);
00817             AN.currentTerm = old;
00818         }
00819 /*
00820         Store the dimension
00821 */
00822         C->dimension[number] = dim;
00823     }
00824     UNLOCK(AM.sbuflock);
00825
00826     *term = old[0]; ss[0] = old[1]; ss[1] = old[2]; ss[2] = old[3]; ss[3] = old[4];
00827     return(number);
00828 }
00829

```

```

00830 /*
00831     #] FindSubterm :
00832     #[ FindLocalSubterm :
00833
00834     In this routine we look up a variable.
00835     If we don't find it we will enter it in the subterm compiler buffer
00836     Searching is by tree structure.
00837     Adding changes the tree.
00838
00839     Notice that in TFORM we should be in sequential mode.
00840 */
00841
00842 WORD FindLocalSubterm(PHEAD WORD *subterm, WORD startebuf)
00843 {
00844     WORD old[5], *ss, *term, number, i, j, *t1, *t2;
00845     CBUF *C = cbuf + AT.ebufnum;
00846     term = subterm-1;
00847     ss = subterm+subterm[1];
00848 /*
00849     Convert to proper term
00850 */
00851     old[0] = *term; old[1] = ss[0]; old[2] = ss[1]; old[3] = ss[2]; old[4] = ss[3];
00852     ss[0] = 1; ss[1] = 1; ss[2] = 3; ss[3] = 0; *term = subterm[1]+4;
00853 /*
00854     First see whether we have this one already in the global buffer.
00855 */
00856     number = FindTree(AM.sbufnum,term);
00857     if ( number > 0 ) goto wearehappy;
00858 /*
00859     Now look whether it is in the ebufnum between startebuf and numrhs
00860     Note however that we need an offset of (numxsymbol-startebuf)
00861 */
00862     for ( i = startebuf+1; i <= C->numrhs; i++ ) {
00863         t1 = C->rhs[i]; t2 = term;
00864         if ( *t1 == *t2 ) {
00865             j = *t1;
00866             while ( *t1 == *t2 && j > 0 ) { t1++; t2++; j--; }
00867             if ( j <= 0 ) {
00868                 number = i-startebuf+numxsymbol;
00869                 goto wearehappy;
00870             }
00871         }
00872     }
00873 /*
00874     Now we have to add it to cbuf[AT.ebufnum]
00875 */
00876     AddRHS(AT.ebufnum,1);
00877     AddNtoC(AT.ebufnum,*term,term,9);
00878     AddToCB(C,0)
00879     number = C->numrhs-startebuf+numxsymbol;
00880 wearehappy:
00881     *term = old[0]; ss[0] = old[1]; ss[1] = old[2]; ss[2] = old[3]; ss[3] = old[4];
00882     return(number);
00883 }
00884
00885 /*
00886     #] FindLocalSubterm :
00887     #[ PrintSubtermList :
00888
00889     Prints all the expressions in the subterm compiler buffer.
00890     The format is such that they give definitions of the temporary
00891     variables of which the contents are stored in this buffer.
00892     These variables have the names Z_123 etc.
00893 */
00894
00895 void PrintSubtermList(int from,int to)
00896 {
00897     UBYTE buffer[80], *out, outbuffer[300];
00898     int first, i, ii, inc = 1;
00899     WORD *term;
00900     CBUF *C = cbuf + AM.sbufnum;
00901 /*
00902     if ( to < from ) inc = -1;
00903     if ( to == from ) inc = 0;
00904 */
00905     if ( from <= to ) {
00906         inc = 1; to += inc;
00907     }
00908     else {
00909         inc = -1; to += inc;
00910     }
00911     AO.OutFill = AO.OutputLine = outbuffer;
00912     AO.OutStop = AO.OutputLine+AC.LineLength;
00913     AO.IsBracket = 0;
00914     AO.OutSkip = 3;
00915
00916     if ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE ) {

```

```

00917         TokenToLine((UBYTE *)"          ");
00918         AO.OutSkip = 7;
00919     }
00920     else if ( ( AO.Optimize.debugflags & 1 ) == 1 ) {}
00921     else if ( AO.OutSkip > 0 ) {
00922         for ( i = 0; i < AO.OutSkip; i++ ) TokenToLine((UBYTE *)" ");
00923     }
00924     i = from;
00925     do {
00926         if ( ( AO.Optimize.debugflags & 1 ) == 1 ) {
00927             TokenToLine((UBYTE *)"id ");
00928             for ( ii = 3; ii < AO.OutSkip; ii++ ) TokenToLine((UBYTE *)" ");
00929         }
00930     /*
00931         if ( AC.OutputMode == NORMALFORMAT ) {
00932             TokenToLine((UBYTE *)"id ");
00933         }
00934     */
00935     else if ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE ) {}
00936     else { TokenToLine((UBYTE *)" "); }
00937
00938     out = StrCopy((UBYTE *)AC.extrasym,buffer);
00939     if ( AC.extrasymbols == 0 ) {
00940         out = NumCopy(i,out);
00941         out = StrCopy((UBYTE *)"_",out);
00942     }
00943     else if ( AC.extrasymbols == 1 ) {
00944         out = AddArrayIndex(i,out);
00945     }
00946     out = StrCopy((UBYTE *)"=",out);
00947     TokenToLine(buffer);
00948     term = C->rhs[i];
00949     first = 1;
00950     if ( *term == 0 ) {
00951         out = StrCopy((UBYTE *)"0",buffer);
00952         if ( AC.OutputMode != FORTRANMODE && AC.OutputMode != PFORTRANMODE ) {
00953             out = StrCopy((UBYTE *)";",out);
00954         }
00955         TokenToLine(buffer);
00956     }
00957     else {
00958         while ( *term ) {
00959             if ( WriteInnerTerm(term,first) ) Terminate(-1);
00960             term += *term;
00961             first = 0;
00962         }
00963         if ( AC.OutputMode != FORTRANMODE && AC.OutputMode != PFORTRANMODE ) {
00964             out = StrCopy((UBYTE *)";",buffer);
00965             TokenToLine(buffer);
00966         }
00967     }
00968 /*
00969     There is a problem with FiniLine because it prepares for a
00970     continuation line in fortran mode.
00971     But the next statement should start on a blank line.
00972 */
00973 /*
00974     FiniLine();
00975     if ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE ) {
00976         AO.OutFill = AO.OutputLine;
00977         TokenToLine((UBYTE *)"          ");
00978         AO.OutSkip = 7;
00979     }
00980 */
00981     if ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE ) {
00982         AO.OutSkip = 6;
00983         FiniLine();
00984         AO.OutSkip = 7;
00985     }
00986     else {
00987         FiniLine();
00988     }
00989     i += inc;
00990 } while ( i != to );
00991 }
00992
00993 /*
00994     #] PrintSubtermList :
00995     #[ PrintExtraSymbol :
00996
00997     Prints the definition of extra symbol num as the contents
00998     of the expression in terms.
00999     The parameter par has three options:
01000         EXTRASYMBOL      num is interpreted as the number of an extra symbol
01001         REGULARSYMBOL    num is interpreted as the number of a symbol.
01002                         It could still be an extra symbol.
01003         EXPRESSIONNUMBER num is the number of an expression.

```

```

01004         terms contains the rhs expression.
01005 */
01006
01007 void PrintExtraSymbol(int num, WORD *terms,int par)
01008 {
01009     UBYTE buffer[80], *out, outbuffer[300];
01010     int first, i;
01011     WORD *term;
01012
01013     AO.OutFill = AO.OutputLine = outbuffer;
01014     AO.OutStop = AO.OutputLine+AC.LineLength;
01015     AO.IsBracket = 0;
01016
01017     if ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE ) {
01018         TokenToLine((UBYTE *) "      ");
01019         AO.OutSkip = 7;
01020     }
01021     else if ( ( AO.Optimize.debugflags & 1 ) == 1 ) {
01022         TokenToLine((UBYTE *) "id ");
01023         for ( i = 3; i < AO.OutSkip; i++ ) TokenToLine((UBYTE *) " ");
01024     }
01025     else if ( AO.OutSkip > 0 ) {
01026         for ( i = 0; i < AO.OutSkip; i++ ) TokenToLine((UBYTE *) " ");
01027     }
01028     out = buffer;
01029     switch ( par ) {
01030         case REGULARSYMBOL:
01031             if ( num >= MAXVARIABLES-cbuf[AM.sbufnum].numrhs ) {
01032                 num = MAXVARIABLES-num;
01033             }
01034             else {
01035                 out = StrCopy(FindSymbol(num),out);
01036                 out = StrCopy(VARNAME(symbols,num),out); */
01037                 break;
01038             }
01039             /* fall through */
01040         case EXTRASYMBOL:
01041             out = StrCopy(FindExtraSymbol(num),out);
01042             /*
01043             out = StrCopy((UBYTE *)AC.extrasym,out);
01044             if ( AC.extrasymbols == 0 ) {
01045                 out = NumCopy(num,out);
01046                 out = StrCopy((UBYTE *) "_",out);
01047             }
01048             else if ( AC.extrasymbols == 1 ) {
01049                 out = AddArrayIndex(num,out);
01050             }
01051             */
01052             break;
01053         case EXPRESSIONNUMBER:
01054             out = StrCopy(EXPRNAME(num),out);
01055             break;
01056         default:
01057             MesPrint("Illegal option in PrintExtraSymbol");
01058             Terminate(-1);
01059     }
01060     out = StrCopy((UBYTE *) "=",out);
01061     TokenToLine(buffer);
01062     term = terms;
01063     first = 1;
01064     if ( *term == 0 ) {
01065         out = StrCopy((UBYTE *) "0",buffer);
01066         TokenToLine(buffer);
01067     }
01068     else {
01069         while ( *term ) {
01070             if ( WriteInnerTerm(term,first) ) Terminate(-1);
01071             term += *term;
01072             first = 0;
01073         }
01074     }
01075     if ( AC.OutputMode != FORTRANMODE && AC.OutputMode != PFORTRANMODE ) {
01076         out = StrCopy((UBYTE *) ";",buffer);
01077         TokenToLine(buffer);
01078     }
01079     FiniLine();
01080 }
01081
01082 /*
01083     #] PrintExtraSymbol :
01084     #[ FindSubexpression :
01085
01086     In this routine we look up a subexpression.
01087     If we don't find it we will enter it in the subterm compiler buffer
01088     Searching is by tree structure.
01089     Adding changes the tree.
01090

```

```

01091         Notice that in TFORM we should be in sequential mode.
01092 */
01093
01094 WORD FindSubexpression(WORD *subexpr)
01095 {
01096     WORD *term, number;
01097     CBUF *C = cbuf + AM.sbufnum;
01098     LONG oldCpointer;
01099
01100     term = subexpr;
01101     while ( *term ) term += *term;
01102     number = term - subexpr;
01103 /*
01104     We may have to add the subexpression to the tree.
01105     This requires a lock.
01106 */
01107     LOCK(AM.sbuflock);
01108
01109     oldCpointer = C->Pointer-C->Buffer; /* Offset of course !!!!!*/
01110     AddRHS(AM.sbufnum,1);
01111 /*
01112     Add the terms to the compiler buffer. Paste on a zero.
01113 */
01114     AddNtoC(AM.sbufnum,number,subexpr,10);
01115     AddToCB(C,0)
01116 /*
01117     See whether we have this one already. If not, insert it in the tree.
01118 */
01119     number = InsTree(AM.sbufnum,C->numrhs);
01120 /*
01121     Restore old values and return what is needed.
01122 */
01123     if ( number < (C->numrhs) ) { /* It existed already */
01124         C->Pointer = oldCpointer + C->Buffer;
01125         C->numrhs--;
01126     }
01127     else {
01128         GETIDENTITY
01129         WORD dim = DimensionExpression(BHEAD subexpr);
01130 /*
01131         Store the dimension
01132 */
01133         C->dimension[number] = dim;
01134     }
01135
01136     UNLOCK(AM.sbuflock);
01137
01138     return(number);
01139 }
01140
01141 /*
01142     #] FindSubexpression :
01143     #[ ExtraSymFun :
01144 */
01145
01146 int ExtraSymFun(PHEAD WORD *term,WORD level)
01147 {
01148     WORD *oldworkpointer = AT.WorkPointer;
01149     WORD *termout, *t1, *t2, *t3, *tstop, *tend, i;
01150     int retval = 0;
01151     tend = termout = term + *term;
01152     tstop = tend - ABS(tend[-1]);
01153     t3 = t1 = term+1; t2 = termout+1;
01154 /*
01155     First refind the function(s). There is at least one.
01156 */
01157     while ( t1 < tstop ) {
01158         if ( *t1 == EXTRASYMFUN && t1[1] == FUNHEAD+2 ) {
01159             if ( t1[FUNHEAD] == -SNUMBER && t1[FUNHEAD+1] <= numxsymbol
01160                 && t1[FUNHEAD+1] > 0 ) {
01161                 i = t1[FUNHEAD+1];
01162             }
01163             else if ( t1[FUNHEAD] == -SYMBOL && t1[FUNHEAD+1] < MAXVARIABLES
01164                 && t1[FUNHEAD+1] >= MAXVARIABLES-numxsymbol ) {
01165                 i = MAXVARIABLES - t1[FUNHEAD+1];
01166             }
01167             else goto nocase;
01168             while ( t3 < t1 ) *t2++ = *t3++;
01169 /*
01170             Now inset the rhs pointer
01171 */
01172             *t2++ = SUBEXPRESSION;
01173             *t2++ = SUBEXPSIZE;
01174             *t2++ = i;
01175             *t2++ = 1;
01176             *t2++ = AM.sbufnum;
01177             FILLSUB(t2)

```

```

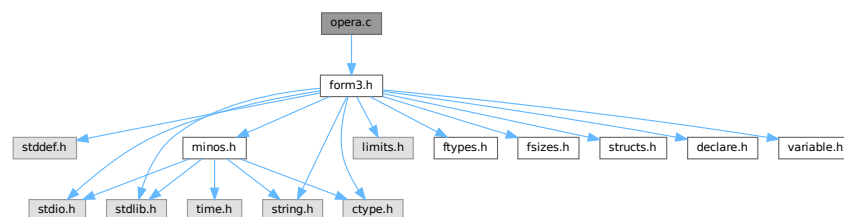
01178         t3 = t1 = t1 + t1[1];
01179     }
01180     else if ( *t1 == EXTRASYMFUN && t1[1] == FUNHEAD ) {
01181         while ( t3 < t1 ) *t2++ = *t3++;
01182         t3 = t1 = t1 + t1[1];
01183     }
01184     else {
01185 nocase:;
01186         t1 = t1 + t1[1];
01187     }
01188 }
01189 while ( t3 < tend ) *t2++ = *t3++;
01190 *termout = t2 - termout;
01191 AT.WorkPointer = t2;
01192 if ( AT.WorkPointer >= AT.WorkTop ) {
01193     MLOCK(ErrorMessageLock);
01194     MesWork();
01195     MUNLOCK(ErrorMessageLock);
01196     AT.WorkPointer = oldworkpointer;
01197     return(-1);
01198 }
01199 retval = Generator(BHEAD termout,level);
01200 AT.WorkPointer = oldworkpointer;
01201 if ( retval < 0 ) {
01202     MLOCK(ErrorMessageLock);
01203     MesCall("ExtraSymFun");
01204     MUNLOCK(ErrorMessageLock);
01205 }
01206 return(retval);
01207 }
01208
01209 /*
01210     #] ExtraSymFun :
01211     #[ PruneExtraSymbols :
01212 */
01213
01214 int PruneExtraSymbols(WORD downto)
01215 {
01216     CBUF *C = cbuf + AM.sbufnum;
01217     if ( downto < C->numrhs && downto >= 0 ) { /* !!!!! */
01218         ClearTree(AM.sbufnum);
01219         C->numrhs = downto;
01220         if ( downto == 0 ) {
01221             C->Pointer = C->Buffer;
01222         }
01223         else {
01224             WORD *w = C->rhs[downto], i;
01225             while ( *w ) w += *w;
01226             C->Pointer = w+1;
01227             for ( i = 1; i <= downto; i++ ) {
01228                 InsTree(AM.sbufnum,i);
01229             }
01230         }
01231     }
01232     return(0);
01233 }
01234
01235 /*
01236     #] PruneExtraSymbols :
01237 */

```

4.90 opera.c File Reference

```
#include "form3.h"
```

Include dependency graph for opera.c:



Functions

- WORD [EpfFind](#) (PHEAD WORD *term, WORD *params)
- WORD [EpfCon](#) (PHEAD WORD *term, WORD *params, WORD num, WORD level)
- WORD [EpfGen](#) (WORD number, WORD *inlist, WORD *kron, WORD *perm, WORD sgn)
- WORD [Trick](#) (WORD *in, [TRACES](#) *t)
- WORD [Trace4no](#) (WORD number, WORD *kron, [TRACES](#) *t)
- WORD [Trace4](#) (PHEAD WORD *term, WORD *params, WORD num, WORD level)
- WORD [Trace4Gen](#) (PHEAD [TRACES](#) *t, WORD number)
- WORD [TraceNno](#) (WORD number, WORD *kron, [TRACES](#) *t)
- WORD [TraceN](#) (PHEAD WORD *term, WORD *params, WORD num, WORD level)
- WORD [TraceNgen](#) (PHEAD [TRACES](#) *t, WORD number)
- WORD [Traces](#) (PHEAD WORD *term, WORD *params, WORD num, WORD level)
- WORD [TraceFind](#) (PHEAD WORD *term, WORD *params)
- WORD [Chisholm](#) (PHEAD WORD *term, WORD level)
- WORD [TenVecFind](#) (PHEAD WORD *term, WORD *params)
- WORD [TenVec](#) (PHEAD WORD *term, WORD *params, WORD num, WORD level)

4.90.1 Detailed Description

Contains the 'operations' These are the trace routines, the contractions of the Levi-Civita tensors and the tensor to vector/vector to tensor routines. The trace and contraction routines are done in a special way (see commentary with the FIXEDGLOBALS struct)

Definition in file [opera.c](#).

4.90.2 Function Documentation

4.90.2.1 EpfFind()

```
WORD EpfFind (
    PHEAD WORD * term,
    WORD * params )
```

Definition at line 58 of file [opera.c](#).

4.90.2.2 EpfCon()

```
WORD EpfCon (
    PHEAD WORD * term,
    WORD * params,
    WORD num,
    WORD level )
```

Definition at line 212 of file [opera.c](#).

4.90.2.3 EpfGen()

```
WORD EpfGen (
    WORD number,
    WORD * inlist,
    WORD * kron,
    WORD * perm,
    WORD sgn )
```

Definition at line 282 of file [opera.c](#).

4.90.2.4 Trick()

```
WORD Trick (
    WORD * in,
    TRACES * t )
```

Definition at line 331 of file [opera.c](#).

4.90.2.5 Trace4no()

```
WORD Trace4no (
    WORD number,
    WORD * kron,
    TRACES * t )
```

Definition at line 418 of file [opera.c](#).

4.90.2.6 Trace4()

```
WORD Trace4 (
    PHEAD WORD * term,
    WORD * params,
    WORD num,
    WORD level )
```

Definition at line 694 of file [opera.c](#).

4.90.2.7 Trace4Gen()

```
WORD Trace4Gen (
    PHEAD TRACES * t,
    WORD number )
```

Definition at line 856 of file [opera.c](#).

4.90.2.8 TraceNno()

```
WORD TraceNno (
    WORD number,
    WORD * kron,
    TRACES * t )
```

Definition at line 1341 of file [opera.c](#).

4.90.2.9 TraceN()

```
WORD TraceN (
    PHEAD WORD * term,
    WORD * params,
    WORD num,
    WORD level )
```

Definition at line 1382 of file [opera.c](#).

4.90.2.10 TraceNgen()

```
WORD TraceNgen (
    PHEAD TRACES * t,
    WORD number )
```

Definition at line 1446 of file [opera.c](#).

4.90.2.11 Traces()

```
WORD Traces (
    PHEAD WORD * term,
    WORD * params,
    WORD num,
    WORD level )
```

Definition at line 1831 of file [opera.c](#).

4.90.2.12 TraceFind()

```
WORD TraceFind (
    PHEAD WORD * term,
    WORD * params )
```

Definition at line 1853 of file [opera.c](#).

4.90.2.13 Chisholm()

```
WORD Chisholm (
    PHEAD WORD * term,
    WORD level )
```

Definition at line 1963 of file [opera.c](#).

4.90.2.14 TenVecFind()

```
WORD TenVecFind (
    PHEAD WORD * term,
    WORD * params )
```

Definition at line 2178 of file [opera.c](#).

4.90.2.15 TenVec()

```
WORD TenVec (
    PHEAD WORD * term,
    WORD * params,
    WORD num,
    WORD level )
```

Definition at line 2312 of file [opera.c](#).

4.91 opera.c

[Go to the documentation of this file.](#)

```
00001
00009 /* #[] License : */
00010 /*
00011 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00012 * When using this file you are requested to refer to the publication
00013 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00014 * This is considered a matter of courtesy as the development was paid
00015 * for by FOM the Dutch physics granting agency and we would like to
00016 * be able to track its scientific use to convince FOM of its value
00017 * for the community.
00018 *
00019 * This file is part of FORM.
00020 *
00021 * FORM is free software: you can redistribute it and/or modify it under the
00022 * terms of the GNU General Public License as published by the Free Software
00023 * Foundation, either version 3 of the License, or (at your option) any later
00024 * version.
00025 *
00026 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00027 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00028 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00029 * details.
00030 *
00031 * You should have received a copy of the GNU General Public License along
00032 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00033 */
00034 /* #[] License : */
00035 /*
00036 #[] Includes : opera.c
00037 */
00038
00039 #include "form3.h"
00040 /*
00041 int hulp;
00042 */
00043
00044 /*
00045 #] Includes :
00046 #[] Operations :
00047 #[] EpfFind : WORD EpfFind(term,params)
00048
00049 Searches for a pair of Levi-Civita tensors that should be
00050 contracted.
00051 If a match is found its settings are recorded in AT.TMout.
00052 type indicates the number of indices that is searched for,
00053 unless all are searched for (type = 0).
00054 number is the number of tensors that should survive contraction.
00055
00056 */
```

```

00057
00058 WORD EpfFind(PHEAD WORD *term, WORD *params)
00059 {
00060     GETBIDENTITY
00061     WORD *t, *m, *r, n1 = 0, n2, min = -1, count, fac;
00062     WORD *c1 = 0, *c2 = 0, sgn = 1;
00063     WORD *tstop, *mstop;
00064     UWORD *facto = (UWORD *)AT.WorkPointer;
00065     WORD ncoef, nfac;
00066     WORD number, type;
00067     if ( ( AT.WorkPointer = (WORD *) (facto + AM.MaxTal) ) > AT.WorkTop ) {
00068         MLOCK(ErrorMessageLock);
00069         MesWork();
00070         MUNLOCK(ErrorMessageLock);
00071         return(-1);
00072     }
00073     number = params[3];
00074     type = params[4];
00075     t = term;
00076     GETSTOP(t, tstop);
00077     t++;
00078     if ( !type ) {
00079         while ( *t != LEVICIVITA && t < tstop ) t += t[1];
00080         if ( t >= tstop ) return(0);
00081         m = t;
00082         while ( *m == LEVICIVITA && m < tstop ) { n1++; m += m[1]; }
00083     AllLev:
00084         if ( n1 <= (number+1) || n1 <= 1 ) return(0);
00085         mstop = m;
00086         m = t + t[1];
00087         do {
00088             while ( m[1] == t[1] ) {
00089                 m += FUNHEAD;
00090                 r = t+FUNHEAD;
00091                 count = fac = n1 = n2 = t[1] - FUNHEAD;
00092                 while ( n1 && n2 ) {
00093                     if ( *m > *r ) {
00094                         r++; n2--;
00095                     }
00096                     else if ( *m < *r ) {
00097                         m++; n1--;
00098                     }
00099                     else {
00100                         if ( *m >= AM.OffsetIndex &&
00101                             ( ( *m >= AM.IndDum && AC.lDefDim == fac ) ||
00102                               ( *m < AM.IndDum &&
00103                                 indices[*m-AM.OffsetIndex].dimension == fac ) ) ) {
00104                             count--;
00105                         }
00106                         r++; m++; n1--; n2--;
00107                     }
00108                 }
00109                 m += n1;
00110                 if ( min < 0 || count < min ) {
00111                     c1 = t;
00112                     c2 = m - fac - FUNHEAD;
00113                     min = count;
00114                 }
00115                 if ( m >= mstop ) break;
00116             }
00117             t += t[1];
00118         } while ( ( m = t + t[1] ) < mstop );
00119     }
00120     else {
00121         fac = type + FUNHEAD;
00122         while ( *t != LEVICIVITA && t < tstop ) t += t[1];
00123         while ( *t == LEVICIVITA && t < tstop && t[1] != fac ) t += t[1];
00124         if ( t >= tstop ) return(0);
00125         m = t;
00126         while ( *m == LEVICIVITA && m < tstop && m[1] == fac ) { n1++; m += m[1]; }
00127         goto AllLev;
00128     }
00129     /*
00130     We have now the two tensors that give the minimum contraction
00131     in c1 and c2.
00132     Prepare the AT.TMout array;
00133     */
00134     if ( min < 0 ) return(0); /* No matching pair! */
00135     t = c1;
00136     mstop = c2;
00137     fac = t[1] - FUNHEAD;
00138     m = AT.TMout;
00139     *m++ = 3 + (min*2); /* The full length */
00140     *m++ = CONTRACT;
00141     if ( number < 0 ) *m++ = 1;
00142     else *m++ = 0;
00143     n1 = n2 = t[1] - FUNHEAD;

```

```

00144     r = c1 + FUNHEAD;
00145     c1 = m;
00146     m = c2 + FUNHEAD;
00147     c2 = c1 + min;
00148     while ( n1 && n2 ) {
00149         if ( *m > *r ) { *c1++ = *r++; n2--; }
00150         else if ( *m < *r ) { *c2++ = *m++; n1--; }
00151         else {
00152             if ( *m < AM.OffsetIndex || ( *m < AM.IndDum &&
00153                 ( indices[*m-AM.OffsetIndex].dimension != fac ) ) ||
00154                 ( *m >= AM.IndDum && AC.lDefDim != fac ) ) {
00155                 *c1++ = *r++; *c2++ = *m++;
00156             }
00157             else { if ( ( n1 ^ n2 ) & 1 ) sgn = -sgn; r++; m++; }
00158             n1--; n2--;
00159         }
00160     }
00161     if ( n1 ) { NCOPY(c2,m,n1); }
00162     else if ( n2 ) { NCOPY(c1,r,n2); }
00163     fac -= min;
00164     m = t + t[1];
00165     while ( m < mstop ) *t++ = *m++;
00166     m += m[1];
00167     while ( m < tstop ) *t++ = *m++;
00168     *t++ = SUBEXPRESSION;
00169     *t++ = SUBEXPSIZE;
00170     *t++ = -1;
00171     *t++ = 1;
00172     *t++ = DUMMYBUFFER;
00173     FILLSUB(t)
00174     r = term;
00175     r += *r - 1;
00176     mstop = r;
00177     ncoef = REDLENG(*r);
00178     tstop = t;
00179     while ( m < mstop ) *t++ = *m++;
00180     if ( Factorial(BHEAD fac,facto,&nfac) || Mully(BHEAD (UWORD *)tstop,&ncoef,facto,nfac) ) {
00181         MLOCK(ErrorMessageLock);
00182         MesCall("EpFFind");
00183         MUNLOCK(ErrorMessageLock);
00184         SETERROR(-1)
00185     }
00186     tstop += (ABS(ncoef))*2;
00187     if ( sgn < 0 ) ncoef = -ncoef;
00188     ncoef *= 2;
00189     *tstop++ = (ncoef<0)?(ncoef-1):(ncoef+1);
00190     *term = WORDDIF(tstop,term);
00191     return(1);
00192 }
00193
00194 /*
00195     #] EpFFind :
00196     #[ EpFCon :          WORD EpFCon(term,params,num,level)
00197
00198     Contraction of two strings of indices/vectors. They come
00199     from LeviCivita tensors that are being contracted.
00200     term is the term with the subterm to be replaced.
00201     params is the full indicator:
00202         Length, number, raise, parameters.
00203     Length is the length of the parameter field.
00204     number is the number of the operation.
00205     raise tells whether level should be raised afterwards.
00206     the parameters are the two strings.
00207     level is the id level.
00208     The factorial has been multiplied in already.
00209
00210 */
00211
00212 WORD EpFCon(PHEAD WORD *term, WORD *params, WORD num, WORD level)
00213 {
00214     GETBIDENTITY
00215     WORD *kron, *perm, *termout, *tstop, size2;
00216     WORD *m, *t, sizes, sgn = 0, i;
00217     sizes = *params - 3;
00218     kron = AT.WorkPointer;
00219     perm = (AT.WorkPointer += sizes);
00220     termout = (AT.WorkPointer += sizes);
00221     AT.WorkPointer = (WORD *)(((UBYTE *) (AT.WorkPointer)) + AM.MaxTer);
00222     if ( AT.WorkPointer > AT.WorkTop ) {
00223         MLOCK(ErrorMessageLock);
00224         MesWork();
00225         MUNLOCK(ErrorMessageLock);
00226         return(-1);
00227     }
00228     params += 2;
00229     if ( !(*params++) ) level--;
00230     size2 = sizes>>1;

```

```

00231     if ( !size2 ) goto DoOnce;
00232     while ( ( sgn = EpfGen(size2,params,kron,perm,sgn) ) != 0 ) {
00233 DoOnce:
00234         t = term;
00235         GETSTOP(t,tstop);
00236         m = termout;
00237         tstop -= SUBEXPSIZE;
00238         while ( t < tstop ) *m++ = *t++;
00239         if ( t[2] != num || *t != SUBEXPRESSION ) {
00240             MLOCK(ErrorMessageLock);
00241             MesPrint("Serious error in EpfCon");
00242             MUNLOCK(ErrorMessageLock);
00243             return(-1);
00244         }
00245         tstop += SUBEXPSIZE;
00246         if ( sizes ) {
00247             *m++ = DELTA;
00248             *m++ = sizes + 2;
00249             t = kron;
00250             i = sizes;
00251             if ( i ) { NCOPY(m,t,i); }
00252         }
00253         t = tstop;
00254         tstop = term + *term;
00255         while ( t < tstop ) *m++ = *t++;
00256         *termout = WORDDIF(m,termout);
00257         m--;
00258         if ( sgn < 0 ) *m = - *m;
00259         if ( *termout ) {
00260             *AN.RepPoint = 1;
00261             AR.expchanged = 1;
00262             AT.WorkPointer = termout + *termout;
00263             if ( Generator(BHEAD termout,level) < 0 ) goto EpfCall;
00264         }
00265     }
00266     AT.WorkPointer = kron;
00267     return(0);
00268 EpfCall:
00269     if ( AM.tracebackflag ) {
00270         MLOCK(ErrorMessageLock);
00271         MesCall("EpfCon");
00272         MUNLOCK(ErrorMessageLock);
00273     }
00274     return(-1);
00275 }
00276
00277 /*
00278     #] EpfCon :
00279     #[ EpfGen :          WORD EpfGen(number,inlist,kron,perm,sgn)
00280 */
00281
00282 WORD EpfGen(WORD number, WORD *inlist, WORD *kron, WORD *perm, WORD sgn)
00283 {
00284     WORD i, *in2, k, a;
00285     if ( !sgn ) {
00286         in2 = inlist + number;
00287         number *= 2;
00288         for ( i = 1; i < number; i += 2 ) {
00289             *perm++ = i;
00290             *perm++ = i;
00291             *kron++ = *inlist++;
00292             *kron++ = *in2++;
00293         }
00294         if ( number <= 0 ) return(0);
00295         else return(1);
00296     }
00297     number *= 2;
00298     i = number - 1;
00299     while ( ( i -= 2 ) >= 0 ) {
00300         if ( ( k = perm[i] ) != i ) {
00301             sgn = -sgn;
00302             a = kron[i];
00303             kron[i] = kron[k];
00304             kron[k] = a;
00305         }
00306         if ( ( k = ( perm[i] += 2 ) ) < number ) {
00307             a = kron[i];
00308             kron[i] = kron[k];
00309             kron[k] = a;
00310             sgn = - sgn;
00311             for ( k = i + 2; k < number; k += 2 ) perm[k] = k;
00312             return(sgn);
00313         }
00314     }
00315     return(0);
00316 }
00317

```

```

00318 /*
00319      #] EpfGen :
00320      #[ Trick :          WORD Trick(in,t)
00321
00322      This routine implements the identity:
00323      g_(j,mu)*g_(j,nu)*g_(j,ro)=e_(mu,nu,ro,si)*g5_(j)*g_(j,si)
00324      +d_(mu,nu)*g_(j,ro)-d_(mu,ro)*g_(j,nu)+d_(nu,ro)*g_(j,mu)
00325      which is for 4 dimensions only!
00326
00327      Note that z->gamm = 1 if there is no g5 present.
00328
00329 */
00330
00331 WORD Trick(WORD *in, TRACES *t)
00332 {
00333     WORD n, n1;
00334     n = t->stap;
00335     n1 = t->step1;
00336     switch ( t->eers[n] ) {
00337     case 0: {
00338         WORD *p;
00339         p = t->pepf + t->mepf[n];
00340         *p++ = *in++;
00341         *p++ = *in++;
00342         *p++ = *in;
00343         *p = ++(t->mdum);
00344         (t->mepf[n1]) += 4;
00345         *in = t->mdum;
00346         t->gamm = - t->gamm;
00347         t->eers[n] = 5;
00348         break;
00349     }
00350     case 4: {
00351         WORD *p;
00352         p = t->pdel + t->mdel[n];
00353         (t->mepf[n1]) -= 4;
00354         (t->mdum)--;
00355         *p++ = *in++;
00356         *p = *in++;
00357         *in = *(t->pepf + t->mepf[n] + 2);
00358         (t->mdel[n1]) += 2;
00359         t->gamm = - t->gamm;
00360         break;
00361     }
00362     case 3: {
00363         t->sign1 = - t->sign1;
00364         *(t->pdel + t->mdel[n] + 1) = in[2];
00365         in[2] = in[1];
00366         break;
00367     }
00368     case 2: {
00369         t->sign1 = - t->sign1;
00370         in[2] = in[0];
00371         *(t->pdel + t->mdel[n]) = in[1];
00372         break;
00373     }
00374     case 1: {
00375         in[2] = *(t->pdel + t->mdel[n] + 1);
00376         (t->mdel[n1]) -= 2;
00377         break;
00378     }
00379     default: {
00380         return(0);
00381     }
00382 }
00383     return(--(t->eers[n]));
00384 }
00385
00386 /*
00387      #] Trick :
00388      #[ Trace4no :          WORD Trace4no(number,kron,t)
00389
00390      Takes the trace of a string of gamma matrices in 4 dimensions.
00391      There is no test for indices or vectors that are the same.
00392      The four dimensions refer to the contraction in the algebra:
00393      g_(i,a,b,c) = e_(a,b,c,d)*g_(i,5_,d) + g_(i,a)*d_(b,c)
00394                  - g_(i,b)*d_(a,c) + g_(i,c)*d_(a,b)
00395      This simplifies life very much and leads to shorter expressions
00396      than in the n dimensional case.
00397
00398      Parameters:
00399      number: the number of vectors/indices in inlist.
00400      inlist: the indices/vectors in the string.
00401      kron:   the output delta's.
00402      gamma5: the potential gamma5 in front.
00403      t:      the struct for scratch manipulations.
00404      stack:  the space to put all scratch arrays in.

```

```

00405
00406     The return value is zero if there are no more terms, 1 if a
00407     term was generated with a positive sign and -1 if a term was
00408     generated with a negative sign.
00409     The value of one is increased to two if the first 4 values
00410     in kron should be interpreted as a Levi-Civita tensor.
00411
00412     Note that kron should have more places reserved than the number
00413     of indices in inlist, because it will contain dummy indices
00414     temporarily. In principle there can be 'number*1/4' extra dummies.
00415
00416 */
00417
00418 WORD Trace4no(WORD number, WORD *kron, TRACES *t)
00419 {
00420     WORD i;
00421     WORD *p, *m;
00422     WORD retval, *stop, oldsign;
00423     if ( !t->sgn ) { /* Startup */
00424         if ( ( number < 0 ) || ( number & 1 ) ) return(0);
00425         if ( number <= 2 ) {
00426             if ( t->gamma5 == GAMMA5 ) return(0);
00427             if ( number == 2 ) {
00428                 *kron++ = *t->inlist;
00429                 *kron++ = t->inlist[1];
00430             }
00431             return(1);
00432         }
00433         t->sgn = 1;
00434         {
00435             WORD nhalf = number >> 1;
00436             WORD ndouble = number * 2;
00437             p = t->eers;
00438             t->eers = p;      p += nhalf;
00439             t->mepf = p;      p += nhalf;
00440             t->mdel = p;      p += nhalf;
00441             t->pdel = p;      p += number + nhalf;
00442             t->pepf = p;      p += ndouble;
00443             t->e4 = p;        p += number;
00444             t->e3 = p;        p += ndouble;
00445             t->nt3 = p;       p += nhalf;
00446             t->nt4 = p;       p += nhalf;
00447             t->j3 = p;        p += ndouble;
00448             t->j4 = p;
00449         }
00450         t->mepf[0] = 0;
00451         t->mdel[0] = 0;
00452         t->mdum = AM.mTraceDum;
00453         t->kstep = -2;
00454         t->step1 = 0;
00455         t->sign1 = 1;
00456         t->lc3 = -1;
00457         t->lc4 = -1;
00458         t->gamm = 1;
00459
00460         do {
00461             t->stap = (t->step1)++;
00462             t->kstep += 2;
00463             t->eers[t->stap] = 0;
00464             t->mepf[t->step1] = t->mepf[t->stap];
00465             t->mdel[t->step1] = t->mdel[t->stap];
00466             CallTrick: while ( !Trick(t->inlist+t->kstep,t) ) {
00467                 t->kstep -= 2;
00468                 t->step1 = (t->stap)--;
00469                 if ( t->stap < 0 ) {
00470                     return(0);
00471                 }
00472             }
00473             while ( t->kstep < (number-4) );
00474         } /*
00475     Take now the trace of the leftover matrices.
00476     If gamma5 causes the term to vanish there will be a
00477     renewed call to Trick for its next term.
00478 */
00479     t->sign2 = t->sign1;
00480     if ( ( t->gamma5 == GAMMA7 ) && ( t->gamm == -1 ) ) {
00481         t->sign2 = - t->sign2;
00482     }
00483     else if ( ( t->gamma5 == GAMMA5 ) && ( t->gamm == 1 ) ) {
00484         goto CallTrick;
00485     }
00486     else if ( ( t->gamma5 == GAMMA1 ) && ( t->gamm == -1 ) ) {
00487         goto CallTrick;
00488     }
00489     p = t->pdel + t->mdel[t->step1];
00490     *p++ = t->inlist[t->kstep+2];
00491     *p++ = t->inlist[t->kstep+3];

```



```

00492 /*
00493     Now the trace has been expressed in terms of Levi-Civita tensors
00494     and Kronecker delta's.
00495     The Levi-Civita tensors are in t->pepf
00496     and there are t->mepf[step1] elements in this array.
00497     The Kronecker delta's are in t->pdel
00498     and there are t->mdel[step1] elements in this array.
00499
00500     Next we rake the Levi-Civita tensors together such that there
00501     is an optimal use of the contractions.
00502 */
00503 {
00504     WORD ae;
00505     ae = t->mepf[t->step1];
00506     t->ad = t->mdel[t->step1]+2;
00507     t->a4 = 0;
00508     t->a3 = 0;
00509     while ( ( ae -= 4 ) >= 0 ) {
00510         if ( t->pepf[ae] > AM.mTraceDum && t->pepf[ae] <= t->mdum ) {
00511             p = t->e3 + t->a3;
00512             m = t->pepf + ae;
00513             for ( i = 0; i < 3; i++ ) {
00514                 p[3] = m[3-i];
00515                 *p++ = m[i-4];
00516             }
00517             t->a3 += 6;
00518             ae -= 4;
00519         }
00520         else {
00521             p = t->e4 + t->a4;
00522             m = t->pepf + ae;
00523             for ( i = 0; i < 4; i++ ) *p++ = *m++;
00524             t->a4 += 4;
00525         }
00526     }
00527 }
00528 /*
00529     Now e3 contains pairs of LeviCivita tensors that have
00530     three indices each and a3 is the total number of indices.
00531     e4 contains individual tensors with 4 indices.
00532     Some indices may be contracted with Kronecker delta's.
00533
00534     Contract the e3 tensors first.
00535 */
00536 while ( t->a3 > 0 ) {
00537     t->nt3[++(t->lc3)] = 0;
00538     while ( ( t->nt3[t->lc3] = EpfGen(3,t->e3+t->a3-6,
00539         t->pdel+t->ad,t->j3+6*t->lc3,oldsign = t->nt3[t->lc3]) ) == 0 ) {
00540         if ( oldsign < 0 ) t->sign2 = - t->sign2;
00541         (t->lc3)--;
00542     NextE3:
00543         if ( t->lc3 < 0 ) goto CallTrick;
00544         t->ad -= 6;
00545         t->a3 += 6;
00546     }
00547     if ( oldsign ) {
00548         if ( oldsign != t->nt3[t->lc3] ) t->sign2 = - t->sign2;
00549     }
00550     else if ( t->nt3[t->lc3] < 0 ) t->sign2 = - t->sign2;
00551     t->a3 -= 6;
00552     t->ad += 6;
00553 }
00554 /*
00555     Contract the e4 tensors.
00556 */
00557 while ( t->a4 > 4 ) {
00558     t->nt4[++(t->lc4)] = 0;
00559     while ( ( t->nt4[t->lc4] = EpfGen(4,t->e4+t->a4-8,
00560         t->pdel+t->ad,t->j4+8*t->lc4,oldsign = t->nt4[t->lc4]) ) == 0 ) {
00561         if ( oldsign < 0 ) t->sign2 = - t->sign2;
00562         (t->lc4)--;
00563     NextE4:
00564         if ( t->lc4 < 0 ) goto NextE3;
00565         t->ad -= 8;
00566         t->a4 += 8;
00567     }
00568     if ( oldsign ) {
00569         if ( oldsign != t->nt4[t->lc4] ) t->sign2 = - t->sign2;
00570     }
00571     else if ( t->nt4[t->lc4] < 0 ) t->sign2 = - t->sign2;
00572     t->a4 -= 8;
00573     t->ad += 8;
00574 }
00575 /*
00576     Finally the extra dummy indices can be eliminated.
00577     Note that there are currently t->ad words in t->pdel forming
00578     t->ad / 2 Kronecker delta's. We are however not allowed to
    alter anything in these arrays, so the results should be

```

```

00579         copied to kron.
00580     */
00581     m = kron;
00582     if ( t->a4 == 4 ) {
00583         p = t->e4;
00584         *m++ = *p++; *m++ = *p++; *m++ = *p++; *m++ = *p++;
00585         retval = 2;
00586     }
00587     else retval = 1;
00588     if ( t->sign2 < 0 ) retval = - retval;
00589     p = t->pdel;
00590     for ( i = 0; i < t->ad; i++ ) *m++ = *p++;
00591     p = kron;
00592     if ( t->a4 == 4 ) {
00593     /*
00594         Test for dummies in the last position of the e_.
00595     */
00596         stop = p + t->ad + 4;
00597         p += 3;
00598         while ( *p >= AM.mTraceDum && *p <= t->mdum ) {
00599             m = p + 1;
00600             do {
00601                 if ( *m == *p ) {
00602                     *p = m[1];
00603                     *m = *--stop;
00604                     m[1] = *--stop;
00605                     break;
00606                 }
00607                 else if ( m[1] == *p ) {
00608                     *p = *m;
00609                     *m = *--stop;
00610                     m[1] = *--stop;
00611                     break;
00612                 }
00613                 else m += 2;
00614             } while ( m < stop );
00615         }
00616         p++;
00617     }
00618     else stop = p + t->ad;
00619     while ( p < (stop-2) ) {
00620         while ( *p >= AM.mTraceDum && *p <= t->mdum ) {
00621             m = p + 2;
00622             do {
00623                 if ( *m == *p ) {
00624                     *p = m[1];
00625                     *m = *--stop;
00626                     m[1] = *--stop;
00627                     break;
00628                 }
00629                 else if ( m[1] == *p ) {
00630                     *p = *m;
00631                     *m = *--stop;
00632                     m[1] = *--stop;
00633                     break;
00634                 }
00635                 else m += 2;
00636             } while ( m < stop );
00637         }
00638         p++;
00639         while ( *p >= AM.mTraceDum && *p <= t->mdum ) {
00640             m = p + 1;
00641             do {
00642                 if ( *m == *p ) {
00643                     *p = m[1];
00644                     *m = *--stop;
00645                     m[1] = *--stop;
00646                     break;
00647                 }
00648                 else if ( m[1] == *p ) {
00649                     *p = *m;
00650                     *m = *--stop;
00651                     m[1] = *--stop;
00652                     break;
00653                 }
00654                 else m += 2;
00655             } while ( m < stop );
00656         }
00657         p++;
00658     }
00659     return(retval);
00660 }
00661 if ( number <= 2 ) return(0);
00662 else { goto NextE4; }
00663 }
00664
00665 /*

```

```

00666      #] Trace4no :
00667      #[ Trace4 :          WORD Trace4(term,params,num,level)
00668
00669      Generates traces of the string of gamma matrices in 'instring'.
00670
00671      The difference with the routine tracen ( for n dimensions )
00672      lies in the treatment of gamma 5 and the specific form
00673      of the Chisholm identities. The identities used here are
00674      g(mu)*g(al)*...*g(an)*g(mu)=
00675      n=odd: -2*g(an)*...*g(al)   ( reversed order )
00676      n=even: 2*g(an)*g(al)*...*g(a(n-1))
00677              +2*g(a(n-1))*...*g(al)*g(an)
00678      There is a special case for n=2 : 4*d(al,a2)*gi
00679
00680      The main difference with the old fortran version lies in
00681      the recursion that is used here. That cleans up the variables
00682      very much.
00683
00684      The contents of the AT.TMout array are:
00685      length,type,gamma5,gamma's
00686
00687      The space for the vectors in t is at most 14 * number.
00688
00689      The condition params[5] == 0 corresponds to finding gamma6*gamma7
00690      during the pick up of the matrices.
00691
00692  */
00693
00694 WORD Trace4(PHEAD WORD *term, WORD *params, WORD num, WORD level)
00695 {
00696     GETBIDENTITY
00697     TRACES *t;
00698     WORD *p, *m, number, i;
00699     WORD *OldW;
00700     WORD j, minimum, minimum2, *min, *stopper;
00701     OldW = AT.WorkPointer;
00702     if ( AN.numtracesctack >= AN.intracestack ) {
00703         number = AN.intracestack + 2;
00704         t = (TRACES *)Malloc1(number*sizeof(TRACES),"TRACES-struct");
00705         if ( AN.tracestack ) {
00706             for ( i = 0; i < AN.intracestack; i++ ) { t[i] = AN.tracestack[i]; }
00707             M_free(AN.tracestack,"TRACES-struct");
00708         }
00709         AN.tracestack = t;
00710         AN.intracestack = number;
00711     }
00712
00713     number = *params - 6;
00714     if ( number < 0 || ( number & 1 ) || !params[5] ) return(0);
00715
00716     t = AN.tracestack + AN.numtracesctack;
00717     AN.numtracesctack++;
00718
00719     t->finalstep = ( params[2] & 16 ) ? 1 : 0;
00720     t->gamma5 = params[3];
00721     if ( t->finalstep && t->gamma5 != GAMMA1 ) {
00722         MLOCK(ErrorMessageLock);
00723         MesPrint("Gamma5 not allowed in this option of the trace command");
00724         MUNLOCK(ErrorMessageLock);
00725         AN.numtracesctack--;
00726         SETERROR(-1)
00727     }
00728     t->inlist = AT.WorkPointer;
00729     t->accu = t->accu = t->inlist + number;
00730     t->perm = t->accu + (number*2);
00731     t->eers = t->perm + number;
00732     if ( ( AT.WorkPointer += 19 * number ) >= AT.WorkTop ) {
00733         MLOCK(ErrorMessageLock);
00734         MesWork();
00735         MUNLOCK(ErrorMessageLock);
00736         return(-1);
00737     }
00738     t->num = num;
00739     t->level = level;
00740     p = t->inlist;
00741     m = params+6;
00742     for ( i = 0; i < number; i++ ) *p++ = *m++;
00743     t->term = term;
00744     t->factor = params[4];
00745     t->allsign = params[5];
00746     if ( number >= 10 || ( t->gamma5 != GAMMA1 && number > 4 ) ) {
00747  /*
00748         The next code should `normal order' the string
00749         We need the lexicographic smallest string, taking also the
00750         reverse strings into account.
00751  */
00752         minimum = 0; min = t->inlist;

```

```

00753     stopper = min + number;
00754     for ( i = 1; i < number; i++ ) {
00755         p = min;
00756         m = t->inlist + i;
00757         for ( j = 0; j < number; j++ ) {
00758             if ( *p < *m ) break;
00759             if ( *p > *m ) {
00760                 min = t->inlist+i;
00761                 minimum = i;
00762                 break;
00763             }
00764             p++; m++;
00765             if ( m >= stopper ) m = t->inlist;
00766             if ( p >= stopper ) p = t->inlist;
00767         }
00768     }
00769     p = min;
00770     min = m = AT.WorkPointer;
00771     i = number;
00772     while ( --i >= 0 ) {
00773         *m++ = *p++;
00774         if ( p >= stopper ) p = t->inlist;
00775     }
00776     p = t->inlist;
00777     m = min;
00778     i = number;
00779     while ( --i >= 0 ) *p++ = *m++;
00780     p = t->inlist;
00781     m = stopper;
00782     while ( p < m ) { /* reverse string */
00783         i = *p; *p++ = *--m; *m = i;
00784     }
00785     minimum2 = 0;
00786     for ( i = 0; i < number; i++ ) {
00787         p = min;
00788         m = t->inlist + i;
00789         for ( j = 0; j < number; j++ ) {
00790             if ( *p < *m ) break;
00791             if ( *p > *m ) {
00792                 m = t->inlist + i;
00793                 p = min;
00794                 j = number;
00795                 while ( --j >= 0 ) {
00796                     *p++ = *m++;
00797                     if ( m >= stopper ) m = t->inlist;
00798                 }
00799                 minimum2 = i;
00800                 break;
00801             }
00802             p++; m++;
00803             if ( m >= stopper ) m = t->inlist;
00804         }
00805     }
00806     minimum ^= minimum2;
00807     if ( ( minimum & 1 ) != 0 ) {
00808         if ( t->gamma5 == GAMMA5 ) t->allsign = - t->allsign;
00809         else if ( t->gamma5 != GAMMA1 )
00810             t->gamma5 = GAMMA6 + GAMMA7 - t->gamma5;
00811     }
00812     p = min; m = t->inlist; i = number;
00813     while ( --i >= 0 ) *m++ = *p++;
00814 /*
00815     Now the trace is in normal order
00816 */
00817 }
00818 number = Trace4Gen(BHEAD t,number);
00819 AT.WorkPointer = OldW;
00820 AN.numtracesctack--;
00821 return(number);
00822 }
00823
00824 /*
00825 #] Trace4 :
00826 #[ Trace4Gen :          WORD Trace4Gen(t,number)
00827
00828 The recursive breakdown of a trace in 4 dimensions.
00829 We test first whether the trace has zero or two gamma's left.
00830 This case can be done quickly.
00831 Next we test whether we can eliminate adjacent objects that are
00832 the same.
00833 Then we test for Chisholm identities (I). First for identities
00834 with an odd number of gamma matrices (II), then for those with an
00835 even number of matrices (III). The special thing here is the demand
00836 that the contraction be between indices with 4 dimensions only.
00837 Then there is a scan for objects that are the same, not regarding
00838 their type (IV). This is exactly the same as in n dimensions.
00839 Finally we have a string left in which all objects are different (V).

```

```

00840      This case is treated by the routine Trace4no (no stands for no
00841      objects are the same).
00842
00843      In case I we have one d_ of which the result of the contraction
00844      has not yet been fixed.
00845      Case II gives just a reordering of the matrices and a factor -2.
00846      Case III gives two terms: one for the anti commutation, such that
00847      the number of intermediate matrices becomes odd and the other
00848      from the Chisholm rule for an odd number of matrices. Both have
00849      a factor 2.
00850      Case IV gives m+1 terms when m is the number of matrices inbetween.
00851      We take the shortest path. The sign alternates and all terms have
00852      a factor two, except for the last one.
00853
00854  */
00855
00856  WORD Trace4Gen(PHEAD TRACES *t, WORD number)
00857  {
00858      GETBIDENTITY
00859      WORD *termout, *stop;
00860      WORD *p, *m, oldval;
00861      WORD *pold, *mold, diff, *oldstring, cp;
00862  /*
00863      #[ Special cases :
00864  */
00865      if ( number <= 2 ) { /* Special cases */
00866          if ( t->gamma5 == GAMMA5 ) return(0);
00867          termout = AT.WorkPointer;
00868          p = t->termp;
00869          stop = p + *p;
00870          m = termout;
00871          p++;
00872          if ( p < stop ) do {
00873              if ( *p == SUBEXPRESSION && p[2] == t->num ) {
00874                  oldstring = p;
00875                  p = t->termp;
00876                  do { *m++ = *p++; } while ( p < oldstring );
00877                  p += p[1];
00878                  *m++ = AC.lUniTrace[0];
00879                  *m++ = AC.lUniTrace[1];
00880                  *m++ = AC.lUniTrace[2];
00881                  *m++ = AC.lUniTrace[3];
00882                  if ( number == 2 || t->accup > t->accu ) {
00883                      oldstring = m;
00884                      *m++ = DELTA;
00885                      *m++ = 4;
00886                      if ( number == 2 ) {
00887                          *m++ = t->inlist[0];
00888                          *m++ = t->inlist[1];
00889                      }
00890                      if ( t->accup > t->accu ) {
00891                          pold = p;
00892                          p = t->accu;
00893                          while ( p < t->accup ) *m++ = *p++;
00894                          oldstring[1] = WORDDIF(m,oldstring);
00895                          p = pold;
00896                      }
00897                  }
00898                  if ( t->factor ) {
00899                      *m++ = SNUMBER;
00900                      *m++ = 4;
00901                      *m++ = 2;
00902                      *m++ = t->factor;
00903                  }
00904                  do { *m++ = *p++; } while ( p < stop );
00905                  *termout = WORDDIF(m,termout);
00906                  if ( t->allsign < 0 ) m[-1] = -m[-1];
00907                  if ( ( AT.WorkPointer = m ) > AT.WorkTop ) {
00908                      MLOCK(ErrorMessageLock);
00909                      MesWork();
00910                      MUNLOCK(ErrorMessageLock);
00911                      return(-1);
00912                  }
00913                  *AN.RepPoint = 1;
00914                  AR.expchanged = 1;
00915                  if ( *termout ) {
00916                      *AN.RepPoint = 1;
00917                      AR.expchanged = 1;
00918                      if ( Generator(BHEAD termout,t->level) ) goto TracCall;
00919                  }
00920                  AT.WorkPointer= termout;
00921                  return(0);
00922              }
00923              p += p[1];
00924          } while ( p < stop );
00925          return(0);
00926      }

```

```

00927 /*
00928         #] Special cases :
00929         #[ Adjacent objects :
00930 */
00931     p = t->inlist;
00932     stop = p + number - 1;
00933     if ( *p == *stop ) {          /* First and last of string */
00934         oldval = *p;
00935         *(t->accup)++ = *p;
00936         *(t->accup)++ = *p;
00937         m = p+1;
00938         while ( m < stop ) *p++ = *m++;
00939         if ( t->gamma5 != GAMMA1 ) {
00940             if ( t->gamma5 == GAMMA5 ) t->allsign = - t->allsign;
00941             else if ( t->gamma5 == GAMMA6 ) t->gamma5 = GAMMA7;
00942             else if ( t->gamma5 == GAMMA7 ) t->gamma5 = GAMMA6;
00943         }
00944         if ( Trace4Gen(BHEAD t,number-2) ) goto TracCall;
00945         t = AN.tracestack + AN.numtracesctack - 1;
00946         if ( t->gamma5 != GAMMA1 ) {
00947             if ( t->gamma5 == GAMMA5 ) t->allsign = - t->allsign;
00948             else if ( t->gamma5 == GAMMA6 ) t->gamma5 = GAMMA7;
00949             else if ( t->gamma5 == GAMMA7 ) t->gamma5 = GAMMA6;
00950         }
00951         while ( p > t->inlist ) *--m = *--p;
00952         *p = *stop = oldval;
00953         t->accup -= 2;
00954         return(0);
00955     }
00956     do {
00957         if ( *p == p[1] ) {          /* Adjacent in string */
00958             oldval = *p;
00959             pold = p;
00960             m = p+2;
00961             *(t->accup)++ = *p;
00962             *(t->accup)++ = *p;
00963             while ( m <= stop ) *p++ = *m++;
00964             if ( Trace4Gen(BHEAD t,number-2) ) goto TracCall;
00965             t = AN.tracestack + AN.numtracesctack - 1;
00966             while ( p > pold ) *--m = *--p;
00967             *p++ = oldval;
00968             *p++ = oldval;
00969             t->accup -= 2;
00970             return(0);
00971         }
00972         p++;
00973     } while ( p < stop );
00974 /*
00975         #] Adjacent objects :
00976         #[ Odd Contraction :
00977 */
00978     p = t->inlist;
00979     stop = p + number;
00980     do {
00981         if ( *p >= AM.OffsetIndex && (
00982             ( *p < WILDOFFSET + AM.OffsetIndex &&
00983               indices[*p-AM.OffsetIndex].dimension == 4 )
00984             || ( *p >= WILDOFFSET + AM.OffsetIndex && AC.lDefDim == 4 ) ) ) {
00985             m = p+2;
00986             while ( m < stop ) {
00987                 if ( *p == *m ) {
00988                     pold = p;
00989                     mold = m;
00990                     oldval = *p;
00991                     (t->factor)++;
00992                     t->allsign = - t->allsign;
00993                     *p++ = *--m;
00994                     m--;
00995                     while ( m > p ) { diff = *p; *p++ = *m; *m-- = diff; }
00996                     p = mold - 1;
00997                     m = mold + 1;
00998                     while ( m < stop ) *p++ = *m++;
00999                     if ( Trace4Gen(BHEAD t,number-2) ) goto TracCall;
01000                     t = AN.tracestack + AN.numtracesctack - 1;
01001                     m--;
01002                     while ( m > mold ) *m-- = *--p;
01003                     p = pold;
01004                     *m-- = oldval;
01005                     *m-- = *p;
01006                     *p++ = oldval;
01007                     while ( m > p ) { diff = *p; *p++ = *m; *m-- = diff; }
01008                     t->allsign = - t->allsign;
01009                     (t->factor)--;
01010                     return(0);
01011                 }
01012                 m += 2;
01013             }

```

```

01014     }
01015     p++;
01016 } while ( p < stop );
01017 /*
01018     #] Odd Contraction :
01019     #[ Even Contraction :
01020     First the case with two matrices inbetween.
01021 */
01022 p = t->inlist;
01023 stop = p + number;
01024 do {
01025     if ( *p >= AM.OffsetIndex && (
01026         ( *p < WILDOFFSET + AM.OffsetIndex &&
01027         indices[*p-AM.OffsetIndex].dimension == 4 )
01028         || ( *p >= WILDOFFSET + AM.OffsetIndex && AC.lDefDim == 4 ) ) ) {
01029         m = p+3;
01030         if ( m >= stop ) m -= number;
01031         if ( *p == *m ) {
01032             WORD oldfactor, old5;
01033             oldstring = AT.WorkPointer;
01034             AT.WorkPointer += number;
01035             oldfactor = t->allsign;
01036             old5 = t->gamma5;
01037             if ( m < p ) cp = (WORDDIF(m,t->inlist) + 1) & 1;
01038             else cp = 0;
01039             if ( cp && ( t->gamma5 != GAMMA1 ) ) {
01040                 if ( t->gamma5 == GAMMA5 ) t->allsign = -t->allsign;
01041                 else if ( t->gamma5 == GAMMA6 ) t->gamma5 = GAMMA7;
01042                 else if ( t->gamma5 == GAMMA7 ) t->gamma5 = GAMMA6;
01043             }
01044             mold = m;
01045             p = oldstring;
01046             m = t->inlist;
01047             while ( m < stop ) *p++ = *m++;
01048 /*
01049         Rotate the string
01050 */
01051             m = mold + 1;
01052             p = t->inlist;
01053             while ( m < stop ) *p++ = *m++;
01054             m = oldstring;
01055             if ( !cp && ((WORDDIF(stop,p))&1) != 0 && ( t->gamma5 != GAMMA1 ) ) {
01056                 if ( t->gamma5 == GAMMA5 ) t->allsign = -t->allsign;
01057                 else if ( t->gamma5 == GAMMA6 ) t->gamma5 = GAMMA7;
01058                 else if ( t->gamma5 == GAMMA7 ) t->gamma5 = GAMMA6;
01059             }
01060             while ( p < stop ) *p++ = *m++;
01061             t->factor += 2;
01062             m = p - 3;
01063             p = t->inlist;
01064             oldval = number - 4;
01065             while ( oldval > 0 ) {
01066                 if ( *p >= AM.OffsetIndex && (
01067                     ( *p < WILDOFFSET + AM.OffsetIndex &&
01068                     indices[*p-AM.OffsetIndex].dimension )
01069                     || ( *p >= WILDOFFSET + AM.OffsetIndex && AC.lDefDim ) ) ) {
01070                     if ( *p == *m ) {
01071                         *p = m[1];
01072                         break;
01073                     }
01074                     else if ( *p == m[1] ) {
01075                         *p = *m;
01076                         break;
01077                     }
01078                 }
01079                 p++; oldval--;
01080             }
01081             if ( oldval <= 0 ) {
01082                 *(t->accup)++ = *m++;
01083                 *(t->accup)++ = *m++;
01084             }
01085             if ( Trace4Gen(BHEAD t,number-4) ) goto TracCall;
01086             t = AN.tracestack + AN.numtracesctack - 1;
01087             t->factor -= 2;
01088             if ( oldval <= 0 ) t->accup -= 2;
01089             t->gamma5 = old5;
01090             t->allsign = oldfactor;
01091             AT.WorkPointer = p = oldstring;
01092             m = t->inlist;
01093             while ( m < stop ) *m++ = *p++;
01094             return(0);
01095         }
01096     }
01097     p++;
01098 } while ( p < stop );
01099 /*
01100     The case with at least 4 matrices inbetween.

```

```

01101 */
01102     p = t->inlist;
01103     stop = p + number;
01104     do {
01105         if ( *p >= AM.OffsetIndex && (
01106             ( *p < WILDOFFSET + AM.OffsetIndex &&
01107               indices[*p-AM.OffsetIndex].dimension == 4 )
01108             || ( *p >= WILDOFFSET + AM.OffsetIndex && AC.lDefDim == 4 ) ) ) {
01109             m = p+5;
01110             while ( m < stop ) {
01111                 if ( *p == *m ) {
01112                     WORD *pex, *mex;
01113                     pold = p;
01114                     mold = m;
01115                     oldval = *p;
01116             /*
01117                 g_(1,mu)*g_(1,al)*...*g_(1,aj)*g_(1,an)*g_(1,mu) ->
01118                 first:
01119                 2*g_(1,an)*g_(1,al)*...*g_(1,aj)
01120             */
01121                 (t->factor)++;
01122             /*
01123                 The variable hulp seems unnecessary
01124                 *p = hulp = m[-1];
01125             */
01126                 *p = m[-1];
01127                 p = m - 1;
01128                 m++;
01129                 while ( m < stop ) *p++ = *m++;
01130                 if ( Trace4Gen(BHEAD t,number-2) ) goto TracCall;
01131                 t = AN.tracestack + AN.numtracesctack - 1;
01132                 pex = p; mex = m;
01133                 p = pold;
01134                 m = mold - 2;
01135                 while ( m > p ) { diff = *p; *p++ = *m; *m-- = diff; }
01136             /*
01137                 and then:
01138                 2*g_(1,aj)*...*g_(1,al)*g_(1,an)
01139             */
01140                 if ( Trace4Gen(BHEAD t,number-2) ) goto TracCall;
01141                 t = AN.tracestack + AN.numtracesctack - 1;
01142                 p = pold;
01143                 m = mold - 2;
01144                 while ( m > p ) { diff = *p; *p++ = *m; *m-- = diff; }
01145                 m = mex;
01146                 p = pex;
01147                 m--;
01148                 while ( m > mold ) *m-- = *--p;
01149                 m = mold;
01150                 *m-- = oldval;
01151                 p = pold;
01152                 *m = *p;
01153                 *p = oldval;
01154                 (t->factor)--;
01155                 return(0);
01156             }
01157             m += 2;
01158         }
01159     }
01160     p++;
01161 } while ( p < stop );
01162 /*
01163     #] Even Contraction :
01164     #[ Same Objects :
01165 */
01166     p = t->inlist;
01167     stop = p + number - 1;
01168     diff = 2;
01169     do {
01170         p = t->inlist;
01171         while ( p <= stop ) {
01172             m = p + diff;
01173             if ( m > stop ) m -= number;
01174             if ( *p == *m ) {
01175                 WORD oldfactor, c, old5;
01176                 oldfactor = t->allsign;
01177                 old5 = t->gamma5;
01178                 cp = (WORDDIF(m,t->inlist)) & 1;
01179                 if ( !cp && ( t->gamma5 != GAMMA1 ) ) {
01180                     if ( t->gamma5 == GAMMA5 ) t->allsign = -t->allsign;
01181                     else if ( t->gamma5 == GAMMA6 ) t->gamma5 = GAMMA7;
01182                     else if ( t->gamma5 == GAMMA7 ) t->gamma5 = GAMMA6;
01183                 }
01184                 oldstring = AT.WorkPointer;
01185                 AT.WorkPointer += number;
01186                 mold = m;
01187                 oldval = *p;

```



```

01188         p = oldstring;
01189         m = t->inlist;
01190         while ( m <= stop ) *p++ = *m++;
01191 /*
01192         Rotate the string
01193 */
01194         m = mold + 1;
01195         p = t->inlist;
01196         while ( m <= stop ) *p++ = *m++;
01197         m = oldstring;
01198         while ( p <= stop ) *p++ = *m++;
01199         (t->factor)++;
01200         p -= diff + 1;
01201         m = stop;
01202         *(t->accup) = oldval;
01203         t->accup += 2;
01204         m--;
01205         while ( m > p ) {
01206             c = t->accup[-1];
01207             t->accup[-1] = *m;
01208             *m = c;
01209             if ( Trace4Gen(BHEAD t,number-2) ) goto Trac4Call;
01210             t = AN.tracestack + AN.numtracesctack - 1;
01211             m--;
01212             t->allsign = - t->allsign;
01213         }
01214         c = t->accup[-1];
01215         t->accup[-1] = *m;
01216         *m = c;
01217         (t->factor)--;
01218         if ( Trace4Gen(BHEAD t,number-2) ) goto Trac4Call;
01219         t = AN.tracestack + AN.numtracesctack - 1;
01220         t->accup -= 2;
01221         t->allsign = oldfactor;
01222         AT.WorkPointer = p = oldstring;
01223         m = t->inlist;
01224         while ( m <= stop ) *m++ = *p++;
01225         t->gamma5 = old5;
01226         return(0);
01227     }
01228     p++;
01229 }
01230 } while ( ++diff <= (number>>1) );
01231 /*
01232     #] Same Objects :
01233     #[ All Different :
01234
01235     Here we have a string with all different objects.
01236
01237 */
01238     t->sgn = 0;
01239     termout = AT.WorkPointer;
01240     for(;;) {
01241         if ( t->finalstep == 0 ) diff = Trace4no(number,t->accup,t);
01242         else diff = TraceNno(number,t->accup,t);
01243 /* while ( ( diff = Trace4no(number,t->accup,t) ) != 0 ) */
01244         if ( diff == 0 ) break;
01245         p = t->termp;
01246         stop = p + *p;
01247         m = termout;
01248         p++;
01249         if ( p < stop ) do {
01250             if ( *p == SUBEXPRESSION && p[2] == t->num ) {
01251                 oldstring = p;
01252                 p = t->termp;
01253                 do { *m++ = *p++; } while ( p < oldstring );
01254                 p += p[1];
01255                 pold = p;
01256                 *m++ = AC.lUniTrace[0];
01257                 *m++ = AC.lUniTrace[1];
01258                 *m++ = AC.lUniTrace[2];
01259                 *m++ = AC.lUniTrace[3];
01260                 *m++ = SNUMBER;
01261                 *m++ = 4;
01262                 *m++ = 2;
01263                 *m++ = t->factor;
01264                 p = t->accup;
01265                 oldval = number;
01266                 if ( diff == 2 || diff == -2 ) {
01267                     *m++ = LEVICIVITA;
01268                     *m++ = 4+FUNHEAD;
01269                     *m++ = DIRTYFLAG;
01270                     FILLFUN3(m)
01271                     *m++ = *p++; *m++ = *p++; *m++ = *p++; *m++ = *p++;
01272                     oldval -= 4;
01273                 }
01274                 if ( oldval > 0 || t->accup > t->accu ) {

```

```

01275         oldstring = m;
01276         *m++ = DELTA;
01277         *m++ = oldval + 2;
01278         if ( oldval > 0 ) NCOPY(m,p,oldval);
01279         if ( t->accup > t->accu ) {
01280             p = t->accu;
01281             while ( p < t->accup ) *m++ = *p++;
01282             oldstring[1] = WORDDIF(m,oldstring);
01283         }
01284     }
01285     p = pold;
01286     do { *m++ = *p++; } while ( p < stop );
01287     *termout = WORDDIF(m,termout);
01288     if ( ( diff ^ t->allsign ) < 0 ) m[-1] = - m[-1];
01289     if ( ( AT.WorkPointer = m ) > AT.WorkTop ) {
01290         MLOCK(ErrorMessageLock);
01291         MesWork();
01292         MUNLOCK(ErrorMessageLock);
01293         return(-1);
01294     }
01295     if ( *termout ) {
01296         *AN.RepPoint = 1;
01297         AR.expchanged = 1;
01298         if ( Generator(BHEAD termout,t->level) ) {
01299             AT.WorkPointer = termout;
01300             goto TracCall;
01301         }
01302         t = AN.tracestack + AN.numtracesctack - 1;
01303     }
01304     break;
01305 }
01306 p += p[1];
01307 } while ( p < stop );
01308 }
01309 AT.WorkPointer = termout;
01310 return(0);
01311
01312 /*
01313         #] All Different :
01314 */
01315 Trac4Call:
01316     AT.WorkPointer = oldstring;
01317 TracCall:
01318     if ( AM.tracebackflag ) {
01319         MLOCK(ErrorMessageLock);
01320         MesCall("Trace4Gen");
01321         MUNLOCK(ErrorMessageLock);
01322     }
01323     return(-1);
01324 }
01325
01326 /*
01327         #] Trace4Gen :
01328         #[ TraceNno :           WORD TraceNno(number,kron,t)
01329
01330         Routine takes the trace in N dimensions of a string
01331         of gamma matrices. It is assumed that there are no
01332         contractions and no vectors that are the same. For
01333         the treatment of those cases there are special routines,
01334         that call this routine as a final stage.
01335         The calling routine must reserve 'number' WORDs for perm
01336         and kron each.
01337         kron and perm may not be altered during the generation!
01338
01339 */
01340
01341 WORD TraceNno(WORD number, WORD *kron, TRACES *t)
01342 {
01343     WORD i, j, a, *p;
01344     if ( !t->sgn ) {
01345         if ( !number || ( number & 1 ) ) return(0);
01346         p = t->inlist;
01347         for ( i = 0; i < number; i++ ) {
01348             t->perm[i] = i;
01349             *kron++ = *p++;
01350         }
01351         t->sgn = 1;
01352         return(1);
01353     }
01354     else {
01355         i = number - 3;
01356         while ( i > 0 ) {
01357             a = kron[i];
01358             p = t->perm + i;
01359             for ( j = i + 1; j <= *p; j++ ) kron[j-1] = kron[j];
01360             kron[( *p )++] = a;
01361             if ( *p < number ) {

```

```

01362         a = kron[*p];
01363         j = *p;
01364         while ( j >= (i+1) ) { kron[j] = kron[j-1]; j--; }
01365         kron[i] = a;
01366         number -= 2;
01367         for ( j = i+2; j < number; j += 2 ) t->perm[j] = j;
01368         t->sgn = - t->sgn;
01369         return(t->sgn);
01370     }
01371     i -= 2;
01372 }
01373 }
01374 return(0);
01375 }
01376
01377 /*
01378     #] TraceNno :
01379     #[ TraceN :          WORD TraceN(term,params,num,level)
01380 */
01381
01382 WORD TraceN(PHEAD WORD *term, WORD *params, WORD num, WORD level)
01383 {
01384     GETBIDENTITY
01385     TRACES *t;
01386     WORD *p, *m, number, i;
01387     WORD *OldW;
01388     if ( params[3] != GAMMA1 ) {
01389         MLOCK(ErrorMessageLock);
01390         MesPrint("Gamma5 not allowed in n-trace");
01391         MUNLOCK(ErrorMessageLock);
01392         SETERROR(-1)
01393     }
01394     OldW = AT.WorkPointer;
01395     if ( AN.numtracesctack >= AN.intracestack ) {
01396         number = AN.intracestack + 2;
01397         t = (TRACES *)Malloca(number*sizeof(TRACES),"TRACES-struct");
01398         if ( AN.tracestack ) {
01399             for ( i = 0; i < AN.intracestack; i++ ) { t[i] = AN.tracestack[i]; }
01400             M_free(AN.tracestack,"TRACES-struct");
01401         }
01402         AN.tracestack = t;
01403         AN.intracestack = number;
01404     }
01405     number = *params - 6;
01406     if ( number < 0 || ( number & 1 ) || !params[5] ) return(0);
01407
01408     t = AN.tracestack + AN.numtracesctack;
01409     AN.numtracesctack++;
01410
01411     t->inlist = AT.WorkPointer;
01412     t->accu = t->accu = t->inlist + number;
01413     t->perm = t->accu + number;
01414     if ( ( AT.WorkPointer += 3 * number ) >= AT.WorkTop ) {
01415         AN.numtracesctack--;
01416         MLOCK(ErrorMessageLock);
01417         MesWork();
01418         MUNLOCK(ErrorMessageLock);
01419         return(-1);
01420     }
01421     t->num = num;
01422     t->level = level;
01423     p = t->inlist;
01424     m = params+6;
01425     for ( i = 0; i < number; i++ ) *p++ = *m++;
01426     t->termp = term;
01427     t->factor = params[4];
01428     t->allsign = params[5];
01429     number = TraceNgen(BHEAD t,number);
01430     AT.WorkPointer = OldW;
01431     AN.numtracesctack--;
01432     return(number);
01433 }
01434
01435 /*
01436     #] TraceN :
01437     #[ TraceNgen :          WORD TraceNgen(t,number)
01438
01439     This routine is a simplified version of Trace4Gen. We know here
01440     only three cases: Adjacent objects, same objects and all different.
01441     The other difference lies of course in the struct which is now
01442     not of type TRACES, but of type TRACES.
01443 */
01444
01445 WORD TraceNgen(PHEAD TRACES *t, WORD number)
01446 {
01447     GETBIDENTITY

```

```

01449     WORD *termout, *stop;
01450     WORD *p, *m, oldval;
01451     WORD *pold, *mold, diff, *oldstring;
01452 /*
01453         #[] Special cases :
01454 */
01455     if ( number <= 2 ) { /* Special cases */
01456         termout = AT.WorkPointer;
01457         p = t->termp;
01458         stop = p + *p;
01459         m = termout;
01460         p++;
01461         if ( p < stop ) do {
01462             if ( *p == SUBEXPRESSION && p[2] == t->num ) {
01463                 oldstring = p;
01464                 p = t->termp;
01465                 do { *m++ = *p++; } while ( p < oldstring );
01466                 p += p[1];
01467                 *m++ = AC.lUniTrace[0];
01468                 *m++ = AC.lUniTrace[1];
01469                 *m++ = AC.lUniTrace[2];
01470                 *m++ = AC.lUniTrace[3];
01471                 if ( number == 2 || t->accup > t->accu ) {
01472                     oldstring = m;
01473                     *m++ = DELTA;
01474                     *m++ = 4;
01475                     if ( number == 2 ) {
01476                         *m++ = t->inlist[0];
01477                         *m++ = t->inlist[1];
01478                     }
01479                     if ( t->accup > t->accu ) {
01480                         pold = p;
01481                         p = t->accu;
01482                         while ( p < t->accup ) *m++ = *p++;
01483                         oldstring[1] = WORDDIFF(m,oldstring);
01484                         p = pold;
01485                     }
01486                 }
01487                 if ( t->factor ) {
01488                     *m++ = SNUMBER;
01489                     *m++ = 4;
01490                     *m++ = 2;
01491                     *m++ = t->factor;
01492                 }
01493                 do { *m++ = *p++; } while ( p < stop );
01494                 *termout = WORDDIFF(m,termout);
01495                 if ( t->allsign < 0 ) m[-1] = -m[-1];
01496                 if ( ( AT.WorkPointer = m ) > AT.WorkTop ) {
01497                     MLOCK(ErrorMessageLock);
01498                     MesWork();
01499                     MUNLOCK(ErrorMessageLock);
01500                     return(-1);
01501                 }
01502                 if ( *termout ) {
01503                     *AN.RepPoint = 1;
01504                     AR.expcchanged = 1;
01505                     if ( Generator(BHEAD termout,t->level) ) goto TracCall;
01506                 }
01507                 AT.WorkPointer= termout;
01508                 return(0);
01509             }
01510             p += p[1];
01511         } while ( p < stop );
01512         return(0);
01513     }
01514 /*
01515         #[] Special cases :
01516         #[] Adjacent objects :
01517 */
01518     p = t->inlist;
01519     stop = p + number - 1;
01520     if ( *p == *stop ) { /* First and last of string */
01521         oldval = *p;
01522         *(t->accup)++ = *p;
01523         *(t->accup)++ = *p;
01524         m = p+1;
01525         while ( m < stop ) *p++ = *m++;
01526         if ( TraceNgen(BHEAD t,number-2) ) goto TracCall;
01527         t = AN.tracestack + AN.numtracesctack - 1;
01528         while ( p > t->inlist ) *--m = *--p;
01529         *p = *stop = oldval;
01530         t->accup -= 2;
01531         return(0);
01532     }
01533     do {
01534         if ( *p == p[1] ) { /* Adjacent in string */
01535             oldval = *p;

```

```

01536         pold = p;
01537         m = p+2;
01538         *(t->accup)++ = *p;
01539         *(t->accup)++ = *p;
01540         while ( m <= stop ) *p++ = *m++;
01541         if ( TraceNgen(BHEAD t,number-2) ) goto TracCall;
01542         t = AN.tracestack + AN.numtracesctack - 1;
01543         while ( p > pold ) *--m = *--p;
01544         *p++ = oldval;
01545         *p++ = oldval;
01546         t->accup -= 2;
01547         return(0);
01548     }
01549     p++;
01550 } while ( p < stop );
01551 /*
01552     #] Adjacent objects :
01553     #[ Same Objects :
01554 */
01555     p = t->inlist;
01556     stop = p + number - 1;
01557     diff = 2;
01558     do {
01559         p = t->inlist;
01560         while ( p <= stop ) {
01561             m = p + diff;
01562             if ( m > stop ) m -= number;
01563             if ( *p == *m ) {
01564                 WORD oldfactor, c;
01565                 oldstring = AT.WorkPointer;
01566                 AT.WorkPointer += number;
01567                 mold = m;
01568                 oldval = *p;
01569                 p = oldstring;
01570                 m = t->inlist;
01571                 while ( m <= stop ) *p++ = *m++;
01572             /*
01573             Rotate the string
01574             */
01575             {
01576                 m = mold + 1;
01577                 p = t->inlist;
01578                 while ( m <= stop ) *p++ = *m++;
01579                 m = oldstring;
01580                 while ( p <= stop ) *p++ = *m++;
01581             }
01582             oldfactor = t->allsign;
01583             (t->factor)++;
01584             p -= diff + 1;
01585             m = stop;
01586             if ( oldval >= ( AM.OffsetIndex + WILDOFFSET ) ||
01587                 ( oldval >= AM.OffsetIndex
01588                   && indices[oldval-AM.OffsetIndex].dimension ) ) {
01589                 m--;
01590             /*
01591             We distinguish 4 cases:
01592             m-p=1   Use g_(1,mu,a,mu) = (2-d_(mu,mu))*g_(1,a)
01593             m-p=2   Use g_(1,mu,a,b,mu) = 4*d_(a,b)+(d_(mu,mu)-4)*g_(1,a,b)
01594             m-p=3   Use g_(1,mu,a,b,c,mu) = -2*g_(1,c,b,a)
01595                     - (d_(mu,mu)-4)*g_(1,a,b,c)
01596             m-p>3   Reduce down to m-p=3 with the old technique
01597             */
01598             while ( m > (p+3) ) {
01599                 c = *p;
01600                 *p = *m;
01601                 *m = c;
01602                 if ( TraceNgen(BHEAD t,number-2) ) goto TracnCall;
01603                 t = AN.tracestack + AN.numtracesctack - 1;
01604                 m--;
01605                 t->allsign = - t->allsign;
01606             }
01607             switch ( WORDDIF(m,p) ) {
01608             case 1:
01609                 c = *p;
01610                 *p = *m;
01611                 *m = c;
01612                 if ( oldval < ( AM.OffsetIndex + WILDOFFSET )
01613                     && indices[oldval-AM.OffsetIndex].nmin4
01614                     != -NMIN4SHIFT ) {
01615                     t->allsign = - t->allsign;
01616                     if ( TraceNgen(BHEAD t,number-2) ) goto TracnCall;
01617                     t = AN.tracestack + AN.numtracesctack - 1;
01618                     (t->factor)--;
01619                     *(t->accup)++ = SUMMEDIND;
01620                     *(t->accup)++ =
01621                         indices[oldval-AM.OffsetIndex].nmin4;
01622                 }

```

```

01623         else
01624         {
01625             if ( TraceNgen(BHEAD t,number-2) ) goto TracnCall;
01626             t = AN.tracestack + AN.numtracesctack - 1;
01627             t->allsign = - t->allsign;
01628             (t->factor)--;
01629             *(t->accup)++ = oldval;
01630             *(t->accup)++ = oldval;
01631         }
01632         if ( TraceNgen(BHEAD t,number-2) ) goto TracnCall;
01633         t = AN.tracestack + AN.numtracesctack - 1;
01634         t->accup -= 2;
01635         break;
01636     case 2:
01637     { WORD one, two;
01638         one = *p = p[1];
01639         two = p[1] = *m;
01640         (t->factor)++;          /* 4 */
01641         *(t->accup)++ = *p;      /* d_(a,b) */
01642         *(t->accup)++ = *m;
01643         if ( TraceNgen(BHEAD t,number-4) ) goto TracnCall;
01644         t = AN.tracestack + AN.numtracesctack - 1;
01645         *p = one; p[1] = two;
01646         t->accup -= 2;
01647         if ( oldval < ( AM.OffsetIndex + WILDOFFSET )
01648             && indices[oldval-AM.OffsetIndex].nmin4
01649             != -NMIN4SHIFT ) {
01650             t->factor -= 2;
01651             *(t->accup)++ = SUMMEDIND;
01652             *(t->accup)++ =
01653                 indices[oldval-AM.OffsetIndex].nmin4;
01654         }
01655         else {
01656             t->allsign = - t->allsign;
01657             if ( TraceNgen(BHEAD t,number-2) ) goto TracnCall;
01658             t = AN.tracestack + AN.numtracesctack - 1;
01659             t->allsign = - t->allsign;
01660             t->factor -= 2;
01661             *(t->accup)++ = oldval;
01662             *(t->accup)++ = oldval;
01663         }
01664         if ( TraceNgen(BHEAD t,number-2) ) goto TracnCall;
01665         t = AN.tracestack + AN.numtracesctack - 1;
01666         t->accup -= 2;
01667     }
01668     break;
01669     default:
01670     c = *p;
01671     *p = *m;
01672     *m = c;
01673     c = m[-1]; m[-1] = m[-2]; m[-2] = c;
01674     t->allsign = - t->allsign;
01675     if ( TraceNgen(BHEAD t,number-2) ) goto TracnCall;
01676     t = AN.tracestack + AN.numtracesctack - 1;
01677     m--;
01678     c = *p;
01679     *p = *m;
01680     *m = c;
01681     (t->factor)--;
01682     if ( oldval < ( AM.OffsetIndex + WILDOFFSET )
01683         && indices[oldval-AM.OffsetIndex].nmin4
01684         != -NMIN4SHIFT ) {
01685         *(t->accup)++ = SUMMEDIND;
01686         *(t->accup)++ =
01687             indices[oldval-AM.OffsetIndex].nmin4;
01688         if ( TraceNgen(BHEAD t,number-2) ) goto TracnCall;
01689         t = AN.tracestack + AN.numtracesctack - 1;
01690         t->accup -= 2;
01691         t->allsign = - t->allsign;
01692     }
01693     else
01694     {
01695         *(t->accup)++ = oldval;
01696         *(t->accup)++ = oldval;
01697         if ( TraceNgen(BHEAD t,number-2) ) goto TracnCall;
01698         t = AN.tracestack + AN.numtracesctack - 1;
01699         t->accup -= 2;
01700         t->allsign = - t->allsign;
01701         t->factor += 2;
01702         if ( TraceNgen(BHEAD t,number-2) ) goto TracnCall;
01703         t = AN.tracestack + AN.numtracesctack - 1;
01704         t->factor -= 2;
01705     }
01706     break;
01707 }
01708 }
01709 else {

```

```

01710         *(t->accup) = oldval;
01711         t->accup += 2;
01712         m--;
01713         while ( m > p ) {
01714             c = t->accup[-1];
01715             t->accup[-1] = *m;
01716             *m = c;
01717             if ( TraceNgen(BHEAD t,number-2) ) goto TracnCall;
01718             t = AN.tracestack + AN.numtracesctack - 1;
01719             m--;
01720             t->allsign = - t->allsign;
01721         }
01722         c = t->accup[-1];
01723         t->accup[-1] = *m;
01724         *m = c;
01725         (t->factor)--;
01726         if ( TraceNgen(BHEAD t,number-2) ) goto TracnCall;
01727         t = AN.tracestack + AN.numtracesctack - 1;
01728         t->accup -= 2;
01729     }
01730     t->allsign = oldfactor;
01731     p = oldstring;
01732     m = t->inlist;
01733     while ( m <= stop ) *m++ = *p++;
01734     AT.WorkPointer = oldstring;
01735     return(0);
01736 }
01737 p++;
01738 }
01739 diff++;
01740 } while ( diff <= (number»1) );
01741 /*
01742     #] Same Objects :
01743     #[ All Different :
01744
01745     Here we have a string with all different objects.
01746 */
01747 */
01748 t->sgn = 0;
01749 termout = AT.WorkPointer;
01750 while ( ( diff = TraceNno(number,t->accup,t) ) != 0 ) {
01751     p = t->termp;
01752     stop = p + *p;
01753     m = termout;
01754     p++;
01755     if ( p < stop ) do {
01756         if ( *p == SUBEXPRESSION && p[2] == t->num ) {
01757             oldstring = p;
01758             p = t->termp;
01759             do { *m++ = *p++; } while ( p < oldstring );
01760             p += p[1];
01761             pold = p;
01762             *m++ = AC.lUniTrace[0];
01763             *m++ = AC.lUniTrace[1];
01764             *m++ = AC.lUniTrace[2];
01765             *m++ = AC.lUniTrace[3];
01766             *m++ = SNUMBER;
01767             *m++ = 4;
01768             *m++ = 2;
01769             *m++ = t->factor;
01770             p = t->accup;
01771             oldval = number;
01772             oldstring = m;
01773             *m++ = DELTA;
01774             *m++ = oldval + 2;
01775             NCOPY(m,p,oldval);
01776             if ( t->accup > t->accu ) {
01777                 p = t->accu;
01778                 while ( p < t->accup ) *m++ = *p++;
01779                 oldstring[1] = WORDDIFF(m,oldstring);
01780             }
01781             p = pold;
01782             do { *m++ = *p++; } while ( p < stop );
01783             *termout = WORDDIFF(m,termout);
01784             if ( ( diff ^ t->allsign ) < 0 ) m[-1] = - m[-1];
01785             if ( ( AT.WorkPointer = m ) > AT.WorkTop ) {
01786                 MLOCK(ErrorMessageLock);
01787                 MesWork();
01788                 MUNLOCK(ErrorMessageLock);
01789                 return(-1);
01790             }
01791             if ( *termout ) {
01792                 *AN.RepPoint = 1;
01793                 AR.expchanged = 1;
01794                 if ( Generator(BHEAD termout,t->level) ) {
01795                     AT.WorkPointer = termout;
01796                     goto TracCall;

```

```

01797         }
01798         t = AN.tracestack + AN.numtracesctack - 1;
01799     }
01800     break;
01801 }
01802 p += p[1];
01803 } while ( p < stop );
01804 }
01805 AT.WorkPointer = termout;
01806 return(0);
01807
01808 /*
01809     #] All Different :
01810 */
01811 TracnCall:
01812     AT.WorkPointer = oldstring;
01813 TracCall:
01814     if ( AM.tracebackflag ) {
01815         MLOCK(ErrorMessageLock);
01816         MesCall("TraceNgen");
01817         MUNLOCK(ErrorMessageLock);
01818     }
01819     return(-1);
01820 }
01821
01822 /*
01823     #] TraceNgen :
01824     #[ Traces :           WORD Traces(term,params,num,level)
01825
01826     The contents of the AT.TMout array are:
01827     length,type,subtype,gamma5,factor,sign,gamma's
01828 */
01829
01830 WORD Traces(PHEAD WORD *term, WORD *params, WORD num, WORD level)
01831 {
01832     GETBIDENTITY
01833     switch ( AT.TMout[2] ) { /* Subtype gives dimension */
01834     case 0:
01835         return(TraceN(BHEAD term,params,num,level));
01836     case 4:
01837         return(Trace4(BHEAD term,params,num,level));
01838     case 12:
01839         return(Trace4(BHEAD term,params,num,level));
01840     case 20:
01841         return(Trace4(BHEAD term,params,num,level));
01842     default:
01843         return(0);
01844     }
01845 }
01846
01847
01848 /*
01849     #] Traces :
01850     #[ TraceFind :       WORD TraceFind(term,params)
01851 */
01852
01853 WORD TraceFind(PHEAD WORD *term, WORD *params)
01854 {
01855     GETBIDENTITY
01856     WORD *p, *m, *to;
01857     WORD *termout, *stop, *stop2, number = 0;
01858     WORD first = 1;
01859     WORD type, spinline, sp;
01860     type = params[3];
01861     spinline = params[4];
01862     if ( spinline < 0 ) { /* $ variable. Evaluate */
01863         sp = DolToIndex(BHEAD -spinline);
01864         if ( AN.ErrorInDollar || sp < 0 ) {
01865             MLOCK(ErrorMessageLock);
01866             MesPrint("$%s does not have an index value in trace statement in module %1",
01867                 DOLLARNAME(Dollars,-spinline),AC.CModule);
01868             MUNLOCK(ErrorMessageLock);
01869             return(0);
01870         }
01871         spinline = sp;
01872     }
01873     to = AT.TMout;
01874     to++;
01875     *to++ = TAKETRACE;
01876     *to++ = type;
01877     *to++ = GAMMA1;
01878     *to++ = 0; /* Powers of two */
01879     *to++ = 1; /* sign */
01880     p = term;
01881     m = p + *p - 1;
01882     stop = m - ABS(*m);
01883     termout = m = AT.WorkPointer;

```



```

01884     m++;
01885     p++;
01886     while ( p < stop ) {
01887         stop2 = p + p[1];
01888         if ( *p == GAMMA && p[FUNHEAD] == spinline ) {
01889             if ( first ) {
01890                 *m++ = SUBEXPRESSION;
01891                 *m++ = SUBEXPSIZE;
01892                 *m++ = -1;
01893                 *m++ = 1;
01894                 *m++ = DUMMYBUFFER;
01895                 FILLSUB(m)
01896                 first = 0;
01897             }
01898             p += FUNHEAD+1;
01899             while ( p < stop2 ) {
01900                 if ( *p == GAMMA5 ) {
01901                     if ( AT.TMout[3] == GAMMA5 ) AT.TMout[3] = GAMMA1;
01902                     else if ( AT.TMout[3] == GAMMA1 ) AT.TMout[3] = GAMMA5;
01903                     else if ( AT.TMout[3] == GAMMA7 ) AT.TMout[5] = -AT.TMout[5];
01904                     if ( number & 1 ) AT.TMout[5] = - AT.TMout[5];
01905                     p++;
01906                 }
01907                 else if ( *p == GAMMA6 ) {
01908                     if ( number & 1 ) goto F7;
01909 F6:                     if ( AT.TMout[3] == GAMMA6 ) (AT.TMout[4])++;
01910                     else if ( AT.TMout[3] == GAMMA1 ) AT.TMout[3] = GAMMA6;
01911                     else if ( AT.TMout[3] == GAMMA5 ) AT.TMout[3] = GAMMA6;
01912                     else if ( AT.TMout[3] == GAMMA7 ) AT.TMout[5] = 0;
01913                     p++;
01914                 }
01915                 else if ( *p == GAMMA7 ) {
01916                     if ( number & 1 ) goto F6;
01917 F7:                     if ( AT.TMout[3] == GAMMA7 ) (AT.TMout[4])++;
01918                     else if ( AT.TMout[3] == GAMMA1 ) AT.TMout[3] = GAMMA7;
01919                     else if ( AT.TMout[3] == GAMMA5 ) {
01920                         AT.TMout[3] = GAMMA7;
01921                         AT.TMout[5] = -AT.TMout[5];
01922                     }
01923                     else if ( AT.TMout[3] == GAMMA6 ) AT.TMout[5] = 0;
01924                     p++;
01925                 }
01926                 else {
01927                     *to++ = *p++;
01928                     number++;
01929                 }
01930             }
01931         }
01932         else {
01933             while ( p < stop2 ) *m++ = *p++;
01934         }
01935     }
01936     if ( first ) return(0);
01937     AT.TMout[0] = WORDDIF(to,AT.TMout);
01938     to = term;
01939     to += *to;
01940     while ( p < to ) *m++ = *p++;
01941     *termout = WORDDIF(m,termout);
01942     to = term;
01943     p = termout;
01944     do { *to++ = *p++; } while ( p < m );
01945     AT.WorkPointer = term + *term;
01946     return(1);
01947 }
01948
01949 /*
01950     #] TraceFind :
01951     #[ Chisholm :          WORD Chisholm(term,level,num)
01952
01953     Routines for reorganizing traces.
01954     The command
01955         Chisholm,1;
01956     will collect the gamma matrices in spinline 1 and see whether
01957     they have an index in common with another gamma matrix. If this
01958     is the case the identity
01959         g_(2,mu)*Tr[g_(1,mu)*S(2)] = S(2)+SR(2)
01960     is applied (SR is the reversed string).
01961 */
01962
01963 WORD Chisholm(PHEAD WORD *term, WORD level)
01964 {
01965     GETBIDENTITY
01966     WORD *t, *r, *m, *s, *tt, *rr;
01967     WORD *mat, *matpoint, *termout, *rdo;
01968     CBUF *C = cbuf+AM.rbufnum;
01969     WORD i, j, num = C->lhs[level][2], gam5;
01970     WORD norm = 0, k, *matp;

```

```

01971 /*
01972 #[ Find : Find possible matrices
01973 */
01974 mat = matpoint = AT.WorkPointer;
01975 t = term;
01976 r = t + *t - 1; r -= ABS(*r);
01977 t++;
01978 i = 0;
01979 gam5 = GAMMA1;
01980 while ( t < r ) {
01981     if ( *t == GAMMA && t[FUNHEAD] == num ) {
01982         m = t + t[1];
01983         t += FUNHEAD+1;
01984         while ( t < m ) {
01985             if ( *t >= 0 || *t < MINSPEC ) i++;
01986             else {
01987                 if ( gam5 == GAMMA1 ) gam5 = *t;
01988                 else if ( gam5 == GAMMA5 ) {
01989                     if ( *t == GAMMA5 ) gam5 = GAMMA1;
01990                     else if ( *t != GAMMA1 ) gam5 = *t;
01991                 }
01992             }
01993             *matpoint++ = *t++;
01994         }
01995     }
01996     else t += t[1];
01997 }
01998 if ( ( i & 1 ) != 0 ) return(0); /* odd trace */
01999 /*
02000 #] Find :
02001 #[ Test : Test for contracted index
02002
02003 This code should be modified.
02004
02005 We have to check for all possible matches if C->lhs[level][3] == 1
02006 and the trace contains a gamma5, gamma6 or gamma7.
02007 Then we normalize by the number of possible contractions (norm) and
02008 do all of them. This way the Levi-Civita tensors have a maximum
02009 chance of cancelling each other. This option is activated with
02010 'contract' and 'symmetrize'. Defaults are that they are on, but
02011 they can be switched off with nocontract and nosymmetrize.
02012 */
02013 s = mat;
02014 while ( s < matpoint ) {
02015     /*
02016         if ( *s < AM.OffsetIndex || ( *s < ( AM.OffsetIndex + WILDOFFSET ) &&
02017             indices[*s-AM.OffsetIndex].dimension == 0 ) ) {
02018     */
02019         if ( *s < AM.OffsetIndex || ( *s < ( AM.OffsetIndex + WILDOFFSET ) &&
02020             indices[*s-AM.OffsetIndex].dimension != 4 )
02021             || ( ( AC.lDefDim != 4 ) && ( *s >= ( AM.OffsetIndex + WILDOFFSET ) ) ) ) {
02022             s++; continue;
02023         }
02024         t = term+1;
02025         while ( t < r ) {
02026             if ( *t == GAMMA && t[FUNHEAD] != num ) {
02027                 m = t + t[1];
02028                 t += FUNHEAD+1;
02029                 while ( t < m ) {
02030                     if ( *t == *s ) {
02031                         norm++;
02032                     }
02033                     t++;
02034                 }
02035             }
02036             else t += t[1];
02037         }
02038         s++;
02039     }
02040     if ( norm == 0 ) return(Generator(BHEAD term,level)); /* No Action */
02041     /*
02042     #] Test :
02043     #[ Do : Process the string
02044
02045     tt: The subterm
02046     t: The matrix
02047     s: The matrix in the relevant string
02048
02049     Cycle the string in mat so that s is at the end.
02050     Copy the part till the critical GAMMA.
02051     Copy inside the critical string, copy S, copy tail inside string.
02052     Important to remember where S is so that we can reverse it later.
02053     Add term UnitTrace/2/norm.
02054     Copy rest of term.
02055     Continue execution with S.
02056     Reverse S.
02057     Continue execution with SR.

```

```

02058 */
02059
02060     if ( C->lhs[level][3] == 0 /* || gam5 == GAMMA1 */ ) norm = 1;
02061
02062     matp = matpoint;
02063     for ( k = 0; k < norm; k++ ) {
02064         matpoint = matp;
02065         s = mat;
02066         while ( s < matpoint ) {
02067             /*
02068                 if ( *s < AM.OffsetIndex || ( *s < ( AM.OffsetIndex + WILDOFFSET ) &&
02069                     indices[*s-AM.OffsetIndex].dimension == 0 ) ) {
02070             */
02071
02072                 if ( *s < AM.OffsetIndex || ( *s < ( AM.OffsetIndex + WILDOFFSET ) &&
02073                     indices[*s-AM.OffsetIndex].dimension != 4 ) ) {
02074                     s++; continue;
02075                 }
02076                 t = term+1;
02077                 while ( t < r ) {
02078                     if ( *t == GAMMA && t[FUNHEAD] != num ) {
02079                         tt = t;
02080                         m = t + t[1];
02081                         t += FUNHEAD+1;
02082                         while ( t < m ) {
02083                             if ( *t == *s ) {
02084                                 i = WORDDIF(t,tt);
02085                                 m = mat;
02086                                 while ( m <= s ) *matpoint++ = *m++;
02087                                 t = mat;
02088                                 while ( m < matpoint ) *t++ = *m++;
02089                                 termout = t;
02090                                 m = termout + 1;
02091                                 t = term + 1;
02092                                 while ( t < tt ) {
02093                                     if ( *t != GAMMA || t[FUNHEAD] != num ) {
02094                                         j = t[1];
02095                                         NCOPY(m,t,j);
02096                                     }
02097                                     else t += t[1];
02098                                 }
02099                                 tt += tt[1];
02100                                 rdo = m;
02101                                 j = i;
02102                                 while ( --j >= 0 ) *m++ = *t++;
02103                                 matpoint = m;
02104                                 s = mat;
02105                                 while ( s < termout ) *m++ = *s++;
02106                                 m--;
02107                                 t++;
02108                                 while ( t < tt ) *m++ = *t++;
02109                                 rdo[1] = WORDDIF(m,rdo);
02110
02111                                 *m++ = AC.lUniTrace[0];
02112                                 *m++ = AC.lUniTrace[1];
02113                                 *m++ = AC.lUniTrace[2];
02114                                 *m++ = AC.lUniTrace[3];
02115                                 *m++ = SNUMBER;
02116                                 *m++ = 4;
02117                                 *m++ = 2*norm;
02118                                 *m++ = -1;
02119
02120                                 while ( t < r ) {
02121                                     if ( *t != GAMMA || t[FUNHEAD] != num ) {
02122                                         j = t[1];
02123                                         NCOPY(m,t,j);
02124                                     }
02125                                     else t += t[1];
02126                                 }
02127                                 rr = term + *term;
02128                                 while ( t < rr ) *m++ = *t++;
02129
02130                                 *termout = WORDDIF(m,termout);
02131                                 rr = m;
02132                                 t = termout;
02133                                 j = *termout;
02134                                 NCOPY(m,t,j);
02135                                 AT.WorkPointer = m;
02136                                 if ( Generator(BHEAD t,level) ) goto ChisCall;
02137
02138                                 j = WORDDIF(termout,mat)-1;
02139                                 t = matpoint;
02140                                 m = t + j;
02141                                 AT.WorkPointer = rr;
02142                                 while ( m > t ) {
02143                                     i = *--m; *m = *t; *t++ = i;
02144                                 }

```

```

02145
02146                                     if ( Generator(BHEAD termout,level) ) goto ChisCall;
02147                                     AT.WorkPointer = mat;
02148
02149                                     goto NextK;
02150                                     }
02151                                     t++;
02152                                     }
02153                                     }
02154                                     else t += t[1];
02155                                     }
02156                                     s++;
02157                                     }
02158 NextK;;
02159     }
02160     return(0);
02161 /*
02162     #] Do :
02163 */
02164 ChisCall:
02165     if ( AM.tracebackflag ) {
02166         MLOCK(ErrorMessageLock);
02167         MesCall("Chisholm");
02168         MUNLOCK(ErrorMessageLock);
02169     }
02170     return(-1);
02171 }
02172
02173 /*
02174     #] Chisholm :
02175     #[ TenVecFind :          WORD TenVecFind(term,params)
02176 */
02177
02178 WORD TenVecFind(PHEAD WORD *term, WORD *params)
02179 {
02180     GETBIDENTITY
02181     WORD *t, *w, *m, *tstop;
02182     WORD i, mode, thevector, thetensor, spectator;
02183     thetensor = params[3];
02184     thevector = params[4];
02185     mode = params[5];
02186     if ( thetensor < 0 ) { /* $-expression */
02187         thetensor = DolToTensor(BHEAD -thetensor);
02188         if ( thetensor < FUNCTION ) {
02189             if ( thevector > 0 ) {
02190                 thetensor = DolToTensor(BHEAD thevector);
02191                 if ( thetensor < FUNCTION ) {
02192                     MLOCK(ErrorMessageLock);
02193                     MesPrint("%s should have been a tensor in module %1"
02194                             ,DOLLARNAME(Dollars,params[4]),AC.CModule);
02195                     MUNLOCK(ErrorMessageLock);
02196                     return(-1);
02197                 }
02198                 thevector = DolToVector(BHEAD -params[3]);
02199                 if ( thevector >= 0 ) {
02200                     MLOCK(ErrorMessageLock);
02201                     MesPrint("%s should have been a vector in module %1"
02202                             ,DOLLARNAME(Dollars,-params[3]),AC.CModule);
02203                     MUNLOCK(ErrorMessageLock);
02204                     return(-1);
02205                 }
02206             }
02207             else {
02208                 MLOCK(ErrorMessageLock);
02209                 MesPrint("%s should have been a tensor in module %1"
02210                         ,DOLLARNAME(Dollars,-params[3]),AC.CModule);
02211                 MUNLOCK(ErrorMessageLock);
02212                 return(-1);
02213             }
02214         }
02215     }
02216     if ( thevector > 0 ) { /* $-expression */
02217         thevector = DolToVector(BHEAD thevector);
02218         if ( thevector >= 0 ) {
02219             MLOCK(ErrorMessageLock);
02220             MesPrint("%s should have been a vector in module %1"
02221                     ,DOLLARNAME(Dollars,params[4]),AC.CModule);
02222             MUNLOCK(ErrorMessageLock);
02223             return(-1);
02224         }
02225     }
02226     if ( ( mode & 1 ) != 0 ) { /* Vector to tensor */
02227         GETSTOP(term,tstop);
02228         t = term + 1;
02229         while ( t < tstop ) {
02230             if ( *t == DOTPRODUCT ) {
02231                 i = t[1] - 2; t += 2;

```

```

02232         while ( i > 0 ) {
02233             spectator = 0;
02234             if ( t[2] < 0 ) {}
02235             else if ( *t == thevector && t[1] == thevector ) {
02236                 if ( ( mode & 2 ) == 0 ) spectator = thevector;
02237             }
02238             else if ( *t == thevector ) spectator = t[1];
02239             else if ( t[1] == thevector ) spectator = *t;
02240             if ( spectator ) {
02241                 if ( ( mode & 8 ) == 0 ) goto match;
02242                 w = SetElements + Sets[params[6]].first;
02243                 m = SetElements + Sets[params[6]].last;
02244                 while ( w < m ) {
02245                     if ( *w == spectator ) break;
02246                     w++;
02247                 }
02248                 if ( w >= m ) goto match;
02249             }
02250             t += 3;
02251             i -= 3;
02252         }
02253     }
02254     else if ( *t == VECTOR ) {
02255         i = t[1] - 2; t += 2;
02256         while ( i > 0 ) {
02257             if ( *t == thevector ) goto match;
02258             t += 2;
02259             i -= 2;
02260         }
02261     }
02262     else if ( *t == thetensor ) t += t[1];
02263     else if ( *t >= FUNCTION ) {
02264         if ( functions[*t-FUNCTION].spec > 0 ) {
02265             w = t + t[1];
02266             t += FUNHEAD;
02267             while ( t < w ) {
02268                 if ( *t == thevector ) goto match;
02269                 t++;
02270             }
02271         }
02272         else if ( ( mode & 4 ) != 0 ) {
02273             w = t + t[1];
02274             t += FUNHEAD;
02275             while ( t < w ) {
02276                 if ( *t == -VECTOR && t[1] == thevector ) goto match;
02277                 else if ( *t > 0 ) t += *t;
02278                 else if ( *t <= -FUNCTION ) t++;
02279                 else t += 2;
02280             }
02281         }
02282         else t += t[1];
02283     }
02284     else t += t[1];
02285 }
02286 }
02287 else { /* Tensor to Vector */
02288     GETSTOP(term,tstop);
02289     t = term+1;
02290     while ( t < tstop ) {
02291         if ( *t == thetensor ) goto match;
02292         t += t[1];
02293     }
02294 }
02295 return(0);
02296 match:
02297     AT.TMout[0] = 5;
02298     AT.TMout[1] = TENVEC;
02299     AT.TMout[2] = thetensor;
02300     AT.TMout[3] = thevector;
02301     AT.TMout[4] = mode;
02302     if ( ( mode & 8 ) != 0 ) { AT.TMout[0] = 6; AT.TMout[5] = params[6]; }
02303     return(1);
02304 }
02305 }
02306
02307 /*
02308     #] TenVecFind :
02309     #[ TenVec :          WORD TenVec(term,params,num,level)
02310 */
02311
02312 WORD TenVec(PHEAD WORD *term, WORD *params, WORD num, WORD level)
02313 {
02314     GETBIDENTITY
02315     WORD *t, *m, *w, *termout, *tstop, *outlist, *ou, *ww, *mm;
02316     WORD i, j, k, x, mode, thevector, thetensor, DumNow, spectator;
02317     DUMMYUSE(num);
02318     thetensor = params[2];

```

```

02319     thevector = params[3];
02320     mode = params[4];
02321     termout = AT.WorkPointer;
02322     DumNow = AR.CurDum = DetCurDum(BHEAD term);
02323     if ( ( mode & 1 ) != 0 ) { /* Vector to tensor */
02324         AT.WorkPointer += *term;
02325         ou = outlist = AT.WorkPointer;
02326         GETSTOP(term,tstop);
02327         t = term + 1;
02328         m = termout + 1;
02329         while ( t < tstop ) {
02330             if ( *t == DOTPRODUCT ) {
02331                 i = t[1] - 2;
02332                 w = m;
02333                 *m++ = *t++; *m++ = *t++;
02334                 while ( i > 0 ) {
02335                     spectator = 0;
02336                     if ( t[2] < 0 ) {
02337                         *m++ = *t++; *m++ = *t++; *m++ = *t++;
02338                     }
02339                     else if ( *t == thevector && t[1] == thevector ) {
02340                         if ( ( mode & 2 ) == 0 ) spectator = thevector;
02341                         else {
02342                             *m++ = *t++; *m++ = *t++; *m++ = *t++;
02343                         }
02344                     }
02345                     else if ( *t == thevector ) spectator = t[1];
02346                     else if ( t[1] == thevector ) spectator = *t;
02347                     else {
02348                         *m++ = *t++; *m++ = *t++; *m++ = *t++;
02349                     }
02350                     if ( spectator ) {
02351                         if ( ( mode & 8 ) == 0 ) goto noveto;
02352                         ww = SetElements + Sets[params[5]].first;
02353                         mm = SetElements + Sets[params[5]].last;
02354                         while ( ww < mm ) {
02355                             if ( *ww == spectator ) break;
02356                             ww++;
02357                         }
02358                         if ( ww < mm ) {
02359                             *m++ = *t++; *m++ = *t++; *m++ = *t++;
02360                         }
02361                         else {
02362 noveto:
02363                             if ( spectator == thevector ) {
02364                                 for ( j = 0; j < t[2]; j++ ) {
02365                                     *ou++ = ++AR.CurDum;
02366                                     *ou++ = AR.CurDum;
02367                                 }
02368                                 t += 3;
02369                             }
02370                             else {
02371                                 for ( j = 0; j < t[2]; j++ ) *ou++ = spectator;
02372                                 t += 3;
02373                             }
02374                             i -= 3;
02375                         }
02376                         w[1] = WORDDIF(m,w);
02377                         if ( w[1] == 2 ) m = w;
02378                     }
02379                 else if ( *t == VECTOR ) {
02380                     i = t[1] - 2; w = m;
02381                     *m++ = *t++; *m++ = *t++;
02382                     while ( i > 0 ) {
02383                         if ( *t == thevector ) {
02384                             *ou++ = t[1];
02385                             t += 2;
02386                         }
02387                         else { *m++ = *t++; *m++ = *t++; }
02388                         i -= 2;
02389                     }
02390                     w[1] = WORDDIF(m,w);
02391                     if ( w[1] == 2 ) m = w;
02392                 }
02393             else if ( *t == thetensor ) {
02394                 i = t[1] - FUNHEAD;
02395                 t += FUNHEAD;
02396                 NCOPY(ou,t,i);
02397             }
02398             else if ( *t >= FUNCTION ) {
02399                 if ( functions[*t-FUNCTION].spec > 0 ) {
02400                     w = t + t[1];
02401                     i = FUNHEAD;
02402                     NCOPY(m,t,i);
02403                     while ( t < w ) {
02404                         if ( *t == thevector ) {
02405                             *m++ = ++AR.CurDum;

```

```

02406             *ou++ = AR.CurDum;
02407             t++;
02408         }
02409         else *m++ = *t++;
02410     }
02411 }
02412 else if ( ( mode & 4 ) != 0 ) {
02413     w = t + t[1];
02414     i = FUNHEAD;
02415     NCOPY(m,t,i);
02416     while ( t < w ) {
02417         if ( *t == -VECTOR && t[1] == thevector ) {
02418             *m++ = -INDEX;
02419             *m++ = ++AR.CurDum;
02420             *ou++ = AR.CurDum;
02421             t += 2;
02422         }
02423         else if ( *t > 0 ) {
02424             i = *t;
02425             NCOPY(m,t,i);
02426         }
02427         else if ( *t <= -FUNCTION ) *m++ = *t++;
02428         else { *m++ = *t++; *m++ = *t++; }
02429     }
02430 }
02431     else goto docopy;
02432 }
02433 else {
02434 docopy:
02435     i = t[1];
02436     NCOPY(m,t,i);
02437 }
02438 }
02439 i = WORDDIF(ou,outlist);
02440 if ( i > 0 ) {
02441     for ( j = 1; j < i; j++ ) {
02442         if ( outlist[j-1] > outlist[j] ) {
02443             x = outlist[j-1]; outlist[j-1] = outlist[j]; outlist[j] = x;
02444             for ( k = j-1; k > 0; k-- ) {
02445                 if ( outlist[k-1] <= outlist[k] ) break;
02446                 x = outlist[k-1]; outlist[k-1] = outlist[k]; outlist[k] = x;
02447             }
02448         }
02449     }
02450 }
02451 *m++ = thetensor;
02452 *m++ = FUNHEAD + i;
02453 *m++ = DIRTYSYMFLAG;
02454 FILLFUN3(m)
02455 ou = outlist;
02456 NCOPY(m,ou,i);
02457 }
02458 w = term + *term;
02459 while ( t < w ) *m++ = *t++;
02460 }
02461 else { /* Tensor to Vector */
02462     GETSTOP(term,tstop);
02463     t = term+1;
02464     m = termout+1;
02465     while ( t < tstop ) {
02466         if ( *t != thetensor ) {
02467             i = t[1];
02468             NCOPY(m,t,i);
02469         }
02470         else {
02471             i = t[1] - FUNHEAD;
02472             t += FUNHEAD;
02473             if ( i > 0 ) {
02474                 w = m; m += 2;
02475                 while ( --i >= 0 ) {
02476                     *m++ = thevector;
02477                     *m++ = *t++;
02478                 }
02479                 *w = DELTA;
02480                 w[1] = WORDDIF(m,w);
02481             }
02482         }
02483     }
02484     w = term + *term;
02485     while ( t < w ) *m++ = *t++;
02486 }
02487 *termout = WORDDIF(m,termout);
02488 AT.WorkPointer = m;
02489 *AT.TMout = 0;
02490 if ( Generator(BHEAD termout,level) ) goto fromTenVec;
02491 AR.CurDum = DumNow;
02492 AT.WorkPointer = termout;

```


Functions

- void [print_instr](#) (const vector< WORD > &instr, WORD num)
- template<class RandomAccessIterator >
void [my_random_shuffle](#) (PHEAD RandomAccessIterator fr, RandomAccessIterator to)
- LONG [get_expression](#) (int exprnr)
- vector< vector< WORD > > [get_brackets](#) ()
- int [count_operators](#) (const WORD *expr, bool print=false)
- int [count_operators](#) (const vector< WORD > &instr, bool print=false)
- vector< WORD > [occurrence_order](#) (const WORD *expr, bool rev)
- WORD [getpower](#) (const WORD *term, int var, int pos)
- void [fixarg](#) (UWORD *t, WORD &n)
- void [GcdLong_fix_args](#) (PHEAD UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc)
- void [DivLong_fix_args](#) (UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc, UWORD *d, WORD *nd)
- void [build_Horner_tree](#) (const WORD **terms, int numterms, int var, int maxvar, int pos, vector< WORD > *res)
- bool [term_compare](#) (const WORD *a, const WORD *b)
- vector< WORD > [Horner_tree](#) (const WORD *expr, const vector< WORD > &order)
- void [print_tree](#) (const vector< WORD > &tree)
- template<typename T >
size_t [hash_range](#) (T *array, int size)
- vector< WORD > [generate_instructions](#) (const vector< WORD > &tree, bool do_CSE)
- int [count_operators_cse](#) (const vector< WORD > &tree)
- [NODE](#) * [buildTree](#) (vector< WORD > &tree)
- int [count_operators_cse_topdown](#) (vector< WORD > &tree)
- vector< WORD > [simulated_annealing](#) ()
- void [find_Horner_MCTS_expand_tree](#) ()
- void [find_Horner_MCTS](#) ()
- vector< WORD > [merge_operators](#) (const vector< WORD > &all_instr, bool move_coeff)
- vector< [optimization](#) > [find_optimizations](#) (const vector< WORD > &instr)
- bool [do_optimization](#) (const [optimization](#) optim, vector< WORD > &instr, int newid)
- int [partial_factorize](#) (vector< WORD > &instr, int n, int improve)
- vector< WORD > [optimize_greedy](#) (vector< WORD > instr, LONG time_limit)
- vector< WORD > [recycle_variables](#) (const vector< WORD > &all_instr)
- void [optimize_expression_given_Horner](#) ()
- void [generate_output](#) (const vector< WORD > &instr, int exprnr, int extraoffset, const vector< vector< WORD > > &brackets)
- WORD [generate_expression](#) (WORD exprnr)
- void [optimize_print_code](#) (int print_expr)
- int [Optimize](#) (WORD exprnr, int do_print)
- int [ClearOptimize](#) ()

Variables

- const WORD [OPER_ADD](#) = -1
- const WORD [OPER_MUL](#) = -2
- const WORD [OPER_COMMA](#) = -3
- WORD * [optimize_expr](#)
- vector< vector< WORD > > [optimize_best_Horner_schemes](#)
- int [optimize_num_vars](#)
- int [optimize_best_num_oper](#)
- vector< WORD > [optimize_best_instr](#)
- vector< WORD > [optimize_best_vars](#)

- bool [mcts_factorized](#)
- bool [mcts_separated](#)
- vector< WORD > [mcts_vars](#)
- [tree_node](#) [mcts_root](#)
- int [mcts_expr_score](#)
- set< pair< int, vector< WORD > > > [mcts_best_schemes](#)

4.92.1 Detailed Description

experimental routines for the optimization of FORTRAN or C output.

Definition in file [optimize.cc](#).

4.92.2 Function Documentation

4.92.2.1 `print_instr()`

```
void print_instr (
    const vector< WORD > & instr,
    WORD num )
```

Definition at line 180 of file [optimize.cc](#).

4.92.2.2 `my_random_shuffle()`

```
template<class RandomAccessIterator >
void my_random_shuffle (
    PHEAD RandomAccessIterator fr,
    RandomAccessIterator to )
```

Random shuffle

4.92.3 Description

Randomly permutes elements in the range [fr,to). Functionality is the same as C++'s "random_shuffle", but it uses Form's "wranf".

Definition at line 202 of file [optimize.cc](#).

Referenced by [find_Horner_MCTS\(\)](#).

4.92.3.1 `get_expression()`

```
LONG get_expression (
    int exprnr )
```

Get expression

4.92.4 Description

Reads an expression from the input file into a buffer (called `optimize_expr`). This buffer is used during the optimization process. Non-symbols are removed by `ConvertToPoly` and are put in temporary symbols.

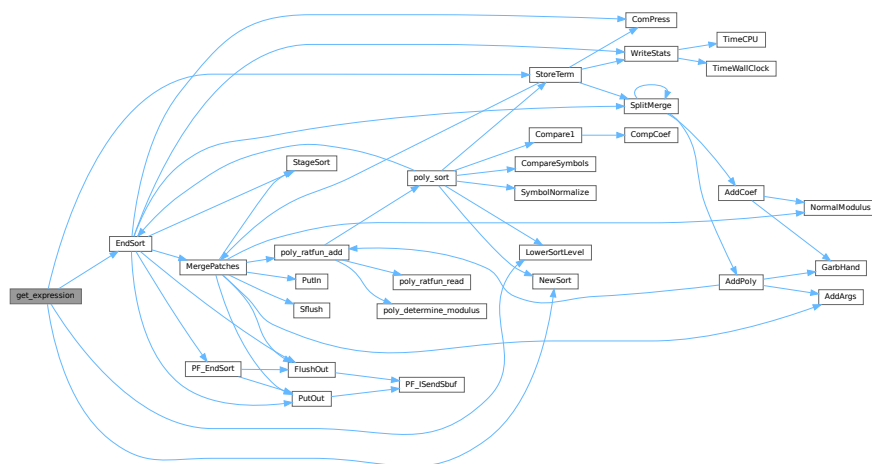
The return value is the length of the expression in WORDs, or a negative number if it failed.

Definition at line 226 of file [optimize.cc](#).

References [EndSort\(\)](#), [LowerSortLevel\(\)](#), [NewSort\(\)](#), and [StoreTerm\(\)](#).

Referenced by [Optimize\(\)](#).

Here is the call graph for this function:



4.92.4.1 get_brackets()

```
vector< vector< WORD > > get_brackets ( )
```

Get brackets

4.92.5 Description

Checks whether the input expression (stored in `optimize_expr`) contains brackets. If so, this method replaces terms outside brackets by powers of SEPERATESYMBOL (equal brackets have equal powers) and the brackets are returned. If not, the result is empty.

Brackets are used for simultaneous optimization. The symbol SEPERATESYMBOL is always the first one used in a Horner scheme.

Definition at line 303 of file [optimize.cc](#).

Referenced by [Optimize\(\)](#).

4.92.5.1 `count_operators()` [1/2]

```
int count_operators (
    const WORD * expr,
    bool print = false )
```

Count operators

4.92.6 Description

Counts the number of operators in a Form-style expression.

Definition at line 423 of file [optimize.cc](#).

Referenced by [find_Horner_MCTS\(\)](#), [generate_instructions\(\)](#), [Optimize\(\)](#), [optimize_expression_given_Horner\(\)](#), and [optimize_greedy\(\)](#).

4.92.6.1 `count_operators()` [2/2]

```
int count_operators (
    const vector< WORD > & instr,
    bool print = false )
```

Count operators

4.92.7 Description

Counts the number of operators in a vector of instructions

Definition at line 477 of file [optimize.cc](#).

4.92.7.1 `occurrence_order()`

```
vector< WORD > occurrence_order (
    const WORD * expr,
    bool rev )
```

Occurrence order

4.92.8 Description

Extracts all variables from an expression and sorts them with most occurring first (or last, with `rev=true`)

Definition at line 520 of file [optimize.cc](#).

Referenced by [Optimize\(\)](#).

4.92.8.1 getpower()

```
WORD getpower (
    const WORD * term,
    int var,
    int pos )
```

Horner tree building

4.92.9 Description

Given a Form-style expression (in a buffer in memory), this builds an expression tree. The tree is determined by a multivariate Horner scheme, i.e., something of the form:

$$1+y+x*(2+y*(1+y)+x*(3-y*(...)))$$

The order of the variables is given to the method "Horner_tree", which rennumbers and reorders the terms of the expression. Next, the recursive method "build_Horner_tree" does the actual tree construction.

The tree is represented in postfix notation. Tokens are of the following forms:

- SNUMBER tokenlength num den coefflength
- SYMBOL tokenlength variable power
- OPER_ADD or OPER_MUL

4.92.10 Note

Sets AN.poly_num_vars and allocates AN.poly_vars. The latter should be freed later. Get power of variable (helper function for build_Horner_tree)

4.92.11 Description

Returns the power of the variable "var", which is at position "pos" in this term, if it is present.

Definition at line 601 of file [optimize.cc](#).

Referenced by [build_Horner_tree\(\)](#).

4.92.11.1 fixarg()

```
void fixarg (
    UWORD * t,
    WORD & n )
```

Call GcdLong/DivLong with leading zeroes

4.92.12 Description

These method remove leading zeroes, so that GcdLong and DivLong can safely be called.

Definition at line 615 of file [optimize.cc](#).

4.92.12.1 GcdLong_fix_args()

```
void GcdLong_fix_args (
    PHEAD UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc )
```

Definition at line 621 of file [optimize.cc](#).

4.92.12.2 DivLong_fix_args()

```
void DivLong_fix_args (
    UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc,
    UWORD * d,
    WORD * nd )
```

Definition at line 627 of file [optimize.cc](#).

4.92.12.3 build_Horner_tree()

```
void build_Horner_tree (
    const WORD ** terms,
    int numterms,
    int var,
    int maxvar,
    int pos,
    vector< WORD > * res )
```

Build the Horner tree

4.92.13 Description

Constructs the Horner tree. The method processes one variable and continues recursively until the Horner scheme is completed.

"terms" is a pointer to the starts of the terms. "numterms" is the number of terms to be processed. "var" is the next variable to be processed (index between 0 and #maxvar) and "maxvar" is the last variable to be processed, so that partial Horner trees can also be constructed. "pos" is the position that the power of "var" should be in (one level further in the recursion, "pos" has increased by 0 or 1 depending on whether the previous power was 0 or not). The result is written at the pointer "res".

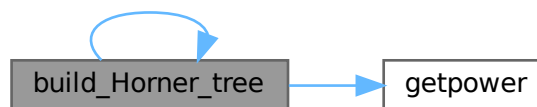
This method also factors out gcds of the coefficients. The result should end with "gcd OPER_MUL" at all times, so that one level higher gcds can be extracted again.

Definition at line 653 of file [optimize.cc](#).

References [build_Horner_tree\(\)](#), and [getpower\(\)](#).

Referenced by [build_Horner_tree\(\)](#), and [Horner_tree\(\)](#).

Here is the call graph for this function:



4.92.13.1 term_compare()

```
bool term_compare (  
    const WORD * a,  
    const WORD * b )
```

Term compare (helper function for `Horner_tree`)

4.92.14 Description

Compares two terms of the form "L SYM 4 x n coeff" or "L coeff". Lower powers of lower-indexed symbols come first. This is used to sort the terms in correct order.

Definition at line 853 of file [optimize.cc](#).

Referenced by [Horner_tree\(\)](#).

4.92.14.1 Horner_tree()

```
vector< WORD > Horner_tree (
    const WORD * expr,
    const vector< WORD > & order )
```

Prepare Horner tree building

4.92.15 Description

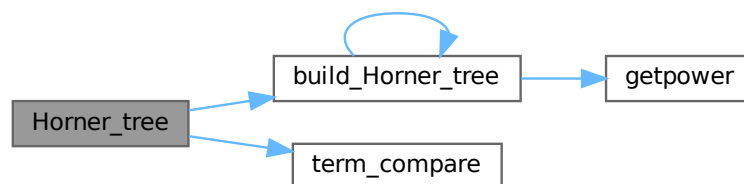
This method rennumbers the variables to 0...#vars-1 according to the specified order. Next, it stored pointer to individual terms and sorts the terms with higher power first. Then the sorted lists of power is used for the construction of the Horner tree.

Definition at line 874 of file [optimize.cc](#).

References [build_Horner_tree\(\)](#), and [term_compare\(\)](#).

Referenced by [optimize_expression_given_Horner\(\)](#).

Here is the call graph for this function:



4.92.15.1 print_tree()

```
void print_tree (
    const vector< WORD > & tree )
```

Definition at line 1002 of file [optimize.cc](#).

4.92.15.2 hash_range()

```
template<typename T >
size_t hash_range (
    T * array,
    int size )
```

Definition at line 1071 of file [optimize.cc](#).

4.92.15.3 generate_instructions()

```
vector< WORD > generate_instructions (
    const vector< WORD > & tree,
    bool do_CSE )
```

Generate instructions

4.92.16 Description

Converts the expression tree to a list of instructions that directly translate to code. Instructions are of the form:

expr.nr operator length [operands]+ trailing.zero

The operands are of the form:

length [(EXTRA)SYMBOL length variable power] coeff

This method only generates binary operators. Merging is done later. The method also checks for common subexpressions and eliminates them and the flag "do_CSE" is set.

4.92.17 Implementation details

The map "ID" keeps track of which subexpressions already exist. The key is formatted as one of the following:

SYMBOL x n SNUMBER coeff OPERATOR LHS RHS

with LHS/RHS formatted as one of the following:

SNUMBER idx 0 (EXTRA)SYMBOL x n

ID[symbol] or ID[operator] equals a subexpression number. ID[coeff] equals the position of the number in the input.

The stack s is used to process the postfix expression tree. Three-word tokens of the form:

SNUMBER idx.of.coeff 0 SYMBOL x n EXTRASYMBOL x 1

are pushed onto it. Operators pop two operands and push the resulting expression.

(Extra)symbols are 1-indexed, because -X is also needed to represent -1 times this term.

There is currently a bug. The notation cannot tell if there is a single bracket and then ignores the bracket.

TODO: check if this method performs properly if do_CSE=false

Definition at line 1133 of file [optimize.cc](#).

References [count_operators\(\)](#).

Referenced by [optimize_expression_given_Horner\(\)](#).

Here is the call graph for this function:



4.92.17.1 count_operators_cse()

```
int count_operators_cse (
    const vector< WORD > & tree )
```

Count number of operators in a binary tree, while removing CSEs on the fly. The instruction set is not created, which makes this method slightly faster.

A hash is created on the fly and is passed through the stack. TODO: find better hash functions

Definition at line 1342 of file [optimize.cc](#).

4.92.17.2 buildTree()

```
NODE * buildTree (
    vector< WORD > & tree )
```

Definition at line 1564 of file [optimize.cc](#).

4.92.17.3 count_operators_cse_topdown()

```
int count_operators_cse_topdown (
    vector< WORD > & tree )
```

Definition at line 1626 of file [optimize.cc](#).

4.92.17.4 simulated_annealing()

```
vector< WORD > simulated_annealing ( )
```

Definition at line 1681 of file [optimize.cc](#).

4.92.17.5 find_Horner_MCTS_expand_tree()

```
void find_Horner_MCTS_expand_tree ( )
```

Definition at line 2054 of file [optimize.cc](#).

4.92.17.6 find_Horner_MCTS()

```
void find_Horner_MCTS ( )
```

Find best Horner schemes using MCTS

4.92.18 Description

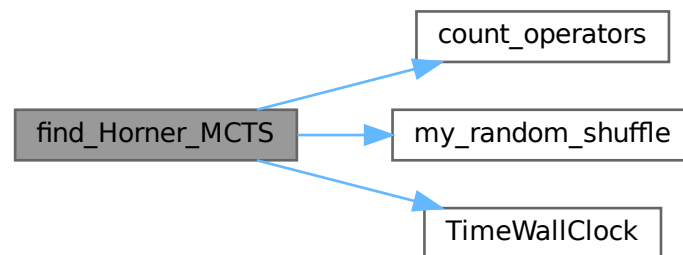
The method governs the MCTS for the best Horner schemes. It does some pre-processing, calls "find_Horner_MCTS_expand_tree" a number of times and does some post-processing.

Definition at line 2255 of file [optimize.cc](#).

References [count_operators\(\)](#), [my_random_shuffle\(\)](#), and [TimeWallClock\(\)](#).

Referenced by [Optimize\(\)](#).

Here is the call graph for this function:



4.92.18.1 merge_operators()

```
vector< WORD > merge_operators (
    const vector< WORD > & all_instr,
    bool move_coeff )
```

Merge operators

4.92.19 Description

The input instructions form a binary DAG. This method merges expressions like

$Z1 = a+b; Z2 = Z1+c;$

into

$Z2 = a+b+c;$

An instruction is merged iff it only has one parent and the operator equals its parent's operator.

This still leaves some freedom: where should the coefficients end up in cases as:

$Z1 = Z2 + x \Leftrightarrow Z1 = 2*Z2 + x \quad Z2 = 2*x*y \quad Z2 = x*y$

Both are relevant, e.g. for CSE of the form "2*x" and "2*Z2". The flag "move_coeff" moves coefficients from LHS-like expressions to RHS-like expressions.

Furthermore, this method removes empty equation ($Z1=0$), that are introduced by some "optimize_greedy" substitutions.

4.92.20 Implementation details

Expressions are mostly traversed via a stack, so that parents are evaluated before their children.

With "move_coeff" set coefficients are moved, but this leads to some tricky cases, e.g.

$$Z1 = Z2 + x \quad Z2 = 2*y$$

Here Z2 reduces to the trivial equation $Z2=y$, which should be eliminated. Here the array skip[i] comes in.

Furthermore in the case

$$Z1 = Z2 + x \quad Z2 = 2*Z3 \quad Z3 = x*Z4 \quad Z4 = y*z$$

after substituting $Z1 = 2*Z3 + x$, the parent expression for Z4 becomes Z3 instead of Z2. This is where renum_par[i] comes in.

Finally, once a coefficient has been moved, skip_coeff[i] is set and this coefficient is copied into the new expression anymore.

Definition at line 2403 of file [optimize.cc](#).

Referenced by [optimize_expression_given_Horner\(\)](#).

4.92.20.1 find_optimizations()

```
vector< optimization > find_optimizations (
    const vector< WORD > & instr )
```

Find optimizations

4.92.21 Description

This method find all optimization of the form described in "class Optimization". It process every equation, looking for possible optimizations and stores them in a fast-access data structure to count the total improvement of an optimization.

Definition at line 2698 of file [optimize.cc](#).

Referenced by [optimize_greedy\(\)](#).

4.92.21.1 do_optimization()

```
bool do_optimization (
    const optimization optim,
    vector< WORD > & instr,
    int newid )
```

Do optimization

4.92.22 Description

This method performs an optimization. It scans through the equations of "optim.eqnidxs" and looks in which this optimization can still be performed (due to other performed optimizations this isn't always the case). If possible, it substitutes the common subexpression by a new extra symbol numbered "newid". Finally, the new extrasymbol is defined accordingly.

Substitutions may lead to trivial equations of the form " $Z_i=Z_j$ ", but these are removed in the end of the method. The method returns whether the substitution has been done once or more (or not).

Definition at line 2931 of file [optimize.cc](#).

Referenced by [optimize_greedy\(\)](#).

4.92.22.1 partial_factorize()

```
int partial_factorize (
    vector< WORD > & instr,
    int n,
    int improve )
```

Partial factorization of instructions

4.92.23 Description

This method performs partial factorization of instructions. In particular the following instructions

$Z_1 = x*a*b$ $Z_2 = x*c*d*e$ $Z_3 = 2*x + Z_1 + Z_2 + \text{more}$

are replaced by

$Z_1 = a*b$ $Z_2 = c*d*e$ $Z_3 = Z_j + \text{more}$ $Z_i = 2 + Z_1 + Z_2$ $Z_j = x*Z_i$

Here it is necessary that no other equations refer to Z_1 and Z_2 . The generation of trivial instructions ($Z_i=Z_j$ or $Z_i=x$) is prevented.

Definition at line 3480 of file [optimize.cc](#).

Referenced by [optimize_greedy\(\)](#).

4.92.23.1 optimize_greedy()

```
vector< WORD > optimize_greedy (
    vector< WORD > instr,
    LONG time_limit )
```

Optimize instructions greedily

4.92.24 Description

This method optimizes an expression greedily. It calls "find_optimizations" to obtain candidates and performs the best one(s) by calling "do_optimization".

How many different optimization are done, before "find_optimization" is called again, is determined by the settings "greedyminnum" and "greedymaxperc".

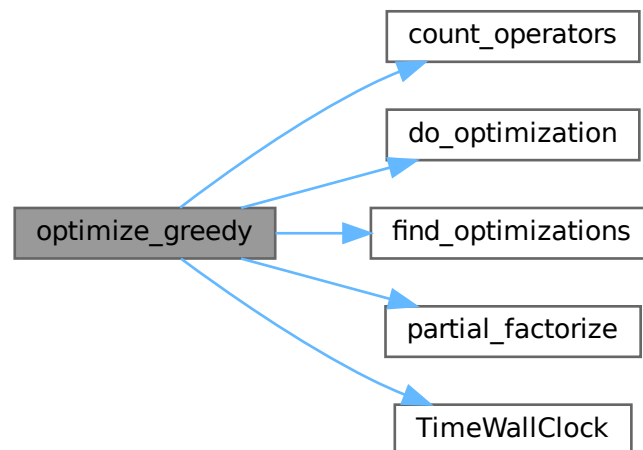
During the optimization process, sequences of zeroes are introduced in the instructions, since moving all instructions when one gets optimized, is very costly. Therefore, in the end, the instructions are "compressed" again to remove these extra zeroes.

Definition at line 3774 of file [optimize.cc](#).

References [count_operators\(\)](#), [do_optimization\(\)](#), [find_optimizations\(\)](#), [partial_factorize\(\)](#), and [TimeWallClock\(\)](#).

Referenced by [optimize_expression_given_Horner\(\)](#).

Here is the call graph for this function:



4.92.24.1 recycle_variables()

```
vector< WORD > recycle_variables (
    const vector< WORD > & all_instr )
```

Recycle variables

4.92.25 Description

The current input uses many temporary variables. Many of them become obsolete at some point during the evaluation of the code, so can be recycled. This method rennumbers the temporary variables, so that they are recycled. Furthermore, the input is order in depth-first order, so that the instructions can be performed consecutively.

4.92.26 Implementation details

First, for each subDAG, an estimate for the number of variables needed is made. This is done by the following recursive formula:

$$\#vars(x) = \max(\#vars(ch_i(x)) + i),$$

with $ch_i(x)$ the i -th child of x , where the childs are ordered w.r.t. $\#vars(ch_i)$. This formula is exact if the input forms a tree, and otherwise gives a reasonable estimate.

Then, the instructions are reordered in a depth-first order with childs ordered w.r.t. $\#vars$. Next, the times that variables become obsolete are found. Each LHS of an instruction is renumbered to the lowest-numbered temporary variable that is available at that time.

Definition at line 3914 of file [optimize.cc](#).

Referenced by [optimize_expression_given_Horner\(\)](#).

4.92.26.1 optimize_expression_given_Horner()

```
void optimize_expression_given_Horner ( )
```

Optimize expression given a Horner scheme

4.92.27 Description

This method picks one Horner scheme from the list of best Horner schemes, applies this scheme to the expression and then, depending on `optimize.settings`, does a common subexpression elimination (CSE) or performs greedy optimizations.

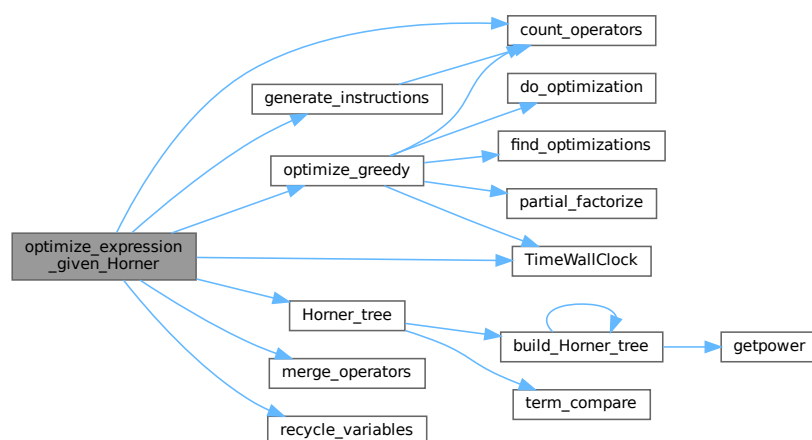
CSE is fast, while greedy might be slow. CSE followed by greedy is faster than greedy alone, but typically results in slightly worse code (not proven; just observed). eventually do greedy optimizations

Definition at line 4061 of file [optimize.cc](#).

References [count_operators\(\)](#), [generate_instructions\(\)](#), [Horner_tree\(\)](#), [merge_operators\(\)](#), [optimize_greedy\(\)](#), [recycle_variables\(\)](#), and [TimeWallClock\(\)](#).

Referenced by [Optimize\(\)](#).

Here is the call graph for this function:



4.92.27.1 generate_output()

```
void generate_output (
    const vector< WORD > & instr,
    int exprnr,
    int extraoffset,
    const vector< vector< WORD > > & brackets )
```

Generate output

4.92.28 Description

This method prepares the instructions for printing. They are stored in Form format, so that they can be printed by "PrintExtraSymbol". The results are stored in the buffer AO.OptimizeResult.

Definition at line [4292](#) of file [optimize.cc](#).

Referenced by [Optimize\(\)](#).

4.92.28.1 generate_expression()

```
WORD generate_expression (
    WORD exprnr )
```

Generate expression

4.92.29 Description

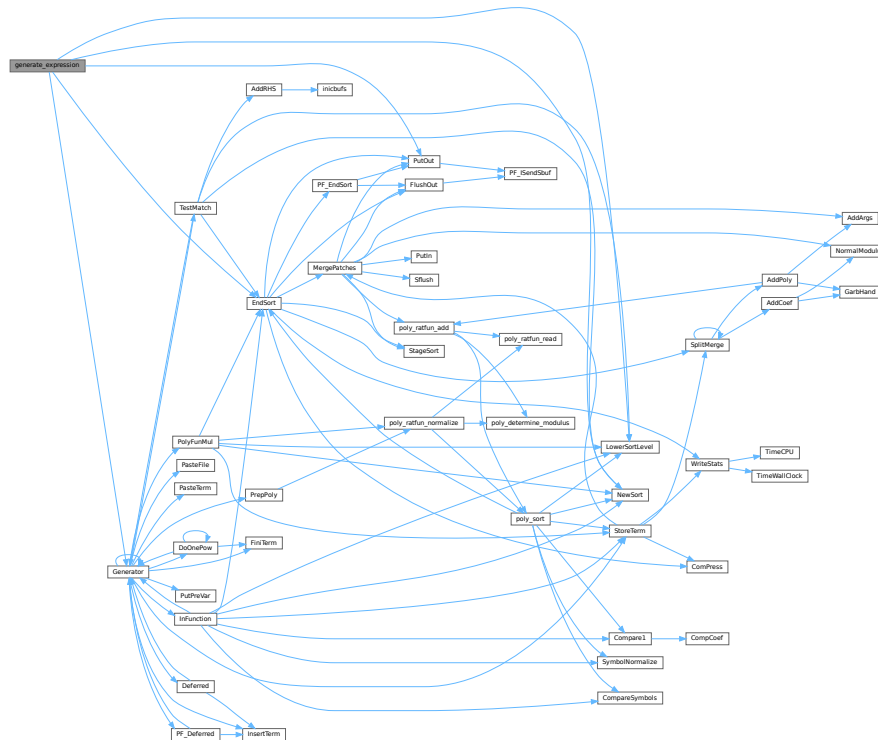
This method modifies the original optimized expression by an expression with extra symbols. This is used for "#Optimize".

Definition at line [4441](#) of file [optimize.cc](#).

References [EndSort\(\)](#), [Generator\(\)](#), [LowerSortLevel\(\)](#), [NewSort\(\)](#), and [PutOut\(\)](#).

Referenced by [Optimize\(\)](#).

Here is the call graph for this function:



4.92.29.1 optimize_print_code()

```
void optimize_print_code (
    int print_expr )
```

Print optimized code

4.92.30 Description

This method prints the optimized code via "PrintExtraSymbol". Depending on the flag, the original expression is printed (for "Print") or not (for "#Optimize / #write "O").

Definition at line 4525 of file [optimize.cc](#).

Referenced by [Optimize\(\)](#).

4.92.30.1 Optimize()

```
int Optimize (
    WORD exprnr,
    int do_print )
```

Optimization of expression

4.92.31 Description

This method takes an input expression and generates optimized code to calculate its value. The following methods are called to do so:

- (1) `get_expression` : to read to expression
- (2) `get_brackets` : find brackets for simultaneous optimization
- (3) `occurrence_order` or `find_Horner_MCTS` : to determine (the) Horner scheme(s) to use; this depends on `AO.optimize.horner`
- (4) `optimize_expression_given_Horner` : to do the optimizations for each Horner scheme; this method does either CSE or greedy optimizations dependings on `AO.optimize.method`
- (5) `generate_output` : to format the output in Form notation and store it in a buffer
- (6a) `optimize_print_code` : to print the expression (for "Print") or (6b) `generate_expression` : to modify the expression (for "#Optimize")

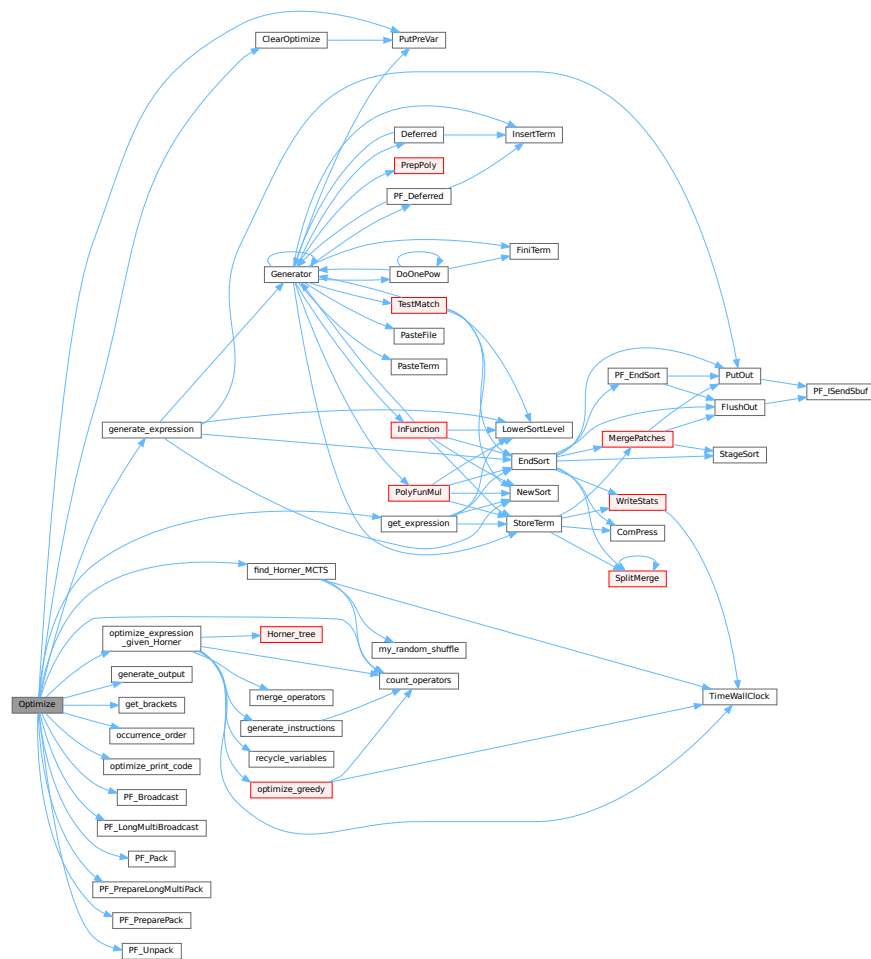
On ParFORM, all the processes must call this function at the same time. Then

- (1) Because only the master can access to the expression to be optimized, the master broadcast the expression to all the slaves after reading the expression (`PF_get_expression`).
- (2) `get_brackets` reads `optimize_expr` as the input and it works also on the slaves. We leave it although the bracket information is not needed on the slaves (used in (5) on the master).
- (3) and (4) `find_Horner_MCTS` and `optimize_expression_given_Horner` are parallelized.
- (5), (6a) and (6b) are needed only on the master.

Definition at line 4638 of file [optimize.cc](#).

References [ClearOptimize\(\)](#), [count_operators\(\)](#), [find_Horner_MCTS\(\)](#), [generate_expression\(\)](#), [generate_output\(\)](#), [get_brackets\(\)](#), [get_expression\(\)](#), [occurrence_order\(\)](#), [optimize_expression_given_Horner\(\)](#), [optimize_print_code\(\)](#), [PF_Broadcast\(\)](#), [PF_LongMultiBroadcast\(\)](#), [PF_Pack\(\)](#), [PF_PrepareLongMultiPack\(\)](#), [PF_PreparePack\(\)](#), [PF_Unpack\(\)](#), and [PutPreVar\(\)](#).

Here is the call graph for this function:



4.92.31.1 ClearOptimize()

```
int ClearOptimize (
    void )
```

Optimization of expression

4.92.32 Description

Clears the buffers that were used for optimization output. Clears the expression from the buffers (marks it to be dropped). Note: we need to use the expression by its name, because numbers may change if we drop other expressions between when we do the optimizations and clear the results (in [execute.c](#)). Also this is not 100% safe, because we could overwrite the optimized expression. But that can be done only in a Local or Global statement and hence we only have to test there that we might have to call ClearOptimize first. (in file [comexpr.c](#))

Definition at line 4975 of file [optimize.cc](#).

References [PutPreVar\(\)](#).

Referenced by [Optimize\(\)](#).

Here is the call graph for this function:



4.92.33 Variable Documentation

4.92.33.1 OPER_ADD

```
const WORD OPER_ADD = -1
```

Definition at line [120](#) of file [optimize.cc](#).

4.92.33.2 OPER_MUL

```
const WORD OPER_MUL = -2
```

Definition at line [121](#) of file [optimize.cc](#).

4.92.33.3 OPER_COMMA

```
const WORD OPER_COMMA = -3
```

Definition at line [122](#) of file [optimize.cc](#).

4.92.33.4 optimize_expr

```
WORD* optimize_expr
```

Definition at line [157](#) of file [optimize.cc](#).

4.92.33.5 optimize_best_Horner_schemes

```
vector<vector<WORD>> > optimize_best_Horner_schemes
```

Definition at line [158](#) of file [optimize.cc](#).

4.92.33.6 optimize_num_vars

```
int optimize_num_vars
```

Definition at line 159 of file [optimize.cc](#).

4.92.33.7 optimize_best_num_oper

```
int optimize_best_num_oper
```

Definition at line 160 of file [optimize.cc](#).

4.92.33.8 optimize_best_instr

```
vector<WORD> optimize_best_instr
```

Definition at line 161 of file [optimize.cc](#).

4.92.33.9 optimize_best_vars

```
vector<WORD> optimize_best_vars
```

Definition at line 162 of file [optimize.cc](#).

4.92.33.10 mcts_factorized

```
bool mcts_factorized
```

Definition at line 165 of file [optimize.cc](#).

4.92.33.11 mcts_separated

```
bool mcts_separated
```

Definition at line 165 of file [optimize.cc](#).

4.92.33.12 mcts_vars

```
vector<WORD> mcts_vars
```

Definition at line 166 of file [optimize.cc](#).

4.92.33.13 mcts_root

```
tree\_node mcts_root
```

Definition at line 167 of file [optimize.cc](#).

4.92.33.14 mcts_expr_score

```
int mcts_expr_score
```

Definition at line 168 of file [optimize.cc](#).

4.92.33.15 mcts_best_schemes

```
set<pair<int,vector<WORD>> > > mcts_best_schemes
```

Definition at line 169 of file [optimize.cc](#).

4.93 optimize.cc

[Go to the documentation of this file.](#)

```
00001
00006 /*  #[ License : */
00007 /*
00008  *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00009  *   When using this file you are requested to refer to the publication
00010  *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00011  *   This is considered a matter of courtesy as the development was paid
00012  *   for by FOM the Dutch physics granting agency and we would like to
00013  *   be able to track its scientific use to convince FOM of its value
00014  *   for the community.
00015  *
00016  *   This file is part of FORM.
00017  *
00018  *   FORM is free software: you can redistribute it and/or modify it under the
00019  *   terms of the GNU General Public License as published by the Free Software
00020  *   Foundation, either version 3 of the License, or (at your option) any later
00021  *   version.
00022  *
00023  *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00024  *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00025  *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00026  *   details.
00027  *
00028  *   You should have received a copy of the GNU General Public License along
00029  *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00030  */
00031 /*
00032  *   #[ License :
00033  *   #[ includes :
00034  */
00035
00036 // #define DEBUG
00037 // #define DEBUG_MORE
00038 // #define DEBUG_MCTS
00039 // #define DEBUG_GREEDY
00040
00041 #ifdef HAVE_CONFIG_H
00042 #ifndef CONFIG_H_INCLUDED
00043 #define CONFIG_H_INCLUDED
00044 #include <config.h>
00045 #endif
00046 #endif
00047
00048 #include <vector>
00049 #include <stack>
00050 #include <algorithm>
00051 #include <set>
00052 #include <map>
00053 #include <climits>
00054 #include <cmath>
00055 #include <string>
00056 #include <algorithm>
00057 #include <iostream>
00058
00059 #ifdef APPLE64
00060 #ifndef HAVE_UNORDERED_MAP
00061 #define HAVE_UNORDERED_MAP
00062 #endif
```

```

00063 #ifndef HAVE_UNORDERED_SET
00064 #define HAVE_UNORDERED_SET
00065 #endif
00066 #endif
00067
00068 #ifdef HAVE_UNORDERED_MAP
00069     #include <unordered_map>
00070     using std::unordered_map;
00071 #elif !defined(HAVE_TR1_UNORDERED_MAP) && defined(HAVE_BOOST_UNORDERED_MAP_HPP)
00072     #include <boost/unordered_map.hpp>
00073     using boost::unordered_map;
00074 #else
00075     #include <tr1/unordered_map>
00076     using std::tr1::unordered_map;
00077 #endif
00078 #ifdef HAVE_UNORDERED_SET
00079     #include <unordered_set>
00080     using std::unordered_set;
00081 #elif !defined(HAVE_TR1_UNORDERED_SET) && defined(HAVE_BOOST_UNORDERED_SET_HPP)
00082     #include <boost/unordered_set.hpp>
00083     using boost::unordered_set;
00084 #else
00085     #include <tr1/unordered_set>
00086     using std::tr1::unordered_set;
00087 #endif
00088
00089 #if defined(HAVE_BUILTIN_POPCOUNT)
00090     static inline int popcount(unsigned int x) {
00091         return __builtin_popcount(x);
00092     }
00093 #elif defined(HAVE_POPCNT)
00094     #include <intrin.h>
00095     static inline int popcount(unsigned int x) {
00096         return __popcnt(x);
00097     }
00098 #else
00099     static inline int popcount(unsigned int x) {
00100         int count = 0;
00101         while (x > 0) {
00102             if ((x & 1) == 1) count++;
00103             x >>= 1;
00104         }
00105         return count;
00106     }
00107 #endif
00108
00109 extern "C" {
00110     #include "form3.h"
00111 }
00112
00113 // #ifdef DEBUG
00114 #include "mytime.h"
00115 // #endif
00116
00117 using namespace std;
00118
00119 // operators
00120 const WORD OPER_ADD = -1;
00121 const WORD OPER_MUL = -2;
00122 const WORD OPER_COMMA = -3;
00123
00124 // class for a node of the MCTS tree
00125 class tree_node {
00126 public:
00127     vector<tree_node> childs;
00128     double sum_results;
00129     int num_visits;
00130     WORD var;
00131     bool finished;
00132     PADPOINTER(1,1,1,1);
00133
00134     tree_node(int _var=0):
00135         sum_results(0), num_visits(0), var(_var), finished(false) {}
00136
00137     tree_node(const tree_node& other):
00138         childs(other.childs),
00139         sum_results(other.sum_results),
00140         num_visits(other.num_visits),
00141         var(other.var),
00142         finished(other.finished) {}
00143
00144     tree_node& operator=(const tree_node& other) {
00145         if (this != &other) {
00146             childs = other.childs;
00147             sum_results = other.sum_results;
00148             num_visits = other.num_visits;
00149             var = other.var;

```

```

00150         finished = other.finished;
00151     }
00152     return *this;
00153 }
00154 };
00155
00156 // global variables for multithreading
00157 WORD *optimize_expr;
00158 vector<vector<WORD> > optimize_best_Horner_schemes;
00159 int optimize_num_vars;
00160 int optimize_best_num_oper;
00161 vector<WORD> optimize_best_instr;
00162 vector<WORD> optimize_best_vars;
00163
00164 // global variables for MCTS
00165 bool mcts_factorized, mcts_separated;
00166 vector<WORD> mcts_vars;
00167 tree_node mcts_root;
00168 int mcts_expr_score;
00169 set<pair<int, vector<WORD> > > mcts_best_schemes;
00170
00171 #ifdef WITHPTHREADS
00172 pthread_mutex_t optimize_lock;
00173 #endif
00174
00175 /*
00176     #] includes :
00177     #[ print_instr :
00178 */
00179
00180 void print_instr (const vector<WORD> &instr, WORD num)
00181 {
00182     const WORD *tbegin = &instr.begin();
00183     const WORD *tend = tbegin+instr.size();
00184     for (const WORD *t=tbegin; t!=tend; t+=*(t+2)) {
00185         MesPrint("< %d> %a", num, t[2], t);
00186     }
00187 }
00188
00189 /*
00190     #] print_instr :
00191     #[ my_random_shuffle :
00192 */
00193
00194 template <class RandomAccessIterator>
00195 void my_random_shuffle (PHEAD RandomAccessIterator fr, RandomAccessIterator to) {
00196     for (int i=to-fr-1; i>0; --i)
00197         swap (fr[i], fr[wrnf(BHEAD0) % (i+1)]);
00198 }
00199
00200 /*
00201     #] my_random_shuffle :
00202     #[ get_expression :
00203 */
00204
00205 static WORD comlist[3] = {TYPETOPOLYNOMIAL, 3, DOALL};
00206
00207 LONG get_expression (int exprnr) {
00208     GETIDENTITY;
00209
00210     AR.NoCompress = 1;
00211
00212     NewSort(BHEAD0);
00213     EXPRESSIONS e = Expressions+exprnr;
00214     SetScratch(AR.infile, &(e->onfile));
00215
00216     // get header term
00217     WORD *term = AT.WorkPointer;
00218     GetTerm(BHEAD term);
00219
00220     NewSort(BHEAD0);
00221
00222     // get terms
00223     while (GetTerm(BHEAD term) > 0) {
00224         AT.WorkPointer = term + *term;
00225         WORD *t1 = term;
00226         WORD *t2 = term + *term;
00227         if (ConvertToPoly(BHEAD t1, t2, comlist, 1) < 0) return -1;
00228         int n = *t2;
00229         NCOPY(t1, t2, n);
00230         AT.WorkPointer = term + *term;
00231         if (StoreTerm(BHEAD term)) return -1;
00232     }
00233
00234     // sort and store in buffer
00235     LONG len = EndSort(BHEAD (WORD *) ((void *) (&optimize_expr)), 2);

```



```

00256     LowerSortLevel();
00257     AT.WorkPointer = term;
00258
00259     return len;
00260 }
00261
00262 /*
00263     #] get_expression :
00264     #[ PF_get_expression :
00265 */
00266 #ifdef WITHMPI
00267
00268 // get_expression for ParFORM
00269 LONG PF_get_expression (int exprnr) {
00270     LONG len;
00271     if (PF.me == MASTER) {
00272         len = get_expression(exprnr);
00273     }
00274     if (PF.numtasks > 1) {
00275         PF_BroadcastBuffer(&optimize_expr, &len);
00276     }
00277     return len;
00278 }
00279
00280 // replace get_expression called in Optimize
00281 #define get_expression PF_get_expression
00282
00283 #endif
00284 /*
00285     #] PF_get_expression :
00286     #[ get_brackets :
00287 */
00288
00303 vector<vector<WORD> > get_brackets () {
00304
00305     // check for brackets in expression
00306     bool has_brackets = false;
00307     for (WORD *t=optimize_expr; *t!=0; t+=*t) {
00308         WORD *tend=t+*t; tend-=ABS(*(tend-1));
00309         for (WORD *t1=t+1; t1<tend; t1+=*(t1+1))
00310             if (*t1 == HAAKJE)
00311                 has_brackets=true;
00312     }
00313
00314     // replace brackets by SEPARATESYMBOL
00315     vector<vector<WORD> > brackets;
00316
00317     if (has_brackets) {
00318         int exprlen=10; // we need potential space for an empty bracket
00319         for (WORD *t=optimize_expr; *t!=0; t+=*t)
00320             exprlen += *t;
00321         WORD *newexpr = (WORD *)Malloc1(exprlen*sizeof(WORD), "optimize newexpr");
00322
00323         int i=0;
00324         int sep_power = 0;
00325
00326         for (WORD *t=optimize_expr; *t!=0; t+=*t) {
00327             WORD *t1 = t+1;
00328
00329             // scan for bracket
00330             vector<WORD> bracket;
00331             for (; *t1!=HAAKJE; t1+=*(t1+1))
00332                 bracket.insert(bracket.end(), t1, t1+*(t1+1));
00333
00334             if (brackets.size()==0 || bracket!=brackets.back()) {
00335                 sep_power++;
00336                 brackets.push_back(bracket);
00337             }
00338             t1+=*(t1+1);
00339
00340             WORD left = t + *t - t1;
00341             bool more_symbols = (left != ABS(*(t+*t-1)));
00342
00343             // add power of SEPARATESYMBOL
00344             newexpr[i++] = 1 + left + (more_symbols ? 2 : 4);
00345             newexpr[i++] = SYMBOL;
00346             newexpr[i++] = (more_symbols ? *(t1+1) + 2 : 4);
00347             newexpr[i++] = SEPARATESYMBOL;
00348             newexpr[i++] = sep_power;
00349
00350             // add remaining terms
00351             if (more_symbols) {
00352                 t1+=2;
00353                 left-=2;
00354             }
00355             while (left-->0)
00356                 newexpr[i++] = *(t1++);

```

```

00357     }
00358     /*
00359     We insert here an empty bracket that is zero.
00360     It is used for the case that there is only a single bracket which is
00361     outside the notation for trees at a later stage.
00362
00363     There may be a problem now in that in the case of sep_power==1
00364     newexpr is bigger than optimize_expr. We have to check that.
00365     */
00366     if ( sep_power == 1 )
00367     {
00368         WORD *t;
00369         vector<WORD> bracket;
00370         bracket.push_back(0);
00371         bracket.push_back(0);
00372         bracket.push_back(0);
00373         bracket.push_back(0);
00374         sep_power++;
00375         brackets.push_back(bracket);
00376         newexpr[i++] = 8;
00377         newexpr[i++] = SYMBOL;
00378         newexpr[i++] = 4;
00379         newexpr[i++] = SEPARATESYMBOL;
00380         newexpr[i++] = sep_power;
00381         newexpr[i++] = 1;
00382         newexpr[i++] = 1;
00383         newexpr[i++] = 3;
00384         newexpr[i++] = 0;
00385         for (t=optimize_expr; *t!=0; t+=*t) {}
00386         if ( t-optimize_expr+1 < i ) { // We have to redo this
00387             M_free(optimize_expr,"$-sort space");
00388             optimize_expr = (WORD *)Malloca(1*sizeof(WORD),"$-sort space");
00389         }
00390     }
00391     else {
00392         newexpr[i++] = 0;
00393     }
00394     memcpy(optimize_expr, newexpr, i*sizeof(WORD));
00395     M_free(newexpr,"optimize newexpr");
00396
00397     // if factorized, replace SEP by FAC and remove brackets
00398     if (brackets[0].size()>0 && brackets[0][2]==FACTORSYMBOL) {
00399         for (WORD *t=optimize_expr; *t!=0; t+=*t) {
00400             if (*t == ABS(*(t+*t-1))+1) continue;
00401             if (t[1]==SYMBOL)
00402                 for (int i=3; i<t[2]; i+=2)
00403                     if (t[i]==SEPARATESYMBOL) t[i]=FACTORSYMBOL;
00404         }
00405         return vector<vector<WORD> >();
00406     }
00407 }
00408
00409 return brackets;
00410 }
00411
00412 /*
00413 #] get_brackets :
00414 #[ count_operators :
00415 */
00416
00423 int count_operators (const WORD *expr, bool print=false) {
00424
00425     int n=0;
00426     while (*(expr+n)!=0) n+=*(expr+n);
00427
00428     int cntpow=0, cntmul=0, cntadd=0, sumpow=0;
00429     WORD maxpowfac=1, maxpowsep=1;
00430
00431     for (const WORD *t=expr; *t!=0; t+=*t) {
00432         if (t!=expr) cntadd++; // new term
00433         if (*t==ABS(*(t+*t-1))+1) continue; // only coefficient
00434
00435         int cntsym=0;
00436
00437         if (t[1]==SYMBOL)
00438             for (int i=3; i<t[2]; i+=2) {
00439                 if (t[i]==FACTORSYMBOL) {
00440                     maxpowfac = max(maxpowfac, t[i+1]);
00441                     continue;
00442                 }
00443                 if (t[i]==SEPARATESYMBOL) {
00444                     maxpowsep = max(maxpowsep, t[i+1]);
00445                     continue;
00446                 }
00447                 if (t[i+1]>2) { // (extra)symbol power>2
00448                     cntpow++;
00449                     sumpow += (int)floor(log(t[i+1])/log(2.0)) + popcount(t[i+1]) - 1;

```

```

00450         }
00451         if (t[i+1]==2) cntmul++; // (extra)symbol squared
00452         cntsym++;
00453     }
00454
00455     if (ABS(*(t++t-1))!=3 || *(t++t-2)!=1 || *(t++t-3)!=1) cntsym++; // non +/-1 coefficient
00456
00457     if (cntsym > 0) cntmul+=cntsym-1;
00458 }
00459
00460 cntadd -= maxpowfac-1;
00461 cntmul += maxpowfac-1;
00462
00463 cntadd -= maxpowsep-1;
00464
00465 if (print)
00466     MesPrint ("*** STATS: original %1P %1M %1A : %1", cntpow,cntmul,cntadd,sumpow+cntmul+cntadd);
00467
00468 return sumpow+cntmul+cntadd;
00469 }
00470
00471 int count_operators (const vector<WORD> &instr, bool print=false) {
00472     int cntpow=0, cntmul=0, cntadd=0, sumpow=0;
00473
00474     const WORD *ebegin = &instr.begin();
00475     const WORD *eend = ebegin+instr.size();
00476
00477     for (const WORD *e=ebegin; e!=eend; e+=*(e+2)) {
00478         for (const WORD *t=e+3; *t!=0; t+=*t) {
00479             if (t!=e+3) {
00480                 if (*(e+1)==OPER_ADD) cntadd++; // new term
00481                 if (*(e+1)==OPER_MUL) cntmul++; // new term
00482             }
00483             if (*t == ABS(*(t++t-1))+1) continue; // only coefficient
00484             if (*(t+1)==SYMBOL || *(t+1)==EXTRASYMBOL) {
00485                 if (*(t+4)==2) cntmul++; // (extra)symbol squared
00486                 if (*(t+4)>2) { // (extra)symbol power>2
00487                     cntpow++;
00488                     sumpow += (int)floor(log(*(t+4))/log(2.0)) + popcount(*(t+4)) - 1;
00489                 }
00490             }
00491             if (ABS(*(t++t-1))!=3 || *(t++t-2)!=1 || *(t++t-3)!=1) cntmul++; // non +/-1 coefficient
00492         }
00493     }
00494
00495     if (print)
00496         MesPrint ("*** STATS: optimized %1P %1M %1A : %1", cntpow,cntmul,cntadd,sumpow+cntmul+cntadd);
00497
00498     return sumpow+cntmul+cntadd;
00499 }
00500
00501
00502 if (print)
00503     MesPrint ("*** STATS: optimized %1P %1M %1A : %1", cntpow,cntmul,cntadd,sumpow+cntmul+cntadd);
00504
00505 return sumpow+cntmul+cntadd;
00506 }
00507
00508 /*
00509 #] count_operators :
00510 #[ occurrence_order :
00511 */
00512
00520 vector<WORD> occurrence_order (const WORD *expr, bool rev) {
00521     // count the number of occurrences of variables
00522     map<WORD,int> cnt;
00523     for (const WORD *t=expr; *t!=0; t+=*t) {
00524         if (*t == ABS(*(t++t-1))+1) continue; // skip constant term
00525         if (t[1] == SYMBOL)
00526             for (int i=3; i<t[2]; i+=2)
00527                 cnt[t[i]]++;
00528     }
00529
00530
00531     bool is_fac=false, is_sep=false;
00532     if (cnt.count(FACTORSYMBOL)) {
00533         is_fac=true;
00534         cnt.erase(FACTORSYMBOL);
00535     }
00536     if (cnt.count(SEPARATESYMBOL)) {
00537         is_sep=true;
00538         cnt.erase(SEPARATESYMBOL);
00539     }
00540
00541     // determine the order of the variables
00542     vector<pair<int,WORD> > cnt_order;
00543     for (map<WORD,int>::iterator i=cnt.begin(); i!=cnt.end(); i++)
00544         cnt_order.push_back(make_pair(i->second, i->first));
00545     sort(cnt_order.rbegin(), cnt_order.rend());
00546
00547     // create resulting order
00548     vector<WORD> order;
00549     for (int i=0; i<(int)cnt_order.size(); i++)

```

```

00550         order.push_back(cnt_order[i].second);
00551
00552         if (rev) reverse(order.begin(),order.end());
00553
00554         // add FACTORSYMBOL/SEPARATESYMBOL
00555         if (is_fac) order.insert(order.begin(), FACTORSYMBOL);
00556         if (is_sep) order.insert(order.begin(), SEPARATESYMBOL);
00557
00558         return order;
00559     }
00560
00561     /*
00562     #] occurrence_order :
00563     #[ Horner_tree :
00564     */
00565
00601 WORD getpower (const WORD *term, int var, int pos) {
00602     if (*term == ABS>(*term+*term-1))+1) return 0; // constant term
00603     if (2*pos+2 >= term[2]) return 0; // too few symbols
00604     if (term[2*pos+3] != var) return 0; // incorrect symbol
00605     return term[2*pos+4]; // return power
00606 }
00607
00615 void fixarg (UWORD *t, WORD &n) {
00616     int an=ABS(n), sn=SGN(n);
00617     while (*(t+an-1)==0) an--;
00618     n=an*sn;
00619 }
00620
00621 void GcdLong_fix_args (PHEAD UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc) {
00622     fixarg(a,na);
00623     fixarg(b,nb);
00624     GcdLong(BHEAD a,na,b,nb,c,nc);
00625 }
00626
00627 void DivLong_fix_args(UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc, UWORD *d, WORD *nd)
00628 {
00629     fixarg(a,na);
00629     fixarg(b,nb);
00630     DivLong(a,na,b,nb,c,nc,d,nd);
00631 }
00632
00653 void build_Horner_tree (const WORD **terms, int numterms, int var, int maxvar, int pos, vector<WORD>
*res) {
00654
00655     GETIDENTITY;
00656
00657     if (var == maxvar) {
00658         // Horner tree is finished, so add remaining terms unfactorized
00659         // (note: since only complete Horner schemes seem to be useful, numterms=1 here
00660
00661         for (int fr=0; fr<numterms; fr++) {
00662
00663             bool empty = true;
00664             const WORD *t = terms[fr];
00665
00666             // add symbols
00667             if (*t != ABS(*t+*t-1))+1)
00668                 for (int i=3+2*pos; i<t[2]; i+=2) {
00669                     res->push_back(SYMBOL);
00670                     res->push_back(4);
00671                     res->push_back(t[i]);
00672                     res->push_back(t[i+1]);
00673                     if (!empty) res->push_back(OPER_MUL);
00674                     empty = false;
00675                 }
00676
00677             // if empty, add a 1, since the result should look like "... coeff *"
00678             if (empty) {
00679                 res->push_back(SNUMBER);
00680                 res->push_back(5);
00681                 res->push_back(1);
00682                 res->push_back(1);
00683                 res->push_back(3);
00684             }
00685
00686             // add coefficient
00687             res->push_back(SNUMBER);
00688             WORD n = ABS(*t+*t-1);
00689             res->push_back(n+2);
00690             for (int i=0; i<n; i++)
00691                 res->push_back(*t+*t-n+i);
00692             res->push_back(OPER_MUL);
00693
00694             if (fr>0) res->push_back(OPER_ADD);
00695         }
00696

```

```

00697         // result should end with gcd of the terms; right now this never
00698         // triggers, but if one would allow for incomplete Horner schemes,
00699         // one should extract the gcd here
00700         if (numterms > 1) {
00701             res->push_back(SNUMBER);
00702             res->push_back(5);
00703             res->push_back(1);
00704             res->push_back(1);
00705             res->push_back(3);
00706             res->push_back(OPER_MUL);
00707         }
00708     }
00709     else {
00710         // extract variable "var" and the gcd and proceed recursively
00711
00712         WORD nnum = 0, nden = 0, ntmp = 0, ndum = 0;
00713         UWORD *num = NumberMalloc("build_Horner_tree");
00714         UWORD *den = NumberMalloc("build_Horner_tree");
00715         UWORD *tmp = NumberMalloc("build_Horner_tree");
00716         UWORD *dum = NumberMalloc("build_Horner_tree");
00717
00718         // previous coefficient for gcd extraction or coefficient multiplication
00719         int prev_coeff_idx = -1;
00720
00721         for (int fr=0; fr<numterms; ) {
00722
00723             // find power of current term
00724             WORD pow = getpower(terms[fr], var, pos);
00725
00726             // find all terms with that power
00727             int to=fr+1;
00728             while (to<numterms && getpower(terms[to], var, pos) == pow) to++;
00729
00730             // recursively build Horner tree of all terms proportional to var^pow
00731             build_Horner_tree (terms+fr, to-fr, var+1, maxvar, pow==0?pos:pos+1, res);
00732
00733             if (AN.poly_vars[var] != FACTORSYMBOL && AN.poly_vars[var] != SEPARATESYMBOL) {
00734                 // if normal symbol, find gcd(numerators) and gcd(denominators)
00735                 WORD n1 = res->at(res->size()-2) / 2;
00736                 WORD *t1 = &res->at(res->size()-2-2*ABS(n1));
00737
00738                 WORD *t2 = fr==0 ? t1 : &res->at(prev_coeff_idx);
00739                 WORD n2 = fr==0 ? n1 : *(t2+*(t2+1)-1) / 2;
00740                 if (fr>0) t2+=2;
00741
00742                 GcdLong_fix_args(BHEAD (UWORD *)t1,n1,(UWORD *)t2,n2,num,&nnum);
00743                 GcdLong_fix_args(BHEAD (UWORD *)t1+ABS(n1),ABS(n1),(UWORD
*)t2+ABS(n2),ABS(n2),den,&nden);
00744
00745                 // divide out gcds; note: leading zeroes can be added here
00746                 for (int i=0; i<2; i++) {
00747                     if (i==1 && fr==0) break;
00748
00749                     WORD *t = i==0 ? t1 : t2;
00750                     WORD n = i==0 ? n1 : n2;
00751
00752                     DivLong_fix_args((UWORD *)t, n, num, nnum, tmp, &ntmp, dum, &ndum);
00753                     for (int j=0; j<ABS(ntmp); j++) *t++ = tmp[j];
00754                     for (int j=0; j<ABS(n)-ABS(ntmp); j++) *t++ = 0;
00755
00756                     if (SGN(ntmp) != SGN(n)) n=-n;
00757
00758                     DivLong_fix_args((UWORD *)t, n, den, nden, tmp, &ntmp, dum, &ndum);
00759                     for (int j=0; j<ABS(ntmp); j++) *t++ = tmp[j];
00760                     for (int j=0; j<ABS(n)-ABS(ntmp); j++) *t++ = 0;
00761
00762                     *t++ = SGN(n) * (2*ABS(n)+1);
00763                 }
00764
00765                 // add the addition operator
00766                 if (fr>0) res->push_back(OPER_ADD);
00767
00768                 // add the power of "var"
00769                 WORD nextpow = (to==numterms ? 0 : getpower(terms[to], var, pos));
00770
00771                 if (pow-nextpow > 0) {
00772                     res->push_back(SYMBOL);
00773                     res->push_back(4);
00774                     res->push_back(var);
00775                     res->push_back(pow-nextpow);
00776                     res->push_back(OPER_MUL);
00777                 }
00778
00779                 // add the extracted gcd
00780                 res->push_back(SNUMBER);
00781                 WORD n = MaX(ABS(nnum),nden);
00782                 res->push_back(n*2+3);

```

```

00783         for (int i=0; i<ABS(nnum); i++) res->push_back(num[i]);
00784         for (int i=0; i<n-ABS(nnum); i++) res->push_back(0);
00785         for (int i=0; i<nden; i++) res->push_back(den[i]);
00786         for (int i=0; i<n-ABS(nden); i++) res->push_back(0);
00787         res->push_back(SGN(nnum)*(2*n+1));
00788         res->push_back(OPER_MUL);
00789
00790         prev_coeff_idx = res->size() - ABS(res->at(res->size()-2)) - 3;
00791     }
00792     else if (AN.poly_vars[var]==FACTORSYMBOL) {
00793
00794         // if factorsymbol, multiply overall integer contents
00795
00796         if (fr>0) {
00797             WORD n1 = res->at(res->size()-2) / 2;
00798             WORD *t1 = &res->at(res->size()-2-2*ABS(n1));
00799             WORD *t2 = &res->at(prev_coeff_idx);
00800             WORD n2 = *(t2+(t2+1)-1) / 2;
00801             t2+=2;
00802
00803             MulRat(BHEAD (UWORD *)t1,n1, (UWORD *)t2,n2,tmp,&ntmp);
00804
00805             // replace previous coefficient with 1
00806             n2=ABS(n2);
00807             for (int i=0; i<ABS(n2); i++)
00808                 t2[i] = t2[n2+i] = i==0 ? 1 : 0;
00809             t2[2*n2] = 2*n2+1;
00810
00811             // remove this coefficient
00812             for (int i=0; i<2*ABS(n1)+2; i++)
00813                 res->pop_back();
00814
00815             // add product
00816             res->back() = 2*ABS(ntmp)+3; // adjust size of term
00817             res->insert(res->end(), tmp, tmp+2*ABS(ntmp)); // num/den coefficient
00818             res->push_back(SGN(ntmp)*(2*ABS(ntmp)+1)); // size of coefficient
00819             res->push_back(OPER_MUL); // operator
00820         }
00821
00822         prev_coeff_idx = res->size() - ABS(res->at(res->size()-2)) - 3;
00823
00824         // multiply previous factors with this factor
00825         if (fr>0)
00826             res->push_back(OPER_MUL);
00827     }
00828     else { // AN.poly_vars[var]==SEPARATESYMBOL
00829         if (fr>0)
00830             res->push_back(OPER_COMMA);
00831         prev_coeff_idx = -1;
00832     }
00833
00834     fr=to;
00835 }
00836
00837 // cleanup
00838 NumberFree(dum,"build_Horner_tree");
00839 NumberFree(tmp,"build_Horner_tree");
00840 NumberFree(den,"build_Horner_tree");
00841 NumberFree(num,"build_Horner_tree");
00842 }
00843 }
00844
00853 bool term_compare (const WORD *a, const WORD *b) {
00854     if (*a == ABS(*(a++a-1))+1) return true; // coefficient comes first
00855     if (*b == ABS(*(b++b-1))+1) return false;
00856     if (a[1]!=SYMBOL) return true;
00857     if (b[1]!=SYMBOL) return false;
00858     for (int i=3; i<a[2] && i<b[2]; i+=2) {
00859         if (a[i] !=b[i] ) return a[i] >b[i];
00860         if (a[i+1]!=b[i+1]) return a[i+1]<b[i+1];
00861     }
00862     return a[2]<b[2];
00863 }
00864
00874 vector<WORD> Horner_tree (const WORD *expr, const vector<WORD> &order) {
00875     #ifdef DEBUG
00876         MesPrint ("*** [%s, w=%w] CALL: Horner_tree(%a)", thetime_str().c_str(), order.size(), &order[0]);
00877     #endif
00878
00879     GETIDENTITY;
00880
00881     // find the renumbering scheme (new numbers are 0,1,...,#vars-1)
00882     map<WORD,WORD> renum;
00883     AN.poly_num_vars = order.size();
00884     AN.poly_vars = (WORD *)Malloc1(AN.poly_num_vars*sizeof(WORD), "AN.poly_vars");
00885     for (int i=0; i<AN.poly_num_vars; i++) {
00886         AN.poly_vars[i] = order[i];

```

```

00887         renum[order[i]] = i;
00888     }
00889
00890     // sort variables in individual terms using bubble sort
00891     WORD* sorted;
00892     WORD* dynamicAlloc = 0;
00893     LONG sumsize = 0;
00894
00895     for (const WORD *t=expr; *t!=0; t+=*t) {
00896         sumsize += *t;
00897     }
00898
00899     // We actually need sumsize + 1 WORDS available, due to the "*sorted = 0;"
00900     // at the end of the sort loop.
00901     if ( AT.WorkPointer + sumsize + 1 > AT.WorkTop ) {
00902         dynamicAlloc = (WORD*)Malloc1(sizeof(*dynamicAlloc)*(sumsize+1), "Horner_tree alloc");
00903         sorted = dynamicAlloc;
00904     }
00905     else {
00906         sorted = AT.WorkPointer;
00907     }
00908
00909     for (const WORD *t=expr; *t!=0; t+=*t) {
00910         memcpy(sorted, t, *t*sizeof(WORD));
00911
00912         if (*t != ABS(*(t+*t-1))+1) {
00913             // non-constant term
00914
00915             // renumber variables, adding new variables if necessary
00916             for (int i=3; i<sorted[2]; i+=2) {
00917                 if (!renum.count(sorted[i])) {
00918                     renum[sorted[i]] = AN.poly_num_vars;
00919
00920                     WORD *new_poly_vars = (WORD *)Malloc1((AN.poly_num_vars+1)*sizeof(WORD),
00921 "AN.poly_vars");
00922                     memcpy(new_poly_vars, AN.poly_vars, AN.poly_num_vars*sizeof(WORD));
00923                     M_free(AN.poly_vars,"poly_vars");
00924                     AN.poly_vars = new_poly_vars;
00925                     AN.poly_vars[AN.poly_num_vars] = sorted[i];
00926                     AN.poly_num_vars++;
00927                 }
00928                 sorted[i] = renum[sorted[i]];
00929             }
00930             // order variables
00931             for (int i=0; i<sorted[2]/2; i++)
00932                 for (int j=3; j+2<sorted[2]; j+=2)
00933                     if (sorted[j] > sorted[j+2]) {
00934                         swap(sorted[j], sorted[j+2]);
00935                         swap(sorted[j+1], sorted[j+3]);
00936                     }
00937         }
00938         sorted += *sorted;
00939     }
00940
00941     *sorted = 0;
00942     if ( dynamicAlloc ) {
00943         sorted = dynamicAlloc;
00944     }
00945     else {
00946         sorted = AT.WorkPointer;
00947     }
00948
00949     // find pointers to all terms and sort them efficiently
00950     vector<const WORD *> terms;
00951     for (const WORD *t=sorted; *t!=0; t+=*t)
00952         terms.push_back(t);
00953     sort(terms.rbegin(), terms.rend(), term_compare);
00954
00955     // construct the Horner tree
00956     vector<WORD> res;
00957     build_Horner_tree(&terms[0], terms.size(), 0, AN.poly_num_vars, 0, &res);
00958
00959     // remove leading zeroes in coefficients
00960     int j=0;
00961     for (int i=0; i<(int)res.size(); ) {
00962         if (res[i]==OPER_ADD || res[i]==OPER_MUL || res[i]==OPER_COMMA)
00963             res[j++] = res[i++];
00964         else if (res[i]==SYMBOL) {
00965             memmove(&res[j], &res[i], res[i+1]*sizeof(WORD));
00966             i+=res[j+1];
00967             j+=res[j+1];
00968         }
00969         else if (res[i]==SNUMBER) {
00970             int n = (res[i+1]-2)/2;
00971             int dn = 0;
00972             while (res[i+1+n-dn]==0 && res[i+1+2*n-dn]==0) dn++;

```

```

00973         res[j++] = SNUMBER;
00974         res[j++] = 2*(n-dn) + 3;
00975         memmove(&res[j], &res[i+2], (n-dn)*sizeof(WORD));
00976         j+=n-dn;
00977         memmove(&res[j], &res[i+n+2], (n-dn)*sizeof(WORD));
00978         j+=n-dn;
00979         res[j++] = SGN(res[i+2*n+2])*(2*(n-dn)+1);
00980         i+=2*n+3;
00981     }
00982 }
00983 res.resize(j);
00984
00985 if ( dynamicAlloc ) {
00986     M_free(dynamicAlloc, "Horner_tree alloc");
00987 }
00988
00989 #ifdef DEBUG
00990     MesPrint ("*** [%s, w=%w] DONE: Horner_tree(%a)", thetime_str().c_str(), order.size(), &order[0]);
00991 #endif
00992
00993     return res;
00994 }
00995
00996 /*
00997  #] Horner_tree :
00998  #[ print_tree :
00999  */
01000
01001 // print Horner tree (for debugging)
01002 void print_tree (const vector<WORD> &tree) {
01003
01004     GETIDENTITY;
01005
01006     for (int i=0; i<(int)tree.size();) {
01007         if (tree[i]==OPER_ADD) {
01008             MesPrint ("%");
01009             i++;
01010         }
01011         else if (tree[i]==OPER_MUL) {
01012             MesPrint ("%");
01013             i++;
01014         }
01015         else if (tree[i]==OPER_COMMA) {
01016             MesPrint (",");
01017             i++;
01018         }
01019         else if (tree[i]==SNUMBER) {
01020             UBYTE buf[100];
01021             int n = tree[i+tree[i+1]-1]/2;
01022             PrtLong((UWORD *)&tree[i+2], n, buf);
01023             int l = strlen((char *)buf);
01024             buf[l]='/';
01025             n=ABS(n);
01026             PrtLong((UWORD *)&tree[i+2+n], n, buf+l+1);
01027             MesPrint ("%s",buf);
01028             i+=tree[i+1];
01029         }
01030         else if (tree[i]==SYMBOL) {
01031             if (AN.poly_vars[tree[i+2]] < 10000)
01032                 MesPrint ("%s^d%", VARNAME(symbols,AN.poly_vars[tree[i+2]]), tree[i+3]);
01033             else
01034                 MesPrint ("%Zd^d%", MAXVARIABLES-AN.poly_vars[tree[i+2]], tree[i+3]);
01035             i+=tree[i+1];
01036         }
01037         else {
01038             MesPrint ("error");
01039             exit(1);
01040         }
01041
01042         MesPrint (" %");
01043     }
01044
01045     MesPrint ("");
01046 }
01047
01048 /*
01049  #] print_tree :
01050  #[ generate_instructions :
01051  */
01052
01053
01054 struct CSEHash {
01055     size_t operator()(const vector<WORD>& n) const {
01056         return n[0];
01057     }
01058 };
01059

```



```

01060 struct CSEEq {
01061     bool operator()(const vector<WORD>& lhs, const vector<WORD>& rhs) const {
01062         if (lhs.size() != rhs.size()) return false;
01063         for (unsigned int i = 1; i < lhs.size(); i++) {
01064             if (lhs[i] != rhs[i]) return false;
01065         }
01066         return true;
01067     }
01068 };
01069
01070
01071 template<typename T> size_t hash_range(T* array, int size) {
01072     size_t hash = 0;
01073
01074     for (int i = 0; i < size; i++) {
01075         hash ^= array[i] + 0x9e3779b9 + (hash << 6) + (hash >> 2);
01076     }
01077
01078     return hash;
01079 }
01080
01133 vector<WORD> generate_instructions (const vector<WORD> &tree, bool do_CSE) {
01134     #ifdef DEBUG
01135         MesPrint ("*** [%s, w=%w] CALL: generate_instructions(cse=%d)",
01136                 thetime_str().c_str(), do_CSE?1:0);
01137     #endif
01138
01139     typedef unordered_map<vector<WORD>, int, CSEHash, CSEEq> csemap;
01140     csemap ID;
01141
01142     // reserve lots of space, to prevent later rehashes
01143     // TODO: what if this is too large? make a parameter?
01144     if (do_CSE) {
01145         ID.rehash(mcts_expr_score * 2);
01146     }
01147
01148     // s is a stack of operands to process when you encounter operators
01149     // in the postfix expression tree. Operands consist of three WORDs,
01150     // formatted as the LHS/RHS of the keys in ID.
01151     stack<int> s;
01152     vector<WORD> instr;
01153     WORD numinstr = 0;
01154     vector<WORD> x;
01155
01156     // process the expression tree
01157     for (int i=0; i<(int)tree.size(); ) {
01158         x.clear();
01159
01160         if (tree[i]==SNUMBER) {
01161             WORD hash = hash_range(&tree[i], tree[i + 1]);
01162             x.push_back(hash);
01163             x.push_back(SNUMBER);
01164             x.insert(x.end(), &tree[i], &tree[i]+tree[i+1]);
01165             int sign = SGN(x.back());
01166             x.back() *= sign;
01167
01168             std::pair<csemap::iterator, bool> suc = ID.insert(csemap::value_type(x, i + 1));
01169             s.push(0);
01170             s.push(sign * suc.first->second);
01171             s.push(SNUMBER);
01172             s.push(hash);
01173             i+=tree[i+1];
01174         }
01175         else if (tree[i]==SYMBOL) {
01176             WORD hash = hash_range(&tree[i], tree[i + 1]);
01177             if (tree[i+3]>1) {
01178                 x.push_back(hash);
01179                 x.push_back(SYMBOL);
01180                 x.push_back(tree[i+2]+1); // variable (1-indexed)
01181                 x.push_back(tree[i+3]); // power
01182
01183                 csemap::iterator it = ID.find(x);
01184                 if (do_CSE && it != ID.end()) {
01185                     // already-seen power of a symbol
01186                     s.push(1);
01187                     s.push(it->second);
01188                     s.push(EXTRASYMBOL);
01189                 } else {
01190                     //MesPrint("strange");
01191                     if (numinstr == MAXPOSITIVE) {
01192                         MesPrint((char *) "ERROR: too many temporary variables needed in
optimization");
01193                         Terminate(-1);
01194                     }
01195
01196                     // new power greater than 1 of a symbol
01197                     instr.push_back(numinstr); // expr.nr

```

```

01198         instr.push_back(OPER_MUL); // operator
01199         instr.push_back(9+OPTHEAD); // length total
01200         instr.push_back(8); // length operand
01201         instr.push_back(SYMBOL); // SYMBOL
01202         instr.push_back(4); // length symbol
01203         instr.push_back(tree[i+2]); // variable
01204         instr.push_back(tree[i+3]); // power
01205         instr.push_back(1); // numerator
01206         instr.push_back(1); // denominator
01207         instr.push_back(3); // length coeff
01208         instr.push_back(0); // trailing 0
01209
01210         ID[x] = ++numinstr;
01211         s.push(1);
01212         s.push(numinstr);
01213         s.push(EXTRASymbol);
01214     }
01215 }
01216 else {
01217     // power of 1
01218     s.push(tree[i+3]); // power
01219     s.push(tree[i+2]+1); // variable (1-indexed)
01220     s.push(SYMBOL);
01221 }
01222
01223 s.push(hash); // push hash
01224 i+=tree[i+1];
01225 }
01226 else { // tree[i]==OPERATOR
01227     int oper = tree[i];
01228     i++;
01229
01230     x.push_back(0); // placeholder for hash
01231     x.push_back(oper);
01232     vector<WORD> hash;
01233     hash.push_back(oper);
01234
01235     // get two operands from the stack
01236     for (int operand=0; operand<2; operand++) {
01237         hash.push_back(s.top()); s.pop();
01238         x.push_back(s.top()); s.pop();
01239         x.push_back(s.top()); s.pop();
01240         x.push_back(s.top()); s.pop();
01241     }
01242
01243     x[0] = hash_range(&hash[0], 3);
01244
01245     // get rid of multiplications by +/-1
01246     if (oper==OPER_MUL) {
01247         bool do_continue = false;
01248
01249         for (int operand=0; operand<2; operand++) {
01250             int idx_oper1 = operand==0 ? 2 : 5;
01251             int idx_oper2 = operand==0 ? 5 : 2;
01252
01253             // check whether operand 1 equals +/-1
01254             if (x[idx_oper1]==SNUMBER) {
01255
01256                 int idx = ABS(x[idx_oper1+1])-1;
01257                 if (tree[idx+2]==1 && tree[idx+3]==1 && ABS(tree[idx+4])==3) {
01258                     // push +/- other operand and continue
01259                     s.push(x[idx_oper2+2]);
01260                     s.push(x[idx_oper2+1]*SGN(x[idx_oper1+1]));
01261                     s.push(x[idx_oper2]);
01262                     s.push(hash[1 + (operand + 1) % 2]);
01263                     do_continue = true;
01264                     break;
01265                 }
01266             }
01267         }
01268
01269         if (do_continue) continue;
01270     }
01271
01272     // check whether this subexpression has been seen before
01273     // if not, generate instruction to define it
01274     csemapi::iterator it = ID.find(x);
01275     if (!do_CSE || it == ID.end()) {
01276         if (numinstr == MAXPOSITIVE) {
01277             MesPrint((char *) "ERROR: too many temporary variables needed in optimization");
01278             Terminate(-1);
01279         }
01280
01281         instr.push_back(numinstr); // expr.nr.
01282         instr.push_back(x[1]); // operator
01283         instr.push_back(3); // length
01284     }

```

```

01285         ID[x] = ++numinstr;
01286
01287         int lenidx = instr.size()-1;
01288
01289         for (int j=0; j<2; j++)
01290             if (x[3*j+2]==SYMBOL || x[3*j+2]==EXTRASymbol) {
01291                 instr.push_back(8); // length total
01292                 instr.push_back(x[3*j+2]); // (extra)symbol
01293                 instr.push_back(4); // length (extra)symbol
01294                 instr.push_back(ABS(x[3*j+3])-1); // variable (0-indexed)
01295                 instr.push_back(x[3*j+4]); // power
01296                 instr.push_back(1); // numerator
01297                 instr.push_back(1); // denominator
01298                 instr.push_back(3*SGN(x[3*j+3])); // length coeff
01299                 instr[lenidx] += 8;
01300             }
01301             else { // x[3*j+1]==SNUMBER
01302                 int t = ABS(x[3*j+3])-1;
01303                 instr.push_back(tree[t+1]-1); // length number
01304                 instr.insert(instr.end(), &tree[t+2], &tree[t+tree[t+1]]); // digits
01305                 instr.back() *= SGN(instr.back()) * SGN(x[3*j+3]);
01306                 instr[lenidx] += tree[t+1]-1;
01307             }
01308
01309         instr.push_back(0); // trailing 0
01310         instr[lenidx]++;
01311     }
01312
01313     // push new expression on the stack
01314     s.push(1);
01315     s.push(ID[x]);
01316     s.push(EXTRASymbol);
01317     s.push(x[0]); // push hash
01318 }
01319 }
01320
01321 #ifdef DEBUG
01322     MesPrint ("*** [%s, w=%w] DONE: generate_instructions(cse=%d) : numoper=%d",
01323             thetime_str().c_str(), do_CSE?1:0, count_operators(instr));
01324 #endif
01325
01326     return instr;
01327 }
01328
01329 /*
01330 #] generate_instructions :
01331 #[ count_operators_cse :
01332 */
01333
01334
01342 int count_operators_cse (const vector<WORD> &tree) {
01343     //MesPrint ("*** [%s] Starting CSEE", thetime_str().c_str());
01344
01345     typedef unordered_map<vector<WORD>, int, CSEHash, CSEEq> csemap;
01346     csemap ID;
01347
01348     // reserve lots of space, to prevent later rehashes
01349     // TODO: what if this is too large? make a parameter?
01350     ID.rehash(mcts_expr_score * 2);
01351
01352     // s is a stack of operands to process when you encounter operators
01353     // in the postfix expression tree. Operands consist of three WORDs,
01354     // formatted as the LHS/RHS of the keys in ID.
01355     stack<int> s;
01356     int numinstr = 0, numcommas = 0;
01357     vector<WORD> x;
01358
01359     // process the expression tree
01360     for (int i=0; i<(int)tree.size(); i) {
01361         x.clear();
01362
01363         if (tree[i]==SNUMBER) {
01364             WORD hash = hash_range(&tree[i], tree[i + 1]);
01365             x.push_back(hash);
01366             x.push_back(SNUMBER);
01367             x.insert(x.end(), &tree[i], &tree[i+tree[i+1]]);
01368             int sign = SGN(x.back());
01369             x.back() *= sign;
01370
01371             std::pair<csemap::iterator, bool> suc = ID.insert(csemap::value_type(x, i + 1));
01372             s.push(0);
01373             s.push(sign * suc.first->second);
01374             s.push(SNUMBER);
01375             s.push(hash);
01376             i+=tree[i+1];
01377         }
01378         else if (tree[i]==SYMBOL) {

```

```

01379     WORD hash = hash_range(&tree[i], tree[i + 1]);
01380     if (tree[i+3]>1) {
01381         x.push_back(hash);
01382         x.push_back(SYMBOL);
01383         x.push_back(tree[i+2]+1); // variable (1-indexed)
01384         x.push_back(tree[i+3]); // power
01385
01386         csemapi::iterator it = ID.find(x);
01387         if (it != ID.end()) {
01388             // already-seen power of a symbol
01389             s.push(1);
01390             s.push(it->second);
01391             s.push(EXTRASYMBOL);
01392         } else {
01393             if (tree[i + 3] == 2)
01394                 numinstr++;
01395             else
01396                 numinstr += (int)floor(log(tree[i+3])/log(2.0)) + popcount(tree[i+3]) - 1;
01397
01398             ID[x] = numinstr;
01399             s.push(1);
01400             s.push(numinstr);
01401             s.push(EXTRASYMBOL);
01402         }
01403     }
01404     else {
01405         // power of 1
01406         s.push(tree[i+3]); // power
01407         s.push(tree[i+2]+1); // variable (1-indexed)
01408         s.push(SYMBOL);
01409     }
01410
01411     s.push(hash); // push hash
01412
01413     i+=tree[i+1];
01414 }
01415 else { // tree[i]==OPERATOR
01416     int oper = tree[i];
01417     i++;
01418
01419     x.push_back(0); // placeholder for hash
01420     x.push_back(oper);
01421     vector<WORD> hash;
01422     hash.push_back(oper);
01423
01424     // get two operands from the stack
01425     for (int operand=0; operand<2; operand++) {
01426         hash.push_back(s.top()); s.pop();
01427         x.push_back(s.top()); s.pop();
01428         x.push_back(s.top()); s.pop();
01429         x.push_back(s.top()); s.pop();
01430     }
01431
01432
01433     x[0] = hash_range(&hash[0], 3);
01434
01435     // get rid of multiplications by +/-1
01436     if (oper==OPER_MUL) {
01437         bool do_continue = false;
01438
01439         for (int operand=0; operand<2; operand++) {
01440             int idx_oper1 = operand==0 ? 2 : 5;
01441             int idx_oper2 = operand==0 ? 5 : 2;
01442
01443             // check whether operand 1 equals +/-1
01444             if (x[idx_oper1]==SNUMBER) {
01445
01446                 int idx = ABS(x[idx_oper1+1])-1;
01447                 if (tree[idx+2]==1 && tree[idx+3]==1 && ABS(tree[idx+4])==3) {
01448                     // push +/- other operand and continue
01449                     s.push(x[idx_oper2+2]);
01450                     s.push(x[idx_oper2+1]*SGN(x[idx_oper1+1]));
01451                     s.push(x[idx_oper2]);
01452                     s.push(hash[1 + (operand + 1) % 2]);
01453                     do_continue = true;
01454                     break;
01455                 }
01456             }
01457         }
01458
01459         if (do_continue) continue;
01460     }
01461
01462     // check whether this subexpression has been seen before
01463     // if not, generate instruction to define it
01464     csemapi::iterator it = ID.find(x);
01465     if (it == ID.end()) {

```

```

01466         if (numinstr == MAXPOSITIVE) {
01467             MesPrint((char *) "ERROR: too many temporary variables needed in optimization");
01468             Terminate(-1);
01469         }
01470
01471         if (oper == OPER_COMMA) numcommas++;
01472         ID[x] = ++numinstr;
01473         s.push(1);
01474         s.push(numinstr);
01475         s.push(EXTRASymbol);
01476     } else {
01477         // push new expression on the stack
01478         s.push(1);
01479         s.push(it->second);
01480         s.push(EXTRASymbol);
01481     }
01482
01483     s.push(x[0]); // push hash
01484 }
01485 }
01486
01487 //MesPrint ("*** [%s] Stopping CSEE", thetime_str().c_str());
01488 return numinstr - numcommas;
01489 }
01490
01491 /*
01492 #] count_operators_cse :
01493 #[ count_operators_cse_topdown :
01494 */
01495
01496 typedef struct node {
01497     const WORD* data;
01498     struct node* l;
01499     struct node* r; // TODO: add l,r to data?
01500     WORD sign; // TODO: use data for this?
01501     UWORD hash;
01502
01503     node() : l(NULL), r(NULL), sign(1), hash(0) {};
01504     node(const WORD* data) : data(data), l(NULL), r(NULL), sign(1), hash(0) {};
01505
01506     // a minus sign in the tree should only count as a different entry if
01507     // it is a compound expression: a = -a, but T+-V != T+V
01508     int cmp(const struct node* rhs) const {
01509         if (this == rhs) return 0;
01510
01511         if (data[0] != rhs->data[0]) return data[0] < rhs->data[0] ? -1 : 1;
01512         int mod = data[0] == SNUMBER ? -1 : 0; // don't check sign, for numbers
01513         if (data[0] == SYMBOL || data[0] == SNUMBER) {
01514             for (int i = 0; i < data[1] + mod; i++) {
01515                 if (data[i] != rhs->data[i]) return data[i] < rhs->data[i] ? -1 : 1;
01516             }
01517         } else {
01518             int lv = l->cmp(rhs->l);
01519             if (lv != 0) return lv;
01520             int rv = r->cmp(rhs->r);
01521             if (rv != 0) return rv;
01522
01523             // TODO: only for ADD operation
01524             if (l->sign != rhs->l->sign) return l->sign < rhs->l->sign ? -1 : 1;
01525             if (r->sign != rhs->r->sign) return r->sign < rhs->r->sign ? -1 : 1;
01526         }
01527
01528         return 0;
01529     }
01530
01531     // less than operator
01532     bool operator() (const struct node* lhs, const struct node* rhs) const
01533     {
01534         return lhs->cmp(rhs) < 0;
01535     }
01536
01537     void calcHash() {
01538         int mod = data[0] == SNUMBER ? -1 : 0; // don't check sign, for numbers
01539         if (data[0] == SYMBOL || data[0] == SNUMBER) {
01540             hash = hash_range(data, data[1] + mod);
01541         } else {
01542             if (l->hash == 0) l->calcHash();
01543             if (r->hash == 0) r->calcHash();
01544
01545             // signs only matter for compound expressions
01546             size_t newr[] = {(size_t)data[0], l->hash, (size_t)l->sign, r->hash, (size_t)r->sign};
01547             hash = hash_range(newr, 5);
01548         }
01549     }
01550 } NODE;
01551
01552 struct NodeHash {

```

```

01553     size_t operator()(const NODE* n) const {
01554         return n->hash; // already computed
01555     }
01556 };
01557
01558 struct NodeEq {
01559     bool operator()(const NODE* lhs, const NODE* rhs) const {
01560         return lhs->cmp(rhs) == 0;
01561     }
01562 };
01563
01564 NODE* buildTree(vector<WORD> &tree) {
01565     //MesPrint ("*** [%s] Starting CSEE topdown", thetime_str().c_str());
01566
01567     // allocate spaces for the tree, cannot be more nodes than tree size
01568     NODE* ar = (NODE*)Mallocl(tree.size() * sizeof(NODE), "CSE tree");
01569     NODE* c = 0;
01570     unsigned int curIndex = 0;
01571
01572     stack<NODE*> st;
01573     for (int i=0; i<(int)tree.size(); ) {
01574         c = ar + curIndex;
01575         new (c) NODE(&tree[i]); // placement new
01576         curIndex++;
01577
01578         if (tree[i]==SYMBOL || tree[i] == SNUMBER) {
01579             // extract the sign to a new class member
01580             if (tree[i] == SNUMBER) {
01581                 c->sign = SGN(tree[i + tree[i + 1] -1]);
01582             }
01583
01584             c->calcHash();
01585             st.push(c);
01586             i+=tree[i+1];
01587         } else {
01588             c->r = st.top(); st.pop();
01589             c->l = st.top(); st.pop();
01590
01591             // filter *1 and *-1
01592             // TODO: also multiply if there are two numbers?
01593             if (c->data[0] == OPER_MUL) {
01594                 NODE* ch[] = {c->r, c->l};
01595                 for (int j = 0; j < 2; j++)
01596                     if (ch[j]->data[0] == SNUMBER && ch[j]->data[1] == 5 && ch[j]->data[2]==1 &&
ch[j]->data[3]==1) {
01597                         ch[(j+1)%2]->sign *= ch[j]->sign; // transfer sign
01598                         c = ch[(j+1)%2];
01599                         break;
01600                     }
01601             }
01602
01603             c->calcHash();
01604             st.push(c);
01605             i++;
01606         }
01607     }
01608
01609     // TODO: reallocate to smaller size? Could save memory
01610     //MesPrint("Memory difference: %d vs %d", curIndex, tree.size());
01611
01612     // we want to make the root of the tree the first element
01613     // so that we can easily free the array.
01614     // we swap the first element with the root
01615     // we need to change the pointer in the operator node that has this element as a child
01616     // TODO: check performance
01617     for (unsigned int i = 0; i < curIndex; i++) {
01618         if (ar[i].l == ar) ar[i].l = st.top();
01619         if (ar[i].r == ar) ar[i].r = st.top();
01620     }
01621
01622     swap(ar[0], *st.top());
01623     return ar;
01624 }
01625
01626 int count_operators_cse_topdown (vector<WORD> &tree) {
01627     typedef unordered_set<NODE*, NodeHash, NodeEq> nodeset;
01628     nodeset ID;
01629
01630     // reserve lots of space, to prevent later rehashes
01631     // TODO: what if this is too large? make a parameter?
01632     ID.rehash(mcts_expr_score * 2);
01633
01634     int numinstr = 0;
01635
01636     NODE* root = buildTree(tree);
01637
01638     stack<NODE*> stack;

```

```

01639     stack.push(root);
01640     while (!stack.empty())
01641     {
01642         NODE* c = stack.top();
01643         stack.pop();
01644
01645         if (c->data[0] == SYMBOL) {
01646             if (c->data[3] > 1) {
01647                 std::pair<nodeset::iterator, bool> suc = ID.insert(c);
01648                 if (suc.second) { // new
01649                     if (c->data[3] == 2)
01650                         numinstr++;
01651                     else
01652                         numinstr += (int)floor(log(c->data[3])/log(2.0)) + popcount(c->data[3]) - 1;
01653                 }
01654             }
01655         } else {
01656             if (c->data[0] != SNUMBER) {
01657                 // operator
01658                 std::pair<nodeset::iterator, bool> suc = ID.insert(c);
01659                 if (suc.second) {
01660                     stack.push(c->r);
01661                     stack.push(c->l);
01662
01663                     // ignore OPER_COMMA
01664                     if (c->data[0] == OPER_MUL || c->data[0] == OPER_ADD)
01665                         numinstr++;
01666                 }
01667             }
01668         }
01669     }
01670
01671     //MesPrint ("*** [%s] Stopping CSEE", thetime_str().c_str());
01672     M_free(root, "CSE tree");
01673
01674     return numinstr;
01675 }
01676
01677 /*
01678 #] count_operators_cse_topdown :
01679 #[ simulated_annealing :
01680 */
01681 vector<WORD> simulated_annealing() {
01682     float minT = AO.Optimize.saMinT.fval;
01683     float maxT = AO.Optimize.saMaxT.fval;
01684     float T = maxT;
01685     float coolrate = pow(minT / maxT, 1 / (float)AO.Optimize.saIter);
01686
01687     GETIDENTITY;
01688
01689     // create a valid state where FACTORSYMBOL/SEPARATESYMBOL remains first
01690     vector<WORD> state = occurrence_order(optimize_expr, false);
01691     int startindex = 0;
01692     if ((state.size() > 0 && state[0] == SEPARATESYMBOL) || (state.size() > 1 && state[1] ==
FACTORSYMBOL)) startindex++;
01693     if (state.size() > 1 && state[1] == FACTORSYMBOL) startindex++;
01694
01695     my_random_shuffle(BHEAD state.begin() + startindex, state.end()); // start from random scheme
01696
01697     vector<WORD> tree = Horner_tree(optimize_expr, state);
01698     int curscore = count_operators_cse_topdown(tree);
01699
01700     // clean poly_vars, that are allocated by Horner_tree
01701     AN.poly_num_vars = 0;
01702     M_free(AN.poly_vars, "poly_vars");
01703
01704     std::vector<WORD> best = state; // best state
01705     int bestscore = curscore;
01706
01707     if (startindex == (int)state.size() || (int)state.size() - startindex < 2) {
01708         return best;
01709     }
01710
01711     for (int o = 0; o < AO.Optimize.saIter; o++) {
01712         int inda = iranf(BHEAD state.size() - startindex) + startindex;
01713         int indb = iranf(BHEAD state.size() - startindex) + startindex;
01714
01715         if (inda == indb) {
01716             continue;
01717         }
01718
01719         swap(state[inda], state[indb]); // swap works best for Horner
01720
01721         vector<WORD> tree = Horner_tree(optimize_expr, state);
01722         int newscore = count_operators_cse_topdown(tree);
01723
01724         // clean poly_vars, that are allocated by Horner_tree

```

```

01725     AN.poly_num_vars = 0;
01726     M_free(AN.poly_vars, "poly_vars");
01727
01728     if (newscore <= curscore || 2.0 * wranf(BHEAD0) / (float)(UWORD) (-1) < exp((curscore -
newscore) / T)) {
01729         curscore = newscore;
01730
01731         if (curscore < bestscore) {
01732             bestscore = curscore;
01733             best = state;
01734         }
01735     } else {
01736         swap(state[inda], state[indb]);
01737     }
01738
01739 #ifdef DEBUG_SA
01740     MesPrint("Score at step %d: %d", o, curscore);
01741 #endif
01742     T *= coolrate;
01743 }
01744
01745 #ifdef DEBUG_SA
01746     MesPrint("Simulated annealing score: %d", bestscore);
01747 #endif
01748
01749     return best;
01750 }
01751
01752 /*
01753     #] simulated_annealing :
01754     #[ printpstree :
01755 */
01756
01757 /*
01758 // print MCTS tree with LaTeX/pstricks (for analysis)
01759 void printpstree_rec (tree_node x, string pre="") {
01760
01761     if (x.num_visits==1) {
01762         MesPrint("%s\\TR{%d}", pre.c_str(), x.var);
01763     }
01764     else {
01765         MesPrint("%s\\pstree%s{\\TR{%d}}{", pre.c_str(),
01766             pre==" "? "[nodesep=0, levels=40]": "",
01767             x.var);
01768         for (int i=0; i<(int)x.chlds.size(); i++)
01769             if (x.chlds[i].num_visits>0)
01770                 printpstree_rec(x.chlds[i], pre+" ");
01771         MesPrint("%s", pre.c_str());
01772     }
01773 }
01774
01775 void printpstree () {
01776     // draw tree with pstricks
01777     MesPrint ("\\documentclass{article}");
01778     MesPrint ("\\usepackage{pstricks,pst-node,pst-tree,graphicx}");
01779     MesPrint ("\\begin{document}");
01780     MesPrint ("\\scalebox{0.02}{");
01781     printpstree_rec(mcts_root, " ");
01782     MesPrint ("}");
01783     MesPrint ("\\end{document}");
01784 }
01785 */
01786
01787 /*
01788     #] printpstree :
01789     #[ find_Horner_MCTS_expand_tree :
01790 */
01791
01792 /*
01793     #[ next_MCTS_scheme :
01794 */
01795
01796 // find a Horner scheme to be used for the next simulation
01797 inline static void next_MCTS_scheme (PHEAD vector<WORD> *porder, vector<WORD> *pscheme,
vector<tree_node > *ppath) {
01798
01799     vector<WORD> &order = *porder;
01800     vector<WORD> &schemev = *pscheme;
01801     vector<tree_node > &path = *ppath;
01802     int depth = 0, nchild0;
01803     float slide_down_factor = 1.0;
01804
01805     order.clear();
01806     path.clear();
01807
01808     // MCTS step I: select
01809     tree_node *select = &mcts_root;

```



```

01841     path.push_back(select);
01842     nchild0 = select->childs.size();
01843     while (select->childs.size() > 0) {
01844         // add virtual loss
01845         select->num_visits++;
01846         select->sum_results+=mcts_expr_score;
01847
01848         //-----
01849         switch ( AO.Optimize.mctsdecaymode ) {
01850             case 1: // Based on http://arxiv.org/abs/arXiv:1312.0841
01851                 slide_down_factor = 1.0-(1.0*AT.optimitimes)/(1.0*AO.Optimize.mctsnumexpand);
01852                 break;
01853             case 2: // This gives a bit more cleanup time at the end.
01854                 if ( 2*AT.optimitimes < AO.Optimize.mctsnumexpand ) {
01855                     slide_down_factor = 1.0*(AO.Optimize.mctsnumexpand-2*AT.optimitimes);
01856                     slide_down_factor /= 1.0*AO.Optimize.mctsnumexpand;
01857                 }
01858                 else {
01859                     slide_down_factor = 0.0001;
01860                 }
01861                 break;
01862             case 3: // depth dependent factor combined with case 1
01863                 float dd = 1.0-(1.0*depth)/(1.0*nchild0);
01864                 slide_down_factor = 1.0-(1.0*AT.optimitimes)/(1.0*AO.Optimize.mctsnumexpand);
01865                 if ( dd <= 0.000001 ) slide_down_factor = 1.0;
01866                 else slide_down_factor /= dd;
01867                 if ( slide_down_factor > 1.0 ) slide_down_factor = 1.0;
01868                 break;
01869         }
01870         //-----
01871
01872         #ifdef DEBUG_MCTS
01873         MesPrint("select %d",select->var);
01874         #endif
01875
01876         // find most-promising node
01877         double best=0;
01878         tree_node *next=NULL;
01879         for (vector<tree_node>::iterator p=select->childs.begin(); p<select->childs.end(); p++) {
01880             double score;
01881             if (p->num_visits >= 1) {
01882
01883                 // there are results calculated, so select with the UCT formula
01884                 score = mcts_expr_score / (p->sum_results/p->num_visits) +
01885                 //-----
01886                 slide_down_factor *
01887                 //-----
01888                 2 * AO.Optimize.mctsconstant.fval * sqrt(2*log(select->num_visits) /
01889                 p->num_visits);
01890
01891                 #ifdef DEBUG_MCTS
01892                 printf("%d: %.2lf [x=%.2lf n=%d fin=%i]\n",p->var,score,mcts_expr_score /
01893                 (p->sum_results/p->num_visits),
01894                 p->num_visits,p->finished?1:0);
01895                 fflush(stdout);
01896                 #endif
01897             }
01898             else {
01899                 // no results yet, so select this node by setting score=infinite
01900                 score = 1e100;
01901                 #ifdef DEBUG_MCTS
01902                 printf("%d: inf\n",p->var); fflush(stdout);
01903                 #endif
01904             }
01905
01906             // update best candidate
01907             if (!p->finished && score>best) {
01908                 best=score;
01909                 next=&*p;
01910             }
01911         }
01912
01913         // if no node is found, this node is finished
01914         if (next==NULL) {
01915             select->finished=true;
01916             break;
01917         }
01918
01919         // traverse down the tree
01920         select = next;
01921         path.push_back(select);
01922         order.push_back(select->var);
01923         depth++;
01924     }
01925     // MCTS step II: expand

```

```

01926
01927 #ifdef DEBUG_MCTS
01928     MesPrint("expand %d",select->var);
01929 #endif
01930
01931     // variables used so far
01932     set<WORD> var_used;
01933
01934     for (int i=0; i<(int)order.size(); i++)
01935         var_used.insert(ABS(order[i])-1);
01936
01937     // if this a new node, create node and add children
01938     if (!select->finished && select->childs.size()==0) {
01939         tree_node new_node(select->var);
01940         int sign = (order.size() == 0) ? 1 : SGN(order.back());
01941         for (int i=0; i<(int)mcts_vars.size(); i++)
01942             if (!var_used.count(mcts_vars[i])) {
01943                 new_node.childs.push_back(tree_node(sign*(mcts_vars[i]+1)));
01944                 if (AO.Optimize.hornerdirection==O_FORWARDANDBACKWARD)
01945                     new_node.childs.push_back(tree_node(-sign*(mcts_vars[i]+1)));
01946             }
01947         my_random_shuffle(BHEAD new_node.childs.begin(), new_node.childs.end());
01948
01949         // here locking is necessary, since operator=(tree_node) is a
01950         // non-atomic operation (using pointers makes this lock obsolete)
01951         LOCK(optimize_lock);
01952         *select = new_node;
01953         UNLOCK(optimize_lock);
01954     }
01955     // set finished if necessary
01956     if (select->childs.size()==0)
01957         select->finished = true;
01958
01959     // add virtual loss of number of operators in original expression
01960     select->num_visits++;
01961     select->sum_results+=mcts_expr_score;
01962
01963     // MCTS step III: simulation
01964
01965     // create complete Horner scheme
01966     deque<WORD> scheme;
01967
01968     for (int i=0; i<(int)mcts_vars.size(); i++)
01969         if (!var_used.count(mcts_vars[i]))
01970             scheme.push_back(mcts_vars[i]);
01971     my_random_shuffle(BHEAD scheme.begin(), scheme.end());
01972
01973     for (int i=(int)order.size()-1; i>=0; i--) {
01974         if (order[i] > 0)
01975             scheme.push_front(order[i]-1);
01976         else
01977             scheme.push_back(-order[i]-1);
01978     }
01979
01980     // add FACTORSYMBOL/SEPARATESYMBOL is necessary
01981     if (mcts_factorized)
01982         scheme.push_front(FACTORSYMBOL);
01983     if (mcts_separated)
01984         scheme.push_front(SEPARATESYMBOL);
01985
01986     // Horner scheme as a vector
01987     schemev = vector<WORD>(scheme.begin(), scheme.end());
01988
01989 }
01990
01991 /*
01992     #] next_MCTS_scheme :
01993     #[ try_MCTS_scheme :
01994 */
01995
01996 // count the number of operators in the given Horner scheme
01997 inline static void try_MCTS_scheme (PHEAD const vector<WORD> &scheme, int *pnum_oper) {
01998
01999     // do Horner, CSE and count the number of operators
02000     vector<WORD> tree = Horner_tree(optimize_expr, scheme);
02001     //vector<WORD> instr = generate_instructions(tree, true);
02002     //int num_oper = count_operators(instr);
02003     //int num_oper2 = count_operators_cse(tree);
02004     //int num_oper2 = count_operators_cse_topdown(tree);
02005     //MesPrint("%d %d", num_oper, num_oper2);
02006
02007     int num_oper = count_operators_cse_topdown(tree);
02008
02009     // clean poly_vars, that is allocated by Horner_tree
02010     AN.poly_num_vars = 0;
02011     M_free(AN.poly_vars,"poly_vars");
02012

```

```

02013     *pnum_oper = num_oper;
02014
02015 }
02016
02017 /*
02018     #] try_MCTS_scheme :
02019     #[ update_MCTS_scheme :
02020 */
02021
02022 // update the best score and the search tree
02023 inline static void update_MCTS_scheme (int num_oper, const vector<WORD> &scheme, vector<tree_node *>
    *ppath) {
02024
02025     vector<tree_node *> &path = *ppath;
02026
02027     // update the (global) list of best Horner scheme
02028     if ((int)mcts_best_schemes.size() < AO.Optimize.mctsnumkeep ||
02029         (--mcts_best_schemes.end()->first > num_oper) {
02030         // here locking is necessary, for otherwise best_schemes may
02031         // become corrupted; lock can be prevented if each thread keeps
02032         // track of it's own list and those lists are merged in the end,
02033         // but this seems not useful to implement
02034         LOCK(optimize_lock);
02035         mcts_best_schemes.insert(make_pair(num_oper,scheme));
02036         if ((int)mcts_best_schemes.size() > AO.Optimize.mctsnumkeep)
02037             mcts_best_schemes.erase(--mcts_best_schemes.end());
02038         UNLOCK(optimize_lock);
02039     }
02040
02041     // MCTS step IV: backpropagate
02042
02043     // add number of operator and subtract mcts_expr_score, which
02044     // behaves as a "virtual loss"
02045     for (vector<tree_node *>::iterator p=path.begin(); p<path.end(); p++)
02046         (*p)->sum_results += num_oper - mcts_expr_score;
02047
02048 }
02049
02050 /*
02051     #] update_MCTS_scheme :
02052 */
02053
02054 void find_Horner_MCTS_expand_tree () {
02055
02056     #ifdef DEBUG
02057         MesPrint ("*** [%s, w=%w] CALL: find_Horner_MCTS_expand_tree", thetime_str().c_str());
02058     #endif
02059
02060     GETIDENTITY;
02061
02062     // the order for the Horner scheme up to the selected node, with signs
02063     // indicating forward or backward.
02064     vector<WORD> order;
02065
02066     // complete Horner scheme
02067     vector<WORD> scheme;
02068
02069     // path to the selected node
02070     vector<tree_node *> path;
02071
02072     // the number of operations obtained by the simulation
02073     int num_oper;
02074
02075     next_MCTS_scheme(BHEAD &order, &scheme, &path);
02076     try_MCTS_scheme(BHEAD scheme, &num_oper);
02077     #ifdef DEBUG_MCTS
02078         // Actually "order" is needed only for this debug output.
02079         MesPrint ("[%a] -> [%a] -> %d", order.size(), &order[0], scheme.size(), &scheme[0], num_oper);
02080     #endif
02081     update_MCTS_scheme(num_oper, scheme, &path);
02082
02083     #ifdef DEBUG
02084         MesPrint ("*** [%s, w=%w] DONE: find_Horner_MCTS_expand_tree(%a-> %d)",
02085             thetime_str().c_str(), scheme.size(), &scheme[0], num_oper);
02086     #endif
02087
02088 }
02089
02090 /*
02091     #] find_Horner_MCTS_expand_tree :
02092     #[ PF_find_Horner_MCTS_expand_tree :
02093 */
02094 #ifdef WITHMPI
02095
02096 // To remember which task is assigned to each slave. A task is represented by
02097 // a pair of a Horner scheme and the selected path for the scheme.
02098 // The index range is from 1 to PF.numtasks-1.

```

```

02099 vector<pair<vector<WORD>, vector<tree_node *> > > PF_opt_MCTS_tasks;
02100
02101 // Initialization.
02102 void PF_find_Horner_MCTS_expand_tree_master_init () {
02103     PF_opt_MCTS_tasks.resize(PF.numtasks);
02104     for (int i = 1; i < PF.numtasks; i++) {
02105         pair<vector<WORD>, vector<tree_node *> > &p = PF_opt_MCTS_tasks[i];
02106         p.first.clear();
02107         p.second.clear();
02108     }
02109 }
02110
02111 }
02112
02113 // Wait for an idle slave and return the process number.
02114 int PF_find_Horner_MCTS_expand_tree_master_next () {
02115     // Find an idle slave.
02116     int next;
02117     PF_Receive(PF_ANY_SOURCE, PF_OPT_MCTS_MSGTAG, &next, NULL);
02118
02119     // Check if the slave had a task.
02120     pair<vector<WORD>, vector<tree_node *> > &p = PF_opt_MCTS_tasks[next];
02121     if (!p.first.empty()) {
02122         // If so, update the result.
02123         int num_oper;
02124         PF_Unpack(&num_oper, 1, PF_INT);
02125         update_MCTS_scheme(num_oper, p.first, &p.second);
02126
02127         // Clear the task.
02128         p.first.clear();
02129         p.second.clear();
02130     }
02131     return next;
02132 }
02133
02134 }
02135
02136 // The main function on the master.
02137 void PF_find_Horner_MCTS_expand_tree_master () {
02138     #ifdef DEBUG
02139         MesPrint ("*** [%s, w=%w] CALL: PF_find_Horner_MCTS_expand_tree_master", thetime_str().c_str());
02140     #endif
02141     vector<WORD> order;
02142     vector<WORD> scheme;
02143     vector<tree_node *> path;
02144
02145     next_MCTS_scheme(BHEAD &order, &scheme, &path);
02146
02147     // Find an idle slave.
02148     int next = PF_find_Horner_MCTS_expand_tree_master_next();
02149
02150     // Send a new task to the slave.
02151     PF_PrepareLongSinglePack();
02152     int len = scheme.size();
02153     PF_LongSinglePack(&len, 1, PF_INT);
02154     PF_LongSinglePack(&scheme[0], len, PF_WORD);
02155     PF_LongSingleSend(next, PF_OPT_MCTS_MSGTAG);
02156
02157     // Remember the task.
02158     pair<vector<WORD>, vector<tree_node *> > &p = PF_opt_MCTS_tasks[next];
02159     p.first = scheme;
02160     p.second = path;
02161
02162     #ifdef DEBUG
02163         MesPrint ("*** [%s, w=%w] DONE: PF_find_Horner_MCTS_expand_tree_master", thetime_str().c_str());
02164     #endif
02165 }
02166
02167 // Wait for all the slaves to finish their tasks.
02168 void PF_find_Horner_MCTS_expand_tree_master_wait () {
02169     #ifdef DEBUG
02170         MesPrint ("*** [%s, w=%w] CALL: PF_find_Horner_MCTS_expand_tree_master_wait",
02171             thetime_str().c_str());
02172     #endif
02173
02174     // Wait for all the slaves.
02175     for (int i = 1; i < PF.numtasks; i++) {
02176         int next = PF_find_Horner_MCTS_expand_tree_master_next();
02177         // Send a null task.
02178         PF_PrepareLongSinglePack();
02179         int len = 0;
02180         PF_LongSinglePack(&len, 1, PF_INT);
02181     }

```

```

02185         PF_LongSingleSend(next, PF_OPT_MCTS_MSGTAG);
02186     }
02187
02188 #ifdef DEBUG
02189     MesPrint ("*** [%s, w=%w] DONE: PF_find_Horner_MCTS_expand_tree_master_wait",
02190             thetime_str().c_str());
02191 #endif
02192 }
02193
02194 // The main function on the slaves.
02195 void PF_find_Horner_MCTS_expand_tree_slave () {
02196
02197 #ifdef DEBUG
02198     MesPrint ("*** [%s, w=%w] CALL: PF_find_Horner_MCTS_expand_tree_slave", thetime_str().c_str());
02199 #endif
02200
02201     vector<WORD> scheme;
02202
02203     {
02204         // Send the first message to the master, which indicates I am idle.
02205         PF_PreparePack();
02206         int dummy = 0;
02207         PF_Pack(&dummy, 1, PF_INT);
02208         PF_Send(MASTER, PF_OPT_MCTS_MSGTAG);
02209     }
02210
02211     for (;;) {
02212         // Get a task from the master.
02213         PF_LongSingleReceive(MASTER, PF_OPT_MCTS_MSGTAG, NULL, NULL);
02214
02215         // Length of the task.
02216         int len;
02217         PF_LongSingleUnpack(&len, 1, PF_INT);
02218
02219         // No task remains.
02220         if (len == 0) break;
02221
02222         // Perform the given task.
02223         scheme.resize(len);
02224         PF_LongSingleUnpack(&scheme[0], len, PF_WORD);
02225         int num_oper;
02226         try_MCTS_scheme(scheme, &num_oper);
02227
02228         // Send the result to the master.
02229         PF_PreparePack();
02230         PF_Pack(&num_oper, 1, PF_INT);
02231         PF_Send(MASTER, PF_OPT_MCTS_MSGTAG);
02232     }
02233
02234 #ifdef DEBUG
02235     MesPrint ("*** [%s, w=%w] DONE: PF_find_Horner_MCTS_expand_tree_slave", thetime_str().c_str());
02236 #endif
02237 }
02238
02239 #endif
02240
02241 /*
02242 #] PF_find_Horner_MCTS_expand_tree :
02243 #[ find_Horner_MCTS :
02244 */
02245
02246 //vector<vector<WORD> > find_Horner_MCTS () {
02247 void find_Horner_MCTS () {
02248
02249 #ifdef WITHMPI
02250     if (PF.me != MASTER) {
02251         if (PF.numtasks <= 1) return;
02252         PF_find_Horner_MCTS_expand_tree_slave();
02253         return;
02254     }
02255 #endif
02256
02257 #ifdef DEBUG
02258     MesPrint ("*** [%s, w=%w] CALL: find_Horner_MCTS", thetime_str().c_str());
02259 #endif
02260
02261     GETIDENTITY;
02262
02263     LONG start_time = TimeWallClock(1);
02264
02265     // initialize the used global variables
02266     mcts_expr_score = count_operators(optimize_expr);
02267     mcts_root = tree_node();
02268
02269     // extract all symbols from the expression
02270     set<WORD> var_set;

```

```

02279     for (WORD *t=optimize_expr; *t!=0; t+=*t)
02280         if (t[1] == SYMBOL)
02281             for (int i=3; i<t[2]; i+=2)
02282                 var_set.insert(t[i]);
02283
02284     // check for factorized/separated expression and make sure that
02285     // FACTORSYMBOL/SEPARATESYMBOL isn't included in the MCTS
02286     mcts_factorized = var_set.count(FACTORSYMBOL);
02287     if (mcts_factorized) var_set.erase(FACTORSYMBOL);
02288     mcts_separated = var_set.count(SEPARATESYMBOL);
02289     if (mcts_separated) var_set.erase(SEPARATESYMBOL);
02290
02291     mcts_vars = vector<WORD>(var_set.begin(), var_set.end());
02292     optimize_num_vars = (int)mcts_vars.size();
02293     // initialize MCTS tree root
02294     for (int i=0; i<(int)mcts_vars.size(); i++) {
02295         if (AO.Optimize.hornerdirection != O_BACKWARD)
02296             mcts_root.childs.push_back(tree_node(+ (mcts_vars[i]+1)));
02297         if (AO.Optimize.hornerdirection != O_FORWARD)
02298             mcts_root.childs.push_back(tree_node(- (mcts_vars[i]+1)));
02299     }
02300     my_random_shuffle(BHEAD mcts_root.childs.begin(), mcts_root.childs.end());
02301
02302     #if defined(WITHMPI)
02303         PF_find_Horner_MCTS_expand_tree_master_init();
02304     #endif
02305
02306     // initialize a potential variable mctsconstant scheme.
02307     AT.optentimes = 0;
02308
02309     // call expand_tree until it is called "mctsnunexpand" times, the
02310     // time limit is reached or the tree is fully finished
02311     for (int times=0; times<AO.Optimize.mctsnunexpand && !mcts_root.finished &&
02312          (AO.Optimize.mctsttimelimit==0 || (TimeWallClock(1)-start_time)/100 <
02313          AO.Optimize.mctsttimelimit);
02314          times++) {
02315         AT.optentimes = times;
02316         // call expand_tree routine depending on threading mode
02317         #if defined(WITHPTHREADS)
02318             if (AM.totalnumberofthreads > 1)
02319                 find_Horner_MCTS_expand_tree_threaded();
02320             else
02321                 #elif defined(WITHMPI)
02322                     if (PF.numtasks > 1)
02323                         PF_find_Horner_MCTS_expand_tree_master();
02324                     else
02325                         #endif
02326                             find_Horner_MCTS_expand_tree();
02327
02328         // if TForm or ParForm, wait for everyone to finish
02329         #ifdef WITHPTHREADS
02330             MasterWaitAll();
02331         #endif
02332         #ifdef WITHMPI
02333             PF_find_Horner_MCTS_expand_tree_master_wait();
02334         #endif
02335         #ifdef DEBUG
02336             MesPrint ("*** [%s, w=%w] DONE: find_Horner_MCTS", thetime_str().c_str());
02337         #endif
02338     }
02339 }
02340
02341 /*
02342     #] find_Horner_MCTS :
02343     #[ merge_operators :
02344 */
02345
02403 vector<WORD> merge_operators (const vector<WORD> &all_instr, bool move_coeff) {
02404
02405     #ifdef DEBUG_MORE
02406         MesPrint ("*** [%s, w=%w] CALL: merge_operators", thetime_str().c_str());
02407     #endif
02408
02409     // get starting positions of instructions
02410     vector<const WORD *> instr;
02411     const WORD *tbegin = &*all_instr.begin();
02412     const WORD *tend = tbegin+all_instr.size();
02413     // copy all instructions to temp space. There will be n of them in instr.
02414     for (const WORD *t=tbegin; t!=tend; t+=*(t+2)) {
02415         instr.push_back(t);
02416     }
02417     int n = instr.size();
02418
02419     // find parents and number of parents of instructions
02420     vector<int> par(n), numpar(n,0);
02421     for (int i=0; i<n; i++) par[i]=i;

```

```

02422
02423     for (int i=0; i<n; i++) {
02424         for (const WORD *t=instr[i]+OPTHEAD; *t!=0; t+=*t) {
02425             if ( (*t+1)==EXTRASymbol && *t!=1+ABS(*t+*t-1)) ) {
02426                 // extra symbol t[3] is referred to in instr i.
02427                 par[*t+3]=i;
02428                 numpar[*t+3]++;
02429
02430                 // if coefficient isn't +/-1 or power > 1, increase numpar,
02431                 // so that this is not merged
02432                 if (*t+*t-3)!=1 || *t+*t-2)!=1 || ABS(*t+*t-1)!=3) numpar[*t+3]++;
02433                 if (*t+4)>1) numpar[*t+3]++;
02434             }
02435         }
02436     }
02437     // determine which instructions to merge
02438     stack<int> s;
02439     s.push(n-1);
02440     vector<bool> vis(n,false);
02441
02442     while (!s.empty()) {
02443         int i=s.top(); s.pop();
02444         if (vis[i]) continue;
02445         vis[i]=true;
02446
02447         for (const WORD *t=instr[i]+OPTHEAD; *t!=0; t+=*t)
02448             if ( (*t+1)==EXTRASymbol && *t!=1+ABS(*t+*t-1)) )
02449                 s.push(*t+3);
02450
02451         // condition: one parent and equal operator as parent
02452         if (numpar[i]==1 && *(instr[i+1])==*(instr[par[i]+1]))
02453             par[i] = par[par[i]]; // The expr into which we subst par[i] to get i
02454         else
02455             par[i] = i;
02456     }
02457
02458     // merge instructions into new instructions
02459     vector<WORD> newinstr;
02460
02461     // stack of new expressions, all 0-indexed
02462     stack<WORD> new_expr;
02463     new_expr.push(n-1);
02464     vis = vector<bool>(n,false);
02465
02466     // skip empty equations (might be introduced by greedy optimizations)
02467     vector<bool> skip(n,false), skipcoeff(n,false);
02468     for (int i=0; i<n; i++)
02469         if (*(instr[i]+OPTHEAD) == 0) skip[i]=skipcoeff[i]=true;
02470
02471     // for renumbering merged parents
02472     vector<int> renum_par(n);
02473     for (int i=0; i<n; i++)
02474         renum_par[i]=i;
02475
02476     while (!new_expr.empty()) {
02477         int x = new_expr.top(); new_expr.pop();
02478         if (vis[x]) continue;
02479         vis[x] = true;
02480
02481         // find all instructions with parent=x and copy their arguments
02482         // into a new expression
02483         bool first_copy=true;
02484         int lenidx = newinstr.size()+2;
02485
02486         // 1-indexed, since signs may occur
02487         stack<WORD> this_expr;
02488         this_expr.push(x+1);
02489
02490         while (!this_expr.empty()) {
02491             // pop from stack, determine expr.nr and sign
02492             int i = this_expr.top(); this_expr.pop();
02493             int sign = SGN(i);
02494             i = ABS(i)-1;
02495             for (const WORD *t=instr[i]+OPTHEAD; *t!=0; t+=*t) { // terms in i
02496                 // don't copy a term if:
02497                 // (1) skip=true, since then it's merged into the parent
02498                 // (2) extrasymbol with parent=x, because its children should be copied
02499                 // (3) coefficient with skipcoeff=true, since it's already copied
02500                 bool copy = !skip[i];
02501                 if (*t!=1+ABS(*t+*t-1)) && *t+1==EXTRASymbol) {
02502                     if (par[*t+3] == x) {
02503                         // parent of term refers to x. we push it with its sign if no skip is true
02504                         // and the sign of the expr.
02505                         this_expr.push(sign * (skip[i]||skipcoeff[i] ? 1 : SGN(*t+*t-1))) *
02506                             (1+*t+3));
02507                     if (*(instr[i+1]) == OPER_MUL) sign=1;

```

```

02508         copy=false;
02509     }
02510     else {
02511         new_expr.push(*(t+3));
02512     }
02513 }
02514
02515 if (*t == 1+ABS(*(t+t-1)) && skipcoeff[i]) {
02516     copy=false;
02517 }
02518
02519 if (copy) {
02520     // first term, so add header
02521     if (first_copy) {
02522         newinstr.push_back(renum_par[x]); // expr.nr.
02523         newinstr.push_back(*(instr[x]+1)); // operator
02524         newinstr.push_back(3); // length OPTHEAD?
02525         first_copy=false;
02526     }
02527
02528     // copy term and adjust sign
02529     int thislenidx = newinstr.size();
02530     newinstr.insert(newinstr.end(), t, t+t); // Put the whole term in newinstr
02531     newinstr.back() *= sign;
02532     if (*(instr[i]+1) == OPER_MUL) sign=1;
02533     newinstr[thislenidx] += *t;
02534
02535     // check for moving coefficients up
02536     // necessary condition: MUL-expression with 1 parent
02537     if (move_coeff && *t!=1+ABS(*(t+t-1)) && *(instr[i]+1)!=OPER_COMMA &&
02538         *(t+1)==EXTRASymbol && numpar[*(t+3)]==1 && *(instr[*(t+3)+1]==OPER_MUL)
02539 {
02540
02541     // coefficient is always the first term (that's how Horner+generate works)
02542     const WORD *t1 = instr[*(t+3)]+OPTHEAD;
02543     const WORD *t2 = t1+t1;
02544
02545     if (*t1 == 1+ABS(*(t1+t1-1))) {
02546         // t1 pointer to a coefficient, so move it
02547
02548         // remove old coefficient of 1
02549         WORD *t3 = &newinstr.end(); //
02550         int sign2 = SGN(t3[-1]); //
02551         newinstr.erase(newinstr.end()-3, newinstr.end());
02552         // count number of arguments; iff it is 2 move the (extra)symbol too
02553         int numargs=0;
02554         for (const WORD *tt=t1; *tt!=0; tt+=*tt) {
02555             numargs++;
02556         }
02557         if (numargs==2 && *(t2+4)==1) {
02558             // replace (extra)symbol
02559             newinstr[newinstr.size()-4] = *(t2+1);
02560             newinstr[newinstr.size()-3] = *(t2+2);
02561             newinstr[newinstr.size()-2] = *(t2+3);
02562             newinstr[newinstr.size()-1] = *(t2+4);
02563             sign2 *= SGN(*(t2+t2-1)); // was t2[7]
02564
02565             // ignore this expression from now on
02566             skip[*(t+3)]=true;
02567             if (*(t2+1)==EXTRASymbol)
02568                 renum_par[*(t+3)] = *(t2+3);
02569         }
02570         else {
02571             // otherwise, ignore coefficient from now on
02572             // we need to collect the signs of the terms
02573             // first and set them to one. This was forgotten
02574             // before. Gave occasional errors.
02575             if ( numargs > 2 || ( numargs == 2 && t2[4] > 1 ) ) {
02576                 for (WORD *tt=(WORD *)t2; *tt!=0; tt+=*tt) {
02577                     if ( tt[*tt-1] < 0 ) {
02578                         tt[*tt-1] = -tt[*tt-1];
02579                         sign2 = -sign2;
02580                     }
02581                 }
02582             }
02583             skipcoeff[*(t+3)]=true;
02584         }
02585
02586         // add new coefficient
02587         newinstr.insert(newinstr.end(), t1+1, t1+t1);
02588         newinstr.back() *= sign2;
02589         newinstr[thislenidx] += ABS(newinstr.back()) - 3;
02590         newinstr[thislenidx] += ABS(newinstr.back()) - 3;
02591     }
02592 }
02593 }

```



```

02594     }
02595
02596     // if something has been copied, add trailing zero
02597     if (!first_copy) {
02598         newinstr.push_back(0);
02599         newinstr[lenidx]++;
02600     }
02601 }
02602
02603 // renumber the expressions to 0,1,2,...; only keep expressions with
02604 // skip=false which are their own parent after a renumbering in case
02605 // of moved coefficients
02606
02607 // find renumber scheme
02608 vector<int> renum(n,-1);
02609 int next=0;
02610 for (int i=0; i<n; i++)
02611     if (!skip[i] && renum_par[par[i]]==i) renum[renum_par[i]]=next++;
02612
02613 // find new instruction index
02614 tbegin = &*newinstr.begin();
02615 tend = tbegin+newinstr.size();
02616 for (const WORD *t=tbegin; t!=tend; t+=*(t+2))
02617     instr[renum[*t]] = t;
02618
02619 // renumbering expressions and sort them lexicographically (in
02620 // new_instr they are in the preorder of the expression tree)
02621 vector<WORD> sortinstr;
02622 for (int i=0; i<next; i++) {
02623     int idx = sortinstr.size();
02624     sortinstr.insert(sortinstr.end(), instr[i], instr[i]+*(instr[i]+2));
02625
02626     sortinstr[idx] = renum[sortinstr[idx]];
02627
02628     // renumber content of an expression
02629     for (WORD *t2=&sortinstr[idx]+3; *t2!=0; t2+=*t2)
02630         if (*t2!=1+ABS(*t2+*t2-1)) && *(t2+1)==EXTRASYMBOL
02631             *(t2+3) = renum[*t2+3];
02632 }
02633
02634 #ifdef DEBUG_MORE
02635     MesPrint ("*** [%s, w=%w] DONE: merge_operators", thetime_str().c_str());
02636 #endif
02637
02638     return sortinstr;
02639 }
02640
02641 /*
02642  #] merge_operators :
02643  #[ class Optimization :
02644  */
02645
02646 class optimization {
02647 public:
02648     int type, arg1, arg2, improve;
02649     vector<WORD> coeff;
02650     vector<int> eqnidxs;
02651
02652     bool operator< (const optimization &a) const {
02653         if (arg1 != a.arg1) return arg1 < a.arg1;
02654         if (arg2 != a.arg2) return arg2 < a.arg2;
02655         if (type != a.type) return type < a.type;
02656         return coeff < a.coeff;
02657     }
02658 };
02659
02660 /*
02661  #] class Optimization :
02662  #[ find_optimizations :
02663  */
02664
02665 vector<optimization> find_optimizations (const vector<WORD> &instr) {
02666
02667     #ifdef DEBUG_GREEDY
02668         MesPrint ("*** [%s, w=%w] CALL: find_optimizations", thetime_str().c_str());
02669     #endif
02670
02671     // #[ Startup :
02672     // the resulting vector of optimizations
02673     vector<optimization> res;
02674
02675     // a map to count the improvement of an optimization; the
02676     // improvement is stored as a vector<int> with equation numbers
02677     map<optimization, vector<int> > cnt;
02678
02679     // a map to identify coefficients
02680     map<vector<int>,int> idx_coeff;

```

```

02714
02715     const WORD *ebegin = &instr.begin();
02716     const WORD *eend = ebegin+instr.size();
02717
02718     for (int optim_type=0; optim_type<=4; optim_type++) {
02719
02720         cnt.clear();
02721         idx_coeff.clear();
02722
02723         optimization optim;
02724         optim.type = optim_type;
02725         // #] Startup :
02726
02727         // #[ type 0 : find optimizations of the form z=x^n (optim.type==0)
02728         if (optim_type == 0) {
02729             for (const WORD *e=ebegin; e!=eend; e+=*(e+2)) {
02730                 if (*(e+1) != OPER_MUL) continue;
02731                 for (const WORD *t=e+3; *t!=0; t+=*t) {
02732                     if (*t == ABS(*(t+*t-1))+1) continue;
02733                     if (*(t+4) > 1) {
02734                         optim.arg1 = (*(t+1)==SYMBOL ? 1 : -1) * (*(t+3) + 1);
02735                         optim.arg2 = *(t+4);
02736                         cnt[optim].push_back(e-ebegin);
02737                     }
02738                 }
02739             }
02740         }
02741         // #] type 0 :
02742         // #[ type 1 : find optimizations of the form z=x*y (optim.type==1)
02743         if (optim_type == 1) {
02744             for (const WORD *e=ebegin; e!=eend; e+=*(e+2)) {
02745                 if (*(e+1) != OPER_MUL) continue;
02746
02747                 for (const WORD *t1=e+3; *t1!=0; t1+=*t1) {
02748                     if (*t1 == ABS(*(t1+*t1-1))+1) continue;
02749                     int x1 = (*(t1+1)==SYMBOL ? 1 : -1) * (*(t1+3) + 1);
02750
02751                     for (const WORD *t2=t1+*t1; *t2!=0; t2+=*t2) {
02752                         if (*t2 == ABS(*(t2+*t2-1))+1) continue;
02753                         int x2 = (*(t2+1)==SYMBOL ? 1 : -1) * (*(t2+3) + 1);
02754
02755                         if (*(t1+4) == *(t2+4)) {
02756                             optim.arg1 = x1;
02757                             optim.arg2 = x2;
02758                             if (optim.arg1 > optim.arg2)
02759                                 swap(optim.arg1, optim.arg2);
02760
02761                             if (*(t1+4) == 1)
02762                                 cnt[optim].push_back(e-ebegin);
02763                             else {
02764                                 // E=x^n*y^n -> z=x*y; E=z^n is double improvement
02765                                 cnt[optim].push_back(e-ebegin);
02766                                 cnt[optim].push_back(e-ebegin);
02767                             }
02768                         }
02769                     }
02770                 }
02771             }
02772         }
02773         // #] type 1 :
02774         // #[ type 2 : find optimizations of the form z=c*x (optim.type==2)
02775         if (optim_type == 2) {
02776             for (const WORD *e=ebegin; e!=eend; e+=*(e+2)) {
02777
02778                 if (*(e+1) == OPER_ADD) {
02779                     // in ADD-equation
02780
02781                     for (const WORD *t=e+3; *t!=0; t+=*t) {
02782                         if (*t == ABS(*(t+*t-1))+1) continue;
02783
02784                         if (*(t+4)==1) {
02785                             if (ABS(*(t+*t-1))==3 && *(t+*t-2)==1 && *(t+*t-3)==1) continue;
02786
02787                             optim.coeff = vector<WORD>(t+*t-ABS(*(t+*t-1)), t+*t);
02788                             optim.coeff.back() = ABS(optim.coeff.back());
02789
02790                             optim.arg1 = (*(t+1)==SYMBOL ? 1 : -1) * (*(t+3) + 1);
02791                             optim.arg2 = 0;
02792
02793                             cnt[optim].push_back(e-ebegin);
02794                         }
02795                     }
02796                 }
02797                 else if (*(e+1) == OPER_MUL) {
02798                     // in MUL-equation
02799                     optim.coeff.clear();
02800

```

```

02801         for (const WORD *t=e+3; *t!=0; t+=*t)
02802             if (*t == ABS(* (t+*t-1))+1) {
02803                 if (ABS(* (t+*t-1))==3 && * (t+*t-2)==1 && * (t+*t-3)==1) continue;
02804                 optim.coeff = vector<WORD>(t+*t-ABS(* (t+*t-1)), t+*t);
02805                 optim.coeff.back() = ABS(optim.coeff.back());
02806             }
02807
02808         if (!optim.coeff.empty())
02809             for (const WORD *t=e+3; *t!=0; t+=*t) {
02810                 if (*t == ABS(* (t+*t-1))+1) continue;
02811                 if (* (t+4) != 1) continue;
02812                 optim.arg1 = (* (t+1)==SYMBOL ? 1 : -1) * (* (t+3) + 1);
02813                 optim.arg2 = 0;
02814                 cnt[optim].push_back(e-ebegin);
02815             }
02816     }
02817 }
02818 }
02819 // #] type 2 :
02820 // #[ type 3 : find optimizations of the form z=x+c (optim.type==3)
02821     if (optim_type == 3) {
02822         for (const WORD *e=ebegin; e!=eend; e+=*(e+2)) {
02823
02824             if (* (e+1) != OPER_ADD) continue;
02825
02826             optim.coeff.clear();
02827
02828             for (const WORD *t=e+3; *t!=0; t+=*t)
02829                 if (*t == ABS(* (t+*t-1))+1) {
02830                     optim.coeff = vector<WORD>(t+*t-ABS(* (t+*t-1)), t+*t);
02831                 }
02832
02833             if (!optim.coeff.empty()) {
02834                 for (const WORD *t=e+3; *t!=0; t+=*t) {
02835                     if (*t == ABS(* (t+*t-1))+1) continue;
02836                     if (ABS(* (t+*t-1))!=3 || * (t+*t-2)!=1 || * (t+*t-3)!=1) continue;
02837                     if (* (t+*t-1)==-3) optim.coeff.back() *= -1;
02838
02839                     optim.arg1 = (* (t+1)==SYMBOL ? 1 : -1) * (* (t+3) + 1);
02840                     optim.arg2 = 0;
02841                     cnt[optim].push_back(e-ebegin);
02842                     if (* (t+*t-1)==-3) optim.coeff.back() *= -1;
02843                 }
02844             }
02845         }
02846     }
02847 // #] type 3 :
02848 // #[ type 4,5 : find optimizations of the form z=x+y or z=x-y (optim.type==4 or 5)
02849     if (optim_type == 4) {
02850         for (const WORD *e=ebegin; e!=eend; e+=*(e+2)) {
02851             if (* (e+1) != OPER_ADD) continue;
02852
02853             for (const WORD *t1=e+3; *t1!=0; t1+=*t1) {
02854                 if (*t1 == ABS(* (t1+*t1-1))+1) continue;
02855                 int x1 = (* (t1+1)==SYMBOL ? 1 : -1) * (* (t1+3) + 1);
02856
02857                 for (const WORD *t2=t1+*t1; *t2!=0; t2+=*t2) {
02858                     if (*t2 == ABS(* (t2+*t2-1))+1) continue;
02859                     int x2 = (* (t2+1)==SYMBOL ? 1 : -1) * (* (t2+3) + 1);
02860
02861                     int sign1 = SGN(* (t1+*t1-1));
02862                     int sign2 = SGN(* (t2+*t2-1));
02863
02864                     if (BigLong((UWORD *)t1+5, ABS(* (t1+*t1-1))-1, (UWORD *)t2+5,
02865 ABS(* (t2+*t2-1))-1) == 0) {
02866
02867                         optim.type = (sign1 * sign2 == 1 ? 4 : 5); // optimization type
02868                         optim.arg1 = x1;
02869                         optim.arg2 = x2;
02870                         if (optim.arg1 > optim.arg2) {
02871                             swap(optim.arg1, optim.arg2);
02872                         }
02873
02874                         if (ABS(* (t1+*t1-1))==3 && * (t1+*t1-2)==1 && * (t1+*t1-3)==1)
02875                             cnt[optim].push_back(e-ebegin);
02876                         else {
02877                             // E=2x+2y -> z=x+y; E=2z is improvement bby itself
02878                             cnt[optim].push_back(e-ebegin);
02879                             cnt[optim].push_back(e-ebegin);
02880                         }
02881                     }
02882                 }
02883             }
02884         }
02885 // #] type 4,5 :
02886 // #[ add :

```

```

02887
02888 // add optimizations with positive improvement to the result
02889 for (map<optimization, vector<int> >::iterator i=cnt.begin(); i!=cnt.end(); i++) {
02890     int improve = i->second.size() - 1;
02891     if (improve > 0) {
02892         res.push_back(i->first);
02893         res.back().improve = improve;
02894         res.back().eqnidxs = i->second;
02895
02896         // remove duplicates, that were add to get the correct improvement
02897         res.back().eqnidxs.erase(unique(res.back().eqnidxs.begin(), res.back().eqnidxs.end()),
res.back().eqnidxs.end());
02898     }
02899 }
02900 }
02901 // #] add :
02902
02903 #ifdef DEBUG_GREEDY
02904     MesPrint ("*** [%s, w=%w] DONE: find_optimizations", thetime_str().c_str());
02905 #endif
02906
02907     return res;
02908 }
02909
02910 /*
02911     #] find_optimizations :
02912     #[ do_optimization :
02913 */
02914
02931 bool do_optimization (const optimization optim, vector<WORD> &instr, int newid) {
02932
02933 // #[ Debug code :
02934 #ifdef DEBUG_GREEDY
02935     if (optim.type==0)
02936         MesPrint ("*** [%s, w=%w] CALL: do_optimization(improve=%d, %c%d^%d)",
02937             thetime_str().c_str(), optim.improve,
02938             optim.arg1>0?'x':'z', ABS(optim.arg1)-1, optim.arg2);
02939     else if (optim.type==1 || optim.type==4)
02940         MesPrint ("*** [%s, w=%w] CALL: do_optimization(improve=%d, %c%d%c%c%d)",
02941             thetime_str().c_str(), optim.improve,
02942             optim.arg1>0?'x':'z', ABS(optim.arg1)-1,
02943             optim.type==1 ? '*' : optim.type==4 ? '+' : '-',
02944             optim.arg2>0?'x':'z', ABS(optim.arg2)-1);
02945     else {
02946         WORD n = optim.coeff.back() / 2;
02947         UBYTE num[BITSIWORD*ABS(n)], den[BITSIWORD*ABS(n)];
02948         PrtLong((UWORD *)&optim.coeff[0], n, num);
02949         PrtLong((UWORD *)&optim.coeff[ABS(n)], ABS(n), den);
02950         MesPrint ("*** [%s, w=%w] CALL: do_optimization(improve=%d, %c%d%c%s/%s)",
02951             thetime_str().c_str(), optim.improve,
02952             optim.arg1>0?'x':'z', ABS(optim.arg1)-1,
02953             optim.type==2 ? '*' : '+', num, den);
02954     }
02955 #endif
02956 // #] Debug code :
02957
02958     bool substituted = false;
02959     WORD *ebegin = &instr.begin();
02960
02961 // #[ type 0 : substitution of the form z=x^n (optim.type==0)
02962     if (optim.type == 0) {
02963
02964         int vartypex = optim.arg1>0 ? SYMBOL : EXTRASYMBOL;
02965         int varnumx = ABS(optim.arg1) - 1;
02966         int n = optim.arg2;
02967
02968         for (int i=0; i<(int)optim.eqnidxs.size(); i++) {
02969             WORD *e = ebegin + optim.eqnidxs[i];
02970             if (*(e+1) != OPER_MUL) continue;
02971
02972             // scan through equation
02973             for (WORD *t=e+3; *t!=0; t+=*t) {
02974                 if (*t == ABS(*(t+*t-1))+1) continue;
02975                 if (*(t+1)==vartypex &&
02976                     *(t+3)==varnumx &&
02977                     *(t+4) % n == 0) {
02978
02979                     // substitute
02980                     *(t+1) = EXTRASYMBOL;
02981                     *(t+3) = newid;
02982                     *(t+4) /= n;
02983
02984                     substituted = true;
02985                 }
02986             }
02987         }
02988     }

```

```

02989         if (!substituted) {
02990 #ifdef DEBUG_GREEDY
02991     MesPrint ("*** [%s, w=%w] DONE: do_optimization : res=false", thetime_str().c_str(),
optim.improve);
02992 #endif
02993         return false;
02994     }
02995
02996     // add extra equation (Tnew = x^n)
02997     instr.push_back(newid); // eqn.nr
02998     instr.push_back(OPER_MUL); // operator
02999     instr.push_back(12); // total length
03000     instr.push_back(8); // term length
03001     instr.push_back(vartypex); // (extra)symbol
03002     instr.push_back(4); // symbol length
03003     instr.push_back(varnumx); // symbol id
03004     instr.push_back(n); // power
03005     instr.push_back(1); // coefficient 1
03006     instr.push_back(1); // coefficient 1
03007     instr.push_back(3); // trailing 0
03008     instr.push_back(0); // trailing 0
03009 }
03010 // #] type 0 :
03011 // #[ type 1 : substitution of the form z=x*y (optim.type==1)
03012 if (optim.type == 1) {
03013
03014     int vartypex = optim.arg1>0 ? SYMBOL : EXTRASYMBOL;
03015     int varnumx = ABS(optim.arg1) - 1;
03016     int vartypey = optim.arg2>0 ? SYMBOL : EXTRASYMBOL;
03017     int varnumy = ABS(optim.arg2) - 1;
03018
03019     for (int i=0; i<(int)optim.eqnidxs.size(); i++) {
03020         WORD *e = ebegin + optim.eqnidxs[i];
03021         if (*(e+1) != OPER_MUL) continue;
03022
03023         // scan through equation
03024         int powx=0, powy=0;
03025         for (WORD *t=e+3; *t!=0; t+=*t) {
03026             if (*t == ABS(*t+*t-1))+1) continue;
03027             if (*(t+1)==vartypex && *(t+3)==varnumx) powx = *(t+4);
03028             if (*(t+1)==vartypey && *(t+3)==varnumy) powy = *(t+4);
03029         }
03030
03031         // substitute if found
03032         if (powx>0 && powy>0 && powx==powy) {
03033
03034             WORD sign = 1;
03035             WORD *newt = e+3;
03036
03037             for (WORD *t=e+3; *t!=0; t+=*t) {
03038                 int dt=*t;
03039
03040                 if (*t == ABS(*t+*t-1))+1 ||
03041                     (!(*t+1)==vartypex && *(t+3)==varnumx) &&
03042                     (!(*t+1)==vartypey && *(t+3)==varnumy)) {
03043                     memmove(newt, t, *t*sizeof(WORD));
03044                     newt += dt;
03045                 }
03046                 else {
03047                     sign *= SGN(*t+*t-1);
03048                 }
03049
03050                 t+=dt;
03051             }
03052
03053             *newt++ = 8; // term length
03054             *newt++ = EXTRASYMBOL; // extrasymbol
03055             *newt++ = 4; // symbol length
03056             *newt++ = newid; // symbol id
03057             *newt++ = powx; // power
03058             *newt++ = 1;
03059             *newt++ = 1; // coefficient +/-1
03060             *newt++ = 3*sign;
03061             *newt++ = 0; // trailing 0
03062
03063             substituted = true;
03064         }
03065     }
03066
03067     if (!substituted) {
03068 #ifdef DEBUG_GREEDY
03069     MesPrint ("*** [%s, w=%w] DONE: do_optimization : res=false", thetime_str().c_str(),
optim.improve);
03070 #endif
03071     return false;
03072 }
03073

```

```

03074         // add extra equation (Tnew = x*y)
03075         instr.push_back(newid); // eqn.nr
03076         instr.push_back(OPER_MUL); // operator
03077         instr.push_back(20); // total length
03078         instr.push_back(8); // LHS length
03079         instr.push_back(vartype); // (extra)symbol
03080         instr.push_back(4); // symbol length
03081         instr.push_back(varnum); // symbol id
03082         instr.push_back(1); // power 1
03083         instr.push_back(1);
03084         instr.push_back(1); // coefficient 1
03085         instr.push_back(3);
03086         instr.push_back(8); // RHS length
03087         instr.push_back(vartype); // (extra)symbol
03088         instr.push_back(4); // symbol length
03089         instr.push_back(varnum); // symbol id
03090         instr.push_back(1); // power 1
03091         instr.push_back(1);
03092         instr.push_back(1); // coefficient 1
03093         instr.push_back(3);
03094         instr.push_back(0); // trailing 0
03095     }
03096     // #] type 1 :
03097     // #[ type 2 : substitution of the form z=c*x (optim.type==2)
03098
03099     if (optim.type == 2) {
03100         int vartype = optim.arg1>0 ? SYMBOL : EXTRASYMBOL;
03101         int varnum = ABS(optim.arg1) - 1;
03102
03103         WORD ncoeff = optim.coeff.back();
03104
03105         // scan through equations
03106         for (int i=0; i<(int)optim.eqnidxs.size(); i++) {
03107             WORD *e = ebegin + optim.eqnidxs[i];
03108
03109             if (*(e+1) == OPER_ADD) {
03110                 // scan through ADD-equation
03111                 for (WORD *t=e+3; *t!=0; t+=*t) {
03112
03113                     if (*t == ABS(*(t+*t-1))+1) continue;
03114
03115                     if (*(t+1)==vartype && ABS(*(t+3))==varnum && *(t+4)==1 &&
03116                         BigLong((UWORD *) &optim.coeff[0], ncoeff-1,
03117                             (UWORD *) t+*t-ABS(*(t+*t-1)), ABS(*(t+*t-1))-1) == 0) {
03118                         // substitute
03119
03120                         int sign = SGN(*(t+*t-1));
03121
03122                         WORD *tend = t;
03123                         while (*tend!=0) tend+=*tend;
03124                         WORD nmove = tend - t - *t;
03125                         memmove(t, t+*t, nmove*sizeof(WORD));
03126                         t += nmove;
03127
03128                         *t++ = 8; // term length
03129                         *t++ = EXTRASYMBOL; // (extra)symbol
03130                         *t++ = 4; // symbol length
03131                         *t++ = newid; // symbol id
03132                         *t++ = 1; // power of 1
03133                         *t++ = 1;
03134                         *t++ = 1; // coefficient of +/-1
03135                         *t++ = 3 * sign;
03136                         *t++ = 0; // trailing 0
03137
03138                         substituted = true;
03139                         break;
03140                     }
03141                 }
03142             }
03143             else if (*(e+1) == OPER_MUL) {
03144
03145                 bool coeff_match=false, var_match=false;
03146                 int sign = 1;
03147
03148                 // scan through MUL-equation
03149                 for (WORD *t=e+3; *t!=0; t+=*t) {
03150                     if (*t == ABS(*(t+*t-1))+1 && BigLong((UWORD *) &optim.coeff[0], ncoeff-1,
03151                         *(t+*t-ABS(*(t+*t-1))), ABS(*(t+*t-1))-1) == 0) {
03152                         coeff_match = true;
03153                         sign *= SGN(*(t+*t-1));
03154                     }
03155                     else if (*(t+1)==vartype && ABS(*(t+3))==varnum && *(t+4)==1) {
03156                         var_match = true;
03157                         sign *= SGN(*(t+*t-1));
03158                     }
03159                 }

```

```

03160
03161         // substitute if found
03162         if (coeff_match && var_match) {
03163
03164             WORD *newt = e+3;
03165
03166             for (WORD *t=e+3; *t!=0;) {
03167
03168                 int dt=*t;
03169
03170                 if (*t!=ABS(*(t+*t-1))+1 && !(*t+1)==vartype && ABS(*(t+3))==varnum &&
03171                     *(t+4)==1) {
03172                     memmove(newt, t, dt*sizeof(WORD));
03173                     newt += dt;
03174                 }
03175                 t+=dt;
03176             }
03177
03178             *newt++ = 8;           // term length
03179             *newt++ = EXTRASYMBOL; // extrasymbol
03180             *newt++ = 4;           // symbol length
03181             *newt++ = newid;        // symbol id
03182             *newt++ = 1;           // power of 1
03183             *newt++ = 1;
03184             *newt++ = 1;           // coefficient of +/-1
03185             *newt++ = 3 * sign;
03186             *newt++ = 0;           // trailing 0
03187
03188             substituted = true;
03189         }
03190     }
03191 }
03192
03193     if (!substituted) {
03194 #ifdef DEBUG_GREEDY
03195         MesPrint ("*** [%s, w=%w] DONE: do_optimization : res=false", thetime_str().c_str(),
03196             optim.improve);
03197 #endif
03198         return false;
03199     }
03200
03201     // add extra equation (Tnew = c*y)
03202     instr.push_back(newid);           // eqn.nr
03203     instr.push_back(OPER_ADD);        // operator
03204     instr.push_back(9+ABS(ncoeff));   // total length
03205     instr.push_back(5+ABS(ncoeff));   // term length
03206     instr.push_back(vartype);         // (extra)symbol
03207     instr.push_back(4);               // symbol length
03208     instr.push_back(varnum);          // symbol id
03209     instr.push_back(1);               // power of 1
03210     for (int i=0; i<ABS(ncoeff); i++) // coefficient
03211         instr.push_back(optim.coeff[i]);
03212     instr.push_back(0);               // trailing 0
03213 }
03214 // #] type 2 :
03215 // #[ type 3 : substitution of the form z=x+c (optim.type==3)
03216 if (optim.type == 3) {
03217     int vartype = optim.arg1>0 ? SYMBOL : EXTRASYMBOL;
03218     int varnum = ABS(optim.arg1) - 1;
03219
03220     WORD ncoeff = optim.coeff.back();
03221
03222     // scan through equation
03223     for (int i=0; i<(int)optim.eqnidxs.size(); i++) {
03224         WORD *e = ebegin + optim.eqnidxs[i];
03225
03226         if (*(e+1) != OPER_ADD) continue;
03227
03228         int coeff_match=0, var_match=0;
03229
03230         for (WORD *t=e+3; *t!=0; t+=*t) {
03231             if (*t == ABS(*(t+*t-1))+1 && BigLong((UWORD *) &optim.coeff[0], ABS(ncoeff)-1,
03232                 *(t+*t-ABS(*(t+*t-1)), ABS(*(t+*t-1))-1) == 0)
03233                 coeff_match = SGN(ncoeff) * SGN(*(t+*t-1));
03234             else if (*t+1==vartype && ABS(*(t+3))==varnum && *(t+4)==1)
03235                 var_match = SGN(*(t+7));
03236         }
03237
03238         // substitute if found (x+c and -x-c and matches)
03239         if (coeff_match * var_match == 1) {
03240             WORD *newt = e+3;
03241
03242             for (WORD *t=e+3; *t!=0;) {
03243                 int dt=*t;

```

```

03244         if (*t!=ABS(*t+*t-1))+1 && !(*t+1)==vartype && ABS(*t+3)==varnum &&
*(t+4)==1) {
03245             memmove(newt, t, dt*sizeof(WORD));
03246             newt += dt;
03247         }
03248         t+=dt;
03249     }
03250
03251     *newt++ = 8;           // term length
03252     *newt++ = EXTRASYMBOL; // extrasymbol
03253     *newt++ = 4;           // symbol length
03254     *newt++ = newid;       // symbol id
03255     *newt++ = 1;           // power of 1
03256     *newt++ = 1;
03257     *newt++ = 1;           // coefficient of +/-1
03258     *newt++ = 3*coeff_match;
03259     *newt++ = 0;           // trailing zero
03260
03261     substituted = true;
03262 }
03263 }
03264
03265     if (!substituted) {
03266 #ifdef DEBUG_GREEDY
03267         MesPrint ("*** [%s, w=%w] DONE: do_optimization : res=false", thetime_str().c_str(),
optim.improve);
03268 #endif
03269         return false;
03270     }
03271
03272     // add extra equation (Tnew = x+c)
03273     instr.push_back(newid);           // eqn.nr
03274     instr.push_back(OPER_ADD);        // operator
03275     instr.push_back(13+ABS(ncoeff));  // total length
03276     instr.push_back(8);               // x-term length
03277     instr.push_back(vartype);         // (extra)symbol
03278     instr.push_back(4);               // symbol length
03279     instr.push_back(varnum);          // symbol id
03280     instr.push_back(1);               // power of 1
03281     instr.push_back(1);
03282     instr.push_back(1);               // coefficient of 1
03283     instr.push_back(3);
03284     instr.push_back(ABS(ncoeff)+1);   // c-term length
03285     for (int i=0; i<ABS(ncoeff); i++) // coefficient
03286         instr.push_back(optim.coeff[i]);
03287     instr.push_back(0);               // trailing zero
03288 }
03289 // #] type 3 :
03290 // #[ type 4,5 : substitution of the form z=x+y or z=x-y (optim.type=4 or 5)
03291 if (optim.type >= 4) {
03292
03293     int vartypex = optim.arg1>0 ? SYMBOL : EXTRASYMBOL;
03294     int varnumx = ABS(optim.arg1) - 1;
03295     int vartypey = optim.arg2>0 ? SYMBOL : EXTRASYMBOL;
03296     int varnumy = ABS(optim.arg2) - 1;
03297
03298     // scan through equations
03299     for (int i=0; i<(int)optim.eqnidxs.size(); i++) {
03300         WORD *e = ebegin + optim.eqnidxs[i];
03301
03302         if (*(e+1) != OPER_ADD) continue;
03303
03304         const WORD *coeffx=NULL, *coeffy=NULL;
03305         WORD ncoeffx=0,ncoeffy=0;
03306
03307         // looks for terms
03308         for (WORD *t=e+3; *t!=0; t+=*t) {
03309             if (*t == ABS(*t+*t-1))+1 continue; // constant
03310             if (*t+1)==vartypex && *t+3==varnumx && *t+4==1) {
03311                 coeffx = t+5;
03312                 ncoeffx = *(t+*t-1);
03313             }
03314             if (*t+1)==vartypey && *t+3==varnumy && *t+4==1) {
03315                 coeffy = t+5;
03316                 ncoeffy = *(t+*t-1);
03317             }
03318         }
03319
03320         // check signs (type=4: x+y and -x-y, type=5: x-y and -x+y) ??????
03321         // check signs (type=4: x+y, type=5: x-y) !!!!!!!!!!!
03322         if (SGN(ncoeffx) * SGN(ncoeffy) * (optim.type==4 ? 1 : -1) == 1) {
03323             // check absolute value of coefficients
03324             if (BigLong((UWORD *)coeffx, ABS(ncoeffx)-1, (UWORD *)coeffy, ABS(ncoeffy)-1) == 0) {
03325                 // substitute
03326                 vector<WORD> coeff(coeffx, coeffx+ABS(ncoeffx));
03327
03328                 WORD *newt = e+3;

```



```

03329 /*
03330 if ( optim.type == 5 ) {
03331     while ( *newt ) newt+=*newt;
03332     int i = (newt - e) - 3;
03333     MesPrint(" < %a",i,e+3);
03334     newt = e+3;
03335 }
03336 */
03337         for (WORD *t=e+3; *t!=0;) {
03338             int dt=*t;
03339             if (*t == ABS(*(t+*t-1))+1 ||
03340                 (!(*t+1)==vartypex && *(t+3)==varnumx) &&
03341                 !(*t+1)==vartypey && *(t+3)==varnumy)) {
03342                 memmove(newt, t, dt*sizeof(WORD));
03343                 newt += dt;
03344             }
03345             t+=dt;
03346         }
03347
03348         *newt++ = 5 + ABS(ncoeffx);           // term length
03349         *newt++ = EXTRASYMBOL;               // extrasymbol
03350         *newt++ = 4;                         // symbol length
03351         *newt++ = newid;                     // symbol id
03352         *newt++ = 1;                         // power of 1
03353         for (int j=0; j<ABS(ncoeffx); j++) // coefficient
03354             *newt++ = coeff[j];
03355         *newt++ = 0;                         // trailing 0
03356         substituted = true;
03357 /*
03358 if ( optim.type == 5 ) {
03359     int i = (newt - e) - 4;
03360     MesPrint(" > %a",i,e+3);
03361 }
03362 */
03363     }
03364 }
03365 }
03366
03367     if (!substituted) {
03368 #ifdef DEBUG_GREEDY
03369         MesPrint ("*** [%s, w=%w] DONE: do_optimization : res=false", thetime_str().c_str(),
03370             optim.improve);
03371 #endif
03372         return false;
03373 }
03374 /*
03375 if ( optim.type == 5 )
03376     MesPrint ("improve=%d, %c%d%c%c%d", optim.improve,
03377         optim.arg1>0?'x':'Z', ABS(optim.arg1)-1,
03378         optim.type==1?'*': optim.type==4?'+' : '- ',
03379         optim.arg2>0?'x':'Z', ABS(optim.arg2)-1);
03380 */
03381         // add extra equation (Tnew = x+/-y)
03382         instr.push_back(newid);           // eqn.nr
03383         instr.push_back(OPER_ADD);       // operator
03384         instr.push_back(20);             // total length
03385         instr.push_back(8);              // term length
03386         instr.push_back(vartypex);       // (extra)symbol
03387         instr.push_back(4);              // symbol length
03388         instr.push_back(varnumx);        // symbol id
03389         instr.push_back(1);              // power of 1
03390         instr.push_back(1);
03391         instr.push_back(1);              // coefficient of 1
03392         instr.push_back(8);              // term length
03393         instr.push_back(vartypey);       // (extra)symbol
03394         instr.push_back(4);              // symbol length
03395         instr.push_back(varnumy);        // symbol id
03396         instr.push_back(1);              // power of 1
03397         instr.push_back(1);
03398         instr.push_back(1);              // coefficient of +/-1
03399         instr.push_back(3*(optim.type==4?1:-1));
03400         instr.push_back(0);              // trailing 0
03401     }
03402 // #] type 4,5 :
03403 // #[ trivial : remove trivial equations of the form Zi = +/-Zj
03404 vector<int> renum(newid+1, 0);
03405 bool do_renum=false;
03406
03407 // vector may be moved when it is extended
03408 ebegin = &*instr.begin();
03409
03410 for (int i=0; i<(int)optim.eqnidxs.size(); i++) {
03411     WORD *e = ebegin + optim.eqnidxs[i];
03412     WORD *t = e+3;
03413     if (*t==0) continue;                // empty (removed) equation
03414     if (*(t+*t)!=0) continue;           // more than 1 term

```

```

03415         if (*t!=8) continue; // term not of correct form
03416         if (*t!=1) continue; // power != 1
03417         if (*t!=1 || *(t+6)!=1) continue; // coeff != +/-1
03418
03419         // trivial term, so renumber this one
03420         renum[*e] = SGN(*(t+7)) * (*(t+3) + 1);
03421         do_renum = true;
03422
03423         // remove equation
03424         *t=0;
03425     }
03426 // #] trivial :
03427 // #[ renumbering :
03428
03429 // there are renumberings to be done, so loop through all equations
03430 if (do_renum) {
03431     WORD *eend = ebegin+instr.size();
03432
03433     for (WORD *e=ebegin; e!=eend; e+=*(e+2)) {
03434         for (WORD *t=e+3; *t!=0; t+=*t) {
03435             if (*t == ABS(*(t+*t-1))+1) continue;
03436             if (*(t+1)==EXTRASYNBOL && renum[*t+3]!=0) {
03437                 *(t+*t-1) *= SGN(renum[*t+3]);
03438                 *(t+3) = ABS(renum[*t+3]) - 1;
03439             }
03440         }
03441     }
03442 }
03443 // #] renumbering :
03444
03445 #ifdef DEBUG_GREEDY
03446     MesPrint ("*** [%s, w=%w] DONE: do_optimization : res=true", thetime_str().c_str(),
03447         optim.improve);
03447 #endif
03448
03449     return true;
03450 }
03451
03452 /*
03453 #] do_optimization :
03454 #[ partial_factorize :
03455 */
03456
03480 int partial_factorize (vector<WORD> &instr, int n, int improve) {
03481
03482 #ifdef DEBUG_GREEDY
03483     MesPrint ("*** [%s, w=%w] CALL: partial_factorize (n=%d)", thetime_str().c_str(), n);
03484 #endif
03485
03486     GETIDENTITY;
03487
03488     // get starting positions of instructions
03489     vector<int> instr_idx(n);
03490     WORD *ebegin = &*instr.begin();
03491     WORD *eend = ebegin+instr.size();
03492     for (WORD *e=ebegin; e!=eend; e+=*(e+2)) {
03493         instr_idx[*e] = e - ebegin;
03494     }
03495
03496     // get reference counts
03497 /*
03498 * The next construction replaces the vector construction which is
03499 * rather costly for valgrind (and maybe also in normal running)
03500 */
03501     int nmax = 2*n;
03502     WORD *numpar = (WORD *)Malloc1(nmax*sizeof(WORD), "numpar");
03503     for (int i = 0; i < nmax; i++) numpar[i] = 0;
03504 // vector<WORD> numpar(n);
03505     for (WORD *e=ebegin; e!=eend; e+=*(e+2))
03506         for (WORD *t=e+3; *t!=0; t+=*t) {
03507             if (*t == ABS(*(t+*t-1))+1) continue;
03508             if (*(t+1) == EXTRASYNBOL) numpar[*t+3]++;
03509         }
03510
03511     // find factorizable expressions
03512     for (int i=0; i<n; i++) {
03513         WORD *e = &*instr.begin() + instr_idx[i];
03514         if (*(e+1) != OPER_ADD) continue;
03515
03516         // count symbol occurrences
03517         map<WORD,WORD> cnt; // 1-indexed, <0:EXTRASYNBOL, >0:SYMBOL
03518
03519         for (WORD *t=e+3; *t!=0; t+=*t) {
03520             if (*t==ABS(*(t+*t-1))+1) continue;
03521
03522             // count symbols in t
03523             if (*(t+4)==1)

```

```

03524         cnt[(*(t+1)==SYMBOL ? 1 : -1) * (*(t+3)+1)]++;
03525
03526         // count symbols in extrasymbols of t
03527         if (*(t+1)==EXTRASymbol && *(t+4)==1 && numpar[*(t+3)]==1) {
03528             WORD *t2 = &*instr.begin() + instr_idx[*(t+3)];
03529             if (*(t2+1) != OPER_MUL) continue;
03530             for (t2+=3; *t2!=0; t2+=*t2) {
03531                 if (*t2 == ABS(*(t2+*t2-1))+1) continue;
03532                 if (*(t2+4)==1)
03533                     cnt[(*(t2+1)==SYMBOL ? 1 : -1) * (*(t2+3)+1)]++;
03534             }
03535         }
03536     }
03537
03538     // find most-occurring symbol
03539     WORD x=0, best=0;
03540     for (map<WORD,WORD>::iterator it=cnt.begin(); it!=cnt.end(); it++)
03541         if (it->second > best) { x=it->first; best=it->second; }
03542
03543     // occurrence>=2 and occurrence>improve, so factorize
03544
03545     if (best>=2 && best>improve) {
03546         // initialize new equation (Zi from example above)
03547         vector<WORD> new_eqn;
03548         new_eqn.push_back(n);
03549         new_eqn.push_back(OPER_ADD);
03550         new_eqn.push_back(0); // length
03551
03552         WORD dt;
03553         WORD *newt=e+3;
03554         for (WORD *t=e+3; *t!=0; t+=dt) {
03555             dt = *t;
03556             bool keep=true;
03557
03558             if (*(t!=ABS(*(t+*t-1))+1) {
03559
03560                 // factorized symbol is in t itself
03561                 if (*(t+4)==1) {
03562                     WORD y = (*(t+1)==SYMBOL ? 1 : -1) * (*(t+3)+1);
03563                     if (y==x) {
03564                         new_eqn.push_back(*t-4);
03565                         new_eqn.insert(new_eqn.end(), t+5, t+dt);
03566                         keep=false;
03567                     }
03568                 }
03569
03570                 // look in extrasymbol of t with ref.count=1
03571                 if (*(t+1)==EXTRASymbol && *(t+4)==1 && numpar[*(t+3)]==1) {
03572                     WORD *t2 = &*instr.begin() + instr_idx[*(t+3)];
03573                     if (*(t2+1) == OPER_MUL) {
03574                         bool has_x=false;
03575                         for (t2+=3; *t2!=0; t2+=*t2) {
03576                             if (*t2 == ABS(*(t2+*t2-1))+1) continue;
03577                             WORD y = (*(t2+1)==SYMBOL ? 1 : -1) * (*(t2+3)+1);
03578                             // extrasymbol has factorized symbol
03579                             if (y==x && *(t2+4)==1) {
03580                                 has_x=true;
03581                                 // copy remaining part
03582                                 WORD *tend=t2+*t2;
03583                                 WORD sign = SGN(*(tend-1));
03584                                 while (*tend!=0) tend+=*tend;
03585                                 int dt2 = tend - (t2+*t2);
03586                                 memmove(t2, t2+*t2, (dt2+1)*sizeof(WORD));
03587                                 t2 += dt2;
03588                                 *(t2-1) *= sign;
03589                                 break;
03590                             }
03591                         }
03592                     if (has_x) {
03593                         // extrasymbol has x, so add it to new equation
03594                         keep=false;
03595                         int thisidx=new_eqn.size();
03596                         new_eqn.insert(new_eqn.end(), t, t+dt);
03597                         t2 = &*instr.begin() + instr_idx[*(t+3)] + 3;
03598                         // if becomes trivial, substitute the term
03599                         if (*(t2+*t2)==0) {
03600                             // it's a number
03601                             if (*t2 == ABS(*(t2+*t2-1))+1) {
03602                                 if (ABS(new_eqn[new_eqn.size()-1])==3 &&
new_eqn[new_eqn.size()-2]==1 && new_eqn[new_eqn.size()-3]==1) {
03603                                     // original equation has coefficient of +/-1, so replace
it
03604                                     WORD sign = SGN(new_eqn.back());
03605                                     new_eqn.erase(new_eqn.begin()+thisidx, new_eqn.end());
03606                                     new_eqn.insert(new_eqn.end(), t2, t2+*t2);
03607                                     new_eqn.back() *= sign;
03608                                     *t2 = 0;

```

```

03609                                     }
03610                                     else {
03611                                         // two non-trivial coefficients, so multiply them
03612                                         // note: untested code (found no way to trigger it)
03613                                         UWORD *tmp = NumberMalloc("partial_factorize");
03614                                         WORD ntmp=0;
03615                                         MulRat(BHEAD (UWORD *)t2+*t2-ABS(*t2+*t2-1)),
03616                                         *t2+*t2-1,
03617                                         (UWORD
03618                                         *) &*(new_eqn.end()-ABS(new_eqn.back())), new_eqn.back(),
03619                                         tmp, &ntmp);
03620                                         new_eqn.erase(new_eqn.begin()+thisidx, new_eqn.end());
03621                                         new_eqn.push_back(ABS(ntmp)+1);
03622                                         new_eqn.insert(new_eqn.end(), tmp, tmp+ABS(ntmp));
03623                                         NumberFree(tmp, "partial_factorize");
03624                                         *t2 = 0;
03625                                     }
03626                                     }
03627                                     else if (*t2+4)==1) {
03628                                         // it's a variable
03629                                         new_eqn.back() *= SGN(*t2+*t2-1));
03630                                         new_eqn[thisidx+1] = *t2+1;
03631                                         new_eqn[thisidx+2] = *t2+2;
03632                                         new_eqn[thisidx+3] = *t2+3;
03633                                         new_eqn[thisidx+4] = *t2+4;
03634                                         *t2 = 0;
03635                                     }
03636                                     }
03637                                     }
03638                                     }
03639                                     }
03640                                     // no x, so copy it
03641                                     if (keep) {
03642                                         memmove(newt, t, dt*sizeof(WORD));
03643                                         newt += dt;
03644                                     }
03645                                     }
03646                                     }
03647                                     // finalize new equation
03648                                     new_eqn.push_back(0);
03649                                     new_eqn[2] = new_eqn.size();
03650                                     bool empty = newt == e+3;
03651                                     if ( n+1 >= nmax ) {
03652                                         int i, newnmax = nmax*2;
03653                                         WORD *newnumpar = (WORD *)Malloc1(newnmax*sizeof(WORD), "newnumpar");
03654                                         for ( i = 0; i < n; i++ ) newnumpar[i] = numpar[i];
03655                                         for ( ; i < newnmax; i++ ) newnumpar[i] = 0;
03656                                         M_free(numpar, "numpar");
03657                                         numpar = newnumpar;
03658                                         nmax = newnmax;
03659                                     }
03660                                     numpar.push_back(0);
03661                                     n++;
03662                                     // if original is not empty, add new equation (Zj) to it
03663                                     // otherwise replace it later
03664                                     if (!empty) {
03665                                         *newt++ = 8;
03666                                         *newt++ = EXTRASYMBOL;
03667                                         *newt++ = 4;
03668                                         *newt++ = n;
03669                                         *newt++ = 1;
03670                                         *newt++ = 1;
03671                                         *newt++ = 1;
03672                                         *newt++ = 3;
03673                                         *newt++ = 0;
03674                                     }
03675                                     // add new equation to instructions
03676                                     instr_idx.push_back(instr.size());
03677                                     instr.insert(instr.end(), new_eqn.begin(), new_eqn.end());
03678                                     // generate another new equation (Zj=x*Zi)
03679                                     new_eqn.clear();
03680                                     new_eqn.push_back(n);
03681                                     new_eqn.push_back(OPER_MUL);
03682                                     new_eqn.push_back(20);
03683                                     new_eqn.push_back(8);
03684                                     // add factorized symbol
03685                                     if (x>0) {
03686                                         new_eqn.push_back(SYMBOL);
03687                                         new_eqn.push_back(4);
03688                                         new_eqn.push_back(x-1);
03689                                     }
03690                                     }
03691                                     }
03692                                     }
03693                                     }

```

```

03694         new_eqn.push_back(1);
03695     }
03696     else {
03697         new_eqn.push_back(EXTRASymbol);
03698         new_eqn.push_back(4);
03699         new_eqn.push_back(-x-1);
03700         new_eqn.push_back(1);
03701     }
03702     new_eqn.push_back(1);
03703     new_eqn.push_back(1);
03704     new_eqn.push_back(3);
03705     new_eqn.push_back(8);
03706     // add new equation (Zi)
03707     new_eqn.push_back(EXTRASymbol);
03708     new_eqn.push_back(4);
03709     new_eqn.push_back(n-1);
03710     new_eqn.push_back(1);
03711     new_eqn.push_back(1);
03712     new_eqn.push_back(1);
03713     new_eqn.push_back(3);
03714     new_eqn.push_back(0);
03715
03716     if (!empty) {
03717         // add new equation (Zj) to instructions
03718         instr_idx.push_back(instr.size());
03719         instr.insert(instr.end(), new_eqn.begin(), new_eqn.end());
03720         if ( n+1 >= nmax ) {
03721             int i, newnmax = nmax*2;
03722             WORD *newnumpar = (WORD *)Malloca1(newnmax*sizeof(WORD), "newnumpar");
03723             for ( i = 0; i < n; i++ ) newnumpar[i] = numpar[i];
03724             for ( ; i < newnmax; i++ ) newnumpar[i] = 0;
03725             M_free(numpar, "numpar");
03726             numpar = newnumpar;
03727             nmax = newnmax;
03728         }
03729         // numpar.push_back(0);
03730         n++;
03731     }
03732     else {
03733         // replace e with Zj
03734         e = &*instr.begin() + instr_idx[i];
03735         e[1] = OPER_MUL;
03736         memcpy(e+3, &new_eqn[3], (new_eqn.size()-3)*sizeof(WORD));
03737     }
03738
03739     // decrease i, so this expression is factorized again if possible
03740     i--;
03741 }
03742 }
03743
03744 #ifdef DEBUG_GREEDY
03745     MesPrint ("*** [%s, w=%w] DONE: partial_factorize (n=%d)", thetime_str().c_str(), n);
03746 #endif
03747     M_free(numpar, "numpar");
03748     return n;
03749 }
03750
03751 /*
03752 #] partial_factorize :
03753 #[ optimize_greedy :
03754 */
03755
03774 vector<WORD> optimize_greedy (vector<WORD> instr, LONG time_limit) {
03775
03776 #ifdef DEBUG
03777     int old_num_oper = count_operators(instr);
03778     MesPrint ("*** [%s, w=%w] CALL: optimize_greedy(numoper=%d)",
03779             thetime_str().c_str(), old_num_oper);
03780 #endif
03781
03782     LONG start_time = TimeWallClock(1);
03783
03784     WORD *ebegin = &*instr.begin();
03785     WORD *eend = ebegin+instr.size();
03786
03787     // store final equation, since it must be the last equation later
03788     int final_eqn_idx = 0;
03789     int next_eqn = 0;
03790
03791     for (WORD *e=ebegin; e!=eend; e+=*(e+2)) {
03792         next_eqn = *e + 1;
03793         final_eqn_idx = e-ebegin;
03794     }
03795     // optimize instructions
03796     while (TimeWallClock(1)-start_time < time_limit) {
03797         int old_next_eqn = next_eqn;
03798

```

```

03799         // find optimizations
03800         vector<optimization> optim = find_optimizations(instr);
03801
03802         // add_eqnidxs contains modified equations, which might have to be updated later again
03803         vector<int> add_eqnidxs;
03804
03805         // number of optimizations to do
03806         int num_do_optims = max(AO.Optimize.greedyminnum,
(int)optim.size()*AO.Optimize.greedymaxperc/100);
03807
03808         // if best improvement is one, do all optimizations
03809         int best_improve=0;
03810         for (int i=0; i<(int)optim.size(); i++)
03811             best_improve = max(best_improve, optim[i].improve);
03812         if (best_improve <= 1)
03813             num_do_optims = MAXPOSITIVE;
03814         // do a number of optimizations
03815         while (optim.size() > 0 && num_do_optims-- > 0) {
03816
03817             // find best optimization
03818             int best=0;
03819             best_improve=0;
03820             for (int i=0; i<(int)optim.size(); i++)
03821                 if (optim[i].improve > best_improve) {
03822                     best=i;
03823                     best_improve=optim[i].improve;
03824                 }
03825
03826             // add extra equations
03827             for (int i=0; i<(int)add_eqnidxs.size(); i++)
03828                 optim[best].eqnidxs.push_back(add_eqnidxs[i]);
03829
03830             // do optimization, update next_eqn if successful
03831             int next_idx = instr.size();
03832             if (do_optimization(optim[best], instr, next_eqn)) {
03833                 next_eqn++;
03834                 add_eqnidxs.push_back(next_idx);
03835             }
03836
03837             optim.erase(optim.begin()+best);
03838         }
03839
03840         // partially factorize with improve >= best_improve
03841         next_eqn = partial_factorize(instr, next_eqn, best_improve);
03842
03843         // check whether nothing has changed
03844         if (next_eqn == old_next_eqn) break;
03845     }
03846
03847     // add final equation to the back (must be by definition)
03848     instr.push_back(next_eqn);
03849     instr.insert(instr.end(), instr.begin()+final_eqn_idx+1,
instr.begin()+final_eqn_idx+instr[final_eqn_idx+2]);
03850
03851     // removed original final equation
03852     instr[final_eqn_idx+3] = 0;
03853
03854     // remove extra zeroes
03855     WORD *t = &instr[0];
03856
03857     ebegin = &*instr.begin();
03858     eend = ebegin+instr.size();
03859     int de=0;
03860
03861     for (WORD *e=ebegin; e!=eend; e+=de) {
03862         de = *(e+2);
03863         int n=3;
03864         while (*(e+n) != 0) n+=*(e+n);
03865         n++;
03866         memmove (t, e, n*sizeof(WORD));
03867         *(t+2) = n;
03868         t += n;
03869     }
03870
03871     instr.resize(t - &instr[0]);
03872
03873 #ifdef DEBUG
03874     MesPrint ("*** [%s, w=%w] DONE: optimize_greedy(numoper=%d) : numoper=%d",
03875             thetime_str().c_str(), old_num_oper, count_operators(instr));
03876 #endif
03877
03878     return instr;
03879 }
03880
03881 /*
03882 #] optimize_greedy :
03883 #[ recycle_variables :

```

```

03884 */
03885
03914 vector<WORD> recycle_variables (const vector<WORD> &all_instr) {
03915
03916 #ifdef DEBUG_MORE
03917     MesPrint ("*** [%s, w=%w] CALL: recycle_variables", thetime_str().c_str());
03918 #endif
03919
03920     // get starting positions of instructions
03921     vector<const WORD *> instr;
03922     const WORD *tbegin = &all_instr.begin();
03923     const WORD *tend = tbegin+all_instr.size();
03924     for (const WORD *t=tbegin; t!=tend; t+=*(t+2))
03925         instr.push_back(t);
03926     int n = instr.size();
03927
03928     // determine with expressions are connected, how many intermediate
03929     // are needed (assuming it's a expression tree instead of a DAG) and
03930     // sort the leaves such that you need a minimal number of variables
03931     vector<int> vars_needed(n);
03932     vector<bool> vis(n,false);
03933     vector<vector<int> > conn(n);
03934
03935     stack<int> s;
03936     s.push(n);
03937
03938     while (!s.empty()) {
03939         int i=s.top(); s.pop();
03940         if (i>0) {
03941             i--;
03942             if (vis[i]) continue;
03943             vis[i]=true;
03944             s.push(-(i+1));
03945
03946             // find all connections
03947             for (const WORD *t=instr[i]+3; *t!=0; t+=*t)
03948                 if (*t!=1+ABS(*(t+*t-1)) && *(t+1)==EXTRASYMBOL) {
03949                     int k = *(t+3);
03950                     conn[i].push_back(k);
03951                     s.push(k+1);
03952                 }
03953         }
03954         else {
03955             i=-i-1;
03956
03957             // sort the childs w.r.t. needed variables
03958             vector<pair<int,int> > need;
03959             for (int j=0; j<(int)conn[i].size(); j++)
03960                 need.push_back(make_pair(vars_needed[conn[i][j]], conn[i][j]));
03961
03962             // keep the comma expression in proper order
03963             if (*(instr[i]+1) != OPER_COMMA)
03964                 sort(need.rbegin(), need.rend());
03965
03966             vars_needed[i] = 1;
03967             for (int j=0; j<(int)need.size(); j++) {
03968                 vars_needed[i] = max(vars_needed[i], need[j].first+j);
03969                 conn[i][j] = need[j].second;
03970             }
03971         }
03972     }
03973
03974     // order the instructions in depth-first order and determine the first
03975     // and last occurrences of variables
03976     vector<int> order, first(n,0), last(n,0);
03977     vis = vector<bool>(n,false);
03978     s.push(n);
03979
03980     while (!s.empty()) {
03981
03982         int i=s.top(); s.pop();
03983
03984         if (i>0) {
03985             i--;
03986             if (vis[i]) continue;
03987             vis[i]=true;
03988             s.push(-(i+1));
03989             for (int j=(int)conn[i].size()-1; j>=0; j--)
03990                 s.push(conn[i][j]+1);
03991         }
03992         else {
03993             i=-i-1;
03994             first[i] = last[i] = order.size();
03995             order.push_back(i);
03996             for (int j=0; j<(int)conn[i].size(); j++) {
03997                 int k = conn[i][j];
03998                 last[k] = max(last[k], first[i]);
03999             }
04000         }
04001     }
04002 }

```

```

03999         }
04000     }
04001 }
04002
04003 // find the renumbering to recycled variables, where at any time the
04004 // lowest-indexed variable that can be used is chosen
04005 int numvar=0;
04006 set<int> var;
04007 vector<int> renum(n);
04008
04009 for (int i=0; i<(int)order.size(); i++) {
04010     for (int j=0; j<(int)conn[order[i]].size(); j++) {
04011         int k = conn[order[i]][j];
04012         if (last[k] == i) var.insert(renum[k]);
04013     }
04014
04015     if (var.empty()) var.insert(numvar++);
04016     renum[order[i]] = *var.begin(); var.erase(var.begin());
04017 }
04018
04019 // put the number of variables used in a preprocessor variable
04020
04021 // generate new instructions with the renumbering
04022 vector<WORD> newinstr;
04023
04024 for (int i=0; i<(int)order.size(); i++) {
04025     int x = order[i];
04026     int j = newinstr.size();
04027     newinstr.insert(newinstr.end(), instr[x], instr[x]+(instr[x]+2));
04028
04029     newinstr[j] = renum[newinstr[j]];
04030
04031     for (WORD *t=&newinstr[j+3]; *t!=0; t+=*t)
04032         if (*t!=1+ABS(*t+*t-1)) && *(t+1)==EXTRASymbol
04033             *(t+3) = renum[*t+3];
04034 }
04035
04036 #ifdef DEBUG_MORE
04037     MesPrint ("*** [%s, w=%w] DONE: recycle_variables", thetime_str().c_str());
04038 #endif
04039
04040     return newinstr;
04041 }
04042
04043 /*
04044 #] recycle_variables :
04045 #[ optimize_expression_given_Horner :
04046 */
04047
04061 void optimize_expression_given_Horner () {
04062
04063     #ifdef DEBUG
04064         MesPrint ("*** [%s, w=%w] CALL: optimize_expression_given_Horner", thetime_str().c_str());
04065     #endif
04066
04067     GETIDENTITY;
04068
04069     // initialize timer
04070     LONG start_time = TimeWallClock(1);
04071     LONG time_limit = 100 * AO.Optimize.greedytimelimit / (AO.Optimize.horner == O_MCTS ?
AO.Optimize.mctsnumkeep : 1);
04072     if (time_limit == 0) time_limit=MAXPOSITIVE;
04073
04074     // pick a Horner scheme from the list
04075     LOCK(optimize_lock);
04076     vector<WORD> Horner_scheme = optimize_best_Horner_schemes.back();
04077     optimize_best_Horner_schemes.pop_back();
04078     UNLOCK(optimize_lock);
04079
04080     // if ( ( AO.Optimize.debugflags&2 ) == 2 ) {
04081     //     MesPrint ("Scheme: %a", Horner_scheme.size(), &(Horner_scheme[0]));
04082     // }
04083
04084     // apply Horner scheme
04085     vector<WORD> tree = Horner_tree(optimize_expr, Horner_scheme);
04086
04087     // generate instructions, eventually with CSE
04088     vector<WORD> instr;
04089
04090     if (AO.Optimize.method == O_CSE || AO.Optimize.method == O_CSEGREEDY)
04091         instr = generate_instructions(tree, true);
04092     else
04093         instr = generate_instructions(tree, false);
04094     if (AO.Optimize.method == O_CSEGREEDY || AO.Optimize.method == O_GREEDY) {
04095         instr = merge_operators(instr, false);
04096         instr = optimize_greedy(instr, time_limit-(TimeWallClock(1)-start_time));
04097         instr = merge_operators(instr, true);
04098     }

```



```

04099     instr = optimize_greedy(instr, time_limit-(TimeWallClock(1)-start_time));
04100 }
04101 instr = merge_operators(instr, true);
04102
04103 // recycle the temporary variables
04104 instr = recycle_variables(instr);
04105
04106 // determine the quality of the code and possibly update the best code
04107 int num_oper = count_operators(instr);
04108
04109 LOCK(optimize_lock);
04110 if (num_oper < optimize_best_num_oper) {
04111     optimize_num_vars = Horner_scheme.size();
04112     optimize_best_num_oper = num_oper;
04113     optimize_best_instr = instr;
04114     optimize_best_vars = vector<WORD>(AN.poly_vars, AN.poly_vars+AN.poly_num_vars);
04115 }
04116 UNLOCK(optimize_lock);
04117
04118 // clean poly_vars, that are allocated by Horner_tree
04119 AN.poly_num_vars = 0;
04120 M_free(AN.poly_vars, "poly_vars");
04121
04122 #ifdef DEBUG
04123     MesPrint ("*** [%s, w=%w] DONE: optimize_expression_given_Horner", thetime_str().c_str());
04124 #endif
04125 }
04126
04127 /*
04128     #] optimize_expression_given_Horner :
04129     #[ PF_optimize_expression_given_Horner :
04130 */
04131 #ifdef WITHMPI
04132
04133 // Initialization.
04134 void PF_optimize_expression_given_Horner_master_init () {
04135     // Nothing to do for now.
04136 }
04137
04138 // Wait for an idle slave and return the process number.
04139 int PF_optimize_expression_given_Horner_master_next () {
04140
04141     // Find an idle slave.
04142     int next;
04143     PF_LongSingleReceive(PF_ANY_SOURCE, PF_OPT_HORNER_MSGTAG, &next, NULL);
04144     return next;
04145 }
04146
04147 // The main function on the master.
04148 void PF_optimize_expression_given_Horner_master () {
04149
04150     #ifdef DEBUG
04151         MesPrint ("*** [%s, w=%w] CALL: PF_optimize_expression_given_Horner_master",
04152             thetime_str().c_str());
04153     #endif
04154
04155     // pick a Horner scheme from the list
04156     vector<WORD> Horner_scheme = optimize_best_Horner_schemes.back();
04157     optimize_best_Horner_schemes.pop_back();
04158
04159     // Find an idle slave.
04160     int next = PF_optimize_expression_given_Horner_master_next();
04161
04162     // Send a new task to the slave.
04163     PF_PrepareLongSinglePack();
04164     int len = Horner_scheme.size();
04165     PF_LongSinglePack(&len, 1, PF_INT);
04166     PF_LongSinglePack(&Horner_scheme[0], len, PF_WORD);
04167     PF_LongSingleSend(next, PF_OPT_HORNER_MSGTAG);
04168
04169     #ifdef DEBUG
04170         MesPrint ("*** [%s, w=%w] DONE: PF_optimize_expression_given_Horner_master",
04171             thetime_str().c_str());
04172     #endif
04173 }
04174
04175 // Wait for all the slaves to finish their tasks.
04176 void PF_optimize_expression_given_Horner_master_wait () {
04177
04178     #ifdef DEBUG
04179         MesPrint ("*** [%s, w=%w] CALL: PF_optimize_expression_given_Horner_master_wait",
04180             thetime_str().c_str());
04181     #endif
04182
04183     // Wait for all the slaves.

```

```

04183     for (int i = 1; i < PF.numtasks; i++) {
04184         int next = PF_optimize_expression_given_Horner_master_next();
04185         // Send a null task.
04186         PF_PrepareLongSinglePack();
04187         int len = 0;
04188         PF_LongSinglePack(&len, 1, PF_INT);
04189         PF_LongSingleSend(next, PF_OPT_HORNER_MSGTAG);
04190     }
04191
04192     // Combine the result from all the slaves.
04193     optimize_best_num_oper = INT_MAX;
04194     for (int i = 1; i < PF.numtasks; i++) {
04195         PF_LongSingleReceive(PF_ANY_SOURCE, PF_OPT_COLLECT_MSGTAG, NULL, NULL);
04196
04197         int len;
04198
04199         // The first integer is the number of operations.
04200         PF_LongSingleUnpack(&len, 1, PF_INT);
04201
04202         if (len >= optimize_best_num_oper) continue;
04203
04204         // Update the best result.
04205         optimize_best_num_oper = len;
04206         PF_LongSingleUnpack(&len, 1, PF_INT);
04207         optimize_best_instr.resize(len);
04208         PF_LongSingleUnpack(&optimize_best_instr[0], len, PF_WORD);
04209         PF_LongSingleUnpack(&len, 1, PF_INT);
04210         optimize_best_vars.resize(len);
04211         PF_LongSingleUnpack(&optimize_best_vars[0], len, PF_WORD);
04212
04213         optimize_num_vars = optimize_best_vars.size(); // TODO
04214     }
04215
04216 #ifdef DEBUG
04217     MesPrint ("*** [%s, w=%w] DONE: PF_optimize_expression_given_Horner_master_wait",
04218             thetime_str().c_str());
04219 #endif
04220 }
04221
04222 // The main function on the slaves.
04223 void PF_optimize_expression_given_Horner_slave () {
04224
04225     #ifdef DEBUG
04226         MesPrint ("*** [%s, w=%w] CALL: PF_optimize_expression_given_Horner_slave",
04227                 thetime_str().c_str());
04228     #endif
04229
04230     optimize_best_Horner_schemes.clear();
04231     optimize_best_num_oper = INT_MAX;
04232
04233     int dummy = 0;
04234     int len;
04235
04236     for (;;) {
04237         // Ask the master for the next task.
04238         PF_PrepareLongSinglePack();
04239         PF_LongSinglePack(&dummy, 1, PF_INT);
04240         PF_LongSingleSend(MASTER, PF_OPT_HORNER_MSGTAG);
04241
04242         // Get a task from the master.
04243         PF_LongSingleReceive(MASTER, PF_OPT_HORNER_MSGTAG, NULL, NULL);
04244
04245         // Length of the task.
04246         PF_LongSingleUnpack(&len, 1, PF_INT);
04247
04248         // No task remains.
04249         if (len == 0) break;
04250
04251         // Perform the given task.
04252         optimize_best_Horner_schemes.push_back(vector<WORD>());
04253         vector<WORD> &Horner_scheme = optimize_best_Horner_schemes.back();
04254         Horner_scheme.resize(len);
04255         PF_LongSingleUnpack(&Horner_scheme[0], len, PF_WORD);
04256         optimize_expression_given_Horner ();
04257     }
04258
04259     // Send the result to the master.
04260     PF_PrepareLongSinglePack();
04261     PF_LongSinglePack(&optimize_best_num_oper, 1, PF_INT);
04262     if (optimize_best_num_oper != INT_MAX) {
04263         len = optimize_best_instr.size();
04264         PF_LongSinglePack(&len, 1, PF_INT);
04265         PF_LongSinglePack(&optimize_best_instr[0], len, PF_WORD);
04266         len = optimize_best_vars.size();
04267         PF_LongSinglePack(&len, 1, PF_INT);
04268         PF_LongSinglePack(&optimize_best_vars[0], len, PF_WORD);

```

```

04268     }
04269     PF_LongSingleSend(MASTER, PF_OPT_COLLECT_MSGTAG);
04270
04271 #ifdef DEBUG
04272     MesPrint ("*** [%s, w=%w] DONE: PF_optimize_expression_given_Horner_slave",
04273             thetime_str().c_str());
04274 #endif
04275 }
04276
04277 #endif
04278 /*
04279 #] PF_optimize_expression_given_Horner :
04280 #[ generate_output :
04281 */
04282
04292 void generate_output (const vector<WORD> &instr, int exprnr, int extraoffset, const
vector<vector<WORD> > &brackets) {
04293
04294 #ifdef DEBUG
04295     MesPrint ("*** [%s, w=%w] CALL: generate_output", thetime_str().c_str());
04296 #endif
04297
04298     GETIDENTITY;
04299     vector<WORD> output;
04300
04301     // one-indexed instead of zero-indexed
04302     extraoffset++;
04303     int num = 0;
04304     int maxsize = (int)instr.size();
04305     for (int i=0; i<maxsize; i+=instr[i+2]) num++;
04306     int *tstart = (int *)Malloca(num*sizeof(int), "nplaces");
04307     num = 0;
04308     for (int i=0; i<maxsize; i+=instr[i+2]) tstart[num++] = i;
04309     for (int j=0; j<num; j++) {
04310         int i = tstart[j];
04311
04312         // copy arguments
04313         WCOPY(AT.WorkPointer, &instr[i+3], (instr[i+2]-3));
04314
04315         // update maximal variable number
04316         AO.OptimizeResult.maxvar = MaX(AO.OptimizeResult.maxvar, instr[i]+extraoffset);
04317
04318         // renumber symbols and extrasymbols to correct values
04319         for (WORD *t=AT.WorkPointer; *t!=0; t+=*t) {
04320             if (*t == ABS(* (t+*t-1))+1) continue;
04321             if (* (t+1)==SYMBOL) * (t+3) = optimize_best_vars[* (t+3)];
04322             if (* (t+1)==EXTRASymbol) {
04323                 * (t+1) = SYMBOL;
04324                 * (t+3) = MAXVARIABLES-* (t+3)-extraoffset;
04325             }
04326         }
04327
04328         // reformat multiplication instructions, since their current
04329         // format is "expr.nr OPER_MUL length arguments", which differs
04330         // from Form's format for a product of symbols
04331         if (instr[i+1]==OPER_MUL) {
04332
04333             WORD *now=AT.WorkPointer+1;
04334             int dt;
04335             bool coeff=false;
04336             int coeff_sign=1;
04337
04338             for (WORD *t=AT.WorkPointer; *t!=0; t+=dt) {
04339                 dt = *t;
04340                 if (*t == ABS(* (t+*t-1))+1) {
04341                     // copy coefficient
04342                     memmove(AT.WorkPointer+instr[i+2], t, *t*sizeof(WORD));
04343                     coeff = true;
04344                 }
04345                 else {
04346                     // move symbol
04347                     int n = * (t+2);
04348                     memmove(now, t+1, n*sizeof(WORD));
04349                     now += n;
04350                     coeff_sign *= SGN(* (t+dt-1));
04351                 }
04352             }
04353
04354             if (coeff) {
04355                 // add existing coefficient
04356                 int n = * (AT.WorkPointer + instr[i+2]) - 1;
04357                 memmove(now, AT.WorkPointer+instr[i+2]+1, n*sizeof(WORD));
04358                 now += n;
04359             }
04360             else {
04361                 // add coefficient of one

```

```

04362         *now+=1;
04363         *now+=1;
04364         *now+=3;
04365     }
04366
04367     *(now-1) *= coeff_sign;
04368
04369     *AT.WorkPointer = now - AT.WorkPointer;
04370     *now++ = 0;
04371 }
04372
04373 // in the case of simultaneous optimization of expressions, add the
04374 // brackets to the final expression
04375 if (instr[i+1]==OPER_COMMA) {
04376     WORD *start = AT.WorkPointer + instr[i+2];
04377     WORD *now = start;
04378     int b=0;
04379     for (const WORD *t=AT.WorkPointer; *t!=0; t+=*t) {
04380         if ( ( brackets[b].size() != 0 ) && ( brackets[b][0] == 0 ) ) break;
04381         *now++ = *t + brackets[b].size();
04382         if (brackets[b].size() != 0) {
04383             memcpy(now, &brackets[b][0], brackets[b].size()*sizeof(WORD));
04384             now += brackets[b].size();
04385         }
04386         memcpy(now, t+1, (*t-1)*sizeof(WORD));
04387         now += *t-1;
04388         b++;
04389     }
04390     *now++ = 0;
04391     memmove(AT.WorkPointer, start, (now-start)*sizeof(WORD));
04392 }
04393
04394 // add the number of the extra symbol; if it is the last one,
04395 // replace the extra symbol number with the expression number
04396 if (i+instr[i+2]<(int)instr.size())
04397     output.push_back(instr[i]+extraoffset);
04398 else {
04399     output.push_back(-(exprnr+1));
04400 }
04401
04402 // add code for this symbol
04403 int n=0;
04404 while (*(AT.WorkPointer+n)!=0)
04405     n += *(AT.WorkPointer+n);
04406 n++;
04407 output.insert(output.end(), AT.WorkPointer, AT.WorkPointer+n);
04408 }
04409
04410 // trailing zero
04411 output.push_back(0);
04412
04413 // clear buffer
04414 if (AO.OptimizeResult.code != NULL)
04415     M_free(AO.OptimizeResult.code, "optimize output");
04416
04417 M_free(tstart, "nplaces");
04418
04419 // copy buffer
04420 AO.OptimizeResult.codesize = output.size();
04421 AO.OptimizeResult.code = (WORD *)Malloc1(output.size()*sizeof(WORD), "optimize output");
04422 memcpy(AO.OptimizeResult.code, &output[0], output.size()*sizeof(WORD));
04423
04424 #ifdef DEBUG
04425     MesPrint ("*** [%s, w=%w] DONE: generate_output", thetime_str().c_str());
04426 #endif
04427 }
04428
04429 /*
04430     #] generate_output :
04431     #[ generate_expression :
04432 */
04433
04441 WORD generate_expression (WORD exprnr) {
04442
04443     #ifdef DEBUG
04444         MesPrint ("*** [%s, w=%w] CALL: generate_expression", thetime_str().c_str());
04445     #endif
04446
04447     GETIDENTITY;
04448
04449     WORD *oldWorkPointer = AT.WorkPointer;
04450
04451     CBUF *C = cbuf+AC.cbunum;
04452     WORD *term = AT.WorkPointer, oldcurexpr = AR.CurExpr;
04453     POSITION position;
04454     EXPRESSIONS e = Expressions+exprnr;
04455     SetScratch(AR.infile, &(e->onfile));

```

```

04456
04457     if ( GetTerm(BHEAD term) <= 0 ) {
04458         MesPrint("Expression %d has problems in scratchfile",exprnr);
04459         Terminate(-1);
04460     }
04461     SeekScratch(AR.outfile,&position);
04462     e->onfile = position;
04463     if ( PutOut(BHEAD term,&position,AR.outfile,0) < 0 ) {
04464         MesPrint("Expression %d has problems in output scratchfile",exprnr);
04465         Terminate(-1);
04466     }
04467
04468     AR.CurExpr = exprnr;
04469     NewSort(BHEAD0);
04470
04471     // scan for the original expression (marked by *t<0) and give the
04472     // terms to Generator
04473     WORD *t = AO.OptimizeResult.code;
04474     {
04475         WORD old = AR.Cnumlhs; AR.Cnumlhs = 0;
04476         // We can use the remaining part of the WorkSpace for every term that
04477         // goes through Generator. We have WorkSpace problems here if we use
04478         // the (modified) AT.WorkPointer every time in the loop.
04479         WORD* currentWorkPointer = AT.WorkPointer;
04480         while (*t!=0) {
04481             bool is_expr = *t < 0;
04482             t++;
04483             while (*t!=0) {
04484                 if (is_expr) {
04485                     memcpy(currentWorkPointer, t, *t*sizeof(WORD));
04486                     Generator(BHEAD currentWorkPointer, C->numlhs);
04487                 }
04488                 t+=*t;
04489             }
04490             t++;
04491         }
04492         AR.Cnumlhs = old;
04493     }
04494
04495     // final sorting
04496     if (EndSort(BHEAD NULL,0) < 0) {
04497         LowerSortLevel();
04498         Terminate(-1);
04499     }
04500
04501     AT.WorkPointer = oldWorkPointer;
04502     AR.CurExpr = oldcurexpr;
04503
04504 #ifdef DEBUG
04505     MesPrint ("*** [%s, w=%w] DONE: generate_expression", thetime_str().c_str());
04506 #endif
04507     return 0;
04508 }
04509
04510 /*
04511 #] generate_expression :
04512 #[ optimize_print_code :
04513 */
04514
04525 void optimize_print_code (int print_expr) {
04526
04527 #ifdef DEBUG
04528     MesPrint ("*** [%s, w=%w] CALL: optimize_print_code", thetime_str().c_str());
04529 #endif
04530     if ( ( AO.Optimize.debugflags & 1 ) != 0 ) {
04531         /*
04532          * The next code is for debugging purposes. We may want the statements
04533          * in reverse order to substitute them all back.
04534          * Jan used a Mathematica program to do this. Here we make that
04535          * Format Ox,debugflag=1;
04536          * Creates reverse order during printing.
04537          * All we have to do is put id in front of the statements. This is done
04538          * in PrintExtraSymbol.
04539          */
04540         WORD *t = AO.OptimizeResult.code;
04541         WORD num = 0; // First we count the number of objects.
04542         while (*t!=0) {
04543             num++;
04544             t++; while (*t!=0) t+=*t; t++;
04545         }
04546         WORD **tstart = (WORD **)Malloca1(num*sizeof(WORD *),"print objects");
04547         t = AO.OptimizeResult.code; num = 0; // Now we get the addresses
04548         while (*t!=0) {
04549             tstart[num++] = t;
04550             t++; while (*t!=0) t+=*t; t++;
04551         }

```

```

04552         // Flip the addresses
04553         int halfnum = num/2;
04554         for (int i=0; i<halfnum; i++) { swap(tstart[i],tstart[num-1-i]); }
04555         for ( int i = 0; i < num; i++ ) {
04556             t = tstart[i];
04557             if (*t > 0)
04558                 PrintExtraSymbol(*t, t+1, EXTRASYMBOL);
04559             else if (print_expr)
04560                 PrintExtraSymbol(-*t-1, t+1, EXPRESSIONNUMBER);
04561         }
04562         CBUF *C = cbuf + AM.sbufnum;
04563         if (C->numrhs >= AO.OptimizeResult.minvar)
04564             PrintSubtermList(AO.OptimizeResult.minvar, C->numrhs);
04565     }
04566     else {
04567         // print extra symbols from ConvertToPoly in optimization
04568         CBUF *C = cbuf + AM.sbufnum;
04569         if (C->numrhs >= AO.OptimizeResult.minvar)
04570             PrintSubtermList(AO.OptimizeResult.minvar, C->numrhs);
04571
04572         WORD *t = AO.OptimizeResult.code;
04573         while (*t!=0) {
04574             if (*t > 0) {
04575                 PrintExtraSymbol(*t, t+1, EXTRASYMBOL);
04576             }
04577             else if (print_expr)
04578                 PrintExtraSymbol(-*t-1, t+1, EXPRESSIONNUMBER);
04579             t++;
04580             while (*t!=0) t+=*t;
04581             t++;
04582         }
04583     }
04584
04585 #ifdef DEBUG
04586     MesPrint ("*** [%s, w=%w] DONE: optimize_print_code", thetime_str().c_str());
04587 #endif
04588 }
04589
04590 /*
04591 #] optimize_print_code :
04592 #[ Optimize :
04593 */
04594
04638 int Optimize (WORD exprnr, int do_print) {
04639
04640 #if defined(WITHMPI) && (defined(DEBUG) || defined(DEBUG_MORE) || defined(DEBUG_MCTS) ||
    defined(DEBUG_GREEDY))
04641     // set AS.printflag negative temporary.
04642     struct save_printflag {
04643         save_printflag() {
04644             oldprintflag = AS.printflag;
04645             AS.printflag = -1;
04646         }
04647         ~save_printflag() {
04648             AS.printflag = oldprintflag;
04649         }
04650         int oldprintflag;
04651     } save_printflag_;
04652 #endif
04653
04654 #ifdef DEBUG
04655     MesPrint ("*** [%s, w=%w] CALL: Optimize", thetime_str().c_str());
04656     MesPrint ("*** %"); PrintRunningTime();
04657 #endif
04658
04659 #ifdef WITHPTHREADS
04660     optimize_lock = dummylock;
04661 #endif
04662
04663     AO.OptimizeResult.minvar = (cbuf + AM.sbufnum)->numrhs + 1;
04664
04665     if (get_expression(exprnr) < 0) return -1;
04666     vector<vector<WORD>> > brackets = get_brackets();
04667
04668 #ifdef DEBUG
04669 #ifdef WITHMPI
04670     if (PF.me == MASTER)
04671 #endif
04672     MesPrint ("*** runtime after preparing the expression: %"); PrintRunningTime();
04673 #endif
04674
04675     if (optimize_expr[0]==0 ||
04676         (optimize_expr[optimize_expr[0]]==0 &&
04677          optimize_expr[0]==ABS(optimize_expr[optimize_expr[0]-1])+1) ||
04677         (optimize_expr[optimize_expr[0]]==0 && optimize_expr[0]==8 &&
04678          optimize_expr[5]==1 && optimize_expr[6]==1 && ABS(optimize_expr[7])==3)) {
04679         if (AO.OptimizeResult.code != NULL)

```

```

04680         M_free(AO.OptimizeResult.code, "optimize output");
04681
04682         // zero terms or one trivial term (number or +/-variable), so no
04683         // optimization, so copy expression; special case because without
04684         // operators the optimization crashes
04685         AO.OptimizeResult.code = (WORD *)Malloc1((optimize_expr[0]+3)*sizeof(WORD), "optimize
output");
04686         AO.OptimizeResult.code[0] = -(exprnr+1);
04687         memcpy(AO.OptimizeResult.code+1, optimize_expr, (optimize_expr[0]+1)*sizeof(WORD));
04688         AO.OptimizeResult.code[optimize_expr[0]+2] = 0;
04689     }
04690     else {
04691         // find Horner scheme(s)
04692         optimize_best_Horner_schemes.clear();
04693         if (AO.Optimize.horner == O_OCCURRENCE) {
04694             if (AO.Optimize.hornerdirection != O_BACKWARD)
04695                 optimize_best_Horner_schemes.push_back(occurrence_order(optimize_expr, false));
04696             if (AO.Optimize.hornerdirection != O_FORWARD)
04697                 optimize_best_Horner_schemes.push_back(occurrence_order(optimize_expr, true));
04698         }
04699         else {
04700             if (AO.Optimize.horner == O_SIMULATED_ANNEALING) {
04701                 optimize_best_Horner_schemes.push_back(simulated_annealing());
04702             }
04703             else {
04704                 mcts_best_schemes.clear();
04705                 if (AO.inscheme) {
04706                     optimize_best_Horner_schemes.push_back(vector<WORD>(AO.schemenum));
04707                     for (int i=0; i < AO.schemenum; i++)
04708                         optimize_best_Horner_schemes[0][i] = AO.inscheme[i];
04709                 }
04710                 else {
04711                     for (int i = 0; i < AO.Optimize.mctsnumrepeat; i++)
04712                         find_Horner_MCTS();
04713                     // generate results
04714                     for (set<pair<int, vector<WORD>> >>::iterator i=mcts_best_schemes.begin();
i!=mcts_best_schemes.end(); i++) {
04715                         optimize_best_Horner_schemes.push_back(i->second);
04716                     #ifdef DEBUG_MCTS
04717                         MesPrint ("%a -> %d", i->second.size(), &i->second[0], i->first);
04718                     #endif
04719                 }
04720             }
04721             // clear the tree by making a new empty one.
04722             mcts_root = tree_node();
04723         }
04724     }
04725     #ifdef DEBUG
04726     #ifdef WITHMPI
04727         if (PF.me == MASTER)
04728     #endif
04729     MesPrint ("*** runtime after Horner: %"); PrintRunningTime();
04730     #endif
04731
04732     #ifdef WITHMPI
04733         if (PF.me == MASTER) {
04734             PF_optimize_expression_given_Horner_master_init();
04735         }
04736     #endif
04737     // find best Horner scheme and results
04738     optimize_best_num_oper = INT_MAX;
04739
04740     int imax = (int)optimize_best_Horner_schemes.size();
04741
04742     for (int i=0; i<imax; i++) {
04743         #if defined(WITHPTHREADS)
04744             if (AM.totalnumberofthreads > 1)
04745                 optimize_expression_given_Horner_threaded();
04746             else
04747         #elif defined(WITHMPI)
04748             if (PF.numtasks > 1)
04749                 PF_optimize_expression_given_Horner_master();
04750             else
04751         #endif
04752                 optimize_expression_given_Horner();
04753     }
04754
04755     #ifdef WITHMPI
04756         PF_optimize_expression_given_Horner_master_wait();
04757     }
04758     else {
04759         if (PF.numtasks > 1)
04760             PF_optimize_expression_given_Horner_slave();
04761     }
04762     #endif
04763
04764     #ifdef WITHPTHREADS

```

```

04765     MasterWaitAll();
04766 #endif
04767     // format results, then print it (for "Print") or modify
04768     // expression (for "#Optimize")
04769 #ifdef WITHMPI
04770     if (PF.me == MASTER)
04771 #endif
04772     generate_output(optimize_best_instr, exprnr, cbuf[AM.sbufnum].numrhs, brackets);
04773 #ifdef WITHMPI
04774     else {
04775         // non-null dummy code for slaves
04776         if (AO.OptimizeResult.code == NULL) {
04777             AO.OptimizeResult.code = (WORD *)Malloca(sizeof(WORD), "optimize output");
04778         }
04779     }
04780 #endif
04781 }
04782
04783 #ifdef WITHMPI
04784 if (PF.me == MASTER) {
04785     PF_PrepPack();
04786     PF_Pack(&AO.OptimizeResult.minvar, 1, PF_WORD);
04787     PF_Pack(&AO.OptimizeResult.maxvar, 1, PF_WORD);
04788 }
04789 PF_Broadcast();
04790 if (PF.me != MASTER) {
04791     PF_Unpack(&AO.OptimizeResult.minvar, 1, PF_WORD);
04792     PF_Unpack(&AO.OptimizeResult.maxvar, 1, PF_WORD);
04793 }
04794 #endif
04795
04796 // set preprocessor variables
04797 char str[100];
04798 snprintf(str, 100, "%d", AO.OptimizeResult.minvar);
04799 PutPreVar((UBYTE *) "optimminvar_", (UBYTE *) str, 0, 1);
04800 snprintf(str, 100, "%d", AO.OptimizeResult.maxvar);
04801 PutPreVar((UBYTE *) "optimmaxvar_", (UBYTE *) str, 0, 1);
04802
04803 if (do_print) {
04804 #ifdef WITHMPI
04805     if (PF.me == MASTER)
04806 #endif
04807     optimize_print_code(1);
04808     ClearOptimize();
04809 }
04810 else {
04811 #ifdef WITHMPI
04812     if (PF.me == MASTER)
04813 #endif
04814     generate_expression(exprnr);
04815 }
04816
04817 #ifdef WITHMPI
04818 if (PF.me == MASTER) {
04819 #endif
04820
04821     if (AO.Optimize.printstats > 0) {
04822         char str[20];
04823         MesPrint("");
04824         count_operators(optimize_expr, true);
04825         int numop = count_operators(optimize_best_instr, true);
04826         snprintf(str, 20, "%d", numop);
04827         PutPreVar((UBYTE *) "optimvalue_", (UBYTE *) str, 0, 1);
04828     }
04829     else {
04830         char str[20];
04831         int numop = count_operators(optimize_best_instr, false);
04832         snprintf(str, 20, "%d", numop);
04833         PutPreVar((UBYTE *) "optimvalue_", (UBYTE *) str, 0, 1);
04834     }
04835
04836     if ( ( AO.Optimize.schemeflags&1 ) == 1 ) {
04837         GETIDENTITY
04838         UBYTE *OutScr, *Out, *old1 = AO.OutputLine, *old2 = AO.OutFill;
04839         int i, sym;
04840         AO.OutputLine = AO.OutFill = (UBYTE *) AT.WorkPointer;
04841         FiniLine();
04842         OutScr = (UBYTE *) AT.WorkPointer + ( TOLONG(AT.WorkTop) - TOLONG(AT.WorkPointer) ) / 2;
04843         TokenToLine((UBYTE *) " Scheme selected: ");
04844         for ( i = 0; i < optimize_num_vars; i++ ) {
04845             Out = OutScr;
04846             sym = optimize_best_vars[i];
04847             if ( i > 0 ) TokenToLine((UBYTE *) ",");
04848             if ( sym < NumSymbols ) {
04849                 StrCopy(FindSymbol(sym), OutScr);
04850                 StrCopy(VARNAME(sym), OutScr);
04851             }

```



```

04852         else {
04853             Out = StrCopy((UBYTE *)AC.extrasym,Out);
04854             if ( AC.extrasymbols == 0 ) {
04855                 Out = NumCopy((MAXVARIABLES-sym),Out);
04856                 Out = StrCopy((UBYTE *)"_",Out);
04857             }
04858             else if ( AC.extrasymbols == 1 ) {
04859                 Out = AddArrayIndex((MAXVARIABLES-sym),Out);
04860             }
04861         }
04862         TokenToLine(OutScr);
04863     }
04864     TokenToLine((UBYTE *)";");
04865     FiniLine();
04866     AO.OutFill = old2;
04867     AO.OutputLine = old1;
04868 }
04869
04870 {
04871     GETIDENTITY
04872     UBYTE *OutScr, *Out, *outstring = 0;
04873     int i, sym;
04874     AO.OutputLine = AO.OutFill = (UBYTE *)AT.WorkPointer;
04875     OutScr = (UBYTE *)AT.WorkPointer + ( TOLONG(AT.WorkTop) - TOLONG(AT.WorkPointer) ) /2;
04876     for ( i = 0; i < optimize_num_vars; i++ ) {
04877         Out = OutScr;
04878         sym = optimize_best_vars[i];
04879         if ( sym < NumSymbols ) {
04880             StrCopy(FindSymbol(sym),OutScr);
04881             /* StrCopy(VARNAME(symbols,sym),OutScr); */
04882         }
04883         else {
04884             Out = StrCopy((UBYTE *)AC.extrasym,Out);
04885             if ( AC.extrasymbols == 0 ) {
04886                 Out = NumCopy((MAXVARIABLES-sym),Out);
04887                 Out = StrCopy((UBYTE *)"_",Out);
04888             }
04889             else if ( AC.extrasymbols == 1 ) {
04890                 Out = AddArrayIndex((MAXVARIABLES-sym),Out);
04891             }
04892         }
04893         outstring = AddToString(outstring,OutScr,1);
04894     }
04895     if ( outstring == 0 ) {
04896         PutPreVar((UBYTE *)"optimscheme_",(UBYTE *)"",0,1);
04897     }
04898     else {
04899         PutPreVar((UBYTE *)"optimscheme_",(UBYTE *)outstring,0,1);
04900         M_free(outstring,"AddToString");
04901     }
04902 }
04903
04904 #ifndef WITHMPI
04905 }
04906
04907 // synchronize optimvalue_ and optimscheme_
04908 if ( PF.me == MASTER ) {
04909     UBYTE *value;
04910     int bytes;
04911
04912     PF_PrepareLongMultiPack();
04913
04914     value = GetPreVar((UBYTE *)"optimvalue_", WITHERROR);
04915     bytes = strlen((char *)value);
04916     PF_LongMultiPack(&bytes, 1, PF_INT);
04917     PF_LongMultiPack(value, bytes, PF_BYTE);
04918
04919     value = GetPreVar((UBYTE *)"optimscheme_", WITHERROR);
04920     bytes = strlen((char *)value);
04921     PF_LongMultiPack(&bytes, 1, PF_INT);
04922     PF_LongMultiPack(value, bytes, PF_BYTE);
04923 }
04924 PF_LongMultiBroadcast();
04925 if ( PF.me != MASTER ) {
04926     static vector<UBYTE> prevarbuf;
04927     UBYTE *value;
04928     int bytes;
04929
04930     PF_LongMultiUnpack(&bytes, 1, PF_INT);
04931     prevarbuf.reserve(bytes + 1);
04932     value = &prevarbuf[0];
04933     PF_LongMultiUnpack(value, bytes, PF_BYTE);
04934     value[bytes] = '\0'; // null terminator
04935     PutPreVar((UBYTE *)"optimvalue_", value, NULL, 1);
04936
04937     PF_LongMultiUnpack(&bytes, 1, PF_INT);
04938     prevarbuf.reserve(bytes + 1);

```

```

04939         value = &prevarbuf[0];
04940         PF_LongMultiUnpack(value, bytes, PF_BYTE);
04941         value[bytes] = '\0'; // null terminator
04942         PutPreVar((UBYTE *) "optimscheme_", value, NULL, 1);
04943     }
04944 #endif
04945
04946     // cleanup
04947     M_free(optimize_expr, "LoadOptim");
04948
04949 #ifdef DEBUG
04950     MesPrint ("*** [%s, w=%w] DONE: Optimize", thetime_str().c_str());
04951 #endif
04952
04953     return 0;
04954 }
04955
04956 /*
04957  #] Optimize :
04958  #[ ClearOptimize :
04959 */
04960
04975 int ClearOptimize()
04976 {
04977     char str[20];
04978     WORD numexpr, *w;
04979     int error = 0;
04980     if ( AO.OptimizeResult.code != NULL ) {
04981         M_free(AO.OptimizeResult.code, "optimize output");
04982         AO.OptimizeResult.code = NULL;
04983         AO.OptimizeResult.codesize = 0;
04984         PutPreVar((UBYTE *) "optimminvar_", (UBYTE *) ("0"), 0, 1);
04985         PutPreVar((UBYTE *) "optimmaxvar_", (UBYTE *) ("0"), 0, 1);
04986         PruneExtraSymbols(AO.OptimizeResult.minvar-1);
04987         cbuf[AM.sbufnum].numrhs = AO.OptimizeResult.minvar-1;
04988         AO.OptimizeResult.minvar = AO.OptimizeResult.maxvar = 0;
04989     }
04990     if ( AO.OptimizeResult.nameofexpr != NULL ) {
04991         /*
04992             We have to pick up the expression by its name. Numbers may change.
04993             Note that this requires that when we overwrite an expression, we
04994             check that it is not an optimized expression. See execute.c and
04995             comexpr.c
04996         */
04997         if ( GetName(AC.exprnames, AO.OptimizeResult.nameofexpr, &numexpr, NOAUTO) != CEXPRESSION ) {
04998             MesPrint("@Internal error while clearing optimized expression %s",
04999                     AO.OptimizeResult.nameofexpr);
05000             Terminate(-1);
05001         }
05002         M_free(AO.OptimizeResult.nameofexpr, "optimize expression name");
05003         AO.OptimizeResult.nameofexpr = NULL;
05004         w = &(Expressions[numexpr].status);
05005         *w = SetExprCases(DROP, 1, *w);
05006         if ( *w < 0 ) error = 1;
05007     }
05008     sprintf (str, 20, "%d", cbuf[AM.sbufnum].numrhs);
05009     PutPreVar (AM.oldnumextrasyms, (UBYTE *) str, 0, 1);
05010     PutPreVar ((UBYTE *) "optimvalue_", (UBYTE *) ("0"), 0, 1);
05011     PutPreVar ((UBYTE *) "optimscheme_", (UBYTE *) ("0"), 0, 1);
05012     return(error);
05013 }
05014 /*
05015  #] ClearOptimize :
05016 */

```

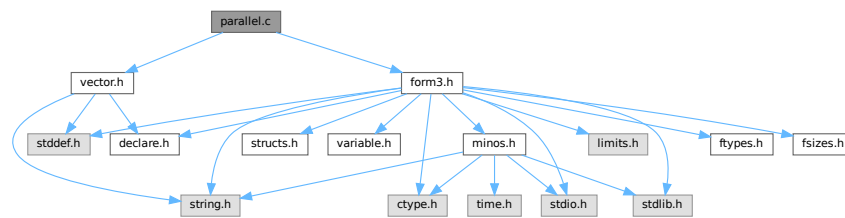
4.94 parallel.c File Reference

```

#include "form3.h"
#include "vector.h"

```

Include dependency graph for parallel.c:



Data Structures

- struct [NoDe](#)
- struct [dollar_buf](#)
- struct [bufIPstruct](#)

Macros

- #define [PRINTFBUF](#)(TEXT, TERM, SIZE) {}
- #define [SWAP](#)(x, y)
- #define [PACK_LONG](#)(p, n)
- #define [UNPACK_LONG](#)(p, n)
- #define [CHECK](#)(condition) [_CHECK](#)(condition, __FILE__, __LINE__)
- #define [_CHECK](#)(condition, file, line) [__CHECK](#)(condition, file, line)
- #define [__CHECK](#)(condition, file, line)
- #define [DBGOUT](#)(lv1, lv2, a) do { if (lv1 >= lv2) { printf a; fflush(stdout); } } while (0)
- #define [DBGOUT_NINTERMS](#)(lv, a)
- #define [PF_STATS_SIZE](#) 5
- #define [PF_SNDFILEBUFSIZE](#) 4096
- #define [recvBuffer](#) logBuffer /* (master) The buffer for receiving messages. */

Typedefs

- typedef struct [NoDe](#) [NODE](#)
- typedef struct [bufIPstruct](#) [bufIPstruct_t](#)

Functions

- LONG [PF_RealTime](#) (int)
- int [PF_LibInit](#) (int *, char ***)
- int [PF_LibTerminate](#) (int)
- int [PF_Probe](#) (int *)
- int [PF_RecvWbuf](#) (WORD *, LONG *, int *)
- int [PF_IRecvRbuf](#) ([PF_BUFFER](#) *, int, int)
- int [PF_WaitRbuf](#) ([PF_BUFFER](#) *, int, LONG *)
- int [PF_RawSend](#) (int dest, void *buf, LONG l, int tag)
- LONG [PF_RawRecv](#) (int *src, void *buf, LONG thesize, int *tag)
- int [PF_RawProbe](#) (int *src, int *tag, int *bytesize)
- int [PF_EndSort](#) (void)

- WORD [PF_Deferred](#) (WORD *term, WORD level)
- int [PF_Processor](#) (EXPRESSIONS e, WORD i, WORD LastExpression)
- int [PF_Init](#) (int *argc, char ***argv)
- int [PF_Terminate](#) (int errorcode)
- LONG [PF_GetSlaveTimes](#) (void)
- LONG [PF_BroadcastNumber](#) (LONG x)
- void [PF_BroadcastBuffer](#) (WORD **buffer, LONG *length)
- int [PF_BroadcastString](#) (UBYTE *str)
- int [PF_BroadcastPreDollar](#) (WORD **dbuffer, LONG *newsize, int *numterms)
- int [PF_CollectModifiedDollars](#) (void)
- int [PF_BroadcastModifiedDollars](#) (void)
- int [PF_BroadcastRedefinedPreVars](#) (void)
- int [PF_StoreInsideInfo](#) (void)
- int [PF_RestoreInsideInfo](#) (void)
- int [PF_BroadcastCBuf](#) (int bufnum)
- int [PF_BroadcastExpFlags](#) (void)
- int [PF_BroadcastExpr](#) (EXPRESSIONS e, FILEHANDLE *file)
- int [PF_BroadcastRHS](#) (void)
- int [PF_InParallelProcessor](#) (void)
- int [PF_SendFile](#) (int to, FILE *fd)
- int [PF_RecvFile](#) (int from, FILE *fd)
- void [PF_MLock](#) (void)
- void [PF_MUnlock](#) (void)
- LONG [PF_WriteFileToFile](#) (int handle, UBYTE *buffer, LONG size)
- void [PF_FlushStdOutBuffer](#) (void)
- void [PF_FreeErrorMessageBuffers](#) (void)

Variables

- [PARALLELVARs PF](#)

4.94.1 Detailed Description

Message passing library independent functions of parform

This file contains functions needed for the parallel version of form3 these functions need no real link to the message passing libraries, they only need some interface dependent preprocessor definitions (check [parallel.h](#)). So there still need two different objectfiles to be compiled for mpi and pvm!

Definition in file [parallel.c](#).

4.94.2 Macro Definition Documentation

4.94.2.1 PRINTFBUF

```
#define PRINTFBUF(
    TEXT,
    TERM,
    SIZE ) {}
```

Definition at line 117 of file [parallel.c](#).

4.94.2.2 SWAP

```
#define SWAP(
    x,
    y )
```

Value:

```
do { \
    char swap_tmp__[sizeof(x) == sizeof(y) ? (int)sizeof(x) : -1]; \
    memcpy(swap_tmp__, &y, sizeof(x)); \
    memcpy(&y, &x, sizeof(x)); \
    memcpy(&x, swap_tmp__, sizeof(x)); \
} while (0)
```

Swaps the variables *x* and *y*. If `sizeof(x) != sizeof(y)` then a compilation error will occur. A set of `memcpy` calls with constant sizes is expected to be inlined by the optimisation.

Definition at line 124 of file [parallel.c](#).

4.94.2.3 PACK_LONG

```
#define PACK_LONG(
    p,
    n )
```

Value:

```
do { \
    *p)++ = (UWORD)((ULONG)(n) & (ULONG)WORDMASK); \
    *p)++ = (UWORD)((ULONG)(n) >> BITSINWORD) & (ULONG)WORDMASK); \
} while (0)
```

Packs a LONG value *n* to a WORD buffer *p*.

Definition at line 135 of file [parallel.c](#).

4.94.2.4 UNPACK_LONG

```
#define UNPACK_LONG(
    p,
    n )
```

Value:

```
do { \
    (n) = (LONG)((((ULONG)(p)[1] & (ULONG)WORDMASK) << BITSINWORD) | ((ULONG)(p)[0] & (ULONG)WORDMASK)); \
    (p) += 2; \
} while (0)
```

Unpacks a LONG value *n* from a WORD buffer *p*.

Definition at line 144 of file [parallel.c](#).

4.94.2.5 CHECK

```
#define CHECK(
    condition ) _CHECK(condition, __FILE__, __LINE__)
```

A simple check for unrecoverable errors.

Definition at line 153 of file [parallel.c](#).

4.94.2.6 _CHECK

```
#define _CHECK(  
    condition,  
    file,  
    line ) __CHECK(condition, file, line)
```

Definition at line 154 of file [parallel.c](#).

4.94.2.7 __CHECK

```
#define __CHECK(  
    condition,  
    file,  
    line )
```

Value:

```
do { \  
    if ( !(condition) ) { \  
        Error0("Fatal error at " file ":" #line); \  
        Terminate(-1); \  
    } \  
} while (0)
```

Definition at line 155 of file [parallel.c](#).

4.94.2.8 DBGOUT

```
#define DBGOUT(  
    lv1,  
    lv2,  
    a ) do { if ( lv1 >= lv2 ) { printf a; fflush(stdout); } } while (0)
```

Definition at line 166 of file [parallel.c](#).

4.94.2.9 DBGOUT_NINTERMS

```
#define DBGOUT_NINTERMS(  
    lv,  
    a )
```

Definition at line 169 of file [parallel.c](#).

4.94.2.10 PF_STATS_SIZE

```
#define PF_STATS_SIZE 5
```

Definition at line 178 of file [parallel.c](#).

4.94.2.11 PF_SNDFILEBUFSIZE

```
#define PF_SNDFILEBUFSIZE 4096
```

Definition at line 4212 of file [parallel.c](#).

4.94.2.12 recvBuffer

```
#define recvBuffer logBuffer /* (master) The buffer for receiving messages. */
```

Definition at line 4320 of file [parallel.c](#).

4.94.3 Typedef Documentation

4.94.3.1 NODE

```
typedef struct NoDe NODE
```

A node for the tree of losers in the final sorting on the master.

4.94.4 Function Documentation

4.94.4.1 PF_RealTime()

```
LONG PF_RealTime (  
    int i )
```

Returns the realtime in 1/100 sec. as a LONG.

Parameters

<i>i</i>	the timer will be reset if <code>i == 0</code> .
----------	--

Returns

the real elapsed time in 1/100 second.

Definition at line 101 of file [mpi.c](#).

Referenced by [PF_Init\(\)](#).

4.94.4.2 PF_LibInit()

```
int PF_LibInit (  
    int * argcp,  
    char *** argvp )
```

Performs all library dependent initializations.

Parameters

<i>argcp</i>	pointer to the number of arguments.
<i>argvp</i>	pointer to the arguments.

Returns

0 if OK, nonzero on error.

Definition at line 123 of file [mpi.c](#).

Referenced by [PF_Init\(\)](#).

4.94.4.3 PF_LibTerminate()

```
int PF_LibTerminate (
    int error )
```

Exits mpi, when there is an error either indicated or happening, returnvalue is 1, else returnvalue is 0.

Parameters

<i>error</i>	an error code.
--------------	----------------

Returns

0 if OK, nonzero on error.

Definition at line 209 of file [mpi.c](#).

Referenced by [PF_Terminate\(\)](#).

4.94.4.4 PF_Probe()

```
int PF_Probe (
    int * src )
```

Probes the next incoming message. If `src == PF_ANY_SOURCE` this operation is blocking, otherwise nonblocking.

Parameters

<i>in, out</i>	<i>src</i>	the source process number. In output, the process number of actual found source.
----------------	------------	--

Returns

the tag value of the next incoming message if found, 0 if a nonblocking probe (input `src != PF_ANY_SOURCE`) did not find any messages. A negative returned value indicates an error.

Definition at line 230 of file [mpi.c](#).

4.94.4.5 PF_RecvWbuf()

```
int PF_RecvWbuf (
    WORD * b,
    LONG * s,
    int * src )
```

Blocking receive of a `WORD` buffer.

Parameters

<i>out</i>	<i>b</i>	the buffer to store the received data.
<i>in, out</i>	<i>s</i>	the size of the buffer. The output value is the actual size of the received data.
<i>in, out</i>	<i>src</i>	the source process number. The output value is the process number of actual source.

Returns

the received message tag. A negative value indicates an error.

Definition at line 337 of file [mpi.c](#).

4.94.4.6 PF_IRecvRbuf()

```
int PF_IRecvRbuf (
    PF_BUFFER * r,
    int bn,
    int from )
```

Posts nonblocking receive for the active receive buffer. The buffer is filled from `full` to `stop`.

Parameters

<i>r</i>	the PF_BUFFER struct for the nonblocking receive.
<i>bn</i>	the index of the cyclic buffer.
<i>from</i>	the source process number.

Returns

0 if OK, nonzero on error.

Definition at line 366 of file [mpi.c](#).

4.94.4.7 PF_WaitRbuf()

```
int PF_WaitRbuf (
    PF_BUFFER * r,
    int bn,
    LONG * size )
```

Waits for the buffer *bn* to finish a pending nonblocking receive. It returns the received tag and in **size* the number of field received. If the receive is already finished, just return the flag and size, else wait for it to finish, but also check for other pending receives.

Parameters

	<i>r</i>	the PF_BUFFER struct for the pending nonblocking receive.
	<i>bn</i>	the index of the cyclic buffer.
out	<i>size</i>	the actual size of received data.

Returns

the received message tag. A negative value indicates an error.

Definition at line [400](#) of file [mpi.c](#).

4.94.4.8 PF_RawSend()

```
int PF_RawSend (
    int dest,
    void * buf,
    LONG l,
    int tag )
```

Sends *l* bytes from *buf* to *dest*. Returns 0 on success, or -1.

Parameters

<i>dest</i>	the destination process number.
<i>buf</i>	the send buffer.
<i>l</i>	the size of data in the send buffer in bytes.
<i>tag</i>	the message tag.

Returns

0 if OK, nonzero on error.

Definition at line [463](#) of file [mpi.c](#).

Referenced by [PF_MUnlock\(\)](#), and [PF_SendFile\(\)](#).

4.94.4.9 PF_RawRecv()

```
LONG PF_RawRecv (
    int * src,
    void * buf,
    LONG thesize,
    int * tag )
```

Receives not more than *thesize* bytes from *src*, returns the actual number of received bytes, or -1 on failure.

Parameters

in, out	<i>src</i>	the source process number. In output, that of the actual received message.
out	<i>buf</i>	the receive buffer.
	<i>thesize</i>	the size of the receive buffer in bytes.
out	<i>tag</i>	the message tag of the actual received message.

Returns

the actual sizeof received data in bytes, or -1 on failure.

Definition at line 484 of file [mpi.c](#).

Referenced by [PF_RecvFile\(\)](#).

4.94.4.10 PF_RawProbe()

```
int PF_RawProbe (
    int * src,
    int * tag,
    int * bytesize )
```

Probes an incoming message.

Parameters

in, out	<i>src</i>	the source process number. In output, that of the actual received message.
in, out	<i>tag</i>	the message tag. In output, that of the actual received message.
out	<i>bytesize</i>	the size of incoming data in bytes.

Returns

0 if OK, nonzero on error.

Definition at line 508 of file [mpi.c](#).

4.94.4.11 PF_EndSort()

```
int PF_EndSort (
    void )
```

Finishes a master sorting with collecting terms from slaves. Called by [EndSort\(\)](#).

If this is not the masterprocess, just initialize the sendbuffers and return 0, else [PF_EndSort\(\)](#) sends the rest of the terms in the sendbuffer to the next slave and a dummy message to all slaves with tag PF_ENDSORT_MSGTAG. Then it receives the sorted terms, sorts them using a recursive 'tree of losers' ([PF_GetLoser\(\)](#)) and writes them to the outputfile.

Returns

1 if the sorting on the master was done. 0 if [EndSort\(\)](#) still must perform a regular sorting because it is not at the ground level or not on the master or in the sequential mode or in the InParallel mode. -1 if an error occurred.

Remarks

The slaves will send the sorted terms back to the master in the regular sorting (after the initialization of the send buffer in [PF_EndSort\(\)](#)). See [PutOut\(\)](#) and [FlushOut\(\)](#).

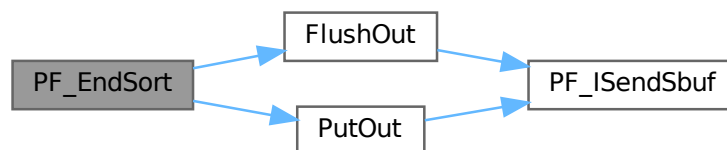
This function has been changed such that when it returns 1, AM.S0->TermsLeft is set correctly. But AM.S0->GenTerms is not set: it will be set after collecting the statistics from the slaves at the end of [PF_Processor\(\)](#). (TU 30 Jun 2011)

Definition at line 864 of file [parallel.c](#).

References [FlushOut\(\)](#), and [PutOut\(\)](#).

Referenced by [EndSort\(\)](#).

Here is the call graph for this function:

**4.94.4.12 PF_Deferred()**

```
WORD PF_Deferred (
    WORD * term,
    WORD level )
```

Replaces [Deferred\(\)](#) on the slaves.

Parameters

<i>term</i>	the term that must be multiplied by the contents of the current bracket.
<i>level</i>	the compiler level.

Returns

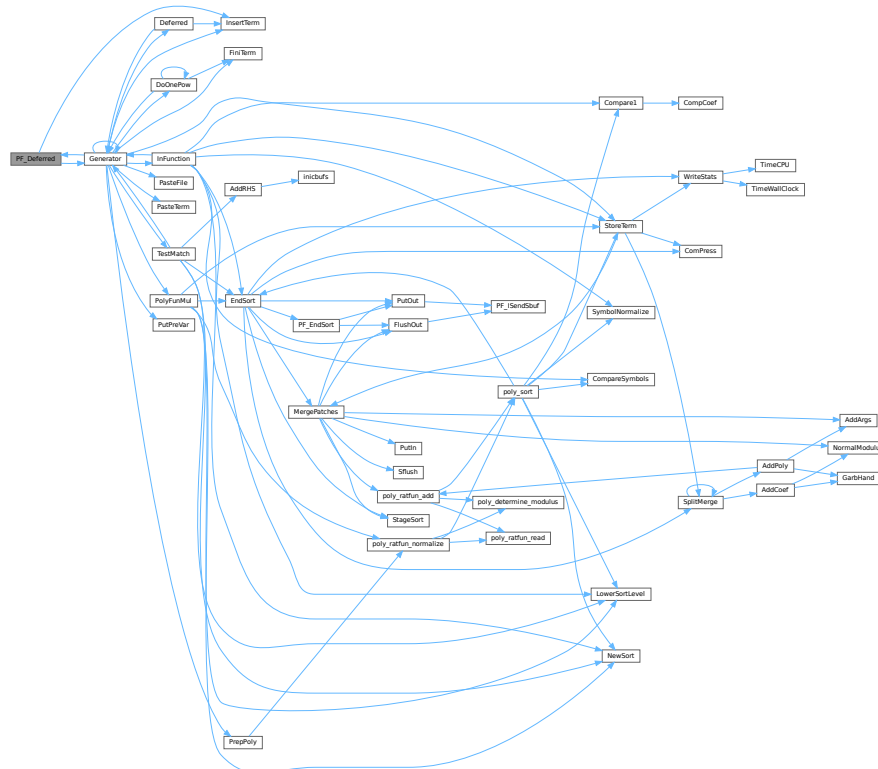
0 if OK, nonzero on error.

Definition at line 1208 of file [parallel.c](#).

References [Generator\(\)](#), and [InsertTerm\(\)](#).

Referenced by [Generator\(\)](#).

Here is the call graph for this function:



4.94.4.13 PF_Processor()

```
int PF_Processor (
    EXPRESSIONS e,
    WORD i,
    WORD LastExpression )
```

Replaces parts of [Processor\(\)](#) on the masters and slaves. On the master [PF_Processor\(\)](#) is responsible for proper distribution of terms from the input file to the slaves. On the slaves it calls [Generator\(\)](#) for all the terms that this process gets, but [PF_GetTerm\(\)](#) gets terms from the master (not directly from infile).

Parameters

<i>e</i>	The pointer to the current expression.
<i>i</i>	The index for the current expression.
<i>LastExpression</i>	The flag indicating whether it is the last expression.

Returns

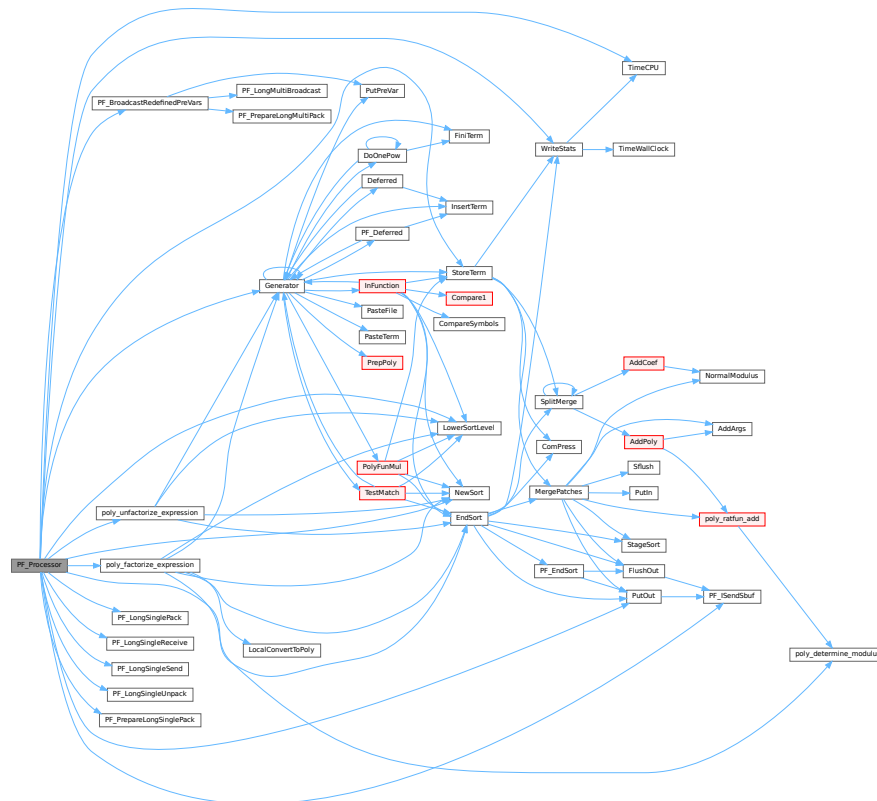
0 if OK, nonzero on error.

Definition at line 1540 of file [parallel.c](#).

References [EndSort\(\)](#), [Generator\(\)](#), [LowerSortLevel\(\)](#), [NewSort\(\)](#), [PACK_LONG](#), [PF_BroadcastRedefinedPreVars\(\)](#), [PF_ISendSbuf\(\)](#), [PF_LongSinglePack\(\)](#), [PF_LongSingleReceive\(\)](#), [PF_LongSingleSend\(\)](#), [PF_LongSingleUnpack\(\)](#), [PF_PrepareLongSinglePack\(\)](#), [poly_factorize_expression\(\)](#), [poly_unfactorize_expression\(\)](#), [PutOut\(\)](#), [StoreTerm\(\)](#), [SWAP](#), [TimeCPU\(\)](#), and [WriteStats\(\)](#).

Referenced by [Processor\(\)](#).

Here is the call graph for this function:



4.94.4.14 PF_Init()

```
int PF_Init (
    int * argc,
    char *** argv )
```

All the library independent stuff. [PF_LibInit\(\)](#) should do all library dependent initializations.

Parameters

<i>argc</i>	pointer to the number of arguments.
<i>argv</i>	pointer to the arguments.

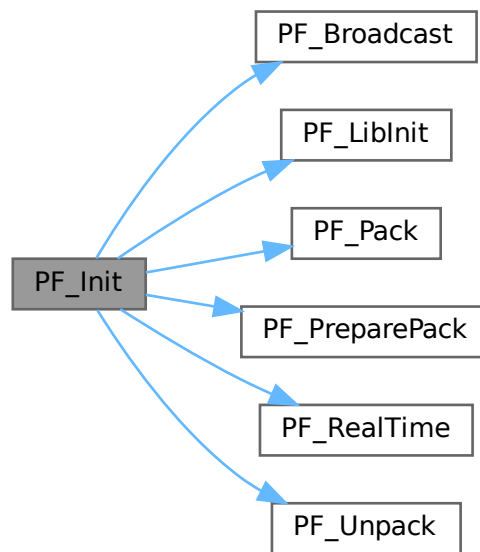
Returns

0 if OK, nonzero on error.

Definition at line 1963 of file [parallel.c](#).

References [PF_Broadcast\(\)](#), [PF_LibInit\(\)](#), [PF_Pack\(\)](#), [PF_PreparePack\(\)](#), [PF_RealTime\(\)](#), and [PF_Unpack\(\)](#).

Here is the call graph for this function:

**4.94.4.15 PF_Terminate()**

```
int PF_Terminate (
    int errorcode )
```

Performs the finalization of ParFORM. To be called by `Terminate()`.

Parameters

<i>error</i>	an error code.
--------------	----------------

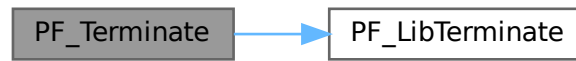
Returns

0 if OK, nonzero on error.

Definition at line 2058 of file [parallel.c](#).

References [PF_LibTerminate\(\)](#).

Here is the call graph for this function:



4.94.4.16 PF_GetSlaveTimes()

```

LONG PF_GetSlaveTimes (
    void )
  
```

Returns the total CPU time of all slaves together. This function must be called on the master and all slaves.

Returns

on the master, the sum of CPU times on all slaves.

Definition at line 2074 of file [parallel.c](#).

References [TimeCPU\(\)](#).

Here is the call graph for this function:



4.94.4.17 PF_BroadcastNumber()

```

LONG PF_BroadcastNumber (
    LONG x )
  
```

Broadcasts a LONG value from the master to the all slaves.

Parameters

x	the number to be broadcast (set on the master).
---	---

Returns

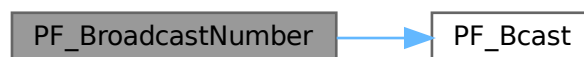
the synchronised result.

Definition at line 2094 of file [parallel.c](#).

References [PF_Bcast\(\)](#).

Referenced by [DoCheckpoint\(\)](#), [PF_BroadcastBuffer\(\)](#), and [StartVariables\(\)](#).

Here is the call graph for this function:

**4.94.4.18 PF_BroadcastBuffer()**

```
void PF_BroadcastBuffer (
    WORD ** buffer,
    LONG * length )
```

Broadcasts a buffer from the master to all the slaves.

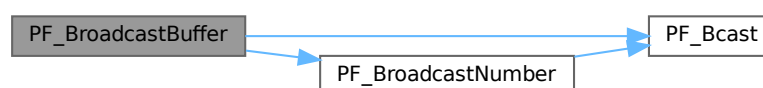
Parameters

<i>in, out</i>	<i>buffer</i>	on the master, the buffer to be broadcast. On the slaves, the buffer will be allocated if the length is greater than 0. The caller must free it.
<i>in, out</i>	<i>length</i>	on the master, the length of the buffer to be broadcast. On the slaves, it receives the length of transferred buffer. The actual transfer occurs only if the length is greater than 0.

Definition at line 2121 of file [parallel.c](#).

References [PF_Bcast\(\)](#), and [PF_BroadcastNumber\(\)](#).

Here is the call graph for this function:



4.94.4.19 PF_BroadcastString()

```
int PF_BroadcastString (
    UBYTE * str )
```

Broadcasts a string from the master to all slaves.

Parameters

<i>in, out</i>	<i>str</i>	The pointer to a null-terminated string.
----------------	------------	--

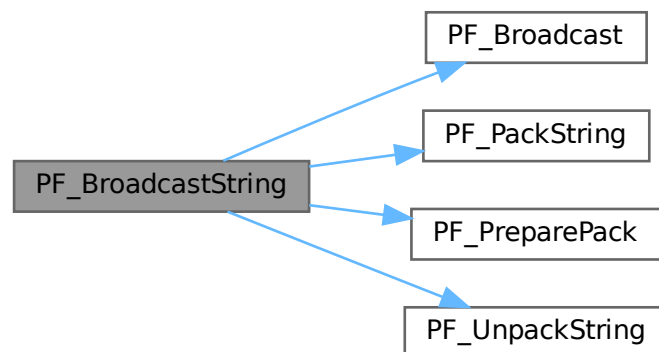
Returns

0 if OK, nonzero on error.

Definition at line 2163 of file [parallel.c](#).

References [PF_Broadcast\(\)](#), [PF_PackString\(\)](#), [PF_PreparePack\(\)](#), and [PF_UnpackString\(\)](#).

Here is the call graph for this function:



4.94.4.20 PF_BroadcastPreDollar()

```
int PF_BroadcastPreDollar (
    WORD ** dbuffer,
    LONG * newsize,
    int * numterms )
```

Broadcasts dollar variables set as a preprocessor variables. Only the master is able to make an assignment like `#$a=g;` where `g` is an expression: only the master has an access to the expression. So, the master broadcasts the result to slaves.

The result is in `*dbuffer` of the size is `*newsize` (in number of WORDs), +1 for trailing zero. For slave `newsize` and `numterms` are output parameters.

Parameters

<i>in, out</i>	<i>dbuffer</i>	the buffer for a dollar variable.
<i>in, out</i>	<i>newsize</i>	the size of the dollar variable in WORDs.
<i>in, out</i>	<i>numterms</i>	the number of terms in the dollar variable.

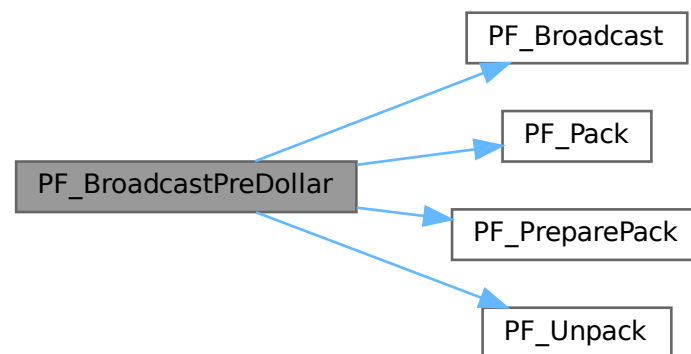
Returns

0 if OK, nonzero on error.

Definition at line 2218 of file [parallel.c](#).

References [PF_Broadcast\(\)](#), [PF_Pack\(\)](#), [PF_PreparePack\(\)](#), and [PF_Unpack\(\)](#).

Here is the call graph for this function:



4.94.4.21 PF_CollectModifiedDollars()

```
int PF_CollectModifiedDollars (
    void )
```

Combines modified dollar variables on the all slaves, and store them into those on the master.

The potentially modified dollar variables are given in `PotModdollars`, and the number of them is given by `NumPotModdollars`.

The current module could be executed in parallel only if all potentially modified variables are listed in `ModOptdollars`, otherwise the module was switched to the sequential mode.

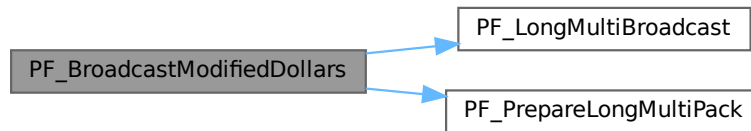
Returns

0 if OK, nonzero on error.

Definition at line 2785 of file [parallel.c](#).

References [PF_LongMultiBroadcast\(\)](#), and [PF_PrepareLongMultiPack\(\)](#).

Here is the call graph for this function:

**4.94.4.23 PF_BroadcastRedefinedPreVars()**

```
int PF_BroadcastRedefinedPreVars (  
    void )
```

Broadcasts preprocessor variables, which were changed by the Redefine statements in the current module, from the master to the all slaves.

The potentially redefined preprocessor variables are given in `AC.pfirstnum`, and the number of them is given by `AC.numpfirstnum`. For an actually redefined variable, the corresponding value in `AC.inputnumbers` is non-negative.

Returns

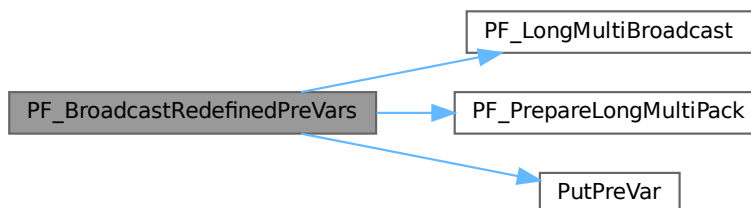
0 if OK, nonzero on error.

Definition at line 3002 of file [parallel.c](#).

References [PF_LongMultiBroadcast\(\)](#), [PF_PrepareLongMultiPack\(\)](#), [PutPreVar\(\)](#), [VectorPtr](#), and [VectorReserve](#).

Referenced by [PF_Processor\(\)](#).

Here is the call graph for this function:



4.94.4.24 PF_StoreInsideInfo()

```
int PF_StoreInsideInfo (
    void )
```

Definition at line 3092 of file [parallel.c](#).

4.94.4.25 PF_RestoreInsideInfo()

```
int PF_RestoreInsideInfo (
    void )
```

Definition at line 3118 of file [parallel.c](#).

4.94.4.26 PF_BroadcastCBuf()

```
int PF_BroadcastCBuf (
    int bufnum )
```

Broadcasts a compiler buffer specified by *bufnum* from the master to the all slaves.

Parameters

<i>bufnum</i>	The index of the compiler buffer to be broadcast.
---------------	---

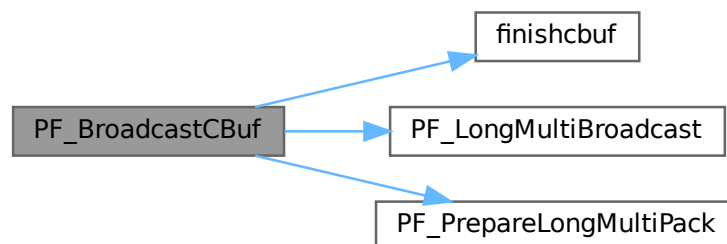
Returns

0 if OK, nonzero on error.

Definition at line 3144 of file [parallel.c](#).

References [CbUf::boomlijst](#), [CbUf::Buffer](#), [CbUf::BufferSize](#), [CbUf::CanCommu](#), [CbUf::dimension](#), [finishcbuf\(\)](#), [CbUf::lhs](#), [CbUf::numdum](#), [CbUf::NumTerms](#), [PF_LongMultiBroadcast\(\)](#), [PF_PrepareLongMultiPack\(\)](#), [CbUf::Pointer](#), [CbUf::rhs](#), and [CbUf::Top](#).

Here is the call graph for this function:



4.94.4.27 PF_BroadcastExpFlags()

```
int PF_BroadcastExpFlags (
    void )
```

Broadcasts AR.expflags and several properties of each expression, e.g., e->vflags, from the master to all slaves.

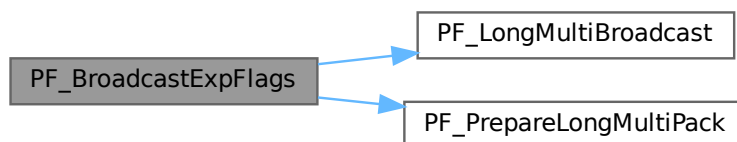
Returns

0 if OK, nonzero on error.

Definition at line 3255 of file [parallel.c](#).

References [PF_LongMultiBroadcast\(\)](#), and [PF_PrepareLongMultiPack\(\)](#).

Here is the call graph for this function:



4.94.4.28 PF_BroadcastExpr()

```
int PF_BroadcastExpr (
    EXPRESSIONS e,
    FILEHANDLE * file )
```

Broadcasts an expression from the master to the all slaves.

Parameters

<i>e</i>	The expression to be broadcast.
<i>file</i>	The file in which the expression is sitting.

Returns

0 if OK, nonzero on error.

Definition at line 3549 of file [parallel.c](#).

Referenced by [PF_BroadcastRHS\(\)](#).

4.94.4.29 PF_BroadcastRHS()

```
int PF_BroadcastRHS (
    void )
```

Broadcasts expressions appearing in the right-hand side from the master to the all slaves.

Returns

0 if OK, nonzero on error.

Definition at line [3577](#) of file [parallel.c](#).

References [PF_BroadcastExpr\(\)](#).

Referenced by [Processor\(\)](#).

Here is the call graph for this function:



4.94.4.30 PF_InParallelProcessor()

```
int PF_InParallelProcessor (
    void )
```

Processes expressions in the InParallel mode, i.e., dividing expressions marked by partodo over the slaves.

Returns

0 if OK, nonzero on error.

Definition at line [3624](#) of file [parallel.c](#).

Referenced by [Processor\(\)](#).

4.94.4.31 PF_SendFile()

```
int PF_SendFile (
    int to,
    FILE * fd )
```

Sends a file to the process specified by *to*.

Parameters

<i>to</i>	the destination process number.
<i>fd</i>	the file to be sent.

Returns

the size of sent data in bytes, or -1 on error.

Definition at line 4221 of file [parallel.c](#).

References [PF_RawSend\(\)](#).

Referenced by [DoCheckpoint\(\)](#).

Here is the call graph for this function:



4.94.4.32 PF_RecvFile()

```
int PF_RecvFile (
    int from,
    FILE * fd )
```

Receives a file from the process specified by *from*.

Parameters

<i>from</i>	the source process number.
<i>fd</i>	the file to save the received data.

Returns

the size of received data in bytes, or -1 on error.

Definition at line 4259 of file [parallel.c](#).

References [PF_RawRecv\(\)](#).

Referenced by [DoCheckpoint\(\)](#).

Here is the call graph for this function:



4.94.4.33 PF_MLock()

```
void PF_MLock (
    void )
```

A function called by MLOCK(ErrorMessageLock) for slaves.

Definition at line 4340 of file [parallel.c](#).

References [VectorClear](#).

4.94.4.34 PF_MUnlock()

```
void PF_MUnlock (
    void )
```

A function called by MUNLOCK(ErrorMessageLock) for slaves.

Definition at line 4356 of file [parallel.c](#).

References [PF_RawSend\(\)](#), [VectorEmpty](#), [VectorPtr](#), and [VectorSize](#).

Here is the call graph for this function:



4.94.4.35 PF_WriteFileToFile()

```
LONG PF_WriteFileToFile (
    int handle,
    UBYTE * buffer,
    LONG size )
```

Replaces WriteFileToFile() on the master and slaves.

It copies the given buffer into internal buffers if called between MLOCK(ErrorMessageLock) and MUNLOCK(ErrorMessageLock) for slaves and handle is StdOut or LogHandle, otherwise calls WriteFileToFile().

Parameters

<i>handle</i>	a file handle that specifies the output.
<i>buffer</i>	a pointer to the source buffer containing the data to be written.
<i>size</i>	the size of data to be written in bytes.

Returns

the actual size of data written to the output in bytes.

Definition at line 4385 of file [parallel.c](#).

References [VectorClear](#), [VectorPtr](#), [VectorPushBacks](#), and [VectorSize](#).

4.94.4.36 PF_FlushStdOutBuffer()

```
void PF_FlushStdOutBuffer (
    void )
```

Explicitly Flushes the buffer for the standard output on the master, which is used if PF_ENABLE_STDOUT_↵
BUFFERING is defined.

Definition at line 4479 of file [parallel.c](#).

References [VectorClear](#), [VectorPtr](#), and [VectorSize](#).

4.94.4.37 PF_FreeErrorMessageBuffers()

```
void PF_FreeErrorMessageBuffers (
    void )
```

Frees the buffers allocated for the synchronized output.

Currently, not used anywhere, but could be used in [PF_Terminate\(\)](#).

Definition at line 4615 of file [parallel.c](#).

References [VectorFree](#).

4.94.5 Variable Documentation

4.94.5.1 PF

[PARALLELVARS](#) PF

Definition at line 90 of file [parallel.c](#).

4.95 parallel.c

[Go to the documentation of this file.](#)

```

00001
00011 /* #[] License : */
00012 /*
00013 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00014 *   When using this file you are requested to refer to the publication
00015 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00016 *   This is considered a matter of courtesy as the development was paid
00017 *   for by FOM the Dutch physics granting agency and we would like to
00018 *   be able to track its scientific use to convince FOM of its value
00019 *   for the community.
00020 *
00021 *   This file is part of FORM.
00022 *
00023 *   FORM is free software: you can redistribute it and/or modify it under the
00024 *   terms of the GNU General Public License as published by the Free Software
00025 *   Foundation, either version 3 of the License, or (at your option) any later
00026 *   version.
00027 *
00028 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00029 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00030 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00031 *   details.
00032 *
00033 *   You should have received a copy of the GNU General Public License along
00034 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00035 */
00036 /* #] License : */
00037 /*
00038     #[] includes :
00039 */
00040 #include "form3.h"
00041 #include "vector.h"
00042
00043 /*
00044 #define PF_DEBUG_BCAST_LONG
00045 #define PF_DEBUG_BCAST_BUF
00046 #define PF_DEBUG_BCAST_PREDOLLAR
00047 #define PF_DEBUG_BCAST_RHSEXP
00048 #define PF_DEBUG_BCAST_DOLLAR
00049 #define PF_DEBUG_BCAST_PREVAR
00050 #define PF_DEBUG_BCAST_CBUF
00051 #define PF_DEBUG_BCAST_EXPRFLAGS
00052 #define PF_DEBUG_REDUCE_DOLLAR
00053 */
00054
00055 /* mpi.c */
00056 LONG PF_RealTime(int);
00057 int PF_LibInit(int*, char**);
00058 int PF_LibTerminate(int);
00059 int PF_Probe(int*);
00060 int PF_RecvWbuf(WORD*, LONG*, int*);
00061 int PF_IRecvRbuf(PF_BUFFER*, int, int);
00062 int PF_WaitRbuf(PF_BUFFER *, int, LONG *);
00063 int PF_RawSend(int dest, void *buf, LONG l, int tag);
00064 LONG PF_RawRecv(int *src, void *buf, LONG thesize, int *tag);
00065 int PF_RawProbe(int *src, int *tag, int *bytesize);
00066
00067 /* Private functions */
00068
00069 static int PF_WaitAllSlaves(void);
00070
00071 static void PF_PackRedefinedPreVars(void);
00072 static void PF_UnpackRedefinedPreVars(void);
00073
00074 static int PF_Wait4MasterIP(int tag);
00075 static int PF_DoOneExpr(void);
00076 static int PF_ReadMaster(void); /* reads directly to its scratch! */
00077 static int PF_Slave2MasterIP(int src); /* both master and slave */
00078 static int PF_Master2SlaveIP(int dest, EXPRESSIONS e);
00079 static int PF_WalkThrough(WORD *t, LONG l, LONG chunk, LONG *count);
00080 static int PF_SendChunkIP(FILEHANDLE *curfile, POSITION *position, int to, LONG thesize);
00081 static int PF_RecvChunkIP(FILEHANDLE *curfile, int from, LONG thesize);
00082
00083 static void PF_ReceiveErrorMessage(int src, int tag);
00084 static void PF_CatchErrorMessages(int *src, int *tag);
00085 static void PF_CatchErrorMessagesForAll(void);
00086 static int PF_ProbeWithCatchingErrorMessages(int *src);
00087
00088 /* Variables */
00089
00090 PARALLELVARS PF;
00091 #ifdef MPI2

```

```

00092 WORD *PF_shared_buff;
00093 #endif
00094
00095 static LONG PF_goutterms; /* (master) Total out terms at PF_EndSort(), used in PF_Statistics().
    */
00096 static POSITION PF_exprsize; /* (master) The size of the expression at PF_EndSort(), used in
    PF_Processor(). */
00097
00098 /*
00099 This will work well only under Linux, see
00100 #ifdef PF_WITH_SCHED_YIELD
00101 below in PF_WaitAllSlaves().
00102 */
00103 #ifndef PF_WITH_SCHED_YIELD
00104 #include <sched.h>
00105 #endif
00106
00107 #ifdef PF_WITHLOG
00108 #define PRINTFBUF(TEXT,TERM,SIZE) { UBYTE lbuf[24]; if(PF.log){ WORD iii;\
00109 NumToStr(lbuf,AC.CModule); \
00110 fprintf(stderr,"%d|s] %s : ",PF.me,lbuf,(char*)TEXT);\
00111 if(TERM){ fprintf(stderr,"%d] ",(int)(*TERM));\
00112 if((SIZE)<500 && (SIZE)>0) for(iii=1;iii<(SIZE);iii++)\
00113 fprintf(stderr,"%d ",TERM[iii]); }\
00114 fprintf(stderr,"\n");\
00115 fflush(stderr); } }
00116 #else
00117 #define PRINTFBUF(TEXT,TERM,SIZE) {}
00118 #endif
00119
00124 #define SWAP(x, y) \
00125 do { \
00126 char swap_tmp__[sizeof(x) == sizeof(y) ? (int)sizeof(x) : -1]; \
00127 memcpy(swap_tmp__, &y, sizeof(x)); \
00128 memcpy(&y, &x, sizeof(x)); \
00129 memcpy(&x, swap_tmp__, sizeof(x)); \
00130 } while (0)
00131
00135 #define PACK_LONG(p, n) \
00136 do { \
00137 * (p)++ = (UWORD)((ULONG)(n) & (ULONG)WORDMASK); \
00138 * (p)++ = (UWORD)((ULONG)(n) » BITSINWORD) & (ULONG)WORDMASK); \
00139 } while (0)
00140
00144 #define UNPACK_LONG(p, n) \
00145 do { \
00146 (n) = (LONG)((((ULONG)(p)[1] & (ULONG)WORDMASK) « BITSINWORD) | ((ULONG)(p)[0] &
(ULONG)WORDMASK)); \
00147 (p) += 2; \
00148 } while (0)
00149
00153 #define CHECK(condition) _CHECK(condition, __FILE__, __LINE__)
00154 #define _CHECK(condition, file, line) __CHECK(condition, file, line)
00155 #define __CHECK(condition, file, line) \
00156 do { \
00157 if ( !(condition) ) { \
00158 Error0("Fatal error at " file ":" #line); \
00159 Terminate(-1); \
00160 } \
00161 } while (0)
00162
00163 /*
00164 * For debugging.
00165 */
00166 #define DBGOUT(lv1, lv2, a) do { if ( lv1 >= lv2 ) { printf a; fflush(stdout); } } while (0)
00167
00168 /* (AN.ninterms of master) == max(AN.ninterms of slaves) == sum(PF_linterms of slaves) at EndSort().
    */
00169 #define DBGOUT_NINTERMS(lv, a)
00170 /* #define DBGOUT_NINTERMS(lv, a) DBGOUT(1, lv, a) */
00171
00172 /*
00173 #] includes :
00174 #[ statistics :
00175 #] variables : (should be part of a struct?)
00176 */
00177 static LONG PF_linterms; /* local interms on this proces: PF_Proces */
00178 #define PF_STATS_SIZE 5
00179 static LONG **PF_stats = NULL; /* space for collecting statistics of all procs */
00180 static LONG PF_laststat; /* last realtime when statistics were printed */
00181 static LONG PF_statsinterval; /* timeinterval for printing statistics */
00182 /*
00183 #] variables :
00184 #[ PF_Statistics :
00185 */
00186
00195 static int PF_Statistics(LONG **stats, int proc)

```

```

00196 {
00197     GETIDENTITY
00198     LONG real, cpu;
00199     WORD rpart, cpart;
00200     int i, j;
00201
00202     if ( AT.SS == AM.S0 && PF.me == MASTER ) {
00203         real = PF_RealTime(PF_TIME); rpart = (WORD)(real%100); real /= 100;
00204
00205         if ( PF_stats == NULL ) {
00206             PF_stats = (LONG**)Malloc1(PF.numtasks*sizeof(LONG*), "PF_stats 1");
00207             for ( i = 0; i < PF.numtasks; i++ ) {
00208                 PF_stats[i] = (LONG*)Malloc1(PF_STATS_SIZE*sizeof(LONG), "PF_stats 2");
00209                 for ( j = 0; j < PF_STATS_SIZE; j++ ) PF_stats[i][j] = 0;
00210             }
00211         }
00212         if ( proc > 0 ) for ( i = 0; i < PF_STATS_SIZE; i++ ) PF_stats[proc][i] = stats[0][i];
00213
00214         if ( real >= PF_laststat + PF_statsinterval || proc == 0 ) {
00215             LONG sum[PF_STATS_SIZE];
00216
00217             for ( i = 0; i < PF_STATS_SIZE; i++ ) sum[i] = 0;
00218             sum[0] = cpu = TimeCPU(1);
00219             cpart = (WORD)(cpu%1000);
00220             cpu /= 1000;
00221             cpart /= 10;
00222             if ( AC.OldParallelStats ) MesPrint("");
00223             if ( proc > 0 && AC.StatsFlag && AC.OldParallelStats ) {
00224                 MesPrint("proc      CPU      in      gen      left      byte");
00225                 MesPrint("%3d : %7l.%2i %10l", 0, cpu, cpart, AN.ninterms);
00226             }
00227             else if ( AC.StatsFlag && AC.OldParallelStats ) {
00228                 MesPrint("proc      CPU      in      gen      out      byte");
00229                 MesPrint("%3d : %7l.%2i %10l %10l %10l", 0, cpu, cpart, AN.ninterms, 0, PF_goutterms);
00230             }
00231
00232             for ( i = 1; i < PF.numtasks; i++ ) {
00233                 cpart = (WORD)(PF_stats[i][0]%1000);
00234                 cpu = PF_stats[i][0] / 1000;
00235                 cpart /= 10;
00236                 if ( AC.StatsFlag && AC.OldParallelStats )
00237                     MesPrint("%3d : %7l.%2i %10l %10l %10l", i, cpu, cpart,
00238                             PF_stats[i][2], PF_stats[i][3], PF_stats[i][4]);
00239                 for ( j = 0; j < PF_STATS_SIZE; j++ ) sum[j] += PF_stats[i][j];
00240             }
00241             cpart = (WORD)(sum[0]%1000);
00242             cpu = sum[0] / 1000;
00243             cpart /= 10;
00244             if ( AC.StatsFlag && AC.OldParallelStats ) {
00245                 MesPrint("Sum = %7l.%2i %10l %10l %10l", cpu, cpart, sum[2], sum[3], sum[4]);
00246                 MesPrint("Real = %7l.%2i %20s (%1) %16s",
00247                         real, rpart, AC.Commercial, AC.CModule, EXPRNAME(AR.CurExpr));
00248                 MesPrint("");
00249             }
00250             PF_laststat = real;
00251         }
00252     }
00253     return(0);
00254 }
00255 /*
00256     #] PF_Statistics :
00257     #] statistics :
00258     #[ sort.c :
00259     #[ sort variables :
00260 */
00261
00262 typedef struct NoDe {
00263     struct NoDe *left;
00264     struct NoDe *rght;
00265     int lloser;
00266     int rloser;
00267     int lsrc;
00268     int rsrc;
00269 } NODE;
00270
00271 /*
00272     should/could be put in one struct
00273 */
00274 static NODE *PF_root; /* root of tree of losers */
00275 static WORD PF_loser; /* this is the last loser */
00276 static WORD **PF_term; /* these point to the active terms */
00277 static WORD **PF_newcpo; /* new coeffs of merged terms */
00278 static WORD *PF_newclen; /* length of new coefficients */
00279
00280 /*
00281     preliminary: could also write somewhere else?
00282 */

```

```

00286
00287 static WORD *PF_WorkSpace; /* used in PF_EndSort() */
00288 static UWORD *PF_ScratchSpace; /* used in PF_GetLoser() */
00289
00290 /*
00291     #] sort variables :
00292     #[ PF_AllocBuf :
00293 */
00294
00311 static PF_BUFFER *PF_AllocBuf(int nbufs, LONG bsize, WORD free)
00312 {
00313     PF_BUFFER *buf;
00314     UBYTE *p, *stop;
00315     LONG allocsize;
00316     int i;
00317
00318     allocsize =
00319         (LONG)(sizeof(PF_BUFFER) + 4*nbufs*sizeof(WORD*) + (nbufs-free)*bsize);
00320
00321     allocsize +=
00322         (LONG)( nbufs * ( 2 * sizeof(MPI_Status)
00323                         + sizeof(MPI_Request)
00324                         + sizeof(MPI_Datatype)
00325                     ) );
00326     allocsize += (LONG)( nbufs * 3 * sizeof(int) );
00327
00328     if ( ( buf = (PF_BUFFER*)Malloc1(allocsize,"PF_AllocBuf") ) == NULL ) return(NULL);
00329
00330     p = ((UBYTE *)buf) + sizeof(PF_BUFFER);
00331     stop = ((UBYTE *)buf) + allocsize;
00332
00333     buf->nbufs = nbufs;
00334     buf->active = 0;
00335
00336     buf->buff = (WORD**)p; p += buf->nbufs*sizeof(WORD*);
00337     buf->fill = (WORD**)p; p += buf->nbufs*sizeof(WORD*);
00338     buf->full = (WORD**)p; p += buf->nbufs*sizeof(WORD*);
00339     buf->stop = (WORD**)p; p += buf->nbufs*sizeof(WORD*);
00340     buf->status = (MPI_Status *)p; p += buf->nbufs*sizeof(MPI_Status);
00341     buf->retstat = (MPI_Status *)p; p += buf->nbufs*sizeof(MPI_Status);
00342     buf->request = (MPI_Request *)p; p += buf->nbufs*sizeof(MPI_Request);
00343     buf->type = (MPI_Datatype *)p; p += buf->nbufs*sizeof(MPI_Datatype);
00344     buf->index = (int *)p; p += buf->nbufs*sizeof(int);
00345
00346     for ( i = 0; i < buf->nbufs; i++ ) buf->request[i] = MPI_REQUEST_NULL;
00347     buf->tag = (int *)p; p += buf->nbufs*sizeof(int);
00348     buf->from = (int *)p; p += buf->nbufs*sizeof(int);
00349 /*
00350     and finally the real bufferspace
00351 */
00352     for ( i = free; i < buf->nbufs; i++ ) {
00353         buf->buff[i] = (WORD*)p; p += bsize;
00354         buf->stop[i] = (WORD*)p;
00355         buf->fill[i] = buf->full[i] = buf->buff[i];
00356     }
00357     if ( p != stop ) {
00358         MesPrint("Error in PF_AllocBuf p = %x stop = %x\n",p,stop);
00359         return(NULL);
00360     }
00361     return(buf);
00362 }
00363
00364 /*
00365     #] PF_AllocBuf :
00366     #[ PF_InitTree :
00367 */
00368
00380 static int PF_InitTree(void)
00381 {
00382     GETIDENTITY
00383     PF_BUFFER **rbuf = PF.rbufs;
00384     UBYTE *p, *stop;
00385     int numrbufs,numtasks = PF.numtasks;
00386     int i, j, src, numnodes;
00387     int numslaves = numtasks - 1;
00388     LONG size;
00389 /*
00390     #] the buffers : for the new coefficients and the terms
00391     we need one for each slave
00392 */
00393     if ( PF_term == NULL ) {
00394         size = 2*numtasks*sizeof(WORD*) + sizeof(WORD)*
00395             ( numtasks*(1 + AM.MaxTal) + (AM.MaxTer/sizeof(WORD)+1) + 2*(AM.MaxTal+2) );
00396
00397         PF_term = (WORD **)Malloc1(size,"PF_term");
00398         stop = ((UBYTE*)PF_term) + size;
00399         p = ((UBYTE*)PF_term) + numtasks*sizeof(WORD*);

```

```

00400
00401     PF_newcpos = (WORD **)p; p += sizeof(WORD*) * numtasks;
00402     PF_newclen = (WORD *)p; p += sizeof(WORD) * numtasks;
00403     for ( i = 0; i < numtasks; i++ ) {
00404         PF_newcpos[i] = (WORD *)p; p += sizeof(WORD)*AM.MaxTal;
00405         PF_newclen[i] = 0;
00406     }
00407     PF_WorkSpace = (WORD *)p; p += AM.MaxTer+sizeof(WORD);
00408     PF_ScratchSpace = (UWORD*)p; p += 2*(AM.MaxTal+2)*sizeof(UWORD);
00409
00410     if ( p != stop ) { MesPrint("error in PF_InitTree"); return(-1); }
00411 }
00412 /*
00413     #] the buffers :
00414     #[ the receive buffers :
00415 */
00416     numrbufs = PF.numrbufs;
00417 /*
00418     this is the size we have in the combined sortbufs for one slave
00419 */
00420     size = (AT.SS->sTop2 - AT.SS->lBuffer - 1)/(PF.numtasks - 1);
00421
00422     if ( rbuf == NULL ) {
00423         if ( ( rbuf = (PF_BUFFER**)Malloca1(numtasks*sizeof(PF_BUFFER*), "Master: rbufs") ) == NULL )
00424             return(-1);
00425         if ( (rbuf[0] = PF_AllocBuf(1,0,1) ) == NULL ) return(-1);
00426         for ( i = 1; i < numtasks; i++ ) {
00427             if (!(rbuf[i] = PF_AllocBuf(numrbufs,sizeof(WORD)*size,1))) return(-1);
00428         }
00429         rbuf[0]->buff[0] = AT.SS->lBuffer;
00430         rbuf[0]->full[0] = rbuf[0]->fill[0] = rbuf[0]->buff[0];
00431         rbuf[0]->stop[0] = rbuf[1]->buff[0] = rbuf[0]->buff[0] + 1;
00432         rbuf[1]->full[0] = rbuf[1]->fill[0] = rbuf[1]->buff[0];
00433         for ( i = 2; i < numtasks; i++ ) {
00434             rbuf[i-1]->stop[0] = rbuf[i]->buff[0] = rbuf[i-1]->buff[0] + size;
00435             rbuf[i]->full[0] = rbuf[i]->fill[0] = rbuf[i]->buff[0];
00436         }
00437         rbuf[numtasks-1]->stop[0] = rbuf[numtasks-1]->buff[0] + size;
00438
00439         for ( i = 1; i < numtasks; i++ ) {
00440             for ( j = 0; j < rbuf[i]->numbufs; j++ ) {
00441                 rbuf[i]->full[j] = rbuf[i]->fill[j] = rbuf[i]->buff[j] + AM.MaxTer/sizeof(WORD) + 2;
00442             }
00443             PF_term[i] = rbuf[i]->fill[rbuf[i]->active];
00444             *PF_term[i] = 0;
00445             PF_IRecvRbuf(rbuf[i],rbuf[i]->active,i);
00446         }
00447         rbuf[0]->active = 0;
00448         PF_term[0] = rbuf[0]->buff[0];
00449         PF_term[0][0] = 0; /* PF_term[0] is used for a zero term. */
00450         PF.rbufs = rbuf;
00451 /*
00452         #] the receive buffers :
00453         #[ the actual tree :
00454
00455         calculate number of nodes in mergetree and allocate space for them
00456 */
00457         if ( numslaves < 3 ) numnodes = 1;
00458         else {
00459             numnodes = 2;
00460             while ( numnodes < numslaves ) numnodes *= 2;
00461             numnodes -= 1;
00462         }
00463
00464         if ( PF_root == NULL )
00465             if ( ( PF_root = (NODE*)Malloca1(sizeof(NODE)*numnodes,"nodes in mergetree") ) == NULL )
00466                 return(-1);
00467 /*
00468         then initialize all the nodes
00469 */
00470         src = 1;
00471         for ( i = 0; i < numnodes; i++ ) {
00472             if ( 2*(i+1) <= numnodes ) {
00473                 PF_root[i].left = &(PF_root[2*(i+1)-1]);
00474                 PF_root[i].lsrc = 0;
00475             }
00476             else {
00477                 PF_root[i].left = 0;
00478                 if ( src < numtasks ) PF_root[i].lsrc = src++;
00479                 else PF_root[i].lsrc = 0;
00480             }
00481             PF_root[i].lloser = 0;
00482         }
00483         for ( i = 0; i < numnodes; i++ ) {
00484             if ( 2*(i+1)+1 <= numnodes ) {
00485                 PF_root[i].rght = &(PF_root[2*(i+1)]);

```



```

00486         PF_root[i].rsrc = 0;
00487     }
00488     else {
00489         PF_root[i].rghet = 0;
00490         if (src<numtasks) PF_root[i].rsrc = src++;
00491         else PF_root[i].rsrc = 0;
00492     }
00493     PF_root[i].rloser = 0;
00494 }
00495 /*
00496     #] the actual tree :
00497 */
00498 return(numnodes);
00499 }
00500
00501 /*
00502     #] PF_InitTree :
00503     #[ PF_PutIn :
00504 */
00505
00524 static WORD *PF_PutIn(int src)
00525 {
00526     int tag;
00527     WORD im, r;
00528     WORD *m1, *m2;
00529     LONG size;
00530     PF_BUFFER *rbuf = PF.rbufs[src];
00531     int a = rbuf->active;
00532     int next = a+1 >= rbuf->numbufs ? 0 : a+1 ;
00533     WORD *lastterm = PF_term[src];
00534     WORD *term = rbuf->fill[a];
00535
00536     if ( src <= 0 ) return(PF_term[0]);
00537
00538     if ( rbuf->full[a] == rbuf->buff[a] + AM.MaxTer/sizeof(WORD) + 2 ) {
00539 /*
00540         very first term from this src
00541 */
00542         tag = PF_WaitRbuf(rbuf,a,&size);
00543         rbuf->full[a] += size;
00544         if ( tag == PF_ENDBUFFER_MSGTAG ) *rbuf->full[a]++ = 0;
00545         else if ( rbuf->numbufs > 1 ) {
00546 /*
00547             post a nonblock. recv. for the next buffer
00548 */
00549             rbuf->full[next] = rbuf->buff[next] + AM.MaxTer/sizeof(WORD) + 2;
00550             size = (LONG)(rbuf->stop[next] - rbuf->full[next]);
00551             PF_IRecvRbuf(rbuf,next,src);
00552         }
00553     }
00554     if ( *term == 0 && term != rbuf->full[a] ) return(PF_term[0]);
00555 /*
00556     exception is for rare cases when the terms fitted exactly into buffer
00557 */
00558     if ( term + *term > rbuf->full[a] || term + 1 >= rbuf->full[a] ) {
00559 newterms:
00560         m1 = rbuf->buff[next] + AM.MaxTer/sizeof(WORD) + 1;
00561         if ( *term < 0 || term == rbuf->full[a] ) {
00562 /*
00563             copy term and lastterm to the new buffer, so that they end at m1
00564 */
00565             m2 = rbuf->full[a] - 1;
00566             while ( m2 >= term ) *m1-- = *m2--;
00567             rbuf->fill[next] = term = m1 + 1;
00568             m2 = lastterm + *lastterm - 1;
00569             while ( m2 >= lastterm ) *m1-- = *m2--;
00570             lastterm = m1 + 1;
00571         }
00572         else {
00573 /*
00574             copy beginning of term to the next buffer so that it ends at m1
00575 */
00576             m2 = rbuf->full[a] - 1;
00577             while ( m2 >= term ) *m1-- = *m2--;
00578             rbuf->fill[next] = term = m1 + 1;
00579         }
00580         if ( rbuf->numbufs == 1 ) {
00581             rbuf->full[a] = rbuf->buff[a] + AM.MaxTer/sizeof(WORD) + 2;
00582             size = (LONG)(rbuf->stop[a] - rbuf->full[a]);
00583             PF_IRecvRbuf(rbuf,a,src);
00584         }
00585 /*
00586         wait for new terms in the next buffer
00587 */
00588         rbuf->full[next] = rbuf->buff[next] + AM.MaxTer/sizeof(WORD) + 2;
00589         tag = PF_WaitRbuf(rbuf,next,&size);
00590         rbuf->full[next] += size;

```

```

00591         if ( tag == PF_ENDBUFFER_MSGTAG ) {
00592             *rbuf->full[next]++ = 0;
00593         }
00594         else if ( rbuf->numbufs > 1 ) {
00595             /*
00596                 post a nonblock. recv. for active buffer, it is not needed anymore
00597             */
00598             rbuf->full[a] = rbuf->buff[a] + AM.MaxTer/sizeof(WORD) + 2;
00599             size = (LONG)(rbuf->stop[a] - rbuf->full[a]);
00600             PF_IRecvRbuf(rbuf,a,src);
00601         }
00602         /*
00603             now safely make next buffer active
00604         */
00605         a = rbuf->active = next;
00606     }
00607     if ( *term < 0 ) {
00608         /*
00609             We need to decompress the term
00610         */
00611         im = *term;
00612         r = term[1] - im + 1;
00613         m1 = term + 2;
00614         m2 = lastterm - im + 1;
00615         while ( ++im <= 0 ) *--m1 = *--m2;
00616         *--m1 = r;
00617         rbuf->fill[a] = term = m1;
00618         if ( term + *term > rbuf->full[a] ) goto newterms;
00619     }
00620     rbuf->fill[a] += *term;
00621     return(term);
00622 }
00623
00624
00625 /*
00626     #] PF_PutIn :
00627     #[ PF_GetLoser :
00628 */
00629
00648 static int PF_GetLoser(NODE *n)
00649 {
00650     GETIDENTITY
00651     WORD comp;
00652
00653     if ( PF_loser == 0 ) {
00654         /*
00655             this is for the right initialization of the tree only
00656         */
00657         if ( n->left ) n->lloser = PF_GetLoser(n->left);
00658         else {
00659             n->lloser = n->lsrc;
00660             if ( *(PF_term[n->lsrc] = PF_PutIn(n->lsrc)) == 0 ) n->lloser = 0;
00661         }
00662         PF_loser = 0;
00663         if ( n->right ) n->rloser = PF_GetLoser(n->right);
00664         else {
00665             n->rloser = n->rsrc;
00666             if ( *(PF_term[n->rsrc] = PF_PutIn(n->rsrc)) == 0 ) n->rloser = 0;
00667         }
00668         PF_loser = 0;
00669     }
00670     else if ( PF_loser == n->lloser ) {
00671         if ( n->left ) n->lloser = PF_GetLoser(n->left);
00672         else {
00673             n->lloser = n->lsrc;
00674             if ( *(PF_term[n->lsrc] = PF_PutIn(n->lsrc)) == 0 ) n->lloser = 0;
00675         }
00676     }
00677     else if ( PF_loser == n->rloser ) {
00678 newright:
00679         if ( n->right ) n->rloser = PF_GetLoser(n->right);
00680         else {
00681             n->rloser = n->rsrc;
00682             if ( *(PF_term[n->rsrc] = PF_PutIn(n->rsrc)) == 0 ) n->rloser = 0;
00683         }
00684     }
00685     if ( n->lloser > 0 && n->rloser > 0 ) {
00686         comp = CompareTerms(BHEAD PF_term[n->lloser],PF_term[n->rloser],(WORD)0);
00687         if ( comp > 0 ) return(n->lloser);
00688         else if (comp < 0 ) return(n->rloser);
00689         else {
00690             /*
00691                 #[ terms are equal :
00692             */
00693             WORD *lcpos, *rcpos;
00694             UWORD *newcpos;
00695             WORD lclen, rclen, newclen, newnlen;

```

```

00696         SORTING *S = AT.SS;
00697
00698         if ( S->PolyWise ) {
00699             /*
00700             #[ Here we work with PolyFun :
00701             */
00702                 WORD *ttl, *w;
00703                 WORD r1,r2;
00704                 WORD *ml = PF_term[n->lloser];
00705                 WORD *mr = PF_term[n->rloser];
00706
00707                 if ( ( r1 = (int)*PF_term[n->lloser] ) <= 0 ) r1 = 20;
00708                 if ( ( r2 = (int)*PF_term[n->rloser] ) <= 0 ) r2 = 20;
00709                 ttl = ml;
00710                 ml += S->PolyWise;
00711                 mr += S->PolyWise;
00712                 if ( S->PolyFlag == 2 ) {
00713                     w = poly_ratfun_add(BHEAD ml,mr);
00714                     if ( *ttl + w[1] - ml[1] > AM.MaxTer/((LONG)sizeof(WORD)) ) {
00715                         MesPrint("Term too complex in PolyRatFun addition. MaxTermSize of %10l is too
small",AM.MaxTer);
00716                         Terminate(-1);
00717                     }
00718                     AT.WorkPointer = w;
00719                 }
00720                 else {
00721                     w = AT.WorkPointer;
00722                     if ( w + ml[1] + mr[1] > AT.WorkTop ) {
00723                         MesPrint("A Workspace of %10l is too small",AM.WorkSize);
00724                         Terminate(-1);
00725                     }
00726                     AddArgs(BHEAD ml,mr,w);
00727                 }
00728                 r1 = w[1];
00729                 if ( r1 <= FUNHEAD || ( w[FUNHEAD] == -SNUMBER && w[FUNHEAD+1] == 0 ) ) {
00730                     goto cancelled;
00731                 }
00732                 if ( r1 == ml[1] ) {
00733                     NCOPY(ml,w,r1);
00734                 }
00735                 else if ( r1 < ml[1] ) {
00736                     r2 = ml[1] - r1;
00737                     mr = w + r1;
00738                     ml += ml[1];
00739                     while ( --r1 >= 0 ) *--ml = *--mr;
00740                     mr = ml - r2;
00741                     r1 = S->PolyWise;
00742                     while ( --r1 >= 0 ) *--ml = *--mr;
00743                     *ml -= r2;
00744                     PF_term[n->lloser] = ml;
00745                 }
00746                 else {
00747                     r2 = r1 - ml[1];
00748                     if ( r2 > 2*AM.MaxTal )
00749                         MesPrint("warning: new term in polyfun is large");
00750                     mr = ttl - r2;
00751                     r1 = S->PolyWise;
00752                     ml = ttl;
00753                     *ml += r2;
00754                     PF_term[n->lloser] = mr;
00755                     NCOPY(mr,ml,r1);
00756                     r1 = w[1];
00757                     NCOPY(mr,w,r1);
00758                 }
00759                 PF_newclen[n->rloser] = 0;
00760                 PF_loser = n->rloser;
00761                 goto newright;
00762             /*
00763             #[ Here we work with PolyFun :
00764             */
00765                 }
00766                 if ( ( lclen = PF_newclen[n->lloser] ) != 0 ) lcpos = PF_newcpos[n->lloser];
00767                 else {
00768                     lcpos = PF_term[n->lloser];
00769                     lclen = *(lcpos += *lcpos - 1);
00770                     lcpos -= ABS(lclen) - 1;
00771                 }
00772                 if ( ( rclen = PF_newclen[n->rloser] ) != 0 ) rcpos = PF_newcpos[n->rloser];
00773                 else {
00774                     rcpos = PF_term[n->rloser];
00775                     rclen = *(rcpos += *rcpos - 1);
00776                     rcpos -= ABS(rclen) - 1;
00777                 }
00778                 lclen = ( (lclen > 0) ? (lclen-1) : (lclen+1) ) >> 1;
00779                 rclen = ( (rclen > 0) ? (rclen-1) : (rclen+1) ) >> 1;
00780                 newcpos = PF_ScratchSpace;
00781                 if ( AddRat(BHEAD (UWORD *)lcpos,lclen,(UWORD *)rcpos,rclen,newcpos,&newnlen) )

```

```

    return(-1);
00782     if ( AN.ncmod != 0 ) {
00783         if ( ( AC.modmode & POSNEG ) != 0 ) {
00784             NormalModulus(newcpos,&newnlen);
00785         }
00786         if ( BigLong(newcpos,newnlen,(UWORD *)AC.cmod,ABS(AN.ncmod)) >=0 ) {
00787             WORD ii;
00788             SubPLon(newcpos,newnlen,(UWORD *)AC.cmod,ABS(AN.ncmod),newcpos,&newnlen);
00789             newcpos[newnlen] = 1;
00790             for ( ii = 1; ii < newnlen; ii++ ) newcpos[newnlen+ii] = 0;
00791         }
00792     }
00793     if ( newnlen == 0 ) {
00794         /*
00795             terms cancel, get loser of left subtree and then of right subtree
00796         */
00797         cancelled:
00798             PF_loser = n->lloser;
00799             PF_newcpos[n->lloser] = 0;
00800             if ( n->left ) n->lloser = PF_GetLoser(n->left);
00801             else {
00802                 n->lloser = n->lsrc;
00803                 if ( *(PF_term[n->lsrc] = PF_PutIn(n->lsrc)) == 0 ) n->lloser = 0;
00804             }
00805             PF_loser = n->rloser;
00806             PF_newcpos[n->rloser] = 0;
00807             goto newright;
00808         }
00809         else {
00810             /*
00811                 keep the left term and get the loser of right subtree
00812             */
00813             newnlen *= 2;
00814             newcpos = ( newnlen > 0 ) ? ( newnlen + 1 ) : ( newnlen - 1 );
00815             if ( newnlen < 0 ) newnlen = -newnlen;
00816             PF_newcpos[n->lloser] = newcpos;
00817             lcp = PF_newcpos[n->lloser];
00818             if ( newcpos < 0 ) newcpos = -newcpos;
00819             while ( newcpos-- ) *lcp++ = *newcpos++;
00820             PF_loser = n->rloser;
00821             PF_newcpos[n->rloser] = 0;
00822             goto newright;
00823         }
00824         /*
00825             #] terms are equal :
00826         */
00827     }
00828 }
00829 if ( n->lloser > 0 ) return(n->lloser);
00830 if ( n->rloser > 0 ) return(n->rloser);
00831 return(0);
00832 }
00833 /*
00834     #] PF_GetLoser :
00835     #[ PF_EndSort :
00836 */
00837
00864 int PF_EndSort(void)
00865 {
00866     GETIDENTITY
00867     FILEHANDLE *fout = AR.outfile;
00868     PF_BUFFER *sbuf=PF.sbuf;
00869     SORTING *S = AT.SS;
00870     WORD *outterm,*pp;
00871     LONG size, noutterms;
00872     POSITION position, oldposition;
00873     WORD i,cc;
00874     int oldgzipCompress;
00875
00876     if ( AT.SS != AT.S0 || !PF.parallel ) return 0;
00877
00878     if ( PF.me != MASTER ) {
00879         /*
00880             #[ the slaves have to initialize their sendbuffer :
00881
00882             this is a slave and it's PObuffer should be the minimum of the
00883             sortiosize on the master and the PObuffer of our file.
00884             First save the original PObuffer and PObuffer of the outfile
00885         */
00886         size = (S->sTop2 - S->lBuffer - 1)/(PF.numtasks - 1);
00887         size -= (AM.MaxTer/sizeof(WORD) + 2);
00888         if ( fout->POsize < (LONG)(size*sizeof(WORD)) ) size = fout->POsize/sizeof(WORD);
00889         if ( sbuf == NULL ) {
00890             if ( (sbuf = PF_AllocBuf(PF.numsbuffers, size*sizeof(WORD), 1)) == NULL ) return -1;
00891             sbuf->active = 0;
00892             PF.sbuf = sbuf;
00893         }

```

```

00894     sbuf->buff[0] = fout->PObuffer;
00895     sbuf->stop[0] = fout->PObuffer+size;
00896     if ( sbuf->stop[0] > fout->POstop ) return -1;
00897     for ( i = 0; i < PF.numsbuffers; i++ )
00898         sbuf->fill[i] = sbuf->full[i] = sbuf->buff[i];
00899
00900     fout->PObuffer = sbuf->buff[sbuf->active];
00901     fout->POstop = sbuf->stop[sbuf->active];
00902     fout->POsize = size*sizeof(WORD);
00903     fout->POfill = fout->POfull = fout->PObuffer;
00904 /*
00905     #] the slaves have to initialize their sendbuffer :
00906 */
00907     return(0);
00908 }
00909 /*
00910     this waits for all slaves to be ready to send terms back
00911 */
00912 PF_WaitAllSlaves(); /* Note, the returned value should be 0 on success. */
00913 /*
00914     Now collect the terms of all slaves and merge them.
00915     PF_GetLoser gives the position of the smallest term, which is the real
00916     work. The smallest term needs to be copied to the outbuf: use PutOut.
00917 */
00918 PF_InitTree();
00919 if ( AR.PolyFun == 0 ) { S->PolyFlag = 0; }
00920 else if ( AR.PolyFunType == 1 ) { S->PolyFlag = 1; }
00921 else if ( AR.PolyFunType == 2 ) {
00922     if ( AR.PolyFunExp == 2
00923         || AR.PolyFunExp == 3 ) S->PolyFlag = 1;
00924     else S->PolyFlag = 2;
00925 }
00926 *AR.CompressPointer = 0;
00927 SeekScratch(fout, &position);
00928 oldposition = position;
00929 oldgzipCompress = AR.gzipCompress;
00930 AR.gzipCompress = 0;
00931
00932 noutterms = 0;
00933
00934 while ( PF_loser >= 0 ) {
00935     if ( (PF_loser = PF_GetLoser(PF_root)) == 0 ) break;
00936     outterm = PF_term[PF_loser];
00937     noutterms++;
00938
00939     if ( PF_newclen[PF_loser] != 0 ) {
00940 /*
00941         #] this is only when new coeff was too long :
00942 */
00943         outterm = PF_WorkSpace;
00944         pp = PF_term[PF_loser];
00945         cc = *pp;
00946         while ( cc-- ) *outterm++ = *pp++;
00947         outterm = (outterm[-1] > 0) ? outterm-outterm[-1] : outterm+outterm[-1];
00948         if ( PF_newclen[PF_loser] > 0 ) cc = (WORD)PF_newclen[PF_loser] - 1;
00949         else cc = -(WORD)PF_newclen[PF_loser] - 1;
00950         pp = PF_newcpos[PF_loser];
00951         while ( cc-- ) *outterm++ = *pp++;
00952         *outterm++ = PF_newclen[PF_loser];
00953         *PF_WorkSpace = outterm - PF_WorkSpace;
00954         outterm = PF_WorkSpace;
00955         *PF_newcpos[PF_loser] = 0;
00956         PF_newclen[PF_loser] = 0;
00957 /*
00958         #] this is only when new coeff was too long :
00959 */
00960     }
00961     PRINTFBUF("PF_EndSort to PutOut: ",outterm,*outterm);
00962     PutOut(BHEAD outterm,&position,fout,1);
00963 }
00964 if ( FlushOut(&position,fout,0) ) {
00965     AR.gzipCompress = oldgzipCompress;
00966     return(-1);
00967 }
00968 S->TermsLeft = PF_goutterms = noutterms;
00969 DIFPOS(PF_exprsize, position, oldposition);
00970 AR.gzipCompress = oldgzipCompress;
00971 return(1);
00972 }
00973
00974 /*
00975     #] PF_EndSort :
00976     #] sort.c :
00977     #] proces.c :
00978     #] variables :
00979 */
00980

```

```

00981 static WORD *PF_CurrentBracket;
00982
00983 /*
00984     #] variables :
00985     #[ PF_GetTerm :
00986 */
00987
01006 static WORD PF_GetTerm(WORD *term)
01007 {
01008     GETIDENTITY
01009     FILEHANDLE *fi = AC.RhsExprInModuleFlag && PF.rhsInParallel ? &PF.slavebuf : AR.infile;
01010     WORD i;
01011     WORD *next, *np, *last, *lp = 0, *nextstop, *tp=term;
01012
01013     /* Only on the slaves. */
01014
01015     AN.deferskipped = 0;
01016     if ( fi->POfill >= fi->POfull || fi->POfull == fi->PObuffer ) {
01017 ReceiveNew:
01018     {
01019     /*
01020         #[ receive new terms from master :
01021     */
01022         int src = MASTER, tag;
01023         int follow = 0;
01024         LONG size,cpu,space = 0;
01025
01026         if ( PF.log ) {
01027             fprintf(stderr,"%d] Starting to send to Master\n",PF.me);
01028             fflush(stderr);
01029         }
01030
01031         cpu = TimeCPU(1);
01032         PF_PrepPack();
01033         PF_Pack(&cpu, 1,PF_LONG);
01034         PF_Pack(&space, 1,PF_LONG);
01035         PF_Pack(&PF_linterms, 1,PF_LONG);
01036         PF_Pack(&(AM.S0->GenTerms), 1,PF_LONG);
01037         PF_Pack(&(AM.S0->TermsLeft), 1,PF_LONG);
01038         PF_Pack(&follow, 1,PF_INT );
01039
01040         if ( PF.log ) {
01041             fprintf(stderr,"%d] Now sending with tag = %d\n",PF.me,PF_READY_MSGTAG);
01042             fflush(stderr);
01043         }
01044
01045         PF_Send(MASTER, PF_READY_MSGTAG);
01046
01047         if ( PF.log ) {
01048             fprintf(stderr,"%d] returning from send\n",PF.me);
01049             fflush(stderr);
01050         }
01051
01052         size = fi->POstop - fi->PObuffer - 1;
01053 #ifndef AbsolutelyExtra
01054         PF_Receive(MASTER,PF_ANY_MSGTAG,&src,&tag);
01055 #endif MPI2
01056         if ( tag == PF_TERM_MSGTAG ) {
01057             PF_Unpack(&size, 1, PF_LONG);
01058             if ( PF_Put_target(src) == 0 ) {
01059                 printf("PF_Put_target error ...\n");
01060             }
01061         }
01062         else {
01063             PF_RecvWbuf(fi->PObuffer,&size,&src);
01064         }
01065 #else
01066         PF_RecvWbuf(fi->PObuffer,&size,&src);
01067 #endif
01068 #endif
01069         tag=PF_RecvWbuf(fi->PObuffer,&size,&src);
01070
01071         fi->POfill = fi->PObuffer;
01072         /* Get AN.ninterms which sits in the first 2 WORDs. */
01073         {
01074             LONG ninterms;
01075             UNPACK_LONG(fi->POfill, ninterms);
01076             if ( fi->POfill < fi->POfull ) {
01077                 DBGOUT_NINTERMS(2, ("PF.me=%d AN.ninterms=%d PF_linterms=%d ninterms=%d GET\n",
(int)PF.me, (int)AN.ninterms, (int)PF_linterms, (int)ninterms));
01078                 AN.ninterms = ninterms - 1;
01079             } else {
01080                 DBGOUT_NINTERMS(2, ("PF.me=%d AN.ninterms=%d PF_linterms=%d ninterms=%d GETEND\n",
(int)PF.me, (int)AN.ninterms, (int)PF_linterms, (int)ninterms));
01081             }
01082         }
01083         fi->POfull = fi->PObuffer + size;

```

```

01084         if ( tag == PF_ENDSORT_MSGTAG ) *fi->POfull++ = 0;
01085 /*
01086     #] receive new terms from master :
01087 */
01088     }
01089     if ( PF_CurrentBracket ) *PF_CurrentBracket = 0;
01090 }
01091 if ( *fi->POfill == 0 ) {
01092     fi->POfill = fi->POfull = fi->PObuffer;
01093     *term = 0;
01094     goto RegRet;
01095 }
01096 if ( AR.DeferFlag ) {
01097     if ( !PF_CurrentBracket ) {
01098 /*
01099     #[ alloc space :
01100 */
01101         PF_CurrentBracket =
01102             (WORD*)Malloc1(AM.MaxTer,"PF_CurrentBracket");
01103         *PF_CurrentBracket = 0;
01104 /*
01105     #] alloc space :
01106 */
01107     }
01108     while ( *PF_CurrentBracket ) { /* "for each term in the buffer" */
01109 /*
01110     #[ test : bracket & skip if it's equal to the last in PF_CurrentBracket
01111 */
01112         next = fi->POfill;
01113         nextstop = next + *next; nextstop -= ABS(nextstop[-1]);
01114         next++;
01115         last = PF_CurrentBracket+1;
01116         while ( next < nextstop ) {
01117 /*
01118             scan the next term and PF_CurrentBracket
01119 */
01120             if ( *last == HAAKJE && *next == HAAKJE ) {
01121 /*
01122                 the part outside brackets is equal => skip this term
01123 */
01124                 PRINTFBUF("PF_GetTerm skips",fi->POfill,*fi->POfill);
01125                 break;
01126             }
01127 /*
01128             check if the current subterms are equal
01129 */
01130             np = next; next += next[1];
01131             lp = last; last += last[1];
01132             while ( np < next ) if ( *lp++ != *np++ ) goto strip;
01133         }
01134 /*
01135         go on to next term
01136 */
01137         fi->POfill += *fi->POfill;
01138         AN.deferskipped++;
01139 /*
01140         the usual checks
01141 */
01142         if ( fi->POfill >= fi->POfull || fi->POfull == fi->PObuffer )
01143             goto ReceiveNew;
01144         if ( *fi->POfill == 0 ) {
01145             fi->POfill = fi->POfull = fi->PObuffer;
01146             *term = 0;
01147             goto RegRet;
01148         }
01149 /*
01150     #] test :
01151 */
01152     }
01153 /*
01154     #[ copy :
01155 */
01156     this term to CurrentBracket and the part outside of bracket
01157     to Workspace at term
01158 */
01159 strip:
01160     next = fi->POfill;
01161     nextstop = next + *next; nextstop -= ABS(nextstop[-1]);
01162     next++;
01163     tp++;
01164     lp = PF_CurrentBracket + 1;
01165     while ( next < nextstop ) {
01166         if ( *next == HAAKJE ) {
01167             fi->POfill += *fi->POfill;
01168             while ( next < fi->POfill ) *lp++ = *next++;
01169             *PF_CurrentBracket = lp - PF_CurrentBracket;
01170             *lp = 0;

```

```

01171             *tp++ = 1;
01172             *tp++ = 1;
01173             *tp++ = 3;
01174             *term = WORDDIF(tp,term);
01175             PRINTFBUF("PF_GetTerm new brack",PF_CurrentBracket,*PF_CurrentBracket);
01176             PRINTFBUF("PF_GetTerm POfill",fi->POfill,*fi->POfill);
01177             goto RegRet;
01178         }
01179         np = next; next += next[1];
01180         while ( np < next ) *tp++ = *lp++ = *np++;
01181     }
01182     tp = term;
01183 /*
01184     #] copy :
01185 */
01186 }
01187
01188     i = *fi->POfill;
01189     while ( i-- ) *tp++ = *fi->POfill++;
01190 RegRet:
01191     PRINTFBUF("PF_GetTerm returns",term,*term);
01192     return(*term);
01193 }
01194
01195 /*
01196     #] PF_GetTerm :
01197     #[ PF_Deferred :
01198 */
01208 WORD PF_Deferred(WORD *term, WORD level)
01209 {
01210     GETIDENTITY
01211     WORD *bra, *bstop;
01212     WORD *tstart;
01213     FILEHANDLE *fi = AC.RhsExprInModuleFlag && PF.rhsInParallel ? &PF.slavebuf : AR.infile;
01214     WORD *next = fi->POfill;
01215     WORD *termout = AT.WorkPointer;
01216     WORD *oldwork = AT.WorkPointer;
01217
01218     AT.WorkPointer = (WORD *) ((UBYTE *) (AT.WorkPointer) + AM.MaxTer);
01219     AR.DeferFlag = 0;
01220
01221     PRINTFBUF("PF_Deferred (Term) ",term,*term);
01222     PRINTFBUF("PF_Deferred (Bracket)",PF_CurrentBracket,*PF_CurrentBracket);
01223
01224     bra = bstop = PF_CurrentBracket;
01225     if ( *bstop > 0 ) {
01226         bstop += *bstop;
01227         bstop -= ABS(bstop[-1]);
01228     }
01229     bra++;
01230     while ( *bra != HAAKJE && bra < bstop ) bra += bra[1];
01231     if ( bra >= bstop ) { /* No deferred action! */
01232         AT.WorkPointer = term + *term;
01233         if ( Generator(BHEAD term,level) ) goto DefCall;
01234         AR.DeferFlag = 1;
01235         AT.WorkPointer = oldwork;
01236         return(0);
01237     }
01238     bstop = bra;
01239     tstart = bra + bra[1];
01240     bra = PF_CurrentBracket;
01241     tstart--;
01242     *tstart = bra + *bra - tstart;
01243     bra++;
01244 /*
01245     Status of affairs:
01246     First bracket content starts at tstart.
01247     Next term starts at next.
01248     The outside of the bracket runs from bra = PF_CurrentBracket to bstop.
01249 */
01250     for(;;) {
01251         if ( InsertTerm(BHEAD term,0,AM.rbufnum,tstart,termout,0) < 0 ) {
01252             goto DefCall;
01253         }
01254 /*
01255         call Generator with new composed term
01256 */
01257         AT.WorkPointer = termout + *termout;
01258         if ( Generator(BHEAD termout,level) ) goto DefCall;
01259         AT.WorkPointer = termout;
01260         tstart = next + 1;
01261         if ( tstart >= fi->POfull ) goto ThatsIt;
01262         next += *next;
01263 /*
01264         compare with current bracket
01265 */

```



```

01266         while ( bra <= bstop ) {
01267             if ( *bra != *tstart ) goto ThatsIt;
01268             bra++; tstart++;
01269         }
01270     /*
01271         now bra and tstart should both be a HAAKJE
01272     */
01273     bra--; tstart--;
01274     if ( *bra != HAAKJE || *tstart != HAAKJE ) goto ThatsIt;
01275     tstart += tstart[1];
01276     tstart--;
01277     *tstart = next - tstart;
01278     bra = PF_CurrentBracket + 1;
01279 }
01280
01281 ThatsIt:
01282 /*
01283     AT.WorkPointer = oldwork;
01284 */
01285     AR.DeferFlag = 1;
01286     return(0);
01287 DefCall:
01288     MesCall("PF_Deferred");
01289     SETERROR(-1);
01290 }
01291
01292 /*
01293     #] PF_Deferred :
01294     #[ PF_Wait4Slave :
01295 */
01296
01297 static LONG **PF_W4Sstats = 0;
01298
01305 static int PF_Wait4Slave(int src)
01306 {
01307     int j, tag, next;
01308
01309     tag = PF_ANY_MSGTAG;
01310     PF_CatchErrorMessages(&src, &tag);
01311     PF_Receive(src, tag, &next, &tag);
01312
01313     if ( tag != PF_READY_MSGTAG ) {
01314         MesPrint("[%d] PF_Wait4Slave: received MSGTAG %d", (WORD)PF.me, (WORD)tag);
01315         return(-1);
01316     }
01317     if ( PF_W4Sstats == 0 ) {
01318         PF_W4Sstats = (LONG**)Malloc1(sizeof(LONG*), "");
01319         PF_W4Sstats[0] = (LONG*)Malloc1(PF_STATS_SIZE*sizeof(LONG), "");
01320     }
01321     PF_Unpack(PF_W4Sstats[0], PF_STATS_SIZE, PF_LONG);
01322     PF_Statistics(PF_W4Sstats, next);
01323
01324     PF_Unpack(&j, 1, PF_INT);
01325
01326     if ( j ) {
01327     /*
01328         actions depending on rest of information in last message
01329     */
01330     }
01331     return(next);
01332 }
01333
01334 /*
01335     #] PF_Wait4Slave :
01336     #[ PF_Wait4SlaveIP :
01337 */
01338 /*
01339     array of expression numbers for PF_InParallel processor.
01340     Each time the master sends expression "i" to the slave
01341     "next" it sets partodoexr[next]=i:
01342 */
01343 static WORD *partodoexr=NULL;
01344
01352 static int PF_Wait4SlaveIP(int *src)
01353 {
01354     int j, tag, next;
01355
01356     tag = PF_ANY_MSGTAG;
01357     PF_CatchErrorMessages(src, &tag);
01358     PF_Receive(*src, tag, &next, &tag);
01359     *src=tag;
01360     if ( PF_W4Sstats == 0 ) {
01361         PF_W4Sstats = (LONG**)Malloc1(sizeof(LONG*), "");
01362         PF_W4Sstats[0] = (LONG*)Malloc1(PF_STATS_SIZE*sizeof(LONG), "");
01363     }
01364
01365     PF_Unpack(PF_W4Sstats[0], PF_STATS_SIZE, PF_LONG);

```

```

01366     if ( tag == PF_DATA_MSGTAG )
01367         AR.CurExpr = partodoexr[next];
01368     PF_Statistics(PF_W4Sstats,next);
01369
01370     PF_Unpack(&j,1,PF_INT);
01371
01372     if ( j ) {
01373         /* actions depending on rest of information in last message */
01374     }
01375
01376     return(next);
01377 }
01378 /*
01379     #] PF_Wait4SlaveIP :
01380     #[ PF_WaitAllSlaves :
01381 */
01382
01391 static int PF_WaitAllSlaves(void)
01392 {
01393     int i, readySlaves, tag, next = PF_ANY_SOURCE;
01394     UBYTE *has_sent = 0;
01395
01396     has_sent = (UBYTE*)Malloc1(sizeof(UBYTE)*(PF.numtasks + 1),"PF_WaitAllSlaves");
01397     for ( i = 0; i < PF.numtasks; i++ ) has_sent[i] = 0;
01398
01399     for ( readySlaves = 1; readySlaves < PF.numtasks; ) {
01400         if ( next != PF_ANY_SOURCE ) { /*Go to the next slave:*/
01401             do{ /*Note, here readySlaves<PF.numtasks, so this loop can't be infinite*/
01402                 if ( ++next >= PF.numtasks ) next = 1;
01403             } while ( has_sent[next] == 1 );
01404         }
01405         /*
01406             Here PF_ProbeWithCatchingErrorMessages() is BLOCKING function if next = PF_ANY_SOURCE:
01407         */
01408         tag = PF_ProbeWithCatchingErrorMessages(&next);
01409         /*
01410             Here next != PF_ANY_SOURCE
01411         */
01412         switch ( tag ) {
01413             case PF_BUFFER_MSGTAG:
01414             case PF_ENDBUFFER_MSGTAG:
01415                 /*
01416                     Slaves are ready to send their results back
01417                 */
01418                 if ( has_sent[next] == 0 ) {
01419                     has_sent[next] = 1;
01420                     readySlaves++;
01421                 }
01422                 else { /*error?*/
01423                     fprintf(stderr,"ERROR next=%d tag=%d\n",next,tag);
01424                 }
01425                 /*
01426                     Note, we do NOT read results here! Messages from these slaves will be read
01427                     only after all slaves are ready, further in caller function
01428                 */
01429                 break;
01430             case 0:
01431                 /*
01432                     The slave is not ready. Just go to the next slave.
01433                     It may appear that there are no more ready slaves, and the master
01434                     will wait them in infinite loop. Stupid situation - the master can
01435                     receive buffers from ready slaves!
01436                 */
01437                 #ifdef PF_WITH_SCHED_YIELD
01438                 /*
01439                     Relinquish the processor:
01440                 */
01441                 sched_yield();
01442                 #endif
01443                 break;
01444             case PF_DATA_MSGTAG:
01445                 tag=next;
01446                 next=PF_Wait4SlaveIP(&tag);
01447         }
01448         /*
01449             tag must be == PF_DATA_MSGTAG!
01450         */
01451         PF_Statistics(PF_stats,0);
01452         PF_Slave2MasterIP(next);
01453         PF_Master2SlaveIP(next,NULL);
01454         if ( has_sent[next] == 0 ) {
01455             has_sent[next]=1;
01456             readySlaves++;
01457         }
01458         else{
01459             /*error?*/
01460             fprintf(stderr,"ERROR next=%d tag=%d\n",next,tag);
01461             /*if ( has_sent[next] == 0 )*/
01462             break;
01463         }
01464     }

```

```

01461         case PF_EMPTY_MSGTAG:
01462             tag=next;
01463             next=PF_Wait4SlaveIP(&tag);
01464         /*
01465          tag must be == PF_EMPTY_MSGTAG!
01466          */
01467             PF_Master2SlaveIP(next,NULL);
01468             if ( has_sent[next] == 0 ) {
01469                 has_sent[next]=1;
01470                 readySlaves++;
01471             }else{
01472                 /*error?*/
01473                 fprintf(stderr,"ERROR next=%d tag=%d\n",next,tag);
01474                 /*if ( has_sent[next] == 0 )*/
01475                 break;
01476         case PF_READY_MSGTAG:
01477         /*
01478             idle slave
01479             May be only PF_READY_MSGTAG:
01480             */
01481             next = PF_Wait4Slave(next);
01482             if ( next == -1 ) return(next); /*Cannot be!*/
01483             if ( has_sent[0] == 0 ) { /*Send the last chunk to the slave*/
01484                 PF.sbuf->active = 0;
01485                 has_sent[0] = 1;
01486             }
01487             else {
01488         /*
01489                 Last chunk was sent, so just send to slave ENDSORT
01490                 AN.ninterms must be sent because the slave expects it:
01491             */
01492                 PACK_LONG(PF.sbuf->fill[next], AN.ninterms);
01493             /*
01494                 This will tell to the slave that there are no more terms:
01495             */
01496                 *(PF.sbuf->fill[next])++ = 0;
01497                 PF.sbuf->active = next;
01498             }
01499         /*
01500                 Send ENDSORT
01501             */
01502                 PF_ISendSbuf(next,PF_ENDSORT_MSGTAG);
01503                 break;
01504         default:
01505         /*
01506                 Error?
01507                 Indicates the error. This will force exit from the main loop:
01508             */
01509                 MesPrint("!!!Unexpected MPI message src=%d tag=%d.", next, tag);
01510                 readySlaves = PF.numtasks+1;
01511                 break;
01512             }
01513         }
01514     }
01515     if ( has_sent ) M_free(has_sent,"PF_WaitAllSlaves");
01516     /*
01517         0 on success (exit from the main loop by loop condition), or -1 if fails
01518         (exit from the main loop since readySlaves=PF.numtasks+1):
01519     */
01520     return(PF.numtasks-readySlaves);
01521 }
01522
01523 /*
01524     #] PF_WaitAllSlaves :
01525     #[ PF_Processor :
01526 */
01527
01540 int PF_Processor(EXPRESSIONS e, WORD i, WORD LastExpression)
01541 {
01542     GETIDENTITY
01543     WORD *term = AT.WorkPointer;
01544     LONG dd = 0;
01545     PF_BUFFER *sb = PF.sbuf;
01546     WORD j, *s, next;
01547     LONG size, cpu;
01548     POSITION position;
01549     int k, src, tag;
01550     FILEHANDLE *oldoutfile = AR.outfile;
01551
01552 #ifndef MPI2
01553     if ( PF_shared_buff == NULL ) {
01554         if ( PF_SMWin_Init() == 0 ) {
01555             MesPrint("PF_SMWin_Init error");
01556             exit(-1);
01557         }
01558     }
01559 #endif

```

```

01560
01561     if ( ( (WORD *) ((UBYTE *) (AT.WorkPointer)) + AM.MaxTer ) > AT.WorkTop ) return(MesWork());
01562
01563     /* For redefine statements. */
01564     if ( AC.numpfirstnum > 0 ) {
01565         for ( j = 0; j < AC.numpfirstnum; j++ ) {
01566             AC.inputnumbers[j] = -1;
01567         }
01568     }
01569
01570     if ( AC.mparallelfalg != PARALLELFLAG ) return(0);
01571
01572     if ( PF.me == MASTER ) {
01573         /*
01574             #[ Master:
01575                 #[ write prototype to outfile:
01576             */
01577             WORD oldBracketOn = AR.BracketOn;
01578             WORD *oldBrackBuf = AT.BrackBuf;
01579             WORD oldbracketindexflag = AT.bracketindexflag;
01580
01581             LONG maxinterms;      /* the maximum number of terms in the bucket */
01582             int cmaxinterms;      /* a variable controlling the transition of maxinterms */
01583             LONG termsinbucket;   /* the number of filled terms in the bucket */
01584             LONG ProcessBucketSize = AC.mProcessBucketSize;
01585
01586             if ( PF.log && AC.CModule >= PF.log )
01587                 MesPrint("[%d] working on expression %s in module %l",PF.me,EXPRNAME(i),AC.CModule);
01588             if ( GetTerm(BHEAD term) <= 0 ) {
01589                 MesPrint("[%d] Expression %d has problems in scratchfile",PF.me,i);
01590                 return(-1);
01591             }
01592             term[3] = i;
01593             if ( AR.outtohide ) {
01594                 SeekScratch(AR.hidefile,&position);
01595                 e->onfile = position;
01596                 if ( PutOut(BHEAD term,&position,AR.hidefile,0) < 0 ) return(-1);
01597             }
01598             else {
01599                 SeekScratch(AR.outfile,&position);
01600                 e->onfile = position;
01601                 if ( PutOut(BHEAD term,&position,AR.outfile,0) < 0 ) return(-1);
01602             }
01603             AR.DeferFlag = 0; /* The master leave the brackets!!! */
01604             AR.Eside = RHSIDE;
01605             if ( ( e->vflags & ISFACTORIZED ) != 0 ) {
01606                 AR.BracketOn = 1;
01607                 AT.BrackBuf = AM.BracketFactors;
01608                 AT.bracketindexflag = 1;
01609             }
01610             if ( AT.bracketindexflag > 0 ) OpenBracketIndex(i);
01611         /*
01612             #[ write prototype to outfile:
01613             #[ initialize sendbuffer if necessary:
01614
01615             the size of the sendbufs is:
01616             MIN(1/PF.numtasks*(AT.SS->sBufsize+AT.SS->lBufsize),AR.infile->POsize)
01617             No allocation for extra buffers necessary, just make sb->buf... point
01618             to the right places in the sortbuffers.
01619         */
01620         NewSort(BHEAD0); /* we need AT.SS to be set for this!!! */
01621         if ( sb == 0 || sb->buff[0] != AT.SS->lBuffer ) {
01622             size = (LONG)((AT.SS->sTop2 - AT.SS->lBuffer)/(PF.numtasks));
01623             if ( size > (LONG)(AR.infile->POsize/sizeof(WORD) - 1) )
01624                 size = AR.infile->POsize/sizeof(WORD) - 1;
01625             if ( sb == 0 ) {
01626                 if ( ( sb = PF_AllocBuf(PF.numtasks,size*sizeof(WORD),PF.numtasks) ) == NULL )
01627                     return(-1);
01628             }
01629             sb->buff[0] = AT.SS->lBuffer;
01630             sb->full[0] = sb->fill[0] = sb->buff[0];
01631             for ( j = 1; j < PF.numtasks; j++ ) {
01632                 sb->stop[j-1] = sb->buff[j] = sb->buff[j-1] + size;
01633             }
01634             sb->stop[PF.numtasks-1] = sb->buff[PF.numtasks-1] + size;
01635             PF.sbuf = sb;
01636         }
01637         for ( j = 0; j < PF.numtasks; j++ ) {
01638             sb->full[j] = sb->fill[j] = sb->buff[j];
01639         }
01640     /*
01641         #[ initialize sendbuffer if necessary:
01642         #[ loop for all terms in infile:
01643     */
01644     /*
01645     * The initial value of maxinterms is determined by the user given
01646     * ProcessBucketSize and the number of terms in the current expression.

```

```

01647      * We make the initial maxinterms smaller, so that we get the all
01648      * workers busy as soon as possible.
01649      */
01650      maxinterms = ProcessBucketSize / 100;
01651      if ( maxinterms > e->counter / (PF.numtasks - 1) / 4 )
01652          maxinterms = e->counter / (PF.numtasks - 1) / 4;
01653      if ( maxinterms < 1 ) maxinterms = 1;
01654      cmaxinterms = 0;
01655      /*
01656      * Copy them always to sb->buff[0]. When that is full, wait for
01657      * the next slave to accept terms, exchange sb->buff[0] and
01658      * sb->buff[next], send sb->buff[next] to next slave and go on
01659      * filling the now empty sb->buff[0].
01660      */
01661      AN.ninterms = 0;
01662      termsinbucket = 0;
01663      PACK_LONG(sb->fill[0], 1);
01664      while ( GetTerm(BHEAD term) ) {
01665          AN.ninterms++; dd = AN.deferskipped;
01666          if ( AC.CollectFun && *term <= (LONG)(AM.MaxTer/(2*sizeof(WORD))) ) {
01667              if ( GetMoreTerms(term) < 0 ) {
01668                  LowerSortLevel(); return(-1);
01669              }
01670          }
01671          PRINTFBUF("PF_Processor gets",term,*term);
01672          if ( termsinbucket >= maxinterms || sb->fill[0] + *term >= sb->stop[0] ) {
01673              next = PF_Wait4Slave(PF_ANY_SOURCE);
01674
01675              sb->fill[next] = sb->fill[0];
01676              sb->full[next] = sb->full[0];
01677              SWAP(sb->stop[next], sb->stop[0]);
01678              SWAP(sb->buff[next], sb->buff[0]);
01679              sb->fill[0] = sb->full[0] = sb->buff[0];
01680              sb->active = next;
01681
01682          #ifdef MPI2
01683              if ( PF_Put_origin(next) == 0 ) {
01684                  printf("PF_Put_origin error...\n");
01685              }
01686          #else
01687              PF_ISendSbuf(next,PF_TERM_MSGTAG);
01688          #endif
01689
01690          /* Initialize the next bucket. */
01691          termsinbucket = 0;
01692          PACK_LONG(sb->fill[0], AN.ninterms);
01693          /*
01694          * For the "slow startup". We double maxinterms up to ProcessBucketSize
01695          * after (hopefully) the all workers got some terms.
01696          */
01697          if ( cmaxinterms >= PF.numtasks - 2 ) {
01698              maxinterms *= 2;
01699              if ( maxinterms >= ProcessBucketSize ) {
01700                  cmaxinterms = -1;
01701                  maxinterms = ProcessBucketSize;
01702              }
01703          }
01704          else if ( cmaxinterms >= 0 ) {
01705              cmaxinterms++;
01706          }
01707          j = *(s = term);
01708          NCOPY(sb->fill[0], s, j);
01709          termsinbucket++;
01710      }
01711      /* NOTE: The last chunk will be sent to a slave at EndSort() => PF_EndSort()
01712      *      => PF_WaitAllSlaves(). */
01713      AN.ninterms += dd;
01714      /*
01715      #] loop for all terms in infile:
01716      #[ Clean up & EndSort:
01717      */
01718      if ( LastExpression ) {
01719          UpdateMaxSize();
01720          if ( AR.infile->handle >= 0 ) {
01721              CloseFile(AR.infile->handle);
01722              AR.infile->handle = -1;
01723              remove(AR.infile->name);
01724              PUTZERO(AR.infile->P0position);
01725          }
01726          AR.infile->P0fill = AR.infile->P0full = AR.infile->P0buffer;
01727      }
01728      if ( AR.outtohide ) AR.outfile = AR.hidefile;
01729      PF.parallel = 1;
01730      if ( EndSort(BHEAD AM.S0->sBuffer,0) < 0 ) return(-1);
01731      PF.parallel = 0;
01732      if ( AR.outtohide ) {
01733          AR.outfile = oldoutfile;

```

```

01734         AR.hidefile->POfull = AR.hidefile->POfill;
01735     }
01736     UpdateMaxSize();
01737     AR.BraceOn = oldBracketOn;
01738     AT.BrackBuf = oldBrackBuf;
01739     if ( ( e->vflags & TOBEFACTORED ) != 0 )
01740         poly_factorize_expression(e);
01741     else if ( ( ( e->vflags & TOBEUNFACTORED ) != 0 )
01742             && ( ( e->vflags & ISFACTORIZED ) != 0 ) )
01743         poly_unfactorize_expression(e);
01744     AT.bracketindexflag = oldbracketindexflag;
01745     AR.GetFile = 0;
01746     AR.outtohide = 0;
01747     /*
01748     * NOTE: e->numdummies, e->vflags and AR.exprflags will be updated
01749     *       after gathering the information from all slaves.
01750     */
01751 /*
01752     #] Clean up & EndSort:
01753     #[ Collect (stats,prepro,...):
01754 */
01755     DBGOUT_NINTERMS(1, ("PF.me=%d AN.ninterms=%d ENDSORT\n", (int)PF.me, (int)AN.ninterms));
01756     PF_CatchErrorMessageForAll();
01757     e->numdummies = 0;
01758     for ( k = 1; k < PF.numtasks; k++ ) {
01759         PF_LongSingleReceive(PF_ANY_SOURCE, PF_ENDSORT_MSGTAG, &src, &tag);
01760         PF_LongSingleUnpack(PF_stats[src], PF_STATS_SIZE, PF_LONG);
01761         {
01762             WORD numdummies, expchanged;
01763             PF_LongSingleUnpack(&numdummies, 1, PF_WORD);
01764             PF_LongSingleUnpack(&expchanged, 1, PF_WORD);
01765             if ( e->numdummies < numdummies ) e->numdummies = numdummies;
01766             AR.expchanged |= expchanged;
01767         }
01768         /* Now handle redefined preprocessor variables. */
01769         if ( AC.numpfirstnum > 0 ) PF_UnpackRedefinedPreVars();
01770     }
01771     /* Broadcast redefined preprocessor variables. */
01772     if ( AC.numpfirstnum > 0 ) {
01773         int RetCode = PF_BroadcastRedefinedPreVars();
01774         if ( RetCode ) return RetCode;
01775     }
01776     if ( ! AC.OldParallelStats ) {
01777         /* Now we can calculate AT.SS->GenTerms from the statistics of the slaves. */
01778         LONG genterms = 0;
01779         for ( k = 1; k < PF.numtasks; k++ ) {
01780             genterms += PF_stats[k][3];
01781         }
01782         AT.SS->GenTerms = genterms;
01783         WriteStats(&PF_exprsize, STATSPOSTSORT, NOCHECKLOGTYPE);
01784         Expressions[AR.CurExpr].size = PF_exprsize;
01785     }
01786     PF_Statistics(PF_stats,0);
01787 /*
01788     #] Collect (stats,prepro,...):
01789     #[ Update flags :
01790 */
01791     if ( AM.S0->TermsLeft ) e->vflags &= ~ISZERO;
01792     else e->vflags |= ISZERO;
01793     if ( AR.expchanged == 0 ) e->vflags |= ISUNMODIFIED;
01794     if ( AM.S0->TermsLeft ) AR.expflags |= ISZERO;
01795     if ( AR.expchanged ) AR.expflags |= ISUNMODIFIED;
01796 /*
01797     #] Update flags :
01798     #] Master:
01799 */
01800 }
01801 else {
01802 /*
01803     #[ Slave :
01804 */
01805 /*
01806         #[ Generator Loop & EndSort :
01807
01808         loop for all terms to get from master, call Generator for each of them
01809         then call EndSort and do cleanup (to be implemented)
01810 */
01811         WORD oldBracketOn = AR.BraceOn;
01812         WORD *oldBrackBuf = AT.BrackBuf;
01813         WORD oldbracketindexflag = AT.bracketindexflag;
01814
01815         /* For redefine statements. */
01816         if ( AC.numpfirstnum > 0 ) {
01817             for ( j = 0; j < AC.numpfirstnum; j++ ) {
01818                 AC.inputnumbers[j] = -1;
01819             }
01820         }

```

```

01821
01822     SeekScratch(AR.outfile,&position);
01823     e->onfile = position;
01824     AR.DeferFlag = AC.ComDefer;
01825     AR.Eside = RHSIDE;
01826     if ( ( e->vflags & ISFACTORIZED ) != 0 ) {
01827         AR.BracketOn = 1;
01828         AT.BrackBuf = AM.BracketFactors;
01829         AT.bracketindexflag = 1;
01830     }
01831     NewSort(BHEAD0);
01832     AR.MaxDum = AM.IndDum;
01833     AN.ninterms = 0;
01834     PF_linterms = 0;
01835     PF.parallel = 1;
01836 #ifdef MPI2
01837     AR.infile->POfull = AR.infile->POfill = AR.infile->PObuffer = PF_shared_buff;
01838 #endif
01839     {
01840         FILEHANDLE *fi = AC.RhsExprInModuleFlag && PF.rhsInParallel ? &PF.slavebuf : AR.infile;
01841         fi->POfull = fi->POfill = fi->PObuffer;
01842     }
01843     /* FIXME: AN.ninterms is still broken when AN.deferskipped is non-zero.
01844      *      It still needs some work, also in PF_GetTerm(). (TU 30 Aug 2011) */
01845     while ( PF_GetTerm(term) ) {
01846         PF_linterms++; AN.ninterms++; dd = AN.deferskipped;
01847         AT.WorkPointer = term + *term;
01848         AN.RepPoint = AT.RepCount + 1;
01849         if ( ( e->vflags & ISFACTORIZED ) != 0 && term[1] == HAAKJE ) {
01850             StoreTerm(BHEAD term);
01851             continue;
01852         }
01853         if ( AR.DeferFlag ) {
01854             AR.CurDum = AN.IndDum = Expressions[AR.CurExpr].numdummies + AM.IndDum;
01855         }
01856         else {
01857             AN.IndDum = AM.IndDum;
01858             AR.CurDum = ReNumber(BHEAD term);
01859         }
01860         if ( AC.SymChangeFlag ) MarkDirty(term,DIRTYSYMLAG);
01861         if ( AN.ncmod ) {
01862             if ( ( AC.modmode & ALSOFUNARGS ) != 0 ) MarkDirty(term,DIRTYFLAG);
01863             else if ( AR.PolyFun ) PolyFunDirty(BHEAD term);
01864         }
01865         else if ( AC.PolyRatFunChanged ) PolyFunDirty(BHEAD term);
01866         if ( ( AR.PolyFunType == 2 ) && ( AC.PolyRatFunChanged == 0 )
01867             && ( e->status == LOCALEXPRESSION || e->status == GLOBALEXPRESSION ) ) {
01868             PolyFunClean(BHEAD term);
01869         }
01870         if ( Generator(BHEAD term,0) ) {
01871             MesPrint("[%d] PF_Processor: Error in Generator",PF.me);
01872             LowerSortLevel(); return(-1);
01873         }
01874         PF_linterms += dd; AN.ninterms += dd;
01875     }
01876     PF_linterms += dd; AN.ninterms += dd;
01877     {
01878         /*
01879          * EndSort() overrides AR.outfile->PObuffer etc. (See also PF_EndSort()),
01880          * but it causes a problem because
01881          * (1) PF_EndSort() sets AR.outfile->PObuffer to a send-buffer.
01882          * (2) RevertScratch() clears AR.infile, but then swaps buffers of AR.infile
01883          *     and AR.outfile.
01884          * (3) RHS expressions are stored to AR.infile->PObuffer.
01885          * (4) Again, PF_EndSort() sets AR.outfile->PObuffer, but now AR.outfile->PObuffer
01886          *     == AR.infile->PObuffer because of (1) and (2).
01887          * (5) The result goes to AR.outfile. This breaks the RHS expressions,
01888          *     which may be needed for the next expression.
01889          * Solution: backup & restore AR.outfile->PObuffer etc. (TU 14 Sep 2011)
01890          */
01891         FILEHANDLE *fout = AR.outfile;
01892         WORD *oldbuff = fout->PObuffer;
01893         WORD *oldstop = fout->POstop;
01894         LONG oldsize = fout->POsize;
01895         if ( EndSort(BHEAD AM.S0->sBuffer, 0) < 0 ) return -1;
01896         fout->PObuffer = oldbuff;
01897         fout->POstop = oldstop;
01898         fout->POsize = oldsize;
01899         fout->POfill = fout->POfull = fout->PObuffer;
01900     }
01901     AR.BracketOn = oldBracketOn;
01902     AT.BrackBuf = oldBrackBuf;
01903     AT.bracketindexflag = oldbracketindexflag;
01904     /*
01905         #] Generator Loop & EndSort :
01906         #[ Collect (stats,prepro...) :
01907     */

```

```

01908     DBGOUT_NINTERMS(1, ("PF.me=%d AN.ninterms=%d PF_linterms=%d ENDSORT\n", (int)PF.me,
(int)AN.ninterms, (int)PF_linterms));
01909     PF_PrepereLongSinglePack();
01910     cpu = TimeCPU(1);
01911     size = 0;
01912     PF_LongSinglePack(&cpu, 1, PF_LONG);
01913     PF_LongSinglePack(&size, 1, PF_LONG);
01914     PF_LongSinglePack(&PF_linterms, 1, PF_LONG);
01915     PF_LongSinglePack(&AM.S0->GenTerms, 1, PF_LONG);
01916     PF_LongSinglePack(&AM.S0->TermsLeft, 1, PF_LONG);
01917     {
01918         WORD numdummies = AR.MaxDum - AM.IndDum;
01919         PF_LongSinglePack(&numdummies, 1, PF_WORD);
01920         PF_LongSinglePack(&AR.expchanged, 1, PF_WORD);
01921     }
01922     /* Now handle redefined preprocessor variables. */
01923     if ( AC.numpfirstnum > 0 ) PF_PackRedefinedPreVars();
01924     PF_LongSingleSend(MASTER, PF_ENDSORT_MSGTAG);
01925     /* Broadcast redefined preprocessor variables. */
01926     if ( AC.numpfirstnum > 0 ) {
01927         int RetCode = PF_BroadcastRedefinedPreVars();
01928         if ( RetCode ) return RetCode;
01929     }
01930     /*
01931         #] Collect (stats,prepro...) :
01932
01933         This operation is moved to the beginning of each block, see PreProcessor
01934         in pre.c.
01935
01936         #] Slave :
01937     */
01938     if ( PF.log ) {
01939         UBYTE lbuf[24];
01940         NumToStr(lbuf, AC.CModule);
01941         fprintf(stderr, "[%d] [%s] Endsort,Collect,Broadcast done\n", PF.me, lbuf);
01942         fflush(stderr);
01943     }
01944 }
01945 return(0);
01946 }
01947
01948 /*
01949     #] PF_Processor :
01950     #] proces.c :
01951     #[ startup :, prepro & compile
01952     #[ PF_Init :
01953 */
01954
01963 int PF_Init(int *argc, char ***argv)
01964 {
01965     /*
01966         this should definitely be somewhere else ...
01967     */
01968     PF_CurrentBracket = 0;
01969
01970     PF.numtasks = 0; /* number of tasks, is determined in PF_LibInit ! */
01971     PF.numbufs = 2; /* might be changed by the environment variable on the master ! */
01972     PF.numrbufs = 2; /* might be changed by the environment variable on the master ! */
01973
01974     int ret = PF_LibInit(argc, argv);
01975     if (ret) { return ret; }
01976     PF_RealTime(PF_RESET);
01977
01978     PF.log = 0;
01979     PF.parallel = 0;
01980     PF_statsinterval = 10;
01981     PF.rhsInParallel=1;
01982     PF.exprbufsize=4096; /*in WORDs*/
01983
01984     #ifdef PF_WITHGETENV
01985     if ( PF.me == MASTER ) {
01986         char *c;
01987     /*
01988         get these from the environment at the moment should be in setfile/tail
01989     */
01990     if ( ( c = getenv("PF_LOG") ) != 0 ) {
01991         if ( *c ) PF.log = (int)atoi(c);
01992         else PF.log = 1;
01993         fprintf(stderr, "[%d] changing PF.log to %d\n", PF.me, PF.log);
01994         fflush(stderr);
01995     }
01996     if ( ( c = (char*)getenv("PF_RBUFS") ) != 0 ) {
01997         PF.numrbufs = (int)atoi(c);
01998         fprintf(stderr, "[%d] changing numrbufs to: %d\n", PF.me, PF.numrbufs);
01999         fflush(stderr);
02000     }
02001     if ( ( c = (char*)getenv("PF_SBUFS") ) != 0 ) {

```



```

02002         PF.numsbufs = (int)atoi(c);
02003         fprintf(stderr, "[%d] changing numsbufs to: %d\n", PF.me, PF.numsbufs);
02004         fflush(stderr);
02005     }
02006     if ( PF.numsbufs > 10 ) PF.numsbufs = 10;
02007     if ( PF.numsbufs < 1 ) PF.numsbufs = 1;
02008     if ( PF.numrbufs > 2 ) PF.numrbufs = 2;
02009     if ( PF.numrbufs < 1 ) PF.numrbufs = 1;
02010
02011     if ( ( c = getenv("PF_STATS") ) ) {
02012         UBYTE lbuf[24];
02013         PF_statsinterval = (int)atoi(c);
02014         NumToStr(lbuf, PF_statsinterval);
02015         fprintf(stderr, "[%d] changing PF_statsinterval to %s\n", PF.me, lbuf);
02016         fflush(stderr);
02017         if ( PF_statsinterval < 1 ) PF_statsinterval = 10;
02018     }
02019 }
02020 #endif
02021 /*
02022  #[ Broadcast settings from getenv: could also be done in PF_DoSetup
02023 */
02024 if ( PF.me == MASTER ) {
02025     PF_PrepPack();
02026     PF_Pack(&PF.log, 1, PF_INT);
02027     PF_Pack(&PF.numrbufs, 1, PF_WORD);
02028     PF_Pack(&PF.numsbufs, 1, PF_WORD);
02029 }
02030 PF_Broadcast();
02031 if ( PF.me != MASTER ) {
02032     PF_Unpack(&PF.log, 1, PF_INT);
02033     PF_Unpack(&PF.numrbufs, 1, PF_WORD);
02034     PF_Unpack(&PF.numsbufs, 1, PF_WORD);
02035     if ( PF.log ) {
02036         fprintf(stderr, "[%d] log=%d rbufs=%d sbufs=%d\n",
02037             PF.me, PF.log, PF.numrbufs, PF.numsbufs);
02038         fflush(stderr);
02039     }
02040 }
02041 /*
02042  #] Broadcast settings from getenv:
02043 */
02044 return(0);
02045 }
02046 /*
02047     #] PF_Init :
02048     #[ PF_Terminate :
02049 */
02050
02058 int PF_Terminate(int errorcode)
02059 {
02060     return PF_LibTerminate(errorcode);
02061 }
02062
02063 /*
02064     #] PF_Terminate :
02065     #[ PF_GetSlaveTimes :
02066 */
02067
02074 LONG PF_GetSlaveTimes(void)
02075 {
02076     LONG slavetimes = 0;
02077     LONG t = PF.me == MASTER ? 0 : AM.SumTime + TimeCPU(1);
02078     MPI_Reduce(&t, &slavetimes, 1, PF_LONG, MPI_SUM, MASTER, PF_COMM);
02079     return slavetimes;
02080 }
02081
02082 /*
02083     #] PF_GetSlaveTimes :
02084     #] startup :
02085     #[ PF_BroadcastNumber :
02086 */
02087
02094 LONG PF_BroadcastNumber(LONG x)
02095 {
02096     #ifdef PF_DEBUG_BCAST_LONG
02097         if ( PF.me == MASTER ) {
02098             MesPrint(">> Broadcast LONG: %l", x);
02099         }
02100     #endif
02101     PF_Bcast(&x, sizeof(LONG));
02102     return x;
02103 }
02104
02105 /*
02106     #] PF_BroadcastNumber :
02107     #[ PF_BroadcastBuffer :

```

```

02108 */
02109
02121 void PF_BroadcastBuffer(WORD **buffer, LONG *length)
02122 {
02123     WORD *p;
02124     LONG rest;
02125 #ifdef PF_DEBUG_BCAST_BUF
02126     if ( PF.me == MASTER ) {
02127         MesPrint("» Broadcast Buffer: length=%l", *length);
02128     }
02129 #endif
02130     /* Initialize the buffer on the slaves. */
02131     if ( PF.me != MASTER ) {
02132         *buffer = NULL;
02133     }
02134     /* Broadcast the length of the buffer. */
02135     *length = PF_BroadcastNumber(*length);
02136     if ( *length <= 0 ) return;
02137     /* Allocate the buffer on the slaves. */
02138     if ( PF.me != MASTER ) {
02139         *buffer = (WORD *)Malloc1(*length * sizeof(WORD), "PF_BroadcastBuffer");
02140     }
02141     /* Broadcast the data in the buffer. */
02142     p = *buffer;
02143     rest = *length;
02144     while ( rest > 0 ) {
02145         int l = rest < (LONG)PF.exprbufsize ? (int)rest : PF.exprbufsize;
02146         PF_Bcast(p, l * sizeof(WORD));
02147         p += l;
02148         rest -= l;
02149     }
02150 }
02151
02152 /*
02153 #] PF_BroadcastBuffer :
02154 #[ PF_BroadcastString :
02155 */
02156
02163 int PF_BroadcastString(UBYTE *str)
02164 {
02165     int clength = 0;
02166     /*
02167         If string does not fit to the PF_buffer, it
02168         will be split into chunks. Next chunk is started at str+clength
02169     */
02170     UBYTE *cstr=str;
02171     /*
02172         Note, compilation is performed INDEPENDENTLY on AC.mparallelfalg!
02173         No if ( AC.mparallelfalg == PARALLELFALG ) !!
02174     */
02175     do {
02176         cstr += clength; /*at each step for all slaves and master */
02177         if ( MASTER == PF.me ) { /*Pack str*/
02178             initialize buffers
02179             if ( PF_PreparePack() != 0 ) Terminate(-1);
02180             if ( ( clength = PF_PackString(cstr) ) < 0 ) Terminate(-1);
02181             PF_Broadcast();
02182             if ( MASTER != PF.me ) {
02183                 Slave - unpack received string
02184                 For slaves buffers are initialised automatically.
02185                 if ( ( clength = PF_UnpackString(cstr) ) < 0 ) Terminate(-1);
02186             }
02187             while ( cstr[clength-1] != '\0' );
02188             return (0);
02189         }
02190     } while ( cstr[clength-1] != '\0' );
02191     return (0);
02192 }
02193
02194 /*
02195 #] PF_BroadcastString :
02196 #[ PF_BroadcastPreDollar :
02197 */
02198
02218 int PF_BroadcastPreDollar(WORD **dbuffer, LONG *newsize, int *numterms)
02219 {
02220     int err = 0;
02221     LONG i;
02222     /*
02223         Note, compilation is performed INDEPENDENTLY on AC.mparallelfalg!
02224         No if(AC.mparallelfalg==PARALLELFALG) !!
02225     */
02226     if ( MASTER == PF.me ) {

```

```

02227 /*
02228     The problem is that sometimes dollar variables are longer
02229     than PF_packbuf! So we split long expression into chunks.
02230     There are n filled chunks and one partially filled chunk:
02231 */
02232 LONG n = ((*newsize)+1)/PF_maxDollarChunkSize;
02233 /*
02234     ...and one more chunk for the rest; if the expression fits to
02235     the buffer without splitting, the latter will be the only one.
02236
02237     PF_maxDollarChunkSize is the maximal number of items fitted to
02238     the buffer. It is calculated in PF_LibInit() in mpi.c.
02239     PF_maxDollarChunkSize is calculated for the first step, when
02240     two fields (numterms and newsize, see below) are already packed.
02241     For simplicity, this value is used also for all steps, in
02242     despite of it is a bit less than maximally available space.
02243 */
02244 WORD *thechunk = *dbuffer;
02245
02246 err = PF_PreparePack(); /* initialize buffers */
02247 err |= PF_Pack(numterms,1,PF_INT);
02248 err |= PF_Pack(newsize,1,PF_LONG); /* pack the size */
02249 /*
02250     Pack and broadcast completely filled chunks.
02251     It may happen, this loop is not entered at all:
02252 */
02253 for ( i = 0; i < n; i++ ) {
02254     err |= PF_Pack(thechunk,PF_maxDollarChunkSize,PF_WORD);
02255     err |= PF_Broadcast();
02256     thechunk += PF_maxDollarChunkSize;
02257     PF_PreparePack();
02258 }
02259 /*
02260     Pack and broadcast the rest:
02261 */
02262 if ( ( n = ( (*newsize)+1)%PF_maxDollarChunkSize ) != 0 ) {
02263     err |= PF_Pack(thechunk,n,PF_WORD);
02264     err |= PF_Broadcast();
02265 }
02266 #ifdef PF_DEBUG_BCAST_PREDOLLAR
02267     MesPrint("> Broadcast PreDollar: newsize=%d numterms=%d", (int)*newsize, *numterms);
02268 #endif
02269 }
02270 if ( MASTER != PF.me ) { /* Slave - unpack received buffer */
02271     WORD *thechunk;
02272     LONG n, therest, thesize;
02273     err |= PF_Broadcast();
02274     err |= PF_Unpack(numterms,1,PF_INT);
02275     err |= PF_Unpack(newsize,1,PF_LONG);
02276 /*
02277     Now we know the buffer size.
02278 */
02279 thesize = (*newsize)+1;
02280 /*
02281     Evaluate the number of completely filled chunks. The last step must be
02282     treated separately, so -1:
02283 */
02284 n = (thesize/PF_maxDollarChunkSize) - 1;
02285 /*
02286     Note, here n can be <0, this is ok.
02287 */
02288 therest = thesize % PF_maxDollarChunkSize;
02289 thechunk = *dbuffer =
02290     (WORD*)Malloc1( thesize * sizeof(WORD), "$-buffer slave");
02291 if ( thechunk == NULL ) return(err|4);
02292 /*
02293     Unpack completely filled chunks and receive the next portion.
02294     It may happen, this loop is not entered at all:
02295 */
02296 for ( i = 0; i < n; i++ ) {
02297     err |= PF_Unpack(thechunk,PF_maxDollarChunkSize,PF_WORD);
02298     thechunk += PF_maxDollarChunkSize;
02299     err |= PF_Broadcast();
02300 }
02301 /*
02302     Now the last completely filled chunk:
02303 */
02304 if ( n >= 0 ) {
02305     err |= PF_Unpack(thechunk,PF_maxDollarChunkSize,PF_WORD);
02306     thechunk += PF_maxDollarChunkSize;
02307     if ( therest != 0 ) err |= PF_Broadcast();
02308 }
02309 /*
02310     Unpack the rest (it is already received!):
02311 */
02312 if ( therest != 0 ) err |= PF_Unpack(thechunk,therest,PF_WORD);
02313 }

```

```

02314     return (err);
02315 }
02316
02317 /*
02318 #] PF_BroadcastPreDollar :
02319 #[ Synchronization of modified dollar variables :
02320     #[ Helper functions :
02321         #[ dollarlen :
02322 */
02323
02327 static inline LONG dollarlen(const WORD *terms)
02328 {
02329     const WORD *p = terms;
02330     while ( *p ) p += *p;
02331     return p - terms; /* Not including the null terminator. */
02332 }
02333
02334 /*
02335     #] dollarlen :
02336     #[ dollar_mod_type :
02337 */
02338
02343 static inline WORD dollar_mod_type(WORD index)
02344 {
02345     int i;
02346     for ( i = 0; i < NumModOptdollars; i++ )
02347         if ( ModOptdollars[i].number == index ) break;
02348     if ( i >= NumModOptdollars ) return -1;
02349     return ModOptdollars[i].type;
02350 }
02351
02352
02353 /*
02354     #] dollar_mod_type :
02355     #] Helper functions :
02356     #[ PF_CollectModifiedDollars :
02357 */
02358
02359 /*
02360     #[ dollar_to_be_collected :
02361 */
02362
02367 static inline int dollar_to_be_collected(WORD index)
02368 {
02369     switch ( dollar_mod_type(index) ) {
02370     case MODSUM:
02371     case MODMAX:
02372     case MODMIN:
02373         return 1;
02374     default:
02375         return 0;
02376     }
02377 }
02378
02379 /*
02380     #] dollar_to_be_collected :
02381     #[ copy_dollar :
02382 */
02383
02388 static inline void copy_dollar(WORD index, WORD type, const WORD *where, LONG size)
02389 {
02390     DOLLARS d = Dollars + index;
02391
02392     CleanDollarFactors(d);
02393
02394     if ( type != DOLZERO && where != NULL && where != &AM.dollarzero && where[0] != 0 && size > 0 ) {
02395         if ( size > d->size || size < d->size / 4 ) { /* Reallocate if not enough or too much. */
02396             if ( d->where && d->where != &AM.dollarzero )
02397                 M_free(d->where, "old content of dollar");
02398             d->where = Malloc1(sizeof(WORD) * size, "copy buffer to dollar");
02399             d->size = size;
02400         }
02401         d->type = type;
02402         WCOPY(d->where, where, size);
02403     }
02404     else {
02405         if ( d->where && d->where != &AM.dollarzero )
02406             M_free(d->where, "old content of dollar");
02407         d->type = DOLZERO;
02408         d->where = &AM.dollarzero;
02409         d->size = 0;
02410     }
02411 }
02412
02413 /*
02414     #] copy_dollar :
02415     #[ compare_two_expressions :

```

```

02416 */
02417
02422 static inline int compare_two_expressions(const WORD *e1, const WORD *e2)
02423 {
02424     GETIDENTITY
02425     /*
02426      * We consider the cases that
02427      *   (1) the expression has no term,
02428      *   (2) the expression has only one term and it is a number,
02429      *   (3) otherwise.
02430      * Assume that the expressions are sorted and all terms are normalized.
02431      * The numerators of the coefficients must never be zero.
02432      *
02433      * Note that TwoExprCompare() is not adequate for our purpose
02434      * (as of 6 Aug. 2013), e.g., TwoExprCompare({0}, {4, 1, 1, -1}, LESS)
02435      * returns TRUE.
02436      */
02437     if ( e1[0] == 0 ) {
02438         if ( e2[0] == 0 ) {
02439             return(0);
02440         }
02441         else if ( e2[e2[0]] == 0 && e2[0] == ABS(e2[e2[0] - 1]) + 1 ) {
02442             if ( e2[e2[0] - 1] > 0 )
02443                 return(-1);
02444             else
02445                 return(+1);
02446         }
02447     }
02448     else if ( e1[e1[0]] == 0 && e1[0] == ABS(e1[e1[0] - 1]) + 1 ) {
02449         if ( e2[0] == 0 ) {
02450             if ( e1[e1[0] - 1] > 0 )
02451                 return(+1);
02452             else
02453                 return(-1);
02454         }
02455         else if ( e2[e2[0]] == 0 && e2[0] == ABS(e2[e2[0] - 1]) + 1 ) {
02456             return(CompCoef((WORD *)e1, (WORD *)e2));
02457         }
02458     }
02459     /* The expressions are not so simple. Define the order by each term. */
02460     while ( e1[0] && e2[0] ) {
02461         int c = CompareTerms(BHEAD (WORD *)e1, (WORD *)e2, 1);
02462         if ( c < 0 )
02463             return(-1);
02464         else if ( c > 0 )
02465             return(+1);
02466         e1 += e1[0];
02467         e2 += e2[0];
02468     }
02469     if ( e1[0] ) return(+1);
02470     if ( e2[0] ) return(-1);
02471     return(0);
02472 }
02473
02474 /*
02475     #] compare_two_expressions :
02476     #[ Variables :
02477 */
02478
02479 typedef struct {
02480     VectorStruct(WORD) buf;
02481     LONG size;
02482     WORD type;
02483     PADPOINTER(1,0,1,0);
02484 } dollar_buf;
02485
02486 /* Buffers used to store data for each variable from each slave. */
02487 static Vector(dollar_buf, dollar_slave_bufs);
02488
02489 /*
02490     #] Variables :
02491 */
02492
02506 int PF_CollectModifiedDollars(void)
02507 {
02508     int i, j, ndollars;
02509     /*
02510      * If the current module was executed in the sequential mode,
02511      * there are no modified module on the slaves.
02512      */
02513     if ( AC.mparallelfalg != PARALLELFLAG && !AC.partodoflag ) return 0;
02514     /*
02515      * Count the number of (potentially) modified dollar variables, which we need to collect.
02516      * Here we need to collect all max/min/sum variables.
02517      */
02518     ndollars = 0;
02519     for ( i = 0; i < NumPotModdollars; i++ ) {

```

```

02520     WORD index = PotModdollars[i];
02521     if ( dollar_to_be_collected(index) ) ndollars++;
02522 }
02523 if ( ndollars == 0 ) return 0; /* No dollars to be collected. */
02524
02525 if ( PF.me == MASTER ) {
02526 /*
02527     #[] Master :
02528 */
02529     int nslaves, nvars;
02530     /* Prepare receive buffers. We need ndollars*(PF.numtasks-1) buffers. */
02531     int nbufs = ndollars * (PF.numtasks - 1);
02532     VectorReserve(dollar_slave_bufs, nbufs);
02533     for ( i = VectorSize(dollar_slave_bufs); i < nbufs; i++ ) {
02534         VectorInit(VectorPtr(dollar_slave_bufs)[i].buf);
02535     }
02536     VectorSize(dollar_slave_bufs) = nbufs;
02537     /* Receive data from each slave. */
02538     for ( nslaves = 1; nslaves < PF.numtasks; nslaves++ ) {
02539         int src;
02540         PF_LongSingleReceive(PF_ANY_SOURCE, PF_DOLLAR_MSGTAG, &src, NULL);
02541         nvars = 0;
02542         for ( i = 0; i < NumPotModdollars; i++ ) {
02543             WORD index = PotModdollars[i];
02544             dollar_buf *b;
02545             if ( !dollar_to_be_collected(index) ) continue;
02546             b = &VectorPtr(dollar_slave_bufs)[(PF.numtasks - 1) * nvars + (src - 1)];
02547             PF_LongSingleUnpack(&b->type, 1, PF_WORD);
02548             if ( b->type != DOLZERO ) {
02549                 LONG size;
02550                 WORD *where;
02551                 PF_LongSingleUnpack(&size, 1, PF_LONG);
02552                 VectorReserve(b->buf, size + 1);
02553                 where = VectorPtr(b->buf);
02554                 PF_LongSingleUnpack(where, size, PF_WORD);
02555                 where[size] = 0; /* The null terminator is needed. */
02556                 b->size = size + 1; /* Including the null terminator. */
02557                 /* Note that we don't collect factored stuff for max/min/sum variables. */
02558             }
02559             else {
02560                 VectorReserve(b->buf, 1);
02561                 VectorPtr(b->buf)[0] = 0;
02562                 b->size = 0;
02563             }
02564             nvars++;
02565         }
02566     }
02567     /*
02568     * Combine received dollars. The FORM reference manual says maximum/minimum/sum
02569     * $-variables must have a numerical value, however, this routine should work also
02570     * for non-numerical cases, although the maximum/minimum value for non-numerical
02571     * terms has ambiguity.
02572     */
02573     nvars = 0;
02574     for ( i = 0; i < NumPotModdollars; i++ ) {
02575         WORD index = PotModdollars[i];
02576         WORD dtype;
02577         DOLLARS d;
02578         dollar_buf *b;
02579         if ( !dollar_to_be_collected(index) ) continue;
02580         d = Dollars + index;
02581         b = &VectorPtr(dollar_slave_bufs)[(PF.numtasks - 1) * nvars];
02582         dtype = dollar_mod_type(index);
02583         switch ( dtype ) {
02584             case MODMAX:
02585             case MODMIN: {
02586 /*
02587             #[] MODMAX & MODMIN :
02588 */
02589                 int selected = 0;
02590                 for ( j = 1; j < PF.numtasks - 1; j++ ) {
02591                     int c = compare_two_expressions(VectorPtr(b[j].buf),
02592 VectorPtr(b[selected].buf));
02593                     if ( (dtype == MODMAX && c > 0) || (dtype == MODMIN && c < 0) )
02594                         selected = j;
02595                 }
02596                 b = b + selected;
02597                 copy_dollar(index, b->type, VectorPtr(b->buf), b->size);
02598 /*
02599             #] MODMAX & MODMIN :
02600 */
02601                 break;
02602             case MODSUM: {
02603 /*
02604             #[] MODSUM :
02605 */

```

```

02606         GETIDENTITY
02607         int err = 0;
02608
02609         CBUF *C = cbuf + AM.rbufnum;
02610         WORD *oldwork = AT.WorkPointer, *oldcterm = AN.cTerm;
02611         WORD olddefer = AR.DeferFlag, oldnumlhs = AR.Cnumlhs, oldnumrhs = C->numrhs;
02612
02613         LONG size;
02614         WORD type, *dbuf;
02615
02616         AN.cTerm = 0;
02617         AR.DeferFlag = 0;
02618
02619         if ( ((WORD *) ((UBYTE *) AT.WorkPointer + AM.MaxTer)) > AT.WorkTop ) {
02620             err = -1;
02621             goto cleanup;
02622             MesWork();
02623         }
02624
02625         if ( NewSort(BHEAD0) ) {
02626             err = -1;
02627             goto cleanup;
02628         }
02629         if ( NewSort(BHEAD0) ) {
02630             LowerSortLevel();
02631             err = -1;
02632             goto cleanup;
02633         }
02634
02635         /*
02636          * Sum up the original $-variable in the master and $-variables on all slaves.
02637          * Note that $-variables on the slaves are set to zero at the beginning of
02638          * the module (See also DoExecute()).
02639          */
02640         for ( j = 0; j < PF.numtasks; j++ ) {
02641             const WORD *r;
02642             for ( r = j == 0 ? Dollars[index].where : VectorPtr(b[j - 1].buf); *r; r += *r
02643         ) {
02644             WCOPY(AT.WorkPointer, r, *r);
02645             AT.WorkPointer += *r;
02646             AR.Cnumlhs = 0;
02647             if ( Generator(BHEAD oldwork, 0) ) {
02648                 LowerSortLevel(); LowerSortLevel();
02649                 err = -1;
02650                 goto cleanup;
02651             }
02652             AT.WorkPointer = oldwork;
02653         }
02654
02655         size = EndSort(BHEAD (WORD *)&dbuf, 2);
02656         if ( size < 0 ) {
02657             LowerSortLevel();
02658             err = -1;
02659             goto cleanup;
02660         }
02661         LowerSortLevel();
02662
02663         /* Find special cases. */
02664         type = DOLTERMS;
02665         if ( dbuf[0] == 0 ) {
02666             type = DOLZERO;
02667         }
02668         else if ( dbuf[dbuf[0]] == 0 ) {
02669             const WORD *t = dbuf, *w;
02670             WORD n, nsize;
02671             n = *t;
02672             nsize = t[n - 1];
02673             if ( nsize < 0 ) nsize = -nsize;
02674             if ( nsize == n - 1 ) {
02675                 nsize = (nsize - 1) / 2;
02676                 w = t + 1 + nsize;
02677                 if ( *w == 1 ) {
02678                     w++; while ( w < t + n - 1 ) { if ( *w ) break; w++; }
02679                     if ( w >= t + n - 1 ) type = DOLNUMBER;
02680                 }
02681                 else if ( n == 7 && t[6] == 3 && t[5] == 1 && t[4] == 1 && t[1] == INDEX
02682                     && t[2] == 3 ) {
02683                     type = DOLINDEX;
02684                     d->index = t[3];
02685                 }
02686             }
02687             copy_dollar(index, type, dbuf, dollarlen(dbuf) + 1);
02688             M_free(dbuf, "temporary dollar buffer");
02689         cleanup:
02690         AR.Cnumlhs = oldnumlhs;

```

```

02691             C->numrhs = oldnumrhs;
02692             AR.DeferFlag = olddefer;
02693             AN.cTerm = oldcterm;
02694             AT.WorkPointer = oldwork;
02695
02696             if ( err ) return err;
02697 /*
02698             #] MODSUM :
02699 */
02700             break;
02701         }
02702     }
02703     if ( d->type == DOLTERMS )
02704         cbuf[AM.dbufnum].CanCommu[index] = numcommute(d->where,
02705 &cbuf[AM.dbufnum].NumTerms[index]);
02706         cbuf[AM.dbufnum].rhs[index] = d->where;
02707         nvars++;
02708 #ifdef PF_DEBUG_REDUCE_DOLLAR
02709     MesPrint("« Reduce $-var: %s", AC.dollarnames->namebuffer + d->name);
02710 #endif
02711 }
02712 /*
02713     #] Master :
02714 */
02715 }
02716 else {
02717     /*
02718         #[ Slave :
02719 */
02720     PF_PrepareLongSinglePack();
02721     /* Pack each variable. */
02722     for ( i = 0; i < NumPotModdollars; i++ ) {
02723         WORD index = PotModdollars[i];
02724         DOLLARS d;
02725         if ( !dollar_to_be_collected(index) ) continue;
02726         d = Dollars + index;
02727         PF_LongSinglePack(&d->type, 1, PF_WORD);
02728         if ( d->type != DOLZERO ) {
02729             /*
02730              * NOTE: d->size is the allocated buffer size for d->where in WORDs.
02731              *       So dollarlen(d->where) can be < d->size-1. (TU 15 Dec 2011)
02732              */
02733             LONG size = dollarlen(d->where);
02734             PF_LongSinglePack(&size, 1, PF_LONG);
02735             PF_LongSinglePack(d->where, size, PF_WORD);
02736             /* Note that we don't collect factored stuff for max/min/sum variables. */
02737         }
02738     }
02739     PF_LongSingleSend(MASTER, PF_DOLLAR_MSGTAG);
02740 /*
02741     #] Slave :
02742 */
02743 }
02744 return 0;
02745 }
02746 /*
02747     #] PF_CollectModifiedDollars :
02748     #[ PF_BroadcastModifiedDollars :
02749 */
02750
02751 /*
02752     #[ dollar_to_be_broadcast :
02753 */
02754
02755 static inline int dollar_to_be_broadcast(WORD index)
02756 {
02757     switch ( dollar_mod_type(index) ) {
02758         case MODLOCAL:
02759             return 0;
02760         default:
02761             return 1;
02762     }
02763 }
02764
02765 /*
02766     #] dollar_to_be_broadcast :
02767 */
02768
02769 int PF_BroadcastModifiedDollars(void)
02770 {
02771     int i, j, ndollars;
02772     /*
02773      * Count the number of (potentially) modified dollar variables, which we need to broadcast.
02774      * Here we need to broadcast all non-local variables.
02775      */
02776     ndollars = 0;

```



```

02793     for ( i = 0; i < NumPotModdollars; i++ ) {
02794         WORD index = PotModdollars[i];
02795         if ( dollar_to_be_broadcast(index) ) ndollars++;
02796     }
02797     if ( ndollars == 0 ) return 0; /* No dollars to be broadcast. */
02798
02799     if ( PF.me == MASTER ) {
02800 /*
02801         #[ Master :
02802 */
02803         PF_PrepareLongMultiPack();
02804         /* Pack each variable. */
02805         for ( i = 0; i < NumPotModdollars; i++ ) {
02806             WORD index = PotModdollars[i];
02807             DOLLARS d;
02808             if ( !dollar_to_be_broadcast(index) ) continue;
02809             d = Dollars + index;
02810             PF_LongMultiPack(&d->type, 1, PF_WORD);
02811             if ( d->type != DOLZERO ) {
02812                 /*
02813                  * NOTE: d->size is the allocated buffer size for d->where in WORDs.
02814                  *       So dollarlen(d->where) can be < d->size-1. (TU 15 Dec 2011)
02815                  */
02816                 LONG size = dollarlen(d->where);
02817                 PF_LongMultiPack(&size, 1, PF_LONG);
02818                 PF_LongMultiPack(d->where, size, PF_WORD);
02819                 /* ...and the factored stuff. */
02820                 PF_LongMultiPack(&d->nfactors, 1, PF_WORD);
02821                 if ( d->nfactors > 1 ) {
02822                     for ( j = 0; j < d->nfactors; j++ ) {
02823                         FACDOLLAR *f = &d->factors[j];
02824                         PF_LongMultiPack(&f->type, 1, PF_WORD);
02825                         PF_LongMultiPack(&f->size, 1, PF_LONG);
02826                         if ( f->size > 0 )
02827                             PF_LongMultiPack(f->where, f->size, PF_WORD);
02828                         else
02829                             PF_LongMultiPack(&f->value, 1, PF_WORD);
02830                     }
02831                 }
02832             }
02833 #ifdef PF_DEBUG_BCAST_DOLLAR
02834             MesPrint("» Broadcast $-var: %s", AC.dollarnames->namebuffer + d->name);
02835 #endif
02836         }
02837 /*
02838         #] Master :
02839 */
02840     }
02841     if ( PF_LongMultiBroadcast() ) return -1;
02842     if ( PF.me != MASTER ) {
02843 /*
02844         #[ Slave :
02845 */
02846         for ( i = 0; i < NumPotModdollars; i++ ) {
02847             WORD index = PotModdollars[i];
02848             DOLLARS d;
02849             if ( !dollar_to_be_broadcast(index) ) continue;
02850             d = Dollars + index;
02851             /* Clear the contents of the dollar variable. */
02852             if ( d->where && d->where != &AM.dollarzero )
02853                 M_free(d->where, "old content of dollar");
02854             d->where = &AM.dollarzero;
02855             d->size = 0;
02856             CleanDollarFactors(d);
02857             /* Unpack and store the contents. */
02858             PF_LongMultiUnpack(&d->type, 1, PF_WORD);
02859             if ( d->type != DOLZERO ) {
02860                 LONG size;
02861                 PF_LongMultiUnpack(&size, 1, PF_LONG);
02862                 d->size = size + 1;
02863                 d->where = (WORD *)Malloca(sizeof(WORD) * d->size, "dollar content");
02864                 PF_LongMultiUnpack(d->where, size, PF_WORD);
02865                 d->where[size] = 0; /* The null terminator is needed. */
02866                 /* ...and the factored stuff. */
02867                 PF_LongMultiUnpack(&d->nfactors, 1, PF_WORD);
02868                 if ( d->nfactors > 1 ) {
02869                     d->factors = (FACDOLLAR *)Malloca(sizeof(FACDOLLAR) * d->nfactors, "dollar
factored stuff");
02870                     for ( j = 0; j < d->nfactors; j++ ) {
02871                         FACDOLLAR *f = &d->factors[j];
02872                         PF_LongMultiUnpack(&f->type, 1, PF_WORD);
02873                         PF_LongMultiUnpack(&f->size, 1, PF_LONG);
02874                         if ( f->size > 0 ) {
02875                             f->where = (WORD *)Malloca(sizeof(WORD) * (f->size + 1), "dollar factor
content");
02876                             PF_LongMultiUnpack(f->where, f->size, PF_WORD);
02877                             f->where[f->size] = 0; /* The null terminator is needed. */

```

```

02878             f->value = 0;
02879         }
02880         else {
02881             f->where = NULL;
02882             PF_LongMultiUnpack(&f->value, 1, PF_WORD);
02883         }
02884     }
02885 }
02886 }
02887 if ( d->type == DOLTERMS )
02888     cbuf[AM.dbufnum].CanCommu[index] = numcommute(d->where,
02889 &cbuf[AM.dbufnum].NumTerms[index]);
02890     cbuf[AM.dbufnum].rhs[index] = d->where;
02891 }
02892 /*
02893     #] Slave :
02894 */
02895 }
02896 return 0;
02897 }
02898 /*
02899     #] PF_BroadcastModifiedDollars :
02900     #] Synchronization of modified dollar variables :
02901     #] Synchronization of redefined preprocessor variables :
02902     #] Variables :
02903 */
02904 /* A buffer used in receivers. */
02905 static Vector(UBYTE, prevarbuf);
02906 /*
02907     #] Variables :
02908     #] PF_PackRedefinedPreVars :
02909 */
02910 static void PF_PackRedefinedPreVars(void)
02911 {
02912     int i;
02913     /* First, pack the number of redefined preprocessor variables. */
02914     int nredefs = 0;
02915     for ( i = 0; i < AC.numpfirstnum; i++ )
02916         if ( AC.inputnumbers[i] >= 0 ) nredefs++;
02917     PF_LongSinglePack(&nredefs, 1, PF_INT);
02918     /* Then, pack each variable. */
02919     for ( i = 0; i < AC.numpfirstnum; i++ )
02920         if ( AC.inputnumbers[i] >= 0 ) {
02921             WORD index = AC.pfirstnum[i];
02922             UBYTE *value = PreVar[index].value;
02923             int bytes = strlen((char *)value);
02924             PF_LongSinglePack(&index, 1, PF_WORD);
02925             PF_LongSinglePack(&bytes, 1, PF_INT);
02926             PF_LongSinglePack(value, bytes, PF_BYTE);
02927             PF_LongSinglePack(&AC.inputnumbers[i], 1, PF_LONG);
02928         }
02929 }
02930 /*
02931     #] PF_PackRedefinedPreVars :
02932     #] PF_UnpackRedefinedPreVars :
02933 */
02934 static void PF_UnpackRedefinedPreVars(void)
02935 {
02936     int i, j;
02937     /* Unpack the number of redefined preprocessor variables. */
02938     int nredefs;
02939     PF_LongSingleUnpack(&nredefs, 1, PF_INT);
02940     if ( nredefs > 0 ) {
02941         /* Then unpack each variable. */
02942         for ( i = 0; i < nredefs; i++ ) {
02943             WORD index;
02944             int bytes;
02945             UBYTE *value;
02946             LONG inputnumber;
02947             PF_LongSingleUnpack(&index, 1, PF_WORD);
02948             PF_LongSingleUnpack(&bytes, 1, PF_INT);
02949             VectorReserve(prevarbuf, bytes + 1);
02950             value = VectorPtr(prevarbuf);
02951             PF_LongSingleUnpack(value, bytes, PF_BYTE);
02952             value[bytes] = '\0'; /* The null terminator is needed. */
02953             PF_LongSingleUnpack(&inputnumber, 1, PF_LONG);
02954             /* Put this variable if it must be updated. */
02955             for ( j = 0; j < AC.numpfirstnum; j++ )
02956                 if ( AC.pfirstnum[j] == index ) break;
02957             if ( AC.inputnumbers[j] < inputnumber ) {
02958                 AC.inputnumbers[j] = inputnumber;

```

```

02981         PutPreVar(PreVar[index].name, value, NULL, 1);
02982     }
02983 }
02984 }
02985 }
02986
02987 /*
02988     #] PF_UnpackRedefinedPreVars :
02989     #[ PF_BroadcastRedefinedPreVars :
02990 */
02991
03002 int PF_BroadcastRedefinedPreVars(void)
03003 {
03004     /*
03005      * NOTE: Because the compilation is performed on the all processes
03006      * independently on AC.mparallelfalg, we always have to broadcast redefined
03007      * preprocessor variables from the master to the all slaves.
03008      */
03009     if ( PF.me == MASTER ) {
03010         /*
03011             #[ Master :
03012         */
03013         int i, nredefs;
03014         PF_PrepereLongMultiPack();
03015         /* First, pack the number of redefined preprocessor variables. */
03016         nredefs = 0;
03017         for ( i = 0; i < AC.numpfirstnum; i++ )
03018             if ( AC.inputnumbers[i] >= 0 ) nredefs++;
03019         PF_LongMultiPack(&nredefs, 1, PF_INT);
03020         /* Then, pack each variable. */
03021         for ( i = 0; i < AC.numpfirstnum; i++ )
03022             if ( AC.inputnumbers[i] >= 0 ) {
03023                 WORD index = AC.pfirstnum[i];
03024                 UBYTE *value = PreVar[index].value;
03025                 int bytes = strlen((char *)value);
03026                 PF_LongMultiPack(&index, 1, PF_WORD);
03027                 PF_LongMultiPack(&bytes, 1, PF_INT);
03028                 PF_LongMultiPack(value, bytes, PF_BYTE);
03029 #ifdef PF_DEBUG_BCAST_PREVAR
03030                 MesPrint("» Broadcast PreVar: %s = \"%s\"", PreVar[index].name, value);
03031 #endif
03032             }
03033         /*
03034             #[ Master :
03035         */
03036     }
03037     if ( PF_LongMultiBroadcast() ) return -1;
03038     if ( PF.me != MASTER ) {
03039         /*
03040             #[ Slave :
03041         */
03042         int i, nredefs;
03043         /* Unpack the number of redefined preprocessor variables. */
03044         PF_LongMultiUnpack(&nredefs, 1, PF_INT);
03045         if ( nredefs > 0 ) {
03046             /* Then unpack each variable and put it. */
03047             for ( i = 0; i < nredefs; i++ ) {
03048                 WORD index;
03049                 int bytes;
03050                 UBYTE *value;
03051                 PF_LongMultiUnpack(&index, 1, PF_WORD);
03052                 PF_LongMultiUnpack(&bytes, 1, PF_INT);
03053                 VectorReserve(prevarbuf, bytes + 1);
03054                 value = VectorPtr(prevarbuf);
03055                 PF_LongMultiUnpack(value, bytes, PF_BYTE);
03056                 value[bytes] = '\0'; /* The null terminator is needed. */
03057                 PutPreVar(PreVar[index].name, value, NULL, 1);
03058             }
03059         }
03060         /*
03061             #[ Slave :
03062         */
03063     }
03064     return 0;
03065 }
03066
03067 /*
03068     #] PF_BroadcastRedefinedPreVars :
03069     #] Synchronization of redefined preprocessor variables :
03070     #[ Preprocessor Inside instruction :
03071     #[ Variables :
03072 */
03073
03074 /* Saved values of AC.RhsExprInModuleFlag, PotModdollars and AC.pfirstnum. */
03075 static WORD oldRhsExprInModuleFlag;
03076 static Vector (WORD, oldPotModdollars);
03077 static Vector (WORD, oldpfirstnum);

```

```

03078
03079 /*
03080     #] Variables :
03081     #[ PF_StoreInsideInfo :
03082 */
03083
03084 /*
03085  * Saves the current values of AC.RhsExprInModuleFlag, PotModdollars
03086  * and AC.pfirstnum.
03087  *
03088  * Called by DoInside().
03089  *
03090  * @return 0 if OK, nonzero on error.
03091  */
03092 int PF_StoreInsideInfo(void)
03093 {
03094     int i;
03095     oldRhsExprInModuleFlag = AC.RhsExprInModuleFlag;
03096     VectorClear(oldPotModdollars);
03097     for ( i = 0; i < NumPotModdollars; i++ )
03098         VectorPushBack(oldPotModdollars, PotModdollars[i]);
03099     VectorClear(oldpfirstnum);
03100     for ( i = 0; i < AC.numpfirstnum; i++ )
03101         VectorPushBack(oldpfirstnum, AC.pfirstnum[i]);
03102     return 0;
03103 }
03104
03105 /*
03106     #] PF_StoreInsideInfo :
03107     #[ PF_RestoreInsideInfo :
03108 */
03109
03110 /*
03111  * Restores the saved values of AC.RhsExprInModuleFlag, PotModdollars
03112  * and AC.pfirstnum.
03113  *
03114  * Called by DoEndInside().
03115  *
03116  * @return 0 if OK, nonzero on error.
03117  */
03118 int PF_RestoreInsideInfo(void)
03119 {
03120     int i;
03121     AC.RhsExprInModuleFlag = oldRhsExprInModuleFlag;
03122     NumPotModdollars = VectorSize(oldPotModdollars);
03123     for ( i = 0; i < NumPotModdollars; i++ )
03124         PotModdollars[i] = VectorPtr(oldPotModdollars)[i];
03125     AC.numpfirstnum = VectorSize(oldpfirstnum);
03126     for ( i = 0; i < AC.numpfirstnum; i++ )
03127         AC.pfirstnum[i] = VectorPtr(oldpfirstnum)[i];
03128     return 0;
03129 }
03130
03131 /*
03132     #] PF_RestoreInsideInfo :
03133     #] Preprocessor Inside instruction :
03134     #[ PF_BroadcastCBuf :
03135 */
03136
03144 int PF_BroadcastCBuf(int bufnum)
03145 {
03146     CBUF *C = cbuf + bufnum;
03147     int i;
03148     LONG l;
03149     if ( PF.me == MASTER ) {
03150 /*
03151     #] Master :
03152 */
03153         PF_PrepareLongMultiPack();
03154         /* Pack CBUF struct except pointers. */
03155         PF_LongMultiPack(&C->BufferSize, 1, PF_LONG);
03156         PF_LongMultiPack(&C->numlhs, 1, PF_INT);
03157         PF_LongMultiPack(&C->numrhs, 1, PF_INT);
03158         PF_LongMultiPack(&C->maxlhs, 1, PF_INT);
03159         PF_LongMultiPack(&C->maxrhs, 1, PF_INT);
03160         PF_LongMultiPack(&C->mnumlhs, 1, PF_INT);
03161         PF_LongMultiPack(&C->mnumrhs, 1, PF_INT);
03162         PF_LongMultiPack(&C->numtree, 1, PF_INT);
03163         PF_LongMultiPack(&C->rootnum, 1, PF_INT);
03164         PF_LongMultiPack(&C->MaxTreeSize, 1, PF_INT);
03165         /* Now pointers. Pointer, lhs and rhs are packed as offsets. We don't pack Top. */
03166         l = C->Pointer - C->Buffer;
03167         PF_LongMultiPack(&l, 1, PF_LONG);
03168         PF_LongMultiPack(C->Buffer, 1, PF_WORD);
03169         for ( i = 0; i < C->numlhs + 1; i++ ) {
03170             l = C->lhs[i] - C->Buffer;
03171             PF_LongMultiPack(&l, 1, PF_LONG);

```

```

03172     }
03173     for ( i = 0; i < C->numrhs + 1; i++ ) {
03174         l = C->rhs[i] - C->Buffer;
03175         PF_LongMultiPack(&l, 1, PF_LONG);
03176     }
03177     PF_LongMultiPack(C->CanCommu, C->numrhs + 1, PF_LONG);
03178     PF_LongMultiPack(C->NumTerms, C->numrhs + 1, PF_LONG);
03179     PF_LongMultiPack(C->numdum, C->numrhs + 1, PF_WORD);
03180     PF_LongMultiPack(C->dimension, C->numrhs + 1, PF_WORD);
03181     if ( C->MaxTreeSize > 0 )
03182         PF_LongMultiPack(C->boomlijst, (C->numtree + 1) * (sizeof(COMPTREE) / sizeof(int)),
PF_INT);
03183 #ifdef PF_DEBUG_BCAST_CBUF
03184     MesPrint("» Broadcast CBuf %d", bufnum);
03185 #endif
03186 /*
03187     #] Master :
03188 */
03189 }
03190 if ( PF_LongMultiBroadcast() ) return -1;
03191 if ( PF.me != MASTER ) {
03192 /*
03193     #[ Slave :
03194 */
03195     /* First, free already allocated buffers. */
03196     finishcbuf(bufnum);
03197     /* Unpack CBUF struct except pointers. */
03198     PF_LongMultiUnpack(&C->BufferSize, 1, PF_LONG);
03199     PF_LongMultiUnpack(&C->numlhs, 1, PF_INT);
03200     PF_LongMultiUnpack(&C->numrhs, 1, PF_INT);
03201     PF_LongMultiUnpack(&C->maxlhs, 1, PF_INT);
03202     PF_LongMultiUnpack(&C->maxrhs, 1, PF_INT);
03203     PF_LongMultiUnpack(&C->mnumlhs, 1, PF_INT);
03204     PF_LongMultiUnpack(&C->mnumrhs, 1, PF_INT);
03205     PF_LongMultiUnpack(&C->numtree, 1, PF_INT);
03206     PF_LongMultiUnpack(&C->rootnum, 1, PF_INT);
03207     PF_LongMultiUnpack(&C->MaxTreeSize, 1, PF_INT);
03208     /* Allocate new buffers. */
03209     C->Buffer = (WORD *)Malloc1(C->BufferSize * sizeof(WORD), "compiler buffer");
03210     C->Top = C->Buffer + C->BufferSize;
03211     C->lhs = (WORD **)Malloc1(C->maxlhs * sizeof(WORD *), "compiler buffer");
03212     C->rhs = (WORD **)Malloc1(C->maxrhs * (sizeof(WORD *) + 2 * sizeof(LONG) + 2 * sizeof(WORD)),
"compiler buffer");
03213     C->CanCommu = (LONG *) (C->rhs + C->maxrhs);
03214     C->NumTerms = C->CanCommu + C->maxrhs;
03215     C->numdum = (WORD *) (C->NumTerms + C->maxrhs);
03216     C->dimension = C->numdum + C->maxrhs;
03217     if ( C->MaxTreeSize > 0 )
03218         C->boomlijst = (COMPTREE *)Malloc1(C->MaxTreeSize * sizeof(COMPTREE), "compiler buffer");
03219     /* Unpack buffers. */
03220     PF_LongMultiUnpack(&l, 1, PF_LONG);
03221     PF_LongMultiUnpack(C->Buffer, 1, PF_WORD);
03222     C->Pointer = C->Buffer + 1;
03223     for ( i = 0; i < C->numlhs + 1; i++ ) {
03224         PF_LongMultiUnpack(&l, 1, PF_LONG);
03225         C->lhs[i] = C->Buffer + 1;
03226     }
03227     for ( i = 0; i < C->numrhs + 1; i++ ) {
03228         PF_LongMultiUnpack(&l, 1, PF_LONG);
03229         C->rhs[i] = C->Buffer + 1;
03230     }
03231     PF_LongMultiUnpack(C->CanCommu, C->numrhs + 1, PF_LONG);
03232     PF_LongMultiUnpack(C->NumTerms, C->numrhs + 1, PF_LONG);
03233     PF_LongMultiUnpack(C->numdum, C->numrhs + 1, PF_WORD);
03234     PF_LongMultiUnpack(C->dimension, C->numrhs + 1, PF_WORD);
03235     if ( C->MaxTreeSize > 0 )
03236         PF_LongMultiUnpack(C->boomlijst, (C->numtree + 1) * (sizeof(COMPTREE) / sizeof(int)),
PF_INT);
03237 /*
03238     #] Slave :
03239 */
03240 }
03241 return 0;
03242 }
03243 /*
03244     #] PF_BroadcastCBuf :
03245     #[ PF_BroadcastExpFlags :
03246 */
03247 int PF_BroadcastExpFlags(void)
03248 {
03249     WORD i;
03250     EXPRESSIONS e;
03251     if ( PF.me == MASTER ) {
03252 /*
03253     #] Master :

```

```

03262 */
03263     PF_PrepareLongMultiPack();
03264     PF_LongMultiPack(&AR.expflags, 1, PF_WORD);
03265     for ( i = 0; i < NumExpressions; i++ ) {
03266         e = &Expressions[i];
03267         PF_LongMultiPack(&e->counter, 1, PF_WORD);
03268         PF_LongMultiPack(&e->vflags, 1, PF_WORD);
03269         PF_LongMultiPack(&e->uflags, 1, PF_WORD);
03270         PF_LongMultiPack(&e->numdummies, 1, PF_WORD);
03271         PF_LongMultiPack(&e->numfactors, 1, PF_WORD);
03272     #ifdef PF_DEBUG_BCAST_EXPRFLAGS
03273         MesPrint("> Broadcast ExprFlags: %s", AC.exprnames->namebuffer + e->name);
03274     #endif
03275     }
03276 /*
03277     #] Master :
03278 */
03279 }
03280 if ( PF_LongMultiBroadcast() ) return -1;
03281 if ( PF.me != MASTER ) {
03282 /*
03283     #[ Slave :
03284 */
03285     PF_LongMultiUnpack(&AR.expflags, 1, PF_WORD);
03286     for ( i = 0; i < NumExpressions; i++ ) {
03287         e = &Expressions[i];
03288         PF_LongMultiUnpack(&e->counter, 1, PF_WORD);
03289         PF_LongMultiUnpack(&e->vflags, 1, PF_WORD);
03290         PF_LongMultiUnpack(&e->uflags, 1, PF_WORD);
03291         PF_LongMultiUnpack(&e->numdummies, 1, PF_WORD);
03292         PF_LongMultiUnpack(&e->numfactors, 1, PF_WORD);
03293     }
03294 /*
03295     #] Slave :
03296 */
03297 }
03298 return 0;
03299 }
03300
03301 /*
03302     #] PF_BroadcastExpFlags :
03303     #[ PF_SetScratch :
03304 */
03305
03312 static void PF_SetScratch(FILEHANDLE *f, POSITION *position)
03313 {
03314     if(
03315         ( f->handle >= 0 ) && ISGEPOS(*position, f->POposition) &&
03316         ( ISGEPOSINC(*position, f->POposition, (f->POfull-f->PObuffer)*sizeof(WORD)) ==0 )
03317     ) /*position is inside the buffer! SetScratch() will do nothing.*/
03318         f->POfull=f->PObuffer; /*force SetScratch() to re-read the position from the beginning:*/
03319     SetScratch(f, position);
03320 }
03321
03322 /*
03323     #] PF_SetScratch :
03324     #[ PF_pushScratch :
03325 */
03326
03333 static int PF_pushScratch(FILEHANDLE *f)
03334 {
03335     LONG size, RetCode;
03336     if ( f->handle < 0 ){
03337         /*Create the file*/
03338         if ( ( RetCode = CreateFile(f->name) ) >= 0 ) {
03339             f->handle = (WORD)RetCode;
03340             PUTZERO(f->filesize);
03341             PUTZERO(f->POposition);
03342         }
03343         else{
03344             MesPrint("Cannot create scratch file %s", f->name);
03345             return(-1);
03346         }
03347     } /*if ( f->handle < 0 )*/
03348     size = (f->POfill-f->PObuffer)*sizeof(WORD);
03349     if( size > 0 ){
03350         SeekFile(f->handle, &(f->POposition), SEEK_SET);
03351         if ( WriteFile(f->handle, (UBYTE *) (f->PObuffer), size) != size ){
03352             MesPrint("Error while writing to disk. Disk full?");
03353             return(-1);
03354         }
03355         ADDPOS(f->filesize, size);
03356         ADDPOS(f->POposition, size);
03357         f->POfill = f->POfull=f->PObuffer;
03358     } /*if( size > 0 )*/
03359     return(0);
03360 }

```

```

03361
03362 /*
03363 #] PF_pushScratch :
03364 #[ Broadcasting RHS expressions :
03365 #] PF_WalkThroughExprMaster :
03366 Returns <=0 if the expression is ready, or dl+1;
03367 */
03368
03369 static int PF_WalkThroughExprMaster(FILEHANDLE *curfile, int dl)
03370 {
03371     LONG l=0;
03372     for(;;){
03373         if(curfile->POfull-curfile->POfill < dl){
03374             POSITION pos;
03375             SeekScratch(curfile,&pos);
03376             PF_SetScratch(curfile,&pos);
03377         }/*if(curfile->POfull-curfile->POfill < dl)*/
03378         curfile->POfill+=dl;
03379         l+=dl;
03380         if( l >= PF.exprbufsize){
03381             if( l == PF.exprbufsize){
03382                 if( *(curfile->POfill) == 0)/*expression is ready*/
03383                     return(0);
03384             }
03385             l-=PF.exprbufsize;
03386             curfile->POfill-=l;
03387             return l+1;
03388         }
03389
03390         dl=*(curfile->POfill);
03391         if(dl == 0)
03392             return l-PF.exprbufsize;
03393
03394         if(dl<0){/*compressed term*/
03395             if(curfile->POfull-curfile->POfill < 1){
03396                 POSITION pos;
03397                 SeekScratch(curfile,&pos);
03398                 PF_SetScratch(curfile,&pos);
03399             }/*if(curfile->POfull-curfile->POfill < 1)*/
03400             dl=*(curfile->POfill+1)+2;
03401         }/*if(*(curfile->POfill)<0)*/
03402     }/*for(;;)*/
03403 }
03404
03405 /*
03406 #] PF_WalkThroughExprMaster :
03407 #] PF_WalkThroughExprSlave :
03408 Returns <=0 if the expression is ready, or dl+1;
03409 */
03410
03411 static int PF_WalkThroughExprSlave(FILEHANDLE *curfile, LONG *counter, int dl)
03412 {
03413     LONG l=0;
03414     for(;;){
03415         if(curfile->POstop-curfile->POfill < dl){
03416             if(PF_pushScratch(curfile))
03417                 return(-PF.exprbufsize-1);
03418         }
03419         curfile->POfill+=dl;
03420         curfile->POfull=curfile->POfill;
03421         l+=dl;
03422         if( l >= PF.exprbufsize){
03423             if( l == PF.exprbufsize){
03424                 /*
03425                  * This access is valid because PF.exprbufsize+1 WORDs are
03426                  * broadcasted, this shortcut is not mandatory though. (TU 15 Sep 2011)
03427                  */
03428                 if( *(curfile->POfill) == 0)/*expression is ready*/
03429                     return(0);
03430             }
03431             l-=PF.exprbufsize;
03432             curfile->POfill-=l;
03433             curfile->POfull=curfile->POfill;
03434             return l+1;
03435         }
03436
03437         dl=*(curfile->POfill);
03438         if(dl == 0)
03439             return l-PF.exprbufsize;
03440         (*counter)++;
03441         if(dl<0){/*compressed term*/
03442             if(curfile->POstop-curfile->POfill < 1){
03443                 if(PF_pushScratch(curfile))
03444                     return(-PF.exprbufsize-1);
03445             }
03446             /*
03447              * This access is always valid because PF.exprbufsize+1 WORDs are

```

```

03448         * broadcasted. (TU 15 Sep 2011)
03449         */
03450         dl=*(curfile->POfill+1)+2;
03451     }/*if*(curfile->POfill)<0)*/
03452 }/*for(;;)*/
03453 }
03454
03455 /*
03456     #] PF_WalkThroughExprSlave :
03457     #[ PF_rhsBCastMaster :
03458 */
03459
03460 static int PF_rhsBCastMaster(FILEHANDLE *curfile, EXPRESSIONS e)
03461 {
03462     LONG l=1; /*PF_WalkThroughExpr returns length + 1*/
03463     SetScratch(curfile, &(e->onfile));
03464     do{
03465         /*
03466          * We need to broadcast PF.exprbufsize+1 WORDs because PF_WalkThroughExprSlave
03467          * may access to an additional 1 WORD. It is better to rewrite the routines
03468          * in such a way as to broadcast only PF.exprbufsize WORDs. (TU 15 Sep 2011)
03469          */
03470         if ( curfile->POfull - curfile->POfill < PF.exprbufsize + 1 ) {
03471             POSITION pos;
03472             SeekScratch(curfile, &pos);
03473             PF_SetScratch(curfile, &pos);
03474         }
03475         if ( PF_Bcast(curfile->POfill, (PF.exprbufsize + 1) * sizeof(WORD)) )
03476             return -1;
03477         l=PF_WalkThroughExprMaster(curfile, l-1);
03478     }while(l>0);
03479     if(l<0)/*The tail is extra, decrease POfill*/
03480         curfile->POfill-=l;
03481     return(0);
03482 }
03483
03484 /*
03485     #] PF_rhsBCastMaster :
03486     #[ PF_rhsBCastSlave :
03487 */
03488
03489 static int PF_rhsBCastSlave(FILEHANDLE *curfile, EXPRESSIONS e)
03490 {
03491     LONG l=1; /*PF_WalkThroughExpr returns length + 1*/
03492     LONG counter = 0;
03493     do{
03494         /*
03495          * We need to broadcast PF.exprbufsize+1 WORDs because PF_WalkThroughExprSlave
03496          * may access to an additional 1 WORD. It is better to rewrite the routines
03497          * in such a way as to broadcast only PF.exprbufsize WORDs. (TU 15 Sep 2011)
03498          */
03499         if ( curfile->POstop - curfile->POfill < PF.exprbufsize + 1 ) {
03500             if(PF_pushScratch(curfile))
03501                 return(-1);
03502         }
03503         if ( PF_Bcast(curfile->POfill, (PF.exprbufsize + 1) * sizeof(WORD)) )
03504             return(-1);
03505         l = PF_WalkThroughExprSlave(curfile, &counter, l - 1);
03506     }while(l>0);
03507     if(l<0)/*The tail is extra, decrease POfill*/
03508         if(l<-PF.exprbufsize)/*error due to a PF_pushScratch() failure */
03509             return(-1);
03510     curfile->POfill-=l;
03511 }
03512
03513 if ( curfile->handle >= 0 ) {
03514     if ( PF_pushScratch(curfile) ) return -1;
03515 }
03516 curfile->POfull=curfile->POfill;
03517 if ( curfile != AR.hidefile ) AR.InInBuf = curfile->POfull-curfile->PObuffer;
03518 else AR.InHiBuf = curfile->POfull-curfile->PObuffer;
03519 CHECK(counter == e->counter + 1); /* The first term is the prototype. */
03520 return(0);
03521 }
03522
03523 /*
03524     #] PF_rhsBCastSlave :
03525     #[ PF_BroadcastExpr :
03526 */
03527
03528 int PF_BroadcastExpr(EXPRESSIONS e, FILEHANDLE *file)
03529 {
03530     if ( PF.me == MASTER ) {
03531         if ( PF_rhsBCastMaster(file, e) ) return -1;
03532     #ifdef PF_DEBUG_BCAST_RHSEXP
03533         MesPrint("> Broadcast RhsExpr: %s", AC.exprnames->namebuffer + e->name);
03534     #endif
03535 }
03536

```



```

03557     else {
03558         POSITION pos;
03559         SetEndHScratch(file, &pos);
03560         e->onfile = pos;
03561         if ( PF_rhsBCastSlave(file, e) ) return -1;
03562     }
03563     return 0;
03564 }
03565
03566 /*
03567     #] PF_BroadcastExpr :
03568     #[ PF_BroadcastRHS :
03569 */
03570
03571 int PF_BroadcastRHS(void)
03572 {
03573     int i;
03574     for ( i = 0; i < NumExpressions; i++ ) {
03575         EXPRESSIONS e = &Expressions[i];
03576         if ( !(e->vflags & ISINRHS) ) continue;
03577         switch ( e->status ) {
03578             case LOCALEXPRESSION:
03579             case SKIPLEXPRESSION:
03580             case DROPLEXPRESSION:
03581             case GLOBALEXPRESSION:
03582             case SKIPGEXPRESSION:
03583             case DROPGEXPRESSION:
03584             case HIDELEXPRESSION:
03585             case HIDEGEXPRESSION:
03586             case INTOHIDELEXPRESSION:
03587             case INTOHIDEGEXPRESSION:
03588                 if ( PF_BroadcastExpr(e, AR.infile) ) return -1;
03589                 break;
03590             case HIDDENLEXPRESSION:
03591             case HIDDENGEXPRESSION:
03592             case DROPHLEXPRESSION:
03593             case DROPHGEXPRESSION:
03594             case UNHIDELEXPRESSION:
03595             case UNHIDEGEXPRESSION:
03596                 if ( PF_BroadcastExpr(e, AR.hidefile) ) return -1;
03597                 break;
03598         }
03599     }
03600     if ( PF.me != MASTER )
03601         UpdatePositions();
03602     return 0;
03603 }
03604
03605 /*
03606     #] PF_BroadcastRHS :
03607     #] Broadcasting RHS expressions :
03608     #[ InParallel mode :
03609     #[ PF_InParallelProcessor :
03610 */
03611
03612 int PF_InParallelProcessor(void)
03613 {
03614     GETIDENTITY
03615     int i, next, tag;
03616     EXPRESSIONS e;
03617     /*
03618      * Skip expressions with zero terms. All the master and slaves need to
03619      * change the "partodo" flag.
03620      */
03621     if ( PF.numtasks >= 3 ) {
03622         for ( i = 0; i < NumExpressions; i++ ) {
03623             e = Expressions + i;
03624             if ( e->partodo > 0 && e->counter == 0 ) {
03625                 e->partodo = 0;
03626             }
03627         }
03628     }
03629     if (PF.me == MASTER){
03630         if ( PF.numtasks >= 3 ) {
03631             partodoexpr = (WORD*)Malloca(sizeof(WORD)*(PF.numtasks+1), "PF_InParallelProcessor");
03632             for ( i = 0; i < NumExpressions; i++ ) {
03633                 e = Expressions+i;
03634                 if ( e->partodo <= 0 ) continue;
03635                 switch(e->status){
03636                     case LOCALEXPRESSION:
03637                     case GLOBALEXPRESSION:
03638                     case UNHIDELEXPRESSION:
03639                     case UNHIDEGEXPRESSION:
03640                     case INTOHIDELEXPRESSION:
03641                     case INTOHIDEGEXPRESSION:
03642                         tag=PF_ANY_SOURCE;
03643                         next=PF_Wait4SlaveIP(&tag);

```

```

03656         if(next<0)
03657             return(-1);
03658         if(tag == PF_DATA_MSGTAG){
03659             PF_Statistics(PF_stats,0);
03660             if(PF_Slave2MasterIP(next))
03661                 return(-1);
03662         }
03663         if(PF_Master2SlaveIP(next,e))
03664             return(-1);
03665         partodoexr[next]=i;
03666         break;
03667     default:
03668         e->partodo = 0;
03669         continue;
03670     }/*switch(e->status)*/
03671     }/*for ( i = 0; i < NumExpressions; i++ )*/
03672     /*Here some slaves are working, other are waiting on PF_Send.
03673     Wait all of them.*/
03674     /*At this point no new slaves may be launched so PF_WaitAllSlaves()
03675     does not modify partodoexr[]*/
03676     if(PF_WaitAllSlaves())
03677         return(-1);
03678
03679     if ( AC.CollectFun ) AR.DeferFlag = 0;
03680     if(partodoexr){
03681         M_free(partodoexr,"PF_InParallelProcessor");
03682         partodoexr=NULL;
03683     }/*if(partodoexr)*/
03684     }/*if ( PF.numtasks >= 3 ) */
03685     else {
03686         for ( i = 0; i < NumExpressions; i++ ) {
03687             Expressions[i].partodo = 0;
03688         }
03689     }
03690     return(0);
03691 }/*if(PF.me == MASTER)*/
03692 /*Slave:*/
03693 if(PF_Wait4MasterIP(PF_EMPTY_MSGTAG))
03694     return(-1);
03695 /*master is ready to listen to me*/
03696 do{
03697     WORD *oldwork= AT.WorkPointer;
03698     tag=PF_ReadMaster();/*reads directly to its scratch!*/
03699     if(tag<0)
03700         return(-1);
03701     if(tag == PF_DATA_MSGTAG){
03702         oldwork = AT.WorkPointer;
03703
03704         /* For redefine statements. */
03705         if ( AC.numpfirstnum > 0 ) {
03706             int j;
03707             for ( j = 0; j < AC.numpfirstnum; j++ ) {
03708                 AC.inputnumbers[j] = -1;
03709             }
03710         }
03711
03712         if(PF_DoOneExpr())/*the processor*/
03713             return(-1);
03714         if(PF_Wait4MasterIP(PF_DATA_MSGTAG))
03715             return(-1);
03716         if(PF_Slave2MasterIP(PF.me))/*both master and slave*/
03717             return(-1);
03718         AT.WorkPointer=oldwork;
03719     }/*if(tag == PF_DATA_MSGTAG)*/
03720     }while(tag!=PF_EMPTY_MSGTAG);
03721     PF.exprtodo=-1;
03722     return(0);
03723 }/*PF_InParallelProcessor*/
03724
03725 /*
03726     #] PF_InParallelProcessor :
03727     #[ PF_Wait4MasterIP :
03728 */
03729
03730 static int PF_Wait4MasterIP(int tag)
03731 {
03732     int follow = 0;
03733     LONG cpu,space = 0;
03734
03735     if(PF.log){
03736         fprintf(stderr,"%d] Starting to send to Master\n",PF.me);
03737         fflush(stderr);
03738     }
03739
03740     PF_PreparePack();
03741     cpu = TimeCPU(1);
03742     PF_Pack(&cpu,1,PF_LONG);

```

```

03743     PF_Pack(&space,1,PF_LONG);
03744     PF_Pack(&PF_linterms,1,PF_LONG);
03745     PF_Pack(&(AM.S0->GenTerms),1,PF_LONG);
03746     PF_Pack(&(AM.S0->TermsLeft),1,PF_LONG);
03747     PF_Pack(&follow,1,PF_INT );
03748
03749     if(PF.log){
03750         fprintf(stderr,"%d] Now sending with tag = %d\n",PF.me,tag);
03751         fflush(stderr);
03752     }
03753
03754     PF_Send(MASTER, tag);
03755
03756     if(PF.log){
03757         fprintf(stderr,"%d] returning from send\n",PF.me);
03758         fflush(stderr);
03759     }
03760     return(0);
03761 }
03762 /*
03763     #] PF_Wait4MasterIP :
03764     #[ PF_DoOneExpr :
03765 */
03766
03774 static int PF_DoOneExpr(void)/*the processor*/
03775 {
03776     GETIDENTITY
03777     EXPRESSIONS e;
03778     int i;
03779     WORD *term;
03780     POSITION position, outposition;
03781     FILEHANDLE *fi, *fout;
03782     LONG dd = 0;
03783     WORD oldBracketOn = AR.BracketOn;
03784     WORD *oldBrackBuf = AT.BrackBuf;
03785     WORD oldbracketindexflag = AT.bracketindexflag;
03786     e = Expressions + PF.exprtodo;
03787     i = PF.exprtodo;
03788     AR.CurExpr = i;
03789     AR.SortType = AC.SortType;
03790     AR.expchanged = 0;
03791     if ( ( e->vflags & ISFACTORIZED ) != 0 ) {
03792         AR.BracketOn = 1;
03793         AT.BrackBuf = AM.BracketFactors;
03794         AT.bracketindexflag = 1;
03795     }
03796
03797     position = AS.OldOnFile[i];
03798     if ( e->status == HIDDENLEXPRESSION || e->status == HIDDENGEXPRESSION ) {
03799         AR.GetFile = 2; fi = AR.hidefile;
03800     }
03801     else {
03802         AR.GetFile = 0; fi = AR.infile;
03803     }
03804 /*
03805     PUTZERO(fi->PPosition);
03806     if ( fi->handle >= 0 ) {
03807         fi->POfill = fi->POfull = fi->PObuffer;
03808     }
03809 */
03810     SetScratch(fi,&position);
03811     term = AT.WorkPointer;
03812     AR.CompressPointer = AR.CompressBuffer;
03813     AR.CompressPointer[0] = 0;
03814     AR.KeptInHold = 0;
03815     if ( GetTerm(BHEAD term) <= 0 ) {
03816         MesPrint("Expression %d has problems in scratchfile",i);
03817         Terminate(-1);
03818     }
03819     if ( AT.bracketindexflag > 0 ) OpenBracketIndex(i);
03820     term[3] = i;
03821     PUTZERO(outposition);
03822     fout = AR.outfile;
03823     fout->POfill = fout->POfull = fout->PObuffer;
03824     fout->PPosition = outposition;
03825     if ( fout->handle >= 0 ) {
03826         fout->POposition = outposition;
03827     }
03828 /*
03829     The next statement is needed because we need the system
03830     to believe that the expression is at position zero for
03831     the moment. In this worker, with no memory of other expressions,
03832     it is. This is needed for when a bracket index is made
03833     because there e->onfile is an offset. Afterwards, when the
03834     expression is written to its final location in the masters
03835     output e->onfile will get its real value.
03836 */

```

```

03837         PUTZERO(e->onfile);
03838         if ( PutOut(BHEAD term,&outposition,fout,0) < 0 ) return -1;
03839
03840         AR.DeferFlag = AC.ComDefer;
03841
03842         /*      AR.sLevel = AB[0]->R.sLevel;*/
03843         term = AT.WorkPointer;
03844         NewSort(BHEAD0);
03845         AR.MaxDum = AM.IndDum;
03846         AN.ninterms = 0;
03847         while ( GetTerm(BHEAD term) ) {
03848             SeekScratch(fi,&position);
03849             AN.ninterms++; dd = AN.deferskipped;
03850             if ( ( e->vflags & ISFACTORIZED ) != 0 && term[1] == HAAKJE ) {
03851                 StoreTerm(BHEAD term);
03852             }
03853             else {
03854                 if ( AC.CollectFun && *term <= (AM.MaxTer/(2*(LONG)sizeof(WORD))) ) {
03855                     if ( GetMoreTerms(term) < 0 ) {
03856                         LowerSortLevel(); return(-1);
03857                     }
03858                     SeekScratch(fi,&position);
03859                 }
03860                 AT.WorkPointer = term + *term;
03861                 AN.RepPoint = AT.RepCount + 1;
03862                 if ( AR.DeferFlag ) {
03863                     AR.CurDum = AN.IndDum = Expressions[PF.exprtodo].numdummies;
03864                 }
03865                 else {
03866                     AN.IndDum = AM.IndDum;
03867                     AR.CurDum = ReNumber(BHEAD term);
03868                 }
03869                 if ( AC.SymChangeFlag ) MarkDirty(term,DIRTYSYMFLAG);
03870                 if ( AN.ncmod ) {
03871                     if ( ( AC.modmode & ALSOFUNARGS ) != 0 ) MarkDirty(term,DIRTYFLAG);
03872                     else if ( AR.PolyFun ) PolyFunDirty(BHEAD term);
03873                 }
03874                 else if ( AC.PolyRatFunChanged ) PolyFunDirty(BHEAD term);
03875                 if ( ( AR.PolyFunType == 2 ) && ( AC.PolyRatFunChanged == 0 )
03876                     && ( e->status == LOCALEXPRESSION || e->status == GLOBALEXPRESSION ) ) {
03877                     PolyFunClean(BHEAD term);
03878                 }
03879                 if ( Generator(BHEAD term,0) ) {
03880                     LowerSortLevel(); return(-1);
03881                 }
03882                 AN.ninterms += dd;
03883             }
03884             SetScratch(fi,&position);
03885             if ( fi == AR.hidefile ) {
03886                 AR.InHiBuf = (fi->POfull-fi->PObuffer)
03887                     -DIFBASE(position,fi->POposition)/sizeof(WORD);
03888             }
03889             else {
03890                 AR.InInBuf = (fi->POfull-fi->PObuffer)
03891                     -DIFBASE(position,fi->POposition)/sizeof(WORD);
03892             }
03893         }
03894         AN.ninterms += dd;
03895         if ( EndSort(BHEAD AM.S0->sBuffer,0) < 0 ) return(-1);
03896         e->numdummies = AR.MaxDum - AM.IndDum;
03897         AR.BraceOn = oldBracketOn;
03898         AT.BrackBuf = oldBrackBuf;
03899         if ( ( e->vflags & TOBEFACTORED ) != 0 )
03900             poly_factorize_expression(e);
03901         else if ( ( ( e->vflags & TOBEUNFACTORED ) != 0 )
03902             && ( ( e->vflags & ISFACTORIZED ) != 0 ) )
03903             poly_unfactorize_expression(e);
03904         if ( AM.S0->TermsLeft ) e->vflags &= ~ISZERO;
03905         else e->vflags |= ISZERO;
03906         if ( AR.expchanged == 0 ) e->vflags |= ISUNMODIFIED;
03907         /*      if ( AM.S0->TermsLeft ) AR.expflags |= ISZERO;
03908             if ( AR.expchanged ) AR.expflags |= ISUNMODIFIED;*/
03909         AR.GetFile = 0;
03910         AT.bracketindexflag = oldbracketindexflag;
03911         fout->POfull = fout->POfill;
03912         return(0);
03913     }
03914 }
03915
03916 /*
03917     #] PF_DoOneExpr :
03918     #[ PF_Slave2MasterIP :
03919 */
03920
03921 typedef struct bufIPstruct {
03922     LONG i;
03923     struct ExprEssiOn e;

```

```

03924 } bufIPstruct_t;
03925
03926 static int PF_Slave2MasterIP(int src)/*both master and slave*/
03927 {
03928     EXPRESSIONS e;
03929     bufIPstruct_t exprData;
03930     int i,l;
03931     FILEHANDLE *fout=AR.outfile;
03932     POSITION pos;
03933     /*Here we know the length of data to send in advance:
03934     slave has the only one expression in its scratch file, and it sends
03935     this information to the master.*/
03936     if (PF.me != MASTER){/*slave*/
03937         e = Expressions + PF.exprtodo;
03938         /*Fill in the expression data:*/
03939         memcpy(&(exprData.e), e, sizeof(struct ExprEsSiOn));
03940         SeekScratch(fout,&pos);
03941         exprData.i=BASEPOSITION(pos);
03942         /*Send the metadata:*/
03943         if (PF_RawSend(MASTER,&exprData,sizeof(bufIPstruct_t),0))
03944             return(-1);
03945         i=exprData.i;
03946         SETBASEPOSITION(pos,0);
03947         do{
03948             int blen=PF.exprbufsize*sizeof(WORD);
03949             if(i<blen)
03950                 blen=i;
03951             l=PF_SendChunkIP(fout,&pos, MASTER, blen);
03952             /*Here always l == blen!*/
03953             if(l<0)
03954                 return(-1);
03955             ADDPOS(pos,l);
03956             i-=l;
03957         }while(i>0);
03958         if ( fout->handle >= 0 ) { /* Now get rid of the file */
03959             CloseFile(fout->handle);
03960             fout->handle = -1;
03961             remove(fout->name);
03962             PUTZERO(fout->POposition);
03963             PUTZERO(fout->filesize);
03964             fout->POfill = fout->POfull = fout->PObuffer;
03965         }
03966         /* Now handle redefined preprocessor variables. */
03967         if ( AC.numpfirstnum > 0 ) {
03968             PF_PrepareLongSinglePack();
03969             PF_PackRedefinedPreVars();
03970             PF_LongSingleSend(MASTER, PF_MISC_MSGTAG);
03971         }
03972         return(0);
03973     }/*if (PF.me != MASTER)*/
03974     /*Master*/
03975     /*partodoexpr[src] is the number of expression.*/
03976     e = Expressions +partodoexpr[src];
03977     /*Get metadata:*/
03978     if (PF_RawRecv(&src, &exprData,sizeof(bufIPstruct_t),&i) != sizeof(bufIPstruct_t))
03979         return(-1);
03980     /*Fill in the expression data:*/
03981     /* memcpy(e, &(exprData.e), sizeof(struct ExprEsSiOn)); */
03982     e->counter = exprData.e.counter;
03983     e->vflags = exprData.e.vflags;
03984     e->uflags = exprData.e.uflags;
03985     e->numdummies = exprData.e.numdummies;
03986     e->numfactors = exprData.e.numfactors;
03987     if ( !(e->vflags & ISZERO) ) AR.expflags |= ISZERO;
03988     if ( !(e->vflags & ISUNMODIFIED) ) AR.expflags |= ISUNMODIFIED;
03989     SeekScratch(fout,&pos);
03990     e->onfile = pos;
03991     i=exprData.i;
03992     while(i>0){
03993         int blen=PF.exprbufsize*sizeof(WORD);
03994         if(i<blen)
03995             blen=i;
03996         l=PF_RecvChunkIP(fout,src,blen);
03997         /*Here always l == blen!*/
03998         if(l<0)
03999             return(-1);
04000         i-=l;
04001     }
04002     /* Now handle redefined preprocessor variables. */
04003     if ( AC.numpfirstnum > 0 ) {
04004         PF_LongSingleReceive(src, PF_MISC_MSGTAG, NULL, NULL);
04005         PF_UnpackRedefinedPreVars();
04006     }
04007     return(0);
04008 }
04009
04010 /*

```

```

04011         #] PF_Slave2MasterIP :
04012         #[ PF_Master2SlaveIP :
04013     */
04014
04015     static int PF_Master2SlaveIP(int dest, EXPRESSIONS e)
04016     {
04017         bufIPstruct_t exprData;
04018         FILEHANDLE *fi;
04019         POSITION pos;
04020         int l;
04021         LONG ll=0, count=0;
04022         WORD *t;
04023         if(e==NULL){/*Say to the slave that no more job:*/
04024             if(PF_RawSend(dest, &exprData, sizeof(bufIPstruct_t), PF_EMPTY_MSGTAG))
04025                 return(-1);
04026             return(0);
04027         }
04028         memcpy(&(exprData.e), e, sizeof(struct ExprEsSiOn));
04029         exprData.i=e-Expressions;
04030         if ( AC.StatsFlag && AC.OldParallelStats ) {
04031             MesPrint("");
04032             MesPrint(" Sending expression %s to slave %d", EXPRNAME(exprData.i), dest);
04033         }
04034         if(PF_RawSend(dest, &exprData, sizeof(bufIPstruct_t), PF_DATA_MSGTAG))
04035             return(-1);
04036         if ( e->status == HIDDENLEXPRESSION || e->status == HIDDENGEXPRESSION )
04037             fi = AR.hidefile;
04038         else
04039             fi = AR.infile;
04040         pos=e->onfile;
04041         SetScratch(fi, &pos);
04042         do{
04043             l=PF_SendChunkIP(fi, &pos, dest, PF.exprbufsize*sizeof(WORD));
04044             if(l<0)
04045                 return(-1);
04046             t=fi->Pobuffer+ (DIFBASE(pos, fi->POposition))/sizeof(WORD);
04047             ll=PF_WalkThrough(t, ll, l/sizeof(WORD), &count);
04048             ADDPOS(pos, l);
04049         }while(ll>=2);
04050         return(0);
04051     }
04052
04053     /*
04054         #] PF_Master2SlaveIP :
04055         #[ PF_ReadMaster :
04056     */
04057
04058     static int PF_ReadMaster(void)/*reads directly to its scratch!*/
04059     {
04060         bufIPstruct_t exprData;
04061         int tag, m=MASTER;
04062         EXPRESSIONS e;
04063         FILEHANDLE *fi;
04064         POSITION pos;
04065         LONG count=0;
04066         WORD *t;
04067         LONG ll=0;
04068         int l;
04069         /*Get metadata:*/
04070         if (PF_RawRecv(&m, &exprData, sizeof(bufIPstruct_t), &tag) != sizeof(bufIPstruct_t))
04071             return(-1);
04072
04073         if(tag == PF_EMPTY_MSGTAG)/*No data, no job*/
04074             return(tag);
04075
04076         /*data expected, tag must be == PF_DATA_MSTAG!*/
04077         PF.exprtodo=exprData.i;
04078         e=Expressions + PF.exprtodo;
04079         /*Fill in the expression data:*/
04080         /* memcpy(e, &(exprData.e), sizeof(struct ExprEsSiOn)); */
04081         if ( e->status == HIDDENLEXPRESSION || e->status == HIDDENGEXPRESSION )
04082             fi = AR.hidefile;
04083         else
04084             fi = AR.infile;
04085         SetEndHScratch(fi, &pos);
04086         e->onfile=AS.OldOnFile[PF.exprtodo]=pos;
04087
04088         do{
04089             l=PF_RecvChunkIP(fi, MASTER, PF.exprbufsize*sizeof(WORD));
04090             if(l<0)
04091                 return(-1);
04092             t=fi->POfull-l/sizeof(WORD);
04093             ll=PF_WalkThrough(t, ll, l/sizeof(WORD), &count);
04094         }while(ll>=2);
04095         /*Now -ll-2 is the number of "extra" elements transferred from the master.*/
04096         fi->POfull=-ll-2;
04097         fi->POfill=fi->POfull;

```

```

04098     return(PF_DATA_MSGTAG);
04099 }
04100
04101 /*
04102     #] PF_ReadMaster :
04103     #[ PF_SendChunkIP :
04104     thesize is in bytes. Returns the number of sent bytes or <0 on error:
04105 */
04106
04107 static int PF_SendChunkIP(FILEHANDLE *curfile, POSITION *position, int to, LONG thesize)
04108 {
04109     LONG l=thesize;
04110     if(
04111         ISLESSPOS(*position,curfile->POposition) ||
04112         ISGEPOSINC(*position,curfile->POposition,
04113             ((curfile->POfull-curfile->PObuffer)*sizeof(WORD)-thesize) )
04114     ){
04115         if(curfile->handle< 0)
04116             l=(curfile->POfull-curfile->PObuffer)*sizeof(WORD) - (LONG)(position->p1);
04117         else{
04118             PF_SetScratch(curfile,position);
04119             if(
04120                 ISGEPOSINC(*position,curfile->POposition,
04121                     ((curfile->POfull-curfile->PObuffer)*sizeof(WORD)-thesize) )
04122             )
04123                 l=(curfile->POfull-curfile->PObuffer)*sizeof(WORD) - (LONG)position->p1;
04124         }
04125     }
04126     /*Now we are able to sent l bytes from the
04127     curfile->PObuffer[position-curfile->POposition]*/
04128     if(PF_RawSend(to,curfile->PObuffer+ (DIFBASE(*position,curfile->POposition))/sizeof(WORD),l,0))
04129         return(-1);
04130     return(l);
04131 }
04132
04133 /*
04134     #] PF_SendChunkIP :
04135     #[ PF_RecvChunkIP :
04136     thesize is in bytes. Returns the number of sent bytes or <0 on error:
04137 */
04138
04139 static int PF_RecvChunkIP(FILEHANDLE *curfile, int from, LONG thesize)
04140 {
04141     LONG receivedBytes;
04142
04143     if( (LONG)((curfile->POstop - curfile->POfull)*sizeof(WORD)) < thesize )
04144         if(PF_pushScratch(curfile))
04145             return(-1);
04146     /*Now there is enough space from curfile->POfill to curfile->POstop*/
04147     /*Block:*/
04148     int tag=0;
04149     receivedBytes=PF_RawRecv(&from,curfile->POfull,thesize,&tag);
04150     /*:Block*/
04151     if(receivedBytes >= 0 ){
04152         curfile->POfull+=receivedBytes/sizeof(WORD);
04153         curfile->POfill=curfile->POfull;
04154     }/*if(receivedBytes >= 0 )*/
04155     return(receivedBytes);
04156 }
04157
04158 /*
04159     #] PF_RecvChunkIP :
04160     #[ PF_WalkThrough :
04161     Returns:
04162     >= 0 -- initial offset,
04163     -1 -- the first element of t contains the length of the tail of compressed term,
04164     <= -2 -- -(d+2), where d is the number of extra transferred elements.
04165     Expects:
04166     l -- initial offset or -1,
04167     chunk -- number of transferred elements (not bytes!)
04168     *count -- incremented each time a new term is found
04169 */
04170
04171 static int PF_WalkThrough(WORD *t, LONG l, LONG chunk, LONG *count)
04172 {
04173     if(l<0) /*== -1!*/
04174         l=(*t)+1; /*the first element of t contains the length of
04175         the tail of compressed term*/
04176     else{
04177         if(l>=chunk) /*next term is out of the chunk*/
04178             return(l-chunk);
04179         t+=l;
04180         chunk-=l; /*note, l was less than chunk so chunk >0!*/
04181         l=*t;
04182     }
04183     /*Main loop:*/
04184     while(l!=0){

```

```

04185         if(l>0){/*an offset to the next term*/
04186             if(l<chunk){
04187                 t+=1;
04188                 chunk-=1;/*note, l was less than chunk so chunk >0!*/
04189                 l=*t;
04190                 (*count)++;
04191             }/*if(l<chunk)*/
04192             else
04193                 return(l-chunk);
04194         }/*if(l>0)*/
04195         else{ /* l<0 */
04196             if(chunk < 2){/*i.e., chunk == 1*/
04197                 return(-1);/*the first WORD in the next chunk is length of the tail of the compressed
term*/
04198                 l=(t+1)+2; /*+2 since
04199                     1. t points to the length field -1,
04200                     2. the size of a tail of compressed term is equal to the number of WORDs in this
tail*/
04201             }
04202         }/*while(l!=0)*/
04203         return(-1-chunk);/* -(2+(chunk-1)), chunk>0 ! */
04204     }
04205     /*
04206     #] PF_WalkThrough :
04207     #] InParallel mode :
04208     #[ PF_SendFile :
04209     */
04210     /*
04211     #define PF_SNDFILEBUFSIZE 4096
04212     int PF_SendFile(int to, FILE *fd)
04213     {
04214         size_t len=0;
04215         if(fd == NULL){
04216             if(PF_RawSend(to,&to,sizeof(int),PF_EMPTY_MSGTAG))
04217                 return(-1);
04218             return(0);
04219         }
04220         for(;;){
04221             char buf[PF_SNDFILEBUFSIZE];
04222             size_t l;
04223             l=fread(buf, 1, PF_SNDFILEBUFSIZE, fd);
04224             len+=l;
04225             if(l==PF_SNDFILEBUFSIZE){
04226                 if(PF_RawSend(to,buf,PF_SNDFILEBUFSIZE,PF_BUFFER_MSGTAG))
04227                     return(-1);
04228             }
04229             else{
04230                 if(PF_RawSend(to,buf,l,PF_ENDBUFFER_MSGTAG))
04231                     return(-1);
04232                 break;
04233             }
04234         }/*for(;;)*/
04235         return(len);
04236     }
04237     /*
04238     #] PF_SendFile :
04239     #[ PF_RecvFile :
04240     */
04241     /*
04242     int PF_RecvFile(int from, FILE *fd)
04243     {
04244         size_t len=0;
04245         int tag;
04246         do{
04247             char buf[PF_SNDFILEBUFSIZE];
04248             int l;
04249             l=PF_RawRecv(&from,buf,PF_SNDFILEBUFSIZE,&tag);
04250             if(l<0)
04251                 return(-1);
04252             if(tag == PF_EMPTY_MSGTAG)
04253                 return(-1);
04254             if( fwrite(buf,l,1,fd) !=1 )
04255                 return(-1);
04256             len+=l;
04257         }while(tag!=PF_ENDBUFFER_MSGTAG);
04258         return(len);
04259     }
04260     /*
04261     #] PF_RecvFile :
04262     #[ Synchronised output :
04263     #[ Explanations :
04264     */

```



```

04284
04285 /*
04286  * If the master and slaves output statistics or error messages to the same stream
04287  * or file (e.g., the standard output or the log file) simultaneously, then
04288  * a mixing of their outputs can occur. To avoid this, TFORM uses a lock of
04289  * ErrorMessageLock, but there is no locking functionality in the original MPI
04290  * specification. We need to synchronise the output from the master and slaves.
04291  *
04292  * The idea of the synchronised output (by, e.g., MesPrint()) implemented here is
04293  * Slaves:
04294  * 1. Save the output by WriteFile() (set to PF_WriteFileToFile())
04295  *    into some buffers between MLOCK(ErrorMessageLock) and
04296  *    MUNLOCK(ErrorMessageLock), which call PF_MLock() and PF_MUnlock(),
04297  *    respectively. The output for AM.Stdout and AC.LogHandle are saved to
04298  *    the buffers.
04299  * 2. At MUNLOCK(ErrorMessageLock), send the output in the buffer to the master,
04300  *    with PF_STDOUT_MSGTAG or PF_LOG_MSGTAG.
04301  * Master:
04302  * 1. Receive the buffered output from slaves, and write them by
04303  *    WriteFileToFile().
04304  * The main problem is how and where the master receives messages from
04305  * the slaves (PF_ReceiveErrorMessage()). For this purpose there are three
04306  * helper functions: PF_CatchErrorMessages() and PF_CatchErrorMessagesForAll()
04307  * which remove messages with PF_STDOUT_MSGTAG or PF_LOG_MSGTAG from the top
04308  * of the message queue, and PF_ProbeWithCatchingErrorMessages() which is same as
04309  * PF_Probe() except removing these messages.
04310 */
04311
04312 /*
04313     #] Explanations :
04314     #[ Variables :
04315 */
04316
04317 static int errorMessageLock = 0; /* (slaves) The lock count. See PF_MLock() and PF_MUnlock(). */
04318 static Vector(UBYTE, stdoutBuffer); /* (slaves) The buffer for AM.Stdout. */
04319 static Vector(UBYTE, logBuffer); /* (slaves) The buffer for AC.LogHandle. */
04320 #define recvBuffer logBuffer /* (master) The buffer for receiving messages. */
04321
04322 /*
04323  * If PF_ENABLE_STDOUT_BUFFERING is defined, the master performs the line buffering
04324  * (using stdoutBuffer) at PF_WriteFileToFile().
04325  */
04326 #ifndef PF_ENABLE_STDOUT_BUFFERING
04327 #ifndef UNIX
04328 #define PF_ENABLE_STDOUT_BUFFERING
04329 #endif
04330 #endif
04331
04332 /*
04333     #] Variables :
04334     #[ PF_MLock :
04335 */
04336
04337 void PF_MLock(void)
04338 {
04339     /* Only on slaves. */
04340     if ( errorMessageLock++ > 0 ) return;
04341     VectorClear(stdoutBuffer);
04342     VectorClear(logBuffer);
04343 }
04344
04345 /*
04346     #] PF_MLock :
04347     #[ PF_MUnlock :
04348 */
04349
04350 void PF_MUnlock(void)
04351 {
04352     /* Only on slaves. */
04353     if ( --errorMessageLock > 0 ) return;
04354     if ( !VectorEmpty(stdoutBuffer) ) {
04355         PF_RawSend(MASTER, VectorPtr(stdoutBuffer), VectorSize(stdoutBuffer), PF_STDOUT_MSGTAG);
04356     }
04357     if ( !VectorEmpty(logBuffer) ) {
04358         PF_RawSend(MASTER, VectorPtr(logBuffer), VectorSize(logBuffer), PF_LOG_MSGTAG);
04359     }
04360 }
04361
04362 /*
04363     #] PF_MUnlock :
04364     #[ PF_WriteFileToFile :
04365 */
04366
04367 LONG PF_WriteFileToFile(int handle, UBYTE *buffer, LONG size)
04368 {
04369     if ( PF.me != MASTER && errorMessageLock > 0 ) {
04370         if ( handle == AM.Stdout ) {

```

```

04389         VectorPushBacks(stdoutBuffer, buffer, size);
04390         return size;
04391     }
04392     else if ( handle == AC.LogHandle ) {
04393         VectorPushBacks(logBuffer, buffer, size);
04394         return size;
04395     }
04396 }
04397 #ifdef PF_ENABLE_STDOUT_BUFFERING
04398 /*
04399  * On my computer, sometimes a single linefeed "\n" sent to the standard
04400  * output is ignored on the execution of mpiexec. A typical example is:
04401  * $ cat foo.c
04402  * #include <unistd.h>
04403  * int main() {
04404  *     write(1, "    ", 4);
04405  *     write(1, "\n", 1);
04406  *     write(1, "    ", 4);
04407  *     write(1, "123\n", 4);
04408  *     return 0;
04409  * }
04410  * or even as a shell script:
04411  * $ cat foo.sh
04412  * #! bin/sh
04413  * printf "    "
04414  * printf "\n"
04415  * printf "    "
04416  * printf "123\n"
04417  * When I ran it on mpiexec
04418  * $ while :; do mpiexec -np 1 ./foo.sh; done
04419  * I observed the single linefeed (printf "\n") was sometimes ignored. Even
04420  * though this phenomenon might be specific to my environment, I added this
04421  * code because someone may encounter a similar phenomenon and feel it
04422  * frustrating. (TU 16 Jun 2011)
04423  *
04424  * Phenomenon:
04425  * A single linefeed sent to the standard output occasionally ignored
04426  * on mpiexec.
04427  *
04428  * Environment:
04429  * openSUSE 11.4 (x86_64)
04430  * kernel: 2.6.37.6-0.5-desktop
04431  * gcc: 4.5.1 20101208
04432  * mpich2-1.3.2pl configured with '--enable-shared --with-pm=smpd'
04433  *
04434  * Solution:
04435  * In Unix (in which Uwrite() calls write() system call without any buffering),
04436  * we perform the line buffering here. A single linefeed is also buffered.
04437  *
04438  * XXX:
04439  * At the end of the program the buffered output (text without LF) will not be flushed,
04440  * i.e., will not be written to the standard output. This is not problematic at a normal run.
04441  * The buffer can be explicitly flushed by PF_FlushStdOutBuffer().
04442  */
04443 if ( PF.me == MASTER && handle == AM.StdOut ) {
04444     size_t oldsize;
04445     /* Assume the newline character is LF (when UNIX is defined). */
04446     if ( (size > 0 && buffer[size - 1] != LINEFEED) || (size == 1 && buffer[0] == LINEFEED) ) {
04447         VectorPushBacks(stdoutBuffer, buffer, size);
04448         return size;
04449     }
04450     if ( (oldsize = VectorSize(stdoutBuffer)) > 0 ) {
04451         LONG ret;
04452         VectorPushBacks(stdoutBuffer, buffer, size);
04453         ret = WriteFileToFile(handle, VectorPtr(stdoutBuffer), VectorSize(stdoutBuffer));
04454         VectorClear(stdoutBuffer);
04455         if ( ret < 0 ) {
04456             return ret;
04457         }
04458         else if ( ret < (LONG)oldsize ) {
04459             return 0; /* This means the buffered output in previous calls is lost. */
04460         }
04461         else {
04462             return ret - (LONG)oldsize;
04463         }
04464     }
04465 }
04466 #endif
04467 return WriteFileToFile(handle, buffer, size);
04468 }
04469 /*
04470 #] PF_WriteFileToFile :
04471 #[ PF_FlushStdOutBuffer :
04472 */
04473 */
04474 void PF_FlushStdOutBuffer(void)

```

```

04480 {
04481 #ifdef PF_ENABLE_STDOUT_BUFFERING
04482     if ( PF.me == MASTER && VectorSize(stdoutBuffer) > 0 ) {
04483         WriteFileToFile(AM.StdOut, VectorPtr(stdoutBuffer), VectorSize(stdoutBuffer));
04484         VectorClear(stdoutBuffer);
04485     }
04486 #endif
04487 }
04488
04489 /*
04490     #] PF_FlushStdOutBuffer :
04491     #[ PF_ReceiveErrorMessage :
04492 */
04493
04502 static void PF_ReceiveErrorMessage(int src, int tag)
04503 {
04504     /* Only on the master. */
04505     int size;
04506     int ret = PF_RawProbe(&src, &tag, &size);
04507     CHECK(ret == 0);
04508     switch ( tag ) {
04509         case PF_STDOUT_MSGTAG:
04510         case PF_LOG_MSGTAG:
04511             VectorReserve(recvBuffer, size);
04512             ret = PF_RawRecv(&src, VectorPtr(recvBuffer), size, &tag);
04513             CHECK(ret == size);
04514             if ( size > 0 ) {
04515                 int handle = (tag == PF_STDOUT_MSGTAG) ? AM.StdOut : AC.LogHandle;
04516 #ifdef PF_ENABLE_STDOUT_BUFFERING
04517                 if ( handle == AM.StdOut ) PF_WriteFileToFile(handle, VectorPtr(recvBuffer), size);
04518                 else
04519 #endif
04520                 WriteFileToFile(handle, VectorPtr(recvBuffer), size);
04521             }
04522             break;
04523     }
04524 }
04525
04526 /*
04527     #] PF_ReceiveErrorMessage :
04528     #[ PF_CatchErrorMessages :
04529 */
04530
04539 static void PF_CatchErrorMessages(int *src, int *tag)
04540 {
04541     /* Only on the master. */
04542     for (;;) {
04543         int asrc = *src;
04544         int atag = *tag;
04545         int ret = PF_RawProbe(&asrc, &atag, NULL);
04546         CHECK(ret == 0);
04547         if ( atag == PF_STDOUT_MSGTAG || atag == PF_LOG_MSGTAG ) {
04548             PF_ReceiveErrorMessage(asrc, atag);
04549             continue;
04550         }
04551         *src = asrc;
04552         *tag = atag;
04553         break;
04554     }
04555 }
04556
04557 /*
04558     #] PF_CatchErrorMessages :
04559     #[ PF_CatchErrorMessagesForAll :
04560 */
04561
04566 static void PF_CatchErrorMessagesForAll(void)
04567 {
04568     /* Only on the master. */
04569     int i;
04570     for ( i = 1; i < PF.numtasks; i++ ) {
04571         int src = i;
04572         int tag = PF_ANY_MSGTAG;
04573         PF_CatchErrorMessages(&src, &tag);
04574     }
04575 }
04576
04577 /*
04578     #] PF_CatchErrorMessagesForAll :
04579     #[ PF_ProbeWithCatchingErrorMessages :
04580 */
04581
04591 static int PF_ProbeWithCatchingErrorMessages(int *src)
04592 {
04593     for (;;) {
04594         int newsrc = *src;
04595         int tag = PF_Probe(&newsrc);

```

```

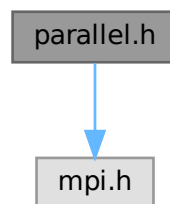
04596         if ( tag == PF_STDOUT_MSGTAG || tag == PF_LOG_MSGTAG ) {
04597             PF_ReceiveErrorMessage(newsrc, tag);
04598             continue;
04599         }
04600         if ( tag > 0 ) *src = newsrc;
04601         return tag;
04602     }
04603 }
04604
04605 /*
04606     #] PF_ProbeWithCatchingErrorMessages :
04607     #[ PF_FreeErrorMessageBuffers :
04608 */
04609
04615 void PF_FreeErrorMessageBuffers(void)
04616 {
04617     VectorFree(stdoutBuffer);
04618     VectorFree(logBuffer);
04619 }
04620
04621 /*
04622     #] PF_FreeErrorMessageBuffers :
04623     #] Synchronised output :
04624 */

```

4.96 parallel.h File Reference

```
#include <mpi.h>
```

Include dependency graph for parallel.h:



Data Structures

- struct [PF_BUFFER](#)
- struct [ParallelVars](#)

Macros

- `#define` [MASTER](#) 0
- `#define` [PF_RESET](#) 0
- `#define` [PF_TIME](#) 1
- `#define` [PF_TERM_MSGTAG](#) 10 /* master -> slave: sending terms */
- `#define` [PF_ENDSORT_MSGTAG](#) 11 /* master -> slave: no more terms to be distributed, slave -> master: after [EndSort\(\)](#) */
- `#define` [PF_DOLLAR_MSGTAG](#) 12 /* slave -> master: sending \$-variables */
- `#define` [PF_BUFFER_MSGTAG](#) 20 /* slave -> master: sending sorted terms, or in [PF_SendFile\(\)](#)/[PF_RecvFile\(\)](#) */

- `#define PF_ENDBUFFER_MSGTAG 21` /* same as PF_BUFFER_MSGTAG, but indicates the end of operation */
- `#define PF_READY_MSGTAG 30` /* slave -> master: slave is idle and can accept terms */
- `#define PF_DATA_MSGTAG 50` /* InParallel, DoCheckpoint() */
- `#define PF_EMPTY_MSGTAG 52` /* InParallel, DoCheckpoint(), PF_SendFile(), PF_RecvFile() */
- `#define PF_STDOUT_MSGTAG 60` /* slave -> master: sending text to the stdout */
- `#define PF_LOG_MSGTAG 61` /* slave -> master: sending text to the log file */
- `#define PF_OPT_MCTS_MSGTAG 70` /* master <-> slave: optimization */
- `#define PF_OPT_HORNER_MSGTAG 71` /* master <-> slave: optimization */
- `#define PF_OPT_COLLECT_MSGTAG 72` /* slave -> master: optimization */
- `#define PF_MISC_MSGTAG 100`
- `#define GNUC_PREREQ(major, minor, patchlevel) 0`
- `#define OMPI_SKIP_MPICXX 1`
- `#define indices ((INDICES))(AC.IndexList.lijst)`
- `#define PF_ANY_SOURCE MPI_ANY_SOURCE`
- `#define PF_ANY_MSGTAG MPI_ANY_TAG`
- `#define PF_COMM MPI_COMM_WORLD`
- `#define PF_BYTE MPI_BYTE`
- `#define PF_INT MPI_INT`
- `#define PF_LongMultiPack(buffer, count, type) PF_LongMultiPackImpl(buffer, count, sizeof_datatype(type), type)`
- `#define PF_LongMultiUnpack(buffer, count, type) PF_LongMultiUnpackImpl(buffer, count, sizeof_↵ datatype(type), type)`

Typedefs

- typedef struct [ParallelVars](#) **PARALLELVARS**

Functions

- int [PF_IsendSbuf](#) (int, int)
- int [PF_Bcast](#) (void *buffer, int count)
- int [PF_RawSend](#) (int, void *, LONG, int)
- LONG [PF_RawRecv](#) (int *, void *, LONG, int *)
- int [PF_PreparePack](#) (void)
- int [PF_Pack](#) (const void *buffer, size_t count, MPI_Datatype type)
- int [PF_Unpack](#) (void *buffer, size_t count, MPI_Datatype type)
- int [PF_PackString](#) (const UBYTE *str)
- int [PF_UnpackString](#) (UBYTE *str)
- int [PF_Send](#) (int to, int tag)
- int [PF_Receive](#) (int src, int tag, int *psrc, int *ptag)
- int [PF_Broadcast](#) (void)
- int [PF_PrepareLongSinglePack](#) (void)
- int [PF_LongSinglePack](#) (const void *buffer, size_t count, MPI_Datatype type)
- int [PF_LongSingleUnpack](#) (void *buffer, size_t count, MPI_Datatype type)
- int [PF_LongSingleSend](#) (int to, int tag)
- int [PF_LongSingleReceive](#) (int src, int tag, int *psrc, int *ptag)
- int [PF_PrepareLongMultiPack](#) (void)
- int [PF_LongMultiPackImpl](#) (const void *buffer, size_t count, size_t eSize, MPI_Datatype type)
- int [PF_LongMultiUnpackImpl](#) (void *buffer, size_t count, size_t eSize, MPI_Datatype type)
- int [PF_LongMultiBroadcast](#) (void)
- int [PF_EndSort](#) (void)
- WORD [PF_Deferred](#) (WORD *, WORD)

- int [PF_Processor](#) ([EXPRESSIONS](#), WORD, WORD)
- int [PF_Init](#) (int *, char ***)
- int [PF_Terminate](#) (int)
- LONG [PF_GetSlaveTimes](#) (void)
- LONG [PF_BroadcastNumber](#) (LONG)
- void [PF_BroadcastBuffer](#) (WORD **buffer, LONG *length)
- int [PF_BroadcastString](#) (UBYTE *)
- int [PF_BroadcastPreDollar](#) (WORD **, LONG *, int *)
- int [PF_CollectModifiedDollars](#) (void)
- int [PF_BroadcastModifiedDollars](#) (void)
- int [PF_BroadcastRedefinedPreVars](#) (void)
- int [PF_BroadcastCBuf](#) (int bufnum)
- int [PF_BroadcastExpFlags](#) (void)
- int [PF_StoreInsideInfo](#) (void)
- int [PF_RestoreInsideInfo](#) (void)
- int [PF_BroadcastExpr](#) ([EXPRESSIONS](#) e, [FILEHANDLE](#) *file)
- int [PF_BroadcastRHS](#) (void)
- int [PF_InParallelProcessor](#) (void)
- int [PF_SendFile](#) (int to, FILE *fd)
- int [PF_RecvFile](#) (int from, FILE *fd)
- void [PF_MLock](#) (void)
- void [PF_MUnlock](#) (void)
- LONG [PF_WriteFileToFile](#) (int, UBYTE *, LONG)
- void [PF_FlushStdOutBuffer](#) (void)

Variables

- [PARALLELVARS](#) PF
- LONG [PF_maxDollarChunkSize](#)

4.96.1 Detailed Description

Header file with things relevant to ParForm.

Definition in file [parallel.h](#).

4.96.2 Macro Definition Documentation

4.96.2.1 MASTER

```
#define MASTER 0
```

Definition at line 39 of file [parallel.h](#).

4.96.2.2 PF_RESET

```
#define PF_RESET 0
```

Definition at line 41 of file [parallel.h](#).

4.96.2.3 PF_TIME

```
#define PF_TIME 1
```

Definition at line 42 of file [parallel.h](#).

4.96.2.4 PF_TERM_MSGTAG

```
#define PF_TERM_MSGTAG 10 /* master -> slave: sending terms */
```

Definition at line 44 of file [parallel.h](#).

4.96.2.5 PF_ENDSORT_MSGTAG

```
#define PF_ENDSORT_MSGTAG 11 /* master -> slave: no more terms to be distributed, slave -> master: after EndSort() */
```

Definition at line 45 of file [parallel.h](#).

4.96.2.6 PF_DOLLAR_MSGTAG

```
#define PF_DOLLAR_MSGTAG 12 /* slave -> master: sending $-variables */
```

Definition at line 46 of file [parallel.h](#).

4.96.2.7 PF_BUFFER_MSGTAG

```
#define PF_BUFFER_MSGTAG 20 /* slave -> master: sending sorted terms, or in PF_SendFile()/PF_RecvFile() */
```

Definition at line 47 of file [parallel.h](#).

4.96.2.8 PF_ENDBUFFER_MSGTAG

```
#define PF_ENDBUFFER_MSGTAG 21 /* same as PF_BUFFER_MSGTAG, but indicates the end of operation */
```

Definition at line 48 of file [parallel.h](#).

4.96.2.9 PF_READY_MSGTAG

```
#define PF_READY_MSGTAG 30 /* slave -> master: slave is idle and can accept terms */
```

Definition at line 49 of file [parallel.h](#).

4.96.2.10 PF_DATA_MSGTAG

```
#define PF_DATA_MSGTAG 50 /* InParallel, DoCheckpoint() */
```

Definition at line 50 of file [parallel.h](#).

4.96.2.11 PF_EMPTY_MSGTAG

```
#define PF_EMPTY_MSGTAG 52 /* InParallel, DoCheckpoint(), PF_SendFile(), PF_RecvFile() */
```

Definition at line 51 of file [parallel.h](#).

4.96.2.12 PF_STDOUT_MSGTAG

```
#define PF_STDOUT_MSGTAG 60 /* slave -> master: sending text to the stdout */
```

Definition at line 52 of file [parallel.h](#).

4.96.2.13 PF_LOG_MSGTAG

```
#define PF_LOG_MSGTAG 61 /* slave -> master: sending text to the log file */
```

Definition at line 53 of file [parallel.h](#).

4.96.2.14 PF_OPT_MCTS_MSGTAG

```
#define PF_OPT_MCTS_MSGTAG 70 /* master <-> slave: optimization */
```

Definition at line 54 of file [parallel.h](#).

4.96.2.15 PF_OPT_HORNER_MSGTAG

```
#define PF_OPT_HORNER_MSGTAG 71 /* master <-> slave: optimization */
```

Definition at line 55 of file [parallel.h](#).

4.96.2.16 PF_OPT_COLLECT_MSGTAG

```
#define PF_OPT_COLLECT_MSGTAG 72 /* slave -> master: optimization */
```

Definition at line 56 of file [parallel.h](#).

4.96.2.17 PF_MISC_MSGTAG

```
#define PF_MISC_MSGTAG 100
```

Definition at line 57 of file [parallel.h](#).

4.96.2.18 GNUC_PREREQ

```
#define GNUC_PREREQ(  
    major,  
    minor,  
    patchlevel ) 0
```

Definition at line 67 of file [parallel.h](#).

4.96.2.19 OMPI_SKIP_MPICXX

```
#define OMPI_SKIP_MPICXX 1
```

Definition at line 108 of file [parallel.h](#).

4.96.2.20 indices

```
#define indices ((INDICES)(AC.IndexList.lijst))
```

Definition at line 113 of file [parallel.h](#).

4.96.2.21 PF_ANY_SOURCE

```
#define PF_ANY_SOURCE MPI_ANY_SOURCE
```

Definition at line 123 of file [parallel.h](#).

4.96.2.22 PF_ANY_MSGTAG

```
#define PF_ANY_MSGTAG MPI_ANY_TAG
```

Definition at line 124 of file [parallel.h](#).

4.96.2.23 PF_COMM

```
#define PF_COMM MPI_COMM_WORLD
```

Definition at line 125 of file [parallel.h](#).

4.96.2.24 PF_BYTE

```
#define PF_BYTE MPI_BYTE
```

Definition at line 126 of file [parallel.h](#).

4.96.2.25 PF_INT

```
#define PF_INT MPI_INT
```

Definition at line 127 of file [parallel.h](#).

4.96.2.26 PF_LongMultiPack

```
#define PF_LongMultiPack(  
    buffer,  
    count,  
    type ) PF_LongMultiPackImpl(buffer, count, sizeof_datatype(type), type)
```

Definition at line 235 of file [parallel.h](#).

4.96.2.27 PF_LongMultiUnpack

```
#define PF_LongMultiUnpack(  
    buffer,  
    count,  
    type ) PF_LongMultiUnpackImpl(buffer, count, sizeof_datatype(type), type)
```

Definition at line 236 of file [parallel.h](#).

4.96.3 Function Documentation

4.96.3.1 PF_IsendSbuf()

```
int PF_IsendSbuf (  
    int to,  
    int tag ) [extern]
```

Nonblocking send operation. It sends everything from *buff* to *fill* of the active buffer. Depending on *tag* it also can do waiting for other sends to finish or set the active buffer to the next one.

Parameters

<i>to</i>	the destination process number.
<i>tag</i>	the message tag.

Returns

0 if OK, nonzero on error.

Definition at line 261 of file [mpi.c](#).

Referenced by [FlushOut\(\)](#), [PF_Processor\(\)](#), and [PutOut\(\)](#).

4.96.3.2 PF_Bcast()

```
int PF_Bcast (
    void * buffer,
    int count ) [extern]
```

Broadcasts a message from the master to slaves.

Parameters

<i>in, out</i>	<i>buffer</i>	the starting address of buffer. The contents in this buffer on the master will be transferred to those on the slaves.
	<i>count</i>	the length of the buffer in bytes.

Returns

0 if OK, nonzero on error.

Definition at line 440 of file [mpi.c](#).

Referenced by [PF_BroadcastBuffer\(\)](#), and [PF_BroadcastNumber\(\)](#).

4.96.3.3 PF_RawSend()

```
int PF_RawSend (
    int dest,
    void * buf,
    LONG l,
    int tag ) [extern]
```

Sends *l* bytes from *buf* to *dest*. Returns 0 on success, or -1.

Parameters

<i>dest</i>	the destination process number.
<i>buf</i>	the send buffer.
<i>l</i>	the size of data in the send buffer in bytes.
<i>tag</i>	the message tag.

Returns

0 if OK, nonzero on error.

Definition at line 463 of file [mpi.c](#).

4.96.3.4 PF_RawRecv()

```
LONG PF_RawRecv (
    int * src,
```

```
void * buf,
LONG thesize,
int * tag ) [extern]
```

Receives not more than *thesize* bytes from *src*, returns the actual number of received bytes, or -1 on failure.

Parameters

in, out	<i>src</i>	the source process number. In output, that of the actual received message.
out	<i>buf</i>	the receive buffer.
	<i>thesize</i>	the size of the receive buffer in bytes.
out	<i>tag</i>	the message tag of the actual received message.

Returns

the actual sizeof received data in bytes, or -1 on failure.

Definition at line 484 of file [mpi.c](#).

4.96.3.5 PF_PreparePack()

```
int PF_PreparePack (
    void ) [extern]
```

Prepares for the next pack operations on the sender.

Returns

0 if OK, nonzero on error.

Definition at line 624 of file [mpi.c](#).

Referenced by [Optimize\(\)](#), [PF_BroadcastPreDollar\(\)](#), [PF_BroadcastString\(\)](#), and [PF_Init\(\)](#).

4.96.3.6 PF_Pack()

```
int PF_Pack (
    const void * buffer,
    size_t count,
    MPI_Datatype type ) [extern]
```

Adds data into the pack buffer.

Parameters

<i>buffer</i>	the pointer to the buffer storing the data to be packed.
<i>count</i>	the number of elements in the buffer.
<i>type</i>	the data type of elements in the buffer.

Returns

0 if OK, nonzero on error.

Definition at line 642 of file [mpi.c](#).

References [MPI_ERRCODE_CHECK](#).

Referenced by [Optimize\(\)](#), [PF_BroadcastPreDollar\(\)](#), and [PF_Init\(\)](#).

4.96.3.7 PF_Unpack()

```
int PF_Unpack (
    void * buffer,
    size_t count,
    MPI_Datatype type ) [extern]
```

Retrieves the next data in the pack buffer.

Parameters

out	<i>buffer</i>	the pointer to the buffer to store the unpacked data.
	<i>count</i>	the number of elements of data to be received.
	<i>type</i>	the data type of elements of data to be received.

Returns

0 if OK, nonzero on error.

Definition at line 671 of file [mpi.c](#).

References [MPI_ERRCODE_CHECK](#).

Referenced by [Optimize\(\)](#), [PF_BroadcastPreDollar\(\)](#), and [PF_Init\(\)](#).

4.96.3.8 PF_PackString()

```
int PF_PackString (
    const UBYTE * str ) [extern]
```

Packs a string *str* into the packed buffer [PF_packbuf](#), including the trailing zero.

The first element ([PF_INT](#)) is the length of the packed portion of the string. If the string does not fit to the buffer [PF_packbuf](#), the function packs only the initial portion. It returns the number of packed bytes, so if ([str](#)[length-1]=='\0') then the whole string fits to the buffer, if not, then the rest ([str](#)+length) must be packed and send again. On error, the function returns the negative error code.

One exception: the string "\0\0" is used as an image of the NULL, so all 3 characters will be packed.

Parameters

<i>str</i>	a string to be packed.
------------	------------------------

Returns

the number of packed bytes, or a negative value on failure.

Definition at line 706 of file [mpi.c](#).

Referenced by [PF_BroadcastString\(\)](#).

4.96.3.9 PF_UnpackString()

```
int PF_UnpackString (
    UBYTE * str ) [extern]
```

Unpacks a string to *str* from the packed buffer PF_packbuf, including the trailing zero.

It returns the number of unpacked bytes, so if (str[length-1]=='\0') then the whole string was unpacked, if not, then the rest must be appended to (str+length). On error, the function returns the negative error code.

Parameters

out	<i>str</i>	the buffer to store the unpacked string
-----	------------	---

Returns

the number of unpacked bytes, or a negative value on failure.

Definition at line 774 of file [mpi.c](#).

Referenced by [PF_BroadcastString\(\)](#).

4.96.3.10 PF_Send()

```
int PF_Send (
    int to,
    int tag ) [extern]
```

Sends the contents in the pack buffer to the process specified by *to*.

Example:

```
if ( PF.me == SRC ) {
    PF_PreparePack();
    // Packing operations here...
    PF_Send(DEST, TAG);
}
else if ( PF.me == DEST ) {
    PF_Receive(SRC, TAG, &actual_src, &actual_tag);
    // Unpacking operations here...
}
```

Parameters

<i>to</i>	the destination process number.
<i>tag</i>	the message tag.

Returns

0 if OK, nonzero on error.

Definition at line 822 of file [mpi.c](#).

References [MPI_ERRCODE_CHECK](#).

4.96.3.11 PF_Receive()

```
int PF_Receive (
    int src,
    int tag,
    int * psrc,
    int * ptag ) [extern]
```

Receives data into the pack buffer from the process specified by *src*. This function allows &*src* == *psrc* or &*tag* == *ptag*. Either *psrc* or *ptag* can be NULL.

See the example of [PF_Send\(\)](#).

Parameters

	<i>src</i>	the source process number (can be PF_ANY_SOURCE).
	<i>tag</i>	the source message tag (can be PF_ANY_TAG).
out	<i>psrc</i>	the actual source process number of received message.
out	<i>ptag</i>	the received message tag.

Returns

0 if OK, nonzero on error.

Definition at line 848 of file [mpi.c](#).

References [MPI_ERRCODE_CHECK](#).

4.96.3.12 PF_Broadcast()

```
int PF_Broadcast (
    void ) [extern]
```

Broadcasts the contents in the pack buffer on the master to those on the slaves.

Example:

```
if ( PF.me == MASTER ) {
    PF_PrepPack();
    // Packing operations here...
}
PF_Broadcast();
if ( PF.me != MASTER ) {
    // Unpacking operations here...
}
```

Returns

0 if OK, nonzero on error.

Definition at line 883 of file [mpi.c](#).

References [MPI_ERRCODE_CHECK](#).

Referenced by [Optimize\(\)](#), [PF_BroadcastPreDollar\(\)](#), [PF_BroadcastString\(\)](#), and [PF_Init\(\)](#).

4.96.3.13 PF_PrepareLongSinglePack()

```
int PF_PrepareLongSinglePack (
    void ) [extern]
```

Prepares for the next long-single-pack operations on the sender.

Returns

0 if OK, nonzero on error.

Definition at line 1451 of file [mpi.c](#).

Referenced by [PF_CollectModifiedDollars\(\)](#), and [PF_Processor\(\)](#).

4.96.3.14 PF_LongSinglePack()

```
int PF_LongSinglePack (
    const void * buffer,
    size_t count,
    MPI_Datatype type ) [extern]
```

Adds data into the "long single" pack buffer.

Parameters

<i>buffer</i>	the pointer to the buffer storing the data to be packed.
<i>count</i>	the number of elements in the buffer.
<i>type</i>	the data type of elements in the buffer.

Returns

0 if OK, nonzero on error.

Definition at line 1469 of file [mpi.c](#).

Referenced by [PF_CollectModifiedDollars\(\)](#), and [PF_Processor\(\)](#).

4.96.3.15 PF_LongSingleUnpack()

```
int PF_LongSingleUnpack (
    void * buffer,
    size_t count,
    MPI_Datatype type ) [extern]
```

Retrieves the next data in the "long single" pack buffer.

Parameters

out	<i>buffer</i>	the pointer to the buffer to store the unpacked data.
	<i>count</i>	the number of elements of data to be received.
	<i>type</i>	the data type of elements of data to be received.

Returns

0 if OK, nonzero on error.

Definition at line 1503 of file [mpi.c](#).

Referenced by [PF_CollectModifiedDollars\(\)](#), and [PF_Processor\(\)](#).

4.96.3.16 PF_LongSingleSend()

```
int PF_LongSingleSend (
    int to,
    int tag ) [extern]
```

Sends the contents in the "long single" pack buffer to the process specified by *to*.

Example:

```
if ( PF.me == SRC ) {
    PF_PrepareLongSinglePack();
    // Packing operations here...
    PF_LongSingleSend(DEST, TAG);
}
else if ( PF.me == DEST ) {
    PF_LongSingleReceive(SRC, TAG, &actual_src, &actual_tag);
    // Unpacking operations here...
}
```

Parameters

<i>to</i>	the destination process number.
<i>tag</i>	the message tag.

Returns

0 if OK, nonzero on error.

Definition at line 1540 of file [mpi.c](#).

Referenced by [PF_CollectModifiedDollars\(\)](#), and [PF_Processor\(\)](#).

4.96.3.17 PF_LongSingleReceive()

```
int PF_LongSingleReceive (
    int src,
    int tag,
    int * psrc,
    int * ptag ) [extern]
```

Receives data into the "long single" pack buffer from the process specified by *src*. This function allows *&src == psrc* or *&tag == ptag*. Either *psrc* or *ptag* can be NULL.

See the example of [PF_LongSingleSend\(\)](#).

Parameters

	<i>src</i>	the source process number (can be PF_ANY_SOURCE).
	<i>tag</i>	the source message tag (can be PF_ANY_TAG).
out	<i>psrc</i>	the actual source process number of received message.
out	<i>ptag</i>	the received message tag.

Returns

0 if OK, nonzero on error.

Definition at line 1583 of file [mpi.c](#).

Referenced by [PF_CollectModifiedDollars\(\)](#), and [PF_Processor\(\)](#).

4.96.3.18 PF_PrepareLongMultiPack()

```
int PF_PrepareLongMultiPack (
    void ) [extern]
```

Prepares for the next long-multi-pack operations on the sender.

Returns

0 if OK, nonzero on error.

Definition at line 1643 of file [mpi.c](#).

Referenced by [Optimize\(\)](#), [PF_BroadcastCBuf\(\)](#), [PF_BroadcastExpFlags\(\)](#), [PF_BroadcastModifiedDollars\(\)](#), and [PF_BroadcastRedefinedPreVars\(\)](#).

4.96.3.19 PF_LongMultiPackImpl()

```
int PF_LongMultiPackImpl (
    const void * buffer,
    size_t count,
    size_t eSize,
    MPI_Datatype type ) [extern]
```

Adds data into the "long multi" pack buffer.

Parameters

<i>buffer</i>	the pointer to the buffer storing the data to be packed.
<i>count</i>	the number of elements in the buffer.
<i>eSize</i>	the byte size of each element of data.
<i>type</i>	the data type of elements in the buffer.

Returns

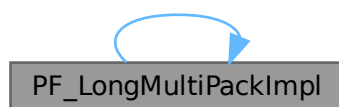
0 if OK, nonzero on error.

Definition at line 1662 of file [mpi.c](#).

References [PF_LongMultiPackImpl\(\)](#).

Referenced by [PF_LongMultiPackImpl\(\)](#).

Here is the call graph for this function:

**4.96.3.20 PF_LongMultiUnpackImpl()**

```

int PF_LongMultiUnpackImpl (
    void * buffer,
    size_t count,
    size_t eSize,
    MPI_Datatype type ) [extern]
  
```

Retrieves the next data in the "long multi" pack buffer.

Parameters

out	<i>buffer</i>	the pointer to the buffer to store the unpacked data.
	<i>count</i>	the number of elements of data to be received.
	<i>eSize</i>	the byte size of each element of data.
	<i>type</i>	the data type of elements of data to be received.

Returns

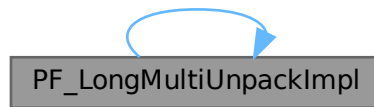
0 if OK, nonzero on error.

Definition at line 1721 of file [mpi.c](#).

References [PF_LongMultiUnpackImpl\(\)](#).

Referenced by [PF_LongMultiUnpackImpl\(\)](#).

Here is the call graph for this function:



4.96.3.21 PF_LongMultiBroadcast()

```
int PF_LongMultiBroadcast (
    void ) [extern]
```

Broadcasts the contents in the "long multi" pack buffer on the master to those on the slaves.

Example:

```
if ( PF.me == MASTER ) {
    PF_PrepareLongMultiPack();
    // Packing operations here...
}
PF_LongMultiBroadcast();
if ( PF.me != MASTER ) {
    // Unpacking operations here...
}
```

Returns

0 if OK, nonzero on error.

Definition at line 1807 of file [mpi.c](#).

Referenced by [Optimize\(\)](#), [PF_BroadcastCBuf\(\)](#), [PF_BroadcastExpFlags\(\)](#), [PF_BroadcastModifiedDollars\(\)](#), and [PF_BroadcastRedefinedPreVars\(\)](#).

4.96.3.22 PF_EndSort()

```
int PF_EndSort (
    void ) [extern]
```

Finishes a master sorting with collecting terms from slaves. Called by [EndSort\(\)](#).

If this is not the masterprocess, just initialize the sendbuffers and return 0, else [PF_EndSort\(\)](#) sends the rest of the terms in the sendbuffer to the next slave and a dummy message to all slaves with tag PF_ENDSORT_MSGTAG. Then it receives the sorted terms, sorts them using a recursive 'tree of losers' ([PF_GetLoser\(\)](#)) and writes them to the outputfile.

Returns

1 if the sorting on the master was done. 0 if [EndSort\(\)](#) still must perform a regular sorting because it is not at the ground level or not on the master or in the sequential mode or in the InParallel mode. -1 if an error occurred.

Remarks

The slaves will send the sorted terms back to the master in the regular sorting (after the initialization of the send buffer in [PF_EndSort\(\)](#)). See [PutOut\(\)](#) and [FlushOut\(\)](#).

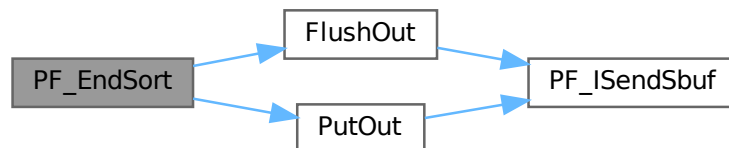
This function has been changed such that when it returns 1, AM.S0->TermsLeft is set correctly. But AM.S0->GenTerms is not set: it will be set after collecting the statistics from the slaves at the end of [PF_Processor\(\)](#). (TU 30 Jun 2011)

Definition at line 864 of file [parallel.c](#).

References [FlushOut\(\)](#), and [PutOut\(\)](#).

Referenced by [EndSort\(\)](#).

Here is the call graph for this function:

**4.96.3.23 PF_Deferred()**

```
WORD PF_Deferred (
    WORD * term,
    WORD level ) [extern]
```

Replaces [Deferred\(\)](#) on the slaves.

Parameters

<i>term</i>	the term that must be multiplied by the contents of the current bracket.
<i>level</i>	the compiler level.

Returns

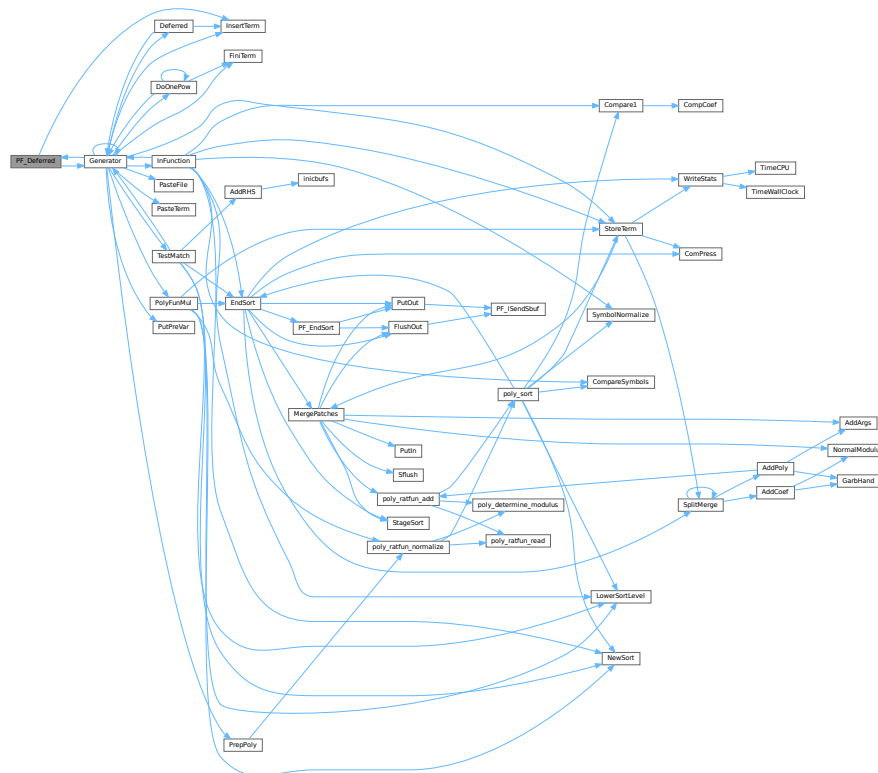
0 if OK, nonzero on error.

Definition at line 1208 of file [parallel.c](#).

References [Generator\(\)](#), and [InsertTerm\(\)](#).

Referenced by [Generator\(\)](#).

Here is the call graph for this function:



4.96.3.24 PF_Processor()

```
int PF_Processor (
    EXPRESSIONS e,
    WORD i,
    WORD LastExpression ) [extern]
```

Replaces parts of [Processor\(\)](#) on the masters and slaves. On the master [PF_Processor\(\)](#) is responsible for proper distribution of terms from the input file to the slaves. On the slaves it calls [Generator\(\)](#) for all the terms that this process gets, but [PF_GetTerm\(\)](#) gets terms from the master (not directly from infile).

Parameters

<i>e</i>	The pointer to the current expression.
<i>i</i>	The index for the current expression.
<i>LastExpression</i>	The flag indicating whether it is the last expression.

Returns

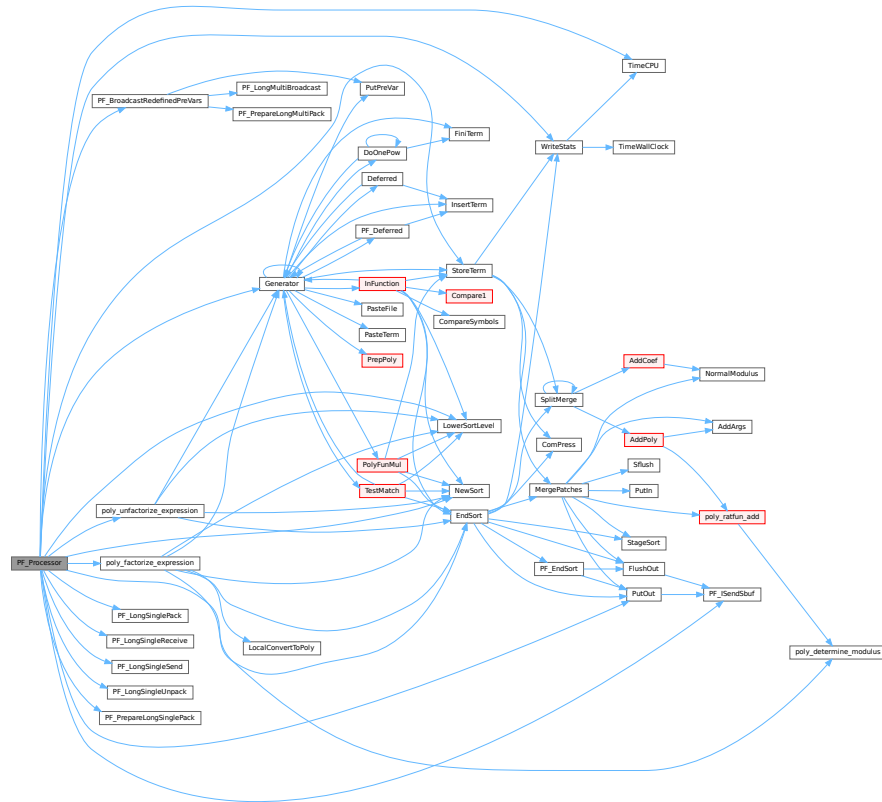
0 if OK, nonzero on error.

Definition at line 1540 of file [parallel.c](#).

References [EndSort\(\)](#), [Generator\(\)](#), [LowerSortLevel\(\)](#), [NewSort\(\)](#), [PACK_LONG](#), [PF_BroadcastRedefinePreVars\(\)](#), [PF_ISendSbuf\(\)](#), [PF_LongSinglePack\(\)](#), [PF_LongSingleReceive\(\)](#), [PF_LongSingleSend\(\)](#), [PF_LongSingleUnpack\(\)](#), [PF_PrepareLongSinglePack\(\)](#), [poly_factorize_expression\(\)](#), [poly_unfactorize_expression\(\)](#), [PutOut\(\)](#), [StoreTerm\(\)](#), [SWAP](#), [TimeCPU\(\)](#), and [WriteStats\(\)](#).

Referenced by [Processor\(\)](#).

Here is the call graph for this function:



4.96.3.25 PF_Init()

```
int PF_Init (
    int * argc,
    char *** argv ) [extern]
```

All the library independent stuff. [PF_LibInit\(\)](#) should do all library dependent initializations.

Parameters

<i>argc</i>	pointer to the number of arguments.
<i>argv</i>	pointer to the arguments.

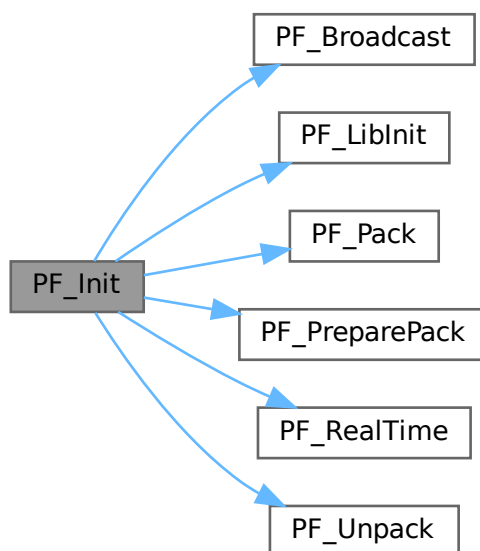
Returns

0 if OK, nonzero on error.

Definition at line 1963 of file [parallel.c](#).

References [PF_Broadcast\(\)](#), [PF_LibInit\(\)](#), [PF_Pack\(\)](#), [PF_PreparePack\(\)](#), [PF_RealTime\(\)](#), and [PF_Unpack\(\)](#).

Here is the call graph for this function:



4.96.3.26 PF_Terminate()

```
int PF_Terminate (
    int errorcode ) [extern]
```

Performs the finalization of ParFORM. To be called by `Terminate()`.

Parameters

<i>error</i>	an error code.
--------------	----------------

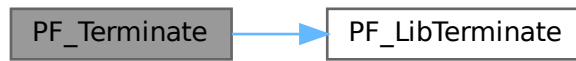
Returns

0 if OK, nonzero on error.

Definition at line 2058 of file [parallel.c](#).

References [PF_LibTerminate\(\)](#).

Here is the call graph for this function:



4.96.3.27 PF_GetSlaveTimes()

```
LONG PF_GetSlaveTimes (
    void ) [extern]
```

Returns the total CPU time of all slaves together. This function must be called on the master and all slaves.

Returns

on the master, the sum of CPU times on all slaves.

Definition at line 2074 of file [parallel.c](#).

References [TimeCPU\(\)](#).

Here is the call graph for this function:



4.96.3.28 PF_BroadcastNumber()

```
LONG PF_BroadcastNumber (
    LONG x ) [extern]
```

Broadcasts a LONG value from the master to the all slaves.

Parameters

x	the number to be broadcast (set on the master).
---	---

Returns

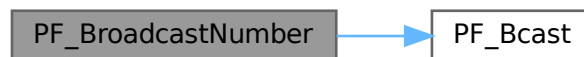
the synchronised result.

Definition at line 2094 of file [parallel.c](#).

References [PF_Bcast\(\)](#).

Referenced by [DoCheckpoint\(\)](#), [PF_BroadcastBuffer\(\)](#), and [StartVariables\(\)](#).

Here is the call graph for this function:

**4.96.3.29 PF_BroadcastBuffer()**

```

void PF_BroadcastBuffer (
    WORD ** buffer,
    LONG * length ) [extern]
  
```

Broadcasts a buffer from the master to all the slaves.

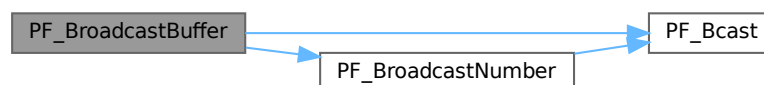
Parameters

in, out	<i>buffer</i>	on the master, the buffer to be broadcast. On the slaves, the buffer will be allocated if the length is greater than 0. The caller must free it.
in, out	<i>length</i>	on the master, the length of the buffer to be broadcast. On the slaves, it receives the length of transferred buffer. The actual transfer occurs only if the length is greater than 0.

Definition at line 2121 of file [parallel.c](#).

References [PF_Bcast\(\)](#), and [PF_BroadcastNumber\(\)](#).

Here is the call graph for this function:



4.96.3.30 PF_BroadcastString()

```
int PF_BroadcastString (
    UBYTE * str ) [extern]
```

Broadcasts a string from the master to all slaves.

Parameters

<i>in, out</i>	<i>str</i>	The pointer to a null-terminated string.
----------------	------------	--

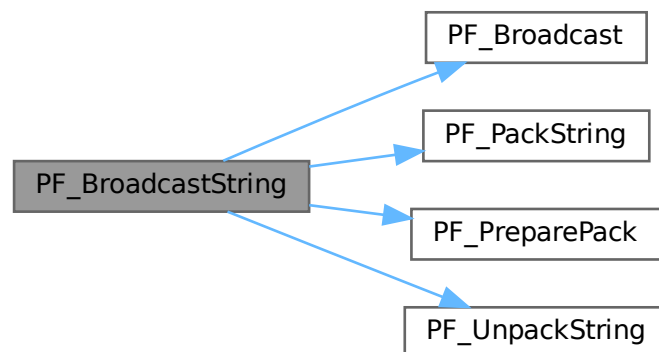
Returns

0 if OK, nonzero on error.

Definition at line 2163 of file [parallel.c](#).

References [PF_Broadcast\(\)](#), [PF_PackString\(\)](#), [PF_PreparePack\(\)](#), and [PF_UnpackString\(\)](#).

Here is the call graph for this function:



4.96.3.31 PF_BroadcastPreDollar()

```
int PF_BroadcastPreDollar (
    WORD ** dbuffer,
    LONG * newsize,
    int * numterms ) [extern]
```

Broadcasts dollar variables set as a preprocessor variables. Only the master is able to make an assignment like `#$a=g;` where `g` is an expression: only the master has an access to the expression. So, the master broadcasts the result to slaves.

The result is in `*dbuffer` of the size is `*newsize` (in number of WORDs), +1 for trailing zero. For slave `newsize` and `numterms` are output parameters.

Parameters

<i>in, out</i>	<i>dbuffer</i>	the buffer for a dollar variable.
<i>in, out</i>	<i>newsize</i>	the size of the dollar variable in WORDs.
<i>in, out</i>	<i>numterms</i>	the number of terms in the dollar variable.

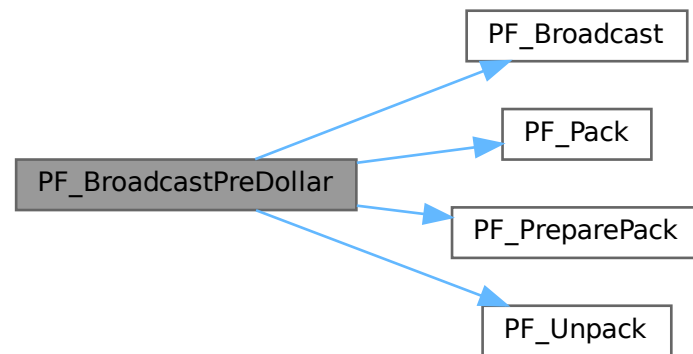
Returns

0 if OK, nonzero on error.

Definition at line 2218 of file [parallel.c](#).

References [PF_Broadcast\(\)](#), [PF_Pack\(\)](#), [PF_PreparePack\(\)](#), and [PF_Unpack\(\)](#).

Here is the call graph for this function:

**4.96.3.32 PF_CollectModifiedDollars()**

```
int PF_CollectModifiedDollars (
    void ) [extern]
```

Combines modified dollar variables on the all slaves, and store them into those on the master.

The potentially modified dollar variables are given in `PotModdollars`, and the number of them is given by `NumPotModdollars`.

The current module could be executed in parallel only if all potentially modified variables are listed in `ModOptdollars`, otherwise the module was switched to the sequential mode.

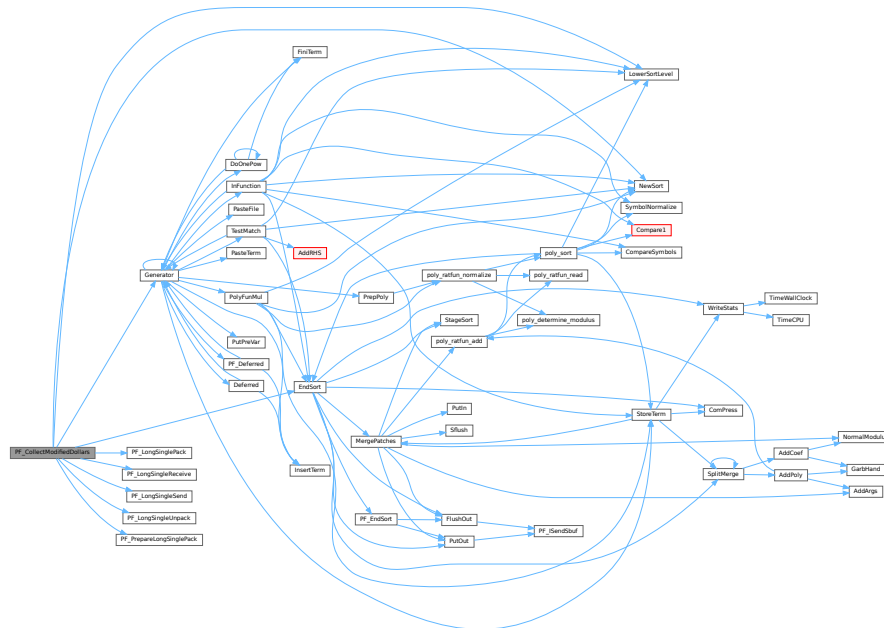
Returns

0 if OK, nonzero on error.

Definition at line 2506 of file parallel.c.

References [EndSort\(\)](#), [Generator\(\)](#), [LowerSortLevel\(\)](#), [NewSort\(\)](#), [PF_LongSinglePack\(\)](#), [PF_LongSingleReceive\(\)](#), [PF_LongSingleSend\(\)](#), [PF_LongSingleUnpack\(\)](#), [PF_PrepareLongSinglePack\(\)](#), [VectorInit](#), [VectorPtr](#), [VectorReserve](#), and [VectorSize](#).

Here is the call graph for this function:



4.96.3.33 PF_BroadcastModifiedDollars()

```
int PF_BroadcastModifiedDollars (
    void ) [extern]
```

Broadcasts modified dollar variables on the master to the all slaves.

The potentially modified dollar variables are given in PotModdollars, and the number of them is given by NumPotModdollars.

The current module could be executed in parallel only if all potentially modified variables are listed in `ModOptdollars`, otherwise the module was switched to the sequential mode. In either cases, we need to broadcast them.

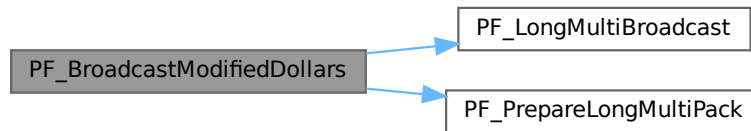
Returns

0 if OK, nonzero on error.

Definition at line 2785 of file [parallel.c](#).

References [PF_LongMultiBroadcast\(\)](#), and [PF_PrepareLongMultiPack\(\)](#).

Here is the call graph for this function:

**4.96.3.34 PF_BroadcastRedefinedPreVars()**

```
int PF_BroadcastRedefinedPreVars (
    void ) [extern]
```

Broadcasts preprocessor variables, which were changed by the Redefine statements in the current module, from the master to the all slaves.

The potentially redefined preprocessor variables are given in `AC.pfirstnum`, and the number of them is given by `AC.numpfirstnum`. For an actually redefined variable, the corresponding value in `AC.inputnumbers` is non-negative.

Returns

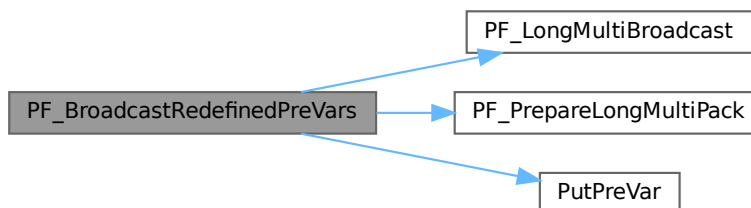
0 if OK, nonzero on error.

Definition at line 3002 of file [parallel.c](#).

References [PF_LongMultiBroadcast\(\)](#), [PF_PrepareLongMultiPack\(\)](#), [PutPreVar\(\)](#), [VectorPtr](#), and [VectorReserve](#).

Referenced by [PF_Processor\(\)](#).

Here is the call graph for this function:



4.96.3.35 PF_BroadcastCBuf()

```
int PF_BroadcastCBuf (
    int bufnum ) [extern]
```

Broadcasts a compiler buffer specified by *bufnum* from the master to the all slaves.

Parameters

<i>bufnum</i>	The index of the compiler buffer to be broadcast.
---------------	---

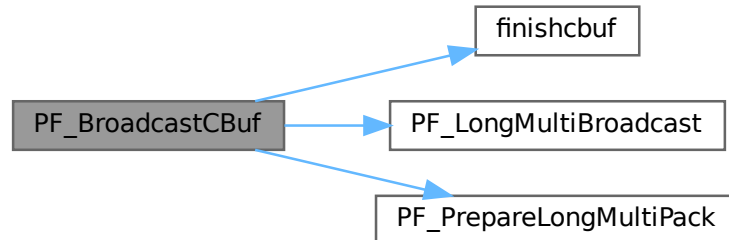
Returns

0 if OK, nonzero on error.

Definition at line 3144 of file [parallel.c](#).

References [CbUf::boomlijst](#), [CbUf::Buffer](#), [CbUf::BufferSize](#), [CbUf::CanCommu](#), [CbUf::dimension](#), [finishcbuf\(\)](#), [CbUf::lhs](#), [CbUf::numdum](#), [CbUf::NumTerms](#), [PF_LongMultiBroadcast\(\)](#), [PF_PrepareLongMultiPack\(\)](#), [CbUf::Pointer](#), [CbUf::rhs](#), and [CbUf::Top](#).

Here is the call graph for this function:



4.96.3.36 PF_BroadcastExpFlags()

```
int PF_BroadcastExpFlags (
    void ) [extern]
```

Broadcasts `AR.expflags` and several properties of each expression, e.g., `e->vflags`, from the master to all slaves.

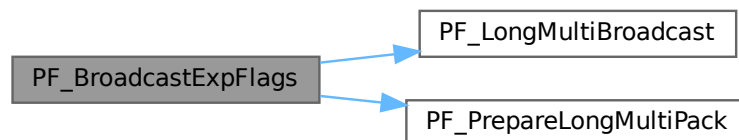
Returns

0 if OK, nonzero on error.

Definition at line 3255 of file [parallel.c](#).

References [PF_LongMultiBroadcast\(\)](#), and [PF_PrepareLongMultiPack\(\)](#).

Here is the call graph for this function:

**4.96.3.37 PF_StoreInsideInfo()**

```
int PF_StoreInsideInfo (
    void ) [extern]
```

Definition at line 3092 of file [parallel.c](#).

4.96.3.38 PF_RestoreInsideInfo()

```
int PF_RestoreInsideInfo (
    void ) [extern]
```

Definition at line 3118 of file [parallel.c](#).

4.96.3.39 PF_BroadcastExpr()

```
int PF_BroadcastExpr (
    EXPRESSIONS e,
    FILEHANDLE * file ) [extern]
```

Broadcasts an expression from the master to the all slaves.

Parameters

<i>e</i>	The expression to be broadcast.
<i>file</i>	The file in which the expression is sitting.

Returns

0 if OK, nonzero on error.

Definition at line 3549 of file [parallel.c](#).

Referenced by [PF_BroadcastRHS\(\)](#).

4.96.3.40 PF_BroadcastRHS()

```
int PF_BroadcastRHS (  
    void ) [extern]
```

Broadcasts expressions appearing in the right-hand side from the master to the all slaves.

Returns

0 if OK, nonzero on error.

Definition at line 3577 of file [parallel.c](#).

References [PF_BroadcastExpr\(\)](#).

Referenced by [Processor\(\)](#).

Here is the call graph for this function:

**4.96.3.41 PF_InParallelProcessor()**

```
int PF_InParallelProcessor (  
    void ) [extern]
```

Processes expressions in the InParallel mode, i.e., dividing expressions marked by `partodo` over the slaves.

Returns

0 if OK, nonzero on error.

Definition at line 3624 of file [parallel.c](#).

Referenced by [Processor\(\)](#).

4.96.3.42 PF_SendFile()

```
int PF_SendFile (  
    int to,  
    FILE * fd ) [extern]
```

Sends a file to the process specified by `to`.

Parameters

<i>to</i>	the destination process number.
<i>fd</i>	the file to be sent.

Returns

the size of sent data in bytes, or -1 on error.

Definition at line 4221 of file [parallel.c](#).

References [PF_RawSend\(\)](#).

Referenced by [DoCheckpoint\(\)](#).

Here is the call graph for this function:

**4.96.3.43 PF_RecvFile()**

```
int PF_RecvFile (
    int from,
    FILE * fd ) [extern]
```

Receives a file from the process specified by *from*.

Parameters

<i>from</i>	the source process number.
<i>fd</i>	the file to save the received data.

Returns

the size of received data in bytes, or -1 on error.

Definition at line 4259 of file [parallel.c](#).

References [PF_RawRecv\(\)](#).

Referenced by [DoCheckpoint\(\)](#).

Here is the call graph for this function:



4.96.3.44 PF_MLock()

```
void PF_MLock (
    void ) [extern]
```

A function called by MLOCK(ErrorMessageLock) for slaves.

Definition at line 4340 of file [parallel.c](#).

References [VectorClear](#).

4.96.3.45 PF_MUnlock()

```
void PF_MUnlock (
    void ) [extern]
```

A function called by MUNLOCK(ErrorMessageLock) for slaves.

Definition at line 4356 of file [parallel.c](#).

References [PF_RawSend\(\)](#), [VectorEmpty](#), [VectorPtr](#), and [VectorSize](#).

Here is the call graph for this function:



4.96.3.46 PF_WriteFileToFile()

```
LONG PF_WriteFileToFile (
    int handle,
    UBYTE * buffer,
    LONG size ) [extern]
```

Replaces WriteFileToFile() on the master and slaves.

It copies the given buffer into internal buffers if called between MLOCK(ErrorMessageLock) and MUNLOCK(ErrorMessageLock) for slaves and handle is StdOut or LogHandle, otherwise calls WriteFileToFile().

Parameters

<i>handle</i>	a file handle that specifies the output.
<i>buffer</i>	a pointer to the source buffer containing the data to be written.
<i>size</i>	the size of data to be written in bytes.

Returns

the actual size of data written to the output in bytes.

Definition at line 4385 of file [parallel.c](#).

References [VectorClear](#), [VectorPtr](#), [VectorPushBacks](#), and [VectorSize](#).

4.96.3.47 PF_FlushStdOutBuffer()

```
void PF_FlushStdOutBuffer (
    void ) [extern]
```

Explicitly Flushes the buffer for the standard output on the master, which is used if PF_ENABLE_STDOUT_↵
BUFFERING is defined.

Definition at line 4479 of file [parallel.c](#).

References [VectorClear](#), [VectorPtr](#), and [VectorSize](#).

4.96.4 Variable Documentation**4.96.4.1 PF**

```
PARALLELVARS PF [extern]
```

Definition at line 90 of file [parallel.c](#).

4.96.4.2 PF_maxDollarChunkSize

```
LONG PF_maxDollarChunkSize [extern]
```

Definition at line 69 of file [mpi.c](#).

4.97 parallel.h

[Go to the documentation of this file.](#)

```

00001 #ifndef __PARALLEL__
00002 #define __PARALLEL__
00003
00009 /* #[ License : */
00010 /*
00011  * Copyright (C) 1984-2023 J.A.M. Vermaseren
00012  * When using this file you are requested to refer to the publication
00013  * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00014  * This is considered a matter of courtesy as the development was paid
00015  * for by FOM the Dutch physics granting agency and we would like to
00016  * be able to track its scientific use to convince FOM of its value
00017  * for the community.
00018  *
00019  * This file is part of FORM.
00020  *
00021  * FORM is free software: you can redistribute it and/or modify it under the
00022  * terms of the GNU General Public License as published by the Free Software
00023  * Foundation, either version 3 of the License, or (at your option) any later
00024  * version.
00025  *
00026  * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00027  * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00028  * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00029  * details.
00030  *
00031  * You should have received a copy of the GNU General Public License along
00032  * with FORM. If not, see <http://www.gnu.org/licenses/>.
00033  */
00034 /* #[ License : */
00035
00036 /*
00037  *#[ macros & definitions :
00038  */
00039 #define MASTER 0
00040
00041 #define PF_RESET 0
00042 #define PF_TIME 1
00043
00044 #define PF_TERM_MSGTAG 10 /* master -> slave: sending terms */
00045 #define PF_ENDSORT_MSGTAG 11 /* master -> slave: no more terms to be distributed, slave ->
master: after EndSort() */
00046 #define PF_DOLLAR_MSGTAG 12 /* slave -> master: sending $-variables */
00047 #define PF_BUFFER_MSGTAG 20 /* slave -> master: sending sorted terms, or in
PF_SendFile()/PF_RecvFile() */
00048 #define PF_ENDBUFFER_MSGTAG 21 /* same as PF_BUFFER_MSGTAG, but indicates the end of operation */
00049 #define PF_READY_MSGTAG 30 /* slave -> master: slave is idle and can accept terms */
00050 #define PF_DATA_MSGTAG 50 /* InParallel, DoCheckpoint() */
00051 #define PF_EMPTY_MSGTAG 52 /* InParallel, DoCheckpoint(), PF_SendFile(), PF_RecvFile() */
00052 #define PF_STDOUT_MSGTAG 60 /* slave -> master: sending text to the stdout */
00053 #define PF_LOG_MSGTAG 61 /* slave -> master: sending text to the log file */
00054 #define PF_OPT_MCTS_MSGTAG 70 /* master <-> slave: optimization */
00055 #define PF_OPT_HORNER_MSGTAG 71 /* master <-> slave: optimization */
00056 #define PF_OPT_COLLECT_MSGTAG 72 /* slave -> master: optimization */
00057 #define PF_MISC_MSGTAG 100
00058
00059 /*
00060  * A macro for checking the version of gcc.
00061  */
00062 #if defined(__GNUC__) && defined(__GNUC_MINOR__) && defined(__GNUC_PATCHLEVEL__)
00063 # define GNUC_PREREQ(major, minor, patchlevel) \
00064     ((__GNUC__ < 16) + ((__GNUC_MINOR__ < 8) + __GNUC_PATCHLEVEL__ >= \
00065     ((major) < 16) + ((minor) < 8) + (patchlevel)))
00066 #else
00067 # define GNUC_PREREQ(major, minor, patchlevel) 0
00068 #endif
00069
00070 /*
00071  * The macro "indices" defined in variable.h collides with some function
00072  * argument names in the MPI-3.0 standard.
00073  */
00074 #undef indices
00075
00076 /* Avoid messy padding warnings which may appear in mpi.h. */
00077 #if GNUC_PREREQ(4, 6, 0)
00078 # pragma GCC diagnostic push
00079 # pragma GCC diagnostic ignored "-Wpadded"
00080 # pragma GCC diagnostic ignored "-Wunused-parameter"
00081 #endif
00082 #if defined(__clang__) && defined(__has_warning)
00083 # pragma clang diagnostic push
00084 # if __has_warning("-Wpadded")
00085 # pragma clang diagnostic ignored "-Wpadded"

```

```

00086 # endif
00087 # if __has_warning("-Wunused-parameter")
00088 # pragma clang diagnostic ignored "-Wunused-parameter"
00089 # endif
00090 #endif
00091
00092 # ifdef __cplusplus
00093 /*
00094  * form3.h (which includes parallel.h) is included from newpoly.h as
00095  * extern "C" {
00096  * #include "form3.h"
00097  * }
00098  * On the other hand, C++ interfaces to MPI are defined in mpi.h if it is
00099  * included from C++ sources. We first leave from the C-linkage, include
00100  * mpi.h, and then go back to the C-linkage.
00101  * (TU 7 Jun 2011)
00102  */
00103 }
00104 # define OMPI_SKIP_MPICXX 1
00105 # include <mpi.h>
00106 extern "C" {
00107 # else
00108 # define OMPI_SKIP_MPICXX 1
00109 # include <mpi.h>
00110 # endif
00111
00112 /* Now redefine "indices" in the same way as in variable.h. */
00113 #define indices ((INDICES) (AC.IndexList.lijst))
00114
00115 /* Restore the warning settings. */
00116 #if GNUC_PREREQ(4, 6, 0)
00117 # pragma GCC diagnostic pop
00118 #endif
00119 #if defined(__clang__) && defined(__has_warning)
00120 # pragma clang diagnostic pop
00121 #endif
00122
00123 # define PF_ANY_SOURCE MPI_ANY_SOURCE
00124 # define PF_ANY_MSGTAG MPI_ANY_TAG
00125 # define PF_COMM MPI_COMM_WORLD
00126 # define PF_BYTE MPI_BYTE
00127 # define PF_INT MPI_INT
00128 # if defined(ILP32)
00129 # define PF_WORD MPI_SHORT
00130 # define PF_LONG MPI_LONG
00131 # elif defined(LLP64)
00132 # define PF_WORD MPI_INT
00133 # define PF_LONG MPI_LONG_LONG_INT
00134 # elif defined(LP64)
00135 # define PF_WORD MPI_INT
00136 # define PF_LONG MPI_LONG
00137 # endif
00138
00139 /*
00140  #] macros & definitions :
00141  #[ s/r-bufs :
00142  */
00143
00144 typedef struct {
00145     WORD **buff;
00146     WORD **fill;
00147     WORD **full;
00148     WORD **stop;
00149     MPI_Status *status;
00150     MPI_Status *retstat;
00151     MPI_Request *request;
00152     MPI_Datatype *type; /* this is needed in PF_Wait for Get_count */
00153     int *index; /* dummies for returnvalues */
00154     int *tag; /* for the version with blocking send/receives */
00155     int *from;
00156     int numbufs; /* number of cyclic buffers */
00157     int active; /* flag telling which buffer is active */
00158     PADPOINTER(0,2,0,0);
00159 } PF_BUFFER;
00160
00161 /*
00162  #] s/r-bufs :
00163  #[ global variables used by the PF_functions : need to be known everywhere
00164  */
00165
00166 typedef struct ParallelVars {
00167     FILEHANDLE slavebuf; /* (slave) allocated if there are RHS expressions */
00168     /* special buffers for nonblocking, unbuffered send/receives */
00169     PF_BUFFER *sbuf; /* set of cyclic send buffers for master_and_ slave */
00170     PF_BUFFER **rbufs; /* array of sets of cyclic receive buffers for master */
00171     int me; /* Internal number of task: master is 0 */
00172     int numtasks; /* total number of tasks */

```

```

00177     int           parallel;           /* flags telling the master and slaves to do the sorting parallel */
00178                                           /* [05nov2003 mt] This flag must be set to 0 in iniModule! */
00179     int           rhsInParallel;      /* flag for parallel executing even if there are RHS expressions */
00180     int           mkSlaveInfile;      /* flag tells that slavebuf is used on the slaves */
00181     int           exprbufsize;        /* buffer size in WORDs to be used for transferring expressions */
00182     int           exprtodo;           /* >= 0: the expression to do in InParallel, -1: otherwise */
00183     int           log;                /* flag for logging mode */
00184     WORD          numsbufs;           /* number of cyclic send buffers (PF.sbuf->numsbufs) */
00185     WORD          numrbufs;           /* number of cyclic receive buffers (PF.rbufs[i]->numsbufs,
i=1,...,numtasks-1) */
00186     PADPOSITION(2,0,8,2,0);
00187 } PARALLELVARS;
00188
00189 extern PARALLELVARS PF;
00190 /*[04oct2005 mt]:*/
00191 /*for broadcasting dollarvars, see parallel.c:PF_BroadcastPreDollar():*/
00192 extern LONG PF_maxDollarChunkSize;
00193 /*:[04oct2005 mt]*/
00194
00195 /*
00196    #] global variables used by the PF_functions :
00197    #[ Function prototypes :
00198 */
00199
00200 /* mpi.c */
00201 extern int PF_ISendSbuf(int,int);
00202 extern int PF_Bcast(void *buffer, int count);
00203 extern int PF_RawSend(int,void *,LONG,int);
00204 extern LONG PF_RawRecv(int *,void *,LONG,int *);
00205
00206 extern int PF_PreparePack(void);
00207 extern int PF_Pack(const void *buffer, size_t count, MPI_Datatype type);
00208 extern int PF_Unpack(void *buffer, size_t count, MPI_Datatype type);
00209 extern int PF_PackString(const UBYTE *str);
00210 extern int PF_UnpackString(UBYTE *str);
00211 extern int PF_Send(int to, int tag);
00212 extern int PF_Receive(int src, int tag, int *psrc, int *ptag);
00213 extern int PF_Broadcast(void);
00214
00215 extern int PF_PrepareLongSinglePack(void);
00216 extern int PF_LongSinglePack(const void *buffer, size_t count, MPI_Datatype type);
00217 extern int PF_LongSingleUnpack(void *buffer, size_t count, MPI_Datatype type);
00218 extern int PF_LongSingleSend(int to, int tag);
00219 extern int PF_LongSingleReceive(int src, int tag, int *psrc, int *ptag);
00220
00221 extern int PF_PrepareLongMultiPack(void);
00222 extern int PF_LongMultiPackImpl(const void *buffer, size_t count, size_t eSize, MPI_Datatype type);
00223 extern int PF_LongMultiUnpackImpl(void *buffer, size_t count, size_t eSize, MPI_Datatype type);
00224 extern int PF_LongMultiBroadcast(void);
00225
00226 static inline size_t sizeof_datatype(MPI_Datatype type)
00227 {
00228     if ( type == PF_BYTE ) return sizeof(char);
00229     if ( type == PF_INT ) return sizeof(int);
00230     if ( type == PF_WORD ) return sizeof(WORD);
00231     if ( type == PF_LONG ) return sizeof(LONG);
00232     return(0);
00233 }
00234
00235 #define PF_LongMultiPack(buffer, count, type) PF_LongMultiPackImpl(buffer, count,
sizeof_datatype(type), type)
00236 #define PF_LongMultiUnpack(buffer, count, type) PF_LongMultiUnpackImpl(buffer, count,
sizeof_datatype(type), type)
00237
00238 /* parallel.c */
00239 extern int PF_EndSort(void);
00240 extern WORD PF_Deferred(WORD *,WORD);
00241 extern int PF_Processor(EXPRESSIONS,WORD,WORD);
00242 extern int PF_Init(int*,char ***);
00243 extern int PF_Terminate(int);
00244 extern LONG PF_GetSlaveTimes(void);
00245 extern LONG PF_BroadcastNumber(LONG);
00246 extern void PF_BroadcastBuffer(WORD **buffer, LONG *length);
00247 extern int PF_BroadcastString(UBYTE *);
00248 extern int PF_BroadcastPreDollar(WORD **, LONG *,int *);
00249 extern int PF_CollectModifiedDollars(void);
00250 extern int PF_BroadcastModifiedDollars(void);
00251 extern int PF_BroadcastRedefinedPreVars(void);
00252 extern int PF_BroadcastCBuf(int bufnum);
00253 extern int PF_BroadcastExpFlags(void);
00254 extern int PF_StoreInsideInfo(void);
00255 extern int PF_RestoreInsideInfo(void);
00256 extern int PF_BroadcastExpr(EXPRESSIONS e, FILEHANDLE *file);
00257 extern int PF_BroadcastRHS(void);
00258 extern int PF_InParallelProcessor(void);
00259 extern int PF_SendFile(int to, FILE *fd);
00260 extern int PF_RecvFile(int from, FILE *fd);

```

```

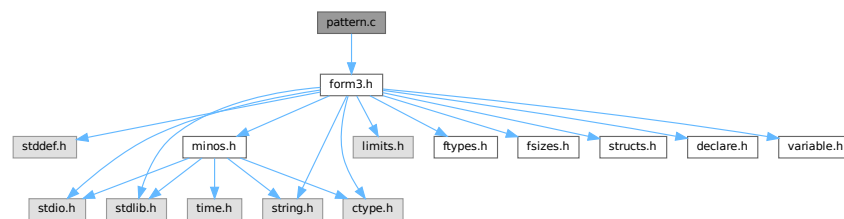
00261 extern void PF_MLock(void);
00262 extern void PF_MUnlock(void);
00263 extern LONG PF_WriteFileToFile(int, UBYTE *, LONG);
00264 extern void PF_FlushStdOutBuffer(void);
00265
00266 /*
00267  #] Function prototypes :
00268  */
00269
00270 #endif

```

4.98 pattern.c File Reference

```
#include "form3.h"
```

Include dependency graph for pattern.c:



Macros

- `#define PutInBuffers(pow)`

Functions

- `WORD TestMatch` (PHEAD WORD *term, WORD *level)
- `void Substitute` (PHEAD WORD *term, WORD *pattern, WORD power)
- `WORD FindAll` (PHEAD WORD *term, WORD *pattern, WORD level, WORD *par)
- `int TestSelect` (WORD *term, WORD *setp)
- `void SubsInAll` (PHEAD0)
- `void TransferBuffer` (int from, int to, int spectator)
- `int TakeIDfunction` (PHEAD WORD *term)

4.98.1 Detailed Description

Top level pattern matching routines. More pattern matching is found in [findpat.c](#), [function.c](#), [symmetr.c](#) and [smart.c](#). The last three files contain the matching inside functions. The file [pattern.c](#) contains also the very important routine `Substitute`. All regular pattern matching is just the finding of the pattern and indicating what are the wildcards etc. The routine `Substitute` does the actual removal of the pattern and replaces it by a subterm of the type SUBEXPRESSION.

Definition in file [pattern.c](#).

4.98.2 Macro Definition Documentation

4.98.2.1 PutInBuffers

```
#define PutInBuffers(
    pow )
```

Value:

```
AddRHS(AT.ebufnum,1); \
*out++ = SUBEXPRESSION; \
*out++ = SUBEXPSIZE; \
*out++ = C->numrhs; \
*out++ = pow; \
*out++ = AT.ebufnum; \
FILLSUB(out) \
r = AT.pWorkSpace[rhs+i]; \
if ( *r > 0 ) { \
    oldinr = r[*r]; r[*r] = 0; \
    AddNtoC(AT.ebufnum, (*r+1-ARGHEAD), (r+ARGHEAD), 14); \
    r[*r] = oldinr; \
} \
else { \
    ToGeneral(r,buffer,1); \
    buffer[buffer[0]] = 0; \
    AddNtoC(AT.ebufnum,buffer[0]+1,buffer,15); \
}
```

Definition at line 2193 of file [pattern.c](#).

4.98.3 Function Documentation

4.98.3.1 TestMatch()

```
WORD TestMatch (
    PHEAD WORD * term,
    WORD * level )
```

This routine governs the pattern matching. If it decides that a substitution should be made, this can be either the insertion of a right hand side (C->rhs) or the automatic generation of terms as a result of an operation (like trace). The object to be replaced is removed from term and a subexpression pointer is inserted. If the substitution is made more than once there can be more subexpression pointers. Its number is positive as it corresponds to the level at which the C->rhs can be found in the compiler output. The subexpression pointer contains the wildcard substitution information. The power is found in *AT.TMout. For operations the subexpression pointer is negative and corresponds to an address in the array AT.TMout. In this array are then the instructions for the routine to be called and its number in the array 'Operations' The format is here: length,functionnumber,length-2 parameters

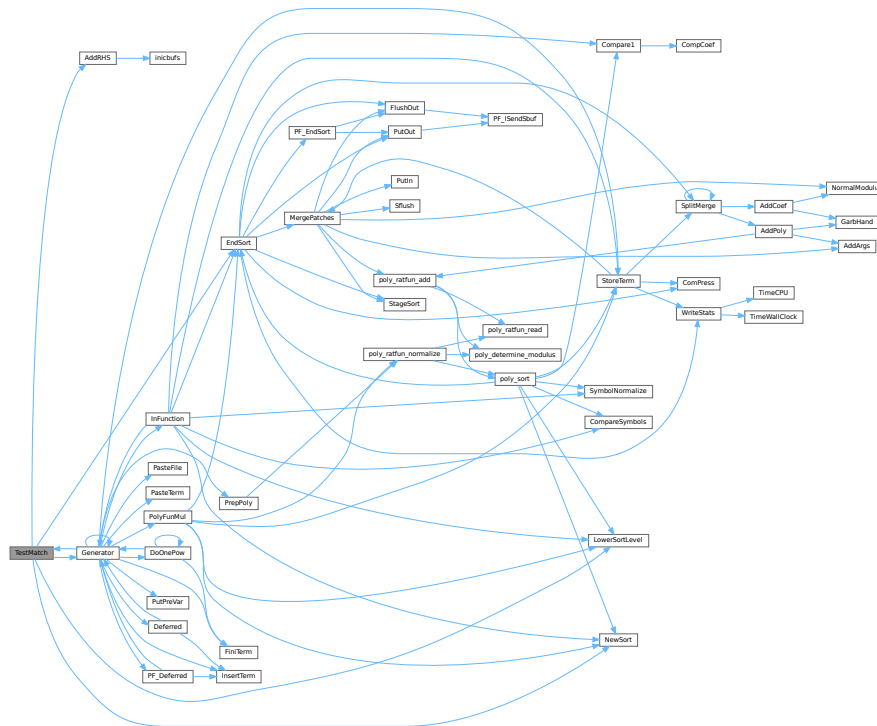
There is a certain complexity wrt repeat levels. Another complication is the poking of the wildcard values in the subexpression prototype in the compiler buffer. This was how things were done in the past with sequential FORM, but with the advent of TFORM this cannot be maintained. Now, for TFORM we make a copy of it. 7-may-2008 (JV): We cannot yet guarantee that this has been done 100% correctly. There are errors that occur in TFORM only and that may indicate problems.

Definition at line 97 of file [pattern.c](#).

References [AddRHS\(\)](#), [EndSort\(\)](#), [Generator\(\)](#), [CbUf::lhs](#), [LowerSortLevel\(\)](#), and [NewSort\(\)](#).

Referenced by [Generator\(\)](#).

Here is the call graph for this function:



4.98.3.2 Substitute()

```
void Substitute (
    PHEAD WORD * term,
    WORD * pattern,
    WORD power )
```

Definition at line 640 of file [pattern.c](#).

4.98.3.3 FindAll()

```
WORD FindAll (
    PHEAD WORD * term,
    WORD * pattern,
    WORD level,
    WORD * par )
```

Definition at line 1197 of file [pattern.c](#).

4.98.3.4 TestSelect()

```
int TestSelect (
    WORD * term,
    WORD * setp )
```

Definition at line 1748 of file [pattern.c](#).

4.98.3.5 SubsInAll()

```
void SubsInAll (
    PHEAD0 )
```

Definition at line 1982 of file [pattern.c](#).

4.98.3.6 TransferBuffer()

```
void TransferBuffer (
    int from,
    int to,
    int spectator )
```

Definition at line 2143 of file [pattern.c](#).

4.98.3.7 TakeIDfunction()

```
int TakeIDfunction (
    PHEAD WORD * term )
```

Definition at line 2213 of file [pattern.c](#).

4.99 pattern.c

[Go to the documentation of this file.](#)

```
00001
00012 /* #[] License : */
00013 /*
00014 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00015 * When using this file you are requested to refer to the publication
00016 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00017 * This is considered a matter of courtesy as the development was paid
00018 * for by FOM the Dutch physics granting agency and we would like to
00019 * be able to track its scientific use to convince FOM of its value
00020 * for the community.
00021 *
00022 * This file is part of FORM.
00023 *
00024 * FORM is free software: you can redistribute it and/or modify it under the
00025 * terms of the GNU General Public License as published by the Free Software
00026 * Foundation, either version 3 of the License, or (at your option) any later
00027 * version.
00028 *
00029 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00030 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00031 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00032 * details.
00033 *
00034 * You should have received a copy of the GNU General Public License along
00035 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00036 */
00037 /* #[] License : */
00038 /*
00039 !!! Notice the change in OnePV in FindAll (7-may-2008 JV).
00040
00041 #[] Includes : pattern.c
00042 */
00043
00044 #include "form3.h"
00045
00046 /*
00047 #[] Includes :
00048 #[] Patterns :
00049 #[] Rules :
```

```

00050
00051     There are several rules governing the allowable replacements.
00052     1: Multi with anything but symbols or dotproducts reverts
00053         to many.
00054     2: Each symbol can have only one (wildcard) power, so
00055         x^2*x^n? is illegal.
00056     3: when a single vector is used it replaces all occurrences
00057         of the vector. Therefore q*q(mu) or q*q(mu) cannot occur.
00058         Also q*q cannot be done.
00059     4: Loose vector elements are replaced with p(mu), dotproducts
00060         with p?.q.
00061     5: p?.q? is allowed.
00062     6: x^n? can revert to n = 0 if there is no power of x.
00063     7: x?^n? must match some x. There could be an ambiguity otherwise.
00064
00065     #] Rules :
00066     #[ TestMatch :          WORD TestMatch(term,level)
00067 */
00068
00097 WORD TestMatch(PHEAD WORD *term, WORD *level)
00098 {
00099     GETBIDENTITY
00100     WORD *ll, *m, *w, *llf, *OldWork, *StartWork, *ww, *mm, *t, *OldTermBuffer = 0;
00101     WORD power = 0, match = 0, i, msign = 0, ll2;
00102     int numdollars = 0, protosize, oldallnumrhs;
00103     CBUF *C = cbuf+AM.rbufnum, *CC;
00104     AT.idallflag = 0;
00105     do {
00106 /*
00107         #] Preliminaries :
00108 */
00109         ll = C->lhs[*level];
00110         if ( *ll == TYPEEXPRESSION ) {
00111 /*
00112             Expressions are not subject to anything.
00113 */
00114             return(0);
00115         }
00116         else if ( *ll == TYPEREPEAT ) {
00117             ++AN.RepPoint = 0;
00118             return(0);          /* Will force the next level */
00119         }
00120         else if ( *ll == TYPEENDREPEAT ) {
00121             if ( *AN.RepPoint ) {
00122                 AN.RepPoint[-1] = 1;          /* Mark the higher level as dirty */
00123                 *AN.RepPoint = 0;
00124                 *level = ll[2];          /* Level to jump back to */
00125             }
00126             else {
00127                 AN.RepPoint--;
00128                 if ( AN.RepPoint < AT.RepCount ) {
00129                     MLOCK(ErrorMessageLock);
00130                     MesPrint("Internal problems with REPEAT count");
00131                     MUNLOCK(ErrorMessageLock);
00132                     Terminate(-1);
00133                 }
00134             }
00135             return(0);          /* Force the next level */
00136         }
00137         else if ( *ll == TYPEOPERATION ) {
00138 /*
00139             Operations have always their own level.
00140 */
00141             if ( (*(FG.OperaFind[ll[2]])) (BHEAD term,ll) ) return(-1);
00142             else return(0);
00143         }
00144 /*
00145         #] Preliminaries :
00146 */
00147         OldWork = AT.WorkPointer;
00148         if ( AT.WorkPointer < term + *term ) AT.WorkPointer = term + *term;
00149         ww = AT.WorkPointer;
00150 /*
00151         Here we need to make a copy of the subexpression object because we
00152         will be writing the values of the wildcards in it.
00153         Originally we copied it into the private version of the compiler buffer
00154         that is used for scratch space (ebufnum). This caused errors in the
00155         routines like ScanFunctions when the ebufnum Buffer was expanded
00156         and inpat was still pointing at the old Buffer. This expansion
00157         could be done in AddWild and hence cannot be fixed at > 100 places.
00158         The solution is to use AN.patternbuffer (JV 16-mar-2009).
00159 */
00160         {
00161             WORD *ta = ll, *ma;
00162             int ja = ta[1];
00163 /*
00164             New code (16-mar-2009) JV

```

```

00165 */
00166     if ( ( ja + 2 ) > AN.patternbuffersize ) {
00167         if ( AN.patternbuffer ) M_free(AN.patternbuffer,"AN.patternbuffer");
00168         AN.patternbuffersize = 2 * ja + 2;
00169         AN.patternbuffer = (WORD *)Malloc(AN.patternbuffersize * sizeof(WORD),
00170             "AN.patternbuffer");
00171     }
00172     ma = AN.patternbuffer;
00173     m = ma + IDHEAD;
00174     NCOPY(ma,ta,ja);
00175     *ma = 0;
00176 }
00177 AN.FullProto = m;
00178 AN.WildValue = w = m + SUBEXPSIZE;
00179 protosize = IDHEAD + m[1];
00180 m += m[1];
00181 AN.WildStop = m;
00182 StartWork = ww;
00183 ll2 = ll[2];
00184 /*
00185     #[ Expand dollars :
00186 */
00187 if ( ( ll[4] & DOLLARFLAG ) != 0 ) { /* We have at least one dollar in the pattern */
00188     WORD oldRepPoint = *AN.RepPoint, olddefer = AR.DeferFlag;
00189     AR.Eside = LHSIDEX;
00190 /*
00191     Copy into WorkSpace. This means that AN.patternbuffer will be free.
00192 */
00193     ww = AT.WorkPointer; i = m[0]; mm = m;
00194     NCOPY(ww,mm,i);
00195     *StartWork += 3;
00196     *ww++ = 1; *ww++ = 1; *ww++ = 3;
00197     AT.WorkPointer = ww;
00198     AR.DeferFlag = 0;
00199     NewSort(BHEAD0);
00200     if ( Generator(BHEAD StartWork,AR.Cnumlhs) ) {
00201         LowerSortLevel();
00202         AT.WorkPointer = OldWork;
00203         AR.DeferFlag = olddefer;
00204         return(-1);
00205     }
00206     AT.WorkPointer = ww;
00207     if ( EndSort(BHEAD ww,0) < 0 ) {}
00208     AR.DeferFlag = olddefer;
00209     if ( *ww == 0 || *(ww+ww) != 0 ) {
00210         if ( AP.lhdollarerror == 0 ) {
00211 /*
00212             If race condition we just get more error messages
00213 */
00214             MLOCK(ErrorMessageLock);
00215             MesPrint("&LHS must be one term");
00216             MUNLOCK(ErrorMessageLock);
00217             AP.lhdollarerror = 1;
00218         }
00219         AT.WorkPointer = OldWork;
00220         return(-1);
00221     }
00222     m = ww; ww = m + *m;
00223     if ( m[*m-1] < 0 ) { msign = 1; m[*m-1] = -m[*m-1]; }
00224     if ( *ww || m[*m-1] != 3 || m[*m-2] != 1 || m[*m-3] != 1 ) {
00225         MLOCK(ErrorMessageLock);
00226         MesPrint("Dollar variable develops into an illegal pattern in id-statement");
00227         MUNLOCK(ErrorMessageLock);
00228         return(-1);
00229     }
00230     *m -= m[*m-1];
00231     if ( ( *m + 1 + protosize ) > AN.patternbuffersize ) {
00232         if ( AN.patternbuffer ) M_free(AN.patternbuffer,"AN.patternbuffer");
00233         AN.patternbuffersize = 2 * (*m) + 2 + protosize;
00234         AN.patternbuffer = (WORD *)Malloc(AN.patternbuffersize * sizeof(WORD),
00235             "AN.patternbuffer");
00236         mm = ll; ww = AN.patternbuffer; i = protosize;
00237         NCOPY(ww,mm,i);
00238         AN.FullProto = AN.patternbuffer + IDHEAD;
00239         AN.WildValue = w = AN.FullProto + SUBEXPSIZE;
00240         AN.WildStop = AN.patternbuffer + protosize;
00241     }
00242     mm = AN.patternbuffer + protosize;
00243     i = *m;
00244     NCOPY(mm,m,i);
00245     m = AN.patternbuffer + protosize;
00246     AR.Eside = RHSIDE;
00247     *mm = 0;
00248 /*
00249     Test the pattern. If only wildcard powers -> SUBONCE
00250 */
00251     {

```

```

00252         WORD *mmm = m + *m, *m1 = m+1, jm, noveto = 0;
00253         while ( m1 < mmm ) {
00254             if ( *m1 == SYMBOL ) {
00255                 for ( jm = 2; jm < m1[1]; jm+=2 ) {
00256                     if ( m1[jm+1] < MAXPOWER && m1[jm+1] > -MAXPOWER ) break;
00257                 }
00258                 if ( jm < m1[1] ) { noveto = 1; break; }
00259             }
00260             else if ( *m1 == DOTPRODUCT ) {
00261                 for ( jm = 2; jm < m1[1]; jm+=3 ) {
00262                     if ( m1[jm+2] < MAXPOWER && m1[jm+2] > -MAXPOWER ) break;
00263                 }
00264                 if ( jm < m1[1] ) { noveto = 1; break; }
00265             }
00266             else { noveto = 1; break; }
00267             m1 += m1[1];
00268         }
00269         if ( noveto == 0 ) {
00270             l12 = l12 & ~SUBMASK;
00271             l12 |= SUBONCE;
00272         }
00273     }
00274     AT.WorkPointer = ww = StartWork;
00275     *AN.RepPoint = oldRepPoint;
00276 }
00277 /*
00278     #] Expand dollars :
00279
00280     In case of id,all we have to check at this point that there are only
00281     functions in the pattern.
00282 */
00283 if ( ( l12 & SUBMASK ) == SUBALL ) {
00284     WORD *t = AN.patternbuffer+IDHEAD, *tt;
00285     WORD *tstop, *ttstop, ii;
00286     t += t[1]; tstop = t + *t; t++;
00287     while ( t < tstop ) {
00288         if ( *t < FUNCTION ) break;
00289         t += t[1];
00290     }
00291     if ( t < tstop ) {
00292         MLOCK(ErrorMessageLock);
00293         MesPrint("Error: id,all can only be used with (products of) functions and/or tensors.");
00294         MUNLOCK(ErrorMessageLock);
00295         return(-1);
00296     }
00297     OldTermBuffer = AN.termbuffer;
00298     AN.termbuffer = TermMalloc("id,all");
00299 /*
00300     Now make sure that only regular functions and tensors can take part.
00301 */
00302     tt = term; ttstop = tt+*tt; ttstop -= ABS(ttstop[-1]); tt++;
00303     t = AN.termbuffer+1;
00304     while ( tt < ttstop ) {
00305         if ( *tt >= FUNCTION && *tt != AR.PolyFun && *tt != AR.PolyFunInv ) {
00306             ii = tt[1]; NCOPY(t,tt,ii);
00307         }
00308         else tt += tt[1];
00309     }
00310     *t++ = 1; *t++ = 1; *t++ = 3; AN.termbuffer[0] = t-AN.termbuffer;
00311 }
00312 /*
00313     To be puristic, we need to check that all wildcards in the prototype
00314     are actually present. If the LHS contained a replace_ this may not be
00315     the case.
00316 */
00317 ClearWild(BHEAD0);
00318 while ( w < AN.WildStop ) {
00319     if ( *w == LOADDOLLAR ) numdollars++;
00320     w += w[1];
00321 }
00322 AN.RepFunNum = 0;
00323 /* rep = */ AN.RepFunList = AT.WorkPointer;
00324 AT.WorkPointer = (WORD *)(((UBYTE *) (AT.WorkPointer)) + AM.MaxTer/2);
00325 if ( AT.WorkPointer >= AT.WorkTop ) {
00326     MLOCK(ErrorMessageLock);
00327     MesWork();
00328     MUNLOCK(ErrorMessageLock);
00329     return(-1);
00330 }
00331 AN.DisOrderFlag = l12 & SUBDISORDER;
00332 AN.nogroundlevel = 0;
00333 switch ( l12 & SUBMASK ) {
00334     case SUBONLY :
00335         /* Must be an exact match */
00336         AN.UseFindOnly = 1; AN.ForFindOnly = 0;
00337         if ( FindRest(BHEAD term,m) && ( AN.UsedOtherFind ||
00338             FindOnly(BHEAD term,m) ) ) {

```

```

00339         power = 1;
00340         if ( msign ) term[term[0]-1] = -term[term[0]-1];
00341     }
00342     else power = 0;
00343     break;
00344     case SUBMANY :
00345         AN.UseFindOnly = -1;
00346         if ( ( power = FindRest(BHEAD term,m) ) > 0 ) {
00347             if ( ( power = FindOnce(BHEAD term,m) ) > 0 ) {
00348                 AN.UseFindOnly = 0;
00349                 do {
00350                     if ( msign ) term[term[0]-1] = -term[term[0]-1];
00351                     Substitute(BHEAD term,m,1);
00352                     if ( numdollars ) {
00353                         WildDollars(BHEAD (WORD *)0);
00354                         numdollars = 0;
00355                     }
00356                     if ( ww < term+term[0] ) ww = term+term[0];
00357                     ClearWild(BHEAD0);
00358                     AT.WorkPointer = ww;
00359                     /*
00360                     if ( rep < ww ) { */
00361                         AN.RepFunNum = 0;
00362                         /* rep = */ AN.RepFunList = ww;
00363                         AT.WorkPointer = (WORD *)(((UBYTE *) (AT.WorkPointer)) + AM.MaxTer/2);
00364                         if ( AT.WorkPointer >= AT.WorkTop ) {
00365                             MLOCK(ErrorMessageLock);
00366                             MesWork();
00367                             MUNLOCK(ErrorMessageLock);
00368                             return(-1);
00369                         }
00370                     }
00371                     else {
00372                         AN.RepFunList = rep;
00373                         AN.RepFunNum = 0;
00374                     }
00375                 /*
00376                 AN.nogroundlevel = 0;
00377             } while ( FindRest(BHEAD term,m) && ( AN.UsedOtherFind ||
00378                 FindOnce(BHEAD term,m) ) );
00379             match = 1;
00380         }
00381         else if ( power < 0 ) {
00382             do {
00383                 if ( msign ) term[term[0]-1] = -term[term[0]-1];
00384                 Substitute(BHEAD term,m,1);
00385                 if ( numdollars ) {
00386                     WildDollars(BHEAD (WORD *)0);
00387                     numdollars = 0;
00388                 }
00389                 if ( ww < term+term[0] ) ww = term+term[0];
00390                 ClearWild(BHEAD0);
00391                 AT.WorkPointer = ww;
00392                 /*
00393                 if ( rep < ww ) { */
00394                     AN.RepFunNum = 0;
00395                     /* rep = */ AN.RepFunList = ww;
00396                     AT.WorkPointer = (WORD *)(((UBYTE *) (AT.WorkPointer)) + AM.MaxTer/2);
00397                     if ( AT.WorkPointer >= AT.WorkTop ) {
00398                         MLOCK(ErrorMessageLock);
00399                         MesWork();
00400                         MUNLOCK(ErrorMessageLock);
00401                         return(-1);
00402                     }
00403                 }
00404                 else {
00405                     AN.RepFunList = rep;
00406                     AN.RepFunNum = 0;
00407                 }
00408             /*
00409             } while ( FindRest(BHEAD term,m) );
00410             match = 1;
00411         }
00412     }
00413     else if ( power < 0 ) {
00414         if ( FindOnce(BHEAD term,m) ) {
00415             do {
00416                 if ( msign ) term[term[0]-1] = -term[term[0]-1];
00417                 Substitute(BHEAD term,m,1);
00418                 if ( numdollars ) {
00419                     WildDollars(BHEAD (WORD *)0);
00420                     numdollars = 0;
00421                 }
00422                 if ( ww < term+term[0] ) ww = term+term[0];
00423                 ClearWild(BHEAD0);
00424                 AT.WorkPointer = ww;
00425                 /*

```

```

00426             AN.RepFunNum = 0;
00427             /* rep = */ AN.RepFunList = ww;
00428             AT.WorkPointer = (WORD *) ((UBYTE *) (AT.WorkPointer)) + AM.MaxTer/2;
00429             if ( AT.WorkPointer >= AT.WorkTop ) {
00430                 MLOCK(ErrorMessageLock);
00431                 MesWork();
00432                 MUNLOCK(ErrorMessageLock);
00433                 return(-1);
00434             }
00435 /*
00436             }
00437             else {
00438                 AN.RepFunList = rep;
00439                 AN.RepFunNum = 0;
00440             }
00441 */
00442             } while ( FindOnce(BHEAD term,m) );
00443             match = 1;
00444         }
00445     }
00446     if ( match ) {
00447         if ( ( ll2 & SUBAFTER ) != 0 ) *level = AC.Labels[ll[3]];
00448     }
00449     else {
00450         if ( ( ll2 & SUBAFTERNOT ) != 0 ) *level = AC.Labels[ll[3]];
00451     }
00452     goto nextlevel;
00453 case SUBONCE :
00454     AN.UseFindOnly = 0;
00455     if ( FindRest(BHEAD term,m) && ( AN.UsedOtherFind || FindOnce(BHEAD term,m) ) ) {
00456         power = 1;
00457         if ( msign ) term[term[0]-1] = -term[term[0]-1];
00458     }
00459     else power = 0;
00460     break;
00461 case SUBMULTI :
00462     power = FindMulti(BHEAD term,m);
00463     if ( ( power & 1 ) != 0 && msign ) term[term[0]-1] = -term[term[0]-1];
00464     break;
00465 case SUBVECTOR :
00466     while ( ( power = FindAll(BHEAD term,m,*level,(WORD *)0) ) != 0 ) {
00467         if ( ( power & 1 ) != 0 && msign ) term[term[0]-1] = -term[term[0]-1];
00468         match = 1;
00469     }
00470     break;
00471 case SUBSELECT :
00472     llf = ll + IDHEAD; llf += llf[1]; llf += *llf;
00473     AN.UseFindOnly = 1; AN.ForFindOnly = llf;
00474     if ( FindRest(BHEAD term,m) && ( AN.UsedOtherFind || FindOnly(BHEAD term,m) ) ) {
00475         if ( msign ) term[term[0]-1] = -term[term[0]-1];
00476 /*
00477         The following code needs to be hacked a bit to allow for
00478         all types of sets and for occurrence anywhere in the term
00479         The code at the end of FindOnly is a bit mysterious.
00480 */
00481         if ( llf[1] > 2 ) {
00482             WORD *t1, *t2;
00483             if ( *term > AN.sizeselecttermundo ) {
00484                 if ( AN.selecttermundo ) M_free(AN.selecttermundo,"AN.selecttermundo");
00485                 AN.sizeselecttermundo = *term + 10;
00486                 AN.selecttermundo = (WORD *) Malloc1(
00487                     AN.sizeselecttermundo*sizeof(WORD),"AN.selecttermundo");
00488             }
00489             t1 = term; t2 = AN.selecttermundo; i = *term;
00490             NCOPY(t2,t1,i);
00491         }
00492         power = 1;
00493         Substitute(BHEAD term,m,power);
00494         if ( llf[1] > 2 ) {
00495             if ( TestSelect(term,llf) ) {
00496                 WORD *t1, *t2;
00497                 power = 0;
00498                 t1 = term; t2 = AN.selecttermundo; i = *t2;
00499                 NCOPY(t1,t2,i);
00500 #if IDHEAD > 3
00501                 if ( ( ll2 & SUBAFTERNOT ) != 0 ) {
00502                     *level = AC.Labels[ll[3]];
00503                 }
00504 #endif
00505                 goto nextlevel;
00506             }
00507         }
00508         if ( numdollars ) {
00509             WildDollars(BHEAD (WORD *)0);
00510             numdollars = 0;
00511         }
00512         match = 1;

```



```

00513         if ( ( ll2 & SUBAFTER ) != 0 ) {
00514             *level = AC.Labels[ll[3]];
00515         }
00516     }
00517     else {
00518         if ( ( ll2 & SUBAFTERNOT ) != 0 ) {
00519             *level = AC.Labels[ll[3]];
00520         }
00521         power = 0;
00522     }
00523     goto nextlevel;
00524 case SUBALL:
00525     AN.UseFindOnly = 0;
00526     CC = cbuf+AT.allbufnum;
00527     oldallnumrhs = CC->numrhs;
00528     t = AddRHS(AT.allbufnum,1);
00529     *t = 0;
00530     AT.idallflag = 1;
00531     AT.idallmaxnum = ll[5];
00532     AT.idallnum = 0;
00533     if ( FindRest(BHEAD AN.termbuffer,m) || AT.idallflag > 1 ) {
00534         WORD *t, *tstop, *tt, first = 1, ii;
00535         power = 1;
00536         *CC->Pointer++ = 0;
00537         if ( msign ) term[term[0]-1] = -term[term[0]-1];
00538     /*
00539         If we come here the matches are all already in the
00540         compiler buffer. All we need to do is take out all
00541         functions and replace them by a SUBEXPRESSION that
00542         points to this buffer.
00543         Note: the PolyFun/PolyRatFun should be excluded from this.
00544         This works because each match writes incrementally to
00545         the buffer using the routine SubsInAll.
00546
00547         The call to WildDollars should be made in Generator.....
00548     */
00549     t = term; tstop = t + *t; ii = ABS(tstop[-1]); tstop -= ii;
00550     tt = AT.WorkPointer+1;
00551     t++;
00552     while ( t < tstop ) {
00553         if ( *t >= FUNCTION && *t != AR.PolyFun && *t != AR.PolyFunInv ) {
00554             if ( first ) { /* SUBEXPRESSION */
00555                 *tt++ = SUBEXPRESSION;
00556                 *tt++ = SUBEXPSIZE;
00557                 *tt++ = CC->numrhs;
00558                 *tt++ = 1;
00559                 *tt++ = AT.allbufnum;
00560                 FILLSUB(tt)
00561                 first = 0;
00562             }
00563             t += t[1];
00564         }
00565         else {
00566             i = t[1]; NCOPY(tt,t,i);
00567         }
00568     }
00569     if ( ( ll[4] & NORMALIZEFLAG ) != 0 ) {
00570     /*
00571         In case of the normalization option, we have to divide
00572         by AT.idallnum;
00573     */
00574         WORD na = t[ii-1];
00575         na = REDLENG(na);
00576         for ( i = 0; i < ii; i++ ) tt[i] = t[i];
00577         Divvy(BHEAD (UWORD *)tt,&na,(UWORD *)(&(AT.idallnum)),1);
00578         na = INCLENG(na);
00579         ii = ABS(na);
00580         tt[ii-1] = na;
00581         tt += ii;
00582     }
00583     else {
00584         NCOPY(tt,t,ii);
00585     }
00586     ii = tt-AT.WorkPointer;
00587     *(AT.WorkPointer) = ii;
00588     tt = AT.WorkPointer; t = term;
00589     NCOPY(t,tt,ii);
00590
00591     if ( ( ll2 & SUBAFTER ) != 0 ) { /* ifmatch -> */
00592         *level = AC.Labels[ll[3]];
00593     }
00594     TermFree(AN.termbuffer,"id,all");
00595     AN.termbuffer = OldTermBuffer;
00596     AT.WorkPointer = AN.RepFunList;
00597     AT.idallflag = 0;
00598     CC->Pointer[0] = 0;
00599     TransferBuffer(AT.aebufnum,AT.ebufnum,AT.allbufnum);

```

```

00600         return(1);
00601     }
00602     AT.idallflag = 0;
00603     power = 0;
00604     CC->numrhs = oldallnumrhs;
00605     TermFree(AN.termbuffer,"id,all");
00606     AN.termbuffer = OldTermBuffer;
00607     break;
00608     default :
00609         break;
00610 }
00611 if ( power ) {
00612     Substitute(BHEAD term,m,power);
00613     if ( numdollars ) {
00614         WildDollars(BHEAD (WORD *)0);
00615         numdollars = 0;
00616     }
00617     match = 1;
00618     if ( ( ll2 & SUBAFTER ) != 0 ) { /* ifmatch -> */
00619         *level = AC.Labels[ll[3]];
00620     }
00621 }
00622 else {
00623     AT.WorkPointer = AN.RepFunList;
00624     if ( ( ll2 & SUBAFTERNOT ) != 0 ) { /* ifnomatch -> */
00625         *level = AC.Labels[ll[3]];
00626     }
00627 }
00628 nextlevel:;
00629 } while ( (*level)++ < AR.Cnumlhs && C->lhs[*level][0] == TYPEIDOLD );
00630 (*level)--;
00631 AT.WorkPointer = AN.RepFunList;
00632 return(match);
00633 }
00634
00635 /*
00636     #] TestMatch :
00637     #[ Substitute :      void Substitute(term,pattern,power)
00638 */
00639
00640 void Substitute(PHEAD WORD *term, WORD *pattern, WORD power)
00641 {
00642     GETBIDENTITY
00643     WORD *TemTerm;
00644     WORD *t, *m;
00645     WORD *tstop, *mstop;
00646     WORD *xstop, *ystop;
00647     WORD nt, *fill, nq, mt;
00648     WORD *q, *subterm, *tcoef, oldvall = 0, newval3, i = 0;
00649     WORD PutExpr = 0, sign = 0;
00650     TemTerm = AT.WorkPointer;
00651     if ( ( (WORD *)((UBYTE *) (AT.WorkPointer)) + AM.MaxTer*2) ) > AT.WorkTop ) {
00652         MLOCK(ErrorMessageLock);
00653         MesWork();
00654         MUNLOCK(ErrorMessageLock);
00655         Terminate(-1);
00656     }
00657     m = pattern;
00658     mstop = m + *m;
00659     m++;
00660     t = term;
00661     t += *term - 1;
00662     tcoef = t;
00663     tstop = t - ABS(*t) + 1;
00664     t = term;
00665     t++;
00666     fill = TemTerm;
00667     fill++;
00668     if ( m < mstop ) { do {
00669 /*
00670         #[ SYMBOLS :
00671 */
00672         if ( *m == SYMBOL ) {
00673             ystop = m + m[1];
00674             m += 2;
00675             while ( *t != SYMBOL && t < tstop ) {
00676                 nq = t[1];
00677                 NCOPY(fill,t,nq);
00678             }
00679             if ( t >= tstop ) goto SubCoef;
00680             *fill++ = SYMBOL;
00681             fill++;
00682             subterm = fill;
00683             xstop = t + t[1];
00684             t += 2;
00685             do {
00686                 if ( *m == *t && t < xstop ) {

```

```

00687         nt = t[1];
00688         mt = m[1];
00689         if ( mt >= 2*MAXPOWER ) {
00690             if ( CheckWild(BHEAD mt-2*MAXPOWER, SYMTONUM, -MAXPOWER, &newval3) ) {
00691                 nt -= AN.oldvalue;
00692                 goto SubsL1;
00693             }
00694         }
00695         else if ( mt <= -2*MAXPOWER ) {
00696             if ( CheckWild(BHEAD -mt-2*MAXPOWER, SYMTONUM, -MAXPOWER, &newval3) ) {
00697                 nt += AN.oldvalue;
00698                 goto SubsL1;
00699             }
00700         }
00701         else {
00702             nt -= mt * power;
00703 SubsL1:         if ( nt ) {
00704                 *fill++ = *t;
00705                 *fill++ = nt;
00706             }
00707         }
00708         m += 2; t += 2;
00709     }
00710     else if ( *m >= 2*MAXPOWER ) {
00711         while ( t < xstop ) { *fill++ = *t++; *fill++ = *t++; }
00712         nq = WORDDIF(fill, subterm);
00713         fill = subterm;
00714         while ( nq > 0 ) {
00715             if ( !CheckWild(BHEAD *m-2*MAXPOWER, SYMTOSYM, *fill, &newval3) ) {
00716                 mt = m[1];
00717                 if ( mt >= 2*MAXPOWER ) {
00718                     if ( CheckWild(BHEAD mt-2*MAXPOWER, SYMTONUM, -MAXPOWER, &newval3) ) {
00719                         if ( fill[1] -= AN.oldvalue ) goto SubsL2;
00720                     }
00721                 }
00722                 else if ( mt <= -2*MAXPOWER ) {
00723                     if ( CheckWild(BHEAD -mt-2*MAXPOWER, SYMTONUM, -MAXPOWER, &newval3) ) {
00724                         if ( fill[1] += AN.oldvalue ) goto SubsL2;
00725                     }
00726                 }
00727                 else {
00728                     if ( fill[1] -= mt * power ) {
00729 SubsL2:         fill += nq;
00730                     nq = 0;
00731                 }
00732             }
00733             break;
00734         }
00735         nq -= 2;
00736         fill += 2;
00737     }
00738     if ( nq ) {
00739         nq -= 2;
00740         q = fill + 2;
00741         while ( --nq >= 0 ) *fill++ = *q++;
00742     }
00743     m += 2;
00744 }
00745     else if ( *m < *t || t >= xstop ) { m += 2; }
00746     else { *fill++ = *t++; *fill++ = *t++; }
00747 } while ( m < ystop );
00748 while ( t < xstop ) *fill++ = *t++;
00749 nq = WORDDIF(fill, subterm);
00750 if ( nq > 0 ) {
00751     nq += 2;
00752     subterm[-1] = nq;
00753 }
00754 else { fill = subterm; fill -= 2; }
00755 }
00756 /*
00757     #] SYMBOLS :
00758     #[ DOTPRODUCTS :
00759 */
00760 else if ( *m == DOTPRODUCT ) {
00761     ystop = m + m[1];
00762     m += 2;
00763     while ( *t > DOTPRODUCT && t < tstop ) {
00764         nq = t[1];
00765         NCOPY(fill, t, nq);
00766     }
00767     if ( t >= tstop ) goto SubCoef;
00768     if ( *t != DOTPRODUCT ) {
00769         m = ystop;
00770         goto EndLoop;
00771     }
00772     *fill++ = DOTPRODUCT;
00773     fill++;

```

```

00774      subterm = fill;
00775      xstop = t + t[1];
00776      t += 2;
00777      do {
00778          if ( *m == *t && m[1] == t[1] && t < xstop ) {
00779              nt = t[2];
00780              mt = m[2];
00781              if ( mt >= 2*MAXPOWER ) {
00782                  if ( CheckWild(BHEAD mt-2*MAXPOWER, SYMTONUM, -MAXPOWER, &newval3) ) {
00783                      nt -= AN.oldvalue;
00784                      goto SubsL3;
00785                  }
00786              }
00787              else if ( mt <= -2*MAXPOWER ) {
00788                  if ( CheckWild(BHEAD -mt-2*MAXPOWER, SYMTONUM, -MAXPOWER, &newval3) ) {
00789                      nt += AN.oldvalue;
00790                      goto SubsL3;
00791                  }
00792              }
00793              else {
00794                  nt -= mt * power;
00795                  if ( nt ) {
00796                      *fill++ = *t++;
00797                      *fill++ = *t;
00798                      *fill++ = nt;
00799                      t += 2;
00800                  }
00801                  else t += 3;
00802              }
00803              m += 3;
00804          }
00805          else if ( *m >= (AM.OffsetVector+WILDOFFSET) ) {
00806              while ( t < xstop ) {
00807                  *fill++ = *t++; *fill++ = *t++; *fill++ = *t++;
00808              }
00809              oldvall = 1;
00810              goto SubsL4;
00811          }
00812          else if ( m[1] >= (AM.OffsetVector+WILDOFFSET) ) {
00813              while ( *m >= *t && t < xstop ) {
00814                  *fill++ = *t++; *fill++ = *t++; *fill++ = *t++;
00815              }
00816              oldvall = 0;
00817              nq = WORDDIF(fill, subterm);
00818              fill = subterm;
00819              while ( nq > 0 ) {
00820                  if ( ( oldvall && ( (
00821                      !CheckWild(BHEAD *m-WILDOFFSET, VECTOVEC, *fill, &newval3)
00822                      && !CheckWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, fill[1], &newval3)
00823                      ) || (
00824                      !CheckWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, *fill, &newval3)
00825                      && !CheckWild(BHEAD *m-WILDOFFSET, VECTOVEC, fill[1], &newval3)
00826                      ) ) || ( !oldvall && ( (
00827                          *m == *fill
00828                          && !CheckWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, fill[1], &newval3)
00829                          ) || (
00830                          !CheckWild(BHEAD m[1]-WILDOFFSET, VECTOVEC, *fill, &newval3)
00831                          && *m == fill[1] ) ) ) ) {
00832                      mt = m[2];
00833                      if ( mt >= 2*MAXPOWER ) {
00834                          if ( CheckWild(BHEAD mt-2*MAXPOWER, SYMTONUM, -MAXPOWER, &newval3) ) {
00835                              if ( fill[2] -= AN.oldvalue )
00836                                  goto SubsL5;
00837                          }
00838                      }
00839                      else if ( mt <= -2*MAXPOWER ) {
00840                          if ( CheckWild(BHEAD -mt-2*MAXPOWER, SYMTONUM, -MAXPOWER, &newval3) ) {
00841                              if ( fill[2] += AN.oldvalue )
00842                                  goto SubsL5;
00843                          }
00844                      }
00845                      else {
00846                          if ( fill[2] -= mt * power ) {
00847                              fill += nq;
00848                              nq = 0;
00849                          }
00850                      }
00851                      m += 3;
00852                      break;
00853                  }
00854                  fill += 3; nq -= 3;
00855              }
00856              if ( nq ) {
00857                  nq -= 3;
00858                  q = fill + 3;
00859                  while ( --nq >= 0 ) *fill++ = *q++;
00860              }

```

```

00861         }
00862         else if ( t >= xstop || *m < *t || ( *m == *t && m[1] < t[1] ) )
00863             { m += 3; }
00864         else {
00865             *fill++ = *t++; *fill++ = *t++; *fill++ = *t++;
00866         }
00867     } while ( m < ystop );
00868     while ( t < xstop ) *fill++ = *t++;
00869     nq = WORDDIF(fill,subterm);
00870     if ( nq > 0 ) {
00871         nq += 2;
00872         subterm[-1] = nq;
00873     }
00874     else { fill = subterm; fill -= 2; }
00875 }
00876 /*
00877     #] DOTPRODUCTS :
00878     #[ FUNCTIONS :
00879 */
00880 else if ( *m >= FUNCTION ) {
00881     while ( *t >= FUNCTION || *t == SUBEXPRESSION ) {
00882         nt = WORDDIF(t,term);
00883         for ( mt = 0; mt < AN.RepFunNum; mt += 2 ) {
00884             if ( nt == AN.RepFunList[mt] ) break;
00885         }
00886         if ( mt >= AN.RepFunNum ) {
00887             nq = t[1];
00888             NCOPY(fill,t,nq);
00889         }
00890         else {
00891             WORD *oldt = 0;
00892             if ( *m == GAMMA && m[1] != FUNHEAD+1 ) {
00893                 oldt = t;
00894                 if ( ( i = AN.RepFunList[mt+1] ) > 0 ) {
00895                     *fill++ = GAMMA;
00896                     *fill++ = i + FUNHEAD+1;
00897                     FILLFUN(fill)
00898                     nq = i + 1;
00899                     t += FUNHEAD;
00900                     NCOPY(fill,t,nq);
00901                 }
00902                 t = oldt;
00903             }
00904             else if ( ( *t == LEVICIVITA ) || ( *t >= FUNCTION
00905                 && (functions[*t-FUNCTION].symmetric & ~REVERSEORDER) == ANTISYMMETRIC )
00906                 ) sign += AN.RepFunList[mt+1];
00907             else if ( *m >= FUNCTION+WILDOFFSET
00908                 && (functions[*m-FUNCTION-WILDOFFSET].symmetric & ~REVERSEORDER) == ANTISYMMETRIC
00909                 ) sign += AN.RepFunList[mt+1];
00910             if ( !PutExpr ) {
00911                 xstop = t + t[1];
00912                 t = AN.FullProto;
00913                 nq = t[1];
00914                 t[3] = power;
00915                 NCOPY(fill,t,nq);
00916                 t = xstop;
00917                 PutExpr = 1;
00918             }
00919             else t += t[1];
00920             if ( *m == GAMMA && m[1] != FUNHEAD+1 ) {
00921                 i = oldt[1] - m[1] - i;
00922                 if ( i > 0 ) {
00923                     *fill++ = GAMMA;
00924                     *fill++ = i + FUNHEAD+1;
00925                     FILLFUN(fill)
00926                     *fill++ = oldt[FUNHEAD];
00927                     t = t - i;
00928                     NCOPY(fill,t,i);
00929                 }
00930             }
00931             break;
00932         }
00933     }
00934     m += m[1];
00935 }
00936 /*
00937     #] FUNCTIONS :
00938     #[ VECTORS :
00939 */
00940 else if ( *m == VECTOR ) {
00941     while ( *t > VECTOR ) {
00942         nq = t[1];
00943         NCOPY(fill,t,nq);
00944     }
00945     xstop = t + t[1];
00946     ystop = m + m[1];
00947     t += 2;

```

```

00948     m += 2;
00949     *fill++ = VECTOR;
00950     fill++;
00951     subterm = fill;
00952     do {
00953         if ( *m == *t && m[1] == t[1] ) {
00954             m += 2; t += 2;
00955         }
00956         else if ( *m >= (AM.OffsetVector+WILDOFFSET) ) {
00957             while ( t < xstop ) *fill++ = *t++;
00958             nq = WORDDIF(fill,subterm);
00959             fill = subterm;
00960             if ( m[1] < (AM.OffsetIndex+WILDOFFSET) ) {
00961                 do {
00962                     if ( m[1] == fill[1] &&
00963                         !CheckWild(BHEAD *m-WILDOFFSET,VECTOVEC,*fill,&newval3) )
00964                         break;
00965                     fill += 2;
00966                     nq -= 2;
00967                 } while ( nq > 0 );
00968             }
00969             else { /* Double wildcard */
00970                 do {
00971                     if ( !CheckWild(BHEAD m[1]-WILDOFFSET,INDTOIND,fill[1],&newval3)
00972                         && !CheckWild(BHEAD *m-WILDOFFSET,VECTOVEC,*fill,&newval3) )
00973                         break;
00974                     if ( *fill == oldval1 && fill[1] == AN.oldvalue ) break;
00975                     fill += 2;
00976                     nq -= 2;
00977                 } while ( nq > 0 );
00978             }
00979             nq -= 2;
00980             q = fill + 2;
00981             if ( nq > 0 ) { NCOPY(fill,q,nq); }
00982             m += 2;
00983         }
00984         else if ( *m <= *t &&
00985             m[1] >= (AM.OffsetIndex + WILDOFFSET) ) {
00986             while ( *m == *t && t < xstop )
00987                 { *fill++ = *t++; *fill++ = *t++; }
00988             nq = WORDDIF(fill,subterm);
00989             fill = subterm;
00990             do {
00991                 if ( *m == *fill &&
00992                     !CheckWild(BHEAD m[1]-WILDOFFSET,INDTOIND,fill[1],&newval3) )
00993                     break;
00994                 nq -= 2;
00995                 fill += 2;
00996             } while ( nq > 0 );
00997             nq -= 2;
00998             q = fill + 2;
00999             if ( nq > 0 ) { NCOPY(fill,q,nq); }
01000             m += 2;
01001         }
01002         else { *fill++ = *t++; *fill++ = *t++; }
01003     } while ( m < ystop );
01004     while ( t < xstop ) *fill++ = *t++;
01005     nq = WORDDIF(fill,subterm);
01006     if ( nq > 0 ) {
01007         nq += 2;
01008         subterm[-1] = nq;
01009     }
01010     else { fill = subterm; fill -= 2; }
01011 }
01012 /*
01013     #] VECTORS :
01014     #[ INDICES :
01015
01016     Currently without wildcards
01017 */
01018     else if ( *m == INDEX ) {
01019         while ( *t > INDEX ) {
01020             nq = t[1];
01021             NCOPY(fill,t,nq);
01022         }
01023         xstop = t + t[1];
01024         ystop = m + m[1];
01025         t += 2;
01026         m += 2;
01027         *fill++ = INDEX;
01028         fill++;
01029         subterm = fill;
01030         do {
01031             if ( *m == *t ) {
01032                 m += 1; t += 1;
01033             }
01034             else if ( *m >= (AM.OffsetIndex+WILDOFFSET) ) {

```

```

01035         while ( t < xstop ) *fill++ = *t++;
01036         nq = WORDDIF(fill, subterm);
01037         fill = subterm;
01038         do {
01039             if ( !CheckWild(BHEAD *m-WILDOFFSET,INDTOIND,*fill,&newval3) ) {
01040                 break;
01041             }
01042             fill += 1;
01043             nq -= 1;
01044         } while ( nq > 0 );
01045         nq -= 1;
01046         if ( nq > 0 ) {
01047             q = fill + 1;
01048             NCOPY(fill,q,nq);
01049         }
01050         m += 1;
01051     }
01052     else {
01053         *fill++ = *t++;
01054     }
01055 } while ( m < ystop );
01056 while ( t < xstop ) *fill++ = *t++;
01057 nq = WORDDIF(fill,subterm);
01058 if ( nq > 0 ) {
01059     nq += 2;
01060     subterm[-1] = nq;
01061 }
01062 else { fill = subterm; fill -= 2; }
01063 }
01064 /*
01065     #] INDICES :
01066     #[ DELTAS :
01067 */
01068 else if ( *m == DELTA ) {
01069     while ( *t > DELTA ) {
01070         nq = t[1];
01071         NCOPY(fill,t,nq);
01072     }
01073     xstop = t + t[1];
01074     ystop = m + m[1];
01075     t += 2;
01076     m += 2;
01077     *fill++ = DELTA;
01078     fill++;
01079     subterm = fill;
01080     do {
01081         if ( *t == *m && t[1] == m[1] ) { m += 2; t += 2; }
01082         else if ( *m >= (AM.OffsetIndex+WILDOFFSET) ) { /* Two dummies */
01083             while ( t < xstop ) *fill++ = *t++;
01084             /* fill = subterm; */
01085             oldvall = 1;
01086             goto SubsL6;
01087         }
01088         else if ( m[1] >= (AM.OffsetIndex+WILDOFFSET) ) {
01089             while ( (*m == *t || *m == t[1]) && ( t < xstop ) ) {
01090                 *fill++ = *t++; *fill++ = *t++;
01091             }
01092             oldvall = 0;
01093             SubsL6:
01094             nq = WORDDIF(fill,subterm);
01095             fill = subterm;
01096             do {
01097                 if ( ( oldvall && ( (
01098                     !CheckWild(BHEAD *m-WILDOFFSET,INDTOIND,*fill,&newval3)
01099                     && !CheckWild(BHEAD m[1]-WILDOFFSET,INDTOIND,fill[1],&newval3)
01100                 ) || (
01101                     !CheckWild(BHEAD m[1]-WILDOFFSET,INDTOIND,*fill,&newval3)
01102                     && !CheckWild(BHEAD *m-WILDOFFSET,INDTOIND,fill[1],&newval3)
01103                 ) ) || ( !oldvall && ( (
01104                     *m == *fill
01105                     && !CheckWild(BHEAD m[1]-WILDOFFSET,INDTOIND,fill[1],&newval3)
01106                 ) || (
01107                     *m == fill[1]
01108                     && !CheckWild(BHEAD m[1]-WILDOFFSET,INDTOIND,*fill,&newval3)
01109                 ) ) ) ) break;
01110                 fill += 2;
01111                 nq -= 2;
01112             } while ( nq > 0 );
01113             nq -= 2;
01114             if ( nq > 0 ) {
01115                 q = fill + 2;
01116                 NCOPY(fill,q,nq);
01117             }
01118             m += 2;
01119         }
01120         else {
01121             *fill++ = *t++; *fill++ = *t++;
01122         }

```

```

01122         } while ( m < ystop );
01123         while ( t < xstop ) *fill++ = *t++;
01124         nq = WORDDIF(fill,subterm);
01125         if ( nq > 0 ) {
01126             nq += 2;
01127             subterm[-1] = nq;
01128         }
01129         else { fill = subterm; fill -= 2; }
01130     }
01131     /*
01132         #] DELTAS :
01133     */
01134 EndLoop;;
01135     } while ( m < mstop ); }
01136     while ( t < tstop ) *fill++ = *t++;
01137 SubCoef:
01138     if ( !PutExpr ) {
01139         t = AN.FullProto;
01140         nq = t[1];
01141         t[3] = power;
01142         NCOPY(fill,t,nq);
01143     }
01144     t = tcoef;
01145     nq = ABS(*t);
01146     t = tstop;
01147     NCOPY(fill,t,nq);
01148     nq = WORDDIF(fill,TemTerm);
01149     fill = term;
01150     t = TemTerm;
01151     *fill++ = nq--;
01152     t++;
01153     NCOPY(fill,t,nq);
01154     if ( sign ) {
01155         if ( ( sign & 1 ) != 0 ) fill[-1] = -fill[-1];
01156     }
01157     if ( AT.WorkPointer < fill ) AT.WorkPointer = fill;
01158     AN.RepFunNum = 0;
01159 }
01160
01161 /*
01162     #] Substitute :
01163     #[ FindSpecial :          WORD FindSpecial(term)
01164
01165     Routine to detect simplifications regarding the special functions
01166     exponent, denominator.
01167
01168 WORD FindSpecial(WORD *term)
01169 {
01170     WORD *t;
01171     WORD *tstop;
01172     t = term; t += *t - 1; tstop = t - ABS(*t) + 1; t = term;
01173     t++;
01174     if ( t < tstop ) { do {
01175         if ( *t == EXPONENT ) {
01176             Exponents can become simpler when:
01177             a: the exponent of an expression becomes an integer.
01178             b: The expression becomes zero.
01179         }
01180         else if ( *t == DENOMINATOR ) {
01181             Denominators can become simpler when:
01182             a: The denominator is a single term without functions.
01183             b: An overall coefficient can be removed.
01184             c: An overall object can be removed.
01185             The task is here to bring the denominator in an unique form.
01186         }
01187         t += *t;
01188     } while ( t < tstop ); }
01189     return(0);
01190 }
01191
01192     #] FindSpecial :
01193     #[ FindAll :          WORD FindAll(term,pattern,level,par)
01194 */
01195
01196 WORD FindAll(PHEAD WORD *term, WORD *pattern, WORD level, WORD *par)
01197 {
01198     GETBIDENTITY
01199     WORD *t, *m, *r, *mm, rnum;
01200     WORD *tstop, *mstop, *TwoProto, *vwhere = 0, oldv, oldvv, vv, level2;
01201     WORD v, nq, OffNum = AM.OffsetVector + WILDOFFSET, i, ii = 0, jj;
01202     WORD fromindex, *intens, notflag1 = 0, notflag2 = 0;
01203     CBUF *C;
01204     C = cbuf+AM.rbufnum;
01205     v = pattern[3]; /* The vector to be found */
01206     m = t = term;
01207     m += *m;

```



```

01209     m -= ABS(m[-1]);
01210     t++;
01211     if ( t < m ) do {
01212         tstop = t + t[1];
01213         fromindex = 2;
01214     /*
01215         #[ VECTOR :
01216     */
01217     if ( *t == VECTOR ) {
01218         r = t;
01219         r += 2;
01220 InVect:
01221         while ( r < tstop ) {
01222             oldv = *r;
01223             if ( v >= OffNum ) {
01224                 vwhere = AN.FullProto + 3 + SUBEXPSIZE;
01225                 if ( vwhere[1] == FROMSET || vwhere[1] == SETTONUM ) {
01226                     WORD *afirst, *alast, j;
01227                     j = vwhere[3];
01228                     if ( j > WILDOFFSET ) { j -= 2*WILDOFFSET; notflag1 = 1; }
01229                     else { notflag1 = 0; }
01230                     afirst = SetElements + Sets[j].first;
01231                     alast = SetElements + Sets[j].last;
01232                     ii = 1;
01233                     if ( notflag1 == 0 ) {
01234                         do {
01235                             if ( *afirst == *r ) {
01236                                 if ( vwhere[1] == SETTONUM ) {
01237                                     AN.FullProto[8+SUBEXPSIZE] = SYMTONUM;
01238                                     AN.FullProto[11+SUBEXPSIZE] = ii;
01239                                 }
01240                                 else if ( vwhere[4] >= 0 ) {
01241                                     oldv = *(afirst - Sets[j].first
01242                                         + Sets[vwhere[4]].first);
01243                                 }
01244                                 goto DoVect;
01245                             }
01246                             ii++;
01247                         } while ( ++afirst < alast );
01248                     }
01249                     else {
01250                         do {
01251                             if ( *afirst == *r ) break;
01252                         } while ( ++afirst < alast );
01253                         if ( afirst >= alast ) goto DoVect;
01254                     }
01255                 }
01256                 else goto DoVect;
01257             }
01258             else if ( v == *r ) {
01259 DoVect:
01260                 m = AT.WorkPointer;
01261                 tstop = t;
01262                 t = term;
01263                 mstop = t + *t;
01264                 do { *m++ = *t++; } while ( t < tstop );
01265                 vwhere = m;
01266                 t = AN.FullProto;
01267                 nq = t[1];
01268                 t[3] = 1;
01269                 NCOPY(m,t,nq);
01270                 t = tstop;
01271                 if ( fromindex == 1 ) m[-1] = FUNNYVEC;
01272                 else m[-1] = r[1]; /* The index is always here! */
01273                 if ( v >= OffNum ) vwhere[3+SUBEXPSIZE] = oldv;
01274                 if ( vwhere[1] > 12+SUBEXPSIZE ) {
01275                     vwhere[11+SUBEXPSIZE] = ii;
01276                     vwhere[8+SUBEXPSIZE] = SYMTONUM;
01277                 }
01278                 if ( t[1] > fromindex+2 ) {
01279                     *m++ = *t++;
01280                     *m++ = *t++ - fromindex;
01281                     while ( t < r ) *m++ = *t++;
01282                     t += fromindex;
01283                 }
01284                 else t += t[1];
01285                 do { *m++ = *t++; } while ( t < mstop );
01286                 *AT.WorkPointer = nq = WORDDIF(m,AT.WorkPointer);
01287                 m = AT.WorkPointer;
01288                 t = term;
01289                 NCOPY(t,m,nq);
01290                 AT.WorkPointer = t;
01291                 return(1);
01292             }
01293             r += fromindex;
01294         }
01295     /*

```

```

01296         #] VECTOR :
01297         #[ DOTPRODUCT :
01298     */
01299     else if ( *t == DOTPRODUCT ) {
01300         r = t;
01301         r += 2;
01302         do {
01303             if ( ( i = r[2] ) < 0 ) goto NextDot;
01304             if ( *r == r[1] ) { /* p.p */
01305                 oldv = *r;
01306                 if ( v == *r ) { /* v.v */
01307                     TwoVec:
01308                         m = AT.WorkPointer;
01309                         tstop = t;
01310                         t = term;
01311                         mstop = t + *t;
01312                         do { *m++ = *t++; } while ( t < tstop );
01313                         do {
01314                             vwhere = m;
01315                             t = AN.FullProto;
01316                             nq = t[1];
01317                             t[3] = 2;
01318                             NCOPY(m,t,nq);
01319                             m[-1] = ++AR.CurDum;
01320                             if ( v >= OffNum ) vwhere[3+SUBEXPSIZE] = oldv;
01321                         } while ( --i > 0 );
01322                         CopRest:
01323                         t = tstop;
01324                         if ( t[1] > 5 ) {
01325                             *m++ = *t++;
01326                             *m++ = *t++ - 3;
01327                             while ( t < r ) *m++ = *t++;
01328                             t += 3;
01329                         }
01330                         else t += t[1];
01331                         do { *m++ = *t++; } while ( t < mstop );
01332                         *AT.WorkPointer = nq = WORDDIF(m,AT.WorkPointer);
01333                         m = AT.WorkPointer;
01334                         t = term;
01335                         NCOPY(t,m,nq);
01336                         AT.WorkPointer = t;
01337                         return(1);
01338                     }
01339                 else if ( v >= OffNum ) { /* v?.v? */
01340                     vwhere = AN.FullProto + 3+SUBEXPSIZE;
01341                     if ( vwhere[1] == FROMSET || vwhere[1] == SETTONUM ) {
01342                         WORD *afirst, *alast, j;
01343                         j = vwhere[3];
01344                         if ( j > WILDOFFSET ) { j -= 2*WILDOFFSET; notflag1 = 1; }
01345                         else { notflag1 = 0; }
01346                         afirst = SetElements + Sets[j].first;
01347                         alast = SetElements + Sets[j].last;
01348                         ii = 1;
01349                         if ( notflag1 == 0 ) {
01350                             do {
01351                                 if ( *afirst == *r ) {
01352                                     if ( vwhere[1] == SETTONUM ) {
01353                                         AN.FullProto[8+SUBEXPSIZE] = SYMTONUM;
01354                                         AN.FullProto[11+SUBEXPSIZE] = ii;
01355                                     }
01356                                     else if ( vwhere[4] >= 0 ) {
01357                                         oldv = *(afirst - Sets[j].first
01358                                             + Sets[vwhere[4]].first);
01359                                         goto TwoVec;
01360                                     }
01361                                 }
01362                                 ii++;
01363                             } while ( ++afirst < alast );
01364                         }
01365                         else {
01366                             do {
01367                                 if ( *afirst == *r ) break;
01368                             } while ( ++afirst < alast );
01369                             if ( afirst >= alast ) goto TwoVec;
01370                         }
01371                     }
01372                     else goto TwoVec;
01373                 }
01374             }
01375         }
01376     else {
01377         if ( v == r[1] ) { r[1] = *r; *r = v; }
01378         oldv = *r;
01379         oldvv = r[1];
01380         if ( v == *r ) {
01381             if ( !par ) { while ( ++level <= AR.Cnumlhs
01382                 && C->lhs[level][0] == TYPEIDOLD ) {
01383                 m = C->lhs[level];
01384                 m += IDHEAD;
01385                 if ( m[-IDHEAD+2] == SUBVECTOR ) {

```

```

01383         if ( ( vv = m[m[1]+3] ) == r[1] ) {
01384 OnePV:       TwoProto = AN.FullProto;
01385 TwoPV:       m = AT.WorkPointer;
01386             tstop = t;
01387             t = term;
01388             mstop = t + *t;
01389             do { *m++ = *t++; } while ( t < tstop );
01390             do {
01391                 t = AN.FullProto;
01392                 vwhere = m + 3 + SUBEXPSIZE;
01393                 nq = t[1];
01394                 t[3] = 1;
01395                 NCOPY(m,t,nq);
01396                 m[-1] = ++AR.CurDum;
01397                 if ( v >= OffNum ) *vwhere = oldv;
01398                 if ( vwhere[-2-SUBEXPSIZE] > 12+SUBEXPSIZE ) {
01399                     vwhere[8] = ii;
01400                     vwhere[5] = SYMTONUM;
01401                 }
01402                 t = TwoProto;
01403                 vwhere = m + 3+SUBEXPSIZE;
01404                 mm = m;
01405                 nq = t[1];
01406                 t[3] = 1;
01407                 NCOPY(m,t,nq);
01408 /*
01409 The next two lines repair a bug. without them it takes twice
01410 the rhs of the first vector.
01411 */
01412                 mm[2] = C->lhs[level][IDHEAD+2];
01413                 mm[4] = C->lhs[level][IDHEAD+4];
01414                 m[-1] = AR.CurDum;
01415                 if ( vv >= OffNum ) *vwhere = oldvv;
01416             } while ( --i > 0 );
01417             goto CopRest;
01418         }
01419     else if ( vv > OffNum ) {
01420         vwhere = AN.FullProto + 3+SUBEXPSIZE;
01421         if ( vwhere[1] == FROMSET || vwhere[1] == SETTONUM ) {
01422             WORD *afirst, *alast, j;
01423             j = vwhere[3];
01424             if ( j > WILDOFFSET ) { j -= 2*WILDOFFSET; notflag1 = 1; }
01425             else { notflag1 = 0; }
01426             afirst = SetElements + Sets[j].first;
01427             alast = SetElements + Sets[j].last;
01428             if ( notflag1 == 0 ) {
01429                 ii = 1;
01430                 do {
01431                     if ( *afirst == r[1] ) {
01432                         if ( vwhere[1] == SETTONUM ) {
01433                             AN.FullProto[8+SUBEXPSIZE] = SYMTONUM;
01434                             AN.FullProto[11+SUBEXPSIZE] = ii;
01435                         }
01436                         else if ( vwhere[4] >= 0 ) {
01437                             oldvv = *(afirst - Sets[j].first
01438                                 + Sets[vwhere[4]].first);
01439                         }
01440                         goto OnePV;
01441                     }
01442                     ii++;
01443                     } while ( ++afirst < alast );
01444                 }
01445             else {
01446                 do {
01447                     if ( *afirst == *r ) break;
01448                     } while ( ++afirst < alast );
01449                     if ( afirst >= alast ) goto OnePV;
01450                 }
01451             }
01452             else goto OnePV;
01453         }
01454     }
01455 }}
01456 /*
01457 v.q with v matching and no match for the q, also
01458 not in following idold statements.
01459 Notice that a following q.p? cannot match.
01460 */
01461 rnum = r[1];
01462 OneOnly:   m = AT.WorkPointer;
01463             tstop = t;
01464             t = term;
01465             mstop = t + *t;
01466             do { *m++ = *t++; } while ( t < tstop );
01467             vwhere = m;
01468             t = AN.FullProto;
01469             nq = t[1];

```

```

01470         t[3] = i;
01471         NCOPY(m,t,nq);
01472         m[-4] = INDTOIND;
01473         m[-1] = rnum;
01474         if ( v >= OffNum ) vwhere[3+SUBEXPSIZE] = oldv;
01475         goto CopRest;
01476     }
01477     else if ( v >= OffNum ) {
01478         vwhere = AN.FullProto + 3+SUBEXPSIZE;
01479         if ( vwhere[1] == FROMSET || vwhere[1] == SETTONUM ) {
01480             WORD *afirst, *alast, *bfirst, *blast, j;
01481             j = vwhere[3];
01482             if ( j > WILDOFFSET ) { j -= 2*WILDOFFSET; notflag1 = 1; }
01483             else { notflag1 = 0; }
01484             afirst = SetElements + Sets[j].first;
01485             alast = SetElements + Sets[j].last;
01486             ii = 1;
01487             if ( notflag1 == 0 ) {
01488                 do {
01489                     if ( *afirst == *r ) {
01490                         if ( vwhere[1] == SETTONUM ) {
01491                             AN.FullProto[8+SUBEXPSIZE] = SYMTONUM;
01492                             AN.FullProto[11+SUBEXPSIZE] = ii;
01493                         }
01494                         else if ( vwhere[4] >= 0 ) {
01495                             oldv = *(afirst - Sets[j].first
01496                                 + Sets[vwhere[4]].first);
01497                         }
01498                         level2 = level;
01499                         do {
01500                             if ( !par ) m = C->lhs[level2];
01501                             else m = par;
01502                             m += IDHEAD;
01503                             if ( m[-IDHEAD+2] == SUBVECTOR ) {
01504                                 if ( ( vv = m[m[1]+3] ) == r[1] )
01505                                     goto OnePV;
01506                                 else if ( vv >= OffNum ) {
01507                                     if ( m[SUBEXPSIZE+4] != FROMSET &&
01508                                         m[SUBEXPSIZE+4] != SETTONUM ) goto OnePV;
01509                                     j = m[SUBEXPSIZE+6];
01510                                     if ( j > WILDOFFSET ) { j -= 2*WILDOFFSET; notflag2 = 1; }
01511                                     else { notflag2 = 0; }
01512                                     bfirst = SetElements + Sets[j].first;
01513                                     blast = SetElements + Sets[j].last;
01514                                     jj = 1;
01515                                     if ( notflag2 == 0 ) {
01516                                         do {
01517                                             if ( *bfirst == r[1] ) {
01518                                                 if ( m[SUBEXPSIZE+4] == SETTONUM ) {
01519                                                     m[SUBEXPSIZE+8] = SYMTONUM;
01520                                                     m[SUBEXPSIZE+11] = jj;
01521                                                 }
01522                                                 else if ( m[SUBEXPSIZE+7] >= 0 ) {
01523                                                     oldvv = *(bfirst - Sets[j].first
01524                                                         + Sets[m[SUBEXPSIZE+7]].first);
01525                                                 }
01526                                                 goto OnePV;
01527                                             }
01528                                             jj++;
01529                                         } while ( ++bfirst < blast );
01530                                     }
01531                                     else {
01532                                         do {
01533                                             if ( *bfirst == r[1] ) break;
01534                                         } while ( ++bfirst < blast );
01535                                         if ( bfirst >= blast ) goto OnePV;
01536                                     }
01537                                 }
01538                             } while ( ++level2 < AR.Cnumlhs &&
01539                                 C->lhs[level2][0] == TYPEIDOLD );
01540                             rnum = r[1];
01541                             goto OneOnly;
01542                         }
01543                     }
01544                     else if ( *afirst == r[1] ) {
01545                         if ( vwhere[1] == SETTONUM ) {
01546                             AN.FullProto[8+SUBEXPSIZE] = SYMTONUM;
01547                             AN.FullProto[11+SUBEXPSIZE] = ii;
01548                         }
01549                         else if ( vwhere[4] >= 0 ) {
01550                             oldv = *(afirst - Sets[j].first
01551                                 + Sets[vwhere[4]].first);
01552                         }
01553                         level2 = level;
01554                         while ( ++level2 < AR.Cnumlhs &&
01555                             C->lhs[level2][0] == TYPEIDOLD ) {
01556                             if ( !par ) m = C->lhs[level2];

```

```

01557         else m = par;
01558         m += IDHEAD;
01559         if ( m[-IDHEAD+2] == SUBVECTOR ) {
01560             if ( ( vv = m[6] ) == *r )
01561                 goto OnePV;
01562             else if ( vv >= OffNum ) {
01563                 if ( m[SUBEXPSIZE+4] != FROMSET && m[SUBEXPSIZE+4]
01564                     != SETTONUM ) {
01565                     j = *r;
01566                     *r = r[1];
01567                     r[1] = j;
01568                     goto OnePV;
01569                 }
01570                 j = m[SUBEXPSIZE+6];
01571                 bfirst = SetElements + Sets[j].first;
01572                 blast = SetElements + Sets[j].last;
01573                 jj = 1;
01574                 do {
01575                     if ( *bfirst == *r ) {
01576                         if ( m[SUBEXPSIZE+4] == SETTONUM ) {
01577                             m[SUBEXPSIZE+8] = SYMTONUM;
01578                             m[SUBEXPSIZE+11] = jj;
01579                         }
01580                         else if ( m[SUBEXPSIZE+7] >= 0 ) {
01581                             oldvv = *(bfirst + Sets[j].first
01582                                 + Sets[m[SUBEXPSIZE+7]].first);
01583                         }
01584                         j = *r;
01585                         *r = r[1];
01586                         r[1] = j;
01587                         j = oldv; oldv = oldvv; oldvv = j;
01588                         goto OnePV;
01589                     }
01590                     jj++;
01591                 } while ( ++bfirst < blast );
01592             }
01593         }
01594     }
01595     jj = *r; *r = r[1]; r[1] = jj;
01596     jj = oldv; oldv = oldvv; oldvv = j;
01597     rnum = r[1];
01598     goto OneOnly;
01599 }
01600 ii++;
01601 } while ( ++afirst < alast );
01602 }
01603 else {
01604     do {
01605         if ( *afirst == *r ) break;
01606     } while ( ++afirst < alast );
01607     if ( afirst >= alast ) goto Hitlevel1;
01608     do {
01609         if ( *afirst == r[1] ) break;
01610     } while ( ++afirst < alast );
01611     if ( afirst >= alast ) goto Hitlevel2;
01612 }
01613 }
01614 else { /* Matches twice */
01615     vv = v;
01616     TwoProto = AN.FullProto;
01617     goto TwoPV;
01618 }
01619 }
01620 }
01621 NextDot:    r += 3;
01622 } while ( r < tstop );
01623 }
01624 /*
01625     #] DOTPRODUCT :
01626     #[ LEVICIVITA :
01627 */
01628 else if ( *t == LEVICIVITA ) {
01629     intens = 0;
01630     r = t;
01631     r += FUNHEAD;
01632 OneVect:;
01633     while ( r < tstop ) {
01634         oldv = *r;
01635         if ( v >= OffNum && *r < -10 ) {
01636             vwhere = AN.FullProto + 3+SUBEXPSIZE;
01637             if ( vwhere[1] == FROMSET || vwhere[1] == SETTONUM ) {
01638                 WORD *afirst, *alast, j;
01639                 j = vwhere[3];
01640                 if ( j > WILDOFFSET ) { j -= 2*WILDOFFSET; notflag1 = 1; }
01641                 else { notflag1 = 0; }
01642                 afirst = SetElements + Sets[j].first;
01643                 alast = SetElements + Sets[j].last;

```

```

01644             ii = 1;
01645             if ( notflag1 == 0 ) {
01646                 do {
01647                     if ( *afirst == *r ) {
01648                         if ( vwhere[1] == SETTONUM ) {
01649                             AN.FullProto[8+SUBEXPSIZE] = SYMTONUM;
01650                             AN.FullProto[11+SUBEXPSIZE] = ii;
01651                         }
01652                         else if ( vwhere[4] >= 0 ) {
01653                             oldv = *(afirst - Sets[j].first
01654                                 + Sets[vwhere[4]].first);
01655                         }
01656                         goto DoVect;
01657                     }
01658                     ii++;
01659                 } while ( ++afirst < alast );
01660             }
01661             else {
01662                 do {
01663                     if ( *afirst == *r ) break;
01664                 } while ( ++afirst < alast );
01665                 if ( afirst >= alast ) goto DoVect;
01666             }
01667         }
01668         else goto LeVect;
01669     }
01670     else if ( v == *r ) {
01671 LeVect:
01672         m = AT.WorkPointer;
01673         mstop = term + *term;
01674         t = term;
01675         *r = ++AR.CurDum;
01676         if ( intens ) *intens = DIRTYSYMFLAG;
01677         do { *m++ = *t++; } while ( t < tstop );
01678         t = AN.FullProto;
01679         nq = t[1];
01680         t[3] = 1;
01681         if ( v >= OffNum ) *vwhere = oldv;
01682         NCOPY(m,t,nq);
01683         m[-1] = AR.CurDum;
01684         t = tstop;
01685         do { *m++ = *t++; } while ( t < mstop );
01686         *AT.WorkPointer = nq = WORDDIF(m,AT.WorkPointer);
01687         m = AT.WorkPointer;
01688         t = term;
01689         NCOPY(t,m,nq);
01690         AT.WorkPointer = t;
01691         return(1);
01692     }
01693     r++;
01694 }
01695 /*
01696     #] LEVICIVITA :
01697     #[ GAMMA :
01698 */
01699     else if ( *t == GAMMA ) {
01700         intens = 0;
01701         r = t;
01702         r += FUNHEAD+1;
01703         if ( r < tstop ) goto OneVect;
01704     }
01705 /*
01706     #] GAMMA :
01707     #[ INDEX :
01708 */
01709     else if ( *t == INDEX ) { /* The 'forgotten' part */
01710         r = t;
01711         r += 2;
01712         fromindex = 1;
01713         goto InVect;
01714     }
01715 /*
01716     #] INDEX :
01717     #[ FUNCTION :
01718 */
01719     else if ( *t >= FUNCTION ) {
01720         if ( *t >= FUNCTION
01721             && functions[*t-FUNCTION].spec >= TENSORFUNCTION
01722             && t[1] > FUNHEAD ) {
01723 /*
01724             Tensors are linear in their vectors!
01725 */
01726             r = t;
01727             r += FUNHEAD;
01728             intens = t+2;
01729             goto OneVect;
01730         }

```

```

01731     }
01732     /*
01733         #] FUNCTION :
01734     */
01735     t += t[1];
01736     } while ( t < m );
01737     return(0);
01738 }
01739
01740 /*
01741     #] FindAll :
01742     #[ TestSelect :
01743
01744     Returns 1 if any of the objects in any of the sets in setp
01745     occur anywhere in the term
01746 */
01747
01748 int TestSelect(WORD *term, WORD *setp)
01749 {
01750     WORD *tstop, *t, *s, *el, *elstop, *termstop, *tt, n, ns;
01751     GETSTOP(term,tstop);
01752     term += 1;
01753     while ( term < tstop ) {
01754         switch ( *term ) {
01755             case SYMBOL:
01756                 n = term[1] - 2;
01757                 t = term + 2;
01758                 while ( n > 0 ) {
01759                     ns = setp[1] - 2;
01760                     s = setp + 2;
01761                     while ( --ns >= 0 ) {
01762                         if ( Sets[*s].type != CSYMBOL ) { s++; continue; }
01763                         el = SetElements + Sets[*s].first;
01764                         elstop = SetElements + Sets[*s].last;
01765                         while ( el < elstop ) {
01766                             if ( *el++ == *t ) return(1);
01767                         }
01768                         s++;
01769                     }
01770                     n -= 2;
01771                     t += 2;
01772                 }
01773                 break;
01774             case VECTOR:
01775                 n = term[1] - 2;
01776                 t = term + 2;
01777                 while ( n > 0 ) {
01778                     ns = setp[1] - 2;
01779                     s = setp + 2;
01780                     while ( --ns >= 0 ) {
01781                         if ( Sets[*s].type != CVECTOR ) { s++; continue; }
01782                         el = SetElements + Sets[*s].first;
01783                         elstop = SetElements + Sets[*s].last;
01784                         while ( el < elstop ) {
01785                             if ( *el++ == *t ) return(1);
01786                         }
01787                         s++;
01788                     }
01789                     t++;
01790                     ns = setp[1] - 2;
01791                     s = setp + 2;
01792                     while ( --ns >= 0 ) {
01793                         if ( Sets[*s].type != CINDEX
01794                             && Sets[*s].type != CNUMBER ) { s++; continue; }
01795                         el = SetElements + Sets[*s].first;
01796                         elstop = SetElements + Sets[*s].last;
01797                         while ( el < elstop ) {
01798                             if ( *el++ == *t ) return(1);
01799                         }
01800                         s++;
01801                     }
01802                     n -= 2;
01803                     t++;
01804                 }
01805                 break;
01806             case INDEX:
01807                 n = term[1] - 2;
01808                 t = term + 2;
01809                 goto dotensor;
01810             case DOTPRODUCT:
01811                 n = term[1] - 2;
01812                 t = term + 2;
01813                 while ( n > 0 ) {
01814                     ns = setp[1] - 2;
01815                     s = setp + 2;
01816                     while ( --ns >= 0 ) {
01817                         if ( Sets[*s].type != CVECTOR ) { s++; continue; }

```

```

01818         el = SetElements + Sets[*s].first;
01819         elstop = SetElements + Sets[*s].last;
01820         while ( el < elstop ) {
01821             if ( *el++ == *t ) return(1);
01822         }
01823         s++;
01824     }
01825     t++;
01826     ns = setp[1] - 2;
01827     s = setp + 2;
01828     while ( --ns >= 0 ) {
01829         if ( Sets[*s].type != CVECTOR ) { s++; continue; }
01830         el = SetElements + Sets[*s].first;
01831         elstop = SetElements + Sets[*s].last;
01832         while ( el < elstop ) {
01833             if ( *el++ == *t ) return(1);
01834         }
01835         s++;
01836     }
01837     n -= 3;
01838     t += 2;
01839 }
01840 break;
01841 case DELTA:
01842     n = term[1] - 2;
01843     t = term + 2;
01844     goto dotensor;
01845 default:
01846     if ( *term < FUNCTION ) break;
01847     ns = setp[1] - 2;
01848     s = setp + 2;
01849     while ( --ns >= 0 ) {
01850         if ( Sets[*s].type != CFUNCTION ) { s++; continue; }
01851         el = SetElements + Sets[*s].first;
01852         elstop = SetElements + Sets[*s].last;
01853         while ( el < elstop ) {
01854             if ( *el++ == *term ) return(1);
01855         }
01856         s++;
01857     }
01858     if ( functions[*term-FUNCTION].spec > 0 ) {
01859         n = term[1] - FUNHEAD;
01860         t = term + FUNHEAD;
01861 dotensor:
01862         while ( n > 0 ) {
01863             ns = setp[1] - 2;
01864             s = setp + 2;
01865             while ( --ns >= 0 ) {
01866                 if ( *t < MINSPEC ) {
01867                     if ( Sets[*s].type != CVECTOR ) { s++; continue; }
01868                 }
01869                 else if ( *t >= 0 ) {
01870                     if ( Sets[*s].type != CINDEX
01871                         && Sets[*s].type != CNUMBER ) { s++; continue; }
01872                 }
01873                 else { s++; continue; }
01874                 el = SetElements + Sets[*s].first;
01875                 elstop = SetElements + Sets[*s].last;
01876                 while ( el < elstop ) {
01877                     if ( *el++ == *t ) return(1);
01878                 }
01879                 s++;
01880             }
01881             t++;
01882             n--;
01883         }
01884     }
01885     else {
01886         termstop = term + term[1];
01887         tt = term + FUNHEAD;
01888         while ( tt < termstop ) {
01889             if ( *tt < 0 ) {
01890                 if ( *tt == -SYMBOL ) {
01891                     ns = setp[1] - 2;
01892                     s = setp + 2;
01893                     while ( --ns >= 0 ) {
01894                         if ( Sets[*s].type != CSYMBOL ) { s++; continue; }
01895                         el = SetElements + Sets[*s].first;
01896                         elstop = SetElements + Sets[*s].last;
01897                         while ( el < elstop ) {
01898                             if ( *el++ == tt[1] ) return(1);
01899                         }
01900                         s++;
01901                     }
01902                     tt += 2;
01903                 }
01904                 else if ( *tt == -VECTOR || *tt == -MINVECTOR ) {

```



```

01905         ns = setp[1] - 2;
01906         s = setp + 2;
01907         while ( --ns >= 0 ) {
01908             if ( Sets[*s].type != CVECTOR ) { s++; continue; }
01909             el = SetElements + Sets[*s].first;
01910             elstop = SetElements + Sets[*s].last;
01911             while ( el < elstop ) {
01912                 if ( *el++ == tt[1] ) return(1);
01913             }
01914             s++;
01915         }
01916         tt += 2;
01917     }
01918     else if ( *tt == -INDEX ) {
01919         ns = setp[1] - 2;
01920         s = setp + 2;
01921         while ( --ns >= 0 ) {
01922             if ( Sets[*s].type != CINDE
01923                 && Sets[*s].type != CNUMBER ) { s++; continue; }
01924             el = SetElements + Sets[*s].first;
01925             elstop = SetElements + Sets[*s].last;
01926             while ( el < elstop ) {
01927                 if ( *el++ == tt[1] ) return(1);
01928             }
01929             s++;
01930         }
01931         tt += 2;
01932     }
01933     else if ( *tt <= -FUNCTION ) {
01934         ns = setp[1] - 2;
01935         s = setp + 2;
01936         while ( --ns >= 0 ) {
01937             if ( Sets[*s].type != CFUNCTION ) { s++; continue; }
01938             el = SetElements + Sets[*s].first;
01939             elstop = SetElements + Sets[*s].last;
01940             while ( el < elstop ) {
01941                 if ( *el++ == -( *tt ) ) return(1);
01942             }
01943             s++;
01944         }
01945         tt++;
01946     }
01947     else tt += 2;
01948 }
01949 else {
01950     t = tt + ARGHEAD;
01951     tt += *tt;
01952     while ( t < tt ) {
01953         if ( TestSelect(t,setp) ) return(1);
01954         t += *t;
01955     }
01956 }
01957 }
01958 }
01959 break;
01960 }
01961 term += term[1];
01962 }
01963 return(0);
01964 }
01965
01966 /*
01967     #] TestSelect :
01968     #[ SubsInAll :      void SubsInAll()
01969
01970     This routine takes a match in id,all and stores it away in
01971     the AT.allbufnum 'compiler' buffer, after taking out the pattern.
01972     The main problem here is that id,all usually has (lots of) wildcards
01973     and their assignments are on stack and the difficult ones are in
01974     AT.ebufnum. Popping the stack while looking for more matches would
01975     loose those. Hence we have to copy them into yet another compiler
01976     buffer: AT.aebufnum. Because this may involve many matches and
01977     because the original term has only a limited number of arguments,
01978     it will pay to look for already existing ones in this buffer.
01979     (to be done later).
01980 */
01981
01982 void SubsInAll(PHEAD0)
01983 {
01984     GETBIDENTITY
01985     WORD *TemTerm;
01986     WORD *t, *m, *term;
01987     WORD *tstop, *mstop, *xstop;
01988     WORD nt, *fill, nq, mt;
01989     WORD *tcoef, i = 0;
01990     WORD PutExpr = 0, sign = 0;
01991 }

```

```

01992     We start with building the term in the Workspace.
01993     Afterwards we will transfer it to AT.allbufnum.
01994     We have to make sure there is room in the Workspace.
01995 */
01996     AT.idallflag = 2;
01997     TemTerm = AT.WorkPointer;
01998     if ( ( (WORD *)(((UBYTE *) (AT.WorkPointer)) + AM.MaxTer*2) ) > AT.WorkTop ) {
01999         MLOCK(ErrorMessageLock);
02000         MesWork();
02001         MUNLOCK(ErrorMessageLock);
02002         Terminate(-1);
02003     }
02004     m = AN.patternbuffer + IDHEAD; m += m[1];
02005     mstop = m + *m;
02006     m++;
02007     term = AN.termbuffer;
02008     tstop = term + *term; tcoef = tstop-1; tstop -= ABS(tstop[-1]);
02009     t = term;
02010     t++;
02011     fill = TemTerm;
02012     fill++;
02013     while ( m < mstop ) {
02014         while ( t < tstop ) {
02015             nt = WORDDIF(t,term);
02016             for ( mt = 0; mt < AN.RepFunNum; mt += 2 ) {
02017                 if ( nt == AN.RepFunList[mt] ) break;
02018             }
02019             if ( mt >= AN.RepFunNum ) {
02020                 nq = t[1];
02021                 NCOPY(fill,t,nq);
02022             }
02023             else {
02024                 WORD *oldt = 0;
02025                 if ( *m == GAMMA && m[1] != FUNHEAD+1 ) {
02026                     oldt = t;
02027                     if ( ( i = AN.RepFunList[mt+1] ) > 0 ) {
02028                         *fill++ = GAMMA;
02029                         *fill++ = i + FUNHEAD+1;
02030                         FILLFUN(fill)
02031                         nq = i + 1;
02032                         t += FUNHEAD;
02033                         NCOPY(fill,t,nq);
02034                     }
02035                     t = oldt;
02036                 }
02037                 else if ( ( *t == LEVICIVITA ) || ( *t >= FUNCTION
02038                     && (functions[*t-FUNCTION].symmetric & ~REVERSEORDER) == ANTISYMMETRIC )
02039                     ) sign += AN.RepFunList[mt+1];
02040                 else if ( *m >= FUNCTION+WILDOFFSET
02041                     && (functions[*m-FUNCTION-WILDOFFSET].symmetric & ~REVERSEORDER) == ANTISYMMETRIC
02042                     ) sign += AN.RepFunList[mt+1];
02043                 if ( !PutExpr ) {
02044                     WORD *pstart = fill, *p, *w, *ww;
02045                     xstop = t + t[1];
02046                     t = AN.FullProto;
02047                     nq = t[1];
02048                     t[3] = 1;
02049                     NCOPY(fill,t,nq);
02050                     t = xstop;
02051                     PutExpr = 1;
02052                 /*
02053                     Here we need provisions for keeping wildcard matches
02054                     that reside in AT.ebufnum. We will move them to
02055                     AT.aebufnum.
02056                     Problem: the SUBEXPRESSION assumes automatically
02057                     that the compiler buffer is AT.ebufnum. We have to
02058                     correct that in TransferBuffer.
02059                 */
02060                 p = pstart + SUBEXPSIZE;
02061                 while ( p < fill ) {
02062                     switch ( *p ) {
02063                         case SYMTOSUB:
02064                         case VECTOSUB:
02065                         case INDTOSUB:
02066                         case ARGTOARG:
02067                         case ARLTOARL:
02068                             w = cbuf[AT.ebufnum].rhs[p[3]];
02069                             ww = cbuf[AT.ebufnum].rhs[p[3]+1];
02070                 /*
02071                     Here we could search for whether this
02072                     object sits in the buffer already.
02073                     To be done later.
02074                     By the way: ww-w fits inside a WORD.
02075                 */
02076                 AddRHS(AT.aebufnum,1);
02077                 AddNtoC(AT.aebufnum,ww-w,w,11);
02078                 p[3] = cbuf[AT.aebufnum].numrhs;

```

```

02079             cbuf[AT.aebufnum].rhs[p[3]+1] = cbuf[AT.aebufnum].Pointer;
02080             p += p[1];
02081             break;
02082             case FROMSET:
02083             case SETTONUM:
02084             case LOADDOLLAR:
02085                 p += p[1];
02086                 break;
02087             default:
02088                 p += p[1];
02089                 break;
02090         }
02091     }
02092 }
02093 }
02094 else t += t[1];
02095 if ( *m == GAMMA && m[1] != FUNHEAD+1 ) {
02096     i = oldt[1] - m[1] - i;
02097     if ( i > 0 ) {
02098         *fill++ = GAMMA;
02099         *fill++ = i + FUNHEAD+1;
02100         FILLFUN(fill)
02101         *fill++ = oldt[FUNHEAD];
02102         t = t - i;
02103         NCOPY(fill,t,i);
02104     }
02105 }
02106 break;
02107 }
02108 }
02109 m += m[1];
02110 }
02111 while ( t < tstop ) *fill++ = *t++;
02112 if ( !PutExpr ) {
02113     t = AN.FullProto;
02114     nq = t[1];
02115     t[3] = 1;
02116     NCOPY(fill,t,nq);
02117 }
02118 t = tcoef;
02119 nq = ABS(*t);
02120 t = tstop;
02121 NCOPY(fill,t,nq);
02122 if ( sign ) {
02123     if ( ( sign & 1 ) != 0 ) fill[-1] = -fill[-1];
02124 }
02125 *TemTerm = fill-TemTerm;
02126 /*
02127 And now we copy this to AT.allbufnum
02128 */
02129 AddNtoC(AT.allbufnum,TemTerm[0],TemTerm,12);
02130 cbuf[AT.allbufnum].Pointer[0] = 0;
02131 AN.RepFunNum = 0;
02132 }
02133
02134 /*
02135     #] SubsInAll :
02136     #[ TransferBuffer :
02137
02138     Adds the whole content of a (compiler)buffer to another buffer.
02139     In spectator we have an expression in the RHS that needs the
02140     wildcard resolutions adapted by an offset.
02141 */
02142
02143 void TransferBuffer(int from,int to,int spectator)
02144 {
02145     CBUF *C = cbuf + spectator;
02146     CBUF *Cf = cbuf + from;
02147     CBUF *Ct = cbuf + to;
02148     int offset = Ct->numrhs;
02149     LONG i;
02150     WORD *t, *tt, *ttt, *tstop, size;
02151     for ( i = 1; i <= Cf->numrhs; i++ ) {
02152         size = Cf->rhs[i+1]-Cf->rhs[i];
02153         AddRHS(to,1);
02154         AddNtoC(to,size,Cf->rhs[i],13);
02155     }
02156     Ct->rhs[Ct->numrhs+1] = Ct->Pointer;
02157     Cf->numrhs = 0;
02158 /*
02159 Now we have to update the 'pointers' in the spectator.
02160 */
02161 t = C->rhs[C->numrhs];
02162 while ( *t ) {
02163     tt = t+1; t += *t;
02164     tstop = t-ABS(t[-1]);
02165     while ( tt < tstop ) {

```

```

02166         if ( *tt == SUBEXPRESSION ) {
02167             ttt = tt+SUBEXPSIZE; tt += tt[1];
02168             while ( ttt < tt ) {
02169                 switch ( *ttt ) {
02170                     case SYMTOSUB:
02171                     case VECTOSUB:
02172                     case INDIOSUB:
02173                     case ARGTOARG:
02174                     case ARLTOARL:
02175                         ttt[3] += offset;
02176                         break;
02177                     default:
02178                         break;
02179                 }
02180                 ttt += 4;
02181             }
02182         }
02183         else tt += tt[1];
02184     }
02185 }
02186 }
02187
02188 /*
02189     #] TransferBuffer :
02190     #[ TakeIDfunction :
02191 */
02192
02193 #define PutInBuffers(pow) \
02194     AddrRHS(AT.ebufnum,1); \
02195     *out++ = SUBEXPRESSION; \
02196     *out++ = SUBEXPSIZE; \
02197     *out++ = C->numrhs; \
02198     *out++ = pow; \
02199     *out++ = AT.ebufnum; \
02200     FILLSUB(out) \
02201     r = AT.pWorkspace[rhs+i]; \
02202     if ( *r > 0 ) { \
02203         oldinr = r[*r]; r[*r] = 0; \
02204         AddNtoC(AT.ebufnum, (*r+1-ARGHEAD), (r+ARGHEAD), 14); \
02205         r[*r] = oldinr; \
02206     } \
02207     else { \
02208         ToGeneral(r,buffer,1); \
02209         buffer[buffer[0]] = 0; \
02210         AddNtoC(AT.ebufnum,buffer[0]+1,buffer,15); \
02211     }
02212
02213 int TakeIDfunction(PHEAD WORD *term)
02214 {
02215     WORD *tstop, *t, *r, *m, *f, *nextf, *funstop, *left, *l, *newterm;
02216     WORD *out, oldinr, pow;
02217     WORD buffer[20];
02218     int i, ii, j, numsub, numfound = 0, first;
02219     LONG lhs,rhs;
02220     CBUF *C;
02221     GETSTOP(term,tstop);
02222     for ( t = term+1; t < tstop; t += t[1] ) { if ( *t == IDFUNCTION ) break; }
02223     if ( t >= tstop ) return(0);
02224 /*
02225     Step 1: test validity
02226 */
02227     funstop = t + t[1]; f = t + FUNHEAD;
02228     left = term + *term;
02229     l = left+1; numsub = 0;
02230     while ( f < funstop ) {
02231         nextf = f; NEXTARG(nextf)
02232         if ( nextf >= funstop ) { return(0); } /* odd number of arguments */
02233         if ( *f == -SYMBOL ) { *l++ = SYMBOL; *l++ = 4; *l++ = f[1]; *l++ = 1; }
02234         else if ( *f < -FUNCTION ) { *l++ = *f; *l++ = FUNHEAD; FILLFUN(1) }
02235         else if ( *f > 0 ) {
02236             if ( *f != f[ARGHEAD]+ARGHEAD ) goto noaction;
02237             if ( nextf[-1] != 3 || nextf[-2] != 1 || nextf[-3] != 1 ) goto noaction;
02238             if ( f[ARGHEAD] <= 4 ) goto noaction;
02239             if ( f[ARGHEAD] != f[ARGHEAD+2]+4 ) goto noaction;
02240             if ( f[ARGHEAD] == 8 && f[ARGHEAD+1] == SYMBOL ) {
02241                 for ( i = 0; i < 4; i++ ) *l++ = f[ARGHEAD+1+i];
02242             }
02243             else if ( f[ARGHEAD] == 9 && f[ARGHEAD+1] == DOTPRODUCT ) {
02244                 for ( i = 0; i < 5; i++ ) *l++ = f[ARGHEAD+1+i];
02245             }
02246             else if ( f[ARGHEAD+1] >= FUNCTION ) {
02247                 for ( i = 0; i < f[ARGHEAD+1]-4; i++ ) *l++ = f[ARGHEAD+1+i];
02248             }
02249             else goto noaction;
02250         }
02251         else goto noaction;
02252         numsub++;

```

```

02253         f = nextf;
02254         NEXTARG(f)
02255     }
02256     C = cbuf+AT.ebufnum;
02257     AT.WorkPointer = l;
02258     *left = l-left;
02259 /*
02260     Put the pointers to the lhs and the rhs in the pointer workspace
02261 */
02262     WantAddPointers(2*numsub);
02263     lhs = AT.pWorkPointer;
02264     rhs = lhs+numsub;
02265     AT.pWorkPointer = rhs+numsub;
02266     f = t + FUNHEAD; l = left+1;
02267     for ( i = 0; i < numsub; i++ ) {
02268         AT.pWorkSpace[lhs+i] = l; l += l[1];
02269         NEXTARG(f);
02270         AT.pWorkSpace[rhs+i] = f;
02271         NEXTARG(f);
02272     }
02273 /*
02274     Take out the patterns and replace them by SUBEXPRESSIONs pointing at
02275     the e buffer. We put the resulting term above the left sides.
02276     Note that we take out only the first id_ if there is more than one!
02277 */
02278     first = 1;
02279     t = term+1; newterm = AT.WorkPointer; out = newterm+1;
02280     while ( t < tstop ) {
02281         if ( *t == IDFUNCTION && first ) { first = 0; t += t[1]; continue; }
02282         if ( *t >= FUNCTION ) {
02283             for ( i = 0; i < numsub; i++ ) {
02284                 m = AT.pWorkSpace[lhs+i];
02285                 if ( *m != *t ) continue;
02286                 for ( j = 1; j < t[1]; j++ ) {
02287                     if ( m[j] != t[j] ) break;
02288                 }
02289                 if ( j != t[1] ) continue;
02290                 numfound++;
02291             }
02292             /*
02293                 We have a match! Set up a SUBEXPRESSION subterm and put the
02294                 corresponding rhs in the eBuffer.
02295             */
02296             PutInBuffers(1)
02297             t += t[1];
02298             if ( i == numsub ) { /* no match. Just copy to output. */
02299                 j = t[1]; NCOPY(out,t,j)
02300             }
02301         }
02302         else if ( *t == SYMBOL ) {
02303             for ( i = 0; i < numsub; i++ ) {
02304                 m = AT.pWorkSpace[lhs+i];
02305                 if ( *m != SYMBOL ) continue;
02306                 for ( ii = 2; ii < t[1]; ii += 2 ) {
02307                     if ( m[2] != t[ii] ) continue;
02308                     pow = t[ii+1]/m[3];
02309                     if ( pow <= 0 ) continue;
02310                     t[ii+1] = t[ii+1]*m[3];
02311                     numfound++;
02312                 }
02313                 /*
02314                     Create the proper rhs in the eBuffer and set up a
02315                     SUBEXPRESSION subterm.
02316                 */
02317                 PutInBuffers(pow)
02318             }
02319             /*
02320                 Now we copy whatever remains of the SYMBOL subterm to the output
02321             */
02322             m = out; *out++ = t[0]; *out++ = t[1];
02323             for ( ii = 2; ii < t[1]; ii += 2 ) {
02324                 if ( t[ii+1] ) { *out++ = t[ii]; *out++ = t[ii+1]; }
02325             }
02326             m[1] = out-m;
02327             if ( m[1] == 2 ) out = m;
02328             t += t[1];
02329         }
02330         else if ( *t == DOTPRODUCT ) {
02331             for ( i = 0; i < numsub; i++ ) {
02332                 m = AT.pWorkSpace[lhs+i];
02333                 if ( *m != DOTPRODUCT ) continue;
02334                 for ( ii = 2; ii < t[1]; ii += 3 ) {
02335                     if ( m[2] != t[ii] || m[3] != t[ii+1] ) continue;
02336                     pow = t[ii+2]/m[4];
02337                     if ( pow <= 0 ) continue;
02338                     t[ii+2] = t[ii+2]*m[4];
02339                     numfound++;

```

```

02340 /*
02341             Create the proper rhs in the eBuffer and set up a
02342             SUBEXPRESSION subterm.
02343 */
02344             PutInBuffers(pow)
02345         }
02346     }
02347 /*
02348             Now we copy whatever remains of the DOTPRODUCT subterm to the output
02349 */
02350     m = out; *out++ = t[0]; *out++ = t[1];
02351     for ( ii = 2; ii < t[1]; ii += 3 ) {
02352         if ( t[ii+2] ) { *out++ = t[ii]; *out++ = t[ii+1]; *out++ = t[ii+2]; }
02353     }
02354     m[1] = out-m;
02355     if ( m[1] == 2 ) out = m;
02356     t += t[1];
02357 }
02358 else {
02359     j = t[1]; NCOPY(out,t,j)
02360 }
02361 }
02362 /*
02363             Copy the coefficient and set the size.
02364 */
02365     t = tstop; r = term+term; while ( t < r ) *out++ = *t++;
02366     *newterm = out-newterm;
02367 /*
02368             Finally we move the new term over the original term.
02369 */
02370     i = *newterm;
02371     t = term; r = newterm; NCOPY(t,r,i)
02372 /*
02373             At this point we can return and if the calling Generator jumps back to
02374             its start, TestSub can take care of the expansions of SUBEXPRESSIONs.
02375 */
02376     AT.pWorkPointer = lhs;
02377     AT.WorkPointer = t;
02378     return(numfound);
02379 noaction:
02380     return(0);
02381 }
02382
02383 /*
02384             #] TakeIDfunction :
02385             #] Patterns :
02386 */
02387

```

4.100 poly.cc

```

00001 /* @file poly.cc
00002 *
00003 * Contains the class for representing sparse multivariate
00004 * polynomials with integer coefficients
00005 */
00006 /* #] License : */
00007 /*
00008 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00009 * When using this file you are requested to refer to the publication
00010 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00011 * This is considered a matter of courtesy as the development was paid
00012 * for by FOM the Dutch physics granting agency and we would like to
00013 * be able to track its scientific use to convince FOM of its value
00014 * for the community.
00015 *
00016 * This file is part of FORM.
00017 *
00018 * FORM is free software: you can redistribute it and/or modify it under the
00019 * terms of the GNU General Public License as published by the Free Software
00020 * Foundation, either version 3 of the License, or (at your option) any later
00021 * version.
00022 *
00023 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00024 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00025 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00026 * details.
00027 *
00028 * You should have received a copy of the GNU General Public License along
00029 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00030 */
00031 /* #] License : */
00032 /*

```

```

00033     #[ includes :
00034 */
00035
00036 #include "poly.h"
00037 #include "polygcd.h"
00038
00039 #include <cassert>
00040 #include <cctype>
00041 #include <cmath>
00042 #include <algorithm>
00043 #include <map>
00044 #include <iostream>
00045
00046 using namespace std;
00047
00048 /*
00049     #] includes :
00050     #[ constructor (small constant polynomial) :
00051 */
00052
00053 // constructor for a small constant polynomial
00054 poly::poly (PHEAD int a, WORD modp, WORD modn):
00055     size_of_terms(AM.MaxTer/(LONG) sizeof(WORD)),
00056     modp(0),
00057     modn(1)
00058 {
00059     POLY_STOREIDENTITY;
00060
00061     terms = TermMalloc("poly::poly(int)");
00062
00063     if (a == 0) {
00064         terms[0] = 1; // length
00065     }
00066     else {
00067         terms[0] = 4 + AN.poly_num_vars; // length
00068         terms[1] = 3 + AN.poly_num_vars; // length
00069         for (int i=0; i<AN.poly_num_vars; i++) terms[2+i] = 0; // powers
00070         terms[2+AN.poly_num_vars] = ABS(a); // coefficient
00071         terms[3+AN.poly_num_vars] = a>0 ? 1 : -1; // length coefficient
00072     }
00073
00074     if (modp!=-1) setmod(modp,modn);
00075 }
00076
00077 /*
00078     #] constructor (small constant polynomial) :
00079     #[ constructor (large constant polynomial) :
00080 */
00081
00082 // constructor for a large constant polynomial
00083 poly::poly (PHEAD const UWORD *a, WORD na, WORD modp, WORD modn):
00084     size_of_terms(AM.MaxTer/(LONG) sizeof(WORD)),
00085     modp(0),
00086     modn(1)
00087 {
00088     POLY_STOREIDENTITY;
00089
00090     terms = TermMalloc("poly::poly(UWORD*,WORD)");
00091
00092     // remove leading zeros
00093     while (*(a+ABS(na)-1)==0)
00094         na -= SGN(na);
00095
00096     terms[0] = 3 + AN.poly_num_vars + ABS(na); // length
00097     terms[1] = terms[0] - 1; // length
00098     for (int i=0; i<AN.poly_num_vars; i++) terms[2+i] = 0; // powers
00099     termscopy((WORD *)a, 2+AN.poly_num_vars, ABS(na)); // coefficient
00100     terms[2+AN.poly_num_vars+ABS(na)] = na; // length coefficient
00101
00102     if (modp!=-1) setmod(modp,modn);
00103 }
00104
00105 /*
00106     #] constructor (large constant polynomial) :
00107     #[ constructor (deep copy polynomial) :
00108 */
00109
00110 // copy constructor: makes a deep copy of a polynomial
00111 poly::poly (const poly &a, WORD modp, WORD modn):
00112     size_of_terms(AM.MaxTer/(LONG) sizeof(WORD))
00113 {
00114     #ifdef POLY_MOVE_DEBUG
00115         std::cout << "poly copy ctor" << std::endl;
00116     #endif
00117     POLY_GETIDENTITY(a);
00118     POLY_STOREIDENTITY;
00119 }

```

```

00120     terms = TermMalloc("poly::poly(poly&)");
00121
00122     *this = a;
00123
00124     if (modp!= -1) setmod(modp,modn);
00125 }
00126
00127 /*
00128  #] constructor (deep copy polynomial) :
00129  #[ destructor :
00130  */
00131
00132 // destructor
00133 poly::~poly () {
00134
00135     POLY_GETIDENTITY(*this);
00136
00137     if (size_of_terms == AM.MaxTer/(LONG)sizeof(WORD))
00138         TermFree(terms, "poly::~poly");
00139     else
00140         M_free(terms, "poly::~poly");
00141 }
00142
00143 /*
00144  #] destructor :
00145  #[ expand_memory :
00146  */
00147
00148 // expands the memory allocated for terms to at least twice its size
00149 void poly::expand_memory (int i) {
00150
00151     POLY_GETIDENTITY(*this);
00152
00153     LONG new_size_of_terms = MaX(2*size_of_terms, i);
00154     if (new_size_of_terms > MAXPOSITIVE) {
00155         MLOCK(ErrorMessageLock);
00156         MesPrint ((char*)"ERROR: polynomials too large (> WORDSIZE)");
00157         MUNLOCK(ErrorMessageLock);
00158         Terminate(1);
00159     }
00160
00161     WORD *newterms = (WORD *)Malloc1(new_size_of_terms * sizeof(WORD), "poly::expand_memory");
00162     WCOPY(newterms, terms, size_of_terms);
00163
00164     if (size_of_terms == AM.MaxTer/(LONG)sizeof(WORD))
00165         TermFree(terms, "poly::expand_memory");
00166     else
00167         M_free(terms, "poly::expand_memory");
00168
00169     terms = newterms;
00170     size_of_terms = new_size_of_terms;
00171 }
00172
00173 /*
00174  #] expand_memory :
00175  #[ setmod :
00176  */
00177
00178 // sets the coefficient space to ZZ/p^n
00179 void poly::setmod(WORD _modp, WORD _modn) {
00180
00181     POLY_GETIDENTITY(*this);
00182
00183     if (_modp>0 && (_modp!=modp || _modn<modn)) {
00184         modp = _modp;
00185         modn = _modn;
00186
00187         WORD nmodq=0;
00188         UWORD *modq=NULL;
00189         bool smallq;
00190
00191         if (modn == 1) {
00192             modq = (UWORD *)&modp;
00193             nmodq = 1;
00194             smallq = true;
00195         }
00196         else {
00197             RaisPowCached(BHEAD modp,modn,&modq,&nmodq);
00198             smallq = false;
00199         }
00200
00201         coefficients_modulo(modq,nmodq,smallq);
00202     }
00203     else {
00204         modp = _modp;
00205         modn = _modn;
00206     }

```



```

00207 }
00208
00209 /*
00210  #] setmod :
00211  #[ coefficients_modulo :
00212 */
00213
00214 // reduces all coefficients of the polynomial modulo a
00215 void poly::coefficients_modulo (UWORD *a, WORD na, bool small) {
00216     POLY_GETIDENTITY(*this);
00217
00218     int j=1;
00219     for (int i=1, di; i<terms[0]; i+=di) {
00220         di = terms[i];
00221
00222         if (i!=j)
00223             for (int k=0; k<di; k++)
00224                 terms[j+k] = terms[i+k];
00225
00226         WORD n = terms[j+terms[j]-1];
00227
00228         if (ABS(n)==1 && small) {
00229             // small number modulo small number
00230             terms[j+1+AN.poly_num_vars] = n*((UWORD)terms[j+1+AN.poly_num_vars] % a[0]);
00231             if (terms[j+1+AN.poly_num_vars] == 0)
00232                 n=0;
00233             else {
00234                 if (terms[j+1+AN.poly_num_vars] > +(WORD)a[0]/2) terms[j+1+AN.poly_num_vars]-=a[0];
00235                 if (terms[j+1+AN.poly_num_vars] < -(WORD)a[0]/2) terms[j+1+AN.poly_num_vars]+=a[0];
00236                 n = SGN(terms[j+1+AN.poly_num_vars]);
00237                 terms[j+1+AN.poly_num_vars] = ABS(terms[j+1+AN.poly_num_vars]);
00238             }
00239         }
00240         else
00241             TakeNormalModulus((UWORD *)&terms[j+1+AN.poly_num_vars], &n, a, na, NOUNPACK);
00242
00243         if (n!=0) {
00244             terms[j] = 2+AN.poly_num_vars+ABS(n);
00245             terms[j+terms[j]-1] = n;
00246             j += terms[j];
00247         }
00248     }
00249     terms[0] = j;
00250 }
00251
00252 }
00253
00254 /*
00255  #] coefficients_modulo :
00256  #[ to_string :
00257 */
00258
00259 // converts an integer to a string
00260 const string int_to_string (WORD x) {
00261     char res[41];
00262     snprintf (res, 41, "%i",x);
00263     return res;
00264 }
00265
00266 // converts a polynomial to a string
00267 const string poly::to_string() const {
00268     POLY_GETIDENTITY(*this);
00269
00270     string res;
00271
00272     int printtimes;
00273     UBYTE *scoeff = (UBYTE *)NumberMalloc("poly::to_string");
00274
00275     if (terms[0]==1)
00276         // zero
00277         res = "0";
00278     else {
00279         for (int i=1; i<terms[0]; i+=terms[i]) {
00280             // sign
00281             WORD ncoeff = terms[i+terms[i]-1];
00282             if (ncoeff < 0) {
00283                 ncoeff*=-1;
00284                 res += "-";
00285             }
00286             else {
00287                 if (i>1) res += "+";
00288             }
00289
00290             if (ncoeff==1 && terms[i+terms[i]-1-ncoeff]==1) {
00291                 // coeff=1, so don't print coefficient and '*'

```

```

00294         printtimes = 0;
00295     }
00296     else {
00297         // print coefficient
00298         PRTLong((UWORD*)&terms[i+terms[i]-1-ncoeff], ncoeff, scoeff);
00299         res += string((char *)scoeff);
00300         printtimes=1;
00301     }
00302
00303     // print variables
00304     for (int j=0; j<AN.poly_num_vars; j++) {
00305         if (terms[i+1+j] > 0) {
00306             if (printtimes) res += "*";
00307             res += string(1,'a'+j);
00308             if (terms[i+1+j] > 1) res += "^" + int_to_string(terms[i+1+j]);
00309             printtimes = 1;
00310         }
00311     }
00312
00313     // iff coeff=1 and all power=0, print '1' after all
00314     if (!printtimes) res += "1";
00315 }
00316 }
00317
00318 // eventual modulo
00319 if (modp>0) {
00320     res += " (mod ";
00321     res += int_to_string(modp);
00322     if (modn>1) {
00323         res += "^";
00324         res += int_to_string(modn);
00325     }
00326     res += ")";
00327 }
00328
00329 NumberFree(scoeff,"poly::to_string");
00330
00331 return res;
00332 }
00333
00334 /*
00335 #] to_string :
00336 #[ ostream operator :
00337 */
00338
00339 // output stream operator
00340 ostream& operator<< (ostream &out, const poly &a) {
00341     return out << a.to_string();
00342 }
00343
00344 /*
00345 #] ostream operator :
00346 #[ monomial_compare :
00347 */
00348
00349 // compares two monomials (0:equal, <0:a smaller, >0:b smaller)
00350 int poly::monomial_compare (PHEAD const WORD *a, const WORD *b) {
00351     for (int i=0; i<AN.poly_num_vars; i++)
00352         if (a[i+1]!=b[i+1]) return a[i+1]-b[i+1];
00353     return 0;
00354 }
00355
00356 /*
00357 #] monomial_compare :
00358 #[ normalize :
00359 */
00360
00361 // brings a polynomial to normal form
00362 const poly & poly::normalize() {
00363
00364     POLY_GETIDENTITY(*this);
00365
00366     WORD nmodq=0;
00367     UWORD *modq=NULL;
00368
00369     if (modp!=0) {
00370         if (modn == 1) {
00371             modq = (UWORD *)&modp;
00372             nmodq = 1;
00373         }
00374         else {
00375             RaisPowCached(BHEAD modp, modn, &modq, &nmodq);
00376         }
00377     }
00378
00379     // find and sort all monomials
00380     // terms[0]/num_vars+3 is an upper bound for number of terms in a

```

```

00381
00382     LONG poffset = AT.pWorkPointer;
00383     WantAddPointers((terms[0]/(AN.poly_num_vars+3)));
00384     AT.pWorkPointer += terms[0]/(AN.poly_num_vars+3);
00385     #define p (&(AT.pWorkspace[poffset]))
00386
00387 // WORD **p = (WORD **)Malloc1(terms[0]/(AN.poly_num_vars+3) * sizeof(WORD*), "poly::normalize");
00388
00389     int nterms = 0;
00390     for (int i=1; i<terms[0]; i+=terms[i])
00391         p[nterms++] = terms + i;
00392
00393     sort(p, p + nterms, monomial_larger(BHEAD0));
00394
00395     WORD *tmp;
00396     if (size_of_terms == AM.MaxTer/(LONG)sizeof(WORD)) {
00397         tmp = (WORD *)TermMalloc("poly::normalize");
00398     }
00399     else {
00400         tmp = (WORD *)Malloc1(size_of_terms * sizeof(WORD), "poly::normalize");
00401     }
00402     int j=1;
00403     int prevj=0;
00404     tmp[0]=0;
00405     tmp[1]=0;
00406
00407     for (int i=0; i<nterms; i++) {
00408         if (i>0 && monomial_compare(BHEAD &tmp[j], p[i])==0) {
00409             // duplicate term, so add coefficients
00410             WORD ncoeff = tmp[j+tmp[j]-1];
00411             AddLong((UWORD *)&tmp[j+1+AN.poly_num_vars], ncoeff,
00412                 (UWORD *)&p[i][1+AN.poly_num_vars], p[i][p[i][0]-1],
00413                 (UWORD *)&tmp[j+1+AN.poly_num_vars], &ncoeff);
00414
00415             tmp[j+1+AN.poly_num_vars+ABS(ncoeff)] = ncoeff;
00416             tmp[j] = 2+AN.poly_num_vars+ABS(ncoeff);
00417         }
00418         else {
00419             // new term
00420             prevj = j;
00421             j += tmp[j];
00422             WCOPY(&tmp[j],p[i],p[i][0]);
00423         }
00424
00425         if (modp!=0) {
00426             // bring coefficient to normal form mod p^n
00427             WORD ntmp = tmp[j+tmp[j]-1];
00428             TakeNormalModulus((UWORD *)&tmp[j+1+AN.poly_num_vars], &ntmp,
00429                 modq,nmodq, NOUNPACK);
00430             tmp[j] = 2+AN.poly_num_vars+ABS(ntmp);
00431             tmp[j+tmp[j]-1] = ntmp;
00432         }
00433
00434         // add terms to polynomial
00435         if (tmp[j+tmp[j]-1]==0) {
00436             tmp[j]=0;
00437             j=prevj;
00438         }
00439     }
00440
00441     j+=tmp[j];
00442
00443     tmp[0] = j;
00444     WCOPY(terms,tmp,tmp[0]);
00445
00446 // M_free(p, "poly::normalize");
00447
00448     if (size_of_terms == AM.MaxTer/(LONG)sizeof(WORD))
00449         TermFree(tmp, "poly::normalize");
00450     else
00451         M_free(tmp, "poly::normalize");
00452
00453     AT.pWorkPointer = poffset;
00454     #undef p
00455
00456     return *this;
00457 }
00458
00459 /*
00460 #] normalize :
00461 #[ last_monomial_index :
00462 */
00463
00464 // index of the last monomial, i.e., the constant term
00465 WORD poly::last_monomial_index () const {
00466     POLY_GETIDENTITY(*this);
00467     return terms[0] - ABS(terms[terms[0]-1]) - AN.poly_num_vars - 2;

```

```

00468 }
00469
00470
00471 // pointer to the last monomial (the constant term)
00472 WORD * poly::last_monomial () const {
00473     return &terms[last_monomial_index()];
00474 }
00475
00476 /*
00477     #] last_monomial_index :
00478     #[ compare_degree_vector :
00479 */
00480
00481 int poly::compare_degree_vector(const poly & b) const {
00482     POLY_GETIDENTITY(*this);
00483
00484     // special cases if one or both are 0
00485     if (terms[0] == 1 && b[0] == 1) return 0;
00486     if (terms[0] == 1) return -1;
00487     if (b[0] == 1) return 1;
00488
00489     for (int i = 0; i < AN.poly_num_vars; i++) {
00490         int d1 = degree(i);
00491         int d2 = b.degree(i);
00492         if (d1 != d2) return d1 - d2;
00493     }
00494
00495     return 0;
00496 }
00497
00498 /*
00499     #] compare_degree_vector :
00500     #[ degree_vector :
00501 */
00502
00503 std::vector<int> poly::degree_vector() const {
00504     POLY_GETIDENTITY(*this);
00505
00506     std::vector<int> degrees(AN.poly_num_vars);
00507     for (int i = 0; i < AN.poly_num_vars; i++) {
00508         degrees[i] = degree(i);
00509     }
00510
00511     return degrees;
00512 }
00513
00514 /*
00515     #] degree_vector :
00516     #[ compare_degree_vector :
00517 */
00518
00519 int poly::compare_degree_vector(const vector<int> & b) const {
00520     POLY_GETIDENTITY(*this);
00521
00522     if (terms[0] == 1) return -1;
00523
00524     for (int i = 0; i < AN.poly_num_vars; i++) {
00525         int d1 = degree(i);
00526         if (d1 != b[i]) return d1 - b[i];
00527     }
00528
00529     return 0;
00530 }
00531
00532 /*
00533     #] compare_degree_vector :
00534     #[ add :
00535 */
00536
00537 // addition of polynomials by merging
00538 void poly::add (const poly &a, const poly &b, poly &c) {
00539     POLY_GETIDENTITY(a);
00540
00541     c.modp = a.modp;
00542     c.modn = a.modn;
00543
00544     WORD nmodq=0;
00545     UWORD *modq=NULL;
00546
00547     bool both_mod_small=false;
00548
00549     if (c.modp!=0) {
00550         if (c.modn == 1) {
00551             modq = (UWORD *)&c.modp;
00552             nmodq = 1;
00553             if (a.modp>0 && b.modp>0 && a.modn==1 && b.modn==1)

```

```

00555         both_mod_small=true;
00556     }
00557     else {
00558         RaisPowCached(BHEAD c.modp, c.modn, &modq, &nmodq);
00559     }
00560 }
00561
00562 int ai=1, bi=1, ci=1;
00563
00564 while (ai<a[0] || bi<b[0]) {
00565
00566     c.check_memory(ci);
00567
00568     int cmp = ai<a[0] && bi<b[0] ? monomial_compare(BHEAD &a[ai], &b[bi]) : 0;
00569
00570     if (bi==b[0] || cmp>0) {
00571         // insert term from a
00572         c.termscopy(&a[ai], ci, a[ai]);
00573         ci+=a[ai];
00574         ai+=a[ai];
00575     }
00576     else if (ai==a[0] || cmp<0) {
00577         // insert term from b
00578         c.termscopy(&b[bi], ci, b[bi]);
00579         ci+=b[bi];
00580         bi+=b[bi];
00581     }
00582     else {
00583         // insert term from a+b
00584         c.termscopy(&a[ai], ci, MaX(a[ai], b[bi]));
00585         WORD nc=0;
00586         if (both_mod_small) {
00587             c[ci+1+AN.poly_num_vars] = ((LONG)a[ai+1+AN.poly_num_vars]*a[ai+a[ai]-1]+
00588 (LONG)b[bi+1+AN.poly_num_vars]*b[bi+b[bi]-1]) % c.modp;
00589             if ((WORD)c[ci+1+AN.poly_num_vars] > +c.modp/2) c[ci+1+AN.poly_num_vars] -= c.modp;
00590             if ((WORD)c[ci+1+AN.poly_num_vars] < -c.modp/2) c[ci+1+AN.poly_num_vars] += c.modp;
00591             nc = (c[ci+1+AN.poly_num_vars]==0 ? 0 : SGN((WORD)c[ci+1+AN.poly_num_vars]));
00592             c[ci+1+AN.poly_num_vars] = ABS((WORD)c[ci+1+AN.poly_num_vars]);
00593         }
00594         else {
00595             AddLong((UWORD *)&a[ai+1+AN.poly_num_vars], a[ai+a[ai]-1],
00596 (UWORD *)&b[bi+1+AN.poly_num_vars], b[bi+b[bi]-1],
00597 (UWORD *)&c[ci+1+AN.poly_num_vars], &nc);
00598             if (c.modp!=0) TakeNormalModulus((UWORD *)&c[ci+1+AN.poly_num_vars], &nc,
00599 modq, nmodq,
00600 NOUNPACK);
00601         }
00602
00603         if (nc!=0) {
00604             c[ci] = 2+AN.poly_num_vars+ABS(nc);
00605             c[ci+c[ci]-1] = nc;
00606             ci += c[ci];
00607         }
00608
00609         ai+=a[ai];
00610         bi+=b[bi];
00611     }
00612 }
00613 c[0]=ci;
00614 }
00615
00616 /*
00617 #] add :
00618 #[ sub :
00619 */
00620
00621 // subtraction of polynomials by merging
00622 void poly::sub (const poly &a, const poly &b, poly &c) {
00623
00624     POLY_GETIDENTITY(a);
00625
00626     c.modp = a.modp;
00627     c.modn = a.modn;
00628
00629     WORD nmodq=0;
00630     UWORD *modq=NULL;
00631
00632     bool both_mod_small=false;
00633
00634     if (c.modp!=0) {
00635         if (c.modn == 1) {
00636             modq = (UWORD *)&c.modp;
00637             nmodq = 1;
00638             if (a.modp>0 && b.modp>0 && a.modn==1 && b.modn==1)

```

```

00639         both_mod_small=true;
00640     }
00641     else {
00642         RaisPowCached(BHEAD c.modp, c.modn, &modq, &nmodq);
00643     }
00644 }
00645
00646 int ai=1, bi=1, ci=1;
00647
00648 while (ai<a[0] || bi<b[0]) {
00649     c.check_memory(ci);
00650
00651     int cmp = ai<a[0] && bi<b[0] ? monomial_compare(BHEAD &a[ai], &b[bi]) : 0;
00652
00653     if (bi==b[0] || cmp>0) {
00654         // insert term from a
00655         c.termscopy(&a[ai], ci, a[ai]);
00656         ci+=a[ai];
00657         ai+=a[ai];
00658     }
00659     else if (ai==a[0] || cmp<0) {
00660         // insert term from b
00661         c.termscopy(&b[bi], ci, b[bi]);
00662         ci+=b[bi];
00663         bi+=b[bi];
00664         c[ci-1]*=-1;
00665     }
00666     else {
00667         // insert term from a+b
00668         c.termscopy(&a[ai], ci, MaX(a[ai], b[bi]));
00669         WORD nc=0;
00670         if (both_mod_small) {
00671             c[ci+1+AN.poly_num_vars] = ((LONG)a[ai+1+AN.poly_num_vars]*a[ai+a[ai]-1]-
00672 (LONG)b[bi+1+AN.poly_num_vars]*b[bi+b[bi]-1]) % c.modp;
00673             if ((WORD)c[ci+1+AN.poly_num_vars] > +c.modp/2) c[ci+1+AN.poly_num_vars] -= c.modp;
00674             if ((WORD)c[ci+1+AN.poly_num_vars] < -c.modp/2) c[ci+1+AN.poly_num_vars] += c.modp;
00675             nc = (c[ci+1+AN.poly_num_vars]==0 ? 0 : SGN((WORD)c[ci+1+AN.poly_num_vars]));
00676             c[ci+1+AN.poly_num_vars] = ABS((WORD)c[ci+1+AN.poly_num_vars]);
00677         }
00678         else {
00679             AddLong((UWORD *)&a[ai+1+AN.poly_num_vars], a[ai+a[ai]-1],
00680 (UWORD *)&b[bi+1+AN.poly_num_vars], -b[bi+b[bi]-1], // -b[...] causes
00681 subtraction
00682 (UWORD *)&c[ci+1+AN.poly_num_vars], &nc);
00683             if (c.modp!=0) TakeNormalModulus((UWORD *)&c[ci+1+AN.poly_num_vars], &nc,
00684 modq, nmodq,
00685 NOUNPACK);
00686         }
00687         if (nc!=0) {
00688             c[ci] = 2+AN.poly_num_vars+ABS(nc);
00689             c[ci+c[ci]-1] = nc;
00690             ci += c[ci];
00691         }
00692         ai+=a[ai];
00693         bi+=b[bi];
00694     }
00695 }
00696 }
00697
00698 c[0]=ci;
00699 }
00700
00701 /*
00702 #] sub :
00703 #[ pop_heap :
00704 */
00705
00706 // pops the largest monomial from the heap and stores it in heap[n]
00707 void poly::pop_heap (PHEAD WORD **heap, int n) {
00708     WORD *old = heap[0];
00709
00710     heap[0] = heap[--n];
00711
00712     int i=0;
00713     while (2*i+2<n && (monomial_compare(BHEAD heap[2*i+1]+3, heap[i]+3)>0 ||
00714 monomial_compare(BHEAD heap[2*i+2]+3, heap[i]+3)>0)) {
00715         if (monomial_compare(BHEAD heap[2*i+1]+3, heap[2*i+2]+3)>0) {
00716             swap(heap[i], heap[2*i+1]);
00717             i=2*i+1;
00718         }
00719         else {
00720

```

```

00722         swap(heap[i], heap[2*i+2]);
00723         i=2*i+2;
00724     }
00725 }
00726
00727     if (2*i+1<n && monomial_compare(BHEAD heap[2*i+1]+3, heap[i]+3)>0)
00728         swap(heap[i], heap[2*i+1]);
00729
00730     heap[n] = old;
00731 }
00732
00733 /*
00734     #] pop_heap :
00735     #[ push_heap :
00736 */
00737
00738 // pushes the monomial in heap[n] onto the heap
00739 void poly::push_heap (PHEAD WORD **heap, int n) {
00740
00741     int i=n-1;
00742
00743     while (i>0 && monomial_compare(BHEAD heap[i]+3, heap[(i-1)/2]+3) > 0) {
00744         swap(heap[(i-1)/2], heap[i]);
00745         i=(i-1)/2;
00746     }
00747 }
00748
00749 /*
00750     #] push_heap :
00751     #[ mul_one_term :
00752 */
00753
00754 // multiplies a polynomial with a monomial
00755 void poly::mul_one_term (const poly &a, const poly &b, poly &c) {
00756
00757     POLY_GETIDENTITY(a);
00758
00759     int ci=1;
00760
00761     WORD nmodq=0;
00762     UWORD *modq=NULL;
00763
00764     bool both_mod_small=false;
00765
00766     if (c.modp!=0) {
00767         if (c.modn == 1) {
00768             modq = (UWORD *)&c.modp;
00769             nmodq = 1;
00770             if (a.modp>0 && b.modp>0 && a.modn==1 && b.modn==1)
00771                 both_mod_small=true;
00772         }
00773         else {
00774             RaisPowCached(BHEAD c.modp, c.modn, &modq, &nmodq);
00775         }
00776     }
00777
00778     for (int ai=1; ai<a[0]; ai+=a[ai])
00779         for (int bi=1; bi<b[0]; bi+=b[bi]) {
00780
00781             c.check_memory(ci);
00782
00783             for (int i=0; i<AN.poly_num_vars; i++)
00784                 c[ci+1+i] = a[ai+1+i] + b[bi+1+i];
00785             WORD nc;
00786
00787             // if both polynomials are modulo p^1, use integer calculus
00788             if (both_mod_small) {
00789                 c[ci+1+AN.poly_num_vars] =
00790                     ((LONG)a[ai+1+AN.poly_num_vars] * a[ai+2+AN.poly_num_vars] *
00791                      b[bi+1+AN.poly_num_vars] * b[bi+2+AN.poly_num_vars]) % c.modp;
00792                 nc = (c[ci+1+AN.poly_num_vars]==0 ? 0 : 1);
00793             }
00794             else {
00795                 // otherwise, use form long calculus
00796                 MulLong((UWORD *)&a[ai+1+AN.poly_num_vars], a[ai+a[ai]-1],
00797                        (UWORD *)&b[bi+1+AN.poly_num_vars], b[bi+b[bi]-1],
00798                        (UWORD *)&c[ci+1+AN.poly_num_vars], &nc);
00799                 if (c.modp!=0) TakeNormalModulus((UWORD *)&c[ci+1+AN.poly_num_vars], &nc,
00800                                                  modq, nmodq,
00801                                                  NOUNPACK);
00802             }
00803
00804             if (nc!=0) {
00805                 c[ci] = 2+AN.poly_num_vars+ABS(nc);
00806                 ci += c[ci];
00807             }
00808
00809             if (both_mod_small) {

```

```

00808             if (c[ci-2] > +c.modp/2) c[ci-2] -= c.modp;
00809             if (c[ci-2] < -c.modp/2) c[ci-2] += c.modp;
00810             c[ci-1] = SGN(c[ci-2]);
00811             c[ci-2] = ABS(c[ci-2]);
00812         }
00813         else {
00814             c[ci-1] = nc;
00815         }
00816     }
00817 }
00818
00819     c[0]=ci;
00820 }
00821
00822 /*
00823     #] mul_one_term :
00824     #[ mul_univar :
00825 */
00826
00827 // dense univariate multiplication, i.e., for each power find all
00828 // pairs of monomials that result in that power
00829 void poly::mul_univar (const poly &a, const poly &b, poly &c, int var) {
00830
00831     POLY_GETIDENTITY(a);
00832
00833     WORD nmodq=0;
00834     UWORD *modq=NULL;
00835
00836     bool both_mod_small=false;
00837
00838     if (c.modp!=0) {
00839         if (c.modn == 1) {
00840             modq = (UWORD *)&c.modp;
00841             nmodq = 1;
00842             if (a.modp>0 && b.modp>0 && a.modn==1 && b.modn==1)
00843                 both_mod_small=true;
00844         }
00845         else {
00846             RaisPowCached(BHEAD c.modp,c.modn,&modq,&nmodq);
00847         }
00848     }
00849
00850     poly t(BHEAD 0);
00851     WORD nt;
00852
00853     int ci=1;
00854
00855     // bounds on the powers in a*b
00856     int minpow = AN.poly_num_vars==0 ? 0 : a.last_monomial()[1+var] + b.last_monomial()[1+var];
00857     int maxpow = AN.poly_num_vars==0 ? 0 : a[2+var]+b[2+var];
00858     int afirst=1, blast=1;
00859
00860     for (int pow=maxpow; pow>=minpow; pow--) {
00861
00862         c.check_memory(ci);
00863
00864         WORD nc=0;
00865
00866         // adjust range in a or b
00867         if (a[afirst+1+var] + b[blast+1+var] > pow) {
00868             if (blast+b[blast] < b[0])
00869                 blast+=b[blast];
00870             else
00871                 afirst+=a[afirst];
00872         }
00873
00874         // find terms that result in the correct power
00875         for (int ai=afirst, bi=blast; ai<a[0] && bi>=1;) {
00876
00877             int thispow = AN.poly_num_vars==0 ? 0 : a[ai+1+var] + b[bi+1+var];
00878
00879             if (thispow == pow) {
00880
00881                 // if both polynomials are modulo p^1, use integer calculus
00882                 if (both_mod_small) {
00883                     c[ci+1+AN.poly_num_vars] =
00884                         ((nc==0 ? 0 : (LONG)c[ci+1+AN.poly_num_vars] * nc) +
00885                         (LONG)a[ai+1+AN.poly_num_vars] * a[ai+2+AN.poly_num_vars] *
00886                         b[bi+1+AN.poly_num_vars] * b[bi+2+AN.poly_num_vars]) % c.modp;
00887                     nc = (c[ci+1+AN.poly_num_vars]==0 ? 0 : 1);
00888                 }
00889                 else {
00890                     // otherwise, use form long calculus
00891
00892                     MulLong((UWORD *)&a[ai+1+AN.poly_num_vars], a[ai+a[ai]-1],
00893                             (UWORD *)&b[bi+1+AN.poly_num_vars], b[bi+b[bi]-1],
00894                             (UWORD *)&t[0], &nt);

```



```

00895
00896         AddLong ((UWORD *)&t[0], nt,
00897                 (UWORD *)&c[ci+1+AN.poly_num_vars], nc,
00898                 (UWORD *)&c[ci+1+AN.poly_num_vars], &nc);
00899
00900         if (c.modp!=0) TakeNormalModulus((UWORD *)&c[ci+1+AN.poly_num_vars], &nc,
00901                                         modq, nmodq,
NOUNPACK);
00902     }
00903
00904     ai += a[ai];
00905     bi -= ABS(b[bi-1]) + 2 + AN.poly_num_vars;
00906 }
00907 else if (thispow > pow)
00908     ai += a[ai];
00909 else
00910     bi -= ABS(b[bi-1]) + 2 + AN.poly_num_vars;
00911 }
00912
00913 // add term to result
00914 if (nc != 0) {
00915     for (int j=0; j<AN.poly_num_vars; j++)
00916         c[ci+1+j] = 0;
00917     if (AN.poly_num_vars > 0)
00918         c[ci+1+var] = pow;
00919
00920     c[ci] = 2+AN.poly_num_vars+ABS(nc);
00921     ci += c[ci];
00922
00923     // if necessary, adjust to range [-p/2,p/2]
00924     if (both_mod_small) {
00925         if (c[ci-2] > +c.modp/2) c[ci-2] -= c.modp;
00926         if (c[ci-2] < -c.modp/2) c[ci-2] += c.modp;
00927         c[ci-1] = SGN(c[ci-2]);
00928         c[ci-2] = ABS(c[ci-2]);
00929     }
00930     else {
00931         c[ci-1] = nc;
00932     }
00933 }
00934 }
00935
00936 c[0] = ci;
00937 }
00938
00939 /*
00940 #] mul_univar :
00941 #[ mul_heap :
00942 */
00943
00944 void poly::mul_heap (const poly &a, const poly &b, poly &c) {
00945
00946     POLY_GETIDENTITY(a);
00947
00948     WORD nmodq=0;
00949     UWORD *modq=NULL;
00950
00951     bool both_mod_small=false;
00952
00953     if (c.modp!=0) {
00954         if (c.modn == 1) {
00955             modq = (UWORD *)&c.modp;
00956             nmodq = 1;
00957             if (a.modp>0 && b.modp>0 && a.modn==1 && b.modn==1)
00958                 both_mod_small=true;
00959         }
00960         else {
00961             RaisPowCached(BHEAD c.modp, c.modn, &modq, &nmodq);
00962         }
00963     }
00964
00965     // find maximum powers in different variables
00966     WORD *maxpower = AT.WorkPointer;
00967     AT.WorkPointer += AN.poly_num_vars;
00968     WORD *maxpowera = AT.WorkPointer;
00969     AT.WorkPointer += AN.poly_num_vars;
00970     WORD *maxpowerb = AT.WorkPointer;
00971     AT.WorkPointer += AN.poly_num_vars;
00972
00973     for (int i=0; i<AN.poly_num_vars; i++)
00974         maxpowera[i] = maxpowerb[i] = 0;
00975
00976     for (int ai=1; ai<a[0]; ai+=a[ai])
00977         for (int j=0; j<AN.poly_num_vars; j++)
00978             maxpowera[j] = MaX(maxpowera[j], a[ai+1+j]);
00979
00980     for (int bi=1; bi<b[0]; bi+=b[bi])

```

```

00998         for (int j=0; j<AN.poly_num_vars; j++)
00999             maxpowerb[j] = MaX(maxpowerb[j], b[bi+1+j]);
01000
01001     for (int i=0; i<AN.poly_num_vars; i++)
01002         maxpower[i] = maxpowera[i] + maxpowerb[i];
01003
01004     // if PROD(maxpower) small, use hash array
01005     bool use_hash = true;
01006     int nhash = 1;
01007
01008     for (int i=0; i<AN.poly_num_vars; i++) {
01009         if (nhash > POLY_MAX_HASH_SIZE / (maxpower[i]+1)) {
01010             nhash = 1;
01011             use_hash = false;
01012             break;
01013         }
01014         nhash *= maxpower[i]+1;
01015     }
01016
01017     // allocate heap and hash
01018     int nheap=a.number_of_terms();
01019
01020     WantAddPointers(nheap+nhash);
01021     WORD **heap = AT.pWorkSpace + AT.pWorkPointer;
01022
01023     for (int ai=1, i=0; ai<a[0]; ai+=a[ai], i++) {
01024         heap[i] = (WORD *) NumberMalloc("poly::mul_heap");
01025         heap[i][0] = ai;
01026         heap[i][1] = 1;
01027         heap[i][2] = -1;
01028         heap[i][3] = 0;
01029         for (int j=0; j<AN.poly_num_vars; j++)
01030             heap[i][4+j] = MAXPOSITIVE;
01031     }
01032
01033     WORD **hash = AT.pWorkSpace + AT.pWorkPointer + nheap;
01034     for (int i=0; i<nhash; i++)
01035         hash[i] = NULL;
01036
01037     int ci = 1;
01038
01039     // multiply
01040     while (nheap > 0) {
01041
01042         c.check_memory(ci);
01043
01044         pop_heap(BHEAD heap, nheap--);
01045         WORD *p = heap[nheap];
01046
01047         // if non-zero
01048         if (p[3] != 0) {
01049             if (use_hash) hash[p[2]] = NULL;
01050
01051             c[0] = ci;
01052
01053             // append this term to the result
01054             if (use_hash || ci==1 || monomial_compare(BHEAD p+3, c.last_monomial())!=0) {
01055                 p[4 + AN.poly_num_vars + ABS(p[3])] = p[3];
01056                 p[3] = 2 + AN.poly_num_vars + ABS(p[3]);
01057                 c.termscopy(&p[3],ci,p[3]);
01058                 ci += c[ci];
01059             }
01060             else {
01061                 // add this term to the last term of the result
01062                 ci = c.last_monomial_index();
01063                 WORD nc = c[ci+c[ci]-1];
01064
01065                 // if both polynomials are modulo p^1, use integer calculus
01066                 if (both_mod_small) {
01067                     c[ci+AN.poly_num_vars+1] = ((LONG)c[ci+AN.poly_num_vars+1]*nc +
01068 p[4+AN.poly_num_vars]*p[3]) % c.modp;
01069                     if (c[ci+1+AN.poly_num_vars]==0)
01070                         nc = 0;
01071                     else {
01072                         if (c[ci+1+AN.poly_num_vars] > +c.modp/2) c[ci+1+AN.poly_num_vars] -= c.modp;
01073                         if (c[ci+1+AN.poly_num_vars] < -c.modp/2) c[ci+1+AN.poly_num_vars] += c.modp;
01074                         nc = SGN(c[ci+1+AN.poly_num_vars]);
01075                         c[ci+1+AN.poly_num_vars] = ABS(c[ci+1+AN.poly_num_vars]);
01076                     }
01077                 }
01078                 else {
01079                     // otherwise, use form long calculus
01080                     AddLong ((UWORD *)&p[4+AN.poly_num_vars], p[3],
01081                             (UWORD *)&c[ci+AN.poly_num_vars+1], nc,
01082                             (UWORD *)&c[ci+AN.poly_num_vars+1],&nc);
01083
01084                     if (c.modp!=0) TakeNormalModulus((UWORD *)&c[ci+1+AN.poly_num_vars], &nc,

```

```

01084                                     modq, nmodq,
NOUNPACK);
01085     }
01086
01087     if (nc!=0) {
01088         c[ci] = 2 + AN.poly_num_vars + ABS(nc);
01089         ci += c[ci];
01090         c[ci-1] = nc;
01091     }
01092 }
01093 }
01094
01095 // add new term to the heap (ai, bi+1)
01096 while (p[1] < b[0]) {
01097
01098     for (int j=0; j<AN.poly_num_vars; j++)
01099         p[4+j] = a[p[0]+1+j] + b[p[1]+1+j];
01100
01101     // if both polynomials are modulo p^1, use integer calculus
01102     if (both_mod_small) {
01103         p[4+AN.poly_num_vars] = ((LONG)a[p[0]+1+AN.poly_num_vars]*a[p[0]+a[p[0]]-1]*
01104 b[p[1]+1+AN.poly_num_vars]*b[p[1]+b[p[1]]-1]) % c.modp;
01105         if (p[4+AN.poly_num_vars]==0)
01106             p[3]=0;
01107         else {
01108             if (p[4+AN.poly_num_vars] > +c.modp/2) p[4+AN.poly_num_vars] -= c.modp;
01109             if (p[4+AN.poly_num_vars] < -c.modp/2) p[4+AN.poly_num_vars] += c.modp;
01110             p[3] = SGN(p[4+AN.poly_num_vars]);
01111             p[4+AN.poly_num_vars] = ABS(p[4+AN.poly_num_vars]);
01112         }
01113     }
01114     else {
01115         // otherwise, use form long calculus
01116         MulLong((UWORD *)&a[p[0]+1+AN.poly_num_vars], a[p[0]+a[p[0]]-1],
01117 (UWORD *)&b[p[1]+1+AN.poly_num_vars], b[p[1]+b[p[1]]-1],
01118 (UWORD *)&p[4+AN.poly_num_vars], &p[3]);
01119
01120         if (c.modp!=0) TakeNormalModulus((UWORD *)&p[4+AN.poly_num_vars], &p[3],
01121                                     modq, nmodq,
NOUNPACK);
01122     }
01123
01124     p[1] += b[p[1]];
01125
01126     if (use_hash) {
01127         int ID = 0;
01128         for (int i=0; i<AN.poly_num_vars; i++)
01129             ID = (maxpower[i]+1)*ID + p[4+i];
01130
01131         // if hash and unused, push onto heap
01132         if (hash[ID] == NULL) {
01133             p[2] = ID;
01134             hash[ID] = p;
01135             push_heap(BHEAD heap, ++nheap);
01136             break;
01137         }
01138         else {
01139             // if hash and used, add to heap element
01140             WORD *h = hash[ID];
01141             // if both polynomials are modulo p^1, use integer calculus
01142             if (both_mod_small) {
01143                 h[4+AN.poly_num_vars] = ((LONG)p[4+AN.poly_num_vars]*p[3] +
h[4+AN.poly_num_vars]*h[3]) % c.modp;
01144                 if (h[4+AN.poly_num_vars]==0)
01145                     h[3]=0;
01146                 else {
01147                     if (h[4+AN.poly_num_vars] > +c.modp/2) h[4+AN.poly_num_vars] -= c.modp;
01148                     if (h[4+AN.poly_num_vars] < -c.modp/2) h[4+AN.poly_num_vars] += c.modp;
01149                     h[3] = SGN(h[4+AN.poly_num_vars]);
01150                     h[4+AN.poly_num_vars] = ABS(h[4+AN.poly_num_vars]);
01151                 }
01152             }
01153             else {
01154                 // otherwise, use form long calculus
01155                 AddLong((UWORD *)&p[4+AN.poly_num_vars], p[3],
01156 (UWORD *)&h[4+AN.poly_num_vars], h[3],
01157 (UWORD *)&h[4+AN.poly_num_vars], &h[3]);
01158
01159                 if (c.modp!=0) TakeNormalModulus((UWORD *)&h[4+AN.poly_num_vars], &h[3],
01160                                     modq, nmodq,
NOUNPACK);
01161             }
01162         }
01163     }
01164     else {
01165         // if no hash, push onto heap

```

```

01166         p[2] = -1;
01167         push_heap(BHEAD heap, ++nheap);
01168         break;
01169     }
01170 }
01171 }
01172
01173 c[0] = ci;
01174
01175 for (int ai=1, i=0; ai<a[0]; ai+=a[ai], i++)
01176     NumberFree(heap[i], "poly::mul_heap");
01177 AT.WorkPointer -= 3 * AN.poly_num_vars;
01178 }
01179
01180 /*
01181  #] mul_heap :
01182  #[ mul :
01183  */
01184
01195 void poly::mul (const poly &a, const poly &b, poly &c) {
01196
01197     c.modp = a.modp;
01198     c.modn = a.modn;
01199
01200     if (a.is_zero() || b.is_zero()) { c[0]=1; return; }
01201     if (a.is_one()) {
01202         c.check_memory(b[0]);
01203         c.termscopy(b.terms, 0, b[0]);
01204         // normalize if necessary
01205         if (a.modp!=b.modp || a.modn!=b.modn) {
01206             c.modp=0;
01207             c.setmod(a.modp, a.modn);
01208         }
01209         return;
01210     }
01211     if (b.is_one()) {
01212         c.check_memory(a[0]);
01213         c.termscopy(a.terms, 0, a[0]);
01214         return;
01215     }
01216
01217     int na=a.number_of_terms();
01218     int nb=b.number_of_terms();
01219
01220     if (na==1 || nb==1) {
01221         mul_one_term(a, b, c);
01222         return;
01223     }
01224
01225     int vara = a.is_dense_univariate();
01226     int varb = b.is_dense_univariate();
01227
01228     if (vara!=-2 && varb!=-2 && vara==varb) {
01229         mul_univar(a, b, c, vara);
01230         return;
01231     }
01232
01233     if (na < nb)
01234         mul_heap(a, b, c);
01235     else
01236         mul_heap(b, a, c);
01237 }
01238
01239 /*
01240  #] mul :
01241  #[ divmod_one_term : :
01242  */
01243
01244 // divides a polynomial by a monomial
01245 void poly::divmod_one_term (const poly &a, const poly &b, poly &q, poly &r, bool only_divides) {
01246
01247     POLY_GETIDENTITY(a);
01248
01249     int qi=1, ri=1;
01250
01251     WORD nmodq=0;
01252     UWORD *modq=NULL;
01253
01254     WORD nltbinv=0;
01255     UWORD *ltbinv=NULL;
01256
01257     bool both_mod_small=false;
01258
01259     if (q.modp!=0) {
01260         if (q.modn == 1) {
01261             modq = (UWORD *) &q.modp;
01262             nmodq = 1;

```

```

01263         if (a.modp>0 && b.modp>0 && a.modn==1 && b.modn==1)
01264             both_mod_small=true;
01265     }
01266     else {
01267         RaisPowCached(BHEAD q.modp,q.modn,&modq,&nmodq);
01268     }
01269     ltbinv = NumberMalloc("poly::div_one_term");
01270
01271     if (both_mod_small) {
01272         WORD ltb = b[b[1]]*b[2+AN.poly_num_vars];
01273         GetModInverses(ltb + (ltb<0?q.modp:0), q.modp, (WORD*)ltbinv, NULL);
01274         nltbinv = 1;
01275     }
01276     else
01277         GetLongModInverses(BHEAD (UWORD *)&b[2+AN.poly_num_vars], b[b[1]], modq, nmodq, ltbinv,
01278 &nltbinv, NULL, NULL);
01279 }
01280
01281 for (int ai=1; ai<a[0]; ai+=a[ai]) {
01282
01283     q.check_memory(qi);
01284     r.check_memory(ri);
01285
01286     // check divisibility of powers
01287     bool div=true;
01288     for (int j=0; j<AN.poly_num_vars; j++) {
01289         q[qi+1+j] = a[ai+1+j]-b[2+j];
01290         r[ri+1+j] = a[ai+1+j];
01291         if (q[qi+1+j] < 0) div=false;
01292     }
01293
01294     WORD nq,nr;
01295
01296     if (div) {
01297         // if variables are divisible, divide coefficient
01298         if (q.modp==0) {
01299             DivLong((UWORD *)&a[ai+1+AN.poly_num_vars], a[ai+a[ai]-1],
01300 (UWORD *)&b[2+AN.poly_num_vars], b[b[1]],
01301 (UWORD *)&q[qi+1+AN.poly_num_vars], &nq,
01302 (UWORD *)&r[ri+1+AN.poly_num_vars], &nr);
01303         }
01304         else {
01305             // if both polynomials are modulo p^1, use integer calculus
01306             if (both_mod_small) {
01307                 q[qi+1+AN.poly_num_vars] =
01308 (LONG)a[ai+1+AN.poly_num_vars] * a[ai+a[ai]-1] * ltbinv[0] * nltbinv) %
01309 q.modp;
01310                 nq = (q[qi+1+AN.poly_num_vars]==0 ? 0 : 1);
01311             }
01312             else {
01313                 // otherwise, use form long calculus
01314                 MulLong((UWORD *)&a[ai+1+AN.poly_num_vars], a[ai+a[ai]-1],
01315 ltbinv, nltbinv,
01316 (UWORD *)&q[qi+1+AN.poly_num_vars], &nq);
01317                 TakeNormalModulus((UWORD *)&q[qi+1+AN.poly_num_vars], &nq,
01318 modq,nmodq, NOUNPACK);
01319             }
01320             nr=0;
01321         }
01322     }
01323     else {
01324         // if not, term becomes part of the remainder
01325         nq=0;
01326         nr=a[ai+a[ai]-1];
01327         r.termscopy(&a[ai+1+AN.poly_num_vars], ri+1+AN.poly_num_vars, ABS(nr));
01328     }
01329
01330     // add terms to quotient/remainder
01331     if (nq!=0) {
01332         q[qi] = 2+AN.poly_num_vars+ABS(nq);
01333         qi += q[qi];
01334
01335         // if necessary, adjust to range [-p/2,p/2]
01336         if (both_mod_small) {
01337             if (q[qi-2] > +q.modp/2) q[qi-2] -= q.modp;
01338             if (q[qi-2] < -q.modp/2) q[qi-2] += q.modp;
01339             q[qi-1] = SGN(q[qi-2]);
01340             q[qi-2] = ABS(q[qi-2]);
01341         }
01342         else {
01343             q[qi-1] = nq;
01344         }
01345     }
01346
01347     if (nr != 0) {

```

```

01348         if (only_divides) { r = poly(BHEAD 1); ri=r[0]; break; }
01349         r[ri] = 2+AN.poly_num_vars+ABS(nr);
01350         ri += r[ri];
01351         r[ri-1] = nr;
01352     }
01353 }
01354
01355 q[0]=qi;
01356 r[0]=ri;
01357
01358 if (q.modp!=0 || ltbinv != NULL) NumberFree(ltbinv,"poly::div_one_term");
01359 }
01360
01361 /*
01362 #] divmod_one_term :
01363 #[ divmod_univar : :
01364 */
01365
01376 void poly::divmod_univar (const poly &a, const poly &b, poly &q, poly &r, int var, bool only_divides)
01377 {
01378     POLY_GETIDENTITY(a);
01379
01380     WORD nmodq=0;
01381     UWORD *modq=NULL;
01382
01383     WORD nltbinv=0;
01384     UWORD *ltbinv=NULL;
01385
01386     bool both_mod_small=false;
01387
01388     if (q.modp!=0) {
01389         if (q.modn == 1) {
01390             modq = (UWORD *) &q.modp;
01391             nmodq = 1;
01392             if (a.modp>0 && b.modp>0 && a.modn==1 && b.modn==1)
01393                 both_mod_small=true;
01394         }
01395         else {
01396             RaisPowCached(BHEAD q.modp,q.modn,&modq,&nmodq);
01397         }
01398         ltbinv = NumberMalloc("poly::div_univar");
01399
01400         if (both_mod_small) {
01401             WORD ltb = b[b[1]]*b[2+AN.poly_num_vars];
01402             GetModInverses(ltb + (ltb<0?q.modp:0), q.modp, (WORD*)ltbinv, NULL);
01403             nltbinv = 1;
01404         }
01405         else
01406             GetLongModInverses(BHEAD (UWORD *) &b[2+AN.poly_num_vars], b[b[1]], modq, nmodq, ltbinv,
01407 &nltbinv, NULL, NULL);
01408     }
01409
01410     WORD ns=0;
01411     WORD nt;
01412     UWORD *s = NumberMalloc("poly::div_univar");
01413     UWORD *t = NumberMalloc("poly::div_univar");
01414     s[0]=0;
01415
01416     int bpow = b[2+var];
01417
01418     int ai=1, qi=1, ri=1;
01419
01420     for (int pow=a[2+var]; pow>=0; pow--) {
01421         q.check_memory(qi);
01422         r.check_memory(ri);
01423
01424         // look for the correct power in a
01425         while (ai<a[0] && a[ai+1+var] > pow)
01426             ai+=a[ai];
01427
01428         // first term of the r.h.s. of the above equation
01429         if (ai<a[0] && a[ai+1+var] == pow) {
01430             ns = a[ai+a[ai]-1];
01431             WCOPY(s, &a[ai+1+AN.poly_num_vars], ABS(ns));
01432         }
01433         else {
01434             ns = 0;
01435         }
01436
01437         int bi=1, qj=qi;
01438
01439         // second term(s) of the r.h.s. of the above equation
01440         while (qj>1 && bi<b[0]) {
01441             qj -= 2 + AN.poly_num_vars + ABS(q[qj-1]);

```

```

01443
01444 while (bi<b[0] && b[bi+1+var]+q[qj+1+var] > pow)
01445     bi += b[bi];
01446
01447 if (bi<b[0] && b[bi+1+var]+q[qj+1+var] == pow) {
01448     // if both polynomials are modulo p^1, use integer calculus
01449
01450     if (both_mod_small) {
01451         s[0] = ((WORD)s[0]*ns - (LONG)b[bi+1+AN.poly_num_vars] * b[bi+b[bi]-1] *
01452             q[qj+1+AN.poly_num_vars] * q[qj+q[qj]-1]) % q.modp;
01453         ns = (s[0]==0 ? 0 : 1);
01454     }
01455     else {
01456         // otherwise, use form long calculus
01457         MulLong((UWORD *)&b[bi+1+AN.poly_num_vars], b[bi+b[bi]-1],
01458             (UWORD *)&q[qj+1+AN.poly_num_vars], q[qj+q[qj]-1],
01459             t, &nt);
01460         nt *= -1;
01461         AddLong(t,nt,s,ns,s,&ns);
01462         if (q.modp!=0) TakeNormalModulus((UWORD *)s,&ns,modq, nmodq, NOUNPACK);
01463     }
01464 }
01465 }
01466
01467 if (ns != 0) {
01468     // if necessary, adjust to range [-p/2,p/2]
01469     if (both_mod_small) {
01470         s[0]*=ns;
01471         if ((WORD)s[0] > +q.modp/2) s[0] -= q.modp;
01472         if ((WORD)s[0] < -q.modp/2) s[0] += q.modp;
01473         ns = SGN((WORD)s[0]);
01474         s[0] = ABS((WORD)s[0]);
01475     }
01476
01477     if (pow >= bpow) {
01478         // large power, so divide by b
01479         if (q.modp == 0) {
01480             DivLong(s, ns,
01481                 (UWORD *)&b[2+AN.poly_num_vars], b[b[1]],
01482                 (UWORD *)&q[qi+1+AN.poly_num_vars], &ns, t, &nt);
01483         }
01484         else {
01485             if (both_mod_small) {
01486                 q[qi+1+AN.poly_num_vars] = ((LONG)s[0]*ns*ltbinv[0]*nltbinv) % q.modp;
01487                 if ((WORD)q[qi+1+AN.poly_num_vars] > +q.modp/2) q[qi+1+AN.poly_num_vars] -=
q.modp;
01488                 if ((WORD)q[qi+1+AN.poly_num_vars] < -q.modp/2) q[qi+1+AN.poly_num_vars] +=
q.modp;
01489                 ns = (q[qi+1+AN.poly_num_vars]==0 ? 0 : SGN((WORD)q[qi+1+AN.poly_num_vars]));
01490                 q[qi+1+AN.poly_num_vars] = ABS((WORD)q[qi+1+AN.poly_num_vars]);
01491             }
01492             else {
01493                 MulLong(s, ns, ltbinv, nltbinv, (UWORD *)&q[qi+1+AN.poly_num_vars], &ns);
01494                 TakeNormalModulus((UWORD *)&q[qi+1+AN.poly_num_vars], &ns,
01495                     modq,nmodq, NOUNPACK);
01496             }
01497             nt=0;
01498         }
01499     }
01500     else {
01501         // small power, so remainder
01502         WCOPY(t,s,ABS(ns));
01503         nt = ns;
01504         ns = 0;
01505     }
01506
01507     // add terms to quotient/remainder
01508     if (ns!=0) {
01509         for (int i=0; i<AN.poly_num_vars; i++)
01510             q[qi+1+i] = 0;
01511         q[qi+1+var] = pow-bpow;
01512
01513         q[qi] = 2+AN.poly_num_vars+ABS(ns);
01514         qi += q[qi];
01515         q[qi-1] = ns;
01516     }
01517
01518     if (nt != 0) {
01519         if (only_divides) { r=poly(BHEAD 1); ri=r[0]; break; }
01520         for (int i=0; i<AN.poly_num_vars; i++)
01521             r[ri+1+i] = 0;
01522         r[ri+1+var] = pow;
01523
01524         for (int i=0; i<ABS(nt); i++)
01525             r[ri+1+AN.poly_num_vars+i] = t[i];
01526

```

```

01527         r[ri] = 2+AN.poly_num_vars+ABS(nt);
01528         ri += r[ri];
01529         r[ri-1] = nt;
01530     }
01531 }
01532 }
01533
01534 q[0] = qi;
01535 r[0] = ri;
01536
01537 NumberFree(s,"poly::div_univar");
01538 NumberFree(t,"poly::div_univar");
01539
01540 if (q.modp!=0 || ltbinv!=NULL) NumberFree(ltbinv,"poly::div_univar");
01541 }
01542
01543 /*
01544 #] divmod_univar :
01545 #[ divmod_heap :
01546 */
01547
01579 void poly::divmod_heap (const poly &a, const poly &b, poly &q, poly &r, bool only_divides, bool
check_div, bool &div_fail) {
01580
01581     POLY_GETIDENTITY(a);
01582
01583     div_fail = false;
01584     q[0] = r[0] = 1;
01585
01586     WORD nmodq=0;
01587     UWORD *modq=NULL;
01588
01589     WORD nltbinv=0;
01590     UWORD *ltbinv=NULL;
01591     LONG oldpWorkPointer = AT.pWorkPointer;
01592
01593     bool both_mod_small=false;
01594
01595     if (q.modp!=0) {
01596         if (q.modn == 1) {
01597             modq = (UWORD *) &q.modp;
01598             nmodq = 1;
01599             if (a.modp>0 && b.modp>0 && a.modn==1 && b.modn==1)
01600                 both_mod_small=true;
01601         }
01602         else {
01603             RaisPowCached(BHEAD q.modp,q.modn,&modq,&nmodq);
01604         }
01605         ltbinv = NumberMalloc("poly::div_heap-a");
01606
01607         if (both_mod_small) {
01608             WORD ltb = b[b[1]]*b[2+AN.poly_num_vars];
01609             GetModInverses(ltb + (ltb<0?q.modp:0), q.modp, (WORD*)ltbinv, NULL);
01610             nltbinv = 1;
01611         }
01612         else
01613             GetLongModInverses(BHEAD (UWORD *) &b[2+AN.poly_num_vars], b[b[1]], modq, nmodq, ltbinv,
&nltbinv, NULL, NULL);
01614     }
01615
01616     // allocate heap
01617     int nb=b.number_of_terms();
01618     WantAddPointers(nb);
01619     WORD **heap = AT.pWorkSpace + AT.pWorkPointer;
01620     AT.pWorkPointer += nb;
01621
01622     for (int i=0; i<nb; i++)
01623         heap[i] = (WORD *) NumberMalloc("poly::div_heap-b");
01624     int nheap = 1;
01625     heap[0][0] = 1;
01626     heap[0][1] = 0;
01627     heap[0][2] = -1;
01628     WCOPY(&heap[0][3], &a[1], a[1]);
01629     heap[0][3] = a[a[1]];
01630
01631     int qi=1, ri=1;
01632
01633     int s = nb;
01634     WORD *t = (WORD *) NumberMalloc("poly::div_heap-c");
01635
01636     // insert contains element that still have to be inserted to the heap
01637     // (exists to avoid code duplication).
01638     vector<pair<int,int> > insert;
01639
01640     while (insert.size()>0 || nheap>0) {
01641
01642         q.check_memory(qi);

```



```

01643         r.check_memory(ri);
01644
01645         // collect a term t for the quotient/remainder
01646         t[0] = -1;
01647
01648         do {
01649
01650             WORD *p = heap[nheap];
01651             bool this_insert;
01652
01653             if (insert.empty()) {
01654                 // extract element from the heap and prepare adding new ones
01655                 this_insert = false;
01656
01657                 pop_heap(BHEAD heap, nheap--);
01658                 p = heap[nheap];
01659
01660                 if (t[0] == -1) {
01661                     WCOPY(t, p, (5+ABS(p[3])+AN.poly_num_vars));
01662                 }
01663                 else {
01664                     // if both polynomials are modulo p^1, use integer calculus
01665                     if (both_mod_small) {
01666                         t[4+AN.poly_num_vars] = ((LONG)t[4+AN.poly_num_vars]*t[3] +
01667 p[4+AN.poly_num_vars]*p[3]) % q.modp;
01668                         if (t[4+AN.poly_num_vars]==0)
01669                             t[3]=0;
01670                         else {
01671                             if (t[4+AN.poly_num_vars] > +q.modp/2) t[4+AN.poly_num_vars] -= q.modp;
01672                             if (t[4+AN.poly_num_vars] < -q.modp/2) t[4+AN.poly_num_vars] += q.modp;
01673                             t[3] = SGN(t[4+AN.poly_num_vars]);
01674                             t[4+AN.poly_num_vars] = ABS(t[4+AN.poly_num_vars]);
01675                         }
01676                     }
01677                     else {
01678                         // otherwise, use form long calculus
01679                         AddLong ((UWORD *)&p[4+AN.poly_num_vars], p[3],
01680                                 (UWORD *)&t[4+AN.poly_num_vars], t[3],
01681                                 (UWORD *)&t[4+AN.poly_num_vars], &t[3]);
01682                         if (q.modp!=0) TakeNormalModulus((UWORD *)&t[4+AN.poly_num_vars], &t[3],
01683                                                         modq, nmodq,
01684 NOUNPACK);
01685                     }
01686                 }
01687             }
01688             else {
01689                 // prepare adding an element of insert to the heap
01690                 this_insert = true;
01691
01692                 p[0] = insert.back().first;
01693                 p[1] = insert.back().second;
01694                 insert.pop_back();
01695
01696                 // add elements to the heap
01697                 while (true) {
01698                     // prepare the element
01699                     if (p[1]==0) {
01700                         p[0] += a[p[0]];
01701                         if (p[0]==a[0]) break;
01702                         WCOPY(&p[3], &a[p[0]], a[p[0]]);
01703                         p[3] = p[2+p[3]];
01704                     }
01705                     else {
01706                         if (!this_insert)
01707                             p[1] += q[p[1]];
01708                         this_insert = false;
01709
01710                         if (p[1]==qi) { s++; break; }
01711
01712                         for (int i=0; i<AN.poly_num_vars; i++)
01713                             p[4+i] = b[p[0]+1+i] + q[p[1]+1+i];
01714
01715                         // if both polynomials are modulo p^1, use integer calculus
01716                         if (both_mod_small) {
01717                             p[4+AN.poly_num_vars] = ((LONG)b[p[0]+1+AN.poly_num_vars]*b[p[0]+b[p[0]]-1]*
01718 q[p[1]+1+AN.poly_num_vars]*q[p[1]+q[p[1]]-1]) % q.modp;
01719                             if (p[4+AN.poly_num_vars]==0)
01720                                 p[3]=0;
01721                             else {
01722                                 if (p[4+AN.poly_num_vars] > +q.modp/2) p[4+AN.poly_num_vars] -= q.modp;
01723                                 if (p[4+AN.poly_num_vars] < -q.modp/2) p[4+AN.poly_num_vars] += q.modp;
01724                                 p[3] = SGN(p[4+AN.poly_num_vars]);
01725                                 p[4+AN.poly_num_vars] = ABS(p[4+AN.poly_num_vars]);
01726                             }
01727                         }
01728                     }
01729                 }
01730             }
01731         }
01732     }

```

```

01727         else {
01728             // otherwise, use form long calculus
01729             MulLong((UWORD *)&b[p[0]+1+AN.poly_num_vars], b[p[0]+b[p[0]]-1],
01730                     (UWORD *)&q[p[1]+1+AN.poly_num_vars], q[p[1]+q[p[1]]-1],
01731                     (UWORD *)&p[4+AN.poly_num_vars], &p[3]);
01732             if (q.modp!=0) TakeNormalModulus((UWORD *)&p[4+AN.poly_num_vars], &p[3],
01733                                             modq, nmodq,
NOUNPACK);
01734         }
01735
01736         p[3] *= -1;
01737     }
01738
01739     // no hashing
01740     p[2] = -1;
01741
01742     // add it to a heap element
01743     swap (heap[nheap],p);
01744     push_heap(BHEAD heap, ++nheap);
01745     break;
01746 }
01747 }
01748 while (t[0]==-1 || (nheap>0 && monomial_compare(BHEAD heap[0]+3, t+3)==0));
01749
01750 if (t[3] == 0) continue;
01751
01752 // check divisibility
01753 bool div = true;
01754 for (int i=0; i<AN.poly_num_vars; i++)
01755     if (t[4+i] < b[2+i]) div=false;
01756
01757 if (!div) {
01758     // not divisible, so append it to the remainder
01759     if (only_divides) { r=poly(BHEAD 1); ri=r[0]; break;}
01760     t[4 + AN.poly_num_vars + ABS(t[3])] = t[3];
01761     t[3] = 2 + AN.poly_num_vars + ABS(t[3]);
01762     r.termscopy(&t[3], ri, t[3]);
01763     ri += t[3];
01764 }
01765 else {
01766     // divisible, so divide coefficient as well
01767     WORD nq, nr;
01768
01769     if (q.modp==0) {
01770         DivLong((UWORD *)&t[4+AN.poly_num_vars], t[3],
01771                 (UWORD *)&b[2+AN.poly_num_vars], b[b[1]],
01772                 (UWORD *)&q[qi+1+AN.poly_num_vars], &nq,
01773                 (UWORD *)&r[ri+1+AN.poly_num_vars], &nr);
01774         if(check_div && nr != 0)
01775         {
01776             // non-zero remainder from coefficient division => result not expressible in
integers
01777             div_fail = true;
01778             break;
01779         }
01780     }
01781     else {
01782         // if both polynomials are modulo p^1, use integer calculus
01783         if (both_mod_small) {
01784             q[qi+1+AN.poly_num_vars] = ((LONG)t[4+AN.poly_num_vars]*t[3]*ltbinv[0]*nltbinv) %
q.modp;
01785             if (q[qi+1+AN.poly_num_vars]==0)
01786                 nq=0;
01787             else {
01788                 if (q[qi+1+AN.poly_num_vars] > +q.modp/2) q[qi+1+AN.poly_num_vars] -= q.modp;
01789                 if (q[qi+1+AN.poly_num_vars] < -q.modp/2) q[qi+1+AN.poly_num_vars] += q.modp;
01790                 nq = SGN(q[qi+1+AN.poly_num_vars]);
01791                 q[qi+1+AN.poly_num_vars] = ABS(q[qi+1+AN.poly_num_vars]);
01792             }
01793         }
01794         else {
01795             // otherwise, use form long calculus
01796             MulLong((UWORD *)&t[4+AN.poly_num_vars], t[3], ltbinv, nltbinv, (UWORD
*)&q[qi+1+AN.poly_num_vars], &nq);
01797             TakeNormalModulus((UWORD *)&q[qi+1+AN.poly_num_vars], &nq, modq, nmodq, NOUNPACK);
01798         }
01799
01800         nr=0;
01801     }
01802
01803     // add terms to quotient and remainder
01804     if (nq != 0) {
01805         int bi = 1;
01806         for (int j=1; j<s; j++) {
01807             bi += b[bi];
01808             insert.push_back(make_pair(bi, qi));
01809         }

```

```

01810             s=1;
01811
01812             q[qi] = 2+AN.poly_num_vars+ABS(nq);
01813             for (int i=0; i<AN.poly_num_vars; i++)
01814                 q[qi+1+i] = t[4+i] - b[2+i];
01815             qi += q[qi];
01816             q[qi-1] = nq;
01817         }
01818
01819         if (nr != 0) {
01820             r[ri] = 2+AN.poly_num_vars+ABS(nr);
01821             for (int i=0; i<AN.poly_num_vars; i++)
01822                 r[ri+1+i] = t[4+i];
01823             ri += r[ri];
01824             r[ri-1] = nr;
01825         }
01826     }
01827 }
01828
01829 q[0] = qi;
01830 r[0] = ri;
01831
01832 for (int i=0; i<nb; i++)
01833     NumberFree(heap[i], "poly::div_heap-b");
01834
01835 NumberFree(t, "poly::div_heap-c");
01836
01837 if (q.modp!=0 || ltbinv!=NULL) NumberFree(ltbinv, "poly::div_heap-a");
01838 AT.pWorkPointer = oldpWorkPointer;
01839 }
01840
01841 /*
01842 #] divmod_heap :
01843 #[ divmod :
01844 */
01845
01856 void poly::divmod (const poly &a, const poly &b, poly &q, poly &r, bool only_divides) {
01857     q.modp = r.modp = a.modp;
01858     q.modn = r.modn = a.modn;
01859
01860     if (a.is_zero()) {
01861         q[0]=1;
01862         r[0]=1;
01863         return;
01864     }
01865
01866     if (b.is_zero()) {
01867         MLOCK(ErrorMessageLock);
01868         MesPrint ((char*)"ERROR: division by zero polynomial");
01869         MUNLOCK(ErrorMessageLock);
01870         Terminate(1);
01871     }
01872
01873     if (b.is_one()) {
01874         q.check_memory(a[0]);
01875         q.termscopy(a.terms,0,a[0]);
01876         r[0]=1;
01877         return;
01878     }
01879
01880     if (b.is_monomial()) {
01881         divmod_one_term(a,b,q,r,only_divides);
01882         return;
01883     }
01884
01885     int vara = a.is_dense_univariate();
01886     int varb = b.is_dense_univariate();
01887
01888     if (vara!=-2 && varb!=-2 && (vara==--1 || varb==--1 || vara==varb)) {
01889         divmod_univar(a,b,q,r,Max(vara,varb),only_divides);
01890     }
01891     else {
01892         bool div_fail;
01893         divmod_heap(a,b,q,r,only_divides,false,div_fail);
01894     }
01895 }
01896
01897 /*
01898 #] divmod :
01899 #[ divides :
01900 */
01901 // returns whether a exactly divides b
01902 bool poly::divides (const poly &a, const poly &b) {
01903     POLY_GETIDENTITY(a);
01904     poly q(BHEAD 0), r(BHEAD 0);
01905     divmod(b,a,q,r,true);
01906     return r.is_zero();

```

```

01907 }
01908
01909 /*
01910     #] divides :
01911     #[ div :
01912 */
01913
01914 // the quotient of two polynomials
01915 void poly::div (const poly &a, const poly &b, poly &c) {
01916     POLY_GETIDENTITY(a);
01917     poly d(BHEAD 0);
01918     divmod(a,b,c,d,false);
01919 }
01920
01921 /*
01922     #] div :
01923     #[ mod :
01924 */
01925
01926 // the remainder of division of two polynomials
01927 void poly::mod (const poly &a, const poly &b, poly &c) {
01928     POLY_GETIDENTITY(a);
01929     poly d(BHEAD 0);
01930     divmod(a,b,c,d,false);
01931 }
01932
01933 /*
01934     #] mod :
01935     #[ copy operator :
01936 */
01937
01938 // copy operator
01939 poly & poly::operator= (const poly &a) {
01940     #ifdef POLY_MOVE_DEBUG
01941         std::cout << "poly copy assign" << std::endl;
01942     #endif
01943     if (&a != this) {
01944         modp = a.modp;
01945         modn = a.modn;
01946         check_memory(a[0]);
01947         termscopy(a.terms,0,a[0]);
01948     }
01949     return *this;
01950 }
01951
01952
01953 /*
01954     #] copy operator :
01955     #[ operator overloads :
01956 */
01957
01958 // binary operators for polynomial arithmetic
01959 const poly poly::operator+ (const poly &a) const { POLY_GETIDENTITY(a); poly b(BHEAD 0);
    add(*this,a,b); return b; }
01960 const poly poly::operator- (const poly &a) const { POLY_GETIDENTITY(a); poly b(BHEAD 0);
    sub(*this,a,b); return b; }
01961 const poly poly::operator* (const poly &a) const { POLY_GETIDENTITY(a); poly b(BHEAD 0);
    mul(*this,a,b); return b; }
01962 const poly poly::operator/ (const poly &a) const { POLY_GETIDENTITY(a); poly b(BHEAD 0);
    div(*this,a,b); return b; }
01963 const poly poly::operator% (const poly &a) const { POLY_GETIDENTITY(a); poly b(BHEAD 0);
    mod(*this,a,b); return b; }
01964
01965 // assignment operators for polynomial arithmetic
01966 poly& poly::operator+= (const poly &a) { return *this = *this + a; }
01967 poly& poly::operator-= (const poly &a) { return *this = *this - a; }
01968 poly& poly::operator*= (const poly &a) { return *this = *this * a; }
01969 poly& poly::operator/= (const poly &a) { return *this = *this / a; }
01970 poly& poly::operator%= (const poly &a) { return *this = *this % a; }
01971
01972 // comparison operators
01973 bool poly::operator== (const poly &a) const {
01974     for (int i=0; i<terms[0]; i++)
01975         if (terms[i] != a[i]) return 0;
01976     return 1;
01977 }
01978
01979 bool poly::operator!= (const poly &a) const { return !(*this == a); }
01980
01981 /*
01982     #] operator overloads :
01983     #[ num_terms :
01984 */
01985
01986 int poly::number_of_terms () const {
01987
01988     int res=0;

```

```

01989     for (int i=1; i<terms[0]; i+=terms[i])
01990         res++;
01991     return res;
01992 }
01993
01994 /*
01995     #] num_terms :
01996     #[ first_variable :
01997 */
01998
01999 // returns the lexicographically first variable of a polynomial
02000 int poly::first_variable () const {
02001
02002     POLY_GETIDENTITY(*this);
02003
02004     int var = AN.poly_num_vars;
02005     for (int j=0; j<var; j++)
02006         if (terms[2+j]>0) return j;
02007     return var;
02008 }
02009
02010 /*
02011     #] first_variable :
02012     #[ all_variables :
02013 */
02014
02015 // returns a list of all variables of a polynomial
02016 const vector<int> poly::all_variables () const {
02017
02018     POLY_GETIDENTITY(*this);
02019
02020     vector<bool> used(AN.poly_num_vars, false);
02021
02022     for (int i=1; i<terms[0]; i+=terms[i])
02023         for (int j=0; j<AN.poly_num_vars; j++)
02024             if (terms[i+1+j]>0) used[j] = true;
02025
02026     vector<int> vars;
02027     for (int i=0; i<AN.poly_num_vars; i++)
02028         if (used[i]) vars.push_back(i);
02029
02030     return vars;
02031 }
02032
02033 /*
02034     #] all_variables :
02035     #[ sign :
02036 */
02037
02038 // returns the sign of the leading coefficient
02039 int poly::sign () const {
02040     if (terms[0]==1) return 0;
02041     return terms[terms[1]] > 0 ? 1 : -1;
02042 }
02043
02044 /*
02045     #] sign :
02046     #[ degree :
02047 */
02048
02049 // returns the degree of x of a polynomial (deg=-1 iff a=0)
02050 int poly::degree (int x) const {
02051     int deg = -1;
02052     for (int i=1; i<terms[0]; i+=terms[i])
02053         deg = MaX(deg, terms[i+1+x]);
02054     return deg;
02055 }
02056
02057 /*
02058     #] degree :
02059     #[ total_degree :
02060 */
02061
02062 // returns the total degree of a polynomial (deg=-1 iff a=0)
02063 int poly::total_degree () const {
02064
02065     POLY_GETIDENTITY(*this);
02066
02067     int tot_deg = -1;
02068     for (int i=1; i<terms[0]; i+=terms[i]) {
02069         int deg=0;
02070         for (int j=0; j<AN.poly_num_vars; j++)
02071             deg += terms[i+1+j];
02072         tot_deg = MaX(deg, tot_deg);
02073     }
02074     return tot_deg;
02075 }

```

```

02076
02077 /*
02078     #] total_degree :
02079     #[ integer_lcoeff :
02080 */
02081
02082 // returns the integer coefficient of the leading monomial
02083 const poly poly::integer_lcoeff () const {
02084
02085     POLY_GETIDENTITY(*this);
02086
02087     poly res(BHEAD 0);
02088     res.modp = modp;
02089     res.modn = modn;
02090
02091     res.termscopy(&terms[1],1,terms[1]);
02092     res[0] = res[1] + 1; // length
02093     for (int i=0; i<AN.poly_num_vars; i++)
02094         res[2+i] = 0; // powers
02095
02096     return res;
02097 }
02098
02099 /*
02100     #] integer_lcoeff :
02101     #[ coefficient :
02102 */
02103
02104 // returns the polynomial coefficient of x^n
02105 const poly poly::coefficient (int x, int n) const {
02106
02107     POLY_GETIDENTITY(*this);
02108
02109     poly res(BHEAD 0);
02110     res.modp = modp;
02111     res.modn = modn;
02112     res[0] = 1;
02113
02114     for (int i=1; i<terms[0]; i+=terms[i])
02115         if (terms[i+1+x] == n) {
02116             res.check_memory(res[0]+terms[i]);
02117             res.termscopy(&terms[i], res[0], terms[i]);
02118             res[res[0]+1+x] -= n; // power of x
02119             res[0] += res[res[0]]; // length
02120         }
02121
02122     return res;
02123 }
02124
02125 /*
02126     #] coefficient :
02127     #[ lcoeff_multivar :
02128 */
02129
02130 // returns the leading coefficient with the polynomial viewed as a
02131 // polynomial in x, so that the result is a polynomial in the
02132 // remaining variables
02133 const poly poly::lcoeff_univar (int x) const {
02134     return coefficient(x, degree(x));
02135 }
02136
02137 // returns the leading coefficient with the polynomial viewed as a
02138 // polynomial with coefficients in ZZ[x], so that the result
02139 // polynomial is in ZZ[x] too
02140 const poly poly::lcoeff_multivar (int x) const {
02141
02142     POLY_GETIDENTITY(*this);
02143
02144     poly res(BHEAD 0,modp,modn);
02145
02146     for (int i=1; i<terms[0]; i+=terms[i]) {
02147         bool same_powers = true;
02148         for (int j=0; j<AN.poly_num_vars; j++)
02149             if (j!=x && terms[i+1+j]!=terms[2+j]) {
02150                 same_powers = false;
02151                 break;
02152             }
02153         if (!same_powers) break;
02154
02155         res.check_memory(res[0]+terms[i]);
02156         res.termscopy(&terms[i],res[0],terms[i]);
02157         for (int k=0; k<AN.poly_num_vars; k++)
02158             if (k!=x) res[res[0]+1+k]=0;
02159
02160         res[0] += terms[i];
02161     }
02162

```

```

02163     return res;
02164 }
02165
02166 /*
02167  #] lcoeff_multivar :
02168  #[ derivative :
02169  */
02170
02171 // returns the derivative with respect to x
02172 const poly poly::derivative (int x) const {
02173
02174     POLY_GETIDENTITY(*this);
02175
02176     poly b(BHEAD 0);
02177     int bi=1;
02178
02179     for (int i=1; i<terms[0]; i+=terms[i]) {
02180
02181         int power = terms[i+1+x];
02182
02183         if (power > 0) {
02184             b.check_memory(bi);
02185             b.termscopy(&terms[i], bi, terms[i]);
02186             b[bi+1+x]--;
02187
02188             WORD nb = b[bi+b[bi]-1];
02189             Product((UWORD *) &b[bi+1+AN.poly_num_vars], &nb, power);
02190
02191             b[bi] = 2 + AN.poly_num_vars + ABS(nb);
02192             b[bi+b[bi]-1] = nb;
02193
02194             bi += b[bi];
02195         }
02196     }
02197
02198     b[0] = bi;
02199     b.setmod(modp, modn);
02200     return b;
02201 }
02202
02203 /*
02204  #] derivative :
02205  #[ is_zero :
02206  */
02207
02208 // returns whether the polynomial is zero
02209 bool poly::is_zero () const {
02210     return terms[0] == 1;
02211 }
02212
02213 /*
02214  #] is_zero :
02215  #[ is_one :
02216  */
02217
02218 // returns whether the polynomial is one
02219 bool poly::is_one () const {
02220
02221     POLY_GETIDENTITY(*this);
02222
02223     if (terms[0] != 4+AN.poly_num_vars) return false;
02224     if (terms[1] != 3+AN.poly_num_vars) return false;
02225     for (int i=0; i<AN.poly_num_vars; i++)
02226         if (terms[2+i] != 0) return false;
02227     if (terms[2+AN.poly_num_vars] != 1) return false;
02228     if (terms[3+AN.poly_num_vars] != 1) return false;
02229
02230     return true;
02231 }
02232
02233 /*
02234  #] is_one :
02235  #[ is_integer :
02236  */
02237
02238 // returns whether the polynomial is an integer
02239 bool poly::is_integer () const {
02240
02241     POLY_GETIDENTITY(*this);
02242
02243     if (terms[0] == 1) return true;
02244     if (terms[0] != terms[1]+1) return false;
02245
02246     for (int j=0; j<AN.poly_num_vars; j++)
02247         if (terms[2+j] != 0)
02248             return false;
02249

```

```

02250     return true;
02251 }
02252
02253 /*
02254 #] is_integer :
02255 #[ is_monomial :
02256 */
02257
02258 // returns whether the polynomial consist of one term
02259 bool poly::is_monomial () const {
02260     return terms[0]>1 && terms[0]==terms[1]+1;
02261 }
02262
02263 /*
02264 #] is_monomial :
02265 #[ is_dense_univariate :
02266 */
02267
02284 int poly::is_dense_univariate () const {
02285     POLY_GETIDENTITY(*this);
02286
02287     int num_terms=0, res=-1;
02288
02289     // test univariate
02290     for (int i=1; i<terms[0]; i+=terms[i]) {
02291         for (int j=0; j<AN.poly_num_vars; j++)
02292             if (terms[i+1+j] > 0) {
02293                 if (res == -1) res = j;
02294                 if (res != j) return -2;
02295             }
02296         num_terms++;
02297     }
02298
02299     // constant polynomial
02300     if (res == -1) return -1;
02301
02302     // test density
02303     int deg = terms[2+res];
02304     if (2*num_terms < deg+1) return -2;
02305
02306     return res;
02307 }
02308
02309 /*
02310 #] is_dense_univariate :
02311 #[ simple_poly (small) :
02312 */
02313
02314 // returns the polynomial (x-a)^b mod p^n with a small
02315 const poly poly::simple_poly (PHEAD int x, int a, int b, int p, int n) {
02316     poly tmp(BHEAD 0,p,n);
02317
02318     int idx=1;
02319     tmp[idx++] = 3 + AN.poly_num_vars; // length
02320
02321     for (int i=0; i<AN.poly_num_vars; i++)
02322         tmp[idx++] = i==x ? 1 : 0; // powers
02323
02324     tmp[idx++] = 1; // coefficient
02325
02326     tmp[idx++] = 1; // length coefficient
02327
02328     if (a != 0) {
02329         tmp[idx++] = 3 + AN.poly_num_vars; // length
02330         for (int i=0; i<AN.poly_num_vars; i++) tmp[idx++] = 0; // powers
02331         tmp[idx++] = ABS(a); // coefficient
02332         tmp[idx++] = -SGN(a); // length coefficient
02333     }
02334
02335     tmp[0] = idx; // length
02336
02337     if (b == 1) return tmp;
02338
02339     poly res(BHEAD 1,p,n);
02340
02341     while (b!=0) {
02342         if (b&1) res*=tmp;
02343         tmp*=tmp;
02344         b>>=1;
02345     }
02346
02347     return res;
02348 }
02349
02350 /*
02351 #] simple_poly (small) :
02352 #[ simple_poly (large) :

```



```

02353 */
02354
02355 // returns the polynomial (x-a)^b mod p^n with a large
02356 const poly poly::simple_poly (PHEAD int x, const poly &a, int b, int p, int n) {
02357
02358     poly res(BHEAD 1,p,n);
02359     poly tmp(BHEAD 0,p,n);
02360
02361     int idx=1;
02362
02363     tmp[idx++] = 3 + AN.poly_num_vars; // length
02364     for (int i=0; i<AN.poly_num_vars; i++)
02365         tmp[idx++] = i==x ? 1 : 0; // powers
02366     tmp[idx++] = 1; // coefficient
02367     tmp[idx++] = 1; // length coefficient
02368
02369     if (!a.is_zero()) {
02370         tmp[idx++] = 2 + AN.poly_num_vars + ABS(a[a[0]-1]); // length
02371         for (int i=0; i<AN.poly_num_vars; i++) tmp[idx++] = 0; // powers
02372         for (int i=0; i<ABS(a[a[0]-1]); i++)
02373             tmp[idx++] = a[2 + AN.poly_num_vars + i]; // coefficient
02374         tmp[idx++] = -a[a[0]-1]; // length coefficient
02375     }
02376
02377     tmp[0] = idx; // length
02378
02379     while (b!=0) {
02380         if (b&1) res*=tmp;
02381         tmp*=tmp;
02382         b>>=1;
02383     }
02384
02385     return res;
02386 }
02387
02388 /*
02389 #] simple_poly (large) :
02390 #[ get_variables :
02391 */
02392
02393 // gets all variables in the expressions and stores them in AN.poly_vars
02394 // (it is assumed that AN.poly_vars=NULL)
02395 void poly::get_variables (PHEAD const vector<WORD *> &es, bool with_arghead, bool sort_vars) {
02396     assert(AN.poly_vars == NULL);
02397
02398     AN.poly_num_vars = 0;
02399
02400     vector<int> vars;
02401     vector<int> degrees;
02402     map<int,int> var_to_idx;
02403
02404     // extract all variables
02405     for (int ei=0; ei<(int)es.size(); ei++) {
02406         const WORD *e = es[ei];
02407
02408         // fast notation
02409         if (*e == -SNUMBER) {
02410         }
02411         else if (*e == -SYMBOL) {
02412             if (!var_to_idx.count(e[1])) {
02413                 vars.push_back(e[1]);
02414                 var_to_idx[e[1]] = AN.poly_num_vars++;
02415                 degrees.push_back(1);
02416             }
02417         }
02418         // JD: Here we need to check for non-symbol/number terms in fast notation.
02419         else if (*e < 0) {
02420             MLOCK(ErrorMessageLock);
02421             MesPrint ((char*)"ERROR: polynomials and polyratfuns must contain symbols only");
02422             MUNLOCK(ErrorMessageLock);
02423             Terminate(1);
02424         }
02425         else {
02426             for (int i=with_arghead ? ARGHEAD : 0; with_arghead ? i<e[0] : e[i]!=0; i+=e[i]) {
02427                 if (i+1<i+e[i]-ABS(e[i+e[i]-1]) && e[i+1]!=SYMBOL) {
02428                     MLOCK(ErrorMessageLock);
02429                     MesPrint ((char*)"ERROR: polynomials and polyratfuns must contain symbols only");
02430                     MUNLOCK(ErrorMessageLock);
02431                     Terminate(1);
02432                 }
02433
02434                 for (int j=i+3; j<i+e[i]-ABS(e[i+e[i]-1]); j+=2) {
02435                     if (!var_to_idx.count(e[j])) {
02436                         vars.push_back(e[j]);
02437                         var_to_idx[e[j]] = AN.poly_num_vars++;
02438                         degrees.push_back(e[j+1]);
02439                     }

```

```

02440         else {
02441             degrees[var_to_idx[e[j]]] = MaX(degrees[var_to_idx[e[j]]], e[j+1]);
02442         }
02443     }
02444 }
02445 }
02446 }
02447
02448 // make sure an eventual FACTORSYMBOL appear as last
02449 if (var_to_idx.count(FACTORSYMBOL)) {
02450     int i = var_to_idx[FACTORSYMBOL];
02451     while (i+1<AN.poly_num_vars) {
02452         swap(vars[i], vars[i+1]);
02453         swap(degrees[i], degrees[i+1]);
02454         i++;
02455     }
02456     degrees[i] = -1; // makes sure it stays last
02457 }
02458
02459 // AN.poly_vars will be deleted in calling functions from polywrap.c
02460 // For that we use the function poly_free_poly_vars
02461 // We add one to the allocation to avoid copying in poly_divmod
02462
02463 if ( (AN.poly_num_vars+1)*sizeof(WORD) > (size_t)(AM.MaxTer) ) {
02464     // This can happen only in expressions with excessively many variables.
02465     AN.poly_vars = (WORD *)Mallc1((AN.poly_num_vars+1)*sizeof(WORD), "AN.poly_vars");
02466     AN.poly_vars_type = 1;
02467 }
02468 else {
02469     AN.poly_vars = TermMalloc("AN.poly_vars");
02470     AN.poly_vars_type = 0;
02471 }
02472
02473 for (int i=0; i<AN.poly_num_vars; i++)
02474     AN.poly_vars[i] = vars[i];
02475
02476 if (sort_vars) {
02477     // bubble sort variables in decreasing order of degree
02478     // (this seems better for factorization)
02479     for (int i=0; i<AN.poly_num_vars; i++)
02480         for (int j=0; j+1<AN.poly_num_vars; j++)
02481             if (degrees[j] < degrees[j+1]) {
02482                 swap(degrees[j], degrees[j+1]);
02483                 swap(AN.poly_vars[j], AN.poly_vars[j+1]);
02484             }
02485 }
02486 else {
02487     // sort lexicographically
02488     sort(AN.poly_vars, AN.poly_vars+AN.poly_num_vars - var_to_idx.count(FACTORSYMBOL));
02489 }
02490 }
02491
02492 /*
02493 #] get_variables :
02494 #[ argument_to_poly :
02495 */
02496
02497 // converts a form expression to a polynomial class "poly"
02498 const poly poly::argument_to_poly (PHEAD WORD *e, bool with_arghead, bool sort_univar, poly *denpoly)
02499 {
02500     map<int,int> var_to_idx;
02501     for (int i=0; i<AN.poly_num_vars; i++)
02502         var_to_idx[AN.poly_vars[i]] = i;
02503
02504     poly res(BHEAD 0);
02505
02506     // fast notation
02507     if (*e == -SNUMBER) {
02508         if (denpoly!=NULL) *denpoly = poly(BHEAD 1);
02509
02510         if (e[1] == 0) {
02511             res[0] = 1;
02512             return res;
02513         }
02514         else {
02515             res[0] = 4 + AN.poly_num_vars;
02516             res[1] = 3 + AN.poly_num_vars;
02517             for (int i=0; i<AN.poly_num_vars; i++)
02518                 res[2+i] = 0;
02519             res[2+AN.poly_num_vars] = ABS(e[1]);
02520             res[3+AN.poly_num_vars] = SGN(e[1]);
02521
02522             return res;
02523         }
02524     }
02525 }

```

```

02526     if (*e == -SYMBOL) {
02527         if (denpoly!=NULL) *denpoly = poly(BHEAD 1);
02528
02529         res[0] = 4 + AN.poly_num_vars;
02530         res[1] = 3 + AN.poly_num_vars;
02531         for (int i=0; i<AN.poly_num_vars; i++)
02532             res[2+i] = 0;
02533         res[2+var_to_idx.find(e[1])>second] = 1;
02534         res[2+AN.poly_num_vars] = 1;
02535         res[3+AN.poly_num_vars] = 1;
02536         return res;
02537     }
02538
02539     // find LCM of denominators of all terms
02540     WORD nden=1, npro=0, ngcd=0, ndum=0;
02541     UWORD *den = NumberMalloc("poly::argument_to_poly");
02542     UWORD *pro = NumberMalloc("poly::argument_to_poly");
02543     UWORD *gcd = NumberMalloc("poly::argument_to_poly");
02544     UWORD *dum = NumberMalloc("poly::argument_to_poly");
02545     den[0]=1;
02546
02547     for (int i=with_arghead ? ARGHEAD : 0; with_arghead ? i<e[0] : e[i]!=0; i+=e[i]) {
02548         int ncoe = ABS(e[i+e[i]-1]/2);
02549         UWORD *coe = (UWORD *) &e[i+e[i]-ncoe-1];
02550         while (ncoe>0 && coe[ncoe-1]==0) ncoe--;
02551         Mullong(den, nden, coe, ncoe, pro, &npro);
02552         GcdLong(BHEAD den, nden, coe, ncoe, gcd, &ngcd);
02553         DivLong(pro, npro, gcd, ngcd, den, &nden, dum, &ndum);
02554     }
02555
02556     if (denpoly!=NULL) *denpoly = poly(BHEAD den, nden);
02557
02558     int ri=1;
02559
02560     // ordinary notation
02561     for (int i=with_arghead ? ARGHEAD : 0; with_arghead ? i<e[0] : e[i]!=0; i+=e[i]) {
02562         res.check_memory(ri);
02563         WORD nc = e[i+e[i]-1]; // length coefficient
02564         for (int j=0; j<AN.poly_num_vars; j++) // powers=0
02565             res[ri+1+j]=0; // coefficient to dummy
02566         WCOPY(dum, &e[i+e[i]-ABS(nc)], ABS(nc)); // remove denominator
02567         nc /= 2; // multiply with
02568         Mulli(BHEAD dum, &nc, den, nden);
02569         overall den
02570             res.termscopy((WORD *)dum, ri+1+AN.poly_num_vars, ABS(nc)); // coefficient to res
02571         res[ri] = ABS(nc) + AN.poly_num_vars + 2; // length
02572         res[ri+res[ri]-1] = nc; // length coefficient
02573         for (int j=i+3; j<i+e[i]-ABS(e[i+e[i]-1]); j+=2)
02574             res[ri+1+var_to_idx.find(e[j])>second] = e[j+1]; // powers
02575         ri += res[ri]; // length
02576     }
02577
02578     res[0] = ri;
02579
02580     // JD: For univariate cases, check whether the ordering is correct or not.
02581     // There are various ways to arrive here, but with an incorrect ordering, such
02582     // as interactions with collect, moving a polyratfun out of another function
02583     // argument, etc.
02584     // If the ordering is not correct, make sure we normalize. In multivariate cases,
02585     // always normalize as before.
02586     if (sort_univar == false && AN.poly_num_vars == 1) {
02587         if (res.number_of_terms() >= 2) {
02588             if (res[2] < res.last_monomial()[2]) {
02589                 sort_univar = true;
02590             }
02591         }
02592     }
02593
02594     if (sort_univar || AN.poly_num_vars>1) {
02595         res.normalize();
02596     }
02597
02598     NumberFree(den, "poly::argument_to_poly");
02599     NumberFree(pro, "poly::argument_to_poly");
02600     NumberFree(gcd, "poly::argument_to_poly");
02601     NumberFree(dum, "poly::argument_to_poly");
02602
02603     return res;
02604 }
02605
02606 /*
02607 #] argument_to_poly :
02608 #[ poly_to_argument :
02609 */
02610 // converts a polynomial class "poly" to a form expression
02611 void poly::poly_to_argument (const poly &a, WORD *res, bool with_arghead) {

```

```

02612
02613     POLY_GETIDENTITY(a);
02614
02615     // special case: a=0
02616     if (a[0]==1) {
02617         if (with_arghead) {
02618             res[0] = -SNUMBER;
02619             res[1] = 0;
02620         }
02621         else {
02622             res[0] = 0;
02623         }
02624         return;
02625     }
02626
02627     if (with_arghead) {
02628         res[1] = AN.poly_num_vars>1 ? DIRTYFLAG : 0; // dirty flag
02629         for (int i=2; i<ARGHEAD; i++)
02630             res[i] = 0; // remainder of arghead
02631     }
02632
02633     int L = with_arghead ? ARGHEAD : 0;
02634
02635     for (int i=1; i!=a[0]; i+=a[i]) {
02636
02637         res[L]=1; // length
02638
02639         bool first=true;
02640
02641         for (int j=0; j<AN.poly_num_vars; j++)
02642             if (a[i+1+j] > 0) {
02643                 if (first) {
02644                     first=false;
02645                     res[L+1] = 1; // symbols
02646                     res[L+2] = 2; // length
02647                 }
02648                 res[L+1+res[L+2]++] = AN.poly_vars[j]; // symbol
02649                 res[L+1+res[L+2]++] = a[i+1+j]; // power
02650             }
02651
02652         if (!first) res[L] += res[L+2]; // fix length
02653
02654         WORD nc = a[i+a[i]-1];
02655         WCOPY(&res[L+res[L]], &a[i+a[i]-1-ABS(nc)], ABS(nc)); // numerator
02656
02657         res[L] += ABS(nc); // fix length
02658         memset(&res[L+res[L]], 0, ABS(nc)*sizeof(WORD)); // denominator one
02659         res[L+res[L]] = 1; // denominator one
02660         res[L] += ABS(nc); // fix length
02661         res[L+res[L]] = SGN(nc) * (2*ABS(nc)+1); // length of coefficient
02662         res[L]++; // fix length
02663         L += res[L]; // fix length
02664     }
02665
02666     if (with_arghead) {
02667         res[0] = L;
02668         // convert to fast notation if possible
02669         ToFast(res,res);
02670     }
02671     else {
02672         res[L] = 0;
02673     }
02674 }
02675
02676 /*
02677 #] poly_to_argument :
02678 #[ poly_to_argument_with_den :
02679 */
02680
02681 // converts a polynomial class "poly" divided by a number (nden, den) to a form expression
02682 // cf. poly::poly_to_argument()
02683 void poly::poly_to_argument_with_den (const poly &a, WORD nden, const UWORD *den, WORD *res, bool
    with_arghead) {
02684
02685     POLY_GETIDENTITY(a);
02686
02687     // special case: a=0
02688     if (a[0]==1) {
02689         if (with_arghead) {
02690             res[0] = -SNUMBER;
02691             res[1] = 0;
02692         }
02693         else {
02694             res[0] = 0;
02695         }
02696         return;
02697     }

```

```

02698
02699     if (with_arghead) {
02700         res[1] = AN.poly_num_vars>1 ? DIRTYFLAG : 0; // dirty flag
02701         for (int i=2; i<ARGHEAD; i++)
02702             res[i] = 0; // remainder of arghead
02703     }
02704
02705     WORD nden1;
02706     UWORD *den1 = (UWORD *)NumberMalloc("poly_to_argument_with_den");
02707
02708     int L = with_arghead ? ARGHEAD : 0;
02709
02710     for (int i=1; i!=a[0]; i+=a[i]) {
02711
02712         res[L]=1; // length
02713
02714         bool first=true;
02715
02716         for (int j=0; j<AN.poly_num_vars; j++)
02717             if (a[i+1+j] > 0) {
02718                 if (first) {
02719                     first=false;
02720                     res[L+1] = 1; // symbols
02721                     res[L+2] = 2; // length
02722                 }
02723                 res[L+1+res[L+2]++] = AN.poly_vars[j]; // symbol
02724                 res[L+1+res[L+2]++] = a[i+1+j]; // power
02725             }
02726
02727         if (!first) res[L] += res[L+2]; // fix length
02728
02729         // numerator
02730         WORD nc = a[i+a[i]-1];
02731         WCOPY(&res[L+res[L]], &a[i+a[i]-1-ABS(nc)], ABS(nc));
02732
02733         // denominator
02734         nden1 = nden;
02735         WCOPY(den1, den, ABS(nden));
02736
02737         if (nden != 1 || den[0] != 1) {
02738             // remove gcd(num,den)
02739             Simplify(BHEAD (UWORD *)&res[L+res[L]], &nc, den1, &nden1);
02740         }
02741
02742         Pack((UWORD *)&res[L+res[L]], &nc, den1, nden1); // format
02743         res[L] += 2*ABS(nc)+1; // fix length
02744         res[L+res[L]-1] = SGN(nc)*(2*ABS(nc)+1); // length of coefficient
02745         L += res[L]; // fix length
02746     }
02747
02748     NumberFree(den1, "poly_to_argument_with_den");
02749
02750     if (with_arghead) {
02751         res[0] = L;
02752         // convert to fast notation if possible
02753         ToFast(res,res);
02754     }
02755     else {
02756         res[L] = 0;
02757     }
02758 }
02759
02760 /*
02761 #] poly_to_argument_with_den :
02762 #[ size_of_form_notation :
02763 */
02764
02765 // the size of the polynomial in form notation (without argheads and fast notation)
02766 int poly::size_of_form_notation () const {
02767
02768     POLY_GETIDENTITY(*this);
02769
02770     // special case: a=0
02771     if (terms[0]==1) return 0;
02772
02773     int len = 0;
02774
02775     for (int i=1; i!=terms[0]; i+=terms[i]) {
02776         len++;
02777         int npow = 0;
02778         for (int j=0; j<AN.poly_num_vars; j++)
02779             if (terms[i+1+j] > 0) npow++;
02780         if (npow > 0) len += 2*npow + 2;
02781         len += 2 * ABS(terms[i+terms[i]-1]) + 1;
02782     }
02783
02784     return len;

```

```

02785 }
02786
02787 /*
02788 #] size_of_form_notation :
02789 #[ size_of_form_notation_with_den :
02790 */
02791
02792 // the size of the polynomial divided by a number (its size is given by nden)
02793 // in form notation (without argheads and fast notation)
02794 // cf. poly::size_of_form_notation()
02795 int poly::size_of_form_notation_with_den (WORD nden) const {
02796
02797     POLY_GETIDENTITY(*this);
02798
02799     // special case: a=0
02800     if (terms[0]==1) return 0;
02801
02802     nden = ABS(nden);
02803     int len = 0;
02804
02805     for (int i=1; i!=terms[0]; i+=terms[i]) {
02806         len++;
02807         int npow = 0;
02808         for (int j=0; j<AN.poly_num_vars; j++)
02809             if (terms[i+1+j] > 0) npow++;
02810         if (npow > 0) len += 2*npow + 2;
02811         len += 2 * MaX(ABS(terms[i+terms[i]-1]), nden) + 1;
02812     }
02813
02814     return len;
02815 }
02816
02817 /*
02818 #] size_of_form_notation_with_den :
02819 #[ to_coefficient_list :
02820 */
02821
02822 // returns the coefficient list of a univariate polynomial
02823 const vector<WORD> poly::to_coefficient_list (const poly &a) {
02824
02825     POLY_GETIDENTITY(a);
02826
02827     if (a.is_zero()) return vector<WORD>();
02828
02829     int x = a.first_variable();
02830     if (x == AN.poly_num_vars) x=0;
02831
02832     vector<WORD> res(1+a[2+x],0);
02833
02834     for (int i=1; i<a[0]; i+=a[i])
02835         res[a[i+1+x]] = (a[i+a[i]-1] * a[i+a[i]-2]) % a.modp;
02836
02837     return res;
02838 }
02839
02840 /*
02841 #] to_coefficient_list :
02842 #[ coefficient_list_divmod :
02843 */
02844
02845 // divides two polynomials represented by coefficient lists
02846 const vector<WORD> poly::coefficient_list_divmod (const vector<WORD> &a, const vector<WORD> &b, WORD
p, int divmod) {
02847
02848     int bsize = (int)b.size();
02849     while (b[bsize-1]==0) bsize--;
02850
02851     WORD inv;
02852     GetModInverses(b[bsize-1] + (b[bsize-1]<0?p:0), p, &inv, NULL);
02853
02854     vector<WORD> q(a.size(),0);
02855     vector<WORD> r(a);
02856
02857     while ((int)r.size() >= bsize) {
02858         LONG mul = ((LONG)inv * r.back()) % p;
02859         int off = r.size()-bsize;
02860         q[off] = mul;
02861         for (int i=0; i<bsize; i++)
02862             r[off+i] = (r[off+i] - mul*b[i]) % p;
02863         while (r.size()>0 && r.back()==0)
02864             r.pop_back();
02865     }
02866
02867     if (divmod==0) {
02868         while (q.size()>0 && q.back()==0)
02869             q.pop_back();
02870         return q;

```

```

02871     }
02872     else {
02873         while (r.size()>0 && r.back()==0)
02874             r.pop_back();
02875         return r;
02876     }
02877 }
02878
02879 /*
02880  #] coefficient_list_divmod :
02881  #[ from_coefficient_list :
02882  */
02883
02884 // converts a coefficient list to a "poly"
02885 const poly poly::from_coefficient_list (PHEAD const vector<WORD> &a, int x, WORD p) {
02886     poly res(BHEAD 0);
02887     int ri=1;
02889     for (int i=(int)a.size()-1; i>=0; i--)
02891         if (a[i] != 0) {
02892             res.check_memory(ri);
02893             res[ri] = AN.poly_num_vars+3; // length
02894             for (int j=0; j<AN.poly_num_vars; j++)
02895                 res[ri+1+j]=0; // powers
02896             res[ri+1+x] = i; // power of x
02897             res[ri+1+AN.poly_num_vars] = ABS(a[i]); // coefficient
02898             res[ri+1+AN.poly_num_vars+1] = SGN(a[i]); // sign
02899             ri += res[ri];
02900         }
02901
02902     res[0]=ri; // total length
02903     res.setmod(p,1);
02904
02905     return res;
02906 }
02907
02908 /*
02909  #] from_coefficient_list :
02910  */

```

4.101 poly.h

```

00001 #pragma once
00002 /* #] License : */
00003 /*
00004  * Copyright (C) 1984-2023 J.A.M. Vermaseren
00005  * When using this file you are requested to refer to the publication
00006  * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00007  * This is considered a matter of courtesy as the development was paid
00008  * for by FOM the Dutch physics granting agency and we would like to
00009  * be able to track its scientific use to convince FOM of its value
00010  * for the community.
00011  *
00012  * This file is part of FORM.
00013  *
00014  * FORM is free software: you can redistribute it and/or modify it under the
00015  * terms of the GNU General Public License as published by the Free Software
00016  * Foundation, either version 3 of the License, or (at your option) any later
00017  * version.
00018  *
00019  * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00020  * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00021  * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00022  * details.
00023  *
00024  * You should have received a copy of the GNU General Public License along
00025  * with FORM. If not, see <http://www.gnu.org/licenses/>.
00026  */
00027 /* #] License : */
00028
00029 extern "C" {
00030 #include "form3.h"
00031 }
00032
00033 #include <iostream>
00034 #include <string>
00035 #include <vector>
00036
00037 // for debugging
00038 // #define POLY_MOVE_DEBUG
00039
00040 // macros for tform

```

```

00041 #ifndef WITHPTHREADS
00042 #define POLY_GETIDENTITY(X)
00043 #define POLY_STOREIDENTITY
00044 #else
00045 #define POLY_GETIDENTITY(X) ALLPRIVATES *B = (X).Bpointer
00046 #define POLY_STOREIDENTITY Bpointer = B
00047 #endif
00048
00049 // maximum size of the hash table used for multiplication and division
00050
00051 const int POLY_MAX_HASH_SIZE = Min(1<<20, MAXPOSITIVE);
00052
00053 class poly {
00054 public:
00055     // variables
00056     #ifdef WITHPTHREADS
00057         ALLPRIVATES *Bpointer;
00058     #endif
00059     WORD *terms;
00060     LONG size_of_terms;
00061     WORD modp, modn;
00062
00063     // constructors/destructor
00064     poly (PHEAD int, WORD=-1, WORD=1);
00065     poly (PHEAD const UWORD *, WORD, WORD=-1, WORD=1);
00066     poly (const poly &, WORD=-1, WORD=1);
00067     ~poly ();
00068
00069     // operators
00070     poly& operator+= (const poly&);
00071     poly& operator-= (const poly&);
00072     poly& operator*= (const poly&);
00073     poly& operator/= (const poly&);
00074     poly& operator%= (const poly&);
00075
00076     const poly operator+ (const poly&) const;
00077     const poly operator- (const poly&) const;
00078     const poly operator* (const poly&) const;
00079     const poly operator/ (const poly&) const;
00080     const poly operator% (const poly&) const;
00081
00082     bool operator== (const poly&) const;
00083     bool operator!= (const poly&) const;
00084     poly& operator= (const poly &);
00085     WORD& operator[] (int);
00086     const WORD& operator[] (int) const;
00087
00088     #if __cplusplus >= 201103L
00089         // support move semantics
00090         poly (poly &&, WORD=-1, WORD=1) noexcept;
00091         poly& operator= (poly &&) noexcept;
00092     #endif
00093
00094     // memory management
00095     void termscopy (const WORD *, int, int);
00096     void check_memory(int);
00097     void expand_memory(int);
00098
00099     // check type of polynomial
00100     bool is_zero () const;
00101     bool is_one () const;
00102     bool is_integer () const;
00103     bool is_monomial () const;
00104     int is_dense_univariate () const;
00105
00106     // properties
00107     int sign () const;
00108     int degree (int) const;
00109     int total_degree () const;
00110     int first_variable () const;
00111     int number_of_terms () const;
00112     const std::vector<int> all_variables () const;
00113     const poly integer_lcoeff () const;
00114     const poly lcoeff_univar (int) const;
00115     const poly lcoeff_multivar (int) const;
00116     const poly coefficient (int, int) const;
00117     const poly derivative (int) const;
00118
00119     // modulo calculus
00120     void setmod(WORD, WORD=1);
00121     void coefficients_modulo (UWORD *, WORD, bool);
00122
00123     // simple polynomials
00124     static const poly simple_poly (PHEAD int, int=0, int=1, int=0, int=1);

```



```

00128     static const poly simple_poly (PHEAD int, const poly&, int=1, int=0, int=1);
00129
00130     // conversion from/to form notation
00131     static void get_variables (PHEAD const std::vector<WORD *> &, bool, bool);
00132     static const poly argument_to_poly (PHEAD WORD *, bool, bool, poly *den=NULL);
00133     static void poly_to_argument (const poly &, WORD *, bool);
00134     static void poly_to_argument_with_den (const poly &, WORD, const UWORD *, WORD *, bool);
00135     int size_of_form_notation () const;
00136     int size_of_form_notation_with_den (WORD) const;
00137     const poly & normalize ();
00138
00139     // operations for coefficient lists
00140     static const poly from_coefficient_list (PHEAD const std::vector<WORD> &, int, WORD);
00141     static const std::vector<WORD> to_coefficient_list (const poly &);
00142     static const std::vector<WORD> coefficient_list_divmod (const std::vector<WORD> &, const
std::vector<WORD> &, WORD, int);
00143
00144     // string output for debugging
00145     const std::string to_string() const;
00146
00147     // monomials
00148     static int monomial_compare (PHEAD const WORD *, const WORD *);
00149     WORD last_monomial_index () const;
00150     WORD* last_monomial () const;
00151     int compare_degree_vector(const poly &) const;
00152     std::vector<int> degree_vector() const;
00153     int compare_degree_vector(const std::vector<int> &) const;
00154
00155     // (internal) mathematical operations
00156     static void add (const poly &, const poly &, poly &);
00157     static void sub (const poly &, const poly &, poly &);
00158     static void mul (const poly &, const poly &, poly &);
00159     static void div (const poly &, const poly &, poly &);
00160     static void mod (const poly &, const poly &, poly &);
00161     static void divmod (const poly &, const poly &, poly &, poly &, bool only_divides);
00162     static bool divides (const poly &, const poly &);
00163
00164     static void mul_one_term (const poly &, const poly &, poly &);
00165     static void mul_univar (const poly &, const poly &, poly &, int);
00166     static void mul_heap (const poly &, const poly &, poly &);
00167
00168     static void divmod_one_term (const poly &, const poly &, poly &, poly &, bool);
00169     static void divmod_univar (const poly &, const poly &, poly &, poly &, int, bool);
00170     static void divmod_heap (const poly &, const poly &, poly &, poly &, bool, bool, bool&);
00171
00172     static void push_heap (PHEAD WORD **, int);
00173     static void pop_heap (PHEAD WORD **, int);
00174
00175     PADPOINTER(1,0,2,0);
00176 };
00177
00178 // comparison class for monomials (for std::sort)
00179 class monomial_larger {
00180
00181 public:
00182
00183     #ifndef WITHPTHREADS
00184         monomial_larger() {}
00185     #else
00186         ALLPRIVATES *B;
00187         monomial_larger(ALLPRIVATES *b): B(b) {}
00188     #endif
00189
00190     bool operator()(const WORD *a, const WORD *b) {
00191         return poly::monomial_compare(BHEAD a, b) > 0;
00192     }
00193 };
00194
00195 // stream operator
00196 std::ostream& operator<< (std::ostream &, const poly &);
00197
00198 // inline function definitions
00199
00200 #if __cplusplus >= 201103L
00201
00202 inline poly::poly (poly &&a, WORD modp, WORD modn) noexcept {
00203     #ifdef POLY_MOVE_DEBUG
00204         std::cout << "poly move ctor" << std::endl;
00205     #endif
00206     POLY_GETIDENTITY(a);
00207     POLY_STOREIDENTITY;
00208     terms = a.terms;
00209     size_of_terms = a.size_of_terms;
00210     this->modp = a.modp;
00211     this->modn = a.modn;
00212     if (modp != -1) {
00213         setmod(modp,modn);

```

```

00214     }
00215     a.terms = nullptr;
00216     a.size_of_terms = -1;
00217 }
00218
00219 inline poly& poly::operator= (poly &a) noexcept {
00220 #ifdef POLY_MOVE_DEBUG
00221     std::cout << "poly move assign" << std::endl;
00222 #endif
00223     if (this != &a) {
00224         WORD *old_terms = terms;
00225         LONG old_size_of_terms = size_of_terms;
00226         terms = a.terms;
00227         size_of_terms = a.size_of_terms;
00228         modp = a.modp;
00229         modn = a.modn;
00230         a.terms = old_terms;
00231         a.size_of_terms = old_size_of_terms;
00232     }
00233     return *this;
00234 }
00235
00236 #endif
00237
00238 /* Checks whether the terms array is large enough to add another
00239 * term (of size AM.MaxTal) to the polynomials. In case not, it is
00240 * expanded.
00241 */
00242 inline void poly::check_memory (int i) {
00243     POLY_GETIDENTITY(*this);
00244     if (i + 3 + AN.poly_num_vars + AM.MaxTal >= size_of_terms) expand_memory(i + AM.MaxTal);
00245 // Used to be i+2 but there should also be space for a trailing zero
00246 }
00247
00248 // indexing operators
00249 inline WORD& poly::operator[] (int i) {
00250     return terms[i];
00251 }
00252
00253 inline const WORD& poly::operator[] (int i) const {
00254     return terms[i];
00255 }
00256
00257 /* Copies "num" WORD-sized terms from the pointer "source" to the
00258 * current polynomial at index "dest"
00259 */
00260 inline void poly::termscopy (const WORD *source, int dest, int num) {
00261 #ifdef POLY_MOVE_DEBUG
00262     std::cout << "poly termscopy: num=" << num << std::endl;
00263 #endif
00264     memcpy (terms+dest, source, num*sizeof(WORD));
00265 }

```

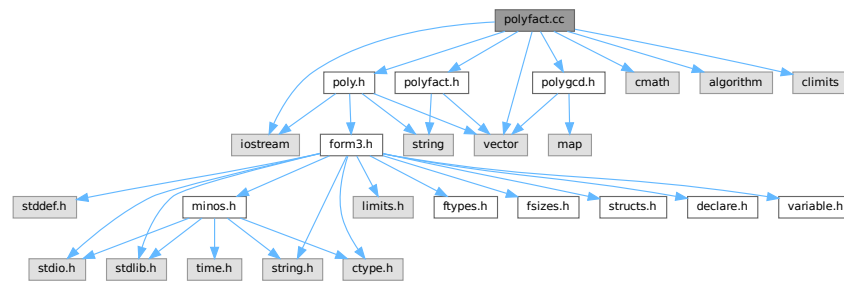
4.102 polyfact.cc File Reference

```

#include "poly.h"
#include "polygcd.h"
#include "polyfact.h"
#include <cmath>
#include <vector>
#include <iostream>
#include <algorithm>
#include <climits>

```

Include dependency graph for polyfact.cc:



Functions

- ostream & operator<< (ostream &out, const [factorized_poly](#) &a)
- template<class T >
ostream & operator<< (ostream &out, const vector< T > &v)

4.102.1 Detailed Description

Contains the routines for factorizing multivariate polynomials

Definition in file [polyfact.cc](#).

4.102.2 Function Documentation

4.102.2.1 operator<<() [1/2]

```
ostream & operator<< (
    ostream & out,
    const factorized\_poly & a )
```

Definition at line 104 of file [polyfact.cc](#).

4.102.2.2 operator<<() [2/2]

```
template<class T >
ostream & operator<< (
    ostream & out,
    const vector< T > & v )
```

Definition at line 109 of file [polyfact.cc](#).

4.103 polyfact.cc

[Go to the documentation of this file.](#)

```

00001
00005 /* #[ License : */
00006 /*
00007 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00008 *   When using this file you are requested to refer to the publication
00009 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00010 *   This is considered a matter of courtesy as the development was paid
00011 *   for by FOM the Dutch physics granting agency and we would like to
00012 *   be able to track its scientific use to convince FOM of its value
00013 *   for the community.
00014 *
00015 *   This file is part of FORM.
00016 *
00017 *   FORM is free software: you can redistribute it and/or modify it under the
00018 *   terms of the GNU General Public License as published by the Free Software
00019 *   Foundation, either version 3 of the License, or (at your option) any later
00020 *   version.
00021 *
00022 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00023 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00024 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00025 *   details.
00026 *
00027 *   You should have received a copy of the GNU General Public License along
00028 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00029 */
00030 /* #[ License : */
00031 /*
00032 *   #[ include :
00033 */
00034
00035 #include "poly.h"
00036 #include "polygcd.h"
00037 #include "polyfact.h"
00038
00039 #include <cmath>
00040 #include <vector>
00041 #include <iostream>
00042 #include <algorithm>
00043 #include <climits>
00044
00045 // #define DEBUG
00046
00047 #ifdef DEBUG
00048 #include "mytime.h"
00049 #endif
00050
00051 using namespace std;
00052
00053 /*
00054 *   #[ include :
00055 *   #[ tostring :
00056 */
00057
00058 // Turns a factorized_poly into a readable string
00059 const string factorized_poly::tostring () const {
00060
00061     // empty
00062     if (factor.size()==0)
00063         return "no_factors";
00064
00065     string res;
00066
00067     // polynomial
00068     for (int i=0; i<(int)factor.size(); i++) {
00069         if (i>0) res += "*";
00070         res += "(";
00071         res += poly(factor[i],0,1).to_string();
00072         res += ")";
00073         if (power[i]>1) {
00074             res += "^";
00075             char tmp[100];
00076             snprintf (tmp,100,"%i",power[i]);
00077             res += tmp;
00078         }
00079     }
00080
00081     // modulo p^n
00082     if (factor[0].modp>0) {
00083         res += " (mod ";
00084         char tmp[12];
00085         snprintf (tmp,12,"%i",factor[0].modp);

```

```

00086         res += tmp;
00087         if (factor[0].modn > 1) {
00088             snprintf(tmp,12,"%i",factor[0].modn);
00089             res += "^";
00090             res += tmp;
00091         }
00092         res += ")";
00093     }
00094
00095     return res;
00096 }
00097
00098 /*
00099  #] toString :
00100  #[ ostream operator :
00101  */
00102
00103 // ostream operator for outputting a factorized_poly
00104 ostream& operator<< (ostream &out, const factorized_poly &a) {
00105     return out << a.toString();
00106 }
00107
00108 // ostream operator for outputting a vector<T>
00109 template<class T> ostream& operator<< (ostream &out, const vector<T> &v) {
00110     out<<"{";
00111     for (int i=0; i<(int)v.size(); i++) {
00112         if (i>0) out<<",";
00113         out<<v[i];
00114     }
00115     out<<"}";
00116     return out;
00117 }
00118
00119 /*
00120  #] ostream operator :
00121  #[ add_factor :
00122  */
00123
00124 // adds a factor f^p to a factorization
00125 void factorized_poly::add_factor(const poly &f, int p) {
00126     factor.push_back(f);
00127     power.push_back(p);
00128 }
00129
00130 /*
00131  #] add_factor :
00132  #[ extended_gcd_Euclidean_lifted :
00133  */
00134
00152 const vector<poly> polyfact::extended_gcd_Euclidean_lifted (const poly &a, const poly &b) {
00153
00154 #ifdef DEBUGALL
00155     cout << "*** [" << thetime() << " ] CALL: extended_Euclidean_lifted(" <<a<<","<<b<<")<<endl;
00156 #endif
00157
00158     POLY_GETIDENTITY(a);
00159
00160     // Calculate s,t,gcd (mod p) with the Extended Euclidean Algorithm.
00161     poly s(a,a.modp,1);
00162     poly t(b,b.modp,1);
00163     poly sa(BHEAD 1,a.modp,1);
00164     poly sb(BHEAD 0,b.modp,1);
00165     poly ta(BHEAD 0,a.modp,1);
00166     poly tb(BHEAD 1,b.modp,1);
00167
00168     while (!t.is_zero()) {
00169         poly x(s/t);
00170         swap(s -=x*t , t);
00171         swap(sa-=x*ta, ta);
00172         swap(sb-=x*tb, tb);
00173     }
00174
00175     // Normalize the result.
00176     sa /= s.integer_lcoeff();
00177     sb /= s.integer_lcoeff();
00178
00179     // Lift the result to modulo p^n with p-adic Newton's iteration.
00180     poly samodp(sa);
00181     poly sbmodp(sb);
00182     poly term(BHEAD 1);
00183
00184     sa.setmod(0,1);
00185     sb.setmod(0,1);
00186
00187     poly amodp(a,a.modp,1);
00188     poly bmodp(b,a.modp,1);
00189     poly error(poly(BHEAD 1) - sa*a - sb*b);

```

```

00190     poly p(BHEAD a.modp);
00191
00192     for (int n=1; n<a.modn && !error.is_zero(); n++) {
00193         error /= p;
00194         term *= p;
00195         poly errormodp(error,a.modp,1);
00196         poly dsa((samodp * errormodp) % bmodp);
00197         // equivalent, but faster than the symmetric
00198         // poly dsb((sbmodp * errormodp) % amodp);
00199         poly dsb((errormodp - dsa*amodp) / bmodp);
00200         sa += term * dsa;
00201         sb += term * dsb;
00202         error -= a*dsa + b*dsb;
00203     }
00204
00205     sa.setmod(a.modp,a.modn);
00206     sb.setmod(a.modp,a.modn);
00207
00208     // Output the result
00209     vector<poly> res;
00210     res.push_back(sa);
00211     res.push_back(sb);
00212
00213 #ifdef DEBUGALL
00214     cout << "*** [" << thetime() << "]" RES : extended_Euclidean_lifted("«a«","«b«") = "«res«endl;
00215 #endif
00216
00217     return res;
00218 }
00219
00220 /*
00221 #] extended_gcd_Euclidean_lifted :
00222 #[ solve_Diophantine_univariate :
00223 */
00224
00225 const vector<poly> polyfact::solve_Diophantine_univariate (const vector<poly> &a, const poly &b) {
00226
00227 #ifdef DEBUGALL
00228     cout << "*** [" << thetime() << "]" CALL: solve_Diophantine_univariate(" «a«","«b«")«endl;
00229 #endif
00230
00231     POLY_GETIDENTITY(b);
00232
00233     vector<poly> s(1,b);
00234
00235     for (int i=0; i+1<(int)a.size(); i++) {
00236         poly A(BHEAD 1,b.modp,b.modn);
00237         for (int j=i+1; j<(int)a.size(); j++) A *= a[j];
00238
00239         vector<poly> t(extended_gcd_Euclidean_lifted(a[i],A));
00240         poly prev(s.back());
00241         s.back() = t[1] * prev % a[i];
00242         s.push_back(t[0] * prev % A);
00243     }
00244
00245 #ifdef DEBUGALL
00246     cout << "*** [" << thetime() << "]" RES : solve_Diophantine_univariate(" «a«","«b«") = "«s«endl;
00247 #endif
00248
00249     return s;
00250 }
00251
00252 /*
00253 #] solve_Diophantine_univariate :
00254 #[ solve_Diophantine_multivariate :
00255 */
00256
00257 const vector<poly> polyfact::solve_Diophantine_multivariate (const vector<poly> &a, const poly &b,
00258     const vector<int> &x, const vector<int> &c, int d) {
00259
00260 #ifdef DEBUGALL
00261     cout << "*** [" << thetime() << "]" CALL: solve_Diophantine_multivariate("
00262 «a«","«b«","«x«","«c«","«d«")«endl;
00263 #endif
00264
00265     POLY_GETIDENTITY(b);
00266
00267     if (b.is_zero()) return vector<poly>(a.size(),poly(BHEAD 0));
00268
00269     if (x.size() == 1) return solve_Diophantine_univariate(a,b);
00270
00271     // Reduce the polynomial with the homomorphism <xm-c{m-1}>
00272     poly simple(poly::simple_poly(BHEAD x.back(),c.back()));
00273
00274     vector<poly> ared(a);
00275     for (int i=0; i<(int)ared.size(); i++)
00276         ared[i] %= simple;

```

```

00337
00338     poly bred(b % simple);
00339     vector<int> xred(x.begin(), x.end()-1);
00340     vector<int> cred(c.begin(), c.end()-1);
00341
00342     // Solve the equation in one less variable
00343     vector<poly> s(solve_Diophantine_multivariate(ared, bred, xred, cred, d));
00344     if (s == vector<poly>()) return vector<poly>();
00345
00346     // Cache the  $A_i = \text{product}(a_j \mid j \neq i)$ .
00347     vector<poly> A(a.size(), poly(BHEAD 1, b.modp, b.modn));
00348     for (int i=0; i<(int)a.size(); i++)
00349         for (int j=0; j<(int)a.size(); j++)
00350             if (i!=j) A[i] *= a[j];
00351
00352     // Add the powers  $(x_m - c_{m-1})^k$  with ideal-adic Newton iteration.
00353     poly term(BHEAD 1, b.modp, b.modn);
00354
00355     poly error(b);
00356     for (int i=0; i<(int)A.size(); i++)
00357         error -= s[i] * A[i];
00358
00359     for (int deg=1; deg<=d; deg++) {
00360
00361         if (error.is_zero()) break;
00362
00363         error /= simple;
00364         term *= simple;
00365
00366         vector<poly> ds(solve_Diophantine_multivariate(ared, error%simple, xred, cred, d));
00367         if (ds == vector<poly>()) return vector<poly>();
00368
00369         for (int i=0; i<(int)s.size(); i++) {
00370             s[i] += ds[i] * term;
00371             error -= ds[i] * A[i];
00372         }
00373     }
00374
00375     if (!error.is_zero()) return vector<poly>();
00376
00377 #ifdef DEBUGALL
00378     cout << "*** [" << thetime() << "] RES : solve_Diophantine_multivariate("
00379         << a << ", " << b << ", " << x << ", " << c << ", " << d << ") = " << s << endl;
00379 #endif
00380
00381     return s;
00382 }
00383
00384 /*
00385 #] solve_Diophantine_multivariate :
00386 #[ lift_coefficients :
00387 */
00388
00421 const vector<poly> polyfact::lift_coefficients (const poly &A, const vector<poly> &a) {
00422
00423 #ifdef DEBUG
00424     cout << "*** [" << thetime() << "] CALL: lift_coefficients(" << A << ", " << a << ")" << endl;
00425 #endif
00426
00427     POLY_GETIDENTITY(A);
00428
00429     poly A(A);
00430     vector<poly> a(a);
00431     poly term(BHEAD 1);
00432
00433     int x = A.first_variable();
00434
00435     // Replace the leading term of all factors with lterm(A) mod p
00436     poly lead(A.integer_lcoeff());
00437     for (int i=0; i<(int)a.size(); i++) {
00438         a[i] *= lead / a[i].integer_lcoeff();
00439         if (i>0) A*=lead;
00440     }
00441
00442     // Solve Diophantine equation
00443     vector<poly> s(solve_Diophantine_univariate(a, poly(BHEAD 1, A.modp, 1)));
00444
00445     // Replace the leading term of all factors with lterm(A) mod  $p^n$ 
00446     for (int i=0; i<(int)a.size(); i++) {
00447         a[i].setmod(A.modp, A.modn);
00448         a[i] += (lead - a[i].integer_lcoeff()) * poly::simple_poly(BHEAD x, 0, a[i].degree(x));
00449     }
00450
00451     // Calculate the error, express it in terms of ai and add corrections.
00452     for (int k=2; k<=A.modn; k++) {
00453         term *= poly(BHEAD A.modp);

```

```

00454
00455     poly error(BHEAD -1);
00456     for (int i=0; i<(int)a.size(); i++) error *= a[i];
00457     error += A;
00458     if (error.is_zero()) break;
00459
00460     error /= term;
00461     error.setmod(A.modp,1);
00462
00463     for (int i=0; i<(int)a.size(); i++)
00464         a[i] += term * (error * s[i] % a[i]);
00465 }
00466
00467 // Fix leading coefficients by dividing out integer contents.
00468 for (int i=0; i<(int)a.size(); i++)
00469     a[i] /= polygcd::integer_content(poly(a[i],0,1));
00470
00471 #ifdef DEBUG
00472     cout << "*** [" << thetime() << "]" RES : lift_coefficients("<_A<","<_a<") = "<a<<endl;
00473 #endif
00474
00475     return a;
00476 }
00477
00478 /*
00479     #] lift_coefficients :
00480     #[ predetermine :
00481 */
00482
00497 void polyfact::predetermine (int dep, const vector<vector<int> > &state, vector<vector<vector<int> > >
&terms, vector<int> &term, int sumdeg) {
00498     // store the term
00499     if (dep == (int)state.size()) {
00500         terms[sumdeg].push_back(term);
00501         return;
00502     }
00503
00504     // recursively create new terms
00505     term.push_back(0);
00506
00507     for (int deg=0; sumdeg+deg<(int)state[dep].size(); deg++)
00508         if (state[dep][deg] > 0) {
00509             term.back() = deg;
00510             predetermine(dep+1, state, terms, term, sumdeg+deg);
00511         }
00512
00513     term.pop_back();
00514 }
00515
00516 /*
00517     #] predetermine :
00518     #[ lift_variables :
00519 */
00520
00558 const vector<poly> polyfact::lift_variables (const poly &A, const vector<poly> &_a, const vector<int>
&x, const vector<int> &c, const vector<poly> &lc) {
00559
00560 #ifdef DEBUG
00561     cout << "*** [" << thetime() << "]" CALL: lift_variables("<A<","<_a<","<x<","<c<","<lc<")\n";
00562 #endif
00563
00564     // If univariate, don't lift
00565     if (x.size()<=1) return _a;
00566
00567     POLY_GETIDENTITY(A);
00568
00569     vector<poly> a(_a);
00570
00571     // First method: predetermine coefficients
00572
00573     // check feasibility, otherwise it tries too many possibilities
00574     int cnt = POLYFACT_MAX_PREDETERMINATION;
00575     for (int i=0; i<(int)a.size(); i++) {
00576         if (a[i].number_of_terms() == 0) return vector<poly>();
00577         cnt /= a[i].number_of_terms();
00578     }
00579
00580     if (cnt>0) {
00581         // state[n][d]: coefficient of x^d in a[n] is
00582         // 0: non-existent, 1: undetermined, 2: determined
00583         int D = A.degree(x[0]);
00584         vector<vector<int> > state(a.size(), vector<int>(D+1, 0));
00585
00586         for (int i=0; i<(int)a.size(); i++)
00587             for (int j=1; j<a[i][0]; j+=a[i][j])
00588                 state[i][a[i][j+1+x[0]]] = j==1 ? 2 : 1;
00589

```



```

00590         // collect all products of terms
00591         vector<vector<vector<int>> > > terms(D+1);
00592         vector<int> term;
00593         predetermine(0, state, terms, term);
00594
00595         // count the number of undetermined coefficients
00596         vector<int> num_undet (terms.size(), 0);
00597         for (int i=0; i<(int)terms.size(); i++)
00598             for (int j=0; j<(int)terms[i].size(); j++)
00599                 for (int k=0; k<(int)terms[i][j].size(); k++)
00600                     if (state[k][terms[i][j][k]] == 1) num_undet[i]++;
00601
00602         // replace the current leading coefficients by the correct ones
00603         for (int i=0; i<(int)a.size(); i++)
00604             a[i] += (lc[i] - a[i].lcoeff_univar(x[0])) * poly::simple_poly(BHEAD
x[0], 0, a[i].degree(x[0]));
00605
00606         bool changed;
00607         do {
00608             changed = false;
00609
00610             for (int i=0; i<(int)terms.size(); i++) {
00611                 // is only one coefficient in a equation is undetermined, solve
00612                 // the equation to determine this coefficient
00613                 if (num_undet[i] == 1) {
00614                     // generate equation
00615                     poly lhs(BHEAD 0), rhs(A.coefficient(x[0], i), A.modp, A.modn);
00616                     int which_idx=-1, which_deg=-1;
00617                     for (int j=0; j<(int)terms[i].size(); j++) {
00618                         poly coeff(BHEAD 1, A.modp, A.modn);
00619                         bool undet=false;
00620                         for (int k=0; k<(int)terms[i][j].size(); k++) {
00621                             if (state[k][terms[i][j][k]] == 1) {
00622                                 undet = true;
00623                                 which_idx=k;
00624                                 which_deg=terms[i][j][k];
00625                             }
00626                             else
00627                                 coeff *= a[k].coefficient(x[0], terms[i][j][k]);
00628                         }
00629                         if (undet)
00630                             lhs = coeff;
00631                         else
00632                             rhs -= coeff;
00633                     }
00634                     // solve equation
00635                     if (A.modn > 1) rhs.setmod(0, 1);
00636                     if (lhs.is_zero() || !(rhs%lhs).is_zero()) return vector<poly>();
00637                     a[which_idx] += (rhs / lhs - a[which_idx].coefficient(x[0], which_deg)) *
poly::simple_poly(BHEAD x[0], 0, which_deg);
00638                     state[which_idx][which_deg] = 2;
00639
00640                     // update number of undetermined coefficients
00641                     for (int j=0; j<(int)terms.size(); j++)
00642                         for (int k=0; k<(int)terms[j].size(); k++)
00643                             if (terms[j][k][which_idx] == which_deg)
00644                                 num_undet[j]--;
00645
00646                     changed = true;
00647                 }
00648             }
00649         } while (changed);
00650
00651         // if this is the complete result, skip lifting
00652         poly check(BHEAD 1, A.modn>1?0:A.modp, 1);
00653         for (int i=0; i<(int)a.size(); i++)
00654             check *= a[i];
00655
00656         if (check == A) return a;
00657     }
00658
00659     // Second method: Hensel lifting
00660
00661     // Calculate A and lc's modulo Ii = <xi-c[i-1], ..., xm-c[m-1]> (for i=2, ..., m)
00662     vector<poly> simple(x.size(), poly(BHEAD 0));
00663     for (int i=(int)x.size()-2; i>=0; i--)
00664         simple[i] = poly::simple_poly(BHEAD x[i+1], c[i], 1);
00665
00666     // Calculate the maximum degree of A in x2, ..., xm
00667     int maxdegA=0;
00668     for (int i=1; i<(int)x.size(); i++)
00669         maxdegA = MaX(maxdegA, A.degree(x[i]));
00670
00671     // Iteratively add the variables x2, ..., xm
00672     for (int xi=1; xi<(int)x.size(); xi++) {
00673         // replace the current leading coefficients by the correct ones

```

```

00675         for (int i=0; i<(int)a.size(); i++)
00676             a[i] += (lc[i] - a[i].lcoeff_univar(x[0])) * poly::simple_poly(BHEAD
x[0],0,a[i].degree(x[0]));
00677
00678         vector<poly> anew(a);
00679         for (int i=0; i<(int)anew.size(); i++)
00680             for (int j=xi-1; j<(int)c.size(); j++)
00681                 anew[i] %= simple[j];
00682
00683         vector<int> xnew(x.begin(), x.begin()+xi);
00684         vector<int> cnew(c.begin(), c.begin()+xi-1);
00685         poly term(BHEAD 1,A.modp,A.modn);
00686
00687         // Iteratively add the powers xi^k
00688         for (int deg=1, maxdeg=A.degree(x[xi]); deg<=maxdeg; deg++) {
00689
00690             term *= simple[xi-1];
00691
00692             // Calculate the error, express it in terms of ai and add corrections.
00693             poly error(BHEAD -1,A.modp,A.modn);
00694             for (int i=0; i<(int)a.size(); i++) error += a[i];
00695             error += A;
00696             for (int i=xi; i<(int)c.size(); i++) error %= simple[i];
00697
00698             if (error.is_zero()) break;
00699
00700             error /= term;
00701             error %= simple[xi-1];
00702
00703             vector<poly> s(solve_Diophantine_multivariate(anew,error,xnew,cnew,maxdegA));
00704             if (s == vector<poly>()) return vector<poly>();
00705
00706             for (int i=0; i<(int)a.size(); i++)
00707                 a[i] += s[i] * term;
00708         }
00709
00710         // check whether PRODUCT(a[i]) = A mod <xi-c{i-1},...,xm-c{m-1}> over the integers or ZZ/p
00711         poly check(BHEAD -1, A.modn>1?0:A.modp, 1);
00712         for (int i=0; i<(int)a.size(); i++) check *= a[i];
00713         check += A;
00714         for (int i=xi; i<(int)c.size(); i++) check %= simple[i];
00715         if (!check.is_zero()) return vector<poly>();
00716     }
00717
00718 #ifdef DEBUG
00719     cout << "*** [" << thetime() << "]" RES : lift_variables("«A«", "«_a«", "«x«", "«c«", "«lc«") = " << a <<
endl;
00720 #endif
00721
00722     return a;
00723 }
00724
00725 /*
00726 #] lift_variables :
00727 #[ choose_prime :
00728 */
00729
00741 WORD polyfact::choose_prime (const poly &a, const vector<int> &x, WORD p) {
00742
00743 #ifdef DEBUG
00744     cout << "*** [" << thetime() << "]" CALL: choose_prime("«a«", "«x«", "«p«") << endl;
00745 #endif
00746
00747     POLY_GETIDENTITY(a);
00748
00749     poly icont_lcoeff(polygcd::integer_content(a.lcoeff_univar(x[0])));
00750
00751     if (p==0) p = POLYFACT_FIRST_PRIME;
00752
00753     poly icont_lcoeff_modp(BHEAD 0);
00754
00755     do {
00756
00757         bool is_prime;
00758
00759         do {
00760             p += 2;
00761             is_prime = true;
00762             for (int d=2; d*d<=p; d++)
00763                 if (p%d==0) { is_prime=false; break; }
00764         }
00765         while (!is_prime);
00766
00767         icont_lcoeff_modp = icont_lcoeff;
00768         icont_lcoeff_modp.setmod(p,1);
00769     }
00770     while (icont_lcoeff_modp.is_zero());

```

```

00771
00772 #ifdef DEBUG
00773     cout << "*** [" << thetime() << "]" RES : choose_prime("«a«", "«x«", ?) = "«p«endl;
00774 #endif
00775
00776     return p;
00777 }
00778
00779 /*
00780 #] choose_prime :
00781 #[ choose_prime_power :
00782 */
00783
00798 WORD polyfact::choose_prime_power (const poly &a, WORD p) {
00799
00800 #ifdef DEBUG
00801     cout << "*** [" << thetime() << "]" CALL: choose_prime_power("«a«", "«p«") "«endl;
00802 #endif
00803
00804     POLY_GETIDENTITY(a);
00805
00806     // analyse the polynomial for calculating the bound
00807     double maxdegree=0, maxlogcoeff=0, numterms=0;
00808
00809     for (int i=1; i<a[0]; i+=a[i]) {
00810         for (int j=0; j<AN.poly_num_vars; j++)
00811             maxdegree = MaX(maxdegree, a[i+1+j]);
00812
00813         maxlogcoeff = MaX(maxlogcoeff,
00814             log(1.0+(UWORD)a[i+a[i]-2]) + // most
00815             significant digit + 1
00816             BITSINWORD*log(2.0)*(ABS(a[i+a[i]-1])-1)); // number of
00817             digits
00818             numterms++;
00819     }
00820
00821     WORD res = (WORD)ceil((log((sqrt(5.0)+1)/2)*maxdegree + maxlogcoeff + 0.5*log(numterms)) /
00822         log((double)p));
00823
00824 #ifdef DEBUG
00825     cout << "*** [" << thetime() << "]" CALL: choose_prime_power("«a«", "«p«") = "«res«endl;
00826 #endif
00827
00828     return res;
00829 }
00830
00831 /*
00832 #] choose_prime_power :
00833 #[ choose_ideal :
00834 */
00835
00860 const vector<int> polyfact::choose_ideal (const poly &a, int p, const factorized_poly &lc, const
00861     vector<int> &x) {
00862
00863 #ifdef DEBUG
00864     cout << "*** [" << thetime() << "]" CALL: polyfact::choose_ideal("
00865         «a«", "«p«", "«lc«", "«x«") "«endl;
00866 #endif
00867
00868     if (x.size()==1) return vector<int>();
00869
00870     POLY_GETIDENTITY(a);
00871
00872     vector<int> c(x.size()-1);
00873
00874     int dega = a.degree(x[0]);
00875     poly amodI(a);
00876
00877     // choose random c
00878     for (int i=0; i<(int)c.size(); i++) {
00879         c[i] = 1 + wranf(BHEAD0) % ((p-1) / POLYFACT_IDEAL_FRACTION);
00880         amodI %= poly::simple_poly(BHEAD x[i+1], c[i], 1);
00881     }
00882
00883     poly amodIp(amodI);
00884     amodIp.setmod(p, 1);
00885
00886     // check if leading coefficient is non-zero [equivalent to degree=old_degree]
00887     if (amodIp.degree(x[0]) != dega)
00888         return c = vector<int>();
00889
00890     // check if leading coefficient is squarefree [equivalent to gcd(a,a')==const]
00891     if (!polygcd::gcd_Euclidean(amodIp, amodIp.derivative(x[0])).is_integer())
00892         return c = vector<int>();
00893
00894     if (a.modp>0 && a.modn==1) return c;
00895

```

```

00895 // check for unique prime factors in each factor lc[i] of the leading coefficient
00896 vector<poly> d(1, polygcd::integer_content(amodI));
00897
00898 for (int i=0; i<(int)lc.factor.size(); i++) {
00899     // constant factor
00900     if (i==0 && lc.factor[i].is_integer()) {
00901         d[0] *= lc.factor[i];
00902         continue;
00903     }
00904
00905     // factor modulo I
00906     poly q(lc.factor[i]);
00907     for (int j=0; j<(int)c.size(); j++)
00908         q %= poly::simple_poly(BHEAD x[j+1],c[j]);
00909     if (q.sign() == -1) q *= poly(BHEAD -1);
00910
00911     // divide out common factors
00912     for (int j=(int)d.size()-1; j>=0; j--) {
00913         poly r(d[j]);
00914         while (!r.is_one()) {
00915             r = polygcd::integer_gcd(r,q);
00916             q /= r;
00917         }
00918     }
00919
00920     // check whether there is some factor left
00921     if (q.is_one()) return vector<int>();
00922     d.push_back(q);
00923 }
00924
00925 #ifdef DEBUG
00926     cout << "*** [" << thetime() << "] RES : polyfact::choose_ideal("
00927         <<a<<","<p<<","<lc<<","<x<<") = "<c<<endl;
00928 #endif
00929
00930     return c;
00931 }
00932
00933 /*
00934 #] choose_ideal :
00935 #[ squarefree_factors_Yun :
00936 */
00937
00938 const factorized_poly polyfact::squarefree_factors_Yun (const poly &a) {
00939
00940     factorized_poly res;
00941     poly a(_a);
00942
00943     int pow = 1;
00944     int x = a.first_variable();
00945
00946     poly b(a.derivative(x));
00947     poly c(polygcd::gcd(a,b));
00948
00949     while (true) {
00950         a /= c;
00951         b /= c;
00952         b -= a.derivative(x);
00953         if (b.is_zero()) break;
00954         c = polygcd::gcd(a,b);
00955         if (!c.is_one()) res.add_factor(c,pow);
00956         pow++;
00957     }
00958
00959     if (!a.is_one()) res.add_factor(a,pow);
00960     return res;
00961 }
00962
00963 /*
00964 #] squarefree_factors_Yun :
00965 #[ squarefree_factors_modp :
00966 */
00967
00968 const factorized_poly polyfact::squarefree_factors_modp (const poly &a) {
00969
00970     factorized_poly res;
00971     poly a(_a);
00972
00973     int pow = 1;
00974     int x = a.first_variable();
00975     poly b(a.derivative(x));
00976
00977     // poly contains terms of the form c(x)^n (n!=c*p)
00978     if (!b.is_zero()) {
00979         poly c(polygcd::gcd(a,b));
00980         a /= c;
00981     }
00982 }

```

```

00994         while (!a.is_one()) {
00995             b = polygcd::gcd(a,c);
00996             a /= b;
00997             if (!a.is_one()) res.add_factor(a,pow);
00998             pow++;
00999             a = b;
01000             c /= a;
01001         }
01002
01003     a = c;
01004 }
01005
01006 // polynomial contains terms of the form c(x)^p
01007 if (!a.is_one()) {
01008     for (int i=1; i<a[1]; i+=a[i])
01009         a[i+1+x] /= a.modp;
01010     factorized_poly res2(squarefree_factors(a));
01011     for (int i=0; i<(int)res2.factor.size(); i++) {
01012         res.factor.push_back(res2.factor[i]);
01013         res.power.push_back(a.modp*res2.power[i]);
01014     }
01015 }
01016
01017 return res;
01018 }
01019
01020 /*
01021 #] squarefree_factors_modp :
01022 #[ squarefree_factors :
01023 */
01024
01045 const factorized_poly polyfact::squarefree_factors (const poly &a) {
01046
01047 #ifdef DEBUG
01048     cout << "*** [" << thetime() << " ] CALL: squarefree_factors(" <<a<<")\n";
01049 #endif
01050
01051     if (a.is_one()) return factorized_poly();
01052
01053     factorized_poly res;
01054
01055     if (a.modp==0)
01056         res = squarefree_factors_Yun(a);
01057     else
01058         res = squarefree_factors_modp(a);
01059
01060 #ifdef DEBUG
01061     cout << "*** [" << thetime() << " ] RES : squarefree_factors(" <<a<<") = "<<res<<"\n";
01062 #endif
01063
01064     return res;
01065 }
01066
01067 /*
01068 #] squarefree_factors :
01069 #[ Berlekamp_Qmatrix :
01070 */
01071
01094 const vector<vector<WORD> > polyfact::Berlekamp_Qmatrix (const poly &a) {
01095
01096 #ifdef DEBUG
01097     cout << "*** [" << thetime() << " ] CALL: Berlekamp_Qmatrix(" <<a<<")\n";
01098 #endif
01099
01100     if (_a.all_variables() == vector<int>())
01101         return vector<vector<WORD> >(0);
01102
01103     POLY_GETIDENTITY(_a);
01104
01105     poly a(_a);
01106     int x = a.first_variable();
01107     int n = a.degree(x);
01108     int p = a.modp;
01109
01110     poly lc(a.integer_lcoeff());
01111     a /= lc;
01112
01113     vector<vector<WORD> > Q(n, vector<WORD>(n));
01114
01115     // c is the vector of coefficients of the polynomial a
01116     vector<WORD> c(n+1,0);
01117     for (int j=1; j<a[0]; j+=a[j])
01118         c[a[j+1+x]] = a[j+1+AN.poly_num_vars] * a[j+2+AN.poly_num_vars];
01119
01120     // d is the vector of coefficients of x^i mod a, starting with i=0
01121     vector<WORD> d(n,0);
01122     d[0]=1;

```

```

01123
01124     for (int i=0; i<=(n-1)*p; i++) {
01125         // store the coefficients of x^(i*p) mod a
01126         if (i%p==0) Q[i/p] = d;
01127
01128         // transform d=x^i mod a into d=x^(i+1) mod a
01129         vector<WORD> e(n);
01130         for (int j=0; j<n; j++) {
01131             e[j] = (- (LONG)d[n-1]*c[j] + (j>0?d[j-1]:0)) % p;
01132             if (e[j]<0) e[j]+=p;
01133         }
01134
01135         d=e;
01136     }
01137
01138     // Q = Q - I
01139     for (int i=0; i<n; i++)
01140         Q[i][i] = (Q[i][i] - 1 + p) % p;
01141
01142     // Gaussian elimination
01143     for (int i=0; i<n; i++) {
01144         // Find pivot
01145         int ii=i; while (ii<n && Q[ii][ii]==0) ii++;
01146         if (ii==n) continue;
01147
01148         for (int k=0; k<n; k++)
01149             swap(Q[k][ii],Q[k][i]);
01150
01151         // normalize row i, which becomes the pivot
01152         WORD mul;
01153         GetModInverses (Q[i][i], p, &mul, NULL);
01154         vector<int> idx;
01155
01156         for (int k=0; k<n; k++) if (Q[k][i] != 0) {
01157             // store indices of non-zero elements for sparse matrices
01158             idx.push_back(k);
01159             Q[k][i] = ((LONG)Q[k][i] * mul) % p;
01160         }
01161
01162         // reduce
01163         for (int j=0; j<n; j++)
01164             if (j!=i && Q[i][j]!=0) {
01165                 LONG mul = Q[i][j];
01166                 for (int k=0; k<(int)idx.size(); k++) {
01167                     Q[idx[k]][j] = (Q[idx[k]][j] - mul*Q[idx[k]][i]) % p;
01168                     if (Q[idx[k]][j] < 0) Q[idx[k]][j]+=p;
01169                 }
01170             }
01171     }
01172
01173     for (int i=0; i<n; i++) {
01174         // Q = Q - I
01175         Q[i][i] = Q[i][i]-1;
01176
01177         // reduce all coefficients in the range 0,1,...,p-1
01178         for (int j=0; j<n; j++) {
01179             Q[i][j] = -Q[i][j]%p;
01180             if (Q[i][j]<0) Q[i][j]+=p;
01181         }
01182     }
01183
01184 #ifndef DEBUG
01185     cout << "*** [" << thetime() << "]" RES : Berlekamp_Qmatrix("«_a«") = "«Q«"\n";
01186 #endif
01187
01188     return Q;
01189 }
01190
01191 /*
01192 #] Berlekamp_Qmatrix :
01193 #[ Berlekamp_find_factors :
01194 */
01195
01219 const vector<poly> polyfact::Berlekamp_find_factors (const poly &a, const vector<vector<WORD> > &q)
01220 {
01221 #ifndef DEBUG
01222     cout << "*** [" << thetime() << "]" CALL: Berlekamp_find_factors("«_a«","«_q«")\n";
01223 #endif
01224
01225     if (_a.all_variables() == vector<int>()) return vector<poly>(1,_a);
01226
01227     POLY_GETIDENTITY(_a);
01228
01229     vector<vector<WORD> > q=_q;
01230
01231     int rank=0;

```

```

01232     for (int i=0; i<(int)q.size(); i++)
01233         if (q[i]!=vector<WORD>(q[i].size(),0)) rank++;
01234
01235     poly a(_a);
01236     int x = a.first_variable();
01237     int n = a.degree(x);
01238     int p = a.modp;
01239
01240     a /= a.integer_lcoeff();
01241
01242     // Vector of factors, represented as dense polynomials mod p
01243     vector<vector<WORD> > fac(1, vector<WORD>(n+1,0));
01244
01245     fac[0] = poly::to_coefficient_list(a);
01246     bool finished=false;
01247
01248     // Loop over the columns of q + constant, i.e., an exhaustive list of possible factors
01249     for (int i=1; i<n && !finished; i++) {
01250         if (q[i] == vector<WORD>(n,0)) continue;
01251
01252         for (int s=0; s<p && !finished; s++) {
01253             for (int j=0; j<(int)fac.size() && !finished; j++) {
01254                 vector<WORD> c = polygcd::coefficient_list_gcd(fac[j], q[i], p);
01255
01256                 // If a non-trivial factor is found, add it to the list
01257                 if (c.size()!=1 && c.size()!=fac[j].size()) {
01258                     fac.push_back(c);
01259                     fac[j] = poly::coefficient_list_divmod(fac[j], c, p, 0);
01260                     if ((int)fac.size() == rank) finished=true;
01261                 }
01262             }
01263
01264             // Increase the constant term by one
01265             q[i][0] = (q[i][0]+1) % p;
01266         }
01267     }
01268
01269     // Convert the densely represented polynomials to sparse ones
01270     vector<poly> res(fac.size(),poly(BHEAD 0, p));
01271     for (int i=0; i<(int)fac.size(); i++)
01272         res[i] = poly::from_coefficient_list(BHEAD fac[i],x,p);
01273
01274 #ifdef DEBUG
01275     cout << "*** [" << thetime() << "] RES : Berlekamp_find_factors(" <<_a<<"," <<q<<) = " <<res<<"\n";
01276 #endif
01277
01278     return res;
01279 }
01280
01281 /*
01282 #] Berlekamp_find_factors :
01283 #[ combine_factors :
01284 */
01285
01307 const vector<poly> polyfact::combine_factors (const poly &a, const vector<poly> &f) {
01308
01309 #ifdef DEBUG
01310     cout << "*** [" << thetime() << "] CALL: combine_factors(" <<a<<"," <<f<<")\n";
01311 #endif
01312
01313     POLY_GETIDENTITY(a);
01314
01315     poly a0(a,0,1);
01316     vector<poly> res;
01317
01318     int num_used = 0;
01319     vector<bool> used(f.size(), false);
01320
01321     // Loop over all bitmasks with num=1,2,...,size(factors)/2 bits
01322     // set, that contain only unused factors
01323     for (int num=1; num<=(int)(f.size() - num_used)/2; num++) {
01324         vector<int> next(f.size() - num_used, 0);
01325         for (int i=0; i<num; i++) next[next.size()-1-i] = 1;
01326
01327         do {
01328             poly fac(BHEAD 1,a.modp,a.modn);
01329             for (int i=0, j=0; i<(int)f.size(); i++)
01330                 if (!used[i] && next[j++]) fac *= f[i];
01331             fac /= fac.integer_lcoeff();
01332             fac *= a.integer_lcoeff();
01333             fac /= polygcd::integer_content(poly(fac,0,1));
01334
01335             if ((a0 % fac).is_zero()) {
01336                 res.push_back(fac);
01337                 for (int i=0, j=0; i<(int)f.size(); i++)
01338                     if (!used[i]) used[i] = next[j++];
01339                 num_used += num;

```

```

01340         num--;
01341         break;
01342     }
01343 }
01344 while (next_permutation(next.begin(), next.end()));
01345 }
01346
01347 // All unused factors together form one more factor
01348 if (num_used != (int)f.size()) {
01349     poly fac(BHEAD 1, a.modp, a.modn);
01350     for (int i=0; i<(int)f.size(); i++)
01351         if (!used[i]) fac *= f[i];
01352     fac /= fac.integer_lcoeff();
01353     fac *= a.integer_lcoeff();
01354     fac /= polygcd::integer_content(poly(fac, 0, 1));
01355     res.push_back(fac);
01356 }
01357
01358 #ifdef DEBUG
01359     cout << "*** [" << thetime() << " ] RES : combine_factors(" << a << ", " << f << ") = " << res << "\n";
01360 #endif
01361
01362     return res;
01363 }
01364
01365 /*
01366 #] combine_factors :
01367 #[ factorize_squarefree :
01368 */
01369
01388 const vector<poly> polyfact::factorize_squarefree (const poly &a, const vector<int> &x) {
01389
01390     #ifdef DEBUG
01391         cout << "*** [" << thetime() << " ] CALL: factorize_squarefree(" << a << ") \n";
01392     #endif
01393
01394     POLY_GETIDENTITY(a);
01395
01396     WORD p=a.modp, n=a.modn;
01397
01398     try_again:
01399
01400     int bestp=0;
01401     int min_factors = INT_MAX;
01402     poly amodI(BHEAD 0), bestamodI(BHEAD 0);
01403     vector<int> c,d,bestc,bestd;
01404     vector<vector<WORD>> > q,bestq;
01405
01406     // Factorize leading coefficient
01407     factorized_poly lc(factorize(a.lcoeff_univar(x[0])));
01408
01409     // Try a number of primes
01410     int prime_tries = 0;
01411
01412     while (prime_tries<POLYFACT_NUM_CONFIRMATIONS && min_factors>1) {
01413         if (a.modp == 0) {
01414             p = choose_prime(a,x,p);
01415             n = 0;
01416             if (a.degree(x[0]) % p == 0) continue;
01417
01418             // Univariate case: check whether the polynomial mod p is squarefree
01419             // Multivariate case: this check is done after choosing I (for efficiency)
01420             if (x.size()==1) {
01421                 poly amodp(a,p,1);
01422                 if (polygcd::gcd_Euclidean(amodp, amodp.derivative(x[0])).degree(x[0]) != 0)
01423                     continue;
01424             }
01425         }
01426
01427         // Try a number of ideals
01428         if (x.size()>1)
01429             for (int ideal_tries=0; ideal_tries<POLYFACT_MAX_IDEAL_TRIES; ideal_tries++) {
01430                 c = choose_ideal(a,p,lc,x);
01431                 if (c.size()>0) break;
01432             }
01433
01434         if (x.size()==1 || c.size()>0) {
01435             amodI = a;
01436             for (int i=0; i<(int)c.size(); i++)
01437                 amodI %= poly::simple_poly(BHEAD x[i+1], c[i]);
01438
01439             // Determine Q-matrix and its rank. Smaller rank is better.
01440             q = Berlekamp_Qmatrix(poly(amodI,p,1));
01441             int rank=0;
01442             for (int i=0; i<(int)q.size(); i++)
01443                 if (q[i]!=vector<WORD>(q[i].size(),0)) rank++;
01444

```



```

01445         if (rank<min_factors) {
01446             bestp=p;
01447             bestc=c;
01448             bestq=q;
01449             bestamodI=amodI;
01450             min_factors = rank;
01451             prime_tries = 0;
01452         }
01453
01454         if (rank==min_factors)
01455             prime_tries++;
01456
01457 #ifdef DEBUG
01458     cout << "*** [" << thetime() << "] (A) : factorize_squarefree("a«
01459         " try p=" << p << " #factors=" << rank << " (min="«min_factors«"x"«prime_tries«")" << endl;
01460 #endif
01461     }
01462 }
01463
01464 p=bestp;
01465 c=bestc;
01466 q=bestq;
01467 amodI=bestamodI;
01468
01469 // Determine to which power of p to lift
01470 if (n==0) {
01471     n = choose_prime_power(amodI,p);
01472     n = MaX(n, choose_prime_power(a,p));
01473 }
01474
01475 amodI.setmod(p,n);
01476
01477 #ifdef DEBUG
01478     cout << "*** [" << thetime() << "] (B) : factorize_squarefree("a«
01479         " chosen c = " << c << " , p^n = "«p«"^"«n«endl;
01480     cout << "*** [" << thetime() << "] (C) : factorize_squarefree("a«
01481         " #factors = " << min_factors << endl;
01482 #endif
01483
01484 // Find factors
01485 vector<poly> f(Berlekamp_find_factors(poly(amodI,p,1),q));
01486
01487 // Lift coefficients
01488 if (f.size() > 1) {
01489     f = lift_coefficients(amodI,f);
01490     if (f==vector<poly>()) {
01491 #ifdef DEBUG
01492         cout << "factor_squarefree failed (lift_coeff step) : " << endl;
01493 #endif
01494         goto try_again;
01495     }
01496
01497 // Combine factors
01498 if (a.modp==0) f = combine_factors(amodI,f);
01499 }
01500
01501 // Lift variables
01502 if (f.size() == 1)
01503     f = vector<poly>(1, a);
01504
01505 if (x.size() > 1 && f.size() > 1) {
01506
01507     // The correct leading coefficients of the factors can be
01508     // reconstructed from prime number factors of the leading
01509     // coefficients modulo I. This is possible since all factors of
01510     // the leading coefficient have unique prime factors for the ideal
01511     // I is chosen as such.
01512
01513     poly amodI(a);
01514     for (int i=0; i<(int)c.size(); i++)
01515         amodI %= poly::simple_poly(BHEAD x[i+1],c[i]);
01516     poly delta(polygcd::integer_content(amodI));
01517
01518     vector<poly> lcmmodI(lc.factor.size(), poly(BHEAD 0));
01519     for (int i=0; i<(int)lc.factor.size(); i++) {
01520         lcmmodI[i] = lc.factor[i];
01521         for (int j=0; j<(int)c.size(); j++)
01522             lcmmodI[i] %= poly::simple_poly(BHEAD x[j+1],c[j]);
01523     }
01524
01525     vector<poly> correct_lc(f.size(), poly(BHEAD 1,p,n));
01526
01527     for (int j=0; j<(int)f.size(); j++) {
01528         poly lc_f(f[j].integer_lcoeff() * delta);
01529         WORD nlc_f = lc_f[lc_f[1]];
01530         poly quo(BHEAD 0,rem(BHEAD 0);
01531         WORD nquo,nrem;

```

```

01532
01533     for (int i=(int)lcmmodI.size()-1; i>=0; i--) {
01534
01535         if (i==0 && lc.factor[i].is_integer()) continue;
01536
01537         do {
01538             DivLong((UWORD *)&lc_f[2+AN.poly_num_vars], nlc_f,
01539                     (UWORD *)&lcmmodI[i][2+AN.poly_num_vars], lcmmodI[i][lcmmodI[i][1]],
01540                     (UWORD *)&quo[0], &nquo,
01541                     (UWORD *)&rem[0], &nrem);
01542
01543             if (nrem == 0) {
01544                 correct_lc[j] *= lc.factor[i];
01545                 lc_f.termscopy(&quo[0], 2+AN.poly_num_vars, ABS(nquo));
01546                 nlc_f = nquo;
01547             }
01548         } while (nrem == 0);
01549     }
01550 }
01551
01552
01553 for (int i=0; i<(int)correct_lc.size(); i++) {
01554     poly correct_modI(correct_lc[i]);
01555     for (int j=0; j<(int)c.size(); j++)
01556         correct_modI %= poly::simple_poly(BHEAD x[j+1],c[j]);
01557
01558     poly d(polygcd::integer_gcd(correct_modI, f[i].integer_lcoeff()));
01559     correct_lc[i] *= f[i].integer_lcoeff() / d;
01560     delta /= correct_modI / d;
01561     f[i] *= correct_modI / d;
01562 }
01563
01564 // increase n, because of multiplying with delta
01565 if (!delta.is_one()) {
01566     poly deltapow(BHEAD 1);
01567     for (int i=1; i<(int)correct_lc.size(); i++)
01568         deltapow *= delta;
01569     while (!deltapow.is_zero()) {
01570         deltapow /= poly(BHEAD p);
01571         n++;
01572     }
01573
01574     for (int i=0; i<(int)f.size(); i++) {
01575         f[i].modn = n;
01576         correct_lc[i].modn = n;
01577     }
01578 }
01579
01580 poly aa(a,p,n);
01581
01582 for (int i=0; i<(int)correct_lc.size(); i++) {
01583     correct_lc[i] *= delta;
01584     f[i] *= delta;
01585     if (i>0) aa *= delta;
01586 }
01587
01588 f = lift_variables(aa,f,x,c,correct_lc);
01589
01590 for (int i=0; i<(int)f.size(); i++)
01591     if (a.modp == 0)
01592         f[i] /= polygcd::integer_content(poly(f[i],0,1));
01593     else
01594         f[i] /= polygcd::content_univar(f[i], x[0]);
01595
01596 if (f==vector<poly>()) {
01597 #ifdef DEBUG
01598     cout << "factor_squarefree failed (lift_var step)" << endl;
01599 #endif
01600     goto try_again;
01601 }
01602
01603 // set modulus of the factors correctly
01604 if (a.modp==0)
01605     for (int i=0; i<(int)f.size(); i++)
01606         f[i].setmod(0,1);
01607
01608 // Final check (not sure if this is necessary, but it doesn't hurt)
01609 poly check(BHEAD 1,a.modp,a.modn);
01610 for (int i=0; i<(int)f.size(); i++)
01611     check *= f[i];
01612
01613 if (check != a) {
01614 #ifdef DEBUG
01615     cout << "factor_squarefree failed (final check) : " << f << endl;
01616 #endif
01617     goto try_again;
01618 }

```

```

01619     }
01620
01621 #ifdef DEBUG
01622     cout << "*** [" << thetime() << "]" RES : factorize_squarefree("«a«", "«x«", "«c«") = "«f«"\n";
01623 #endif
01624
01625     return f;
01626 }
01627
01628 /*
01629 #] factorize_squarefree :
01630 #[ factorize :
01631 */
01632
01641 const factorized_poly polyfact::factorize (const poly &a) {
01642
01643 #ifdef DEBUG
01644     cout << "*** [" << thetime() << "]" CALL: factorize("«a«")\n";
01645 #endif
01646     vector<int> x = a.all_variables();
01647
01648     // No variables, so just one factor
01649     if (x.size() == 0) {
01650         factorized_poly res;
01651         if (a.is_one()) return res;
01652         res.add_factor(a,1);
01653         return res;
01654     }
01655
01656     // Remove content
01657     poly conta(polygcd::content_univar(a,x[0]));
01658
01659     factorized_poly faca(factorize(conta));
01660
01661     poly ppa(a / conta);
01662
01663     // Find a squarefree factorization
01664     factorized_poly b(squarefree_factors(ppa));
01665
01666 #ifdef DEBUG
01667     cout << "*** [" << thetime() << "]" ... : factorize("«a«") : SFF = "«b«"\n";
01668 #endif
01669
01670     // Factorize each squarefree factor and build the "factorized_poly"
01671     for (int i=0; i<(int)b.factor.size(); i++) {
01672
01673         vector<poly> c(factorize_squarefree(b.factor[i], x));
01674
01675         for (int j=0; j<(int)c.size(); j++) {
01676             faca.factor.push_back(c[j]);
01677             faca.power.push_back(b.power[i]);
01678         }
01679     }
01680
01681 #ifdef DEBUG
01682     cout << "*** [" << thetime() << "]" RES : factorize("«a«") = "«faca«"\n";
01683 #endif
01684
01685     return faca;
01686 }
01687
01688 /*
01689 #] factorize :
01690 */

```

4.104 polyfact.h

```

00001 #pragma once
00002 /* #[ License : */
00003 /*
00004 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00005 * When using this file you are requested to refer to the publication
00006 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00007 * This is considered a matter of courtesy as the development was paid
00008 * for by FOM the Dutch physics granting agency and we would like to
00009 * be able to track its scientific use to convince FOM of its value
00010 * for the community.
00011 *
00012 * This file is part of FORM.
00013 *
00014 * FORM is free software: you can redistribute it and/or modify it under the
00015 * terms of the GNU General Public License as published by the Free Software
00016 * Foundation, either version 3 of the License, or (at your option) any later

```

```

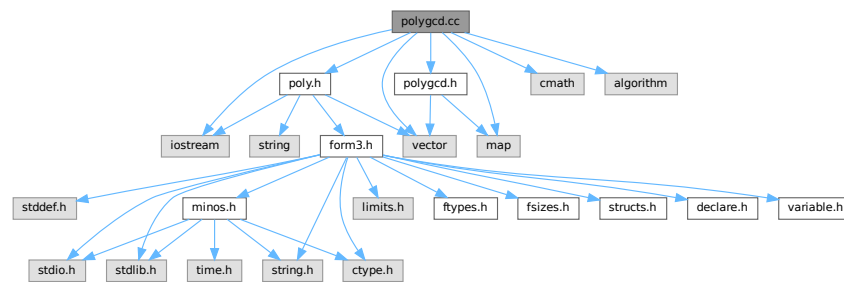
00017 *   version.
00018 *
00019 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00020 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00021 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00022 *   details.
00023 *
00024 *   You should have received a copy of the GNU General Public License along
00025 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00026 */
00027 /* #] License : */
00028
00029 #include <vector>
00030 #include <string>
00031
00032 // First prime modulo which factorization is tried. Too small results
00033 // in more unsuccessful attempts; too large is slower.
00034 const int POLYFACT_FIRST_PRIME = 17;
00035
00036 // Fraction of [1,p) that is used for substitutions of variables. Too
00037 // small results in more unsuccessful attempts; too large is slower.
00038 const int POLYFACT_IDEAL_FRACTION = 5;
00039
00040 // Number of ideals that are tried before failure due to unlucky
00041 // choices is accepted.
00042 const int POLYFACT_MAX_IDEAL_TRIES = 3;
00043
00044 // Number of confirmations for the minimal number of factors before
00045 // Hensel lifting is started.
00046 const int POLYFACT_NUM_CONFIRMATIONS = 3;
00047
00048 // Maximum number of equations for predetermination of coefficients
00049 // for multivariate Hensel lifting
00050 const int POLYFACT_MAX_PREDETERMINATION = 10000;
00051
00052 class poly;
00053
00054 class factorized_poly {
00055     /* Class for representing a factorized polynomial
00056      * The polynomial is given by: PRODUCT(factor[i] ^ power[i])
00057      */
00058 public:
00059     std::vector<poly> factor;
00060     std::vector<int> power;
00061
00062     void add_factor(const poly &f, int p=1);
00063     const std::string toString() const;
00064     friend std::ostream& operator<< (std::ostream &out, const poly &p);
00065 };
00066
00067 std::ostream& operator<< (std::ostream &out, const factorized_poly &a);
00068
00069 namespace polyfact {
00070
00071     // factorization routine
00072     const factorized_poly factorize (const poly &a);
00073
00074     // methods for squarefree factorization
00075     const factorized_poly squarefree_factors (const poly &a);
00076     const factorized_poly squarefree_factors_Yun (const poly &a);
00077     const factorized_poly squarefree_factors_modp (const poly &a);
00078
00079     // methods for choosing suitable reductions
00080     const std::vector<poly> factorize_squarefree (const poly &a, const std::vector<int> &x);
00081     WORD choose_prime (const poly &a, const std::vector<int> &x, WORD p=0);
00082     WORD choose_prime_power (const poly &a, WORD p);
00083     const std::vector<int> choose_ideal (const poly &a, int p, const factorized_poly &lc, const
std::vector<int> &x);
00084
00085     // methods for univariate factorization
00086     const std::vector<std::vector<WORD> > Berlekamp_Qmatrix (const poly &a);
00087     const std::vector<poly> Berlekamp_find_factors (const poly &a, const std::vector<std::vector<WORD>
> &Q);
00088     const std::vector<poly> combine_factors (const poly &a, const std::vector<poly> &f);
00089
00090     // methods for Hensel lifting
00091     const std::vector<poly> extended_gcd_Euclidean_lifted (const poly &a, const poly &b);
00092     const std::vector<poly> solve_Diophantine_univariate (const std::vector<poly> &a, const poly &b);
00093     const std::vector<poly> solve_Diophantine_multivariate (const std::vector<poly> &a, const poly &b,
const std::vector<int> &x, const std::vector<int> &c, int d);
00094     const std::vector<poly> lift_coefficients (const poly &a, const std::vector<poly> &f);
00095     const std::vector<poly> lift_variables (const poly &a, const std::vector<poly> &f, const
std::vector<int> &x, const std::vector<int> &c, const std::vector<poly> &lc);
00096     void predetermine (int dep, const std::vector<std::vector<int> > &state,
std::vector<std::vector<std::vector<int> > > &terms, std::vector<int> &term, int sumdeg=0);
00097
00098 }

```

4.105 polygcd.cc File Reference

```
#include "poly.h"
#include "polygcd.h"
#include <iostream>
#include <vector>
#include <cmath>
#include <map>
#include <algorithm>
```

Include dependency graph for polygcd.cc:



Data Structures

- struct [BracketInfo](#)

Functions

- bool [gcd_heuristic_possible](#) (const [poly](#) &a)
- const [poly](#) [gcd_linear_helper](#) (const [poly](#) &a, const [poly](#) &b)

4.105.1 Detailed Description

Contains the routines for calculating greatest common divisors of multivariate polynomials

Definition in file [polygcd.cc](#).

4.105.2 Function Documentation

4.105.2.1 gcd_heuristic_possible()

```
bool gcd_heuristic_possible (
    const poly & a )
```

Heuristic greatest common divisor of multivariate polynomials

4.105.3 Description

Checks whether the heuristic seems possible by estimating

$\text{MAX_terms} \{ \text{coeff} \wedge \text{PROD}_{\{i=1..\#\text{vars}\}} (\text{pow}_i+1) \}$

and comparing this with `GCD_HEURISTIC_MAX_DIGITS`.

4.105.4 Notes

- For small polynomials, this consumes time and never triggers.

Definition at line 1142 of file [polygcd.cc](#).

4.105.4.1 gcd_linear_helper()

```
const poly gcd_linear_helper (
    const poly & a,
    const poly & b )
```

Definition at line 1403 of file [polygcd.cc](#).

4.106 polygcd.cc

[Go to the documentation of this file.](#)

```
00001
00006 /* # License : */
00007 /*
00008 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00009 * When using this file you are requested to refer to the publication
00010 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00011 * This is considered a matter of courtesy as the development was paid
00012 * for by FOM the Dutch physics granting agency and we would like to
00013 * be able to track its scientific use to convince FOM of its value
00014 * for the community.
00015 *
00016 * This file is part of FORM.
00017 *
00018 * FORM is free software: you can redistribute it and/or modify it under the
00019 * terms of the GNU General Public License as published by the Free Software
00020 * Foundation, either version 3 of the License, or (at your option) any later
00021 * version.
00022 *
00023 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00024 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00025 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00026 * details.
00027 *
00028 * You should have received a copy of the GNU General Public License along
00029 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00030 */
00031 /* # License : */
00032 /*
00033 # include :
00034 */
00035
00036 #include "poly.h"
00037 #include "polygcd.h"
00038
00039 #include <iostream>
00040 #include <vector>
00041 #include <cmath>
00042 #include <map>
00043 #include <algorithm>
```

```

00044
00045 //define DEBUG
00046 //define DEBUGALL
00047
00048 #ifdef DEBUG
00049 #include "mytime.h"
00050 #endif
00051
00052 using namespace std;
00053
00054 /*
00055     #] include :
00056     #[ ostream operator :
00057 */
00058
00059 #ifdef DEBUG
00060 // ostream operator for outputting vector<T>s for debugging purposes
00061 template<class T> ostream& operator<< (ostream &out, const vector<T> &x) {
00062     out<<"{";
00063     for (int i=0; i<(int)x.size(); i++) {
00064         if (i>0) out<<" ";
00065         out<<x[i];
00066     }
00067     out<<" ";
00068     return out;
00069 }
00070 #endif
00071
00072 /*
00073     #] ostream operator :
00074     #[ integer_gcd :
00075 */
00076
00091 const poly polygcd::integer_gcd (const poly &a, const poly &b) {
00092
00093     #ifdef DEBUGALL
00094         cout << "*** [" << thetime() << " ] CALL: integer_gcd(" << a << ", " << b << ")" << endl;
00095     #endif
00096
00097     POLY_GETIDENTITY(a);
00098
00099     if (a.is_zero()) return b;
00100     if (b.is_zero()) return a;
00101
00102     poly c(BHEAD 0, a.modp, a.modn);
00103     WORD nc;
00104
00105     GcdLong(BHEAD
00106             (UWORD *)&a[AN.poly_num_vars+2],a[a[0]-1],
00107             (UWORD *)&b[AN.poly_num_vars+2],b[b[0]-1],
00108             (UWORD *)&c[AN.poly_num_vars+2],&nc);
00109
00110     WORD x = 2 + AN.poly_num_vars + ABS(nc);
00111     c[1] = x; // term length
00112     c[0] = x+1; // total length
00113     c[x] = nc; // coefficient length
00114
00115     for (int i=0; i<AN.poly_num_vars; i++)
00116         c[2+i] = 0; // powers
00117
00118     #ifdef DEBUGALL
00119         cout << "*** [" << thetime() << " ] RES : integer_gcd(" << a << ", " << b << ") = " << c << endl;
00120     #endif
00121
00122     return c;
00123 }
00124
00125 /*
00126     #] integer_gcd :
00127     #[ integer_content :
00128 */
00129
00143 const poly polygcd::integer_content (const poly &a) {
00144
00145     #ifdef DEBUGALL
00146         cout << "*** [" << thetime() << " ] CALL: integer_content(" << a << ")" << endl;
00147     #endif
00148
00149     POLY_GETIDENTITY(a);
00150
00151     if (a.modp>0) return a.integer_lcoeff();
00152
00153     poly c(BHEAD 0, 0, 1);
00154     WORD *d = (WORD *)NumberMalloc("polygcd::integer_content");
00155     WORD nc=0;
00156
00157     for (int i=0; i<AN.poly_num_vars; i++)

```

```

00158         c[2+i] = 0;
00159
00160     for (int i=1; i<a[0]; i+=a[i]) {
00161
00162         WCOPY(d,&c[2+AN.poly_num_vars],nc);
00163
00164         GcdLong(BHEAD (UWORD *)d, nc,
00165                 (UWORD *)&a[i+1+AN.poly_num_vars], a[i+a[i]-1],
00166                 (UWORD *)&c[2+AN.poly_num_vars], &nc);
00167
00168         WORD x = 2 + AN.poly_num_vars + ABS(nc);
00169         c[1] = x; // term length
00170         c[0] = x+1; // total length
00171         c[x] = nc; // coefficient length
00172     }
00173
00174     if (a.sign() != c.sign()) c *= poly(BHEAD -1);
00175
00176     NumberFree(d,"polygcd::integer_content");
00177
00178 #ifdef DEBUGALL
00179     cout << "*** [" << thetime() << "] RES : integer_content(" << a << ") = " << c << endl;
00180 #endif
00181
00182     return c;
00183 }
00184
00185 /*
00186 #] integer_content :
00187 #[ content_univar :
00188 */
00189
00205 const poly polygcd::content_univar (const poly &a, int x) {
00206
00207 #ifdef DEBUG
00208     cout << "*** [" << thetime() << "] CALL: content_univar(" << a << ", " << string(1,'a'+x) << ") << endl;
00209 #endif
00210
00211     POLY_GETIDENTITY(a);
00212
00213     poly res(BHEAD 0, a.modp, a.modn);
00214
00215     for (int i=1; i<a[0];) {
00216         poly b(BHEAD 0, a.modp, a.modn);
00217         int deg = a[i+1+x];
00218
00219         for (; i<a[0] && a[i+1+x]==deg; i+=a[i]) {
00220             b.check_memory(b[0]+a[i]);
00221             b.termscopy(&a[i],b[0],a[i]);
00222             b[b[0]+1+x] = 0;
00223             b[0] += a[i];
00224         }
00225
00226         res = gcd(res, b);
00227
00228         if (res.is_integer()) {
00229             res = integer_content(a);
00230             break;
00231         }
00232     }
00233
00234     if (a.sign() != res.sign()) res *= poly(BHEAD -1);
00235
00236 #ifdef DEBUG
00237     cout << "*** [" << thetime() << "] RES : content_univar(" << a << ", " << string(1,'a'+x) << ") = " << res
00238     << endl;
00239 #endif
00240     return res;
00241 }
00242
00243 /*
00244 #] content_univar :
00245 #[ content_multivar :
00246 */
00247
00263 const poly polygcd::content_multivar (const poly &a, int x) {
00264
00265 #ifdef DEBUGALL
00266     cout << "*** [" << thetime() << "] CALL: content_multivar(" << a << ", " << string(1,'a'+x) << ") <<
00267     endl;
00268 #endif
00269
00270     POLY_GETIDENTITY(a);
00271
00272     poly res(BHEAD 0, a.modp, a.modn);
00273

```



```

00273     for (int i=1,j; i<a[0]; i=j) {
00274         poly b(BHEAD 0, a.modp, a.modn);
00275
00276         for (j=i; j<a[0]; j+=a[j]) {
00277             bool same_powers = true;
00278             for (int k=0; k<AN.poly_num_vars; k++)
00279                 if (k!=x && a[i+1+k]!=a[j+1+k]) {
00280                     same_powers = false;
00281                     break;
00282                 }
00283             if (!same_powers) break;
00284
00285             b.check_memory(b[0]+a[j]);
00286             b.termscopy(&a[j],b[0],a[j]);
00287             for (int k=0; k<AN.poly_num_vars; k++)
00288                 if (k!=x) b[b[0]+1+k]=0;
00289
00290             b[0] += a[j];
00291         }
00292
00293         res = gcd_Euclidean(res, b);
00294
00295         if (res.is_integer()) {
00296             res = poly(BHEAD a.sign(),a.modp,a.modn);
00297             break;
00298         }
00299     }
00300
00301 #ifdef DEBUGALL
00302     cout << "*** [" << thetime() << "] RES : content_multivar(" << a << ", " << string(1,'a'+x) << ") = " <<
00303     res << endl;
00304 #endif
00305     return res;
00306 }
00307
00308 /*
00309     #] content_multivar :
00310         #[ coefficient_list_gcd :
00311 */
00312
00325 const vector<WORD> polygcd::coefficient_list_gcd (const vector<WORD> &a, const vector<WORD> &b, WORD
p) {
00326
00327 #ifdef DEBUGALL
00328     cout << "*** [" << thetime() << "] CALL: coefficient_list_gcd("<<_a<<","<<_b<<","<<p<<")<<endl;
00329 #endif
00330
00331     vector<WORD> a(_a), b(_b);
00332
00333     while (b.size() != 0) {
00334         a = poly::coefficient_list_divmod(a,b,p,1);
00335         swap(a,b);
00336     }
00337
00338     while (a.back()==0) a.pop_back();
00339
00340     WORD inv;
00341     GetModInverses(a.back() + (a.back()<0?p:0), p, &inv, NULL);
00342
00343     for (int i=0; i<(int)a.size(); i++)
00344         a[i] = (LONG)inv*a[i] % p;
00345
00346 #ifdef DEBUGALL
00347     cout << "*** [" << thetime() << "] RES : coefficient_list_gcd("<<_a<<","<<_b<<","<<p<<") = "<<a<<endl;
00348 #endif
00349
00350     return a;
00351 }
00352
00353 /*
00354     #] coefficient_list_gcd :
00355     #[ gcd_Euclidean :
00356 */
00357
00374 const poly polygcd::gcd_Euclidean (const poly &a, const poly &b) {
00375
00376 #ifdef DEBUG
00377     cout << "*** [" << thetime() << "] CALL: gcd_Euclidean("<<a<<","<<b<<")<<endl;
00378 #endif
00379
00380     POLY_GETIDENTITY(a);
00381
00382     if (a.is_zero()) return b;
00383     if (b.is_zero()) return a;
00384     if (a.is_integer() || b.is_integer()) return integer_gcd(a,b);
00385

```

```

00386     poly res(BHEAD 0);
00387
00388     if (a.is_dense_univariate()>=-1 && b.is_dense_univariate()>=-1) {
00389         vector<WORD> coeff = coefficient_list_gcd(poly::to_coefficient_list(a),
00390 poly::to_coefficient_list(b), a.modp);
00391         res = poly::from_coefficient_list(BHEAD coeff, a.first_variable(), a.modp);
00392     }
00393     else {
00394         res = a;
00395         poly rem(b);
00396         while (!rem.is_zero())
00397             swap(res%=rem, rem);
00398         res /= res.integer_lcoeff();
00399     }
00400
00401 #ifdef DEBUG
00402     cout << "*** [" << thetime() << "] RES : gcd_Euclidean(" << a << ", " << b << ") = " << res << endl;
00403 #endif
00404
00405     return res;
00406 }
00407
00408 /*
00409 #] gcd_Euclidean :
00410 #[ chinese_remainder :
00411 */
00412
00426 const poly polygcd::chinese_remainder (const poly &a1, const poly &m1, const poly &a2, const poly &m2)
00427 {
00428 #ifdef DEBUGALL
00429     cout << "*** [" << thetime() << "] CALL: chinese_remainder(" << a1 << ", " << m1 << ", " << a2 << ", " << m2 <<
00430 ") " << endl;
00431 #endif
00432
00433     POLY_GETIDENTITY(a1);
00434
00435     WORD nx,ny,nz;
00436     UWORD *x = (UWORD *)NumberMalloc("polygcd::chinese_remainder");
00437     UWORD *y = (UWORD *)NumberMalloc("polygcd::chinese_remainder");
00438     UWORD *z = (UWORD *)NumberMalloc("polygcd::chinese_remainder");
00439
00440     GetLongModInverses(BHEAD (UWORD *)&m1[2+AN.poly_num_vars], m1[m1[1]],
00441 (UWORD *)&m2[2+AN.poly_num_vars], m2[m2[1]],
00442 (UWORD *)x, &nx, NULL, NULL);
00443
00444     AddLong((UWORD *)&a2[2+AN.poly_num_vars], a2.is_zero() ? 0 : a2[a2[1]],
00445 (UWORD *)&a1[2+AN.poly_num_vars], a1.is_zero() ? 0 : -a1[a1[1]],
00446 y, &ny);
00447
00448     MulLong (x,nx,y,ny,z,&nz);
00449     MulLong (z,nz,(UWORD *)&m1[2+AN.poly_num_vars],m1[m1[1]],x,&nx);
00450
00451     AddLong (x,nx,(UWORD *)&a1[2+AN.poly_num_vars], a1.is_zero() ? 0 : a1[a1[1]],y,&ny);
00452
00453     MulLong ((UWORD *)&m1[2+AN.poly_num_vars], m1[m1[1]],
00454 (UWORD *)&m2[2+AN.poly_num_vars], m2[m2[1]],
00455 (UWORD *)z,&nz);
00456
00457     TakeNormalModulus (y,&ny,(UWORD *)z,nz,NOUNPACK);
00458
00459     poly res(BHEAD y,ny);
00460
00461     NumberFree(x,"polygcd::chinese_remainder");
00462     NumberFree(y,"polygcd::chinese_remainder");
00463     NumberFree(z,"polygcd::chinese_remainder");
00464
00465 #ifdef DEBUGALL
00466     cout << "*** [" << thetime() << "] RES : chinese_remainder(" << a1 << ", " << m1 << ", " << a2 << ", " << m2 <<
00467 ") = " << res << endl;
00468 #endif
00469
00470     return res;
00471 }
00472
00473 /*
00474 #] chinese_remainder :
00475 #[ substitute :
00476 */
00477
00488 const poly polygcd::substitute(const poly &a, int x, int c) {
00489
00490     POLY_GETIDENTITY(a);
00491
00492     poly b(BHEAD 0);
00493

```

```

00494     if (a.is_zero()) {
00495         return b;
00496     }
00497
00498     bool zero=true;
00499     int bi=1;
00500
00501     // cache size is bounded by the degree in x, twice the number of terms of
00502     // the polynomial and a constant
00503     vector<WORD> cache(min(a.degree(x)+1,min(2*a.number_of_terms(),
00504         POLYGCD_RAISPOWMOD_CACHE_SIZE)), 0);
00505
00506     for (int ai=1; ai<=a[0]; ai+=a[ai]) {
00507         // last term or different power, then add term to b iff non-zero
00508         if (!zero) {
00509             bool add=false;
00510             if (ai==a[0])
00511                 add=true;
00512             else {
00513                 for (int i=0; i<AN.poly_num_vars; i++)
00514                     if (i!=x && a[ai+1+i]!=b[bi+1+i]) {
00515                         zero=true;
00516                         add=true;
00517                         break;
00518                     }
00519             }
00520
00521             if (add) {
00522                 if (b[bi+AN.poly_num_vars+1] < 0)
00523                     b[bi+AN.poly_num_vars+1] += a.modp;
00524                 bi+=b[bi];
00525             }
00526
00527             if (ai==a[0]) break;
00528         }
00529
00530         b.check_memory(bi);
00531
00532         // create new term in b
00533         if (zero) {
00534             b[bi] = 3+AN.poly_num_vars;
00535             for (int i=0; i<AN.poly_num_vars; i++)
00536                 b[bi+1+i] = a[ai+1+i];
00537             b[bi+1+x] = 0;
00538             b[bi+AN.poly_num_vars+1] = 0;
00539             b[bi+AN.poly_num_vars+2] = 1;
00540         }
00541
00542         // add term of a to the current term in b
00543         LONG coeff = a[ai+1+AN.poly_num_vars] * a[ai+2+AN.poly_num_vars];
00544         int pow = a[ai+1+x];
00545
00546         if (pow<(int)cache.size()) {
00547             if (cache[pow]==0)
00548                 cache[pow] = RaisPowMod(c, pow, a.modp);
00549             coeff = (coeff * cache[pow]) % a.modp;
00550         }
00551         else {
00552             coeff = (coeff * RaisPowMod(c, pow, a.modp)) % a.modp;
00553         }
00554
00555         b[bi+AN.poly_num_vars+1] = (coeff + b[bi+AN.poly_num_vars+1]) % a.modp;
00556         if (b[bi+AN.poly_num_vars+1] != 0) zero=false;
00557     }
00558
00559     b[0]=bi;
00560     b.setmod(a.modp);
00561
00562     return b;
00563 }
00564
00565 /*
00566     #] substitute :
00567     #[ sparse_interpolation helper functions :
00568 */
00569
00570 // Returns a list of size #terms(a) with entries PROD(ci^powi, i=2..n)
00571 const vector<int> polygcd::sparse_interpolation_get_mul_list (const poly &a, const vector<int> &x,
00572     const vector<int> &c) {
00573     // cache size for variable x is bounded by the degree in x, twice
00574     // the number of terms of the polynomial and a constant
00575     vector<vector<WORD>> > cache(c.size());
00576     int max_cache_size = min(2*a.number_of_terms(),POLYGCD_RAISPOWMOD_CACHE_SIZE);
00577     for (int i=0; i<(int)c.size(); i++)
00578         cache[i] = vector<WORD>(min(a.degree(x[i+1])+1,max_cache_size), 0);

```

```

00579     vector<int> res;
00580     for (int i=1; i<a[0]; i+=a[i]) {
00581         LONG coeff=1;
00582         for (int j=0; j<(int)c.size(); j++) {
00583             int pow = a[i+1+x[j+1]];
00584             if (pow<(int)cache[j].size()) {
00585                 if (cache[j][pow]==0)
00586                     cache[j][pow] = RaisPowMod(c[j], pow, a.modp);
00587                 coeff = (coeff * cache[j][pow]) % a.modp;
00588             }
00589             else {
00590                 coeff = (coeff * RaisPowMod(c[j], pow, a.modp)) % a.modp;
00591             }
00592         }
00593         res.push_back(coeff);
00594     }
00595     return res;
00596 }
00597
00598 // Multiplies the coefficients of a with the entries of mul
00599 void polygcd::sparse_interpolation_mul_poly (poly &a, const vector<int> &mul) {
00600     for (int i=1, j=0; i<a[0]; i+=a[i], j++)
00601         a[i+a[i]-2] = ((LONG)a[i+a[i]-2]*mul[j]) % a.modp;
00602 }
00603
00604 // Sets all coefficients to the range 0..modp-1 and the powers of x2...xn to 0
00605 const poly polygcd::sparse_interpolation_reduce_poly (const poly &a, const vector<int> &x) {
00606     poly res(a);
00607     for (int i=1; i<a[0]; i+=a[i]) {
00608         for (int j=1; j<(int)x.size(); j++)
00609             res[i+1+x[j]]=0;
00610         if (res[i+a[i]-1]==-1) {
00611             res[i+a[i]-1]=1;
00612             res[i+a[i]-2]=a.modp-res[i+a[i]-2];
00613         }
00614     }
00615     return res;
00616 }
00617
00618 // Collects entries with equal powers, so that the result is a proper polynomial
00619 const poly polygcd::sparse_interpolation_fix_poly (const poly &a, int x) {
00620
00621     POLY_GETIDENTITY(a);
00622     poly res(BHEAD 0, a.modp, 1);
00623
00624     int j=1;
00625     bool newterm=true;
00626
00627     for (int i=1; i<a[0]; i+=a[i]) {
00628         if (newterm)
00629             res.termscopy(&a[i], j, a[i]);
00630         else
00631             res[j+res[j]-2] = ((LONG)res[j+res[j]-2] + a[i+a[i]-2]) % a.modp;
00632
00633         newterm = i+a[i] == a[0] || res[j+1+x] != a[i+a[i]+1+x];
00634         if (newterm && res[j+res[j]-2]!=0) j += res[j];
00635     }
00636
00637     res[0]=j;
00638     return res;
00639 }
00640
00641 /*
00642     #] sparse_interpolation helper functions :
00643     #[ gcd_modular_sparse_interpolation :
00644 */
00645
00677 const poly polygcd::gcd_modular_sparse_interpolation (const poly &origa, const poly &origb, const
vector<int> &x, const poly &s) {
00678
00679 #ifdef DEBUG
00680     cout << "*** [" << thetime() << " ] CALL: gcd_modular_sparse_interpolation("
00681         << origa << ", " << origb << ", " << x << ", " << " << s << ")" << endl;
00682 #endif
00683
00684     POLY_GETIDENTITY(origa);
00685
00686     // strip multivariate content
00687     poly conta(content_multivar(origa, x.back()));
00688     poly contb(content_multivar(origb, x.back()));
00689     poly gcdconts(gcd_Euclidean(conta, contb));
00690     const poly& a = conta.is_one() ? origa : origa/conta;
00691     const poly& b = contb.is_one() ? origb : origb/contb;
00692
00693     // for non-monic cases, we need to normalize with the gcd of the lcoeffs of a poly in x[0]
00694     // or else the shape fitting does not work.
00695     // FIXME: the current implementation still rejects some valid shapes.

```

```

00696     poly lcgcd(BHEAD 1, a.modp);
00697     if (!s.lcoeff_univar(x[0]).is_integer()) {
00698         lcgcd = gcd_modular_dense_interpolation(a.lcoeff_univar(x[0]), b.lcoeff_univar(x[0]), x,
poly(BHEAD 0));
00699     }
00700
00701     // reduce polynomials
00702     poly ared(sparse_interpolation_reduce_poly(a,x));
00703     poly bred(sparse_interpolation_reduce_poly(b,x));
00704     poly sred(sparse_interpolation_reduce_poly(s,x));
00705     poly lred(sparse_interpolation_reduce_poly(lcgcd,x));
00706
00707     // set all coefficients to 1
00708     for (int i=1; i<sred[0]; i+=sred[i]) {
00709         sred[i+sred[i]-2] = sred[i+sred[i]-1] = 1;
00710     }
00711
00712     // generate random numbers and check there this set doesn't result
00713     // in a singular matrix
00714     vector<int> c(x.size()-1);
00715     vector<int> smul;
00716
00717     bool duplicates;
00718     do {
00719         for (int i=0; i<(int)c.size(); i++)
00720             c[i] = 1 + wranf(BHEAD0) % (a.modp-1);
00721         smul = sparse_interpolation_get_mul_list(s,x,c);
00722
00723         duplicates = false;
00724
00725         int fr=0,to=0;
00726         for (int i=1; i<s[0];) {
00727             int pow = s[i+1+x[0]];
00728             while (i<s[0] && s[i+1+x[0]]==pow) i+=s[i], to++;
00729             for (int j=fr; j<to; j++)
00730                 for (int k=fr; k<j; k++)
00731                     if (smul[j] == smul[k])
00732                         duplicates = true;
00733             fr=to;
00734         }
00735     } while (duplicates);
00736
00737     // get the lists to multiply the polynomials with every iteration
00738     vector<int> amul(sparse_interpolation_get_mul_list(a,x,c));
00739     vector<int> bmul(sparse_interpolation_get_mul_list(b,x,c));
00740     vector<int> lmul(sparse_interpolation_get_mul_list(lcgcd,x,c));
00741
00742     vector<vector<vector<LONG> > > M;
00743     vector<vector<LONG> > V;
00744
00745     int maxMsize=0;
00746
00747     // create (empty) matrices
00748     for (int i=1; i<s[0]; i+=s[i]) {
00749         if (i==1 || s[i+1+x[0]]!=s[i+1+x[0]-s[i]]) {
00750             M.push_back(vector<vector<LONG> >());
00751             V.push_back(vector<LONG>());
00752         }
00753         M.back().push_back(vector<LONG>());
00754         V.back().push_back(0);
00755         maxMsize = max(maxMsize, (int)M.back().size());
00756     }
00757
00758     // generate linear equations
00759     for (int numg=0; numg<maxMsize; numg++) {
00760         poly amodI(sparse_interpolation_fix_poly(ared,x[0]));
00761         poly bmodI(sparse_interpolation_fix_poly(bred,x[0]));
00762         poly lmodI(sparse_interpolation_fix_poly(lred,x[0]));
00763
00764         // A fix for non-monic gcds. This could be slow if lmodI has many terms,
00765         // since it overfits the gcd now. Another gcd has to be run to remove the
00766         // extra terms.
00767         poly gcd(lmodI * gcd_Euclidean(amodI,bmodI));
00768
00769         // if correct gcd
00770         if (!gcd.is_zero() && gcd[2+x[0]]==sred[2+x[0]]) {
00771             // for each power in the gcd, generate an equation if needed
00772             int gi=1, midx=0;
00773
00774             for (int si=1; si<s[0];) {
00775                 // if the term exists, set Vi=coeff, otherwise Vi remains 0
00776                 if (gi<gcd[0] && gcd[gi+1+x[0]]==sred[si+1+x[0]]) {
00777                     if (numg < (int)V[midx].size())
00778                         V[midx][numg] = gcd[gi+gcd[gi]-1]*gcd[gi+gcd[gi]-2];
00779                 }
00780                 gi+=s[si];
00781             }

```

```

00782         gi += gcd[gi];
00783     }
00784
00785     // add the coefficients of s to the matrix M
00786     for (int i=0; i<(int)M[midx].size(); i++) {
00787         if (numg < (int)M[midx].size())
00788             M[midx][numg].push_back(sred[si+1+AN.poly_num_vars]);
00789         si += s[si];
00790     }
00791
00792     midx++;
00793 }
00794 }
00795 else {
00796     // incorrect gcd
00797     if (!gcd.is_zero() && gcd[2+x[0]]<sred[2+x[0]])
00798         return poly(BHEAD 0);
00799     numg--;
00800 }
00801
00802 // multiply polynomials by the lists to obtain new ones
00803 sparse_interpolation_mul_poly(ared, amul);
00804 sparse_interpolation_mul_poly(bred, bmul);
00805 sparse_interpolation_mul_poly(sred, smul);
00806 sparse_interpolation_mul_poly(lred, lmul);
00807 }
00808
00809 // solve the linear equations
00810 for (int i=0; i<(int)M.size(); i++) {
00811     int n = M[i].size();
00812
00813     // Gaussian elimination
00814     for (int j=0; j<n; j++) {
00815         for (int k=0; k<j; k++) {
00816             LONG x = M[i][j][k];
00817             for (int l=k; l<n; l++)
00818                 M[i][j][l] = (M[i][j][l] - M[i][k][l]*x) % a.modp;
00819             V[i][j] = (V[i][j] - V[i][k]*x) % a.modp;
00820         }
00821
00822         // normalize row
00823         WORD x = M[i][j][j]; // WORD for GetModInverses
00824         GetModInverses(x + (x<0?a.modp:0), a.modp, &x, NULL);
00825         for (int k=0; k<n; k++)
00826             M[i][j][k] = (M[i][j][k]*x) % a.modp;
00827         V[i][j] = (V[i][j]*x) % a.modp;
00828     }
00829
00830     // solve
00831     for (int j=n-1; j>=0; j--)
00832         for (int k=j+1; k<n; k++)
00833             V[i][j] = (V[i][j] - M[i][j][k]*V[i][k]) % a.modp;
00834 }
00835
00836 // create coefficient list
00837 vector<LONG> coeff;
00838 for (int i=0; i<(int)V.size(); i++)
00839     for (int j=0; j<(int)V[i].size(); j++)
00840         coeff.push_back(V[i][j]);
00841
00842 // create resulting polynomial
00843 poly res(BHEAD 0);
00844 int ri=1, i=0;
00845 for (int si=1; si<s[0]; si+=s[si]) {
00846     res.check_memory(ri);
00847     res[ri] = 3 + AN.poly_num_vars; // term length
00848     for (int j=0; j<AN.poly_num_vars; j++)
00849         res[ri+1+j] = s[si+1+j]; // powers
00850     res[ri+1+AN.poly_num_vars] = ABS(coeff[i]); // coefficient
00851     res[ri+2+AN.poly_num_vars] = SGN(coeff[i]); // coefficient length
00852     i++;
00853     ri += res[ri];
00854 }
00855 res[0]=ri; // total length
00856 res.setmod(a.modp,1);
00857
00858 if (!poly::divides(res, lcgcd * a) || !poly::divides(res, lcgcd * b)) {
00859     return poly(BHEAD 0); // bad shape
00860 } else {
00861     // refine gcd
00862     if (!poly::divides(res, a))
00863         res = gcd_modular_dense_interpolation(res, a, x, poly(BHEAD 0));
00864     if (!poly::divides(res, b))
00865         res = gcd_modular_dense_interpolation(res, b, x, poly(BHEAD 0));
00866 }
00867
00868 #ifdef DEBUG

```

```

00869     cout << "*** [" << thetime() << "]" RES : gcd_modular_sparse_interpolation("
00870         << a << ", " << b << ", " << x << ", " << ", " << s << ") = " << res << endl;
00871 #endif
00872
00873     return gcdconts * res;
00874 }
00875
00876 /*
00877     #] gcd_modular_sparse_interpolation :
00878     #[ gcd_modular_dense_interpolation :
00879 */
00880
00899 const poly polygcd::gcd_modular_dense_interpolation (const poly &a, const poly &b, const vector<int>
&x, const poly &s) {
00900
00901 #ifdef DEBUG
00902     cout << "*** [" << thetime() << "]" CALL: gcd_modular_dense_interpolation(" << a << ", " << b << ", " << x <<
", " << s << ") << endl;
00903 #endif
00904
00905     POLY_GETIDENTITY(a);
00906
00907     // if univariate, then use Euclidean algorithm
00908     if (x.size() == 1) {
00909         return gcd_Euclidean(a,b);
00910     }
00911
00912     // if shape is known, use sparse interpolation
00913     if (!s.is_zero()) {
00914         poly res = gcd_modular_sparse_interpolation (a,b,x,s);
00915         if (!res.is_zero()) return res;
00916         // apparently the shape was not correct. continue.
00917     }
00918
00919     // divide out multivariate content in last variable
00920     int X = x.back();
00921
00922     poly conta(content_multivar(a,X));
00923     poly contb(content_multivar(b,X));
00924     poly gcdconts(gcd_Euclidean(conta,contb));
00925     const poly& ppa = conta.is_one() ? a : poly(a/conta);
00926     const poly& ppb = contb.is_one() ? b : poly(b/contb);
00927
00928     // gcd of leading coefficients
00929     poly lcoeffa(ppa.lcoeff_multivar(X));
00930     poly lcoeffb(ppb.lcoeff_multivar(X));
00931     poly gcdlcoeffs(gcd_Euclidean(lcoeffa,lcoeffb));
00932
00933     // calculate the degree bound for each variable
00934     int m = Min(ppa.degree(x[x.size() - 2]),ppb.degree(x[x.size() - 2]));
00935
00936     poly res(BHEAD 0);
00937     poly oldres(BHEAD 0);
00938     poly newshape(BHEAD 0);
00939     poly modpoly(BHEAD 1,a.modp);
00940
00941     while (true) {
00942         // generate random constants and substitute it
00943         int c = 1 + wranf(BHEAD0) % (a.modp-1);
00944         if (substitute(gcdlcoeffs,X,c).is_zero()) continue;
00945         if (substitute(modpoly,X,c).is_zero()) continue;
00946
00947         poly amodc(substitute(ppa,X,c));
00948         poly bmodc(substitute(ppb,X,c));
00949
00950         // calculate gcd recursively
00951         poly gcdmodc(gcd_modular_dense_interpolation(amodc,bmodc,vector<int>(x.begin(),x.end()-1),
newshape));
00952         int n = gcdmodc.degree(x[x.size() - 2]);
00953
00954         // normalize
00955         gcdmodc = (gcdmodc * substitute(gcdlcoeffs,X,c)) / gcdmodc.integer_lcoeff();
00956         poly simple(poly::simple_poly(BHEAD X,c,1,a.modp)); // (X-c) mod p
00957
00958         // if power is smaller, the old one was wrong
00959         if ((res.is_zero() && n == m) || n < m) {
00960             m = n;
00961             res = gcdmodc;
00962             newshape = gcdmodc; // set a new shape (interpolation does not change it)
00963             modpoly = simple;
00964         }
00965         else if (n == m) {
00966             oldres = res;
00967             // equal powers, so interpolate results
00968             poly coeff_poly(substitute(modpoly,X,c));
00969             WORD coeff_word = coeff_poly[2+AN.poly_num_vars] * coeff_poly[3+AN.poly_num_vars];
00970             if (coeff_word < 0) coeff_word += a.modp;

```

```

00971
00972         GetModInverses(coeff_word, a.modp, &coeff_word, NULL);
00973
00974         res.setmod(a.modp); // make sure the mod is set before substituting
00975         res += poly(BHEAD coeff_word, a.modp, 1) * modpoly * (gcdmodc - substitute(res,X,c));
00976         modpoly *= simple;
00977     }
00978
00979     // check whether this is the complete gcd
00980     if (!res.is_zero() && res == oldres) {
00981         poly nres = res / content_multivar(res, X);
00982         if (poly::divides(nres,ppa) && poly::divides(nres,ppb)) {
00983 #ifdef DEBUG
00984             cout << "*** [" << thetime() << "] RES : gcd_modular_dense_interpolation(" << a << ", " << b
00985             << ", "
00986             << x << ", " << ", " << s << ") = " << gcdconts * nres << endl;
00987 #endif
00988             return gcdconts * nres;
00989         }
00990         // At this point, the gcd may be too large due to bad luck
00991         // TODO: create an efficient fail state that tries to find a smaller
00992         // polynomial without interpolating bad ones?
00993         newshape = poly(BHEAD 0); // reset the shape, important!
00994     }
00995 }
00996 }
00997
00998 /*
00999 #] gcd_modular_dense_interpolation : :
01000 #[ gcd_modular :
01001 */
01002
01019 const poly polygcd::gcd_modular (const poly &origa, const poly &origb, const vector<int> &x) {
01020
01021 #ifdef DEBUG
01022     cout << "*** [" << thetime() << "] CALL: gcd_modular(" << origa << ", " << origb << ", " << x << ")" << endl;
01023 #endif
01024
01025     POLY_GETIDENTITY(origa);
01026
01027     if (origa.is_zero()) return origa.modp==0 ? origb : origb / origb.integer_lcoeff();
01028     if (origb.is_zero()) return origa.modp==0 ? origa : origa / origa.integer_lcoeff();
01029     if (origa==origb) return origa.modp==0 ? origa : origa / origa.integer_lcoeff();
01030
01031     poly ac = integer_content(origa);
01032     poly bc = integer_content(origb);
01033     const poly& a = ac.is_one() ? origa : poly(origa/ac);
01034     const poly& b = bc.is_one() ? origb : poly(origb/bc);
01035     poly ic = integer_gcd(ac, bc);
01036     poly g = integer_gcd(a.integer_lcoeff(), b.integer_lcoeff());
01037
01038     int pnum=0;
01039
01040     poly d(BHEAD 0);
01041     poly m1(BHEAD 1);
01042     int mindeg=MAXPOSITIVE;
01043
01044     while (true) {
01045         // choose a prime and solve modulo the prime
01046         WORD p = NextPrime(BHEAD pnum++);
01047         if (poly(a.integer_lcoeff(),p).is_zero()) continue;
01048         if (poly(b.integer_lcoeff(),p).is_zero()) continue;
01049
01050         poly c(gcd_modular_dense_interpolation(poly(a,p),poly(b,p),x,poly(d,p)));
01051         c = (c * poly(g,p)) / c.integer_lcoeff(); // normalize so that lcoeff(c) = g mod p
01052
01053         if (c.is_zero()) {
01054             // unlucky choices somewhere, so start all over again
01055             d = poly(BHEAD 0);
01056             m1 = poly(BHEAD 1);
01057             mindeg = MAXPOSITIVE;
01058             continue;
01059         }
01060
01061         if (!(poly(a,p)%c).is_zero()) continue;
01062         if (!(poly(b,p)%c).is_zero()) continue;
01063
01064         int deg = c.degree(x[0]);
01065
01066         if (deg < mindeg) {
01067             // small degree, so the old one is wrong
01068             d=c;
01069             d.modp=a.modp;
01070             d.modn=a.modn;
01071             m1 = poly(BHEAD p);
01072             mindeg=deg;

```



```

01073     }
01074     else if (deg == mindeg) {
01075         // same degree, so use Chinese Remainder Algorithm
01076         poly newd(BHEAD 0);
01077
01078         for (int ci=1,di=1; ci<c[0]||di<d[0]; ) {
01079             int comp = ci==c[0] ? -1 : di==d[0] ? +1 : poly::monomial_compare(BHEAD
&c[ci],&d[di]);
01080             poly a1(BHEAD 0),a2(BHEAD 0);
01081
01082             newd.check_memory(newd[0]);
01083
01084             if (comp <= 0) {
01085                 newd.termscopy(&d[di],newd[0],1+AN.poly_num_vars);
01086                 a1 = poly(BHEAD (UWORD *)&d[di+1+AN.poly_num_vars],d[di+d[di]-1]);
01087                 di+=d[di];
01088             }
01089             if (comp >= 0) {
01090                 newd.termscopy(&c[ci],newd[0],1+AN.poly_num_vars);
01091                 a2 = poly(BHEAD (UWORD *)&c[ci+1+AN.poly_num_vars],c[ci+c[ci]-1]);
01092                 ci+=c[ci];
01093             }
01094
01095             poly e(chinese_remainder(a1,m1,a2,poly(BHEAD p)));
01096             newd.termscopy(&e[2+AN.poly_num_vars], newd[0]+1+AN.poly_num_vars, ABS(e[e[1]])+1);
01097             newd[newd[0]] = 2 + AN.poly_num_vars + ABS(e[e[1]]);
01098             newd[0] += newd[newd[0]];
01099         }
01100
01101         m1 *= poly(BHEAD p);
01102         d=newd;
01103     }
01104
01105     // divide out spurious integer content
01106     poly ppd(d / integer_content(d));
01107
01108     // check whether this is the complete gcd
01109     if (poly::divides(ppd,a) && poly::divides(ppd,b)) {
01110 #ifdef DEBUG
01111         cout << "*** [" << thetime() << "]" RES : gcd_modular(" << origa << "," << origb << "," << x << ")
= "
01112             << ic * ppd << endl;
01113 #endif
01114         return ic * ppd;
01115     }
01116 #ifdef DEBUG
01117     cout << "*** [" << thetime() << "]" Retrying modular_gcd with new prime" << endl;
01118 #endif
01119 }
01120 }
01121
01122 /*
01123 #] gcd_modular :
01124 #[ gcd_heuristic_possible :
01125 */
01126
01142 bool gcd_heuristic_possible (const poly &a) {
01143     POLY_GETIDENTITY(a);
01144
01145     double prod_deg = 1;
01146     for (int j=0; j<AN.poly_num_vars; j++)
01147         prod_deg *= a[2+j]+1;
01148
01149     double digits = ABS(a[1+a[1]-1]);
01150     double lead = a[1+1+AN.poly_num_vars];
01151
01152     return prod_deg*(digits-1+log(2*ABS(lead))/log(2.0)/(BITSINWORD/2)) <
POLYGCD_HEURISTIC_MAX_DIGITS;
01154 }
01155
01156 /*
01157 #] gcd_heuristic_possible :
01158 #[ gcd_heuristic :
01159 */
01160
01161
01188 const poly polygcd::gcd_heuristic (const poly &a, const poly &b, const vector<int> &x, int max_tries)
{
01189
01190 #ifdef DEBUG
01191     cout << "*** [" << thetime() << "]" CALL: gcd_heuristic("<a<","<b<","<x<")\n";
01192 #endif
01193
01194     if (a.is_integer()) return integer_gcd(a,integer_content(b));
01195     if (b.is_integer()) return integer_gcd(integer_content(a),b);
01196

```

```

01197     POLY_GETIDENTITY(a);
01198
01199     // Calculate xi = 2*min(max(coefficients a),max(coefficients b))+2
01200
01201     UWORD *pxi=NULL;
01202     WORD nxi=0;
01203
01204     for (int i=1; i<a[0]; i+=a[i]) {
01205         WORD na = ABS(a[i+a[i]-1]);
01206         if (BigLong((UWORD *)&a[i+a[i]-1-na], na, pxi, nxi) > 0) {
01207             pxi = (UWORD *)&a[i+a[i]-1-na];
01208             nxi = na;
01209         }
01210     }
01211
01212     for (int i=1; i<b[0]; i+=b[i]) {
01213         WORD nb = ABS(b[i+b[i]-1]);
01214         if (BigLong((UWORD *)&b[i+b[i]-1-nb], nb, pxi, nxi) > 0) {
01215             pxi = (UWORD *)&b[i+b[i]-1-nb];
01216             nxi = nb;
01217         }
01218     }
01219
01220     poly xi(BHEAD pxi,nxi);
01221
01222     // Addition of another random factor gives better performance
01223     xi = xi*poly(BHEAD 2) + poly(BHEAD 2 + wranf(BHEAD0)%POLYGCD_HEURISTIC_MAX_ADD_RANDOM);
01224
01225     // If degree*digits(xi) is too large, throw exception
01226     if (max(a.degree(x[0]),b.degree(x[0])) * xi[xi[1]] >= Min(AM.MaxTal,
POLYGCD_HEURISTIC_MAX_DIGITS)) {
01227 #ifdef DEBUG
01228     cout << "*** [" << thetime() << " ] RES : gcd_heuristic("«a«","«b«","«x«" = overflow\n";
01229 #endif
01230     throw(gcd_heuristic_failed());
01231 }
01232
01233     for (int times=0; times<max_tries; times++) {
01234
01235         // Recursively calculate the gcd
01236
01237         poly gamma(gcd_heuristic(a % poly::simple_poly(BHEAD x[0],xi),
01238             b % poly::simple_poly(BHEAD x[0],xi),
01239             vector<int>(x.begin()+1,x.end()),1));
01240
01241         // If a gcd is found, reconstruct the powers of x
01242         if (!gamma.is_zero()) {
01243             // res is construct is reverse order. idx/len are for reversing
01244             // it in the correct order
01245             poly res(BHEAD 0), c(BHEAD 0);
01246             vector<int> idx, len;
01247
01248             for (int power=0; !gamma.is_zero(); power++) {
01249
01250                 // calculate c = gamma % xi (c and gamma are polynomials, xi is integer)
01251                 c = gamma;
01252                 c.coefficients_modulo((UWORD *)&xi[2+AN.poly_num_vars], xi[xi[0]-1], false);
01253
01254                 // Add the terms c * x^power to res
01255                 res.check_memory(res[0]+c[0]);
01256                 res.termscopy(&c[1],res[0],c[0]-1);
01257                 for (int i=1; i<c[0]; i+=c[i])
01258                     res[res[0]-1+i+1+x[0]] = power;
01259
01260                 // Store idx/len for reversing
01261                 if (!c.is_zero()) {
01262                     idx.push_back(res[0]);
01263                     len.push_back(c[0]-1);
01264                 }
01265
01266                 res[0] += c[0]-1;
01267
01268                 // Divide gamma by xi
01269                 gamma = (gamma - c) / xi;
01270             }
01271
01272             // Reverse the resulting polynomial
01273             poly rev(BHEAD 0);
01274             rev.check_memory(res[0]);
01275
01276             rev[0] = 1;
01277             for (int i=idx.size()-1; i>=0; i--) {
01278                 rev.termscopy(&res[idx[i]], rev[0], len[i]);
01279                 rev[0] += len[i];
01280             }
01281             res = rev;
01282

```

```

01283         poly ppres = res / integer_content(res);
01284
01285         if ((a%ppres).is_zero() && (b%ppres).is_zero()) {
01286 #ifdef DEBUG
01287             cout << "*** [" << thetime() << "] RES : gcd_heuristic(" << a << ", " << b << ", " << x << ") = " << res << "\n";
01288 #endif
01289             return res;
01290         }
01291     }
01292
01293     // Next xi by multiplying with the golden ratio to avoid correlated errors
01294     xi = xi * poly(BHEAD 28657) / poly(BHEAD 17711) + poly(BHEAD wranf(BHEAD0) %
POLY_GCD_HEURISTIC_MAX_ADD_RANDOM);
01295 }
01296
01297 #ifdef DEBUG
01298     cout << "*** [" << thetime() << "] RES : gcd_heuristic(" << a << ", " << b << ", " << x << ") = failed\n";
01299 #endif
01300
01301     return poly(BHEAD 0);
01302 }
01303
01304 /*
01305 #] gcd_heuristic :
01306 #[ bracket :
01307 */
01308
01309 const map<vector<int>,poly> polygcd::full_bracket(const poly &a, const vector<int>& filter) {
01310     POLY_GETIDENTITY(a);
01311
01312     map<vector<int>,poly> bracket;
01313     for (int ai=1; ai<a[0]; ai+=a[ai]) {
01314         vector<int> varpattern(AN.poly_num_vars);
01315         for (int i=0; i<AN.poly_num_vars; i++)
01316             if (filter[i] == 1 && a[ai + i + 1] > 0)
01317                 varpattern[i] = a[ai + i + 1];
01318
01319         // create monomial
01320         poly mon(BHEAD 1);
01321         mon.setmod(a.modp);
01322         mon[0] = a[ai] + 1;
01323         for (int i=0; i<a[ai]; i++)
01324             if (i > 0 && i <= AN.poly_num_vars && varpattern[i - 1])
01325                 mon[1+i] = 0;
01326             else
01327                 mon[1+i] = a[ai+i];
01328
01329         map<vector<int>,poly>::iterator i = bracket.find(varpattern);
01330         if (i == bracket.end()) {
01331             bracket.insert(std::make_pair(varpattern, mon));
01332         } else {
01333             i->second += mon;
01334         }
01335     }
01336
01337     return bracket;
01338 }
01339
01340 const poly polygcd::bracket(const poly &a, const vector<int>& pattern, const vector<int>& filter) {
01341     POLY_GETIDENTITY(a);
01342
01343     poly bracket(BHEAD 0);
01344     for (int ai=1; ai<a[0]; ai+=a[ai]) {
01345         bool ispat = true;
01346         for (int i=0; i<AN.poly_num_vars; i++)
01347             if (filter[i] == 1 && pattern[i] != a[ai + i + 1]) {
01348                 ispat = false;
01349                 break;
01350             }
01351
01352         if (ispat) {
01353             poly mon(BHEAD 1);
01354             mon.setmod(a.modp);
01355             mon[0] = a[ai] + 1;
01356             for (int i=0; i<a[ai]; i++)
01357                 if (i > 0 && i <= AN.poly_num_vars && pattern[i - 1])
01358                     mon[1+i] = 0;
01359                 else
01360                     mon[1+i] = a[ai+i];
01361             bracket += mon;
01362         }
01363     }
01364
01365     return bracket;
01366 }
01367
01368 const map<vector<int>,int> polygcd::bracket_count(const poly &a, const vector<int>& filter) {

```

```

01369     POLY_GETIDENTITY(a);
01370
01371     map<vector<int>,int> bracket;
01372     for (int ai=1; ai<a[0]; ai+=a[ai]) {
01373         vector<int> varpattern(AN.poly_num_vars);
01374         for (int i=0; i<AN.poly_num_vars; i++)
01375             if (filter[i] == 1 && a[ai + i + 1] > 0)
01376                 varpattern[i] = a[ai + i + 1];
01377
01378         map<vector<int>,int>::iterator i = bracket.find(varpattern);
01379         if (i == bracket.end()) {
01380             bracket.insert(std::make_pair(varpattern, 0));
01381         } else {
01382             i->second++;
01383         }
01384     }
01385
01386     return bracket;
01387 }
01388
01389 struct BracketInfo {
01390     std::vector<int> pattern;
01391     int num_terms, dummy = 0;
01392     const poly* p;
01393
01394     BracketInfo(const std::vector<int>& pattern, int num_terms, const poly* p) : pattern(pattern),
01395         num_terms(num_terms), p(p) {}
01396 };
01397
01398 /*
01399     #] bracket :
01400     #[ gcd_linear:
01401 */
01402
01403 const poly gcd_linear_helper (const poly &a, const poly &b) {
01404     POLY_GETIDENTITY(a);
01405
01406     for (int i = 0; i < AN.poly_num_vars; i++)
01407         if (a.degree(i) == 1) {
01408             vector<int> filter(AN.poly_num_vars);
01409             filter[i] = 1;
01410
01411             // bracket the linear variable
01412             map<vector<int>,poly> ba = polygcd::full_bracket(a, filter);
01413
01414             poly subgcd(BHEAD 1);
01415             if (ba.size() == 2)
01416                 subgcd = polygcd::gcd_linear(ba.begin()->second, (++ba.begin()->second);
01417             else
01418                 subgcd = ba.begin()->second;
01419
01420             poly linfac = a / subgcd;
01421             if (poly::divides(linfac,b))
01422                 return linfac * polygcd::gcd_linear(subgcd, b / linfac);
01423
01424             return polygcd::gcd_linear(subgcd, b);
01425         }
01426
01427     return poly(BHEAD 0);
01428 }
01429
01430 const poly polygcd::gcd_linear (const poly &a, const poly &b) {
01431     #ifdef DEBUG
01432         cout << "*** [" << thetime() << " ] CALL: gcd_linear(" <<a<<","<<b<<")\n";
01433     #endif
01434
01435     POLY_GETIDENTITY(a);
01436
01437     if (a.is_zero()) return a.modp==0 ? b : b / b.integer_lcoeff();
01438     if (b.is_zero()) return a.modp==0 ? a : a / a.integer_lcoeff();
01439     if (a==b) return a.modp==0 ? a : a / a.integer_lcoeff();
01440
01441     if (a.is_integer() || b.is_integer()) {
01442         if (a.modp > 0) return poly(BHEAD 1,a.modp,a.modn);
01443         return poly(integer_gcd(integer_content(a),integer_content(b)),0,1);
01444     }
01445
01446     poly h = gcd_linear_helper(a, b);
01447     if (!h.is_zero()) return h;
01448
01449     h = gcd_linear_helper(b, a);
01450     if (!h.is_zero()) return h;
01451
01452     vector<int> xa = a.all_variables();
01453     vector<int> xb = b.all_variables();

```

```

01458
01459     vector<int> used(AN.poly_num_vars,0);
01460     for (int i=0; i<(int)xa.size(); i++) used[xa[i]]++;
01461     for (int i=0; i<(int)xb.size(); i++) used[xb[i]]++;
01462     vector<int> x;
01463     for (int i=0; i<AN.poly_num_vars; i++)
01464         if (used[i]) x.push_back(i);
01465
01466     return gcd_modular(a,b,x);
01467 }
01468
01469 /*
01470 #] gcd_linear:
01471 #[ gcd :
01472 */
01473
01474 const poly polygcd::gcd (const poly &a, const poly &b) {
01475
01476 #ifdef DEBUG
01477     cout << "*** [" << thetime() << " ] CALL: gcd(" << a << ", " << b << ") \n";
01478 #endif
01479
01480     POLY_GETIDENTITY(a);
01481
01482     if (a.is_zero()) return a.modp==0 ? b : b / b.integer_lcoeff();
01483     if (b.is_zero()) return a.modp==0 ? a : a / a.integer_lcoeff();
01484     if (a==b) return a.modp==0 ? a : a / a.integer_lcoeff();
01485
01486     if (a.is_integer() || b.is_integer()) {
01487         if (a.modp > 0) return poly(BHEAD 1,a.modp,a.modn);
01488         return poly(integer_gcd(integer_content(a),integer_content(b)),0,1);
01489     }
01490
01491     // Generate a list of variables of a and b
01492     vector<int> xa = a.all_variables();
01493     vector<int> xb = b.all_variables();
01494
01495     vector<int> used(AN.poly_num_vars,0);
01496     for (int i=0; i<(int)xa.size(); i++) used[xa[i]]++;
01497     for (int i=0; i<(int)xb.size(); i++) used[xb[i]]++;
01498     vector<int> x;
01499     for (int i=0; i<AN.poly_num_vars; i++)
01500         if (used[i]) x.push_back(i);
01501
01502     if (a.is_monomial() || b.is_monomial()) {
01503
01504         poly res(BHEAD 1,a.modp,a.modn);
01505         if (a.modp == 0) res = integer_gcd(integer_content(a),integer_content(b));
01506
01507         for (int i=0; i<(int)x.size(); i++)
01508             res[2+x[i]] = 1<<(BITSINWORD-2);
01509
01510         for (int i=1; i<a[0]; i+=a[i])
01511             for (int j=0; j<(int)x.size(); j++)
01512                 res[2+x[j]] = MiN(res[2+x[j]], a[i+1+x[j]]);
01513
01514         for (int i=1; i<b[0]; i+=b[i])
01515             for (int j=0; j<(int)x.size(); j++)
01516                 res[2+x[j]] = MiN(res[2+x[j]], b[i+1+x[j]]);
01517
01518         return res;
01519     }
01520
01521     // Calculate the contents, their gcd and the primitive parts
01522     poly conta(x.size()==1 ? integer_content(a) : content_univar(a,x[0]));
01523     poly contb(x.size()==1 ? integer_content(b) : content_univar(b,x[0]));
01524     poly gcdconts(x.size()==1 ? integer_gcd(conta,contb) : gcd(conta,contb));
01525     const poly& ppa = conta.is_one() ? a : poly(a/conta);
01526     const poly& ppb = contb.is_one() ? b : poly(b/contb);
01527
01528     if (ppa == ppb)
01529         return ppa * gcdconts;
01530
01531     poly res(BHEAD 0);
01532
01533 #ifdef POLY_GCD_USE_HEURISTIC
01534     // Try the heuristic gcd algorithm
01535     if (a.modp==0 && gcd_heuristic_possible(a) && gcd_heuristic_possible(b)) {
01536         try {
01537             res = gcd_heuristic(ppa,ppb,x);
01538             if (!res.is_zero()) res /= integer_content(res);
01539         }
01540         catch (gcd_heuristic_failed) {}
01541     }
01542 #endif
01543
01544     // If gcd==0, the heuristic algorithm failed, so we do more extensive checks.

```

```

01565 // First, we filter out variables that appear in only one of the expressions.
01566 if (res.is_zero()) {
01567     bool unusedVars = false;
01568     for (unsigned int i = 0; i < used.size(); i++) {
01569         if (used[i] == 1) {
01570             unusedVars = true;
01571             break;
01572         }
01573     }
01574
01575     // if there are no unused variables, go to the linear routine directly
01576     if (!unusedVars) {
01577         res = gcd_linear(ppa,ppb);
01578 #ifdef DEBUG
01579         cout << "New GCD attempt (unused vars): " << res << endl;
01580 #endif
01581     }
01582
01583     // if res is not the gcd, it is 0 or larger than the gcd.
01584     // we bracket the expression in all the variables that appear only in one expr.
01585     // and we refine the gcd.
01586     bool diva = !res.is_zero() && poly::divides(res,ppa);
01587     bool divb = !res.is_zero() && poly::divides(res,ppb);
01588     if (!diva || !divb) {
01589         vector<BracketInfo> bracketinfo;
01590
01591         if (!diva) {
01592             map<vector<int>,int> ba = bracket_count(ppa, used);
01593             for (map<vector<int>,int>::iterator it = ba.begin(); it != ba.end(); it++)
01594                 bracketinfo.push_back(BracketInfo(it->first, it->second, &ppa));
01595         }
01596
01597         if (!divb) {
01598             map<vector<int>,int> bb = bracket_count(ppb, used);
01599             for (map<vector<int>,int>::iterator it = bb.begin(); it != bb.end(); it++)
01600                 bracketinfo.push_back(BracketInfo(it->first, it->second, &ppb));
01601         }
01602
01603         // sort so that the smallest bracket will be last
01604         sort(bracketinfo.begin(), bracketinfo.end());
01605
01606         if (res.is_zero()) {
01607             res = bracket(*bracketinfo.back().p, bracketinfo.back().pattern, used);
01608             bracketinfo.pop_back();
01609         }
01610
01611         while (bracketinfo.size() > 0) {
01612             poly subpoly(bracket(*bracketinfo.back().p, bracketinfo.back().pattern, used));
01613             if (!poly::divides(res,subpoly)) {
01614                 // if we can filter out more variables, call gcd again
01615                 if (res.all_variables() != subpoly.all_variables())
01616                     res = gcd(subpoly,res);
01617             } else
01618                 res = gcd_linear(subpoly,res);
01619             bracketinfo.pop_back();
01620         }
01621
01622         if (res.is_zero() || !poly::divides(res,ppa) || !poly::divides(res,ppb)) {
01623             MesPrint("Bad gcd found.");
01624             std::cout << "Bad gcd:" << res << " for " << ppa << " " << ppb << std::endl;
01625             Terminate(1);
01626         }
01627     }
01628
01629     res *= gcdconts * poly(BHEAD res.sign());
01630
01631 #ifdef DEBUG
01632     cout << "*** [" << thetime() << "] RES : gcd(" << a << ", " << b << ") = " << res << endl;
01633 #endif
01634
01635     return res;
01636 }
01637
01638 /*
01639 #] gcd :
01640 */

```

4.107 polygcd.h

```
00001 #pragma once
```

```

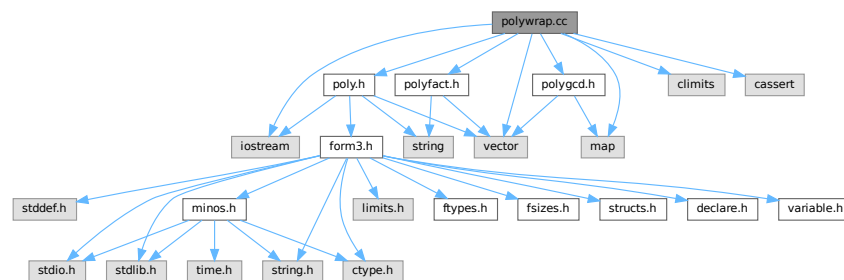
00002 /* #[ License : */
00003 /*
00004 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00005 *   When using this file you are requested to refer to the publication
00006 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00007 *   This is considered a matter of courtesy as the development was paid
00008 *   for by FOM the Dutch physics granting agency and we would like to
00009 *   be able to track its scientific use to convince FOM of its value
00010 *   for the community.
00011 *
00012 *   This file is part of FORM.
00013 *
00014 *   FORM is free software: you can redistribute it and/or modify it under the
00015 *   terms of the GNU General Public License as published by the Free Software
00016 *   Foundation, either version 3 of the License, or (at your option) any later
00017 *   version.
00018 *
00019 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00020 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00021 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00022 *   details.
00023 *
00024 *   You should have received a copy of the GNU General Public License along
00025 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00026 */
00027 /* #[ License : */
00028
00029 #include <vector>
00030 #include <map>
00031
00032 class poly; // polynomial class
00033 class gcd_heuristic_failed {}; // class for throwing exceptions
00034
00035 // whether or not to use the heuristic before Zippel's algorithm
00036 #define POLYGCD_USE_HEURISTIC
00037
00038 // maximum number of words in a coefficient for gcd_heuristic to continue
00039 const int POLYGCD_HEURISTIC_MAX_DIGITS = 1000;
00040
00041 // maximum number of retries after the heuristic has failed
00042 const int POLYGCD_HEURISTIC_MAX_TRIES = 10;
00043
00044 // a fudge factor, which improves efficiency
00045 const int POLYGCD_HEURISTIC_MAX_ADD_RANDOM = 10;
00046
00047 // maximum cached power in substitute_last and sparse_interpolation_get_mul_list
00048 const int POLYGCD_RAISPOWMOD_CACHE_SIZE = 1000;
00049
00050 namespace polygcd {
00051
00052     // functions to call the gcd routines
00053     const poly integer_gcd (const poly &a, const poly &b);
00054     const poly integer_content (const poly &a);
00055
00056     const poly gcd (const poly &a, const poly &b);
00057     const poly content_univar (const poly &a, int x);
00058     const poly content_multivar (const poly &a, int x);
00059
00060     const std::vector<WORD> coefficient_list_gcd (const std::vector<WORD> &a, const std::vector<WORD>
&b, WORD p);
00061
00062     // internal functions
00063     const poly gcd_heuristic (const poly &a, const poly &b, const std::vector<int> &x, int
max_tries=POLYGCD_HEURISTIC_MAX_TRIES);
00064     const poly gcd_Euclidean (const poly &a, const poly &b);
00065     const poly gcd_modular (const poly &a, const poly &b, const std::vector<int> &x);
00066     const poly gcd_modular_dense_interpolation (const poly &a, const poly &b, const std::vector<int>
&x, const poly &s);
00067     const poly gcd_modular_sparse_interpolation (const poly &a, const poly &b, const std::vector<int>
&x, const poly &s);
00068
00069     const std::vector<int> sparse_interpolation_get_mul_list (const poly &a, const std::vector<int>
&x, const std::vector<int> &c);
00070     void sparse_interpolation_mul_poly (poly &a, const std::vector<int> &m);
00071     const poly sparse_interpolation_reduce_poly (const poly &a, const std::vector<int> &x);
00072     const poly sparse_interpolation_fix_poly (const poly &a, int x);
00073
00074     const poly chinese_remainder (const poly &a1, const poly &m1, const poly &a2, const poly &m2);
00075     const poly substitute(const poly &a, int x, int c);
00076     const std::map<std::vector<int>,poly> full_bracket(const poly &a, const std::vector<int>& filter);
00077     const poly bracket(const poly &a, const std::vector<int>& pattern, const std::vector<int>&
filter);
00078     const std::map<std::vector<int>,int> bracket_count(const poly &a, const std::vector<int>& filter);
00079     const poly gcd_linear (const poly &a, const poly &b);
00080 }

```

4.108 polywrap.cc File Reference

```
#include "poly.h"
#include "polygcd.h"
#include "polyfact.h"
#include <iostream>
#include <vector>
#include <map>
#include <climits>
#include <cassert>
```

Include dependency graph for polywrap.cc:



Functions

- WORD [poly_determine_modulus](#) (PHEAD bool multi_error, bool is_fun_arg, string message)
- WORD * [poly_gcd](#) (PHEAD WORD *a, WORD *b, WORD fit)
- WORD * [poly_divmod](#) (PHEAD WORD *a, WORD *b, int divmod, WORD fit)
- WORD * [poly_div](#) (PHEAD WORD *a, WORD *b, WORD fit)
- WORD * [poly_rem](#) (PHEAD WORD *a, WORD *b, WORD fit)
- void [poly_ratfun_read](#) (WORD *a, [poly](#) &num, [poly](#) &den)
- void [poly_sort](#) (PHEAD WORD *a)
- WORD * [poly_ratfun_add](#) (PHEAD WORD *t1, WORD *t2)
- int [poly_ratfun_normalize](#) (PHEAD WORD *term)
- void [poly_fix_minus_signs](#) ([factorized_poly](#) &a)
- WORD * [poly_factorize](#) (PHEAD WORD *argin, WORD *argout, bool with_arghead, bool is_fun_arg)
- int [poly_factorize_argument](#) (PHEAD WORD *argin, WORD *argout)
- WORD * [poly_factorize_dollar](#) (PHEAD WORD *argin)
- int [poly_factorize_expression](#) (EXPRESSIONS expr)
- int [poly_unfactorize_expression](#) (EXPRESSIONS expr)
- WORD * [poly_inverse](#) (PHEAD WORD *arga, WORD *argb)
- WORD * [poly_mul](#) (PHEAD WORD *a, WORD *b)
- void [poly_free_poly_vars](#) (PHEAD const char *text)

Variables

- const int [POLYWRAP_DENOMPOWER_INCREASE_FACTOR](#) = 2

4.108.1 Detailed Description

Contains methods to call the polynomial methods (written in C++) from the rest of Form (written in C). These include polynomial gcd computation, factorization and polyratfuns.

Definition in file [polywrap.cc](#).

4.108.2 Function Documentation

4.108.2.1 poly_determine_modulus()

```
WORD poly_determine_modulus (
    PHEAD bool multi_error,
    bool is_fun_arg,
    string message )
```

Modulus for polynomial algebra

4.108.3 Description

This method determines whether polynomial algebra is done with a modulus or not. This depends on AC.ncmod. If only_funargs is set it also depends on (AC.modmode & ALSOFUNARGS).

The program terminates if the feature is not implemented. Polynomial algebra modulo $M > \text{WORDSIZE}$ is not implemented. If multi_error is set, multivariate algebra mod M is not implemented.

4.108.4 Notes

- If AC.ncmod>0 and only_funargs=true and AC.modmode&ALSOFUNARGS=false, AN.ncmod is set to zero, for otherwise RaisPow calculates mod M .

Definition at line 79 of file [polywrap.cc](#).

Referenced by [poly_factorize\(\)](#), [poly_factorize_expression\(\)](#), [poly_gcd\(\)](#), [poly_ratfun_add\(\)](#), and [poly_ratfun_normalize\(\)](#).

4.108.4.1 poly_gcd()

```
WORD * poly_gcd (
    PHEAD WORD * a,
    WORD * b,
    WORD fit )
```

Polynomial gcd

4.108.5 Description

This method calculates the greatest common divisor of two polynomials, given by two zero-terminated Form-style term lists.

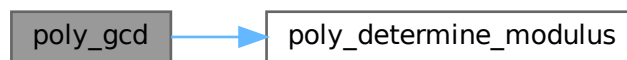
4.108.6 Notes

- The result is written at newly allocated memory
- Called from [ratio.c](#)
- Calls `polygcd::gcd`

Definition at line [124](#) of file [polywrap.cc](#).

References [poly_determine_modulus\(\)](#).

Here is the call graph for this function:



4.108.6.1 `poly_divmod()`

```
WORD * poly_divmod (  
    PHEAD WORD * a,  
    WORD * b,  
    int divmod,  
    WORD fit )
```

Definition at line [203](#) of file [polywrap.cc](#).

4.108.6.2 `poly_div()`

```
WORD * poly_div (  
    PHEAD WORD * a,  
    WORD * b,  
    WORD fit )
```

Definition at line [435](#) of file [polywrap.cc](#).

4.108.6.3 `poly_rem()`

```
WORD * poly_rem (  
    PHEAD WORD * a,  
    WORD * b,  
    WORD fit )
```

Definition at line [463](#) of file [polywrap.cc](#).

4.108.6.4 poly_ratfun_read()

```
void poly_ratfun_read (
    WORD * a,
    poly & num,
    poly & den )
```

Read a PolyRatFun

4.108.7 Description

This method reads a polyratfun starting at the pointer a. The resulting numerator and denominator are written in num and den. If MUSTCLEANPRF, the result is normalized.

4.108.8 Notes

- Calls polygcd::gcd

Definition at line 496 of file [polywrap.cc](#).

Referenced by [poly_ratfun_add\(\)](#), and [poly_ratfun_normalize\(\)](#).

4.108.8.1 poly_sort()

```
void poly_sort (
    PHEAD WORD * a )
```

Sort the polynomial terms

4.108.9 Description

Sorts the terms of a polynomial in Form [poly\(rat\)](#)fun order, i.e. lexicographical order with highest degree first.

4.108.10 Notes

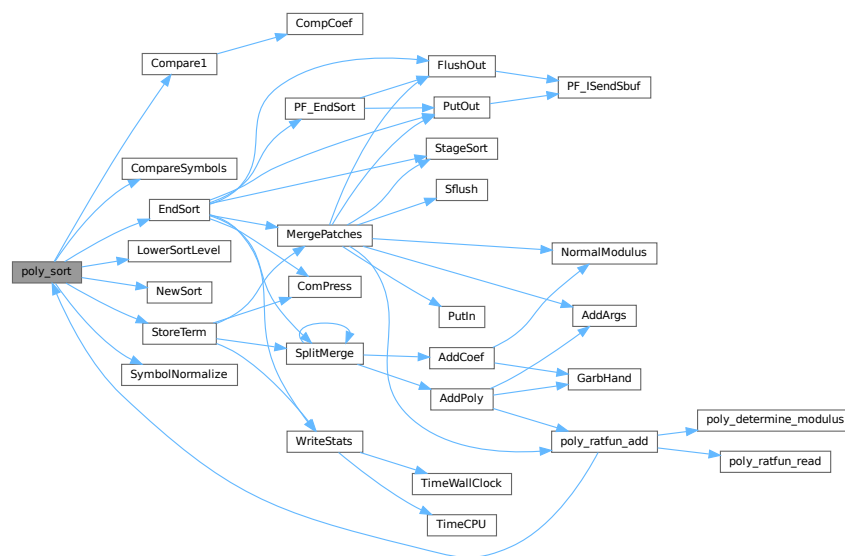
- Uses Form sort routines with custom compare

Definition at line 590 of file [polywrap.cc](#).

References [Compare1\(\)](#), [CompareSymbols\(\)](#), [EndSort\(\)](#), [LowerSortLevel\(\)](#), [NewSort\(\)](#), [StoreTerm\(\)](#), and [SymbolNormalize\(\)](#).

Referenced by [poly_ratfun_add\(\)](#), and [poly_ratfun_normalize\(\)](#).

Here is the call graph for this function:



4.108.10.1 poly_ratfun_add()

```
WORD * poly_ratfun_add (
    PHEAD WORD * t1,
    WORD * t2 )
```

Addition of PolyRatFuns

4.108.11 Description

This method gets two pointers to polyratfuns with up to two arguments each and calculates the sum.

4.108.14 Notes

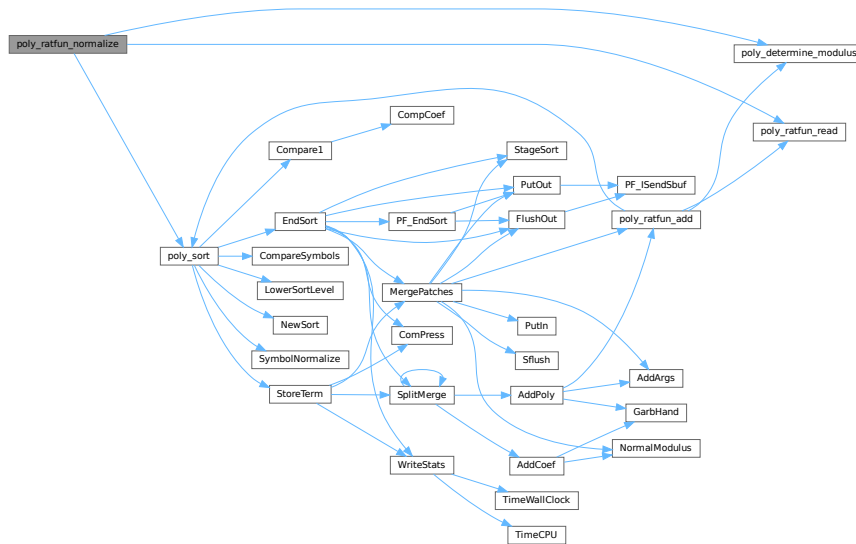
- The result overwrites the original term
- Called from [proces.c](#)
- Calls `poly::operators` and `polygcd::gcd`

Definition at line 769 of file [polywrap.cc](#).

References [poly_determine_modulus\(\)](#), [poly_ratfun_read\(\)](#), and [poly_sort\(\)](#).

Referenced by [PolyFunMul\(\)](#), and [PrepPoly\(\)](#).

Here is the call graph for this function:



4.108.14.1 poly_fix_minus_signs()

```
void poly_fix_minus_signs (
    factorized_poly & a )
```

Definition at line 921 of file [polywrap.cc](#).

4.108.14.2 poly_factorize()

```
WORD * poly_factorize (
    PHEAD WORD * argin,
    WORD * argout,
    bool with_arghead,
    bool is_fun_arg )
```

Factorization of function arguments / dollars

4.108.15 Description

This method factorizes a Form style argument or zero-terminated term list.

4.108.16 Notes

- Called from `poly_factorize_{argument,dollar}`
- Calls `polyfact::factorize`

Definition at line 986 of file `polywrap.cc`.

References `poly_determine_modulus()`.

Referenced by `poly_factorize_argument()`, and `poly_factorize_dollar()`.

Here is the call graph for this function:



4.108.16.1 poly_factorize_argument()

```
int poly_factorize_argument (  
    PHEAD WORD * argin,  
    WORD * argout )
```

Factorization of function arguments

4.108.17 Description

This method factorizes the Form-style argument `argin`.

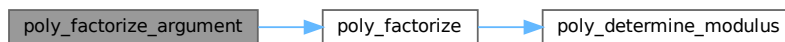
4.108.18 Notes

- The result is written at argout
- Called from [argument.c](#)
- Calls `poly_factorize`

Definition at line 1112 of file [polywrap.cc](#).

References [poly_factorize\(\)](#).

Here is the call graph for this function:



4.108.18.1 poly_factorize_dollar()

```
WORD * poly_factorize_dollar (
    PHEAD WORD * argin )
```

Factorization of dollar variables

4.108.19 Description

This method factorizes a dollar variable.

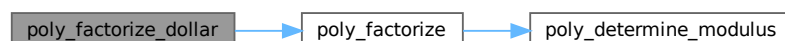
4.108.20 Notes

- The result is written at newly allocated memory.
- Called from [dollar.c](#)
- Calls `poly_factorize`

Definition at line 1146 of file [polywrap.cc](#).

References [poly_factorize\(\)](#).

Here is the call graph for this function:

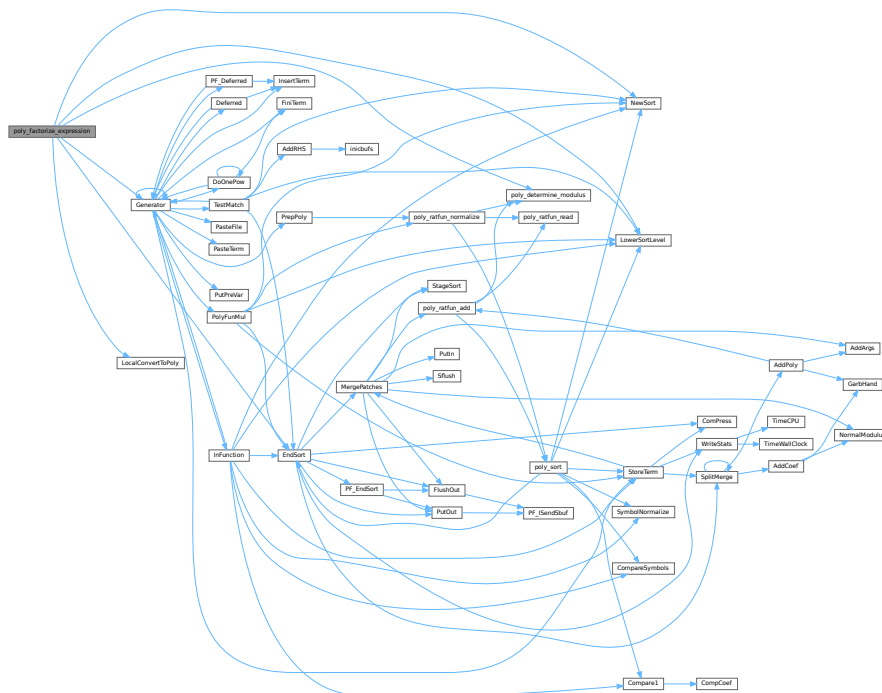



```
int poly_factorize_expression (
    EXPRESSIONS expr )
```

4.108.21 Description

4.108.22 Notes

- Here is the call graph for this function:



4.108.22.1 poly_unfactorize_expression()

```
int poly_unfactorize_expression (
    EXPRESSIONS expr )
```

Unfactorization of expressions

4.108.23 Description

This method expands a factorized expression.

4.108.24 Notes

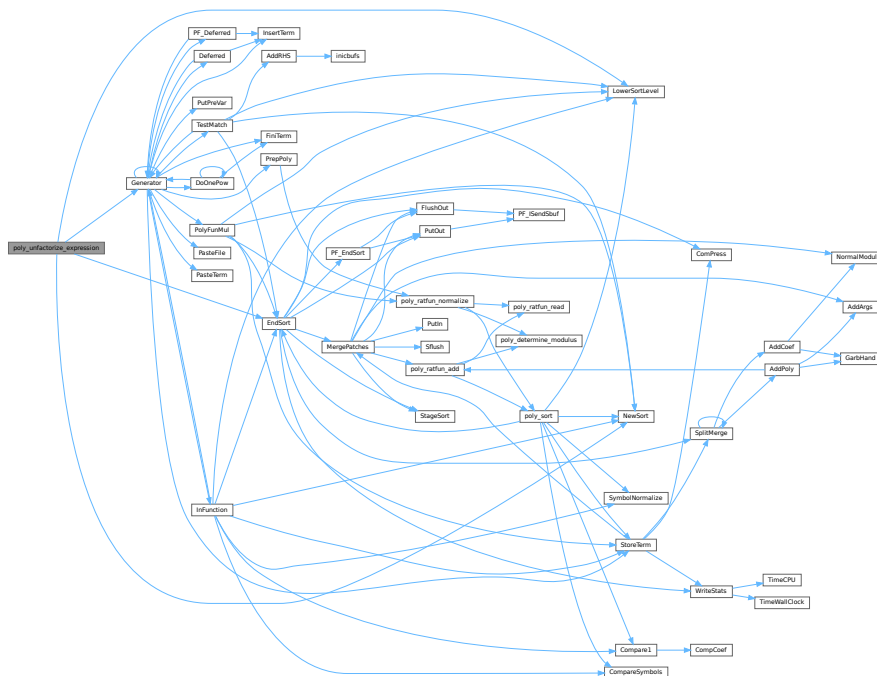
- The result overwrites the input expression
- Called from [proces.c](#)

Definition at line 1535 of file [polywrap.cc](#).

References [EndSort\(\)](#), [Generator\(\)](#), [FiLe::handle](#), [LowerSortLevel\(\)](#), and [NewSort\(\)](#).

Referenced by [PF_Processor\(\)](#), and [Processor\(\)](#).

Here is the call graph for this function:



4.108.24.1 poly_inverse()

```
WORD * poly_inverse (
    PHEAD WORD * arga,
    WORD * argb )
```

Definition at line [1743](#) of file [polywrap.cc](#).

4.108.24.2 poly_mul()

```
WORD * poly_mul (
    PHEAD WORD * a,
    WORD * b )
```

Definition at line [1948](#) of file [polywrap.cc](#).

4.108.24.3 poly_free_poly_vars()

```
void poly_free_poly_vars (
    PHEAD const char * text )
```

Definition at line [2014](#) of file [polywrap.cc](#).

4.108.25 Variable Documentation**4.108.25.1 POLYWRAP_DENOMPOWER_INCREASE_FACTOR**

```
const int POLYWRAP_DENOMPOWER_INCREASE_FACTOR = 2
```

Definition at line [52](#) of file [polywrap.cc](#).

4.109 polywrap.cc

[Go to the documentation of this file.](#)

```
00001
00008 /* # [ License : */
00009 /*
00010 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00011 * When using this file you are requested to refer to the publication
00012 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00013 * This is considered a matter of courtesy as the development was paid
00014 * for by FOM the Dutch physics granting agency and we would like to
00015 * be able to track its scientific use to convince FOM of its value
00016 * for the community.
00017 *
00018 * This file is part of FORM.
00019 *
00020 * FORM is free software: you can redistribute it and/or modify it under the
00021 * terms of the GNU General Public License as published by the Free Software
00022 * Foundation, either version 3 of the License, or (at your option) any later
00023 * version.
00024 *
00025 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00026 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00027 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00028 * details.
00029 *
```

```

00030 *   You should have received a copy of the GNU General Public License along
00031 *   with FORM.  If not, see <http://www.gnu.org/licenses/>.
00032 */
00033 /* #] License : */
00034
00035 #include "poly.h"
00036 #include "polygcd.h"
00037 #include "polyfact.h"
00038
00039 #include <iostream>
00040 #include <vector>
00041 #include <map>
00042 #include <climits>
00043 #include <cassert>
00044
00045 // #define DEBUG
00046
00047 #ifdef DEBUG
00048 #include "mytime.h"
00049 #endif
00050
00051 // denompower is increased by this factor when divmod_heap fails
00052 const int POLYWRAP_DENOMPOWER_INCREASE_FACTOR = 2;
00053
00054 using namespace std;
00055
00056 /*
00057  #[ poly_determine_modulus :
00058 */
00059
00079 WORD poly_determine_modulus (PHEAD bool multi_error, bool is_fun_arg, string message) {
00080
00081     if (AC.ncmod==0) return 0;
00082
00083     if (!is_fun_arg || (AC.modmode & ALSOFUNARGS)) {
00084
00085         if (ABS(AC.ncmod)>1) {
00086             MLOCK(ErrorMessageLock);
00087             MesPrint ((char*)"ERROR: %s with modulus > WORDSIZE not implemented",message.c_str());
00088             MUNLOCK(ErrorMessageLock);
00089             Terminate(-1);
00090         }
00091
00092         if (multi_error && AN.poly_num_vars>1) {
00093             MLOCK(ErrorMessageLock);
00094             MesPrint ((char*)"ERROR: multivariate %s with modulus not implemented",message.c_str());
00095             MUNLOCK(ErrorMessageLock);
00096             Terminate(-1);
00097         }
00098
00099         return *AC.cmod;
00100     }
00101
00102     AN.ncmod = 0;
00103     return 0;
00104 }
00105
00106 /*
00107  #] poly_determine_modulus :
00108  #[ poly_gcd :
00109 */
00110
00124 WORD *poly_gcd(PHEAD WORD *a, WORD *b, WORD fit) {
00125
00126 #ifdef WITHFLINT
00127     if ( AC.FlintPolyFlag && AC.ncmod==0 ) {
00128         WORD *ret = flint_gcd(BHEAD a, b, fit);
00129         return ret;
00130     }
00131 #endif
00132
00133 #ifdef DEBUG
00134     cout << "*** [" << thetime() << "]" << " CALL : poly_gcd" << endl;
00135 #endif
00136
00137 //
00138 //MesPrint("Calling poly_gcd with:");
00139 //{
00140 // WORD *at = a;
00141 // MesPrint("      a:");
00142 // while ( *at ) {
00143 //     MesPrint("      %a",*at,at);
00144 //     at += *at;
00145 // }
00146 // MesPrint("      b:");
00147 // at = b;
00148 // while ( *at ) {

```

```

00149 //      MesPrint("    %a",*at,at);
00150 //      at += *at;
00151 //  }
00152 //}
00153
00154 // Extract variables
00155 vector<WORD *> e;
00156 e.reserve(2);
00157 e.push_back(a);
00158 e.push_back(b);
00159 poly::get_variables(BHEAD e, false, true);
00160
00161 // Check for modulus calculus
00162 WORD modp=poly_determine_modulus(BHEAD true, true, "polynomial GCD");
00163
00164 // Convert to polynomials
00165 poly pa(poly::argument_to_poly(BHEAD a, false, true), modp, 1);
00166 poly pb(poly::argument_to_poly(BHEAD b, false, true), modp, 1);
00167
00168 // Calculate gcd
00169 poly gcd(polygcd::gcd(pa,pb));
00170
00171 // Allocate new memory and convert to Form notation
00172 int newsize = (gcd.size_of_form_notation()+1);
00173 WORD *res;
00174 if ( fit ) {
00175     if ( newsize*sizeof(WORD) >= (size_t)(AM.MaxTer) ) {
00176         MLOCK(ErrorMessageLock);
00177         MesPrint("poly_gcd: Term too complex (%d words). Maybe increasing MaxTermSize (%d words)
can help",
00178                 newsize, AM.MaxTer/sizeof(WORD));
00179         MUNLOCK(ErrorMessageLock);
00180         Terminate(-1);
00181     }
00182     res = TermMalloc("poly_gcd");
00183 }
00184 else {
00185     res = (WORD *)Malloca1(newsize*sizeof(WORD), "poly_gcd");
00186 }
00187 poly::poly_to_argument(gcd, res, false);
00188
00189 poly_free_poly_vars(BHEAD "AN.poly_vars_qcd");
00190
00191 // reset modulo calculation
00192 AN.ncmod = AC.ncmod;
00193 return res;
00194 }
00195
00196 /*
00197 #] poly_gcd :
00198 #[ poly_divmod :
00199
00200 if fit == 1 the answer must fit inside a term.
00201 */
00202
00203 WORD *poly_divmod(PHEAD WORD *a, WORD *b, int divmod, WORD fit) {
00204
00205 #ifdef DEBUG
00206     cout << "*** [" << thetime() << "]" CALL : poly_divmod" << endl;
00207 #endif
00208
00209 // check for modulus calculus
00210 WORD modp=poly_determine_modulus(BHEAD false, true, "polynomial division");
00211
00212 // get variables
00213 vector<WORD *> e;
00214 e.reserve(2);
00215 e.push_back(a);
00216 e.push_back(b);
00217 poly::get_variables(BHEAD e, false, false);
00218
00219 // add extra variables to keep track of denominators
00220 const int DENOMSYMBOL = MAXPOSITIVE;
00221
00222 // WORD *new_poly_vars = (WORD *)Malloca1((AN.poly_num_vars+1)*sizeof(WORD), "AN.poly_vars");
00223 // WCOPY(new_poly_vars, AN.poly_vars, AN.poly_num_vars);
00224 // new_poly_vars[AN.poly_num_vars] = DENOMSYMBOL;
00225 // if (AN.poly_num_vars > 0)
00226 //     M_free(AN.poly_vars, "AN.poly_vars");
00227 // AN.poly_num_vars++;
00228 // AN.poly_vars = new_poly_vars;
00229 // AN.poly_vars[AN.poly_num_vars++] = DENOMSYMBOL;
00230
00231 // convert to polynomials
00232 poly dena(BHEAD 0);
00233 poly denb(BHEAD 0);
00234 poly pa(poly::argument_to_poly(BHEAD a, false, true, &dena), modp, 1);

```

```

00235     poly pb(poly::argument_to_poly(BHEAD b, false, true, &denb), modp, 1);
00236
00237     // remove contents
00238     poly numres(polygcd::integer_content(pa));
00239     poly denres(polygcd::integer_content(pb));
00240     pa /= numres;
00241     pb /= denres;
00242
00243     if (divmod==0) {
00244         numres *= denb;
00245         denres *= dena;
00246     }
00247     else {
00248         denres = dena;
00249     }
00250
00251     poly gcdres(polygcd::integer_gcd(numres,denres));
00252     numres /= gcdres;
00253     denres /= gcdres;
00254
00255     // determine lcoeff(b)
00256     poly lcoeffb(pb.integer_lcoeff());
00257
00258     poly pres(BHEAD 0);
00259
00260     if (!lcoeffb.is_one()) {
00261
00262         if (AN.poly_num_vars > 2) {
00263             // the original polynomial is multivariate (one dummy variable has
00264             // been added), so it is not trivial to determine which power of
00265             // lcoeff(b) can be in the answer
00266
00267             int denompower = 0, prevdenompower = 0;
00268             poly pq(BHEAD 0);
00269             poly pr(BHEAD 0);
00270
00271             // try denompower = 0, if this fails increase denompower until division succeeds
00272             bool div_fail = true;
00273             do
00274             {
00275                 if(denompower < prevdenompower)
00276                 {
00277                     // denompower increased beyond INT_MAX
00278                     MLOCK(ErrorMessageLock);
00279                     MesPrint ((char*)"ERROR: pseudo-division failed in poly_divmod (denompower >
INT_MAX)");
00280                     MUNLOCK(ErrorMessageLock);
00281                     Terminate(1);
00282                 }
00283
00284                 if(denompower != 0)
00285                 {
00286                     // multiply a by lcoeffb^(denompower-prevdenompower)
00287                     WORD n = lcoeffb[lcoeffb[1]];
00288                     RaisPow(BHEAD (UWORD *)&lcoeffb[2+AN.poly_num_vars], &n,
denompower-prevdenompower);
00289                     lcoeffb[1] = 2 + AN.poly_num_vars + ABS(n);
00290                     lcoeffb[0] = 1 + lcoeffb[1];
00291                     lcoeffb[lcoeffb[1]] = n;
00292
00293                     pa *= lcoeffb;
00294                     denres *= lcoeffb;
00295                 }
00296
00297                 // try division
00298                 poly ppow(BHEAD 0);
00299                 poly::divmod_heap(pa,pb,pq,pr,false,true,div_fail); // sets div_fail
00300
00301                 // increase denompower for next iteration
00302                 prevdenompower = denompower;
00303                 denompower = (denompower==0 ? 1 : denompower*POLYWRAP_DENOMPPOWER_INCREASE_FACTOR+1 );
00304             }
00305             while(div_fail);
00306
00307             pres = (divmod==0 ? pq : pr);
00308
00309         }
00310         else {
00311             // one variable, so the power is the difference of the degrees
00312
00313             int denompower = MaX(0, pa.degree(0) - pb.degree(0) + 1);
00314
00315             // multiply a by that power
00316             WORD n = lcoeffb[lcoeffb[1]];
00317             RaisPow(BHEAD (UWORD *)&lcoeffb[2+AN.poly_num_vars], &n, denompower);
00318             lcoeffb[1] = 2 + AN.poly_num_vars + ABS(n);

```

```

00319         lcoeffb[0] = 1 + lcoeffb[1];
00320         lcoeffb[lcoeffb[1]] = n;
00321
00322         pa *= lcoeffb;
00323         denres *= lcoeffb;
00324
00325         pres = (divmod==0 ? pa/pb : pa%pb);
00326
00327     }
00328 }
00329 }
00330 else {
00331
00332     pres = (divmod==0 ? pa/pb : pa%pb);
00333
00334 }
00335
00336 // convert to Form notation
00337 // NOTE: this part can be rewritten with poly::size_of_form_notation_with_den()
00338 // and poly::poly_to_argument_with_den().
00339 WORD *res;
00340
00341 // special case: a=0
00342 if (pres[0]==1) {
00343     if ( fit ) {
00344         res = TermMalloc("poly_divmod");
00345     }
00346     else {
00347         res = (WORD *)Malloca(sizeof(WORD), "poly_divmod");
00348     }
00349     res[0] = 0;
00350 }
00351 else {
00352     pres *= numres;
00353
00354     WORD nden;
00355     UWORD *den = (UWORD *)NumberMalloc("poly_divmod");
00356
00357     // allocate the memory; note that this overestimates the size,
00358     // since the estimated denominators are too large
00359     int ressize = pres.size_of_form_notation() + pres.number_of_terms()*2*ABS(denres[denres[1]]) +
1;
00360
00361     if ( fit ) {
00362         if ( ressize*sizeof(WORD) > (size_t)(AM.MaxTer) ) {
00363             MLOCK(ErrorMessageLock);
00364             MesPrint("poly_divmod: Term too complex (%d words). Maybe increasing MaxTermSize (%d
words) can help",
00365                     ressize, AM.MaxTer/sizeof(WORD));
00366             MUNLOCK(ErrorMessageLock);
00367             Terminate(-1);
00368         }
00369         res = TermMalloc("poly_divmod");
00370     }
00371     else {
00372         res = (WORD *)Malloca(ressize*sizeof(WORD), "poly_divmod");
00373     }
00374     int L=0;
00375
00376     for (int i=1; i!=pres[0]; i+=pres[i]) {
00377
00378         res[L]=1; // length
00379         bool first = true;
00380
00381         for (int j=0; j<AN.poly_num_vars; j++)
00382             if (pres[i+1+j] > 0) {
00383                 if (first) {
00384                     first = false;
00385                     res[L+1] = 1; // symbols
00386                     res[L+2] = 2; // length
00387                 }
00388                 res[L+1+res[L+2]++] = AN.poly_vars[j]; // symbol
00389                 res[L+1+res[L+2]++] = pres[i+1+j]; // power
00390             }
00391
00392         if (!first) res[L] += res[L+2]; // fix length
00393
00394         // numerator
00395         WORD nnum = pres[i+pres[i]-1];
00396         WCOPY(&res[L+res[L]], &pres[i+pres[i]-1-ABS(nnum)], ABS(nnum));
00397
00398         // calculate denominator
00399         nden = denres[denres[1]];
00400         WCOPY(den, &denres[2+AN.poly_num_vars], ABS(nden));
00401
00402         if (nden!=1 || den[0]!=1)
00403             Simplify(BHEAD (UWORD *)&res[L+res[L]], &nnum, den, &nden); // gcd(num,den)

```

```

00404         Pack((UWORD *)&res[L+res[L]], &nnum, den, nden);           // format
00405         res[L] += 2*ABS(nnum)+1;                                     // fix length
00406         res[L+res[L]-1] = SGN(nnum)*(2*ABS(nnum)+1);               // length of coefficient
00407         L += res[L];                                               // fix length
00408     }
00409
00410     res[L] = 0;
00411
00412     NumberFree(den, "poly_divmod");
00413 }
00414
00415 // clean up
00416 poly_free_poly_vars(BHEAD "AN.poly_vars_divmod");
00417
00418 // reset modulo calculation
00419 AN.ncmod = AC.ncmod;
00420
00421 return res;
00422 }
00423
00424 /*
00425 #] poly_divmod :
00426 #[ poly_div :
00427
00428 Routine divides the expression in arg1 by the expression in arg2.
00429 We did not take out special cases.
00430 The arguments are zero terminated sequences of term(s).
00431 The action is to divide arg1 by arg2: [arg1/arg2].
00432 The answer should be a buffer (allocated by Malloc1) with a zero
00433 terminated sequence of terms (or just zero).
00434 */
00435 WORD *poly_div(PHEAD WORD *a, WORD *b, WORD fit) {
00436
00437 #ifdef WITHFLINT
00438     if ( AC.FlintPolyFlag && AC.ncmod==0 ) {
00439         WORD *ret = flint_div(BHEAD a, b, fit);
00440         return ret;
00441     }
00442 #endif
00443
00444 #ifdef DEBUG
00445     cout << "*** [" << thetime() << "]" CALL : poly_div" << endl;
00446 #endif
00447
00448     return poly_divmod(BHEAD a, b, 0, fit);
00449 }
00450
00451 /*
00452 #] poly_div :
00453 #[ poly_rem :
00454
00455 Routine divides the expression in arg1 by the expression in arg2
00456 and takes the remainder.
00457 We did not take out special cases.
00458 The arguments are zero terminated sequences of term(s).
00459 The action is to divide arg1 by arg2 and take the remainder: [arg1%arg2].
00460 The answer should be a buffer (allocated by Malloc1) with a zero
00461 terminated sequence of terms (or just zero).
00462 */
00463 WORD *poly_rem(PHEAD WORD *a, WORD *b, WORD fit) {
00464
00465 #ifdef WITHFLINT
00466     if ( AC.FlintPolyFlag && AC.ncmod==0 ) {
00467         WORD *ret = flint_rem(BHEAD a, b, fit);
00468         return ret;
00469     }
00470 #endif
00471
00472 #ifdef DEBUG
00473     cout << "*** [" << thetime() << "]" CALL : poly_rem" << endl;
00474 #endif
00475
00476     return poly_divmod(BHEAD a, b, 1, fit);
00477 }
00478
00479 /*
00480 #] poly_rem :
00481 #[ poly_ratfun_read :
00482 */
00483
00496 void poly_ratfun_read (WORD *a, poly &nnum, poly &den) {
00497
00498 #ifdef DEBUG
00499     cout << "*** [" << thetime() << "]" CALL : poly_ratfun_read" << endl;
00500 #endif
00501
00502     POLY_GETIDENTITY(nnum);

```



```

00503
00504     int modp = num.modp;
00505
00506     WORD *astop = a+a[1];
00507
00508     bool clean = (a[2] & MUSTCLEANPRF) == 0;
00509
00510     a += FUNHEAD;
00511     if (a >= astop) {
00512         MLOCK(ErrorMessageLock);
00513         MesPrint ((char*)"ERROR: PolyRatFun cannot have zero arguments");
00514         MUNLOCK(ErrorMessageLock);
00515         Terminate(-1);
00516     }
00517
00518     poly den_num(BHEAD 1), den_den(BHEAD 1);
00519
00520     num = poly::argument_to_poly(BHEAD a, true, !clean, &den_num);
00521     num.setmod(modp,1);
00522     NEXTARG(a);
00523
00524     if (a < astop) {
00525         den = poly::argument_to_poly(BHEAD a, true, !clean, &den_den);
00526         den.setmod(modp,1);
00527         NEXTARG(a);
00528     }
00529     else {
00530         den = poly(BHEAD 1, modp, 1);
00531     }
00532
00533     if (a < astop) {
00534         MLOCK(ErrorMessageLock);
00535         MesPrint ((char*)"ERROR: PolyRatFun cannot have more than two arguments");
00536         MUNLOCK(ErrorMessageLock);
00537         Terminate(-1);
00538     }
00539
00540     // JD: At this point, num and den are certainly sorted into the correct order by
00541     // poly::argument_to_poly, but we can't rely on the clean flag to know if there
00542     // are any negative powers. Check for them, and set clean = false if there are any.
00543     vector<WORD> minpower(AN.poly_num_vars, MAXPOSITIVE);
00544
00545     for (int i=1; i<num[0]; i+=num[i]) {
00546         for (int j=0; j<AN.poly_num_vars; j++) {
00547             minpower[j] = Min(minpower[j], num[i+1+j]);
00548         }
00549     }
00550     for (int i=1; i<den[0]; i+=den[i]) {
00551         for (int j=0; j<AN.poly_num_vars; j++) {
00552             minpower[j] = Min(minpower[j], den[i+1+j]);
00553             if ( minpower[j] < 0 ) clean = false;
00554         }
00555     }
00556
00557     if (!clean) {
00558         for (int i=1; i<num[0]; i+=num[i])
00559             for (int j=0; j<AN.poly_num_vars; j++)
00560                 num[i+1+j] -= minpower[j];
00561         for (int i=1; i<den[0]; i+=den[i])
00562             for (int j=0; j<AN.poly_num_vars; j++)
00563                 den[i+1+j] -= minpower[j];
00564
00565         num *= den_den;
00566         den *= den_num;
00567
00568         poly gcd = polygcd::gcd(num,den);
00569         num /= gcd;
00570         den /= gcd;
00571     }
00572 }
00573
00574 /*
00575 #] poly_ratfun_read :
00576 #[ poly_sort :
00577 */
00578
00590 void poly_sort(PHEAD WORD *a) {
00591
00592 #ifdef DEBUG
00593     cout << "*** [" << thetime() << "]" << " CALL : poly_sort" << endl;
00594 #endif
00595     if (NewSort(BHEAD0)) { Terminate(-1); }
00596     AR.CompareRoutine = (COMPAREDUMMY)(&CompareSymbols);
00597
00598     for (int i=ARGHEAD; i<a[0]; i+=a[i]) {
00599         if (SymbolNormalize(a+i)<0 || StoreTerm(BHEAD a+i)) {
00600             AR.CompareRoutine = (COMPAREDUMMY)(&Compare1);

```

```

00601         LowerSortLevel();
00602         Terminate(-1);
00603     }
00604 }
00605
00606 if (EndSort(BHEAD a+ARGHEAD,1) < 0) {
00607     AR.CompareRoutine = (COMPAREDUMMY)(&Compare1);
00608     Terminate(-1);
00609 }
00610
00611 AR.CompareRoutine = (COMPAREDUMMY)(&Compare1);
00612 a[1] = 0; // set dirty flag to zero
00613 }
00614
00615 /*
00616 #] poly_sort :
00617 #[ poly_ratfun_add :
00618 */
00619
00633 WORD *poly_ratfun_add (PHEAD WORD *t1, WORD *t2) {
00634
00635 #ifdef WITHFLINT
00636     if ( AC.FlintPolyFlag && AC.ncmod==0 ) {
00637         WORD *ret = flint_ratfun_add(BHEAD t1, t2);
00638         return ret;
00639     }
00640 #endif
00641
00642     if ( AR.PolyFunExp == 1 ) return PolyRatFunSpecial(BHEAD t1, t2);
00643
00644 #ifdef DEBUG
00645     cout << "*** [" << thetime() << "]" CALL : poly_ratfun_add" << endl;
00646 #endif
00647
00648     WORD *oldworkpointer = AT.WorkPointer;
00649
00650     // Extract variables
00651     vector<WORD *> e;
00652     e.reserve(4);
00653
00654     for (WORD *t=t1+FUNHEAD; t<t1+t1[1];) {
00655         e.push_back(t);
00656         NEXTARG(t);
00657     }
00658     for (WORD *t=t2+FUNHEAD; t<t2+t2[1];) {
00659         e.push_back(t);
00660         NEXTARG(t);
00661     }
00662
00663     assert(e.size() == 4);
00664
00665     poly::get_variables(BHEAD e, true, true);
00666
00667     // Check for modulus calculus
00668     WORD modp=poly_determine_modulus(BHEAD true, true, "PolyRatFun");
00669
00670     // Find numerators / denominators
00671     poly num1(BHEAD 0,modp,1), den1(BHEAD 0,modp,1), num2(BHEAD 0,modp,1), den2(BHEAD 0,modp,1);
00672
00673     poly_ratfun_read(t1, num1, den1);
00674     poly_ratfun_read(t2, num2, den2);
00675
00676     poly num(BHEAD 0),den(BHEAD 0),gcd(BHEAD 0);
00677
00678     // Calculate result
00679     if (den1 != den2) {
00680         gcd = polygcd::gcd(den1,den2);
00681 #ifdef OLDADDITION
00682         num = num1*(den2/gcd) + num2*(den1/gcd);
00683         den = (den1/gcd)*den2;
00684         gcd = polygcd::gcd(num,den);
00685 #else
00686         den = den1/gcd;
00687         num = num1*(den2/gcd) + num2*den;
00688         den = den*den2;
00689         gcd = polygcd::gcd(num,gcd);
00690 #endif
00691     }
00692     else {
00693         num = num1 + num2;
00694         den = den1;
00695         gcd = polygcd::gcd(num,den);
00696     }
00697
00698     num /= gcd;
00699     den /= gcd;
00700

```

```

00701 // Fix sign
00702 if (den.sign() == -1) { num*=poly(BHEAD -1); den*=poly(BHEAD -1); }
00703
00704 // Check size: include FUNHEAD for the prf itself, an ARGHEAD each for num and den,
00705 // and 3 for the final coeff "1/1". We don't know here what the rest of the term looks like,
00706 // but it certainly has at least its total size (so +1):
00707 if ((num.size_of_form_notation() + den.size_of_form_notation() + FUNHEAD + 2*ARGHEAD + 3 + 1)
00708     > AM.MaxTer/(int)sizeof(WORD)) {
00709
00710     MLOCK(ErrorMessageLock);
00711     MesPrint ("ERROR: PolyRatFun doesn't fit in a term");
00712     MesPrint ("(1) num size = %d, den size = %d, rest = %d, MaxTermSize = %d words",
00713             num.size_of_form_notation()+ARGHEAD,
00714             den.size_of_form_notation()+ARGHEAD,
00715             FUNHEAD + 3 + 1,
00716             AM.MaxTer/sizeof(WORD));
00717     MUNLOCK(ErrorMessageLock);
00718     Terminate(-1);
00719 }
00720
00721 // Format result in Form notation
00722 WORD *t = oldworkpointer;
00723
00724 *t++ = AR.PolyFun; // function
00725 *t++ = 0; // length (to be determined)
00726 // *t++ &= ~MUSTCLEANPRF; // clean polyratfun
00727 *t++ = 0;
00728 FILLFUN3(t); // header
00729 poly::poly_to_argument(num,t, true); // argument 1 (numerator)
00730 if (*t>0 && t[1]==DIRTYFLAG) // to Form order
00731     poly_sort(BHEAD t);
00732 t += (*t>0 ? *t : 2);
00733 poly::poly_to_argument(den,t, true); // argument 2 (denominator)
00734 if (*t>0 && t[1]==DIRTYFLAG) // to Form order
00735     poly_sort(BHEAD t);
00736 t += (*t>0 ? *t : 2);
00737
00738 oldworkpointer[1] = t - oldworkpointer; // length
00739 AT.WorkPointer = t;
00740
00741 poly_free_poly_vars(BHEAD "AN.poly_vars_ratfun_add");
00742
00743 // reset modulo calculation
00744 AN.ncmod = AC.ncmod;
00745
00746 return oldworkpointer;
00747 }
00748
00749 /*
00750 #] poly_ratfun_add :
00751 #[ poly_ratfun_normalize :
00752 */
00753
00769 int poly_ratfun_normalize (PHEAD WORD *term) {
00770
00771 #ifdef WITHFLINT
00772     if ( AC.FlintPolyFlag && AC.ncmod==0 ) {
00773         flint_ratfun_normalize(BHEAD term);
00774         return 0;
00775     }
00776 #endif
00777
00778 #ifdef DEBUG
00779     cout << "*** [" << thetime() << " ] CALL : poly_ratfun_normalize" << endl;
00780 #endif
00781
00782 // Strip coefficient
00783 WORD *tstop = term + *term;
00784 int ncoeff = tstop[-1];
00785 tstop -= ABS(ncoeff);
00786
00787 // if only one clean polyratfun, return immediately
00788 int num_polyratfun = 0;
00789
00790 for (WORD *t=term+1; t<tstop; t+=t[1])
00791     if (*t == AR.PolyFun) {
00792         num_polyratfun++;
00793         if ((t[2] & MUSTCLEANPRF) != 0)
00794             num_polyratfun = INT_MAX;
00795         if (num_polyratfun > 1) break;
00796     }
00797
00798 if (num_polyratfun <= 1) return 0;
00799
00800 WORD oldsorttype = AR.SortType;
00801 AR.SortType = SORTHIGFIRST;
00802

```

```

00803 /*
00804     When there are polyratfun's with only one variable: rename them
00805     temporarily to TMPPOLYFUN.
00806 */
00807 for (WORD *t=term+1; t<tstop; t+=t[1]) {
00808     if (*t == AR.PolyFun && (t[1] == FUNHEAD+t[FUNHEAD]
00809         || t[1] == FUNHEAD+2 ) ) { *t = TMPPOLYFUN; }
00810 }
00811
00812 // Extract all variables in the polyfuns
00813 vector<WORD *> e;
00814
00815 for (WORD *t=term+1; t<tstop; t+=t[1]) {
00816     if (*t == AR.PolyFun)
00817         for (WORD *t2 = t+FUNHEAD; t2<t+t[1];) {
00818             e.push_back(t2);
00819             NEXTARG(t2);
00820         }
00821 }
00822
00823 poly::get_variables(BHEAD e, true, true);
00824
00825 // Check for modulus calculus
00826 WORD modp=poly_determine_modulus(BHEAD true, true, "PolyRatFun");
00827
00828 // Accumulate total denominator/numerator and copy the remaining terms
00829 // We start with 'trivial' polynomials
00830 poly num1(BHEAD (UWORD *)tstop, ncoeff/2, modp, 1);
00831 poly den1(BHEAD (UWORD *)tstop+ABS(ncoeff/2), ABS(ncoeff)/2, modp, 1);
00832
00833 WORD *s = term+1;
00834
00835 for (WORD *t=term+1; t<tstop;)
00836     if (*t == AR.PolyFun) {
00837         poly num2(BHEAD 0, modp, 1);
00838         poly den2(BHEAD 0, modp, 1);
00839         poly_ratfun_read(t, num2, den2);
00840
00841         if ((t[2] & MUSTCLEANPRF) != 0) { // first normalize
00842             poly gcd1(polygcd::gcd(num2, den2));
00843             num2 = num2/gcd1;
00844             den2 = den2/gcd1;
00845         }
00846         t += t[1];
00847         poly gcd1(polygcd::gcd(num1, den2));
00848         poly gcd2(polygcd::gcd(num2, den1));
00849
00850         num1 = (num1 / gcd1) * (num2 / gcd2);
00851         den1 = (den1 / gcd2) * (den2 / gcd1);
00852     }
00853     else {
00854         int i = t[1];
00855         if (s != t) { NCOPY(s, t, i) }
00856         else { t += i; s += i; }
00857     }
00858 }
00859
00860 // Fix sign
00861 if (den1.sign() == -1) { num1*=poly(BHEAD -1); den1*=poly(BHEAD -1); }
00862
00863 // Check size: include FUNHEAD for the prf itself, an ARGHEAD each for num and den,
00864 // s-term for the copied term so far, and 3 for final coeff "1/1"
00865 if ((num1.size_of_form_notation() + den1.size_of_form_notation() + FUNHEAD + 2*ARGHEAD
00866     + s-term + 3) > AM.MaxTer/(int)sizeof(WORD)) {
00867     MLOCK(ErrorMessageLock);
00868     MesPrint ("ERROR: PolyRatFun doesn't fit in a term");
00869     MesPrint ("(2) num size = %d, den size = %d, rest = %d, MaxTermSize = %d words",
00870         num1.size_of_form_notation()+ARGHEAD,
00871         den1.size_of_form_notation()+ARGHEAD,
00872         FUNHEAD + s-term + 3,
00873         AM.MaxTer/sizeof(WORD));
00874     MUNLOCK(ErrorMessageLock);
00875     Terminate(-1);
00876 }
00877
00878 // Format result in Form notation
00879 WORD *t = s;
00880 *t++ = AR.PolyFun; // function
00881 *t++ = 0; // size (to be determined)
00882 *t++ &= ~MUSTCLEANPRF; // clean polyratfun
00883 FILLFUN3(t); // header
00884 poly::poly_to_argument(num1, t, true); // argument 1 (numerator)
00885 if (*t>0 && t[1]==DIRTYFLAG) // to Form order
00886     poly_sort(BHEAD t);
00887 t += (*t>0 ? *t : 2);
00888 poly::poly_to_argument(den1, t, true); // argument 2 (denominator)

```

```

00890     if (*t>0 && t[1]==DIRTYFLAG)           // to Form order
00891         poly_sort(BHEAD t);
00892     t += (*t>0 ? *t : 2);
00893
00894     s[1] = t - s;                           // function length
00895
00896     *t++ = 1;                               // term coefficient
00897     *t++ = 1;
00898     *t++ = 3;
00899
00900     term[0] = t-term;                       // term length
00901
00902     poly_free_poly_vars(BHEAD "AN.poly_vars_ratfun_normalize");
00903
00904     // reset modulo calculation
00905     AN.ncmod = AC.ncmod;
00906
00907     tstop = term + *term; tstop -= ABS(tstop[-1]);
00908     for (WORD *t=term+1; t<tstop; t+=t[1]) {
00909         if (*t == TMPPOLYFUN) *t = AR.PolyFun;
00910     }
00911
00912     AR.SortType = oldsorttype;
00913     return 0;
00914 }
00915
00916 /*
00917 #] poly_ratfun_normalize :
00918 #[ poly_fix_minus_signs :
00919 */
00920
00921 void poly_fix_minus_signs (factorized_poly &a) {
00922
00923     if ( a.factor.empty() ) return;
00924
00925     POLY_GETIDENTITY(a.factor[0]);
00926
00927     int overall_sign = +1;
00928
00929     // find term with maximum power of highest symbol
00930     for (int i=0; i<(int)a.factor.size(); i++) {
00931
00932         int maxvar = -1;
00933         int maxpow = -1;
00934         int sign = +1;
00935
00936         WORD *tstop = a.factor[i].terms; tstop += *tstop;
00937         for (WORD *t=a.factor[i].terms+1; t<tstop; t+=*t)
00938             for (int j=0; j<AN.poly_num_vars; j++) {
00939                 int var = AN.poly_vars[j];
00940                 int pow = t[1+j];
00941                 if (pow>0 && (var>maxvar || (var==maxvar && pow>maxpow))) {
00942                     maxvar = var;
00943                     maxpow = pow;
00944                     sign = SGN(*(t+*t-1));
00945                 }
00946             }
00947
00948         // if negative coefficient, multiply by -1
00949         if (sign== -1) {
00950             a.factor[i] *= poly(BHEAD sign);
00951             if (a.power[i] % 2 == 1) overall_sign*=-1;
00952         }
00953     }
00954
00955     // if overall minus sign
00956     if (overall_sign == -1) {
00957         // look at constant factor and multiply by -1
00958         for (int i=0; i<(int)a.factor.size(); i++)
00959             if (a.factor[i].is_integer()) {
00960                 a.factor[i] *= poly(BHEAD -1);
00961                 return;
00962             }
00963
00964         // otherwise, add a factor of -1
00965         a.add_factor(poly(BHEAD -1), 1);
00966     }
00967 }
00968
00969 /*
00970 #] poly_fix_minus_signs :
00971 #[ poly_factorize :
00972 */
00973
00986 WORD *poly_factorize (PHEAD WORD *argin, WORD *argout, bool with_arghead, bool is_fun_arg) {
00987
00988 #ifdef DEBUG

```

```

00989     cout << "*** [" << thetime() << "]" CALL : poly_factorize" << endl;
00990 #endif
00991
00992     poly::get_variables(BHEAD vector<WORD*>(1,argin), with_arghead, true);
00993     poly den(BHEAD 0);
00994     poly a(poly::argument_to_poly(BHEAD argin, with_arghead, true, &den));
00995
00996     // check for modulus calculus
00997     WORD modp=poly_determine_modulus(BHEAD true, is_fun_arg, "polynomial factorization");
00998     a.setmod(modp,1);
00999
01000     // factorize
01001     factorized_poly f(polyfact::factorize(a));
01002     poly_fix_minus_signs(f);
01003
01004     poly num(BHEAD 1);
01005     for (int i=0; i<(int)f.factor.size(); i++)
01006         if (f.factor[i].is_integer())
01007             num = f.factor[i];
01008
01009     // determine size
01010     int len = with_arghead ? ARGHEAD : 0;
01011
01012     if (!num.is_one() || !den.is_one()) {
01013         len++;
01014         len += MAX(ABS(num[num[1]]), den[den[1]])*2+1;
01015         len += with_arghead ? ARGHEAD : 1;
01016     }
01017
01018     for (int i=0; i<(int)f.factor.size(); i++) {
01019         if (!f.factor[i].is_integer()) {
01020             len += f.power[i] * f.factor[i].size_of_form_notation();
01021             len += f.power[i] * (with_arghead ? ARGHEAD : 1);
01022         }
01023     }
01024
01025     len++;
01026
01027     if (argout != NULL) {
01028         // check size
01029         if (len >= AM.MaxTer) {
01030             MLOCK(ErrorMessageLock);
01031             MesPrint ("ERROR: factorization doesn't fit in a term (len = %d, MaxTermSize = %d words)",
01032                 len/sizeof(WORD), AM.MaxTer/sizeof(WORD));
01033             MUNLOCK(ErrorMessageLock);
01034             Terminate(-1);
01035         }
01036     }
01037     else {
01038         // allocate size
01039         argout = (WORD*) Malloc1(len*sizeof(WORD), "poly_factorize");
01040     }
01041
01042     WORD *old_argout = argout;
01043
01044     // constant factor
01045     if (!num.is_one() || !den.is_one()) {
01046         int n = max(ABS(num[num[1]]), ABS(den[den[1]]));
01047
01048         if (with_arghead) {
01049             *argout++ = ARGHEAD + 2 + 2*n;
01050             for (int i=1; i<ARGHEAD; i++)
01051                 *argout++ = 0;
01052         }
01053
01054         *argout++ = 2 + 2*n;
01055
01056         for (int i=0; i<n; i++)
01057             *argout++ = i<ABS(num[num[1]]) ? num[2+AN.poly_num_vars+i] : 0;
01058         for (int i=0; i<n; i++)
01059             *argout++ = i<ABS(den[den[1]]) ? den[2+AN.poly_num_vars+i] : 0;
01060
01061         *argout++ = SGN(num[num[1]]) * (2*n+1);
01062
01063         if (!with_arghead)
01064             *argout++ = 0;
01065         else {
01066             if (ToFast(old_argout, old_argout))
01067                 argout = old_argout+2;
01068         }
01069     }
01070
01071     // non-constant factors
01072     for (int i=0; i<(int)f.factor.size(); i++)
01073         if (!f.factor[i].is_integer())
01074             for (int j=0; j<f.power[i]; j++) {
01075                 poly::poly_to_argument(f.factor[i], argout, with_arghead);

```

```

01076
01077         if (with_arghead)
01078             argout += *argout > 0 ? *argout : 2;
01079         else {
01080             while (*argout!=0) argout+=*argout;
01081             argout++;
01082         }
01083     }
01084
01085     *argout=0;
01086
01087     poly_free_poly_vars(BHEAD "AN.poly_vars_factorize");
01088
01089     // reset modulo calculation
01090     AN.ncmod = AC.ncmod;
01091
01092     return old_argout;
01093 }
01094
01095 /*
01096 #] poly_factorize :
01097 #[ poly_factorize_argument :
01098 */
01099
01100 int poly_factorize_argument(PHEAD WORD *argin, WORD *argout) {
01101
01102 #ifdef DEBUG
01103     cout << "*** [" << thetime() << " ] CALL : poly_factorize_argument" << endl;
01104 #endif
01105
01106 #ifdef WITHFLINT
01107     if ( AC.FlintPolyFlag && AC.ncmod==0 ) {
01108         flint_factorize_argument(BHEAD argin, argout);
01109         return 0;
01110     }
01111 #endif
01112
01113     poly_factorize(BHEAD argin,argout,true,true);
01114     return 0;
01115 }
01116
01117 /*
01118 #] poly_factorize_argument :
01119 #[ poly_factorize_dollar :
01120 */
01121
01122 WORD *poly_factorize_dollar (PHEAD WORD *argin) {
01123
01124 #ifdef DEBUG
01125     cout << "*** [" << thetime() << " ] CALL : poly_factorize_dollar" << endl;
01126 #endif
01127
01128 #ifdef WITHFLINT
01129     if ( AC.FlintPolyFlag && AC.ncmod==0 ) {
01130         return flint_factorize_dollar(BHEAD argin);
01131     }
01132 #endif
01133
01134     return poly_factorize(BHEAD argin,NULL,false,false);
01135 }
01136
01137 /*
01138 #] poly_factorize_dollar :
01139 #[ poly_factorize_expression :
01140 */
01141
01142 int poly_factorize_expression(EXPRESSIONS expr) {
01143
01144 #ifdef DEBUG
01145     cout << "*** [" << thetime() << " ] CALL : poly_factorize_expression" << endl;
01146 #endif
01147
01148     GETIDENTITY;
01149
01150     if (AT.WorkPointer + AM.MaxTer > AT.WorkTop ) {
01151         MLOCK(ErrorMessageLock);
01152         MesWork();
01153         MUNLOCK(ErrorMessageLock);
01154         Terminate(-1);
01155     }
01156
01157     WORD *term = AT.WorkPointer;
01158     WORD startebuf = cbuf[AT.ebufnum].numrhs;
01159     FILEHANDLE *file;
01160     POSITION pos;
01161
01162     FILEHANDLE *oldinfile = AR.infile;

```

```

01199 FILEHANDLE *oldoutfile = AR.outfile;
01200 WORD oldBracketOn = AR.BraceOn;
01201 WORD *oldBrackBuf = AT.BrackBuf;
01202 WORD oldbracketindexflag = AT.bracketindexflag;
01203 char oldCommercial[COMMERCIALSIZE+2];
01204
01205 strcpy(oldCommercial, (char*)AC.Commercial);
01206 strcpy((char*)AC.Commercial, "factorize");
01207
01208 // locate is the input
01209 if (expr->status == HIDDENEXPRESSION || expr->status == HIDDENLEEXPRESSION ||
01210     expr->status == INTOHIDEEXPRESSION || expr->status == INTOHIDELEEXPRESSION) {
01211     AR.InHiBuf = 0; file = AR.hidefile; AR.GetFile = 2;
01212 }
01213 else {
01214     AR.InInBuf = 0; file = AR.outfile; AR.GetFile = 0;
01215 }
01216
01217 // read and write to expression file
01218 AR.infile = AR.outfile = file;
01219
01220 // dummy indices are not allowed
01221 if (expr->numdummies > 0) {
01222     MesPrint("ERROR: factorization with dummy indices not implemented");
01223     Terminate(-1);
01224 }
01225
01226 // determine whether the expression is on file or in memory
01227 if (file->handle >= 0) {
01228     pos = expr->onfile;
01229     SeekFile(file->handle, &pos, SEEK_SET);
01230     if (ISNOTEQUALPOS(pos, expr->onfile)) {
01231         MesPrint("ERROR: something wrong in scratch file [poly_factorize_expression]");
01232         Terminate(-1);
01233     }
01234     file->POposition = expr->onfile;
01235     file->POfull = file->PObuffer;
01236     if (expr->status == HIDDENEXPRESSION)
01237         AR.InHiBuf = 0;
01238     else
01239         AR.InInBuf = 0;
01240 }
01241 else {
01242     file->POfill = (WORD *) ((UBYTE *) (file->PObuffer) + BASEPOSITION(expr->onfile));
01243 }
01244
01245 SetScratch(AR.infile, &(expr->onfile));
01246
01247 // read the first header term
01248 WORD size = GetTerm(BHEAD term);
01249 if (size <= 0) {
01250     MesPrint("ERROR: something wrong with expression [poly_factorize_expression]");
01251     Terminate(-1);
01252 }
01253
01254 // store position: this is where the output will go
01255 pos = expr->onfile;
01256 ADDPOS(pos, size*sizeof(WORD));
01257
01258 // use polynomial as buffer, because it is easy to extend
01259 poly buffer(BHEAD 0);
01260 int bufpos = 0;
01261 int sumcommu = 0;
01262
01263 // read all terms
01264 while (GetTerm(BHEAD term)) {
01265     // substitute non-symbols by extra symbols
01266     sumcommu += DoesCommu(term);
01267     if (sumcommu > 1) {
01268         MesPrint("ERROR: Cannot factorize an expression with more than one noncommuting object");
01269         Terminate(-1);
01270     }
01271     buffer.check_memory(bufpos);
01272     if (LocalConvertToPoly(BHEAD term, buffer.terms + bufpos, startebuf, 0) < 0) {
01273         MesPrint("ERROR: in LocalConvertToPoly [factorize_expression]");
01274         Terminate(-1);
01275     }
01276     bufpos += *(buffer.terms + bufpos);
01277 }
01278 buffer[bufpos] = 0;
01279
01280 // parse the polynomial
01281
01282 AN.poly_num_vars = 0;
01283 poly::get_variables(BHEAD vector<WORD*>(1, buffer.terms), false, true);
01284 poly den(BHEAD 0);
01285 poly a(poly::argument_to_poly(BHEAD buffer.terms, false, true, &den));

```



```

01286
01287 // check for modulus calculus
01288 WORD modp=poly_determine_modulus(BHEAD true, false, "polynomial factorization");
01289 a.setmod(modp,1);
01290
01291 // create output
01292 SetScratch(file, &pos);
01293 NewSort(BHEAD0);
01294
01295 CBUF *C = cbuf+AC.cbufnum;
01296 CBUF *CC = cbuf+AT.ebufnum;
01297
01298 // turn brackets on. We force the existence of a bracket index.
01299 WORD nexpr = expr - Expressions;
01300 AR.BracketOn = 1;
01301 AT.BrackBuf = AM.BracketFactors;
01302 AT.bracketindexflag = 1;
01303 ClearBracketIndex(-nexpr-2); // Clears the index made during primary generation
01304 OpenBracketIndex(nexpr); // Set up a new index
01305
01306 if (a.is_zero()) {
01307     expr->numfactors = 1;
01308 }
01309 else if (a.is_one() && den.is_one()) {
01310     expr->numfactors = 1;
01311
01312     term[0] = 8;
01313     term[1] = SYMBOL;
01314     term[2] = 4;
01315     term[3] = FACTORSYMBOL;
01316     term[4] = 1;
01317     term[5] = 1;
01318     term[6] = 1;
01319     term[7] = 3;
01320
01321     AT.WorkPointer += *term;
01322     Generator(BHEAD term, C->numlhs);
01323     AT.WorkPointer = term;
01324 }
01325 else {
01326     factorized_poly fac;
01327     bool iszero = false;
01328
01329     if (!(expr->vflags & ISFACTORIZED)) {
01330         // factorize the polynomial
01331         fac = polyfact::factorize(a);
01332         poly_fix_minus_signs(fac);
01333     }
01334     else {
01335         int factorsymbol=-1;
01336         for (int i=0; i<AN.poly_num_vars; i++)
01337             if (AN.poly_vars[i] == FACTORSYMBOL)
01338                 factorsymbol = i;
01339
01340         poly denpow(BHEAD 1);
01341
01342         // already factorized, so factorize the factors
01343         for (int i=1; i<=expr->numfactors; i++) {
01344             poly origfac(a.coefficient(factorsymbol, i));
01345             factorized_poly fac2;
01346             if (origfac.is_zero())
01347                 iszero=true;
01348             else {
01349                 fac2 = polyfact::factorize(origfac);
01350                 poly_fix_minus_signs(fac2);
01351                 denpow *= den;
01352             }
01353             for (int j=0; j<(int)fac2.power.size(); j++)
01354                 fac.add_factor(fac2.factor[j], fac2.power[j]);
01355         }
01356
01357         // update denominator, since each factor was scaled
01358         den=denpow;
01359     }
01360
01361     expr->numfactors = 0;
01362
01363     // coefficient
01364     poly num(BHEAD 1);
01365     for (int i=0; i<(int)fac.factor.size(); i++)
01366         if (fac.factor[i].is_integer())
01367             num *= fac.factor[i];
01368
01369     poly gcd(polygcd::integer_gcd(num,den));
01370     den/=gcd;
01371     num/=gcd;
01372

```

```

01373     if (iszero)
01374         expr->numfactors++;
01375
01376     if (!iszero || (expr->vflags & KEEPZERO)) {
01377         if (!num.is_one() || !den.is_one()) {
01378             expr->numfactors++;
01379
01380             int n = max(ABS(num[num[1]]), ABS(den[den[1]]));
01381
01382             term[0] = 6 + 2*n;
01383             term[1] = SYMBOL;
01384             term[2] = 4;
01385             term[3] = FACTORSYMBOL;
01386             term[4] = expr->numfactors;
01387             for (int i=0; i<n; i++) {
01388                 term[5+i] = i<ABS(num[num[1]]) ? num[2+AN.poly_num_vars+i] : 0;
01389                 term[5+n+i] = i<ABS(den[den[1]]) ? den[2+AN.poly_num_vars+i] : 0;
01390             }
01391             term[5+2*n] = SGN(num[num[1]]) * (2*n+1);
01392             AT.WorkPointer += *term;
01393             Generator(BHEAD term, C->numlhs);
01394             AT.WorkPointer = term;
01395         }
01396
01397         vector<poly> fac_arg(fac.factor.size(), poly(BHEAD 0));
01398
01399         // convert the non-constant factors to Form-style arguments
01400         for (int i=0; i<(int)fac.factor.size(); i++)
01401             if (!fac.factor[i].is_integer()) {
01402                 buffer.check_memory(fac.factor[i].size_of_form_notation()+1);
01403                 poly::poly_to_argument(fac.factor[i], buffer.terms, false);
01404                 NewSort(BHEAD0);
01405
01406                 for (WORD *t=buffer.terms; *t!=0; t+=*t) {
01407                     // substitute extra symbols
01408                     if (ConvertFromPoly(BHEAD t, term, numxsymbol,
CC->numrhs-startebuf+numxsymbol,
01409                                     startebuf-numxsymbol, 1) <= 0 ) {
01410                         MesPrint("ERROR: in ConvertFromPoly [factorize_expression]");
01411                         Terminate(-1);
01412                         return(-1);
01413                     }
01414
01415                     // store term
01416                     AT.WorkPointer += *term;
01417                     Generator(BHEAD term, C->numlhs);
01418                     AT.WorkPointer = term;
01419                 }
01420
01421                 // sort and store in buffer
01422                 WORD *buffer;
01423                 if (EndSort(BHEAD (WORD *)((void *)(&buffer)),2) < 0) return -1;
01424
01425                 LONG bufsize=0;
01426                 for (WORD *t=buffer; *t!=0; t+=*t)
01427                     bufsize+=*t;
01428
01429                 fac_arg[i].check_memory(bufsize+ARGHEAD+1);
01430
01431                 for (int j=0; j<ARGHEAD; j++)
01432                     fac_arg[i].terms[j] = 0;
01433                 fac_arg[i].terms[0] = ARGHEAD + bufsize;
01434                 WCOPY(fac_arg[i].terms+ARGHEAD, buffer, bufsize);
01435                 M_free(buffer, "polynomial factorization");
01436             }
01437
01438         // compare and sort the factors in Form notation
01439         vector<int> order;
01440         vector<vector<int>> comp(fac.factor.size(), vector<int>(fac.factor.size(), 0));
01441
01442         for (int i=0; i<(int)fac.factor.size(); i++)
01443             if (!fac.factor[i].is_integer()) {
01444                 order.push_back(i);
01445
01446                 for (int j=i+1; j<(int)fac.factor.size(); j++)
01447                     if (!fac.factor[j].is_integer()) {
01448                         comp[i][j] = CompArg(fac_arg[j].terms, fac_arg[i].terms);
01449                         comp[j][i] = -comp[i][j];
01450                     }
01451             }
01452
01453         for (int i=0; i<(int)order.size(); i++)
01454             for (int j=0; j+1<(int)order.size(); j++)
01455                 if (comp[order[i]][order[j]] == 1)
01456                     swap(order[i], order[j]);
01457
01458         // create the final expression

```

```

01459         for (int i=0; i<(int)order.size(); i++)
01460             for (int j=0; j<fac.power[order[i]]; j++) {
01461
01462                 expr->numfactors++;
01463
01464                 WORD *tstop = fac_arg[order[i]].terms + *fac_arg[order[i]].terms;
01465                 for (WORD *t=fac_arg[order[i]].terms+ARGHEAD; t<tstop; t+=*t) {
01466
01467                     WCOPY(term+4, t, *t);
01468
01469                     // add special symbol "factor_"
01470                     *term = *(term+4) + 4;
01471                     *(term+1) = SYMBOL;
01472                     *(term+2) = 4;
01473                     *(term+3) = FACTORSYMBOL;
01474                     *(term+4) = expr->numfactors;
01475
01476                     // store term
01477                     AT.WorkPointer += *term;
01478                     Generator(BHEAD term, C->numlhs);
01479                     AT.WorkPointer = term;
01480                 }
01481             }
01482     }
01483 }
01484
01485 // final sorting
01486 if (EndSort(BHEAD NULL,0) < 0) {
01487     LowerSortLevel();
01488     Terminate(-1);
01489 }
01490
01491 // set factorized flag
01492 if (expr->numfactors > 0)
01493     expr->vflags |= ISFACTORIZED;
01494
01495 // clean up
01496 AR.infile = oldinfile;
01497 AR.outfile = oldoutfile;
01498 AR.BracketOn = oldBracketOn;
01499 AT.BrackBuf = oldBrackBuf;
01500 AT.bracketindexflag = oldbracketindexflag;
01501 strcpy((char*)AC.Commercial, oldCommercial);
01502
01503 poly_free_poly_vars(BHEAD "AN.poly_vars_factorize_expression");
01504
01505 return 0;
01506 }
01507
01508 /*
01509 #] poly_factorize_expression :
01510 #[ poly_unfactorize_expression :
01511 */
01512
01513 #if ( SUBEXPSIZE == 5 )
01514 static WORD genericterm[] = {38,1,4,FACTORSYMBOL,0
01515 ,EXPRESSION,15,0,1,0,13,10,8,1,4,FACTORSYMBOL,0,1,1,3
01516 ,EXPRESSION,15,0,1,0,13,10,8,1,4,FACTORSYMBOL,0,1,1,3
01517 ,1,1,3,0};
01518 static WORD genericterm2[] = {23,1,4,FACTORSYMBOL,0
01519 ,EXPRESSION,15,0,1,0,13,10,8,1,4,FACTORSYMBOL,0,1,1,3
01520 ,1,1,3,0};
01521 #endif
01522
01523 int poly_unfactorize_expression(EXPRESSIONS expr)
01524 {
01525     GETIDENTITY;
01526     int i, j, nfac = expr->numfactors, nfacp, nexpr = expr - Expressions;
01527     int expriszero = 0;
01528
01529     FILEHANDLE *oldinfile = AR.infile;
01530     FILEHANDLE *oldoutfile = AR.outfile;
01531     char oldCommercial[COMMERCIALSIZE+2];
01532
01533     WORD *oldworkpointer = AT.WorkPointer;
01534     WORD *term = AT.WorkPointer, *t, *w, size;
01535
01536     FILEHANDLE *file;
01537     POSITION pos, oldpos;
01538
01539     WORD oldBracketOn = AR.BracketOn;
01540     WORD *oldBrackBuf = AT.BrackBuf;
01541     CBUF *C = cbuf+AC.cbunum;
01542
01543     if ( ( expr->vflags & ISFACTORIZED ) == 0 ) return(0);
01544
01545     if ( AT.WorkPointer + AM.MaxTer > AT.WorkTop ) {

```

```

01558         MLOCK(ErrorMessageLock);
01559         MesWork();
01560         MUNLOCK(ErrorMessageLock);
01561         Terminate(-1);
01562     }
01563
01564     oldpos = AS.OldOnFile[nexpr];
01565     AS.OldOnFile[nexpr] = expr->onfile;
01566
01567     strcpy(oldCommercial, (char*)AC.Commercial);
01568     strcpy((char*)AC.Commercial, "unfactorize");
01569 /*
01570     locate the input
01571 */
01572     if ( expr->status == HIDDENEXPRESSION || expr->status == HIDDENLEXPRESSION ||
01573         expr->status == INTOHIDEGEXPRESSION || expr->status == INTOHIDELEXPRESSION ) {
01574         AR.InHiBuf = 0; file = AR.hidefile; AR.GetFile = 2;
01575     }
01576     else {
01577         AR.InInBuf = 0; file = AR.outfile; AR.GetFile = 0;
01578     }
01579 /*
01580     read and write to expression file
01581 */
01582     AR.infile = AR.outfile = file;
01583 /*
01584     set the input file to the correct position
01585 */
01586     if ( file->handle >= 0 ) {
01587         pos = expr->onfile;
01588         SeekFile(file->handle, &pos, SEEK_SET);
01589         if (ISNOTEQUALPOS(pos, expr->onfile)) {
01590             MesPrint("ERROR: something wrong in scratch file unfactorize_expression");
01591             Terminate(-1);
01592         }
01593         file->POposition = expr->onfile;
01594         file->POfull = file->PObuffer;
01595         if ( expr->status == HIDDENEXPRESSION )
01596             AR.InHiBuf = 0;
01597         else
01598             AR.InInBuf = 0;
01599     }
01600     else {
01601         file->POfill = (WORD *) ((UBYTE *) (file->PObuffer) + BASEPOSITION(expr->onfile));
01602     }
01603     SetScratch(AR.infile, &(expr->onfile));
01604 /*
01605     Test for whether the first factor is zero.
01606 */
01607     if ( GetFirstBracket(term, nexpr) < 0 ) Terminate(-1);
01608     if ( term[4] != 1 || *term != 8 || term[1] != SYMBOL || term[3] != FACTORSYMBOL || term[4] != 1 )
01609     {
01610         expriszero = 1;
01611     }
01612     SetScratch(AR.infile, &(expr->onfile));
01613 /*
01614     Read the prototype. After this we have the file ready for the output at pos.
01615 */
01616     size = GetTerm(BHEAD term);
01617     if ( size <= 0 ) {
01618         MesPrint ("ERROR: something wrong with expression unfactorize_expression");
01619         Terminate(-1);
01620     }
01621     pos = expr->onfile;
01622     ADDPOS(pos, size*sizeof(WORD));
01623 /*
01624     Set the brackets straight
01625 */
01626     AR.BracketOn = 1;
01627     AT.BrackBuf = AM.BracketFactors;
01628     AT.bracketinfo = 0;
01629     while ( nfac > 2 ) {
01630         nfacp = nfac - nfac%2;
01631 /*
01632     Prepare the bracket index. We have:
01633         e->bracketinfo: the old input bracket index
01634         e->newbracketinfo: the bracket index made for our current input
01635     We need to keep e->bracketinfo in case other workers need it (InParallel)
01636     Hence we work with AT.bracketinfo which takes priority.
01637     Note that in Processor we forced a newbracketinfo to be made.
01638 */
01639     if ( AT.bracketinfo != 0 ) ClearBracketIndex(-1);
01640     AT.bracketinfo = expr->newbracketinfo;
01641     OpenBracketIndex(nexpr);
01642 /*
01643     Now emulate the terms:
01644         sum_(i,0,nfacp,2,factor_^(i/2+1)*F[factor_^(i+1)]*F[factor_^(i+2)])

```

```

01644         +factor_^(nfacp/2+1)*F[factor_^nfac]
01645     */
01646     NewSort(BHEAD0);
01647     if ( expriszero == 0 ) {
01648         for ( i = 0; i < nfacp; i += 2 ) {
01649             t = genericterm; w = term = oldworkpointer;
01650             j = *t; NCOPY(w,t,j);
01651             term[4] = i/2+1;
01652             term[7] = nexpr;
01653             term[16] = i+1;
01654             term[22] = nexpr;
01655             term[31] = i+2;
01656             AT.WorkPointer = term + *term;
01657             Generator(BHEAD term, C->numlhs);
01658         }
01659         if ( nfac > nfacp ) {
01660             t = genericterm2; w = term = oldworkpointer;
01661             j = *t; NCOPY(w,t,j);
01662             term[4] = i/2+1;
01663             term[7] = nexpr;
01664             term[16] = nfac;
01665             AT.WorkPointer = term + *term;
01666             Generator(BHEAD term, C->numlhs);
01667         }
01668     }
01669     if ( EndSort(BHEAD AM.S0->sBuffer,0) < 0 ) {
01670         LowerSortLevel();
01671         Terminate(-1);
01672     }
01673     /*
01674     Set the file back into reading position
01675     */
01676     SetScratch(file, &pos);
01677     nfac = (nfac+1)/2;
01678     if ( expriszero ) { nfac = 1; }
01679 }
01680 if ( AT.bracketinfo != 0 ) ClearBracketIndex(-1);
01681 AT.bracketinfo = expr->newbracketinfo;
01682 expr->newbracketinfo = 0;
01683 /*
01684 Reset the brackets to make them ready for the final pass
01685 */
01686 AR.BracketOn = oldBracketOn;
01687 AT.BrackBuf = oldBrackBuf;
01688 if ( AR.BracketOn ) OpenBracketIndex(nexpr);
01689 /*
01690 We distinguish two cases: nfac == 2 and nfac == 1
01691 After preparing the term we skip the factor_ part.
01692 */
01693 NewSort(BHEAD0);
01694 if ( expriszero == 0 ) {
01695     if ( nfac == 1 ) {
01696         t = genericterm2; w = term = oldworkpointer;
01697         j = *t; NCOPY(w,t,j);
01698         term[7] = nexpr;
01699         term[16] = nfac;
01700     }
01701     else if ( nfac == 2 ) {
01702         t = genericterm; w = term = oldworkpointer;
01703         j = *t; NCOPY(w,t,j);
01704         term[7] = nexpr;
01705         term[16] = 1;
01706         term[22] = nexpr;
01707         term[31] = 2;
01708     }
01709     else {
01710         AS.OldOnFile[nexpr] = oldpos;
01711         return(-1);
01712     }
01713     term[4] = term[0]-4;
01714     term += 4;
01715     AT.WorkPointer = term + *term;
01716     Generator(BHEAD term, C->numlhs);
01717 }
01718 if ( EndSort(BHEAD AM.S0->sBuffer,0) < 0 ) {
01719     LowerSortLevel();
01720     Terminate(-1);
01721 }
01722 /*
01723 Final Cleanup
01724 */
01725 expr->numfactors = 0;
01726 expr->vflags &= ~ISFACTORIZED;
01727 if ( AT.bracketinfo != 0 ) ClearBracketIndex(-1);
01728
01729 AR.infile = oldinfile;
01730 AR.outfile = oldoutfile;

```

```

01731     strcpy((char*)AC.Commercial, oldCommercial);
01732     AT.WorkPointer = oldworkpointer;
01733     AS.OldOnFile[nexpr] = oldpos;
01734
01735     return(0);
01736 }
01737
01738 /*
01739 #] poly_unfactorize_expression :
01740 #[ poly_inverse :
01741 */
01742
01743 WORD *poly_inverse(PHEAD WORD *arga, WORD *argb) {
01744
01745 #ifdef WITHFLINT
01746     if ( AC.FlintPolyFlag && AC.ncmod==0 ) {
01747         return flint_inverse(BHEAD arga, argb);
01748     }
01749 #endif
01750
01751 #ifdef DEBUG
01752     cout << "*** [" << thetime() << "]" CALL : poly_inverse" << endl;
01753 #endif
01754
01755     // Extract variables
01756     vector<WORD *> e;
01757     e.reserve(2);
01758     e.push_back(arga);
01759     e.push_back(argb);
01760     poly::get_variables(BHEAD e, false, true);
01761
01762     if (AN.poly_num_vars > 1) {
01763         MLOCK(ErrorMessageLock);
01764         MesPrint ((char*)"ERROR: multivariate polynomial inverse is generally impossible");
01765         MUNLOCK(ErrorMessageLock);
01766         Terminate(-1);
01767     }
01768
01769     poly finalden(BHEAD 1), finalres(BHEAD 1);
01770     int ressize = 0;
01771     WORD *res = NULL;
01772
01773     // Convert to polynomials
01774     poly dena(BHEAD 0); // We need to keep the overall denominator of arga, to multiply the result
01775     poly a(poly::argument_to_poly(BHEAD arga, false, true, &dena));
01776     poly b(poly::argument_to_poly(BHEAD argb, false, true));
01777
01778     // Divide out the integer content, FORM has not already done this.
01779     poly content_a(BHEAD 0), content_b(BHEAD 0);
01780     content_a = polygcd::integer_content(a);
01781     content_b = polygcd::integer_content(b);
01782     a /= content_a;
01783     b /= content_b;
01784
01785     poly invamodp(BHEAD 0), invbmodp(BHEAD 0);
01786     WORD modp = 0;
01787
01788     // Special cases:
01789     // Possibly strange that we give 1 for inverse_(x1,1) but here we take MMA's convention.
01790     if ((a.is_one() && b.is_one()) || a.is_one()) {
01791         finalres = poly(BHEAD 1);
01792     }
01793     else if (b.is_one()) {
01794         finalres = poly(BHEAD 0);
01795     }
01796     else {
01797         // Check for modulus calculus
01798         modp=poly_determine_modulus(BHEAD true, true, "polynomial inverse");
01799         const bool mod_calc = modp==0 ? false : true;
01800         a.setmod(modp,1);
01801         b.setmod(modp,1);
01802
01803         // Check the gcd of a,b: if it is != 1, the inverse does not exist.
01804         poly gcd(polygcd::gcd(a,b));
01805         if (!gcd.is_one()) {
01806             MLOCK(ErrorMessageLock);
01807             if (mod_calc) {
01808                 MesPrint ((char*)"ERROR: polynomial inverse does not exist (mod %d)", modp);
01809             }
01810             else {
01811                 MesPrint ((char*)"ERROR: polynomial inverse does not exist");
01812             }
01813             MUNLOCK(ErrorMessageLock);
01814             Terminate(-1);
01815         }
01816
01817         bool inv_exists = true;

```

```

01818     do {
01819         // If we are not using modulus calculus, find a suitable prime for xgcd:
01820         if (!mod_calc) {
01821             vector<int> x(1,0);
01822             modp = polyfact::choose_prime(a.integer_lcoeff()*b.integer_lcoeff(), x, modp);
01823         }
01824
01825         poly amodp(a,modp,1);
01826         poly bmodp(b,modp,1);
01827
01828         // Calculate gcd
01829         vector<poly> xgcd(polyfact::extended_gcd_Euclidean_lifted(amodp,bmodp));
01830         invamodp = poly(xgcd[0]);
01831         invbmodp = poly(xgcd[1]);
01832
01833         inv_exists = ((invamodp * amodp) % bmodp).is_one();
01834         if (!inv_exists && mod_calc) {
01835             // Control should not reach here!
01836             MLOCK(ErrorMessageLock);
01837             MesPrint ((char*)"ERROR: polynomial inverse does not exist (mod %d) B", modp);
01838             MUNLOCK(ErrorMessageLock);
01839             Terminate(-1);
01840         }
01841
01842         // If the inverse does not exist and we are not working in modulus calculus,
01843         // choose a new prime and try again. This loop should always terminate, as
01844         // we have already checked that gcd(a,b) == 1.
01845     } while (!inv_exists);
01846
01847     // estimate of the size of the Form notation; might be extended later
01848     ressize = invamodp.size_of_form_notation()+1;
01849     res = (WORD *)Malloca(ressize*sizeof(WORD), "poly_inverse");
01850
01851     // initialize polynomials to store the result
01852     poly primepower(BHEAD modp);
01853     poly inva(invamodp,modp,1);
01854     poly invb(invbmodp,modp,1);
01855
01856     while (true) {
01857         // convert to Form notation
01858         int j=0;
01859         WORD n=0;
01860         for (int i=1; i<inva[0]; i+=inva[i]) {
01861
01862             // check whether res should be extended
01863             while (ressize < j + 2*ABS(inva[i+inva[i]-1]) + (inva[i+1]>0?4:0) + 3) {
01864                 int newressize = 2*ressize;
01865
01866                 WORD *newres = (WORD *)Malloca(newressize*sizeof(WORD), "poly_inverse");
01867                 WCOPY(newres, res, ressize);
01868                 M_free(res, "poly_inverse");
01869                 res = newres;
01870                 ressize = newressize;
01871             }
01872
01873             res[j] = 1;
01874             if (inva[i+1]>0) {
01875                 res[j+res[j]++] = SYMBOL;
01876                 res[j+res[j]++] = 4;
01877                 res[j+res[j]++] = AN.poly_vars[0];
01878                 res[j+res[j]++] = inva[i+1];
01879             }
01880             MakeLongRational(BHEAD (UWORD *)&inva[i+2], inva[i+inva[i]-1],
01881                             (UWORD *)&primepower.terms[3],
01882                             primepower.terms[primepower.terms[1]],
01883                             (UWORD *)&res[j+res[j]], &n);
01884             res[j] += 2*ABS(n);
01885             res[j+res[j]++] = SGN(n)*(2*ABS(n)+1);
01886             j += res[j];
01887         }
01888         res[j]=0;
01889
01890         // if modulus calculus is set, this is the answer
01891         if (a.modp != 0) break;
01892
01893         // otherwise check over integers
01894         poly den(BHEAD 0);
01895         poly check(poly::argument_to_poly(BHEAD res, false, true, &den));
01896         // Shortcut: if b's lcoeff doesn't divide the lcoeff of check*a,
01897         // b certainly doesn't divide check*a:
01898         if (poly::divides(b.integer_lcoeff(), check.integer_lcoeff()*a.integer_lcoeff())) {
01899             check = check*a - den;
01900             if (poly::divides(b, check)) break;
01901         }
01902
01903         // if incorrect, lift with quadratic p-adic Newton's iteration.
01904         poly error((poly(BHEAD 1) - a*inva - b*invb) / primepower);

```

```

01904         poly errormodpp(error, modp, inva.modn);
01905
01906         inva.modn *= 2;
01907         invb.modn *= 2;
01908
01909         poly dinva((inva * errormodpp) % b);
01910         poly dinvb((invb * errormodpp) % a);
01911
01912         inva += dinva * primepower;
01913         invb += dinvb * primepower;
01914
01915         primepower *= primepower;
01916     }
01917
01918     // One more round trip from form -> poly -> form, to multiply by dena in an easy way
01919     finalres = poly(poly::argument_to_poly(BHEAD res, false, true, &finalden));
01920 }
01921
01922 finalres *= dena;
01923 // The overall denominator additionally needs to be multiplied by content_a:
01924 finalden *= content_a;
01925 const WORD finalden_size = finalden.terms[finalden.terms[1]];
01926 const int finalsize = finalres.size_of_form_notation_with_den(finalden_size)+1;
01927 if (ressize < finalsize) {
01928     if (res != NULL) {
01929         M_free(res, "poly_inverse");
01930     }
01931     res = (WORD *)Malloca(finalsize*sizeof(WORD), "poly_inverse");
01932 }
01933 poly::poly_to_argument_with_den(finalres, finalden_size,
01934     (UWORD*)&(finalden.terms[finalden.terms[1] - ABS(finalden_size)]), res, false);
01935
01936 // clean up and reset modulo calculation
01937 poly_free_poly_vars(BHEAD "AN.poly_vars_inverse");
01938
01939 AN.ncmod = AC.ncmod;
01940 return res;
01941 }
01942
01943 /*
01944 #] poly_inverse :
01945 #[ poly_mul :
01946 */
01947
01948 WORD *poly_mul(PHEAD WORD *a, WORD *b) {
01949
01950 #ifdef DEBUG
01951     cout << "*** [" << thetime() << "]" CALL : poly_mul" << endl;
01952 #endif
01953
01954 #ifdef WITHFLINT
01955     if ( AC.FlintPolyFlag && AC.ncmod==0 ) {
01956         return flint_mul(BHEAD a, b);
01957     }
01958 #endif
01959
01960     // Extract variables
01961     vector<WORD *> e;
01962     e.reserve(2);
01963     e.push_back(a);
01964     e.push_back(b);
01965     poly::get_variables(BHEAD e, false, false); // TODO: any performance effect by sort_vars=true?
01966
01967     // Convert to polynomials
01968     poly dena(BHEAD 0);
01969     poly denb(BHEAD 0);
01970     poly pa(poly::argument_to_poly(BHEAD a, false, true, &dena));
01971     poly pb(poly::argument_to_poly(BHEAD b, false, true, &denb));
01972
01973     // Check for modulus calculus
01974     WORD modp = poly_determine_modulus(BHEAD true, true, "polynomial multiplication");
01975     pa.setmod(modp, 1);
01976
01977     // NOTE: mul_ is currently implemented by translating negative powers of
01978     // symbols to extra symbols. For future improvement, it may be better to
01979     // compute
01980     // (content(a) * content(b)) * (a/content(a)) * (b/content(b))
01981     // to avoid introducing extra symbols for "mixed" cases, e.g.,
01982     // (1+x) * (1/x) -> (1+x) * (1+Z1_).
01983     assert(dena.is_integer());
01984     assert(denb.is_integer());
01985     assert(modp == 0 || dena.is_one());
01986     assert(modp == 0 || denb.is_one());
01987
01988     // multiplication
01989     pa *= pb;
01990

```



```

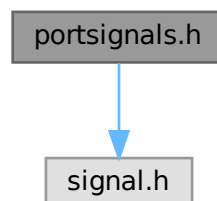
01991     // convert to Form notation
01992     WORD *res;
01993     if (dena.is_one() && denb.is_one()) {
01994         res = (WORD *)Malloca1((pa.size_of_form_notation() + 1) * sizeof(WORD), "poly_mul");
01995         poly::poly_to_argument(pa, res, false);
01996     } else {
01997         dena *= denb;
01998         res = (WORD *)Malloca1((pa.size_of_form_notation_with_den(dena[dena[1]]) + 1) * sizeof(WORD),
01999 "poly_mul");
01999         poly::poly_to_argument_with_den(pa, dena[dena[1]], (const UWORD *)&dena[2+AN.poly_num_vars],
02000 res, false);
02001     }
02002     // clean up and reset modulo calculation
02003     poly_free_poly_vars(BHEAD "AN.poly_vars_mul");
02004     AN.ncmod = AC.ncmod;
02005     return res;
02006 }
02007 }
02008
02009 /*
02010 #] poly_mul :
02011 #[ poly_free_poly_vars :
02012 */
02013
02014 void poly_free_poly_vars(PHEAD const char *text)
02015 {
02016     if ( AN.poly_vars_type == 0 ) {
02017         TermFree(AN.poly_vars, text);
02018     }
02019     else {
02020         M_free(AN.poly_vars, text);
02021     }
02022     AN.poly_num_vars = 0;
02023     AN.poly_vars = 0;
02024 }
02025
02026 /*
02027 #] poly_free_poly_vars :
02028 */

```

4.110 portsignals.h File Reference

#include <signal.h>

Include dependency graph for portsignals.h:



Macros

- #define [FATAL_SIG_ERROR](#) 4
- #define [NSIG](#) (1024)
- #define [SIGSEGV](#) (NSIG+1)
- #define [SIGFPE](#) (NSIG+2)

- `#define SIGILL (NSIG+3)`
- `#define SIGEMT (NSIG+4)`
- `#define SIGSYS (NSIG+5)`
- `#define SIGPIPE (NSIG+6)`
- `#define SIGLOST (NSIG+7)`
- `#define SIGXCPU (NSIG+8)`
- `#define SIGXFSZ (NSIG+9)`
- `#define SIGTERM (NSIG+10)`
- `#define SIGINT (NSIG+11)`
- `#define SIGQUIT (NSIG+12)`
- `#define SIGHUP (NSIG+13)`
- `#define SIGALRM (NSIG+14)`
- `#define SIGVTALRM (NSIG+15)`
- `#define SIGPROF (NSIG+16)`

4.110.1 Detailed Description

Contains definitions for signals used/intercepted in FORM.

Some systems (especially LINUX) have not enough signals available so some of the (!documented!) signals are not defined. This file contains the definition of all signals used in the program. If the signal is not defined we define it as unused ($>NSIG$).

The include of signal.h must be first, before we try to define undefined signals.

Definition in file [portsignals.h](#).

4.110.2 Macro Definition Documentation

4.110.2.1 FATAL_SIG_ERROR

```
#define FATAL_SIG_ERROR 4
```

Definition at line 47 of file [portsignals.h](#).

4.110.2.2 NSIG

```
#define NSIG (1024)
```

Definition at line 53 of file [portsignals.h](#).

4.110.2.3 SIGSEGV

```
#define SIGSEGV (NSIG+1)
```

Definition at line 57 of file [portsignals.h](#).

4.110.2.4 SIGFPE

```
#define SIGFPE (NSIG+2)
```

Definition at line 60 of file [portsignals.h](#).

4.110.2.5 SIGILL

```
#define SIGILL (NSIG+3)
```

Definition at line 63 of file [portsignals.h](#).

4.110.2.6 SIGEMT

```
#define SIGEMT (NSIG+4)
```

Definition at line 66 of file [portsignals.h](#).

4.110.2.7 SIGSYS

```
#define SIGSYS (NSIG+5)
```

Definition at line 69 of file [portsignals.h](#).

4.110.2.8 SIGPIPE

```
#define SIGPIPE (NSIG+6)
```

Definition at line 72 of file [portsignals.h](#).

4.110.2.9 SIGLOST

```
#define SIGLOST (NSIG+7)
```

Definition at line 75 of file [portsignals.h](#).

4.110.2.10 SIGXCPU

```
#define SIGXCPU (NSIG+8)
```

Definition at line 78 of file [portsignals.h](#).

4.110.2.11 SIGXFSZ

```
#define SIGXFSZ (NSIG+9)
```

Definition at line 81 of file [portsignals.h](#).

4.110.2.12 SIGTERM

```
#define SIGTERM (NSIG+10)
```

Definition at line 84 of file [portsignals.h](#).

4.110.2.13 SIGINT

```
#define SIGINT (NSIG+11)
```

Definition at line 87 of file [portsignals.h](#).

4.110.2.14 SIGQUIT

```
#define SIGQUIT (NSIG+12)
```

Definition at line 90 of file [portsignals.h](#).

4.110.2.15 SIGHUP

```
#define SIGHUP (NSIG+13)
```

Definition at line 93 of file [portsignals.h](#).

4.110.2.16 SIGALRM

```
#define SIGALRM (NSIG+14)
```

Definition at line 96 of file [portsignals.h](#).

4.110.2.17 SIGVTALRM

```
#define SIGVTALRM (NSIG+15)
```

Definition at line 99 of file [portsignals.h](#).

4.110.2.18 SIGPROF

```
#define SIGPROF (NSIG+16)
```

Definition at line 102 of file [portsignals.h](#).

4.111 portsignals.h

[Go to the documentation of this file.](#)

```
00001 #ifndef PORTSIGNAL_H
00002 #define PORTSIGNAL_H
00003
00018 /* #[] License : */
00019 /*
00020  * Copyright (C) 1984-2023 J.A.M. Vermaseren
00021  * When using this file you are requested to refer to the publication
00022  * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00023  * This is considered a matter of courtesy as the development was paid
00024  * for by FOM the Dutch physics granting agency and we would like to
00025  * be able to track its scientific use to convince FOM of its value
00026  * for the community.
00027  *
00028  * This file is part of FORM.
00029  *
00030  * FORM is free software: you can redistribute it and/or modify it under the
00031  * terms of the GNU General Public License as published by the Free Software
00032  * Foundation, either version 3 of the License, or (at your option) any later
00033  * version.
00034  *
00035  * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00036  * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00037  * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00038  * details.
00039  *
00040  * You should have received a copy of the GNU General Public License along
00041  * with FORM. If not, see <http://www.gnu.org/licenses/>.
00042  */
00043 /* #[] License : */
00044
00045 #include <signal.h>
00046
00047 #define FATAL_SIG_ERROR 4
00048
00049 #ifndef NSIG
00050 /*
00051  * The value of NSIG must be enough to fall outside the range of defined signals
00052  */
00053 #define NSIG (1024)
00054 #endif
00055
00056 #ifndef SIGSEGV
00057 #define SIGSEGV (NSIG+1)
00058 #endif
00059 #ifndef SIGFPE
00060 #define SIGFPE (NSIG+2)
00061 #endif
00062 #ifndef SIGILL
00063 #define SIGILL (NSIG+3)
00064 #endif
00065 #ifndef SIGEMT
00066 #define SIGEMT (NSIG+4)
00067 #endif
00068 #ifndef SIGSYS
00069 #define SIGSYS (NSIG+5)
00070 #endif
00071 #ifndef SIGPIPE
00072 #define SIGPIPE (NSIG+6)
00073 #endif
00074 #ifndef SIGLOST
00075 #define SIGLOST (NSIG+7)
00076 #endif
00077 #ifndef SIGXCPU
00078 #define SIGXCPU (NSIG+8)
00079 #endif
00080 #ifndef SIGXFSZ
00081 #define SIGXFSZ (NSIG+9)
00082 #endif
00083 #ifndef SIGTERM
00084 #define SIGTERM (NSIG+10)
00085 #endif
00086 #ifndef SIGINT
00087 #define SIGINT (NSIG+11)
00088 #endif
00089 #ifndef SIGQUIT
00090 #define SIGQUIT (NSIG+12)
00091 #endif
00092 #ifndef SIGHUP
00093 #define SIGHUP (NSIG+13)
00094 #endif
00095 #ifndef SIGALRM
00096 #define SIGALRM (NSIG+14)
```

```

00097 #endif
00098 #ifndef SIGVTALRM
00099 #define SIGVTALRM (NSIG+15)
00100 #endif
00101 #ifndef SIGPROF
00102 #define SIGPROF (NSIG+16)
00103 #endif
00104
00105 #endif

```

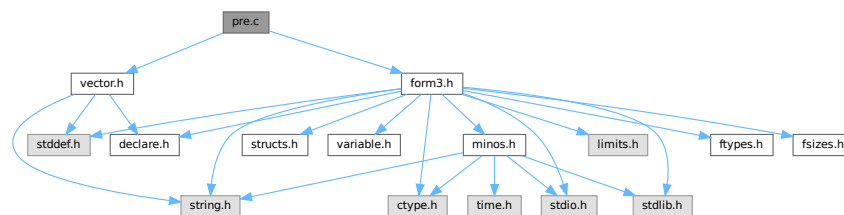
4.112 pre.c File Reference

```

#include "form3.h"
#include "vector.h"

```

Include dependency graph for pre.c:



Macros

- #define [STRINGIFY](#)(x) [STRINGIFY__](#)(x)
- #define [STRINGIFY__](#)(x) #x
- #define [SKIPBUFSIZE](#) 20
- #define [KILL](#) "kill"
- #define [KILLALL](#) "killall"
- #define [DAEMON](#) "daemon"
- #define [SHELL](#) "shell"
- #define [STDERR](#) "stderr"
- #define [TRUE_EXPR](#) "true"
- #define [FALSE_EXPR](#) "false"
- #define [NOSHELL](#) "noshell"
- #define [TERMINAL](#) "terminal"

Functions

- UBYTE [GetInput](#) (void)
- void [ClearPushback](#) (void)
- UBYTE [GetChar](#) (int level)
- void [CharOut](#) (UBYTE c)
- void [UnsetAllowDelay](#) (void)
- UBYTE * [GetPreVar](#) (UBYTE *name, int flag)
- int [PutPreVar](#) (UBYTE *name, UBYTE *value, UBYTE *args, int mode)
- void [PopPreVars](#) (int tonumber)
- void [IniModule](#) (int type)
- void [IniSpecialModule](#) (int type)

- void [PreProcessor](#) (void)
- int [PreProInstruction](#) (void)
- int [LoadInstruction](#) (int mode)
- int [LoadStatement](#) (int type)
- int [ExpandTripleDots](#) (int par)
- [KEYWORD](#) * [FindKeyWord](#) (UBYTE *theword, [KEYWORD](#) *table, int size)
- [KEYWORD](#) * [FindInKeyWord](#) (UBYTE *theword, [KEYWORD](#) *table, int size)
- int [TheDefine](#) (UBYTE *s, int mode)
- int [DoCommentChar](#) (UBYTE *s)
- int [DoPreAssign](#) (UBYTE *s)
- int [DoDefine](#) (UBYTE *s)
- int [DoRedefine](#) (UBYTE *s)
- int [ClearMacro](#) (UBYTE *name)
- int [TheUndefine](#) (UBYTE *name)
- int [DoUndefine](#) (UBYTE *s)
- int [DoInclude](#) (UBYTE *s)
- int [DoReverseInclude](#) (UBYTE *s)
- int [Include](#) (UBYTE *s, int type)
- int [DoPreExchange](#) (UBYTE *s)
- int [DoCall](#) (UBYTE *s)
- int [DoDebug](#) (UBYTE *s)
- int [DoTerminate](#) (UBYTE *s)
- int [DoContinueDo](#) (UBYTE *s)
- int [DoDo](#) (UBYTE *s)
- int [DoBreakDo](#) (UBYTE *s)
- int [DoElse](#) (UBYTE *s)
- int [DoElseif](#) (UBYTE *s)
- int [DoEnddo](#) (UBYTE *s)
- int [DoEndif](#) (UBYTE *s)
- int [DoEndprocedure](#) (UBYTE *s)
- int [Dof](#) (UBYTE *s)
- int [Dofldef](#) (UBYTE *s, int par)
- int [Dofydef](#) (UBYTE *s)
- int [Dofndef](#) (UBYTE *s)
- int [DoInside](#) (UBYTE *s)
- int [DoEndInside](#) (UBYTE *s)
- int [DoMessage](#) (UBYTE *s)
- int [DoPipe](#) (UBYTE *s)
- int [DoPrcExtension](#) (UBYTE *s)
- int [DoPreOut](#) (UBYTE *s)
- int [DoPrePrintTimes](#) (UBYTE *s)
- int [DoPreSortReallocate](#) (UBYTE *s)
- int [DoPreAppend](#) (UBYTE *s)
- int [DoPreCreate](#) (UBYTE *s)
- int [DoPreRemove](#) (UBYTE *s)
- int [DoPreClose](#) (UBYTE *s)
- int [DoPreWrite](#) (UBYTE *s)
- int [DoProcedure](#) (UBYTE *s)
- int [DoPreBreak](#) (UBYTE *s)
- int [DoPreCase](#) (UBYTE *s)
- int [DoPreDefault](#) (UBYTE *s)
- int [DoPreEndSwitch](#) (UBYTE *s)
- int [DoPreSwitch](#) (UBYTE *s)
- int [DoPreShow](#) (UBYTE *s)
- int [DoSystem](#) (UBYTE *s)

- int [PreLoad](#) ([PRELOAD](#) *p, UBYTE *start, UBYTE *stop, int mode, char *message)
- int [PreSkip](#) (UBYTE *start, UBYTE *stop, int mode)
- void [StartPrepro](#) (void)
- int [EvalPrelf](#) (UBYTE *s)
- UBYTE * [PrelfEval](#) (UBYTE *s, int *value)
- int [PreCmp](#) (int type, int val, UBYTE *t, int type2, int val2, UBYTE *t2, int cmpop)
- int [PreEq](#) (int type, int val, UBYTE *t, int type2, int val2, UBYTE *t2, int eqop)
- UBYTE * [pParseObject](#) (UBYTE *s, int *type, LONG *val2)
- UBYTE * [PreCalc](#) (void)
- UBYTE * [PreEval](#) (UBYTE *s, LONG *x)
- void [AddToPreTypes](#) (int type)
- void [MessPreNesting](#) (int par)
- int [DoPreAddSeparator](#) (UBYTE *s)
- int [DoPreRmSeparator](#) (UBYTE *s)
- int [DoExternal](#) (UBYTE *s)
- int [DoPrompt](#) (UBYTE *s)
- int [DoSetExternal](#) (UBYTE *s)
- int [DoSetExternalAttr](#) (UBYTE *s)
- int [DoRmExternal](#) (UBYTE *s)
- int [DoFromExternal](#) (UBYTE *s)
- int [DoToExternal](#) (UBYTE *s)
- UBYTE * [defineChannel](#) (UBYTE *s, [HANDLERS](#) *h)
- int [writeToChannel](#) (int wtype, UBYTE *s, [HANDLERS](#) *h)
- int [DoFactDollar](#) (UBYTE *s)
- WORD [GetDollarNumber](#) (UBYTE **inp, [DOLLARS](#) d)
- int [DoSetRandom](#) (UBYTE *s)
- int [DoOptimize](#) (UBYTE *s)
- int [DoClearOptimize](#) (UBYTE *s)
- int [DoSkipExtraSymbols](#) (UBYTE *s)
- int [DoPreReset](#) (UBYTE *s)
- int [DoPreAppendPath](#) (UBYTE *s)
- int [DoPrePrependPath](#) (UBYTE *s)
- int [DoTimeOutAfter](#) (UBYTE *s)
- int [DoNamespace](#) (UBYTE *s)
- int [DoEndNamespace](#) (UBYTE *s)
- UBYTE * [SkipName](#) (UBYTE *s)
- UBYTE * [ConstructName](#) (UBYTE *s, UBYTE type)
- int [DoUse](#) (UBYTE *s)
- int [UserFlags](#) (UBYTE *s, int par)
- int [DoClearUserFlag](#) (UBYTE *s)
- int [DoSetUserFlag](#) (UBYTE *s)

4.112.1 Detailed Description

This is the preprocessor and all its routines.

Definition in file [pre.c](#).

4.112.2 Macro Definition Documentation

4.112.2.1 STRINGIFY

```
#define STRINGIFY(  
    x ) STRINGIFY__(x)
```

Definition at line 4251 of file [pre.c](#).

4.112.2.2 STRINGIFY__

```
#define STRINGIFY__(  
    x ) #x
```

Definition at line 4252 of file [pre.c](#).

4.112.2.3 SKIPBUFSIZE

```
#define SKIPBUFSIZE 20
```

Definition at line 4375 of file [pre.c](#).

4.112.2.4 KILL

```
#define KILL "kill"
```

Definition at line 5595 of file [pre.c](#).

4.112.2.5 KILLALL

```
#define KILLALL "killall"
```

Definition at line 5596 of file [pre.c](#).

4.112.2.6 DAEMON

```
#define DAEMON "daemon"
```

Definition at line 5597 of file [pre.c](#).

4.112.2.7 SHELL

```
#define SHELL "shell"
```

Definition at line 5598 of file [pre.c](#).

4.112.2.8 STDERR

```
#define STDERR "stderr"
```

Definition at line [5599](#) of file [pre.c](#).

4.112.2.9 TRUE_EXPR

```
#define TRUE_EXPR "true"
```

Definition at line [5601](#) of file [pre.c](#).

4.112.2.10 FALSE_EXPR

```
#define FALSE_EXPR "false"
```

Definition at line [5602](#) of file [pre.c](#).

4.112.2.11 NOSHELL

```
#define NOSHELL "noshell"
```

Definition at line [5603](#) of file [pre.c](#).

4.112.2.12 TERMINAL

```
#define TERMINAL "terminal"
```

Definition at line [5604](#) of file [pre.c](#).

4.112.3 Function Documentation

4.112.3.1 GetInput()

```
UBYTE GetInput (  
    void )
```

Definition at line [132](#) of file [pre.c](#).

4.112.3.2 ClearPushback()

```
void ClearPushback (  
    void )
```

Definition at line [161](#) of file [pre.c](#).

4.112.3.3 GetChar()

```
UBYTE GetChar (
    int level )
```

Definition at line 179 of file [pre.c](#).

4.112.3.4 CharOut()

```
void CharOut (
    UBYTE c )
```

Definition at line 489 of file [pre.c](#).

4.112.3.5 UnsetAllowDelay()

```
void UnsetAllowDelay (
    void )
```

Definition at line 510 of file [pre.c](#).

4.112.3.6 GetPreVar()

```
UBYTE * GetPreVar (
    UBYTE * name,
    int flag )
```

Definition at line 536 of file [pre.c](#).

4.112.3.7 PutPreVar()

```
int PutPreVar (
    UBYTE * name,
    UBYTE * value,
    UBYTE * args,
    int mode )
```

Inserts/Updates a preprocessor variable in the name administration.

Parameters

<i>name</i>	Character string with the variable name.
<i>value</i>	Character string with a possible value. Special case: if this argument is zero, then we have no value. Note: This is different from having an empty argument! This should only occur when the name starts with a ?
<i>args</i>	Character string with possible arguments.
<i>mode</i>	=0: always create a new name entry, =1: try to do a redefinition if possible.

Returns

Index of used entry in name list.

Definition at line 718 of file [pre.c](#).

Referenced by [ClearOptimize\(\)](#), [Generator\(\)](#), [Optimize\(\)](#), [PF_BroadcastRedefinedPreVars\(\)](#), [StartVariables\(\)](#), and [TheDefine\(\)](#).

4.112.3.8 PopPreVars()

```
void PopPreVars (
    int tonumber )
```

Definition at line 820 of file [pre.c](#).

4.112.3.9 IniModule()

```
void IniModule (
    int type )
```

Definition at line 835 of file [pre.c](#).

4.112.3.10 IniSpecialModule()

```
void IniSpecialModule (
    int type )
```

Definition at line 937 of file [pre.c](#).

4.112.3.11 PreProcessor()

```
void PreProcessor (
    void )
```

Definition at line 947 of file [pre.c](#).

4.112.3.12 PreProInstruction()

```
int PreProInstruction (
    void )
```

Definition at line 1166 of file [pre.c](#).

4.112.3.13 LoadInstruction()

```
int LoadInstruction (
    int mode )
```

Definition at line 1254 of file [pre.c](#).

4.112.3.14 LoadStatement()

```
int LoadStatement (
    int type )
```

Definition at line 1457 of file [pre.c](#).

4.112.3.15 ExpandTripleDots()

```
int ExpandTripleDots (
    int par )
```

Definition at line 1595 of file [pre.c](#).

4.112.3.16 FindKeyWord()

```
KEYWORD * FindKeyWord (
    UBYTE * theword,
    KEYWORD * table,
    int size )
```

Definition at line 1963 of file [pre.c](#).

4.112.3.17 FindInKeyWord()

```
KEYWORD * FindInKeyWord (
    UBYTE * theword,
    KEYWORD * table,
    int size )
```

Definition at line 1994 of file [pre.c](#).

4.112.3.18 TheDefine()

```
int TheDefine (
    UBYTE * s,
    int mode )
```

Preprocessor assignment. Possible arguments and values are treated and the new preprocessor variable is put into the name administration.

Parameters

<i>s</i>	Pointer to the character string following the preprocessor command.
<i>mode</i>	Bitmask. 0-bit clear: always create a new name entry, 0-bit set: try to redefine an existing name, 1-bit set: ignore preprocessor if/switch status.

Returns

zero: no errors, negative number: errors.

Definition at line [2024](#) of file [pre.c](#).

References [PutPreVar\(\)](#).

Here is the call graph for this function:

**4.112.3.19 DoCommentChar()**

```
int DoCommentChar (
    UBYTE * s )
```

Definition at line [2097](#) of file [pre.c](#).

4.112.3.20 DoPreAssign()

```
int DoPreAssign (
    UBYTE * s )
```

Definition at line [2128](#) of file [pre.c](#).

4.112.3.21 DoDefine()

```
int DoDefine (
    UBYTE * s )
```

Definition at line [2159](#) of file [pre.c](#).

4.112.3.22 DoRedefine()

```
int DoRedefine (
    UBYTE * s )
```

Definition at line [2169](#) of file [pre.c](#).

4.112.3.23 ClearMacro()

```
int ClearMacro (
    UBYTE * name )
```

Definition at line 2181 of file [pre.c](#).

4.112.3.24 TheUndefine()

```
int TheUndefine (
    UBYTE * name )
```

Definition at line 2209 of file [pre.c](#).

4.112.3.25 DoUndefine()

```
int DoUndefine (
    UBYTE * s )
```

Definition at line 2263 of file [pre.c](#).

4.112.3.26 DoInclude()

```
int DoInclude (
    UBYTE * s )
```

Definition at line 2315 of file [pre.c](#).

4.112.3.27 DoReverseInclude()

```
int DoReverseInclude (
    UBYTE * s )
```

Definition at line 2322 of file [pre.c](#).

4.112.3.28 Include()

```
int Include (
    UBYTE * s,
    int type )
```

Definition at line 2329 of file [pre.c](#).

4.112.3.29 DoPreExchange()

```
int DoPreExchange (
    UBYTE * s )
```

Definition at line 2490 of file [pre.c](#).

4.112.3.30 DoCall()

```
int DoCall (
    UBYTE * s )
```

Definition at line [2551](#) of file [pre.c](#).

4.112.3.31 DoDebug()

```
int DoDebug (
    UBYTE * s )
```

Definition at line [2720](#) of file [pre.c](#).

4.112.3.32 DoTerminate()

```
int DoTerminate (
    UBYTE * s )
```

Definition at line [2757](#) of file [pre.c](#).

4.112.3.33 DoContinueDo()

```
int DoContinueDo (
    UBYTE * s )
```

Definition at line [2780](#) of file [pre.c](#).

4.112.3.34 DoDo()

```
int DoDo (
    UBYTE * s )
```

Definition at line [2811](#) of file [pre.c](#).

4.112.3.35 DoBreakDo()

```
int DoBreakDo (
    UBYTE * s )
```

Definition at line [3026](#) of file [pre.c](#).

4.112.3.36 DoElse()

```
int DoElse (
    UBYTE * s )
```

Definition at line [3095](#) of file [pre.c](#).

4.112.3.37 DoElseif()

```
int DoElseif (
    UBYTE * s )
```

Definition at line 3132 of file [pre.c](#).

4.112.3.38 DoEnddo()

```
int DoEnddo (
    UBYTE * s )
```

Definition at line 3167 of file [pre.c](#).

4.112.3.39 DoEndif()

```
int DoEndif (
    UBYTE * s )
```

Definition at line 3313 of file [pre.c](#).

4.112.3.40 DoEndprocedure()

```
int DoEndprocedure (
    UBYTE * s )
```

Definition at line 3341 of file [pre.c](#).

4.112.3.41 Dolf()

```
int DoIf (
    UBYTE * s )
```

Definition at line 3367 of file [pre.c](#).

4.112.3.42 Dolfdef()

```
int DoIfdef (
    UBYTE * s,
    int par )
```

Definition at line 3391 of file [pre.c](#).

4.112.3.43 Dolfydef()

```
int DoIfydef (
    UBYTE * s )
```

Definition at line 3416 of file [pre.c](#).

4.112.3.44 DoIfndef()

```
int DoIfndef (
    UBYTE * s )
```

Definition at line [3426](#) of file [pre.c](#).

4.112.3.45 DoInside()

```
int DoInside (
    UBYTE * s )
```

Definition at line [3451](#) of file [pre.c](#).

4.112.3.46 DoEndInside()

```
int DoEndInside (
    UBYTE * s )
```

Definition at line [3544](#) of file [pre.c](#).

4.112.3.47 DoMessage()

```
int DoMessage (
    UBYTE * s )
```

Definition at line [3676](#) of file [pre.c](#).

4.112.3.48 DoPipe()

```
int DoPipe (
    UBYTE * s )
```

Definition at line [3690](#) of file [pre.c](#).

4.112.3.49 DoPrcExtension()

```
int DoPrcExtension (
    UBYTE * s )
```

Definition at line [3713](#) of file [pre.c](#).

4.112.3.50 DoPreOut()

```
int DoPreOut (
    UBYTE * s )
```

Definition at line [3747](#) of file [pre.c](#).

4.112.3.51 DoPrePrintTimes()

```
int DoPrePrintTimes (
    UBYTE * s )
```

Definition at line [3770](#) of file [pre.c](#).

4.112.3.52 DoPreSortReallocate()

```
int DoPreSortReallocate (
    UBYTE * s )
```

Definition at line [3784](#) of file [pre.c](#).

4.112.3.53 DoPreAppend()

```
int DoPreAppend (
    UBYTE * s )
```

Definition at line [3804](#) of file [pre.c](#).

4.112.3.54 DoPreCreate()

```
int DoPreCreate (
    UBYTE * s )
```

Definition at line [3847](#) of file [pre.c](#).

4.112.3.55 DoPreRemove()

```
int DoPreRemove (
    UBYTE * s )
```

Definition at line [3887](#) of file [pre.c](#).

4.112.3.56 DoPreClose()

```
int DoPreClose (
    UBYTE * s )
```

Definition at line [3920](#) of file [pre.c](#).

4.112.3.57 DoPreWrite()

```
int DoPreWrite (
    UBYTE * s )
```

Definition at line [3964](#) of file [pre.c](#).

4.112.3.58 DoProcedure()

```
int DoProcedure (
    UBYTE * s )
```

Definition at line [4005](#) of file [pre.c](#).

4.112.3.59 DoPreBreak()

```
int DoPreBreak (
    UBYTE * s )
```

Definition at line [4047](#) of file [pre.c](#).

4.112.3.60 DoPreCase()

```
int DoPreCase (
    UBYTE * s )
```

Definition at line [4071](#) of file [pre.c](#).

4.112.3.61 DoPreDefault()

```
int DoPreDefault (
    UBYTE * s )
```

Definition at line [4110](#) of file [pre.c](#).

4.112.3.62 DoPreEndSwitch()

```
int DoPreEndSwitch (
    UBYTE * s )
```

Definition at line [4134](#) of file [pre.c](#).

4.112.3.63 DoPreSwitch()

```
int DoPreSwitch (
    UBYTE * s )
```

Definition at line [4161](#) of file [pre.c](#).

4.112.3.64 DoPreShow()

```
int DoPreShow (
    UBYTE * s )
```

Definition at line [4215](#) of file [pre.c](#).

4.112.3.65 DoSystem()

```
int DoSystem (
    UBYTE * s )
```

Definition at line [4254](#) of file [pre.c](#).

4.112.3.66 PreLoad()

```
int PreLoad (
    PRELOAD * p,
    UBYTE * start,
    UBYTE * stop,
    int mode,
    char * message )
```

Definition at line [4294](#) of file [pre.c](#).

4.112.3.67 PreSkip()

```
int PreSkip (
    UBYTE * start,
    UBYTE * stop,
    int mode )
```

Definition at line [4377](#) of file [pre.c](#).

4.112.3.68 StartPrepro()

```
void StartPrepro (
    void )
```

Definition at line [4432](#) of file [pre.c](#).

4.112.3.69 EvalPrelf()

```
int EvalPreIf (
    UBYTE * s )
```

Definition at line [4460](#) of file [pre.c](#).

4.112.3.70 PrelfEval()

```
UBYTE * PrelfEval (
    UBYTE * s,
    int * value )
```

Definition at line [4495](#) of file [pre.c](#).

4.112.3.71 PreCmp()

```
int PreCmp (
    int type,
    int val,
    UBYTE * t,
    int type2,
    int val2,
    UBYTE * t2,
    int cmpop )
```

Definition at line [4624](#) of file [pre.c](#).

4.112.3.72 PreEq()

```
int PreEq (
    int type,
    int val,
    UBYTE * t,
    int type2,
    int val2,
    UBYTE * t2,
    int eqop )
```

Definition at line [4648](#) of file [pre.c](#).

4.112.3.73 pParseObject()

```
UBYTE * pParseObject (
    UBYTE * s,
    int * type,
    LONG * val2 )
```

Definition at line [4677](#) of file [pre.c](#).

4.112.3.74 PreCalc()

```
UBYTE * PreCalc (
    void )
```

Definition at line [5113](#) of file [pre.c](#).

4.112.3.75 PreEval()

```
UBYTE * PreEval (
    UBYTE * s,
    LONG * x )
```

Definition at line [5191](#) of file [pre.c](#).

4.112.3.76 AddToPreTypes()

```
void AddToPreTypes (
    int type )
```

Definition at line [5341](#) of file [pre.c](#).

4.112.3.77 MessPreNesting()

```
void MessPreNesting (
    int par )
```

Definition at line [5359](#) of file [pre.c](#).

4.112.3.78 DoPreAddSeparator()

```
int DoPreAddSeparator (
    UBYTE * s )
```

Definition at line [5382](#) of file [pre.c](#).

4.112.3.79 DoPreRmSeparator()

```
int DoPreRmSeparator (
    UBYTE * s )
```

Definition at line [5407](#) of file [pre.c](#).

4.112.3.80 DoExternal()

```
int DoExternal (
    UBYTE * s )
```

Definition at line [5424](#) of file [pre.c](#).

4.112.3.81 DoPrompt()

```
int DoPrompt (
    UBYTE * s )
```

Definition at line [5506](#) of file [pre.c](#).

4.112.3.82 DoSetExternal()

```
int DoSetExternal (
    UBYTE * s )
```

Definition at line [5539](#) of file [pre.c](#).

4.112.3.83 DoSetExternalAttr()

```
int DoSetExternalAttr (
    UBYTE * s )
```

Definition at line 5609 of file [pre.c](#).

4.112.3.84 DoRmExternal()

```
int DoRmExternal (
    UBYTE * s )
```

Definition at line 5726 of file [pre.c](#).

4.112.3.85 DoFromExternal()

```
int DoFromExternal (
    UBYTE * s )
```

Definition at line 5791 of file [pre.c](#).

4.112.3.86 DoToExternal()

```
int DoToExternal (
    UBYTE * s )
```

Definition at line 5986 of file [pre.c](#).

4.112.3.87 defineChannel()

```
UBYTE * defineChannel (
    UBYTE * s,
    HANDLERS * h )
```

Definition at line 6033 of file [pre.c](#).

4.112.3.88 writeToChannel()

```
int writeToChannel (
    int wtype,
    UBYTE * s,
    HANDLERS * h )
```

Definition at line 6068 of file [pre.c](#).

4.112.3.89 DoFactDollar()

```
int DoFactDollar (
    UBYTE * s )
```

Definition at line 6633 of file [pre.c](#).

4.112.3.90 GetDollarNumber()

```
WORD GetDollarNumber (
    UBYTE ** inp,
    DOLLARS d )
```

Definition at line 6670 of file [pre.c](#).

4.112.3.91 DoSetRandom()

```
int DoSetRandom (
    UBYTE * s )
```

Definition at line 6760 of file [pre.c](#).

4.112.3.92 DoOptimize()

```
int DoOptimize (
    UBYTE * s )
```

Definition at line 6803 of file [pre.c](#).

4.112.3.93 DoClearOptimize()

```
int DoClearOptimize (
    UBYTE * s )
```

Definition at line 6936 of file [pre.c](#).

4.112.3.94 DoSkipExtraSymbols()

```
int DoSkipExtraSymbols (
    UBYTE * s )
```

Definition at line 6956 of file [pre.c](#).

4.112.3.95 DoPreReset()

```
int DoPreReset (
    UBYTE * s )
```

Definition at line 6995 of file [pre.c](#).

4.112.3.96 DoPreAppendPath()

```
int DoPreAppendPath (
    UBYTE * s )
```

Appends the given path (absolute or relative to the current file directory) to the FORM path.

Syntax: #appendpath <path>

Definition at line 7153 of file [pre.c](#).

4.112.3.97 DoPrePrependPath()

```
int DoPrePrependPath (
    UBYTE * s )
```

Prepends the given path (absolute or relative to the current file directory) to the FORM path.

Syntax: #prependpath <path>

Definition at line 7170 of file [pre.c](#).

4.112.3.98 DoTimeOutAfter()

```
int DoTimeOutAfter (
    UBYTE * s )
```

Definition at line 7182 of file [pre.c](#).

4.112.3.99 DoNamespace()

```
int DoNamespace (
    UBYTE * s )
```

Definition at line 7234 of file [pre.c](#).

4.112.3.100 DoEndNamespace()

```
int DoEndNamespace (
    UBYTE * s )
```

Definition at line 7282 of file [pre.c](#).

4.112.3.101 SkipName()

```
UBYTE * SkipName (
    UBYTE * s )
```

Definition at line 7305 of file [pre.c](#).

4.112.3.102 ConstructName()

```
UBYTE * ConstructName (
    UBYTE * s,
    UBYTE type )
```

Definition at line [7405](#) of file [pre.c](#).

4.112.3.103 DoUse()

```
int DoUse (
    UBYTE * s )
```

Definition at line [7497](#) of file [pre.c](#).

4.112.3.104 UserFlags()

```
int UserFlags (
    UBYTE * s,
    int par )
```

Definition at line [7545](#) of file [pre.c](#).

4.112.3.105 DoClearUserFlag()

```
int DoClearUserFlag (
    UBYTE * s )
```

Definition at line [7652](#) of file [pre.c](#).

4.112.3.106 DoSetUserFlag()

```
int DoSetUserFlag (
    UBYTE * s )
```

Definition at line [7662](#) of file [pre.c](#).

4.113 pre.c

[Go to the documentation of this file.](#)

```

00001
00005 /* #[ License : */
00006 /*
00007 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00008 *   When using this file you are requested to refer to the publication
00009 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00010 *   This is considered a matter of courtesy as the development was paid
00011 *   for by FOM the Dutch physics granting agency and we would like to
00012 *   be able to track its scientific use to convince FOM of its value
00013 *   for the community.
00014 *
00015 *   This file is part of FORM.
00016 *
00017 *   FORM is free software: you can redistribute it and/or modify it under the
00018 *   terms of the GNU General Public License as published by the Free Software
00019 *   Foundation, either version 3 of the License, or (at your option) any later
00020 *   version.
00021 *
00022 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00023 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00024 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00025 *   details.
00026 *
00027 *   You should have received a copy of the GNU General Public License along
00028 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00029 */
00030 /* #[ License : */
00031 /*
00032     #[ Includes :
00033 */
00034 #include "form3.h"
00035
00036 static UBYTE pushbackchar = 0;
00037 static int oldmode = 0;
00038 static int stopdelay = 0;
00039 static STREAM *oldstream = 0;
00040 static UBYTE underscore[2] = {'_', 0};
00041 static PREVAR *ThePreVar = 0;
00042
00043 static KEYWORD precommands[] = {
00044     {"add" , DoPreAdd , 0, 0}
00045     , {"addseparator" , DoPreAddSeparator, 0, 0}
00046     , {"append" , DoPreAppend , 0, 0}
00047     , {"appendpath" , DoPreAppendPath, 0, 0}
00048     , {"assign" , DoPreAssign , 0, 0}
00049     , {"break" , DoPreBreak , 0, 0}
00050     , {"breakdo" , DoBreakDo , 0, 0}
00051     , {"call" , DoCall , 0, 0}
00052     , {"case" , DoPreCase , 0, 0}
00053     , {"clearflag" , DoClearUserFlag, 0, 0}
00054     , {"clearoptimize", DoClearOptimize, 0, 0}
00055     , {"close" , DoPreClose , 0, 0}
00056     , {"closedictionary", DoPreCloseDictionary, 0, 0}
00057     , {"commentchar" , DoCommentChar , 0, 0}
00058     , {"continuedo" , DoContinueDo , 0, 0}
00059     , {"create" , DoPreCreate , 0, 0}
00060     , {"debug" , DoDebug , 0, 0}
00061     , {"default" , DoPreDefault , 0, 0}
00062     , {"define" , DoDefine , 0, 0}
00063     , {"do" , DoDo , 0, 0}
00064     , {"else" , DoElse , 0, 0}
00065     , {"elseif" , DoElseif , 0, 0}
00066     , {"enddo" , DoEnddo , 0, 0}
00067 #ifdef WITHFLOAT
00068     , {"endfloat" , DoEndFloat , 0, 0}
00069 #endif
00070     , {"endif" , DoEndif , 0, 0}
00071     , {"endinside" , DoEndInside , 0, 0}
00072     , {"endnamespace" , DoEndNamespace , 0, 0}
00073     , {"endprocedure" , DoEndprocedure , 0, 0}
00074     , {"endswitch" , DoPreEndSwitch , 0, 0}
00075     , {"exchange" , DoPreExchange , 0, 0}
00076     , {"external" , DoExternal , 0, 0}
00077     , {"factdollar" , DoFactDollar , 0, 0}
00078     , {"fromexternal" , DoFromExternal , 0, 0}
00079     , {"if" , DoIf , 0, 0}
00080     , {"ifdef" , DoIfydef , 0, 0}
00081     , {"ifndef" , DoIfndef , 0, 0}
00082     , {"include" , DoInclude , 0, 0}
00083     , {"inside" , DoInside , 0, 0}
00084     , {"message" , DoMessage , 0, 0}
00085     , {"namespace" , DoNamespace , 0, 0}

```

```

00086     ,{"opendictionary", DoPreOpenDictionary,0,0}
00087     ,{"optimize"       , DoOptimize       , 0, 0}
00088     ,{"pipe"           , DoPipe           , 0, 0}
00089     ,{"preout"         , DoPreOut         , 0, 0}
00090     ,{"prependpath"    , DoPrePrependPath,0, 0}
00091     ,{"printtimes"     , DoPrePrintTimes, 0, 0}
00092     ,{"procedure"      , DoProcedure      , 0, 0}
00093     ,{"procedureextension", DoPrCExtension , 0, 0}
00094     ,{"prompt"         , DoPrompt         , 0, 0}
00095     ,{"redefine"       , DoRedefine       , 0, 0}
00096     ,{"remove"         , DoPreRemove      , 0, 0}
00097     ,{"reset"          , DoPreReset       , 0, 0}
00098     ,{"reverseinclude" , DoReverseInclude , 0, 0}
00099     ,{"rmexternal"     , DoRmExternal     , 0, 0}
00100     ,{"rmseparator"    , DoPreRmSeparator,0, 0}
00101     ,{"setexternal"    , DoSetExternal    , 0, 0}
00102     ,{"setexternalattr", DoSetExternalAttr , 0, 0}
00103     ,{"setflag"        , DoSetUserFlag    , 0, 0}
00104     ,{"setrandom"      , DoSetRandom      , 0, 0}
00105     ,{"show"           , DoPreShow        , 0, 0}
00106     ,{"skipextrasymbols", DoSkipExtraSymbols, 0, 0}
00107     ,{"sortreallocate", DoPreSortReallocate, 0, 0}
00108 #ifdef WITHFLOAT
00109     ,{"startfloat"     , DoStartFloat     , 0, 0}
00110 #endif
00111     ,{"switch"         , DoPreSwitch      , 0, 0}
00112     ,{"system"         , DoSystem         , 0, 0}
00113     ,{"terminate"      , DoTerminate      , 0, 0}
00114     ,{"timeoutafter"   , DoTimeOutAfter   , 0, 0}
00115     ,{"toexternal"     , DoToExternal     , 0, 0}
00116     ,{"undefine"       , DoUndefine       , 0, 0}
00117     ,{"use"            , DoUse            , 0, 0}
00118     ,{"usedictionary", DoPreUseDictionary,0,0}
00119     ,{"write"          , DoPreWrite       , 0, 0}
00120 };
00121
00122 /*
00123     #] Includes :
00124     # [ PreProcessor :
00125         #[ GetInput :
00126
00127         Gets one input character. If we reach the end of a stream
00128         we pop to the previous stream and try again.
00129         If there are no more streams we let this be known.
00130     */
00131
00132 UBYTE GetInput(void)
00133 {
00134     UBYTE c;
00135     while ( AC.CurrentStream ) {
00136         c = GetFromStream(AC.CurrentStream);
00137         if ( c != ENDOFSTREAM ) {
00138             #ifdef WITHMPI
00139                 if ( PF.me == MASTER
00140                     && AC.NoShowInput <= 0
00141                     && AC.CurrentStream->type != PREVARSTREAM )
00142             #else
00143                 if ( AC.NoShowInput <= 0 && AC.CurrentStream->type != PREVARSTREAM )
00144             #endif
00145                 CharOut(c);
00146                 return(c);
00147             }
00148             AC.CurrentStream = CloseStream(AC.CurrentStream);
00149             if ( stopdelay && AC.CurrentStream == oldstream ) {
00150                 stopdelay = 0; AP.AllowDelay = 1;
00151             }
00152         }
00153         return(ENDOFINPUT);
00154     }
00155
00156 /*
00157     #] GetInput :
00158     #[ ClearPushback :
00159 */
00160
00161 void ClearPushback(void)
00162 {
00163     pushbackchar = 0;
00164 }
00165
00166 /*
00167     #] ClearPushback :
00168     #[ GetChar :
00169
00170     Reads one character. If it encounters a quote it immediately
00171     takes the whole preprocessor variable and opens a stream
00172     for it and starts reading the stream.

```

```

00173         Note that we have to take special precautions for escaped quotes.
00174         That is why we remember the previous character. We allow the
00175         (dubious?) construction of ending a stream with a backslash and
00176         then using it to escape an object in the parent stream.
00177     */
00178
00179     UBYTE GetChar(int level)
00180     {
00181         UBYTE namebuf[MAXPRENAMESIZE+2], c, *s, *t;
00182         static UBYTE lastchar, charinbuf = 0;
00183         int i, j, raiselow, olddelay;
00184         STREAM *stream;
00185         if ( level > 0 ) {
00186             lastchar = '';
00187             goto higherlevel;
00188         }
00189         if ( pushbackchar ) { c = pushbackchar; pushbackchar = 0; return(c); }
00190         if ( charinbuf ) { c = charinbuf; charinbuf = 0; return(c); }
00191         c = GetInput();
00192         for (;;) {
00193             if ( c == '\\\\' ) {
00194                 charinbuf = GetInput();
00195                 if ( charinbuf != LINEFEED ) {
00196                     pushbackchar = charinbuf;
00197                     charinbuf = 0;
00198                     break;
00199                 }
00200                 charinbuf = 0; /* Escaped linefeed -> skip leading blanks */
00201                 while ( ( c = GetInput() ) == ' ' || c == '\t' ) {}
00202             }
00203             else if ( c == '\"' || c == '\'' ) {
00204                 if ( AP.DelayPrevar == 1 && c == '\"' ) {
00205                     AP.DelayPrevar = 0;
00206                     break;
00207                 }
00208                 lastchar = c;
00209             higherlevel:
00210                 c = GetInput();
00211                 if ( c == '!' && lastchar == '\'' ) {
00212                     if ( stopdelay == 0 ) oldstream = AC.CurrentStream;
00213                     AP.AllowDelay = 0;
00214                     stopdelay = 1;
00215                     c = GetInput();
00216                 }
00217                 if ( c == '~' && lastchar == '\'' ) {
00218                     if ( AP.AllowDelay ) {
00219                         pushbackchar = c;
00220                         c = lastchar;
00221                         AP.DelayPrevar = 1;
00222                         break;
00223                     }
00224                 }
00225                 else {
00226                     pushbackchar = c;
00227                 }
00228                 olddelay = AP.DelayPrevar;
00229                 AP.DelayPrevar = 0;
00230                 i = 0; lastchar = 0;
00231                 for (;;) {
00232                     if ( pushbackchar ) { c = pushbackchar; pushbackchar = 0; }
00233                     else { c = GetInput(); }
00234                     if ( c == ENDOFINPUT || ( ( c == '\"' || c == LINEFEED )
00235                         && lastchar != '\\\\' ) ) {
00236                         break;
00237                     }
00238                     if ( c == '{' ) { /* Try the preprocessor calculator */
00239                         if ( PreCalc() == 0 ) Terminate(-1);
00240                         c = GetInput(); /* This is either a { or a number */
00241                         if ( c == '{' ) {
00242                             MesPrint("@Illegal set inside preprocessor variable name");
00243                             Terminate(-1);
00244                         }
00245                     }
00246                     if ( c == '\'' && lastchar != '\\\\' ) {
00247                         c = GetChar(1);
00248                         if ( c == ENDOFINPUT || ( ( c == '\"' || c == LINEFEED )
00249                             && lastchar != '\\\\' ) ) {
00250                             break;
00251                         }
00252                     }
00253                     if ( lastchar == '\\\\' ) { i--; lastchar = 0; }
00254                     else lastchar = c;
00255                     namebuf[i++] = c;
00256                     if ( i > MAXPRENAMESIZE ) {
00257                         namebuf[i] = 0;
00258                         Error1("Preprocessor variable name too long: ",namebuf);
00259                     }

```

```

00260     }
00261     namebuf[i++] = 0;
00262     if ( c != '"' ) {
00263         Error1("Unmatched quotes for preprocessor variable",namebuf);
00264     }
00265     AP.DelayPrevar = olddelay;
00266     if ( namebuf[0] == '$' ) {
00267         raiselow = PRENOACTION;
00268         if ( AP.PreproFlag && *AP.preStart ) {
00269             s = EndOfToken(AP.preStart);
00270             c = *s; *s = 0;
00271             if ( ( StrICmp(AP.preStart,(UBYTE *) "ifdef") == 0
00272                 || StrICmp(AP.preStart,(UBYTE *) "ifndef") == 0 )
00273                 && GetDollar(namebuf+1) < 0 ) {
00274                 *s = c; c = ' ';
00275                 break;
00276             }
00277             *s = c;
00278         }
00279         else {
00280             s = EndOfToken(namebuf+1);
00281             if ( *s == '[' ) { while ( *s ) s++; }
00282         }
00283         if ( *s == '-' && s[1] == '-' && s[2] == 0 )
00284             raiselow = PRELOWERAFTER;
00285         else if ( *s == '+' && s[1] == '+' && s[2] == 0 )
00286             raiselow = PRERAISEAFTER;
00287         c = *s; *s = 0;
00288         if ( OpenStream(namebuf+1,DOLLARSTREAM,0,raiselow) == 0 ) {
00289             *s = c;
00290             MesPrint("@Undefined variable %s used as preprocessor variable",
00291                     namebuf);
00292             Terminate(-1);
00293         }
00294         *s = c;
00295     }
00296     else {
00297         raiselow = PRENOACTION;
00298         if ( AP.PreproFlag && *AP.preStart ) {
00299             s = EndOfToken(AP.preStart);
00300             c = *s; *s = 0;
00301             if ( ( StrICmp(AP.preStart,(UBYTE *) "ifdef") == 0
00302                 || StrICmp(AP.preStart,(UBYTE *) "ifndef") == 0 )
00303                 && GetPreVar(namebuf,WITHOUTERROR) == 0 ) {
00304                 *s = c; c = ' ';
00305                 break;
00306             }
00307             *s = c;
00308         }
00309         s = EndOfToken(namebuf);
00310         if ( *s == '-' ) s++;
00311         if ( *s == '-' && s[1] == '-' && s[2] == 0 )
00312             raiselow = PRELOWERAFTER;
00313         else if ( *s == '+' && s[1] == '+' && s[2] == 0 )
00314             raiselow = PRERAISEAFTER;
00315         else if ( *s == '(' && namebuf[i-2] == ')' ) {
00316             /*
00317              Now count the arguments and separate them by zeroes
00318              Check on the ?var construction and if present, reset
00319              some comma's.
00320              Make the assignments of the variables
00321              Run the macro.
00322              Undefine the variables
00323             */
00324             int nargs = 1;
00325             PREVAR *p;
00326             size_t p_offset;
00327             *s++ = 0; namebuf[i-2] = 0;
00328             if ( StrICmp(namebuf,(UBYTE *) "random_") == 0 ) {
00329                 UBYTE *ranvalue;
00330                 ranvalue = PreRandom(s);
00331                 PutPreVar(namebuf,ranvalue,(UBYTE *) "?a",1);
00332                 M_free(ranvalue,"PreRandom");
00333                 goto dostream;
00334             }
00335             else if ( StrICmp(namebuf,(UBYTE *) "tolower_") == 0 ) {
00336                 UBYTE *ss = s;
00337                 while ( *ss ) { *ss = (UBYTE)(tolower(*ss)); ss++; }
00338                 PutPreVar(namebuf,s,(UBYTE *) "?a",1);
00339                 goto dostream;
00340             }
00341             else if ( StrICmp(namebuf,(UBYTE *) "toupper_") == 0 ) {
00342                 UBYTE *ss = s;
00343                 while ( *ss ) { *ss = (UBYTE)(toupper(*ss)); ss++; }
00344                 PutPreVar(namebuf,s,(UBYTE *) "?a",1);
00345                 goto dostream;
00346             }

```

```

00347     else if ( StrICmp(namebuf, (UBYTE *)"takeleft_") == 0 ) {
00348         UBYTE *ss = s;
00349         int x = 0, nsize;
00350         while ( *ss != ',' && *ss ) ss++;
00351         nsize = ss-s;
00352         if ( *ss ) {
00353             *ss++ = 0;
00354             while ( FG.cTable[*ss] == 1 ) x = 10*x + (*ss++ - '0');
00355             if ( x > nsize ) x = nsize;
00356         }
00357         else x = 0;
00358         PutPreVar(namebuf,s+x, (UBYTE *)"?a",1);
00359         goto dostream;
00360     }
00361     else if ( StrICmp(namebuf, (UBYTE *)"takeright_") == 0 ) {
00362         UBYTE *ss = s;
00363         int x = 0, nsize;
00364         while ( *ss != ',' && *ss ) ss++;
00365         nsize = ss-s;
00366         if ( *ss ) {
00367             *ss++ = 0;
00368             while ( FG.cTable[*ss] == 1 ) x = 10*x + (*ss++ - '0');
00369             if ( x > nsize ) x = nsize;
00370         }
00371         else x = 0;
00372         x = nsize - x;
00373         s[x] = 0;
00374         PutPreVar(namebuf,s, (UBYTE *)"?a",1);
00375         goto dostream;
00376     }
00377     else if ( StrICmp(namebuf, (UBYTE *)"keepleft_") == 0 ) {
00378         UBYTE *ss = s;
00379         int x = 0, nsize;
00380         while ( *ss != ',' && *ss ) ss++;
00381         nsize = ss-s;
00382         if ( *ss ) {
00383             *ss++ = 0;
00384             while ( FG.cTable[*ss] == 1 ) x = 10*x + (*ss++ - '0');
00385             if ( x > nsize ) x = nsize;
00386         }
00387         else x = nsize;
00388         s[x] = 0;
00389         PutPreVar(namebuf,s, (UBYTE *)"?a",1);
00390         goto dostream;
00391     }
00392     else if ( StrICmp(namebuf, (UBYTE *)"keppright_") == 0 ) {
00393         UBYTE *ss = s;
00394         int x = 0, nsize;
00395         while ( *ss != ',' && *ss ) ss++;
00396         nsize = ss-s;
00397         if ( *ss ) {
00398             *ss++ = 0;
00399             while ( FG.cTable[*ss] == 1 ) x = 10*x + (*ss++ - '0');
00400             if ( x > nsize ) x = nsize;
00401         }
00402         else x = nsize;
00403         x = nsize-x;
00404         PutPreVar(namebuf,s+x, (UBYTE *)"?a",1);
00405         goto dostream;
00406     }
00407     while ( *s ) {
00408         if ( *s == '\\\\' ) s++;
00409         if ( *s == ',' ) { *s = 0; nargs++; }
00410         s++;
00411     }
00412     GetPreVar(namebuf,WITHEXERROR);
00413     p = ThePreVar;
00414     if ( p == 0 ) {
00415         MesPrint("@Illegal use of arguments in preprocessor variable %s",namebuf);
00416         Terminate(-1);
00417     }
00418     if ( p->nargs <= 0 || ( p->wildarg == 0 && nargs != p->nargs )
00419         || ( p->wildarg > 0 && nargs < p->nargs-1 ) ) {
00420         MesPrint("@Arguments of macro %s do not match",namebuf);
00421         Terminate(-1);
00422     }
00423     if ( p->wildarg > 0 ) {
00424         /*
00425          *
00426          *
00427          *
00428          *
00429          *
00430          *
00431          *
00432          *
00433          *

```



```

00434             *s++ = ',';
00435         }
00436     }
00437 /*
00438         Now we can make the assignments
00439 */
00440     s = namebuf;
00441     while ( *s ) s++;
00442     s++;
00443     t = p->argnames;
00444     p_offset = p - PreVar;
00445     for ( j = 0; j < p->nargs; j++ ) {
00446         if ( ( nargs == p->nargs-1 ) && ( *t == '?' ) ) {
00447             PutPreVar(t,0,0,0);
00448         }
00449         else {
00450             PutPreVar(t,s,0,0);
00451             while ( *s ) s++;
00452             s++;
00453         }
00454         p = PreVar + p_offset;
00455         while ( *t ) t++;
00456         t++;
00457     }
00458 }
00459 dostream;;
00460     if ( ( stream = OpenStream(namebuf,PREVARSTREAM,0,raiselow) ) == 0 ) {
00461 /*
00462         Eat comma before or after. This is `no value'
00463 */
00464     }
00465     else if ( stream->inbuffer == 0 ) {
00466         c = GetInput();
00467         if ( level > 0 && c == '\\' ) return(c);
00468         goto endofloop;
00469     }
00470     }
00471     c = GetInput();
00472 }
00473 else if ( c == '{' ) { /* Try the preprocessor calculator */
00474     if ( PreCalc() == 0 ) Terminate(-1);
00475     c = GetInput(); /* This is either a { or a number */
00476     break;
00477 }
00478 else break;
00479 endofloop;;
00480 }
00481 return(c);
00482 }
00483
00484 /*
00485     #] GetChar :
00486     #[ CharOut :
00487 */
00488
00489 void CharOut(UBYTE c)
00490 {
00491     if ( c == LINEFEED ) {
00492         AM.OutBuffer[AP.InOutBuf++] = c;
00493         WriteString(INPUTOUT,AM.OutBuffer,AP.InOutBuf);
00494         AP.InOutBuf = 0;
00495     }
00496     else {
00497         if ( AP.InOutBuf >= AM.OutBufSize || c == LINEFEED ) {
00498             WriteString(INPUTOUT,AM.OutBuffer,AP.InOutBuf);
00499             AP.InOutBuf = 0;
00500         }
00501         AM.OutBuffer[AP.InOutBuf++] = c;
00502     }
00503 }
00504
00505 /*
00506     #] CharOut :
00507     #[ UnsetAllowDelay :
00508 */
00509
00510 void UnsetAllowDelay(void)
00511 {
00512     if ( ThePreVar != 0 ) {
00513         if ( ThePreVar->nargs > 0 ) AP.AllowDelay = 0;
00514     }
00515 }
00516
00517 /*
00518     #] UnsetAllowDelay :
00519     #[ GetPreVar :
00520

```

```

00521         We use the model of a heap. If the same name has been used more
00522         than once the last definition is used. This gives the impression
00523         of local variables.
00524
00525         There are two types: The regular ones and the expression variables.
00526         The last ones are like UNCHANGED_exprname and ZERO_exprname or
00527         UNCHANGED_ and ZERO_.
00528     */
00529
00530     static UBYTE *yes = (UBYTE *) "1";
00531     static UBYTE *no = (UBYTE *) "0";
00532     static UBYTE numintopolynomial[12];
00533     #include "vector.h"
00534     static Vector(UBYTE, exprstr); /* Used for numactiveexprs_ and activeexprnames_. */
00535
00536     UBYTE *GetPreVar(UBYTE *name, int flag)
00537     {
00538         GETIDENTITY
00539         int i, mode;
00540         WORD number;
00541         UBYTE *t, c = 0, *tt = 0;
00542         t = name; while ( *t ) t++;
00543         if ( t[-1] == '-' && t[-2] == '-' && t-2 > name && t[-3] != '_' ) {
00544             t -= 2; c = *t; *t = 0; tt = t;
00545         }
00546         else if ( t[-1] == '+' && t[-2] == '+' && t-2 > name && t[-3] != '_' ) {
00547             t -= 2; c = *t; *t = 0; tt = t;
00548         }
00549         else if ( StrICmp(name, (UBYTE *) "time_") == 0 ) {
00550             UBYTE millibuf[24];
00551             LONG millitime, timepart;
00552             int timepart1, timepart2;
00553             static char timestring[40];
00554             /* millitime = TimeCPU(1); */
00555             millitime = GetRunningTime();
00556             timepart = millitime%1000;
00557             millitime /= 1000;
00558             timepart /= 10;
00559             timepart1 = timepart / 10;
00560             timepart2 = timepart % 10;
00561             NumToStr(millibuf, millitime);
00562             snprintf(timestring, 40, "%s.%ld%ld", (GetRunningTime() - AP.StopWatchZero));
00563             return((UBYTE *) timestring);
00564         }
00565         else if ( ( StrICmp(name, (UBYTE *) "timer_") == 0 )
00566                 || ( StrICmp(name, (UBYTE *) "stopwatch_") == 0 ) ) {
00567             static char timestring[40];
00568             snprintf(timestring, 40, "%ld", (GetRunningTime() - AP.StopWatchZero));
00569             return((UBYTE *) timestring);
00570         }
00571         else if ( StrICmp(name, (UBYTE *) "numactiveexprs_") == 0 ) {
00572             /* the number of active expressions */
00573             int n = 0;
00574             for ( i = 0; i < NumExpressions; i++ ) {
00575                 EXPRESSIONS e = Expressions + i;
00576                 switch ( e->status ) {
00577                     case LOCALEXPRESSION:
00578                     case GLOBALEXPRESSION:
00579                     case UNHIDELEXPRESSION:
00580                     case UNHIDEGEXPRESSION:
00581                     case INTOHIDELEXPRESSION:
00582                     case INTOHIDEGEXPRESSION:
00583                         n++;
00584                         break;
00585                 }
00586             }
00587             VectorReserve(exprstr, 41); /* up to 128-bit */
00588             LongCopy(n, (char *) VectorPtr(exprstr));
00589             return VectorPtr(exprstr);
00590         }
00591         else if ( StrICmp(name, (UBYTE *) "activeexprnames_") == 0 ) {
00592             /* the list of active expressions separated by commas */
00593             int j = 0;
00594             VectorReserve(exprstr, 16); /* at least 1 character for '\0' */
00595             for ( i = 0; i < NumExpressions; i++ ) {
00596                 UBYTE *p, *s;
00597                 int len, k;
00598                 EXPRESSIONS e = Expressions + i;
00599                 switch ( e->status ) {
00600                     case LOCALEXPRESSION:
00601                     case GLOBALEXPRESSION:
00602                     case UNHIDELEXPRESSION:
00603                     case UNHIDEGEXPRESSION:
00604                     case INTOHIDELEXPRESSION:
00605                     case INTOHIDEGEXPRESSION:
00606                         s = AC.exprnames->namebuffer + e->name;
00607                         len = StrLen(s);

```

```

00608         VectorSize(exprstr) = j; /* j bytes must be copied in extending the buffer. */
00609         VectorReserve(exprstr, j + len * 2 + 1);
00610         p = VectorPtr(exprstr);
00611         if ( j > 0 ) p[j++] = ',';
00612         for ( k = 0; k < len; k++ ) {
00613             if ( s[k] == ',' || s[k] == '|' ) p[j++] = '\\';
00614             p[j++] = s[k];
00615         }
00616         break;
00617     }
00618 }
00619 VectorPtr(exprstr)[j] = '\0';
00620 return VectorPtr(exprstr);
00621 }
00622 else if ( StrICmp(name, (UBYTE *)"path_") == 0 ) {
00623     /* the current FORM path (for debugging both in .c and .frm) */
00624     if ( AM.Path ) {
00625         return(AM.Path);
00626     }
00627     else {
00628         return((UBYTE *)"");
00629     }
00630 }
00631 t = name;
00632 while ( *t && *t != '_' ) t++;
00633 for ( i = NumPre-1; i >= 0; i-- ) {
00634     if ( *t == '_' && ( StrICmp(name, PreVar[i].name) == 0 ) ) {
00635         if ( c ) *tt = c;
00636         ThePreVar = PreVar+i;
00637         return(PreVar[i].value);
00638     }
00639     else if ( StrCmp(name, PreVar[i].name) == 0 ) {
00640         if ( c ) *tt = c;
00641         ThePreVar = PreVar+i;
00642         return(PreVar[i].value);
00643     }
00644 }
00645 if ( *t == '_' ) {
00646     if ( StrICmp(name, (UBYTE *)"EXTRASYNBOLS_") == 0 ) goto extrashort;
00647     *t = 0;
00648     if ( StrICmp(name, (UBYTE *)"UNCHANGED") == 0 ) mode = 1;
00649     else if ( StrICmp(name, (UBYTE *)"ZERO") == 0 ) mode = 0;
00650     else if ( StrICmp(name, (UBYTE *)"SHOWINPUT") == 0 ) {
00651         *t++ = '_';
00652         if ( c ) *tt = c;
00653         if ( AC.NoShowInput > 0 ) return(no);
00654         else return(yes);
00655     }
00656     else if ( StrICmp(name, (UBYTE *)"EXTRASYNBOLS") == 0 ) {
00657         *t++ = '_';
00658 extrashort:;
00659         number = cbuf[AM.sbufnum].numrhs;
00660         t = numintopolynomial;
00661         NumCopy(number, t);
00662         return(numintopolynomial);
00663     }
00664     else mode = -1;
00665     *t++ = '_';
00666     if ( mode >= 0 ) {
00667         ThePreVar = 0;
00668         if ( *t ) {
00669             if ( GetName(AC.exprnames, t, &number, NOAUTO) == CEXPRESSION ) {
00670                 if ( c ) *tt = c;
00671                 if ( ( Expressions[number].vflags & ( 1 << mode ) ) != 0 )
00672                     return(yes);
00673                 else return(no);
00674             }
00675         }
00676         else {
00677             /*
00678              Here we have to test all active results.
00679              These are in 'negative' so the flags have to be zero.
00680             */
00681             if ( c ) *tt = c;
00682             if ( ( AR.expflags & ( 1 << mode ) ) == 0 ) return(yes);
00683             else return(no);
00684         }
00685     }
00686 }
00687 if ( ( t = (UBYTE *) (getenv((char *) (name))) ) != 0 ) {
00688     if ( c ) *tt = c;
00689     ThePreVar = 0;
00690     return(t);
00691 }
00692 if ( c ) *tt = c;
00693 if ( flag == WITHERROR ) {
00694     Error1("Undefined preprocessor variable", name);

```

```

00695     }
00696     return(0);
00697 }
00698
00699 /*
00700     #] GetPreVar :
00701     #[ PutPreVar :
00702 */
00703
00718 int PutPreVar(UBYTE *name, UBYTE *value, UBYTE *args, int mode)
00719 {
00720     int i, ii, num = 2, nnum = 2, numargs = 0;
00721     UBYTE *s, *t, *u = 0;
00722     PREVAR *p;
00723     if ( value == 0 && name[0] != '?' ) {
00724         MesPrint("@Illegal empty value for preprocessor variable %s",name);
00725         Terminate(-1);
00726     }
00727     if ( args ) {
00728         s = args; num++;
00729         while ( *s ) {
00730             if ( *s != ' ' && *s != '\t' ) num++;
00731             s++;
00732         }
00733     }
00734     if ( mode == 1 ) {
00735         i = NumPre;
00736         while ( --i >= 0 ) {
00737             if ( StrCmp(name,PreVar[i].name) == 0 ) {
00738                 u = PreVar[i].name;
00739                 break;
00740             }
00741         }
00742     }
00743     else i = -1;
00744     if ( i < 0 ) { p = (PREVAR *)FromList(&AP.PreVarList); ii = p - PreVar; }
00745     else { p = &(PreVar[i]); ii = i; }
00746     if ( value ) {
00747         s = value; while ( *s ) { s++; num++; }
00748     }
00749     else num = 1;
00750     if ( i >= 0 ) {
00751         if ( p->value ) {
00752             s = p->value;
00753             while ( *s ) { s++; nnum++; }
00754         }
00755         else nnum = 1;
00756         if ( nnum >= num ) {
00757             /*
00758                 We can keep this in place
00759             */
00760             if ( value && p->value ) {
00761                 s = value;
00762                 t = p->value;
00763                 while ( *s ) *t++ = *s++;
00764                 *t = 0;
00765             }
00766             else p->value = 0;
00767             return(i);
00768         }
00769     }
00770     s = name; while ( *s ) { s++; num++; }
00771     t = (UBYTE *)Malloc1(num,"PreVariable");
00772     p->name = t;
00773     s = name; while ( *s ) *t++ = *s++; *t++ = 0;
00774     if ( value ) {
00775         p->value = t;
00776         s = value; while ( *s ) *t++ = *s++; *t = 0;
00777         if ( AM.atstartup && t[-1] == '\n' ) t[-1] = 0;
00778     }
00779     else p->value = 0;
00780     p->wildarg = 0;
00781     if ( args ) {
00782         int first = 1;
00783         t++; p->argnames = t;
00784         s = args;
00785         while ( *s ) {
00786             if ( *s == ' ' || *s == '\t' ) { s++; continue; }
00787             if ( *s == ',' ) {
00788                 s++; *t++ = 0; numargs++;
00789                 while ( *s == ' ' || *s == '\t' ) s++;
00790                 if ( *s == '?' ) {
00791                     if ( p->wildarg > 0 ) {
00792                         Error0("More than one ?var in #define");
00793                     }
00794                     p->wildarg = numargs;
00795                 }

```

```

00796         }
00797         else if ( *s == '?' && first ) {
00798             p->wildarg = 1; *t++ = *s++;
00799         }
00800         else { *t++ = *s++; }
00801         first = 0;
00802     }
00803     *t = 0;
00804     numargs++;
00805     p->nargs = numargs;
00806 }
00807 else {
00808     p->nargs = 0;
00809     p->argnames = 0;
00810 }
00811 if ( u ) M_free(u,"replace PreVar value");
00812 return(ii);
00813 }
00814
00815 /*
00816     #] PutPreVar :
00817     #[ PopPreVars :
00818 */
00819
00820 void PopPreVars(int tonumber)
00821 {
00822     PREVAR *p = &(PreVar[NumPre]);
00823     while ( NumPre > tonumber ) {
00824         NumPre--; p--;
00825         M_free(p->name,"popping PreVar");
00826         p->name = p->value = 0;
00827     }
00828 }
00829
00830 /*
00831     #] PopPreVars :
00832     #[ IniModule :
00833 */
00834
00835 void IniModule(int type)
00836 {
00837     GETIDENTITY
00838     WORD **w, i;
00839     CBUF *C = cbuf+AC.cbufnum;
00840     /*[05nov2003 mt]:*/
00841 #ifdef WITHMPI
00842     /* To prevent
00843      * (1) FlushOut() and PutOut() on the slaves to send a mess to the master
00844      * compiling a module,
00845      * (2) EndSort() called from poly_factorize_expression() on the master
00846      * waits for the slaves.
00847      */
00848     PF.parallel=0;
00849     /*BTW, this was the bug preventing usage of more than 1 expression!*/
00850 #endif
00851
00852     AR.BracketOn = 0;
00853     AR.StoreData.dirtyflag = 0;
00854     AC.bracketindexflag = 0;
00855     AT.bracketindexflag = 0;
00856
00857     /*[06nov2003 mt]:*/
00858 #ifdef WITHMPI
00859     /* This flag may be set in the procedure tokenize(). */
00860     AC.RhsExprInModuleFlag = 0;
00861     /*[20oct2009 mt]:*/
00862     PF.mkSlaveInfile=0;
00863     PF.slavebuf.PObuffer=NULL;
00864     for(i=0; i<NumExpressions; i++)
00865         Expressions[i].vflags &= ~ISINRHS;
00866     /*:[20oct2009 mt]:*/
00867 #endif
00868     /*:[06nov2003 mt]:*/
00869
00870     /*[19nov2003 mt]:*/
00871     /*The module counter:*/
00872     (AC.CModule)++;
00873     /*:[19nov2003 mt]:*/
00874
00875     if ( !type ) {
00876         if ( C->rhs ) {
00877             w = C->rhs; i = C->maxrhs;
00878             do { *w++ = 0; } while ( --i > 0 );
00879         }
00880         if ( C->lhs ) {
00881             w = C->lhs; i = C->maxlhs;
00882             do { *w++ = 0; } while ( --i > 0 );

```

```

00883     }
00884 }
00885 C->numlhs = C->numrhs = 0;
00886 ClearTree(AC.cbufnum);
00887 while ( AC.NumLabels > 0 ) {
00888     AC.NumLabels--;
00889     if ( AC.LabelNames[AC.NumLabels] ) M_free(AC.LabelNames[AC.NumLabels], "LabelName");
00890 }
00891
00892 C->Pointer = C->Buffer;
00893
00894 AC.Commercial[0] = 0;
00895
00896 AC.IfStack = AC.IfHeap;
00897 AC.arglevel = 0;
00898 AC.termlevel = 0;
00899 AC.IfLevel = 0;
00900 AC.WhileLevel = 0;
00901 AC.RepLevel = 0;
00902 AC.insidelevel = 0;
00903 AC.dolooplevel = 0;
00904 AC.MustTestTable = 0;
00905 AO.PrintType = 0; /* Otherwise statistics can get spoiled */
00906 AC.ComDefer = 0;
00907 AC.CollectFun = 0;
00908 AM.S0->PolyWise = 0;
00909 AC.SymChangeFlag = 0;
00910 AP.lhdollarerror = 0;
00911 AR.PolyFun = AC.lPolyFun;
00912 AR.PolyFunInv = AC.lPolyFunInv;
00913 AR.PolyFunType = AC.lPolyFunType;
00914 AR.PolyFunExp = AC.lPolyFunExp;
00915 AR.PolyFunVar = AC.lPolyFunVar;
00916 AR.PolyFunPow = AC.lPolyFunPow;
00917 AC.mparallelfalg = AC.parallelfalg | AM.hparallelfalg;
00918 AC.inparallelfalg = 0;
00919 AC.mProcessBucketSize = AC.ProcessBucketSize;
00920 NumPotModdollars = 0;
00921 AC.topolynomialflag = 0;
00922 #ifdef WITHPTHREADS
00923     if ( AM.totalnumberofthreads > 1 ) AS.MultiThreaded = 1;
00924     else AS.MultiThreaded = 0;
00925     for ( i = 1; i < AM.totalnumberofthreads; i++ ) {
00926         AB[i]->T.S0->PolyWise = 0;
00927     }
00928 #endif
00929     OpenTemp();
00930 }
00931
00932 /*
00933     #] IniModule :
00934     #[ IniSpecialModule :
00935 */
00936
00937 void IniSpecialModule(int type)
00938 {
00939     DUMMYUSE(type);
00940 }
00941
00942 /*
00943     #] IniSpecialModule :
00944     #[ PreProcessor :
00945 */
00946
00947 void PreProcessor(void)
00948 {
00949     int moduletype = FIRSTMODULE;
00950     int specialtype = 0;
00951     int error1 = 0, error2 = 0, retcode, retval;
00952     UBYTE c, *t, *s;
00953     AP.StopWatchZero = GetRunningTime();
00954     AC.compiletype = 0;
00955     AP.PreContinuation = 0;
00956     AP.PreAssignLevel = 0;
00957     AP.gNumPre = NumPre;
00958     AC.iPointer = AC.iBuffer;
00959     AC.iPointer[0] = 0;
00960
00961     if ( AC.CheckpointFlag == -1 ) DoRecovery(&moduletype);
00962     AC.CheckpointStamp = Timer(0);
00963
00964     for(;;) {
00965         /* if ( A.StatisticsFlag ) CharOut(LINEFEED); */
00966
00967         IniModule(moduletype);
00968
00969         /*Re-define preprocessor variable CMODULE_ as a current module number, starting from 1*/

```

```

00970      /*The module counter is AC.CModule, it is incremented in IniModule*/
00971      {
00972          UBYTE buf[24];/*64/Log_2[10] = 19.3, this is enough for any integer*/
00973          NumToStr(buf,AC.CModule);
00974          PutPreVar((UBYTE *) "CMODULE_",buf,0,1);
00975      }
00976
00977      if ( specialtype ) IniSpecialModule(specialtype);
00978
00979      for(;;) { /* Read a single line/statement */
00980          c = GetChar(0);
00981          if ( c == AP.ComChar ) { /* This line is commentary */
00982              LoadInstruction(5);
00983              if ( AC.CurrentStream->FoldName ) {
00984                  t = AP.preStart;
00985                  if ( *t && t[1] && t[2] == '#' && t[3] == ']' ) {
00986                      t += 4;
00987                      while ( *t == ' ' || *t == '\t' ) t++;
00988                      s = AC.CurrentStream->FoldName;
00989                      while ( *s == *t ) { s++; t++; }
00990                      if ( *s == 0 && ( *t == ' ' || *t == '\t'
00991                          || *t == ':' ) ) {
00992                          while ( *t == ' ' || *t == '\t' ) t++;
00993                          if ( *t == ':' ) {
00994                              AC.CurrentStream = CloseStream(AC.CurrentStream);
00995                          }
00996                      }
00997                  }
00998              }
00999              *AP.preStart = 0;
01000              continue;
01001          }
01002          while ( c == ' ' || c == '\t' ) c = GetChar(0);
01003          if ( c == LINEFEED ) continue;
01004          if ( c == ENDOFINPUT ) {
01005              /* CharOut(LINEFEED); */
01006              Warning(".end instruction generated");
01007              moduletype = ENDMODULE; specialtype = 0;
01008              goto endmodule; /* Fake one */
01009          }
01010          if ( c == '#' ) {
01011              if ( PreProInstruction() ) { error1++; error2++; AP.preError++; }
01012              *AP.preStart = 0;
01013          }
01014          else if ( c == '.' ) {
01015              if ( ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) ||
01016                  ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) ) {
01017                  LoadInstruction(1);
01018                  continue;
01019              }
01020              if ( ModuleInstruction(&moduletype,&specialtype) ) { error2++; AP.preError++; }
01021              if ( specialtype ) SetSpecialMode(moduletype,specialtype);
01022              if ( AP.PreInsideLevel != 0 ) {
01023                  MesPrint("@end of module instructions may not be used inside");
01024                  MesPrint("@the scope of a %#inside %#endinside construction.");
01025                  Terminate(-1);
01026              }
01027              if ( AC.RepLevel > 0 ) {
01028                  MesPrint("&EndRepeat statement(s) missing");
01029                  error2++; AP.preError++;
01030              }
01031              if ( AC.tablecheck == 0 ) {
01032                  AC.tablecheck = 1;
01033                  if ( TestTables() ) { error2++; AP.preError++; }
01034              }
01035              if ( AP.PreContinuation ) {
01036                  error1++; error2++;
01037                  MesPrint("&Unfinished statement. Missing ;?");
01038              }
01039              if ( moduletype == GLOBALMODULE ) MakeGlobal();
01040              else {
01041                  endmodule:
01042                  if ( error2 == 0 && AM.qError == 0 ) {
01043                      retcode = ExecModule(moduletype);
01044                      #ifdef WITHMPI
01045                      if (PF.slavebuf.PObuffer!=NULL) {
01046                          M_free(PF.slavebuf.PObuffer,"PF inbuf");
01047                          PF.slavebuf.PObuffer=NULL;
01048                      }
01049                      #endif
01050                      UpdatePositions();
01051                      if ( retcode < 0 ) error1++;
01052                      if ( retcode ) { error2++; AP.preError++; }
01053                  }
01054                  else {
01055                      EXPRESSIONS e;
01056                      WORD j;
01057                      for ( j = 0, e = Expressions; j < NumExpressions; j++, e++ ) {

```

```

01057                                     if ( e->replace == NEWLYDEFINEDEXPRESSION ) e->replace =
REGULAREXPRESSION;
01058                                     }
01059                                     }
01060                                     switch ( moduletype ) {
01061                                     case STOREMODULE:
01062                                         if ( ExecStore() ) error1++;
01063                                         break;
01064                                     case CLEARMODULE:
01065                                         FullCleanUp();
01066                                         error1 = error2 = AP.preError = 0;
01067                                         AM.atstartup = 1;
01068                                         PutPreVar((UBYTE *) "DATE_", (UBYTE *) MakeDate(), 0, 1);
01069                                         AM.atstartup = 0;
01070                                         if ( AM.resetTimeOnClear ) {
01071 #ifdef WITHPTHREADS
01072                                             ClearAllThreads();
01073 #endif
01074                                             AM.SumTime += TimeCPU(1);
01075                                             TimeCPU(0);
01076                                         }
01077                                         AP.StopWatchZero = GetRunningTime();
01078                                         break;
01079                                     case ENDMODULE:
01080                                         Terminate( -( error1 | error2 ) );
01081                                     }
01082                                     }
01083                                     AC.tablecheck = 0;
01084                                     AC.compiletype = 0;
01085                                     if ( AC.exprfillwarning > 0 ) {
01086                                         AC.exprfillwarning = 0;
01087                                     }
01088                                     if ( AC.CheckpointFlag && error1 == 0 && error2 == 0 ) DoCheckpoint(moduletype);
01089                                     break; /* start a new module */
01090                                 }
01091                                 else {
01092                                     if ( ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) ||
01093                                         ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) ) {
01094                                         pushbackchar = c;
01095                                         LoadInstruction(5);
01096                                         continue;
01097                                     }
01098                                     UngetChar(c);
01099                                     if ( AP.PreContinuation ) {
01100                                         retval = LoadStatement(OLDSTATEMENT);
01101                                     }
01102                                     else {
01103                                         AC.CurrentStream->prevline = AC.CurrentStream->linenumber;
01104                                         retval = LoadStatement(NEWSTATEMENT);
01105                                     }
01106                                     if ( retval < 0 ) {
01107                                         error1++;
01108                                         if ( retval == -1 ) AP.PreContinuation = 0;
01109                                         else AP.PreContinuation = 1;
01110                                         TryRecover(0);
01111                                     }
01112                                     else if ( retval > 0 ) AP.PreContinuation = 0;
01113                                     else AP.PreContinuation = 1;
01114                                     if ( error1 == 0 && !AP.PreContinuation ) {
01115                                         if ( ( AP.PreDebug & PREPROONLY ) == 0 ) {
01116                                             int onpmd = NumPotModdollars;
01117 #ifdef WITHMPI
01118                                             WORD oldRhsExprInModuleFlag = AC.RhsExprInModuleFlag;
01119                                             if ( AP.PreAssignFlag ) AC.RhsExprInModuleFlag = 0;
01120 #endif
01121                                             if ( AP.PreOut || ( AP.PreDebug & DUMPTOCOMPILER )
01122                                                 == DUMPTOCOMPILER )
01123                                                 MesPrint(" %s", AC.iBuffer+AP.PreAssignStack[AP.PreAssignLevel]);
01124                                             retcode = CompileStatement(AC.iBuffer+AP.PreAssignStack[AP.PreAssignLevel]);
01125                                             if ( retcode < 0 ) error1++;
01126                                             if ( retcode ) { error2++; AP.preError++; }
01127                                             if ( AP.PreAssignFlag ) {
01128                                                 if ( retcode == 0 ) {
01129                                                     if ( ( retcode = CatchDollar(0) ) < 0 ) error1++;
01130                                                     else if ( retcode > 0 ) { error2++; AP.preError++; }
01131                                                 }
01132                                                 else CatchDollar(-1);
01133                                                 POPPREASSIGNLEVEL;
01134                                                 if ( AP.PreAssignLevel <= 0 )
01135                                                     AP.PreAssignFlag = 0;
01136                                                 NumPotModdollars = onpmd;
01137 #ifdef WITHMPI
01138                                                 AC.RhsExprInModuleFlag = oldRhsExprInModuleFlag;
01139 #endif
01140                                             }
01141                                         }
01142                                     }
01143                                 }
01144                                 else {

```



```

01143             MesPrint(" %s",AC.iBuffer+AP.PreAssignStack[AP.PreAssignLevel]);
01144         }
01145     }
01146     else if ( !AP.PreContinuation ) {
01147         if ( AP.PreAssignLevel > 0 ) {
01148             POPPREASSIGNLEVEL;
01149             if ( AP.PreAssignLevel <=0 )
01150                 AP.PreAssignFlag = 0;
01151         }
01152     }
01153     /*
01154         if ( !AP.PreContinuation ) AP.PreAssignFlag = 0;
01155     */
01156 }
01157 }
01158 }
01159 }
01160
01161 /*
01162     #] PreProcessor :
01163     #[ PreProInstruction :
01164 */
01165
01166 int PreProInstruction(void)
01167 {
01168     UBYTE *s, *t;
01169     KEYWORD *key;
01170     AP.PreproFlag = 1;
01171     AP.preFill = 0;
01172     AP.AllowDelay = 0;
01173     AP.DelayPrevar = 0;
01174
01175     oldmode = 0;
01176     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) {
01177         LoadInstruction(3);
01178         if ( ( StrICmp(AP.preStart, (UBYTE *)"case") == 0
01179             || StrICmp(AP.preStart, (UBYTE *)"default") == 0 )
01180             && AP.PreSwitchModes[AP.PreSwitchLevel] == SEARCHINGPRECASE ) {
01181             LoadInstruction(0);
01182         }
01183         else if ( StrICmp(AP.preStart, (UBYTE *)"assign ") == 0 ) {}
01184         else { LoadInstruction(1); }
01185     }
01186     else if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) {
01187         LoadInstruction(3);
01188         if ( ( StrICmp(AP.preStart, (UBYTE *)"else") == 0
01189             || StrICmp(AP.preStart, (UBYTE *)"elseif") == 0 )
01190             && AP.PreIfStack[AP.PreIfLevel] == LOOKINGFORELSE ) {
01191             LoadInstruction(0);
01192         }
01193         else if ( StrICmp(AP.preStart, (UBYTE *)"assign ") == 0 ) {}
01194         else {
01195             LoadInstruction(1);
01196         }
01197     }
01198     else {
01199         LoadInstruction(0);
01200     }
01201     AP.PreproFlag = 0;
01202     t = AP.preStart;
01203     if ( *t == '-' ) {
01204         if ( AP.PreSwitchModes[AP.PreSwitchLevel] == EXECUTINGPRESWITCH
01205             && AP.PreIfStack[AP.PreIfLevel] == EXECUTINGIF )
01206             AC.NoShowInput = 1;
01207     }
01208     else if ( *t == '+' ) {
01209         if ( AP.PreSwitchModes[AP.PreSwitchLevel] == EXECUTINGPRESWITCH
01210             && AP.PreIfStack[AP.PreIfLevel] == EXECUTINGIF )
01211             AC.NoShowInput = 0;
01212     }
01213     else if ( *t == ':' ) {}
01214     else {
01215         retry:;
01216         key = FindKeyWord(t,precommands,sizeof(precommands)/sizeof(KEYWORD));
01217         s = EndOfToken(t);
01218         if ( key == 0 ) {
01219             if ( *s == ';' ) {
01220                 *s = 0; goto retry;
01221             }
01222             else {
01223                 *s = 0;
01224                 MesPrint("@Unrecognized preprocessor instruction: %s",t);
01225                 return(-1);
01226             }
01227         }
01228         while ( *s == ' ' || *s == '\t' || *s == ',' ) s++;
01229         t = s;

```

```

01230         while ( *t ) t++;
01231         while ( ( t[-1] == ';' ) && ( t[-2] != '\\') ) {
01232             t--; *t = 0;
01233         }
01234         return((key->func)(s));
01235     }
01236     return(0);
01237 }
01238
01239 /*
01240     #] PreProInstruction :
01241     #[ LoadInstruction :
01242
01243     0:  preprocessor instruction that may involve matching of brackets
01244     1:  runs straight to end-of-line
01245     2:  runs to ;
01246     3:  only gets one word without ` interpretation.
01247     5:  with pushbackchar, but inside commentary. -> 1
01248
01249 To be added:
01250     In define, redefine, call and listed do we may have delayed substitution
01251     of preprocessor variables.
01252 */
01253
01254 int LoadInstruction(int mode)
01255 {
01256     UBYTE *s, *sstart, *t, c, cp;
01257     LONG position, fillpos = 0;
01258     int bralevel = 0, parlevel = 0, first = 1;
01259     int quotelevel = 0;
01260     if ( AP.preFill ) {
01261         s = AP.preFill;
01262         AP.preFill = 0;
01263         if ( s[1] != LINEFEED && s[1] != ENDOFINPUT ) {
01264             s[0] = s[1]; s++;
01265         }
01266         else { oldmode = mode; return(0); }
01267     }
01268     else { s = AP.preStart; }
01269     sstart = s; *s = 0;
01270     for(;;) {
01271         if ( ( mode & 1 ) == 1 ) {
01272             if ( pushbackchar && ( mode == 3 || mode == 5 ) ) {
01273                 c = pushbackchar; pushbackchar = 0;
01274             }
01275             else c = GetInput();
01276         }
01277         else {
01278             c = GetChar(0);
01279         }
01280
01281         if ( mode == 2 && c == ';' ) break;
01282         if ( ( mode == 1 || mode == 5 ) && c == LINEFEED ) break;
01283         if ( mode == 3 && FG.cTable[c] != 0 ) {
01284             if ( c == '$' ) {
01285                 pushbackchar = '$';
01286                 *s++ = 'a'; *s++ = 's'; *s++ = 's'; *s++ = 'i';
01287                 *s++ = 'g'; *s++ = 'n'; *s++ = ' '; *s = 0;
01288             }
01289             if ( c == '\\' || c == '`' ) { /* we do not expand preprocessor variables */
01290                 mode = 1;
01291             }
01292             else {
01293                 AP.preFill = s; *s++ = 0; *s = c;
01294                 oldmode = mode;
01295                 return(0);
01296             }
01297         }
01298         if ( mode == 0 && first ) {
01299             if ( c == '$' ) {
01300                 dodollar:
01301                 s = sstart;
01302                 *s++ = 'a'; *s++ = 's'; *s++ = 's'; *s++ = 'i';
01303                 *s++ = 'g'; *s++ = 'n'; *s = 0;
01304                 pushbackchar = c;
01305                 oldmode = mode;
01306                 return(0);
01307             }
01308             if ( c == ' ' || c == '\t' || c == ',' ) {}
01309             else first = 0;
01310         }
01311         else if ( mode == 1 && first && c == '$' && oldmode == 3 ) goto dodollar;
01312         if ( c == ENDOFINPUT || ( c == LINEFEED
01313             && bralevel == 0
01314             && quotelevel == 0 ) ) {
01315             if ( mode == 2 && c == ENDOFINPUT ) {
01316                 MesPrint("@Unexpected end of instruction");
01317                 oldmode = mode;

```

```

01317         return(-1);
01318     }
01319     /*
01320     if ( mode == 0 && bralevel ) {
01321         MesPrint("@Unmatched brackets");
01322         oldmode = mode;
01323         return(-1);
01324     }
01325     */
01326     if ( mode != 2 ) break;
01327 }
01328 if ( quotelevel ) {
01329     if ( c == '\\\\' ) {
01330         if ( ( mode == 1 ) || ( mode == 5 ) ) c = GetInput();
01331         else {
01332             c = GetChar(0);
01333         }
01334         if ( c == ENDOFINPUT ) {
01335             MesPrint("@Unmatched \\");
01336             if ( mode == 2 && c == ENDOFINPUT ) {
01337                 MesPrint("@Unexpected end of instruction");
01338             }
01339             /*
01340             if ( mode == 0 && bralevel ) {
01341                 MesPrint("@Unmatched brackets");
01342             }
01343             */
01344             oldmode = mode;
01345             return(-1);
01346         }
01347         else if ( c == LINEFEED ) {}
01348         else if ( c == '"' ) { *s++ = '\\\\'; }
01349         else {
01350             *s++ = '\\\\';
01351         }
01352     }
01353     else if ( c == '"' ) {
01354         quotelevel = 0;
01355         AP.AllowDelay = 0;
01356     }
01357 }
01358 else if ( c == '\\\\' ) {
01359     if ( ( mode == 1 ) || ( mode == 5 ) ) cp = GetInput();
01360     else {
01361         cp = GetChar(0);
01362     }
01363     if ( cp == LINEFEED ) continue;
01364     if ( mode != 2 || cp != ';' ) *s++ = c;
01365     c = cp;
01366 }
01367 else if ( c == '"' ) {
01368     /*
01369     Now look back in the buffer and determine what the keyword is.
01370     If it is define or redefine, put AllowDelay to 1.
01371     */
01372     t = AP.preStart;
01373     while ( FG.cTable[*t] <= 1 ) t++;
01374     cp = *t; *t = 0;
01375     if ( ( StrICmp(AP.preStart, (UBYTE *) "define") == 0 )
01376         || ( StrICmp(AP.preStart, (UBYTE *) "redefine") == 0 ) ) {
01377         AP.AllowDelay = 1;
01378         oldstream = AC.CurrentStream;
01379     }
01380     *t = cp;
01381     quotelevel = 1;
01382 }
01383 else if ( quotelevel == 0 && bralevel == 0 && c == '(' ) {
01384     t = AP.preStart;
01385     while ( FG.cTable[*t] <= 1 ) t++;
01386     cp = *t; *t = 0;
01387     if ( ( parlevel == 0 )
01388         && ( StrICmp(AP.preStart, (UBYTE *) "call") == 0 ) ) {
01389         AP.AllowDelay = 1;
01390         oldstream = AC.CurrentStream;
01391     }
01392     *t = cp;
01393     parlevel++;
01394 }
01395 else if ( quotelevel == 0 && bralevel == 0 && c == ')' ) {
01396     parlevel--;
01397 }
01398 else if ( quotelevel == 0 && parlevel == 0 && c == '{' ) {
01399     t = AP.preStart;
01400     while ( FG.cTable[*t] <= 1 ) t++;
01401     cp = *t; *t = 0;
01402     if ( ( bralevel == 0 )
01403         && ( StrICmp(AP.preStart, (UBYTE *) "call") == 0 )

```

```

01404         || ( StrICmp(AP.preStart, (UBYTE *) "do" ) == 0 ) ) {
01405         AP.AllowDelay = 1;
01406         oldstream = AC.CurrentStream;
01407     }
01408     *t = cp;
01409     bralevel++;
01410 }
01411 else if ( quotelevel == 0 && parlevel == 0 && c == '}' ) {
01412     bralevel--;
01413     if ( bralevel < 0 ) {
01414         if ( mode != 5 ) {
01415             MesPrint("@Unmatched brackets");
01416             oldmode = mode;
01417             return(-1);
01418         }
01419         bralevel = 0;
01420     }
01421 }
01422 if ( s >= (AP.preStop-1) ) {
01423     UBYTE **ppp;
01424     position = s - AP.preStart;
01425     if ( AP.preFill ) fillpos = AP.preFill - AP.preStart;
01426     ppp = &(AP.preStart); /* to avoid a compiler warning */
01427     if ( DoubleList((void ***)ppp, &AP.pSize, sizeof(UBYTE),
01428         "instruction buffer") ) { *s = 0; oldmode = mode; return(-1); }
01429     AP.preStop = AP.preStart + AP.pSize-3;
01430     s = AP.preStart + position;
01431     if ( AP.preFill ) AP.preFill = fillpos + AP.preStart;
01432 }
01433 *s++ = c;
01434 }
01435 *s = 0;
01436 oldmode = mode;
01437 if ( mode == 0 ) {
01438     if ( ExpandTripleDots(1) < 0 ) return(-1);
01439 }
01440 return(0);
01441 }
01442 /*
01443 #] LoadInstruction :
01444 #[ LoadStatement :
01445
01446 Puts the current string together in the input buffer.
01447 Does things like placing comma's where needed and expand ...
01448 We force a comma after the keyword. Before 8-sep-2009 the program might
01449 not put a comma if a + or - followed. And then the compiler ate
01450 the + or - and we needed repair code in the routines that used the
01451 + or - (Print, modulus, multiply and (a)bracket). This worked but
01452 the problem was with statements like Dimension -4; which then would
01453 be processed as Dimension 4; (JV)
01454 */
01455 */
01456
01457 int LoadStatement(int type)
01458 {
01459     UBYTE *s, c, cp;
01460     int retval = 0, stringlevel = 0, newstatement = 0;
01461     if ( type == NEWSTATEMENT ) { AP.eat = 1; newstatement = 1;
01462         s = AC.iBuffer+AP.PreAssignStack[AP.PreAssignLevel]; }
01463     else { s = AC.iPointer; *s = 0; c = ' '; goto blank; }
01464     *s = 0;
01465     for(;;) {
01466         c = GetChar(0);
01467         if ( c == ENDOFINPUT ) { retval = -1; break; }
01468         if ( stringlevel == 0 ) {
01469             if ( c == LINEFEED ) {
01470                 if ( AP.eat < 0 ) { s--; AP.eat = 0; }
01471                 retval = 0; break;
01472             }
01473             if ( c == ';' ) {
01474                 if ( AP.eat < 0 ) { s--; AP.eat = 0; }
01475                 while ( ( c = GetChar(0) ) == ' ' || c == '\t' ) {}
01476                 if ( c != LINEFEED ) UngetChar(c);
01477                 retval = 1;
01478                 break;
01479             }
01480         }
01481         if ( c == '\\\ ' ) {
01482             cp = GetChar(0);
01483             if ( cp == LINEFEED ) continue;
01484             *s++ = c;
01485             c = cp;
01486         }
01487         if ( c == '"' ) {
01488             if ( stringlevel == 0 ) stringlevel = 1;
01489             else stringlevel = 0;
01490             AP.eat = 0;

```

```

01491     }
01492     else if ( stringlevel == 0 ) {
01493         if ( c == '\t' ) c = ' ';
01494         if ( c == ' ' ) {
01495             blank: if ( newstatement < 0 ) newstatement = 0;
01496                     if ( AP.eat && ( newstatement == 0 ) ) continue;
01497                     c = ' ';
01498                     AP.eat = -2;
01499                     if ( newstatement > 0 ) newstatement = -1;
01500                 }
01501         else if ( chartype[c] <= 3 ) {
01502             AP.eat = 0;
01503             if ( newstatement < 0 ) newstatement = 0;
01504         }
01505         else if ( c == ',' ) {
01506             if ( newstatement > 0 ) {
01507                 newstatement = -1;
01508                 AP.eat = -2;
01509             }
01510             /* else if ( AP.eat == -2 ) { s--; } */
01511             else if ( AP.eat == -2 ) { AP.eat = 1; continue; }
01512             else { goto doall; }
01513         }
01514         else {
01515             doall:: if ( AP.eat < 0 ) {
01516                     if ( newstatement == 0 ) s--;
01517                     else { newstatement = 0; }
01518                 }
01519                 else if ( newstatement == 1 ) newstatement = 0;
01520                 AP.eat = 1;
01521                 if ( c == '*' && s > AC.iBuffer+AP.PreAssignStack[AP.PreAssignLevel] && s[-1] == '*' )
01522                     {
01523                         s[-1] = '^';
01524                         continue;
01525                     }
01526             }
01527             if ( s >= AC.iStop ) {
01528                 if ( !AP.iBufError ) {
01529                     LONG position = s - AC.iBuffer;
01530                     LONG position2 = AC.iPointer - AC.iBuffer;
01531                     UBYTE **ppp = &(AC.iBuffer); /* to avoid a compiler warning */
01532                     if ( DoubleList((void ***)ppp,&AC.iBufferSize
01533                                     ,sizeof(UBYTE),"statement buffer") ) {
01534                         *s = 0; retval = -1; AP.iBufError = 1;
01535                     }
01536                     AC.iPointer = AC.iBuffer + position2;
01537                     AC.iStop = AC.iBuffer + AC.iBufferSize-2;
01538                     s = AC.iBuffer + position;
01539                 }
01540                 if ( AP.iBufError ) {
01541                     for(;;){
01542                         c = GetChar(0);
01543                         if ( c == ENDOFINPUT ) { retval = -1; break; }
01544                         if ( c == '"' ) {
01545                             if ( stringlevel > 0 ) stringlevel = 0;
01546                             else stringlevel = 1;
01547                         }
01548                         else if ( c == LINEFEED && !stringlevel ) { retval = -2; break; }
01549                         else if ( c == ';' && !stringlevel ) {
01550                             while ( ( c = GetChar(0) ) == ' ' || c == '\t' ) {}
01551                             if ( c != LINEFEED ) UngetChar(c);
01552                             retval = -1;
01553                             break;
01554                         }
01555                         else if ( c == '\\\\' ) c = GetChar(0);
01556                     }
01557                     break;
01558                 }
01559             }
01560             *s++ = c;
01561         }
01562         AC.iPointer = s;
01563         *s = 0;
01564         if ( stringlevel > 0 ) {
01565             MesPrint("@Unbalanced \". Runaway string");
01566             retval = -1;
01567         }
01568         if ( retval == 1 ) {
01569             if ( ExpandTripleDots(0) < 0 ) retval = -1;
01570         }
01571         return(retval);
01572     }
01573 }
01574 /*
01575     #] LoadStatement :
01576     #[ ExpandTripleDots :

```

```

01577 */
01578
01579 static inline int IsSignChar(UBYTE c)
01580 {
01581     return c == '+' || c == '-';
01582 }
01583
01584 static inline int IsAlphanumericChar(UBYTE c)
01585 {
01586     return FG.cTable[c] == 0 || FG.cTable[c] == 1;
01587 }
01588
01589 static inline int CanParseSignedNumber(const UBYTE *s)
01590 {
01591     while ( IsSignChar(*s) ) s++;
01592     return FG.cTable[*s] == 1;
01593 }
01594
01595 int ExpandTripleDots(int par)
01596 {
01597     UBYTE *s, *s1, *s2, *n1, *n2, *t1, *t2, *startp, operator1, operator2, c, cc;
01598     UBYTE *nBuffer, *strngs, *Buffer, *Stop;
01599     LONG withquestion, x1, x2, y1, y2, number, inc, newsize, pow, fullsize;
01600     int i, error = 0, i1, i2, ii, *nums = 0;
01601
01602     if ( par == 0 ) {
01603         Buffer = AC.iBuffer+AP.PreAssignStack[AP.PreAssignLevel]; Stop = AC.iStop;
01604     }
01605     else {
01606         Buffer = AP.preStart; Stop = AP.preStop;
01607     }
01608     s = Buffer; while ( *s ) s++;
01609     fullsize = s - Buffer;
01610     if ( fullsize < 7 ) return(error);
01611
01612     s = Buffer+2;
01613     while ( *s ) {
01614         if ( *s != '.' || ( s[-1] != ',' && FG.cTable[s[-1]] != 5 ) )
01615             { s++; continue; }
01616         if ( s[-1] == '%' || s[-1] == '^' || s[1] != '.' || s[2] != '.' )
01617             { s++; continue; }
01618         s1 = s - 2;
01619         s += 3;
01620         if ( *s != s[-4] && ( *s != '+' || s[-4] != '-' )
01621             && ( *s != '-' || s[-4] != '+' ) ) {
01622             MesPrint("&Improper operators for ...");
01623             error = -1;
01624         }
01625         operator1 = s[-4];
01626         operator2 = s++;
01627         if ( operator1 == ':' ) operator1 = '.';
01628         if ( operator2 == ':' ) operator2 = '.';
01629     /*
01630         We have now O1...O2 (0 stands for operator)
01631         Full syntax is
01632         [str]#1[?]O1...O2[str]#2[?] (Special case)
01633         in which both strings are identical and if one ? then also the other.
01634         <pattern1>O1...O2<pattern2> (General case)
01635         in which the difference in the patterns is just numerical.
01636     */
01637     s2 = s; /* the beginning of the second string */
01638     if ( *s2 != '<' || *s1 != '>' ) { /* Special case */
01639         startp = s1+1;
01640         withquestion = ( *s1 == '?' ); s1--;
01641         while ( FG.cTable[*s1] == 1 && s1 >= Buffer ) s1--;
01642         n1 = s1+1; /* Beginning of first number */
01643         if ( FG.cTable[*n1] != 1 ) {
01644             MesPrint("&No first number in ... operator");
01645             error = -1;
01646         }
01647         while ( FG.cTable[*s1] <= 1 && s1 >= Buffer ) s1--;
01648         s1++;
01649     /*
01650         We have now the first string from s1 to n1, number from n1
01651     */
01652     t1 = s1; t2 = s2;
01653     while ( t1 < n1 && *t1 == *t2 ) { t1++; t2++; }
01654     n2 = t2;
01655     if ( FG.cTable[*t2] != 1 ) {
01656         MesPrint("&No second number in ... operator");
01657         error = -1;
01658     }
01659     x2 = 0;
01660     while ( FG.cTable[*t2] == 1 ) x2 = 10*x2 + *t2++ - '0';
01661     x1 = 0;
01662     while ( FG.cTable[*t1] == 1 ) x1 = 10*x1 + *t1++ - '0';
01663     if ( withquestion != ( *t2 == '?' ) ) {

```

```

01664         MesPrint("&Improper use of ? in ... operator");
01665         if ( *t2 == '?' ) t2++;
01666         error = -1;
01667     }
01668     else if ( withquestion ) t2++;
01669     if ( FG.cTable[*t2] <= 2 ) {
01670         MesPrint("&Illegal object after ... construction");
01671         error = -1;
01672     }
01673     c = *n1; *n1 = 0; s = t2;
01674     if ( error ) continue;
01675     /*
01676     At this point the syntax has been fulfilled. We have
01677     str in s1.
01678     x1,x2 are #1,#2
01679     operator1,operator2 are the two operators.
01680     s points at whatever comes after.
01681     Expansion will have to be computed.
01682     */
01683     if ( x2 < x1 ) { number = x1-x2; inc = -1; y1 = x2; y2 = x1; }
01684     else { number = x2-x1; inc = 1; y1 = x1; y2 = x2; }
01685     newsize = (number+1)*(n1-s1) /* the strings */
01686             + number /* the operators */
01687             +(number+1)*(withquestion?1:0) /* questionmarks */
01688             +(number+1); /* last digits */
01689     pow = 10;
01690     for ( i = 1; i < 10; i++, pow *= 10 ) {
01691         if ( y1 >= pow ) newsize += number+1;
01692         else if ( y2 >= pow ) newsize += y2-pow+1;
01693         else break;
01694     }
01695     while ( Buffer+(fullsize+newsize-(s-s1)) >= Stop ) {
01696         LONG strpos = s1-Buffer;
01697         LONG endstr = n1-Buffer;
01698         LONG startq = startp - Buffer;
01699         LONG position = s - Buffer;
01700         UBYTE **ppp;
01701         if ( par == 0 ) {
01702             LONG position2 = AC.iPointer - AC.iBuffer;
01703             ppp = &(AC.iBuffer); /* to avoid a compiler warning */
01704             if ( DoubleList((void ***)ppp,&AC.iBufferSize
01705                 ,sizeof(UBYTE),"statement buffer") ) {
01706                 Terminate(-1);
01707             }
01708             AC.iPointer = AC.iBuffer + position2;
01709             AC.iStop = AC.iBuffer + AC.iBufferSize-2;
01710             Buffer = AC.iBuffer+AP.PreAssignStack[AP.PreAssignLevel]; Stop = AC.iStop;
01711         }
01712         else {
01713             LONG fillpos = 0;
01714             if ( AP.preFill ) fillpos = AP.preFill - AP.preStart;
01715             ppp = &(AP.preStart); /* to avoid a compiler warning */
01716             if ( DoubleList((void ***)ppp,&AP.pSize,sizeof(UBYTE),
01717                 "instruction buffer") ) {
01718                 Terminate(-1);
01719             }
01720             AP.preStop = AP.preStart + AP.pSize-3;
01721             if ( AP.preFill ) AP.preFill = fillpos + AP.preStart;
01722             Buffer = AP.preStart; Stop = AP.preStop;
01723         }
01724         s = Buffer + position;
01725         n1 = Buffer + endstr;
01726         s1 = Buffer + strpos;
01727         startp = Buffer + startq;
01728     }
01729     /*
01730     We have space for the expansion in the buffer.
01731     There are two cases: new size > old size
01732                        old size >= new size
01733     Note that wherever we move things, it will be at least startp.
01734     */
01735     if ( newsize > (s-s1) ) {
01736         t2 = Buffer + fullsize;
01737         t1 = t2 + (newsize - (s-s1));
01738         *t1 = 0;
01739         while ( t2 > s ) { *--t1 = *--t2; }
01740     }
01741     else if ( newsize < (s-s1) ) {
01742         t1 = s1 + newsize; t2 = s; s = t1;
01743         while ( *t2 ) *t1++ = *t2++;
01744         *t1 = 0;
01745     }
01746     for ( x1 += inc, t1 = startp; number > 0; number--, x1 += inc ) {
01747         *t1++ = operator1;
01748         cc = operator1; operator1 = operator2; operator2 = cc;
01749         t2 = s1; while ( *t2 ) *t1++ = *t2++;
01750         x2 = x1; n2 = t1;

```

```

01751         do {
01752             *t1++ = '0' + x2 % 10;
01753             x2 /= 10;
01754         } while ( x2 );
01755         s2 = t1 - 1;
01756         while ( s2 > n2 ) { cc = *s2; *s2 = *n2; *n2++ = cc; s2--; }
01757         if ( withquestion ) *t1++ = '?';
01758     }
01759     fullsize += newsize - ( s - s1 );
01760     *n1 = c;
01761 }
01762 else { /* General case. Find the patterns first */
01763     t1 = s1; s1--;
01764     while ( s1 > Buffer ) {
01765         if ( *s1 == '<' ) break;
01766         s1--;
01767     }
01768     t2 = s2;
01769     while ( *t2 ) {
01770         if ( *t2 == '>' ) break;
01771         t2++;
01772     }
01773     if ( *s1 != '<' || *t2 != '>' ) {
01774         MesPrint("&Illegal attempt to use ... operator");
01775         return(-1);
01776     }
01777     s1++; s2++; /* Pointers to the patterns */
01778     nums = (int *)Malloc1((t1-s1)*2*(sizeof(int)+sizeof(UBYTE))
01779         , "Expand ...");
01780     strngs = (UBYTE *) (nums + 2*(t1-s1));
01781     n1 = s1; n2 = s2; ii = -1; i = 0;
01782     s = strngs;
01783     while ( n1 < t1 || n2 < t2 ) {
01784         /* Check the next characters can be parsed as numbers including signs. */
01785         if ( CanParseSignedNumber(n1) && CanParseSignedNumber(n2) ) {
01786             /*
01787              * Don't allow the cases that one has the sign and the other doesn't,
01788              * and the meaning changes without the sign. For example,
01789              * <f(1)>+...<f(3)> Allowed
01790              * <f(-2)>+...<f(2)> Allowed
01791              * <f(x-2)>+...<f(x+2)> Allowed
01792              * <f(x-2)>+...<f(x2)> Not allowed
01793              */
01794             int sign1 = IsSignChar(*n1);
01795             int sign2 = IsSignChar(*n2);
01796             int inword1 = s1 < n1 && IsAlphanumericChar(n1[-1]);
01797             int inword2 = s2 < n2 && IsAlphanumericChar(n2[-1]);
01798             if ( ( sign1 ^ sign2 ) && ( inword1 || inword2 ) ) break; /* Not allowed. */
01799             if ( sign1 || sign2 ) {
01800                 *s++ = '+'; /* Marker indicating we need the sign. */
01801             }
01802         } else {
01803             /* If they are not numbers, they should be same. */
01804             if ( *n1 == *n2 ) { *s++ = *n1++; n2++; continue; }
01805             else break;
01806         }
01807         ParseSignedNumber(x1,n1)
01808         ParseSignedNumber(x2,n2)
01809         if ( x1 == x2 ) {
01810             if ( s != strngs && ( s[-1] == '+' || s[-1] == '-' ) ) {
01811                 /* We need the sign. */
01812                 s--;
01813                 if ( x1 >= 0 ) {
01814                     *s++ = '+';
01815                 }
01816             }
01817             s = NumCopy(x1, s);
01818         }
01819         else {
01820             nums[2*i] = x1; nums[2*i+1] = x2;
01821             i++; *s++ = 0;
01822         }
01823     }
01824     if ( n1 < t1 || n2 < t2 ) {
01825         MesPrint("&Improper use of ... operator.");
01826 theend: M_free(nums, "Expand ...");
01827         return(-1);
01828     }
01829     *s = 0;
01830     if ( i == 0 ) ii = 0;
01831     else {
01832         ii = nums[0] - nums[1];
01833         if ( ii < 0 ) ii = -ii;
01834         for ( x1 = 1; x1 < i; x1++ ) {
01835             x2 = nums[2*x1]-nums[2*x1+1];
01836             if ( x2 < 0 ) x2 = -x2;
01837             if ( x2 != ii ) {

```



```

01838             MesPrint("&Improper synchronization of numbers in ... operator");
01839             goto theend;
01840         }
01841     }
01842 }
01843 ii++;
01844 /*
01845     We have now proper syntax.
01846     There are i+1 strings in strngs and i pairs of numbers
01847     in nums. Each time a start value and a finish value.
01848     We have ii steps. If ii <= 2, it will fit in the existing
01849     allocation. But this is hardly useful.
01850     We make a new allocation and copy from the old.
01851     Compute space.
01852 */
01853 x2 = s - strngs - i; /* -1 for end-of-string and +1 for the operator*/
01854 for ( i1 = 0; i1 < i; i1++ ) {
01855     i2 = nums[2*i1];
01856     x1 = nums[2*i1+1];
01857     if ( i2 < 0 ) i2 = -i2;
01858     if ( x1 < 0 ) x1 = -x1;
01859     if ( x1 > i2 ) i2 = x1;
01860     x1 = 2;
01861     while ( i2 > 0 ) { i2 /= 10; x1++; }
01862     x2 += x1;
01863 }
01864 x2 *= ii; /* Space for the expanded string (a bit more) */
01865 x2 += fullsize;
01866 x2 += 5; /* This will definitely hold everything */
01867 x2 += sizeof(UBYTE *);
01868 x2 = x2 - (x2 & (sizeof(UBYTE *)-1));
01869 nBuffer = (UBYTE *)Malloc1(x2,"input buffer");
01870 n1 = nBuffer; s = Buffer; s1--;
01871 while ( s < s1 ) *n1++ = *s++;
01872 /*
01873     Solution of the special case that no comma was generated
01874     due to the presence of < to start the pattern.
01875     We get a comma when the word before ends in an alphanumeric
01876     character, a _ or a ] and the word inside starts with an
01877     alphanumeric character, a [ (or an _ (for future considerations))
01878 */
01879 if ( ( ( n1 > nBuffer ) && ( ( FG.cTable[n1[-1]] <= 1 )
01880 || ( n1[-1] == '_' ) || ( n1[-1] == ']' ) ) ) &&
01881 ( ( FG.cTable[strngs[0]] <= 1 ) || ( strngs[0] == '[' )
01882 || ( strngs[0] == '_' ) ) ) *n1++ = ',';
01883
01884 for ( i1 = 0; i1 < ii; i1++ ) {
01885     s = strngs; while ( *s ) *n1++ = *s++;
01886     for ( i2 = 0; i2 < i; i2++ ) {
01887         if ( n1 > nBuffer && IsSignChar(n1[-1]) ) {
01888             /* We need the sign of counters. */
01889             n1--;
01890             if ( nums[2*i2] >= 0 ) {
01891                 *n1++ = '+';
01892             }
01893         }
01894         n1 = NumCopy((WORD)(nums[2*i2]),n1);
01895         if ( nums[2*i2] > nums[2*i2+1] ) nums[2*i2]--;
01896         else nums[2*i2]++;
01897         s++; while ( *s ) *n1++ = *s++;
01898     }
01899     if ( ( i1 & 1 ) == 0 ) *n1++ = operator1;
01900     else *n1++ = operator2;
01901 }
01902 n1--; /* drop the trailing operator */
01903 s = t2 + 1; n2 = n1;
01904 /*
01905     Similar extra comma
01906 */
01907 if ( ( ( ( FG.cTable[n1[-1]] <= 1 )
01908 || ( n1[-1] == '_' ) || ( n1[-1] == ']' ) ) ) &&
01909 ( ( FG.cTable[s[0]] <= 1 ) || ( s[0] == '[' )
01910 || ( s[0] == '_' ) ) ) *n1++ = ',';
01911
01912 while ( *s ) *n1++ = *s++;
01913 *n1 = 0;
01914 if ( par == 0 ) {
01915     LONG nnn1 = n1-nBuffer;
01916     LONG nnn2 = n2-nBuffer;
01917     LONG nnn3;
01918     while ( AC.iBuffer+AP.PreAssignStack[AP.PreAssignLevel] + x2 >= AC.iStop ) {
01919         LONG position = s-Buffer;
01920         LONG position2 = AC.iPointer - AC.iBuffer;
01921         UBYTE **ppp;
01922         ppp = &(AC.iBuffer); /* to avoid a compiler warning */
01923         if ( DoubleLList((void ***)ppp,&AC.iBufferSize

```

```

01925         ,sizeof(UBYTE),"statement buffer") ) {
01926             Terminate(-1);
01927         }
01928         AC.iPointer = AC.iBuffer + position2;
01929         AC.iStop = AC.iBuffer + AC.iBufferSize-2;
01930         Buffer = AC.iBuffer+AP.PreAssignStack[AP.PreAssignLevel]; Stop = AC.iStop;
01931         s = Buffer + position;
01932     }
01933     /*
01934         This can be improved. We only have to start from the first term.
01935     */
01936     for ( nnn3 = 0; nnn3 < nnn1; nnn3++ ) Buffer[nnn3] = nBuffer[nnn3];
01937     Buffer[nnn3] = 0;
01938     n1 = Buffer + nnn1;
01939     n2 = Buffer + nnn2;
01940     M_free(nBuffer,"input buffer");
01941     M_free(nums,"Expand ...");
01942 }
01943 else { /* Comes here only inside a real preprocessor instruction */
01944     AP.preStop = nBuffer + x2 - 2;
01945     AP.pSize = x2;
01946     M_free(AP.preStart,"input buffer");
01947     M_free(nums,"Expand ...");
01948     AP.preStart = nBuffer;
01949     Buffer = AP.preStart; Stop = AP.preStop;
01950 }
01951 fullsize = n1 - Buffer;
01952 s = n2;
01953 }
01954 }
01955 return(error);
01956 }
01957
01958 /*
01959     #] ExpandTripleDots :
01960     #[ FindKeyword :
01961 */
01962
01963 KEYWORD *FindKeyword(UBYTE *theword, KEYWORD *table, int size)
01964 {
01965     int low,med,hi;
01966     UBYTE *s1, *s2;
01967     low = 0;
01968     hi = size-1;
01969     while ( hi >= low ) {
01970         med = (hi+low)/2;
01971         s1 = (UBYTE *) (table[med].name);
01972         s2 = theword;
01973         while ( *s1 && tolower(*s1) == tolower(*s2) ) { s1++; s2++; }
01974         if ( *s1 == 0 &&
01975             /*[30apr2004 mt]:*/
01976             /* The bug!!:
01977                 FG.cTable[*s2] != 1 && FG.cTable[*s2] != 2
01978             */
01979                 FG.cTable[*s2] != 0 && FG.cTable[*s2] != 1
01980             /*
01981                 ( *s2 == ' ' || *s2 == '\t' || *s2 == 0 || *s2 == ',' || *s2 == '(' ) */
01982             )
01983             return(table+med);
01984         if ( tolower(*s2) > tolower(*s1) ) low = med+1;
01985         else hi = med - 1;
01986     }
01987     return(0);
01988 }
01989 /*
01990     #] FindKeyword :
01991     #[ FindInKeyword :
01992 */
01993
01994 KEYWORD *FindInKeyword(UBYTE *theword, KEYWORD *table, int size)
01995 {
01996     int i;
01997     UBYTE *s1, *s2;
01998     for ( i = 0; i < size; i++ ) {
01999         s1 = (UBYTE *) (table[i].name);
02000         s2 = theword;
02001         while ( *s1 && tolower(*s1) == tolower(*s2) ) { s1++; s2++; }
02002         if ( *s2 == 0 || *s2 == ' ' || *s2 == ',' || *s2 == '\t' )
02003             return(table+i);
02004     }
02005     return(0);
02006 }
02007
02008 /*
02009     #] FindInKeyword :
02010     #[ TheDefine :
02011 */

```

```

02012
02024 int TheDefine(UBYTE *s, int mode)
02025 {
02026     UBYTE *name, *value, *valpoin, *args = 0, c;
02027     if ( ( mode & 2 ) == 0 ) {
02028         if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
02029         if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
02030     }
02031     else { mode &= ~2; }
02032     name = s;
02033     if ( chartype[*s] != 0 ) goto illname;
02034     s++;
02035     while ( chartype[*s] <= 1 ) s++;
02036     value = s;
02037     while ( *s == ' ' || *s == '\t' ) s++;
02038     c = *s; *value = 0;
02039     if ( c == 0 ) {
02040         if ( PutPreVar(name, (UBYTE *) "1", 0, mode) < 0 ) return(-1);
02041         return(0);
02042     }
02043     if ( c == '(' ) { /* arguments. scan for correctness */
02044         s++; args = s;
02045         for (;;) {
02046             if ( chartype[*s] != 0 ) goto illarg;
02047             s++;
02048             while ( chartype[*s] <= 1 ) s++;
02049             while ( *s == ' ' || *s == '\t' ) s++;
02050             if ( *s == ')' ) break;
02051             if ( *s != ',' ) goto illargs;
02052             s++;
02053             while ( *s == ' ' || *s == '\t' ) s++;
02054         }
02055         *s++ = 0;
02056         while ( *s == ' ' || *s == '\t' ) s++;
02057         c = *s;
02058     }
02059     if ( c == '"' ) {
02060         s++; valpoin = value = s;
02061         while ( *s != '"' ) {
02062             if ( *s == '\\' ) {
02063                 if ( s[1] == 'n' ) { *valpoin++ = LINEFEED; s += 2; }
02064                 else if ( s[1] == '"' ) { *valpoin++ = '"'; s += 2; }
02065                 else if ( s[1] == 0 ) goto illval;
02066                 else { *valpoin++ = *s++; *valpoin++ = *s++; }
02067             }
02068             else *valpoin++ = *s++;
02069         }
02070         *valpoin = 0;
02071         if ( PutPreVar(name, value, args, mode) < 0 ) return(-1);
02072     }
02073     else {
02074         MesPrint("@Illegal string for preprocessor variable %s. Forgotten double quotes (\") ?", name);
02075         return(-1);
02076     }
02077     return(0);
02078 illname:;
02079     MesPrint("@Illegally formed name of preprocessor variable");
02080     return(-1);
02081 illarg:;
02082     MesPrint("@Illegally formed name of argument of preprocessor definition");
02083     return(-1);
02084 illargs:;
02085     MesPrint("@Illegally formed arguments of preprocessor definition");
02086     return(-1);
02087 illval:;
02088     MesPrint("@Illegal valpoin for preprocessor variable %s", name);
02089     return(-1);
02090 }
02091
02092 /*
02093     #] TheDefine :
02094     #[ DoCommentChar :
02095 */
02096
02097 int DoCommentChar(UBYTE *s)
02098 {
02099     UBYTE c;
02100     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
02101     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
02102     while ( *s == ' ' || *s == '\t' ) s++;
02103     if ( *s == 0 || *s == '\n' ) {
02104         MesPrint("@No valid comment character specified");
02105         return(-1);
02106     }
02107     c = *s++;
02108     while ( *s == ' ' || *s == '\t' ) s++;
02109     if ( *s != 0 && *s != '\n' ) {

```

```

02110         MesPrint("@Comment character should be a single valid character");
02111         return(-1);
02112     }
02113     AP.ComChar = c;
02114     return(0);
02115 }
02116
02117 /*
02118     #] DoCommentChar :
02119     #[ DoPreAssign :
02120
02121     Routine assigns a 'value' to a $variable.
02122     Syntax: #assign
02123           next line(s) a statement of the type
02124           $name = expression;
02125     Note: at the moment of the assign there cannot be an 'open' statement.
02126 */
02127
02128 int DoPreAssign(UBYTE *s)
02129 {
02130     int error = 0;
02131     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) {
02132         return(0);
02133     }
02134     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) {
02135         return(0);
02136     }
02137     if ( *s ) {
02138         MesPrint("@Illegal characters in %#assign instruction");
02139         error = 1;
02140     }
02141     PUSHPREASSIGNLEVEL;
02142     AP.PreAssignFlag = 1;
02143 /*
02144     if ( AP.PreContinuation ) {
02145         MesPrint("@Assign instructions cannot occur inside statements");
02146         MesPrint("@Missing ; ?");
02147         AP.PreContinuation = 0;
02148         error = 1;
02149     }
02150 */
02151     return(error);
02152 }
02153
02154 /*
02155     #] DoPreAssign :
02156     #[ DoDefine :
02157 */
02158
02159 int DoDefine(UBYTE *s)
02160 {
02161     return(TheDefine(s,0));
02162 }
02163
02164 /*
02165     #] DoDefine :
02166     #[ DoRedefine :
02167 */
02168
02169 int DoRedefine(UBYTE *s)
02170 {
02171     return(TheDefine(s,1));
02172 }
02173
02174 /*
02175     #] DoRedefine :
02176     #[ ClearMacro :
02177
02178     Undefines the arguments of a macro after its use.
02179 */
02180
02181 int ClearMacro(UBYTE *name)
02182 {
02183     int i;
02184     PREVAR *p;
02185     UBYTE *s;
02186     for ( i = NumPre-1, p = &(PreVar[NumPre-1]); i >= 0; i--, p-- ) {
02187         if ( StrCmp(name,p->name) == 0 ) break;
02188     }
02189     if ( i < 0 ) return(-1);
02190     if ( p->nargs <= 0 ) return(0);
02191     s = p->argnames;
02192     for ( i = 0; i < p->nargs; i++ ) {
02193         TheUndefine(s);
02194         while ( *s ) s++;
02195         s++;
02196     }

```

```

02197     return(0);
02198 }
02199
02200 /*
02201     #] ClearMacro :
02202     #[ TheUndefine :
02203
02204     There is a complication here. If there are redefine statements
02205     they will be pointing at the wrong variable if their number is
02206     greater than the number of the variable we pop.
02207 */
02208
02209 int TheUndefine(UBYTE *name)
02210 {
02211     int i, inum, error = 0;
02212     PREVAR *p;
02213     for ( i = NumPre-1, p = &(PreVar[NumPre-1]); i >= 0; i--, p-- ) {
02214         if ( StrCmp(name,p->name) == 0 ) {
02215             M_free(p->name,"undefining PreVar");
02216             NumPre--;
02217             inum = i;
02218             while ( i < NumPre ) {
02219                 p->name = p[1].name;
02220                 p->value = p[1].value;
02221                 p++; i++;
02222             }
02223             p->name = 0; p->value = 0;
02224             {
02225                 CBUF *CC = cbuf + AC.cbunum;
02226                 int j, k;
02227                 for ( j = 1; j <= CC->numlhs; j++ ) {
02228                     if ( CC->lhs[j][0] == TYPEREDEFPRE ) {
02229                         if ( CC->lhs[j][2] > inum ) CC->lhs[j][2]--;
02230                         else if ( CC->lhs[j][2] == inum ) {
02231                             for ( k = inum - 1; k >= 0; k-- )
02232                                 if ( StrCmp(name, PreVar[k].name) == 0 ) break;
02233                             if ( k >= 0 ) CC->lhs[j][2] = k;
02234                             else {
02235                                 MesPrint("@Conflict between undefining a preprocessor variable and a
02236                                 redefine statement");
02237                                 error = 1;
02238                             }
02239                         }
02240                     }
02241                 }
02242                 #ifdef PARALLELCODE
02243                 for ( j = 0; j < AC.pfirstnum; j++ ) {
02244                     if ( AC.pfirstnum[j] > inum ) AC.pfirstnum[j]--;
02245                     else if ( AC.pfirstnum[j] == inum ) {
02246                         for ( k = inum - 1; k >= 0; k-- )
02247                             if ( StrCmp(name, PreVar[k].name) == 0 ) break;
02248                         if ( k >= 0 ) AC.pfirstnum[j] = k;
02249                     }
02250                 }
02251                 #endif
02252                 break;
02253             }
02254         }
02255     }
02256     return(error);
02257 }
02258 /*
02259     #] TheUndefine :
02260     #[ DoUndefine :
02261 */
02262
02263 int DoUndefine(UBYTE *s)
02264 {
02265     UBYTE *name, *t;
02266     int error = 0, retval;
02267     /*
02268     int i;
02269     PREVAR *p;
02270 */
02271     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
02272     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
02273     name = s;
02274     if ( chartype[*s] != 0 ) goto illname;
02275     s++;
02276     while ( chartype[*s] <= 1 ) s++;
02277     t = s;
02278     if ( *s && *s != ' ' && *s != '\t' ) goto illname;
02279     while ( *s == ' ' || *s == '\t' ) s++;
02280     if ( *s ) {
02281         MesPrint("@Undefine should just have a variable name");
02282         error = -1;

```

```

02283     }
02284     *t = 0;
02285     if ( ( retval = TheUndefine(name) ) != 0 ) {
02286         if ( error == 0 ) return(retval);
02287         if ( error > 0 ) error = retval;
02288     }
02289     /*
02290     for ( i = NumPre-1, p = &(PreVar[NumPre-1]); i >= 0; i--, p-- ) {
02291         if ( StrCmp(name,p->name) == 0 ) {
02292             M_free(p->name,"undefining PreVar");
02293             NumPre--;
02294             while ( i < NumPre ) {
02295                 p->name = p[1].name;
02296                 p->value = p[1].value;
02297                 p++; i++;
02298             }
02299             p->name = 0; p->value = 0;
02300             break;
02301         }
02302     }
02303     */
02304     return(error);
02305 illname:;
02306     MesPrint("@Illegally formed name of preprocessor variable");
02307     return(-1);
02308 }
02309
02310 /*
02311     #] DoUndefine :
02312     #[ DoInclude :
02313 */
02314
02315 int DoInclude(UBYTE *s) { return(Include(s,FILESTREAM)); }
02316
02317 /*
02318     #] DoInclude :
02319     #[ DoReverseInclude :
02320 */
02321
02322 int DoReverseInclude(UBYTE *s) { return(Include(s,REVERSEFILESTREAM)); }
02323
02324 /*
02325     #] DoReverseInclude :
02326     #[ Include :
02327 */
02328
02329 int Include(UBYTE *s, int type)
02330 {
02331     UBYTE *name = s, *fold, *t, c, c1 = 0, c2 = 0, c3 = 0;
02332     int strtoloffset, withnolist = AC.NoShowInput;
02333     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
02334     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
02335     if ( *s == '-' || *s == '+' ) {
02336         if ( *s == '-' ) withnolist = 1;
02337         else withnolist = 0;
02338         s++;
02339         while ( *s == ' ' || *s == '\t' ) s++;
02340         name = s;
02341     }
02342     if ( *s == '"' ) {
02343         while ( *s && *s != '"' ) {
02344             if ( *s == '\\' ) s++;
02345             s++;
02346         }
02347         t = s++;
02348     }
02349     else {
02350         while ( *s && *s != ' ' && *s != '\t' ) {
02351             if ( *s == '\\' ) s++;
02352             s++;
02353         }
02354         t = s;
02355     }
02356     while ( *s == ' ' || *s == '\t' ) s++;
02357     if ( *s == '#' ) {
02358         *t = 0;
02359         s++;
02360         while ( *s == ' ' || *s == '\t' ) s++;
02361         fold = s;
02362         if ( *s == 0 ) {
02363             MesPrint("@Empty fold name");
02364             return(-1);
02365         }
02366     continue_fold:
02367         while ( *s && *s != ' ' && *s != '\t' ) {
02368             if ( *s == '\\' ) s++;
02369             s++;

```

```

02370     }
02371     t = s;
02372     while ( *s == ' ' || *s == '\t' ) s++;
02373     if ( *s ) {
02374         /*
02375          * A non-whitespace character is found. Continue parsing the fold.
02376          */
02377         goto continue_fold;
02378     }
02379 }
02380 else if ( *s == 0 ) {
02381     fold = 0;
02382 }
02383 else {
02384     MesPrint("@Improper syntax for file name");
02385     return(-1);
02386 }
02387 *t = 0;
02388 if ( fold ) {
02389     fold = strDup1(fold,"foldname");
02390 }
02391 /*
02392 We have the name of the file in 'name' and the fold in 'fold' (or NULL)
02393 */
02394 if ( OpenStream(name,type,0,PRENOACTION) == 0 ) {
02395     if ( fold ) { M_free(fold,"foldname"); fold = 0; }
02396     return(-1);
02397 }
02398 if ( fold ) {
02399     LONG position = -1;
02400     int foldopen = 0;
02401     LONG linenum = 0, prevline = 0;
02402     name = strDup1(name,"name of include file");
02403     AC.CurrentStream->FoldName = strDup1(fold,"name of fold");
02404     AC.NoShowInput++;
02405     for(;;) {
02406         c = GetFromStream(AC.CurrentStream);
02407         if ( c == ENDOFSTREAM ) {
02408             AC.CurrentStream = CloseStream(AC.CurrentStream);
02409             goto nofold;
02410         }
02411         if ( c == AP.ComChar ) {
02412             strloffset = AC.CurrentStream-AC.Streams;
02413             LoadInstruction(1);
02414             if ( AC.CurrentStream != strloffset+AC.Streams ) {
02415                 c = ENDOFSTREAM;
02416             }
02417             else {
02418                 t = AP.preStart;
02419                 if ( t[2] == '#' && ( t[3] == '[' && !foldopen )
02420                     || ( t[3] == ']' && foldopen ) ) {
02421                     t += 4;
02422                     while ( *t == ' ' || *t == '\t' ) t++;
02423                     s = AC.CurrentStream->FoldName;
02424                     while ( *s == *t ) { s++; t++; }
02425                     if ( *s == 0 && ( *t == ' ' || *t == '\t'
02426                         || *t == ':' ) ) {
02427                         while ( *t == ' ' || *t == '\t' ) t++;
02428                         if ( *t == ':' ) {
02429                             if ( foldopen == 0 ) {
02430                                 foldopen = 1;
02431                                 position = GetStreamPosition(AC.CurrentStream);
02432                                 linenum = AC.CurrentStream->linenum;
02433                                 prevline = AC.CurrentStream->prevline;
02434                                 c3 = AC.CurrentStream->isnextchar;
02435                                 c1 = AC.CurrentStream->nextchar[0];
02436                                 c2 = AC.CurrentStream->nextchar[1];
02437                             }
02438                             else {
02439                                 foldopen = 0;
02440                                 PositionStream(AC.CurrentStream,position);
02441                                 AC.CurrentStream->linenum = linenum;
02442                                 AC.CurrentStream->prevline = prevline;
02443                                 AC.CurrentStream->eqnum = 1;
02444                                 AC.NoShowInput--;
02445                                 AC.CurrentStream->isnextchar = c3;
02446                                 AC.CurrentStream->nextchar[0] = c1;
02447                                 AC.CurrentStream->nextchar[1] = c2;
02448                                 break;
02449                             }
02450                         }
02451                     }
02452                 }
02453             }
02454         }
02455         else {
02456             while ( c != LINEFEED && c != ENDOFSTREAM ) {

```

```

02457             c = GetFromStream(AC.CurrentStream);
02458             if ( c == ENDOFSTREAM ) {
02459                 AC.CurrentStream = CloseStream(AC.CurrentStream);
02460                 break;
02461             }
02462         }
02463     }
02464     if ( c == ENDOFSTREAM ) {
02465 nofold:
02466         MesPrint("@Cannot find fold %s in file %s",fold,name);
02467         UngetChar(c);
02468         AC.NoShowInput--;
02469         M_free(name,"name of include file");
02470         Terminate(-1);
02471     }
02472 }
02473 M_free(name,"name of include file");
02474 }
02475 AC.NoShowInput = withnolist;
02476 if ( fold ) { M_free(fold,"foldname"); fold = 0; }
02477 return(0);
02478 }
02479
02480 /*
02481     #] Include :
02482     #[ DoPreExchange :
02483
02484     Exchanges the names of expressions or the contents of dollars
02485     Syntax:
02486         #exchange expr1,expr2
02487         #exchange $var1,$var2
02488 */
02489
02490 int DoPreExchange(UBYTE *s)
02491 {
02492     int error = 0;
02493     UBYTE *s1, *s2;
02494     WORD num1, num2;
02495     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
02496     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
02497     while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
02498     if ( *s == '$' ) {
02499         s++; s1 = s; while ( FG.cTable[*s] <= 1 ) s++;
02500         if ( *s != ',' && *s != ' ' && *s != '\t' ) goto syntax;
02501         *s++ = 0;
02502         while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
02503         if ( *s != '$' ) goto syntax;
02504         s++; s2 = s; while ( FG.cTable[*s] <= 1 ) s++;
02505         if ( *s != 0 && *s != ';' ) goto syntax;
02506         *s = 0;
02507         if ( ( num1 = GetDollar(s1) ) <= 0 ) {
02508             MesPrint("@$%s has not been defined (yet)",s1);
02509             error = 1;
02510         }
02511         if ( ( num2 = GetDollar(s2) ) <= 0 ) {
02512             MesPrint("@$%s has not been defined (yet)",s2);
02513             error = 1;
02514         }
02515         if ( error == 0 ) {
02516             ExchangeDollars((int)num1,(int)num2);
02517         }
02518     }
02519     else {
02520         s1 = s; s = SkipAName(s);
02521         if ( *s != ',' && *s != ' ' && *s != '\t' ) goto syntax;
02522         *s++ = 0;
02523         while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
02524         if ( FG.cTable[*s] != 0 && *s != '[' ) goto syntax;
02525         s2 = s; s = SkipAName(s);
02526         if ( *s != 0 && *s != ';' ) goto syntax;
02527         *s = 0;
02528         if ( GetName(AC.exprnames,s1,&num1,NOAUTO) != CEXPRESSION ) {
02529             MesPrint("@%s is not an expression",s1);
02530             error = 1;
02531         }
02532         if ( GetName(AC.exprnames,s2,&num2,NOAUTO) != CEXPRESSION ) {
02533             MesPrint("@%s is not an expression",s2);
02534             error = 1;
02535         }
02536         if ( error == 0 ) {
02537             ExchangeExpressions((int)num1,(int)num2);
02538         }
02539     }
02540     return(error);
02541 syntax:
02542     MesPrint("@Proper syntax: %#exchange expr1,expr2 or %#exchange $var1,$var2");
02543     return(1);

```



```

02544 }
02545
02546 /*
02547     #] DoPreExchange :
02548     #[ DoCall :
02549 */
02550
02551 int DoCall(UBYTE *s)
02552 {
02553     UBYTE *t, *u, *v, *name, c, cp, *args1, *args2, *t1, *t2, *wild = 0;
02554     int bratype = 0, wildargs = 0, inwildargs = 0, nwildargs = 0;
02555     PROCEDURE *p;
02556     int streamoffset;
02557     int i, namesize, narg1, narg2, bralevel, numpre;
02558     LONG i1, i2;
02559     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
02560     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
02561 /*
02562     1: Get the name of the procedure.
02563     2: Locate the procedure.
02564 */
02565     name = s; s = EndOfToken(s); c = *s; *s = 0;
02566     for ( i = NumProcedures-1; i >= 0; i-- ) {
02567         if ( StrCmp(Procedures[i].name,name) == 0 ) break;
02568     }
02569     p = (PROCEDURE *)FromList(&AP.ProcList);
02570     if ( i < 0 ) { /* Try to find a file */
02571         namesize = 0;
02572         t = name;
02573         while ( *t ) { t++; namesize++; }
02574         t = AP.procedureExtension;
02575         while ( *t ) { t++; namesize++; }
02576         t = p->name = (UBYTE *)Malloc1(namesize+2,"procedure");
02577         u = name;
02578         while ( *u ) *t++ = *u++;
02579         *t++ = '.';
02580         v = AP.procedureExtension;
02581         while ( *v ) *t++ = *v++;
02582         *t = 0;
02583         p->loadmode = 0; /* buffer should be freed at end */
02584         p->p.buffer = LoadInputFile(p->name,PROCEDUREFILE);
02585         if ( p->p.buffer == 0 ) return(-1);
02586         t[-4] = 0;
02587     }
02588     else {
02589         p->p.buffer = Procedures[i].p.buffer;
02590         p->name = Procedures[i].name;
02591         p->loadmode = 1;
02592     }
02593     t = p->p.buffer;
02594     SKIPBLANKS(t)
02595     if ( *t++ != '#' ) goto wrongfile;
02596     SKIPBLANKS(t)
02597     t += 9;
02598     SKIPBLANKS(t)
02599     u = EndOfToken(t);
02600     cp = *u; *u = 0;
02601     if ( StrCmp(t,name) != 0 ) goto wrongfile;
02602     *u = cp;
02603     *s = c;
02604 /*
02605     The pointer p points to the contents of the procedure (in memory)
02606     Now we have to match the arguments. u points to after the name
02607     in the 'file', s to after the name in the call statement.
02608 */
02609     bralevel = narg1 = narg2 = 0; args2 = u;
02610     SKIPBLANKS(u)
02611     if ( *u == '(' ) {
02612         u++; SKIPBLANKS(u)
02613         args2 = u;
02614         while ( *u != ')' ) {
02615             if ( *u == '?' ) { wildargs++; u++; nwildargs = narg2+1; }
02616             narg2++; u = EndOfToken(u); SKIPBLANKS(u)
02617             if ( *u == ',' ) { u++; SKIPBLANKS(u) }
02618             else if ( *u != ')' || ( wildargs > 1 ) ) {
02619                 MesPrint("@Illegal argument field in procedure %s",p->name);
02620                 return(-1);
02621             }
02622         }
02623     }
02624     while ( *u != LINEFEED ) u++;
02625     SKIPBLANKS(s)
02626     args1 = s+1;
02627     if ( *s == '(' ) bratype = 1;
02628     do {
02629         if ( *s == '{' && bratype == 0 ) bralevel++;
02630         else if ( *s == '(' && bratype == 1 ) bralevel++;

```

```

02631     else if ( *s == '}' && bratype == 0 ) {
02632         bralevel--;
02633         if ( bralevel == 0 ) {
02634             *s = 0; nargs1++;
02635             if ( wildargs && nargs1 == nwildargs ) wild = s;
02636         }
02637     }
02638     else if ( *s == ')' && bratype == 1 ) {
02639         bralevel--;
02640         if ( bralevel == 0 ) {
02641             *s = 0; nargs1++;
02642             if ( wildargs && nargs1 == nwildargs ) wild = s;
02643         }
02644     }
02645     /*[12dec2003 mt]:*/
02646     /*else if ( *s == ',' || *s == '|' ) {*/
02647     else if (set_in(*s,AC.separators)) {/*Function set_in see in
02648                                         file tools.c*/
02649         /*:[12dec2003 mt]:*/
02650         *s = 0; nargs1++;
02651         if ( wildargs && nargs1 == nwildargs ) wild = s;
02652     }
02653     else if ( *s == '\\') s++;
02654     s++;
02655 } while ( bralevel > 0 );
02656 if ( wildargs && nargs1 >= nargs2-1 ) {
02657     inwildargs = nargs1-nargs2+1;
02658     if ( inwildargs == 0 ) nwildargs = 0;
02659     else {
02660         while ( inwildargs > 1 ) {
02661             *wild = ',';
02662             while ( *wild ) wild++;
02663             inwildargs--;
02664         }
02665     }
02666 }
02667 else if ( nargs1 != nargs2 && ( nargs2 != 0 || nargs1 != 1 || *args1 != 0 ) ) {
02668     MesPrint("@Arguments of procedure %s are not matching",p->name);
02669     return(-1);
02670 }
02671 numpre = -NumPre-1; /* For the stream */
02672 for ( i = 0; i < nargs2; i++ ) {
02673     t = args2;
02674     if ( *t == '?' ) {
02675         args2++;
02676     }
02677     if ( *t == '?' && inwildargs == 0 ) {
02678         args2 = EndOfToken(args2); c = *args2; *args2 = 0;
02679         if ( PutPreVar(t,(UBYTE *)"",0,0) < 0 ) return(-1);
02680     }
02681     else {
02682         args2 = EndOfToken(args2); c = *args2; *args2 = 0;
02683         t1 = t2 = args1;
02684         while ( *t1 ) {
02685             if ( *t1 == '\\') t1++;
02686             if ( t1 != t2 ) *t2 = *t1;
02687             t2++; t1++;
02688         }
02689         *t2 = 0;
02690         if ( PutPreVar(t,args1,0,0) < 0 ) return(-1);
02691         args1 = t1+1; /* Next argument */
02692     }
02693     *args2 = c; SKIPBLANKS(args2) /* skip to next name */
02694     args2++; SKIPBLANKS(args2)
02695 }
02696 streamoffset = AC.CurrentStream - AC.Streams;
02697 args1 = AC.CurrentStream->name;
02698 AC.CurrentStream->name = p->name;
02699 i1 = AC.CurrentStream->linenumber;
02700 i2 = AC.CurrentStream->prevline;
02701 AC.CurrentStream->prevline =
02702 AC.CurrentStream->linenumber = 2;
02703 OpenStream(u+1,PREREADSTREAM3,numpre,PRENOACTION);
02704 AC.Streams[streamoffset].name = args1;
02705 AC.Streams[streamoffset].linenumber = i1;
02706 AC.Streams[streamoffset].prevline = i2;
02707 AddToPreTypes(PRETYPEPROCEDURE);
02708 return(0);
02709 wrongfile:;
02710 if ( i < 0 ) MesPrint("@File %s is not a proper procedure",p->name);
02711 else MesPrint("!!!Internal error with procedure names: %s",name);
02712 return(-1);
02713 }
02714
02715 /*
02716 #] DoCall :
02717 #[ DoDebug :

```

```

02718 */
02719
02720 int DoDebug(UBYTE *s)
02721 {
02722     int x;
02723     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
02724     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
02725     NeedNumber(x,s,nonumber)
02726     if ( x < 0 || x > (PREPROONLY
02727         | DUMPTOCOMPILER
02728         | DUMPOUTTERMS
02729         | DUMPINTERMS
02730         | DUMPTOSORT
02731         | DUMPTOPARALLEL
02732 #ifdef WITHPTHREADS
02733         | THREADSDEBUG
02734 #endif
02735         ) ) goto nonumber;
02736     AP.PreDebug = 0;
02737     if ( ( x & PREPROONLY ) != 0 ) AP.PreDebug |= PREPROONLY; /* 1 */
02738     if ( ( x & DUMPTOCOMPILER ) != 0 ) AP.PreDebug |= DUMPTOCOMPILER; /* 2 */
02739     if ( ( x & DUMPOUTTERMS ) != 0 ) AP.PreDebug |= DUMPOUTTERMS; /* 4 */
02740     if ( ( x & DUMPINTERMS ) != 0 ) AP.PreDebug |= DUMPINTERMS; /* 8 */
02741     if ( ( x & DUMPTOSORT ) != 0 ) AP.PreDebug |= DUMPTOSORT; /* 16 */
02742     if ( ( x & DUMPTOPARALLEL ) != 0 ) AP.PreDebug |= DUMPTOPARALLEL; /* 32 */
02743 #ifdef WITHPTHREADS
02744     if ( ( x & THREADSDEBUG ) != 0 ) AP.PreDebug |= THREADSDEBUG; /* 64 */
02745 #endif
02746     return(0);
02747 nonumber:
02748     MesPrint("@Illegal argument for debug instruction");
02749     return(1);
02750 }
02751
02752 /*
02753     #] DoDebug :
02754     #[ DoTerminate :
02755 */
02756
02757 int DoTerminate(UBYTE *s)
02758 {
02759     int x;
02760     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
02761     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
02762     if ( *s ) {
02763         NeedNumber(x,s,nonumber)
02764         Terminate(x);
02765     }
02766     else {
02767         Terminate(-1);
02768     }
02769     return(0);
02770 nonumber:
02771     MesPrint("@Illegal argument for terminate instruction");
02772     return(1);
02773 }
02774
02775 /*
02776     #] DoTerminate :
02777     #[ DoContinueDo :
02778 */
02779
02780 int DoContinueDo(UBYTE *s)
02781 {
02782     DOLOOP *loop;
02783
02784     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
02785     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
02786
02787     if ( NumDoLoops <= 0 ) {
02788         MesPrint("@%#continuedo without %#do");
02789         return(1);
02790     }
02791
02792     loop = &(DoLoops[NumDoLoops-1]);
02793     AP.NumPreTypes = loop->NumPreTypes+1;
02794     AP.PreIfLevel = loop->PreIfLevel;
02795     AP.PreSwitchLevel = loop->PreSwitchLevel;
02796
02797     return(DoEnddo(s));
02798 }
02799
02800 /*
02801     #] DoContinueDo :
02802     #[ DoDo :
02803
02804     The do loop has three varieties:

```

```

02805         #do i = num1,num2 [,num3]
02806         #do i = {string1,string2,...,stringn}
02807         The | as separator is also allowed for backwards compatibility
02808         #do i = expression      One by one all terms of the expression
02809     */
02810
02811     int DoDo(UBYTE *s)
02812     {
02813         GETIDENTITY
02814         UBYTE *t, c, *u, *uu;
02815         DOLOOP *loop;
02816         WORD expnum;
02817         LONG linenum = AC.CurrentStream->linenumber;
02818         int oldNoShowInput = AC.NoShowInput, i, oldpreassignflag;
02819
02820         if ( ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH )
02821         || ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) ) {
02822             if ( PreSkip((UBYTE *) "do", (UBYTE *) "enddo", 1) ) return(-1);
02823             return(0);
02824         }
02825
02826     /*
02827         if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
02828         if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
02829     */
02830     AddToPreTypes(PRETYPEPDO);
02831
02832     loop = (DOLOOP *) FromList(&AP.LoopList);
02833     loop->firstdollar = loop->lastdollar = loop->incdollar = -1;
02834     loop->NumPreTypes = AP.NumPreTypes-1;
02835     loop->PreIfLevel = AP.PreIfLevel;
02836     loop->PreSwitchLevel = AP.PreSwitchLevel;
02837     AC.NoShowInput = 1;
02838     if ( PreLoad(&(loop->p), (UBYTE *) "do", (UBYTE *) "enddo", 1, "doloop") ) return(-1);
02839     AC.NoShowInput = oldNoShowInput;
02840     loop->NoShowInput = AC.NoShowInput;
02841     /*
02842         Get now the name. We have to take great care when the name is terminated!
02843     */
02844     s = loop->p.buffer + (s - AP.preStart);
02845     SKIPBLANKS(s)
02846     loop->name = s;
02847     if ( chartype[*s] != 0 ) goto illname;
02848     s++;
02849     while ( chartype[*s] <= 1 ) s++;
02850     t = s;
02851     while ( *s == ' ' || *s == '\t' ) s++;
02852     if ( *s != '=' ) goto illdo;
02853     s++;
02854     while ( *s == ' ' || *s == '\t' ) s++;
02855     *t = 0;
02856
02857     if ( *s == '{' ) {
02858         loop->type = LISTEDLOOP;
02859         s++; loop->vars = s;
02860         loop->lastnum = 0;
02861         while ( *s != '}' && *s != 0 ) {
02862             if ( set_in(*s, AC.separators) ) { *s = 0; loop->lastnum++; }
02863             else if ( *s == '\\\'' ) s++;
02864             s++;
02865         }
02866         if ( *s == 0 ) goto illdo;
02867         *s++ = 0;
02868         loop->lastnum++;
02869         loop->firstnum = 0;
02870         loop->contents = s;
02871     }
02872     else if ( *s == '-' || *s == '+' || chartype[*s] == 1 || *s == '$' ) {
02873         loop->type = NUMERICALLOOP;
02874         t = s;
02875         while ( *s && *s != ',' ) s++;
02876         if ( *s == 0 ) goto illdo;
02877         if ( *t == '$' ) {
02878             c = *s; *s = 0;
02879             if ( GetName(AC.dollarnames, t+1, &loop->firstdollar, NOAUTO) != CDOLLAR ) {
02880                 MesPrint("@%s is undefined in first parameter in %ndo instruction", t);
02881                 return(-1);
02882             }
02883             loop->firstnum = DolToLong(BHEAD loop->firstdollar);
02884             if ( AN.ErrorInDollar ) {
02885                 MesPrint("@%s does not evaluate into a valid loop parameter", t);
02886                 return(-1);
02887             }
02888             *s++ = c;
02889         }
02890         else {
02891             *s = ' ';

```

```

02892         if ( PreEval(t,&loop->firstnum) == 0 ) goto illdo;
02893         *s++ = ',';
02894     }
02895     t = s;
02896     while ( *s && *s != ',' && *s != ';' && *s != LINEFEED ) s++;
02897     c = *s;
02898     if ( *t == '$' ) {
02899         *s = 0;
02900         if ( GetName(AC.dollarnames,t+1,&loop->lastdollar,NOAUTO) != CDOLLAR ) {
02901             MesPrint("@%s is undefined in second parameter in %sdo instruction",t);
02902             return(-1);
02903         }
02904         loop->lastnum = DolToLong(BHEAD loop->lastdollar);
02905         if ( AN.ErrorInDollar ) {
02906             MesPrint("@%s does not evaluate into a valid loop parameter",t);
02907             return(-1);
02908         }
02909         *s++ = c;
02910     }
02911     else {
02912         *s = ' ';
02913         if ( PreEval(t,&loop->lastnum) == 0 ) goto illdo;
02914         *s++ = c;
02915     }
02916     if ( c == ',' ) {
02917         t = s;
02918         while ( *s && *s != ';' && *s != LINEFEED ) s++;
02919         if ( *t == '$' ) {
02920             c = *s; *s = 0;
02921             if ( GetName(AC.dollarnames,t+1,&loop->incdollar,NOAUTO) != CDOLLAR ) {
02922                 MesPrint("@%s is undefined in third parameter in %sdo instruction",t);
02923                 return(-1);
02924             }
02925             loop->incnum = DolToLong(BHEAD loop->incdollar);
02926             if ( AN.ErrorInDollar ) {
02927                 MesPrint("@%s does not evaluate into a valid loop parameter",t);
02928                 return(-1);
02929             }
02930             *s++ = c;
02931         }
02932         else {
02933             c = *s; *s = ' ';
02934             if ( PreEval(t,&loop->incnum) == 0 ) goto illdo;
02935             *s++ = c;
02936         }
02937     }
02938     else loop->incnum = 1;
02939     loop->contents = s;
02940 }
02941 else if ( ( chartype[*s] == 0 ) || ( *s == '[' ) ) {
02942     int oldNumPotModdollars = NumPotModdollars;
02943 #ifndef WITHMPI
02944     WORD oldRhsExprInModuleFlag = AC.RhsExprInModuleFlag;
02945     AC.RhsExprInModuleFlag = 0;
02946 #endif
02947     t = s;
02948     if ( ( s = SkipAName(s) ) == 0 ) goto illdo;
02949     c = *s; *s = 0;
02950     if ( GetName(AC.exprnames,t,&expnum,NOAUTO) == CEXPRESSION ) {
02951         loop->type = ONEEXPRESSION;
02952     /*
02953         We should remember the expression by name for when it gets
02954         renumbered!!! If it gets deleted there will be a crash or at
02955         least the loop terminates.
02956     */
02957         loop->vars = t;
02958     }
02959     else goto illdo;
02960     if ( c == ',' || c == '\t' || c == ';' ) { s++; }
02961     else if ( c != 0 && c != '\n' ) goto illdo;
02962     while ( *s == ',' || *s == '\t' || *s == ';' ) s++;
02963     if ( *s != 0 && *s != '\n' ) goto illdo;
02964     loop->firstnum = 0;
02965     s++;
02966     loop->contents = s;
02967     loop->incnum = 0;
02968 /*
02969     Next determine size of statement and allocate space
02970 */
02971     while ( *t ) t++;
02972     i = t - loop->vars;
02973     t = loop->name;
02974     while ( *t ) { t++; i++; }
02975     i += 4;
02976     loop->dollarname = Malloc1((LONG)i,"do-loop instruction");
02977 /*
02978     Construct the statement

```

```

02979 */
02980     u = loop->dollarname;
02981     *u++ = '$'; t = loop->name; while ( *t ) *u++ = *t++;
02982     *u++ = '_'; uu = u; *u++ = '='; t = loop->vars;
02983     while ( *t ) *u++ = *t++;
02984     *t = 0; *u = 0;
02985 /*
02986     Compile and put in dollar variable.
02987     Note that we remember the dollar by name and that this name ends in _
02988 */
02989     oldpreassignflag = AP.PreAssignFlag;
02990     AP.PreAssignFlag = 2;
02991     CompileStatement(loop->dollarname);
02992     if ( CatchDollar(0) ) {
02993         MesPrint("@Cannot load expression in do loop");
02994         return(-1);
02995     }
02996     AP.PreAssignFlag = oldpreassignflag;
02997     NumPotModdollars = oldNumPotModdollars;
02998 #ifdef WITHMPI
02999     AC.RhsExprInModuleFlag = oldRhsExprInModuleFlag;
03000 #endif
03001     *uu = 0;
03002 }
03003 else goto illdo; /* Syntax problems */
03004 loop->errorsinloop = 0;
03005 /* loop->startlinenumber = linenum+1; 5-oct-2000 One too much? */
03006 loop->startlinenumber = linenum;
03007 PutPreVar(loop->name, (UBYTE *) "0", 0, 0);
03008 loop->firstloopcall = 1;
03009 return(DoEnddo(s));
03010 illname:;
03011 MesPrint("@Improper name for do loop variable");
03012 return(-1);
03013 illdo:;
03014 MesPrint("@Improper syntax in do loop instruction");
03015 return(-1);
03016 }
03017
03018 /*
03019     #] DoDo :
03020     #[ DoBreakDo :
03021
03022     #breakdo [num]
03023     jumps out of num #do-loops (if there are that many) (default is 1)
03024 */
03025
03026 int DoBreakDo(UBYTE *s)
03027 {
03028     DOLOOP *loop;
03029     WORD levels;
03030
03031     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
03032     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
03033
03034     if ( NumDoLoops <= 0 ) {
03035         MesPrint("@%#breakdo without %#do");
03036         return(1);
03037     }
03038 /*
03039     if ( AP.PreTypes[AP.NumPreTypes] != PRETYPEDO ) { MessPreNesting(4); return(-1); }
03040 */
03041     while ( *s && ( *s == ',' || *s == ' ' || *s == '\t' ) ) s++;
03042     if ( *s == 0 ) {
03043         levels = 1;
03044     }
03045     else if ( FG.cTable[*s] == 1 ) {
03046         levels = 0;
03047         while ( *s >= '0' && *s <= '9' ) { levels = 10*levels + *s++ - '0'; }
03048         if ( *s != 0 ) goto improper;
03049     }
03050     else {
03051 improper:
03052         MesPrint("@Improper syntax of %#breakdo instruction");
03053         return(1);
03054     }
03055     if ( levels > NumDoLoops ) {
03056         MesPrint("@Too many loop levels requested in %#breakdo instruction");
03057         Terminate(-1);
03058     }
03059     while ( levels > 0 ) {
03060         while ( AC.CurrentStream->type != PREREADSTREAM
03061                && AC.CurrentStream->type != PREREADSTREAM2
03062                && AC.CurrentStream->type != PREREADSTREAM3 ) {
03063             AC.CurrentStream = CloseStream(AC.CurrentStream);
03064         }
03065         while ( AP.PreTypes[AP.NumPreTypes] != PRETYPEDO

```

```

03066     && AP.PreTypes[AP.NumPreTypes] != PRETYPEPROCEDURE ) AP.NumPreTypes--;
03067     if ( AC.CurrentStream->type == PREREADSTREAM3
03068     || AP.PreTypes[AP.NumPreTypes] == PRETYPEPROCEDURE ) {
03069         MesPrint("@Trying to jump out of a procedure with a %#breakdo instruction");
03070         Terminate(-1);
03071     }
03072     loop = &(DoLoops[NumDoLoops-1]);
03073     AP.NumPreTypes = loop->NumPreTypes;
03074     AP.PreIfLevel = loop->PreIfLevel;
03075     AP.PreSwitchLevel = loop->PreSwitchLevel;
03076     /*
03077     AP.NumPreTypes--;
03078     */
03079     NumDoLoops--;
03080     DoUndefine(loop->name);
03081     M_free(loop->p.buffer, "loop->p.buffer");
03082     loop->firstloopcall = 0;
03083
03084     AC.CurrentStream = CloseStream(AC.CurrentStream);
03085     levels--;
03086 }
03087 return(0);
03088 }
03089
03090 /*
03091     #] DoBreakDo :
03092     #[ DoElse :
03093     */
03094
03095 int DoElse(UBYTE *s)
03096 {
03097     if ( AP.PreTypes[AP.NumPreTypes] != PRETYPEEIF ) {
03098         if ( AP.PreIfLevel <= 0 ) MesPrint("@%#else without corresponding %#if");
03099         else MessPreNesting(1);
03100         return(-1);
03101     }
03102     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
03103     while ( *s == ' ' ) s++;
03104     if ( tolower(*s) == 'i' && tolower(s[1]) == 'f' && s[2]
03105         && FG.cTable[s[2]] > 1 && s[2] != '_' ) {
03106         s += 2;
03107         while ( *s == ' ' ) s++;
03108         return(DoElseif(s));
03109     }
03110     if ( AP.PreIfLevel <= 0 ) {
03111         MesPrint("@%#else without corresponding %#if");
03112         return(-1);
03113     }
03114     switch ( AP.PreIfStack[AP.PreIfLevel] ) {
03115         case EXECUTINGIF:
03116             AP.PreIfStack[AP.PreIfLevel] = LOOKINGFORENDIF;
03117             break;
03118         case LOOKINGFORELSE:
03119             AP.PreIfStack[AP.PreIfLevel] = EXECUTINGIF;
03120             break;
03121         case LOOKINGFORENDIF:
03122             break;
03123     }
03124     return(0);
03125 }
03126
03127 /*
03128     #] DoElse :
03129     #[ DoElseif :
03130     */
03131
03132 int DoElseif(UBYTE *s)
03133 {
03134     int condition;
03135     if ( AP.PreTypes[AP.NumPreTypes] != PRETYPEEIF ) {
03136         if ( AP.PreIfLevel <= 0 ) MesPrint("@%#elseif without corresponding %#if");
03137         else MessPreNesting(2);
03138         return(-1);
03139     }
03140     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
03141     if ( AP.PreIfLevel <= 0 ) {
03142         MesPrint("@%#elseif without corresponding %#if");
03143         return(-1);
03144     }
03145     switch ( AP.PreIfStack[AP.PreIfLevel] ) {
03146         case EXECUTINGIF:
03147             AP.PreIfStack[AP.PreIfLevel] = LOOKINGFORENDIF;
03148             break;
03149         case LOOKINGFORELSE:
03150             if ( ( condition = EvalPreIf(s) ) < 0 ) return(-1);
03151             AP.PreIfStack[AP.PreIfLevel] = condition;
03152             break;

```

```

03153         case LOOKINGFORENDIF:
03154             break;
03155     }
03156     return(0);
03157 }
03158
03159 /*
03160     #] DoElseif :
03161     #[ DoEnddo :
03162
03163     At the first call there is no stream yet.
03164     After that we have to close the stream and start a new one.
03165 */
03166
03167 int DoEnddo(UBYTE *s)
03168 {
03169     GETIDENTITY
03170     DOLOOP *loop;
03171     UBYTE *t, *tt, *value, numstr[16];
03172     LONG xval;
03173     int xsign, retval;
03174     DUMMYUSE(s);
03175     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
03176     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
03177 /*
03178     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ||
03179         AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) {
03180         if ( AP.PreTypes[AP.NumPreTypes] == PRETYPEDO ) AP.NumPreTypes--;
03181         else { MessPreNesting(3); return(-1); }
03182         return(0);
03183     }
03184 */
03185     if ( NumDoLoops <= 0 ) {
03186         MesPrint("@%#enddo without %#do");
03187         return(1);
03188     }
03189     if ( AP.PreTypes[AP.NumPreTypes] != PRETYPEDO ) { MessPreNesting(4); return(-1); }
03190     loop = &(DoLoops[NumDoLoops-1]);
03191     if ( !loop->firstloopcall ) AC.CurrentStream = CloseStream(AC.CurrentStream);
03192
03193     if ( loop->errorsinloop ) {
03194         MesPrint("++++Errors in Loop");
03195         goto finish;
03196     }
03197     if ( loop->type == LISTEDLOOP ) {
03198         if ( loop->firstnum >= loop->lastnum ) goto finish;
03199         loop->firstnum++;
03200         t = value = loop->vars;
03201         while ( *value ) value++;
03202         value++;
03203         loop->vars = value;
03204         value = tt = t;
03205         while ( *value ) {
03206             if ( *value == '\\\\' ) value++;
03207             *tt++ = *value++;
03208         }
03209         *tt = 0;
03210         PutPreVar(loop->name,t,0,1); /* We overwrite the definition */
03211     }
03212     else if ( loop->type == NUMERICALLOOP ) {
03213
03214         if ( !loop->firstloopcall ) {
03215 /*
03216             Test whether the variable was changed inside the loop into
03217             a different numerical value. If so, adjust.
03218 */
03219             t = GetPreVar(loop->name,WITHOUTERROR);
03220             if ( t ) {
03221                 value = t;
03222                 xsign = 1;
03223                 while ( *value && ( *value == ' ' ||
03224                     || *value == '-' || *value == '+' ) ) {
03225                     if ( *value == '-' ) xsign = -xsign;
03226                     value++;
03227                 }
03228                 t = value; xval = 0;
03229                 while ( *value >= '0' && *value <= '9' ) xval = 10*xval + *value++ - '0';
03230                 while ( *value && *value == ' ' ) value++;
03231                 if ( *value == 0 ) {
03232 /*
03233                     Now we may substitute the loopvalue.
03234 */
03235                     if ( xsign < 0 ) xval = -xval;
03236                     if ( loop->incdollar >= 0 ) {
03237                         loop->incnum = DolToLong(BHEAD loop->incdollar);
03238                         if ( AN.ErrorInDollar ) {
03239                             MesPrint("@%s does not evaluate into a valid third loop

```



```

parameter",DOLLARNAME(Dollars,loop->incdollar));
03240         return(-1);
03241     }
03242     }
03243     loop->firstnum = xval + loop->incnum;
03244     }
03245     }
03246     if ( loop->lastdollar >= 0 ) {
03247         loop->lastnum = DolToLong(BHEAD loop->lastdollar);
03248         if ( AN.ErrorInDollar ) {
03249             MesPrint("@%s does not evaluate into a valid second loop
parameter",DOLLARNAME(Dollars,loop->lastdollar));
03250             return(-1);
03251         }
03252     }
03253     }
03254     if ( ( loop->incnum > 0 && loop->firstnum > loop->lastnum )
|| ( loop->incnum < 0 && loop->firstnum < loop->lastnum ) ) goto finish;
03255     NumToStr(numstr,loop->firstnum);
03256     t = numstr;
03257     loop->firstnum += loop->incnum;
03258     PutPreVar(loop->name,t,0,1); /* We overwrite the definition */
03259 }
03260 else if ( loop->type == ONEEXPRESSION ) {
03261 /*
03262     Find the dollar expression
03263 */
03264     WORD numdollar = GetDollar(loop->dollarname+1);
03265     DOLLARS d = Dollars + numdollar;
03266     WORD *w, *dw, v, *ww;
03267     if ( (d->where) == 0 ) {
03268         d->type = DOLUNDEFINED;
03269         M_free(loop->dollarname,"do-loop instruction");
03270         goto finish;
03271     }
03272     w = d->where + loop->incnum;
03273     if ( *w == 0 ) {
03274         M_free(d->where,"dollar");
03275         d->where = 0;
03276         d->type = DOLUNDEFINED;
03277         M_free(loop->dollarname,"do-loop instruction");
03278         goto finish;
03279     }
03280     loop->incnum += *w;
03281 /*
03282     Now the term has to be converted to text.
03283 */
03284     ww = w + *w; v = *ww; *ww = 0;
03285     dw = d->where; d->where = w;
03286     t = WriteDollarToBuffer(numdollar,1);
03287     d->where = dw; *ww = v;
03288     PutPreVar(loop->name,t,0,1); /* We overwrite the definition */
03289     M_free(t,"dollar");
03290 }
03291 if ( loop->firstloopcall ) OpenStream(loop->contents,PREREADSTREAM2,0,PRENOACTION);
03292 else OpenStream(loop->contents,PREREADSTREAM,0,PRENOACTION);
03293 AC.CurrentStream->prevline =
03294 AC.CurrentStream->linenumber = loop->startlinenumber;
03295 AC.CurrentStream->eqnum = 0;
03296 loop->firstloopcall = 0;
03297 return(0);
03298 finish:;
03299 NumDoLoops--;
03300 retval = DoUndefine(loop->name);
03301 M_free(loop->p.buffer,"loop->p.buffer");
03302 loop->firstloopcall = 0;
03303 AP.NumPreTypes--;
03304 return(retval);
03305 }
03306 }
03307 /*
03308     #] DoEnddo :
03309     #[ DoEndif :
03310 */
03311 int DoEndif(UBYTE *s)
03312 {
03313     DUMMYUSE(s);
03314     if ( AP.PreTypes[AP.NumPreTypes] != PRETYPEEIF ) {
03315         if ( AP.PreIfLevel <= 0 ) MesPrint("@%#endif without corresponding %#if");
03316         else MesPreNesting(5);
03317         return(-1);
03318     }
03319     AP.NumPreTypes--;
03320     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
03321     if ( AP.PreIfLevel <= 0 ) {
03322         MesPrint("@%#endif without corresponding %#if");
03323     }
03324 }

```

```

03325         return(-1);
03326     }
03327     AP.PreIfLevel--;
03328     return(0);
03329 }
03330
03331 /*
03332     #] DoEndif :
03333     #[ DoEndprocedure :
03334
03335     Action is simple: close the current stream if it is still
03336     the stream from which the statement came.
03337     Then pop the current procedure and all its local derivatives.
03338     if loadmode > 1 the procedure was defined locally.
03339 */
03340
03341 int DoEndprocedure(UBYTE *s)
03342 {
03343     DUMMYUSE(s);
03344     if ( AP.PreTypes[AP.NumPreTypes] != PRETYPEPROCEDURE ) {
03345         MessPreNesting(6);
03346         return(-1);
03347     }
03348     AP.NumPreTypes--;
03349     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
03350     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
03351     AC.CurrentStream = CloseStream(AC.CurrentStream);
03352     do {
03353         NumProcedures--;
03354         if ( Procedures[NumProcedures].loadmode == 0 ) {
03355             M_free(Procedures[NumProcedures].p.buffer, "procedures buffer");
03356             M_free(Procedures[NumProcedures].name, "procedures name");
03357         }
03358     } while ( Procedures[NumProcedures].loadmode > 1 );
03359     return(0);
03360 }
03361
03362 /*
03363     #] DoEndprocedure :
03364     #[ DoIf :
03365 */
03366
03367 int DoIf(UBYTE *s)
03368 {
03369     int condition;
03370     AddToPreTypes(PRETYPEIF);
03371     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
03372     if ( AP.PreIfStack[AP.PreIfLevel] == EXECUTINGIF ) {
03373         condition = EvalPreIf(s);
03374         if ( condition < 0 ) return(-1);
03375     }
03376     else condition = LOOKINGFORENDIF;
03377     if ( AP.PreIfLevel+1 >= AP.MaxPreIfLevel ) {
03378         int **ppp = &AP.PreIfStack; /* To avoid a compiler warning */
03379         if ( DoubleList((void ***)ppp, &AP.MaxPreIfLevel, sizeof(int),
03380             "PreIfLevels") ) return(-1);
03381     }
03382     AP.PreIfStack[++AP.PreIfLevel] = condition;
03383     return(0);
03384 }
03385
03386 /*
03387     #] DoIf :
03388     #[ DoIfdef :
03389 */
03390
03391 int DoIfdef(UBYTE *s, int par)
03392 {
03393     int condition;
03394     AddToPreTypes(PRETYPEIF);
03395     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
03396     if ( AP.PreIfStack[AP.PreIfLevel] == EXECUTINGIF ) {
03397         while ( *s == ' ' || *s == '\t' ) s++;
03398         if ( ( *s == 0 ) == ( par == 1 ) ) condition = LOOKINGFORELSE;
03399         else condition = EXECUTINGIF;
03400     }
03401     else condition = LOOKINGFORENDIF;
03402     if ( AP.PreIfLevel+1 >= AP.MaxPreIfLevel ) {
03403         int **ppp = &AP.PreIfStack; /* to avoid a compiler warning */
03404         if ( DoubleList((void ***)ppp, &AP.MaxPreIfLevel, sizeof(int),
03405             "PreIfLevels") ) return(-1);
03406     }
03407     AP.PreIfStack[++AP.PreIfLevel] = condition;
03408     return(0);
03409 }
03410
03411 /*

```

```

03412         #] DoIfdef :
03413         #[ DoIfydef :
03414     */
03415
03416     int DoIfydef(UBYTE *s)
03417     {
03418         return DoIfdef(s,1);
03419     }
03420
03421     /*
03422         #] DoIfydef :
03423         #[ DoIfndef :
03424     */
03425
03426     int DoIfndef(UBYTE *s)
03427     {
03428         return DoIfdef(s,2);
03429     }
03430
03431     /*
03432         #] DoIfndef :
03433         #[ DoInside :
03434
03435         #inside $var1,...,$varn
03436         statements without .sort
03437         #endinside
03438
03439         executes the statements on the contents of the $ variables as if they
03440         are a module. The results are put back in the dollar variables.
03441         To do this right we need a struct with
03442             old compiler buffer
03443             list of numbers of dollars
03444             length of the list
03445             length of the array containing the list
03446         Because we need to compose statements, the statement buffer must be
03447         empty. This means that we have to test for that. Same at the end. We
03448         must have a completed statement.
03449     */
03450
03451     int DoInside(UBYTE *s)
03452     {
03453         GETIDENTITY
03454         int numdol, error = 0;
03455         WORD *nb, newsize, i;
03456         UBYTE *name, c;
03457         if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
03458         if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
03459         if ( AP.PreInsideLevel != 0 ) {
03460             MesPrint("@Illegal nesting of %#inside/%#endinside instructions");
03461             return(-1);
03462         }
03463     /*
03464         if ( AP.PreContinuation ) {
03465             error = -1;
03466             MesPrint("@%#inside cannot be inside a regular statement");
03467         }
03468     */
03469     PUSHPREASSIGNLEVEL
03470     /*
03471         Now the dollars to do
03472     */
03473     AP.inside.numdollars = 0;
03474     for(;;) {
03475         while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
03476         if ( *s == 0 ) break;
03477         if ( *s != '$' ) {
03478             MesPrint("@%#inside instruction can have only $ variables for parameters");
03479             return(-1);
03480         }
03481         s++;
03482         name = s;
03483         while (chartype[*s] <= 1 ) s++;
03484         c = *s; *s = 0;
03485         if ( ( numdol = GetDollar(name) ) < 0 ) {
03486             MesPrint("@%#inside: $%s has not (yet) been defined",name);
03487             *s = c;
03488             error = -1;
03489         }
03490         else {
03491             *s = c;
03492             if ( AP.inside.numdollars >= AP.inside.size ) {
03493                 if ( AP.inside.buffer == 0 ) newsize = 20;
03494                 else newsize = 2*AP.inside.size;
03495                 nb = (WORD *)Malloca1(newsize*sizeof(WORD),"insidebuffer");
03496                 if ( AP.inside.buffer ) {
03497                     for ( i = 0; i < AP.inside.size; i++ ) nb[i] = AP.inside.buffer[i];
03498                     M_free(AP.inside.buffer,"insidebuffer");

```

```

03499         }
03500         AP.inside.buffer = nb;
03501         AP.inside.size = newsiz;
03502     }
03503     AP.inside.buffer[AP.inside.numdollars++] = numdol;
03504 }
03505 }
03506 /*
03507 We have to store the configuration of the compiler buffer, so that
03508 we know where to start executing and how to reset the buffer.
03509 */
03510 AP.inside.oldcomPILEtype = AC.comPILEtype;
03511 AP.inside.oldparallelflag = AC.mparallelflag;
03512 AP.inside.oldnumpotmoddollars = NumPotModdollars;
03513 AP.inside.olddcbuf = AC.cbunum;
03514 AP.inside.olddrbuf = AM.rbufnum;
03515 AP.inside.olddcnulhs = AR.Cnulhs,
03516 AddToPreTypes(PRETYPEINSIDE);
03517 AP.PreInsideLevel = 1;
03518 AC.cbunum = AP.inside.inscbuf;
03519 AM.rbufnum = AP.inside.inscbuf;
03520 clearcbuf(AC.cbunum);
03521 AC.comPILEtype = 0;
03522 AC.mparallelflag = PARALLELFLAG;
03523 #ifdef WITHMPI
03524 /*
03525 * We use AC.RhsExprInModuleFlag, PotModdollars, and AC.pfirstnum
03526 * in order to check (1) whether there are expression names in RHS,
03527 * (2) which dollar variables can be modified, and (3) which
03528 * preprocessor variables can be redefined, in #inside.
03529 * We store the current values of them, and then reset them.
03530 */
03531 PF_StoreInsideInfo();
03532 AC.RhsExprInModuleFlag = 0;
03533 NumPotModdollars = 0;
03534 AC.numpfirstnum = 0;
03535 #endif
03536 return(error);
03537 }
03538
03539 /*
03540 #] DoInside :
03541 #[ DoEndInside :
03542 */
03543
03544 int DoEndInside(UBYTE *s)
03545 {
03546     GETIDENTITY
03547     WORD numdol, *oldworkpointer = AT.WorkPointer, *term, *t, j, i;
03548     DOLLARS d, nd;
03549     WORD oldbracketon = AR.BraceOn;
03550     WORD *oldcompresspointer = AR.CompressPointer;
03551     int oldmultithreaded = AS.MultiThreaded;
03552     /* int oldmparallelflag = AC.mparallelflag; */
03553     FILEHANDLE *f;
03554     #ifdef WITHMPI
03555     int error = 0;
03556     #endif
03557     DUMMYUSE(s);
03558     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
03559     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
03560     if ( AP.PreTypes[AP.NumPreTypes] != PRETYPEINSIDE ) {
03561         if ( AP.PreInsideLevel != 1 ) MesPrint("@%#endinside without corresponding %#inside");
03562         else MessPreNesting(11);
03563         return(-1);
03564     }
03565     AP.NumPreTypes--;
03566     if ( AP.PreInsideLevel != 1 ) {
03567         MesPrint("@%#endinside without corresponding %#inside");
03568         return(-1);
03569     }
03570     if ( AP.PreContinuation ) {
03571         MesPrint("@%#endinside: previous statement not terminated.");
03572         Terminate(-1);
03573     }
03574     AC.comPILEtype = AP.inside.oldcomPILEtype;
03575     AR.Cnulhs = cbunum[AM.rbufnum].nulhs;
03576     #ifdef WITHMPI
03577     /*
03578     * If the #inside...#endinside contains expressions in RHS, only the master executes it
03579     * and then broadcasts the result to the all slaves. If not, the all processes execute
03580     * it and in this case no MPI interactions are needed.
03581     */
03582     if ( PF.me == MASTER || !AC.RhsExprInModuleFlag ) {
03583     #endif
03584         AR.BraceOn = 0;
03585         AS.MultiThreaded = 0;

```

```

03586      /* AC.mparallelflag = PARALLELFLAG; */
03587      if ( AR.CompressPointer == 0 ) AR.CompressPointer = AR.CompressBuffer;
03588      f = AR.infile; AR.infile = AR.outfile; AR.outfile = f;
03589      /*
03590      Now we have to execute the statements on the proper dollars.
03591      */
03592      for ( i = 0; i < AP.inside.numdollars; i++ ) {
03593          numdol = AP.inside.buffer[i];
03594          nd = d = Dollars + numdol;
03595          if ( d->type != DOLZERO ) {
03596              if ( d->type != DOLTERMS ) nd = DolToTerms(BHEAD numdol);
03597              term = nd->where;
03598              NewSort(BHEAD0);
03599              NewSort(BHEAD0);
03600              AR.MaxDum = AM.IndDum;
03601              while ( *term ) {
03602                  t = oldworkpointer; j = *term;
03603                  NCOPY(t,term,j);
03604                  AT.WorkPointer = t;
03605                  AN.IndDum = AM.IndDum;
03606                  AR.CurDum = ReNumber(BHEAD term);
03607                  if ( Generator(BHEAD oldworkpointer,0) ) {
03608                      MesPrint("@Called from %#endinside");
03609                      MesPrint("@Evaluating variable %s",DOLLARNAME(Dollars,numdol));
03610                      Terminate(-1);
03611                  }
03612              }
03613              AT.WorkPointer = oldworkpointer;
03614              CleanDollarFactors(d);
03615              if ( d->where ) { M_free(d->where,"dollar contents"); d->where = 0; }
03616              EndSort(BHEAD (WORD *) ((void *) (&(d->where))),2);
03617              LowerSortLevel();
03618              term = d->where; while ( *term ) term += *term;
03619              d->size = term - d->where;
03620              if ( nd != d ) M_free(nd,"Copy of dollar variable");
03621              if ( d->where[0] == 0 ) {
03622                  M_free(d->where,"dollar contents"); d->where = 0;
03623                  d->type = DOLZERO;
03624              }
03625          }
03626      }
03627      #ifdef WITHMPI
03628      {
03629          if ( AC.RhsExprInModuleFlag ) {
03630              /*
03631               * The only master executed the statements in #inside.
03632               * We need to broadcast the result to the all slaves.
03633               */
03634              for ( i = 0; i < AP.inside.numdollars; i++ ) {
03635                  /*
03636                   * Mark $-variables specified in the #inside instruction as modified
03637                   * such that they will be broadcast.
03638                   */
03639                  AddPotModddollar(AP.inside.buffer[i]);
03640              }
03641              /* Now actual broadcast of modified variables. */
03642              if ( NumPotModddollars > 0 ) {
03643                  error = PF_BroadcastModifiedDollars();
03644                  if ( error ) goto cleanup;
03645              }
03646              if ( AC.numpfirstnum > 0 ) {
03647                  error = PF_BroadcastRedefinedPreVars();
03648                  if ( error ) goto cleanup;
03649              }
03650          }
03651      cleanup:
03652      #endif
03653      f = AR.infile; AR.infile = AR.outfile; AR.outfile = f;
03654      AC.cbunum = AP.inside.oldcbunum;
03655      AM.rbufnum = AP.inside.olldrbuf;
03656      AR.Cnumlhs = AP.inside.olddcnumlhs;
03657      AR.BracketOn = oldbracketon;
03658      AP.PreInsideLevel = 0;
03659      AR.CompressPointer = oldcompresspointer;
03660      AS.MultiThreaded = oldmultithreaded;
03661      AC.mparallelflag = AP.inside.oldparallelflag;
03662      NumPotModddollars = AP.inside.oldnumpotmodddollars;
03663      POPPREASSIGNLEVEL
03664      #ifdef WITHMPI
03665      PF_RestoreInsideInfo();
03666      if ( error ) return error;
03667      #endif
03668      return(0);
03669      }
03670
03671      /*
03672      #] DoEndInside :

```

```

03673         #] DoMessage :
03674     */
03675
03676 int DoMessage(UBYTE *s)
03677 {
03678     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
03679     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
03680     while ( *s == ' ' || *s == '\t' ) s++;
03681     MesPrint("~~~%s",s);
03682     return(0);
03683 }
03684
03685 /*
03686     #] DoMessage :
03687     #] DoPipe :
03688 */
03689
03690 int DoPipe(UBYTE *s)
03691 {
03692     #ifndef WITHPIPE
03693         DUMMYUSE(s);
03694     #endif
03695     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
03696     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
03697     #ifdef WITHPIPE
03698         FLUSHCONSOLE;
03699         while ( *s == ' ' || *s == '\t' ) s++;
03700         if ( OpenStream(s,PIPESTREAM,0,PREENOACTION) == 0 ) return(-1);
03701         return(0);
03702     #else
03703         Error0("Pipes not implemented on this computer/system");
03704         return(-1);
03705     #endif
03706 }
03707
03708 /*
03709     #] DoPipe :
03710     #] DoProcExtension :
03711 */
03712
03713 int DoProcExtension(UBYTE *s)
03714 {
03715     UBYTE *t, *u, c;
03716     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
03717     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
03718     while ( *s == ' ' || *s == '\t' ) s++;
03719     if ( *s == 0 || *s == '\n' ) {
03720         MesPrint("@No valid procedure extension specified");
03721         return(-1);
03722     }
03723     if ( FG.cTable[*s] != 0 ) {
03724         MesPrint("@Procedure extension should be a string starting with an alphabetic character. No
03725         whitespace.");
03726         return(-1);
03727     }
03728     t = s;
03729     while ( *s && *s != '\n' && *s != ' ' && *s != '\t' ) s++;
03730     u = s;
03731     while ( *s == ' ' || *s == '\t' ) s++;
03732     if ( *s != 0 && *s != '\n' ) {
03733         MesPrint("@Too many parameters in ProcedureExtension instruction");
03734         return(-1);
03735     }
03736     c = *u; *u = 0;
03737     if ( AP.procedureExtension ) M_free(AP.procedureExtension,"ProcedureExtension");
03738     AP.procedureExtension = strDupl(t,"ProcedureExtension");
03739     *u = c;
03740     return(0);
03741 }
03742
03743 /*
03744     #] DoProcExtension :
03745     #] DoPreOut :
03746 */
03747
03748 int DoPreOut(UBYTE *s)
03749 {
03750     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
03751     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
03752     if ( tolower(*s) == 'o' ) {
03753         if ( tolower(s[1]) == 'n' && s[2] == 0 ) {
03754             AP.PreOut = 1;
03755             return(0);
03756         }
03757         if ( tolower(s[1]) == 'f' && tolower(s[2]) == 'f' && s[3] == 0 ) {
03758             AP.PreOut = 0;
03759             return(0);
03760         }
03761     }

```

```

03759     }
03760 }
03761 MesPrint("@Illegal option in PreOut instruction");
03762 return(-1);
03763 }
03764
03765 /*
03766     #] DoPreOut :
03767     #[ DoPrePrintTimes :
03768 */
03769
03770 int DoPrePrintTimes(UBYTE *s)
03771 {
03772     DUMMYUSE(s);
03773     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
03774     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
03775     PrintRunningTime();
03776     return(0);
03777 }
03778
03779 /*
03780     #] DoPrePrintTimes :
03781     #[ DoPreSortReallocate :
03782 */
03783
03784 int DoPreSortReallocate(UBYTE *s)
03785 {
03786     DUMMYUSE(s);
03787     if ( AC.SortReallocateFlag == 0 ) {
03788         /* Currently off, so set to 2. Then the reallocation code knows the flag was
03789            set here, since "On sortreallocate;" sets it to 1. */
03790         AC.SortReallocateFlag = 2;
03791     }
03792     /* If the flag is already on, do nothing. */
03793     return(0);
03794 }
03795
03796 /*
03797     #] DoPreSortReallocate :
03798     #[ DoPreAppend :
03799
03800     Syntax:
03801     #append <filename>
03802 */
03803
03804 int DoPreAppend(UBYTE *s)
03805 {
03806     UBYTE *name, *to;
03807
03808     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
03809     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
03810     if ( AP.preError ) return(0);
03811     while ( *s == ' ' || *s == '\t' ) s++;
03812     /*
03813     Determine where to write
03814     */
03815     if ( *s == '<' ) {
03816         s++;
03817         name = to = s;
03818         while ( *s && *s != '>' ) {
03819             if ( *s == '\\\\' ) s++;
03820             *to++ = *s++;
03821         }
03822         if ( *s == 0 ) {
03823             MesPrint("@Improper termination of filename");
03824             return(-1);
03825         }
03826         s++;
03827         *to = 0;
03828         if ( *name ) { GetAppendChannel((char *)name); }
03829         else goto improper;
03830     }
03831     else {
03832 improper:
03833         MesPrint("@Proper syntax is: %#append <filename>");
03834         return(-1);
03835     }
03836     return(0);
03837 }
03838
03839 /*
03840     #] DoPreAppend :
03841     #[ DoPreCreate :
03842
03843     Syntax:
03844     #create <filename>
03845     */

```

```

03846
03847 int DoPreCreate(UBYTE *s)
03848 {
03849     UBYTE *name, *to;
03850
03851     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
03852     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
03853     if ( AP.preError ) return(0);
03854     while ( *s == ' ' || *s == '\t' ) s++;
03855     /*
03856     Determine where to write
03857     */
03858     if ( *s == '<' ) {
03859         s++;
03860         name = to = s;
03861         while ( *s && *s != '>' ) {
03862             if ( *s == '\\' ) s++;
03863             *to++ = *s++;
03864         }
03865         if ( *s == 0 ) {
03866             MesPrint("@Improper termination of filename");
03867             return(-1);
03868         }
03869         s++;
03870         *to = 0;
03871         if ( *name ) { GetChannel((char *)name,0); }
03872         else goto improper;
03873     }
03874     else {
03875 improper:
03876         MesPrint("@Proper syntax is: %#create <filename>");
03877         return(-1);
03878     }
03879     return(0);
03880 }
03881
03882 /*
03883     #] DoPreCreate :
03884     #[ DoPreRemove :
03885 */
03886
03887 int DoPreRemove(UBYTE *s)
03888 {
03889     UBYTE *name, *to;
03890     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
03891     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
03892     if ( AP.preError ) return(0);
03893     while ( *s == ' ' || *s == '\t' ) s++;
03894     if ( *s == '<' ) { s++; }
03895     else {
03896         MesPrint("@Proper syntax is: %#remove <filename>");
03897         return(-1);
03898     }
03899     name = to = s;
03900     while ( *s && *s != '>' ) {
03901         if ( *s == '\\' ) s++;
03902         *to++ = *s++;
03903     }
03904     if ( *s == 0 ) {
03905         MesPrint("@Improper filename");
03906         return(-1);
03907     }
03908     s++;
03909     *to = 0;
03910     CloseChannel((char *)name);
03911     remove((char *)name);
03912     return(0);
03913 }
03914
03915 /*
03916     #] DoPreRemove :
03917     #[ DoPreClose :
03918 */
03919
03920 int DoPreClose(UBYTE *s)
03921 {
03922     UBYTE *name, *to;
03923     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
03924     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
03925     if ( AP.preError ) return(0);
03926     while ( *s == ' ' || *s == '\t' ) s++;
03927     if ( *s == '<' ) { s++; }
03928     else {
03929         MesPrint("@Proper syntax is: %#close <filename>");
03930         return(-1);
03931     }
03932     name = to = s;

```



```

03933     while ( *s && *s != '>' ) {
03934         if ( *s == '\\\\' ) s++;
03935         *to++ = *s++;
03936     }
03937     if ( *s == 0 ) {
03938         MesPrint("@Improper filename");
03939         return(-1);
03940     }
03941     s++;
03942     *to = 0;
03943     return(CloseChannel((char *)name));
03944 }
03945
03946 /*
03947     #] DoPreClose :
03948     #[ DoPreWrite :
03949
03950     Syntax:
03951     #write [<filename>] "formatstring" [,objects]
03952     The format string can contain the following special objects/codes
03953     \n  newline
03954     \t  tab
03955     \!  if last entry in string: no linefeed at end
03956     \b  put \ in output
03957     %$  $-variable (to be found among the objects)
03958     %e  expression (name to be found among the objects)
03959     %E  expression without ; (name to be found among the objects)
03960     %s  string (to be found among the objects) (with or without "")
03961     %S  subterms (see PrintSubtermList)
03962 */
03963
03964 int DoPreWrite(UBYTE *s)
03965 {
03966     HANDLERS h;
03967
03968     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
03969     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
03970     if ( AP.preError ) return(0);
03971
03972 #ifdef WITHMPI
03973     if ( PF.me != MASTER ) return 0;
03974 #endif
03975
03976     h.oldsilent      = AM.silent;
03977     h.newlogonly     = h.oldlogonly = AM.FileOnlyFlag;
03978     h.newhandle      = h.oldhandle  = AC.LogHandle;
03979     h.oldprinttype   = AO.PrintType;
03980
03981     while ( *s == ' ' || *s == '\\t' ) s++;
03982
03983     /* Determine where to write
03984     */
03985     if ( (s=defineChannel(s,&h))==0 ) return(-1);
03986
03987     return(writeToChannel(WRITEOUT,s,&h));
03988 }
03989
03990 /*
03991     #] DoPreWrite :
03992     #[ DoProcedure :
03993
03994     We have to read this procedure into a buffer.
03995     The only complications are:
03996     1: we have to seek through the file to do this efficiently
03997        the file operations under VMS cannot do this properly
03998        (unless we use the proper ANSI structs?)
03999        This is the reason why we read whole input files under VMS.
04000     2: what to do when the same name is used twice.
04001     Note that we have to do the reading without substitution of
04002     preprocessor variables.
04003 */
04004
04005 int DoProcedure(UBYTE *s)
04006 {
04007     UBYTE c;
04008     PROCEDURE *p;
04009     LONG i;
04010     if ( ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH )
04011         || ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) ) {
04012         if ( PreSkip((UBYTE *) "procedure", (UBYTE *) "endprocedure", 1) ) return(-1);
04013         return(0);
04014     }
04015     p = (PROCEDURE *)FromList(&AP.ProcList);
04016     if ( PreLoad(&(p->p), (UBYTE *) "procedure", (UBYTE *) "endprocedure"
04017         , 1, (char *) "procedure") ) return(-1);
04018
04019     p->loadmode = 2;

```

```

04020     s = p->p.buffer + 10;
04021     while ( *s == ' ' || *s == LINEFEED ) s++;
04022     if ( chartype[*s] ) {
04023         MesPrint("@Illegal name for procedure");
04024         return(-1);
04025     }
04026     p->name = s++;
04027     while ( chartype[*s] == 0 || chartype[*s] == 1 ) s++;
04028     c = *s; *s = 0;
04029     p->name = strdup( p->name, "procedure" );
04030     *s = c;
04031 /*
04032     Check for double names
04033 */
04034     for ( i = NumProcedures-2; i >= 0; i-- ) {
04035         if ( StrCmp(Procedures[i].name,p->name) == 0 ) {
04036             Error1("Multiple occurrence of procedure name ",p->name);
04037         }
04038     }
04039     return(0);
04040 }
04041
04042 /*
04043     #] DoProcedure :
04044     #[ DoPreBreak :
04045 */
04046
04047 int DoPreBreak(UBYTE *s)
04048 {
04049     DUMMYUSE(s);
04050     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
04051     if ( AP.PreTypes[AP.NumPreTypes] != PRETYPESWITCH ) {
04052         if ( AP.PreSwitchLevel <= 0 )
04053             MesPrint("@Break without corresponding Switch");
04054         else MessPreNesting(7);
04055         return(-1);
04056     }
04057     if ( AP.PreSwitchLevel <= 0 ) {
04058         MesPrint("@Break without corresponding Switch");
04059         return(-1);
04060     }
04061     if ( AP.PreSwitchModes[AP.PreSwitchLevel] == EXECUTINGPRESWITCH )
04062         AP.PreSwitchModes[AP.PreSwitchLevel] = SEARCHINGPREENDSWITCH;
04063     return(0);
04064 }
04065
04066 /*
04067     #] DoPreBreak :
04068     #[ DoPreCase :
04069 */
04070
04071 int DoPreCase(UBYTE *s)
04072 {
04073     UBYTE *t;
04074     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
04075     if ( AP.PreTypes[AP.NumPreTypes] != PRETYPESWITCH ) {
04076         if ( AP.PreSwitchLevel <= 0 )
04077             MesPrint("@Case without corresponding Switch");
04078         else MessPreNesting(8);
04079         return(-1);
04080     }
04081     if ( AP.PreSwitchLevel <= 0 ) {
04082         MesPrint("@Case without corresponding Switch");
04083         return(-1);
04084     }
04085     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != SEARCHINGPRECASE ) return(0);
04086
04087     SKIPBLANKS(s)
04088     t = s;
04089     while ( *s ) { if ( *s == '\\\\' ) s++; s++; }
04090     while ( s > t && ( s[-1] == ' ' || s[-1] == '\\t' ) && s[-2] != '\\\\' ) {
04091         if ( s[-2] == '\\\\' ) s--;
04092         s--;
04093     }
04094     if ( *t == '"' && s > t+1 && s[-1] == '"' && s[-2] != '\\\\' ) {
04095         t++; s--; *s = 0;
04096     }
04097     else *s = 0;
04098     s = AP.PreSwitchStrings[AP.PreSwitchLevel];
04099     while ( *t == *s && *t ) { s++; t++; }
04100     if ( *t || *s ) return(0); /* case did not match */
04101     AP.PreSwitchModes[AP.PreSwitchLevel] = EXECUTINGPRESWITCH;
04102     return(0);
04103 }
04104
04105 /*
04106     #] DoPreCase :

```

```

04107         #[ DoPreDefault :
04108     /*
04109
04110 int DoPreDefault(UBYTE *s)
04111 {
04112     DUMMYUSE(s);
04113     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
04114     if ( AP.PreTypes[AP.NumPreTypes] != PRETYPESWITCH ) {
04115         if ( AP.PreSwitchLevel <= 0 )
04116             MesPrint("@Default without corresponding Switch");
04117         else MessPreNesting(9);
04118         return(-1);
04119     }
04120     if ( AP.PreSwitchLevel <= 0 ) {
04121         MesPrint("@Default without corresponding Switch");
04122         return(-1);
04123     }
04124     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != SEARCHINGPRECASE ) return(0);
04125     AP.PreSwitchModes[AP.PreSwitchLevel] = EXECUTINGPRESWITCH;
04126     return(0);
04127 }
04128
04129 /*
04130         #[ DoPreDefault :
04131         #[ DoPreEndSwitch :
04132     /*
04133
04134 int DoPreEndSwitch(UBYTE *s)
04135 {
04136     DUMMYUSE(s);
04137     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
04138     if ( AP.PreTypes[AP.NumPreTypes] != PRETYPESWITCH ) {
04139         if ( AP.PreSwitchLevel <= 0 )
04140             MesPrint("@EndSwitch without corresponding Switch");
04141         else MessPreNesting(10);
04142         return(-1);
04143     }
04144     AP.NumPreTypes--;
04145     if ( AP.PreSwitchLevel <= 0 ) {
04146         MesPrint("@EndSwitch without corresponding Switch");
04147         return(-1);
04148     }
04149     M_free(AP.PreSwitchStrings[AP.PreSwitchLevel--], "pre switch string");
04150     return(0);
04151 }
04152
04153 /*
04154         #[ DoPreEndSwitch :
04155         #[ DoPreSwitch :
04156
04157     There should be a string after this.
04158     We have to store it somewhere.
04159     /*
04160
04161 int DoPreSwitch(UBYTE *s)
04162 {
04163     UBYTE *t, *switchstring, **newstrings;
04164     int newnum, i, *newmodes;
04165     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
04166     SKIPBLANKS(s)
04167     t = s;
04168     while ( *s ) { if ( *s == '\\\\' ) s++; s++; }
04169     while ( s > t && ( s[-1] == ' ' || s[-1] == '\\t' ) && s[-2] != '\\\\' ) {
04170         if ( s[-2] == '\\\\' ) s--;
04171         s--;
04172     }
04173     if ( *t == '"' && s > t+1 && s[-1] == '"' && s[-2] != '\\\\' ) {
04174         t++; s--; *s = 0;
04175     }
04176     else *s = 0;
04177     switchstring = (UBYTE *)Malloc1((s-t)+1, "case string");
04178     s = switchstring;
04179     while ( *t ) {
04180         if ( *t == '\\\\' ) t++;
04181         *s++ = *t++;
04182     }
04183     *s = 0;
04184     if ( AP.PreSwitchLevel >= AP.NumPreSwitchStrings ) {
04185         newnum = 2*AP.NumPreSwitchStrings;
04186         newstrings = (UBYTE **)Malloc1(sizeof(UBYTE *)*(newnum+1), "case strings");
04187         newmodes = (int *)Malloc1(sizeof(int)*(newnum+1), "case strings");
04188         for ( i = 0; i < AP.NumPreSwitchStrings; i++ )
04189             newstrings[i] = AP.PreSwitchStrings[i];
04190         M_free(AP.PreSwitchStrings, "AP.PreSwitchStrings");
04191         for ( i = 0; i <= AP.NumPreSwitchStrings; i++ )
04192             newmodes[i] = AP.PreSwitchModes[i];
04193         M_free(AP.PreSwitchModes, "AP.PreSwitchModes");

```

```

04194     AP.PreSwitchStrings = newstrings;
04195     AP.PreSwitchModes    = newmodes;
04196     AP.NumPreSwitchStrings = newnum;
04197 }
04198 AP.PreSwitchStrings[++AP.PreSwitchLevel] = switchstring;
04199 if ( ( AP.PreSwitchLevel > 1 )
04200     && ( AP.PreSwitchModes[AP.PreSwitchLevel-1] != EXECUTINGPRESWITCH ) )
04201     AP.PreSwitchModes[AP.PreSwitchLevel] = SEARCHINGPREENDSWITCH;
04202 else
04203     AP.PreSwitchModes[AP.PreSwitchLevel] = SEARCHINGPRECASE;
04204 AddToPreTypes(PRETYPEPSWITCH);
04205 return(0);
04206 }
04207
04208 /*
04209     #] DoPreSwitch :
04210     #[ DoPreShow :
04211
04212     Print the contents of the preprocessor variables
04213 */
04214
04215 int DoPreShow(UBYTE *s)
04216 {
04217     int i;
04218     UBYTE *name, c;
04219     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
04220     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
04221     while ( *s == ' ' || *s == '\t' ) s++;
04222     if ( *s == 0 ) {
04223         MesPrint("%#The preprocessor variables:");
04224         for ( i = 0; i < NumPre; i++ ) {
04225             MesPrint("%d: %s = \"%s\"", i, PreVar[i].name, PreVar[i].value);
04226         }
04227     }
04228     else {
04229         while ( *s ) {
04230             name = s; while ( *s && *s != ' ' && *s != '\t' && *s != ',' ) s++;
04231             c = *s; *s = 0;
04232             for ( i = 0; i < NumPre; i++ ) {
04233                 if ( StrCmp(PreVar[i].name, name) == 0 )
04234                     MesPrint("%d: %s = \"%s\"", i, PreVar[i].name, PreVar[i].value);
04235             }
04236             *s = c;
04237             while ( *s == ' ' || *s == '\t' ) s++;
04238         }
04239     }
04240     return(0);
04241 }
04242
04243 /*
04244     #] DoPreShow :
04245     #[ DoSystem :
04246 */
04247
04248 /*
04249 * A macro for translating the contents of `x' into a string after expanding.
04250 */
04251 #define STRINGIFY(x)  STRINGIFY__(x)
04252 #define STRINGIFY__(x) #x
04253
04254 int DoSystem(UBYTE *s)
04255 {
04256     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
04257     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
04258     if ( AP.PreError ) return(0);
04259     #ifdef WITHSYSTEM
04260     FLUSHCONSOLE;
04261     while ( *s == ' ' || *s == '\t' ) s++;
04262     if ( *s == '-' && s[1] == 'e' ) {
04263         LONG err;
04264         UBYTE str[24];
04265         s += 2;
04266         if ( *s != ' ' ) {
04267             MesPrint("@Syntax error in #system command.");
04268             return(-1);
04269         }
04270         while ( *s == ' ' || *s == '\t' ) s++;
04271         err = system((char *)s);
04272         NumToStr(str, err);
04273         PutPreVar((UBYTE *) "SYSTEMERROR_", str, 0, 1);
04274     }
04275     else if ( system((char *)s) ) {
04276         MesPrint("@System call returned with error condition");
04277         Terminate(-1);
04278     }
04279     return(0);
04280 #else

```

```

04281     Error0("External programs not implemented on this computer/system");
04282     return(-1);
04283 #endif
04284 }
04285
04286 /*
04287     #] DoSystem :
04288     #[ PreLoad :
04289
04290     Loads a loop or procedure into a special buffer.
04291     Note: The current instruction is already in the preStart buffer
04292 */
04293
04294 int PreLoad(PRELOAD *p, UBYTE *start, UBYTE *stop, int mode, char *message)
04295 {
04296     UBYTE *s, *t, *top, *newbuffer, c;
04297     LONG i, ppsize, linenum = AC.CurrentStream->linenum;
04298     int size1, size2, level, com=0, last=1, strng = 0;
04299     p->size = AP.pSize;
04300     p->buffer = (UBYTE *)Malloc1(p->size+1,message);
04301     top = p->buffer + p->size - 2;
04302     t = p->buffer; *t++ = '#';
04303     s = start; size1 = size2 = 0;
04304     while ( *s ) { s++; size1++; }
04305     s = stop; while ( *s ) { s++; size2++; }
04306     s = AP.preStart; while ( *s ) *t++ = *s++; *t++ = LINEFEED;
04307     level = 1;
04308     i = 100;
04309     for (;;) {
04310         c = GetInput();
04311         if ( c == ENDOFINPUT ) {
04312             MesPrint("@Missing %s, Should match line %1",stop,linenum);
04313             return(-1);
04314         }
04315         if ( c == AP.ComChar && last == 1 ) com = 1;
04316         if ( c == LINEFEED ) { last = 1; com = 0; }
04317         else last = 0;
04318
04319         if ( ( c == '"' ) && ( com == 0 ) ) { strng ^= 1; }
04320
04321         if ( ( c == '#' ) && ( com == 0 ) ) i = 0;
04322         else i++;
04323
04324         if ( t >= top ) {
04325             ppsize = t - p->buffer;
04326             p->size *= 2;
04327             newbuffer = (UBYTE *)Malloc1(p->size,message);
04328             t = newbuffer; s = p->buffer;
04329             while ( --ppsize >= 0 ) *t++ = *s++;
04330             M_free(p->buffer,"loading do loop");
04331             p->buffer = newbuffer;
04332             top = p->buffer + p->size - 2;
04333         }
04334         *t++ = c;
04335         if ( strng == 0 ) {
04336             if ( ( i == size2 ) && ( com == 0 ) ) {
04337                 *t = 0;
04338                 if ( StrICmp(t-size2,(UBYTE *) (stop)) == 0 ) {
04339                     while ( ( c = GetInput() ) != LINEFEED && c != ENDOFINPUT ) {}
04340                     level--;
04341                     if ( level <= 0 ) break;
04342                     if ( c == ENDOFINPUT ) Error1("Missing #",stop);
04343                     *t++ = LINEFEED; *t = 0; last = 1;
04344                 }
04345             }
04346             if ( ( i == size1 ) && mode && ( com == 0 ) ) {
04347                 *t = 0;
04348                 if ( StrICmp(t-size1,(UBYTE *) (start)) == 0 ) {
04349                     /*
04350                         while ( ( c = GetInput() ) != LINEFEED && c != ENDOFINPUT ) {}
04351                         if ( c == ENDOFINPUT ) Error1("Missing #",stop);
04352                     */
04353                     level++;
04354                 }
04355             }
04356             if ( i == 1 && t[-2] == LINEFEED ) {
04357                 if ( c == '-' ) AC.NoShowInput = 1;
04358                 else if ( c == '+' ) AC.NoShowInput = 0;
04359             }
04360         }
04361     }
04362     *t++ = LINEFEED;
04363     *t = 0;
04364     return(0);
04365 }
04366
04367 /*

```

```

04368         #] PreLoad :
04369         #[ PreSkip :
04370
04371         Skips a loop or procedure.
04372         Note: The current instruction is already in the preStart buffer
04373     */
04374
04375     #define SKIPBUFSIZE 20
04376
04377     int PreSkip(UBYTE *start, UBYTE *stop, int mode)
04378     {
04379         UBYTE *s, *t, buffer[SKIPBUFSIZE+2], c;
04380         LONG i, linenum = AC.CurrentStream->linenum;
04381         int size1, size2, level, com=0, last=1;
04382
04383         t = buffer; *t++ = '#';
04384         s = start; size1 = size2 = 0;
04385         while ( *s ) { s++; size1++; }
04386         s = stop; while ( *s ) { s++; size2++; }
04387         level = 1;
04388         i = 0;
04389         for (;;) {
04390             c = GetInput();
04391             if ( c == ENDOFINPUT ) {
04392                 MesPrint("@Missing %s, Should match line %l",stop,linenum);
04393                 return(-1);
04394             }
04395             if ( c == AP.ComChar && last == 1 ) com = 1;
04396             if ( c == LINEFEED ) { last = 1; com = 0; i = 0; t = buffer; }
04397             else last = 0;
04398             if ( ( c == '#' ) && ( com == 0 ) ) { i = 0; t = buffer; }
04399             else i++;
04400
04401             if ( i < SKIPBUFSIZE ) *t++ = c;
04402             if ( ( i == size2 ) && ( com == 0 ) ) {
04403                 *t = 0;
04404                 if ( StrICmp(t-size2,(UBYTE *) (stop)) == 0 ) {
04405                     while ( ( c = GetInput() ) != LINEFEED && c != ENDOFINPUT ) {}
04406                     level--;
04407                     if ( level <= 0 ) {
04408                         pushbackchar = LINEFEED;
04409                         break;
04410                     }
04411                     if ( c == ENDOFINPUT ) Error1("Missing #",stop);
04412                     i = 0; t = buffer;
04413                 }
04414             }
04415             if ( ( i == size1 ) && mode && ( com == 0 ) ) {
04416                 *t = 0;
04417                 if ( StrICmp(t-size1,(UBYTE *) (start)) == 0 ) {
04418                     while ( ( c = GetInput() ) != LINEFEED && c != ENDOFINPUT ) {}
04419                     level++;
04420                     i = 0; t = buffer;
04421                 }
04422             }
04423         }
04424         return(0);
04425     }
04426
04427     /*
04428         #] PreSkip :
04429         #[ StartPrepro :
04430     */
04431
04432     void StartPrepro(void)
04433     {
04434         int **ppp;
04435         AP.MaxPreIfLevel = 2;
04436         ppp = &AP.PreIfStack;
04437         if ( DoubleList((void ***) ppp, &AP.MaxPreIfLevel, sizeof(int),
04438             "PreIfLevels" ) Terminate(-1);
04439         AP.PreIfLevel = 0; AP.PreIfStack[0] = EXECUTINGIF;
04440
04441         AP.NumPreSwitchStrings = 10;
04442         AP.PreSwitchStrings = (UBYTE **) Malloc1(sizeof(UBYTE *) *
04443             (AP.NumPreSwitchStrings+1), "case strings");
04444         AP.PreSwitchModes = (int *) Malloc1(sizeof(int) *
04445             (AP.NumPreSwitchStrings+1), "case strings");
04446         AP.PreSwitchModes[0] = EXECUTINGPRESWITCH;
04447         AP.PreSwitchLevel = 0;
04448     }
04449
04450     /*
04451         #] StartPrepro :
04452         #[ EvalPreIf :
04453
04454         Evaluates the condition in an if instruction.

```

```

04455         The return value is EXECUTINGIF if the condition is true.
04456         If it is false the returnvalue is LOOKINGFORELSE.
04457         An error gives a return value of -1
04458     */
04459
04460 int EvalPreIf(UBYTE *s)
04461 {
04462     UBYTE *t, *u;
04463     int val;
04464     t = s;
04465     while ( *t ) t++;
04466     *t++ = '\0';
04467     *t = 0;
04468     if ( ( u = PreIfEval(s,&val) ) == 0 ) return(-1);
04469     if ( u < t ) {
04470         MesPrint("@Unmatched parentheses in condition");
04471         return(-1);
04472     }
04473     if ( val ) return(EXECUTINGIF);
04474     else      return(LOOKINGFORELSE);
04475 }
04476
04477 /*
04478     #] EvalPreIf :
04479     #[ PreIfEval :
04480
04481     Used for recursions in the evaluation of a preprocessor if-condition.
04482     It determines whether the contents of () is true or false
04483     (or in error).
04484     The return value is the address of the first character after the
04485     closing parenthesis or null if there is an error.
04486     In value we find true(1) or false(0)
04487     We enter after the opening parenthesis.
04488     There are levels:
04489         0: orlevel:  a || b
04490         1: andlevel: a && b
04491         2: eqlevel:  a == b or a != b or a = b
04492         3: cmplevel: a > b or a >= b or a < b or a <= b or a >~ b etc
04493 */
04494
04495 UBYTE *PreIfEval(UBYTE *s, int *value)
04496 {
04497     int orlevel = 0, andlevel = 0, eqlevel = 0, cmplevel = 0;
04498     int type, val;
04499     LONG val2;
04500     int orte, orval, cmptype, cmpval, eqtype, eqval, andtype, andval;
04501     UBYTE *t, *eqt, *cmpt, c;
04502     int eqop, cmpop;
04503     orte = orval = cmptype = cmpval = eqtype = eqval = andtype = andval = 0;
04504     eqop = cmpop = 0;
04505     eqt = cmpt = 0;
04506     *value = 0;
04507     while ( *s != '\0' ) {
04508         while ( *s == ' ' || *s == '\t' || *s == '\n' || *s == '\r' ) s++;
04509         t = s;
04510         s = pParseObject(s,&type,&val2);
04511         if ( s == 0 ) return(0);
04512         val = val2;
04513         c = *s;
04514         *s++ = 0; /* in case the object is a string without " */
04515         while ( c == ' ' || c == '\t' || c == '\n' || c == '\r' ) {
04516             c = *s; *s++ = 0;
04517         }
04518         if ( *t == '"' ) t++;
04519         switch(c) {
04520             case '|':
04521                 if ( *s != '|' ) goto illoper;
04522                 s++;
04523                 /* fall through */
04524             case ')':
04525                 if ( cmplevel ) {
04526                     if ( type == 0 || cmptype == 0 ) goto illobject;
04527                     val = PreCmp(type,val,t,cmptype,cmpval,cmpt,cmpop);
04528                     type = 0;
04529                     cmplevel = 0;
04530                 }
04531                 if ( eqlevel ) {
04532                     val = PreEq(type,val,t,eqtype,eqval,eqt,eqop);
04533                     type = 0;
04534                     eqlevel = 0;
04535                 }
04536                 if ( andlevel ) {
04537                     if ( andtype != 0 || type != 0 ) goto illobject;
04538                     val &= andval;
04539                     andlevel = 0;
04540                 }
04541                 if ( orlevel ) {

```

```

04542         if ( orte != 0 || type != 0 ) goto illobject;
04543         val |= orte;
04544     }
04545     if ( c == ')' ) {
04546         *value = val;
04547         return(s);
04548     }
04549     orte = 1;
04550     orte = val;
04551     orte = type;
04552     break;
04553 case '&':
04554     if ( *s != '&' ) goto illoper;
04555     s++;
04556     if ( cmplevel ) {
04557         if ( type == 0 || cmptype == 0 ) goto illobject;
04558         val = PreCmp(type, val, t, cmptype, cmpval, cmpt, cmpop);
04559         type = 0;
04560         cmplevel = 0;
04561     }
04562     if ( eqlevel ) {
04563         val = PreEq(type, val, t, eqtype, eqval, eqt, eqop);
04564         type = 0;
04565         eqlevel = 0;
04566     }
04567     if ( andlevel ) {
04568         if ( andtype != 0 || type != 0 ) goto illobject;
04569         val &= andval;
04570     }
04571     andlevel = 1;
04572     andval = val;
04573     andtype = type;
04574     break;
04575 case '!':
04576 case '=':
04577     if ( eqlevel ) goto illorder;
04578     if ( cmplevel ) {
04579         if ( type == 0 || cmptype == 0 ) goto illobject;
04580         val = PreCmp(type, val, t, cmptype, cmpval, cmpt, cmpop);
04581         type = 0;
04582         cmplevel = 0;
04583     }
04584     if ( c == '!' && *s != '=' ) goto illoper;
04585     if ( *s == '=' ) s++;
04586     if ( c == '!' ) eqop = 1;
04587     else eqop = 0;
04588     eqlevel = 1; eqt = t; eqval = val; eqtype = type;
04589     break;
04590 case '>':
04591 case '<':
04592     if ( cmplevel ) goto illorder;
04593     if ( c == '<' ) cmpop = -1;
04594     else cmpop = 1;
04595     cmplevel = 1; cmpt = t; cmpval = val; cmptype = type;
04596     if ( *s == '=' ) {
04597         s++;
04598         if ( *s == '~' ) { s++; cmpop *= 4; }
04599         else cmpop *= 2;
04600     }
04601     else if ( *s == '~' ) { s++; cmpop *= 3; }
04602     break;
04603 default:
04604     goto illoper;
04605 }
04606 }
04607 return(s);
04608 illorder:
04609     MesPrint("@illegal order of operators");
04610     return(0);
04611 illobject:
04612     MesPrint("@illegal object for this operator");
04613     return(0);
04614 illoper:
04615     MesPrint("@illegal operator");
04616     return(0);
04617 }
04618
04619 /*
04620     #] PreIfEval :
04621     #[ PreCmp :
04622 */
04623
04624 int PreCmp(int type, int val, UBYTE *t, int type2, int val2, UBYTE *t2, int cmpop)
04625 {
04626     if ( type == 2 || type2 == 2 || cmpop < -2 || cmpop > 2 ) {
04627         if ( cmpop < 0 && cmpop > -3 ) cmpop -= 2;
04628         if ( cmpop > 0 && cmpop < 3 ) cmpop += 2;
04629     }

```



```

04629         if ( cmpop == 3 ) val = StrCmp(t2,t) > 0;
04630     else if ( cmpop == 4 ) val = StrCmp(t2,t) >= 0;
04631     else if ( cmpop == -3 ) val = StrCmp(t2,t) < 0;
04632     else if ( cmpop == -4 ) val = StrCmp(t2,t) <= 0;
04633 }
04634 else {
04635     if ( cmpop == 1 ) val = ( val2 > val );
04636     else if ( cmpop == 2 ) val = ( val2 >= val );
04637     else if ( cmpop == -1 ) val = ( val2 < val );
04638     else if ( cmpop == -2 ) val = ( val2 <= val );
04639 }
04640 return(val);
04641 }
04642
04643 /*
04644     #] PreCmp :
04645     #[ PreEq :
04646 */
04647
04648 int PreEq(int type, int val, UBYTE *t, int type2, int val2, UBYTE *t2, int eqop)
04649 {
04650     UBYTE str[20];
04651     if ( type == 2 || type2 == 2 ) {
04652         if ( type != 2 ) { NumToStr(str,val); t = str; }
04653         if ( type2 != 2 ) { NumToStr(str,val2); t2 = str; }
04654         if ( eqop == 1 ) val = StrCmp(t,t2) != 0;
04655         else val = StrCmp(t,t2) == 0;
04656     }
04657     else {
04658         if ( eqop ) val = val != val2;
04659         else val = val == val2;
04660     }
04661     return(val);
04662 }
04663
04664 /*
04665     #] PreEq :
04666     #[ pParseObject :
04667
04668     Parses a preprocessor object. We can have:
04669     1: a number (type = 1)
04670     2: a string (type = 2)
04671     3: an expression between parentheses (type = 0)
04672     4: a special function (type = 3)
04673     If the object is not a number, an expression or a special operator
04674     we try to interpret it as a string.
04675 */
04676
04677 UBYTE *pParseObject(UBYTE *s, int *type, LONG *val2)
04678 {
04679     UBYTE *t, c;
04680     int sign, val = 0;
04681     LONG x;
04682     while ( *s == ' ' || *s == '\t' ) s++;
04683     if ( *s == '(' ) {
04684         s++;
04685         while ( *s == ' ' || *s == '\t' || *s == '\n' || *s == '\r' ) s++;
04686         s = PreIfEval(s,&val);
04687         *type = 0;
04688         *val2 = val;
04689         return(s);
04690     }
04691     else if ( *s == '$' && s[1] == '(' ) {
04692         s += 2;
04693         while ( *s == ' ' || *s == '\t' || *s == '\n' || *s == '\r' ) s++;
04694         s = PreIfDollarEval(s,&val);
04695         *type = 0; *val2 = val;
04696         return(s);
04697     }
04698     if ( *s == 0 ) {
04699 illend:
04700         MesPrint("@illegal end of condition");
04701         return(0);
04702     }
04703     if ( *s == '"' ) {
04704         s++;
04705         while ( *s && *s != '"' ) {
04706             if ( *s == '\\' ) s++;
04707             s++;
04708         }
04709         if ( *s == 0 ) goto illend;
04710         else *s = 0;
04711         *type = 2;
04712         s++;
04713
04714         while ( *s == ' ' || *s == '\t' || *s == '\n' || *s == '\r' ) s++;
04715

```

```

04716         return(s);
04717     }
04718     t = s; sign = 1; x = 0;
04719     if ( chartype[*t] == 0 ) { /* Special operators and strings without "" */
04720         do { t++; } while ( chartype[*t] <= 1 );
04721         if ( *t == '(' ) {
04722             WORD ttype;
04723             c = *t; *t = 0;
04724             if ( StrICmp(s, (UBYTE *) "termsin") == 0 ) {
04725                 UBYTE *tt;
04726                 WORD numdol, numexp;
04727                 ttype = 0;
04728 together:
04729                 *t++ = c;
04730                 while ( *t == ' ' || *t == '\t' || *t == '\n' || *t == '\r' ) t++;
04731                 if ( *t == '$' ) {
04732                     t++; tt = t; while ( chartype[*tt] <= 1 ) tt++;
04733                     c = *tt; *tt = 0;
04734                     if ( ( numdol = GetDollar(t) ) > 0 ) {
04735                         *tt = c;
04736                         if ( ttype == 1 ) {
04737                             x = SizeOfDollar(numdol);
04738                         }
04739                         else {
04740                             x = TermsInDollar(numdol);
04741                         }
04742                     }
04743                     else {
04744                         MesPrint("@%s has not (yet) been defined",t);
04745                         *tt = c;
04746                         Terminate(-1);
04747                     }
04748                 }
04749                 else {
04750                     tt = SkipAName(t);
04751                     c = *tt; *tt = 0;
04752                     if ( GetName(AC.exprnames,t,&numexp,NOAUTO) == NAMENOTFOUND ) {
04753                         MesPrint("@%s has not (yet) been defined",t);
04754                         *tt = c;
04755                         Terminate(-1);
04756                     }
04757                     else {
04758                         *tt = c;
04759                         if ( ttype == 1 ) {
04760                             x = SizeOfExpression(numexp);
04761                         }
04762                         else {
04763                             x = TermsInExpression(numexp);
04764                         }
04765                     }
04766                 }
04767                 while ( *tt == ' ' || *tt == '\t'
04768                     || *tt == '\n' || *tt == '\r' ) tt++;
04769                 if ( *tt != ')' ) {
04770                     MesPrint("@Improper use of terms($var) or terms(expr)");
04771                     Terminate(-1);
04772                 }
04773                 *ttype = 3;
04774                 s = tt+1;
04775                 *val2 = x;
04776                 return(s);
04777             }
04778             else if ( StrICmp(s, (UBYTE *) "sizeof") == 0 ) {
04779                 ttype = 1;
04780                 goto together;
04781             }
04782             else if ( StrICmp(s, (UBYTE *) "exists") == 0 ) {
04783                 UBYTE *tt;
04784                 WORD numdol, numexp;
04785                 *t++ = c;
04786                 while ( *t == ' ' || *t == '\t' || *t == '\n' || *t == '\r' ) t++;
04787                 if ( *t == '$' ) {
04788                     t++; tt = t; while ( chartype[*tt] <= 1 ) tt++;
04789                     c = *tt; *tt = 0;
04790                     if ( ( numdol = GetDollar(t) ) >= 0 ) { x = 1; }
04791                     else { x = 0; }
04792                     *tt = c;
04793                 }
04794                 else if ( *t == '"' ) { /* see whether a file exists */
04795                     UBYTE *name, *oldname; /*
04796                     t++; tt = t;
04797                     for (;;) {
04798                         if ( *tt == '\\ ' ) tt++;
04799                         else if ( *tt == '"' ) break;
04800                         tt++;
04801                     }
04802                     c = *tt; *tt = 0;

```

```

04803 /*
04804 Try to open the file. If possible, return 1.
04805 Afterwards close it.
04806 We do have to run through the FORMPATH. Hence we use LocateFile.
04807 This routine may change the name to the full name.
04808
04809 oldname = name = strDupl(t,"name in exists");
04810 x = LocateFile(&name,-1);
04811 */
04812 x = OpenFile((char *)t);
04813 if ( x >= 0 ) {
04814     CloseFile(x);
04815     x = 1;
04816 /*
04817     if ( name != oldname ) M_free(name,"name from LocateFile");
04818 */
04819 }
04820 else x = 0;
04821 /*
04822 M_free(oldname,"name in exists");
04823 */
04824 *tt++ = c;
04825 }
04826 else {
04827     tt = SkipAName(t);
04828     c = *tt; *tt = 0;
04829     if ( GetName(AC.exprnames,t,&numexp,NOAUTO) == NAMENOTFOUND ) { x = 0; }
04830     else { x = 1; }
04831     *tt = c;
04832 }
04833 while ( *tt == ' ' || *tt == '\t'
04834         || *tt == '\n' || *tt == '\r' ) tt++;
04835 if ( *tt != ')' ) {
04836     MesPrint("@Improper use of exists($var) or exists(expr)");
04837     Terminate(-1);
04838 }
04839 *type = 3;
04840 s = tt+1;
04841 *val2 = x;
04842 return(s);
04843 }
04844 else if ( StrICmp(s,(UBYTE *)"isnumerical") == 0 ) {
04845     GETIDENTITY
04846     UBYTE *tt;
04847     WORD numdol, numexp;
04848     *tt++ = c;
04849     while ( *t == ' ' || *t == '\t' || *t == '\n' || *t == '\r' ) t++;
04850     if ( *t == '$' ) {
04851         t++; tt = t; while ( chartype[*tt] <= 1 ) tt++;
04852         c = *tt; *tt = 0;
04853         if ( ( numdol = GetDollar(t) ) < 0 ) {
04854             MesPrint("@$ variable in isnumerical(%s) does not exist",t);
04855             Terminate(-1);
04856         }
04857         x = DolToLong(BHEAD numdol);
04858         if ( AN.ErrorInDollar ) {
04859             DOLLARS d = Dollars + numdol;
04860             x = 0;
04861             if ( d->type == DOLNUMBER || d->type == DOLTERMS ) {
04862                 if ( d->where[0] == 0 ) x = 1;
04863                 else if ( d->where[d->where[0]] == 0 ) {
04864                     if ( ABS(d->where[d->where[0]-1]) == d->where[0]-1 )
04865                         x = 1;
04866                 }
04867             }
04868         }
04869         else x = 1;
04870         *tt = c;
04871     }
04872     else {
04873         tt = SkipAName(t);
04874         c = *tt; *tt = 0;
04875         if ( GetName(AC.exprnames,t,&numexp,NOAUTO) == NAMENOTFOUND ) {
04876             MesPrint("@expression in isnumerical(%s) does not exist",t);
04877             Terminate(-1);
04878         }
04879         x = TermsInExpression(numexp);
04880         if ( x != 1 ) x = 0;
04881         else {
04882             WORD *term = AT.WorkPointer;
04883             if ( GetFirstTerm(term,numexp,1) < 0 ) {
04884                 MesPrint("@error reading expression in isnumerical(%s)",t);
04885                 Terminate(-1);
04886             }
04887             if ( *term == ABS(term[*term-1])+1 ) x = 1;
04888             else x = 0;
04889         }
04890     }
04891 }

```

```

04890         *tt = c;
04891     }
04892     while ( *tt == ' ' || *tt == '\t'
04893           || *tt == '\n' || *tt == '\r' ) tt++;
04894     if ( *tt != ')' ) {
04895         MesPrint("@Improper use of isnumerical($var) or numerical(expr)");
04896         Terminate(-1);
04897     }
04898     *type = 3;
04899     s = tt+1;
04900     *val2 = x;
04901     return(s);
04902 }
04903 else if ( StrICmp(s, (UBYTE *)("maxpowerof")) == 0 ) {
04904     UBYTE *tt;
04905     WORD numsym;
04906     int stype;
04907     *tt++ = c;
04908     while ( *t == ' ' || *t == '\t' || *t == '\n' || *t == '\r' ) t++;
04909     tt = SkipAName(t);
04910     c = *tt; *tt = 0;
04911     if ( ( stype = GetName(AC.varnames,t,&numsym,NOAUTO) ) == NAMENOTFOUND ) {
04912         MesPrint("@%s has not (yet) been defined",t);
04913         *tt = c;
04914         Terminate(-1);
04915     }
04916     else if ( stype != CSYMBOL ) {
04917         MesPrint("@%s should be a symbol",t);
04918         *tt = c;
04919         Terminate(-1);
04920     }
04921     else {
04922         *tt = c;
04923         x = symbols[numsym].maxpower;
04924     }
04925     while ( *tt == ' ' || *tt == '\t'
04926           || *tt == '\n' || *tt == '\r' ) tt++;
04927     if ( *tt != ')' ) {
04928         MesPrint("@Improper use of maxpowerof(symbol)");
04929         Terminate(-1);
04930     }
04931     *type = 3;
04932     s = tt+1;
04933     *val2 = x;
04934     return(s);
04935 }
04936 else if ( StrICmp(s, (UBYTE *)("minpowerof")) == 0 ) {
04937     UBYTE *tt;
04938     WORD numsym;
04939     int stype;
04940     *tt++ = c;
04941     while ( *t == ' ' || *t == '\t' || *t == '\n' || *t == '\r' ) t++;
04942     tt = SkipAName(t);
04943     c = *tt; *tt = 0;
04944     if ( ( stype = GetName(AC.varnames,t,&numsym,NOAUTO) ) == NAMENOTFOUND ) {
04945         MesPrint("@%s has not (yet) been defined",t);
04946         *tt = c;
04947         Terminate(-1);
04948     }
04949     else if ( stype != CSYMBOL ) {
04950         MesPrint("@%s should be a symbol",t);
04951         *tt = c;
04952         Terminate(-1);
04953     }
04954     else {
04955         *tt = c;
04956         x = symbols[numsym].minpower;
04957     }
04958     while ( *tt == ' ' || *tt == '\t'
04959           || *tt == '\n' || *tt == '\r' ) tt++;
04960     if ( *tt != ')' ) {
04961         MesPrint("@Improper use of minpowerof(symbol)");
04962         Terminate(-1);
04963     }
04964     *type = 3;
04965     s = tt+1;
04966     *val2 = x;
04967     return(s);
04968 }
04969 else if ( StrICmp(s, (UBYTE *)("isfactorized")) == 0 ) {
04970     UBYTE *tt;
04971     WORD numdol, numexp;
04972     *tt++ = c;
04973     while ( *t == ' ' || *t == '\t' || *t == '\n' || *t == '\r' ) t++;
04974     if ( *t == '$' ) {
04975         t++; tt = t; while (chartype[*tt] <= 1 ) tt++;
04976         c = *tt; *tt = 0;

```

```

04977         if ( ( numdol = GetDollar(t) ) > 0 ) {
04978             if ( Dollars[numdol].factors != 0 ) x = 1;
04979             else x = 0;
04980         }
04981         else {
04982             MesPrint("@ %s should be the name of an expression or a $ variable",t-1);
04983             Terminate(-1);
04984         }
04985         *tt = c;
04986     }
04987     else {
04988         tt = SkipAName(t);
04989         c = *tt; *tt = 0;
04990         if ( GetName(AC.exprnames,t,&numexp,NOAUTO) == NAMENOTFOUND ) {
04991             MesPrint("@ %s should be the name of an expression or a $ variable",t);
04992             Terminate(-1);
04993         }
04994         else {
04995             if ( ( Expressions[numexp].vflags & ISFACTORIZED ) != 0 ) x = 1;
04996             else x = 0;
04997         }
04998         *tt = c;
04999     }
05000     while ( *tt == ' ' || *tt == '\t'
05001             || *tt == '\n' || *tt == '\r' ) tt++;
05002     if ( *tt != ')' ) {
05003         MesPrint("@Improper use of isfactorized($var) or isfactorized(expr)");
05004         Terminate(-1);
05005     }
05006     *type = 3;
05007     s = tt+1;
05008     *val2 = x;
05009     return(s);
05010 }
05011 else if ( StrICmp(s,(UBYTE *)"isdefined") == 0 ) {
05012     UBYTE *tt;
05013     *t++ = c;
05014     while ( *t == ' ' || *t == '\t' || *t == '\n' || *t == '\r' ) t++;
05015     tt = SkipAName(t);
05016     c = *tt; *tt = 0;
05017     if ( GetPreVar(t,WITHOUTERROR) != 0 ) x = 1;
05018     else x = 0;
05019     *tt = c;
05020     while ( *tt == ' ' || *tt == '\t'
05021             || *tt == '\n' || *tt == '\r' ) tt++;
05022     if ( *tt != ')' ) {
05023         MesPrint("@Improper use of isdefined(var)");
05024         Terminate(-1);
05025     }
05026     *type = 3;
05027     s = tt+1;
05028     *val2 = x;
05029     return(s);
05030 }
05031 else if ( StrICmp(s,(UBYTE *)"flag") == 0 ) {
05032     UBYTE *tt;
05033     WORD x = 0, numexp;
05034     {
05035         *t++ = c;
05036         while ( *t == ' ' || *t == '\t' ) t++;
05037         if ( FG.cTable[*t] != 1 ) goto flagerror;
05038         while ( FG.cTable[*t] == 1 ) x = 10*x + (*t++ - '0');
05039         if ( x < 1 || x > BITSINWORD ) {
05040             MesPrint("@Illegal number %d for flag in flag condition",x);
05041             goto flagerror;
05042         }
05043         while ( *t == ' ' || *t == '\t' ) t++;
05044         if ( *t != ',' ) goto flagerror;
05045         t++;
05046         while ( *t == ' ' || *t == '\t' ) t++;
05047         tt = SkipAName(t);
05048         c = *tt; *tt = 0;
05049         if ( GetName(AC.exprnames,t,&numexp,NOAUTO) == NAMENOTFOUND ) {
05050             MesPrint("@ %s should be the name of an expression",t);
05051             goto flagerror;
05052         }
05053         *tt = c;
05054         while ( *t == ' ' || *t == '\t' ) t++;
05055         if ( *tt != ')' ) {
05056 flagerror:
05057             MesPrint("@Improper use of flag(num,expr)");
05058             Terminate(-1);
05059             return(0);
05060         }
05061         if ( ( Expressions[numexp].uflags & ( 1 << (x-1) ) ) != 0 )
05062             *val2 = 1;
05063         else *val2 = 0;

```

```

05064         *type = 3;
05065         s = tt+1;
05066         return(s);
05067     }
05068 }
05069     else *t = c;
05070 }
05071     else if ( *t == '=' || *t == '<' || *t == '>' || *t == '!'
05072 || *t == ')' || *t == ' ' || *t == '\\t' || *t == 0 || *t == '\\n' ) {
05073     *val2 = 0;
05074     *type = 2;
05075     return(t);
05076 }
05077     else {
05078         MesPrint("@Illegal use of string in preprocessor condition: %s",s);
05079         Terminate(-1);
05080     }
05081 }
05082 while ( *t == '-' || *t == '+' || *t == ' ' || *t == '\\t' ) {
05083     if ( *t == '-' ) sign = -sign;
05084     t++;
05085 }
05086 while ( chartype[*t] == 1 ) { x = 10*x + *t++ - '0'; }
05087 while ( *t == ' ' || *t == '\\t' ) t++;
05088 if ( chartype[*t] == 8 || *t == ')' || *t == '=' || *t == 0 ) {
05089     *val2 = sign > 0 ? x: -x;
05090     *type = 1;
05091     return(t);
05092 }
05093 while ( chartype[*t] != 8 && *t != ')' && *t != '=' && *t ) t++;
05094 while ( ( t > s ) && ( t[-1] == ' ' || t[-1] == '\\t' ) ) t--;
05095 *type = 2;
05096 *val2 = val;
05097 return(t);
05098 }
05099
05100 /*
05101     #] pParseObject :
05102     #[ PreCalc :
05103
05104     To be called when a { is encountered.
05105     Action: read first till matching }. This is to be stored.
05106     Next we look whether this is a set or whether it can be
05107     evaluated. If it is a set we consider it as a new stream.
05108     The stream will have to be deallocated when read completely.
05109     If it is to be evaluated we do that and put the result in
05110     a stream.
05111 */
05112
05113 UBYTE *PreCalc(void)
05114 {
05115     UBYTE *buff, *s = 0, *t, *newb, c;
05116     int size, i, n, parlevel = 0, bralevel = 0;
05117     LONG answer;
05118     ULONG uanswer;
05119     size = n = 0;
05120     buff = 0; c = '{';
05121     for (;;) {
05122         if ( n >= size ) {
05123             if ( size == 0 ) size = 72;
05124             else size *= 2;
05125             if ( ( newb = (UBYTE *)Malloc1(size+2,"{ }") ) == 0 ) return(0);
05126             s = newb;
05127             if ( buff ) {
05128                 i = n;
05129                 t = buff;
05130                 NCOPYB(s,t,i);
05131                 M_free(buff,"pre calc buffer");
05132             }
05133             else s = newb;
05134             buff = newb;
05135         }
05136         *s++ = c; n++;
05137         c = GetChar(0);
05138         if ( c == 0 ) {
05139             Error0("Unmatched {}");
05140             M_free(buff,"precalc buffer");
05141             return(0);
05142         }
05143         else if ( c == '{' ) { bralevel++; }
05144         else if ( c == '}' ) {
05145             if ( --bralevel < 0 ) { *s++ = c; *s = 0; break; }
05146         }
05147         else if ( c == '(' ) { parlevel++; }
05148         else if ( c == ')' ) {
05149             if ( --parlevel < 0 ) { *s++ = c; *s = 0; goto setstring; }
05150         }
05151     }

```

```

05151         else if ( chartype[c] != 1 && chartype[c] != 5
05152             && chartype[c] != 6 && c != '!' && c != '&'
05153             && c != '|' && c != '\\') { *s++ = c; *s = 0; goto setstring; }
05154     }
05155     if ( parlevel > 0 ) goto setstring;
05156 /*
05157     Try now to evaluate the string.
05158     If it works, copy the resulting value back into buff as a string.
05159 */
05160     answer = 0;
05161     if ( PreEval(buff+1,&answer) == 0 ) goto setstring;
05162     t = buff + size;
05163     s = buff;
05164     if ( answer < 0 ) { *s++ = '-'; }
05165     uanswer = LongAbs(answer);
05166     n = 0;
05167     do {
05168         *--t = ( uanswer % 10 ) + '0';
05169         uanswer /= 10;
05170         n++;
05171     } while ( uanswer > 0 );
05172     NCOPYB(s,t,n);
05173     *s = 0;
05174     setstring;
05175 /*
05176     Open a stream that contains the current string.
05177     Mark it to be removed after termination.
05178 */
05179     if ( OpenStream(buff,PRECALCSTREAM,0,PRENOACTION) == 0 ) return(0);
05180     return(buff);
05181 }
05182
05183 /*
05184     #] PreCalc :
05185     #[ PreEval :
05186
05187     Operations are:
05188     +, -, *, /, %, &, |, ^, !, ^% (postfix 2log), ^/ (postfix sqrt)
05189 */
05190
05191 UBYTE *PreEval(UBYTE *s, LONG *x)
05192 {
05193     LONG y, z, a;
05194     int tobemultiplied, tobeadded = 1, expsign, i;
05195     UBYTE *t;
05196     *x = 0; a = 1;
05197     while ( *s == ' ' || *s == '\t' ) s++;
05198     for(;;){
05199         if ( *s == '+' || *s == '-' ) {
05200             if ( *s == '-' ) tobeadded = -1;
05201             else tobeadded = 1;
05202             s++;
05203             while ( *s == '-' || *s == '+' || *s == ' ' || *s == '\t' ) {
05204                 if ( *s == '-' ) tobeadded = -tobeadded;
05205                 s++;
05206             }
05207         }
05208         tobemultiplied = 0;
05209         for(;;){
05210             while ( *s == ' ' || *s == '\t' ) s++;
05211             if ( *s <= '9' && *s >= '0' ) {
05212                 ULONG uy;
05213                 ParseNumber(uy,s)
05214                 y = uy; /* may cause an implementation-defined behaviour */
05215             }
05216             else if ( *s == '(' || *s == '{' ) {
05217                 if ( ( t = PreEval(s+1,&y) ) == 0 ) return(0);
05218                 s = t;
05219             }
05220             else return(0);
05221             while ( *s == ' ' || *s == '\t' ) s++;
05222             expsign = 1;
05223             while ( *s == '^' || *s == '!' ) {
05224                 s++;
05225                 if ( s[-1] == '!' ) { /* factorial of course */
05226                     while ( *s == ' ' || *s == '\t' ) s++;
05227                     if ( y < 0 ) {
05228                         MesPrint("@Negative value in preprocessor factorial: %1",y);
05229                         return(0);
05230                     }
05231                     else if ( y == 0 ) y = 1;
05232                     else if ( y > 1 ) {
05233                         z = y-1;
05234                         while ( z > 0 ) { y = y*z; z--; }
05235                     }
05236                     continue;
05237                 }

```

```

05238     else if ( *s == '%' ) { /* ^% is postfix 2log */
05239         s++;
05240         while ( *s == ' ' || *s == '\t' ) s++;
05241         z = y;
05242         if ( z <= 0 ) {
05243             MesPrint("@Illegal value in preprocessor logarithm: %1",z);
05244             return(0);
05245         }
05246         y = 0; z >= 1;
05247         while ( z ) { y++; z >= 1; }
05248         continue;
05249     }
05250     else if ( *s == '/' ) { /* ^/ is postfix sqrt */
05251         LONG yy, zz;
05252         s++;
05253         while ( *s == ' ' || *s == '\t' ) s++;
05254         z = y;
05255         if ( z <= 0 ) {
05256             MesPrint("@Illegal value in preprocessor square root: %1",z);
05257             return(0);
05258         }
05259         if ( z > 8 ) { /* Very crude integer square root */
05260             zz = z;
05261             yy = 0; zz >= 1;
05262             while ( zz ) { yy++; zz >= 1; }
05263             zz = z > (yy/2); i = 10; y = 0;
05264             do {
05265                 yy = zz/2 + z/(2*zz); i--;
05266                 if ( y == yy ) break;
05267                 y = zz; zz = yy;
05268             } while ( y != yy && i > 0 );
05269             while ( y*y < z ) y++;
05270             while ( y*y > z ) y--;
05271         }
05272         else if ( z >= 4 ) y = 2;
05273         else if ( z == 0 ) y = 0;
05274         else y = 1;
05275         continue;
05276     }
05277     while ( *s == ' ' || *s == '\t' ) s++;
05278     while ( *s == '-' || *s == '+' || *s == ' ' || *s == '\t' ) {
05279         if ( *s == '-' ) expsign = -expsign;
05280     }
05281     if ( *s <= '9' && *s >= '0' ) {
05282         ParseNumber(z,s)
05283     }
05284     else if ( *s == '(' || *s == '{' ) {
05285         if ( ( t = PreEval(s+1,&z) ) == 0 ) return(0);
05286         s = t;
05287     }
05288     else return(0);
05289     while ( *s == ' ' || *s == '\t' ) s++;
05290     y = iexp(y,(int)z);
05291 }
05292 if ( tobemultiplied == 0 ) {
05293     if ( expsign < 0 ) a = 1/y;
05294     else a = y;
05295 }
05296 else {
05297     if ( tobemultiplied > 2 && expsign != 1 ) {
05298         MesPrint("&Incorrect use of ^ with & or |. Use brackets!");
05299         Terminate(-1);
05300     }
05301     tobemultiplied *= expsign;
05302     if ( tobemultiplied == 1 ) a *= y;
05303     else if ( tobemultiplied == 3 ) a &= y;
05304     else if ( tobemultiplied == 4 ) a |= y;
05305     else {
05306         if ( y == 0 || tobemultiplied == -2 ) {
05307             MesPrint("@Division by zero in preprocessor calculator");
05308             Terminate(-1);
05309         }
05310         if ( tobemultiplied == 2 ) a %= y;
05311         else a /= y;
05312     }
05313 }
05314 if ( *s == '%' ) tobemultiplied = 2;
05315 else if ( *s == '*' ) tobemultiplied = 1;
05316 else if ( *s == '/' ) tobemultiplied = -1;
05317 else if ( *s == '&' ) tobemultiplied = 3;
05318 else if ( *s == '|' ) tobemultiplied = 4;
05319 else {
05320     ULONG ux, ua;
05321     ux = *x;
05322     ua = a;
05323     if ( tobeadded >= 0 ) ux += ua;
05324     else ux -= ua;

```



```

05325         *x = ULongToLong(ux);
05326         if ( *s == ')' || *s == '}' ) return(s+1);
05327         else if ( *s == '-' || *s == '+' ) { tobeadded = 1; break; }
05328         else return(0);
05329     }
05330     s++;
05331 }
05332 }
05333 /* return(0); */
05334 }
05335
05336 /*
05337     #] PreEval :
05338     #[ AddToPreTypes :
05339 */
05340
05341 void AddToPreTypes(int type)
05342 {
05343     if ( AP.NumPreTypes >= AP.MaxPreTypes ) {
05344         int i, *newlist = (int *)Malloca(sizeof(int)*(2*AP.MaxPreTypes+1)
05345             , "preprocessor type lists");
05346         for ( i = 0; i <= AP.MaxPreTypes; i++ ) newlist[i] = AP.PreTypes[i];
05347         M_free(AP.PreTypes, "preprocessor type lists");
05348         AP.PreTypes = newlist;
05349         AP.MaxPreTypes = 2*AP.MaxPreTypes;
05350     }
05351     AP.PreTypes[++AP.NumPreTypes] = type;
05352 }
05353
05354 /*
05355     #] AddToPreTypes :
05356     #[ MessPreNesting :
05357 */
05358
05359 void MessPreNesting(int par)
05360 {
05361     MesPrint("@(%d)Illegal nesting of %sif, %sdo, %sprocedure and/or %sswitch",par);
05362 }
05363
05364 /*
05365     #] MessPreNesting :
05366     #[ DoPreAddSeparator :
05367
05368     Preprocessor directives "addseparator" and "rmseparator" add/remove
05369     separator characters used to separate function arguments.
05370     Example:
05371
05372         #define QQ "a|g|a"
05373         #addseparator %
05374         *Comma must be quoted!:
05375         #rmseparator ",",
05376         #rmseparator |
05377         #call H(a,a`QQ')
05378
05379     Characters ' ', '\t' and '"' are ignored!
05380 */
05381
05382 int DoPreAddSeparator(UBYTE *s)
05383 {
05384     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
05385     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
05386     for (; *s != '\0'; s++) {
05387         while ( *s == ' ' || *s == '\t' || *s == '"' ) s++;
05388         /* Todo:
05389         if ( set_in(*s, invalidseparators) ) {
05390             MesPrint("@Invalid separator specified");
05391             return(-1);
05392         }
05393         */
05394         set_set(*s, AC.separators);
05395     }
05396     return(0);
05397 }
05398
05399 /*
05400     #] DoPreAddSeparator :
05401     #[ DoPreRmSeparator :
05402
05403     See commentary with DoPreAddSeparator
05404
05405     Characters ' ', '\t' and '"' are ignored!
05406 */
05407 int DoPreRmSeparator(UBYTE *s)
05408 {
05409     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
05410     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
05411     for (; *s != '\0'; s++) {

```

```

05412         while ( *s == ' ' || *s == '\t' || *s == '"' ) s++;
05413         set_del(*s,AC.separators);
05414     }
05415     return(0);
05416 }
05417
05418 /*
05419     #] DoPreRmSeparator :
05420     #[ DoExternal:
05421
05422     #external ["prevar"] command
05423 */
05424 int DoExternal(UBYTE *s)
05425 {
05426     #ifdef WITHEXTERNALCHANNEL
05427         UBYTE *prevar=0;
05428         int externalD= 0;
05429     #else
05430         DUMMYUSE(s);
05431     #endif
05432     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
05433     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
05434     if ( AP.preError ) return(0);
05435
05436     #ifdef WITHEXTERNALCHANNEL
05437         while ( *s == ' ' || *s == '\t' ) s++;
05438         if(*s == '"'){/*prevar to store the descriptor is defined*/
05439             prevar++;
05440
05441             if ( chartye[*s] == 0 )for(;*s != '"'; s++)switch(chartye[*s]){
05442                 case 10:/*'\0' fits here*/
05443                     MesPrint("@Can't finde closing \"");
05444                     Terminate(-1);
05445                 case 0:case 1: continue;
05446                 default:
05447                     break;
05448             }
05449             if(*s != '"'){
05450                 MesPrint("@Illegal name of preprocessor variable to store external channel");
05451                 return(-1);
05452             }
05453             *s='\0';
05454             for(s++; *s == ' ' || *s == '\t'; s++);
05455         }
05456
05457         if(*s == '\0'){
05458             MesPrint("@Illegal external command");
05459             return(-1);
05460         }
05461         /*here s is a command*/
05462         /*See the file extcmd.c*/
05463         /*[08may2006 mt]:*/
05464         externalD=openExternalChannel(
05465             s,
05466             AX.daemonize,
05467             AX.shellname,
05468             AX.stderrname);
05469         /*:[08may2006 mt]*/
05470         if(externalD<1){/*error?*/
05471             /*Not quite correct - terminate the program on error:*/
05472             Error1("Can't start external program",s);
05473             return(-1);
05474         }
05475         /*Now external command runs.*/
05476
05477         if(prevar){/*Store the external channel descriptor in the provided variable:*/
05478             UBYTE buf[21];/* 64/Log_2[10] = 19.3, so this is enough forever...*/
05479             NumToStr(buf,externalD);
05480             if ( PutPreVar(prevar,buf,0,1) < 0 ) return(-1);
05481         }
05482
05483         AX.currentExternalChannel=externalD;
05484         /*[08may2006 mt]:*/
05485         if(AX.currentPrompt!=0){/*Change default terminator*/
05486             if(setTerminatorForExternalChannel( (char *)AX.currentPrompt)){
05487                 MesPrint("@Prompt is too long");
05488                 return(-1);
05489             }
05490         }
05491         setKillModeForExternalChannel(AX.killSignal,AX.killWholeGroup);
05492         /*:[08may2006 mt]*/
05493         return(0);
05494     #else /*ifndef WITHEXTERNALCHANNEL*/
05495         Error0("External channel: not implemented on this computer/system");
05496         return(-1);
05497     #endif /*ifndef WITHEXTERNALCHANNEL ... else*/
05498 }

```

```

05499
05500 /*
05501     #] DoExternal:
05502     #[ DoPrompt:
05503         #prompt string
05504 */
05505
05506 int DoPrompt(UBYTE *s)
05507 {
05508     #ifndef WITHEXTERNALCHANNEL
05509         DUMMYUSE(s);
05510     #endif
05511     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
05512     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
05513
05514     #ifdef WITHEXTERNALCHANNEL
05515         while ( *s == ' ' || *s == '\t' ) s++;
05516         if ( AX.currentPrompt )
05517             M_free(AX.currentPrompt,"external channel prompt");
05518         if ( *s == '\0' )
05519             AX.currentPrompt = (UBYTE *)strDupl((UBYTE *)"", "external channel prompt");
05520         else
05521             AX.currentPrompt = strDupl(s, "external channel prompt");
05522         if( setTerminatorForExternalChannel( (char *)AX.currentPrompt) > 0 ){
05523             MesPrint("@Prompt is too long");
05524             return(-1);
05525         }
05526         /*else: if 0, ok; if -1, there is no current channel-ok, just prompt is stored.*/
05527         return(0);
05528     #else /*ifdef WITHEXTERNALCHANNEL*/
05529         Error0("External channel: not implemented on this computer/system");
05530         return(-1);
05531     #endif /*ifdef WITHEXTERNALCHANNEL ... else*/
05532 }
05533 /*
05534     #] DoPrompt:
05535     #[ DoSetExternal:
05536         #setexternal n
05537 */
05538
05539 int DoSetExternal(UBYTE *s)
05540 {
05541     #ifdef WITHEXTERNALCHANNEL
05542         int n=0;
05543     #else
05544         DUMMYUSE(s);
05545     #endif
05546     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
05547     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
05548     if ( AP.preError ) return(0);
05549
05550     #ifdef WITHEXTERNALCHANNEL
05551         while ( *s == ' ' || *s == '\t' ) s++;
05552         while ( charType[*s] == 1 ) { n = 10*n + *s++ - '0'; }
05553         while ( *s == ' ' || *s == '\t' ) s++;
05554         if(*s!='\0'){
05555             MesPrint("@setexternal: number expected");
05556             return(-1);
05557         }
05558         if(selectExternalChannel(n)<0){
05559             MesPrint("@setexternal: invalid number");
05560             return(-1);
05561         }
05562         AX.currentExternalChannel=n;
05563         return(0);
05564     #else /*ifdef WITHEXTERNALCHANNEL*/
05565         Error0("External channel: not implemented on this computer/system");
05566         return(-1);
05567     #endif /*ifdef WITHEXTERNALCHANNEL ... else*/
05568 }
05569 /*
05570     #] DoSetExternal:
05571     #[ DoSetExternalAttr:
05572 */
05573
05574 static FORM_INLINE UBYTE *pickupword(UBYTE *s)
05575 {
05576
05577     for(;*s>' ';s++)switch(*s){
05578         case '=':
05579         case ',':
05580         case ';':
05581             return(s);
05582     }/*for(;*s>' ';s++)switch(*s)*/
05583     return(s);
05584 }
05585 /*Returns 0 if the first string (case insensitively) equal to

```

```

05586     the beginning of the second string (of length n):
05587     */
05588     static inline int strINComp(UBYTE *a, UBYTE *b, int n)
05589     {
05590         for (;n>0;n--) if (tolower(*a++)!=tolower(*b++))
05591             return(1);
05592         return(*a != '\0');
05593     }
05594
05595     #define KILL "kill"
05596     #define KILLALL "killall"
05597     #define DAEMON "daemon"
05598     #define SHELL "shell"
05599     #define STDERR "stderr"
05600
05601     #define TRUE_EXPR "true"
05602     #define FALSE_EXPR "false"
05603     #define NOSHELL "noshell"
05604     #define TERMINAL "terminal"
05605
05606     /*
05607         Expects comma-separated list of pairs name=value
05608     */
05609     int DoSetExternalAttr(UBYTE *s)
05610     {
05611         #ifdef WITHEXTERNALCHANNEL
05612             int lnam,lval;
05613             UBYTE *nam,*val;
05614             #else
05615                 DUMMYUSE(s);
05616             #endif
05617             if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
05618             if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
05619             if ( AP.preError ) return(0);
05620
05621             #ifdef WITHEXTERNALCHANNEL
05622                 do{
05623                     /*Read the name:*/
05624                     while ( *s == ' ' || *s == '\t' ) s++;
05625                     s=pickupword(nam=s);
05626                     lnam=s-nam;
05627                     while ( *s == ' ' || *s == '\t' ) s++;
05628                     if (*s++!='='){
05629                         MesPrint("@External channel:'=' expected instead of %s",s-1);
05630                         return(-1);
05631                     }
05632                     /*Read the value:*/
05633                     while ( *s == ' ' || *s == '\t' ) s++;
05634                     val=s;
05635
05636                     for(;;){
05637                         UBYTE *m;
05638                         s=pickupword(s);
05639                         m=s;
05640                         while ( *s == ' ' || *s == '\t' ) s++;
05641                         if ( (*s == ',') || (*s == '\n') || (*s == ';') || (*s == '\0') ){
05642                             s=m;
05643                             break;
05644                         }
05645                     } /*for(;;)*/
05646
05647                     lval=s-val;
05648                     while ( *s == ' ' || *s == '\t' ) s++;
05649
05650                     if(strINComp((UBYTE *)SHELL,nam,lnam)==0){
05651                         if (AX.shellname!=NULL)
05652                             M_free(AX.shellname,"external channel shellname");
05653                         if(strINComp((UBYTE *)NOSHELL,val,lval)==0)
05654                             AX.shellname=NULL;
05655                         else{
05656                             UBYTE *ch,*b;
05657                             b=ch=AX.shellname=Malloc1(lval+1,"external channel shellname");
05658                             while(ch-b<lval)
05659                                 *ch++=*val++;
05660                             *ch='\0';
05661                         }
05662                     } else if(strINComp((UBYTE *)DAEMON,nam,lnam)==0){
05663                         if(strINComp((UBYTE *)TRUE_EXPR,val,lval)==0)
05664                             AX.daemonize = 1;
05665                         else if(strINComp((UBYTE *)FALSE_EXPR,val,lval)==0)
05666                             AX.daemonize = 0;
05667                         else{
05668                             MesPrint("@External channel:true or false expected for %s",DAEMON);
05669                             return(-1);
05670                         }
05671                     } else if(strINComp((UBYTE *)KILLALL,nam,lnam)==0){
05672                         if(strINComp((UBYTE *)TRUE_EXPR,val,lval)==0)

```

```

05673         AX.killWholeGroup = 1;
05674     else if(strINCMp((UBYTE *)FALSE_EXPR,val,lval)==0)
05675         AX.killWholeGroup = 0;
05676     else{
05677         MesPrint("@External channel: true or false expected for %s",KILLALL);
05678         return(-1);
05679     }
05680 }else if(strINCMp((UBYTE *)KILL,nam,lnam)==0){
05681     int i,n=0;
05682     for(i=0;i<lval;i++) {
05683         if( *val>='0' && *val<= '9' )
05684             n = 10*n + *val++ - '0';
05685         else{
05686             MesPrint("@External channel: number expected for %s",KILL);
05687             return(-1);
05688         }
05689     }
05690     AX.killSignal=n;
05691 }else if(strINCMp((UBYTE *)STDERR,nam,lnam)==0){
05692     if( AX.stdername != NULL ) {
05693         M_free(AX.stdername,"external channel stderrname");
05694     }
05695     if(strINCMp((UBYTE *)TERMINAL,val,lval)==0)
05696         AX.stdername = NULL;
05697     else{
05698         UBYTE *ch,*b;
05699         b=ch=AX.stdername=Malloc1(lval+1,"external channel stderrname");
05700         while(ch-b<lval)
05701             *ch++=*val++;
05702         *ch='\0';
05703     }
05704 }else{
05705     nam[lnam+1]='\0';
05706     MesPrint("@External channel: unrecognized attribute",nam);
05707     return(-1);
05708 }
05709 }while(*s++ == ' ');
05710 if( (*s-1)>' ' && (*s-1)!=';' ) {
05711     MesPrint("@External channel: syntax error: %s",s-1);
05712     return(-1);
05713 }
05714 return(0);
05715 #else /*ifdef WITHEXTERNALCHANNEL*/
05716     Error0("External channel: not implemented on this computer/system");
05717     return(-1);
05718 #endif /*ifdef WITHEXTERNALCHANNEL ... else*/
05719 }
05720 /*
05721     #] DoSetExternalAttr:
05722     #[ DoRmExternal:
05723         #rmexternal [n] (if 0, close all)
05724 */
05725 int DoRmExternal(UBYTE *s)
05726 {
05727     #ifdef WITHEXTERNALCHANNEL
05728         int n = -1;
05729     #else
05730         DUMMYUSE(s);
05731     #endif
05732     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
05733     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
05734     if ( AP.preError ) return(0);
05735     #ifdef WITHEXTERNALCHANNEL
05736     while ( *s == ' ' || *s == '\t' ) s++;
05737     if( chartype[*s] == 1 ){
05738         for(n=0; chartype[*s] == 1 ; s++) { n = 10*n + *s - '0'; }
05739         while ( *s == ' ' || *s == '\t' ) s++;
05740     }
05741     if(*s!='\0'){
05742         MesPrint("@rmexternal: invalid number");
05743         return(-1);
05744     }
05745     switch(n){
05746     case 0:/*Close all opened channels*/
05747         closeAllExternalChannels();
05748         AX.currentExternalChannel=0;
05749         /*Do not clean AX.currentPrompt!*/
05750         return(0);
05751     case -1:/*number is not specified - try current*/
05752         n=AX.currentExternalChannel;
05753         /* fall through */
05754     default:
05755         closeExternalChannel(n);/*No reaction for possible error*/
05756     }
05757     if (n == AX.currentExternalChannel)/*cleaned up by closeExternalChannel()*/

```

```

05760     AX.currentExternalChannel=0;
05761     return(0);
05762 #else /*ifdef WITHEXTERNALCHANNEL*/
05763     Error0("External channel: not implemented on this computer/system");
05764     return(-1);
05765 #endif /*ifdef WITHEXTERNALCHANNEL ... else*/
05766 }
05767 /*
05768 #] DoRmExternal:
05769 #[ DoFromExternal :
05770     #fromexternal
05771         is used to read the text from the running external
05772         program, the syntax is similar to the #include
05773         directive.
05774     #fromexternal "varname"
05775         is used to read the text from the running external
05776         program into the preprocessor variable varname.
05777         directive.
05778     #fromexternal "varname" maxlength
05779         is used to read the text from the running external
05780         program into the preprocessor variable varname.
05781         directive. Only first maxlength characters are
05782         stored.
05783     FORM continues to read the running external
05784     program output until the external program outputs a
05785     prompt.
05786 */
05787
05788 int DoFromExternal(UBYTE *s)
05789 {
05790     #ifdef WITHEXTERNALCHANNEL
05791         UBYTE *prevar=0;
05792         int lbuf=-1;
05793         int withNoList=AC.NoShowInput;
05794         int oldpreassignflag;
05795     #else
05796         DUMMYUSE(s);
05797     #endif
05798     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
05799     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
05800     if ( AP.preError ) return(0);
05801     #ifdef WITHEXTERNALCHANNEL
05802         FLUSHCONSOLE;
05803         while ( *s == ' ' || *s == '\t' ) s++;
05804         /*[17may2006 mt]:*/
05805         if ( *s == '-' || *s == '+' ) {
05806             if ( *s == '-' )
05807                 withNoList = 1;
05808             else
05809                 withNoList = 0;
05810             s++;
05811             while ( *s == ' ' || *s == '\t' ) s++;
05812         }/*if ( *s == '-' || *s == '+' )*/
05813         /*[17may2006 mt]:*/
05814         /*[02feb2006 mt]:*/
05815         if(*s == '"'){/*prevar to store the output is defined*/
05816             prevar=++s;
05817             if ( *s=='$' || chartype[*s] == 0 )for(;;s != '"' ; s++)switch(chartype[*s]){
05818                 case 10:/*'\0' fits here*/
05819                     MesPrint("@Can't finde closing \"");
05820                     Terminate(-1);
05821                 case 0:case 1: continue;
05822                 default:
05823                     break;
05824             }
05825             if(*s != '"'){
05826                 MesPrint("@Illegal name to store output of external channel");
05827                 return(-1);
05828             }
05829             *s='\0';
05830             for(s++; *s == ' ' || *s == '\t'; s++);
05831         }/*if(*s == '"')*/
05832         if(*s != '\0'){
05833             if( chartype[*s] == 1 ){
05834                 for(lbuf=0; chartype[*s] == 1 ; s++) { lbuf = 10*lbuf + *s - '0'; }
05835                 while ( *s == ' ' || *s == '\t' ) s++;
05836             }
05837             if( (*s!='\0')||(lbuf<0) ){
05838                 MesPrint("@Illegal buffer length in fromexternal");
05839                 return(-1);
05840             }
05841         }
05842     #endif
05843 }

```

```

05847     }
05848 }/*if(*s != '\0')*/
05849 /*:[02feb20006 mt]*/
05850 if(getCurrentExternalChannel() != AX.currentExternalChannel)
05851     /*[08may20006 mt]*/
05852     /*selectExternalChannel(AX.currentExternalChannel);*/
05853     if(selectExternalChannel(AX.currentExternalChannel)){
05854         MesPrint("@No current external channel");
05855         return(-1);
05856     }
05857     /*:[08may20006 mt]*/
05858
05859 /*[02feb2006 mt]*/
05860 if(prevar!=0){/*The result must be stored into preprovar*/
05861     UBYTE *buf;
05862     int cc = 0;
05863     if(lbuf == -1){/*Unlimited buffer, everything must be stored*/
05864         int i;
05865         buf=Malloc1( (lbuf=255)+1,"Fromexternal");
05866         /*[18may20006 mt]*/
05867         /*for(i=0; (cc=getcFromExtChannel())!=EOF;i++){*/
05868         /* May 2006: now getcFromExtChannelOk returns EOF while
05869         getcFromExtChannelFailure returns -2 (see comments in
05870         exctcmd.c)*/
05871         for(i=0; (cc=getcFromExtChannel())>0;i++){
05872             /*:[18may20006 mt]*/
05873             if(i==lbuf){
05874                 int j;
05875                 UBYTE *tmp=Malloc1( (lbuf*=2)+1,"Fromexternal");
05876                 for (j=0; j<i; j++) tmp[j]=buf[j];
05877                 M_free(buf, "Fromexternal");
05878                 buf=tmp;
05879             }
05880             buf[i]=(UBYTE) (cc);
05881             /*for(i=0; (cc=getcFromExtChannel())>0;i++){*/
05882             /*:[18may20006 mt]*/
05883             if(cc == -2){
05884                 MesPrint("@No current external channel");
05885                 return(-1);
05886             }
05887             lbuf=i;
05888             /*:[18may20006 mt]*/
05889             buf[i]='\0';
05890         }else{/*Fixed buffer, only lbuf chars must be stored*/
05891             int i;
05892             buf=Malloc1(lbuf+1,"Fromexternal");
05893             for(i=0; i<lbuf;i++){
05894                 /*:[18may20006 mt]*/
05895                 /*if( (cc=getcFromExtChannel())==EOF )*/
05896                 /* May 2006: now getcFromExtChannelOk returns EOF while
05897                 getcFromExtChannelFailure returns -2 (see comments in
05898                 exctcmd.c)*/
05899                 if( (cc=getcFromExtChannel())<1 )
05900                 /*:[18may20006 mt]*/
05901                 break;
05902                 buf[i]=(UBYTE) (cc);
05903             }
05904             buf[i]='\0';
05905             /*:[18may20006 mt]*/
05906             /*if(cc!=EOF)
05907             while(getcFromExtChannel()!=EOF);*//*Eat the rest*/
05908             /* May 2006: now getcFromExtChannelOk returns EOF while
05909             getcFromExtChannelFailure returns -2 (see comments in
05910             exctcmd.c)*/
05911             if(cc>0)
05912                 while(getcFromExtChannel())>0; /*Eat the rest*/
05913             else if(cc == -2){
05914                 MesPrint("@No current external channel");
05915                 return(-1);
05916             }
05917             /*:[18may20006 mt]*/
05918         }
05919         /*:[18may20006 mt]*/
05920         if(*prevar == '$'){/*Put the answer to the dollar variable*/
05921             int oldNumPotModdollars = NumPotModdollars;
05922 #ifdef WITHMPI
05923             WORD oldRhsExprInModuleFlag = AC.RhsExprInModuleFlag;
05924             AC.RhsExprInModuleFlag = 0;
05925 #endif
05926             /*Here lbuf is the actual length of buf!*/
05927             /*"prevar=buf'\0'":*/
05928             UBYTE *pbuf=Malloc1(StrLen(prevar)+1+lbuf+1,"Fromexternal to dollar");
05929             UBYTE *c=pbuf;
05930             UBYTE *b=prevar;
05931             while(*b!='\0'){*c++ = *b++;}
05932             *c++='=';
05933             b=buf;

```

```

05934         while( (*c++==*b++)!='\0' );
05935         oldpreassignflag = AP.PreAssignFlag;
05936         AP.PreAssignFlag = 1;
05937         if ( ( cc = CompileStatement(pbuf) ) || ( cc = CatchDollar(0) ) ) {
05938             Error1("External channel: can't assign output to dollar variable ",prevar);
05939         }
05940         AP.PreAssignFlag = oldpreassignflag;
05941         NumPotModdollars = oldNumPotModdollars;
05942 #ifndef WITHMPI
05943         AC.RhsExprInModuleFlag = oldRhsExprInModuleFlag;
05944 #endif
05945         M_free(pbuf,"Fromexternal to dollar");
05946     }else{
05947         cc = PutPreVar(prevar, buf, 0, 1) < 0;
05948     }
05949     /*:[18may20006 mt]*/
05950     M_free(buf,"Fromexternal");
05951     if ( cc ) return(-1);
05952     return(0);
05953 }
05954 /*:[02feb2006 mt]*/
05955 if ( OpenStream(s,EXTERNALCHANNELSTREAM,0,PREENOACTION) == 0 ) return(-1);
05956 /*:[17may2006 mt]*/
05957 AC.NoShowInput = withNoList;
05958 /*:[17may2006 mt]*/
05959 return(0);
05960 #else
05961     Error0("External channel: not implemented on this computer/system");
05962     return(-1);
05963 #endif
05964 }
05965 /*
05966     #] DoFromExternal :
05967     #[ DoToExternal :
05968         #toexetrnal
05969 */
05970 */
05971
05972 #ifndef WITHEXTERNALCHANNEL
05973
05974 /*A wrapper to writeBufToExtChannel, see the file extcmd.c:*/
05975 LONG WriteToExternalChannel(int handle, UBYTE *buffer, LONG size)
05976 {
05977     /*ATT! handle is not used! Actual output is performed to
05978         the current external channel, see extcmd.c!*/
05979     DUMMYUSE(handle);
05980     if(writeBufToExtChannel((char*)buffer,size))
05981         return(-1);
05982     return(size);
05983 }
05984 #endif /*ifndef WITHEXTERNALCHANNEL*/
05985
05986 int DoToExternal(UBYTE *s)
05987 {
05988 #ifndef WITHEXTERNALCHANNEL
05989     HANDLERS h;
05990     LONG (*OldWrite)(int handle, UBYTE *buffer, LONG size) = WriteFile;
05991     int ret=-1;
05992 #else
05993     DUMMYUSE(s);
05994 #endif
05995     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
05996     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
05997     if ( AP.preError ) return(0);
05998 #ifndef WITHEXTERNALCHANNEL
05999     h.oldsilent=AM.silent;
06000     h.newlogonly = h.oldlogonly = AM.FileOnlyFlag;
06001     h.newhandle = h.oldhandle = AC.LogHandle;
06002     h.oldprinttype = AO.PrintType;
06003
06004     WriteFile=&WriteToExternalChannel;
06005
06006     while ( *s == ' ' || *s == '\t' ) s++;
06007
06008     if(AX.currentExternalChannel==0){
06009         MesPrint("@No current external channel");
06010         goto DoToExternalReady;
06011     }
06012
06013     if(getCurrentExternalChannel()!=AX.currentExternalChannel)
06014         selectExternalChannel(AX.currentExternalChannel);
06015
06016     ret=writeToChannel(EXTERNALCHANNELOUT,s,&h);
06017     DoToExternalReady:
06018         WriteFile=OldWrite;
06019     return(ret);
06020

```



```

06021 #else /*ifdef WITHEXTERNALCHANNEL*/
06022     Error0("External channel: not implemented on this computer/system");
06023     return(-1);
06024 #endif /*ifdef WITHEXTERNALCHANNEL ... else*/
06025
06026 }
06027
06028 /*
06029     #] DoToExternal :
06030     #[ defineChannel :
06031 */
06032
06033 UBYTE *defineChannel(UBYTE *s, HANDLERS *h)
06034 {
06035     UBYTE *name,*to;
06036
06037     if ( *s != '<' )
06038         return(s);
06039
06040     s++;
06041     name = to = s;
06042     while ( *s && *s != '>' ) {
06043         if ( *s == '\\\\' ) s++;
06044         *to++ = *s++;
06045     }
06046     if ( *s == 0 ) {
06047         MesPrint("@Improper termination of filename");
06048         return(0);
06049     }
06050     s++;
06051     *to = 0;
06052     if ( *name ) {
06053         h->newhandle = GetChannel((char *)name,0);
06054         h->newlogonly = 1;
06055     }
06056     else if ( AC.LogHandle >= 0 ) {
06057         h->newhandle = AC.LogHandle;
06058         h->newlogonly = 1;
06059     }
06060     return(s);
06061 }
06062
06063 /*
06064     #] defineChannel :
06065     #[ writeToChannel :
06066 */
06067
06068 int writeToChannel(int wtype, UBYTE *s, HANDLERS *h)
06069 {
06070     UBYTE *to, *fstring, *ss, *sss, *sl, c, cl;
06071     WORD num, number, nfac;
06072     WORD oldOptimizationLevel;
06073     UBYTE Out[MAXLINELENGTH+14], *stopper;
06074     int nosemi, i;
06075     int plus = 0;
06076
06077     /*
06078     Now determine the format string
06079 */
06080     while ( *s == ',' || *s == ' ' ) s++;
06081     if ( *s != '"' ) {
06082         MesPrint("@No format string present");
06083         return(-1);
06084     }
06085     s++; fstring = to = s;
06086     while ( *s ) {
06087         if ( *s == '\\\\' ) {
06088             s++;
06089             if ( *s == '\\\\' ) {
06090                 *to++ = *s++;
06091                 if ( *s == '\\\\' ) *to++ = *s++;
06092             }
06093             else if ( *s == '"' ) *to++ = *s++;
06094             else { *to++ = '\\\\'; *to++ = *s++; }
06095         }
06096         else if ( *s == '"' ) break;
06097         else *to++ = *s++;
06098     }
06099     if ( *s != '"' ) {
06100         MesPrint("@No closing \" in format string");
06101         return(-1);
06102     }
06103     *to = 0; s++;
06104     if ( AC.LineLength > 20 && AC.LineLength <= MAXLINELENGTH ) stopper = Out + AC.LineLength;
06105     else stopper = Out + MAXLINELENGTH;
06106     to = Out;
06107     /*

```

```

06108     s points now at the list of objects (if any)
06109     we can start executing the format string.
06110 */
06111     AM.silent = 0;
06112     AC.LogHandle = h->newhandle;
06113     AM.FileOnlyFlag = h->newlogonly;
06114     if ( h->newhandle >= 0 ) {
06115         AO.PrintType |= PRINTLFILE;
06116     }
06117     while ( *fstring ) {
06118         if ( to >= stopper ) {
06119             if ( AC.OutputMode == FORTRANMODE && AC.IsFortran90 == ISFORTRAN90 ) {
06120                 *to++ = '&';
06121             }
06122             num = to - Out;
06123             WriteString(wtype,Out,num);
06124             to = Out;
06125             if ( AC.OutputMode == FORTRANMODE
06126                 || AC.OutputMode == PFORTRANMODE ) {
06127                 number = 7;
06128                 for ( i = 0; i < number; i++ ) *to++ = ' ';
06129                 to[-2] = '&';
06130             }
06131         }
06132         if ( *fstring == '\\\\' ) {
06133             fstring++;
06134             if ( *fstring == 'n' ) {
06135                 num = to - Out;
06136                 WriteString(wtype,Out,num);
06137                 to = Out;
06138                 fstring++;
06139             }
06140             else if ( *fstring == 't' ) { *to++ = '\\t'; fstring++; }
06141             else if ( *fstring == 'b' ) { *to++ = '\\b'; fstring++; }
06142             else *to++ = *fstring++;
06143         }
06144         else if ( *fstring == '%' ) {
06145             plus = 0;
06146             retry:
06147                 fstring++;
06148                 if ( *fstring == 'd' ) {
06149                     int sign,dig;
06150                     number = -1;
06151                     donumber:
06152                         while ( *s == ',' || *s == ' ' || *s == '\\t' ) s++;
06153                         sign = 1;
06154                         while ( *s == '+' || *s == '-' ) {
06155                             if ( *s == '-' ) sign = -sign;
06156                             s++;
06157                         }
06158                         dig = 0; ss = s; if ( sign < 0 ) { ss--; *ss = '-'; dig++; }
06159                         while ( *s >= '0' && *s <= '9' ) { s++; dig++; }
06160                         if ( number < 0 ) {
06161                             while ( ss < s ) {
06162                                 if ( to >= stopper ) {
06163                                     num = to - Out;
06164                                     WriteString(wtype,Out,num);
06165                                     to = Out;
06166                                 }
06167                                 if ( *ss == '\\\\' ) ss++;
06168                                 *to++ = *ss++;
06169                             }
06170                         }
06171                         else {
06172                             if ( number < dig ) { dig = number; ss = s - dig; }
06173                             while ( number > dig ) {
06174                                 if ( to >= stopper ) {
06175                                     num = to - Out;
06176                                     WriteString(wtype,Out,num);
06177                                     to = Out;
06178                                 }
06179                                 *to++ = ' '; number--;
06180                             }
06181                             while ( ss < s ) {
06182                                 if ( to >= stopper ) {
06183                                     num = to - Out;
06184                                     WriteString(wtype,Out,num);
06185                                     to = Out;
06186                                 }
06187                                 if ( *ss == '\\\\' ) ss++;
06188                                 *to++ = *ss++;
06189                             }
06190                         }
06191                         fstring++;
06192                     }
06193                     else if ( *fstring == '$' ) {
06194                         UBYTE *dolalloc;

```

```

06195         number = AO.OutSkip;
06196 dodollar:
06197         while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
06198         if ( AC.OutputMode == FORTRANMODE
06199             || AC.OutputMode == PFORTRANMODE ) {
06200             number = 7;
06201         }
06202         if ( *s != '$' ) {
06203             MesPrint("@$-variable expected in #write instruction");
06204             AM.FileOnlyFlag = h->oldlogonly;
06205             AC.LogHandle = h->oldhandle;
06206             AO.PrintType = h->oldprinttype;
06207             AM.silent = h->oldsilent;
06208             return(-1);
06209         }
06210         s++; ss = s;
06211         while ( chartype[*s] <= 1 ) s++;
06212         if ( s == ss ) goto nodollar;
06213         c = *s; *s = 0;
06214         num = GetDollar(ss);
06215         if ( num < 0 ) {
06216             MesPrint("@#write instruction: %s has not been defined",ss);
06217             AM.FileOnlyFlag = h->oldlogonly;
06218             AC.LogHandle = h->oldhandle;
06219             AO.PrintType = h->oldprinttype;
06220             AM.silent = h->oldsilent;
06221             return(-1);
06222         }
06223         *s = c;
06224         if ( *s == '[' ) {
06225             if ( Dollars[num].nfactors <= 0 ) {
06226                 *s = 0;
06227                 MesPrint("@#write instruction: %s has not been factorized",ss);
06228                 AM.FileOnlyFlag = h->oldlogonly;
06229                 AC.LogHandle = h->oldhandle;
06230                 AO.PrintType = h->oldprinttype;
06231                 AM.silent = h->oldsilent;
06232                 return(-1);
06233             }
06234             /*
06235             Now get the number between the []
06236             */
06237             nfac = GetDollarNumber(&s,Dollars+num);
06238
06239             if ( Dollars[num].nfactors == 1 && nfac == 1 ) goto writewhole;
06240
06241             if ( ( dolalloc = WriteDollarFactorToBuffer(num,nfac,0) ) == 0 ) {
06242                 AM.FileOnlyFlag = h->oldlogonly;
06243                 AC.LogHandle = h->oldhandle;
06244                 AO.PrintType = h->oldprinttype;
06245                 AM.silent = h->oldsilent;
06246                 return(-1);
06247             }
06248             goto writealloc;
06249         }
06250         else if ( *s && *s != ' ' && *s != ',' && *s != '\t' ) {
06251             MesPrint("@#write instruction: illegal characters after $-variable");
06252             AM.FileOnlyFlag = h->oldlogonly;
06253             AC.LogHandle = h->oldhandle;
06254             AO.PrintType = h->oldprinttype;
06255             AM.silent = h->oldsilent;
06256             return(-1);
06257         }
06258         else {
06259             writewhole:
06260             if ( ( dolalloc = WriteDollarToBuffer(num,0) ) == 0 ) {
06261                 AM.FileOnlyFlag = h->oldlogonly;
06262                 AC.LogHandle = h->oldhandle;
06263                 AO.PrintType = h->oldprinttype;
06264                 AM.silent = h->oldsilent;
06265                 return(-1);
06266             }
06267             else {
06268                 writealloc:
06269                 ss = dolalloc;
06270                 while ( *ss ) {
06271                     if ( to >= stopper ) {
06272                         if ( AC.OutputMode == FORTRANMODE && AC.IsFortran90 == ISFORTRAN90 ) {
06273                             *to++ = '&';
06274                         }
06275                         num = to - Out;
06276                         WriteString(wtype,Out,num);
06277                         to = Out;
06278                         for ( i = 0; i < number; i++ ) *to++ = ' ';
06279                         if ( AC.OutputMode == FORTRANMODE
06280                             || AC.OutputMode == PFORTRANMODE ) to[-2] = '&';
06281                     }

```

```

06282         if ( chartye[*ss] > 3 ) { *to++ = *ss++; }
06283     else {
06284         sss = ss; while ( chartye[*ss] <= 3 ) ss++;
06285         if ( ( to + (ss-sss) ) >= stopper ) {
06286             if ( (ss-sss) >= (stopper-Out) ) {
06287                 if ( ( to - stopper ) < 10 ) {
06288                     if ( AC.OutputMode == FORTRANMODE && AC.IsFortran90 ==
ISFORTRAN90 ) {
06289                         *to++ = '&';
06290                     }
06291                     num = to - Out;
06292                     WriteString(wtype,Out,num);
06293                     to = Out;
06294                     for ( i = 0; i < number; i++ ) *to++ = ' ';
06295                     if ( AC.OutputMode == FORTRANMODE
|| AC.OutputMode == PFORTRANMODE ) to[-2] = '&';
06296                 }
06297                 while ( (ss-sss) >= (stopper-Out) ) {
06298                     while ( to < stopper-1 ) {
06299                         *to++ = *sss++;
06300                     }
06301                     if ( AC.OutputMode == FORTRANMODE && AC.IsFortran90 ==
ISFORTRAN90 ) {
06302                         *to++ = '&';
06303                     }
06304                     else {
06305                         *to++ = '\\';
06306                     }
06307                     num = to - Out;
06308                     WriteString(wtype,Out,num);
06309                     to = Out;
06310                     if ( AC.OutputMode == FORTRANMODE
|| AC.OutputMode == PFORTRANMODE ) {
06311                         for ( i = 0; i < number; i++ ) *to++ = ' ';
06312                         to[-2] = '&';
06313                     }
06314                 }
06315             }
06316         }
06317     }
06318     else {
06319         if ( AC.OutputMode == FORTRANMODE && AC.IsFortran90 == ISFORTRAN90
) {
06320             *to++ = '&';
06321         }
06322         num = to - Out;
06323         WriteString(wtype,Out,num);
06324         to = Out;
06325         for ( i = 0; i < number; i++ ) *to++ = ' ';
06326         if ( AC.OutputMode == FORTRANMODE
|| AC.OutputMode == PFORTRANMODE ) to[-2] = '&';
06327     }
06328 }
06329 }
06330 while ( sss < ss ) *to++ = *sss++;
06331 }
06332 }
06333 }
06334 M_free(dolalloc,"written dollar");
06335 fstring++;
06336 }
06337 }
06338 else if ( *fstring == 's' ) {
06339     fstring++;
06340     while ( *s == ',' || *s == ' ' || *s == '\\t' ) s++;
06341     if ( *s == '"' ) {
06342         s++; ss = s;
06343         while ( *s ) {
06344             if ( *s == '\\\\' ) s++;
06345             else if ( *s == '"' ) break;
06346             s++;
06347         }
06348         if ( *s == 0 ) {
06349             MesPrint("@#write instruction: Missing \" in string");
06350             AM.FileOnlyFlag = h->oldlogonly;
06351             AC.LogHandle = h->oldhandle;
06352             AO.PrintType = h->oldprinttype;
06353             AM.silent = h->oldsilent;
06354             return(-1);
06355         }
06356         while ( ss < s ) {
06357             if ( to >= stopper ) {
06358                 num = to - Out;
06359                 WriteString(wtype,Out,num);
06360                 to = Out;
06361             }
06362             if ( *ss == '\\\\' ) ss++;
06363             *to++ = *ss++;
06364         }
06365         s++;

```

```

06366     }
06367     else {
06368         sss = ss = s;
06369         while ( *s && *s != ' ' ) {
06370             if ( *s == '\\\\' ) { s++; sss = s+1; }
06371             s++;
06372         }
06373         while ( s > sss+1 && ( s[-1] == ' ' || s[-1] == '\\t' ) ) s--;
06374         while ( ss < s ) {
06375             if ( to >= stopper ) {
06376                 num = to - Out;
06377                 WriteString(wtype,Out,num);
06378                 to = Out;
06379             }
06380             if ( *ss == '\\\\' ) ss++;
06381             *to++ = *ss++;
06382         }
06383     }
06384 }
06385 else if ( *fstring == 'X' ) {
06386     fstring++;
06387     if ( cbuf[AM.sbufnum].numrhs > 0 ) {
06388         /*
06389             This should be only to the value of AM.oldnumextrasympols
06390         */
06391         UBYTE *s = GetPreVar(AM.oldnumextrasympols,0);
06392         WORD x = 0;
06393         while ( *s >= '0' && *s <= '9' ) x = 10*x + *s++ - '0';
06394         if ( x > 0 )
06395             PrintSubtermList(1,x);
06396         else
06397             PrintSubtermList(1,cbuf[AM.sbufnum].numrhs);
06398     }
06399 }
06400 else if ( *fstring == 'O' ) {
06401     number = AO.OutSkip;
06402 dooptim:
06403     fstring++;
06404     /*
06405         First test whether there is an optimization buffer
06406     */
06407     if ( AO.OptimizeResult.code == NULL && AO.OptimizationLevel != 0 ) {
06408         MesPrint("@In #write instruction: no optimization results available!");
06409         return(-1);
06410     }
06411     num = to - Out;
06412     WriteString(wtype,Out,num);
06413     to = Out;
06414     if ( AO.OptimizationLevel != 0 ) {
06415         WORD oldoutskip = AO.OutSkip;
06416         AO.OutSkip = number;
06417         optimize_print_code(0);
06418         AO.OutSkip = oldoutskip;
06419     }
06420 }
06421 else if ( *fstring == 'e' || *fstring == 'E' ) {
06422     if ( *fstring == 'E'
06423         || AC.OutputMode == FORTRANMODE
06424         || AC.OutputMode == PFORTRANMODE ) nosemi = 1;
06425     else nosemi = 0;
06426     fstring++;
06427     while ( *s == ' ' || *s == '\\ ' || *s == '\\t' ) s++;
06428     if ( chartype[*s] != 0 && *s != '[' ) {
06429 noexpr:
06430         MesPrint("@expression name expected in #write instruction");
06431         AM.FileOnlyFlag = h->oldlogonly;
06432         AC.LogHandle = h->oldhandle;
06433         AO.PrintType = h->oldprinttype;
06434         AM.silent = h->oldsilent;
06435         return(-1);
06436     }
06437     ss = s;
06438     if ( ( s = SkipAName(ss) ) == 0 || s[-1] == '_' ) goto noexpr;
06439     s1 = s; c = c1 = *s1;
06440     if ( c1 == '(' ) {
06441         SKIPBRA3(s)
06442         if ( *s == ')' ) {
06443             AO.CurBufWrt = s1+1;
06444             c = *s; *s = 0;
06445         }
06446         else {
06447             MesPrint("@Illegal () specifier in expression name in #write");
06448             AM.FileOnlyFlag = h->oldlogonly;
06449             AC.LogHandle = h->oldhandle;
06450             AO.PrintType = h->oldprinttype;
06451             AM.silent = h->oldsilent;
06452             return(-1);
06453         }
06454     }

```

```

06453     }
06454     else AO.CurBufWrt = (UBYTE *)underscore;
06455     *s1 = 0;
06456     num = to - Out;
06457     if ( num > 0 ) WriteUnfinString(wtype,Out,num);
06458     to = Out;
06459     oldOptimizationLevel = AO.OptimizationLevel;
06460     AO.OptimizationLevel = 0;
06461     if ( WriteOne(ss,(int)num,nosemi,plus) < 0 ) {
06462         AM.FileOnlyFlag = h->oldlogonly;
06463         AC.LogHandle = h->oldhandle;
06464         AO.PrintType = h->oldprinttype;
06465         AM.silent = h->oldsilent;
06466         return(-1);
06467     }
06468     AO.OptimizationLevel = oldOptimizationLevel;
06469     *s1 = c1;
06470     if ( s > s1 ) *s++ = c;
06471 }
06472 /*
06473 File content
06474 */
06475 else if ( ( *fstring == 'f' ) || ( *fstring == 'F' ) ) {
06476     LONG n;
06477     while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
06478     ss = s;
06479     while ( *s && *s != ',' ) {
06480         if ( *s == '\\' ) s++;
06481         s++;
06482     }
06483     c = *s; *s = 0;
06484     s1 = LoadInputFile(ss,HEADERFILE);
06485     *s = c;
06486 /*
06487     There should have been a way to pass the file size.
06488     Also there should be conversions for \r\n etc.
06489 */
06490     if ( s1 ) {
06491         ss = s1; while ( **s ) ss++;
06492         n = ss-s1;
06493         WriteString(wtype,s1,n);
06494         M_free(s1,"copy file");
06495     }
06496     else if ( *fstring == 'F' ) {
06497         *s = 0;
06498         MesPrint("@Error in #write: could not open file %s",ss);
06499         *s = c;
06500         goto ReturnWithError;
06501     }
06502     fstring++;
06503 }
06504 else if ( *fstring == '%' ) {
06505     *to++ = *fstring++;
06506 }
06507 else if ( FG.cTable[*fstring] == 1 ) { /* %#S */
06508     number = 0;
06509     while ( FG.cTable[*fstring] == 1 ) {
06510         number = 10*number + *fstring++ - '0';
06511     }
06512     if ( *fstring == 'O' ) goto dooptim;
06513     else if ( *fstring == 'd' ) goto donumber;
06514     else if ( *fstring == '$' ) goto dodollar;
06515     else if ( *fstring == 't' ) { /* 'tab' position */
06516         if ( number < (WORD)(stopper-Out) ) {
06517             while ( (WORD)(to-Out) < number ) *to++ = ' ';
06518         }
06519         fstring++;
06520     }
06521 }
06522 else if ( *fstring == 'X' || *fstring == 'x' ) {
06523     if ( number > 0 && number <= cbuf[AM.sbufnum].numrhs ) {
06524         UBYTE buffer[80], *out, *old1, *old2, *old3;
06525         WORD *term, first;
06526         if ( *fstring == 'X' ) {
06527             out = StrCopy((UBYTE *)AC.extrasym,buffer);
06528             if ( AC.extrasymbols == 0 ) {
06529                 out = NumCopy(number,out);
06530                 out = StrCopy((UBYTE *)"_",out);
06531             }
06532             else if ( AC.extrasymbols == 1 ) {
06533                 if ( AC.OutputMode == CMODE ) {
06534                     out = StrCopy((UBYTE *)"[",out);
06535                     out = NumCopy(number,out);
06536                     out = StrCopy((UBYTE *)"]",out);
06537                 }
06538                 else {
06539                     out = StrCopy((UBYTE *)"(",out);
06540                     out = NumCopy(number,out);
06541                 }
06542             }
06543         }
06544     }
06545 }

```

```

06540         out = StrCopy((UBYTE *)"),out);
06541     }
06542 }
06543 out = StrCopy((UBYTE *)"=",out);
06544 ss = buffer;
06545 while ( ss < out ) {
06546     if ( to >= stopper ) {
06547         num = to - Out;
06548         WriteString(wtype,Out,num);
06549         to = Out;
06550     }
06551     *to++ = *ss++;
06552 }
06553 }
06554 term = cbuf[AM.sbufnum].rhs[number];
06555 first = 1;
06556 if ( *term == 0 ) {
06557     *to++ = '0';
06558 }
06559 else {
06560     old1 = AO.OutFill;
06561     old2 = AO.OutputLine;
06562     old3 = AO.OutStop;
06563     AO.OutFill = to;
06564     AO.OutputLine = Out;
06565     AO.OutStop = Out + AC.LineLength;
06566     while ( *term ) {
06567         if ( WriteInnerTerm(term,first) ) Terminate(-1);
06568         term += *term;
06569         first = 0;
06570     }
06571     to = Out + (AO.OutFill-AO.OutputLine);
06572     AO.OutFill = old1;
06573     AO.OutputLine = old2;
06574     AO.OutStop = old3;
06575 }
06576 }
06577 fstring++;
06578 }
06579 else {
06580     goto IllegControlSequence;
06581 }
06582 }
06583 else if ( *fstring == '+' ) {
06584     plus = 1; goto retry;
06585 }
06586 else if ( *fstring == 0 ) {
06587     *to++ = 0;
06588 }
06589 else {
06590     IllegControlSequence:
06591     MesPrint("@Illegal control sequence in format string in #write instruction");
06592     ReturnWithError:
06593     AM.FileOnlyFlag = h->oldlogonly;
06594     AC.LogHandle = h->oldhandle;
06595     AO.PrintType = h->oldprinttype;
06596     AM.silent = h->oldsilent;
06597     return(-1);
06598 }
06599 }
06600 else {
06601     *to++ = *fstring++;
06602 }
06603 }
06604 /*
06605 Now flush the output
06606 */
06607 num = to - Out;
06608 /*[15apr2004 mt]:*/
06609 if(wtype==EXTERNALCHANNELOUT){
06610     if(num!=0)
06611         WriteUnfinString(wtype,Out,num);
06612 }else
06613     /*:[15apr2004 mt]*/
06614     WriteString(wtype,Out,num);
06615 /*
06616 and restore original parameters
06617 */
06618 AM.FileOnlyFlag = h->oldlogonly;
06619 AC.LogHandle = h->oldhandle;
06620 AO.PrintType = h->oldprinttype;
06621 AM.silent = h->oldsilent;
06622 return(0);
06623 }
06624
06625 /*
06626 #] writeToChannel :

```

```

06627     #[] DoFactDollar :
06628
06629     Executes the #factdollar $var
06630         instruction
06631 */
06632
06633 int DoFactDollar(UBYTE *s)
06634 {
06635     GETIDENTITY
06636     WORD numdollar, *oldworkpointer;
06637
06638     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
06639     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
06640     while ( *s == ' ' || *s == '\t' ) s++;
06641     if ( *s == '$' ) {
06642         if ( GetName(AC.dollarnames,s+1,&numdollar,NOAUTO) != CDOLLAR ) {
06643             MesPrint("@%s is undefined",s);
06644             return(-1);
06645         }
06646         s = SkipAName(s+1);
06647         if ( *s != 0 ) {
06648             MesPrint("@#FactDollar should have a single $variable for its argument");
06649             return(-1);
06650         }
06651         NewSort(BHEAD0);
06652         oldworkpointer = AT.WorkPointer;
06653         if ( DollarFactorize(BHEAD numdollar) ) return(-1);
06654         AT.WorkPointer = oldworkpointer;
06655         LowerSortLevel();
06656         return(0);
06657     }
06658     else if ( ParenthesesTest(s) ) return(-1);
06659     else {
06660         MesPrint("@#FactDollar should have a single $variable for its argument");
06661         return -1;
06662     }
06663 }
06664
06665 /*
06666     #[] DoFactDollar :
06667     #[] GetDollarNumber :
06668 */
06669
06670 WORD GetDollarNumber(UBYTE **inp, DOLLARS d)
06671 {
06672     UBYTE *s = *inp, c, *name;
06673     WORD number, nfac, *w;
06674     DOLLARS dd;
06675     s++;
06676     if ( *s == '$' ) {
06677         s++; name = s;
06678         while ( FG.cTable[*s] < 2 ) s++;
06679         c = *s; *s = 0;
06680         if ( GetName(AC.dollarnames,name,&number,NOAUTO) == NAMENOTFOUND ) {
06681             MesPrint("@dollar in #write should have been defined previously");
06682             Terminate(-1);
06683         }
06684         *s = c;
06685         dd = Dollars + number;
06686         if ( c == '[' ) {
06687             *inp = s;
06688             nfac = GetDollarNumber(inp,dd);
06689             s = *inp;
06690             if ( *s != ']' ) {
06691                 MesPrint("@Illegal factor for dollar variable");
06692                 Terminate(-1);
06693             }
06694             *inp = s+1;
06695             if ( nfac == 0 ) {
06696                 if ( dd->nfactors > d->nfactors ) {
06697 TooBig:
06698                     MesPrint("@Factor number for dollar variable too large");
06699                     Terminate(-1);
06700                 }
06701                 return(dd->nfactors);
06702             }
06703             w = dd->factors[nfac-1].where;
06704             if ( w == 0 ) {
06705                 if ( dd->factors[nfac-1].value > d->nfactors ||
06706                     dd->factors[nfac-1].value < 0 ) goto TooBig;
06707                 return(dd->factors[nfac-1].value);
06708             }
06709             if ( *w == 4 && w[4] == 0 && w[3] == 3 && w[2] == 1
06710                 && w[1] <= d->nfactors ) return(w[1]);
06711             if ( w[*w] == 0 && w[*w-1] == *w-1 ) goto TooBig;
06712 IllNum:
06713             MesPrint("@Illegal factor number for dollar variable");

```



```

06714         Terminate(-1);
06715     }
06716     else { /* The dollar should be a number */
06717         if ( dd->type == DOLZERO ) {
06718             return(0);
06719         }
06720         else if ( dd->type == DOLTERMS || dd->type == DOLNUMBER ) {
06721             w = dd->where;
06722             if ( *w == 4 && w[4] == 0 && w[3] == 3 && w[2] == 1
06723                 && w[1] <= d->nfactors ) return(w[1]);
06724             if ( w[*w] == 0 && w[*w-1] == *w-1 ) goto TooBig;
06725             goto IllNum;
06726         }
06727         else goto IllNum;
06728     }
06729 }
06730 else if ( FG.cTable[*s] == 1 ) {
06731     WORD x = *s++ - '0';
06732     while ( FG.cTable[*s] == 1 ) {
06733         x = 10*x + *s++ - '0';
06734         if ( x > d->nfactors ) {
06735             MesPrint("@Factor number %d for dollar variable too large",x);
06736             Terminate(-1);
06737         }
06738     }
06739     if ( *s != ']' ) {
06740         MesPrint("@Illegal factor number for dollar variable");
06741         Terminate(-1);
06742     }
06743     s++; *inp = s;
06744     return(x);
06745 }
06746 else {
06747     MesPrint("@Illegal factor indicator for dollar variable");
06748     Terminate(-1);
06749 }
06750 return(-1);
06751 }
06752
06753 /*
06754     #] GetDollarNumber :
06755     #[ DoSetRandom :
06756
06757     Executes the #SetRandom number
06758 */
06759
06760 int DoSetRandom(UBYTE *s)
06761 {
06762     ULONG x;
06763     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
06764     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
06765     while ( *s == ' ' || *s == '\t' ) s++;
06766     x = 0;
06767     while ( FG.cTable[*s] == 1 ) {
06768         x = 10*x + (*s++ - '0');
06769     }
06770     while ( *s == ' ' || *s == '\t' ) s++;
06771     if ( *s == 0 ) {
06772 #ifdef WITHPTHREADS
06773 #ifdef WITHSORTBOTS
06774         int id, totnum = MaX(2*AM.totalnumberofthreads-3,AM.totalnumberofthreads);
06775 #else
06776         int id, totnum = AM.totalnumberofthreads;
06777 #endif
06778         for ( id = 0; id < totnum; id++ ) {
06779             AB[id]->R.wranfseed = x;
06780             if ( AB[id]->R.wranfia ) M_free(AB[id]->R.wranfia,"wranf");
06781             AB[id]->R.wranfia = 0;
06782         }
06783 #else
06784         AR.wranfseed = x;
06785         if ( AR.wranfia ) M_free(AR.wranfia,"wranf");
06786         AR.wranfia = 0;
06787 #endif
06788         return(0);
06789     }
06790     else {
06791         MesPrint("@proper syntax is #SetRandom number");
06792         return(-1);
06793     }
06794 }
06795
06796 /*
06797     #] DoSetRandom :
06798     #[ DoOptimize :
06799
06800     Executes the #Optimize(expr) instruction.

```

```

06801 */
06802
06803 int DoOptimize(UBYTE *s)
06804 {
06805     GETIDENTITY
06806     UBYTE *exprname;
06807     WORD numexpr;
06808     int error = 0, i;
06809     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
06810     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
06811     DUMMYUSE(*s)
06812     exprname = s; s = SkipAName(s);
06813     if ( *s != 0 && *s != ';' ) {
06814         MesPrint("@proper syntax is #Optimize,expression");
06815         return(-1);
06816     }
06817     *s = 0;
06818     if ( GetName(AC.exprnames,exprname,&numexpr,NOAUTO) != CEXPRESSION ) {
06819         MesPrint("@%s is not an expression",exprname);
06820         error = 1;
06821     }
06822     else if ( AP.preError == 0 ) {
06823         EXPRESSIONS e = Expressions + numexpr;
06824         POSITION position;
06825         int firstterm;
06826         WORD *term = AT.WorkPointer;
06827         ClearOptimize();
06828         if ( AO.OptimizationLevel == 0 ) return(0);
06829         switch ( e->status ) {
06830             case LOCALEXPRESSION:
06831             case GLOBALEXPRESSION:
06832                 break;
06833             default:
06834                 MesPrint("@Expression %s is not an active unhidden local or global
expression.",exprname);
06835                 Terminate(-1);
06836                 break;
06837         }
06838         #ifdef WITHMPI
06839         if ( PF.me == MASTER )
06840         #endif
06841             RevertScratch();
06842         for ( i = NumExpressions-1; i >= 0; i-- ) {
06843             AS.OldOnFile[i] = Expressions[i].onfile;
06844             AS.OldNumFactors[i] = Expressions[i].numfactors;
06845             AS.Oldvflags[i] = Expressions[i].vflags;
06846             Expressions[i].vflags &= ~(ISUNMODIFIED|ISZERO);
06847         }
06848         for ( i = 0; i < NumExpressions; i++ ) {
06849             if ( i == numexpr ) {
06850                 PutPreVar(AM.oldnumextrasyms,
06851                     GetPreVar((UBYTE *) "EXTRASYMBOLS_",0),0,1);
06852                 Optimize(numexpr, 0);
06853                 AO.OptimizeResult.nameofexpr = strdup(exprname,"optimize expression name");
06854                 continue;
06855             }
06856             #ifdef WITHMPI
06857             if ( PF.me == MASTER ) {
06858             #endif
06859                 e = Expressions + i;
06860                 switch ( e->status ) {
06861                     case LOCALEXPRESSION:
06862                     case SKIPLEXPRESSION:
06863                     case DROPLEXPRESSION:
06864                     case DROPEDEXPRESSION:
06865                     case GLOBALEXPRESSION:
06866                     case SKIPGEXPRESSION:
06867                     case DROPGEXPRESSION:
06868                     case HIDELEXPRESSION:
06869                     case HIDEGEXPRESSION:
06870                     case DROPHLEXPRESSION:
06871                     case DROPHGEXPRESSION:
06872                     case INTOHIDELEXPRESSION:
06873                     case INTOHIDEGEXPRESSION:
06874                         break;
06875                     default:
06876                         continue;
06877                 }
06878                 AR.GetFile = 0;
06879                 SetScratch(AR.infile,&(e->onfile));
06880                 if ( GetTerm(BHEAD term) <= 0 ) {
06881                     MesPrint("@Expression %d has problems reading from scratchfile",i);
06882                     Terminate(-1);
06883                 }
06884                 term[3] = i;
06885                 AR.DeferFlag = 0;
06886                 SeekScratch(AR.outfile,&position);

```

```

06887         e->onfile = position;
06888         *AM.S0->sBuffer = 0; firstterm = -1;
06889         do {
06890             WORD *oldipointer = AR.CompressPointer;
06891             WORD *comprtop = AR.ComprTop;
06892             AR.ComprTop = AM.S0->sTop;
06893             AR.CompressPointer = AM.S0->sBuffer;
06894             if ( firstterm > 0 ) {
06895                 if ( PutOut(BHEAD term,&position,AR.outfile,1) < 0 ) goto DoSerr;
06896             }
06897             else if ( firstterm < 0 ) {
06898                 if ( PutOut(BHEAD term,&position,AR.outfile,0) < 0 ) goto DoSerr;
06899                 firstterm++;
06900             }
06901             else {
06902                 if ( PutOut(BHEAD term,&position,AR.outfile,-1) < 0 ) goto DoSerr;
06903                 firstterm++;
06904             }
06905             AR.CompressPointer = oldipointer;
06906             AR.ComprTop = comprtop;
06907         } while ( GetTerm(BHEAD term) );
06908         if ( FlushOut(&position,AR.outfile,1) ) {
06909 DoSerr:
06910             MesPrint("@Expression %d has problems writing to scratchfile",i);
06911             Terminate(-1);
06912         }
06913 #ifdef WITHMPI
06914         }
06915 #endif
06916     }
06917     /*
06918         Now some administration and we are done
06919     */
06920     UpdateMaxSize();
06921 }
06922 else {
06923     ClearOptimize();
06924 }
06925 return(error);
06926 }
06927 }
06928
06929 /*
06930     #] DoOptimize :
06931     #[ DoClearOptimize :
06932
06933     Clears all relevant buffers of the output optimization
06934 */
06935
06936 int DoClearOptimize(UBYTE *s)
06937 {
06938     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
06939     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
06940     DUMMYUSE(*s);
06941     return(ClearOptimize());
06942 }
06943
06944 /*
06945     #] DoClearOptimize :
06946     #[ DoSkipExtraSymbols :
06947
06948     Adds the intermediate variables of the previous optimization
06949     to the list of extra symbols, provided it has not yet been erased
06950     by a #clearoptimize
06951     To remove them again one needs to use the 'delete extrasymbols;'
06952     or the 'delete extrasymbols>num;' statement in which num is the
06953     old number of extra symbols.
06954 */
06955
06956 int DoSkipExtraSymbols(UBYTE *s)
06957 {
06958     CBUF *C = cbuf + AM.sbufnum;
06959     WORD tt = 0, j = 0, oldval = AO.OptimizeResult.minvar;
06960     if ( AO.OptimizeResult.code == NULL ) return(0);
06961     if ( AO.OptimizationLevel == 0 ) return(0);
06962     while ( *s == ',' ) s++;
06963     if ( *s == 0 ) {
06964         AO.OptimizeResult.minvar = AO.OptimizeResult.maxvar+1;
06965     }
06966     else {
06967         while ( *s <= '9' && *s >= '0' ) j = 10*j + *s++ - '0';
06968         if ( *s ) {
06969             MesPrint("@Illegal use of #SkipExtraSymbols instruction");
06970             Terminate(-1);
06971         }
06972         AO.OptimizeResult.minvar += j;
06973         if ( AO.OptimizeResult.minvar > AO.OptimizeResult.maxvar )

```

```

06974         AO.OptimizeResult.minvar = AO.OptimizeResult.maxvar+1;
06975     }
06976     j = AO.OptimizeResult.minvar - oldval;
06977     while ( j > 0 ) {
06978         AddRHS(AM.sbufnum,1);
06979         AddNtoC(AM.sbufnum,1,&tt,16);
06980         AddToCB(C,0);
06981         InsTree(AM.sbufnum,C->numrhs);
06982         j--;
06983     }
06984     return(0);
06985 }
06986
06987 /*
06988     #] DoSkipExtraSymbols :
06989     #[ DoPreReset :
06990
06991     Does a reset of variables.
06992     Currently only the timer (stopwatch) of `timer_`
06993 */
06994
06995 int DoPreReset(UBYTE *s)
06996 {
06997     UBYTE *ss, c;
06998     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
06999     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
07000     while ( *s == ' ' || *s == '\t' ) s++;
07001     if ( *s == 0 ) {
07002         MesPrint("@proper syntax is #Reset variable");
07003         return(-1);
07004     }
07005     ss = s;
07006     while ( FG.cTable[*s] == 0 ) s++;
07007     c = *s; *s = 0;
07008     if ( ( StrICmp(ss,(UBYTE *)"timer") == 0 )
07009         || ( StrICmp(ss,(UBYTE *)"stopwatch") == 0 ) ) {
07010         *s = c;
07011         AP.StopWatchZero = GetRunningTime();
07012         return(0);
07013     }
07014     else {
07015         *s = c;
07016         MesPrint("@proper syntax is #Reset variable");
07017         return(-1);
07018     }
07019 }
07020
07021 /*
07022     #] DoPreReset :
07023     #[ DoPreAppendPath :
07024 */
07025
07026 static int DoAddPath(UBYTE *s, int bPrepend)
07027 {
07028     /* NOTE: this doesn't support some file systems, e.g., 0x5c with CP932. */
07029
07030     UBYTE *path, *path_end, *current_dir, *current_dir_end, *NewPath, *t;
07031     int bRelative, n;
07032
07033     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
07034     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
07035
07036     /* Parse the path in the input. */
07037     while ( *s == ' ' || *s == '\t' ) s++; /* skip spaces */
07038     if ( *s == '"' ) { /* the path is given by "..." */
07039         path = ++s;
07040         while ( *s && *s != '"' ) {
07041             if ( SEPARATOR != '\\ ' && *s == '\\ ' ) { /* escape character, e.g., "\\\" */
07042                 if ( !s[1] ) goto ImproperPath;
07043                 s++;
07044             }
07045             s++;
07046         }
07047         if ( *s != '"' ) goto ImproperPath;
07048         path_end = s++;
07049     }
07050     else {
07051         path = s;
07052         while ( *s && *s != ' ' && *s != '\t' ) {
07053             if ( SEPARATOR != '\\ ' && *s == '\\ ' ) { /* escape character, e.g., "\" */
07054                 if ( !s[1] ) goto ImproperPath;
07055                 s++;
07056             }
07057             s++;
07058         }
07059         path_end = s;
07060     }

```

```

07061     if ( path == path_end ) goto ImproperPath; /* empty path */
07062     while ( *s == ' ' || *s == '\t' ) s++; /* skip spaces */
07063     if ( *s ) goto ImproperPath; /* extra tokens found */
07064
07065     /* Check if the path is an absolute path. */
07066     bRelative = 1;
07067     if ( path[0] == SEPARATOR ) { /* starts with the directory separator */
07068         bRelative = 0;
07069     }
07070 #ifndef WINDOWS
07071     else if ( chartype[path[0]] == 0 && path[1] == ':' ) { /* starts with (drive letter): */
07072         bRelative = 0;
07073     }
07074 #endif
07075
07076     /* Get the current file directory when a relative path is given. */
07077     if ( bRelative ) {
07078         if ( !AC.CurrentStream ) goto FileNameUnavailable;
07079         if ( AC.CurrentStream->type != FILESTREAM && AC.CurrentStream->type != REVERSEFILESTREAM )
07080             goto FileNameUnavailable;
07081         if ( !AC.CurrentStream->name ) goto FileNameUnavailable;
07082         s = current_dir = current_dir_end = AC.CurrentStream->name;
07083         while ( *s ) {
07084             if ( SEPARATOR != '\\ ' && *s == '\\ ' && s[1] ) { /* escape character, e.g., "\\\" */
07085                 s += 2;
07086                 continue;
07087             }
07088             if ( *s == SEPARATOR ) {
07089                 current_dir_end = s;
07090             }
07091             s++;
07092         }
07093     }
07094     else {
07095         current_dir = current_dir_end = NULL;
07096     }
07097     /* Allocate a buffer for new AM.Path. */
07098     n = path_end - path;
07099     if ( AM.Path ) n += StrLen(AM.Path) + 1;
07100     if ( current_dir != current_dir_end ) n += current_dir_end - current_dir + 1;
07101     s = NewPath = (UBYTE *)Malloca(n + 1, "add path");
07102
07103     /* Construct new FORM path. */
07104     if ( bPrepend ) {
07105         if ( current_dir != current_dir_end ) {
07106             t = current_dir;
07107             while ( t != current_dir_end ) *s++ = *t++;
07108             *s++ = SEPARATOR;
07109         }
07110         t = path;
07111         while ( t != path_end ) *s++ = *t++;
07112         if ( AM.Path ) *s++ = PATHSEPARATOR;
07113     }
07114     if ( AM.Path ) {
07115         t = AM.Path;
07116         while ( *t ) *s++ = *t++;
07117     }
07118     if ( !bPrepend ) {
07119         if ( AM.Path ) *s++ = PATHSEPARATOR;
07120         if ( current_dir != current_dir_end ) {
07121             t = current_dir;
07122             while ( t != current_dir_end ) *s++ = *t++;
07123             *s++ = SEPARATOR;
07124         }
07125         t = path;
07126         while ( t != path_end ) *s++ = *t++;
07127     }
07128     *s = '\0';
07129
07130     /* Update AM.Path. */
07131     if ( AM.Path ) M_free(AM.Path, "add path");
07132     AM.Path = NewPath;
07133
07134     return(0);
07135
07136 ImproperPath:
07137     MesPrint("@Improper syntax for %sPath", bPrepend ? "Prepend" : "Append");
07138     return(-1);
07139
07140 FileNameUnavailable:
07141     /* This may be improved in future. */
07142     MesPrint("@Sorry, %sPath can't resolve the current file name from here", bPrepend ? "Prepend" :
07143 "Append");
07144     return(-1);
07145 }

```

```

07153 int DoPreAppendPath(UBYTE *s)
07154 {
07155     return DoAddPath(s, 0);
07156 }
07157
07158 /*
07159     #] DoPreAppendPath :
07160     #[ DoPrePrependPath :
07161 */
07162
07170 int DoPrePrependPath(UBYTE *s)
07171 {
07172     return DoAddPath(s, 1);
07173 }
07174
07175 /*
07176     #] DoPrePrependPath :
07177     #[ DoTimeOutAfter :
07178
07179     Executes the #timeoutafter number
07180 */
07181
07182 int DoTimeOutAfter(UBYTE *s)
07183 {
07184     #ifdef WITH_ALARM
07185         ULONG x;
07186     #else
07187         DUMMYUSE(s);
07188     #endif
07189     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
07190     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
07191     #ifdef WITH_ALARM
07192         while ( *s == ' ' || *s == '\t' ) s++;
07193         x = 0;
07194         while ( FG.cTable[*s] == 1 ) {
07195             x = 10*x + (*s++-'0');
07196         }
07197         while ( *s == ' ' || *s == '\t' ) s++;
07198         if ( *s == 0 ) {
07199             alarm(x);
07200             return(0);
07201         }
07202         else {
07203             MesPrint("@proper syntax is #TimeoutAfter number");
07204             return(-1);
07205         }
07206     #else
07207         Error0("#timeoutafter not implemented on this computer/system");
07208         return(-1);
07209     #endif
07210 }
07211
07212 /*
07213     #] DoTimeOutAfter :
07214     #[ DoNamespace :
07215
07216     Syntax:
07217         #Namespace name
07218         .....
07219         #use variables
07220         .....
07221         #EndNamespace
07222     Effect:
07223         All variables/expressions defined inside the range of the
07224         namespace get name_ prepended.
07225         This holds also for $-variables, names of procedures and
07226         names of files.
07227     Namespaces can be used in a nested way. cf this_is_deep_x
07228     A leading _ takes the role of what is super:: in some other languages.
07229     Remarks:
07230         Names of preprocessor variables are excluded!
07231         Names of built in objects are excluded! (like sum_, d_ etc.)
07232 */
07233
07234 int DoNamespace(UBYTE *s)
07235 {
07236     UBYTE *s1, *s2, c;
07237     NAMESPACE *namespace;
07238     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
07239     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
07240     while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
07241     if ( FG.cTable[*s] != 0 ) {
07242         MesPrint("@Illegal name in #namespace instruction: %s",s);
07243         return(-1);
07244     }
07245     s1 = s;
07246     while ( FG.cTable[*s1] <= 1 ) s1++;

```

```

07247     s2 = s1;
07248     while ( *s2 == ' ' || *s2 == ',' || *s2 == '\t' ) s2++;
07249     if ( *s2 != 0 ) {
07250         MesPrint("@A #namespace instruction can only have one name with only alphanumeric
characters.");
07251         return(-1);
07252     }
07253     c = *s1; *s1 = 0;
07254 /*
07255     Now we have the name and the statement is legal.
07256     We can proceed creating the namespace and its use tree.
07257 */
07258     namespace = (NAMESPACE *)Malloc1(sizeof(NAMESPACE),"namespace");
07259     namespace->name = strdup1(s,"namespace_name");
07260     namespace->usenames = MakeNameTree();
07261     if ( AP.firstnamespace == 0 ) {
07262         namespace->previous = 0;
07263         namespace->next = 0;
07264         AP.firstnamespace = namespace;
07265         AP.lastnamespace = namespace;
07266     }
07267     else {
07268         AP.lastnamespace->next = namespace;
07269         namespace->next = 0;
07270         namespace->previous = AP.lastnamespace;
07271         AP.lastnamespace = namespace;
07272     }
07273     *s1 = c;
07274     return(0);
07275 }
07276
07277 /*
07278     #] DoNamespace :
07279     #[ DoEndNamespace :
07280 */
07281
07282 int DoEndNamespace(UBYTE *s)
07283 {
07284     NAMESPACE *namespace;
07285     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
07286     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
07287     while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
07288     if ( *s != 0 ) {
07289         MesPrint("@Illegal #endnamespace instruction");
07290         return(-1);
07291     }
07292     namespace = AP.lastnamespace;
07293     AP.lastnamespace = namespace->previous;
07294     M_free(namespace->name,"namespace_name");
07295     FreeNameTree(namespace->usenames);
07296     M_free(namespace,"namespace");
07297     return(0);
07298 }
07299
07300 /*
07301     #] DoEndNamespace :
07302     #[ SkipName :
07303 */
07304
07305 UBYTE *SkipName(UBYTE *s)
07306 {
07307     UBYTE *t = s, *s1, c;
07308     int num = 0, block = 0;
07309     if ( *s == '[' ) {
07310         straight:
07311         SKIPBRA1(s)
07312         if ( *s == 0 ) {
07313             MesPrint("&Illegal name: '%s'",t);
07314             return(0);
07315         }
07316         s++; s1 = s;
07317         while ( FG.cTable[*s] <= 1 || *s == '_' ) s++;
07318         if ( s1 != s ) goto witherror;
07319     }
07320     else if ( *s == '$' ) {
07321         s++;
07322         while ( *s == '_' ) { s++; num++; }
07323         block = 1;
07324         while ( FG.cTable[*s] <= 1 || *s == '_' ) {
07325             if ( FG.cTable[*s] != 0 && block == 1 ) {
07326                 blocked:
07327                 MesPrint("&Illegally formed name: %s",t);
07328                 return(0);
07329             }
07330             if ( *s == '_' ) { num++; block = 1; }
07331             else block = 0;
07332             s++;

```

```

07333     }
07334     if ( s[-1] == '_' && num > 1 ) goto built;
07335 }
07336 else if ( FG.cTable[*s] == 0 ) {
07337 regular:
07338     while ( FG.cTable[*s] <= 1 || *s == '_' ) {
07339         if ( FG.cTable[*s] != 0 && block == 1 ) goto blocked;
07340         if ( *s == '_' ) { block = 1; num++; }
07341         else block = 0;
07342         s++;
07343     }
07344     if ( *s == '[' ) goto straight;
07345     if ( s[-1] == '_' && num > 1 ) {
07346 built:
07347         c = *s; *s = 0;
07348         MesPrint("&Built in objects cannot be part of namespaces: %s",t);
07349         *s = c;
07350         return(0);
07351     }
07352 }
07353 else if ( *s == '_' ) {
07354     while ( *s == '_' ) { s++; num++; }
07355     if ( FG.cTable[*s] == 0 ) { block = 0; goto regular; }
07356     goto witherror;
07357 }
07358 else if ( *s == '@' ) {
07359     s++; block = 1;
07360     if ( *s == '_' || FG.cTable[*s] == 1 ) {
07361         MesPrint("@Illegally formed name: %s",s-1);
07362     }
07363     goto regular;
07364 }
07365 else if ( *s == '#' ) { /* name of a procedure */
07366     s++;
07367     while ( *s == '_' ) { s++; num++; }
07368     block = 1;
07369     while ( FG.cTable[*s] <= 1 || *s == '_' ) {
07370         if ( FG.cTable[*s] != 0 && block == 1 ) goto blocked;
07371         if ( *s == '_' ) { num++; block = 1; }
07372         else block = 0;
07373         s++;
07374     }
07375     if ( s[-1] == '_' ) {
07376 witherror:
07377         c = *s; *s = 0;
07378         MesPrint("&Illegally formed name: %s",t);
07379         *s = c;
07380         return(0);
07381     }
07382 }
07383 else if ( *s == '<' ) { /* name of a file. Can be anything (more or less) */
07384     s++;
07385     while ( *s && *s != '>' ) s++;
07386     if ( *s != '>' ) goto witherror;
07387     s++;
07388 }
07389 return(s);
07390 }
07391
07392 /*
07393     #] SkipName :
07394     #[ ConstructName :
07395
07396     Routine gets a 'raw' name and modifies it if the namespace
07397     settings ask for it. It puts the new name in a buffer that
07398     may be expanded if the names become rather long.
07399     Note that eventually that name needs to be copied, because
07400     we do not allocate new buffers for each name.
07401
07402     type tells what kind of name we look for
07403 */
07404
07405 UBYTE *ConstructName(UBYTE *s,UBYTE type)
07406 {
07407     int len;
07408     UBYTE *t, *u;
07409     WORD number;
07410     NAMESPACE *namespace;
07411     if ( AP.lastnamespace == 0 ) return(s);
07412     if ( *s == '@' ) return(s+1);
07413     if ( GetName(AP.lastnamespace->usenames,s,&number,NOAUTO) !=
07414         NAMENOTFOUND ) return(s);
07415 /*
07416     Now the real stuff
07417     First we have to compute the size of the new name.
07418 */
07419     len = StrLen(s) + 1;

```



```

07420     namespace = AP.firstnamespace;
07421     while ( namespace ) {
07422         len += StrLen(namespace->name)+1;
07423         namespace = namespace->previous;
07424     }
07425     if ( len > AP.fullnamespace ) {
07426         while ( len > AP.fullnamespace ) AP.fullnamespace *= 2;
07427         M_free(AP.fullname, "AP.fullname");
07428         AP.fullname = (UBYTE *)Malloc1(AP.fullnamespace*sizeof(UBYTE *), "AP.fullname");
07429     }
07430     namespace = AP.firstnamespace;
07431     t = AP.fullname;
07432     switch ( type ) {
07433         case ' ':
07434             case 0:
07435                 while ( namespace ) {
07436                     u = namespace->name;
07437                     while ( *u ) *t++ = *u++;
07438                     *t++ = ' ';
07439                     namespace = namespace->previous;
07440                 }
07441                 while ( *s ) *t++ = *s++;
07442                 *t = 0;
07443                 break;
07444             case '$':
07445             case '#':
07446                 *t++ = type;
07447                 while ( namespace ) {
07448                     u = namespace->name;
07449                     while ( *u ) *t++ = *u++;
07450                     *t++ = ' ';
07451                     namespace = namespace->previous;
07452                 }
07453                 if ( type == '$' ) s++;
07454                 while ( *s ) *t++ = *s++;
07455                 *t = 0;
07456                 break;
07457             case '<':
07458                 while ( namespace ) {
07459                     u = namespace->name;
07460                     while ( *u ) *t++ = *u++;
07461                     *t++ = ' ';
07462                     namespace = namespace->previous;
07463                 }
07464                 s++;
07465                 while ( *s ) *t++ = *s++;
07466                 t--; /* strip the '>' */
07467                 *t = 0;
07468                 break;
07469             default:
07470                 MesPrint("&Unrecognized datatype in ConstructName");
07471                 *t = 0;
07472                 break;
07473     }
07474     return(AP.fullname);
07475 }
07476
07477 /*
07478     #] ConstructName :
07479     #[ DoUse :
07480
07481     Routine makes (inside the confines of the current namespace)
07482     a list of variables that are excluded from the namespace.
07483     Once the namespace is ended, the list is removed.
07484     The list can include names of variables, dollars and procedures.
07485     Preprocessor variables are excluded from the namespace. Their
07486     inclusion would be too complicated for the input streams.
07487     Note that the preprocessor variables that are arguments in a #do
07488     or in a procedure are on a stack and should not cause problems.
07489     Names of variables can be just that.
07490     Names of $-variables are also straightforward.
07491     Names of procedures should be preceded by a # character.
07492     Names of files (like in #include) should be enclosed by <>.
07493     The names are stored in a balanced tree. Each namespace may have
07494     its own tree. The toplevel (no namespace) does not allow a #use.
07495 */
07496
07497 int DoUse(UBYTE *s)
07498 {
07499     NAMESPACE *namespace;
07500     UBYTE *t, c;
07501     int number;
07502     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
07503     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
07504     if ( AP.lastnamespace == 0 ) {
07505         MesPrint("@It is not allowed to use #use outside the scope of a namespace.");
07506         return(-1);

```

```

07507     }
07508     namespace = AP.lastnamespace;
07509     while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
07510     while ( *s ) {
07511         t = s;
07512         if ( ( s = SkipName(t) ) == 0 ) return(-1);
07513         if ( s == t ) {
07514             MesPrint("@Unrecognized object in #use instruction: %s",t);
07515             return(-1);
07516         }
07517         c = *s; *s = 0;
07518     /*
07519         In usernames we only need the names to know whether they are 'protected'.
07520         We need to keep the $, # and <> to avoid potential double names.
07521     */
07522         AddName(namespace->usernames,t,0,0,&number);
07523         *s = c;
07524         while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
07525     }
07526     return(0);
07527 }
07528
07529 /*
07530     #] DoUse :
07531     #[ UserFlags :
07532
07533     Syntax:
07534     #ClearFlag number(s),expression(s)
07535     #ClearFlag number(s)
07536     #ClearFlag expression(s)
07537     #ClearFlag
07538     #SetFlag number(s),expression(s)
07539     #SetFlag number(s)
07540     #SetFlag expression(s)
07541     #SetFlag
07542     par == 0: Clear, par == 1: Set.
07543 */
07544
07545 int UserFlags(UBYTE *s,int par)
07546 {
07547     int mask = 0, error = 0, i;
07548     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
07549     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
07550     while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
07551     if ( *s == 0 ) { /* Treat all flags in all active expressions */
07552         allexpr:
07553         for ( i = 0; i < NumExpressions; i++ ) {
07554             switch ( Expressions[i].status ) {
07555                 case UNHIDELEXPRESSION:
07556                 case UNHIDEGEXPRESSION:
07557                 case INTOHIDELEXPRESSION:
07558                 case INTOHIDEGEXPRESSION:
07559                 case LOCALEXPRESSION:
07560                 case GLOBALEXPRESSION:
07561                 case SKIPLEXPRESSION:
07562                 case SKIPGEXPRESSION:
07563                 case HIDELEXPRESSION:
07564                 case HIDEGEXPRESSION:
07565                     if ( par == 1 ) Expressions[i].uflags |= ~mask;
07566                     else Expressions[i].uflags &= mask;
07567                     break;
07568                 case DROPEDEXPRESSION:
07569                 case DROPLEXPRESSION:
07570                 case DROPGEXPRESSION:
07571                 case DROPHGEXPRESSION:
07572                 case STOREDEXPRESSION:
07573                 case HIDDENLEXPRESSION:
07574                 case HIDDENGEXPRESSION:
07575                 case SPECTATOREXPRESSION:
07576                 default:
07577                     break;
07578             }
07579         }
07580     }
07581     else if ( FG.cTable[*s] == 1 ) {
07582         mask = (int)WORDMASK;
07583         while ( FG.cTable[*s] == 1 ) {
07584             int x = 0;
07585             while ( FG.cTable[*s] == 1 ) { x = 10*x + (*s++-'0'); }
07586             if ( x < 1 || x > BITSINWORD ) {
07587                 MesPrint("@Illegal number %d for flag in #...Flag instruction",x);
07588                 return(1);
07589             }
07590             mask ^= (1<<(x-1));
07591             while ( *s == ' ' || *s == ',' || *s == '\t' ) s++;
07592         }
07593     }

```

```

07594         if ( *s == 0 ) goto allexp;
07595     }
07596     else { /* Clear all flags in all expressions that are specified */
07597         mask = (int)WORDMASK;
07598     }
07599     while ( *s ) { /* now read the expressions */
07600         UBYTE *s1, c;
07601         WORD num1;
07602         if ( FG.cTable[*s] != 0 && *s != '[' ) goto syntax;
07603         s1 = s; s = SkipAName(s);
07604         c = *s; *s = 0;
07605         if ( GetName(AC.exprnames,s1,&num1,NOAUTO) != CEXPRESSION ) {
07606             MesPrint("@%s is not an active expression",s1);
07607             error = 1;
07608             return(error);
07609         }
07610         switch ( Expressions[num1].status ) {
07611             case UNHIDELEXPRESSION:
07612             case UNHIDEGEXPRESSION:
07613             case INTOHIDELEXPRESSION:
07614             case INTOHIDEGEXPRESSION:
07615             case LOCALEXPRESSION:
07616             case GLOBALEXPRESSION:
07617             case SKIPLEXPRESSION:
07618             case SKIPGEXPRESSION:
07619             case HIDELEXPRESSION:
07620             case HIDEGEXPRESSION:
07621             if ( par == 1 ) Expressions[num1].uflags |= ~mask;
07622             else Expressions[num1].uflags &= mask;
07623             break;
07624             case DROPEDEXPRESSION:
07625             case DROPLEXPRESSION:
07626             case DROPGEXPRESSION:
07627             case DROPHLEXPRESSION:
07628             case DROPHGEXPRESSION:
07629             case STOREDEXPRESSION:
07630             case HIDDENLEXPRESSION:
07631             case HIDDENGEXPRESSION:
07632             case SPECTATOREXPRESSION:
07633             default:
07634                 MesPrint("@%s is not an active expression",s1);
07635                 error = 1;
07636                 break;
07637         }
07638         *s = c;
07639         while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
07640     }
07641     return(error);
07642 syntax:
07643     MesPrint("@Illegal name in #...Flag instruction.");
07644     return(1);
07645 }
07646
07647 /*
07648     #] UserFlags :
07649     #[ DoClearUserFlag :
07650 */
07651
07652 int DoClearUserFlag(UBYTE *s)
07653 {
07654     return(UserFlags(s,0));
07655 }
07656
07657 /*
07658     #] DoClearUserFlag :
07659     #[ DoSetUserFlag :
07660 */
07661
07662 int DoSetUserFlag(UBYTE *s)
07663 {
07664     return(UserFlags(s,1));
07665 }
07666
07667 /*
07668     #] DoSetUserFlag :
07669     #[ DoStartFloat :
07670
07671     If there is a number follwing, it will be the new default precision.
07672     If float has been started before, the old one will be removed first.
07673     If there are two numbers, the second one is the maximum weight for
07674     MZV's.
07675 */
07676 #ifdef WITHFLOAT
07677
07678 int DoStartFloat(UBYTE *s)
07679 {
07680     GETIDENTITY

```

```

07681     int error = 0;
07682     LONG x;
07683     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
07684     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
07685     if ( AR.PolyFun != 0 ) {
07686         MesPrint("@Simultaneous use of Poly(Rat)Fun and float_ is not allowed.");
07687         error = 1;
07688     }
07689     if ( AC.ncmod != 0 ) {
07690         MesPrint("@Simultaneous use of floating point and modulus arithmetic makes no sense.");
07691         error = 1;
07692     }
07693     while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
07694     if ( *s >= '0' && *s <= '9' ) {
07695         x = 0;
07696         do {
07697             x = 10*x + (*s++ - '0');
07698         } while ( *s >= '0' && *s <= '9' );
07699         AC.tDefaultPrecision = x;
07700         while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
07701         if ( *s >= '0' && *s <= '9' ) {
07702             x = 0;
07703             do {
07704                 x = 10*x + (*s++ - '0');
07705             } while ( *s >= '0' && *s <= '9' );
07706             AC.tMaxWeight = x;
07707             while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
07708         }
07709         else {
07710             AC.tMaxWeight = 0;
07711         }
07712         if ( *s ) goto IllPar;
07713     }
07714     else if ( *s != 0 ) {
07715 IllPar:
07716         MesPrint("@Illegal parameter in %#StartFloat instruction: %s ",s);
07717         error = 1;
07718     }
07719     if ( error == 0 ) {
07720         if ( AC.tDefaultPrecision && ( AC.tDefaultPrecision != AC.DefaultPrecision
07721         || AT.aux_ == 0 ) ) {
07722             AC.DefaultPrecision = AC.tDefaultPrecision;
07723             AC.tDefaultPrecision = 0;
07724         }
07725         if ( AC.tMaxWeight && ( AC.tMaxWeight != AC.MaxWeight
07726         || AT.aux_ == 0 ) ) {
07727             AC.MaxWeight = AC.tMaxWeight;
07728             AC.tMaxWeight = 0;
07729         }
07730         SetFloatPrecision(AC.DefaultPrecision+AC.MaxWeight+1);
07731         SetupMPFTables();
07732         if ( AC.MaxWeight > 0 ) SetupMZVTables();
07733         SetfFloatPrecision(AC.DefaultPrecision);
07734     }
07735     else {
07736         AC.tDefaultPrecision = 0;
07737         AC.tMaxWeight = 0;
07738     }
07739     return(error);
07740 }
07741
07742 #endif
07743 /*
07744     #] DoStartFloat :
07745     #[ DoEndFloat :
07746 */
07747 #ifdef WITHFLOAT
07748
07749 int DoEndFloat(UBYTE *s)
07750 {
07751     int error = 0;
07752     if ( AP.PreSwitchModes[AP.PreSwitchLevel] != EXECUTINGPRESWITCH ) return(0);
07753     if ( AP.PreIfStack[AP.PreIfLevel] != EXECUTINGIF ) return(0);
07754     while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
07755     if ( *s != 0 ) {
07756         MesPrint("@Illegal parameter in %#EndFloat instruction: %s ",s);
07757         error = 1;
07758     }
07759     if ( error == 0 ) {
07760         ClearfFloat();
07761         ClearMZVTables();
07762     }
07763     return(error);
07764 }
07765
07766 #endif
07767 /*

```

```

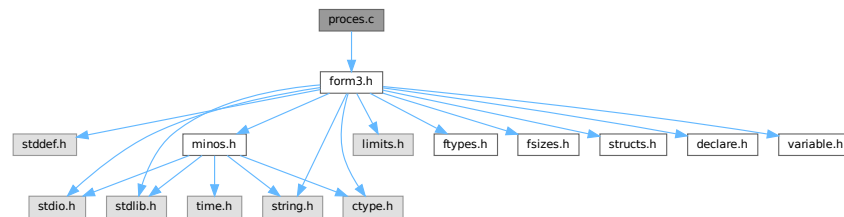
07768      #] DoEndFloat :
07769      # ] PreProcessor :
07770  */

```

4.114 proces.c File Reference

```
#include "form3.h"
```

Include dependency graph for proces.c:



Macros

- `#define DONE(x) { retvalue = x; goto Done; }`

Functions

- WORD [Processor](#) (void)
- WORD [TestSub](#) (PHEAD WORD *term, WORD level)
- WORD [InFunction](#) (PHEAD WORD *term, WORD *termout)
- WORD [InsertTerm](#) (PHEAD WORD *term, WORD replac, WORD extractbuff, WORD *position, WORD *termout, WORD tepos)
- LONG [PasteFile](#) (PHEAD WORD number, WORD *accum, [POSITION](#) *position, WORD **accfill, [RENUMBER](#) renumber, WORD *freeze, WORD nexpr)
- WORD * [PasteTerm](#) (PHEAD WORD number, WORD *accum, WORD *position, WORD times, WORD divby)
- WORD [FiniTerm](#) (PHEAD WORD *term, WORD *accum, WORD *termout, WORD number, WORD tepos)
- WORD [Generator](#) (PHEAD WORD *term, WORD level)
- WORD [DoOnePow](#) (PHEAD WORD *term, WORD power, WORD nexpr, WORD *accum, WORD *aa, WORD level, WORD *freeze)
- WORD [Deferred](#) (PHEAD WORD *term, WORD level)
- WORD [PrepPoly](#) (PHEAD WORD *term, WORD par)
- WORD [PolyFunMul](#) (PHEAD WORD *term)

Variables

- WORD [printscratch](#) [2]

4.114.1 Detailed Description

Contains the central terms processor routines. This is the core of the virtual machine. All other files are to help these routines.

Definition in file [proces.c](#).

4.114.2 Macro Definition Documentation

4.114.2.1 DONE

```
#define DONE(
    x ) { retvalue = x; goto Done; }
```

TestSub hunts for subexpression pointers. If one is found its power is given in AN.TeSuOut. and the returnvalue is 'expressionnumber'. If the expression number is negative it is an expression on disk.

In addition this routine tries to locate subexpression pointers in functions. It also notices that action must be taken with any of the special functions.

Parameters

<i>term</i>	The term in which TestSub hunts for potential action
<i>level</i>	The number of the 'level' in the compiler buffer.

Returns

The number of the (sub)expression that was encountered.

Other values that are returned are in AN.TeSuOut, AR.TePos, AT.TMbuff, AN.TelFun, AN.Frozen, AT.TMaddr

The level in the compiler buffer is more or less the number of the statement in the module. Hence it refers to the element in the lhs array.

This routine is one of the most important routines in FORM.

Definition at line 682 of file [proces.c](#).

4.114.3 Function Documentation

4.114.3.1 Processor()

```
WORD Processor (
    void )
```

This is the central processor. It accepts a stream of Expressions which is accessed by calls to GetTerm. The expressions reside either in AR.infile or AR.hidefile The definitions of an expression are seen as an id-statement, so the primary Expressions should be written to the system of scratch files as single terms with an expression pointer. Each expression is terminated with a zero and the whole is terminated by two zeroes.

The routine DoExecute should determine whether results are to be printed, should revert the scratch I/O directions etc. In principle it is DoExecute that calls Processor.

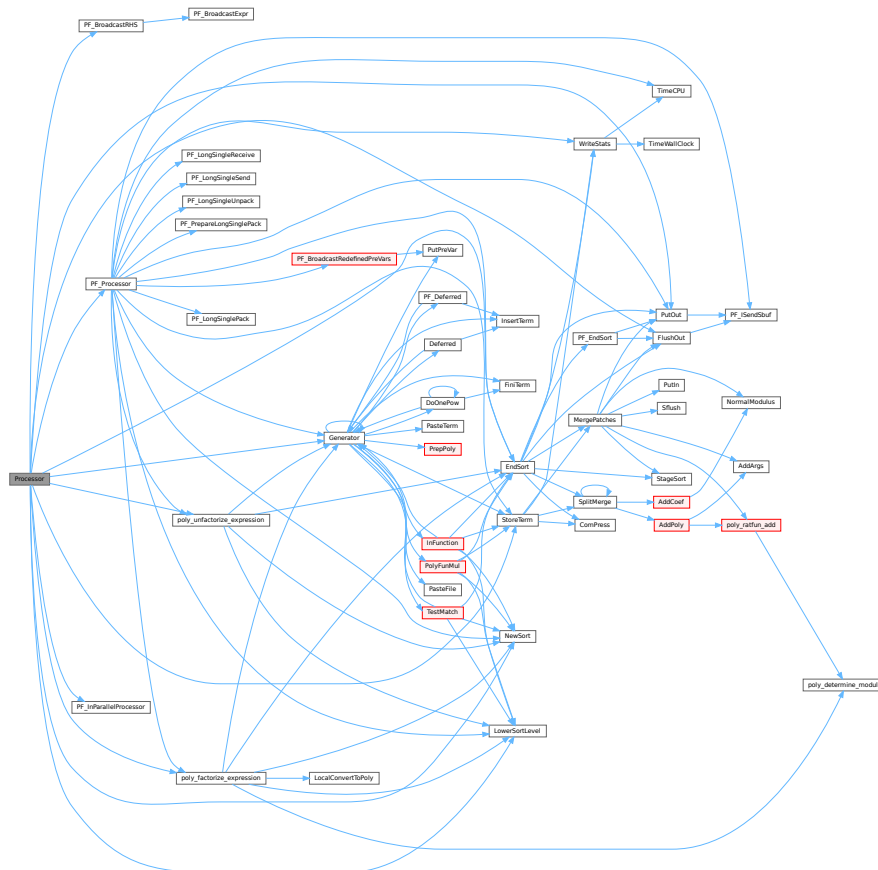
Returns

if everything OK: 0. Otherwise error. The preprocessor may continue with compilation though. Really fatal errors should return on the spot by calling `Terminate`.

Definition at line 64 of file [proces.c](#).

References [CbUf::Buffer](#), [EndSort\(\)](#), [FlushOut\(\)](#), [Generator\(\)](#), [CbUf::lhs](#), [LowerSortLevel\(\)](#), [NewSort\(\)](#), [PF_BroadcastRHS\(\)](#), [PF_InParallelProcessor\(\)](#), [PF_Processor\(\)](#), [CbUf::Pointer](#), [poly_factorize_expression\(\)](#), [poly_unfactorize_expression\(\)](#), [PutOut\(\)](#), [CbUf::rhs](#), and [StoreTerm\(\)](#).

Here is the call graph for this function:



4.114.3.2 TestSub()

```
WORD TestSub (
    PHEAD WORD * term,
    WORD level )
```

Definition at line 684 of file [proces.c](#).

4.114.3.3 InFunction()

```
WORD InFunction (
    PHEAD WORD * term,
    WORD * termout )
```

Makes the replacement of the subexpression with the number 'replac' in a function argument. Additional information is passed in some of the AR, AN, AT variables.

Parameters

<i>term</i>	The input term
<i>termout</i>	The output term

Returns

0: everything is fine, Negative: fatal, Positive: error.

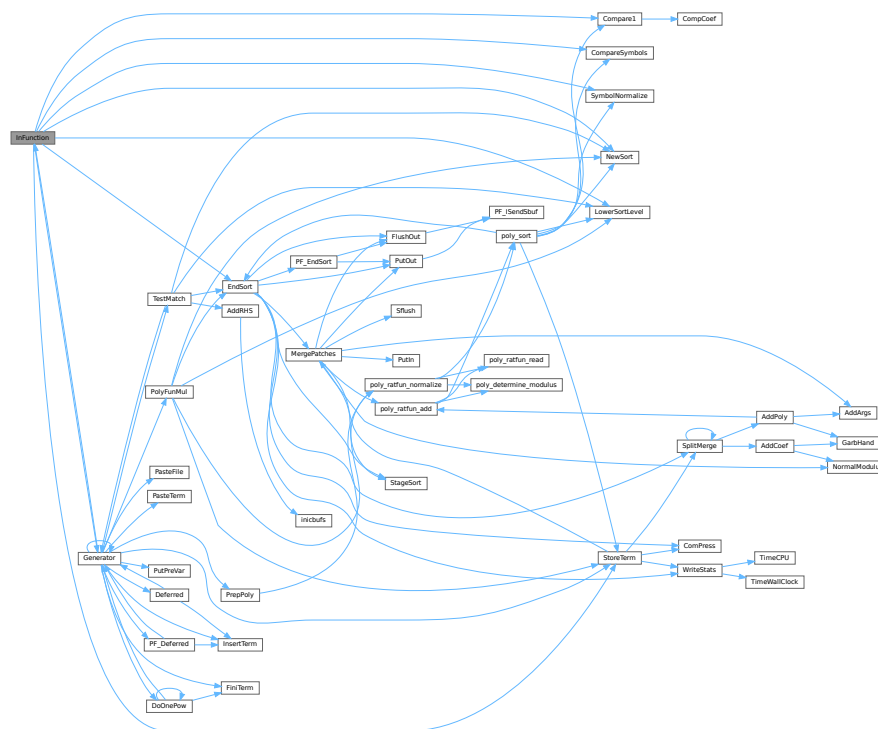
Special attention should be given to nested functions!

Definition at line 2174 of file [proces.c](#).

References [Compare1\(\)](#), [CompareSymbols\(\)](#), [EndSort\(\)](#), [Generator\(\)](#), [LowerSortLevel\(\)](#), [NewSort\(\)](#), [StoreTerm\(\)](#), and [SymbolNormalize\(\)](#).

Referenced by [Generator\(\)](#).

Here is the call graph for this function:



4.114.3.4 InsertTerm()

```
WORD InsertTerm (
    PHEAD WORD * term,
    WORD replac,
    WORD extractbuff,
    WORD * position,
```



```
WORD * termout,  
WORD tepos )
```

Puts the terms 'term' and 'position' together into a single legal term in termout. replac is the number of the subexpression that should be replaced. It must be a positive term. When action is needed in the argument of a function all terms in that argument are dealt with recursively. The subexpression is sorted. Only one subexpression is done at a time this way.

Parameters

<i>term</i>	the input term
<i>replac</i>	number of the subexpression pointer to replace
<i>extractbuff</i>	number of the compiler buffer <i>replac</i> refers to
<i>position</i>	position from where to take the term in the compiler buffer
<i>termout</i>	the output term
<i>tepos</i>	offset in term where the subexpression is.

Returns

Normal conventions (OK = 0).

Definition at line 2744 of file [proces.c](#).

Referenced by [Deferred\(\)](#), [Generator\(\)](#), and [PF_Deferred\(\)](#).

4.114.3.5 PasteFile()

```

LONG PasteFile (
    PHEAD WORD number,
    WORD * accum,
    POSITION * position,
    WORD ** accfill,
    RENUMBER renumber,
    WORD * freeze,
    WORD nexpr )

```

Gets a term from stored expression *expr* and puts it in the accumulator at position *number*. It returns the length of the term that came from file.

Parameters

<i>number</i>	number of partial terms to skip in <i>accum</i>
<i>accum</i>	the accumulator
<i>position</i>	file position from where to get the stored term
<i>accfill</i>	returns tail position in <i>accum</i>
<i>renumber</i>	the renumber struct for the variables in the stored expression
<i>freeze</i>	information about if we need only the contents of a bracket
<i>nexpr</i>	the number of the stored expression

Returns

Normal conventions (OK = 0).

Definition at line 2880 of file [proces.c](#).

Referenced by [Generator\(\)](#).

4.114.3.6 PasteTerm()

```
WORD * PasteTerm (
    PHEAD WORD number,
    WORD * accum,
    WORD * position,
    WORD times,
    WORD divby )
```

Puts the term at position in the accumulator *accum* at position '*number*+1'. if *times* > 0 the coefficient of this term is multiplied by *times*/*divby*.

Parameters

<i>number</i>	The number of term fragments in <i>accum</i> that should be skipped
<i>accum</i>	The accumulator of term fragments
<i>position</i>	A position in (typically) a compiler buffer from where a (piece of a) term comes.
<i>times</i>	Multiply the result by this
<i>divby</i>	Divide the result by this.

This routine is typically used when we have to replace a (sub)expression pointer by a power of a (sub)expression. This uses mostly a binomial expansion and the new term is the old term multiplied one by one by terms of the new expression. The factors *times* and *divby* keep track of the binomial coefficient. Once this is complete, the routine *FiniTerm* will make the contents of the accumulator into a proper term that still needs to be normalized.

Definition at line 3002 of file [proces.c](#).

Referenced by [Generator\(\)](#).

4.114.3.7 FiniTerm()

```
WORD FiniTerm (
    PHEAD WORD * term,
    WORD * accum,
    WORD * termout,
    WORD number,
    WORD tepos )
```

Concatenates the contents of the accumulator into a single legal term, which replaces the subexpression pointer

Parameters

<i>term</i>	the input term with the (sub)expression subterm
<i>accum</i>	the accumulator with the term fragments
<i>termout</i>	the location where the output should be written
<i>number</i>	the number of term fragments in the accumulator
<i>tepos</i>	the position of the subterm in term to be replaced

Definition at line 3067 of file [proces.c](#).

Referenced by [DoOnePow\(\)](#), and [Generator\(\)](#).

4.114.3.8 Generator()

```
WORD Generator (
    PHEAD WORD * term,
    WORD level )
```

The heart of the program. Here the expansion tree is set up in one giant recursion

Parameters

<i>term</i>	the input term. may be overwritten
<i>level</i>	the level in the compiler buffer (number of statement)

Returns

Normal conventions (OK = 0).

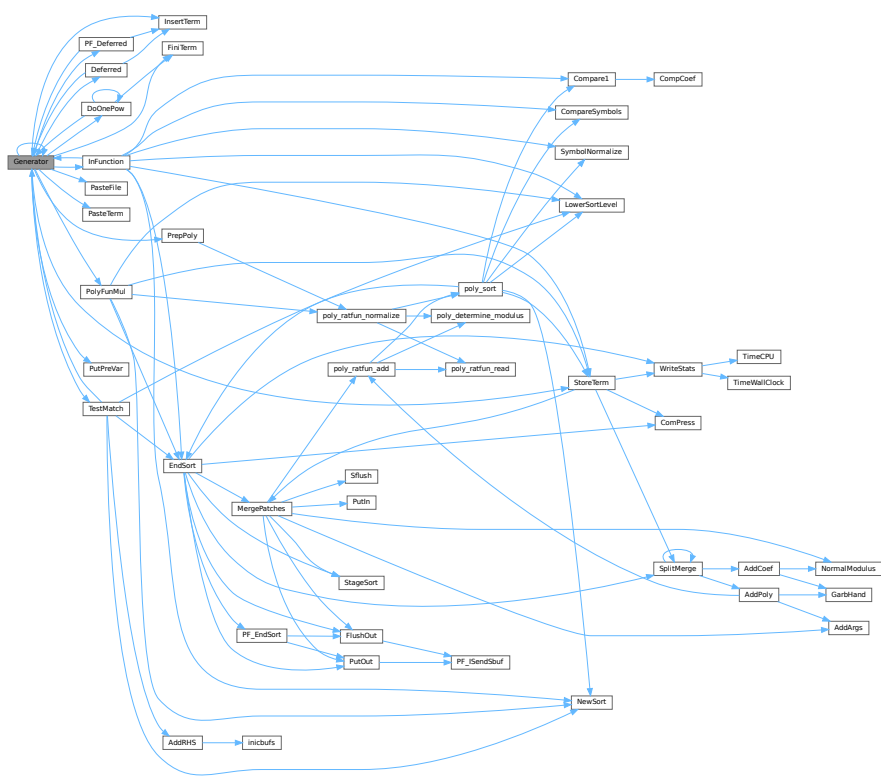
The routine looks first whether there are unsubstituted (sub)expressions. If so, one of them gets inserted term by term and the new term is used in a renewed call to Generator. If there are no (sub)expressions, the term is normalized, the compiler level is raised (next statement) and the program looks what type of statement this is. If this is a special statement it is either treated on the spot or the appropriate routine is called. If it is a substitution, the pattern matcher is called (TestMatch) which tells whether there was a match. If so we need to call TestSub again to test for (sub)expressions. If we run out of levels, the term receives a final treatment for modulus calculus and/or brackets and is then sent off to the sorting routines.

Definition at line 3266 of file [proces.c](#).

References [Deferred\(\)](#), [DoOnePow\(\)](#), [FiniTerm\(\)](#), [Generator\(\)](#), [InFunction\(\)](#), [InsertTerm\(\)](#), [CbUf::lhs](#), [VaRrEnUm::lo](#), [PasteFile\(\)](#), [PasteTerm\(\)](#), [PF_Deferred\(\)](#), [CbUf::Pointer](#), [PolyFunMul\(\)](#), [PrepPoly\(\)](#), [PutPreVar\(\)](#), [CbUf::rhs](#), [StoreTerm\(\)](#), [ReNuMbEr::symb](#), and [TestMatch\(\)](#).

Referenced by [Deferred\(\)](#), [DoOnePow\(\)](#), [generate_expression\(\)](#), [Generator\(\)](#), [InFunction\(\)](#), [MakeDollarInteger\(\)](#), [MakeDollarMod\(\)](#), [PF_CollectModifiedDollars\(\)](#), [PF_Deferred\(\)](#), [PF_Processor\(\)](#), [poly_factorize_expression\(\)](#), [poly_unfactorize_expression\(\)](#), [Processor\(\)](#), and [TestMatch\(\)](#).

Here is the call graph for this function:



4.114.3.9 DoOnePow()

```
WORD DoOnePow (
    PHEAD WORD * term,
    WORD power,
    WORD nexp,
    WORD * accum,
    WORD * aa,
    WORD level,
    WORD * freeze )
```

Routine gets one power of an expression in the scratch system. If there are more powers needed there will be a recursion.

No attempt is made to use binomials because we have no information about commuting properties.

There is a searching for the contents of brackets if needed. This searching may be rather slow because of the single links.

Parameters

<i>term</i>	is the term we are adding to.
<i>power</i>	is the power of the expression that we need.
<i>nexp</i>	is the number of the expression.
<i>accum</i>	is the accumulator of terms. It accepts the termfragments that are made into a proper term in FiniTerm

Parameters

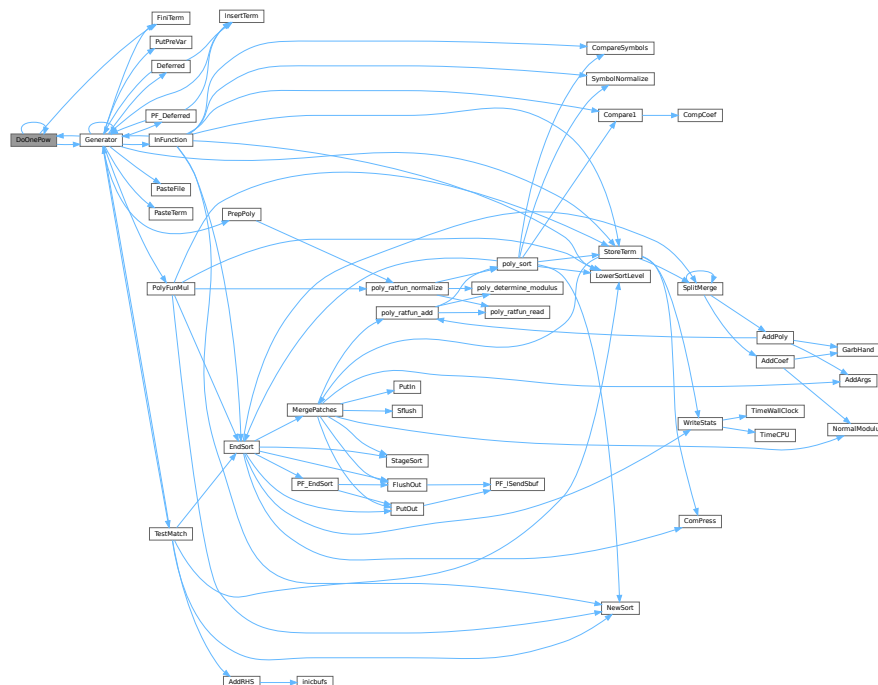
<i>aa</i>	points to the start of the entire accumulator. In *aa we store the number of term fragments that are in the accumulator.
<i>level</i>	is the current depth in the tree of statements. It is needed to continue to the next operation/substitution with each generated term
<i>freeze</i>	is the pointer to the bracket information that should be matched.

Definition at line 4623 of file [proces.c](#).

References [DoOnePow\(\)](#), [FiniTerm\(\)](#), [Generator\(\)](#), and [FiLe::handle](#).

Referenced by [DoOnePow\(\)](#), and [Generator\(\)](#).

Here is the call graph for this function:



4.114.3.10 Deferred()

```
WORD Deferred (
    PHEAD WORD * term,
    WORD level )
```

Picks up the deferred brackets. These are the bracket contents of which we postpone the reading when we use the 'Keep Brackets' statement. These contents are multiplying the terms just before they are sent to the sorting system. Special attention goes to having it thread-safe We have to lock positioning the file and reading it in a thread specific buffer.

Parameters

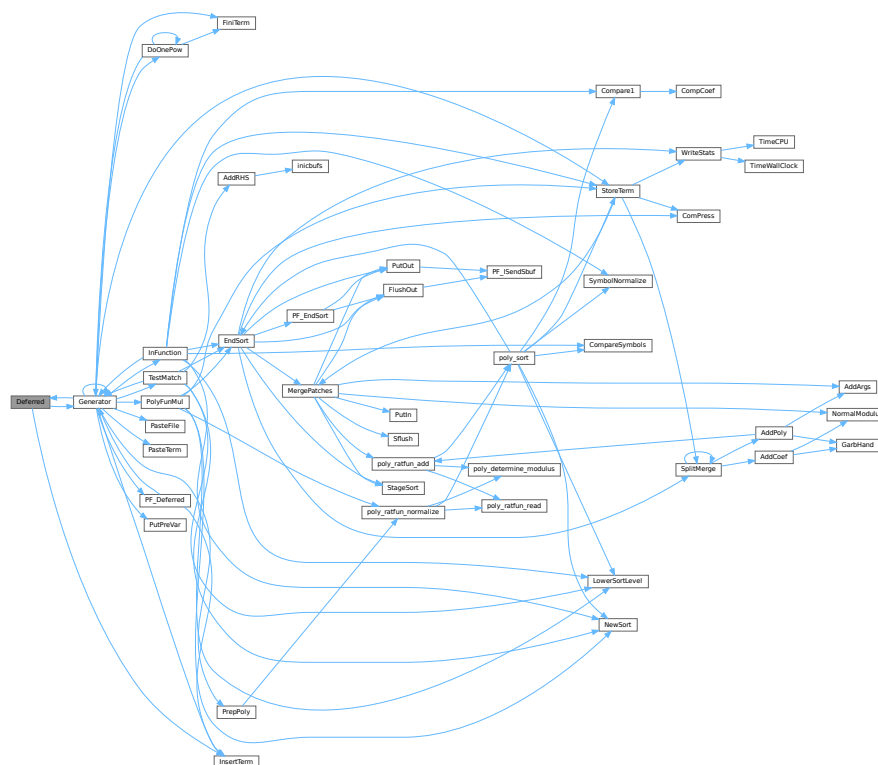
<i>term</i>	The term that must be multiplied by the contents of the current bracket
<i>level</i>	The compiler level. This is needed because after multiplying term by term we call Generator again.

Definition at line 4844 of file [proces.c](#).

References [Generator\(\)](#), and [InsertTerm\(\)](#).

Referenced by [Generator\(\)](#).

Here is the call graph for this function:



4.114.3.11 PrepPoly()

```

WORD PrepPoly (
    PHEAD WORD * term,
    WORD par )

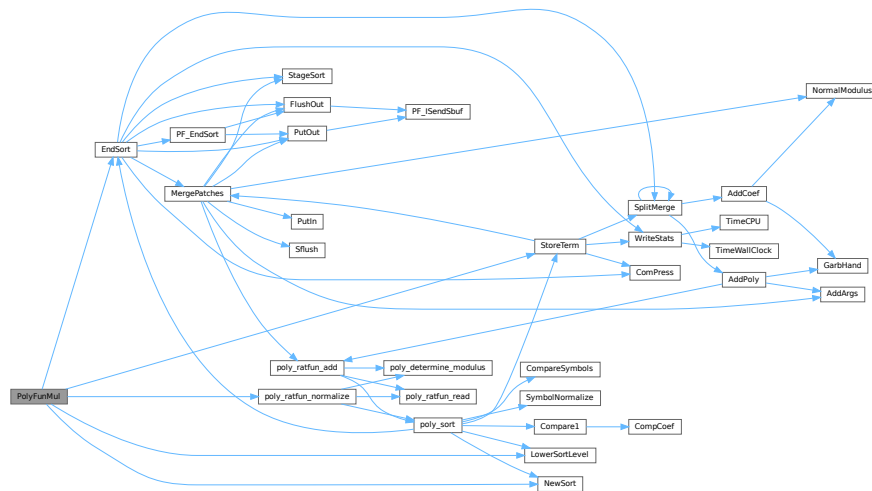
```

Routine checks whether the count of function AR.PolyFun is zero or one. If it is one and it has one scalarlike argument the coefficient of the term is pulled inside the argument. If the count is zero a new function is made with the coefficient as its only argument. The function should be placed at its proper position.

When this function is active it places the PolyFun as last object before the coefficient. This is needed because otherwise the compress algorithm has problems in MergePatches.

Referenced by [Generator\(\)](#).

Here is the call graph for this function:



4.114.4 Variable Documentation

4.114.4.1 printscratch

WORD printscratch[2]

Definition at line 39 of file [proces.c](#).

4.115 proces.c

[Go to the documentation of this file.](#)

```

00001
00006 /* # [ License : */
00007 /*
00008  * Copyright (C) 1984-2023 J.A.M. Vermaseren
00009  * When using this file you are requested to refer to the publication
00010  * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00011  * This is considered a matter of courtesy as the development was paid
00012  * for by FOM the Dutch physics granting agency and we would like to
00013  * be able to track its scientific use to convince FOM of its value
00014  * for the community.
00015  *
00016  * This file is part of FORM.
00017  *
00018  * FORM is free software: you can redistribute it and/or modify it under the
00019  * terms of the GNU General Public License as published by the Free Software
00020  * Foundation, either version 3 of the License, or (at your option) any later
00021  * version.
00022  *
00023  * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00024  * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00025  * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00026  * details.
00027  *
00028  * You should have received a copy of the GNU General Public License along
00029  * with FORM. If not, see <http://www.gnu.org/licenses/>.
00030  */
00031 /* # [ License : */
00032 /*
00033 #define HIDEDEBUG

```

```

00034     # [ Includes : proces.c
00035 */
00036
00037 #include "form3.h"
00038
00039 WORD printscratch[2];
00040
00041 /*
00042     #] Includes :
00043     # [ Processor :
00044         # [ Processor :          WORD Processor()
00045 */
00064 WORD Processor(void)
00065 {
00066     GETIDENTITY
00067     WORD *term, *t, i, retval = 0, size;
00068     EXPRESSIONS e;
00069     POSITION position;
00070     WORD last, LastExpression, fromspectator;
00071     LONG dd = 0;
00072     CBUF *C = cbuf+AC.cbunum;
00073     int firstterm;
00074     CBUF *CC = cbuf+AT.ebunum;
00075     WORD **w, *cpo, *cbo;
00076     FILEHANDLE *curfile, *oldoutfile = AR.outfile;
00077     WORD oldBracketOn = AR.BracketOn;
00078     WORD *oldBrackBuf = AT.BrackBuf;
00079     WORD oldbracketindexflag = AT.bracketindexflag;
00080 #ifdef WITHPTHREADS
00081     int OldMultiThreaded = AS.MultiThreaded, Oldmparallelflag = AC.mparallelflag;
00082 #endif
00083     if ( CC->numrhs > 0 || CC->numlhs > 0 ) {
00084         if ( CC->rhs ) {
00085             w = CC->rhs; i = CC->numrhs;
00086             do { *w++ = 0; } while ( --i > 0 );
00087         }
00088         if ( CC->lhs ) {
00089             w = CC->lhs; i = CC->numlhs;
00090             do { *w++ = 0; } while ( --i > 0 );
00091         }
00092         CC->numlhs = CC->numrhs = 0;
00093         ClearTree(AT.ebunum);
00094         CC->Pointer = CC->Buffer;
00095     }
00096
00097     if ( NumExpressions == 0 ) return(0);
00098     AR.expflags = 0;
00099     AR.CompressPointer = AR.CompressBuffer;
00100     AR.NoCompress = AC.NoCompress;
00101     term = AT.WorkPointer;
00102     if ( ( (WORD *)(((UBYTE *) (AT.WorkPointer)) + AM.MaxTer) ) > AT.WorkTop ) return(MesWork());
00103     UpdatePositions();
00104     C->rhs[C->numrhs+1] = C->Pointer;
00105     AR.KeptInHold = 0;
00106     if ( AC.CollectFun ) AR.DeferFlag = 0;
00107     AR.outtohide = 0;
00108     AN.PolyFunTodo = 0;
00109 #ifdef HIDEDEBUG
00110     MesPrint("Status at the start of Processor (HideLevel = %d)", AC.HideLevel);
00111     for ( i = 0; i < NumExpressions; i++ ) {
00112         e = Expressions+i;
00113         ExprStatus(e);
00114     }
00115 #endif
00116 /*
00117     Next determine the last expression. This is used for removing the
00118     input file when the final stage of the sort of this expression is
00119     reached. That can save up to 1/3 in disk space.
00120 */
00121     for ( i = NumExpressions-1; i >= 0; i-- ) {
00122         e = Expressions+i;
00123         if ( e->status == LOCALEXPRESSION || e->status == GLOBALEXPRESSION
00124             || e->status == HIDELEXPRESSION || e->status == HIDEGEXPRESSION
00125             || e->status == SKIPLEXPRESSION || e->status == SKIPGEXPRESSION
00126             || e->status == UNHIDELEXPRESSION || e->status == UNHIDEGEXPRESSION
00127             || e->status == INTOHIDELEXPRESSION || e->status == INTOHIDEGEXPRESSION
00128         ) break;
00129     }
00130     last = i;
00131     for ( i = NumExpressions-1; i >= 0; i-- ) {
00132         AS.OldOnFile[i] = Expressions[i].onfile;
00133         AS.OldNumFactors[i] = Expressions[i].numfactors;
00134         AS.Oldvflags[i] = e[i].vflags; */
00135         AS.Oldvflags[i] = Expressions[i].vflags;
00136         AS.Olduflags[i] = Expressions[i].uflags;
00137         Expressions[i].vflags &= ~(ISUNMODIFIED|ISZERO);
00138     }

```

```

00139 #ifdef WITHPTHREADS
00140 /*
00141      When we run with threads we have to make sure that all local input
00142      buffers are pointed correctly. Of course this isn't needed if we
00143      run on a single thread only.
00144 */
00145     if ( AC.partodoflag && AM.totalnumberofthreads > 1 ) {
00146         AS.MultiThreaded = 1; AC.mparallelfld = PARALLELFLAG;
00147     }
00148     if ( AS.MultiThreaded && AC.mparallelfld == PARALLELFLAG ) {
00149         SetWorkerFiles();
00150     }
00151 /*
00152      We start with running the expressions with expr->partodo in parallel.
00153      The current model is: give each worker an expression. Wait for
00154      workers to finish and tell them where to write.
00155      Then give them a new expression. Workers may have to wait for each
00156      other. This is also the case with the last one.
00157 */
00158     if ( AS.MultiThreaded && AC.mparallelfld == PARALLELFLAG ) {
00159         if ( InParallelProcessor() ) {
00160             retval = 1;
00161         }
00162         AS.MultiThreaded = OldMultiThreaded;
00163         AC.mparallelfld = Oldmparallelfld;
00164     }
00165 #endif
00166 #ifdef WITHMPI
00167     if ( AC.RhsExprInModuleFlag && PF.rhsInParallel && (AC.mparallelfld == PARALLELFLAG ||
AC.partodofld) ) {
00168         if ( PF_BroadcastRHS() ) {
00169             retval = -1;
00170         }
00171     }
00172     PF.exprtodo = -1; /* This means, the slave does not perform inparallel */
00173     if ( AC.partodofld > 0 ) {
00174         if ( PF_InParallelProcessor() ) {
00175             retval = -1;
00176         }
00177     }
00178 #endif
00179     for ( i = 0; i < NumExpressions; i++ ) {
00180 #ifdef INNERTEST
00181         if ( AC.InnerTest ) {
00182             if ( StrCmp(AC.TestValue, (UBYTE *)INNERTEST) == 0 ) {
00183                 MesPrint("Testing(Processor): value = %s",AC.TestValue);
00184             }
00185         }
00186 #endif
00187         e = Expressions+i;
00188 #ifdef WITHPTHREADS
00189         if ( AC.partodofld > 0 && e->partodo > 0 && AM.totalnumberofthreads > 2 ) {
00190             e->partodo = 0;
00191             continue;
00192         }
00193 #endif
00194 #ifdef WITHMPI
00195         if ( AC.partodofld > 0 && e->partodo > 0 && PF.numtasks > 2 ) {
00196             e->partodo = 0;
00197             continue;
00198         }
00199 #endif
00200         AS.CollectOverFlag = 0;
00201         AR.expchanged = 0;
00202         if ( i == last ) LastExpression = 1;
00203         else LastExpression = 0;
00204         if ( e->inmem ) {
00205 /*
00206             #[ in memory : Memory allocated by poly.c only thusfar.
00207             Here GetTerm cannot work.
00208             For the moment we ignore this for parallelization.
00209 */
00210             WORD j;
00211
00212             AR.GetFile = 0;
00213             SetScratch(AR.infile,&(e->onfile));
00214             if ( GetTerm(BHEAD term) <= 0 ) {
00215                 MesPrint("(1) Expression %d has problems in scratchfile",i);
00216                 retval = -1;
00217                 break;
00218             }
00219             term[3] = i;
00220             AR.CurExpr = i;
00221             SeekScratch(AR.outfile,&position);
00222             e->onfile = position;
00223             if ( PutOut (BHEAD term,&position,AR.outfile,0) < 0 ) goto ProcErr;
00224             AR.DeferFlag = AC.ComDefer;

```

```

00225     NewSort(BHEAD0);
00226     AN.ninterms = 0;
00227     t = e->inmem;
00228     while ( *t ) {
00229         for ( j = 0; j < *t; j++ ) term[j] = t[j];
00230         t += *t;
00231         AN.ninterms++; dd = AN.deferskipped;
00232         if ( AC.CollectFun && *term <= (AM.MaxTer/(2*(LONG)(sizeof(WORD)))) ) {
00233             if ( GetMoreFromMem(term,&t) ) {
00234                 LowerSortLevel(); goto ProcErr;
00235             }
00236         }
00237         AT.WorkPointer = term + *term;
00238         AN.RepPoint = AT.RepCount + 1;
00239         AN.IndDum = AM.IndDum;
00240         AR.CurDum = ReNumber(BHEAD term);
00241         if ( AC.SymChangeFlag ) MarkDirty(term,DIRTYSYMFLAG);
00242         if ( AN.ncmod ) {
00243             if ( ( AC.modmode & ALSOFUNARGS ) != 0 ) MarkDirty(term,DIRTYFLAG);
00244             else if ( AR.PolyFun ) PolyFunDirty(BHEAD term);
00245         }
00246         else if ( AC.PolyRatFunChanged ) PolyFunDirty(BHEAD term);
00247         if ( Generator(BHEAD term,0) ) {
00248             LowerSortLevel(); goto ProcErr;
00249         }
00250         AN.ninterms += dd;
00251     }
00252     AN.ninterms += dd;
00253     if ( EndSort(BHEAD AM.S0->sBuffer,0) < 0 ) goto ProcErr;
00254     if ( AM.S0->TermsLeft ) e->vflags &= ~ISZERO;
00255     else e->vflags |= ISZERO;
00256     if ( AR.expchanged == 0 ) e->vflags |= ISUNMODIFIED;
00257     if ( AM.S0->TermsLeft ) AR.expflags |= ISZERO;
00258     if ( AR.expchanged ) AR.expflags |= ISUNMODIFIED;
00259     AR.GetFile = 0;
00260     /*
00261     #] in memory :
00262     */
00263 }
00264 else {
00265     AR.CurExpr = i;
00266     switch ( e->status ) {
00267     case UNHIDELEXPRESSION:
00268     case UNHIDEGEXPRESSION:
00269         AR.GetFile = 2;
00270 #ifdef WITHMPI
00271         if ( PF.me == MASTER ) SetScratch(AR.hidefile,&(e->onfile));
00272 #else
00273         SetScratch(AR.hidefile,&(e->onfile));
00274         AR.InHiBuf = AR.hidefile->POfull-AR.hidefile->POfill;
00275 #ifdef HIDEDEBUG
00276         MesPrint("Hidefile: onfile: %15p, PPosition: %15p, filesize: %15p",&(e->onfile)
00277             ,&(AR.hidefile->PPosition),&(AR.hidefile->filesize));
00278         MesPrint("Set hidefile to buffer position %1/%1; AR.InHiBuf = %1"
00279             ,(AR.hidefile->POfill-AR.hidefile->PObuffer)*sizeof(WORD)
00280             ,(AR.hidefile->POfull-AR.hidefile->PObuffer)*sizeof(WORD)
00281             ,AR.InHiBuf
00282             );
00283 #endif
00284 #endif
00285         curfile = AR.hidefile;
00286         goto commonread;
00287     case INTOHIDELEXPRESSION:
00288     case INTOHIDEGEXPRESSION:
00289         AR.outtohide = 1;
00290     /*
00291         BugFix 12-feb-2016
00292         This may not work when the file is open and we move around.
00293         AR.hidefile->POfill = AR.hidefile->POfull;
00294     */
00295         SetEndHScratch(AR.hidefile,&position);
00296         /* fall through */
00297     case LOCALEXPRESSION:
00298     case GLOBALEXPRESSION:
00299         AR.GetFile = 0;
00300     /*[20oct2009 mt]:*/
00301 #ifdef WITHMPI
00302         if( ( PF.me == MASTER ) || (PF.mkSlaveInfile) )
00303 #endif
00304         SetScratch(AR.infile,&(e->onfile));
00305     /*:[20oct2009 mt]:*/
00306         curfile = AR.infile;
00307 commonread:;
00308 #ifdef WITHMPI
00309         if ( PF_Processor(e,i,LastExpression) ) {
00310             MesPrint("Error in PF_Processor");
00311             goto ProcErr;

```

```

00312     }
00313     /*[20oct2009 mt]:*/
00314     if ( AC.mparallelflag != PARALLELFLAG ){
00315         if(PF.me != MASTER)
00316             break;
00317     #endif
00318     /*:[20oct2009 mt]:*/
00319     if ( GetTerm(BHEAD term) <= 0 ) {
00320     #ifdef HIDEDEBUG
00321         MesPrint("Error condition 1a");
00322         ExprStatus(e);
00323     #endif
00324         MesPrint("(2) Expression %d has problems in scratchfile(process)",i);
00325         retval = -1;
00326         break;
00327     }
00328     term[3] = i;
00329     if ( term[5] < 0 ) {      /* Fill with spectator */
00330         fromspectator = -term[5];
00331         PUTZERO(AM.SpectatorFiles[fromspectator-1].readpos);
00332         term[5] = AC.cbufnum;
00333     }
00334     else fromspectator = 0;
00335     if ( AR.outtohide ) {
00336         SeekScratch(AR.hidefile,&position);
00337         e->onfile = position;
00338         if ( PutOut(BHEAD term,&position,AR.hidefile,0) < 0 ) goto ProcErr;
00339     }
00340     else {
00341         SeekScratch(AR.outfile,&position);
00342         e->onfile = position;
00343         if ( PutOut(BHEAD term,&position,AR.outfile,0) < 0 ) goto ProcErr;
00344     }
00345     AR.DeferFlag = AC.ComDefer;
00346     AR.Eside = RHSIDE;
00347     if ( ( e->vflags & ISFACTORIZED ) != 0 ) {
00348         AR.BraceOn = 1;
00349         AT.BrackBuf = AM.BraceFactors;
00350         AT.bracketindexflag = 1;
00351     }
00352     if ( AT.bracketindexflag > 0 ) OpenBracketIndex(i);
00353     #ifdef WITHPTHREADS
00354     if ( AS.MultiThreaded && AC.mparallelflag == PARALLELFLAG ) {
00355         if ( ThreadsProcessor(e,LastExpression,fromspectator) ) {
00356             MesPrint("Error in ThreadsProcessor");
00357             goto ProcErr;
00358         }
00359         if ( AR.outtohide ) {
00360             AR.outfile = oldoutfile;
00361             AR.hidefile->POfull = AR.hidefile->POfill;
00362         }
00363     }
00364     else
00365     #endif
00366     {
00367         NewSort(BHEAD0);
00368         AR.MaxDum = AM.IndDum;
00369         AN.ninterms = 0;
00370         for(;;) {
00371             if ( fromspectator ) size = GetFromSpectator(term,fromspectator-1);
00372             else size = GetTerm(BHEAD term);
00373             if ( size <= 0 ) break;
00374             SeekScratch(curfile,&position);
00375             if ( ( e->vflags & ISFACTORIZED ) != 0 && term[1] == HAAKJE ) {
00376                 StoreTerm(BHEAD term);
00377             }
00378             else {
00379                 AN.ninterms++; dd = AN.deferskipped;
00380                 if ( AC.CollectFun && *term <= (AM.MaxTer/(2*(LONG)(sizeof(WORD)))) ) {
00381                     if ( GetMoreTerms(term) < 0 ) {
00382                         LowerSortLevel(); goto ProcErr;
00383                     }
00384                     SeekScratch(curfile,&position);
00385                 }
00386                 AT.WorkPointer = term + *term;
00387                 AN.RepPoint = AT.RepCount + 1;
00388                 if ( AR.DeferFlag ) {
00389                     AN.IndDum = Expressions[AR.CurExpr].numdummies + AM.IndDum;
00390                     AR.CurDum = AN.IndDum;
00391                 }
00392                 else {
00393                     AN.IndDum = AM.IndDum;
00394                     AR.CurDum = ReNumber(BHEAD term);
00395                 }
00396                 if ( AC.SymChangeFlag ) MarkDirty(term,DIRTSYMFLAG);
00397                 if ( AN.ncmod ) {
00398                     if ( ( AC.modmode & ALSOFUNARGS ) != 0 ) MarkDirty(term,DIRTYFLAG);

```

```

00399         else if ( AR.PolyFun ) PolyFunDirty(BHEAD term);
00400     }
00401     else if ( AC.PolyRatFunChanged ) PolyFunDirty(BHEAD term);
00402     if ( ( AR.PolyFunType == 2 ) && ( AC.PolyRatFunChanged == 0 )
00403         && ( e->status == LOCALEXPRESSION || e->status == GLOBALEXPRESSION ) ) {
00404         PolyFunClean(BHEAD term);
00405     }
00406     if ( Generator(BHEAD term,0) ) {
00407         LowerSortLevel(); goto ProcErr;
00408     }
00409     AN.ninterms += dd;
00410 }
00411 SetScratch(curfile,&position);
00412 if ( AR.GetFile == 2 ) {
00413     AR.InHiBuf = (curfile->POfull-curfile->PObuffer)
00414         -DIFBASE(position,curfile->POposition)/sizeof(WORD);
00415 }
00416 else {
00417     AR.InInBuf = (curfile->POfull-curfile->PObuffer)
00418         -DIFBASE(position,curfile->POposition)/sizeof(WORD);
00419 }
00420 }
00421 AN.ninterms += dd;
00422 if ( LastExpression ) {
00423     UpdateMaxSize();
00424     if ( AR.infile->handle >= 0 ) {
00425         CloseFile(AR.infile->handle);
00426         AR.infile->handle = -1;
00427         remove(AR.infile->name);
00428         PUTZERO(AR.infile->POposition);
00429     }
00430     AR.infile->POfill = AR.infile->POfull = AR.infile->PObuffer;
00431 }
00432 if ( AR.outtohide ) AR.outfile = AR.hidefile;
00433 if ( EndSort(BHEAD AM.S0->sBuffer,0) < 0 ) goto ProcErr;
00434 if ( AR.outtohide ) {
00435     AR.outfile = oldoutfile;
00436     AR.hidefile->POfull = AR.hidefile->POfill;
00437 }
00438 e->numdummies = AR.MaxDum - AM.IndDum;
00439 UpdateMaxSize();
00440 }
00441 AR.BracketOn = oldBracketOn;
00442 AT.BrackBuf = oldBrackBuf;
00443 if ( ( e->vflags & TOBEFACTORED ) != 0 ) {
00444     poly_factorize_expression(e);
00445 }
00446 else if ( ( ( e->vflags & TOBEUNFACTORED ) != 0 )
00447     && ( ( e->vflags & ISFACTORIZED ) != 0 ) ) {
00448     poly_unfactorize_expression(e);
00449 }
00450 AT.bracketindexflag = oldbracketindexflag;
00451 if ( AM.S0->TermsLeft ) e->vflags &= ~ISZERO;
00452 else e->vflags |= ISZERO;
00453 if ( AR.expchanged == 0 ) e->vflags |= ISUNMODIFIED;
00454 if ( AM.S0->TermsLeft ) AR.expflags |= ISZERO;
00455 if ( AR.expchanged ) AR.expflags |= ISUNMODIFIED;
00456 AR.GetFile = 0;
00457 AR.outtohide = 0;
00458 /*[20oct2009 mt]:*/
00459 #ifdef WITHMPI
00460     }
00461 #endif
00462 #ifdef WITHPTHREADS
00463     if ( e->status == INTOHIDELEXPRESSION ||
00464         e->status == INTOHIDEGEXPRESSION ) {
00465         SetHideFiles();
00466     }
00467 #endif
00468     break;
00469 case SKIPLEXPRESSION:
00470 case SKIPGEXPRESSION:
00471 /*
00472     This can be greatly improved of course by file-to-file copy.
00473 */
00474 #ifdef WITHMPI
00475     if ( PF.me != MASTER ) break;
00476 #endif
00477     AR.GetFile = 0;
00478     SetScratch(AR.infile,&(e->onfile));
00479     if ( GetTerm(BHEAD term) <= 0 ) {
00480 #ifdef HIDEDEBUG
00481         MesPrint("Error condition 1b");
00482         ExprStatus(e);
00483 #endif
00484         MesPrint("(3) Expression %d has problems in scratchfile",i);
00485         retval = -1;

```

```

00486         break;
00487     }
00488     term[3] = i;
00489     AR.DeferFlag = 0;
00490     SeekScratch(AR.outfile,&position);
00491     e->onfile = position;
00492     *AM.S0->sBuffer = 0; firstterm = -1;
00493     do {
00494         WORD *oldipointer = AR.CompressPointer;
00495         WORD *comprtop = AR.ComprTop;
00496         AR.ComprTop = AM.S0->sTop;
00497         AR.CompressPointer = AM.S0->sBuffer;
00498         if ( firstterm > 0 ) {
00499             if ( PutOut(BHEAD term,&position,AR.outfile,1) < 0 ) goto ProcErr;
00500         }
00501         else if ( firstterm < 0 ) {
00502             if ( PutOut(BHEAD term,&position,AR.outfile,0) < 0 ) goto ProcErr;
00503             firstterm++;
00504         }
00505         else {
00506             if ( PutOut(BHEAD term,&position,AR.outfile,-1) < 0 ) goto ProcErr;
00507             firstterm++;
00508         }
00509         AR.CompressPointer = oldipointer;
00510         AR.ComprTop = comprtop;
00511     } while ( GetTerm(BHEAD term) );
00512     if ( FlushOut(&position,AR.outfile,1) ) goto ProcErr;
00513     UpdateMaxSize();
00514     break;
00515     case HIDELEXPRESSION:
00516     case HIDEGEXPRESSION:
00517     #ifdef WITHMPI
00518         if ( PF.me != MASTER ) break;
00519     #endif
00520     AR.GetFile = 0;
00521     SetScratch(AR.infile,&(e->onfile));
00522     if ( GetTerm(BHEAD term) <= 0 ) {
00523     #ifdef HIDEDEBUG
00524         MesPrint("Error condition 1c");
00525         ExprStatus(e);
00526     #endif
00527         MesPrint("(4) Expression %d has problems in scratchfile",i);
00528         retval = -1;
00529         break;
00530     }
00531     term[3] = i;
00532     AR.DeferFlag = 0;
00533     SetEndHScratch(AR.hidefile,&position);
00534     e->onfile = position;
00535     #ifdef HIDEDEBUG
00536     if ( AR.hidefile->handle >= 0 ) {
00537         POSITION possize,pos;
00538         PUTZERO(possize);
00539         PUTZERO(pos);
00540         SeekFile(AR.hidefile->handle,&pos,SEEK_CUR);
00541         SeekFile(AR.hidefile->handle,&possize,SEEK_END);
00542         SeekFile(AR.hidefile->handle,&pos,SEEK_SET);
00543         MesPrint("Processor Hidel: filesize(th) = %12p, filesize(ex) = %12p",&(position),
00544                 &(possize));
00545         MesPrint("      in buffer:
00546         %1", (AR.hidefile->Pofill-AR.hidefile->PObuffer)*sizeof(WORD));
00547     }
00548     #endif
00549     *AM.S0->sBuffer = 0; firstterm = -1;
00550     cbo = cpo = AM.S0->sBuffer;
00551     do {
00552         WORD *oldipointer = AR.CompressPointer;
00553         WORD *oldibuffer = AR.CompressBuffer;
00554         WORD *comprtop = AR.ComprTop;
00555         AR.ComprTop = AM.S0->sTop;
00556         AR.CompressPointer = cpo;
00557         AR.CompressBuffer = cbo;
00558         if ( firstterm > 0 ) {
00559             if ( PutOut(BHEAD term,&position,AR.hidefile,1) < 0 ) goto ProcErr;
00560         }
00561         else if ( firstterm < 0 ) {
00562             if ( PutOut(BHEAD term,&position,AR.hidefile,0) < 0 ) goto ProcErr;
00563             firstterm++;
00564         }
00565         else {
00566             if ( PutOut(BHEAD term,&position,AR.hidefile,-1) < 0 ) goto ProcErr;
00567             firstterm++;
00568         }
00569         cpo = AR.CompressPointer;
00570         cbo = AR.CompressBuffer;
00571         AR.CompressPointer = oldipointer;
00572         AR.CompressBuffer = oldibuffer;

```

```

00572         AR.ComprTop = comprtop;
00573     } while ( GetTerm(BHEAD term) );
00574 #ifdef HIIDEDEBUG
00575     if ( AR.hidefile->handle >= 0 ) {
00576         POSITION posize,pos;
00577         PUTZERO(posize);
00578         PUTZERO(pos);
00579         SeekFile(AR.hidefile->handle,&pos,SEEK_CUR);
00580         SeekFile(AR.hidefile->handle,&posize,SEEK_END);
00581         SeekFile(AR.hidefile->handle,&pos,SEEK_SET);
00582         MesPrint("Processor Hide2: filesize(th) = %12p, filesize(ex) = %12p",&(position),
00583                 &(posize));
00584         MesPrint("    in buffer:
%1", (AR.hidefile->POfill-AR.hidefile->PObuffer)*sizeof(WORD));
00585     }
00586 #endif
00587     if ( FlushOut(&position,AR.hidefile,1) ) goto ProcErr;
00588     AR.hidefile->POfull = AR.hidefile->POfill;
00589 #ifdef HIIDEDEBUG
00590     if ( AR.hidefile->handle >= 0 ) {
00591         POSITION posize,pos;
00592         PUTZERO(posize);
00593         PUTZERO(pos);
00594         SeekFile(AR.hidefile->handle,&pos,SEEK_CUR);
00595         SeekFile(AR.hidefile->handle,&posize,SEEK_END);
00596         SeekFile(AR.hidefile->handle,&pos,SEEK_SET);
00597         MesPrint("Processor Hide3: filesize(th) = %12p, filesize(ex) = %12p",&(position),
00598                 &(posize));
00599         MesPrint("    in buffer:
%1", (AR.hidefile->POfill-AR.hidefile->PObuffer)*sizeof(WORD));
00600     }
00601 #endif
00602 /*
00603     Because we direct the e->onfile already to the hide file, we
00604     need to change the status of the expression. Otherwise the use
00605     of parts (or the whole) of the expression looks in the infile
00606     while the position is that of the hide file.
00607     We choose to get everything from the hide file. On average that
00608     should give least file activity.
00609 */
00610     if ( e->status == HIDELEXPRESSION ) {
00611         e->status = HIDDENLEXPRESSION;
00612         AS.OldOnFile[i] = e->onfile;
00613         AS.OldNumFactors[i] = Expressions[i].numfactors;
00614     }
00615     if ( e->status == HIDEGEXPRESSION ) {
00616         e->status = HIDDENGEXPRESSION;
00617         AS.OldOnFile[i] = e->onfile;
00618         AS.OldNumFactors[i] = Expressions[i].numfactors;
00619     }
00620 #ifdef WITHPTHREADS
00621     SetHideFiles();
00622 #endif
00623     UpdateMaxSize();
00624     break;
00625 case DROPEDEXPRESSION:
00626 case DROPLEXPRESSION:
00627 case DROPGEXPRESSION:
00628 case DROPHLEXPRESSION:
00629 case DROPHGEXPRESSION:
00630 case STOREDEXPRESSION:
00631 case HIDDENLEXPRESSION:
00632 case HIDDENGEXPRESSION:
00633 case SPECTATOREXPRESSION:
00634     default:
00635         break;
00636     }
00637 }
00638 AR.KeptInHold = 0;
00639 }
00640 AR.DeferFlag = 0;
00641 AT.WorkPointer = term;
00642 #ifdef HIIDEDEBUG
00643 MesPrint("Status at the end of Processor (HideLevel = %d)",AC.HideLevel);
00644 for ( i = 0; i < NumExpressions; i++ ) {
00645     e = Expressions+i;
00646     ExprStatus(e);
00647 }
00648 #endif
00649 return(retval);
00650 ProcErr:
00651 AT.WorkPointer = term;
00652 if ( AM.tracebackflag ) MesCall("Processor");
00653 return(-1);
00654 }
00655 /*
00656 #] Processor :

```



```

00657      # [ TestSub :          WORD TestSub (term, level)
00658      /*
00682 #define DONE(x) { retvalue = x; goto Done; }
00683
00684 WORD TestSub (PHEAD WORD *term, WORD level)
00685 {
00686     GETBIDENTITY
00687     WORD *m, *t, *r, retvalue = 0, funflag, j, oldncmod, nexpr, *Tpattern = 0;
00688     WORD *stop, *t1, *t2, funnum, wilds, tbufnum, stilldirty = 0;
00689     NESTING n;
00690     CBUF *C = cbuf+AT.ebufnum;
00691     LONG isp, i;
00692     TABLES T;
00693     COMPARE oldcompareroutine = (COMPARE) (AR.CompareRoutine);
00694     WORD oldsorttype = AR.SortType;
00695     ReStart:
00696     tbufnum = 0; i = 0; retvalue = 0;
00697     AT.TMbuff = AM.rbufnum;
00698     funflag = 0;
00699     t = term;
00700     r = t + *t - 1;
00701     m = r - ABS(*r) + 1;
00702     t++;
00703     if ( t < m ) do {
00704         if ( *t == SUBEXPRESSION ) {
00705             /*
00706                 Subexpression encountered
00707                 There may be more than one.
00708                 The old strategy was to take the last.
00709                 A newer strategy was to take the lowest power first.
00710                 The current strategy is that we compute the number of terms
00711                 generated by this subexpression and take the minimum of that.
00712             */
00713
00714 #ifdef WHICHSUBEXPRESSION
00715
00716             WORD *tmin = t, AN.nbino;
00717             /* LONG minval = MAXLONG; */
00718             LONG minval = -1;
00719             LONG mm, mnuml = 1;
00720             if ( AN.BinoScrat == 0 ) {
00721                 AN.BinoScrat = (UWORD *) Malloc1 ((AM.MaxTal+2)*sizeof(UWORD), "GetBinoScrat");
00722             }
00723 #endif
00724             if ( t[3] ) {
00725                 r = t + t[1];
00726                 while ( AN.subsubveto == 0 &&
00727                     *r == SUBEXPRESSION && r < m && r[3] ) {
00728 #ifdef WHICHSUBEXPRESSION
00729                     mnuml++;
00730 #endif
00731                 if ( r[1] == t[1] && r[2] == t[2] && r[4] == t[4] ) {
00732                     j = t[1] - SUBEXPSIZE;
00733                     t1 = t + SUBEXPSIZE;
00734                     t2 = r + SUBEXPSIZE;
00735                     while ( j > 0 && *t1++ == *t2++ ) j--;
00736                     if ( j <= 0 ) {
00737                         t[3] += r[3];
00738                         if ( t[3] == 0 ) {
00739                             t1 = r + r[1];
00740                             t2 = term + *term;
00741                             *term -= r[1]+t[1];
00742                             r = t;
00743                             while ( t1 < t2 ) *r++ = *t1++;
00744                             goto ReStart;
00745                         }
00746                         else {
00747                             t1 = r + r[1];
00748                             t2 = term + *term;
00749                             *term -= r[1];
00750                             m -= r[1];
00751                             while ( t1 < t2 ) *r++ = *t1++;
00752                             r = t;
00753                         }
00754                     }
00755                 }
00756 #ifdef WHICHSUBEXPRESSION
00757                 else if ( t[2] >= 0 ) {
00758                     /*
00760                         Compute Binom(numterms+power-1, power-1)
00761                         We need potentially long arithmetic.
00762                         That is why we had to allocate AN.BinoScrat
00763                     */
00764                     if ( AN.last1 == t[3] && AN.last2 == cbuf[t[4]].NumTerms[t[2]] + t[3] - 1 ) {
00765                         if ( AN.last3 > minval ) {
00766                             minval = AN.last3; tmin = t;

```

```

00767     }
00768     }
00769     else {
00770     AN.last1 = t[3]; mm = AN.last2 = cbuf[t[4]].NumTerms[t[2]] + t[3] - 1;
00771     if ( t[3] == 1 ) {
00772         if ( mm > minval ) {
00773             minval = mm; tmin = t;
00774         }
00775     }
00776     else if ( t[3] > 0 ) {
00777         if ( mm > MAXPOSITIVE ) goto TooMuch;
00778         GetBinom(AN.BinoScrat,&AN.nbino,(WORD)mm,t[3]);
00779         if ( AN.nbino > 2 ) goto TooMuch;
00780         if ( AN.nbino == 2 ) {
00781             mm = AN.BinoScrat[1];
00782             mm = ( mm « BITSINWORD ) + AN.BinoScrat[0];
00783         }
00784         else if ( AN.nbino == 1 ) mm = AN.BinoScrat[0];
00785         else mm = 0;
00786         if ( mm > minval ) {
00787             minval = mm; tmin = t;
00788         }
00789     }
00790     AN.last3 = mm;
00791     }
00792     }
00793 #endif
00794     t = r;
00795     r += r[1];
00796 }
00797 #ifdef WHICHSUBEXPRESSION
00798     if ( mnum1 > 1 && t[2] >= 0 ) {
00799     /*
00800         To keep the flowcontrol simple we duplicate some code here
00801     */
00802     if ( AN.last1 == t[3] && AN.last2 == cbuf[t[4]].NumTerms[t[2]] + t[3] - 1 ) {
00803         if ( AN.last3 > minval ) {
00804             minval = AN.last3; tmin = t;
00805         }
00806     }
00807     else {
00808     AN.last1 = t[3]; mm = AN.last2 = cbuf[t[4]].NumTerms[t[2]] + t[3] - 1;
00809     if ( t[3] == 1 ) {
00810         if ( mm > minval ) {
00811             minval = mm; tmin = t;
00812         }
00813     }
00814     else if ( t[3] > 0 ) {
00815         if ( mm > MAXPOSITIVE ) {
00816         /*
00817             We will generate more terms than we can count
00818         */
00819         TooMuch::;
00820             MLOCK(ErrorMessageLock);
00821             MesPrint("Attempt to generate more terms than FORM can count");
00822             MUNLOCK(ErrorMessageLock);
00823             Terminate(-1);
00824         }
00825         GetBinom(AN.BinoScrat,&AN.nbino,(WORD)mm,t[3]);
00826         if ( AN.nbino > 2 ) goto TooMuch;
00827         if ( AN.nbino == 2 ) {
00828             mm = AN.BinoScrat[1];
00829             mm = ( mm « BITSINWORD ) + AN.BinoScrat[0];
00830         }
00831         else if ( AN.nbino == 1 ) mm = AN.BinoScrat[0];
00832         else mm = 0;
00833         if ( mm > minval ) {
00834             minval = mm; tmin = t;
00835         }
00836     }
00837     AN.last3 = mm;
00838     }
00839     }
00840     t = tmin;
00841 #endif
00842 /*
00843     AR.TePos = WORDDIF(t,term);
00844     AT.TMbuff = t[4];
00845     if ( t[4] == AM.dbufnum && (t+t[1]) < m && t[t[1]] == DOLLAREXP2 ) {
00846         if ( t[t[1]+2] < 0 ) AT.TMdolfac = -t[t[1]+2];
00847         else { /* resolve the element number */
00848             AT.TMdolfac = GetDolNum(BHEAD t+t[1],m)+1;
00849         }
00850     }
00851     else AT.TMdolfac = 0;
00852     if ( t[3] < 0 ) {
00853         AN.TeInFun = 1;

```

```

00854         AR.TePos = WORDDIF(t,term);
00855         DONE(t[2])
00856     }
00857     else {
00858         AN.TeInFun = 0;
00859         AN.TeSuOut = t[3];
00860     }
00861     if ( t[2] < 0 ) {
00862         AN.TeSuOut = -t[3];
00863         DONE(-t[2])
00864     }
00865     DONE(t[2])
00866 }
00867 }
00868 else if ( *t == EXPRESSION ) {
00869     WORD *toTMaddr;
00870     i = -t[2] - 1;
00871     if ( t[3] < 0 ) {
00872         AN.TeInFun = 1;
00873         AR.TePos = WORDDIF(t,term);
00874         DONE(i)
00875     }
00876     nexpr = t[3];
00877     toTMaddr = m = AT.WorkPointer;
00878     AN.Frozen = 0;
00879 /*
00880     We have to be very careful with respect to setting variables
00881     like AN.TeInFun, because we may still call Generator and that
00882     may change those variables. That is why we set them at the
00883     last moment only.
00884 */
00885     j = t[1];
00886     AT.WorkPointer += j;
00887     r = t;
00888     NCOPY(m,r,j);
00889     r = t + t[1];
00890     t += SUBEXPSIZE;
00891     while ( t < r ) {
00892         if ( *t == FROMBRAC ) {
00893             WORD *ttstop,*tttstop;
00894 /*
00895             Note: Convention is that wildcards are done
00896             after the expression has been picked up. So
00897             no wildcard substitutions are needed here.
00898 */
00899             t += 2;
00900             AN.Frozen = m = AT.WorkPointer;
00901 /*
00902             We should check now for subexpressions and if necessary
00903             we substitute them. Keep in mind: only one term allowed!
00904
00905             In retrospect (26-jan-2010): take also functions that
00906             have a dirty flag on
00907 */
00908             j = *t; tttstop = t + j;
00909             GETSTOP(t,tttstop);
00910             *m++ = j; t++;
00911             while ( t < tttstop ) {
00912                 if ( *t == SUBEXPRESSION ) break;
00913                 if ( *t >= FUNCTION && ( ( t[2] & DIRTYFLAG ) == DIRTYFLAG ) ) break;
00914                 j = t[1]; NCOPY(m,t,j);
00915             }
00916             if ( t < tttstop ) {
00917 /*
00918                 We ran into a subexpression or a function with a
00919                 'dirty' argument. It could also be a $ or
00920                 just e[(a^2)*b]. In all cases we should evaluate
00921 */
00922                 while ( t < tttstop ) *m++ = *t++;
00923                 *AT.WorkPointer = m-AT.WorkPointer;
00924                 m = AT.WorkPointer;
00925                 AT.WorkPointer = m + *m;
00926                 NewSort(BHEAD0);
00927                 if ( Generator(BHEAD m,AR.Cnumlhs) ) {
00928                     LowerSortLevel(); goto EndTest;
00929                 }
00930                 if ( EndSort(BHEAD m,0) < 0 ) goto EndTest;
00931                 AN.Frozen = m;
00932                 if ( *m == 0 ) {
00933                     *m++ = 4; *m++ = 1; *m++ = 1; *m++ = 3;
00934                 }
00935                 else if ( m[*m] != 0 ) {
00936                     MLOCK(ErrorMessageLock);
00937                     MesPrint("Bracket specification in expression should be one single term");
00938                     MUNLOCK(ErrorMessageLock);
00939                     Terminate(-1);
00940                 }

```

```

00941         else {
00942             m += *m;
00943             m -= ABS(m[-1]);
00944             *m++ = 1; *m++ = 1; *m++ = 3;
00945             *AN.Frozen = m - AN.Frozen;
00946         }
00947     }
00948     else {
00949         while ( t < tttstop ) *m++ = *t++;
00950         *AT.WorkPointer = m-AT.WorkPointer;
00951         m = AT.WorkPointer;
00952         AT.WorkPointer = m + *m;
00953         if ( Normalize(BHEAD m) ) {
00954             MLOCK(ErrorMessageLock);
00955             MesPrint("Error while picking up contents of bracket");
00956             MUNLOCK(ErrorMessageLock);
00957             Terminate(-1);
00958         }
00959         if ( !*m ) {
00960             *m++ = 4; *m++ = 1; *m++ = 1; *m++ = 3;
00961         }
00962         else m += *m;
00963     }
00964     AT.WorkPointer = m;
00965     break;
00966 }
00967 t += t[1];
00968 }
00969 AN.TeInFun = 0;
00970 AR.TePos = 0;
00971 AN.TeSuOut = nexpr;
00972 AT.TMaddr = toTMaddr;
00973 DONE(i)
00974 }
00975 else if ( *t >= FUNCTION ) {
00976     if ( t[0] == EXPONENT ) {
00977         if ( t[1] == FUNHEAD+4 && t[FUNHEAD] == -SYMBOL &&
00978             t[FUNHEAD+2] == -SNUMBER && t[FUNHEAD+3] < MAXPOWER
00979             && t[FUNHEAD+3] > -MAXPOWER ) {
00980             t[0] = SYMBOL;
00981             t[1] = 4;
00982             t[2] = t[FUNHEAD+1];
00983             t[3] = t[FUNHEAD+3];
00984             r = term + *term;
00985             m = t + FUNHEAD+4;
00986             t += 4;
00987             while ( m < r ) *t++ = *m++;
00988             *term = WORDDIF(t,term);
00989             goto ReStart;
00990         }
00991         else if ( t[1] == FUNHEAD+ARGHEAD+11 && t[FUNHEAD] == ARGHEAD+9
00992             && t[FUNHEAD+ARGHEAD] == 9 && t[FUNHEAD+ARGHEAD+1] == DOTPRODUCT
00993             && t[FUNHEAD+ARGHEAD+8] == 3
00994             && t[FUNHEAD+ARGHEAD+7] == 1
00995             && t[FUNHEAD+ARGHEAD+6] == 1
00996             && t[FUNHEAD+ARGHEAD+5] == 1
00997             && t[FUNHEAD+ARGHEAD+9] == -SNUMBER
00998             && t[FUNHEAD+ARGHEAD+10] < MAXPOWER
00999             && t[FUNHEAD+ARGHEAD+10] > -MAXPOWER ) {
01000             t[0] = DOTPRODUCT;
01001             t[1] = 5;
01002             t[2] = t[FUNHEAD+ARGHEAD+3];
01003             t[3] = t[FUNHEAD+ARGHEAD+4];
01004             t[4] = t[FUNHEAD+ARGHEAD+10];
01005             r = term + *term;
01006             m = t + FUNHEAD+ARGHEAD+11;
01007             t += 5;
01008             while ( m < r ) *t++ = *m++;
01009             *term = WORDDIF(t,term);
01010             goto ReStart;
01011         }
01012     }
01013     funnum = *t;
01014     if ( *t >= FUNCTION + WILDOFFSET ) funnum -= WILDOFFSET;
01015     if ( *t == EXPONENT ) {
01016         /*
01017             Test whether the second argument is an integer
01018         */
01019         r = t+FUNHEAD;
01020         NEXTARG(r)
01021         if ( *r == -SNUMBER && r[1] < MAXPOWER && r+2 == t+t[1] &&
01022             t[FUNHEAD] > -FUNCTION && ( t[FUNHEAD] != -SNUMBER
01023             || t[FUNHEAD+1] != 0 ) && t[FUNHEAD] != ARGHEAD ) {
01024             if ( r[1] == 0 ) {
01025                 if ( t[FUNHEAD] == -SNUMBER && t[FUNHEAD+1] == 0 ) {
01026                     MLOCK(ErrorMessageLock);
01027                     MesPrint("Encountered 0^0. Fatal error.");

```

```

01028             MUNLOCK(ErrorMessageLock);
01029             SETERROR(-1);
01030         }
01031         *t = DUMMYFUN;
01032     /*
01033             Now mark it clean to avoid further interference.
01034             Normalize will remove this object.
01035     */
01036     t[2] = 0;
01037 }
01038 else {
01039     /* Note that the case 0^ is treated in Normalize */
01040
01041     t1 = AddRHS(AT.ebufnum,1);
01042     m = t + FUNHEAD;
01043     if ( *m > 0 ) {
01044         m += ARGHEAD;
01045         i = t[FUNHEAD] - ARGHEAD;
01046         while ( (t1 + i + 10) > C->Top )
01047             t1 = DoubleCbuffer(AT.ebufnum,t1,9);
01048         while ( --i >= 0 ) *t1++ = *m++;
01049     }
01050     else {
01051         if ( (t1 + 20) > C->Top )
01052             t1 = DoubleCbuffer(AT.ebufnum,t1,10);
01053         ToGeneral(m,t1,1);
01054         t1 += *t1;
01055     }
01056     *t1++ = 0;
01057     C->rhs[C->numrhs+1] = t1;
01058     C->Pointer = t1;
01059
01060     /* No provisions yet for commuting objects */
01061
01062     C->CanCommu[C->numrhs] = 1;
01063     *t++ = SUBEXPRESSION;
01064     *t++ = SUBEXPSIZE;
01065     *t++ = C->numrhs;
01066     *t++ = r[1];
01067     *t++ = AT.ebufnum;
01068     #if SUBEXPSIZE > 5
01069     Important: we may not have enough spots here
01070     #endif
01071     FILLSUB(t) /* Important: We have maybe only 5 spots! */
01072     r += 2;
01073     m = term + *term;
01074     do { *t++ = *r++; } while ( r < m );
01075     *term -= WORDDIF(r,t);
01076     goto ReStart;
01077 }
01078 }
01079 }
01080 else if ( *t == SUMF1 || *t == SUMF2 ) {
01081     /*
01082         What we are looking for is:
01083         1-st argument: Single symbol or index.
01084         2-nd argument: Number.
01085         3-rd argument: Number.
01086         (4-th argument):Number.
01087         One more argument.
01088         This would activate the summation procedure.
01089         Note that the initiated recursion here can be done
01090         without upsetting the regular procedures.
01091     */
01092     WORD *tstop, lcounter, lcmin, lcmax, lcinc;
01093     tstop = t + t[1];
01094     r = t+FUNHEAD;
01095     if ( r+6 < tstop && r[2] == -SNUMBER && r[4] == -SNUMBER
01096         && ( r[0] == -SYMBOL )
01097         || ( r[0] == -INDEX && r[1] >= AM.OffsetIndex
01098             && r[3] >= 0 && r[3] < AM.OffsetIndex
01099             && r[5] >= 0 && r[5] < AM.OffsetIndex ) ) {
01100         lcounter = r[0] == -INDEX ? -r[1]: r[1]; /* The loop counter */
01101         lcmin = r[3];
01102         lcmax = r[5];
01103         r += 6;
01104         if ( *r == -SNUMBER && r+2 < tstop ) {
01105             lcinc = r[1];
01106             r += 2;
01107         }
01108         else lcinc = 1;
01109         if ( r < tstop && ( ( *r > 0 && (r++*r) == tstop )
01110             || ( *r <= -FUNCTION && r+1 == tstop )
01111             || ( *r > -FUNCTION && *r < 0 && r+2 == tstop ) ) ) {
01112             m = AddRHS(AT.ebufnum,1);
01113             if ( *r > 0 ) {
01114                 i = *r - ARGHEAD;

```

```

01115         r += ARGHEAD;
01116         while ( (m + i + 10) > C->Top )
01117             m = DoubleCbuffer(AT.ebufnum,m,11);
01118         while ( --i >= 0 ) *m++ = *r++;
01119     }
01120     else {
01121         while ( (m + 20) > C->Top )
01122             m = DoubleCbuffer(AT.ebufnum,m,12);
01123         ToGeneral(r,m,1);
01124         m += *m;
01125     }
01126     *m++ = 0;
01127     C->rhs[C->numrhs+1] = m;
01128     C->Pointer = m;
01129     m = AT.TMout;
01130     *m++ = 6;
01131     if ( *t == SUMF1 ) *m++ = SUMNUM1;
01132     else *m++ = SUMNUM2;
01133     *m++ = lcounter;
01134     *m++ = lcmin;
01135     *m++ = lcmax;
01136     *m++ = lcinc;
01137     m = t + t[1];
01138     r = C->rhs[C->numrhs];
01139 /*
01140     Test now if the argument was already evaluated.
01141     In that case it needs a new subexpression prototype.
01142     In either case we replace the function now by a
01143     subexpression prototype.
01144 */
01145     if ( *r >= (SUBEXPSIZE+4)
01146         && ABS(*(r+r-1)) < (*r - 1)
01147         && r[1] == SUBEXPRESSION ) {
01148         r++;
01149         i = r[1] - 5;
01150         *t++ = *r++; *t++ = *r++; *t++ = C->numrhs;
01151         r++; *t++ = *r++; *t++ = AT.ebufnum; r++;
01152         while ( --i >= 0 ) *t++ = *r++;
01153     }
01154     else {
01155         *t++ = SUBEXPRESSION;
01156         *t++ = 4+SUBEXPSIZE;
01157         *t++ = C->numrhs;
01158         *t++ = 1;
01159         *t++ = AT.ebufnum;
01160         FILLSUB(t)
01161         if ( lcounter < 0 ) {
01162             *t++ = INDTOIND;
01163             *t++ = 4;
01164             *t++ = -lcounter;
01165         }
01166         else {
01167             *t++ = SYMTONUM;
01168             *t++ = 4;
01169             *t++ = lcounter;
01170         }
01171         *t++ = lcmin;
01172     }
01173     t2 = term + *term;
01174     while ( m < t2 ) *t++ = *m++;
01175     *term = WORDDIF(t,term);
01176     AN.TeInFun = -C->numrhs;
01177     AR.TePos = 0;
01178     AN.TeSuOut = 0;
01179     AT.TMbuff = AT.ebufnum;
01180     DONE(C->numrhs)
01181 }
01182 }
01183 }
01184 else if ( *t == TOPOLOGIES ) {
01185 /*
01186     Syntax:
01187     topologies_(nloops,nlegs,setvertexsizes,setext,setint[,options])
01188 */
01189     t1 = t+FUNHEAD; t2 = t+t[1];
01190     if ( *t1 == -SNUMBER && t1[1] >= 0 &&
01191         t1[2] == -SNUMBER && ( t1[3] >= 0 || t1[3] == -2 ) &&
01192         t1[4] == -SETSET && Sets[t1[5]].type == CNUMBER &&
01193         t1[6] == -SETSET && Sets[t1[7]].type == CVECTOR &&
01194         t1[8] == -SETSET && Sets[t1[9]].type == CVECTOR &&
01195         t1+10 <= t2 ) {
01196         if ( t1+10 == t2 || ( t1+12 <= t2 && ( t1[10] == -SNUMBER ||
01197             ( t1[10] == -SETSET &&
01198                 Sets[t1[5]].last-Sets[t1[5]].first ==
01199                 Sets[t1[11]].last-Sets[t1[11]].first ) ) ) {
01200             AN.TeInFun = -15;
01201             AN.TeSuOut = 0;

```

```

01202         AR.TePos = -1;
01203         AR.funoffset = t - term;
01204         DONE(1)
01205     }
01206 }
01207 }
01208 else if ( *t == DIAGRAMS ) {
01209 /*
01210     Syntax:
01211     diagrams_(model,setinparticles,setoutparticles,
01212         setextmomenta,setintmomenta,couplings or loops)
01213 */
01214     if ( AC.nummodels > 0 ) { /* No model, no diagrams */
01215         t1 = t+FUNHEAD; t2 = t+t[1];
01216         if (
01217             t1[0] == -SETSET && Sets[t1[1]].type == CMODEL &&
01218             t1[2] == -SETSET && ( Sets[t1[3]].type == CFUNCTION
01219                 || ( Sets[t1[3]].type == ANYTYPE && ( Sets[t1[3]].first == Sets[t1[3]].last ) ) )
01220             &&
01221             t1[4] == -SETSET && ( Sets[t1[5]].type == CFUNCTION
01222                 || ( Sets[t1[5]].type == ANYTYPE && ( Sets[t1[5]].first == Sets[t1[5]].last ) ) )
01223             &&
01224             t1[6] == -SETSET && Sets[t1[7]].type == CVECTOR &&
01225             t1[8] == -SETSET && Sets[t1[9]].type == CVECTOR &&
01226             t1+12 <= t2 ) {
01227             /*
01228             Test that the sets of particles correspond to particles
01229             of the set model.
01230             */
01231             MODEL *m = AC.models[SetElements[Sets[t1[1]].first]];
01232             int nn0,nn1,nn2;
01233             for ( nn0 = 3; nn0 <= 5; nn0 += 2 ) {
01234                 for ( nn1 = Sets[t1[nn0]].first; nn1 < Sets[t1[nn0]].last; nn1++ ) {
01235                     for ( nn2 = 0; nn2 < m->nparticles; nn2++ ) {
01236                         if ( m->vertices[nn2]->particles[0].number == SetElements[nn1]
01237                             || m->vertices[nn2]->particles[1].number == SetElements[nn1] ) break;
01238                     }
01239                     if ( nn2 >= m->nparticles ) goto doesnotwork;
01240                 }
01241             }
01242             /*
01243             Now test for a single argument indicating the order
01244             in perturbation theory.
01245             */
01246             if ( ( t1[10] == -SNUMBER && t1[11] >= 0 && t1+12 == t2 )
01247                 || ( t1[10] == -SYMBOL && t1+12 == t2 )
01248                 || ( t1+10+t1[10] == t2 && t1+10+ARGHEAD+t1[10+ARGHEAD] == t2
01249                     && t1[11+ARGHEAD] == SYMBOL ) ) {
01250                 /*
01251                 Now test that all symbols are valid coupling constants.
01252                 */
01253                 if ( t1+12 > t2 ) {
01254                     WORD *ttl = t1+13+ARGHEAD, im;
01255                     t2 -= ABS(t2[-1]);
01256                     while ( ttl < t2 ) {
01257                         for ( im = 0; im < m->ncouplings; im++ ) {
01258                             if ( *ttl == m->couplings[im] ) break;
01259                         }
01260                         if ( im >= m->ncouplings ) goto doesnotwork;
01261                         ttl += 2;
01262                     }
01263                     AN.TeInFun = -16;
01264                     AN.TeSuOut = 0;
01265                     AR.TePos = -1;
01266                     AR.funoffset = t - term;
01267                     DONE(1)
01268                 }
01269                 else if ( ( ( t1[10] == -SNUMBER && t1[11] >= 0 && t1+12 == t2-2 )
01270                     || ( t1[10] == -SYMBOL && t1+12 == t2-2 )
01271                     || ( t1+10+t1[10] == t2-2 && t1+10+ARGHEAD+t1[10+ARGHEAD] == t2-2
01272                         && t1[11+ARGHEAD] == SYMBOL ) ) && t2[-2] == -SNUMBER ) {
01273                     /*
01274                     With options at t2[-2],t2[-1]
01275                     Now test that all symbols are valid coupling constants.
01276                     */
01277                     t2 -= 2;
01278                     if ( t1+12 > t2 ) {
01279                         WORD *ttl = t1+13+ARGHEAD, im;
01280                         t2 -= ABS(t2[-1]);
01281                         while ( ttl < t2 ) {
01282                             for ( im = 0; im < m->ncouplings; im++ ) {
01283                                 if ( *ttl == m->couplings[im] ) break;
01284                             }
01285                             if ( im >= m->ncouplings ) goto doesnotwork;
01286                             ttl += 2;
01287                         }
01288                     }
01289                 }
01290             }
01291         }
01292     }
01293 }

```

```

01287     }
01288     AN.TeInFun = -16;
01289     AN.TeSuOut = 0;
01290     AR.TePos = -1;
01291     AR.funoffset = t - term;
01292     DONE(1)
01293 }
01294 doesnotwork;;
01295     }
01296     }
01297 }
01298 if ( functions[funnum-FUNCTION].spec <= 0
01299     || ( t[2] & (DIRTYFLAG|MUSTCLEANPRF) ) != 0 ) {
01300     funflag = 1;
01301 }
01302 if ( *t <= MAXBUILTINFUNCTION ) {
01303     if ( *t <= DELTAP && *t >= THETA ) { /* Speeds up by 2 or 3 compares */
01304         if ( *t == THETA || *t == THETA2 ) {
01305             WORD *tstop, *tt2, kk;
01306             tstop = t + t[1];
01307             tt2 = t + FUNHEAD;
01308             while ( tt2 < tstop ) {
01309                 if ( *tt2 > 0 && tt2[1] != 0 ) goto DoSpec;
01310                 NEXTARG(tt2)
01311             }
01312             if ( !AT.RecFlag ) {
01313                 if ( ( kk = DoTheta(BHEAD t) ) == 0 ) {
01314                     *term = 0;
01315                     DONE(0)
01316                 }
01317                 else if ( kk > 0 ) {
01318                     m = t + t[1];
01319                     r = term + *term;
01320                     while ( m < r ) *t++ = *m++;
01321                     *term = WORDDIF(t,term);
01322                     goto ReStart;
01323                 }
01324             }
01325         }
01326     else if ( *t == DELTA2 || *t == DELTAP ) {
01327         WORD *tstop, *tt2, kk;
01328         tstop = t + t[1];
01329         tt2 = t + FUNHEAD;
01330         while ( tt2 < tstop ) {
01331             if ( *tt2 > 0 && tt2[1] != 0 ) goto DoSpec;
01332             NEXTARG(tt2)
01333         }
01334         if ( !AT.RecFlag ) {
01335             if ( ( kk = DoDelta(t) ) == 0 ) {
01336                 *term = 0;
01337                 DONE(0)
01338             }
01339             else if ( kk > 0 ) {
01340                 m = t + t[1];
01341                 r = term + *term;
01342                 while ( m < r ) *t++ = *m++;
01343                 *term = WORDDIF(t,term);
01344                 goto ReStart;
01345             }
01346         }
01347     }
01348     else if ( *t == DISTRIBUTION && t[FUNHEAD] == -SNUMBER
01349         && t[FUNHEAD+1] >= -2 && t[FUNHEAD+1] <= 2
01350         && t[FUNHEAD+2] == -SNUMBER
01351         && t[FUNHEAD+4] <= -FUNCTION
01352         && t[FUNHEAD+5] <= -FUNCTION ) {
01353         WORD *ttt = t+FUNHEAD+6, *tttstop = t+t[1];
01354         while ( ttt < tttstop ) {
01355             if ( *ttt == -DOLLAREXPRESSION ) break;
01356             NEXTARG(ttt);
01357         }
01358         if ( ttt >= tttstop ) {
01359             AN.TeInFun = -1;
01360             AN.TeSuOut = 0;
01361             AR.TePos = -1;
01362             DONE(1)
01363         }
01364     }
01365     else if ( *t == DELTA3 && ((t[1]-FUNHEAD) & 1) == 0 ) {
01366         AN.TeInFun = -2;
01367         AN.TeSuOut = 0;
01368         AR.TePos = -1;
01369         DONE(1)
01370     }
01371     else if ( ( *t == TABLEFUNCTION ) && ( t[FUNHEAD] <= -FUNCTION )
01372         && ( T = functions[-t[FUNHEAD]-FUNCTION].tabl ) != 0
01373         && ( t[1] >= FUNHEAD+1+2*ABS(T->numind) ) )

```



```

01374      && ( t[FUNHEAD+1] == -SYMBOL ) ) {
01375  /*
01376      The case of table_(tab,sym1,...,symn)
01377  */
01378      for ( isp = 0; isp < ABS(T->numind); isp++ ) {
01379          if ( t[FUNHEAD+1+2*isp] != -SYMBOL ) break;
01380      }
01381      if ( isp >= ABS(T->numind) ) {
01382          AN.TeInFun = -3;
01383          AN.TeSuOut = 0;
01384          AR.TePos = -1;
01385          DONE(1)
01386      }
01387  }
01388  else if ( *t == TABLEFUNCTION && t[FUNHEAD] <= -FUNCTION
01389  && ( T = functions[-t[FUNHEAD]-FUNCTION].tabl ) != 0
01390  && ( t[1] == FUNHEAD+2 )
01391  && ( t[FUNHEAD+1] <= -FUNCTION ) ) {
01392  /*
01393      The case of table_(tab,fun)
01394  */
01395      AN.TeInFun = -3;
01396      AN.TeSuOut = 0;
01397      AR.TePos = -1;
01398      DONE(1)
01399  }
01400  else if ( *t == FACTORIN ) {
01401      if ( t[1] == FUNHEAD+2 && t[FUNHEAD] == -DOLLAREXPRESSION ) {
01402          AN.TeInFun = -4;
01403          AN.TeSuOut = 0;
01404          AR.TePos = -1;
01405          DONE(1)
01406      }
01407      else if ( t[1] == FUNHEAD+2 && t[FUNHEAD] == -EXPRESSION ) {
01408          AN.TeInFun = -5;
01409          AN.TeSuOut = 0;
01410          AR.TePos = -1;
01411          DONE(1)
01412      }
01413  }
01414  else if ( *t == TERMSINBRACKET ) {
01415      if ( t[1] == FUNHEAD || (
01416          t[1] == FUNHEAD+2
01417          && t[FUNHEAD] == -SNUMBER
01418          && t[FUNHEAD+1] == 0
01419      ) ) {
01420          AN.TeInFun = -6;
01421          AN.TeSuOut = 0;
01422          AR.TePos = -1;
01423          DONE(1)
01424      }
01425  /*
01426      The other cases have not yet been implemented
01427      We still have to add the case of short arguments
01428      First the different bracket in same expression
01429
01430      else if ( t[1] > FUNHEAD+ARGHEAD
01431          && t[FUNHEAD] == t[1]-FUNHEAD
01432          && t[FUNHEAD+ARGHEAD] == t[1]-FUNHEAD-ARGHEAD
01433          && t[t[1]-1] == 3
01434          && t[t[1]-2] == 1
01435          && t[t[1]-3] == 1 ) {
01436          AN.TeInFun = -6;
01437          AN.TeSuOut = 0;
01438          AR.TePos = -1;
01439          DONE(1)
01440      }
01441
01442      Next the bracket in an other expression
01443
01444      else if ( t[1] > FUNHEAD+ARGHEAD+2
01445          && t[FUNHEAD] == -EXPRESSION
01446          && t[FUNHEAD+2] == t[1]-FUNHEAD-2
01447          && t[FUNHEAD+ARGHEAD+2] == t[1]-FUNHEAD-ARGHEAD-2
01448          && t[t[1]-1] == 3
01449          && t[t[1]-2] == 1
01450          && t[t[1]-3] == 1 ) {
01451          AN.TeInFun = -6;
01452          AN.TeSuOut = 0;
01453          AR.TePos = -1;
01454          DONE(1)
01455      }
01456  */
01457  }
01458  else if ( *t == EXTRASYMFUN ) {
01459      if ( t[1] == FUNHEAD+2 && (
01460          ( t[FUNHEAD] == -SNUMBER && t[FUNHEAD+1] <= cbuf[AM.sbufnum].numrhs

```

```

01461         && t[FUNHEAD+1] > 0 ) ||
01462         ( t[FUNHEAD] == -SYMBOL && t[FUNHEAD+1] < MAXVARIABLES
01463         && t[FUNHEAD+1] >= MAXVARIABLES-cbuf[AM.sbufnum].numrhs ) ) {
01464     AN.TeInFun = -7;
01465     AN.TeSuOut = 0;
01466     AR.TePos = -1;
01467     DONE(1)
01468 }
01469 else if ( t[1] == FUNHEAD ) {
01470     AN.TeInFun = -7;
01471     AN.TeSuOut = 0;
01472     AR.TePos = -1;
01473     DONE(1)
01474 }
01475 }
01476 else if ( *t == DIVFUNCTION || *t == REMFUNCTION
01477         || *t == INVERSEFUNCTION || *t == MULFUNCTION
01478         || *t == GCDFUNCTION ) {
01479     WORD *tf;
01480     int todo = 1, numargs = 0;
01481     tf = t + FUNHEAD;
01482     while ( tf < t + t[1] ) {
01483         DOLLARS d;
01484         if ( *tf == -DOLLEXPRESSION ) {
01485             d = Dollars + tf[1];
01486             if ( d->type == DOLWILDARGS ) {
01487                 WORD *tterm = AT.WorkPointer, *tw;
01488                 WORD *ta = term, *tb = tterm, *tc, *td = term + *tterm;
01489                 while ( ta < t ) *tb++ = *ta++;
01490                 tc = tb;
01491                 while ( ta < tf ) *tb++ = *ta++;
01492                 tw = d->where+1;
01493                 while ( *tw ) {
01494                     if ( *tw < 0 ) {
01495                         if ( *tw > -FUNCTION ) *tb++ = *tw++;
01496                         *tb++ = *tw++;
01497                     }
01498                     else {
01499                         int ia;
01500                         for ( ia = 0; ia < *tw; ia++ ) *tb++ = *tw++;
01501                     }
01502                 }
01503                 NEXTARG(ta)
01504                 while ( ta < t+t[1] ) *tb++ = *ta++;
01505                 tc[1] = tb-tc;
01506                 while ( ta < td ) *tb++ = *ta++;
01507                 *tterm = tb - tterm;
01508                 {
01509                     int ia, na = *tterm;
01510                     ta = tterm; tb = term;
01511                     for ( ia = 0; ia < na; ia++ ) *tb++ = *ta++;
01512                 }
01513                 if ( tb > AT.WorkTop ) {
01514                     MLOCK(ErrorMessageLock);
01515                     MesWork();
01516                     goto EndTest2;
01517                 }
01518                 AT.WorkPointer = tb;
01519                 goto ReStart;
01520             }
01521         }
01522         NEXTARG(tf);
01523     }
01524     tf = t + FUNHEAD;
01525     while ( tf < t + t[1] ) {
01526         numargs++;
01527         if ( *tf > 0 && tf[1] != 0 ) todo = 0;
01528         NEXTARG(tf);
01529     }
01530     if ( todo && numargs == 2 ) {
01531         if ( *t == DIVFUNCTION ) AN.TeInFun = -9;
01532         else if ( *t == REMFUNCTION ) AN.TeInFun = -10;
01533         else if ( *t == INVERSEFUNCTION ) AN.TeInFun = -11;
01534         else if ( *t == MULFUNCTION ) AN.TeInFun = -14;
01535         else if ( *t == GCDFUNCTION ) AN.TeInFun = -8;
01536         AN.TeSuOut = 0;
01537         AR.TePos = -1;
01538         DONE(1)
01539     }
01540     else if ( todo && numargs == 3 ) {
01541         if ( *t == DIVFUNCTION ) AN.TeInFun = -9;
01542         else if ( *t == REMFUNCTION ) AN.TeInFun = -10;
01543         else if ( *t == GCDFUNCTION ) AN.TeInFun = -8;
01544         AN.TeSuOut = 0;
01545         AR.TePos = -1;
01546         DONE(1)
01547     }

```

```

01548         else if ( todo && *t == GCDFUNCTION ) {
01549             AN.TeInFun = -8;
01550             AN.TeSuOut = 0;
01551             AR.TePos = -1;
01552             DONE(1)
01553         }
01554     }
01555     else if ( *t == PERMUTATIONS && ( ( t[1] >= FUNHEAD+1
01556         && t[FUNHEAD] <= -FUNCTION ) || ( t[1] >= FUNHEAD+3
01557         && t[FUNHEAD] == -SNUMBER && t[FUNHEAD+2] <= -FUNCTION ) ) ) {
01558         AN.TeInFun = -12;
01559         AN.TeSuOut = 0;
01560         AR.TePos = -1;
01561         DONE(1)
01562     }
01563     else if ( *t == PARTITIONS ) {
01564         if ( TestPartitions(t,&(AT.partitions)) ) {
01565             AT.partitions.where = t-term;
01566             AN.TeInFun = -13;
01567             AN.TeSuOut = 0;
01568             AR.TePos = -1;
01569             DONE(1)
01570         }
01571     }
01572 }
01573 }
01574 t += t[1];
01575 } while ( t < m );
01576 if ( funflag ) { /* Search in functions */
01577 DoSpec:
01578     t = term;
01579     AT.NestPoin->termsize = t;
01580     if ( AT.NestPoin == AT.Nest ) AN.EndNest = t + *t;
01581     t++;
01582     oldncmod = AN.ncmod;
01583     if ( t < m ) do {
01584         if ( *t < FUNCTION ) {
01585             t += t[1]; continue;
01586         }
01587         if ( AN.ncmod && ( ( AC.modmode & ALSOFUNARGS ) == 0 ) ) {
01588             if ( *t != AR.PolyFun ) AN.ncmod = 0;
01589             else AN.ncmod = oldncmod;
01590         }
01591         r = t + t[1];
01592         funnum = *t;
01593         if ( *t >= FUNCTION + WILDOFFSET ) funnum -= WILDOFFSET;
01594         if ( ( *t == NUMFACTORS || *t == FIRSTTERM || *t == CONTENTTERM )
01595             && t[1] == FUNHEAD+2 &&
01596             ( t[FUNHEAD] == -EXPRESSION || t[FUNHEAD] == -DOLLAREXPRESSION ) ) {
01597 /*
01598             if ( *t == NUMFACTORS ) {
01599                 This we leave for Normalize
01600             }
01601 */
01602         }
01603         else if ( functions[funnum-FUNCTION].spec <= 0 ) {
01604             AT.NestPoin->funsize = t + 1;
01605             t1 = t;
01606             t += FUNHEAD;
01607             while ( t < r ) { /* Sum over arguments */
01608                 if ( *t > 0 && t[1] ) { /* Argument is dirty */
01609                     AT.NestPoin->argsize = t;
01610                     AT.NestPoin++;
01611 /*
01612                     stop = t + *t; */
01613                     t2 = t;
01614                     t += ARGHEAD;
01615                     while ( t < AT.NestPoin[-1].argsize+(AT.NestPoin[-1].argsize) ) {
01616                         /* Sum over terms */
01617                         AT.RecFlag++;
01618                         i = *t;
01619                         AN.subsubveto = 1;
01620 /*
01621                         AN.subsubveto repairs a bug that became apparent
01622                         in an example by York Schroeder:
01623                         f(k1.k1)*replace_(k1,2*k2)
01624                         Is it possible to repair the counting of the various
01625                         length indicators? (JV 1-jun-2010)
01626                         if ( ( retvalue = TestSub(BHEAD t,level) ) != 0 ) {
01627 /*
01628                             Possible size changes:
01629                             Note defs at 471,467,460,400,425,328
01630 */
01631 redosize:
01632                             if ( i > *t ) {
01633 /*
01634                             i -= *t;

```

```

01635         *t2 -= i;
01636         t1[1] -= i;
01637         t += *t;
01638         r = t + i;
01639         m = term + *term;
01640         while ( r < m ) *t++ = *r++;
01641         *term -= i;
01642     /*
01643         i -= *t;
01644         t += *t;
01645         r = t + i;
01646         m = AN.EndNest;
01647         while ( r < m ) *t++ = *r++;
01648         t = AT.NestPoin[-1].argsize + ARGHEAD;
01649         n = AT.Nest;
01650         while ( n < AT.NestPoin ) {
01651             *(n->argsize) -= i;
01652             *(n->funsize) -= i;
01653             *(n->termsize) -= i;
01654             n++;
01655         }
01656         AN.EndNest -= i;
01657     }
01658     AN.subsubveto = 0;
01659     t1[2] = 1;
01660     if ( *t1 == AR.PolyFun && AR.PolyFunType == 2 )
01661         t1[2] |= MUSTCLEANPRF;
01662     AT.RecFlag--;
01663     AT.NestPoin--;
01664     AN.TeInFun++;
01665     AR.TePos = 0;
01666     AN.ncmod = oldncmod;
01667     DONE(retval)
01668 }
01669 else {
01670     /*
01671      * Somehow the next line fixes Issue #106.
01672      */
01673     i = *t;
01674     Normalize(BHEAD t);
01675     /* if ( i > *t ) { retval = 1; goto redosize; } */
01676     /*
01677      * Experimentally, the next line fixes Issue #105.
01678      */
01679     if ( *t == 0 ) { retval = 1; goto redosize; }
01680     {
01681         WORD *tend = t + *t, *tt = t+1;
01682         stilldirty = 0;
01683         tend -= ABS(tend[-1]);
01684         while ( tt < tend ) {
01685             if ( *tt == SUBEXPRESSION || *tt == EXPRESSION ) {
01686                 stilldirty = 1; break;
01687             }
01688             tt += tt[1];
01689         }
01690     }
01691     if ( i > *t ) {
01692         /*
01693          * We should not forget to correct the Nest
01694          * stack. That caused trouble in the past.
01695          */
01696         retval = 1;
01697         i -= *t;
01698         t += *t;
01699         r = t + i;
01700         m = AN.EndNest;
01701         while ( r < m ) *t++ = *r++;
01702         t = AT.NestPoin[-1].argsize + ARGHEAD;
01703         n = AT.Nest;
01704         while ( n < AT.NestPoin ) {
01705             *(n->argsize) -= i;
01706             *(n->funsize) -= i;
01707             *(n->termsize) -= i;
01708             n++;
01709         }
01710         AN.EndNest -= i;
01711     }
01712 }
01713 AN.subsubveto = 0;
01714 AT.RecFlag--;
01715 t += *t;
01716 }
01717 AT.NestPoin--;
01718 /*
01719 Argument contains no subexpressions.
01720 It should be normalized and sorted.
01721 The main problem is the storage.

```

```

01722 */
01723
01724     t = AT.NestPoin->argsize;
01725     j = *t;
01726     t += ARGHEAD;
01727     NewSort(BHEAD0);
01728     if ( *t1 == AR.PolyFun && AR.PolyFunType == 2 ) {
01729         AR.CompareRoutine = (COMPAREDUMMY)(&CompareSymbols);
01730         AR.SortType = SORTHIGHFIRST;
01731     }
01732     if ( AT.WorkPointer < term + *term )
01733         AT.WorkPointer = term + *term;
01734
01735     while ( t < AT.NestPoin->argsize+(AT.NestPoin->argsize) ) {
01736         m = AT.WorkPointer;
01737         r = t + *t;
01738         do { *m++ = *t++; } while ( t < r );
01739         r = AT.WorkPointer;
01740         AT.WorkPointer = r + *r;
01741         if ( Normalize(BHEAD r) ) {
01742             if ( *t1 == AR.PolyFun && AR.PolyFunType == 2 ) {
01743                 AR.SortType = oldsorttype;
01744                 AR.CompareRoutine = (COMPAREDUMMY)oldcompareroutine;
01745                 t1[2] |= MUSTCLEANPRF;
01746             }
01747             LowerSortLevel(); goto EndTest;
01748         }
01749         if ( AN.ncmod != 0 ) {
01750             if ( *r ) {
01751                 if ( Modulus(r) ) {
01752                     LowerSortLevel();
01753                     AT.WorkPointer = r;
01754                     if ( *t1 == AR.PolyFun && AR.PolyFunType == 2 ) {
01755                         AR.SortType = oldsorttype;
01756                         AR.CompareRoutine = (COMPAREDUMMY)oldcompareroutine;
01757                         t1[2] |= MUSTCLEANPRF;
01758                     }
01759                     goto EndTest;
01760                 }
01761             }
01762             if ( AR.PolyFun > 0 ) {
01763                 if ( PrepPoly(BHEAD r,1) != 0 ) goto EndTest;
01764             }
01765             if ( *r ) StoreTerm(BHEAD r);
01766             AT.WorkPointer = r;
01767         }
01768         if ( EndSort(BHEAD AT.WorkPointer+ARGHEAD,1) < 0 ) goto EndTest;
01769         m = AT.WorkPointer+ARGHEAD;
01770         if ( *t1 == AR.PolyFun && AR.PolyFunType == 2 ) {
01771             AR.SortType = oldsorttype;
01772             AR.CompareRoutine = (COMPAREDUMMY)oldcompareroutine;
01773             t1[2] |= MUSTCLEANPRF;
01774         }
01775         while ( *m ) m += *m;
01776         i = WORDDIF(m,AT.WorkPointer);
01777         *AT.WorkPointer = i;
01778         AT.WorkPointer[1] = stilldirty;
01779         if ( ToFast(AT.WorkPointer,AT.WorkPointer) ) {
01780             m = AT.WorkPointer;
01781             if ( *m <= -FUNCTION ) { m++; i = 1; }
01782             else { m += 2; i = 2; }
01783         }
01784         j = i - j;
01785         if ( j > 0 ) {
01786             r = m + j;
01787             if ( r > AT.WorkTop ) {
01788                 MLOCK(ErrorMessageLock);
01789                 MesWork();
01790                 goto EndTest2;
01791             }
01792             do { *--r = *--m; } while ( m > AT.WorkPointer );
01793             AT.WorkPointer = r;
01794             m = AN.EndNest;
01795             r = m + j;
01796             stop = AT.NestPoin->argsize+(AT.NestPoin->argsize);
01797             do { *--r = *--m; } while ( m >= stop );
01798         }
01799         else if ( j < 0 ) {
01800             m = AT.NestPoin->argsize+(AT.NestPoin->argsize);
01801             r = m + j;
01802             do { *r++ = *m++; } while ( m < AN.EndNest );
01803         }
01804         m = AT.NestPoin->argsize;
01805         r = AT.WorkPointer;
01806         while ( --i >= 0 ) *m++ = *r++;
01807         n = AT.Nest;
01808         while ( n <= AT.NestPoin ) {

```

```

01809             if ( *(n->argsize) > 0 && n != AT.NestPoin )
01810                 *(n->argsize) += j;
01811             *(n->funsize) += j;
01812             *(n->termsize) += j;
01813             n++;
01814         }
01815         AN.EndNest += j;
01816         /* (AT.NestPoin->argsize)[1] = 0; */
01817         if ( funnum == DENOMINATOR || funnum == EXPONENT ) {
01818             if ( Normalize(BHEAD term) ) {
01819                 /*
01820                  In this case something has been substituted
01821                  Either a $ or a replace_?????
01822                  Originally we had here:
01823
01824                  goto EndTest;
01825
01826                  It seems better to restart.
01827                 */
01828                 AN.ncmod = oldncmod;
01829                 goto ReStart;
01830             }
01831             /*
01832              And size changes here?????
01833             */
01834         }
01835         AN.ncmod = oldncmod;
01836         goto ReStart;
01837     }
01838     else if ( *t == -DOLLAREXPRESSION ) {
01839         if ( ( *t1 == TERMSINEXPR || *t1 == SIZEOFFUNCTION )
01840             && t1[1] == FUNHEAD+2 ) {}
01841         else {
01842             if ( AR.Eside != LHSIDE ) {
01843                 AN.TeInFun = 1; AR.TePos = 0;
01844                 AT.Tmbuff = AM.dbufnum; t1[2] |= DIRTYFLAG;
01845                 AN.ncmod = oldncmod;
01846                 DONE(1)
01847             }
01848             AC.lhdollarflag = 1;
01849         }
01850     }
01851     else if ( *t == -TERMSINBRACKET ) {
01852         if ( AR.Eside != LHSIDE ) {
01853             AN.TeInFun = 1; AR.TePos = 0;
01854             t1[2] |= DIRTYFLAG;
01855             AN.ncmod = oldncmod;
01856             DONE(1)
01857         }
01858     }
01859     else if ( AN.ncmod != 0 && *t == -SNUMBER ) {
01860         if ( AN.ncmod == 1 || AN.ncmod == -1 ) {
01861             isp = (UWORD)(AC.cmod[0]);
01862             isp = t[1] % isp;
01863             if ( ( AC.modmode & POSNEG ) != 0 ) {
01864                 if ( isp > (UWORD)(AC.cmod[0])/2 ) isp = isp - (UWORD)(AC.cmod[0]);
01865                 else if ( -isp > (UWORD)(AC.cmod[0])/2 ) isp = isp +
01866                     (UWORD)(AC.cmod[0]);
01867             }
01868             else {
01869                 if ( isp < 0 ) isp += (UWORD)(AC.cmod[0]);
01870             }
01871             if ( isp <= MAXPOSITIVE && isp >= -MAXPOSITIVE ) {
01872                 t[1] = isp;
01873             }
01874         }
01875         NEXTARG(t)
01876     }
01877     if ( funnum >= FUNCTION && functions[funnum-FUNCTION].tabl ) {
01878         /*
01879          Test whether the table catches
01880          Test 1: index arguments and range. i will be the number
01881                  of the element in the table.
01882         */
01883         WORD rhsnumber, *oldwork = AT.WorkPointer;
01884         WORD ii, *p, *pp, *ppstop;
01885         MINMAX *mm;
01886         T = functions[funnum-FUNCTION].tabl;
01887         /*
01888          Because of tables with a variable number of indices
01889          we need to make a copy of the pattern.
01890          If we do this in the Workspace we get problems with EndNest.
01891          This is why we use TermMalloc.
01892          Now Tpattern is a copy that can be modified.
01893         */
01894         #ifdef WITHPTHREADS

```

```

01895         pp = T->pattern[AT.identity];
01896     #else
01897     pp = T->pattern;
01898 #endif
01899     if ( Tpattern == 0 ) Tpattern = TermMalloc("Tpattern");
01900     p = Tpattern;
01901     ii = pp[1];
01902     for ( i = 0; i < ii; i++ ) *p++ = *pp++;
01903
01904     p = Tpattern + FUNHEAD+1;
01905     mm = T->mm;
01906     if ( T->sparse ) {
01907         WORD xx;
01908         t = t1+FUNHEAD;
01909         if ( T->numind == 0 ) { isp = 0; xx = 0; }
01910         else {
01911             if ( T->numind < 0 ) {
01912                 if ( *t != -SNUMBER && t[2] != -SNUMBER ) {
01913                     xx = ABS(T->numind);
01914                     goto teststrict;
01915                 }
01916                 xx = t[1]+1;
01917                 if ( xx < 2 || xx > -T->numind ) goto teststrict;
01918             }
01919             else xx = T->numind;
01920             for ( i = 0; i < xx; i++, t += 2 ) {
01921                 if ( *t != -SNUMBER ) break;
01922             }
01923             if ( i < xx ) goto teststrict;
01924             isp = FindTableTree(T,t1+FUNHEAD,2);
01925         }
01926         if ( isp < 0 ) {
01927             if ( T->strict == -2 ) {
01928                 rhsnumber = AM.zerorhs;
01929                 tbufnum = AM.zbufnum;
01930             }
01931             else if ( T->strict == -3 ) {
01932                 rhsnumber = AM.onerhs;
01933                 tbufnum = AM.zbufnum;
01934             }
01935             else if ( T->strict < 0 ) goto NextFun;
01936             else {
01937                 MLOCK(ErrorMessageLock);
01938                 MesPrint("Element in table is undefined");
01939                 if ( Tpattern ) {
01940                     TermFree(Tpattern,"Tpattern");
01941                     Tpattern = 0;
01942                 }
01943                 goto showtable;
01944             }
01945             /*
01946             Copy the indices;
01947             */
01948             t = t1+FUNHEAD+1;
01949             for ( i = 0; i < xx; i++ ) {
01950                 *p = *t; p+=2; t+=2;
01951             }
01952         }
01953         else {
01954             rhsnumber = T->tablepointers[isp+ABS(T->numind)];
01955             #if ( TABLEEXTENSION == 2 )
01956             tbufnum = T->bbufnum;
01957             #else
01958             tbufnum = T->tablepointers[isp+ABS(T->numind)+1];
01959             #endif
01960             t = t1+FUNHEAD+1;
01961             ii = xx;
01962             while ( --ii >= 0 ) {
01963                 *p = *t; t += 2; p += 2;
01964             }
01965         }
01966         if ( xx < ABS(T->numind) ) {
01967             p--; ppstop = Tpattern+Tpattern[1];
01968             pp = p+2*(-T->numind-xx);
01969             while ( pp < ppstop ) *p++ = *pp++;
01970             Tpattern[1] = p - Tpattern;
01971         }
01972         goto caughttable;
01973     }
01974     else {
01975         i = 0;
01976         t = t1 + FUNHEAD;
01977         j = T->numind;
01978         while ( --j >= 0 ) {
01979             if ( *t != -SNUMBER ) goto NextFun;
01980             t++;
01981             if ( *t < mm->mini || *t > mm->maxi ) {

```

```

01982         if ( T->bounds ) {
01983             MLOCK(ErrorMessageLock);
01984             MesPrint("Table boundary check. Argument %d",
01985                 T->numind-j);
01986 showtable:         AO.OutFill = AO.OutputLine = (UBYTE *)m;
01987                     AO.OutSkip = 8;
01988                     IniLine(0);
01989                     WriteSubTerm(tl,1);
01990                     FiniLine();
01991                     MUNLOCK(ErrorMessageLock);
01992                     if ( Tpattern ) {
01993                         TermFree(Tpattern,"Tpattern");
01994                         Tpattern = 0;
01995                     }
01996                     SETERROR(-1)
01997                 }
01998                 if ( Tpattern ) {
01999                     TermFree(Tpattern,"Tpattern");
02000                     Tpattern = 0;
02001                 }
02002                 goto NextFun;
02003             }
02004             i += ( *t - mm->mini ) * (LONG) (mm->size);
02005             *p = *t++;
02006             p += 2;
02007             mm++;
02008         }
02009 /*
02010         Test now whether the entry exists.
02011 */
02012         i *= TABLEEXTENSION;
02013         if ( T->tablepointers[i] == -1 ) {
02014             if ( T->strict == -2 ) {
02015                 rhsnumber = AM.zerorhs;
02016                 tbufnum = AM.zbufnum;
02017             }
02018             else if ( T->strict == -3 ) {
02019                 rhsnumber = AM.onerhs;
02020                 tbufnum = AM.zbufnum;
02021             }
02022             else if ( T->strict < 0 ) {
02023                 if ( Tpattern ) {
02024                     TermFree(Tpattern,"Tpattern");
02025                     Tpattern = 0;
02026                 }
02027                 goto NextFun;
02028             }
02029             else {
02030                 MLOCK(ErrorMessageLock);
02031                 MesPrint("Element in table is undefined");
02032                 if ( Tpattern ) {
02033                     TermFree(Tpattern,"Tpattern");
02034                     Tpattern = 0;
02035                 }
02036                 goto showtable;
02037             }
02038         }
02039         else {
02040             rhsnumber = T->tablepointers[i];
02041             #if ( TABLEEXTENSION == 2 )
02042                 tbufnum = T->tbufnum;
02043             #else
02044                 tbufnum = T->tablepointers[i+1];
02045             #endif
02046         }
02047     }
02048 /*
02049     If there are more arguments we have to do some
02050     pattern matching. This should be easy. We adapted the
02051     pattern, so that the array indices match already.
02052     Note that if there is no match the program will become
02053     very slow.
02054 */
02055 caughttable:
02056 #ifdef WITHPTHREADS
02057     AN.FullProto = T->prototype[AT.identity];
02058 #else
02059     AN.FullProto = T->prototype;
02060 #endif
02061     AN.WildValue = AN.FullProto + SUBEXPSIZE;
02062     AN.WildStop = AN.FullProto+AN.FullProto[1];
02063     ClearWild(BHEAD0);
02064     AN.RepFunNum = 0;
02065     AN.RepFunList = AN.EndNest;
02066     AT.WorkPointer = (WORD *)(((UBYTE *) (AN.EndNest)) + AM.MaxTer/2);
02067     if ( AT.WorkPointer >= AT.WorkTop ) {
02068         MLOCK(ErrorMessageLock);

```



```

02069         MesWork();
02070         MUNLOCK(ErrorMessageLock);
02071     }
02072     wilds = 0;
02073     if ( MatchFunction(BHEAD Tpattern,t1,&wilds) > 0 ) {
02074         AT.WorkPointer = oldwork;
02075         if ( AT.NestPoin != AT.Nest ) {
02076             AN.ncmod = oldncmod;
02077             if ( Tpattern ) {
02078                 TermFree(Tpattern,"Tpattern");
02079                 Tpattern = 0;
02080             }
02081             DONE(1)
02082         }
02083
02084         m = AN.FullProto;
02085         retvalue = m[2] = rhsnumber;
02086         m[4] = tbufnum;
02087         t = t1;
02088         j = t[1];
02089         i = m[1];
02090         if ( j > i ) {
02091             j = i - j;
02092             NCOPY(t,m,i);
02093             m = term + *term;
02094             while ( r < m ) *t++ = *r++;
02095             *term += j;
02096         }
02097         else if ( j < i ) {
02098             j = i-j;
02099             t = term + *term;
02100             while ( t >= r ) { t[j] = *t; t--; }
02101             t = t1;
02102             NCOPY(t,m,i);
02103             *term += j;
02104         }
02105         else {
02106             NCOPY(t,m,j);
02107         }
02108         AN.TeInFun = 0;
02109         AR.TePos = 0;
02110         AN.TeSuOut = -1;
02111         if ( AT.WorkPointer < term + *term ) AT.WorkPointer = term + *term;
02112         AT.TMbuff = tbufnum;
02113         AN.ncmod = oldncmod;
02114         DONE(retvalue);
02115     }
02116     AT.WorkPointer = oldwork;
02117     if ( Tpattern ) {
02118         TermFree(Tpattern,"Tpattern");
02119         Tpattern = 0;
02120     }
02121 }
02122 NextFun;;
02123 }
02124     else if ( ( t[2] & DIRTYFLAG ) != 0 ) {
02125         t += FUNHEAD;
02126         while ( t < r ) {
02127             if ( *t == FUNNYDOLLAR ) {
02128                 if ( AR.Eside != LHSIDE ) {
02129                     AN.TeInFun = 1;
02130                     AR.TePos = 0;
02131                     AT.TMbuff = AM.dbufnum;
02132                     AN.ncmod = oldncmod;
02133                     DONE(1)
02134                 }
02135                 AC.lhdollarflag = 1;
02136             }
02137             t++;
02138         }
02139     }
02140     t = r;
02141     AN.ncmod = oldncmod;
02142     } while ( t < m );
02143 }
02144 Done:
02145     if ( Tpattern ) {
02146         TermFree(Tpattern,"Tpattern");
02147         Tpattern = 0;
02148     }
02149     return(retvalue);
02150 EndTest;;
02151     MLOCK(ErrorMessageLock);
02152 EndTest2;;
02153     MesCall("TestSub");
02154     MUNLOCK(ErrorMessageLock);
02155     SETERROR(-1)

```

```

02156 }
02157
02158 /*
02159     #] TestSub :
02160     #[ InFunction :      WORD InFunction(term,termout)
02161 */
02174 WORD InFunction(PHEAD WORD *term, WORD *termout)
02175 {
02176     GETBIDENTITY
02177     WORD *m, *t, *r, *rr, sign = 1, oldncmod;
02178     WORD *u, *v, *w, *from, *to,
02179     ipp, olddefer = AR.DeferFlag, oldPolyFun = AR.PolyFun, i, j;
02180     LONG numterms;
02181     from = t = term;
02182     r = t + *t - 1;
02183     m = r - ABS(*r) + 1;
02184     t++;
02185     while ( t < m ) {
02186         if ( *t >= FUNCTION+WILDOFFSET ) ipp = *t - WILDOFFSET;
02187         else ipp = *t;
02188         if ( AR.TePos ) {
02189             if ( ( term + AR.TePos ) == t ) {
02190                 m = termout;
02191                 while ( from < t ) *m++ = *from++;
02192                 *m++ = DENOMINATOR;
02193                 *m++ = t[1] + 4 + FUNHEAD + ARGHEAD;
02194                 *m++ = DIRTYFLAG;
02195                 FILLFUN3(m)
02196                 *m++ = t[1] + 4 + ARGHEAD;
02197                 *m++ = 1;
02198                 FILLARG(m)
02199                 *m++ = t[1] + 4;
02200                 t[3] = -t[3];
02201                 v = t + t[1];
02202                 while ( t < v ) *m++ = *t++;
02203                 from[3] = -from[3];
02204                 *m++ = 1;
02205                 *m++ = 1;
02206                 *m++ = 3;
02207                 r = term + *term;
02208                 while ( t < r ) *m++ = *t++;
02209                 if ( (m-termout) > (LONG)(AM.MaxTer/sizeof(WORD)) ) {
02210                     MLOCK(ErrorMessageLock);
02211                     MesPrint("Output term too large (%d words) (MaxTermSize: %d words)", m-termout,
02212                             AM.MaxTer/sizeof(WORD));
02213                     MUNLOCK(ErrorMessageLock);
02214                     goto TooLarge;
02215                 }
02216                 *termout = WORDDIF(m,termout);
02217                 return(0);
02218             }
02219         }
02220         else if ( ( *t >= FUNCTION && functions[ipp-FUNCTION].spec <= 0 )
02221                 && ( t[2] & DIRTYFLAG ) == DIRTYFLAG ) {
02222             m = termout;
02223             r = t + t[1];
02224             u = t;
02225             t += FUNHEAD;
02226             oldncmod = AN.ncmod;
02227             while ( t < r ) { /* t points at an argument */
02228                 if ( *t > 0 && t[1] ) { /* Argument has been modified */
02229                     WORD oldsorttype = AR.SortType;
02230                     /* This whole argument must be redone */
02231
02232                     if ( ( AN.ncmod != 0 )
02233                         && ( ( AC.modmode & ALSOFUNARGS ) == 0 )
02234                         && ( *u != AR.PolyFun ) ) { AN.ncmod = 0; }
02235                     AR.DeferFlag = 0;
02236                     v = t + *t;
02237                     t += ARGHEAD; /* First term */
02238                     w = 0; /* to appease the compilers warning devices */
02239                     while ( from < t ) {
02240                         if ( from == u ) w = m;
02241                         *m++ = *from++;
02242                     }
02243                     to = m;
02244                     NewSort(BHEAD0);
02245                     if ( *u == AR.PolyFun && AR.PolyFunType == 2 ) {
02246                         AR.CompareRoutine = (COMPAREDUMMY)(&CompareSymbols);
02247                         AR.SortType = SORTHIGHFIRST;
02248                     }
02249                 }
02250             }
02251             AR.PolyFun = 0;
02252             while ( t < v ) {
02253                 i = *t;
02254                 NCOPY(m,t,i);
02255             }

```

```

02254         m = to;
02255         if ( AT.WorkPointer < m+m ) AT.WorkPointer = m + *m;
02256         if ( Generator(BHEAD m,AR.Cnumlhs) ) {
02257             AN.ncmod = oldncmod;
02258             LowerSortLevel(); goto InFunc;
02259         }
02260     }
02261     /* w = the function */
02262     /* v = the next argument */
02263     /* u = the function */
02264     /* to is new argument */
02265
02266     to -= ARGHEAD;
02267     if ( EndSort(BHEAD m,1) < 0 ) {
02268         AN.ncmod = oldncmod;
02269         goto InFunc;
02270     }
02271     AR.PolyFun = oldPolyFun;
02272     if ( *u == AR.PolyFun && AR.PolyFunType == 2 ) {
02273         AR.CompareRoutine = (COMPAREDUMMY)(&Compare1);
02274         AR.SortType = oldsorttype;
02275     }
02276     while ( *m ) m += *m;
02277     *to = WORDDIF(m,to);
02278     to[1] = 1; /* ?????? or rather 0?. 24-mar-2006 JV */
02279     if ( ToFast(to,to) ) {
02280         if ( *to <= -FUNCTION ) m = to+1;
02281         else m = to+2;
02282     }
02283     w[1] = WORDDIF(m,w) + WORDDIF(r,v);
02284     r = term + *term;
02285     t = v;
02286     while ( t < r ) *m++ = *t++;
02287     if ( (m-termout) > (LONG)(AM.MaxTer/sizeof(WORD)) ) {
02288         MLOCK(ErrorMessageLock);
02289         MesPrint("Output term too large (%d words) (MaxTermSize: %d words)",
m-termout, AM.MaxTer/sizeof(WORD));
02290         MUNLOCK(ErrorMessageLock);
02291         goto TooLarge;
02292     }
02293     *termout = WORDDIF(m,termout);
02294     AR.DeferFlag = olddefer;
02295     AN.ncmod = oldncmod;
02296     return(0);
02297 }
02298 else if ( *t == -DOLLAREXPRESSION ) {
02299     if ( AR.Eside == LHSIDE ) {
02300         NEXTARG(t)
02301         AC.lhdollarflag = 1;
02302     }
02303     else {
02304         /*
02305             This whole argument must be redone
02306         */
02307         DOLLARS d = Dollars + t[1];
02308 #ifdef WITHPTHREADS
02309         int nummodopt, dtype = -1;
02310         if ( AS.MultiThreaded ) {
02311             for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
02312                 if ( t[1] == ModOptdollars[nummodopt].number ) break;
02313             }
02314             if ( nummodopt < NumModOptdollars ) {
02315                 dtype = ModOptdollars[nummodopt].type;
02316                 if ( dtype == MODLOCAL ) {
02317                     d = ModOptdollars[nummodopt].dstruct+AT.identity;
02318                 }
02319                 else {
02320                     LOCK(d->pthreadlockread);
02321                 }
02322             }
02323         }
02324 #endif
02325         oldncmod = AN.ncmod;
02326         if ( ( AN.ncmod != 0 )
02327             && ( ( AC.modmode & ALSOFUNARGS ) == 0 )
02328             && ( *u != AR.PolyFun ) ) { AN.ncmod = 0; }
02329         AR.DeferFlag = 0;
02330         v = t + 2;
02331         w = 0; /* to appease the compilers warning devices */
02332         while ( from < t ) {
02333             if ( from == u ) w = m;
02334             *m++ = *from++;
02335         }
02336         to = m;
02337         switch ( d->type ) {
02338             case DOLINDEX:
02339                 if ( d->index >= 0 && d->index < AM.OffsetIndex ) {

```

```

02340             *m++ = -SNUMBER; *m++ = d->index;
02341         }
02342         else { *m++ = -INDEX; *m++ = d->index; }
02343         break;
02344     case DOLZERO:
02345         *m++ = -SNUMBER; *m++ = 0; break;
02346     case DOLNUMBER:
02347         if ( d->where[0] == 4 &&
02348             ( d->where[1] & MAXPOSITIVE ) == d->where[1] ) {
02349             *m++ = -SNUMBER;
02350             if ( d->where[3] >= 0 ) *m++ = d->where[1];
02351             else *m++ = -d->where[1];
02352             break;
02353         }
02354         /* fall through */
02355     case DOLTERMS:
02356         /*
02357         Here we have the special case of the PolyRatFun
02358         That function may have a different sort of the
02359         terms in the argument.
02360         */
02361         to = m; r = d->where;
02362         *m++ = 0; *m++ = 1;
02363         FILLARG(m)
02364         while ( *r ) {
02365             i = *r; NCOPY(m,r,i)
02366         }
02367         *to = m-to;
02368         if ( ToFast(to,to) ) {
02369             if ( *to <= -FUNCTION ) m = to+1;
02370             else m = to+2;
02371         }
02372         else if ( *u == AR.PolyFun && AR.PolyFunType == 2 ) {
02373             AR.PolyFun = 0;
02374             NewSort(BHEAD0);
02375             AR.CompareRoutine = (COMPAREDUMMY)(&CompareSymbols);
02376             r = to + ARGHEAD;
02377             while ( r < m ) {
02378                 rr = r; r += *r;
02379                 if ( SymbolNormalize(rr) ) goto InFunc;
02380                 if ( StoreTerm(BHEAD rr) ) {
02381                     AR.CompareRoutine = (COMPAREDUMMY)(&Compare1);
02382                     LowerSortLevel();
02383                     Terminate(-1);
02384                 }
02385             }
02386             if ( EndSort(BHEAD to+ARGHEAD,1) < 0 ) goto InFunc;
02387             AR.PolyFun = oldPolyFun;
02388             AR.CompareRoutine = (COMPAREDUMMY)(&Compare1);
02389             m = to+ARGHEAD;
02390             if ( *m == 0 ) {
02391                 *to = -SNUMBER;
02392                 to[1] = 0;
02393                 m = to + 2;
02394             }
02395             else {
02396                 while ( *m ) m += *m;
02397                 *t = m - to;
02398                 if ( ToFast(to,to) ) {
02399                     if ( *to <= -FUNCTION ) m = to+1;
02400                     else m = to+2;
02401                 }
02402             }
02403         }
02404         w[1] = w[1] - 2 + (m-to);
02405         break;
02406     case DOLSUBTERM:
02407         to = m; r = d->where;
02408         i = r[1];
02409         *m++ = i+4+ARGHEAD; *m++ = 1;
02410         FILLARG(m)
02411         *m++ = i+4;
02412         while ( --i >= 0 ) *m++ = *r++;
02413         *m++ = 1; *m++ = 1; *m++ = 3;
02414         if ( ToFast(to,to) ) {
02415             if ( *to <= -FUNCTION ) m = to+1;
02416             else m = to+2;
02417         }
02418         w[1] = w[1] - 2 + (m-to);
02419         break;
02420     case DOLARGUMENT:
02421         to = m; r = d->where;
02422         if ( *r > 0 ) {
02423             i = *r - 2;
02424             *m++ = *r++; *m++ = 1; r++;
02425             while ( --i >= 0 ) *m++ = *r++;
02426         }

```

```

02427         else if ( *r <= -FUNCTION ) *m++ = *r++;
02428     else { *m++ = *r++; *m++ = *r++; }
02429     w[1] = w[1] - 2 + (m-to);
02430     break;
02431 case DOLWILDARGS:
02432     to = m; r = d->where;
02433     if ( *r > 0 ) { /* Tensor arguments */
02434         i = *r++;
02435         while ( --i >= 0 ) {
02436             if ( *r < 0 ) {
02437                 *m++ = -VECTOR; *m++ = *r++;
02438             }
02439             else if ( *r >= AM.OffsetIndex ) {
02440                 *m++ = -INDEX; *m++ = *r++;
02441             }
02442             else { *m++ = -SNUMBER; *m++ = *r++; }
02443         }
02444     }
02445     else { /* Regular arguments */
02446         r++;
02447         while ( *r ) {
02448             if ( *r > 0 ) {
02449                 i = *r - 2;
02450                 *m++ = *r++; *m++ = 1; r++;
02451                 while ( --i >= 0 ) *m++ = *r++;
02452             }
02453             else if ( *r <= -FUNCTION ) *m++ = *r++;
02454             else { *m++ = *r++; *m++ = *r++; }
02455         }
02456     }
02457     w[1] = w[1] - 2 + (m-to);
02458     break;
02459 case DOLUNDEFINED:
02460 default:
02461     MLOCK(ErrorMessageLock);
02462     MesPrint("!!!Undefined $-variable: %s!!!",
02463         AC.dollarnames->namebuffer+d->name);
02464     MUNLOCK(ErrorMessageLock);
02465 #ifdef WITHPTHREADS
02466     if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
02467 #endif
02468     Terminate(-1);
02469 }
02470 #ifdef WITHPTHREADS
02471 if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
02472 #endif
02473 r = term + *term;
02474 t = v;
02475 while ( t < r ) *m++ = *t++;
02476 if ( (m-termout) > (LONG)(AM.MaxTer/sizeof(WORD)) ) {
02477     MLOCK(ErrorMessageLock);
02478     MesPrint("Output term too large (%d words) (MaxTermSize: %d words)",
02479 m-termout, AM.MaxTer/sizeof(WORD));
02479     MUNLOCK(ErrorMessageLock);
02480     goto TooLarge;
02481 }
02482 *termout = WORDDIFF(m,termout);
02483 AR.DeferFlag = olddefer;
02484 AN.ncmod = oldncmod;
02485 return(0);
02486 }
02487 }
02488 else if ( *t == -TERMSINBRACKET ) {
02489     if ( AC.ComDefer ) numterms = CountTerms1(BHEAD0);
02490     else numterms = 1;
02491 /*
02492     Compose the output term
02493     First copy the part till this function argument
02494     m points at the output term space
02495     u points at the start of the function
02496     t points at the start of the argument
02497 */
02498     w = 0;
02499     while ( from < t ) {
02500         if ( from == u ) w = m;
02501         *m++ = *from++;
02502     }
02503     if ( ( numterms & MAXPOSITIVE ) == numterms ) {
02504         *m++ = -SNUMBER; *m++ = numterms & MAXPOSITIVE;
02505         w[1] += 1;
02506     }
02507     else if ( ( i = numterms >> BITSINWORD ) == 0 ) {
02508         *m++ = ARGHEAD+4;
02509         for ( j = 1; j < ARGHEAD; j++ ) *m++ = 0;
02510         *m++ = 4; *m++ = numterms & WORDMASK; *m++ = 1; *m++ = 3;
02511         w[1] += ARGHEAD+3;
02512     }

```

```

02513         else {
02514             *m++ = ARGHEAD+6;
02515             for ( j = 1; j < ARGHEAD; j++ ) *m++ = 0;
02516             *m++ = 6; *m++ = numterms & WORDMASK;
02517             *m++ = i; *m++ = 1; *m++ = 0; *m++ = 5;
02518             w[1] += ARGHEAD+5;
02519         }
02520         from++; /* Skip our function */
02521         r = term + *term;
02522         while ( from < r ) *m++ = *from++;
02523         if ( (m-termout) > (LONG)(AM.MaxTer/sizeof(WORD)) ) {
02524             MLOCK(ErrorMessageLock);
02525             MesPrint("Output term too large (%d words) (MaxTermSize: %d words)",
m-termout, AM.MaxTer/sizeof(WORD));
02526             MUNLOCK(ErrorMessageLock);
02527             goto TooLarge;
02528         }
02529         *termout = WORDDIFF(m,termout);
02530         return(0);
02531     }
02532     else { NEXTARG(t) }
02533 }
02534 t = u;
02535 }
02536 else if ( ( *t >= FUNCTION && functions[ipp-FUNCTION].spec > 0 )
&& ( t[2] & DIRTYFLAG ) == DIRTYFLAG ) { /* Could be FUNNYDOLLAR */
02537     u = t; v = t + t[1];
02538     t += FUNHEAD;
02539     while ( t < v ) {
02540         if ( *t == FUNNYDOLLAR ) {
02541             if ( AR.Eside != LHSIDE ) {
02542                 DOLLARS d = Dollars + t[1];
02543 #ifndef WITHPTHREADS
02544                 int nummodopt, dtype = -1;
02545                 if ( AS.MultiThreaded ) {
02546                     for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
02547                         if ( t[1] == ModOptdollars[nummodopt].number ) break;
02548                     }
02549                     if ( nummodopt < NumModOptdollars ) {
02550                         dtype = ModOptdollars[nummodopt].type;
02551                         if ( dtype == MODLOCAL ) {
02552                             d = ModOptdollars[nummodopt].dstruct+AT.identity;
02553                         }
02554                         else {
02555                             LOCK(d->pthreadlockread);
02556                         }
02557                     }
02558                 }
02559             }
02560 #endif
02561             oldncmod = AN.ncmod;
02562             if ( ( AN.ncmod != 0 )
&& ( ( AC.modmode & ALSOFUNARGS ) == 0 )
&& ( *u != AR.PolyFun ) ) { AN.ncmod = 0; }
02563             m = termout; w = 0;
02564             while ( from < t ) {
02565                 if ( from == u ) w = m;
02566                 *m++ = *from++;
02567             }
02568             to = m;
02569             switch ( d->type ) {
02570                 case DOLINDEX:
02571                     *m++ = d->index; break;
02572                 case DOLZERO:
02573                     *m++ = 0; break;
02574                 case DOLNUMBER:
02575                 case DOLTERMS:
02576                     if ( d->where[0] == 4 && d->where[4] == 0
&& d->where[3] == 3 && d->where[2] == 1
&& d->where[1] < AM.OffsetIndex ) {
02577                         *m++ = d->where[1];
02578                     }
02579                     else {
02580                         wrongtype;;
02581 #ifndef WITHPTHREADS
02582                         if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
02583                         MLOCK(ErrorMessageLock);
02584                         MesPrint("%s has wrong type for tensor substitution",
AC.dollarnames->namebuffer+d->name);
02585                         MUNLOCK(ErrorMessageLock);
02586                         AN.ncmod = oldncmod;
02587                         return(-1);
02588                     }
02589                     break;
02590                 case DOLARGUMENT:
02591                     if ( d->where[0] == -INDEX ) {
02592                         *m++ = d->where[1]; break;
02593                     }

```

```

02599         }
02600         else if ( d->where[0] == -VECTOR ) {
02601             *m++ = d->where[1]; break;
02602         }
02603         else if ( d->where[0] == -MINVECTOR ) {
02604             *m++ = d->where[1];
02605             sign = -sign;
02606             break;
02607         }
02608         else if ( d->where[0] == -SNUMBER ) {
02609             if ( d->where[1] >= 0
02610                 && d->where[1] < AM.OffsetIndex ) {
02611                 *m++ = d->where[1]; break;
02612             }
02613         }
02614         goto wrongtype;
02615     case DOLWILDARGS:
02616         if ( d->where[0] > 0 ) {
02617             r = d->where; i = *r++;
02618             while ( --i >= 0 ) *m++ = *r++;
02619         }
02620         else {
02621             r = d->where + 1;
02622             while ( *r ) {
02623                 if ( *r == -INDEX ) {
02624                     *m++ = r[1]; r += 2; continue;
02625                 }
02626                 else if ( *r == -VECTOR ) {
02627                     *m++ = r[1]; r += 2; continue;
02628                 }
02629                 else if ( *r == -MINVECTOR ) {
02630                     *m++ = r[1]; r += 2;
02631                     sign = -sign; continue;
02632                 }
02633                 else if ( *r == -SNUMBER ) {
02634                     if ( r[1] >= 0
02635                         && r[1] < AM.OffsetIndex ) {
02636                         *m++ = r[1]; r += 2; continue;
02637                     }
02638                 }
02639                 goto wrongtype;
02640             }
02641         }
02642         break;
02643     case DOLSUBTERM:
02644         r = d->where;
02645         if ( *r == INDEX && r[1] == 3 ) {
02646             *m++ = r[2];
02647         }
02648         else goto wrongtype;
02649         break;
02650     case DOLUNDEFINED:
02651 #ifdef WITHPTHREADS
02652         if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
02653 #endif
02654         MLOCK(ErrorMessageLock);
02655         MesPrint("$%s is undefined in tensor substitution",
02656             AC.dollarnames->namebuffer+d->name);
02657         MUNLOCK(ErrorMessageLock);
02658         AN.ncmod = oldncmod;
02659         return(-1);
02660     }
02661 #ifdef WITHPTHREADS
02662     if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
02663 #endif
02664     w[1] = w[1] - 2 + (m-to);
02665     from += 2;
02666     term += *term;
02667     while ( from < term ) *m++ = *from++;
02668     if ( sign < 0 ) m[-1] = -m[-1];
02669     if ( (m-termout) > (LONG)(AM.MaxTer/sizeof(WORD)) ) {
02670         MLOCK(ErrorMessageLock);
02671         MesPrint("Output term too large (%d words) (MaxTermSize: %d words)",
02672             m-termout, AM.MaxTer/sizeof(WORD));
02672         MUNLOCK(ErrorMessageLock);
02673         goto TooLarge;
02674     }
02675     *termout = m - termout;
02676     AN.ncmod = oldncmod;
02677     return(0);
02678 }
02679 else {
02680     AC.lhdollarflag = 1;
02681 }
02682 }
02683 t++;
02684 }

```

```

02685         t = u;
02686     }
02687     t += t[1];
02688 }
02689 MLOCK(ErrorMessageLock);
02690 MesPrint("Internal error in InFunction: Function not encountered.");
02691 if ( AM.tracebackflag ) {
02692     MesPrint("%w: AR.TePos = %d",AR.TePos);
02693     MesPrint("%w: AN.TeInFun = %d",AN.TeInFun);
02694     termout = term;
02695     AO.OutFill = AO.OutputLine = (UBYTE *)AT.WorkPointer + AM.MaxTer;
02696     AO.OutSkip = 3;
02697     FiniLine();
02698     i = *termout;
02699     while ( --i >= 0 ) {
02700         TalToLine((UWORD) (*termout++));
02701         TokenToLine((UBYTE *) " ");
02702     }
02703     AO.OutSkip = 0;
02704     FiniLine();
02705     MesCall("InFunction");
02706 }
02707 MUNLOCK(ErrorMessageLock);
02708 return(1);
02709
02710 InFunc:
02711 MLOCK(ErrorMessageLock);
02712 MesCall("InFunction");
02713 MUNLOCK(ErrorMessageLock);
02714 SETERROR(-1)
02715
02716 TooLarge:
02717 MLOCK(ErrorMessageLock);
02718 MesCall("InFunction");
02719 MUNLOCK(ErrorMessageLock);
02720 SETERROR(-1)
02721 }
02722
02723 /*
02724     #] InFunction :
02725     #[ InsertTerm :      WORD InsertTerm(term,replac,extractbuff,position,termout)
02726 */
02744 WORD InsertTerm(PHEAD WORD *term, WORD replac, WORD extractbuff, WORD *position, WORD *termout,
02745                 WORD tepos)
02746 {
02747     GETBIDENTITY
02748     WORD *m, *t, *r, i, l2, j;
02749     WORD *u, *v, ll, *coef;
02750     coef = AT.WorkPointer;
02751     if ( ( AT.WorkPointer = coef + 2*AM.MaxTal ) > AT.WorkTop ) {
02752         MLOCK(ErrorMessageLock);
02753         MesWork();
02754         MUNLOCK(ErrorMessageLock);
02755         return(-1);
02756     }
02757     t = term;
02758     r = t + *t;
02759     ll = l2 = r[-1];
02760     m = r - ABS(l2);
02761     if ( tepos > 0 ) {
02762         t = term + tepos;
02763         goto foundit;
02764     }
02765     t++;
02766     while ( t < m ) {
02767         if ( *t == SUBEXPRESSION && t[2] == replac && t[3] && t[4] == extractbuff ) {
02768             r = t + t[1];
02769             while ( *r == SUBEXPRESSION && r[2] == replac && r[3] && r < m && r[4] == extractbuff ) {
02770                 t = r; r += r[1];
02771             }
02772 foundit:;
02773             u = m;
02774             r = term;
02775             m = termout;
02776             do { *m++ = *r++; } while ( r < t );
02777             if ( t[1] > SUBEXPSIZE ) {
02778 /*
02779                 if this is a dollar expression there are no wildcards
02780 */
02781                 i = *--m;
02782                 if ( ( l2 = WildFill(BHEAD m,position,t) ) < 0 ) goto InsCall;
02783                 *m = i;
02784                 m += l2-1;
02785                 l2 = *m;
02786                 i = ( j = ABS(l2) ) - 1;
02787                 r = coef + i;
02788                 do { *--r = *--m; } while ( --i > 0 );

```



```

02789     }
02790     else {
02791         v = t;
02792         t = position;
02793         r = t + *t;
02794         l2 = r[-1];
02795         r -= ( j = ABS(l2) );
02796         t++;
02797         if ( t < r ) do { *m++ = *t++; } while ( t < r );
02798         t = v;
02799     }
02800     t += t[1];
02801     while ( t < u && *t == DOLLAREXP2 ) t += t[1];
02802 ComAct:   if ( t < u ) do { *m++ = *t++; } while ( t < u );
02803     if ( *r == 1 && r[1] == 1 && j == 3 ) {
02804         if ( l2 < 0 ) l1 = -l1;
02805         i = ABS(l1)-1;
02806         NCOPY(m,t,i);
02807         *m++ = l1;
02808     }
02809     else {
02810         if ( MulRat(BHEAD (UWORD *)u,REDLENG(l1), (UWORD *)r,REDLENG(l2),
02811             (UWORD *)m,&l1) ) goto InsCall;
02812         l2 = l1;
02813         l2 *= 2;
02814         if ( l2 < 0 ) {
02815             m -= l2;
02816             *m++ = l2-1;
02817         }
02818         else {
02819             m += l2;
02820             *m++ = l2+1;
02821         }
02822     }
02823     *termout = WORDDIF(m,termout);
02824     if ( (*termout)*((LONG)sizeof(WORD)) > AM.MaxTer ) {
02825         MLOCK(ErrorMessageLock);
02826         MesPrint("Term too complex during substitution (%d words). MaxTermSize (%l words) is
too small.", *termout, AM.MaxTer/(LONG)sizeof(WORD) );
02827         goto InsCall2;
02828     }
02829     AT.WorkPointer = coef;
02830     return(0);
02831 }
02832     t += t[1];
02833 }
02834 /*
02835 The next action is for when there is no subexpression pointer.
02836 We append the extra term. Effectively the routine becomes now a
02837 merge routine for two terms.
02838 */
02839 v = t;
02840 u = m;
02841 r = term;
02842 m = termout;
02843 do { *m++ = *r++; } while ( r < t );
02844 t = position;
02845 r = t + *t;
02846 l2 = r[-1];
02847 r -= ( j = ABS(l2) );
02848 t++;
02849 if ( t < r ) do { *m++ = *t++; } while ( t < r );
02850 t = v;
02851 goto ComAct;
02852
02853 InsCall:
02854     MLOCK(ErrorMessageLock);
02855 InsCall2:
02856     MesCall("InsertTerm");
02857     MUNLOCK(ErrorMessageLock);
02858     SETERROR(-1)
02859 }
02860
02861 /*
02862 #] InsertTerm :
02863 #[ PasteFile :          WORD PasteFile(num,acc,pos,accf,renum,freeze,nexpr)
02864 */
02880 LONG PasteFile(PHEAD WORD number, WORD *accum, POSITION *position, WORD **accfill,
02881     RENUMBER renumber, WORD *freeze, WORD nexpr)
02882 {
02883     GETBIDENTITY
02884     WORD *r, l, *m, i;
02885     WORD *stop, *s1, *s2;
02886 /* POSITION AccPos; bug 12-apr-2008 JV */
02887     WORD InCompState;
02888     WORD *oldipointer;
02889     LONG retlength;

```

```

02890     stop = (WORD *) ((UBYTE *) (accum)) + 2*AM.MaxTer);
02891     *accum++ = number;
02892     while ( --number >= 0 ) accum += *accum;
02893     if ( freeze ) {
02894 /*       AccPos = *position; bug 12-apr-2008 JV */
02895       oldipointer = AR.CompressPointer;
02896       do {
02897         AR.CompressPointer = oldipointer;
02898 /*       if ( ( l = GetFromStore(accum,&AccPos,renumber,&InCompState,nexpr) ) < 0 ) bug 12-apr-2008
JV */
02899         if ( ( l = GetFromStore(accum,position,renumber,&InCompState,nexpr) ) < 0 )
02900           goto PasErr;
02901         if ( !l ) { *accum = 0; return(0); }
02902         r = accum;
02903         m = r + *r;
02904         m -= ABS(m[-1]);
02905         r++;
02906         while ( r < m && *r != HAAKJE ) r += r[1];
02907         if ( r >= m ) {
02908           if ( *freeze != 4 ) l = -1;
02909         }
02910         else {
02911 /*
02912           The algorithm for accepting terms with a given (freeze)
02913           representation outside brackets is rather crude. A refinement
02914           would be to store the part outside the bracket and skip the
02915           term when this part doesn't alter (and is unacceptable).
02916           Once accepting one can keep accepting till the bracket alters
02917           and then one may stop the generation. It is necessary to
02918           set up a struct to remember the bracket and the progress
02919           status.
02920 */
02921           m = AT.WorkPointer;
02922           s2 = r;
02923           r = accum;
02924           *m++ = WORDDIF(s2,r) + 3;
02925           r++;
02926           while ( r < s2 ) *m++ = *r++;
02927           *m++ = 1; *m++ = 1; *m++ = 3;
02928           m = AT.WorkPointer;
02929           if ( Normalize(BHEAD AT.WorkPointer) ) goto PasErr;
02930           r = freeze;
02931           i = *m;
02932           while ( --i >= 0 && *m++ == *r++ ) {}
02933           if ( i > 0 ) {
02934             l = -1;
02935           }
02936           else { /* Term to be accepted */
02937             r = accum;
02938             s1 = r + *r;
02939             r++;
02940             m = s2;
02941             m += m[1];
02942             do { *r++ = *m++; } while ( m < s1 );
02943             *accum = 1 = WORDDIF(r,accum);
02944           }
02945         }
02946       } while ( l < 0 );
02947       retlength = InCompState;
02948 /*     retlength = DIFBASE(AccPos,*position) / sizeof(WORD); bug 12-apr-2008 JV */
02949     }
02950     else {
02951       if ( ( l = GetFromStore(accum,position,renumber,&InCompState,nexpr) ) < 0 ) {
02952         MLOCK(ErrorMessageLock);
02953         MesCall("PasteFile");
02954         MUNLOCK(ErrorMessageLock);
02955         SETERROR(-1)
02956       }
02957       if ( l == 0 ) { *accum = 0; return(0); }
02958       retlength = InCompState;
02959     }
02960     accum += 1;
02961     if ( accum > stop ) {
02962       MLOCK(ErrorMessageLock);
02963       MesPrint("Buffer too small in PasteFile");
02964       MUNLOCK(ErrorMessageLock);
02965       SETERROR(-1)
02966     }
02967     *accum = 0;
02968     *accfill = accum;
02969     return(retlength);
02970 PasErr:
02971     MLOCK(ErrorMessageLock);
02972     MesCall("PasteFile");
02973     MUNLOCK(ErrorMessageLock);
02974     SETERROR(-1)
02975 }

```

```

02976
02977 /*
02978     #] PasteFile :
02979     #[ PasteTerm :          WORD PasteTerm(number,accum,position,times,divby)
02980 */
03002 WORD *PasteTerm(PHEAD WORD number, WORD *accum, WORD *position, WORD times, WORD divby)
03003 {
03004     GETBIDENTITY
03005     WORD *t, *r, x, y, z;
03006     WORD *m, *u, ll, a[2];
03007     m = (WORD *)(((UBYTE *) (accum)) + AM.MaxTer);
03008 /*     m = (WORD *)(((UBYTE *) (accum)) + 2*AM.MaxTer); */
03009     *accum++ = number;
03010     while ( --number >= 0 ) accum += *accum;
03011     if ( times == divby ) {
03012         t = position;
03013         r = t + *t;
03014         if ( t < r ) do { *accum++ = *t++; } while ( t < r );
03015     }
03016     else {
03017         u = accum;
03018         t = position;
03019         r = t + *t - 1;
03020         ll = *r;
03021         r -= ABS(*r) - 1;
03022         if ( t < r ) do { *accum++ = *t++; } while ( t < r );
03023         if ( divby > times ) { x = divby; y = times; }
03024         else { x = times; y = divby; }
03025         z = x*y;
03026         while ( z ) { x = y; y = z; z = x*y; }
03027         if ( y != 1 ) { divby /= y; times /= y; }
03028         a[1] = divby;
03029         a[0] = times;
03030         if ( MulRat(BHEAD (UWORD *)t,REDLENG(ll),(UWORD *)a,1,(UWORD *)accum,&ll) ) {
03031             MLOCK(ErrorMessageLock);
03032             MesCall("PasteTerm");
03033             MUNLOCK(ErrorMessageLock);
03034             return(0);
03035         }
03036         x = ll;
03037         x *= 2;
03038         if ( x < 0 ) { accum -= x; *accum++ = x - 1; }
03039         else { accum += x; *accum++ = x + 1; }
03040         *u = WORDDIF(accum,u);
03041     }
03042     if ( accum >= m ) {
03043         MLOCK(ErrorMessageLock);
03044         MesPrint("Buffer too small in PasteTerm");
03045         MUNLOCK(ErrorMessageLock);
03046         return(0);
03047     }
03048     *accum = 0;
03049     return(accum);
03050 }
03051
03052 /*
03053     #] PasteTerm :
03054     #[ FiniTerm :          WORD FiniTerm(term,accum,termout,number)
03055 */
03067 WORD FiniTerm(PHEAD WORD *term, WORD *accum, WORD *termout, WORD number, WORD tepos)
03068 {
03069     GETBIDENTITY
03070     WORD *m, *t, *r, i, numacc, l2, ipp;
03071     WORD *u, *v, ll, *coef = AT.WorkPointer, *oldaccum;
03072     if ( ( AT.WorkPointer = coef + 2*AM.MaxTal ) > AT.WorkTop ) {
03073         MLOCK(ErrorMessageLock);
03074         MesWork();
03075         MUNLOCK(ErrorMessageLock);
03076         return(-1);
03077     }
03078     oldaccum = accum;
03079     t = term;
03080     m = t + *t - 1;
03081     ll = REDLENG(*m);
03082     i = ABS(*m) - 1;
03083     r = coef + i;
03084     do { *--r = *--m; } while ( --i > 0 ); /* Copies coefficient */
03085     if ( tepos > 0 ) {
03086         t = term + tepos;
03087         goto foundit;
03088     }
03089     t++;
03090     if ( t < m ) do {
03091         if ( ( (*t == SUBEXPRESSION && (*r=t+t[1]) != SUBEXPRESSION
03092             || r >= m || !r[3] ) ) || *t == EXPRESSION ) && t[2] == number && t[3] ) {
03093 foundit:;
03094             u = m;

```

```

03095         r = term;
03096         m = termout;
03097         if ( r < t ) do { *m++ = *r++; } while ( r < t );
03098         numacc = *accum++;
03099         if ( numacc >= 0 ) do {
03100             if ( *t == EXPRESSION ) {
03101                 v = t + t[1];
03102                 r = t + SUBEXPSIZE;
03103                 while ( r < v ) {
03104                     if ( *r == WILDCARDS ) {
03105                         r += 2;
03106                         i = *--m;
03107                         if ( ( l2 = WildFill(BHEAD m,accum,r) ) < 0 ) goto FiniCall;
03108                         goto AllWild;
03109                     }
03110                     r += r[1];
03111                 }
03112                 goto NoWild;
03113             }
03114             else if ( t[1] > SUBEXPSIZE && t[SUBEXPSIZE] != FROMBRAC ) {
03115                 i = *--m;
03116                 if ( ( l2 = WildFill(BHEAD m,accum,t) ) < 0 ) goto FiniCall;
03117 AllWild:
03118                 *m = i;
03119                 m += l2-1;
03120                 l2 = *m;
03121                 m -= ABS(l2) - 1;
03122                 r = m;
03123             }
03124 NoWild:
03125             r = accum;
03126             v = r + *r - 1;
03127             l2 = *v;
03128             v -= ABS(l2) - 1;
03129             r++;
03130             if ( r < v ) do { *m++ = *r++; } while ( r < v );
03131         }
03132         if ( *r == 1 && r[1] == 1 && ABS(l2) == 3 ) {
03133             if ( l2 < 0 ) l1 = -l1;
03134         }
03135         else {
03136             l2 = REDLENG(l2);
03137             if ( l2 == 0 ) {
03138                 t = oldaccum;
03139                 numacc = *t++;
03140                 AO.OutSkip = 3;
03141                 FiniLine();
03142                 while ( --numacc >= 0 ) {
03143                     i = *t;
03144                     while ( --i >= 0 ) {
03145                         TalToLine((UWORD)(*t++));
03146                         TokenToLine((UBYTE *)" ");
03147                     }
03148                 }
03149                 AO.OutSkip = 0;
03150                 FiniLine();
03151                 goto FiniCall;
03152             }
03153             if ( MulRat(BHEAD (UWORD *)coef,l1,(UWORD *)r,l2,(UWORD *)coef,&l1) ) goto
FiniCall;
03154             if ( AN.ncmod != 0 && TakeModulus((UWORD
*)coef,&l1,AC.cmod,AN.ncmod,UNPACK|AC.modmode) ) goto FiniCall;
03155             accum += *accum;
03156             } while ( --numacc >= 0 );
03157             if ( *t == SUBEXPRESSION ) {
03158                 while ( t+t[1] < u && t[t[1]] == DOLLAREXP2 ) t += t[1];
03159             }
03160             t += t[1];
03161             if ( t < u ) do { *m++ = *t++; } while ( t < u );
03162             l2 = l1;
03163 /*
03164 Code to economize when taking x = (a+b)/2
03165 */
03166         r = termout+1;
03167         while ( r < m ) {
03168             if ( *r == SUBEXPRESSION ) {
03169                 t = r + r[1];
03170                 l1 = (WORD)(cbuf[r[4]].CanCommu[r[2]]);
03171                 while ( t < m ) {
03172                     if ( *t == SUBEXPRESSION &&
03173                         t[1] == r[1] && t[2] == r[2] && t[4] == r[4] ) {
03174                         i = t[1] - SUBEXPSIZE;
03175                         u = r + SUBEXPSIZE; v = t + SUBEXPSIZE;
03176                         while ( i > 0 ) {
03177                             if ( *v++ != *u++ ) break;
03178                             i--;
03179                         }

```

```

03180             if ( i <= 0 ) {
03181                 u = r;
03182                 r[3] += t[3];
03183                 r = t + t[1];
03184                 while ( r < m ) *t++ = *r++;
03185                 m = t;
03186                 r = u;
03187                 goto Nextr;
03188             }
03189             if ( l1 && cbuf[t[4]].CanCommu[t[2]] ) break;
03190             while ( t+t[1] < m && t[t[1]] == DOLLAREXP2 ) t += t[1];
03191         }
03192         else if ( l1 ) {
03193             if ( *t == SUBEXPRESSION && cbuf[t[4]].CanCommu[t[2]] )
03194                 break;
03195             if ( *t >= FUNCTION+WILDOFFSET )
03196                 ipp = *t - WILDOFFSET;
03197             else ipp = *t;
03198             if ( *t >= FUNCTION
03199                 && functions[ipp-FUNCTION].commute && l1 ) break;
03200             if ( *t == EXPRESSION ) break;
03201         }
03202         t += t[1];
03203     }
03204     r += r[1];
03205 }
03206 else r += r[1];
03207 Nextr;;
03208 }
03209
03210 i = ABS(12);
03211 i *= 2;
03212 i++;
03213 l2 = ( l2 >= 0 ) ? i: -i;
03214 r = coef;
03215 while ( --i > 0 ) *m++ = *r++;
03216 *m++ = l2;
03217 *termout = WORDDIF(m,termout);
03218 AT.WorkPointer = coef;
03219 return(0);
03220 }
03221 t += t[1];
03222 } while ( t < m );
03223 AT.WorkPointer = coef;
03224 return(1);
03225
03226 FiniCall:
03227 MLOCK(ErrorMessageLock);
03228 MesCall("FiniTerm");
03229 MUNLOCK(ErrorMessageLock);
03230 SETERROR(-1)
03231 }
03232
03233 /*
03234 #] FiniTerm :
03235 #[ Generator :      WORD Generator(BHEAD term,level)
03236 */
03237
03238 static WORD zeroDollar[] = { 0, 0 };
03239 /*
03240 static LONG debugcounter = 0;
03241 */
03242
03266 WORD Generator(PHEAD WORD *term, WORD level)
03267 {
03268     GETBIDENTITY
03269     WORD replac, *accum, *termout, *t, i, j, tepos, applyflag = 0, *StartBuf;
03270     WORD *a, power, powerl, DumNow = AR.CurDum, oldtoprhs, oldatoprhs, retnorm, extractbuff;
03271     int *RepSto = AN.RepPoint, iscopy = 0;
03272     CBUF *C = cbuf+AM.rbufnum, *CC = cbuf + AT.ebufnum, *CCC = cbuf + AT.aebufnum;
03273     LONG posisub, oldcpointer, oldacpointer;
03274     DOLLARS d = 0;
03275     WORD numfac[5], idfunctionflag;
03276 #ifdef WITHPTHREADS
03277     int nummodopt, dtype = -1, id;
03278 #endif
03279     oldtoprhs = CC->numrhs;
03280     oldcpointer = CC->Pointer - CC->Buffer;
03281     oldatoprhs = CCC->numrhs;
03282     oldacpointer = CCC->Pointer - CCC->Buffer;
03283 ReStart:
03284     if ( ( replac = TestSub(BHEAD term,level) ) == 0 ) {
03285         if ( applyflag ) { TableReset(); applyflag = 0; }
03286     /*
03287         if ( AN.PolyNormFlag > 1 ) {
03288             if ( PolyFunMul(BHEAD term) < 0 ) goto GenCall;
03289             AN.PolyNormFlag = 0;

```

```

03290         if ( !*term ) goto Return0;
03291     }
03292 */
03293 Renormalize:
03294     AN.PolyNormFlag = 0;
03295     AN.idfunctionflag = 0;
03296     if ( ( retnorm = Normalize(BHEAD term) ) != 0 ) {
03297         if ( retnorm > 0 ) {
03298             if ( AT.WorkPointer < term + *term ) AT.WorkPointer = term + *term;
03299             goto ReStart;
03300         }
03301         goto GenCall;
03302     }
03303     idfunctionflag = AN.idfunctionflag;
03304     if ( !*term ) { AN.PolyNormFlag = 0; goto Return0; }
03305
03306     if ( AN.PolyNormFlag ) {
03307         if ( AN.PolyFunTodo == 0 ) {
03308             if ( PolyFunMul(BHEAD term) < 0 ) goto GenCall;
03309             if ( !*term ) { AN.PolyNormFlag = 0; goto Return0; }
03310         }
03311         else {
03312             WORD oldPolyFunExp = AR.PolyFunExp;
03313             AR.PolyFunExp = 0;
03314             if ( PolyFunMul(BHEAD term) < 0 ) goto GenCall;
03315             AT.WorkPointer = term+*term;
03316             AR.PolyFunExp = oldPolyFunExp;
03317             if ( !*term ) { AN.PolyNormFlag = 0; goto Return0; }
03318             if ( Normalize(BHEAD term) < 0 ) goto GenCall;
03319             if ( !*term ) { AN.PolyNormFlag = 0; goto Return0; }
03320             AT.WorkPointer = term+*term;
03321             if ( AN.PolyNormFlag ) {
03322                 if ( PolyFunMul(BHEAD term) < 0 ) goto GenCall;
03323                 if ( !*term ) { AN.PolyNormFlag = 0; goto Return0; }
03324                 AT.WorkPointer = term+*term;
03325             }
03326             AN.PolyFunTodo = 0;
03327         }
03328     }
03329     if ( idfunctionflag > 0 ) {
03330         if ( TakeIDfunction(BHEAD term) ) {
03331             AT.WorkPointer = term + *term;
03332             goto ReStart;
03333         }
03334     }
03335     if ( AT.WorkPointer < (WORD *)(((UBYTE *) (term)) + AM.MaxTer) )
03336         AT.WorkPointer = (WORD *)(((UBYTE *) (term)) + AM.MaxTer);
03337     do {
03338 SkipCount: level++;
03339         if ( level > AR.Cnumlhs ) {
03340             if ( AR.DeferFlag && AR.sLevel <= 0 ) {
03341 #ifdef WITHMPI
03342                 if ( PF.me != MASTER && AC.mparallelflag == PARALLELFLAG && PF.exprtoto < 0 ) {
03343                     if ( PF_Deferred(term,level) ) goto GenCall;
03344                 }
03345                 else
03346 #endif
03347                 {
03348                     if ( Deferred(BHEAD term,level) ) goto GenCall;
03349                 }
03350                 goto Return0;
03351             }
03352             if ( AN.ncmod != 0 ) {
03353                 if ( Modulus(term) ) goto GenCall;
03354                 if ( !*term ) goto Return0;
03355             }
03356             if ( AR.CurDum > AM.IndDum && AR.sLevel <= 0 ) {
03357                 WORD olddummies = AN.IndDum;
03358                 AN.IndDum = AM.IndDum;
03359                 ReNumber(BHEAD term);
03360                 Normalize(BHEAD term);
03361                 AN.IndDum = olddummies;
03362                 if ( !*term ) goto Return0;
03363                 olddummies = DetCurDum(BHEAD term);
03364                 if ( olddummies > AR.MaxDum ) AR.MaxDum = olddummies;
03365             }
03366             if ( AR.PolyFun > 0 && ( AR.sLevel <= 0 || AN.FunSorts[AR.sLevel]->PolyFlag > 0 ) ) {
03367                 if ( PrepPoly(BHEAD term,0) != 0 ) goto Return0;
03368             }
03369             else if ( AR.PolyFun > 0 ) {
03370                 if ( PrepPoly(BHEAD term,1) != 0 ) goto Return0;
03371             }
03372             if ( AR.sLevel <= 0 && AR.BracketOn ) {
03373                 if ( AT.WorkPointer < term + *term ) AT.WorkPointer = term + *term;
03374                 termout = AT.WorkPointer;
03375                 if ( AT.WorkPointer + *term + 3 > AT.WorkTop ) goto OverWork;
03376                 if ( PutBracket(BHEAD term) ) return(-1);

```

```

03377         AN.RepPoint = RepSto;
03378         *AT.WorkPointer = 0;
03379         i = StoreTerm(BHEAD termout);
03380         AT.WorkPointer = termout;
03381         CC->numrhs = oldtoprhs;
03382         CC->Pointer = CC->Buffer + oldcpointer;
03383         CCC->numrhs = oldatoprhs;
03384         CCC->Pointer = CCC->Buffer + oldacpointer;
03385         return(i);
03386     }
03387     else {
03388         if ( AT.WorkPointer < term + *term ) AT.WorkPointer = term + *term;
03389         if ( AT.WorkPointer >= AT.WorkTop ) goto OverWork;
03390         *AT.WorkPointer = 0;
03391         AN.RepPoint = RepSto;
03392         i = StoreTerm(BHEAD term);
03393         CC->numrhs = oldtoprhs;
03394         CC->Pointer = CC->Buffer + oldcpointer;
03395         CCC->numrhs = oldatoprhs;
03396         CCC->Pointer = CCC->Buffer + oldacpointer;
03397         return(i);
03398     }
03399 }
03400 i = C->lhs[level][0];
03401 if ( i >= TYPECOUNT ) {
03402     /*
03403     #[ Special action :
03404     */
03405     switch ( i ) {
03406     case TYPECOUNT:
03407         if ( CountDo(term,C->lhs[level]) < C->lhs[level][2] ) {
03408             AT.WorkPointer = term + *term;
03409             goto Return0;
03410         }
03411         break;
03412     case TYPEMULT:
03413         if ( MultDo(BHEAD term,C->lhs[level]) ) goto GenCall;
03414         goto ReStart;
03415     case TYPEGOTO:
03416         level = AC.Labels[C->lhs[level][2]];
03417         break;
03418     case TYPEDISCARD:
03419         AT.WorkPointer = term + *term;
03420         goto Return0;
03421     case TYPEIF:
03422 #ifdef WITHPTHREADS
03423     {
03424         /*
03425         We may be writing in the space here when wildcards
03426         are involved in a match(). Hence we have to make
03427         a private copy here!!!!
03428         */
03429         WORD ic, jc, *ifcode, *jfcode;
03430         jfcode = C->lhs[level]; jc = jfcode[1];
03431         ifcode = AT.WorkPointer; AT.WorkPointer += jc;
03432         for ( ic = 0; ic < jc; ic++ ) ifcode[ic] = jfcode[ic];
03433         while ( !DoIfStatement(BHEAD ifcode,term) ) {
03434             level = C->lhs[level][2];
03435             if ( C->lhs[level][0] != TYPEELIF ) break;
03436         }
03437         AT.WorkPointer = ifcode;
03438     }
03439 #else
03440     while ( !DoIfStatement(BHEAD C->lhs[level],term) ) {
03441         level = C->lhs[level][2];
03442         if ( C->lhs[level][0] != TYPEELIF ) break;
03443     }
03444 #endif
03445     break;
03446     case TYPEELIF:
03447     do {
03448         level = C->lhs[level][2];
03449     } while ( C->lhs[level][0] == TYPEELIF );
03450     break;
03451     case TYPEELSE:
03452     case TYPEENDIF:
03453         level = C->lhs[level][2];
03454         break;
03455     case TYPESUMFIX:
03456     {
03457         WORD *cp = AR.CompressPointer, *op = AR.CompressPointer;
03458         WORD *tlhs = C->lhs[level] + 3, *m, *lhs;
03459         WORD theindex = C->lhs[level][2];
03460         if ( theindex < 0 ) { /* $-variable */
03461 #ifdef WITHPTHREADS
03462             int ddtype = -1;
03463             theindex = -theindex;

```

```

03464         d = Dollars + theindex;
03465         if ( AS.MultiThreaded ) {
03466             for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
03467                 if ( theindex == ModOptdollars[nummodopt].number ) break;
03468             }
03469             if ( nummodopt < NumModOptdollars ) {
03470                 ddtype = ModOptdollars[nummodopt].type;
03471                 if ( ddtype == MODLOCAL ) {
03472                     d = ModOptdollars[nummodopt].dstruct+AT.identity;
03473                 }
03474                 else {
03475                     LOCK(d->pthreadlockread);
03476                 }
03477             }
03478         }
03479     #else
03480         theindex = -theindex;
03481         d = Dollars + theindex;
03482     #endif
03483
03484     if ( d->type != DOLINDEX
03485         || d->index < AM.OffsetIndex
03486         || d->index >= AM.OffsetIndex + WILDOFFSET ) {
03487         MLOCK(ErrorMessageLock);
03488         MesPrint("%$s should have been an index"
03489             ,AC.dollarnames->namebuffer+d->name);
03490         AN.currentTerm = term;
03491         MesPrint("Current term: %t");
03492         AN.listinprint = printscratch;
03493         printscratch[0] = DOLLAREXPRESSION;
03494         printscratch[1] = theindex;
03495         MesPrint("%$s = %$"
03496             ,AC.dollarnames->namebuffer+d->name);
03497         MUNLOCK(ErrorMessageLock);
03498     #ifdef WITHPTHREADS
03499         if ( ddtype > 0 && ddtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
03500     #endif
03501         goto GenCall;
03502     }
03503     theindex = d->index;
03504     #ifdef WITHPTHREADS
03505     if ( ddtype > 0 && ddtype != MODLOCAL ) { UNLOCK(d->pthreadlockread); }
03506     #endif
03507 }
03508 cp[1] = SUBEXPSIZE+4;
03509 cp += SUBEXPSIZE;
03510 *cp++ = INDTOIND;
03511 *cp++ = 4;
03512 *cp++ = theindex;
03513 i = C->lhs[level][1] - 3;
03514 cp++;
03515 AR.CompressPointer = cp;
03516 while ( --i >= 0 ) {
03517     cp[-1] = *tlhs++;
03518     termout = AT.WorkPointer;
03519     if ( ( jlhs = WildFill(BHEAD termout,term,op) ) < 0 )
03520         goto GenCall;
03521     m = term;
03522     jlhs = *m;
03523     while ( --jlhs >= 0 ) {
03524         if ( *m++ != *termout++ ) break;
03525     }
03526     if ( jlhs >= 0 ) {
03527         termout = AT.WorkPointer;
03528         AT.WorkPointer = termout + *termout;
03529         if ( Generator(BHEAD termout,level) ) goto GenCall;
03530         AT.WorkPointer = termout;
03531     }
03532     else {
03533         AR.CompressPointer = op;
03534         goto SkipCount;
03535     }
03536 }
03537 AR.CompressPointer = op;
03538 goto CommonEnd;
03539 }
03540 case TYPESUM:
03541 {
03542     WORD *wp, *cp = AR.CompressPointer, *op = AR.CompressPointer;
03543     WORD theindex;
03544     WORD *ow;
03545     /*
03546     At this point it is safest to determine CurDum
03547     */
03548     AR.CurDum = DetCurDum(BHEAD term);
03549     i = C->lhs[level][1]-2;
03550     wp = C->lhs[level] + 2;

```



```

03551         cp[1] = SUBEXPSIZE+4*i;
03552         cp += SUBEXPSIZE;
03553         while ( --i >= 0 ) {
03554             theindex = *wp++;
03555             if ( theindex < 0 ) { /* $-variable */
03556 #ifdef WITHPTHREADS
03557                 int ddtype = -1;
03558                 theindex = -theindex;
03559                 d = Dollars + theindex;
03560                 if ( AS.MultiThreaded ) {
03561                     for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
03562                         if ( theindex == ModOptdollars[nummodopt].number ) break;
03563                     }
03564                     if ( nummodopt < NumModOptdollars ) {
03565                         ddtype = ModOptdollars[nummodopt].type;
03566                         if ( ddtype == MODLOCAL ) {
03567                             d = ModOptdollars[nummodopt].dstruct+AT.identity;
03568                         }
03569                         else {
03570                             LOCK(d->pthreadslackread);
03571                         }
03572                     }
03573                 }
03574             #else
03575                 theindex = -theindex;
03576                 d = Dollars + theindex;
03577             #endif
03578             if ( d->type != DOLINDEX
03579                 || d->index < AM.OffsetIndex
03580                 || d->index >= AM.OffsetIndex + WILDOFFSET ) {
03581                 MLOCK(ErrorMessageLock);
03582                 MesPrint("%s should have been an index"
03583                     ,AC.dollarnames->namebuffer+d->name);
03584                 AN.currentTerm = term;
03585                 MesPrint("Current term: %t");
03586                 AN.listinprint = printscratch;
03587                 printscratch[0] = DOLLAREXPRESSION;
03588                 printscratch[1] = theindex;
03589                 MesPrint("%s = %s"
03590                     ,AC.dollarnames->namebuffer+d->name);
03591                 MUNLOCK(ErrorMessageLock);
03592 #ifdef WITHPTHREADS
03593                 if ( ddtype > 0 && ddtype != MODLOCAL ) {
03594                     UNLOCK(d->pthreadslackread); }
03595                 #endif
03596                 goto GenCall;
03597             }
03598             theindex = d->index;
03599             if ( ddtype > 0 && ddtype != MODLOCAL ) { UNLOCK(d->pthreadslackread); }
03600         #endif
03601     }
03602     *cp++ = INDTOIND;
03603     *cp++ = 4;
03604     *cp++ = theindex;
03605     *cp++ = ++AR.CurDum;
03606 }
03607 ow = AT.WorkPointer;
03608 AR.CompressPointer = cp;
03609 if ( WildFill(BHEAD ow,term,op) < 0 ) goto GenCall;
03610 AR.CompressPointer = op;
03611 i = ow[0];
03612 WORD term_changed = 0;
03613 for ( j = 0; j < i; j++ ) {
03614     if ( term[j] != ow[j] ) term_changed = 1;
03615     term[j] = ow[j];
03616 }
03617 // If the term was modified by WildFill, set RepCount.
03618 if ( term_changed ) *AN.RepPoint = 1;
03619 AT.WorkPointer = ow;
03620 ReNumber(BHEAD term);
03621 goto Renormalize;
03622 }
03623 case TYPECHISHOLM:
03624     if ( Chisholm(BHEAD term,level) ) goto GenCall;
03625 CommonEnd:
03626     AT.WorkPointer = term + *term;
03627     goto Return0;
03628 case TYPEARG:
03629     if ( ( i = execarg(BHEAD term,level) ) < 0 ) goto GenCall;
03630     level = C->lhs[level][2];
03631     if ( i > 0 ) goto ReStart;
03632     break;
03633 case TYPENORM:
03634 case TYPENORM2:
03635 case TYPENORM3:

```

```

03636         case TYPENORM4:
03637         case TYPESPLITARG:
03638         case TYPESPLITARG2:
03639         case TYPESPLITFIRSTARG:
03640         case TYPESPLITLASTARG:
03641         case TYPEARGTOEXTRASymbol:
03642             if ( execarg(BHEAD term,level) < 0 ) goto GenCall;
03643             level = C->lhs[level][2];
03644             break;
03645         case TYPEFACTARG:
03646         case TYPEFACTARG2:
03647             { WORD jjj;
03648             if ( ( jjj = execarg(BHEAD term,level) ) < 0 ) goto GenCall;
03649             if ( jjj > 0 ) goto ReStart;
03650             level = C->lhs[level][2];
03651             break; }
03652         case TYPEEXIT:
03653             if ( C->lhs[level][2] > 0 ) {
03654                 MLOCK(ErrorMessageLock);
03655                 MesPrint("%s",C->lhs[level]+3);
03656                 MUNLOCK(ErrorMessageLock);
03657             }
03658             Terminate(-1);
03659             goto GenCall;
03660         case TYPESETEXIT:
03661             AM.exitflag = 1; /* no danger of race conditions */
03662             break;
03663         case TYPEPRINT:
03664             AN.currentTerm = term;
03665             AN.numlistinprint = (C->lhs[level][1] - C->lhs[level][4] - 5)/2;
03666             AN.listinprint = C->lhs[level]+5+C->lhs[level][4];
03667             MLOCK(ErrorMessageLock);
03668             AO.ErrorBlock = 1;
03669             MesPrint((char *) (C->lhs[level]+5));
03670             AO.ErrorBlock = 0;
03671             MUNLOCK(ErrorMessageLock);
03672             break;
03673         case TYPEFPRINT:
03674             {
03675                 int oldFOflag;
03676                 WORD oldPrintType, oldLogHandle = AC.LogHandle;
03677                 AC.LogHandle = C->lhs[level][2];
03678                 MLOCK(ErrorMessageLock);
03679                 oldFOflag = AM.FileOnlyFlag;
03680                 oldPrintType = AO.PrintType;
03681                 if ( AC.LogHandle >= 0 ) {
03682                     AM.FileOnlyFlag = 1;
03683                     AO.PrintType |= PRINTLFILE;
03684                 }
03685                 AO.PrintType |= C->lhs[level][3];
03686                 AN.currentTerm = term;
03687                 AN.numlistinprint = (C->lhs[level][1] - C->lhs[level][4] - 5)/2;
03688                 AN.listinprint = C->lhs[level]+5+C->lhs[level][4];
03689                 MesPrint((char *) (C->lhs[level]+5));
03690                 AO.PrintType = oldPrintType;
03691                 AM.FileOnlyFlag = oldFOflag;
03692                 MUNLOCK(ErrorMessageLock);
03693                 AC.LogHandle = oldLogHandle;
03694             }
03695             break;
03696         case TYPEREDEFPRE:
03697             j = C->lhs[level][2];
03698         #ifdef WITHMPI
03699             {
03700                 /*
03701                 * Regardless of parallel/nonparallel switch, we need to set
03702                 * AC.inputnumbers[ii], which indicates that the corresponding
03703                 * preprocessor variable is redefined and so we need to
03704                 * send/broadcast it.
03705                 */
03706                 int ii;
03707                 for ( ii = 0; ii < AC.numpfirstnum; ii++ ) {
03708                     if ( AC.pfirstnum[ii] == j ) break;
03709                 }
03710                 AC.inputnumbers[ii] = AN.ninterms;
03711             }
03712         #endif
03713         #ifdef WITHPTHREADS
03714             if ( AS.MultiThreaded ) {
03715                 int ii;
03716                 for ( ii = 0; ii < AC.numpfirstnum; ii++ ) {
03717                     if ( AC.pfirstnum[ii] == j ) break;
03718                 }
03719                 if ( AN.inputnumber < AC.inputnumbers[ii] ) break;
03720                 LOCK(AP.PreVarLock);
03721                 if ( AN.inputnumber >= AC.inputnumbers[ii] ) {
03722                     a = C->lhs[level]+4;

```

```

03723         if ( a[a[-1]] == 0 )
03724             PutPreVar(PreVar[j].name, (UBYTE *) (a), 0, 1);
03725         else
03726             PutPreVar(PreVar[j].name, (UBYTE *) (a)
03727                 , (UBYTE *) (a+a[-1]+1), 1);
03728     /*
03729         PutPreVar(PreVar[j].name, (UBYTE *) (C->lhs[level]+4), 0, 1);
03730     */
03731     AC.inputnumbers[ii] = AN.inputnumber;
03732     }
03733     UNLOCK(AP.PreVarLock);
03734 }
03735 else
03736 #endif
03737 {
03738     a = C->lhs[level]+4;
03739     LOCK(AP.PreVarLock);
03740     if ( a[a[-1]] == 0 )
03741         PutPreVar(PreVar[j].name, (UBYTE *) (a), 0, 1);
03742     else
03743         PutPreVar(PreVar[j].name, (UBYTE *) (a)
03744             , (UBYTE *) (a+a[-1]+1), 1);
03745     UNLOCK(AP.PreVarLock);
03746 }
03747 break;
03748 case TYPERENUMBER:
03749     AT.WorkPointer = term + *term;
03750     if ( FullRenummer(BHEAD term, C->lhs[level][2]) ) goto GenCall;
03751     AT.WorkPointer = term + *term;
03752     if ( *term == 0 ) goto Return0;
03753     break;
03754 case TYPETRY:
03755     if ( TryDo(BHEAD term, C->lhs[level], level) ) goto GenCall;
03756     AT.WorkPointer = term + *term;
03757     goto Return0;
03758 case TYPEASSIGN:
03759     { WORD onc = AR.NoCompress, oldEside = AR.Eside;
03760       WORD oldrepeat = *AN.RepPoint;
03761     /*
03762         Here we have to assign an expression to a $ variable.
03763     */
03764     AR.Eside = RHSIDE;
03765     AR.NoCompress = 1;
03766     AN.cTerm = AN.currentTerm = term;
03767     AT.WorkPointer = term + *term;
03768     *AT.WorkPointer++ = 0;
03769     if ( AssignDollar(BHEAD term, level) ) goto GenCall;
03770     AT.WorkPointer = term + *term;
03771     AN.cTerm = 0;
03772     *AN.RepPoint = oldrepeat;
03773     AR.NoCompress = onc;
03774     AR.Eside = oldEside;
03775     break;
03776     }
03777 case TYPEFINDLOOP:
03778     if ( Lus(term, C->lhs[level][3], C->lhs[level][4],
03779         C->lhs[level][5], C->lhs[level][6], C->lhs[level][2]) ) {
03780         AT.WorkPointer = term + *term;
03781         goto Renormalize;
03782     }
03783     break;
03784 case TYPEINSIDE:
03785     if ( InsideDollar(BHEAD C->lhs[level], level) < 0 ) goto GenCall;
03786     level = C->lhs[level][2];
03787     break;
03788 case TYPETERM:
03789     retnorm = execterm(BHEAD term, level);
03790     AN.RepPoint = RepSto;
03791     AR.CurDum = DumNow;
03792     CC->numrhs = oldtoprhs;
03793     CC->Pointer = CC->Buffer + oldcpointer;
03794     CCC->numrhs = oldatoprhs;
03795     CCC->Pointer = CCC->Buffer + oldacpointer;
03796     return(retnorm);
03797 case TYPEDETCURDUM:
03798     AT.WorkPointer = term + *term;
03799     AR.CurDum = DetCurDum(BHEAD term);
03800     break;
03801 case TYPEINEXPRESSION:
03802     {WORD *ll = C->lhs[level];
03803       int numexprs = (int)(ll[1]-3);
03804       ll += 3;
03805       while ( numexprs-- >= 0 ) {
03806         if ( *ll == AR.CurExpr ) break;
03807         ll++;
03808       }
03809       if ( numexprs < 0 ) level = C->lhs[level][2];

```

```

03810         }
03811         break;
03812     case TYPEMERGE:
03813         AT.WorkPointer = term + *term;
03814         if ( DoShuffle(term,level,C->lhs[level][2],C->lhs[level][3]) )
03815             goto GenCall;
03816         AT.WorkPointer = term + *term;
03817         goto Return0;
03818     case TYPESTUFFLE:
03819         AT.WorkPointer = term + *term;
03820         if ( DoStuffle(term,level,C->lhs[level][2],C->lhs[level][3]) )
03821             goto GenCall;
03822         AT.WorkPointer = term + *term;
03823         goto Return0;
03824     case TYPETESTUSE:
03825         AT.WorkPointer = term + *term;
03826         if ( TestUse(term,level) ) goto GenCall;
03827         AT.WorkPointer = term + *term;
03828         break;
03829     case TYPEAPPLY:
03830         AT.WorkPointer = term + *term;
03831         if ( ApplyExec(term,C->lhs[level][2],level) < C->lhs[level][2] ) {
03832             AT.WorkPointer = term + *term;
03833             *AN.RepPoint = 1;
03834             goto ReStart;
03835         }
03836         AT.WorkPointer = term + *term;
03837         break;
03838     /*
03839     case TYPEAPPLYRESET:
03840         AT.WorkPointer = term + *term;
03841         if ( ApplyReset(level) ) goto GenCall;
03842         AT.WorkPointer = term + *term;
03843         break;
03844     */
03845     case TYPECHAININ:
03846         { int lter = *term;
03847         AT.WorkPointer = term + *term;
03848         if ( ChainIn(BHEAD term,C->lhs[level][2]) ) goto GenCall;
03849         AT.WorkPointer = term + *term;
03850         /* Symmetry properties might mean the term has vanished */
03851         if ( *term == 0 ) goto Return0;
03852         if ( *term != lter ) *AN.RepPoint = 1;
03853         }
03854         break;
03855     case TYPECHAINOUT:
03856         { int lter = *term;
03857         AT.WorkPointer = term + *term;
03858         if ( ChainOut(BHEAD term,C->lhs[level][2]) ) goto GenCall;
03859         AT.WorkPointer = term + *term;
03860         if ( *term != lter ) *AN.RepPoint = 1;
03861         }
03862         break;
03863     case TYPEFACTOR:
03864         AT.WorkPointer = term + *term;
03865         if ( DollarFactorize(BHEAD C->lhs[level][2]) ) goto GenCall;
03866         AT.WorkPointer = term + *term;
03867         break;
03868     case TYPEARGIMPLODE:
03869         AT.WorkPointer = term + *term;
03870         if ( ArgumentImplode(BHEAD term,C->lhs[level]) ) goto GenCall;
03871         AT.WorkPointer = term + *term;
03872         break;
03873     case TYPEARGEXPLODE:
03874         AT.WorkPointer = term + *term;
03875         if ( ArgumentExplode(BHEAD term,C->lhs[level]) ) goto GenCall;
03876         AT.WorkPointer = term + *term;
03877         break;
03878     case TYPEDENOMINATORS:
03879         if ( DenToFunction(term,C->lhs[level][2]) ) goto ReStart;
03880         break;
03881     case TYPEDROPCOEFFICIENT:
03882         DropCoefficient(BHEAD term);
03883         break;
03884     case TYPETRANSFORM:
03885         AT.WorkPointer = term + *term;
03886         if ( RunTransform(BHEAD term,C->lhs[level]+2) ) goto GenCall;
03887         AT.WorkPointer = term + *term;
03888         if ( *term == 0 ) goto Return0;
03889         goto ReStart;
03890     case TYPETOPOLYNOMIAL:
03891         AT.WorkPointer = term + *term;
03892         termout = AT.WorkPointer;
03893         if ( ConvertToPoly(BHEAD term,termout,C->lhs[level],0) < 0 ) goto GenCall;
03894         if ( *termout == 0 ) goto Return0;
03895         i = termout[0]; t = term; NCOPY(t,termout,i);
03896         AT.WorkPointer = term + *term;

```

```

03897         break;
03898     case TYPEFROMPOLYNOMIAL:
03899         AT.WorkPointer = term + *term;
03900         termout = AT.WorkPointer;
03901         if ( ConvertFromPoly(BHEAD term,termout,0,numxsymbol,0,0) < 0 ) goto GenCall;
03902         if ( *term == 0 ) goto Return0;
03903         i = termout[0]; t = term; NCOPY(t,termout,i);
03904         AT.WorkPointer = term + *term;
03905         goto ReStart;
03906     case TYPEDOLOOP:
03907         level = TestDoLoop(BHEAD C->lhs[level],level);
03908         if ( level < 0 ) goto GenCall;
03909         break;
03910     case TYPEENDDOLOOP:
03911         level = TestEndDoLoop(BHEAD C->lhs[C->lhs[level][2]],C->lhs[level][2]);
03912         if ( level < 0 ) goto GenCall;
03913         break;
03914     case TYPEDROPSYMBOLS:
03915         DropSymbols(BHEAD term);
03916         break;
03917     case TYPEPUTINSIDE:
03918         AT.WorkPointer = term + *term;
03919         if ( PutInside(BHEAD term,C->lhs[level]) < 0 ) goto GenCall;
03920         AT.WorkPointer = term + *term;
03921         /*
03922          * We need to call Generator() to convert slow notation to
03923          * fast notation, which fixes Issue #30.
03924          */
03925         if ( Generator(BHEAD term,level) < 0 ) goto GenCall;
03926         goto Return0;
03927     case TYPETOSPECTATOR:
03928         if ( PutInSpectator(term,C->lhs[level][2]) < 0 ) goto GenCall;
03929         goto Return0;
03930     case TYPECANONICALIZE:
03931         AT.WorkPointer = term + *term;
03932         if ( DoCanonicalize(BHEAD term,C->lhs[level]) ) goto GenCall;
03933         AT.WorkPointer = term + *term;
03934         if ( *term == 0 ) goto Return0;
03935         break;
03936     case TYPESWITCH:
03937         AT.WorkPointer = term + *term;
03938         if ( DoSwitch(BHEAD term,C->lhs[level]) ) goto GenCall;
03939         goto Return0;
03940     case TYPEENDSWITCH:
03941         AT.WorkPointer = term + *term;
03942         if ( DoEndSwitch(BHEAD term,C->lhs[level]) ) goto GenCall;
03943         goto Return0;
03944     case TYPESETUSERFLAG:
03945         Expressions[AR.CurExpr].uflags |= 1 « (C->lhs[level][2]);
03946         break;
03947     case TYPECLEARUSERFLAG:
03948         Expressions[AR.CurExpr].uflags &= ~(1 « (C->lhs[level][2]));
03949         break;
03950     case TYPEALLLOOPS:
03951         AT.WorkPointer = term + *term;
03952         if ( AllLoops(BHEAD term,level) ) goto GenCall;
03953         goto Return0;
03954     case TYPEALLPATHS:
03955         AT.WorkPointer = term + *term;
03956         if ( AllPaths(BHEAD term,level) ) goto GenCall;
03957         goto Return0;
03958 #ifdef WITHFLOAT
03959     case TYPEEVALUATE:
03960         AT.WorkPointer = term + *term;
03961         if ( C->lhs[level][2] == MZV
03962             || C->lhs[level][2] == EULER
03963             || C->lhs[level][2] == MZVHALF
03964             || C->lhs[level][2] == ALLMZVFUNCTIONS
03965             ) {
03966             if ( EvaluateEuler(BHEAD term,level,C->lhs[level][2]) ) goto GenCall;
03967         }
03968         else {
03969             if ( EvaluateFun(BHEAD term,level,C->lhs[level]) ) goto GenCall;
03970         }
03971         /*
03972         else if ( C->lhs[level][2] == SQRTFUNCTION ) {
03973             if ( EvaluateSqrt(BHEAD term,level,C->lhs[level][2]) ) goto GenCall;
03974         }
03975         else {
03976             MLOCK(ErrorMessageLock);
03977             MesPrint("Illegal function %d in evaluate statement.",C->lhs[level][2]);
03978             MUNLOCK(ErrorMessageLock);
03979             goto GenCall;
03980         }
03981         */
03982         goto Return0;
03983     case TYPETOFLOAT:

```

```

03984         AT.WorkPointer = term + *term;
03985         if ( ToFloat(BHEAD term,level) ) goto GenCall;
03986         goto Return0;
03987     case TYPETORAT:
03988         AT.WorkPointer = term + *term;
03989         if ( ToRat(BHEAD term,level) ) goto GenCall;
03990         goto Return0;
03991 #endif
03992     }
03993     goto SkipCount;
03994 /*
03995     #] Special action :
03996 */
03997 }
03998 } while ( ( i = TestMatch(BHEAD term,&level) ) == 0 );
03999 if ( AT.WorkPointer < term + *term ) AT.WorkPointer = term + *term;
04000 if ( i > 0 ) replac = TestSub(BHEAD term,level);
04001 else replac = i;
04002 if ( replac >= 0 || AT.TMout[1] != SYMMETRIZE ) {
04003     *AN.RepPoint = 1;
04004     AR.expchanged = 1;
04005 }
04006 if ( replac < 0 ) { /* Terms come from automatic generation */
04007     AutoGen: i = *AT.TMout;
04008     t = termout = AT.WorkPointer;
04009     if ( ( AT.WorkPointer += i ) > AT.WorkTop ) goto OverWork;
04010     accum = AT.TMout;
04011     while ( --i >= 0 ) *t++ = *accum++;
04012     if ( (*FG.Operation[termout[1]])(BHEAD term,termout,replac,level) ) goto GenCall;
04013     AT.WorkPointer = termout;
04014     goto Return0;
04015 }
04016 }
04017 if ( applyflag ) { TableReset(); applyflag = 0; }
04018
04019 if ( AN.TeInFun ) { /* Match in function argument */
04020     if ( AN.TeInFun < 0 && !AN.TeSuOut ) {
04021
04022         if ( AR.TePos >= 0 ) goto AutoGen;
04023         switch ( AN.TeInFun ) {
04024             case -1:
04025                 if ( DoDistrib(BHEAD term,level) ) goto GenCall;
04026                 break;
04027             case -2:
04028                 if ( DoDelta3(BHEAD term,level) ) goto GenCall;
04029                 break;
04030             case -3:
04031                 if ( DoTableExpansion(term,level) ) goto GenCall;
04032                 break;
04033             case -4:
04034                 if ( FactorIn(BHEAD term,level) ) goto GenCall;
04035                 break;
04036             case -5:
04037                 if ( FactorInExpr(BHEAD term,level) ) goto GenCall;
04038                 break;
04039             case -6:
04040                 if ( TermsInBracket(BHEAD term,level) < 0 ) goto GenCall;
04041                 break;
04042             case -7:
04043                 if ( ExtraSymFun(BHEAD term,level) < 0 ) goto GenCall;
04044                 break;
04045             case -8:
04046                 if ( GCDfunction(BHEAD term,level) < 0 ) goto GenCall;
04047                 break;
04048             case -9:
04049                 if ( DIVfunction(BHEAD term,level,0) < 0 ) goto GenCall;
04050                 break;
04051             case -10:
04052                 if ( DIVfunction(BHEAD term,level,1) < 0 ) goto GenCall;
04053                 break;
04054             case -11:
04055                 if ( DIVfunction(BHEAD term,level,2) < 0 ) goto GenCall;
04056                 break;
04057             case -12:
04058                 if ( DoPermutations(BHEAD term,level) ) goto GenCall;
04059                 break;
04060             case -13:
04061                 if ( DoPartitions(BHEAD term,level) ) goto GenCall;
04062                 break;
04063             case -14:
04064                 if ( DIVfunction(BHEAD term,level,3) < 0 ) goto GenCall;
04065                 break;
04066             case -15:
04067                 if ( GenTopologies(BHEAD term,level) < 0 ) goto GenCall;
04068                 break;
04069             case -16:
04070                 if ( GenDiagrams(BHEAD term,level) < 0 ) goto GenCall;

```

```

04071         break;
04072     }
04073 }
04074 else {
04075     termout = AT.WorkPointer;
04076     AT.WorkPointer = (WORD *) (((UBYTE *) (AT.WorkPointer)) + AM.MaxTer);
04077     if ( AT.WorkPointer > AT.WorkTop ) goto OverWork;
04078     if ( InFunction(BHEAD term,termout) ) goto GenCall;
04079     AT.WorkPoint = termout + *termout;
04080     *AN.RepPoint = 1;
04081     AR.expchanged = 1;
04082     if ( *termout && Generator(BHEAD termout,level) < 0 ) goto GenCall;
04083     AT.WorkPointer = termout;
04084 }
04085 }
04086 else if ( replac > 0 ) {
04087     power = AN.TeSuOut;
04088     tepos = AR.TePos;
04089     if ( power < 0 ) { /* Table expansion */
04090         power = -power; tepos = 0;
04091     }
04092     extractbuff = AT.TMbuff;
04093     if ( extractbuff == AM.dbufnum ) {
04094         d = DolToTerms(BHEAD replac);
04095         if ( d && d->where != 0 ) {
04096             iscopy = 1;
04097             if ( AT.TMdolfac > 0 ) { /* We need a factor */
04098                 if ( AT.TMdolfac == 1 ) {
04099                     if ( d->nfactors ) {
04100                         numfac[0] = 4;
04101                         numfac[1] = d->nfactors;
04102                         numfac[2] = 1;
04103                         numfac[3] = 3;
04104                         numfac[4] = 0;
04105                     }
04106                     else {
04107                         numfac[0] = 0;
04108                     }
04109                     StartBuf = numfac;
04110                 }
04111                 else {
04112                     if ( (AT.TMdolfac-1) > d->nfactors && d->nfactors > 0 ) {
04113                         MLOCK(ErrorMessageLock);
04114                         MesPrint("Attempt to use an nonexisting factor %d of a
04115 $-variable", (WORD) (AT.TMdolfac-1));
04116                         if ( d->nfactors == 1 )
04117                             MesPrint("There is only one factor");
04118                         else
04119                             MesPrint("There are only %d factors", (WORD) (d->nfactors));
04120                         MUNLOCK(ErrorMessageLock);
04121                         goto GenCall;
04122                     }
04123                     if ( d->nfactors > 1 ) {
04124                         DOLLARS dd;
04125                         LONG dsize;
04126                         WORD *td1, *td2;
04127                         dd = Dollars + replac;
04128 #ifdef WITHPTHREADS
04129                     {
04130                         int nummodopt, dtype = -1;
04131                         if ( AS.MultiThreaded && ( AC.mparallelflag == PARALLELFLAG ) ) {
04132                             for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
04133                                 if ( replac == ModOptdollars[nummodopt].number ) break;
04134                             }
04135                             if ( nummodopt < NumModOptdollars ) {
04136                                 dtype = ModOptdollars[nummodopt].type;
04137                                 if ( dtype == MODLOCAL ) {
04138                                     dd = ModOptdollars[nummodopt].dstruct+AT.identity;
04139                                 }
04140                             }
04141                         }
04142 #endif
04143                     dsize = dd->factors[AT.TMdolfac-2].size;
04144                     /*
04145                     We copy only the factor we need
04146                     */
04147                     if ( dsize == 0 ) {
04148                         numfac[0] = 4;
04149                         numfac[1] = d->factors[AT.TMdolfac-2].value;
04150                         numfac[2] = 1;
04151                         numfac[3] = 3;
04152                         numfac[4] = 0;
04153                         StartBuf = numfac;
04154                         if ( numfac[1] < 0 ) {
04155                             numfac[1] = -numfac[1];
04156                             numfac[3] = -numfac[3];

```

```

04157         }
04158     }
04159     else {
04160         d->factors[AT.TMdolfac-2].where = td2 = (WORD *)Malloc1(
04161             (dsize+1)*sizeof(WORD), "Copy of factor");
04162         td1 = dd->factors[AT.TMdolfac-2].where;
04163         StartBuf = td2;
04164         d->size = dsize; d->type = DOLTERMS;
04165         NCOPY(td2,td1,dsize);
04166         *td2 = 0;
04167     }
04168 }
04169 else if ( d->nfactors == 1 ) {
04170     StartBuf = d->where;
04171 }
04172 else {
04173     MLOCK(ErrorMessageLock);
04174     if ( d->nfactors == 0 ) {
04175         MesPrint("Attempt to use factor %d of an unfactored
$-variable", (WORD) (AT.TMdolfac-1));
04176     }
04177     else {
04178         MesPrint("Internal error. Illegal number of factors for $-variable");
04179     }
04180     MUNLOCK(ErrorMessageLock);
04181     goto GenCall;
04182 }
04183 }
04184 }
04185 else StartBuf = d->where;
04186 }
04187 else {
04188     d = Dollars + replac;
04189     StartBuf = zeroDollar;
04190 }
04191 posisub = 0;
04192 i = DetCommu(d->where);
04193 #ifdef WITHPTHREADS
04194     if ( AS.MultiThreaded ) {
04195         for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
04196             if ( replac == ModOptdollars[nummodopt].number ) break;
04197         }
04198         if ( nummodopt < NumModOptdollars ) {
04199             dtype = ModOptdollars[nummodopt].type;
04200             if ( dtype != MODLOCAL && dtype != MODSUM ) {
04201                 if ( StartBuf[0] && StartBuf[StartBuf[0]] ) {
04202                     MLOCK(ErrorMessageLock);
04203                     MesPrint("A dollar variable with modoption max or min can have only one
term");
04204                     MUNLOCK(ErrorMessageLock);
04205                     goto GenCall;
04206                 }
04207                 LOCK(d->pthreadlockread);
04208             }
04209         }
04210     }
04211 #endif
04212 }
04213 else {
04214     StartBuf = cbuf[extractbuff].Buffer;
04215     posisub = cbuf[extractbuff].rhs[replac] - StartBuf;
04216     i = (WORD)cbuf[extractbuff].CanCommu[replac];
04217 }
04218 if ( power == 1 ) { /* Just a single power */
04219     termout = AT.WorkPointer;
04220     AT.WorkPointer = (WORD *)(((UBYTE *) (AT.WorkPointer)) + AM.MaxTer);
04221     if ( AT.WorkPointer > AT.WorkTop ) goto OverWork;
04222     while ( StartBuf[posisub] ) {
04223         if ( extractbuff == AT.allbufnum ) WildDollars(BHEAD &(StartBuf[posisub]));
04224         AT.WorkPointer = (WORD *)(((UBYTE *) (termout)) + AM.MaxTer);
04225         if ( InsertTerm(BHEAD term,replac,extractbuff,
&(StartBuf[posisub]),termout,tepos) < 0 ) goto GenCall;
04226         AT.WorkPointer = termout + *termout;
04227         *AN.RepPoint = 1;
04228         AR.expchanged = 1;
04229         posisub += StartBuf[posisub];
04230     }
04231 /*
04232     For multiple table substitutions it may be better to
04233     do modulus arithmetic right here
04234     Turns out to be not very effective.
04235
04236     if ( AN.ncmod != 0 ) {
04237         if ( Modulus(termout) ) goto GenCall;
04238         if ( !*termout ) goto Return0;
04239     }
04240 */
04241 #ifdef WITHPTHREADS

```



```

04242         if ( dtype > 0 && dtype != MODLOCAL && dtype != MODSUM ) {
04243             UNLOCK(d->pthreadlockread); }
04243         if ( ( AS.Balancing && CC->numrhs == 0 ) && StartBuf[posisub] ) {
04244             if ( ( id = ConditionalGetAvailableThread() ) >= 0 ) {
04245                 if ( BalanceRunThread(BHEAD id,termout,level) < 0 ) goto GenCall;
04246             }
04247         }
04248         else
04249 #endif
04250         if ( Generator(BHEAD termout,level) < 0 ) goto GenCall;
04251 #ifdef WITHPTHREADS
04252         if ( dtype > 0 && dtype != MODLOCAL ) { dtype = 0; break; }
04253 #endif
04254         if ( iscopy == 0 && ( extractbuff != AM.dbufnum ) ) {
04255             /*
04256              * There are cases in which a bigger buffer is created
04257              * on the fly, like with wildcard buffers.
04258              * We play it safe here. Maybe we can be more selective
04259              * in some distant future?
04260              */
04261             StartBuf = cbuf[extractbuff].Buffer;
04262         }
04263     }
04264     if ( extractbuff == AT.allbufnum ) {
04265         CBUF *Ce = cbuf + extractbuff;
04266         Ce->Pointer = Ce->rhs[Ce->numrhs--];
04267     }
04268 #ifdef WITHPTHREADS
04269     if ( dtype > 0 && dtype != MODLOCAL && dtype != MODSUM ) { UNLOCK(d->pthreadlockread);
04270 dtype = 0; }
04271 #endif
04271     if ( iscopy ) {
04272         if ( d->nfactors > 1 ) {
04273             int j;
04274             for ( j = 0; j < d->nfactors; j++ ) {
04275                 if ( d->factors[j].where ) M_free(d->factors[j].where,"Copy of factor");
04276             }
04277             M_free(d->factors,"Dollar factors");
04278         }
04279         M_free(d,"Copy of dollar variable");
04280         d = 0; iscopy = 0;
04281     }
04282     AT.WorkPointer = termout;
04283 }
04284 else if ( i <= 1 ) { /* Use binomials */
04285     LONG posit, olw;
04286     WORD *same, *ow = AT.WorkPointer;
04287     LONG olpw = AT.posWorkPointer;
04288     power1 = power+1;
04289     WantAddLongs(power1);
04290     olw = posit = AT.lWorkPointer; AT.lWorkPointer += power1;
04291     same = ++AT.WorkPointer;
04292     a = accum = ( AT.WorkPointer += power1+1 );
04293     AT.WorkPointer = (WORD *)(((UBYTE *) (AT.WorkPointer)) + 2*AM.MaxTer);
04294     if ( AT.WorkPointer > AT.WorkTop ) goto OverWork;
04295     AT.lWorkspace[posit] = posisub;
04296     same[-1] = 0;
04297     *same = 1;
04298     *accum = 0;
04299     tepos = AR.TePos;
04300     i = 1;
04301     do {
04302         if ( StartBuf[AT.lWorkspace[posit]] ) {
04303             if ( ( a = PasteTerm(BHEAD i-1,accum,
04304                 &(StartBuf[AT.lWorkspace[posit]]),i,*same) ) == 0 )
04305                 goto GenCall;
04306             AT.lWorkspace[posit+1] = AT.lWorkspace[posit];
04307             same[1] = *same + 1;
04308             if ( i > 1 && AT.lWorkspace[posit] < AT.lWorkspace[posit-1] ) *same = 1;
04309             AT.lWorkspace[posit] += StartBuf[AT.lWorkspace[posit]];
04310             i++;
04311             posit++;
04312             same++;
04313         }
04314         else {
04315             i--; posit--; same--;
04316         }
04317     } if ( i > power ) {
04318         termout = AT.WorkPointer = a;
04319         AT.WorkPointer = (WORD *)(((UBYTE *) (AT.WorkPointer)) + 2*AM.MaxTer);
04320         if ( AT.WorkPointer > AT.WorkTop )
04321             goto OverWork;
04322         if ( FiniTerm(BHEAD term,accum,termout,replac,tepos) ) goto GenCall;
04323         AT.WorkPointer = termout + *termout;
04324         *AN.RepPoint = 1;
04325         AR.expchanged = 1;
04326 #ifdef WITHPTHREADS

```

```

04327         if ( dtype > 0 && dtype != MODLOCAL && dtype != MODSUM ) {
04328             UNLOCK(d->pthreadlockread); }
04328         if ( ( AS.Balancing && CC->numrhs == 0 ) && ( i > 0 )
04329             && ( id = ConditionalGetAvailableThread() ) >= 0 ) {
04330             if ( BalanceRunThread(BHEAD id,termout,level) < 0 ) goto GenCall;
04331         }
04332         else
04333         #endif
04334         if ( Generator(BHEAD termout,level) ) goto GenCall;
04335     #ifdef WITHPTHREADS
04336         if ( dtype > 0 && dtype != MODLOCAL ) { dtype = 0; break; }
04337     #endif
04338         if ( iscopy == 0 && ( extractbuff != AM.dbufnum ) )
04339             StartBuf = cbuf[extractbuff].Buffer;
04340         i--; posit--; same--;
04341     }
04342     } while ( i > 0 );
04343     #ifdef WITHPTHREADS
04344         if ( dtype > 0 && dtype != MODLOCAL && dtype != MODSUM ) { UNLOCK(d->pthreadlockread);
04345             dtype = 0; }
04346         #endif
04347         if ( iscopy ) {
04348             if ( d->nfactors > 1 ) {
04349                 int j;
04350                 for ( j = 0; j < d->nfactors; j++ ) {
04351                     if ( d->factors[j].where ) M_free(d->factors[j].where,"Copy of factor");
04352                 }
04353                 M_free(d->factors,"Dollar factors");
04354                 M_free(d,"Copy of dollar variable");
04355                 d = 0; iscopy = 0;
04356             }
04357             AT.WorkPointer = ow; AT.lWorkPointer = olw; AT.posWorkPointer = olpw;
04358         }
04359         else {
04360             /* No binomials */
04361             LONG posit, olw, olpw = AT.posWorkPointer;
04362             WantAddLongs(power);
04363             posit = olw = AT.lWorkPointer; AT.lWorkPointer += power;
04364             a = accum = AT.WorkPointer;
04365             AT.WorkPointer = (WORD *)(((UBYTE *) (AT.WorkPointer)) + 2*AM.MaxTer);
04366             if ( AT.WorkPointer > AT.WorkTop ) goto OverWork;
04367             for ( i = 0; i < power; i++ ) AT.lWorkspace[posit++] = posisub;
04368             posit = olw;
04369             *accum = 0;
04370             tepos = AR.TePos;
04371             i = 0;
04372             while ( i >= 0 ) {
04373                 if ( StartBuf[AT.lWorkspace[posit]] ) {
04374                     if ( ( a = PasteTerm(BHEAD i,accum,
04375                         &(StartBuf[AT.lWorkspace[posit]]),1,1) ) == 0 ) goto GenCall;
04376                     AT.lWorkspace[posit] += StartBuf[AT.lWorkspace[posit]];
04377                     i++; posit++;
04378                 }
04379                 else {
04380                     AT.lWorkspace[posit--] = posisub;
04381                     i--;
04382                 }
04383                 if ( i >= power ) {
04384                     termout = AT.WorkPointer = a;
04385                     AT.WorkPointer = (WORD *)(((UBYTE *) (AT.WorkPointer)) + 2*AM.MaxTer);
04386                     if ( AT.WorkPointer > AT.WorkTop ) goto OverWork;
04387                     if ( FiniTerm(BHEAD term,accum,termout,replac,tepos) ) goto GenCall;
04388                     AT.WorkPointer = termout + *termout;
04389                     *AN.RepPoint = 1;
04390                     AR.expchanged = 1;
04391                 }
04392                 if ( dtype > 0 && dtype != MODLOCAL && dtype != MODSUM ) {
04393                     UNLOCK(d->pthreadlockread); }
04394                 if ( ( AS.Balancing && CC->numrhs == 0 ) && ( i > 0 ) && ( id =
04395                     ConditionalGetAvailableThread() ) >= 0 ) {
04396                     if ( BalanceRunThread(BHEAD id,termout,level) < 0 ) goto GenCall;
04397                 }
04398                 else
04399                 #endif
04400                 if ( Generator(BHEAD termout,level) ) goto GenCall;
04401             #ifdef WITHPTHREADS
04402                 if ( dtype > 0 && dtype != MODLOCAL && dtype != MODSUM ) { dtype = 0; break; }
04403             #endif
04404             if ( iscopy == 0 && ( extractbuff != AM.dbufnum ) )
04405                 StartBuf = cbuf[extractbuff].Buffer;
04406             i--; posit--;
04407         }
04408     }
04409     #ifdef WITHPTHREADS
04410         if ( dtype > 0 && dtype != MODLOCAL && dtype != MODSUM ) { UNLOCK(d->pthreadlockread);
04411             dtype = 0; }
04412         #endif

```

```

04409         if ( iscopy ) {
04410             if ( d->nfactors > 1 ) {
04411                 int j;
04412                 for ( j = 0; j < d->nfactors; j++ ) {
04413                     if ( d->factors[j].where ) M_free(d->factors[j].where,"Copy of factor");
04414                 }
04415                 M_free(d->factors,"Dollar factors");
04416             }
04417             M_free(d,"Copy of dollar variable");
04418             d = 0; iscopy = 0;
04419         }
04420         AT.WorkPointer = accum;
04421         AT.lWorkPointer = olw;
04422         AT.posWorkPointer = olpw;
04423     }
04424 }
04425 else { /* Expression from disk */
04426     POSITION StartPos;
04427     LONG position, olpw, opw, comprev, extra;
04428     RENUMBER renumber;
04429     WORD *Freeze, *aa, *dummies;
04430     replac = -replac-1;
04431     power = AN.TeSuOut;
04432     Freeze = AN.Frozen;
04433     if ( Expressions[replac].status == STOREDEXPRESSION ) {
04434         POSITION firstpos;
04435         SETSTARTPOS(firstpos);
04436
04437         /* Note that AT.TMaddr is needed for GetTable just once! */
04438         /*
04439         We need space for the previous term in the compression
04440         This is made available in AR.CompressBuffer, although we may get
04441         problems with this sooner or later. Hence we need to keep
04442         a set of pointers in AR.CompressBuffer
04443         Note that after the last call there has been no use made
04444         of AR.CompressPointer, so it points automatically at its original
04445         position!
04446         */
04447         WantAddPointers(power+1);
04448         comprev = opw = AT.pWorkPointer;
04449         AT.pWorkPointer += power+1;
04450         WantAddPositions(power+1);
04451         position = olpw = AT.posWorkPointer;
04452         AT.posWorkPointer += power + 1;
04453
04454         AT.pWorkSpace[comprev++] = AR.CompressPointer;
04455
04456         for ( i = 0; i < power; i++ ) {
04457             PUTZERO(AT.posWorkSpace[position]); position++;
04458         }
04459         position = olpw;
04460         if ( ( renumber = GetTable(replac,&(AT.posWorkSpace[position]),1) ) == 0 ) goto GenCall;
04461         dummies = AT.WorkPointer;
04462         *dummies++ = AR.CurDum;
04463         AT.WorkPointer += power+2;
04464         accum = AT.WorkPointer;
04465         AT.WorkPointer = (WORD *)(((UBYTE *) (AT.WorkPointer)) + 2*AM.MaxTer);
04466         if ( AT.WorkPointer > AT.WorkTop ) goto OverWork;
04467         aa = AT.WorkPointer;
04468         *accum = 0;
04469         i = 0; StartPos = AT.posWorkSpace[position];
04470         dummies[i] = AR.CurDum;
04471         while ( i >= 0 ) {
04472             skippedfirst:
04473                 AR.CompressPointer = AT.pWorkSpace[comprev-1];
04474                 if ( ( extra = PasteFile(BHEAD i,accum,&(AT.posWorkSpace[position])
04475                     ,&a,renumber,Freeze,replac) ) < 0 ) goto GenCall;
04476                 if ( Expressions[replac].numdummies > 0 ) {
04477                     AR.CurDum = dummies[i] + Expressions[replac].numdummies;
04478                 }
04479                 if ( NOTSTARTPOS(firstpos) ) {
04480                     if ( ISMINPOS(firstpos) || ISEQUALPOS(firstpos,AT.posWorkSpace[position]) ) {
04481                         firstpos = AT.posWorkSpace[position];
04482                     /*
04483                     ADDPOS(AT.posWorkSpace[position],extra * sizeof(WORD));
04484                     */
04485                     goto skippedfirst;
04486                 }
04487             }
04488             if ( extra ) {
04489                 /*
04490                 ADDPOS(AT.posWorkSpace[position],extra * sizeof(WORD));
04491                 */
04492                 i++; AT.posWorkSpace[++position] = StartPos;
04493                 AT.pWorkSpace[comprev++] = AR.CompressPointer;
04494                 dummies[i] = AR.CurDum;
04495             }

```

```

04496         else {
04497             PUTZERO(AT.posWorkSpace[position]); position--; i--;
04498             AR.CurDum = dummies[i];
04499             comprev--;
04500         }
04501         if ( i >= power ) {
04502             termout = AT.WorkPointer = a;
04503             AT.WorkPointer = (WORD *) (((UBYTE *) (AT.WorkPointer)) + 2*AM.MaxTer);
04504             if ( AT.WorkPointer > AT.WorkTop ) goto OverWork;
04505             if ( FiniTerm(BHEAD term,accum,termout,replac,0) ) goto GenCall;
04506             if ( *termout ) {
04507                 AT.WorkPointer = termout + *termout;
04508                 *AN.RepPoint = 1;
04509                 AR.expchanged = 1;
04510 #ifndef WITHPTHREADS
04511                 if ( ( AS.Balancing && CC->numrhs == 0 ) && ( i > 0 ) && ( id =
ConditionalGetAvailableThread() ) >= 0 ) {
04512                     if ( BalanceRunThread(BHEAD id,termout,level) < 0 ) goto GenCall;
04513                 }
04514             }
04515             else
04516 #endif
04517                 if ( Generator(BHEAD termout,level) ) goto GenCall;
04518         }
04519         i--; position--;
04520         AR.CurDum = dummies[i];
04521         comprev--;
04522     }
04523     AT.WorkPointer = aa;
04524 }
04525 AT.WorkPointer = accum;
04526 AT.posWorkPointer = olpw;
04527 AT.pWorkPointer = opw;
04528 /*
04529     Bug fix. See also GetTable
04530 #ifndef WITHPTHREADS
04531     M_free(renumber->symb.lo,"VarSpace");
04532     M_free(renumber,"Renumbr");
04533 #endif
04534 */
04535     if ( renumber->symb.lo != AN.dummyrenumlist )
04536         M_free(renumber->symb.lo,"VarSpace");
04537     M_free(renumber,"Renumbr");
04538 }
04539 else { /* Active expression */
04540     aa = accum = AT.WorkPointer;
04541     if ( ( (WORD *) (((UBYTE *) (AT.WorkPointer)) + 2 * AM.MaxTer + sizeof(WORD)) ) > AT.WorkTop
)
04542         goto OverWork;
04543     *accum++ = -1; AT.WorkPointer++;
04544     if ( DoOnePow(BHEAD term,power,replac,accum,aa,level,Freeze) ) goto GenCall;
04545     AT.WorkPointer = aa;
04546 }
04547 }
04548 }
04549 Return0:
04550     AR.CurDum = DumNow;
04551     AN.RepPoint = RepSto;
04552     CC->numrhs = oldtoprhs;
04553     CC->Pointer = CC->Buffer + oldcpointer;
04554     CCC->numrhs = oldatoprhs;
04555     CCC->Pointer = CCC->Buffer + oldacpointer;
04556     return(0);
04557
04558 GenCall:
04559     if ( AM.tracebackflag ) {
04560         termout = term;
04561         MLOCK(ErrorMessageLock);
04562         AO.OutFill = AO.OutputLine = (UBYTE *) AT.WorkPointer;
04563         AO.OutSkip = 3;
04564         FiniLine();
04565         i = *termout;
04566         while ( --i >= 0 ) {
04567             TalToLine((UWORD) (*termout++));
04568             TokenToLine((UBYTE *) " ");
04569         }
04570         AO.OutSkip = 0;
04571         FiniLine();
04572         MesCall("Generator");
04573         MUNLOCK(ErrorMessageLock);
04574     }
04575     CC->numrhs = oldtoprhs;
04576     CC->Pointer = CC->Buffer + oldcpointer;
04577     CCC->numrhs = oldatoprhs;
04578     CCC->Pointer = CCC->Buffer + oldacpointer;
04579     return(-1);
04580 OverWork:

```

```

04581     CC->numrhs = oldtoprhs;
04582     CC->Pointer = CC->Buffer + oldcpointer;
04583     CCC->numrhs = oldatoprhs;
04584     CCC->Pointer = CCC->Buffer + oldacpointer;
04585     MLOCK(ErrorMessageLock);
04586     MesWork();
04587     MUNLOCK(ErrorMessageLock);
04588     return(-1);
04589 }
04590
04591 /*
04592     #] Generator :
04593     #[ DoOnePow :           WORD DoOnePow(term,power,nexp,accum,aa,level,freeze)
04594 */
04619 #ifdef WITHPTHREADS
04620 char freezestring[] = "freeze<-xxxx";
04621 #endif
04622
04623 WORD DoOnePow(PHEAD WORD *term, WORD power, WORD nexp, WORD * accum,
04624              WORD *aa, WORD level, WORD *freeze)
04625 {
04626     GETBIDENTITY
04627     POSITION oldposition, startposition;
04628     WORD *acc, *termout, fromfreeze = 0;
04629     WORD *oldipointer = AR.CompressPointer;
04630     FILEHANDLE *fi;
04631     WORD type, retval;
04632     WORD oldGetOneFile = AR.GetOneFile;
04633     WORD olddummies = AR.CurDum;
04634     WORD extradummies = Expressions[nexp].numdummies;
04635 /*
04636     The next code is for some tricky debugging. (5-jan-2010 JV)
04637     Normally it should be disabled.
04638 */
04639 /*
04640 #ifdef WITHPTHREADS
04641     if ( freeze ) {
04642         MLOCK(ErrorMessageLock);
04643         if ( AT.identity < 10 ) {
04644             freezestring[8] = '0'+AT.identity;
04645             freezestring[9] = '>';
04646             freezestring[10] = 0;
04647         }
04648         else if ( AT.identity < 100 ) {
04649             freezestring[8] = '0'+AT.identity/10;
04650             freezestring[9] = '0'+AT.identity%10;
04651             freezestring[10] = '>';
04652             freezestring[11] = 0;
04653         }
04654         else {
04655             freezestring[8] = 0;
04656         }
04657         PrintTerm(freeze,freezestring);
04658         MUNLOCK(ErrorMessageLock);
04659     }
04660 #else
04661     if ( freeze ) PrintTerm(freeze,"freeze");
04662 #endif
04663 */
04664     type = Expressions[nexp].status;
04665     if ( type == HIDDENLEXPRESSION || type == HIDDENGEEXPRESSION
04666         || type == DROPHLEXPRESSION || type == DROPHGEEXPRESSION
04667         || type == UNHIDELEXPRESSION || type == UNHIDEGEEXPRESSION ) {
04668         AR.GetOneFile = 2; fi = AR.hidefile;
04669     }
04670     else {
04671         AR.GetOneFile = 0; fi = AR.infile;
04672     }
04673     if ( fi->handle >= 0 ) {
04674         PUTZERO(oldposition);
04675 #ifdef WITHSEEK
04676         LOCK(AS.inputslock);
04677         SeekFile(fi->handle,&oldposition,SEEK_CUR);
04678         UNLOCK(AS.inputslock);
04679 #endif
04680     }
04681     else {
04682         SETBASEPOSITION(oldposition,fi->POfill-fi->PObuffer);
04683     }
04684     if ( freeze && ( Expressions[nexp].bracketinfo != 0 ) ) {
04685         POSITION *brapos;
04686 /*
04687         There is a bracket index
04688         AR.CompressPointer = oldipointer;
04689 */
04690         (*aa)++;
04691         power--;

```

```

04692         if ( ( brapos = FindBracket(nexp,freeze) ) == 0 )
04693             goto EndExpr;
04694         startposition = *brapos;
04695         goto doterms;
04696     }
04697     startposition = AS.OldOnFile[nexp];
04698     retval = GetOneTerm(BHEAD accum,fi,&startposition,0);
04699     if ( retval > 0 ) {          /* Skip prototype */
04700         (*aa)++;
04701         power--;
04702     doterms:
04703         AR.CompressPointer = oldipointer;
04704         for (;;) {
04705             retval = GetOneTerm(BHEAD accum,fi,&startposition,0);
04706             if ( retval <= 0 ) break;
04707         /*
04708             Here should come the code to test for [].
04709         */
04710         if ( freeze ) {
04711             WORD *t, *m, *r, *mstop;
04712             WORD *tset;
04713             t = accum;
04714             m = freeze;
04715             m += *m;
04716             m -= ABS(m[-1]);
04717             mstop = m;
04718             m = freeze + 1;
04719             r = t;
04720             r += *t;
04721             r -= ABS(r[-1]);
04722             t++;
04723             tset = t;
04724             while ( t < r && *t != HAAKJE ) t += t[1];
04725             if ( t >= r ) {
04726                 if ( m < mstop ) {
04727                     if ( fromfreeze ) goto EndExpr;
04728                     goto NextTerm;
04729                 }
04730                 t = tset;
04731             }
04732             else {
04733                 r = tset;
04734                 while ( r < t && m < mstop ) {
04735                     if ( *r == *m ) { m++; r++; }
04736                     else {
04737                         if ( fromfreeze ) goto EndExpr;
04738                         goto NextTerm;
04739                     }
04740                 }
04741                 if ( r < t || m < mstop ) {
04742                     if ( fromfreeze ) goto EndExpr;
04743                     goto NextTerm;
04744                 }
04745             }
04746             fromfreeze = 1;
04747             r = tset;
04748             m = accum;
04749             m += *m;
04750             while ( t < m ) *r++ = *t++;
04751             *accum = WORDDIFF(r,accum);
04752         }
04753         if ( extradummies > 0 ) {
04754             if ( olddummies > AM.IndDum ) {
04755                 MoveDummies(BHEAD accum,olddummies-AM.IndDum);
04756             }
04757             AR.CurDum = olddummies+extradummies;
04758         }
04759         acc = accum;
04760         acc += *acc;
04761         if ( power <= 0 ) {
04762             termout = acc;
04763             AT.WorkPointer = (WORD *)(((UBYTE *) (acc)) + 2*AM.MaxTer);
04764             if ( AT.WorkPointer > AT.WorkTop ) {
04765                 MLOCK(ErrorMessageLock);
04766                 MesWork();
04767                 MUNLOCK(ErrorMessageLock);
04768                 return(-1);
04769             }
04770             if ( FiniTerm(BHEAD term,aa,termout,nexp,0) ) goto PowCall;
04771             if ( *termout ) {
04772                 MarkPolyRatFunDirty(termout)
04773                 PolyFunDirty(BHEAD termout); /*
04774             AT.WorkPointer = termout + *termout;
04775             *AN.RepPoint = 1;
04776             AR.expchanged = 1;
04777             if ( Generator(BHEAD termout,level) ) goto PowCall;
04778         }

```

```

04779         }
04780         else {
04781             if ( acc > AT.WorkTop ) {
04782                 MLOCK(ErrorMessageLock);
04783                 MesWork();
04784                 MUNLOCK(ErrorMessageLock);
04785                 return(-1);
04786             }
04787             if ( DoOnePow(BHEAD term,power,nexp,acc,aa,level,freeze) ) goto PowCall;
04788         }
04789     NextTerm;;
04790     AR.CompressPointer = olddipointer;
04791 }
04792 EndExpr:
04793     (*aa)--;
04794 }
04795     AR.CompressPointer = olddipointer;
04796     if ( fi->handle >= 0 ) {
04797 #ifdef WITHSEEK
04798         LOCK(AS.inputslock);
04799         SeekFile(fi->handle,&oldposition,SEEK_SET);
04800         UNLOCK(AS.inputslock);
04801         if ( ISNEGPOS(oldposition) ) {
04802             MLOCK(ErrorMessageLock);
04803             MesPrint("File error");
04804             goto PowCall2;
04805         }
04806 #endif
04807     }
04808     else {
04809         fi->POfill = fi->PObuffer + BASEPOSITION(oldposition);
04810     }
04811     AR.GetOneFile = oldGetOneFile;
04812     AR.CurDum = olddummies;
04813     return(0);
04814 PowCall;;
04815     MLOCK(ErrorMessageLock);
04816 #ifdef WITHSEEK
04817 PowCall2;;
04818 #endif
04819     MesCall("DoOnePow");
04820     MUNLOCK(ErrorMessageLock);
04821     SETERROR(-1)
04822 }
04823
04824 /*
04825     #] DoOnePow :
04826     #[ Deferred :          WORD Deferred(term,level)
04827 */
04844 WORD Deferred(PHEAD WORD *term, WORD level)
04845 {
04846     GETBIDENTITY
04847     POSITION startposition;
04848     WORD *t, *m, *mstop, *tstart, *tstop, *termout, i, *oldwork, retval;
04849     WORD *olddipointer = AR.CompressPointer, *oldPOfill = AR.infile->POfill;
04850     WORD oldGetOneFile = AR.GetOneFile;
04851     AR.GetOneFile = 1;
04852     oldwork = AT.WorkPointer;
04853     AT.WorkPointer = (WORD *) ((UBYTE *) (AT.WorkPointer)) + AM.MaxTer;
04854     termout = AT.WorkPointer;
04855     AR.DeferFlag = 0;
04856     startposition = AR.DefPosition;
04857 /*
04858     Store old position
04859 */
04860     if ( AR.infile->handle >= 0 ) {
04861 /*
04862         PUTZERO(oldposition);
04863         SeekFile(AR.infile->handle,&oldposition,SEEK_CUR);
04864 */
04865     }
04866     else {
04867 /*
04868         SETBASEPOSITION(oldposition,AR.infile->POfill-AR.infile->PObuffer);
04869 */
04870         AR.infile->POfill = (WORD *) ((UBYTE *) (AR.infile->PObuffer)
04871             +BASEPOSITION(startposition));
04872     }
04873 /*
04874     Look in the CompressBuffer where the bracket contents start
04875 */
04876     t = m = AR.CompressBuffer;
04877     t += *t;
04878     mstop = t - ABS(t[-1]);
04879     m++;
04880     while ( *m != HAAKJE && m < mstop ) m += m[1];
04881     if ( m >= mstop ) { /* No deferred action! */

```

```

04882     AT.WorkPointer = term + *term;
04883     if ( Generator(BHEAD term,level) ) goto DefCall;
04884     AR.DeferFlag = 1;
04885     AT.WorkPointer = oldwork;
04886     AR.GetOneFile = oldGetOneFile;
04887     return(0);
04888 }
04889 mstop = m + m[1];
04890 decr = WORDDIFF(mstop,AR.CompressBuffer)-1;
04891 tstart = AR.CompressPointer + decr;
04892
04893 m = AR.CompressBuffer;
04894 t = AR.CompressPointer;
04895 i = *m;
04896 NCOPY(t,m,i);
04897 oldb = *tstart;
04898 AR.TePos = 0;
04899 AN.TeSuOut = 0;
04900 /*
04901     Status:
04902     First bracket content starts at mstop.
04903     Next term starts at startposition.
04904     Decompression information is in AR.CompressPointer.
04905     The outside of the bracket runs from AR.CompressBuffer+1 to mstop.
04906 */
04907 for(;;) {
04908     *tstart = *(AR.CompressPointer)-decr;
04909     AR.CompressPointer = AR.CompressPointer+AR.CompressPointer[0];
04910     if ( InsertTerm(BHEAD term,0,AM.rbufnum,tstart,termout,0) < 0 ) {
04911         goto DefCall;
04912     }
04913     *tstart = oldb;
04914     AT.WorkPointer = termout + *termout;
04915     if ( Generator(BHEAD termout,level) ) goto DefCall;
04916     AR.CompressPointer = oldpointer;
04917     AT.WorkPointer = termout;
04918     retval = GetOneTerm(BHEAD AT.WorkPointer,AR.infile,&startposition,0);
04919     if ( retval >= 0 ) AR.CompressPointer = oldpointer;
04920     if ( retval <= 0 ) break;
04921     t = AR.CompressPointer;
04922     if ( *t < (1 + decr + ABS(*(t+*t-1))) ) break;
04923     t++;
04924     m = AR.CompressBuffer+1;
04925     while ( m < mstop ) {
04926         if ( *m != *t ) goto Thatsit;
04927         m++; t++;
04928     }
04929 }
04930 Thatsit::;
04931 /*
04932     Finished. Reposition the file, restore information and return.
04933 */
04934 if ( AR.infile->handle < 0 ) AR.infile->POfill = oldPOfill;
04935 AR.DeferFlag = 1;
04936 AR.GetOneFile = oldGetOneFile;
04937 AT.WorkPointer = oldwork;
04938 return(0);
04939 DefCall::;
04940 MLOCK(ErrorMessageLock);
04941 MesCall("Deferred");
04942 MUNLOCK(ErrorMessageLock);
04943 SETERROR(-1)
04944 }
04945
04946 /*
04947     #] Deferred :
04948     #[ PrepPoly :      WORD PrepPoly(term,par)
04949 */
04972 WORD PrepPoly(PHEAD WORD *term,WORD par)
04973 {
04974     GETBIDENTITY
04975     WORD count = 0, i, jcoef, ncoef;
04976     WORD *t, *m, *r, *tstop, *poly = 0, *v, *w, *vv, *ww;
04977     WORD *oldworkpointer = AT.WorkPointer;
04978 /*
04979     The problem here is that the function will be forced into 'long'
04980     notation. After this -SNUMBER,1 becomes 6,0,4,1,1,3 and the
04981     pattern matcher cannot match a short 1 with a long 1.
04982     But because this is an undocumented feature for very special
04983     purposes, we don't do anything about it. (30-aug-2011)
04984 */
04985     if ( AR.PolyFunType == 2 && AR.PolyFunExp != 2 ) {
04986         WORD oldtype = AR.SortType;
04987         AR.SortType = SORTHIGHFIRST;
04988         if ( poly_ratfun_normalize(BHEAD term) != 0 ) Terminate(-1);
04989 /*
04990         if ( ReadPolyRatFun(BHEAD term) != 0 ) Terminate(-1); */
04990         oldworkpointer = AT.WorkPointer;

```



```

04991     AR.SortType = oldtype;
04992 }
04993 AT.PolyAct = 0;
04994 t = term;
04995 GETSTOP(t,tstop);
04996 t++;
04997 while ( t < tstop ) {
04998     if ( *t == AR.PolyFun ) {
04999         if ( count > 0 ) return(0);
05000         poly = t;
05001         count++;
05002     }
05003     t += t[1];
05004 }
05005 r = m = term + *term;
05006 i = ABS(m[-1]);
05007 if ( par > 0 ) {
05008     if ( count == 0 ) return(0);
05009     else if ( AR.PolyFunType == 1 || (AR.PolyFunType == 2 && AR.PolyFunExp == 2) )
05010         goto DoOne;
05011     else if ( AR.PolyFunType == 2 )
05012         goto DoTwo;
05013     else
05014         goto DoError;
05015 }
05016 else if ( count == 0 ) {
05017     /*
05018     #! Create a PolyFun :
05019     */
05020     poly = t = tstop;
05021     if ( i == 3 && m[-2] == 1 && (m[-3]&MAXPOSITIVE) == m[-3] ) {
05022         *m++ = AR.PolyFun;
05023         if ( AR.PolyFunType == 1 || (AR.PolyFunType == 2 && AR.PolyFunExp == 2) ) {
05024             *m++ = FUNHEAD+2;
05025             FILLFUN(m)
05026             *m++ = -SNUMBER;
05027             *m = m[-2-FUNHEAD] < 0 ? -m[-4-FUNHEAD]: m[-4-FUNHEAD];
05028             m++;
05029         }
05030         else if ( AR.PolyFunType == 2 ) {
05031             *m++ = FUNHEAD+4;
05032             FILLFUN(m)
05033             *m++ = -SNUMBER;
05034             *m = m[-2-FUNHEAD] < 0 ? -m[-4-FUNHEAD]: m[-4-FUNHEAD];
05035             m++;
05036             *m++ = -SNUMBER;
05037             *m++ = 1;
05038         }
05039     }
05040     else {
05041         WORD *vm;
05042         r = tstop;
05043         if ( AR.PolyFunType == 1 || (AR.PolyFunType == 2 && AR.PolyFunExp == 2) ) {
05044             *m++ = AR.PolyFun;
05045             *m++ = FUNHEAD+ARGHEAD+i+1;
05046             FILLFUN(m)
05047             *m++ = ARGHEAD+i+1;
05048             *m++ = 0;
05049             FILLARG(m)
05050             *m++ = i+1;
05051             NCOPY(m,r,i);
05052         }
05053         else if ( AR.PolyFunType == 2 ) {
05054             WORD *num, *den, size, sign, sizenum, sizeden;
05055             if ( m[-1] < 0 ) { sign = -1; size = -m[-1]; }
05056             else { sign = 1; size = m[-1]; }
05057             num = m - size; size = (size-1)/2; den = num + size;
05058             sizenum = size; while ( num[sizenum-1] == 0 ) sizenum--;
05059             sizeden = size; while ( den[sizeden-1] == 0 ) sizeden--;
05060             v = m;
05061             AT.PolyAct = WORDDIF(v,term);
05062             *v++ = AR.PolyFun;
05063             v++;
05064             FILLFUN(v);
05065             vm = v;
05066             *v++ = ARGHEAD+2*sizenum+2;
05067             *v++ = 0;
05068             FILLARG(v);
05069             *v++ = 2*sizenum+2;
05070             for ( i = 0; i < sizenum; i++ ) *v++ = num[i];
05071             *v++ = 1;
05072             for ( i = 1; i < sizenum; i++ ) *v++ = 0;
05073             *v++ = sign*(2*sizenum+1);
05074             if ( ToFast(vm,vm) ) v = vm+2;
05075             vm = v;
05076             *v++ = ARGHEAD+2*sizeden+2;
05077             *v++ = 0;

```

```

05078         FILLARG(v);
05079         *v++ = 2*sizeden+2;
05080         for ( i = 0; i < sizeden; i++ ) *v++ = den[i];
05081         *v++ = 1;
05082         for ( i = 1; i < sizeden; i++ ) *v++ = 0;
05083         *v++ = 2*sizeden+1;
05084         if ( ToFast(vm,vm) ) v = vm+2;
05085         i = v-m;
05086         m[1] = i;
05087         w = num;
05088         NCOPY(w,m,i);
05089         *w++ = 1; *w++ = 1; *w++ = 3; *term = w - term;
05090         return(0);
05091     }
05092 }
05093 /*
05094     #] Create a PolyFun :
05095 */
05096 }
05097 else if ( AR.PolyFunType == 1 || (AR.PolyFunType == 2 && AR.PolyFunExp == 2) ) {
05098     DoOne;;
05099 /*
05100     #[ One argument :
05101 */
05102     m = term + *term;
05103     r = poly + poly[1];
05104     if ( ( poly[1] == FUNHEAD+2 && poly[FUNHEAD+1] == 0
05105         && poly[FUNHEAD] == -SNUMBER ) || poly[1] == FUNHEAD ) return(1);
05106     t = poly + FUNHEAD;
05107     if ( t >= r ) return(0);
05108     if ( m[-1] == 3 && *tstop == 1 && tstop[1] == 1 ) {
05109         i = poly[1];
05110         t = poly;
05111         NCOPY(m,t,i);
05112     }
05113     else if ( *t <= -FUNCTION ) {
05114         if ( t+1 < r ) return(0); /* More than one argument */
05115         r = tstop;
05116         *m++ = AR.PolyFun;
05117         *m++ = FUNHEAD*2+ARGHEAD+i+1;
05118         FILLFUN(m)
05119         *m++ = FUNHEAD+ARGHEAD+i+1;
05120         *m++ = 0;
05121         FILLARG(m)
05122         *m++ = FUNHEAD+i+1;
05123         *m++ = -*t++;
05124         *m++ = FUNHEAD;
05125         FILLFUN(m)
05126         NCOPY(m,r,i);
05127     }
05128     else if ( *t < 0 ) {
05129         if ( t+2 < r ) return(0); /* More than one argument */
05130         r = tstop;
05131         if ( *t == -SNUMBER ) {
05132             if ( t[1] == 0 ) return(1); /* Term should be zero now */
05133             *m = AR.PolyFun;
05134             w = m+1;
05135             m += FUNHEAD+ARGHEAD;
05136             v = m;
05137             *m++ = 5+i;
05138             *m++ = SNUMBER;
05139             *m++ = 4;
05140             *m++ = t[1];
05141             *m++ = 1;
05142             NCOPY(m,r,i);
05143             if ( m >= AT.WorkSpace && m < AT.WorkTop )
05144                 AT.WorkPointer = m;
05145             if ( Normalize(BHEAD v) ) Terminate(-1);
05146             AT.WorkPointer = oldworkpointer;
05147             m = w;
05148             if ( *v == 4 && v[2] == 1 && (v[1]&MAXPOSITIVE) == v[1] ) {
05149                 *m++ = FUNHEAD+2;
05150                 FILLFUN(m)
05151                 *m++ = -SNUMBER;
05152                 *m++ = v[3] < 0 ? -v[1] : v[1];
05153             }
05154             else if ( *v == 0 ) return(1);
05155             else {
05156                 *m++ = FUNHEAD+ARGHEAD+*v;
05157                 FILLFUN(m)
05158                 *m++ = ARGHEAD+*v;
05159                 *m++ = 0;
05160                 FILLARG(m)
05161                 m = v + *v;
05162             }
05163         }
05164     }
05165     else if ( *t == -SYMBOL ) {

```

```

05165         *m++ = AR.PolyFun;
05166         *m++ = FUNHEAD+ARGHEAD+5+i;
05167         FILLFUN(m)
05168         *m++ = ARGHEAD+5+i;
05169         *m++ = 0;
05170         FILLARG(m)
05171         *m++ = 5+i;
05172         *m++ = SYMBOL;
05173         *m++ = 4;
05174         *m++ = t[1];
05175         *m++ = 1;
05176         NCOPY(m,r,i);
05177     }
05178     else return(0);          /* Not symbol-like */
05179 }
05180 else {
05181     if ( t + *t < r ) return(0); /* More than one argument */
05182     i = m[-1];
05183     *m++ = AR.PolyFun;
05184     w = m;
05185     m += ARGHEAD+FUNHEAD-1;
05186     t += ARGHEAD;
05187     jcoef = i < 0 ? (i+1)>>1:(i-1)>>1;
05188     v = t;
05189 /*
05190     Test now the scalar nature of the argument.
05191     No indices allowed.
05192 */
05193     while ( t < r ) {
05194         WORD *vstop;
05195         vv = t + *t;
05196         vstop = vv - ABS(vv[-1]);
05197         t++;
05198         while( t < vstop ) {
05199             if ( *t == INDEX ) return(0);
05200             t += t[1];
05201         }
05202         t = vv;
05203     }
05204 /*
05205     Now multiply each term by the coefficient.
05206 */
05207     t = v;
05208     while ( t < r ) {
05209         ww = m;
05210         v = t + *t;
05211         ncoef = v[-1];
05212         vv = v - ABS(ncoef);
05213         if ( ncoef < 0 ) ncoef++;
05214         else ncoef--;
05215         ncoef >= 1;
05216         while ( t < vv ) *m++ = *t++;
05217         if ( MulRat(BHEAD (UWORD *)vv,ncoef, (UWORD *)tstop,jcoef,
05218             (UWORD *)m,&ncoef) ) Terminate(-1);
05219         ncoef *= 2;
05220         m += ABS(ncoef);
05221         if ( ncoef < 0 ) ncoef--;
05222         else ncoef++;
05223         *m++ = ncoef;
05224         *ww = WORDDIF(m,ww);
05225         if ( AN.ncmod != 0 ) {
05226             if ( Modulus(ww) ) Terminate(-1);
05227             if ( *ww == 0 ) return(1);
05228             m = ww + *ww;
05229         }
05230         t = v;
05231     }
05232     *w = (WORDDIF(m,w))+1;
05233     w[FUNHEAD-1] = w[0] - FUNHEAD;
05234     w[FUNHEAD] = 0;
05235     w[1] = 0; /* omission survived for years. 23-mar-2006 JV */
05236     w += FUNHEAD-1;
05237     if ( ToFast(w,w) ) {
05238         if ( *w <= -FUNCTION ) { w[-FUNHEAD+1] = FUNHEAD+1; m = w+1; }
05239         else { w[-FUNHEAD+1] = FUNHEAD+2; m = w+2; }
05240     }
05241 }
05242 }
05243 t = poly + poly[1];
05244 while ( t < tstop ) *poly++ = *t++;
05245 /*
05246     #] One argument :
05247 */
05248 }
05249 else if ( AR.PolyFunType == 2 ) {
05250     DoTwo;;
05251 /*

```

```

05252      #] Two arguments :
05253  /*
05254      WORD *num, *den, size, sign, sizenum, sizeden;
05255  /*
05256      First make sure that the PolyFun is last
05257  /*
05258      m = term + *term;
05259      if ( poly + poly[1] < tstop ) {
05260          for ( i = 0; i < poly[1]; i++ ) m[i] = poly[i];
05261          t = poly; v = poly + poly[1];
05262          while ( v < tstop ) *t++ = *v++;
05263          poly = t;
05264          for ( i = 0; i < m[1]; i++ ) t[i] = m[i];
05265          t += m[1];
05266      }
05267      AT.PolyAct = WORDDIF(poly,term);
05268  /*
05269      If needed we convert the coefficient into a PolyRatFun and then
05270      we call poly_ratfun_normalize
05271  /*
05272      if ( m[-1] == 3 && m[-2] == 1 && m[-3] == 1 ) return(0);
05273      if ( AR.PolyFunExp != 1 ) {
05274          if ( m[-1] < 0 ) { sign = -1; size = -m[-1]; } else { sign = 1; size = m[-1]; }
05275          num = m - size; size = (size-1)/2; den = num + size;
05276          sizenum = size; while ( num[sizenum-1] == 0 ) sizenum--;
05277          sizeden = size; while ( den[sizeden-1] == 0 ) sizeden--;
05278          v = m;
05279          *v++ = AR.PolyFun;
05280          *v++ = FUNHEAD + 2*(ARGHEAD+sizenum+sizeden+2);
05281  /*
05282          *v++ = MUSTCLEANPRF; */
05283          *v++ = 0;
05284          FILLFUN3(v);
05285          *v++ = ARGHEAD+2*sizenum+2;
05286          *v++ = 0;
05287          FILLARG(v);
05288          *v++ = 2*sizenum+2;
05289          for ( i = 0; i < sizenum; i++ ) *v++ = num[i];
05290          *v++ = 1;
05291          for ( i = 1; i < sizenum; i++ ) *v++ = 0;
05292          *v++ = sign*(2*sizenum+1);
05293          *v++ = ARGHEAD+2*sizeden+2;
05294          *v++ = 0;
05295          FILLARG(v);
05296          *v++ = 2*sizeden+2;
05297          for ( i = 0; i < sizeden; i++ ) *v++ = den[i];
05298          *v++ = 1;
05299          for ( i = 1; i < sizeden; i++ ) *v++ = 0;
05300          *v++ = 2*sizeden+1;
05301          w = num;
05302          i = v - m;
05303          NCOPY(w,m,i);
05304      }
05305      else {
05306          w = m-ABS(m[-1]);
05307      }
05308      *w++ = 1; *w++ = 1; *w++ = 3; *term = w - term;
05309      {
05310          WORD oldtype = AR.SortType;
05311          AR.SortType = SORTHIGHFIRST;
05312  /*
05313          if ( count > 0 )
05314              poly_ratfun_normalize(BHEAD term);
05315          else
05316              ReadPolyRatFun(BHEAD term);
05317  /*
05318          poly_ratfun_normalize(BHEAD term);
05319  /*
05320          oldworkpointer = AT.WorkPointer; */
05321          AR.SortType = oldtype;
05322      }
05323  /*
05324      #] Two arguments :
05325  /*
05326      }
05327      else {
05328          DoError;;
05329          MLOCK(ErrorMessageLock);
05330          MesPrint("Illegal value for PolyFunType in PrepPoly");
05331          MUNLOCK(ErrorMessageLock);
05332          Terminate(-1);
05333      }
05334      r = term + *term;
05335      AT.PolyAct = WORDDIF(poly,term);
05336      while ( r < m ) *poly++ = *r++;
05337      *poly++ = 1;
05338      *poly++ = 1;

```

```

05339     *poly++ = 3;
05340     *term = WORDDIFF(poly,term);
05341 endofit;;
05342     return(0);
05343 }
05344
05345 /*
05346     #] PrepPoly :
05347     #[ PolyFunMul :          WORD PolyFunMul(term)
05348 */
05360 WORD PolyFunMul(PHEAD WORD *term)
05361 {
05362     GETBIDENTITY
05363     WORD *t, *fun1, *fun2, *t1, *t2, *m, *w, *ww, *tt1, *tt2, *tt4, *arg1, *arg2;
05364     WORD *tstop, i, dirty = 0, OldPolyFunPow = AR.PolyFunPow, minp1, minp2;
05365     WORD n1, n2, i1, i2, l1, l2, l3, l4, action = 0, noac = 0, retval = 0;
05366     if ( AR.PolyFunType == 2 && AR.PolyFunExp == 1 ) {
05367         WORD pow = 0, pow1;
05368         t = term + 1; t1 = term + *term; t1 -= ABS(t1[-1]);
05369         w = t;
05370         while ( t < t1 ) {
05371             if ( *t != AR.PolyFun ) {
05372 SkipFun:
05373                 if ( t == w ) { t += t[1]; w = t; }
05374                 else { i = t[1]; NCOPY(w,t,i) }
05375                 continue;
05376             }
05377             pow1 = 0;
05378             t2 = t + t[1]; t += FUNHEAD;
05379             if ( *t < 0 ) {
05380                 if ( *t == -SYMBOL && t[1] == AR.PolyFunVar ) pow1++;
05381                 else if ( *t != -SNUMBER ) goto NoLegal;
05382                 t += 2;
05383             }
05384             else if ( t[0] == ARGHEAD+8 && t[ARGHEAD] == 8
05385                     && t[ARGHEAD+1] == SYMBOL && t[ARGHEAD+3] == AR.PolyFunVar
05386                     && t[ARGHEAD+5] == 1 && t[ARGHEAD+6] == 1 && t[ARGHEAD+7] == 3 ) {
05387                 pow1 += t[ARGHEAD+4];
05388                 t += *t;
05389             }
05390             else {
05391 NoLegal:
05392                 MLOCK(ErrorMessageLock);
05393                 MesPrint("Illegal term with divergence in PolyRatFun");
05394                 MesCall("PolyFunMul");
05395                 MUNLOCK(ErrorMessageLock);
05396                 Terminate(-1);
05397             }
05398             if ( *t < 0 ) {
05399                 if ( *t == -SYMBOL && t[1] == AR.PolyFunVar ) pow1--;
05400                 else if ( *t != -SNUMBER ) goto NoLegal;
05401                 t += 2;
05402             }
05403             else if ( t[0] == ARGHEAD+8 && t[ARGHEAD] == 8
05404                     && t[ARGHEAD+1] == SYMBOL && t[ARGHEAD+3] == AR.PolyFunVar
05405                     && t[ARGHEAD+5] == 1 && t[ARGHEAD+6] == 1 && t[ARGHEAD+7] == 3 ) {
05406                 pow1 -= t[ARGHEAD+4];
05407                 t += *t;
05408             }
05409             else goto NoLegal;
05410             if ( t == t2 ) pow += pow1;
05411             else goto SkipFun;
05412         }
05413         m = w;
05414         *w++ = AR.PolyFun; *w++ = 0; FILLFUN(w);
05415         if ( pow > 1 ) {
05416             *w++ = 8+ARGHEAD; *w++ = 0; FILLARG(w);
05417             *w++ = 8; *w++ = SYMBOL; *w++ = 4; *w++ = AR.PolyFunVar; *w++ = pow;
05418             *w++ = 1; *w++ = 1; *w++ = 3; *w++ = -SNUMBER; *w++ = 1;
05419         }
05420         else if ( pow == 1 ) {
05421             *w++ = -SYMBOL; *w++ = AR.PolyFunVar; *w++ = -SNUMBER; *w++ = 1;
05422         }
05423         else if ( pow < -1 ) {
05424             *w++ = -SNUMBER; *w++ = 1; *w++ = 8+ARGHEAD; *w++ = 0; FILLARG(w);
05425             *w++ = 8; *w++ = SYMBOL; *w++ = 4; *w++ = AR.PolyFunVar; *w++ = -pow;
05426             *w++ = 1; *w++ = 1; *w++ = 3;
05427         }
05428         else if ( pow == -1 ) {
05429             *w++ = -SNUMBER; *w++ = 1; *w++ = -SYMBOL; *w++ = AR.PolyFunVar;
05430         }
05431         else {
05432             *w++ = -SNUMBER; *w++ = 1; *w++ = -SNUMBER; *w++ = 1;
05433         }
05434         m[1] = w - m;
05435         *w++ = 1; *w++ = 1; *w++ = 3;
05436         *term = w - term;

```

```

05437         if ( w > AT.WorkSpace && w < AT.WorkTop ) AT.WorkPointer = w;
05438         return(0);
05439     }
05440 ReStart:
05441     if ( AR.PolyFunType == 2 && ( ( AR.PolyFunExp != 2 )
05442         || ( AR.PolyFunExp == 2 && AN.PolyNormFlag > 1 ) ) ) {
05443         WORD count1 = 0, count2 = 0, count3;
05444         WORD oldtype = AR.SortType;
05445         t = term + 1; t1 = term + *term; t1 -= ABS(t1[-1]);
05446         while ( t < t1 ) {
05447             if ( *t == AR.PolyFun ) {
05448                 if ( t[2] && dirty == 0 ) { /* Any dirty flag on? */
05449                     dirty = 1;
05450                     /* ReadPolyRatFun(BHEAD term); */
05451                     /* ToPolyFunGeneral(BHEAD term); */
05452                     poly_ratfun_normalize(BHEAD term);
05453                     if ( term[0] == 0 ) return(0);
05454                     count1 = 0;
05455                     action++;
05456                     goto ReStart;
05457                 }
05458                 t2 = t + t[1]; tt2 = t+FUNHEAD; count3 = 0;
05459                 while ( tt2 < t2 ) { count3++; NEXTARG(tt2); }
05460                 if ( count3 == 2 ) {
05461                     count1++;
05462                     if ( ( t[2] & MUSTCLEANPRF ) != 0 ) { /* Better civilize this guy */
05463                         action++;
05464                         w = AT.WorkPointer;
05465                         AR.SortType = SORTHIGHFIRST;
05466                         t2 = t + t[1]; tt2 = t+FUNHEAD;
05467                         while ( tt2 < t2 ) {
05468                             if ( *tt2 > 0 ) {
05469                                 tt4 = tt2; tt1 = tt2 + ARGHEAD; tt2 += *tt2;
05470                                 NewSort(BHEAD0);
05471                                 while ( tt1 < tt2 ) {
05472                                     i = *tt1; ww = w; NCOPY(ww,tt1,i);
05473                                     AT.WorkPointer = ww;
05474                                     Normalize(BHEAD w);
05475                                     StoreTerm(BHEAD w);
05476                                 }
05477                                 EndSort(BHEAD w,1);
05478                                 ww = w; while ( *ww ) ww += *ww;
05479                                 if ( ww-w != *tt4-ARGHEAD ) { /* Little problem */
05480                                     /*
05481                                     Solution: brute force copy
05482                                     Maybe it will never come here????
05483                                     */
05484                                     WORD *r1 = TermMalloc("PolyFunMul");
05485                                     WORD ii = (ww-w)-(*tt4-ARGHEAD); /* increment */
05486                                     WORD *r2 = tt4+ARGHEAD, *r3, *r4 = r1;
05487                                     i = r2 - term; r3 = term; NCOPY(r4,r3,i);
05488                                     i = ww-w; ww = w; NCOPY(r4,ww,i);
05489                                     r3 = tt2; i = term+*term-tt2; NCOPY(r4,r3,i);
05490                                     *r1 = i = r4-r1; r4 = term; r3 = r1;
05491                                     NCOPY(r4,r3,i);
05492                                     t[1] += ii; t1 += ii; *tt4 += ii;
05493                                     tt2 = tt4 + *tt4;
05494                                     TermFree(r1,"PolyFunMul");
05495                                 }
05496                                 else {
05497                                     i = ww-w; ww = w; tt1 = tt4+ARGHEAD;
05498                                     NCOPY(tt1,ww,i);
05499                                     AT.WorkPointer = w;
05500                                 }
05501                             }
05502                             else if ( *tt2 <= -FUNCTION ) tt2++;
05503                             else tt2 += 2;
05504                         }
05505                         AR.SortType = oldtype;
05506                     }
05507                 }
05508             }
05509             t += t[1];
05510         }
05511         if ( count1 <= 1 ) { goto checkaction; }
05512         if ( AR.PolyFunExp == 1 ) {
05513             t = term + *term; t -= ABS(t[-1]);
05514             *t++ = 1; *t++ = 1; *t++ = 3; *term = t - term;
05515         }
05516         {
05517             AR.SortType = SORTHIGHFIRST;
05518             /* retval = ReadPolyRatFun(BHEAD term); */
05519             /* ToPolyFunGeneral(BHEAD term); */
05520             retval = poly_ratfun_normalize(BHEAD term);
05521             if ( *term == 0 ) return(retval);
05522             AR.SortType = oldtype;
05523         }

```

```

05524     t = term + 1; t1 = term + *term; t1 -= ABS(t1[-1]);
05525     while ( t < t1 ) {
05526         if ( *t == AR.PolyFun ) {
05527             t2 = t + t[1]; tt2 = t+FUNHEAD; count3 = 0;
05528             while ( tt2 < t2 ) { count3++; NEXTARG(tt2); }
05529             if ( count3 == 2 ) {
05530                 count2++;
05531             }
05532         }
05533     }
05534     t += t[1];
05535 }
05536 if ( count1 >= count2 ) {
05537     t = term + 1;
05538     while ( t < t1 ) {
05539         if ( *t == AR.PolyFun ) {
05540             t2 = t;
05541             t = t + t[1];
05542             t2[2] |= (DIRTYFLAG|MUSTCLEANPRF);
05543             t2 += FUNHEAD;
05544             while ( t2 < t ) {
05545                 if ( *t2 > 0 ) t2[1] = DIRTYFLAG;
05546                 NEXTARG(t2);
05547             }
05548         }
05549         else t += t[1];
05550     }
05551 }
05552
05553 w = term + *term;
05554 if ( w > AT.WorkSpace && w < AT.WorkTop ) AT.WorkPointer = w;
05555 checkaction:
05556 if ( action ) retval = action;
05557 return(retval);
05558 }
05559 retry:
05560 if ( term >= AT.WorkSpace && term+*term < AT.WorkTop )
05561     AT.WorkPointer = term + *term;
05562 GETSTOP(term,tstop);
05563 t = term+1;
05564 while ( *t != AR.PolyFun && t < tstop ) t += t[1];
05565 while ( t < tstop && *t == AR.PolyFun ) {
05566     if ( t[1] > FUNHEAD ) {
05567         if ( t[FUNHEAD] < 0 ) {
05568             if ( t[FUNHEAD] <= -FUNCTION && t[1] == FUNHEAD+1 ) break;
05569             if ( t[FUNHEAD] > -FUNCTION && t[1] == FUNHEAD+2 ) {
05570                 if ( t[FUNHEAD] == -SNUMBER && t[FUNHEAD+1] == 0 ) {
05571                     *term = 0;
05572                     return(0);
05573                 }
05574                 break;
05575             }
05576         }
05577         else if ( t[FUNHEAD] == t[1] - FUNHEAD ) break;
05578     }
05579     noac = 1;
05580     t += t[1];
05581 }
05582 if ( *t != AR.PolyFun || t >= tstop ) goto done;
05583 fun1 = t;
05584 t += t[1];
05585 while ( t < tstop && *t == AR.PolyFun ) {
05586     if ( t[1] > FUNHEAD ) {
05587         if ( t[FUNHEAD] < 0 ) {
05588             if ( t[FUNHEAD] <= -FUNCTION && t[1] == FUNHEAD+1 ) break;
05589             if ( t[FUNHEAD] > -FUNCTION && t[1] == FUNHEAD+2 ) {
05590                 if ( t[FUNHEAD] == -SNUMBER && t[FUNHEAD+1] == 0 ) {
05591                     *term = 0;
05592                     return(0);
05593                 }
05594                 break;
05595             }
05596         }
05597         else if ( t[FUNHEAD] == t[1] - FUNHEAD ) break;
05598     }
05599     noac = 1;
05600     t += t[1];
05601 }
05602 if ( *t != AR.PolyFun || t >= tstop ) goto done;
05603 fun2 = t;
05604 /*
05605 We have two functions of the proper type.
05606 Count terms (needed for the specials)
05607 */
05608 t = fun1 + FUNHEAD;
05609 if ( *t < 0 ) {
05610     n1 = 1; arg1 = AT.WorkPointer;

```

```

05611         ToGeneral(t,arg1,1);
05612         AT.WorkPointer = arg1 + *arg1;
05613     }
05614     else {
05615         t += ARGHEAD;
05616         n1 = 0; t1 = fun1 + fun1[1]; arg1 = t;
05617         while ( t < t1 ) { n1++; t += *t; }
05618     }
05619     t = fun2 + FUNHEAD;
05620     if ( *t < 0 ) {
05621         n2 = 1; arg2 = AT.WorkPointer;
05622         ToGeneral(t,arg2,1);
05623         AT.WorkPointer = arg2 + *arg2;
05624     }
05625     else {
05626         t += ARGHEAD;
05627         n2 = 0; t2 = fun2 + fun2[1]; arg2 = t;
05628         while ( t < t2 ) { n2++; t += *t; }
05629     }
05630 /*
05631     Now we can start the multiplications. We first multiply the terms
05632     without coefficients, then normalize, and finally put the coefficients
05633     in place. This is because one has often truncated series and the
05634     high powers may get killed, while their coefficients are the most
05635     expensive ones.
05636     Note: We may run into fun(-SNUMBER,value)
05637 */
05638     w = AT.WorkPointer;
05639     NewSort(BHEAD0);
05640     if ( AR.PolyFunType == 2 && AR.PolyFunExp == 2 ) {
05641         AT.TrimPower = 1;
05642 /*
05643         We have to find the lowest power in both polynomials.
05644         This will be needed to temporarily correct the AR.PolyFunPow
05645 */
05646         minp1 = MAXPOWER;
05647         for ( t1 = arg1, i1 = 0; i1 < n1; i1++, t1 += *t1 ) {
05648             if ( *t1 == 4 ) {
05649                 if ( minp1 > 0 ) minp1 = 0;
05650             }
05651             else if ( ABS(t1[*t1-1]) == (*t1-1) ) {
05652                 if ( minp1 > 0 ) minp1 = 0;
05653             }
05654             else {
05655                 if ( t1[1] == SYMBOL && t1[2] == 4 && t1[3] == AR.PolyFunVar ) {
05656                     if ( t1[4] < minp1 ) minp1 = t1[4];
05657                 }
05658                 else {
05659                     MesPrint("Illegal term in expanded polyratfun.");
05660                     goto PolyCall;
05661                 }
05662             }
05663         }
05664         minp2 = MAXPOWER;
05665         for ( t2 = arg2, i2 = 0; i2 < n2; i2++, t2 += *t2 ) {
05666             if ( *t2 == 4 ) {
05667                 if ( minp2 > 0 ) minp2 = 0;
05668             }
05669             else if ( ABS(t2[*t2-1]) == (*t2-1) ) {
05670                 if ( minp2 > 0 ) minp2 = 0;
05671             }
05672             else {
05673                 if ( t2[1] == SYMBOL && t2[2] == 4 && t2[3] == AR.PolyFunVar ) {
05674                     if ( t2[4] < minp2 ) minp2 = t2[4];
05675                 }
05676                 else {
05677                     MesPrint("Illegal term in expanded polyratfun.");
05678                     goto PolyCall;
05679                 }
05680             }
05681         }
05682         AR.PolyFunPow += minp1+minp2;
05683     }
05684     for ( t1 = arg1, i1 = 0; i1 < n1; i1++, t1 += *t1 ) {
05685     for ( t2 = arg2, i2 = 0; i2 < n2; i2++, t2 += *t2 ) {
05686         m = w;
05687         m++;
05688         GETSTOP(t1,tt1);
05689         t = t1 + 1;
05690         while ( t < tt1 ) *m++ = *t++;
05691         GETSTOP(t2,tt2);
05692         t = t2+1;
05693         while ( t < tt2 ) *m++ = *t++;
05694         *m++ = 1; *m++ = 1; *m++ = 3; *w = WORDDIF(m,w);
05695         AT.WorkPointer = m;
05696         if ( Normalize(BHEAD w) ) { LowerSortLevel(); goto PolyCall; }
05697         if ( *w ) {

```



```

05698         m = w + *w;
05699         if ( m[-1] != 3 || m[-2] != 1 || m[-3] != 1 ) {
05700             l3 = REDLENG(m[-1]);
05701             m -= ABS(m[-1]);
05702             t = t1 + *t1 - 1;
05703             l1 = REDLENG(*t);
05704             if ( MulRat(BHEAD (UWORD *)m,l3,(UWORD *)t1,l1,(UWORD *)m,&l4) ) {
05705                 LowerSortLevel(); goto PolyCall; }
05706             if ( AN.ncmod != 0 && TakeModulus((UWORD *)m,&l4,AC.cmod,AN.ncmod,UNPACK|AC.modmode) )
{
05707                 LowerSortLevel(); goto PolyCall; }
05708                 if ( l4 == 0 ) continue;
05709                 t = t2 + *t2 - 1;
05710                 l2 = REDLENG(*t);
05711                 if ( MulRat(BHEAD (UWORD *)m,l4,(UWORD *)t2,l2,(UWORD *)m,&l3) ) {
05712                     LowerSortLevel(); goto PolyCall; }
05713                 if ( AN.ncmod != 0 && TakeModulus((UWORD *)m,&l3,AC.cmod,AN.ncmod,UNPACK|AC.modmode) )
{
05714                     LowerSortLevel(); goto PolyCall; }
05715                 }
05716                 else {
05717                     m -= 3;
05718                     t = t1 + *t1 - 1;
05719                     l1 = REDLENG(*t);
05720                     t = t2 + *t2 - 1;
05721                     l2 = REDLENG(*t);
05722                     if ( MulRat(BHEAD (UWORD *)t1,l1,(UWORD *)t2,l2,(UWORD *)m,&l3) ) {
05723                         LowerSortLevel(); goto PolyCall; }
05724                     if ( AN.ncmod != 0 && TakeModulus((UWORD *)m,&l3,AC.cmod,AN.ncmod,UNPACK|AC.modmode) )
{
05725                         LowerSortLevel(); goto PolyCall; }
05726                     }
05727                     if ( l3 == 0 ) continue;
05728                     l3 = INCLENG(l3);
05729                     m += ABS(l3);
05730                     m[-1] = l3;
05731                     *w = WORDDIF(m,w);
05732                     AT.WorkPointer = m;
05733                     if ( StoreTerm(BHEAD w) ) { LowerSortLevel(); goto PolyCall; }
05734                 }
05735             }
05736         }
05737         if ( EndSort(BHEAD w,0) < 0 ) goto PolyCall;
05738         AR.PolyFunPow = OldPolyFunPow;
05739         AT.TrimPower = 0;
05740         if ( *w == 0 ) {
05741             *term = 0;
05742             return(0);
05743         }
05744         t = w;
05745         while ( *t ) t += *t;
05746         AT.WorkPointer = t;
05747         n1 = WORDDIF(t,w);
05748         t1 = term;
05749         while ( t1 < fun1 ) *t++ = *t1++;
05750         t2 = t;
05751         *t++ = AR.PolyFun;
05752         *t++ = FUNHEAD+ARGHEAD+n1;
05753         *t++ = 0;
05754         FILLFUN3(t)
05755         *t++ = ARGHEAD+n1;
05756         *t++ = 0;
05757         FILLARG(t)
05758         NCOPY(t,w,n1);
05759         if ( ToFast(t2+FUNHEAD,t2+FUNHEAD) ) {
05760             if ( t2[FUNHEAD] > -FUNCTION ) t2[1] = FUNHEAD+2;
05761             else t2[FUNHEAD] = FUNHEAD+1;
05762             t = t2 + t2[1];
05763         }
05764         t1 = fun1 + fun1[1];
05765         while ( t1 < fun2 ) *t++ = *t1++;
05766         t1 = fun2 + fun2[1];
05767         t2 = term + *term;
05768         while ( t1 < t2 ) *t++ = *t1++;
05769         *AT.WorkPointer = n1 = WORDDIF(t,AT.WorkPointer);
05770         if ( n1*(LONG)sizeof(WORD) > AM.MaxTer ) {
05771             MLOCK(ErrorMessageLock);
05772             MesPrint("Term too complex (%d words). MaxTermSize (%l words) is too small.", n1,
AM.MaxTer/(LONG)sizeof(WORD) );
05773             goto PolyCall2;
05774         }
05775         m = term; t = AT.WorkPointer;
05776         NCOPY(m,t,n1);
05777         action++;
05778         goto retry;
05779     done:
05780         AT.WorkPointer = term + *term;

```

```

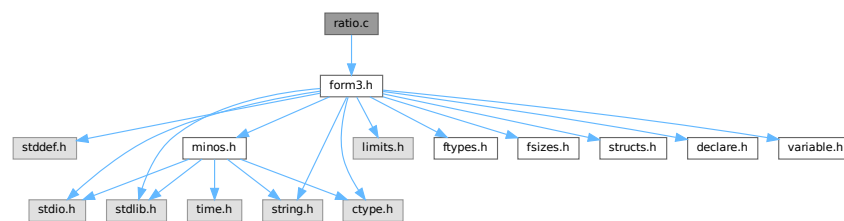
05781     if ( action && noac ) {
05782         if ( Normalize(BHEAD term) ) goto PolyCall;
05783         AT.WorkPointer = term + *term;
05784     }
05785     return(0);
05786 PolyCall;;
05787     MLOCK(ErrorMessageLock);
05788 PolyCall2;;
05789     AR.PolyFunPow = OldPolyFunPow;
05790     MesCall("PolyFunMul");
05791     MUNLOCK(ErrorMessageLock);
05792     SETERROR(-1)
05793 }
05794
05795 /*
05796     #] PolyFunMul :
05797     #] Processor :
05798 */

```

4.116 ratio.c File Reference

```
#include "form3.h"
```

Include dependency graph for ratio.c:



Data Structures

- struct [ARGBUFFER](#)

Functions

- WORD [RatioFind](#) (PHEAD WORD *term, WORD *params)
- WORD [RatioGen](#) (PHEAD WORD *term, WORD *params, WORD num, WORD level)
- WORD [BinomGen](#) (PHEAD WORD *term, WORD level, WORD **tstops, WORD x1, WORD x2, WORD pow1, WORD pow2, WORD sign, UWORD *coef, WORD ncoef)
- WORD [DoSumF1](#) (PHEAD WORD *term, WORD *params, WORD replac, WORD level)
- WORD [Glue](#) (PHEAD WORD *term1, WORD *term2, WORD *sub, WORD insert)
- WORD [DoSumF2](#) (PHEAD WORD *term, WORD *params, WORD replac, WORD level)
- int [GCDfunction](#) (PHEAD WORD *term, WORD level)
- WORD * [GCDfunction3](#) (PHEAD WORD *in1, WORD *in2)
- WORD * [PutExtraSymbols](#) (PHEAD WORD *in, WORD startbuf, int *actionflag)
- WORD * [TakeExtraSymbols](#) (PHEAD WORD *in, WORD startbuf)
- WORD * [MultiplyWithTerm](#) (PHEAD WORD *in, WORD *term, WORD par)
- WORD * [TakeContent](#) (PHEAD WORD *in, WORD *term)
- int [MergeSymbolLists](#) (PHEAD WORD *old, WORD *extra, int par)
- int [MergeDotproductLists](#) (PHEAD WORD *old, WORD *extra, int par)
- WORD * [CreateExpression](#) (PHEAD WORD nex)

- int [GCDterms](#) (PHEAD WORD *term1, WORD *term2, WORD *termout)
- int [ReadPolyRatFun](#) (PHEAD WORD *term)
- int [FromPolyRatFun](#) (PHEAD WORD *fun, WORD **numout, WORD **denout)
- WORD * [TakeSymbolContent](#) (PHEAD WORD *in, WORD *term)
- void [GCDclean](#) (PHEAD WORD *num, WORD *den)
- WORD * [PolyDiv](#) (PHEAD WORD *a, WORD *b, char *text)
- int [DIVfunction](#) (PHEAD WORD *term, WORD level, int par)
- WORD * [MULfunc](#) (PHEAD WORD *p1, WORD *p2)
- WORD * [ConvertArgument](#) (PHEAD WORD *arg, int *type)
- int [ExpandRat](#) (PHEAD WORD *fun)
- int [InvPoly](#) (PHEAD WORD *inpoly, WORD maxpow, WORD sym)

Variables

- WORD [divrem](#) [4] = { DIVFUNCTION, REMFUNCTION, INVERSEFUNCTION, MULFUNCTION }
- char * [TheErrorMessage](#) []

4.116.1 Detailed Description

A variety of routines: The ratio command for partial fractioning (rather old. Schoonschip inheritance) The sum routines.

Definition in file [ratio.c](#).

4.116.2 Function Documentation

4.116.2.1 RatioFind()

```
WORD RatioFind (
    PHEAD WORD * term,
    WORD * params )
```

Definition at line 62 of file [ratio.c](#).

4.116.2.2 RatioGen()

```
WORD RatioGen (
    PHEAD WORD * term,
    WORD * params,
    WORD num,
    WORD level )
```

Definition at line 170 of file [ratio.c](#).

4.116.2.3 BinomGen()

```
WORD BinomGen (
    PHEAD WORD * term,
    WORD level,
    WORD ** tstops,
    WORD x1,
    WORD x2,
    WORD pow1,
    WORD pow2,
    WORD sign,
    UWORD * coef,
    WORD ncoef )
```

Definition at line 320 of file [ratio.c](#).

4.116.2.4 DoSumF1()

```
WORD DoSumF1 (
    PHEAD WORD * term,
    WORD * params,
    WORD replac,
    WORD level )
```

Definition at line 395 of file [ratio.c](#).

4.116.2.5 Glue()

```
WORD Glue (
    PHEAD WORD * term1,
    WORD * term2,
    WORD * sub,
    WORD insert )
```

Definition at line 461 of file [ratio.c](#).

4.116.2.6 DoSumF2()

```
WORD DoSumF2 (
    PHEAD WORD * term,
    WORD * params,
    WORD replac,
    WORD level )
```

Definition at line 520 of file [ratio.c](#).

4.116.2.7 GCDfunction()

```
int GCDfunction (
    PHEAD WORD * term,
    WORD level )
```

Definition at line 609 of file [ratio.c](#).

4.116.2.8 GCDfunction3()

```
WORD * GCDfunction3 (
    PHEAD WORD * in1,
    WORD * in2 )
```

Definition at line 1141 of file [ratio.c](#).

4.116.2.9 PutExtraSymbols()

```
WORD * PutExtraSymbols (
    PHEAD WORD * in,
    WORD startebuf,
    int * actionflag )
```

Definition at line 1244 of file [ratio.c](#).

4.116.2.10 TakeExtraSymbols()

```
WORD * TakeExtraSymbols (
    PHEAD WORD * in,
    WORD startebuf )
```

Definition at line 1273 of file [ratio.c](#).

4.116.2.11 MultiplyWithTerm()

```
WORD * MultiplyWithTerm (
    PHEAD WORD * in,
    WORD * term,
    WORD par )
```

Definition at line 1312 of file [ratio.c](#).

4.116.2.12 TakeContent()

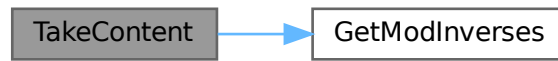
```
WORD * TakeContent (
    PHEAD WORD * in,
    WORD * term )
```

Implements part of the old ExecArg in which we take common factors from arguments with more than one term. Here the input is a sequence of terms in 'in' and the answer is a content-free sequence of terms. This sequence has been allocated by the Malloc1 routine in a call to EndSort, unless the expression was already content-free. In that case the input pointer is returned. The content is returned in term. This is supposed to be a separate allocation, made by TermMalloc in the calling routine.

Definition at line 1376 of file [ratio.c](#).

References [GetModInverses\(\)](#).

Here is the call graph for this function:



4.116.2.13 MergeSymbolLists()

```
int MergeSymbolLists (
    PHEAD WORD * old,
    WORD * extra,
    int par )
```

Definition at line 1808 of file [ratio.c](#).

4.116.2.14 MergeDotproductLists()

```
int MergeDotproductLists (
    PHEAD WORD * old,
    WORD * extra,
    int par )
```

Definition at line 1927 of file [ratio.c](#).

4.116.2.15 CreateExpression()

```
WORD * CreateExpression (
    PHEAD WORD nexp )
```

Definition at line 2020 of file [ratio.c](#).

4.116.2.16 GCDterms()

```
int GCDterms (
    PHEAD WORD * term1,
    WORD * term2,
    WORD * termout )
```

Definition at line 2076 of file [ratio.c](#).

4.116.2.17 ReadPolyRatFun()

```
int ReadPolyRatFun (
    PHEAD WORD * term )
```

Definition at line 2264 of file [ratio.c](#).

4.116.2.18 FromPolyRatFun()

```
int FromPolyRatFun (
    PHEAD WORD * fun,
    WORD ** numout,
    WORD ** denout )
```

Definition at line 2383 of file [ratio.c](#).

4.116.2.19 TakeSymbolContent()

```
WORD * TakeSymbolContent (
    PHEAD WORD * in,
    WORD * term )
```

Implements part of the old ExecArg in which we take common factors from arguments with more than one term. We allow only symbols as this code is used for the polyratfun only. We have a special routine, because the generic TakeContent does too much work and speed is at a premium here. Input: *in* is the input expression as a sequence of terms. **Output:** *term*: the content return value: the contentfree expression. it is in new allocation, made by TermMalloc. (should be in a TermMalloc space?)

Definition at line 2434 of file [ratio.c](#).

References [GetModInverses\(\)](#).

Here is the call graph for this function:



4.116.2.20 GCDclean()

```
void GCDclean (
    PHEAD WORD * num,
    WORD * den )
```

Definition at line 2644 of file [ratio.c](#).

4.116.2.21 PolyDiv()

```
WORD * PolyDiv (
    PHEAD WORD * a,
    WORD * b,
    char * text )
```

Definition at line [2741](#) of file [ratio.c](#).

4.116.2.22 DIVfunction()

```
int DIVfunction (
    PHEAD WORD * term,
    WORD level,
    int par )
```

Definition at line [2788](#) of file [ratio.c](#).

4.116.2.23 MULfunc()

```
WORD * MULfunc (
    PHEAD WORD * p1,
    WORD * p2 )
```

Definition at line [2957](#) of file [ratio.c](#).

4.116.2.24 ConvertArgument()

```
WORD * ConvertArgument (
    PHEAD WORD * arg,
    int * type )
```

Definition at line [3018](#) of file [ratio.c](#).

4.116.2.25 ExpandRat()

```
int ExpandRat (
    PHEAD WORD * fun )
```

Definition at line [3113](#) of file [ratio.c](#).

4.116.2.26 InvPoly()

```
int InvPoly (
    PHEAD WORD * inpoly,
    WORD maxpow,
    WORD sym )
```

Definition at line [3592](#) of file [ratio.c](#).

4.116.3 Variable Documentation

4.116.3.1 divrem

WORD divrem[4] = { DIVFUNCTION, REMFUNCTION, INVERSEFUNCTION, MULFUNCTION }

Definition at line 2786 of file [ratio.c](#).

4.116.3.2 TheErrorMessage

char* TheErrorMessage[]

Initial value:

```
= {
    "PolyRatFun not of a type that FORM will expand: incorrect variable inside."
    , "Division by zero in PolyRatFun encountered in ExpandRat."
    , "Irregular code in PolyRatFun encountered in ExpandRat."
    , "Called from ExpandRat."
    , "WorkSpace overflow. Change parameter WorkSpace in setup file?"
}
```

Definition at line 3105 of file [ratio.c](#).

4.117 ratio.c

[Go to the documentation of this file.](#)

```
00001
00008 /* #[ License : */
00009 /*
00010 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00011 *   When using this file you are requested to refer to the publication
00012 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00013 *   This is considered a matter of courtesy as the development was paid
00014 *   for by FOM the Dutch physics granting agency and we would like to
00015 *   be able to track its scientific use to convince FOM of its value
00016 *   for the community.
00017 *
00018 *   This file is part of FORM.
00019 *
00020 *   FORM is free software: you can redistribute it and/or modify it under the
00021 *   terms of the GNU General Public License as published by the Free Software
00022 *   Foundation, either version 3 of the License, or (at your option) any later
00023 *   version.
00024 *
00025 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00026 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00027 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00028 *   details.
00029 *
00030 *   You should have received a copy of the GNU General Public License along
00031 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00032 */
00033 /* #] License : */
00034 /*
00035 *   #[ Includes : ratio.c
00036 */
00037
00038 #include "form3.h"
00039
00040 /*
00041 *   #] Includes :
00042 *   #[ Ratio :
00043
00044 *   These are the special operations regarding simple polynomials.
00045 *   The first and most needed is the partial fractioning expansion.
00046 *   Ratio,x1,x2,x3
00047
00048 *   The files belonging to the ratio command serve also as a good example
00049 *   of how to implement a new operation.
00050
```

```

00051      # [ RatioFind :
00052
00053      The routine that should locate the need for a ratio command.
00054      If located the corresponding symbols are removed and the
00055      operational parameters are loaded. A subexpression pointer
00056      is inserted and the code for success is returned.
00057
00058      params points at the compiler output block defined in RatioComp.
00059
00060  */
00061
00062  WORD RatioFind(PHEAD WORD *term, WORD *params)
00063  {
00064      GETBIDENTITY
00065      WORD *t, *m, *r;
00066      WORD x1, x2, i;
00067      WORD *y1, *y2, n1 = 0, n2 = 0;
00068      x1 = params[3];
00069      x2 = params[4];
00070      m = t = term;
00071      m += *m;
00072      m -= ABS(m[-1]);
00073      t++;
00074      if ( t < m ) do {
00075          if ( *t == SYMBOL ) {
00076              y1 = 0;
00077              y2 = 0;
00078              r = t + t[1];
00079              m = t + 2;
00080              do {
00081                  if ( *m == x1 ) { y1 = m; n1 = m[1]; }
00082                  else if ( *m == x2 ) { y2 = m; n2 = m[1]; }
00083                  m += 2;
00084              } while ( m < r );
00085              if ( !y1 || !y2 || ( n1 > 0 && n2 > 0 ) ) return(0);
00086              m -= 2;
00087              if ( y1 > y2 ) { r = y1; y1 = y2; y2 = r; }
00088              *y2 = *m; y2[1] = m[1];
00089              m -= 2;
00090              *y1 = *m; y1[1] = m[1];
00091              i = WORDDIFF(m,t);
00092              #if SUBEXPSIZE > 6
00093              We have to revise the code for the second case.
00094              #endif
00095              if ( i > 2 ) {          /* Subexpression fits exactly */
00096                  t[1] = i;
00097                  y1 = term+*term;
00098                  y2 = y1+SUBEXPSIZE-4;
00099                  r = m+4;
00100                  while ( y1 > r ) --y2 = --y1;
00101                  *m++ = SUBEXPRESSION;
00102                  *m++ = SUBEXPSIZE;
00103                  *m++ = -1;
00104                  *m++ = 1;
00105                  *m++ = DUMMYBUFFER;
00106                  FILLSUB(m)
00107                  *term += SUBEXPSIZE-4;
00108              }
00109              else {                  /* All symbols are gone. Rest has to be moved */
00110                  m -= 2;
00111                  *m++ = SUBEXPRESSION;
00112                  *m++ = SUBEXPSIZE;
00113                  *m++ = -1;
00114                  *m++ = 1;
00115                  *m++ = DUMMYBUFFER;
00116                  FILLSUB(m)
00117                  t = term;
00118                  t += *t;
00119                  *term += SUBEXPSIZE-6;
00120                  r = m + 6-SUBEXPSIZE;
00121                  do { *m++ = *r++; } while ( r < t );
00122              }
00123              t = AT.TMout;          /* Load up the TM out array for the generator */
00124              *t++ = 7;
00125              *t++ = RATIO;
00126              *t++ = x1;
00127              *t++ = x2;
00128              *t++ = params[5];
00129              *t++ = n1;
00130              *t++ = n2;
00131              return(1);
00132          }
00133          t += t[1];
00134      } while ( t < m );
00135      return(0);
00136  }
00137

```

```

00138 /*
00139      #] RatioFind :
00140      #[ RatioGen :
00141
00142      The algorithm:
00143      x1^-n1*x2^n2 ==> x2 --> x1 + x3
00144      x1^n1*x2^-n2 ==> x1 --> x2 - x3
00145      x1^-n1*x2^-n2 ==>
00146
00147          +sum(i=0,n1-1){ (-1)^i*binom(n2-1+i,n2-1)
00148              *x3^-(n2+i)*x1^-(n1-i) }
00149          +sum(i=0,n2-1){ (-1)^(n1)*binom(n1-1+i,n1-1)
00150              *x3^-(n1+i)*x2^-(n2-i) }
00151
00152      Actually there is an amount of arbitrariness in the first two
00153      formulae and the replacement x2 -> x1 + x3 could be made 'by hand'.
00154      It is better to use the nontrivial 'minimal change' formula:
00155
00156      x1^-n1*x2^n2:   if ( n1 >= n2 ) {
00157                      +sum(i=0,n2){ x3^i*x1^-(n1-n2+i)*binom(n2,i) }
00158                      }
00159                      else {
00160                          sum(i=0,n2-n1){ x2^(n2-n1-i)*x3^i*binom(n1-1+i,n1-1) }
00161                          +sum(i=0,n1-1){ x3^(n2-i)*x1^-(n1-i)*binom(n2,i) }
00162                      }
00163      x1^n1*x2^-n2:   Same but x3 -> -x3.
00164
00165      The contents of the AT.TMout/params array are:
00166      length,type,x1,x2,x3,n1,n2
00167
00168 */
00169
00170 WORD RatioGen(PHEAD WORD *term, WORD *params, WORD num, WORD level)
00171 {
00172     GETBIDENTITY
00173     WORD *t, *m;
00174     WORD *tstops[3];
00175     WORD n1, n2, i, j;
00176     WORD x1,x2,x3;
00177     UWORD *coef;
00178     WORD ncoef, sign = 0;
00179     coef = (UWORD *)AT.WorkPointer;
00180     t = term;
00181     tstops[2] = m = t + *t;
00182     m -= ABS(m[-1]);
00183     t++;
00184     do {
00185         if ( *t == SUBEXPRESSION && t[2] == num ) break;
00186         t += t[1];
00187     } while ( t < m );
00188     tstops[0] = t;
00189     tstops[1] = t + t[1];
00190
00191     /* Copying to termout will be from term to tstop1, then the induced part
00192     and finally from tstop2 to tstop3
00193
00194     Now separate the various cases:
00195
00196 */
00197     t = params + 2;
00198     x1 = *t++;
00199     x2 = *t++;
00200     x3 = *t++;
00201     n1 = *t++;
00202     n2 = *t++;
00203     if ( n1 > 0 ) { /* Flip the variables and indicate -x3 */
00204         n2 = -n2;
00205         sign = 1;
00206         i = n1; n1 = n2; n2 = i;
00207         i = x1; x1 = x2; x2 = i;
00208         goto PosNeg;
00209     }
00210     else if ( n2 > 0 ) {
00211         n1 = -n1;
00212     PosNeg:
00213         if ( n2 <= n1 ) { /* x1 -> x2 + x3 */
00214             *coef = 1;
00215             ncoef = 1;
00216             AT.WorkPointer = (WORD *) (coef + 1);
00217             j = n2;
00218             for ( i = 0; i <= n2; i++ ) {
00219                 if ( BinomGen(BHEAD term,level,tstops,x1,x3,n2-n1-i,i,sign&i
00220                     ,coef,ncoef) ) goto RatioCall;
00221                 if ( i < n2 ) {
00222                     if ( Product(coef,&ncoef,j) ) goto RatioCall;
00223                     if ( Quotient(coef,&ncoef,i+1) ) goto RatioCall;
00224                     j--;

```

```

00225         AT.WorkPointer = (WORD *) (coef + ABS(ncoef));
00226     }
00227 }
00228 AT.WorkPointer = (WORD *) (coef);
00229 return(0);
00230 }
00231 else {
00232 /*
00233     sum(i=0,n2-n1){x2^(n2-n1-i)*x3^i*binom(n1-1+i,n1-1)}
00234 +sum(i=0,n1-1){x3^(n2-i)*x1^(n1-i)*binom(n2,i)}
00235 */
00236     *coef = 1;
00237     ncoef = 1;
00238     AT.WorkPointer = (WORD *) (coef + 1);
00239     j = n2 - n1;
00240     for ( i = 0; i <= j; i++ ) {
00241         if ( BinomGen(BHEAD term,level,tstops,x2,x3,n2-n1-i,i,sign&i
00242             ,coef,ncoef) ) goto RatioCall;
00243         if ( i < j ) {
00244             if ( Product(coef,&ncoef,n1+i) ) goto RatioCall;
00245             if ( Quotient(coef,&ncoef,i+1) ) goto RatioCall;
00246             AT.WorkPointer = (WORD *) (coef + ABS(ncoef));
00247         }
00248     }
00249     *coef = 1;
00250     ncoef = 1;
00251     AT.WorkPointer = (WORD *) (coef + 1);
00252     j = n1-1;
00253     for ( i = 0; i <= j; i++ ) {
00254         if ( BinomGen(BHEAD term,level,tstops,x1,x3,i-n1,n2-i,sign&(n2-i)
00255             ,coef,ncoef) ) goto RatioCall;
00256         if ( i < j ) {
00257             if ( Product(coef,&ncoef,n2-i) ) goto RatioCall;
00258             if ( Quotient(coef,&ncoef,i+1) ) goto RatioCall;
00259             AT.WorkPointer = (WORD *) (coef + ABS(ncoef));
00260         }
00261     }
00262     AT.WorkPointer = (WORD *) (coef);
00263     return(0);
00264 }
00265 }
00266 else {
00267     n2 = -n2;
00268     n1 = -n1;
00269 /*
00270     +sum(i=0,n1-1){(-1)^i*binom(n2-1+i,n2-1)
00271     *x3^(n2+i)*x1^(n1-i)}
00272 +sum(i=0,n2-1){(-1)^(n1)*binom(n1-1+i,n1-1)
00273     *x3^(n1+i)*x2^(n2-i)}
00274 */
00275     *coef = 1;
00276     ncoef = 1;
00277     AT.WorkPointer = (WORD *) (coef + 1);
00278     j = n1-1;
00279     for ( i = 0; i <= j; i++ ) {
00280         if ( BinomGen(BHEAD term,level,tstops,x1,x3,i-n1,-n2-i,i&1
00281             ,coef,ncoef) ) goto RatioCall;
00282         if ( i < j ) {
00283             if ( Product(coef,&ncoef,n2+i) ) goto RatioCall;
00284             if ( Quotient(coef,&ncoef,i+1) ) goto RatioCall;
00285             AT.WorkPointer = (WORD *) (coef + ABS(ncoef));
00286         }
00287     }
00288     *coef = 1;
00289     ncoef = 1;
00290     AT.WorkPointer = (WORD *) (coef + 1);
00291     j = n2-1;
00292     for ( i = 0; i <= j; i++ ) {
00293         if ( BinomGen(BHEAD term,level,tstops,x2,x3,i-n2,-n1-i,n1&1
00294             ,coef,ncoef) ) goto RatioCall;
00295         if ( i < j ) {
00296             if ( Product(coef,&ncoef,n1+i) ) goto RatioCall;
00297             if ( Quotient(coef,&ncoef,i+1) ) goto RatioCall;
00298             AT.WorkPointer = (WORD *) (coef + ABS(ncoef));
00299         }
00300     }
00301     AT.WorkPointer = (WORD *) (coef);
00302     return(0);
00303 }
00304 }
00305 RatioCall:
00306     MLOCK(ErrorMessageLock);
00307     MesCall("RatioGen");
00308     MUNLOCK(ErrorMessageLock);
00309     SETERROR(-1)
00310 }
00311

```

```

00312 /*
00313     #] RatioGen :
00314     #[ BinomGen :
00315
00316     Routine for the generation of terms in a binomialtype expansion.
00317
00318 */
00319
00320 WORD BinomGen(PHEAD WORD *term, WORD level, WORD **tstops, WORD x1, WORD x2,
00321     WORD pow1, WORD pow2, WORD sign, UWORD *coef, WORD ncoef)
00322 {
00323     GETBIDENTITY
00324     WORD *t, *r;
00325     WORD *termout;
00326     WORD k;
00327     termout = AT.WorkPointer;
00328     t = termout;
00329     r = term;
00330     do { *t++ = *r++; } while ( r < tstops[0] );
00331     *t++ = SYMBOL;
00332     if ( pow2 == 0 ) {
00333         if ( pow1 == 0 ) t--;
00334         else { *t++ = 4; *t++ = x1; *t++ = pow1; }
00335     }
00336     else if ( pow1 == 0 ) {
00337         *t++ = 4; *t++ = x2; *t++ = pow2;
00338     }
00339     else {
00340         *t++ = 6; *t++ = x1; *t++ = pow1; *t++ = x2; *t++ = pow2;
00341     }
00342     *t++ = LNUMBER;
00343     *t++ = ABS(ncoef) + 3;
00344     *t = ncoef;
00345     if ( sign ) *t = -*t;
00346     t++;
00347     ncoef = ABS(ncoef);
00348     for ( k = 0; k < ncoef; k++ ) *t++ = coef[k];
00349     r = tstops[1];
00350     do { *t++ = *r++; } while ( r < tstops[2] );
00351     *termout = WORDDIF(t,termout);
00352     AT.WorkPointer = t;
00353     if ( AT.WorkPointer > AT.WorkTop ) {
00354         MLOCK(ErrorMessageLock);
00355         MesWork();
00356         MUNLOCK(ErrorMessageLock);
00357         return(-1);
00358     }
00359     *AN.RepPoint = 1;
00360     AR.expchanged = 1;
00361     if ( Generator(BHEAD termout,level) ) {
00362         MLOCK(ErrorMessageLock);
00363         MesCall("BinomGen");
00364         MUNLOCK(ErrorMessageLock);
00365         SETERROR(-1)
00366     }
00367     AT.WorkPointer = termout;
00368     return(0);
00369 }
00370
00371 /*
00372     #] BinomGen :
00373     #] Ratio :
00374     #[ Sum :
00375     #[ DoSumF1 :
00376
00377     Routine expands a sum_ function.
00378     Its arguments are:
00379     The term in which the function occurs.
00380     The parameter list:
00381         length of parameter field
00382         function number (SUMNUM1)
00383         number of the symbol that is loop parameter
00384         min value
00385         max value
00386         increment
00387     the number of the subexpression to be removed
00388     the level in the generation tree.
00389
00390     Note that the insertion of the loop parameter in the argument
00391     is done via the regular wildcard substitution mechanism.
00392
00393 */
00394
00395 WORD DoSumF1(PHEAD WORD *term, WORD *params, WORD replac, WORD level)
00396 {
00397     GETBIDENTITY
00398     WORD *termout, *t, extractbuff = AT.TMbuff;

```

```

00399     WORD isum, ival, iinc;
00400     LONG from;
00401     CBUF *C;
00402     ival = params[3];
00403     iinc = params[5];
00404     if ( ( iinc > 0 && params[4] >= ival )
00405         || ( iinc < 0 && params[4] <= ival ) ) {
00406         isum = (params[4] - ival)/iinc + 1;
00407     }
00408     else return(0);
00409     termout = AT.WorkPointer;
00410     AT.WorkPointer = (WORD *)((UBYTE *) (AT.WorkPointer)) + AM.MaxTer;
00411     if ( AT.WorkPointer > AT.WorkTop ) {
00412         MLOCK(ErrorMessageLock);
00413         MesWork();
00414         MUNLOCK(ErrorMessageLock);
00415         return(-1);
00416     }
00417     t = term + 1;
00418     while ( *t != SUBEXPRESSION || t[2] != replac || t[4] != extractbuff )
00419         t += t[1];
00420     C = cbuf+t[4];
00421     t += SUBEXPSIZE;
00422     if ( params[2] < 0 ) {
00423         while ( *t != INDTOIND || t[2] != -params[2] ) t += t[1];
00424         *t = INDTOIND;
00425     }
00426     else {
00427         while ( *t > SYMTOSUB || t[2] != params[2] ) t += t[1];
00428         *t = SYMTONUM;
00429     }
00430     do {
00431         t[3] = ival;
00432         from = C->rhs[replac] - C->Buffer;
00433         while ( C->Buffer[from] ) {
00434             if ( InsertTerm(BHEAD term, replac, extractbuff, C->Buffer+from, termout, 0) < 0 ) goto
SumFlCall;
00435             AT.WorkPointer = termout + *termout;
00436             if ( Generator(BHEAD termout, level) < 0 ) goto SumFlCall;
00437             from += C->Buffer[from];
00438         }
00439         ival += iinc;
00440     } while ( --isum > 0 );
00441     AT.WorkPointer = termout;
00442     return(0);
00443 SumFlCall:
00444     MLOCK(ErrorMessageLock);
00445     MesCall("DoSumFl");
00446     MUNLOCK(ErrorMessageLock);
00447     SETERROR(-1)
00448 }
00449
00450 /*
00451     #] DoSumFl :
00452     #[ Glue :
00453
00454     Routine multiplies two terms. The second term is subject
00455     to the wildcard substitutions in sub.
00456     Output in the first term. This routine is a variation on
00457     the routine InsertTerm.
00458
00459 */
00460
00461 WORD Glue(PHEAD WORD *term1, WORD *term2, WORD *sub, WORD insert)
00462 {
00463     GETBIDENTITY
00464     UWORD *coef;
00465     WORD ncoef, *t, *t1, *t2, i, nc2, nc3, old, newer;
00466     coef = (UWORD *) (TermMalloc("Glue"));
00467     t = term1;
00468     t += *t;
00469     i = t[-1];
00470     t -= ABS(i);
00471     old = WORDDIF(t, term1);
00472     ncoef = REDLENG(i);
00473     if ( i < 0 ) i = -i;
00474     i--;
00475     t1 = t;
00476     t2 = (WORD *) coef;
00477     while ( --i >= 0 ) *t2++ = *t1++;
00478     i = *--t;
00479     nc2 = WildFill(BHEAD t, term2, sub);
00480     *t = i;
00481     t += nc2;
00482     nc2 = t[-1];
00483     t -= ABS(nc2);
00484     newer = WORDDIF(t, term1);

```

```

00485     if ( MulRat (BHEAD (UWORD *)t, REDLENG(nc2), coef, ncoef, (UWORD *)t, &nc3) ) {
00486         MLOCK(ErrorMessageLock);
00487         MesCall("Glue");
00488         MUNLOCK(ErrorMessageLock);
00489         TermFree(coef, "Glue");
00490         SETERROR(-1)
00491     }
00492     i = (ABS(nc3))*2;
00493     t += i++;
00494     *t++ = (nc3 >= 0)?i:-i;
00495     *term1 = WORDDIF(t, term1);
00496 /*
00497     Switch the new piece with the old tail, so that noncommuting
00498     variables get into their proper spot.
00499 */
00500     i = old - insert;
00501     t1 = t;
00502     t2 = term1+insert;
00503     NCOPY(t1, t2, i);
00504     i = newer - old;
00505     t1 = term1+insert;
00506     t2 = term1+old;
00507     NCOPY(t1, t2, i);
00508     t2 = t;
00509     i = old - insert;
00510     NCOPY(t1, t2, i);
00511     TermFree(coef, "Glue");
00512     return(0);
00513 }
00514
00515 /*
00516     #] Glue :
00517     #[ DoSumF2 :
00518 */
00519
00520 WORD DoSumF2(PHEAD WORD *term, WORD *params, WORD replac, WORD level)
00521 {
00522     GETBIDENTITY
00523     WORD *termout, *t, *from, *sub, *to, extractbuff = AT.TMbuff;
00524     WORD isum, ival, iinc, insert, i;
00525     CBUF *C;
00526     ival = params[3];
00527     iinc = params[5];
00528     if ( ( iinc > 0 && params[4] >= ival )
00529         || ( iinc < 0 && params[4] <= ival ) ) {
00530         isum = (params[4] - ival)/iinc + 1;
00531     }
00532     else return(0);
00533     termout = AT.WorkPointer;
00534     AT.WorkPointer = (WORD *)((UBYTE *) (AT.WorkPointer)) + AM.MaxTer;
00535     if ( AT.WorkPointer > AT.WorkTop ) {
00536         MLOCK(ErrorMessageLock);
00537         MesWork();
00538         MUNLOCK(ErrorMessageLock);
00539         return(-1);
00540     }
00541     t = term + 1;
00542     while ( *t != SUBEXPRESSION || t[2] != replac || t[4] != extractbuff ) t += t[1];
00543     insert = WORDDIF(t, term);
00544
00545     from = term;
00546     to = termout;
00547     while ( from < t ) *to++ = *from++;
00548     from += t[1];
00549     sub = term + *term;
00550     while ( from < sub ) *to++ = *from++;
00551     *termout -= t[1];
00552
00553     sub = t;
00554     C = cbuf+t[4];
00555     t += SUBEXPSIZE;
00556     if ( params[2] < 0 ) {
00557         while ( *t != INDTOIND || t[2] != -params[2] ) t += t[1];
00558         *t = INDTOIND;
00559     }
00560     else {
00561         while ( *t > SYMTOSUB || t[2] != params[2] ) t += t[1];
00562         *t = SYMTONUM;
00563     }
00564     t[3] = ival;
00565     for(;;) {
00566         AT.WorkPointer = termout + *termout;
00567         to = AT.WorkPointer;
00568         if ( ( to + *termout ) > AT.WorkTop ) {
00569             MLOCK(ErrorMessageLock);
00570             MesWork();
00571             MUNLOCK(ErrorMessageLock);

```

```

00572         return(-1);
00573     }
00574     from = termout;
00575     i = *termout;
00576     NCOPY(to, from, i);
00577     from = AT.WorkPointer;
00578     AT.WorkPointer = to;
00579     if ( Generator(BHEAD from, level) < 0 ) goto SumF2Call;
00580     if ( --isum <= 0 ) break;
00581     ival += iinc;
00582     t[3] = ival;
00583     if ( Glue(BHEAD termout, C->rhs[replac], sub, insert) < 0 ) goto SumF2Call;
00584 }
00585 AT.WorkPointer = termout;
00586 return(0);
00587 SumF2Call:
00588 MLOCK(ErrorMessageLock);
00589 MesCall("DoSumF2");
00590 MUNLOCK(ErrorMessageLock);
00591 SETERROR(-1)
00592 }
00593
00594 /*
00595     #] DoSumF2 :
00596     #] Sum :
00597     #[ GCDfunction :
00598     #[ GCDfunction :
00599 */
00600
00601 typedef struct {
00602     WORD *buffer;
00603     DOLLARS dollar;
00604     LONG size;
00605     int type;
00606     int dummy;
00607 } ARGBUFFER;
00608
00609 int GCDfunction(PHEAD WORD *term, WORD level)
00610 {
00611     GETBIDENTITY
00612     WORD *t, *tstop, *tf, *termout, *tin, *tout, *m, *mnext, *mstop, *mm;
00613     int todo, i, ii, j, istart, sign = 1, action = 0;
00614     WORD firstshort = 0, firstvalue = 0, gcdisone = 0, mlength, tlength, newlength;
00615     WORD totargs = 0, numargs, argsdone = 0, *mh, oldvall, *g, *gcdout = 0;
00616     WORD *argl, *arg2;
00617     UWORD x1, x2, x3;
00618     LONG args;
00619     #if ( FUNHEAD > 4 )
00620     WORD sh[FUNHEAD+5];
00621     #else
00622     WORD sh[9];
00623     #endif
00624     DOLLARS d;
00625     ARGBUFFER *abuf = 0, ab;
00626 /*
00627     #[ Find Function. Count arguments :
00628
00629     First find the proper function
00630 */
00631     t = term + *term; tlength = t[-1];
00632     tstop = t - ABS(tlength);
00633     t = term + 1;
00634     while ( t < tstop ) {
00635         if ( *t != GCDFUNCTION ) { t += t[1]; continue; }
00636         todo = 1; totargs = 0;
00637         tf = t + FUNHEAD;
00638         while ( tf < t + t[1] ) {
00639             totargs++;
00640             if ( *tf > 0 && tf[1] != 0 ) todo = 0;
00641             NEXTARG(tf);
00642         }
00643         if ( todo ) break;
00644         t += t[1];
00645     }
00646     if ( t >= tstop ) {
00647         MLOCK(ErrorMessageLock);
00648         MesPrint("Internal error. Indicated gcd_ function not encountered.");
00649         MUNLOCK(ErrorMessageLock);
00650         Terminate(-1);
00651     }
00652     WantAddPointers(totargs);
00653     args = AT.pWorkPointer; AT.pWorkPointer += totargs;
00654 /*
00655     #] Find Function. Count arguments :
00656     #[ Do short arguments :
00657
00658     The function we need, in agreement with TestSub, is now in t

```



```

00659     Make first a compilation of the short arguments (except $-s and expressions)
00660     to see whether we need to do much work.
00661     This means that after this scan we can ignore all short arguments with
00662     the exception of unevaluated $-s and expressions.
00663  */
00664     numargs = 0;
00665     firstshort = 0;
00666     tf = t + FUNHEAD;
00667     while ( tf < t + t[1] ) {
00668         if ( *tf == -SNUMBER && tf[1] == 0 ) { NEXTARG(tf); continue; }
00669         if ( *tf > 0 || *tf == -DOLLAREXPRESSION || *tf == -EXPRESSION ) {
00670             AT.pWorkspace[args+numargs++] = tf;
00671             NEXTARG(tf); continue;
00672         }
00673         if ( firstshort == 0 ) {
00674             firstshort = *tf;
00675             if ( *tf <= -FUNCTION ) { firstvalue = -( *tf ); }
00676             else { firstvalue = tf[1]; }
00677             NEXTARG(tf);
00678             argsdone++;
00679             continue;
00680         }
00681         else if ( *tf != firstshort ) {
00682             if ( *tf != -INDEX && *tf != -VECTOR && *tf != -MINVECTOR ) {
00683                 argsdone++; gcdisone = 1; break;
00684             }
00685             if ( firstshort != -INDEX && firstshort != -VECTOR && firstshort != -MINVECTOR ) {
00686                 argsdone++; gcdisone = 1; break;
00687             }
00688             if ( tf[1] != firstvalue ) {
00689                 argsdone++; gcdisone = 1; break;
00690             }
00691             if ( *t == -MINVECTOR ) { firstshort = -VECTOR; }
00692             if ( firstshort == -MINVECTOR ) { firstshort = -VECTOR; }
00693         }
00694         else if ( *tf > -FUNCTION && *tf != -SNUMBER && tf[1] != firstvalue ) {
00695             argsdone++; gcdisone = 1; break;
00696         }
00697         if ( *tf == -SNUMBER && firstvalue != tf[1] ) {
00698             /*
00699                 make a new firstvalue which is gcd_(firstvalue,tf[1])
00700             */
00701             if ( firstvalue == 1 || tf[1] == 1 ) { gcdisone = 1; break; }
00702             if ( firstvalue < 0 && tf[1] < 0 ) {
00703                 x1 = -firstvalue; x2 = -tf[1]; sign = -1;
00704             }
00705             else {
00706                 x1 = ABS(firstvalue); x2 = ABS(tf[1]); sign = 1;
00707             }
00708             while ( ( x3 = x1%x2 ) != 0 ) { x1 = x2; x2 = x3; }
00709             firstvalue = ((WORD)x2)*sign;
00710             argsdone++;
00711             if ( firstvalue == 1 ) { gcdisone = 1; break; }
00712         }
00713         NEXTARG(tf);
00714     }
00715     termout = AT.WorkPointer;
00716     AT.WorkPointer = (WORD *)(((UBYTE *) (AT.WorkPointer)) + AM.MaxTer);
00717     if ( AT.WorkPointer > AT.WorkTop ) {
00718         MLOCK(ErrorMessageLock);
00719         MesWork();
00720         MUNLOCK(ErrorMessageLock);
00721         return(-1);
00722     }
00723  */
00724     #] Do short arguments :
00725     #[ Do trivial GCD :
00726
00727     Copy head
00728  */
00729     i = t - term; tin = term; tout = termout;
00730     NCOPY(tout,tin,i);
00731     if ( gcdisone || ( firstshort == -SNUMBER && firstvalue == 1 ) ) {
00732         sign = 1;
00733     gcdone:
00734         tin += t[1]; tstop = term + *term;
00735         while ( tin < tstop ) *tout++ = *tin++;
00736         *termout = tout - termout;
00737         if ( sign < 0 ) tout[-1] = -tout[-1];
00738         AT.WorkPointer = tout;
00739         if ( argsdone && Generator(BHEAD termout,level) < 0 ) goto CalledFrom;
00740         AT.WorkPointer = termout;
00741         AT.pWorkPointer = args;
00742         return(0);
00743     }
00744  */
00745     #] Do trivial GCD :

```

```

00746     #[ Do short argument GCD :
00747     */
00748     if ( numargs == 0 ) {      /* basically we are done */
00749 doshort:
00750         sign = 1;
00751         if ( firstshort == 0 ) goto gcdone;
00752         if ( firstshort == -SNUMBER ) {
00753             *tout++ = SNUMBER; *tout++ = 4; *tout++ = firstvalue; *tout++ = 1;
00754             goto gcdone;
00755         }
00756         else if ( firstshort == -SYMBOL ) {
00757             *tout++ = SYMBOL; *tout++ = 4; *tout++ = firstvalue; *tout++ = 1;
00758             goto gcdone;
00759         }
00760         else if ( firstshort == -VECTOR || firstshort == -INDEX ) {
00761             *tout++ = INDEX; *tout++ = 3; *tout++ = firstvalue; goto gcdone;
00762         }
00763         else if ( firstshort == -MINVECTOR ) {
00764             sign = -1;
00765             *tout++ = INDEX; *tout++ = 3; *tout++ = firstvalue; goto gcdone;
00766         }
00767         else if ( firstshort <= -FUNCTION ) {
00768             *tout++ = firstvalue; *tout++ = FUNHEAD; FILLFUN(tout);
00769             goto gcdone;
00770         }
00771         else {
00772             MLOCK(ErrorMessageLock);
00773             MesPrint("Internal error. Illegal short argument in GCDfunction.");
00774             MUNLOCK(ErrorMessageLock);
00775             Terminate(-1);
00776         }
00777     }
00778     /*
00779     #[ Do short argument GCD :
00780     #[ Convert short argument :
00781
00782     Now we allocate space for the arguments in general notation.
00783     First the special one if there were short arguments
00784     */
00785     if ( firstshort ) {
00786         switch ( firstshort ) {
00787             case -SNUMBER:
00788                 sh[0] = 4; sh[1] = ABS(firstvalue); sh[2] = 1;
00789                 if ( firstvalue < 0 ) sh[3] = -3;
00790                 else sh[3] = 3;
00791                 sh[4] = 0;
00792                 break;
00793             case -MINVECTOR:
00794             case -VECTOR:
00795             case -INDEX:
00796                 sh[0] = 8; sh[1] = INDEX; sh[2] = 3; sh[3] = firstvalue;
00797                 sh[4] = 1; sh[5] = 1;
00798                 if ( firstshort == -MINVECTOR ) sh[6] = -3;
00799                 else sh[6] = 3;
00800                 sh[7] = 0;
00801                 break;
00802             case -SYMBOL:
00803                 sh[0] = 8; sh[1] = SYMBOL; sh[2] = 4; sh[3] = firstvalue; sh[4] = 1;
00804                 sh[5] = 1; sh[6] = 1; sh[7] = 3; sh[8] = 0;
00805                 break;
00806             default:
00807                 sh[0] = FUNHEAD+4; sh[1] = firstshort; sh[2] = FUNHEAD;
00808                 for ( i = 2; i < FUNHEAD; i++ ) sh[i+1] = 0;
00809                 sh[FUNHEAD+1] = 1; sh[FUNHEAD+2] = 1; sh[FUNHEAD+3] = 3; sh[FUNHEAD+4] = 0;
00810                 break;
00811         }
00812     }
00813     /*
00814     #[ Convert short argument :
00815     #[ Sort arguments :
00816
00817     Now we should sort the arguments in a way that the dollars and the
00818     expressions come last. That way we may never need them.
00819     */
00820     for ( i = 1; i < numargs; i++ ) {
00821         for ( ii = i; ii > 0; ii-- ) {
00822             arg1 = AT.pWorkSpace[args+ii];
00823             arg2 = AT.pWorkSpace[args+ii-1];
00824             if ( *arg1 < 0 ) {
00825                 if ( *arg2 < 0 ) {
00826                     if ( *arg1 == -EXPRESSION ) break;
00827                     if ( *arg2 == -DOLLAREXPRESSION ) break;
00828                     AT.pWorkSpace[args+ii] = arg2;
00829                     AT.pWorkSpace[args+ii-1] = arg1;
00830                 }
00831                 else break;
00832             }

```

```

00833         else if ( *arg2 < 0 ) {
00834             AT.pWorkSpace[args+ii] = arg2;
00835             AT.pWorkSpace[args+ii-1] = arg1;
00836         }
00837         else {
00838             if ( *arg1 > *arg2 ) {
00839                 AT.pWorkSpace[args+ii] = arg2;
00840                 AT.pWorkSpace[args+ii-1] = arg1;
00841             }
00842             else break;
00843         }
00844     }
00845 }
00846 /*
00847 #] Sort arguments :
00848 #[ There is a single term argument :
00849 */
00850 if ( firstshort ) {
00851     mh = sh; istart = 0;
00852 oneterm:;
00853     for ( i = istart; i < numargs; i++ ) {
00854         arg1 = AT.pWorkSpace[args+i];
00855         if ( *arg1 > 0 ) {
00856             oldvall = arg1[*arg1]; arg1[*arg1] = 0;
00857             m = arg1+ARGHEAD;
00858             while ( *m ) {
00859                 GCDterms(BHEAD mh,m,mh); m += *m;
00860                 if ( mh[0] == 4 && mh[1] == 1 && mh[2] == 1 && mh[3] == 3 ) {
00861                     gcdisone = 1; sign = 1; arg1[*arg1] = oldvall; goto gcdone;
00862                 }
00863             }
00864             arg1[*arg1] = oldvall;
00865         }
00866         else if ( *arg1 == -DOLLAREXPRESSION ) {
00867             if ( ( d = DolToTerms(BHEAD arg1[1]) ) != 0 ) {
00868                 m = d->where;
00869                 while ( *m ) {
00870                     GCDterms(BHEAD mh,m,mh); m += *m;
00871                     argsdone++;
00872                     if ( mh[0] == 4 && mh[1] == 1 && mh[2] == 1 && mh[3] == 3 ) {
00873                         gcdisone = 1; sign = 1;
00874                         if ( d->factors ) M_free(d->factors,"Dollar factors");
00875                         M_free(d,"Copy of dollar variable"); goto gcdone;
00876                     }
00877                 }
00878                 if ( d->factors ) M_free(d->factors,"Dollar factors");
00879                 M_free(d,"Copy of dollar variable");
00880             }
00881         }
00882         else {
00883             mm = CreateExpression(BHEAD arg1[1]);
00884             m = mm;
00885             while ( *m ) {
00886                 GCDterms(BHEAD mh,m,mh); m += *m;
00887                 argsdone++;
00888                 if ( mh[0] == 4 && mh[1] == 1 && mh[2] == 1 && mh[3] == 3 ) {
00889                     gcdisone = 1; sign = 1; M_free(mm,"CreateExpression"); goto gcdone;
00890                 }
00891             }
00892             M_free(mm,"CreateExpression");
00893         }
00894     }
00895     if ( firstshort ) {
00896         if ( mh[0] == 4 ) {
00897             firstshort = -SNUMBER; firstvalue = mh[1] * (mh[3]/3);
00898         }
00899         else if ( mh[1] == SYMBOL ) {
00900             firstshort = -SYMBOL; firstvalue = mh[3];
00901         }
00902         else if ( mh[1] == INDEX ) {
00903             firstshort = -INDEX; firstvalue = mh[3];
00904             if ( mh[6] == -3 ) firstshort = -MINVECTOR;
00905         }
00906         else if ( mh[1] >= FUNCTION ) {
00907             firstshort = -mh[1]; firstvalue = mh[1];
00908         }
00909         goto doshort;
00910     }
00911     else {
00912         /*
00913         We have a GCD that is only a single term.
00914         Paste it in and combine the coefficients.
00915         */
00916         mh[mh[0]] = 0;
00917         mm = mh;
00918         ii = 0;
00919         goto multiterms;

```

```

00920     }
00921 }
00922 /*
00923 Now we have only regular arguments.
00924 But some have not yet been expanded.
00925 Check whether there are proper long arguments and if so if there is
00926 one with just a single term
00927 */
00928 for ( i = 0; i < numargs; i++ ) {
00929     arg1 = AT.pWorkSpace[args+i];
00930     if ( *arg1 > 0 && arg1[ARGHEAD]+ARGHEAD == *arg1 ) {
00931 /*
00932         We have an argument with a single term
00933 */
00934         if ( i != 0 ) {
00935             arg2 = AT.pWorkSpace[args];
00936             AT.pWorkSpace[args] = arg1;
00937             AT.pWorkSpace[args+1] = arg2;
00938         }
00939         m = mh = AT.WorkPointer;
00940         mm = arg1+ARGHEAD; i = *mm;
00941         NCOPY(m,mm,i);
00942         AT.WorkPointer = m;
00943         istart = 1;
00944         argsdone++;
00945         goto oneterm;
00946     }
00947 }
00948 /*
00949 #] There is a single term argument :
00950 #[ Expand $ and expr :
00951
00952 We have: 1: regular multiterm arguments
00953          2: dollars
00954          3: expressions.
00955 The sum of them is numargs. Their addresses are in args. The problem is
00956 that expansion will lead to allocations that we have to return and all
00957 these allocations are different in nature.
00958 */
00959 action = 1;
00960 abuf = (ARGBUFFER *)Malloc1(numargs*sizeof(ARGBUFFER),"argbuffer");
00961 for ( i = 0; i < numargs; i++ ) {
00962     arg1 = AT.pWorkSpace[args+i];
00963     if ( *arg1 > 0 ) {
00964         m = (WORD *)Malloc1(*arg1*sizeof(WORD),"argbuffer type 0");
00965         abuf[i].buffer = m;
00966         abuf[i].type = 0;
00967         mm = arg1+ARGHEAD;
00968         j = *arg1-ARGHEAD;
00969         abuf[i].size = j;
00970         if ( j ) argsdone++;
00971         NCOPY(m,mm,j);
00972         *m = 0;
00973     }
00974     else if ( *arg1 == -DOLLAREXPRESSION ) {
00975         d = DolToTerms(BHEAD arg1[1]);
00976         abuf[i].buffer = d->where;
00977         abuf[i].type = 1;
00978         abuf[i].dollar = d;
00979         m = abuf[i].buffer;
00980         if ( *m ) argsdone++;
00981         while ( *m ) m+= *m;
00982         abuf[i].size = m-abuf[i].buffer;
00983     }
00984     else if ( *arg1 == -EXPRESSION ) {
00985         abuf[i].buffer = CreateExpression(BHEAD arg1[1]);
00986         abuf[i].type = 2;
00987         m = abuf[i].buffer;
00988         if ( *m ) argsdone++;
00989         while ( *m ) m+= *m;
00990         abuf[i].size = m-abuf[i].buffer;
00991     }
00992     else {
00993         MLOCK(ErrorMessageLock);
00994         MesPrint("What argument is this?");
00995         MUNLOCK(ErrorMessageLock);
00996         goto CalledFrom;
00997     }
00998 }
00999 for ( i = 0; i < numargs; i++ ) {
01000     arg1 = abuf[i].buffer;
01001     if ( *arg1 == 0 ) {}
01002     else if ( arg1[*arg1] == 0 ) {
01003 /*
01004         After expansion there is an argument with a single term
01005 */
01006         ab = abuf[i]; abuf[i] = abuf[0]; abuf[0] = ab;

```

```

01007         mh = abuf[0].buffer;
01008         for ( j = 1; j < numargs; j++ ) {
01009             m = abuf[j].buffer;
01010             while ( *m ) {
01011                 GCDterms(BHEAD mh,m,mh); m += *m;
01012                 argsdone++;
01013                 if ( mh[0] == 4 && mh[1] == 1 && mh[2] == 1 && mh[3] == 3 ) {
01014                     gcdisone = 1; sign = 1; break;
01015                 }
01016             }
01017             if ( *m ) break;
01018         }
01019         mm = mh + *mh; if ( mm[-1] < 0 ) { sign = -1; mm[-1] = -mm[-1]; }
01020         mstop = mm - mm[-1]; m = mh+1; mlength = mm[-1];
01021         while ( tin < t ) *tout++ = *tin++;
01022         while ( m < mstop ) *tout++ = *m++;
01023         tin += tin[1];
01024         while ( tin < tstop ) *tout++ = *tin++;
01025         tlength = REDLENG(tlength);
01026         mlength = REDLENG(mlength);
01027         if ( MulRat(BHEAD (UWORD *)tstop,tlength,(UWORD *)mstop,mlength,
01028             (UWORD *)tout,&newlength) < 0 ) goto CalledFrom;
01029         mlength = INCLENG(newlength);
01030         tout += ABS(mlength);
01031         tout[-1] = mlength*sign;
01032         *termout = tout - termout;
01033         AT.WorkPointer = tout;
01034         if ( argsdone && Generator(BHEAD termout,level) < 0 ) goto CalledFrom;
01035         goto cleanup;
01036     }
01037 }
01038 /*
01039  There are only arguments with more than one term.
01040  We order them by size to make the computations as easy as possible.
01041 */
01042 for ( i = 1; i < numargs; i++ ) {
01043     for ( ii = i; ii > 0; ii-- ) {
01044         if ( abuf[ii-1].size <= abuf[ii].size ) break;
01045         ab = abuf[ii-1]; abuf[ii-1] = abuf[ii]; abuf[ii] = ab;
01046     }
01047 }
01048 /*
01049  #] Expand $ and expr :
01050  #[ Multiterm subexpressions :
01051 */
01052 ii = 0;
01053 gcdout = abuf[ii].buffer;
01054 for ( i = 0; i < numargs; i++ ) {
01055     if ( abuf[i].buffer[0] ) { gcdout = abuf[i].buffer; ii = i; i++; argsdone++; break; }
01056 }
01057 for ( ; i < numargs; i++ ) {
01058     if ( abuf[i].buffer[0] ) {
01059         g = GCDfunction3(BHEAD gcdout,abuf[i].buffer);
01060         argsdone++;
01061         if ( gcdout != abuf[ii].buffer ) M_free(gcdout,"gcdout");
01062         gcdout = g;
01063         if ( gcdout[*gcdout] == 0 && gcdout[0] == 4 && gcdout[1] == 1
01064             && gcdout[2] == 1 && gcdout[3] == 3 ) break;
01065     }
01066 }
01067 mm = gcdout;
01068 multiterms:;
01069 tlength = REDLENG(tlength);
01070 while ( *mm ) {
01071     tin = term; tout = termout; while ( tin < t ) *tout++ = *tin++;
01072     tin += t[1];
01073     mnext = mm + *mm; mlength = mnext[-1]; mstop = mnext - ABS(mlength);
01074     mm++;
01075     while ( mm < mstop ) *tout++ = *mm++;
01076     while ( tin < tstop ) *tout++ = *tin++;
01077     mlength = REDLENG(mlength);
01078     if ( MulRat(BHEAD (UWORD *)tstop,tlength,(UWORD *)mm,mlength,
01079         (UWORD *)tout,&newlength) < 0 ) goto CalledFrom;
01080     mlength = INCLENG(newlength);
01081     tout += ABS(mlength);
01082     tout[-1] = mlength;
01083     *termout = tout - termout;
01084     AT.WorkPointer = tout;
01085     if ( argsdone && Generator(BHEAD termout,level) < 0 ) goto CalledFrom;
01086     mm = mnext; /* next term */
01087 }
01088 if ( action && ( gcdout != abuf[ii].buffer ) ) M_free(gcdout,"gcdout");
01089 /*
01090  #] Multiterm subexpressions :
01091  #[ Cleanup :
01092 */
01093 cleanup:;

```

```

01094     if ( action ) {
01095         for ( i = 0; i < numargs; i++ ) {
01096             if ( abuf[i].type == 0 ) { M_free(abuf[i].buffer,"argbuffer type 0"); }
01097             else if ( abuf[i].type == 1 ) {
01098                 d = abuf[i].dollar;
01099                 if ( d->factors ) M_free(d->factors,"Dollar factors");
01100                 M_free(d,"Copy of dollar variable");
01101             }
01102             else if ( abuf[i].type == 2 ) { M_free(abuf[i].buffer,"CreateExpression"); }
01103         }
01104         M_free(abuf,"argbuffer");
01105     }
01106 /*
01107     #] Cleanup :
01108 */
01109     AT.pWorkPointer = args;
01110     AT.WorkPointer = termout;
01111     return(0);
01112
01113 CalledFrom:
01114     MLOCK(ErrorMessageLock);
01115     MesCall("GCDfunction");
01116     MUNLOCK(ErrorMessageLock);
01117     SETERROR(-1)
01118     return(-1);
01119 }
01120
01121 /*
01122     #] GCDfunction :
01123     #[ GCDfunction3 :
01124
01125     Finds the GCD of the two arguments which are buffers with terms.
01126     In principle the first buffer can have only one term.
01127
01128     If both buffers have more than one term, we need to replace all
01129     non-symbolic objects by generated symbols and substitute that back
01130     afterwards. The rest we leave to the powerful routines.
01131     Philosophical problem: What do we do with GCD_(x/z+y,x+y*z) ?
01132
01133     Method:
01134     If we have only negative powers of z and no positive powers we let
01135     the EXTRASYMBOLS do their job. When mixed, multiply the arguments with
01136     the negative powers with enough powers of z to eliminate the negative powers.
01137     The DENOMINATOR function is always eliminated with the mechanism as we
01138     cannot tell whether there are positive powers of its contents.
01139 */
01140 WORD *GCDfunction3(PHEAD WORD *in1, WORD *in2)
01141 {
01142     GETBIDENTITY
01143     WORD oldsorttype = AR.SortType, *ow = AT.WorkPointer;;
01144     WORD *t, *tt, *gcdout, *term1, *term2, *confree1, *confree2, *gcdout1, *proper1, *proper2;
01145     int i, actionflag1, actionflag2;
01146     WORD startebuf = cbuf[AT.ebufnum].numrhs;
01147     WORD tryterm1, tryterm2;
01148     if ( in2[*in2] == 0 ) { t = in1; in1 = in2; in2 = t; }
01149     if ( in1[*in1] == 0 ) { /* First input with only one term */
01150         gcdout = (WORD *)Malloc1(([*in1+1]*sizeof(WORD),"gcdout");
01151         i = *in1; t = gcdout; tt = in1; NCOPY(t,tt,i); *t = 0;
01152         t = in2;
01153         while ( *t ) {
01154             GCDterms(BHEAD gcdout,t,gcdout);
01155             if ( gcdout[0] == 4 && gcdout[1] == 1
01156                 && gcdout[2] == 1 && gcdout[3] == 3 ) break;
01157             t += *t;
01158         }
01159         AT.WorkPointer = ow;
01160         return(gcdout);
01161     }
01162 }
01163 /*
01164     We need to take out the content from the two expressions
01165     and determine their GCD. This plays with the negative powers!
01166 */
01167     AR.SortType = SORTHIGFIRST;
01168     term1 = TermMalloc("GCDfunction3-a");
01169     term2 = TermMalloc("GCDfunction3-b");
01170
01171     confree1 = TakeContent(BHEAD in1,term1);
01172     tryterm1 = AN.tryterm; AN.tryterm = 0;
01173     confree2 = TakeContent(BHEAD in2,term2);
01174     tryterm2 = AN.tryterm; AN.tryterm = 0;
01175 /*
01176     confree1 = TakeSymbolContent(BHEAD in1,term1);
01177     confree2 = TakeSymbolContent(BHEAD in2,term2);
01178 */
01179     GCDterms(BHEAD term1,term2,term1);
01180     TermFree(term2,"GCDfunction3-b");

```

```

01181 /*
01182     Now we have to replace all non-symbols and symbols to a negative power
01183     by extra symbols.
01184 */
01185 if ( ( proper1 = PutExtraSymbols(BHEAD confree1,startebuf,&actionflag1) ) == 0 ) goto CalledFrom;
01186 if ( confree1 != in1 ) {
01187     if ( tryterm1 ) { TermFree(confree1,"TakeContent"); }
01188     else { M_free(confree1,"TakeContent"); }
01189 }
01190 /*
01191     TermFree(confree1,"TakeSymbolContent");
01192 */
01193 if ( ( proper2 = PutExtraSymbols(BHEAD confree2,startebuf,&actionflag2) ) == 0 ) goto CalledFrom;
01194 if ( confree2 != in2 ) {
01195     if ( tryterm2 ) { TermFree(confree2,"TakeContent"); }
01196     else { M_free(confree2,"TakeContent"); }
01197 }
01198 /*
01199     TermFree(confree2,"TakeSymbolContent");
01200 */
01201 /*
01202     And now the real work:
01203 */
01204 gcdout1 = poly_gcd(BHEAD proper1,proper2,0);
01205 M_free(proper1,"PutExtraSymbols");
01206 M_free(proper2,"PutExtraSymbols");
01207
01208 AR.SortType = oldsorttype;
01209 if ( actionflag1 || actionflag2 ) {
01210     if ( ( gcdout = TakeExtraSymbols(BHEAD gcdout1,startebuf) ) == 0 ) goto CalledFrom;
01211     M_free(gcdout1,"gcdout");
01212 }
01213 else {
01214     gcdout = gcdout1;
01215 }
01216
01217 cbuf[AT.ebufnum].numrhs = startebuf;
01218 /*
01219     Now multiply gcdout by term1
01220 */
01221 if ( term1[0] != 4 || term1[3] != 3 || term1[1] != 1 || term1[2] != 1 ) {
01222     AN.tryterm = -1;
01223     if ( ( gcdout1 = MultiplyWithTerm(BHEAD gcdout,term1,2) ) == 0 ) goto CalledFrom;
01224     AN.tryterm = 0;
01225     M_free(gcdout,"gcdout");
01226     gcdout = gcdout1;
01227 }
01228 TermFree(term1,"GCDfunction3-a");
01229 AT.WorkPointer = ow;
01230 return(gcdout);
01231 CalledFrom:
01232 AN.tryterm = 0;
01233 MLOCK(ErrorMessageLock);
01234 MesCall("GCDfunction3");
01235 MUNLOCK(ErrorMessageLock);
01236 return(0);
01237 }
01238
01239 /*
01240     #] GCDfunction3 :
01241     #[ PutExtraSymbols :
01242 */
01243
01244 WORD *PutExtraSymbols(PHEAD WORD *in,WORD startebuf,int *actionflag)
01245 {
01246     WORD *termout = AT.WorkPointer;
01247     int action;
01248     *actionflag = 0;
01249     NewSort(BHEAD0);
01250     while ( *in ) {
01251         if ( ( action = LocalConvertToPoly(BHEAD in,termout,startebuf,0) ) < 0 ) {
01252             LowerSortLevel();
01253             goto CalledFrom;
01254         }
01255         if ( action > 0 ) *actionflag = 1;
01256         StoreTerm(BHEAD termout);
01257         in += *in;
01258     }
01259     if ( EndSort(BHEAD (WORD *)((void *)(&termout)),2) < 0 ) goto CalledFrom;
01260     return(termout);
01261 CalledFrom:
01262     MLOCK(ErrorMessageLock);
01263     MesCall("PutExtraSymbols");
01264     MUNLOCK(ErrorMessageLock);
01265     return(0);
01266 }
01267

```

```

01268 /*
01269      #] PutExtraSymbols :
01270      #[ TakeExtraSymbols :
01271 */
01272
01273 WORD *TakeExtraSymbols(PHEAD WORD *in,WORD startebuf)
01274 {
01275     CBUF *C = cbuf+AC.cbunum;
01276     CBUF *CC = cbuf+AT.ebunum;
01277     WORD *oldworkpointer = AT.WorkPointer, *termout;
01278
01279     termout = AT.WorkPointer;
01280     NewSort(BHEAD0);
01281     while ( *in ) {
01282         if ( ConvertFromPoly(BHEAD
in,termout,numxsymbol,CC->numrhs-startebuf+numxsymbol,startebuf-numxsymbol,1) <= 0 ) {
01283             LowerSortLevel();
01284             goto CalledFrom;
01285         }
01286         in += *in;
01287         AT.WorkPointer = termout + *termout;
01288     /*
01289         ConvertFromPoly leaves terms with subexpressions. Hence:
01290     */
01291         if ( Generator(BHEAD termout,C->numlhs) ) {
01292             LowerSortLevel();
01293             goto CalledFrom;
01294         }
01295     }
01296     AT.WorkPointer = oldworkpointer;
01297     if ( EndSort(BHEAD (WORD *)((void *)(&termout)),2) < 0 ) goto CalledFrom;
01298     return(termout);
01299
01300 CalledFrom:
01301     MLOCK(ErrorMessageLock);
01302     MesCall("TakeExtraSymbols");
01303     MUNLOCK(ErrorMessageLock);
01304     return(0);
01305 }
01306
01307 /*
01308      #] TakeExtraSymbols :
01309      #[ MultiplyWithTerm :
01310 */
01311
01312 WORD *MultiplyWithTerm(PHEAD WORD *in, WORD *term, WORD par)
01313 {
01314     WORD *termout, *t, *tt, *tstop, *ttstop;
01315     WORD length, length1, length2;
01316     WORD oldsorttype = AR.SortType;
01317     COMPARE oldcompareroutine = (COMPARE) (AR.CompareRoutine);
01318     AR.CompareRoutine = (COMPAREDUMMY) (&CompareSymbols);
01319
01320     if ( par == 0 || par == 2 ) AR.SortType = SORTHIGHFIRST;
01321     else AR.SortType = SORTLOWFIRST;
01322     termout = AT.WorkPointer;
01323     NewSort(BHEAD0);
01324     while ( *in ) {
01325         tt = termout + 1;
01326         tstop = in + *in; tstop -= ABS(tstop[-1]); t = in + 1;
01327         while ( t < tstop ) *t++ = *t++;
01328         ttstop = term + *term; ttstop -= ABS(ttstop[-1]); t = term + 1;
01329         while ( t < ttstop ) *t++ = *t++;
01330         length1 = REDLENG(in[*in-1]); length2 = REDLENG(term[*term-1]);
01331         if ( MulRat(BHEAD (UWORD *)tstop,length1,
01332             (UWORD *)ttstop,length2,(UWORD *)tt,&length) ) goto CalledFrom;
01333         length = INCLENG(length);
01334         tt += ABS(length); tt[-1] = length;
01335         *termout = tt - termout;
01336         SymbolNormalize(termout);
01337         StoreTerm(BHEAD termout);
01338         in += *in;
01339     }
01340     if ( par == 2 ) {
01341     /*
01342         if ( AN.tryterm == 0 ) AN.tryterm = 1; */
01343         AN.tryterm = 0; /* For now */
01344         if ( EndSort(BHEAD (WORD *)((void *)(&termout)),2) < 0 ) goto CalledFrom;
01345     }
01346     else {
01347         if ( EndSort(BHEAD termout,1) < 0 ) goto CalledFrom;
01348     }
01349     AR.CompareRoutine = (COMPAREDUMMY)oldcompareroutine;
01350
01351     AR.SortType = oldsorttype;
01352     return(termout);
01353 }

```



```

01354 CalledFrom:
01355     MLOCK(ErrorMessageLock);
01356     MesCall("MultiplyWithTerm");
01357     MUNLOCK(ErrorMessageLock);
01358     return(0);
01359 }
01360
01361 /*
01362     #] MultiplyWithTerm :
01363     #[ TakeContent :
01364 */
01376 WORD *TakeContent(PHEAD WORD *in, WORD *term)
01377 {
01378     GETBIDENTITY
01379     WORD *t, *tstop, *tcom, *tout, *tstore, *r, *rstop, *m, *mm, *w, *ww, *wterm;
01380     WORD *tnext, *tt, *tterm, code[2];
01381     WORD *inp, a, *den;
01382     int i, j, k, action = 0, sign;
01383     UWORD *GCDbuffer, *GCDbuffer2, *LCMbuffer, *LCMbuffer2, *ap;
01384     WORD GCDlen, GCDlen2, LCMlen, LCMlen2, length, redlength, len1, len2;
01385     tout = tstore = term+1;
01386 /*
01387     #[ INDEX :
01388 */
01389     t = in;
01390     tnext = t + *t;
01391     tstop = tnext-ABS(tnext[-1]);
01392     t++;
01393     while ( t < tstop ) {
01394         if ( *t == INDEX ) {
01395             i = t[1]; NCOPY(tout,t,i); break;
01396         }
01397         else t += t[1];
01398     }
01399     if ( tout > tstore ) { /* There are indices in the first term */
01400         t = tnext;
01401         while ( *t ) {
01402             tnext = t + *t;
01403             rstop = tnext - ABS(tnext[-1]);
01404             r = t+1;
01405             if ( r == rstop ) goto noindices;
01406             while ( r < rstop ) {
01407                 if ( *r != INDEX ) { r += r[1]; continue; }
01408                 m = tstore+2;
01409                 while ( m < tout ) {
01410                     for ( i = 2; i < r[1]; i++ ) {
01411                         if ( *m == r[i] ) break;
01412                         if ( *m > r[i] ) continue;
01413                         mm = m+1;
01414                         while ( mm < tout ) { mm[-1] = mm[0]; mm++; }
01415                         tout--; tstore[1]--; m--;
01416                         break;
01417                     }
01418                     m++;
01419                 }
01420             }
01421             if ( r >= rstop || tout <= tstore+2 ) {
01422                 tout = tstore; break;
01423             }
01424         }
01425         if ( tout > tstore+2 ) { /* Now we have to take out what is in tstore */
01426             t = in; w = in;
01427             while ( *t ) {
01428                 wterm = w;
01429                 tnext = t + *t; t++; w++;
01430                 while ( *t != INDEX ) { i = t[1]; NCOPY(w,t,i); }
01431                 tt = t + t[1]; t += 2; r = tstore+2; ww = w; *w++ = INDEX; w++;
01432                 while ( r < tout && t < tt ) {
01433                     if ( *r > *t ) { *w++ = *t++; }
01434                     else if ( *r == *t ) { r++; t++; }
01435                     else goto CalledFrom;
01436                 }
01437                 if ( r < tout ) goto CalledFrom;
01438                 while ( t < tt ) *w++ = *t++;
01439                 ww[1] = w - ww;
01440                 if ( ww[1] == 2 ) w = ww;
01441                 while ( t < tnext ) *w++ = *t++;
01442                 *wterm = w - wterm;
01443             }
01444             *w = 0;
01445         }
01446     noindices:
01447         tstore = tout;
01448     }
01449 /*
01450     #] INDEX :
01451     #[ VECTOR/DELTA :

```

```

01452 */
01453 code[0] = VECTOR; code[1] = DELTA;
01454 for ( k = 0; k < 2; k++ ) {
01455     t = in;
01456     tnext = t + *t;
01457     tstop = tnext-ABS(tnext[-1]);
01458     t++;
01459     while ( t < tstop ) {
01460         if ( *t == code[k] ) {
01461             i = t[1]; NCOPY(tout,t,i); break;
01462         }
01463         else t += t[1];
01464     }
01465     if ( tout > tstore ) { /* There are vectors in the first term */
01466         t = tnext;
01467         while ( *t ) {
01468             tnext = t + *t;
01469             rstop = tnext - ABS(tnext[-1]);
01470             r = t+1;
01471             if ( r == rstop ) { tstore = tout; goto novectors; }
01472             while ( r < rstop ) {
01473                 if ( *r != code[k] ) { r += r[1]; continue; }
01474                 m = tstore+2;
01475                 while ( m < tout ) {
01476                     for ( i = 2; i < r[1]; i += 2 ) {
01477                         if ( *m == r[i] && m[1] == r[i+1] ) break;
01478                         if ( *m > r[i] || ( *m == r[i] && m[1] > r[i+1] ) ) continue;
01479                         mm = m+2;
01480                         while ( mm < tout ) { mm[-2] = mm[0]; mm[-1] = mm[1]; mm += 2; }
01481                         tout -= 2; tstore[1] -= 2; m -= 2;
01482                         break;
01483                     }
01484                     m += 2;
01485                 }
01486             }
01487             if ( r >= rstop || tout <= tstore+2 ) {
01488                 tout = tstore; break;
01489             }
01490         }
01491         if ( tout > tstore+2 ) { /* Now we have to take out what is in tstore */
01492             t = in; w = in;
01493             while ( *t ) {
01494                 wterm = w;
01495                 tnext = t + *t; t++; w++;
01496                 while ( *t != code[k] ) { i = t[1]; NCOPY(w,t,i); }
01497                 tt = t + t[1]; t += 2; r = tstore+2; ww = w; *w++ = code[k]; w++;
01498                 while ( r < tout && t < tt ) {
01499                     if ( ( *r > *t ) || ( *r == *t && r[1] > t[1] ) )
01500                         { *w++ = *t++; *w++ = *t++; }
01501                     else if ( *r == *t && r[1] == t[1] ) { r += 2; t += 2; }
01502                     else goto CalledFrom;
01503                 }
01504                 if ( r < tout ) goto CalledFrom;
01505                 while ( t < tt ) *w++ = *t++;
01506                 ww[1] = w - ww;
01507                 if ( ww[1] == 2 ) w = ww;
01508                 while ( t < tnext ) *w++ = *t++;
01509                 *wterm = w - wterm;
01510             }
01511             *w = 0;
01512         }
01513         tstore = tout;
01514     }
01515 }
01516 novectors;;
01517 /*
01518 #] VECTOR/DELTA :
01519 #[ FUNCTIONS :
01520 */
01521 t = in;
01522 tnext = t + *t;
01523 tstop = tnext-ABS(tnext[-1]);
01524 t++;
01525 tcom = 0;
01526 while ( t < tstop ) {
01527     if ( *t >= FUNCTION ) {
01528         if ( functions[*t-FUNCTION].commute ) {
01529             if ( tcom == 0 ) { tcom = tstore; }
01530             else {
01531                 for ( i = 0; i < t[1]; i++ ) {
01532                     if ( t[i] != tcom[i] ) {
01533                         MLOCK(ErrorMessageLock);
01534                         MesPrint("GCD or factorization of more than one noncommuting object not
allowed");
01535                         MUNLOCK(ErrorMessageLock);
01536                         goto CalledFrom;
01537                     }

```

```

01538         }
01539     }
01540 }
01541     i = t[1]; NCOPY(tstore,t,i);
01542 }
01543     else t += t[1];
01544 }
01545 if ( tout > tstore ) {
01546     t = tnext;
01547     while ( *t ) {
01548         tnext = t + *t; tstop = tnext - ABS(tnext[-1]); t++;
01549         if ( t == tstop ) goto nofunctions;
01550         r = tstore;
01551         while ( r < tout ) {
01552             tt = t;
01553             while ( tt < tstop ) {
01554                 for ( i = 0; i < r[1]; i++ ) {
01555                     if ( r[i] != tt[i] ) break;
01556                 }
01557                 if ( i == r[1] ) { r += r[1]; goto nextrl; }
01558             }
01559             /*
01560              Not encountered in this term. Scratch from list
01561             */
01562             m = r; mm = r + r[1];
01563             while ( mm < tout ) *m++ = *mm++;
01564             tout = m;
01565 nextrl:;
01566         }
01567         if ( tout <= tstore ) break;
01568         t += *t;
01569     }
01570 }
01571 if ( tout > tstore ) {
01572     /*
01573      Now we have one or more functions left that are common in all terms.
01574      Take them out. We do this one by one.
01575     */
01576     r = tstore;
01577     while ( r < tout ) {
01578         t = in; ww = in; w = ww+1;
01579         while ( *t ) {
01580             tnext = t + *t;
01581             t++;
01582             for(;;) {
01583                 for ( i = 0; i < r[1]; i++ ) {
01584                     if ( t[i] != r[i] ) {
01585                         j = t[1]; NCOPY(w,t,j);
01586                         break;
01587                     }
01588                 }
01589                 if ( i == r[1] ) {
01590                     t += t[1];
01591                     while ( t < tnext ) *w++ = *t++;
01592                     *ww = w - ww;
01593                     break;
01594                 }
01595             }
01596         }
01597         r += r[1];
01598         *w = 0;
01599     }
01600 nofunctions:
01601     tstore = tout;
01602 }
01603 /*
01604  #] FUNCTIONS :
01605  #[ SYMBOL :
01606
01607  We make a list of symbols and their minimal powers.
01608  This includes negative powers. In the end we have to multiply by the
01609  inverse of this list. That takes out all negative powers and leaves
01610  things ready for further processing.
01611  */
01612 tterm = AT.WorkPointer; tt = tterm+1;
01613 tout[0] = SYMBOL; tout[1] = 2;
01614 t = in;
01615 tnext = t + *t; tstop = tnext - ABS(tnext[-1]); t++;
01616 while ( t < tstop ) {
01617     if ( *t == SYMBOL ) {
01618         for ( i = 0; i < t[1]; i++ ) tout[i] = t[i];
01619         break;
01620     }
01621     t += t[1];
01622 }
01623 t = tnext;
01624 while ( *t ) {

```

```

01625         tnext = t + *t; tstop = tnext - ABS(tnext[-1]); t++;
01626         if ( t == tstop ) {
01627             tout[1] = 2;
01628             break;
01629         }
01630         else {
01631             while ( t < tstop ) {
01632                 if ( *t == SYMBOL ) {
01633                     MergeSymbolLists(BHEAD tout,t,-1);
01634                     break;
01635                 }
01636                 t += t[1];
01637             }
01638             t = tnext;
01639         }
01640     }
01641     if ( tout[1] > 2 ) {
01642         t = tout;
01643         tt[0] = t[0]; tt[1] = t[1];
01644         for ( i = 2; i < t[1]; i += 2 ) {
01645             tt[i] = t[i]; tt[i+1] = -t[i+1];
01646         }
01647         tt += tt[1];
01648         tout += tout[1];
01649         action++;
01650     }
01651     /*
01652     #] SYMBOL :
01653     #[ DOTPRODUCT :
01654
01655     We make a list of dotproducts and their minimal powers.
01656     This includes negative powers. In the end we have to multiply by the
01657     inverse of this list. That takes out all negative powers and leaves
01658     things ready for further processing.
01659     */
01660     tout[0] = DOTPRODUCT; tout[1] = 2;
01661     t = in;
01662     while ( *t ) {
01663         tnext = t + *t; tstop = tnext - ABS(tnext[-1]); t++;
01664         if ( t == tstop ) {
01665             tout[1] = 2;
01666             break;
01667         }
01668         while ( t < tstop ) {
01669             if ( *t == DOTPRODUCT ) {
01670                 MergeDotproductLists(BHEAD tout,t,-1);
01671                 break;
01672             }
01673             t += t[1];
01674         }
01675         t = tnext;
01676     }
01677     if ( tout[1] > 2 ) {
01678         t = tout;
01679         tt[0] = t[0]; tt[1] = t[1];
01680         for ( i = 2; i < t[1]; i += 3 ) {
01681             tt[i] = t[i]; tt[i+1] = t[i+1]; tt[i+2] = -t[i+2];
01682         }
01683         tt += tt[1];
01684         tout += tout[1];
01685         action++;
01686     }
01687     /*
01688     #] DOTPRODUCT :
01689     #[ Coefficient :
01690
01691     Now we have to collect the GCD of the numerators
01692     and the LCM of the denominators.
01693     */
01694     AT.WorkPointer = tt;
01695     if ( AN.cmod != 0 ) {
01696         WORD x, ix, ip;
01697         t = in; tnext = t + *t; tstop = tnext - ABS(tnext[-1]);
01698         x = tstop[0];
01699         if ( tnext[-1] < 0 ) x += AC.cmod[0];
01700         if ( GetModInverses(x, (WORD) (AN.cmod[0]), &ix, &ip) ) goto CalledFrom;
01701         *tout++ = x; *tout++ = 1; *tout++ = 3;
01702         *tt++ = ix; *tt++ = 1; *tt++ = 3;
01703     }
01704     else {
01705         GCDBuffer = NumberMalloc("MakeInteger");
01706         GCDBuffer2 = NumberMalloc("MakeInteger");
01707         LCMbuffer = NumberMalloc("MakeInteger");
01708         LCMbuffer2 = NumberMalloc("MakeInteger");
01709         t = in;
01710         tnext = t + *t; length = tnext[-1];
01711         if ( length < 0 ) { sign = -1; length = -length; }

```

```

01712     else { sign = 1; }
01713     tstop = tnext - length;
01714     redlength = (length-1)/2;
01715     for ( i = 0; i < redlength; i++ ) {
01716         GCDbuffer[i] = (UWORD) (tstop[i]);
01717         LCMbuffer[i] = (UWORD) (tstop[redlength+i]);
01718     }
01719     GCDlen = LCMlen = redlength;
01720     while ( GCDbuffer[GCDlen-1] == 0 ) GCDlen--;
01721     while ( LCMbuffer[LCMlen-1] == 0 ) LCMlen--;
01722     t = tnext;
01723     while ( *t ) {
01724         tnext = t + *t; length = ABS(tnext[-1]);
01725         tstop = tnext - length; redlength = (length-1)/2;
01726         len1 = len2 = redlength;
01727         den = tstop + redlength;
01728         while ( tstop[len1-1] == 0 ) len1--;
01729         while ( den[len2-1] == 0 ) len2--;
01730         if ( GCDlen == 1 && GCDbuffer[0] == 1 ) {}
01731         else {
01732             GcdLong(BHEAD (UWORD *)tstop, len1, GCDbuffer, GCDlen, GCDbuffer2, &GCDlen2);
01733             ap = GCDbuffer; GCDbuffer = GCDbuffer2; GCDbuffer2 = ap;
01734             a = GCDlen; GCDlen = GCDlen2; GCDlen2 = a;
01735         }
01736         if ( len2 == 1 && den[0] == 1 ) {}
01737         else {
01738             GcdLong(BHEAD LCMbuffer, LCMlen, (UWORD *)den, len2, LCMbuffer2, &LCMlen2);
01739             DivLong((UWORD *)den, len2, LCMbuffer2, LCMlen2,
01740                 GCDbuffer2, &GCDlen2, (UWORD *)AT.WorkPointer, &a);
01741             Mullong(LCMbuffer, LCMlen, GCDbuffer2, GCDlen2, LCMbuffer2, &LCMlen2);
01742             ap = LCMbuffer; LCMbuffer = LCMbuffer2; LCMbuffer2 = ap;
01743             a = LCMlen; LCMlen = LCMlen2; LCMlen2 = a;
01744         }
01745         t = tnext;
01746     }
01747     if ( GCDlen != 1 || GCDbuffer[0] != 1 || LCMlen != 1 || LCMbuffer[0] != 1 ) {
01748         redlength = GCDlen; if ( LCMlen > GCDlen ) redlength = LCMlen;
01749         for ( i = 0; i < GCDlen; i++ ) *tout++ = (WORD) (GCDbuffer[i]);
01750         for ( ; i < redlength; i++ ) *tout++ = 0;
01751         for ( i = 0; i < LCMlen; i++ ) *tout++ = (WORD) (LCMbuffer[i]);
01752         for ( ; i < redlength; i++ ) *tout++ = 0;
01753         *tout++ = (2*redlength+1)*sign;
01754         for ( i = 0; i < LCMlen; i++ ) *ttt++ = (WORD) (LCMbuffer[i]);
01755         for ( ; i < redlength; i++ ) *ttt++ = 0;
01756         for ( i = 0; i < GCDlen; i++ ) *ttt++ = (WORD) (GCDbuffer[i]);
01757         for ( ; i < redlength; i++ ) *ttt++ = 0;
01758         *ttt++ = (2*redlength+1)*sign;
01759         action++;
01760     }
01761     else {
01762         *tout++ = 1; *tout++ = 1; *tout++ = 3*sign;
01763         *ttt++ = 1; *ttt++ = 1; *ttt++ = 3*sign;
01764         if ( sign != 1 ) action++;
01765     }
01766     *tout = 0;
01767     NumberFree(LCMbuffer2, "MakeInteger");
01768     NumberFree(LCMbuffer, "MakeInteger");
01769     NumberFree(GCDbuffer2, "MakeInteger");
01770     NumberFree(GCDbuffer, "MakeInteger");
01771 }
01772 /*
01773 #] Coefficient :
01774 #[ Multiply by the inverse content :
01775 */
01776 if ( action ) {
01777     *tterm = tt - tterm;
01778     AT.WorkPointer = tt;
01779     inp = MultiplyWithTerm(BHEAD in, tterm, 2);
01780     AT.WorkPointer = tterm;
01781     in = inp;
01782 }
01783 /*
01784 #] Multiply by the inverse content :
01785 */
01786 *term = tout - term;
01787 AT.WorkPointer = tterm;
01788 return(in);
01789 CalledFrom:
01790 MLOCK(ErrorMessageLock);
01791 MesCall("TakeContent");
01792 MUNLOCK(ErrorMessageLock);
01793 return(0);
01794 }
01795
01796 /*
01797 #] TakeContent :
01798 #[ MergeSymbolLists :

```

```

01799
01800     Merges the extra list into the old.
01801     If par == -1 we take minimum powers
01802     If par == 1 we take maximum powers
01803     If par == 0 we take minimum of the absolute value of the powers
01804             if one is positive and the other negative we get zero.
01805     We assume that the symbols are in order in both lists
01806 */
01807
01808 int MergeSymbolLists(PHEAD WORD *old, WORD *extra, int par)
01809 {
01810     GETBIDENTITY
01811     WORD *new = TermMalloc("MergeSymbolLists");
01812     WORD *t1, *t2, *fill;
01813     int i1, i2;
01814     fill = new + 2; *new = SYMBOL;
01815     i1 = old[1] - 2; i2 = extra[1] - 2;
01816     t1 = old + 2; t2 = extra + 2;
01817     switch ( par ) {
01818     case -1:
01819         while ( i1 > 0 && i2 > 0 ) {
01820             if ( *t1 > *t2 ) {
01821                 if ( t2[1] < 0 ) { *fill++ = *t2++; *fill++ = *t2++; }
01822                 else t2 += 2;
01823                 i2 -= 2;
01824             }
01825             else if ( *t1 < *t2 ) {
01826                 if ( t1[1] < 0 ) { *fill++ = *t1++; *fill++ = *t1++; }
01827                 else t1 += 2;
01828                 i1 -= 2;
01829             }
01830             else if ( t1[1] < t2[1] ) {
01831                 *fill++ = *t1++; *fill++ = *t1++; t2 += 2;
01832                 i1 -= 2; i2 -= 2;
01833             }
01834             else {
01835                 *fill++ = *t2++; *fill++ = *t2++; t1 += 2;
01836                 i1 -= 2; i2 -= 2;
01837             }
01838         }
01839         for ( ; i1 > 0; i1 -= 2 ) {
01840             if ( t1[1] < 0 ) { *fill++ = *t1++; *fill++ = *t1++; }
01841             else t1 += 2;
01842         }
01843         for ( ; i2 > 0; i2 -= 2 ) {
01844             if ( t2[1] < 0 ) { *fill++ = *t2++; *fill++ = *t2++; }
01845             else t2 += 2;
01846         }
01847         break;
01848     case 1:
01849         while ( i1 > 0 && i2 > 0 ) {
01850             if ( *t1 > *t2 ) {
01851                 if ( t2[1] > 0 ) { *fill++ = *t2++; *fill++ = *t2++; }
01852                 else t2 += 2;
01853                 i2 -= 2;
01854             }
01855             else if ( *t1 < *t2 ) {
01856                 if ( t1[1] > 0 ) { *fill++ = *t1++; *fill++ = *t1++; }
01857                 else t1 += 2;
01858                 i1 -= 2;
01859             }
01860             else if ( t1[1] > t2[1] ) {
01861                 *fill++ = *t1++; *fill++ = *t1++; t2 += 2;
01862                 i1 -= 2; i2 -= 2;
01863             }
01864             else {
01865                 *fill++ = *t2++; *fill++ = *t2++; t1 += 2;
01866                 i1 -= 2; i2 -= 2;
01867             }
01868         }
01869         for ( ; i1 > 0; i1 -= 2 ) {
01870             if ( t1[1] > 0 ) { *fill++ = *t1++; *fill++ = *t1++; }
01871             else t1 += 2;
01872         }
01873         for ( ; i2 > 0; i2 -= 2 ) {
01874             if ( t2[1] > 0 ) { *fill++ = *t2++; *fill++ = *t2++; }
01875             else t2 += 2;
01876         }
01877         break;
01878     case 0:
01879         while ( i1 > 0 && i2 > 0 ) {
01880             if ( *t1 > *t2 ) {
01881                 t2 += 2; i2 -= 2;
01882             }
01883             else if ( *t1 < *t2 ) {
01884                 t1 += 2; i1 -= 2;
01885             }

```

```

01886         else if ( ( t1[1] > 0 ) && ( t2[1] < 0 ) ) { t1 += 2; t2 += 2; i1 -= 2; i2 -= 2; }
01887         else if ( ( t1[1] < 0 ) && ( t2[1] > 0 ) ) { t1 += 2; t2 += 2; i1 -= 2; i2 -= 2; }
01888         else if ( t1[1] > 0 ) {
01889             if ( t1[1] < t2[1] ) {
01890                 *fill++ = *t1++; *fill++ = *t1++; t2 += 2; i2 -= 2;
01891             }
01892             else {
01893                 *fill++ = *t2++; *fill++ = *t2++; t1 += 2; i1 -= 2;
01894             }
01895         }
01896         else {
01897             if ( t2[1] < t1[1] ) {
01898                 *fill++ = *t2++; *fill++ = *t2++; t1 += 2; i1 -= 2; i2 -= 2;
01899             }
01900             else {
01901                 *fill++ = *t1++; *fill++ = *t1++; t2 += 2; i1 -= 2; i2 -= 2;
01902             }
01903         }
01904     }
01905     for ( ; i1 > 0; i1-- ) *fill++ = *t1++;
01906     for ( ; i2 > 0; i2-- ) *fill++ = *t2++;
01907     break;
01908 }
01909 i1 = new[1] = fill - new;
01910 t2 = new; t1 = old; NCOPY(t1,t2,i1);
01911 TermFree(new,"MergeSymbolLists");
01912 return(0);
01913 }
01914
01915 /*
01916     #] MergeSymbolLists :
01917     #[ MergeDotproductLists :
01918
01919     Merges the extra list into the old.
01920     If par == -1 we take minimum powers
01921     If par == 1 we take maximum powers
01922     If par == 0 we take minimum of the absolute value of the powers
01923         if one is positive and the other negative we get zero.
01924     We assume that the dotproducts are in order in both lists
01925 */
01926
01927 int MergeDotproductLists(PHEAD WORD *old, WORD *extra, int par)
01928 {
01929     GETBIDENTITY
01930     WORD *new = TermMalloc("MergeDotproductLists");
01931     WORD *t1, *t2, *fill;
01932     int i1,i2;
01933     fill = new + 2;
01934     i1 = old[1] - 2; i2 = extra[1] - 2;
01935     t1 = old + 2; t2 = extra + 2;
01936     switch ( par ) {
01937         case -1:
01938             while ( i1 > 0 && i2 > 0 ) {
01939                 if ( ( *t1 > *t2 ) || ( *t1 == *t2 && t1[1] > t2[1] ) ) {
01940                     if ( t2[2] < 0 ) { *fill++ = *t2++; *fill++ = *t2++; *fill++ = *t2++; }
01941                     else t2 += 3;
01942                 }
01943                 else if ( ( *t1 < *t2 ) || ( *t1 == *t2 && t1[1] < t2[1] ) ) {
01944                     if ( t1[2] < 0 ) { *fill++ = *t1++; *fill++ = *t1++; *fill++ = *t1++; }
01945                     else t1 += 3;
01946                 }
01947                 else if ( t1[2] < t2[2] ) {
01948                     *fill++ = *t1++; *fill++ = *t1++; *fill++ = *t1++; t2 += 3;
01949                 }
01950                 else {
01951                     *fill++ = *t2++; *fill++ = *t2++; *fill++ = *t2++; t1 += 3;
01952                 }
01953             }
01954             break;
01955         case 1:
01956             while ( i1 > 0 && i2 > 0 ) {
01957                 if ( ( *t1 > *t2 ) || ( *t1 == *t2 && t1[1] > t2[1] ) ) {
01958                     if ( t2[2] > 0 ) { *fill++ = *t2++; *fill++ = *t2++; *fill++ = *t2++; }
01959                     else t2 += 3;
01960                 }
01961                 else if ( ( *t1 < *t2 ) || ( *t1 == *t2 && t1[1] < t2[1] ) ) {
01962                     if ( t1[2] > 0 ) { *fill++ = *t1++; *fill++ = *t1++; *fill++ = *t1++; }
01963                     else t1 += 3;
01964                 }
01965                 else if ( t1[2] > t2[2] ) {
01966                     *fill++ = *t1++; *fill++ = *t1++; *fill++ = *t1++; t2 += 3;
01967                 }
01968                 else {
01969                     *fill++ = *t2++; *fill++ = *t2++; *fill++ = *t2++; t1 += 3;
01970                 }
01971             }
01972             break;

```

```

01973         case 0:
01974             while ( i1 > 0 && i2 > 0 ) {
01975                 if ( ( *t1 > *t2 ) || ( *t1 == *t2 && t1[1] > t2[1] ) ) {
01976                     t2 += 3;
01977                 }
01978                 else if ( ( *t1 < *t2 ) || ( *t1 == *t2 && t1[1] < t2[1] ) ) {
01979                     t1 += 3;
01980                 }
01981                 else if ( ( t1[2] > 0 ) && ( t2[2] < 0 ) ) { t1 += 3; t2 += 3; }
01982                 else if ( ( t1[2] < 0 ) && ( t2[2] > 0 ) ) { t1 += 3; t2 += 3; }
01983                 else if ( t1[2] > 0 ) {
01984                     if ( t1[2] < t2[2] ) {
01985                         *fill++ = *t1++; *fill++ = *t1++; *fill++ = *t1++; t2 += 3;
01986                     }
01987                     else {
01988                         *fill++ = *t2++; *fill++ = *t2++; *fill++ = *t2++; t1 += 3;
01989                     }
01990                 }
01991                 else {
01992                     if ( t2[2] < t1[2] ) {
01993                         *fill++ = *t2++; *fill++ = *t2++; *fill++ = *t2++; t1 += 3;
01994                     }
01995                     else {
01996                         *fill++ = *t1++; *fill++ = *t1++; *fill++ = *t1++; t2 += 3;
01997                     }
01998                 }
01999             }
02000             break;
02001         }
02002         i1 = new[1] = fill - new;
02003         t2 = new; t1 = old; NCOPY(t1,t2,i1);
02004         TermFree(new, "MergeDotproductLists");
02005         return(0);
02006     }
02007
02008     /*
02009         #] MergeDotproductLists :
02010         #[ CreateExpression :
02011
02012         Looks for the expression in the argument, reads it and puts it
02013         in a buffer. Returns the address of the buffer.
02014         We send the expression through the Generator system, because there
02015         may be unsubstituted (sub)expressions as in
02016         Local F = (a+b);
02017         Local G = gcd_(F,...);
02018     */
02019
02020     WORD *CreateExpression(PHEAD WORD nexp)
02021     {
02022         GETBIDENTITY
02023         CBUF *C = cbuf+AC.cbunum;
02024         POSITION startposition, oldposition;
02025         FILEHANDLE *fi;
02026         WORD *term, *oldipointer = AR.CompressPointer;
02027     ;
02028         switch ( Expressions[nexp].status ) {
02029             case HIDDENLEXPRESSION:
02030             case HIDDENGEXPRESSION:
02031             case DROPHLEXPRESSION:
02032             case DROPHGEXPRESSION:
02033             case UNHIDELEXPRESSION:
02034             case UNHIDEGEXPRESSION:
02035                 AR.GetOneFile = 2; fi = AR.hidefile;
02036                 break;
02037             default:
02038                 AR.GetOneFile = 0; fi = AR.infile;
02039                 break;
02040         }
02041         SeekScratch(fi,&oldposition);
02042         startposition = AS.OldOnFile[nexp];
02043         term = AT.WorkPointer;
02044         if ( GetOneTerm(BHEAD term,fi,&startposition,0) <= 0 ) goto CalledFrom;
02045         NewSort (BHEAD0);
02046         AR.CompressPointer = oldipointer;
02047         while ( GetOneTerm(BHEAD term,fi,&startposition,0) > 0 ) {
02048             AT.WorkPointer = term + *term;
02049             if ( Generator(BHEAD term,C->numlhs) ) {
02050                 LowerSortLevel();
02051                 goto CalledFrom;
02052             }
02053             AR.CompressPointer = oldipointer;
02054         }
02055         AT.WorkPointer = term;
02056         if ( EndSort (BHEAD (WORD *) ((void *) (&term)),2) < 0 ) goto CalledFrom;
02057         SetScratch(fi,&oldposition);
02058         return (term);
02059     CalledFrom:

```



```

02060     MLOCK(ErrorMessageLock);
02061     MesCall("CreateExpression");
02062     MUNLOCK(ErrorMessageLock);
02063     Terminate(-1);
02064     return(0);
02065 }
02066
02067 /*
02068     #] CreateExpression :
02069     #[ GCDterms :      GCD of two terms
02070
02071     Computes the GCD of two terms.
02072     Output in termout.
02073     termout may overlap with term1.
02074 */
02075
02076 int GCDterms(PHEAD WORD *term1, WORD *term2, WORD *termout)
02077 {
02078     GETBIDENTITY
02079     WORD *t1, *t1stop, *t1next, *t2, *t2stop, *t2next, *tout, *tt1, *tt2;
02080     int count1, count2, i, ii, x1, sign;
02081     WORD length1, length2;
02082     t1 = term1 + *term1; t1stop = t1 - ABS(t1[-1]); t1 = term1+1;
02083     t2 = term2 + *term2; t2stop = t2 - ABS(t2[-1]); t2 = term2+1;
02084     tout = termout+1;
02085     while ( t1 < t1stop ) {
02086         t1next = t1 + t1[1];
02087         t2 = term2+1;
02088         if ( *t1 == SYMBOL ) {
02089             while ( t2 < t2stop && *t2 != SYMBOL ) t2 += t2[1];
02090             if ( t2 < t2stop && *t2 == SYMBOL ) {
02091                 t2next = t2+t2[1];
02092                 tt1 = t1+2; tt2 = t2+2; count1 = 0;
02093                 while ( tt1 < t1next && tt2 < t2next ) {
02094                     if ( *tt1 < *tt2 ) tt1 += 2;
02095                     else if ( *tt1 > *tt2 ) tt2 += 2;
02096                     else if ( ( tt1[1] > 0 && tt2[1] < 0 ) ||
02097                             ( tt2[1] > 0 && tt1[1] < 0 ) ) {
02098                         tt1 += 2; tt2 += 2;
02099                     }
02100                     else {
02101                         x1 = tt1[1];
02102                         if ( tt1[1] < 0 ) { if ( tt2[1] > x1 ) x1 = tt2[1]; }
02103                         else { if ( tt2[1] < x1 ) x1 = tt2[1]; }
02104                         tout[count1+2] = *tt1;
02105                         tout[count1+3] = x1;
02106                         tt1 += 2; tt2 += 2;
02107                         count1 += 2;
02108                     }
02109                 }
02110                 if ( count1 > 0 ) {
02111                     *tout = SYMBOL; tout[1] = count1+2; tout += tout[1];
02112                 }
02113             }
02114         }
02115         else if ( *t1 == DOTPRODUCT ) {
02116             while ( t2 < t2stop && *t2 != DOTPRODUCT ) t2 += t2[1];
02117             if ( t2 < t2stop && *t2 == DOTPRODUCT ) {
02118                 t2next = t2+t2[1];
02119                 tt1 = t1+2; tt2 = t2+2; count1 = 0;
02120                 while ( tt1 < t1next && tt2 < t2next ) {
02121                     if ( *tt1 < *tt2 || ( *tt1 == *tt2 && tt1[1] < tt2[1] ) ) tt1 += 3;
02122                     else if ( *tt1 > *tt2 || ( *tt1 == *tt2 && tt1[1] > tt2[1] ) ) tt2 += 3;
02123                     else if ( ( tt1[2] > 0 && tt2[2] < 0 ) ||
02124                             ( tt2[2] > 0 && tt1[2] < 0 ) ) {
02125                         tt1 += 3; tt2 += 3;
02126                     }
02127                     else {
02128                         x1 = tt1[2];
02129                         if ( tt1[2] < 0 ) { if ( tt2[2] > x1 ) x1 = tt2[2]; }
02130                         else { if ( tt2[2] < x1 ) x1 = tt2[2]; }
02131                         tout[count1+2] = *tt1;
02132                         tout[count1+3] = tt1[1];
02133                         tout[count1+4] = x1;
02134                         tt1 += 3; tt2 += 3;
02135                         count1 += 3;
02136                     }
02137                 }
02138                 if ( count1 > 0 ) {
02139                     *tout = DOTPRODUCT; tout[1] = count1+2; tout += tout[1];
02140                 }
02141             }
02142         }
02143         else if ( *t1 == VECTOR ) {
02144             while ( t2 < t2stop && *t2 != VECTOR ) t2 += t2[1];
02145             if ( t2 < t2stop && *t2 == VECTOR ) {
02146                 t2next = t2+t2[1];

```

```

02147         tt1 = t1+2; tt2 = t2+2; count1 = 0;
02148         while ( tt1 < tlnext && tt2 < t2next ) {
02149             if ( *tt1 < *tt2 || ( *tt1 == *tt2 && tt1[1] < tt2[1] ) ) tt1 += 2;
02150             else if ( *tt1 > *tt2 || ( *tt1 == *tt2 && tt1[1] > tt2[1] ) ) tt2 += 2;
02151             else {
02152                 tout[count1+2] = *tt1;
02153                 tout[count1+3] = tt1[1];
02154                 tt1 += 2; tt2 += 2;
02155                 count1 += 2;
02156             }
02157         }
02158         if ( count1 > 0 ) {
02159             *tout = VECTOR; tout[1] = count1+2; tout += tout[1];
02160         }
02161     }
02162 }
02163 else if ( *t1 == INDEX ) {
02164     while ( t2 < t2stop && *t2 != INDEX ) t2 += t2[1];
02165     if ( t2 < t2stop && *t2 == INDEX ) {
02166         t2next = t2+t2[1];
02167         tt1 = t1+2; tt2 = t2+2; count1 = 0;
02168         while ( tt1 < tlnext && tt2 < t2next ) {
02169             if ( *tt1 < *tt2 ) tt1 += 1;
02170             else if ( *tt1 > *tt2 ) tt2 += 1;
02171             else {
02172                 tout[count1+2] = *tt1;
02173                 tt1 += 1; tt2 += 1;
02174                 count1 += 1;
02175             }
02176         }
02177         if ( count1 > 0 ) {
02178             *tout = INDEX; tout[1] = count1+2; tout += tout[1];
02179         }
02180     }
02181 }
02182 else if ( *t1 == DELTA ) {
02183     while ( t2 < t2stop && *t2 != DELTA ) t2 += t2[1];
02184     if ( t2 < t2stop && *t2 == DELTA ) {
02185         t2next = t2+t2[1];
02186         tt1 = t1+2; tt2 = t2+2; count1 = 0;
02187         while ( tt1 < tlnext && tt2 < t2next ) {
02188             if ( *tt1 < *tt2 || ( *tt1 == *tt2 && tt1[1] < tt2[1] ) ) tt1 += 2;
02189             else if ( *tt1 > *tt2 || ( *tt1 == *tt2 && tt1[1] > tt2[1] ) ) tt2 += 2;
02190             else {
02191                 tout[count1+2] = *tt1;
02192                 tout[count1+3] = tt1[1];
02193                 tt1 += 2; tt2 += 2;
02194                 count1 += 2;
02195             }
02196         }
02197         if ( count1 > 0 ) {
02198             *tout = DELTA; tout[1] = count1+2; tout += tout[1];
02199         }
02200     }
02201 }
02202 else if ( *t1 >= FUNCTION ) { /* noncommuting functions? Forbidden! */
02203     /*
02204         Count how many times this function occurs.
02205         Then count how many times it is in term2.
02206     */
02207     count1 = 1;
02208     while ( tlnext < t1stop && *t1 == *tlnext && t1[1] == tlnext[1] ) {
02209         for ( i = 2; i < t1[1]; i++ ) {
02210             if ( t1[i] != tlnext[i] ) break;
02211         }
02212         if ( i < t1[1] ) break;
02213         count1++;
02214         tlnext += tlnext[1];
02215     }
02216     count2 = 0;
02217     while ( t2 < t2stop ) {
02218         if ( *t2 == *t1 && t2[1] == t1[1] ) {
02219             for ( i = 2; i < t1[1]; i++ ) {
02220                 if ( t2[i] != t1[i] ) break;
02221             }
02222             if ( i >= t1[1] ) count2++;
02223         }
02224         t2 += t2[1];
02225     }
02226     if ( count1 < count2 ) count2 = count1; /* number of common occurrences */
02227     if ( count2 > 0 ) {
02228         if ( tout == t1 ) {
02229             while ( count2 > 0 ) { tout += tout[1]; count2--; }
02230         }
02231         else {
02232             i = t1[1]*count2;
02233             NCOPY(tout,t1,i);

```

```

02234     }
02235     }
02236 }
02237 t1 = tlnext;
02238 }
02239 /*
02240     Now the coefficients. They are in t1stop and t2stop. Should go to tout.
02241 */
02242 sign = 1;
02243 length1 = term1[*term1-1]; ii = i = ABS(length1); t1 = t1stop;
02244 if ( t1 != tout ) { NCOPY(tout,t1,i); tout -= ii; }
02245 length2 = term2[*term2-1];
02246 if ( length1 < 0 && length2 < 0 ) sign = -1;
02247 if ( AccumGCD(BHEAD (UWORD *)tout,&length1,(UWORD *)t2stop,length2) ) {
02248     MLOCK(ErrorMessageLock);
02249     MesCall("GCDterms");
02250     MUNLOCK(ErrorMessageLock);
02251     SETERROR(-1)
02252 }
02253 if ( sign < 0 && length1 > 0 ) length1 = -length1;
02254 tout += ABS(length1); tout[-1] = length1;
02255 *termout = tout - termout; *tout = 0;
02256 return(0);
02257 }
02258 /*
02259     #] GCDterms :
02260     #[ ReadPolyRatFun :
02261 */
02262 /*
02263 int ReadPolyRatFun(PHEAD WORD *term)
02264 {
02265     WORD *oldworkpointer = AT.WorkPointer;
02266     int flag, i;
02267     WORD *t, *fun, *nextt, *num, *den, *t1, *t2, size, numsize, densize;
02268     WORD *term1, *term2, *confree1, *confree2, *gcd, *num1, *den1, move, *newnum, *newden;
02269     WORD *tstop, *m1, *m2;
02270     WORD oldsorttype = AR.SortType;
02271     COMPARE oldcompareroutine = (COMPARE) (AR.CompareRoutine);
02272     AR.SortType = SORTHIGFIRST;
02273     AR.CompareRoutine = (COMPAREDUMMY) (&CompareSymbols);
02274     tstop = term + *term; tstop -= ABS(tstop[-1]);
02275     if ( term + *term == AT.WorkPointer ) flag = 1;
02276     else flag = 0;
02277     t = term+1;
02278     while ( t < tstop ) {
02279         if ( *t != AR.PolyFun ) { t += t[1]; continue; }
02280         if ( ( t[2] & MUSTCLEANPRF ) == 0 ) { t += t[1]; continue; }
02281         fun = t;
02282         nextt = t + t[1];
02283         if ( fun[1] > FUNHEAD && fun[FUNHEAD] == -SNUMBER && fun[FUNHEAD+1] == 0 )
02284             { *term = 0; break; }
02285         if ( FromPolyRatFun(BHEAD fun, &num, &den) > 0 ) { t = nextt; continue; }
02286         if ( *num == ARGHEAD ) { *term = 0; break; }
02287     }
02288     /*
02289         Now we have num and den. Both are in general argument notation,
02290         but can also be used as expressions as in num+ARGHEAD, den+ARGHEAD.
02291         We need the gcd. For this we have to take out the contents
02292         because PreGCD does not like contents.
02293     */
02294     term1 = TermMalloc("ReadPolyRatFun");
02295     term2 = TermMalloc("ReadPolyRatFun");
02296     confree1 = TakeSymbolContent(BHEAD num+ARGHEAD,term1);
02297     confree2 = TakeSymbolContent(BHEAD den+ARGHEAD,term2);
02298     GCDclean(BHEAD term1,term2);
02299     gcd = PreGCD(BHEAD confree1,confree2,1); /*
02300     gcd = poly_gcd(BHEAD confree1,confree2,1);
02301     newnum = PolyDiv(BHEAD confree1,gcd,"ReadPolyRatFun");
02302     newden = PolyDiv(BHEAD confree2,gcd,"ReadPolyRatFun");
02303     TermFree(confree2,"ReadPolyRatFun");
02304     TermFree(confree1,"ReadPolyRatFun");
02305     num1 = MULfunc(BHEAD term1,newnum);
02306     den1 = MULfunc(BHEAD term2,newden);
02307     TermFree(newnum,"ReadPolyRatFun");
02308     TermFree(newden,"ReadPolyRatFun");
02309     /*
02310     M_free(gcd,"poly_gcd"); /*
02311     TermFree(gcd,"poly_gcd");
02312     TermFree(term1,"ReadPolyRatFun");
02313     TermFree(term2,"ReadPolyRatFun");
02314     /*
02315         Now we can put the function back together.
02316         Notice that we cannot use ToFast, because there is no reservation
02317         for the header of the argument. Fortunately there are only two
02318         types of fast arguments.
02319     */
02320     if ( num1[0] == 4 && num1[4] == 0 && num1[2] == 1 && num1[1] > 0 ) {

```

```

02321         numsize = 2; numl[0] = -SNUMBER;
02322         if ( numl[3] < 0 ) numl[1] = -numl[1];
02323     }
02324     else if ( numl[0] == 8 && numl[8] == 0 && numl[7] == 3 && numl[6] == 1
02325         && numl[5] == 1 && numl[1] == SYMBOL && numl[4] == 1 ) {
02326         numsize = 2; numl[0] = -SYMBOL; numl[1] = numl[3];
02327     }
02328     else { m1 = numl; while ( *m1 ) m1 += *m1; numsize = (m1-numl)+ARGHEAD; }
02329     if ( denl[0] == 4 && denl[4] == 0 && denl[2] == 1 && denl[1] > 0 ) {
02330         densize = 2; denl[0] = -SNUMBER;
02331         if ( denl[3] < 0 ) denl[1] = -denl[1];
02332     }
02333     else if ( denl[0] == 8 && denl[8] == 0 && denl[7] == 3 && denl[6] == 1
02334         && denl[5] == 1 && denl[1] == SYMBOL && denl[4] == 1 ) {
02335         densize = 2; denl[0] = -SYMBOL; denl[1] = denl[3];
02336     }
02337     else { m2 = denl; while ( *m2 ) m2 += *m2; densize = (m2-denl)+ARGHEAD; }
02338     size = FUNHEAD+numsize+densize;
02339
02340     if ( size > fun[1] ) {
02341         move = size - fun[1];
02342         t1 = term+*term; t2 = t1+move;
02343         while ( t1 > nexttt ) *--t2 = *--t1;
02344         tstop += move; nexttt += move;
02345         *term += move;
02346     }
02347     else if ( size < fun[1] ) {
02348         move = fun[1]-size;
02349         t2 = fun+size; t1 = nexttt;
02350         tstop -= move; nexttt -= move;
02351         t = term+*term;
02352         while ( t1 < t ) *t2++ = *t1++;
02353         *term -= move;
02354     }
02355     else { /* no need to move anything */ }
02356     fun[1] = size; fun[2] = 0;
02357     t2 = fun+FUNHEAD; t1 = numl;
02358     if ( *numl < 0 ) { *t2++ = numl[0]; *t2++ = numl[1]; }
02359     else { *t2++ = numsize; *t2++ = 0; FILLARG(t2);
02360         i = numsize-ARGHEAD; NCOPY(t2,t1,i) }
02361     t1 = denl;
02362     if ( *denl < 0 ) { *t2++ = denl[0]; *t2++ = denl[1]; }
02363     else { *t2++ = densize; *t2++ = 0; FILLARG(t2);
02364         i = densize-ARGHEAD; NCOPY(t2,t1,i) }
02365
02366     TermFree(numl,"MULfunc");
02367     TermFree(denl,"MULfunc");
02368     t = nexttt;
02369 }
02370
02371 if ( flag ) AT.WorkPointer = term + *term;
02372 else AT.WorkPointer = oldworkpointer;
02373 AR.CompareRoutine = (COMPAREDUMMY)oldcompareroutine;
02374 AR.SortType = oldsorttype;
02375 return(0);
02376 }
02377
02378 /*
02379     #] ReadPolyRatFun :
02380     #[ FromPolyRatFun :
02381 */
02382
02383 int FromPolyRatFun(PHEAD WORD *fun, WORD **numout, WORD **denout)
02384 {
02385     WORD *nextfun, *tt, *num, *den;
02386     int i;
02387     nextfun = fun + fun[1];
02388     fun += FUNHEAD;
02389     num = AT.WorkPointer;
02390     if ( *fun < 0 ) {
02391         if ( *fun != -SNUMBER && *fun != -SYMBOL ) goto Improper;
02392         ToGeneral(fun,num,0);
02393         tt = num + *num; *tt++ = 0;
02394         fun += 2;
02395     }
02396     else { i = *fun; tt = num; NCOPY(tt,fun,i); *tt++ = 0; }
02397     den = tt;
02398     if ( *fun < 0 ) {
02399         if ( *fun != -SNUMBER && *fun != -SYMBOL ) goto Improper;
02400         ToGeneral(fun,den,0);
02401         tt = den + *den; *tt++ = 0;
02402         fun += 2;
02403     }
02404     else { i = *fun; tt = den; NCOPY(tt,fun,i); *tt++ = 0; }
02405     *numout = num; *denout = den;
02406     if ( fun != nextfun ) { return(1); }
02407     AT.WorkPointer = tt;

```

```

02408     return(0);
02409 Improper:
02410     MLOCK(ErrorMessageLock);
02411     MesPrint("Improper use of PolyRatFun");
02412     MesCall("FromPolyRatFun");
02413     MUNLOCK(ErrorMessageLock);
02414     SETERROR(-1);
02415 }
02416
02417 /*
02418     #] FromPolyRatFun :
02419     #[ TakeSymbolContent :
02420 */
02434 WORD *TakeSymbolContent (PHEAD WORD *in, WORD *term)
02435 {
02436     GETBIDENTITY
02437     WORD *t, *tstop, *tout, *tstore;
02438     WORD *tnext, *tt, *tterm;
02439     WORD *inp, a, *den, *oldworkpointer = AT.WorkPointer;
02440     int i, action = 0, sign, first;
02441     UWORD *GCDbuffer, *GCDbuffer2, *LCMbuffer, *LCMbuffer2, *ap;
02442     WORD GCDlen, GCDlen2, LCMlen, LCMlen2, length, redlength, len1, len2;
02443     LONG j;
02444     tout = tstore = term+1;
02445 /*
02446     #[ SYMBOL :
02447
02448     We make a list of symbols and their minimal powers.
02449     This includes negative powers. In the end we have to multiply by the
02450     inverse of this list. That takes out all negative powers and leaves
02451     things ready for further processing.
02452 */
02453     tterm = AT.WorkPointer; tt = tterm+1;
02454     tout[0] = SYMBOL; tout[1] = 2;
02455     t = in; first = 1;
02456     while ( *t ) {
02457         tnext = t + *t; tstop = tnext - ABS(tnext[-1]); t++;
02458         while ( t < tstop ) {
02459             if ( first ) {
02460                 if ( *t == SYMBOL ) {
02461                     for ( i = 0; i < t[1]; i++ ) tout[i] = t[i];
02462                     goto didwork;
02463                 }
02464                 else {
02465                     MLOCK(ErrorMessageLock);
02466                     MesPrint ((char*)"ERROR: polynomials and polyratfuns must contain symbols only");
02467                     MUNLOCK(ErrorMessageLock);
02468                     Terminate(1);
02469                 }
02470             }
02471             else if ( *t == SYMBOL ) {
02472                 MergeSymbolLists(BHEAD tout,t,-1);
02473                 goto didwork;
02474             }
02475             else {
02476                 t += t[1];
02477             }
02478         }
02479 /*
02480         Here we come when there were no symbols. Only keep the negative ones.
02481 */
02482         if ( first == 0 ) {
02483             int j = 2;
02484             for ( i = 2; i < tout[1]; i += 2 ) {
02485                 if ( tout[i+1] < 0 ) {
02486                     if ( i == j ) { j += 2; }
02487                     else { tout[j] = tout[i]; tout[j+1] = tout[i+1]; j += 2; }
02488                 }
02489             }
02490             tout[1] = j;
02491         }
02492     didwork:;
02493     first = 0;
02494     t = tnext;
02495 }
02496 if ( tout[1] > 2 ) {
02497     t = tout;
02498     tt[0] = t[0]; tt[1] = t[1];
02499     for ( i = 2; i < t[1]; i += 2 ) {
02500         tt[i] = t[i]; tt[i+1] = -t[i+1];
02501     }
02502     tt += tt[1];
02503     tout += tout[1];
02504     action++;
02505 }
02506 /*
02507     #] SYMBOL :

```

```

02508     #[ Coefficient :
02509
02510     Now we have to collect the GCD of the numerators
02511     and the LCM of the denominators.
02512 */
02513     AT.WorkPointer = tt;
02514     if ( AN.cmod != 0 ) {
02515         WORD x, ix, ip;
02516         t = in; tnext = t + *t; tstop = tnext - ABS(tnext[-1]);
02517         x = tstop[0];
02518         if ( tnext[-1] < 0 ) x += AC.cmod[0];
02519         if ( GetModInverses(x, (WORD) (AN.cmod[0]), &ix, &ip) ) goto CalledFrom;
02520         *tout++ = x; *tout++ = 1; *tout++ = 3;
02521         *tt++ = ix; *tt++ = 1; *tt++ = 3;
02522     }
02523     else {
02524         GCDbuffer = NumberMalloc("MakeInteger");
02525         GCDbuffer2 = NumberMalloc("MakeInteger");
02526         LCMbuffer = NumberMalloc("MakeInteger");
02527         LCMbuffer2 = NumberMalloc("MakeInteger");
02528         t = in;
02529         tnext = t + *t; length = tnext[-1];
02530         if ( length < 0 ) { sign = -1; length = -length; }
02531         else { sign = 1; }
02532         tstop = tnext - length;
02533         redlength = (length-1)/2;
02534         for ( i = 0; i < redlength; i++ ) {
02535             GCDbuffer[i] = (UWORD) (tstop[i]);
02536             LCMbuffer[i] = (UWORD) (tstop[redlength+i]);
02537         }
02538         GCDlen = LCMlen = redlength;
02539         while ( GCDbuffer[GCDlen-1] == 0 ) GCDlen--;
02540         while ( LCMbuffer[LCMlen-1] == 0 ) LCMlen--;
02541         t = tnext;
02542         while ( *t ) {
02543             tnext = t + *t; length = ABS(tnext[-1]);
02544             tstop = tnext - length; redlength = (length-1)/2;
02545             len1 = len2 = redlength;
02546             den = tstop + redlength;
02547             while ( tstop[len1-1] == 0 ) len1--;
02548             while ( den[len2-1] == 0 ) len2--;
02549             if ( GCDlen == 1 && GCDbuffer[0] == 1 ) {}
02550             else {
02551                 GcdLong(BHEAD (UWORD *)tstop, len1, GCDbuffer, GCDlen, GCDbuffer2, &GCDlen2);
02552                 ap = GCDbuffer; GCDbuffer = GCDbuffer2; GCDbuffer2 = ap;
02553                 a = GCDlen; GCDlen = GCDlen2; GCDlen2 = a;
02554             }
02555             if ( len2 == 1 && den[0] == 1 ) {}
02556             else {
02557                 GcdLong(BHEAD LCMbuffer, LCMlen, (UWORD *)den, len2, LCMbuffer2, &LCMlen2);
02558                 DivLong((UWORD *)den, len2, LCMbuffer2, LCMlen2,
02559                     GCDbuffer2, &GCDlen2, (UWORD *)AT.WorkPointer, &a);
02560                 MullLong(LCMbuffer, LCMlen, GCDbuffer2, GCDlen2, LCMbuffer2, &LCMlen2);
02561                 ap = LCMbuffer; LCMbuffer = LCMbuffer2; LCMbuffer2 = ap;
02562                 a = LCMlen; LCMlen = LCMlen2; LCMlen2 = a;
02563             }
02564             t = tnext;
02565         }
02566         if ( GCDlen != 1 || GCDbuffer[0] != 1 || LCMlen != 1 || LCMbuffer[0] != 1 ) {
02567             redlength = GCDlen; if ( LCMlen > GCDlen ) redlength = LCMlen;
02568             for ( i = 0; i < GCDlen; i++ ) *tout++ = (WORD) (GCDbuffer[i]);
02569             for ( ; i < redlength; i++ ) *tout++ = 0;
02570             for ( i = 0; i < LCMlen; i++ ) *tout++ = (WORD) (LCMbuffer[i]);
02571             for ( ; i < redlength; i++ ) *tout++ = 0;
02572             *tout++ = (2*redlength+1)*sign;
02573             for ( i = 0; i < LCMlen; i++ ) *tt++ = (WORD) (LCMbuffer[i]);
02574             for ( ; i < redlength; i++ ) *tt++ = 0;
02575             for ( i = 0; i < GCDlen; i++ ) *tt++ = (WORD) (GCDbuffer[i]);
02576             for ( ; i < redlength; i++ ) *tt++ = 0;
02577             *tt++ = (2*redlength+1)*sign;
02578             action++;
02579         }
02580         else {
02581             *tout++ = 1; *tout++ = 1; *tout++ = 3*sign;
02582             *tt++ = 1; *tt++ = 1; *tt++ = 3*sign;
02583             if ( sign != 1 ) action++;
02584         }
02585         NumberFree(LCMbuffer2, "MakeInteger");
02586         NumberFree(LCMbuffer, "MakeInteger");
02587         NumberFree(GCDbuffer2, "MakeInteger");
02588         NumberFree(GCDbuffer, "MakeInteger");
02589     }
02590 */
02591     #[ Coefficient :
02592     #[ Multiply by the inverse content :
02593 */
02594     if ( action ) {

```

```

02595     *term = tout - term; *tout = 0;
02596     *tterm = tt - tterm; *tt = 0;
02597     AT.WorkPointer = tt;
02598     inp = MultiplyWithTerm(BHEAD in,tterm,2);
02599     AT.WorkPointer = tterm;
02600     t = inp; while ( *t ) t += *t;
02601     j = (t-inp); t = inp;
02602     if ( j*sizeof(WORD) > (size_t)(AM.MaxTer) ) goto OverWork;
02603     in = tout = TermMalloc("TakeSymbolContent");
02604     NCOPY(tout,t,j); *tout = 0;
02605     if ( AN.tryterm > 0 ) { TermFree(inp,"MultiplyWithTerm"); AN.tryterm = 0; }
02606     else { M_free(inp,"MultiplyWithTerm"); }
02607 }
02608 else {
02609     t = in; while ( *t ) t += *t;
02610     j = (t-in); t = in;
02611     if ( j*sizeof(WORD) > (size_t)(AM.MaxTer) ) goto OverWork;
02612     in = tout = TermMalloc("TakeSymbolContent");
02613     NCOPY(tout,t,j); *tout = 0;
02614     term[0] = 4; term[1] = 1; term[2] = 1; term[3] = 3; term[4] = 0;
02615 }
02616 /*
02617 #] Multiply by the inverse content :
02618 AT.WorkPointer = tterm + *tterm;
02619 */
02620 AT.WorkPointer = oldworkpointer;
02621
02622 return(in);
02623 OverWork:
02624 MLOCK(ErrorMessageLock);
02625 MesPrint("Term too complex. Maybe increasing MaxTermSize can help");
02626 MUNLOCK(ErrorMessageLock);
02627 CalledFrom:
02628 MLOCK(ErrorMessageLock);
02629 MesCall("TakeSymbolContent");
02630 MUNLOCK(ErrorMessageLock);
02631 Terminate(-1);
02632 return(0);
02633 }
02634
02635 /*
02636 #] TakeSymbolContent :
02637 #[ GCDclean :
02638
02639     Takes a numerator and a denominator that each consist of a
02640     single term with only a coefficient and symbols and makes them
02641     into a proper fraction. Output overwrites input.
02642 */
02643
02644 void GCDclean(PHEAD WORD *num, WORD *den)
02645 {
02646     WORD *out1 = TermMalloc("GCDclean");
02647     WORD *out2 = TermMalloc("GCDclean");
02648     WORD *t1, *t2, *r1, *r2, *t1stop, *t2stop, csizel, csize2, csize3, pow, sign;
02649     int i;
02650     t1stop = num+*num; sign = ( t1stop[-1] < 0 ) ? -1 : 1;
02651     csizel = ABS(t1stop[-1]); t1stop -= csizel;
02652     t2stop = den+*den; if ( t2stop[-1] < 0 ) sign = -sign;
02653     csize2 = ABS(t2stop[-1]); t2stop -= csize2;
02654     t1 = num+1; t2 = den+1;
02655     r1 = out1+3; r2 = out2+3;
02656     if ( t1 == t1stop ) {
02657         if ( t2 < t2stop ) {
02658             for ( i = 2; i < t2[1]; i += 2 ) {
02659                 if ( t2[i+1] < 0 ) { *r1++ = t2[i]; *r1++ = -t2[i+1]; }
02660                 else { *r2++ = t2[i]; *r2++ = t2[i+1]; }
02661             }
02662         }
02663     }
02664     else if ( t2 == t2stop ) {
02665         for ( i = 2; i < t1[1]; i += 2 ) {
02666             if ( t1[i+1] < 0 ) { *r2++ = t1[i]; *r2++ = -t1[i+1]; }
02667             else { *r1++ = t1[i]; *r1++ = t1[i+1]; }
02668         }
02669     }
02670     else {
02671         t1 += 2; t2 += 2;
02672         while ( t1 < t1stop && t2 < t2stop ) {
02673             if ( *t1 < *t2 ) {
02674                 if ( t1[1] > 0 ) { *r1++ = *t1; *r1++ = t1[1]; t1 += 2; }
02675                 else if ( t1[1] < 0 ) { *r2++ = *t1; *r2++ = -t1[1]; t1 += 2; }
02676             }
02677             else if ( *t1 > *t2 ) {
02678                 if ( t2[1] > 0 ) { *r2++ = *t2; *r2++ = t2[1]; t2 += 2; }
02679                 else if ( t2[1] < 0 ) { *r1++ = *t2; *r1++ = -t2[1]; t2 += 2; }
02680             }
02681             else {

```

```

02682         pow = t1[1]-t2[1];
02683         if ( pow > 0 ) { *r1++ = *t1; *r1++ = pow; }
02684         else if ( pow < 0 ) { *r2++ = *t1; *r2++ = -pow; }
02685         t1 += 2; t2 += 2;
02686     }
02687 }
02688 while ( t1 < t1stop ) {
02689     if ( t1[1] < 0 ) { *r2++ = *t1; *r2++ = -t1[1]; }
02690     else { *r1++ = *t1; *r1++ = t1[1]; }
02691     t1 += 2;
02692 }
02693 while ( t2 < t2stop ) {
02694     if ( t2[1] < 0 ) { *r1++ = *t2; *r1++ = -t2[1]; }
02695     else { *r2++ = *t2; *r2++ = t2[1]; }
02696     t2 += 2;
02697 }
02698 }
02699 if ( r1 > out1+3 ) { out1[1] = SYMBOL; out1[2] = r1 - out1 - 1; }
02700 else r1 = out1+1;
02701 if ( r2 > out2+3 ) { out2[1] = SYMBOL; out2[2] = r2 - out2 - 1; }
02702 else r2 = out2+1;
02703 /*
02704     Now the coefficients.
02705 */
02706 csize1 = REDLENG(csize1);
02707 csize2 = REDLENG(csize2);
02708 if ( DivRat(BHEAD (UWORD *)t1stop,csize1,(UWORD *)t2stop,csize2,(UWORD *)r1,&csize3) ) {
02709     MLOCK(ErrorMessageLock);
02710     MesCall("GCDclean");
02711     MUNLOCK(ErrorMessageLock);
02712     Terminate(-1);
02713 }
02714 UnPack((UWORD *)r1,csize3,&csize2,&csize1);
02715 t2 = r1+ABS(csize3);
02716 for ( i = 0; i < csize2; i++ ) r2[i] = t2[i];
02717 r2 += csize2; *r2++ = 1;
02718 for ( i = 1; i < csize2; i++ ) *r2++ = 0;
02719 csize2 = INCLENG(csize2); *r2++ = csize2; *out2 = r2-out2;
02720 r1 += ABS(csize1); *r1++ = 1;
02721 for ( i = 1; i < ABS(csize1); i++ ) *r1++ = 0;
02722 csize1 = INCLENG(csize1); *r1++ = csize1; *out1 = r1-out1;
02723
02724 t1 = num; t2 = out1; i = *out1; NCOPY(t1,t2,i); *t1 = 0;
02725 if ( sign < 0 ) t1[-1] = -t1[-1];
02726 t1 = den; t2 = out2; i = *out2; NCOPY(t1,t2,i); *t1 = 0;
02727
02728 TermFree(out2,"GCDclean");
02729 TermFree(out1,"GCDclean");
02730 }
02731
02732 /*
02733     #] GCDclean :
02734     #[ PolyDiv :
02735
02736     Special stub function for polynomials that should fit inside a term.
02737     We make sure that the space is allocated by TermMalloc.
02738     This makes things much easier on the calling routines.
02739 */
02740
02741 WORD *PolyDiv(PHEAD WORD *a,WORD *b,char *text)
02742 {
02743     /*
02744         Probably the following would work now
02745     */
02746     DUMMYUSE(text);
02747     return(poly_div(BHEAD a,b,1));
02748     /*
02749         WORD *quo, *qq;
02750         WORD *x, *xx;
02751         LONG i;
02752         quo = poly_div(BHEAD a,b,1);
02753         x = TermMalloc(text);
02754         qq = quo; while ( *qq ) qq += *qq;
02755         i = (qq-quo+1);
02756         if ( i*sizeof(WORD) > (size_t)(AM.MaxTer) ) {
02757             DUMMYUSE(text);
02758             MLOCK(ErrorMessageLock);
02759             MesPrint("PolyDiv: Term too complex. Maybe increasing MaxTermSize can help");
02760             MUNLOCK(ErrorMessageLock);
02761             Terminate(-1);
02762         }
02763         xx = x; qq = quo;
02764         NCOPY(xx,qq,i)
02765         TermFree(quo,"poly_div");
02766         return(xx);
02767     */
02768 }

```



```

02769
02770 /*
02771     #] PolyDiv :
02772     #] GCDfunction :
02773     #] DIVfunction :
02774
02775     Input: a div_ function that has two arguments inside a term.
02776     Action: Calculates [arg1/arg2] using polynomial techniques if needed.
02777     Output: The output result is combined with the remainder of the term
02778     and sent to Generator for further processing.
02779     Note that the output can be just a number or many terms.
02780     In case par == 0 the output is [arg1/arg2]
02781     In case par == 1 the output is [arg1%arg2]
02782     In case par == 2 the output is [inverse of arg1 modulus arg2]
02783     In case par == 3 the output is [arg1*arg2]
02784 */
02785
02786 WORD divrem[4] = { DIVFUNCTION, REMFUNCTION, INVERSEFUNCTION, MULFUNCTION };
02787
02788 int DIVfunction(PHEAD WORD *term,WORD level,int par)
02789 {
02790     GETBIDENTITY
02791     WORD *t, *tt, *r, *arg1 = 0, *arg2 = 0, *arg3 = 0, *termout;
02792     WORD *tstop, *tend, *r3, *rr, *rstop, tlength, rlength, newlength;
02793     WORD *proper1, *proper2, *proper3 = 0, numdol = -1;
02794     int numargs = 0, type1, type2, actionflag1, actionflag2;
02795     WORD startebuf = cbuf[AT.ebufnum].numrhs;
02796     int division = ( par <= 2 ); /* false for mul_ */
02797     if ( par < 0 || par > 3 ) {
02798         MLOCK(ErrorMessageLock);
02799         MesPrint("Internal error. Illegal parameter %d in DIVfunction.",par);
02800         MUNLOCK(ErrorMessageLock);
02801         Terminate(-1);
02802     }
02803     /*
02804     Find the function
02805     */
02806     tend = term + *term; tstop = tend - ABS(tend[-1]);
02807     t = term+1;
02808     while ( t < tstop ) {
02809         if ( *t != divrem[par] ) { t += t[1]; continue; }
02810         r = t + FUNHEAD;
02811         tt = t + t[1]; numargs = 0;
02812         while ( r < tt ) {
02813             if ( numargs == 0 ) { arg1 = r; }
02814             if ( numargs == 1 ) { arg2 = r; }
02815             if ( numargs == 2 && *r == -DOLLAREXPRESSION ) { numdol = r[1]; }
02816             numargs++;
02817             NEXTARG(r);
02818         }
02819         if ( numargs == 2 ) break;
02820         if ( division && numargs == 3 ) break;
02821         t = tt;
02822     }
02823     if ( t >= tstop ) {
02824         MLOCK(ErrorMessageLock);
02825         MesPrint("Internal error. Indicated div_ or rem_ function not encountered.");
02826         MUNLOCK(ErrorMessageLock);
02827         Terminate(-1);
02828     }
02829     /*
02830     We have two arguments in arg1 and arg2.
02831     */
02832     if ( division && *arg1 == -SNUMBER && arg1[1] == 0 ) {
02833         if ( *arg2 == -SNUMBER && arg2[1] == 0 ) {
02834             zerozero:;
02835             MLOCK(ErrorMessageLock);
02836             MesPrint("0/0 in either div_ or rem_ function.");
02837             MUNLOCK(ErrorMessageLock);
02838             Terminate(-1);
02839         }
02840         if ( numdol >= 0 ) PutTermInDollar(0,numdol);
02841         return(0);
02842     }
02843     if ( division && *arg2 == -SNUMBER && arg2[1] == 0 ) {
02844         divzero:;
02845         MLOCK(ErrorMessageLock);
02846         MesPrint("Division by zero in either div_ or rem_ function.");
02847         MUNLOCK(ErrorMessageLock);
02848         Terminate(-1);
02849     }
02850     if ( !division ) {
02851         if ( (*arg1 == -SNUMBER && arg1[1] == 0) ||
02852             (*arg2 == -SNUMBER && arg2[1] == 0) ) {
02853             return(0);
02854         }
02855     }

```

```

02856     if ( ( arg1 = ConvertArgument(BHEAD arg1, &type1) ) == 0 ) goto CalledFrom;
02857     if ( ( arg2 = ConvertArgument(BHEAD arg2, &type2) ) == 0 ) goto CalledFrom;
02858     if ( division && *arg1 == 0 ) {
02859         if ( *arg2 == 0 ) {
02860             M_free(arg2,"DIVfunction");
02861             M_free(arg1,"DIVfunction");
02862             goto zerozero;
02863         }
02864         M_free(arg2,"DIVfunction");
02865         M_free(arg1,"DIVfunction");
02866         if ( numdol >= 0 ) PutTermInDollar(0,numdol);
02867         return(0);
02868     }
02869     if ( division && *arg2 == 0 ) {
02870         M_free(arg2,"DIVfunction");
02871         M_free(arg1,"DIVfunction");
02872         goto divzero;
02873     }
02874     if ( !division && (*arg1 == 0 || *arg2 == 0) ) {
02875         M_free(arg2,"DIVfunction");
02876         M_free(arg1,"DIVfunction");
02877         return(0);
02878     }
02879     if ( ( proper1 = PutExtraSymbols(BHEAD arg1,startebuf,&actionflag1) ) == 0 ) goto CalledFrom;
02880     if ( ( proper2 = PutExtraSymbols(BHEAD arg2,startebuf,&actionflag2) ) == 0 ) goto CalledFrom;
02881     /*
02882     if ( type2 == 0 ) M_free(arg2,"DIVfunction");
02883     else {
02884         DOLLARS d = ((DOLLARS)arg2)-1;
02885         if ( d->factors ) M_free(d->factors,"Dollar factors");
02886         M_free(d,"Copy of dollar variable");
02887     }
02888     */
02889     M_free(arg2,"DIVfunction");
02890     /*
02891     if ( type1 == 0 ) M_free(arg1,"DIVfunction");
02892     else {
02893         DOLLARS d = ((DOLLARS)arg1)-1;
02894         if ( d->factors ) M_free(d->factors,"Dollar factors");
02895         M_free(d,"Copy of dollar variable");
02896     }
02897     */
02898     M_free(arg1,"DIVfunction");
02899     if ( par == 0 ) proper3 = poly_div(BHEAD proper1, proper2,0);
02900     else if ( par == 1 ) proper3 = poly_rem(BHEAD proper1, proper2,0);
02901     else if ( par == 2 ) proper3 = poly_inverse(BHEAD proper1, proper2);
02902     else if ( par == 3 ) proper3 = poly_mul(BHEAD proper1, proper2);
02903     if ( proper3 == 0 ) goto CalledFrom;
02904     if ( actionflag1 || actionflag2 ) {
02905         if ( ( arg3 = TakeExtraSymbols(BHEAD proper3,startebuf) ) == 0 ) goto CalledFrom;
02906         M_free(proper3,"DIVfunction");
02907     }
02908     else {
02909         arg3 = proper3;
02910     }
02911     M_free(proper2,"DIVfunction");
02912     M_free(proper1,"DIVfunction");
02913     cbuf[AT.ebufnum].numrhs = startebuf;
02914     if ( *arg3 ) {
02915         termout = AT.WorkPointer;
02916         tlength = tend[-1];
02917         tlength = REDLENG(tlength);
02918         r3 = arg3;
02919         while ( *r3 ) {
02920             tt = term + 1; rr = termout + 1;
02921             while ( tt < t ) *rr++ = *tt++;
02922             r = r3 + 1;
02923             r3 = r3 + *r3;
02924             rstop = r3 - ABS(r3[-1]);
02925             while ( r < rstop ) *rr++ = *r++;
02926             tt += t[1];
02927             while ( tt < tstop ) *rr++ = *ttt++;
02928             rlength = r3[-1];
02929             rlength = REDLENG(rlength);
02930             if ( MulRat(BHEAD (UWORD *)tstop,tlength,(UWORD *)rstop,rlength,
02931                 (UWORD *)rr,&newlength) < 0 ) goto CalledFrom;
02932             rlength = INCLENG(newlength);
02933             rr += ABS(rlength);
02934             rr[-1] = rlength;
02935             *termout = rr - termout;
02936             AT.WorkPointer = rr;
02937             if ( Generator(BHEAD termout,level) ) goto CalledFrom;
02938         }
02939         AT.WorkPointer = termout;
02940     }
02941     M_free(arg3,"DIVfunction");
02942     return(0);

```

```

02943 CalledFrom:
02944     MLOCK(ErrorMessageLock);
02945     MesCall("DIVfunction");
02946     MUNLOCK(ErrorMessageLock);
02947     SETERROR(-1)
02948 }
02949
02950 /*
02951 #] DIVfunction :
02952 #[ MULfunc :
02953
02954     Multiplies two polynomials and puts the results in TermMalloc space.
02955 */
02956
02957 WORD *MULfunc(PHEAD WORD *p1, WORD *p2)
02958 {
02959     WORD *prod, size1, size2, size3, *t, *tfill, *ps1, *ps2, sign1, sign2, error, *p3;
02960     UWORD *num1, *num2, *num3;
02961     int i;
02962     WORD oldsorttype = AR.SortType;
02963     COMPARE oldcompareroutine = (COMPARE) (AR.CompareRoutine);
02964     AR.SortType = SORTHIGHFIRST;
02965     AR.CompareRoutine = (COMPAREDUMMY) (&CompareSymbols);
02966     num3 = NumberMalloc("MULfunc");
02967     prod = TermMalloc("MULfunc");
02968     NewSort(BHEAD0);
02969     while ( *p1 ) {
02970         ps1 = p1+*p1; num1 = (UWORD *) (ps1 - ABS(ps1[-1])); size1 = ps1[-1];
02971         if ( size1 < 0 ) { sign1 = -1; size1 = -size1; }
02972         else sign1 = 1;
02973         size1 = (size1-1)/2;
02974         p3 = p2;
02975         while ( *p3 ) {
02976             ps2 = p3+*p3; num2 = (UWORD *) (ps2 - ABS(ps2[-1])); size2 = ps2[-1];
02977             if ( size2 < 0 ) { sign2 = -1; size2 = -size2; }
02978             else sign2 = 1;
02979             size2 = (size2-1)/2;
02980             if ( MulLong(num1, size1, num2, size2, num3, &size3) ) {
02981                 error = 1;
02982             }
02983             CalledFrom:
02984                 MLOCK(ErrorMessageLock);
02985                 MesPrint(" Error %d", error);
02986                 MesCall("MulFunc");
02987                 MUNLOCK(ErrorMessageLock);
02988                 Terminate(-1);
02989             }
02990             tfill = prod+1;
02991             t = p1+1; while ( t < (WORD *) num1 ) *tfill++ = *t++;
02992             t = p3+1; while ( t < (WORD *) num2 ) *tfill++ = *t++;
02993             t = (WORD *) num3;
02994             for ( i = 0; i < size3; i++ ) *tfill++ = *t++;
02995             *tfill++ = 1;
02996             for ( i = 1; i < size3; i++ ) *tfill++ = 0;
02997             *tfill++ = (2*size3+1)*sign1*sign2;
02998             prod[0] = tfill - prod;
02999             if ( SymbolNormalize(prod) ) { error = 2; goto CalledFrom; }
03000             if ( StoreTerm(BHEAD prod) ) { error = 3; goto CalledFrom; }
03001             p3 += *p3;
03002         }
03003         p1 += *p1;
03004     }
03005     NumberFree(num3, "MULfunc");
03006     EndSort(BHEAD prod, 1);
03007     AR.CompareRoutine = (COMPAREDUMMY) oldcompareroutine;
03008     AR.SortType = oldsorttype;
03009     return(prod);
03010 }
03011
03012 /*
03013 #] MULfunc :
03014 #[ ConvertArgument :
03015
03016     Converts an argument to a general notation in allocated space.
03017 */
03018 WORD *ConvertArgument(PHEAD WORD *arg, int *type)
03019 {
03020     WORD *output, *t, *r;
03021     int i, size;
03022     if ( *arg > 0 ) {
03023         output = (WORD *) Malloc1((*arg)*sizeof(WORD), "ConvertArgument");
03024         i = *arg - ARGHEAD; t = arg + ARGHEAD; r = output;
03025         NCOPY(r, t, i);
03026         *r = 0; *type = 0;
03027         return(output);
03028     }
03029     if ( *arg == -EXPRESSION ) {

```

```

03030         *type = 0;
03031         return(CreateExpression(BHEAD arg[1]));
03032     }
03033     if ( *arg == -DOLLAREXPRESSION ) {
03034         DOLLARS d;
03035         *type = 1;
03036         d = DolToTerms(BHEAD arg[1]);
03037     /*
03038         The problem is that DolToTerms creates a copy of the dollar variable.
03039         If we just return d->where we create a memory leak. Hence we have to
03040         copy the contents of d->where to a new buffer
03041     */
03042         output = (WORD *)Malloc1((d->size+1)*sizeof(WORD),"Copy of dollar content");
03043         WCOPY(output,d->where,d->size+1);
03044         if ( d->factors ) { M_free(d->factors,"Dollar factors"); d->factors = 0; }
03045         M_free(d,"Copy of dollar variable");
03046         return(output);
03047     }
03048     #if ( FUNHEAD > 4 )
03049         size = FUNHEAD+5;
03050     #else
03051         size = 9;
03052     #endif
03053     output = (WORD *)Malloc1(size*sizeof(WORD),"ConvertArgument");
03054     switch(*arg) {
03055         case -SYMBOL:
03056             output[0] = 8; output[1] = SYMBOL; output[2] = 4; output[3] = arg[1];
03057             output[4] = 1; output[5] = 1; output[6] = 1; output[7] = 3;
03058             output[8] = 0;
03059             break;
03060         case -INDEX:
03061         case -VECTOR:
03062         case -MINVECTOR:
03063             output[0] = 7; output[1] = INDEX; output[2] = 3; output[3] = arg[1];
03064             output[4] = 1; output[5] = 1;
03065             if ( *arg == -MINVECTOR ) output[6] = -3;
03066             else output[6] = 3;
03067             output[7] = 0;
03068             break;
03069         case -SNUMBER:
03070             output[0] = 4;
03071             if ( arg[1] < 0 ) {
03072                 output[1] = -arg[1]; output[2] = 1; output[3] = -3;
03073             }
03074             else {
03075                 output[1] = arg[1]; output[2] = 1; output[3] = 3;
03076             }
03077             output[4] = 0;
03078             break;
03079         default:
03080             output[0] = FUNHEAD+4;
03081             output[1] = -*arg;
03082             output[2] = FUNHEAD;
03083             for ( i = 3; i <= FUNHEAD; i++ ) output[i] = 0;
03084             output[FUNHEAD+1] = 1;
03085             output[FUNHEAD+2] = 1;
03086             output[FUNHEAD+3] = 3;
03087             output[FUNHEAD+4] = 0;
03088             break;
03089     }
03090     *type = 0;
03091     return(output);
03092 }
03093
03094 /*
03095 #] ConvertArgument :
03096 #[ ExpandRat :
03097
03098 Expands the denominator of a PolyRatFun in the variable PolyFunVar.
03099 The output is a polyratfun with a single argument.
03100 In the case that there is a polyratfun with more than one argument
03101 or the dirtyflag is on, the argument(s) is/are normalized.
03102 The output overwrites the input.
03103 */
03104
03105 char *TheErrorMessage[] = {
03106     "PolyRatFun not of a type that FORM will expand: incorrect variable inside."
03107     ,"Division by zero in PolyRatFun encountered in ExpandRat."
03108     ,"Irregular code in PolyRatFun encountered in ExpandRat."
03109     ,"Called from ExpandRat."
03110     ,"Workspace overflow. Change parameter Workspace in setup file?"
03111 };
03112
03113 int ExpandRat(PHEAD WORD *fun)
03114 {
03115     WORD *r, *rr, *rrr, *tt, *tnext, *arg1, *arg2, *rmin = 0, *rmininv;
03116     WORD *rcoef, rsize, rcopy, *ow = AT.WorkPointer;

```

```

03117     WORD *numerator, *denominator, *rnext;
03118     WORD *thecopy, *rc, ncoef, newcoef, *m, *mm, nco, *outarg = 0;
03119     UWORD co[2], col[2], co2[2];
03120     WORD OldPolyFunPow = AR.PolyFunPow;
03121     int i, j, minpow = 0, eppow, first, error = 0, ipoly;
03122     if ( fun[1] == FUNHEAD ) { return(0); }
03123     tnext = fun + fun[1];
03124     if ( fun[1] == fun[FUNHEAD]+FUNHEAD ) { /* Single argument */
03125         if ( fun[2] == 0 ) { goto done; }
03126     /*
03127         We have to normalize the argument. This could make it shorter.
03128     */
03129     NormArg;;
03130         if ( outarg == 0 ) outarg = TermMalloc("ExpandRat")+ARGHEAD;
03131         AT.TrimPower = 1;
03132         NewSort(BHEAD0);
03133         r = fun+FUNHEAD+ARGHEAD;
03134         if ( AR.PolyFunExp == 2 ) { /* Find minimum power */
03135             WORD minpow2 = MAXPOWER, *rrm;
03136             rrm = r;
03137             while ( rrm < tnext ) {
03138                 if ( *rrm == 4 ) {
03139                     if ( minpow2 > 0 ) minpow2 = 0;
03140                 }
03141                 else if ( ABS(rrm[*rrm-1]) == (*rrm-1) ) {
03142                     if ( minpow2 > 0 ) minpow2 = 0;
03143                 }
03144                 else {
03145                     if ( rrm[1] == SYMBOL && rrm[2] == 4 && rrm[3] == AR.PolyFunVar ) {
03146                         if ( rrm[4] < minpow2 ) minpow2 = rrm[4];
03147                     }
03148                     else {
03149                         MesPrint("Illegal term in expanded polyratfun.");
03150                         goto onerror;
03151                     }
03152                 }
03153                 rrm += *rrm;
03154             }
03155             AR.PolyFunPow += minpow2;
03156         }
03157         while ( r < tnext ) {
03158             rr = r + *r;
03159             i = *r; rrr = outarg; NCOPY(rrr,r,i);
03160             Normalize(BHEAD outarg);
03161             if ( *outarg > 0 ) StoreTerm(BHEAD outarg);
03162         }
03163         r = fun+FUNHEAD+ARGHEAD;
03164         EndSort(BHEAD r,1);
03165         AT.TrimPower = 0;
03166         if ( *r == 0 ) {
03167             fun[FUNHEAD] = -SNUMBER; fun[FUNHEAD+1] = 0;
03168             fun[1] = FUNHEAD+2;
03169         }
03170         else {
03171             rr = fun+FUNHEAD;
03172             if ( ToFast(rr,rr) ) {
03173                 NEXTARG(rr); fun[1] = rr - fun;
03174             }
03175             else {
03176                 while ( *r ) r += *r;
03177                 *rr = r-rr; rr[1] = CLEANFLAG;
03178                 fun[1] = r - fun;
03179             }
03180         }
03181         fun[2] = CLEANFLAG;
03182         goto done;
03183     }
03184     /*
03185     First test whether we have only AR.PolyFunVar in the denominator
03186     */
03187     tt = fun + FUNHEAD;
03188     arg1 = arg2 = 0;
03189     if ( tt < tnext ) {
03190         arg1 = tt; NEXTARG(tt);
03191         if ( tt < tnext ) {
03192             arg2 = tt; NEXTARG(tt);
03193             if ( tt != tnext ) { arg1 = arg2 = 0; } /* more than two arguments */
03194         }
03195     }
03196     if ( arg2 == 0 ) {
03197         if ( *arg1 < 0 ) { fun[2] = CLEANFLAG; goto done; }
03198         if ( fun[2] == CLEANFLAG ) goto done;
03199         goto NormArg; /* Note: should not come here */
03200     }
03201     /*
03202     Produce the output argument in outarg
03203     */

```

```

03204     if ( outarg == 0 ) outarg = TermMalloc("ExpandRat")+ARGHEAD;
03205
03206     if ( *arg2 <= 0 ) {
03207 /*
03208         These cases are trivial.
03209         We try as much as possible to write the output directly into the
03210         function. We just have to be extremely careful not to overwrite
03211         relevant information before we are finished with it.
03212 */
03213         if ( *arg2 == -SYMBOL && arg2[1] == AR.PolyFunVar ) {
03214             rr = r = fun+FUNHEAD+ARGHEAD;
03215             if ( *arg1 < 0 ) {
03216                 if ( *arg1 == -SYMBOL ) {
03217                     if ( arg1[1] == AR.PolyFunVar ) {
03218                         *r++ = 4; *r++ = 1; *r++ = 1; *r++ = 3; *r = 0;
03219                     }
03220                     else {
03221                         *r++ = 10; *r++ = SYMBOL; *r++ = 6;
03222                         *r++ = arg1[1]; *r++ = 1;
03223                         *r++ = AR.PolyFunVar; *r++ = -1;
03224                         *r++ = 1; *r++ = 1; *r++ = 3; *r = 0;
03225                         Normalize(BHEAD rr);
03226                     }
03227                 }
03228                 else if ( *arg1 == -SNUMBER ) {
03229                     nco = arg1[1];
03230                     if ( nco == 0 ) { *r++ = 0; }
03231                     else {
03232                         *r++ = 8; *r++ = SYMBOL; *r++ = 4;
03233                         *r++ = AR.PolyFunVar; *r++ = -1;
03234                         *r++ = ABS(nco); *r++ = 1;
03235                         if ( nco < 0 ) *r++ = -3;
03236                         else *r++ = 3;
03237                         *r = 0;
03238                     }
03239                 }
03240                 else { error = 2; goto onerror; } /* should not happen! */
03241             }
03242             else { /* Multi-term numerator. */
03243                 m = arg1+ARGHEAD;
03244                 NewSort(BHEAD0); /* Technically maybe not needed */
03245                 if ( AR.PolyFunExp == 2 ) { /* Find minimum power */
03246                     WORD minpow2 = MAXPOWER, *rrm;
03247                     rrm = m;
03248                     while ( rrm < arg2 ) {
03249                         if ( *rrm == 4 ) {
03250                             if ( minpow2 > 0 ) minpow2 = 0;
03251                         }
03252                         else if ( ABS(rrm[*rrm-1]) == (*rrm-1) ) {
03253                             if ( minpow2 > 0 ) minpow2 = 0;
03254                         }
03255                         else {
03256                             if ( rrm[1] == SYMBOL && rrm[2] == 4 && rrm[3] == AR.PolyFunVar ) {
03257                                 if ( rrm[4] < minpow2 ) minpow2 = rrm[4];
03258                             }
03259                             else {
03260                                 MesPrint("Illegal term in expanded polyratfun.");
03261                                 goto onerror;
03262                             }
03263                         }
03264                         rrm += *rrm;
03265                     }
03266                     AR.PolyFunPow += minpow2-1;
03267                 }
03268                 while ( m < arg2 ) {
03269                     r = outarg;
03270                     rrr = r++; mm = m + *m;
03271                     *r++ = SYMBOL; *r++ = 4; *r++ = AR.PolyFunVar; *r++ = -1;
03272                     m++; while ( m < mm ) *r++ = *m++;
03273                     *rrr = r-rrr;
03274                     Normalize(BHEAD rrr);
03275                     StoreTerm(BHEAD rrr);
03276                 }
03277                 EndSort(BHEAD rr,1);
03278                 r = rr; while ( *r ) r += *r;
03279             }
03280             if ( *rr == 0 ) {
03281                 fun[FUNHEAD] = -SNUMBER; fun[FUNHEAD+1] = CLEANFLAG;
03282                 fun[1] = FUNHEAD+2;
03283             }
03284             else {
03285                 rr = fun+FUNHEAD;
03286                 *rr = r-rr;
03287                 rr[1] = CLEANFLAG;
03288                 if ( ToFast(rr,rr) ) {
03289                     NEXTARG(rr);
03290                     fun[1] = rr - fun;

```

```

03291         }
03292         else { fun[1] = r - fun; }
03293     }
03294     fun[2] = CLEANFLAG;
03295     goto done;
03296 }
03297 else if ( *arg2 == -SNUMBER ) {
03298     rr = r = outarg;
03299     if ( arg2[1] == 0 ) { error = 1; goto onerror; }
03300     if ( *arg1 == -SNUMBER ) { /* Things may not be normalized */
03301         if ( arg1[1] == 0 ) { *r++ = 0; }
03302         else {
03303             col[0] = ABS(arg1[1]); col[1] = 1;
03304             co2[0] = 1; co2[1] = ABS(arg2[1]);
03305             MulRat(BHEAD col,1,co2,1,co,&nco);
03306             *r++ = 4; *r++ = (WORD)(co[0]); *r++ = (WORD)(co[1]);
03307             if ( ( arg1[1] < 0 && arg2[1] > 0 ) ||
03308                 ( arg1[1] > 0 && arg2[1] < 0 ) ) *r++ = -3;
03309             else *r++ = 3;
03310             *r = 0;
03311         }
03312     }
03313     else if ( *arg1 == -SYMBOL ) {
03314         *r++ = 8; *r++ = SYMBOL; *r++ = 4;
03315         *r++ = arg1[1]; *r++ = 1;
03316         *r++ = 1; *r++ = ABS(arg2[1]);
03317         if ( arg2[1] < 0 ) *r++ = -3;
03318         else *r++ = 3;
03319         *r = 0;
03320     }
03321     else if ( *arg1 < 0 ) { error = 2; goto onerror; }
03322     else { /* Multi-term numerator. */
03323         m = arg1+ARGHEAD;
03324         NewSort(BHEAD0); /* Technically maybe not needed */
03325         if ( AR.PolyFunExp == 2 ) { /* Find minimum power */
03326             WORD minpow2 = MAXPOWER, *rrm;
03327             rrm = m;
03328             while ( rrm < arg2 ) {
03329                 if ( *rrm == 4 ) {
03330                     if ( minpow2 > 0 ) minpow2 = 0;
03331                 }
03332                 else if ( ABS(rrm[*rrm-1]) == (*rrm-1) ) {
03333                     if ( minpow2 > 0 ) minpow2 = 0;
03334                 }
03335                 else {
03336                     if ( rrm[1] == SYMBOL && rrm[2] == 4 && rrm[3] == AR.PolyFunVar ) {
03337                         if ( rrm[4] < minpow2 ) minpow2 = rrm[4];
03338                     }
03339                     else {
03340                         MesPrint("Illegal term in expanded polyratfun.");
03341                         goto onerror;
03342                     }
03343                 }
03344                 rrm += *rrm;
03345             }
03346             AR.PolyFunPow += minpow2;
03347         }
03348         while ( m < arg2 ) {
03349             r = rr;
03350             rrr = r++; mm = m + *m;
03351             *r++ = DENOMINATOR; *r++ = FUNHEAD + 2; *r++ = DIRTYFLAG;
03352             FILLFUN3(r);
03353             *r++ = arg2[0]; *r++ = arg2[1];
03354             m++; while ( m < mm ) *r++ = *m++;
03355             *rrr = r-rrr;
03356             if ( r < AT.WorkTop && r >= AT.WorkSpace )
03357                 AT.WorkPointer = r;
03358             Normalize(BHEAD rrr);
03359             if ( ABS(rrr[*rrr-1]) == *rrr-1 ) {
03360                 if ( AR.PolyFunPow >= 0 ) {
03361                     StoreTerm(BHEAD rrr);
03362                 }
03363             }
03364             else if ( rrr[1] == SYMBOL && rrr[2] == 4 &&
03365                 rrr[3] == AR.PolyFunVar && rrr[4] <= AR.PolyFunPow ) {
03366                 StoreTerm(BHEAD rrr);
03367             }
03368         }
03369         EndSort(BHEAD rr,1);
03370     }
03371     r = rr; while ( *r ) r += *r;
03372     i = r-rr;
03373     r = fun + FUNHEAD + ARGHEAD;
03374     NCOPY(r,rr,i);
03375     rr = fun + FUNHEAD;
03376     *rr = r - rr; rr[1] = CLEANFLAG;
03377     if ( ToFast(rr,rr) ) {

```

```

03378         NEXTARG(rr);
03379         fun[1] = rr - fun;
03380     }
03381     else { fun[1] = r - fun; }
03382     fun[2] = CLEANFLAG;
03383     goto done;
03384 }
03385 else { error = 0; goto onerror; }
03386 }
03387 else {
03388     r = arg2+ARGHEAD; /* The argument ends at tnext */
03389     first = 1;
03390     while ( r < tnext ) {
03391         rr = r + *r; rr -= ABS(rr[-1]);
03392         if ( r+1 == rr ) {
03393             if ( first ) { minpow = 0; first = 0; rmin = r; }
03394             else if ( minpow > 0 ) { minpow = 0; rmin = r; }
03395         }
03396         else if ( r[1] != SYMBOL || r[2] != 4 || r[3] != AR.PolyFunVar
03397             || r[4] > MAXPOWER ) { error = 0; goto onerror; }
03398         else if ( first ) { minpow = r[4]; first = 0; rmin = r; }
03399         else if ( r[4] < minpow ) { minpow = r[4]; rmin = r; }
03400         r += *r;
03401     }
03402     /*
03403     We have now:
03404     1: a numerator in arg1 which can contain several variables.
03405     2: a denominator in arg2 with at most only AR.PolyFunVar (ep).
03406     3: the minimum power in the denominator is minpow and the
03407        term with that minimum power is in rmin.
03408     Divide numerator and denominator by this minimum power.
03409     Determine the power range in the numerator.
03410     Call InvPoly.
03411     Multiply by the inverse in such a way that we never take more
03412     powers of ep than necessary.
03413     */
03414     /*
03415     One: put 1/rmin in AT.WorkPointer -> rmininv
03416     */
03417     AT.WorkPointer += AM.MaxTer/sizeof(WORD);
03418     if ( AT.WorkPointer + (AM.MaxTer/sizeof(WORD)) >= AT.WorkTop ) {
03419         error = 4; goto onerror;
03420     }
03421     rmininv = r = AT.WorkPointer;
03422     rr = rmin; i = *rmin; NCOPY(r,rr,i)
03423     if ( minpow != 0 ) { rmininv[4] = -rmininv[4]; }
03424     rsize = ABS(r[-1]);
03425     rcoef = r - rsize;
03426     rsize = (rsize-1)/2; rr = rcoef + rsize;
03427     for ( i = 0; i < rsize; i++ ) {
03428         rcopy = rcoef[i]; rcoef[i] = rr[i]; rr[i] = rcopy;
03429     }
03430     AT.WorkPointer = r;
03431     if ( *arg1 < 0 ) {
03432         ToGeneral(arg1,r,0);
03433         arg1 = r; r += *r; *r++ = 0; rcopy = 0;
03434         AT.WorkPointer = r;
03435     }
03436     else {
03437         r = arg1 + *arg1;
03438         rcopy = *r; *r++ = 0;
03439     }
03440     /*
03441     We can use MultiplyWithTerm.
03442     */
03443     AT.LeaveNegative = 1;
03444     numerator = MultiplyWithTerm(BHEAD arg1+ARGHEAD,rmininv,0);
03445     AT.LeaveNegative = 0;
03446     r[-1] = rcopy;
03447     r = numerator; while ( *r ) r += *r;
03448     AT.WorkPointer = r+1;
03449     rcopy = arg2[*arg2]; arg2[*arg2] = 0;
03450     denominator = MultiplyWithTerm(BHEAD arg2+ARGHEAD,rmininv,1);
03451     arg2[*arg2] = rcopy;
03452     r = denominator; while ( *r ) r += *r;
03453     AT.WorkPointer = r+1;
03454     /*
03455     Now find the minimum power of ep in the numerator.
03456     */
03457     r = numerator;
03458     first = 1;
03459     while ( *r ) {
03460         rr = r + *r; rr -= ABS(rr[-1]);
03461         if ( r+1 == rr ) {
03462             if ( first ) { minpow = 0; first = 0; }
03463             else if ( minpow > 0 ) { minpow = 0; }
03464         }

```



```

03465         else if ( r[1] != SYMBOL ) { error = 0; goto onerror; }
03466     else {
03467         for ( i = 3; i < r[2]; i += 2 ) {
03468             if ( r[i] == AR.PolyFunVar ) {
03469                 if ( first ) { minpow = r[i+1]; first = 0; }
03470                 else if ( r[i+1] < minpow ) minpow = r[i+1];
03471                 break;
03472             }
03473         }
03474         if ( i >= r[2] ) {
03475             if ( first ) { minpow = 0; first = 0; }
03476             else if ( minpow > 0 ) minpow = 0;
03477         }
03478     }
03479     r += *r;
03480 }
03481 /*
03482 We can invert the denominator.
03483 Note that the return value is an offset in AT.pWorkSpace.
03484 Hence there is no need to free memory afterwards.
03485 */
03486 if ( AR.PolyFunExp == 3 ) {
03487     ipoly = InvPoly(BHEAD denominator, AR.PolyFunPow-minpow, AR.PolyFunVar);
03488 }
03489 else {
03490     ipoly = InvPoly(BHEAD denominator, AR.PolyFunPow, AR.PolyFunVar);
03491 }
03492 /*
03493 Now we start the multiplying
03494 */
03495 NewSort(BHEAD0);
03496 r = numerator;
03497 while ( *r ) {
03498     /*
03499         1: Find power of ep.
03500     */
03501     rnext = r + *r;
03502     rrr = rnext - ABS(rnext[-1]);
03503     rr = r+1;
03504     eppow = 0;
03505     if ( rr < rrr ) {
03506         j = rr[1] - 2; rr += 2;
03507         while ( j > 0 ) {
03508             if ( *rr == AR.PolyFunVar ) { eppow = rr[1]; break; }
03509             j -= 2; rr += 2;
03510         }
03511     }
03512     /*
03513         2: Multiply by the proper terms in ipoly
03514     */
03515     for ( i = 0; i <= AR.PolyFunPow-eppow+minpow; i++ ) {
03516         if ( AT.pWorkSpace[ipoly+i] == 0 ) continue;
03517         /*
03518             Copy the term, add i to the power of ep and multiply coef.
03519         */
03520         rc = r;
03521         rr = thecopy = AT.WorkPointer;
03522         while ( rc < rrr ) *rr++ = *rc++;
03523         if ( i != 0 ) {
03524             *rr++ = SYMBOL; *rr++ = 4; *rr++ = AR.PolyFunVar; *rr++ = i;
03525         }
03526         ncoef = REDLENG(rnext[-1]);
03527         MulRat(BHEAD (UWORD *)rrr, ncoef,
03528             (UWORD *) (AT.pWorkSpace[ipoly+i]+1), AT.pWorkSpace[ipoly+i][0],
03529             (UWORD *)rr, &newcoef);
03530         ncoef = ABS(newcoef); rr += 2*ncoef;
03531         newcoef = INCLENG(newcoef);
03532         *rr++ = newcoef;
03533         *thecopy = rr - thecopy;
03534         AT.WorkPointer = rr;
03535         Normalize(BHEAD thecopy);
03536         if ( *thecopy > 0 ) StoreTerm(BHEAD thecopy);
03537         AT.WorkPointer = thecopy;
03538     }
03539     r = rnext;
03540 }
03541 /*
03542 Now we have all.
03543 */
03544 rr = fun + FUNHEAD; r = rr + ARGHEAD;
03545 EndSort(BHEAD r, 1);
03546 if ( *r == 0 ) {
03547     fun[1] = FUNHEAD+2; fun[2] = CLEANFLAG;
03548     fun[FUNHEAD] = -SNUMBER; fun[FUNHEAD+1] = 0;
03549 }
03550 else {
03551     while ( *r ) r += *r;

```

```

03552         rr[0] = r-rr; rr[1] = CLEANFLAG;
03553         if ( ToFast(rr,rr) ) { NEXTARG(rr); fun[1] = rr-fun; }
03554         else { fun[1] = r-fun; }
03555         fun[2] = CLEANFLAG;
03556     }
03557 }
03558 done:
03559     if ( outarg ) TermFree(outarg-ARGHEAD,"ExpandRat");
03560     AR.PolyFunPow = OldPolyFunPow;
03561     AT.WorkPointer = ow;
03562     AN.PolyNormFlag = 1;
03563     return(0);
03564 onerror:
03565     if ( outarg ) TermFree(outarg-ARGHEAD,"ExpandRat");
03566     AR.PolyFunPow = OldPolyFunPow;
03567     AT.WorkPointer = ow;
03568     MLOCK(ErrorMessageLock);
03569     MesPrint(TheErrorMessage[error]);
03570     MUNLOCK(ErrorMessageLock);
03571     Terminate(-1);
03572     return(-1);
03573 }
03574
03575 /*
03576 #] ExpandRat :
03577 #[ InvPoly :
03578
03579 The input polynomial is represented as a sequence of terms in ascending
03580 power. The first coefficient is 1. If we call this 1-a and
03581  $a = \sum_{j=1,n} x^j a(j)$ , and  $b = 1/(1-a)$  we can find the coefficients
03582 of b with the recursion
03583  $b(0) = 1, b(n) = \sum_{j=1,n} a(j) * b(n-j)$ 
03584 The variable is the symbol sym and we need maxpow powers in the answer.
03585 The answer is an array of pointers to the coefficients of the various
03586 powers as rational numbers in the notation signedsize,numerator,denominator
03587 We put these powers in the workspace and the answer is in AT.pWorkspace.
03588 Hence the return value is an offset in the pWorkspace.
03589 A zero pointer indicates that this coefficient is zero.
03590 */
03591
03592 int InvPoly(PHEAD WORD *inpoly, WORD maxpow, WORD sym)
03593 {
03594     int needed, inpointers, outpointers, maxinput = 0, i, j;
03595     WORD *t, *tt, *ttt, *w, *c, *cc, *ccc, lenc, lenc1, lenc2, rc, *c1, *c2;
03596     /*
03597     Step 0: allocate the space
03598     */
03599     needed = (maxpow+1)*2;
03600     WantAddPointers(needed);
03601     inpointers = AT.pWorkPointer;
03602     outpointers = AT.pWorkPointer+maxpow+1;
03603     for ( i = 0; i < needed; i++ ) AT.pWorkspace[inpointers+i] = 0;
03604     /*
03605     Step 1: determine the coefficients in inpoly
03606     often there is a maximum power that is much smaller than maxpow.
03607     keeping track of this can speed up things.
03608     */
03609     t = inpoly;
03610     w = AT.WorkPointer;
03611     while ( *t ) {
03612         if ( *t == 4 ) {
03613             if ( t[1] != 1 || t[2] != 1 || t[3] != 3 ) goto onerror;
03614             AT.pWorkspace[inpointers] = 0;
03615         }
03616         else if ( t[1] != SYMBOL || t[2] != 4 || t[3] != sym || t[4] < 0 ) goto onerror;
03617         else if ( t[4] > maxpow ) {} /* power outside useful range */
03618         else {
03619             if ( t[4] > maxinput ) maxinput = t[4];
03620             AT.pWorkspace[inpointers+t[4]] = w;
03621             tt = t + *t; rc = *--tt; /* we need - the coefficient! */
03622             rc = REDLENG(rc); *w++ = rc;
03623             ttt = t+5;
03624             while ( ttt < tt ) *w++ = *ttt++;
03625         }
03626         t += *t;
03627     }
03628     /*
03629     Step 2: compute the output. b(0) = 1.
03630     then the recursion starts.
03631     */
03632     AT.pWorkspace[outpointers] = w;
03633     *w++ = 1; *w++ = 1; *w++ = 1;
03634     c = TermMalloc("InvPoly");
03635     c1 = TermMalloc("InvPoly");
03636     c2 = TermMalloc("InvPoly");
03637     for ( j = 1; j <= maxpow; j++ ) {
03638     /*

```

```

03639         Start at c = a(j)*b(0) = a(j)
03640     */
03641     if ( ( cc = AT.pWorkspace[inpointers+j] ) != 0 ) {
03642         lenc = *cc++; /* reduced length */
03643         i = 2*ABS(lenc); ccc = c;
03644         NCOPY(ccc,cc,i);
03645     }
03646     else { lenc = 0; }
03647     for ( i = Min(j-1,maxinput); i > 0; i-- ) {
03648     /*
03649         c -> c + a(i)*b(j-i)
03650     */
03651         if ( AT.pWorkspace[inpointers+i] == 0
03652             || AT.pWorkspace[outpointers+j-i] == 0 ) {
03653         }
03654         else {
03655             if ( MulRat(BHEAD (UWORD
*) (AT.pWorkspace[inpointers+i]+1),AT.pWorkspace[inpointers+i][0],
03656                 (UWORD *) (AT.pWorkspace[outpointers+j-i]+1),AT.pWorkspace[outpointers+j-i][0],
03657                 (UWORD *)c1,&lenc1) ) goto calcerror;
03658             if ( lenc == 0 ) {
03659                 cc = c; c = c1; c1 = cc;
03660                 lenc = lenc1;
03661             }
03662             else {
03663                 if ( AddRat(BHEAD (UWORD *)c,lenc, (UWORD *)c1,lenc1, (UWORD *)c2,&lenc2) )
03664                     goto calcerror;
03665                 cc = c; c = c2; c2 = cc;
03666                 lenc = lenc2;
03667             }
03668         }
03669     }
03670     /*
03671     Copy c to the proper location
03672     */
03673     if ( lenc == 0 ) AT.pWorkspace[outpointers+j] = 0;
03674     else {
03675         AT.pWorkspace[outpointers+j] = w;
03676         *w++ = lenc;
03677         i = 2*ABS(lenc); ccc = c;
03678         NCOPY(w,ccc,i);
03679     }
03680 }
03681 AT.WorkPointer = w;
03682 TermFree(c2,"InvPoly");
03683 TermFree(c1,"InvPoly");
03684 TermFree(c,"InvPoly");
03685
03686 return(outpointers);
03687 onerror:
03688     MLOCK(ErrorMessageLock);
03689     MesPrint("Incorrect symbol field in InvPoly.");
03690     MUNLOCK(ErrorMessageLock);
03691     Terminate(-1);
03692     return(-1);
03693 calcerror:
03694     MLOCK(ErrorMessageLock);
03695     MesPrint("Called from InvPoly.");
03696     MUNLOCK(ErrorMessageLock);
03697     Terminate(-1);
03698     return(-1);
03699 }
03700
03701 /*
03702 #] InvPoly :
03703 */

```

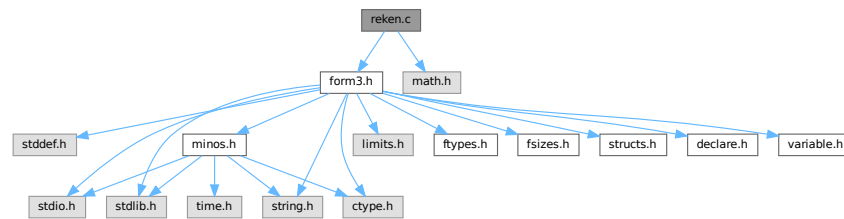
4.118 reken.c File Reference

```

#include "form3.h"
#include <math.h>

```

Include dependency graph for reken.c:



Macros

- `#define GCDMAX 3`
- `#define NEWTRICK 1`
- `#define COPYLONG(x1, nx1, x2, nx2) { int i; for(i=0;i<ABS(nx2);i++)x1[i]=x2[i];nx1=nx2; }`
- `#define WARMUP 6`

Functions

- void `Pack` (UWORD *a, WORD *na, UWORD *b, WORD nb)
- void `UnPack` (UWORD *a, WORD na, WORD *denom, WORD *numer)
- WORD `Mully` (PHEAD UWORD *a, WORD *na, UWORD *b, WORD nb)
- WORD `Divvy` (PHEAD UWORD *a, WORD *na, UWORD *b, WORD nb)
- WORD `AddRat` (PHEAD UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc)
- WORD `MulRat` (PHEAD UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc)
- WORD `DivRat` (PHEAD UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc)
- WORD `Simplify` (PHEAD UWORD *a, WORD *na, UWORD *b, WORD *nb)
- WORD `AccumGCD` (PHEAD UWORD *a, WORD *na, UWORD *b, WORD nb)
- int `TakeRatRoot` (UWORD *a, WORD *n, WORD power)
- WORD `AddLong` (UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc)
- WORD `AddPLon` (UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc)
- void `SubPLon` (UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc)
- WORD `MulLong` (UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc)
- WORD `BigLong` (UWORD *a, WORD na, UWORD *b, WORD nb)
- WORD `DivLong` (UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc, UWORD *d, WORD *nd)
- WORD `RaisPow` (PHEAD UWORD *a, WORD *na, UWORD b)
- void `RaisPowCached` (PHEAD WORD x, WORD n, UWORD **c, WORD *nc)
- WORD `RaisPowMod` (WORD x, WORD n, WORD m)
- int `NormalModulus` (UWORD *a, WORD *na)
- int `MakeInverses` (void)
- int `GetModInverses` (WORD m1, WORD m2, WORD *im1, WORD *im2)
- int `GetLongModInverses` (PHEAD UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *ia, WORD *nia, UWORD *ib, WORD *nib)
- WORD `Product` (UWORD *a, WORD *na, WORD b)
- UWORD `Quotient` (UWORD *a, WORD *na, WORD b)
- WORD `Remain10` (UWORD *a, WORD *na)
- WORD `Remain4` (UWORD *a, WORD *na)
- void `PrtLong` (UWORD *a, WORD na, UBYTE *s)
- WORD `GetLong` (UBYTE *s, UWORD *a, WORD *na)

- WORD [GcdLong](#) (PHEAD UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc)
- WORD [GetBinom](#) (UWORD *a, WORD *na, WORD i1, WORD i2)
- WORD [LcmLong](#) (PHEAD UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc)
- int [TakeLongRoot](#) (UWORD *a, WORD *n, WORD power)
- int [MakeRational](#) (WORD a, WORD m, WORD *b, WORD *c)
- int [MakeLongRational](#) (PHEAD UWORD *a, WORD na, UWORD *m, WORD nm, UWORD *b, WORD *nb)
- WORD [CompCoef](#) (WORD *term1, WORD *term2)
- WORD [Modulus](#) (WORD *term)
- WORD [TakeModulus](#) (UWORD *a, WORD *na, UWORD *cmodvec, WORD ncmmod, WORD par)
- WORD [TakeNormalModulus](#) (UWORD *a, WORD *na, UWORD *c, WORD nc, WORD par)
- WORD [MakeModTable](#) (void)
- int [Factorial](#) (PHEAD WORD n, UWORD *a, WORD *na)
- int [Bernoulli](#) (WORD n, UWORD *a, WORD *na)
- WORD [NextPrime](#) (PHEAD WORD num)
- WORD [Moebius](#) (PHEAD WORD nn)
- void [iniwranf](#) (PHEAD0)
- UWORD [wranf](#) (PHEAD0)
- UWORD [iranf](#) (PHEAD UWORD imax)
- UBYTE * [PreRandom](#) (UBYTE *s)

4.118.1 Detailed Description

This file contains the numerical routines. The arithmetic in FORM is normally over the rational numbers. Hence there are routines for dealing with integers and with rational of 'arbitrary precision' (within limits) There are also routines for that calculus modulus an integer. In addition there are the routines for factorials and Bernoulli numbers. The random number function is currently only for internal purposes.

Definition in file [reken.c](#).

4.118.2 Macro Definition Documentation

4.118.2.1 GCDMAX

```
#define GCDMAX 3
```

Definition at line 49 of file [reken.c](#).

4.118.2.2 NEWTRICK

```
#define NEWTRICK 1
```

Definition at line 51 of file [reken.c](#).

4.118.2.3 COPYLONG

```
#define COPYLONG(
    x1,
    nx1,
    x2,
    nx2 ) { int i; for(i=0;i<ABS(nx2);i++) x1[i]=x2[i];nx1=nx2; }
```

Definition at line 2901 of file [reken.c](#).

4.118.2.4 WARMUP

```
#define WARMUP 6
```

Definition at line [3801](#) of file [reken.c](#).

4.118.3 Function Documentation

4.118.3.1 Pack()

```
void Pack (
    UWORD * a,
    WORD * na,
    UWORD * b,
    WORD nb )
```

Definition at line [62](#) of file [reken.c](#).

4.118.3.2 UnPack()

```
void UnPack (
    UWORD * a,
    WORD na,
    WORD * denom,
    WORD * numer )
```

Definition at line [100](#) of file [reken.c](#).

4.118.3.3 Mully()

```
WORD Mully (
    PHEAD UWORD * a,
    WORD * na,
    UWORD * b,
    WORD nb )
```

Definition at line [126](#) of file [reken.c](#).

4.118.3.4 Divvy()

```
WORD Divvy (
    PHEAD UWORD * a,
    WORD * na,
    UWORD * b,
    WORD nb )
```

Definition at line [172](#) of file [reken.c](#).

4.118.3.5 AddRat()

```
WORD AddRat (
    PHEAD UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc )
```

Definition at line 210 of file [reken.c](#).

4.118.3.6 MulRat()

```
WORD MulRat (
    PHEAD UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc )
```

Definition at line 378 of file [reken.c](#).

4.118.3.7 DivRat()

```
WORD DivRat (
    PHEAD UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc )
```

Definition at line 500 of file [reken.c](#).

4.118.3.8 Simplify()

```
WORD Simplify (
    PHEAD UWORD * a,
    WORD * na,
    UWORD * b,
    WORD * nb )
```

Definition at line 530 of file [reken.c](#).

4.118.3.9 AccumGCD()

```
WORD AccumGCD (
    PHEAD UWORD * a,
    WORD * na,
    UWORD * b,
    WORD nb )
```

Definition at line 647 of file [reken.c](#).

4.118.3.10 TakeRatRoot()

```
int TakeRatRoot (
    UWORD * a,
    WORD * n,
    WORD power )
```

Definition at line 680 of file [reken.c](#).

4.118.3.11 AddLong()

```
WORD AddLong (
    UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc )
```

Definition at line 705 of file [reken.c](#).

4.118.3.12 AddPLon()

```
WORD AddPLon (
    UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc )
```

Definition at line 750 of file [reken.c](#).

4.118.3.13 SubPLon()

```
void SubPLon (
    UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc )
```

Definition at line 803 of file [reken.c](#).

4.118.3.14 MulLong()

```
WORD MulLong (
    UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc )
```

Definition at line 841 of file [reken.c](#).

4.118.3.15 BigLong()

```
WORD BigLong (
    UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb )
```

Definition at line 944 of file [reken.c](#).

4.118.3.16 DivLong()

```
WORD DivLong (
    UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc,
    UWORD * d,
    WORD * nd )
```

Definition at line 972 of file [reken.c](#).

4.118.3.17 RaisPow()

```
WORD RaisPow (
    PHEAD UWORD * a,
    WORD * na,
    UWORD b )
```

Definition at line 1228 of file [reken.c](#).

4.118.3.18 RaisPowCached()

```
void RaisPowCached (
    PHEAD WORD x,
    WORD n,
    UWORD ** c,
    WORD * nc )
```

Computes power x^n and caches the value

4.118.4 Description

Calculates the power x^n and stores the results for caching purposes. The pointer c (i.e., the pointer, and not what it points to) is overwritten. What it points to should not be overwritten in the calling function.

4.118.5 Notes

- Caching is done in `AT.small_power[]`. This array is extended if necessary.

Definition at line 1296 of file [reken.c](#).

Referenced by [poly::divmod_heap\(\)](#), [poly::divmod_univar\(\)](#), and [poly::mul_heap\(\)](#).

4.118.5.1 RaisPowMod()

```
WORD RaisPowMod (
    WORD x,
    WORD n,
    WORD m )
```

Definition at line 1383 of file [reken.c](#).

4.118.5.2 NormalModulus()

```
int NormalModulus (
    UWORD * a,
    WORD * na )
```

Brings a modular representation in the range $-p/2$ to $+p/2$ The return value tells whether anything was done. Routine made in the general modulus revamp of July 2008 (JV).

Definition at line 1403 of file [reken.c](#).

Referenced by [AddCoef\(\)](#), and [MergePatches\(\)](#).

4.118.5.3 MakeInverses()

```
int MakeInverses (
    void )
```

Makes a table of inverses in modular calculus The modulus is in `AC.cmod` and `AC.ncmod` One should notice that the table of inverses can only be made if the modulus fits inside a single FORM word. Otherwise the table lookup becomes too difficult and the table too long.

Definition at line 1440 of file [reken.c](#).

References [GetModInverses\(\)](#).

Here is the call graph for this function:



4.118.5.4 GetModInverses()

```
int GetModInverses (
    WORD m1,
    WORD m2,
    WORD * im1,
    WORD * im2 )
```

Input m1 and m2, which are relative prime. determines $a*m1+b*m2 = 1$ (and 1 is the gcd of m1 and m2) then $a*m1 = 1 \bmod m2$ and hence $im1 = a$. and $b*m2 = 1 \bmod m1$ and hence $im2 = b$. Set $m1 = 0*m1+1*m2 = a1*m1+b1*m2$ $m2 = 1*m1+0*m2 = a2*m1+b2*m2$ If everything is OK, the return value is zero

Definition at line [1476](#) of file [reken.c](#).

Referenced by [poly::divmod_heap\(\)](#), [poly::divmod_univar\(\)](#), [MakeDollarMod\(\)](#), [MakeInverses\(\)](#), [MakeMod\(\)](#), [TakeContent\(\)](#), and [TakeSymbolContent\(\)](#).

4.118.5.5 GetLongModInverses()

```
int GetLongModInverses (
    PHEAD UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * ia,
    WORD * nia,
    UWORD * ib,
    WORD * nib )
```

Definition at line [1508](#) of file [reken.c](#).

4.118.5.6 Product()

```
WORD Product (
    UWORD * a,
    WORD * na,
    WORD b )
```

Definition at line [1585](#) of file [reken.c](#).

4.118.5.7 Quotient()

```
UWORD Quotient (
    UWORD * a,
    WORD * na,
    WORD b )
```

Definition at line [1620](#) of file [reken.c](#).

4.118.5.8 Remain10()

```
WORD Remain10 (
    UWORD * a,
    WORD * na )
```

Definition at line [1659](#) of file [reken.c](#).

4.118.5.9 Remain4()

```
WORD Remain4 (
    UWORD * a,
    WORD * na )
```

Definition at line [1686](#) of file [reken.c](#).

4.118.5.10 PrtLong()

```
void PrtLong (
    UWORD * a,
    WORD na,
    UBYTE * s )
```

Definition at line [1712](#) of file [reken.c](#).

4.118.5.11 GetLong()

```
WORD GetLong (
    UBYTE * s,
    UWORD * a,
    WORD * na )
```

Definition at line [1774](#) of file [reken.c](#).

4.118.5.12 GcdLong()

```
WORD GcdLong (
    PHEAD UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc )
```

Definition at line [2276](#) of file [reken.c](#).

4.118.5.13 GetBinom()

```
WORD GetBinom (
    UWORD * a,
    WORD * na,
    WORD i1,
    WORD i2 )
```

Definition at line 2667 of file [reken.c](#).

4.118.5.14 LcmLong()

```
WORD LcmLong (
    PHEAD UWORD * a,
    WORD na,
    UWORD * b,
    WORD nb,
    UWORD * c,
    WORD * nc )
```

Definition at line 2701 of file [reken.c](#).

4.118.5.15 TakeLongRoot()

```
int TakeLongRoot (
    UWORD * a,
    WORD * n,
    WORD power )
```

Definition at line 2735 of file [reken.c](#).

4.118.5.16 MakeRational()

```
int MakeRational (
    WORD a,
    WORD m,
    WORD * b,
    WORD * c )
```

Definition at line 2855 of file [reken.c](#).

4.118.5.17 MakeLongRational()

```
int MakeLongRational (
    PHEAD UWORD * a,
    WORD na,
    UWORD * m,
    WORD nm,
    UWORD * b,
    WORD * nb )
```

Definition at line 2903 of file [reken.c](#).

4.118.5.18 CompCoef()

```
WORD CompCoef (
    WORD * term1,
    WORD * term2 )
```

Routine takes $a_1 \bmod m_1$ and $a_2 \bmod m_2$ and returns $a \bmod m_1*m_2$ with
 $a \bmod m_1 = a_1$ and $a \bmod m_2 = a_2$

Chinese remainder: $a \bmod (m_1*m_2) = q_1*m_1 + a_1$ $a \bmod (m_1*m_2) = q_2*m_2 + a_2$ Compute n_1 such that $(n_1*m_1) \bmod m_2$ is one
 Compute n_2 such that $(n_2*m_2) \bmod m_1$ is one Then $(a_1*n_2*m_2 + a_2*n_1*m_1) \bmod (m_1*m_2)$ is $a \bmod (m_1*m_2)$

Definition at line 3047 of file [reken.c](#).

Referenced by [Compare1\(\)](#).

4.118.5.19 Modulus()

```
WORD Modulus (
    WORD * term )
```

Definition at line 3102 of file [reken.c](#).

4.118.5.20 TakeModulus()

```
WORD TakeModulus (
    UWORD * a,
    WORD * na,
    UWORD * cmovec,
    WORD ncmod,
    WORD par )
```

Definition at line 3149 of file [reken.c](#).

4.118.5.21 TakeNormalModulus()

```
WORD TakeNormalModulus (
    UWORD * a,
    WORD * na,
    UWORD * c,
    WORD nc,
    WORD par )
```

Definition at line 3321 of file [reken.c](#).

4.118.5.22 MakeModTable()

```
WORD MakeModTable (
    void )
```

Definition at line 3363 of file [reken.c](#).

4.118.5.23 Factorial()

```
int Factorial (
    PHEAD WORD n,
    UWORD * a,
    WORD * na )
```

Definition at line 3449 of file [reken.c](#).

4.118.5.24 Bernoulli()

```
int Bernoulli (
    WORD n,
    UWORD * a,
    WORD * na )
```

Definition at line 3529 of file [reken.c](#).

4.118.5.25 NextPrime()

```
WORD NextPrime (
    PHEAD WORD num )
```

Gives the next prime number in the list of prime numbers.

If the list isn't long enough we expand it. For ease in ParForm and because these lists shouldn't be very big we let each worker keep its own list.

The list is cut off at MAXPOWER, because we don't want to get into trouble that the power of a variable gets larger than the prime number.

Definition at line 3664 of file [reken.c](#).

4.118.5.26 Moebius()

```
WORD Moebius (
    PHEAD WORD nn )
```

Definition at line 3728 of file [reken.c](#).

4.118.5.27 iniwranf()

```
void iniwranf (
    PHEAD0 )
```

Definition at line 3819 of file [reken.c](#).

4.118.5.28 wranf()

```
UWORD wranf (
    PHEAD0 )
```

Definition at line [3868](#) of file [reken.c](#).

4.118.5.29 iranf()

```
UWORD iranf (
    PHEAD UWORD imax )
```

Definition at line [3887](#) of file [reken.c](#).

4.118.5.30 PreRandom()

```
UBYTE * PreRandom (
    UBYTE * s )
```

Definition at line [3912](#) of file [reken.c](#).

4.119 reken.c

[Go to the documentation of this file.](#)

```
00001
00011 /* #[] License : */
00012 /*
00013 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00014 *   When using this file you are requested to refer to the publication
00015 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00016 *   This is considered a matter of courtesy as the development was paid
00017 *   for by FOM the Dutch physics granting agency and we would like to
00018 *   be able to track its scientific use to convince FOM of its value
00019 *   for the community.
00020 *
00021 *   This file is part of FORM.
00022 *
00023 *   FORM is free software: you can redistribute it and/or modify it under the
00024 *   terms of the GNU General Public License as published by the Free Software
00025 *   Foundation, either version 3 of the License, or (at your option) any later
00026 *   version.
00027 *
00028 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00029 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00030 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00031 *   details.
00032 *
00033 *   You should have received a copy of the GNU General Public License along
00034 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00035 */
00036 /* #[] License : */
00037 /*
00038 *   #[] Includes : reken.c
00039 */
00040
00041 #include "form3.h"
00042 #include <math.h>
00043
00044 #ifdef WITHGMP
00045 #include <gmp.h>
00046 #define GMPSPREAD (GMP_LIMB_BITS/BITSINWORD)
00047 #endif
00048
00049 #define GCDMAX 3
00050
00051 #define NEWTRICK 1
```



```

00052 /*
00053  #] Includes :
00054  #[ RekenRational :
00055      #] Pack :          void Pack(a,na,b,nb)
00056
00057      Packs the contents of the numerator a and the denominator b into
00058      one normalized fraction a.
00059 */
00060
00061 void Pack(UWORD *a, WORD *na, UWORD *b, WORD nb)
00062 {
00063     WORD c, sgn = 1, i;
00064     UWORD *to,*from;
00065     if ( (c = *na) == 0 ) {
00066         MLOCK(ErrorMessageLock);
00067         MesPrint("Caught a zero in Pack");
00068         MUNLOCK(ErrorMessageLock);
00069         return;
00070     }
00071     if ( nb == 0 ) {
00072         MLOCK(ErrorMessageLock);
00073         MesPrint("Division by zero in Pack");
00074         MUNLOCK(ErrorMessageLock);
00075         return;
00076     }
00077     if ( *na < 0 ) { sgn = -sgn; c = -c; }
00078     if ( nb < 0 ) { sgn = -sgn; nb = -nb; }
00079     *na = MaX(c,nb);
00080     to = a + c;
00081     i = *na - c;
00082     while ( --i >= 0 ) *to++ = 0;
00083     i = *na - nb;
00084     from = b;
00085     NCOPY(to,from,nb);
00086     while ( --i >= 0 ) *to++ = 0;
00087     if ( sgn < 0 ) *na = -*na;
00088 }
00089
00090
00091 /*
00092  #] Pack :
00093  #[ UnPack :          void UnPack(a,na,denom,number)
00094
00095      Determines the sizes of the numerator and the denominator in the
00096      normalized fraction a with length na.
00097 */
00098
00099 void UnPack(UWORD *a, WORD na, WORD *denom, WORD *number)
00100 {
00101     UWORD *pos;
00102     WORD i, sgn = na;
00103     if ( na < 0 ) { na = -na; }
00104     i = na;
00105     if ( i > 1 ) {          /* Find the respective leading words */
00106         a += i;
00107         a--;
00108         pos = a + i;
00109         while ( !(*a) ) { i--; a--; }
00110         while ( !(*pos) ) { na--; pos--; }
00111     }
00112     *denom = na;
00113     if ( sgn < 0 ) i = -i;
00114     *number = i;
00115 }
00116
00117
00118 /*
00119  #] UnPack :
00120  #[ Mully :          WORD Mully(a,na,b,nb)
00121
00122      Multiplies the rational a by the Long b.
00123 */
00124
00125 WORD Mully(PHEAD UWORD *a, WORD *na, UWORD *b, WORD nb)
00126 {
00127     GETBIDENTITY
00128     UWORD *d, *e;
00129     WORD i, sgn = 1;
00130     WORD nd, ne, adenom, anumer;
00131     if ( !nb ) { *na = 0; return(0); }
00132     else if ( *b == 1 ) {
00133         if ( nb == 1 ) return(0);
00134         else if ( nb == -1 ) { *na = -*na; return(0); }
00135     }
00136     if ( *na < 0 ) { sgn = -sgn; *na = -*na; }
00137     if ( nb < 0 ) { sgn = -sgn; nb = -nb; }

```

```

00139     UnPack(a,*na,&adenom,&anumer);
00140     d = NumberMalloc("Mully"); e = NumberMalloc("Mully");
00141     for ( i = 0; i < nb; i++ ) { e[i] = *b++; }
00142     ne = nb;
00143     if ( Simplify(BHEAD a+*na,&adenom,e,&ne) ) goto MullyEr;
00144     if ( MullLong(a,anumer,e,ne,d,&nd) ) goto MullyEr;
00145     b = a+*na;
00146     for ( i = 0; i < *na; i++ ) { e[i] = *b++; }
00147     ne = adenom;
00148     *na = nd;
00149     b = d;
00150     *na = nd;
00151     for ( i = 0; i < *na; i++ ) { a[i] = *b++; }
00152     Pack(a,na,e,ne);
00153     if ( sgn < 0 ) *na = -*na;
00154     NumberFree(d,"Mully"); NumberFree(e,"Mully");
00155     return(0);
00156 MullyEr:
00157     MLOCK(ErrorMessageLock);
00158     MesCall("Mully");
00159     MUNLOCK(ErrorMessageLock);
00160     NumberFree(d,"Mully"); NumberFree(e,"Mully");
00161     SETERROR(-1)
00162 }
00163
00164 /*
00165     #] Mully :
00166     #[ Divvy :          WORD Divvy(a,na,b,nb)
00167
00168     Divides the rational a by the Long b.
00169 */
00170
00171 WORD Divvy(PHEAD UWORD *a, WORD *na, UWORD *b, WORD nb)
00172 {
00173     GETBIDENTITY
00174     UWORD *d,*e;
00175     WORD i, sgn = 1;
00176     WORD nd, ne, adenom, anumer;
00177     if ( !nb ) {
00178         MLOCK(ErrorMessageLock);
00179         MesPrint("Division by zero in Divvy");
00180         MUNLOCK(ErrorMessageLock);
00181         return(-1);
00182     }
00183     d = NumberMalloc("Divvy"); e = NumberMalloc("Divvy");
00184     if ( nb < 0 ) { sgn = -sgn; nb = -nb; }
00185     if ( *na < 0 ) { sgn = -sgn; *na = -*na; }
00186     UnPack(a,*na,&adenom,&anumer);
00187     for ( i = 0; i < nb; i++ ) { e[i] = *b++; }
00188     ne = nb;
00189     if ( Simplify(BHEAD a,&anumer,e,&ne) ) goto DivvyEr;
00190     if ( MullLong(a+*na,adenom,e,ne,d,&nd) ) goto DivvyEr;
00191     *na = anumer;
00192     Pack(a,na,d,nd);
00193     if ( sgn < 0 ) *na = -*na;
00194     NumberFree(d,"Divvy"); NumberFree(e,"Divvy");
00195     return(0);
00196 DivvyEr:
00197     MLOCK(ErrorMessageLock);
00198     MesCall("Divvy");
00199     MUNLOCK(ErrorMessageLock);
00200     NumberFree(d,"Divvy"); NumberFree(e,"Divvy");
00201     SETERROR(-1)
00202 }
00203
00204 /*
00205     #] Divvy :
00206     #[ AddRat :          WORD AddRat(a,na,b,nb,c,nc)
00207
00208 */
00209
00210 WORD AddRat(PHEAD UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc)
00211 {
00212     GETBIDENTITY
00213     UWORD *d, *e, *f, *g;
00214     WORD nd, ne, nf, ng, adenom, anumer, bdenom, bnumer;
00215     if ( !na ) {
00216         WORD i;
00217         *nc = nb;
00218         if ( nb < 0 ) nb = -nb;
00219         nb *= 2;
00220         for ( i = 0; i < nb; i++ ) *c++ = *b++;
00221         return(0);
00222     }
00223     else if ( !nb ) {
00224         WORD i;
00225         *nc = na;

```

```

00226         if ( na < 0 ) na = -na;
00227         na *= 2;
00228         for ( i = 0; i < na; i++ ) *c++ = *a++;
00229         return(0);
00230     }
00231     else if ( b[1] == 1 && a[1] == 1 ) {
00232         if ( na == 1 ) {
00233             if ( nb == 1 ) {
00234                 *c = *a + *b;
00235                 c[1] = 1;
00236                 if ( *c < *a ) { c[2] = 1; c[3] = 0; *nc = 2; }
00237                 else { *nc = 1; }
00238                 return(0);
00239             }
00240             else if ( nb == -1 ) {
00241                 if ( *b > *a ) {
00242                     *c = *b - *a; *nc = -1;
00243                 }
00244                 else if ( *b < *a ) {
00245                     *c = *a - *b; *nc = 1;
00246                 }
00247                 else *nc = 0;
00248                 c[1] = 1;
00249                 return(0);
00250             }
00251         }
00252         else if ( na == -1 ) {
00253             if ( nb == -1 ) {
00254                 c[1] = 1;
00255                 *c = *a + *b;
00256                 if ( *c < *a ) { c[2] = 1; c[3] = 0; *nc = -2; }
00257                 else { *nc = -1; }
00258                 return(0);
00259             }
00260             else if ( nb == 1 ) {
00261                 if ( *b > *a ) {
00262                     *c = *b - *a; *nc = 1;
00263                 }
00264                 else if ( *b < *a ) {
00265                     *c = *a - *b; *nc = -1;
00266                 }
00267                 else *nc = 0;
00268                 c[1] = 1;
00269                 return(0);
00270             }
00271         }
00272     }
00273     UnPack(a,na,&adenom,&anumer);
00274     UnPack(b,nb,&bdenom,&bnumber);
00275     if ( na < 0 ) na = -na;
00276     if ( nb < 0 ) nb = -nb;
00277     if ( na == 1 && nb == 1 ) {
00278         RLONG t1, t2, t3;
00279         t3 = ((RLONG)a[1])*((RLONG)b[1]);
00280         t1 = ((RLONG)a[0])*((RLONG)b[1]);
00281         t2 = ((RLONG)a[1])*((RLONG)b[0]);
00282         if ( ( anumer > 0 && bnumber > 0 ) || ( anumer < 0 && bnumber < 0 ) ) {
00283             if ( ( t1 = t1 + t2 ) < t2 ) {
00284                 c[2] = 1;
00285                 c[0] = (UWORD)t1;
00286                 c[1] = (UWORD)(t1 » BITSINWORD);
00287                 *nc = 3;
00288             }
00289             else {
00290                 c[0] = (UWORD)t1;
00291                 if ( ( c[1] = (UWORD)(t1 » BITSINWORD) ) != 0 ) *nc = 2;
00292                 else *nc = 1;
00293             }
00294         }
00295         else {
00296             if ( t1 == t2 ) { *nc = 0; return(0); }
00297             if ( t1 > t2 ) {
00298                 t1 -= t2;
00299             }
00300             else {
00301                 t1 = t2 - t1;
00302                 anumer = -anumer;
00303             }
00304             c[0] = (UWORD)t1;
00305             if ( ( c[1] = (UWORD)(t1 » BITSINWORD) ) != 0 ) *nc = 2;
00306             else *nc = 1;
00307         }
00308         if ( anumer < 0 ) *nc = -*nc;
00309         d = NumberMalloc("AddRat");
00310         d[0] = (UWORD)t3;
00311         if ( ( d[1] = (UWORD)(t3 » BITSINWORD) ) != 0 ) nd = 2;
00312         else nd = 1;

```

```

00313         if ( Simplify(BHEAD c,nc,d,&nd) ) goto AddRer1;
00314     }
00315 /*
00316     else if ( a[na] == 1 && b[nb] == 1 && adenom == 1 && bdenom == 1 ) {
00317         if ( AddLong(a,na,b,nb,c,&nc) ) goto AddRer2;
00318         i = ABS(nc); d = c + i; *d++ = 1;
00319         while ( --i > 0 ) *d++ = 0 ;
00320         return(0);
00321     }
00322 */
00323     else {
00324         d = NumberMalloc("AddRat"); e = NumberMalloc("AddRat");
00325         f = NumberMalloc("AddRat"); g = NumberMalloc("AddRat");
00326         if ( GcdLong(BHEAD a+na,adenom,b+nb,bdenom,d,&nd) ) goto AddRer;
00327         if ( *d == 1 && nd == 1 ) nd = 0;
00328         if ( nd ) {
00329             if ( DivLong(a+na,adenom,d,nd,e,&ne,c,nc) ) goto AddRer;
00330             if ( DivLong(b+nb,bdenom,d,nd,f,&nf,c,nc) ) goto AddRer;
00331             if ( MulLong(a,anumer,f,nf,c,nc) ) goto AddRer;
00332             if ( MulLong(b,bnumer,e,ne,g,&ng) ) goto AddRer;
00333         }
00334         else {
00335             if ( MulLong(a+na,adenom,b,bnumer,c,nc) ) goto AddRer;
00336             if ( MulLong(b+nb,bdenom,a,anumer,g,&ng) ) goto AddRer;
00337         }
00338         if ( AddLong(c,*nc,g,ng,c,nc) ) goto AddRer;
00339         if ( !*nc ) {
00340             NumberFree(g,"AddRat"); NumberFree(f,"AddRat");
00341             NumberFree(e,"AddRat"); NumberFree(d,"AddRat");
00342             return(0);
00343         }
00344         if ( nd ) {
00345             if ( Simplify(BHEAD c,nc,d,&nd) ) goto AddRer;
00346             if ( MulLong(e,ne,d,nd,g,&ng) ) goto AddRer;
00347             if ( MulLong(g,ng,f,nf,d,&nd) ) goto AddRer;
00348         }
00349         else {
00350             if ( MulLong(a+na,adenom,b+nb,bdenom,d,&nd) ) goto AddRer;
00351         }
00352         NumberFree(g,"AddRat"); NumberFree(f,"AddRat"); NumberFree(e,"AddRat");
00353     }
00354     Pack(c,nc,d,nd);
00355     NumberFree(d,"AddRat");
00356     return(0);
00357 AddRer:
00358     NumberFree(g,"AddRat"); NumberFree(f,"AddRat"); NumberFree(e,"AddRat");
00359 AddRer1:
00360     NumberFree(d,"AddRat");
00361 /* AddRer2: */
00362     MLOCK(ErrorMessageLock);
00363     MesCall("AddRat");
00364     MUNLOCK(ErrorMessageLock);
00365     SETERROR(-1)
00366 }
00367
00368 /*
00369     #] AddRat :
00370     #[ MulRat :          WORD MulRat(a,na,b,nb,c,nc)
00371
00372     Multiplies the rationals a and b. The Gcd of the individual
00373     pieces is divided out first to minimize the chances of spurious
00374     overflows.
00375
00376 */
00377
00378 WORD MulRat(PHEAD UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc)
00379 {
00380     WORD i;
00381     WORD sgn = 1;
00382     if ( *b == 1 && b[1] == 1 ) {
00383         if ( nb == 1 ) {
00384             *nc = na;
00385             i = ABS(na); i *= 2;
00386             while ( --i >= 0 ) *c++ = *a++;
00387             return(0);
00388         }
00389         else if ( nb == -1 ) {
00390             *nc = - na;
00391             i = ABS(na); i *= 2;
00392             while ( --i >= 0 ) *c++ = *a++;
00393             return(0);
00394         }
00395     }
00396     if ( *a == 1 && a[1] == 1 ) {
00397         if ( na == 1 ) {
00398             *nc = nb;
00399             i = ABS(nb); i *= 2;

```

```

00400         while ( --i >= 0 ) *c++ = *b++;
00401         return(0);
00402     }
00403     else if ( na == -1 ) {
00404         *nc = - nb;
00405         i = ABS(nb); i *= 2;
00406         while ( --i >= 0 ) *c++ = *b++;
00407         return(0);
00408     }
00409 }
00410 if ( na < 0 ) { na = -na; sgn = -sgn; }
00411 if ( nb < 0 ) { nb = -nb; sgn = -sgn; }
00412 if ( !na || !nb ) { *nc = 0; return(0); }
00413 if ( na != 1 || nb != 1 ) {
00414     GETBIDENTITY
00415     UWORD *xd,*xe, *xf,*xg;
00416     WORD dden, dnumr, eden, enumr;
00417     UnPack(a,na,&dden,&dnumr);
00418     UnPack(b,nb,&eden,&enumr);
00419     xd = NumberMalloc("MulRat"); xf = NumberMalloc("MulRat");
00420     for ( i = 0; i < dnumr; i++ ) xd[i] = a[i];
00421     a += na;
00422     for ( i = 0; i < dden; i++ ) xf[i] = a[i];
00423     xe = NumberMalloc("MulRat"); xg = NumberMalloc("MulRat");
00424     for ( i = 0; i < enumr; i++ ) xe[i] = b[i];
00425     b += nb;
00426     for ( i = 0; i < eden; i++ ) xg[i] = b[i];
00427     if ( Simplify(BHEAD xd,&dnumr,xg,&eden) ||
00428         Simplify(BHEAD xe,&enumr,xf,&dden) ||
00429         MulLong(xd,dnumr,xe,enumr,c,nc) ||
00430         MulLong(xf,dden,xg,eden,xd,&dnumr) ) {
00431         MLOCK(ErrorMessageLock);
00432         MesCall("MulRat");
00433         MUNLOCK(ErrorMessageLock);
00434         NumberFree(xd,"MulRat"); NumberFree(xe,"MulRat"); NumberFree(xf,"MulRat");
00435         NumberFree(xg,"MulRat");
00436         SETERROR(-1)
00437     }
00438     Pack(c,nc,xd,dnumr);
00439     NumberFree(xd,"MulRat"); NumberFree(xe,"MulRat"); NumberFree(xf,"MulRat");
00440     NumberFree(xg,"MulRat");
00441 }
00442 else {
00443     UWORD y;
00444     UWORD a0,a1,b0,b1;
00445     RLONG xx;
00446     y = a[0]; b1=b[1];
00447     do { a0 = y % b1; y = b1; } while ( ( b1 = a0 ) != 0 );
00448     if ( y != 1 ) {
00449         a0 = a[0] / y;
00450         b1 = b[1] / y;
00451     }
00452     else {
00453         a0 = a[0];
00454         b1 = b[1];
00455     }
00456     y=b[0]; a1=a[1];
00457     do { b0 = y % a1; y = a1; } while ( ( a1 = b0 ) != 0 );
00458     if ( y != 1 ) {
00459         a1 = a[1] / y;
00460         b0 = b[0] / y;
00461     }
00462     else {
00463         a1 = a[1];
00464         b0 = b[0];
00465     }
00466     xx = ((RLONG)a0)*b0;
00467     if ( xx & AWORDMASK ) {
00468         *nc = 2;
00469         c[0] = (UWORD)xx;
00470         c[1] = (UWORD)(xx >> BITSINWORD);
00471         xx = ((RLONG)a1)*b1;
00472         c[2] = (UWORD)xx;
00473         c[3] = (UWORD)(xx >> BITSINWORD);
00474     }
00475     else {
00476         c[0] = (UWORD)xx;
00477         xx = ((RLONG)a1)*b1;
00478         if ( xx & AWORDMASK ) {
00479             c[1] = 0;
00480             c[2] = (UWORD)xx;
00481             c[3] = (UWORD)(xx >> BITSINWORD);
00482             *nc = 2;
00483         }
00484         else {
00485             c[1] = (UWORD)xx;
00486             *nc = 1;

```

```

00485     }
00486     }
00487 }
00488 if ( sgn < 0 ) *nc = -*nc;
00489 return(0);
00490 }
00491
00492 /*
00493     #] MulRat :
00494     #[ DivRat :      WORD DivRat(a,na,b,nb,c,nc)
00495
00496     Divides the rational a by the rational b.
00497 */
00498
00499 WORD DivRat(PHEAD UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc)
00500 {
00501     GETBIDENTITY
00502     WORD i, j;
00503     UWORD *xd,*xe,xx;
00504     if ( !nb ) {
00505         MLOCK(ErrorMessageLock);
00506         MesPrint("Rational division by zero");
00507         MUNLOCK(ErrorMessageLock);
00508         return(-1);
00509     }
00510     j = i = (nb >= 0)? nb: -nb;
00511     xd = b; xe = b + i;
00512     do { xx = *xd; *xd++ = *xe; *xe++ = xx; } while ( --j > 0 );
00513     j = MulRat(BHEAD a,na,b,nb,c,nc);
00514     xd = b; xe = b + i;
00515     do { xx = *xd; *xd++ = *xe; *xe++ = xx; } while ( --i > 0 );
00516     return(j);
00517 }
00518
00519 /*
00520     #] DivRat :
00521     #[ Simplify :      WORD Simplify(a,na,b,nb)
00522
00523     Determines the greatest common denominator of a and b and
00524     divides both by it. A possible sign is put in a. This is
00525     the simplification of the fraction a/b.
00526 */
00527
00528 WORD Simplify(PHEAD UWORD *a, WORD *na, UWORD *b, WORD *nb)
00529 {
00530     GETBIDENTITY
00531     UWORD *x1,*x2,*x3;
00532     UWORD *x4;
00533     WORD n1,n2,n3,n4,sgn = 1;
00534     WORD i;
00535     UWORD *Siscrat5, *Siscrat6, *Siscrat7, *Siscrat8;
00536     if ( *na < 0 ) { *na = -*na; sgn = -sgn; }
00537     if ( *nb < 0 ) { *nb = -*nb; sgn = -sgn; }
00538     Siscrat5 = NumberMalloc("Simplify"); Siscrat6 = NumberMalloc("Simplify");
00539     Siscrat7 = NumberMalloc("Simplify"); Siscrat8 = NumberMalloc("Simplify");
00540     x1 = Siscrat8; x2 = Siscrat7;
00541     if ( *nb == 1 ) {
00542         x3 = Siscrat6;
00543         if ( DivLong(a,*na,b,*nb,x1,&n1,x2,&n2) ) goto SimpErr;
00544         if ( !n2 ) {
00545             for ( i = 0; i < n1; i++ ) *a++ = *x1++;
00546             *na = n1;
00547             *b = 1;
00548         }
00549     }
00550     else {
00551         UWORD y1, y2, y3;
00552         y2 = *b;
00553         y3 = *x2;
00554         do { y1 = y2 % y3; y2 = y3; } while ( ( y3 = y1 ) != 0 );
00555         if ( ( *x2 = y2 ) != 1 ) {
00556             *b /= y2;
00557             if ( DivLong(a,*na,x2,(WORD)1,x1,&n1,x3,&n3) ) goto SimpErr;
00558             for ( i = 0; i < n1; i++ ) *a++ = *x1++;
00559             *na = n1;
00560         }
00561     }
00562 }
00563 }
00564 #ifndef NEWTRICK
00565 else if ( *na >= GCDMAX && *nb >= GCDMAX ) {
00566     n1 = i = *na; x3 = a;
00567     NCOPY(x1,x3,i);
00568     x3 = b; n2 = i = *nb;
00569     NCOPY(x2,x3,i);
00570     x4 = Siscrat5;
00571     x2 = Siscrat6;

```

```

00572     x3 = Siscrat7;
00573     if ( GcdLong(BHEAD Siscrat8,n1,Siscrat7,n2,x2,&n3) ) goto SimpErr;
00574     n2 = n3;
00575     if ( *x2 != 1 || n2 != 1 ) {
00576         DivLong(a,*na,x2,n2,x1,&n1,x4,&n4);
00577         *na = i = n1;
00578         NCOPY(a,x1,i);
00579         DivLong(b,*nb,x2,n2,x3,&n3,x4,&n4);
00580         *nb = i = n3;
00581         NCOPY(b,x3,i);
00582     }
00583 }
00584 #endif
00585 else {
00586     x4 = Siscrat5;
00587     n1 = i = *na; x3 = a;
00588     NCOPY(x1,x3,i);
00589     x3 = b; n2 = i = *nb;
00590     NCOPY(x2,x3,i);
00591     x1 = Siscrat8; x2 = Siscrat7; x3 = Siscrat6;
00592     for(;;){
00593         if ( DivLong(x1,n1,x2,n2,x4,&n4,x3,&n3) ) goto SimpErr;
00594         if ( !n3 ) break;
00595         if ( n2 == 1 ) {
00596             while ( ( *x1 = (*x2) % (*x3) ) != 0 ) { *x2 = *x3; *x3 = *x1; }
00597             *x2 = *x3;
00598             break;
00599         }
00600         if ( DivLong(x2,n2,x3,n3,x4,&n4,x1,&n1) ) goto SimpErr;
00601         if ( !n1 ) { x2 = x3; n2 = n3; x3 = Siscrat7; break; }
00602         if ( n3 == 1 ) {
00603             while ( ( *x2 = (*x3) % (*x1) ) != 0 ) { *x3 = *x1; *x1 = *x2; }
00604             *x2 = *x1;
00605             n2 = 1;
00606             break;
00607         }
00608         if ( DivLong(x3,n3,x1,n1,x4,&n4,x2,&n2) ) goto SimpErr;
00609         if ( !n2 ) { x2 = x1; n2 = n1; x1 = Siscrat7; break; }
00610         if ( n1 == 1 ) {
00611             while ( ( *x3 = (*x1) % (*x2) ) != 0 ) { *x1 = *x2; *x2 = *x3; }
00612             break;
00613         }
00614     }
00615     if ( *x2 != 1 || n2 != 1 ) {
00616         DivLong(a,*na,x2,n2,x1,&n1,x4,&n4);
00617         *na = i = n1;
00618         NCOPY(a,x1,i);
00619         DivLong(b,*nb,x2,n2,x3,&n3,x4,&n4);
00620         *nb = i = n3;
00621         NCOPY(b,x3,i);
00622     }
00623 }
00624 if ( sgn < 0 ) *na = -*na;
00625 NumberFree(Siscrat5,"Simplify"); NumberFree(Siscrat6,"Simplify");
00626 NumberFree(Siscrat7,"Simplify"); NumberFree(Siscrat8,"Simplify");
00627 return(0);
00628 SimpErr:
00629 MLOCK(ErrorMessageLock);
00630 MesCall("Simplify");
00631 MUNLOCK(ErrorMessageLock);
00632 NumberFree(Siscrat5,"Simplify"); NumberFree(Siscrat6,"Simplify");
00633 NumberFree(Siscrat7,"Simplify"); NumberFree(Siscrat8,"Simplify");
00634 SETERROR(-1)
00635 }
00636
00637 /*
00638     #] Simplify :
00639     #[ AccumGCD :      WORD AccumGCD(PHEAD a,na,b,nb)
00640
00641     Routine takes the rational GCD of the fractions in a and b and
00642     replaces a by the GCD of the two.
00643     The rational GCD is defined as the rational that consists of
00644     the GCD of the numerators divided by the GCD of the denominators
00645 */
00646
00647 WORD AccumGCD(PHEAD UWORD *a, WORD *na, UWORD *b, WORD nb)
00648 {
00649     GETBIDENTITY
00650     WORD nna,nnb,numa,numb,dena,denb,numc,denc;
00651     UWORD *GCDbuffer = NumberMalloc("AccumGCD");
00652     int i;
00653     nna = *na; if ( nna < 0 ) nna = -nna; nna = (nna-1)/2;
00654     nnb = nb; if ( nnb < 0 ) nnb = -nnb; nnb = (nnb-1)/2;
00655     UnPack(a,nna,&dena,&numa);
00656     UnPack(b,nnb,&denb,&numb);
00657     if ( GcdLong(BHEAD a,numa,b,numb,GCDbuffer,&numc) ) goto AccErr;
00658     numa = numc;

```

```

00659     for ( i = 0; i < numa; i++ ) a[i] = GCDbuffer[i];
00660     if ( GcdLong(BHEAD a+nna,dena,b+nnb,denb,GCDbuffer,&denc) ) goto AccErr;
00661     dena = denc;
00662     for ( i = 0; i < dena; i++ ) a[i+nna] = GCDbuffer[i];
00663     Pack(a,&numa,a+nna,dena);
00664     *na = INCLENG(numa);
00665     NumberFree(GCDbuffer,"AccumGCD");
00666     return(0);
00667 AccErr:
00668     MLOCK(ErrorMessageLock);
00669     MesCall("AccumGCD");
00670     MUNLOCK(ErrorMessageLock);
00671     NumberFree(GCDbuffer,"AccumGCD");
00672     SETERROR(-1)
00673 }
00674
00675 /*
00676     #] AccumGCD :
00677     #[ TakeRatRoot:
00678 */
00679
00680 int TakeRatRoot( UWORD *a, WORD *n, WORD power)
00681 {
00682     WORD numer,denom, nn;
00683     if ( ( power & 1 ) == 0 && *n < 0 ) return(1);
00684     if ( ABS(*n) == 1 && a[0] == 1 && a[1] == 1 ) return(0);
00685     nn = ABS(*n);
00686     UnPack(a,nn,&denom,&numer);
00687     if ( TakeLongRoot(a+nn,&denom,power) ) return(1);
00688     if ( TakeLongRoot(a,&numer,power) ) return(1);
00689     Pack(a,&numer,a+nn,denom);
00690     if ( *n < 0 ) *n = -numer;
00691     else *n = numer;
00692     return(0);
00693 }
00694
00695 /*
00696     #] TakeRatRoot:
00697     #] RekenRational :
00698     #[ RekenLong :
00699     #[ AddLong :          WORD AddLong(a,na,b,nb,c,nc)
00700
00701     Long addition. Uses addition and subtraction of positive numbers.
00702
00703 */
00704
00705 WORD AddLong( UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc)
00706 {
00707     WORD sgn, res;
00708     if ( na < 0 ) {
00709         if ( nb < 0 ) {
00710             if ( AddPLon(a,-na,b,-nb,c,nc) ) return(-1);
00711             *nc = -*nc;
00712             return(0);
00713         }
00714         else {
00715             na = -na;
00716             sgn = -1;
00717         }
00718     }
00719     else {
00720         if ( nb < 0 ) {
00721             nb = -nb;
00722             sgn = 1;
00723         }
00724         else { return( AddPLon(a,na,b,nb,c,nc) ); }
00725     }
00726     if ( ( res = BigLong(a,na,b,nb) ) > 0 ) {
00727         SubPLon(a,na,b,nb,c,nc);
00728         if ( sgn < 0 ) *nc = -*nc;
00729     }
00730     else if ( res < 0 ) {
00731         SubPLon(b,nb,a,na,c,nc);
00732         if ( sgn > 0 ) *nc = -*nc;
00733     }
00734     else {
00735         *nc = 0;
00736         *c = 0;
00737     }
00738     return(0);
00739 }
00740
00741 /*
00742     #] AddLong :
00743     #[ AddPLon :          WORD AddPLon(a,na,b,nb,c,nc)
00744
00745     Adds two long integers a and b and puts the result in c.

```



```

00746     The length of a and b are na and nb. The length of c is returned in nc.
00747     c can be a or b.
00748 */
00749
00750 WORD AddPLon(WORD *a, WORD na, WORD *b, WORD nb, WORD *c, WORD *nc)
00751 {
00752     UWORD carry = 0, e, nd = 0;
00753     while ( na && nb ) {
00754         e = *a;
00755         *c = e + *b + carry;
00756         if ( carry ) {
00757             if ( e < *c ) carry = 0;
00758         }
00759         else {
00760             if ( e > *c ) carry = 1;
00761         }
00762         a++; b++; c++; nd++; na--; nb--;
00763     }
00764     while ( na ) {
00765         if ( carry ) {
00766             *c = *a++ + carry;
00767             if ( *c++ ) carry = 0;
00768         }
00769         else *c++ = *a++;
00770         nd++; na--;
00771     }
00772     while ( nb ) {
00773         if ( carry ) {
00774             *c = *b++ + carry;
00775             if ( *c++ ) carry = 0;
00776         }
00777         else *c++ = *b++;
00778         nd++; nb--;
00779     }
00780     if ( carry ) {
00781         nd++;
00782         if ( nd > (UWORD)AM.MaxTal ) {
00783             MLOCK(ErrorMessageLock);
00784             MesPrint("Overflow in addition");
00785             MUNLOCK(ErrorMessageLock);
00786             return(-1);
00787         }
00788         *c++ = carry;
00789     }
00790     *nc = nd;
00791     return(0);
00792 }
00793
00794 /*
00795     #] AddPLon :
00796     #[ SubPLon :      void SubPLon(a,na,b,nb,c,nc)
00797
00798     Subtracts b from a. Assumes that a > b. Result in c.
00799     c can be a or b.
00800
00801 */
00802
00803 void SubPLon(WORD *a, WORD na, WORD *b, WORD nb, WORD *c, WORD *nc)
00804 {
00805     UWORD borrow = 0, e, nd = 0;
00806     while ( nb ) {
00807         e = *a;
00808         if ( borrow ) {
00809             *c = e - *b - borrow;
00810             if ( *c < e ) borrow = 0;
00811         }
00812         else {
00813             *c = e - *b;
00814             if ( *c > e ) borrow = 1;
00815         }
00816         a++; b++; c++; na--; nb--; nd++;
00817     }
00818     while ( na ) {
00819         if ( borrow ) {
00820             if ( *a ) { *c++ = *a++ - 1; borrow = 0; }
00821             else { *c++ = (UWORD)(-1); a++; }
00822         }
00823         else *c++ = *a++;
00824         na--; nd++;
00825     }
00826     while ( nd && !*--c ) { nd--; }
00827     *nc = (WORD)nd;
00828 }
00829
00830 /*
00831     #] SubPLon :
00832     #[ MulLong :      WORD MulLong(a,na,b,nb,c,nc)

```

```

00833
00834     Does a Long multiplication. Assumes that WORD is half the size
00835     of a LONG to work out the scheme! The number of operations is
00836     the canonical na*nb multiplications.
00837     c should not overlap with a or b.
00838
00839 */
00840
00841 WORD MullLong(WORD *a, WORD na, WORD *b, WORD nb, WORD *c, WORD *nc)
00842 {
00843     WORD sgn = 1;
00844     WORD i, *ic, *ia;
00845     RLONG t, bb;
00846     if ( !na || !nb ) { *nc = 0; return(0); }
00847     if ( na < 0 ) { na = -na; sgn = -sgn; }
00848     if ( nb < 0 ) { nb = -nb; sgn = -sgn; }
00849     *nc = i = na + nb;
00850     if ( i > (WORD)(AM.MaxTal+1) ) goto MulLov;
00851     ic = c;
00852 /*
00853     #[ GMP stuff :
00854 */
00855 #ifdef WITHGMP
00856     if (na > 3 && nb > 3) {
00857 /*         mp_limb_t res; */
00858         WORD *to, *from;
00859         int j;
00860         GETIDENTITY
00861         WORD *DLscrat9 = NumberMalloc("MulLong"), *DLscratA = NumberMalloc("MulLong"), *DLscratB =
00862         NumberMalloc("MulLong");
00863         #if ( GMPSPREAD != 1 )
00864             if ( na & 1 ) {
00865                 from = a; a = to = DLscrat9; j = na; NCOPY(to, from, j);
00866                 a[na++] = 0;
00867                 ++*nc;
00868             } else
00869             #endif
00870             if ( (LONG)a & (sizeof(mp_limb_t)-1) ) {
00871                 from = a; a = to = DLscrat9; j = na; NCOPY(to, from, j);
00872             }
00873             #if ( GMPSPREAD != 1 )
00874             if ( nb & 1 ) {
00875                 from = b; b = to = DLscratA; j = nb; NCOPY(to, from, j);
00876                 b[nb++] = 0;
00877                 ++*nc;
00878             } else
00879             #endif
00880             if ( (LONG)b & (sizeof(mp_limb_t)-1) ) {
00881                 from = b; b = to = DLscratA; j = nb; NCOPY(to, from, j);
00882             }
00883             if ( ( *nc > (WORD)i ) || ( (LONG)c & (LONG)(sizeof(mp_limb_t)-1) ) ) {
00884                 ic = DLscratB;
00885             }
00886             if ( na < nb ) {
00887                 /* res = */
00888                 mpn_mul((mp_ptr)ic, (mp_srcptr)b, nb/GMPSPREAD, (mp_srcptr)a, na/GMPSPREAD);
00889             } else {
00890                 /* res = */
00891                 mpn_mul((mp_ptr)ic, (mp_srcptr)a, na/GMPSPREAD, (mp_srcptr)b, nb/GMPSPREAD);
00892             }
00893             while ( ic[i-1] == 0 ) i--;
00894             *nc = i;
00895 /*
00896             if ( res == 0 ) *nc -= GMPSPREAD;
00897             else if ( res <= WORDMASK ) --*nc;
00898 */
00899             if ( ic != c ) {
00900                 j = *nc; NCOPY(c, ic, j);
00901             }
00902             if ( sgn < 0 ) *nc = -(*nc);
00903             NumberFree(DLscrat9, "MulLong"); NumberFree(DLscratA, "MulLong");
00904             NumberFree(DLscratB, "MulLong");
00905             return(0);
00906         }
00907     #endif
00908 /*
00909     #] GMP stuff :
00910 */
00911     do { *ic++ = 0; } while ( --i > 0 );
00912     do {
00913         ia = a;
00914         ic = c++;
00915         t = 0;
00916         i = na;
00917         bb = (RLONG) (*b++);

```

```

00918         do {
00919             t = (*ia++) * bb + t + *ic;
00920             *ic++ = (WORD)t;
00921             t >>= BITSINWORD;          /* should actually be a swap */
00922         } while ( --i > 0 );
00923         if ( t ) *ic = (UWORD)t;
00924     } while ( --nb > 0 );
00925     if ( !*ic ) (*nc)--;
00926     if ( *nc > AM.MaxTal ) goto MulLov;
00927     if ( sgn < 0 ) *nc = -(*nc);
00928     return(0);
00929 MulLov:
00930     MLOCK(ErrorMessageLock);
00931     MesPrint("Overflow in Multiplication");
00932     MUNLOCK(ErrorMessageLock);
00933     return(-1);
00934 }
00935
00936 /*
00937     #] MulLong :
00938     #[ BigLong :          WORD BigLong(a,na,b,nb)
00939
00940     Returns > 0 if a > b, < 0 if b > a and 0 if a == b
00941
00942 */
00943
00944 WORD BigLong(UWORD *a, WORD na, UWORD *b, WORD nb)
00945 {
00946     a += na;
00947     b += nb;
00948     while ( na && !*--a ) na--;
00949     while ( nb && !*--b ) nb--;
00950     if ( nb < na ) return(1);
00951     if ( nb > na ) return(-1);
00952     while ( --na >= 0 ) {
00953         if ( *a > *b ) return(1);
00954         else if ( *b > *a ) return(-1);
00955         a--; b--;
00956     }
00957     return(0);
00958 }
00959
00960 /*
00961     #] BigLong :
00962     #[ DivLong :          WORD DivLong(a,na,b,nb,c,nc,d,nd)
00963
00964     This is the long division which knows a couple of exceptions.
00965     It uses therefore a recursive call for the renormalization.
00966     The quotient comes in c and the remainder in d.
00967     d may be overlapping with b. It may also be identical to a.
00968     c should not overlap with a, but it can overlap with b.
00969
00970 */
00971
00972 WORD DivLong(UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c,
00973             WORD *nc, UWORD *d, WORD *nd)
00974 {
00975     WORD sgn = 1, sgnb = 1, ne, nf, ng, nh;
00976     WORD i, ni;
00977     UWORD *w1, *w2;
00978     RLONG t, v;
00979     UWORD *e, *f, *ff, *g, norm, estim;
00980 #ifdef WITHGMP
00981     UWORD *DLscrat9, *DLscratA, *DLscratB, *DLscratC;
00982 #endif
00983     RLONG esthelp;
00984     if ( !nb ) {
00985         MLOCK(ErrorMessageLock);
00986         MesPrint("Division by zero");
00987         MUNLOCK(ErrorMessageLock);
00988         return(-1);
00989     }
00990     if ( !na ) { *nc = *nd = 0; return(0); }
00991     if ( na < 0 ) { sgn = -sgn; na = -na; }
00992     if ( nb < 0 ) { sgnb = -sgnb; nb = -nb; }
00993     if ( na < nb ) {
00994         for ( i = 0; i < na; i++ ) *d++ = *a++;
00995         *nd = na;
00996         *nc = 0;
00997     }
00998     else if ( nb == na && ( i = BigLong(b,nb,a,na) ) >= 0 ) {
00999         if ( i > 0 ) {
01000             for ( i = 0; i < na; i++ ) *d++ = *a++;
01001             *nd = na;
01002             *nc = 0;
01003         }
01004         else {

```

```

01005         *c = 1;
01006         *nc = 1;
01007         *nd = 0;
01008     }
01009 }
01010 else if ( nb == 1 ) {
01011     if ( *b == 1 ) {
01012         for ( i = 0; i < na; i++ ) *c++ = *a++;
01013         *nc = na;
01014         *nd = 0;
01015     }
01016     else {
01017         w1 = a+na;
01018         *nc = ni = na;
01019         *nd = 1;
01020         w2 = c+ni;
01021         v = (RLONG) (*b);
01022         t = (RLONG) (*--w1);
01023         while ( --ni >= 0 ) {
01024             *--w2 = t / v;
01025             t -= v * (*w2);
01026             if ( ni ) {
01027                 t <= BITSINWORD;
01028                 t += *--w1;
01029             }
01030         }
01031         if ( ( *d = (UWORD)t ) == 0 ) *nd = 0;
01032         if ( !(c+na-1) ) (*nc)--;
01033     }
01034 }
01035 else {
01036     GETIDENTITY
01037 /*
01038     #! GMP stuff :
01039
01040     We start with copying a and b.
01041     Then we make space for c and d.
01042     Next we call mpn_tdiv_qr
01043     We adjust sizes and copy to c and d if needed.
01044     Finally the signs are settled.
01045 */
01046 #ifdef WITHGMP
01047     if ( na > 4 && nb > 3 ) {
01048         UWORD *ic, *id, *to, *from;
01049         int j = na - nb;
01050         DLscrat9 = NumberMalloc("DivLong"); DLscratA = NumberMalloc("DivLong");
01051         DLscratB = NumberMalloc("DivLong"); DLscratC = NumberMalloc("DivLong");
01052
01053 #if ( GMPSPREAD != 1 )
01054         if ( na & 1 ) {
01055             from = a; a = to = DLscrat9; i = na; NCOPY(to, from, i);
01056             a[na++] = 0;
01057         } else
01058 #endif
01059         if ( (LONG)a & (sizeof(mp_limb_t)-1) ) {
01060             from = a; a = to = DLscrat9; i = na; NCOPY(to, from, i);
01061         }
01062
01063 #if ( GMPSPREAD != 1 )
01064         if ( nb & 1 ) {
01065             from = b; b = to = DLscratA; i = nb; NCOPY(to, from, i);
01066             b[nb++] = 0;
01067         } else
01068 #endif
01069         if ( ( (LONG)b & (sizeof(mp_limb_t)-1) ) != 0 ) {
01070             from = b; b = to = DLscratA; i = nb; NCOPY(to, from, i);
01071         }
01072 #if ( GMPSPREAD != 1 )
01073 /*
01074     Not recognizing this case was a nasty and extremely rare bug.
01075     In the case that c == a and na is odd and nc == na and b
01076     still needs to be used, the least significant UWORD of b got
01077     overwritten by zero. (22-mar-2023)
01078 */
01079         ic = DLscratB; id = DLscratC;
01080 #else
01081         if ( ( (LONG)c & (sizeof(mp_limb_t)-1) ) != 0 ) ic = DLscratB;
01082         else ic = c;
01083
01084         if ( ( (LONG)d & (sizeof(mp_limb_t)-1) ) != 0 ) id = DLscratC;
01085         else id = d;
01086 #endif
01087         mpn_tdiv_qr((mp_limb_t *)ic, (mp_limb_t *)id, (mp_size_t)0,
01088             (const mp_limb_t *)a, (mp_size_t)(na/GMPSPREAD),
01089             (const mp_limb_t *)b, (mp_size_t)(nb/GMPSPREAD));
01090         while ( j >= 0 && ic[j] == 0 ) j--;
01091         j++; *nc = j;

```

```

01092         if ( c != ic ) { NCOPY(c,ic,j); }
01093         j = nb-1;
01094         while ( j >= 0 && id[j] == 0 ) j--;
01095         j++; *nd = j;
01096         if ( d != id ) { NCOPY(d,id,j); }
01097         if ( sgna < 0 ) { *nc = -(*nc); *nd = -(*nd); }
01098         if ( sgna < 0 ) { *nc = -(*nc); }
01099         NumberFree(DLscrat9,"DivLong"); NumberFree(DLscratA,"DivLong");
01100         NumberFree(DLscratB,"DivLong"); NumberFree(DLscratC,"DivLong");
01101         return(0);
01102     }
01103 #endif
01104 /*
01105     #] GMP stuff :
01106 */
01107 /* Start with normalization operation */
01108
01109 e = NumberMalloc("DivLong"); f = NumberMalloc("DivLong"); g = NumberMalloc("DivLong");
01110 if ( b[nb-1] == (FULLMAX-1) ) norm = 1;
01111 else {
01112     norm = (UWORD) (((ULONG)FULLMAX) / (ULONG) (b[nb-1]+1L));
01113 }
01114 f[na] = 0;
01115 if ( MulLong(b,nb,&norm,1,e,&ne) ||
01116     MulLong(a,na,&norm,1,f,&nf) ) {
01117     NumberFree(e,"DivLong"); NumberFree(f,"DivLong"); NumberFree(g,"DivLong");
01118     return(-1);
01119 }
01120 if ( BigLong(f+nf-ne,ne,e,ne) >= 0 ) {
01121     SubPLon(f+nf-ne,ne,e,ne,f+nf-ne,&nh);
01122     w1 = c + (nf-ne);
01123     *nc = nf-ne+1;
01124 }
01125 else {
01126     nh = ne;
01127     *nc = nf-ne;
01128     w1 = 0;
01129 }
01130 w2 = c; i = *nc; do { *w2++ = 0; } while ( --i > 0 );
01131 nf = na;
01132 ni = nf-ne;
01133 esthelp = (RLONG) (e[ne-1]) + 1L;
01134 while ( nf >= ne ) {
01135     if ( (WORD)esthelp == 0 ) {
01136         estim = (WORD) ((( (RLONG) (f[nf])) << BITSINWORD) + f[nf-1]) >> BITSINWORD;
01137     }
01138     else {
01139         estim = (WORD) ((( (RLONG) (f[nf])) << BITSINWORD) + f[nf-1]) / esthelp;
01140     }
01141     /* This estimate may be up to two too small */
01142     if ( estim ) {
01143         MulLong(e,ne,&estim,1,g,&ng);
01144         nh = ne + 1; if ( !f[ni+ne] ) nh--;
01145         SubPLon(f+ni,nh,g,ng,f+ni,&nh);
01146     }
01147     else {
01148         w2 = f+ni+ne; nh = ne+1;
01149         while ( ( nh > 0 ) && !*w2 ) { nh--; w2--; }
01150     }
01151     if ( BigLong(f+ni,nh,e,ne) >= 0 ) {
01152         estim++;
01153         SubPLon(f+ni,nh,e,ne,f+ni,&nh);
01154         if ( BigLong(f+ni,nh,e,ne) >= 0 ) {
01155             estim++;
01156             SubPLon(f+ni,nh,e,ne,f+ni,&nh);
01157             if ( BigLong(f+ni,nh,e,ne) >= 0 ) {
01158                 MLOCK(ErrorMessageLock);
01159                 MesPrint("Problems in DivLong");
01160                 AO.OutSkip = 3;
01161                 FiniLine();
01162                 i = na;
01163                 while ( --i >= 0 ) { TalToLine((UWORD) (*a++)); TokenToLine((UBYTE *) " "); }
01164                 FiniLine();
01165                 i = nb;
01166                 while ( --i >= 0 ) { TalToLine((UWORD) (*b++)); TokenToLine((UBYTE *) " "); }
01167                 AO.OutSkip = 0;
01168                 FiniLine();
01169                 MUNLOCK(ErrorMessageLock);
01170                 NumberFree(e,"DivLong"); NumberFree(f,"DivLong"); NumberFree(g,"DivLong");
01171                 return(-1);
01172             }
01173         }
01174     }
01175     c[ni] = estim;
01176     nf--;
01177     ni--;
01178 }

```

```

01179         if ( w1 ) *w1 = 1;
01180
01181         /* Finish with the renormalization operation */
01182
01183         if ( nh > 0 ) {
01184             if ( norm == 1 ) {
01185                 *nd = i = nh; ff = f;
01186                 NCOPY(d,ff,i);
01187             }
01188             else {
01189                 w1 = f+nh;
01190                 *nd = ni = nh;
01191                 w2 = d+ni;
01192                 v = norm;
01193                 t = (RLONG) (*--w1);
01194                 while ( --ni >= 0 ) {
01195                     *--w2 = t / v;
01196                     t -= v * (*w2);
01197                     if ( ni ) {
01198                         t <= BITSINWORD;
01199                         t += *--w1;
01200                     }
01201                 }
01202                 if ( t ) {
01203                     MLOCK(ErrorMessageLock);
01204                     MesPrint("Error in DivLong");
01205                     MUNLOCK(ErrorMessageLock);
01206                     NumberFree(e,"DivLong"); NumberFree(f,"DivLong"); NumberFree(g,"DivLong");
01207                     return(-1);
01208                 }
01209                 if ( !(d+nh-1) ) (*nd)--;
01210             }
01211         }
01212         else { *nd = 0; }
01213         NumberFree(e,"DivLong"); NumberFree(f,"DivLong"); NumberFree(g,"DivLong");
01214     }
01215     if ( sgna < 0 ) { *nc = -(*nc); *nd = -(*nd); }
01216     if ( sgnb < 0 ) { *nc = -(*nc); }
01217     return(0);
01218 }
01219
01220 /*
01221     #] DivLong :
01222     #[ RaisPow :      WORD RaisPow(a,na,b)
01223
01224     Raises a to the power b. a is a Long integer and b >= 0.
01225     The method that is used works with a bitdecomposition of b.
01226 */
01227
01228 WORD RaisPow(PHEAD UWORD *a, WORD *na, UWORD b)
01229 {
01230     GETBIDENTITY
01231     WORD i, nu;
01232     UWORD *it, *iu, c;
01233     UWORD *is, *iss;
01234     WORD ns, nt, nmod;
01235     nmod = ABS(AN.ncmod);
01236     if ( !*na || ( ( *na == 1 ) && ( *a == 1 ) ) ) return(0);
01237     if ( !b ) { *na=1; *a=1; return(0); }
01238     is = NumberMalloc("RaisPow");
01239     it = NumberMalloc("RaisPow");
01240     for ( i = 0; i < ABS(*na); i++ ) is[i] = a[i];
01241     ns = *na;
01242     c = b;
01243     for ( i = 0; i < BITSINWORD; i++ ) {
01244         if ( !c ) break;
01245         c /= 2;
01246     }
01247     i--;
01248     c = 1 << i;
01249     while ( --i >= 0 ) {
01250         c /= 2;
01251         if(MulLong(is,ns,is,ns,it,&nt)) goto RaisOvl;
01252         if ( b & c ) {
01253             if ( MulLong(it,nt,a,*na,is,&ns) ) goto RaisOvl;
01254         }
01255         else {
01256             iu = is; is = it; it = iu;
01257             nu = ns; ns = nt; nt = nu;
01258         }
01259         if ( nmod != 0 ) {
01260             if ( DivLong(is,ns,(UWORD *)AC.cmod,nmod,it,&nt,is,&ns) ) goto RaisOvl;
01261         }
01262     }
01263     if ( ( nmod != 0 ) && ( ( AC.modmode & POSNEG ) != 0 ) ) {
01264         NormalModulus(is,&ns);
01265     }

```

```

01266     if ( ( *na = i = ns ) != 0 ) { iss = is; i=ABS(i); NCOPY(a,iss,i); }
01267     NumberFree(is,"RaisPow"); NumberFree(it,"RaisPow");
01268     return(0);
01269 RaisOvl:
01270     MLOCK(ErrorMessageLock);
01271     MesCall("RaisPow");
01272     MUNLOCK(ErrorMessageLock);
01273     NumberFree(is,"RaisPow"); NumberFree(it,"RaisPow");
01274     SETERROR(-1)
01275 }
01276
01277 /*
01278     #] RaisPow :
01279     #[ RaisPowCached :
01280 */
01281
01296 void RaisPowCached (PHEAD WORD x, WORD n, UWORD **c, WORD *nc) {
01297
01298     int i,j;
01299     WORD new_small_power_maxx, new_small_power_maxn, ID;
01300     WORD *new_small_power_n;
01301     UWORD **new_small_power;
01302
01303     /* check whether to extend the array */
01304     if (x>=AT.small_power_maxx || n>=AT.small_power_maxn) {
01305
01306         new_small_power_maxx = AT.small_power_maxx;
01307         if (x>=AT.small_power_maxx)
01308             new_small_power_maxx = MaX(2*AT.small_power_maxx, x+1);
01309
01310         new_small_power_maxn = AT.small_power_maxn;
01311         if (n>=AT.small_power_maxn)
01312             new_small_power_maxn = MaX(2*AT.small_power_maxn, n+1);
01313
01314         new_small_power_n = (WORD*)
01315         Malloc1(new_small_power_maxx*new_small_power_maxn*sizeof(WORD),"RaisPowCached");
01316         new_small_power = (UWORD **) Malloc1(new_small_power_maxx*new_small_power_maxn*sizeof(UWORD
01317         *),"RaisPowCached");
01318
01319         for (i=0; i<new_small_power_maxx * new_small_power_maxn; i++) {
01320             new_small_power_n[i] = 0;
01321             new_small_power [i] = NULL;
01322         }
01323
01324         for (i=0; i<AT.small_power_maxx; i++)
01325             for (j=0; j<AT.small_power_maxn; j++) {
01326                 new_small_power_n[i*new_small_power_maxn+j] =
01327                 AT.small_power_n[i*AT.small_power_maxn+j];
01328                 new_small_power [i*new_small_power_maxn+j] = AT.small_power
01329                 [i*AT.small_power_maxn+j];
01330             }
01331
01332         if (AT.small_power_n != NULL) {
01333             M_free(AT.small_power_n,"RaisPowCached");
01334             M_free(AT.small_power,"RaisPowCached");
01335         }
01336
01337         AT.small_power_maxx = new_small_power_maxx;
01338         AT.small_power_maxn = new_small_power_maxn;
01339         AT.small_power_n = new_small_power_n;
01340         AT.small_power = new_small_power;
01341     }
01342
01343     /* check whether the results is already calculated */
01344     ID = x * AT.small_power_maxn + n;
01345
01346     if (AT.small_power[ID] == NULL) {
01347         #ifdef OLDRAISPOWCACHED
01348             AT.small_power[ID] = NumberMalloc("RaisPowCached");
01349             AT.small_power_n[ID] = 1;
01350             AT.small_power[ID][0] = x;
01351             RaisPow(BHEAD AT.small_power[ID], &AT.small_power_n[ID], n);
01352         #else
01353             UWORD *c = NumberMalloc("RaisPowCached");
01354             WORD i, k = 1;
01355             c[0] = x;
01356             RaisPow(BHEAD c, &k, n);
01357
01358             /* And now get the proper amount. */
01359
01360             if ( AT.InNumMem < k ) { /* We should start a new buffer */
01361                 AT.InNumMem = 5*AM.MaxTal;
01362                 AT.NumMem = (UWORD *) Malloc1(AT.InNumMem*sizeof(UWORD),"RaisPowCached");
01363                 MesPrint(" Got an extra %l UWORDS in RaisPowCached",AT.InNumMem);
01364             }
01365         }
01366     }

```

```

01363         for ( i = 0; i < k; i++ ) AT.NumMem[i] = c[i];
01364         AT.small_power[ID] = AT.NumMem;
01365         AT.small_power_n[ID] = k;
01366         AT.NumMem += k;
01367         AT.InNumMem -= k;
01368         NumberFree(c,"RaisPowCached");
01369 #endif
01370     }
01371
01372     /* return the result */
01373     *c = AT.small_power[ID];
01374     *nc = AT.small_power_n[ID];
01375 }
01376
01377 /*
01378     #] RaisPowCached :
01379     #[ RaisPowMod :
01380
01381     Computes the power x^n mod m
01382 */
01383 WORD RaisPowMod (WORD x, WORD n, WORD m) {
01384     LONG y=1, z=x;
01385     while (n) {
01386         if (n&1) { y*=z; y%=m; }
01387         z*=z; z%=m;
01388         n /= 2;
01389     }
01390     return (WORD)y;
01391 }
01392
01393 /*
01394     #] RaisPowMod :
01395     #[ NormalModulus : int NormalModulus (UWORD *a,WORD *na)
01396 */
01403 int NormalModulus (UWORD *a,WORD *na)
01404 {
01405     WORD n;
01406     if ( AC.halfmod == 0 ) {
01407         LOCK(AC.halfmodlock);
01408         if ( AC.halfmod == 0 ) {
01409             UWORD two[1],remain[1];
01410             WORD dummy;
01411             two[0] = 2;
01412             AC.halfmod = (UWORD *)Malloc1((ABS(AC.ncmod))*sizeof(UWORD),"halfmod");
01413             DivLong((UWORD *)AC.cmod,(ABS(AC.ncmod)),two,1
01414                 ,(UWORD *)AC.halfmod,&(AC.nhalfmod),remain,&dummy);
01415         }
01416         UNLOCK(AC.halfmodlock);
01417     }
01418     n = ABS(*na);
01419     if ( BigLong(a,n,AC.halfmod,AC.nhalfmod) > 0 ) {
01420         SubPLon((UWORD *)AC.cmod,(ABS(AC.ncmod)),a,n,a,&n);
01421         if ( *na > 0 ) { *na = -n; }
01422         else { *na = n; }
01423         return(1);
01424     }
01425     return(0);
01426 }
01427
01428 /*
01429     #] NormalModulus :
01430     #[ MakeInverses :
01431 */
01440 int MakeInverses(void)
01441 {
01442     WORD n = AC.cmod[0], i, inv2;
01443     if ( AC.ncmod != 1 ) return(1);
01444     if ( AC.modinverses == 0 ) {
01445         LOCK(AC.halfmodlock);
01446         if ( AC.modinverses == 0 ) {
01447             AC.modinverses = (UWORD *)Malloc1(n*sizeof(UWORD),"modinverses");
01448             AC.modinverses[0] = 0;
01449             AC.modinverses[1] = 1;
01450             for ( i = 2; i < n; i++ ) {
01451                 if ( GetModInverses(i,n,
01452                     (WORD *)(&(AC.modinverses[i])),&inv2) ) {
01453                     SETERROR(-1)
01454                 }
01455             }
01456         }
01457         UNLOCK(AC.halfmodlock);
01458     }
01459     return(0);
01460 }
01461
01462 /*
01463     #] MakeInverses :

```



```

01464         #] GetModInverses :
01465     */
01476 int GetModInverses(WORD m1, WORD m2, WORD *im1, WORD *im2)
01477 {
01478     WORD a1, a2, a3;
01479     WORD b1, b2, b3;
01480     WORD x = m1, y, c, d = m2;
01481     if ( x < 1 || d <= 1 ) goto somethingwrong;
01482     a1 = 0; a2 = 1;
01483     b1 = 1; b2 = 0;
01484     for(;;) {
01485         c = d/x; y = d%x; /* a good compiler makes this faster than y=d-c*x */
01486         if ( y == 0 ) break;
01487         a3 = a1-c*a2; a1 = a2; a2 = a3;
01488         b3 = b1-c*b2; b1 = b2; b2 = b3;
01489         d = x; x = y;
01490     }
01491     if ( x != 1 ) goto somethingwrong;
01492     if ( a2 < 0 ) a2 += m2;
01493     if ( b2 < 0 ) b2 += m1;
01494     if (im1!=NULL) *im1 = a2;
01495     if (im2!=NULL) *im2 = b2;
01496     return(0);
01497 somethingwrong:
01498     MLOCK(ErrorMessageLock);
01499     MesPrint("Error trying to determine inverses in GetModInverses");
01500     MUNLOCK(ErrorMessageLock);
01501     return(-1);
01502 }
01503 /*
01504     #] GetModInverses :
01505     #] GetLongModInverses :
01506 */
01507
01508 int GetLongModInverses(PHEAD UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *ia, WORD *nia, UWORD *ib,
01509     WORD *nib) {
01510     UWORD *s, *t, *sa, *sb, *ta, *tb, *x, *y, *swap1;
01511     WORD ns, nt, nsa, nsb, nta, ntb, nx, ny, swap2;
01512
01513     s = NumberMalloc("GetLongModInverses");
01514     ns = na;
01515     WCOPY(s, a, ABS(ns));
01516
01517     t = NumberMalloc("GetLongModInverses");
01518     nt = nb;
01519     WCOPY(t, b, ABS(nt));
01520
01521     sa = NumberMalloc("GetLongModInverses");
01522     nsa = 1;
01523     sa[0] = 1;
01524
01525     sb = NumberMalloc("GetLongModInverses");
01526     nsb = 0;
01527
01528     ta = NumberMalloc("GetLongModInverses");
01529     nta = 0;
01530
01531     tb = NumberMalloc("GetLongModInverses");
01532     ntb = 1;
01533     tb[0] = 1;
01534
01535     x = NumberMalloc("GetLongModInverses");
01536     y = NumberMalloc("GetLongModInverses");
01537
01538     while (nt != 0) {
01539         DivLong(s, ns, t, nt, x, &nx, y, &ny);
01540         swap1=s; s=y; y=swap1;
01541         ns=ny;
01542         MullLong(x, nx, ta, nta, y, &ny);
01543         AddLong(sa, nsa, y, -ny, sa, &nsa);
01544         MullLong(x, nx, tb, ntb, y, &ny);
01545         AddLong(sb, nsb, y, -ny, sb, &nsb);
01546
01547         swap1=s; s=t; t=swap1;
01548         swap2=ns; ns=nt; nt=swap2;
01549         swap1=sa; sa=ta; ta=swap1;
01550         swap2=nsa; nsa=nta; nta=swap2;
01551         swap1=sb; sb=tb; tb=swap1;
01552         swap2=nsb; nsb=ntb; ntb=swap2;
01553     }
01554
01555     if (ia!=NULL) {
01556         *nia = nsa*ns;
01557         WCOPY(ia, sa, ABS(*nia));
01558     }
01559 }

```

```

01560     if (ib!=NULL) {
01561         *nib = nsb*ns;
01562         WCOPY(ib,sb,ABS(*nib));
01563     }
01564
01565     NumberFree(s,"GetLongModInverses");
01566     NumberFree(t,"GetLongModInverses");
01567     NumberFree(sa,"GetLongModInverses");
01568     NumberFree(sb,"GetLongModInverses");
01569     NumberFree(ta,"GetLongModInverses");
01570     NumberFree(tb,"GetLongModInverses");
01571     NumberFree(x,"GetLongModInverses");
01572     NumberFree(y,"GetLongModInverses");
01573
01574     return 0;
01575 }
01576
01577 /*
01578     #] GetLongModInverses :
01579     #[ Product :          WORD Product(a,na,b)
01580
01581     Multiplies the Long number in a with the WORD b.
01582 */
01583
01584 WORD Product(WORD *a, WORD *na, WORD b)
01585 {
01586     WORD i, sgn = 1;
01587     RLONG t, u;
01588     if ( *na < 0 ) { *na = -(*na); sgn = -sgn; }
01589     if ( b < 0 ) { b = -b; sgn = -sgn; }
01590     t = 0;
01591     u = (RLONG)b;
01592     for ( i = 0; i < *na; i++ ) {
01593         t += *a * u;
01594         *a++ = (UWORD)t;
01595         t >>= BITSINWORD;
01596     }
01597     if ( t > 0 ) {
01598         if ( ++(*na) > AM.MaxTal ) {
01599             MLOCK(ErrorMessageLock);
01600             MesPrint("Overflow in Product");
01601             MUNLOCK(ErrorMessageLock);
01602             return(-1);
01603         }
01604         *a = (UWORD)t;
01605     }
01606     if ( sgn < 0 ) *na = -(*na);
01607     return(0);
01608 }
01609
01610 /*
01611     #] Product :
01612     #[ Quotient :          UWORD Quotient(a,na,b)
01613
01614     Routine divides the long number a by b with the assumption that
01615     there is no remainder (like while computing binomials).
01616 */
01617
01618 UWORD Quotient(WORD *a, WORD *na, WORD b)
01619 {
01620     RLONG v, t;
01621     WORD i, j, sgn = 1;
01622     if ( ( i = *na ) < 0 ) { sgn = -1; i = -i; }
01623     if ( b < 0 ) { b = -b; sgn = -sgn; }
01624     if ( i == 1 ) {
01625         if ( ( *a /= (UWORD)b ) == 0 ) *na = 0;
01626         if ( sgn < 0 ) *na = -*na;
01627         return(0);
01628     }
01629     a += i;
01630     j = i;
01631     v = (RLONG)b;
01632     t = (RLONG)(--a);
01633     while ( --i >= 0 ) {
01634         *a = t / v;
01635         t -= v * (*a);
01636         if ( i ) {
01637             t <<= BITSINWORD;
01638             t += *--a;
01639         }
01640     }
01641     a += j - 1;
01642     if ( !*a ) j--;
01643     if ( sgn < 0 ) j = -j;
01644     *na = j;

```

```

01647     return(0);
01648 }
01649
01650 /*
01651     #] Quotient :
01652     #[ Remain10 :      WORD Remain10(a,na)
01653
01654     Routine divides a by 10 and gives the remainder as return value.
01655     The value of a will be the quotient! a must be positive.
01656 */
01657
01658
01659 WORD Remain10(UWORD *a, WORD *na)
01660 {
01661     WORD i;
01662     RLONG t, u;
01663     UWORD *b;
01664     i = *na;
01665     t = 0;
01666     b = a + i - 1;
01667     while ( --i >= 0 ) {
01668         t += *b;
01669         *b-- = u = t / 10;
01670         t -= u * 10;
01671         if ( i > 0 ) t <= BITSINWORD;
01672     }
01673     if ( ( *na > 0 ) && !a[*na-1] ) (*na)--;
01674     return((WORD)t);
01675 }
01676
01677 /*
01678     #] Remain10 :
01679     #[ Remain4 :      WORD Remain4(a,na)
01680
01681     Routine divides a by 10000 and gives the remainder as return value.
01682     The value of a will be the quotient! a must be positive.
01683 */
01684
01685
01686 WORD Remain4(UWORD *a, WORD *na)
01687 {
01688     WORD i;
01689     RLONG t, u;
01690     UWORD *b;
01691     i = *na;
01692     t = 0;
01693     b = a + i - 1;
01694     while ( --i >= 0 ) {
01695         t += *b;
01696         *b-- = u = t / 10000;
01697         t -= u * 10000;
01698         if ( i > 0 ) t <= BITSINWORD;
01699     }
01700     if ( ( *na > 0 ) && !a[*na-1] ) (*na)--;
01701     return((WORD)t);
01702 }
01703
01704 /*
01705     #] Remain4 :
01706     #[ PrtLong :      void PrtLong(a,na,s)
01707
01708     Puts the long number a in string s.
01709 */
01710
01711
01712 void PrtLong(UWORD *a, WORD na, UBYTE *s)
01713 {
01714     GETIDENTITY
01715     WORD q, i;
01716     UBYTE *sa, *sb;
01717     UBYTE c;
01718     UWORD *bb, *b;
01719
01720     if ( na < 0 ) {
01721         *s++ = '-';
01722         na = -na;
01723     }
01724
01725     b = NumberMalloc("PrtLong");
01726     bb = b;
01727     i = na; while ( --i >= 0 ) *bb++ = *a++;
01728     a = b;
01729     if ( na > 2 ) {
01730         sa = s;
01731         do {
01732             q = Remain4(a,&na);
01733             *sa++ = (UBYTE)('0' + (q%10));

```

```

01734         q /= 10;
01735         *sa++ = (UBYTE) ('0' + (q%10));
01736         q /= 10;
01737         *sa++ = (UBYTE) ('0' + (q%10));
01738         q /= 10;
01739         *sa++ = (UBYTE) ('0' + (q%10));
01740     } while ( na );
01741     while ( sa[-1] == '0' ) sa--;
01742     sb = s;
01743     s = sa;
01744     sa--;
01745     while ( sa > sb ) { c = *sa; *sa = *sb; *sb = c; sa--; sb++; }
01746 }
01747 else if ( na ) {
01748     sa = s;
01749     do {
01750         q = Remain10(a,&na);
01751         *sa++ = (UBYTE) ('0' + q);
01752     } while ( na );
01753     sb = s;
01754     s = sa;
01755     sa--;
01756     while ( sa > sb ) { c = *sa; *sa = *sb; *sb = c; sa--; sb++; }
01757 }
01758 else *s++ = '0';
01759 *s = '\0';
01760 NumberFree(b,"PrtLong");
01761 }
01762
01763 /*
01764     #] PrtLong :
01765     #[ GetLong :      WORD GetLong(s,a,na)
01766
01767     Reads a long number from a string.
01768     The string is zero terminated and contains only digits!
01769
01770     New algorithm: try to read 4 digits together before the result
01771     is accumulated.
01772 */
01773
01774 WORD GetLong(UBYTE *s, UWORD *a, WORD *na)
01775 {
01776     /*
01777         UWORD digit;
01778         *a = 0;
01779         *na = 0;
01780         while ( FG.cTable[*s] == 1 ) {
01781             digit = *s++ - '0';
01782             if ( *na && Product(a,na,(WORD)10) ) return(-1);
01783             if ( digit && AddLong(a,*na,&digit,(WORD)1,a,na) ) return(-1);
01784         }
01785         return(0);
01786     */
01787     UWORD digit, x = 0, y = 0;
01788     *a = 0;
01789     *na = 0;
01790     while ( FG.cTable[*s] == 1 ) {
01791         x = *s++ - '0';
01792         if ( FG.cTable[*s] != 1 ) { y = 10; break; }
01793         x = 10*x + *s++ - '0';
01794         if ( FG.cTable[*s] != 1 ) { y = 100; break; }
01795         x = 10*x + *s++ - '0';
01796         if ( FG.cTable[*s] != 1 ) { y = 1000; break; }
01797         x = 10*x + *s++ - '0';
01798         if ( *na && Product(a,na,(WORD)10000) ) return(-1);
01799         if ( ( digit = x ) != 0 && AddLong(a,*na,&digit,(WORD)1,a,na) )
01800             return(-1);
01801         y = 0;
01802     }
01803     if ( y ) {
01804         if ( *na && Product(a,na,(WORD)y) ) return(-1);
01805         if ( ( digit = x ) != 0 && AddLong(a,*na,&digit,(WORD)1,a,na) )
01806             return(-1);
01807     }
01808     return(0);
01809 }
01810
01811 /*
01812     #] GetLong :
01813     #[ GCD :      WORD GCD(a,na,b,nb,c,nc)
01814
01815     Algorithm to compute the GCD of two long numbers.
01816     See Knuth, sec 4.5.2 algorithm L.
01817
01818     We assume that both numbers are positive
01819
01820     NOTE!!!!. NumberMalloc gets called and it may not be freed

```

```

01821 */
01822
01823 #ifdef EXTRAGCD
01824
01825 #define Convert(ia,aa,naa) \
01826     if ( (LONG)ia < 0 ) { \
01827         ia = (ULONG)(~(LONG)ia); \
01828         aa[0] = ia; \
01829         if ( ( aa[1] = ia » BITSINWORD ) != 0 ) naa = -2; \
01830         else naa = -1; \
01831     } \
01832     else if ( ia == 0 ) { aa[0] = 0; naa = 0; } \
01833     else { \
01834         aa[0] = ia; \
01835         if ( ( aa[1] = ia » BITSINWORD ) != 0 ) naa = 2; \
01836         else naa = 1; \
01837     }
01838
01839 void GCD(UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc)
01840 {
01841     int ja = 0, jb = 0, j;
01842     UWORD *r,*t;
01843     UWORD *x1, *x2, *x3;
01844     WORD nd,naa,nbb;
01845     ULONG ia,ib,ic,id,u,v,w,q,T;
01846     UWORD aa[2], bb[2];
01847 /*
01848     First eliminate easy powers of 2^...
01849 */
01850     while ( a[0] == 0 ) { na--; ja++; a++; }
01851     while ( b[0] == 0 ) { nb--; jb++; b++; }
01852     if ( ja > jb ) ja = jb;
01853     if ( ja > 0 ) {
01854         j = ja;
01855         do { *c++ = 0; } while ( --j > 0 );
01856     }
01857 /*
01858     Now arrange things such that a >= b
01859 */
01860     if ( na < nb ) {
01861         jb = na; na = nb; nb = jb;
01862     }
01863     r = a; a = b; b = r;
01864 }
01865     else if ( na == nb ) {
01866         r = a+na;
01867         t = b+nb;
01868         j = na;
01869         while ( --j >= 0 ) {
01870             if ( *--r > *--t ) break;
01871             if ( *r < *t ) goto exch;
01872         }
01873         if ( j < 0 ) {
01874     out:
01875             j = nb;
01876             NCOPY(c,b,j);
01877             *nc = nb+ja;
01878             return;
01879         }
01880     }
01881 /*
01882     {
01883         MLOCK(ErrorMessageLock);
01884         MesPrint("Ordered input, ja = %d", (WORD)ja);
01885         AO.OutSkip = 3;
01886         FiniLine();
01887         j = na; r = a;
01888         while ( --j >= 0 ) { TalToLine((UWORD)(*r++)); TokenToLine((UBYTE *)" "); }
01889         FiniLine();
01890         j = nb; r = b;
01891         while ( --j >= 0 ) { TalToLine((UWORD)(*r++)); TokenToLine((UBYTE *)" "); }
01892         AO.OutSkip = 0;
01893         FiniLine();
01894         MUNLOCK(ErrorMessageLock);
01895     }
01896 */
01897 /*
01898     We have now that A > B
01899     The loop recognizes the case that na-nb >= 1
01900     In that case we just have to divide!
01901 */
01902     r = x1 = NumberMalloc("GCD"); t = x2 = NumberMalloc("GCD"); x3 = NumberMalloc("GCD");
01903     j = na;
01904     NCOPY(r,a,j);
01905     j = nb;
01906     NCOPY(t,b,j);
01907

```

```

01908     for(;;) {
01909
01910         while ( na > nb ) {
01911 toobad:
01912             DivLong(x1,na,x2,nb,c,nc,x3,&nd);
01913             if ( nd == 0 ) { b = x2; goto out; }
01914             t = x1; x1 = x2; x2 = x3; x3 = t; na = nb; nb = nd;
01915             if ( na == 2 ) break;
01916         }
01917 /*
01918     Here we can use the shortcut.
01919 */
01920     if ( na == 2 ) {
01921         v = x1[0] + ( ((ULONG)x1[1]) « BITSINWORD );
01922         w = x2[0];
01923         if ( nb == 2 ) w += ((ULONG)x2[1]) « BITSINWORD;
01924 #ifdef EXTRAGCD2
01925         v = GCD2(v,w);
01926 #else
01927         do { u = v%w; v = w; w = u; } while ( w );
01928 #endif
01929         c[0] = (UWORD)v;
01930         if ( ( c[1] = (UWORD)(v » BITSINWORD) ) != 0 ) *nc = 2+ja;
01931         else *nc = 1+ja;
01932         NumberFree(x1,"GCD"); NumberFree(x2,"GCD"); NumberFree(x3,"GCD");
01933         return;
01934     }
01935     if ( na == 1 ) {
01936         UWORD ui, uj;
01937         ui = x1[0]; uj = x2[0];
01938 #ifdef EXTRAGCD2
01939         ui = (UWORD)GCD2((ULONG)ui, (ULONG)uj);
01940 #else
01941         do { nd = ui%uj; ui = uj; uj = nd; } while ( nd );
01942 #endif
01943         c[0] = ui;
01944         *nc = 1 + ja;
01945         NumberFree(x1,"GCD"); NumberFree(x2,"GCD"); NumberFree(x3,"GCD");
01946         return;
01947     }
01948     ia = 1; ib = 0; ic = 0; id = 1;
01949     u = ( ((ULONG)x1[na-1]) « BITSINWORD ) + x1[na-2];
01950     v = ( ((ULONG)x2[nb-1]) « BITSINWORD ) + x2[nb-2];
01951
01952     while ( v+ic != 0 && v+id != 0 &&
01953           ( q = (u+ia)/(v+ic) ) == (u+ib)/(v+id) ) {
01954         T = ia-q*ic; ia = ic; ic = T;
01955         T = ib-q*id; ib = id; id = T;
01956         T = u - q*v; u = v; v = T;
01957     }
01958     if ( ib == 0 ) goto toobad;
01959     Convert(ia,aa,naa);
01960     Convert(ib,bb,nbb);
01961     MulLong(x1,na,aa,naa,x3,&nd);
01962     MulLong(x2,nb,bb,nbb,c,nc);
01963     AddLong(x3,nd,c,*nc,c,nc);
01964     Convert(ic,aa,naa);
01965     Convert(id,bb,nbb);
01966     MulLong(x1,na,aa,naa,x3,&nd);
01967     t = c; na = j = *nc; r = x1;
01968     NCOPY(r,t,j);
01969     MulLong(x2,nb,bb,nbb,c,nc);
01970     AddLong(x3,nd,c,*nc,x2,&nb);
01971 }
01972 }
01973 #endif
01974
01975 /*
01976 #] GCD :
01977 #[ GcdLong :          WORD GcdLong(a,na,b,nb,c,nc)
01978
01979 Returns the Greatest Common Divider of a and b in c.
01980 If a and or b are zero an error message will be returned.
01981 The answer is always positive.
01982 In principle a and c can be the same.
01983 */
01984 #ifndef NEWTRICK
01985 #ifdef NEWTRICK
01986 #] Old Routine :
01987 #[
01988 */
01989
01990 WORD GcdLong(PHEAD UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc)
01991 {
01992     GETBIDENTITY
01993     if ( !na || !nb ) {

```

```

01995         if ( !na && !nb ) {
01996             MLOCK(ErrorMessageLock);
01997             MesPrint("Cannot take gcd");
01998             MUNLOCK(ErrorMessageLock);
01999             return(-1);
02000         }
02001
02002         if ( !na ) {
02003             *nc = abs(nb);
02004             NCOPY(c,b,*nc);
02005             *nc = abs(nb);
02006             return(0);
02007         }
02008
02009         *nc = abs(na);
02010         NCOPY(c,a,*nc);
02011         *nc = abs(na);
02012         return(0);
02013     }
02014     if ( na < 0 ) na = -na;
02015     if ( nb < 0 ) nb = -nb;
02016     if ( na == 1 && nb == 1 ) {
02017 #ifdef EXTRAGCD2
02018         *c = (UWORD)GCD2((ULONG)*a,(ULONG)*b);
02019 #else
02020         UWORD x,y,z;
02021         x = *a;
02022         y = *b;
02023         do { z = x % y; x = y; } while ( ( y = z ) != 0 );
02024         *c = x;
02025 #endif
02026         *nc = 1;
02027     }
02028     else if ( na <= 2 && nb <= 2 ) {
02029         RLONG lx,ly,lz;
02030         if ( na == 2 ) { lx = ((RLONG)(a[1]))«BITSINWORD + *a; }
02031         else { lx = *a; }
02032         if ( nb == 2 ) { ly = ((RLONG)(b[1]))«BITSINWORD + *b; }
02033         else { ly = *b; }
02034 #ifdef LONGWORD
02035 #ifdef EXTRAGCD2
02036         lx = GCD2(lx,ly);
02037 #else
02038         do { lz = lx % ly; lx = ly; } while ( ( ly = lz ) != 0 );
02039 #endif
02040 #else
02041         if ( lx < ly ) { lz = lx; lx = ly; ly = lz; }
02042         do {
02043             lz = lx % ly; lx = ly;
02044         } while ( ( ly = lz ) != 0 && ( lx & AWORDMASK ) != 0 );
02045         if ( ly ) {
02046             do { *c = ((UWORD)lx)%((UWORD)ly); lx = ly; } while ( ( ly = *c ) != 0 );
02047             *c = (UWORD)lx;
02048             *nc = 1;
02049         }
02050         else
02051 #endif
02052     {
02053         *c++ = (UWORD)lx;
02054         if ( ( *c = (UWORD)(lx » BITSINWORD) ) != 0 ) *nc = 2;
02055         else *nc = 1;
02056     }
02057 }
02058 else {
02059 #ifdef EXTRAGCD
02060     GCD(a,na,b,nb,c,nc);
02061 #else
02062 #ifdef NEWGCD
02063     UWORD *x3,*x1,*x2, *GLscrat7, *GLscrat8;
02064     WORD n1,n2,n3,n4;
02065     WORD i, j;
02066     x1 = c; x3 = a; n1 = i = na;
02067     NCOPY(x1,x3,i);
02068     GLscrat7 = NumberMalloc("GcdLong"); GLscrat8 = NumberMalloc("GcdLong");
02069     x2 = GLscrat8; x3 = b; n2 = i = nb;
02070     NCOPY(x2,x3,i);
02071     x1 = c; i = 0;
02072     while ( x1[0] == 0 ) { i += BITSINWORD; x1++; n1--; }
02073     while ( ( x1[0] & 1 ) == 0 ) { i++; SCHUIF(x1,n1) }
02074     x2 = GLscrat8; j = 0;
02075     while ( x2[0] == 0 ) { j += BITSINWORD; x2++; n2--; }
02076     while ( ( x2[0] & 1 ) == 0 ) { j++; SCHUIF(x2,n2) }
02077     if ( j > i ) j = i; /* powers of two in GCD */
02078     for(;;){
02079         if ( n1 > n2 ) {
02080 firstbig:
02081             SubPLon(x1,n1,x2,n2,x1,&n3);

```

```

02082         n1 = n3;
02083         if ( n1 == 0 ) {
02084             x1 = c;
02085             n1 = i = n2; NCOPY(x1,x2,i);
02086             break;
02087         }
02088         while ( ( x1[0] & 1 ) == 0 ) SCHUIF(x1,n1)
02089         if ( n1 == 1 ) {
02090             if ( DivLong(x2,n2,x1,n1,GLscrat7,&n3,x2,&n4) ) goto GcdErr;
02091             n2 = n4;
02092             if ( n2 == 0 ) {
02093                 i = n1; x2 = c; NCOPY(x2,x1,i);
02094                 break;
02095             }
02096 #ifdef EXTRAGCD2
02097             *c = (UWORD)GCD2( (ULONG)x1[0], (ULONG)x2[0] );
02098 #else
02099             {
02100                 UWORD x,y,z;
02101                 x = x1[0];
02102                 y = x2[0];
02103                 do { z = x % y; x = y; } while ( ( y = z ) != 0 );
02104                 *c = x;
02105             }
02106 #endif
02107             n1 = 1;
02108             break;
02109         }
02110     }
02111     else if ( n1 < n2 ) {
02112 lastbig:
02113         SubPLon(x2,n2,x1,n1,x2,&n3);
02114         n2 = n3;
02115         if ( n2 == 0 ) {
02116             i = n1; x2 = c; NCOPY(x2,x1,i);
02117             break;
02118         }
02119         while ( ( x2[0] & 1 ) == 0 ) SCHUIF(x2,n2)
02120         if ( n2 == 1 ) {
02121             if ( DivLong(x1,n1,x2,n2,GLscrat7,&n3,x1,&n4) ) goto GcdErr;
02122             n1 = n4;
02123             if ( n1 == 0 ) {
02124                 x1 = c;
02125                 n1 = i = n2; NCOPY(x1,x2,i);
02126                 break;
02127             }
02128 #ifdef EXTRAGCD2
02129             *c = (UWORD)GCD2( (ULONG)x2[0], (ULONG)x1[0] );
02130 #else
02131             {
02132                 UWORD x,y,z;
02133                 x = x2[0];
02134                 y = x1[0];
02135                 do { z = x % y; x = y; } while ( ( y = z ) != 0 );
02136                 *c = x;
02137             }
02138 #endif
02139             n1 = 1;
02140             break;
02141         }
02142     }
02143     else {
02144         for ( i = n1-1; i >= 0; i-- ) {
02145             if ( x1[i] > x2[i] ) goto firstbig;
02146             else if ( x1[i] < x2[i] ) goto lastbig;
02147         }
02148         i = n1; x2 = c; NCOPY(x2,x1,i);
02149         break;
02150     }
02151 }
02152 /*
02153 Now the GCD is in c but still needs j powers of 2.
02154 */
02155 x1 = c;
02156 while ( j >= BITSINWORD ) {
02157     for ( i = n1; i > 0; i-- ) x1[i] = x1[i-1];
02158     x1[0] = 0; n1++;
02159     j -= BITSINWORD;
02160 }
02161 if ( j > 0 ) {
02162     ULONG a1,a2 = 0;
02163     for ( i = 0; i < n1; i++ ) {
02164         a1 = x1[i]; a1 «= j;
02165         a2 += a1;
02166         x1[i] = a2;
02167         a2 »= BITSINWORD;
02168     }

```



```

02169         if ( a2 != 0 ) {
02170             x1[n1++] = a2;
02171         }
02172     }
02173     *nc = n1;
02174     NumberFree(GLscrat7,"GcdLong"); NumberFree(GLscrat8,"GcdLong");
02175 #else
02176     UWORD *x1,*x2,*x3,*x4,*c1,*c2;
02177     WORD n1,n2,n3,n4,i;
02178     x1 = c; x3 = a; n1 = i = na;
02179     NCOPY(x1,x3,i);
02180     x1 = c; c1 = x2 = NumberMalloc("GcdLong"); x3 = NumberMalloc("GcdLong"); x4 =
NumberMalloc("GcdLong");
02181     c2 = b; n2 = i = nb;
02182     NCOPY(c1,c2,i);
02183     for(;;){
02184         if ( DivLong(x1,n1,x2,n2,x4,&n4,x3,&n3) ) goto GcdErr;
02185         if ( !n3 ) { x1 = x2; n1 = n2; break; }
02186         if ( DivLong(x2,n2,x3,n3,x4,&n4,x1,&n1) ) goto GcdErr;
02187         if ( !n1 ) { x1 = x3; n1 = n3; break; }
02188         if ( DivLong(x3,n3,x1,n1,x4,&n4,x2,&n2) ) goto GcdErr;
02189         if ( !n2 ) {
02190             *nc = n1;
02191             NumberFree(x2,"GcdLong"); NumberFree(x3,"GcdLong"); NumberFree(x4,"GcdLong");
02192             return(0);
02193         }
02194     }
02195     *nc = i = n1;
02196     NCOPY(c,x1,i);
02197     NumberFree(x2,"GcdLong"); NumberFree(x3,"GcdLong"); NumberFree(x4,"GcdLong");
02198 #endif
02199 #endif
02200 }
02201 return(0);
02202 GcdErr:
02203     MLOCK(ErrorMessageLock);
02204     MesCall("GcdLong");
02205     MUNLOCK(ErrorMessageLock);
02206     SETERROR(-1)
02207 }
02208 /*
02209 #] Old Routine :
02210 */
02211 #else
02212
02213 /*
02214     New routine for GcdLong that uses smart shortcuts.
02215     Algorithm by J. Vermaseren 15-nov-2006.
02216     It runs faster for very big numbers but only by a fixed factor.
02217     There is no improvement in the power behaviour.
02218     Improvement on the whole of hf9 (multiple zeta values at weight 9):
02219         Better than a factor 2 on a 32 bits architecture and 2.76 on a
02220         64 bits architecture.
02221     On hf10 (MZV's at weight 10), 64 bits architecture: factor 7.
02222
02223     If we have two long numbers (na,nb > GCDMAX) we will work in a
02224     truncated way. At the moment of writing (15-nov-2006) it isn't
02225     clear whether this algorithm is an invention or a reinvention.
02226     A short search on the web didn't show anything.
02227
02228     31-jul-2007:
02229     A better search shows that this is an adaptation of the Lehmer-Euclid
02230     algorithm, already described in Knuth. Here we can work without upper
02231     and lower limit because we are only interested in the GCD, not the
02232     extra numbers. Also it takes already some features of the double
02233     digit Lehmer-Euclid algorithm of Jebelean it seems.
02234
02235     Maybe this can be programmed slightly better and we can get another
02236     few percent speed increase. Further improvements for the asymptotic
02237     case come from splitting the calculation as in Karatsuba and working
02238     with FFT divisions and multiplications etc. But this is when hundreds
02239     of words are involved at the least.
02240
02241     Algorithm
02242
02243     1: while ( na > nb || nb < GCDMAX ) {
02244         if ( nb == 0 ) { result in a }
02245         c = a % b;
02246         a = b;
02247         b = c;
02248     }
02249     2: Make the truncated values in which a and b are the combinations
02250     of the top two words of a and b. The whole numbers are aa and bb now.
02251     3: ma1 = 1; ma2 = 0; mb1 = 0; mb2 = 1;
02252     4: A = a; B = b; m = a/b; c = a - m*b;
02253         c = ma1*a+ma2*b-m*(mb1*a+mb2*b) = (ma1-m*mb1)*a+(ma2-m*mb2)*b
02254         mc1 = ma1-m*mb1; mc2 = ma2-m*mb2;

```

```

02255     5: a = b; ma1 = mb1; ma2 = mb2;
02256     b = c; mb1 = mc1; mb2 = mc2;
02257     6: if ( b != 0 && nb >= FULLMAX ) goto 4;
02258     7: Now construct the new quantities
02259         ma1*aa+ma2*bb and mb1*aa+mb2*bb
02260     8: goto 1;
02261
02262     The essence of the above algorithm is that we do the divisions only
02263     on relatively short numbers. Also usually there are many steps 4&5
02264     for each step 7. This eliminates many operations.
02265     The termination at FULLMAX is that we make errors by not considering
02266     the tail of the number. If we run b down all the way, the errors combine
02267     in such a way that the new numbers may be of the same order as the old
02268     numbers. By stopping halfway we don't get the error beyond halfway
02269     either. Unfortunately this means that a >= FULLMAX and hence na > nb
02270     which means that next we will have a complete division. But just once.
02271     Running the steps 4-6 till a < FULLMAX runs already into problems.
02272     It may be necessary to experiment a bit to obtain the optimum value
02273     of GCDMAX.
02274 */
02275
02276 WORD GcdLong(PHEAD UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc)
02277 {
02278     GETBIDENTITY
02279     UWORD x,y,z;
02280     UWORD *x1,*x2,*x3,*x4,*x5,*d;
02281     UWORD *GLscrat6, *GLscrat7, *GLscrat8, *GLscrat9, *GLscrat10;
02282     WORD n1,n2,n3,n4,n5,i;
02283     RLONG lx,ly,lz;
02284     LONG ma1, ma2, mb1, mb2, mc1, mc2, m;
02285     if ( !na || !nb ) {
02286         if ( !na && !nb ) {
02287             MLOCK(ErrorMessageLock);
02288             MesPrint("Cannot take gcd");
02289             MUNLOCK(ErrorMessageLock);
02290             return(-1);
02291         }
02292
02293         if ( !na ) {
02294             *nc = abs(nb);
02295             NCOPY(c,b,*nc);
02296             *nc = abs(nb);
02297             return(0);
02298         }
02299
02300         *nc = abs(na);
02301         NCOPY(c,a,*nc);
02302         *nc = abs(na);
02303         return(0);
02304     }
02305     if ( na < 0 ) na = -na;
02306     if ( nb < 0 ) nb = -nb;
02307 /*
02308     #[ GMP stuff :
02309 */
02310 #ifdef WITHGMP
02311     if ( na > 3 && nb > 3 ) {
02312         int ii;
02313         mp_limb_t *upa, *upb, *upc, xx;
02314         UWORD *uw, *u1, *u2;
02315         unsigned int tcounta, tcountb, tcountal, tcountbl;
02316         mp_size_t ana, anb, anc;
02317
02318         u1 = uw = NumberMalloc("GcdLong");
02319         upa = (mp_limb_t *)u1;
02320         ana = na; tcountal = 0;
02321         while ( a[0] == 0 ) { a++; ana--; tcountal++; }
02322         for ( ii = 0; ii < ana; ii++ ) { *uw++ = *a++; }
02323         if ( ( ana & 1 ) != 0 ) { *uw = 0; ana++; }
02324         ana /= 2;
02325
02326         u2 = uw = NumberMalloc("GcdLong");
02327         upb = (mp_limb_t *)u2;
02328         anb = nb; tcountbl = 0;
02329         while ( b[0] == 0 ) { b++; anb--; tcountbl++; }
02330         for ( ii = 0; ii < anb; ii++ ) { *uw++ = *b++; }
02331         if ( ( anb & 1 ) != 0 ) { *uw = 0; anb++; }
02332         anb /= 2;
02333
02334         xx = upa[0]; tcounta = 0;
02335         while ( ( xx & 15 ) == 0 ) { tcounta += 4; xx >>= 4; }
02336         while ( ( xx & 1 ) == 0 ) { tcounta += 1; xx >>= 1; }
02337         xx = upb[0]; tcountb = 0;
02338         while ( ( xx & 15 ) == 0 ) { tcountb += 4; xx >>= 4; }
02339         while ( ( xx & 1 ) == 0 ) { tcountb += 1; xx >>= 1; }
02340
02341         if ( tcounta ) {

```

```

02342         mpn_rshift(upa,upa,ana,tcounta);
02343         if ( upa[ana-1] == 0 ) ana--;
02344     }
02345     if ( tcountb ) {
02346         mpn_rshift(upb,upb,anb,tcountb);
02347         if ( upb[anb-1] == 0 ) anb--;
02348     }
02349
02350     upc = (mp_limb_t *) (NumberMalloc("GcdLong"));
02351     if ( ( ana > anb ) || ( ( ana == anb ) && ( upa[ana-1] >= upb[ana-1] ) ) ) {
02352         anc = mpn_gcd(upc,upa,ana,upb,anb);
02353     }
02354     else {
02355         anc = mpn_gcd(upc,upb,anb,upa,ana);
02356     }
02357
02358     tcounta = tcounta*BITSINWORD + tcounta;
02359     tcountb = tcountb*BITSINWORD + tcountb;
02360     if ( tcountb > tcounta ) tcountb = tcounta;
02361     tcounta = tcountb/BITSINWORD;
02362     tcountb = tcountb%BITSINWORD;
02363
02364     if ( tcountb ) {
02365         xx = mpn_lshift(upc,upc,anc,tcountb);
02366         if ( xx ) { upc[anc] = xx; anc++; }
02367     }
02368
02369     uw = (UWORD *)upc; anc *= 2;
02370     while ( uw[anc-1] == 0 ) anc--;
02371     for ( ii = 0; ii < (int)tcounta; ii++ ) *c++ = 0;
02372     for ( ii = 0; ii < anc; ii++ ) *c++ = *uw++;
02373     *nc = anc + tcounta;
02374     NumberFree(u1,"GcdLong"); NumberFree(u2,"GcdLong"); NumberFree((UWORD *) (upc),"GcdLong");
02375     return(0);
02376 }
02377 #endif
02378 /*
02379 #] GMP stuff :
02380 */
02381 /*
02382 #[ Easy cases :
02383 */
02384     if ( na == 1 && nb == 1 ) {
02385         x = *a;
02386         y = *b;
02387         do { z = x % y; x = y; } while ( ( y = z ) != 0 );
02388         *c = x;
02389         *nc = 1;
02390         return(0);
02391     }
02392     else if ( na <= 2 && nb <= 2 ) {
02393         if ( na == 2 ) { lx = ((RLONG) (a[1]))«BITSINWORD + *a; }
02394         else { lx = *a; }
02395         if ( nb == 2 ) { ly = ((RLONG) (b[1]))«BITSINWORD + *b; }
02396         else { ly = *b; }
02397         if ( lx < ly ) { lz = lx; lx = ly; ly = lz; }
02398         #if ( BITSINWORD == 16 )
02399             do {
02400                 lz = lx % ly; lx = ly;
02401             } while ( ( ly = lz ) != 0 );
02402         #else
02403             do {
02404                 lz = lx % ly; lx = ly;
02405             } while ( ( ly = lz ) != 0 && ( lx & AWORDMASK ) != 0 );
02406             if ( ly ) {
02407                 x = (UWORD)lx; y = (UWORD)ly;
02408                 do { *c = x % y; x = y; } while ( ( y = *c ) != 0 );
02409                 *c = x;
02410                 *nc = 1;
02411             }
02412             else
02413         #endif
02414         {
02415             *c++ = (UWORD)lx;
02416             if ( ( *c = (UWORD) (lx » BITSINWORD) ) != 0 ) *nc = 2;
02417             else *nc = 1;
02418         }
02419         return(0);
02420     }
02421     /*
02422     #] Easy cases :
02423     */
02424     GLscrat6 = NumberMalloc("GcdLong"); GLscrat7 = NumberMalloc("GcdLong");
02425     GLscrat8 = NumberMalloc("GcdLong");
02426     GLscrat9 = NumberMalloc("GcdLong"); GLscrat10 = NumberMalloc("GcdLong");
02427     restart;;
02428     /*

```

```

02429     #] Easy cases :
02430     /*
02431     if ( na == 1 && nb == 1 ) {
02432         x = *a;
02433         y = *b;
02434         do { z = x % y; x = y; } while ( ( y = z ) != 0 );
02435         *c = x;
02436         *nc = 1;
02437     }
02438     else if ( na <= 2 && nb <= 2 ) {
02439         if ( na == 2 ) { lx = ((RLONG)(a[1]))«BITSINWORD + *a; }
02440         else { lx = *a; }
02441         if ( nb == 2 ) { ly = ((RLONG)(b[1]))«BITSINWORD + *b; }
02442         else { ly = *b; }
02443         if ( lx < ly ) { lz = lx; lx = ly; ly = lz; }
02444     #if ( BITSINWORD == 16 )
02445         do {
02446             lz = lx % ly; lx = ly;
02447         } while ( ( ly = lz ) != 0 );
02448     #else
02449         do {
02450             lz = lx % ly; lx = ly;
02451         } while ( ( ly = lz ) != 0 && ( lx & AWORDMASK ) != 0 );
02452         if ( ly ) {
02453             x = (UWORD)lx; y = (UWORD)ly;
02454             do { *c = x % y; x = y; } while ( ( y = *c ) != 0 );
02455             *c = x;
02456             *nc = 1;
02457         }
02458         else
02459     #endif
02460     {
02461         *c++ = (UWORD)lx;
02462         if ( ( *c = (UWORD)(lx » BITSINWORD) ) != 0 ) *nc = 2;
02463         else *nc = 1;
02464     }
02465     }
02466     /*
02467     #] Easy cases :
02468     #] Original code :
02469     /*
02470     else if ( na < GCDMAX || nb < GCDMAX || na != nb ) {
02471         if ( na < nb ) {
02472             x2 = GLscrat8; x3 = a; n2 = i = na;
02473             NCOPY(x2,x3,i);
02474             x1 = c; x3 = b; n1 = i = nb;
02475             NCOPY(x1,x3,i);
02476         }
02477         else {
02478             x1 = c; x3 = a; n1 = i = na;
02479             NCOPY(x1,x3,i);
02480             x2 = GLscrat8; x3 = b; n2 = i = nb;
02481             NCOPY(x2,x3,i);
02482         }
02483         x1 = c; x2 = GLscrat8; x3 = GLscrat7; x4 = GLscrat6;
02484         for(;;){
02485             if ( DivLong(x1,n1,x2,n2,x4,&n4,x3,&n3) ) goto GcdErr;
02486             if ( !n3 ) { x1 = x2; n1 = n2; break; }
02487             if ( n2 <= 2 ) { a = x2; b = x3; na = n2; nb = n3; goto restart; }
02488             if ( n3 >= GCDMAX && n2 == n3 ) {
02489                 a = GLscrat9; b = GLscrat10; na = n2; nb = n3;
02490                 for ( i = 0; i < na; i++ ) a[i] = x2[i];
02491                 for ( i = 0; i < nb; i++ ) b[i] = x3[i];
02492                 goto newtrick;
02493             }
02494             if ( DivLong(x2,n2,x3,n3,x4,&n4,x1,&n1) ) goto GcdErr;
02495             if ( !n1 ) { x1 = x3; n1 = n3; break; }
02496             if ( n3 <= 2 ) { a = x3; b = x1; na = n3; nb = n1; goto restart; }
02497             if ( n1 >= GCDMAX && n1 == n3 ) {
02498                 a = GLscrat9; b = GLscrat10; na = n3; nb = n1;
02499                 for ( i = 0; i < na; i++ ) a[i] = x3[i];
02500                 for ( i = 0; i < nb; i++ ) b[i] = x1[i];
02501                 goto newtrick;
02502             }
02503             if ( DivLong(x3,n3,x1,n1,x4,&n4,x2,&n2) ) goto GcdErr;
02504             if ( !n2 ) { *nc = n1; goto normalend; }
02505             if ( n1 <= 2 ) { a = x1; b = x2; na = n1; nb = n2; goto restart; }
02506             if ( n2 >= GCDMAX && n2 == n1 ) {
02507                 a = GLscrat9; b = GLscrat10; na = n1; nb = n2;
02508                 for ( i = 0; i < na; i++ ) a[i] = x1[i];
02509                 for ( i = 0; i < nb; i++ ) b[i] = x2[i];
02510                 goto newtrick;
02511             }
02512         }
02513         *nc = i = n1;
02514         NCOPY(c,x1,i);
02515     }

```

```

02516 /*
02517     #] Original code :
02518     #[ New code :
02519 */
02520     else {
02521 /*
02522         This is the new algorithm starting at step 3.
02523
02524         3: ma1 = 1; ma2 = 0; mb1 = 0; mb2 = 1;
02525         4: A = a; B = b; m = a/b; c = a - m*b;
02526           c = ma1*a+ma2*b-m*(mb1*a+mb2*b) = (ma1-m*mb1)*a+(ma2-m*mb2)*b
02527           mc1 = ma1-m*mb1; mc2 = ma2-m*mb2;
02528         5: a = b; ma1 = mb1; ma2 = mb2;
02529           b = c; mb1 = mc1; mb2 = mc2;
02530         6: if ( b != 0 ) goto 4;
02531 */
02532 newtrick;;
02533     ma1 = 1; ma2 = 0; mb1 = 0; mb2 = 1;
02534     lx = ((RLONG)(a[na-1]))«BITSINWORD + a[na-2];
02535     ly = ((RLONG)(b[nb-1]))«BITSINWORD + b[nb-2];
02536     if ( ly > lx ) { lz = lx; lx = ly; ly = lz; d = a; a = b; b = d; }
02537     do {
02538         m = lx/ly;
02539         mc1 = ma1-m*mb1; mc2 = ma2-m*mb2;
02540         ma1 = mb1; ma2 = mb2; mb1 = mc1; mb2 = mc2;
02541         lz = lx - m*ly; lx = ly; ly = lz;
02542     } while ( ly >= FULLMAX );
02543 /*
02544     Next the construction of the two new numbers
02545
02546     7: Now construct the new quantities
02547       a = ma1*aa+ma2*bb and b = mb1*aa+mb2*bb
02548 */
02549     x1 = GLscrat6;
02550     x2 = GLscrat7;
02551     x3 = GLscrat8;
02552     x5 = GLscrat10;
02553     if ( ma1 < 0 ) {
02554         ma1 = -ma1;
02555         x1[0] = (UWORD)ma1;
02556         x1[1] = (UWORD)(ma1 » BITSINWORD);
02557         if ( x1[1] ) n1 = -2;
02558         else n1 = -1;
02559     }
02560     else {
02561         x1[0] = (UWORD)ma1;
02562         x1[1] = (UWORD)(ma1 » BITSINWORD);
02563         if ( x1[1] ) n1 = 2;
02564         else n1 = 1;
02565     }
02566     if ( MulLong(a,na,x1,n1,x2,&n2) ) goto GcdErr;
02567     if ( ma2 < 0 ) {
02568         ma2 = -ma2;
02569         x1[0] = (UWORD)ma2;
02570         x1[1] = (UWORD)(ma2 » BITSINWORD);
02571         if ( x1[1] ) n1 = -2;
02572         else n1 = -1;
02573     }
02574     else {
02575         x1[0] = (UWORD)ma2;
02576         x1[1] = (UWORD)(ma2 » BITSINWORD);
02577         if ( x1[1] ) n1 = 2;
02578         else n1 = 1;
02579     }
02580     if ( MulLong(b,nb,x1,n1,x3,&n3) ) goto GcdErr;
02581     if ( AddLong(x2,n2,x3,n3,c,&n4) ) goto GcdErr;
02582     if ( mb1 < 0 ) {
02583         mb1 = -mb1;
02584         x1[0] = (UWORD)mb1;
02585         x1[1] = (UWORD)(mb1 » BITSINWORD);
02586         if ( x1[1] ) n1 = -2;
02587         else n1 = -1;
02588     }
02589     else {
02590         x1[0] = (UWORD)mb1;
02591         x1[1] = (UWORD)(mb1 » BITSINWORD);
02592         if ( x1[1] ) n1 = 2;
02593         else n1 = 1;
02594     }
02595     if ( MulLong(a,na,x1,n1,x2,&n2) ) goto GcdErr;
02596     if ( mb2 < 0 ) {
02597         mb2 = -mb2;
02598         x1[0] = (UWORD)mb2;
02599         x1[1] = (UWORD)(mb2 » BITSINWORD);
02600         if ( x1[1] ) n1 = -2;
02601         else n1 = -1;
02602     }

```

```

02603         else {
02604             x1[0] = (UWORD)mb2;
02605             x1[1] = (UWORD)(mb2 » BITSINWORD);
02606             if ( x1[1] ) n1 = 2;
02607             else n1 = 1;
02608         }
02609         if ( MulLong(b,nb,x1,n1,x3,&n3) ) goto GcdErr;
02610         if ( AddLong(x2,n2,x3,n3,x5,&n5) ) goto GcdErr;
02611         a = c; na = n4; b = x5; nb = n5;
02612         if ( nb == 0 ) { *nc = n4; goto normalend; }
02613         x4 = GLscrat9;
02614         for ( i = 0; i < na; i++ ) x4[i] = a[i];
02615         a = x4;
02616         if ( na < 0 ) na = -na;
02617         if ( nb < 0 ) nb = -nb;
02618     /*
02619         The typical case now is that in a we have the last step to go
02620         to loose the leading word, while in b we have lost the leading word.
02621         We could go to DivLong now but we can also add an extra step that
02622         is less wasteful.
02623         In the case that the new leading word of b is extrememly short (like 1)
02624         we make a rather large error of course. In the worst case the whole
02625         will be intercepted by DivLong after all, but that is so rare that
02626         it shouldn't influence any timing in a measurable way.
02627     */
02628     if ( nb >= GCDMAX && na == nb+1 && b[nb-1] >= HALFMAX && b[nb-1] > a[na-1] ) {
02629         lx = (((RLONG)(a[na-1]))«BITSINWORD) + a[na-2];
02630         x1[0] = lx/b[nb-1]; n1 = 1;
02631         Mullong(b,nb,x1,n1,x2,&n2);
02632         n2 = -n2;
02633         AddLong(a,na,x2,n2,x4,&n4);
02634         if ( n4 == 0 ) {
02635             *nc = nb;
02636             for ( i = 0; i < nb; i++ ) c[i] = b[i];
02637             goto normalend;
02638         }
02639         if ( n4 < 0 ) n4 = -n4;
02640         a = b; na = nb; b = x4; nb = n4;
02641     }
02642     goto restart;
02643 /*
02644     #] New code :
02645 */
02646 }
02647 normalend:
02648     NumberFree(GLscrat6,"GcdLong"); NumberFree(GLscrat7,"GcdLong"); NumberFree(GLscrat8,"GcdLong");
02649     NumberFree(GLscrat9,"GcdLong"); NumberFree(GLscrat10,"GcdLong");
02650     return(0);
02651 GcdErr:
02652     MLOCK(ErrorMessageLock);
02653     MesCall("GcdLong");
02654     MUNLOCK(ErrorMessageLock);
02655     NumberFree(GLscrat6,"GcdLong"); NumberFree(GLscrat7,"GcdLong"); NumberFree(GLscrat8,"GcdLong");
02656     NumberFree(GLscrat9,"GcdLong"); NumberFree(GLscrat10,"GcdLong");
02657     SETERROR(-1)
02658 }
02659 #endif
02660 /*
02661     #] GcdLong :
02662     #[ GetBinom :      WORD GetBinom(a,na,i1,i2)
02663 */
02664 WORD GetBinom(UWORD *a, WORD *na, WORD i1, WORD i2)
02665 {
02666     GETIDENTITY
02667     WORD j, k, l;
02668     UWORD *GBscrat3, *GBscrat4;
02669     if ( i1-i2 < i2 ) i2 = i1-i2;
02670     if ( i2 == 0 ) { *a = 1; *na = 1; return(0); }
02671     if ( i2 > i1 ) { *a = 0; *na = 0; return(0); }
02672     *a = i1; *na = 1;
02673     GBscrat3 = NumberMalloc("GetBinom"); GBscrat4 = NumberMalloc("GetBinom");
02674     for ( j = 2; j <= i2; j++ ) {
02675         GBscrat3[0] = i1+1-j;
02676         if ( MulLong(a,*na,GBscrat3,(WORD)1,GBscrat4,&k) ) goto CalledFrom;
02677         GBscrat3[0] = j;
02678         if ( DivLong(GBscrat4,k,GBscrat3,(WORD)1,a,na,GBscrat3,&l) ) goto CalledFrom;
02679     }
02680     NumberFree(GBscrat3,"GetBinom"); NumberFree(GBscrat4,"GetBinom");
02681     return(0);
02682 CalledFrom:
02683     MLOCK(ErrorMessageLock);
02684     MesCall("GetBinom");
02685     MUNLOCK(ErrorMessageLock);
02686     NumberFree(GBscrat3,"GetBinom"); NumberFree(GBscrat4,"GetBinom");

```

```

02690     SETERROR(-1)
02691 }
02692
02693 /*
02694     #] GetBinom :
02695     #[ LcmLong :      WORD LcmLong(a,na,b,nb)
02696
02697     Computes the LCM of the long numbers a and b and puts the result
02698     in c. c is allowed to be equal to a.
02699 */
02700
02701 WORD LcmLong(PHEAD UWORD *a, WORD na, UWORD *b, WORD nb, UWORD *c, WORD *nc)
02702 {
02703     WORD error = 0;
02704     UWORD *d = NumberMalloc("LcmLong");
02705     UWORD *e = NumberMalloc("LcmLong");
02706     UWORD *f = NumberMalloc("LcmLong");
02707     WORD nd, ne, nf;
02708     GcdLong(BHEAD a, na, b, nb, d, &nd);
02709     DivLong(a,na,d,nd,e,&ne,f,&nf);
02710     if ( MullLong(b,nb,e,ne,c,nc) ) {
02711         MLOCK(ErrorMessageLock);
02712         MesCall("LcmLong");
02713         MUNLOCK(ErrorMessageLock);
02714         error = -1;
02715     }
02716     NumberFree(f,"LcmLong");
02717     NumberFree(e,"LcmLong");
02718     NumberFree(d,"LcmLong");
02719     return(error);
02720 }
02721
02722 /*
02723     #] LcmLong :
02724     #[ TakeLongRoot:   int TakeLongRoot(a,n,power)
02725
02726     Takes the 'power'-root of the long number in a.
02727     If the root could be taken the return value is zero.
02728     If the root could not be taken, the return value is 1.
02729     The root will be in a if it could be taken, otherwise there will be garbage
02730     Algorithm: (assume b is guess of root, b' better guess)
02731         b' = (a-(power-1)*b^power)/(n*b^(power-1))
02732     Note: power should be positive!
02733 */
02734
02735 int TakeLongRoot(UWORD *a, WORD *n, WORD power)
02736 {
02737     GETIDENTITY
02738     int numbits, guessbits, i, retval = 0;
02739     UWORD x, *b, *c, *d, *e;
02740     WORD na, nb, nc, nd, ne;
02741     if ( *n < 0 && ( power & 1 ) == 0 ) return(1);
02742     if ( power == 1 ) return(0);
02743     if ( *n < 0 ) { na = -*n; }
02744     else          { na = *n; }
02745     if ( na == 1 ) {
02746         /*      Special cases that are the most frequent */
02747         if ( a[0] == 1 ) return(0);
02748         if ( power < BITSINWORD && na == 1 && a[0] == (UWORD)(1<<power) ) {
02749             a[0] = 2; return(0);
02750         }
02751         if ( 2*power < BITSINWORD && na == 1 && a[0] == (UWORD)(1<<(2*power)) ) {
02752             a[0] = 4; return(0);
02753         }
02754     }
02755     /*
02756     Step 1: make a guess. We count the number of bits.
02757     numbits will be the 1+2log(a)
02758 */
02759     numbits = BITSINWORD*(na-1);
02760     x = a[na-1];
02761     while ( ( x >> 8 ) != 0 ) { numbits += 8; x >>= 8; }
02762     if ( ( x >> 4 ) != 0 ) { numbits += 4; x >>= 4; }
02763     if ( ( x >> 2 ) != 0 ) { numbits += 2; x >>= 2; }
02764     if ( ( x >> 1 ) != 0 ) numbits++;
02765     guessbits = numbits / power;
02766     if ( guessbits <= 0 ) return(1); /* root < 2 and 1 we did already */
02767     nb = guessbits/BITSINWORD;
02768     /*
02769     The recursion is:
02770     (b'-b) = (a/b^(power-1)-b)/n
02771             = (a/c-b)/n
02772             = (d-b)/n      (remainder of a/c is e)
02773             = c/n      (we reuse the scratch array c)
02774     Termination can be tricky. When a/c has no remainder and = b we have a root.
02775     When d = b but the remainder of a/c != 0, there is definitely no root.
02776 */

```

```

02777     b = NumberMalloc("TakeLongRoot"); c = NumberMalloc("TakeLongRoot");
02778     d = NumberMalloc("TakeLongRoot"); e = NumberMalloc("TakeLongRoot");
02779     for ( i = 0; i < nb; i++ ) { b[i] = 0; }
02780     b[nb] = 1 « (guessbits%BITSINWORD);
02781     nb++;
02782     for(;;) {
02783         nc = nb;
02784         for ( i = 0; i < nb; i++ ) c[i] = b[i];
02785         if ( RaisPow(BHEAD c,&nc,power-1) ) goto TLcall;
02786         if ( DivLong(a,na,c,nc,d,&nd,e,&ne) ) goto TLcall;
02787         nb = -nb;
02788         if ( AddLong(d,nd,b,nb,c,&nc) ) goto TLcall;
02789         nb = -nb;
02790         if ( nc == 0 ) {
02791             if ( ne == 0 ) break;
02792             retval = 1; break;
02793         /*
02794             else {
02795                 NumberFree(b,"TakeLongRoot"); NumberFree(c,"TakeLongRoot");
02796                 NumberFree(d,"TakeLongRoot"); NumberFree(e,"TakeLongRoot");
02797                 return(1);
02798             }
02799         */
02800     }
02801     DivLong(c,nc,(UWORD *)(&power),1,d,&nd,e,&ne);
02802     if ( nd == 0 ) {
02803         retval = 1;
02804         break;
02805     /*
02806         NumberFree(b,"TakeLongRoot"); NumberFree(c,"TakeLongRoot");
02807         NumberFree(d,"TakeLongRoot"); NumberFree(e,"TakeLongRoot");
02808         return(1);
02809     */
02810     /*
02811         This code tries b+1 as a final possibility.
02812         We believe this is not needed
02813         UWORD one = 1;
02814         if ( AddLong(b,nb,&one,1,c,&nc) ) goto TLcall;
02815         if ( RaisPow(BHEAD c,&nc,power-1) ) goto TLcall;
02816         if ( DivLong(a,na,c,nc,d,&nd,e,&ne) ) goto TLcall;
02817         if ( ne != 0 ) return(1);
02818         nb = -nb;
02819         if ( SubLong(d,nd,b,nb,c,&nc) ) goto TLcall;
02820         nb = -nb;
02821         if ( nc != 0 ) {
02822             NumberFree(b,"TakeLongRoot"); NumberFree(c,"TakeLongRoot");
02823             NumberFree(d,"TakeLongRoot"); NumberFree(e,"TakeLongRoot");
02824             return(1);
02825         }
02826         break;
02827     */
02828     }
02829     if ( AddLong(b,nb,d,nd,b,&nb) ) goto TLcall;
02830 }
02831 for ( i = 0; i < nb; i++ ) a[i] = b[i];
02832 if ( *n < 0 ) *n = -nb;
02833 else *n = nb;
02834 NumberFree(b,"TakeLongRoot"); NumberFree(c,"TakeLongRoot");
02835 NumberFree(d,"TakeLongRoot"); NumberFree(e,"TakeLongRoot");
02836 return(retval);
02837 TLcall:
02838 MLOCK(ErrorMessageLock);
02839 MesCall("TakeLongRoot");
02840 MUNLOCK(ErrorMessageLock);
02841 NumberFree(b,"TakeLongRoot"); NumberFree(c,"TakeLongRoot");
02842 NumberFree(d,"TakeLongRoot"); NumberFree(e,"TakeLongRoot");
02843 Terminate(-1);
02844 return(-1);
02845 }
02846
02847 /*
02848     #] TakeLongRoot:
02849     #[ MakeRational:
02850
02851     Makes the integer a mod m into a traction b/c with |b|,|c| < sqrt(m)
02852     For the algorithm, see MakeLongRational.
02853 */
02854
02855 int MakeRational(WORD a,WORD m, WORD *b, WORD *c)
02856 {
02857     LONG x1,x2,x3,x4,y1,y2;
02858     if ( a < 0 ) { a = a+m; }
02859     if ( a <= 1 ) {
02860         if ( a > m/2 ) a = a-m;
02861         *b = a; *c = 1; return(0);
02862     }
02863     x1 = m; x2 = a;

```



```

02864     if ( x2*x2 >= m ) {
02865         y1 = x1/x2; y2 = x1%x2; x3 = 1; x4 = -y1; x1 = x2; x2 = y2;
02866         while ( x2*x2 >= m ) {
02867             y1 = x1/x2; y2 = x1%x2; x1 = x2; x2 = y2; y2 = x3-y1*x4; x3 = x4; x4 = y2;
02868         }
02869     }
02870     else x4 = 1;
02871     if ( x2 == 0 ) { return(1); }
02872     if ( x2 > m/2 ) *b = x2-m;
02873     else *b = x2;
02874     if ( x4 > m/2 ) { *c = x4-m; *c = -*c; *b = -*b; }
02875     else if ( x4 <= -m/2 ) { x4 += m; *c = x4; }
02876     else if ( x4 < 0 ) { x4 = -x4; *c = x4; *b = -*b; }
02877     else *c = x4;
02878     return(0);
02879 }
02880
02881 /*
02882     #] MakeRational:
02883     #[ MakeLongRational:
02884
02885     Converts the long number a mod m into the fraction b
02886     One of the properties of b is that num,den < sqrt(m)
02887     The algorithm: Start with:  m 0
02888                               a 1
02889     Make now c=m/a, cl=m/a      c c2=0-cl*1
02890     Make now d=a%c  dl=a/c      d d2=1-dl*c2
02891     Make now e=c%d  el=c/d      e e2=1-el*d2
02892     etc till in the first column we get a number < sqrt(m)
02893     We have then f,f2 and the fraction is f/f2.
02894     If at any moment we get a zero, m contained an unlucky prime.
02895
02896     Note that this can be made a lot faster when we make the same
02897     improvements as in the GCD routine. That is something for later.
02898 #ifdef WITHMAKERATIONAL
02899 */
02900
02901 #define COPYLONG(x1,nx1,x2,nx2) { int i; for(i=0;i<ABS(nx2);i++)x1[i]=x2[i];nx1=nx2; }
02902
02903 int MakeLongRational(PHEAD UWORD *a, WORD na, UWORD *m, WORD nm, UWORD *b, WORD *nb)
02904 {
02905     UWORD *root = NumberMalloc("MakeRational");
02906     UWORD *x1 = NumberMalloc("MakeRational");
02907     UWORD *x2 = NumberMalloc("MakeRational");
02908     UWORD *x3 = NumberMalloc("MakeRational");
02909     UWORD *x4 = NumberMalloc("MakeRational");
02910     UWORD *y1 = NumberMalloc("MakeRational");
02911     UWORD *y2 = NumberMalloc("MakeRational");
02912     WORD nroot,nx1,nx2,nx3,nx4,ny1,ny2 = 0,retval = 0;
02913     WORD sign = 1;
02914     /*
02915     Step 1: Take the square root of m
02916     */
02917     COPYLONG(root,nroot,m,nm)
02918     TakeLongRoot(root,&nroot,2);
02919     /*
02920     Step 2: Set the start values
02921     */
02922     if ( na < 0 ) { na = -na; sign = -sign; }
02923     COPYLONG(x1,nx1,m,nm)
02924     COPYLONG(x2,nx2,a,na)
02925     /*
02926     x3[0] = 0, nx3 = 0;
02927     x4[0] = 1, nx4 = 1;
02928     */
02929     /*
02930     The start operation needs some special attention because of the zero.
02931     */
02932     if ( BigLong(x2,nx2,root,nroot) <= 0 ) {
02933         x4[0] = 1, nx4 = 1;
02934         goto gottheanswer;
02935     }
02936     DivLong(x1,nx1,x2,nx2,y1,&ny1,y2,&ny2);
02937     if ( ny2 == 0 ) { retval = 1; goto cleanup; }
02938     COPYLONG(x1,nx1,x2,nx2)
02939     COPYLONG(x2,nx2,y2,ny2)
02940     x3[0] = 1; nx3 = 1;
02941     COPYLONG(x4,nx4,y1,ny1)
02942     nx4 = -nx4;
02943     /*
02944     Now the loop.
02945     */
02946     while ( BigLong(x2,nx2,root,nroot) > 0 ) {
02947         DivLong(x1,nx1,x2,nx2,y1,&ny1,y2,&ny2);
02948         if ( ny2 == 0 ) { retval = 1; goto cleanup; }
02949         COPYLONG(x1,nx1,x2,nx2)
02950         COPYLONG(x2,nx2,y2,ny2)

```

```

02951         MulLong(y1,ny1,x4,nx4,y2,&ny2);
02952         ny2 = -ny2;
02953         AddLong(x3,nx3,y2,ny2,y1,&ny1);
02954         COPYLONG(x3,nx3,x4,nx4);
02955         COPYLONG(x4,nx4,y1,ny1);
02956     }
02957     /*
02958         Now we have the answer. It is x2/x4. It has to be packed into b.
02959     */
02960     gottheanswer:
02961         if ( nx4 < 0 ) { sign = -sign; nx4 = -nx4; }
02962         COPYLONG(b,*nb,x2,nx2);
02963         Pack(b,nb,x4,nx4);
02964         if ( sign < 0 ) *nb = -*nb;
02965     cleanup:
02966         NumberFree(y2,"MakeRational");
02967         NumberFree(y1,"MakeRational");
02968         NumberFree(x4,"MakeRational");
02969         NumberFree(x3,"MakeRational");
02970         NumberFree(x2,"MakeRational");
02971         NumberFree(x1,"MakeRational");
02972         NumberFree(root,"MakeRational");
02973         return(retval);
02974     }
02975
02976     /*
02977     #endif
02978         #] MakeLongRational:
02979         #[ ChineseRemainder:
02980     */
02992     #ifdef WITHCHINESEREMAINDER
02993
02994     int ChineseRemainder(PHEAD MODNUM *a1, MODNUM *a2, MODNUM *a)
02995     {
02996         UWORD *inv1 = NumberMalloc("ChineseRemainder");
02997         UWORD *inv2 = NumberMalloc("ChineseRemainder");
02998         UWORD *fac1 = NumberMalloc("ChineseRemainder");
02999         UWORD *fac2 = NumberMalloc("ChineseRemainder");
03000         UWORD two[1];
03001         WORD ninv1, ninv2, nfac1, nfac2;
03002         if ( a1->na < 0 ) {
03003             AddLong(a1->a,a1->na,a1->m,a1->nm,a1->a,&(a1->na));
03004         }
03005         if ( a2->na < 0 ) {
03006             AddLong(a2->a,a2->na,a2->m,a2->nm,a2->a,&(a2->na));
03007         }
03008         MulLong(a1->m,a1->nm,a2->m,a2->nm,a->m,&(a->nm));
03009
03010         GetLongModInverses(BHEAD a1->m,a1->nm,a2->m,a2->nm,inv1,&ninv1,inv2,&ninv2);
03011         MulLong(inv1,ninv1,a1->m,a1->nm,fac1,&nfac1);
03012         MulLong(inv2,ninv2,a2->m,a2->nm,fac2,&nfac2);
03013
03014         MulLong(fac1,nfac1,a2->a,a2->na,inv1,&ninv1);
03015         MulLong(fac2,nfac2,a1->a,a1->na,inv2,&ninv2);
03016         AddLong(inv1,ninv1,inv2,ninv2,a->a,&(a->na));
03017
03018         two[0] = 2;
03019         MulLong(a->a,a->na,two,1,fac1,&nfac1);
03020         if ( BigLong(fac1,nfac1,a->m,a->nm) > 0 ) {
03021             a->nm = -a->nm;
03022             AddLong(a->a,a->na,a->m,a->nm,a->a,&(a->na));
03023             a->nm = -a->nm;
03024         }
03025         NumberFree(fac2,"ChineseRemainder");
03026         NumberFree(fac1,"ChineseRemainder");
03027         NumberFree(inv2,"ChineseRemainder");
03028         NumberFree(inv1,"ChineseRemainder");
03029         return(0);
03030     }
03031
03032     #endif
03033
03034     /*
03035         #] ChineseRemainder:
03036         #] RekenLong :
03037         #[ RekenTerms :
03038             #[ CompCoef :          WORD CompCoef(term1,term2)
03039
03040         Compares the coefficients of term1 and term2 by subtracting them.
03041         This does more work than needed but this routine is only called
03042         when sorting functions and function arguments.
03043         (and comparing values
03044     */
03045     /* #define 64SAVE */
03046
03047     WORD CompCoef(WORD *term1, WORD *term2)
03048     {

```

```

03049     GETIDENTITY
03050     UWORD *c;
03051     WORD n1,n2,n3,*a;
03052     GETCOEF(term1,n1);
03053     GETCOEF(term2,n2);
03054     if ( term1[1] == 0 && n1 == 1 ) {
03055         if ( term2[1] == 0 && n2 == 1 ) return(0);
03056         if ( n2 < 0 ) return(1);
03057         return(-1);
03058     }
03059     else if ( term2[1] == 0 && n2 == 1 ) {
03060         if ( n1 < 0 ) return(-1);
03061         return(1);
03062     }
03063     if ( n1 > 0 ) {
03064         if ( n2 < 0 ) return(1);
03065     }
03066     else {
03067         if ( n2 > 0 ) return(-1);
03068         a = term1; term1 = term2; term2 = a;
03069         n3 = -n1; n1 = -n2; n2 = n3;
03070     }
03071     if ( term1[1] == 1 && term2[1] == 1 && n1 == 1 && n2 == 1 ) {
03072         if ( (UWORD)*term1 > (UWORD)*term2 ) return(1);
03073         else if ( (UWORD)*term1 < (UWORD)*term2 ) return(-1);
03074         else return(0);
03075     }
03076
03077     /*
03078     The next call should get dedicated code, as AddRat does more than
03079     strictly needed. Also more attention should be given to overflow.
03080     */
03081     c = NumberMalloc("CompCoef");
03082     if ( AddRat(BHEAD (UWORD *)term1,n1,(UWORD *)term2,-n2,c,&n3) ) {
03083         MLOCK(ErrorMessageLock);
03084         MesCall("CompCoef");
03085         MUNLOCK(ErrorMessageLock);
03086         NumberFree(c,"CompCoef");
03087         SETERROR(-1)
03088     }
03089     NumberFree(c,"CompCoef");
03090     return(n3);
03091 }
03092
03093 /*
03094     #] CompCoef :
03095     #[ Modulus :      WORD Modulus(term)
03096
03097     Routine takes the coefficient of term modulus b. The answer
03098     is in term again and the length of term is adjusted.
03099
03100     */
03101
03102 WORD Modulus(WORD *term)
03103 {
03104     WORD *t;
03105     WORD n1;
03106     t = term;
03107     GETCOEF(t,n1);
03108     if ( TakeModulus((UWORD *)t,&n1,AC.cmod,AC.ncmod,UNPACK) ) {
03109         MLOCK(ErrorMessageLock);
03110         MesCall("Modulus");
03111         MUNLOCK(ErrorMessageLock);
03112         SETERROR(-1)
03113     }
03114     if ( !n1 ) {
03115         *term = 0;
03116         return(0);
03117     }
03118     else if ( n1 > 0 ) {
03119         n1 *= 2;
03120         t += n1;          /* Note that n1 >= 0 */
03121         n1++;
03122     }
03123     else if ( n1 < 0 ) {
03124         n1 *= 2;
03125         t += -n1;
03126         n1--;
03127     }
03128     *t++ = n1;
03129     *term = WORDDIFF(t,term);
03130     return(0);
03131 }
03132
03133 /*
03134     #] Modulus :
03135     #[ TakeModulus :  WORD TakeModulus(a,na,cmo,ncmod,par)

```

```

03136
03137     Routine gets the rational number in a with reduced length na.
03138     It is called when AC.ncmod != 0 and the number in AC.cmod is the
03139     number wrt which we need the modulus.
03140     The result is returned in a and na again.
03141
03142     If par == NOUNPACK we only do a single number, not a fraction.
03143     In addition we don't do fancy. We want a positive number and
03144     the input was supposed to be positive.
03145     We don't pack the result. The calling routine is responsible for that.
03146     This may not be a good idea. To be checked.
03147 */
03148
03149 WORD TakeModulus(UWORD *a, WORD *na, UWORD *cmodvec, WORD ncmod, WORD par)
03150 {
03151     GETIDENTITY
03152     UWORD *c, *d, *e, *f, *g, *h;
03153     UWORD *x4,*x2;
03154     UWORD *x3,*x1,*x5,*x6,*x7,*x8;
03155     WORD y3,y1,y5,y6;
03156     WORD n1, i, y2, y4;
03157     WORD nh, tdenom, tnumer, nmod;
03158     LONG x;
03159     if ( ncmod == 0 ) return(0);          /* No modulus operation */
03160     nmod = ABS(ncmod);
03161     n1 = *na;
03162     if ( ( par & UNPACK ) != 0 ) UnPack(a,n1,&tdenom,&tnumer);
03163     else { tnumer = n1; }
03164 /*
03165     We fish out the special case that the coefficient is short as well.
03166     There is no need to make lots of calls etc
03167 */
03168     if ( ( ( par & UNPACK ) == 0 ) && nmod == 1 && ( n1 == 1 || n1 == -1 ) ) {
03169         goto simplecase;
03170     }
03171     else if ( nmod == 1 && ( n1 == 1 || n1 == -1 ) ) {
03172         if ( a[1] != 1 ) {
03173             a[1] = a[1] % cmodvec[0];
03174             if ( a[1] == 0 ) {
03175                 MesPrint("Division by zero in short modulus arithmetic");
03176                 return(-1);
03177             }
03178             y1 = 0;
03179             if ( ( AC.modinverses != 0 ) && ( ( par & NOINVERSES ) == 0 ) ) {
03180                 y1 = AC.modinverses[a[1]];
03181             }
03182             else {
03183                 GetModInverses(a[1],cmodvec[0],&y1,&y2);
03184             }
03185             x = a[0];
03186             a[0] = (x*y1) % cmodvec[0];
03187             a[1] = 1;
03188         }
03189         else {
03190 simplecase:
03191             a[0] = a[0] % cmodvec[0];
03192         }
03193         if ( a[0] == 0 ) { *na = 0; return(0); }
03194         if ( ( AC.modmode & POSNEG ) != 0 ) {
03195             if ( a[0] > (UWORD)(cmodvec[0]/2) ) {
03196                 a[0] = cmodvec[0] - a[0];
03197                 *na = -*na;
03198             }
03199         }
03200         else if ( *na < 0 ) {
03201             *na = 1; a[0] = cmodvec[0] - a[0];
03202         }
03203         return(0);
03204     }
03205     c = NumberMalloc("TakeModulus"); d = NumberMalloc("TakeModulus"); e = NumberMalloc("TakeModulus");
03206     f = NumberMalloc("TakeModulus"); g = NumberMalloc("TakeModulus"); h = NumberMalloc("TakeModulus");
03207     n1 = ABS(n1);
03208     if ( DivLong(a,tnumer,(UWORD *)cmodvec,nmod,
03209         c,&nh,a,&tnumer) ) goto ModErr;
03210     if ( tnumer == 0 ) { *na = 0; goto normalreturn; }
03211     if ( ( par & UNPACK ) == 0 ) {
03212         if ( ( AC.modmode & POSNEG ) != 0 ) {
03213             NormalModulus(a,&tnumer);
03214         }
03215         else if ( tnumer < 0 ) {
03216             SubPLon((UWORD *)cmodvec,nmod,a,-tnumer,a,&tnumer);
03217         }
03218         *na = tnumer;
03219         goto normalreturn;
03220     }
03221     if ( tdenom == 1 && a[n1] == 1 ) {
03222         if ( ( AC.modmode & POSNEG ) != 0 ) {

```

```

03223         NormalModulus(a,&tnumer);
03224     }
03225     else if ( tnumer < 0 ) {
03226         SubPLon((UWORD *)cmodvec,nmod,a,-tnumer,a,&tnumer);
03227     }
03228     *na = tnumer;
03229     i = ABS(tnumer);
03230     a += i;
03231     *a++ = 1;
03232     while ( --i > 0 ) *a++ = 0;
03233     goto normalreturn;
03234 }
03235 if ( DivLong(a+n1,tdenom,(UWORD *)cmodvec,nmod,c,&nh,a+n1,&tdenom) ) goto ModErr;
03236 if ( !tdenom ) {
03237     MLOCK(ErrorMessageLock);
03238     MesPrint("Division by zero in modulus arithmetic");
03239     if ( AP.DebugFlag ) {
03240         AO.OutSkip = 3;
03241         FiniLine();
03242         i = *na;
03243         if ( i < 0 ) i = -i;
03244         while ( --i >= 0 ) { TalToLine((UWORD)(*a++)); TokenToLine((UBYTE *)" "); }
03245         i = *na;
03246         if ( i < 0 ) i = -i;
03247         while ( --i >= 0 ) { TalToLine((UWORD)(*a++)); TokenToLine((UBYTE *)" "); }
03248         TalToLine((UWORD)(*na));
03249         AO.OutSkip = 0;
03250         FiniLine();
03251     }
03252     MUNLOCK(ErrorMessageLock);
03253     NumberFree(c,"TakeModulus"); NumberFree(d,"TakeModulus"); NumberFree(e,"TakeModulus");
03254     NumberFree(f,"TakeModulus"); NumberFree(g,"TakeModulus"); NumberFree(h,"TakeModulus");
03255     return(-1);
03256 }
03257 if ( ( AC.modinverses != 0 ) && ( ( par & NOINVERSES ) == 0 )
03258 && ( tdenom == 1 || tdenom == -1 ) ) {
03259     *d = AC.modinverses[a[n1]]; y1 = 1; y2 = tdenom;
03260     if ( MulLong(a,tnumer,d,y1,c,&y3) ) goto ModErr;
03261     if ( DivLong(c,y3,(UWORD *)cmodvec,nmod,d,&y5,a,&tdenom) ) goto ModErr;
03262     if ( y2 < 0 ) tdenom = -tdenom;
03263 }
03264 else {
03265     x2 = (UWORD *)cmodvec; x1 = c; i = nmod; while ( --i >= 0 ) *x1++ = *x2++;
03266     x1 = c; x2 = a+n1; x3 = d; x4 = e; x5 = f; x6 = g;
03267     y1 = nmod; y2 = tdenom; y4 = 0; y5 = 1; *x5 = 1;
03268     for(;;) {
03269         if ( DivLong(x1,y1,x2,y2,h,&nh,x3,&y3) ) goto ModErr;
03270         if ( MulLong(x5,y5,h,nh,x6,&y6) ) goto ModErr;
03271         if ( AddLong(x4,y4,x6,-y6,x6,&y6) ) goto ModErr;
03272         if ( !y3 ) {
03273             if ( y2 != 1 || *x2 != 1 ) {
03274                 MLOCK(ErrorMessageLock);
03275                 MesPrint("Inverse in modulus arithmetic doesn't exist");
03276                 MesPrint("Denominator and modulus are not relative prime");
03277                 MUNLOCK(ErrorMessageLock);
03278                 goto ModErr;
03279             }
03280             break;
03281         }
03282         x7 = x1; x1 = x2; y1 = y2; x2 = x3; y2 = y3; x3 = x7;
03283         x8 = x4; x4 = x5; y4 = y5; x5 = x6; y5 = y6; x6 = x8;
03284     }
03285     if ( y5 < 0 && AddLong((UWORD *)cmodvec,nmod,x5,y5,x5,&y5) ) goto ModErr;
03286     if ( MulLong(a,tnumer,x5,y5,c,&y3) ) goto ModErr;
03287     if ( DivLong(c,y3,(UWORD *)cmodvec,nmod,d,&y5,a,&tdenom) ) goto ModErr;
03288 }
03289 if ( !tdenom ) { *na = 0; goto normalreturn; }
03290 if ( ( ( AC.modmode & POSNEG ) != 0 ) && ( ( par & FROMFUNCTION ) == 0 ) ) {
03291     NormalModulus(a,&tdenom);
03292 }
03293 else if ( tdenom < 0 ) {
03294     SubPLon((UWORD *)cmodvec,nmod,a,-tdenom,a,&tdenom);
03295 }
03296 *na = tdenom;
03297 i = ABS(tdenom);
03298 a += i;
03299 *a++ = 1;
03300 while ( --i > 0 ) *a++ = 0;
03301 normalreturn:
03302     NumberFree(c,"TakeModulus"); NumberFree(d,"TakeModulus"); NumberFree(e,"TakeModulus");
03303     NumberFree(f,"TakeModulus"); NumberFree(g,"TakeModulus"); NumberFree(h,"TakeModulus");
03304     return(0);
03305 ModErr:
03306     MLOCK(ErrorMessageLock);
03307     MesCall("TakeModulus");
03308     MUNLOCK(ErrorMessageLock);
03309     NumberFree(c,"TakeModulus"); NumberFree(d,"TakeModulus"); NumberFree(e,"TakeModulus");

```

```

03310     NumberFree(f, "TakeModulus"); NumberFree(g, "TakeModulus"); NumberFree(h, "TakeModulus");
03311     SETERROR(-1)
03312 }
03313
03314 /*
03315     #] TakeModulus :
03316     #[ TakeNormalModulus : WORD TakeNormalModulus(a, na, par)
03317
03318     added by Jan [01-09-2010]
03319 */
03320
03321 WORD TakeNormalModulus (UWORD *a, WORD *na, UWORD *c, WORD nc, WORD par)
03322 {
03323     WORD n;
03324     WORD nhalfc;
03325     UWORD *halfc;
03326
03327     GETIDENTITY;
03328
03329     /* determine c/2 by right shifting */
03330     halfc = NumberMalloc("TakeNormalModulus");
03331     nhalfc=nc;
03332     WCOPY(halfc, c, nc);
03333
03334     for (n=0; n<nhalfc; n++) {
03335         halfc[n] /= 2;
03336         if (n+1<nc) halfc[n] |= c[n+1] << (BITSINWORD-1);
03337     }
03338
03339     if (halfc[nhalfc-1]==0)
03340         nhalfc--;
03341
03342     /* takes care of the number never expanding, e.g., -1(mod 100) -> 99 -> -1 */
03343     if (BigLong(a, ABS(*na), halfc, nhalfc) > 0) {
03344         TakeModulus(a, na, c, nc, par);
03345
03346         n = ABS(*na);
03347         if (BigLong(a, n, halfc, nhalfc) > 0) {
03348             SubPLon(c, nc, a, n, a, &n);
03349             *na = (*na > 0 ? -n : n);
03350         }
03351     }
03352 }
03353
03354 NumberFree(halfc, "TakeNormalModulus");
03355 return(0);
03356 }
03357
03358 /*
03359     #] TakeNormalModulus :
03360     #[ MakeModTable : WORD MakeModTable()
03361 */
03362
03363 WORD MakeModTable(void)
03364 {
03365     LONG size, i, j, n;
03366     n = ABS(AC.ncmod);
03367     if ( AC.modpowers ) {
03368         M_free(AC.modpowers, "AC.modpowers");
03369         AC.modpowers = NULL;
03370     }
03371     if ( n > 2 ) {
03372         MLOCK(ErrorMessageLock);
03373         MesPrint("&No memory for modulus generator power table");
03374         MUNLOCK(ErrorMessageLock);
03375         Terminate(-1);
03376     }
03377     if ( n == 0 ) return(0);
03378     size = (LONG) (*AC.cmod);
03379     if ( n == 2 ) size += (((LONG)AC.cmod[1])<<BITSINWORD);
03380     AC.modpowers = (UWORD *)Malloca1(size*n*sizeof(UWORD), "table for powers of modulus");
03381     if ( n == 1 ) {
03382         j = 1;
03383         for ( i = 0; i < size; i++ ) AC.modpowers[i] = 0;
03384         for ( i = 0; i < size; i++ ) {
03385             AC.modpowers[j] = (WORD)i;
03386             j *= *AC.powmod;
03387             j %= *AC.cmod;
03388         }
03389         for ( i = 2; i < size; i++ ) {
03390             if ( AC.modpowers[i] == 0 ) {
03391                 MLOCK(ErrorMessageLock);
03392                 MesPrint("&improper generator for this modulus");
03393                 MUNLOCK(ErrorMessageLock);
03394                 M_free(AC.modpowers, "AC.modpowers");
03395                 return(-1);
03396             }

```

```

03397     }
03398     AC.modpowers[1] = 0;
03399 }
03400 else {
03401     GETIDENTITY
03402     WORD nSkrat, n2;
03403     UWORD *MMskrat7 = NumberMalloc("MakeModTable"), *MMskratC = NumberMalloc("MakeModTable");
03404     *MMskratC = 1;
03405     nSkrat = 1;
03406     j = size * 2;
03407     for ( i = 0; i < j; i+=2 ) { AC.modpowers[i] = 0; AC.modpowers[i+1] = 0; }
03408     for ( i = 0; i < size; i++ ) {
03409         j = *MMskratC + (((LONG)MMskratC[1])«BITSINWORD);
03410         j *= 2;
03411         AC.modpowers[j] = (WORD)(i & WORDMASK);
03412         AC.modpowers[j+1] = (WORD)(i » BITSINWORD);
03413         MullLong((UWORD *)MMskratC,nSkrat,(UWORD *)AC.powmod,
03414             AC.npowmod,(UWORD *)MMskrat7,&n2);
03415         TakeModulus(MMskrat7,&n2,AC.cmod,AC.ncmod,NOUNPACK);
03416         *MMskratC = *MMskrat7; MMskratC[1] = MMskrat7[1]; nSkrat = n2;
03417     }
03418     NumberFree(MMskrat7,"MakeModTable"); NumberFree(MMskratC,"MakeModTable");
03419     j = size * 2;
03420     for ( i = 4; i < j; i+=2 ) {
03421         if ( AC.modpowers[i] == 0 && AC.modpowers[i+1] == 0 ) {
03422             MLOCK(ErrorMessageLock);
03423             MesPrint("&improper generator for this modulus");
03424             MUNLOCK(ErrorMessageLock);
03425             M_free(AC.modpowers,"AC.modpowers");
03426             return(-1);
03427         }
03428     }
03429     AC.modpowers[2] = AC.modpowers[3] = 0;
03430 }
03431 return(0);
03432 }
03433
03434 /*
03435     #] MakeModTable :
03436     #] RekenTerms :
03437     #[ Functions :
03438         #[ Factorial :      WORD Factorial(n,a,na)
03439
03440     Starts with only the value of fac_(0).
03441     Builds up what is needed and remembers it for the next time.
03442
03443     We have:
03444         AT.nfac: the number of the highest stored factorial
03445         AT.pfac: the array of locations in the array of stored factorials
03446         AT.factorials: the array with stored factorials
03447 */
03448
03449 int Factorial(PHEAD WORD n, UWORD *a, WORD *na)
03450 {
03451     GETBIDENTITY
03452     UWORD *b, *c;
03453     WORD nc;
03454     int i, j;
03455     LONG ii;
03456     if ( n > AT.nfac ) {
03457         if ( AT.factorials == 0 ) {
03458             AT.nfac = 0; AT.mfac = 50; AT.sfact = 400;
03459             AT.pfac = (LONG *)Mallcl1((AT.mfac+2)*sizeof(LONG),"factorials");
03460             AT.factorials = (UWORD *)Mallcl1(AT.sfact*sizeof(UWORD),"factorials");
03461             AT.factorials[0] = 1; AT.pfac[0] = 0; AT.pfac[1] = 1;
03462         }
03463         b = a;
03464         c = AT.factorials+AT.pfac[AT.nfac];
03465         nc = i = AT.pfac[AT.nfac+1] - AT.pfac[AT.nfac];
03466         while ( --i >= 0 ) *b++ = *c++;
03467         for ( j = AT.nfac+1; j <= n; j++ ) {
03468             Product(a,&nc,j);
03469             if ( nc > AM.MaxTal ) {
03470                 MLOCK(ErrorMessageLock);
03471                 MesPrint("Overflow in factorial. MaxTal = %d",AM.MaxTal);
03472                 MesPrint("Increase MaxTerm in %s",setupfilename);
03473                 MUNLOCK(ErrorMessageLock);
03474                 return(-1);
03475             }
03476             if ( j > AT.mfac ) { /* double the pfac buffer */
03477                 LONG *p;
03478                 p = (LONG *)Mallcl1((AT.mfac*2+2)*sizeof(LONG),"factorials");
03479                 i = AT.mfac;
03480                 for ( i = AT.mfac+1; i >= 0; i-- ) p[i] = AT.pfac[i];
03481                 M_free(AT.pfac,"factorial offsets"); AT.pfac = p; AT.mfac *= 2;
03482             }
03483             if ( AT.pfac[j] + nc >= AT.sfact ) { /* double the factorials buffer */

```

```

03484         UWORD *f;
03485         f = (UWORD *)Malloca(AT.sfact*2*sizeof(UWORD),"factorials");
03486         ii = AT.sfact;
03487         c = AT.factorials; b = f;
03488         while ( --ii >= 0 ) *b++ = *c++;
03489         M_free(AT.factorials,"factorials");
03490         AT.factorials = f;
03491         AT.sfact *= 2;
03492     }
03493     b = a; c = AT.factorials + AT.pfac[j]; i = nc;
03494     while ( --i >= 0 ) *c++ = *b++;
03495     AT.pfac[j+1] = AT.pfac[j] + nc;
03496 }
03497 *na = nc;
03498 AT.nfac = n;
03499 }
03500 else if ( n == 0 ) {
03501     *a = 1; *na = 1;
03502 }
03503 else {
03504     *na = i = AT.pfac[n+1] - AT.pfac[n];
03505     b = AT.factorials + AT.pfac[n];
03506     while ( --i >= 0 ) *a++ = *b++;
03507 }
03508 return(0);
03509 }
03510
03511 /*
03512     #] Factorial :
03513     #[ Bernoulli :      WORD Bernoulli(n,a,na)
03514
03515     Starts with only the value of bernoulli_(0).
03516     Builds up what is needed and remembers it for the next time.
03517     b_0 = 1
03518     (n+1)*b_n = -b_{n-1}-sum_{i,1,n-1,b_i*b_{n-i}}
03519     The n-1 plays only a role for b_2.
03520     We have hard coded b_0,b_1,b_2 and b_odd. After that:
03521     (2n+1)*b_2n = -sum_{i,1,n-1,b_2i*b_{2n-2i}}
03522
03523     We have:
03524     AT.nBer: the number of the highest stored Bernoulli number
03525     AT.pBer: the array of locations in the array of stored Bernoulli numbers
03526     AT.bernoullis: the array with stored Bernoulli numbers
03527 */
03528
03529 int Bernoulli(WORD n, UWORD *a, WORD *na)
03530 {
03531     GETIDENTITY
03532     UWORD *b, *c, *scrib, *ntop, *ntop1;
03533     WORD i, i1, i2, nhalf, nqua, nscrib, nntop, nntop1, *oldworkpointer;
03534     UWORD twee = 2, twonplus1;
03535     int j;
03536     LONG ii;
03537     if ( n <= 1 ) {
03538         if ( n == 0 ) { a[0] = a[1] = 1; *na = 3; }
03539         else if ( n == 1 ) { a[0] = 1; a[1] = 2; *na = 3; }
03540         return(0);
03541     }
03542     if ( ( n & 1 ) != 0 ) { a[0] = a[1] = 0; *na = 0; return(0); }
03543     nhalf = n/2;
03544     if ( nhalf > AT.nBer ) {
03545         oldworkpointer = AT.WorkPointer;
03546         if ( AT.bernoullis == 0 ) {
03547             AT.nBer = 1; AT.mBer = 50; AT.sBer = 400;
03548             AT.pBer = (LONG *)Malloca((AT.mBer+2)*sizeof(LONG),"bernoullis");
03549             AT.bernoullis = (UWORD *)Malloca(AT.sBer*sizeof(UWORD),"bernoullis");
03550             AT.pBer[1] = 0; AT.pBer[2] = 3;
03551             AT.bernoullis[0] = 3; AT.bernoullis[1] = 1; AT.bernoullis[2] = 12;
03552             if ( nhalf == 1 ) {
03553                 a[0] = 1; a[1] = 12; *na = 3; return(0);
03554             }
03555         }
03556         while ( nhalf > AT.mBer ) {
03557             LONG *p;
03558             p = (LONG *)Malloca((AT.mBer*2+1)*sizeof(LONG),"bernoullis");
03559             i = AT.mBer;
03560             for ( i = AT.mBer; i >= 0; i-- ) p[i] = AT.pBer[i];
03561             M_free(AT.pBer,"factorial pointers"); AT.pBer = p; AT.mBer *= 2;
03562         }
03563         for ( n = AT.nBer+1; n <= nhalf; n++ ) {
03564             scrib = (UWORD *) (AT.WorkPointer);
03565             nqua = n/2;
03566             if ( ( n & 1 ) == 1 ) {
03567                 nscrib = 0; ntop = scrib;
03568             }
03569             else {
03570                 b = AT.bernoullis + AT.pBer[nqua];

```



```

03571         nscrib = *b++;
03572         i = (WORD) (REDLENG(nscrib));
03573         MulRat(BHEAD b,i,b,i,scrib,&nscrib);
03574         ntop = scrib + 2*nscrib;
03575         nqua--;
03576     }
03577     for ( j = 1; j <= nqua; j++ ) {
03578         b = AT.bernoullis + AT.pBer[j];
03579         c = AT.bernoullis + AT.pBer[n-j];
03580         i1 = (WORD) (*b); i2 = (WORD) (*c);
03581         i1 = REDLENG(i1);
03582         i2 = REDLENG(i2);
03583         MulRat(BHEAD b+1,i1,c+1,i2,ntop,&nntop);
03584         Mullt(BHEAD ntop,&nntop,&twee,1);
03585         if ( nscrib ) {
03586             i = (WORD) nntop; if ( i < 0 ) i = -i;
03587             ntop1 = ntop + 2*i;
03588             AddRat(BHEAD ntop,nntop,scrib,nscrib,ntop1,&nntop1);
03589         }
03590         else {
03591             ntop1 = ntop; nntop1 = nntop;
03592         }
03593         nscrib = i1 = (WORD) nntop1;
03594         if ( i1 < 0 ) i1 = -i1;
03595         i1 = 2*i1;
03596         for ( i = 0; i < i1; i++ ) scrib[i] = ntop1[i];
03597         ntop = scrib + i1;
03598     }
03599     twonplus1 = 2*n+1;
03600     Divvy(BHEAD scrib,&nscrib,&twonplus1,-1);
03601     i1 = INCLENG(nscrib);
03602     i2 = i1; if ( i2 < 0 ) i2 = -i2;
03603     i = (WORD) (AT.bernoullis[AT.pBer[n-1]]);
03604     if ( i < 0 ) i = -i;
03605     AT.pBer[n] = AT.pBer[n-1]+i;
03606     if ( AT.pBer[n] + i2 >= AT.sBer ) {
03607         UWORD *f;
03608         f = (UWORD *) Malloc1(AT.sBer*2*sizeof(UWORD), "bernoullis");
03609         ii = AT.sBer;
03610         c = AT.bernoullis; b = f;
03611         while ( --ii >= 0 ) *b++ = *c++;
03612         M_free(AT.bernoullis, "bernoullis");
03613         AT.bernoullis = f;
03614         AT.sBer *= 2;
03615     }
03616     c = AT.bernoullis + AT.pBer[n]; b = scrib;
03617     *c++ = i1;
03618     for ( i = 1; i < i2; i++ ) *c++ = *b++;
03619 }
03620 AT.nBer = nhalf;
03621 AT.WorkPointer = oldworkpointer;
03622 }
03623 b = AT.bernoullis + AT.pBer[nhalf];
03624 *na = i = (WORD) (*b++);
03625 if ( i < 0 ) i = -i;
03626 i--;
03627 while ( --i >= 0 ) *a++ = *b++;
03628 return(0);
03629 }
03630
03631 /*
03632     #] Bernoulli :
03633     #[ NextPrime :
03634 */
03646 #if ( BITSINWORD == 32 )
03647 void StartPrimeList(PHEAD0)
03648 {
03650     int i, j;
03651     AR.PrimeList[AR.numinprimelist++] = 3;
03652     for ( i = 5; i < 46340; i += 2 ) {
03653         for ( j = 0; j < AR.numinprimelist && AR.PrimeList[j]*AR.PrimeList[j] <= i; j++ ) {
03654             if ( i % AR.PrimeList[j] == 0 ) goto nexti;
03655         }
03656         AR.PrimeList[AR.numinprimelist++] = i;
03657     nexti;
03658     }
03659     AR.notfirstprime = 1;
03660 }
03661
03662 #endif
03663
03664 WORD NextPrime(PHEAD WORD num)
03665 {
03666     int i, j;
03667     WORD *newpl;
03668     LONG newsize, x;

```

```

03669 #if ( BITSINWORD == 32 )
03670     if ( AR.notfirstprime == 0 ) StartPrimeList(BHEAD0);
03671 #endif
03672     if ( num > AT.inprimelist ) {
03673         while ( AT.inprimelist < num ) {
03674             if ( num >= AT.sizeprimelist ) {
03675                 if ( AT.sizeprimelist == 0 ) newsize = 32;
03676                 else newsize = 2*AT.sizeprimelist;
03677                 while ( num >= newsize ) newsize = newsize*2;
03678                 newpl = (WORD *)Malloc1(newsize*sizeof(WORD), "NextPrime");
03679                 for ( i = 0; i < AT.sizeprimelist; i++ ) {
03680                     newpl[i] = AT.primelist[i];
03681                 }
03682                 if ( AT.sizeprimelist > 0 ) {
03683                     M_free(AT.primelist, "NextPrime");
03684                 }
03685                 AT.sizeprimelist = newsize;
03686                 AT.primelist = newpl;
03687             }
03688             if ( AT.inprimelist < 0 ) { i = MAXPOSITIVE; }
03689             else { i = AT.primelist[AT.inprimelist]; }
03690             while ( i > MAXPOWER ) {
03691                 i -= 2; x = i;
03692             }
03693             #if ( BITSINWORD == 32 )
03694                 for ( j = 0; j < AR.numinprimelist && AR.PrimeList[j]*(LONG)(AR.PrimeList[j]) <= x;
03695                     j++ ) {
03696                     if ( x % AR.PrimeList[j] == 0 ) goto nexti;
03697                 }
03698             #else
03699                 for ( j = 3; j*((LONG)j) <= x; j += 2 ) {
03700                     if ( x % j == 0 ) goto nexti;
03701                 }
03702             #endif
03703             AT.inprimelist++;
03704             AT.primelist[AT.inprimelist] = i;
03705             break;
03706         }
03707         nexti;
03708     }
03709     if ( i < MAXPOWER ) {
03710         MLOCK(ErrorMessageLock);
03711         MesPrint("There are not enough short prime numbers for this calculation");
03712         MesPrint("Try to use a computer with a %d-bits architecture",
03713             (int)(BITSINWORD*4));
03714         MUNLOCK(ErrorMessageLock);
03715         Terminate(-1);
03716     }
03717 }
03718 return(AT.primelist[num]);
03719 }
03720 /*
03721 #] NextPrime :
03722 #[ Moebius :
03723 Returns the value of the Moebius function if n fits inside
03724 a WORD.
03725 The method we use is a bit like the sieve of Erathostenes.
03726 */
03727
03728 WORD Moebius(PHEAD WORD nn)
03729 {
03730     WORD i, n = nn, x;
03731     LONG newsize;
03732     SBYTE *newtable, mu;
03733     #if ( BITSINWORD == 32 )
03734         if ( AR.notfirstprime == 0 ) StartPrimeList(BHEAD0);
03735     #endif
03736     /*
03737     First we make sure that:
03738     a: the table is big enough.
03739     b: the number is not already in the table.
03740     */
03741     if ( nn >= AR.moebiustablesz ) {
03742         if ( AR.moebiustablesz <= 0 ) { newsize = (LONG)nn + 20; }
03743         else { newsize = (LONG)nn*2; }
03744         if ( newsize > MAXPOSITIVE ) newsize = MAXPOSITIVE;
03745         newtable = (SBYTE *)Malloc1(newsize*sizeof(SBYTE), "Moebius");
03746         for ( i = 0; i < AR.moebiustablesz; i++ ) newtable[i] = AR.moebiustable[i];
03747         for ( ; i < newsize; i++ ) newtable[i] = 2;
03748         if ( AR.moebiustablesz > 0 ) M_free(AR.moebiustable, "Moebius");
03749         AR.moebiustable = newtable;
03750         AR.moebiustablesz = newsize;
03751     }
03752     /* NOTE: nn == MAXPOSITIVE never fits in moebiustable. */
03753     if ( nn != MAXPOSITIVE && AR.moebiustable[nn] != 2 ) return((WORD)AR.moebiustable[nn]);
03754     mu = 1;

```

```

03755     if ( n == 1 ) goto putvalue;
03756     if ( n % 2 == 0 ) {
03757         n /= 2;
03758         if ( n % 2 == 0 ) { mu = 0; goto putvalue; }
03759         if ( AR.moebiustable[n] != 2 ) { mu = -AR.moebiustable[n]; goto putvalue; }
03760         mu = -mu;
03761         if ( n == 1 ) goto putvalue;
03762     }
03763     #if ( BITSINWORD == 32 )
03764     for ( i = 0; i < AR.numinprimelist; i++ ) {
03765         x = AR.PrimeList[i];
03766     #else
03767     for ( x = 3; x < MAXPOSITIVE; x += 2 ) {
03768     #endif
03769         if ( n % x == 0 ) {
03770             n /= x;
03771             if ( n % x == 0 ) { mu = 0; goto putvalue; }
03772             if ( AR.moebiustable[n] != 2 ) { mu = -AR.moebiustable[n]; goto putvalue; }
03773             mu = -mu;
03774             if ( n == 1 ) goto putvalue;
03775         }
03776         if ( n < x*x ) break; /* notice that x*x always fits inside a WORD */
03777     }
03778     mu = -mu;
03779     putvalue:
03780     if ( nn != MAXPOSITIVE ) AR.moebiustable[nn] = mu;
03781     return((WORD)mu);
03782 }
03783
03784 /*
03785     #] Moebius :
03786     #[ wranf :
03787
03788     A random number generator that generates random WORDs with a very
03789     long sequence. It is based on the Knuth generator.
03790
03791     We take some care that each thread can run its own, but each
03792     uses its own startup. Hence the seed includes the identity of
03793     the thread.
03794
03795     For NPAIR1, NPAIR2 we can use any pair from the table on page 28.
03796     Candidates are 24,55 (the example on the pages 171,172)
03797     or (33,97) or (38,89)
03798     These values are defined in fsizes.h and used in startup.c and threads.c
03799 */
03800
03801 #define WARMUP 6
03802
03803 static void wranfnew(PHEAD0)
03804 {
03805     int i;
03806     LONG j;
03807     for ( i = 0; i < AR.wranfnpair1; i++ ) {
03808         j = AR.wranfia[i] - AR.wranfia[i+(AR.wranfnpair2-AR.wranfnpair1)];
03809         if ( j < 0 ) j += (LONG)1 « (2*BITSINWORD-2);
03810         AR.wranfia[i] = j;
03811     }
03812     for ( i = AR.wranfnpair1; i < AR.wranfnpair2; i++ ) {
03813         j = AR.wranfia[i] - AR.wranfia[i-AR.wranfnpair1];
03814         if ( j < 0 ) j += (LONG)1 « (2*BITSINWORD-2);
03815         AR.wranfia[i] = j;
03816     }
03817 }
03818
03819 void iniwranf(PHEAD0)
03820 {
03821     int imax = AR.wranfnpair2-1;
03822     ULONG i, ii, seed = AR.wranfseed;
03823     LONG j, k;
03824     ULONG offset = 12345;
03825     #ifdef PARALLELCODE
03826     int id;
03827     #if defined(WITHPTHREADS)
03828     id = AT.identity;
03829     #elif defined(WITHMPI)
03830     id = PF.me;
03831     #endif
03832     seed += id;
03833     i = id + 1;
03834     if ( i > 1 ) {
03835         ULONG pow, accu;
03836         pow = offset; accu = 1;
03837         while ( i ) {
03838             if ( ( i & 1 ) != 0 ) accu *= pow;
03839             i /= 2; pow = pow*pow;
03840         }
03841         offset = accu;

```

```

03842     }
03843 #endif
03844     if ( seed < ((LONG)1«(BITSINWORD-1)) ) {
03845         j = ( (seed+31459L) « (BITSINWORD-2))+offset;
03846     }
03847     else if ( seed < ((LONG)1«(BITSINWORD+10-1)) ) {
03848         j = ( (seed+31459L) « (BITSINWORD-10-2))+offset;
03849     }
03850     else {
03851         j = ( (seed+31459L) « 1)+offset;
03852     }
03853     if ( ( seed & 1 ) == 1 ) seed++;
03854     j += seed;
03855     AR.wranfia[imax] = j;
03856     k = 1;
03857     for ( i = 0; i <= (ULONG)(imax); i++ ) {
03858         ii = (AR.wranfnpair1*i)%AR.wranfnpair2;
03859         AR.wranfia[ii] = k;
03860         k = ULongToLong((ULONG)j - (ULONG)k);
03861         if ( k < 0 ) k += (LONG)1 « (2*BITSINWORD-2);
03862         j = AR.wranfia[ii];
03863     }
03864     for ( i = 0; i < WARMUP; i++ ) wranfnew(BHEAD0);
03865     AR.wranfcall = 0;
03866 }
03867
03868 UWORD wranf(PHEAD0)
03869 {
03870     UWORD wval;
03871     if ( AR.wranfia == 0 ) {
03872         AR.wranfia = (ULONG *)Malloc1(AR.wranfnpair2*sizeof(ULONG),"wranf");
03873         iniwranf(BHEAD0);
03874     }
03875     if ( AR.wranfcall >= AR.wranfnpair2 ) {
03876         wranfnew(BHEAD0);
03877         AR.wranfcall = 0;
03878     }
03879     wval = (UWORD)(AR.wranfia[AR.wranfcall++])«(BITSINWORD-1));
03880     return(wval);
03881 }
03882
03883 /*
03884 Returns a random UWORD in the range (0,...,imax-1)
03885 */
03886
03887 UWORD iranf(PHEAD UWORD imax)
03888 {
03889     UWORD i;
03890     ULONG x, xmax;
03891     if (imax < 2) return 0;
03892     x = (LONG)1 « BITSINWORD;
03893     xmax = x - x%imax;
03894     while ( ( i = wranf(BHEAD0) ) >= xmax ) {}
03895     return(i%imax);
03896 }
03897
03898 /*
03899 #] wranf :
03900 #[ PreRandom :
03901
03902 The random number generator of the preprocessor.
03903 This one is completely different from the execution time generator
03904 random_(number). In the preprocessor we generate a floating point
03905 number in a string according to a distribution.
03906 Currently allowed are:
03907     RANDOM_(log,min,max)
03908     RANDOM_(lin,min,max)
03909 The return value is a string with the floating point number.
03910 */
03911
03912 UBYTE *PreRandom(UBYTE *s)
03913 {
03914     GETIDENTITY
03915     UBYTE *mode,*mins = 0,*maxs = 0, *outval;
03916     float num;
03917     double minval, maxval, value = 0;
03918     int linlog = -1;
03919     mode = s;
03920     while ( FG.cTable[*s] <= 1 ) s++;
03921     if ( *s == ',' ) { *s = 0; s++; }
03922     mins = s;
03923     while ( *s && *s != ',' ) s++;
03924     if ( *s == ',' ) { *s = 0; s++; }
03925     maxs = s;
03926     while ( *s && *s != ',' ) s++;
03927     if ( *s || *maxs == 0 || *mins == 0 ) {
03928         MesPrint("@Illegal arguments in macro RANDOM_");

```

```

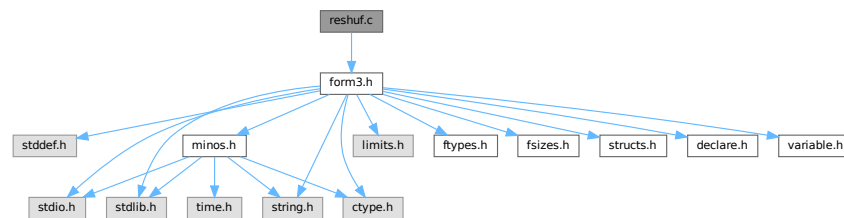
03929         Terminate(-1);
03930     }
03931     if ( StrICmp(mode, (UBYTE *)"lin") == 0 ) {
03932         linlog = 0;
03933     }
03934     else if ( StrICmp(mode, (UBYTE *)"log") == 0 ) {
03935         linlog = 1;
03936     }
03937     else {
03938         MesPrint("@Illegal mode argument in macro RANDOM_");
03939         Terminate(-1);
03940     }
03941
03942     sscanf((char *)mins, "%f", &num); minval = num;
03943     sscanf((char *)maxs, "%f", &num); maxval = num;
03944
03945     /*
03946     * Note on ParFORM: we should use the same random number on all the
03947     * processes in the complication phase. The random number is generated
03948     * on the master and broadcast to the other processes.
03949     */
03950     {
03951         UWORD x;
03952         double xx;
03953 #ifdef WITHMPI
03954         x = 0;
03955         if ( PF.me == MASTER ) {
03956             x = wranf(BHEAD0);
03957         }
03958         x = (UWORD)PF_BroadcastNumber((LONG)x);
03959 #else
03960         x = wranf(BHEAD0);
03961 #endif
03962         xx = x/pow(2.0, (double) (BITSINWORD-1));
03963         if ( linlog == 0 ) {
03964             value = minval + (maxval-minval)*xx;
03965         }
03966         else if ( linlog == 1 ) {
03967             value = minval * pow(maxval/minval, xx);
03968         }
03969     }
03970
03971     outval = (UBYTE *)Malloc1(64, "PreRandom");
03972     if ( ABS(value) < 0.00001 || ABS(value) > 1000000. ) {
03973         snprintf((char *)outval, 64, "%e", value);
03974     }
03975     else if ( ABS(value) < 0.0001 ) { snprintf((char *)outval, 64, "%10f", value); }
03976     else if ( ABS(value) < 0.001 ) { snprintf((char *)outval, 64, "%9f", value); }
03977     else if ( ABS(value) < 0.01 ) { snprintf((char *)outval, 64, "%8f", value); }
03978     else if ( ABS(value) < 0.1 ) { snprintf((char *)outval, 64, "%7f", value); }
03979     else if ( ABS(value) < 1. ) { snprintf((char *)outval, 64, "%6f", value); }
03980     else if ( ABS(value) < 10. ) { snprintf((char *)outval, 64, "%5f", value); }
03981     else if ( ABS(value) < 100. ) { snprintf((char *)outval, 64, "%4f", value); }
03982     else if ( ABS(value) < 1000. ) { snprintf((char *)outval, 64, "%3f", value); }
03983     else if ( ABS(value) < 10000. ) { snprintf((char *)outval, 64, "%2f", value); }
03984     else { snprintf((char *)outval, 64, "%1f", value); }
03985     return(outval);
03986 }
03987
03988 /*
03989     #] PreRandom :
03990     #] Functions :
03991 */

```

4.120 reshuf.c File Reference

```
#include "form3.h"
```

Include dependency graph for reshuf.c:



Functions

- WORD [ReNumber](#) (PHEAD WORD *term)
- void [FunLevel](#) (PHEAD WORD *term)
- WORD [DetCurDum](#) (PHEAD WORD *t)
- int [FullRenumber](#) (PHEAD WORD *term, WORD par)
- void [MoveDummies](#) (PHEAD WORD *term, WORD shift)
- void [AdjustRenumScratch](#) (PHEAD0)
- WORD [CountDo](#) (WORD *term, WORD *instruct)
- WORD [CountFun](#) (WORD *term, WORD *countfun)
- WORD [DimensionSubterm](#) (WORD *subterm)
- WORD [DimensionTerm](#) (WORD *term)
- WORD [DimensionExpression](#) (PHEAD WORD *expr)
- WORD [MultDo](#) (PHEAD WORD *term, WORD *pattern)
- WORD [TryDo](#) (PHEAD WORD *term, WORD *pattern, WORD level)
- WORD [DoDistrib](#) (PHEAD WORD *term, WORD level)
- WORD [EqualArg](#) (WORD *parms, WORD num1, WORD num2)
- WORD [DoDelta3](#) (PHEAD WORD *term, WORD level)
- WORD [TestPartitions](#) (WORD *tfun, [PARTI](#) *parti)
- WORD [DoPartitions](#) (PHEAD WORD *term, WORD level)
- WORD [DoPermutations](#) (PHEAD WORD *term, WORD level)
- WORD [DoShuffle](#) (WORD *term, WORD level, WORD fun, WORD option)
- int [Shuffle](#) (WORD *from1, WORD *from2, WORD *to)
- int [FinishShuffle](#) (WORD *fini)
- WORD [DoShuffle](#) (WORD *term, WORD level, WORD fun, WORD option)
- int [Shuffle](#) (WORD *from1, WORD *from2, WORD *to)
- int [FinishShuffle](#) (WORD *fini)
- WORD * [StuffRootAdd](#) (WORD *t1, WORD *t2, WORD *to)

4.120.1 Detailed Description

Mixed routines: Routines for relabelling dummy indices. The multiply command The distrib_ function The tryreplace statement

Definition in file [reshuf.c](#).

4.120.2 Macro Definition Documentation

4.120.2.1 NEWCODE

```
#define NEWCODE
```

Definition at line 35 of file [reshuf.c](#).

4.120.3 Function Documentation

4.120.3.1 ReNumber()

```
WORD ReNumber (
    PHEAD WORD * term )
```

Definition at line 80 of file [reshuf.c](#).

4.120.3.2 FunLevel()

```
void FunLevel (
    PHEAD WORD * term )
```

Definition at line 130 of file [reshuf.c](#).

4.120.3.3 DetCurDum()

```
WORD DetCurDum (
    PHEAD WORD * t )
```

Definition at line 252 of file [reshuf.c](#).

4.120.3.4 FullRenumber()

```
int FullRenumber (
    PHEAD WORD * term,
    WORD par )
```

Definition at line 324 of file [reshuf.c](#).

4.120.3.5 MoveDummies()

```
void MoveDummies (
    PHEAD WORD * term,
    WORD shift )
```

Definition at line 436 of file [reshuf.c](#).

4.120.3.6 AdjustRenumScratch()

```
void AdjustRenumScratch (
    PHEAD0 )
```

Definition at line 506 of file [reshuf.c](#).

4.120.3.7 CountDo()

```
WORD CountDo (
    WORD * term,
    WORD * instruct )
```

Definition at line 552 of file [reshuf.c](#).

4.120.3.8 CountFun()

```
WORD CountFun (
    WORD * term,
    WORD * countfun )
```

Definition at line 706 of file [reshuf.c](#).

4.120.3.9 DimensionSubterm()

```
WORD DimensionSubterm (
    WORD * subterm )
```

Definition at line 867 of file [reshuf.c](#).

4.120.3.10 DimensionTerm()

```
WORD DimensionTerm (
    WORD * term )
```

Definition at line 968 of file [reshuf.c](#).

4.120.3.11 DimensionExpression()

```
WORD DimensionExpression (
    PHEAD WORD * expr )
```

Definition at line 1000 of file [reshuf.c](#).

4.120.3.12 MultDo()

```
WORD MultDo (  
    PHEAD WORD * term,  
    WORD * pattern )
```

Definition at line [1044](#) of file [reshuf.c](#).

4.120.3.13 TryDo()

```
WORD TryDo (  
    PHEAD WORD * term,  
    WORD * pattern,  
    WORD level )
```

Definition at line [1072](#) of file [reshuf.c](#).

4.120.3.14 DoDistrib()

```
WORD DoDistrib (  
    PHEAD WORD * term,  
    WORD level )
```

Definition at line [1119](#) of file [reshuf.c](#).

4.120.3.15 EqualArg()

```
WORD EqualArg (  
    WORD * parms,  
    WORD num1,  
    WORD num2 )
```

Definition at line [1379](#) of file [reshuf.c](#).

4.120.3.16 DoDelta3()

```
WORD DoDelta3 (  
    PHEAD WORD * term,  
    WORD level )
```

Definition at line [1405](#) of file [reshuf.c](#).

4.120.3.17 TestPartitions()

```
WORD TestPartitions (  
    WORD * tfun,  
    PARTI * parti )
```

Definition at line [1607](#) of file [reshuf.c](#).

4.120.3.18 DoPartitions()

```
WORD DoPartitions (  
    PHEAD WORD * term,  
    WORD level )
```

Definition at line 1722 of file [reshuf.c](#).

4.120.3.19 DoPermutations()

```
WORD DoPermutations (  
    PHEAD WORD * term,  
    WORD level )
```

Definition at line 2007 of file [reshuf.c](#).

4.120.3.20 DoShuffle()

```
WORD DoShuffle (  
    WORD * term,  
    WORD level,  
    WORD fun,  
    WORD option )
```

Definition at line 2095 of file [reshuf.c](#).

4.120.3.21 Shuffle()

```
int Shuffle (  
    WORD * from1,  
    WORD * from2,  
    WORD * to )
```

Definition at line 2229 of file [reshuf.c](#).

4.120.3.22 FinishShuffle()

```
int FinishShuffle (  
    WORD * fini )
```

Definition at line 2421 of file [reshuf.c](#).

4.120.3.23 DoStuffle()

```
WORD DoStuffle (  
    WORD * term,  
    WORD level,  
    WORD fun,  
    WORD option )
```

Definition at line 2487 of file [reshuf.c](#).

4.120.3.24 Stuffle()

```
int Stuffle (
    WORD * from1,
    WORD * from2,
    WORD * to )
```

Definition at line [2702](#) of file [reshuf.c](#).

4.120.3.25 FinishStuffle()

```
int FinishStuffle (
    WORD * fini )
```

Definition at line [2829](#) of file [reshuf.c](#).

4.120.3.26 StuffRootAdd()

```
WORD * StuffRootAdd (
    WORD * t1,
    WORD * t2,
    WORD * to )
```

Definition at line [2876](#) of file [reshuf.c](#).

4.121 reshuf.c

[Go to the documentation of this file.](#)

```
00001
00009 /* #[ License : */
00010 /*
00011 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00012 * When using this file you are requested to refer to the publication
00013 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00014 * This is considered a matter of courtesy as the development was paid
00015 * for by FOM the Dutch physics granting agency and we would like to
00016 * be able to track its scientific use to convince FOM of its value
00017 * for the community.
00018 *
00019 * This file is part of FORM.
00020 *
00021 * FORM is free software: you can redistribute it and/or modify it under the
00022 * terms of the GNU General Public License as published by the Free Software
00023 * Foundation, either version 3 of the License, or (at your option) any later
00024 * version.
00025 *
00026 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00027 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00028 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00029 * details.
00030 *
00031 * You should have received a copy of the GNU General Public License along
00032 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00033 */
00034 /* #[ License : */
00035 #define NEWCODE
00036 /*
00037 * #[ Includes : reshuf.c
00038 */
00039
00040 #include "form3.h"
00041
00042 /*
00043 * #[ Includes :
```

```

00044     #[ Reshuf :
00045
00046     Routines to rearrange dummy indices, so that
00047     a: The notation becomes reasonably unique (the perfect job
00048         may consume very much time).
00049     b: The value of AR.CurDum is reset.
00050
00051     Also some routines are needed to aid in the reading of stored
00052     expressions. Also in those expressions there can be dummy
00053     indices, and there should be no conflict with the already
00054     existing dummies.
00055
00056     #[ ReNumber :
00057
00058     Reads the term, tests for dummies, and puts them in order.
00059     Note that this is kind of a first order approximation.
00060     There is quite some room to make this routine 'smart'
00061     First order:
00062         First index will be lowest, second will be next etc.
00063     Second order:
00064         Functions with more than one index and symmetry properties
00065         have some look ahead to see which index is the first to
00066         find its partner.
00067     Third order:
00068         Take the ordering of the functions into account.
00069     Fourth order:
00070         Try all permutations and see which one gives the 'minimal' term.
00071     Currently we use only the first order.
00072
00073     We need a scratch array for the numbers we find, and one for
00074     the addresses at which these numbers are.
00075     We can use the space for the Scrut arrays. There are 13 of those
00076     and each is AM.MaxTal UWORDS long.
00077
00078 /*
00079
00080 WORD ReNumber(PHEAD WORD *term)
00081 {
00082     GETBIDENTITY
00083     WORD *d, *e, **p, **f;
00084     WORD n, i, j, old;
00085     AN.DumFound = AN.RenumScratch;
00086     AN.DumPlace = AN.PoinScratch;
00087     AN.DumFunPlace = AN.FunScratch;
00088     AN.NumFound = 0;
00089     FunLevel(BHEAD term);
00090     d = AN.RenumScratch;
00091     p = AN.PoinScratch;
00092     f = AN.FunScratch;
00093 /*
00094     Now the first level sorting.
00095 */
00096     i = AN.IndDum;
00097     n = AN.NumFound;
00098     while ( --n >= 0 ) {
00099         if ( *d > 0 ) {
00100             old = **p;
00101             **p = ++i;
00102             if ( *f ) **f |= DIRTYSYMFLAG;
00103             e = d;
00104             e++;
00105             for ( j = 1; j <= n; j++ ) {
00106                 if ( *e && *(p[j]) == old ) {
00107                     *(p[j]) = i;
00108                     if ( f[j] ) *(f[j]) |= DIRTYSYMFLAG;
00109                     *e = 0;
00110                 }
00111                 e++;
00112             }
00113         }
00114         p++;
00115         d++;
00116         f++;
00117     }
00118     return(i);
00119 }
00120
00121 /*
00122     #[ ReNumber :
00123     #[ FunLevel :
00124
00125     Does one term in determining where the dummies are.
00126     Made to work recursively for functions.
00127
00128 */
00129
00130 void FunLevel(PHEAD WORD *term)

```

```

00131 {
00132     GETBIDENTITY
00133     WORD *t, *tstop, *r, *fun;
00134     WORD *m;
00135     t = r = term;
00136     r += *r;
00137     tstop = r - ABS(r[-1]);
00138     t++;
00139     if ( t < tstop ) do {
00140         r = t + t[1];
00141         switch ( *t ) {
00142             case SYMBOL:
00143             case DOTPRODUCT:
00144                 break;
00145             case VECTOR:
00146                 t += 3;
00147                 do {
00148                     if ( *t > AN.IndDum ) {
00149                         if ( AN.NumFound >= AN.MaxRenumScratch ) AdjustRenumScratch(BHEAD0);
00150                         AN.NumFound++;
00151                         *AN.DumFound++ = *t;
00152                         *AN.DumPlace++ = t;
00153                         *AN.DumFunPlace++ = 0;
00154                     }
00155                     t += 2;
00156                 } while ( t < r );
00157                 break;
00158             case SUBEXPRESSION:
00159 /*
00160             Still must hunt down the wildcards(?)
00161 */
00162                 break;
00163             case GAMMA:
00164                 t += FUNHEAD-2;
00165                 /* fall through */
00166             case DELTA:
00167             case INDEX:
00168                 t += 2;
00169                 while ( t < r ) {
00170                     if ( *t > AN.IndDum ) {
00171                         if ( AN.NumFound >= AN.MaxRenumScratch ) AdjustRenumScratch(BHEAD0);
00172                         AN.NumFound++;
00173                         *AN.DumFound++ = *t;
00174                         *AN.DumPlace++ = t;
00175                         *AN.DumFunPlace++ = 0;
00176                     }
00177                     t++;
00178                 }
00179                 break;
00180             case HAAKJE:
00181             case EXPRESSION:
00182             case SNUMBER:
00183             case LNUMBER:
00184                 break;
00185             default:
00186                 if ( *t < FUNCTION ) {
00187                     MLOCK(ErrorMessageLock);
00188                     MesPrint("Unexpected code in ReNumber");
00189                     MUNLOCK(ErrorMessageLock);
00190                     Terminate(-1);
00191                 }
00192                 fun = t+2;
00193                 if ( *t >= FUNCTION && functions[*t-FUNCTION].spec
00194                 >= TENSORFUNCTION ) {
00195                     t += FUNHEAD;
00196                     while ( t < r ) {
00197                         if ( *t > AN.IndDum ) {
00198                             if ( AN.NumFound >= AN.MaxRenumScratch ) AdjustRenumScratch(BHEAD0);
00199                             AN.NumFound++;
00200                             *AN.DumFound++ = *t;
00201                             *AN.DumPlace++ = t;
00202                             *AN.DumFunPlace++ = fun;
00203                         }
00204                         t++;
00205                     }
00206                     break;
00207                 }
00208                 t += FUNHEAD;
00209                 while ( t < r ) {
00210                     if ( *t > 0 ) {
00211
00212                         /* General function. Enter 'recursion'. */
00213
00214                         m = t + *t;
00215                         t += ARGHEAD;
00216                         while ( t < m ) {

```

```

00218             FunLevel(BHEAD t);
00219             t += *t;
00220         }
00221     }
00222     else {
00223         if ( *t == -INDEX ) {
00224             t++;
00225             if ( *t >= AN.IndDum ) {
00226                 if ( AN.NumFound >= AN.MaxRenumScratch ) AdjustRenumScratch(BHEAD0);
00227                 AN.NumFound++;
00228                 *AN.DumFound++ = *t;
00229                 *AN.DumPlace++ = t;
00230                 *AN.DumFunPlace++ = fun;
00231             }
00232             t++;
00233         }
00234         else if ( *t <= -FUNCTION ) t++;
00235         else t += 2;
00236     }
00237 }
00238 break;
00239 }
00240 t = r;
00241 } while ( t < tstop );
00242 }
00243
00244 /*
00245     #] FunLevel :
00246     #[ DetCurDum :
00247
00248     We look for indices in the range AM.IndDum to AM.IndDum+MAXDUMMIES.
00249     The maximum value is returned.
00250 */
00251
00252 WORD DetCurDum(PHEAD WORD *t)
00253 {
00254     GETBIDENTITY
00255     WORD maxval = AN.IndDum;
00256     WORD maxtop = AM.IndDum + WILDOFFSET;
00257     WORD *tstop, *m, *r, i;
00258     tstop = t + *t - 1;
00259     tstop -= ABS(*tstop);
00260     t++;
00261     while ( t < tstop ) {
00262         if ( *t == VECTOR ) {
00263             m = t + 3;
00264             t += t[1];
00265             while ( m < t ) {
00266                 if ( *m > maxval && *m < maxtop ) maxval = *m;
00267                 m += 2;
00268             }
00269         }
00270         else if ( *t == DELTA || *t == INDEX ) {
00271             m = t + 2;
00272         Singles:
00273             t += t[1];
00274             while ( m < t ) {
00275                 if ( *m > maxval && *m < maxtop ) maxval = *m;
00276                 m++;
00277             }
00278         }
00279         else if ( *t >= FUNCTION ) {
00280             if ( functions[*t-FUNCTION].spec >= TENSORFUNCTION ) {
00281                 m = t + FUNHEAD;
00282                 goto Singles;
00283             }
00284             r = t + FUNHEAD;
00285             t += t[1];
00286             while ( r < t ) { /* The arguments */
00287                 if ( *r < 0 ) {
00288                     if ( *r <= -FUNCTION ) r++;
00289                     else if ( *r == -INDEX ) {
00290                         if ( r[1] > maxval && r[1] < maxtop ) maxval = r[1];
00291                         r += 2;
00292                     }
00293                     else r += 2;
00294                 }
00295                 else {
00296                     m = r + ARGHEAD;
00297                     r += *r;
00298                     while ( m < r ) { /* Terms in the argument */
00299                         i = DetCurDum(BHEAD m);
00300                         if ( i > maxval && i < maxtop ) maxval = i;
00301                         m += *m;
00302                     }
00303                 }
00304             }

```

```

00305     }
00306     else {
00307         t += t[1];
00308     }
00309 }
00310 return(maxval);
00311 }
00312
00313 /*
00314     #] DetCurDum :
00315     #[ FullRenumber :
00316
00317     Does a full renumbering. May be slow if there are many indices.
00318     par = 1 Goes with a factorial!
00319     par = 0 All single exchanges only till there is no more improvement.
00320     Notice that there is a hole in the defense with respect to
00321     arguments inside functions inside functions.
00322 */
00323
00324 int FullRenumber(PHEAD WORD *term, WORD par)
00325 {
00326     GETBIDENTITY
00327     WORD *d, **p, **f, *w, *t, *best, *stac, *perm, a, *termtry;
00328     WORD n, i, j, k, ii;
00329     WORD *oldworkpointer = AT.WorkPointer;
00330     n = ReNumber(BHEAD term) - AM.IndDum;
00331     if ( n <= 1 ) return(0);
00332     Normalize(BHEAD term);
00333     if ( *term == 0 ) return(0);
00334     n = ReNumber(BHEAD term) - AM.IndDum;
00335     d = AN.RenumScratch;
00336     p = AN.PoinScratch;
00337     f = AN.FunScratch;
00338     if ( AT.WorkPointer < term + *term ) AT.WorkPointer = term + *term;
00339     k = AN.NumFound;
00340     best = w = AT.WorkPointer; t = term;
00341     for ( i = *term; i > 0; i-- ) *w++ = *t++;
00342     AT.WorkPointer = w;
00343     Normalize(BHEAD best);
00344     AT.WorkPointer = w = best + *best;
00345     stac = w+100;
00346     perm = stac + n + 1;
00347     termtry = perm + n + 1;
00348     for ( i = 1; i <= n; i++ ) perm[i] = i + AM.IndDum;
00349     for ( i = 1; i <= n; i++ ) stac[i] = i;
00350     for ( i = 0; i < k; i++ ) d[i] = *(p[i]) - AM.IndDum;
00351     if ( par == 0 ) { /* All single exchanges */
00352         for ( i = 1; i < n; i++ ) {
00353             for ( j = i+1; j <= n; j++ ) {
00354                 a = perm[j]; perm[j] = perm[i]; perm[i] = a;
00355                 for ( ii = 0; ii < k; ii++ ) {
00356                     *(p[ii]) = perm[d[ii]];
00357                     if ( f[ii] ) *(f[ii]) |= DIRTYSYMFLAG;
00358                 }
00359                 t = term; w = termtry;
00360                 for ( ii = 0; ii < *term; ii++ ) *w++ = *t++;
00361                 AT.WorkPointer = w;
00362                 if ( Normalize(BHEAD termtry) == 0 ) {
00363                     if ( *termtry == 0 ) goto Return0;
00364                     if ( ( ii = CompareTerms(BHEAD termtry,best,0) ) > 0 ) {
00365                         t = termtry; w = best;
00366                         for ( ii = 0; ii < *termtry; ii++ ) *w++ = *t++;
00367                         i = 0; break; /* restart from beginning */
00368                     }
00369                     else if ( ii == 0 && CompCoef(termtry,best) != 0 )
00370                         goto Return0;
00371                 }
00372                 /* if no success, set back */
00373                 a = perm[j]; perm[j] = perm[i]; perm[i] = a;
00374             }
00375         }
00376     }
00377     else if ( par == 1 ) { /* all permutations */
00378         j = n-1;
00379         for (;;) {
00380             if ( stac[j] == n ) {
00381                 a = perm[j]; perm[j] = perm[n]; perm[n] = a;
00382                 stac[j] = j;
00383                 j--;
00384                 if ( j <= 0 ) break;
00385                 continue;
00386             }
00387             if ( j != stac[j] ) {
00388                 a = perm[j]; perm[j] = perm[stac[j]]; perm[stac[j]] = a;
00389             }
00390             (stac[j])++;
00391             a = perm[j]; perm[j] = perm[stac[j]]; perm[stac[j]] = a;

```

```

00392     {
00393         for ( i = 0; i < k; i++ ) {
00394             *(p[i]) = perm[d[i]];
00395             if ( f[i] ) *(f[i]) |= DIRTSYMFFLAG;
00396         }
00397         t = term; w = termtry;
00398         for ( i = 0; i < *term; i++ ) *w++ = *t++;
00399         AT.WorkPointer = w;
00400         if ( Normalize(BHEAD termtry) == 0 ) {
00401             if ( *termtry == 0 ) goto Return0;
00402             if ( ( ii = CompareTerms(BHEAD termtry,best,0) ) > 0 ) {
00403                 t = termtry; w = best;
00404                 for ( i = 0; i < *termtry; i++ ) *w++ = *t++;
00405             }
00406             else if ( ii == 0 && CompCoef(termtry,best) != 0 )
00407                 goto Return0;
00408         }
00409     }
00410     if ( j < n-1 ) { j = n-1; }
00411 }
00412 }
00413 t = term; w = best;
00414 n = *best;
00415 for ( i = 0; i < n; i++ ) *t++ = *w++;
00416 AT.WorkPointer = oldworkpointer;
00417 return(0);
00418 Return0:
00419 *term = 0;
00420 AT.WorkPointer = oldworkpointer;
00421 return(0);
00422 }
00423
00424 /*
00425     #] FullRenumber :
00426     #[ MoveDummies :
00427
00428     Routine shifts the dummy indices by an amount 'shift'.
00429     It is an adaptation of DetCurDum.
00430     Needed for = ...*expression^power*...
00431     in which expression contains dummy indices.
00432     Note that this code should have been in ver1 already and has
00433     always been missing. Routine made 29-jan-2007.
00434 */
00435
00436 void MoveDummies(PHEAD WORD *term, WORD shift)
00437 {
00438     GETBIDENTITY
00439     WORD maxval = AN.IndDum;
00440     WORD maxtop = AM.IndDum + WILDOFFSET;
00441     WORD *tstop, *m, *r;
00442     tstop = term + *term - 1;
00443     tstop -= ABS(*tstop);
00444     term++;
00445     while ( term < tstop ) {
00446         if ( *term == VECTOR ) {
00447             m = term + 3;
00448             term += term[1];
00449             while ( m < term ) {
00450                 if ( *m > maxval && *m < maxtop ) *m += shift;
00451                 m += 2;
00452             }
00453         }
00454         else if ( *term == DELTA || *term == INDEX ) {
00455             m = term + 2;
00456         Singles:
00457             term += term[1];
00458             while ( m < term ) {
00459                 if ( *m > maxval && *m < maxtop ) *m += shift;
00460                 m++;
00461             }
00462         }
00463         else if ( *term >= FUNCTION ) {
00464             if ( functions[*term-FUNCTION].spec >= TENSORFUNCTION ) {
00465                 m = term + FUNHEAD;
00466                 goto Singles;
00467             }
00468             r = term + FUNHEAD;
00469             term += term[1];
00470             while ( r < term ) { /* The arguments */
00471                 if ( *r < 0 ) {
00472                     if ( *r <= -FUNCTION ) r++;
00473                     else if ( *r == -INDEX ) {
00474                         if ( r[1] > maxval && r[1] < maxtop ) r[1] += shift;
00475                         r += 2;
00476                     }
00477                     else r += 2;
00478                 }

```



```

00479         else {
00480             m = r + ARGHEAD;
00481             r += *r;
00482             while ( m < r ) { /* Terms in the argument */
00483                 MoveDummies(BHEAD m, shift);
00484                 m += *m;
00485             }
00486         }
00487     }
00488 }
00489 else {
00490     term += term[1];
00491 }
00492 }
00493 }
00494
00495 /*
00496     #] MoveDummies :
00497     #[ AdjustRenumScratch :
00498
00499     Extends the buffer for number of dummies that can be found in
00500     a term. Originally we had a fixed buffer at size 300 in the AN
00501     struct. Thomas Hahn ran out of that. Hence we have now made it
00502     a dynamical buffer.
00503     Note that the pointers used in FunLevel need adjustment as well.
00504 */
00505 void AdjustRenumScratch(PHEAD0)
00506 {
00507     GETBIDENTITY
00508     WORD newsize;
00509     int i;
00510     WORD **newpoin, *newnum;
00511     if ( AN.MaxRenumScratch == 0 ) newsize = 100;
00512     else newsize = AN.MaxRenumScratch*2;
00513     if ( newsize > MAXPOSITIVE/2 ) newsize = MAXPOSITIVE/2+1;
00514
00515     newpoin = (WORD **)Malloca1(newsize*sizeof(WORD *), "PoinScratch");
00516     for ( i = 0; i < AN.NumFound; i++ ) newpoin[i] = AN.PoinScratch[i];
00517     for ( ; i < newsize; i++ ) newpoin[i] = 0;
00518     if ( AN.PoinScratch ) M_free(AN.PoinScratch, "PoinScratch");
00519     AN.PoinScratch = newpoin;
00520     AN.DumPlace = newpoin + AN.NumFound;
00521
00522     newpoin = (WORD **)Malloca1(newsize*sizeof(WORD *), "FunScratch");
00523     for ( i = 0; i < AN.NumFound; i++ ) newpoin[i] = AN.FunScratch[i];
00524     for ( ; i < newsize; i++ ) newpoin[i] = 0;
00525     if ( AN.FunScratch ) M_free(AN.FunScratch, "FunScratch");
00526     AN.FunScratch = newpoin;
00527     AN.DumFunPlace = newpoin + AN.NumFound;
00528
00529     newnum = (WORD *)Malloca1(newsize*sizeof(WORD), "RenumScratch");
00530     for ( i = 0; i < AN.NumFound; i++ ) newnum[i] = AN.RenumScratch[i];
00531     for ( ; i < newsize; i++ ) newnum[i] = 0;
00532     if ( AN.RenumScratch ) M_free(AN.RenumScratch, "RenumScratch");
00533     AN.RenumScratch = newnum;
00534     AN.DumFound = newnum + AN.NumFound;
00535
00536     AN.MaxRenumScratch = newsize;
00537 }
00538
00539 /*
00540     #] AdjustRenumScratch :
00541     #[ Reshuf :
00542     #[ Count :
00543     #[ CountDo :
00544
00545     This function executes the counting action in a count
00546     operation. The return value is the count of the term.
00547     Input is the term and a pointer to the instruction.
00548 */
00549 WORD CountDo(WORD *term, WORD *instruct)
00550 {
00551     WORD *m, *r, i, j, count = 0;
00552     WORD *stopper, *tstop, *r1 = 0, *r2 = 0;
00553     m = instruct;
00554     stopper = m + m[1];
00555     instruct += 3;
00556     tstop = term + *term; tstop -= ABS(tstop[-1]); term++;
00557     while ( term < tstop ) {
00558         switch ( *term ) {
00559             case SYMBOL:
00560                 i = term[1] - 2;
00561                 term += 2;
00562                 while ( i > 0 ) {

```

```

00566         m = instruct;
00567         while ( m < stopper ) {
00568             if ( *m == SYMBOL && m[2] == *term ) {
00569                 count += m[3] * term[1];
00570             }
00571             m += m[1];
00572         }
00573         term += 2;
00574         i -= 2;
00575     }
00576     break;
00577 case DOTPRODUCT:
00578     i = term[1] - 2;
00579     term += 2;
00580     while ( i > 0 ) {
00581         m = instruct;
00582         while ( m < stopper ) {
00583             if ( *m == DOTPRODUCT && ( ( m[2] == *term &&
00584                 m[3] == term[1]) || ( m[2] == term[1] &&
00585                 m[3] == *term ) ) ) {
00586                 count += m[4] * term[2];
00587                 break;
00588             }
00589             m += m[1];
00590         }
00591         m = instruct;
00592         while ( m < stopper ) {
00593             if ( *m == VECTOR && m[2] == *term &&
00594                 ( m[3] & DOTPBIT ) != 0 ) {
00595                 count += m[m[1]-1] * term[2];
00596             }
00597             m += m[1];
00598         }
00599         m = instruct;
00600         while ( m < stopper ) {
00601             if ( *m == VECTOR && m[2] == term[1] &&
00602                 ( m[3] & DOTPBIT ) != 0 ) {
00603                 count += m[m[1]-1] * term[2];
00604             }
00605             m += m[1];
00606         }
00607         term += 3;
00608         i -= 3;
00609     }
00610     break;
00611 case INDEX:
00612     j = 1;
00613     goto VectInd;
00614 case VECTOR:
00615     j = 2;
00616 VectInd:
00617     i = term[1] - 2;
00618     term += 2;
00619     while ( i > 0 ) {
00620         m = instruct;
00621         while ( m < stopper ) {
00622             if ( *m == VECTOR && m[2] == *term &&
00623                 ( m[3] & VECTBIT ) != 0 ) {
00624                 count += m[m[1]-1];
00625             }
00626             m += m[1];
00627         }
00628         term += j;
00629         i -= j;
00630     }
00631     break;
00632 default:
00633     if ( *term >= FUNCTION ) {
00634         i = *term;
00635         m = instruct;
00636         while ( m < stopper ) {
00637             if ( *m == FUNCTION && m[2] == i ) count += m[3];
00638             m += m[1];
00639         }
00640         if ( functions[i-FUNCTION].spec >= TENSORFUNCTION ) {
00641             i = term[1] - FUNHEAD;
00642             term += FUNHEAD;
00643             while ( i > 0 ) {
00644                 if ( *term < 0 ) {
00645                     m = instruct;
00646                     while ( m < stopper ) {
00647                         if ( *m == VECTOR && m[2] == *term &&
00648                             ( m[3] & FUNBIT ) != 0 ) {
00649                             count += m[m[1]-1];
00650                         }
00651                         m += m[1];
00652                     }

```

```

00653             term++;
00654             i--;
00655         }
00656     }
00657     else {
00658         r = term + term[1];
00659         term += FUNHEAD;
00660         while ( term < r ) {
00661             if ( ( *term == -INDEX || *term == -VECTOR
00662                 || *term == -MINVECTOR ) && term[1] < MINSPEC ) {
00663                 m = instruct;
00664                 while ( m < stopper ) {
00665                     if ( *m == VECTOR && term[1] == m[2]
00666                         && ( m[3] & SETBIT ) != 0 ) {
00667                         r1 = SetElements + Sets[m[4]].first;
00668                         r2 = SetElements + Sets[m[4]].last;
00669                         while ( r1 < r2 ) {
00670                             if ( *r1 == i ) {
00671                                 count += m[m[1]-1];
00672                                 goto NextFF;
00673                             }
00674                             r1++;
00675                         }
00676                     }
00677                     m += m[1];
00678                 }
00679 NextFF:
00680                 term += 2;
00681             }
00682             else { NEXTARG(term) }
00683         }
00684     }
00685     break;
00686 }
00687 else {
00688     term += term[1];
00689 }
00690 break;
00691 }
00692 }
00693 return(count);
00694 }
00695
00696 /*
00697 #] CountDo :
00698 #[ CountFun :
00699
00700 This is the count function.
00701 The return value is the count of the term.
00702 Input is the term and a pointer to the count function.
00703
00704 */
00705
00706 WORD CountFun(WORD *term, WORD *countfun)
00707 {
00708     WORD *m, *r, i, j, count = 0, *instruct, *stopper, *tstop;
00709     m = countfun;
00710     stopper = m + m[1];
00711     instruct = countfun + FUNHEAD;
00712     tstop = term + *term; tstop -= ABS(tstop[-1]); term++;
00713     while ( term < tstop ) {
00714         switch ( *term ) {
00715             case SYMBOL:
00716                 i = term[1] - 2;
00717                 term += 2;
00718                 while ( i > 0 ) {
00719                     m = instruct;
00720                     while ( m < stopper ) {
00721                         if ( *m == -SNUMBER ) { NEXTARG(m) continue; }
00722                         if ( *m == -SYMBOL && m[1] == *term
00723                             && m[2] == -SNUMBER && ( m + 2 ) < stopper ) {
00724                             count += m[3] * term[1]; m += 4;
00725                         }
00726                         else { NEXTARG(m) }
00727                     }
00728                     term += 2;
00729                     i -= 2;
00730                 }
00731                 break;
00732             case DOTPRODUCT:
00733                 i = term[1] - 2;
00734                 term += 2;
00735                 while ( i > 0 ) {
00736                     m = instruct;
00737                     while ( m < stopper ) {
00738                         if ( *m == -SNUMBER ) { NEXTARG(m) continue; }
00739                         if ( *m == 9+ARGHEAD && m[ARGHEAD] == 9

```

```

00740         && m[ARGHEAD+1] == DOTPRODUCT
00741         && m[ARGHEAD+9] == -SNUMBER && ( m + ARGHEAD+9 ) < stopper
00742         && ( ( m[ARGHEAD+3] == *term &&
00743         m[ARGHEAD+4] == term[1] ) ||
00744         ( m[ARGHEAD+3] == term[1] &&
00745         m[ARGHEAD+4] == *term ) ) {
00746             count += m[ARGHEAD+10] * term[2];
00747             m += ARGHEAD+11;
00748         }
00749         else { NEXTARG(m) }
00750     }
00751     m = instruct;
00752     while ( m < stopper ) {
00753         if ( *m == -SNUMBER ) { NEXTARG(m) continue; }
00754         if ( ( *m == -VECTOR || *m == -MINVECTOR )
00755         && m[1] == *term &&
00756         m[2] == -SNUMBER && ( m+2 ) < stopper ) {
00757             count += m[3] * term[2]; m += 4;
00758         }
00759         NEXTARG(m)
00760     }
00761     m = instruct;
00762     while ( m < stopper ) {
00763         if ( *m == -SNUMBER ) { NEXTARG(m) continue; }
00764         if ( ( *m == -VECTOR || *m == -MINVECTOR )
00765         && m[1] == term[1] &&
00766         m[2] == -SNUMBER && ( m+2 ) < stopper ) {
00767             count += m[3] * term[2];
00768             m += 4;
00769         }
00770         NEXTARG(m)
00771     }
00772     term += 3;
00773     i -= 3;
00774 }
00775 break;
00776 case INDEX:
00777     j = 1;
00778     goto VectInd;
00779 case VECTOR:
00780     j = 2;
00781 VectInd:
00782     i = term[1] - 2;
00783     term += 2;
00784     while ( i > 0 ) {
00785         m = instruct;
00786         while ( m < stopper ) {
00787             if ( *m == -SNUMBER ) { NEXTARG(m) continue; }
00788             if ( ( *m == -VECTOR || *m == -MINVECTOR )
00789             && m[1] == *term &&
00790             m[2] == -SNUMBER && ( m+2 ) < stopper ) {
00791                 count += m[3]; m += 4;
00792             }
00793             NEXTARG(m)
00794         }
00795         term += j;
00796         i -= j;
00797     }
00798     break;
00799 default:
00800     if ( *term >= FUNCTION ) {
00801         i = *term;
00802         m = instruct;
00803         while ( m < stopper ) {
00804             if ( *m == -SNUMBER ) { NEXTARG(m) continue; }
00805             if ( *m == -i && m[1] == -SNUMBER && ( m+1 ) < stopper ) {
00806                 count += m[2]; m += 3;
00807             }
00808             NEXTARG(m)
00809         }
00810         if ( functions[i-FUNCTION].spec >= TENSORFUNCTION ) {
00811             i = term[1] - FUNHEAD;
00812             term += FUNHEAD;
00813             while ( i > 0 ) {
00814                 if ( *term < 0 ) {
00815                     m = instruct;
00816                     while ( m < stopper ) {
00817                         if ( *m == -SNUMBER ) { NEXTARG(m) continue; }
00818                         if ( ( *m == -VECTOR || *m == -INDEX
00819                         || *m == -MINVECTOR ) && m[1] == *term &&
00820                         m[2] == -SNUMBER && ( m+2 ) < stopper ) {
00821                             count += m[3]; m += 4;
00822                         }
00823                         else { NEXTARG(m) }
00824                     }
00825                 }
00826                 term++;
00827                 i--;

```

```

00827     }
00828     }
00829     else {
00830         r = term + term[1];
00831         term += FUNHEAD;
00832         while ( term < r ) {
00833             if ( ( *term == -INDEX || *term == -VECTOR
00834                 || *term == -MINVECTOR ) && term[1] < MINSPEC ) {
00835                 m = instruct;
00836                 while ( m < stopper ) {
00837                     if ( *m == -SNUMBER ) { NEXTARG(m) continue; }
00838                     if ( *m == -VECTOR && m[1] == term[1]
00839                         && m[2] == -SNUMBER && (m+2) < stopper ) {
00840                         count += m[3];
00841                         m += 4;
00842                     }
00843                     else { NEXTARG(m) }
00844                 }
00845                 term += 2;
00846             }
00847             else { NEXTARG(term) }
00848         }
00849     }
00850     break;
00851 }
00852 else {
00853     term += term[1];
00854 }
00855 break;
00856 }
00857 }
00858 return(count);
00859 }
00860
00861 /*
00862     #] CountFun :
00863     #] Count :
00864     #[ DimensionSubterm :
00865 */
00866
00867 WORD DimensionSubterm(WORD *subterm)
00868 {
00869     WORD *r, *rstop, dim, i;
00870     LONG x = 0;
00871     rstop = subterm + subterm[1];
00872     if ( *subterm == SYMBOL ) {
00873         r = subterm + 2;
00874         while ( r < rstop ) {
00875             if ( *r <= NumSymbols && *r > -MAXPOWER ) {
00876                 dim = symbols[*r].dimension;
00877                 if ( dim == MAXPOSITIVE ) goto undefined;
00878                 x += dim * r[1];
00879                 if ( x >= MAXPOSITIVE || x <= -MAXPOSITIVE ) goto outofrange;
00880                 r += 2;
00881             }
00882             else if ( *r <= MAXVARIABLES ) {
00883 /*
00884                 Here we have an extra symbol. Store dimension in the compiler buffer
00885 */
00886                 i = MAXVARIABLES - *r;
00887                 dim = cbuf[AM.sbufnum].dimension[i];
00888                 if ( dim == MAXPOSITIVE ) goto undefined;
00889                 if ( dim == -MAXPOSITIVE ) goto outofrange;
00890                 x += dim * r[1];
00891                 if ( x >= MAXPOSITIVE || x <= -MAXPOSITIVE ) goto outofrange;
00892                 r += 2;
00893             }
00894             else { r += 2; }
00895         }
00896     }
00897     else if ( *subterm == DOTPRODUCT ) {
00898         r = subterm + 2;
00899         while ( r < rstop ) {
00900             dim = vectors[*r-AM.OffsetVector].dimension;
00901             if ( dim == MAXPOSITIVE ) goto undefined;
00902             x += dim * r[2];
00903             if ( x >= MAXPOSITIVE || x <= -MAXPOSITIVE ) goto outofrange;
00904             dim = vectors[r[1]-AM.OffsetVector].dimension;
00905             if ( dim == MAXPOSITIVE ) goto undefined;
00906             x += dim * r[2];
00907             if ( x >= MAXPOSITIVE || x <= -MAXPOSITIVE ) goto outofrange;
00908             r += 3;
00909         }
00910     }
00911     else if ( *subterm == VECTOR ) {
00912         r = subterm + 2;
00913         while ( r < rstop ) {

```

```

00914         dim = vectors[*r-AM.OffsetVector].dimension;
00915         if ( dim == MAXPOSITIVE ) goto undefined;
00916         x += dim;
00917         if ( x >= MAXPOSITIVE || x <= -MAXPOSITIVE ) goto outofrange;
00918         r += 2;
00919     }
00920 }
00921 else if ( *subterm == INDEX ) {
00922     r = subterm + 2;
00923     while ( r < rstop ) {
00924         if ( *r < 0 ) {
00925             dim = vectors[*r-AM.OffsetVector].dimension;
00926             if ( dim == MAXPOSITIVE ) goto undefined;
00927             x += dim;
00928             if ( x >= MAXPOSITIVE || x <= -MAXPOSITIVE ) goto outofrange;
00929         }
00930         r++;
00931     }
00932 }
00933 else if ( *subterm >= FUNCTION ) {
00934     dim = functions[*subterm-FUNCTION].dimension;
00935     if ( dim == MAXPOSITIVE ) goto undefined;
00936     x += dim;
00937     if ( x >= MAXPOSITIVE || x <= -MAXPOSITIVE ) goto outofrange;
00938     if ( functions[*subterm-FUNCTION].spec > 0 ) { /* tensor */
00939         r = subterm + FUNHEAD;
00940         while ( r < rstop ) {
00941             if ( *r < MINSPEC ) {
00942                 dim = vectors[*r-AM.OffsetVector].dimension;
00943                 if ( dim == MAXPOSITIVE ) goto undefined;
00944                 x += dim;
00945                 if ( x >= MAXPOSITIVE || x <= -MAXPOSITIVE ) goto outofrange;
00946             }
00947             r++;
00948         }
00949     }
00950 }
00951 return((WORD)x);
00952 undefined:
00953     return((WORD)MAXPOSITIVE);
00954 outofrange:
00955     return(-(WORD)MAXPOSITIVE);
00956 }
00957 /*
00958 #] DimensionSubterm :
00959 #[ DimensionTerm :
00960
00961 Returns the dimension of the given term.
00962 If there is any variable of which the dimension is not defined
00963 we return the code for undefined which is MAXPOSITIVE
00964 When the value is out of range we return -MAXPOSITIVE
00965 */
00966
00967 WORD DimensionTerm(WORD *term)
00968 {
00969     WORD *t, *tstop, dim;
00970     LONG x = 0;
00971     tstop = term + *term; tstop -= ABS(tstop[-1]);
00972     t = term+1;
00973     while ( t < tstop ) {
00974         dim = DimensionSubterm(t);
00975         if ( dim == MAXPOSITIVE ) goto undefined;
00976         if ( dim == -MAXPOSITIVE ) goto outofrange;
00977         x += dim;
00978         if ( x >= MAXPOSITIVE || x <= -MAXPOSITIVE ) goto outofrange;
00979         t += t[1];
00980     }
00981     return((WORD)x);
00982 undefined:
00983     return((WORD)MAXPOSITIVE);
00984 outofrange:
00985     return(-(WORD)MAXPOSITIVE);
00986 }
00987 /*
00988 #] DimensionTerm :
00989 #[ DimensionExpression :
00990
00991 Returns the dimension of the given expression.
00992 If there is any variable of which the dimension is not defined
00993 we return the code for undefined which is MAXPOSITIVE
00994 When the value is out of range we return -MAXPOSITIVE
00995 When the value is not consistent we return -MAXPOSITIVE.
00996 */
00997
00998 WORD DimensionExpression(PHEAD WORD *expr)

```

```

01001 {
01002     WORD dim, *term, *old, x = 0;
01003     int first = 1;
01004     term = expr;
01005     while ( *term ) {
01006         dim = DimensionTerm(term);
01007         if ( dim == MAXPOSITIVE ) goto undefined;
01008         if ( dim == -MAXPOSITIVE ) goto outofrange;
01009         if ( first ) { x = dim; }
01010         else if ( x != dim ) {
01011             old = AN.currentTerm;
01012             MLOCK(ErrorMessageLock);
01013             MesPrint("Dimension is not the same in the terms of the expression");
01014             term = expr;
01015             while ( *term ) {
01016                 AN.currentTerm = term;
01017                 MesPrint("  %T");
01018             }
01019             MUNLOCK(ErrorMessageLock);
01020             AN.currentTerm = old;
01021             return(-(WORD)MAXPOSITIVE);
01022         }
01023         term += *term;
01024     }
01025     return((WORD)x);
01026 undefined:
01027     return((WORD)MAXPOSITIVE);
01028 outofrange:
01029     old = AN.currentTerm;
01030     AN.currentTerm = term;
01031     MLOCK(ErrorMessageLock);
01032     MesPrint("Dimension out of range in %t in subexpression");
01033     MUNLOCK(ErrorMessageLock);
01034     AN.currentTerm = old;
01035     return(-(WORD)MAXPOSITIVE);
01036 }
01037
01038 /*
01039  #] DimensionExpression :
01040  #[ Multiply Term :
01041  #[ MultDo :
01042  */
01043
01044 WORD MultDo(PHEAD WORD *term, WORD *pattern)
01045 {
01046     GETBIDENTITY
01047     WORD *t, *r, i;
01048     t = term + *term;
01049     if ( pattern[2] > 0 ) {          /* Left multiply */
01050         i = *term - 1;
01051     }
01052     else {                          /* Right multiply */
01053         i = ABS(t[-1]);
01054     }
01055     *term += SUBEXPSIZE;
01056     r = t + SUBEXPSIZE;
01057     do { *--r = *--t; } while ( --i > 0 );
01058     r = pattern + 3;
01059     i = r[1];
01060     while ( --i >= 0 ) *t++ = *r++;
01061     AT.WorkPointer = term + *term;
01062     return(0);
01063 }
01064
01065 /*
01066  #] MultDo :
01067  #[ Multiply Term :
01068  #[ Try Term(s) :
01069  #[ TryDo :
01070  */
01071
01072 WORD TryDo(PHEAD WORD *term, WORD *pattern, WORD level)
01073 {
01074     GETBIDENTITY
01075     WORD *t, *r, *m, i, j;
01076     ReNumber(BHEAD term);
01077     Normalize(BHEAD term);
01078     m = r = term + *term;
01079     m++;
01080     i = pattern[2];
01081     t = pattern + 3;
01082     NCOPY(m,t,i)
01083     j = *term - 1;
01084     t = term + 1;
01085     NCOPY(m,t,j)
01086     *r = WORDDIFF(m,r);
01087     AT.WorkPointer = m;

```

```

01088     if ( ( j = Normalize(BHEAD r) ) == 0 || j == 1 ) {
01089         if ( *r == 0 ) return(0);
01090         ReNumber(BHEAD r); Normalize(BHEAD r);
01091         if ( *r == 0 ) return(0);
01092         if ( ( i = CompareTerms(BHEAD term,r,0) ) < 0 ) {
01093             *AN.RepPoint = 1;
01094             AR.expchanged = 1;
01095             return(Generator(BHEAD r,level));
01096         }
01097         if ( i == 0 && CompCoef(term,r) != 0 ) { return(0); }
01098     }
01099     AT.WorkPointer = r;
01100     return(Generator(BHEAD term,level));
01101 }
01102
01103 /*
01104     #] TryDo :
01105     #] Try Term(s) :
01106     #[ Distribute :
01107     #[ DoDistrib :
01108
01109     The routine that generates the terms ordered by a distrib_ function.
01110     The presence of a replaceable distrib_ function has been sensed
01111     in the routine TestSub and has been passed on to Generator.
01112     It is then Generator that calls this function in a way that is
01113     similar to calling the trace routines, except for that for the
01114     trace routines and the Levi-Civita tensors the arguments are put
01115     in temporary storage and here we leave them inside the term,
01116     because there is no knowing how long the field will be.
01117 */
01118
01119 WORD DoDistrib(PHEAD WORD *term, WORD level)
01120 {
01121     GETBIDENTITY
01122     WORD *t, *m, *r = 0, *stop, *tstop, *termout, *endhead, *starttail, *parms;
01123     WORD i, j, k, n, nn, ntype, fun1 = 0, fun2 = 0, typ1 = 0, typ2 = 0;
01124     WORD *arg, *oldwork, *mf, ktype = 0, atype = 0;
01125     WORD sgn, dirtyflag;
01126     AN.TeInFun = AR.TePos = 0;
01127     t = term;
01128     tstop = t + *t;
01129     stop = tstop - ABS(tstop[-1]);
01130     t++;
01131     while ( t < stop ) {
01132         r = t + t[1];
01133         if ( *t == DISTRIBUTION && t[FUNHEAD] == -SNUMBER
01134             && t[FUNHEAD+1] >= -2 && t[FUNHEAD+1] <= 2
01135             && t[FUNHEAD+2] == -SNUMBER
01136             && t[FUNHEAD+4] <= -FUNCTION
01137             && t[FUNHEAD+5] <= -FUNCTION ) {
01138             WORD *ttt = t+FUNHEAD+6, *tttstop = t+t[1];
01139             while ( ttt < tttstop ) {
01140                 if ( *ttt == -DOLLEXPRESSION ) break;
01141                 NEXTARG(ttt);
01142             }
01143             if ( ttt >= tttstop ) {
01144                 fun1 = -t[FUNHEAD+4];
01145                 fun2 = -t[FUNHEAD+5];
01146                 typ1 = functions[fun1-FUNCTION].spec;
01147                 typ2 = functions[fun2-FUNCTION].spec;
01148                 if ( typ1 > 0 || typ2 > 0 ) {
01149                     m = t + FUNHEAD+6;
01150                     r = t + t[1];
01151                     while ( m < r ) {
01152                         if ( *m != -INDEX && *m != -VECTOR && *m != -MINVECTOR )
01153                             break;
01154                         m += 2;
01155                     }
01156                     if ( m < r ) {
01157                         MLOCK(ErrorMessageLock);
01158                         MesPrint("Incompatible function types and arguments in distrib_");
01159                         MUNLOCK(ErrorMessageLock);
01160                         SETERROR(-1)
01161                     }
01162                 }
01163                 break;
01164             }
01165         }
01166         t = r;
01167     }
01168     dirtyflag = t[2];
01169     ntype = t[FUNHEAD+1];
01170     n = t[FUNHEAD+3];
01171 /*
01172     t points at the distrib_ function to be expanded.
01173     fun1,fun2 and typ1,typ2 are the two functions and their types.
01174     ntype indicates the action:

```



```

01175      0: Make all possible divisions: 2^nargs
01176      1: fun1 should get n arguments: nargs! / ( n! (nargs-n)! )
01177      2: fun2 should get n arguments: nargs! / ( n! (nargs-n)! )
01178 The distinction between 1 and two is for noncommuting objects.
01179      3: fun1 should get n arguments. Super symmetric option.
01180      4: fun2 idem
01181 The super symmetric option involves:
01182      a: arguments get sorted
01183      b: identical arguments are seen as such. Hence not all their
01184         distributions are taken into account. It is as if after the
01185         distrib there is a symmetrize fun1; symmetrize fun2;
01186      c: Hence if the occurrence of each argument is a,b,c,...
01187         and their occurrence in fun1 is a1,b1,c1,... and in fun2
01188         is a2,b2,c2,... then each term is generated (a1+a2)!/a1!/a2!
01189         (b1+b2)!/b1!/b2! (c1+c2)!/c1!/c2! ... times.
01190      d: We have to make an array of occurrences and positions.
01191      e: Then we sort the arguments indirectly.
01192      f: Next we generate the argument lists in the same way as we
01193         generate powers of expressions with binomials. Hence we need
01194         a third array to keep track of the 'powers'
01195 */
01196     endhead = t;
01197     starttail = r;
01198     parms = m = t + FUNHEAD+6;
01199     i = 0;
01200     while ( m < r ) { /* Count arguments */
01201         i++;
01202         NEXTARG(m);
01203     }
01204     oldwork = AT.WorkPointer;
01205     arg = AT.WorkPointer + 1;
01206     arg[-1] = 0;
01207     termout = arg + i;
01208     switch ( ntype ) {
01209         case 0: ktype = 1; atype = n < 0 ? 1: 0; n = 0; break;
01210         case 1: ktype = 1; atype = 0; break;
01211         case 2: ktype = 0; atype = 0; break;
01212         case -1: ktype = 1; atype = 1; break;
01213         case -2: ktype = 0; atype = 1; break;
01214     }
01215     do {
01216 /*
01217         All distributions with n elements. We generate the array arg with
01218         all possible 1 and 0 patterns. 1 means in fun1 and 0 means in fun2.
01219 */
01220         if ( n > i ) return(0); /* 0 elements */
01221
01222         for ( j = 0; j < n; j++ ) arg[j] = 1;
01223         for ( j = n; j < i; j++ ) arg[j] = 0;
01224         for(;;) {
01225             sgn = 0;
01226             t = term;
01227             m = termout;
01228             while ( t < endhead ) *m++ = *t++;
01229             mf = m;
01230             *m++ = fun1;
01231             *m++ = FUNHEAD;
01232             *m++ = dirtyflag;
01233             #if FUNHEAD > 3
01234                 k = FUNHEAD -3;
01235                 while ( k-- > 0 ) *m++ = 0;
01236             #endif
01237             r = parms;
01238             for ( k = 0; k < i; k++ ) {
01239                 if ( arg[k] == ktype ) {
01240                     if ( *r <= -FUNCTION ) *m++ = *r++;
01241                     else if ( *r < 0 ) {
01242                         if ( typ1 > 0 ) {
01243                             if ( *r == -MINVECTOR ) sgn ^= 1;
01244                             r++;
01245                             *m++ = *r++;
01246                         }
01247                     } else { *m++ = *r++; *m++ = *r++; }
01248                 }
01249                 else {
01250                     nn = *r;
01251                     NCOPY(m,r,nn);
01252                 }
01253             }
01254             else { NEXTARG(r) }
01255         }
01256         mf[1] = WORDDIFF(m,mf);
01257         mf = m;
01258         *m++ = fun2;
01259         *m++ = FUNHEAD;
01260         *m++ = dirtyflag;
01261         #if FUNHEAD > 3

```

```

01262         k = FUNHEAD -3;
01263         while ( k-- > 0 ) *m++ = 0;
01264     #endif
01265     r = parms;
01266     for ( k = 0; k < i; k++ ) {
01267         if ( arg[k] != ktype ) {
01268             if ( *r <= -FUNCTION ) *m++ = *r++;
01269             else if ( *r < 0 ) {
01270                 if ( typ2 > 0 ) {
01271                     if ( *r == -MINVECTOR ) sgn ^= 1;
01272                     r++;
01273                     *m++ = *r++;
01274                 }
01275                 else { *m++ = *r++; *m++ = *r++; }
01276             }
01277             else {
01278                 nn = *r;
01279                 NCOPY(m,r,nn);
01280             }
01281         }
01282         else { NEXTARG(r) }
01283     }
01284     mf[1] = WORDDIF(m,mf);
01285     #ifndef NUOVO
01286     if ( atype == 0 ) {
01287         WORD k1,k2;
01288         for ( k = 0; k < i-1; k++ ) {
01289             if ( arg[k] == 0 ) continue;
01290             k1 = 1; k2 = k;
01291             while ( k < i-1 && EqualArg(parms,k,k+1) ) { k++; k1++; }
01292             while ( k2 <= k && arg[k2] == 1 ) k2++;
01293             k2 = k-k2+1;
01294             /*
01295             Now we need k1!/(k2! (k1-k2)!)
01296             */
01297             if ( k2 != k1 && k2 != 0 ) {
01298                 if ( GetBinom((UWORD *)m+3,m+2,k1,k2) ) {
01299                     MLOCK(ErrorMessageLock);
01300                     MesCall("DoDistrib");
01301                     MUNLOCK(ErrorMessageLock);
01302                     SETERROR(-1)
01303                 }
01304                 m[1] = ( m[2] < 0 ? -m[2]: m[2] ) + 3;
01305                 *m = LNUMBER;
01306                 m += m[1];
01307             }
01308         }
01309     }
01310     #endif
01311     r = starttail;
01312     while ( r < tstop ) *m++ = *r++;
01313
01314     if ( atype ) { /* antisymmetric field */
01315         k = n;
01316         nn = 0;
01317         for ( j = 0; j < i && k > 0; j++ ) {
01318             if ( arg[j] == 1 ) k--;
01319             else nn += k;
01320         }
01321         sgn ^= nn & 1;
01322     }
01323
01324     if ( sgn ) m[-1] = -m[-1];
01325     *termout = WORDDIF(m,termout);
01326     AT.WorkPointer = m;
01327     if ( AT.WorkPointer > AT.WorkTop ) {
01328         MLOCK(ErrorMessageLock);
01329         MesWork();
01330         MUNLOCK(ErrorMessageLock);
01331         return(-1);
01332     }
01333     *AN.RepPoint = 1;
01334     AR.expchanged = 1;
01335     if ( Generator(BHEAD termout,level) ) Terminate(-1);
01336     #ifndef NUOVO
01337     {
01338         WORD k1;
01339         j = i - 1;
01340         k = 0;
01341         redok: while ( arg[j] == 1 && j >= 0 ) { j--; k++; }
01342         while ( arg[j] == 0 && j >= 0 ) j--;
01343         if ( j < 0 ) break;
01344         k1 = j;
01345         arg[j] = 0;
01346         while ( !atype && EqualArg(parms,j,j+1) ) {
01347             j++;
01348             if ( j >= i - k - 1 ) { j = k1; k++; goto redok; }

```

```

01349         arg[j] = 0;
01350     }
01351     while ( k >= 0 ) { j++; arg[j] = 1; k--; }
01352     j++;
01353     while ( j < i ) { arg[j] = 0; j++; }
01354 }
01355 #else
01356     j = i - 1;
01357     k = 0;
01358     while ( arg[j] == 1 && j >= 0 ) { j--; k++; }
01359     while ( arg[j] == 0 && j >= 0 ) j--;
01360     if ( j < 0 ) break;
01361     arg[j] = 0;
01362     while ( k >= 0 ) { j++; arg[j] = 1; k--; }
01363     j++;
01364     while ( j < i ) { arg[j] = 0; j++; }
01365 #endif
01366 }
01367 } while ( ntype == 0 && ++n <= i );
01368 AT.WorkPointer = oldwork;
01369 return(0);
01370 }
01371
01372 /*
01373     #] DoDistrib :
01374     #[ EqualArg :
01375
01376     Returns 1 if the arguments in the field are identical.
01377 */
01378
01379 WORD EqualArg(WORD *parms, WORD num1, WORD num2)
01380 {
01381     WORD *t1, *t2;
01382     WORD i;
01383     t1 = parms;
01384     while ( --num1 >= 0 ) { NEXTARG(t1); }
01385     t2 = parms;
01386     while ( --num2 >= 0 ) { NEXTARG(t2); }
01387     if ( *t1 != *t2 ) return(0);
01388     if ( *t1 < 0 ) {
01389         if ( *t1 <= -FUNCTION || t1[1] == t2[1] ) return(1);
01390         return(0);
01391     }
01392     i = *t1;
01393     while ( --i >= 0 ) {
01394         if ( *t1 != *t2 ) return(0);
01395         t1++; t2++;
01396     }
01397     return(1);
01398 }
01399
01400 /*
01401     #] EqualArg :
01402     #[ DoDelta3 :
01403 */
01404
01405 WORD DoDelta3(PHEAD WORD *term, WORD level)
01406 {
01407     GETBIDENTITY
01408     WORD *t, *m, *m1, *m2, *stopper, *tstop, *termout, *dels, *taken;
01409     WORD *ic, *jc, *factors;
01410     WORD num, num2, i, j, k, knum, a;
01411     AN.TeInFun = AR.TePos = 0;
01412     tstop = term + *term;
01413     stopper = tstop - ABS(tstop[-1]);
01414     t = term+1;
01415     while ( ( *t != DELTA3 || ((t[1]-FUNHEAD) & 1) != 0 ) && t < stopper )
01416         t += t[1];
01417     if ( t >= stopper ) {
01418         MLOCK(ErrorMessageLock);
01419         MesPrint("Internal error with dd_ function");
01420         MUNLOCK(ErrorMessageLock);
01421         Terminate(-1);
01422     }
01423     m1 = t; m2 = t + t[1];
01424     num = t[1] - FUNHEAD;
01425     if ( num == 0 ) {
01426         termout = t = AT.WorkPointer;
01427         m = term;
01428         while ( m < m1 ) *t++ = *m++;
01429         m = m2; while ( m < tstop ) *t++ = *m++;
01430         *termout = WORDDIF(t,termout);
01431         AT.WorkPointer = t;
01432         *AN.RepPoint = 1;
01433         AR.expchanged = 1;
01434         if ( Generator(BHEAD termout,level) ) {
01435             MLOCK(ErrorMessageLock);

```

```

01436         MesCall("Do dd_");
01437         MUNLOCK(ErrorMessageLock);
01438         SETERROR(-1)
01439     }
01440     AT.WorkPointer = termout;
01441     return(0);
01442 }
01443 t += FUNHEAD;
01444 /*
01445 Step 1: sort the arguments
01446 */
01447 for ( i = 1; i < num; i++ ) {
01448     if ( t[i] < t[i-1] ) {
01449         a = t[i]; t[i] = t[i-1]; t[i-1] = a;
01450         j = i - 1;
01451         while ( j > 0 ) {
01452             if ( t[j] >= t[j-1] ) break;
01453             a = t[j]; t[j] = t[j-1]; t[j-1] = a;
01454             j--;
01455         }
01456     }
01457 }
01458 /*
01459 Step 2: Order them by occurrence
01460 In 'taken' we have the array with the number of occurrences.
01461 in 'dels' is the type of object.
01462 */
01463 m = taken = AT.WorkPointer;
01464 for ( i = 0; i < num; i++ ) *m++ = 0;
01465 dels = m; knum = 0;
01466 for ( i = 0; i < num; knum++ ) {
01467     *m++ = t[i]; i++; taken[knum] = 1;
01468     while ( i < num ) {
01469         if ( t[i] != t[i-1] ) break;
01470         i++; (taken[knum])++;
01471     }
01472 }
01473 for ( i = 0; i < knum; i++ ) *m++ = taken[i];
01474 ic = m; num2 = num/2;
01475 jc = ic + num2;
01476 factors = jc + num2;
01477 termout = factors + num2;
01478 /*
01479 The recursion has num/2 steps
01480 */
01481 k = 0;
01482 while ( k >= 0 ) {
01483     if ( k >= num2 ) {
01484         t = termout; m = term;
01485         while ( m < m1 ) *t++ = *m++;
01486         *t++ = DELTA; *t++ = num+2;
01487         for ( i = 0; i < num2; i++ ) {
01488             *t++ = dels[ic[i]]; *t++ = dels[jc[i]];
01489         }
01490         for ( i = 0; i < num2; i++ ) {
01491             if ( ic[i] == jc[i] ) {
01492                 j = 1;
01493                 while ( i < num2-1 && ic[i] == ic[i+1] && ic[i] == jc[i+1] )
01494                     { i++; j++; }
01495                 for ( a = 1; a < j; a++ ) {
01496                     *t++ = SNUMBER; *t++ = 4; *t++ = 2*a+1; *t++ = 1;
01497                 }
01498                 for ( a = 0; a+1+i < num2; a++ ) {
01499                     if ( ic[a+i] != ic[a+i+1] ) break;
01500                 }
01501                 if ( a > 0 ) {
01502                     if ( GetBinom((UWORD *) (t+3), t+2, 2*j+a, a) ) {
01503                         MLOCK(ErrorMessageLock);
01504                         MesCall("Do dd_");
01505                         MUNLOCK(ErrorMessageLock);
01506                         SETERROR(-1)
01507                     }
01508                     t[1] = ( t[2] < 0 ? -t[2]: t[2] ) + 3;
01509                     *t = LNUMBER;
01510                     t += t[1];
01511                 }
01512             }
01513             else if ( factors[i] != 1 ) {
01514                 *t++ = SNUMBER; *t++ = 4; *t++ = factors[i]; *t++ = 1;
01515             }
01516         }
01517         for ( i = 0; i < num2-1; i++ ) {
01518             if ( ic[i] == jc[i] ) continue;
01519             j = 1;
01520             while ( i < num2-1 && jc[i] == jc[i+1] && ic[i] == ic[i+1] ) {
01521                 i++; j++;
01522             }

```

```

01523         for ( a = 0; a+i < num2-1; a++ ) {
01524             if ( ic[i+a] != ic[i+a+1] ) break;
01525         }
01526         if ( a > 0 ) {
01527             if ( GetBinom((UWORD *) (t+3),t+2,j+a,a) ) {
01528                 MLOCK(ErrorMessageLock);
01529                 MesCall("Do dd_");
01530                 MUNLOCK(ErrorMessageLock);
01531                 SETERROR(-1)
01532             }
01533             t[1] = ( t[2] < 0 ? -t[2]: t[2] ) + 3;
01534             *t = LNUMBER;
01535             t += t[1];
01536         }
01537     }
01538     m = m2;
01539     while ( m < tstop ) *t++ = *m++;
01540     *termout = WORDDIFF(t,termout);
01541     AT.WorkPointer = t;
01542     *AN.RepPoint = 1;
01543     AR.expchanged = 1;
01544     if ( Generator(BHEAD termout,level) ) {
01545         MLOCK(ErrorMessageLock);
01546         MesCall("Do dd_");
01547         MUNLOCK(ErrorMessageLock);
01548         SETERROR(-1)
01549     }
01550     k--;
01551     if ( k >= 0 ) goto nextj;
01552     else break;
01553 }
01554 for ( ic[k] = 0; ic[k] < knum; ic[k]++ ) {
01555     if ( taken[ic[k]] > 0 ) break;
01556 }
01557 if ( k > 0 && ic[k-1] == ic[k] ) jc[k] = jc[k-1];
01558 else jc[k] = ic[k];
01559 for ( ; jc[k] < knum; jc[k]++ ) {
01560     if ( taken[jc[k]] <= 0 ) continue;
01561     if ( ic[k] == jc[k] ) {
01562         if ( taken[jc[k]] <= 1 ) continue;
01563     /*
01564         factors[k] = taken[ic[k]];
01565         if ( ( factors[k] & 1 ) == 0 ) (factors[k])--;
01566     */
01567         taken[ic[k]] -= 2;
01568     }
01569     else {
01570         factors[k] = taken[jc[k]];
01571         (taken[ic[k]])--; (taken[jc[k]])--;
01572     }
01573     k++;
01574     goto nextk; /* This is the simulated recursion */
01575 nextj::
01576     (taken[ic[k]])++; (taken[jc[k]])++;
01577 }
01578 k--;
01579 if ( k >= 0 ) goto nextj;
01580 nextk::
01581 }
01582 AT.WorkPointer = taken;
01583 return(0);
01584 }
01585
01586 /*
01587 #] DoDelta3 :
01588 #[ TestPartitions :
01589
01590 Checks whether the function in tfun is a partitions_ function
01591 that can be expanded. If it can a number of relevant objects is
01592 inside the struct parti.
01593 This test is not entirely trivial because there are many restrictions
01594 w.r.t. the arguments.
01595 Syntax (still to be implemented)
01596 partitions_(number_of_partition_entries,[function,number,]^nope,arguments)
01597 [function,number,]: can be
01598     f,3     for a partition of 3 arguments
01599     f,0     for the remaining arguments (should be last)
01600     num1,f,num2 with num1 effectively a number of partitions but this
01601                counts as num1 entries.
01602     0,f,num2: all partitions have num2 arguments. No number of partition
01603                entries needed. If num2 does not divide the number of
01604                arguments there will be no action.
01605 */
01606
01607 WORD TestPartitions(WORD *tfun, PARTI *parti)
01608 {
01609     WORD *tnext = tfun + tfun[1];

```

```

01610     WORD *t, *tt;
01611     WORD argcount = 0, sum = 0, i, ipart, argremain;
01612     WORD tensorflag = 0;
01613     parti->psize = parti->nfun = parti->args = parti->nargs = 0;
01614     parti->numargs = parti->numpart = parti->where = 0;
01615     tt = t = tfun + FUNHEAD;
01616     while ( t < tnext ) { argcount++; NEXTARG(t); }
01617     if ( argcount < 1 ) goto No;
01618     t = tt;
01619     if ( *t != -SNUMBER ) goto No;
01620     if ( t[1] == 0 ) {
01621         t += 2;
01622         if ( *t <= -FUNCTION && t[1] == -SNUMBER && t[2] > 0 ) {
01623             if ( functions[-*t-FUNCTION].spec > 0 ) tensorflag = 1;
01624             if ( argcount-3 < 0 ) goto No;
01625             if ( ( argcount-3 ) % t[2] != 0 ) goto No;
01626         }
01627         else goto No;
01628         parti->numpart = (argcount-3)/t[2];
01629         parti->numargs = argcount - 3;
01630         parti->psize = (WORD *)Malloc1((parti->numpart*2+parti->numargs*2+2)
01631             *sizeof(WORD),"partitions");
01632         parti->nfun = parti->psize + parti->numpart;
01633         parti->args = parti->nfun + parti->numpart;
01634         parti->nargs = parti->args + parti->numargs;
01635         for ( i = 0; i < parti->numpart; i++ ) {
01636             parti->psize[i] = t[2];
01637             parti->nfun[i] = -t[0];
01638         }
01639         t += 3;
01640     }
01641     else if ( t[1] > 0 ) { /* Number of partitions */
01642     /*
01643         We can have sequences of function,number for one partition
01644         or number1,function,number2 for number1 partitions of size number2.
01645         The last partition can have number=0. It must be a single partition
01646         and it will take all remaining arguments.
01647         If any of the functions is a tensor, all arguments must be either
01648         vector or index.
01649     */
01650         parti->numpart = t[1]; t += 2;
01651         ipart = sum = 0; argremain = argcount - 1;
01652     /*
01653         At this point is seems better to make an allocation already that
01654         may be too big. The alternative is having to pass this code twice.
01655     */
01656         parti->psize = (WORD *)Malloc1((argcount*4+2)*sizeof(WORD),"partitions");
01657         parti->nfun = parti->psize+argcount;
01658         parti->args = parti->nfun+argcount;
01659         parti->nargs = parti->args+argcount;
01660         while ( ipart < parti->numpart ) {
01661             if ( *t <= -FUNCTION && t[1] == -SNUMBER && t[2] >= 0 ) {
01662                 if ( functions[-*t-FUNCTION].spec > 0 ) tensorflag = 1;
01663                 if ( t[2] == 0 ) {
01664                     if ( ipart+1 != parti->numpart ) goto WhatAPity;
01665                     argremain -= 2;
01666                     parti->nfun[ipart] = -*t;
01667                     parti->psize[ipart++] = argremain-sum;
01668                     ipart++;
01669                     sum = argremain;
01670                 }
01671                 else {
01672                     parti->nfun[ipart] = -*t;
01673                     parti->psize[ipart++] = t[2];
01674                     argremain -= 2;
01675                     sum += t[2];
01676                 }
01677                 t += 3;
01678             }
01679             else if ( *t == -SNUMBER && t[1] > 0 && ipart+t[1] <= parti->numpart
01680                 && t[2] <= -FUNCTION && t[3] == -SNUMBER && t[4] > 0 ) {
01681                 if ( functions[-t[2]-FUNCTION].spec > 0 ) tensorflag = 1;
01682                 argremain -= 3;
01683                 for ( i = 0; i < t[1]; i++ ) {
01684                     parti->nfun[ipart] = -t[2];
01685                     parti->psize[ipart++] = t[4];
01686                     sum += t[4];
01687                 }
01688                 if ( sum > argremain ) goto WhatAPity;
01689                 t += 5;
01690             }
01691             else goto WhatAPity;
01692         }
01693         if ( sum != argremain ) goto WhatAPity;
01694         parti->numargs = argremain;
01695     }
01696     else goto No;

```

```

01697 /*
01698 Now load the offsets of the arguments and check if needed whether OK with tensor
01699 */
01700 for ( i = 0; i < parti->numargs; i++ ) {
01701     parti->args[i] = t - tfun;
01702     if ( tensorflag && ( *t != -VECTOR && *t != -INDEX ) ) goto WhatAPity;
01703     NEXTARG(t);
01704 }
01705 return(1);
01706 WhatAPity:
01707 M_free(parti->psize,"partitions");
01708 parti->psize = parti->nfun = parti->args = parti->nargs = 0;
01709 parti->numargs = parti->numpart = parti->where = 0;
01710 No:
01711 return(0);
01712 }
01713
01714 /*
01715     #] TestPartitions :
01716     #[ DoPartitions :
01717
01718     As we have only one AT.partitions we need to copy it locally
01719     if we keep needing it.
01720 */
01721
01722 WORD DoPartitions(PHEAD WORD *term, WORD level)
01723 {
01724     WORD x, i, j, im, *fun, ndiff, siz, tensorflag = 0;
01725     PARTI part = AT.partitions;
01726     WORD *array, **j3, **j3fill, **j3where;
01727     WORD a, pfill, *j2, *j2fill, j3size, ncoeff, ncoeffnum, nfac, ncoeff2, ncoeff3, n;
01728     UWORD *coeff, *coeffnum, *cfac, *coeff2, *coeff3, *c;
01729     /* Make AT.partitions ready for future use (if there is another function) */
01730     AT.partitions.psize = AT.partitions.nfun = AT.partitions.args = AT.partitions.nargs = 0;
01731     AT.partitions.numargs = AT.partitions.numpart = AT.partitions.where = 0;
01732 /*
01733 Start with bubble sorting the list of arguments. And the list of partitions.
01734 */
01735 fun = term + part.where;
01736 if ( functions[*fun-FUNCTION].spec > 0 ) tensorflag = 1;
01737 for ( i = 1; i < part.numargs; i++ ) {
01738     for ( j = i-1; j >= 0; j-- ) {
01739         if ( CompArg(fun+part.args[j+1],fun+part.args[j]) >= 0 ) break;
01740         x = part.args[j+1]; part.args[j+1] = part.args[j]; part.args[j] = x;
01741     }
01742 }
01743 for ( i = 1; i < part.numpart; i++ ) {
01744     for ( j = i-1; j >= 0; j-- ) {
01745         if ( part.psize[j+1] < part.psize[j] ) break;
01746         if ( part.psize[j+1] == part.psize[j] && part.nfun[j+1] <= part.nfun[j] ) break;
01747         x = part.psize[j+1]; part.psize[j+1] = part.psize[j]; part.psize[j] = x;
01748         x = part.nfun[j+1]; part.nfun[j+1] = part.nfun[j]; part.nfun[j] = x;
01749     }
01750 }
01751 /*
01752 Now we have the partitions sorted from high to low and the arguments
01753 have been sorted the regular way arguments are sorted in a symmetrize.
01754 The important thing is that identical arguments are adjacent.
01755 Assign the numbers (identical arguments have identical numbers).
01756 */
01757 ndiff = 1; part.nargs[0] = ndiff;
01758 for ( i = 1; i < part.numargs; i++ ) {
01759     if ( CompArg(fun+part.args[i],fun+part.args[i-1]) != 0 ) ndiff++;
01760     part.nargs[i] = ndiff;
01761 }
01762 part.nargs[part.numargs] = 0;
01763 coeffnum = NumberMalloc("partitionsn");
01764 coeff = NumberMalloc("partitions");
01765 coeff2 = NumberMalloc("partitions2");
01766 coeff3 = NumberMalloc("partitions3");
01767 cfac = NumberMalloc("partitions!");
01768 ncoeffnum = 1; coeffnum[0] = 1;
01769 /*
01770 The numerator of the coefficient will be n1!*n2!*...*n(ndiff)!
01771 We compute it only once.
01772 */
01773 j = 0;
01774 for ( i = 1; i <= ndiff; i++ ) {
01775     n = 0;
01776     while ( part.nargs[j] == i ) { n++; j++; }
01777     if ( n > 1 ) { /* 1! needs no attention */
01778         if ( Factorial(BHEAD n, cfac, &nfac) ) Terminate(-1);
01779         if ( MulLong(coeffnum,ncoeffnum,cfac,nfac,coeff2,&ncoeff2) ) Terminate(-1);
01780         c = coeffnum; coeffnum = coeff2; coeff2 = c;
01781         n = ncoeffnum; ncoeffnum = ncoeff2; ncoeff2 = n;
01782     }
01783 }

```

```

01784 /*
01785     Now comes the part where we have to make sure that
01786     a: we generate all partitions.
01787     b: we generate only different partitions.
01788     c: we get the proper combinatorics factor.
01789     Method:
01790     Suppose the largest partition needs n objects and there are m partitions.
01791     We allocate m arrays of n 'digits'. Make in the smaller partitions the
01792     appropriate leading digits zero.
01793     Divide the largest numbers (of the arguments) over the partitions as
01794     leftmost digits (after possible zeroes). The arrays, seen as numbers,
01795     should be such that each is less or equal to its left neighbour. Take the
01796     next largest numbers, etc. This generates unique partitions and all of
01797     them. Because we have a formula for the multiplicity, this should do it.
01798
01799     The general case. At a later stage we might put in a more economical
01800     version for special cases.
01801 */
01802     siz = part.psize[0];
01803     j3size = 2*(part.numpart+1)+2*(part.numargs+1);
01804     array = (WORD *)Malloc1((part.numpart+1)*siz*sizeof(WORD), "parts");
01805     j3 = (WORD **)Malloc1(j3size*sizeof(WORD *), "parts3");
01806     j2 = (WORD *)Malloc1((part.numpart+part.numargs+2)*sizeof(WORD), "parts2");
01807     j3fill = j3+(part.numpart+1);
01808     j3where = j3fill+(part.numpart+1);
01809     for ( i = 0; i < j3size; i++ ) j3[i] = 0;
01810     j2fill = j2+(part.numpart+1);
01811     for ( i = 0; i < part.numargs; i++ ) j2fill[i] = 0;
01812     for ( i = 0; i < part.numpart; i++ ) {
01813         j3[i] = array+i*siz;
01814         for ( j = 0; j < siz; j++ ) j3[i][j] = 0;
01815         j3fill[i] = j3[i]+(siz-part.psize[i]);
01816         j2[i] = part.psize[i]; /* Number of places still available */
01817     }
01818     j3[part.numpart] = array+part.numpart*siz;
01819     j2[part.numpart] = 0;
01820 /*
01821     Now comes a complicated two-level recursion in a and pfill.
01822 */
01823     a = part.numargs-1;
01824     pfill = 0;
01825 /*
01826     We start putting the last number in part.nargs in the first partition in array.
01827     For backtracking we need to know where we put this number. Hence j3where.
01828 */
01829     while ( a < part.numargs ) {
01830         while ( j2[pfill] <= 0 ) {
01831             pfill++;
01832             while ( pfill >= part.numpart ) { /* we have to pop */
01833                 a++;
01834                 if ( a >= part.numargs ) goto Done;
01835                 pfill = j2fill[a];
01836                 j2[pfill]++;
01837                 j3where[a][0] = 0;
01838                 j3fill[pfill]--;
01839                 pfill++;
01840             }
01841         }
01842         j3where[a] = j3fill[pfill];
01843         *(j3fill[pfill])++ = part.nargs[a];
01844         j2[pfill]--; j2fill[a] = pfill;
01845 /*
01846         Now test whether this is allowed.
01847 */
01848         if ( pfill > 0 && part.psize[pfill] == part.psize[pfill-1]
01849             && part.nfun[pfill] == part.nfun[pfill-1] ) { /* First check whether allowed */
01850             for ( im = 0; im < siz; im++ ) {
01851                 if ( j3[pfill-1][im] < j3[pfill][im] ) break;
01852                 if ( j3[pfill-1][im] > j3[pfill][im] ) im = siz;
01853             }
01854             if ( im < siz ) { /* not ordered. undo and raise pfill */
01855                 pfill = j2fill[a];
01856                 j2[pfill]++;
01857                 j3where[a][0] = 0;
01858                 j3fill[pfill]--;
01859                 pfill++;
01860                 continue; /* Note that j2[part.numpart] = 0 */
01861             }
01862         }
01863         a--;
01864         if ( a < 0 ) { /* Solution */
01865 /*
01866             #[ Solution :
01867
01868             Now we compose the output term. The input term contains
01869             three parts: head, partitions_, tail.
01870             partitions_ starts at term+part.where.

```



```

01871      We first put the function parts and worry about the coefficient later.
01872  */
01873      WORD *t, *to, *twhere = term+part.where, *t2, *tend = term+term, *termout;
01874      WORD num, jj, *targ, *tfun;
01875      t2 = twhere+twhere[1];
01876      to = termout = AT.WorkPointer;
01877      if ( termout + *term + part.numpart*FUNHEAD + AM.MaxTal >= AT.WorkTop ) {
01878          return(MesWork());
01879      }
01880      for ( i = 0; i < ncoeffnum; i++ ) coeff[i] = coeffnum[i];
01881      ncoeff = ncoeffnum;
01882      t = term; while ( t < twhere ) *to++ = *t++;
01883  /*
01884      Now the partitions
01885  */
01886      for ( i = 0; i < part.numpart; i++ ) {
01887          tfun = to;
01888          *to++ = part.nfun[i]; to++; FILLFUN(to);
01889          for ( j = 1; j <= part.psize[i]; j++ ) {
01890              num = j3[i][siz-j]; /* now we need an argument with this number */
01891              for ( jj = num-1; jj < part.numargs; jj++ ) {
01892                  if ( part.nargs[jj] == num ) break;
01893              }
01894              targ = part.args[jj]+twhere;
01895              if ( *targ < 0 ) {
01896                  if ( tensorflag ) targ++;
01897                  else if ( *targ > -FUNCTION ) *to++ = *targ++;
01898                  *to++ = *targ++;
01899              }
01900              else { jj = *targ; NCOPY(to,targ,jj); }
01901          }
01902          tfun[1] = to - tfun;
01903      }
01904  /*
01905      Now the denominators of the coefficient
01906      First identical functions/partitions
01907  */
01908      j = 1; n = 1;
01909      while ( j < part.numpart ) {
01910          for ( im = 0; im < siz; im++ ) {
01911              if ( part.nfun[j-1] != part.nfun[j] ) break;
01912              if ( j3[j-1][im] < j3[j][im] ) break;
01913              if ( j3[j-1][im] > j3[j][im] ) im = 2*siz+2;
01914          }
01915          if ( im == siz ) { n++; j++; continue; }
01916          if ( n > 1 ) {
01917              div1: if ( Factorial(BHEAD n, cfac, &nfac) ) Terminate(-1);
01918                  if ( DivLong(coeff,ncoeff,cfac,nfac,coeff2,&ncoeff2,coeff3,&ncoeff3) )
01919                      Terminate(-1);
01920                      c = coeff; coeff = coeff2; coeff2 = c;
01921                      n = ncoeff; ncoeff = ncoeff2; ncoeff2 = n;
01922                      }
01923                      n = 1; j++;
01924                      }
01925                      if ( n > 1 ) goto div1;
01926  /*
01927      Now identical elements inside the partitions
01928  */
01929      for ( i = 0; i < part.numpart; i++ ) {
01930          j = 0; while ( j3[i][j] == 0 ) j++;
01931          n = 1; j++;
01932          while ( j < siz ) {
01933              if ( j3[i][j-1] == j3[i][j] ) { n++; j++; }
01934              else {
01935                  if ( n > 1 ) {
01936                      div2: if ( Factorial(BHEAD n, cfac, &nfac) ) Terminate(-1);
01937                          if ( DivLong(coeff,ncoeff,cfac,nfac,coeff2,&ncoeff2,coeff3,&ncoeff3) )
01938                              Terminate(-1);
01939                              c = coeff; coeff = coeff2; coeff2 = c;
01940                              n = ncoeff; ncoeff = ncoeff2; ncoeff2 = n;
01941                              }
01942                              n = 1; j++;
01943                              }
01944                              if ( n > 1 ) goto div2;
01945  /*
01946      And put this inside the term. Normalize will take care of it.
01947  */
01948      if ( ncoeff != 1 || coeff[0] > 1 ) {
01949          if ( ncoeff == 1 && coeff[0] <= MAXPOSITIVE ) {
01950              *to++ = SNUMBER; *to++ = 4; *to++ = (WORD)(coeff[0]); *to++ = 1;
01951          }
01952          else {
01953              *to++ = LNUMBER; *to++ = ncoeff+3; *to++ = ncoeff;
01954              for ( i = 0; i < ncoeff; i++ ) *to++ = ((WORD *)coeff)[i];
01955          }

```

```

01956         }
01957     /*
01958         And the tail
01959     */
01960     while ( t2 < tend ) *to++ = *t2++;
01961     *termout = to-termout;
01962     AT.WorkPointer = to;
01963     if ( Generator(BHEAD termout,level) ) Terminate(-1);
01964     AT.WorkPointer = termout;
01965 /*
01966     #] Solution :
01967
01968     Now we can pop all a with the lowest value and one more.
01969 */
01970     a = 0;
01971     while ( part.nargs[a] == 1 ) {
01972         pfill = j2fill[a]; j2[pfill]++; j3where[a][0] = 0; j3fill[pfill]--; a++;
01973     }
01974     if ( a < part.numargs ) {
01975         pfill = j2fill[a]; j2[pfill]++; j3where[a][0] = 0; j3fill[pfill]--; a++;
01976     }
01977     a--;
01978     pfill++;
01979 }
01980 else if ( part.nargs[a] == part.nargs[a+1] ) {}
01981 else { pfill = 0; }
01982 }
01983 Done:
01984     M_free(j2,"parts2");
01985     M_free(j3,"parts3");
01986     M_free(array,"parts");
01987     NumberFree(cfac,"partitions!");
01988     NumberFree(coeff3,"partitions3");
01989     NumberFree(coeff2,"partitions2");
01990     NumberFree(coeff,"partitions");
01991     NumberFree(coeffnum,"partitionsn");
01992     M_free(part.psize,"partitions");
01993     part.psize = part.nfun = part.args = part.nargs = 0;
01994     part.numargs = part.numpart = part.where = 0;
01995     return(0);
01996 }
01997
01998 /*
01999     #] DoPartitions :
02000     #] Distribute :
02001     #[ DoPermutations :
02002
02003     Routine replaces the function perm_(f,args) by occurrences of f with
02004     all permutations of the args. This should always fit!
02005 */
02006
02007 WORD DoPermutations(PHEAD WORD *term, WORD level)
02008 {
02009     PERMP perm;
02010     WORD *oldworkpointer = AT.WorkPointer, *termout = AT.WorkPointer;
02011     WORD *t, *tstop, *tt, *ttstop, odd = 0;
02012     WORD *args[MAXMATCH], nargs, i, first, skip, *to, *from;
02013 /*
02014     Find function and count arguments. Check for odd/even
02015 */
02016     tstop = term+*term; tstop -= ABS(tstop[-1]);
02017     t = term+1;
02018     while ( t < tstop ) {
02019         if ( *t == PERMUTATIONS ) {
02020             if ( t[1] >= FUNHEAD+1 && t[FUNHEAD] <= -FUNCTION ) {
02021                 odd = 0; skip = 1;
02022             }
02023             else if ( t[1] >= FUNHEAD+3 && t[FUNHEAD] == -SNUMBER && t[FUNHEAD+2] <= -FUNCTION ) {
02024                 if ( t[FUNHEAD+1] % 2 == 1 ) odd = -1;
02025                 else odd = 0;
02026                 skip = 3;
02027             }
02028             else { t += t[1]; continue; }
02029             tt = t+FUNHEAD+skip; ttstop = t + t[1];
02030             nargs = 0;
02031             while ( tt < ttstop ) { NEXTARG(tt); nargs++; }
02032             tt = t+FUNHEAD+skip;
02033             if ( nargs > MAXMATCH ) {
02034                 MLOCK(ErrorMessageLock);
02035                 MesPrint("Too many arguments in function perm_. %d! is way too big", (WORD)MAXMATCH);
02036                 MUNLOCK(ErrorMessageLock);
02037                 SETERROR(-1)
02038             }
02039             i = 0;
02040             while ( tt < ttstop ) { args[i++] = tt; NEXTARG(tt); }
02041             perm.n = nargs;
02042             perm.sign = 0;

```

```

02043     perm.objects = args;
02044     first = 1;
02045     while ( (first = PermuteP(&perm,first) ) == 0 ) {
02046 /*
02047         Compose the output term
02048 */
02049         to = termout; from = term;
02050         while ( from < t ) *to++ = *from++;
02051         *to++ = -t[FUNHEAD+skip-1];
02052         *to++ = t[1] - skip;
02053         for ( i = 2; i < FUNHEAD; i++ ) *to++ = t[i];
02054         for ( i = 0; i < nargs; i++ ) {
02055             from = args[i];
02056             COPY1ARG(to,from);
02057         }
02058         from = t+t[1];
02059         tstop = term + *term;
02060         while ( from < tstop ) *to++ = *from++;
02061         if ( odd && ( ( perm.sign & 1 ) != 0 ) ) to[-1] = -to[-1];
02062         *termout = to - termout;
02063         AT.WorkPointer = to;
02064         if ( Generator(BHEAD termout,level) ) Terminate(-1);
02065         AT.WorkPointer = oldworkpointer;
02066     }
02067     return(0);
02068 }
02069 t += t[1];
02070 }
02071 return(0);
02072 }
02073
02074 /*
02075 #] DoPermutations :
02076 #[ DoShuffle :
02077
02078 Merges the arguments of all occurrences of function fun into a
02079 single occurrence of fun. The opposite of Distrib_
02080 Syntax:
02081     Shuffle[,once|all],fun;
02082     Shuffle[,once|all],$fun;
02083 The expansion of the dollar should give a single function.
02084 The dollar is indicated as usual with a negative value.
02085 option = 1 (once): generate identical results only once
02086 option = 0 (all): generate identical results with combinatorics (default)
02087 */
02088
02089 /*
02090 We use the Shuffle routine which has a large amount of combinatorics.
02091 It doesn't have grouped combinatorics as in (0,1,2)*(0,1,3) where the
02092 groups (0,1) also cause double terms.
02093 */
02094
02095 WORD DoShuffle(WORD *term, WORD level, WORD fun, WORD option)
02096 {
02097     GETIDENTITY
02098     SHvariables SHback, *SH = &(AN.SHvar);
02099     WORD *t1, *t2, *tstop, ncoef, n = fun, *to, *from;
02100     int i, error;
02101     LONG k;
02102     UWORD *newcombi;
02103
02104     if ( n < 0 ) {
02105         if ( ( n = DolToFunction(BHEAD -n) ) == 0 ) {
02106             MLOCK(ErrorMessageLock);
02107             MesPrint("$-variable in merge statement did not evaluate to a function.");
02108             MUNLOCK(ErrorMessageLock);
02109             return(1);
02110         }
02111     }
02112     if ( AT.WorkPointer + 3*(*term) + AM.MaxTal > AT.WorkTop ) {
02113         MLOCK(ErrorMessageLock);
02114         MesWork();
02115         MUNLOCK(ErrorMessageLock);
02116         return(-1);
02117     }
02118
02119     tstop = term + *term;
02120     ncoef = tstop[-1];
02121     tstop -= ABS(ncoef);
02122     t1 = term + 1;
02123     while ( t1 < tstop ) {
02124         if ( ( *t1 == n ) && ( t1+t1[1] < tstop ) && ( t1[1] > FUNHEAD ) ) {
02125             t2 = t1 + t1[1];
02126             if ( t2 >= tstop ) {
02127                 return(Generator(BHEAD term,level));
02128             }
02129             while ( t2 < tstop ) {

```

```

02130         if ( ( *t2 == n ) && ( t2[1] > FUNHEAD ) ) break;
02131         t2 += t2[1];
02132     }
02133     if ( t2 < tstop ) break;
02134 }
02135 t1 += t1[1];
02136 }
02137 if ( t1 >= tstop ) {
02138     return(Generator(BHEAD term,level));
02139 }
02140 *AN.RepPoint = 1;
02141 /*
02142 Now we have two occurrences of the function.
02143 Back up all relevant variables and load all the stuff that needs to be
02144 passed on.
02145 */
02146 SHback = AN.SHvar;
02147 SH->finishuf = &FinishShuffle;
02148 SH->do_uffle = &DoShuffle;
02149 SH->outterm = AT.WorkPointer;
02150 AT.WorkPointer += *term;
02151 SH->stop1 = t1 + t1[1];
02152 SH->stop2 = t2 + t2[1];
02153 SH->thefunction = n;
02154 SH->option = option;
02155 SH->level = level;
02156 SH->incoef = tstop;
02157 SH->nincoef = ncoef;
02158
02159 if ( AN.SHcombi == 0 || AN.SHcombisize == 0 ) {
02160     AN.SHcombisize = 200;
02161     AN.SHcombi = (UWORD *)Malloca(AN.SHcombisize*sizeof(UWORD), "AN.SHcombi");
02162     SH->combilast = 0;
02163     SHback.combilast = 0;
02164 }
02165 else {
02166     SH->combilast += AN.SHcombi[SH->combilast]+1;
02167     if ( SH->combilast >= AN.SHcombisize - 100 ) {
02168         newcombi = (UWORD *)Malloca(2*AN.SHcombisize*sizeof(UWORD), "AN.SHcombi");
02169         for ( k = 0; k < AN.SHcombisize; k++ ) newcombi[k] = AN.SHcombi[k];
02170         M_free(AN.SHcombi, "AN.SHcombi");
02171         AN.SHcombi = newcombi;
02172         AN.SHcombisize *= 2;
02173     }
02174 }
02175 AN.SHcombi[SH->combilast] = 1;
02176 AN.SHcombi[SH->combilast+1] = 1;
02177
02178 i = t1-term; to = SH->outterm; from = term;
02179 NCOPY(to,from,i)
02180 SH->outfun = to;
02181 for ( i = 0; i < FUNHEAD; i++ ) { *to++ = t1[i]; }
02182
02183 error = Shuffle(t1+FUNHEAD,t2+FUNHEAD,to);
02184
02185 AT.WorkPointer = SH->outterm;
02186 AN.SHvar = SHback;
02187 if ( error ) {
02188     MesCall("DoShuffle");
02189     return(-1);
02190 }
02191 return(0);
02192 }
02193
02194 /*
02195 #] DoShuffle :
02196 #[ Shuffle :
02197
02198 How to make shuffles:
02199
02200 We have two lists of arguments. We have to make a single
02201 shuffle of them. All combinations. Doubles should have as
02202 much as possible a combinatorics factor. Sometimes this is
02203 very difficult as in:
02204 (0,1,2)x(0,1,3) = -> (0,1) is a repeated pattern and the
02205 factor on that is difficult
02206 Simple way: (without combinatorics)
02207 repeat id f0(?c)*f(x1?,?a)*f(x2?,?b) =
02208         +f0(?c,x1)*f(?a)*f(x2,?b)
02209         +f0(?c,x2)*f(x1,?a)*f(?b);
02210
02211 Refinement:
02212 if ( x1 == x2 ) check how many more there are of the same.
02213 --> (n1,x) and (n2,x)
02214 id f0(?c)*f1((n1,x),?b)*f2((n2,x),?c) =
02215         +binom_(n1+n2,n1)*f0(?c,(n1+n2,x))*f1(?a)*f2(?b)
02216         +sum_(j,0,n1-1,binom_(n2+j,j)*f0(?c,(j+n2,x))
02217             *f1((n1-j),?a)*f2(?b))*force2

```

```

02217             +sum_(j,0,n2-1,binom_(n1+j,j)*f0(?c,(j+n1,x))
02218                 *f1(?a)*f2((n2-j),?b))*force1
02219     The force operation can be executed directly
02220
02221     The next question is how to program this: recursively or linearly
02222     which would require simulation of a recursion. Recursive is clearest
02223     but we need to pass a number of arguments from the calling routine
02224     to the final routine. This is done with AN.SHvar.
02225
02226     We need space for the accumulation of the combinatoric factors.
02227 */
02228
02229 int Shuffle(WORD *from1, WORD *from2, WORD *to)
02230 {
02231     GETIDENTITY
02232     WORD *t, *fr, *next1, *next2, na, *fn1, *fn2, *tt;
02233     int i, n, n1, n2, j;
02234     LONG combilast;
02235     SHvariables *SH = &(AN.SHvar);
02236     if ( from1 == SH->stop1 && from2 == SH->stop2 ) {
02237         return(FiniShuffle(to));
02238     }
02239     else if ( from1 == SH->stop1 ) {
02240         i = SH->stop2 - from2; t = to; tt = from2; NCOPY(t,tt,i)
02241         return(FiniShuffle(t));
02242     }
02243     else if ( from2 == SH->stop2 ) {
02244         i = SH->stop1 - from1; t = to; tt = from1; NCOPY(t,tt,i)
02245         return(FiniShuffle(t));
02246     }
02247 /*
02248     Compare lead arguments
02249 */
02250     if ( AreArgsEqual(from1,from2) ) {
02251 /*
02252         First find out how many of each
02253 */
02254         next1 = from1; n1 = 1; NEXTARG(next1)
02255         while ( ( next1 < SH->stop1 ) && AreArgsEqual(from1,next1) ) {
02256             n1++; NEXTARG(next1)
02257         }
02258         next2 = from2; n2 = 1; NEXTARG(next2)
02259         while ( ( next2 < SH->stop2 ) && AreArgsEqual(from2,next2) ) {
02260             n2++; NEXTARG(next2)
02261         }
02262         combilast = SH->combilast;
02263 /*
02264         +binom_(n1+n2,n1)*f0(?c,(n1+n2,x))*f1(?a)*f2(?b)
02265 */
02266         t = to;
02267         n = n1 + n2;
02268         while ( --n >= 0 ) { fr = from1; CopyArg(t,fr) }
02269         if ( GetBinom((UWORD *) (t),&na,n1+n2,n1) ) goto shuffcall;
02270         if ( combilast + AN.SHcombi[combilast] + na + 2 >= AN.SHcombisize ) {
02271 /*
02272             We need more memory in this stack. Fortunately this is the
02273             only place where we have to do this, because the other factors
02274             are definitely smaller.
02275             Layout:  size, LongInteger, size, LongInteger, .....
02276             We start pointing at the last one.
02277 */
02278             UWORD *combi = (UWORD *)Malloca(2*AN.SHcombisize*2,"AN.SHcombi");
02279             LONG jj;
02280             for ( jj = 0; jj < AN.SHcombisize; jj++ ) combi[jj] = AN.SHcombi[jj];
02281             AN.SHcombisize *= 2;
02282             M_free(AN.SHcombi,"AN.SHcombi");
02283             AN.SHcombi = combi;
02284         }
02285         if ( MulLong((UWORD *) (AN.SHcombi+combilast+1),AN.SHcombi[combilast],
02286             (UWORD *) (t),na,
02287             (UWORD *) (AN.SHcombi+combilast+AN.SHcombi[combilast]+2),
02288             (WORD *) (AN.SHcombi+combilast+AN.SHcombi[combilast]+1)) ) goto shuffcall;
02289         SH->combilast = combilast + AN.SHcombi[combilast] + 1;
02290         if ( next1 >= SH->stop1 ) {
02291             fr = next2; i = SH->stop2 - fr;
02292             NCOPY(t,fr,i)
02293             if ( FiniShuffle(t) ) goto shuffcall;
02294         }
02295         else if ( next2 >= SH->stop2 ) {
02296             fr = next1; i = SH->stop1 - fr;
02297             NCOPY(t,fr,i)
02298             if ( FiniShuffle(t) ) goto shuffcall;
02299         }
02300         else {
02301             if ( Shuffle(next1,next2,t) ) goto shuffcall;
02302         }
02303         SH->combilast = combilast;

```

```

02304  /*
02305      +sum_(j,0,n1-1,binom_(n2+j,j)*f0(?c,(j+n2,x))
02306      *f1((n1-j),?a)*f2(?b))*force2
02307  */
02308      if ( next2 < SH->stop2 ) {
02309          t = to;
02310          n = n2;
02311          while ( --n >= 0 ) { fr = from1; CopyArg(t,fr) }
02312          for ( j = 0; j < n1; j++ ) {
02313              if ( GetBinom((UWORD *) (t),&na,n2+j,j) ) goto shuffcall;
02314              if ( Mullong((UWORD *) (AN.SHcombi+combilast+1),AN.SHcombi[combilast],
02315                          (UWORD *) (t),na,
02316                          (UWORD *) (AN.SHcombi+combilast+AN.SHcombi[combilast]+2),
02317                          (WORD *) (AN.SHcombi+combilast+AN.SHcombi[combilast]+1)) ) goto shuffcall;
02318              SH->combilast = combilast + AN.SHcombi[combilast] + 1;
02319              if ( j > 0 ) { fr = from1; CopyArg(t,fr) }
02320              fn2 = next2; tt = t;
02321              CopyArg(tt,fn2)
02322          }
02323          if ( fn2 >= SH->stop2 ) {
02324              n = n1-j;
02325              while ( --n >= 0 ) { fr = from1; CopyArg(tt,fr) }
02326              fr = next1; i = SH->stop1 - fr;
02327              NCOPY(tt,fr,i)
02328              if ( FiniShuffle(tt) ) goto shuffcall;
02329          }
02330          else {
02331              n = j; fn1 = from1; while ( --n >= 0 ) { NEXTARG(fn1) }
02332              if ( Shuffle(fn1,fn2,tt) ) goto shuffcall;
02333          }
02334          SH->combilast = combilast;
02335      }
02336  }
02337  /*
02338      +sum_(j,0,n2-1,binom_(n1+j,j)*f0(?c,(j+n1,x))
02339      *f1(?a)*f2((n2-j),?b))*force1
02340  */
02341      if ( next1 < SH->stop1 ) {
02342          t = to;
02343          n = n1;
02344          while ( --n >= 0 ) { fr = from1; CopyArg(t,fr) }
02345          for ( j = 0; j < n2; j++ ) {
02346              if ( GetBinom((UWORD *) (t),&na,n1+j,j) ) goto shuffcall;
02347              if ( Mullong((UWORD *) (AN.SHcombi+combilast+1),AN.SHcombi[combilast],
02348                          (UWORD *) (t),na,
02349                          (UWORD *) (AN.SHcombi+combilast+AN.SHcombi[combilast]+2),
02350                          (WORD *) (AN.SHcombi+combilast+AN.SHcombi[combilast]+1)) ) goto shuffcall;
02351              SH->combilast = combilast + AN.SHcombi[combilast] + 1;
02352              if ( j > 0 ) { fr = from1; CopyArg(t,fr) }
02353              fn1 = next1; tt = t;
02354              CopyArg(tt,fn1)
02355          }
02356          if ( fn1 >= SH->stop1 ) {
02357              n = n2-j;
02358              while ( --n >= 0 ) { fr = from1; CopyArg(tt,fr) }
02359              fr = next2; i = SH->stop2 - fr;
02360              NCOPY(tt,fr,i)
02361              if ( FiniShuffle(tt) ) goto shuffcall;
02362          }
02363          else {
02364              n = j; fn2 = from2; while ( --n >= 0 ) { NEXTARG(fn2) }
02365              if ( Shuffle(fn1,fn2,tt) ) goto shuffcall;
02366          }
02367          SH->combilast = combilast;
02368      }
02369  }
02370  }
02371  else {
02372  /*
02373      Argument from first list
02374  */
02375      t = to;
02376      fr = from1;
02377      CopyArg(t,fr)
02378      if ( fr >= SH->stop1 ) {
02379          fr = from2; i = SH->stop2 - fr;
02380          NCOPY(t,fr,i)
02381          if ( FiniShuffle(t) ) goto shuffcall;
02382      }
02383      else {
02384          if ( Shuffle(fr,from2,t) ) goto shuffcall;
02385      }
02386  /*
02387      Argument from second list
02388  */
02389      t = to;
02390      fr = from2;

```

```

02391     CopyArg(t,fr)
02392     if ( fr >= SH->stop2 ) {
02393         fr = from1; i = SH->stop1 - fr;
02394         NCOPY(t,fr,i)
02395         if ( FiniShuffle(t) ) goto shuffcall;
02396     }
02397     else {
02398         if ( Shuffle(from1,fr,t) ) goto shuffcall;
02399     }
02400 }
02401 return(0);
02402 shuffcall:
02403 MesCall("Shuffle");
02404 return(-1);
02405 }
02406
02407 /*
02408 #] Shuffle :
02409 #[ FinishShuffle :
02410
02411 The complications here are:
02412 1: We want to save space. We put the output term in 'out' straight
02413    on top of what we produced thusfar. We have to copy the early
02414    piece because once the term goes back to Generator, Normalize can
02415    change it in situ
02416 2: There can be other occurrence of the function between the two
02417    that we did. For shuffles that isn't likely, but we use this
02418    routine also for the stuffles and there it can happen.
02419 */
02420
02421 int FinishShuffle(WORD *fini)
02422 {
02423     GETIDENTITY
02424     WORD *t, *t1, *oldworkpointer = AT.WorkPointer, *tcoef, ntcoef, *out;
02425     int i;
02426     SHvariables *SH = &(AN.SHvar);
02427     SH->outfun[1] = fini - SH->outfun;
02428     if ( functions[SH->outfun[0]-FUNCTION].symmetric != 0 )
02429         SH->outfun[2] |= DIRTYSYMFLAG;
02430     out = fini; i = fini - SH->outterm; t = SH->outterm;
02431     NCOPY(fini,t,i)
02432     t = SH->stop1;
02433     t1 = t + t[1];
02434     while ( t1 < SH->stop2 ) { t = t1; t1 = t + t[1]; }
02435     t1 = SH->stop1;
02436     while ( t1 < t ) *fini++ = *t1++;
02437     t = SH->stop2;
02438     while ( t < SH->incoef ) *fini++ = *t++;
02439     tcoef = fini;
02440     ntcoef = SH->nincoef;
02441     i = ABS(ntcoef);
02442     NCOPY(fini,t,i);
02443     ntcoef = REDLENG(ntcoef);
02444     Mully(BHEAD (UWORD *)tcoef,&ntcoef,
02445         (UWORD *) (AN.SHcombi+SH->combilast+1),AN.SHcombi[SH->combilast]));
02446     ntcoef = INCLENG(ntcoef);
02447     fini = tcoef + ABS(ntcoef);
02448     if ( ( ( SH->option & 2 ) != 0 ) && ( ( SH->option & 256 ) != 0 ) ) ntcoef = -ntcoef;
02449     fini[-1] = ntcoef;
02450     i = *out = fini - out;
02451 /*
02452 Now check whether we have to do more
02453 */
02454 AT.WorkPointer = out + *out;
02455 if ( ( SH->option & 1 ) == 1 ) {
02456     if ( Generator(BHEAD out,SH->level) ) goto Finicall;
02457 }
02458 else {
02459     if ( DoStuffle(out,SH->level,SH->thefunction,SH->option) ) goto Finicall;
02460 }
02461 AT.WorkPointer = oldworkpointer;
02462 return(0);
02463 Finicall:
02464 AT.WorkPointer = oldworkpointer;
02465 MesCall("FinishShuffle");
02466 return(-1);
02467 }
02468
02469 /*
02470 #] FinishShuffle :
02471 #[ DoStuffle :
02472
02473 Stuffling is a variation of shuffling.
02474 In the stuffling we insist that the arguments are (short) integers. nonzero.
02475 The stuffle sum is  $x \text{ st } y = \text{sig}_-(x) * \text{sig}_-(y) * (\text{abs}(x) + \text{abs}(y))$ 
02476 The way we do this is:
02477 1: count the arguments in each function: n1, n2

```

```

02478         2: take the minimum minval = min(n1,n2).
02479         3: for ( j = 0; j <= min; j++ ) take j elements in each of the lists.
02480         4: the j+1 groups of remaining arguments have to each be shuffled
02481         5: the j selected pairs have to be stuffle added.
02482         We can use many of the shuffle things.
02483         Considering the recursive nature of the generation we actually don't
02484         need to know n1, n2, minval.
02485     */
02486
02487 WORD DoStuffle(WORD *term, WORD level, WORD fun, WORD option)
02488 {
02489     GETIDENTITY
02490     SHvariables SHback, *SH = &(AN.SHvar);
02491     WORD *t1, *t2, *tstop, *t1stop, *t2stop, ncoef, n = fun, *to, *from;
02492     WORD *r1, *r2;
02493     int i, error;
02494     LONG k;
02495     UWORD *newcombi;
02496 #ifdef NEWCODE
02497     WORD *rr1, *rr2, i1, i2;
02498 #endif
02499     if ( n < 0 ) {
02500         if ( ( n = DolToFunction(BHEAD -n) ) == 0 ) {
02501             MLOCK(ErrorMessageLock);
02502             MesPrint("$-variable in merge statement did not evaluate to a function.");
02503             MUNLOCK(ErrorMessageLock);
02504             return(1);
02505         }
02506     }
02507     if ( AT.WorkPointer + 3*(term) + AM.MaxTal > AT.WorkTop ) {
02508         MLOCK(ErrorMessageLock);
02509         MesWork();
02510         MUNLOCK(ErrorMessageLock);
02511         return(-1);
02512     }
02513
02514     tstop = term + *term;
02515     ncoef = tstop[-1];
02516     tstop -= ABS(ncoef);
02517     t1 = term + 1;
02518     retry1:;
02519     while ( t1 < tstop ) {
02520         if ( ( *t1 == n ) && ( t1+t1[1] < tstop ) && ( t1[1] > FUNHEAD ) ) {
02521             t2 = t1 + t1[1];
02522             if ( t2 >= tstop ) {
02523                 return(Generator(BHEAD term,level));
02524             }
02525             retry2:;
02526             while ( t2 < tstop ) {
02527                 if ( ( *t2 == n ) && ( t2[1] > FUNHEAD ) ) break;
02528                 t2 += t2[1];
02529             }
02530             if ( t2 < tstop ) break;
02531         }
02532         t1 += t1[1];
02533     }
02534     if ( t1 >= tstop ) {
02535         return(Generator(BHEAD term,level));
02536     }
02537     /*
02538     Next we have to check that the arguments are of the correct type
02539     At the same time we can count them.
02540     */
02541 #ifndef NEWCODE
02542     t1stop = t1 + t1[1];
02543     r1 = t1 + FUNHEAD;
02544     while ( r1 < t1stop ) {
02545         if ( *r1 != -SNUMBER ) break;
02546         if ( r1[1] == 0 ) break;
02547         r1 += 2;
02548     }
02549     if ( r1 < t1stop ) { t1 = t2; goto retry1; }
02550     t2stop = t2 + t2[1];
02551     r2 = t2 + FUNHEAD;
02552     while ( r2 < t2stop ) {
02553         if ( *r2 != -SNUMBER ) break;
02554         if ( r2[1] == 0 ) break;
02555         r2 += 2;
02556     }
02557     if ( r2 < t2stop ) { t2 = t2 + t2[1]; goto retry2; }
02558 #else
02559     t1stop = t1 + t1[1];
02560     r1 = t1 + FUNHEAD;
02561     while ( r1 < t1stop ) {
02562         if ( *r1 == -SNUMBER ) {
02563             if ( r1[1] == 0 ) break;
02564             r1 += 2; continue;

```



```

02565     }
02566     else if ( *r1 == -SYMBOL ) {
02567         if ( ( symbols[r1[1]].complex & VARTYPEROOTOFUNITY ) != VARTYPEROOTOFUNITY )
02568             break;
02569         r1 += 2; continue;
02570     }
02571     if ( *r1 > 0 && *r1 == r1[ARGHEAD]+ARGHEAD ) {
02572         if ( ABS(r1[r1[0]-1]) == r1[0]-ARGHEAD-1 ) {}
02573         else if ( r1[ARGHEAD+1] == SYMBOL ) {
02574             rr1 = r1 + ARGHEAD + 3;
02575             i1 = rr1[-1]-2;
02576             while ( i1 > 0 ) {
02577                 if ( ( symbols[*rr1].complex & VARTYPEROOTOFUNITY ) != VARTYPEROOTOFUNITY )
02578                     break;
02579                 i1 -= 2; rr1 += 2;
02580             }
02581             if ( i1 > 0 ) break;
02582         }
02583         else break;
02584         rr1 = r1+*r1-1;
02585         i1 = (ABS(*rr1)-1)/2;
02586         while ( i1 > 1 ) {
02587             if ( rr1[-1] ) break;
02588             i1--; rr1--;
02589         }
02590         if ( i1 > 1 || rr1[-1] != 1 ) break;
02591         r1 += *r1;
02592     }
02593     else break;
02594 }
02595 if ( r1 < t1stop ) { t1 = t2; goto retry1; }
02596 t2stop = t2 + t2[1];
02597 r2 = t2 + FUNHEAD;
02598
02599 while ( r2 < t2stop ) {
02600     if ( *r2 == -SNUMBER ) {
02601         if ( r2[1] == 0 ) break;
02602         r2 += 2; continue;
02603     }
02604     else if ( *r2 == -SYMBOL ) {
02605         if ( ( symbols[r2[1]].complex & VARTYPEROOTOFUNITY ) != VARTYPEROOTOFUNITY )
02606             break;
02607         r2 += 2; continue;
02608     }
02609     if ( *r2 > 0 && *r2 == r2[ARGHEAD]+ARGHEAD ) {
02610         if ( ABS(r2[r2[0]-1]) == r2[0]-ARGHEAD-1 ) {}
02611         else if ( r2[ARGHEAD+1] == SYMBOL ) {
02612             rr2 = r2 + ARGHEAD + 3;
02613             i2 = rr2[-1]-2;
02614             while ( i2 > 0 ) {
02615                 if ( ( symbols[*rr2].complex & VARTYPEROOTOFUNITY ) != VARTYPEROOTOFUNITY )
02616                     break;
02617                 i2 -= 2; rr2 += 2;
02618             }
02619             if ( i2 > 0 ) break;
02620         }
02621         else break;
02622         rr2 = r2+*r2-1;
02623         i2 = (ABS(*rr2)-1)/2;
02624         while ( i2 > 1 ) {
02625             if ( rr2[-1] ) break;
02626             i2--; rr2--;
02627         }
02628         if ( i2 > 1 || rr2[-1] != 1 ) break;
02629         r2 += *r2;
02630     }
02631     else break;
02632 }
02633 if ( r2 < t2stop ) { t2 = t2 + t2[1]; goto retry2; }
02634 #endif
02635 /*
02636 OK, now we got two objects that can be used.
02637 */
02638 *AN.RepPoint = 1;
02639
02640 SHback = AN.SHvar;
02641 SH->finishuf = &FinishStuffle;
02642 SH->do_uffle = &DoStuffle;
02643 SH->outterm = AT.WorkPointer;
02644 AT.WorkPointer += *term;
02645 SH->ststop1 = t1 + t1[1];
02646 SH->ststop2 = t2 + t2[1];
02647 SH->thefunction = n;
02648 SH->option = option;
02649 SH->level = level;
02650 SH->incoef = tstop;
02651 SH->nincoef = ncoef;

```

```

02652     if ( AN.SHcombi == 0 || AN.SHcombisize == 0 ) {
02653         AN.SHcombisize = 200;
02654         AN.SHcombi = (UWORD *)Malloc1(AN.SHcombisize*sizeof(UWORD), "AN.SHcombi");
02655         SH->combilast = 0;
02656         SHback.combilast = 0;
02657     }
02658     else {
02659         SH->combilast += AN.SHcombi[SH->combilast]+1;
02660         if ( SH->combilast >= AN.SHcombisize - 100 ) {
02661             newcombi = (UWORD *)Malloc1(2*AN.SHcombisize*sizeof(UWORD), "AN.SHcombi");
02662             for ( k = 0; k < AN.SHcombisize; k++ ) newcombi[k] = AN.SHcombi[k];
02663             M_free(AN.SHcombi, "AN.SHcombi");
02664             AN.SHcombi = newcombi;
02665             AN.SHcombisize *= 2;
02666         }
02667     }
02668     AN.SHcombi[SH->combilast] = 1;
02669     AN.SHcombi[SH->combilast+1] = 1;
02670
02671     i = t1-term; to = SH->outterm; from = term;
02672     NCOPY(to, from, i)
02673     SH->outfun = to;
02674     for ( i = 0; i < FUNHEAD; i++ ) { *to++ = t1[i]; }
02675
02676     error = Stuffle(t1+FUNHEAD, t2+FUNHEAD, to);
02677
02678     AT.WorkPointer = SH->outterm;
02679     AN.SHvar = SHback;
02680     if ( error ) {
02681         MesCall("DoStuffle");
02682         return(-1);
02683     }
02684     return(0);
02685 }
02686
02687 /*
02688 #] DoStuffle :
02689 #[ Stuffle :
02690
02691 The way to generate the stuffles
02692 1: select an argument in the first list (for(j1=0;j1<last;j1++))
02693 2: select an argument in the second list (for(j2=0;j2<last;j2++))
02694 3: put values for SH->ststop1 and SH->ststop2 at these arguments.
02695 4: generate all shuffles of the arguments in front.
02696 5: Then put the stuffle sum of arg(j1) and arg(j2)
02697 6: Then continue calling Stuffle
02698 7: Once one gets exhausted, we can clean up the list and call FinishShuffle
02699 8: if ( ( SH->option & 2 ) != 0 ) the stuffle sum is negative.
02700 */
02701
02702 int Stuffle(WORD *from1, WORD *from2, WORD *to)
02703 {
02704     GETIDENTITY
02705     WORD *t, *tf, *next1, *next2, *st1, *st2, *savel, *save2;
02706     SHvariables *SH = &(AN.SHvar);
02707     int i, retval;
02708
02709     /*
02710     First the special cases (exhausted list(s)):
02711     */
02712     savel = SH->stop1; save2 = SH->stop2;
02713     if ( from1 >= SH->ststop1 && from2 == SH->ststop2 ) {
02714         SH->stop1 = SH->ststop1;
02715         SH->stop2 = SH->ststop2;
02716         retval = FinishShuffle(to);
02717         SH->stop1 = savel; SH->stop2 = save2;
02718         return(retval);
02719     }
02720     else if ( from1 >= SH->ststop1 ) {
02721         i = SH->ststop2 - from2; t = to; tf = from2; NCOPY(t, tf, i)
02722         SH->stop1 = SH->ststop1;
02723         SH->stop2 = SH->ststop2;
02724         retval = FinishShuffle(t);
02725         SH->stop1 = savel; SH->stop2 = save2;
02726         return(retval);
02727     }
02728     else if ( from2 >= SH->ststop2 ) {
02729         i = SH->ststop1 - from1; t = to; tf = from1; NCOPY(t, tf, i)
02730         SH->stop1 = SH->ststop1;
02731         SH->stop2 = SH->ststop2;
02732         retval = FinishShuffle(t);
02733         SH->stop1 = savel; SH->stop2 = save2;
02734         return(retval);
02735     }
02736
02737     /*
02738     Now the case that we have no stuffle sums.
02739     */
02740     SH->stop1 = SH->ststop1;

```

```

02739     SH->stop2 = SH->ststop2;
02740     SH->finishuf = &FinishShuffle;
02741     if ( Shuffle(from1,from2,to) ) goto stuffcall;
02742     SH->finishuf = &FinishStuffle;
02743     /*
02744     Now we have to select a pair, one from 1 and one from 2.
02745     */
02746     #ifndef NEWCODE
02747     st1 = from1; next1 = st1+2;          /* <----- */
02748     #else
02749     st1 = next1 = from1;
02750     NEXTARG(next1)
02751     #endif
02752     while ( next1 <= SH->ststop1 ) {
02753     #ifndef NEWCODE
02754     st2 = from2; next2 = st2+2;          /* <----- */
02755     #else
02756     next2 = st2 = from2;
02757     NEXTARG(next2)
02758     #endif
02759     while ( next2 <= SH->ststop2 ) {
02760         SH->stop1 = st1;
02761         SH->stop2 = st2;
02762         if ( st1 == from1 && st2 == from2 ) {
02763             t = to;
02764             #ifndef NEWCODE
02765             *t++ = -SNUMBER; *t++ = StuffAdd(st1[1],st2[1]);
02766             #else
02767             t = StuffRootAdd(st1,st2,t);
02768             #endif
02769             SH->option ^= 256;
02770             if ( Stuffle(next1,next2,t) ) goto stuffcall;
02771             SH->option ^= 256;
02772         }
02773         else if ( st1 == from1 ) {
02774             i = st2-from2;
02775             t = to; tf = from2; NCOPY(t,tf,i)
02776             #ifndef NEWCODE
02777             *t++ = -SNUMBER; *t++ = StuffAdd(st1[1],st2[1]);
02778             #else
02779             t = StuffRootAdd(st1,st2,t);
02780             #endif
02781             SH->option ^= 256;
02782             if ( Stuffle(next1,next2,t) ) goto stuffcall;
02783             SH->option ^= 256;
02784         }
02785         else if ( st2 == from2 ) {
02786             i = st1-from1;
02787             t = to; tf = from1; NCOPY(t,tf,i)
02788             #ifndef NEWCODE
02789             *t++ = -SNUMBER; *t++ = StuffAdd(st1[1],st2[1]);
02790             #else
02791             t = StuffRootAdd(st1,st2,t);
02792             #endif
02793             SH->option ^= 256;
02794             if ( Stuffle(next1,next2,t) ) goto stuffcall;
02795             SH->option ^= 256;
02796         }
02797         else {
02798             if ( Shuffle(from1,from2,to) ) goto stuffcall;
02799         }
02800     #ifndef NEWCODE
02801     st2 = next2; next2 += 2;          /* <----- */
02802     #else
02803     st2 = next2;
02804     NEXTARG(next2)
02805     #endif
02806     }
02807     #ifndef NEWCODE
02808     st1 = next1; next1 += 2;          /* <----- */
02809     #else
02810     st1 = next1;
02811     NEXTARG(next1)
02812     #endif
02813     }
02814     SH->stop1 = save1; SH->stop2 = save2;
02815     return(0);
02816 stuffcall:;
02817     MesCall("Stuffle");
02818     return(-1);
02819 }
02820
02821 /*
02822 #] Stuffle :
02823 #[ FinishStuffle :
02824
02825 The program only comes here from the Shuffle routine.

```

```

02826     It should add the stuffle sum and then call Stuffle again.
02827 */
02828
02829 int FinishStuffle(WORD *fini)
02830 {
02831     GETIDENTITY
02832     SHvariables *SH = &(AN.SHvar);
02833     #ifdef NEWCODE
02834     WORD *next1 = SH->stop1, *next2 = SH->stop2;
02835     fini = StuffRootAdd(next1,next2,fini);
02836     #else
02837     *fini++ = -SNUMBER; *fini++ = StuffAdd(SH->stop1[1],SH->stop2[1]);
02838     #endif
02839     SH->option ^= 256;
02840     #ifdef NEWCODE
02841     NEXTARG(next1)
02842     NEXTARG(next2)
02843     if ( Stuffle(next1,next2,fini) ) goto stuffcall;
02844     #else
02845     if ( Stuffle(SH->stop1+2,SH->stop2+2,fini) ) goto stuffcall;
02846     #endif
02847     SH->option ^= 256;
02848     return(0);
02849 stuffcall:;
02850     MesCall("FinishStuffle");
02851     return(-1);
02852 }
02853
02854 /*
02855     #] FinishStuffle :
02856     #[ StuffRootAdd :
02857
02858     Makes the stuffle sum of two arguments.
02859     The arguments can be of one of three types:
02860     1: -SNUMBER,num
02861     2: -SYMBOL,symbol
02862     3: Numerical (long) argument.
02863     4: Generic argument with (only) symbols that are roots of unity and
02864        a coefficient.
02865     We have excluded the case that both t1 and t2 are of type 1:
02866     The output should be written to 'to' and the new fill position should
02867     be the return value.
02868     'to' is inside the workspace.
02869
02870     The stuffle sum is sig_(t2)*t1+sig_(t1)*t2
02871     or sig_(t1)*sig_(t2)*(abs_(t1)+abs_(t2))
02872 */
02873
02874 #ifdef NEWCODE
02875 WORD *StuffRootAdd(WORD *t1, WORD *t2, WORD *to)
02876 {
02877     int type1, type2, type3, sgn, sgn1, sgn2, sgn3, pow, root, nosymbols, i;
02878     WORD *tt1, *tt2, it1, it2, *t3, *r, size1, size2, size3;
02879     WORD scratch[2];
02880     LONG x;
02881     if ( *t1 == -SNUMBER ) { type1 = 1; if ( t1[1] < 0 ) sgn1 = -1; else sgn1 = 1; }
02882     else if ( *t1 == -SYMBOL ) { type1 = 2; sgn1 = 1; }
02883     else if ( ABS(t1[*t1-1]) == *t1-ARGHEAD-1 ) {
02884         type1 = 3; if ( t1[*t1-1] < 0 ) sgn1 = -1; else sgn1 = 1; }
02885     else { type1 = 4; if ( t1[*t1-1] < 0 ) sgn1 = -1; else sgn1 = 1; }
02886     if ( *t2 == -SNUMBER ) { type2 = 1; if ( t2[1] < 0 ) sgn2 = -1; else sgn2 = 1; }
02887     else if ( *t2 == -SYMBOL ) { type2 = 2; sgn2 = 1; }
02888     else if ( ABS(t2[*t2-1]) == *t2-ARGHEAD-1 ) {
02889         type2 = 3; if ( t2[*t2-1] < 0 ) sgn2 = -1; else sgn2 = 1; }
02890     else { type2 = 4; if ( t2[*t2-1] < 0 ) sgn2 = -1; else sgn2 = 1; }
02891     if ( type1 > type2 ) {
02892         t3 = t1; t1 = t2; t2 = t3;
02893         type3 = type1; type1 = type2; type2 = type3;
02894         sgn3 = sgn1; sgn1 = sgn2; sgn2 = sgn3;
02895     }
02896     nosymbols = 1; sgn3 = 1;
02897     switch ( type1 ) {
02898     case 1:
02899         if ( type2 == 1 ) {
02900             x = sgn2 * t1[1];
02901             x += sgn1 * t2[1];
02902             if ( x > MAXPOSITIVE || x < -(MAXPOSITIVE+1) ) {
02903                 if ( x < 0 ) { sgn1 = -3; x = -x; }
02904                 else sgn1 = 3;
02905                 *to++ = ARGHEAD+4;
02906                 *to++ = 0;
02907                 FILLARG(to)
02908                 *to++ = 4; *to++ = (UWORD)x; *to++ = 1; *to++ = sgn1;
02909             }
02910         }
02911         else { *to++ = -SNUMBER; *to++ = (WORD)x; }
02912     }

```

```

02913     else if ( type2 == 2 ) {
02914         *to++ = ARGHEAD+8; *to++ = 0; FILLARG(to)
02915         *to++ = 8; *to++ = SYMBOL; *to++ = 4; *to++ = t2[1]; *to++ = 1;
02916         *to++ = ABS(t1[1])+1;
02917         *to++ = 1;
02918         *to++ = 3*sgn1;
02919     }
02920     else if ( type2 == 3 ) {
02921         tt1 = (WORD *)scratch; tt1[0] = ABS(t1[1]); size1 = 1;
02922         tt2 = t2+ARGHEAD+1; size2 = (ABS(t2[*t2-1])-1)/2;
02923         t3 = to;
02924         *to++ = 0; *to++ = 0; FILLARG(to) *to++ = 0;
02925         goto DoCoeffi;
02926     }
02927     else {
02928         /*
02929         t1 is (short) numeric, t2 has the symbol(s).
02930         */
02931         tt1 = (WORD *)scratch; tt1[0] = ABS(t1[1]); size1 = 1;
02932         tt2 = t2+ARGHEAD+1; tt2 += tt2[1]; size2 = (ABS(t2[*t2-1])-1)/2;
02933         t3 = to; i = tt2 - t2; r = t2;
02934         NCOPY(to,r,i)
02935         nosymbols = 0;
02936         goto DoCoeffi;
02937     }
02938     break;
02939 case 2:
02940     if ( type2 == 2 ) {
02941         if ( t1[1] == t2[1] ) {
02942             if ( ( symbols[t1[1]].maxpower == 4 )
02943                 && ( ( symbols[t1[1]].complex & VARTYPEMINUS ) == VARTYPEMINUS ) ) {
02944                 *to++ = -SNUMBER; *to++ = -2;
02945             }
02946             else if ( symbols[t1[1]].maxpower == 2 ) {
02947                 *to++ = -SNUMBER; *to++ = 2;
02948             }
02949             else {
02950                 *to++ = ARGHEAD+8; *to++ = 0; FILLARG(to)
02951                 *to++ = 8; *to++ = SYMBOL; *to++ = 4;
02952                 *to++ = t1[1]; *to++ = 2;
02953                 *to++ = 2; *to++ = 1; *to++ = 3;
02954             }
02955         }
02956         else {
02957             *to++ = ARGHEAD+10; *to++ = 0; FILLARG(to)
02958             *to++ = 10; *to++ = SYMBOL; *to++ = 6;
02959             if ( t1[1] < t2[1] ) {
02960                 *to++ = t1[1]; *to++ = 1; *to++ = t2[1]; *to++ = 1;
02961             }
02962             else {
02963                 *to++ = t2[1]; *to++ = 1; *to++ = t1[1]; *to++ = 1;
02964             }
02965             *to++ = 2; *to++ = 1; *to++ = 3;
02966         }
02967     }
02968     else if ( type2 == 3 ) {
02969         t3 = to;
02970         *to++ = 0; *to++ = 0; FILLARG(to) *to++ = 0;
02971         *to++ = SYMBOL; *to++ = 4; *to++ = t1[1]; *to++ = 1;
02972         tt1 = scratch; tt1[1] = 1; size1 = 1;
02973         tt2 = t2+ARGHEAD+1; size2 = (ABS(t2[*t2-1])-1)/2;
02974         nosymbols = 0;
02975         goto DoCoeffi;
02976     }
02977     else {
02978         tt1 = scratch; tt1[0] = 1; size1 = 1;
02979         t3 = to;
02980         *to++ = 0; *to++ = 0; FILLARG(to) *to++ = 0;
02981         *to++ = SYMBOL; *to++ = 0;
02982         tt2 = t2 + ARGHEAD+3; it2 = tt2[-1]-2;
02983         while ( it2 > 0 ) {
02984             if ( *tt2 == t1[1] ) {
02985                 pow = tt2[1]+1;
02986                 root = symbols[*tt2].maxpower;
02987                 if ( pow >= root ) pow -= root;
02988                 if ( ( symbols[*tt2].complex & VARTYPEMINUS ) == VARTYPEMINUS ) {
02989                     if ( ( root & 1 ) == 0 && pow >= root/2 ) {
02990                         pow -= root/2; sgn3 = -sgn3;
02991                     }
02992                 }
02993                 if ( pow != 0 ) {
02994                     *to++ = *tt2; *to++ = pow;
02995                 }
02996                 tt2 += 2; it2 -= 2;
02997                 break;
02998             }
02999             else if ( t1[1] < *tt2 ) {

```

```

03000          *to++ = t1[1]; *to++ = 1; break;
03001      }
03002      else {
03003          *to++ = *tt2++; *to++ = *tt2++; it2 -= 2;
03004          if ( it2 <= 0 ) { *to++ = t1[1]; *to++ = 1; }
03005      }
03006  }
03007  while ( it2 > 0 ) { *to++ = *tt2++; *to++ = *tt2++; it2 -= 2; }
03008  if ( (to - t3) > ARGHEAD+3 ) {
03009      t3[ARGHEAD+2] = (to-t3)-ARGHEAD-1; /* size of the SYMBOL field */
03010      nosymbols = 0;
03011  }
03012  else {
03013      to = t3+ARGHEAD+1; /* no SYMBOL field */
03014  }
03015  size2 = (ABS(t2[*t2-1])-1)/2;
03016  goto DoCoeffi;
03017  }
03018  break;
03019  case 3:
03020      if ( type2 == 3 ) {
03021  /*
03022          Both are numeric
03023  */
03024          tt1 = t1+ARGHEAD+1; size1 = (ABS(t1[*t1-1])-1)/2;
03025          tt2 = t2+ARGHEAD+1; size2 = (ABS(t2[*t2-1])-1)/2;
03026          t3 = to;
03027          *to++ = 0; *to++ = 0; FILLARG(to) *to++ = 0;
03028          goto DoCoeffi;
03029      }
03030      else {
03031  /*
03032          t1 is (long) numeric, t2 has the symbol(s).
03033  */
03034          tt1 = t1+ARGHEAD+1; size1 = (ABS(t1[*t1-1])-1)/2;
03035          tt2 = t2+ARGHEAD+1; tt2 += tt2[1]; size2 = (ABS(t2[*t2-1])-1)/2;
03036          t3 = to; i = tt2 - t2; r = t2;
03037          NCOPY(to,r,i)
03038          nosymbols = 0;
03039          goto DoCoeffi;
03040      }
03041  break;
03042  case 4:
03043  /*
03044          Both have roots of unity
03045          1: Merge the lists and simplify if possible
03046  */
03047          tt1 = t1+ARGHEAD+3; it1 = tt1[-1]-2;
03048          tt2 = t2+ARGHEAD+3; it2 = tt2[-1]-2;
03049          t3 = to;
03050          *to++ = 0; *to++ = 0; FILLARG(to)
03051          *to++ = 0; *to++ = SYMBOL; *to++ = 0;
03052          while ( it1 > 0 && it2 > 0 ) {
03053              if ( *tt1 == *tt2 ) {
03054                  pow = tt1[1]+tt2[1];
03055                  root = symbols[*tt1].maxpower;
03056                  if ( pow >= root ) pow -= root;
03057                  if ( ( symbols[*tt1].complex & VARTYPEMINUS ) == VARTYPEMINUS ) {
03058                      if ( ( root & 1 ) == 0 && pow >= root/2 ) {
03059                          pow -= root/2; sgn3 = -sgn3;
03060                      }
03061                  }
03062                  if ( pow != 0 ) {
03063                      *to++ = *tt1; *to++ = pow;
03064                  }
03065                  tt1 += 2; tt2 += 2; it1 -= 2; it2 -= 2;
03066              }
03067              else if ( *tt1 < *tt2 ) {
03068                  *to++ = *tt1++; *to++ = *tt1++; it1 -= 2;
03069              }
03070              else {
03071                  *to++ = *tt2++; *to++ = *tt2++; it2 -= 2;
03072              }
03073          }
03074          while ( it1 > 0 ) { *to++ = *tt1++; *to++ = *tt1++; it1 -= 2; }
03075          while ( it2 > 0 ) { *to++ = *tt2++; *to++ = *tt2++; it2 -= 2; }
03076          if ( (to - t3) > ARGHEAD+3 ) {
03077              t3[ARGHEAD+2] = (to-t3)-ARGHEAD-1; /* size of the SYMBOL field */
03078              nosymbols = 0;
03079          }
03080          else {
03081              to = t3+ARGHEAD+1; /* no SYMBOL field */
03082          }
03083          size1 = (ABS(t1[*t1-1])-1)/2;
03084          size2 = (ABS(t2[*t2-1])-1)/2;
03085  /*
03086          Now tt1 and tt2 are pointing at their coefficients.

```

```

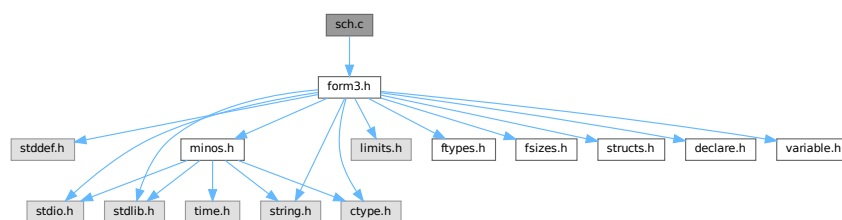
03087         sgn1 is the sign of 1, sgn2 is the sign of 2 and sgn3 is an extra
03088         overall sign.
03089     */
03090 DoCoefffi:
03091     if ( AddLong((UWORD *)tt1,size1,(UWORD *)tt2,size2,(UWORD *)to,&size3) ) {
03092         MLOCK(ErrorMessageLock);
03093         MesPrint("Called from StuffRootAdd");
03094         MUNLOCK(ErrorMessageLock);
03095         Terminate(-1);
03096     }
03097     sgn = sgn1*sgn2*sgn3;
03098     if ( nosymbols && size3 == 1 ) {
03099         if ( (UWORD)(to[0]) <= MAXPOSITIVE && sgn > 0 ) {
03100             sgn1 = to[0];
03101             to = t3; *to++ = -SNUMBER; *to++ = sgn1;
03102         }
03103         else if ( (UWORD)(to[0]) <= (MAXPOSITIVE+1) && sgn < 0 ) {
03104             sgn1 = to[0];
03105             to = t3; *to++ = -SNUMBER; *to++ = -sgn1;
03106         }
03107         else goto genericcoef;
03108     }
03109     else {
03110 genericcoef:
03111         to += size3;
03112         sgn = sgn*(2*size3+1);
03113         *to++ = 1;
03114         while ( size3 > 1 ) { *to++ = 0; size3--; }
03115         *to++ = sgn;
03116         t3[0] = to - t3;
03117         t3[ARGHEAD] = t3[0] - ARGHEAD;
03118     }
03119     break;
03120 }
03121 return(to);
03122 }
03123 #endif
03124
03125 /*
03126 #] StuffRootAdd :
03127 */

```

4.122 sch.c File Reference

```
#include "form3.h"
```

Include dependency graph for sch.c:



Macros

- #define `va_dcl` int va_alist;
- #define `va_start`(list) list = (UBYTE *) &va_alist
- #define `va_end`(list)
- #define `va_arg`(list, mode) (((mode *) (list += sizeof(mode)))[-1])

Typedefs

- typedef UBYTE * [va_list](#)

Functions

- UBYTE * [StrCopy](#) (UBYTE *from, UBYTE *to)
- void [AddToLine](#) (UBYTE *s)
- void [FiniLine](#) (void)
- void [IniLine](#) (WORD extrablank)
- void [LongToLine](#) (UWORD *a, WORD na)
- void [RatToLine](#) (UWORD *a, WORD na)
- void [TalToLine](#) (UWORD x)
- void [TokenToLine](#) (UBYTE *s)
- UBYTE * [CodeToLine](#) (WORD number, UBYTE *Out)
- void [MultiplyToLine](#) (void)
- UBYTE * [AddArrayIndex](#) (WORD num, UBYTE *out)
- void [PrtTerms](#) (void)
- UBYTE * [WrtPower](#) (UBYTE *Out, WORD Power)
- void [PrintTime](#) (UBYTE *mess)
- void [WriteLists](#) (void)
- void [WriteDictionary](#) (DICTIONARY *dict)
- void [WriteArgument](#) (WORD *t)
- WORD [WriteSubTerm](#) (WORD *sterm, WORD first)
- WORD [WritInnerTerm](#) (WORD *term, WORD first)
- WORD [WriteTerm](#) (WORD *term, WORD *lbrac, WORD first, WORD prtf, WORD br)
- WORD [WriteExpression](#) (WORD *terms, LONG ltot)
- WORD [WriteAll](#) (void)
- WORD [WriteOne](#) (UBYTE *name, int alreadyinline, int nosemi, WORD plus)

4.122.1 Detailed Description

Contains the functions that deal with the writing of expressions/terms in a textual representation. (Dutch schrijven = to write)

Definition in file [sch.c](#).

4.122.2 Macro Definition Documentation

4.122.2.1 [va_dcl](#)

```
#define va_dcl int va_alist;
```

Definition at line 48 of file [sch.c](#).

4.122.2.2 [va_start](#)

```
#define va_start(  
    list ) list = (UBYTE *) &va_alist
```

Definition at line 49 of file [sch.c](#).

4.122.2.3 va_end

```
#define va_end(  
    list )
```

Definition at line 50 of file [sch.c](#).

4.122.2.4 va_arg

```
#define va_arg(  
    list,  
    mode ) (((mode *) (list += sizeof(mode))) [-1])
```

Definition at line 51 of file [sch.c](#).

4.122.3 Typedef Documentation

4.122.3.1 va_list

```
typedef UBYTE* va_list
```

Definition at line 47 of file [sch.c](#).

4.122.4 Function Documentation

4.122.4.1 StrCopy()

```
UBYTE * StrCopy (  
    UBYTE * from,  
    UBYTE * to )
```

Definition at line 67 of file [sch.c](#).

4.122.4.2 AddToLine()

```
void AddToLine (  
    UBYTE * s )
```

Definition at line 82 of file [sch.c](#).

4.122.4.3 FiniLine()

```
void FiniLine (  
    void )
```

Definition at line 182 of file [sch.c](#).

4.122.4.4 IniLine()

```
void IniLine (
    WORD extrablk )
```

Definition at line [267](#) of file [sch.c](#).

4.122.4.5 LongToLine()

```
void LongToLine (
    UWORD * a,
    WORD na )
```

Definition at line [303](#) of file [sch.c](#).

4.122.4.6 RatToLine()

```
void RatToLine (
    UWORD * a,
    WORD na )
```

Definition at line [337](#) of file [sch.c](#).

4.122.4.7 TalToLine()

```
void TalToLine (
    UWORD x )
```

Definition at line [522](#) of file [sch.c](#).

4.122.4.8 TokenToLine()

```
void TokenToLine (
    UBYTE * s )
```

Definition at line [551](#) of file [sch.c](#).

4.122.4.9 CodeToLine()

```
UBYTE * CodeToLine (
    WORD number,
    UBYTE * Out )
```

Definition at line [658](#) of file [sch.c](#).

4.122.4.10 MultiplyToLine()

```
void MultiplyToLine (  
    void )
```

Definition at line 671 of file [sch.c](#).

4.122.4.11 AddArrayIndex()

```
UBYTE * AddArrayIndex (  
    WORD num,  
    UBYTE * out )
```

Definition at line 696 of file [sch.c](#).

4.122.4.12 PrtTerms()

```
void PrtTerms (  
    void )
```

Definition at line 716 of file [sch.c](#).

4.122.4.13 WrtPower()

```
UBYTE * WrtPower (  
    UBYTE * Out,  
    WORD Power )
```

Definition at line 738 of file [sch.c](#).

4.122.4.14 PrintTime()

```
void PrintTime (  
    UBYTE * mess )
```

Definition at line 784 of file [sch.c](#).

4.122.4.15 WriteLists()

```
void WriteLists (  
    void )
```

Definition at line 810 of file [sch.c](#).

4.122.4.16 WriteDictionary()

```
void WriteDictionary (  
    DICTIONARY * dict )
```

Definition at line 1307 of file [sch.c](#).

4.122.4.17 WriteArgument()

```
void WriteArgument (
    WORD * t )
```

Definition at line 1424 of file [sch.c](#).

4.122.4.18 WriteSubTerm()

```
WORD WriteSubTerm (
    WORD * sterm,
    WORD first )
```

Definition at line 1542 of file [sch.c](#).

4.122.4.19 WriteInnerTerm()

```
WORD WriteInnerTerm (
    WORD * term,
    WORD first )
```

Definition at line 1951 of file [sch.c](#).

4.122.4.20 WriteTerm()

```
WORD WriteTerm (
    WORD * term,
    WORD * lbrac,
    WORD first,
    WORD prtft,
    WORD br )
```

Definition at line 2148 of file [sch.c](#).

4.122.4.21 WriteExpression()

```
WORD WriteExpression (
    WORD * terms,
    LONG ltot )
```

Definition at line 2470 of file [sch.c](#).

4.122.4.22 WriteAll()

```
WORD WriteAll (
    void )
```

Definition at line 2501 of file [sch.c](#).

4.122.4.23 WriteOne()

```
WORD WriteOne (
    UBYTE * name,
    int alreadyinline,
    int nosemi,
    WORD plus )
```

Definition at line 2727 of file [sch.c](#).

4.123 sch.c

[Go to the documentation of this file.](#)

```
00001
00006 /* #[] License : */
00007 /*
00008 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00009 * When using this file you are requested to refer to the publication
00010 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00011 * This is considered a matter of courtesy as the development was paid
00012 * for by FOM the Dutch physics granting agency and we would like to
00013 * be able to track its scientific use to convince FOM of its value
00014 * for the community.
00015 *
00016 * This file is part of FORM.
00017 *
00018 * FORM is free software: you can redistribute it and/or modify it under the
00019 * terms of the GNU General Public License as published by the Free Software
00020 * Foundation, either version 3 of the License, or (at your option) any later
00021 * version.
00022 *
00023 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00024 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00025 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00026 * details.
00027 *
00028 * You should have received a copy of the GNU General Public License along
00029 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00030 */
00031 /* #[] License : */
00032 /*
00033 #[] Includes : sch.c
00034 */
00035
00036 #include "form3.h"
00037
00038 #ifdef ANSI
00039 #include <stdarg.h>
00040 #else
00041 #ifdef mBSD
00042 #include <varargs.h>
00043 #else
00044 #ifdef VMS
00045 #include <varargs.h>
00046 #else
00047 typedef UBYTE *va_list;
00048 #define va_dcl int va_alist;
00049 #define va_start(list) list = (UBYTE *) &va_alist
00050 #define va_end(list)
00051 #define va_arg(list,mode) (((mode *) (list += sizeof(mode)))[-1])
00052 #endif
00053 #endif
00054 #endif
00055
00056 static int startinline = 0;
00057 static char fcontchar = '&';
00058 static int noextralinefeed = 0;
00059 static int lowestlevel = 1;
00060
00061 /*
00062 #[] Includes :
00063 #[] schryf-Utilities :
00064 #[] StrCopy : UBYTE *StrCopy(from,to)
00065 */
00066
00067 UBYTE *StrCopy(UBYTE *from, UBYTE *to)
```

```

00068 {
00069     while( ( *to++ = *from++ ) != 0 );
00070     return(to-1);
00071 }
00072
00073 /*
00074     #] StrCopy :
00075     #[ AddToLine :          void AddToLine(s)
00076
00077     Puts the characters of s in the outputline. If the line becomes
00078     filled it is written.
00079
00080 */
00081
00082 void AddToLine(UBYTE *s)
00083 {
00084     UBYTE *Out;
00085     LONG num;
00086     int i;
00087     if ( AO.OutInBuffer ) { AddToDollarBuffer(s); return; }
00088     Out = AO.OutFill;
00089     while ( *s ) {
00090         if ( Out >= AO.OutStop ) {
00091             if ( AC.OutputMode == FORTRANMODE && AC.IsFortran90 == ISFORTRAN90 ) {
00092                 *Out++ = fcontchar;
00093             }
00094             #ifdef WITHRETURN
00095                 *Out++ = CARRIAGERETURN;
00096             #endif
00097             *Out++ = LINEFEED;
00098             AO.PortFirst = 0;
00099             num = Out - AO.OutputLine;
00100
00101             if ( AC.LogHandle >= 0 ) {
00102                 if ( WriteFile(AC.LogHandle,AO.OutputLine+startinline
00103                               ,num-startinline) != (num-startinline) ) {
00104                     /*
00105                         We cannot write to an otherwise open log file.
00106                         The disk could be full of course.
00107                     */
00108                     #ifdef DEBUGGER
00109                         if ( BUG.logfileflag == 0 ) {
00110                             fprintf(stderr,"Panic: Cannot write to log file! Disk full?\n");
00111                             BUG.logfileflag = 1;
00112                         }
00113                         BUG.eflag = 1; BUG.printflag = 1;
00114                     #else
00115                         Terminate(-1);
00116                     #endif
00117                 }
00118             }
00119
00120             if ( ( AO.PrintType & PRINTLFILE ) == 0 ) {
00121                 #ifdef WITHRETURN
00122                 if ( num > 1 && AO.OutputLine[num-2] == CARRIAGERETURN ) {
00123                     AO.OutputLine[num-2] = LINEFEED;
00124                     num--;
00125                 }
00126                 #endif
00127                 if ( WriteFile(AM.StdOut,AO.OutputLine+startinline
00128                               ,num-startinline) != (num-startinline) ) {
00129                     #ifdef DEBUGGER
00130                         if ( BUG.stdoutflag == 0 ) {
00131                             fprintf(stderr,"Panic: Cannot write to standard output!\n");
00132                             BUG.stdoutflag = 1;
00133                         }
00134                         BUG.eflag = 1; BUG.printflag = 1;
00135                     #else
00136                         Terminate(-1);
00137                     #endif
00138                 }
00139             }
00140             /* thomasr 23/04/09: A continuation line has been started.
00141              * In Fortran90 we do not want a space after the initial
00142              * '&' character otherwise we might end up with something
00143              * like:
00144              * ... 2.&
00145              * & 0 ...
00146              */
00147             startinline = 0;
00148             for ( i = 0; i < AO.OutSkip; i++ ) AO.OutputLine[i] = ' ';
00149             Out = AO.OutputLine + AO.OutSkip;
00150             if ( ( AC.OutputMode == FORTRANMODE
00151                 || AC.OutputMode == PFORTRANMODE ) && AO.OutSkip == 7 ) {
00152                 /* thomasr 23/04/09: fix leading blank in fortran90 mode */
00153                 if(AC.IsFortran90 == ISFORTRAN90) {
00154                     Out[-1] = fcontchar;

```

```

00155         }
00156         else {
00157             Out[-2] = fcontchar;
00158             Out[-1] = ' ';
00159         }
00160     }
00161     if ( AO.IsBracket ) { *Out++ = ' ';
00162         if ( AC.OutputSpaces == NORMALFORMAT ) {
00163             *Out++ = ' '; *Out++ = ' '; }
00164     }
00165     *Out = '\0';
00166     if ( AC.OutputMode == FORTRANMODE
00167         || ( AC.OutputMode == CMODE && AO.FactorMode == 0 )
00168         || AC.OutputMode == PFORTRANMODE )
00169         AO.InFbrack++;
00170     }
00171     *Out++ = *s++;
00172 }
00173 *Out = '\0';
00174 AO.OutFill = Out;
00175 }
00176
00177 /*
00178     #] AddToLine :
00179     #[ FiniLine :      void FiniLine()
00180 */
00181
00182 void FiniLine(void)
00183 {
00184     UBYTE *Out;
00185     WORD i;
00186     LONG num;
00187     if ( AO.OutInBuffer ) return;
00188     Out = AO.OutFill;
00189     while ( Out > AO.OutputLine ) {
00190         if ( Out[-1] == ' ' ) Out--;
00191         else break;
00192     }
00193     i = (WORD)(Out-AO.OutputLine);
00194     if ( noextralinefeed == 0 ) {
00195         if ( AC.OutputMode == FORTRANMODE && AC.IsFortran90 == ISFORTRAN90
00196             && Out > AO.OutputLine ) {
00197             /*
00198                 *Out++ = fcontchar;
00199             */
00200         }
00201         #ifdef WITHRETURN
00202         *Out++ = CARRIAGERETURN;
00203         #endif
00204         *Out++ = LINEFEED;
00205         AO.FortFirst = 0;
00206     }
00207     num = Out - AO.OutputLine;
00208
00209     if ( AC.LogHandle >= 0 ) {
00210         if ( WriteFile(AC.LogHandle,AO.OutputLine+startinline,
00211             ,num-startinline) != (num-startinline) ) {
00212             #ifdef DEBUGGER
00213                 if ( BUG.logfileflag == 0 ) {
00214                     fprintf(stderr,"Panic: Cannot write to log file! Disk full?\n");
00215                     BUG.logfileflag = 1;
00216                 }
00217                 BUG.eflag = 1; BUG.printflag = 1;
00218             #else
00219                 Terminate(-1);
00220             #endif
00221         }
00222     }
00223
00224     if ( ( AO.PrintType & PRINTLFILE ) == 0 ) {
00225         #ifdef WITHRETURN
00226         if ( num > 1 && AO.OutputLine[num-2] == CARRIAGERETURN ) {
00227             AO.OutputLine[num-2] = LINEFEED;
00228             num--;
00229         }
00230         #endif
00231         if ( WriteFile(AM.Stdout,AO.OutputLine+startinline,
00232             ,num-startinline) != (num-startinline) ) {
00233             #ifdef DEBUGGER
00234                 if ( BUG.stdoutflag == 0 ) {
00235                     fprintf(stderr,"Panic: Cannot write to standard output!\n");
00236                     BUG.stdoutflag = 1;
00237                 }
00238                 BUG.eflag = 1; BUG.printflag = 1;
00239             #else
00240                 Terminate(-1);
00241             #endif

```

```

00242     }
00243 }
00244 startinline = 0;
00245 if ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE
00246     || ( AC.OutputMode == CMODE && AO.FactorMode == 0 ) ) AO.InFbrack++;
00247 Out = AO.OutputLine;
00248 AO.OutStop = Out + AC.LineLength;
00249 i = AO.OutSkip;
00250 while ( --i >= 0 ) *Out++ = ' ';
00251 if ( ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE )
00252     && AO.OutSkip == 7 ) {
00253     Out[-2] = fcontchar;
00254     Out[-1] = ' ';
00255 }
00256 AO.OutFill = Out;
00257 }
00258
00259 /*
00260     #] FiniLine :
00261     #[ IniLine :          void IniLine(extrablank)
00262
00263     Initializes the output line for the type of output
00264 */
00265 void IniLine(WORD extrablank)
00266 {
00267     UBYTE *Out;
00270     Out = AO.OutputLine;
00271     AO.OutStop = Out + AC.LineLength;
00272     *Out++ = ' ';
00273     *Out++ = ' ';
00274     *Out++ = ' ';
00275     *Out++ = ' ';
00276     *Out++ = ' ';
00277     if ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE ) {
00278         *Out++ = fcontchar;
00279         AO.OutSkip = 7;
00280     }
00281     else
00282         AO.OutSkip = 6;
00283     *Out++ = ' ';
00284     while ( extrablank > 0 ) {
00285         *Out++ = ' ';
00286         extrablank--;
00287     }
00288     AO.OutFill = Out;
00289 }
00290
00291 /*
00292     #] IniLine :
00293     #[ LongToLine :          void LongToLine(a,na)
00294
00295     Puts a Long integer in the output line. If it is only a single
00296     word long it is put in the line as a single token.
00297     The sign of a is ignored.
00298 */
00299 static UBYTE *LLscratch = 0;
00300
00301 void LongToLine(WORD *a, WORD na)
00302 {
00303     UBYTE *OutScratch;
00306     if ( LLscratch == 0 ) {
00307         LLscratch = (UBYTE *)Malloca(4*(AM.MaxTal*sizeof(WORD)+2)*sizeof(UBYTE), "LongToLine");
00308     }
00309     OutScratch = LLscratch;
00310     if ( na < 0 ) na = -na;
00311     if ( na > 1 ) {
00312         PrtLong(a, na, OutScratch);
00313         if ( AO.NoSpacesInNumbers || AC.OutputMode == REDUCEMODE ) {
00314             AO.BlockSpaces = 1;
00315             TokenToLine(OutScratch);
00316             AO.BlockSpaces = 0;
00317         }
00318         else {
00319             TokenToLine(OutScratch);
00320         }
00321     }
00322     else if ( !na ) TokenToLine((UBYTE *)"0");
00323     else TalToLine(*a);
00324 }
00325
00326 /*
00327     #] LongToLine :
00328     #[ RatToLine :          void RatToLine(a,na)

```



```

00329
00330     Puts a rational number in the output line. The sign is ignored.
00331
00332 */
00333
00334 static UBYTE *RLscratch = 0;
00335 static UWORD *RLscratE = 0;
00336
00337 void RatToLine(UWORD *a, WORD na)
00338 {
00339     GETIDENTITY
00340     WORD adenom, anumer;
00341     UWORD maxInt;
00342
00343     if ( AC.OutputMode == CMODE ) {
00344         // In C, integer literals over 2^32-1 are automatically promoted to longer types up to
00345         // unsigned long long, however FORTRAN has always given integers over 2^32-1 a floating suffix
00346         // in C mode. Retain that behaviour here for now. In the future, maxInt could be a user-
00347         // configurable parameter.
00348         maxInt = 4294967295;
00349     }
00350     else {
00351         // In Fortran modes, literals over 2^31-1 should be printed with a float suffix
00352         maxInt = 2147483647;
00353     }
00354
00355     if ( na < 0 ) na = -na;
00356
00357     if ( AC.OutNumberType == RATIONALMODE ) {
00358
00359         // Work out what is to be printed:
00360         WORD isOne = 0, isOneOver = 0, isIntegral = 0;
00361         WORD isLongNum = 0, isLongDen = 0;
00362         UnPack(a, na, &adenom, &anumer);
00363         if ( na == 1 && a[0] == 1 && a[1] == 1 ) { isOne = 1; isIntegral = 1; }
00364         else if ( adenom == 1 && a[na] == 1 ) { isIntegral = 1; }
00365         else if ( anumer == 1 && a[0] == 1 ) { isOneOver = 1; }
00366         if ( anumer > 1 || ( anumer == 1 && a[0] > maxInt ) ) { isLongNum = 1; };
00367         if ( adenom > 1 || ( adenom == 1 && a[na] > maxInt ) ) { isLongDen = 1; };
00368
00369         // Now sort out the float suffix for the numerator:
00370         UBYTE* suffNum = (UBYTE*)" ";
00371         UBYTE* suffDen = (UBYTE*)" ";
00372         if ( isLongNum || !isIntegral || AC.Fortran90Kind || ( AO.DoubleFlag & 4 ) == 4 ) {
00373             if ( AC.OutputMode == FORTRANMODE && AC.IsFortran90 == ISFORTRAN90 ) {
00374                 if ( AC.Fortran90Kind ) { suffNum = AC.Fortran90Kind; }
00375                 else { suffNum = (UBYTE*)". "; }
00376             }
00377             else if ( AC.OutputMode == FORTRANMODE || AC.OutputMode == CMODE ) {
00378                 if ( ( AO.DoubleFlag & 2 ) == 2 ) { suffNum = (UBYTE*)".Q0"; }
00379                 else if ( ( AO.DoubleFlag & 1 ) == 1 ) { suffNum = (UBYTE*)".D0"; }
00380                 else { suffNum = (UBYTE*)". "; }
00381             }
00382         }
00383         // The same again, for the denominator:
00384         if ( isLongDen || !isIntegral || AC.Fortran90Kind || ( AO.DoubleFlag & 4 ) == 4 ) {
00385             if ( AC.OutputMode == FORTRANMODE && AC.IsFortran90 == ISFORTRAN90 ) {
00386                 if ( AC.Fortran90Kind ) { suffDen = AC.Fortran90Kind; }
00387                 else { suffDen = (UBYTE*)". "; }
00388             }
00389             else if ( AC.OutputMode == FORTRANMODE || AC.OutputMode == CMODE ) {
00390                 if ( ( AO.DoubleFlag & 2 ) == 2 ) { suffDen = (UBYTE*)".Q0"; }
00391                 else if ( ( AO.DoubleFlag & 1 ) == 1 ) { suffDen = (UBYTE*)".D0"; }
00392                 else { suffDen = (UBYTE*)". "; }
00393             }
00394         }
00395         // In PFORTAN mode, rationals don't get a suffix if the integers are not long
00396         // Also the suffix is at least ".D0" (and never ".").
00397         if ( AC.OutputMode == PFORTANMODE ) {
00398             if ( isLongNum ) {
00399                 if ( ( AO.DoubleFlag & 2 ) == 2 ) { suffNum = (UBYTE*)".Q0"; }
00400                 else { suffNum = (UBYTE*)".D0"; }
00401             }
00402             if ( isLongDen ) {
00403                 if ( ( AO.DoubleFlag & 2 ) == 2 ) { suffDen = (UBYTE*)".Q0"; }
00404                 else { suffDen = (UBYTE*)".D0"; }
00405             }
00406         }
00407
00408         // Finally, print the number:
00409         // In PFORTAN we use
00410         //   one      if denom = numerator = 1
00411         //   integer   if denom = 1
00412         //   (one/integer) if numerator = 1
00413         //   ((one*integer)/integer) in the general case
00414         if ( AC.OutputMode == PFORTANMODE ) {
00415             if ( isOne ) {

```

```

00416         AddToLine((UBYTE *) "one");
00417     }
00418     else if ( isOneOver ) {
00419         AddToLine((UBYTE *) "(one/");
00420         LongToLine(a+na,adenom);
00421         AddToLine(suffDen);
00422         AddToLine((UBYTE*)")");
00423     }
00424     else if ( isIntegral ) {
00425         LongToLine(a,anumer);
00426         AddToLine(suffNum);
00427     }
00428     else {
00429         // rational
00430         AddToLine((UBYTE *) "((one*");
00431         LongToLine(a,anumer);
00432         AddToLine(suffNum);
00433         AddToLine((UBYTE*)")/");
00434         LongToLine(a+na,adenom);
00435         AddToLine(suffDen);
00436         AddToLine((UBYTE*)")");
00437     }
00438 }
00439 // All other modes use the same printing code:
00440 else {
00441     // Numerator:
00442     LongToLine(a,anumer);
00443     AddToLine(suffNum);
00444     // Denominator, if it is not 1:
00445     if (!isIntegral) {
00446         AddToLine((UBYTE*)"/");
00447         LongToLine(a+na,adenom);
00448         AddToLine(suffDen);
00449     }
00450 }
00451 }
00452
00453 else {
00454 /*
00455     This is the float mode
00456 */
00457     UBYTE *OutScratch;
00458     WORD exponent = 0, i, ndig, newl;
00459     UWORD *c, *den, b = 10, dig[10];
00460     UBYTE *o, *out, cc;
00461 /*
00462     First we have to adjust the numerator and denominator
00463 */
00464     if ( RLscratch == 0 ) {
00465         RLscratch = (UBYTE *) Malloc1(4*(AM.MaxTal+2)*sizeof(UBYTE), "RatToLine");
00466         RLscratE = (UWORD *) Malloc1(2*(AM.MaxTal+2)*sizeof(UWORD), "RatToLine");
00467     }
00468     out = OutScratch = RLscratch;
00469     c = RLscratE; for ( i = 0; i < 2*na; i++ ) c[i] = a[i];
00470     UnPack(c, na, &adenom, &anumer);
00471     while ( BigLong(c, anumer, c+na, adenom) >= 0 ) {
00472         Divvy(BHEAD c, &na, &b, 1);
00473         UnPack(c, na, &adenom, &anumer);
00474         exponent++;
00475     }
00476     while ( BigLong(c, anumer, c+na, adenom) < 0 ) {
00477         Mully(BHEAD c, &na, &b, 1);
00478         UnPack(c, na, &adenom, &anumer);
00479         exponent--;
00480     }
00481 /*
00482     Now division will give a number between 1 and 9
00483 */
00484     den = c + na; i = 1;
00485     DivLong(c, anumer, den, adenom, dig, &ndig, c, &newl);
00486     *out++ = (UBYTE)(dig[0]+'0'); *out++ = '.';
00487     while ( newl && i < AC.OutNumberType ) {
00488         Pack(c, &newl, den, adenom);
00489         Mully(BHEAD c, &newl, &b, 1);
00490         na = newl;
00491         UnPack(c, na, &adenom, &anumer);
00492         den = c + na;
00493         DivLong(c, anumer, den, adenom, dig, &ndig, c, &newl);
00494         if ( ndig == 0 ) *out++ = '0';
00495         else *out++ = (UBYTE)(dig[0]+'0');
00496         i++;
00497     }
00498     *out++ = 'E';
00499     if ( exponent < 0 ) { exponent = -exponent; *out++ = '-'; }
00500     else { *out++ = '+'; }
00501     o = out;
00502     do {

```

```

00503         *out++ = (UBYTE)((exponent % 10)+'0');
00504         exponent /= 10;
00505     } while ( exponent );
00506     *out = 0; out--;
00507     while ( o < out ) { cc = *o; *o = *out; *out = cc; o++; out--; }
00508     TokenToLine(OutScratch);
00509 }
00510 }
00511
00512 /*
00513     #] RatToLine :
00514     #[ TalToLine :          void TalToLine(x)
00515
00516     Writes the unsigned number x to the output as a single token.
00517     Par indicates the number of leading blanks in the line.
00518     This parameter is needed here for the WriteLists routine.
00519
00520 */
00521
00522 void TalToLine(UWORD x)
00523 {
00524     UBYTE t[BITSINWORD/3+1];
00525     UBYTE *s;
00526     WORD i = 0, j;
00527     s = t;
00528     do { *s++ = (UBYTE)((x % 10)+'0'); i++; } while ( ( x /= 10 ) != 0 );
00529     *s-- = '\0';
00530     j = ( i - 1 ) >> 1;
00531     while ( j >= 0 ) {
00532         i = t[j]; t[j] = s[-j]; s[-j] = (UBYTE)i; j--;
00533     }
00534     TokenToLine(t);
00535 }
00536
00537 /*
00538     #] TalToLine :
00539     #[ TokenToLine :      void TokenToLine(s)
00540
00541     Puts s in the output buffer. If it doesn't fit the buffer is
00542     flushed first. This routine keeps tokens as one unit.
00543     Par indicates the number of leading blanks in the line.
00544     This parameter is needed here for the WriteLists routine.
00545
00546     Remark (27-oct-2007): i and j must be longer than WORD!
00547     It can happen that a number is so long that it has more than 2^15 or 2^31
00548     digits!
00549 */
00550
00551 void TokenToLine(UBYTE *s)
00552 {
00553     UBYTE *t, *Out;
00554     LONG num, i = 0, j;
00555     if ( AO.OutInBuffer ) { AddToDollarBuffer(s); return; }
00556     t = s; Out = AO.OutFill;
00557     while ( *t++ ) i++;
00558     while ( i > 0 ) {
00559         if ( ( Out + i ) >= AO.OutStop && ( ( i < ((AC.LineLength-AO.OutSkip)>1) )
00560             || ( (AO.OutStop-Out) < (i>2) ) ) ) {
00561             if ( AC.OutputMode == FORTRANMODE && AC.IsFortran90 == ISFORTRAN90 ) {
00562                 *Out++ = fcontchar;
00563             }
00564             #ifdef WITHRETURN
00565             *Out++ = CARRIAGERETURN;
00566             #endif
00567             *Out++ = LINEFEED;
00568             AO.FortFirst = 0;
00569             num = Out - AO.OutputLine;
00570             if ( AC.LogHandle >= 0 ) {
00571                 if ( WriteFile(AC.LogHandle, AO.OutputLine+startinline,
00572                     num-startinline) != (num-startinline) ) {
00573                     #ifdef DEBUGGER
00574                     if ( BUG.logfileflag == 0 ) {
00575                         fprintf(stderr, "Panic: Cannot write to log file! Disk full?\n");
00576                         BUG.logfileflag = 1;
00577                     }
00578                     BUG.eflag = 1; BUG.printflag = 1;
00579                 #else
00580                 Terminate(-1);
00581                 #endif
00582             }
00583             }
00584             if ( ( AO.PrintType & PRINTLFILE ) == 0 ) {
00585                 #ifdef WITHRETURN
00586                 if ( num > 1 && AO.OutputLine[num-2] == CARRIAGERETURN ) {
00587                     AO.OutputLine[num-2] = LINEFEED;
00588                     num--;
00589                 }

```

```

00590 #endif
00591         if ( WriteFile(AM.Stdout,AO.OutputLine+startinline,
00592             num-startinline) != (num-startinline) ) {
00593 #ifdef DEBUGGER
00594             if ( BUG.stdoutflag == 0 ) {
00595                 fprintf(stderr,"Panic: Cannot write to standard output!\n");
00596                 BUG.stdoutflag = 1;
00597             }
00598             BUG.eflag = 1; BUG.printflag = 1;
00599 #else
00600             Terminate(-1);
00601 #endif
00602         }
00603     }
00604     startinline = 0;
00605     Out = AO.OutputLine;
00606     if ( AO.BlockSpaces == 0 ) {
00607         for ( j = 0; j < AO.OutSkip; j++ ) { *Out++ = ' '; }
00608         if ( ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE ) ) {
00609             if ( AO.OutSkip == 7 ) {
00610                 Out[-2] = fcontchar;
00611                 Out[-1] = ' ';
00612             }
00613         }
00614     }
00615     /*
00616     Out = AO.OutputLine + AO.OutSkip;
00617     if ( ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE )
00618         && AO.OutSkip == 7 ) {
00619         Out[-2] = fcontchar;
00620         Out[-1] = ' ';
00621     }
00622     else {
00623         for ( j = 0; j < AO.OutSkip; j++ ) { AO.OutputLine[j] = ' '; }
00624     }
00625     */
00626     if ( AO.IsBracket ) { *Out++ = ' '; *Out++ = ' '; *Out++ = ' '; }
00627     *Out = '\0';
00628     if ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE
00629         || ( AC.OutputMode == CMODE && AO.FactorMode == 0 ) ) AO.InFbrack++;
00630 }
00631 if ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE ) {
00632     /* Very long numbers */
00633     if ( i > (WORD)(AO.OutStop-Out) ) j = (WORD)(AO.OutStop - Out);
00634     else j = i;
00635     i -= j;
00636     NCOPYB(Out,s,j);
00637 }
00638 else {
00639     if ( i > (WORD)(AO.OutStop-Out) ) j = (WORD)(AO.OutStop - Out - 1);
00640     else j = i;
00641     i -= j;
00642     NCOPYB(Out,s,j);
00643     if ( i > 0 ) *Out++ = '\\';
00644 }
00645 }
00646 *Out = '\0';
00647 AO.OutFill = Out;
00648 }
00649
00650 /*
00651     #] TokenToLine :
00652     #[ CodeToLine :          void CodeToLine(name,number,mode)
00653
00654     Writes a name and possibly its number to output as a single token.
00655
00656     */
00657
00658 UBYTE *CodeToLine(WORD number, UBYTE *Out)
00659 {
00660     Out = StrCopy((UBYTE *) "(", Out);
00661     Out = NumCopy(number, Out);
00662     Out = StrCopy((UBYTE *) ")", Out);
00663     return(Out);
00664 }
00665
00666 /*
00667     #] CodeToLine :
00668     #[ MultiplyToLine :
00669     */
00670
00671 void MultiplyToLine(void)
00672 {
00673     int i;
00674     if ( AO.CurrentDictionary > 0 && AO.CurDictSpecials > 0
00675         && AO.CurDictSpecials == DICT_DOSPECIALS ) {
00676         DICTIONARY *dict = AO.Dictionaries[AO.CurrentDictionary-1];

```

```

00677 /*
00678     Find the star:
00679 */
00680     for ( i = 0; i < dict->numelements; i++ ) {
00681         if ( dict->elements[i]->type != DICT_SPECIALCHARACTER ) continue;
00682         if ( (UBYTE)dict->elements[i]->lhs[0] == (UBYTE)('*') ) {
00683             TokenToLine((UBYTE *) (dict->elements[i]->rhs));
00684             return;
00685         }
00686     }
00687 }
00688 TokenToLine((UBYTE *) "*");
00689 }
00690
00691 /*
00692     #] MultiplyToLine :
00693     #[ AddArrayIndex :
00694 */
00695
00696 UBYTE *AddArrayIndex(WORD num, UBYTE *out)
00697 {
00698     if ( AC.OutputMode == CMODE ) {
00699         out = StrCopy((UBYTE *) "[", out);
00700         out = NumCopy(num, out);
00701         out = StrCopy((UBYTE *) "]", out);
00702     }
00703     else {
00704         out = StrCopy((UBYTE *) "(", out);
00705         out = NumCopy(num, out);
00706         out = StrCopy((UBYTE *) ")", out);
00707     }
00708     return(out);
00709 }
00710
00711 /*
00712     #] AddArrayIndex :
00713     #[ PrtTerms :          void PrtTerms()
00714 */
00715
00716 void PrtTerms(void)
00717 {
00718     UWORD a[2];
00719     WORD na;
00720     a[0] = (UWORD)AO.NumInBrack;
00721     a[1] = (UWORD)(AO.NumInBrack >> BITSINWORD);
00722     if ( a[1] ) na = 2;
00723     else na = 1;
00724     TokenToLine((UBYTE *) " ");
00725     LongToLine(a, na);
00726     if ( a[0] == 1 && na == 1 ) {
00727         TokenToLine((UBYTE *) " term");
00728     }
00729     else TokenToLine((UBYTE *) " terms");
00730     AO.NumInBrack = 0;
00731 }
00732
00733 /*
00734     #] PrtTerms :
00735     #[ WrtPower :
00736 */
00737
00738 UBYTE *WrtPower(UBYTE *Out, WORD Power)
00739 {
00740     if ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE
00741         || AC.OutputMode == REDUCEMODE ) {
00742         *Out++ = '*'; *Out++ = '*';
00743     }
00744     else if ( AC.OutputMode == CMODE ) *Out++ = ',';
00745     else {
00746         UBYTE *Out1 = IsExponentSign();
00747         if ( Out1 == 0 ) *Out++ = '^';
00748         else {
00749             while ( *Out1 ) *Out++ = *Out1++;
00750             *Out = 0;
00751         }
00752     }
00753     if ( Power >= 0 ) {
00754         if ( Power < 2*MAXPOWER )
00755             Out = NumCopy(Power, Out);
00756         else
00757             Out = StrCopy(FindSymbol((WORD)((LONG)Power-2*MAXPOWER)), Out);
00758     /*
00759         Out = StrCopy(VARNAME(symbols, (LONG)Power-2*MAXPOWER), Out); */
00759     if ( AC.OutputMode == CMODE ) *Out++ = ')';
00760     *Out = 0;
00761 }
00762 else {
00763     if ( ( AC.OutputMode >= FORTRANMODE || AC.OutputMode >= PFORTRANMODE

```

```

00764         || AC.OutputMode >= REDUCEMODE ) && AC.OutputMode != CMODE )
00765         *Out++ = '(';
00766     *Out++ = '-';
00767     if ( Power > -2*MAXPOWER )
00768         Out = NumCopy(-Power,Out);
00769     else
00770         Out = StrCopy(FindSymbol( (WORD) ( (LONG)Power-2*MAXPOWER) ),Out);
00771 /*     Out = StrCopy(VARNAME(symbols, (LONG) (-Power)-2*MAXPOWER),Out); */
00772     if ( AC.OutputMode >= FORTRANMODE || AC.OutputMode >= PFORTRANMODE
00773         || AC.OutputMode >= REDUCEMODE ) *Out++ = ')';
00774     *Out = 0;
00775 }
00776 return(Out);
00777 }
00778
00779 /*
00780     #] WrtPower :
00781     #[ PrintTime :
00782 */
00783 void PrintTime(UBYTE *mess)
00784 {
00785     LONG millitime = TimeCPU(1);
00786     WORD timepart = (WORD)(millitime%1000);
00787     millitime /= 1000;
00788     timepart /= 10;
00789     MesPrint("At %s: Time = %7l.%2i sec",mess,millitime,timepart);
00790 }
00791
00792 /*
00793     #] PrintTime :
00794     #] schryf-Utilities :
00795     #[ schryf-Writes :
00796     #[ WriteLists :         void WriteLists()
00797
00798     Writes the namelists. If mode > 0 also the internal codes are given.
00799
00800 */
00801 static UBYTE *symname[] = {
00802     (UBYTE *)"(cyclic)", (UBYTE *)"(reversecyclic)"
00803     , (UBYTE *)"(symmetric)", (UBYTE *)"(antisymmetric)" };
00804 static UBYTE *rsymname[] = {
00805     (UBYTE *)"(-cyclic)", (UBYTE *)"(-reversecyclic)"
00806     , (UBYTE *)"(-symmetric)", (UBYTE *)"(-antisymmetric)" };
00807 void WriteLists(void)
00808 {
00809     GETIDENTITY
00810     WORD i, j, k, *skip;
00811     int first, startvalue;
00812     UBYTE *OutScr, *Out;
00813     EXPRESSIONS e;
00814     CBUF *C = cbuf+AC.cbunum;
00815     int olddict = AO.CurrentDictionary;
00816     skip = &AO.OutSkip;
00817     *skip = 0;
00818     AO.OutputLine = AO.OutFill = (UBYTE *)AT.WorkPointer;
00819     AO.CurrentDictionary = 0;
00820     FiniLine();
00821     OutScr = (UBYTE *)AT.WorkPointer + ( TOLONG(AT.WorkTop) - TOLONG(AT.WorkPointer) ) /2;
00822     if ( AC.CodesFlag || AC.NamesFlag > 1 ) startvalue = 0;
00823     else startvalue = FIRSTUSERSYMBOL;
00824 /*
00825     #[ Symbols :
00826 */
00827     if ( ( j = NumSymbols ) > startvalue ) {
00828         TokenToLine((UBYTE *)" Symbols");
00829         *skip = 3;
00830         FiniLine();
00831         for ( i = startvalue; i < j; i++ ) {
00832             if ( i >= BUILTINSYMBOLS && i < FIRSTUSERSYMBOL ) continue;
00833             Out = StrCopy(VARNAME(symbols,i),OutScr);
00834             if ( symbols[i].minpower > -MAXPOWER || symbols[i].maxpower < MAXPOWER ) {
00835                 Out = StrCopy((UBYTE *)"(",Out);
00836                 if ( symbols[i].minpower > -MAXPOWER )
00837                     Out = NumCopy(symbols[i].minpower,Out);
00838                 Out = StrCopy((UBYTE *)":",Out);
00839                 if ( symbols[i].maxpower < MAXPOWER )
00840                     Out = NumCopy(symbols[i].maxpower,Out);
00841                 Out = StrCopy((UBYTE *)")",Out);
00842             }
00843             if ( ( symbols[i].complex & VARTYPEIMAGINARY ) == VARTYPEIMAGINARY ) {
00844                 Out = StrCopy((UBYTE *)"#i",Out);
00845             }
00846             else if ( ( symbols[i].complex & VARTYPECOMPLEX ) == VARTYPECOMPLEX ) {
00847                 Out = StrCopy((UBYTE *)"#c",Out);
00848             }
00849         }
00850     }

```

```

00851     }
00852     else if ( ( symbols[i].complex & VARTYPEROOTOFUNITY ) == VARTYPEROOTOFUNITY ) {
00853         Out = StrCopy((UBYTE *)"#",Out);
00854         if ( ( symbols[i].complex & VARTYPEMINUS ) == VARTYPEMINUS ) {
00855             Out = StrCopy((UBYTE *)"-",Out);
00856         }
00857         else {
00858             Out = StrCopy((UBYTE *)"+",Out);
00859         }
00860         Out = NumCopy(symbols[i].maxpower,Out);
00861     }
00862     if ( AC.CodesFlag ) Out = CodeToLine(i,Out);
00863     if ( ( symbols[i].complex & VARTYPECOMPLEX ) == VARTYPECOMPLEX ) i++;
00864     StrCopy((UBYTE *)" ",Out);
00865     TokenToLine(OutScr);
00866 }
00867 *skip = 0;
00868 FiniLine();
00869 }
00870 /*
00871 #] Symbols :
00872 #[ Indices :
00873 */
00874 if ( AC.CodesFlag || AC.NamesFlag > 1 ) startvalue = 0;
00875 else startvalue = BUILTININDICES;
00876 if ( ( j = NumIndices ) > startvalue ) {
00877     TokenToLine((UBYTE *)" Indices");
00878     *skip = 3;
00879     FiniLine();
00880     for ( i = startvalue; i < j; i++ ) {
00881         Out = StrCopy(FindIndex(i+AM.OffsetIndex),OutScr);
00882         Out = StrCopy(VARNAME(indices,i),OutScr);
00883         if ( indices[i].dimension >= 0 ) {
00884             if ( indices[i].dimension != AC.lDefDim ) {
00885                 Out = StrCopy((UBYTE *)"=",Out);
00886                 Out = NumCopy(indices[i].dimension,Out);
00887             }
00888         }
00889         else if ( indices[i].dimension < 0 ) {
00890             Out = StrCopy((UBYTE *)"=",Out);
00891             Out = StrCopy(VARNAME(symbols,-indices[i].dimension),Out);
00892             if ( indices[i].nmin4 < -NMIN4SHIFT ) {
00893                 Out = StrCopy((UBYTE *)":",Out);
00894                 Out = StrCopy(VARNAME(symbols,-indices[i].nmin4-NMIN4SHIFT),Out);
00895             }
00896         }
00897         if ( AC.CodesFlag ) Out = CodeToLine(i+AM.OffsetIndex,Out);
00898         StrCopy((UBYTE *)" ",Out);
00899         TokenToLine(OutScr);
00900     }
00901     *skip = 0;
00902     FiniLine();
00903 }
00904 /*
00905 #] Indices :
00906 #[ Vectors :
00907 */
00908 if ( AC.CodesFlag || AC.NamesFlag > 1 ) startvalue = 0;
00909 else startvalue = BUILTINVECTORS;
00910 if ( ( j = NumVectors ) > startvalue ) {
00911     TokenToLine((UBYTE *)" Vectors");
00912     *skip = 3;
00913     FiniLine();
00914     for ( i = startvalue; i < j; i++ ) {
00915         Out = StrCopy(VARNAME(vectors,i),OutScr);
00916         if ( AC.CodesFlag ) Out = CodeToLine(i+AM.OffsetVector,Out);
00917         StrCopy((UBYTE *)" ",Out);
00918         TokenToLine(OutScr);
00919     }
00920     *skip = 0;
00921     FiniLine();
00922 }
00923 /*
00924 #] Vectors :
00925 #[ Functions :
00926 */
00927 if ( AC.CodesFlag || AC.NamesFlag > 1 ) startvalue = 0;
00928 else startvalue = AM.NumFixedFunctions;
00929 for ( k = 0; k < 2; k++ ) {
00930     first = 1;
00931     j = NumFunctions;
00932     for ( i = startvalue; i < j; i++ ) {
00933         if ( i > MAXBUILTINFUNCTION-FUNCTION
00934             && i < FIRSTUSERFUNCTION-FUNCTION ) continue;
00935         if ( ( k == 0 && functions[i].commute )
00936             || ( k != 0 && !functions[i].commute ) ) {
00937             if ( first ) {

```

```

00938             TokenToLine((UBYTE *) (FG.FunNam[k]));
00939             *skip = 3;
00940             FiniLine();
00941             first = 0;
00942         }
00943         Out = StrCopy(VARNAME(functions,i),OutScr);
00944         if ( ( functions[i].complex & VARTYPEIMAGINARY ) == VARTYPEIMAGINARY ) {
00945             Out = StrCopy((UBYTE *)"#i",Out);
00946         }
00947         else if ( ( functions[i].complex & VARTYPECOMPLEX ) == VARTYPECOMPLEX ) {
00948             Out = StrCopy((UBYTE *)"#c",Out);
00949         }
00950         if ( functions[i].spec == VERTEXFUNCTION ) {
00951             Out = StrCopy((UBYTE *)"(Particle)",Out);
00952         }
00953         else if ( functions[i].spec >= TENSORFUNCTION ) {
00954             Out = StrCopy((UBYTE *)"(Tensor)",Out);
00955         }
00956         if ( functions[i].symmetric > 0 ) {
00957             if ( ( functions[i].symmetric & REVERSEORDER ) != 0 ) {
00958                 Out = StrCopy((UBYTE *) (rsymname[(functions[i].symmetric &
~REVERSEORDER)-1]),Out);
00959             }
00960             else {
00961                 Out = StrCopy((UBYTE *) (symname[functions[i].symmetric-1]),Out);
00962             }
00963         }
00964         if ( AC.CodesFlag ) Out = CodeToLine(i+FUNCTION,Out);
00965         if ( ( functions[i].complex & VARTYPECOMPLEX ) == VARTYPECOMPLEX ) i++;
00966         StrCopy((UBYTE *)" ",Out);
00967         TokenToLine(OutScr);
00968     }
00969 }
00970 *skip = 0;
00971 if ( first == 0 ) FiniLine();
00972 }
00973 /*
00974     #] Functions :
00975     #[ Sets :
00976 */
00977 if ( AC.CodesFlag || AC.NamesFlag > 1 ) startvalue = 0;
00978 else startvalue = AM.NumFixedSets;
00979 if ( ( j = AC.SetList.num ) > startvalue ) {
00980     WORD element, LastElement, type, number;
00981     TokenToLine((UBYTE *)" Sets");
00982     for ( i = startvalue; i < j; i++ ) {
00983         *skip = 3;
00984         FiniLine();
00985         if ( Sets[i].name < 0 ) {
00986             Out = StrCopy((UBYTE *)"{}",OutScr);
00987         }
00988         else {
00989             Out = StrCopy(VARNAME(Sets,i),OutScr);
00990         }
00991         if ( AC.CodesFlag ) Out = CodeToLine(i,Out);
00992         StrCopy((UBYTE *)":",Out);
00993         TokenToLine(OutScr);
00994         if ( i < AM.NumFixedSets ) {
00995             TokenToLine((UBYTE *)" ");
00996             TokenToLine((UBYTE *)fixedsets[i].description);
00997         }
00998         else if ( Sets[i].type == CRANGE ) {
00999             int iflag = 0;
01000             if ( Sets[i].first == 3*MAXPOWER ) {
01001             }
01002             else if ( Sets[i].first >= MAXPOWER ) {
01003                 TokenToLine((UBYTE *)"<=");
01004                 NumCopy(Sets[i].first-2*MAXPOWER,OutScr);
01005                 TokenToLine(OutScr);
01006                 iflag = 1;
01007             }
01008             else {
01009                 TokenToLine((UBYTE *)"<");
01010                 NumCopy(Sets[i].first,OutScr);
01011                 TokenToLine(OutScr);
01012                 iflag = 1;
01013             }
01014             if ( Sets[i].last == -3*MAXPOWER ) {
01015             }
01016             else if ( Sets[i].last <= -MAXPOWER ) {
01017                 if ( iflag ) TokenToLine((UBYTE *)",");
01018                 TokenToLine((UBYTE *)">=");
01019                 NumCopy(Sets[i].last+2*MAXPOWER,OutScr);
01020                 TokenToLine(OutScr);
01021             }
01022             else {
01023                 if ( iflag ) TokenToLine((UBYTE *)",");

```



```

01024         TokenToLine((UBYTE *)">");
01025         NumCopy(Sets[i].last, OutScr);
01026         TokenToLine(OutScr);
01027     }
01028 }
01029 else {
01030     element = Sets[i].first;
01031     LastElement = Sets[i].last;
01032     type = Sets[i].type;
01033     while ( element < LastElement ) {
01034         TokenToLine((UBYTE *)" ");
01035         number = SetElements[element++];
01036         switch ( type ) {
01037             case CSYMBOL:
01038                 if ( number < 0 ) {
01039                     StrCopy(VARNAME(symbols, -number), OutScr);
01040                     StrCopy((UBYTE *) "?", Out);
01041                     TokenToLine(OutScr);
01042                 }
01043                 else if ( number < MAXPOWER )
01044                     TokenToLine(VARNAME(symbols, number));
01045                 else {
01046                     NumCopy(number-2*MAXPOWER, OutScr);
01047                     TokenToLine(OutScr);
01048                 }
01049                 break;
01050             case CINDEX:
01051                 if ( number >= AM.IndDum ) {
01052                     Out = StrCopy((UBYTE *) "N", OutScr);
01053                     Out = NumCopy(number-(AM.IndDum), Out);
01054                     StrCopy((UBYTE *) "_?", Out);
01055                     TokenToLine(OutScr);
01056                 }
01057                 else if ( number >= AM.OffsetIndex + (WORD)WILDMASK ) {
01058                     Out = StrCopy(VARNAME(indices, number
01059                                     -AM.OffsetIndex-WILDMASK), OutScr);
01060                     StrCopy((UBYTE *) "?", Out);
01061                     TokenToLine(OutScr);
01062                 }
01063                 else if ( number >= AM.OffsetIndex ) {
01064                     TokenToLine(VARNAME(indices, number-AM.OffsetIndex));
01065                 }
01066                 else {
01067                     NumCopy(number, OutScr);
01068                     TokenToLine(OutScr);
01069                 }
01070                 break;
01071             case CVECTOR:
01072                 Out = OutScr;
01073                 if ( number < AM.OffsetVector ) {
01074                     number += WILDMASK;
01075                     Out = StrCopy((UBYTE *) "-", Out);
01076                 }
01077                 if ( number >= AM.OffsetVector + WILDOFFSET ) {
01078                     Out = StrCopy(VARNAME(vectors, number
01079                                     -AM.OffsetVector-WILDOFFSET), Out);
01080                     StrCopy((UBYTE *) "?", Out);
01081                 }
01082                 else {
01083                     Out = StrCopy(VARNAME(vectors, number-AM.OffsetVector), Out);
01084                 }
01085                 TokenToLine(OutScr);
01086                 break;
01087             case CFUNCTION:
01088                 if ( number >= FUNCTION + (WORD)WILDMASK ) {
01089                     Out = StrCopy(VARNAME(functions, number
01090                                     -FUNCTION-WILDMASK), OutScr);
01091                     StrCopy((UBYTE *) "?", Out);
01092                     TokenToLine(OutScr);
01093                 }
01094                 TokenToLine(VARNAME(functions, number-FUNCTION));
01095                 break;
01096             default:
01097                 NumCopy(number, OutScr);
01098                 TokenToLine(OutScr);
01099                 break;
01100         }
01101     }
01102 }
01103 }
01104 *skip = 0;
01105 FiniLine();
01106 }
01107 /*
01108 #] Sets :
01109 #[ Expressions :
01110 */

```

```

01111     if ( AS.ExecMode ) {
01112         e = Expressions;
01113         j = NumExpressions;
01114         first = 1;
01115         for ( i = 0; i < j; i++, e++ ) {
01116             if ( e->status >= 0 ) {
01117                 if ( first ) {
01118                     TokenToLine((UBYTE *) " Expressions");
01119                     *skip = 3;
01120                     FiniLine();
01121                     first = 0;
01122                 }
01123                 Out = StrCopy(AC.exprnames->namebuffer+e->name, OutScr);
01124                 Out = StrCopy((UBYTE *) (FG.ExprStat[e->status]), Out);
01125                 if ( AC.CodesFlag ) Out = CodeToLine(i, Out);
01126                 StrCopy((UBYTE *) " ", Out);
01127                 TokenToLine(OutScr);
01128             }
01129         }
01130         if ( !first ) {
01131             *skip = 0;
01132             FiniLine();
01133         }
01134     }
01135     e = Expressions;
01136     j = NumExpressions;
01137     first = 1;
01138     for ( i = 0; i < j; i++ ) {
01139         if ( e->printflag && ( e->status == LOCALEXPRESSION ||
01140             e->status == GLOBALEXPRESSION || e->status == UNHIDELEXPRESSION
01141             || e->status == UNHIDEGEXPRESSION ) ) {
01142             if ( first ) {
01143                 TokenToLine((UBYTE *) " Expressions to be printed");
01144                 *skip = 3;
01145                 FiniLine();
01146                 first = 0;
01147             }
01148             Out = StrCopy(AC.exprnames->namebuffer+e->name, OutScr);
01149             StrCopy((UBYTE *) " ", Out);
01150             TokenToLine(OutScr);
01151         }
01152         e++;
01153     }
01154     if ( !first ) {
01155         *skip = 0;
01156         FiniLine();
01157     }
01158     /*
01159         #] Expressions :
01160         #[ Dollars :
01161     */
01162
01163     if ( AC.CodesFlag || AC.NamesFlag > 1 ) startvalue = 0;
01164     else startvalue = BUILTINDOLLARS;
01165     if ( ( j = NumDollars ) > startvalue ) {
01166         TokenToLine((UBYTE *) " Dollar variables");
01167         *skip = 3;
01168         FiniLine();
01169         for ( i = startvalue; i < j; i++ ) {
01170             Out = StrCopy((UBYTE *) "$", OutScr);
01171             Out = StrCopy(DOLLARNAME(Dollars, i), Out);
01172             if ( AC.CodesFlag ) Out = CodeToLine(i, Out);
01173             StrCopy((UBYTE *) " ", Out);
01174             TokenToLine(OutScr);
01175         }
01176         *skip = 0;
01177         FiniLine();
01178     }
01179
01180     if ( ( j = NumPotModdollars ) > 0 ) {
01181         TokenToLine((UBYTE *) " Dollar variables to be modified");
01182         *skip = 3;
01183         FiniLine();
01184         for ( i = 0; i < j; i++ ) {
01185             Out = StrCopy((UBYTE *) "$", OutScr);
01186             Out = StrCopy(DOLLARNAME(Dollars, PotModdollars[i]), Out);
01187             for ( k = 0; k < NumModOptdollars; k++ )
01188                 if ( ModOptdollars[k].number == PotModdollars[i] ) break;
01189             if ( k < NumModOptdollars ) {
01190                 switch ( ModOptdollars[k].type ) {
01191                     case MODSUM:
01192                         Out = StrCopy((UBYTE *) "(sum)", Out);
01193                         break;
01194                     case MODMAX:
01195                         Out = StrCopy((UBYTE *) "(maximum)", Out);
01196                         break;
01197                     case MODMIN:

```

```

01198             Out = StrCopy((UBYTE *)"(minimum)", Out);
01199             break;
01200         case MODLOCAL:
01201             Out = StrCopy((UBYTE *)"(local)", Out);
01202             break;
01203         default:
01204             Out = StrCopy((UBYTE *)"(?)", Out);
01205             break;
01206     }
01207 }
01208 StrCopy((UBYTE *)" ", Out);
01209 TokenToLine(OutScr);
01210 }
01211 *skip = 0;
01212 FiniLine();
01213 }
01214 /*
01215 #] Dollars :
01216 */
01217
01218 if ( AC.ncmod != 0 ) {
01219     TokenToLine((UBYTE *)"All arithmetic is modulus ");
01220     LongToLine((UWORD *)AC.cmod,ABS(AC.ncmod));
01221     if ( AC.ncmod > 0 ) TokenToLine((UBYTE *)" with powerreduction");
01222     else TokenToLine((UBYTE *)" without powerreduction");
01223     if ( ( AC.modmode & POSNEG ) != 0 ) TokenToLine((UBYTE *)" centered around 0");
01224     else TokenToLine((UBYTE *)" positive numbers only");
01225     FiniLine();
01226 }
01227 if ( AC.lDefDim != 4 ) {
01228     TokenToLine((UBYTE *)"The default dimension is ");
01229     if ( AC.lDefDim >= 0 ) {
01230         NumCopy(AC.lDefDim,OutScr);
01231         TokenToLine(OutScr);
01232     }
01233     else {
01234         TokenToLine(VARNAME(symbols,-AC.lDefDim));
01235         if ( AC.lDefDim4 != -NMIN4SHIFT ) {
01236             TokenToLine((UBYTE *)"");
01237             if ( AC.lDefDim4 >= -NMIN4SHIFT ) {
01238                 NumCopy(AC.lDefDim4,OutScr);
01239                 TokenToLine(OutScr);
01240             }
01241             else {
01242                 TokenToLine(VARNAME(symbols,-AC.lDefDim4-NMIN4SHIFT));
01243             }
01244         }
01245     }
01246     FiniLine();
01247 }
01248 if ( AC.lUnitTrace != 4 ) {
01249     TokenToLine((UBYTE *)"The trace of the unit matrix is ");
01250     if ( AC.lUnitTrace >= 0 ) {
01251         NumCopy(AC.lUnitTrace,OutScr);
01252         TokenToLine(OutScr);
01253     }
01254     else {
01255         TokenToLine(VARNAME(symbols,-AC.lUnitTrace));
01256     }
01257     FiniLine();
01258 }
01259 if ( AO.NumDictionaries > 0 ) {
01260     for ( i = 0; i < AO.NumDictionaries; i++ ) {
01261         WriteDictionary(AO.Dictionaries[i]);
01262     }
01263     if ( olddict > 0 )
01264         MesPrint("\nCurrently dictionary %s is active\n",
01265             AO.Dictionaries[olddict-1]->name);
01266     else
01267         MesPrint("\nCurrently there is no active dictionary\n");
01268 }
01269 if ( AC.CodesFlag ) {
01270     if ( C->numlhs > 0 ) {
01271         TokenToLine((UBYTE *)" Left Hand Sides:");
01272         AO.OutSkip = 3;
01273         for ( i = 1; i <= C->numlhs; i++ ) {
01274             FiniLine();
01275             skip = C->lhs[i];
01276             j = skip[1];
01277             while ( --j >= 0 ) { TalToLine((UWORD)(*skip++)); TokenToLine((UBYTE *)" "); }
01278         }
01279         AO.OutSkip = 0;
01280         FiniLine();
01281     }
01282     if ( C->numrhs > 0 ) {
01283         TokenToLine((UBYTE *)" Right Hand Sides:");
01284         AO.OutSkip = 3;

```

```

01285         for ( i = 1; i <= C->numrhs; i++ ) {
01286             FiniLine();
01287             skip = C->rhs[i];
01288             while ( ( j = skip[0] ) != 0 ) {
01289                 while ( --j >= 0 ) { TalToLine((UWORD) (*skip++)); TokenToLine((UBYTE *) " "); }
01290             }
01291             FiniLine();
01292         }
01293         AO.OutSkip = 0;
01294         FiniLine();
01295     }
01296 }
01297 AO.CurrentDictionary = olddict;
01298 }
01299
01300 /*
01301     #] WriteLists :
01302     #[ WriteDictionary :
01303
01304     This routine is part of WriteLists and should be called from there.
01305 */
01306
01307 void WriteDictionary(DICTIONARY *dict)
01308 {
01309     GETIDENTITY
01310     int i, first;
01311     WORD *skip, na, *a, spec, *t, *tstop, j;
01312     UBYTE str[2], *OutScr, *Out;
01313     WORD oldoutputmode = AC.OutputMode, oldoutputsaces = AC.OutputSpaces;
01314     WORD oldoutskip = AO.OutSkip;
01315     AC.OutputMode = NORMALFORMAT;
01316     AC.OutputSpaces = NOSPACEFORMAT;
01317     MesPrint("===Contents of dictionary %s===", dict->name);
01318     skip = &AO.OutSkip;
01319     *skip = 3;
01320     AO.OutputLine = AO.OutFill = (UBYTE *) AT.WorkPointer;
01321     for ( j = 0; j < *skip; j++ ) *(AO.OutFill)++ = ' ';
01322
01323     OutScr = (UBYTE *) AT.WorkPointer + ( TOLONG(AT.WorkTop) - TOLONG(AT.WorkPointer) ) /2;
01324     for ( i = 0; i < dict->numelements; i++ ) {
01325         switch ( dict->elements[i]->type ) {
01326             case DICT_INTEGERNUMBER:
01327                 LongToLine((UWORD *) (dict->elements[i]->lhs), dict->elements[i]->size);
01328                 Out = OutScr; *Out = 0;
01329                 break;
01330             case DICT_RATIONALNUMBER:
01331                 a = dict->elements[i]->lhs;
01332                 na = a[a[0]-1]; na = (ABS(na)-1)/2;
01333                 RatToLine((UWORD *) (a+1), na);
01334                 Out = OutScr; *Out = 0;
01335                 break;
01336             case DICT_SYMBOL:
01337                 na = dict->elements[i]->lhs[0];
01338                 Out = StrCopy(VARNAME(symbols, na), OutScr);
01339                 break;
01340             case DICT_VECTOR:
01341                 na = dict->elements[i]->lhs[0]-AM.OffsetVector;
01342                 Out = StrCopy(VARNAME(vectors, na), OutScr);
01343                 break;
01344             case DICT_INDEX:
01345                 na = dict->elements[i]->lhs[0]-AM.OffsetIndex;
01346                 Out = StrCopy(VARNAME(indices, na), OutScr);
01347                 break;
01348             case DICT_FUNCTION:
01349                 na = dict->elements[i]->lhs[0]-FUNCTION;
01350                 Out = StrCopy(VARNAME(functions, na), OutScr);
01351                 break;
01352             case DICT_FUNCTION_WITH_ARGUMENTS:
01353                 t = dict->elements[i]->lhs;
01354                 na = *t-FUNCTION;
01355                 Out = StrCopy(VARNAME(functions, na), OutScr);
01356                 spec = functions[*t - FUNCTION].spec;
01357                 tstop = t + t[1];
01358                 first = 1;
01359                 if ( t[1] <= FUNHEAD ) {}
01360                 else if ( spec >= TENSORFUNCTION ) {
01361                     t += FUNHEAD; *Out++ = (UBYTE) '(';
01362                     while ( t < tstop ) {
01363                         if ( first == 0 ) *Out++ = (UBYTE) (',');
01364                         else first = 0;
01365                         j = *t++;
01366                         if ( j >= 0 ) {
01367                             if ( j < AM.OffsetIndex ) { Out = NumCopy(j, Out); }
01368                             else if ( j < AM.IndDum ) {
01369                                 Out = StrCopy(VARNAME(indices, j-AM.OffsetIndex), Out);
01370                             }
01371                             else {

```

```

01372                                     MesPrint("Currently wildcards are not allowed in dictionary
elements");
01373                                     Terminate(-1);
01374                                     }
01375                                     }
01376                                     else {
01377                                         Out = StrCopy(VARNAME(vectors,j-AM.OffsetVector),Out);
01378                                     }
01379                                     }
01380                                     *Out++ = (UBYTE)'\''; *Out = 0;
01381                                     }
01382                                     else {
01383                                         t += FUNHEAD; *Out++ = (UBYTE)'\''; *Out = 0;
01384                                         TokenToLine(OutScr);
01385                                         while ( t < tstop ) {
01386                                             if ( !first ) TokenToLine((UBYTE *)"", "");
01387                                             WriteArgument(t);
01388                                             NEXTARG(t);
01389                                             first = 0;
01390                                         }
01391                                         Out = OutScr;
01392                                         *Out++ = (UBYTE)'\''; *Out = 0;
01393                                     }
01394                                     break;
01395                                     case DICT_SPECIALCHARACTER:
01396                                         str[0] = (UBYTE)(dict->elements[i]->lhs[0]);
01397                                         str[1] = 0;
01398                                         Out = StrCopy(str,OutScr);
01399                                         break;
01400                                     default:
01401                                         Out = OutScr; *Out = 0;
01402                                         break;
01403                                     }
01404                                     Out = StrCopy((UBYTE *)": \"",Out);
01405                                     Out = StrCopy((UBYTE *) (dict->elements[i]->rhs),Out);
01406                                     Out = StrCopy((UBYTE *) "\"",Out);
01407                                     TokenToLine(OutScr);
01408                                     FiniLine();
01409                                     }
01410                                     MesPrint("====End of dictionary %s====",dict->name);
01411                                     AC.OutputMode = oldoutputmode;
01412                                     AC.OutputSpaces = oldoutputspaces;
01413                                     AO.OutSkip = oldoutskip;
01414     }
01415
01416     /*
01417         #] WriteDictionary :
01418         #[ WriteArgument :      void WriteArgument(WORD *t)
01419
01420         Write a single argument field. The general field goes to
01421         WriteExpression and the fast field is dealt with here.
01422     */
01423
01424     void WriteArgument(WORD *t)
01425     {
01426         UBYTE buffer[180];
01427         UBYTE *Out;
01428         WORD i;
01429         int oldoutsidefun, oldlowestlevel = lowestlevel;
01430         lowestlevel = 0;
01431         if ( *t > 0 ) {
01432             oldoutsidefun = AC.outsidefun; AC.outsidefun = 0;
01433             WriteExpression(t+ARGHEAD, (LONG) (*t-ARGHEAD));
01434             AC.outsidefun = oldoutsidefun;
01435             goto CleanUp;
01436         }
01437         Out = buffer;
01438         if ( *t == -SNUMBER ) {
01439             NumCopy(t[1],Out);
01440         }
01441         else if ( *t == -SYMBOL ) {
01442             if ( t[1] >= MAXVARIABLES-cbuf[AM.sbufnum].numrhs ) {
01443                 Out = StrCopy(FindExtraSymbol(MAXVARIABLES-t[1]),Out);
01444             }
01445             Out = StrCopy((UBYTE *)AC.extrasym,Out);
01446             if ( AC.extrasymbols == 0 ) {
01447                 Out = NumCopy((MAXVARIABLES-t[1]),Out);
01448                 Out = StrCopy((UBYTE *) "_",Out);
01449             }
01450             else if ( AC.extrasymbols == 1 ) {
01451                 Out = AddArrayIndex((MAXVARIABLES-t[1]),Out);
01452             }
01453         }
01454         /*
01455             else if ( AC.extrasymbols == 2 ) {
01456                 Out = NumCopy((MAXVARIABLES-t[1]),Out);
01457             }

```

```

01458 */
01459     }
01460     else {
01461         StrCopy(FindSymbol(t[1]),Out);
01462     /*      StrCopy(VARNAME(symbols,t[1]),Out); */
01463     }
01464 }
01465 else if ( *t == -VECTOR ) {
01466     if ( t[1] == FUNNYVEC ) { *Out++ = '?'; *Out = 0; }
01467     else
01468         StrCopy(FindVector(t[1]),Out);
01469 /*      StrCopy(VARNAME(vectors,t[1] - AM.OffsetVector),Out); */
01470 }
01471 else if ( *t == -MINVECTOR ) {
01472     *Out++ = '-';
01473     StrCopy(FindVector(t[1]),Out);
01474 /*      StrCopy(VARNAME(vectors,t[1] - AM.OffsetVector),Out); */
01475 }
01476 else if ( *t == -INDEX ) {
01477     if ( t[1] >= 0 ) {
01478         if ( t[1] < AM.OffsetIndex ) { NumCopy(t[1],Out); }
01479         else {
01480             i = t[1];
01481             if ( i >= AM.IndDum ) {
01482                 i -= AM.IndDum;
01483                 *Out++ = 'N';
01484                 Out = NumCopy(i,Out);
01485                 *Out++ = '-';
01486                 *Out++ = '?';
01487                 *Out = 0;
01488             }
01489             else {
01490                 i -= AM.OffsetIndex;
01491                 Out = StrCopy(FindIndex(i%WILDOFFSET+AM.OffsetIndex),Out);
01492 /*      Out = StrCopy(VARNAME(indices,i%WILDOFFSET),Out); */
01493                 if ( i >= WILDOFFSET ) { *Out++ = '?'; *Out = 0; }
01494             }
01495         }
01496     }
01497     else if ( t[1] == FUNNYVEC ) { *Out++ = '?'; *Out = 0; }
01498     else
01499         StrCopy(FindVector(t[1]),Out);
01500 /*      StrCopy(VARNAME(vectors,t[1] - AM.OffsetVector),Out); */
01501 }
01502 else if ( *t == -DOLLAREXPRESSION ) {
01503     DOLLARS d = Dollars + t[1];
01504     *Out++ = '$';
01505     StrCopy(AC.dollarnames->namebuffer+d->name,Out);
01506 }
01507 else if ( *t == -EXPRESSION ) {
01508     StrCopy(EXPRNAME(t[1]),Out);
01509 }
01510 else if ( *t == -SETSET ) {
01511     StrCopy(VARNAME(Sets,t[1]),Out);
01512 }
01513 else if ( *t <= -FUNCTION ) {
01514     StrCopy(FindFunction(-*t),Out);
01515 /*      StrCopy(VARNAME(functions,-*t-FUNCTION),Out); */
01516 }
01517 else {
01518     MesPrint("Illegal function argument while writing");
01519     goto CleanUp;
01520 }
01521 TokenToLine(buffer);
01522 CleanUp:
01523 lowestlevel = oldlowestlevel;
01524 return;
01525 }
01526
01527 /*
01528     #] WriteArgument :
01529     #[ WriteSubTerm :      WORD WriteSubTerm(sterm,first)
01530
01531     Writes a single subterm field to the output line.
01532     There is a recursion for functions.
01533
01534 #define NUMSPECS 8
01535 UBYTE *specfunnames[NUMSPECS] = {
01536     (UBYTE *)"fac", (UBYTE *)"nargs", (UBYTE *)"binom"
01537     , (UBYTE *)"sign", (UBYTE *)"mod", (UBYTE *)"min", (UBYTE *)"max"
01538     , (UBYTE *)"invfac" };
01539 */
01540
01541 WORD WriteSubTerm(WORD *sterm, WORD first)
01542 {
01543     UBYTE buffer[80];

```

```

01545     UBYTE *Out, closepar[2] = { (UBYTE)'\'', 0};
01546     WORD *stopper, *t, *tt, i, j, po = 0;
01547     int oldoutsidefun;
01548     stopper = sterm + sterm[1];
01549     t = sterm + 2;
01550     switch ( *sterm ) {
01551     case SYMBOL :
01552         while ( t < stopper ) {
01553             if ( lowestlevel && ( ( AO.PrintType & PRINTALL ) != 0 ) ) {
01554                 FiniLine();
01555                 if ( AC.OutputSpaces == NOSPACEFORMAT ) IniLine(1);
01556                 else IniLine(3);
01557                 if ( first ) TokenToLine((UBYTE *) " ");
01558             }
01559             if ( !first ) MultiplyToLine();
01560             if ( AC.OutputMode == CMODE && t[1] != 1 ) {
01561                 if ( AC.Cnumpows >= t[1] && t[1] > 0 ) {
01562                     po = t[1];
01563                     Out = StrCopy((UBYTE *) "POW",buffer);
01564                     Out = NumCopy(po,Out);
01565                     Out = StrCopy((UBYTE *) "(",Out);
01566                     TokenToLine(buffer);
01567                 }
01568                 else {
01569                     TokenToLine((UBYTE *) "pow(");
01570                 }
01571             }
01572             if ( *t < NumSymbols ) {
01573                 Out = StrCopy(FindSymbol(*t),buffer); t++;
01574                 /* Out = StrCopy(VARNAME(symbols,*t),buffer); t++; */
01575             }
01576             else {
01577                 /* see also routine PrintSubtermList.
01578
01579                 */
01580                 Out = StrCopy(FindExtraSymbol(MAXVARIABLES-*t),buffer);
01581                 /* Out = StrCopy((UBYTE *) AC.extrasym,buffer);
01582                 if ( AC.extrasymbols == 0 ) {
01583                     Out = NumCopy((MAXVARIABLES-*t),Out);
01584                     Out = StrCopy((UBYTE *) "_",Out);
01585                 }
01586                 else if ( AC.extrasymbols == 1 ) {
01587                     Out = AddArrayIndex((MAXVARIABLES-*t),Out);
01588                 }
01589
01590                 */
01591                 /* else if ( AC.extrasymbols == 2 ) {
01592                     Out = NumCopy((MAXVARIABLES-*t),Out);
01593                 }
01594
01595                 */
01596                 t++;
01597             }
01598             if ( AC.OutputMode == CMODE && po > 1
01599                 && AC.Cnumpows >= po ) {
01600                 Out = StrCopy((UBYTE *) ")",Out);
01601                 po = 0;
01602             }
01603             else if ( *t != 1 ) WrtPower(Out,*t);
01604             TokenToLine(buffer);
01605             t++;
01606             first = 0;
01607         }
01608         break;
01609     case VECTOR :
01610         while ( t < stopper ) {
01611             if ( lowestlevel && ( ( AO.PrintType & PRINTALL ) != 0 ) ) {
01612                 FiniLine();
01613                 if ( AC.OutputSpaces == NOSPACEFORMAT ) IniLine(1);
01614                 else IniLine(3);
01615                 if ( first ) TokenToLine((UBYTE *) " ");
01616             }
01617             if ( !first ) MultiplyToLine();
01618
01619             Out = StrCopy(FindVector(*t),buffer);
01620             /* Out = StrCopy(VARNAME(vectors,*t - AM.OffsetVector),buffer); */
01621             t++;
01622             if ( AC.OutputMode == MATHEMATICAMODE ) *Out++ = '[';
01623             else *Out++ = '(';
01624             if ( *t >= AM.OffsetIndex ) {
01625                 i = *t++;
01626                 if ( i >= AM.IndDum ) {
01627                     i -= AM.IndDum;
01628                     *Out++ = 'N';
01629                     Out = NumCopy(i,Out);
01630                     *Out++ = '_';
01631                     *Out++ = '?';

```

```

01632         *Out = 0;
01633     }
01634     else
01635         Out = StrCopy(FindIndex(i),Out);
01636 /*         Out = StrCopy(VARNAME(indices,i - AM.OffsetIndex),Out); */
01637     }
01638     else if ( *t == FUNNYVEC ) { *Out++ = '?'; *Out = 0; }
01639     else {
01640         Out = NumCopy(*t++,Out);
01641     }
01642     if ( AC.OutputMode == MATHEMATICAMODE ) *Out++ = ']';
01643     else *Out++ = ')';
01644     *Out = 0;
01645     TokenToLine(buffer);
01646     first = 0;
01647 }
01648 break;
01649 case INDEX :
01650     while ( t < stopper ) {
01651         if ( lowestlevel && ( ( AO.PrintType & PRINTALL ) != 0 ) ) {
01652             FiniLine();
01653             if ( AC.OutputSpaces == NOSPACEFORMAT ) IniLine(1);
01654             else IniLine(3);
01655             if ( first ) TokenToLine((UBYTE *) " ");
01656         }
01657         if ( !first ) MultiplyToLine();
01658         if ( *t >= 0 ) {
01659             if ( *t < AM.OffsetIndex ) {
01660                 TalToLine((UWORD)(*t++));
01661             }
01662             else {
01663                 i = *t++;
01664                 if ( i >= AM.IndDum ) {
01665                     i -= AM.IndDum;
01666                     Out = buffer;
01667                     *Out++ = 'N';
01668                     Out = NumCopy(i,Out);
01669                     *Out++ = '_';
01670                     *Out++ = '?';
01671                     *Out = 0;
01672                 }
01673                 else {
01674                     i -= AM.OffsetIndex;
01675                     Out = StrCopy(FindIndex(i%WILDOFFSET+AM.OffsetIndex),buffer);
01676 /*                     Out = StrCopy(VARNAME(indices,i%WILDOFFSET),buffer); */
01677                     if ( i >= WILDOFFSET ) { *Out++ = '?'; *Out = 0; }
01678                 }
01679                 TokenToLine(buffer);
01680             }
01681         }
01682         else {
01683             TokenToLine(FindVector(*t)); t++;
01684 /*             TokenToLine(VARNAME(vectors,*t - AM.OffsetVector)); t++; */
01685         }
01686         first = 0;
01687     }
01688     break;
01689 case DOLLAREXPRESSION:
01690     {
01691         DOLLARS d = Dollars + sterm[2];
01692         Out = StrCopy((UBYTE *) "$",buffer);
01693         Out = StrCopy(AC.dollarnames->namebuffer+d->name,Out);
01694         if ( sterm[3] != 1 ) WrtPower(Out,sterm[3]);
01695         TokenToLine(buffer);
01696     }
01697     first = 0;
01698     break;
01699 case DELTA :
01700     while ( t < stopper ) {
01701         if ( lowestlevel && ( ( AO.PrintType & PRINTALL ) != 0 ) ) {
01702             FiniLine();
01703             if ( AC.OutputSpaces == NOSPACEFORMAT ) IniLine(1);
01704             else IniLine(3);
01705             if ( first ) TokenToLine((UBYTE *) " ");
01706         }
01707         if ( !first ) MultiplyToLine();
01708         Out = StrCopy((UBYTE *) "d_",buffer);
01709         if ( *t >= AM.OffsetIndex ) {
01710             if ( *t < AM.IndDum ) {
01711                 Out = StrCopy(FindIndex(*t),Out);
01712 /*                 Out = StrCopy(VARNAME(indices,*t - AM.OffsetIndex),Out); */
01713                 t++;
01714             }
01715             else {
01716                 *Out++ = 'N';
01717                 Out = NumCopy( *t++ - AM.IndDum, Out);
01718                 *Out++ = '_';

```



```

01719         *Out++ = '?';
01720         *Out = 0;
01721     }
01722 }
01723 else if ( *t == FUNNYVEC ) { *Out++ = '?'; *Out = 0; }
01724 else {
01725     Out = NumCopy(*t++,Out);
01726 }
01727 *Out++ = ',';
01728 if ( *t >= AM.OffsetIndex ) {
01729     if ( *t < AM.IndDum ) {
01730         Out = StrCopy(FindIndex(*t),Out);
01731         Out = StrCopy(VARNAME(indices,*t - AM.OffsetIndex),Out); /*
01732         t++;
01733     }
01734     else {
01735         *Out++ = 'N';
01736         Out = NumCopy(*t++ - AM.IndDum,Out);
01737         *Out++ = '_';
01738         *Out++ = '?';
01739     }
01740 }
01741 else {
01742     Out = NumCopy(*t++,Out);
01743 }
01744 *Out++ = ')';
01745 *Out = 0;
01746 TokenToLine(buffer);
01747 first = 0;
01748 }
01749 break;
01750 case DOTPRODUCT :
01751     while ( t < stopper ) {
01752         if ( lowestlevel && ( ( AO.PrintType & PRINTALL ) != 0 ) ) {
01753             FiniLine();
01754             if ( AC.OutputSpaces == NOSPACEFORMAT ) IniLine(1);
01755             else IniLine(3);
01756             if ( first ) TokenToLine((UBYTE *) " ");
01757         }
01758         if ( !first ) MultiplyToLine();
01759         if ( AC.OutputMode == CMODE && t[2] != 1 )
01760             TokenToLine((UBYTE *) "pow(");
01761         if ( AC.OutputMode == MATHEMATICAMODE )
01762             TokenToLine((UBYTE *) "(");
01763         Out = StrCopy(FindVector(*t),buffer);
01764         Out = StrCopy(VARNAME(vectors,*t - AM.OffsetVector),buffer); /*
01765         t++;
01766         if ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE
01767             || AC.OutputMode == CMODE )
01768             *Out++ = AO.FortDotChar;
01769         else *Out++ = '.';
01770         Out = StrCopy(FindVector(*t),Out);
01771         Out = StrCopy(VARNAME(vectors,*t - AM.OffsetVector),Out); /*
01772         if ( AC.OutputMode == MATHEMATICAMODE ) {
01773             *Out++ = ')';
01774             *Out = 0;
01775         }
01776         t++;
01777         if ( *t != 1 ) WrtPower(Out,*t);
01778         t++;
01779         TokenToLine(buffer);
01780         first = 0;
01781     }
01782     break;
01783 case EXPONENT :
01784     #if FUNHEAD != 2
01785         t += FUNHEAD - 2;
01786     #endif
01787     if ( !first ) MultiplyToLine();
01788     if ( AC.OutputMode == CMODE ) TokenToLine((UBYTE *) "pow(");
01789     else TokenToLine((UBYTE *) "(");
01790     WriteArgument(t);
01791     if ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE
01792         || AC.OutputMode == REDUCEMODE )
01793         TokenToLine((UBYTE *) ")*(");
01794     else if ( AC.OutputMode == CMODE ) TokenToLine((UBYTE *) ",");
01795     else {
01796         UBYTE *Out1 = IsExponentSign();
01797         if ( Out1 ) {
01798             TokenToLine((UBYTE *) Out1);
01799             TokenToLine(Out1);
01800             TokenToLine((UBYTE *) "(");
01801         }
01802         else TokenToLine((UBYTE *) "^(");
01803     }
01804     NEXTARG(t)
01805     WriteArgument(t);

```

```

01806         TokenToLine((UBYTE *)"");
01807         break;
01808     case DENOMINATOR :
01809 #if FUNHEAD != 2
01810         t += FUNHEAD - 2;
01811 #endif
01812         if ( first ) TokenToLine((UBYTE *)"1/(");
01813         else TokenToLine((UBYTE *)"/(");
01814         WriteArgument(t);
01815         TokenToLine((UBYTE *)"");
01816         break;
01817     case SUBEXPRESSION:
01818         if ( !first ) MultiplyToLine();
01819         TokenToLine((UBYTE *)"(");
01820         t = cbuf[sterm[4]].rhs[sterm[2]];
01821         tt = t;
01822         while ( *tt ) tt += *tt;
01823         oldoutsidfun = AC.outsidfun; AC.outsidfun = 0;
01824         if ( *t ) {
01825             WriteExpression(t, (LONG)(tt-t));
01826         }
01827         else {
01828             TokenToLine((UBYTE *)"0");
01829         }
01830         AC.outsidfun = oldoutsidfun;
01831         TokenToLine((UBYTE *)"");
01832         if ( sterms[3] != 1 ) {
01833             UBYTE *Out1 = IsExponentSign();
01834             if ( Out1 ) TokenToLine(Out1);
01835             else TokenToLine((UBYTE *)"^");
01836             Out = buffer;
01837             NumCopy(sterms[3], Out);
01838             TokenToLine(buffer);
01839         }
01840         break;
01841     default :
01842         if ( lowestlevel && ( ( AO.PrintType & PRINTALL ) != 0 ) ) {
01843             FiniLine();
01844             if ( AC.OutputSpaces == NOSPACEFORMAT ) IniLine(1);
01845             else IniLine(3);
01846             if ( first ) TokenToLine((UBYTE *)" ");
01847         }
01848         if ( *sterm < FUNCTION ) {
01849             return(MesPrint("Illegal subterm while writing"));
01850         }
01851         if ( !first ) MultiplyToLine();
01852         first = 1;
01853         { UBYTE *tmp;
01854             if ( ( tmp = FindFunWithArgs(sterm) ) != 0 ) {
01855                 TokenToLine(tmp);
01856                 break;
01857             }
01858         }
01859         t += FUNHEAD-2;
01860
01861         if ( *sterm == GAMMA && t[-FUNHEAD+1] == FUNHEAD+1 ) {
01862             TokenToLine((UBYTE *)"gi_");
01863         }
01864         else {
01865             if ( *sterm != DUMFUN ) {
01866                 Out = StrCopy(FindFunction(*sterm), buffer);
01867                 Out = StrCopy(VARNAME(functions, *sterm - FUNCTION), buffer); /*
01868             }
01869             else { Out = buffer; *Out = 0; }
01870             if ( t >= stopper ) {
01871                 TokenToLine(buffer);
01872                 break;
01873             }
01874             if ( AC.OutputMode == MATHEMATICAMODE ) { *Out++ = '['; closepar[0] = (UBYTE)']'; }
01875             else { *Out++ = '('; }
01876             *Out = 0;
01877             TokenToLine(buffer);
01878         }
01879         i = functions[*sterm - FUNCTION].spec;
01880         if ( i >= TENSORFUNCTION ) {
01881             int curdict = AO.CurrentDictionary;
01882             if ( AO.CurrentDictionary && AO.CurDictNotInFunctions > 0 )
01883                 AO.CurrentDictionary = 0;
01884             t = sterms + FUNHEAD;
01885             while ( t < stopper ) {
01886                 if ( !first ) TokenToLine((UBYTE *)",");
01887                 else first = 0;
01888                 j = *t++;
01889                 if ( j >= 0 ) {
01890                     if ( j < AM.OffsetIndex ) TalToLine((UWORD)(j));
01891                     else if ( j < AM.IndDum ) {
01892                         i = j - AM.OffsetIndex;

```

```

01893                                     Out = StrCopy(FindIndex(i%WILDOFFSET+AM.OffsetIndex),buffer);
01894 /*                                     Out = StrCopy(VARNAME(indices,i%WILDOFFSET),buffer); */
01895                                     if ( i >= WILDOFFSET ) { *Out++ = '?'; *Out = 0; }
01896                                     TokenToLine(buffer);
01897                                     }
01898                                     else {
01899                                         Out = buffer;
01900                                         *Out++ = 'N';
01901                                         Out = NumCopy(j - AM.IndDum,Out);
01902                                         *Out++ = '_';
01903                                         *Out++ = '?';
01904                                         *Out = 0;
01905                                         TokenToLine(buffer);
01906                                     }
01907                                     }
01908                                     else if ( j == FUNNYVEC ) { TokenToLine((UBYTE *)"?"); }
01909                                     else if ( j > -WILDOFFSET ) {
01910                                         Out = buffer;
01911                                         Out = NumCopy((UWORD)(-j + 4),Out);
01912                                         *Out++ = '_';
01913                                         *Out = 0;
01914                                         TokenToLine(buffer);
01915                                     }
01916                                     else {
01917                                         TokenToLine(FindVector(j));
01918 /*                                     TokenToLine(VARNAME(vectors,j - AM.OffsetVector)); */
01919                                     }
01920                                     }
01921                                     AO.CurrentDictionary = curdict;
01922                                     }
01923                                     else {
01924                                         int curdict = AO.CurrentDictionary;
01925                                         if ( AO.CurrentDictionary && AO.CurDictNotInFunctions > 0 )
01926                                             AO.CurrentDictionary = 0;
01927                                         while ( t < stopper ) {
01928                                             if ( !first ) TokenToLine((UBYTE *)",");
01929                                             WriteArgument(t);
01930                                             NEXTARG(t);
01931                                             first = 0;
01932                                         }
01933                                         AO.CurrentDictionary = curdict;
01934                                     }
01935                                     TokenToLine(closepar);
01936                                     closepar[0] = (UBYTE)'\'';
01937                                     break;
01938                                     }
01939                                     return(0);
01940     }
01941
01942 /*
01943     #] WriteSubTerm :
01944     #[ WriteInnerTerm :      WORD WriteInnerTerm(term,first)
01945
01946     Writes the contents of term to the output.
01947     Only the part that is inside parentheses is written.
01948
01949 */
01950
01951 WORD WriteInnerTerm(WORD *term, WORD first)
01952 {
01953     WORD *t, *s, *s1, *s2, n, i, pow;
01954 #ifdef WITHFLOAT
01955     int FloatChars = 0;
01956     GETIDENTITY
01957 #endif
01958     t = term;
01959     s = t+1;
01960     GETCOEF(t,n);
01961     while ( s < t ) {
01962         if ( *s == HAAKJE ) break;
01963         s += s[1];
01964     }
01965     if ( s < t ) { s += s[1]; }
01966     else { s = term+1; }
01967
01968     if ( n < 0 || !first ) {
01969         if ( n > 0 ) { TOKENTOLINE(" + ","+") }
01970         else if ( n < 0 ) { n = -n; TOKENTOLINE(" - ","-") }
01971     }
01972     if ( AC.modpowers ) {
01973         if ( n == 1 && *t == 1 && t > s ) first = 1;
01974         else if ( ABS(AC.ncmod) == 1 ) {
01975             UBYTE *Out1 = IsExponentSign();
01976             LongToLine((UWORD *)AC.powmod,AC.npowmod);
01977             if ( Out1 ) TokenToLine(Out1);
01978             else TokenToLine((UBYTE *)"^");
01979             TalToLine(AC.modpowers[(LONG)((UWORD)*t)]);

```

```

01980         first = 0;
01981     }
01982     else {
01983         LONG jj;
01984         UBYTE *Out1 = IsExponentSign();
01985         LongToLine((UWORD *)AC.powmod,AC.npowmod);
01986         if ( Out1 ) TokenToLine(Out1);
01987         else TokenToLine((UBYTE *)"^");
01988         jj = (UWORD)*t;
01989         if ( n == 2 ) jj += ((LONG)t[1])«BITSINWORD;
01990         if ( AC.modpowers[jj+1] == 0 ) {
01991             TalToLine(AC.modpowers[jj]);
01992         }
01993         else {
01994             LongToLine(AC.modpowers+jj,2);
01995         }
01996         first = 0;
01997     }
01998 }
01999 else if ( n != 1 || *t != 1 || t[1] != 1 || t <= s ) {
02000     if ( lowestlevel && ( ( AO.PrintType & PRINTONEFUNCTION ) != 0 ) ) {
02001         FiniLine();
02002         if ( AC.OutputSpaces == NOSPACEFORMAT ) IniLine(1);
02003         else IniLine(3);
02004     }
02005     if ( AO.CurrentDictionary > 0 ) TransformRational((UWORD *)t,n);
02006     else RatToLine((UWORD *)t,n);
02007     first = 0;
02008 }
02009 #ifdef WITHFLOAT
02010 /*
02011     Check whether there is a 'proper' float_ function and no raw mode.
02012     If so, print as float.
02013     Raw mode is indicated as AO.FloatPrec < 0.
02014     AO.FloatPrec == 0 indicated as many digits as the precision allows.
02015 */
02016     else if ( AO.FloatPrec >= 0 && AT.aux_ != 0 ) {
02017         WORD *ss = s;
02018         while ( ss < t ) {
02019             if ( *ss == FLOATFUN ) {
02020                 if ( ( FloatChars = PrintFloat(ss,AO.FloatPrec) ) != 0 ) {
02021                     TokenToLine(AO.floatspace);
02022                     first = 0;
02023                 }
02024                 break;
02025             }
02026             ss += ss[1];
02027         }
02028         if ( ss >= t ) first = 1;
02029     }
02030 #endif
02031     else first = 1;
02032     while ( s < t ) {
02033         if ( lowestlevel && ( (AO.PrintType & (PRINTONEFUNCTION | PRINTALL)) == PRINTONEFUNCTION ) ) {
02034             FiniLine();
02035             if ( AC.OutputSpaces == NOSPACEFORMAT ) IniLine(1);
02036             else IniLine(3);
02037         }
02038     }
02039 /*
02040     #[ NEWGAMMA :
02041 */
02042 #ifdef NEWGAMMA
02043     if ( *s == GAMMA ) { /* String them up */
02044         WORD *tt,*ss;
02045         ss = AT.WorkPointer;
02046         *ss++ = GAMMA;
02047         *ss++ = s[1];
02048         FILLFUN(ss)
02049         *ss++ = s[FUNHEAD];
02050         tt = s + FUNHEAD + 1;
02051         n = s[1] - FUNHEAD-1;
02052         do {
02053             while ( --n >= 0 ) *ss++ = *tt++;
02054             tt = s + s[1];
02055             while ( *tt == GAMMA && tt[FUNHEAD] == s[FUNHEAD] && tt < t ) {
02056                 s = tt;
02057                 tt += FUNHEAD + 1;
02058                 n = s[1] - FUNHEAD-1;
02059                 if ( n > 0 ) break;
02060             } while ( n > 0 );
02061             tt = AT.WorkPointer;
02062             AT.WorkPointer = ss;
02063             tt[1] = WORDDIF(ss,tt);
02064             if ( WriteSubTerm(tt,first) ) {
02065                 MesCall("WriteInnerTerm");

```

```

02067         SETERROR(-1)
02068     }
02069     AT.WorkPointer = tt;
02070 }
02071 else
02072 #endif
02073 /*
02074     #] NEWGAMMA :
02075 */
02076 #ifdef WITHFLOAT
02077     if ( *s == FLOATFUN && AO.FloatPrec >= 0 && AT.aux_ != 0 ) {
02078     }
02079     else
02080 #endif
02081     {
02082         if ( *s >= FUNCTION && AC.funpowers > 0
02083             && functions[*s-FUNCTION].spec == 0 && ( AC.funpowers == ALLFUNPOWERS ||
02084             ( AC.funpowers == COMFUNPOWERS && functions[*s-FUNCTION].commute == 0 ) ) ) {
02085             pow = 1;
02086             for(;;) {
02087                 s1 = s; s2 = s + s[1]; i = s[1];
02088                 if ( s2 < t ) {
02089                     while ( --i >= 0 && *s1 == *s2 ) { s1++; s2++; }
02090                     if ( i < 0 ) {
02091                         pow++; s = s+s[1];
02092                     }
02093                     else break;
02094                 }
02095                 else break;
02096             }
02097             if ( pow > 1 ) {
02098                 if ( AC.OutputMode == CMODE ) {
02099                     if ( !first ) MultiplyToLine();
02100                     TokenToLine((UBYTE *) "pow(");
02101                     first = 1;
02102                 }
02103                 if ( WriteSubTerm(s,first) ) {
02104                     MesCall("WriteInnerTerm");
02105                     SETERROR(-1)
02106                 }
02107                 if ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE
02108                     || AC.OutputMode == REDUCEMODE ) { TokenToLine((UBYTE *) "***"); }
02109                 else if ( AC.OutputMode == CMODE ) { TokenToLine((UBYTE *) ","); }
02110                 else {
02111                     UBYTE *Out1 = IsExponentSign();
02112                     if ( Out1 ) TokenToLine(Out1);
02113                     else TokenToLine((UBYTE *) "^");
02114                 }
02115                 TalToLine(pow);
02116                 if ( AC.OutputMode == CMODE ) TokenToLine((UBYTE *) ")");
02117             }
02118             else if ( WriteSubTerm(s,first) ) {
02119                 MesCall("WriteInnerTerm");
02120                 SETERROR(-1)
02121             }
02122         }
02123         else if ( WriteSubTerm(s,first) ) {
02124             MesCall("WriteInnerTerm");
02125             SETERROR(-1)
02126         }
02127     }
02128     first = 0;
02129     s += s[1];
02130 }
02131 return(0);
02132 }
02133
02134 /*
02135     #] WriteInnerTerm :
02136     #[ WriteTerm :      WORD WriteTerm(term,lbrac,first,prtf,br)
02137
02138     Writes a term to output. It tests the bracket information first.
02139     If there are no brackets or the bracket is the same all is passed
02140     to WriteInnerTerm. If there are brackets and the bracket is not
02141     the same as for the predecessor the old bracket is closed and
02142     a new one is opened.
02143     br indicates whether we are in a subexpression, barring zeroing
02144     AO.IsBracket
02145
02146 */
02147
02148 WORD WriteTerm(WORD *term, WORD *lbrac, WORD first, WORD prtf, WORD br)
02149 {
02150     WORD *t, *stopper, *b, n;
02151     int oldIsFortran90 = AC.IsFortran90, i;
02152     if ( *lbrac >= 0 ) {
02153         t = term + 1;

```

```

02154     stopper = (term + *term - 1);
02155     stopper -= ABS(*stopper) - 1;
02156     while ( t < stopper ) {
02157         if ( *t == HAAKJE ) {
02158             stopper = t;
02159             t = term+1;
02160             if ( *lbrac == ( n = WORDDIF(stopper,t) ) ) {
02161                 b = AO.bracket + 1;
02162                 t = term + 1;
02163                 while ( n > 0 && ( *b++ == *t++ ) ) { n--; }
02164                 if ( n <= 0 && ( ( AM.FortranCont <= 0 || AO.InFbrack < AM.FortranCont )
02165                     || ( lowestlevel == 0 ) ) ) {
02166                     /*
02167                         We continue inside a bracket.
02168                     */
02169                     AO.IsBracket = 1;
02170                     if ( ( prtft & PRINTCONTENTS ) != 0 ) {
02171                         AO.NumInBrack++;
02172                     }
02173                     else {
02174                         if ( WriteInnerTerm(term,0) ) goto WrtTmes;
02175                         if ( ( AO.PrintType & PRINTONETERM ) != 0 ) {
02176                             FiniLine();
02177                             TokenToLine((UBYTE *) "    ");
02178                         }
02179                     }
02180                     return(0);
02181                 }
02182                 t = term + 1;
02183                 n = WORDDIF(stopper,t);
02184             }
02185             /*
02186                 Close the bracket
02187             */
02188             if ( *lbrac ) {
02189                 if ( ( prtft & PRINTCONTENTS ) ) PrtTerms();
02190                 TOKENTOLINE(" )","")
02191                 if ( AC.OutputMode == CMODE && AO.FactorMode == 0 )
02192                     TokenToLine((UBYTE *) ";");
02193                 else if ( AO.FactorMode && ( n == 0 ) ) {
02194                     /*
02195                         This should not happen.
02196                     */
02197                     return(0);
02198                 }
02199                 AC.IsFortran90 = ISNOTFORTRAN90;
02200                 FiniLine();
02201                 AC.IsFortran90 = oldIsFortran90;
02202                 if ( AC.OutputMode != FORTRANMODE && AC.OutputMode != PFORTRANMODE
02203                     && AC.OutputSpaces == NORMALFORMAT
02204                     && AO.FactorMode == 0 ) FiniLine();
02205             }
02206             else {
02207                 if ( AC.OutputMode == CMODE && AO.FactorMode == 0 )
02208                     TokenToLine((UBYTE *) ";");
02209                 if ( AO.FortFirst == 0 ) {
02210                     if ( !first ) {
02211                         AC.IsFortran90 = ISNOTFORTRAN90;
02212                         FiniLine();
02213                         AC.IsFortran90 = oldIsFortran90;
02214                     }
02215                 }
02216             }
02217             if ( AO.FactorMode == 0 ) {
02218                 if ( ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE )
02219                     && !first ) {
02220                     WORD oldmode = AC.OutputMode;
02221                     AC.OutputMode = 0;
02222                     IniLine(0);
02223                     AC.OutputMode = oldmode;
02224                     AO.OutSkip = 7;
02225                 }
02226                 if ( AO.FortFirst == 0 ) {
02227                     TokenToLine(AO.CurBufWrt);
02228                     TOKENTOLINE(" = ","=")
02229                     TokenToLine(AO.CurBufWrt);
02230                 }
02231                 else {
02232                     AO.FortFirst = 0;
02233                     TokenToLine(AO.CurBufWrt);
02234                     TOKENTOLINE(" = ","=")
02235                 }
02236             }
02237             else if ( AC.OutputMode == CMODE && !first ) {
02238                 IniLine(0);
02239                 if ( AO.FortFirst == 0 ) {
02240                     TokenToLine(AO.CurBufWrt);

```

```

02241         TOKENTOLINE(" += ", "+=")
02242     }
02243     else {
02244         AO.FortFirst = 0;
02245         TokenToLine(AO.CurBufWrt);
02246         TOKENTOLINE(" = ", "=")
02247     }
02248 }
02249 else if ( startinline == 0 ) {
02250     IniLine(0);
02251 }
02252 AO.InFbrack = 0;
02253 if ( ( *lbrac = n ) > 0 ) {
02254     b = AO.bracket;
02255     *b++ = n + 4;
02256     while ( --n >= 0 ) *b++ = *t++;
02257     *b++ = 1; *b++ = 1; *b = 3;
02258     AO.IsBracket = 0;
02259     if ( WriteInnerTerm(AO.bracket,0) ) {
02260         /* Error message */
02261         WORD i;
02262         t = term;
02263         AO.OutSkip = 3;
02264         FiniLine();
02265         i = *t;
02266         while ( --i >= 0 ) { TalToLine((UWORD)(t++));
02267             if ( AC.OutputSpaces == NORMALFORMAT )
02268                 TokenToLine((UBYTE *)" "); }
02269         AO.OutSkip = 0;
02270         FiniLine();
02271         MesCall("WriteTerm");
02272         SETERROR(-1)
02273     }
02274     TOKENTOLINE(" * ( ", "*(")
02275     AO.NumInBrack = 0;
02276     AO.IsBracket = 1;
02277     if ( ( prt & PRINTONETERM ) != 0 ) {
02278         first = 0;
02279         FiniLine();
02280         TokenToLine((UBYTE *)" ");
02281     }
02282     else first = 1;
02283 }
02284 else {
02285     AO.IsBracket = 0;
02286     first = 0;
02287 }
02288 }
02289 else {
02290     /*
02291     Here is the code that writes the glue between two factors.
02292     We should not forget factors that are zero!
02293     */
02294     if ( ( *lbrac = n ) > 0 ) {
02295         b = AO.bracket;
02296         *b++ = n + 4;
02297         while ( --n >= 0 ) *b++ = *t++;
02298         *b++ = 1; *b++ = 1; *b = 3;
02299         for ( i = AO.FactorNum+1; i < AO.bracket[4]; i++ ) {
02300             if ( first ) {
02301                 TOKENTOLINE(" ( 0 )", "(0)")
02302                 first = 0;
02303             }
02304             else {
02305                 TOKENTOLINE(" * ( 0 )", "* (0)")
02306             }
02307             FiniLine();
02308             IniLine(0);
02309         }
02310         AO.FactorNum = AO.bracket[4];
02311     }
02312     else {
02313         AO.NumInBrack = 0;
02314         return(0);
02315     }
02316     if ( first == 0 ) { TOKENTOLINE(" * ( ", "*(") }
02317     else { TOKENTOLINE(" ( ", "(" ) }
02318     AO.NumInBrack = 0;
02319     first = 1;
02320 }
02321 if ( ( prt & PRINTCONTENTS ) != 0 ) AO.NumInBrack++;
02322 else if ( WriteInnerTerm(term,first) ) goto WrtTmes;
02323 if ( ( AO.PrintType & PRINTONETERM ) != 0 ) {
02324     FiniLine();
02325     TokenToLine((UBYTE *)" ");
02326 }
02327 return(0);

```

```

02328         }
02329         else t += t[1];
02330     }
02331     if ( *lbrac > 0 ) {
02332         if ( ( prtfl & PRINTCONTENTS ) != 0 ) PrtTerms();
02333         TokenToLine((UBYTE *) " ");
02334         if ( AC.OutputMode == CMODE ) TokenToLine((UBYTE *) "");
02335         if ( AO.FortFirst == 0 ) {
02336             AC.IsFortran90 = ISNOTFORTRAN90;
02337             FiniLine();
02338             AC.IsFortran90 = oldIsFortran90;
02339         }
02340         if ( AC.OutputMode != FORTRANMODE && AC.OutputMode != PFORTRANMODE
02341             && AC.OutputSpaces == NORMALFORMAT ) FiniLine();
02342         if ( ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE )
02343             && !first ) {
02344             WORD oldmode = AC.OutputMode;
02345             AC.OutputMode = 0;
02346             IniLine(0);
02347             AC.OutputMode = oldmode;
02348             AO.OutSkip = 7;
02349             if ( AO.FortFirst == 0 ) {
02350                 TokenToLine(AO.CurBufWrt);
02351                 TOKENTOLINE(" = ", "=");
02352                 TokenToLine(AO.CurBufWrt);
02353             }
02354             else {
02355                 AO.FortFirst = 0;
02356                 TokenToLine(AO.CurBufWrt);
02357                 TOKENTOLINE(" = ", "=");
02358             }
02359             /*
02360             TokenToLine(AO.CurBufWrt);
02361             TOKENTOLINE(" = ", "=");
02362             if ( AO.FortFirst == 0 )
02363                 TokenToLine(AO.CurBufWrt);
02364             else AO.FortFirst = 0;
02365             */
02366         }
02367         else if ( AC.OutputMode == CMODE && !first ) {
02368             IniLine(0);
02369             if ( AO.FortFirst == 0 ) {
02370                 TokenToLine(AO.CurBufWrt);
02371                 TOKENTOLINE(" += ", "+=");
02372             }
02373             else {
02374                 AO.FortFirst = 0;
02375                 TokenToLine(AO.CurBufWrt);
02376                 TOKENTOLINE(" = ", "=");
02377             }
02378             /*
02379             TokenToLine(AO.CurBufWrt);
02380             if ( AO.FortFirst == 0 ) { TOKENTOLINE(" += ", "+=") }
02381             else {
02382                 TOKENTOLINE(" = ", "=");
02383                 AO.FortFirst = 0;
02384             }
02385             */
02386         }
02387         else IniLine(0);
02388         *lbrac = 0;
02389         first = 1;
02390     }
02391 }
02392 if ( !br ) AO.IsBracket = 0;
02393 if ( ( AM.FortranCont > 0 && AO.InFbrack >= AM.FortranCont ) && lowestlevel ) {
02394     if ( AC.OutputMode == CMODE ) TokenToLine((UBYTE *) "");
02395     if ( ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE )
02396         && !first ) {
02397         WORD oldmode = AC.OutputMode;
02398         if ( AO.FortFirst == 0 ) {
02399             AC.IsFortran90 = ISNOTFORTRAN90;
02400             FiniLine();
02401             AC.IsFortran90 = oldIsFortran90;
02402             AC.OutputMode = 0;
02403             IniLine(0);
02404             AC.OutputMode = oldmode;
02405             AO.OutSkip = 7;
02406             TokenToLine(AO.CurBufWrt);
02407             TOKENTOLINE(" = ", "=");
02408             TokenToLine(AO.CurBufWrt);
02409         }
02410         else {
02411             AO.FortFirst = 0;
02412             /*
02413             TokenToLine(AO.CurBufWrt);
02414             TOKENTOLINE(" = ", "=");

```



```

02415 */
02416     }
02417 /*
02418     TokenToLine(AO.CurBufWrt);
02419     TOKENTOLINE(" = ", "=")
02420     if ( AO.FortFirst == 0 )
02421         TokenToLine(AO.CurBufWrt);
02422     else AO.FortFirst = 0;
02423 */
02424 }
02425 else if ( AC.OutputMode == CMODE && !first ) {
02426     FiniLine();
02427     IniLine(0);
02428     if ( AO.FortFirst == 0 ) {
02429         TokenToLine(AO.CurBufWrt);
02430         TOKENTOLINE(" += ", "+=")
02431     }
02432     else {
02433         AO.FortFirst = 0;
02434         TokenToLine(AO.CurBufWrt);
02435         TOKENTOLINE(" = ", "=")
02436     }
02437 /*
02438     TokenToLine(AO.CurBufWrt);
02439     if ( AO.FortFirst == 0 ) { TOKENTOLINE(" += ", "+=") }
02440     else {
02441         TOKENTOLINE(" = ", "=")
02442         AO.FortFirst = 0;
02443     }
02444 */
02445 }
02446 else {
02447     FiniLine();
02448     IniLine(0);
02449 }
02450 AO.InFbrack = 0;
02451 }
02452 if ( WriteInnerTerm(term,first) ) goto WrtTmes;
02453 if ( ( AO.PrintType & PRINTONETERM ) != 0 ) {
02454     FiniLine();
02455     IniLine(0);
02456 }
02457 return(0);
02458 }
02459
02460 /*
02461     #] WriteTerm :
02462     #[ WriteExpression :      WORD WriteExpression(terms,ltot)
02463
02464     Writes a subexpression to output.
02465     The subexpression is in terms and contains ltot words.
02466     This is only used for function arguments.
02467
02468 */
02469
02470 WORD WriteExpression(WORD *terms, LONG ltot)
02471 {
02472     WORD *stopper;
02473     WORD first, btot;
02474     WORD OldIsBracket, OldPrintType = AO.PrintType;
02475     if ( !AC.outsidefun ) { AO.PrintType &= ~PRINTONETERM; first = 1; }
02476     else first = 0;
02477     stopper = terms + ltot;
02478     btot = -1;
02479     while ( terms < stopper ) {
02480         AO.IsBracket = OldIsBracket;
02481         if ( WriteTerm(terms,&btot,first,0,1) ) {
02482             MesCall("WriteExpression");
02483             SETERROR(-1)
02484         }
02485         first = 0;
02486         terms += *terms;
02487     }
02488 /* AO.IsBracket = 0; */
02489 AO.IsBracket = OldIsBracket;
02490 AO.PrintType = OldPrintType;
02491 return(0);
02492 }
02493
02494 /*
02495     #] WriteExpression :
02496     #[ WriteAll :          WORD WriteAll()
02497
02498     Writes all expressions that should be written
02499 */
02500
02501 WORD WriteAll(void)

```

```

02502 {
02503     GETIDENTITY
02504     WORD lbrac, first;
02505     WORD *t, *stopper, n, prtf;
02506     int oldIsFortran90 = AC.IsFortran90, i;
02507     POSITION pos;
02508     FILEHANDLE *f;
02509     EXPRESSIONS e;
02510     if ( AM.exitflag ) return(0);
02511 #ifdef WITHMPI
02512     if ( PF.me != MASTER ) {
02513         /*
02514          * For the slaves, we need to call Optimize() the same number of times
02515          * as the master. The first argument doesn't have any important role.
02516          */
02517         for ( n = 0; n < NumExpressions; n++ ) {
02518             e = &Expressions[n];
02519             if ( (!e->printflag) & PRINTON ) continue;
02520             switch ( e->status ) {
02521                 case LOCALEXPRESSION:
02522                 case GLOBALEXPRESSION:
02523                 case UNHIDELEXPRESSION:
02524                 case UNHIDEGEXPRESSION:
02525                     break;
02526                 default:
02527                     continue;
02528             }
02529             e->printflag = 0;
02530             PutPreVar(AM.oldnumextrasymbols, GetPreVar((UBYTE *)"EXTRASYNBOLS_", 0), 0, 1);
02531             if ( AO.OptimizationLevel > 0 ) {
02532                 if ( Optimize(0, 1) ) return(-1);
02533             }
02534         }
02535         return(0);
02536     }
02537 #endif
02538     SeekScratch(AR.outfile, &pos);
02539     if ( ResetScratch() ) {
02540         MesCall("WriteAll");
02541         SETERROR(-1)
02542     }
02543     AO.termbuf = AT.WorkPointer;
02544     AO.bracket = (WORD *)((UBYTE *) (AT.WorkPointer)) + AM.MaxTer;
02545     AT.WorkPointer = (WORD *)((UBYTE *) (AT.WorkPointer)) + AM.MaxTer*2;
02546     AO.OutFill = AO.OutputLine = (UBYTE *)AT.WorkPointer;
02547     AT.WorkPointer += 2*AC.LineLength;
02548     *(AR.CompressBuffer) = 0;
02549     first = 0;
02550     for ( n = 0; n < NumExpressions; n++ ) {
02551         if ( ( Expressions[n].printflag & PRINTON ) != 0 ) { first = 1; break; }
02552     }
02553     if ( !first ) goto EndWrite;
02554     AO.IsBracket = 0;
02555     AO.OutSkip = 3;
02556     AR.DeferFlag = 0;
02557     while ( GetTerm(BHEAD AO.termbuf) ) {
02558         t = AO.termbuf + 1;
02559         e = Expressions + AO.termbuf[3];
02560         n = e->status;
02561         if ( ( n == LOCALEXPRESSION || n == GLOBALEXPRESSION
02562             || n == UNHIDELEXPRESSION || n == UNHIDEGEXPRESSION ) &&
02563             ( ( prtf = e->printflag ) & PRINTON ) != 0 ) {
02564             e->printflag = 0;
02565             AO.NumInBrack = 0;
02566             PutPreVar(AM.oldnumextrasymbols,
02567                 GetPreVar((UBYTE *)"EXTRASYNBOLS_", 0), 0, 1);
02568             if ( ( prtf & PRINTLFILE ) != 0 ) {
02569                 if ( AC.LogHandle < 0 ) prtf &= ~PRINTLFILE;
02570             }
02571             AO.PrintType = prtf;
02572             /*
02573              if ( AC.OutputMode == VORTRANMODE ) {
02574                  UBYTE *oldOutFill = AO.OutFill, *oldOutputLine = AO.OutputLine;
02575                  AO.OutSkip = 6;
02576                  if ( Optimize(AO.termbuf[3], 1) ) goto AboWrite;
02577                  AO.OutSkip = 3;
02578                  AO.OutFill = oldOutFill; AO.OutputLine = oldOutputLine;
02579                  FiniLine();
02580                  continue;
02581              }
02582              else
02583              */
02584             if ( AO.OptimizationLevel > 0 ) {
02585                 UBYTE *oldOutFill = AO.OutFill, *oldOutputLine = AO.OutputLine;
02586                 AO.OutSkip = 6;
02587                 if ( Optimize(AO.termbuf[3], 1) ) goto AboWrite;
02588                 AO.OutSkip = 3;

```

```

02589         AO.OutFill = oldOutFill; AO.OutputLine = oldOutputLine;
02590         FiniLine();
02591         continue;
02592     }
02593     if ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE )
02594         AO.OutSkip = 6;
02595     FiniLine();
02596     AO.CurBufWrt = EXPRNAME(AO.termbuf[3]);
02597     TokenToLine(AO.CurBufWrt);
02598     stopper = t + t[1];
02599     t += SUBEXPSIZE;
02600     if ( t < stopper ) {
02601         TokenToLine((UBYTE *) "(");
02602         first = 1;
02603         while ( t < stopper ) {
02604             n = *t;
02605             if ( !first ) TokenToLine((UBYTE *) ",");
02606             switch ( n ) {
02607                 case SYMTOSYM :
02608                     TokenToLine(FindSymbol(t[2]));
02609                     TokenToLine(VARNAME(symbols,t[2])); /*
02610                     break;
02611                 case VECTOVEC :
02612                     TokenToLine(FindVector(t[2]));
02613                     TokenToLine(VARNAME(vectors,t[2] - AM.OffsetVector)); /*
02614                     break;
02615                 case INDTOIND :
02616                     TokenToLine(FindIndex(t[2]));
02617                     TokenToLine(VARNAME(indices,t[2] - AM.OffsetIndex)); /*
02618                     break;
02619                 default :
02620                     TokenToLine(FindFunction(t[2]));
02621                     TokenToLine(VARNAME(functions,t[2] - FUNCTION)); /*
02622                     break;
02623             }
02624             t += t[1];
02625             first = 0;
02626         }
02627         TokenToLine((UBYTE *) ")");
02628     }
02629     TOKENTOLINE(" =", "=");
02630     if ( AC.OutputMode == MATHEMATICAMODE ) {
02631         TOKENTOLINE(" (","(");
02632     }
02633     lbrac = 0;
02634     AO.InFbrack = 0;
02635     if ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE )
02636         AO.FortFirst = 1;
02637     else
02638         AO.FortFirst = 0;
02639     first = 1;
02640     if ( ( e->vflags & ISFACTORIZED ) != 0 ) {
02641         AO.FactorMode = 1+e->numfactors;
02642         AO.FactorNum = 0; /* Which factor are we doing. For factors that are zero */
02643     }
02644     else {
02645         AO.FactorMode = 0;
02646     }
02647     while ( GetTerm(BHEAD AO.termbuf) ) {
02648         WORD *m;
02649         GETSTOP(AO.termbuf,m);
02650         if ( ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE )
02651             && ( ( prtf & PRINTONETERM ) != 0 ) ) {}
02652         else {
02653             if ( first ) {
02654                 FiniLine();
02655                 IniLine(0);
02656             }
02657             if ( ( prtf & PRINTONETERM ) != 0 ) first = 0;
02658             if ( WriteTerm(AO.termbuf,&lbrac,first,prtf,0) )
02659                 goto AboWrite;
02660             first = 0;
02661         }
02662     }
02663     if ( AO.FactorMode ) {
02664         if ( first ) { AO.FactorNum = 1; TOKENTOLINE(" ( 0 )"," (0)") }
02665         else TOKENTOLINE(" )","");
02666         for ( i = AO.FactorNum+1; i <= e->numfactors; i++ ) {
02667             FiniLine();
02668             IniLine(0);
02669             TOKENTOLINE(" * ( 0 )","*(0)");
02670         }
02671         AO.FactorNum = e->numfactors;
02672         if ( AC.OutputMode != FORTRANMODE && AC.OutputMode != PFORTRANMODE )
02673             TokenToLine((UBYTE *) ",");
02674     }
02675     else if ( AO.FactorMode == 0 || first ) {

```

```

02676         if ( first ) { TOKENTOline(" 0","0" ) }
02677     else if ( lbrac ) {
02678         if ( ( prtf & PRINTCONTENTS ) != 0 ) PrtTerms();
02679         TOKENTOline(" ",")")
02680     }
02681     else if ( ( prtf & PRINTCONTENTS ) != 0 ) {
02682         TOKENTOline(" + 1 * ( ", "+1*(")
02683         PrtTerms();
02684         TOKENTOline(" )",")")
02685     }
02686     if ( AC.OutputMode == MATHEMATICAMODE ) {
02687         TokenToLine((UBYTE *)");");
02688     }
02689     if ( AC.OutputMode != FORTRANMODE && AC.OutputMode != PFORTRANMODE )
02690         TokenToLine((UBYTE *)";");
02691 }
02692 AO.OutSkip = 3;
02693 AC.IsFortran90 = ISNOTFORTRAN90;
02694 FiniLine();
02695 AC.IsFortran90 = oldIsFortran90;
02696 AO.FactorMode = 0;
02697 }
02698 else {
02699     do { } while ( GetTerm(BHEAD AO.termbuf) );
02700 }
02701 }
02702 if ( AC.OutputSpaces == NORMALFORMAT ) FiniLine();
02703 EndWrite:
02704 if ( AR.infile->handle >= 0 ) {
02705     SeekFile(AR.infile->handle, &(AR.infile->filesize), SEEK_SET);
02706 }
02707 AO.IsBracket = 0;
02708 AT.WorkPointer = AO.termbuf;
02709 SetScratch(AR.infile, &pos);
02710 f = AR.outfile; AR.outfile = AR.infile; AR.infile = f;
02711 return(0);
02712 AboWrite:
02713 SetScratch(AR.infile, &pos);
02714 f = AR.outfile; AR.outfile = AR.infile; AR.infile = f;
02715 MesCall("WriteAll");
02716 Terminate(-1);
02717 return(-1);
02718 }
02719
02720 /*
02721     #] WriteAll :
02722     #[ WriteOne :          WORD WriteOne(name,alreadyinline)
02723
02724     Writes one expression from the preprocessor
02725 */
02726
02727 WORD WriteOne(UBYTE *name, int alreadyinline, int nosemi, WORD plus)
02728 {
02729     GETIDENTITY
02730     WORD number;
02731     WORD lbrac, first;
02732     POSITION pos;
02733     FILEHANDLE *f;
02734     WORD prf;
02735
02736     if ( GetName(AC.exprnames, name, &number, NOAUTO) != CEXPRESSION ) {
02737         MesPrint("@%s is not an expression", name);
02738         return(-1);
02739     }
02740     switch ( Expressions[number].status ) {
02741         case HIDDENLEXPRESSION:
02742         case HIDDENGEXPRESSION:
02743         case HIDELEXPRESSION:
02744         case HIDEGEXPRESSION:
02745         case UNHIDELEXPRESSION:
02746         case UNHIDEGEXPRESSION:
02747     /*
02748         case DROPHLEXPRESSION:
02749         case DROPHGEXPRESSION:
02750     */
02751         AR.GetFile = 2;
02752         break;
02753         case LOCALEXPRESSION:
02754         case GLOBALEXPRESSION:
02755         case SKIPLEXPRESSION:
02756         case SKIPGEXPRESSION:
02757     /*
02758         case DROPLEXPRESSION:
02759         case DROPGEXPRESSION:
02760     */
02761         AR.GetFile = 0;
02762         break;

```

```

02763         default:
02764             MesPrint("@expressions %s is not active. It cannot be written",name);
02765             return(-1);
02766     }
02767     SeekScratch(AR.outfile,&pos);
02768
02769     f = AR.outfile; AR.outfile = AR.infile; AR.infile = f;
02770 /*
02771     if ( ResetScratch() ) {
02772         MesCall("WriteOne");
02773         SETERROR(-1)
02774     }
02775 */
02776     if ( AR.GetFile == 2 ) f = AR.hidefile;
02777     else f = AR.infile;
02778     prf = Expressions[number].printflag;
02779     if ( plus ) prf |= PRINTONETERM;
02780 /*
02781         Now position the file
02782 */
02783     if ( f->handle >= 0 ) {
02784         SetScratch(f,&(Expressions[number].onfile));
02785     }
02786     else {
02787         f->POfill = (WORD *) ((UBYTE *) (f->PObuffer)
02788             + BASEPOSITION(Expressions[number].onfile));
02789     }
02790     AO.termbuf = AT.WorkPointer;
02791     AO.bracket = (WORD *) ((UBYTE *) (AT.WorkPointer)) + AM.MaxTer;
02792     AT.WorkPointer = (WORD *) ((UBYTE *) (AT.WorkPointer)) + AM.MaxTer*2);
02793
02794     AO.OutFill = AO.OutputLine = (UBYTE *) AT.WorkPointer;
02795     AT.WorkPointer += 2*AC.LineLength;
02796     *(AR.CompressBuffer) = 0;
02797
02798     AO.IsBracket = 0;
02799     AO.OutSkip = 3;
02800     AR.DeferFlag = 0;
02801
02802     if ( AC.OutputMode == FORTRANMODE || AC.OutputMode == PFORTRANMODE )
02803         AO.OutSkip = 6;
02804     if ( GetTerm(BHEAD AO.termbuf) <= 0 ) {
02805         MesPrint("@ReadError in expression %s",name);
02806         goto AboWrite;
02807     }
02808 /*
02809     PutPreVar(AM.oldnumextrasymbols,
02810         GetPreVar((UBYTE *) "EXTRASYMBOLS_",0),0,1);
02811 */
02812 /*
02813     * Currently WriteOne() is called only from writeToChannel() with setting
02814     * AO.OptimizationLevel = 0, which means Optimize() is never called here.
02815     * So we don't need to think about how to ensure that the master and the
02816     * slaves call Optimize() at the same time. (TU 26 Jul 2013)
02817     */
02818     if ( AO.OptimizationLevel > 0 ) {
02819         AO.OutSkip = 6;
02820         if ( Optimize(AO.termbuf[3], 1) ) goto AboWrite;
02821         AO.OutSkip = 3;
02822         FiniLine();
02823     }
02824     else {
02825         lbrac = 0;
02826         AO.InFbrack = 0;
02827         AO.FortFirst = 0;
02828         first = 1;
02829         while ( GetTerm(BHEAD AO.termbuf) ) {
02830             WORD *m;
02831             GETSTOP(AO.termbuf,m);
02832             if ( first ) {
02833                 IniLine(0);
02834                 startinline = alreadyinline;
02835                 AO.OutFill = AO.OutputLine + startinline;
02836                 if ( WriteTerm(AO.termbuf,&lbrac,first,0,0) )
02837                     goto AboWrite;
02838                 first = 0;
02839             }
02840             else {
02841                 if ( ( prf & PRINTONETERM ) != 0 ) first = 1;
02842                 if ( first ) {
02843                     FiniLine();
02844                     IniLine(0);
02845                 }
02846                 first = 0;
02847                 if ( WriteTerm(AO.termbuf,&lbrac,first,0,0) )
02848                     goto AboWrite;
02849             }

```

```

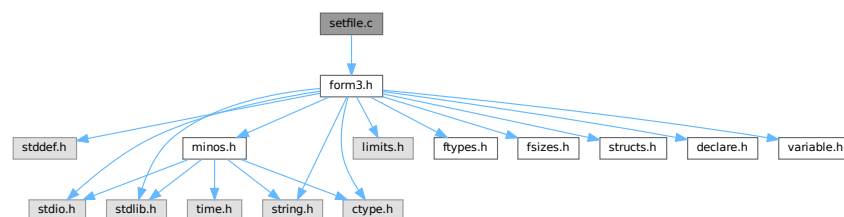
02850     }
02851     if ( first ) {
02852         IniLine(0);
02853         startinline = alreadyinline;
02854         AO.OutFill = AO.OutputLine + startinline;
02855         TOKENTOLINE(" 0","0");
02856     }
02857     else if ( lbrac ) {
02858         TOKENTOLINE(" )","");
02859     }
02860     if ( AC.OutputMode != FORTRANMODE && AC.OutputMode != PFORTRANMODE
02861         && nosemi == 0 ) TokenToLine((UBYTE *) "");
02862     AO.OutSkip = 3;
02863     if ( AC.OutputSpaces == NORMALFORMAT && nosemi == 0 ) {
02864         FiniLine();
02865     }
02866     else {
02867         noextralinefeed = 1;
02868         FiniLine();
02869         noextralinefeed = 0;
02870     }
02871 }
02872 AO.IsBracket = 0;
02873 AT.WorkPointer = AO.termbuf;
02874 SetScratch(f,&pos);
02875 f = AR.outfile; AR.outfile = AR.infile; AR.infile = f;
02876 AO.InFbrack = 0;
02877 return(0);
02878 AboWrite:
02879 SetScratch(AR.infile,&pos);
02880 f->POposition = pos;
02881 f = AR.outfile; AR.outfile = AR.infile; AR.infile = f;
02882 MesCall("WriteOne");
02883 Terminate(-1);
02884 return(-1);
02885 }
02886
02887 /*
02888     #] WriteOne :
02889     #] schryf-Writes :
02890 */

```

4.124 setfile.c File Reference

```
#include "form3.h"
```

Include dependency graph for setfile.c:



Macros

- #define NUMERICALVALUE 0
- #define STRINGVALUE 1
- #define PATHVALUE 2
- #define ONOFFVALUE 3
- #define DEFINEVALUE 4
- #define SETBUFSIZE 257

Functions

- int [DoSetups](#) (void)
- int [ProcessOption](#) (UBYTE *s1, UBYTE *s2, int filetype)
- [SETUPPARAMETERS](#) * [GetSetupPar](#) (UBYTE *s)
- int [RecalcSetups](#) (void)
- int [AllocSetups](#) (void)
- void [WriteSetup](#) (void)
- [SORTING](#) * [AllocSort](#) (LONG inLargeSize, LONG inSmallSize, LONG inSmallEsize, LONG inTermsInSmall, int inMaxPatches, int inMaxFpatches, LONG inIOsize, int level)
- void [AllocSortFileName](#) ([SORTING](#) *sort)
- [FILEHANDLE](#) * [AllocFileHandle](#) (WORD par, char *name)
- void [DeAllocFileHandle](#) ([FILEHANDLE](#) *fh)
- int [MakeSetupAllocs](#) (void)
- int [TryFileSetups](#) (void)
- int [TryEnvironment](#) (void)

Variables

- char [curdirp](#) [] = "."
- char [cursortdirp](#) [] = "."
- char [commentchar](#) [] = "*"
- char [dotchar](#) [] = "_"
- char [highfirst](#) [] = "highfirst"
- char [lowfirst](#) [] = "lowfirst"
- char [procedureextension](#) [] = "prc"
- [SETUPPARAMETERS](#) [setupparameters](#) []

4.124.1 Detailed Description

The routines that deal with the setup parameters.

Definition in file [setfile.c](#).

4.124.2 Macro Definition Documentation

4.124.2.1 NUMERICALVALUE

```
#define NUMERICALVALUE 0
```

Definition at line 47 of file [setfile.c](#).

4.124.2.2 STRINGVALUE

```
#define STRINGVALUE 1
```

Definition at line 48 of file [setfile.c](#).

4.124.2.3 PATHVALUE

```
#define PATHVALUE 2
```

Definition at line 49 of file [setfile.c](#).

4.124.2.4 ONOFFVALUE

```
#define ONOFFVALUE 3
```

Definition at line 50 of file [setfile.c](#).

4.124.2.5 DEFINEVALUE

```
#define DEFINEVALUE 4
```

Definition at line 51 of file [setfile.c](#).

4.124.2.6 SETBUFSIZE

```
#define SETBUFSIZE 257
```

Definition at line 1140 of file [setfile.c](#).

4.124.3 Function Documentation

4.124.3.1 DoSetups()

```
int DoSetups (  
    void )
```

Definition at line 132 of file [setfile.c](#).

4.124.3.2 ProcessOption()

```
int ProcessOption (  
    UBYTE * s1,  
    UBYTE * s2,  
    int filetype )
```

Definition at line 182 of file [setfile.c](#).

4.124.3.3 GetSetupPar()

```
SETUPPARAMETERS * GetSetupPar (  
    UBYTE * s )
```

Definition at line 337 of file [setfile.c](#).

4.124.3.4 RecalcSetups()

```
int RecalcSetups (
    void )
```

Definition at line 357 of file [setfile.c](#).

4.124.3.5 AllocSetups()

```
int AllocSetups (
    void )
```

Definition at line 404 of file [setfile.c](#).

4.124.3.6 WriteSetup()

```
void WriteSetup (
    void )
```

Definition at line 811 of file [setfile.c](#).

4.124.3.7 AllocSort()

```
SORTING * AllocSort (
    LONG inLargeSize,
    LONG inSmallSize,
    LONG inSmallEsize,
    LONG inTermsInSmall,
    int inMaxPatches,
    int inMaxFpatches,
    LONG inIOSize,
    int level )
```

Definition at line 869 of file [setfile.c](#).

4.124.3.8 AllocSortFileName()

```
void AllocSortFileName (
    SORTING * sort )
```

Definition at line 1019 of file [setfile.c](#).

4.124.3.9 AllocFileHandle()

```
FILEHANDLE * AllocFileHandle (
    WORD par,
    char * name )
```

Definition at line 1044 of file [setfile.c](#).

4.124.3.10 DeAllocFileHandle()

```
void DeAllocFileHandle (
    FILEHANDLE * fh )
```

Definition at line 1105 of file [setfile.c](#).

4.124.3.11 MakeSetupAllocs()

```
int MakeSetupAllocs (
    void )
```

Definition at line 1122 of file [setfile.c](#).

4.124.3.12 TryFileSetups()

```
int TryFileSetups (
    void )
```

Definition at line 1142 of file [setfile.c](#).

4.124.3.13 TryEnvironment()

```
int TryEnvironment (
    void )
```

Definition at line 1228 of file [setfile.c](#).

4.124.4 Variable Documentation

4.124.4.1 curdirp

```
char curdirp[] = "."
```

Definition at line 39 of file [setfile.c](#).

4.124.4.2 cursortdirp

```
char cursortdirp[] = "."
```

Definition at line 40 of file [setfile.c](#).

4.124.4.3 commentchar

```
char commentchar[] = "*"
```

Definition at line 41 of file [setfile.c](#).

4.124.4.4 dotchar

```
char dotchar[] = "_"
```

Definition at line 42 of file [setfile.c](#).

4.124.4.5 highfirst

```
char highfirst[] = "highfirst"
```

Definition at line 43 of file [setfile.c](#).

4.124.4.6 lowfirst

```
char lowfirst[] = "lowfirst"
```

Definition at line 44 of file [setfile.c](#).

4.124.4.7 procedureextension

```
char procedureextension[] = "prc"
```

Definition at line 45 of file [setfile.c](#).

4.124.4.8 setupparameters

```
SETUPPARAMETERS setupparameters[]
```

Definition at line 53 of file [setfile.c](#).

4.125 setfile.c

[Go to the documentation of this file.](#)

```
00001
00005 /* # [ License : */
00006 /*
00007 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00008 * When using this file you are requested to refer to the publication
00009 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00010 * This is considered a matter of courtesy as the development was paid
00011 * for by FOM the Dutch physics granting agency and we would like to
00012 * be able to track its scientific use to convince FOM of its value
00013 * for the community.
00014 *
00015 * This file is part of FORM.
00016 *
00017 * FORM is free software: you can redistribute it and/or modify it under the
00018 * terms of the GNU General Public License as published by the Free Software
00019 * Foundation, either version 3 of the License, or (at your option) any later
00020 * version.
00021 *
00022 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00023 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00024 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00025 * details.
00026 *
```

```

00027 *   You should have received a copy of the GNU General Public License along
00028 *   with FORM.  If not, see <http://www.gnu.org/licenses/>.
00029 */
00030 /* #] License : */
00031 /*
00032     #[ Includes :
00033
00034     Routines that deal with settings and the setup file
00035 */
00036
00037 #include "form3.h"
00038
00039 char curdirp[] = ".";
00040 char cursortdirp[] = ".";
00041 char commentchar[] = "*";
00042 char dotchar[] = "_";
00043 char highfirst[] = "highfirst";
00044 char lowfirst[] = "lowfirst";
00045 char procedureextension[] = "prc";
00046
00047 #define NUMERICALVALUE 0
00048 #define STRINGVALUE 1
00049 #define PATHVALUE 2
00050 #define ONOFFVALUE 3
00051 #define DEFINEVALUE 4
00052
00053 SETUPPARAMETERS setupparameters[] =
00054 {
00055     {(UBYTE *) "bracketindexsize",          NUMERICALVALUE, 0, (LONG) MAXBRACKETBUFFERSIZE}
00056     , {(UBYTE *) "commentchar",              STRINGVALUE, 0, (LONG) commentchar}
00057     , {(UBYTE *) "compresssize",             NUMERICALVALUE, 0, (LONG) COMPRESSBUFFER}
00058     , {(UBYTE *) "constindex",              NUMERICALVALUE, 0, (LONG) NUMFIXED}
00059     , {(UBYTE *) "continuationlines",        NUMERICALVALUE, 0, (LONG) FORTRANCONTINUATIONLINES}
00060 #ifdef WITHFLOAT
00061     , {(UBYTE *) "defaultprecision",         NUMERICALVALUE, 0, (LONG) DEFAULTTPRECISION}
00062 #endif
00063     , {(UBYTE *) "define",                   DEFINEVALUE, 0, (LONG) 0}
00064     , {(UBYTE *) "dotchar",                  STRINGVALUE, 0, (LONG) dotchar}
00065     , {(UBYTE *) "factorizationcache",       NUMERICALVALUE, 0, (LONG) FBUFFERSIZE}
00066     , {(UBYTE *) "filepatches",              NUMERICALVALUE, 0, (LONG) MAXFPATCHES}
00067     , {(UBYTE *) "functionlevels",           NUMERICALVALUE, 0, (LONG) MAXFLEVELS}
00068     , {(UBYTE *) "hidesize",                 NUMERICALVALUE, 0, (LONG) 0}
00069     , {(UBYTE *) "indir",                    PATHVALUE, 0, (LONG) curdirp}
00070     , {(UBYTE *) "indentspace",              NUMERICALVALUE, 0, (LONG) INDENTSPACE}
00071     , {(UBYTE *) "insidefirst",              ONOFFVALUE, 0, (LONG) 1}
00072     , {(UBYTE *) "jumpratio",                NUMERICALVALUE, 0, (LONG) JUMPRATIO}
00073     , {(UBYTE *) "largepatches",             NUMERICALVALUE, 0, (LONG) MAXPATCHES}
00074     , {(UBYTE *) "largesize",                NUMERICALVALUE, 0, (LONG) LARGEBUFFER}
00075     , {(UBYTE *) "maxnumbersize",            NUMERICALVALUE, 0, (LONG) 0}
00076     /* , {(UBYTE *) "maxnumbersize",         NUMERICALVALUE, 0, (LONG) MAXNUMBERSIZE} */
00077     , {(UBYTE *) "maxtermsize",              NUMERICALVALUE, 0, (LONG) MAXTER}
00078 #ifdef WITHFLOAT
00079     , {(UBYTE *) "maxweight",                NUMERICALVALUE, 0, (LONG) MAXWEIGHT}
00080 #endif
00081     , {(UBYTE *) "maxwildcards",             NUMERICALVALUE, 0, (LONG) MAXWILDC}
00082     , {(UBYTE *) "nospacesinnnumbers",        ONOFFVALUE, 0, (LONG) 0}
00083     , {(UBYTE *) "numstorecaches",           NUMERICALVALUE, 0, (LONG) NUMSTORECACHES}
00084     , {(UBYTE *) "nwritefinalstatistics",     ONOFFVALUE, 0, (LONG) 0}
00085     , {(UBYTE *) "nwriteprocessstatistics",   ONOFFVALUE, 0, (LONG) 0}
00086     , {(UBYTE *) "nwritestatistics",         ONOFFVALUE, 0, (LONG) 0}
00087     , {(UBYTE *) "nwritethreadstatistics",    ONOFFVALUE, 0, (LONG) 0}
00088     , {(UBYTE *) "oldfactarg",               ONOFFVALUE, 0, (LONG) NEWFACTARG}
00089     , {(UBYTE *) "oldqcd",                   ONOFFVALUE, 0, (LONG) 1}
00090     , {(UBYTE *) "oldorder",                 ONOFFVALUE, 0, (LONG) 0}
00091     , {(UBYTE *) "oldparallelstatistics",     ONOFFVALUE, 0, (LONG) 0}
00092     , {(UBYTE *) "parentheses",              NUMERICALVALUE, 0, (LONG) MAXPARLEVEL}
00093     , {(UBYTE *) "path",                     PATHVALUE, 0, (LONG) curdirp}
00094     , {(UBYTE *) "procedureextension",        STRINGVALUE, 0, (LONG) procedureextension}
00095     , {(UBYTE *) "processbucketsize",        NUMERICALVALUE, 0, (LONG) DEFAULTPROCESSBUCKETSIZE}
00096     , {(UBYTE *) "resettimeonclear",          ONOFFVALUE, 0, (LONG) 1}
00097     , {(UBYTE *) "scratchsize",               NUMERICALVALUE, 0, (LONG) SCRATCHSIZE}
00098     , {(UBYTE *) "shmwinsize",                NUMERICALVALUE, 0, (LONG) SHMWINSIZE}
00099     , {(UBYTE *) "sizestorecache",            NUMERICALVALUE, 0, (LONG) SIZESTORECACHE}
00100     , {(UBYTE *) "smallextension",           NUMERICALVALUE, 0, (LONG) SMALLOVERFLOW}
00101     , {(UBYTE *) "smallsize",                 NUMERICALVALUE, 0, (LONG) SMALLBUFFER}
00102     , {(UBYTE *) "sortiosize",                NUMERICALVALUE, 0, (LONG) SORTIOSIZE}
00103     , {(UBYTE *) "sorttype",                  STRINGVALUE, 0, (LONG) lowfirst}
00104     , {(UBYTE *) "spectatorsize",            NUMERICALVALUE, 0, (LONG) SPECTATORSIZE}
00105     , {(UBYTE *) "subfilepatches",           NUMERICALVALUE, 0, (LONG) SMAXFPATCHES}
00106     , {(UBYTE *) "sublargepatches",          NUMERICALVALUE, 0, (LONG) SMAXPATCHES}
00107     , {(UBYTE *) "sublargesize",             NUMERICALVALUE, 0, (LONG) SLARGEBUFFER}
00108     , {(UBYTE *) "subsmallextension",        NUMERICALVALUE, 0, (LONG) SSMALLOVERFLOW}
00109     , {(UBYTE *) "subsmallsize",             NUMERICALVALUE, 0, (LONG) SMALLBUFFER}
00110     , {(UBYTE *) "subsortiosize",            NUMERICALVALUE, 0, (LONG) SSORTIOSIZE}
00111     , {(UBYTE *) "subtermsinsmall",          NUMERICALVALUE, 0, (LONG) STERMSSMALL}
00112     , {(UBYTE *) "tempdir",                   STRINGVALUE, 0, (LONG) curdirp}
00113     , {(UBYTE *) "tempsortdir",              STRINGVALUE, 0, (LONG) cursortdirp}

```

```

00114     ,{(UBYTE *)"termsinsmall",          NUMERICALVALUE, 0, (LONG)TERMSSMALL}
00115     ,{(UBYTE *)"threadbucketsize",      NUMERICALVALUE, 0, (LONG)DEFAULTTHREADBUCKETSIZE}
00116     ,{(UBYTE *)"threadloadbalancing",    ONOFFVALUE, 0, (LONG)DEFAULTTHREADLOADBALANCING}
00117     ,{(UBYTE *)"threads",                NUMERICALVALUE, 0, (LONG)DEFAULTTHREADS}
00118     ,{(UBYTE *)"threadscratchoutsize",    NUMERICALVALUE, 0, (LONG)THREADSCRATCHOUTSIZE}
00119     ,{(UBYTE *)"threadscratchsize",      NUMERICALVALUE, 0, (LONG)THREADSCRATCHSIZE}
00120     ,{(UBYTE *)"threadsortfilesynch",    ONOFFVALUE, 0, (LONG)0}
00121     ,{(UBYTE *)"totalsize",              ONOFFVALUE, 0, (LONG)2}
00122     ,{(UBYTE *)"workspace",              NUMERICALVALUE, 0, (LONG)WORKBUFFER}
00123     ,{(UBYTE *)"wtimestats",             ONOFFVALUE, 0, (LONG)2}
00124 };
00125
00126 /*
00127     #] Includes :
00128     #[ Setups :
00129         #[ DoSetups :
00130 */
00131
00132 int DoSetups(void)
00133 {
00134     UBYTE *setbuffer, *s, *t, *u /*, c */;
00135     int errors = 0;
00136     setbuffer = LoadInputFile((UBYTE *)setupfilename,SETUPFILE);
00137     if ( setbuffer ) {
00138 /*
00139         The contents of the file are now in setbuffer.
00140         Each line is commentary or a single command.
00141         The buffer is terminated with a zero.
00142 */
00143         s = setbuffer;
00144         while ( *s ) {
00145             if ( *s == ' ' || *s == '\t' || *s == '*' || *s == '#' || *s == '\n' ) {
00146                 while ( *s && *s != '\n' ) s++;
00147             }
00148             else if ( tolower(*s) < 'a' || tolower(*s) > 'z' ) {
00149                 t = s;
00150                 while ( *s && *s != '\n' ) s++;
00151 /*
00152                 c = *s; *s = 0;
00153                 Error1("Setup file: Illegal statement: ",t);
00154                 errors++; *s = c;
00155 */
00156             }
00157             else {
00158                 t = s; /* name of the option */
00159                 while ( tolower(*s) >= 'a' && tolower(*s) <= 'z' ) s++;
00160                 *s++ = 0;
00161                 while ( *s == ' ' || *s == '\t' ) s++;
00162                 u = s; /* 'value' of the option */
00163                 while ( *s && *s != '\n' && *s != '\r' ) s++;
00164                 if ( *s ) *s++ = 0;
00165                 errors += ProcessOption(t,u,0);
00166             }
00167             while ( *s == '\n' || *s == '\r' ) s++;
00168         }
00169         M_free(setbuffer,"setup file buffer");
00170     }
00171     if ( errors ) return(1);
00172     else return(0);
00173 }
00174
00175 /*
00176     #] DoSetups :
00177     #[ ProcessOption :
00178 */
00179
00180 static char *proopl[3] = { "Setup file", "Setups in .frm file", "Setup in environment" };
00181
00182 int ProcessOption(UBYTE *s1, UBYTE *s2, int filetype)
00183 {
00184     SETUPPARAMETERS *sp;
00185     int n, giveback = 0, error = 0;
00186     UBYTE *s, *t, *s2ret;
00187     LONG x;
00188     sp = GetSetupPar(s1);
00189     if ( sp ) {
00190 /*
00191         We check now whether there are ` variables to be looked up in the
00192         environment. This is new (30-may-2008). This is only allowed in s2.
00193 */
00194 restart:;
00195         {
00196             UBYTE *s3,*s4,*s5,*s6, c, *start;
00197             int n1,n2,n3;
00198             s = s2;
00199             while ( *s ) {
00200                 if ( *s == '\\ ' ) s += 2;

```

```

00201         else if ( *s == ' ' ) {
00202             start = s; s++;
00203             while ( *s && *s != '\n' ) {
00204                 if ( *s == '\\ ' ) s++;
00205                 s++;
00206             }
00207             if ( *s == 0 ) {
00208                 MesPrint("%s: Illegal use of ` character for parameter %s"
00209                     ,proopl[filetype],s1);
00210                 return(1);
00211             }
00212             c = *s; *s = 0;
00213             s3 = (UBYTE *)getenv((char *) (start+1));
00214             if ( s3 == 0 ) {
00215                 MesPrint("%s: Cannot find environment variable %s for parameter %s"
00216                     ,proopl[filetype],start+1,s1);
00217                 return(1);
00218             }
00219             }
00220             *s = c; s++;
00221             n1 = start - s2; s4 = s3; n2 = 0;
00222             while ( *s4 ) {
00223                 if ( *s4 == '\\ ' ) { s4++; n2++; }
00224                 s4++; n2++;
00225             }
00226             s4 = s; n3 = 0;
00227             while ( *s4 ) {
00228                 if ( *s4 == '\\ ' ) { s4++; n3++; }
00229                 s4++; n3++;
00230             }
00231             s4 = (UBYTE *)Malloc1((n1+n2+n3+1)*sizeof(UBYTE),"environment in setup");
00232             s5 = s2; s6 = s4;
00233             while ( n1-- > 0 ) *s6++ = *s5++;
00234             s5 = s3;
00235             while ( n2-- > 0 ) *s6++ = *s5++;
00236             s5 = s;
00237             while ( n3-- > 0 ) *s6++ = *s5++;
00238             *s6 = 0;
00239             if ( giveback ) M_free(s2,"environment in setup");
00240             s2 = s4;
00241             giveback = 1;
00242             goto restart;
00243         }
00244         else s++;
00245     }
00246 }
00247 n = sp->type;
00248 s2ret = s2;
00249 switch ( n ) {
00250     case NUMERICALVALUE:
00251         ParseNumber(x,s2);
00252         if ( *s2 == 'K' ) { x = x * 1000; s2++; }
00253         else if ( *s2 == 'M' ) { x = x * 1000000; s2++; }
00254         else if ( *s2 == 'G' ) { x = x * 1000000000; s2++; }
00255         else if ( *s2 == 'T' ) { x = x * 1000000000000; s2++; }
00256         if ( *s2 && *s2 != ' ' && *s2 != '\\t' ) {
00257             MesPrint("%s: Numerical value expected for parameter %s"
00258                 ,proopl[filetype],s1);
00259             error = 1; break;
00260         }
00261         sp->value = x;
00262         sp->flags = USEDFLAG;
00263         break;
00264     case STRINGVALUE:
00265         if ( StrICmp(s1,(UBYTE *)"tempsortdir") == 0 ) AM.havesortdir = 1;
00266         s = s2; t = s2;
00267         while ( *s ) {
00268             if ( *s == ' ' || *s == '\\t' ) break;
00269             if ( *s == '\\ ' ) s++;
00270             *t++ = *s++;
00271         }
00272         *t = 0;
00273         if ( sp->flags == USEDFLAG && sp->value != 0 )
00274             M_free((void *) (sp->value),"Process option");
00275         sp->value = (LONG)strDupl(s2,"Process option");
00276         sp->flags = USEDFLAG;
00277         break;
00278     case PATHVALUE:
00279         if ( StrICmp(s1,(UBYTE *)"incdir") == 0 ) {
00280             AM.IncDir = 0;
00281         }
00282         else if ( StrICmp(s1,(UBYTE *)"path") == 0 ) {
00283             if ( AM.Path ) M_free(AM.Path,"path");
00284             AM.Path = 0;
00285         }
00286         else {
00287             MesPrint("Setups: %s not yet implemented",s1);

```

```

00288         error = 1;
00289         break;
00290     }
00291     if ( sp->flags == USEDFLAG && sp->value != 0 )
00292         M_free((void *) (sp->value), "Process option");
00293     sp->value = (LONG)strDup1(s2, "Process option");
00294     sp->flags = USEDFLAG;
00295     break;
00296 case ONOFFVALUE:
00297     if ( tolower(*s2) == 'o' && tolower(s2[1]) == 'n'
00298         && ( s2[2] == 0 || s2[2] == ' ' || s2[2] == '\t' ) )
00299         sp->value = 1;
00300     else if ( tolower(*s2) == 'o' && tolower(s2[1]) == 'f'
00301         && tolower(s2[2]) == 'f'
00302         && ( s2[3] == 0 || s2[3] == ' ' || s2[3] == '\t' ) )
00303         sp->value = 0;
00304     else {
00305         MesPrint("%s: Unrecognized option for parameter %s: %s"
00306             , proopl[filetype], s1, s2);
00307         error = 1; break;
00308     }
00309     sp->flags = USEDFLAG;
00310     break;
00311 case DEFINEVALUE:
00312 /*
00313     if ( sp->value ) M_free((UBYTE *) (sp->value), "Process option");
00314     sp->value = (LONG)strDup1(s2, "Process option");
00315 */
00316     if ( TheDefine(s2, 2) ) error = 1;
00317     break;
00318 default:
00319     Error1("Error in setupparameter table for:", s1);
00320     error = 1;
00321     break;
00322 }
00323 }
00324 else {
00325     MesPrint("%s: Keyword not recognized: %s", proopl[filetype], s1);
00326     error = 1;
00327 }
00328 if ( giveback ) M_free(s2ret, "environment in setup");
00329 return(error);
00330 }
00331
00332 /*
00333     #] ProcessOption :
00334     #[ GetSetupPar :
00335 */
00336
00337 SETUPPARAMETERS *GetSetupPar(UBYTE *s)
00338 {
00339     int hi, med, lo, i;
00340     lo = 0;
00341     hi = sizeof(setupparameters)/sizeof(SETUPPARAMETERS);
00342     do {
00343         med = ( hi + lo ) / 2;
00344         i = StrICmp(s, (UBYTE *) setupparameters[med].parameter);
00345         if ( i == 0 ) return(setupparameters+med);
00346         if ( i < 0 ) hi = med-1;
00347         else lo = med+1;
00348     } while ( hi >= lo );
00349     return(0);
00350 }
00351
00352 /*
00353     #] GetSetupPar :
00354     #[ RecalcSetups :
00355 */
00356
00357 int RecalcSetups(void)
00358 {
00359     SETUPPARAMETERS *sp, *spl;
00360
00361     spl = GetSetupPar((UBYTE *) "threads");
00362     if ( AM.totalnumberofthreads > 1 ) spl->value = AM.totalnumberofthreads - 1;
00363     else spl->value = 0;
00364 /*
00365     if ( spl->value > 0 ) AM.totalnumberofthreads = spl->value+1;
00366     if ( AM.totalnumberofthreads == 0 ) AM.totalnumberofthreads = 1;
00367 */
00368     sp = GetSetupPar((UBYTE *) "filepatches");
00369     if ( sp->value < AM.totalnumberofthreads-1 )
00370         sp->value = AM.totalnumberofthreads - 1;
00371
00372     sp = GetSetupPar((UBYTE *) "smallsize");
00373     spl = GetSetupPar((UBYTE *) "smallestextension");
00374     if ( 6*spl->value < 7*sp->value ) spl->value = (7*sp->value)/6;

```

```

00375     sp = GetSetupPar((UBYTE *)"termsinsmall");
00376     sp->value = ( sp->value + 15 ) & (-16L);
00377 #ifdef WITHPTHREADS
00378 {
00379     SETUPPARAMETERS *sp2;
00380     LONG totalsize, minimumsize;
00381     sp = GetSetupPar((UBYTE *)"largesize");
00382     totalsize = spl->value+sp->value;
00383     sp2 = GetSetupPar((UBYTE *)"maxtermsize");
00384     AM.MaxTer = sp2->value*sizeof(WORD);
00385     if ( AM.MaxTer < 200*(LONG)(sizeof(WORD)) ) AM.MaxTer = 200*(LONG)(sizeof(WORD));
00386     if ( AM.MaxTer > MAXPOSITIVE - 200*(LONG)(sizeof(WORD)) ) AM.MaxTer = MAXPOSITIVE -
200*(LONG)(sizeof(WORD));
00387     AM.MaxTer /= sizeof(WORD);
00388     AM.MaxTer *= sizeof(WORD);
00389     minimumsize = (AM.totalnumberofthreads-1)*(AM.MaxTer+
00390     NUMBEROFBLOCKSINSERT*MINIMUMNUMBEROFTERMS*AM.MaxTer);
00391     if ( totalsize < minimumsize ) {
00392         sp->value = minimumsize - spl->value;
00393     }
00394 }
00395 #endif
00396     return(0);
00397 }
00398
00399 /*
00400     #] RecalcSetups :
00401     #[ AllocSetups :
00402 */
00403
00404 int AllocSetups(void)
00405 {
00406     SETUPPARAMETERS *sp;
00407     LONG LargeSize, SmallSize, SmallEsize, TermsInSmall, IOSize;
00408     int MaxPatches, MaxFpatches, error = 0, i, size;
00409     UBYTE *s;
00410 #ifndef WITHPTHREADS
00411     int j;
00412 #endif
00413     sp = GetSetupPar((UBYTE *)"threads");
00414     if ( sp->value > 0 ) AM.totalnumberofthreads = sp->value+1;
00415
00416     AM.OutBuffer = (UBYTE *)Malloca1(AM.OutBufSize+1,"OutputBuffer");
00417     AP.PreAssignStack = (LONG *)Malloca1(AP.MaxPreAssignLevel*sizeof(LONG *),"PreAssignStack");
00418     for ( i = 0; i < AP.MaxPreAssignLevel; i++ ) AP.PreAssignStack[i] = 0;
00419     AC.iBuffer = (UBYTE *)Malloca1(AC.iBufferSize+1,"statement buffer");
00420     AC.iStop = AC.iBuffer + AC.iBufferSize-2;
00421     AP.preStart = (UBYTE *)Malloca1(AP.pSize,"instruction buffer");
00422     AP.preStop = AP.preStart + AP.pSize - 3;
00423     /* AP.PreIfStack is already allocated in StartPrepro(), but to be sure we
00424        "if" the freeing */
00425     if ( AP.PreIfStack ) M_free(AP.PreIfStack,"PreIfStack");
00426     AP.PreIfStack = (int *)Malloca1(AP.MaxPreIfLevel*sizeof(int),
00427        "Preprocessor if stack");
00428     AP.PreIfStack[0] = EXECUTINGIF;
00429     sp = GetSetupPar((UBYTE *)"insidefirst");
00430     AM.ginsidefirst = AC.minsidefirst = AC.insidefirst = sp->value;
00431 /*
00432     We need to consider eliminating this variable
00433 */
00434     sp = GetSetupPar((UBYTE *)"maxtermsize");
00435     AM.MaxTer = sp->value*sizeof(WORD);
00436     if ( AM.MaxTer < 200*(LONG)(sizeof(WORD)) ) AM.MaxTer = 200*(LONG)(sizeof(WORD));
00437     if ( AM.MaxTer > MAXPOSITIVE - 200*(LONG)(sizeof(WORD)) ) AM.MaxTer = MAXPOSITIVE -
200*(LONG)(sizeof(WORD));
00438     AM.MaxTer /= (LONG)sizeof(WORD);
00439     AM.MaxTer *= (LONG)sizeof(WORD);
00440 /*
00441     Allocate workspace.
00442 */
00443     sp = GetSetupPar((UBYTE *)"workspace");
00444     AM.WorkSize = sp->value;
00445 #ifdef WITHPTHREADS
00446 #else
00447     AT.WorkSpace = (WORD *)Malloca1(AM.WorkSize*sizeof(WORD), (char *) (sp->parameter));
00448     AT.WorkTop = AT.WorkSpace + AM.WorkSize;
00449     AT.WorkPointer = AT.WorkSpace;
00450 #endif
00451 /*
00452     Fixed indices
00453 */
00454     sp = GetSetupPar((UBYTE *)"constindex");
00455     if ( ( sp->value+100+5*WILDOFFSET ) > MAXPOSITIVE ) {
00456         MesPrint("Setting of %s in setupfile too large","constindex");
00457         AM.OffsetIndex = MAXPOSITIVE - 5*WILDOFFSET - 100;
00458         MesPrint("value corrected to maximum allowed: %d",AM.OffsetIndex);
00459     }

```



```

00460     else AM.OffsetIndex = sp->value + 1;
00461     AC.FixIndices = (WORD *)Malloc1((AM.OffsetIndex)*sizeof(WORD), (char *) (sp->parameter));
00462     AM.WilInd = AM.OffsetIndex + WILDOFFSET;
00463     AM.DumInd = AM.OffsetIndex + 2*WILDOFFSET;
00464     AM.IndDum = AM.DumInd + WILDOFFSET;
00465 #ifndef WITHPTHREADS
00466     AR.CurDum = AN.IndDum = AM.IndDum;
00467 #endif
00468     AM.mTraceDum = AM.IndDum + 2*WILDOFFSET;
00469
00470     sp = GetSetupPar((UBYTE *) "parentheses");
00471     AM.MaxParLevel = sp->value+1;
00472     AC.tokenarglevel = (WORD *)Malloc1((sp->value+1)*sizeof(WORD), (char *) (sp->parameter));
00473 /*
00474     Space during calculations
00475 */
00476     sp = GetSetupPar((UBYTE *) "maxnumbersize");
00477 /*
00478     size = ( sp->value + 11 ) & (-4);
00479     AM.MaxTal = size - 2;
00480     if ( AM.MaxTal > (AM.MaxTer/sizeof(WORD)-2)/2 )
00481         AM.MaxTal = (AM.MaxTer/sizeof(WORD)-2)/2;
00482     if ( AM.MaxTal < (AM.MaxTer/sizeof(WORD)-2)/4 )
00483         AM.MaxTal = (AM.MaxTer/sizeof(WORD)-2)/4;
00484 */
00485 /*
00486     There is too much confusion about MaxTal cq maxnumbersize.
00487     It seems better to fix it at its maximum value. This way we only worry
00488     about maxtermsize. This can be understood better by the 'innocent' user.
00489 */
00490     if ( sp->value == 0 ) {
00491         AM.MaxTal = (AM.MaxTer/sizeof(WORD)-2)/2;
00492     }
00493     else {
00494         size = ( sp->value + 11 ) & (-4);
00495         AM.MaxTal = size - 2;
00496         if ( (size_t)AM.MaxTal > (size_t)((AM.MaxTer/sizeof(WORD)-2)/2) )
00497             AM.MaxTal = (AM.MaxTer/sizeof(WORD)-2)/2;
00498     }
00499     AM.MaxTal &= -sizeof(WORD)*2;
00500
00501     sp->value = AM.MaxTal;
00502     AC.cmod = (UWORD *)Malloc1(AM.MaxTal*4*sizeof(UWORD), (char *) (sp->parameter));
00503     AM.gcmod = AC.cmod + AM.MaxTal;
00504     AC.powmod = AM.gcmod + AM.MaxTal;
00505     AM.gpowmod = AC.powmod + AM.MaxTal;
00506 /*
00507     The IO buffers for the input and output expressions.
00508     Fscr[2] will be assigned in a later stage for hiding expressions from
00509     the regular action. That will make the program faster.
00510 */
00511     sp = GetSetupPar((UBYTE *) "scratchsize");
00512     AM.ScratSize = sp->value/sizeof(WORD);
00513     if ( AM.ScratSize < 4*AM.MaxTer ) AM.ScratSize = 4*AM.MaxTer;
00514     AM.HideSize = AM.ScratSize;
00515     sp = GetSetupPar((UBYTE *) "hidesize");
00516     if ( sp->value > 0 ) {
00517         AM.HideSize = sp->value/sizeof(WORD);
00518         if ( AM.HideSize < 4*AM.MaxTer ) AM.HideSize = 4*AM.MaxTer;
00519     }
00520     sp = GetSetupPar((UBYTE *) "factorizationcache");
00521     AM.fbufferSize = sp->value;
00522 #ifdef WITHPTHREADS
00523     sp = GetSetupPar((UBYTE *) "threadscratchsize");
00524     AM.ThreadSkratSize = sp->value/sizeof(WORD);
00525     sp = GetSetupPar((UBYTE *) "threadscratchoutsize");
00526     AM.ThreadSkratOutSize = sp->value/sizeof(WORD);
00527 #endif
00528 #ifndef WITHPTHREADS
00529     for ( j = 0; j < 2; j++ ) {
00530         WORD *ScratchBuf;
00531         ScratchBuf = (WORD *)Malloc1(AM.ScratSize*sizeof(WORD), "scratchsize");
00532         AR.Fscr[j].POsize = AM.ScratSize * sizeof(WORD);
00533         AR.Fscr[j].POfull = AR.Fscr[j].POfill = AR.Fscr[j].PObuffer = ScratchBuf;
00534         AR.Fscr[j].POstop = AR.Fscr[j].PObuffer + AM.ScratSize;
00535         PUTZERO(AR.Fscr[j].POposition);
00536     }
00537     AR.Fscr[2].PObuffer = 0;
00538 #endif
00539     sp = GetSetupPar((UBYTE *) "threadbucketsize");
00540     AC.ThreadBucketSize = AM.gThreadBucketSize = AM.ggThreadBucketSize = sp->value;
00541     sp = GetSetupPar((UBYTE *) "threadloadbalancing");
00542     AC.ThreadBalancing = AM.gThreadBalancing = AM.ggThreadBalancing = sp->value;
00543     sp = GetSetupPar((UBYTE *) "threadsortfilesynch");
00544     AC.ThreadSortFileSynch = AM.gThreadSortFileSynch = AM.ggThreadSortFileSynch = sp->value;
00545 /*
00546     The size for shared memory window for onside MPI2 communications

```

```

00547 */
00548     sp = GetSetupPar((UBYTE *)"shmwinSize");
00549     AM.shmWinSize = sp->value/sizeof(WORD);
00550     if ( AM.shmWinSize < 4*AM.MaxTer ) AM.shmWinSize = 4*AM.MaxTer;
00551 /*
00552     The sort buffer
00553 */
00554     sp = GetSetupPar((UBYTE *)"smallSize");
00555     SmallSize = sp->value;
00556     sp = GetSetupPar((UBYTE *)"smallextension");
00557     SmallEsize = sp->value;
00558     sp = GetSetupPar((UBYTE *)"largesize");
00559     LargeSize = sp->value;
00560     sp = GetSetupPar((UBYTE *)"termsinsmall");
00561     TermsInSmall = sp->value;
00562     sp = GetSetupPar((UBYTE *)"largepatches");
00563     MaxPatches = sp->value;
00564     sp = GetSetupPar((UBYTE *)"filepatches");
00565     MaxFpatches = sp->value;
00566     sp = GetSetupPar((UBYTE *)"sortiosize");
00567     IOsize = sp->value;
00568     if ( IOsize < AM.MaxTer ) { IOsize = AM.MaxTer; sp->value = IOsize; }
00569 #ifndef WITHPTHREADS
00570 #ifdef WITHZLIB
00571     for ( j = 0; j < 2; j++ ) { AR.Fscr[j].ziosize = IOsize; }
00572 #endif
00573 #endif
00574     AM.S0 = 0;
00575     AM.S0 = AllocSort(LargeSize,SmallSize,SmallEsize,TermsInSmall
00576                     ,MaxPatches,MaxFpatches,IOsize,0);
00577     /* AM.S0->file.ziosize was already set to a (larger) value by AllocSort, here it is re-set. */
00578 #ifdef WITHZLIB
00579     AM.S0->file.ziosize = IOsize;
00580 #ifndef WITHPTHREADS
00581     AR.FoStage4[0].ziosize = IOsize;
00582     AR.FoStage4[1].ziosize = IOsize;
00583     AT.S0 = AM.S0;
00584 #endif
00585 #else
00586 #ifndef WITHPTHREADS
00587     AT.S0 = AM.S0;
00588 #endif
00589 #endif
00590 #ifndef WITHPTHREADS
00591     AR.FoStage4[0].POsize = ((IOsize+sizeof(WORD)-1)/sizeof(WORD))*sizeof(WORD);
00592     AR.FoStage4[1].POsize = ((IOsize+sizeof(WORD)-1)/sizeof(WORD))*sizeof(WORD);
00593 #endif
00594     sp = GetSetupPar((UBYTE *)"subsmallSize");
00595     AM.SSmallSize = sp->value;
00596     sp = GetSetupPar((UBYTE *)"subsmallextension");
00597     AM.SSmallEsize = sp->value;
00598     sp = GetSetupPar((UBYTE *)"sublargesize");
00599     AM.SLargeSize = sp->value;
00600     sp = GetSetupPar((UBYTE *)"subtermsinsmall");
00601     AM.STermsInSmall = sp->value;
00602     sp = GetSetupPar((UBYTE *)"sublargepatches");
00603     AM.SMaxPatches = sp->value;
00604     sp = GetSetupPar((UBYTE *)"subfilepatches");
00605     AM.SMaxFpatches = sp->value;
00606     sp = GetSetupPar((UBYTE *)"subsortiosize");
00607     AM.SIOsize = sp->value;
00608     /* As for IOsize, make sure SIOsize is at least as large as AM.MaxTer. */
00609     if ( AM.SIOsize < AM.MaxTer ) { AM.SIOsize = AM.MaxTer; sp->value = AM.SIOsize; }
00610     sp = GetSetupPar((UBYTE *)"spectatorsize");
00611     AM.SpectatorSize = sp->value;
00612 /*
00613     The next code is just for the moment (26-jan-1997) because we have
00614     the new parts combined with the old. Once the old parts are gone
00615     from the program, we can eliminate this code too.
00616 */
00617     sp = GetSetupPar((UBYTE *)"functionlevels");
00618     AM.maxFlevels = sp->value + 1;
00619 #ifdef WITHPTHREADS
00620 #else
00621     AT.Nest = (NESTING)Malloc1((LONG)sizeof(struct NeStInG)*AM.maxFlevels,"functionlevels");
00622     AT.NestStop = AT.Nest + AM.maxFlevels;
00623     AT.NestPoin = AT.Nest;
00624 #endif
00625
00626     sp = GetSetupPar((UBYTE *)"maxwildcards");
00627     AM.MaxWildcards = sp->value;
00628 #ifdef WITHPTHREADS
00629 #else
00630     AT.WildMask = (WORD *)Malloc1((LONG)AM.MaxWildcards*sizeof(WORD),"maxwildcards");
00631 #endif
00632
00633     sp = GetSetupPar((UBYTE *)"compresssize");

```

```

00634     if ( sp->value < 2*AM.MaxTer ) sp->value = 2*AM.MaxTer;
00635     AM.CompressSize = sp->value;
00636 #ifndef WITHPTHREADS
00637     AR.CompressBuffer = (WORD *)Malloc1( (AM.CompressSize+10)*sizeof(WORD), "compresssize");
00638     AR.CompressPointer = AR.CompressBuffer;
00639     AR.ComprTop = AR.CompressBuffer + AM.CompressSize;
00640 #endif
00641     sp = GetSetupPar( (UBYTE *) "bracketindexsize");
00642     if ( sp->value < 20*AM.MaxTer ) sp->value = 20*AM.MaxTer;
00643     AM.MaxBracketBufferSize = sp->value/sizeof(WORD);
00644
00645     sp = GetSetupPar( (UBYTE *) "dotchar");
00646     AO.FortDotChar = ((UBYTE *) (sp->value))[0];
00647     sp = GetSetupPar( (UBYTE *) "commentchar");
00648     AP.cComChar = AP.ComChar = ((UBYTE *) (sp->value))[0];
00649     sp = GetSetupPar( (UBYTE *) "procedureextension");
00650 /*
00651     Check validity first.
00652 */
00653     s = (UBYTE *) (sp->value);
00654     if ( FG.cTable[*s] != 0 ) {
00655         MesPrint(" Illegal string for procedure extension %s", (UBYTE *) sp->value);
00656         error = -2;
00657     }
00658     else {
00659         s++;
00660         while ( *s ) {
00661             if ( *s == ' ' || *s == '\t' || *s == '\n' ) {
00662                 MesPrint(" Illegal string for procedure extension %s", (UBYTE *) sp->value);
00663                 error = -2;
00664                 break;
00665             }
00666             s++;
00667         }
00668     }
00669     AP.cprocedureExtension = strDup1( (UBYTE *) (sp->value), "procedureExtension");
00670     AP.procedureExtension = strDup1( AP.cprocedureExtension, "procedureExtension");
00671
00672     sp = GetSetupPar( (UBYTE *) "totalsize");
00673     if ( sp->value != 2 ) AM.PrintTotalSize = sp->value;
00674
00675     sp = GetSetupPar( (UBYTE *) "continuationlines");
00676     AM.FortranCont = sp->value;
00677     sp = GetSetupPar( (UBYTE *) "oldorder");
00678     AM.OldOrderFlag = sp->value;
00679     sp = GetSetupPar( (UBYTE *) "resettimeonclear");
00680     AM.resetTimeOnClear = sp->value;
00681     sp = GetSetupPar( (UBYTE *) "nospacesinnumbers");
00682     AO.NoSpacesInNumbers = AM.gNoSpacesInNumbers = AM.ggNoSpacesInNumbers = sp->value;
00683     sp = GetSetupPar( (UBYTE *) "indentspace");
00684     AO.IndentSpace = AM.gIndentSpace = AM.ggIndentSpace = sp->value;
00685     sp = GetSetupPar( (UBYTE *) "jumpratio");
00686     AM.jumpratio = sp->value;
00687     sp = GetSetupPar( (UBYTE *) "nwritestatistics");
00688     AC.StatsFlag = AM.gStatsFlag = AM.ggStatsFlag = 1-sp->value;
00689     sp = GetSetupPar( (UBYTE *) "nwritefinalstatistics");
00690     AC.FinalStats = AM.gFinalStats = AM.ggFinalStats = 1-sp->value;
00691     sp = GetSetupPar( (UBYTE *) "nwritethreadstatistics");
00692     AC.ThreadStats = AM.gThreadStats = AM.ggThreadStats = 1-sp->value;
00693     sp = GetSetupPar( (UBYTE *) "nwriteprocessstatistics");
00694     AC.ProcessStats = AM.gProcessStats = AM.ggProcessStats = 1-sp->value;
00695     sp = GetSetupPar( (UBYTE *) "oldparallelstatistics");
00696     AC.OldParallelStats = AM.gOldParallelStats = AM.ggOldParallelStats = sp->value;
00697     sp = GetSetupPar( (UBYTE *) "oldfactarg");
00698     AC.OldFactArgFlag = AM.gOldFactArgFlag = AM.ggOldFactArgFlag = sp->value;
00699     sp = GetSetupPar( (UBYTE *) "oldgcd");
00700     AC.OldGCDflag = AM.gOldGCDflag = AM.ggOldGCDflag = sp->value;
00701     sp = GetSetupPar( (UBYTE *) "wtimestats");
00702     if ( sp->value == 2 ) sp->value = AM.ggWTimeStatsFlag;
00703     AC.WTimeStatsFlag = AM.gWTimeStatsFlag = AM.ggWTimeStatsFlag = sp->value;
00704 #ifdef WITHFLOAT
00705     sp = GetSetupPar( (UBYTE *) "maxweight");
00706     AC.MaxWeight = AM.gMaxWeight = AM.ggMaxWeight = sp->value;
00707     sp = GetSetupPar( (UBYTE *) "defaultprecision");
00708     AC.DefaultPrecision = AM.gDefaultPrecision = AM.ggDefaultPrecision = sp->value;
00709 #endif
00710     sp = GetSetupPar( (UBYTE *) "sorttype");
00711     if ( StrICmp( (UBYTE *) "lowfirst", (UBYTE *) sp->value) == 0 ) {
00712         AC.lSortType = SORTLOWFIRST;
00713     }
00714     else if ( StrICmp( (UBYTE *) "highfirst", (UBYTE *) sp->value) == 0 ) {
00715         AC.lSortType = SORTHIGHFIRST;
00716     }
00717     else {
00718         MesPrint(" Illegal SortType specification: %s", (UBYTE *) sp->value);
00719         error = -2;
00720     }

```

```

00721
00722     sp = GetSetupPar((UBYTE *)"processbucketsize");
00723     AM.hProcessBucketSize = AM.gProcessBucketSize =
00724     AC.ProcessBucketSize = AC.mProcessBucketSize = sp->value;
00725 /*
00726     The store caches (code installed 15-aug-2006 JV)
00727 */
00728     sp = GetSetupPar((UBYTE *)"numstorecaches");
00729     AM.NumStoreCaches = sp->value;
00730     sp = GetSetupPar((UBYTE *)"sizestorecache");
00731     AM.SizeStoreCache = sp->value;
00732     /* Make sure this is a multiple of sizeof(WORD). */
00733     AM.SizeStoreCache = ((AM.SizeStoreCache+sizeof(WORD)-1)/sizeof(WORD))*sizeof(WORD);
00734 #ifndef WITHPTHREADS
00735 /*
00736     Install the store caches (15-aug-2006 JV)
00737     Note that in the case of PTHREADS this is done in InitializeOneThread
00738 */
00739     AT.StoreCache = AT.StoreCacheAlloc = 0;
00740     if ( AM.NumStoreCaches > 0 ) {
00741         STORECACHE sa, sb;
00742         size = sizeof(struct StOrEcAcHe)+AM.SizeStoreCache;
00743         size = ((size-1)/sizeof(size_t)+1)*sizeof(size_t);
00744         AT.StoreCacheAlloc = (STORECACHE)Malloc1(size*AM.NumStoreCaches,"StoreCaches");
00745         AT.StoreCache = AT.StoreCacheAlloc;
00746         sa = AT.StoreCache;
00747         for ( j = 0; j < AM.NumStoreCaches; j++ ) {
00748             sb = (STORECACHE)(void *)((UBYTE *)sa+size);
00749             if ( j == AM.NumStoreCaches-1 ) {
00750                 sa->next = 0;
00751             }
00752             else {
00753                 sa->next = sb;
00754             }
00755             SETBASEPOSITION(sa->position,-1);
00756             SETBASEPOSITION(sa->toppos,-1);
00757             sa = sb;
00758         }
00759     }
00760 #endif
00761 /*
00762     And now some order sensitive things
00763 */
00764     if ( AM.Path == 0 ) {
00765         sp = GetSetupPar((UBYTE *)"path");
00766         AM.Path = strDup1((UBYTE *) (sp->value),"path");
00767     }
00768     if ( AM.IncDir == 0 ) {
00769         sp = GetSetupPar((UBYTE *)"incdir");
00770         AM.IncDir = strDup1((UBYTE *) (sp->value),"incdir");
00771     }
00772 /*
00773     if ( AM.TempDir == 0 ) {
00774         sp = GetSetupPar((UBYTE *)"tempdir");
00775         AM.TempDir = strDup1((UBYTE *) (sp->value),"tempdir");
00776     }
00777 */
00778 /*
00779     As part of the setup, we possibly need to work around a flint multi-threading
00780     bug which was fixed in version 3.1.0. This function should be called by the
00781     master only, before the worker threads start processing terms.
00782 */
00783 #ifdef WITHFLINT
00784     #if __FLINT_RELEASE < 30100
00785         flint_startup_init();
00786     #endif
00787 #endif
00788     return(error);
00789 }
00790
00791 /*
00792     #] AllocSetups :
00793     #[ WriteSetup :
00794
00795     The routine writes the values of the setup parameters.
00796     We should do this better. (JV, 21-may-2008)
00797     The way it should be done is:
00798     a: write the raw values.
00799     b: give readjusted values.
00800     c: give derived values.
00801     Because this is a difficult subject, it would be nice to have a LaTeX
00802     document that explains this all exactly. There should then be a
00803     mechanism to poke the values of the setup into the LaTeX document.
00804     probably the easiest way is to make a file with lots of \def definitions

```

```

00808     and have that included into the LaTeX file.
00809 */
00810
00811 void WriteSetup(void)
00812 {
00813     int n = sizeof(setupparameters)/sizeof(SETUPPARAMETERS);
00814     SETUPPARAMETERS *sp;
00815     MesPrint(" The setup parameters are:");
00816     for ( sp = setupparameters; n > 0; n--, sp++ ) {
00817         switch(sp->type){
00818             case NUMERICALVALUE:
00819                 MesPrint("    %s: %1", sp->parameter, sp->value);
00820                 break;
00821             case PATHVALUE:
00822                 if ( StrICmp(sp->parameter, (UBYTE *) "path") == 0 && AM.Path ) {
00823                     MesPrint("    %s: '%s'", sp->parameter, (UBYTE *) (AM.Path));
00824                     break;
00825                 }
00826                 if ( StrICmp(sp->parameter, (UBYTE *) "incdir") == 0 && AM.IncDir ) {
00827                     MesPrint("    %s: '%s'", sp->parameter, (UBYTE *) (AM.IncDir));
00828                     break;
00829                 }
00830                 /* fall through */
00831             case STRINGVALUE:
00832                 if ( StrICmp(sp->parameter, (UBYTE *) "tempdir") == 0 && AM.TempDir ) {
00833                     MesPrint("    %s: '%s'", sp->parameter, (UBYTE *) (AM.TempDir));
00834                 }
00835                 else if ( StrICmp(sp->parameter, (UBYTE *) "tempSortDir") == 0 && AM.TempSortDir ) {
00836                     MesPrint("    %s: '%s'", sp->parameter, (UBYTE *) (AM.TempSortDir));
00837                 }
00838                 else {
00839                     MesPrint("    %s: '%s'", sp->parameter, (UBYTE *) (sp->value));
00840                 }
00841                 break;
00842             case ONOFFVALUE:
00843                 if ( sp->value == 0 )
00844                     MesPrint("    %s: OFF", sp->parameter);
00845                 else if ( sp->value == 1 )
00846                     MesPrint("    %s: ON", sp->parameter);
00847                 break;
00848             case DEFINEVALUE:
00849                 /*
00850                  MesPrint("    %s: '%s'", sp->parameter, (UBYTE *) (sp->value));
00851                  */
00852                 break;
00853         }
00854     }
00855     AC.SetupFlag = 0;
00856 }
00857
00858 /*
00859     #] WriteSetup :
00860     #[ AllocSort :
00861
00862     Routine allocates a complete struct for sorting.
00863     To be used for the main allocation of the sort buffers, and
00864     in a later stage for the function and subroutine sort buffers.
00865     The level arg denotes a main buffer allocation (0) or a sub-buffer
00866     allocation (1), used only for printing warning messages for buffer
00867     size adjustments in DEBUGGING mode.
00868 */
00869 SORTING *AllocSort(LONG inLargeSize, LONG inSmallSize, LONG inSmallEsize, LONG inTermsInSmall,
00870     int inMaxPatches, int inMaxFpatches, LONG inIOsize, int level)
00871 {
00872     DUMMYUSE(level); /* This is only used in DEBUGGING mode */
00873     LONG LargeSize = inLargeSize;
00874     LONG SmallSize = inSmallSize;
00875     LONG SmallEsize = inSmallEsize;
00876     LONG TermsInSmall = inTermsInSmall;
00877     int MaxPatches = inMaxPatches;
00878     int MaxFpatches = inMaxFpatches;
00879     LONG IOsize = inIOsize;
00880
00881     LONG longer, terms2insmall, sortsize, longerp;
00882     LONG IObuffersize = IOsize;
00883     LONG IOtry;
00884     SORTING *sort;
00885     int fname2Size = 0, j = 0;
00886     char *s;
00887     if ( AM.S0 != 0 ) {
00888         s = FG.fname2; fname2Size = 0;
00889         while ( *s ) { s++; fname2Size++; }
00890         fname2Size += 16;
00891     }
00892     if ( MaxFpatches < 4 ) MaxFpatches = 4;
00893     longer = MaxPatches > MaxFpatches ? MaxPatches : MaxFpatches;
00894     longerp = longer;

```

```

00895     while ( (1 << j) < longerp ) j++;
00896     longerp = (1 << j) + 1;
00897     longerp += sizeof(WORD*) - (longerp*sizeof(WORD *));
00898     longer++;
00899     longer += sizeof(WORD*) - (longerp*sizeof(WORD *));
00900     if ( SmallSize < 16*AM.MaxTer ) SmallSize = 16*AM.MaxTer+16;
00901     TermsInSmall = (TermsInSmall+15) & (-16L);
00902     terms2insmall = 2*TermsInSmall; /* Used to be just + 100 rather than *2 */
00903     if ( SmallSize < (SmallSize+3)/2 ) SmallSize = (SmallSize+3)/2;
00904     if ( LargeSize > 0 && LargeSize < 2*SmallSize ) LargeSize = 2*SmallSize;
00905     SmallEsize = (SmallEsize+15) & (-16L);
00906     if ( LargeSize < 0 ) LargeSize = 0;
00907     sortsize = sizeof(SORTING);
00908     sortsize = (sortsize+15)&(-16L);
00909     IObuffersize = (IObuffersize+sizeof(WORD)-1)/sizeof(WORD);
00910 /*
00911     The next statement fixes a bug. In the rare case that we have a
00912     problem here, we expand the size of the large buffer or the
00913     small extension
00914 */
00915     if ( (ULONG) ( LargeSize+SmallEsize ) < MaxFpatches*((IObuffersize
00916     +COMPINC)*sizeof(WORD)+2*AM.MaxTer) ) {
00917         if ( LargeSize == 0 )
00918             SmallSize = MaxFpatches*((IObuffersize+COMPINC)*sizeof(WORD)+2*AM.MaxTer);
00919         else
00920             LargeSize = MaxFpatches*((IObuffersize+COMPINC)*sizeof(WORD)+2*AM.MaxTer)
00921                 - SmallEsize;
00922     }
00923
00924     IOtry = ((LargeSize+SmallEsize)/MaxFpatches-2*AM.MaxTer)/sizeof(WORD)-COMPINC;
00925
00926     /* Here both IObuffersize and IOtry are in units of sizeof(WORD). */
00927     if ( IObuffersize < IOtry ) {
00928         IObuffersize = IOtry;
00929     }
00930
00931 #if DEBUGGING
00932     char *prefix;
00933     if ( level == 0 ) { prefix = ""; }
00934     else { prefix = "Sub"; }
00935     if ( LargeSize != inLargeSize ) { MesPrint("Warning: %sLargeSize adjusted: %l -> %l", prefix,
inLargeSize, LargeSize); }
00936     if ( SmallSize != inSmallSize ) { MesPrint("Warning: %sSmallSize adjusted: %l -> %l", prefix,
inSmallSize, SmallSize); }
00937     if ( SmallEsize != inSmallEsize ) { MesPrint("Warning: %sSmallEsize adjusted: %l -> %l", prefix,
inSmallEsize, SmallEsize); }
00938     if ( TermsInSmall != inTermsInSmall ) { MesPrint("Warning: %sTermsInSmall adjusted: %l -> %l",
prefix, inTermsInSmall, TermsInSmall); }
00939     if ( MaxPatches != inMaxPatches ) { MesPrint("Warning: MaxPatches adjusted: %d -> %d",
inMaxPatches, MaxPatches); }
00940     if ( MaxFpatches != inMaxFpatches ) { MesPrint("Warning: MaxFPatches adjusted: %d -> %d",
inMaxFpatches, MaxFpatches); }
00941     /* This one is always changed if the LargeSize has not been... */
00942     /* if ( IObuffersize != inIOsize/(LONG)sizeof(WORD) ) { MesPrint("Warning: IOsize adjusted: %l ->
%l", inIOsize/sizeof(WORD), IObuffersize); } */
00943 #endif
00944
00945     /* Allocate separate buffers for most struct members, for better testing and debugging with
valgrind. */
00946     sort = Malloc1(sizeof(*sort), "AllocSort: sorting struct");
00947
00948     sort->LargeSize = LargeSize/sizeof(WORD);
00949     sort->SmallSize = SmallSize/sizeof(WORD);
00950     sort->SmallEsize = SmallEsize/sizeof(WORD);
00951     sort->MaxPatches = MaxPatches;
00952     sort->MaxFpatches = MaxFpatches;
00953     sort->TermsInSmall = TermsInSmall;
00954     sort->Terms2InSmall = terms2insmall;
00955
00956     sort->sPointer = Malloc1(sizeof(*(sort->sPointer ))*terms2insmall, "AllocSort: sPointer");
00957     sort->Patches = Malloc1(sizeof(*(sort->Patches ))*longer, "AllocSort: Patches");
00958     sort->pStop = Malloc1(sizeof(*(sort->pStop ))*longer, "AllocSort: pStop");
00959     sort->poina = Malloc1(sizeof(*(sort->poina ))*longerp, "AllocSort: poina");
00960     sort->poin2a = Malloc1(sizeof(*(sort->poin2a ))*longerp, "AllocSort: poin2a");
00961
00962     sort->fPatches = Malloc1(sizeof(*(sort->fPatches ))*longer, "AllocSort: fPatches");
00963     sort->fPatchesStop = Malloc1(sizeof(*(sort->fPatchesStop))*longer, "AllocSort: fPatchesStop");
00964     sort->inPatches = Malloc1(sizeof(*(sort->inPatches ))*longer, "AllocSort: inPatches");
00965     sort->tree = Malloc1(sizeof(*(sort->tree ))*longerp, "AllocSort: tree");
00966     sort->used = Malloc1(sizeof(*(sort->used ))*longerp, "AllocSort: used");
00967
00968 #ifdef WITHZLIB
00969     sort->fpcompressed = Malloc1(sizeof(*(sort->fpcompressed ))*(longerp+2), "AllocSort:
fpcompressed");
00970     sort->fpincompressed = Malloc1(sizeof(*(sort->fpincompressed))*(longerp+2), "AllocSort:
fpincompressed");
00971     sort->zsparray = 0;

```

```

00972 #endif
00973
00974     sort->ktoi          = Malloc1(sizeof(*(sort->ktoi))*(longerp+2), "AllocSort: ktoi");
00975
00976     // The combined Large buffer and Small buffer (+ extension) are used.
00977     // They must be allocated together.
00978     sort->lBuffer        = Malloc1(sizeof(*(sort->lBuffer))*(sort->LargeSize+sort->SmallEsize),
"AllocSort: lBuffer+sBuffer");
00979     sort->lTop = sort->lBuffer+sort->LargeSize;
00980
00981     sort->sBuffer = sort->lTop;
00982     if ( sort->LargeSize == 0 ) { sort->lBuffer = 0; sort->lTop = 0; }
00983     sort->sTop = sort->sBuffer + sort->SmallSize;
00984     sort->sTop2 = sort->sBuffer + sort->SmallEsize;
00985     sort->sHalf = sort->sBuffer + (LONG)((sort->SmallSize+sort->SmallEsize)>>1);
00986
00987     sort->file.PObuffer = Malloc1(IObuffersize*sizeof(*(sort->file.PObuffer))+fname2Size+16,
"AllocSort: PObuffer");
00988     sort->file.POstop = sort->file.PObuffer+IObuffersize;
00989     sort->file.POsize = IObuffersize * sizeof(WORD);
00990     sort->file.POfill = sort->file.POfull = sort->file.PObuffer;
00991     sort->file.active = 0;
00992     sort->file.handle = -1;
00993     PUTZERO(sort->file.POposition);
00994 #ifdef WITHPTHREADS
00995     sort->file.pthreadlock = dummylock;
00996 #endif
00997 #ifdef WITHZLIB
00998     sort->file.ziosize = IObuffersize*sizeof(WORD);
00999     sort->file.ziobuffer = 0;
01000 #endif
01001     if ( AM.S0 != 0 ) {
01002         sort->file.name = (char *) (sort->file.PObuffer + IObuffersize);
01003         AllocSortFileName(sort);
01004     }
01005     else sort->file.name = 0;
01006     sort->cBuffer = 0;
01007     sort->cBufferSize = 0;
01008     sort->f = 0;
01009     sort->PolyWise = 0;
01010
01011     return(sort);
01012 }
01013
01014 /*
01015     #] AllocSort :
01016     #[ AllocSortFileName :
01017 */
01018
01019 void AllocSortFileName(SORTING *sort)
01020 {
01021     GETIDENTITY
01022     char *s, *t;
01023     /*
01024         This is not the allocation before the tempfiles are determined.
01025         Hence we can use the name in FG.fname2 and modify the tail
01026     */
01027     s = FG.fname2; t = sort->file.name;
01028     while ( *s ) *t++ = *s++;
01029 #ifdef WITHPTHREADS
01030     t[-2] = 'F';
01031     snprintf(t-1,FG.fname2size-(t-1-sort->file.name),"%d.%d",identity,AN.filename);
01032 #else
01033     t[-2] = 'f';
01034     snprintf(t-1,FG.fname2size-(t-1-sort->file.name),"%d",AN.filename);
01035 #endif
01036     AN.filename++;
01037 }
01038
01039 /*
01040     #] AllocSortFileName :
01041     #[ AllocFileHandle :
01042 */
01043
01044 FILEHANDLE *AllocFileHandle(WORD par,char *name)
01045 {
01046     GETIDENTITY
01047     LONG allocation, Ssize;
01048     FILEHANDLE *fh;
01049     int i = 0;
01050     char *s, *t;
01051
01052     s = FG.fname2; i = 0;
01053     while ( *s ) { s++; i++; }
01054     if ( par == 0 ) { i += 16; Ssize = AM.SIOsize; }
01055     else { s = name; while ( *s ) { i++; s++; } i+= 2; Ssize = AM.SpectatorSize; }
01056

```

```

01057     allocation = sizeof(FILEHANDLE) + (Ssize+1)*sizeof(WORD) + i*sizeof(char)
01058                 +FG.fname2size+20;
01059     fh = (FILEHANDLE *)Malloc1(allocation,"FileHandle");
01060
01061     fh->PObuffer = (WORD *) (fh+1);
01062     fh->POstop = fh->PObuffer+Ssize;
01063     fh->POsize = Ssize * sizeof(WORD);
01064     fh->active = 0;
01065     fh->handle = -1;
01066     PUTZERO(fh->POposition);
01067 #ifdef WITHPTHREADS
01068     fh->pthreadlock = dummylock;
01069 #endif
01070     if ( par == 0 ) { /* sort file */
01071         if ( AM.S0 != 0 ) {
01072             fh->name = (char *) (fh->POstop + 1);
01073             s = FG.fname2; t = fh->name;
01074             while ( *s ) *t++ = *s++;
01075 #ifdef WITHPTHREADS
01076             t[-2] = 'F';
01077             snprintf(t-1,FG.fname2size-(t-1-fh->name), "%d-%d", identity, AN.filenum);
01078 #else
01079             t[-2] = 'f';
01080             snprintf(t-1,FG.fname2size-(t-1-fh->name), "%d", AN.filenum);
01081 #endif
01082             AN.filenum++;
01083         }
01084         else fh->name = 0;
01085     }
01086     else { /* Spectator file */
01087         fh->name = (char *) (fh->POstop + 1);
01088         s = FG.fname; t = fh->name;
01089         for ( i = 0; i < FG.fnamebase; i++ ) *t++ = *s++;
01090         s = name;
01091         while ( *s ) *t++ = *s++;
01092         *t = 0;
01093     }
01094     fh->POfill = fh->POfull = fh->PObuffer;
01095     return(fh);
01096 }
01097
01098 /*
01099     #] AllocFileHandle :
01100     #[ DeAllocFileHandle :
01101
01102     Made to repair deallocation of AN.filenum. 21-sep-2000
01103 */
01104
01105 void DeAllocFileHandle(FILEHANDLE *fh)
01106 {
01107     GETIDENTITY
01108     if ( fh->handle >= 0 ) {
01109         CloseFile(fh->handle);
01110         fh->handle = -1;
01111         remove(fh->name);
01112     }
01113     AN.filenum--; /* free namespace. was forgotten in first reading */
01114     M_free(fh,"Temporary FileHandle");
01115 }
01116
01117 /*
01118     #] DeAllocFileHandle :
01119     #[ MakeSetupAllocs :
01120 */
01121
01122 int MakeSetupAllocs(void)
01123 {
01124     if ( RecalcSetups() || AllocSetups() ) return(1);
01125     else return(0);
01126 }
01127
01128 /*
01129     #] MakeSetupAllocs :
01130     #[ TryFileSetups :
01131
01132     Routine looks in the input file for a start of the type
01133     [#-]
01134     #: setupparameter value
01135     It keeps looking until the first line that does not start with
01136     #-, #+ or #:
01137     Then it rewinds the input.
01138 */
01139
01140 #define SETBUFSIZE 257
01141
01142 int TryFileSetups(void)
01143 {

```



```

01144     LONG oldstreamposition;
01145     int oldstream;
01146     int error = 0, eqnum;
01147     int oldNoShowInput = AC.NoShowInput;
01148     UBYTE buff[SETBUFSIZE+1], *s, *t, *u, *settop, c;
01149     LONG linenum, prevline;
01150
01151     if ( AC.CurrentStream == 0 ) return(error);
01152     oldstream = AC.CurrentStream - AC.Streams;
01153     oldstreamposition = GetStreamPosition(AC.CurrentStream);
01154     linenum = AC.CurrentStream->linenum;
01155     prevline = AC.CurrentStream->prevline;
01156     eqnum = AC.CurrentStream->eqnum;
01157     AC.NoShowInput = 1;
01158     settop = buff + SETBUFSIZE;
01159     if ( AC.CurrentStream->type == INPUTSTREAM && oldstreamposition == 0 )
01160         AC.CurrentStream->fileposition = 0;
01161     for(;;) {
01162         c = GetInput();
01163         if ( c == '*' || c == '\n' ) {
01164             while ( c != '\n' && c != ENDOFINPUT ) c = GetInput();
01165             if ( c == ENDOFINPUT ) goto eoi;
01166             continue;
01167         }
01168         if ( c == ENDOFINPUT ) goto eoi;
01169         if ( c != '#' ) break;
01170         c = GetInput();
01171         if ( c == ENDOFINPUT ) goto eoi;
01172         if ( c != '-' && c != '+' && c != ':' ) break;
01173         if ( c != ':' ) {
01174             while ( c != '\n' && c != ENDOFINPUT ) c = GetInput();
01175             continue;
01176         }
01177         s = buff;
01178         while ( ( c = GetInput() ) == ' ' || c == '\t' || c == '\r' ) {}
01179         if ( c == ENDOFINPUT ) break;
01180         if ( c == LINEFEED ) continue;
01181         if ( c == 0 || c == ENDOFINPUT ) break;
01182         while ( c != LINEFEED ) {
01183             *s++ = c;
01184             c = GetInput();
01185             if ( c != LINEFEED && c != '\r' ) continue;
01186             if ( s >= settop ) {
01187                 while ( c != '\n' && c != ENDOFINPUT ) c = GetInput();
01188                 MesPrint("Setups in .frm file: Line too long. setup ignored");
01189                 error++; goto nextline;
01190             }
01191             *s++ = '\n';
01192             t = s = buff; /* name of the option */
01193             while ( tolower(*s) >= 'a' && tolower(*s) <= 'z' ) s++;
01194             if ( *s != '\n' ) {
01195                 *s++ = 0;
01196                 while ( *s == ' ' || *s == '\t' ) s++;
01197                 u = s; /* 'value' of the option */
01198                 while ( *s && *s != '\n' && *s != '\r' ) s++;
01199                 if ( *s ) *s++ = 0;
01200             }
01201             else {
01202                 /* The value is empty. */
01203                 u = s;
01204                 *s++ = 0;
01205             }
01206             error += ProcessOption(t,u,1);
01207         nextline:;
01208     }
01209     AC.NoShowInput = oldNoShowInput;
01210     AC.CurrentStream = AC.Streams + oldstream;
01211     PositionStream(AC.CurrentStream,oldstreamposition);
01212     AC.CurrentStream->linenum = linenum;
01213     AC.CurrentStream->prevline = prevline;
01214     AC.CurrentStream->eqnum = eqnum;
01215     ClearPushback();
01216     if ( AC.CurrentStream->type == INPUTSTREAM ) AC.CurrentStream->fileposition = -1;
01217     return(error);
01218 eoi:
01219     MesPrint("Input file without a program.");
01220     return(-1);
01221 }
01222
01223 /*
01224     #] TryFileSetups :
01225     #[ TryEnvironment :
01226 */
01227
01228 int TryEnvironment(void)
01229 {

```

```

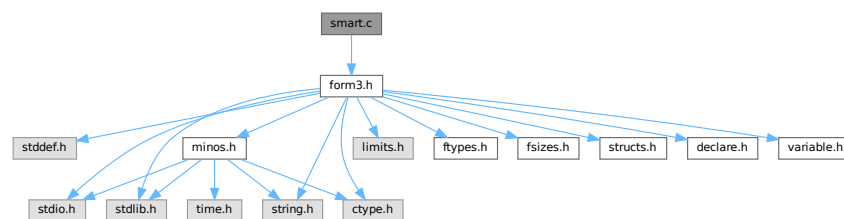
01230     char *s, *t, *u, varname[100];
01231     int i,imax = sizeof(setupparameters)/sizeof(SETUPPARAMETERS);
01232     int error = 0;
01233     varname[0] = 'F'; varname[1] = 'O'; varname[2] = 'R'; varname[3] = 'M';
01234     varname[4] = '_'; varname[5] = 0;
01235     for ( i = 0; i < imax; i++ ) {
01236         t = s = (char *) (setupparameters[i].parameter);
01237         u = varname+5;
01238         while ( *s ) { *u++ = (char) (toupper((unsigned char)*s)); s++; }
01239         *u = 0;
01240         s = (char *) (getenv(varname));
01241         if ( s ) {
01242             error += ProcessOption((UBYTE *)t, (UBYTE *)s,2);
01243         }
01244     }
01245     return(error);
01246 }
01247
01248 /*
01249     #] TryEnvironment :
01250     #] Setups :
01251 */

```

4.126 smart.c File Reference

#include "form3.h"

Include dependency graph for smart.c:



Functions

- int [StudyPattern](#) (WORD *lhs)
- int [MatchesPossible](#) (WORD *pattern, WORD *term)

4.126.1 Detailed Description

The functions for smart pattern searches in combinations of functions. When many wildcards are involved and the functions are (anti)symmetric an exhaustive search for all possibilities may take very much time (like factorial in the number of wildcards) while a human can often see immediately that there cannot be a match. The routines here try to make FORM a bit smarter in this respect.

This is just the beginning. It still needs lots of work!

Definition in file [smart.c](#).

4.126.2 Function Documentation

4.126.2.1 StudyPattern()

```
int StudyPattern (
    WORD * lhs )
```

Definition at line 62 of file [smart.c](#).

4.126.2.2 MatchIsPossible()

```
int MatchIsPossible (
    WORD * pattern,
    WORD * term )
```

Definition at line 299 of file [smart.c](#).

4.127 smart.c

[Go to the documentation of this file.](#)

```
00001
00012 /* #[] License : */
00013 /*
00014 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00015 * When using this file you are requested to refer to the publication
00016 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00017 * This is considered a matter of courtesy as the development was paid
00018 * for by FOM the Dutch physics granting agency and we would like to
00019 * be able to track its scientific use to convince FOM of its value
00020 * for the community.
00021 *
00022 * This file is part of FORM.
00023 *
00024 * FORM is free software: you can redistribute it and/or modify it under the
00025 * terms of the GNU General Public License as published by the Free Software
00026 * Foundation, either version 3 of the License, or (at your option) any later
00027 * version.
00028 *
00029 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00030 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00031 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00032 * details.
00033 *
00034 * You should have received a copy of the GNU General Public License along
00035 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00036 */
00037 /* #[] License : */
00038 /*
00039 #[] Includes : function.c
00040 */
00041
00042 #include "form3.h"
00043
00044 /*
00045 #[] Includes :
00046 #[] StudyPattern :
00047
00048 Argument is a complete lhs of an id-statement
00049 Its last word is a zero (new convention(18-may-1997)) to indicate
00050 that no extra information is following. We can add there information
00051 about the pattern that will help during the actual pattern matching.
00052 Currently the WorkPointer points to just after this lhs.
00053
00054 Task of this routine: To study the functions, their symmetry properties
00055 and their wildcards to determine in which order the functions can
00056 be matched best. If the order should be different we can change it here.
00057
00058 Problem encountered 22-dec-2008 (JV): we don't take noncommuting
00059 functions into account.
```

```

00060 */
00061
00062 int StudyPattern(WORD *lhs)
00063 {
00064     GETIDENTITY
00065     WORD *fullproto, *pat, *p, *p1, *p2, *pstop, *info, f, nn;
00066     int numfun = 0, numsym = 0, allwilds = 0, i, j, k, nc;
00067     FUN_INFO *finf, *fmin, *f1, *f2, funscratch;
00068
00069     fullproto = lhs + IDHEAD;
00070     /* if ( *lhs == TYPEIF ) fullproto--; */
00071     pat = fullproto + fullproto[1];
00072     info = pat + *pat;
00073
00074     p = pat + 1;
00075     while ( p < info ) {
00076         if ( *p >= FUNCTION ) {
00077             numfun++;
00078             nn = *p - FUNCTION;
00079             if ( nn >= WILDOFFSET ) nn -= WILDOFFSET;
00080         /*
00081             We check here for cases that are not allowed like ?a inside
00082             symmetric functions or tensors.
00083         */
00084         if ( ( functions[nn].symmetric == SYMMETRIC ) ||
00085             ( functions[nn].symmetric == ANTISYMMETRIC ) ) {
00086             p2 = p+p[1]; p1 = p+FUNHEAD;
00087             if ( functions[nn].spec > 0 ) {
00088                 while ( p1 < p2 ) {
00089                     if ( *p1 == FUNNYWILD ) {
00090                         MesPrint("&Argument field wildcards are not allowed inside (anti)symmetric
functions or tensors");
00091                         return(1);
00092                     }
00093                     p1++;
00094                 }
00095             }
00096             else {
00097                 while ( p1 < p2 ) {
00098                     if ( *p1 == -ARGWILD ) {
00099                         MesPrint("&Argument field wildcards are not allowed inside (anti)symmetric
functions or tensors");
00100                         return(1);
00101                     }
00102                     NEXTARG(p1);
00103                 }
00104             }
00105         }
00106         p += p[1];
00107     }
00108     if ( numfun == 0 ) return(0);
00109     if ( ( lhs[2] & SUBMASK ) == SUBALL ) {
00110         p = pat + 1;
00111         while ( p < info ) {
00112             if ( *p == SYMBOL || *p == VECTOR || *p == DOTPRODUCT || *p == INDEX ) {
00113                 MesPrint("&id,all can have only functions and/or tensors in the lhs.");
00114                 return(1);
00115             }
00116             p += p[1];
00117         }
00118     }
00119 }
00120 /*
00121     We need now some room for the information about the functions
00122 */
00123 if ( numfun > AN.numfuninfo ) {
00124     if ( AN.FunInfo ) M_free(AN.FunInfo,"funinfo");
00125     AN.numfuninfo = numfun + 10;
00126     AN.FunInfo = (FUN_INFO *)Malloc1(AN.numfuninfo*sizeof(FUN_INFO),"funinfo");
00127 }
00128 /*
00129     Now collect the information. First the locations.
00130 */
00131 p = pat + 1; i = 0;
00132 while ( p < info ) {
00133     if ( *p >= FUNCTION ) AN.FunInfo[i++].location = p;
00134     p += p[1];
00135 }
00136 for ( i = 0, finf = AN.FunInfo; i < numfun; i++, finf++ ) {
00137     p = finf->location;
00138     pstop = p + p[1];
00139     f = *p;
00140     if ( f > FUNCTION+WILDOFFSET ) f -= WILDOFFSET;
00141     finf->numargs = finf->numfunnies = finf->numwildcards = 0;
00142     finf->symmet = functions[f-FUNCTION].symmetric;
00143     finf->tensor = functions[f-FUNCTION].spec;
00144     finf->commute = functions[f-FUNCTION].commute;

```

```

00145     if ( finf->tensor >= TENSORFUNCTION ) {
00146         p += FUNHEAD;
00147         while ( p < pstop ) {
00148             if ( *p == FUNNYWILD ) {
00149                 finf->numfunnies++; p+= 2; continue;
00150             }
00151             else if ( *p < 0 ) {
00152                 if ( *p >= AM.OffsetVector + WILDOFFSET && *p < MINSPEC ) {
00153                     finf->numwildcards++;
00154                 }
00155             }
00156             else {
00157                 if ( *p >= AM.OffsetIndex + WILDOFFSET &&
00158                     *p <= AM.OffsetIndex + 2*WILDOFFSET ) finf->numwildcards++;
00159             }
00160             finf->numargs++;
00161             p++;
00162         }
00163     }
00164     else {
00165         p += FUNHEAD;
00166         while ( p < pstop ) {
00167             if ( *p > 0 ) { finf->numargs++; p += *p; continue; }
00168             if ( *p <= -FUNCTION ) {
00169                 if ( *p <= -FUNCTION - WILDOFFSET ) finf->numwildcards++;
00170                 p++;
00171             }
00172             else if ( *p == -SYMBOL ) {
00173                 if ( p[1] >= 2*MAXPOWER ) finf->numwildcards++;
00174                 p += 2;
00175             }
00176             else if ( *p == -INDEX ) {
00177                 if ( p[1] >= AM.OffsetIndex + WILDOFFSET &&
00178                     p[1] <= AM.OffsetIndex + 2*WILDOFFSET ) finf->numwildcards++;
00179                 p += 2;
00180             }
00181             else if ( *p == -VECTOR || *p == -MINVECTOR ) {
00182                 if ( p[1] >= AM.OffsetVector + WILDOFFSET && p[1] < MINSPEC ) {
00183                     finf->numwildcards++;
00184                 }
00185                 p += 2;
00186             }
00187             else if ( *p == -ARGWILD ) {
00188                 finf->numfunnies++;
00189                 p += 2;
00190             }
00191             else { p += 2; }
00192             finf->numargs++;
00193         }
00194     }
00195     if ( finf->symmet ) {
00196         numsym++;
00197         allwilds += finf->numwildcards + finf->numfunnies;
00198     }
00199 }
00200 if ( numsym == 0 ) return(0);
00201 if ( allwilds == 0 ) return(0);
00202 /*
00203 We have the information in the array AN.FunInfo.
00204 We sort things and then write the sorted pattern.
00205 Of course we may not play with the order of the noncommuting functions.
00206 Of course we have to become even smarter in the future and look during
00207 the sorting which wildcards are assigned when.
00208 But for now this should do.
00209 */
00210 for ( nc = numfun-1; nc >= 0; nc-- ) { if ( AN.FunInfo[nc].commute ) break; }
00211
00212 finf = AN.FunInfo;
00213 for ( i = nc+2; i < numfun; i++ ) {
00214     fmin = finf; finf++;
00215     if ( ( finf->symmet < fmin->symmet ) || (
00216         ( finf->symmet == fmin->symmet ) &&
00217         ( ( finf->numwildcards+finf->numfunnies < fmin->numwildcards+fmin->numfunnies )
00218         || ( ( finf->numwildcards+finf->numfunnies == fmin->numwildcards+fmin->numfunnies )
00219         && ( finf->numwildcards < fmin->numfunnies ) ) ) ) ) {
00220         funscratch = AN.FunInfo[i];
00221         AN.FunInfo[i] = AN.FunInfo[i-1];
00222         AN.FunInfo[i-1] = funscratch;
00223         for ( j = i-1; j > nc && j > 0; j-- ) {
00224             f1 = AN.FunInfo[j];
00225             f2 = f1-1;
00226             if ( ( f1->symmet < f2->symmet ) || (
00227                 ( f1->symmet == f2->symmet ) &&
00228                 ( ( f1->numwildcards+f1->numfunnies < f2->numwildcards+f2->numfunnies )
00229                 || ( ( f1->numwildcards+f1->numfunnies == f2->numwildcards+f2->numfunnies )
00230                 && ( f1->numwildcards < f2->numfunnies ) ) ) ) ) {
00231                 funscratch = AN.FunInfo[j];

```

```

00232             AN.FunInfo[j] = AN.FunInfo[j-1];
00233             AN.FunInfo[j-1] = funscratch;
00234         }
00235         else break;
00236     }
00237 }
00238 }
00239 /*
00240 Now we rewrite the pattern. First into the space after it and then we
00241 copy it back. Be careful with the non-commutative functions. There the
00242 worst one should decide.
00243 */
00244 p = pat + 1;
00245 p2 = info;
00246 for ( i = 0; i < numfun; i++ ) {
00247     if ( i == nc ) {
00248         for ( k = 0; k <= nc; k++ ) {
00249             if ( AN.FunInfo[k].commute ) {
00250                 p1 = AN.FunInfo[k].location; j = p1[1];
00251                 NCOPY(p2,p1,j)
00252             }
00253         }
00254     }
00255     else if ( AN.FunInfo[i].commute == 0 ) {
00256         p1 = AN.FunInfo[i].location; j = p1[1];
00257         NCOPY(p2,p1,j)
00258     }
00259 }
00260 p = pat + 1; p1 = info;
00261 while ( p1 < p2 ) *p++ = *p1++;
00262 /*
00263 And now we place the relevant information in the info part
00264 */
00265 p2 = info+1;
00266 for ( i = 0; i < numfun; i++ ) {
00267     if ( i == nc ) {
00268         for ( k = 0; k <= nc; k++ ) {
00269             if ( AN.FunInfo[k].commute ) {
00270                 finf = AN.FunInfo + k;
00271                 *p2++ = finf->numargs;
00272                 *p2++ = finf->numwildcards;
00273                 *p2++ = finf->numfunnies;
00274                 *p2++ = finf->symmet;
00275             }
00276         }
00277     }
00278     else if ( AN.FunInfo[i].commute == 0 ) {
00279         finf = AN.FunInfo + i;
00280         *p2++ = finf->numargs;
00281         *p2++ = finf->numwildcards;
00282         *p2++ = finf->numfunnies;
00283         *p2++ = finf->symmet;
00284     }
00285 }
00286 *info = p2-info;
00287 lhs[1] = p2-lhs;
00288 return(0);
00289 }
00290
00291 /*
00292 #] StudyPattern :
00293 #[ MatchIsPossible :
00294
00295 We come here when there are functions and there is nontrivial
00296 symmetry related wildcarding.
00297 */
00298
00299 int MatchIsPossible(WORD *pattern, WORD *term)
00300 {
00301     GETIDENTITY
00302     WORD *info = pattern + *pattern;
00303     WORD *t, *tstop, *tt, *inf, *p;
00304     int numfun = 0, inpat, i, j, numargs;
00305     FUN_INFO *finf;
00306 /*
00307 We need a list of functions and their number of arguments
00308 */
00309 GETSTOP(term,tstop);
00310 t = term + 1;
00311 while ( t < tstop ) {
00312     if ( *t >= FUNCTION ) numfun++;
00313     t += t[1];
00314 }
00315 if ( numfun == 0 ) goto NotPossible;
00316 if ( *info == SETSET ) info += info[1];
00317 inpat = (*info-1)/4;
00318 if ( inpat > numfun ) goto NotPossible;

```

```

00319 /*
00320 We need now some room for the information about the functions
00321 */
00322 if ( numfun > AN.numfuninfo ) {
00323     if ( AN.FunInfo ) M_free(AN.FunInfo,"funinfo");
00324     AN.numfuninfo = numfun + 10;
00325     AN.FunInfo = (FUN_INFO *)Malloc1(AN.numfuninfo*sizeof(FUN_INFO),"funinfo");
00326 }
00327 t = term + 1; finf = AN.FunInfo;
00328 while ( t < tstop ) {
00329     if ( *t >= FUNCTION ) {
00330         finf->location = t;
00331         if ( functions[*t-FUNCTION].spec >= TENSORFUNCTION ) {
00332             numargs = t[1]-FUNHEAD;
00333             t += t[1];
00334         }
00335         else {
00336             numargs = 0;
00337             tt = t + t[1];
00338             t += FUNHEAD;
00339             while ( t < tt ) {
00340                 numargs++;
00341                 NEXTARG(t)
00342             }
00343         }
00344         finf->numargs = numargs;
00345         finf++;
00346     }
00347     else t += t[1];
00348 }
00349 /*
00350 Now we first find a partner for each function in the pattern
00351 with a fixed number of arguments
00352 */
00353 for ( i = 0, inf = info+1, p = pattern+1; i < inpat; i++, inf += 4, p+=p[1] ) {
00354     if ( inf[2] ) continue;
00355     if ( *p >= (FUNCTION+WILDOFFSET) ) continue;
00356     for ( j = 0, finf = AN.FunInfo; j < numfun; j++, finf++ ) {
00357         if ( *p == *(finf->location) && *inf == finf->numargs ) {
00358             finf->numargs = -finf->numargs-1;
00359             break;
00360         }
00361     }
00362     if ( j >= numfun ) goto NotPossible;
00363 }
00364 for ( i = 0, inf = info+1, p = pattern+1; i < inpat; i++, inf += 4, p+=p[1] ) {
00365     if ( inf[2] ) continue;
00366     if ( *p < (FUNCTION+WILDOFFSET) ) continue;
00367     for ( j = 0, finf = AN.FunInfo; j < numfun; j++, finf++ ) {
00368         if ( *inf == finf->numargs ) {
00369             finf->numargs = -finf->numargs-1;
00370             break;
00371         }
00372     }
00373     if ( j >= numfun ) goto NotPossible;
00374 }
00375 for ( i = 0, inf = info+1, p = pattern+1; i < inpat; i++, inf += 4, p+=p[1] ) {
00376     if ( inf[2] == 0 ) continue;
00377     if ( *p >= (FUNCTION+WILDOFFSET) ) continue;
00378     for ( j = 0, finf = AN.FunInfo; j < numfun; j++, finf++ ) {
00379         if ( *p == *(finf->location) && (*inf-inf[2]) <= finf->numargs ) {
00380             finf->numargs = -finf->numargs-1;
00381             break;
00382         }
00383     }
00384     if ( j >= numfun ) goto NotPossible;
00385 }
00386 for ( i = 0, inf = info+1, p = pattern+1; i < inpat; i++, inf += 4, p+=p[1] ) {
00387     if ( inf[2] == 0 ) continue;
00388     if ( *p < (FUNCTION+WILDOFFSET) ) continue;
00389     for ( j = 0, finf = AN.FunInfo; j < numfun; j++, finf++ ) {
00390         if ( (*inf-inf[2]) <= finf->numargs ) {
00391             finf->numargs = -finf->numargs-1;
00392             break;
00393         }
00394     }
00395     if ( j >= numfun ) goto NotPossible;
00396 }
00397 /*
00398 Thus far we have determined that for each function in the pattern
00399 there is a potential partner with enough arguments.
00400 For now the rest is up to the real pattern matcher.
00401
00402 To undo the disabling of the number of arguments we need this code
00403 for ( i = 0, finf = AN.FunInfo; i < numfun; i++, finf++ ) {
00404     if ( finf->numargs < 0 ) finf->numargs = -finf->numargs-1;
00405 }

```

```

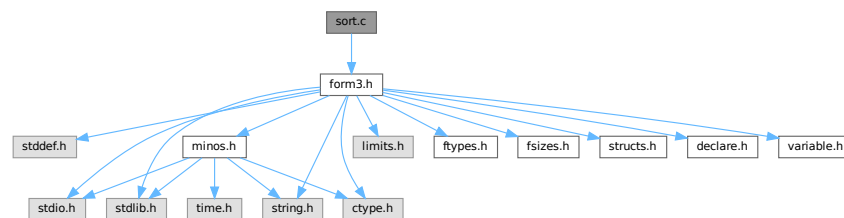
00406 */
00407     return(1);
00408 NotPossible:
00409 /*
00410     PrintTerm(pattern,"p");
00411     PrintTerm(term,"t");
00412 */
00413     return(0);
00414 }
00415
00416 /*
00417     #] MatchIsPossible :
00418 */

```

4.128 sort.c File Reference

```
#include "form3.h"
```

Include dependency graph for sort.c:



Macros

- `#define NEWSPLITMERGE`
- `#define NEWSPLITMERGETIMSORT`
- `#define HUMANSTRLEN 12`
- `#define HUMANSUFFLEN 4`
- `#define INSLENGTH(x) w[1] = FUNHEAD+ARGHEAD+x; w[FUNHEAD] = ARGHEAD+x;`

Functions

- void `HumanString` (char *string, float input, const char suffix[HUMANSUFFLEN][4])
- void `WriteStats` (POSITION *plspace, WORD par, WORD checkLogType)
- WORD `NewSort` (PHEAD0)
- LONG `EndSort` (PHEAD WORD *buffer, int par)
- LONG `PutIn` (FILEHANDLE *file, POSITION *position, WORD *buffer, WORD **take, int npat)
- WORD `Sflush` (FILEHANDLE *fi)
- WORD `PutOut` (PHEAD WORD *term, POSITION *position, FILEHANDLE *fi, WORD ncomp)
- WORD `FlushOut` (POSITION *position, FILEHANDLE *fi, int compr)
- WORD `AddCoef` (PHEAD WORD **ps1, WORD **ps2)
- WORD `AddPoly` (PHEAD WORD **ps1, WORD **ps2)
- void `AddArgs` (PHEAD WORD *s1, WORD *s2, WORD *m)
- WORD `Compare1` (PHEAD WORD *term1, WORD *term2, WORD level)
- WORD `CompareSymbols` (PHEAD WORD *term1, WORD *term2, WORD par)
- WORD `CompareHSymbols` (PHEAD WORD *term1, WORD *term2, WORD par)
- LONG `ComPress` (WORD **ss, LONG *n)

- LONG [SplitMerge](#) (PHEAD WORD **Pointer, LONG number)
- void [GarbHand](#) (void)
- WORD [MergePatches](#) (WORD par)
- WORD [StoreTerm](#) (PHEAD WORD *term)
- void [StageSort](#) (FILEHANDLE *fout)
- WORD [SortWild](#) (WORD *w, WORD nw)
- void [CleanUpSort](#) (int num)
- void [LowerSortLevel](#) (void)
- WORD * [PolyRatFunSpecial](#) (PHEAD WORD *t1, WORD *t2)
- void [SimpleSplitMergeRec](#) (WORD *array, WORD num, WORD *auxarray)
- void [SimpleSplitMerge](#) (WORD *array, WORD num)
- WORD [BinarySearch](#) (WORD *array, WORD num, WORD x)

Variables

- LONG [numcompares](#)
- char * [totterms](#) [] = { " ", ">>", "-->" }
- const char [humanTermsSuffix](#) [HUMANSUFFLEN][4] = {"K ", "M ", "B ", "T "}
- const char [humanBytesSuffix](#) [HUMANSUFFLEN][4] = {"KiB", "MiB", "GiB", "TiB"}

4.128.1 Detailed Description

Contains the sort routines. We distinguish levels of sorting. The ground level is the sorting of terms in an expression. When a term has functions, the arguments can contain terms that need sorting, which this then done by raising the level. This can happen recursively. NewSort and EndSort automatically raise and lower the level. Because the ground level does some special things, sometimes we have to raise immediately to the second level skipping the ground level.

Special routines for the parallel sorting are in the file [threads.c](#). Also the sorting of terms in polynomials is special but most of that is controlled by changing the address of the compare routine. Other routines relevant for adding rational polynomials are in the file [polynito.c](#).

Definition in file [sort.c](#).

4.128.2 Macro Definition Documentation

4.128.2.1 NEWSPLITMERGE

```
#define NEWSPLITMERGE
```

Definition at line 53 of file [sort.c](#).

4.128.2.2 NEWSPLITMERGETIMSORT

```
#define NEWSPLITMERGETIMSORT
```

Definition at line 55 of file [sort.c](#).

4.128.2.3 HUMANSTRLEN

```
#define HUMANSTRLEN 12
```

Definition at line 85 of file [sort.c](#).

4.128.2.4 HUMANSUFFLEN

```
#define HUMANSUFFLEN 4
```

Definition at line 86 of file [sort.c](#).

4.128.2.5 INSLENGTH

```
#define INSLENGTH(  
    x ) w[1] = FUNHEAD+ARGHEAD+x; w[FUNHEAD] = ARGHEAD+x;
```

Definition at line 2334 of file [sort.c](#).

4.128.3 Function Documentation

4.128.3.1 HumanString()

```
void HumanString (  
    char * string,  
    float input,  
    const char suffix[HUMANSUFFLEN][4] )
```

Definition at line 89 of file [sort.c](#).

4.128.3.2 WriteStats()

```
void WriteStats (  
    POSITION * plspace,  
    WORD par,  
    WORD checkLogType )
```

Writes the statistics.

Parameters

<i>plspace</i>	The size in bytes currently occupied
<i>par</i>	par = 0 = STATSSPLITMERGE after a splitmerge. par = 1 = STATSMERGETOFILE after merge to sortfile. par = 2 = STATSPOSTSORT after the sort
<i>checkLogType</i>	== CHECKLOGTYPE: The output should not go to the log file if AM.LogType is 1.

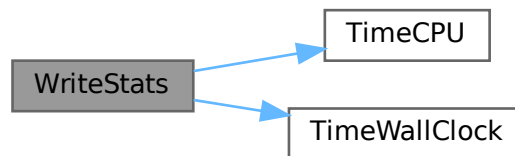
current expression is to be found in AR.CurExpr. terms are in S->TermsLeft. S->GenTerms.

Definition at line 121 of file [sort.c](#).

References [TimeCPU\(\)](#), and [TimeWallClock\(\)](#).

Referenced by [EndSort\(\)](#), [PF_Processor\(\)](#), and [StoreTerm\(\)](#).

Here is the call graph for this function:



4.128.3.3 NewSort()

```
WORD NewSort (
    PHEAD0 )
```

Starts a new sort. At the lowest level this is a 'main sort' with the struct according to the parameters in S0. At higher levels this is a sort for functions, subroutines or dollars. We prepare the arrays and structs.

Returns

Regular convention (OK -> 0)

Definition at line 643 of file [sort.c](#).

Referenced by [generate_expression\(\)](#), [get_expression\(\)](#), [InFunction\(\)](#), [MakeDollarInteger\(\)](#), [MakeDollarMod\(\)](#), [PF_CollectModifiedDollars\(\)](#), [PF_Processor\(\)](#), [poly_factorize_expression\(\)](#), [poly_sort\(\)](#), [poly_unfactorize_expression\(\)](#), [PolyFunMul\(\)](#), [Processor\(\)](#), [TakeArgContent\(\)](#), and [TestMatch\(\)](#).

4.128.3.4 EndSort()

```
LONG EndSort (
    PHEAD WORD * buffer,
    int par )
```

Finishes a sort. At `AR.sLevel == 0` the output is to the regular output stream. When `AR.sLevel > 0`, the parameter `par` determines the actual output. The `AR.sLevel` will be popped. All ongoing stages are finished and if the sortfile is open it is closed. The statistics are printed when `AR.sLevel == 0` `par == 0` [Output](#) to the buffer. `par == 1` Sort for function arguments. The output will be copied into the buffer. It is assumed that this is in the `WorkSpace`. `par == 2` Sort for \$-variable. We return the address of the buffer that contains the output in `buffer` (treated like `WORD **`). We first catch the output in a file (unless we can intercept things after the small buffer has been sorted) Then we read from the file into a buffer. Only when `par == 0` data compression can be attempted at `AT.SS==AT.S0`.

Parameters

<i>buffer</i>	buffer for output when needed
<i>par</i>	See above

Returns

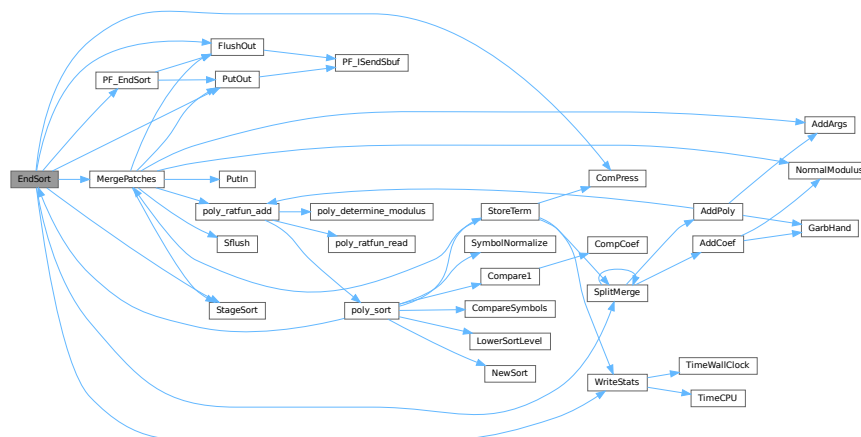
If negative: error. If positive: number of words in output.

Definition at line 733 of file [sort.c](#).

References [ComPress\(\)](#), [FlushOut\(\)](#), [MergePatches\(\)](#), [PF_EndSort\(\)](#), [PutOut\(\)](#), [SplitMerge\(\)](#), [StageSort\(\)](#), and [WriteStats\(\)](#).

Referenced by [generate_expression\(\)](#), [get_expression\(\)](#), [InFunction\(\)](#), [MakeDollarInteger\(\)](#), [MakeDollarMod\(\)](#), [PF_CollectModifiedDollars\(\)](#), [PF_Processor\(\)](#), [poly_factorize_expression\(\)](#), [poly_sort\(\)](#), [poly_unfactorize_expression\(\)](#), [PolyFunMul\(\)](#), [Processor\(\)](#), [TakeArgContent\(\)](#), and [TestMatch\(\)](#).

Here is the call graph for this function:



4.128.3.5 PutIn()

```

LONG PutIn (
    FILEHANDLE * file,
    POSITION * position,
    WORD * buffer,
    WORD ** take,
    int npat )
  
```

Reads a new patch from position in file handle. It is put at buffer, anything after take is moved forward. This would be part of a term that hasn't been used yet. Because of this there should be some space before the start of the buffer

Parameters

<i>file</i>	The file system from which to read
<i>position</i>	The position from which to read
<i>buffer</i>	The buffer into which to read
<i>take</i>	The unused tail should be moved before the buffer
<i>npat</i>	The number of the patch. Is needed if the information was compressed with gzip, because each patch has its own independent gzip encoding.

Definition at line 1320 of file [sort.c](#).

References [FiLe::handle](#).

Referenced by [MergePatches\(\)](#).

4.128.3.6 Sflush()

```
WORD Sflush (
    FILEHANDLE * fi )
```

Puts the contents of a buffer to output Only to be used when there is a single patch in the large buffer.

Parameters

<i>fi</i>	The filesystem (or its cache) to which the patch should be written
-----------	--

Definition at line 1380 of file [sort.c](#).

References [FiLe::handle](#).

Referenced by [MergePatches\(\)](#).

4.128.3.7 PutOut()

```
WORD PutOut (
    PHEAD WORD * term,
    POSITION * position,
    FILEHANDLE * fi,
    WORD ncomp )
```

Routine writes one term to file handle at position. It returns the new value of the position.

NOTE: For 'final output' we may have to index the brackets. See the struct BRACKETINDEX. We should maintain:
 1: a list with brackets array with the brackets
 2: a list of objects of type BRACKETINDEX. It contains array with either pointers or offsets to the list of brackets. starting positions in the file. The index may be tied to a maximum size. In that case we may have to prune the list occasionally.

Parameters

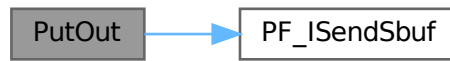
<i>term</i>	The term to be written
<i>position</i>	The position in the file. Afterwards it is updated
<i>fi</i>	The file (or its cache) to which should be written
<i>ncomp</i>	Information about what type of compression should be used

Definition at line 1466 of file [sort.c](#).

References [FiLe::handle](#), and [PF_ISendSbuf\(\)](#).

Referenced by [EndSort\(\)](#), [generate_expression\(\)](#), [MergePatches\(\)](#), [PF_EndSort\(\)](#), [PF_Processor\(\)](#), and [Processor\(\)](#).

Here is the call graph for this function:



4.128.3.8 FlushOut()

```
WORD FlushOut (
    POSITION * position,
    FILEHANDLE * fi,
    int compr )
```

Completes output to an output file and writes the trailing zero.

Parameters

<i>position</i>	The position in the file after writing
<i>fi</i>	The file (or its cache)
<i>compr</i>	Indicates whether there should be compression with gzip.

Returns

Regular conventions (OK -> 0).

Definition at line 1828 of file [sort.c](#).

References [FiLe::handle](#), [BrAcKeTiNfO::indexbuffer](#), and [PF_ISendSbuf\(\)](#).

Referenced by [EndSort\(\)](#), [MergePatches\(\)](#), [PF_EndSort\(\)](#), and [Processor\(\)](#).

Here is the call graph for this function:



4.128.3.9 AddCoef()

```
WORD AddCoef (
    PHEAD WORD ** ps1,
    WORD ** ps2 )
```

Adds the coefficients of the terms *ps1 and *ps2. The problem comes when there is not enough space for a new longer coefficient. First a local solution is tried. If this is not successful we need to move terms around. The possibility of a garbage collection should not be ignored, as avoiding this costs very much extra space which is nearly wasted otherwise.

If the return value is zero the terms cancelled.

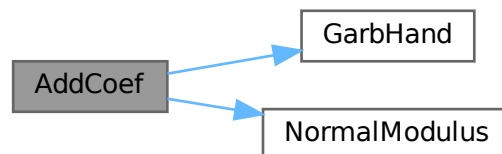
The resulting term is left in *ps1.

Definition at line 2042 of file [sort.c](#).

References [GarbHand\(\)](#), and [NormalModulus\(\)](#).

Referenced by [SplitMerge\(\)](#).

Here is the call graph for this function:

**4.128.3.10 AddPoly()**

```
WORD AddPoly (
    PHEAD WORD ** ps1,
    WORD ** ps2 )
```

Routine should be called when `S->PolyWise != 0`. It points then to the position of `AR.PolyFun` in both terms.

We add the contents of the arguments of the two polynomials. Special attention has to be given to special arguments. We have to reserve a space equal to the size of one term + the size of the argument of the other. The addition has to be done in this routine because not all objects are reentrant.

Newer addition (12-nov-2007). The `PolyFun` can have two arguments. In that case `S->PolyFlag` is 2 and we have to call the routine for adding rational polynomials. We have to be rather careful what happens with: The location of the output The order of the terms in the arguments At first we allow only univariate polynomials in the `PolyFun`. This restriction will be lifted a.s.a.p.

Parameters

<i>ps1</i>	A pointer to the position of the first term
<i>ps2</i>	A pointer to the position of the second term

Returns

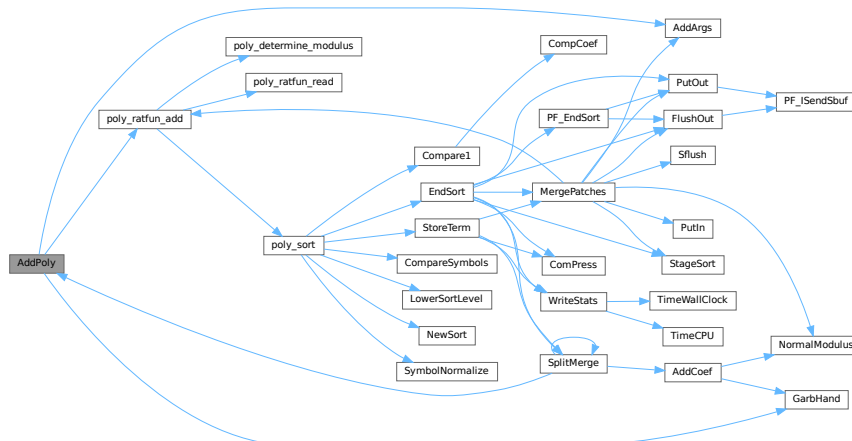
If zero the terms cancel. Otherwise the new term is in *ps1.

Definition at line 2171 of file [sort.c](#).

References [AddArgs\(\)](#), [GarbHand\(\)](#), and [poly_ratfun_add\(\)](#).

Referenced by [SplitMerge\(\)](#).

Here is the call graph for this function:



4.128.3.11 AddArgs()

```

void AddArgs (
    PHEAD WORD * s1,
    WORD * s2,
    WORD * m )

```

Adds the arguments of two occurrences of the PolyFun.

Parameters

<i>s1</i>	Pointer to the first occurrence.
<i>s2</i>	Pointer to the second occurrence.
<i>m</i>	Pointer to where the answer should be.

Definition at line 2343 of file [sort.c](#).

Referenced by [AddPoly\(\)](#), and [MergePatches\(\)](#).

4.128.3.12 Compare1()

```
WORD Compare1 (
    PHEAD WORD * term1,
    WORD * term2,
    WORD level )
```

Compares two terms. The answer is: 0 equal (with exception of the coefficient if level == 0.) >0 term1 comes first. <0 term2 comes first. Some special precautions may be needed to keep the CompCoef routine from generating overflows, although this is very unlikely in subterms. This routine should not return an error condition.

Originally this routine was called Compare. With the treatment of special polynomials with terms that contain only symbols and the need for extreme speed for the polynomial routines we made a special compare routine and now we store the address of the current compare routine in AR.CompareRoutine and have a macro Compare which makes all existing code work properly and we can just replace the routine on a thread by thread basis (each thread has its own AR struct).

Parameters

<i>term1</i>	First input term
<i>term2</i>	Second input term
<i>level</i>	The sorting level (may influence on the result)

Returns

0 equal (with exception of the coefficient if level == 0.) >0 term1 comes first. <0 term2 comes first.

When there are floating point numbers active (float_ = FLOATFUN) the presence of one or more float_ functions is returned in AT.SortFloatMode: 0: no float_ 1: float_ in term1 only 2: float_ in term2 only 3: float_ in both terms

Definition at line 2636 of file [sort.c](#).

References [CompCoef\(\)](#).

Referenced by [InFunction\(\)](#), [poly_sort\(\)](#), and [StartVariables\(\)](#).

Here is the call graph for this function:



4.128.3.13 CompareSymbols()

```
WORD CompareSymbols (
    PHEAD WORD * term1,
    WORD * term2,
    WORD par )
```

Compares the terms, based on the value of AN.polysortflag. If $\text{term1} < \text{term2}$ the return value is -1 If $\text{term1} > \text{term2}$ the return value is 1 If $\text{term1} = \text{term2}$ the return value is 0 The coefficients may differ. The terms contain only a single subterm of type SYMBOL. If AN.polysortflag = 0 it is a 'regular' compare. If AN.polysortflag = 1 the sum of the powers is more important par is a dummy parameter to make the parameter field identical to that of Compare1 which is the regular compare routine in [sort.c](#)

Definition at line 3100 of file [sort.c](#).

Referenced by [InFunction\(\)](#), and [poly_sort\(\)](#).

4.128.3.14 CompareHSymbols()

```
WORD CompareHSymbols (
    PHEAD WORD * term1,
    WORD * term2,
    WORD par )
```

Compares terms that can have only SYMBOL and HAAKJE subterms. If $\text{term1} < \text{term2}$ the return value is -1 If $\text{term1} > \text{term2}$ the return value is 1 If $\text{term1} = \text{term2}$ the return value is 0 par is a dummy parameter to make the parameter field identical to that of Compare1 which is the regular compare routine in [sort.c](#)

Definition at line 3143 of file [sort.c](#).

4.128.3.15 ComPress()

```
LONG ComPress (
    WORD ** ss,
    LONG * n )
```

Gets a list of pointers to terms and compresses the terms. In n it collects the number of terms and the return value of the function is the space that is occupied.

We have to pay some special attention to the compression of terms with a PolyFun. This PolyFun should occur only straight before the coefficient, so we can use the same trick as for the coefficient to sabotage compression of this object (Replace in the history the function pointer by zero. This is safe, because terms that would be identical otherwise would have been added).

Parameters

<i>ss</i>	Array of pointers to terms to be compressed.
<i>n</i>	Number of pointers in ss.

Returns

Total number of words needed for the compressed result.

Definition at line 3196 of file [sort.c](#).

Referenced by [EndSort\(\)](#), and [StoreTerm\(\)](#).

4.128.3.16 SplitMerge()

```
LONG SplitMerge (
    PHEAD WORD ** Pointer,
    LONG number )
```

Algorithm by J.A.M.Vermaseren (31-7-1988)

Note that AN.SplitScratch and AN.InScratch are used also in GarbHand

Merge sort in memory. The input is an array of pointers. Sorting is done recursively by dividing the array in two equal parts and calling SplitMerge for each. When the parts are small enough we can do the compare and take the appropriate action. An addition is that we look for 'runs'. Sequences that are already ordered. This happens a lot when there is very little action in a module. This made FORM faster by a few percent.

Parameters

<i>Pointer</i>	The array of pointers to the terms to be sorted.
<i>number</i>	The number of pointers in Pointer.

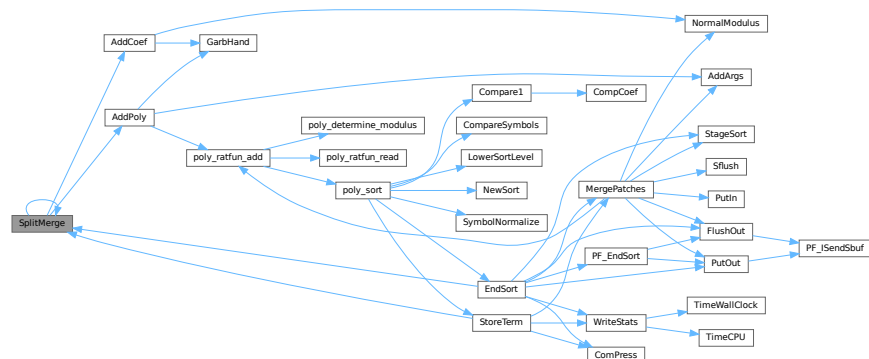
The terms are supposed to be sitting in the small buffer and there is supposed to be an extension to this buffer for when there are two terms that should be added and the result takes more space than each of the original terms. The notation guarantees that the result never needs more space than the sum of the spaces of the original terms.

Definition at line 3362 of file [sort.c](#).

References [AddCoef\(\)](#), [AddPoly\(\)](#), and [SplitMerge\(\)](#).

Referenced by [EndSort\(\)](#), [SplitMerge\(\)](#), and [StoreTerm\(\)](#).

Here is the call graph for this function:



4.128.3.17 GarbHand()

```
void GarbHand (
    void )
```

Garbage collection that takes place when the small extension is full and we need to place more terms there. When this is the case there are many holes in the small buffer and the whole can be compactified. The major complication is the buffer for SplitMerge. There are two options for temporary memory: 1: find some buffer that has enough space (maybe in the large buffer). 2: allocate a buffer. Give it back afterwards of course. If the small extension is properly dimensioned this routine should be called very rarely. Most of the time it will be called when the polyfun or polyratfun is active.

Definition at line [3642](#) of file [sort.c](#).

Referenced by [AddCoef\(\)](#), and [AddPoly\(\)](#).

4.128.3.18 MergePatches()

```
WORD MergePatches (
    WORD par )
```

The general merge routine. Can be used for the large buffer and the file merging. The array S->Patches tells where the patches start S->pStop tells where they end (has to be computed first). The end of a 'line to be merged' is indicated by a zero. If the end is reached without running into a zero or a term runs over the boundary of a patch it is a file merging operation and a new piece from the file is read in.

Parameters

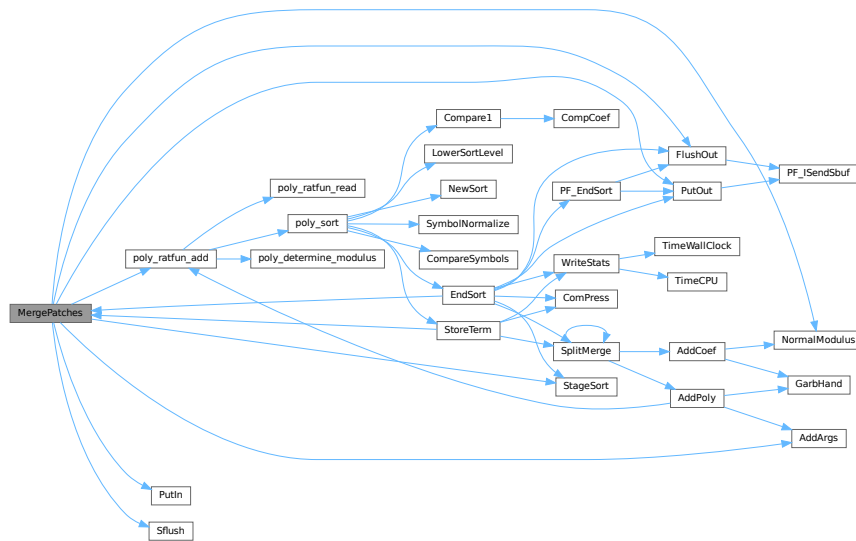
<i>par</i>	If <i>par</i> == 0 the sort is for file -> outputfile. If <i>par</i> == 1 the sort is for large buffer -> sortfile. If <i>par</i> == 2 the sort is for large buffer -> outputfile.
------------	--

Definition at line [3757](#) of file [sort.c](#).

References [AddArgs\(\)](#), [FlushOut\(\)](#), [File::handle](#), [NormalModulus\(\)](#), [poly_ratfun_add\(\)](#), [PutIn\(\)](#), [PutOut\(\)](#), [Sflush\(\)](#), and [StageSort\(\)](#).

Referenced by [EndSort\(\)](#), and [StoreTerm\(\)](#).

Here is the call graph for this function:



4.128.3.19 StoreTerm()

```
WORD StoreTerm (
    PHEAD WORD * term )
```

The central routine to accept terms, store them and keep things at least partially sorted. A call to `EndSort` will then complete storing and sorting.

Parameters

<i>term</i>	The term to be stored
-------------	-----------------------

Returns

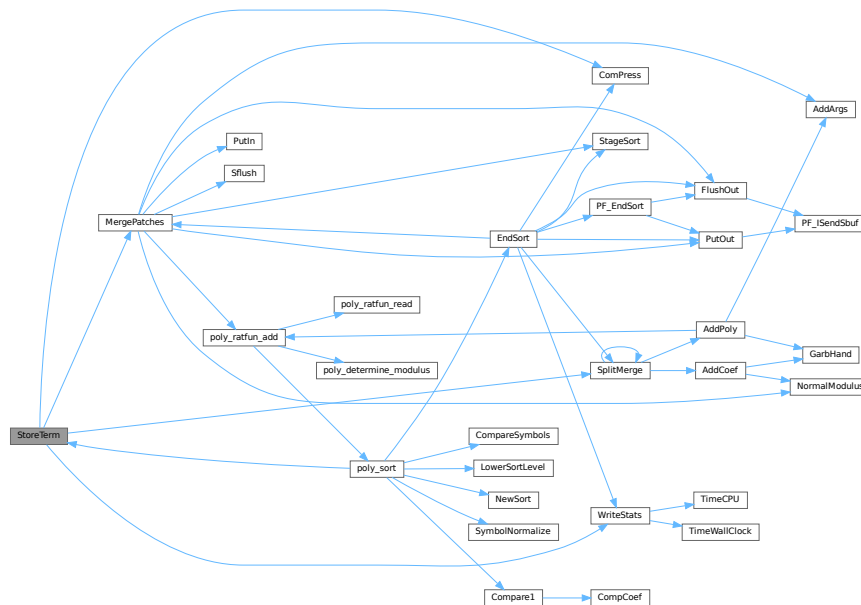
Regular return conventions (OK -> 0)

Definition at line 4540 of file `sort.c`.

References `ComPress()`, `MergePatches()`, `SplitMerge()`, and `WriteStats()`.

Referenced by `Generator()`, `get_expression()`, `InFunction()`, `PF_Processor()`, `poly_sort()`, `PolyFunMul()`, `Processor()`, and `TakeArgContent()`.

Here is the call graph for this function:



4.128.3.20 StageSort()

```
void StageSort (
    FILEHANDLE * fout )
```

Prepares a stage 4 or higher sort. Stage 4 sorts occur when the sort file contains more patches than can be merged in one pass.

Definition at line 4654 of file [sort.c](#).

References [FiLe::handle](#).

Referenced by [EndSort\(\)](#), and [MergePatches\(\)](#).

4.128.3.21 SortWild()

```
WORD SortWild (
    WORD * w,
    WORD nw )
```

Sorts the wildcard entries in the parameter w. Double entries are removed. Full space taken is nw words. Routine serves for the reading of wildcards in the compiler. The entries come in the format: (type,4,number,0) in which the zero is reserved for the future replacement of 'number'.

Parameters

<i>w</i>	buffer with wildcard entries.
<i>nw</i>	number of wildcard entries.

Returns

Normal conventions (OK -> 0)

Definition at line [4753](#) of file [sort.c](#).

4.128.3.22 CleanUpSort()

```
void CleanUpSort (
    int num )
```

Partially or completely frees function sort buffers.

Definition at line [4845](#) of file [sort.c](#).

4.128.3.23 LowerSortLevel()

```
void LowerSortLevel (
    void )
```

Lowers the level in the sort system.

Definition at line [4945](#) of file [sort.c](#).

Referenced by [generate_expression\(\)](#), [get_expression\(\)](#), [InFunction\(\)](#), [PF_CollectModifiedDollars\(\)](#), [PF_Processor\(\)](#), [poly_factorize_expression\(\)](#), [poly_sort\(\)](#), [poly_unfactorize_expression\(\)](#), [PolyFunMul\(\)](#), [Processor\(\)](#), and [TestMatch\(\)](#).

4.128.3.24 PolyRatFunSpecial()

```
WORD * PolyRatFunSpecial (
    PHEAD WORD * t1,
    WORD * t2 )
```

Definition at line [4962](#) of file [sort.c](#).

4.128.3.25 SimpleSplitMergeRec()

```
void SimpleSplitMergeRec (
    WORD * array,
    WORD num,
    WORD * auxarray )
```

Definition at line [5064](#) of file [sort.c](#).

4.128.3.26 SimpleSplitMerge()

```
void SimpleSplitMerge (
    WORD * array,
    WORD num )
```

Definition at line [5092](#) of file [sort.c](#).

4.128.3.27 BinarySearch()

```
WORD BinarySearch (
    WORD * array,
    WORD num,
    WORD x )
```

Definition at line [5110](#) of file [sort.c](#).

4.128.4 Variable Documentation

4.128.4.1 numcompares

```
LONG numcompares
```

Definition at line [75](#) of file [sort.c](#).

4.128.4.2 toterms

```
char* toterms[] = { " ", " >>", "-->" }
```

Definition at line [83](#) of file [sort.c](#).

4.128.4.3 humanTermsSuffix

```
const char humanTermsSuffix[HUMANSUFFLEN][4] = {"K ", "M ", "B ", "T "}
```

Definition at line [87](#) of file [sort.c](#).

4.128.4.4 humanBytesSuffix

```
const char humanBytesSuffix[HUMANSUFFLEN][4] = {"KiB", "MiB", "GiB", "TiB"}
```

Definition at line [88](#) of file [sort.c](#).

4.129 sort.c

[Go to the documentation of this file.](#)

```

00001
00017 /* #[ License : */
00018 /*
00019 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00020 *   When using this file you are requested to refer to the publication
00021 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00022 *   This is considered a matter of courtesy as the development was paid
00023 *   for by FOM the Dutch physics granting agency and we would like to
00024 *   be able to track its scientific use to convince FOM of its value
00025 *   for the community.
00026 *
00027 *   This file is part of FORM.
00028 *
00029 *   FORM is free software: you can redistribute it and/or modify it under the
00030 *   terms of the GNU General Public License as published by the Free Software
00031 *   Foundation, either version 3 of the License, or (at your option) any later
00032 *   version.
00033 *
00034 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00035 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00036 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00037 *   details.
00038 *
00039 *   You should have received a copy of the GNU General Public License along
00040 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00041 */
00042 /* #[ License : */
00043 /*
00044 *   #[ Includes : sort.c
00045 *
00046 *   Sort routines according to new conventions (25-jun-1997).
00047 *   This is more object oriented.
00048 *   The active sort is indicated by AT.SS which should agree with
00049 *   AN.FunSorts[AR.sLevel];
00050
00051 #define GZIPDEBUG
00052 */
00053 #define NEWSPLITMERGE
00054 /* Comment to turn off Timsort in SplitMerge for debugging: */
00055 #define NEWSPLITMERGETIMSORT
00056 /* During SplitMerge, print pointer array state on entry. Very spammy, for debugging. */
00057 /* #define SPLITMERGEDEBUG */
00058 /* Debug printing for GarbHand */
00059 /* #define TESTGARB */
00060
00061 #include "form3.h"
00062
00063 #ifdef WITHPTHREADS
00064 UBYTE THRbuf[100];
00065 #endif
00066
00067 #ifdef WITHSTATS
00068 extern LONG numwrites;
00069 extern LONG numreads;
00070 extern LONG numseeks;
00071 extern LONG nummallocs;
00072 extern LONG numfrees;
00073 #endif
00074
00075 LONG numcompares;
00076
00077 /*
00078 *   #[ Includes :
00079 *   #[ SortUtilities :
00080 *   #[ WriteStats :          void WriteStats(lspace,par,checkLogType)
00081 */
00082
00083 char *totterms[] = { " ", " ", " »", "-->" };
00084
00085 #define HUMANSTRLEN 12
00086 #define HUMANSUFFLEN 4
00087 const char humanTermsSuffix[HUMANSUFFLEN][4] = {"K ", "M ", "B ", "T "};
00088 const char humanBytesSuffix[HUMANSUFFLEN][4] = {"KiB", "MiB", "GiB", "TiB"};
00089 void HumanString(char* string, float input, const char suffix[HUMANSUFFLEN][4]) {
00090     int ind = -1;
00091     while (ind < 0 || (input >= 1000.0 && ind < HUMANSUFFLEN) ) {
00092         input /= 1000.0;
00093         ind++;
00094     }
00095     if ( input < 0.5 ) {
00096         snprintf(string, HUMANSTRLEN,
00097             " ( <1 %s)", suffix[ind]);

```

```

00098     }
00099     else {
00100         snprintf(string, HUMANSTRLEN,
00101             " (%3.f %s)", input, suffix[ind]);
00102     }
00103 }
00104
00121 void WriteStats(POSITION *plspace, WORD par, WORD checkLogType)
00122 {
00123     GETIDENTITY
00124     LONG millitime, y = 0x7FFFFFFFL >> 1;
00125     WORD timepart;
00126     SORTING *S;
00127     POSITION pp;
00128     int use_wtime;
00129     if ( AT.SS == AT.S0 && AC.StatsFlag ) {
00130 #ifdef WITHPTHREADS
00131         if ( AC.ThreadStats == 0 && identity > 0 ) return;
00132 #elif defined(WITHMPI)
00133         if ( AC.OldParallelStats ) return;
00134         if ( ! AC.ProcessStats && PF.me != MASTER ) return;
00135 #endif
00136         if ( Expressions == 0 ) return;
00137
00138         if ( par == STATSSPLITMERGE ) {
00139             if ( AC.ShortStatsMax == 0 ) return;
00140             AR.ShortSortCount++;
00141             if ( AR.ShortSortCount < AC.ShortStatsMax ) return;
00142         }
00143         AR.ShortSortCount = 0;
00144
00145         S = AT.SS;
00146
00147         char humanGenTermsText[HUMANSTRLEN] = "";
00148         char humanTermsLeftText[HUMANSTRLEN] = "";
00149         char humanBytesText[HUMANSTRLEN] = "";
00150         if ( AC.HumanStatsFlag ) {
00151             HumanString(humanGenTermsText, (float)(S->GenTerms), humanTermsSuffix);
00152             HumanString(humanTermsLeftText, (float)(S->TermsLeft), humanTermsSuffix);
00153             HumanString(humanBytesText, (float)(BASEPOSITION(*plspace)), humanBytesSuffix);
00154         }
00155
00156         MLOCK(ErrorMessageLock);
00157
00158         /* If the statistics should not go to the log file, temporarily hide the
00159          * LogHandle. This must be done within the ErrorMessageLock region to
00160          * avoid a data race between threads. */
00161         const WORD oldLogHandle = AC.LogHandle;
00162         if ( checkLogType && AM.LogType ) {
00163             AC.LogHandle = -1;
00164         }
00165
00166         if ( AC.ShortStats ) {}
00167         else {
00168 #ifdef WITHPTHREADS
00169             if ( identity > 0 ) {
00170                 MesPrint("                Thread %d reporting",identity);
00171             }
00172             else {
00173                 MesPrint("");
00174             }
00175 #elif defined(WITHMPI)
00176             if ( PF.me != MASTER ) {
00177                 MesPrint("                Process %d reporting",PF.me);
00178             }
00179             else {
00180                 MesPrint("");
00181             }
00182 #else
00183             MesPrint("");
00184 #endif
00185         }
00186         /*
00187          * We define WTimeStatsFlag as a flag to print the wall-clock time on
00188          * the *master*, not in workers. This can be confusing in thread
00189          * statistics when short statistics is used. Technically,
00190          * TimeWallClock() is not thread-safe in TFORM.
00191          */
00192         use_wtime = AC.WTimeStatsFlag;
00193 #if defined(WITHPTHREADS)
00194         if ( use_wtime && identity > 0 ) use_wtime = 0;
00195 #elif defined(WITHMPI)
00196         if ( use_wtime && PF.me != MASTER ) use_wtime = 0;
00197 #endif
00198         millitime = use_wtime ? TimeWallClock(1) * 10 : TimeCPU(1);
00199         timepart = (WORD)(millitime%1000);
00200         millitime /= 1000;

```

```

00201         timepart /= 10;
00202         if ( AC.ShortStats ) {
00203             #if defined(WITHPTHREADS) || defined(WITHMPI)
00204             #ifdef WITHPTHREADS
00205                 if ( identity > 0 ) {
00206                     #else
00207                     if ( PF.me != MASTER ) {
00208                         const int identity = PF.me;
00209                     #endif
00210                     if ( par == STATSSPLITMERGE || par == STATSPOSTSORT ) {
00211                         SETBASEPOSITION(pp,y);
00212                         if ( ISLESSPOS(*plspace,pp) ) {
00213                             MesPrint("%d: %7l.%2is %8l>%10l%3s%10l:%10p %s %s",identity,
00214                                 millitime,timepart,AN.ninterms,S->GenTerms,toterms[par],
00215                                 S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00216                             /*
00217                             MesPrint("%d: %14s %7l.%2is %8l>%10l%3s%10l:%10p",identity,
00218                                 EXPRNAME(AR.CurExpr),AC.Commercial,millitime,timepart,
00219                                 AN.ninterms,S->GenTerms,toterms[par],S->TermsLeft,plspace);
00220                             */
00221                         }
00222                     else {
00223                         y = 1000000000L;
00224                         SETBASEPOSITION(pp,y);
00225                         MULPOS(pp,100);
00226                         if ( ISLESSPOS(*plspace,pp) ) {
00227                             MesPrint("%d: %7l.%2is %8l>%10l%3s%10l:%11p %s %s",identity,
00228                                 millitime,timepart,AN.ninterms,S->GenTerms,toterms[par],
00229                                 S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00230                         }
00231                     else {
00232                         MULPOS(pp,10);
00233                         if ( ISLESSPOS(*plspace,pp) ) {
00234                             MesPrint("%d: %7l.%2is %8l>%10l%3s%10l:%12p %s %s",identity,
00235                                 millitime,timepart,AN.ninterms,S->GenTerms,toterms[par],
00236                                 S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00237                         }
00238                     else {
00239                         MULPOS(pp,10);
00240                         if ( ISLESSPOS(*plspace,pp) ) {
00241                             MesPrint("%d: %7l.%2is %8l>%10l%3s%10l:%13p %s %s",identity,
00242                                 millitime,timepart,AN.ninterms,S->GenTerms,toterms[par],
00243                                 S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00244                         }
00245                     else {
00246                         MULPOS(pp,10);
00247                         if ( ISLESSPOS(*plspace,pp) ) {
00248                             MesPrint("%d: %7l.%2is %8l>%10l%3s%10l:%14p %s %s",identity,
00249                                 millitime,timepart,AN.ninterms,S->GenTerms,toterms[par],
00250                                 S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00251                         }
00252                     else {
00253                         MULPOS(pp,10);
00254                         if ( ISLESSPOS(*plspace,pp) ) {
00255                             MesPrint("%d: %7l.%2is %8l>%10l%3s%10l:%15p %s %s",identity,
00256                                 millitime,timepart,AN.ninterms,S->GenTerms,toterms[par],
00257                                 S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00258                         }
00259                     else {
00260                         MULPOS(pp,10);
00261                         if ( ISLESSPOS(*plspace,pp) ) {
00262                             MesPrint("%d: %7l.%2is %8l>%10l%3s%10l:%16p %s %s",identity,
00263                                 millitime,timepart,AN.ninterms,S->GenTerms,toterms[par],
00264                                 S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00265                         }
00266                     else {
00267                         MULPOS(pp,10);
00268                         if ( ISLESSPOS(*plspace,pp) ) {
00269                             MesPrint("%d: %7l.%2is %8l>%10l%3s%10l:%17p %s %s",identity,
00270                                 millitime,timepart,AN.ninterms,S->GenTerms,toterms[par],
00271                                 S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00272                         }
00273                     } } } } }
00274                 }
00275             }
00276         }
00277     else if ( par == STATSMERGETOFILE ) {
00278         SETBASEPOSITION(pp,y);
00279         if ( ISLESSPOS(*plspace,pp) ) {
00280             MesPrint("%d: %7l.%2is %10l:%10p",identity,millitime,timepart,
00281                 S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00282         }
00283     else {
00284         y = 1000000000L;
00285         SETBASEPOSITION(pp,y);
00286         MULPOS(pp,100);
00287         if ( ISLESSPOS(*plspace,pp) ) {

```

```

00288         MesPrint("%d: %7l.%2is %10l:%11p",identity,millitime,timepart,
00289         S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00290     }
00291     else {
00292         MULPOS(pp,10);
00293         if ( ISLESSPOS(*plspace,pp) ) {
00294             MesPrint("%d: %7l.%2is %10l:%12p",identity,millitime,timepart,
00295             S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00296         }
00297         else {
00298             MULPOS(pp,10);
00299             if ( ISLESSPOS(*plspace,pp) ) {
00300                 MesPrint("%d: %7l.%2is %10l:%13p",identity,millitime,timepart,
00301                 S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00302             }
00303             else {
00304                 MULPOS(pp,10);
00305                 if ( ISLESSPOS(*plspace,pp) ) {
00306                     MesPrint("%d: %7l.%2is %10l:%14p",identity,millitime,timepart,
00307                     S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00308                 }
00309                 else {
00310                     MULPOS(pp,10);
00311                     if ( ISLESSPOS(*plspace,pp) ) {
00312                         MesPrint("%d: %7l.%2is %10l:%15p",identity,millitime,timepart,
00313                         S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00314                     }
00315                     else {
00316                         MULPOS(pp,10);
00317                         if ( ISLESSPOS(*plspace,pp) ) {
00318                             MesPrint("%d: %7l.%2is %10l:%16p",identity,millitime,timepart,
00319                             S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00320                         }
00321                         else {
00322                             MULPOS(pp,10);
00323                             if ( ISLESSPOS(*plspace,pp) ) {
00324                                 MesPrint("%d: %7l.%2is %10l:%17p",identity,millitime,timepart,
00325                                 S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00326                             }
00327                             } } } } }
00328     }
00329 }
00330 } } else
00331 #endif
00332 {
00333 if ( par == STATSSPLITMERGE || par == STATSPOSTSORT ) {
00334     SETBASEPOSITION(pp,y);
00335     if ( ISLESSPOS(*plspace,pp) ) {
00336         MesPrint("%7l.%2is %8l>%10l%3s%10l:%10p %s %s",
00337         millitime,timepart,AN.ninterms,S->GenTerms,toterms[par],
00338         S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00339         /*
00340         MesPrint("%14s %17s %7l.%2is %8l>%10l%3s%10l:%10p",
00341         EXPRNAME(AR.CurExpr),AC.Commercial,millitime,timepart,
00342         AN.ninterms,S->GenTerms,toterms[par],S->TermsLeft,plspace);
00343         */
00344     }
00345     else {
00346         y = 1000000000L;
00347         SETBASEPOSITION(pp,y);
00348         MULPOS(pp,100);
00349         if ( ISLESSPOS(*plspace,pp) ) {
00350             MesPrint("%7l.%2is %8l>%10l%3s%10l:%11p %s %s",
00351             millitime,timepart,AN.ninterms,S->GenTerms,toterms[par],
00352             S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00353         }
00354         else {
00355             MULPOS(pp,10);
00356             if ( ISLESSPOS(*plspace,pp) ) {
00357                 MesPrint("%7l.%2is %8l>%10l%3s%10l:%12p %s %s",
00358                 millitime,timepart,AN.ninterms,S->GenTerms,toterms[par],
00359                 S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00360             }
00361             else {
00362                 MULPOS(pp,10);
00363                 if ( ISLESSPOS(*plspace,pp) ) {
00364                     MesPrint("%7l.%2is %8l>%10l%3s%10l:%13p %s %s",
00365                     millitime,timepart,AN.ninterms,S->GenTerms,toterms[par],
00366                     S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00367                 }
00368                 else {
00369                     MULPOS(pp,10);
00370                     if ( ISLESSPOS(*plspace,pp) ) {
00371                         MesPrint("%7l.%2is %8l>%10l%3s%10l:%14p %s %s",
00372                         millitime,timepart,AN.ninterms,S->GenTerms,toterms[par],
00373                         S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00374                     }

```

```

00375         else {
00376             MULPOS(pp,10);
00377             if ( ISLESSPOS(*plspace,pp) ) {
00378                 MesPrint("%7l.%2is %8l>%10l%3s%10l:%15p %s %s",
00379                     millitime,timepart,AN.ninterms,S->GenTerms,toterms[par],
00380                     S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00381             }
00382         }
00383         else {
00384             MULPOS(pp,10);
00385             if ( ISLESSPOS(*plspace,pp) ) {
00386                 MesPrint("%7l.%2is %8l>%10l%3s%10l:%16p %s %s",
00387                     millitime,timepart,AN.ninterms,S->GenTerms,toterms[par],
00388                     S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00389             }
00390         }
00391         else {
00392             MULPOS(pp,10);
00393             if ( ISLESSPOS(*plspace,pp) ) {
00394                 MesPrint("%7l.%2is %8l>%10l%3s%10l:%17p %s %s",
00395                     millitime,timepart,AN.ninterms,S->GenTerms,toterms[par],
00396                     S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00397             }
00398         }
00399     }
00400     else if ( par == STATSMERGETOFILE ) {
00401         SETBASEPOSITION(pp,y);
00402         if ( ISLESSPOS(*plspace,pp) ) {
00403             MesPrint("%7l.%2is %10l:%10p",millitime,timepart,
00404                 S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00405         }
00406         else {
00407             y = 1000000000L;
00408             SETBASEPOSITION(pp,y);
00409             MULPOS(pp,100);
00410             if ( ISLESSPOS(*plspace,pp) ) {
00411                 MesPrint("%7l.%2is %10l:%11p",millitime,timepart,
00412                     S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00413             }
00414             else {
00415                 MULPOS(pp,10);
00416                 if ( ISLESSPOS(*plspace,pp) ) {
00417                     MesPrint("%7l.%2is %10l:%12p",millitime,timepart,
00418                         S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00419                 }
00420                 else {
00421                     MULPOS(pp,10);
00422                     if ( ISLESSPOS(*plspace,pp) ) {
00423                         MesPrint("%7l.%2is %10l:%13p",millitime,timepart,
00424                             S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00425                     }
00426                     else {
00427                         MULPOS(pp,10);
00428                         if ( ISLESSPOS(*plspace,pp) ) {
00429                             MesPrint("%7l.%2is %10l:%14p",millitime,timepart,
00430                                 S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00431                         }
00432                         else {
00433                             MULPOS(pp,10);
00434                             if ( ISLESSPOS(*plspace,pp) ) {
00435                                 MesPrint("%7l.%2is %10l:%15p",millitime,timepart,
00436                                     S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00437                             }
00438                             else {
00439                                 MULPOS(pp,10);
00440                                 if ( ISLESSPOS(*plspace,pp) ) {
00441                                     MesPrint("%7l.%2is %10l:%16p",millitime,timepart,
00442                                         S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00443                                 }
00444                                 else {
00445                                     MULPOS(pp,10);
00446                                     if ( ISLESSPOS(*plspace,pp) ) {
00447                                         MesPrint("%7l.%2is %10l:%17p",millitime,timepart,
00448                                             S->TermsLeft,plspace,EXPRNAME(AR.CurExpr),AC.Commercial);
00449                                     }
00450                                 }
00451                             }
00452                         }
00453                     }
00454                 }
00455             }
00456         else if ( par == STATSMERGETOFILE ) {
00457             if ( use_wtime ) {
00458                 MesPrint("WTime = %7l.%2i sec",millitime,timepart);
00459             }
00460             else {
00461                 MesPrint("Time = %7l.%2i sec",millitime,timepart);

```

```

00462     }
00463     }
00464     else {
00465     #if ( BITSINLONG > 32 )
00466         if ( S->GenTerms >= 10000000000L ) {
00467             if ( use_wtime ) {
00468                 MesPrint("WTime = %7l.%2i sec    Generated terms = %16l%s",
00469                     millitime,timepart,S->GenTerms,humanGenTermsText);
00470             }
00471             else {
00472                 MesPrint("Time = %7l.%2i sec    Generated terms = %16l%s",
00473                     millitime,timepart,S->GenTerms,humanGenTermsText);
00474             }
00475         }
00476     else {
00477         if ( use_wtime ) {
00478             MesPrint("WTime = %7l.%2i sec    Generated terms = %10l%s",
00479                 millitime,timepart,S->GenTerms,humanGenTermsText);
00480         }
00481         else {
00482             MesPrint("Time = %7l.%2i sec    Generated terms = %10l%s",
00483                 millitime,timepart,S->GenTerms,humanGenTermsText);
00484         }
00485     }
00486 #else
00487     if ( use_wtime ) {
00488         MesPrint("WTime = %7l.%2i sec    Generated terms = %10l%s",
00489             millitime,timepart,S->GenTerms,humanGenTermsText);
00490     }
00491     else {
00492         MesPrint("Time = %7l.%2i sec    Generated terms = %10l%s",
00493             millitime,timepart,S->GenTerms,humanGenTermsText);
00494     }
00495 #endif
00496 }
00497 #if ( BITSINLONG > 32 )
00498     if ( par == STATSSPLITMERGE )
00499         if ( S->TermsLeft >= 10000000000L ) {
00500             MesPrint("%16s%8l Terms %s = %16l%s",EXPRNAME(AR.CurExpr),
00501                 AN.ninterms,FG.swmes[par],S->TermsLeft,humanTermsLeftText);
00502         }
00503         else {
00504             MesPrint("%16s%8l Terms %s = %10l%s",EXPRNAME(AR.CurExpr),
00505                 AN.ninterms,FG.swmes[par],S->TermsLeft,humanTermsLeftText);
00506         }
00507     else {
00508         if ( S->TermsLeft >= 10000000000L ) {
00509             #ifdef WITHPTHREADS
00510                 if ( identity > 0 && par == STATSPOSTSORT ) {
00511                     MesPrint("%16s          Terms in thread = %16l%s",
00512                         EXPRNAME(AR.CurExpr),S->TermsLeft,humanTermsLeftText);
00513                 }
00514                 else
00515                     #elif defined(WITHMPI)
00516                     if ( PF.me != MASTER && par == STATSPOSTSORT ) {
00517                         MesPrint("%16s          Terms in process= %16l%s",
00518                             EXPRNAME(AR.CurExpr),S->TermsLeft,humanTermsLeftText);
00519                     }
00520                     else
00521                         #endif
00522                         {
00523                             MesPrint("%16s          Terms %s = %16l%s",
00524                                 EXPRNAME(AR.CurExpr),FG.swmes[par],S->TermsLeft,humanTermsLeftText);
00525                         }
00526                 }
00527             else {
00528                 #ifdef WITHPTHREADS
00529                     if ( identity > 0 && par == STATSPOSTSORT ) {
00530                         MesPrint("%16s          Terms in thread = %10l%s",
00531                             EXPRNAME(AR.CurExpr),S->TermsLeft,humanTermsLeftText);
00532                     }
00533                     else
00534                         #elif defined(WITHMPI)
00535                         if ( PF.me != MASTER && par == STATSPOSTSORT ) {
00536                             MesPrint("%16s          Terms in process= %10l%s",
00537                                 EXPRNAME(AR.CurExpr),S->TermsLeft,humanTermsLeftText);
00538                         }
00539                         else
00540                             #endif
00541                             {
00542                                 MesPrint("%16s          Terms %s = %10l%s",
00543                                     EXPRNAME(AR.CurExpr),FG.swmes[par],S->TermsLeft,humanTermsLeftText);
00544                             }
00545                 }
00546             }
00547         }
00548     #else
00549         if ( par == STATSSPLITMERGE )

```

```

00549         MesPrint("%16s%8l Terms %s = %10l%s",EXPRNAME(AR.CurExpr),
00550         AN.ninterms,FG.swmes[par],S->TermsLeft,humanTermsLeftText);
00551     else {
00552 #ifdef WITHPTHREADS
00553         if ( identity > 0 && par == STATSPOSTSORT ) {
00554             MesPrint("%16s          Terms in thread = %10l%s",
00555             EXPRNAME(AR.CurExpr),S->TermsLeft,humanTermsLeftText);
00556         }
00557     else
00558 #elif defined(WITHMPI)
00559         if ( PF.me != MASTER && par == STATSPOSTSORT ) {
00560             MesPrint("%16s          Terms in process= %10l%s",
00561             EXPRNAME(AR.CurExpr),S->TermsLeft,humanTermsLeftText);
00562         }
00563     else
00564 #endif
00565     {
00566         MesPrint("%16s          Terms %s = %10l%s",
00567         EXPRNAME(AR.CurExpr),FG.swmes[par],S->TermsLeft,humanTermsLeftText);
00568     }
00569 }
00570 #endif
00571 SETBASEPOSITION(pp,y);
00572 if ( ISLESSPOS(*plspace,pp) ) {
00573     MesPrint("%24s Bytes used      = %10p%s",AC.Commercial,plspace,humanBytesText);
00574 }
00575 else {
00576     y = 1000000000L;
00577     SETBASEPOSITION(pp,y);
00578     MULPOS(pp,100);
00579     if ( ISLESSPOS(*plspace,pp) ) {
00580         MesPrint("%24s Bytes used      =%11p%s",AC.Commercial,plspace,humanBytesText);
00581     }
00582     else {
00583         MULPOS(pp,10);
00584         if ( ISLESSPOS(*plspace,pp) ) {
00585             MesPrint("%24s Bytes used      =%12p%s",AC.Commercial,plspace,humanBytesText);
00586         }
00587         else {
00588             MULPOS(pp,10);
00589             if ( ISLESSPOS(*plspace,pp) ) {
00590                 MesPrint("%24s Bytes used      =%13p%s",AC.Commercial,plspace,humanBytesText);
00591             }
00592             else {
00593                 MULPOS(pp,10);
00594                 if ( ISLESSPOS(*plspace,pp) ) {
00595                     MesPrint("%24s Bytes used      =%14p%s",AC.Commercial,plspace,humanBytesText);
00596                 }
00597                 else {
00598                     MULPOS(pp,10);
00599                     if ( ISLESSPOS(*plspace,pp) ) {
00600                         MesPrint("%24s Bytes used      =%15p%s",AC.Commercial,plspace,humanBytesText);
00601                     }
00602                     else {
00603                         MULPOS(pp,10);
00604                         if ( ISLESSPOS(*plspace,pp) ) {
00605                             MesPrint("%24s Bytes used      =%16p%s",AC.Commercial,plspace,humanBytesText);
00606                         }
00607                         else {
00608                             MULPOS(pp,10);
00609                             if ( ISLESSPOS(*plspace,pp) ) {
00610                                 MesPrint("%24s Bytes used=%17p%s",AC.Commercial,plspace,humanBytesText);
00611                             }
00612                             } } } } }
00613     }
00614 } }
00615 #ifdef WITHSTATS
00616 MesPrint("Total number of writes: %l, reads: %l, seeks, %l"
00617         ,numwrites,numreads,numseeks);
00618 MesPrint("Total number of mallocs: %l, frees: %l"
00619         ,nummallocs,numfrees);
00620 #endif
00621 /* Put back the original LogHandle, it was changed if the statistics were
00622  * not printed in the log file. */
00623 AC.LogHandle = oldLogHandle;
00624
00625 MUNLOCK(ErrorMessageLock);
00626 }
00627 }
00628
00629 /*
00630     #] WriteStats :
00631     #[ NewSort :          WORD NewSort()
00632 */
00643 WORD NewSort(PHEAD0)
00644 {
00645     GETBIDENTITY

```

```

00646     SORTING *S, **newFS;
00647     int i, newsize;
00648     if ( AN.SoScratC == 0 )
00649         AN.SoScratC = (UWORD *)Malloc1(2*(AM.MaxTal+2)*sizeof(UWORD), "NewSort");
00650     AR.sLevel++;
00651     if ( AR.sLevel >= AN.NumFunSorts ) {
00652         if ( AN.NumFunSorts == 0 ) newsize = 100;
00653         else newsize = 2*AN.NumFunSorts;
00654         newFS = (SORTING **)Malloc1((newsize+1)*sizeof(SORTING *), "FunSort pointers");
00655         for ( i = 0; i < AN.NumFunSorts; i++ ) newFS[i] = AN.FunSorts[i];
00656         for ( ; i <= newsize; i++ ) newFS[i] = 0;
00657         if ( AN.FunSorts ) M_free(AN.FunSorts, "FunSort pointers");
00658         AN.FunSorts = newFS; AN.NumFunSorts = newsize;
00659     }
00660     if ( AR.sLevel == 0 ) {
00661         numcompares = 0;
00662
00663         AN.FunSorts[0] = AT.S0;
00664         if ( AR.PolyFun == 0 ) { AT.S0->PolyFlag = 0; }
00665         else if ( AR.PolyFunType == 1 ) { AT.S0->PolyFlag = 1; }
00666         else if ( AR.PolyFunType == 2 ) {
00667             if ( AR.PolyFunExp == 2
00668                 || AR.PolyFunExp == 3 ) AT.S0->PolyFlag = 1;
00669             else AT.S0->PolyFlag = 2;
00670         }
00671         AR.ShortSortCount = 0;
00672     }
00673     else {
00674         if ( AN.FunSorts[AR.sLevel] == 0 ) {
00675             AN.FunSorts[AR.sLevel] = AllocSort(
00676                 AM.SLargeSize, AM.SSmallSize, AM.SSmallEsize, AM.STermsInSmall
00677                 , AM.SMaxPatches, AM.SMaxFpatches, AM.SIOsize, 1);
00678         }
00679         AN.FunSorts[AR.sLevel]->PolyFlag = 0;
00680     }
00681     AT.SS = S = AN.FunSorts[AR.sLevel];
00682     S->sFill = S->sBuffer;
00683     S->lFill = S->lBuffer;
00684     S->lPatch = 0;
00685     S->fPatchN = 0;
00686     S->GenTerms = S->TermsLeft = S->GenSpace = S->SpaceLeft = 0;
00687     S->PoinFill = S->sPointer;
00688     *S->PoinFill = S->sFill;
00689     if ( AR.sLevel > 0 ) { S->PolyWise = 0; }
00690     PUTZERO(S->SizeInFile[0]); PUTZERO(S->SizeInFile[1]); PUTZERO(S->SizeInFile[2]);
00691     S->sTerms = 0;
00692     PUTZERO(S->file.POposition);
00693     S->stage4 = 0;
00694     if ( AR.sLevel > AN.MaxFunSorts ) AN.MaxFunSorts = AR.sLevel;
00695     /*
00696     The next variable is for the staged sort only.
00697     It should be treated differently
00698
00699     PUTZERO(AN.OldPosOut);
00700     */
00701     return(0);
00702 }
00703
00704 /*
00705 #] NewSort :
00706 #[ EndSort : WORD EndSort(PHEAD buffer, par)
00707 */
00733 LONG EndSort(PHEAD WORD *buffer, int par)
00734 {
00735     GETBIDENTITY
00736     SORTING *S = AT.SS;
00737     WORD j, **ss, *to, *t;
00738     LONG sSpace, over, tover, spare, retval = 0, jj;
00739     POSITION position, pp;
00740     off_t lSpace;
00741     FILEHANDLE *fout = 0, *oldoutfile = 0, *newout = 0;
00742
00743     if ( AM.exitflag && AR.sLevel == 0 ) return(0);
00744     #ifndef WITHMPI
00745     if( (retval = PF_EndSort()) > 0){
00746         oldoutfile = AR.outfile;
00747         retval = 0;
00748         goto RetRetval;
00749     }
00750     else if(retval < 0){
00751         retval = -1;
00752         goto RetRetval;
00753     }
00754     /* PF_EndSort returned 0: for S != AM.S0 and slaves still do the regular sort */
00755     #endif /* WITHMPI */
00756     oldoutfile = AR.outfile;

```



```

00757 /*      PolyFlag repair action
00758     if ( S == AT.S0 ) {
00759         if ( AR.PolyFun == 0 ) { S->PolyFlag = 0; }
00760         else if ( AR.PolyFunType == 1 ) { S->PolyFlag = 1; }
00761         else if ( AR.PolyFunType == 2 ) {
00762             if ( AR.PolyFunExp == 2
00763                 || AR.PolyFunExp == 3 ) S->PolyFlag = 1;
00764             else S->PolyFlag = 2;
00765         }
00766         S->PolyWise = 0;
00767     }
00768     else {
00769         S->PolyFlag = S->PolyWise = 0;
00770     }
00771 */
00772     S->PolyWise = 0;
00773     *(S->PoinFill) = 0;
00774 #ifdef SPLITTIME
00775     PrintTime((UBYTE *)"EndSort, before SplitMerge");
00776 #endif
00777     S->sPointer[SplitMerge(BHEAD S->sPointer,S->sTerms)] = 0;
00778 #ifdef SPLITTIME
00779     PrintTime((UBYTE *)"Endsort, after SplitMerge");
00780 #endif
00781     sSpace = 0;
00782     tover = over = S->sTerms;
00783     ss = S->sPointer;
00784     if ( over >= 0 ) {
00785         if ( S->lPatch > 0 || S->file.handle >= 0 ) {
00786             ss[over] = 0;
00787             sSpace = ComPress(ss,&sSpace);
00788             S->TermsLeft -= over - sSpace;
00789             if ( par == 1 ) { AR.outfile = newout = AllocFileHandle(0,(char *)0); }
00790         }
00791         else if ( S != AT.S0 ) {
00792             ss[over] = 0;
00793             if ( par == 2 ) {
00794                 sSpace = 3;
00795                 while ( ( t = *ss++ ) != 0 ) { sSpace += *t; }
00796                 if ( AN.tryterm > 0 && ( (sSpace+1)*sizeof(WORD) < (size_t)(AM.MaxTer) ) ) {
00797                     to = TermMalloc("$-sort space");
00798                 }
00799                 else {
00800                     LONG allocsp = sSpace+1;
00801                     if ( allocsp < MINALLOC ) allocsp = MINALLOC;
00802                     allocsp = (allocsp+7)/8*8;
00803                     to = (WORD *)Malloc1(allocsp*sizeof(WORD),"$-sort space");
00804                     if ( AN.tryterm > 0 ) AN.tryterm = 0;
00805                 }
00806                 *((WORD **)buffer) = to;
00807                 ss = S->sPointer;
00808                 while ( ( t = *ss++ ) != 0 ) {
00809                     j = *t; while ( --j >= 0 ) *to++ = *t++;
00810                 }
00811                 *to = 0;
00812                 retval = sSpace + 1;
00813             }
00814             else {
00815                 to = buffer;
00816                 sSpace = 0;
00817                 while ( ( t = *ss++ ) != 0 ) {
00818                     j = *t;
00819                     if ( ( sSpace += j ) > AM.MaxTer/((LONG)sizeof(WORD)) ) {
00820                         /* Too big! Get the total size for useful error message */
00821                         while ( ( t = *ss++ ) != 0 ) {
00822                             sSpace += *t;
00823                         }
00824                         MLOCK(ErrorMessageLock);
00825                         MesPrint("Sorted function argument too long (%d words). Increase MaxTermSize
00826 (%l words).", sSpace, AM.MaxTer/((LONG)sizeof(WORD)));
00827                         MUNLOCK(ErrorMessageLock);
00828                         retval = -1; goto RetRetval;
00829                     }
00830                     while ( --j >= 0 ) *to++ = *t++;
00831                 }
00832                 *to = 0;
00833                 retval = to - buffer;
00834             }
00835             goto RetRetval;
00836         }
00837         else {
00838             POSITION oldpos;
00839             if ( S == AT.S0 ) {
00840                 fout = AR.outfile;
00841                 *AR.CompressPointer = 0;
00842                 SeekScratch(AR.outfile,&position);

```

```

00843         else {
00844             fout = &(S->file);
00845             PUTZERO(position);
00846         }
00847         oldpos = position;
00848         S->TermsLeft = 0;
00849         /*
00850             Here we can go directly to the output.
00851         */
00852         #ifdef WITHZLIB
00853             { int oldgzipCompress = AR.gzipCompress;
00854               AR.gzipCompress = 0;
00855             #endif
00856             if ( tover > 0 ) {
00857                 ss = S->sPointer;
00858                 while ( ( t = *ss++ ) != 0 ) {
00859                     if ( *t ) S->TermsLeft++;
00860                 #ifdef WITHPTHREADS
00861                     if ( AS.MasterSort && ( fout == AR.outfile ) ) { PutToMaster(BHEAD t); }
00862                     else
00863                 #endif
00864                     if ( PutOut(BHEAD t,&position,fout,1) < 0 ) {
00865                         retval = -1; goto RetRetval;
00866                     }
00867                 }
00868             }
00869             #ifdef WITHPTHREADS
00870             if ( AS.MasterSort && ( fout == AR.outfile ) ) { PutToMaster(BHEAD 0); }
00871             else
00872         #endif
00873             if ( FlushOut(&position,fout,1) ) {
00874                 retval = -1; goto RetRetval;
00875             }
00876             #ifdef WITHZLIB
00877                 AR.gzipCompress = oldgzipCompress;
00878             }
00879             #endif
00880             #ifdef WITHPTHREADS
00881             if ( AS.MasterSort && ( fout == AR.outfile ) ) goto RetRetval;
00882             #endif
00883             #ifdef WITHMPI
00884             if ( PF.me != MASTER && PF.exprtodo < 0 ) goto RetRetval;
00885             #endif
00886             DIFPOS(oldpos,position,oldpos);
00887             S->SpaceLeft = BASEPOSITION(oldpos);
00888             WriteStats(&oldpos,STATSPOSTSORT,NOCHECKLOGTYPE);
00889             pp = oldpos;
00890             goto RetRetval;
00891         }
00892     }
00893     else if ( par == 1 && newout == 0 ) { AR.outfile = newout = AllocFileHandle(0,(char *)0); }
00894     sSpace++;
00895     lSpace = sSpace + (S->lFill - S->lBuffer) - (LONG)S->lPatch*(AM.MaxTer/sizeof(WORD));
00896     /* Note wrt MaxTer and lPatch: each patch starts with space for decompression */
00897     /* Not needed if only large buffer, but needed when using files (?) */
00898     SETBASEPOSITION(pp,lSpace);
00899     MULPOS(pp,sizeof(WORD));
00900     if ( S->file.handle >= 0 ) {
00901         ADD2POS(pp,S->fPatches[S->fPatchN]);
00902     }
00903     if ( S == AT.S0 ) {
00904         if ( S->lPatch > 0 || S->file.handle >= 0 ) {
00905             WriteStats(&pp,STATSSPLITMERGE,CHECKLOGTYPE);
00906         }
00907     }
00908     if ( par == 2 ) { AR.outfile = newout = AllocFileHandle(0,(char *)0); }
00909     if ( S->lPatch > 0 ) {
00910         if ( ( S->lPatch >= S->MaxPatches ) ||
00911             ( ( (WORD *)(((UBYTE *) (S->lFill + sSpace)) + 2*AM.MaxTer) ) >= S->lTop ) ) {
00912             /*
00913                 The large buffer is too full. Merge and write it
00914             */
00915             #ifdef GZIPDEBUG
00916                 MLOCK(ErrorMessageLock);
00917                 MesPrint("%w EndSort: lPatch = %d, MaxPatches = %d,lFill = %x, sSpace = %ld, MaxTer = %d,
00918 lTop = %x"
00919                     ,S->lPatch,S->MaxPatches,S->lFill,sSpace,AM.MaxTer/sizeof(WORD),S->lTop);
00920                 MUNLOCK(ErrorMessageLock);
00921             #endif
00922             if ( MergePatches(1) ) {
00923                 MLOCK(ErrorMessageLock);
00924                 MesCall("EndSort");
00925                 MUNLOCK(ErrorMessageLock);
00926                 retval = -1; goto RetRetval;
00927             }
00928             S->lPatch = 0;

```

```

00929         pp = S->SizeInFile[1];
00930         MULPOS(pp, sizeof(WORD));
00931 #ifndef WITHPTHREADS
00932         if ( S == AT.S0 )
00933 #endif
00934         {
00935             POSITION pppp;
00936             SETBASEPOSITION(pppp, 0);
00937             SeekFile(S->file.handle, &pppp, SEEK_CUR);
00938             SeekFile(S->file.handle, &pp, SEEK_END);
00939             SeekFile(S->file.handle, &pppp, SEEK_SET);
00940             WriteStats(&pp, STATSMERGETOFILE, CHECKLOGTYPE);
00941             UpdateMaxSize();
00942         }
00943     }
00944     else {
00945         S->Patches[S->lPatch++] = S->lFill;
00946         to = (WORD *)(((UBYTE *) (S->lFill)) + AM.MaxTer);
00947         if ( tover > 0 ) {
00948             ss = S->sPointer;
00949             while ( ( t = *ss++ ) != 0 ) {
00950                 j = *t;
00951                 if ( j < 0 ) j = t[1] + 2;
00952                 while ( --j >= 0 ) *to++ = *t++;
00953             }
00954         }
00955         *to++ = 0;
00956         S->lFill = to;
00957         if ( S->file.handle < 0 ) {
00958             if ( MergePatches(2) ) {
00959                 MLOCK(ErrorMessageLock);
00960                 MesCall("EndSort");
00961                 MUNLOCK(ErrorMessageLock);
00962                 retval = -1; goto RetRetVal;
00963             }
00964             if ( S == AT.S0 ) {
00965                 pp = S->SizeInFile[2];
00966                 MULPOS(pp, sizeof(WORD));
00967 #ifdef WITHPTHREADS
00968                 if ( AS.MasterSort && ( fout == AR.outfile ) ) goto RetRetVal;
00969 #endif
00970                 WriteStats(&pp, STATSPOSTSORT, NOCHECKLOGTYPE);
00971                 UpdateMaxSize();
00972             }
00973             else {
00974                 if ( par == 2 && newout->handle >= 0 ) {
00975                     POSITION zeropos;
00976                     PUTZERO(zeropos);
00977 #ifdef ALLLOCK
00978                     LOCK(newout->pthreadlock);
00979 #endif
00980                     SeekFile(newout->handle, &zeropos, SEEK_SET);
00981                     to = (WORD *)Malloc1(BASEPOSITION(newout->filesize)+sizeof(WORD)*2
00982                                     , "$-buffer reading");
00983                     if ( AN.tryterm > 0 ) AN.tryterm = 0;
00984                     if ( ( retval = ReadFile(newout->handle, (UBYTE
00985 *)to, BASEPOSITION(newout->filesize)) ) !=
00986                         BASEPOSITION(newout->filesize) ) {
00987                         MLOCK(ErrorMessageLock);
00988                         MesPrint("Error reading information for $ variable");
00989                         MUNLOCK(ErrorMessageLock);
00990                         M_free(to, "$-buffer reading");
00991                         retval = -1;
00992                     }
00993                     else {
00994                         *((WORD **)buffer) = to;
00995                         retval /= sizeof(WORD);
00996                     }
00997 #ifdef ALLLOCK
00998                     UNLOCK(newout->pthreadlock);
00999 #endif
01000                 }
01001                 else if ( newout->handle >= 0 ) {
01002                     We land here if par == 1 (function arg sort) and PutOut has created a file.
01003                     This means that the term is larger than SIOsize, which we ensure is at least
01004                     as large as MaxTermSize in setfile.c. Thus we know already that the term won't
01005                     fit.
01006                     /*
01007                     TooLarge:
01008                     MLOCK(ErrorMessageLock);
01009                     MesPrint("(1)Output should fit inside a single term. Increase MaxTermSize?");
01010                     MesCall("EndSort");
01011                     MUNLOCK(ErrorMessageLock);
01012                     retval = -1; goto RetRetVal;
01013                 }
01014             }
01015         }

```

```

01014         t = newout->PObuffer;
01015         if ( par == 2 ) {
01016             jj = newout->POfill - t;
01017             if ( AN.tryterm > 0 && ( (jj+2)*sizeof(WORD) < (size_t) (AM.MaxTer) ) ) {
01018                 to = TermMalloc("$-sort space");
01019             }
01020             else {
01021                 LONG allocsp = jj+2;
01022                 if ( allocsp < MINALLOC ) allocsp = MINALLOC;
01023                 allocsp = ((allocsp+7)/8)*8;
01024                 to = (WORD *)Malloc1(allocsp*sizeof(WORD), "$-sort space");
01025                 if ( AN.tryterm > 0 ) AN.tryterm = 0;
01026             }
01027             *((WORD **)buffer) = to;
01028             NCOPY(to,t,jj);
01029             /* we should not set retval here in this par==2 case, it being 0 has
meaning
01030                 after RetRetval, and it is set properly there. */
01031         }
01032         else {
01033             j = newout->POfill - t;
01034             to = buffer;
01035             if ( to >= AT.WorkSpace && to < AT.WorkTop && to+j > AT.WorkTop )
01036                 goto WorkspaceError;
01037             if ( j > AM.MaxTer ) {
01038                 MLOCK(ErrorMessageLock);
01039                 MesPrint("Encountered term of size: %d words.", j/(LONG)sizeof(WORD)
);
01040                 MUNLOCK(ErrorMessageLock);
01041                 goto TooLarge;
01042             }
01043             NCOPY(to,t,j);
01044             retval = to - buffer - 1;
01045         }
01046     }
01047 }
01048 goto RetRetval;
01049 }
01050 if ( MergePatches(1) ) { /* --> SortFile */
01051     MLOCK(ErrorMessageLock);
01052     MesCall("EndSort");
01053     MUNLOCK(ErrorMessageLock);
01054     retval = -1; goto RetRetval;
01055 }
01056 UpdateMaxSize();
01057 pp = S->SizeInFile[1];
01058 MULPOS(pp,sizeof(WORD));
01059 #ifndef WITHPTHREADS
01060     if ( S == AT.S0 )
01061 #endif
01062     {
01063         POSITION pppp;
01064         SETBASEPOSITION(pppp,0);
01065         SeekFile(S->file.handle,&pppp,SEEK_CUR);
01066         SeekFile(S->file.handle,&pp,SEEK_END);
01067         SeekFile(S->file.handle,&pppp,SEEK_SET);
01068         WriteStats(&pp,STATSMERGETOFILE,CHECKLOGTYPE);
01069     }
01070 #ifdef WITHERRORXXX
01071     if ( S != AT.S0 ) {
01072         /*
01073             This is wrong! We have sorted to the sort file.
01074             Things are not sitting in the output yet.
01075         */
01076         if ( newout->handle >= 0 ) goto TooLarge;
01077         t = newout->PObuffer;
01078         j = newout->POfill - t;
01079         to = buffer;
01080         if ( to >= AT.WorkSpace && to < AT.WorkTop && to+j > AT.WorkTop )
01081             goto WorkspaceError;
01082         if ( j > AM.MaxTer ) goto TooLarge;
01083         NCOPY(to,t,j);
01084         goto RetRetval;
01085     }
01086 #endif
01087 }
01088 }
01089 if ( S->file.handle >= 0 ) {
01090     #ifdef GZIPDEBUG
01091         MLOCK(ErrorMessageLock);
01092         MesPrint("%w EndSort: fPatchN = %d, lPatch = %d, position = %12p"
01093             ,S->fPatchN,S->lPatch,&(S->fPatches[S->fPatchN]));
01094         MUNLOCK(ErrorMessageLock);
01095     #endif
01096     if ( S->lPatch <= 0 ) {
01097         StageSort(&(S->file));
01098         position = S->fPatches[S->fPatchN];

```

```

01099         ss = S->sPointer;
01100         if ( *ss ) {
01101             *AR.CompressPointer = 0;
01102 #ifdef WITHZLIB
01103             if ( S == AT.S0 && AR.NoCompress == 0 && AR.gzipCompress > 0 )
01104                 S->fpcompressed[S->fPatchN] = 1;
01105             else
01106                 S->fpcompressed[S->fPatchN] = 0;
01107             SetupOutputGZIP(&(S->file));
01108 #endif
01109             while ( ( t = *ss++ ) != 0 ) {
01110                 if ( PutOut(BHEAD t,&position,&(S->file),1) < 0 ) {
01111                     retval = -1; goto RetRetval;
01112                 }
01113             }
01114             if ( FlushOut(&position,&(S->file),1) ) {
01115                 retval = -1; goto RetRetval;
01116             }
01117             ++(S->fPatchN);
01118             S->fPatches[S->fPatchN] = position;
01119             UpdateMaxSize();
01120 #ifdef GZIPDEBUG
01121             MLOCK(ErrorMessageLock);
01122             MesPrint("%w EndSort+: fPatchN = %d, lPatch = %d, position = %12p"
01123                 ,S->fPatchN,S->lPatch,&(S->fPatches[S->fPatchN]));
01124             MUNLOCK(ErrorMessageLock);
01125 #endif
01126         }
01127     }
01128     AR.Stage4Name = 0;
01129 #ifdef WITHPTHREADS
01130     if ( AS.MasterSort && AC.ThreadSortFileSynch ) {
01131         if ( S->file.handle >= 0 ) {
01132             SynchFile(S->file.handle);
01133         }
01134     }
01135 #endif
01136     UpdateMaxSize();
01137     if ( MergePatches(0) ) {
01138         MLOCK(ErrorMessageLock);
01139         MesCall("EndSort");
01140         MUNLOCK(ErrorMessageLock);
01141         retval = -1; goto RetRetval;
01142     }
01143     S->stage4 = 0;
01144 #ifdef WITHPTHREADS
01145     if ( AS.MasterSort && ( fout == AR.outfile ) ) goto RetRetval;
01146 #endif
01147     pp = S->SizeInFile[0];
01148     MULPOS(pp,sizeof(WORD));
01149     WriteStats(&pp,STATSPOSTSORT,NOCHECKLOGTYPE);
01150     UpdateMaxSize();
01151 }
01152 RetRetval:
01153
01154 #ifdef WITHMPI
01155     /* NOTE: PF_EndSort has been changed such that it sets S->TermsLeft. (TU 30 Jun 2011) */
01156     if ( AR.sLevel == 0 && (PF.me == MASTER || PF.exprtodo >= 0) ) {
01157         Expressions[AR.CurExpr].counter = S->TermsLeft;
01158         Expressions[AR.CurExpr].size = pp;
01159     }
01160 #else
01161     if ( AR.sLevel == 0 ) {
01162         Expressions[AR.CurExpr].counter = S->TermsLeft;
01163         Expressions[AR.CurExpr].size = pp;
01164     } /* if ( AR.sLevel == 0 ) */
01165 #endif
01166 /*:[25nov2003 mt]*/
01167     if ( S->file.handle >= 0 && ( par != 1 ) && ( par != 2 ) ) {
01168         /* sortfile is still open */
01169         UpdateMaxSize();
01170 #ifdef WITHZLIB
01171         ClearSortGZIP(&(S->file));
01172 #endif
01173         CloseFile(S->file.handle);
01174         S->file.handle = -1;
01175         remove(S->file.name);
01176 #ifdef GZIPDEBUG
01177         MLOCK(ErrorMessageLock);
01178         MesPrint("%wEndSort: sortfile %s removed",S->file.name);
01179         MUNLOCK(ErrorMessageLock);
01180 #endif
01181     }
01182     AR.outfile = oldoutfile;
01183     AR.sLevel--;
01184     if ( AR.sLevel >= 0 ) AT.SS = AN.FunSorts[AR.sLevel];
01185     if ( par == 1 ) {

```

```

01186         if ( retval < 0 ) {
01187             UpdateMaxSize();
01188             if ( newout ) {
01189                 DeAllocFileHandle(newout);
01190                 newout = 0;
01191             }
01192         }
01193     else if ( newout ) {
01194         if ( newout->handle >= 0 ) {
01195             MLOCK(ErrorMessageLock);
01196             MesPrint("(2)Output should fit inside a single term. Increase MaxTermSize?");
01197             MesCall("EndSort");
01198             MUNLOCK(ErrorMessageLock);
01199             Terminate(-1);
01200         }
01201         else if ( newout->POfill > newout->PObuffer ) {
01202             /*
01203              Here we have to copy the contents of the 'file' into
01204              the buffer. We assume that this buffer lies in the Workspace.
01205              Hence
01206             */
01207             j = newout->POfill-newout->PObuffer;
01208             if ( buffer >= AT.WorkSpace && buffer < AT.WorkTop && buffer+j > AT.WorkTop )
01209                 goto WorkspaceError;
01210             else {
01211                 to = buffer; t = newout->PObuffer;
01212                 while ( j-- > 0 ) *to++ = *t++;
01213             }
01214             UpdateMaxSize();
01215         }
01216         DeAllocFileHandle(newout);
01217         newout = 0;
01218     }
01219 }
01220 else if ( par == 2 ) {
01221     if ( newout ) {
01222         if ( retval == 0 ) {
01223             if ( newout->handle >= 0 ) {
01224                 /*
01225                  output resides at the moment in a file
01226                  Find the size, make a buffer, copy into the buffer and clean up.
01227                 */
01228                 POSITION zeropos;
01229                 PUTZERO(position);
01230 #ifdef ALLLOCK
01231                 LOCK(newout->pthreadslck);
01232 #endif
01233                 SeekFile(newout->handle,&position,SEEK_END);
01234                 PUTZERO(zeropos);
01235                 SeekFile(newout->handle,&zeropos,SEEK_SET);
01236                 to = (WORD *)Malloc1(BASEPOSITION(position)+sizeof(WORD)*3
01237                                     ,"$-buffer reading");
01238                 if ( AN.tryterm > 0 ) AN.tryterm = 0;
01239                 if ( ( retval = ReadFile(newout->handle,(UBYTE *)to,BASEPOSITION(position)) ) !=
01240                     BASEPOSITION(position) ) {
01241                     MLOCK(ErrorMessageLock);
01242                     MesPrint("Error reading information for $ variable");
01243                     MUNLOCK(ErrorMessageLock);
01244                     M_free(to,"$-buffer reading");
01245                     retval = -1;
01246                 }
01247                 else {
01248                     *((WORD **)buffer) = to;
01249                     retval /= sizeof(WORD);
01250                 }
01251 #ifdef ALLLOCK
01252                 UNLOCK(newout->pthreadslck);
01253 #endif
01254             }
01255         else {
01256             /*
01257              output resides in the cache buffer and the file was never opened
01258             */
01259             LONG wsiz = newout->POfill - newout->PObuffer;
01260             if ( AN.tryterm > 0 && ( (wsiz+2)*sizeof(WORD) < (size_t)(AM.MaxTer) ) ) {
01261                 to = TermMalloc("$-sort space");
01262             }
01263             else {
01264                 LONG allocsp = wsiz+2;
01265                 if ( allocsp < MINALLOC ) allocsp = MINALLOC;
01266                 allocsp = ((allocsp+7)/8)*8;
01267                 to = (WORD *)Malloc1(allocsp*sizeof(WORD),"$-buffer reading");
01268                 if ( AN.tryterm > 0 ) AN.tryterm = 0;
01269             }
01270             *((WORD **)buffer) = to; t = newout->PObuffer;
01271             retval = wsiz;
01272             NCOPY(to,t,wsiz);

```

```

01273         }
01274     }
01275     UpdateMaxSize();
01276     DeAllocFileHandle(newout);
01277     newout = 0;
01278 }
01279 }
01280 else {
01281     if ( newout ) {
01282         DeAllocFileHandle(newout);
01283         newout = 0;
01284     }
01285 }
01286 /*
01287 if ( AR.sLevel < 0 ) {
01288     MesPrint(" number of calls to compare was %l",numcompares);
01289 }
01290 */
01291 return(retval);
01292 WorkspaceError:
01293 MLOCK(ErrorMessageLock);
01294 MesWork();
01295 MesCall("EndSort");
01296 MUNLOCK(ErrorMessageLock);
01297 Terminate(-1);
01298 return(-1);
01299 }
01300
01301 /*
01302     #] EndSort :
01303     #[ PutIn :                LONG PutIn(handle,position,buffer,take,ncpat)
01304 */
01305 LONG PutIn(FILEHANDLE *file, POSITION *position, WORD *buffer, WORD **take, int npat)
01306 {
01307     LONG i, RetCode;
01308     WORD *from, *to;
01309 #ifndef WITHZLIB
01310     DUMMYUSE(ncpat);
01311 #endif
01312     from = buffer + ( file->POsize * sizeof(UBYTE) ) / sizeof(WORD);
01313     i = from - *take;
01314     if ( i*((LONG)(sizeof(WORD))) > AM.MaxTer ) {
01315         MLOCK(ErrorMessageLock);
01316         MesPrint("Problems in PutIn");
01317         MUNLOCK(ErrorMessageLock);
01318         Terminate(-1);
01319     }
01320     to = buffer;
01321     while ( --i >= 0 ) *--to = *--from;
01322     *take = to;
01323 #ifdef WITHZLIB
01324     if ( ( RetCode = FillInputGZIP(file,position,(UBYTE *)buffer
01325                                     ,file->POsize,ncpat) ) < 0 ) {
01326         MLOCK(ErrorMessageLock);
01327         MesPrint("PutIn: We have RetCode = %x while reading %x bytes",
01328                 RetCode,file->POsize);
01329         MUNLOCK(ErrorMessageLock);
01330         Terminate(-1);
01331     }
01332 #else
01333     #ifdef ALLLOCK
01334     LOCK(file->pthreadlock);
01335     #endif
01336     SeekFile(file->handle,position,SEEK_SET);
01337     if ( ( RetCode = ReadFile(file->handle,(UBYTE *)buffer,file->POsize) ) < 0 ) {
01338         #ifdef ALLLOCK
01339         UNLOCK(file->pthreadlock);
01340         #endif
01341         MLOCK(ErrorMessageLock);
01342         MesPrint("PutIn: We have RetCode = %x while reading %x bytes",
01343                 RetCode,file->POsize);
01344         MUNLOCK(ErrorMessageLock);
01345         Terminate(-1);
01346     }
01347     #ifdef ALLLOCK
01348     UNLOCK(file->pthreadlock);
01349     #endif
01350     #endif
01351     return(RetCode);
01352 }
01353 /*
01354     #] PutIn :
01355     #[ Sflush :                WORD Sflush(file)
01356 */
01357 WORD Sflush(FILEHANDLE *fi)
01358 {

```

```

01382     LONG size, RetCode;
01383 #ifdef WITHZLIB
01384     GETIDENTITY
01385     int dobracketindex = 0;
01386     if ( AR.sLevel <= 0 && Expressions[AR.CurExpr].newbracketinfo
01387         && ( fi == AR.outfile || fi == AR.hidefile ) ) dobracketindex = 1;
01388 #endif
01389     if ( fi->handle < 0 ) {
01390         if ( ( RetCode = CreateFile(fi->name) ) >= 0 ) {
01391 #ifdef GZIPDEBUG
01392             MLOCK(ErrorMessageLock);
01393             MesPrint("%w Sflush created scratch file %s",fi->name);
01394             MUNLOCK(ErrorMessageLock);
01395 #endif
01396             fi->handle = (WORD)RetCode;
01397             PUTZERO(fi->filesize);
01398             PUTZERO(fi->POposition);
01399         }
01400         else {
01401             MLOCK(ErrorMessageLock);
01402             MesPrint("Cannot create scratch file %s",fi->name);
01403             MUNLOCK(ErrorMessageLock);
01404             return(-1);
01405         }
01406     }
01407 #ifdef WITHZLIB
01408     if ( AT.SS == AT.S0 && !AR.NoCompress && AR.gzipCompress > 0
01409         && dobracketindex == 0 ) {
01410         if ( FlushOutputGZIP(fi) ) return(-1);
01411         fi->POfill = fi->PObuffer;
01412     }
01413     else
01414 #endif
01415     {
01416 #ifdef ALLLOCK
01417         LOCK(fi->pthreadlock);
01418 #endif
01419         size = (fi->POfill-fi->PObuffer)*sizeof(WORD);
01420         SeekFile(fi->handle,&(fi->POposition),SEEK_SET);
01421         if ( WriteFile(fi->handle,(UBYTE *) (fi->PObuffer),size) != size ) {
01422 #ifdef ALLLOCK
01423             UNLOCK(fi->pthreadlock);
01424 #endif
01425             MLOCK(ErrorMessageLock);
01426             MesPrint("Write error while finishing sort. Disk full?");
01427             MUNLOCK(ErrorMessageLock);
01428             return(-1);
01429         }
01430         ADDPOS(fi->filesize,size);
01431         ADDPOS(fi->POposition,size);
01432         fi->POfill = fi->PObuffer;
01433 #ifdef ALLLOCK
01434         UNLOCK(fi->pthreadlock);
01435 #endif
01436     }
01437     return(0);
01438 }
01439
01440 /*
01441     #] Sflush :
01442     #[ PutOut :                WORD PutOut(term,position,file,ncomp)
01443 */
01444 WORD PutOut(PHEAD WORD *term, POSITION *position, FILEHANDLE *fi, WORD ncomp)
01445 {
01446     GETBIDENTITY
01447     WORD i, *p, ret, *r, *rr, j, k, first;
01448     int dobracketindex = 0;
01449     LONG RetCode;
01450
01451     if ( AT.SS != AT.S0 ) {
01452 /*
01453         For this case no compression should be used
01454 */
01455         if ( ( i = *term ) <= 0 ) return(0);
01456         ret = i;
01457         ADDPOS(*position,i*sizeof(WORD));
01458         p = fi->POfill;
01459         do {
01460             if ( p >= fi->POstop ) {
01461                 if ( fi->handle < 0 ) {
01462                     if ( ( RetCode = CreateFile(fi->name) ) >= 0 ) {
01463 #ifdef GZIPDEBUG
01464                         MLOCK(ErrorMessageLock);
01465                         MesPrint("%w PutOut created sortfile %s",fi->name);
01466                         MUNLOCK(ErrorMessageLock);
01467 #endif
01468                         fi->handle = (WORD)RetCode;

```



```

01491             PUTZERO(fi->filesize);
01492             PUTZERO(fi->POposition);
01493  /*
01494             Should not be here anymore?
01495  #ifndef WITHZLIB
01496             fi->ziobuffer = 0;
01497  #endif
01498  */
01499             }
01500             else {
01501                 MLOCK(ErrorMessageLock);
01502                 MesPrint("Cannot create scratch file %s",fi->name);
01503                 MUNLOCK(ErrorMessageLock);
01504                 return(-1);
01505             }
01506         }
01507 #ifdef ALLLOCK
01508         LOCK(fi->pthreadlock);
01509 #endif
01510         if ( fi == AR.hidefile ) {
01511             LOCK(AS.inputslock);
01512         }
01513         SeekFile(fi->handle,&(fi->POposition),SEEK_SET);
01514         if ( ( RetCode = WriteFile(fi->handle,(UBYTE *) (fi->PObuffer),fi->POsize) ) !=
01515             fi->POsize ) {
01516             if ( fi == AR.hidefile ) {
01517                 UNLOCK(AS.inputslock);
01518             }
01519 #ifdef ALLLOCK
01520             UNLOCK(fi->pthreadlock);
01521 #endif
01522             MLOCK(ErrorMessageLock);
01523             MesPrint("Write error during sort. Disk full?");
01524             MesPrint("Attempt to write %l bytes on file %d at position %15p",
01525                 fi->POsize,fi->handle,&(fi->POposition));
01526             MesPrint("RetCode = %l, Buffer address = %l",RetCode,(LONG) (fi->PObuffer));
01527             MUNLOCK(ErrorMessageLock);
01528             return(-1);
01529         }
01530         ADDPOS(fi->filesize,fi->POsize);
01531         p = fi->PObuffer;
01532         ADDPOS(fi->POposition,fi->POsize);
01533         if ( fi == AR.hidefile ) {
01534             UNLOCK(AS.inputslock);
01535         }
01536 #ifdef ALLLOCK
01537         UNLOCK(fi->pthreadlock);
01538 #endif
01539         if ( AS.MasterSort && AC.ThreadSortFileSynch ) {
01540             if ( fi->handle >= 0 ) SynchFile(fi->handle);
01541         }
01542     #endif
01543     }
01544     *p++ = *term++;
01545 } while ( --i > 0 );
01546 fi->POfull = fi->POfill = p;
01547 return(ret);
01548 }
01549 if ( ( AP.PreDebug & DUMPOUTTERMS ) == DUMPOUTTERMS ) {
01550     MLOCK(ErrorMessageLock);
01551 #ifdef WITHPTHREADS
01552     sprintf((char *) (THRbuf),100,"PutOut(%d)",AT.identity);
01553     PrintTerm(term,(char *) (THRbuf));
01554 #else
01555     PrintTerm(term,"PutOut");
01556 #endif
01557     MesPrint("ncomp = %d, AR.NoCompress = %d, AR.sLevel = %d",ncomp,AR.NoCompress,AR.sLevel);
01558     MesPrint("File %s, position %p",fi->name,position);
01559     MUNLOCK(ErrorMessageLock);
01560 }
01561 if ( AR.sLevel <= 0 && Expressions[AR.CurExpr].newbracketinfo
01562     && ( fi == AR.outfile || fi == AR.hidefile ) ) dobracketindex = 1;
01563 r = rr = AR.CompressPointer;
01564 first = j = k = ret = 0;
01565 if ( ( i = *term ) != 0 ) {
01566     if ( i < 0 ) { /* Compressed term */
01567         i = term[l] + 2;
01568         if ( fi == AR.outfile || fi == AR.hidefile ) {
01569             MLOCK(ErrorMessageLock);
01570             MesPrint("Ran into precompressed term");
01571             MUNLOCK(ErrorMessageLock);
01572             Crash();
01573             return(-1);
01574         }
01575     }
01576 }

```

```

01577     else if ( !AR.NoCompress && ( ncomp > 0 ) && AR.sLevel <= 0 ) { /* Must compress */
01578         if ( dobracketindex ) {
01579             PutBracketInIndex(BHEAD term,position);
01580         }
01581         j = *r++ - 1;
01582         p = term + 1;
01583         i--;
01584         if ( AR.PolyFun ) {
01585             WORD *polystop, *sa;
01586             sa = p + i;
01587             sa -= ABS(sa[-1]);
01588             polystop = p;
01589             while ( polystop < sa && *polystop != AR.PolyFun ) {
01590                 polystop += polystop[1];
01591             }
01592             if ( polystop < sa ) {
01593                 if ( AR.PolyFunType == 2 ) polystop[2] &= ~MUSTCLEANPRF;
01594                 while ( i > 0 && j > 0 && *p == *r && p < polystop ) {
01595                     i--; j--; k--; p++; r++;
01596                 }
01597             }
01598             else {
01599                 while ( i > 0 && j > 0 && *p == *r && p < sa ) { i--; j--; k--; p++; r++; }
01600             }
01601         }
01602 #ifdef WITHFLOAT
01603         else if ( AC.DefaultPrecision ) {
01604             WORD *floatstop, *sa;
01605             sa = p + i;
01606             sa -= ABS(sa[-1]);
01607             floatstop = p;
01608             while ( floatstop < sa && *floatstop != FLOATFUN ) {
01609                 floatstop += floatstop[1];
01610             }
01611             if ( floatstop < sa ) {
01612                 while ( i > 0 && j > 0 && *p == *r && p < floatstop ) {
01613                     i--; j--; k--; p++; r++;
01614                 }
01615             }
01616             else {
01617                 while ( i > 0 && j > 0 && *p == *r && p < sa ) { i--; j--; k--; p++; r++; }
01618             }
01619         }
01620 #endif
01621         else {
01622             WORD *sa;
01623             sa = p + i;
01624             sa -= ABS(sa[-1]);
01625             while ( i > 0 && j > 0 && *p == *r && p < sa ) { i--; j--; k--; p++; r++; }
01626         }
01627         if ( k > -2 ) {
01628 nocompress:
01629             j = i = *term;
01630             k = 0;
01631             p = term;
01632             r = rr;
01633             NCOPY(r,p,j);
01634         }
01635         else {
01636             *rr = *term;
01637             term = p;
01638             j = i;
01639             NCOPY(r,p,j);
01640             j = i;
01641             i += 2;
01642             first = 2;
01643         }
01644         /* Sabotage getting into the coefficient next time */
01645         r[-(ABS(r[-1]))] = 0;
01646         if ( r >= AR.ComprTop ) {
01647             MLOCK(ErrorMessageLock);
01648             MesPrint("CompressSize of %10l is insufficient",AM.CompressSize);
01649             MUNLOCK(ErrorMessageLock);
01650             Crash();
01651             return(-1);
01652         }
01653     }
01654     else if ( !AR.NoCompress && ( ncomp < 0 ) && AR.sLevel <= 0 ) {
01655         /* No compress but put in compress buffer anyway */
01656         if ( dobracketindex ) {
01657             PutBracketInIndex(BHEAD term,position);
01658         }
01659         j = *r++ - 1;
01660         p = term + 1;
01661         i--;
01662         if ( AR.PolyFun ) {
01663             WORD *polystop, *sa;

```

```

01664         sa = p + i;
01665         sa -= ABS(sa[-1]);
01666         polystop = p;
01667         while ( polystop < sa && *polystop != AR.PolyFun ) {
01668             polystop += polystop[1];
01669         }
01670         if ( polystop < sa ) {
01671             if ( AR.PolyFunType == 2 ) polystop[2] &= ~MUSTCLEANPRF;
01672             while ( i > 0 && j > 0 && *p == *r && p < polystop ) {
01673                 i--; j--; k--; p++; r++;
01674             }
01675         }
01676         else {
01677             while ( i > 0 && j > 0 && *p == *r ) { i--; j--; k--; p++; r++; }
01678         }
01679     }
01680     else {
01681         while ( i > 0 && j > 0 && *p == *r ) { i--; j--; k--; p++; r++; }
01682     }
01683     goto nocompress;
01684 }
01685 else {
01686     if ( AR.PolyFunType == 2 ) {
01687         WORD *t, *tstop;
01688         tstop = term + *term;
01689         tstop -= ABS(tstop[-1]);
01690         t = term+1;
01691         while ( t < tstop ) {
01692             if ( *t == AR.PolyFun ) {
01693                 t[2] &= ~MUSTCLEANPRF;
01694             }
01695             t += t[1];
01696         }
01697     }
01698     if ( dobracketindex ) {
01699         PutBracketInIndex(BHEAD term,position);
01700     }
01701 }
01702 ret = i;
01703 ADDPOS(*position,i*sizeof(WORD));
01704 p = fi->POfill;
01705 do {
01706     if ( p >= fi->POstop ) {
01707 #ifdef WITHMPI /* [16mar1998 ar] */
01708         if ( PF.me != MASTER && AR.sLevel <= 0 && (fi == AR.outfile || fi == AR.hidefile) &&
PF.parallel && PF.exprtodo < 0 ) {
01709             PF_BUFFER *sbuf = PF.sbuf;
01710             sbuf->fill[sbuf->active] = fi->POstop;
01711             PF_IsendSbuf(MASTER,PF_BUFFER_MSGTAG);
01712             p = fi->PObuffer = fi->POfill = fi->POfull =
01713             sbuf->buff[sbuf->active];
01714             fi->POstop = sbuf->stop[sbuf->active];
01715         }
01716         else
01717 #endif /* WITHMPI [16mar1998 ar] */
01718         {
01719             if ( fi->handle < 0 ) {
01720                 if ( ( RetCode = CreateFile(fi->name) ) >= 0 ) {
01721 #ifdef GZIPDEBUG
01722                     MLOCK(ErrorMessageLock);
01723                     MesPrint("%w PutOut created sortfile %s",fi->name);
01724                     MUNLOCK(ErrorMessageLock);
01725 #endif
01726                     fi->handle = (WORD)RetCode;
01727                     PUTZERO(fi->filesize);
01728                     PUTZERO(fi->POposition);
01729                 /*
01730                     Should not be here?
01731                 */
01732 #ifdef WITHZLIB
01733                 fi->ziobuffer = 0;
01734 #endif
01735             }
01736             else {
01737                 MLOCK(ErrorMessageLock);
01738                 MesPrint("Cannot create scratch file %s",fi->name);
01739                 MUNLOCK(ErrorMessageLock);
01740                 return(-1);
01741             }
01742         }
01743 #ifdef WITHZLIB
01744         if ( !AR.NoCompress && ncomp > 0 && AR.zipCompress > 0
&& dobracketindex == 0 && fi->zsp != 0 ) {
01745             fi->POfill = p;
01746             if ( PutOutputGZIP(fi) ) return(-1);
01747             p = fi->PObuffer;
01748         }
01749 #endif

```

```

01750         else
01751     #endif
01752     {
01753     #ifdef ALLLOCK
01754         LOCK(fi->pthreadlock);
01755     #endif
01756         if ( fi == AR.hidefile ) {
01757             LOCK(AS.inputslock);
01758         }
01759         SeekFile(fi->handle,&(fi->POposition),SEEK_SET);
01760         if ( ( RetCode = WriteFile(fi->handle,(UBYTE *) (fi->PObuffer),fi->POsize) ) !=
fi->POsize ) {
01761             if ( fi == AR.hidefile ) {
01762                 UNLOCK(AS.inputslock);
01763             }
01764     #ifdef ALLLOCK
01765         UNLOCK(fi->pthreadlock);
01766     #endif
01767         MLOCK(ErrorMessageLock);
01768         MesPrint("Write error during sort. Disk full?");
01769         MesPrint("Attempt to write %l bytes on file %d at position %l5p",
01770             fi->POsize,fi->handle,&(fi->POposition));
01771         MesPrint("RetCode = %l, Buffer address = %l",RetCode,(LONG) (fi->PObuffer));
01772         MUNLOCK(ErrorMessageLock);
01773         return(-1);
01774     }
01775     ADDPOS(fi->filesize,fi->POsize);
01776     p = fi->PObuffer;
01777     ADDPOS(fi->POposition,fi->POsize);
01778     if ( fi == AR.hidefile ) {
01779         UNLOCK(AS.inputslock);
01780     }
01781     #ifdef ALLLOCK
01782     UNLOCK(fi->pthreadlock);
01783     #endif
01784     #ifdef WITHPTHREADS
01785         if ( AS.MasterSort && AC.ThreadSortFileSynch ) {
01786             if ( fi->handle >= 0 ) SynchFile(fi->handle);
01787         }
01788     #endif
01789     }
01790     }
01791     }
01792     if ( first ) {
01793         if ( first == 2 ) *p++ = k;
01794         else *p++ = j;
01795         first--;
01796     }
01797     else *p++ = *term++;
01798     /*
01799         if ( AP.DebugFlag ) {
01800             TalToLine((UWORD) (p[-1])); TokenToLine((UBYTE *) " ");
01801         }
01802     */
01803     } while ( --i > 0 );
01804     fi->POfull = fi->POfill = p;
01805     }
01806     /*
01807         if ( AP.DebugFlag ) {
01808             AO.OutSkip = 0;
01809             FiniLine();
01810         }
01811     */
01812     return(ret);
01813 }
01814 /*
01815 #] PutOut :
01816 #[ FlushOut :                WORD FlushOut(position,file,compr)
01817 */
01828 WORD FlushOut(POSITION *position, FILEHANDLE *fi, int compr)
01829 {
01830     GETIDENTITY
01831     LONG size, RetCode;
01832     int dobracketindex = 0;
01833     #ifndef WITHZLIB
01834     DUMMYUSE(compr);
01835     #endif
01836     if ( AR.sLevel <= 0 && Expressions[AR.CurExpr].newbracketinfo
01837         && ( fi == AR.outfile || fi == AR.hidefile ) ) dobracketindex = 1;
01838     #ifdef WITHMPI /* [16mar1998 ar] */
01839     if ( PF.me != MASTER && AR.sLevel <= 0 && (fi == AR.outfile || fi == AR.hidefile) && PF.parallel
01840         && PF.exprtoto < 0 ) {
01841         PF_BUFFER *sbuf = PF.sbuf;
01842         if ( fi->POfill >= fi->POstop ){
01843             sbuf->fill[sbuf->active] = fi->POstop;
01844             PF_ISendSbuf(MASTER,PF_BUFFER_MSGTAG);

```

```

01844         fi->POfull = fi->POfill = fi->PObuffer = sbuf->buff[sbuf->active];
01845         fi->POstop = sbuf->stop[sbuf->active];
01846     }
01847     *(fi->POfill)++ = 0;
01848     sbuf->fill[sbuf->active] = fi->POfill;
01849     PF_SendSbuf(MASTER,PF_ENDBUFFER_MSGTAG);
01850     fi->PObuffer = fi->POfill = fi->POfull = sbuf->buff[sbuf->active];
01851     fi->POstop = sbuf->stop[sbuf->active];
01852     return(0);
01853 }
01854 #endif /* WITHMPI [16mar1998 ar] */
01855 if ( fi->POfill >= fi->POstop ) {
01856     if ( fi->handle < 0 ) {
01857         if ( ( RetCode = CreateFile(fi->name) ) >= 0 ) {
01858             #ifdef GZIPDEBUG
01859                 MLOCK(ErrorMessageLock);
01860                 MesPrint("%w FlushOut created scratch file %s",fi->name);
01861                 MUNLOCK(ErrorMessageLock);
01862             #endif
01863             PUTZERO(fi->filesize);
01864             PUTZERO(fi->POposition);
01865             fi->handle = (WORD)RetCode;
01866             /*
01867              Should not be here?
01868             #ifdef WITHZLIB
01869                 fi->ziobuffer = 0;
01870             #endif
01871             */
01872         }
01873         else {
01874             MLOCK(ErrorMessageLock);
01875             MesPrint("Cannot create scratch file %s",fi->name);
01876             MUNLOCK(ErrorMessageLock);
01877             return(-1);
01878         }
01879     }
01880     #ifdef WITHZLIB
01881     if ( AT.SS == AT.S0 && !AR.NoCompress && AR.gzipCompress > 0
01882         && dobracketindex == 0 && ( compr > 0 ) && fi->zsp != 0 ) {
01883         if ( PutOutputGZIP(fi) ) return(-1);
01884         fi->POfill = fi->PObuffer;
01885     }
01886     else
01887     #endif
01888     {
01889         #ifdef ALLLOCK
01890             LOCK(fi->pthreadlock);
01891         #endif
01892         if ( fi == AR.hidefile ) {
01893             LOCK(AS.inputslock);
01894         }
01895         SeekFile(fi->handle,&(fi->POposition),SEEK_SET);
01896         if ( ( RetCode = WriteFile(fi->handle,(UBYTE *) (fi->PObuffer),fi->POsize) ) != fi->POsize )
01897         {
01898             #ifdef ALLLOCK
01899                 UNLOCK(fi->pthreadlock);
01900             #endif
01901             if ( fi == AR.hidefile ) {
01902                 UNLOCK(AS.inputslock);
01903             }
01904             MLOCK(ErrorMessageLock);
01905             MesPrint("Write error while sorting. Disk full?");
01906             MesPrint("Attempt to write %l bytes on file %d at position %l5p",
01907                 fi->POsize,fi->handle,&(fi->POposition));
01907             MesPrint("RetCode = %l, Buffer address = %l",RetCode,(LONG) (fi->PObuffer));
01908             MUNLOCK(ErrorMessageLock);
01909             return(-1);
01910         }
01911         ADDPOS(fi->filesize,fi->POsize);
01912         fi->POfill = fi->PObuffer;
01913         ADDPOS(fi->POposition,fi->POsize);
01914         if ( fi == AR.hidefile ) {
01915             UNLOCK(AS.inputslock);
01916         }
01917         #ifdef ALLLOCK
01918             UNLOCK(fi->pthreadlock);
01919         #endif
01920         #ifdef WITHPTHREADS
01921         if ( AS.MasterSort && AC.ThreadSortFileSynch && fi != AR.hidefile ) {
01922             if ( fi->handle >= 0 ) SynchFile(fi->handle);
01923         }
01924     #endif
01925     }
01926 }
01927 *(fi->POfill)++ = 0;
01928 fi->POfull = fi->POfill;
01929 /*

```

```

01930     {
01931         UBYTE OutBuf[140];
01932         if ( AP.DebugFlag ) {
01933             AO.OutFill = AO.OutputLine = OutBuf;
01934             AO.OutSkip = 3;
01935             FiniLine();
01936             TokenToLine((UBYTE *)"End of expression written");
01937             FiniLine();
01938         }
01939     }
01940 */
01941     size = (fi->POfill-fi->PObuffer)*sizeof(WORD);
01942     if ( fi->handle >= 0 ) {
01943 #ifdef WITHZLIB
01944         if ( AT.SS == AT.S0 && !AR.NoCompress && AR.gzipCompress > 0
01945             && dobracketindex == 0 && ( compr > 0 ) && fi->zsp != 0 ) {
01946             if ( FlushOutputGZIP(fi) ) return(-1);
01947             fi->POfill = fi->PObuffer;
01948         }
01949     else
01950 #endif
01951     {
01952 #ifdef ALLLOCK
01953         LOCK(fi->pthreadlock);
01954 #endif
01955         if ( fi == AR.hidefile ) {
01956             LOCK(AS.inputslock);
01957         }
01958         SeekFile(fi->handle,&(fi->POposition),SEEK_SET);
01959 /*
01960         MesPrint("FlushOut: writing %l bytes to position %l2p",size,&(fi->POposition));
01961 */
01962         if ( ( RetCode = WriteFile(fi->handle,(UBYTE *) (fi->PObuffer),size) ) != size ) {
01963 #ifdef ALLLOCK
01964             UNLOCK(fi->pthreadlock);
01965 #endif
01966             if ( fi == AR.hidefile ) {
01967                 UNLOCK(AS.inputslock);
01968             }
01969             MLOCK(ErrorMessageLock);
01970             MesPrint("Write error while finishing sorting. Disk full?");
01971             MesPrint("Attempt to write %l bytes on file %d at position %l5p",
01972                 size,fi->handle,&(fi->POposition));
01973             MesPrint("RetCode = %l, Buffer address = %l",RetCode,(LONG) (fi->PObuffer));
01974             MUNLOCK(ErrorMessageLock);
01975             return(-1);
01976         }
01977         ADDPOS(fi->filesize,size);
01978         ADDPOS(fi->POposition,size);
01979         fi->POfill = fi->PObuffer;
01980         if ( fi == AR.hidefile ) {
01981             UNLOCK(AS.inputslock);
01982         }
01983 #ifdef ALLLOCK
01984         UNLOCK(fi->pthreadlock);
01985 #endif
01986 #ifdef WITHPTHREADS
01987         if ( AS.MasterSort && AC.ThreadSortFileSynch ) {
01988             if ( fi->handle >= 0 ) SynchFile(fi->handle);
01989         }
01990 #endif
01991     }
01992     if ( dobracketindex ) {
01993         BRACKETINFO *b = Expressions[AR.CurExpr].newbracketinfo;
01994         if ( b->indexfill > 0 ) {
01995             DIFPOS(b->indexbuffer[b->indexfill-1].next,*position,Expressions[AR.CurExpr].onfile);
01996         }
01997     }
01998 }
01999 #ifdef WITHZLIB
02000     if ( AT.SS == AT.S0 && !AR.NoCompress && AR.gzipCompress > 0
02001         && dobracketindex == 0 && ( compr > 0 ) && fi->zsp != 0 ) {
02002         PUTZERO(*position);
02003         if ( fi->handle >= 0 ) {
02004 #ifdef ALLLOCK
02005             LOCK(fi->pthreadlock);
02006 #endif
02007             SeekFile(fi->handle,position,SEEK_END);
02008 #ifdef ALLLOCK
02009             UNLOCK(fi->pthreadlock);
02010 #endif
02011         }
02012     else {
02013         ADDPOS(*position,((UBYTE *) fi->POfill-(UBYTE *) fi->PObuffer));
02014     }
02015 }
02016     else

```

```

02017 #endif
02018 {
02019     ADDPOS(*position, sizeof(WORD));
02020 }
02021 return(0);
02022 }
02023
02024 /*
02025     #] FlushOut :
02026     #[ AddCoef :                WORD AddCoef(pterm1,pterm2)
02027 */
02042 WORD AddCoef(PHEAD WORD **ps1, WORD **ps2)
02043 {
02044     GETBIDENTITY
02045     SORTING *S = AT.SS;
02046     WORD *s1, *s2;
02047     WORD l1, l2, i;
02048     WORD OutLen, *t, j;
02049     UWORD *OutCoef;
02050 #ifndef WITHFLOAT
02051     if ( AT.SortFloatMode ) return(AddWithFloat(BHEAD ps1,ps2));
02052 #endif
02053     OutCoef = AN.SoScratC;
02054     s1 = *ps1; s2 = *ps2;
02055     GETCOEF(s1,l1);
02056     GETCOEF(s2,l2);
02057     if ( AddRat(BHEAD (UWORD *)s1,l1, (UWORD *)s2,l2,OutCoef,&OutLen) ) {
02058         MLOCK(ErrorMessageLock);
02059         MesCall("AddCoef");
02060         MUNLOCK(ErrorMessageLock);
02061         Terminate(-1);
02062     }
02063     if ( AN.ncmod != 0 ) {
02064         if ( ( AC.modmode & POSNEG ) != 0 ) {
02065             NormalModulus(OutCoef,&OutLen);
02066         }
02067         /*
02068             We had forgotten that this can also become smaller but the
02069             denominator isn't there. Correct in the other case
02070             17-may-2009 [JV]
02071         */
02072         j = ABS(OutLen); OutCoef[j] = 1;
02073         for ( i = 1; i < j; i++ ) OutCoef[j+i] = 0;
02074     }
02075     else if ( BigLong(OutCoef,OutLen, (UWORD *)AC.cmod,ABS(AN.ncmod)) >= 0 ) {
02076         SubPLon(OutCoef,OutLen, (UWORD *)AC.cmod,ABS(AN.ncmod),OutCoef,&OutLen);
02077         OutCoef[OutLen] = 1;
02078         for ( i = 1; i < OutLen; i++ ) OutCoef[OutLen+i] = 0;
02079     }
02080     if ( !OutLen ) { *ps1 = *ps2 = 0; return(0); }
02081     OutLen *= 2;
02082     if ( OutLen < 0 ) i = - ( --OutLen );
02083     else i = ++OutLen;
02084     if ( l1 < 0 ) l1 = -l1;
02085     l1 *= 2; l1++;
02086     if ( i <= l1 ) { /* Fits in 1 */
02087         l1 -= i;
02088         **ps1 -= l1;
02089         s2 = (WORD *)OutCoef;
02090         while ( --i > 0 ) *s1++ = *s2++;
02091         *s1++ = OutLen;
02092         while ( --l1 >= 0 ) *s1++ = 0;
02093         goto RegEnd;
02094     }
02095     if ( l2 < 0 ) l2 = -l2;
02096     l2 *= 2; l2++;
02097     if ( i <= l2 ) { /* Fits in 2 */
02098         l2 -= i;
02099         **ps2 -= l2;
02100         s1 = (WORD *)OutCoef;
02101         while ( --i > 0 ) *s2++ = *s1++;
02102         *s2++ = OutLen;
02103         while ( --l2 >= 0 ) *s2++ = 0;
02104         *ps1 = *ps2;
02105         goto RegEnd;
02106     }
02107
02108     /* Doesn't fit. Make a new term. */
02109
02110     t = s1;
02111     s1 = *ps1;
02112     j = *s1++ + i - l1; /* Space needed */
02113     if ( ( S->sFill + j ) >= S->sTop2 ) {
02114         GarbHand();
02115         s1 = *ps1;
02116         t = s1 + *s1 - 1;
02117         j = *s1++ + i - l1; /* Space needed */

```

```

02118         l1 = *t;
02119         if ( l1 < 0 ) l1 = - l1;
02120         t -= l1-1;
02121     }
02122     s2 = S->sFill;
02123     *s2++ = j;
02124     while ( s1 < t ) *s2++ = *s1++;
02125     s1 = (WORD *)OutCoef;
02126     while ( --i > 0 ) *s2++ = *s1++;
02127     *s2++ = OutLen;
02128     *ps1 = S->sFill;
02129     S->sFill = s2;
02130 RegEnd:
02131     *ps2 = 0;
02132     if ( **ps1 > AM.MaxTer/((LONG)(sizeof(WORD))) ) {
02133         MLOCK(ErrorMessageLock);
02134         MesPrint("Term too complex after addition in sort. MaxTermSize = %10l",
02135             AM.MaxTer/sizeof(WORD));
02136         MUNLOCK(ErrorMessageLock);
02137         Terminate(-1);
02138     }
02139     return(1);
02140 }
02141
02142 /*
02143     #] AddCoef :
02144     #[ AddPoly :                WORD AddPoly(pterm1,pterm2)
02145 */
02171 WORD AddPoly(PHEAD WORD **ps1, WORD **ps2)
02172 {
02173     GETBIDENTITY
02174     SORTING *S = AT.SS;
02175     WORD i;
02176     WORD *s1, *s2, *m, *w, *t, oldpw = S->PolyWise;
02177     s1 = *ps1 + S->PolyWise;
02178     s2 = *ps2 + S->PolyWise;
02179     w = AT.WorkPointer;
02180 /*
02181     Add here the two arguments. Is a straight merge.
02182 */
02183     if ( S->PolyFlag == 2 && AR.PolyFunExp != 2 && AR.PolyFunExp != 3 ) {
02184         WORD **oldSplitScratch = AN.SplitScratch;
02185         LONG oldSplitScratchSize = AN.SplitScratchSize;
02186         LONG oldInScratch = AN.InScratch;
02187         WORD oldtype = AR.SortType;
02188         if ( (WORD *)((UBYTE *)w + AM.MaxTer) >= AT.WorkTop ) {
02189             MLOCK(ErrorMessageLock);
02190             MesPrint("Program was adding polyratfun arguments");
02191             MesWork();
02192             MUNLOCK(ErrorMessageLock);
02193         }
02194         AR.SortType = SORTHIGHFIRST;
02195         S->PolyWise = 0;
02196         AN.SplitScratch = AN.SplitScratch1;
02197         AN.SplitScratchSize = AN.SplitScratchSize1;
02198         AN.InScratch = AN.InScratch1;
02199         poly_ratfun_add(BHEAD s1,s2);
02200         S->PolyWise = oldpw;
02201         AN.SplitScratch1 = AN.SplitScratch;
02202         AN.SplitScratchSize1 = AN.SplitScratchSize;
02203         AN.InScratch1 = AN.InScratch;
02204         AN.SplitScratch = oldSplitScratch;
02205         AN.SplitScratchSize = oldSplitScratchSize;
02206         AN.InScratch = oldInScratch;
02207         AT.WorkPointer = w;
02208         AR.SortType = oldtype;
02209         if ( w[1] <= FUNHEAD ||
02210             ( w[FUNHEAD] == -SNUMBER && w[FUNHEAD+1] == 0 ) ) {
02211             *ps1 = *ps2 = 0; return(0);
02212         }
02213     }
02214     else {
02215         if ( w + s1[1] + s2[1] + 12 + ARGHEAD >= AT.WorkTop ) {
02216             MLOCK(ErrorMessageLock);
02217             MesPrint("Program was adding polyfun arguments");
02218             MesWork();
02219             MUNLOCK(ErrorMessageLock);
02220         }
02221         AddArgs(BHEAD s1,s2,w);
02222     }
02223 /*
02224     Now we need to store the result in a convenient place.
02225 */
02226     if ( w[1] <= FUNHEAD ) { *ps1 = *ps2 = 0; return(0); }
02227     if ( w[1] <= s1[1] || w[1] <= s2[1] ) { /* Fits in place. */
02228         if ( w[1] > s1[1] ) {
02229             *ps1 = *ps2;

```



```

02230         s1 = s2;
02231     }
02232     t = s1 + s1[1];
02233     m = *ps1 + **ps1;
02234     i = w[1];
02235     NCOPY(s1,w,i);
02236     if ( s1 != t ) {
02237         while ( t < m ) *s1++ = *t++;
02238         **ps1 = WORDDIF(s1,(*ps1));
02239     }
02240     *ps2 = 0;
02241 }
02242 else { /* Make new term */
02243 #ifdef TESTGARB
02244     s2 = *ps2;
02245 #endif
02246     *ps2 = 0;
02247     if ( (S->sFill + (**ps1 + w[1] - s1[1])) >= S->sTop2 ) {
02248 #ifdef TESTGARB
02249         MesPrint("-----Garbage collection-----");
02250 #endif
02251         AT.WorkPointer += w[1];
02252         GarbHand();
02253         AT.WorkPointer = w;
02254         s1 = *ps1;
02255         if ( (S->sFill + (**ps1 + w[1] - s1[1])) >= S->sTop2 ) {
02256 #ifdef TESTGARB
02257             UBYTE OutBuf[140];
02258             MLOCK(ErrorMessageLock);
02259             AO.OutFill = AO.OutputLine = OutBuf;
02260             AO.OutSkip = 3;
02261             FiniLine();
02262             i = *s2;
02263             while ( --i >= 0 ) {
02264                 TalToLine((UWORD) (*s2++)); TokenToLine((UBYTE *) " ");
02265             }
02266             FiniLine();
02267             AO.OutFill = AO.OutputLine = OutBuf;
02268             AO.OutSkip = 3;
02269             FiniLine();
02270             s2 = *ps1;
02271             i = *s2;
02272             while ( --i >= 0 ) {
02273                 TalToLine((UWORD) (*s2++)); TokenToLine((UBYTE *) " ");
02274             }
02275             FiniLine();
02276             AO.OutFill = AO.OutputLine = OutBuf;
02277             AO.OutSkip = 3;
02278             FiniLine();
02279             s2 = w;
02280             i = w[1];
02281             while ( --i >= 0 ) {
02282                 TalToLine((UWORD) (*s2++)); TokenToLine((UBYTE *) " ");
02283             }
02284             FiniLine();
02285             if ( AR.sLevel > 0 ) {
02286                 MesPrint("Please increase SubSmallExtension setup parameter.");
02287             }
02288             else {
02289                 MesPrint("Please increase SmallExtension setup parameter.");
02290             }
02291             MUNLOCK(ErrorMessageLock);
02292 #else
02293             MLOCK(ErrorMessageLock);
02294             if ( AR.sLevel > 0 ) {
02295                 MesPrint("Please increase SubSmallExtension setup parameter.");
02296             }
02297             else {
02298                 MesPrint("Please increase SmallExtension setup parameter.");
02299             }
02300             MUNLOCK(ErrorMessageLock);
02301 #endif
02302         Terminate(-1);
02303     }
02304 }
02305 t = *ps1;
02306 s2 = S->sFill;
02307 m = s2;
02308 i = S->PolyWise;
02309 NCOPY(s2,t,i);
02310 i = w[1];
02311 NCOPY(s2,w,i);
02312 t = t + t[1];
02313 w = *ps1 + **ps1;
02314 while ( t < w ) *s2++ = *t++;
02315 *m = WORDDIF(s2,m);
02316 *ps1 = m;

```

```

02317         S->sFill = s2;
02318         if ( *m > AM.MaxTer/((LONG)sizeof(WORD)) ) {
02319             MLOCK(ErrorMessageLock);
02320             MesPrint("Term too complex after polynomial addition. MaxTermSize = %101",
02321                 AM.MaxTer/sizeof(WORD));
02322             MUNLOCK(ErrorMessageLock);
02323             Terminate(-1);
02324         }
02325     }
02326     return(1);
02327 }
02328
02329 /*
02330     #] AddPoly :
02331     #[ AddArgs :          void AddArgs(arg1,arg2,to)
02332 */
02333
02334 #define INSLNGTH(x)  w[1] = FUNHEAD+ARGHEAD+x; w[FUNHEAD] = ARGHEAD+x;
02335
02336 void AddArgs(PHEAD WORD *s1, WORD *s2, WORD *m)
02337 {
02338     GETBIDENTITY
02339     WORD i1, i2;
02340     WORD *w = m, *mm, *t, *t1, *t2, *tstop1, *tstop2;
02341     WORD tempterm[8+FUNHEAD];
02342
02343     *m++ = AR.PolyFun; *m++ = 0; FILLFUN(m)
02344     *m++ = 0; *m++ = 0; FILLARG(m)
02345     if ( s1[FUNHEAD] < 0 || s2[FUNHEAD] < 0 ) {
02346         if ( s1[FUNHEAD] < 0 ) {
02347             if ( s2[FUNHEAD] < 0 ) { /* Both are special */
02348                 if ( s1[FUNHEAD] <= -FUNCTION ) {
02349                     if ( s2[FUNHEAD] == s1[FUNHEAD] ) {
02350                         *m++ = 4+FUNHEAD; *m++ = -s1[FUNHEAD]; *m++ = FUNHEAD;
02351                         FILLFUN(m)
02352                         *m++ = 2; *m++ = 1; *m++ = 3;
02353                         INSLNGTH(4+FUNHEAD)
02354                     }
02355                 } else if ( s2[FUNHEAD] <= -FUNCTION ) {
02356                     i1 = functions[-FUNCTION-s1[FUNHEAD]].commute != 0;
02357                     i2 = functions[-FUNCTION-s2[FUNHEAD]].commute != 0;
02358                     if ( ( !i1 && i2 ) || ( i1 == i2 && i1 > i2 ) ) {
02359                         i1 = s2[FUNHEAD];
02360                         s2[FUNHEAD] = s1[FUNHEAD];
02361                         s1[FUNHEAD] = i1;
02362                     }
02363                     *m++ = 4+FUNHEAD; *m++ = -s1[FUNHEAD]; *m++ = FUNHEAD;
02364                     FILLFUN(m)
02365                     *m++ = 1; *m++ = 1; *m++ = 3;
02366                     *m++ = 4+FUNHEAD; *m++ = -s2[FUNHEAD]; *m++ = FUNHEAD;
02367                     FILLFUN(m)
02368                     *m++ = 1; *m++ = 1; *m++ = 3;
02369                     INSLNGTH(8+2*FUNHEAD)
02370                 }
02371             } else if ( s2[FUNHEAD] == -SYMBOL ) {
02372                 *m++ = 8; *m++ = SYMBOL; *m++ = 4; *m++ = s2[FUNHEAD+1]; *m++ = 1;
02373                 *m++ = 1; *m++ = 1; *m++ = 3;
02374                 *m++ = 4+FUNHEAD; *m++ = -s1[FUNHEAD]; *m++ = FUNHEAD;
02375                 FILLFUN(m)
02376                 *m++ = 1; *m++ = 1; *m++ = 3;
02377                 INSLNGTH(12+FUNHEAD)
02378             }
02379         } else { /* number */
02380             *m++ = 4;
02381             *m++ = ABS(s2[FUNHEAD+1]); *m++ = 1; *m++ = s2[FUNHEAD+1] < 0 ? -3: 3;
02382             *m++ = 4+FUNHEAD; *m++ = -s1[FUNHEAD]; *m++ = FUNHEAD;
02383             FILLFUN(m)
02384             *m++ = 1; *m++ = 1; *m++ = 3;
02385             INSLNGTH(8+FUNHEAD)
02386         }
02387     }
02388     else if ( s1[FUNHEAD] == -SYMBOL ) {
02389         if ( s2[FUNHEAD] == s1[FUNHEAD] ) {
02390             if ( s1[FUNHEAD+1] == s2[FUNHEAD+1] ) {
02391                 *m++ = 8; *m++ = SYMBOL; *m++ = 4; *m++ = s1[FUNHEAD+1];
02392                 *m++ = 1; *m++ = 2; *m++ = 1; *m++ = 3;
02393                 INSLNGTH(8)
02394             }
02395         } else {
02396             if ( s1[FUNHEAD+1] > s2[FUNHEAD+1] )
02397                 { i1 = s2[FUNHEAD+1]; i2 = s1[FUNHEAD+1]; }
02398             else { i1 = s1[FUNHEAD+1]; i2 = s2[FUNHEAD+1]; }
02399             *m++ = 8; *m++ = SYMBOL; *m++ = 4; *m++ = i1;
02400             *m++ = 1; *m++ = 1; *m++ = 1; *m++ = 3;
02401             *m++ = 8; *m++ = SYMBOL; *m++ = 4; *m++ = i2;
02402             *m++ = 1; *m++ = 1; *m++ = 1; *m++ = 3;
02403             INSLNGTH(16)
02404         }
02405     }
02406 }

```

```

02411     }
02412 }
02413 else if ( s2[FUNHEAD] <= -FUNCTION ) {
02414     *m++ = 8; *m++ = SYMBOL; *m++ = 4; *m++ = s1[FUNHEAD+1]; *m++ = 1;
02415     *m++ = 1; *m++ = 1; *m++ = 3;
02416     *m++ = 4+FUNHEAD; *m++ = -s2[FUNHEAD]; *m++ = FUNHEAD;
02417     FILLFUN(m)
02418     *m++ = 1; *m++ = 1; *m++ = 3;
02419     INSLNGTH(12+FUNHEAD)
02420 }
02421 else {
02422     *m++ = 4;
02423     *m++ = ABS(s2[FUNHEAD+1]); *m++ = 1; *m++ = s2[FUNHEAD+1] < 0 ? -3: 3;
02424     *m++ = 8; *m++ = SYMBOL; *m++ = 4; *m++ = s1[FUNHEAD+1]; *m++ = 1;
02425     *m++ = 1; *m++ = 1; *m++ = 3;
02426     INSLNGTH(12)
02427 }
02428 }
02429 else { /* Must be -SNUMBER! */
02430     if ( s2[FUNHEAD] <= -FUNCTION ) {
02431         *m++ = 4;
02432         *m++ = ABS(s1[FUNHEAD+1]); *m++ = 1; *m++ = s1[FUNHEAD+1] < 0 ? -3: 3;
02433         *m++ = 4+FUNHEAD; *m++ = -s2[FUNHEAD]; *m++ = FUNHEAD;
02434         FILLFUN(m)
02435         *m++ = 1; *m++ = 1; *m++ = 3;
02436         INSLNGTH(8+FUNHEAD)
02437     }
02438     else if ( s2[FUNHEAD] == -SYMBOL ) {
02439         *m++ = 4;
02440         *m++ = ABS(s1[FUNHEAD+1]); *m++ = 1; *m++ = s1[FUNHEAD+1] < 0 ? -3: 3;
02441         *m++ = 8; *m++ = SYMBOL; *m++ = 4; *m++ = s2[FUNHEAD+1]; *m++ = 1;
02442         *m++ = 1; *m++ = 1; *m++ = 3;
02443         INSLNGTH(12)
02444     }
02445     else { /* Both are numbers. add. */
02446         LONG x1;
02447         x1 = (LONG)s1[FUNHEAD+1] + (LONG)s2[FUNHEAD+1];
02448         if ( x1 < 0 ) { i1 = (WORD)(-x1); i2 = -3; }
02449         else { i1 = (WORD)x1; i2 = 3; }
02450         if ( x1 && AN.ncmod != 0 ) {
02451             m[0] = 4;
02452             m[1] = i1;
02453             m[2] = 1;
02454             m[3] = i2;
02455             if ( Modulus(m) ) Terminate(-1);
02456             if ( *m == 0 ) w[1] = 0;
02457             else {
02458                 if ( *m == 4 && ( m[1] & MAXPOSITIVE ) == m[1]
02459                     && m[3] == 3 ) {
02460                     i1 = m[1];
02461                     m -= ARGHEAD;
02462                     *m++ = -SNUMBER;
02463                     *m++ = i1;
02464                     INSLNGTH(4)
02465                 }
02466                 else {
02467                     INSLNGTH(*m)
02468                     m += *m;
02469                 }
02470             }
02471         }
02472     }
02473     else {
02474         if ( x1 == 0 ) {
02475             w[1] = FUNHEAD;
02476         }
02477         else if ( ( i1 & MAXPOSITIVE ) == i1 ) {
02478             m -= ARGHEAD;
02479             *m++ = -SNUMBER;
02480             *m++ = (WORD)x1;
02481             w[1] = FUNHEAD+2;
02482         }
02483         else {
02484             *m++ = 4; *m++ = i1; *m++ = 1; *m++ = i2;
02485             INSLNGTH(4)
02486         }
02487     }
02488 }
02489 }
02490 else { /* Only s1 is special */
02491     slonly:
02492     /*
02493     Compose a term in `tempterm'
02494     */
02495     t = tempterm;
02496     if ( s1[FUNHEAD] <= -FUNCTION ) {
02497         *t++ = 4+FUNHEAD; *t++ = -s1[FUNHEAD]; *t++ = FUNHEAD;

```

```

02498             FILLFUN(t)
02499             *t++ = 1; *t++ = 1; *t++ = 3;
02500         }
02501         else if ( s1[FUNHEAD] == -SYMBOL ) {
02502             *t++ = 8; *t++ = SYMBOL; *t++ = 4;
02503             *t++ = s1[FUNHEAD+1]; *t++ = 1;
02504             *t++ = 1; *t++ = 1; *t++ = 3;
02505         }
02506         else {
02507             *t++ = 4; *t++ = ABS(s1[FUNHEAD+1]);
02508             *t++ = 1; *t++ = s1[FUNHEAD+1] < 0 ? -3: 3;
02509         }
02510         tstop1 = t;
02511         s1 = tempterm;
02512         goto twogen;
02513     }
02514 }
02515 else { /* Only s2 is special */
02516     t = s1;
02517     s1 = s2;
02518     s2 = t;
02519     goto slonly;
02520 }
02521 }
02522 else {
02523     int oldPolyFlag;
02524     tstop1 = s1 + s1[1];
02525     s1 += FUNHEAD+ARGHEAD;
02526 twogen:
02527     tstop2 = s2 + s2[1];
02528     s2 += FUNHEAD+ARGHEAD;
02529 /*
02530     Now we should merge the expressions in s1 and s2 into m.
02531 */
02532     oldPolyFlag = AT.SS->PolyFlag;
02533     AT.SS->PolyFlag = 0;
02534     while ( s1 < tstop1 && s2 < tstop2 ) {
02535         i1 = CompareTerms(BHEAD s1,s2, (WORD) (-1));
02536         if ( i1 > 0 ) {
02537             i2 = *s1;
02538             NCOPY(m,s1,i2);
02539         }
02540         else if ( i1 < 0 ) {
02541             i2 = *s2;
02542             NCOPY(m,s2,i2);
02543         }
02544         else { /* Coefficients should be added. */
02545             WORD i;
02546             t = s1+*s1;
02547             i1 = t[-1];
02548             i2 = *s1 - ABS(i1);
02549             t2 = s2 + i2;
02550             s2 += *s2;
02551             mm = m;
02552             NCOPY(m,s1,i2);
02553             t1 = s1;
02554             s1 = t;
02555             i2 = s2[-1];
02556 /*
02557             t1,i1 is the first coefficient
02558             t2,i2 is the second coefficient
02559             It should be placed at m,i1
02560 */
02561             i1 = REDLENG(i1);
02562             i2 = REDLENG(i2);
02563             if ( AddRat(BHEAD (UWORD *)t1,i1,(UWORD *)t2,i2,(UWORD *)m,&i) ) {
02564                 MLOCK(ErrorMessageLock);
02565                 MesPrint("Addition of coefficients of PolyFun");
02566                 MUNLOCK(ErrorMessageLock);
02567                 Terminate(-1);
02568             }
02569             if ( i == 0 ) {
02570                 m = mm;
02571             }
02572             else {
02573                 i1 = INCLENG(i);
02574                 m += ABS(i1);
02575                 m[-1] = i1;
02576                 *mm = WORDDIFF(m,mm);
02577                 if ( AN.ncmod != 0 ) {
02578                     if ( Modulus(mm) ) Terminate(-1);
02579                     if ( !*mm ) m = mm;
02580                     else m = mm + *mm;
02581                 }
02582             }
02583         }
02584     }

```

```

02585     while ( s1 < tstop1 ) *m++ = *s1++;
02586     while ( s2 < tstop2 ) *m++ = *s2++;
02587     w[1] = WORDDIF(m,w);
02588     w[FUNHEAD] = w[1] - FUNHEAD;
02589     if ( ToFast(w+FUNHEAD,w+FUNHEAD) ) {
02590         if ( w[FUNHEAD] <= -FUNCTION ) w[1] = FUNHEAD+1;
02591         else w[1] = FUNHEAD+2;
02592         if ( w[FUNHEAD] == -SNUMBER && w[FUNHEAD+1] == 0 ) w[1] = FUNHEAD;
02593     }
02594     /* AT.SS->PolyFlag = AR.PolyFunType;*/
02595     AT.SS->PolyFlag = oldPolyFlag;
02596 }
02597 }
02598
02599 /*
02600     #] AddArgs :
02601     #[ Compare1 :                WORD Compare1(term1,term2,level)
02602 */
02636 WORD Compare1(PHEAD WORD *term1, WORD *term2, WORD level)
02637 {
02638     SORTING *S = AT.SS;
02639     WORD *stopper1, *stopper2, *t2;
02640     WORD *s1, *s2, *t1;
02641     WORD *stopex1, *stopex2;
02642     WORD c1, c2;
02643     WORD prevorder;
02644     WORD count = -1, localPoly, polyhit = -1;
02645
02646     if ( AR.sLevel == 0 ) {
02647         numcompares++;
02648     }
02649
02650     if ( S->PolyFlag ) {
02651         /*
02652             if ( S->PolyWise != 0 ) {
02653                 MLOCK(ErrorMessageLock);
02654                 MesPrint("S->PolyWise is not zero!!!!");
02655                 MUNLOCK(ErrorMessageLock);
02656             }
02657         */
02658         count = 0; localPoly = 1; S->PolyWise = polyhit = 0;
02659         S->PolyFlag = AR.PolyFunType;
02660         if ( AR.PolyFunType == 2 &&
02661             ( AR.PolyFunExp == 2 || AR.PolyFunExp == 3 ) ) S->PolyFlag = 1;
02662     }
02663     else { localPoly = 0; }
02664     #ifdef WITHFLOAT
02665     AT.SortFloatMode = 0;
02666     #endif
02667     prevorder = 0;
02668     GETSTOP(term1,s1);
02669     stopper1 = s1;
02670     GETSTOP(term2,stopper2);
02671     t1 = term1 + 1;
02672     t2 = term2 + 1;
02673     while ( t1 < stopper1 && t2 < stopper2 ) {
02674         if ( *t1 != *t2 ) {
02675             if ( *t1 == HAAKJE ) return(PREV(-1));
02676             if ( *t2 == HAAKJE ) return(PREV(1));
02677             if ( *t1 >= (FUNCTION-1) ) {
02678                 if ( *t2 < (FUNCTION-1) ) return(PREV(-1));
02679                 if ( *t1 < FUNCTION && *t2 < FUNCTION ) return(PREV(*t2-*t1));
02680                 if ( *t1 < FUNCTION ) return(PREV(1));
02681                 if ( *t2 < FUNCTION ) return(PREV(-1));
02682                 c1 = functions[*t1-FUNCTION].commute;
02683                 c2 = functions[*t2-FUNCTION].commute;
02684                 if ( !c1 ) {
02685                     if ( c2 ) return(PREV(1));
02686                     else return(PREV(*t2-*t1));
02687                 }
02688                 else {
02689                     if ( !c2 ) return(PREV(-1));
02690                     else return(PREV(*t2-*t1));
02691                 }
02692             }
02693             else return(PREV(*t2-*t1));
02694         }
02695         s1 = t1 + 2;
02696         s2 = t2 + 2;
02697         c1 = *t1;
02698         t1 += t1[1];
02699         t2 += t2[1];
02700         if ( localPoly && c1 < FUNCTION ) {
02701             polyhit = 1;
02702         }
02703         if ( c1 <= (FUNCTION-1)
02704             || ( c1 >= FUNCTION && functions[c1-FUNCTION].spec > 0 ) ) {

```

```

02705         if ( c1 == SYMBOL ) {
02706             if ( *s1 == FACTORSYMBOL && *s2 == FACTORSYMBOL
02707                 && s1[-1] == 4 && s2[-1] == 4
02708                 && ( ( t1 < stopper1 && *t1 == HAAKJE )
02709                     || ( t1 == stopper1 && AT.fromindex ) ) ) {
02710                 /*
02711                     We have to be very careful with the criteria here, because
02712                     Compare1 is called both in the regular sorting and by the
02713                     routine that makes the bracket index. In the last case
02714                     there is no HAAKJE subterm.
02715                 */
02716                 if ( s1[1] != s2[1] ) return(s2[1]-s1[1]);
02717                 s1 += 2; s2 += 2;
02718             }
02719             else if ( AR.SortType >= SORTPOWERFIRST ) {
02720                 WORD i1 = 0, *r1;
02721                 r1 = s1;
02722                 while ( s1 < t1 ) { i1 += s1[1]; s1 += 2; }
02723                 s1 = r1; r1 = s2;
02724                 while ( s2 < t2 ) { i1 -= s2[1]; s2 += 2; }
02725                 s2 = r1;
02726                 if ( i1 ) {
02727                     if ( AR.SortType >= SORTANTIPOWER ) i1 = -i1;
02728                     return(PREV(i1));
02729                 }
02730             }
02731             while ( s1 < t1 ) {
02732                 if ( s2 >= t2 ) {
02733                     /*
02734                         return(PREV(1)); */
02735                     if ( AR.SortType==SORTLOWFIRST ) {
02736                         return(PREV((s1[1]>0?-1:1)));
02737                     }
02738                     else {
02739                         return(PREV((s1[1]<0?-1:1)));
02740                     }
02741                 }
02742                 if ( *s1 != *s2 ) {
02743                     return(PREV(*s2-*s1)); /*
02744                     if ( AR.SortType==SORTLOWFIRST ) {
02745                         if ( *s1 < *s2 ) {
02746                             return(PREV((s1[1]<0?1:-1)));
02747                         }
02748                         else {
02749                             return(PREV((s2[1]<0?-1:1)));
02750                         }
02751                     }
02752                     else {
02753                         if ( *s1 < *s2 ) {
02754                             return(PREV((s1[1]<0?-1:1)));
02755                         }
02756                         else {
02757                             return(PREV((s2[1]<0?1:-1)));
02758                         }
02759                     }
02760                     s1++; s2++;
02761                     if ( *s1 != *s2 ) return(
02762                         PREV((AR.SortType==SORTLOWFIRST?*s2-*s1:*s1-*s2)));
02763                     s1++; s2++;
02764                 }
02765                 if ( s2 < t2 ) {
02766                     /*
02767                         return(PREV(-1)); */
02768                     if ( AR.SortType==SORTLOWFIRST ) {
02769                         return(PREV((s2[1]<0?-1:1)));
02770                     }
02771                     else {
02772                         return(PREV((s2[1]<0?1:-1)));
02773                     }
02774                 }
02775             }
02776             else if ( c1 == DOTPRODUCT ) {
02777                 if ( AR.SortType >= SORTPOWERFIRST ) {
02778                     WORD i1 = 0, *r1;
02779                     r1 = s1;
02780                     while ( s1 < t1 ) { i1 += s1[2]; s1 += 3; }
02781                     s1 = r1; r1 = s2;
02782                     while ( s2 < t2 ) { i1 -= s2[2]; s2 += 3; }
02783                     s2 = r1;
02784                     if ( i1 ) {
02785                         if ( AR.SortType >= SORTANTIPOWER ) i1 = -i1;
02786                         return(PREV(i1));
02787                     }
02788                 }
02789                 while ( s1 < t1 ) {
02790                     if ( s2 >= t2 ) return(PREV(1));
02791                     if ( *s1 != *s2 ) return(PREV(*s2-*s1));
02792                     s1++; s2++;

```

```

02792         if ( *s1 != *s2 ) return(PREV(*s2-*s1));
02793         s1++; s2++;
02794         if ( *s1 != *s2 ) return(
02795             PREV((AR.SortType==SORTLOWFIRST?*s2-*s1:*s1-*s2)));
02796         s1++; s2++;
02797     }
02798     if ( s2 < t2 ) return(PREV(-1));
02799 }
02800 else {
02801     while ( s1 < t1 ) {
02802         if ( s2 >= t2 ) return(PREV(1));
02803         if ( *s1 != *s2 ) return(PREV(*s2-*s1));
02804         s1++; s2++;
02805     }
02806     if ( s2 < t2 ) return(PREV(-1));
02807 }
02808 }
02809 else {
02810     #if FUNHEAD != 2
02811         s1 += FUNHEAD-2;
02812         s2 += FUNHEAD-2;
02813     #endif
02814     if ( localPoly && c1 == AR.PolyFun ) {
02815         if ( count == 0 ) {
02816             if ( S->PolyFlag == 1 ) {
02817                 WORD i1, i2;
02818                 if ( *s1 > 0 ) i1 = *s1;
02819                 else if ( *s1 <= -FUNCTION ) i1 = 1;
02820                 else i1 = 2;
02821                 if ( *s2 > 0 ) i2 = *s2;
02822                 else if ( *s2 <= -FUNCTION ) i2 = 1;
02823                 else i2 = 2;
02824                 if ( s1+i1 == t1 && s2+i2 == t2 ) { /* This is the stuff */
02825                     /*
02826                         Test for scalar nature
02827                     */
02828                     if ( !polyhit ) {
02829                         WORD *u1, *u2, *ustop;
02830                         if ( *s1 < 0 ) {
02831                             if ( *s1 != -SNUMBER && *s1 != -SYMBOL && *s1 > -FUNCTION )
02832                                 goto NoPoly;
02833                         }
02834                         else {
02835                             u1 = s1 + ARGHEAD;
02836                             while ( u1 < t1 ) {
02837                                 u2 = u1 + *u1;
02838                                 ustop = u2 - ABS(u2[-1]);
02839                                 u1++;
02840                                 while ( u1 < ustop ) {
02841                                     if ( *u1 == INDEX ) goto NoPoly;
02842                                     u1 += u1[1];
02843                                 }
02844                                 u1 = u2;
02845                             }
02846                         }
02847                         if ( *s2 < 0 ) {
02848                             if ( *s2 != -SNUMBER && *s2 != -SYMBOL && *s2 > -FUNCTION )
02849                                 goto NoPoly;
02850                         }
02851                         else {
02852                             u1 = s2 + ARGHEAD;
02853                             while ( u1 < t2 ) {
02854                                 u2 = u1 + *u1;
02855                                 ustop = u2 - ABS(u2[-1]);
02856                                 u1++;
02857                                 while ( u1 < ustop ) {
02858                                     if ( *u1 == INDEX ) goto NoPoly;
02859                                     u1 += u1[1];
02860                                 }
02861                                 u1 = u2;
02862                             }
02863                         }
02864                     }
02865                     S->PolyWise = WORDDIFF(s1,term1);
02866                     S->PolyWise -= FUNHEAD;
02867                     count = 1;
02868                     continue;
02869                 }
02870             }
02871             NoPoly:
02872             S->PolyWise = localPoly = 0;
02873         }
02874     }
02875     else if ( AR.PolyFunType == 2 ) {
02876         WORD i1, i2, i1a, i2a;
02877         if ( *s1 > 0 ) i1 = *s1;
02878         else if ( *s1 <= -FUNCTION ) i1 = 1;

```

```

02879         else i1 = 2;
02880         if ( *s2 > 0 ) i2 = *s2;
02881         else if ( *s2 <= -FUNCTION ) i2 = 1;
02882         else i2 = 2;
02883         if ( s1[i1] > 0 ) i1a = s1[i1];
02884         else if ( s1[i1] <= -FUNCTION ) i1a = 1;
02885         else i1a = 2;
02886         if ( s2[i2] > 0 ) i2a = s2[i2];
02887         else if ( s2[i2] <= -FUNCTION ) i2a = 1;
02888         else i2a = 2;
02889         if ( s1+i1+i1a == t1 && s2+i2+i2a == t2 ) { /* This is the stuff */
02890 /*
02891         Test for scalar nature
02892 */
02893         if ( !polyhit ) {
02894             WORD *u1, *u2, *ustop;
02895             if ( *s1 < 0 ) {
02896                 if ( *s1 != -SNUMBER && *s1 != -SYMBOL && *s1 > -FUNCTION )
02897                     goto NoPoly;
02898             }
02899             else {
02900                 u1 = s1 + ARGHEAD;
02901                 while ( u1 < s1+i1 ) {
02902                     u2 = u1 + *u1;
02903                     ustop = u2 - ABS(u2[-1]);
02904                     u1++;
02905                     while ( u1 < ustop ) {
02906                         if ( *u1 == INDEX ) goto NoPoly;
02907                         u1 += u1[1];
02908                     }
02909                     u1 = u2;
02910                 }
02911             }
02912             if ( s1[i1] < 0 ) {
02913                 if ( s1[i1] != -SNUMBER && s1[i1] != -SYMBOL && s1[i1] > -FUNCTION )
02914                     goto NoPoly;
02915             }
02916             else {
02917                 u1 = s1 +i1 + ARGHEAD;
02918                 while ( u1 < t1 ) {
02919                     u2 = u1 + *u1;
02920                     ustop = u2 - ABS(u2[-1]);
02921                     u1++;
02922                     while ( u1 < ustop ) {
02923                         if ( *u1 == INDEX ) goto NoPoly;
02924                         u1 += u1[1];
02925                     }
02926                     u1 = u2;
02927                 }
02928             }
02929             if ( *s2 < 0 ) {
02930                 if ( *s2 != -SNUMBER && *s2 != -SYMBOL && *s2 > -FUNCTION )
02931                     goto NoPoly;
02932             }
02933             else {
02934                 u1 = s2 + ARGHEAD;
02935                 while ( u1 < s2+i2 ) {
02936                     u2 = u1 + *u1;
02937                     ustop = u2 - ABS(u2[-1]);
02938                     u1++;
02939                     while ( u1 < ustop ) {
02940                         if ( *u1 == INDEX ) goto NoPoly;
02941                         u1 += u1[1];
02942                     }
02943                     u1 = u2;
02944                 }
02945             }
02946             if ( s2[i2] < 0 ) {
02947                 if ( s2[i2] != -SNUMBER && s2[i2] != -SYMBOL && s2[i2] > -FUNCTION )
02948                     goto NoPoly;
02949             }
02950             else {
02951                 u1 = s2 + i2 + ARGHEAD;
02952                 while ( u1 < t2 ) {
02953                     u2 = u1 + *u1;
02954                     ustop = u2 - ABS(u2[-1]);
02955                     u1++;
02956                     while ( u1 < ustop ) {
02957                         if ( *u1 == INDEX ) goto NoPoly;
02958                         u1 += u1[1];
02959                     }
02960                     u1 = u2;
02961                 }
02962             }
02963         }
02964         S->PolyWise = WORDDIFF(s1,term1);
02965         S->PolyWise -= FUNHEAD;

```



```

02966         count = 1;
02967         continue;
02968     }
02969     else {
02970         S->PolyWise = localPoly = 0;
02971     }
02972 }
02973 else {
02974     S->PolyWise = localPoly = 0;
02975 }
02976 }
02977 else {
02978     t1 = term1 + S->PolyWise;
02979     t2 = term2 + S->PolyWise;
02980     S->PolyWise = 0;
02981     localPoly = 0;
02982     continue;
02983 }
02984 }
02985 #ifndef WITHFLOAT
02986     if ( c1 == FLOATFUN && t1 == stopper1 && t2 == stopper2 && AT.aux_ != 0 ) {
02987 /*
02988         We have two FLOATFUN's. Test whether they are 'legal'
02989 */
02990         if ( TestFloat(s1-FUNHEAD) ) {
02991             if ( TestFloat(s2-FUNHEAD) ) { AT.SortFloatMode = 3; return(0); }
02992             else { return(1); }
02993         }
02994         else if ( TestFloat(s2-FUNHEAD) ) { return(-1); }
02995     }
02996 #endif
02997     while ( s1 < t1 ) {
02998 /*
02999         The next statement was added 9-nov-2001. It repaired a bad error
03000 */
03001         if ( s2 >= t2 ) return(PREV(-1));
03002 /*
03003         There is a little problem here with fast arguments
03004         We don't want to sacrifice speed, but we like to
03005         keep a rational ordering. This last one suffers in
03006         the solution that has been chosen here.
03007 */
03008         if ( AC.properorderflag ) {
03009             WORD oldpolyflag;
03010             oldpolyflag = S->PolyFlag;
03011             S->PolyFlag = 0;
03012             if ( ( c2 = -CompArg(s1,s2) ) != 0 ) {
03013                 S->PolyFlag = oldpolyflag; return(PREV(c2));
03014             }
03015             S->PolyFlag = oldpolyflag;
03016             NEXTARG(s1)
03017             NEXTARG(s2)
03018         }
03019         else {
03020             if ( *s1 > 0 ) {
03021                 if ( *s2 > 0 ) {
03022                     WORD oldpolyflag;
03023                     stopex1 = s1 + *s1;
03024                     if ( s2 >= t2 ) return(PREV(-1));
03025                     stopex2 = s2 + *s2;
03026                     s1 += ARGHEAD; s2 += ARGHEAD;
03027                     oldpolyflag = S->PolyFlag;
03028                     S->PolyFlag = 0;
03029                     while ( s1 < stopex1 ) {
03030                         if ( s2 >= stopex2 ) {
03031                             S->PolyFlag = oldpolyflag; return(PREV(-1));
03032                         }
03033                         if ( ( c2 = CompareTerms(BHEAD s1,s2,(WORD)1) ) != 0 ) {
03034                             S->PolyFlag = oldpolyflag; return(PREV(c2));
03035                         }
03036                         s1 += *s1;
03037                         s2 += *s2;
03038                     }
03039                     S->PolyFlag = oldpolyflag;
03040                     if ( s2 < stopex2 ) return(PREV(1));
03041                 }
03042                 else return(PREV(1));
03043             }
03044             else {
03045                 if ( *s2 > 0 ) return(PREV(-1));
03046                 if ( *s1 != *s2 ) { return(PREV(*s1-*s2)); }
03047                 if ( *s1 > -FUNCTION ) {
03048                     if ( ++s1 != ++s2 ) { return(PREV(*s2-*s1)); }
03049                 }
03050                 s1++; s2++;
03051             }
03052         }

```

```

03053         }
03054         if ( s2 < t2 ) return(PREV(1));
03055     }
03056 }
03057 #ifdef WITHFLOAT
03058 if ( t1 < stopper1 && *t1 == FLOATFUN && t1+t1[1] == stopper1
03059     && TestFloat(t1) ) {
03060     AT.SortFloatMode = 1; return(0);
03061 }
03062 else if ( t2 < stopper2 && *t2 == FLOATFUN && t2+t2[1] == stopper2
03063     && TestFloat(t2) ) {
03064     AT.SortFloatMode = 2; return(0);
03065 }
03066 #endif
03067 {
03068     if ( AR.SortType != SORTLOWFIRST ) {
03069         if ( t1 < stopper1 ) return(PREV(1));
03070         if ( t2 < stopper2 ) return(PREV(-1));
03071     }
03072     else {
03073         if ( t1 < stopper1 ) return(PREV(-1));
03074         if ( t2 < stopper2 ) return(PREV(1));
03075     }
03076 }
03077 if ( level == 3 ) return(CompCoef(term1,term2));
03078 if ( level >= 1 )
03079     return(CompCoef(term2,term1));
03080 return(0);
03081 }
03082
03083 /*
03084     #] Compare1 :
03085     #[ CompareSymbols :          WORD CompareSymbols(term1,term2,par)
03086 */
03100 WORD CompareSymbols(PHEAD WORD *term1, WORD *term2, WORD par)
03101 {
03102     int sum1, sum2;
03103     WORD *t1, *t2, *tt1, *tt2;
03104     int low, high;
03105     DUMMYUSE(par);
03106     if ( AR.SortType == SORTLOWFIRST ) { low = 1; high = -1; }
03107     else { low = -1; high = 1; }
03108     t1 = term1 + 1; tt1 = term1+*term1; tt1 -= ABS(tt1[-1]); t1 += 2;
03109     t2 = term2 + 1; tt2 = term2+*term2; tt2 -= ABS(tt2[-1]); t2 += 2;
03110     if ( AN.polysortflag > 0 ) {
03111         sum1 = 0; sum2 = 0;
03112         while ( t1 < tt1 ) { sum1 += t1[1]; t1 += 2; }
03113         while ( t2 < tt2 ) { sum2 += t2[1]; t2 += 2; }
03114         if ( sum1 < sum2 ) return(low);
03115         if ( sum1 > sum2 ) return(high);
03116         t1 = term1+3; t2 = term2 + 3;
03117     }
03118     while ( t1 < tt1 && t2 < tt2 ) {
03119         if ( *t1 > *t2 ) return(low);
03120         if ( *t1 < *t2 ) return(high);
03121         if ( t1[1] < t2[1] ) return(low);
03122         if ( t1[1] > t2[1] ) return(high);
03123         t1 += 2; t2 += 2;
03124     }
03125     if ( t1 < tt1 ) return(high);
03126     if ( t2 < tt2 ) return(low);
03127     return(0);
03128 }
03129
03130 /*
03131     #] CompareSymbols :
03132     #[ CompareHSymbols :          WORD CompareHSymbols(term1,term2,par)
03133 */
03143 WORD CompareHSymbols(PHEAD WORD *term1, WORD *term2, WORD par)
03144 {
03145     WORD *t1, *t2, *tt1, *tt2, *ttt1, *ttt2;
03146     DUMMYUSE(par);
03147     DUMMYUSE(AT.WorkPointer);
03148     t1 = term1 + 1; tt1 = term1+*term1; tt1 -= ABS(tt1[-1]); t1 += 2;
03149     t2 = term2 + 1; tt2 = term2+*term2; tt2 -= ABS(tt2[-1]); t2 += 2;
03150     while ( t1 < tt1 && t2 < tt2 ) {
03151         if ( *t1 != *t2 ) {
03152             if ( t1[0] < t2[0] ) return(-1);
03153             return(1);
03154         }
03155         else if ( *t1 == HAAKJE ) {
03156             t1 += 3; t2 += 3; continue;
03157         }
03158         ttt1 = t1+t1[1]; ttt2 = t2+t2[1];
03159         while ( t1 < ttt1 && t2 < ttt2 ) {
03160             if ( *t1 > *t2 ) return(-1);
03161             if ( *t1 < *t2 ) return(1);

```

```

03162         if ( t1[1] < t2[1] ) return(-1);
03163         if ( t1[1] > t2[1] ) return(1);
03164         t1 += 2; t2 += 2;
03165     }
03166     if ( t1 < ttt1 ) return(1);
03167     if ( t2 < ttt2 ) return(-1);
03168 }
03169 if ( t1 < tt1 ) return(1);
03170 if ( t2 < tt2 ) return(-1);
03171 return(0);
03172 }
03173
03174 /*
03175     #] CompareHSymbols :
03176     #[ ComPress :                LONG ComPress(ss,n)
03177 */
03196 LONG ComPress(WORD **ss, LONG *n)
03197 {
03198     GETIDENTITY
03199     WORD *t, *s, j, k;
03200     LONG size = 0;
03201     int newsize, i;
03202     /*
03203         #] debug :
03204
03205     WORD **sss = ss;
03206
03207     if ( AP.DebugFlag ) {
03208         UBYTE OutBuf[140];
03209         MLOCK(ErrorMessageLock);
03210         MesPrint("ComPress:");
03211         AO.OutFill = AO.OutputLine = OutBuf;
03212         AO.OutSkip = 3;
03213         FiniLine();
03214         ss = sss;
03215         while ( *ss ) {
03216             s = *ss++;
03217             j = *s;
03218             if ( j < 0 ) {
03219                 j = s[1] + 2;
03220             }
03221             while ( --j >= 0 ) {
03222                 TalToLine((UWORD) (*s++)); TokenToLine((UBYTE *) " ");
03223             }
03224             FiniLine();
03225         }
03226         AO.OutSkip = 0;
03227         FiniLine();
03228         MUNLOCK(ErrorMessageLock);
03229         ss = sss;
03230     }
03231
03232     #] debug :
03233 */
03234     *n = 0;
03235     if ( AT.SS == AT.S0 && !AR.NoCompress ) {
03236         if ( AN.compressSize == 0 ) {
03237             if ( *ss ) { AN.compressSize = *ss + 64; }
03238             else { AN.compressSize = AM.MaxTer/sizeof(WORD) + 2; }
03239             AN.compressSpace = (WORD *)Malloc1(AN.compressSize*sizeof(WORD), "Compression");
03240         }
03241         AN.compressSpace[0] = 0;
03242         while ( *ss ) {
03243             k = 0;
03244             s = *ss;
03245             j = *s++;
03246             if ( j > AN.compressSize ) {
03247                 newsize = j + 64;
03248                 t = (WORD *)Malloc1(newsize*sizeof(WORD), "Compression");
03249                 t[0] = 0;
03250                 if ( AN.compressSpace ) {
03251                     for ( i = 0; i < *AN.compressSpace; i++ ) t[i] = AN.compressSpace[i];
03252                     M_free(AN.compressSpace, "Compression");
03253                 }
03254                 AN.compressSpace = t;
03255                 AN.compressSize = newsize;
03256             }
03257             t = AN.compressSpace;
03258             i = *t - 1;
03259             *t++ = j; j--;
03260             if ( AR.PolyFun ) {
03261                 WORD *polystop, *sa;
03262                 sa = s + j;
03263                 sa -= ABS(sa[-1]);
03264                 polystop = s;
03265                 while ( polystop < sa && *polystop != AR.PolyFun ) {
03266                     polystop += polystop[1];

```

```

03267         }
03268         while ( i > 0 && j > 0 && *s == *t && s < polystop ) {
03269             i--; j--; s++; t++; k--;
03270         }
03271     }
03272     else {
03273         WORD *sa;
03274         sa = s + j;
03275         sa -= ABS(sa[-1]);
03276         while ( i > 0 && j > 0 && *s == *t && s < sa ) { i--; j--; s++; t++; k--; }
03277     }
03278     if ( k < -1 ) {
03279         s[-1] = j;
03280         s[-2] = k;
03281         *ss = s-2;
03282         size += j + 2;
03283     }
03284     else {
03285         size += *AN.compressSpace;
03286         if ( k == -1 ) { t--; s--; j++; }
03287     }
03288     while ( --j >= 0 ) *t++ = *s++;
03289     /* Sabotage getting into the coefficient next time */
03290     t = AN.compressSpace + *AN.compressSpace;
03291     t[-(ABS(t[-1]))] = 0;
03292     ss++;
03293     (*n)++;
03294 }
03295 }
03296 else {
03297     while ( *ss ) {
03298         size += *(*ss++);
03299         (*n)++;
03300     }
03301 }
03302 /*
03303     #[] debug :
03304
03305     if ( AP.DebugFlag ) {
03306         UBYTE OutBuf[140];
03307         AO.OutFill = AO.OutputLine = OutBuf;
03308         AO.OutSkip = 3;
03309         FiniLine();
03310         ss = sss;
03311         while ( *ss ) {
03312             s = *ss++;
03313             j = *s;
03314             if ( j < 0 ) {
03315                 j = s[1] + 2;
03316             }
03317             while ( --j >= 0 ) {
03318                 TalToLine((UWORD) (*s++)); TokenToLine((UBYTE *) " ");
03319             }
03320             FiniLine();
03321         }
03322         AO.OutSkip = 0;
03323         FiniLine();
03324     }
03325
03326     #[] debug :
03327 */
03328     return(size);
03329 }
03330
03331 /*
03332     #[] ComPress :
03333     #[] SplitMerge :          void SplitMerge(Point,number)
03334 */
03360 #ifdef NEWSPLITMERGE
03361
03362 LONG SplitMerge(PHEAD WORD **Pointer, LONG number)
03363 {
03364     GETBIDENTITY
03365     SORTING *S = AT.SS;
03366     WORD **pp3, **pp1, **pp2, **pptop;
03367     LONG i, newleft, newright, split;
03368
03369 #ifdef SPLITMERGEDEBUG
03370     /* Print current array state on entry. */
03371     printf("%4ld: ", number);
03372     for (int ii = 0; ii < S->sTerms; ii++) {
03373         if ( (S->sPointer)[ii] ) {
03374             printf("%4d ", (unsigned) (S->sPointer[ii]-S->sBuffer));
03375         }
03376         else {
03377             printf(".... ");
03378         }
03379     }

```

```

03379     }
03380     printf("\n");
03381     fflush(stdout);
03382 #endif
03383
03384     if ( number < 2 ) return(number);
03385     if ( number == 2 ) {
03386         pp1 = Pointer; pp2 = pp1 + 1;
03387         if ( ( i = CompareTerms(BHEAD *pp1,*pp2,(WORD)0) ) < 0 ) {
03388             pp3 = (WORD **)(*pp1); *pp1 = *pp2; *pp2 = (WORD *)pp3;
03389         }
03390         else if ( i == 0 ) {
03391             number--;
03392             if ( S->PolyWise ) { if ( AddPoly(BHEAD pp1,pp2) == 0 ) number = 0; }
03393             else { if ( AddCoef(BHEAD pp1,pp2) == 0 ) number = 0; }
03394         }
03395         return(number);
03396     }
03397     pptop = Pointer + number;
03398     split = number/2;
03399     newleft = SplitMerge(BHEAD Pointer,split);
03400     newright = SplitMerge(BHEAD Pointer+split,number-split);
03401     if ( newright == 0 ) return(newleft);
03402 /*
03403     We compare the last of the left with the first of the right
03404     If they are already in order, we will be done quickly.
03405     We may have to compactify the buffer because the recursion may
03406     have created holes. Also this compare may result in equal terms.
03407     Addition of 23-jul-1999. It makes things a bit faster.
03408 */
03409     if ( newleft > 0 && newright > 0 &&
03410         ( i = CompareTerms(BHEAD Pointer[newleft-1],Pointer[split],(WORD)0) ) >= 0 ) {
03411         pp2 = Pointer+split; pp1 = Pointer+newleft-1;
03412         if ( i == 0 ) {
03413             if ( S->PolyWise ) {
03414                 if ( AddPoly(BHEAD pp1,pp2) > 0 ) pp1++;
03415                 else newleft--;
03416             }
03417             else {
03418                 if ( AddCoef(BHEAD pp1,pp2) > 0 ) pp1++;
03419                 else newleft--;
03420             }
03421             *pp2++ = 0; newright--;
03422         }
03423         else pp1++;
03424         newleft += newright;
03425         if ( pp1 < pp2 ) {
03426             while ( --newright >= 0 ) *pp1++ = *pp2++;
03427             while ( pp1 < pptop ) *pp1++ = 0;
03428         }
03429         return(newleft);
03430     }
03431
03432     if ( split >= AN.SplitScratchSize ) {
03433         AN.SplitScratchSize = (split*3)/2+100;
03434         if ( AN.SplitScratchSize > S->Terms2InSmall/2 )
03435             AN.SplitScratchSize = S->Terms2InSmall/2;
03436         if ( AN.SplitScratch ) M_free(AN.SplitScratch,"AN.SplitScratch");
03437         AN.SplitScratch = (WORD **)Malloc1(AN.SplitScratchSize*sizeof(WORD *),"AN.SplitScratch");
03438     }
03439
03440     pp3 = AN.SplitScratch; pp1 = Pointer;
03441     /* Move rather than copy, so GarbHand can't double-count. */
03442     for ( i = 0; i < newleft; i++ ) { *pp3++ = *pp1; *pp1++ = 0; }
03443     AN.InScratch = newleft;
03444     pp1 = AN.SplitScratch; pp2 = Pointer + split; pp3 = Pointer;
03445
03446 #ifdef NEWSPLITMERGETIMSORT
03447 /*
03448     An improvement in the style of Timsort
03449 */
03450     while ( newleft > 8 ) {
03451         /* Check the middle of the LHS terms */
03452         LONG nnleft = newleft/2;
03453         if ( ( i = CompareTerms(BHEAD pp1[nnleft],*pp2,(WORD)0) ) < 0 ) {
03454             /* The terms are not in order. Break out and continue as normal. */
03455             break;
03456         }
03457         /* The terms merge or are in order. Copy pointers up to this point. */
03458         /* In the copy, zero the skipped pointers so GarbHand can't double-count. */
03459         for (int iii = 0; iii < nnleft; iii++) {
03460             *pp3++ = *pp1;
03461             *pp1++ = 0;
03462         }
03463         newleft -= nnleft;
03464         if ( i == 0 ) {
03465             if ( S->PolyWise ) { i = AddPoly(BHEAD pp1,pp2); }

```

```

03466         else { i = AddCoef(BHEAD pp1,pp2); }
03467         if ( i == 0 ) {
03468             /* The terms cancelled. The next term goes in *pp3. Don't move. */
03469         }
03470         else {
03471             /* The terms added. Advance pp3. */
03472             *pp3++ = *pp1;
03473         }
03474         /* We have taken a LHS (copy) and RHS term. */
03475         *pp2++ = 0;
03476         newright--;
03477         *pp1++ = 0;
03478         newleft--;
03479         break;
03480     }
03481 }
03482 #endif
03483
03484 while ( newleft > 0 && newright > 0 ) {
03485     if ( ( i = CompareTerms(BHEAD *pp1,*pp2,(WORD)0) ) < 0 ) {
03486         *pp3++ = *pp2;
03487         *pp2++ = 0;
03488         newright--;
03489     }
03490     else if ( i > 0 ) {
03491         *pp3++ = *pp1;
03492         *pp1++ = 0;
03493         newleft--;
03494     }
03495     else {
03496         if ( S->PolyWise ) { if ( AddPoly(BHEAD pp1,pp2) > 0 ) *pp3++ = *pp1; }
03497         else { if ( AddCoef(BHEAD pp1,pp2) > 0 ) *pp3++ = *pp1; }
03498         *pp1++ = 0; *pp2++ = 0; newleft--; newright--;
03499     }
03500 }
03501 for ( i = 0; i < newleft; i++ ) { *pp3++ = *pp1; *pp1++ = 0; }
03502 if ( pp3 == pp2 ) {
03503     pp3 += newright;
03504 } else {
03505     for ( i = 0; i < newright; i++ ) { *pp3++ = *pp2++; }
03506 }
03507 newleft = pp3 - Pointer;
03508 while ( pp3 < pptop ) *pp3++ = 0;
03509 AN.InScratch = 0;
03510 return(newleft);
03511 }
03512
03513 #else
03514
03515 LONG SplitMerge(PHEAD WORD **Pointer, LONG number)
03516 {
03517     GETBIDENTITY
03518     SORTING *S = AT.SS;
03519     WORD **pp3, **pp1, **pp2;
03520     LONG nleft, nright, i, newleft, newright;
03521     WORD **pptop;
03522
03523 #ifdef SPLITMERGEDEBUG
03524     /* Print current array state on entry. */
03525     printf("%4ld: ", number);
03526     for (int ii = 0; ii < S->sTerms; ii++) {
03527         if ( (S->sPointer)[ii] ) {
03528             printf("%4d ", (unsigned)(S->sPointer[ii]-S->sBuffer));
03529         }
03530         else {
03531             printf(".... ");
03532         }
03533     }
03534     printf("\n");
03535     fflush(stdout);
03536 #endif
03537
03538     if ( number < 2 ) return(number);
03539     if ( number == 2 ) {
03540         pp1 = Pointer; pp2 = pp1 + 1;
03541         if ( ( i = CompareTerms(BHEAD *pp1,*pp2,(WORD)0) ) < 0 ) {
03542             pp3 = (WORD **)(*pp1); *pp1 = *pp2; *pp2 = (WORD *)pp3;
03543         }
03544         else if ( i == 0 ) {
03545             number--;
03546             if ( S->PolyWise ) { if ( AddPoly(BHEAD pp1,pp2) == 0 ) { number = 0; } }
03547             else { if ( AddCoef(BHEAD pp1,pp2) == 0 ) { number = 0; } }
03548         }
03549         return(number);
03550     }
03551     pptop = Pointer + number;
03552     nleft = number » 1; nright = number - nleft;

```

```

03553     newleft  = SplitMerge(BHEAD Pointer,newleft);
03554     newright = SplitMerge(BHEAD Pointer+newleft,nright);
03555 /*
03556     We compare the last of the left with the first of the right
03557     If they are already in order, we will be done quickly.
03558     We may have to compactify the buffer because the recursion may
03559     have created holes. Also this compare may result in equal terms.
03560     Addition of 23-jul-1999. It makes things a bit faster.
03561 */
03562     if ( newleft > 0 && newright > 0 &&
03563         ( i = CompareTerms(BHEAD Pointer[newleft-1],Pointer[nleft],(WORD)0) ) >= 0 ) {
03564         pp2 = Pointer+nleft; pp1 = Pointer+newleft-1;
03565         if ( i == 0 ) {
03566             if ( S->PolyWise ) {
03567                 if ( AddPoly(BHEAD pp1,pp2) > 0 ) pp1++;
03568                 else newleft--;
03569             }
03570             else {
03571                 if ( AddCoef(BHEAD pp1,pp2) > 0 ) pp1++;
03572                 else newleft--;
03573             }
03574             *pp2++ = 0; newright--;
03575         }
03576         else pp1++;
03577         newleft += newright;
03578         if ( pp1 < pp2 ) {
03579             while ( --newright >= 0 ) *pp1++ = *pp2++;
03580             while ( pp1 < pptop ) *pp1++ = 0;
03581         }
03582         return(newleft);
03583     }
03584     if ( nleft > AN.SplitScratchSize ) {
03585         AN.SplitScratchSize = (nleft*3)/2+100;
03586         if ( AN.SplitScratchSize > S->Terms2InSmall/2 )
03587             AN.SplitScratchSize = S->Terms2InSmall/2;
03588         if ( AN.SplitScratch ) M_free(AN.SplitScratch,"AN.SplitScratch");
03589         AN.SplitScratch = (WORD **)Malloca(AN.SplitScratchSize*sizeof(WORD *),"AN.SplitScratch");
03590     }
03591     pp3 = AN.SplitScratch; pp1 = Pointer; i = nleft;
03592     do { *pp3++ = *pp1; *pp1++ = 0; } while ( *pp1 && --i > 0 );
03593     if ( i > 0 ) { *pp3 = 0; i--; }
03594     AN.InScratch = nleft - i;
03595     pp1 = AN.SplitScratch; pp2 = Pointer + nleft; pp3 = Pointer;
03596     while ( nleft > 0 && nright > 0 && *pp1 && *pp2 ) {
03597         if ( ( i = CompareTerms(BHEAD *pp1,*pp2,(WORD)0) ) < 0 ) {
03598             *pp3++ = *pp2;
03599             *pp2++ = 0;
03600             nright--;
03601         }
03602         else if ( i > 0 ) {
03603             *pp3++ = *pp1;
03604             *pp1++ = 0;
03605             nleft--;
03606         }
03607         else {
03608             if ( S->PolyWise ) { if ( AddPoly(BHEAD pp1,pp2) > 0 ) *pp3++ = *pp1; }
03609             else { if ( AddCoef(BHEAD pp1,pp2) > 0 ) *pp3++ = *pp1; }
03610             *pp1++ = 0; *pp2++ = 0; nleft--; nright--;
03611         }
03612     }
03613     while ( --nleft >= 0 && *pp1 ) { *pp3++ = *pp1; *pp1++ = 0; }
03614     while ( --nright >= 0 && *pp2 ) { *pp3++ = *pp2++; }
03615     nleft = pp3 - Pointer;
03616     while ( pp3 < pptop ) *pp3++ = 0;
03617     AN.InScratch = 0;
03618     return(nleft);
03619 }
03620
03621 #endif
03622
03623 /*
03624     #] SplitMerge :
03625     #[ GarbHand :          void GarbHand()
03626 */
03642 void GarbHand(void)
03643 {
03644     GETIDENTITY
03645     SORTING *S = AT.SS;
03646     WORD **Point, *s2, *t, *garbuf, i;
03647     LONG k, total = 0;
03648     int tobereturned = 0;
03649 /*
03650     Compute the size needed. Put it in total.
03651 */
03652 #ifdef TESTGARB
03653     MLOCK(ErrorMessageLock);
03654     MesPrint("in:  S->sFill = %l, S->sTop2 = %l",S->sFill-S->sBuffer,S->sTop2-S->sBuffer);

```

```

03655 #endif
03656     Point = S->sPointer;
03657     k = S->sTerms;
03658     while ( --k >= 0 ) {
03659         if ( ( s2 = *Point++ ) != 0 ) { total += *s2; }
03660     }
03661     Point = AN.SplitScratch;
03662     k = AN.InScratch;
03663     while ( --k >= 0 ) {
03664         if ( ( s2 = *Point++ ) != 0 ) { total += *s2; }
03665     }
03666 #ifdef TESTGARB
03667     MesPrint("total = %1, nterms = %1",total,AN.InScratch);
03668     MUNLOCK(ErrorMessageLock);
03669 #endif
03670 /*
03671     Test now whether it fits. If so deal with the problem inside
03672     the memory at the tail of the large buffer.
03673 */
03674     if ( S->lBuffer != 0 && S->lFill + total <= S->lTop ) {
03675         garbuf = S->lFill;
03676     }
03677     else {
03678         garbuf = (WORD *)Malloc1(total*sizeof(WORD),"Garbage buffer");
03679         tobereturned = 1;
03680     }
03681     t = garbuf;
03682     Point = S->sPointer;
03683     k = S->sTerms;
03684     while ( --k >= 0 ) {
03685         if ( *Point ) {
03686             s2 = *Point++;
03687             i = *s2;
03688             NCOPY(t,s2,i);
03689         }
03690         else { Point++; }
03691     }
03692     Point = AN.SplitScratch;
03693     k = AN.InScratch;
03694     while ( --k >= 0 ) {
03695         if ( *Point ) {
03696             s2 = *Point++;
03697             i = *s2;
03698             NCOPY(t,s2,i);
03699         }
03700         else Point++;
03701     }
03702     s2 = S->sBuffer;
03703     t = garbuf;
03704     Point = S->sPointer;
03705     k = S->sTerms;
03706     while ( --k >= 0 ) {
03707         if ( *Point ) {
03708             *Point++ = s2;
03709             i = *t;
03710             NCOPY(s2,t,i);
03711         }
03712         else { Point++; }
03713     }
03714     Point = AN.SplitScratch;
03715     k = AN.InScratch;
03716     while ( --k >= 0 ) {
03717         if ( *Point ) {
03718             *Point++ = s2;
03719             i = *t;
03720             NCOPY(s2,t,i);
03721         }
03722         else Point++;
03723     }
03724     S->sFill = s2;
03725 #ifdef TESTGARB
03726     MLOCK(ErrorMessageLock);
03727     MesPrint("out: S->sFill = %1, S->sTop2 = %1",S->sFill-S->sBuffer,S->sTop2-S->sBuffer);
03728     if ( S->sFill >= S->sTop2 ) {
03729         MesPrint("We are in deep trouble");
03730     }
03731     MUNLOCK(ErrorMessageLock);
03732 #endif
03733     if ( tobereturned ) M_free(garbuf,"Garbage buffer");
03734     return;
03735 }
03736
03737 /*
03738     #] GarbHand :
03739     #[ MergePatches :          WORD MergePatches(par)
03740 */
03757 WORD MergePatches(WORD par)

```



```

03758 {
03759     GETIDENTITY
03760     SORTING *S = AT.SS;
03761     WORD **poin, **poin2, ul, k, i, im, *ml;
03762     WORD *p, lpat, mpat, level, l1, l2, r1, r2, r3, c;
03763     WORD *m2, *m3, r3l, r33, ki, *rr;
03764     UWORD *coef;
03765     POSITION position;
03766     FILEHANDLE *fin, *fout;
03767     int fhandle;
03768     /*
03769         UBYTE *s;
03770     */
03771     #ifdef WITHZLIB
03772         POSITION position2;
03773         int oldgzipCompress = AR.gzipCompress;
03774         if ( par == 2 ) {
03775             AR.gzipCompress = 0;
03776         }
03777     #endif
03778     fin = &S->file;
03779     fout = &(AR.FoStage4[0]);
03780     NewMerge:
03781     coef = AN.SoScratC;
03782     poin = S->poina; poin2 = S->poin2a;
03783     rr = AR.CompressPointer;
03784     *rr = 0;
03785     /*
03786         #! Setup :
03787     */
03788     if ( par == 1 ) {
03789         fout = &(S->file);
03790         if ( fout->handle < 0 ) {
03791     FileMake:
03792             PUTZERO(AN.OldPosOut);
03793             if ( ( fhandle = CreateFile(fout->name) ) < 0 ) {
03794                 MLOCK(ErrorMessageLock);
03795                 MesPrint("Cannot create file %s",fout->name);
03796                 MUNLOCK(ErrorMessageLock);
03797                 goto ReturnError;
03798             }
03799     #ifdef GZIPDEBUG
03800             MLOCK(ErrorMessageLock);
03801             MesPrint("%w MergePatches created output file %s",fout->name);
03802             MUNLOCK(ErrorMessageLock);
03803     #endif
03804             fout->handle = fhandle;
03805             PUTZERO(fout->filesize);
03806             PUTZERO(fout->POposition);
03807         /*
03808             Should not be here?
03809     #ifdef WITHZLIB
03810             fout->ziobuffer = 0;
03811     #endif
03812         */
03813     #ifdef ALLLOCK
03814             LOCK(fout->pthreadlock);
03815     #endif
03816             SeekFile(fout->handle, &(fout->filesize), SEEK_SET);
03817     #ifdef ALLLOCK
03818             UNLOCK(fout->pthreadlock);
03819     #endif
03820             S->fPatchN = 0;
03821             PUTZERO(S->fPatches[0]);
03822             fout->POfill = fout->PObuffer;
03823             PUTZERO(fout->POposition);
03824         }
03825     ConMer:
03826         StageSort(fout);
03827     #ifdef WITHZLIB
03828         if ( S == AT.S0 && AR.NoCompress == 0 && AR.gzipCompress > 0 )
03829             S->fpcompressed[S->fPatchN] = 1;
03830         else
03831             S->fpcompressed[S->fPatchN] = 0;
03832         SetupOutputGZIP(fout);
03833     #endif
03834     }
03835     else if ( par == 0 && S->stage4 > 0 ) {
03836     /*
03837         We will have to do our job more than once.
03838         Input is from S->file and output will go to AR.FoStage4.
03839         The file corresponding to this last one must be made now.
03840     */
03841         AR.Stage4Name ^= 1;
03842     /*
03843         s = (UBYTE *) (fout->name); while ( *s ) s++;
03844         if ( AR.Stage4Name ) s[-1] += 1;

```

```

03845         else                s[-1] -= 1;
03846     /*
03847         S->iPatches = S->fPatches;
03848         S->fPatches = S->inPatches;
03849         S->inPatches = S->iPatches;
03850         (S->inNum) = S->fPatchN;
03851         AN.OldPosIn = AN.OldPosOut;
03852     #ifdef WITHZLIB
03853         m1 = S->fpincompressed;
03854         S->fpincompressed = S->fpcompressed;
03855         S->fpcompressed = m1;
03856         for ( i = 0; i < S->inNum; i++ ) {
03857             S->fPatchesStop[i] = S->iPatches[i+1];
03858     #ifdef GZIPDEBUG
03859             MLOCK(ErrorMessageLock);
03860             MesPrint("%w fPatchesStop[%d] = %l0p",i,&(S->fPatchesStop[i]));
03861             MUNLOCK(ErrorMessageLock);
03862     #endif
03863         }
03864     #endif
03865         S->stage4 = 0;
03866         goto FileMake;
03867     }
03868     else {
03869     #ifdef WITHZLIB
03870     /*
03871         The next statement is just for now
03872     */
03873         AR.gzipCompress = 0;
03874     #endif
03875         if ( par == 0 ) {
03876             S->iPatches = S->fPatches;
03877             S->inNum = S->fPatchN;
03878     #ifdef WITHZLIB
03879             m1 = S->fpincompressed;
03880             S->fpincompressed = S->fpcompressed;
03881             S->fpcompressed = m1;
03882             for ( i = 0; i < S->inNum; i++ ) {
03883                 S->fPatchesStop[i] = S->fPatches[i+1];
03884     #ifdef GZIPDEBUG
03885                 MLOCK(ErrorMessageLock);
03886                 MesPrint("%w fPatchesStop[%d] = %l0p",i,&(S->fPatchesStop[i]));
03887                 MUNLOCK(ErrorMessageLock);
03888     #endif
03889             }
03890     #endif
03891         }
03892         fout = AR.outfile;
03893     }
03894     if ( par ) { /* Mark end of patches */
03895         S->Patches[S->lPatch] = S->lFill;
03896         for ( i = 0; i < S->lPatch; i++ ) {
03897             S->pStop[i] = S->Patches[i+1]-1;
03898             S->Patches[i] = (WORD *)(((UBYTE *) (S->Patches[i])) + AM.MaxTer);
03899         }
03900     }
03901     else { /* Load the patches */
03902         S->lPatch = (S->inNum);
03903     #ifdef WITHMPTI
03904         if ( S->lPatch > 1 || ( (PF.exprtodo < 0) && (fout == AR.outfile || fout == AR.hidefile ) ) ) {
03905     #else
03906         if ( S->lPatch > 1 ) {
03907     #endif
03908     #ifdef WITHZLIB
03909         SetupAllInputGZIP(S);
03910     #endif
03911         p = S->lBuffer;
03912         for ( i = 0; i < S->lPatch; i++ ) {
03913             p = (WORD *)(((UBYTE *)p)+2*AM.MaxTer+COMPINC*sizeof(WORD));
03914             S->Patches[i] = p;
03915             p = (WORD *)(((UBYTE *)p) + fin->POsize);
03916             S->pStop[i] = m2 = p;
03917     #ifdef WITHZLIB
03918             PutIn(fin,&(S->iPatches[i]),S->Patches[i],&m2,i);
03919     #else
03920             ADDPOS(S->iPatches[i],PutIn(fin,&(S->iPatches[i]),S->Patches[i],&m2,i));
03921     #endif
03922         }
03923     }
03924     }
03925     if ( fout->handle >= 0 ) {
03926         PUTZERO(position);
03927     #ifdef ALLLOCK
03928         LOCK(fout->pthreadlock);
03929     #endif
03930         SeekFile(fout->handle,&position,SEEK_END);
03931         ADDPOS(position,((fout->POfill-fout->PObuffer)*sizeof(WORD)));

```

```

03932 #ifdef ALLLOCK
03933     UNLOCK(fout->pthreadlock);
03934 #endif
03935 }
03936 else {
03937     SETBASEPOSITION(position, (fout->POfill-fout->PObuffer)*sizeof(WORD));
03938 }
03939 /*
03940     #] Setup :
03941
03942     The old code had to be replaced because all output needs to go
03943     through PutOut. For this we have to go term by term and keep
03944     track of the compression.
03945 */
03946 if ( S->lPatch == 1 ) { /* Single patch --> direct copy. Very rare. */
03947     LONG length;
03948
03949     if ( fout->handle < 0 ) if ( Sflush(fout) ) goto PatCall;
03950     if ( par ) { /* Memory to file */
03951 #ifdef WITHZLIB
03952     /*
03953         We fix here the problem that the thing needs to go through PutOut
03954     */
03955         m2 = m1 = *S->Patches; /* The m2 is to keep the compiler from complaining */
03956         while ( *m1 ) {
03957             if ( *m1 < 0 ) { /* Need to uncompress */
03958                 i = -( *m1++ ); m2 += i; im = *m1+i+1;
03959                 while ( i > 0 ) { *m1-- = *m2--; i--; }
03960                 *m1 = im;
03961             }
03962 #ifdef WITHPTHREADS
03963             /* Check here (and in the following) that we are at ground level
03964             (so S == AT.S0) to use PutToMaster. Control can reach here,
03965             with AS.MasterSort, but with S != AT.S0, when the SortBots are
03966             adding PolyRatFun and the sorting of their arguments requires
03967             large buffer patches to be merged. In this case, terms escape
03968             the PolyRatFun argument and end up at ground level, if we use
03969             PutToMaster. */
03970             if ( AS.MasterSort && ( fout == AR.outfile ) && S == AT.S0 ) {
03971                 im = PutToMaster(BHEAD m1);
03972             }
03973             else
03974 #endif
03975                 if ( ( im = PutOut(BHEAD m1,&position,fout,1) ) < 0 ) goto ReturnError;
03976             ADDPOS(S->SizeInFile[par],im);
03977             m2 = m1;
03978             m1 += *m1;
03979         }
03980 #ifdef WITHPTHREADS
03981         if ( AS.MasterSort && ( fout == AR.outfile ) && S == AT.S0 ) {
03982             PutToMaster(BHEAD 0);
03983         }
03984         else
03985 #endif
03986             if ( FlushOut(&position,fout,1) ) goto ReturnError;
03987             ADDPOS(S->SizeInFile[par],1);
03988     #else
03989     /* old code */
03990     length = (LONG)(S->pStop)-(LONG)(S->Patches)+sizeof(WORD);
03991     if ( WriteFile(fout->handle, (UBYTE *) (S->Patches), length) != length )
03992         goto PatwCall;
03993     ADDPOS(position,length);
03994     ADDPOS(fout->POposition,length);
03995     ADDPOS(fout->filesize,length);
03996     ADDPOS(S->SizeInFile[par],length/sizeof(WORD));
03997 #endif
03998     }
03999     else { /* File to file */
04000 #ifdef WITHZLIB
04001     /*
04002         Note: if we change FRONTSIZE we need to make the minimum value
04003         of SmallEsize in AllocSort correspondingly larger or smaller.
04004         Theoretically we could get close to 2*AM.MaxTer!
04005     */
04006     #define FRONTSIZE (2*AM.MaxTer)
04007     WORD *copybuf = (WORD *) (((UBYTE *) (S->sBuffer)) + FRONTSIZE);
04008     WORD *copytop;
04009     SetupAllInputGZIP(S);
04010     m1 = m2 = copybuf;
04011     position2 = S->iPatches[0];
04012     while ( ( length = FillInputGZIP(fin,&position2,
04013         (UBYTE *) copybuf,
04014         (S->SmallEsize*sizeof(WORD)-FRONTSIZE),0) ) > 0 ) {
04015         copytop = (WORD *) (((UBYTE *) copybuf)+length);
04016         while ( *m1 && ( ( *m1 > 0 && m1+m1 < copytop ) ||
04017             ( *m1 < 0 && ( m1+1 < copytop ) && ( m1+m1[1]+1 < copytop ) ) ) )
04018         /*

```

```

04019      22-jun-2013 JV  Extremely nasty bug that has been around for a while.
04020                      What if the end is in the remaining part? We will loose terms!
04021                      while ( *m1 && ( (WORD *)(((UBYTE *) (m1)) + AM.MaxTer ) < S->sTop2 ) )
04022  */
04023                      {
04024                          if ( *m1 < 0 ) { /* Need to uncompress */
04025                              i = -( *m1++ ); m2 += i; im = *m1+i+1;
04026                              while ( i > 0 ) { *m1-- = *m2--; i--; }
04027                              *m1 = im;
04028                          }
04029  #ifndef WITHPTHREADS
04030                          if ( AS.MasterSort && ( fout == AR.outfile ) && S == AT.S0 ) {
04031                              im = PutToMaster(BHEAD m1);
04032                          }
04033                          else
04034  #endif
04035                          if ( ( im = PutOut(BHEAD m1,&position,fout,1) ) < 0 ) goto ReturnError;
04036                          ADDPOS(S->SizeInFile[par],im);
04037                          m2 = m1;
04038                          m1 += *m1;
04039                      }
04040                      if ( m1 < copytop && *m1 == 0 ) break;
04041  /*
04042                      Now move the remaining part 'back'
04043  */
04044                      m3 = copybuf;
04045                      m1 = copytop;
04046                      while ( m1 > m2 ) *--m3 = *--m1;
04047                      m2 = m3;
04048                      m1 = m2 + *m2;
04049                      }
04050                      if ( length < 0 ) {
04051                          MLOCK(ErrorMessageLock);
04052                          MesPrint("Readerror");
04053                          goto PatCall2;
04054                      }
04055  #ifndef WITHPTHREADS
04056                      if ( AS.MasterSort && ( fout == AR.outfile ) && S == AT.S0 ) {
04057                          PutToMaster(BHEAD 0);
04058                      }
04059                      else
04060  #endif
04061                      if ( FlushOut(&position,fout,1) ) goto ReturnError;
04062                      ADDPOS(S->SizeInFile[par],1);
04063  #else
04064  /* old code */
04065                      SeekFile(fin->handle,&(S->iPatches[0]),SEEK_SET); /* needed for stage4 */
04066                      while ( ( length = ReadFile(fin->handle,
04067                          (UBYTE *) (S->sBuffer),S->SmallEsize*sizeof(WORD)) ) > 0 ) {
04068                          if ( WriteFile(fout->handle,(UBYTE *) (S->sBuffer),length) != length )
04069                              goto PatwCall;
04070                          ADDPOS(position,length);
04071                          ADDPOS(fout->PPosition,length);
04072                          ADDPOS(fout->filesize,length);
04073                          ADDPOS(S->SizeInFile[par],length/sizeof(WORD));
04074                      }
04075                      if ( length < 0 ) {
04076                          MLOCK(ErrorMessageLock);
04077                          MesPrint("Readerror");
04078                          goto PatCall2;
04079                      }
04080  #endif
04081                      }
04082                      goto EndOfAll;
04083                  }
04084                  else if ( S->lPatch > 0 ) {
04085
04086                      /* More than one patch. Construct the tree. */
04087
04088                      lpat = 1;
04089                      do { lpat *= 2; } while ( lpat < S->lPatch );
04090                      mpat = ( lpat » 1 ) - 1;
04091                      k = lpat - S->lPatch;
04092
04093                      /* k is the number of empty places in the tree. they will
04094                      be at the even positions from 2 to 2*k */
04095
04096                      for ( i = 1; i < lpat; i++ ) {
04097                          S->tree[i] = -1;
04098                      }
04099                      for ( i = 1; i <= k; i++ ) {
04100                          im = ( i * 2 ) - 1;
04101                          poin[im] = S->Patches[i-1];
04102                          poin2[im] = poin[im] + *(poin[im]);
04103                          S->used[i] = im;
04104                          S->ktoi[im] = i-1;
04105                          S->tree[mpat+i] = 0;

```

```

04106         poin[im-1] = poin2[im-1] = 0;
04107     }
04108     for ( i = (k*2)+1; i <= lpat; i++ ) {
04109         S->used[i-k] = i;
04110         S->ktoi[i] = i-k-1;
04111         poin[i] = S->Patches[i-k-1];
04112         poin2[i] = poin[i] + *(poin[i]);
04113     }
04114     /*
04115     the array poin tells the position of the i-th element of the S->tree
04116     'S->used' is a stack with the S->tree elements that need to be entered
04117     into the S->tree. at the beginning this is S->lPatch. during the
04118     sort there will be only very few elements.
04119     poin2 is the next value of poin. it has to be determined
04120     before the comparisons as the position or the size of the
04121     term indicated by poin may change.
04122     S->ktoi translates a S->tree element back to its stream number.
04123
04124     start the sort
04125     */
04126     level = S->lPatch;
04127
04128     /* introduce one term */
04129 OneTerm:
04130     k = S->used[level];
04131     i = k + lpat - 1;
04132     if ( !(poin[k]) ) {
04133         do { if ( !( i >= 1 ) ) goto EndOfMerge; } while ( !S->tree[i] );
04134         if ( S->tree[i] == -1 ) {
04135             S->tree[i] = 0;
04136             level--;
04137             goto OneTerm;
04138         }
04139         k = S->tree[i];
04140         S->used[level] = k;
04141         S->tree[i] = 0;
04142     }
04143     /*
04144     move terms down the tree
04145     */
04146     while ( i >= 1 ) {
04147         if ( S->tree[i] > 0 ) {
04148             if ( ( c = CompareTerms(BHEAD poin[S->tree[i]],poin[k],(WORD)0) ) > 0 ) {
04149                 /*
04150                 S->tree[i] is the smaller. Exchange and go on.
04151                 */
04152                 S->used[level] = S->tree[i];
04153                 S->tree[i] = k;
04154                 k = S->used[level];
04155             }
04156             else if ( !c ) { /* Terms are equal */
04157                 S->TermsLeft--;
04158                 /*
04159                 Here the terms are equal and their coefficients
04160                 have to be added.
04161                 */
04162                 l1 = *( m1 = poin[S->tree[i]] );
04163                 l2 = *( m2 = poin[k] );
04164                 if ( S->PolyWise ) { /* Here we work with PolyFun */
04165                     WORD *ttl, *w;
04166                     ttl = m1;
04167                     m1 += S->PolyWise;
04168                     m2 += S->PolyWise;
04169                     if ( S->PolyFlag == 2 ) {
04170                         w = poly_ratfun_add(BHEAD m1,m2);
04171                         if ( *ttl + w[1] - m1[1] > AM.MaxTer/((LONG)sizeof(WORD)) ) {
04172                             MLOCK(ErrorMessageLock);
04173                             MesPrint("Term too complex in PolyRatFun addition. MaxTermSize of %10l
04174 is too small",AM.MaxTer);
04175                             MUNLOCK(ErrorMessageLock);
04176                             Terminate(-1);
04177                         }
04178                         AT.WorkPointer = w;
04179                     }
04180                     else {
04181                         w = AT.WorkPointer;
04182                         if ( w + m1[1] + m2[1] > AT.WorkTop ) {
04183                             MLOCK(ErrorMessageLock);
04184                             MesPrint("A Workspace of %10l is too small",AM.WorkSize);
04185                             MUNLOCK(ErrorMessageLock);
04186                             Terminate(-1);
04187                         }
04188                         AddArgs(BHEAD m1,m2,w);
04189                     }
04190                     r1 = w[1];
04191                     if ( r1 <= FUNHEAD
|| ( w[FUNHEAD] == -SNUMBER && w[FUNHEAD+1] == 0 ) )

```

```

04192             { goto cancelled; }
04193     if ( r1 == m1[1] ) {
04194         NCOPY(m1,w,r1);
04195     }
04196     else if ( r1 < m1[1] ) {
04197         r2 = m1[1] - r1;
04198         m2 = w + r1;
04199         m1 += m1[1];
04200         while ( --r1 >= 0 ) *--m1 = *--m2;
04201         m2 = m1 - r2;
04202         r1 = S->PolyWise;
04203         while ( --r1 >= 0 ) *--m1 = *--m2;
04204         *m1 -= r2;
04205         poin[S->tree[i]] = m1;
04206     }
04207     else {
04208         r2 = r1 - m1[1];
04209         m2 = ttl - r2;
04210         r1 = S->PolyWise;
04211         m1 = ttl;
04212         *m1 += r2;
04213         poin[S->tree[i]] = m2;
04214         NCOPY(m2,m1,r1);
04215         r1 = w[1];
04216         NCOPY(m2,w,r1);
04217     }
04218 }
04219 #ifdef WITHFLOAT
04220     else if ( AT.SortFloatMode ) {
04221         WORD *term1, *term2;
04222         term1 = poin[S->tree[i]];
04223         term2 = poin[k];
04224         if ( MergeWithFloat(BHEAD &term1,&term2) == 0 )
04225             goto cancelled;
04226         poin[S->tree[i]] = term1;
04227     }
04228 #endif
04229     else {
04230         r1 = *( m1 += 11 - 1 );
04231         m1 -= ABS(r1) - 1;
04232         r1 = ( ( r1 > 0 ) ? (r1-1) : (r1+1) ) >> 1;
04233         r2 = *( m2 += 12 - 1 );
04234         m2 -= ABS(r2) - 1;
04235         r2 = ( ( r2 > 0 ) ? (r2-1) : (r2+1) ) >> 1;
04236
04237         if ( AddRat(BHEAD (UWORD *)m1,r1,(UWORD *)m2,r2,coef,&r3) ) {
04238             MLOCK(ErrorMessageLock);
04239             MesCall("MergePatches");
04240             MUNLOCK(ErrorMessageLock);
04241             SETERROR(-1)
04242         }
04243
04244         if ( AN.ncmod != 0 ) {
04245             if ( ( AC.modmode & POSNEG ) != 0 ) {
04246                 NormalModulus(coef,&r3);
04247             }
04248             else if ( BigLong(coef,r3,(UWORD *)AC.cmod,ABS(AN.ncmod)) >= 0 ) {
04249                 WORD ii;
04250                 SubPLon(coef,r3,(UWORD *)AC.cmod,ABS(AN.ncmod),coef,&r3);
04251                 coef[r3] = 1;
04252                 for ( ii = 1; ii < r3; ii++ ) coef[r3+ii] = 0;
04253             }
04254         }
04255         r3 *= 2;
04256         r33 = ( r3 > 0 ) ? ( r3 + 1 ) : ( r3 - 1 );
04257         if ( r3 < 0 ) r3 = -r3;
04258         if ( r1 < 0 ) r1 = -r1;
04259         r1 *= 2;
04260         r31 = r3 - r1;
04261         if ( !r3 ) { /* Terms cancel */
04262 cancelled:
04263             ul = S->used[level] = S->tree[i];
04264             S->tree[i] = -1;
04265             /*
04266             We skip to the next term in stream ul
04267             */
04268             im = *poin2[ul];
04269             if ( im < 0 ) {
04270                 r1 = poin2[ul][1] - im + 1;
04271                 m1 = poin2[ul] + 2;
04272                 m2 = poin[ul] - im + 1;
04273                 while ( ++im <= 0 ) *--m1 = *--m2;
04274                 *--m1 = r1;
04275                 poin2[ul] = m1;
04276                 im = r1;
04277             }
04278             poin[ul] = poin2[ul];

```

```

04279         ki = S->ktoi[ul];
04280         if ( !par && (poin[ul] + im + COMPINC) >= S->pStop[ki]
04281             && im > 0 ) {
04282             #ifdef WITHZLIB
04283                 PutIn(fin,&(S->iPatches[ki]),S->Patches[ki],&(poin[ul]),ki);
04284             #else
04285                 ADDPOS(S->iPatches[ki],PutIn(fin,&(S->iPatches[ki]),
04286                     S->Patches[ki],&(poin[ul]),ki));
04287             #endif
04288             poin2[ul] = poin[ul] + im;
04289         }
04290         else {
04291             poin2[ul] += im;
04292         }
04293         S->used[++level] = k;
04294         S->TermsLeft--;
04295     }
04296     else if ( !r31 ) {           /* copy coef into term1 */
04297         goto CopCof2;
04298     }
04299     else if ( r31 < 0 ) {        /* copy coef into term1
04300                                and adjust the length of term1 */
04301         goto CopCoef;
04302     }
04303     else {
04304         /*
04305            this is the dreaded calamity.
04306            is there enough space?
04307         */
04308         if( (poin[S->tree[i]]+l1+r31) >= poin2[S->tree[i]] ) {
04309             /*
04310                no space! now the special trick for which
04311                we left 2*maxing spaces open at the beginning
04312                of each patch.
04313             */
04314             if ( (l1 + r31) > AM.MaxTer/((LONG)sizeof(WORD)) ) {
04315                 MLOCK(ErrorMessageLock);
04316                 MesPrint("Coefficient overflow during sort");
04317                 MUNLOCK(ErrorMessageLock);
04318                 goto ReturnError;
04319             }
04320             m2 = poin[S->tree[i]];
04321             m3 = ( poin[S->tree[i]] -= r31 );
04322             do { *m3++ = *m2++; } while ( m2 < m1 );
04323             m1 = m3;
04324         }
04325         CopCoef:
04326             *(poin[S->tree[i]]) += r31;
04327         CopCof2:
04328             m2 = (WORD *)coef; im = r3;
04329             NCOPY(m1,m2,im);
04330             *m1 = r33;
04331         }
04332     }
04333     /*
04334        Now skip to the next term in stream k.
04335     */
04336     NextTerm:
04337         im = poin2[k][0];
04338         if ( im < 0 ) {
04339             r1 = poin2[k][1] - im + 1;
04340             m1 = poin2[k] + 2;
04341             m2 = poin[k] - im + 1;
04342             while ( ++im <= 0 ) *--m1 = *--m2;
04343             *--m1 = r1;
04344             poin2[k] = m1;
04345             im = r1;
04346         }
04347         poin[k] = poin2[k];
04348         ki = S->ktoi[k];
04349         if ( !par && ( (poin[k] + im + COMPINC) >= S->pStop[ki] )
04350             && im > 0 ) {
04351             #ifdef WITHZLIB
04352                 PutIn(fin,&(S->iPatches[ki]),S->Patches[ki],&(poin[k]),ki);
04353             #else
04354                 ADDPOS(S->iPatches[ki],PutIn(fin,&(S->iPatches[ki]),
04355                     S->Patches[ki],&(poin[k]),ki));
04356             #endif
04357             poin2[k] = poin[k] + im;
04358         }
04359         else {
04360             poin2[k] += im;
04361         }
04362         goto OneTerm;
04363     }
04364 }
04365     else if ( S->tree[i] < 0 ) {

```

```

04366         S->tree[i] = k;
04367         level--;
04368         goto OneTerm;
04369     }
04370 }
04371 /*
04372     found the smallest in the set. indicated by k.
04373     write to its destination.
04374 */
04375 #ifdef WITHPTHREADS
04376     if ( AS.MasterSort && ( fout == AR.outfile ) && S == AT.S0 ) {
04377         im = PutToMaster(BHEAD poin[k]);
04378     }
04379     else
04380 #endif
04381     if ( ( im = PutOut(BHEAD poin[k],&position,fout,1) ) < 0 ) {
04382         MLOCK(ErrorMessageLock);
04383         MesPrint("Called from MergePatches with k = %d (stream %d)",k,S->ktoi[k]);
04384         MUNLOCK(ErrorMessageLock);
04385         goto ReturnError;
04386     }
04387     ADDPOS(S->SizeInFile[par],im);
04388     goto NextTerm;
04389 }
04390 else {
04391     goto NormalReturn;
04392 }
04393 EndOfMerge:
04394 #ifdef WITHPTHREADS
04395     if ( AS.MasterSort && ( fout == AR.outfile ) && S == AT.S0 ) {
04396         PutToMaster(BHEAD 0);
04397     }
04398     else
04399 #endif
04400     if ( FlushOut(&position,fout,1) ) goto ReturnError;
04401     ADDPOS(S->SizeInFile[par],1);
04402 EndOfAll:
04403     if ( par == 1 ) { /* Set the fpatch pointers */
04404         #ifdef WITHZLIB
04405             SeekFile(fout->handle,&position,SEEK_CUR);
04406         #endif
04407         (S->fPatchN)++;
04408         S->fPatches[S->fPatchN] = position;
04409     }
04410     if ( par == 0 && fout != AR.outfile ) {
04411         /*
04412             Output went to sortfile. We have two possibilities:
04413             1: We are not finished with the current in-out cycle
04414                In that case we should pop to the next set of patches
04415             2: We finished a cycle and should clean up the in file
04416                Then we restart the sort.
04417         */
04418         (S->fPatchN)++;
04419         S->fPatches[S->fPatchN] = position;
04420         if ( ISNOTZEROPOS(AN.OldPosIn) ) { /* We are not done */
04421             SeekFile(fin->handle,&(AN.OldPosIn),SEEK_SET);
04422         }
04423         /*
04424             We don't need extra provisions for the zlib compression here.
04425             If part of an expression has been sorted, the whole has been so.
04426             This means that S->fpincompressed[] will remain the same
04427         */
04428         if ( (ULONG)ReadFile(fin->handle,(UBYTE *)(&(S->inNum)),(LONG)sizeof(WORD)) !=
04429             sizeof(WORD)
04430             || (ULONG)ReadFile(fin->handle,(UBYTE *)(&AN.OldPosIn),(LONG)sizeof(POSITION)) !=
04431             sizeof(POSITION)
04432             || (ULONG)ReadFile(fin->handle,(UBYTE *)S->iPatches,(LONG)((S->inNum)+1)
04433                 *sizeof(POSITION)) != ((S->inNum)+1)*sizeof(POSITION) ) {
04434             MLOCK(ErrorMessageLock);
04435             MesPrint("Read error fourth stage sorting");
04436             MUNLOCK(ErrorMessageLock);
04437             goto ReturnError;
04438         }
04439         *rr = 0;
04440         #ifdef WITHZLIB
04441         for ( i = 0; i < S->inNum; i++ ) {
04442             S->fPatchesStop[i] = S->iPatches[i+1];
04443         }
04444         #ifdef GZIPDEBUG
04445             MLOCK(ErrorMessageLock);
04446             MesPrint("%w fPatchesStop[%d] = %10p",i,&(S->fPatchesStop[i]));
04447             MUNLOCK(ErrorMessageLock);
04448         #endif
04449     }
04450     goto ConMer;
04451 }
04452 else {

```



```

04453 /*
04454         if ( fin == &(AR.FoStage4[0]) ) {
04455             s = (UBYTE *) (fin->name); while ( *s ) s++;
04456             if ( AR.Stage4Name == 1 ) s[-1] -= 1;
04457             else s[-1] += 1;
04458         }
04459 */
04460 /*         TruncateFile(fin->handle); */
04461         UpdateMaxSize();
04462 #ifdef WITHZLIB
04463         ClearSortGZIP(fin);
04464 #endif
04465         CloseFile(fin->handle);
04466         remove(fin->name); /* Gives disk space free again. */
04467 #ifdef GZIPDEBUG
04468         MLOCK(ErrorMessageLock);
04469         MesPrint("%w MergePatches removed in file %s", fin->name);
04470         MUNLOCK(ErrorMessageLock);
04471 #endif
04472 /*
04473         if ( fin == &(AR.FoStage4[0]) ) {
04474             s = (UBYTE *) (fin->name); while ( *s ) s++;
04475             if ( AR.Stage4Name == 1 ) s[-1] += 1;
04476             else s[-1] -= 1;
04477         }
04478 */
04479         fin->handle = -1;
04480         { FILEHANDLE *ff = fin; fin = fout; fout = ff; }
04481         PUTZERO(S->SizeInFile[0]);
04482         goto NewMerge;
04483     }
04484 }
04485 if ( par == 0 ) {
04486 /*         TruncateFile(fin->handle); */
04487         UpdateMaxSize();
04488 #ifdef WITHZLIB
04489         ClearSortGZIP(fin);
04490 #endif
04491         CloseFile(fin->handle);
04492         remove(fin->name);
04493         fin->handle = -1;
04494 #ifdef GZIPDEBUG
04495         MLOCK(ErrorMessageLock);
04496         MesPrint("%w MergePatches removed in file %s", fin->name);
04497         MUNLOCK(ErrorMessageLock);
04498 #endif
04499     }
04500 NormalReturn:
04501 #ifdef WITHZLIB
04502     AR.zipCompress = oldgzipCompress;
04503 #endif
04504     return(0);
04505 ReturnError:
04506 #ifdef WITHZLIB
04507     AR.zipCompress = oldgzipCompress;
04508 #endif
04509     return(-1);
04510 #ifndef WITHZLIB
04511 PatwCall:
04512     MLOCK(ErrorMessageLock);
04513     MesPrint("Error while writing to file.");
04514     goto PatCall2;
04515 #endif
04516 PatCall:;
04517     MLOCK(ErrorMessageLock);
04518 PatCall2:;
04519     MesCall("MergePatches");
04520     MUNLOCK(ErrorMessageLock);
04521 #ifdef WITHZLIB
04522     AR.zipCompress = oldgzipCompress;
04523 #endif
04524     SETERROR(-1)
04525 }
04526
04527 /*
04528     #] MergePatches :
04529     #[ StoreTerm : WORD StoreTerm(term)
04530 */
04540 WORD StoreTerm(PHEAD WORD *term)
04541 {
04542     GETBIDENTITY
04543     SORTING *S = AT.SS;
04544     WORD **ss, *lfill, j, *t;
04545     POSITION pp;
04546     LONG lSpace, sSpace, RetCode, over, tover;
04547
04548     if ( ( ( AP.PreDebug & DUMPTOSORT ) == DUMPTOSORT ) && AR.sLevel == 0 ) {

```

```

04549 #ifdef WITHPTHREADS
04550     snprintf((char *) (THRbuf), 100, "StoreTerm(%d)", AT.identity);
04551     PrintTerm(term, (char *) (THRbuf));
04552 #else
04553     PrintTerm(term, "StoreTerm");
04554 #endif
04555 }
04556 if ( AM.exitflag && AR.sLevel == 0 ) return(0);
04557 S->sFill = *(S->PoinFill);
04558 if ( S->sTerms >= S->TermsInSmall || ( S->sFill + *term ) >= S->sTop ) {
04559 /*
04560     The small buffer is full. It has to be sorted and written.
04561 */
04562     tover = over = S->sTerms;
04563     ss = S->sPointer;
04564     ss[over] = 0;
04565 #ifdef SPLITTIME
04566     PrintTime((UBYTE *) "Before SplitMerge");
04567 #endif
04568     ss[SplitMerge(BHEAD ss, over)] = 0;
04569 #ifdef SPLITTIME
04570     PrintTime((UBYTE *) "After SplitMerge");
04571 #endif
04572     sSpace = 0;
04573     if ( over > 0 ) {
04574         sSpace = ComPress(ss, &RetCode);
04575         S->TermsLeft -= over - RetCode;
04576     }
04577     sSpace++;
04578
04579     lSpace = sSpace + (S->lFill - S->lBuffer)
04580             - (AM.MaxTer/sizeof(WORD))*((LONG)S->lPatch);
04581     SETBASEPOSITION(pp, lSpace);
04582     MULPOS(pp, sizeof(WORD));
04583     if ( S->file.handle >= 0 ) {
04584         ADD2POS(pp, S->fPatches[S->fPatchN]);
04585     }
04586     if ( S == AT.S0 ) { /* Only statistics at ground level */
04587         WriteStats(&pp, STATSSPLITMERGE, CHECKLOGTYPE);
04588     }
04589     if ( ( S->lPatch >= S->MaxPatches ) ||
04590         ( ( WORD *) (((UBYTE *) (S->lFill + sSpace)) + 2*AM.MaxTer ) ) >= S->lTop ) ) {
04591 /*
04592     The large buffer is too full. Merge and write it
04593 */
04594     if ( MergePatches(1) ) goto StoreCall;
04595 /*
04596     pp = S->SizeInFile[1];
04597     ADDPOS(pp, sSpace);
04598     MULPOS(pp, sizeof(WORD));
04599 */
04600     SETBASEPOSITION(pp, sSpace);
04601     MULPOS(pp, sizeof(WORD));
04602     ADD2POS(pp, S->fPatches[S->fPatchN]);
04603
04604     if ( S == AT.S0 ) { /* Only statistics at ground level */
04605         WriteStats(&pp, STATSMERGETOFILE, CHECKLOGTYPE);
04606     }
04607     S->lPatch = 0;
04608     S->lFill = S->lBuffer;
04609 }
04610 S->Patches[S->lPatch++] = S->lFill;
04611 lfill = (WORD *) (((UBYTE *) (S->lFill)) + AM.MaxTer);
04612 if ( tover > 0 ) {
04613     ss = S->sPointer;
04614     while ( ( t = *ss++ ) != 0 ) {
04615         j = *t;
04616         if ( j < 0 ) j = t[1] + 2;
04617         while ( --j >= 0 ){
04618             *lfill++ = *t++;
04619         }
04620     }
04621 }
04622 *lfill++ = 0;
04623 S->lFill = lfill;
04624 S->sTerms = 0;
04625 S->PoinFill = S->sPointer;
04626 *(S->PoinFill) = S->sFill = S->sBuffer;
04627 }
04628 j = *term;
04629 while ( --j >= 0 ) *S->sFill++ = *term++;
04630 S->sTerms++;
04631 S->GenTerms++;
04632 S->TermsLeft++;
04633 **S->PoinFill = S->sFill;
04634
04635 return(0);

```

```

04636
04637 StoreCall:
04638     MLOCK(ErrorMessageLock);
04639     MesCall("StoreTerm");
04640     MUNLOCK(ErrorMessageLock);
04641     SETERROR(-1)
04642 }
04643
04644 /*
04645     #] StoreTerm :
04646     #[ StageSort :                void StageSort(FILEHANDLE *fout)
04647 */
04654 void StageSort(FILEHANDLE *fout)
04655 {
04656     GETIDENTITY
04657     SORTING *S = AT.SS;
04658     if ( S->fPatchN >= S->MaxFpatches ) {
04659         POSITION position;
04660         if ( S != AT.S0 ) {
04661             /*
04662                 There are no proper provisions for stage 4 or higher sorts
04663                 for function arguments and $ variables. The reason:
04664                 The current code maps out the patches, based on the size of
04665                 the buffers in the FoStage4 structs, while they are used
04666                 inside the S->file struct that may have far smaller buffers.
04667                 By itself that might still be repairable, but it goes completely
04668                 wrong when during the sort polyRatFuns have to be added and they
04669                 would go into stage4 (very rare but possible).
04670                 The only really correct solution would be to put FoStage4 structs
04671                 in all sort levels. Messy. (JV 8-oct-2018).
04672             */
04673             MLOCK(ErrorMessageLock);
04674             MesPrint("Currently Stage 4 sorts are not allowed for function arguments or $
variables.");
04675             MesPrint("Please increase correspondingsorting parameters (sub-) in the setup.");
04676             MUNLOCK(ErrorMessageLock);
04677             Terminate(-1);
04678         }
04679         PUTZERO(position);
04680         MLOCK(ErrorMessageLock);
04681         #ifdef WITHPTHREADS
04682             MesPrint("StageSort in thread %d",identity);
04683         #elif defined(WITHMPI)
04684             MesPrint("StageSort in process %d",PF.me);
04685         #else
04686             MesPrint("StageSort");
04687         #endif
04688         MUNLOCK(ErrorMessageLock);
04689         SeekFile(fout->handle,&position,SEEK_END);
04690         /*
04691             No extra compression data has to be written.
04692             S->fpincompressed should remain valid.
04693         */
04694         if ( (ULONG)WriteFile(fout->handle,(UBYTE *) (&(S->fPatchN)), (LONG)sizeof(WORD)) !=
sizeof(WORD)
|| (ULONG)WriteFile(fout->handle,(UBYTE *) (&(AN.OldPosOut)), (LONG)sizeof(POSITION)) !=
sizeof(POSITION)
|| (ULONG)WriteFile(fout->handle,(UBYTE *) (S->fPatches), (LONG) (S->fPatchN+1)
*sizeof(POSITION)) != (S->fPatchN+1)*sizeof(POSITION) ) {
04699             MLOCK(ErrorMessageLock);
04700             MesPrint("Write error while staging sort. Disk full?");
04701             MUNLOCK(ErrorMessageLock);
04702             Terminate(-1);
04703         }
04704         AN.OldPosOut = position;
04705         fout->filesize = position;
04706         ADDPOS(fout->filesize, (S->fPatchN+2)*sizeof(POSITION) + sizeof(WORD));
04707         fout->Pposition = fout->filesize;
04708         S->fPatches[0] = fout->filesize;
04709         S->fPatchN = 0;
04710
04711         if ( AR.FoStage4[0].PObuffer == 0 ) {
04712             AR.FoStage4[0].PObuffer = (WORD *)Malloca1(AR.FoStage4[0].POsize*sizeof(WORD)
,"Stage 4 buffer");
04714             AR.FoStage4[0].POfill = AR.FoStage4[0].PObuffer;
04715             AR.FoStage4[0].POstop = AR.FoStage4[0].PObuffer
+ AR.FoStage4[0].POsize/sizeof(WORD);
04717         #ifdef WITHPTHREADS
04718             AR.FoStage4[0].pthreadlock = dummylock;
04719         #endif
04720     }
04721     if ( AR.FoStage4[1].PObuffer == 0 ) {
04722         AR.FoStage4[1].PObuffer = (WORD *)Malloca1(AR.FoStage4[1].POsize*sizeof(WORD)
,"Stage 4 buffer");
04724         AR.FoStage4[1].POfill = AR.FoStage4[1].PObuffer;
04725         AR.FoStage4[1].POstop = AR.FoStage4[1].PObuffer
+ AR.FoStage4[1].POsize/sizeof(WORD);
04727

```

```

04728 #ifdef WITHPTHREADS
04729     AR.FoStage4[1].pthreadlock = dummylock;
04730 #endif
04731 }
04732 S->stage4 = 1;
04733 }
04734 }
04735
04736 /*
04737     #] StageSort :
04738     #[ SortWild :           WORD SortWild(w,nw)
04739 */
04753 WORD SortWild(WORD *w, WORD nw)
04754 {
04755     GETIDENTITY
04756     WORD *v, *s, *m, k, i;
04757     WORD *pScreat, *stop, *sv, error = 0;
04758     pScreat = AT.WorkPointer;
04759     if ( ( AT.WorkPointer + 8 * AM.MaxWildcards ) >= AT.WorkTop ) {
04760         MLOCK(ErrorMessageLock);
04761         MesWork();
04762         MUNLOCK(ErrorMessageLock);
04763         return(-1);
04764     }
04765     stop = w + nw;
04766     i = 0;
04767     while ( i < nw ) {
04768         m = w + i;
04769         v = m + m[1];
04770         while ( v < stop && (
04771             *v == FROMSET || *v == SETTONUM || *v == LOADDOLLAR ) ) v += v[1];
04772         while ( v < stop ) {
04773             if ( *v >= 0 ) {
04774                 if ( AM.Ordering[*v] < AM.Ordering[*m] ) {
04775                     m = v;
04776                 }
04777                 else if ( *v == *m ) {
04778                     if ( v[2] < m[2] ) {
04779                         m = v;
04780                     }
04781                     else if ( v[2] == m[2] ) {
04782                         s = m + m[1];
04783                         sv = v + v[1];
04784                         if ( s < stop && ( *s == FROMSET
04785                             || *s == SETTONUM || *s == LOADDOLLAR ) ) {
04786                             if ( sv < stop && ( *sv == FROMSET
04787                                 || *sv == SETTONUM || *sv == LOADDOLLAR ) ) {
04788                                 if ( s[2] != sv[2] ) {
04789                                     error = -1;
04790                                     MLOCK(ErrorMessageLock);
04791                                     MesPrint("&Wildcard set conflict");
04792                                     MUNLOCK(ErrorMessageLock);
04793                                 }
04794                             }
04795                             *v = -1;
04796                         }
04797                     }
04798                     else {
04799                         if ( sv < stop && ( *sv == FROMSET
04800                             || *sv == SETTONUM || *sv == LOADDOLLAR ) ) {
04801                             *m = -1;
04802                             m = v;
04803                         }
04804                         else {
04805                             *v = -1;
04806                         }
04807                     }
04808                 }
04809             }
04810             v += v[1];
04811             while ( v < stop && ( *v == FROMSET
04812                 || *v == SETTONUM || *v == LOADDOLLAR ) ) v += v[1];
04813         }
04814         s = pScreat;
04815         v = m;
04816         k = m[1];
04817         NCOPY(s,m,k);
04818         while ( m < stop && ( *m == FROMSET
04819             || *m == SETTONUM || *m == LOADDOLLAR ) ) {
04820             k = m[1];
04821             NCOPY(s,m,k);
04822         }
04823         *v = -1;
04824         pScreat = s;
04825         i = 0;
04826         while ( i < nw && ( w[i] < 0 || w[i] == FROMSET
04827             || w[i] == SETTONUM || w[i] == LOADDOLLAR ) ) i += w[i+1];

```

```

04828     }
04829     AC.NwildC = k = WORDDIFF(pScrat,AT.WorkPointer);
04830     s = AT.WorkPointer;
04831     m = w;
04832     NCOPY(m,s,k);
04833     AC.WildC = m;
04834     return(error);
04835 }
04836
04837 /*
04838     #] SortWild :
04839     #[ CleanupSort :          void CleanupSort(num)
04840 */
04841 void CleanupSort(int num)
04842 {
04843     GETIDENTITY
04844     SORTING *S;
04845     int minnum = num, i;
04846     if ( AN.FunSorts ) {
04847         if ( num == -1 ) {
04848             if ( AN.MaxFunSorts > 3 ) {
04849                 minnum = (AN.MaxFunSorts+4)/2;
04850             }
04851             else minnum = 4;
04852         }
04853         else if ( minnum == 0 ) minnum = 1;
04854         for ( i = minnum; i < AN.NumFunSorts; i++ ) {
04855             S = AN.FunSorts[i];
04856             if ( S ) {
04857                 if ( S->file.handle >= 0 ) {
04858                     /* TruncateFile(S->file.handle); */
04859                     UpdateMaxSize();
04860                     #ifdef WITHZLIB
04861                     ClearSortGZIP(&(S->file));
04862                     #endif
04863                     CloseFile(S->file.handle);
04864                     S->file.handle = -1;
04865                     remove(S->file.name);
04866                     #ifdef GZIPDEBUG
04867                     MLOCK(ErrorMessageLock);
04868                     MesPrint("%w CleanupSort removed file %s",S->file.name);
04869                     MUNLOCK(ErrorMessageLock);
04870                     #endif
04871                 }
04872                 M_free(S->sPointer, "CleanupSort: sPointer");
04873                 M_free(S->Patches, "CleanupSort: Patches");
04874                 M_free(S->pStop, "CleanupSort: pStop");
04875                 M_free(S->poina, "CleanupSort: poina");
04876                 M_free(S->poin2a, "CleanupSort: poin2a");
04877                 M_free(S->fPatches, "CleanupSort: fPatches");
04878                 M_free(S->fPatchesStop, "CleanupSort: fPatchesStop");
04879                 M_free(S->inPatches, "CleanupSort: inPatches");
04880                 M_free(S->tree, "CleanupSort: tree");
04881                 M_free(S->used, "CleanupSort: used");
04882                 #ifdef WITHZLIB
04883                 M_free(S->fpcompressed, "CleanupSort: fpcompressed");
04884                 M_free(S->fpincompressed, "CleanupSort: fpincompressed");
04885                 #endif
04886                 M_free(S->ktoi, "CleanupSort: ktoi");
04887                 M_free(S->lBuffer, "CleanupSort: lBuffer+sBuffer");
04888                 M_free(S->file.PObuffer, "CleanupSort: PObuffer");
04889                 M_free(S, "CleanupSort: sorting struct");
04890             }
04891             AN.FunSorts[i] = 0;
04892         }
04893         AN.MaxFunSorts = minnum;
04894         if ( num == 0 ) {
04895             S = AN.FunSorts[0];
04896             if ( S ) {
04897                 if ( S->file.handle >= 0 ) {
04898                     /* TruncateFile(S->file.handle); */
04899                     UpdateMaxSize();
04900                     #ifdef WITHZLIB
04901                     ClearSortGZIP(&(S->file));
04902                     #endif
04903                     CloseFile(S->file.handle);
04904                     S->file.handle = -1;
04905                     remove(S->file.name);
04906                     #ifdef GZIPDEBUG
04907                     MLOCK(ErrorMessageLock);
04908                     MesPrint("%w CleanupSort removed file %s",S->file.name);
04909                     MUNLOCK(ErrorMessageLock);
04910                     #endif
04911                 }
04912             }
04913         }
04914     }
04915 }
04916
04917 }
04918

```

```

04919     for ( i = 0; i < 2; i++ ) {
04920         if ( AR.FoStage4[i].handle >= 0 ) {
04921             UpdateMaxSize();
04922 #ifdef WITHZLIB
04923             ClearSortGZIP(&(AR.FoStage4[i]));
04924 #endif
04925             CloseFile(AR.FoStage4[i].handle);
04926             remove(AR.FoStage4[i].name);
04927             AR.FoStage4[i].handle = -1;
04928 #ifdef GZIPDEBUG
04929             MLOCK(ErrorMessageLock);
04930             MesPrint("%w CleanUpSort removed stage4 file %s",AR.FoStage4[i].name);
04931             MUNLOCK(ErrorMessageLock);
04932 #endif
04933         }
04934     }
04935 }
04936
04937 /*
04938     #] CleanUpSort :
04939     #[ LowerSortLevel :          void LowerSortLevel()
04940 */
04941 void LowerSortLevel(void)
04942 {
04943     GETIDENTITY
04944     if ( AR.sLevel >= 0 ) {
04945         AR.sLevel--;
04946         if ( AR.sLevel >= 0 ) AT.SS = AN.FunSorts[AR.sLevel];
04947     }
04948 }
04949
04950 /*
04951     #] LowerSortLevel :
04952     #[ PolyRatFunSpecial :
04953
04954     Keeps only the most divergent term in AR.PolyFunVar
04955     We assume that the terms are already in that notation.
04956 */
04957 WORD *PolyRatFunSpecial(PHEAD WORD *t1, WORD *t2)
04958 {
04959     WORD *oldworkpointer = AT.WorkPointer, *t, *r;
04960     WORD expl, exp2;
04961     int i;
04962     t = t1+FUNHEAD;
04963     if ( *t == -SYMBOL ) {
04964         if ( t[1] != AR.PolyFunVar ) goto Illegal;
04965         expl = 1;
04966         if ( t[2] != -SNUMBER ) goto Illegal;
04967         t[3] = 1;
04968     }
04969     else if ( *t == -SNUMBER ) {
04970         t[1] = 1;
04971         t += 2;
04972         if ( *t == -SYMBOL ) {
04973             if ( t[1] != AR.PolyFunVar ) goto Illegal;
04974             expl = -1;
04975         }
04976         else if ( *t == -SNUMBER ) {
04977             t[1] = 1;
04978             expl = 0;
04979         }
04980     }
04981     else if ( *t == ARGHEAD+8 && t[ARGHEAD] == 8 && t[ARGHEAD+1] == SYMBOL
04982             && t[ARGHEAD+3] == AR.PolyFunVar ) {
04983         t[ARGHEAD+5] = 1;
04984         t[ARGHEAD+6] = 1;
04985         t[ARGHEAD+7] = 3;
04986         expl = -t[ARGHEAD+4];
04987     }
04988     else goto Illegal;
04989 }
04990
04991 else if ( *t == ARGHEAD+8 && t[ARGHEAD] == 8 && t[ARGHEAD+1] == SYMBOL
04992         && t[ARGHEAD+3] == AR.PolyFunVar ) {
04993     t[ARGHEAD+5] = 1;
04994     t[ARGHEAD+6] = 1;
04995     t[ARGHEAD+7] = 3;
04996     expl = t[ARGHEAD+4];
04997     t += *t;
04998     if ( *t != -SNUMBER ) goto Illegal;
04999     t[1] = 1;
05000 }
05001
05002 else goto Illegal;
05003
05004 t = t2+FUNHEAD;
05005 if ( *t == -SYMBOL ) {
05006     if ( t[1] != AR.PolyFunVar ) goto Illegal;
05007     exp2 = 1;

```

```

05010         if ( t[2] != -SNUMBER ) goto Illegal;
05011         t[3] = 1;
05012     }
05013     else if ( *t == -SNUMBER ) {
05014         t[1] = 1;
05015         t += 2;
05016         if ( *t == -SYMBOL ) {
05017             if ( t[1] != AR.PolyFunVar ) goto Illegal;
05018             exp2 = -1;
05019         }
05020         else if ( *t == -SNUMBER ) {
05021             t[1] = 1;
05022             exp2 = 0;
05023         }
05024         else if ( *t == ARGHEAD+8 && t[ARGHEAD] == 8 && t[ARGHEAD+1] == SYMBOL
05025             && t[ARGHEAD+3] == AR.PolyFunVar ) {
05026             t[ARGHEAD+5] = 1;
05027             t[ARGHEAD+6] = 1;
05028             t[ARGHEAD+7] = 3;
05029             exp2 = -t[ARGHEAD+4];
05030         }
05031         else goto Illegal;
05032     }
05033     else if ( *t == ARGHEAD+8 && t[ARGHEAD] == 8 && t[ARGHEAD+1] == SYMBOL
05034         && t[ARGHEAD+3] == AR.PolyFunVar ) {
05035         t[ARGHEAD+5] = 1;
05036         t[ARGHEAD+6] = 1;
05037         t[ARGHEAD+7] = 3;
05038         exp2 = t[ARGHEAD+4];
05039         t += *t;
05040         if ( *t != -SNUMBER ) goto Illegal;
05041         t[1] = 1;
05042     }
05043     else goto Illegal;
05044
05045     if ( exp1 <= exp2 ) { i = t1[1]; r = t1; }
05046     else { i = t2[1]; r = t2; }
05047     t = oldworkpointer;
05048     NCOPY(t,r,i)
05049
05050     return(oldworkpointer);
05051 Illegal:
05052     MesPrint("Illegal occurrence of PolyRatFun with divergent option");
05053     Terminate(-1);
05054     return(0);
05055 }
05056
05057 /*
05058     #] PolyRatFunSpecial :
05059     #[ SimpleSplitMerge :
05060
05061     Sorts an array of WORDs. No adding of equal objects.
05062 */
05063
05064 void SimpleSplitMergeRec(WORD *array,WORD num,WORD *auxarray)
05065 {
05066     WORD n1,n2,i,j,k,*t1,*t2;
05067     if ( num < 2 ) return;
05068     if ( num == 2 ) {
05069         if ( array[0] > array[1] ) {
05070             EXCH(array[0],array[1])
05071         }
05072         return;
05073     }
05074     n1 = num/2;
05075     n2 = num - n1;
05076     SimpleSplitMergeRec(array,n1,auxarray);
05077     SimpleSplitMergeRec(array+n1,n2,auxarray);
05078     if ( array[n1-1] <= array[n1] ) return;
05079
05080     t1 = array; t2 = auxarray; i = n1; NCOPY(t2,t1,i);
05081     i = 0; j = n1; k = 0;
05082     while ( i < n1 && j < num ) {
05083         if ( auxarray[i] <= array[j] ) { array[k++] = auxarray[i++]; }
05084         else { array[k++] = array[j++]; }
05085     }
05086     while ( i < n1 ) array[k++] = auxarray[i++];
05087 /*
05088     Remember: remnants of j are still in place!
05089 */
05090 }
05091
05092 void SimpleSplitMerge(WORD *array,WORD num)
05093 {
05094     WORD *auxarray = Malloc1(sizeof(WORD)*num/2,"SimpleSplitMerge");
05095     SimpleSplitMergeRec(array,num,auxarray);
05096     M_free(auxarray,"SimpleSplitMerge");

```

```

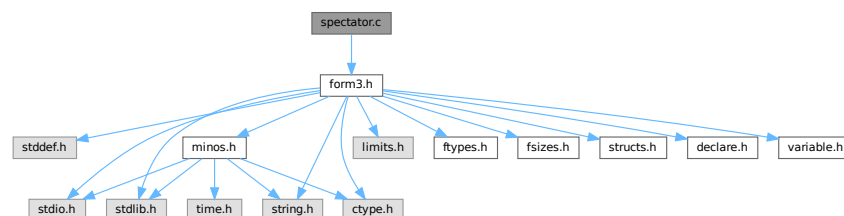
05097 }
05098
05099 /*
05100      #] SimpleSplitMerge :
05101      #[ BinarySearch :
05102
05103      Searches in the sorted array with length num for the object x.
05104      If x is in the list, it returns the number of the array element
05105      that matched. If it is not in the list, it returns -1.
05106      If there are identical objects in the list, which one will
05107      match is quasi random.
05108 */
05109
05110 WORD BinarySearch(WORD *array,WORD num,WORD x)
05111 {
05112     WORD i, bot, top, med;
05113     if ( num < 8 ) {
05114         for ( i = 0; i < num; i++ ) if ( array[i] == x ) return(i);
05115         return(-1);
05116     }
05117     if ( array[0] > x || array[num-1] < x ) return(-1);
05118     bot = 0; top = num-1; med = (top+bot)/2;
05119     do {
05120         if ( array[med] == x ) return(med);
05121         if ( array[med] < x ) { bot = med+1; }
05122         else { top = med-1; }
05123         med = (top+bot)/2;
05124     } while ( med >= bot && med <= top );
05125     return(-1);
05126 }
05127
05128 /*
05129      #] BinarySearch :
05130      #] SortUtilities :
05131 */

```

4.130 spectator.c File Reference

#include "form3.h"

Include dependency graph for spectator.c:



Functions

- int [CoCreateSpectator](#) (UBYTE *inp)
- int [CoToSpectator](#) (UBYTE *inp)
- int [CoRemoveSpectator](#) (UBYTE *inp)
- int [CoEmptySpectator](#) (UBYTE *inp)
- int [PutInSpectator](#) (WORD *term, WORD specnum)
- void [FlushSpectators](#) (void)
- int [CoCopySpectator](#) (UBYTE *inp)
- WORD [GetFromSpectator](#) (WORD *term, WORD specnum)
- void [ClearSpectators](#) (WORD par)

4.130.1 Detailed Description

File contains the code for the spectator files and their control.

Definition in file [spectator.c](#).

4.130.2 Function Documentation

4.130.2.1 CoCreateSpectator()

```
int CoCreateSpectator (
    UBYTE * inp )
```

Definition at line [89](#) of file [spectator.c](#).

4.130.2.2 CoToSpectator()

```
int CoToSpectator (
    UBYTE * inp )
```

Definition at line [181](#) of file [spectator.c](#).

4.130.2.3 CoRemoveSpectator()

```
int CoRemoveSpectator (
    UBYTE * inp )
```

Definition at line [233](#) of file [spectator.c](#).

4.130.2.4 CoEmptySpectator()

```
int CoEmptySpectator (
    UBYTE * inp )
```

Definition at line [293](#) of file [spectator.c](#).

4.130.2.5 PutInSpectator()

```
int PutInSpectator (
    WORD * term,
    WORD specnum )
```

Definition at line [345](#) of file [spectator.c](#).

4.130.2.6 FlushSpectators()

```
void FlushSpectators (
    void )
```

Definition at line 402 of file [spectator.c](#).

4.130.2.7 CoCopySpectator()

```
int CoCopySpectator (
    UBYTE * inp )
```

Definition at line 459 of file [spectator.c](#).

4.130.2.8 GetFromSpectator()

```
WORD GetFromSpectator (
    WORD * term,
    WORD specnum )
```

Definition at line 601 of file [spectator.c](#).

4.130.2.9 ClearSpectators()

```
void ClearSpectators (
    WORD par )
```

Definition at line 675 of file [spectator.c](#).

4.131 spectator.c

[Go to the documentation of this file.](#)

```
00001
00006 /*  #[ License : */
00007 /*
00008 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00009 *   When using this file you are requested to refer to the publication
00010 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00011 *   This is considered a matter of courtesy as the development was paid
00012 *   for by FOM the Dutch physics granting agency and we would like to
00013 *   be able to track its scientific use to convince FOM of its value
00014 *   for the community.
00015 *
00016 *   This file is part of FORM.
00017 *
00018 *   FORM is free software: you can redistribute it and/or modify it under the
00019 *   terms of the GNU General Public License as published by the Free Software
00020 *   Foundation, either version 3 of the License, or (at your option) any later
00021 *   version.
00022 *
00023 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00024 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00025 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00026 *   details.
00027 *
00028 *   You should have received a copy of the GNU General Public License along
00029 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00030 */
00031 /*  #] License : */
```

```

00032 /*
00033     #[ includes :
00034 */
00035
00036 #include "form3.h"
00037
00038 /*
00039     #] includes :
00040     #[ Commentary :
00041
00042     We use an array of SPECTATOR structs in AM.SpectatorFiles.
00043     When a spectator is removed this leaves a hole. This means that
00044     we cannot use AM.NumSpectatorFiles but always have to scan up to
00045     AM.SizeForSpectatorFiles which is the size of the array.
00046     An element is in use when it has a name. This is the name of the
00047     expression that is associated with it. There is also the number of
00048     the expression, but because the expressions are frequently renumbered
00049     at the end of a module, we always search for the spectators by name.
00050     The expression number is only valid in the current module.
00051     During execution we use the number of the spectator.
00052
00053     The FILEHANDLE struct is borrowed from the structs for the scratch
00054     files, but we do not keep copies for all workers as with the scratch
00055     files. This brings some limitations (but saves much space). Basically
00056     the reading can only be done by one master or worker. And because
00057     we use the buffer both for writing and for reading we cannot read and
00058     write in the same module.
00059
00060     Processor can see that an expression is to be filled with a spectator
00061     because we replace the compiler buffer number in the prototype by
00062     -specnum-1. Of course, after this filling has taken place we should
00063     make sure that in the next module there is a nonnegative number there.
00064     The input is then obtained from GetFromSpectator instead from GetTerm.
00065     This needed an extra argument in ThreadsProcessor. InParallelProcessor
00066     can figure it out by itself. ParFORM still needs to be treated for this.
00067
00068     The writing is to a single buffer. Hence it needs a lock. It is possible
00069     to give all workers their own buffers (at great memory cost) and merge
00070     the results when needed. That would be friendlier on ParFORM. We ALWAYS
00071     assume that the order of the terms in the spectator file is random.
00072
00073     In the first version there is no compression in the file. This could
00074     change in later versions because both the writing and the reading are
00075     purely sequential. Brackets are not possible.
00076
00077     Currently, ParFORM allows use of spectators only in the sequential
00078     mode. The parallelization is switched off in modules containing
00079     ToSpectator or CopySpectator. Workers never create or access to
00080     spectator files. Their file handles are always -1. We leave the
00081     parallelization of modules with spectators for future work.
00082
00083     #] Commentary :
00084     #[ CoCreateSpectator :
00085
00086     Syntax: CreateSpectator name_of_expr "filename";
00087 */
00088
00089 int CoCreateSpectator(UBYTE *inp)
00090 {
00091     UBYTE *p, *q, *filename, c, cc;
00092     WORD c1, c2, numexpr = 0, specnum, HadOne = 0;
00093     FILEHANDLE *fh;
00094     while ( (*inp == ',' ) inp++ );
00095     if ( ( q = SkipAName(inp) ) == 0 || q[-1] == '_' ) {
00096         MesPrint("&Illegal name for expression");
00097         return(1);
00098     }
00099     c = *q; *q = 0;
00100     if ( GetVar(inp,&c1,&c2,ALLVARIABLES,NOAUTO) != NAMENOTFOUND ) {
00101         if ( c2 == CEXPRESSION &&
00102             Expressions[c1].status == DROPSPECTATOREXPRESSION ) {
00103             numexpr = c1;
00104             Expressions[numexpr].status = SPECTATOREXPRESSION;
00105             HadOne = 1;
00106         }
00107         else {
00108             MesPrint("&The name %s has been used already.",inp);
00109             *q = c;
00110             return(1);
00111         }
00112     }
00113     p = q+1;
00114     while ( *p == ',' ) p++;
00115     if ( *p != '"' ) goto Syntax;
00116     p++; filename = p;
00117     while ( *p && *p != '"' ) {
00118         if ( *p == '\\\\' ) p++;

```

```

00119         p++;
00120     }
00121     if ( *p != '"' ) goto Syntax;
00122     q = p+1;
00123     while ( *q && ( *q == ',' || *q == ' ' || *q == '\t' ) ) q++;
00124     if ( *q ) goto Syntax;
00125     cc = *p; *p = 0;
00126     /*
00127     Now we need to: create a struct for the spectator file.
00128     */
00129     if ( HadOne == 0 )
00130         numexpr = EntVar(CEXPRESSION,inp,SPECTATOREXPRESSION,0,0,0);
00131     fh = AllocFileHandle(1,(char *)filename);
00132     /*
00133     Make sure there is space in the AM.spectatorfiles array
00134     */
00135     if ( AM.NumSpectatorFiles >= AM.SizeForSpectatorFiles || AM.SpectatorFiles == 0 ) {
00136         int newsize, i;
00137         SPECTATOR *newspectators;
00138         if ( AM.SizeForSpectatorFiles == 0 ) {
00139             newsize = 10;
00140             AM.NumSpectatorFiles = AM.SizeForSpectatorFiles = 0;
00141         }
00142         else newsize = AM.SizeForSpectatorFiles*2;
00143         newspectators = (SPECTATOR *)Malloc1(newsize*sizeof(SPECTATOR),"Spectators");
00144         for ( i = 0; i < AM.NumSpectatorFiles; i++ )
00145             newspectators[i] = AM.SpectatorFiles[i];
00146         for ( ; i < newsize; i++ ) {
00147             newspectators[i].fh = 0;
00148             newspectators[i].name = 0;
00149             newspectators[i].exprnumber = -1;
00150             newspectators[i].flags = 0;
00151             PUTZERO(newspectators[i].position);
00152             PUTZERO(newspectators[i].readpos);
00153         }
00154         AM.SizeForSpectatorFiles = newsize;
00155         if ( AM.SpectatorFiles != 0 ) M_free(AM.SpectatorFiles,"Spectators");
00156         AM.SpectatorFiles = newspectators;
00157         specnum = AM.NumSpectatorFiles++;
00158     }
00159     else {
00160         for ( specnum = 0; specnum < AM.SizeForSpectatorFiles; specnum++ ) {
00161             if ( AM.SpectatorFiles[specnum].name == 0 ) break;
00162         }
00163         AM.NumSpectatorFiles++;
00164     }
00165     PUTZERO(AM.SpectatorFiles[specnum].position);
00166     AM.SpectatorFiles[specnum].name = (char *) (strDup1(inp,"Spectator expression name"));
00167     AM.SpectatorFiles[specnum].fh = fh;
00168     AM.SpectatorFiles[specnum].exprnumber = numexpr;
00169     *p = cc;
00170     return(0);
00171 Syntax:
00172     MesPrint("&Proper syntax is: CreateSpectator,exprname,\"filename\";");
00173     return(-1);
00174 }
00175
00176 /*
00177 #] CoCreateSpectator :
00178 #[ CoToSpectator :
00179 */
00180
00181 int CoToSpectator(UBYTE *inp)
00182 {
00183     UBYTE *q;
00184     WORD c1, numexpr;
00185     int i;
00186     while ( *inp == ',' ) inp++;
00187     if ( ( q = SkipAName(inp) ) == 0 || q[-1] == '_' ) {
00188         MesPrint("&Illegal name for expression");
00189         return(1);
00190     }
00191     if ( *q != 0 ) goto Syntax;
00192     if ( GetVar(inp,&c1,&numexpr,ALLVARIABLES,NOAUTO) == NAMENOTFOUND ||
00193         c1 != CEXPRESSION ) {
00194         MesPrint("&%s is not a valid expression.",inp);
00195         return(1);
00196     }
00197     if ( Expressions[numexpr].status != SPECTATOREXPRESSION ) {
00198         MesPrint("&%s is not an active spectator.",inp);
00199         return(1);
00200     }
00201     for ( i = 0; i < AM.SizeForSpectatorFiles; i++ ) {
00202         if ( AM.SpectatorFiles[i].name != 0 ) {
00203             if ( StrCmp((UBYTE *) (AM.SpectatorFiles[i].name), (UBYTE *) (inp)) == 0 ) break;
00204         }
00205     }

```

```

00206     if ( i >= AM.SizeForSpectatorFiles ) {
00207         MesPrint("&Spectator %s not found.",inp);
00208         return(1);
00209     }
00210     if ( ( AM.SpectatorFiles[i].flags & READSPECTATORFLAG ) != 0 ) {
00211         MesPrint("&Spectator %s: It is not permitted to read from and write to the same spectator in
one module.",inp);
00212         return(1);
00213     }
00214     AM.SpectatorFiles[i].exprnumber = numexpr;
00215     Add3Com(TYPETOSPECTATOR,i);
00216 #ifdef WITHMPI
00217     /*
00218      * In ParFORM, ToSpectator has to be executed on the master.
00219      */
00220     AC.mparallelf1ag |= NOPARALLEL_SPECTATOR;
00221 #endif
00222     return(0);
00223 Syntax:
00224     MesPrint("&Proper syntax is: ToSpectator,exprname;");
00225     return(-1);
00226 }
00227
00228 /*
00229 #] CoToSpectator :
00230 #[ CoRemoveSpectator :
00231 */
00232
00233 int CoRemoveSpectator(UBYTE *inp)
00234 {
00235     UBYTE *q;
00236     WORD c1, numexpr;
00237     int i;
00238     SPECTATOR *sp;
00239     while ( *inp == ',' ) inp++;
00240     if ( ( q = SkipAName(inp) ) == 0 || q[-1] == '_' ) {
00241         MesPrint("&Illegal name for expression");
00242         return(1);
00243     }
00244     if ( *q != 0 ) goto Syntax;
00245     if ( GetVar(inp,&c1,&numexpr,ALLVARIABLES,NOAUTO) == NAMENOTFOUND ||
00246         c1 != CEXPRESSION ) {
00247         MesPrint("&%s is not a valid expression.",inp);
00248         return(1);
00249     }
00250     if ( Expressions[numexpr].status != SPECTATOREXPRESSION ) {
00251         MesPrint("&%s is not a spectator.",inp);
00252         return(1);
00253     }
00254     for ( i = 0; i < AM.SizeForSpectatorFiles; i++ ) {
00255         if ( AM.SpectatorFiles[i].name != 0 ) {
00256             if ( StrCmp((UBYTE *) (AM.SpectatorFiles[i].name), (UBYTE *) (inp)) == 0 ) {
00257                 break;
00258             }
00259         }
00260     }
00261     if ( i >= AM.SizeForSpectatorFiles ) {
00262         MesPrint("&Spectator %s not found.",inp);
00263         return(1);
00264     }
00265     sp = AM.SpectatorFiles+i;
00266     Expressions[numexpr].status = DROPSPECTATOREXPRESSION;
00267     if ( sp->fh->handle != -1 ) {
00268         CloseFile(sp->fh->handle);
00269         sp->fh->handle = -1;
00270         remove(sp->fh->name);
00271     }
00272     M_free(sp->fh,"Temporary FileHandle");
00273     M_free(sp->name,"Spectator expression name");
00274
00275     PUTZERO(sp->position);
00276     PUTZERO(sp->readpos);
00277     sp->fh = 0;
00278     sp->name = 0;
00279     sp->exprnumber = -1;
00280     sp->flags = 0;
00281     AM.NumSpectatorFiles--;
00282     return(0);
00283 Syntax:
00284     MesPrint("&Proper syntax is: RemoveSpectator,exprname;");
00285     return(-1);
00286 }
00287
00288 /*
00289 #] CoRemoveSpectator :
00290 #[ CoEmptySpectator :
00291 */

```

```

00292
00293 int CoEmptySpectator(UBYTE *inp)
00294 {
00295     UBYTE *q;
00296     WORD c1, numexpr;
00297     int i;
00298     SPECTATOR *sp;
00299     while ( *inp == ',' ) inp++;
00300     if ( ( q = SkipAName(inp) ) == 0 || q[-1] == '_' ) {
00301         MesPrint("&Illegal name for expression");
00302         return(1);
00303     }
00304     if ( *q != 0 ) goto Syntax;
00305     if ( GetVar(inp,&c1,&numexpr,ALLVARIABLES,NOAUTO) == NAMENOTFOUND ||
00306         c1 != CEXPRESSION ) {
00307         MesPrint("&%s is not a valid expression.",inp);
00308         return(1);
00309     }
00310     if ( Expressions[numexpr].status != SPECTATOREXPRESSION ) {
00311         MesPrint("&%s is not a spectator.",inp);
00312         return(1);
00313     }
00314     for ( i = 0; i < AM.SizeForSpectatorFiles; i++ ) {
00315         if ( StrCmp((UBYTE *) (AM.SpectatorFiles[i].name), (UBYTE *) (inp)) == 0 ) break;
00316     }
00317     if ( i >= AM.SizeForSpectatorFiles ) {
00318         MesPrint("&Spectator %s not found.",inp);
00319         return(1);
00320     }
00321     sp = AM.SpectatorFiles+i;
00322     if ( sp->fh->handle != -1 ) {
00323         CloseFile(sp->fh->handle);
00324         sp->fh->handle = -1;
00325         remove(sp->fh->name);
00326     }
00327     sp->fh->POfill = sp->fh->POfull = sp->fh->PObuffer;
00328     PUTZERO(sp->position);
00329     PUTZERO(sp->readpos);
00330     return(0);
00331 Syntax:
00332     MesPrint("&Proper syntax is: EmptySpectator,exprname;");
00333     return(-1);
00334 }
00335
00336 /*
00337 #] CoEmptySpectator :
00338 #[ PutInSpectator :
00339
00340 We need to use locks! There is only one file.
00341 The code was copied (and modified) from PutOut.
00342 Here we use no compression.
00343 */
00344
00345 int PutInSpectator(WORD *term,WORD specnum)
00346 {
00347     GETBIDENTITY
00348     WORD i, *p, ret;
00349     LONG RetCode;
00350     SPECTATOR *sp = &(AM.SpectatorFiles[specnum]);
00351     FILEHANDLE *fi = sp->fh;
00352
00353     if ( ( i = *term ) <= 0 ) return(0);
00354     LOCK(fi->pthreadlock);
00355     ret = i;
00356     p = fi->POfill;
00357     do {
00358         if ( p >= fi->POstop ) {
00359             if ( fi->handle < 0 ) {
00360                 if ( ( RetCode = CreateFile(fi->name) ) >= 0 ) {
00361                     fi->handle = (WORD)RetCode;
00362                     PUTZERO(fi->filesize);
00363                     PUTZERO(fi->POposition);
00364                 }
00365                 else {
00366                     MLOCK(ErrorMessageLock);
00367                     MesPrint("Cannot create spectator file %s",fi->name);
00368                     MUNLOCK(ErrorMessageLock);
00369                     UNLOCK(fi->pthreadlock);
00370                     return(-1);
00371                 }
00372             }
00373             SeekFile(fi->handle,&(sp->position),SEEK_SET);
00374             if ( ( RetCode = WriteFile(fi->handle, (UBYTE *) (fi->PObuffer),fi->POsize) ) != fi->POsize
00375         ) {
00376             MLOCK(ErrorMessageLock);
00377             MesPrint("Error during spectator write. Disk full?");
00378             MesPrint("Attempt to write %l bytes on file %d at position %l5p",

```

```

00378             fi->POsize,fi->handle,&(fi->POposition));
00379             MesPrint("RetCode = %1, Buffer address = %1",RetCode,(LONG)(fi->PObuffer));
00380             MUNLOCK(ErrorMessageLock);
00381             UNLOCK(fi->pthreadlock);
00382             return(-1);
00383         }
00384         ADDPOS(fi->filesize,fi->POsize);
00385         p = fi->PObuffer;
00386         ADDPOS(sp->position,fi->POsize);
00387         fi->POposition = sp->position;
00388     }
00389     *p++ = *term++;
00390 } while ( --i > 0 );
00391 fi->POfull = fi->POfill = p;
00392 Expressions[AM.SpectatorFiles[specnum].exprnumber].counter++;
00393 UNLOCK(fi->pthreadlock);
00394 return(ret);
00395 }
00396
00397 /*
00398 #] PutInSpectator :
00399 #[ FlushSpectators :
00400 */
00401
00402 void FlushSpectators(void)
00403 {
00404     SPECTATOR *sp = AM.SpectatorFiles;
00405     FILEHANDLE *fh;
00406     LONG RetCode;
00407     int i;
00408     LONG size;
00409     if ( AM.NumSpectatorFiles <= 0 ) return;
00410     for ( i = 0; i < AM.SizeForSpectatorFiles; i++, sp++ ) {
00411         if ( sp->name == 0 ) continue;
00412         fh = sp->fh;
00413         if ( ( sp->flags & READSPECTATORFLAG ) != 0 ) { /* reset for writing */
00414             sp->flags &= ~READSPECTATORFLAG;
00415             fh->POfill = fh->PObuffer;
00416             if ( fh->handle >= 0 ) {
00417                 SeekFile(fh->handle,&(sp->position),SEEK_SET);
00418                 fh->POposition = sp->position;
00419             }
00420             continue;
00421         }
00422         if ( fh->POfill <= fh->PObuffer ) continue; /* is clean */
00423         if ( fh->handle < 0 ) { /* File needs to be created */
00424             if ( ( RetCode = CreateFile(fh->name) ) >= 0 ) {
00425                 PUTZERO(fh->filesize);
00426                 PUTZERO(fh->POposition);
00427                 fh->handle = (WORD)RetCode;
00428             }
00429             else {
00430                 MLOCK(ErrorMessageLock);
00431                 MesPrint("Cannot create spectator file %s",fh->name);
00432                 MUNLOCK(ErrorMessageLock);
00433                 Terminate(-1);
00434             }
00435             PUTZERO(sp->position);
00436         }
00437         SeekFile(fh->handle,&(sp->position),SEEK_SET);
00438         size = (fh->POfill - fh->PObuffer)*sizeof(WORD);
00439         if ( ( RetCode = WriteFile(fh->handle,(UBYTE *) (fh->PObuffer),size) ) != size ) {
00440             MLOCK(ErrorMessageLock);
00441             MesPrint("Write error synching spectator file. Disk full?");
00442             MesPrint("Attempt to write %1 bytes on file %s at position %15p",
00443                 size,fh->name,&(sp->position));
00444             MUNLOCK(ErrorMessageLock);
00445             Terminate(-1);
00446         }
00447         fh->POfill = fh->PObuffer;
00448         SeekFile(fh->handle,&(sp->position),SEEK_END);
00449         fh->POposition = sp->position;
00450     }
00451     return;
00452 }
00453
00454 /*
00455 #] FlushSpectators :
00456 #[ CoCopySpectator :
00457 */
00458
00459 int CoCopySpectator(UBYTE *inp)
00460 {
00461     GETIDENTITY
00462     UBYTE *q, c, *exprname, *p;
00463     WORD c1, c2, numexpr;
00464     int specnum, error = 0;

```

```

00465     SPECTATOR *sp;
00466     while ( *inp == ',' ) inp++;
00467     if ( ( q = SkipAName(inp) ) == 0 || q[-1] == '_' ) {
00468         MesPrint("&Illegal name for expression");
00469         return(1);
00470     }
00471     if ( *q == 0 ) goto Syntax;
00472     c = *q; *q = 0;
00473     if ( GetVar(inp,&c1,&c2,ALLVARIABLES,NOAUTO) != NAMENOTFOUND ) {
00474         MesPrint("&%s is the name of an existing variable.",inp);
00475         return(1);
00476     }
00477     numexpr = EntVar(CEXPRESSION,inp,LOCALEXPRESSION,0,0,0);
00478     p = q;
00479     exprname = inp;
00480     *q = c;
00481     while ( *q == ' ' || *q == ',' || *q == '\t' ) q++;
00482     if ( *q != '=' ) goto Syntax;
00483     q++;
00484     while ( *q == ' ' || *q == ',' || *q == '\t' ) q++;
00485     inp = q;
00486     if ( ( q = SkipAName(inp) ) == 0 || q[-1] == '_' ) {
00487         MesPrint("&Illegal name for spectator expression");
00488         return(1);
00489     }
00490     if ( *q != 0 ) goto Syntax;
00491     if ( AM.NumSpectatorFiles <= 0 ) {
00492         MesPrint("&CopySpectator: There are no spectator expressions!");
00493         return(1);
00494     }
00495     sp = AM.SpectatorFiles;
00496     for ( specnum = 0; specnum < AM.SizeForSpectatorFiles; specnum++, sp++ ) {
00497         if ( sp->name != 0 ) {
00498             if ( StrCmp((UBYTE *) (sp->name), (UBYTE *) (inp)) == 0 ) break;
00499         }
00500     }
00501     if ( specnum >= AM.SizeForSpectatorFiles ) {
00502         MesPrint("&Spectator %s not found.",inp);
00503         return(1);
00504     }
00505     sp->flags |= READSPECTATORFLAG;
00506     PUTZERO(sp->fh->Pposition);
00507     PUTZERO(sp->readpos);
00508     sp->fh->Pofill = sp->fh->PObuffer;
00509     if ( sp->fh->handle >= 0 ) {
00510         SeekFile(sp->fh->handle, &(sp->fh->Pposition), SEEK_SET);
00511     }
00512     /*
00513     Now we have:
00514     1: The name of the target expression: numexpr
00515     2: The spectator: sp (or specnum).
00516     Time for some action. We need:
00517     a: Write a prototype to create the expression
00518     b: Signal to Processor that this is a spectator.
00519     We do this by giving a negative compiler buffer number.
00520     */
00521     {
00522         WORD *OldWork, *w;
00523         POSITION pos;
00524         OldWork = w = AT.WorkPointer;
00525         *w++ = TYPEEXPRESSION;
00526         *w++ = 3+SUBEXPSIZE;
00527         *w++ = numexpr;
00528         AC.ProtoType = w;
00529         AR.CurExpr = numexpr; /* Block expression numexpr */
00530         *w++ = SUBEXPRESSION;
00531         *w++ = SUBEXPSIZE;
00532         *w++ = numexpr;
00533         *w++ = 1;
00534         *w++ = -specnum-1; /* Indicates "spectator" to Processor */
00535         FILLSUB(w)
00536         *w++ = 1;
00537         *w++ = 1;
00538         *w++ = 3;
00539         *w++ = 0;
00540         SeekScratch(AR.outfile,&pos);
00541         Expressions[numexpr].counter = 1;
00542         Expressions[numexpr].onfile = pos;
00543         Expressions[numexpr].whichbuffer = 0;
00544     #ifdef PARALLELLECODE
00545         Expressions[numexpr].partodo = AC.inparallelflag;
00546     #endif
00547         OldWork[2] = w - OldWork - 3;
00548         AT.WorkPointer = w;
00549
00550         if ( PutOut(BHEAD OldWork+2,&pos,AR.outfile,0) < 0 ) {
00551             c = *p; *p = 0;

```



```

00552         MesPrint("&Cannot create expression %s",exprname);
00553         *p = c;
00554         error = -1;
00555     }
00556     else {
00557         OldWork[2] = 4+SUBEXPSIZE;
00558         OldWork[4] = SUBEXPSIZE;
00559         OldWork[5] = numexpr;
00560         OldWork[SUBEXPSIZE+3] = 1;
00561         OldWork[SUBEXPSIZE+4] = 1;
00562         OldWork[SUBEXPSIZE+5] = 3;
00563         OldWork[SUBEXPSIZE+6] = 0;
00564         if ( PutOut (BHEAD OldWork+2,&pos,AR.outfile,0) < 0
00565             || FlushOut (&pos,AR.outfile,0) ) {
00566             c = *p; *p = 0;
00567             MesPrint("&Cannot create expression %s",exprname);
00568             *p = c;
00569             error = -1;
00570         }
00571         AR.outfile->POfull = AR.outfile->POfill;
00572     }
00573     OldWork[2] = numexpr;
00574     /*
00575     Seems unnecessary (13-feb-2018)
00576
00577     AddNtoL(OldWork[1],OldWork);
00578 */
00579     AT.WorkPointer = OldWork;
00580     if ( AC.dumnumflag ) Add2Com(TYPEDETCURDUM)
00581 }
00582 #ifdef WITHMPI
00583 /*
00584  * In ParFORM, substitutions of spectators has to be done on the master.
00585  */
00586 AC.mparallelflag |= NOPARALLEL_SPECTATOR;
00587 #endif
00588 return(error);
00589 Syntax:
00590 MesPrint("&Proper syntax is: CopySpectator,exprname=spectatorname,");
00591 return(-1);
00592 }
00593
00594 /*
00595 #] CoCopySpectator :
00596 #[ GetFromSpectator :
00597
00598 Note that if we did things right, we do not need a lock for the reading.
00599 */
00600
00601 WORD GetFromSpectator(WORD *term,WORD specnum)
00602 {
00603     SPECTATOR *sp = &(AM.SpectatorFiles[specnum]);
00604     FILEHANDLE *fh = sp->fh;
00605     WORD i, size, *t = term;
00606     LONG InIn;
00607     if ( fh-> handle < 0 ) {
00608         *term = 0;
00609         return(0);
00610     }
00611     /*
00612     sp->position marks the 'end' of the file: the point where writing should
00613     take place. sp->readpos marks from where to read.
00614     fh->POposition marks where the file is currently positioned.
00615     Note that when we read, we need to
00616     */
00617     if ( ISZEROPOS(sp->readpos) ) { /* we start reading. Fill buffer. */
00618     FillBuffer:
00619         SeekFile(fh->handle,&(sp->readpos),SEEK_SET);
00620         InIn = ReadFile(fh->handle,(UBYTE *) (fh->PObuffer),fh->POsize);
00621         if ( InIn < 0 || ( InIn & 1 ) ) {
00622             MLOCK(ErrorMessageLock);
00623             MesPrint("Error reading information for %s spectator",sp->name);
00624             MUNLOCK(ErrorMessageLock);
00625             Terminate(-1);
00626         }
00627         InIn /= sizeof(WORD);
00628         if ( InIn == 0 ) { *term = 0; return(0); }
00629         SeekFile(fh->handle,&(sp->readpos),SEEK_CUR);
00630         fh->POposition = sp->readpos;
00631         fh->POfull = fh->PObuffer+InIn;
00632         fh->POfill = fh->PObuffer;
00633     }
00634     if ( fh->POfill == fh->POfull ) { /* not even the size of the term! */
00635         if ( ISLESSPOS(sp->readpos,sp->position) ) goto FillBuffer;
00636         *term = 0;
00637         return(0);
00638     }

```

```

00639     size = *fh->POfill++; *t++ = size;
00640     for ( i = 1; i < size; i++ ) {
00641         if ( fh->POfill >= fh->POfull ) {
00642             SeekFile(fh->handle, &(sp->readpos), SEEK_SET);
00643             InIn = ReadFile(fh->handle, (UBYTE *) (fh->PObuffer), fh->POsize);
00644             if ( InIn < 0 || ( InIn & 1 ) ) {
00645                 MLOCK(ErrorMessageLock);
00646                 MesPrint("Error reading information for %s spectator", sp->name);
00647                 MUNLOCK(ErrorMessageLock);
00648                 Terminate(-1);
00649             }
00650             InIn /= sizeof(WORD);
00651             if ( InIn == 0 ) {
00652                 MLOCK(ErrorMessageLock);
00653                 MesPrint("Reading incomplete information for %s spectator", sp->name);
00654                 MUNLOCK(ErrorMessageLock);
00655                 Terminate(-1);
00656             }
00657             SeekFile(fh->handle, &(sp->readpos), SEEK_CUR);
00658             fh->POposition = sp->readpos;
00659             fh->POfull = fh->PObuffer+InIn;
00660             fh->POfill = fh->PObuffer;
00661         }
00662         *t++ = *fh->POfill++;
00663     }
00664     return(size);
00665 }
00666
00667 /*
00668 #] GetFromSpectator :
00669 #[ ClearSpectators :
00670
00671 Removes all spectators.
00672 In case of .store, the ones that are protected by .global stay.
00673 */
00674
00675 void ClearSpectators(WORD par)
00676 {
00677     SPECTATOR *sp = AM.SpectatorFiles;
00678     WORD numexpr, c1;
00679     int i;
00680     if ( AM.NumSpectatorFiles > 0 ) {
00681         for ( i = 0; i < AM.SizeForSpectatorFiles; i++, sp++ ) {
00682             if ( sp->name == 0 ) continue;
00683             if ( ( sp->flags & GLOBALSPECTATORFLAG ) == 1 && par == STOREMODULE ) continue;
00684
00685             if ( GetVar((UBYTE *) (sp->name), &c1, &numexpr, ALLVARIABLES, NOAUTO) == NAMENOTFOUND ||
00686                  c1 != CEXPRESSION ) {
00687                 MesPrint("%&s is not a valid expression.", sp->name);
00688                 continue;
00689             }
00690             Expressions[numexpr].status = DROPPEDEXPRESSION;
00691             if ( sp->fh->handle != -1 ) {
00692                 CloseFile(sp->fh->handle);
00693                 sp->fh->handle = -1;
00694                 remove(sp->fh->name);
00695             }
00696             M_free(sp->fh, "Temporary FileHandle");
00697             M_free(sp->name, "Spectator expression name");
00698             PUTZERO(sp->position);
00699             sp->fh = 0;
00700             sp->name = 0;
00701             sp->exprnumber = -1;
00702             sp->flags = 0;
00703             AM.NumSpectatorFiles--;
00704         }
00705     }
00706 }
00707
00708 /*
00709 #] ClearSpectators :
00710 */

```

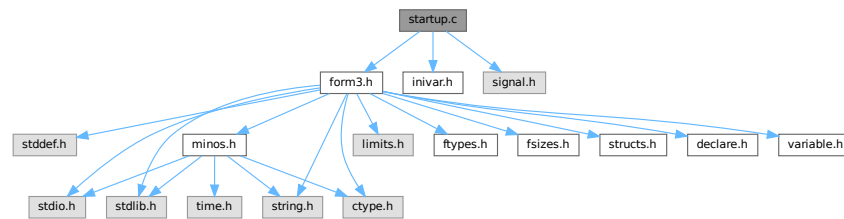
4.132 startup.c File Reference

```

#include "form3.h"
#include "inivar.h"
#include <signal.h>

```

Include dependency graph for startup.c:



Macros

- `#define STRINGIFY(x) STRINGIFY__(x)`
- `#define STRINGIFY__(x) #x`
- `#define FORMNAME "FORM"`
- `#define VERSIONSTR__ STRINGIFY(MAJORVERSION) "." STRINGIFY(MINORVERSION) "Beta"`
- `#define VERSIONSTR FORMNAME " " VERSIONSTR__ " (" PRODUCTIONDATE ")"`
- `#define TAKEPATH(x) if(s[1]== '='){x=s+2;} else{x=*argv++;argc--;}`
- `#define DEFAULTFNAMELENGTH 16`

Functions

- `int DoTail (int argc, UBYTE **argv)`
- `int OpenInput (void)`
- `void ReserveTempFiles (int par)`
- `void StartVariables (void)`
- `void StartMore (void)`
- `WORD IniVars (void)`
- `int main (int argc, char **argv)`
- `void CleanUp (WORD par)`
- `void TerminateImpl (int errorcode, const char *file, int line, const char *function)`
- `void PrintDeprecation (const char *feature, const char *issue)`
- `void PrintRunningTime (void)`
- `LONG GetRunningTime (void)`

Variables

- `UBYTE * emptystring = (UBYTE *)""`
- `UBYTE * defaulttempfilename = (UBYTE *)"xformxxx.str"`

4.132.1 Detailed Description

This file contains the main program. It also deals with the very early stages of the startup of FORM and the final stages when the program attempts some cleanup. Here is the routine that analyses the command tail.

Definition in file [startup.c](#).

4.132.2 Macro Definition Documentation

4.132.2.1 STRINGIFY

```
#define STRINGIFY(  
    x ) STRINGIFY__(x)
```

Definition at line 57 of file [startup.c](#).

4.132.2.2 STRINGIFY__

```
#define STRINGIFY__(  
    x ) #x
```

Definition at line 58 of file [startup.c](#).

4.132.2.3 FORMNAME

```
#define FORMNAME "FORM"
```

Definition at line 68 of file [startup.c](#).

4.132.2.4 VERSIONSTR__

```
#define VERSIONSTR__ STRINGIFY(MAJORVERSION) "." STRINGIFY(MINORVERSION) "Beta"
```

Definition at line 97 of file [startup.c](#).

4.132.2.5 VERSIONSTR

```
#define VERSIONSTR FORMNAME " " VERSIONSTR__ " (" PRODUCTIONDATE ")"
```

Definition at line 101 of file [startup.c](#).

4.132.2.6 TAKEPATH

```
#define TAKEPATH(  
    x ) if(s[1]== '=' ){x=s+2;} else{x=*argv++;argc--;}  
    }
```

Definition at line 235 of file [startup.c](#).

4.132.2.7 DEFAULTFNAMELENGTH

```
#define DEFAULTFNAMELENGTH 16
```

Definition at line 664 of file [startup.c](#).

4.132.3 Function Documentation

4.132.3.1 DoTail()

```
int DoTail (
    int argc,
    UBYTE ** argv )
```

Definition at line 237 of file [startup.c](#).

4.132.3.2 OpenInput()

```
int OpenInput (
    void )
```

Definition at line 542 of file [startup.c](#).

4.132.3.3 ReserveTempFiles()

```
void ReserveTempFiles (
    int par )
```

Definition at line 666 of file [startup.c](#).

4.132.3.4 StartVariables()

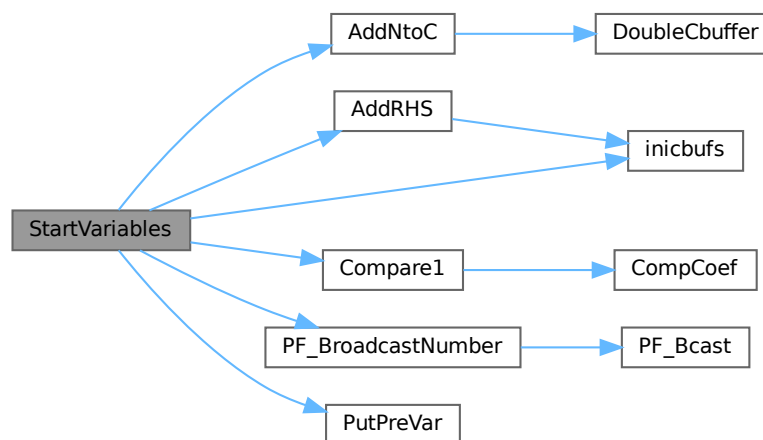
```
void StartVariables (
    void )
```

All functions (well, nearly all) are declared here.

Definition at line 905 of file [startup.c](#).

References [AddNtoC\(\)](#), [AddRHS\(\)](#), [Compare1\(\)](#), [inicbufs\(\)](#), [FuNcTiOn::name](#), [PF_BroadcastNumber\(\)](#), [PutPreVar\(\)](#), and [FuNcTiOn::symmetric](#).

Here is the call graph for this function:



4.132.3.5 StartMore()

```
void StartMore (
    void )
```

Definition at line 1316 of file [startup.c](#).

4.132.3.6 IniVars()

```
WORD IniVars (
    void )
```

Definition at line 1354 of file [startup.c](#).

4.132.3.7 main()

```
int main (
    int argc,
    char ** argv )
```

Definition at line 1662 of file [startup.c](#).

4.132.3.8 CleanUp()

```
void CleanUp (
    WORD par )
```

Definition at line 1761 of file [startup.c](#).

4.132.3.9 TerminateImpl()

```
void TerminateImpl (
    int errorcode,
    const char * file,
    int line,
    const char * function )
```

Definition at line 1850 of file [startup.c](#).

4.132.3.10 PrintDeprecation()

```
void PrintDeprecation (
    const char * feature,
    const char * issue )
```

Prints a deprecation warning for a given feature.

Parameters

<i>feature</i>	The name of the deprecated feature.
<i>issue</i>	The associated issue, e.g., "issues/700".

Definition at line [2063](#) of file [startup.c](#).

4.132.3.11 PrintRunningTime()

```
void PrintRunningTime (  
    void )
```

Definition at line [2087](#) of file [startup.c](#).

4.132.3.12 GetRunningTime()

```
LONG GetRunningTime (  
    void )
```

Definition at line [2128](#) of file [startup.c](#).

4.132.4 Variable Documentation

4.132.4.1 emptystring

```
UBYTE* emptystring = (UBYTE *)"."
```

Definition at line [660](#) of file [startup.c](#).

4.132.4.2 defaulttempfilename

```
UBYTE* defaulttempfilename = (UBYTE *)"xformxxx.str"
```

Definition at line [661](#) of file [startup.c](#).

4.133 startup.c

[Go to the documentation of this file.](#)

```

00001
00008 /* #[ License : */
00009 /*
00010 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00011 *   When using this file you are requested to refer to the publication
00012 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00013 *   This is considered a matter of courtesy as the development was paid
00014 *   for by FOM the Dutch physics granting agency and we would like to
00015 *   be able to track its scientific use to convince FOM of its value
00016 *   for the community.
00017 *
00018 *   This file is part of FORM.
00019 *
00020 *   FORM is free software: you can redistribute it and/or modify it under the
00021 *   terms of the GNU General Public License as published by the Free Software
00022 *   Foundation, either version 3 of the License, or (at your option) any later
00023 *   version.
00024 *
00025 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00026 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00027 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00028 *   details.
00029 *
00030 *   You should have received a copy of the GNU General Public License along
00031 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00032 */
00033 /* #[ License : */
00034 /*
00035         #[ includes :
00036 */
00037
00038 #include "form3.h"
00039 #include "inivar.h"
00040
00041 #ifdef TRAP_SIGNALS
00042 #include "portsignals.h"
00043 #else
00044 #include <signal.h>
00045 #endif
00046 #ifdef ENABLE_BACKTRACE
00047     #include <execinfo.h>
00048 #ifdef LINUX
00049         #include <stdint.h>
00050         #include <inttypes.h>
00051 #endif
00052 #endif
00053
00054 /*
00055 * A macro for translating the contents of `x' into a string after expanding.
00056 */
00057 #define STRINGIFY(x) STRINGIFY__(x)
00058 #define STRINGIFY__(x) #x
00059
00060 /*
00061 * FORMNAME = "FORM" or "TFORM" or "ParFORM".
00062 */
00063 #if defined(WITHPTHREADS)
00064     #define FORMNAME "TFORM"
00065 #elif defined(WITHMPI)
00066     #define FORMNAME "ParFORM"
00067 #else
00068     #define FORMNAME "FORM"
00069 #endif
00070
00071 /*
00072 * VERSIONSTR is the version information printed in the header line.
00073 */
00074 #ifdef HAVE_CONFIG_H
00075     /* We have also version.h. */
00076     #include "version.h"
00077     #ifndef REPO_VERSION
00078         #define REPO_VERSION STRINGIFY(REPO_MAJOR_VERSION) "." STRINGIFY(REPO_MINOR_VERSION)
00079     #endif
00080     #ifndef REPO_DATE
00081         /* The build date, instead of the repo date. */
00082         #define REPO_DATE __DATE__
00083     #endif
00084     #ifndef REPO_REVISION
00085         #define VERSIONSTR FORMNAME " " REPO_VERSION " (" REPO_DATE ", " REPO_REVISION ")"
00086     #else
00087         #define VERSIONSTR FORMNAME " " REPO_VERSION " (" REPO_DATE ")"
00088     #endif

```



```

00089     #define MAJORVERSION REPO_MAJOR_VERSION
00090     #define MINORVERSION REPO_MINOR_VERSION
00091 #else
00092     /*
00093      * Otherwise, form3.h defines MAJORVERSION, MINORVERSION and PRODUCTIONDATE,
00094      * possibly BETAVERSION.
00095      */
00096     #ifndef BETAVERSION
00097         #define VERSIONSTR__ STRINGIFY(MAJORVERSION) "." STRINGIFY(MINORVERSION) "Beta"
00098     #else
00099         #define VERSIONSTR__ STRINGIFY(MAJORVERSION) "." STRINGIFY(MINORVERSION)
00100     #endif
00101     #define VERSIONSTR FORMNAME " " VERSIONSTR__ " (" PRODUCTIONDATE ")"
00102 #endif
00103
00104 /*
00105     #] includes :
00106     #[ PrintHeader :
00107 */
00108
00114 static void PrintHeader(int with_full_info)
00115 {
00116     #ifdef WITHMPI
00117         if ( PF.me == MASTER && !AM.silent ) {
00118     #else
00119         if ( !AM.silent ) {
00120     #endif
00121         char buffer1[250], buffer2[80], *s = buffer1, *t = buffer2;
00122         WORD length, n;
00123         for ( n = 0; n < 250; n++ ) buffer1[n] = ' ';
00124         /*
00125          * NOTE: we expect that the compiler optimizes strlen("string literal")
00126          * to just a number.
00127          */
00128         if ( strlen(VERSIONSTR) <= 100 ) {
00129             strcpy(s,VERSIONSTR);
00130             s += strlen(VERSIONSTR);
00131             *s = 0;
00132         }
00133         else {
00134             /*
00135              * Truncate when it is too long.
00136              */
00137             strncpy(s,VERSIONSTR,97);
00138             s[97] = '.';
00139             s[98] = '.';
00140             s[99] = ')';
00141             s[100] = '\0';
00142             s += 100;
00143         }
00144         /*
00145          By now we omit the message about 32/64 bits. It should all be 64.
00146
00147          s += sprintf(s,250-(s-buffer1), " %d-bits", (WORD) (sizeof(WORD)*16));
00148          *s = 0;
00149          */
00150         if ( with_full_info ) {
00151     #if defined(WITHPTHREADS) || defined(WITHMPI)
00152     #if defined(WITHPTHREADS)
00153             int nworkers = AM.totalnumberofthreads-1;
00154     #elif defined(WITHMPI)
00155             int nworkers = PF.numtasks-1;
00156     #endif
00157             s += sprintf(s,250-(s-buffer1), " %d worker",nworkers);
00158             *s = 0;
00159             /*
00160              while ( *s ) s++; */
00161             if ( nworkers != 1 ) {
00162                 *s++ = 's';
00163                 *s = '\0';
00164             }
00165     #endif
00166             sprintf(t,80-(t-buffer2),"Run: %s",MakeDate());
00167             while ( *t ) t++;
00168
00169             /*
00170              * Align the date to the right, if it fits in a line.
00171              */
00172             length = (s-buffer1) + (t-buffer2);
00173             if ( length+2 <= AC.LineLength ) {
00174                 for ( n = AC.LineLength-length; n > 0; n-- ) *s++ = ' ';
00175                 *s = 0;
00176                 strcat(s,buffer2);
00177                 while ( *s ) s++;
00178             }
00179             else {
00180                 *s = 0;

```

```

00181         strcat(s," ");
00182         while ( *s ) s++;
00183         *s = 0;
00184         strcat(s,buffer2);
00185         while ( *s ) s++;
00186     }
00187 }
00188
00189 /*
00190  * If the header information doesn't fit in a line, we need to extend
00191  * the line length temporarily.
00192  */
00193 length = s-buffer1;
00194 if ( length <= AC.LineLength ) {
00195     MesPrint("%s",buffer1);
00196 }
00197 else {
00198     WORD oldLineLength = AC.LineLength;
00199     AC.LineLength = length;
00200     MesPrint("%s",buffer1);
00201     AC.LineLength = oldLineLength;
00202 }
00203 }
00204 #ifdef WINDOWS
00205     PrintDeprecation("the native Windows version", "issues/623");
00206 #endif
00207 #ifdef ILP32
00208     PrintDeprecation("the 32-bit version", "issues/624");
00209 #endif
00210 #ifdef WITHMPI
00211     PrintDeprecation("the MPI version (ParFORM)", "issues/625");
00212 #endif
00213 if ( AC.CheckpointFlag ) {
00214     PrintDeprecation("the checkpoint mechanism", "issues/626");
00215 }
00216 }
00217
00218 /*
00219     #] PrintHeader :
00220     #[ DoTail :
00221
00222     Routine reads the command tail and handles the commandline options.
00223     It sets the flags for later actions and stored pathnames for
00224     the setup file, include/prc/sub directories etc.
00225     Finally the name of the program is passed on.
00226     Note that we do not support interactive use yet. This will come
00227     to pass in the distant future when we can couple STedi to FORM.
00228     Routine made 23-feb-1993 by J.Vermaseren
00229 */
00230
00231 #ifdef WITHINTERACTION
00232 static UBYTE deflogname[] = "formsession.log";
00233 #endif
00234
00235 #define TAKEPATH(x) if(s[l]== ' '){x=s+2;} else{x=*argv++;argc--;}
00236
00237 int DoTail(int argc, UBYTE **argv)
00238 {
00239     int errorflag = 0, onlyversion = 1;
00240     UBYTE *s, *t, *copy;
00241     int threadnum = 0;
00242     argc--; argv++;
00243     AM.ClearStore = 0;
00244     AM.TimeLimit = 0;
00245     AM.LogType = -1;
00246     AM.HoldFlag = AM.qError = AM.Interact = AM.FileOnlyFlag = 0;
00247     AM.InputFileName = AM.LogFileName = AM.IncDir = AM.TempDir = AM.TempSortDir =
00248     AM.SetupDir = AM.SetupFile = AM.Path = 0;
00249     AM.FromStdin = 0;
00250     /* Always use MultiRun, "-M" option is now ignored. */
00251     AM.MultiRun = 1;
00252     if ( argc < 1 ) {
00253         onlyversion = 0;
00254         goto printversion;
00255     }
00256     while ( argc >= 1 ) {
00257         s = *argv++; argc--;
00258         if ( *s == '-' || ( *s == '/' && ( argc > 0 || AM.Interact ) ) ) {
00259             s++;
00260             switch (*s) {
00261                 case 'c': /* Error checking only */
00262                     AM.qError = 1; break;
00263                 case 'C': /* Next arg is filename of log file */
00264                     TAKEPATH(AM.LogFileName) break;
00265                 case 'D':
00266                 case 'd': /* Next arg is define preprocessor var. */
00267                     t = copy = strDupl(*argv,"Dotail");

```

```

00268         while ( *t && *t != '=' ) t++;
00269         if ( *t == 0 ) {
00270             if ( PutPreVar(copy, (UBYTE *) "1", 0, 0) < 0 ) return(-1);
00271         }
00272         else {
00273             *t++ = 0;
00274             if ( PutPreVar(copy, t, 0, 0) < 0 ) return(-1);
00275             t[-1] = '=';
00276         }
00277         M_free(copy, "-d prevar");
00278         argv++; argc--; break;
00279     case 'f': /* Output only to regular log file */
00280         AM.FileOnlyFlag = 1; AM.LogType = 0; break;
00281     case 'F': /* Output only to log file. Further like L. */
00282         AM.FileOnlyFlag = 1; AM.LogType = 1; break;
00283     case 'h': /* For old systems: wait for key before exit */
00284         AM.HoldFlag = 1; break;
00285     case 'i':
00286         if ( StrCmp(s, (UBYTE *) "ignore-deprecation") == 0 ) {
00287             AM.IgnoreDeprecation = 1;
00288             break;
00289         }
00290 #ifdef WITHINTERACTION
00291         /* Interactive session (not used yet) */
00292         AM.Interact = 1;
00293         break;
00294     #else
00295         goto IllegalOption;
00296 #endif
00297     case 'I': /* Next arg is dir for inc/prc/sub files */
00298         TAKEPATH(AM.IncDir) break;
00299     case 'l': /* Make regular log file */
00300         if ( s[1] == 'l' ) AM.LogType = 1; /*compatibility! */
00301         else
00302             AM.LogType = 0;
00303         break;
00304     case 'L': /* Make log file with only final statistics */
00305         AM.LogType = 1; break;
00306     case 'M': /* Multirun. Name of tempfiles will contain PID */
00307         /* This option is now ignored. We always use MultiRun. */
00308         /* AM.MultiRun = 1; */
00309         break;
00310     case 'm': /* Read number of threads */
00311     case 'w': /* Read number of workers */
00312         t = s++;
00313         threadnum = 0;
00314         while ( *s >= '0' && *s <= '9' )
00315             threadnum = 10*threadnum + *s++ - '0';
00316         if ( *s ) {
00317             if ( PF.me == MASTER )
00318                 printf("Illegal value for option m or w: %s\n", t);
00319             errorflag++;
00320         }
00321         if ( threadnum == 1 ) threadnum = 0; /*
00322         threadnum++;
00323         break;
00324     case 'W': /* Print the wall-clock time on the master. */
00325         AM.ggWTimeStatsFlag = 1;
00326         break;
00327     /*
00328     case 'n':
00329         Reserved for number of slaves without MPI
00330     */
00331     case 'p':
00332 #ifdef WITHEXTERNALCHANNEL
00333     /*There are two possibilities: -p|-pipe*/
00334     if (s[1]=='i') {
00335         if ( (s[2]=='p') && (s[3]=='e') && (s[4]=='\0') ) {
00336             argc--;
00337             /*Initialize pre-set external channels, see
00338             the file extcmd.c:*/
00339             if (initPresetExternalChannels(*argv++, AX.timeout) < 1) {
00340                 if ( PF.me == MASTER )
00341                     printf("Error initializing preset external channels\n");
00342                 errorflag++;
00343             }
00344             AX.timeout=-1; /*This indicates that preset channels
00345                             are initialized from cmdline*/
00346         }
00347     }
00348     #else
00349     }
00350     #endif
00351     if ( PF.me == MASTER )
00352         printf("Illegal option in call of FORM: %s\n", s);
00353     errorflag++;

```

```

00355     }
00356     }else
00357 #else
00358     if ( s[1] ) {
00359         if ( ( s[1]=='i' ) && ( s[2] == 'p' ) && (s[3] == 'e' )
00360             && ( s[4] == '\0' ) ){
00361 #ifdef WITHMPI
00362             if ( PF.me == MASTER )
00363 #endif
00364                 printf("Illegal option: Pipes not supported on this system.\n");
00365         }
00366     else {
00367 #ifdef WITHMPI
00368         if ( PF.me == MASTER )
00369 #endif
00370             printf("Illegal option: %s\n",s);
00371     }
00372     errorflag++;
00373 }
00374 else
00375 #endif
00376 {
00377     /* Next arg is a path variable like in environment */
00378     TAKEPATH(AM.Path)
00379 }
00380 break;
00381 case 'q': /* Quiet option. Only output. Same as -si */
00382     AM.silent = 1; break;
00383 case 'R': /* recover from saved snapshot */
00384     AC.CheckpointFlag = -1;
00385     break;
00386 case 's': /* Next arg is dir with form.set to be used */
00387     if ( ( s[1] == 'o' ) && ( s[2] == 'r' ) && ( s[3] == 't' ) ) {
00388         if(s[4]=='=') {
00389             AM.TempSortDir = s+5;
00390         }
00391         else {
00392             AM.TempSortDir = *argv++;
00393             argc--;
00394         }
00395     }
00396     else if ( s[1] == 'i' ) { /* compatibility: silent/quiet */
00397         AM.silent = 1;
00398     }
00399     else {
00400         TAKEPATH(AM.SetupDir)
00401     }
00402     break;
00403 case 'S': /* Next arg is setup file */
00404     TAKEPATH(AM.SetupFile) break;
00405 case 't': /* Next arg is directory for temp files */
00406     if ( s[1] == 's' ) {
00407         s++;
00408         AM.havesortdir = 1;
00409         TAKEPATH(AM.TempSortDir)
00410     }
00411     else {
00412         TAKEPATH(AM.TempDir)
00413     }
00414     break;
00415 case 'T': /* Print the total size used at end of job */
00416     AM.PrintTotalSize = 1; break;
00417 case 'v':
00418     printversion;
00419 #ifdef WITHMPI
00420     if ( PF.me == MASTER )
00421 #endif
00422         PrintHeader(0);
00423         if ( onlyversion ) return(1);
00424         goto NoFile;
00425 case 'y': /* Preprocessor dumps output. No compilation. */
00426     AP.PreDebug = PREPROONLY; break;
00427 case 'z': /* The number following is a time limit in sec. */
00428     t = s++;
00429     AM.TimeLimit = 0;
00430     while ( *s >= '0' && *s <= '9' )
00431         AM.TimeLimit = 10*AM.TimeLimit + *s++ - '0';
00432     break;
00433 case 'Z': /* Removes the .str file on crash, no matter its contents */
00434     AM.ClearStore = 1; break;
00435 case '\0': /* "-" to use STDIN for the input. */
00436 #ifdef WITHMPI
00437     /* At the moment, ParFORM doesn't implement STDIN broadcasts. */
00438     if ( PF.me == MASTER )
00439         printf("Sorry, reading STDIN as input is currently not supported by
ParFORM\n");
00440     errorflag++;

```

```

00441 #endif
00442                                     AM.FromStdin = 1;
00443                                     AC.NoShowInput = 1; // disable input echoing by default
00444                                     break;
00445                                     default:
00446                                     if ( FG.cTable[*s] == 1 ) {
00447                                     AM.SkipClears = 0; t = s;
00448                                     while ( FG.cTable[*t] == 1 )
00449                                     AM.SkipClears = 10*AM.SkipClears + *t++ - '0';
00450                                     if ( *t != 0 ) {
00451 #ifdef WITHMPI
00452                                     if ( PF.me == MASTER )
00453 #endif
00454                                     printf("Illegal numerical option in call of FORM: %s\n",s);
00455                                     errorflag++;
00456                                     }
00457                                     }
00458                                     else {
00459 IllegalOption:
00460 #ifdef WITHMPI
00461                                     if ( PF.me == MASTER )
00462 #endif
00463                                     printf("Illegal option in call of FORM: %s\n",s);
00464                                     errorflag++;
00465                                     }
00466                                     break;
00467                                     }
00468                                     }
00469                                     else if ( argc == 0 && !AM.Interact ) AM.InputFileName = argv[-1];
00470                                     else {
00471 #ifdef WITHMPI
00472                                     if ( PF.me == MASTER )
00473 #endif
00474                                     printf("Illegal option in call of FORM: %s\n",s);
00475                                     errorflag++;
00476                                     }
00477                                     }
00478                                     AM.totalnumberofthreads = threadnum;
00479                                     if ( AM.InputFileName ) {
00480                                     if ( AM.FromStdin ) {
00481                                     printf("STDIN and the input filename cannot be specified simultaneously\n");
00482                                     errorflag++;
00483                                     }
00484                                     s = AM.InputFileName;
00485                                     while ( *s ) s++;
00486                                     if ( s < AM.InputFileName+4 ||
00487                                     s[-4] != '.' || s[-3] != 'f' || s[-2] != 'r' || s[-1] != 'm' ) {
00488                                     t = (UBYTE *)Malloc1((s-AM.InputFileName)+5,"adding .frm");
00489                                     s = AM.InputFileName;
00490                                     AM.InputFileName = t;
00491                                     while ( *s ) *t++ = *s++;
00492                                     *t++ = '.'; *t++ = 'f'; *t++ = 'r'; *t++ = 'm'; *t = 0;
00493                                     }
00494                                     if ( AM.LogType >= 0 && AM.LogFileName == 0 ) {
00495                                     AM.LogFileName = strdup(AM.InputFileName,"name of logfile");
00496                                     s = AM.LogFileName;
00497                                     while ( *s ) s++;
00498                                     s[-3] = 'l'; s[-2] = 'o'; s[-1] = 'g';
00499                                     }
00500                                     }
00501 #ifdef WITHINTERACTION
00502                                     else if ( AM.Interact ) {
00503                                     if ( AM.LogType >= 0 ) {
00504                                     /*
00505                                     We may have to do better than just taking a name.
00506                                     It is not unique! This will be left for later.
00507                                     */
00508                                     AM.LogFileName = deflogname;
00509                                     }
00510                                     }
00511 #endif
00512                                     else if ( AM.FromStdin ) {
00513                                     /* Do nothing. */
00514                                     }
00515                                     else {
00516 NoFile:
00517 #ifdef WITHMPI
00518                                     if ( PF.me == MASTER )
00519 #endif
00520                                     printf("No filename specified in call of FORM\n");
00521                                     errorflag++;
00522                                     }
00523                                     if ( AM.Path == 0 ) AM.Path = (UBYTE *)getenv("FORMPATH");
00524                                     if ( AM.Path ) {
00525                                     /*
00526                                     * AM.Path is taken from argv or getenv. Reallocate it to avoid invalid
00527                                     * frees when AM.Path has to be changed.

```

```

00528      */
00529      AM.Path = strdup(AM.Path,"DoTail Path");
00530  }
00531  return(errorflag);
00532 }
00533
00534 /*
00535      #] DoTail :
00536      #[ OpenInput :
00537
00538      Major task here after opening is to skip the proper number of
00539      .clear instructions if so desired without using interpretation
00540 */
00541
00542 int OpenInput(void)
00543 {
00544     int oldNoShowInput = AC.NoShowInput;
00545     UBYTE c;
00546     if ( !AM.Interact ) {
00547         if ( AM.FromStdin ) {
00548             if ( OpenStream(0,INPUTSTREAM,0,PREENOACTION) == 0 ) {
00549                 Error0("Cannot open STDIN");
00550                 return(-1);
00551             }
00552         }
00553         else {
00554             if ( OpenStream(AM.InputFileName,FILESTREAM,0,PREENOACTION) == 0 ) {
00555                 Error1("Cannot open file",AM.InputFileName);
00556                 return(-1);
00557             }
00558             if ( AC.CurrentStream->inbuffer <= 0 ) {
00559                 Error1("No input in file",AM.InputFileName);
00560                 return(-1);
00561             }
00562         }
00563         AC.NoShowInput = 1;
00564         while ( AM.SkipClears > 0 ) {
00565             c = GetInput();
00566             if ( c == ENDOFINPUT ) {
00567                 Error0("Not enough .clear instructions in input file");
00568             }
00569             if ( c == '\\\\' ) {
00570                 c = GetInput();
00571                 if ( c == ENDOFINPUT )
00572                     Error0("Not enough .clear instructions in input file");
00573                 continue;
00574             }
00575             if ( c == ' ' || c == '\t' ) continue;
00576             if ( c == '.' ) {
00577                 c = GetInput();
00578                 if ( tolower(c) == 'c' ) {
00579                     c = GetInput();
00580                     if ( tolower(c) == 'l' ) {
00581                         c = GetInput();
00582                         if ( tolower(c) == 'e' ) {
00583                             c = GetInput();
00584                             if ( tolower(c) == 'a' ) {
00585                                 c = GetInput();
00586                                 if ( tolower(c) == 'r' ) {
00587                                     c = GetInput();
00588                                     if ( FG.cTable[c] > 2 ) {
00589                                         AM.SkipClears--;
00590                                     }
00591                                 }
00592                             }
00593                         }
00594                     }
00595                 }
00596             }
00597             while ( c != '\n' && c != '\r' && c != ENDOFINPUT ) {
00598                 c = GetInput();
00599                 if ( c == '\\\\' ) c = GetInput();
00600             }
00601             else if ( c == '\n' || c == '\r' ) continue;
00602             else {
00603                 while ( ( c = GetInput() ) != '\n' && c != '\r' ) {
00604                     if ( c == ENDOFINPUT ) {
00605                         Error0("Not enough .clear instructions in input file");
00606                     }
00607                 }
00608             }
00609         }
00610         AC.NoShowInput = oldNoShowInput;
00611     }
00612     if ( AM.LogFileName ) {
00613 #ifndef WITHMPI
00614         if ( PF.me != MASTER ) {

```

```

00615         /*
00616         * Only the master writes to the log file. On slaves, we need
00617         * a dummy handle, without opening the file.
00618         */
00619         extern FILES **filelist; /* in tools.c */
00620         int i = CreateHandle();
00621         RWLOCKW(AM.handlelock);
00622         filelist[i] = (FILES *)123; /* Must be nonzero to prevent a reuse in CreateHandle. */
00623         UNRWLOCK(AM.handlelock);
00624         AC.LogHandle = i;
00625     }
00626     else
00627 #endif
00628     if ( AC.CheckpointFlag != -1 ) {
00629         if ( ( AC.LogHandle = CreateLogFile((char *) (AM.LogFileName)) ) < 0 ) {
00630             Error1("Cannot create logfile",AM.LogFileName);
00631             return(-1);
00632         }
00633     }
00634     else {
00635         if ( ( AC.LogHandle = OpenAddFile((char *) (AM.LogFileName)) ) < 0 ) {
00636             Error1("Cannot re-open logfile",AM.LogFileName);
00637             return(-1);
00638         }
00639     }
00640 }
00641 return(0);
00642 }
00643
00644 /*
00645     #] OpenInput :
00646     #[ ReserveTempFiles :
00647
00648     Order of preference:
00649     a: if there is a path in the commandtail, take that.
00650     b: if none, try in the form.set file.
00651     c: if still none, try in the environment for the variable FORMTMP
00652     d: if still none, try the current directory.
00653
00654     The parameter indicates action in the case of multithreaded running.
00655     par = 0 : We just run on a single processor. Keep everything normal.
00656     par = 1 : Multithreaded running startup phase 1.
00657     par = 2 : Multithreaded running startup phase 2.
00658 */
00659
00660 UBYTE *emptystring = (UBYTE *)"";
00661 UBYTE *defaulttempfilename = (UBYTE *)"xformxxx.str";
00662 /* This is the length of the above default, but with 7 spaces for PID digits
00663    instead of the "xxx". (Previously FORM used 5 digits and this value was 14) */
00664 #define DEFAULTFNAMELENGTH 16
00665
00666 void ReserveTempFiles(int par)
00667 {
00668     GETIDENTITY
00669     SETUPPARAMETERS *sp;
00670     UBYTE *s, *t, *tenddir, *tenddir2, c;
00671     int i = 0;
00672     WORD j;
00673     if ( par == 0 || par == 1 ) {
00674         if ( AM.TempDir == 0 ) {
00675             sp = GetSetupPar((UBYTE *)"tempdir");
00676             if ( ( sp->flags & USEDFLAG ) != USEDFLAG ) {
00677                 AM.TempDir = (UBYTE *)getenv("FORMTMP");
00678                 if ( AM.TempDir == 0 ) AM.TempDir = emptystring;
00679             }
00680             else AM.TempDir = (UBYTE *) (sp->value);
00681         }
00682         if ( AM.TempSortDir == 0 ) {
00683             if ( AM.havesortdir ) {
00684                 sp = GetSetupPar((UBYTE *)"tempdir");
00685                 AM.TempSortDir = (UBYTE *) (sp->value);
00686             }
00687             else {
00688                 AM.TempSortDir = (UBYTE *)getenv("FORMTMP");
00689                 if ( AM.TempSortDir == 0 ) AM.TempSortDir = AM.TempDir;
00690             }
00691         }
00692     }
00693     /*
00694     We have now in principle a path but we will use its first element only.
00695     Later that should become more complicated. Then we will use a path and
00696     when one device is full we can continue on the next one.
00697 */
00697     s = AM.TempDir; i = 200; /* Some extra for VMS */
00698     while ( *s && *s != PATHSEPARATOR ) { if ( *s == '\\/' ) s++; s++; i++; }
00699     FG.fnamesize = sizeof(UBYTE)*(i+DEFAULTFNAMELENGTH);
00700     FG.fname = (char *)Malloc1(FG.fnamesize,"name for temporary files");
00701     s = AM.TempDir; t = (UBYTE *)FG.fname;

```

```

00702     while ( *s && *s != PATHSEPARATOR ) { if ( *s == '\\\' ) s++; *t++ = *s++; }
00703     if ( (char *)t > FG.fname && t[-1] != SEPARATOR && t[-1] != ALTSEPARATOR )
00704         *t++ = SEPARATOR;
00705     *t = 0;
00706     tenddir = t;
00707     FG.fnamebase = t-(UBYTE *) (FG.fname);
00708
00709     s = AM.TempSortDir; i = 200; /* Some extra for VMS */
00710     while ( *s && *s != PATHSEPARATOR ) { if ( *s == '\\\' ) s++; s++; i++; }
00711
00712     FG.fname2size = sizeof(UBYTE)*(i+DEFAULTFNAMELLENGTH);
00713     FG.fname2 = (char *)Malloc1(FG.fname2size,"name for sort files");
00714     s = AM.TempSortDir; t = (UBYTE *)FG.fname2;
00715     while ( *s && *s != PATHSEPARATOR ) { if ( *s == '\\\' ) s++; *t++ = *s++; }
00716     if ( (char *)t > FG.fname2 && t[-1] != SEPARATOR && t[-1] != ALTSEPARATOR )
00717         *t++ = SEPARATOR;
00718     *t = 0;
00719     tenddir2 = t;
00720     FG.fname2base = t-(UBYTE *) (FG.fname2);
00721
00722     t = tenddir;
00723     s = defaulttmpfilename;
00724 #ifdef WITHMPI
00725     {
00726         int iii;
00727         iii = sprintf((char*)t,FG.fnamesize-((char*)t-FG.fname),"%d",PF.me);
00728         t+= iii;
00729         s+= iii; /* in case defaulttmpfilename is too short */
00730     }
00731 #endif
00732     while ( *s ) *t++ = *s++;
00733     *t = 0;
00734 /*
00735     There are problems when running many FORM jobs at the same time
00736     from make or minos. If they start up simultaneously, occasionally
00737     they can make the same .str file. We prevent this with first trying
00738     a file that contains the digits of the pid. If this file
00739     has already been taken we fall back on the old scheme.
00740     The whole is controlled with the -M (MultiRun) parameter in the
00741     command tail.
00742 */
00743     if ( AM.MultiRun ) {
00744         int num = ((int)GetPID())%10000000;
00745         t += 4;
00746         *t = 0;
00747         t[-1] = 'r';
00748         t[-2] = 't';
00749         t[-3] = 's';
00750         t[-4] = '.';
00751         t[-5] = (UBYTE)('0' + num%10);
00752         t[-6] = (UBYTE)('0' + (num/10)%10);
00753         t[-7] = (UBYTE)('0' + (num/100)%10);
00754         t[-8] = (UBYTE)('0' + (num/1000)%10);
00755         t[-9] = (UBYTE)('0' + (num/10000)%10);
00756         t[-10] = (UBYTE)('0' + (num/100000)%10);
00757         t[-11] = (UBYTE)('0' + num/1000000);
00758         if ( ( AC.StoreHandle = CreateFile((char *)FG.fname) ) < 0 ) {
00759             t[-5] = 'x'; t[-6] = 'x'; t[-7] = 'x'; t[-8] = 'x'; t[-9] = 'x'; t[-10] = 'x'; t[-11] =
00760             'x';
00761             goto classic;
00762         }
00763     }
00764     {
00765     classic:;
00766         for(;;) {
00767             if ( ( AC.StoreHandle = OpenFile((char *)FG.fname) ) < 0 ) {
00768                 if ( ( AC.StoreHandle = CreateFile((char *)FG.fname) ) >= 0 ) break;
00769             }
00770             else CloseFile(AC.StoreHandle);
00771             c = t[-5];
00772             if ( c == 'x' ) t[-5] = '0';
00773             else if ( c == '9' ) {
00774                 t[-5] = '0';
00775                 c = t[-6];
00776                 if ( c == 'x' ) t[-6] = '0';
00777                 else if ( c == '9' ) {
00778                     t[-6] = '0';
00779                     c = t[-7];
00780                     if ( c == 'x' ) t[-7] = '0';
00781                     else if ( c == '9' ) {
00782                         /*
00783                         Note that we tried 1111 names!
00784                         */
00785                         MesPrint("Name space for temp files exhausted");
00786                         t[-7] = 0;
00787                         MesPrint("Please remove files of the type %s or try a different directory"

```



```

00788             ,FG.fname);
00789             Terminate(-1);
00790         }
00791         else t[-7] = (BYTE)(c+1);
00792     }
00793     else t[-6] = (BYTE)(c+1);
00794 }
00795 else t[-5] = (BYTE)(c+1);
00796 }
00797 }
00798 /*
00799 Now we should make sure that the tempsortdir cq tempsortfilename makes it
00800 into a similar construction.
00801 */
00802 s = tenddir; t = tenddir2; while ( *s ) *t++ = *s++;
00803 *t = 0;
00804
00805 /*
00806 Now we should assign a name to the main sort file and the two stage 4 files.
00807 */
00808 AM.S0->file.name = (char *)Malloc1(sizeof(char)*(i+DEFAULTFNAMELENGTH),"name for temporary
files");
00809 s = (BYTE *)AM.S0->file.name;
00810 t = (BYTE *)FG.fname2;
00811 i = 1;
00812 while ( *t ) { *s++ = *t++; i++; }
00813 s[-2] = 'o'; *s = 0;
00814 }
00815 /*
00816 Try to create the sort file already, so we can Terminate earlier if this fails.
00817 */
00818 #ifdef WITHPTHREADS
00819 if ( par <= 1 ) {
00820 #endif
00821     if ( ( AM.S0->file.handle = CreateFile((char *)AM.S0->file.name) ) < 0 ) {
00822         MesPrint("Could not create sort file: %s", AM.S0->file.name);
00823         Terminate(-1);
00824     };
00825     /* Close and clean up the test file */
00826     CloseFile(AM.S0->file.handle);
00827     AM.S0->file.handle = -1;
00828     remove(AM.S0->file.name);
00829 #ifdef WITHPTHREADS
00830 }
00831 #endif
00832 /*
00833 With the stage4 and scratch file names we have to be a bit more careful.
00834 They are to be allocated after the threads are initialized when there
00835 are threads of course.
00836 */
00837 if ( par == 0 ) {
00838     s = (BYTE *)((void *) (FG.fname2)); i = 0;
00839     while ( *s ) { s++; i++; }
00840     /* +1 for null terminator */
00841     s = (BYTE *)Malloc1(sizeof(char)*(i+1),"name for stage4 file a");
00842     AR.FoStage4[1].name = (char *)s;
00843     t = (BYTE *)FG.fname2;
00844     while ( *t ) *s++ = *t++;
00845     s[-2] = '4'; s[-1] = 'a'; *s = 0;
00846     s = (BYTE *)((void *) (FG.fname2)); i = 0;
00847     while ( *s ) { s++; i++; }
00848     /* +1 for null terminator */
00849     s = (BYTE *)Malloc1(sizeof(char)*(i+1),"name for stage4 file b");
00850     AR.FoStage4[0].name = (char *)s;
00851     t = (BYTE *)FG.fname;
00852     while ( *t ) *s++ = *t++;
00853     s[-2] = '4'; s[-1] = 'b'; *s = 0;
00854     for ( j = 0; j < 3; j++ ) {
00855         /* +1 for null terminator */
00856         s = (BYTE *)Malloc1(sizeof(char)*(i+1),"name for scratch file");
00857         AR.Fscr[j].name = (char *)s;
00858         t = (BYTE *)FG.fname;
00859         while ( *t ) *s++ = *t++;
00860         s[-2] = 'c'; s[-1] = (BYTE)('0'+j); *s = 0;
00861     }
00862 }
00863 #ifdef WITHPTHREADS
00864 else if ( par == 2 ) {
00865     size_t tname;
00866     s = (BYTE *)((void *) (FG.fname2)); i = 0;
00867     while ( *s ) { s++; i++; }
00868     /* +1 for null terminator, +10 for 32bit int, +1 for "." */
00869     tname = sizeof(char)*(i+12);
00870     s = (BYTE *)Malloc1(tname,"name for stage4 file a");
00871     snprintf((char *)s,tname,"%s.%d",FG.fname2,AT.identity);
00872     s[i-2] = '4'; s[i-1] = 'a';
00873     AR.FoStage4[1].name = (char *)s;

```

```

00874     s = (UBYTE *) ((void *) (FG.fname)); i = 0;
00875     while ( *s ) { s++; i++; }
00876     /* +1 for null terminator, +10 for 32bit int, +1 for "." */
00877     tname = sizeof(char)*(i+12);
00878     s = (UBYTE *) Malloc1(tname, "name for stage4 file b");
00879     snprintf((char *)s, tname, "%s.%d", FG.fname, AT.identity);
00880     s[i-2] = '4'; s[i-1] = 'b';
00881     AR.FoStage4[0].name = (char *)s;
00882     if ( AT.identity == 0 ) {
00883         for ( j = 0; j < 3; j++ ) {
00884             /* +1 for null terminator */
00885             s = (UBYTE *) Malloc1(sizeof(char)*(i+1), "name for scratch file");
00886             AR.Fscr[j].name = (char *)s;
00887             t = (UBYTE *) FG.fname;
00888             while ( *t ) *s++ = *t++;
00889             s[-2] = 'c'; s[-1] = (UBYTE) ('0'+j); *s = 0;
00890         }
00891     }
00892 }
00893 #endif
00894 }
00895
00896 /*
00897     #] ReserveTempFiles :
00898     #[ StartVariables :
00899 */
00900
00901 #ifdef WITHPTHREADS
00902 ALLPRIVATES *DummyPointer = 0;
00903 #endif
00904
00905 void StartVariables(void)
00906 {
00907     int i, ii;
00908     PUTZERO(AM.zeropos);
00909     StartPrepro();
00910 /*
00911     The module counter:
00912 */
00913     AC.CModule=0;
00914 #ifdef WITHPTHREADS
00915 /*
00916     We need a value in AB because in the startup some routines may call AB[0].
00917 */
00918     AB = (ALLPRIVATES **) &DummyPointer;
00919 #endif
00920 /*
00921     separators used to delimit arguments in #call and #do, by default ',' and '|':
00922     Be sure, it is an empty set:
00923 */
00924     set_sub(AC.separators, AC.separators, AC.separators);
00925     set_set(',', AC.separators);
00926     set_set('|', AC.separators);
00927
00928     AM.BracketFactors[0] = 8;
00929     AM.BracketFactors[1] = SYMBOL;
00930     AM.BracketFactors[2] = 4;
00931     AM.BracketFactors[3] = FACTORSYMBOL;
00932     AM.BracketFactors[4] = 1;
00933     AM.BracketFactors[5] = 1;
00934     AM.BracketFactors[6] = 1;
00935     AM.BracketFactors[7] = 3;
00936
00937     AM.SkipClears = 0;
00938     AC.Cnumpows = 0;
00939     AC.OutputMode = 72;
00940     AC.OutputSpaces = NORMALFORMAT;
00941     AC.LineLength = 79;
00942     AM.gIsFortran90 = AC.IsFortran90 = ISNOTFORTHAN90;
00943     AM.gFortran90Kind = AC.Fortran90Kind = 0;
00944     AM.gCnumpows = 0;
00945     AC.exprfillwarning = 0;
00946     AM.gLineLength = 79;
00947     AM.OutBufSize = 80;
00948     AM.MaxStreamSize = MAXFILESTREAMSIZE;
00949     AP.MaxPreAssignLevel = 4;
00950     AC.iBufferSize = 512;
00951     AP.pSize = 128;
00952     AP.MaxPreIfLevel = 10;
00953     AP.cComChar = AP.ComChar = '*';
00954     AP.firstnamespace = 0;
00955     AP.lastnamespace = 0;
00956     AP.fullnamesize = 127;
00957     AP.fullname = (UBYTE *) Malloc1((AP.fullnamesize+1)*sizeof(UBYTE), "AP.fullname");
00958     AM.OffsetVector = -2*WILDOFFSET+MINSPEC;
00959     AC.cbuffList.num = 0;
00960     AM.hparallelflag = AM.gparallelflag =

```

```

00961     AC.parallelfld = AC.mparallelfld = PARALLELFld;
00962 #ifdef WITHMPI
00963     if ( PF.numtasks < 2 ) AM.hparallelfld |= NOPARALLEL_NPROC;
00964 #endif
00965     AC.tablefilling = 0;
00966     AC.models = 0;
00967     AC.nummodels = 0;
00968     AC.ModelLevel = 0;
00969     AC.modelspace = 0;
00970     AM.resetTimeOnClear = 1;
00971     AM.gnumextrasym = AM.ggnumextrasym = 0;
00972     AM.havesortdir = 0;
00973     AM.SpectatorFiles = 0;
00974     AM.NumSpectatorFiles = 0;
00975     AM.SizeForSpectatorFiles = 0;
00976 /*
00977     Information for the lists of variables. Part of error message and size:
00978 */
00979     AP.ProcList.message = "procedure";
00980     AP.ProcList.size = sizeof(PROCEDURE);
00981     AP.LoopList.message = "doloop";
00982     AP.LoopList.size = sizeof(DOLOOP);
00983     AP.PreVarList.message = "PreVariable";
00984     AP.PreVarList.size = sizeof(PREVAR);
00985     AC.SymbolList.message = "symbol";
00986     AC.SymbolList.size = sizeof(struct SyMbOl);
00987     AC.IndexList.message = "index";
00988     AC.IndexList.size = sizeof(struct InDeX);
00989     AC.VectorList.message = "vector";
00990     AC.VectorList.size = sizeof(struct VeCtOr);
00991     AC.FunctionList.message = "function";
00992     AC.FunctionList.size = sizeof(struct FuNcTiOn);
00993     AC.SetList.message = "set";
00994     AC.SetList.size = sizeof(struct SeTs);
00995     AC.SetElementList.message = "set element";
00996     AC.SetElementList.size = sizeof(WORD);
00997     AC.ExpressionList.message = "expression";
00998     AC.ExpressionList.size = sizeof(struct ExPrEsSiOn);
00999     AC.cbufList.message = "compiler buffer";
01000     AC.cbufList.size = sizeof(CBUF);
01001     AC.Channellist.message = "channel buffer";
01002     AC.Channellist.size = sizeof(CHANNEL);
01003     AP.DollarList.message = "$-variable";
01004     AP.DollarList.size = sizeof(struct DoLLArS);
01005     AC.DubiousList.message = "ambiguous variable";
01006     AC.DubiousList.size = sizeof(struct DuBiOuS);
01007     AC.TableBaseList.message = "list of tablebases";
01008     AC.TableBaseList.size = sizeof(DBASE);
01009     AC.TestValue = 0;
01010     AC.InnerTest = 0;
01011
01012     AC.AutoSymbolList.message = "autosymbol";
01013     AC.AutoSymbolList.size = sizeof(struct SyMbOl);
01014     AC.AutoIndexList.message = "autoindex";
01015     AC.AutoIndexList.size = sizeof(struct InDeX);
01016     AC.AutoVectorList.message = "autovector";
01017     AC.AutoVectorList.size = sizeof(struct VeCtOr);
01018     AC.AutoFunctionList.message = "autofunction";
01019     AC.AutoFunctionList.size = sizeof(struct FuNcTiOn);
01020     AC.PotModDolList.message = "potentially modified dollar";
01021     AC.PotModDolList.size = sizeof(WORD);
01022     AC.ModOptDolList.message = "moduleoptiondollar";
01023     AC.ModOptDolList.size = sizeof(MODOPTDOLLAR);
01024
01025     AO.FortDotChar = '_';
01026     AO.ErrorBlock = 0;
01027     AC.firstconstindex = 1;
01028     AO.Optimize.mctsconstant.fval = 1.0;
01029     AO.Optimize.horner = O_MCTS;
01030     AO.Optimize.hornerdirection = O_FORWARDORBACKWARD;
01031     AO.Optimize.method = O_GREEDY;
01032     AO.Optimize.mctstimelimit = 0;
01033     AO.Optimize.mctsnunexpand = 1000;
01034     AO.Optimize.mctsnunkeep = 10;
01035     AO.Optimize.mctsnunrepeat = 1;
01036     AO.Optimize.greedytimelimit = 0;
01037     AO.Optimize.greedyminnum = 10;
01038     AO.Optimize.greedymaxperc = 5;
01039     AO.Optimize.printstats = 0;
01040     AO.Optimize.debugflags = 0;
01041     AO.OptimizeResult.code = NULL;
01042     AO.inscheme = 0;
01043     AO.schemenum = 0;
01044     AO.wpos = 0;
01045     AO.wpoin = 0;
01046     AO.wlen = 0;
01047     AM.dollarzero = 0;

```

```

01048     AC.doloopstack = 0;
01049     AC.doloopstacksize = 0;
01050     AC.dolooplevel = 0;
01051 /*
01052     Set up the main name trees:
01053 */
01054     AC.varnames = MakeNameTree();
01055     AC.exprnames = MakeNameTree();
01056     AC.dollarnames = MakeNameTree();
01057     AC.autonames = MakeNameTree();
01058     AC.activenames = &(AC.varnames);
01059     AP.preError = 0;
01060 /*
01061     Initialize the compiler:
01062 */
01063     inictable();
01064     AM.rbufnum = inicbufs(); /* Regular compiler buffer */
01065 #ifndef WITHPTHREADS
01066     AT.ebufnum = inicbufs(); /* Buffer for extras during execution */
01067     AT.fbufnum = inicbufs(); /* Buffer for caching in factorization */
01068     AT.allbufnum = inicbufs(); /* Buffer for id,all */
01069     AT.aebufnum = inicbufs(); /* Buffer for id,all */
01070     AN.tryterm = 0;
01071 #else
01072     AS.MasterSort = 0;
01073 #endif
01074     AM.dbufnum = inicbufs(); /* Buffer for dollar variables */
01075     AM.sbufnum = inicbufs(); /* Subterm buffer for polynomials and optimization */
01076     AC.ffbufnum = inicbufs(); /* Buffer number for user defined factorizations */
01077     AM.zbufnum = inicbufs(); /* For very special values */
01078     {
01079         CBUF *C = cbuf+AM.zbufnum;
01080         WORD one[5] = {4,1,1,3,0};
01081         WORD zero = 0;
01082         AddRHS(AM.zbufnum,1);
01083         AM.zerorhs = C->numrhs;
01084         AddNtoC(AM.zbufnum,1,&zero,17);
01085         AddRHS(AM.zbufnum,1);
01086         AM.onerhs = C->numrhs;
01087         AddNtoC(AM.zbufnum,5,one,17);
01088     }
01089     AP.inside.inscbuf = inicbufs(); /* For the #inside instruction */
01090 /*
01091     Enter the built in objects
01092 */
01093     AC.Symbols = &(AC.SymbolList);
01094     AC.Indices = &(AC.IndexList);
01095     AC.Vectors = &(AC.VectorList);
01096     AC.Functions = &(AC.FunctionList);
01097     AC.vetofilling = 0;
01098
01099     AddDollar((UBYTE *)"$",DOLUNDEFINED,0,0);
01100
01101     cbuf[AM.dbufnum].mnumlhs = cbuf[AM.dbufnum].numlhs;
01102     cbuf[AM.dbufnum].mnumrhs = cbuf[AM.dbufnum].numrhs;
01103
01104     AddSymbol((UBYTE *)"i_",-MAXPOWER,MAXPOWER,VARTYPEIMAGINARY,0);
01105     AddSymbol((UBYTE *)"pi_",-MAXPOWER,MAXPOWER,VARTYPENONE,0);
01106 /*
01107     coeff_ should have the number COEFFSYMBOL and den_ the number DENOMINATOR
01108     and the three should be in this order!
01109 */
01110     AddSymbol((UBYTE *)"coeff_",-MAXPOWER,MAXPOWER,VARTYPENONE,0);
01111     AddSymbol((UBYTE *)"num_",-MAXPOWER,MAXPOWER,VARTYPENONE,0);
01112     AddSymbol((UBYTE *)"den_",-MAXPOWER,MAXPOWER,VARTYPENONE,0);
01113     AddSymbol((UBYTE *)"xarg_",-MAXPOWER,MAXPOWER,VARTYPENONE,0);
01114     AddSymbol((UBYTE *)"dimension_",-MAXPOWER,MAXPOWER,VARTYPENONE,0);
01115     AddSymbol((UBYTE *)"factor_",-MAXPOWER,MAXPOWER,VARTYPENONE,0);
01116     AddSymbol((UBYTE *)"sep_",-MAXPOWER,MAXPOWER,VARTYPENONE,0);
01117     AddSymbol((UBYTE *)"ee_",-MAXPOWER,MAXPOWER,VARTYPENONE,0);
01118     AddSymbol((UBYTE *)"em_",-MAXPOWER,MAXPOWER,VARTYPENONE,0);
01119     i = BUILTINSYMBOLS; /* update this in ftypes.h when we add new symbols */
01120 /*
01121     Next we add a number of dummy symbols for ensuring that the user defined
01122     symbols start at a fixed given number FIRSTUSERSYMBOL
01123     We do want to give them unique names though that the user cannot access.
01124 */
01125     {
01126         char dumstr[20];
01127         for ( ; i < FIRSTUSERSYMBOL; i++ ) {
01128             sprintf(dumstr,20,"%d:",i);
01129             AddSymbol((UBYTE *)dumstr,-MAXPOWER,MAXPOWER,VARTYPENONE,0);
01130         }
01131     }
01132
01133     AddIndex((UBYTE *)"iarg_",4,0);
01134     AddVector((UBYTE *)"parg_",VARTYPENONE,0);

```

```

01135
01136     AM.NumFixedFunctions = sizeof(fixedfunctions)/sizeof(struct fixedfun);
01137     for ( i = 0; i < AM.NumFixedFunctions; i++ ) {
01138         ii = AddFunction((UBYTE *)fixedfunctions[i].name
01139             ,fixedfunctions[i].commu
01140             ,fixedfunctions[i].tensor
01141             ,fixedfunctions[i].complx
01142             ,fixedfunctions[i].symmetric
01143             ,0,-1,-1);
01144         if ( fixedfunctions[i].tensor == GAMMAFUNCTION )
01145             functions[ii].flags |= COULDCOMMUTE;
01146     }
01147     /*
01148     Next we add a number of dummy functions for ensuring that the user defined
01149     functions start at a fixed given number FIRSTUSERFUNCTION.
01150     We do want to give them unique names though that the user cannot access.
01151     */
01152     {
01153         char dumstr[20];
01154         for ( i < FIRSTUSERFUNCTION-FUNCTION; i++ ) {
01155             snprintf(dumstr,20,"%d:",i);
01156             AddFunction((UBYTE *)dumstr,0,0,0,0,-1,-1);
01157         }
01158     }
01159     AM.NumFixedSets = sizeof(fixedsets)/sizeof(struct fixedset);
01160     for ( i = 0; i < AM.NumFixedSets; i++ ) {
01161         ii = AddSet((UBYTE *)fixedsets[i].name,fixedsets[i].dimension);
01162         Sets[ii].type = fixedsets[i].type;
01163     }
01164     AM.RepMax = MAXREPEAT;
01165     #ifndef WITHPTHREADS
01166     AT.RepCount = (int *)Malloc1((LONG)((AM.RepMax+3)*sizeof(int)),"repeat buffers");
01167     AN.RepPoint = AT.RepCount;
01168     AT.RepTop = AT.RepCount + AM.RepMax;
01169     AN.polysortflag = 0;
01170     AN.subsubveto = 0;
01171     #endif
01172     AC.NumWildcardNames = 0;
01173     AC.WildcardBufferSize = 50;
01174     AC.WildcardNames = (UBYTE *)Malloc1((LONG)AC.WildcardBufferSize,"argument list names");
01175     #ifndef WITHPTHREADS
01176     AT.WildArgTaken = (WORD *)Malloc1((LONG)AC.WildcardBufferSize*sizeof(WORD)/2
01177         ,"argument list names");
01178     AT.WildcardBufferSize = AC.WildcardBufferSize;
01179     AR.CompareRoutine = (COMPAREDDUMMY)(&Compare1);
01180     AT.nfac = AT.nBer = 0;
01181     AT.factorials = 0;
01182     AT.bernoullis = 0;
01183     AR.wranfia = 0;
01184     AR.wranfcall = 0;
01185     AR.wranfnpair1 = NPAIR1;
01186     AR.wranfnpair2 = NPAIR2;
01187     AR.wranfseed = 0;
01188     #endif
01189     AM.atstartup = 1;
01190     AM.oldnumextrasymbols = strDup1((UBYTE *)"OLDNUMEXTRASYMBOLS_","oldnumextrasymbols");
01191     PutPreVar((UBYTE *)"VERSION_",(UBYTE *)STRINGIFY(MAJORVERSION),0,0);
01192     PutPreVar((UBYTE *)"SUBVERSION_",(UBYTE *)STRINGIFY(MINORVERSION),0,0);
01193     PutPreVar((UBYTE *)"DATE_",(UBYTE *)MakeDate(),0,0);
01194     PutPreVar((UBYTE *)"random_",(UBYTE *)"_____",(UBYTE *)"?a",0);
01195     PutPreVar((UBYTE *)"optimminvar_",(UBYTE *)("0"),0,0);
01196     PutPreVar((UBYTE *)"optimmaxvar_",(UBYTE *)("0"),0,0);
01197     PutPreVar(AM.oldnumextrasymbols,(UBYTE *)("0"),0,0);
01198     PutPreVar((UBYTE *)"optimvalue_",(UBYTE *)("0"),0,0);
01199     PutPreVar((UBYTE *)"optimscheme_",(UBYTE *)("0"),0,0);
01200     PutPreVar((UBYTE *)"tolower_",(UBYTE *)("0"),(UBYTE *)("?a"),0);
01201     PutPreVar((UBYTE *)"toupper_",(UBYTE *)("0"),(UBYTE *)("?a"),0);
01202     PutPreVar((UBYTE *)"takeleft_",(UBYTE *)("0"),(UBYTE *)("?a"),0);
01203     PutPreVar((UBYTE *)"takeright_",(UBYTE *)("0"),(UBYTE *)("?a"),0);
01204     PutPreVar((UBYTE *)"keepleft_",(UBYTE *)("0"),(UBYTE *)("?a"),0);
01205     PutPreVar((UBYTE *)"keepright_",(UBYTE *)("0"),(UBYTE *)("?a"),0);
01206     PutPreVar((UBYTE *)"SYSTEMERROR_",(UBYTE *)("0"),0,0);
01207     /*
01208     Next are a few 'constants' for diagram generation
01209     */
01210     PutPreVar((UBYTE *)"ONEPI_",(UBYTE *)("1"),0,0);
01211     PutPreVar((UBYTE *)"WITHOUTINSERTIONS_",(UBYTE *)("2"),0,0);
01212     PutPreVar((UBYTE *)"NOTADPOLES_",(UBYTE *)("4"),0,0);
01213     PutPreVar((UBYTE *)"SYMMETRIZE_",(UBYTE *)("8"),0,0);
01214     PutPreVar((UBYTE *)"TOPOLOGIESONLY_",(UBYTE *)("16"),0,0);
01215     PutPreVar((UBYTE *)"NONODES_",(UBYTE *)("32"),0,0);
01216     PutPreVar((UBYTE *)"WITHEDGES_",(UBYTE *)("64"),0,0);
01217     /* Note that CHECKEXTERN is 128 */
01218     PutPreVar((UBYTE *)"WITHBLOCKS_",(UBYTE *)("256"),0,0);
01219     PutPreVar((UBYTE *)"WITHONEPISETS_",(UBYTE *)("512"),0,0);
01220     PutPreVar((UBYTE *)"NOSNAILS_",(UBYTE *)("1024"),0,0);
01221     PutPreVar((UBYTE *)"NOEXTSELF_",(UBYTE *)("2048"),0,0);

```

```

01222
01223     {
01224         char buf[41]; /* up to 128-bit */
01225         LONG pid;
01226 #ifndef WITHMPI
01227         pid = GetPID();
01228 #else
01229         pid = ( PF.me == MASTER ) ? GetPID() : (LONG)0;
01230         pid = PF_BroadcastNumber(pid);
01231 #endif
01232         LongCopy(pid,buf);
01233         PutPreVar((UBYTE *) "PID_", (UBYTE *)buf,0,0);
01234     }
01235     AM.atstartup = 0;
01236     AP.MaxPreTypes = 10;
01237     AP.NumPreTypes = 0;
01238     AP.PreTypes = (int *)Malloc1(sizeof(int)*(AP.MaxPreTypes+1),"preprocessor types");
01239     AP.inside.buffer = 0;
01240     AP.inside.size = 0;
01241
01242     AC.SortType = AC.lSortType = AM.gSortType = SORTLOWFIRST;
01243 #ifdef WITHPTHREADS
01244 #else
01245     AR.SortType = AC.SortType;
01246 #endif
01247     AC.LogHandle = -1;
01248     AC.SetList.numtemp = AC.SetList.num;
01249     AC.SetElementList.numtemp = AC.SetElementList.num;
01250
01251     GetName(AC.varnames, (UBYTE *) "exp_", &AM.expnum, NOAUTO);
01252     GetName(AC.varnames, (UBYTE *) "denom_", &AM.denomnum, NOAUTO);
01253     GetName(AC.varnames, (UBYTE *) "fac_", &AM.facnum, NOAUTO);
01254     GetName(AC.varnames, (UBYTE *) "invfac_", &AM.invfacnum, NOAUTO);
01255     GetName(AC.varnames, (UBYTE *) "sum_", &AM.sumnum, NOAUTO);
01256     GetName(AC.varnames, (UBYTE *) "sump_", &AM.sumpnum, NOAUTO);
01257     GetName(AC.varnames, (UBYTE *) "term_", &AM.termfunnum, NOAUTO);
01258     GetName(AC.varnames, (UBYTE *) "match_", &AM.matchfunnum, NOAUTO);
01259     GetName(AC.varnames, (UBYTE *) "count_", &AM.countfunnum, NOAUTO);
01260     AM.termfunnum += FUNCTION;
01261     AM.matchfunnum += FUNCTION;
01262     AM.countfunnum += FUNCTION;
01263
01264     AC.ThreadStats = AM.gThreadStats = AM.ggThreadStats = 1;
01265     AC.FinalStats = AM.gFinalStats = AM.ggFinalStats = 1;
01266     AC.StatsFlag = AM.gStatsFlag = AM.ggStatsFlag = 1;
01267     AC.ThreadsFlag = AM.gThreadsFlag = AM.ggThreadsFlag = 1;
01268     AC.ThreadBalancing = AM.gThreadBalancing = AM.ggThreadBalancing = 1;
01269     AC.ThreadSortFileSynch = AM.gThreadSortFileSynch = AM.ggThreadSortFileSynch = 0;
01270     AC.ProcessStats = AM.gProcessStats = AM.ggProcessStats = 1;
01271     AC.OldParallelStats = AM.gOldParallelStats = AM.ggOldParallelStats = 0;
01272     AC.OldFactArgFlag = AM.gOldFactArgFlag = AM.ggOldFactArgFlag = NEWFACTARG;
01273     AC.OldGCDflag = AM.gOldGCDflag = AM.ggOldGCDflag = 1;
01274     AC.WTimeStatsFlag = AM.gWTimeStatsFlag = AM.ggWTimeStatsFlag = 0;
01275     AM.gcNumDollars = AP.DollarList.num;
01276     AC.SizeCommuteInSet = AM.gSizeCommuteInSet = 0;
01277 #ifdef WITHFLOAT
01278     AC.MaxWeight = AM.gMaxWeight = AM.ggMaxWeight = MAXWEIGHT;
01279     AC.DefaultPrecision = AM.gDefaultPrecision = AM.ggDefaultPrecision = DEFAULTPRECISION;
01280 #endif
01281     AC.CommuteInSet = 0;
01282
01283     AM.PrintTotalSize = 0;
01284
01285     AO.NoSpacesInNumbers = AM.gNoSpacesInNumbers = AM.ggNoSpacesInNumbers = 0;
01286     AO.IndentSpace = AM.gIndentSpace = AM.ggIndentSpace = INDENTSPACE;
01287     AO.BlockSpaces = 0;
01288     AO.OptimizationLevel = 0;
01289     PUTZERO(AS.MaxExprSize);
01290     PUTZERO(AC.StoreFileSize);
01291
01292 #ifdef WITHPTHREADS
01293     AC.inputnumbers = 0;
01294     AC.pfirstnum = 0;
01295     AC.numpfirstnum = AC.sizepfirstnum = 0;
01296 #endif
01297     AC.MemDebugFlag = 1;
01298
01299 #ifdef WITHEXTERNALCHANNEL
01300     AX.currentExternalChannel=0;
01301     AX.killSignal=SIGKILL;
01302     AX.killWholeGroup=1;
01303     AX.daemonize=1;
01304     AX.currentPrompt=0;
01305     AX.timeout=1000; /*One second to initialize preset channels*/
01306     AX.shellname=strDup1((UBYTE *) "/bin/sh -c", "external channel shellname");
01307     AX.stdername=strDup1((UBYTE *) "/dev/null", "external channel stdername");
01308 #endif

```

```

01309 }
01310
01311 /*
01312     #] StartVariables :
01313     #[ StartMore :
01314 */
01315
01316 void StartMore(void)
01317 {
01318     #ifdef WITHEXTERNALCHANNEL
01319         /*If env.variable "FORM_PIPES" is defined, we have to initialize
01320         corresponding pre-set external channels, see file extcmd.c.*/
01321         /*This line must be after all setup settings: in future, timeout
01322         could be changed at setup.*/
01323         if (AX.timeout >= 0) /*if AX.timeout < 0, this was done by cmdline option -pipe*/
01324             initPresetExternalChannels((UBYTE*)getenv("FORM_PIPES"), AX.timeout);
01325     #endif
01326
01327     #ifdef WITHMPI
01328         /*
01329         Define preprocessor variable PARALLELTASK_ as a process number, 0 is the master
01330         Define preprocessor variable NPARALLELTASKS_ as a total number of processes
01331         */
01332         {
01333             UBYTE buf[32];
01334             snprintf((char*)buf, 32, "%d", PF.me);
01335             PutPreVar((UBYTE *) "PARALLELTASK_", buf, 0, 0);
01336             snprintf((char*)buf, 32, "%d", PF.numtasks);
01337             PutPreVar((UBYTE *) "NPARALLELTASKS_", buf, 0, 0);
01338         }
01339     #else
01340         PutPreVar((UBYTE *) "PARALLELTASK_", (UBYTE *) "0", 0, 0);
01341         PutPreVar((UBYTE *) "NPARALLELTASKS_", (UBYTE *) "1", 0, 0);
01342     #endif
01343
01344     PutPreVar((UBYTE *) "NAME_", AM.InputFileName ? AM.InputFileName : (UBYTE *) "STDIN", 0, 0);
01345 }
01346
01347 /*
01348     #] StartMore :
01349     #[ IniVars :
01350
01351     This routine initializes the parameters that may change during the run.
01352 */
01353
01354 WORD IniVars(void)
01355 {
01356     #ifdef WITHPTHREADS
01357         GETIDENTITY
01358     #else
01359         WORD *t;
01360     #endif
01361     WORD *fi, i, one = 1;
01362     CBUF *C = cbuf + AC.cbunum;
01363
01364     #ifdef WITHPTHREADS
01365         UBYTE buf[32];
01366         snprintf((char*)buf, 32, "%d", AM.totalnumberofthreads);
01367         PutPreVar((UBYTE *) "NTHREADS_", buf, 0, 1);
01368     #else
01369         PutPreVar((UBYTE *) "NTHREADS_", (UBYTE *) "1", 0, 1);
01370     #endif
01371
01372     AC.ShortStats = 0;
01373     AC.WarnFlag = 1;
01374     AR.SortType = AC.SortType = AC.lSortType = AM.gSortType;
01375     AC.OutputMode = 72;
01376     AC.OutputSpaces = NORMALFORMAT;
01377     AR.Eside = 0;
01378     AC.DumNum = 0;
01379     AC.ncmod = AM.gncmod = 0;
01380     AC.modmode = AM.gmodmode = 0;
01381     AC.npowmod = AM.gnpowmod = 0;
01382     AC.halfmod = 0; AC.nhalfmod = 0;
01383     AC.modinverses = 0;
01384     AC.lPolyFun = AM.gPolyFun = 0;
01385     AC.lPolyFunInv = AM.gPolyFunInv = 0;
01386     AC.lPolyFunType = AM.gPolyFunType = 0;
01387     AC.lPolyFunExp = AM.gPolyFunExp = 0;
01388     AC.lPolyFunVar = AM.gPolyFunVar = 0;
01389     AC.lPolyFunPow = AM.gPolyFunPow = 0;
01390     #ifdef WITHFLINT
01391     AC.FlintPolyFlag = 1;
01392     #else
01393     AC.FlintPolyFlag = 0;
01394     #endif
01395     AC.DirtPow = 0;

```

```

01396     AC.lDefDim = AM.gDefDim = 4;
01397     AC.lDefDim4 = AM.gDefDim4 = 0;
01398     AC.lUnitTrace = AM.gUnitTrace = 4;
01399     AC.NamesFlag = AM.gNamesFlag = 0;
01400     AC.CodesFlag = AM.gCodesFlag = 0;
01401     /* Printing a backtrace on crash is on by default for both normal and debug
01402        modes if FORM has been compiled with backtrace support. */
01403     #ifdef ENABLE_BACKTRACE
01404         AC.PrintBacktraceFlag = 1;
01405     #else
01406         AC.PrintBacktraceFlag = 0;
01407     #endif
01408     /* Human-readable statistics are off by default */
01409     AC.HumanStatsFlag = 0;
01410     AC.extrasymbols = AM.gextrasymbols = AM.ggextrasymbols = 0;
01411     AC.extrasym = (UBYTE *)Malloc1(2*sizeof(UBYTE), "extrasym");
01412     AM.gextrasym = (UBYTE *)Malloc1(2*sizeof(UBYTE), "extrasym");
01413     AM.ggextrasym = (UBYTE *)Malloc1(2*sizeof(UBYTE), "extrasym");
01414     AC.extrasym[0] = AM.gextrasym[0] = AM.ggextrasym[0] = '2';
01415     AC.extrasym[1] = AM.gextrasym[1] = AM.ggextrasym[1] = 0;
01416     AC.TokensWriteFlag = AM.gTokensWriteFlag = 0;
01417     AC.SetupFlag = 0;
01418     AC.LineLength = AM.gLineLength = 79;
01419     AC.NwildC = 0;
01420     AC.OutputMode = 0;
01421     AM.gOutputMode = 0;
01422     AC.OutputSpaces = NORMALFORMAT;
01423     AM.gOutputSpaces = NORMALFORMAT;
01424     AC.OutNumberType = RATIONALMODE;
01425     AM.gOutNumberType = RATIONALMODE;
01426     #ifdef WITHZLIB
01427         AR.gzipCompress = GZIPDEFAULT;
01428         AR.FoStage4[0].ziobuffer = 0;
01429         AR.FoStage4[1].ziobuffer = 0;
01430     #ifdef WITHZSTD
01431         /* Zstd compression is on by default, if we have compiled with it */
01432         ZWRAP_useZSTDcompression(1);
01433     #endif
01434     #endif
01435     AR.BracketOn = 0;
01436     AC.bracketindexflag = 0;
01437     AT.bracketindexflag = 0;
01438     AT.bracketinfo = 0;
01439     AO.IsBracket = 0;
01440     AM.gfunpowers = AC.funpowers = COMFUNPOWERS;
01441     AC.parallelfalg = AM.gparallelfalg;
01442     AC.properorderflag = AM.gproperorderflag = PROPERORDERFLAG;
01443     AC.ProcessBucketSize = AC.mProcessBucketSize = AM.gProcessBucketSize;
01444     AC.ThreadBucketSize = AM.gThreadBucketSize;
01445     AC.ShortStatsMax = 0;
01446     AM.gShortStatsMax = 0;
01447     AM.ggShortStatsMax = 0;
01448
01449     GlobalSymbols      = NumSymbols;
01450     GlobalIndices      = NumIndices;
01451     GlobalVectors      = NumVectors;
01452     GlobalFunctions    = NumFunctions;
01453     GlobalSets         = NumSets;
01454     GlobalSetElements  = NumSetElements;
01455     AC.modpowers = (UWORD *)0;
01456
01457     i = AM.OffsetIndex;
01458     fi = AC.FixIndices;
01459     if ( i > 0 ) do { *fi++ = one; } while ( --i >= 0 );
01460     AR.sLevel = -1;
01461     AM.Ordering[0] = 5;
01462     AM.Ordering[1] = 6;
01463     AM.Ordering[2] = 7;
01464     AM.Ordering[3] = 0;
01465     AM.Ordering[4] = 1;
01466     AM.Ordering[5] = 2;
01467     AM.Ordering[6] = 3;
01468     AM.Ordering[7] = 4;
01469     for ( i = 8; i < 15; i++ ) AM.Ordering[i] = i;
01470     AM.gUnitTrace[0] =
01471     AC.lUnitTrace[0] = SNUMBER;
01472     AM.gUnitTrace[1] =
01473     AC.lUnitTrace[1] =
01474     AM.gUnitTrace[2] =
01475     AC.lUnitTrace[2] = 4;
01476     AM.gUnitTrace[3] =
01477     AC.lUnitTrace[3] = 1;
01478     #ifdef WITHPTHREADS
01479     AS.Balancing = 0;
01480     #else
01481     AT.MinVecArg[0] = 7+ARGHEAD;
01482     AT.MinVecArg[ARGHEAD] = 7;

```



```

01483     AT.MinVecArg[1+ARGHEAD] = INDEX;
01484     AT.MinVecArg[2+ARGHEAD] = 3;
01485     AT.MinVecArg[3+ARGHEAD] = 0;
01486     AT.MinVecArg[4+ARGHEAD] = 1;
01487     AT.MinVecArg[5+ARGHEAD] = 1;
01488     AT.MinVecArg[6+ARGHEAD] = -3;
01489     t = AT.FunArg;
01490     *t++ = 4+ARGHEAD+FUNHEAD;
01491     for ( i = 1; i < ARGHEAD; i++ ) *t++ = 0;
01492     *t++ = 4+FUNHEAD;
01493     *t++ = 0;
01494     *t++ = FUNHEAD;
01495     for ( i = 2; i < FUNHEAD; i++ ) *t++ = 0;
01496     *t++ = 1; *t++ = 1; *t++ = 3;
01497
01498 #ifdef WITHMPI
01499     AS.printflag = 0;
01500 #endif
01501
01502     AT.comsym[0] = 8;
01503     AT.comsym[1] = SYMBOL;
01504     AT.comsym[2] = 4;
01505     AT.comsym[3] = 0;
01506     AT.comsym[4] = 1;
01507     AT.comsym[5] = 1;
01508     AT.comsym[6] = 1;
01509     AT.comsym[7] = 3;
01510     AT.comnum[0] = 4;
01511     AT.comnum[1] = 1;
01512     AT.comnum[2] = 1;
01513     AT.comnum[3] = 3;
01514     AT.comfun[0] = FUNHEAD+4;
01515     AT.comfun[1] = FUNCTION;
01516     AT.comfun[2] = FUNHEAD;
01517     AT.comfun[3] = 0;
01518     #if FUNHEAD == 4
01519     AT.comfun[4] = 0;
01520 #endif
01521     AT.comfun[FUNHEAD+1] = 1;
01522     AT.comfun[FUNHEAD+2] = 1;
01523     AT.comfun[FUNHEAD+3] = 3;
01524     AT.comind[0] = 7;
01525     AT.comind[1] = INDEX;
01526     AT.comind[2] = 3;
01527     AT.comind[3] = 0;
01528     AT.comind[4] = 1;
01529     AT.comind[5] = 1;
01530     AT.comind[6] = 3;
01531     AT.locwildvalue[0] = SUBEXPRESSION;
01532     AT.locwildvalue[1] = SUBEXPSIZE;
01533     for ( i = 2; i < SUBEXPSIZE; i++ ) AT.locwildvalue[i] = 0;
01534     AT.mulpat[0] = TYPMULT;
01535     AT.mulpat[1] = SUBEXPSIZE+3;
01536     AT.mulpat[2] = 0;
01537     AT.mulpat[3] = SUBEXPRESSION;
01538     AT.mulpat[4] = SUBEXPSIZE;
01539     AT.mulpat[5] = 0;
01540     AT.mulpat[6] = 1;
01541     for ( i = 7; i < SUBEXPSIZE+5; i++ ) AT.mulpat[i] = 0;
01542     AT.proexp[0] = SUBEXPSIZE+4;
01543     AT.proexp[1] = EXPRESSION;
01544     AT.proexp[2] = SUBEXPSIZE;
01545     AT.proexp[3] = -1;
01546     AT.proexp[4] = 1;
01547     for ( i = 5; i < SUBEXPSIZE+1; i++ ) AT.proexp[i] = 0;
01548     AT.proexp[SUBEXPSIZE+1] = 1;
01549     AT.proexp[SUBEXPSIZE+2] = 1;
01550     AT.proexp[SUBEXPSIZE+3] = 3;
01551     AT.proexp[SUBEXPSIZE+4] = 0;
01552     AT.dummysubexp[0] = SUBEXPRESSION;
01553     AT.dummysubexp[1] = SUBEXPSIZE+4;
01554     for ( i = 2; i < SUBEXPSIZE; i++ ) AT.dummysubexp[i] = 0;
01555     AT.dummysubexp[SUBEXPSIZE] = WILDDUMMY;
01556     AT.dummysubexp[SUBEXPSIZE+1] = 4;
01557     AT.dummysubexp[SUBEXPSIZE+2] = 0;
01558     AT.dummysubexp[SUBEXPSIZE+3] = 0;
01559
01560     AT.inprimelist = -1;
01561     AT.sizeprimelist = 0;
01562     AT.primelist = 0;
01563     AT.LeaveNegative = 0;
01564     AT.TrimPower = 0;
01565     AN.SplitScratch = 0;
01566     AN.SplitScratchSize = AN.InScratch = 0;
01567     AN.SplitScratch1 = 0;
01568     AN.SplitScratchSize1 = AN.InScratch1 = 0;
01569     AN.idfunctionflag = 0;

```

```

01570 #endif
01571     AO.OutputLine = AO.OutFill = BufferForOutput;
01572     AO.FactorMode = 0;
01573     C->Pointer = C->Buffer;
01574
01575     AP.PreOut = 0;
01576     AP.ComChar = AP.cComChar;
01577     AC.cbufnum = AM.rbufnum;          /* Select the default compiler buffer */
01578     AC.HideLevel = 0;
01579     AP.PreAssignFlag = 0;
01580     return(0);
01581 }
01582
01583 /*
01584     #] IniVars :
01585     #[ Signal handlers :
01586 */
01587 /*[28apr2004 mt]:*/
01588 #ifdef TRAP_SIGNALS
01589
01590 static int exitInProgress = 0;
01591 static int trappedTerminate = 0;
01592
01593 /*INTSIGHANDLER : some systems require a signal handler to return an integer,
01594 so define the macro INTSIGHANDLER if compiler fails:*/
01595 #ifdef INTSIGHANDLER
01596 static int onErrSig(int i)
01597 #else
01598 static void onErrSig(int i)
01599 #endif
01600 {
01601     if (exitInProgress){
01602         signal(i, SIG_DFL); /* Use default behaviour*/
01603         raise (i); /*reproduce trapped signal*/
01604 #ifdef INTSIGHANDLER
01605         return(i);
01606 #else
01607         return;
01608 #endif
01609     }
01610     trappedTerminate = 1;
01611     /*[13jul2005 mt]:*/ /*TerminateImpl(-1) on signal is here:*/
01612     Terminate(-1);
01613 }
01614
01615 #ifdef INTSIGHANDLER
01616 static void setNewSig(int i, int (*handler)(int))
01617 #else
01618 static void setNewSig(int i, void (*handler)(int))
01619 #endif
01620 {
01621     if (! (i<NSIG) ) /* Invalid signal -- see comments in the file */
01622         return;
01623     if ( signal(i, SIG_IGN) !=SIG_IGN)
01624         /* if compiler fails here, try to define INTSIGHANDLER:*/
01625         signal(i, handler);
01626 }
01627
01628 void setSignalHandlers(void)
01629 {
01630     /* Reset various unrecoverable error signals:*/
01631     setNewSig(SIGSEGV, onErrSig);
01632     setNewSig(SIGFPE, onErrSig);
01633     setNewSig(SIGILL, onErrSig);
01634     setNewSig(SIGEMT, onErrSig);
01635     setNewSig(SIGSYS, onErrSig);
01636     setNewSig(SIGPIPE, onErrSig);
01637     setNewSig(SIGLOST, onErrSig);
01638     setNewSig(SIGXCPU, onErrSig);
01639     setNewSig(SIGXFSZ, onErrSig);
01640
01641     /* Reset interrupt signals:*/
01642     setNewSig(SIGTERM, onErrSig);
01643     setNewSig(SIGINT, onErrSig);
01644     setNewSig(SIGQUIT, onErrSig);
01645     setNewSig(SIGHUP, onErrSig);
01646     setNewSig(SIGALRM, onErrSig);
01647     setNewSig(SIGVTALRM, onErrSig);
01648     /* setNewSig(SIGPROF, onErrSig); */ /* Why did Tentukov forbid profilers?? */
01649 }
01650
01651 #endif
01652 /*[28apr2004 mt]:*/
01653 /*
01654     #] Signal handlers :
01655     #[ main :
01656 */

```

```

01657
01658 #ifdef WITHPTHREADS
01659 ALLPRIVATES *ABdummy[10];
01660 #endif
01661
01662 int main(int argc, char **argv)
01663 {
01664     int retval;
01665     bzero((void *)(&A), sizeof(A)); /* make sure A is initialized at zero */
01666     iniTools();
01667 #ifdef TRAP_SIGNALS
01668     setSignalHandlers();
01669 #endif
01670 #ifdef WINDOWS
01671     _setmode(_fileno(stdout), O_BINARY);
01672 #endif
01673
01674 #ifdef WITHPTHREADS
01675     AB = ABdummy;
01676     StartHandleLock();
01677     BeginIdentities();
01678 #else
01679     AM.SumTime = TimeCPU(0);
01680     TimeWallClock(0);
01681 #endif
01682
01683 #ifdef WITHMPI
01684     if ( PF_Init(&argc, &argv) ) exit(-1);
01685 #endif
01686
01687     StartFiles();
01688     StartVariables();
01689 #ifdef WITHMPI
01690     /*
01691     * Here MesPrint() is ready. We turn on AS.printflag to print possible
01692     * errors occurring on slaves in the initialization. With AS.printflag = -1
01693     * MesPrint() does not use the synchronized output. This may lead broken
01694     * texts in the output somewhat, but it is safer to implement in this way
01695     * for the situation in which some of MesPrint() calls use MLOCK()-MUNLOCK()
01696     * and some do not. In future if we set AS.printflag = 1 and modify the
01697     * source code such that all MesPrint() calls are sandwiched by MLOCK()-
01698     * MUNLOCK(), we need also to modify the code for the master to catch
01699     * messages corresponding to MUNLOCK() calls at some point.
01700     *
01701     * AS.printflag will be set to 0 in IniVars() to prevent slaves from
01702     * printing redundant errors in the preprocessor and compiler (e.g., syntax
01703     * errors).
01704     */
01705     AS.printflag = -1;
01706 #endif
01707
01708     if ( ( retval = DoTail(argc, (UBYTE **)argv) ) != 0 ) {
01709         if ( retval > 0 ) Terminate(0);
01710         else Terminate(-1);
01711     }
01712     if ( DoSetups() ) Terminate(-2);
01713 #ifdef WITHMPI
01714     /* It is messy if all errors in OpenInput() on slaves are printed. */
01715     AS.printflag = 0;
01716 #endif
01717     if ( OpenInput() ) Terminate(-3);
01718 #ifdef WITHMPI
01719     AS.printflag = -1;
01720 #endif
01721     if ( TryEnvironment() ) Terminate(-2);
01722     if ( TryFileSetups() ) Terminate(-2);
01723     if ( MakeSetupAllocs() ) Terminate(-2);
01724     StartMore();
01725     InitRecovery();
01726     CheckRecoveryFile();
01727     if ( AM.totalnumberofthreads == 0 ) AM.totalnumberofthreads = 1;
01728     AS.MultiThreaded = 0;
01729 #ifdef WITHPTHREADS
01730     if ( AM.totalnumberofthreads > 1 ) AS.MultiThreaded = 1;
01731     ReserveTempFiles(1);
01732     StartAllThreads(AM.totalnumberofthreads);
01733     IniFbufs();
01734 #else
01735     ReserveTempFiles(0);
01736     IniFbuffer(AT.fbufnum);
01737 #endif
01738     if ( !AM.FromStdin ) PrintHeader(1);
01739     IniVars();
01740     Globalize(1);
01741 #ifdef WITH_ALARM
01742     if ( AM.TimeLimit > 0 ) alarm(AM.TimeLimit);
01743 #endif

```

```

01744     TimeCPU(0);
01745     TimeChildren(0);
01746     TimeWallClock(0);
01747     PreProcessor();
01748     Terminate(0);
01749     return(0);
01750 }
01751 /*
01752     #] main :
01753     #[ Cleanup :
01754
01755     if par < 0 we have to keep the storage file.
01756     when par > 0 we ran into a .clear statement.
01757     In that case we keep the zero level input and the log file.
01758
01759 */
01760
01761 void Cleanup(WORD par)
01762 {
01763     GETIDENTITY
01764     int i;
01765
01766     if ( FG.fname ) {
01767 #ifdef WITHPTHREADS
01768         if ( B ) {
01769 #endif
01770             CleanupSort(0);
01771             for ( i = 0; i < 3; i++ ) {
01772                 if ( AR.Fscr[i].handle >= 0 ) {
01773                     if ( AR.Fscr[i].name ) {
01774 /*
01775                         If there are more threads referring to the same file
01776                         only the one with the name is the owner of the file.
01777 */
01778                         CloseFile(AR.Fscr[i].handle);
01779                         remove(AR.Fscr[i].name);
01780                     }
01781                     AR.Fscr[i].handle = - 1;
01782                     AR.Fscr[i].POfill = 0;
01783                 }
01784             }
01785 #ifdef WITHPTHREADS
01786         }
01787 #endif
01788         if ( par > 0 ) {
01789 /*
01790             Close all input levels above the lowest?
01791 */
01792         }
01793         if ( AC.StoreHandle >= 0 && par <= 0 ) {
01794 #ifdef TRAP_SIGNALS
01795             if ( trappedTerminate ) { /* We don't throw .str if it has contents */
01796                 POSITION pos;
01797                 PUTZERO(pos);
01798                 SeekFile(AC.StoreHandle,&pos,SEEK_END);
01799                 if ( ISNOTZEROPOS(pos) ) {
01800                     CloseFile(AC.StoreHandle);
01801                     goto dontremove;
01802                 }
01803             }
01804             CloseFile(AC.StoreHandle);
01805 #ifdef WITHPTHREADS
01806             if ( B )
01807 #endif
01808                 if ( par >= 0 || AR.StoreData.Handle < 0 || AM.ClearStore ) {
01809                     remove(FG.fname);
01810                 }
01811             dontremove;;
01812         #else
01813             CloseFile(AC.StoreHandle);
01814 #ifdef WITHPTHREADS
01815             if ( B )
01816 #endif
01817                 if ( par >= 0 || AR.StoreData.Handle < 0 || AM.ClearStore > 0 ) {
01818                     remove(FG.fname);
01819                 }
01820             #endif
01821         }
01822     }
01823     ClearSpectators(CLEARMODULE);
01824 /*
01825     Remove recovery file on exit if everything went well
01826 */
01827     if ( par == 0 ) {
01828         DeleteRecoveryFile();
01829     }
01830 /*

```

```

01831     Now the final message concerning the total time
01832  */
01833     if ( AC.LogHandle >= 0 && par <= 0 ) {
01834         WORD lh = AC.LogHandle;
01835         AC.LogHandle = -1;
01836 #ifdef WITHMPI
01837         if ( PF.me == MASTER ) /* Only the master opened the real file. */
01838 #endif
01839         CloseFile(lh);
01840     }
01841 }
01842
01843 /*
01844     #] CleanUp :
01845     #[ TerminateImpl :
01846  */
01847
01848 static int firstterminate = 1;
01849
01850 void TerminateImpl(int errorcode, const char* file, int line, const char* function)
01851 {
01852     if ( errorcode && firstterminate ) {
01853         firstterminate = 0;
01854
01855         MLOCK(ErrorMessageLock);
01856 #ifdef WITHPTHREADS
01857         MesPrint("Program terminating in thread %w at %");
01858 #elif defined(WITHMPI)
01859         MesPrint("Program terminating in process %w at %");
01860 #else
01861         MesPrint("Program terminating at %");
01862 #endif
01863         MesPrint("Terminate called from %s:%d (%s)", file, line, function);
01864
01865         if ( AC.PrintBacktraceFlag ) {
01866 #ifdef ENABLE_BACKTRACE
01867             void *stack[64];
01868             int stacksize, stop = 0;
01869             stacksize = backtrace(stack, sizeof(stack)/sizeof(stack[0]));
01870
01871             /* First check whether eu-addr2line is available */
01872             if ( !system("command -v eu-addr2line > /dev/null 2>&1") ) {
01873                 MesPrint("Backtrace:");
01874                 for (int i = 0; i < stacksize && !stop; i++) {
01875                     FILE *fp;
01876                     char cmd[512];
01877                     // Leave an initial space
01878                     cmd[0] = ' ';
01879                     MesPrint("%#2d:%", i);
01880                     snprintf(cmd+1, sizeof(cmd)-1, "eu-addr2line -s --pretty-print -f -i '%p'
--pid=%d\n", stack[i], getpid());
01881                     fp = popen(cmd+1, "r");
01882                     while ( fgets(cmd+1, sizeof(cmd)-1, fp) != NULL ) {
01883                         MesPrint("%s", cmd);
01884                         /* Don't show functions lower than "main" (or thread equivalent) */
01885                         if ( strstr(cmd, " main ") || strstr(cmd, " RunThread ") || strstr(cmd, "
RunSortBot ") ) {
01886                             stop = 1;
01887                         }
01888                     }
01889                     pclose(fp);
01890                 }
01891             }
01892 #ifdef LINUX
01893             else if ( !system("command -v addr2line > /dev/null 2>&1") ) {
01894                 /* Get the executable path. */
01895                 char exe_path[PATH_MAX];
01896                 {
01897                     ssize_t len = readlink("/proc/self/exe", exe_path, sizeof(exe_path) - 1);
01898                     if ( len != -1 ) {
01899                         exe_path[len] = '\0';
01900                     }
01901                     else {
01902                         goto backtrace_fallback;
01903                     }
01904                 }
01905                 /* Assume PIE binary and get the base address. */
01906                 uintptr_t base_address = 0;
01907                 {
01908                     char line[256];
01909                     FILE *maps = fopen("/proc/self/maps", "r");
01910                     if ( !maps ) {
01911                         goto backtrace_fallback;
01912                     }
01913                     /* See the format used by nommu_region_show() in fs/proc/nommu.c of the Linux
source. */
01914                     if ( fgets(line, sizeof(line), maps) ) {

```

```

01915             sscanf(line, "%s SCNxPTR -", &base_address);
01916         }
01917         else {
01918             fclose(maps);
01919             goto backtrace_fallback;
01920         }
01921         fclose(maps);
01922     }
01923     char **strings;
01924     strings = backtrace_symbols(stack, stacksize);
01925     MesPrint("Backtrace:");
01926     for ( int i = 0; i < stacksize && !stop; i++ ) {
01927         FILE *fp;
01928         char cmd[PATH_MAX + 512];
01929         // Leave an initial space
01930         cmd[0] = ' ';
01931         uintptr_t addr = (uintptr_t)stack[i] - base_address;
01932         MesPrint("###%2d:", i);
01933         snprintf(cmd+1, sizeof(cmd)-1, "addr2line -e \"%s\" -i -p -s -f -C 0x%" PRIxPTR,
exe_path, addr);
01934         fp = popen(cmd+1, "r");
01935         while ( fgets(cmd+1, sizeof(cmd)-1, fp) != NULL ) {
01936             MesPrint("%s", cmd);
01937             /* Don't show functions lower than "main" */
01938             if ( strstr(cmd, " main ") || strstr(cmd, " RunThread ") || strstr(cmd, "
RunSortBot ") ) {
01939                 stop = 1;
01940             }
01941         }
01942         pclose(fp);
01943     }
01944     free(strings);
01945 }
01946 #endif
01947     else {
01948         /* eu-addr2line not found */
01949 #ifdef LINUX
01950 backtrace_fallback: ;
01951 #endif
01952         char **strings;
01953         strings = backtrace_symbols(stack, stacksize);
01954         MesPrint("Backtrace:");
01955         for ( int i = 0; i < stacksize && !stop; i++ ) {
01956             char *p = strings[i];
01957             while ( *p && *p != '(' ) p++;
01958             MesPrint("###%2d: %s\n", i, p);
01959             /* Don't show functions lower than "main" (or thread equivalent) */
01960             if ( strstr(p, "(main+") || strstr(p, "(RunThread+") || strstr(p, "(RunSortBot+")
) {
01961                 stop = 1;
01962             }
01963         }
01964 #ifdef LINUX
01965         MesPrint("Please install addr2line or eu-addr2line for readable stack information.");
01966     #else
01967         MesPrint("Please install eu-addr2line for readable stack information.");
01968     #endif
01969         free(strings);
01970     }
01971     #else
01972         MesPrint("FORM compiled without backtrace support.");
01973     #endif
01974     } /* if ( AC.PrintBacktraceFlag) { */
01975 }
01976 MUNLOCK(ErrorMessageLock);
01977 Crash();
01978 }
01979 #ifdef TRAP_SIGNALS
01980     exitInProgress=1;
01981 #endif
01982 #ifdef WITHEXTERNALCHANNEL
01983     /*
01984     This function can be called from the error handler, so it is better to
01985     clean up all started processes before any activity:
01986     */
01987     closeAllExternalChannels();
01988     AX.currentExternalChannel=0;
01989     /*[08may2006 mt]:*/
01990     AX.killSignal=SIGKILL;
01991     AX.killWholeGroup=1;
01992     AX.daemonize=1;
01993     /*:[08may2006 mt]:*/
01994     if(AX.currentPrompt){
01995         M_free(AX.currentPrompt,"external channel prompt");
01996         AX.currentPrompt=0;
01997     }
01998 }

```

```

01999      /*[08may2006 mt]:*/
02000      if (AX.shellname) {
02001          M_free(AX.shellname, "external channel shellname");
02002          AX.shellname=0;
02003      }
02004      if (AX.stdername) {
02005          M_free(AX.stdername, "external channel stdername");
02006          AX.stdername=0;
02007      }
02008      /*:[08may2006 mt]:*/
02009      #endif
02010      #ifdef WITHPTHREADS
02011      if ( !WhoAmI() && !errorcode ) {
02012          TerminateAllThreads();
02013      }
02014      #endif
02015      if ( AC.FinalStats ) {
02016          if ( AM.PrintTotalSize ) {
02017              MesPrint("Max. space for expressions: %19p bytes",&(AS.MaxExprSize));
02018          }
02019          PrintRunningTime();
02020      }
02021      #ifdef WITHMPI
02022      if ( AM.HoldFlag && PF.me == MASTER ) {
02023          WriteFile(AM.Stdout, (UBYTE *) ("Hit any key "),12);
02024          PF_FlushStdOutBuffer();
02025          getchar();
02026      }
02027      #else
02028      if ( AM.HoldFlag ) {
02029          WriteFile(AM.Stdout, (UBYTE *) ("Hit any key "),12);
02030          getchar();
02031      }
02032      #endif
02033      #ifdef WITHMPI
02034          PF_Terminate(errorcode);
02035      #endif
02036      /*
02037      We are about to terminate the program. If we are using flint, call the cleanup function.
02038      This keeps valgrind happy.
02039      */
02040      #ifdef WITHFLINT
02041          flint_final_cleanup_master();
02042      #endif
02043          CleanUp(errorcode);
02044          M_print();
02045      #ifdef VMS
02046          P_term(errorcode? 0: 1);
02047      #else
02048          P_term(errorcode);
02049      #endif
02050      }
02051      /*
02052          #] TerminateImpl :
02053          #[ PrintDeprecation :
02054          */
02055      void PrintDeprecation(const char *feature, const char *issue) {
02056      #ifdef WITHMPI
02057          if ( PF.me != MASTER ) return;
02058      #endif
02059          if ( AM.IgnoreDeprecation ) return;
02060          UBYTE *e = (UBYTE *)getenv("FORM_IGNORE_DEPRECATION");
02061          if ( e && e[0] && StrCmp(e, (UBYTE *)"0") && StrCmp(e, (UBYTE *)"false") && StrCmp(e, (UBYTE *)
02062          *) "no" ) ) return;
02063          MesPrint("DeprecationWarning: We are considering deprecating %s.", feature);
02064          MesPrint("If you want this support to continue, leave a comment at:");
02065          MesPrint("");
02066          MesPrint("    https://github.com/vermaseren/form/%s", issue);
02067          MesPrint("");
02068          MesPrint("Otherwise, it will be discontinued in the future.");
02069          MesPrint("To suppress this warning, use the -ignore-deprecation command line option or");
02070          MesPrint("set the environment variable FORM_IGNORE_DEPRECATION=1.");
02071      }
02072      /*
02073          #] PrintDeprecation :
02074          #[ PrintRunningTime :
02075          */
02076      void PrintRunningTime(void)
02077      {
02078      #if (defined(WITHPTHREADS) && (defined(WITHPOSIXCLOCK) || defined(WINDOWS))) || defined(WITHMPI)
02079          LONG mastertime;

```

```

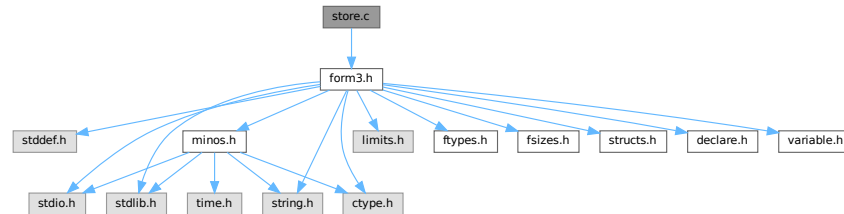
02091     LONG workertime;
02092     LONG wallclocktime;
02093     LONG totaltime;
02094 #if defined(WITHPTHREADS)
02095     if ( AB[0] != 0 ) {
02096         workertime = GetWorkerTimes();
02097 #else
02098     workertime = PF_GetSlaveTimes(); /* must be called on all processors */
02099     if ( PF.me == MASTER ) {
02100 #endif
02101         mastertime = AM.SumTime + TimeCPU(1);
02102         wallclocktime = TimeWallClock(1);
02103         totaltime = mastertime+workertime;
02104         if ( !AM.silent ) {
02105             MesPrint(" %l.%2i sec + %l.%2i sec: %l.%2i sec out of %l.%2i sec",
02106                 mastertime/1000, (WORD) (mastertime%1000)/10,
02107                 workertime/1000, (WORD) (workertime%1000)/10,
02108                 totaltime/1000, (WORD) (totaltime%1000)/10,
02109                 wallclocktime/100, (WORD) (wallclocktime%100));
02110         }
02111     }
02112 #else
02113     LONG mastertime = AM.SumTime + TimeCPU(1);
02114     LONG wallclocktime = TimeWallClock(1);
02115     if ( !AM.silent ) {
02116         MesPrint(" %l.%2i sec out of %l.%2i sec",
02117             mastertime/1000, (WORD) (mastertime%1000)/10,
02118             wallclocktime/100, (WORD) (wallclocktime%100));
02119     }
02120 #endif
02121 }
02122
02123 /*
02124     #] PrintRunningTime :
02125     #[ GetRunningTime :
02126 */
02127
02128 LONG GetRunningTime(void)
02129 {
02130 #if defined(WITHPTHREADS) && (defined(WITHPOSIXCLOCK) || defined(WINDOWS))
02131     LONG mastertime;
02132     if ( AB[0] != 0 ) {
02133 /*
02134 #if ( defined(APPLE64) || defined(APPLE32) )
02135     mastertime = AM.SumTime + TimeCPU(1);
02136     return(mastertime);
02137 #else
02138 */
02139         LONG workertime = GetWorkerTimes();
02140         mastertime = AM.SumTime + TimeCPU(1);
02141         return(mastertime+workertime);
02142 /*
02143 #endif
02144 */
02145     }
02146     else {
02147         return(AM.SumTime + TimeCPU(1));
02148     }
02149 #elif defined(WITHMPI)
02150     LONG mastertime, t = 0;
02151     LONG workertime = PF_GetSlaveTimes(); /* must be called on all processors */
02152     if ( PF.me == MASTER ) {
02153         mastertime = AM.SumTime + TimeCPU(1);
02154         t = mastertime + workertime;
02155     }
02156     return PF_BroadcastNumber(t); /* must be called on all processors */
02157 #else
02158     return(AM.SumTime + TimeCPU(1));
02159 #endif
02160 }
02161
02162 /*
02163     #] GetRunningTime :
02164 */

```


4.134 store.c File Reference

```
#include "form3.h"
```

Include dependency graph for store.c:



Macros

- #define [SAVEREVISION](#) 0x02

Functions

- WORD [OpenTemp](#) (void)
- void [SeekScratch](#) (FILEHANDLE *fi, POSITION *pos)
- void [SetEndScratch](#) (FILEHANDLE *f, POSITION *position)
- void [SetEndHScratch](#) (FILEHANDLE *f, POSITION *position)
- void [SetScratch](#) (FILEHANDLE *f, POSITION *position)
- WORD [RevertScratch](#) (void)
- WORD [ResetScratch](#) (void)
- int [ReadFromScratch](#) (FILEHANDLE *fi, POSITION *pos, UBYTE *buffer, POSITION *length)
- int [AddToScratch](#) (FILEHANDLE *fi, POSITION *pos, UBYTE *buffer, POSITION *length, int withflush)
- int [CoSave](#) (UBYTE *inp)
- int [CoLoad](#) (UBYTE *inp)
- WORD [DeleteStore](#) (WORD par)
- WORD [PutInStore](#) (INDEXENTRY *ind, WORD num)
- WORD [GetTerm](#) (PHEAD WORD *term)
- WORD [GetOneTerm](#) (PHEAD WORD *term, FILEHANDLE *fi, POSITION *pos, int par)
- WORD [GetMoreTerms](#) (WORD *term)
- WORD [GetMoreFromMem](#) (WORD *term, WORD **tpoin)
- WORD [GetFromStore](#) (WORD *to, POSITION *position, RENUMBER renumber, WORD *InCompState, WORD nexpr)
- void [DetVars](#) (WORD *term, WORD par)
- WORD [ToStorage](#) (EXPRESSIONS e, POSITION *length)
- INDEXENTRY * [NextFileIndex](#) (POSITION *indexpos)
- WORD [SetFileIndex](#) (void)
- WORD [VarStore](#) (UBYTE *s, WORD n, WORD name, WORD namesize)
- WORD [TermRenumber](#) (WORD *term, RENUMBER renumber, WORD nexpr)
- WORD [FindrNumber](#) (WORD n, VARRENUM *v)
- INDEXENTRY * [FindInIndex](#) (WORD expr, FILEDATA *f, WORD par, WORD mode)
- RENUMBER [GetTable](#) (WORD expr, POSITION *position, WORD mode)
- int [CopyExpression](#) (FILEHANDLE *from, FILEHANDLE *to)
- WORD [WriteStoreHeader](#) (WORD handle)
- WORD [ReadSaveHeader](#) (void)

- WORD [ReadSaveIndex](#) ([FILEINDEX](#) *fileind)
- WORD [ReadSaveVariables](#) (UBYTE *buffer, UBYTE *top, LONG *size, LONG *outsize, [INDEXENTRY](#) *ind, LONG *stage)
- UBYTE * [ReadSaveTerm32](#) (UBYTE *bin, UBYTE *binend, UBYTE **bout, UBYTE *boutend, UBYTE *top, int terminbuf)
- WORD [ReadSaveExpression](#) (UBYTE *buffer, UBYTE *top, LONG *size, LONG *outsize)

4.134.1 Detailed Description

Contains all functions that deal with store-files and the system independent save-files.

Definition in file [store.c](#).

4.134.2 Macro Definition Documentation

4.134.2.1 SAVEREVISION

```
#define SAVEREVISION 0x02
```

Definition at line [3961](#) of file [store.c](#).

4.134.3 Function Documentation

4.134.3.1 OpenTemp()

```
WORD OpenTemp (  
    void )
```

Definition at line [48](#) of file [store.c](#).

4.134.3.2 SeekScratch()

```
void SeekScratch (  
    FILEHANDLE * fi,  
    POSITION * pos )
```

Definition at line [64](#) of file [store.c](#).

4.134.3.3 SetEndScratch()

```
void SetEndScratch (  
    FILEHANDLE * f,  
    POSITION * position )
```

Definition at line [75](#) of file [store.c](#).

4.134.3.4 SetEndHScratch()

```
void SetEndHScratch (
    FILEHANDLE * f,
    POSITION * position )
```

Definition at line 89 of file [store.c](#).

4.134.3.5 SetScratch()

```
void SetScratch (
    FILEHANDLE * f,
    POSITION * position )
```

Definition at line 115 of file [store.c](#).

4.134.3.6 RevertScratch()

```
WORD RevertScratch (
    void )
```

Definition at line 190 of file [store.c](#).

4.134.3.7 ResetScratch()

```
WORD ResetScratch (
    void )
```

Definition at line 235 of file [store.c](#).

4.134.3.8 ReadFromScratch()

```
int ReadFromScratch (
    FILEHANDLE * fi,
    POSITION * pos,
    UBYTE * buffer,
    POSITION * length )
```

Definition at line 273 of file [store.c](#).

4.134.3.9 AddToScratch()

```
int AddToScratch (
    FILEHANDLE * fi,
    POSITION * pos,
    UBYTE * buffer,
    POSITION * length,
    int withflush )
```

Definition at line 300 of file [store.c](#).

4.134.3.10 CoSave()

```
int CoSave (
    UBYTE * inp )
```

Definition at line [363](#) of file [store.c](#).

4.134.3.11 CoLoad()

```
int CoLoad (
    UBYTE * inp )
```

Definition at line [573](#) of file [store.c](#).

4.134.3.12 DeleteStore()

```
WORD DeleteStore (
    WORD par )
```

Definition at line [773](#) of file [store.c](#).

4.134.3.13 PutInStore()

```
WORD PutInStore (
    INDEXENTRY * ind,
    WORD num )
```

Definition at line [874](#) of file [store.c](#).

4.134.3.14 GetTerm()

```
WORD GetTerm (
    PHEAD WORD * term )
```

Definition at line [993](#) of file [store.c](#).

4.134.3.15 GetOneTerm()

```
WORD GetOneTerm (
    PHEAD WORD * term,
    FILEHANDLE * fi,
    POSITION * pos,
    int par )
```

Definition at line [1268](#) of file [store.c](#).

4.134.3.16 GetMoreTerms()

```
WORD GetMoreTerms (
    WORD * term )
```

Definition at line 1426 of file [store.c](#).

4.134.3.17 GetMoreFromMem()

```
WORD GetMoreFromMem (
    WORD * term,
    WORD ** tpoin )
```

Definition at line 1522 of file [store.c](#).

4.134.3.18 GetFromStore()

```
WORD GetFromStore (
    WORD * to,
    POSITION * position,
    RENUMBER renumber,
    WORD * InCompState,
    WORD nexpr )
```

Definition at line 1629 of file [store.c](#).

4.134.3.19 DetVars()

```
void DetVars (
    WORD * term,
    WORD par )
```

Definition at line 1830 of file [store.c](#).

4.134.3.20 ToStorage()

```
WORD ToStorage (
    EXPRESSIONS e,
    POSITION * length )
```

Definition at line 2017 of file [store.c](#).

4.134.3.21 NextFileIndex()

```
INDEXENTRY * NextFileIndex (
    POSITION * indexpos )
```

Definition at line 2258 of file [store.c](#).

4.134.3.22 SetFileIndex()

```
WORD SetFileIndex (
    void )
```

Reads the next file index and puts it into AR.StoreData.Index. TODO

Returns

= 0 everything okay, != 0 an error occurred

Definition at line [2321](#) of file [store.c](#).

References [WriteStoreHeader\(\)](#).

Here is the call graph for this function:

**4.134.3.23 VarStore()**

```
WORD VarStore (
    UBYTE * s,
    WORD n,
    WORD name,
    WORD namesize )
```

Definition at line [2370](#) of file [store.c](#).

4.134.3.24 TermRenumber()

```
WORD TermRenumber (
    WORD * term,
    RENUMBER renumber,
    WORD nexpr )
```

```
!! WORD *memterm=term; static LONG ctrap=0; !!!
```

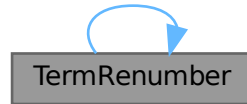
```
!! ctrap++; !!!
```

Definition at line [2428](#) of file [store.c](#).

References [ReNuMbEr::func](#), [ReNuMbEr::funnum](#), [ReNuMbEr::indi](#), [ReNuMbEr::indnum](#), [ReNuMbEr::symb](#), [ReNuMbEr::symnum](#), [TermRenumber\(\)](#), [ReNuMbEr::vecnum](#), and [ReNuMbEr::vect](#).

Referenced by [TermRenumber\(\)](#).

Here is the call graph for this function:



4.134.3.25 FindrNumber()

```
WORD FindrNumber (
    WORD n,
    VARRENUM * v )
```

Definition at line [2588](#) of file [store.c](#).

4.134.3.26 FindInIndex()

```
INDEXENTRY * FindInIndex (
    WORD expr,
    FILEDATA * f,
    WORD par,
    WORD mode )
```

Definition at line [2665](#) of file [store.c](#).

4.134.3.27 GetTable()

```
RENUMBER GetTable (
    WORD expr,
    POSITION * position,
    WORD mode )
```

Definition at line [2826](#) of file [store.c](#).

4.134.3.28 CopyExpression()

```
int CopyExpression (
    FILEHANDLE * from,
    FILEHANDLE * to )
```

Definition at line [3308](#) of file [store.c](#).

4.134.3.29 WriteStoreHeader()

```
WORD WriteStoreHeader (
    WORD handle )
```

Writes header with information about system architecture and FORM revision to an open store file.

Called by [SetFileIndex\(\)](#).

Parameters

<i>handle</i>	specifies open file to which header will be written
---------------	---

Returns

= 0 everything okay, != 0 an error occurred

Definition at line 3972 of file [store.c](#).

References [STOREHEADER::endianness](#), [STOREHEADER::lenLONG](#), [STOREHEADER::lenPOINTER](#), [STOREHEADER::lenPOS](#), [STOREHEADER::lenWORD](#), [STOREHEADER::maxpower](#), [STOREHEADER::sFun](#), [STOREHEADER::sInd](#), [STOREHEADER::sSym](#), [STOREHEADER::sVec](#), and [STOREHEADER::wildoffset](#).

Referenced by [SetFileIndex\(\)](#).

4.134.3.30 ReadSaveHeader()

```
WORD ReadSaveHeader (
    void )
```

Reads the header in the save file and sets function pointers and flags according to the information found there. Must be called before any other ReadSave... function.

Currently works only for the exchange between 32bit and 64bit machines (WORD size must be 2 or 4 bytes)!

It is called by CoLoad().

Returns

= 0 everything okay, != 0 an error occurred

Definition at line 4065 of file [store.c](#).

4.134.3.31 ReadSaveIndex()

```
WORD ReadSaveIndex (
    FILEINDEX * fileind )
```

Reads a FILEINDEX from the open save file specified by AO.SaveData.Handle. Translations for adjusting endianness and data sizes are done if necessary.

Depends on the assumption that sizeof(FILEINDEX) is the same everywhere. If FILEINDEX or INDEXENTRY change, then this functions has to be adjusted.

Called by CoLoad() and FindInIndex().

Parameters

<i>fileind</i>	contains the read FILEINDEX after successful return. must point to allocated, big enough memory.
----------------	--

Returns

= 0 everything okay, != 0 an error occurred

Definition at line 4183 of file [store.c](#).

References [INFILEINDEX](#), [FiLeInDeX::next](#), [FiLeInDeX::number](#), and [FuNcTiOn::number](#).

4.134.3.32 ReadSaveVariables()

```
WORD ReadSaveVariables (
    UBYTE * buffer,
    UBYTE * top,
    LONG * size,
    LONG * outsize,
    INDEXENTRY * ind,
    LONG * stage )
```

Reads the variables from the open file specified by AO.SaveData.Handle. It reads the *size bytes and writes them to the *buffer. It is called by PutInStore().

If translation is necessary, the data might shrink or grow in size, then *size is adjusted so that the reading and writing fits into the memory from the buffer to the top. The actual number of read bytes is returned in *size, the number of written bytes is returned in *outsize.

If the *size is smaller than the actual size of the variables, this function will be called several times and needs to remember the current position in the variable structure. The parameter *stage* does this job. When [ReadSaveVariables\(\)](#) is called for the first time, this parameter should have the value -1.

The parameter *ind* is used to get the number of variables.

Parameters

<i>buffer</i>	read variables are written into this allocated memory
<i>top</i>	upper end of allocated memory
<i>size</i>	number of bytes to read. might return a smaller number of read bytes if translation was necessary
<i>outsize</i>	if translation has be done, outsize contains the number of written bytes
<i>ind</i>	pointer of INDEXENTRY for the current expression. read-only
<i>stage</i>	should be -1 for the first call, will be increased by ReadSaveVariables to memorize the position in the variable structure

Returns

= 0 everything okay, != 0 an error occurred

Definition at line 4369 of file [store.c](#).

References [InDeXeNtRy::nfunctions](#), [InDeXeNtRy::nindices](#), [InDeXeNtRy::nsymbols](#), [InDeXeNtRy::nvectors](#), and [FuNcTiOn::spec](#).

4.134.3.33 ReadSaveTerm32()

```

UBYTE * ReadSaveTerm32 (
    UBYTE * bin,
    UBYTE * binend,
    UBYTE ** bout,
    UBYTE * boutend,
    UBYTE * top,
    int terminbuf )

```

Reads a single term from the given buffer at *bin* and write the translated term back to this buffer at *bout*.

[ReadSaveTerm32\(\)](#) is currently the only instantiation of a ReadSaveTerm-function. It only deals with data that already has the correct endianness and that is resized to 32bit words but without being renumbered or translated in any other way. It uses the compress buffer AR.CompressBuffer.

The function is reentrant in order to cope with nested function arguments. It is called by [ReadSaveExpression\(\)](#) and itself.

The *return value* indicates the position in the input buffer up to which the data has already been successfully processed. The parameter *bout* returns the corresponding position in the output buffer.

Parameters

<i>bin</i>	start of the input buffer
<i>binend</i>	end of the input buffer
<i>bout</i>	as input points to the beginning of the output buffer, as output points behind the already translated data in the output buffer
<i>boutend</i>	end of already decompressed data in output buffer
<i>top</i>	end of output buffer
<i>terminbuf</i>	flag whether decompressed data is already in the output buffer. used in recursive calls

Returns

pointer to the next unprocessed data in the input buffer

Definition at line [4736](#) of file [store.c](#).

References [ReadSaveTerm32\(\)](#).

Referenced by [ReadSaveExpression\(\)](#), and [ReadSaveTerm32\(\)](#).

Here is the call graph for this function:



4.134.3.34 ReadSaveExpression()

```
WORD ReadSaveExpression (
    UBYTE * buffer,
    UBYTE * top,
    LONG * size,
    LONG * outsize )
```

Reads an expression from the open file specified by AO.SaveData.Handle. The endianness flip and a resizing without renumbering is done in this function. Thereafter the buffer consists of chunks with a uniform maximal word size (32bit at the moment). The actual renumbering is then done by calling the function [ReadSaveTerm32\(\)](#). The result is returned in *buffer*.

If the translation at some point doesn't fit into the buffer anymore, the function returns and must be called again. In any case *size* returns the number of successfully read bytes, *outsize* returns the number of successfully written bytes, and the file will be positioned at the next byte after the successfully read data.

It is called by PutInStore().

Parameters

<i>buffer</i>	output buffer, holds the (translated) expression
<i>top</i>	end of buffer
<i>size</i>	number of read bytes
<i>outsize</i>	number of written bytes

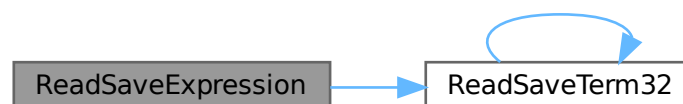
Returns

= 0 everything okay, != 0 an error occurred

Definition at line 5146 of file [store.c](#).

References [ReadSaveTerm32\(\)](#).

Here is the call graph for this function:



4.135 store.c

[Go to the documentation of this file.](#)

```
00001
00006 /* #[ License : */
```

```

00007 /*
00008 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00009 * When using this file you are requested to refer to the publication
00010 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00011 * This is considered a matter of courtesy as the development was paid
00012 * for by FOM the Dutch physics granting agency and we would like to
00013 * be able to track its scientific use to convince FOM of its value
00014 * for the community.
00015 *
00016 * This file is part of FORM.
00017 *
00018 * FORM is free software: you can redistribute it and/or modify it under the
00019 * terms of the GNU General Public License as published by the Free Software
00020 * Foundation, either version 3 of the License, or (at your option) any later
00021 * version.
00022 *
00023 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00024 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00025 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00026 * details.
00027 *
00028 * You should have received a copy of the GNU General Public License along
00029 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00030 */
00031 /* #] License : */
00032 /*
00033 #define HIDEDEBUG
00034     #[ Includes : store.c
00035 */
00036
00037 #include "form3.h"
00038
00039 /*
00040 #] Includes :
00041 #[ StoreExpressions :
00042     #[ OpenTemp :
00043
00044         Opens the scratch files for the input -> output operations.
00045
00046 */
00047
00048 WORD OpenTemp(void)
00049 {
00050     GETIDENTITY
00051     if ( AR.outfile->handle >= 0 ) {
00052         SeekFile(AR.outfile->handle,&(AR.outfile->filesize),SEEK_SET);
00053         AR.outfile->POposition = AR.outfile->filesize;
00054         AR.outfile->POfill = AR.outfile->PObuffer;
00055     }
00056     return(0);
00057 }
00058
00059 /*
00060     #] OpenTemp :
00061     #[ SeekScratch :
00062 */
00063
00064 void SeekScratch(FILEHANDLE *fi, POSITION *pos)
00065 {
00066     *pos = fi->POposition;
00067     ADDPOS(*pos, (TOLONG(fi->POfill)-TOLONG(fi->PObuffer)));
00068 }
00069
00070 /*
00071     #] SeekScratch :
00072     #[ SetEndScratch :
00073 */
00074
00075 void SetEndScratch(FILEHANDLE *f, POSITION *position)
00076 {
00077     if ( f->handle < 0 ) {
00078         SETBASEPOSITION(*position, (f->POfull-f->PObuffer)*sizeof(WORD));
00079     }
00080     else *position = f->filesize;
00081     SetScratch(f,position);
00082 }
00083
00084 /*
00085     #] SetEndScratch :
00086     #[ SetEndHScratch :
00087 */
00088
00089 void SetEndHScratch(FILEHANDLE *f, POSITION *position)
00090 {
00091     if ( f->handle < 0 ) {
00092         SETBASEPOSITION(*position, (f->POfull-f->PObuffer)*sizeof(WORD));
00093         f->POfill = f->POfull;

```

```

00094     }
00095     else {
00096 #ifdef HIDEDEBUG
00097     POSITION possize;
00098     PUTZERO(possize);
00099     SeekFile(f->handle,&possize,SEEK_END);
00100     MesPrint("SetEndHScratch: filesize(th) = %12p, filesize(ex) = %12p",&(f->filesize),
00101             &(possize));
00102 #endif
00103     *position = f->filesize;
00104     f->POposition = f->filesize;
00105     f->POfill = f->POfull = f->PObuffer;
00106     }
00107 /* SetScratch(f,position); */
00108 }
00109
00110 /*
00111     #] SetEndHScratch :
00112     #[ SetScratch :
00113 */
00114
00115 void SetScratch(FILEHANDLE *f, POSITION *position)
00116 {
00117     GETIDENTITY
00118     POSITION possize;
00119     LONG size, *whichInInBuf;
00120     if ( f == AR.hidefile ) whichInInBuf = &(AR.InHiBuf);
00121     else                    whichInInBuf = &(AR.InInBuf);
00122 #ifdef HIDEDEBUG
00123     if ( f == AR.hidefile ) MesPrint("In the hide file");
00124     else MesPrint("In the input file");
00125     MesPrint("SetScratch to position %15p",position);
00126     MesPrint("POposition = %15p, full = %1, fill = %1"
00127             ,&(f->POposition),&(f->POfull-&(f->PObuffer)*sizeof(WORD)
00128             ,&(f->POfill-&(f->PObuffer)*sizeof(WORD));
00129 #endif
00130     if ( ISLESSPOS(*position,f->POposition) ||
00131         ISGEPOSINC(*position,f->POposition,&(f->POfull-&(f->PObuffer)*sizeof(WORD)) ) ) {
00132         if ( f->handle < 0 ) {
00133             if ( ISEQUALPOSINC(*position,f->POposition,
00134                               &(f->POfull-&(f->PObuffer)*sizeof(WORD)) ) goto endpos;
00135             MesPrint("Illegal position in SetScratch");
00136             Terminate(-1);
00137         }
00138         possize = *position;
00139         LOCK(AS.inputslock);
00140         SeekFile(f->handle,&possize,SEEK_SET);
00141         if ( ISNOTEQUALPOS(possize,*position) ) {
00142             UNLOCK(AS.inputslock);
00143             MesPrint("Cannot position file in SetScratch");
00144             Terminate(-1);
00145         }
00146 #ifdef HIDEDEBUG
00147         MesPrint("SetScratch1(%w): position = %12p, size = %1, address =
00148 %x",position,f->POsize,f->PObuffer);
00149 #endif
00150         if ( ( size = ReadFile(f->handle,(UBYTE *) (f->PObuffer),f->POsize) ) < 0
00151             || ( size & 1 ) != 0 ) {
00152             UNLOCK(AS.inputslock);
00153             MesPrint("Read error in SetScratch");
00154             Terminate(-1);
00155         }
00156         UNLOCK(AS.inputslock);
00157         if ( size == 0 ) {
00158             f->PObuffer[0] = 0;
00159         }
00160         f->POfill = f->PObuffer;
00161         f->POposition = *position;
00162 #ifdef WORD2
00163         *whichInInBuf = size >> 1;
00164 #else
00165         *whichInInBuf = size / TABLESIZE(WORD,UBYTE);
00166 #endif
00167         f->POfull = f->PObuffer + *whichInInBuf;
00168 #ifdef HIDEDEBUG
00169         MesPrint("SetScratch2: size = %1, InInBuf = %1, fill = %1, full = %1"
00170             ,size,*whichInInBuf,&(f->POfill-&(f->PObuffer)*sizeof(WORD)
00171             ,&(f->POfull-&(f->PObuffer)*sizeof(WORD));
00172 #endif
00173     }
00174     else {
00175         endpos:
00176         DIFPOS(possize,*position,f->POposition);
00177         f->POfill = (WORD *) (BASEPOSITION(possize)+(UBYTE *) (f->PObuffer));
00178         *whichInInBuf = f->POfull-&(f->POfill);
00179     }
00180 }

```

```

00180
00181 /*
00182     #] SetScratch :
00183     #[ RevertScratch :
00184
00185     Reverts the input/output directions. This way input comes
00186     always from AR.infile
00187
00188 */
00189
00190 WORD RevertScratch(void)
00191 {
00192     GETIDENTITY
00193     FILEHANDLE *f;
00194     if ( AR.infile->handle >= 0 && AR.infile->handle != AR.outfile->handle ) {
00195         CloseFile(AR.infile->handle);
00196         AR.infile->handle = -1;
00197         remove(AR.infile->name);
00198     }
00199     f = AR.infile; AR.infile = AR.outfile; AR.outfile = f;
00200     AR.infile->POfull = AR.infile->POfill;
00201     AR.infile->POfill = AR.infile->PObuffer;
00202     if ( AR.infile->handle >= 0 ) {
00203         POSITION scrpos;
00204         PUTZERO(scrpos);
00205         SeekFile(AR.infile->handle,&scrpos,SEEK_SET);
00206         if ( ISNOTZEROPOS(scrpos) ) {
00207             return(MesPrint("Error with scratch output."));
00208         }
00209         if ( ( AR.InInBuf = ReadFile(AR.infile->handle,(UBYTE *) (AR.infile->PObuffer)
00210             ,AR.infile->POsize) ) < 0 || AR.InInBuf & 1 ) {
00211             return(MesPrint("Error while reading from scratch file"));
00212         }
00213         else {
00214             AR.InInBuf /= TABLESIZE(WORD,UBYTE);
00215         }
00216         AR.infile->POfull = AR.infile->PObuffer + AR.InInBuf;
00217     }
00218     PUTZERO(AR.infile->POposition);
00219     AR.outfile->POfill = AR.outfile->POfull = AR.outfile->PObuffer;
00220     PUTZERO(AR.outfile->POposition);
00221     PUTZERO(AR.outfile->filesize);
00222     return(0);
00223 }
00224
00225 /*
00226     #] RevertScratch :
00227     #[ ResetScratch :
00228
00229     Resets the output scratch file to its beginning in such a way
00230     that the write routines can read it. The output buffers are
00231     left untouched as they may still be needed for extra declarations.
00232
00233 */
00234
00235 WORD ResetScratch(void)
00236 {
00237     GETIDENTITY
00238     FILEHANDLE *f;
00239     if ( AR.infile->handle >= 0 ) {
00240         CloseFile(AR.infile->handle); AR.infile->handle = -1;
00241         remove(AR.infile->name);
00242         PUTZERO(AR.infile->POposition);
00243         AR.infile->POfill = AR.infile->POfull = AR.infile->PObuffer;
00244     }
00245     if ( AR.outfile->handle >= 0 ) {
00246         POSITION scrpos;
00247         PUTZERO(scrpos);
00248         SeekFile(AR.outfile->handle,&scrpos,SEEK_SET);
00249         if ( ISNOTZEROPOS(scrpos) ) {
00250             return(MesPrint("Error with scratch output."));
00251         }
00252         if ( ( AR.InInBuf = ReadFile(AR.outfile->handle,(UBYTE *) (AR.outfile->PObuffer)
00253             ,AR.outfile->POsize) ) < 0 || AR.InInBuf & 1 ) {
00254             return(MesPrint("Error while reading from scratch file"));
00255         }
00256         else AR.InInBuf /= TABLESIZE(WORD,UBYTE);
00257         AR.outfile->POfull = AR.outfile->PObuffer + AR.InInBuf;
00258     }
00259     else AR.outfile->POfull = AR.outfile->POfill;
00260     AR.outfile->POfill = AR.outfile->PObuffer;
00261     PUTZERO(AR.outfile->POposition);
00262     f = AR.outfile; AR.outfile = AR.infile; AR.infile = f;
00263     return(0);
00264 }
00265
00266 /*

```

```

00267         #] ResetScratch :
00268         #[ ReadFromScratch :
00269
00270         Routine is used to copy files from scratch to hide.
00271     */
00272
00273 int ReadFromScratch(FILEHANDLE *fi, POSITION *pos, UBYTE *buffer, POSITION *length)
00274 {
00275     GETIDENTITY
00276     LONG l = BASEPOSITION(*length);
00277     if ( fi->handle < 0 ) {
00278         memcpy(buffer,fi->POfill,l);
00279     }
00280     else {
00281         SeekFile(fi->handle,pos,SEEK_SET);
00282         if ( ReadFile(fi->handle,buffer,l) != l ) {
00283             if ( fi == AR.hidefile )
00284                 MesPrint("Error reading from hide file.");
00285             else
00286                 MesPrint("Error reading from scratch file.");
00287             return(-1);
00288         }
00289     }
00290     return(0);
00291 }
00292
00293 /*
00294     #] ReadFromScratch :
00295     #[ AddToScratch :
00296
00297     Routine is used to copy files from scratch to hide.
00298 */
00299
00300 int AddToScratch(FILEHANDLE *fi, POSITION *pos, UBYTE *buffer, POSITION *length,
00301                 int withflush)
00302 {
00303     GETIDENTITY
00304     LONG l = BASEPOSITION(*length), avail;
00305     DUMMYUSE(pos)
00306     fi->POfill = fi->POfull;
00307     while ( fi->POfill+l/sizeof(WORD) > fi->POstop ) {
00308         avail = (fi->POstop-fi->POfill)*sizeof(WORD);
00309         if ( avail > 0 ) {
00310             memcpy(fi->POfill,buffer,avail);
00311             l -= avail; buffer += avail;
00312         }
00313         if ( fi->handle < 0 ) {
00314             if ( ( fi->handle = (WORD)CreateFile(fi->name) ) < 0 ) {
00315                 if ( fi == AR.hidefile )
00316                     MesPrint("Cannot create hide file %s",fi->name);
00317                 else
00318                     MesPrint("Cannot create scratch file %s",fi->name);
00319                 return(-1);
00320             }
00321             PUTZERO(fi->POposition);
00322         }
00323         SeekFile(fi->handle,&(fi->POposition),SEEK_SET);
00324         if ( WriteFile(fi->handle,(UBYTE *)fi->PObuffer,fi->POsize) != fi->POsize )
00325             goto writeerror;
00326         ADDPOS(fi->POposition,fi->POsize);
00327         fi->POfill = fi->POfull = fi->PObuffer;
00328     }
00329     if ( l > 0 ) {
00330         memcpy(fi->POfill,buffer,l);
00331         fi->POfill += l/sizeof(WORD);
00332         fi->POfull = fi->POfill;
00333     }
00334     if ( withflush && fi->handle >= 0 && fi->POfill > fi->PObuffer ) { /* flush */
00335         l = (LONG)fi->POfill - (LONG)fi->PObuffer;
00336         SeekFile(fi->handle,&(fi->POposition),SEEK_SET);
00337         if ( WriteFile(fi->handle,(UBYTE *)fi->PObuffer,l) != l ) goto writeerror;
00338         ADDPOS(fi->POposition,fi->POsize);
00339         fi->POfill = fi->POfull = fi->PObuffer;
00340     }
00341     if ( withflush && fi->handle >= 0 )
00342         SETBASEPOSITION(fi->filesize,TellFile(fi->handle));
00343     return(0);
00344 writeerror:
00345     if ( fi == AR.hidefile )
00346         MesPrint("Error writing to hide file. Disk full?");
00347     else
00348         MesPrint("Error writing to scratch file. Disk full?");
00349     return(-1);
00350 }
00351
00352 /*
00353     #] AddToScratch :

```

```

00354      #! CoSave :
00355
00356      The syntax of the save statement is:
00357
00358      save filename
00359      save filename expr1 expr2
00360
00361  */
00362
00363  int CoSave(UBYTE *inp)
00364  {
00365      GETIDENTITY
00366      UBYTE *p, c;
00367      WORD n = 0, i;
00368      WORD error = 0, type, number;
00369      WORD exprInStorageFlag = 0;
00370      LONG RetCode = 0, wSize;
00371      EXPRESSIONS e;
00372      INDEXENTRY *ind;
00373      INDEXENTRY *indold;
00374      WORD TMproto[SUBEXPSIZE];
00375      POSITION scrpos, scrpos1, filesize;
00376      int ii, j = sizeof(FILEINDEX)/(sizeof(LONG));
00377      LONG *lo;
00378      while ( *inp == ',' ) inp++;
00379      p = inp;
00380
00381  #ifdef WITHMPI
00382      if ( PF.me != MASTER ) return(0);
00383  #endif
00384
00385      if ( !*p ) return(MesPrint("No filename in save statement"));
00386      if ( FG.cTable[*p] > 1 && ( *p != '.' ) && ( *p != SEPARATOR ) && ( *p != ALTSEPARATOR ) )
00387          return(MesPrint("Illegal filename"));
00388      while ( ++p && *p != ',' ) {}
00389      c = *p;
00390      *p = 0;
00391      if ( !AP.preError ) {
00392          if ( ( RetCode = CreateFile((char *)inp) ) < 0 ) {
00393              return(MesPrint("Cannot open file %s",inp));
00394          }
00395      }
00396      AO.SaveData.Handle = (WORD)RetCode;
00397      PUTZERO(filesize);
00398
00399      e = Expressions;
00400      n = NumExpressions;
00401      if ( c ) { /* There follows a list of expressions */
00402          *p++ = c;
00403          inp = p;
00404          i = (WORD) (INFILEINDEX);
00405          if ( WriteStoreHeader(AO.SaveData.Handle) ) return(MesPrint("Error writing storage file
header"));
00406          /* PUTZERO(AO.SaveData.Index.number); */
00407          /* PUTZERO(AO.SaveData.Index.next); */
00408          lo = (LONG *)(&AO.SaveData.Index);
00409          for ( ii = 0; ii < j; ii++ ) *lo++ = 0;
00410          SETBASEPOSITION(AO.SaveData.Position, (LONG)sizeof(STOREHEADER));
00411          ind = AO.SaveData.Index.expression;
00412          if ( !AP.preError && WriteFile(AO.SaveData.Handle, (UBYTE *)(&(AO.SaveData.Index))
, (LONG)sizeof(struct FileInDeX))!= (LONG)sizeof(struct FileInDeX) ) goto SavWrt;
00413          SeekFile(AO.SaveData.Handle, &(filesize), SEEK_END);
00414          /* ADDPOS(filesize, sizeof(struct FileInDeX)); */
00415
00416          do { /* Scan the list */
00417              if ( !FG.cTable[*p] || *p == '[' ) {
00418                  p = SkipAName(p);
00419                  if ( p == 0 ) return(-1);
00420              }
00421              c = *p; *p = 0;
00422              if ( GetVar(inp, &type, &number, CEXPRESSION, NOAUTO) != NAMENOTFOUND ) {
00423                  if ( e[number].status == STOREDEXPRESSION ) {
00424                      if ( StrLen(AC.exprnames->namebuffer+e[number].name) > MAXENAME ) {
00425                          char msg[100];
00426                          sprintf(msg, sizeof(msg), "saved expr name over %d char: %s", MAXENAME,
AC.exprnames->namebuffer+e[number].name);
00427                          Warning(msg);
00428                      }
00429                  }
00430              }
00431              /* Here we have to locate the stored expression, copy its index entry
00432              possibly after making a new fileindex and then copy the whole
00433              expression.
00434              */
00435              if ( AP.preError ) goto NextExpr;
00436              TMproto[0] = EXPRESSION;
00437              TMproto[1] = SUBEXPSIZE;
00438              TMproto[2] = number;

```



```

00439         TMproto[3] = 1;
00440         { int ie; for ( ie = 4; ie < SUBEXPSIZE; ie++ ) TMproto[ie] = 0; }
00441         AT.TMaddr = TMproto;
00442         if ( ( indold = FindInIndex(number,&AR.StoreData,0,0) ) != 0 ) {
00443             if ( i <= 0 ) {
00444                 /*
00445                     AO.SaveData.Index.next = filesize;
00446                 */
00447                 SeekFile(AO.SaveData.Handle,&(AO.SaveData.Index.next),SEEK_END);
00448                 scrpos = AO.SaveData.Position;
00449                 SeekFile(AO.SaveData.Handle,&scrpos,SEEK_SET);
00450                 if ( ISNOTEQUALPOS(scrpos,AO.SaveData.Position) ) goto SavWrt;
00451                 if ( WriteFile(AO.SaveData.Handle,(UBYTE *)(&(AO.SaveData.Index))
00452                     ,(LONG)sizeof(struct FileInDeX)) != (LONG)sizeof(struct FileInDeX)
00453                 )
00454                     goto SavWrt;
00455                 i = (WORD)(INFILINDEX);
00456                 AO.SaveData.Position = AO.SaveData.Index.next;
00457                 lo = (LONG *)(&AO.SaveData.Index);
00458                 for ( ii = 0; ii < j; ii++ ) *lo++ = 0;
00459                 ind = AO.SaveData.Index.expression;
00460                 scrpos = AO.SaveData.Position;
00461                 SeekFile(AO.SaveData.Handle,&scrpos,SEEK_SET);
00462                 if ( ISNOTEQUALPOS(scrpos,AO.SaveData.Position) ) goto SavWrt;
00463                 if ( WriteFile(AO.SaveData.Handle,(UBYTE *)(&(AO.SaveData.Index))
00464                     ,(LONG)sizeof(struct FileInDeX)) != (LONG)sizeof(struct FileInDeX) )
00465                     goto SavWrt;
00466                 ADDPOS(filesize,sizeof(struct FileInDeX));
00467                 *ind = *indold;
00468             /*
00469                 ind->variables = SeekFile(AO.SaveData.Handle,&(AM.zeropos),SEEK_END);
00470             */
00471             ind->variables = filesize;
00472             ind->position = ind->variables;
00473             ADDPOS(ind->position,DIFBASE(indold->position,indold->variables));
00474             SeekFile(AR.StoreData.Handle,&(indold->variables),SEEK_SET);
00475             wSize = TOLONG(AT.WorkTop) - TOLONG(AT.WorkPointer);
00476             scrpos = ind->length;
00477             ADDPOS(scrpos,DIFBASE(ind->position,ind->variables));
00478             ADD2POS(filesize,scrpos);
00479             SETBASEPOSITION(scrpos1,wSize);
00480             do {
00481                 if ( ISLESSPOS(scrpos,scrpos1) ) wSize = BASEPOSITION(scrpos);
00482                 if ( ReadFile(AR.StoreData.Handle,(UBYTE *)AT.WorkPointer,wSize)
00483                     != wSize ) {
00484                     MesPrint("ReadError");
00485                     error = -1;
00486                     goto EndSave;
00487                 }
00488                 if ( WriteFile(AO.SaveData.Handle,(UBYTE *)AT.WorkPointer,wSize)
00489                     != wSize ) goto SavWrt;
00490                 ADDPOS(scrpos,-wSize);
00491             } while ( ISPOSPOS(scrpos) );
00492             ADDPOS(AO.SaveData.Index.number,1);
00493             ind++;
00494         }
00495         else error = -1;
00496         i--;
00497     }
00498     else {
00499         MesPrint("%s is not a stored expression",inp);
00500         error = -1;
00501     }
00502 NextExpr;;
00503 }
00504 else {
00505     MesPrint("%s is not an expression",inp);
00506     error = -1;
00507 }
00508 *p = c;
00509 if ( c != ',' && c ) {
00510     MesComp("Illegal character",inp,p);
00511     error = -1;
00512     goto EndSave;
00513 }
00514 if ( c ) c = ++p;
00515 inp = p;
00516 } while ( c );
00517 if ( !AP.preError ) {
00518     scrpos = AO.SaveData.Position;
00519     SeekFile(AO.SaveData.Handle,&scrpos,SEEK_SET);
00520     if ( ISNOTEQUALPOS(scrpos,AO.SaveData.Position) ) goto SavWrt;
00521 }
00522 if ( !AP.preError &&
00523     WriteFile(AO.SaveData.Handle,(UBYTE *)(&(AO.SaveData.Index))
00524         ,(LONG)sizeof(struct FileInDeX)) != (LONG)sizeof(struct FileInDeX) ) goto SavWrt;

```

```

00525     }
00526     else if ( !AP.preError ) { /* All stored expressions should be saved. Easy */
00527         /* Make sure there is at least one stored expression: */
00528         if ( n > 0 ) { do {
00529             if ( StrLen(AC.exprnames->namebuffer+e->name) > MAXENAME ) {
00530                 char msg[100];
00531                 snprintf(msg, sizeof(msg), "saved expr name over %d char: %s", MAXENAME,
AC.exprnames->namebuffer+e->name);
00532                 Warning(msg);
00533             }
00534             if ( e->status == STOREDEXPRESSION ) exprInStorageFlag = 1;
00535             e++;
00536         } while ( --n > 0 ); }
00537         if ( exprInStorageFlag ) {
00538             wSize = TOLONG(AT.WorkTop) - TOLONG(AT.WorkPointer);
00539             PUTZERO(scrpos);
00540             SeekFile(AR.StoreData.Handle,&scrpos,SEEK_SET); /* Start at the beginning */
00541             scrpos = AR.StoreData.Fill; /* Number of bytes to be copied */
00542             SETBASEPOSITION(scrpos1,wSize);
00543             do {
00544                 if ( ISLESSPOS(scrpos,scrpos1) ) wSize = BASEPOSITION(scrpos);
00545                 if ( ReadFile(AR.StoreData.Handle,(UBYTE *)AT.WorkPointer,wSize) != wSize ) {
00546                     MesPrint("ReadError");
00547                     error = -1;
00548                     goto EndSave;
00549                 }
00550                 if ( WriteFile(AO.SaveData.Handle,(UBYTE *)AT.WorkPointer,wSize) != wSize )
00551                     goto SavWrt;
00552                 ADDPOS(scrpos,-wSize);
00553             } while ( ISPOSPOS(scrpos) );
00554         }
00555     }
00556 EndSave:
00557     if ( !AP.preError ) {
00558         CloseFile(AO.SaveData.Handle);
00559         AO.SaveData.Handle = -1;
00560     }
00561     return(error);
00562 SavWrt:
00563     MesPrint("WriteError");
00564     error = -1;
00565     goto EndSave;
00566 }
00567 /*
00568     #] CoSave :
00569     #[ CoLoad :
00570 */
00571 */
00572
00573 int CoLoad(UBYTE *inp)
00574 {
00575     GETIDENTITY
00576     INDEXENTRY *ind;
00577     LONG RetCode;
00578     UBYTE *p, c;
00579     WORD num, i, error = 0;
00580     WORD type, number, silentload = 0;
00581     WORD TMproto[SUBEXPSIZE];
00582     POSITION scrpos,firstposition;
00583     while ( *inp == ',' ) inp++;
00584     p = inp;
00585     if ( ( *p == ',' && p[1] == '-' ) || *p == '-' ) {
00586         if ( *p == ',' ) p++;
00587         p++;
00588         if ( *p == 's' || *p == 'S' ) {
00589             silentload = 1;
00590             while ( *p && ( *p != ',' && *p != '-' && *p != '+'
&& *p != SEPARATOR && *p != ALTSEPARATOR && *p != '.' ) ) p++;
00592         }
00593         else if ( *p != ',' ) {
00594             return(MesPrint("Illegal option in Load statement"));
00595         }
00596         while ( *p == ',' ) p++;
00597     }
00598     inp = p;
00599     if ( !*p ) return(MesPrint("No filename in load statement"));
00600     if ( FG.CTable[*p] > 1 && ( *p != '.' ) && ( *p != SEPARATOR ) && ( *p != ALTSEPARATOR ) )
00601         return(MesPrint("Illegal filename"));
00602     while ( ++p && *p != ',' ) {}
00603     c = *p;
00604     *p = 0;
00605     if ( ( RetCode = OpenFile((char *)inp) ) < 0 ) {
00606         return(MesPrint("Cannot open file %s",inp));
00607     }
00608
00609     if ( SetFileIndex() ) {
00610         MesCall("CoLoad");

```

```

00611     SETERROR(-1)
00612 }
00613
00614     AO.SaveData.Handle = (WORD) (RetCode);
00615
00616 #ifdef SYSDEPENDENTSAVE
00617     if ( ReadFile(AO.SaveData.Handle, (UBYTE *) (&(AO.SaveData.Index)),
00618         (LONG)sizeof(struct FileInDeX) != (LONG)sizeof(struct FileInDeX) ) goto LoadRead;
00619 #else
00620     if ( ReadSaveHeader() ) goto LoadRead;
00621     TELLFILE(AO.SaveData.Handle,&firstposition);
00622     if ( ReadSaveIndex(&AO.SaveData.Index) ) goto LoadRead;
00623 #endif
00624     if ( c ) { /* There follows a list of expressions */
00625         *p++ = c;
00626         inp = p;
00627
00628         do { /* Scan the list */
00629             if ( !FG.cTable[*p] || *p == '[' ) {
00630                 p = SkipAName(p);
00631                 if ( p == 0 ) return(-1);
00632             }
00633             c = *p; *p = 0;
00634             if ( GetVar(inp,&type,&number,ALLVARIABLES,NOAUTO) != NAMENOTFOUND ) {
00635                 MesPrint("Conflicting name: %s",inp);
00636                 error = -1;
00637             }
00638             else {
00639                 if ( ( num = EntVar(CEXPRESSION,inp,STOREDEXPRESSION,0,0,0) ) >= 0 ) {
00640                     TMproto[0] = EXPRESSION;
00641                     TMproto[1] = SUBEXPSIZE;
00642                     TMproto[2] = num;
00643                     TMproto[3] = 1;
00644                     { int ie; for ( ie = 4; ie < SUBEXPSIZE; ie++ ) TMproto[ie] = 0; }
00645                     AT.TMaddr = TMproto;
00646                     SeekFile(AO.SaveData.Handle,&firstposition,SEEK_SET);
00647                     AO.SaveData.Position = firstposition;
00648                     if ( ReadSaveIndex(&AO.SaveData.Index) ) goto LoadRead;
00649                     if ( ( ind = FindInIndex(num,&AO.SaveData,1,0) ) != 0 ) {
00650                         if ( !error ) {
00651                             if ( PutInStore(ind,num) ) error = -1;
00652                             else if ( !AM.silent && silentload == 0 )
00653                                 MesPrint(" %s loaded",ind->name);
00654                         }
00655                         /*
00656                         !!! Added 1-feb-1998
00657                         */
00658                         Expressions[num].counter = -1;
00659                     }
00660                     else {
00661                         MesPrint(" %s not found",inp);
00662                         error = -1;
00663                     }
00664                 }
00665                 else error = -1;
00666             }
00667             *p = c;
00668             if ( c != ',' && c ) {
00669                 MesComp("Illegal character",inp,p);
00670                 error = -1;
00671                 goto EndLoad;
00672             }
00673             if ( c ) c = *++p;
00674             inp = p;
00675         } while ( c );
00676         scrpos = AR.StoreData.Position;
00677         SeekFile(AR.StoreData.Handle,&scrpos,SEEK_SET);
00678         if ( ISNOTEQUALPOS(scrpos,AR.StoreData.Position) ) goto LoadWrt;
00679         if ( WriteFile(AR.StoreData.Handle, (UBYTE *) (&(AR.StoreData.Index))
00680             , (LONG)sizeof(struct FileInDeX) != (LONG)sizeof(struct FileInDeX) ) goto LoadWrt;
00681     }
00682     else { /* All saved expressions should be stored. Easy */
00683         i = (WORD)BASEPOSITION(AO.SaveData.Index.number);
00684         ind = AO.SaveData.Index.expression;
00685 #ifdef SYSDEPENDENTSAVE
00686         if ( i > 0 ) { do {
00687             if ( GetVar((UBYTE *) (ind->name),&type,&number,ALLVARIABLES,NOAUTO) != NAMENOTFOUND ) {
00688                 MesPrint("Conflicting name: %s",ind->name);
00689                 error = -1;
00690             }
00691             else {
00692                 if ( ( num = EntVar(CEXPRESSION, (UBYTE *) (ind->name),STOREDEXPRESSION,0,0,0) ) >= 0 )
00693                     {
00694                         if ( !error ) {
00695                             if ( PutInStore(ind,num) ) error = -1;
00696                             else if ( !AM.silent && silentload == 0 )
00697                                 MesPrint(" %s loaded",ind->name);

```

```

00697         }
00698     }
00699     else error = -1;
00700 }
00701 i--;
00702 if ( i == 0 && ISNOTZEROPOS(AO.SaveData.Index.next) ) {
00703     SeekFile(AO.SaveData.Handle, &(AO.SaveData.Index.next), SEEK_SET);
00704     if ( ReadFile(AO.SaveData.Handle, (UBYTE *) (&(AO.SaveData.Index)),
00705         (LONG)sizeof(struct FileInDeX) ) != (LONG)sizeof(struct FileInDeX) ) goto LoadRead;
00706     i = (WORD)BASEPOSITION(AO.SaveData.Index.number);
00707     ind = AO.SaveData.Index.expression;
00708 }
00709 else ind++;
00710 } while ( i > 0 ); }
00711 #else
00712 if ( i > 0 ) {
00713     do {
00714         if ( GetVar((UBYTE *) (ind->name), &type, &number, ALLVARIABLES, NOAUTO) != NAMENOTFOUND )
00715         {
00716             MesPrint("Conflicting name: %s", ind->name);
00717             error = -1;
00718         }
00719         else {
00720             if ( ( num = EntVar(CEXPRESSION, (UBYTE *) (ind->name), STOREDEXPRESSION, 0, 0, 0) ) >=
00721 0 ) {
00722                 if ( !error ) {
00723                     if ( PutInStore(ind, num) ) error = -1;
00724                     else if ( !AM.silent && silentload == 0 )
00725                         MesPrint(" %s loaded", ind->name);
00726                 }
00727                 else error = -1;
00728             }
00729             i--;
00730             if ( i == 0 && (ISNOTZEROPOS(AO.SaveData.Index.next) || AO.bufferedInd) ) {
00731                 SeekFile(AO.SaveData.Handle, &(AO.SaveData.Index.next), SEEK_SET);
00732                 if ( ReadSaveIndex(&AO.SaveData.Index) ) goto LoadRead;
00733                 i = (WORD)BASEPOSITION(AO.SaveData.Index.number);
00734                 ind = AO.SaveData.Index.expression;
00735             }
00736             else ind++;
00737         } while ( i > 0 );
00738     }
00739 }
00740 #endif
00741 EndLoad:
00742 #ifndef SYSDEPENDENTSAVE
00743     if ( AO.powerFlag ) {
00744         MesPrint("WARNING: min-/maxpower had to be adjusted!");
00745     }
00746     if ( AO.resizeFlag ) {
00747         MesPrint("ERROR: could not downsize data!");
00748         return ( -2 );
00749     }
00750 #endif
00751 CloseFile(AO.SaveData.Handle);
00752 AO.SaveData.Handle = -1;
00753 SeekFile(AR.StoreData.Handle, &(AC.StoreFileSize), SEEK_END);
00754 return(error);
00755 LoadWrt:
00756 MesPrint("WriteError");
00757 error = -1;
00758 goto EndLoad;
00759 LoadRead:
00760 MesPrint("ReadError");
00761 error = -1;
00762 goto EndLoad;
00763 }
00764 /*
00765     #] CoLoad :
00766     #[ DeleteStore :
00767
00768     Routine deletes the contents of the entire storage file.
00769     We close the file and recreate it.
00770     If par > 0 we have to remove the expressions from the namelists.
00771 */
00772 WORD DeleteStore(WORD par)
00773 {
00774     GETIDENTITY
00775     char *s;
00776     WORD j, n = 0;
00777     EXPRESSIONS e_in, e_out;
00778     WORD DidClean = 0;
00779     if ( AR.StoreData.Handle >= 0 ) {
00780         if ( par > 0 ) {

```

```

00782         n = NumExpressions;
00783         j = 0;
00784         e_in = e_out = Expressions;
00785         if ( n > 0 ) { do {
00786             if ( e_in->status == STOREDEXPRESSION ) {
00787                 NAMENODE *node = GetNode(AC.exprnames,
00788                     AC.exprnames->namebuffer+e_in->name);
00789                 node->type = CDELETE;
00790                 DidClean = 1;
00791             }
00792             else {
00793                 if ( e_out != e_in ) {
00794                     NAMENODE *node;
00795                     node = GetNode(AC.exprnames,
00796                         AC.exprnames->namebuffer+e_in->name);
00797                     node->number = (WORD)(e_out - Expressions);
00798                     e_out->onfile = e_in->onfile;
00799                     e_out->prototype = e_in->prototype;
00800                     e_out->printfflag = 0;
00801                     e_out->status = e_in->status;
00802                     e_out->name = e_in->name;
00803                     e_out->inmem = e_in->inmem;
00804                     e_out->counter = e_in->counter;
00805                     e_out->numfactors = e_in->numfactors;
00806                     e_out->numdummies = e_in->numdummies;
00807                     e_out->compression = e_in->compression;
00808                     e_out->namesize = e_in->namesize;
00809                     e_out->whichbuffer = e_in->whichbuffer;
00810                     e_out->hidelevel = e_in->hidelevel;
00811                     e_out->node = e_in->node;
00812                     e_out->replace = e_in->replace;
00813                     e_out->vflags = e_in->vflags;
00814                     e_out->uflags = e_in->uflags;
00815 #ifdef PARALLELCODE
00816                     e_out->partodo = e_in->partodo;
00817 #endif
00818                 }
00819                 e_out++;
00820                 j++;
00821             }
00822             e_in++;
00823         } while ( --n > 0 ); }
00824         NumExpressions = j;
00825         if ( DidClean ) CompactifyTree(AC.exprnames, EXPRNAMES);
00826     }
00827     AR.StoreData.Handle = -1;
00828     CloseFile(AC.StoreHandle);
00829     AC.StoreHandle = -1;
00830     {
00831         /*
00832             Knock out the storage caches (25-apr-1990!)
00833         */
00834         STORECACHE st;
00835         st = (STORECACHE) (AT.StoreCache);
00836         while ( st ) {
00837             SETBASEPOSITION(st->position,-1);
00838             SETBASEPOSITION(st->toppos,-1);
00839             st = st->next;
00840         }
00841 #ifdef WITHPTHREADS
00842         for ( j = 1; j < AM.totalnumberofthreads; j++ ) {
00843             st = (STORECACHE) (AB[j]->T.StoreCache);
00844             while ( st ) {
00845                 SETBASEPOSITION(st->position,-1);
00846                 SETBASEPOSITION(st->toppos,-1);
00847                 st = st->next;
00848             }
00849         }
00850 #endif
00851     }
00852     PUTZERO(AC.StoreFileSize);
00853     s = FG.fname; while ( *s ) s++;
00854 #ifdef VMS
00855     *s = ','; s[1] = '*'; s[2] = 0;
00856     remove(FG.fname);
00857     *s = 0;
00858 #endif
00859     return(AC.StoreHandle = CreateFile(FG.fname));
00860 }
00861 else return(0);
00862 }
00863
00864 /*
00865     #] DeleteStore :
00866     #[ PutInStore :
00867
00868     Copies the expression indicated by ind from a load file to the

```

```

00869         internal storage file. A return value of zero indicates that
00870         everything is OK.
00871
00872 */
00873
00874 WORD PutInStore(INDEXENTRY *ind, WORD num)
00875 {
00876     GETIDENTITY
00877     INDEXENTRY *newind;
00878     LONG wSize;
00879 #ifndef SYSDEPENDENTSAVE
00880     LONG wSizeOut;
00881     LONG stage;
00882 #endif
00883     POSITION scrpos, scrpos1;
00884     newind = NextFileIndex(&(Expressions[num].onfile));
00885     *newind = *ind;
00886 #ifndef SYSDEPENDENTSAVE
00887     SETBASEPOSITION(newind->length, 0);
00888 #endif
00889     newind->variables = AR.StoreData.Fill;
00890     SeekFile(AR.StoreData.Handle, &(newind->variables), SEEK_SET);
00891     if ( ISNOTEQUALPOS(newind->variables, AR.StoreData.Fill) ) goto PutErrS;
00892     newind->position = newind->variables;
00893 #ifdef SYSDEPENDENTSAVE
00894     ADDPOS(newind->position, DIFBASE(ind->position, ind->variables));
00895 #endif
00896     /* set read position to ind->variables */
00897     scrpos = ind->variables;
00898     SeekFile(AO.SaveData.Handle, &scrpos, SEEK_SET);
00899     if ( ISNOTEQUALPOS(scrpos, ind->variables) ) goto PutErrS;
00900     /* set max size for read-in */
00901     wSize = TOLONG(AT.WorkTop) - TOLONG(AT.WorkPointer);
00902 #ifdef SYSDEPENDENTSAVE
00903     scrpos = ind->length;
00904     ADDPOS(scrpos, DIFBASE(ind->position, ind->variables));
00905     ADD2POS(AR.StoreData.Fill, scrpos);
00906 #endif
00907     SETBASEPOSITION(scrpos1, wSize);
00908 #ifndef SYSDEPENDENTSAVE
00909     /* prepare look-up table for tensor functions */
00910     if ( ind->nfunctions ) {
00911         AO.tensorList = (UBYTE *)Malloca1(MAXSAVEFUNCTION, "PutInStore");
00912     }
00913     SETBASEPOSITION(scrpos, DIFBASE(ind->position, ind->variables));
00914     /* copy variables first */
00915     stage = -1;
00916     do {
00917         wSize = TOLONG(AT.WorkTop) - TOLONG(AT.WorkPointer);
00918         if ( ISLESSPOS(scrpos, scrpos1) ) wSize = BASEPOSITION(scrpos);
00919         wSizeOut = wSize;
00920         if ( ReadSaveVariables(
00921             (UBYTE *)AT.WorkPointer, (UBYTE *)AT.WorkTop, &wSize, &wSizeOut, ind, &stage) ) {
00922             goto PutErrS;
00923         }
00924         if ( WriteFile(AR.StoreData.Handle, (UBYTE *)AT.WorkPointer, wSizeOut)
00925             != wSizeOut ) goto PutErrS;
00926         ADDPOS(scrpos, -wSize);
00927         ADDPOS(newind->position, wSizeOut);
00928         ADDPOS(AR.StoreData.Fill, wSizeOut);
00929     } while ( ISPOSPOS(scrpos) );
00930     /* then copy the expression itself */
00931     wSize = TOLONG(AT.WorkTop) - TOLONG(AT.WorkPointer);
00932     scrpos = ind->length;
00933 #endif
00934     do {
00935         wSize = TOLONG(AT.WorkTop) - TOLONG(AT.WorkPointer);
00936         if ( ISLESSPOS(scrpos, scrpos1) ) wSize = BASEPOSITION(scrpos);
00937 #ifdef SYSDEPENDENTSAVE
00938         if ( ReadFile(AO.SaveData.Handle, (UBYTE *)AT.WorkPointer, wSize)
00939             != wSize ) goto PutErrS;
00940         if ( WriteFile(AR.StoreData.Handle, (UBYTE *)AT.WorkPointer, wSize)
00941             != wSize ) goto PutErrS;
00942         ADDPOS(scrpos, -wSize);
00943 #else
00944         wSizeOut = wSize;
00945
00946         if ( ReadSaveExpression((UBYTE *)AT.WorkPointer, (UBYTE *)AT.WorkTop, &wSize, &wSizeOut) ) {
00947             goto PutErrS;
00948         }
00949
00950         if ( WriteFile(AR.StoreData.Handle, (UBYTE *)AT.WorkPointer, wSizeOut)
00951             != wSizeOut ) goto PutErrS;
00952         ADDPOS(scrpos, -wSize);
00953         ADDPOS(AR.StoreData.Fill, wSizeOut);
00954         ADDPOS(newind->length, wSizeOut);
00955 #endif

```

```

00956     } while ( ISPOSPOS(scrpos) );
00957     /* free look-up table for tensor functions */
00958     if ( ind->nfunctions ) {
00959         M_free(AO.tensorList,"PutInStore");
00960     }
00961     scrpos = AR.StoreData.Position;
00962     SeekFile(AR.StoreData.Handle,&scrpos,SEEK_SET);
00963     if ( ISNOTEQUALPOS(scrpos,AR.StoreData.Position) ) goto PutErrS;
00964     if ( WriteFile(AR.StoreData.Handle,(UBYTE *)(&AR.StoreData.Index),(LONG)sizeof(FILEINDEX))
00965 == (LONG)sizeof(FILEINDEX) ) return(0);
00966 PutErrS:
00967     return(MesPrint("File error"));
00968 }
00969
00970 /*
00971     #] PutInStore :
00972     #[ GetTerm :
00973
00974     Gets one term from input scratch stream.
00975     Puts it in 'term'.
00976     Returns the length of the term.
00977
00978     Used by Processor      (proces.c)
00979     WriteAll               (sch.c)
00980     WriteOne               (sch.c)
00981     GetMoreTerms           (store.c)
00982     ToStorage              (store.c)
00983     CoFillExpression       (comexpr.c)
00984     FactorInExpr           (factor.c)
00985     LoadOpti              (optim.c)
00986     PF_Processor           (parallel.c)
00987     ThreadsProcessor       (threads.c)
00988     In multi thread/processor mode all calls are done by the master.
00989     Note however that other routines, used by the threads, can use
00990     the same file. Hence we need to be careful about SeekFile and locks.
00991 */
00992
00993 WORD GetTerm(PHEAD WORD *term)
00994 {
00995     GETBIDENTITY
00996     WORD *inp, i, j = 0, len;
00997     LONG InIn, *whichInInBuf;
00998     WORD *r, *m, *mstop = 0, minsiz = 0, *bra = 0, *from;
00999     WORD first, *start = 0, testing = 0;
01000     FILEHANDLE *fi;
01001     AN.deferskipped = 0;
01002     if ( AR.GetFile == 2 ) {
01003         fi = AR.hidefile;
01004         whichInInBuf = &(AR.InHiBuf);
01005     }
01006     else {
01007         fi = AR.infile;
01008         whichInInBuf = &(AR.InInBuf);
01009     }
01010     InIn = *whichInInBuf;
01011     from = term;
01012     if ( AR.KeptInHold ) {
01013         r = AR.CompressBuffer;
01014         i = *r;
01015         AR.KeptInHold = 0;
01016         if ( i <= 0 ) { *term = 0; goto RegRet; }
01017         m = term;
01018         NCOPY(m,r,i);
01019         goto RegRet;
01020     }
01021     if ( AR.DeferFlag ) {
01022         m = AR.CompressBuffer;
01023         if ( *m > 0 ) {
01024             mstop = m + *m;
01025             mstop -= ABS(mstop[-1]);
01026             m++;
01027             while ( m < mstop ) {
01028                 if ( *m == HAAKJE ) {
01029                     testing = 1;
01030                     mstop = m + m[1];
01031                     bra = (WORD *)(((UBYTE *) (term)) + 2*AM.MaxTer);
01032                     m = AR.CompressBuffer+1;
01033                     r = bra;
01034                     while ( m < mstop ) *r++ = *m++;
01035                     mstop = r;
01036                     minsiz = WORDDIF(mstop,bra);
01037                     goto ReStart;
01038                 }
01039                 /*
01040                 We have the bracket to be tested in bra till mstop
01041                 */
01042                 m += m[1];

```

```

01043     }
01044     }
01045     bra = (WORD *) (( (BYTE *) (term)) + 2*AM.MaxTer);
01046     mstop = bra+1;
01047     *bra = 0;
01048     minsiz = 1;
01049     testing = 1;
01050 }
01051 ReStart:
01052     first = 0;
01053     r = AR.CompressBuffer;
01054     if ( fi->handle >= 0 ) {
01055         if ( InIn <= 0 ) {
01056             ADDPOS(fi->PPosition, (fi->POfull-fi->PObuffer)*sizeof(WORD));
01057             LOCK(AS.inputslock);
01058             SeekFile(fi->handle, &(fi->PPosition), SEEK_SET);
01059             InIn = ReadFile(fi->handle, (BYTE *) (fi->PObuffer), fi->POsize);
01060             UNLOCK(AS.inputslock);
01061             if ( ( InIn < 0 ) || ( InIn & 1 ) ) {
01062                 goto GTerr;
01063             }
01064 #ifdef WORD2
01065             InIn >= 1;
01066 #else
01067             InIn /= TABLESIZE(WORD, BYTE);
01068 #endif
01069             *whichInInBuf = InIn;
01070             if ( !InIn ) { *r = 0; *from = 0; goto RegRet; }
01071             fi->POfill = fi->PObuffer;
01072             fi->POfull = fi->PObuffer + InIn;
01073         }
01074         inp = fi->POfill;
01075         if ( ( len = i = *inp ) == 0 ) {
01076             (*whichInInBuf)--;
01077             (fi->POfill)++;
01078             *r = 0;
01079             *from = 0;
01080             goto RegRet;
01081         }
01082         if ( i < 0 ) {
01083             InIn--;
01084             inp++;
01085             r++;
01086             start = term;
01087             *term++ = -i + 1;
01088             while ( ++i <= 0 ) *term++ = *r++;
01089             if ( InIn > 0 ) {
01090                 i = *inp++;
01091                 InIn--;
01092                 *start += i;
01093                 *(AR.CompressBuffer) = len = *start;
01094             }
01095             else {
01096                 first = 1;
01097                 goto NewIn;
01098             }
01099         }
01100         InIn -= i;
01101         if ( InIn < 0 ) {
01102             j = (WORD) (- InIn);
01103             i -= j;
01104         }
01105         else j = 0;
01106         while ( --i >= 0 ) {
01107             *r++ = *term++ = *inp++;
01108         }
01109         if ( j ) {
01110 NewIn:
01111             ADDPOS(fi->PPosition, (fi->POfull-fi->PObuffer)*sizeof(WORD));
01112             LOCK(AS.inputslock);
01113             SeekFile(fi->handle, &(fi->PPosition), SEEK_SET);
01114             InIn = ReadFile(fi->handle, (BYTE *) (fi->PObuffer), fi->POsize);
01115             UNLOCK(AS.inputslock);
01116             if ( ( InIn <= 0 ) || ( InIn & 1 ) ) {
01117                 goto GTerr;
01118             }
01119 #ifdef WORD2
01120             InIn >= 1;
01121 #else
01122             InIn /= TABLESIZE(WORD, BYTE);
01123 #endif
01124             inp = fi->PObuffer;
01125             fi->POfull = inp + InIn;
01126         }
01127         if ( first ) {
01128             j = *inp++;
01129             InIn--;

```



```

01130         *start += j;
01131         *(AR.CompressBuffer) = len = *start;
01132     }
01133     InIn -= j;
01134     while ( --j >= 0 ) { *r++ = *term++ = *inp++; }
01135 }
01136 fi->POfill = inp;
01137 *whichInInBuf = InIn;
01138 AR.DefPosition = fi->POposition;
01139 ADDPOS(AR.DefPosition, ((UBYTE *) (fi->POfill) - (UBYTE *) (fi->PObuffer)));
01140 }
01141 else {
01142     inp = fi->POfill;
01143     if ( inp >= fi->POfull ) { *from = 0; goto RegRet; }
01144     len = j = *inp;
01145     if ( j < 0 ) {
01146         inp++;
01147         *term++ = *r++ = len = - j + 1 + *inp;
01148         while ( ++j <= 0 ) *term++ = *r++;
01149         j = *inp++;
01150     }
01151     else if ( !j ) j = 1;
01152     while ( --j >= 0 ) { *r++ = *term++ = *inp++; }
01153     fi->POfill = inp;
01154     /*%%%%ADDED 7-apr-2006 for Keep Brackets in bucket */
01155     SETBASEPOSITION(AR.DefPosition, ((UBYTE *) (fi->POfill) - (UBYTE *) (fi->PObuffer)));
01156     if ( inp > fi->POfull ) {
01157         goto GTerr;
01158     }
01159 }
01160 if ( r >= AR.ComprTop ) {
01161     MesPrint("CompressSize of %10l is insufficient", AM.CompressSize);
01162     Terminate(-1);
01163 }
01164 AR.CompressPointer = r; *r = 0;
01165 /*
01166     The next *from is a bug fix that made the program read in forbidden
01167     territory.
01168 */
01169 if ( testing && *from != 0 ) {
01170     WORD jj;
01171     r = from;
01172     jj = *r - 1 - ABS(*(r+r-1));
01173     if ( jj < minsiz ) goto strip;
01174     r++;
01175     m = bra;
01176     while ( m < mstop ) {
01177         if ( *m != *r ) {
01178 strip:
01179             r = from;
01180             m = r + *r;
01181             mstop = m - ABS(m[-1]);
01182             r++;
01183             while ( r < mstop ) {
01184                 if ( *r == HAAKJE ) {
01185                     *r++ = 1;
01186                     *r++ = 1;
01187                     *r++ = 3;
01188                     len = WORDDIF(r, from);
01189                     *from = len;
01190                     goto RegRet;
01191                 }
01192                 r += r[1];
01193             }
01194             goto RegRet;
01195         }
01196         m++;
01197         r++;
01198     }
01199     term = from;
01200     AN.deferskipped++;
01201     goto ReStart;
01202 }
01203 RegRet:;
01204 /*
01205     #[ debug :
01206 */
01207 {
01208     UBYTE OutBuf[140];
01209     /* if ( AP.DebugFlag ) { */
01210     if ( ( AP.PreDebug & DUMPINTERMS ) == DUMPINTERMS ) {
01211         MLOCK(ErrorMessageLock);
01212         AO.OutFill = AO.OutputLine = OutBuf;
01213         AO.OutSkip = 3;
01214         FiniLine();
01215         r = from;
01216         i = *r;
01217         TokenToLine((UBYTE *) ("Input: "));
01218     }
01219     /* } */

```

```

01217         if ( i == 0 ) {
01218             TokenToLine((UBYTE *)"zero");
01219         }
01220     else if ( i < 0 ) {
01221         TokenToLine((UBYTE *)"negative!!");
01222     }
01223     else {
01224         while ( --i >= 0 ) {
01225             TalToLine((UWORD) (*r++)); TokenToLine((UBYTE *)" ");
01226         }
01227     }
01228     FiniLine();
01229     MUNLOCK(ErrorMessageLock);
01230 }
01231 }
01232 /*
01233     #] debug :
01234 */
01235     return(*from);
01236 GTerr:
01237     MesPrint("Error while reading scratch file in GetTerm");
01238     Terminate(-1);
01239     return(-1);
01240 }
01241
01242 /*
01243     #] GetTerm :
01244     #[ GetOneTerm :
01245
01246     Gets one term from stream AR.infile->handle.
01247     Puts it in 'term'.
01248     Returns the length of the term.
01249     Input is unbuffered.
01250     Compression via AR.CompressPointer
01251     par is actually in all calls a file handle
01252
01253     Routine is called from
01254         DoOnePow           Get one power of an expression
01255         Deferred           Get the contents of a bracket
01256         GetFirstBracket
01257         FindBracket
01258     We should do something about the lack of buffering.
01259     Maybe a buffer of a few times AM.MaxTer (MaxTermSize*sizeof(WORD)).
01260     Each thread will need its own buffer!
01261
01262     If par == 0 we use ReadPosFile which can fill the whole buffer.
01263     If par == 1 we use ReadFile and do actual read operations.
01264
01265     Note: we cannot use ReadPosFile when running in the master thread.
01266 */
01267
01268 WORD GetOneTerm(PHEAD WORD *term, FILEHANDLE *fi, POSITION *pos, int par)
01269 {
01270     GETBIDENTITY
01271     WORD i, *p;
01272     LONG j, siz;
01273     WORD *r, *rr = AR.CompressPointer;
01274     int error = 0;
01275     r = rr;
01276     if ( fi->handle >= 0 ) {
01277 #ifdef READONEBYONE
01278 #ifdef WITHPTHREADS
01279 /*
01280         This code needs some investigation.
01281         It may be that we should do this always.
01282         It may be that even for workers it is no good.
01283         We may have to make a variable like AM.ReadDirect with
01284         if ( AM.ReadDirect ) par = 1;
01285         and a user command like
01286         On ReadDirect;
01287 */
01288         if ( AT.identity > 0 ) par = 1;
01289 #endif
01290 #endif
01291 /*
01292         To be changed:
01293         1: check first whether the term lies completely inside the buffer
01294         2: if not a: use old strategy for AT.identity == 0 (master)
01295             b: for workers, position file and read buffer
01296 */
01297         if ( par == 0 ) {
01298             siz = ReadPosFile(BHEAD fi, (UBYTE *)term, 1L, pos);
01299         }
01300         else {
01301             LOCK(AS.inputslock);
01302             SeekFile(fi->handle, pos, SEEK_SET);
01303             siz = ReadFile(fi->handle, (UBYTE *)term, sizeof(WORD));

```

```

01304         UNLOCK(AS.inputslock);
01305         ADDPOS(*pos,siz);
01306     }
01307     if ( siz == sizeof(WORD) ) {
01308         p = term;
01309         j = i = *term++;
01310         if ( ( i > AM.MaxTer/((WORD)sizeof(WORD)) ) || ( -i >= AM.MaxTer/((WORD)sizeof(WORD)) ) )
01311         {
01312             error = 1;
01313             goto ErrGet;
01314         }
01315         r++;
01316         if ( i < 0 ) { /* Loading a compressed term */
01317             *p = -i + 1;
01318             while ( ++i <= 0 ) *term++ = *r++;
01319             if ( par == 0 ) {
01320                 siz = ReadPosFile(BHEAD fi, (UBYTE *)term,1L,pos);
01321             }
01322             else {
01323                 LOCK(AS.inputslock);
01324                 SeekFile(fi->handle,pos,SEEK_SET);
01325                 siz = ReadFile(fi->handle, (UBYTE *)term,sizeof(WORD));
01326                 UNLOCK(AS.inputslock);
01327                 ADDPOS(*pos,sizeof(WORD));
01328             }
01329             if ( siz != sizeof(WORD) ) {
01330                 error = 2;
01331                 goto ErrGet;
01332             }
01333             *p += *term;
01334             j = *term;
01335             if ( ( j > AM.MaxTer/((WORD)sizeof(WORD)) ) || ( j <= 0 ) )
01336             {
01337                 error = 3;
01338                 goto ErrGet;
01339             }
01340             *rr = *p; /* Write the proper term size to *AR.CompressPointer */
01341         }
01342         else { /* Loading a regular term */
01343             if ( !j ) return(0);
01344             *rr = i; /* Write the proper term size to *AR.CompressPointer */
01345             j--;
01346         }
01347         i = (WORD)j;
01348         if ( par == 0 ) {
01349             siz = ReadPosFile(BHEAD fi, (UBYTE *)term,j,pos);
01350             j *= TABLESIZE(WORD,UBYTE);
01351         }
01352         else {
01353             j *= TABLESIZE(WORD,UBYTE);
01354             LOCK(AS.inputslock);
01355             SeekFile(fi->handle,pos,SEEK_SET);
01356             siz = ReadFile(fi->handle, (UBYTE *)term,j);
01357             UNLOCK(AS.inputslock);
01358             ADDPOS(*pos,j);
01359         }
01360         if ( siz != j ) {
01361             error = 4;
01362             goto ErrGet;
01363         }
01364         while ( --i >= 0 ) *r++ = *term++;
01365         if ( r >= AR.ComprTop ) {
01366             MLOCK(ErrorMessageLock);
01367             MesPrint("CompressSize of %10l is insufficient",AM.CompressSize);
01368             MUNLOCK(ErrorMessageLock);
01369             Terminate(-1);
01370         }
01371         AR.CompressPointer = r; *r = 0;
01372         return(*p);
01373     }
01374     error = 5;
01375 }
01376 else {
01377     /*
01378     Here the whole expression is in the buffer.
01379     */
01380     fi->POfill = (WORD *)((UBYTE *) (fi->PObuffer) + BASEPOSITION(*pos));
01381     p = fi->POfill;
01382     if ( p >= fi->POfull ) { *term = 0; return(0); }
01383     j = i = *p;
01384     if ( i < 0 ) {
01385         p++;
01386         j = *r++ = *term++ = -i + 1 + *p;
01387         while ( ++i <= 0 ) *term++ = *r++;
01388         i = *p++;
01389     }
01390     if ( i == 0 ) { i = 1; *r++ = 0; *term++ = 0; }

```

```

01391         else { while ( --i >= 0 ) { *r++ = *term++ = *p++; } }
01392         fi->POfill = p;
01393         SETBASEPOSITION(*pos, (UBYTE *) (fi->POfill) - (UBYTE *) (fi->PObuffer));
01394         if ( p <= fi->POfull ) {
01395             if ( r >= AR.ComprTop ) {
01396                 MLOCK(ErrorMessageLock);
01397                 MesPrint("CompressSize of %10l is insufficient", AM.CompressSize);
01398                 MUNLOCK(ErrorMessageLock);
01399                 Terminate(-1);
01400             }
01401             AR.CompressPointer = r; *r = 0;
01402             return((WORD)j);
01403         }
01404         error = 6;
01405     }
01406 ErrGet:
01407     MLOCK(ErrorMessageLock);
01408     MesPrint("Error while reading scratch file in GetOneTerm (%d)", error);
01409     MUNLOCK(ErrorMessageLock);
01410     Terminate(-1);
01411     return(-1);
01412 }
01413
01414 /*
01415     #] GetOneTerm :
01416     #[ GetMoreTerms :
01417     Routine collects more contents of brackets inside a function,
01418     indicated by the number in AC.CollectFun.
01419     The first term is in term already.
01420     We can keep calling GetTerm either till a bracket is finished
01421     or till it would make the term too long (> AM.MaxTer/2)
01422     In all cases this function makes that the routine GetTerm
01423     has a term in 'hold', so the AR.KeptInHold flag must be turned on.
01424 */
01425
01426 WORD GetMoreTerms(WORD *term)
01427 {
01428     GETIDENTITY
01429     WORD *t, *r, *m, *h, *tstop, i, inc, same;
01430     WORD extra;
01431     WORD retval = 0;
01432 /*
01433     We use 23% as a quasi-random default value.
01434 */
01435     extra = ((AM.MaxTer/sizeof(WORD)) * ((LONG)100 - AC.CollectPercentage))/100;
01436     if ( extra < 23 ) extra = 23;
01437 /*
01438     First find the bracket pointer
01439 */
01440     t = term + *term;
01441     tstop = t - ABS(t[-1]);
01442     h = term+1;
01443     while ( *h != HAAKJE && h < tstop ) h += h[1];
01444     if ( h >= tstop ) return(retval);
01445     inc = FUNHEAD+ARGHEAD+1-h[1];
01446     same = WORDDIFF(h, term) + h[1] - 1;
01447     r = m = t + inc;
01448     tstop = h + h[1];
01449     while ( t > tstop ) *--r = *--t;
01450     r--;
01451     *r = WORDDIFF(m, r);
01452     while ( GetTerm(BHEAD m) > 0 ) {
01453         r = m + 1;
01454         t = m + *m - 1;
01455         if ( same > ( i = ( *m - ABS(*t) - 1 ) ) ) { /* Must fail */
01456             if ( AC.AltCollectFun && AS.CollectOverFlag == 2 ) AS.CollectOverFlag = 3;
01457             break;
01458         }
01459         t = term+1;
01460         i = same;
01461         while ( --i >= 0 ) {
01462             if ( *r != *t ) {
01463                 if ( AC.AltCollectFun && AS.CollectOverFlag == 2 ) AS.CollectOverFlag = 3;
01464                 goto FullTerm;
01465             }
01466             r++; t++;
01467         }
01468         if ( ( WORDDIFF(m, term) + i + extra ) > (WORD)(AM.MaxTer/sizeof(WORD)) ) {
01469 /* 23 = 3 + 20. The 20 is to have some extra for substitutions or whatever */
01470             if ( AS.CollectOverFlag == 0 && AC.AltCollectFun == 0 ) {
01471                 Warning("Bracket contents too long in Collect statement");
01472                 Warning("Contents spread over more than one term");
01473                 Warning("If possible: increase MaxTermSize in setfile");
01474                 AS.CollectOverFlag = 1;
01475             }
01476             else if ( AC.AltCollectFun ) {
01477                 AS.CollectOverFlag = 2;

```

```

01478         }
01479         break;
01480     }
01481     tstop = m + *m;
01482     *m -= same;
01483     m++;
01484     while ( r < tstop ) *m++ = *r++;
01485     retval++;
01486     if ( extra == 23 ) extra = ((AM.MaxTer/sizeof(WORD))/6);
01487 }
01488 FullTerm:
01489     h[1] = WORDDIF(m,h);
01490     if ( AS.CollectOverFlag > 1 ) {
01491         *h = AC.AltCollectFun;
01492         if ( AS.CollectOverFlag == 3 ) AS.CollectOverFlag = 1;
01493     }
01494     else *h = AC.CollectFun;
01495     h[2] |= DIRTYFLAG;
01496     h[FUNHEAD] = h[1] - FUNHEAD;
01497     h[FUNHEAD+1] = 0;
01498     if ( ToFast(h+FUNHEAD,h+FUNHEAD) ) {
01499         if ( h[FUNHEAD] <= -FUNCTION ) {
01500             h[1] = FUNHEAD+1;
01501             m = h + FUNHEAD+1;
01502         }
01503         else {
01504             h[1] = FUNHEAD+2;
01505             m = h + FUNHEAD+2;
01506         }
01507     }
01508     *m++ = 1;
01509     *m++ = 1;
01510     *m++ = 3;
01511     *term = WORDDIF(m,term);
01512     AR.KeptInHold = 1;
01513     return(retval);
01514 }
01515
01516 /*
01517     #] GetMoreTerms :
01518     #[ GetMoreFromMem :
01519
01520 */
01521
01522 WORD GetMoreFromMem(WORD *term, WORD **tpoin)
01523 {
01524     GETIDENTITY
01525     WORD *t, *r, *m, *h, *tstop, i, j, inc, same;
01526     LONG extra = 23;
01527
01528     /* First find the bracket pointer
01529     */
01530     t = term + *term;
01531     tstop = t - ABS(t[-1]);
01532     h = term+1;
01533     while ( *h != HAAKJE && h < tstop ) h += h[1];
01534     if ( h >= tstop ) return(0);
01535     inc = FUNHEAD+ARGHEAD+1-h[1];
01536     same = WORDDIF(h,term) + h[1] - 1;
01537     r = m = t + inc;
01538     tstop = h + h[1];
01539     while ( t > tstop ) *--r = *--t;
01540     r--;
01541     *r = WORDDIF(m,r);
01542     while ( **tpoin ) {
01543         r = *tpoin; j = *r;
01544         for ( i = 0; i < j; i++ ) m[i] = *r++;
01545         *tpoin = r;
01546         r = m + 1;
01547         t = m + *m - 1;
01548         if ( same > ( i = ( *m - ABS(*t) -1 ) ) ) { /* Must fail */
01549             if ( AC.AltCollectFun && AS.CollectOverFlag == 2 ) AS.CollectOverFlag = 3;
01550             break;
01551         }
01552         t = term+1;
01553         i = same;
01554         while ( --i >= 0 ) {
01555             if ( *r != *t ) {
01556                 if ( AC.AltCollectFun && AS.CollectOverFlag == 2 ) AS.CollectOverFlag = 3;
01557                 goto FullTerm;
01558             }
01559             r++; t++;
01560         }
01561         if ( ( WORDDIF(m,term) + i + extra ) > (LONG)(AM.MaxTer/(2*sizeof(WORD))) ) {
01562             /* 23 = 3 +20. The 20 is to have some extra for substitutions or whatever */
01563             if ( AS.CollectOverFlag == 0 && AC.AltCollectFun == 0 ) {
01564                 Warning("Bracket contents too long in Collect statement");

```

```

01565         Warning("Contents spread over more than one term");
01566         Warning("If possible: increase MaxTermSize in setfile");
01567         AS.CollectOverFlag = 1;
01568     }
01569     else if ( AC.AltCollectFun ) {
01570         AS.CollectOverFlag = 2;
01571     }
01572     break;
01573 }
01574 tstop = m + *m;
01575 *m -= same;
01576 m++;
01577 while ( r < tstop ) *m++ = *r++;
01578 if ( extra == 23 ) extra = ((AM.MaxTer/sizeof(WORD))/6);
01579 }
01580 FullTerm:
01581 h[1] = WORDDIF(m,h);
01582 if ( AS.CollectOverFlag > 1 ) {
01583     *h = AC.AltCollectFun;
01584     if ( AS.CollectOverFlag == 3 ) AS.CollectOverFlag = 1;
01585 }
01586 else *h = AC.CollectFun;
01587 h[2] |= DIRTYFLAG;
01588 h[FUNHEAD] = h[1] - FUNHEAD;
01589 h[FUNHEAD+1] = 0;
01590 if ( ToFast(h+FUNHEAD,h+FUNHEAD) ) {
01591     if ( h[FUNHEAD] <= -FUNCTION ) {
01592         h[1] = FUNHEAD+1;
01593         m = h + FUNHEAD+1;
01594     }
01595     else {
01596         h[1] = FUNHEAD+2;
01597         m = h + FUNHEAD+2;
01598     }
01599 }
01600 *m++ = 1;
01601 *m++ = 1;
01602 *m++ = 3;
01603 *term = WORDDIF(m,term);
01604 AR.KeptInHold = 1;
01605 return(0);
01606 }
01607
01608 /*
01609     #] GetMoreFromMem :
01610     #[ GetFromStore :
01611
01612     Gets a single term from the storage file at position and puts
01613     it at 'to'.
01614     The value to be returned is the number of words read.
01615     Renumbering is done also.
01616     This is controlled by the renumber table, given in 'renumber'
01617
01618     This routine should work with a number of cache buffers. The
01619     exact number should be definable in form.set.
01620     The parameters are:
01621     AM.SizeStoreCache    (4096)
01622     The numbers are the proposed default values.
01623
01624     The cache is a pure read cache.
01625 */
01626
01627 static int gfs = 0;
01628
01629 WORD GetFromStore(WORD *to, POSITION *position, RENUMBER renumber, WORD *InCompState, WORD nexpr)
01630 {
01631     GETIDENTITY
01632     LONG RetCode, num, first = 0;
01633     WORD *from, *m;
01634     struct StOrEcAcHe sc;
01635     STORECACHE s;
01636     STORECACHE snext, sold;
01637     WORD *r, *rr = AR.CompressPointer;
01638     r = rr;
01639     gfs++;
01640     sc.next = AT.StoreCache;
01641     sold = s = &sc;
01642     snext = s->next;
01643     while ( snext ) {
01644         sold = s;
01645         s = snext;
01646         snext = s->next;
01647         if ( BASEPOSITION(s->position) == -1 ) break;
01648         if ( ISLESSPOS(*position,s->toppos) &&
01649             ISGEPOS(*position,s->position) ) { /* Hit */
01650             if ( AT.StoreCache != s ) {
01651                 sold->next = s->next;

```

```

01652         s->next = AT.StoreCache->next;
01653         AT.StoreCache = s;
01654     }
01655     from = (WORD *)(((UBYTE *) (s->buffer)) + DIFBASE(*position,s->position));
01656     num = *from;
01657     if ( !num ) { return(*to = 0); }
01658     *InCompState = (WORD)num;
01659     m = to;
01660     if ( num < 0 ) {
01661         from++;
01662         ADDPOS(*position,sizeof(WORD));
01663         *m++ = (WORD) (-num+1);
01664         r++;
01665         while ( ++num <= 0 ) *m++ = *r++;
01666         if ( ISLESSPOS(*position,s->toppos) ) {
01667             num = *from++;
01668             *to += (WORD)num;
01669             ADDPOS(*position,sizeof(WORD));
01670             *InCompState = (WORD) (num + 2);
01671         }
01672         else {
01673             first = 1;
01674             goto InNew;
01675         }
01676     }
01677 PastCon:;
01678     while ( num > 0 && ISLESSPOS(*position,s->toppos) ) {
01679         *r++ = *m++ = *from++; ADDPOS(*position,sizeof(WORD)); num--;
01680     }
01681     if ( num > 0 ) {
01682 InNew:
01683         SETBASEPOSITION(s->position,-1);
01684         SETBASEPOSITION(s->toppos,-1);
01685         LOCK(AM.storefilelock);
01686         SeekFile(AR.StoreData.Handle,position,SEEK_SET);
01687         RetCode = ReadFile(AR.StoreData.Handle,(UBYTE *) (s->buffer),AM.SizeStoreCache);
01688         UNLOCK(AM.storefilelock);
01689         if ( RetCode < 0 ) goto PastErr;
01690         if ( !RetCode ) return( *to = 0 );
01691         s->position = *position;
01692         s->toppos = *position;
01693         ADDPOS(s->toppos,RetCode);
01694         from = s->buffer;
01695         if ( first ) {
01696             num = *from++;
01697             ADDPOS(*position,sizeof(WORD));
01698             *to += (WORD)num;
01699             /* This next line has always been commented, but uncommenting it
01700              fixes a rare bug when loading certain save files. */
01701             first = 0;
01702             *InCompState = (WORD) (num + 2);
01703         }
01704         goto PastCon;
01705     }
01706     goto PastEnd;
01707 }
01708 }
01709 if ( AT.StoreCache ) { /* Fill the last buffer */
01710     s->position = *position;
01711     LOCK(AM.storefilelock);
01712     SeekFile(AR.StoreData.Handle,position,SEEK_SET);
01713     RetCode = ReadFile(AR.StoreData.Handle,(UBYTE *) (s->buffer),AM.SizeStoreCache);
01714     UNLOCK(AM.storefilelock);
01715     if ( RetCode < 0 ) goto PastErr;
01716     if ( !RetCode ) return( *to = 0 );
01717     s->toppos = *position;
01718     ADDPOS(s->toppos,RetCode);
01719     if ( AT.StoreCache != s ) {
01720         sold->next = s->next;
01721         s->next = AT.StoreCache->next;
01722         AT.StoreCache = s;
01723     }
01724     m = to;
01725     from = s->buffer;
01726     num = *from;
01727     if ( !num ) { return( *to = 0 ); }
01728     *InCompState = (WORD)num;
01729     if ( num < 0 ) {
01730         *m++ = (WORD) (-num+1);
01731         r++;
01732         from++;
01733         ADDPOS(*position,sizeof(WORD));
01734         while ( ++num <= 0 ) *m++ = *r++;
01735         num = *from++;
01736         *to += (WORD)num;
01737         ADDPOS(*position,sizeof(WORD));
01738         *InCompState = (WORD) (num+2);

```

```

01739     }
01740     goto PastCon;
01741 }
01742 /*      No caching available */
01743 LOCK(AM.storefilelock);
01744 SeekFile(AR.StoreData.Handle,position,SEEK_SET);
01745 RetCode = ReadFile(AR.StoreData.Handle,(UBYTE *)to,(LONG)sizeof(WORD));
01746 SeekFile(AR.StoreData.Handle,position,SEEK_CUR);
01747 UNLOCK(AM.storefilelock);
01748 if ( RetCode != sizeof(WORD) ) {
01749     *to = 0;
01750     return((WORD)RetCode);
01751 }
01752 if ( !*to ) return(0);
01753 m = to;
01754 if ( *to < 0 ) {
01755     num = *m++;
01756     *to = *r++ = (WORD)(-num + 1);
01757     while ( ++num <= 0 ) *m++ = *r++;
01758     LOCK(AM.storefilelock);
01759     SeekFile(AR.StoreData.Handle,position,SEEK_SET);
01760     RetCode = ReadFile(AR.StoreData.Handle,(UBYTE *)m,(LONG)sizeof(WORD));
01761     SeekFile(AR.StoreData.Handle,position,SEEK_CUR);
01762     UNLOCK(AM.storefilelock);
01763     if ( RetCode != sizeof(WORD) ) {
01764         MLOCK(ErrorMessageLock);
01765         MesPrint("@Error in compression of store file");
01766         MUNLOCK(ErrorMessageLock);
01767         return(-1);
01768     }
01769     num = *m;
01770     *to += (WORD)num;
01771     *InCompState = (WORD)(num + 2);
01772 }
01773 else {
01774     *InCompState = *to;
01775     num = *to - 1; m = to + 1; r = rr + 1;
01776 }
01777 first = num;
01778 num *= wsizeof(WORD);
01779 if ( num < 0 ) {
01780     MLOCK(ErrorMessageLock);
01781     MesPrint("@Error in stored expressions file at position %9p",position);
01782     MUNLOCK(ErrorMessageLock);
01783     return(-1);
01784 }
01785 LOCK(AM.storefilelock);
01786 SeekFile(AR.StoreData.Handle,position,SEEK_SET);
01787 RetCode = ReadFile(AR.StoreData.Handle,(UBYTE *)m,num);
01788 SeekFile(AR.StoreData.Handle,position,SEEK_CUR);
01789 UNLOCK(AM.storefilelock);
01790 if ( RetCode != num ) {
01791     MLOCK(ErrorMessageLock);
01792     MesPrint("@Error in stored expressions file at position %9p",position);
01793     MUNLOCK(ErrorMessageLock);
01794     return(-1);
01795 }
01796 NCOPY(r,m,first);
01797 PastErr:
01798 *rr = *to;
01799 if ( r >= AR.ComprTop ) {
01800     MLOCK(ErrorMessageLock);
01801     MesPrint("CompressSize of %10l is insufficient",AM.CompressSize);
01802     MUNLOCK(ErrorMessageLock);
01803     Terminate(-1);
01804 }
01805 AR.CompressPointer = r; *r = 0;
01806 if ( !TermRenumber(to,renumber,nexpr) ) {
01807     MarkDirty(to,DIRTYSYMFLAG);
01808     if ( AR.CurDum > AM.IndDum && Expressions[nexpr].numdummies > 0 )
01809         MoveDummies(BHEAD to,AR.CurDum - AM.IndDum);
01810     return((WORD)*to);
01811 }
01812 PastErr:
01813 MLOCK(ErrorMessageLock);
01814 MesCall("GetFromStore");
01815 MUNLOCK(ErrorMessageLock);
01816 SETERROR(-1)
01817 }
01818
01819 /*
01820     #] GetFromStore :
01821     #[ DetVars :          void DetVars(term)
01822
01823     Determines which variables are used in term.
01824
01825     When par = 1 we are scanning a prototype expression which involves

```



```

01826     completely different rules.
01827
01828 */
01829
01830 void DetVars(WORD *term, WORD par)
01831 {
01832     GETIDENTITY
01833     WORD *stopper;
01834     WORD *t, sym;
01835     WORD *sarg;
01836     stopper = term + *term - 1;
01837     stopper = stopper - ABS(*stopper) + 1;
01838     term++;
01839     if ( par ) {          /* Prototype expression */
01840         WORD n;
01841         if ( ( n = NumSymbols ) > 0 ) {
01842             SYMBOLS tt;
01843             tt = symbols;
01844             do {
01845                 (tt++)->flags &= ~INUSE;
01846             } while ( --n > 0 );
01847         }
01848         if ( ( n = NumIndices ) > 0 ) {
01849             INDICES tt;
01850             tt = indices;
01851             do {
01852                 (tt++)->flags &= ~INUSE;
01853             } while ( --n > 0 );
01854         }
01855         if ( ( n = NumVectors ) > 0 ) {
01856             VECTORS tt;
01857             tt = vectors;
01858             do {
01859                 (tt++)->flags &= ~INUSE;
01860             } while ( --n > 0 );
01861         }
01862         if ( ( n = NumFunctions ) > 0 ) {
01863             FUNCTIONS tt;
01864             tt = functions;
01865             do {
01866                 (tt++)->flags &= ~INUSE;
01867             } while ( --n > 0 );
01868         }
01869         term += SUBEXPSIZE;
01870         while ( term < stopper ) {
01871             if ( *term == SYMTOSYM || *term == SYMTONUM ) {
01872                 term += 2;
01873                 AN.UsedSymbol[*term] = 1;
01874                 symbols[*term].flags |= INUSE;
01875             }
01876             else if ( *term == VECTOVEC ) {
01877                 term += 2;
01878                 AN.UsedVector[*term-AM.OffsetVector] = 1;
01879                 vectors[*term-AM.OffsetVector].flags |= INUSE;
01880             }
01881             else if ( *term == INDTOIND ) {
01882                 term += 2;
01883                 sym = indices[*term - AM.OffsetIndex].dimension;
01884                 if ( sym < 0 ) AN.UsedSymbol[-sym] = 1;
01885                 AN.UsedIndex[*term - AM.OffsetIndex] = 1;
01886                 sym = indices[*term-AM.OffsetIndex].nmin4;
01887                 if ( sym < -NMIN4SHIFT ) AN.UsedSymbol[-sym-NMIN4SHIFT] = 1;
01888                 indices[*term-AM.OffsetIndex].flags |= INUSE;
01889             }
01890             else if ( *term == FUNTOFUN ) {
01891                 term += 2;
01892                 AN.UsedFunction[*term-FUNCTION] = 1;
01893                 functions[*term-FUNCTION].flags |= INUSE;
01894             }
01895             term += 2;
01896         }
01897     }
01898     else {
01899         while ( term < stopper ) {
01900             t = term + term[1];
01901             if ( *term == SYMBOL ) {
01902                 term += 2;
01903                 do {
01904                     AN.UsedSymbol[*term] = 1;
01905                     term += 2;
01906                 } while ( term < t );
01907             }
01908             else if ( *term == DOTPRODUCT ) {
01909                 term += 2;
01910                 do {
01911                     AN.UsedVector[( *term++ ) - AM.OffsetVector] = 1;
01912                     AN.UsedVector[( *term ) - AM.OffsetVector] = 1;

```

```

01913         term += 2;
01914     } while ( term < t );
01915 }
01916 else if ( *term == VECTOR ) {
01917     term += 2;
01918     do {
01919         AN.UsedVector[( *term++ ) - AM.OffsetVector] = 1;
01920         if ( *term >= AM.OffsetIndex && *term < AM.DumInd ) {
01921             sym = indices[*term - AM.OffsetIndex].dimension;
01922             if ( sym < 0 ) AN.UsedSymbol[-sym] = 1;
01923             AN.UsedIndex[*term - AM.OffsetIndex] = 1;
01924             sym = indices[( *term++ ) - AM.OffsetIndex].nmin4;
01925             if ( sym < -NMIN4SHIFT ) AN.UsedSymbol[-sym-NMIN4SHIFT] = 1;
01926         }
01927         else term++;
01928     } while ( term < t );
01929 }
01930 else if ( *term == INDEX || *term == LEVICIVITA || *term == GAMMA
01931 || *term == DELTA ) {
01932     /*
01933     Tensors:
01934         term += 2;
01935     */
01936     if ( *term == INDEX || *term == DELTA ) term += 2;
01937     else {
01938         Tensors:
01939         term += FUNHEAD;
01940     }
01941     while ( term < t ) {
01942         if ( *term >= AM.OffsetIndex && *term < AM.DumInd ) {
01943             sym = indices[*term - AM.OffsetIndex].dimension;
01944             if ( sym < 0 ) AN.UsedSymbol[-sym] = 1;
01945             AN.UsedIndex[( *term ) - AM.OffsetIndex] = 1;
01946             sym = indices[*term-AM.OffsetIndex].nmin4;
01947             if ( sym < -NMIN4SHIFT ) AN.UsedSymbol[-sym-NMIN4SHIFT] = 1;
01948         }
01949         else if ( *term < (WILDOFFSET+AM.OffsetVector) )
01950             AN.UsedVector[( *term ) - AM.OffsetVector] = 1;
01951         term++;
01952     }
01953 }
01954 else if ( *term == HAAKJE ) term = t;
01955 else {
01956     if ( *term > MAXBUILTINFUNCTION )
01957         AN.UsedFunction[( *term ) - FUNCTION] = 1;
01958     if ( *term >= FUNCTION && functions[*term-FUNCTION].spec
01959 >= TENSORFUNCTION && term[1] > FUNHEAD ) goto Tensors;
01960     term += FUNHEAD; /* First argument */
01961     while ( term < t ) {
01962         sarg = term;
01963         NEXTARG(sarg)
01964         if ( *term > 0 ) {
01965             sarg = term + *term; /* End of argument */
01966             term += ARGHEAD; /* First term in argument */
01967             if ( term < sarg ) { do {
01968                 DetVars(term,par);
01969                 term += *term;
01970             } while ( term < sarg ); }
01971         }
01972         else {
01973             if ( *term < -MAXBUILTINFUNCTION ) {
01974                 AN.UsedFunction[-*term-FUNCTION] = 1;
01975             }
01976             else if ( *term == -SYMBOL ) {
01977                 AN.UsedSymbol[term[1]] = 1;
01978             }
01979             else if ( *term == -INDEX ) {
01980                 if ( term[1] < (WILDOFFSET+AM.OffsetVector) ) {
01981                     AN.UsedVector[term[1]-AM.OffsetVector] = 1;
01982                 }
01983                 else if ( term[1] >= AM.OffsetIndex && term[1] < AM.DumInd ) {
01984                     sym = indices[term[1] - AM.OffsetIndex].dimension;
01985                     if ( sym < 0 ) AN.UsedSymbol[-sym] = 1;
01986                     AN.UsedIndex[term[1] - AM.OffsetIndex] = 1;
01987                     sym = indices[term[1]-AM.OffsetIndex].nmin4;
01988                     if ( sym < -NMIN4SHIFT ) AN.UsedSymbol[-sym-NMIN4SHIFT] = 1;
01989                 }
01990             }
01991             else if ( *term == -VECTOR || *term == -MINVECTOR ) {
01992                 AN.UsedVector[term[1]-AM.OffsetVector] = 1;
01993             }
01994         }
01995         term = sarg; /* Next argument */
01996     }
01997     term = t;
01998 }
01999 }

```

```

02000     }
02001 }
02002
02003 /*
02004     #] DetVars :
02005     #[ ToStorage :
02006
02007     This routine takes an expression in the scratch buffer (indicated by e)
02008     and puts it in the storage file. The necessary actions are:
02009
02010     1: determine the list of the used variables.
02011     2: make an index entry.
02012     3: write the namelists.
02013     4: copy the 'length' bytes of the expression.
02014
02015 */
02016
02017 WORD ToStorage(EXPRESSIONS e, POSITION *length)
02018 {
02019     GETIDENTITY
02020     WORD *w, i, j;
02021     WORD *term;
02022     INDEXENTRY *indexent;
02023     LONG size;
02024     POSITION indexpos, scrpos;
02025     FILEHANDLE *f;
02026     if ( ( indexent = NextFileIndex(&indexpos) ) == 0 ) {
02027         MesCall("ToStorage");
02028         SETERROR(-1)
02029     }
02030     indexent->CompressSize = 0; /* thus far no compression */
02031     f = AR.infile; AR.infile = AR.outfile; AR.outfile = f;
02032     if ( e->status == HIDDENEXPRESSION ) {
02033         AR.InHiBuf = 0; f = AR.hidefile; AR.GetFile = 2;
02034     }
02035     else {
02036         AR.InInBuf = 0; f = AR.infile; AR.GetFile = 0;
02037     }
02038     if ( f->handle >= 0 ) {
02039         scrpos = e->onfile;
02040         SeekFile(f->handle, &scrpos, SEEK_SET);
02041         if ( ISNOTEQUALPOS(scrpos, e->onfile) ) {
02042             MesPrint(":::Error in Scratch file");
02043             goto ErrReturn;
02044         }
02045         f->POposition = e->onfile;
02046         f->POfull = f->PObuffer;
02047         if ( e->status == HIDDENEXPRESSION ) AR.InHiBuf = 0;
02048         else AR.InInBuf = 0;
02049     }
02050     else {
02051         f->POfill = (WORD *) ((UBYTE *) (f->PObuffer) + BASEPOSITION(e->onfile));
02052     }
02053     w = AT.WorkPointer;
02054     AN.UsedSymbol = w; w += NumSymbols;
02055     AN.UsedVector = w; w += NumVectors;
02056     AN.UsedIndex = w; w += NumIndices;
02057     AN.UsedFunction = w; w += NumFunctions;
02058     term = w;
02059     w = (WORD *) ((UBYTE *) (w)) + AM.MaxTer;
02060     if ( w > AT.WorkTop ) {
02061         MesWork();
02062         goto ErrReturn;
02063     }
02064     w = AN.UsedSymbol;
02065     i = NumSymbols + NumVectors + NumIndices + NumFunctions;
02066     do { *w++ = 0; } while ( --i > 0 );
02067     if ( GetTerm(BHEAD term) > 0 ) {
02068         DetVars(term, 1);
02069         if ( GetTerm(BHEAD term) ) {
02070             do { DetVars(term, 0); } while ( GetTerm(BHEAD term) > 0 );
02071         }
02072     }
02073     j = 0;
02074     w = AN.UsedSymbol;
02075     i = NumSymbols;
02076     while ( --i >= 0 ) { if ( *w++ ) j++; }
02077     indexent->nsymbols = j;
02078     /* size = j * sizeof(struct SyMbOl); */
02079     j = 0;
02080     w = AN.UsedIndex;
02081     i = NumIndices;
02082     while ( --i >= 0 ) { if ( *w++ ) j++; }
02083     indexent->nindices = j;
02084     /* size += j * sizeof(struct InDeX); */
02085     j = 0;
02086     w = AN.UsedVector;

```

```

02087     i = NumVectors;
02088     while ( --i >= 0 ) { if ( *w++ ) j++; }
02089     indexent->nvectors = j;
02090     /* size += j * sizeof(struct VeCtOr); */
02091     j = 0;
02092     w = AN.UsedFunction;
02093     i = NumFunctions;
02094     while ( --i >= 0 ) { if ( *w++ ) j++; }
02095     indexent->nfunctions = j;
02096     /* size += j * sizeof(struct FuNcTiOn); */
02097     indexent->length = *length;
02098     indexent->variables = AR.StoreData.Fill;
02099     /* indexent->position = AR.StoreData.Fill + size; */
02100     StrCopy(AC.exprnames->namebuffer+e->name, (UBYTE *) (indexent->name));
02101     SeekFile(AR.StoreData.Handle, &(AR.StoreData.Fill), SEEK_SET);
02102     AO.wlen = 100000;
02103     AO.wpos = (UBYTE *) Malloc1(AO.wlen, "AO.wpos buffer");
02104     AO.wpoin = AO.wpos;
02105     {
02106         SYMBOLS a;
02107         w = AN.UsedSymbol;
02108         a = symbols;
02109         j = 0;
02110         i = indexent->nsymbols;
02111         while ( --i >= 0 ) {
02112             while ( !*w ) { w++; a++; j++; }
02113             a->number = j;
02114             if ( VarStore((UBYTE *)a, (WORD) (sizeof(struct SyMbOl)), a->name,
02115                 a->namesize) ) goto ErrToSto;
02116             w++; j++; a++;
02117         }
02118     }
02119     {
02120         INDICES a;
02121         w = AN.UsedIndex;
02122         a = indices;
02123         j = 0;
02124         i = indexent->nindices;
02125         while ( --i >= 0 ) {
02126             while ( !*w ) { w++; a++; j++; }
02127             a->number = j;
02128             if ( VarStore((UBYTE *)a, (WORD) (sizeof(struct InDeX)), a->name,
02129                 a->namesize) ) goto ErrToSto;
02130             w++; j++; a++;
02131         }
02132     }
02133     {
02134         VECTORS a;
02135         w = AN.UsedVector;
02136         a = vectors;
02137         j = 0;
02138         i = indexent->nvectors;
02139         while ( --i >= 0 ) {
02140             while ( !*w ) { w++; a++; j++; }
02141             a->number = j;
02142             if ( VarStore((UBYTE *)a, (WORD) (sizeof(struct VeCtOr)), a->name,
02143                 a->namesize) ) goto ErrToSto;
02144             w++; j++; a++;
02145         }
02146     }
02147     {
02148         FUNCTIONS a;
02149         w = AN.UsedFunction;
02150         a = functions;
02151         j = 0;
02152         i = indexent->nfunctions;
02153         while ( --i >= 0 ) {
02154             while ( !*w ) { w++; a++; j++; }
02155             a->number = j;
02156             if ( VarStore((UBYTE *)a, (WORD) (sizeof(struct FuNcTiOn)), a->name,
02157                 a->namesize) ) goto ErrToSto;
02158             w++; a++; j++;
02159         }
02160     }
02161     if ( VarStore((UBYTE *)0L, (WORD)0, (WORD)0, (WORD)0) ) goto ErrToSto; /* Flush buffer */
02162     TELLFILE(AR.StoreData.Handle, &(indexent->position));
02163     indexent->size = (WORD) DIFBASE(indexent->position, indexent->variables);
02164     /*
02165         The following code was added when it became apparent (30-jan-2007)
02166         that we need provisions for extra space without upsetting existing
02167         .sav files. Here we can put as much as we want.
02168         Look in GetTable on how to recover numdummies.
02169         Forgetting numdummies has been in there from the beginning.
02170     */
02171     if ( WriteFile(AR.StoreData.Handle, (UBYTE *) (&(e->numdummies)), (LONG) sizeof(WORD)) !=
02172         sizeof(WORD) ) {
02173         MesPrint("Error while writing storage file");

```

```

02174         goto ErrReturn;
02175     }
02176     if ( WriteFile(AR.StoreData.Handle, (UBYTE *) (&(e->numfactors)), (LONG) sizeof(WORD)) !=
02177         sizeof(WORD) ) {
02178         MesPrint("Error while writing storage file");
02179         goto ErrReturn;
02180     }
02181     if ( WriteFile(AR.StoreData.Handle, (UBYTE *) (&(e->vflags)), (LONG) sizeof(WORD)) !=
02182         sizeof(WORD) ) {
02183         MesPrint("Error while writing storage file");
02184         goto ErrReturn;
02185     }
02186     if ( WriteFile(AR.StoreData.Handle, (UBYTE *) (&(e->uflags)), (LONG) sizeof(WORD)) !=
02187         sizeof(WORD) ) {
02188         MesPrint("Error while writing storage file");
02189         goto ErrReturn;
02190     }
02191     TELLFILE(AR.StoreData.Handle, &(indexent->position));
02192     if ( f->handle >= 0 ) {
02193         POSITION llength;
02194         llength = *length;
02195         SeekFile(f->handle, &(e->onfile), SEEK_SET);
02196         while ( ISPOSPOS(llength) ) {
02197             SETBASEPOSITION(scrpos, AO.wlen);
02198             if ( ISLESSPOS(llength, scrpos) ) size = BASEPOSITION(llength);
02199             else size = AO.wlen;
02200             if ( ReadFile(f->handle, AO.wpos, size) != size ) {
02201                 MesPrint("Error while reading scratch file");
02202                 goto ErrReturn;
02203             }
02204             if ( WriteFile(AR.StoreData.Handle, AO.wpos, size) != size ) {
02205                 MesPrint("Error while writing storage file");
02206                 goto ErrReturn;
02207             }
02208             ADDPOS(llength, -size);
02209         }
02210     }
02211     else {
02212         WORD *ppp;
02213         ppp = (WORD *) ((UBYTE *) (f->PObuffer) + BASEPOSITION(e->onfile));
02214         if ( WriteFile(AR.StoreData.Handle, (UBYTE *) ppp, BASEPOSITION(*length)) !=
02215             BASEPOSITION(*length) ) {
02216             MesPrint("Error while writing storage file");
02217             goto ErrReturn;
02218         }
02219     }
02220     ADD2POS(*length, indexent->position);
02221     e->onfile = indexpos;
02222     /*
02223     AR.StoreData.Fill = SeekFile(AR.StoreData.Handle, &(AM.zeropos), SEEK_END);
02224     */
02225     AR.StoreData.Fill = *length;
02226     SeekFile(AR.StoreData.Handle, &(AR.StoreData.Fill), SEEK_SET);
02227     scrpos = AR.StoreData.Position;
02228     ADDPOS(scrpos, sizeof(POSITION));
02229     SeekFile(AR.StoreData.Handle, &scrpos, SEEK_SET);
02230     if ( WriteFile(AR.StoreData.Handle, ((UBYTE *) &(AR.StoreData.Index.number))
02231         , (LONG) (sizeof(POSITION))) != sizeof(POSITION) ) goto ErrInSto;
02232     SeekFile(AR.StoreData.Handle, &indexpos, SEEK_SET);
02233     if ( WriteFile(AR.StoreData.Handle, (UBYTE *) indexent, (LONG) (sizeof(INDEXENTRY)) !=
02234         sizeof(INDEXENTRY) ) goto ErrInSto;
02235     FlushFile(AR.StoreData.Handle);
02236     SeekFile(AR.StoreData.Handle, &(AC.StoreFileSize), SEEK_END);
02237     f = AR.infile; AR.infile = AR.outfile; AR.outfile = f;
02238     if ( AO.wpos ) M_free(AO.wpos, "AO.wpos buffer");
02239     AO.wpos = AO.wpoin = 0;
02240     return(0);
02241 ErrToSto:
02242     MesPrint("---Error while storing namelists");
02243     goto ErrReturn;
02244 ErrInSto:
02245     MesPrint("Error in storage");
02246 ErrReturn:
02247     if ( AO.wpos ) M_free(AO.wpos, "AO.wpos buffer");
02248     AO.wpos = AO.wpoin = 0;
02249     f = AR.infile; AR.infile = AR.outfile; AR.outfile = f;
02250     return(-1);
02251 }
02252
02253 /*
02254     #] ToStorage :
02255     #[ NextFileIndex :
02256 */
02257
02258 INDEXENTRY *NextFileIndex(POSITION *indexpos)
02259 {
02260     GETIDENTITY

```

```

02261     INDEXENTRY *ind;
02262     int i, j = sizeof(FILEINDEX)/(sizeof(LONG));
02263     LONG *lo;
02264     if ( AR.StoreData.Handle <= 0 ) {
02265         if ( SetFileIndex() ) {
02266             MesCall("NextFileIndex");
02267             return(0);
02268         }
02269         SETBASEPOSITION(AR.StoreData.Index.number,1);
02270 #ifdef SYSDEPENDENTSAVE
02271         SETBASEPOSITION(*indexpos, (2*sizeof(POSITION)));
02272 #else
02273         SETBASEPOSITION(*indexpos, (2*sizeof(POSITION)+sizeof(STOREHEADER)));
02274 #endif
02275         return(AR.StoreData.Index.expression);
02276     }
02277     while ( BASEPOSITION(AR.StoreData.Index.number) >= (LONG)(INFILEINDEX) ) {
02278         if ( ISNOTZEROPOS(AR.StoreData.Index.next) ) {
02279             SeekFile(AR.StoreData.Handle, &(AR.StoreData.Index.next), SEEK_SET);
02280             AR.StoreData.Position = AR.StoreData.Index.next;
02281             if ( ReadFile(AR.StoreData.Handle, (UBYTE
*)(&AR.StoreData.Index), (LONG)(sizeof(FILEINDEX))) !=
(LONG)(sizeof(FILEINDEX)) ) goto ErrNextS;
02282         }
02283         else {
02284             PUTZERO(AR.StoreData.Index.number);
02285             SeekFile(AR.StoreData.Handle, &(AR.StoreData.Position), SEEK_SET);
02286             if ( WriteFile(AR.StoreData.Handle, (UBYTE
*)(&AR.StoreData.Fill)), (LONG)(sizeof(POSITION)))
!= (LONG)(sizeof(POSITION)) ) goto ErrNextS;
02287             PUTZERO(AR.StoreData.Index.next);
02288             SeekFile(AR.StoreData.Handle, &(AR.StoreData.Fill), SEEK_SET);
02289             AR.StoreData.Position = AR.StoreData.Fill;
02290             lo = (LONG *)(&AR.StoreData.Index);
02291             for ( i = 0; i < j; i++ ) *lo++ = 0;
02292             if ( WriteFile(AR.StoreData.Handle, (UBYTE
*)(&AR.StoreData.Index), (LONG)(sizeof(FILEINDEX))) !=
(LONG)(sizeof(FILEINDEX)) ) goto ErrNextS;
02293             ADDPOS(AR.StoreData.Fill, sizeof(FILEINDEX));
02294         }
02295     }
02296     *indexpos = AR.StoreData.Position;
02297     ADDPOS(*indexpos, (2*sizeof(POSITION)) +
BASEPOSITION(AR.StoreData.Index.number) * sizeof(INDEXENTRY));
02300     ind = &AR.StoreData.Index.expression[BASEPOSITION(AR.StoreData.Index.number)];
02301     ADDPOS(AR.StoreData.Index.number, 1);
02302     return(ind);
02303 ErrNextS:
02304     MesPrint("Error in storage file");
02305     return(0);
02306 }
02307
02308 /*
02309 #] NextFileIndex :
02310 #[ SetFileIndex :
02311 */
02312
02313 WORD SetFileIndex(void)
02314 {
02315     GETIDENTITY
02316     int i, j = sizeof(FILEINDEX)/(sizeof(LONG));
02317     LONG *lo;
02318     if ( AR.StoreData.Handle < 0 ) {
02319         AR.StoreData.Handle = AC.StoreHandle;
02320         PUTZERO(AR.StoreData.Index.next);
02321         PUTZERO(AR.StoreData.Index.number);
02322 #ifdef SYSDEPENDENTSAVE
02323         SETBASEPOSITION(AR.StoreData.Fill, sizeof(FILEINDEX));
02324 #else
02325         if ( WriteStoreHeader(AR.StoreData.Handle) ) return(MesPrint("Error writing storage file
header"));
02326         SETBASEPOSITION(AR.StoreData.Fill, (LONG)sizeof(FILEINDEX)+(LONG)sizeof(STOREHEADER));
02327 #endif
02328         lo = (LONG *)(&AR.StoreData.Index);
02329         for ( i = 0; i < j; i++ ) *lo++ = 0;
02330         if ( WriteFile(AR.StoreData.Handle, (UBYTE *)(&AR.StoreData.Index), (LONG)(sizeof(FILEINDEX)))
!=
(LONG)(sizeof(FILEINDEX)) ) return(MesPrint("Error writing storage file"));
02331     }
02332     else {
02333         POSITION scrpos;
02334 #ifdef SYSDEPENDENTSAVE
02335         PUTZERO(scrpos);
02336 #else
02337         SETBASEPOSITION(scrpos, (LONG)(sizeof(STOREHEADER)));
02338 #endif
02339         SeekFile(AR.StoreData.Handle, &scrpos, SEEK_SET);

```

```

02349         if ( ReadFile(AR.StoreData.Handle, (UBYTE *)(&AR.StoreData.Index), (LONG)(sizeof(FILEINDEX))) !=
02350             (LONG)(sizeof(FILEINDEX)) ) return(MesPrint("Error reading storage file"));
02351     }
02352 #ifdef SYSDEPENDENTSAVE
02353     PUTZERO(AR.StoreData.Position);
02354 #else
02355     SETBASEPOSITION(AR.StoreData.Position, (LONG)(sizeof(STOREHEADER)));
02356 #endif
02357     return(0);
02358 }
02359
02360 /*
02361     #] SetFileIndex :
02362     #[ VarStore :
02363
02364     The n -= sizeof(WORD); makes that the real length comes in the
02365     padding space, provided there is padding space (it seems so).
02366     The reading of the information assumes this is the case and hence
02367     things work....
02368 */
02369
02370 WORD VarStore(UBYTE *s, WORD n, WORD name, WORD namesize)
02371 {
02372     GETIDENTITY
02373     UBYTE *t, *u;
02374     if ( s ) {
02375         n -= sizeof(WORD);
02376         t = (UBYTE *)AO.wpoin;
02377     /*
02378         u = (UBYTE *)AT.WorkTop;
02379     */
02380         u = AO.wpos+AO.wlen;
02381         while ( n > 0 && t < u ) { *t++ = *s++; n--; }
02382         while ( t >= u ) {
02383             if ( WriteFile(AR.StoreData.Handle, AO.wpos, AO.wlen) != AO.wlen ) return(-1);
02384             t = AO.wpos;
02385             while ( n > 0 && t < u ) { *t++ = *s++; n--; }
02386         }
02387         s = AC.varnames->namebuffer + name;
02388         n = namesize;
02389         n += sizeof(void *)-1; n &= -(sizeof(void *));
02390         *((WORD *)t) = n;
02391         t += sizeof(WORD);
02392         while ( n > 0 && t < u ) {
02393             if ( namesize > 0 ) { *t++ = *s++; namesize--; }
02394             else { *t++ = 0; }
02395             n--;
02396         }
02397         while ( t >= u ) {
02398             if ( WriteFile(AR.StoreData.Handle, AO.wpos, AO.wlen) != AO.wlen ) return(-1);
02399             t = AO.wpos;
02400             while ( n > 0 && t < u ) {
02401                 if ( namesize > 0 ) { *t++ = *s++; namesize--; }
02402                 else { *t++ = 0; }
02403                 n--;
02404             }
02405         }
02406         AO.wpoin = t;
02407     }
02408     else {
02409         LONG size;
02410         size = AO.wpoin - AO.wpos;
02411         if ( WriteFile(AR.StoreData.Handle, AO.wpos, size) != size ) return(-1);
02412         AO.wpoin = AO.wpos;
02413     }
02414     return(0);
02415 }
02416
02417 /*
02418     #] VarStore :
02419     #[ TermRenumber :
02420
02421     rennumbers the variables inside term according to the information
02422     in struct renumber.
02423     The search is binary. This avoided having to read/write the
02424     expression twice when it was stored.
02425 */
02426
02427
02428 WORD TermRenumber(WORD *term, RENUMBER renumber, WORD nexpr)
02429 {
02430     WORD *stopper;
02431     WORD *t, *sarg, n;
02432     stopper = term + *term - 1;
02433     stopper = stopper - ABS(*stopper) + 1;
02434     term++;
02435     while ( term < stopper ) {

```

```

02443     if ( *term == SYMBOL ) {
02444         t = term + term[1];
02445         term += 2;
02446         do {
02447             if ( ( n = FindrNumber(*term,&(renumber->syml)) ) < 0 ) goto ErrR;
02448             *term = renumber->symnum[n];
02449             term += 2;
02450         } while ( term < t );
02451     }
02452     else if ( *term == DOTPRODUCT ) {
02453         t = term + term[1];
02454         term += 2;
02455         do {
02456             if ( ( n = FindrNumber(*term,&(renumber->vect)) )
02457                 < 0 ) goto ErrR;
02458             *term++ = renumber->vecnum[n];
02459             if ( ( n = FindrNumber(*term,&(renumber->vect)) )
02460                 < 0 ) goto ErrR;
02461             *term = renumber->vecnum[n];
02462             term += 2;
02463         } while ( term < t );
02464     }
02465     else if ( *term == VECTOR ) {
02466         t = term + term[1];
02467         term += 2;
02468         do {
02469             if ( ( n = FindrNumber(*term,&(renumber->vect)) )
02470                 < 0 ) goto ErrR;
02471             *term++ = renumber->vecnum[n];
02472             if ( ( *term >= AM.OffsetIndex ) && ( *term < AM.IndDum ) ) {
02473                 if ( ( n = FindrNumber(*term,&(renumber->indi)) )
02474                     < 0 ) goto ErrR;
02475                 *term++ = renumber->indnum[n];
02476             }
02477             else term++;
02478         } while ( term < t );
02479     }
02480     else if ( *term == INDEX || *term == LEVICIVITA || *term == GAMMA
02481 || *term == DELTA ) {
02482 Tensors:
02483         t = term + term[1];
02484         if ( *term == INDEX || *term == DELTA ) term += 2;
02485         else term += FUNHEAD;
02486 /*
02487         term += 2;
02488 */
02489         while ( term < t ) {
02490             if ( *term >= AM.OffsetIndex + WILDOFFSET ) {
02491 /*
02492             Still TOBEDONE
02493             */
02494                 }
02495                 else if ( ( *term >= AM.OffsetIndex ) && ( *term < AM.IndDum ) ) {
02496                     if ( ( n = FindrNumber(*term,&(renumber->indi)) )
02497                         < 0 ) goto ErrR;
02498                     *term = renumber->indnum[n];
02499                 }
02500                 else if ( *term < (WILDOFFSET+AM.OffsetVector) ) {
02501                     if ( ( n = FindrNumber(*term,&(renumber->vect)) )
02502                         < 0 ) goto ErrR;
02503                     *term = renumber->vecnum[n];
02504                 }
02505                 term++;
02506             }
02507         }
02508     else if ( *term == HAAKJE ) term += term[1];
02509     else {
02510         if ( *term > MAXBUILTINFUNCTION ) {
02511             if ( ( n = FindrNumber(*term,&(renumber->func)) )
02512                 < 0 ) goto ErrR;
02513             *term = renumber->funnum[n];
02514         }
02515         if ( *term >= FUNCTION && functions[*term-FUNCTION].spec
02516             >= TENSORFUNCTION && term[1] > FUNHEAD ) goto Tensors;
02517         t = term + term[1]; /* General stopper */
02518         term += FUNHEAD; /* First argument */
02519         while ( term < t ) {
02520             sarg = term;
02521             NEXTARG(sarg)
02522             if ( *term > 0 ) {
02523 /*
02524                 Problem here:
02525                 Marking the argument as dirty attacks the heap
02526                 very heavily and costs much computer time.
02527             */
02528                 *term = 1;
02529                 term += ARGHEAD-1;

```



```

02530         while ( term < sarg ) {
02531             if ( TermRenumber(term,renumber,nexpr) ) goto ErrR;
02532             term += *term;
02533         }
02534     }
02535     else {
02536         if ( *term <= -MAXBUILTINFUNCTION ) {
02537             if ( ( n = FindrNumber(-*term,&(renumber->func)) )
02538                 < 0 ) goto ErrR;
02539             *term = -renumber->funnum[n];
02540         }
02541         else if ( *term == -SYMBOL ) {
02542             term++;
02543             if ( ( n = FindrNumber(*term,
02544                 &(renumber->symb)) ) < 0 ) goto ErrR;
02545             *term = renumber->symnum[n];
02546         }
02547         else if ( *term == -INDEX ) {
02548             term++;
02549             if ( *term >= AM.OffsetIndex + WILDOFFSET ) {
02550                 /*
02551                 Still TOBEDONE
02552                 */
02553             }
02554             else if ( ( *term >= AM.OffsetIndex ) && ( *term < AM.IndDum ) ) {
02555                 if ( ( n = FindrNumber(*term,&(renumber->indi)) )
02556                     < 0 ) goto ErrR;
02557                 *term = renumber->indnum[n];
02558             }
02559             else if ( *term < (WILDOFFSET+AM.OffsetVector) ) {
02560                 if ( ( n = FindrNumber(*term,&(renumber->vect)) )
02561                     < 0 ) goto ErrR;
02562                 *term = renumber->vecnum[n];
02563             }
02564         }
02565         else if ( *term == -VECTOR || *term == -MINVECTOR ) {
02566             term++;
02567             if ( ( n = FindrNumber(*term,&(renumber->vect)) )
02568                 < 0 ) goto ErrR;
02569             *term = renumber->vecnum[n];
02570         }
02571     }
02572     term = sarg;          /* Next argument */
02573 }
02574 term = t;
02575 }
02576 }
02577 return(0);
02578 ErrR:
02579     MesCall("TermRenumber");
02580     SETERROR(-1)
02581 }
02582
02583 /*
02584     #] TermRenumber :
02585     #[ FindrNumber :
02586 */
02587
02588 WORD FindrNumber(WORD n, VARRENUM *v)
02589 {
02590     WORD *hi,*med,*lo;
02591     hi = v->hi;
02592     lo = v->lo;
02593     med = v->start;
02594     if ( *hi == 0 ) {
02595         if ( n != *hi ) {
02596             MesPrint("Serious problems coming up in FindrNumber");
02597             return(-1);
02598         }
02599         return(*hi);
02600     }
02601     while ( *med != n ) {
02602         if ( *med < n ) {
02603             if ( med == hi ) goto ErrFindr;
02604             lo = med;
02605             med = hi - ((WORDDIF(hi,med))/2);
02606         }
02607         else {
02608             if ( med == lo ) goto ErrFindr;
02609             hi = med;
02610             med = lo + ((WORDDIF(med,lo))/2);
02611         }
02612     }
02613     return(WORDDIF(med,v->lo));
02614 ErrFindr:
02615     /*
02616     Reconstruction:

```

```

02617 */
02618 {
02619     int i;
02620     i = WORDDIF(v->hi,v->lo);
02621     MesPrint("FindrNumber: n = %d, list has %d members",n,i);
02622     while ( i >= 0 ) {
02623         MesPrint("v->lo[%d] = %d",i,v->lo[i]); i--;
02624     }
02625     hi = v->hi;
02626     lo = v->lo;
02627     med = v->start;
02628     MesPrint("Start with %d,%d,%d",0,WORDDIF(med,v->lo),WORDDIF(hi,v->lo));
02629     while ( *med != n ) {
02630         if ( *med < n ) {
02631             if ( med == hi ) goto ErrFindr2;
02632             lo = med;
02633             med = hi - ((WORDDIF(hi,med))/2);
02634         }
02635         else {
02636             if ( med == lo ) goto ErrFindr2;
02637             hi = med;
02638             med = ((WORDDIF(med,lo))/2) + lo;
02639         }
02640         MesPrint("New: %d,%d,%d, *med = %d",WORDDIF(lo,v->lo),WORDDIF(med,v->lo),WORDDIF(hi,v->lo),*med);
02641     }
02642 }
02643 return(WORDDIF(med,v->lo));
02644 ErrFindr2:
02645     return(MesPrint("Renumbering problems"));
02646 }
02647
02648 /*
02649     #] FindrNumber :
02650     #[ FindInIndex :
02651
02652     Finds an expression in the storage index if it exists.
02653     If found it returns a pointer to the index entry, otherwise zero.
02654     par = 0      Search by address (--> f == &AR.StoreData, called by GetTable, CoSave )
02655     par = 1      Search by name      (--> f == &AO.SaveData, called by CoLoad )
02656
02657     When comparing parameter fields the parameters of the expression
02658     to be searched are in AT.TMaddr. This includes the primary expression
02659     and a possible FROMBRAC information. The FROMBRAC is always last.
02660
02661     The parameter mode tells whether we should worry about arguments of
02662     a stored expression.
02663 */
02664
02665 INDEXENTRY *FindInIndex(WORD expr, FILEDATA *f, WORD par, WORD mode)
02666 {
02667     GETIDENTITY
02668     INDEXENTRY *ind;
02669     WORD i, hand, *m;
02670     WORD *start, *stop, *stop2, *m2, nomatch = 0;
02671     POSITION stindex, indexpos, scrpos;
02672     LONG number, num;
02673     stindex = f->Position;
02674     m = AT.TMaddr;
02675     stop = m + m[1];
02676     m += SUBEXPSIZE;
02677     start = m;
02678     while ( m < stop ) {
02679         if ( *m == FROMBRAC || *m == WILDCARDS ) break;
02680         m += m[1];
02681     }
02682     stop = m;
02683     if ( !par ) hand = AR.StoreData.Handle;
02684     else hand = AO.SaveData.Handle;
02685     for(;;) {
02686         if ( ( i = (WORD)BASEPOSITION(f->Index.number) ) != 0 ) {
02687             indexpos = f->Position;
02688             ADDPOS(indexpos,(2*sizeof(POSITION)));
02689             ind = f->Index.expression;
02690             do {
02691                 if ( ( !par && ISEQUALPOS(indexpos,Expressions[expr].onfile) )
02692                     || ( par && !StrCmp(EXPRNAME(expr),(UBYTE *) (ind->name)) ) ) {
02693                     nomatch = 1;
02694                     /*
02695                     MesPrint("index: position: %8p",&(ind->position));
02696                     MesPrint("index: length: %8p",&(ind->length));
02697                     MesPrint("index: variables: %8p",&(ind->variables));
02698                     MesPrint("index: nsymbols: %d",ind->nsymbols);
02699                     MesPrint("index: nindices: %d",ind->nindices);
02700                     MesPrint("index: nvectors: %d",ind->nvectors);
02701                     MesPrint("index: nfunctions: %d",ind->nfunctions);
02702                     MesPrint("index: size: %d",ind->size);

```

```

02703 */
02704         if ( par ) return(ind);
02705         scrpos = ind->position;
02706         SeekFile(hand,&scrpos,SEEK_SET);
02707         if ( ISNOTEQUALPOS(scrpos,ind->position) ) goto ErrGt2;
02708         if ( ReadFile(hand,(UBYTE *)AT.WorkPointer,(LONG)sizeof(WORD)) !=
02709             sizeof(WORD) || !*AT.WorkPointer ) goto ErrGt2;
02710         num = *AT.WorkPointer - 1;
02711         num *= sizeof(WORD);
02712         if ( *AT.WorkPointer < 0 ||
02713             ReadFile(hand,(UBYTE *) (AT.WorkPointer+1),num) != num ) goto ErrGt2;
02714         m = start; /* start of parameter field to be searched */
02715         m2 = AT.WorkPointer + 1;
02716         stop2 = m2 + m2[1];
02717         m2 += SUBEXPSIZE;
02718         while ( m < stop && m2 < stop2 ) {
02719             if ( *m == SYMBOL ) {
02720                 if ( *m2 != SYMTOSYM ) break;
02721                 m2[3] = m[2];
02722             }
02723             else if ( *m == INDEX ) {
02724                 if ( m[2] >= 0 ) {
02725                     if ( *m2 != INDTOIND ) break;
02726                 }
02727                 else {
02728                     if ( *m2 != VECTOVEC ) break;
02729                 }
02730                 m2[3] = m[2];
02731             }
02732             else if ( *m >= FUNCTION ) {
02733                 if ( *m2 != FUNTOFUN ) break;
02734                 m2[3] = *m;
02735             }
02736             else {}
02737             m += m[1];
02738             m2 += m2[1];
02739         }
02740         if ( ( m >= stop && m2 >= stop2 ) || mode == 0 ) {
02741             AT.WorkPointer = stop2;
02742             return(ind);
02743         }
02744     }
02745     }
02746     ind++;
02747     ADDPOS(indexpos,sizeof(INDEXENTRY));
02748     } while ( --i > 0 );
02749 }
02750 f->Position = f->Index.next;
02751 #ifndef SYSDEPENDENTSAVE
02752     if ( !ISNOTZEROPOS(f->Position) ) ADDPOS(f->Position,sizeof(STOREHEADER));
02753     number = sizeof(struct FileInDeX);
02754 #endif
02755     if ( ISEQUALPOS(f->Position,stindex) && !AO.bufferedInd ) goto ErrGetTab;
02756     if ( !par ) {
02757         SeekFile(AR.StoreData.Handle,&(f->Position),SEEK_SET);
02758         if ( ISNOTEQUALPOS(f->Position,AR.StoreData.Position) ) goto ErrGt2;
02759 #ifndef SYSDEPENDENTSAVE
02760         if ( ReadFile(f->Handle, (UBYTE *) (&(f->Index)), number) != number ) goto ErrGt2;
02761 #endif
02762     }
02763     else {
02764         SeekFile(AO.SaveData.Handle,&(f->Position),SEEK_SET);
02765         if ( ISNOTEQUALPOS(f->Position,AO.SaveData.Position) ) goto ErrGt2;
02766 #ifndef SYSDEPENDENTSAVE
02767         if ( ReadSaveIndex(&f->Index) ) goto ErrGt2;
02768 #endif
02769     }
02770 #ifndef SYSDEPENDENTSAVE
02771     number = sizeof(struct FileInDeX);
02772     if ( ReadFile(f->Handle,(UBYTE *) (&(f->Index)),number) !=
02773         number ) goto ErrGt2;
02774 #endif
02775 }
02776 ErrGetTab:
02777     if ( nomatch ) {
02778         MesPrint("Parameters of expression %s don't match."
02779             ,EXPRNAME(expr));
02780     }
02781     else {
02782         MesPrint("Cannot find expression %s",EXPRNAME(expr));
02783     }
02784     return(0);
02785 ErrGt2:
02786     MesPrint("Readerror in IndexSearch");
02787     return(0);
02788 }
02789

```

```

02790 /*
02791     #] FindInIndex :
02792     #[ GetTable :
02793
02794     Locates stored files and constructs the renumbering tables.
02795     They are allocated in the Workspace.
02796     First the expression data are located. The Index is treated
02797     as a circularly linked buffer which is paged forwardly.
02798     If the indexentry is located (in ind) the two renumber tables
02799     have to be constructed.
02800     Finally the prototype has to be put in the proper buffer, so
02801     that wildcards can be passed. There should be a test with
02802     an already existing prototype that is constructed by the
02803     pattern matcher. This has not been put in yet.
02804
02805     There is a problem with the parallel processing.
02806     Feeding in the variables that were erased by a .store could in
02807     principle happen in different orders (ParFORM) or simultaneously
02808     (TFORM). The proper resolution is to have the compiler call GetTable
02809     when a stored expression is encountered.
02810
02811     This has been mended in development of TFORM by reading the
02812     symbol tables during compilation. See the call to GetTable
02813     in the CodeGenerator.
02814
02815     Next is the problem of FindInIndex which writes in AR.StoreData
02816     Copying this is expensive!
02817
02818     This Doesn't work well for TFORM yet!!!!!!!
02819     e[x1,x2] versus e[x2,x1] messes up.
02820     For the rest is the reloading during execution not thread safe.
02821
02822     The parameter mode tells whether we should worry about arguments
02823     of a stored expression.
02824 */
02825
02826 RENUMBER GetTable(WORD expr, POSITION *position, WORD mode)
02827 {
02828     GETIDENTITY
02829     WORD i, j;
02830     WORD *w;
02831     RENUMBER r;
02832     LONG num, nsize, xx;
02833     WORD jsym, jind, jvec, jfun;
02834     WORD k, type, error = 0, *oldw, *neww, *oldwork = AT.WorkPointer;
02835     struct SyMbOl SyM;
02836     struct InDeX InD;
02837     struct VeCtOr VeC;
02838     struct FuNcTiOn FuN;
02839     INDEXENTRY *ind;
02840 /*
02841     Prepare for FindInIndex to put the prototype in the Workspace.
02842     oldw will point at the "wildcards"
02843 */
02844 /*
02845     Bug fix. Look also in Generator.
02846 #ifndef WITHPTHREADS
02847
02848     if ( ( r = Expressions[expr].renum ) != 0 ) { }
02849     else {
02850         Expressions[expr].renum =
02851         r = (RENUMBER)Mallocl(sizeof(struct ReNuMbEr), "Renum");
02852     }
02853 #else
02854     r = (RENUMBER)Mallocl(sizeof(struct ReNuMbEr), "Renum");
02855 #endif
02856 */
02857     r = (RENUMBER)Mallocl(sizeof(struct ReNuMbEr), "Renum");
02858
02859     oldw = AT.WorkPointer + 1 + SUBEXPSIZE;
02860 /*
02861     The prototype is loaded in the Workspace by the Index routine.
02862     After all it has to find an occurrence with the proper arguments.
02863     This sets the WorkPointer. Hence be careful now.
02864 */
02865     LOCK(AM.storefilelock);
02866     if ( ( ind = FindInIndex(expr, &AR.StoreData, 0, mode) ) == 0 ) {
02867         UNLOCK(AM.storefilelock);
02868         return(0);
02869     }
02870
02871     xx = ind->nsymbols+ind->nindices+ind->nvectors+ind->nfunctions;
02872     if ( xx == 0 ) {
02873         Expressions[expr].renumlists =
02874         w = AN.dummyrenumlist;
02875     }
02876     else {

```

```

02877 /*
02878 #ifndef WITHPTHREADS
02879     Expressions[expr].renumlists =
02880 #endif
02881 */
02882     w = (WORD *)Malloc1(sizeof(WORD)*(xx*2),"VarSpace");
02883 }
02884 r->symb.lo = w;
02885 r->symb.start = w + ind->nsymbols/2;
02886 w += ind->nsymbols;
02887 r->symb.hi = w - 1;
02888 r->symnum = w;
02889 w += ind->nsymbols;
02890
02891 r->indi.lo = w;
02892 r->indi.start = w + ind->nindices/2;
02893 w += ind->nindices;
02894 r->indi.hi = w - 1;
02895 r->indnum = w;
02896 w += ind->nindices;
02897
02898 r->vect.lo = w;
02899 r->vect.start = w + ind->nvectors/2;
02900 w += ind->nvectors;
02901 r->vect.hi = w - 1;
02902 r->vecnum = w;
02903 w += ind->nvectors;
02904
02905 r->func.lo = w;
02906 r->func.start = w + ind->nfunctions/2;
02907 w += ind->nfunctions;
02908 r->func.hi = w - 1;
02909 r->funnum = w;
02910 /* w += ind->nfunctions; */
02911
02912 SeekFile(AR.StoreData.Handle,&(ind->variables),SEEK_SET);
02913 *position = ind->position;
02914 jsym = ind->nsymbols;
02915 jvec = ind->nvectors;
02916 jind = ind->nindices;
02917 jfun = ind->nfunctions;
02918 /*
02919     # Symbols :
02920 */
02921 {
02922     SYMBOLS s = &Sym;
02923     w = r->symb.lo; j = jsym;
02924     for ( i = 0; i < j; i++ ) {
02925         if ( ReadFile(AR.StoreData.Handle,(UBYTE *)s,(LONG)(sizeof(struct Symbol)))
02926             != sizeof(struct Symbol) ) goto ErrGt2;
02927         nsize = s->namesize; nsize += sizeof(void *)-1;
02928         nsize &= -sizeof(void *);
02929         if ( ReadFile(AR.StoreData.Handle,(UBYTE *) (AT.WorkPointer),nsize)
02930             != nsize ) goto ErrGt2;
02931         *w = s->number;
02932         if ( ( s->flags & INUSE ) != 0 ) {
02933             /* Find the replacement. It must exist! */
02934             neww = oldw;
02935             while ( *neww != SYMTOSYM || neww[2] != *w ) neww += neww[1];
02936             k = neww[3];
02937         }
02938         else if ( GetVar((UBYTE *)AT.WorkPointer,&type,&k,ALLVARIABLES,NOAUTO) != NAMENOTFOUND ) {
02939             if ( type != CSYMBOL ) {
02940                 MesPrint("Error: Conflicting types for %s", (AT.WorkPointer));
02941                 error = -1;
02942             }
02943             else {
02944                 if ( ( s->complex & (VARTYPEIMAGINARY|VARTYPECOMPLEX) ) !=
02945                     ( symbols[k].complex & (VARTYPEIMAGINARY|VARTYPECOMPLEX) ) ) {
02946                     MesPrint("Warning: Conflicting complexity for %s",AT.WorkPointer);
02947                     error = -1;
02948                 }
02949                 if ( ( s->complex & (VARTYPEROOTOFUNITY) ) !=
02950                     ( symbols[k].complex & (VARTYPEROOTOFUNITY) ) ) {
02951                     MesPrint("Warning: Conflicting root of unity properties for %s",AT.WorkPointer);
02952                     error = -1;
02953                 }
02954                 if ( ( s->complex & VARTYPEROOTOFUNITY ) == VARTYPEROOTOFUNITY ) {
02955                     if ( s->maxpower != symbols[k].maxpower ) {
02956                         MesPrint("Warning: Conflicting n in n-th root of unity properties for
02957 %s",AT.WorkPointer);
02958                         error = -1;
02959                     }
02960                 }
02961                 else if ( ( s->minpower !=
02962                     symbols[k].minpower || s->maxpower !=
02963                     symbols[k].maxpower ) && AC.WarnFlag ) {

```

```

02963         MesPrint("Warning: Conflicting power restrictions for %s",AT.WorkPointer);
02964     }
02965 }
02966 }
02967 else {
02968     if ( ( k = EntVar(CSYMBOL, (UBYTE *) (AT.WorkPointer), s->complex, s->minpower,
02969         s->maxpower, s->dimension) ) < 0 ) goto GetTcall;
02970 }
02971 *(w+j) = k;
02972 w++;
02973 }
02974 }
02975 /*
02976     #] Symbols :
02977     #[ Indices :
02978 */
02979 {
02980     INDICES s = &InD;
02981     w = r->indi.lo; j = jind;
02982     for ( i = 0; i < j; i++ ) {
02983         if ( ReadFile(AR.StoreData.Handle, (UBYTE *)s, (LONG) (sizeof(struct InDeX))
02984             != sizeof(struct InDeX) ) goto ErrGt2;
02985         nsize = s->namesize; nsize += sizeof(void *)-1;
02986         nsize &= -sizeof(void *);
02987         if ( ReadFile(AR.StoreData.Handle, (UBYTE *) (AT.WorkPointer), nsize)
02988             != nsize ) goto ErrGt2;
02989         *w = s->number + AM.OffsetIndex;
02990         if ( s->dimension < 0 ) { /* Relabel the dimension */
02991             s->dimension = -r->symnum[FindrNumber(-s->dimension, &(r->symb))];
02992             if ( s->nmin4 < -NMIN4SHIFT ) { /* Relabel n-4 */
02993                 s->nmin4 = -r->symnum[FindrNumber(-s->nmin4-NMIN4SHIFT
02994                     , &(r->symb))]-NMIN4SHIFT;
02995             }
02996         }
02997         if ( ( s->flags & INUSE ) != 0 ) {
02998             /* Find the replacement. It must exist! */
02999             neww = oldw;
03000             while ( *neww != INDTOIND || neww[2] != *w ) neww += neww[1];
03001             k = neww[3] - AM.OffsetIndex;
03002         }
03003         else if ( s->type == DUMMY ) {
03004             /*
03005             -----> Here we may have to execute some renumbering
03006             */
03007         }
03008         else if ( GetVar((UBYTE *) (AT.WorkPointer), &type, &k, ALLVARIABLES, NOAUTO) != NAMENOTFOUND ) {
03009             if ( type != CINDEX ) {
03010                 MesPrint("Error: Conflicting types for %s", (AT.WorkPointer));
03011                 error = -1;
03012             }
03013             else {
03014                 if ( s->type !=
03015                     indices[k].type ) {
03016                     MesPrint("Warning: %s is also a dummy index", (AT.WorkPointer));
03017                     error = -1;
03018                     goto GetTb3;
03019                 }
03020                 if ( s->dimension != indices[k].dimension ) {
03021                     MesPrint("Warning: Conflicting dimensions for %s", (AT.WorkPointer));
03022                     error = -1;
03023                 }
03024             }
03025         }
03026         else {
03027             GetTb3:
03028                 if ( ( k = EntVar(CINDEX, (UBYTE *) (AT.WorkPointer),
03029                     s->dimension, 0, s->nmin4, 0) ) < 0 ) goto GetTcall;
03030         }
03031         *(w+j) = k + AM.OffsetIndex;
03032         w++;
03033     }
03034 }
03035 }
03036 /*
03037     #] Indices :
03038     #[ Vectors :
03039 */
03040 {
03041     VECTORS s = &VeC;
03042     w = r->vect.lo; j = jvec;
03043     for ( i = 0; i < j; i++ ) {
03044         if ( ReadFile(AR.StoreData.Handle, (UBYTE *)s, (LONG) (sizeof(struct VeCtOr))
03045             != sizeof(struct VeCtOr) ) goto ErrGt2;
03046         nsize = s->namesize; nsize += sizeof(void *)-1;
03047         nsize &= -sizeof(void *);
03048         if ( ReadFile(AR.StoreData.Handle, (UBYTE *) (AT.WorkPointer), nsize)
03049             != nsize ) goto ErrGt2;

```

```

03050     *w = s->number + AM.OffsetVector;
03051     if ( ( s->flags & INUSE ) != 0 ) {
03052         /* Find the replacement. It must exist! */
03053         neww = oldw;
03054         while ( *neww != VECTOVEC || neww[2] != *w ) neww += neww[1];
03055         k = neww[3] - AM.OffsetVector;
03056     }
03057     else if ( GetVar((UBYTE *) (AT.WorkPointer), &type, &k, ALLVARIABLES, NOAUTO) != NAMENOTFOUND ) {
03058         if ( type != CVECTOR ) {
03059             MesPrint("Error: Conflicting types for %s", (AT.WorkPointer));
03060             error = -1;
03061         }
03062         else {
03063             if ( ( s->complex & (VARTYPEIMAGINARY|VARTYPECOMPLEX) ) !=
03064                 ( vectors[k].complex & (VARTYPEIMAGINARY|VARTYPECOMPLEX) ) ) {
03065                 MesPrint("Warning: Conflicting complexity for %s", (AT.WorkPointer));
03066                 error = -1;
03067             }
03068         }
03069     }
03070     else {
03071         if ( ( k = EntVar(CVECTOR, (UBYTE *) (AT.WorkPointer),
03072             s->complex, 0, 0, s->dimension) ) < 0 ) goto GetTcall;
03073     }
03074     *(w+j) = k + AM.OffsetVector;
03075     w++;
03076 }
03077 }
03078 /*
03079     #] Vectors :
03080     #[ Functions :
03081 */
03082 {
03083     FUNCTIONS s = &FuN;
03084     w = r->func.lo; j = jfun;
03085     for ( i = 0; i < j; i++ ) {
03086         if ( ReadFile(AR.StoreData.Handle, (UBYTE *)s, (LONG)(sizeof(struct FuNctiOn)))
03087             != sizeof(struct FuNctiOn) ) goto ErrGt2;
03088         nsize = s->namesize; nsize += sizeof(void *)-1;
03089         nsize &= -sizeof(void *);
03090         if ( ReadFile(AR.StoreData.Handle, (UBYTE *) (AT.WorkPointer), nsize)
03091             != nsize ) goto ErrGt2;
03092         *w = s->number + FUNCTION;
03093         if ( ( s->flags & INUSE ) != 0 ) {
03094             /* Find the replacement. It must exist! */
03095             neww = oldw;
03096             while ( *neww != FUNTOFUN || neww[2] != *w ) neww += neww[1];
03097             k = neww[3] - FUNCTION;
03098         }
03099         else if ( GetVar((UBYTE *) (AT.WorkPointer), &type, &k, ALLVARIABLES, NOAUTO) != NAMENOTFOUND ) {
03100             if ( type != CFUNCTION ) {
03101                 MesPrint("Error: Conflicting types for %s", (AT.WorkPointer));
03102                 error = -1;
03103             }
03104             else {
03105                 if ( s->complex != functions[k].complex ) {
03106                     MesPrint("Warning: Conflicting complexity for %s", (AT.WorkPointer));
03107                     error = -1;
03108                 }
03109                 else if ( s->symmetric != functions[k].symmetric ) {
03110                     MesPrint("Warning: Conflicting symmetry properties for %s", (AT.WorkPointer));
03111                     error = -1;
03112                 }
03113                 else if ( ( s->maxnumargs != functions[k].maxnumargs )
03114                     || ( s->minnumargs != functions[k].minnumargs ) ) {
03115                     MesPrint("Warning: Conflicting argument restriction properties for
03116 %s", (AT.WorkPointer));
03117                     error = -1;
03118                 }
03119             }
03120         }
03121         else {
03122             if ( ( k = EntVar(CFUNCTION, (UBYTE *) (AT.WorkPointer),
03123                 s->complex, s->commute, s->spec, s->dimension) ) < 0 ) goto GetTcall;
03124             functions[k].symmetric = s->symmetric;
03125             functions[k].maxnumargs = s->maxnumargs;
03126             functions[k].minnumargs = s->minnumargs;
03127         }
03128         *(w+j) = k + FUNCTION;
03129         w++;
03130     }
03131 }
03132 /*
03133     #] Functions :
03134     Now we skip the prototype. This sets the start position at the first term
03135 */

```

```

03136     if ( error ) {
03137         UNLOCK(AM.storefilelock);
03138         AT.WorkPointer = oldwork;
03139         return(0);
03140     }
03141
03142     {
03143     /*
03144         For clarity we look where we are.
03145         We want to know: is this position already known?
03146         Could we have inserted extra information here?
03147
03148         nummystery indicates extra words. We have currently in order
03149         (if they exist)
03150             numdummies
03151             numfactors
03152             vflags
03153             uflags
03154     */
03155         POSITION pos;
03156         int nummystery;
03157         TELLFILE(AR.StoreData.Handle,&pos);
03158         nummystery = DIFBASE(ind->position,pos);
03159     /*
03160         MesPrint("--> We are at position      %8p",&pos);
03161         MesPrint("--> The index says          at %8p",&(ind->position));
03162         MesPrint("--> There are %d mystery bytes",nummystery);
03163     */
03164         if ( nummystery > 0 ) {
03165             if ( ReadFile(AR.StoreData.Handle, (UBYTE *)AT.WorkPointer, (LONG)sizeof(WORD)) !=
03166                 sizeof(WORD) ) {
03167                 UNLOCK(AM.storefilelock);
03168                 AT.WorkPointer = oldwork;
03169                 return(0);
03170             }
03171             Expressions[expr].numdummies = *AT.WorkPointer;
03172     /*
03173             MesPrint("--> numdummies = %d",Expressions[expr].numdummies);
03174     */
03175             nummystery -= sizeof(WORD);
03176         }
03177         else {
03178             Expressions[expr].numdummies = 0;
03179         }
03180         if ( nummystery > 0 ) {
03181             if ( ReadFile(AR.StoreData.Handle, (UBYTE *)AT.WorkPointer, (LONG)sizeof(WORD)) !=
03182                 sizeof(WORD) ) {
03183                 UNLOCK(AM.storefilelock);
03184                 AT.WorkPointer = oldwork;
03185                 return(0);
03186             }
03187             if ( ( AS.OldNumFactors == 0 ) || ( AS.NumOldNumFactors < NumExpressions ) ) {
03188                 WORD *buffer;
03189                 int capacity = 20;
03190                 if (capacity < NumExpressions) capacity = NumExpressions * 2;
03191
03192                 buffer = (WORD *)Malloca1(capacity * sizeof(WORD), "numfactors pointers");
03193                 if (AS.OldNumFactors) {
03194                     WCOPY(buffer, AS.OldNumFactors, AS.NumOldNumFactors);
03195                     M_free(AS.OldNumFactors, "numfactors pointers");
03196                 }
03197                 AS.OldNumFactors = buffer;
03198
03199                 buffer = (WORD *)Malloca1(capacity * sizeof(WORD), "vflags pointers");
03200                 if (AS.Oldvflags) {
03201                     WCOPY(buffer, AS.Oldvflags, AS.NumOldNumFactors);
03202                     M_free(AS.Oldvflags, "vflags pointers");
03203                 }
03204                 AS.Oldvflags = buffer;
03205
03206                 buffer = (WORD *)Malloca1(capacity * sizeof(WORD), "uflags pointers");
03207                 if (AS.Olduflags) {
03208                     WCOPY(buffer, AS.Olduflags, AS.NumOldNumFactors);
03209                     M_free(AS.Olduflags, "uflags pointers");
03210                 }
03211                 AS.Oldvflags = buffer;
03212
03213                 AS.NumOldNumFactors = capacity;
03214             }
03215
03216             AS.OldNumFactors[expr] =
03217             Expressions[expr].numfactors = *AT.WorkPointer;
03218     /*
03219             MesPrint("--> numfactors = %d",Expressions[expr].numfactors);
03220     */
03221             nummystery -= sizeof(WORD);
03222         }

```



```

03223     else {
03224         Expressions[expr].numfactors = 0;
03225     }
03226     if ( nummystery > 0 ) {
03227         if ( ReadFile(AR.StoreData.Handle, (UBYTE *)AT.WorkPointer, (LONG)sizeof(WORD)) !=
03228             sizeof(WORD) ) {
03229             UNLOCK(AM.storefilelock);
03230             AT.WorkPointer = oldwork;
03231             return(0);
03232         }
03233         AS.Oldvflags[expr] =
03234             Expressions[expr].vflags = *AT.WorkPointer;
03235         /*
03236             MesPrint("--> vflags = %d",Expressions[expr].vflags);
03237         */
03238         nummystery -= sizeof(WORD);
03239     }
03240     else {
03241         Expressions[expr].vflags = 0;
03242     }
03243     if ( nummystery > 0 ) {
03244         if ( ReadFile(AR.StoreData.Handle, (UBYTE *)AT.WorkPointer, (LONG)sizeof(WORD)) !=
03245             sizeof(WORD) ) {
03246             UNLOCK(AM.storefilelock);
03247             AT.WorkPointer = oldwork;
03248             return(0);
03249         }
03250         AS.Olduflags[expr] =
03251             Expressions[expr].uflags = *AT.WorkPointer;
03252         /*
03253             MesPrint("--> uflags = %d",Expressions[expr].uflags);
03254         */
03255         nummystery -= sizeof(WORD);
03256     }
03257     else {
03258         Expressions[expr].uflags = 0;
03259     }
03260 }
03261
03262 SeekFile(AR.StoreData.Handle,&(ind->position),SEEK_SET);
03263 if ( ReadFile(AR.StoreData.Handle, (UBYTE *)AT.WorkPointer, (LONG)sizeof(WORD)) !=
03264     sizeof(WORD) || !*AT.WorkPointer ) {
03265     UNLOCK(AM.storefilelock);
03266     AT.WorkPointer = oldwork;
03267     return(0);
03268 }
03269 num = *AT.WorkPointer - 1;
03270 num *= sizeof(WORD);
03271 if ( *AT.WorkPointer < 0 ||
03272     ReadFile(AR.StoreData.Handle, (UBYTE *) (AT.WorkPointer+1),num) != num ) {
03273     MesPrint("@@Error in stored expressions file at position %10p",*position);
03274     UNLOCK(AM.storefilelock);
03275     AT.WorkPointer = oldwork;
03276     return(0);
03277 }
03278 UNLOCK(AM.storefilelock);
03279 ADDPOS(*position,num+sizeof(WORD));
03280 r->startposition = *position;
03281 AT.WorkPointer = oldwork;
03282 return(r);
03283 GetTcall:
03284     UNLOCK(AM.storefilelock);
03285     AT.WorkPointer = oldwork;
03286     MesCall("GetTable");
03287     return(0);
03288 ErrGt2:
03289     UNLOCK(AM.storefilelock);
03290     AT.WorkPointer = oldwork;
03291     MesPrint("Readererror in GetTable");
03292     return(0);
03293 }
03294
03295 /*
03296     #] GetTable :
03297     #[ CopyExpression :
03298
03299     Copies from one scratch buffer to another.
03300     We assume here that the complete 'from' scratch buffer is taken.
03301     We also assume that the 'from' buffer is positioned at the end of
03302     the expression.
03303
03304     The locks should be placed in the calling routine. We need basically
03305     AS.outputslock.
03306 */
03307
03308 int CopyExpression(FILEHANDLE *from, FILEHANDLE *to)
03309 {

```

```

03310     POSITION posfrom, poscopy;
03311     LONG fullsize,i;
03312     WORD *t1, *t2;
03313     int RetCode;
03314     SeekScratch(from,&posfrom);
03315     if ( from->handle < 0 ) { /* input is in memory */
03316         fullsize = (BASEPOSITION(posfrom))/sizeof(WORD);
03317         if ( ( to->PStop - to->PFull ) >= fullsize ) {
03318             /*
03319              Fits inside the buffer of the output. This will be fast.
03320             */
03321             t1 = from->Pbuffer;
03322             t2 = to->Pfull;
03323             NCOPY(t2,t1,fullsize)
03324             to->Pfull = to->Pfill = t2;
03325             goto WriteTrailer;
03326         }
03327         if ( to->handle < 0 ) { /* First open the file */
03328             if ( ( RetCode = CreateFile(to->name) ) >= 0 ) {
03329                 to->handle = (WORD)RetCode;
03330                 PUTZERO(to->filesize);
03331                 PUTZERO(to->Pposition);
03332             }
03333             else {
03334                 MLOCK(ErrorMessageLock);
03335                 MesPrint("Cannot create scratch file %s",to->name);
03336                 MUNLOCK(ErrorMessageLock);
03337                 return(-1);
03338             }
03339         }
03340         t1 = from->Pbuffer;
03341         while ( fullsize > 0 ) {
03342             i = to->PStop - to->Pfull;
03343             if ( i > fullsize ) i = fullsize;
03344             fullsize -= i;
03345             t2 = to->Pfull;
03346             NCOPY(t2,t1,i)
03347             if ( fullsize > 0 ) {
03348                 SeekFile(to->handle,&(to->Pposition),SEEK_SET);
03349                 if ( WriteFile(to->handle,((UBYTE *) (to->Pbuffer)),to->Psize) != to->Psize ) {
03350                     MLOCK(ErrorMessageLock);
03351                     MesPrint("Error while writing to disk. Disk full?");
03352                     MUNLOCK(ErrorMessageLock);
03353                     return(-1);
03354                 }
03355                 ADDPOS(to->Pposition,to->Psize);
03356             /*
03357              SeekFile(to->handle,&(to->Pposition),SEEK_CUR); */
03358             to->filesize = to->Pposition;
03359             to->Pfill = to->Pfull = to->Pbuffer;
03360         }
03361         else {
03362             to->Pfill = to->Pfull = t2;
03363         }
03364         goto WriteTrailer;
03365     }
03366     /*
03367     Now the input involves a file. This needs the use of the Pbuffer of from.
03368     First make sure the tail of the buffer has been written
03369     */
03370     if ( ((UBYTE *) (from->Pfill))-((UBYTE *) (from->Pbuffer)) > 0 ) {
03371         if ( WriteFile(from->handle,((UBYTE *) (from->Pbuffer)),((UBYTE *) (from->Pfill))-((UBYTE *) (from->Pbuffer)))
03372             != ((UBYTE *) (from->Pfill))-((UBYTE *) (from->Pbuffer)) ) {
03373             MLOCK(ErrorMessageLock);
03374             MesPrint("Error while writing to disk. Disk full?");
03375             MUNLOCK(ErrorMessageLock);
03376             return(-1);
03377         }
03378         SeekFile(from->handle,&(from->Pposition),SEEK_CUR);
03379         posfrom = from->filesize = from->Pposition;
03380         from->Pfill = from->Pfull = from->Pbuffer;
03381     }
03382     /*
03383     Now copy the complete contents
03384     */
03385     PUTZERO(poscopy);
03386     SeekFile(from->handle,&poscopy,SEEK_SET);
03387     while ( !ISLESSPOS(poscopy,posfrom) ) {
03388         fullsize = ReadFile(from->handle,((UBYTE *) (from->Pbuffer)),from->Psize);
03389         if ( fullsize < 0 || ( fullsize % sizeof(WORD) ) != 0 ) {
03390             MLOCK(ErrorMessageLock);
03391             MesPrint("Error while reading from disk while copying expression.");
03392             MUNLOCK(ErrorMessageLock);
03393             return(-1);
03394         }
03395         fullsize /= sizeof(WORD);

```

```

03396     from->POfull = from->PObuffer + fullsize;
03397     t1 = from->PObuffer;
03398
03399     if ( ( to->POstop - to->POfull ) >= fullsize ) {
03400 /*
03401         Fits inside the buffer of the output. This will be fast.
03402 */
03403         t2 = to->POfull;
03404         NCOPY(t2,t1,fullsize)
03405         to->POfill = to->POfull = t2;
03406     }
03407     else {
03408         if ( to->handle < 0 ) { /* First open the file */
03409             if ( ( RetCode = CreateFile(to->name) ) >= 0 ) {
03410                 to->handle = (WORD)RetCode;
03411                 PUTZERO(to->POposition);
03412                 PUTZERO(to->filesize);
03413             }
03414             else {
03415                 MLOCK(ErrorMessageLock);
03416                 MesPrint("Cannot create scratch file %s",to->name);
03417                 MUNLOCK(ErrorMessageLock);
03418                 return(-1);
03419             }
03420         }
03421         while ( fullsize > 0 ) {
03422             i = to->POstop - to->POfull;
03423             if ( i > fullsize ) i = fullsize;
03424             fullsize -= i;
03425             t2 = to->POfull;
03426             NCOPY(t2,t1,i)
03427             if ( fullsize > 0 ) {
03428                 SeekFile(to->handle,&(to->POposition),SEEK_SET);
03429                 if ( WriteFile(to->handle,((UBYTE *) (to->PObuffer)),to->POsize) != to->POsize ) {
03430                     MLOCK(ErrorMessageLock);
03431                     MesPrint("Error while writing to disk. Disk full?");
03432                     MUNLOCK(ErrorMessageLock);
03433                     return(-1);
03434                 }
03435                 ADDPOS(to->POposition,to->POsize);
03436 /*
03437                 SeekFile(to->handle,&(to->POposition),SEEK_CUR); */
03438                 to->filesize = to->POposition;
03439                 to->POfill = to->POfull = to->PObuffer;
03440             }
03441             else {
03442                 to->POfill = to->POfull = t2;
03443             }
03444         }
03445         SeekFile(from->handle,&poscopy,SEEK_CUR);
03446     }
03447 WriteTrailer:
03448     if ( ( to->handle >= 0 ) && ( to->POfill > to->PObuffer ) ) {
03449         fullsize = (UBYTE *) (to->POfill) - (UBYTE *) (to->PObuffer);
03450 /*
03451         PUTZERO(to->POposition);
03452         SeekFile(to->handle,&(to->POposition),SEEK_END);
03453 */
03454         SeekFile(to->handle,&(to->filesize),SEEK_SET);
03455         if ( WriteFile(to->handle,((UBYTE *) (to->PObuffer)),fullsize) != fullsize ) {
03456             MLOCK(ErrorMessageLock);
03457             MesPrint("Error while writing to disk. Disk full?");
03458             MUNLOCK(ErrorMessageLock);
03459             return(-1);
03460         }
03461         ADDPOS(to->filesize,fullsize);
03462         to->POposition = to->filesize;
03463         to->POfill = to->POfull = to->PObuffer;
03464     }
03465
03466     return(0);
03467 }
03468
03469 /*
03470     #] CopyExpression :
03471     #[ ExprStatus :
03472 */
03473
03474 #ifndef HIDEDEBUG
03475
03476 static UBYTE *statusexpr[] = {
03477     (UBYTE *) "LOCALEXPRESSION"
03478     , (UBYTE *) "SKIPLEXPRESSION"
03479     , (UBYTE *) "DROPLEXPRESSION"
03480     , (UBYTE *) "DROPEXPRESSION"
03481     , (UBYTE *) "GLOBALEXPRESSION"
03482     , (UBYTE *) "SKIPGEXPRESSION"

```

```

03483     , (UBYTE *) "DROPGEXPRESSION"
03484     , (UBYTE *) "UNKNOWN"
03485     , (UBYTE *) "STOREDEXPRESSION"
03486     , (UBYTE *) "HIDDENLEXPRESSION"
03487     , (UBYTE *) "HIDELEXPRESSION"
03488     , (UBYTE *) "DROPHLEXPRESSION"
03489     , (UBYTE *) "UNHIDELEXPRESSION"
03490     , (UBYTE *) "HIDDENGEXPRESSION"
03491     , (UBYTE *) "HIDEGEXPRESSION"
03492     , (UBYTE *) "DROPHGEXPRESSION"
03493     , (UBYTE *) "UNHIDEGEXPRESSION"
03494     , (UBYTE *) "INTOHIDELEXPRESSION"
03495     , (UBYTE *) "INTOHIDEGEXPRESSION"
03496 };
03497
03498 void ExprStatus (EXPRESSIONS e)
03499 {
03500     MesPrint ("Expression %s(%d) has status %s(%d,%d). Buffer: %d, Position: %15p",
03501         AC.exprnames->namebuffer+e->name, (WORD) (e-Expressions),
03502         statusexpr[e->status], e->status, e->hidelevel,
03503         e->whichbuffer, &(e->onfile));
03504 }
03505
03506 #endif
03507
03508 /*
03509     #] ExprStatus :
03510     #] StoreExpressions :
03511     #[ System Independent Saved Expressions :
03512
03513     All functions concerned with the system independent reading of save-files
03514     are here. They are called by the functions CoLoad, PutInStore,
03515     SetFileIndex, FindInIndex. In case no translation (endianness flip,
03516     resizing of words, renumbering) has to be done, they just do simple file
03517     reading. The function SaveFileHeader() for writing a header with
03518     information about the system architecture, FORM version, etc. is also
03519     located here.
03520
03521     #[ Flip :
03522 */
03523
03524 #ifndef INT16
03525 #error "INT16 not defined!"
03526 #endif
03527 #ifndef INT32
03528 #error "INT32 not defined!"
03529 #endif
03530
03531 static void FlipN(UBYTE *p, int length)
03532 {
03533     UBYTE *q, buf;
03534     q = p + length;
03535     do {
03536         --q;
03537         buf = *p; *p = *q; *q = buf;
03538     } while ( ++p != q );
03539 }
03540
03541 static void Flip16(UBYTE *p)
03542 {
03543     INT16 in = *((INT16 *)p);
03544     INT16 out = (INT16) ( ((in) >> 8) & 0x00FF | (((in) << 8) & 0xFF00) );
03545     *((INT16 *)p) = out;
03546 }
03547
03548 static void Flip32(UBYTE *p)
03549 {
03550     INT32 in = *((INT32 *)p);
03551     INT32 out =
03552         ( (((in) >> 24) & 0x000000FF) | (((in) >> 8) & 0x0000FF00) | \
03553         (((in) << 8) & 0x00FF0000) | (((in) << 24) & 0xFF000000) );
03554     *((INT32 *)p) = out;
03555 }
03556
03557 #ifdef INT64
03558 static void Flip64(UBYTE *p)
03559 {
03560     INT64 in = *((INT64 *)p);
03561     INT64 out =
03562         ( (((in) >> 56) & (INT64)0x00000000000000FFLL) | (((in) >> 40) & (INT64)0x000000000000FF00LL) | \
03563         (((in) >> 24) & (INT64)0x0000000000FF0000LL) | (((in) >> 8) & (INT64)0x00000000FF000000LL) | \
03564         (((in) << 8) & (INT64)0x000000FF00000000LL) | (((in) << 24) & (INT64)0x0000FF0000000000LL) | \
03565         (((in) << 40) & (INT64)0x00FF000000000000LL) | (((in) << 56) & (INT64)0xFF00000000000000LL) );
03566     *((INT64 *)p) = out;
03567 }
03568 #endif

```

```

03587 }
03588 #else
03589 static void Flip64(UBYTE *p) { FlipN(p, 8); }
03590 #endif /* INT64 */
03591
03593 static void Flip128(UBYTE *p) { FlipN(p, 16); }
03594
03595 /*
03596     #] Flip :
03597     #[ Resize :
03598 */
03599
03611 static void ResizeDataBE(UBYTE *src, int slen, UBYTE *dst, int dlen)
03612 {
03613     if ( slen > dlen ) {
03614         src += slen - dlen;
03615         while ( dlen-- ) { *dst++ = *src++; }
03616     }
03617     else {
03618         int i = dlen - slen;
03619         while ( i-- ) { *dst++ = 0; }
03620         while ( slen-- ) { *dst++ = *src++; }
03621     }
03622 }
03623
03627 static void ResizeDataLE(UBYTE *src, int slen, UBYTE *dst, int dlen)
03628 {
03629     if ( slen > dlen ) {
03630         while ( dlen-- ) { *dst++ = *src++; }
03631     }
03632     else {
03633         int i = dlen - slen;
03634         while ( slen-- ) { *dst++ = *src++; }
03635         while ( i-- ) { *dst++ = 0; }
03636     }
03637 }
03638
03651 static void Resize16t16(UBYTE *src, UBYTE *dst)
03652 {
03653     *((INT16 *)dst) = *((INT16 *)src);
03654 }
03655
03657 static void Resize16t32(UBYTE *src, UBYTE *dst)
03658 {
03659     INT16 in = *((INT16 *)src);
03660     INT32 out = (INT32)in;
03661     *((INT32 *)dst) = out;
03662 }
03663
03665 #ifndef INT64
03666 static void Resize16t64(UBYTE *src, UBYTE *dst)
03667 {
03668     INT16 in = *((INT16 *)src);
03669     INT64 out = (INT64)in;
03670     *((INT64 *)dst) = out;
03671 }
03672 #else
03673 static void Resize16t64(UBYTE *src, UBYTE *dst) { AO.ResizeData(src, 2, dst, 8); }
03674 #endif /* INT64 */
03675
03677 static void Resize16t128(UBYTE *src, UBYTE *dst) { AO.ResizeData(src, 2, dst, 16); }
03678
03680 static void Resize32t32(UBYTE *src, UBYTE *dst)
03681 {
03682     *((INT32 *)dst) = *((INT32 *)src);
03683 }
03684
03686 #ifndef INT64
03687 static void Resize32t64(UBYTE *src, UBYTE *dst)
03688 {
03689     INT32 in = *((INT32 *)src);
03690     INT64 out = (INT64)in;
03691     *((INT64 *)dst) = out;
03692 }
03693 #else
03694 static void Resize32t64(UBYTE *src, UBYTE *dst) { AO.ResizeData(src, 4, dst, 8); }
03695 #endif /* INT64 */
03696
03698 static void Resize32t128(UBYTE *src, UBYTE *dst) { AO.ResizeData(src, 4, dst, 16); }
03699
03701 #ifndef INT64
03702 static void Resize64t64(UBYTE *src, UBYTE *dst)
03703 {
03704     *((INT64 *)dst) = *((INT64 *)src);
03705 }
03706 #else
03707 static void Resize64t64(UBYTE *src, UBYTE *dst) { AO.ResizeData(src, 8, dst, 8); }

```

```

03708 #endif /* INT64 */
03709
03711 static void Resize64t128(UBYTE *src, UBYTE *dst) { AO.ResizeData(src, 8, dst, 16); }
03712
03714 static void Resize128t128(UBYTE *src, UBYTE *dst) { AO.ResizeData(src, 16, dst, 16); }
03715
03717 static void Resize32t16(UBYTE *src, UBYTE *dst)
03718 {
03719     INT32 in = *((INT32 *)src);
03720     INT16 out = (INT16)in;
03721     if ( in > (1<15)-1 || in < -(1<15)+1 ) AO.resizeFlag |= 1;
03722     *((INT16 *)dst) = out;
03723 }
03724
03731 static void Resize32t16NC(UBYTE *src, UBYTE *dst)
03732 {
03733     INT32 in = *((INT32 *)src);
03734     INT16 out = (INT16)in;
03735     *((INT16 *)dst) = out;
03736 }
03737
03738 #ifdef INT64
03740 static void Resize64t16(UBYTE *src, UBYTE *dst)
03741 {
03742     INT64 in = *((INT64 *)src);
03743     INT16 out = (INT16)in;
03744     if ( in > (1<15)-1 || in < -(1<15)+1 ) AO.resizeFlag |= 1;
03745     *((INT16 *)dst) = out;
03746 }
03748 static void Resize64t16NC(UBYTE *src, UBYTE *dst)
03749 {
03750     INT64 in = *((INT64 *)src);
03751     INT16 out = (INT16)in;
03752     *((INT16 *)dst) = out;
03753 }
03754 #else
03756 static void Resize64t16(UBYTE *src, UBYTE *dst) { AO.ResizeData(src, 8, dst, 2); }
03758 static void Resize64t16NC(UBYTE *src, UBYTE *dst) { AO.ResizeData(src, 8, dst, 2); }
03759 #endif /* INT64 */
03760
03761 #ifdef INT64
03763 static void Resize64t32(UBYTE *src, UBYTE *dst)
03764 {
03765     INT64 in = *((INT64 *)src);
03766     INT32 out = (INT32)in;
03767     if ( in > ((INT64)1<31)-1 || in < -((INT64)1<31)+1 ) AO.resizeFlag |= 1;
03768     *((INT32 *)dst) = out;
03769 }
03771 static void Resize64t32NC(UBYTE *src, UBYTE *dst)
03772 {
03773     INT64 in = *((INT64 *)src);
03774     INT32 out = (INT32)in;
03775     *((INT32 *)dst) = out;
03776 }
03777 #else
03779 static void Resize64t32(UBYTE *src, UBYTE *dst) { AO.ResizeData(src, 8, dst, 4); }
03781 static void Resize64t32NC(UBYTE *src, UBYTE *dst) { AO.ResizeData(src, 8, dst, 4); }
03782 #endif /* INT64 */
03783
03785 static void Resize128t16(UBYTE *src, UBYTE *dst) { AO.ResizeData(src, 16, dst, 2); }
03786
03788 static void Resize128t16NC(UBYTE *src, UBYTE *dst) { AO.ResizeData(src, 16, dst, 2); }
03789
03791 static void Resize128t32(UBYTE *src, UBYTE *dst) { AO.ResizeData(src, 16, dst, 4); }
03792
03794 static void Resize128t32NC(UBYTE *src, UBYTE *dst) { AO.ResizeData(src, 16, dst, 4); }
03795
03797 static void Resize128t64(UBYTE *src, UBYTE *dst) { AO.ResizeData(src, 16, dst, 8); }
03798
03800 static void Resize128t64NC(UBYTE *src, UBYTE *dst) { AO.ResizeData(src, 16, dst, 8); }
03801
03802 /*
03803     #] Resize :
03804     #[ CheckPower and RenumberVec :
03805 */
03806
03813 static void CheckPower32(UBYTE *p)
03814 {
03815     if ( *((INT32 *)p) < -MAXPOWER ) {
03816         AO.powerFlag |= 0x01;
03817         *((INT32 *)p) = -MAXPOWER;
03818     }
03819     p += sizeof(INT32);
03820     if ( *((INT32 *)p) > MAXPOWER ) {
03821         AO.powerFlag |= 0x02;
03822         *((INT32 *)p) = MAXPOWER;
03823     }

```

```

03824 }
03825
03833 static void RenumberVec32(UBYTE *p)
03834 {
03835     /* INT32 wilddoffset = *((INT32 *)AO.SaveHeader.wilddoffset); */
03836     void *dummy = (void *)AO.SaveHeader.wilddoffset; /* to remove a warning about strict-aliasing
rules in gcc */
03837     INT32 wilddoffset = *((INT32 *)dummy);
03838     INT32 in = *((INT32 *)p);
03839     in = in + 2*wilddoffset;
03840     in = in - 2*WILDOFFSET;
03841     *((INT32 *)p) = in;
03842 }
03843
03844 /*
03845     #] CheckPower and RenumberVec :
03846     #[ ResizeCoeff :
03847 */
03848
03862 static void ResizeCoeff32(UBYTE **bout, UBYTE *bend, UBYTE *top)
03863 {
03864     int i;
03865     INT32 sign;
03866     INT32 *in, *p;
03867     INT32 *out = (INT32 *)*bout;
03868     INT32 *end = (INT32 *)bend;
03869
03870     if ( sizeof(WORD) == 2 ) {
03871         /* 4 -> 2 */
03872         INT32 len = (end - 1 - out) / 2;
03873         int zeros = 2;
03874         p = out + len - 1;
03875
03876         if ( *p & 0xFFFF0000 ) --zeros;
03877         p += len;
03878         if ( *p & 0xFFFF0000 ) --zeros;
03879
03880         in = end - 1;
03881         sign = ( *in-- > 0 ) ? 1 : -1;
03882         p = out + 4*len;
03883         if ( zeros == 2 ) p -= 2;
03884         out = p--;
03885
03886         if ( zeros < 2 ) *p-- = *in >> 16;
03887         *p-- = *in-- & 0x0000FFFF;
03888         for ( i = 1; i < len; ++i ) {
03889             *p-- = *in >> 16;
03890             *p-- = *in-- & 0x0000FFFF;
03891         }
03892         if ( zeros < 2 ) *p-- = *in >> 16;
03893         *p-- = *in-- & 0x0000FFFF;
03894         for ( i = 1; i < len; ++i ) {
03895             *p-- = *in >> 16;
03896             *p-- = *in-- & 0x0000FFFF;
03897         }
03898
03899         *out = (out - p) * sign;
03900         *bout = (UBYTE *) (out+1);
03901     }
03902     else {
03903         /* 2 -> 4 */
03904         INT32 len = (end - 1 - out) / 2;
03905         if ( len == 1 ) {
03906             *out = *(unsigned INT16 *)out;
03907             ++out;
03908             *out = *(unsigned INT16 *)out;
03909             ++out;
03910             ++out;
03911             ++out;
03912         }
03913         else {
03914             p = out;
03915             *out = *(unsigned INT16 *)out;
03916             in = out + 1;
03917             for ( i = 1; i < len; ++i ) {
03918                 /* shift */
03919                 *out = (unsigned INT32) (*(unsigned INT16 *)out)
03920                     + ((unsigned INT32) (*(unsigned INT16 *)in) << 16);
03921                 ++in;
03922                 if ( ++i == len ) break;
03923                 /* copy */
03924                 ++out;
03925                 *out = *(unsigned INT16 *)in;
03926                 ++in;
03927             }
03928             ++out;
03929             *out = *(unsigned INT16 *)in;

```

```

03930         ++in;
03931         for ( i = 1; i < len; ++i ) {
03932             /* shift */
03933             *out = (unsigned INT32)(*(unsigned INT16 *)out)
03934                 + ((unsigned INT32)(*(unsigned INT16 *)in) << 16);
03935             ++in;
03936             if ( ++i == len ) break;
03937             /* copy */
03938             ++out;
03939             *out = *(unsigned INT16 *)in;
03940             ++in;
03941         }
03942         ++out;
03943         if ( *in < 0 ) *out = -(out - p + 1);
03944         else *out = out - p + 1;
03945         ++out;
03946     }
03947
03948     if ( out > (INT32 *)top ) {
03949         MesPrint("Error in resizing coefficient!");
03950     }
03951
03952     *bout = (UBYTE *)out;
03953 }
03954 }
03955
03956 /*
03957     #] ResizeCoeff :
03958     #[ WriteStoreHeader :
03959 */
03960
03961 #define SAVEREVISION 0x02
03962
03972 WORD WriteStoreHeader(WORD handle)
03973 {
03974     /* template of the STOREHEADER */
03975     static STOREHEADER sh = {
03976         { 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF }, /* store header mark */
03977         0, 0, 0, 0, /* sizeof of WORD, LONG, POSITION, void* */
03978         { 0 }, /* endianness check number */
03979         0, 0, 0, 0, /* sizeof variable structs */
03980         { 0 }, /* maxpower */
03981         { 0 }, /* wilddoffset */
03982         SAVEREVISION, /* revision */
03983         { 0 } }; /* reserved */
03984     int endian, i;
03985
03986     /* if called for the first time ... */
03987     if ( sh.lenWORD == 0 ) {
03988         sh.lenWORD = sizeof(WORD);
03989         sh.lenLONG = sizeof(LONG);
03990         sh.lenPOS = sizeof(POSITION);
03991         sh.lenPOINTER = sizeof(void *);
03992
03993         endian = 1;
03994         for ( i = 1; i < (int)sizeof(int); ++i ) {
03995             endian <<= 8;
03996             endian += i+1;
03997         }
03998         for ( i = 0; i < (int)sizeof(int); ++i ) sh.endianness[i] = ((char *)&endian)[i];
03999
04000         sh.sSym = sizeof(struct SymB01);
04001         sh.sInd = sizeof(struct InDeX);
04002         sh.sVec = sizeof(struct VeCtOr);
04003         sh.sFun = sizeof(struct FuNcTiOn);
04004
04005         /* *((WORD *)sh.maxpower) = MAXPOWER;
04006         *((WORD *)sh.wilddoffset) = WILDOFFSET; */
04007         {
04008             WORD dumw[8];
04009             UBYTE *dummy;
04010             for ( i = 0; i < 8; i++ ) dumw[i] = 0;
04011             dummy = (UBYTE *)dumw;
04012             dumw[0] = (WORD)MAXPOWER;
04013             for ( i = 0; i < 16; i++ ) sh.maxpower[i] = dummy[i];
04014             dumw[0] = (WORD)WILDOFFSET;
04015             for ( i = 0; i < 16; i++ ) sh.wilddoffset[i] = dummy[i];
04016         }
04017     }
04018
04019     return ( WriteFile(handle, (UBYTE *)(&sh), (LONG)(sizeof(STOREHEADER)))
04020             != (LONG)(sizeof(STOREHEADER)) );
04021 }
04022
04023 /*
04024     #] WriteStoreHeader :
04025     #[ CompactifySizeof :

```



```

04026 */
04027
04035 static unsigned int CompactifySizeof(unsigned int size)
04036 {
04037     switch ( size ) {
04038         case 2: return 0;
04039         case 4: return 1;
04040         case 8: return 2;
04041         case 16: return 3;
04042         default: MesPrint("Error compactifying size.");
04043             return 3;
04044     }
04045 }
04046
04047 /*
04048     #] CompactifySizeof :
04049     #[ ReadSaveHeader :
04050 */
04051
04065 WORD ReadSaveHeader(void)
04066 {
04067     /* Read-only tables of function pointers for conversions. */
04068     static void (*flipJumpTable[4])(UBYTE *) =
04069         { Flip16, Flip32, Flip64, Flip128 };
04070     static void (*resizeJumpTable[4][4])(UBYTE *, UBYTE *) = /* "own x saved"-sizes */
04071         { { Resize16t16, Resize32t16, Resize64t16, Resize128t16 },
04072           { Resize16t32, Resize32t32, Resize64t32, Resize128t32 },
04073           { Resize16t64, Resize32t64, Resize64t64, Resize128t64 },
04074           { Resize16t128, Resize32t128, Resize64t128, Resize128t128 } };
04075     static void (*resizeNCJumpTable[4][4])(UBYTE *, UBYTE *) = /* "own x saved"-sizes */
04076         { { Resize16t16NC, Resize32t16NC, Resize64t16NC, Resize128t16NC },
04077           { Resize16t32, Resize32t32, Resize64t32NC, Resize128t32NC },
04078           { Resize16t64, Resize32t64, Resize64t64, Resize128t64NC },
04079           { Resize16t128, Resize32t128, Resize64t128, Resize128t128 } };
04080
04081     int endian, i;
04082     WORD idxW = CompactifySizeof(sizeof(WORD));
04083     WORD idxL = CompactifySizeof(sizeof(LONG));
04084     WORD idxP = CompactifySizeof(sizeof(POSITION));
04085     WORD idxVP = CompactifySizeof(sizeof(void *));
04086
04087     AO.transFlag = 0;
04088     AO.powerFlag = 0;
04089     AO.resizeFlag = 0;
04090     AO.bufferedInd = 0;
04091
04092     if ( ReadFile(AO.SaveData.Handle, (UBYTE *)(&AO.SaveHeader),
04093         (LONG)sizeof(STOREHEADER)) != (LONG)sizeof(STOREHEADER) )
04094         return(MesPrint("Error reading save file header"));
04095
04096     /* check whether save-file has no header. if yes then it is an old version
04097     of FORM -> go back to position 0 in file which then contains the first
04098     index and skip the rest. */
04099     for ( i = 0; i < 8; ++i ) {
04100         if ( AO.SaveHeader.headermark[i] != 0xFF ) {
04101             POSITION p;
04102             PUTZERO(p);
04103             SeekFile(AO.SaveData.Handle, &p, SEEK_SET);
04104             return ( 0 );
04105         }
04106     }
04107
04108     if ( AO.SaveHeader.revision != SAVEREVISION ) {
04109         return(MesPrint("Save file header from an old version. Cannot read this file.));
04110     }
04111
04112     endian = 1;
04113     for ( i = 1; i < (int)sizeof(int); ++i ) {
04114         endian <= 8;
04115         endian += i+1;
04116     }
04117     if ( ((char *)&endian)[0] < ((char *)&endian)[1] ) {
04118         /* this machine is big-endian */
04119         AO.ResizeData = ResizeDataBE;
04120     }
04121     else {
04122         /* this machine is little-endian */
04123         AO.ResizeData = ResizeDataLE;
04124     }
04125
04126     /* set AO.transFlag if ANY conversion has to be done later */
04127     if ( AO.SaveHeader.endianness[0] > AO.SaveHeader.endianness[1] ) {
04128         AO.transFlag = ( ((char *)&endian)[0] < ((char *)&endian)[1] );
04129     }
04130     else {
04131         AO.transFlag = ( ((char *)&endian)[0] > ((char *)&endian)[1] );
04132     }

```

```

04133     if ( (WORD)AO.SaveHeader.lenWORD != sizeof(WORD) ) AO.transFlag |= 0x02;
04134     if ( (WORD)AO.SaveHeader.lenLONG != sizeof(LONG) ) AO.transFlag |= 0x04;
04135     if ( (WORD)AO.SaveHeader.lenPOS != sizeof(POSITION) ) AO.transFlag |= 0x08;
04136     if ( (WORD)AO.SaveHeader.lenPOINTER != sizeof(void *) ) AO.transFlag |= 0x10;
04137
04138     AO.FlipWORD = flipJumpTable[idxW];
04139     AO.FlipLONG = flipJumpTable[idxL];
04140     AO.FlipPOS = flipJumpTable[idxP];
04141     AO.FlipPOINTER = flipJumpTable[idxVP];
04142
04143     /* Works only for machines where WORD is not greater than 32bit ! */
04144     AO.CheckPower = CheckPower32;
04145     AO.RenumberVec = RenumberVec32;
04146
04147     AO.ResizeWORD = resizeJumpTable[idxW][CompactifySizeof(AO.SaveHeader.lenWORD)];
04148     AO.ResizeNCWORD = resizeNCJumpTable[idxW][CompactifySizeof(AO.SaveHeader.lenWORD)];
04149     AO.ResizeLONG = resizeJumpTable[idxL][CompactifySizeof(AO.SaveHeader.lenLONG)];
04150     AO.ResizePOS = resizeJumpTable[idxP][CompactifySizeof(AO.SaveHeader.lenPOS)];
04151     AO.ResizePOINTER = resizeJumpTable[idxVP][CompactifySizeof(AO.SaveHeader.lenPOINTER)];
04152
04153     {
04154         WORD dumw[8];
04155         UBYTE *dummy;
04156         for ( i = 0; i < 8; i++ ) dumw[i] = 0;
04157         dummy = (UBYTE *)dumw;
04158         for ( i = 0; i < 16; i++ ) dummy[i] = AO.SaveHeader.maxpower[i];
04159         AO.mpower = dumw[0];
04160     }
04161
04162     return ( 0 );
04163 }
04164
04165 /*
04166     #] ReadSaveHeader :
04167     #[ ReadSaveIndex :
04168 */
04169
04183 WORD ReadSaveIndex(FILEINDEX *fileind)
04184 {
04185     /* do we need some translation for the FILEINDEX? */
04186     if ( AO.transFlag ) {
04187         /* if a translated FILEINDEX can hold less entries than the original
04188            FILEINDEX, then we need to buffer the extra entries in this static
04189            variable (can happen going from 32bit to 64bit */
04190         static FILEINDEX sbuffer;
04191
04192         FILEINDEX buffer;
04193         UBYTE *p, *q;
04194         int i;
04195
04196         /* shortcuts */
04197         int lenW = AO.SaveHeader.lenWORD;
04198         int lenL = AO.SaveHeader.lenLONG;
04199         int lenP = AO.SaveHeader.lenPOS;
04200
04201         /* if we have a buffered FILEINDEX then just return it */
04202         if ( AO.bufferedInd ) {
04203             *fileind = sbuffer;
04204             AO.bufferedInd = 0;
04205             return ( 0 );
04206         }
04207
04208         if ( ReadFile(AO.SaveData.Handle, (UBYTE *)fileind, sizeof(FILEINDEX))
04209             != sizeof(FILEINDEX) ) {
04210             return ( MesPrint("Error(1) reading stored expression.") );
04211         }
04212
04213         /* do we need to flip the endianness? */
04214         if ( AO.transFlag & 1 ) {
04215             LONG number;
04216             /* padding bytes */
04217             int padp = lenL - ((lenW*5+(MAXENAME + 1)) & (lenL-1));
04218             p = (UBYTE *)fileind;
04219             AO.FlipPOS(p); p += lenP;          /* next */
04220             AO.FlipPOS(p);                      /* number */
04221             AO.ResizePOS(p, (UBYTE *)&number);
04222             p += lenP;
04223             for ( i = 0; i < number; ++i ) {
04224                 AO.FlipPOS(p); p += lenP;      /* position */
04225                 AO.FlipPOS(p); p += lenP;      /* length */
04226                 AO.FlipPOS(p); p += lenP;      /* variables */
04227                 AO.FlipLONG(p); p += lenL;      /* CompressSize */
04228                 AO.FlipWORD(p); p += lenW;      /* nsymbols */
04229                 AO.FlipWORD(p); p += lenW;      /* nindices */
04230                 AO.FlipWORD(p); p += lenW;      /* nvectors */
04231                 AO.FlipWORD(p); p += lenW;      /* nfunctions */
04232                 AO.FlipWORD(p); p += lenW;      /* size */

```

```

04233         p += padp;
04234     }
04235 }
04236
04237 /* do we need to resize data? */
04238 if ( AO.transFlag > 1 ) {
04239     LONG number, maxnumber;
04240     int n;
04241     /* padding bytes */
04242     int padp = lenL - ((lenW*5+(MAXENAME + 1)) & (lenL-1));
04243     int padq = sizeof(LONG) - ((sizeof(WORD)*5+(MAXENAME + 1)) & (sizeof(LONG)-1));
04244
04245     p = (UBYTE *)fileind; q = (UBYTE *)&buffer;
04246     AO.ResizePOS(p, q); /* next */
04247     p += lenP; q += sizeof(POSITION);
04248     AO.ResizePOS(p, q); /* number */
04249     p += lenP;
04250     number = BASEPOSITION(*((POSITION *)q));
04251     /* if FILEINDEX in file contains more entries than the FILEINDEX in
04252     memory can contain, then adjust the numbers and prepare for
04253     buffering */
04254     if ( number > (LONG)INFILEINDEX ) {
04255         AO.bufferedInd = number-INFILEINDEX;
04256         if ( AO.bufferedInd > (WORD)INFILEINDEX ) {
04257             /* can happen when reading 32bit and writing >=128bit.
04258             Fix: more than one static buffer for FILEINDEX */
04259             return ( MesPrint("Too many index entries.") );
04260         }
04261         maxnumber = INFILEINDEX;
04262         SETBASEPOSITION(*((POSITION *)q), INFILEINDEX);
04263     }
04264     else {
04265         maxnumber = number;
04266     }
04267     q += sizeof(POSITION);
04268     /* read all INDEXENTRY that fit into the output buffer */
04269     for ( i = 0; i < maxnumber; ++i ) {
04270         AO.ResizePOS(p, q); /* position */
04271         p += lenP; q += sizeof(POSITION);
04272         AO.ResizePOS(p, q); /* length */
04273         p += lenP; q += sizeof(POSITION);
04274         AO.ResizePOS(p, q); /* variables */
04275         p += lenP; q += sizeof(POSITION);
04276         AO.ResizeLONG(p, q); /* CompressSize */
04277         p += lenL; q += sizeof(LONG);
04278         AO.ResizeWORD(p, q); /* nsymbols */
04279         p += lenW; q += sizeof(WORD);
04280         AO.ResizeWORD(p, q); /* nindices */
04281         p += lenW; q += sizeof(WORD);
04282         AO.ResizeWORD(p, q); /* nvectors */
04283         p += lenW; q += sizeof(WORD);
04284         AO.ResizeWORD(p, q); /* nfunctions */
04285         p += lenW; q += sizeof(WORD);
04286         AO.ResizeWORD(p, q); /* size (unchanged!) */
04287         p += lenW; q += sizeof(WORD);
04288         n = MAXENAME + 1;
04289         NCOPYB(q, p, n)
04290         p += padp;
04291         q += padq;
04292     }
04293     /* read all the remaining INDEXENTRY and put them into the static buffer */
04294     if ( AO.bufferedInd ) {
04295         sbuffer.next = buffer.next;
04296         SETBASEPOSITION(sbuffer.number, AO.bufferedInd);
04297         q = (UBYTE *)&sbuffer + sizeof(POSITION) + sizeof(LONG);
04298         for ( i = maxnumber; i < number; ++i ) {
04299             AO.ResizePOS(p, q); /* position */
04300             p += lenP; q += sizeof(POSITION);
04301             AO.ResizePOS(p, q); /* length */
04302             p += lenP; q += sizeof(POSITION);
04303             AO.ResizePOS(p, q); /* variables */
04304             p += lenP; q += sizeof(POSITION);
04305             AO.ResizeLONG(p, q); /* CompressSize */
04306             p += lenL; q += sizeof(LONG);
04307             AO.ResizeWORD(p, q); /* nsymbols */
04308             p += lenW; q += sizeof(WORD);
04309             AO.ResizeWORD(p, q); /* nindices */
04310             p += lenW; q += sizeof(WORD);
04311             AO.ResizeWORD(p, q); /* nvectors */
04312             p += lenW; q += sizeof(WORD);
04313             AO.ResizeWORD(p, q); /* nfunctions */
04314             p += lenW; q += sizeof(WORD);
04315             AO.ResizeWORD(p, q); /* size (unchanged!) */
04316             p += lenW; q += sizeof(WORD);
04317             n = MAXENAME + 1;
04318             NCOPYB(q, p, n)
04319             p += padp;

```

```

04320         q += padq;
04321     }
04322 }
04323 /* copy to output */
04324 p = (UBYTE *)fileind; q = (UBYTE *)&buffer; n = sizeof(FILEINDEX);
04325 NCOPYB(p, q, n)
04326 }
04327 return ( 0 );
04328 } else {
04329     return ( ReadFile(AO.SaveData.Handle, (UBYTE *)fileind, sizeof(FILEINDEX))
04330         != sizeof(FILEINDEX) );
04331 }
04332 }
04333
04334 /*
04335     #] ReadSaveIndex :
04336     #[ ReadSaveVariables :
04337 */
04338
04369 WORD ReadSaveVariables(UBYTE *buffer, UBYTE *top, LONG *size, LONG *outsize, \
04370     INDEXENTRY *ind, LONG *stage)
04371 {
04372     /* do we need some translation for the variables? */
04373     if ( AO.transFlag ) {
04374         /* counters for the number of already read symbols, indices, ... that
04375         need to remain valid between different calls to ReadSaveVariables().
04376         are initialized if stage == -1 */
04377         static WORD numReadSym;
04378         static WORD numReadInd;
04379         static WORD numReadVec;
04380         static WORD numReadFun;
04381
04382         POSITION pos;
04383         UBYTE *in, *out, *pp = 0, *end, *outbuf;
04384         LONG numread;
04385         WORD namelen, realnamelen;
04386         /* shortcuts */
04387         WORD lenW = AO.SaveHeader.lenWORD;
04388         WORD lenL = AO.SaveHeader.lenLONG;
04389         WORD lenP = AO.SaveHeader.lenPOINTER;
04390         WORD flip = AO.transFlag & 1;
04391
04392         /* remember file position in case we have to rewind */
04393         TELLFILE(AO.SaveData.Handle,&pos);
04394
04395         /* decide on the position of the in and out buffers.
04396         if the input is "bigger" than the output, we resize in-place, i.e.
04397         we immediately overwrite the source data by the translated data. in
04398         and out buffers start at the same place.
04399         if not, we read from the end of the given buffer and write at the
04400         beginning. */
04401         if ( (lenW > (WORD)sizeof(WORD))
04402         || ( (lenW == (WORD)sizeof(WORD))
04403             && ( (lenL > (WORD)sizeof(LONG))
04404                 || ( (lenL == (WORD)sizeof(LONG)) && lenP > (WORD)sizeof(void *))
04405             )
04406         ) ) {
04407             in = out = buffer;
04408             end = buffer + *size;
04409         }
04410         else {
04411             /* data will grow roughly by sizeof(WORD)/lenW. the exact value is
04412             not important. if reading and writing areas start to overlap, the
04413             reading will already be near the end of the data and overwriting
04414             doesn't matter. */
04415             LONG newsize = (top - buffer) / (1 + sizeof(WORD)/lenW);
04416             end = top;
04417             out = buffer;
04418             in = end - newsize;
04419             if ( *size > newsize ) *size = newsize;
04420         }
04421
04422         if ( ( numread = ReadFile(AO.SaveData.Handle, in, *size) ) != *size ) {
04423             return ( MesPrint("Error(2) reading stored expression.") );
04424         }
04425
04426         *size = 0;
04427         *outsize = 0;
04428
04429         /* first time in ReadSaveVariables(). initialize counters. */
04430         if ( *stage == -1 ) {
04431             numReadSym = 0;
04432             numReadInd = 0;
04433             numReadVec = 0;
04434             numReadFun = 0;
04435             ++*stage;
04436         }

```

```

04437
04438     while ( in < end ) {
04439         /* Symbols */
04440         if ( *stage == 0 ) {
04441             if ( ind->nsymbols <= numReadSym ) {
04442                 ++stage;
04443                 continue;
04444             }
04445             if ( end - in < AO.SaveHeader.sSym ) {
04446                 goto RSVEnd;
04447             }
04448             if ( flip ) {
04449                 pp = in;
04450                 AO.FlipLONG(pp); pp += lenL;
04451                 while ( pp < in + AO.SaveHeader.sSym ) {
04452                     AO.FlipWORD(pp); pp += lenW;
04453                 }
04454             }
04455             pp = in + AO.SaveHeader.sSym;
04456             AO.ResizeLONG(in, out); in += lenL; out += sizeof(LONG); /* name */
04457             AO.CheckPower(in);
04458             AO.ResizeWORD(in, out); in += lenW;
04459             if ( *((WORD *)out) == -AO.mpower ) *((WORD *)out) = -MAXPOWER;
04460             out += sizeof(WORD); /* minpower */
04461             AO.ResizeWORD(in, out); in += lenW;
04462             if ( *((WORD *)out) == AO.mpower ) *((WORD *)out) = MAXPOWER;
04463             out += sizeof(WORD); /* maxpower */
04464             AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* complex */
04465             AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* number */
04466             AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* flags */
04467             AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* node */
04468             AO.ResizeWORD(in, out); in += lenW; /* namesize */
04469             realnamelen = *((WORD *)out);
04470             realnamelen += sizeof(void *)-1; realnamelen &= -(sizeof(void *));
04471             out += sizeof(WORD);
04472             AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* dimension */
04473             while ( in < pp ) {
04474                 AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD);
04475             }
04476             namelen = *((WORD *)out-1); /* cares for padding "bug" */
04477             if ( end - in < namelen ) {
04478                 goto RSVEnd;
04479             }
04480             *((WORD *)out-1) = realnamelen;
04481             *size += AO.SaveHeader.sSym + namelen;
04482             *outsize += sizeof(struct SyMbOl) + realnamelen;
04483             if ( realnamelen > namelen ) {
04484                 int j = namelen;
04485                 NCOPYB(out, in, j);
04486                 out += realnamelen - namelen;
04487             }
04488             else {
04489                 int j = realnamelen;
04490                 NCOPYB(out, in, j);
04491                 in += namelen - realnamelen;
04492             }
04493             ++numReadSym;
04494             continue;
04495         }
04496         /* Indices */
04497         if ( *stage == 1 ) {
04498             if ( ind->nindices <= numReadInd ) {
04499                 ++stage;
04500                 continue;
04501             }
04502             if ( end - in < AO.SaveHeader.sInd ) {
04503                 goto RSVEnd;
04504             }
04505             if ( flip ) {
04506                 pp = in;
04507                 AO.FlipLONG(pp); pp += lenL;
04508                 while ( pp < in + AO.SaveHeader.sInd ) {
04509                     AO.FlipWORD(pp); pp += lenW;
04510                 }
04511             }
04512             pp = in + AO.SaveHeader.sInd;
04513             AO.ResizeLONG(in, out); in += lenL; out += sizeof(LONG); /* name */
04514             AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* type */
04515             AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* dimension */
04516             AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* number */
04517             AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* flags */
04518             AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* nmin4 */
04519             AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* node */
04520             AO.ResizeWORD(in, out); in += lenW; /* namesize */
04521             realnamelen = *((WORD *)out);
04522             realnamelen += sizeof(void *)-1; realnamelen &= -(sizeof(void *));
04523             out += sizeof(WORD);

```

```

04524         while ( in < pp ) {
04525             AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD);
04526         }
04527         namelen = *((WORD *)out-1); /* cares for padding "bug" */
04528         if ( end - in < namelen ) {
04529             goto RSVEnd;
04530         }
04531         *((WORD *)out-1) = realnamelen;
04532         *size += AO.SaveHeader.sInd + namelen;
04533         *outsize += sizeof(struct InDeX) + realnamelen;
04534         if ( realnamelen > namelen ) {
04535             int j = namelen;
04536             NCOPYB(out, in, j);
04537             out += realnamelen - namelen;
04538         }
04539         else {
04540             int j = realnamelen;
04541             NCOPYB(out, in, j);
04542             in += namelen - realnamelen;
04543         }
04544         ++numReadInd;
04545         continue;
04546     }
04547     /* Vectors */
04548     if ( *stage == 2 ) {
04549         if ( ind->nvectors <= numReadVec ) {
04550             ++*stage;
04551             continue;
04552         }
04553         if ( end - in < AO.SaveHeader.sVec ) {
04554             goto RSVEnd;
04555         }
04556         if ( flip ) {
04557             pp = in;
04558             AO.FlipLONG(pp); pp += lenL;
04559             while ( pp < in + AO.SaveHeader.sVec ) {
04560                 AO.FlipWORD(pp); pp += lenW;
04561             }
04562         }
04563         pp = in + AO.SaveHeader.sVec;
04564         AO.ResizeLONG(in, out); in += lenL; out += sizeof(LONG); /* name */
04565         AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* complex */
04566         AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* number */
04567         AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* flags */
04568         AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* node */
04569         AO.ResizeWORD(in, out); in += lenW; /* namesize */
04570         realnamelen = *((WORD *)out);
04571         realnamelen += sizeof(void *)-1; realnamelen &= -(sizeof(void *));
04572         out += sizeof(WORD);
04573         AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* dimension */
04574         while ( in < pp ) {
04575             AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD);
04576         }
04577         namelen = *((WORD *)out-1); /* cares for padding "bug" */
04578         if ( end - in < namelen ) {
04579             goto RSVEnd;
04580         }
04581         *((WORD *)out-1) = realnamelen;
04582         *size += AO.SaveHeader.sVec + namelen;
04583         *outsize += sizeof(struct VeCtOr) + realnamelen;
04584         if ( realnamelen > namelen ) {
04585             int j = namelen;
04586             NCOPYB(out, in, j);
04587             out += realnamelen - namelen;
04588         }
04589         else {
04590             int j = realnamelen;
04591             NCOPYB(out, in, j);
04592             in += namelen - realnamelen;
04593         }
04594         ++numReadVec;
04595         continue;
04596     }
04597     /* Functions */
04598     if ( *stage == 3 ) {
04599         if ( ind->nfunctions <= numReadFun ) {
04600             ++*stage;
04601             continue;
04602         }
04603         if ( end - in < AO.SaveHeader.sFun ) {
04604             goto RSVEnd;
04605         }
04606         if ( flip ) {
04607             pp = in;
04608             AO.FlipPOINTER(pp); pp += lenP;
04609             AO.FlipLONG(pp); pp += lenL;
04610             AO.FlipLONG(pp); pp += lenL;

```

```

04611         while ( pp < in + AO.SaveHeader.sFun ) {
04612             AO.FlipWORD(pp); pp += lenW;
04613         }
04614     }
04615     pp = in + AO.SaveHeader.sFun;
04616     outbuf = out;
04617     AO.ResizePOINTER(in, out); in += lenP; out += sizeof(void *); /* tabl */
04618     AO.ResizeLONG(in, out); in += lenL; out += sizeof(LONG); /* symminfo */
04619     AO.ResizeLONG(in, out); in += lenL; out += sizeof(LONG); /* name */
04620     AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* commute */
04621     AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* complex */
04622     AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* number */
04623     AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* flags */
04624     AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* spec */
04625     AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* symmetric */
04626     AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* numargs */
04627     AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* node */
04628     AO.ResizeWORD(in, out); in += lenW; /* namesize */
04629     realnamelen = *((WORD *)out);
04630     realnamelen += sizeof(void *)-1; realnamelen &= -(sizeof(void *));
04631     out += sizeof(WORD);
04632     AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD); /* dimension */
04633     while ( in < pp ) {
04634         AO.ResizeWORD(in, out); in += lenW; out += sizeof(WORD);
04635     }
04636     namelen = *((WORD *)out-1); /* cares for padding "bug" */
04637     if ( end - in < namelen ) {
04638         goto RSVEnd;
04639     }
04640     *((WORD *)out-1) = realnamelen;
04641     *size += AO.SaveHeader.sFun + namelen;
04642     *outsize += sizeof(struct FuNcTiOn) + realnamelen;
04643     if ( realnamelen > namelen ) {
04644         int j = namelen;
04645         NCOPYB(out, in, j);
04646         out += realnamelen - namelen;
04647     }
04648     else {
04649         int j = realnamelen;
04650         NCOPYB(out, in, j);
04651         in += namelen - realnamelen;
04652     }
04653     ++numReadFun;
04654     /* we use the information whether a function is tensorial later in ReadSaveTerm */
04655     AO.tensorList[((FUNCTIONS)outbuf)->number+FUNCTION] =
04656         (UBYTE)((FUNCTIONS)outbuf)->spec == TENSORFUNCTION);
04657     continue;
04658 }
04659 /* handle numdummies */
04660 if ( end - in >= lenW ) {
04661     if ( flip ) AO.FlipWORD(in);
04662     AO.ResizeWORD(in, out);
04663     *size += lenW;
04664     *outsize += sizeof(WORD);
04665 }
04666 /* handle numfactors */
04667 if ( end - in >= lenW ) {
04668     if ( flip ) AO.FlipWORD(in);
04669     AO.ResizeWORD(in, out);
04670     *size += lenW;
04671     *outsize += sizeof(WORD);
04672 }
04673 /* handle vflags */
04674 if ( end - in >= lenW ) {
04675     if ( flip ) AO.FlipWORD(in);
04676     AO.ResizeWORD(in, out);
04677     *size += lenW;
04678     *outsize += sizeof(WORD);
04679 }
04680 /* handle uflags */
04681 if ( end - in >= lenW ) {
04682     if ( flip ) AO.FlipWORD(in);
04683     AO.ResizeWORD(in, out);
04684     *size += lenW;
04685     *outsize += sizeof(WORD);
04686 }
04687 return ( 0 );
04688 }
04689
04690 RSVEnd:
04691 /* we are here because the remaining buffer cannot hold the next
04692 struct. we position the file behind the last successfully translated
04693 struct and return. */
04694 ADDPOS(pos, *size);
04695 SeekFile(AO.SaveData.Handle, &pos, SEEK_SET);
04696 return ( 0 );
04697 } else {

```

```

04698         return ( ReadFile(AO.SaveData.Handle, buffer, *size) != *size );
04699     }
04700 }
04701
04702 /*
04703     #] ReadSaveVariables :
04704     #[ ReadSaveTerm :
04705 */
04706
04735 UBYTE *
04736 ReadSaveTerm32(UBYTE *bin, UBYTE *binend, UBYTE **bout, UBYTE *boutend, UBYTE *top, int terminbuf)
04737 {
04738     GETIDENTITY
04739
04740     UBYTE *boutbuf;
04741     INT32 len, j, id;
04742     INT32 *r, *t, *coeff, *end, *newtermsize, *rend;
04743     INT32 *newsubterm;
04744     INT32 *in = (INT32 *)bin;
04745     INT32 *out = (INT32 *)*bout;
04746
04747     /* if called recursively the term is already decompressed in buffer.
04748        is this the case? */
04749     if ( terminbuf ) {
04750         /* don't do any decompression, just adjust the pointers */
04751         len = *out;
04752         end = out + len;
04753         r = in + 1;
04754         rend = (INT32 *)boutend;
04755         coeff = end - ABS(*end-1);
04756         newtermsize = (INT32 *)*bout;
04757         out = newtermsize + 1;
04758     }
04759     else {
04760         /* do decompression of necessary. always return if the space in the
04761            buffer is not sufficient */
04762         INT32 rbuf;
04763         r = (INT32 *)AR.CompressBuffer;
04764         rbuf = *r;
04765         len = j = *in;
04766         /* first copy from AR.CompressBuffer if necessary */
04767         if ( j < 0 ) {
04768             ++in;
04769             if ( (UBYTE *)in >= binend ) {
04770                 return ( bin );
04771             }
04772             *out = len = -j + 1 + *in;
04773             end = out + *out;
04774             if ( (UBYTE *)end >= top ) {
04775                 return ( bin );
04776             }
04777             ++out;
04778             *r++ = len;
04779             while ( ++j <= 0 ) {
04780                 INT32 bb = *r++;
04781                 *out++ = bb;
04782             }
04783             j = *in++;
04784         }
04785         else if ( j == 0 ) {
04786             /* care for padding words */
04787             while ( (UBYTE *)in < binend ) {
04788                 *out++ = 0;
04789                 if ( (UBYTE *)out > top ) {
04790                     return ( (UBYTE *)bin );
04791                 }
04792             }
04793             *r++ = 0;
04794             ++in;
04795         }
04796         *bout = (UBYTE *)out;
04797         return ( (UBYTE *)in );
04798     }
04799     else {
04800         end = out + len;
04801         if ( (UBYTE *)end >= top ) {
04802             return ( bin );
04803         }
04804     }
04805     if ( (UBYTE *)in + j >= binend ) {
04806         *(AR.CompressBuffer) = rbuf;
04807         return ( bin );
04808     }
04809     if ( (UBYTE *)out + j >= top ) {
04810         return ( bin );
04811     }
04812     /* second copy from input buffer */

```



```

04813     while ( --j >= 0 ) {
04814         INT32 bb = *in++;
04815         *r++ = *out++ = bb;
04816     }
04817
04818     rend = r;
04819     r = (INT32 *)AR.CompressBuffer + 1;
04820     coeff = end - ABS(*out-1);
04821     newtermsize = (INT32 *)*bout;
04822     out = newtermsize + 1;
04823 }
04824
04825 /* iterate over subterms */
04826 while ( out < coeff ) {
04827
04828     id = *out++;
04829     ++r;
04830     t = out + *out - 1;
04831     newsubtermp = out;
04832     ++out; ++r;
04833
04834     if ( id == SYMBOL ) {
04835         while ( out < t ) {
04836             ++out; ++r; /* symbol number */
04837             /* if exponent is too big, rewrite as exponent function */
04838             if ( ABS(*out) >= MAXPOWER ) {
04839                 INT32 *a, *b;
04840                 INT32 n;
04841                 INT32 num = *(out-1);
04842                 INT32 exp = *out;
04843                 coeff += 9;
04844                 end += 9;
04845                 t += 9;
04846                 if ( (UBYTE *)end > top ) return ( bin );
04847                 out -= 3;
04848                 *out++ = EXPONENT; /* id */
04849                 *out++ = 13; /* size */
04850                 *out++ = 1; /* dirtyflag */
04851                 *out++ = -SYMBOL; /* first short arg */
04852                 *out++ = num;
04853                 *out++ = 8; /* second arg, size */
04854                 *out++ = 0; /* dirtyflag */
04855                 *out++ = 6; /* term size */
04856                 *out++ = ABS(exp) & 0x0000FFFF;
04857                 *out++ = ABS(exp) » 16;
04858                 *out++ = 1;
04859                 *out++ = 0;
04860                 *out++ = ( exp < 0 ) ? -5 : 5;
04861                 a = ++r;
04862                 b = out;
04863                 n = rend - r;
04864                 NCOPYI32(b, a, n)
04865             }
04866             else {
04867                 ++out; ++r;
04868             }
04869         }
04870     }
04871     else if ( id == DOTPRODUCT ) {
04872         while ( out < t ) {
04873             AO.RenumberVec((UBYTE *)out); /* vector 1 */
04874             ++out; ++r;
04875             AO.RenumberVec((UBYTE *)out); /* vector 2 */
04876             ++out; ++r;
04877             /* if exponent is too big, rewrite as exponent function */
04878             if ( ABS(*out) >= MAXPOWER ) {
04879                 INT32 *a, *b;
04880                 INT32 n;
04881                 INT32 num1 = *(out-2);
04882                 INT32 num2 = *(out-1);
04883                 INT32 exp = *out;
04884                 coeff += 17;
04885                 end += 17;
04886                 t += 17;
04887                 if ( (UBYTE *)end > top ) return ( bin );
04888                 out -= 4;
04889                 *out++ = EXPONENT; /* id */
04890                 *out++ = 22; /* size */
04891                 *out++ = 1; /* dirtyflag */
04892                 *out++ = 11; /* first arg, size */
04893                 *out++ = 0; /* dirtyflag */
04894                 *out++ = 9; /* term size */
04895                 *out++ = DOTPRODUCT; /* p1.p2 */
04896                 *out++ = 5; /* subterm size */
04897                 *out++ = num1; /* p1 */
04898                 *out++ = num2; /* p2 */
04899                 *out++ = 1; /* exponent */

```

```

04900             *out++ = 1;                /* coeff */
04901             *out++ = 1;
04902             *out++ = 3;
04903             *out++ = 8;                /* second arg, size */
04904             *out++ = 0;                /* dirtyflag */
04905             *out++ = 6;                /* term size */
04906             *out++ = ABS(exp) & 0x0000FFFF;
04907             *out++ = ABS(exp) » 16;
04908             *out++ = 1;
04909             *out++ = 0;
04910             *out++ = ( exp < 0 ) ? -5 : 5;
04911             a = ++r;
04912             b = out;
04913             n = rend - r;
04914             NCOPYI32(b, a, n)
04915         }
04916         else {
04917             ++out; ++r;
04918         }
04919     }
04920 }
04921 else if ( id == VECTOR ) {
04922     while ( out < t ) {
04923         AO.RenumberVec((UBYTE *)out); /* vector number */
04924         ++out; ++r;
04925         ++out; ++r; /* index, do nothing */
04926     }
04927 }
04928 else if ( id == INDEX ) {
04929     /* INT32 vectoroffset = -2 * *((INT32 *)AO.SaveHeader.wildoffset); */
04930     void *dummy = (void *)AO.SaveHeader.wildoffset; /* to remove a warning about
strict-aliasing rules in gcc */
04931     INT32 vectoroffset = -2 * *((INT32 *)dummy);
04932     while ( out < t ) {
04933         /* if there is a vector, renumber it */
04934         if ( *out < vectoroffset ) {
04935             AO.RenumberVec((UBYTE *)out);
04936         }
04937         ++out; ++r;
04938     }
04939 }
04940 else if ( id == SUBEXPRESSION ) {
04941     /* nothing to translate */
04942     while ( out < t ) {
04943         ++out; ++r;
04944     }
04945 }
04946 else if ( id == DELTA ) {
04947     /* nothing to translate */
04948     r += t - out;
04949     out = t;
04950 }
04951 else if ( id == HAAKJE ) {
04952     /* nothing to translate */
04953     r += t - out;
04954     out = t;
04955 }
04956 else if ( id == GAMMA || id == LEVICIVITA || (id >= FUNCTION && AO.tensorList[id]) ) {
04957     /* INT32 vectoroffset = -2 * *((INT32 *)AO.SaveHeader.wildoffset); */
04958     void *dummy = (void *)AO.SaveHeader.wildoffset; /* to remove a warning about
strict-aliasing rules in gcc */
04959     INT32 vectoroffset = -2 * *((INT32 *)dummy);
04960     while ( out < t ) {
04961         /* if there is a vector as an argument, renumber it */
04962         if ( *out < vectoroffset ) {
04963             AO.RenumberVec((UBYTE *)out);
04964         }
04965         ++out; ++r;
04966     }
04967 }
04968 else if ( id >= FUNCTION ) {
04969     INT32 *argEnd;
04970     UBYTE *newbin;
04971
04972     ++out; ++r; /* dirty flags */
04973
04974     /* loop over arguments */
04975     while ( out < t ) {
04976         if ( *out < 0 ) {
04977             /* short notation arguments */
04978             switch ( -*out ) {
04979                 case SYMBOL:
04980                     ++out; ++r;
04981                     ++out; ++r;
04982                     break;
04983                 case SNUMBER:
04984                     argEnd = out+2;

```

```

04985         ++out; ++r;
04986         if ( sizeof(WORD) == 2 ) {
04987             /* resize if needed */
04988             if ( *out > (1<15)-1 || *out < -(1<15)+1 ) {
04989                 INT32 *a, *b;
04990                 INT32 n;
04991                 INT32 num = *out;
04992                 coeff += 6;
04993                 end += 6;
04994                 argEnd += 6;
04995                 t += 6;
04996                 if ( (UBYTE *)end > top ) return ( bin );
04997                 --out;
04998                 *out++ = 8;          /* argument size */
04999                 *out++ = 0;          /* dirtyflag */
05000                 *out++ = 6;          /* term size */
05001                 *out++ = ABS(num) & 0x0000FFFF;
05002                 *out++ = ABS(num) » 16;
05003                 *out++ = 1;
05004                 *out++ = 0;
05005                 *out++ = ( num < 0 ) ? -5 : 5;
05006                 a = ++r;
05007                 b = out;
05008                 n = rend - r;
05009                 NCOPYI32(b, a, n)
05010             }
05011             else {
05012                 ++out; ++r;
05013             }
05014         }
05015         else {
05016             ++out; ++r;
05017         }
05018         break;
05019     case VECTOR:
05020         ++out; ++r;
05021         AO.RenumberVec((UBYTE *)out);
05022         ++out; ++r;
05023         break;
05024     case INDEX:
05025         ++out; ++r;
05026         ++out; ++r;
05027         break;
05028     case MINVECTOR:
05029         ++out; ++r;
05030         AO.RenumberVec((UBYTE *)out);
05031         ++out; ++r;
05032         break;
05033     default:
05034         if ( -*out >= FUNCTION ) {
05035             ++out; ++r;
05036             break;
05037         } else {
05038             MesPrint("short function code %d not implemented.", *out);
05039             return ( (UBYTE *)in );
05040         }
05041     }
05042 }
05043 else {
05044     /* long arguments */
05045     INT32 *newargsize = out;
05046     argEnd = out + *out;
05047     ++out; ++r;
05048     ++out; ++r; /* dirty flags */
05049     while ( out < argEnd ) {
05050         INT32 *keepsizetp = out + *out;
05051         INT32 lenbuf = *out;
05052         INT32 *ppp = &out; /* to avoid a compiler warning */
05053         /* recursion */
05054         newbin = ReadSaveTerm32((UBYTE *)r, binend, (UBYTE **)ppp, (UBYTE *)rend, top,
1);
05055         r += lenbuf;
05056         if ( newbin == (UBYTE *)r ) {
05057             return ( (UBYTE *)in );
05058         }
05059         /* if the term done by recursion has changed in size,
05060            we need to move the rest of the data accordingly */
05061         if ( out > keepsizetp ) {
05062             INT32 *a, *b;
05063             INT32 n;
05064             INT32 extention = out - keepsizetp;
05065             a = r;
05066             b = out;
05067             n = rend - r;
05068             NCOPYI32(b, a, n)
05069             coeff += extention;
05070             end += extention;

```

```

05071             argEnd += extention;
05072             t += extention;
05073         }
05074         else if ( out < keepsizep ) {
05075             INT32 *a, *b;
05076             INT32 n;
05077             INT32 extention = keepsizep - out;
05078             a = keepsizep;
05079             b = out;
05080             n = rend - r;
05081             NCOPYI32(b, a, n)
05082             coeff -= extention;
05083             end -= extention;
05084             argEnd -= extention;
05085             t -= extention;
05086         }
05087     }
05088     *newargsize = out - newargsize;
05089 }
05090 }
05091 }
05092 else {
05093     MesPrint("ID %d not recognized.", id);
05094     return ( (UBYTE *)in );
05095 }
05096
05097 *newsubtermp = out - newsubtermp + 1;
05098 }
05099
05100 if ( (UBYTE *)end >= top ) {
05101     return ( bin );
05102 }
05103
05104 /* do coefficient and adjust term size */
05105 boutbuf = *bout;
05106 *bout = (UBYTE *)out;
05107
05108 ResizeCoeff32(bout, (UBYTE *)end, top);
05109
05110 if ( *bout >= top ) {
05111     *bout = boutbuf;
05112     return ( bin );
05113 }
05114
05115 *newtermsize = (INT32 *)*bout - newtermsize;
05116
05117 return ( (UBYTE *)in );
05118 }
05119
05120 /*
05121     #] ReadSaveTerm :
05122     #[ ReadSaveExpression :
05123 */
05124
05146 WORD ReadSaveExpression(UBYTE *buffer, UBYTE *top, LONG *size, LONG *outsize)
05147 {
05148     if ( AO.transFlag ) {
05149         UBYTE *in, *end, *out, *outend, *p;
05150         POSITION pos;
05151         LONG half;
05152         WORD lenW = AO.SaveHeader.lenWORD;
05153
05154         /* remember the last file position in case an expression cannot be
05155            fully processed */
05156         TELLFILE(AO.SaveData.Handle,&pos);
05157
05158         /* adjust 'size' depending on whether the translated data is bigger or
05159            smaller */
05160         half = (top-buffer)/2;
05161         if ( *size > half ) *size = half;
05162         if ( lenW < (WORD)sizeof(WORD) ) {
05163             if ( *size * (LONG)sizeof(WORD)/lenW > half ) *size = half*lenW/(LONG)sizeof(WORD);
05164         }
05165         else {
05166             if ( *size > half ) *size = half;
05167         }
05168
05169         /* depending on the necessary resizing we position the input pointer
05170            either at the start of the buffer or in the middle. if the data will
05171            roughly remain the same size, we need only one processing step, so
05172            we put the 'in' at the middle and 'out' and the beginning. in the
05173            other cases we need two processing steps, so first we put 'in' at
05174            the beginning and write at the middle. the second step can then read
05175            from the middle and put its results at the beginning. */
05176         in = out = buffer;
05177         if ( lenW == sizeof(WORD) ) in += half;
05178         else out += half;

```

```

05179         end = in + *size;
05180         outend = out + *size;
05181
05182         if ( ReadFile(AO.SaveData.Handle, in, *size) != *size ) {
05183             return ( MesPrint("Error(3) reading stored expression.") );
05184         }
05185
05186         if ( AO.transFlag & 1 ) {
05187             p = in;
05188             end -= lenW;
05189             while ( p <= end ) {
05190                 AO.FlipWORD(p); p += lenW;
05191             }
05192             end += lenW;
05193         }
05194
05195         if ( lenW > (WORD)sizeof(WORD) ) {
05196             /* renumber first */
05197             do {
05198                 outend = out+*size;
05199                 if ( outend > top ) outend = top;
05200                 p = ReadSaveTerm32(in, end, &out, outend, top, 0);
05201                 if ( p == in ) break;
05202                 in = p;
05203             } while ( in <= end - lenW );
05204             /* then resize */
05205             *size = in - buffer;
05206             in = buffer + half;
05207             end = out;
05208             out = buffer;
05209
05210             while ( in < end ) {
05211                 /* resize without checking */
05212                 AO.ResizeNCWORD(in, out);
05213                 in += lenW; out += sizeof(WORD);
05214             }
05215         }
05216         else {
05217             if ( lenW < (WORD)sizeof(WORD) ) {
05218                 /* resize first */
05219                 while ( in < end ) {
05220                     AO.ResizeWORD(in, out);
05221                     in += lenW; out += sizeof(WORD);
05222                 }
05223                 in = buffer + half;
05224                 end = out;
05225                 out = buffer;
05226             }
05227             /* then renumber */
05228             do {
05229                 p = ReadSaveTerm32(in, end, &out, buffer+half, buffer+half, 0);
05230                 if ( p == in ) break;
05231                 in = p;
05232             } while ( in <= end - sizeof(WORD) );
05233             *size = (in - buffer - half) * lenW / (ULONG)sizeof(WORD);
05234         }
05235         *outsize = out - buffer;
05236         ADDPOS(pos, *size);
05237         SeekFile(AO.SaveData.Handle, &pos, SEEK_SET);
05238
05239         return ( 0 );
05240     }
05241     else {
05242         return ( ReadFile(AO.SaveData.Handle, buffer, *size) != *size );
05243     }
05244 }
05245
05246 /*
05247     #] ReadSaveExpression :
05248     #] System Independent Saved Expressions :
05249 */

```

4.136 structs.h File Reference

This graph shows which files directly or indirectly include this file:



Data Structures

- struct [PoSiTiOn](#)
- struct [STOREHEADER](#)
- struct [InDeXeNtRy](#)
- struct [FiLeInDeX](#)
- struct [FiLeDaTa](#)
- struct [VaRrEnUm](#)
- struct [ReNuMbEr](#)
- struct [LIST](#)
- struct [KEYWORD](#)
- struct [KEYWORDV](#)
- struct [NaMeNode](#)
- struct [NaMeTree](#)
- struct [tree](#)
- struct [MiNmAx](#)
- struct [BrAcKeTiNdEx](#)
- struct [BrAcKeTiNfO](#)
- struct [TaBIes](#)
- struct [ExPrEsSiOn](#)
- struct [SyMbOl](#)
- struct [InDeX](#)
- struct [VeCtOr](#)
- struct [FuNcTiOn](#)
- struct [SeTs](#)
- struct [DuBiOuS](#)
- struct [FaCdOILaR](#)
- struct [DoLIaRS](#)
- struct [MoDoPtDoLIaRS](#)
- struct [fixedset](#)
- struct [TaBIEbAsEsUbInDeX](#)
- struct [TaBIEbAsE](#)
- struct [FUN_INFO](#)
- struct [PaRtlcLe](#)
- struct [VeRtEx](#)
- struct [MoDeL](#)
- struct [NaMeSpAcE](#)
- struct [FiLe](#)
- struct [StreaM](#)
- struct [SpecTatoR](#)
- struct [TrAcEs](#)
- struct [TrAcEn](#)
- struct [pReVaR](#)
- struct [INSIDEINFO](#)
- struct [PRELOAD](#)
- struct [PROCEDURE](#)
- struct [DoLoOp](#)
- struct [bit_field](#)
- struct [HANDLERS](#)
- struct [CbUf](#)
- struct [ChAnNeL](#)
- struct [SETUPPARAMETERS](#)
- struct [NeStInG](#)
- struct [StOrEcAcHe](#)
- struct [PeRmUtE](#)

- struct [PeRmUtEp](#)
- struct [DiStRiBuTe](#)
- struct [PaRtl](#)
- struct [sOrT](#)
- struct [POLYMOD](#)
- struct [SHvariables](#)
- struct [COST](#)
- struct [MODNUM](#)
- struct [OPTIMIZE](#)
- struct [OPTIMIZERESULT](#)
- struct [DICTIONARY_ELEMENT](#)
- struct [DICTIONARY](#)
- struct [SWITCHTABLE](#)
- struct [SWITCH](#)
- struct [TERMINFO](#)
- struct [M_const](#)
- struct [P_const](#)
- struct [C_const](#)
- struct [S_const](#)
- struct [R_const](#)
- struct [T_const](#)
- struct [N_const](#)
- struct [O_const](#)
- struct [X_const](#)
- struct [AllGlobals](#)
- struct [FixedGlobals](#)

Macros

- #define [INFILEINDEX](#) ((512-2*sizeof([POSITION](#)))/sizeof([INDEXENTRY](#)))
- #define [EMPTYININDEX](#) (512-2*sizeof([POSITION](#))-[INFILEINDEX](#)*sizeof([INDEXENTRY](#)))
- #define [PHEAD](#)
- #define [PHEAD0](#) void
- #define [BHEAD](#)
- #define [BHEAD0](#)

Typedefs

- typedef struct [PoSiTiOn](#) **POSITION**
- typedef struct [InDeXeNtRy](#) **INDEXENTRY**
- typedef struct [FiLeInDeX](#) **FILEINDEX**
- typedef struct [FiLeDaTa](#) **FILEDATA**
- typedef struct [VaRrEnUm](#) **VARRENUM**
- typedef struct [ReNuMbEr](#) * **RENUMBER**
- typedef struct [NaMeNode](#) **NAMENODE**
- typedef struct [NaMeTree](#) **NAMETREE**
- typedef struct [tree](#) **COMPTREE**
- typedef struct [MiNmAx](#) **MINMAX**
- typedef struct [BrAcKeTiNdEx](#) **BRACKETINDEX**
- typedef struct [BrAcKeTiNfO](#) **BRACKETINFO**
- typedef struct [TaBIes](#) * **TABLES**
- typedef struct [ExPrEsSiOn](#) * **EXPRESSIONS**
- typedef struct [SyMbOl](#) * **SYMBOLS**

- typedef struct [InDeX](#) * **INDICES**
- typedef struct [VeCtOr](#) * **VECTORS**
- typedef struct [FuNcTiOn](#) * **FUNCTIONS**
- typedef struct [SeTs](#) * **SETS**
- typedef struct [DuBiOuS](#) * **DUBIOUSV**
- typedef struct [FaCdOILaR](#) **FACDOLLAR**
- typedef struct [DoLIaR\\$](#) * **DOLLARS**
- typedef struct [MoDoPtDoLIaR\\$](#) **MODOPTDOLLAR**
- typedef struct [fixedset](#) **FIXEDSET**
- typedef struct [TaBIeBa\\$EsUbInDeX](#) **TABLEBASESUBINDEX**
- typedef struct [TaBIeBa\\$E](#) **TABLEBASE**
- typedef struct [PaRtlcLe](#) **PARTICLE**
- typedef struct [VeRtEx](#) **VERTEX**
- typedef struct [MoDeL](#) **MODEL**
- typedef struct [NaMeSpAcE](#) **NAMESPACE**
- typedef struct [FiLe](#) **FILEHANDLE**
- typedef struct [StreaM](#) **STREAM**
- typedef struct [SpecTatoR](#) **SPECTATOR**
- typedef struct [TrAcEs](#) **TRACES**
- typedef struct [TrAcEn](#) * **TRACEN**
- typedef struct [pReVaR](#) **PREVAR**
- typedef struct [DoLoOp](#) **DOLOOP**
- typedef struct [bit_field](#) [set_of_char](#)[32]
- typedef struct [bit_field](#) * [one_byte](#)
- typedef WORD(* [FINISHUFFLE](#)) (WORD *)
- typedef WORD(* [DO_UFFLE](#)) (WORD *, WORD, WORD, WORD)
- typedef WORD(* [COMPAREDUMMY](#)) (WORD *, WORD *, WORD)
- typedef struct [CbUf](#) **CBUF**
- typedef struct [ChAnNeL](#) **CHANNEL**
- typedef struct [NeStInG](#) * **NESTING**
- typedef struct [StOrEcAcHe](#) * **STORECACHE**
- typedef struct [PeRmUtE](#) **PERM**
- typedef struct [PeRmUtEp](#) **PERMP**
- typedef struct [DiStRiBuTe](#) **DISTRIBUTE**
- typedef struct [PaRtl](#) **PARTI**
- typedef struct [sOrT](#) **SORTING**
- typedef struct [AllGlobals](#) **ALLGLOBALS**
- typedef WORD(* [WCN](#)) ()
- typedef WORD(* [WCN2](#)) ()
- typedef WORD(* [COMPARE](#)) (PHEAD WORD *, WORD *, WORD)
- typedef struct [FixedGlobals](#) **FIXEDGLOBALS**

Functions

- **STATIC_ASSERT** (sizeof([STOREHEADER](#))==512)
- **STATIC_ASSERT** (sizeof([FILEINDEX](#))==512)

4.136.1 Detailed Description

Contains definitions for global structs.

!!!CAUTION!!! Changes in this file will most likely have consequences for the recovery mechanism (see [checkpoint.c](#)). You need to care for the code in [checkpoint.c](#) as well and modify the code there accordingly!

The marker [D] is used in comments in this file to mark pointers to which dynamically allocated memory is assigned by a call to malloc() during runtime (in contrast to pointers that point into already allocated memory). This information is especially helpful if one needs to know which pointers need to be freed (cf. [checkpoint.c](#)).

Definition in file [structs.h](#).

4.136.2 Macro Definition Documentation

4.136.2.1 INFILEINDEX

```
#define INFILEINDEX ((512-2*sizeof(POSITION))/sizeof(INDEXENTRY))
```

Maximum number of entries (struct [InDeXeNtRy](#)) in a file index (struct [FiLeInDeX](#)). Number is calculated such that the size of a file index is no more than 512 bytes.

Definition at line [119](#) of file [structs.h](#).

4.136.2.2 EMPTYININDEX

```
#define EMPTYININDEX (512-2*sizeof(POSITION)-INFILEINDEX*sizeof(INDEXENTRY))
```

Number of empty filling bytes for a file index (struct [FiLeInDeX](#)). It is calculated such that the size of a file index is always 512 bytes.

Definition at line [124](#) of file [structs.h](#).

4.136.2.3 PHEAD

```
#define PHEAD
```

Definition at line [2647](#) of file [structs.h](#).

4.136.2.4 PHEAD0

```
#define PHEAD0 void
```

Definition at line [2648](#) of file [structs.h](#).

4.136.2.5 BHEAD

```
#define BHEAD
```

Definition at line [2649](#) of file [structs.h](#).

4.136.2.6 BHEAD0

```
#define BHEAD0
```

Definition at line [2650](#) of file [structs.h](#).

4.136.3 Typedef Documentation

4.136.3.1 INDEXENTRY

```
typedef struct InDeXeNtRy INDEXENTRY
```

Defines the structure of an entry in a file index (see struct [FiLeInDeX](#)).

It represents one expression in the file.

4.136.3.2 FILEINDEX

```
typedef struct FiLeInDeX FILEINDEX
```

Defines the structure of a file index in store-files and save-files.

It contains several entries (see struct [InDeXeNtRy](#)) up to a maximum of [INFILEINDEX](#).

The variable number has been made of type POSITION to avoid padding problems with some types of computers/↔ OS and keep system independence of the .sav files.

This struct is always 512 bytes long.

4.136.3.3 VARRENUM

```
typedef struct VaRrEnUm VARRENUM
```

Contains the pointers to an array in which a binary search will be performed.

4.136.3.4 RENUMBER

```
typedef struct ReNuMbEr * RENUMBER
```

Only symb.lo gets dynamically allocated. All other pointers points into this memory.

4.136.3.5 NAMENODE

```
typedef struct NaMeNode NAMENODE
```

The names of variables are kept in an array. Elements of type [NAMENODE](#) define a tree (that is kept balanced) that make it easy and fast to look for variables. See also [NAMETREE](#).

4.136.3.6 NAMETREE

```
typedef struct NaMeTree NAMETREE
```

A struct of type [NAMETREE](#) controls a complete (balanced) tree of names for the compiler. The compiler maintains several of such trees and the system has been set up in such a way that one could define more of them if we ever want to work with local name spaces.

4.136.3.7 COMPTREE

```
typedef struct tree COMPTREE
```

The subexpressions in the compiler are kept track of in a (balanced) tree to reduce the need for subexpressions and hence save much space in large rhs expressions (like when we have xxxxxx occurrences of objects like $f(x+1, x+1)$ in which each $x+1$ becomes a subexpression. The struct that controls this tree is COMPTREE.

4.136.3.8 TABLES

```
typedef struct TaBleS * TABLES
```

buffers, mm, flags, and prototype are always dynamically allocated, tablepointers only if needed (=0 if unallocated), boomlijst and argtail only for sparse tables.

Allocation is done for both the normal and the stub instance (spare), except for prototype and argtail which share memory.

4.136.3.9 FUNCTIONS

```
typedef struct FuNcTiOn * FUNCTIONS
```

Contains all information about a function. Also used for tables. It is used in the [LIST](#) elements of AC.

4.136.3.10 FILEHANDLE

```
typedef struct FiLe FILEHANDLE
```

The type FILEHANDLE is the struct that controls all relevant information of a file, whether it is open or not. The file may even not yet exist. There is a system of caches (PObuffer) and as long as the information to be written still fits inside the cache the file may never be created. There are variables that can store information about different types of files, like scratch files or sort files. Depending on what is available in the system we may also have information about gzip compression (currently sort file only) or locks (TFORM).

4.136.3.11 STREAM

```
typedef struct StreaM STREAM
```

Input is read from 'streams' which are represented by objects of type STREAM. A stream can be a file, a do-loop, a procedure, the string value of a preprocessor variable When a new stream is opened we have to keep information about where to fall back in the parent stream to allow this to happen even in the middle of reading names etc as would be the case with a`i`b

4.136.3.12 TRACES

```
typedef struct TrAcEs TRACES
```

The struct TRACES keeps track of the progress during the expansion of a 4-dimensional trace. Each time a term gets generated the expansion tree continues in the next statement. When it returns it has to know where to continue. The 4-dimensional traces are more complicated than the n-dimensional traces (see TRACEN) because of the extra tricks that can be used. They are responsible for the shorter final expressions.

4.136.3.13 TRACEN

```
typedef struct TrAcEn * TRACEN
```

The struct TRACEN keeps track of the progress during the expansion of a 4-dimensional trace. Each time a term gets generated the expansion tree continues in the next statement. When it returns it has to know where to continue.

4.136.3.14 PREVAR

```
typedef struct pReVaR PREVAR
```

An element of the type PREVAR is needed for each preprocessor variable.

4.136.3.15 DOLOOP

```
typedef struct DoLoOp DOLOOP
```

Each preprocessor do loop has a struct of type DOLOOP to keep track of all relevant parameters like where the beginning of the loop is, what the boundaries, increment and value of the loop parameter are, etc. Also we keep the whole loop inside a buffer of type [PRELOAD](#)

4.136.3.16 set_of_char

```
typedef struct bit_field set_of_char[32]
```

Used in `set_in`, `set_set`, `set_del` and `set_sub`.

Definition at line [952](#) of file [structs.h](#).

4.136.3.17 one_byte

```
typedef struct bit_field* one_byte
```

Used in `set_in`, `set_set`, `set_del` and `set_sub`.

Definition at line [958](#) of file [structs.h](#).

4.136.3.18 FINISHUFFLE

```
typedef WORD(* FINISHUFFLE) (WORD *)
```

Definition at line [978](#) of file [structs.h](#).

4.136.3.19 DO_UFFLE

```
typedef WORD(* DO_UFFLE) (WORD *, WORD, WORD, WORD)
```

Definition at line [979](#) of file [structs.h](#).

4.136.3.20 COMPAREDUMMY

```
typedef WORD(* COMPAREDUMMY) (WORD *, WORD *, WORD)
```

Definition at line 984 of file [structs.h](#).

4.136.3.21 CBUF

```
typedef struct CbUf CBUF
```

The CBUF struct is used by the compiler. It is a compiler buffer of which since version 3.0 there can be many.

4.136.3.22 CHANNEL

```
typedef struct ChAnNeL CHANNEL
```

When we read input from text files we have to remember not only their handle but also their name. This is needed for error messages. Hence we call such a file a channel and reserve a struct of type [CHANNEL](#) to allow to lay this link.

4.136.3.23 NESTING

```
typedef struct NeStInG * NESTING
```

The NESTING struct is used when we enter the argument of functions and there is the possibility that we have to change something there. Because functions can be nested we have to keep track of all levels of functions in case we have to move the outer layers to make room for a larger function argument.

4.136.3.24 STORECACHE

```
typedef struct StOrEcAcHe * STORECACHE
```

The struct of type STORECACHE is used by a caching system for reading terms from stored expressions. Each thread should have its own system of caches.

4.136.3.25 PERM

```
typedef struct PeRmUtE PERM
```

The struct PERM is used to generate all permutations when the pattern matcher has to try to match (anti)symmetric functions.

4.136.3.26 PERMP

```
typedef struct PeRmUtEp PERMP
```

Like struct PERM but works with pointers.

4.136.3.27 DISTRIBUTE

```
typedef struct DiStRiBuTe DISTRIBUTE
```

The struct DISTRIBUTE is used to help the pattern matcher when matching antisymmetric tensors.

4.136.3.28 PARTI

```
typedef struct PaRtI PARTI
```

The struct PARTI is used to help determining whether a `partition_` function can be replaced.

4.136.3.29 SORTING

```
typedef struct sOrT SORTING
```

The struct SORTING is used to control a sort operation. It includes a small and a large buffer and arrays for keeping track of various stages of the (merge) sorts. Each sort level has its own struct and different levels can have different sizes for its arrays. Also different threads have their own set of SORTING structs.

4.136.3.30 ALLGLOBALS

```
typedef struct AllGlobals ALLGLOBALS
```

Without pthreads (FORM) the ALLGLOBALS struct has all the global variables

4.136.3.31 WCN

```
typedef WORD(* WCN) ()
```

Definition at line 2657 of file [structs.h](#).

4.136.3.32 WCN2

```
typedef WORD(* WCN2) ()
```

Definition at line 2658 of file [structs.h](#).

4.136.3.33 COMPARE

```
typedef WORD(* COMPARE) (PHEAD WORD *, WORD *, WORD)
```

Definition at line 2661 of file [structs.h](#).

4.136.3.34 FIXEDGLOBALS

```
typedef struct FixedGlobals FIXEDGLOBALS
```

The FIXEDGLOBALS struct is an anachronism. It started as the struct with global variables that needed initialization. It contains the elements Operation and OperaFind which define a very early way of automatically jumping to the proper operation. We find the results of it in parts of the file [opera.c](#) Later operations were treated differently in a more transparent way. We never changed the existing code. The most important part is currently the cTable which is used intensively in the compiler.

4.137 structs.h

[Go to the documentation of this file.](#)

```
00001
00017 /* #[] License : */
00018 /*
00019 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00020 *   When using this file you are requested to refer to the publication
00021 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00022 *   This is considered a matter of courtesy as the development was paid
00023 *   for by FOM the Dutch physics granting agency and we would like to
00024 *   be able to track its scientific use to convince FOM of its value
00025 *   for the community.
00026 *
00027 *   This file is part of FORM.
00028 *
00029 *   FORM is free software: you can redistribute it and/or modify it under the
00030 *   terms of the GNU General Public License as published by the Free Software
00031 *   Foundation, either version 3 of the License, or (at your option) any later
00032 *   version.
00033 *
00034 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00035 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00036 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00037 *   details.
00038 *
00039 *   You should have received a copy of the GNU General Public License along
00040 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00041 */
00042 /* #[] License : */
00043
00044 #ifndef __STRUCTS__
00045 #define __STRUCTS__
00046 #ifdef _MSC_VER
00047 #include <wchar.h> /* off_t */
00048 #endif
00049
00050
00051 /*
00052 *   #[] sav&store :
00053 */
00054
00059 typedef struct PoSiTiOn {
00060     off_t pl;
00061 } POSITION;
00062
00063 /* Next are the index structs for stored and saved expressions */
00064
00075 typedef struct {
00076     UBYTE headermark[8];
00077     UBYTE lenWORD;
00078     UBYTE lenLONG;
00079     UBYTE lenPOS;
00080     UBYTE lenPOINTER;
00081     UBYTE endianness[16];
00082     UBYTE sSym;
00083     UBYTE sInd;
00084     UBYTE sVec;
00085     UBYTE sFun;
00086     UBYTE maxpower[16];
00087     UBYTE wildoffset[16];
00088     UBYTE revision;
00089     UBYTE reserved[512-8-4-16-4-16-16-1];
00090 } STOREHEADER;
00091
00092
00093 STATIC_ASSERT(sizeof(STOREHEADER) == 512);
```

```

00094
00100 typedef struct InDeXeNtRy {
00101     POSITION    position;
00102     POSITION    length;
00103     POSITION    variables;
00104     LONG       CompressSize;
00105     WORD       nsymbols;
00106     WORD       nindices;
00107     WORD       nvectors;
00108     WORD       nfunctions;
00109     WORD       size;
00110     SBYTE      name[MAXENAME+1];
00111     PADPOSITION(0,1,0,5,MAXENAME+1);
00112 } INDEXENTRY;
00113
00119 #define INFILEINDEX ((512-2*sizeof(POSITION))/sizeof(INDEXENTRY))
00124 #define EMPTYINDEX (512-2*sizeof(POSITION)-INFILEINDEX*sizeof(INDEXENTRY))
00125
00138 typedef struct FiLeInDeX {
00139     POSITION    next;
00140     POSITION    number;
00141     INDEXENTRY expression[INFILEINDEX];
00142     SBYTE      empty[EMPTYINDEX];
00143 } FILEINDEX;
00144
00145 STATIC_ASSERT(sizeof(FILEINDEX) == 512);
00146
00151 typedef struct FiLeDaTa {
00152     FILEINDEX Index;
00153     POSITION Fill;
00154     POSITION Position;
00155     WORD Handle;
00156     WORD dirtyflag;
00157     PADPOSITION(0,0,0,2,0);
00158 } FILEDATA;
00159
00166 typedef struct VaRrEnUm {
00167     WORD *start;
00168     WORD *lo;
00169     WORD *hi;
00170 } VARRENUM;
00171
00178 typedef struct ReNuMbEr {
00179     POSITION startposition;
00180     /* First stage renumbering */
00181     VARRENUM symb;
00182     VARRENUM indi;
00183     VARRENUM vect;
00184     VARRENUM func;
00185     /* Second stage renumbering */
00186     WORD *symnum;
00187     WORD *indnum;
00188     WORD *vecnum;
00189     WORD *funnum;
00190     PADPOSITION(4,0,0,0,sizeof(VARRENUM)*4);
00191 } *RENUMBER;
00192
00193 /*
00194     #] sav&store :
00195     #[ Variables :
00196 */
00197
00205 typedef struct {
00206     void *lijst;
00207     char *message;
00208     int num;
00209     int maxnum;
00210     int size;
00211     int numglobal;
00212     int numtemp;
00213     int numclear;
00214     PADPOINTER(0,6,0,0);
00215 } LIST;
00216
00222 typedef struct {
00223     char *name;
00224     TFUN func;
00225     int type;
00226     int flags;
00227 } KEYWORD;
00228
00234 typedef struct {
00235     char *name;
00236     int *var;
00237     int type;
00238     int flags;
00239 } KEYWORDV;

```



```

00240
00247 typedef struct NaMeNode {
00248     LONG name;
00249     WORD parent;
00250     WORD left;
00251     WORD right;
00252     WORD balance;
00253     WORD type;
00254     WORD number;
00255     PADLONG(0,6,0);
00256 } NAMENODE;
00257
00265 typedef struct NaMeTree {
00266     NAMENODE *namenode;
00268     UBYTE *namebuffer;
00271     LONG nodesize;
00272     LONG nodefill;
00273     LONG namesize;
00274     LONG namefill;
00275     LONG oldnamefill;
00276     LONG oldnodefill;
00277     LONG globalnamefill;
00279     LONG globalnodefill;
00280     LONG clearnamefill;
00281     LONG clearnodefill;
00282     WORD headnode;
00283     PADPOINTER(10,0,1,0);
00284 } NAMETREE;
00285
00294 typedef struct tree {
00295     int parent;
00296     int left;
00297     int right;
00298     int value;
00299     int blnce;
00300     int usage;
00301 } COMPTREE;
00302
00307 typedef struct MiNmAx {
00308     WORD mini;
00309     WORD maxi;
00310     WORD size;
00311 } MINMAX;
00312
00317 typedef struct BrAcKeTiNdEx { /* For indexing brackets in local expressions */
00318     POSITION start; /* Place where bracket starts - start of expr */
00319     POSITION next; /* Place of next indexed bracket in expr */
00320     LONG bracket; /* Offset of position in bracketbuffer */
00321     LONG termsinbracket;
00322     PADPOSITION(0,2,0,0,0);
00323 } BRACKETINDEX;
00324
00329 typedef struct BrAcKeTiNfO {
00330     BRACKETINDEX *indexbuffer;
00331     WORD *bracketbuffer;
00332     LONG bracketbuffersize;
00333     LONG indexbuffersize;
00334     LONG bracketfill;
00335     LONG indexfill;
00336     WORD SortType;
00337     PADPOINTER(4,0,1,0);
00338 } BRACKETINFO;
00339
00350 typedef struct TaBlEs {
00351     WORD *tablepointers;
00352 #ifdef WITHPTHREADS
00353     WORD **prototype;
00354     WORD **pattern;
00355 #else
00356     WORD *prototype;
00357     WORD *pattern;
00358 #endif
00359     MINMAX *mm;
00360     WORD *flags;
00361     COMPTREE *boomlijst;
00362     UBYTE *argtail;
00364     struct TaBlEs *spare;
00365     WORD *buffers;
00366     LONG totind;
00367     LONG reserved;
00368     LONG defined;
00369     LONG mdefined;
00370     int prototypeSize;
00371     int numind;
00372     int bounds;
00373     int strict;
00374     int sparse;

```

```

00375     int      numtree;
00376     int      rootnum;
00377     int      MaxTreeSize;
00378     WORD     bufnum;
00379     WORD     buffersize;
00380     WORD     buffersfill;
00381     WORD     tablenum;
00382     WORD     mode;
00383     WORD     numdummies;
00384     PADPOINTER(4,8,6,0);
00385 } *TABLES;
00386
00391 typedef struct ExprEssiOn {
00392     POSITION   onfile;
00393     POSITION   prototype;
00394     POSITION   size;
00395     RENUMBER  renum; /* For Renumbering of global stored expressions */
00396     BRACKETINFO *bracketinfo;
00397     BRACKETINFO *newbracketinfo;
00398     WORD      *renumlists;
00400     WORD      *inmem; /* If in memory like e.g. a polynomial */
00401     LONG      counter;
00402     LONG      name;
00403     WORD      hidelevel;
00404     WORD      vflags; /* Various flags */
00405     WORD      printflag;
00406     WORD      status;
00407     WORD      replace;
00408     WORD      node;
00409     WORD      whichbuffer;
00410     WORD      namesize;
00411     WORD      compression;
00412     WORD      numdummies;
00413     WORD      numfactors;
00414     WORD      sizeprototype;
00415     WORD      uflags; /* User flags */
00416 #ifdef PARALLELCODE
00417     WORD      partodo; /* Whether to be done in parallel mode */
00418     PADPOSITION(5,2,0,14,0);
00419 #else
00420     PADPOSITION(5,2,0,13,0);
00421 #endif
00422 } *EXPRESSIONS;
00423
00428 typedef struct SyMbOl { /* Don't change unless altering .sav too */
00429     LONG      name; /* Location in names buffer */
00430     WORD      minpower; /* Minimum power admissible */
00431     WORD      maxpower; /* Maximum power admissible */
00432     WORD      complex; /* Properties wrt complex conjugation */
00433     WORD      number; /* Number when stored in file */
00434     WORD      flags; /* Used to indicate usage when storing */
00435     WORD      node;
00436     WORD      namesize;
00437     WORD      dimension; /* For dimensionality checks */
00438     PADLONG(0,8,0);
00439 } *SYMBOLS;
00440
00445 typedef struct InDeX { /* Don't change unless altering .sav too */
00446     LONG      name; /* Location in names buffer */
00447     WORD      type; /* Regular or dummy */
00448     WORD      dimension; /* Value of d(n,n) or -number of symbol */
00449     WORD      number; /* Number when stored in file */
00450     WORD      flags; /* Used to indicate usage when storing */
00451     WORD      nmin4; /* Used for n-4 if dimension < 0 */
00452     WORD      node;
00453     WORD      namesize;
00454     PADLONG(0,7,0);
00455 } *INDICES;
00456
00461 typedef struct VeCtOr { /* Don't change unless altering .sav too */
00462     LONG      name; /* Location in names buffer */
00463     WORD      complex; /* Properties under complex conjugation */
00464     WORD      number; /* Number when stored in file */
00465     WORD      flags; /* Used to indicate usage when storing */
00466     WORD      node;
00467     WORD      namesize;
00468     WORD      dimension; /* For dimensionality checks */
00469     PADLONG(0,6,0);
00470 } *VECTORS;
00471
00477 typedef struct FuNcTiOn { /* Don't change unless altering .sav too */
00478     TABLES  tabl;
00479     LONG      symminfo;
00480     LONG      name;
00481     WORD      commute;
00482     WORD      complex;
00483     WORD      number;

```

```

00484     WORD    flags;
00485     WORD    spec;
00486     WORD    symmetric;
00487     WORD    node;
00488     WORD    namesize;
00489     WORD    dimension;      /* For dimensionality checks */
00490     WORD    maxnumargs;
00491     WORD    minnumargs;
00492     PADPOINTER(2,0,11,0);
00493 } *FUNCTIONS;
00494
00499 typedef struct SeTs {
00500     LONG    name;           /* Location in names buffer */
00501     WORD    type;           /* Symbol, vector, index or function */
00502     WORD    first;          /* First element in setstore */
00503     WORD    last;           /* Last element in setstore (excluding) */
00504     WORD    node;
00505     WORD    namesize;
00506     WORD    dimension;      /* For dimensionality checks */
00507     WORD    flags;          /* Like: ordered */
00508     PADLONG(0,7,0);
00509 } *SETS;
00510
00515 typedef struct DuBiOuS {    /* Undeclared objects. Just for compiler. */
00516     LONG    name;           /* Location in names buffer */
00517     WORD    node;
00518     WORD    dummy;
00519     PADLONG(0,2,0);
00520 } *DUBIOUSV;
00521
00522 typedef struct FaCdOllLaR {
00523     WORD    *where;         /* A pointer(!) to the content */
00524     LONG    size;
00525     WORD    type;           /* Type can be DOLNUMBER or DOLTERMS */
00526     WORD    value;          /* in case it is a (short) number */
00527     PADPOINTER(1,0,2,0);
00528 } FACDOLLAR;
00529
00530 typedef struct DoLlArS {
00531     WORD    *where;         /* A pointer(!) to the object */
00532     FACDOLLAR *factors;     /* an array of factors. nfactors elements */
00533     #ifdef WITHPTHREADS
00534     pthread_mutex_t pthreadslockread;
00535     pthread_mutex_t pthreadslockwrite;
00536     #endif
00537     LONG    size;           /* The number of words */
00538     LONG    name;
00539     WORD    type;
00540     WORD    node;
00541     WORD    index;
00542     WORD    zero;
00543     WORD    numdummies;
00544     WORD    nfactors;
00545     #ifdef WITHPTHREADS
00546     PADPOINTER(2,0,6,sizeof(pthread_mutex_t)*2);
00547     #else
00548     PADPOINTER(2,0,6,0);
00549     #endif
00550 } *DOLLARS;
00551
00556 typedef struct MoDoPtDoLlArS {
00557     #ifdef WITHPTHREADS
00558     DOLLARS dstruct;        /* If local dollar: list of DOLLARS for each thread */
00559     #endif
00560     WORD    number;
00561     WORD    type;
00562     #ifdef WITHPTHREADS
00563     PADPOINTER(0,0,2,0);
00564     #endif
00565 } MODOPTDOLLAR;
00566
00571 typedef struct fixedset {
00572     char *name;
00573     char *description;
00574     int type;
00575     int dimension;
00576 } FIXEDSET;
00577
00582 typedef struct TaBlEbAsEsUbInDeX {
00583     POSITION where;
00584     LONG size;
00585     PADPOSITION(0,1,0,0,0);
00586 } TABLEBASESUBINDEX;
00587
00592 typedef struct TaBlEbAsE {
00593     POSITION fillpoint;
00594     POSITION current;

```

```

00595     UBYTE *name;
00596     int *tablenumbers;          /* Number of each table */
00597     TABLEBASESUBINDEX *subindex; /* For each table */
00598     int numtables;
00599     PADPOSITION(3,0,1,0,0);
00600 } TABLEBASE;
00601
00608 typedef struct {
00609     WORD *location;
00610     int numargs;
00611     int numfunnies;
00612     int numwildcards;
00613     int symmet;
00614     int tensor;
00615     int commute;
00616     PADPOINTER(0,6,0,0);
00617 } FUN_INFO;
00618
00619 typedef struct PaRtIcLe {
00620     WORD number; /* Number of the function */
00621     WORD spin; /* +/- dimension of SU(2) representation */
00622     WORD mass; /* Number of symbol or 0 */
00623     WORD type; /* -1: anti particle, 1: particle, 0: own antiparticle */
00624 } PARTICLE;
00625
00626 typedef struct VerTex {
00627     PARTICLE particles[MAXPARTICLES];
00628     WORD couplings[2*MAXCOUPLINGS];
00629     WORD nparticles;
00630     WORD ncouplings;
00631     WORD type;
00632     WORD error;
00633     WORD externonly;
00634     WORD spare;
00635 } VERTEX;
00636
00637 typedef struct MoDeL {
00638     VERTEX **vertices;
00639     WORD *couplings;
00640     UBYTE *name;
00641     void *grccmodel;
00642     WORD legcouple[MAXLEGS+2];
00643     WORD nparticles;
00644     WORD nvertices;
00645     WORD invertices;
00646     WORD sizevertices;
00647     WORD sizecouplings;
00648     WORD ncouplings;
00649     WORD error;
00650     WORD dummy;
00651 } MODEL;
00652
00653 /*
00654  * The struct NAMESPACE is used for the namespace info. It is a
00655  * doubly linked list of nested namespaces.
00656  */
00657
00658 typedef struct NaMeSpAcE {
00659     struct NaMeSpAcE *previous;
00660     struct NaMeSpAcE *next;
00661     NAMETREE *usenames;
00662     UBYTE *name;
00663 } NAMESPACE;
00664
00665
00666 /*
00667  #] Variables :
00668  #[ Files :
00669  */
00670
00682 typedef struct FiLe {
00683     POSITION PPosition; /* File position */
00684     POSITION filesize; /* Because SEEK_END is unsafe on IBM */
00685     WORD *PObuffer; /* Address of the intermediate buffer */
00686     WORD *POstop; /* End of the buffer */
00687     WORD *POfill; /* Fill position of the buffer */
00688     WORD *POfull; /* Full buffer when only cached */
00689 #ifdef WITHPTHREADS
00690     WORD *wPObuffer; /* Address of the intermediate worker buffer */
00691     WORD *wPOstop; /* End of the worker buffer */
00692     WORD *wPOfill; /* Fill position of the worker buffer */
00693     WORD *wPOfull; /* Full buffer when only worker cached */
00694 #endif
00695     char *name; /* name of the file */
00696 #ifdef WITHZLIB
00697     z_stream zsp; /* The pointer to the stream struct for gzip */
00698     Bytef *ziobuffer; /* The output buffer for compression */

```

```

00699 #endif
00700     ULONG numblocks;           /* Number of blocks in file */
00701     ULONG inbuffer;           /* Block in the buffer */
00702     LONG PSize;               /* size of the buffer */
00703 #ifdef WITHZLIB
00704     LONG ziosize;             /* size of the zoutbuffer */
00705 #endif
00706 #ifdef WITHPTHREADS
00707     LONG wPSize;              /* size of the worker buffer */
00708     pthread_mutex_t pthreadslock;
00709 #endif
00710     int handle;
00711     int active;                /* File is open or closed. Not used. */
00712 #ifdef WITHPTHREADS
00713 #ifdef WITHZLIB
00714     PADPOSITION(11,5,2,0,sizeof(pthread_mutex_t));
00715 #else
00716     PADPOSITION(9,4,2,0,sizeof(pthread_mutex_t));
00717 #endif
00718 #else
00719 #ifdef WITHZLIB
00720     PADPOSITION(7,4,2,0,0);
00721 #else
00722     PADPOSITION(5,3,2,0,0);
00723 #endif
00724 #endif
00725 } FILEHANDLE;
00726
00736 typedef struct Stream {
00737     off_t fileposition;
00738     off_t linenumber;
00739     off_t prevline;
00740     UBYTE *buffer;
00741     UBYTE *pointer;
00742     UBYTE *top;
00743     UBYTE *FoldName;
00744     UBYTE *name;
00745     UBYTE *pname;
00746     LONG buffersize;
00747     LONG bufferposition;
00748     LONG inbuffer;
00749     int previous;
00750     int handle;
00751     int type;
00752     int prevars;
00753     int previousNoShowInput;
00754     int eqnum;
00755     int afterwards;
00756     int olddelay;
00757     int oldnoshowinput;
00758     UBYTE isnextchar;
00759     UBYTE nextchar[2];
00760     UBYTE reserved;
00761     PADPOSITION(6,3,9,0,4);
00762 } STREAM;
00763
00765 typedef struct SpecTator {
00766     POSITION position;          /* The place where we will be writing */
00767     POSITION readpos;           /* The place from which we read */
00768     FILEHANDLE *fh;
00769     char *name;                /* We identify the spectator by the name of the expression */
00770     WORD exprnumber;           /* During running we use the number. */
00771     WORD flags;                /* local, global? */
00772     PADPOSITION(2,0,0,2,0);
00773 } SPECTATOR;
00774
00775 /*
00776 #] Files :
00777 #[ Traces :
00778 */
00779
00789 typedef struct TrAcEs {           /* For computing 4 dimensional traces */
00790     WORD *accu;                /* NUMBER * 2 */
00791     WORD *accup;
00792     WORD *termp;
00793     WORD *perm;               /* number */
00794     WORD *inlist;             /* number */
00795     WORD *nt3;                /* number/2 */
00796     WORD *nt4;                /* number/2 */
00797     WORD *j3;                 /* number*2 */
00798     WORD *j4;                 /* number*2 */
00799     WORD *e3;                 /* number*2 */
00800     WORD *e4;                 /* number */
00801     WORD *eers;               /* number/2 */
00802     WORD *mepf;               /* number/2 */
00803     WORD *mdel;               /* number/2 */
00804     WORD *pepf;               /* number*2 */

```

```

00805     WORD      *pdel;          /* number*3/2 */
00806     WORD      sgn;
00807     WORD      stap;
00808     WORD      step1,kstep,mdum;
00809     WORD      gamm,ad,a3,a4,lc3,lc4;
00810     WORD      sign1,sign2,gamma5,num,level,factor,allsign;
00811     WORD      finalstep;
00812     PADPOINTER(0,0,19,0);
00813 } TRACES;
00814
00822 typedef struct TrAcEn {          /* For computing n dimensional traces */
00823     WORD      *accu;          /* NUMBER */
00824     WORD      *accup;
00825     WORD      *termp;
00826     WORD      *perm;          /* number */
00827     WORD      *inlist;        /* number */
00828     WORD      sgn,num,level,factor,allsign;
00829     PADPOINTER(0,0,5,0);
00830 } *TRACEN;
00831
00832 /*
00833     #] Traces :
00834     #[ Preprocessor :
00835 */
00836
00841 typedef struct pReVaR {
00842     UBYTE *name;
00843     UBYTE *value;
00844     UBYTE *argnames;
00845     int nargs;
00846     int wildarg;
00847     PADPOINTER(0,2,0,0);
00848 } PREVAR;
00849
00854 typedef struct {
00855     WORD *buffer;
00856     int oldcompiletype;
00857     int oldparallelflag;
00858     int oldnumpotmoddollars;
00859     WORD size;
00860     WORD numdollars;
00861     WORD oldcbuf;
00862     WORD oldrbuf;
00863     WORD inscbuf;
00864     WORD oldcnumlhs;
00865     PADPOINTER(0,3,6,0);
00866 } INSIDEINFO;
00867
00873 typedef struct {
00874     UBYTE *buffer;
00875     LONG size;
00876     PADPOINTER(1,0,0,0);
00877 } PRELOAD;
00878
00883 typedef struct {
00884     PRELOAD p;
00885     UBYTE *name;
00886     int loadmode;
00887     PADPOINTER(0,1,0,0);
00888 } PROCEDURE;
00889
00897 typedef struct DoLoOp {
00898     PRELOAD p;
00899     UBYTE *name;
00900     UBYTE *vars;          /* for {} or name of expression */
00901     UBYTE *contents;
00902     UBYTE *dollarname;
00903     LONG startlinenumber;
00904     LONG firstnum;
00905     LONG lastnum;
00906     LONG incnum;
00907     int type;
00908     int NoShowInput;
00909     int errorsinloop;
00910     int firstloopcall;
00911     WORD firstdollar;      /* When >= 0 we have to get the value from a dollar */
00912     WORD lastdollar;       /* When >= 0 we have to get the value from a dollar */
00913     WORD incdollar;        /* When >= 0 we have to get the value from a dollar */
00914     WORD NumPreTypes;
00915     WORD PreIfLevel;
00916     WORD PreSwitchLevel;
00917     PADPOINTER(4,4,6,0);
00918 } DOLOOP;
00919
00927 struct bit_field {          /* Assume 8 bits per byte */
00928     UBYTE bit_0          : 1;
00929     UBYTE bit_1          : 1;

```

```

00930     UBYTE bit_2      : 1;
00931     UBYTE bit_3      : 1;
00932     UBYTE bit_4      : 1;
00933     UBYTE bit_5      : 1;
00934     UBYTE bit_6      : 1;
00935     UBYTE bit_7      : 1;
00936 /*
00937     UINT bit_0        : 1;
00938     UINT bit_1        : 1;
00939     UINT bit_2        : 1;
00940     UINT bit_3        : 1;
00941     UINT bit_4        : 1;
00942     UINT bit_5        : 1;
00943     UINT bit_6        : 1;
00944     UINT bit_7        : 1;
00945 */
00946 };
00947
00952 typedef struct bit_field set_of_char[32];
00953
00958 typedef struct bit_field *one_byte;
00959
00964 typedef struct {
00965     WORD newlogonly;
00966     WORD newhandle;
00967     WORD oldhandle;
00968     WORD oldlogonly;
00969     WORD oldprinttype;
00970     WORD oldsilent;
00971 } HANDLERS;
00972
00973 /*
00974     #] Preprocessor :
00975     #[ Varia :
00976 */
00977
00978 typedef WORD (*FINISHUFFLE)(WORD *);
00979 typedef WORD (*DO_UFFLE)(WORD *,WORD,WORD,WORD);
00980
00981 #ifdef WITHPTHREADS
00982 typedef WORD (*COMPAREDUMMY)(void *,WORD *,WORD *,WORD);
00983 #else
00984 typedef WORD (*COMPAREDUMMY)(WORD *,WORD *,WORD);
00985 #endif
00986
00992 typedef struct CbUf {
00993     WORD *Buffer;
00994     WORD *Top;
00995     WORD *Pointer;
00996     WORD **lhs;
00997     WORD **rhs;
00998     LONG *CanCommu;
00999     LONG *NumTerms;
01000     WORD *numdum;
01001     WORD *dimension;
01002     COMPTREE *boomlijst;
01003     LONG BufferSize;
01004     int numlhs;
01005     int numrhs;
01006     int maxlhs;
01007     int maxrhs;
01008     int mnumlhs;
01009     int mnumrhs;
01010     int numtree;
01011     int rootnum;
01012     int MaxTreeSize;
01013     PADPOINTER(1,9,0,0);
01014 } CBUF;
01015
01023 typedef struct ChAnNeL {
01024     char *name;
01025     int handle;
01026     PADPOINTER(0,1,0,0);
01027 } CHANNEL;
01028
01038 typedef struct {
01039     UBYTE *parameter;
01040     int type;
01041     int flags;
01042     LONG value;
01043 } SETUPPARAMETERS;
01044
01053 typedef struct NeStInG {
01054     WORD *termsize;
01055     WORD *funsize;
01056     WORD *argsize;
01057 } *NESTING;

```

```

01058
01065 typedef struct StOrEcAcHe {
01066     POSITION position;
01067     POSITION toppos;
01068     struct StOrEcAcHe *next;
01069     WORD buffer[2];
01070     PADPOSITION(1,0,0,2,0);
01071 } *STORECACHE;
01072
01078 typedef struct PerMUtE {
01079     WORD *objects;
01080     WORD sign;
01081     WORD n;
01082     WORD cycle[MAXMATCH];
01083     PADPOINTER(0,0,MAXMATCH+2,0);
01084 } PERM;
01085
01090 typedef struct PerMUtEp {
01091     WORD **objects;
01092     WORD sign;
01093     WORD n;
01094     WORD cycle[MAXMATCH];
01095     PADPOINTER(0,0,MAXMATCH+2,0);
01096 } PERMP;
01097
01103 typedef struct DiStriBuTe {
01104     WORD *obj1;
01105     WORD *obj2;
01106     WORD *out;
01107     WORD sign;
01108     WORD n1;
01109     WORD n2;
01110     WORD n;
01111     WORD cycle[MAXMATCH];
01112     PADPOINTER(0,0,(MAXMATCH+4),0);
01113 } DISTRIBUTE;
01114
01120 typedef struct PaRtI {
01121     WORD *psize; /* the sizes of the partitions */
01122     WORD *args; /* the offsets of the arguments to be partitioned */
01123     WORD *nargs; /* argument numbers (different number = different argument) */
01124     WORD *nfun; /* the functions into which the partitions go */
01125     WORD numargs; /* the number of arguments to be partitioned */
01126     WORD numpart; /* the number of partitions */
01127     WORD where; /* offset of the function in the term */
01128     PADPOINTER(0,0,3,0);
01129 } PARTI;
01130
01140 typedef struct sOrT {
01141     FILEHANDLE file; /* The own sort file */
01142     POSITION SizeInFile[3]; /* Sizes in the various files */
01143     WORD *lBuffer; /* The large buffer */
01144     WORD *lTop; /* End of the large buffer */
01145     WORD *lFill; /* The filling point of the large buffer */
01146     WORD *used; /* auxiliary during actual sort */
01147     WORD *sBuffer; /* The small buffer */
01148     WORD *sTop; /* End of the small buffer */
01149     WORD *sTop2; /* End of the extension of the small buffer */
01150     WORD *sHalf; /* Halfway point in the extension */
01151     WORD *sFill; /* Filling point in the small buffer */
01152     WORD **sPointer; /* Pointers to terms in the small buffer */
01153     WORD **PoinFill; /* Filling point for pointers to the sm.buf */
01154     WORD *cBuffer; /* Compress buffer (if it exists) */
01155     WORD **Patches; /* Positions of patches in large buffer */
01156     WORD **pStop; /* Ends of patches in the large buffer */
01157     WORD **poina; /* auxiliary during actual sort */
01158     WORD **poin2a; /* auxiliary during actual sort */
01159     WORD *ktoi; /* auxiliary during actual sort */
01160     WORD *tree; /* auxiliary during actual sort */
01161 #ifdef WITHZLIB
01162     WORD *fpccompressed; /* is output filepatch compressed? */
01163     WORD *fpincompressed; /* is input filepatch compressed? */
01164     z_stream *zspparray; /* the array of zstreams for decompression */
01165 #endif
01166     POSITION *fPatches; /* Positions of output file patches */
01167     POSITION *inPatches; /* Positions of input file patches */
01168     POSITION *fPatchStop; /* Positions of output file patches */
01169     POSITION *iPatches; /* Input file patches, Points to fPatches or inPatches */
01170     FILEHANDLE *f; /* The actual output file */
01171     FILEHANDLE **ff; /* Handles for a staged sort */
01172     LONG sTerms; /* Terms in small buffer */
01173     LONG LargeSize; /* Size of large buffer (in words) */
01174     LONG SmallSize; /* Size of small buffer (in words) */
01175     LONG SmallEsize; /* Size of small + extension (in words) */
01176     LONG TermsInSmall; /* Maximum number of terms in small buffer */
01177     LONG Terms2InSmall; /* with extension for polyfuns etc. */
01178     LONG GenTerms; /* Number of generated terms */

```



```

01179     LONG TermsLeft;           /* Number of terms still in existence */
01180     LONG GenSpace;           /* Amount of space of generated terms */
01181     LONG SpaceLeft;          /* Space needed for still existing terms */
01182     LONG putinsize;          /* Size of buffer in putin */
01183     LONG ninterms;           /* Which input term ? */
01184     int MaxPatches;           /* Maximum number of patches in large buffer */
01185     int MaxFpatches;          /* Maximum number of patches in one filesort */
01186     int type;                 /* Main, function or sub(routine) */
01187     int lPatch;               /* Number of patches in the large buffer */
01188     int fPatchN1;             /* Number of patches in input file */
01189     int PolyWise;             /* Is there a polyfun and if so, where? */
01190     int PolyFlag;             /* */
01191     int cBufferSize;          /* Size of the compress buffer */
01192     int maxtermsize;          /* Keeps track for buffer allocations */
01193     int newmaxtermsize;       /* Auxiliary for maxtermsize */
01194     int outputmode;           /* Tells where the output is going */
01195     int stagelevel;           /* In case we have a 'staged' sort */
01196     WORD fPatchN;             /* Number of patches on file (output) */
01197     WORD inNum;               /* Number of patches on file (input) */
01198     WORD stage4;              /* Are we using stage4? */
01199 #ifndef WITH2LIB
01200     PADPOSITION(27,12,12,3,0);
01201 #else
01202     PADPOSITION(24,12,12,3,0);
01203 #endif
01204 } SORTING;
01205
01206 #ifdef WITHPTHREADS
01207 typedef struct SoRtBlOcK {
01208     pthread_mutex_t *MasterBlockLock;
01209     WORD **MasterStart;
01210     WORD **MasterFill;
01211     WORD **MasterStop;
01212     int MasterNumBlocks;
01213     int MasterBlock;
01214     int FillBlock;
01215     PADPOINTER(0,3,0,0);
01216 } SORTBLOCK;
01217 #endif
01218
01219 #ifdef DEBUGGER
01220 typedef struct DeBuGgInG {
01221     int eflag;
01222     int printfflag;
01223     int logfileflag;
01224     int stdoutflag;
01225 } DEBUGSTR;
01226 #endif
01227
01228 #ifdef WITHPTHREADS
01229 typedef struct ThReAdBuCkEt {
01230     POSITION *deferbuffer; /* For Keep Brackets: remember position */
01231     WORD *threadbuffer; /* Here are the (primary) terms */
01232     WORD *compressbuffer; /* For keep brackets we need the compressbuffer */
01233     LONG threadbuffersize; /* Number of words in threadbuffer */
01234     LONG ddterms; /* Number of primary+secondary terms represented */
01235     LONG firstterm; /* The number of the first term in the bucket */
01236     LONG firstbracket; /* When doing complete brackets */
01237     LONG lastbracket; /* When doing complete brackets */
01238     pthread_mutex_t lock; /* For the load balancing phase */
01239     int free; /* Status of the bucket */
01240     int totnum; /* Total number of primary terms */
01241     int usenum; /* Which is the term being used at the moment */
01242     int busy; /* */
01243     int type; /* Doing brackets? */
01244     PADPOINTER(5,5,0,sizeof(pthread_mutex_t));
01245 } THREADBUCKET;
01246 #endif
01247
01248 typedef struct {
01249     WORD *coefs; /* The array of coefficients */
01250     WORD numsym; /* The number of the symbol in the polynomial */
01251     WORD arraysize; /* The size of the allocation of coefs */
01252     WORD polysize; /* The maximum power in the polynomial */
01253     WORD modnum; /* The prime number of the modulus */
01254 } POLYMOD;
01255
01256 typedef struct {
01257     WORD *outterm; /* Used in DoShuffle/Merge/FinishShuffle system */
01258     WORD *outfun;
01259     WORD *incoef;
01260     WORD *stop1;
01261     WORD *stop2;
01262     WORD *ststop1;

```

```

01283     WORD    *ststop2;
01284     FINISHUFFLE finishuf;
01285     DO_UFFLE   do_uffle;
01286     LONG     combilast;
01287     WORD     nincoef;
01288     WORD     level;
01289     WORD     thefunction;
01290     WORD     option;
01291     PADPOINTER(1,0,4,0);
01292 } SHvariables;
01293
01294 typedef struct {                                /* Used for computing calculational cost in optim.c */
01295     LONG     add;
01296     LONG     mul;
01297     LONG     div;
01298     LONG     pow;
01299 } COST;
01300
01301 typedef struct {
01302     UWORD    *a;      /* The number array */
01303     UWORD    *m;      /* The modulus array */
01304     WORD     na;      /* Size of the number */
01305     WORD     nm;      /* size of the number in the modulus array */
01306 } MODNUM;
01307
01308 /*
01309     Struct for optimizing outputs. If changed, do not forget to change
01310     the padding information in the AO struct.
01311 */
01312 typedef struct {
01313     union { /* we do this to allow padding */
01314         float fval;
01315         int ival[2]; /* This should be enough */
01316     } mctsconstant;
01317     int horner;
01318     int hornerdirection;
01319     int method;
01320     int mctstimelimit;
01321     int mctsnumexpand;
01322     int mctsnumkeep;
01323     int mctsnumrepeat;
01324     int greedytimelimit;
01325     int greedyminnum;
01326     int greedymaxperc;
01327     int printstats;
01328     int debugflags;
01329     int schemeflags;
01330     int mctsdecaymode;
01331     int saIter; /* Simulated annealing updates */
01332     union {
01333         float fval;
01334         int ival[2];
01335     } saMaxT; /* Maximum temperature of SA */
01336     union {
01337         float fval;
01338         int ival[2];
01339     } saMinT; /* Minimum temperature of SA */
01340     int spare;
01341 } OPTIMIZE;
01342
01343 typedef struct {
01344     WORD *code;
01345     UBYTE *nameofexpr; /* It is easier to remember an expression by name */
01346     LONG codesize; /* We need this for the checkpoints */
01347     WORD exprnr; /* Problem here is: we renumber them in execute.c */
01348     WORD minvar;
01349     WORD maxvar;
01350
01351     PADPOSITION(2,1,0,3,0);
01352 } OPTIMIZERESULT;
01353
01354 typedef struct {
01355     WORD *lhs; /* Object to be replaced */
01356     WORD *rhs; /* Depending on the type it will be UBYTE* or WORD* */
01357     int type;
01358     int size; /* Size of the lhs */
01359 } DICTIONARY_ELEMENT;
01360
01361 typedef struct {
01362     DICTIONARY_ELEMENT **elements;
01363     UBYTE *name;
01364     int sizeelements;
01365     int numelements;
01366     int numbers; /* deal with numbers */
01367     int variables; /* deal with single variables */
01368     int characters; /* deal with special characters */
01369     int funwith; /* deal with functions with arguments */

```

```

01370     int gnumelements; /* if .global shrinks the dictionary */
01371     int ranges;
01372 } DICTIONARY;
01373
01374 typedef struct {
01375     WORD ncase;
01376     WORD value;
01377     WORD compbuffer;
01378 } SWITCHTABLE;
01379
01380 typedef struct {
01381     SWITCHTABLE *table;
01382     SWITCHTABLE defaultcase;
01383     SWITCHTABLE endswitch;
01384     WORD typetable;
01385     WORD maxcase;
01386     WORD mincase;
01387     WORD numcases;
01388     WORD tablesize;
01389     WORD caseoffset;
01390     WORD iflevel;
01391     WORD whilelevel;
01392     WORD nestingsum;
01393     WORD padding;
01394 } SWITCH;
01395
01396 typedef struct {
01397     WORD *term;
01398     void *currentModel;
01399     void *currentMODEL;
01400     WORD *legcouple[MAXLEGS+1];
01401     LONG numtopo;
01402     LONG numdia;
01403     WORD diaoffset;
01404     WORD level;
01405     WORD externalset;
01406     WORD internalset;
01407     WORD numextern;
01408     WORD flags;
01409 } TERMINFO;
01410
01411 /*
01412 #] Varia :
01413 #[ A :
01414     #[ M : The M struct is for global settings at startup or .clear
01415 */
01416
01424 struct M_const {
01425     POSITION zeropos; /* (M) is zero */
01426     SORTING *S0;
01427     UWORD *gcmmod;
01428     UWORD *gpowmod;
01429     UBYTE *TempDir; /* (M) Path with where the temporary files go */
01430     UBYTE *TempSortDir; /* (M) Path with where the sort files go */
01431     UBYTE *IncDir; /* (M) Directory path for include files */
01432     UBYTE *InputFileName; /* (M) */
01433     UBYTE *LogFileName; /* (M) */
01434     UBYTE *OutBuffer; /* (M) Output buffer in pre.c */
01435     UBYTE *Path; /* (M) */
01436     UBYTE *SetupDir; /* (M) Directory with setup file */
01437     UBYTE *SetupFile; /* (M) Name of setup file */
01438     UBYTE *gFortran90Kind;
01439     UBYTE *gextrasy;
01440     UBYTE *ggextrasy;
01441     UBYTE *oldnumextrasy;
01442     SPECTATOR *SpectatorFiles;
01443 #ifdef WITHPTHREADS
01444     pthread_rwlock_t handlelock; /* (M) */
01445     pthread_mutex_t storefilelock; /* (M) */
01446     pthread_mutex_t sbufilelock; /* (M) Lock for writing in the AM.sbuffer */
01447     LONG ThreadScratSize; /* (M) Size of Fscr[0/2] buffers of the workers */
01448     LONG ThreadScratOutSize; /* (M) Size of Fscr[1] buffers of the workers */
01449 #endif
01450     LONG MaxTer; /* (M) Maximum term size. Fixed at setup. In Bytes!!! */
01451     LONG CompressSize; /* (M) Size of Compress buffer */
01452     LONG ScratSize; /* (M) Size of Fscr[] buffers */
01453     LONG HideSize; /* (M) Size of Fscr[2] buffer */
01454     LONG SizeStoreCache; /* (M) Size of the caches for reading global expr. */
01455     LONG MaxStreamSize; /* (M) Maximum buffer size in reading streams */
01456     LONG SIOsize; /* (M) Sort InputOutput buffer size */
01457     LONG SLargeSize; /* (M) */
01458     LONG SSmallEsize; /* (M) */
01459     LONG SSmallSize; /* (M) */
01460     LONG STermsInSmall; /* (M) */
01461     LONG MaxBracketBufferSize; /* (M) Max Size for B+ or AB+ per expression */
01462     LONG hProcessBucketSize; /* (M) */
01463     LONG gProcessBucketSize; /* (M) */

```

```

01464     LONG     shmWinSize;          /* (M) size for shared memory window used in communications */
01465     LONG     OldChildTime;         /* (M) Zero time. Needed in timer. */
01466     LONG     OldSecTime;           /* (M) Zero time for measuring wall clock time */
01467     LONG     OldMilliTime;         /* (M) Same, but milli seconds */
01468     LONG     WorkSize;             /* (M) Size of Workspace */
01469     LONG     gThreadBucketSize;    /* (C) */
01470     LONG     ggThreadBucketSize;   /* (C) */
01471     LONG     SumTime;              /* Used in .clear */
01472     LONG     SpectatorSize;        /* Size of the buffer in bytes */
01473     LONG     TimeLimit;            /* Limit in sec to the total real time */
01474 #ifdef WITHFLOAT
01475     LONG     gDefaultPrecision;    /* (M) Default precision in bits for float_ */
01476     LONG     gMaxWeight;           /* (M) Maximum weight for MZV or Euler */
01477     LONG     ggDefaultPrecision;   /* (M) Default precision in bits for float_ */
01478     LONG     ggMaxWeight;          /* (M) Maximum weight for MZV or Euler */
01479 #endif
01480     int      FileOnlyFlag;         /* (M) Writing only to file */
01481     int      Interact;             /* (M) Interactive mode flag */
01482     int      MaxParLevel;          /* (M) Maximum nesting of parentheses */
01483     int      OutBufSize;           /* (M) size of OutBuffer */
01484     int      SMaxFpatches;         /* (M) */
01485     int      SMaxPatches;          /* (M) */
01486     int      StdOut;               /* (M) Regular output channel */
01487     int      ginsidefirst;         /* (M) Not used yet */
01488     int      gDefDim;              /* (M) */
01489     int      gDefDim4;             /* (M) */
01490     int      NumFixedSets;         /* (M) Number of the predefined sets */
01491     int      NumFixedFunctions;    /* (M) Number of built in functions */
01492     int      rbufnum;              /* (M) startup compiler buffer */
01493     int      dbufnum;              /* (M) dollar variables */
01494     int      sbufnum;              /* (M) subterm variables */
01495     int      zbufnum;              /* (M) special values */
01496     int      SkipClears;           /* (M) Number of .clear to skip at start */
01497     int      gTokensWriteFlag;     /* (M) */
01498     int      gfunpowers;           /* (M) */
01499     int      gStatsFlag;           /* (M) */
01500     int      gNamesFlag;          /* (M) */
01501     int      gCodesFlag;           /* (M) */
01502     int      gSortType;            /* (M) */
01503     int      gproperorderflag;     /* (M) */
01504     int      hparallelflag;        /* (M) */
01505     int      gparallelflag;        /* (M) */
01506     int      totalnumberofthreads; /* (M) */
01507     int      gSizeCommuteInSet;
01508     int      gThreadStats;
01509     int      ggThreadStats;
01510     int      gFinalStats;
01511     int      ggFinalStats;
01512     int      gThreadsFlag;
01513     int      ggThreadsFlag;
01514     int      gThreadBalancing;
01515     int      ggThreadBalancing;
01516     int      gThreadSortFileSynch;
01517     int      ggThreadSortFileSynch;
01518     int      gProcessStats;
01519     int      ggProcessStats;
01520     int      gOldParallelStats;
01521     int      ggOldParallelStats;
01522     int      maxFlevels;           /* () maximum function levels */
01523     int      resetTimeOnClear;     /* (M) */
01524     int      gcNumDollars;         /* () number of dollars for .clear */
01525     int      MultiRun;
01526     int      gNoSpacesInNumbers;   /* For very long numbers */
01527     int      ggNoSpacesInNumbers;  /* For very long numbers */
01528     int      gIsFortran90;
01529     int      PrintTotalSize;
01530     int      fbufferSize;          /* Size for the AT.fbufnum factorization caches */
01531     int      gOldFactArgFlag;
01532     int      ggOldFactArgFlag;
01533     int      gnumextrasyms;
01534     int      gnumextrasyms;
01535     int      NumSpectatorFiles;     /* Elements used in AM.spectatorfiles; */
01536     int      SizeForSpectatorFiles; /* Size in AM.spectatorfiles; */
01537     int      gOldGCDflag;
01538     int      ggOldGCDflag;
01539     int      gWTimeStatsFlag;
01540     int      ggWTimeStatsFlag;
01541     int      jumpratio;
01542     WORD     MaxTal;               /* (M) Maximum number of words in a number */
01543     WORD     IndDum;               /* (M) Basis value for dummy indices */
01544     WORD     DumInd;               /* (M) */
01545     WORD     WillInd;              /* (M) Offset for wildcard indices */
01546     WORD     gncmod;               /* (M) Global setting of modulus. size of gcmmod */
01547     WORD     gnpowmod;             /* (M) Global printing as powers. size gpowmod */
01548     WORD     gmodmode;             /* (M) Global mode for modulus */
01549     WORD     gUnitTrace;           /* (M) Global value of Tr[1] */
01550     WORD     gOutputMode;          /* (M) */

```

```

01551 WORD gOutputSpaces; /* (M) */
01552 WORD gOutNumberType; /* (M) */
01553 WORD gCnumpows; /* (M) */
01554 WORD gUniTrace[4]; /* (M) */
01555 WORD MaxWildcards; /* (M) Maximum number of wildcards */
01556 WORD mTraceDum; /* (M) Position/Offset for generated dummies */
01557 WORD OffsetIndex; /* (M) */
01558 WORD OffsetVector; /* (M) */
01559 WORD RepMax; /* (M) Max repeat levels */
01560 WORD LogType; /* (M) Type of writing to log file */
01561 WORD ggStatsFlag; /* (M) */
01562 WORD gLineLength; /* (M) */
01563 WORD qError; /* (M) Only error checking {-c option} */
01564 WORD FortranCont; /* (M) Fortran Continuation character */
01565 WORD HoldFlag; /* (M) Exit on termination? */
01566 WORD Ordering[15]; /* (M) Auxiliary for ordering wildcards */
01567 WORD silent; /* (M) Silent flag. Only results in output. */
01568 WORD tracebackflag; /* (M) For tracing errors */
01569 WORD expnum; /* (M) internal number of ^ function */
01570 WORD denomnum; /* (M) internal number of / function */
01571 WORD facnum; /* (M) internal number of fac_ function */
01572 WORD invfacnum; /* (M) internal number of invfac_ function */
01573 WORD sumnum; /* (M) internal number of sum_ function */
01574 WORD sumpnum; /* (M) internal number of sump_ function */
01575 WORD OldOrderFlag; /* (M) Flag for allowing old statement order */
01576 WORD termfunnum; /* (M) internal number of term_ function */
01577 WORD matchfunnum; /* (M) internal number of match_ function */
01578 WORD countfunnum; /* (M) internal number of count_ function */
01579 WORD gPolyFun; /* (M) global value of PolyFun */
01580 WORD gPolyFunInv; /* (M) global value of Inverse of PolyFun */
01581 WORD gPolyFunType; /* (M) global value of PolyFun */
01582 WORD gPolyFunExp;
01583 WORD gPolyFunVar;
01584 WORD gPolyFunPow;
01585 WORD dollarzero; /* (M) for dollars with zero value */
01586 WORD atstartup; /* To protect against DATE_ ending in \n */
01587 WORD exitflag; /* (R) For the exit statement */
01588 WORD NumStoreCaches; /* ( ) Number of storage caches per processor */
01589 WORD gIndentSpace; /* For indentation in output */
01590 WORD ggIndentSpace;
01591 WORD gShortStatsMax;
01592 WORD ggShortStatsMax;
01593 WORD gextrasymbols;
01594 WORD ggextrasymbols;
01595 WORD zerorhs;
01596 WORD onerhs;
01597 WORD havesortdir;
01598 WORD vectorzero; /* p0_ */
01599 WORD ClearStore;
01600 WORD BracketFactors[8];
01601 BOOL FromStdin; /* read the input from STDIN */
01602 BOOL IgnoreDeprecation; /* ignore deprecation warning */
01603 #ifdef WITHFLOAT
01604 #ifdef WITHPTHREADS
01605 PADPOSITION(17,30,62,84,(sizeof(pthread_rwlock_t)+sizeof(pthread_mutex_t)*2)+2);
01606 #else
01607 PADPOSITION(17,28,62,84,2);
01608 #endif
01609 #else
01610 #ifdef WITHPTHREADS
01611 PADPOSITION(17,30,62,84,(sizeof(pthread_rwlock_t)+sizeof(pthread_mutex_t)*2)+2);
01612 #else
01613 PADPOSITION(17,28,62,84,2);
01614 #endif
01615 #endif
01616 };
01617 /*
01618 #] M :
01619 #[ P : The P struct defines objects set by the preprocessor
01620 */
01621 struct P_const {
01622 LIST DollarList; /* (R) Dollar variables. Contains pointers
01623 to contents of the variables.*/
01624 LIST PreVarList; /* (R) List of preprocessor variables
01625 Points to contents. Can be changed */
01626 LIST LoopList; /* (P) List of do loops */
01627 LIST ProcList; /* (P) List of procedures */
01628 INSIDEINFO inside; /* Information during #inside/#endside */
01629 NAMESPACE *firstnamespace; /* is zero when no namespace specified */
01630 NAMESPACE *lastnamespace; /* is zero when no namespace specified */
01631 UBYTE *fullname; /* buffer for namespace expanded names */
01632 UBYTE **PreSwitchStrings; /* (P) The string in a switch */
01633 UBYTE *preStart; /* (P) Preprocessor instruction buffer */
01634 UBYTE *preStop; /* (P) end of preStart */
01635 UBYTE *preFill; /* (P) Filling point in preStart */
01636 UBYTE *procedureExtension; /* (P) Extension for procedure files (prc) */
01637 UBYTE *cprocedureExtension; /* (P) Extension after .clear */

```

```

01645     LONG    *PreAssignStack;        /* For nesting #name assignments */
01646     int    *PreIfStack;              /* (P) Tracks nesting of #if */
01647     int    *PreSwitchModes;          /* (P) Stack of switch status */
01648     int    *PreTypes;                /* (P) stack of #call, #do etc nesting */
01649 #ifdef WITHPTHREADS
01650     pthread_mutex_t PreVarLock;      /* (P) */
01651 #endif
01652     LONG    StopWatchZero;           /* For `timer_' and #reset timer */
01653     LONG    InOutBuf;                /* (P) Characters in the output buf in pre.c */
01654     LONG    pSize;                   /* (P) size of preStart */
01655     int     PreAssignFlag;            /* (C) Indicates #assign -> catch dollar */
01656     int     PreContinuation;         /* (C) Indicates whether the statement is new */
01657     int     PreproFlag;              /* (P) Internal use to mark work on prepro instr. */
01658     int     iBufError;               /* (P) Flag for errors with input buffer */
01659     int     PreOut;                  /* (P) Flag for #+ #- */
01660     int     PreSwitchLevel;          /* (P) Nesting of #switch */
01661     int     NumPreSwitchStrings;     /* (P) Size of PreSwitchStrings */
01662     int     MaxPreTypes;             /* (P) Size of PreTypes */
01663     int     NumPreTypes;             /* (P) Number of nesting objects in PreTypes */
01664     int     MaxPreIfLevel;           /* (C) Maximum number of nested #if. Dynamic */
01665     int     PreffLevel;              /* (C) Current position if PreIfStack */
01666     int     PreInsideLevel;          /* (C) #inside active? */
01667     int     DelayPrevar;             /* (P) Delaying prevar substitution */
01668     int     AllowDelay;              /* (P) Allow delayed prevar substitution */
01669     int     lhdollarerror;           /* (R) */
01670     int     eat;                     /* ( ) */
01671     int     gNumPre;                 /* (P) Number of preprocessor variables for .clear */
01672     int     PreDebug;                /* (C) */
01673     int     OpenDictionary;
01674     int     PreAssignLevel;          /* For nesting #name = ...; assignments */
01675     int     MaxPreAssignLevel;       /* For nesting #name = ...; assignments */
01676     int     fullnamesize;            /* size of the fullname buffer */
01677     WORD    DebugFlag;               /* (P) For debugging purposes */
01678     WORD    preError;                /* (P) Blocks certain types of execution */
01679     UBYTE    ComChar;                /* (P) Commentary character */
01680     UBYTE    cComChar;               /* (P) Old commentary character for .clear */
01681     PADPOINTER(3,22,2,2);
01682 };
01683
01684 /*
01685     #] P :
01686     #[ C : The C struct defines objects changed by the compiler
01687 */
01688
01697 struct C_const {
01698     set_of_char separators;
01699     POSITION StoreFileSize;           /* ( ) Size of store file */
01700     NAMETREE *dollarnames;
01701     NAMETREE *exprnames;
01702     NAMETREE *varnames;
01703     LIST    ChannelList;
01704
01705     /* Later also for write? */
01706     LIST    DubiousList;
01707     LIST    FunctionList;
01708     LIST    ExpressionList;
01709     LIST    IndexList;
01710     LIST    SetElementList;
01711     LIST    SetList;
01712     LIST    SymbolList;
01713     LIST    VectorList;
01714     LIST    PotModDollList;
01715     LIST    ModOptDollList;
01716     LIST    TableBaseList;
01717
01718     /* Compile buffer variables */
01719
01720     LIST    cbufList;
01721
01722     /* Objects for auto declarations */
01723
01724     LIST    AutoSymbolList;          /* (C) */
01725     LIST    AutoIndexList;           /* (C) */
01726     LIST    AutoVectorList;          /* (C) */
01727     LIST    AutoFunctionList;        /* (C) */
01728     NAMETREE *autonames;
01729     LIST    *Symbols;                /* (C) Pointer for autodeclare. Which list is
01730                                     it searching. Later also for subroutines */
01731
01732     LIST    *Indices;                /* (C) id. */
01733     LIST    *Vectors;                /* (C) id. */
01734     LIST    *Functions;              /* (C) id. */
01735     NAMETREE **activenames;
01736     STREAM    *Streams;
01737     STREAM    *CurrentStream;
01738     SWITCH    *SwitchArray;
01739     MODEL     **models;
01740     WORD       *SwitchHeap;
01741     LONG       *termstack;

```

```

01744     LONG      *termsortstack;
01745     UWORD     *cmod;
01746     UWORD     *powmod;
01747     UWORD     *modpowers;
01748     UWORD     *halfmod;           /* (C) half the modulus when not zero */
01749     WORD      *ProtoType;        /* (C) The subexpression prototype {wildcards} */
01750     WORD      *WildC;            /* (C) Filling point for wildcards. */
01751     LONG      *IfHeap;
01752     LONG      *IfCount;
01753     LONG      *IfStack;
01754     UBYTE     *iBuffer;
01755     UBYTE     *iPointer;
01756     UBYTE     *iStop;
01757     UBYTE     **LabelNames;
01758     WORD      *FixIndices;
01759     WORD      *termssumcheck;
01760     UBYTE     *WildcardNames;
01761     int        *Labels;
01762     SBYTE     *tokens;
01763     SBYTE     *toptokens;
01764     SBYTE     *endoftokens;
01765     WORD      *tokenarglevel;
01766     UWORD     *modinverses;      /* Table for inverses of primes */
01767     UBYTE     *Fortran90Kind;    /* The kind of number in Fortran 90 as in _ki */
01768     WORD      **MultiBracketBuf; /* Array of buffers for multi-level brackets */
01769     UBYTE     *extrasyms;        /* Array with the name for extra symbols in ToPolynomial */
01770     WORD      *doloopstack;      /* To keep track of begin and end of doloops */
01771     WORD      *doloopnest;      /* To keep track of nesting of doloops etc */
01772     char      *CheckpointRunAfter;
01773     char      *CheckpointRunBefore;
01774     WORD      *IfSumCheck;
01775     WORD      *CommuteInSet;     /* groups of noncommuting functions that can commute */
01776     UBYTE     *TestValue;        /* For debugging */
01777 #ifdef PARALLELCODE
01778     LONG      *inputnumbers;
01779     WORD      *pfirstnum;
01780 #endif
01781 #ifdef WITHPTHREADS
01782     pthread_mutex_t halfmodlock; /* () Lock for adding buffer for halfmod */
01783 #endif
01784     LONG      argstack[MAXNEST]; /* (C) {contents} Stack for nesting of Argument */
01785     LONG      insidestack[MAXNEST]; /* (C) {contents} Stack for Argument or Inside. */
01786     LONG      inextrstack[MAXNEST]; /* (C) {contents} Stack for Argument or Inside. */
01787     LONG      iBufferSize;        /* (C) Size of the input buffer */
01788     LONG      TransEname;         /* (C) Used when a new definition overwrites
01789                                     an old expression. */
01790     LONG      ProcessBucketSize;  /* (C) */
01791     LONG      mProcessBucketSize; /* (C) */
01792     LONG      CModule;           /* (C) Counter of current module */
01793     LONG      ThreadBucketSize;   /* (C) Roughly the maximum number of input terms */
01794     LONG      CheckpointStamp;
01795     LONG      CheckpointInterval;
01796 #ifdef WITHFLOAT
01797     LONG      DefaultPrecision;   /* (C) Default precision in bits for float_ */
01798     LONG      MaxWeight;          /* (C) Maximum weight for MZV or Euler */
01799     LONG      tDefaultPrecision;  /* (C) Default precision in bits for float_ */
01800     LONG      tMaxWeight;         /* (C) Maximum weight for MZV or Euler */
01801 #endif
01802     int        cbufnum;
01803     int        AutoDeclareFlag;
01804     int        NoShowInput;      /* (C) No listing of input as in .prc, #do */
01805     int        ShortStats;       /* (C) */
01806     int        compiletype;      /* (C) type of statement {DECLARATION etc} */
01807     int        firstconstindex;  /* (C) flag for giving first error message */
01808     int        insidefirst;      /* (C) Not used yet */
01809     int        minsidefirst;     /* (?) Not used yet */
01810     int        wildflag;         /* (C) Flag for marking use of wildcards */
01811     int        NumLabels;        /* (C) Number of labels {in Labels} */
01812     int        MaxLabels;        /* (C) Size of Labels array */
01813     int        lDefDim;          /* (C) */
01814     int        lDefDim4;         /* (C) */
01815     int        NumWildcardNames; /* (C) Number of ?a variables */
01816     int        WildcardBufferSize; /* (C) size of WildcardNames buffer */
01817     int        MaxIf;            /* (C) size of IfHeap, IfSumCheck, IfCount */
01818     int        NumStreams;       /* (C) */
01819     int        MaxNumStreams;    /* (C) */
01820     int        firstctypemessage; /* (C) Flag for giving first st order error */
01821     int        tablecheck;       /* (C) For table checking */
01822     int        idoption;         /* (C) */
01823     int        BottomLevel;      /* (C) For propercount. Not used!!! */
01824     int        CompileLevel;     /* (C) Subexpression level */
01825     int        TokensWriteFlag;  /* (C) */
01826     int        UnsureDollarMode; /* (C) ?Controls error messages undefined $'s */
01827     int        outsidefun;       /* (C) Used for writing Tables to file */
01828     int        funpowers;        /* (C) */
01829     int        WarnFlag;         /* (C) */
01830     int        StatsFlag;        /* (C) */

```



```

18135     int      NamesFlag;          /* (C) */
18136     int      CodesFlag;          /* (C) */
18137     int      SetupFlag;          /* (C) */
18138     int      SortType;           /* (C) */
18139     int      lSortType;          /* (C) */
18140     int      ThreadStats;        /* (C) */
18141     int      FinalStats;         /* (C) */
18142     int      OldParallelStats;   /* (C) */
18143     int      ThreadsFlag;        /* (C) */
18144     int      ThreadBalancing;    /* (C) */
18145     int      ThreadSortFileSynch; /* (C) */
18146     int      ProcessStats;       /* (C) */
18147     int      BracketNormalize;   /* (C) Indicates whether the bracket st is normalized */
18148     int      maxtermlevel;       /* (C) Size of termstack */
18149     int      dumnumflag;         /* (C) Where there dummy indices in tokenizer? */
18150     int      bracketindexflag;   /* (C) Are brackets going to be indexed? */
18151     int      parallelflag;       /* (C) parallel allowed? */
18152     int      mparallelflag;      /* (C) parallel allowed in this module? */
18153     int      inparallelflag;     /* (C) inparallel allowed? */
18154     int      partodoflag;        /* (C) parallel allowed? */
18155     int      properorderflag;    /* (C) clean normalizing. */
18156     int      vetofilling;        /* (C) vetoes overwriting in tablebase stubs */
18157     int      tablefilling;       /* (C) to notify AddrHS we are filling a table */
18158     int      vetotablebasefill;  /* (C) For the load in tablebase */
18159     int      exprfillwarning;    /* (C) Warning has been printed for expressions in fill statements */
*/
18160     int      lhdollarflag;       /* (R) left hand dollar present */
18161     int      NoCompress;         /* (R) Controls native compression */
18162     int      IsFortran90;        /* Tells whether the Fortran is Fortran90 */
18163     int      MultiBracketLevels; /* Number of elements in MultiBracketBuf */
18164     int      topolynomialflag;    /* To avoid ToPolynomial and FactArg together */
18165     int      ffbufnum;           /* Buffer number for user defined factorizations */
18166     int      OldFactArgFlag;     /* (C) */
18167     int      MemDebugFlag;       /* Only used when MALLOCDiEBUG in tools.c */
18168     int      OldGCDFlag;         /* (C) */
18169     int      WTimeStatsFlag;     /* (C) */
18170     int      SortReallocateFlag; /* Controls reallocation of large+small buffer at module end.
18171                                     0 : Off
18172                                     1 : On, every module (set by On sortreallocate;)
18173                                     2 : On, single module (set by #sortreallocate) */
18174     int      PrintBacktraceFlag; /* Print backtrace on terminate? */
18175     int      FlintPolyFlag;      /* Use Flint for polynomial arithmetic */
18176     int      HumanStatsFlag;     /* Print human-readable stats in the stats print? */
18177     int      doloopstacksize;    /* (C) */
18178     int      dolooplevel;        /* (C) */
18179     int      CheckpointFlag;     /* (C) */
18180     int      SizeCommuteInSet;    /* Size of the CommuteInSet buffer */
18181 #ifdef PARALLELCODE
18182     int      numpfirstnum;       /* For redefine */
18183     int      sizepfirstnum;      /* For redefine */
18184 #endif
18185     int      origin;             /* Determines whether .sort or ModuleOption */
18186     int      vectorlikeLHS;      /* (C) */
18187     int      nummodels;          /* (C) */
18188     int      modelspace;         /* (C) */
18189     int      ModelLevel;         /* (C) */
18190     WORD     argsumcheck[MAXNEST]; /* (C) Checking of nesting */
18191     WORD     insidesumcheck[MAXNEST]; /* (C) Checking of nesting */
18192     WORD     inexprsumcheck[MAXNEST]; /* (C) Checking of nesting */
18193     WORD     RepSumCheck[MAXREPEAT]; /* Checks nesting of repeat, if, argument */
18194     WORD     lUnitTrace[4];      /* (C) */
18195     WORD     RepLevel;           /* (C) Tracks nesting of repeat. */
18196     WORD     arglevel;           /* (C) level of nested argument statements */
18197     WORD     insidelevel;        /* (C) level of nested inside statements */
18198     WORD     inexprlevel;        /* (C) level of nested inexpr statements */
18199     WORD     termlevel;          /* (C) level of nested term statements */
18200     WORD     MustTestTable;      /* (C) Indicates whether tables have been changed */
18201     WORD     DumNum;             /* (C) */
18202     WORD     ncmod;              /* (C) Local setting of modulus. size of cmod */
18203     WORD     npowmod;            /* (C) Local printing as powers. size powmod */
18204     WORD     modmode;            /* (C) Mode for modulus calculus */
18205     WORD     nhalfmod;           /* relevant word size of halfmod when defined */
18206     WORD     DirtPow;            /* (C) Flag for changes in printing mod powers */
18207     WORD     lUnitTrace;         /* (C) Local value of Tr[1] */
18208     WORD     NwildC;             /* (C) Wildcard counter */
18209     WORD     ComDefer;           /* (C) defer brackets */
18210     WORD     CollectFun;         /* (C) Collect function number */
18211     WORD     AltCollectFun;      /* (C) Alternate Collect function number */
18212     WORD     OutputMode;         /* (C) */
18213     WORD     Cnumpows;           /* (C) */
18214     WORD     OutputSpaces;       /* (C) */
18215     WORD     OutNumberType;      /* (C) Controls RATIONAL/FLOAT output */
18216     WORD     DidClean;           /* (C) Test whether nametree needs cleaning */
18217     WORD     IfLevel;            /* (C) */
18218     WORD     WhileLevel;        /* (C) */
18219     WORD     SwitchLevel;        /* (C) */
18220     WORD     SwitchInArray;      /* (C) */

```



```

01924     WORD    MaxSwitch;
01925     WORD    LogHandle;           /* (C) The Log File */
01926     WORD    LineLength;         /* (C) */
01927     WORD    StoreHandle;        /* (C) Handle of .str file */
01928     WORD    HideLevel;          /* (C) Hiding indicator */
01929     WORD    lPolyFun;           /* (C) local value of PolyFun */
01930     WORD    lPolyFunInv;        /* (C) local value of Inverse of PolyFun */
01931     WORD    lPolyFunType;       /* (C) local value of PolyFunType */
01932     WORD    lPolyFunExp;
01933     WORD    lPolyFunVar;
01934     WORD    lPolyFunPow;
01935     WORD    SymChangeFlag;      /* (C) */
01936     WORD    CollectPercentage;  /* (C) Collect function percentage */
01937     WORD    ShortStatsMax;      /* For On FewerStatistics 10; */
01938     WORD    extrasymbols;       /* Flag for the extra symbols output mode */
01939     WORD    PolyRatFunChanged;  /* Keeps track whether we changed in the compiler */
01940     WORD    ToBeInFactors;
01941     WORD    InnerTest;          /* For debugging */
01942 #ifdef WITHMPI
01943     WORD    RhsExprInModuleFlag; /* (C) Set by the compiler if RHS expressions exists. */
01944 #endif
01945     UBYTE    Commercial[COMMERCIALSIZE+2]; /* (C) Message to be printed in statistics */
01946     UBYTE    debugFlags[MAXFLAGS+2]; /* On/Off Flag number(s) */
01947 #ifdef WITHFLOAT
01948 #if defined(WITHPTHREADS)
01949     PADPOSITION(50,12+3*MAXNEST,77,48+3*MAXNEST+MAXREPEAT,COMMERCIALSIZE+MAXFLAGS+4+sizeof(LIST)*17+sizeof(pthread_mutex_t));
01950 #elif defined(WITHMPI)
01951     PADPOSITION(50,12+3*MAXNEST,77,49+3*MAXNEST+MAXREPEAT,COMMERCIALSIZE+MAXFLAGS+4+sizeof(LIST)*17);
01952 #else
01953     PADPOSITION(48,12+3*MAXNEST,75,48+3*MAXNEST+MAXREPEAT,COMMERCIALSIZE+MAXFLAGS+4+sizeof(LIST)*17);
01954 #endif
01955 #else
01956 #if defined(WITHPTHREADS)
01957     PADPOSITION(50,8+3*MAXNEST,77,48+3*MAXNEST+MAXREPEAT,COMMERCIALSIZE+MAXFLAGS+4+sizeof(LIST)*17+sizeof(pthread_mutex_t));
01958 #elif defined(WITHMPI)
01959     PADPOSITION(50,8+3*MAXNEST,77,49+3*MAXNEST+MAXREPEAT,COMMERCIALSIZE+MAXFLAGS+4+sizeof(LIST)*17);
01960 #else
01961     PADPOSITION(48,8+3*MAXNEST,75,48+3*MAXNEST+MAXREPEAT,COMMERCIALSIZE+MAXFLAGS+4+sizeof(LIST)*17);
01962 #endif
01963 #endif
01964 };
01965 /*
01966     #] C :
01967     #[ S : The S struct defines objects changed at the start of the run (Processor)
01968           Basically only set by the master.
01969 */
01977 struct S_const {
01978     POSITION MaxExprSize;        /* ( ) Maximum size of in/out/sort */
01979 #ifdef WITHPTHREADS
01980     pthread_mutex_t inputslock;
01981     pthread_mutex_t outputslock;
01982     pthread_mutex_t MaxExprSizeLock;
01983 #endif
01984     POSITION *OldOnFile;         /* (S) File positions of expressions */
01985     WORD *OldNumFactors;        /* ( ) NumFactors in (old) expression */
01986     WORD *Oldvflags;           /* ( ) vflags in (old) expression */
01987     WORD *Olduflags;           /* ( ) uflags in (old) expression */
01988 #ifdef WITHFLOAT
01989     void *delta_1;
01990 #endif
01991     int NumOldOnFile;           /* (S) Number of expressions in OldOnFile */
01992     int NumOldNumFactors;       /* (S) Number of expressions in OldNumFactors */
01993     int MultiThreaded;         /* (S) Are we running multi-threaded? */
01994 #ifdef WITHPTHREADS
01995     int MasterSort;            /* Final stage of sorting to the master */
01996 #endif
01997 #ifdef WITHMPI
01998     int printflag;             /* controls MesPrint() on each slave */
01999 #endif
02000     int Balancing;             /* For second stage loadbalancing */
02001     WORD ExecMode;             /* (S) */
02002
02003     WORD CollectOverFlag;      /* (R) Indicates overflow at Collect */
02004 #ifdef WITHPTHREADS
02005     WORD sLevel;               /* Copy of AR0.sLevel because it can get messy */
02006 #endif
02007 #if defined(WITHPTHREADS)
02008     PADPOSITION(3,0,5,3,sizeof(pthread_mutex_t)*3);
02009 #elif defined(WITHMPI)
02010     PADPOSITION(4,0,5,2,0);
02011 #else
02012     PADPOSITION(4,0,4,2,0);
02013 #endif
02014 };
02015 /*

```

```

02016         #] S :
02017         #[ R : The R struct defines objects changed at run time.
02018             They determine the environment that has to be transferred
02019             together with a term during multithreaded execution.
02020     */
02029 struct R_const {
02030     FILEDATA    StoreData;          /* (O) */
02031     FILEHANDLE  Fscr[3];            /* (R) Dollars etc play with it too */
02032     FILEHANDLE  FoStage4[2];        /* (R) In Sort. Stage 4. */
02033     POSITION     DefPosition;         /* (R) Deferred position of keep brackets. */
02034     FILEHANDLE  *infile;             /* (R) Points alternately to Fscr[0] or Fscr[1] */
02035     FILEHANDLE  *outfile;           /* (R) Points alternately to Fscr[1] or Fscr[0] */
02036     FILEHANDLE  *hidefile;          /* (R) Points to Fscr[2] */
02037
02038     WORD        *CompressBuffer;     /* (M) */
02039     WORD        *ComprTop;           /* (M) */
02040     WORD        *CompressPointer;    /* (R) */
02041     COMPAREDUMMY CompareRoutine;
02042     ULONG       *wranfia;
02043     SBYTE       *moebiustable;
02044
02045     LONG        OldTime;             /* (R) Zero time. Needed in timer. */
02046     LONG        InInBuf;             /* (R) Characters in input buffer. Scratch files. */
02047     LONG        InHiBuf;            /* (R) Characters in hide buffer. Scratch file. */
02048     LONG        pWorkSize;           /* (R) Size of pWorkSpace */
02049     LONG        lWorkSize;           /* (R) Size of lWorkSpace */
02050     LONG        posWorkSize;         /* (R) Size of posWorkSpace */
02051     ULONG       wranfseed;
02052     int         NoCompress;           /* (R) Controls native compression */
02053     int         gzipCompress;        /* (R) Controls gzip compression */
02054     int         Cnumlhs;             /* Local copy of cbuf[rbufnum].numlhs */
02055     int         outtohide;           /* Indicates that output is directly to hide */
02056 #ifdef WITHPTHREADS
02057     int         exptodo;             /* The expression to do in parallel mode */
02058 #endif
02059     int         wranfcall;
02060     int         wranfnpair1;
02061     int         wranfnpair2;
02062 #if ( BITSINWORD == 32 )
02063     WORD        PrimeList[5000];
02064     WORD        numinprimelist;
02065     WORD        notfirstprime;
02066 #endif
02067     WORD        GetFile;             /* (R) Where to get the terms {like Hide} */
02068     WORD        KeptInHold;          /* (R) */
02069     WORD        BracketOn;           /* (R) Intensely used in poly_ */
02070     WORD        MaxBracket;          /* (R) Size of BrackBuf. Changed by poly_ */
02071     WORD        CurDum;              /* (R) Current maximum dummy number */
02072     WORD        DeferFlag;           /* (R) For deferred brackets */
02073     WORD        TePos;               /* (R) */
02074     WORD        sLevel;              /* (R) Sorting level */
02075     WORD        Stage4Name;          /* (R) Sorting only */
02076     WORD        GetOneFile;          /* (R) Getting from hide or regular */
02077     WORD        PolyFun;             /* (C) Number of the PolyFun function */
02078     WORD        PolyFunInv;          /* (C) Number of the Inverse of the PolyFun function */
02079     WORD        PolyFunType;         /* (I) value of PolyFunType */
02080     WORD        PolyFunExp;
02081     WORD        PolyFunVar;
02082     WORD        PolyFunPow;
02083     WORD        Eside;               /* (I) Tells which side of = sign */
02084     WORD        MaxDum;              /* Maximum dummy value in an expression */
02085     WORD        level;               /* Running level in Generator */
02086     WORD        expchanged;          /* (R) Info about expression */
02087     WORD        expflags;            /* (R) Info about expression */
02088     WORD        CurExpr;             /* (S) Number of current expression */
02089     WORD        SortType;            /* A copy of AC.SortType to play with */
02090     WORD        ShortSortCount;      /* For On FewerStatistics 10; */
02091     WORD        modeloptions;
02092     WORD        funoffset;
02093     WORD        moebiustablesize;
02094 #if ( BITSINWORD == 32 )
02095 #ifdef WITHPTHREADS
02096     PADPOSITION(8,7,8,5029,0);
02097 #else
02098     PADPOSITION(8,7,7,5029,0);
02099 #endif
02100 #else
02101 #ifdef WITHPTHREADS
02102     PADPOSITION(8,7,8,27,0);
02103 #else
02104     PADPOSITION(8,7,7,27,0);
02105 #endif
02106 #endif
02107 };
02108
02109 /*
02110     #] R :

```

```

02111     # [ T : These are variables that stay in each thread during multi threaded execution.
02112     */
02121 struct T_const {
02122     #ifdef WITHPTHREADS
02123         SORTBLOCK SB;
02124     #endif
02125     SORTING *S0; /* (-) The thread specific sort buffer */
02126     SORTING *SS; /* (R) Current sort buffer */
02127     NESTING Nest; /* (R) Nesting of function levels etc. */
02128     NESTING NestStop; /* (R) */
02129     NESTING NestPoin; /* (R) */
02130     WORD *BrackBuf; /* (R) Bracket buffer. Used by poly_ at runtime. */
02131     STORECACHE StoreCache; /* (R) Cache for picking up stored expr. */
02132     STORECACHE StoreCacheAlloc; /* (R) Permanent address of StoreCache to keep valgrind happy */
02133     WORD **pWorkSpace; /* (R) Workspace for pointers. Dynamic. */
02134     LONG *lWorkSpace; /* (R) Workspace for LONG. Dynamic. */
02135     POSITION *posWorkSpace; /* (R) Workspace for file positions */
02136     WORD *WorkSpace; /* (M) */
02137     WORD *WorkTop; /* (M) */
02138     WORD *WorkPointer; /* (R) Pointer in the Workspace heap. */
02139     int *RepCount; /* (M) Buffer for repeat nesting */
02140     int *RepTop; /* (M) Top of RepCount buffer */
02141     WORD *WildArgTaken; /* (N) Stack for wildcard pattern matching */
02142     UWORD *factorials; /* (T) buffer of factorials. Dynamic. */
02143     WORD *small_power_n; /* length of the number */
02144     UWORD **small_power; /* the number */
02145     UWORD *bernoullis; /* (T) The buffer with Bernoulli numbers. Dynamic. */
02146     WORD *primelist;
02147     LONG *pfac; /* (T) array of positions of factorials. Dynamic. */
02148     LONG *pBer; /* (T) array of positions of Bernoulli's. Dynamic. */
02149     WORD *TMaddr; /* (R) buffer for TestSub */
02150     WORD *WildMask; /* (N) Wildcard info during pattern matching */
02151     WORD *previousEfactor; /* (I) Cache for factors in expressions */
02152     WORD **TermMemHeap; /* For TermMalloc. Set zero in Checkpoint */
02153     UWORD **NumberMemHeap; /* For NumberMalloc. Set zero in Checkpoint */
02154     UWORD **CacheNumberMemHeap; /* For CacheNumberMalloc. Set zero in Checkpoint */
02155     BRACKETINFO *bracketinfo;
02156     WORD **ListPoly;
02157     WORD *ListSymbols;
02158     UWORD *NumMem;
02159     WORD *TopologiesTerm;
02160     WORD *TopologiesStart;
02161     #ifdef WITHFLOAT
02162         int *indil;
02163         int *indi2;
02164         void *mpf_tab1;
02165         void *mpf_tab2;
02166         void *aux_;
02167         void *auxr_;
02168     #endif
02169     PARTI partitions;
02170     LONG sBer; /* (T) Size of the Bernoullis buffer */
02171     LONG pWorkPointer; /* (R) Offset-pointer in pWorkSpace */
02172     LONG lWorkPointer; /* (R) Offset-pointer in lWorkSpace */
02173     LONG posWorkPointer; /* (R) Offset-pointer in posWorkSpace */
02174     LONG InNumMem;
02175     int sfact; /* (T) size of the factorials buffer */
02176     int mfac; /* (T) size of the pfac array. */
02177     int ebufnum; /* (R) extra compiler buffer */
02178     int fbufnum; /* extra compiler buffer for factorization cache */
02179     int allbufnum; /* extra compiler buffer for id,all */
02180     int aebufnum; /* extra compiler buffer for id,all */
02181     int idallflag; /* indicates use of id,all buffers */
02182     int idallnum;
02183     int idallmaxnum;
02184     int WildcardBufferSize; /* (I) local copy for updates */
02185     #ifdef WITHPTHREADS
02186         int identity; /* (I) When we work with B->T */
02187         int LoadBalancing; /* Needed for synchronization */
02188     #ifdef WITHSORTBOTS
02189         int SortBotIn1; /* Input stream 1 for a SortBot */
02190         int SortBotIn2; /* Input stream 2 for a SortBot */
02191     #endif
02192     #endif
02193     int TermMemMax; /* For TermMalloc. Set zero in Checkpoint */
02194     int TermMemTop; /* For TermMalloc. Set zero in Checkpoint */
02195     int NumberMemMax; /* For NumberMalloc. Set zero in Checkpoint */
02196     int NumberMemTop; /* For NumberMalloc. Set zero in Checkpoint */
02197     int CacheNumberMemMax; /* For CacheNumberMalloc. Set zero in Checkpoint */
02198     int CacheNumberMemTop; /* For CacheNumberMalloc. Set zero in Checkpoint */
02199     int bracketindexflag; /* Are brackets going to be indexed? */
02200     int optintimes; /* Number of the evaluation of the MCTS tree */
02201     int ListSymbolsSize;
02202     int NumListSymbols;
02203     int numpoly;
02204     int LeaveNegative;
02205     int TrimPower; /* Indicates trimming in polyratfun expansion */

```

```

02206 WORD    small_power_maxx;      /* size of the cache for small powers */
02207 WORD    small_power_maxn;      /* size of the cache for small powers */
02208 WORD    dummysubexp[SUBEXPSIZE+4]; /* () used in normal.c */
02209 WORD    comsym[8];              /* () Used in tools.c = {8,SYMBOL,4,0,1,1,1,3} */
02210 WORD    comnum[4];              /* () Used in tools.c = { 4,1,1,3 } */
02211 WORD    comfun[FUNHEAD+4];      /* () Used in tools.c = {7,FUNCTION,3,0,1,1,3} */
02212                                /* or { 8,FUNCTION,4,0,0,1,1,3 } */
02213 WORD    comind[7];              /* () Used in tools.c = {7,INDEX,3,0,1,1,3} */
02214 WORD    MinVecArg[7+ARGHEAD];   /* (N) but should be more local */
02215 WORD    FunArg[4+ARGHEAD+FUNHEAD]; /* (N) but can be more local */
02216 WORD    locwildvalue[SUBEXPSIZE]; /* () Used in argument.c = {SUBEXPRESSION,SUBEXPSIZE,0,0,0} */
02217 WORD    mulpat[SUBEXPSIZE+5];   /* () Used in argument.c = {TYPEMULT, SUBEXPSIZE+3, 0, */
02218                                /* SUBEXPRESSION, SUBEXPSIZE, 0, 1, 0, 0, 0 } */
02219 WORD    proexp[SUBEXPSIZE+5];    /* () Used in poly.c */
02220 WORD    TMout[40];              /* (R) Passing info */
02221 WORD    TMbuff;                 /* (R) Communication between TestSub and Genera */
02222 WORD    TMdolfac;               /* factor number for dollar */
02223 WORD    nfac;                   /* (T) Number of highest stored factorial */
02224 WORD    nBer;                   /* (T) Number of highest Bernoulli number. */
02225 WORD    mBer;                   /* (T) Size of buffer pBer. */
02226 WORD    PolyAct;                /* (R) Used for putting the PolyFun at end. ini at 0 */
02227 WORD    RecFlag;                /* (R) Used in TestSub. ini at zero. */
02228 WORD    inprimest;
02229 WORD    sizeprimelist;
02230 WORD    fromindex;              /* Tells the compare routine whether call from index */
02231 WORD    setinterntopo;          /* Set of internal momenta for topogen */
02232 WORD    setexterntopo;          /* Set of external momenta for topogen */
02233 WORD    TopologiesLevel;
02234 WORD    TopologiesOptions[2];
02235 #ifdef WITHFLOAT
02236 WORD    FloatPos;
02237 WORD    SortFloatMode;
02238 #endif
02239 #ifdef WITHFLOAT
02240 #ifdef WITHPTHREADS
02241 #ifdef WITHSORTBOTS
02242     PADPOINTER(5,27,107+SUBEXPSIZE*4+FUNHEAD*2+ARGHEAD*2,0);
02243 #else
02244     PADPOINTER(5,25,107+SUBEXPSIZE*4+FUNHEAD*2+ARGHEAD*2,0);
02245 #endif
02246 #else
02247     PADPOINTER(5,23,107+SUBEXPSIZE*4+FUNHEAD*2+ARGHEAD*2,0);
02248 #endif
02249 #else
02250 #ifdef WITHPTHREADS
02251 #ifdef WITHSORTBOTS
02252     PADPOINTER(5,27,105+SUBEXPSIZE*4+FUNHEAD*2+ARGHEAD*2,0);
02253 #else
02254     PADPOINTER(5,25,105+SUBEXPSIZE*4+FUNHEAD*2+ARGHEAD*2,0);
02255 #endif
02256 #else
02257     PADPOINTER(5,23,105+SUBEXPSIZE*4+FUNHEAD*2+ARGHEAD*2,0);
02258 #endif
02259 #endif
02260 };
02261 /*
02262     #] T :
02263     #[ N : The N struct contains variables used in running information
02264            that is inside blocks that should not be split, like pattern
02265            matching, traces etc. They are local for each thread.
02266            They don't need initializations.
02267 */
02276 struct N_const {
02277     POSITION OldPosIn;            /* (R) Used in sort. */
02278     POSITION OldPosOut;           /* (R) Used in sort */
02279     POSITION theposition;         /* () Used in index.c */
02280     WORD    *EndNest;            /* (R) Nesting of function levels etc. */
02281     WORD    *Frozen;             /* (R) Bracket info */
02282     WORD    *FullProto;          /* (R) Prototype of a subexpression or table */
02283     WORD    *cTerm;              /* (R) Current term for coef_ and term_ */
02284     int      *RepPoint;          /* (R) Pointer in RepCount buffer. Tracks repeat */
02285     WORD    *WildValue;          /* (N) Wildcard info during pattern matching */
02286     WORD    *WildStop;           /* (N) Wildcard info during pattern matching */
02287     WORD    *argaddress;         /* (N) Used in pattern matching of arguments */
02288     WORD    *RepFunList;         /* (N) For pattern matching */
02289     WORD    *patstop;            /* (N) Used in pattern matching */
02290     WORD    *terstop;            /* (N) Used in pattern matching */
02291     WORD    *terstart;           /* (N) Used in pattern matching */
02292     WORD    *terfirstcomm;       /* (N) Used in pattern matching */
02293     WORD    *DumFound;           /* (N) For renumbering indices {make local?} */
02294     WORD    **DumPlace;          /* (N) For renumbering indices {make local?} */
02295     WORD    **DumFunPlace;       /* (N) For renumbering indices {make local?} */
02296     WORD    *UsedSymbol;         /* (N) When storing terms of a global expr. */
02297     WORD    *UsedVector;         /* (N) When storing terms of a global expr. */
02298     WORD    *UsedIndex;          /* (N) When storing terms of a global expr. */
02299     WORD    *UsedFunction;       /* (N) When storing terms of a global expr. */
02300     WORD    *MaskPointer;        /* (N) For wildcard pattern matching */

```

```

02301     WORD    *ForFindOnly;        /* (N) For wildcard pattern matching */
02302     WORD    *findTerm;           /* (N) For wildcard pattern matching */
02303     WORD    *findPattern;        /* (N) For wildcard pattern matching */
02304 #ifdef WITHZLIB
02305     Bytef    **ziobufnum;        /* () Used in compress.c */
02306     Bytef    *ziobuffers;        /* () Used in compress.c */
02307 #endif
02308     WORD    *dummyrenumlist;      /* () Used in execute.c and store.c */
02309     int      *funargs;            /* () Used in lus.c */
02310     WORD    **funlocs;            /* () Used in lus.c */
02311     int      *funinds;            /* () Used in lus.c */
02312     UWORD    *NoScrat2;           /* () Used in normal.c */
02313     WORD    *ReplaceScrat;        /* () Used in normal.c */
02314     TRACES    *tracestack;        /* () used in opera.c */
02315     WORD    *selecttermundo;      /* () Used in pattern.c */
02316     WORD    *patternbuffer;       /* () Used in pattern.c */
02317     WORD    *termbuffer;          /* () Used in pattern.c */
02318     WORD    **PoinScrat;          /* () used in reshuf.c */
02319     WORD    **FunScrat;           /* () used in reshuf.c */
02320     WORD    *RenumScrat;          /* () used in reshuf.c */
02321     FUN_INFO *FunInfo;            /* () Used in smart.c */
02322     WORD    **SplitScrat;         /* () Used in sort.c */
02323     WORD    **SplitScrat1;        /* () Used in sort.c */
02324     SORTING  **FunSorts;          /* () Used in sort.c */
02325     UWORD    *SoScratC;           /* () Used in sort.c */
02326     WORD    *listinprint;         /* () Used in proces.c and message.c */
02327     WORD    *currentTerm;         /* () Used in proces.c and message.c */
02328     WORD    **arglist;            /* () Used in function.c */
02329     int      *tlistbuf;           /* () used in lus.c */
02330 #ifdef WHICHSUBEXPRESSION
02331     UWORD    *BinoScrat;          /* () Used in proces.c */
02332 #endif
02333     WORD    *compressSpace;       /* () Used in sort.c */
02334 #ifdef WITHPTHREADS
02335     THREADBUCKET *threadbuck;
02336     EXPRESSIONS expr;
02337 #endif
02338     UWORD    *SHcombi;
02339     WORD    *poly_vars;
02340     UWORD    *cmmod;              /* Local setting of modulus. Pointer to value. */
02341     SHvariables SHvar;
02342     LONG     deferskipped;        /* () Used in proces.c store.c and parallel.c */
02343     LONG     InScrat;             /* () Used in sort.c */
02344     LONG     SplitScratSize;      /* () Used in sort.c */
02345     LONG     InScrat1;            /* () Used in sort.c */
02346     LONG     SplitScratSize1;     /* () Used in sort.c */
02347     LONG     ninterms;            /* () Used in proces.c and sort.c */
02348 #ifdef WITHPTHREADS
02349     LONG     inputnumber;         /* () For use in redefine */
02350     LONG     lastinindex;
02351 #endif
02352 #ifdef WHICHSUBEXPRESSION
02353     LONG     last2;               /* () Used in proces.c */
02354     LONG     last3;               /* () Used in proces.c */
02355 #endif
02356     LONG     SHcombiSize;
02357     int      NumTotWildArgs;      /* (N) Used in pattern matching */
02358     int      UseFindOnly;         /* (N) Controls pattern routines */
02359     int      UsedOtherFind;       /* (N) Controls pattern routines */
02360     int      ErrorInDollar;       /* (R) */
02361     int      numfargs;            /* () Used in lus.c */
02362     int      numflocs;            /* () Used in lus.c */
02363     int      nargs;               /* () Used in lus.c */
02364     int      tohunt;              /* () Used in lus.c */
02365     int      numoffuns;           /* () Used in lus.c */
02366     int      funisize;            /* () Used in lus.c */
02367     int      Rssize;              /* () Used in normal.c */
02368     int      numtracesctack;      /* () used in opera.c */
02369     int      intracestack;        /* () used in opera.c */
02370     int      numfuninfo;          /* () Used in smart.c */
02371     int      NumFunSorts;         /* () Used in sort.c */
02372     int      MaxFunSorts;         /* () Used in sort.c */
02373     int      arglistsize;         /* () Used in function.c */
02374     int      tlistsize;           /* () used in lus.c */
02375     int      filenum;             /* () used in setfile.c */
02376     int      compressSize;        /* () Used in sort.c */
02377     int      polysortflag;
02378     int      nogroundlevel;       /* () Used to see whether pattern matching at groundlevel */
02379     int      subsubveto;          /* () Sabotage combining subexpressions in TestSub */
02380     WORD    *MaxRenumScrat;       /* () used in reshuf.c */
02381     WORD    oldtype;              /* (N) WildCard info at pattern matching */
02382     WORD    oldvalue;             /* (N) WildCard info at pattern matching */
02383     WORD    NumWild;              /* (N) Used in Wildcard */
02384     WORD    RepFunNum;            /* (N) Used in pattern matching */
02385     WORD    DisOrderFlag;         /* (N) Disorder option? Used in pattern matching */
02386     WORD    WildDirt;             /* (N) dirty in wildcard substitution. */
02387     WORD    NumFound;             /* (N) in reshuf only. Local? */

```

```

02388     WORD    WildReserve;           /* (N) Used in the wildcards */
02389     WORD    TeInFun;               /* (R) Passing type of action */
02390     WORD    TeSuOut;               /* (R) Passing info. Local? */
02391     WORD    WildArgs;              /* (R) */
02392     WORD    WildEat;               /* (R) */
02393     WORD    PolyNormFlag;          /* (R) For polynomial arithmetic */
02394     WORD    PolyFunTodo;           /* deals with expansions and multiplications */
02395     WORD    sizeselecttermundo;    /* () Used in pattern.c */
02396     WORD    patternbuffersize;     /* () Used in pattern.c */
02397     WORD    numlistinprint;        /* () Used in process.c */
02398     WORD    ncmod;                 /* () used as some type of flag to disable */
02399     WORD    ExpectedSign;
02400     WORD    SignCheck;
02401     WORD    IndDum;                /* Active dummy indices */
02402     WORD    poly_num_vars;
02403     WORD    idfunctionflag;
02404     WORD    poly_vars_type;        /* type of allocation. For free. */
02405     WORD    tryterm;               /* For EndSort(...,2) */
02406 #ifdef WHICHSUBEXPRESSION
02407     WORD    nbino;                 /* () Used in proces.c */
02408     WORD    last1;                 /* () Used in proces.c */
02409 #endif
02410 #ifdef WITHPTHREADS
02411 #ifdef WHICHSUBEXPRESSION
02412 #ifdef WITHZLIB
02413     PADPOSITION(55,11,23,28,sizeof(SHvariables));
02414 #else
02415     PADPOSITION(53,11,23,28,sizeof(SHvariables));
02416 #endif
02417 #else
02418 #ifdef WITHZLIB
02419     PADPOSITION(54,9,23,26,sizeof(SHvariables));
02420 #else
02421     PADPOSITION(52,9,23,26,sizeof(SHvariables));
02422 #endif
02423 #endif
02424 #else
02425 #ifdef WHICHSUBEXPRESSION
02426 #ifdef WITHZLIB
02427     PADPOSITION(53,9,23,28,sizeof(SHvariables));
02428 #else
02429     PADPOSITION(51,9,23,28,sizeof(SHvariables));
02430 #endif
02431 #else
02432 #ifdef WITHZLIB
02433     PADPOSITION(52,7,23,26,sizeof(SHvariables));
02434 #else
02435     PADPOSITION(50,7,23,26,sizeof(SHvariables));
02436 #endif
02437 #endif
02438 #endif
02439 };
02440
02441 /*
02442     #] N :
02443     #[ O : The O struct concerns output variables
02444 */
02453 struct O_const {
02454     FILEDATA    SaveData;           /* (O) */
02455     STOREHEADER SaveHeader;         /* () System Independent save-Files */
02456     OPTIMIZERESULT OptimizeResult;
02457     UBYTE    *OutputLine;           /* (O) Sits also in debug statements */
02458     UBYTE    *OutStop;              /* (O) Top of OutputLine buffer */
02459     UBYTE    *OutFill;              /* (O) Filling point in OutputLine buffer */
02460     WORD    *bracket;               /* (O) For writing brackets */
02461     WORD    *termbuf;               /* (O) For writing terms */
02462     WORD    *tabstring;
02463     UBYTE    *wpos;                 /* (O) Only when storing file {local?} */
02464     UBYTE    *wpoin;                /* (O) Only when storing file {local?} */
02465     UBYTE    *DollarOutBuffer;      /* (O) Outputbuffer for Dollars */
02466     UBYTE    *CurBufWrt;           /* (O) Name of currently written expr. */
02467     void    (*FlipWORD)(UBYTE *); /* () Function pointers for translations. Initialized by
ReadSaveHeader() */
02468     void    (*FlipLONG)(UBYTE *);
02469     void    (*FlipPOS)(UBYTE *);
02470     void    (*FlipPOINTER)(UBYTE *);
02471     void    (*ResizeData)(UBYTE *,int,UBYTE *,int);
02472     void    (*ResizeWORD)(UBYTE *,UBYTE *);
02473     void    (*ResizeNCWORD)(UBYTE *,UBYTE *);
02474     void    (*ResizeLONG)(UBYTE *,UBYTE *);
02475     void    (*ResizePOS)(UBYTE *,UBYTE *);
02476     void    (*ResizePOINTER)(UBYTE *,UBYTE *);
02477     void    (*CheckPower)(UBYTE *);
02478     void    (*ReNUMBERVec)(UBYTE *);
02479     DICTIONARY **Dictionaries;
02480     UBYTE    *tensorList;           /* Dynamically allocated list with functions that are tensorial. */
02481     WORD    *inscheme;              /* for feeding a Horner scheme to Optimize */

```



```

02482 #ifdef WITHFLOAT
02483     UBYTE    *floatspace;
02484     LONG     floatsize;
02485 #endif
02486 /*-----Leave NumInBrack as first non-pointer. This is used by the checkpoints---*/
02487     LONG     NumInBrack;          /* (0) For typing [] option in print */
02488     LONG     wlen;                /* (0) Used to store files. */
02489     LONG     DollarOutSizeBuffer; /* (0) Size of DollarOutBuffer */
02490     LONG     DollarInOutBuffer;   /* (0) Characters in DollarOutBuffer */
02491 #if defined(mBSD) && defined(MICROTIME)
02492     LONG     wrap;                /* (0) For statistics time. wrap around */
02493     LONG     wrapnum;             /* (0) For statistics time. wrap around */
02494 #endif
02495     OPTIMIZE Optimize;
02496     int      OutInBuffer;         /* (0) Which routine does the writing */
02497     int      NoSpacesInNumbers;   /* For very long numbers */
02498     int      BlockSpaces;         /* For very long numbers */
02499     int      CurrentDictionary;
02500     int      SizeDictionaries;
02501     int      NumDictionaries;
02502     int      CurDictNumbers;
02503     int      CurDictVariables;
02504     int      CurDictSpecials;
02505     int      CurDictFunWithArgs;
02506     int      CurDictNumberWarning;
02507     int      CurDictNotInFunctions;
02508     int      CurDictInDollars;
02509     int      gNumDictionaries;
02510 #ifdef WITHFLOAT
02511     int      FloatPrec;
02512 #endif
02513     WORD     schemenum;           /* for feeding a Horner scheme to Optimize */
02514     WORD     transFlag;           /* () >0 indicates that translations have to be done */
02515     WORD     powerFlag;           /* () >0 indicates that some exponents/powers had to be adjusted */
02516 /*
02517     WORD     mpower;              /* For maxpower adjustment to larger value */
02518     WORD     resizeFlag;          /* () >0 indicates that something went wrong when resizing words */
02519 /*
02518     WORD     bufferedInd;         /* () Contains extra INDEXENTRIES, see ReadSaveIndex() for an
    explanation */
02519     WORD     OutSkip;             /* (0) How many chars to skip in output line */
02520     WORD     IsBracket;           /* (0) Controls brackets */
02521     WORD     InFbrack;            /* (0) For writing only */
02522     WORD     PrintType;           /* (0) */
02523     WORD     FortFirst;           /* (0) Only in sch.c */
02524     WORD     DoubleFlag;          /* (0) Output in double precision */
02525     WORD     IndentSpace;         /* For indentation in output */
02526     WORD     FactorMode;          /* When the output should be written as factors */
02527     WORD     FactorNum;           /* Number of factor currently treated */
02528     WORD     ErrorBlock;
02529     WORD     OptimizationLevel;   /* Level of optimization in the output */
02530     UBYTE    FortDotChar;        /* (0) */
02531 */
02532     /* For the padding, please count also the number of int's in the OPTIMIZE struct.
02533 */
02534 #ifdef WITHFLOAT
02535 #if defined(mBSD) && defined(MICROTIME)
02536     PADPOSITION(25,7,36,18,1);
02537 #else
02538     PADPOSITION(25,5,36,18,1);
02539 #endif
02540 #else
02541 #if defined(mBSD) && defined(MICROTIME)
02542     PADPOSITION(25,6,36,17,1);
02543 #else
02544     PADPOSITION(25,4,36,17,1);
02545 #endif
02546 #endif
02547 };
02548 /*
02549     #] O :
02550     #[ X : The X struct contains variables that deal with the external channel
02551 */
02552 struct X_const {
02553     UBYTE    *currentPrompt;
02554     UBYTE    *shellname;          /* if !=NULL (default is "/bin/sh -c"), start in
    the specified subshell*/
02555     UBYTE    *stderrname;         /* If !=NULL (default if "/dev/null"), stderr is
    redirected to the specified file*/
02556     int      timeout;             /* timeout to initialize preset channels.
    If timeout<0, the preset channels are
    already initialized*/
02557     int      killSignal;          /* signal number, SIGKILL by default*/
02558     int      killWholeGroup;      /* if 0, the signal is sent only to a process,
    if !=0 (default) is sent to a whole process group*/
02559     int      daemonize;           /* if !=0 (default), start in a daemon mode */
02560     int      currentExternalChannel;

```

```

02573     PADPOINTER(0,5,0,0);
02574 };
02575 /*
02576     #] X :
02577     #[ Definitions :
02578
02579     Note: we changed the definition from C to Cc in version 5.
02580     The reason is that everywhere the pointer to the compiler
02581     buffer was called C as well. Somehow that gave no problems but
02582     who knows what the future might bring.
02583 */
02584
02585 #ifdef WITHPTHREADS
02586
02592 typedef struct AllGlobals {
02593     struct M_const M;
02594     struct C_const Cc;
02595     struct S_const S;
02596     struct O_const O;
02597     struct P_const P;
02598     struct X_const X;
02599     PADPOSITION(0,0,0,0,sizeof(struct P_const)+sizeof(struct X_const));
02600 } ALLGLOBALS;
02601
02607 typedef struct AllPrivates {
02608     struct R_const R;
02609     struct N_const N;
02610     struct T_const T;
02611     PADPOSITION(0,0,0,0,sizeof(struct T_const));
02612 } ALLPRIVATES;
02613
02614 #else
02615
02620 typedef struct AllGlobals {
02621     struct M_const M;
02622     struct C_const Cc;
02623     struct S_const S;
02624     struct R_const R;
02625     struct N_const N;
02626     struct O_const O;
02627     struct P_const P;
02628     struct T_const T;
02629     struct X_const X;
02630     PADPOSITION(0,0,0,0,sizeof(struct P_const)+sizeof(struct T_const)+sizeof(struct X_const));
02631 } ALLGLOBALS;
02632
02633 #endif
02634
02635 /*
02636     #] Definitions :
02637     #] A :
02638     #[ FG :
02639 */
02640
02641 #ifdef WITHPTHREADS
02642 #define PHEAD ALLPRIVATES *B,
02643 #define PHEAD0 ALLPRIVATES *B
02644 #define BHEAD B,
02645 #define BHEAD0 B
02646 #else
02647 #define PHEAD
02648 #define PHEAD0 void
02649 #define BHEAD
02650 #define BHEAD0
02651 #endif
02652
02653 #ifdef ANSI
02654 typedef WORD (*WCN) (PHEAD WORD *,WORD *,WORD,WORD);
02655 typedef WORD (*WCN2) (PHEAD WORD *,WORD *);
02656 #else
02657 typedef WORD (*WCN) ();
02658 typedef WORD (*WCN2) ();
02659 #endif
02660
02661 typedef WORD (*COMPARE) (PHEAD WORD *,WORD *,WORD);
02662
02674 typedef struct FixedGlobals {
02675     WCN      Operation[8];
02676     WCN2     OperaFind[6];
02677     char     *VarType[10];
02678     char     *ExprStat[21];
02679     char     *FunNam[2];
02680     char     *swmes[3];
02681     char     *fname;
02682     char     *fname2;
02683     UBYTE    *s_one;
02684     WORD     fnamebase;

```



```

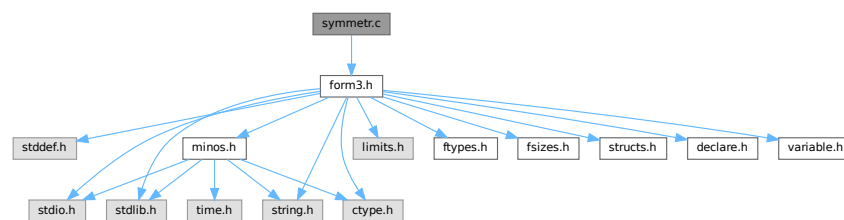
02685     WORD    fname2base;
02686     WORD    fname2size;
02687     WORD    fname2size;
02688     UINT    cTable[256];
02689 } FIXEDGLOBALS;
02690
02691 /*
02692  #] FG :
02693  */
02694
02695 #endif

```

4.138 symmetr.c File Reference

```
#include "form3.h"
```

Include dependency graph for symmetr.c:



Functions

- WORD [MatchE](#) (PHEAD WORD *pattern, WORD *fun, WORD *inter, WORD par)
- WORD [Permute](#) (PERM *perm, WORD first)
- WORD [PermuteP](#) (PERMP *perm, WORD first)
- WORD [Distribute](#) (DISTRIBUTE *d, WORD first)
- int [MatchCy](#) (PHEAD WORD *pattern, WORD *fun, WORD *inter, WORD par)
- int [FunMatchCy](#) (PHEAD WORD *pattern, WORD *fun, WORD *inter, WORD par)
- int [FunMatchSy](#) (PHEAD WORD *pattern, WORD *fun, WORD *inter, WORD par)
- int [MatchArgument](#) (PHEAD WORD *arg, WORD *pat)

4.138.1 Detailed Description

The routines that deal with the pattern matching of functions with symmetric properties.

Definition in file [symmetr.c](#).

4.138.2 Function Documentation

4.138.2.1 MatchE()

```

WORD MatchE (
    PHEAD WORD * pattern,
    WORD * fun,
    WORD * inter,
    WORD par )

```

Definition at line 56 of file [symmetr.c](#).

4.138.2.2 Permute()

```
WORD Permute (
    PERM * perm,
    WORD first )
```

Definition at line 443 of file [symmetr.c](#).

4.138.2.3 PermuteP()

```
WORD PermuteP (
    PERMP * perm,
    WORD first )
```

Definition at line 477 of file [symmetr.c](#).

4.138.2.4 Distribute()

```
WORD Distribute (
    DISTRIBUTE * d,
    WORD first )
```

Definition at line 509 of file [symmetr.c](#).

4.138.2.5 MatchCy()

```
int MatchCy (
    PHEAD WORD * pattern,
    WORD * fun,
    WORD * inter,
    WORD par )
```

Definition at line 570 of file [symmetr.c](#).

4.138.2.6 FunMatchCy()

```
int FunMatchCy (
    PHEAD WORD * pattern,
    WORD * fun,
    WORD * inter,
    WORD par )
```

Definition at line 1066 of file [symmetr.c](#).

4.138.2.7 FunMatchSy()

```
int FunMatchSy (
    PHEAD WORD * pattern,
    WORD * fun,
    WORD * inter,
    WORD par )
```

Definition at line 1524 of file [symmetr.c](#).

4.138.2.8 MatchArgument()

```
int MatchArgument (
    PHEAD WORD * arg,
    WORD * pat )
```

Definition at line 2051 of file [symmetr.c](#).

4.139 symmetr.c

[Go to the documentation of this file.](#)

```
00001
00006 /* #[] License : */
00007 /*
00008 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00009 * When using this file you are requested to refer to the publication
00010 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00011 * This is considered a matter of courtesy as the development was paid
00012 * for by FOM the Dutch physics granting agency and we would like to
00013 * be able to track its scientific use to convince FOM of its value
00014 * for the community.
00015 *
00016 * This file is part of FORM.
00017 *
00018 * FORM is free software: you can redistribute it and/or modify it under the
00019 * terms of the GNU General Public License as published by the Free Software
00020 * Foundation, either version 3 of the License, or (at your option) any later
00021 * version.
00022 *
00023 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00024 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00025 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00026 * details.
00027 *
00028 * You should have received a copy of the GNU General Public License along
00029 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00030 */
00031 /* #[] License : */
00032 /*
00033 #[] Includes : function.c
00034 */
00035
00036 #include "form3.h"
00037
00038 /*
00039 #[] Includes :
00040 #[] MatchE : WORD MatchE(pattern,fun,inter,par)
00041
00042 Matches symmetric and antisymmetric tensors.
00043 Pattern and fun point at a tensor.
00044 Problem is the wildcarding and all its possible permutations.
00045 This routine loops over all of them and calls for each
00046 possible wildcarding the recursion in ScanFunctions.
00047 Note that this can be very costly.
00048
00049 Originally this routine did only Levi Civita tensors and hence
00050 it dealt only with commuting objects.
00051 Because of the backtracking we cannot fall back to the calling
00052 ScanFunctions routine and check the sequence of functions when
00053 non-commuting objects are involved.
00054 */
00055
00056 WORD MatchE(PHEAD WORD *pattern, WORD *fun, WORD *inter, WORD par)
00057 {
00058     GETBIDENTITY
00059     WORD *m, *t, *r, i, retval;
00060     WORD *mstop, *tstop, j, newvalue, newfun;
00061     WORD fixvec[MAXMATCH],wcvec[MAXMATCH],fixind[MAXMATCH],wcind[MAXMATCH];
00062     WORD tfixvec[MAXMATCH],tfixind[MAXMATCH];
00063     WORD vvc,vfix,ifix,iwc,tvfix,tfix,nv,ni;
00064     WORD sign = 0, *rstop, first1, first2, first3, funwild;
00065     WORD *OldWork, nwstore, oRepFunNum;
00066     PERM perm1,perm2;
00067     DISTRIBUTE distr;
00068     WORD *newpat, /* *newter, *instart, */ offset;
00069     /* instart = fun; */
00070     offset = WORDDIF(fun,AN.terstart);
```

```

00071     if ( pattern[1] != fun[1] ) return(0);
00072     if ( *pattern >= FUNCTION+WILDOFFSET ) {
00073         if ( CheckWild(BHEAD *pattern-WILDOFFSET,FUNTOFUN,*fun,&newfun) ) return(0);
00074         funwild = 1;
00075     }
00076     else funwild = 0;
00077     mstop = pattern + pattern[1];
00078     tstop = fun + fun[1];
00079     m = pattern + FUNHEAD;
00080     t = fun + FUNHEAD;
00081     while ( m < mstop ) {
00082         if ( *m != *t ) break;
00083         m++; t++;
00084     }
00085     if ( m >= mstop ) {
00086         AN.RepFunList[AN.RepFunNum++] = offset;
00087         AN.RepFunList[AN.RepFunNum++] = 0;
00088         newpat = pattern + pattern[1];
00089         if ( funwild ) {
00090             m = AN.WildValue;
00091             t = OldWork = AT.WorkPointer;
00092             nwstore = i = (m[-SUBEXPSIZE+1]-SUBEXPSIZE)/4;
00093             r = AT.WildMask;
00094             if ( i > 0 ) {
00095                 do {
00096                     *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *r++;
00097                 } while ( --i > 0 );
00098             }
00099             if ( t >= AT.WorkTop ) {
00100                 MLOCK(ErrorMessageLock);
00101                 MesWork();
00102                 MUNLOCK(ErrorMessageLock);
00103                 return(-1);
00104             }
00105             AT.WorkPointer = t;
00106             AddWild(BHEAD *pattern-WILDOFFSET,FUNTOFUN,newfun);
00107             if ( newpat >= AN.patstop ) {
00108                 if ( AN.UseFindOnly == 0 ) {
00109                     if ( FindOnce(BHEAD AN.findTerm,AN.findPattern) ) {
00110                         AN.UsedOtherFind = 1;
00111                         return(1);
00112                     }
00113                     retval = 0;
00114                 }
00115                 else return(1);
00116             }
00117             else {
00118                 /* newter = instart; */
00119                 retval = ScanFunctions(BHEAD newpat,inter,par);
00120             }
00121             if ( retval == 0 ) {
00122                 m = AN.WildValue;
00123                 t = OldWork; r = AT.WildMask; i = nwstore;
00124                 if ( i > 0 ) {
00125                     do {
00126                         *m++ = *t++; *m++ = *t++; *m++ = *t++; *m++ = *t++; *m++ = *t++; *r++ = *t++;
00127                     } while ( --i > 0 );
00128                 }
00129             }
00130             AT.WorkPointer = OldWork;
00131             return(retval);
00132         }
00133         else {
00134             if ( newpat >= AN.patstop ) {
00135                 if ( AN.UseFindOnly == 0 ) {
00136                     if ( FindOnce(BHEAD AN.findTerm,AN.findPattern) ) {
00137                         AN.UsedOtherFind = 1;
00138                         return(1);
00139                     }
00140                     else return(0);
00141                 }
00142                 else return(1);
00143             }
00144             /* newter = instart; */
00145             i = ScanFunctions(BHEAD newpat,inter,par);
00146             return(i);
00147         }
00148     }
00149     /* Now the recursion
00150     */
00151 }
00152 /*
00153 Strategy:
00154 1: match the fixed arguments
00155 2: match, permuting the wildcards if needed.
00156 3: keep track of sign.
00157 */

```

```

00158     vwc = 0;
00159     vfix = 0;
00160     ifix = 0;
00161     iwc = 0;
00162     r = pattern+FUNHEAD;
00163     while ( r < mstop ) {
00164         if ( *r < (AM.OffsetVector+WILDOFFSET) ) {
00165             fixvec[vfix++] = *r;          /* Fixed vectors */
00166             sign += vwc + ifix + iwc;
00167         }
00168         else if ( *r < MINSPEC ) {
00169             wcvec[vwc++] = *r;          /* Wildcard vectors */
00170             sign += ifix + iwc;
00171         }
00172         else if ( *r < (AM.OffsetIndex+WILDOFFSET) ) {
00173             fixind[ifix++] = *r;        /* Fixed indices */
00174             sign += iwc;
00175         }
00176         else if ( *r < (AM.OffsetIndex+(WILDOFFSET«1)) ) {
00177             wcind[iwc++] = *r;          /* Wildcard indices */
00178         }
00179         else {
00180             fixind[ifix++] = *r;        /* Generated indices ~ fixed */
00181             sign += iwc;
00182         }
00183         r++;
00184     }
00185     if ( iwc == 0 && vwc == 0 ) return(0);
00186     tvfix = tifix = 0;
00187     t = fun + FUNHEAD;
00188     m = fixvec;
00189     mstop = m + vfix;
00190     r = fixind;
00191     rstop = r + ifix;
00192     nv = 0; ni = 0;
00193     while ( t < tstop ) {
00194         if ( *t < 0 ) {
00195             nv++;
00196             if ( m < mstop && *t == *m ) {
00197                 m++;
00198             }
00199             else {
00200                 sign += WORDDIF(mstop,m);
00201                 tfixvec[tfix++] = *t;
00202             }
00203         }
00204         else {
00205             ni++;
00206             if ( r < rstop && *r == *t ) {
00207                 r++;
00208             }
00209             else {
00210                 sign += WORDDIF(rstop,r);
00211                 tfixind[tfix++] = *t;
00212             }
00213         }
00214         t++;
00215     }
00216     if ( m < mstop || r < rstop ) return(0);
00217     if ( tvfix < vwc || (tvfix+tifix) < (vwc+iwc) ) return(0);
00218     sign += ( nv - vfix - vwc ) & ni;
00219     /*
00220     Take now the wildcards that have an assignment already.
00221     See whether they match.
00222     */
00223     {
00224         WORD *wv, *wm, n;
00225         wm = AT.WildMask;
00226         wv = AN.WildValue;
00227         n = AN.NumWild;
00228         do {
00229             if ( *wm ) {
00230                 if ( *wv == VECTOVEC ) {
00231                     for ( ni = 0; ni < vwc; ni++ ) {
00232                         if ( wcvec[ni]-WILDOFFSET == wv[2] ) { /* Has been assigned */
00233                             sign += ni;
00234                             vwc--;
00235                             while ( ni < vwc ) {
00236                                 wcvec[ni] = wcvec[ni+1];
00237                                 ni++;
00238                             }
00239                             /* TryVect: */
00240                             for ( ni = 0; ni < tvfix; ni++ ) {
00241                                 if ( tfixvec[ni] == wv[3] ) {
00242                                     sign += ni;
00243                                     tvfix--;
00244                                     while ( ni < tvfix ) {

```

```

00245         tfixvec[ni] = tfixvec[ni+1];
00246         ni++;
00247     }
00248     goto NextWV;
00249 }
00250 }
00251 return(0);
00252 }
00253 }
00254 }
00255 else if ( *wv == INDTOIND ) {
00256     for ( ni = 0; ni < iwc; ni++ ) {
00257         if ( wcind[ni]-WILDOFFSET == wv[2] ) { /* Has been assigned */
00258             sign += ni;
00259             iwc--;
00260             while ( ni < iwc ) {
00261                 wcind[ni] = wcind[ni+1];
00262                 ni++;
00263             }
00264             for ( ni = 0; ni < tfix; ni++ ) {
00265                 if ( tfixind[ni] == wv[3] ) {
00266                     sign += ni;
00267                     tfix--;
00268                     while ( ni < tfix ) {
00269                         tfixind[ni] = tfixind[ni+1];
00270                         ni++;
00271                     }
00272                     goto NextWV;
00273                 }
00274             }
00275             /* goto TryVect; */
00276             return(0);
00277         }
00278     }
00279 }
00280 }
00281 else if ( *wv == VECTOSUB ) {
00282     for ( ni = 0; ni < vwc; ni++ ) {
00283         if ( wcvect[ni]-WILDOFFSET == wv[2] ) return(0);
00284     }
00285 }
00286 else if ( *wv == INDTOSUB ) {
00287     for ( ni = 0; ni < iwc; ni++ ) {
00288         if ( wcind[ni]-WILDOFFSET == wv[2] ) return(0);
00289     }
00290 }
00291 }
00292 NextWV:
00293     wm++;
00294     wv += wv[1];
00295     n--;
00296     if ( n > 0 ) {
00297         while ( n > 0 && ( *wv == FROMSET || *wv == SETTONUM
00298             || *wv == LOADDOLLAR ) ) { wv += wv[1]; wm++; n--; }
00299     /*
00300         Freak problem: doesn't test for n and ran into a remaining
00301         code equal to SETTONUM followed by a big number and then
00302         ran out of the memory.
00303
00304         while ( *wv == FROMSET || *wv == SETTONUM
00305             || ( *wv == LOADDOLLAR && n > 0 ) ) { wv += wv[1]; wm++; n--; }
00306     */
00307     }
00308     } while ( n > 0 );
00309 }
00310 /*
00311     Now there are only free wildcards left.
00312     Possibly the assigned values ate too many vectors.
00313     The rest has to be done the 'hard way' via permutations.
00314     This is too bad when there are 10 indices.
00315     This could cause 10! tries.
00316     We try to avoid the worst case by using a very special
00317     (somewhat slow) permutation routine that has as its worst
00318     cases some rather unlikely configurations, rather than some
00319     common ones (as would have been the case with the conventional
00320     permutation routine).
00321     assume:
00322         vvvvvvvvvvvviiiiiii      (tvfix in tfixvec and tfix in tfixind)
00323         VVVVVVVVVIIIIIIIIII     (vwc in wcvect and iwc in wcind)
00324     Note: all further assignments are possible at this point!
00325     Strategy:
00326         permute v
00327         permute i
00328         loop over the ordered distribution of the leftover v's
00329         through the i's.
00330 */
00331     if ( tvfix < vwc ) { return(0); }

```

```

00332     perm1.n = tvfix;
00333     perm1.sign = 0;
00334     perm1.objects = tfixvec;
00335     perm2.n = tfix;
00336     perm2.sign = 0;
00337     perm2.objects = tfixind;
00338     distr.n1 = tvfix - vwc;
00339     distr.n2 = tfix;
00340     distr.obj1 = tfixvec + vwc;
00341     distr.obj2 = tfixind;
00342     distr.out = fixvec; /* For scratch */
00343     first1 = 1;
00344 /*
00345         Store the current Wildcard assignments
00346 */
00347     m = AN.WildValue;
00348     t = OldWork = AT.WorkPointer;
00349     nwstore = i = (m[-SUBEXPSIZE+1]-SUBEXPSIZE)/4;
00350     r = AT.WildMask;
00351     if ( i > 0 ) {
00352         do {
00353             *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *r++;
00354         } while ( --i > 0 );
00355     }
00356     if ( t >= AT.WorkTop ) {
00357         MLOCK(ErrorMessageLock);
00358         MesWork();
00359         MUNLOCK(ErrorMessageLock);
00360         return(-1);
00361     }
00362     AT.WorkPointer = t;
00363     while ( (first1 = Permute(&perm1,first1) ) == 0 ) {
00364         first2 = 1;
00365         while ( (first2 = Permute(&perm2,first2) ) == 0 ) {
00366             first3 = 1;
00367             while ( (first3 = Distribute(&distr,first3) ) == 0 ) {
00368 /*
00369                 Make now the wildcard assignments
00370 */
00371                 for ( i = 0; i < vwc; i++ ) {
00372                     j = wvvec[i] - WILDOFFSET;
00373                     if ( CheckWild(BHEAD j,VECTOVEC,tfixvec[i],&newvalue) )
00374                         goto NoCaseB;
00375                     AddWild(BHEAD j,VECTOVEC,newvalue);
00376                 }
00377                 for ( i = 0; i < iwc; i++ ) {
00378                     j = wcind[i] - WILDOFFSET;
00379                     if ( CheckWild(BHEAD j,INDTOIND,fixvec[i],&newvalue) )
00380                         goto NoCaseB;
00381                     AddWild(BHEAD j,INDTOIND,newvalue);
00382                 }
00383 /*
00384                 Go into the recursion
00385 */
00386                 oRepFunNum = AN.RepFunNum;
00387                 AN.RepFunList[AN.RepFunNum++] = offset;
00388                 AN.RepFunList[AN.RepFunNum++] =
00389                     ( perm1.sign + perm2.sign + distr.sign + sign ) & 1;
00390                 newpat = pattern + pattern[1];
00391                 if ( funwild ) AddWild(BHEAD *pattern-WILDOFFSET,FUNTOFUN,newfun);
00392                 if ( newpat >= AN.patstop ) {
00393                     if ( AN.UseFindOnly == 0 ) {
00394                         if ( FindOnce(BHEAD AN.findTerm,AN.findPattern) ) {
00395                             AN.UsedOtherFind = 1;
00396                             return(1);
00397                         }
00398                     }
00399                     else return(1);
00400                 }
00401                 else {
00402 /*
00403                     newter = instart; */
00404                     if ( ScanFunctions(BHEAD newpat,inter,par) ) { return(1); }
00405 /*
00406                     Restore the old Wildcard assignments
00407 */
00408                     AN.RepFunNum = oRepFunNum;
00409 NoCaseB: m = AN.WildValue;
00410                     t = OldWork; r = AT.WildMask; i = nwstore;
00411                     if ( i > 0 ) {
00412                         do {
00413                             *m++ = *t++; *m++ = *t++; *m++ = *t++; *m++ = *t++; *r++ = *t++;
00414                         } while ( --i > 0 );
00415                     }
00416                     AT.WorkPointer = t;
00417                 }
00418             }
00419         }
00420     }

```

```

00419     }
00420     AT.WorkPointer = OldWork;
00421     return(0);
00422 }
00423
00424 /*
00425 #] MatchE :
00426 #[ Permute :                WORD Permute(perm,first)
00427
00428     Special permutation function.
00429     Works recursively.
00430     The aim is to cycle through in as fast a way as possible,
00431     to take care that each object hits the various positions
00432     already early in the game.
00433
00434     Start at two: -> cycle of two
00435     then three -> cycle of three
00436     etc;
00437     The innermost cycle is the longest. This is the opposite
00438     of the usual way of generating permutations and it is
00439     certainly not the fastest one. It allows for the fastest
00440     hit in the assignment of wildcards though.
00441 */
00442
00443 WORD Permute(PERM *perm, WORD first)
00444 {
00445     WORD *s, c, i, j;
00446     if ( first ) {
00447         perm->sign = ( perm->sign <= 1 ) ? 0: 1;
00448         for ( i = 0; i < perm->n; i++ ) perm->cycle[i] = 0;
00449         return(0);
00450     }
00451     i = perm->n;
00452     while ( --i > 0 ) {
00453         s = perm->objects;
00454         c = s[0];
00455         j = i;
00456         while ( --j >= 0 ) { *s = s[1]; s++; }
00457         *s = c;
00458         if ( ( i & 1 ) != 0 ) perm->sign ^= 1;
00459         if ( perm->cycle[i] < i ) {
00460             (perm->cycle[i])++;
00461             return(0);
00462         }
00463         else {
00464             perm->cycle[i] = 0;
00465         }
00466     }
00467     return(1);
00468 }
00469
00470 /*
00471 #] Permute :
00472 #[ PermuteP :                WORD PermuteP(perm,first)
00473
00474     Like Permute, but works on an array of pointers
00475 */
00476
00477 WORD PermuteP(PERMP *perm, WORD first)
00478 {
00479     WORD **s, *c, i, j;
00480     if ( first ) {
00481         perm->sign = ( perm->sign <= 1 ) ? 0: 1;
00482         for ( i = 0; i < perm->n; i++ ) perm->cycle[i] = 0;
00483         return(0);
00484     }
00485     i = perm->n;
00486     while ( --i > 0 ) {
00487         s = perm->objects;
00488         c = s[0];
00489         j = i;
00490         while ( --j >= 0 ) { *s = s[1]; s++; }
00491         *s = c;
00492         if ( ( i & 1 ) != 0 ) perm->sign ^= 1;
00493         if ( perm->cycle[i] < i ) {
00494             (perm->cycle[i])++;
00495             return(0);
00496         }
00497         else {
00498             perm->cycle[i] = 0;
00499         }
00500     }
00501     return(1);
00502 }
00503
00504 /*
00505 #] PermuteP :

```



```

00506     #[ Distribute :
00507     */
00508
00509 WORD Distribute(DISTRIBUTE *d, WORD first)
00510 {
00511     WORD *to, *from, *inc, *from2, i, j;
00512     if ( first ) {
00513         d->n = d->n1 + d->n2;
00514         to = d->out;
00515         from = d->obj2;
00516         for ( i = 0; i < d->n2; i++ ) {
00517             d->cycle[i] = 0;
00518             *to++ = *from++;
00519         }
00520         from = d->obj1;
00521         while ( i < d->n ) {
00522             d->cycle[i++] = 1;
00523             *to++ = *from++;
00524         }
00525         d->sign = 0;
00526         return(0);
00527     }
00528     if ( d->n1 == 0 || d->n2 == 0 ) return(1);
00529     j = 0;
00530     i = 0;
00531     inc = d->cycle;
00532     from = inc + d->n;
00533     while ( *inc ) { j++; inc++; }
00534     while ( inc < from && !*inc ) { i++; inc++; }
00535     if ( inc >= from ) return(1);
00536     d->sign ^= ((i&j)-j+1) & 1;
00537     *inc = 0;
00538     *--inc = 1;
00539     while ( --j >= 0 ) *--inc = 1;
00540     while ( --i > 0 ) *--inc = 0;
00541     to = d->out;
00542     from = d->obj1;
00543     from2 = d->obj2;
00544     for ( i = 0; i < d->n; i++ ) {
00545         if ( *inc++ ) {
00546             *to++ = *from++;
00547         }
00548         else {
00549             *to++ = *from2++;
00550         }
00551     }
00552     return(0);
00553 }
00554
00555 /*
00556     #[ Distribute :
00557     #[ MatchCy :
00558
00559         Matching of (r)cyclic tensors.
00560         Parameters like in MatchE.
00561         The structure of the routine is much simpler, because the number
00562         of possibilities is much more limited.
00563         The major complication is the ?a-type wildcards
00564         We need a strategy for T(il?,?a,il?,?b). Which is the shorter
00565         match: ?a or ?b ? (if possible of course)
00566         This is also relevant in the case of the shortest match if there
00567         is more than one choice for il.
00568     */
00569
00570 int MatchCy(PHEAD WORD *pattern, WORD *fun, WORD *inter, WORD par)
00571 {
00572     GETBIDENTITY
00573     WORD *t, *tstop, *p, *pstop, *m, *r, *oldworkpointer = AT.WorkPointer;
00574     WORD *thewildcards, *multiplicity, *renum, wc, newvalue, oldwilval = 0;
00575     WORD *params, *lowlevel = 0;
00576     int argcount = 0, funnycount = 0, tcount = fun[1] - FUNHEAD;
00577     int type = 0, pnun, i, j, k, nwstore, iraise, itop, sumeat;
00578     CBUF *C = cbuf+AT.ebufnum;
00579     int ntw = 3*AN.NumTotWildArgs+1;
00580     LONG oldcpointer = C->Pointer - C->Buffer;
00581     WORD offset = fun-AN.terstart, *newpat;
00582
00583     if ( (functions[fun[0]-FUNCTION].symmetric & ~REVERSEORDER) == RCYCLESYMMETRIC ) type = 1;
00584     pnun = pattern[0];
00585     nwstore = (AN.WildValue[-SUBEXPSIZE+1]-SUBEXPSIZE)/4;
00586     if ( pnun > FUNCTION + WILDOFFSET ) {
00587         pnun -= WILDOFFSET;
00588         if ( CheckWild(BHEAD pnun,FUNTOFUN,fun[0],&newvalue) ) return(0);
00589         oldwilval = 1;
00590         t = lowlevel = AT.WorkPointer;
00591         m = AN.WildValue;
00592         i = nwstore;

```

```

00593     r = AT.WildMask;
00594     if ( i > 0 ) {
00595         do {
00596             *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *r++;
00597         } while ( --i > 0 );
00598     }
00599     *t++ = C->numrhs;
00600     if ( t >= AT.WorkTop ) {
00601         MLOCK(ErrorMessageLock);
00602         MesWork();
00603         MUNLOCK(ErrorMessageLock);
00604         return(-1);
00605     }
00606     AT.WorkPointer = t;
00607     AddWild(BHEAD pnun,FUNTOFUN,newvalue);
00608 }
00609 if ( (functions[pnum-FUNCTION].symmetric & ~REVERSEORDER) == RCYCLESYMMETRIC ) type = 1;
00610
00611 /* First we have to make an inventory. Are there FUNNYWILD pointers? */
00612
00613 p = pattern + FUNHEAD;
00614 pstop = pattern + pattern[1];
00615 while ( p < pstop ) {
00616     if ( *p == FUNNYWILD ) { p += 2; funnycount++; }
00617     else { p++; argcount++; }
00618 }
00619 if ( argcount > tcount ) goto NoSuccess;
00620 if ( argcount < tcount && funnycount == 0 ) goto NoSuccess;
00621 if ( argcount == 0 && tcount == 0 && funnycount == 0 ) {
00622     AN.RepFunList[AN.RepFunNum++] = offset;
00623     AN.RepFunList[AN.RepFunNum++] = 0;
00624     newpat = pattern + pattern[1];
00625     if ( newpat >= AN.patstop ) {
00626         if ( AN.UseFindOnly == 0 ) {
00627             if ( FindOnce(BHEAD AN.findTerm,AN.findPattern) ) {
00628                 AT.WorkPointer = oldworkpointer;
00629                 AN.UsedOtherFind = 1;
00630                 return(1);
00631             }
00632             j = 0;
00633         }
00634         else {
00635             AT.WorkPointer = oldworkpointer;
00636             return(1);
00637         }
00638     }
00639     else j = ScanFunctions(BHEAD newpat,inter,par);
00640     if ( j ) return(j);
00641     goto NoSuccess;
00642 }
00643 tstop = fun + fun[1];
00644
00645 /* Store the wildcard assignments */
00646
00647 params = AT.WorkPointer;
00648 thewildcards = t = params + tcount;
00649 t += ntw;
00650 if ( oldwilval ) lowlevel = oldworkpointer;
00651 else lowlevel = t;
00652 m = AN.WildValue;
00653 i = nwstore;
00654 if ( i > 0 ) {
00655     r = AT.WildMask;
00656     do {
00657         *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *r++;
00658     } while ( --i > 0 );
00659     *t++ = C->numrhs;
00660 }
00661 if ( t >= AT.WorkTop ) {
00662     MLOCK(ErrorMessageLock);
00663     MesWork();
00664     MUNLOCK(ErrorMessageLock);
00665     return(-1);
00666 }
00667 AT.WorkPointer = t;
00668 /*
00669 #[ Case 1: no funnies or all funnies must be empty. We just cycle through.
00670 */
00671 if ( argcount == tcount ) {
00672     if ( funnycount > 0 ) { /* Test all funnies first */
00673         p = pattern + FUNHEAD;
00674         t = fun + FUNHEAD;
00675         while ( p < pstop ) {
00676             if ( *p != FUNNYWILD ) { p++; continue; }
00677             AN.argaddress = t;
00678             if ( CheckWild(BHEAD p[1],ARGTOARG,0,t) ) goto nomatch;
00679             AddWild(BHEAD p[1],ARGTOARG,0);

```

```

00680         p += 2;
00681     }
00682     oldwilval = 1;
00683 }
00684 for ( k = 0; k <= type; k++ ) {
00685     if ( k == 0 ) {
00686         p = params; t = fun + FUNHEAD;
00687         while ( t < tstop ) *p++ = *t++;
00688     }
00689     else {
00690         p = params+tcount; t = fun + FUNHEAD;
00691         while ( t < tstop ) *--p = *t++;
00692     }
00693     for ( i = 0; i < tcount; i++ ) { /* The various cycles */
00694         p = pattern + FUNHEAD;
00695         wc = 0;
00696         for ( j = 0; j < tcount; j++, p++ ) { /* The arguments */
00697             while ( *p == FUNNYWILD ) p += 2;
00698             t = params + (i+j)%tcount;
00699             if ( *t == *p ) continue;
00700             if ( *p >= AM.OffsetIndex + WILDOFFSET
00701                 && *p < AM.OffsetIndex + 2*WILDOFFSET ) {
00702
00703                 /* Test wildcard index */
00704
00705                 wc = *p - WILDOFFSET;
00706                 if ( CheckWild(BHEAD wc,INDTOIND,*t,&newvalue) ) break;
00707                 AddWild(BHEAD wc,INDTOIND,newvalue);
00708             }
00709             else if ( *t < MINSPEC && p[j] < MINSPEC
00710                 && *p >= AM.OffsetVector + WILDOFFSET ) {
00711
00712                 /* Test wildcard vector */
00713
00714                 wc = *p - WILDOFFSET;
00715                 if ( CheckWild(BHEAD wc,VECTOVEC,*t,&newvalue) ) break;
00716                 AddWild(BHEAD wc,VECTOVEC,newvalue);
00717             }
00718             else break;
00719         }
00720         if ( j >= tcount ) { /* Match! */
00721
00722             /* Continue with other functions. Make sure of the funnies */
00723
00724             AN.RepFunList[AN.RepFunNum++] = offset;
00725             AN.RepFunList[AN.RepFunNum++] = 0;
00726
00727             if ( funnycount > 0 ) {
00728                 p = pattern + FUNHEAD;
00729                 t = fun + FUNHEAD;
00730                 while ( p < pstop ) {
00731                     if ( *p != FUNNYWILD ) { p++; continue; }
00732                     AN.argaddress = t;
00733                     AddWild(BHEAD p[1],ARGTOARG,0);
00734                     p += 2;
00735                 }
00736             }
00737             newpat = pattern + pattern[1];
00738             if ( newpat >= AN.patstop ) {
00739                 if ( AN.UseFindOnly == 0 ) {
00740                     if ( FindOnce(BHEAD AN.findTerm,AN.findPattern) ) {
00741                         AT.WorkPointer = oldworkpointer;
00742                         AN.UsedOtherFind = 1;
00743                         return(1);
00744                     }
00745                     j = 0;
00746                 }
00747                 else {
00748                     AT.WorkPointer = oldworkpointer;
00749                     return(1);
00750                 }
00751             }
00752             else j = ScanFunctions(BHEAD newpat,inter,par);
00753             if ( j ) {
00754                 AT.WorkPointer = oldworkpointer;
00755                 return(j); /* Full match. Return our success */
00756             }
00757             AN.RepFunNum -= 2;
00758         }
00759     }
00760     /* No (deeper) match. -> reset wildcards and continue */
00761
00762     if ( wc && nwstore > 0 ) {
00763         j = nwstore;
00764         m = AN.WildValue;
00765         t = thewildcards + ntw; r = AT.WildMask;
00766         if ( j > 0 ) {

```

```

00767         do {
00768             *m++ = *t++; *m++ = *t++; *m++ = *t++; *m++ = *t++; *r++ = *t++;
00769         } while ( --j > 0 );
00770     }
00771     C->numrhs = *t++;
00772     C->Pointer = C->Buffer + oldcpointer;
00773 }
00774 }
00775 }
00776 goto NoSuccess;
00777 }
00778 /*
00779 #] Case 1:
00780 #[ Case 2: One FUNNYWILD. Fix its length.
00781 */
00782 if ( funnycount == 1 ) {
00783     funnycount = tcount - argcount; /* Number or arguments to be eaten */
00784     for ( k = 0; k <= type; k++ ) {
00785         if ( k == 0 ) {
00786             p = params; t = fun + FUNHEAD;
00787             while ( t < tstop ) *p++ = *t++;
00788         }
00789         else {
00790             p = params+tcount; t = fun + FUNHEAD;
00791             while ( t < tstop ) *--p = *t++;
00792         }
00793         for ( i = 0; i < tcount; i++ ) { /* The various cycles */
00794             p = pattern + FUNHEAD;
00795             t = params;
00796             wc = 0;
00797             for ( j = 0; j < tcount; j++, p++, t++ ) { /* The arguments */
00798                 if ( *t == *p ) continue;
00799                 if ( *p == FUNNYWILD ) {
00800                     p++; wc = 1;
00801                     AN.argaddress = t;
00802                     if ( CheckWild(BHEAD *p,ARGTOARG|EATTENSOR,funnycount,t) ) break;
00803                     AddWild(BHEAD *p,ARGTOARG|EATTENSOR,funnycount);
00804                     j += funnycount-1; t += funnycount-1;
00805                 }
00806                 else if ( *p >= AM.OffsetIndex + WILDOFFSET
00807                     && *p < AM.OffsetIndex + 2*WILDOFFSET ) {
00808
00809                     /* Test wildcard index */
00810
00811                     wc = *p - WILDOFFSET;
00812                     if ( CheckWild(BHEAD wc,INDTOIND,*t,&newvalue) ) break;
00813                     AddWild(BHEAD wc,INDTOIND,newvalue);
00814                 }
00815                 else if ( *t < MINSPEC && *p < MINSPEC
00816                     && *p >= AM.OffsetVector + WILDOFFSET ) {
00817
00818                     /* Test wildcard vector */
00819
00820                     wc = *p - WILDOFFSET;
00821                     if ( CheckWild(BHEAD wc,VECTOVEC,*t,&newvalue) ) break;
00822                     AddWild(BHEAD wc,VECTOVEC,newvalue);
00823                 }
00824                 else break;
00825             }
00826             if ( j >= tcount ) { /* Match! */
00827
00828                 /* Continue with other functions. Make sure of the funnies */
00829
00830                 AN.RepFunList[AN.RepFunNum++] = offset;
00831                 AN.RepFunList[AN.RepFunNum++] = 0;
00832                 newpat = pattern + pattern[1];
00833                 if ( newpat >= AN.patstop ) {
00834                     if ( AN.UseFindOnly == 0 ) {
00835                         if ( FindOnce(BHEAD AN.findTerm,AN.findPattern) ) {
00836                             AT.WorkPointer = oldworkpointer;
00837                             AN.UsedOtherFind = 1;
00838                             return(1);
00839                         }
00840                         j = 0;
00841                     }
00842                     else {
00843                         AT.WorkPointer = oldworkpointer;
00844                         return(1);
00845                     }
00846                 }
00847                 else j = ScanFunctions(BHEAD newpat,inter,par);
00848                 if ( j ) {
00849                     AT.WorkPointer = oldworkpointer;
00850                     return(j); /* Full match. Return our success */
00851                 }
00852                 AN.RepFunNum -= 2;
00853             }

```

```

00854
00855         /* No (deeper) match. -> reset wildcards and continue */
00856
00857         if ( wc ) {
00858             j = nwstore;
00859             m = AN.WildValue;
00860             t = thewildcards + ntw; r = AT.WildMask;
00861             if ( j > 0 ) {
00862                 do {
00863                     *m++ = *t++; *m++ = *t++; *m++ = *t++; *m++ = *t++; *r++ = *t++;
00864                 } while ( --j > 0 );
00865             }
00866             C->numrhs = *t++;
00867             C->Pointer = C->Buffer + oldcpointer;
00868         }
00869         t = params;
00870         wc = *t;
00871         for ( j = 1; j < tcount; j++ ) { *t = t[1]; t++; }
00872         *t = wc;
00873     }
00874 }
00875 goto NoSuccess;
00876 }
00877 /*
00878 #] Case 2:
00879 #[ Case 3: More than one FUNNYWILD. Complicated.
00880 */
00881
00882 sumeat = tcount - argcount; /* Total number to be eaten by Funnies */
00883 /*
00884 In the first funnycount elements of 'thewildcards' we arrange
00885 for the summing over the various possibilities.
00886 The renumbering table is in thewildcards[2*funnycount]
00887 The multiplicity table is in thewildcards[funnycount]
00888 The number of arguments for each is in thewildcards[]
00889 */
00890 p = pattern+FUNHEAD;
00891 for ( i = funnycount; i < ntw; i++ ) thewildcards[i] = -1;
00892 multiplicity = thewildcards + funnycount;
00893 renum = multiplicity + funnycount;
00894 j = 0;
00895 while ( p < pstop ) {
00896     if ( *p != FUNNYWILD ) { p++; continue; }
00897     p++;
00898     if ( renum[*p] < 0 ) {
00899         renum[*p] = j;
00900         multiplicity[j] = 1;
00901         j++;
00902     }
00903     else multiplicity[renum[*p]]++;
00904     p++;
00905 }
00906 /*
00907 Strategy: First 'declared' has a tendency to be smaller
00908 */
00909 for ( i = 1; i < AN.NumTotWildArgs; i++ ) {
00910     if ( renum[i] < 0 ) continue;
00911     for ( j = i+1; j <= AN.NumTotWildArgs; j++ ) {
00912         if ( renum[j] < 0 ) continue;
00913         if ( renum[i] < renum[j] ) continue;
00914         k = multiplicity[renum[i]];
00915         multiplicity[renum[i]] = multiplicity[renum[j]];
00916         multiplicity[renum[j]] = k;
00917         k = renum[i]; renum[i] = renum[j]; renum[j] = k;
00918     }
00919 }
00920 for ( i = 0; i < funnycount; i++ ) thewildcards[i] = 0;
00921 iraise = funnycount-1;
00922 for ( ;; ) {
00923     for ( i = 0, j = sumeat; i < iraise; i++ )
00924         j -= thewildcards[i]*multiplicity[i];
00925     if ( j < 0 || j % multiplicity[iraise] != 0 ) {
00926         if ( j > 0 ) {
00927             thewildcards[iraise-1]++;
00928             continue;
00929         }
00930         itop = iraise-1;
00931         while ( itop > 0 && j < 0 ) {
00932             j += thewildcards[itop]*multiplicity[itop];
00933             thewildcards[itop] = 0;
00934             itop--;
00935         }
00936         if ( itop <= 0 && j <= 0 ) break;
00937         thewildcards[itop]++;
00938         continue;
00939     }
00940     thewildcards[iraise] = j / multiplicity[iraise];

```

```

00941
00942     for ( k = 0; k <= type; k++ ) {
00943         if ( k == 0 ) {
00944             p = params; t = fun + FUNHEAD;
00945             while ( t < tstop ) *p++ = *t++;
00946         }
00947         else {
00948             p = params+tcount; t = fun + FUNHEAD;
00949             while ( t < tstop ) *--p = *t++;
00950         }
00951     }
00952     for ( i = 0; i < tcount; i++ ) { /* The various cycles */
00953         p = pattern + FUNHEAD;
00954         t = params;
00955         wc = 0;
00956         for ( j = 0; j < tcount; j++, p++, t++ ) { /* The arguments */
00957             if ( *t == *p ) continue;
00958             if ( *p == FUNNYWILD ) {
00959                 p++; wc = thewildcards[renum[*p]];
00960                 AN.argaddress = t;
00961                 if ( CheckWild(BHEAD *p,ARGTOARG|EATTENSOR,wc,t) ) break;
00962                 AddWild(BHEAD *p,ARGTOARG|EATTENSOR,wc);
00963                 j += wc-1; t += wc-1; wc = 1;
00964             }
00965             else if ( *p >= AM.OffsetIndex + WILDOFFSET
00966                 && *p < AM.OffsetIndex + 2*WILDOFFSET ) {
00967
00968                 /* Test wildcard index */
00969
00970                 wc = *p - WILDOFFSET;
00971                 if ( CheckWild(BHEAD wc,INDTOIND,*t,&newvalue) ) break;
00972                 AddWild(BHEAD wc,INDTOIND,newvalue);
00973             }
00974             else if ( *t < MINSPEC && *p < MINSPEC
00975                 && *p >= AM.OffsetVector + WILDOFFSET ) {
00976
00977                 /* Test wildcard vector */
00978
00979                 wc = *p - WILDOFFSET;
00980                 if ( CheckWild(BHEAD wc,VECTOVEC,*t,&newvalue) ) break;
00981                 AddWild(BHEAD wc,VECTOVEC,newvalue);
00982             }
00983             else break;
00984         }
00985         if ( j >= tcount ) { /* Match! */
00986
00987             /* Continue with other functions. Make sure of the funnies */
00988
00989             AN.RepFunList[AN.RepFunNum++] = offset;
00990             AN.RepFunList[AN.RepFunNum++] = 0;
00991             newpat = pattern + pattern[1];
00992             if ( newpat >= AN.patstop ) {
00993                 if ( AN.UseFindOnly == 0 ) {
00994                     if ( FindOnce(BHEAD AN.findTerm,AN.findPattern) ) {
00995                         AT.WorkPointer = oldworkpointer;
00996                         AN.UsedOtherFind = 1;
00997                         return(1);
00998                     }
00999                     j = 0;
01000                 }
01001                 else {
01002                     AT.WorkPointer = oldworkpointer;
01003                     return(1);
01004                 }
01005             }
01006             else j = ScanFunctions(BHEAD newpat,inter,par);
01007             if ( j ) {
01008                 AT.WorkPointer = oldworkpointer;
01009                 return(j); /* Full match. Return our success */
01010             }
01011             AN.RepFunNum -= 2;
01012         }
01013     }
01014
01015     /* No (deeper) match. -> reset wildcards and continue */
01016
01017     if ( wc ) {
01018         j = nwstore;
01019         m = AN.WildValue;
01020         t = thewildcards + ntwa; r = AT.WildMask;
01021         if ( j > 0 ) {
01022             do {
01023                 *m++ = *t++; *m++ = *t++; *m++ = *t++; *m++ = *t++; *r++ = *t++;
01024             } while ( --j > 0 );
01025         }
01026         C->numrhs = *t++;
01027         C->Pointer = C->Buffer + oldcpointer;
01028     }
01029     t = params;

```

```

01028         wc = *t;
01029         for ( j = 1; j < tcount; j++ ) { *t = t[1]; t++; }
01030         *t = wc;
01031     }
01032 }
01033 (thewildcards[iraise-1])++;
01034 }
01035 /*
01036 #] Case 3:
01037 */
01038 NoSuccess:
01039     if ( oldwilval > 0 ) {
01040 nomatch:;
01041         j = nwstore;
01042         if ( j > 0 ) {
01043             m = AN.WildValue;
01044             t = lowlevel; r = AT.WildMask;
01045             if ( j > 0 ) {
01046                 do {
01047                     *m++ = *t++; *m++ = *t++; *m++ = *t++; *m++ = *t++; *r++ = *t++;
01048                 } while ( --j > 0 );
01049             }
01050             C->numrhs = *t++;
01051             C->Pointer = C->Buffer + oldcpointer;
01052         }
01053     }
01054     AT.WorkPointer = oldworkpointer;
01055     return(0);
01056 }
01057
01058 /*
01059 #] MatchCy :
01060 #[ FunMatchCy :
01061
01062     Matching of (r)cyclic functions.
01063     Like MatchCy, but now for general functions.
01064 */
01065
01066 int FunMatchCy(PHEAD WORD *pattern, WORD *fun, WORD *inter, WORD par)
01067 {
01068     GETBIDENTITY
01069     WORD *t, *tstop, *p, *pstop, *m, *r, *oldworkpointer = AT.WorkPointer;
01070     WORD **a, *thewildcards, *multiplicity, *renum, wc, wcc, oldwilval = 0;
01071     LONG ow = AT.pWorkPointer;
01072     WORD newvalue, *lowlevel = 0;
01073     int argcount = 0, funnycount = 0, tcount = 0;
01074     int type = 0, pnum, i, j, k, nwstore, iraise, itop, sumeat;
01075     CBUF *C = cbuf+AT.ebufnum;
01076     int ntw = 3*AN.NumTotWildArgs+1;
01077     LONG oldcpointer = C->Pointer - C->Buffer;
01078     WORD offset = fun-AN.terstart, *newpat;
01079
01080     if ( (functions[fun[0]-FUNCTION].symmetric & ~REVERSEORDER) == RCYCLESYMMETRIC ) type = 1;
01081     pnum = pattern[0];
01082     nwstore = (AN.WildValue[-SUBEXPSIZE+1]-SUBEXPSIZE)/4;
01083     if ( pnum > FUNCTION + WILDOFFSET ) {
01084         pnum -= WILDOFFSET;
01085         if ( CheckWild(BHEAD pnum,FUNTOFUN,fun[0],&newvalue) ) return(0);
01086         oldwilval = 1;
01087         t = lowlevel = oldworkpointer;
01088         m = AN.WildValue;
01089         i = nwstore;
01090         r = AT.WildMask;
01091         if ( i > 0 ) {
01092             do {
01093                 *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *r++;
01094             } while ( --i > 0 );
01095         }
01096         *t++ = C->numrhs;
01097         if ( t >= AT.WorkTop ) {
01098             MLOCK(ErrorMessageLock);
01099             MesWork();
01100             MUNLOCK(ErrorMessageLock);
01101             return(-1);
01102         }
01103         AT.WorkPointer = t;
01104         AddWild(BHEAD pnum,FUNTOFUN,newvalue);
01105     }
01106     if ( (functions[pnum-FUNCTION].symmetric & ~REVERSEORDER) == RCYCLESYMMETRIC ) type = 1;
01107
01108     /* First we have to make an inventory. Are there -ARGWILD pointers? */
01109
01110     p = pattern + FUNHEAD;
01111     pstop = pattern + pattern[1];
01112     while ( p < pstop ) {
01113         if ( *p == -ARGWILD ) { p += 2; funnycount++; }
01114         else { NEXTARG(p); argcount++; }

```

```

01115     }
01116     t = fun + FUNHEAD;
01117     tstop = fun + fun[1];
01118     while ( t < tstop ) { NEXTARG(t); tcount++; }
01119
01120     if ( argcount > tcount ) return(0);
01121     if ( argcount < tcount && funnycount == 0 ) return(0);
01122     if ( argcount == 0 && tcount == 0 && funnycount == 0 ) {
01123         AN.RepFunList[AN.RepFunNum++] = offset;
01124         AN.RepFunList[AN.RepFunNum++] = 0;
01125         newpat = pattern + pattern[1];
01126         if ( newpat >= AN.patstop ) {
01127             if ( AN.UseFindOnly == 0 ) {
01128                 if ( FindOnce(BHEAD AN.findTerm,AN.findPattern) ) {
01129                     AT.WorkPointer = oldworkpointer;
01130                     AN.UsedOtherFind = 1;
01131                     return(1);
01132                 }
01133                 j = 0;
01134             }
01135             else {
01136                 AT.WorkPointer = oldworkpointer;
01137                 return(1);
01138             }
01139         }
01140         else j = ScanFunctions(BHEAD newpat,inter,par);
01141         if ( j ) return(j);
01142         goto NoSuccess;
01143     }
01144
01145     /* Store the wildcard assignments */
01146
01147     WantAddPointers(tcount);
01148     AT.pWorkPointer += tcount;
01149     thewildcards = t = AT.WorkPointer;
01150     t += ntwa;
01151     if ( oldwilval ) lowlevel = oldworkpointer;
01152     else lowlevel = t;
01153     m = AN.WildValue;
01154     i = nwstore;
01155     if ( i > 0 ) {
01156         r = AT.WildMask;
01157         do {
01158             *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *r++;
01159         } while ( --i > 0 );
01160         *t++ = C->numrhs;
01161     }
01162     if ( t >= AT.WorkTop ) {
01163         MLOCK(ErrorMessageLock);
01164         MesWork();
01165         MUNLOCK(ErrorMessageLock);
01166         return(-1);
01167     }
01168     AT.WorkPointer = t;
01169     /*
01170     #[ Case 1: no funnies or all funnies must be empty. We just cycle through.
01171     */
01172     if ( argcount == tcount ) {
01173         if ( funnycount > 0 ) { /* Test all funnies first */
01174             p = pattern + FUNHEAD;
01175             t = fun + FUNHEAD;
01176             while ( p < pstop ) {
01177                 if ( *p != -ARGWILD ) { p++; continue; }
01178                 AN.argaddress = t;
01179                 if ( CheckWild(BHEAD p[1],ARGTOARG,0,t) ) goto nomatch;
01180                 AddWild(BHEAD p[1],ARGTOARG,0);
01181                 p += 2;
01182             }
01183             oldwilval = 1;
01184         }
01185         for ( k = 0; k <= type; k++ ) {
01186             if ( k == 0 ) {
01187                 a = AT.pWorkSpace+oww; t = fun + FUNHEAD;
01188                 while ( t < tstop ) { *a++ = t; NEXTARG(t); }
01189             }
01190             else {
01191                 a = AT.pWorkSpace+oww+tcount; t = fun + FUNHEAD;
01192                 while ( t < tstop ) { *--a = t; NEXTARG(t); }
01193             }
01194             for ( i = 0; i < tcount; i++ ) { /* The various cycles */
01195                 p = pattern + FUNHEAD;
01196                 wc = 0;
01197                 for ( j = 0; j < tcount; j++ ) { /* The arguments */
01198                     while ( *p == -ARGWILD ) p += 2;
01199                     t = AT.pWorkSpace[oww+((i+j)%tcount)];
01200                     if ( ( wcc = MatchArgument(BHEAD t,p) ) == 0 ) break;
01201                     if ( wcc > 1 ) wc = 1;

```



```

01202         NEXTARG(p);
01203     }
01204     if ( j >= tcount ) { /* Match! */
01205
01206         /* Continue with other functions. Make sure of the funnies */
01207
01208         AN.RepFunList[AN.RepFunNum++] = offset;
01209         AN.RepFunList[AN.RepFunNum++] = 0;
01210
01211         if ( funnycount > 0 ) {
01212             p = pattern + FUNHEAD;
01213             t = fun + FUNHEAD;
01214             while ( p < pstop ) {
01215                 if ( *p != -ARGWILD ) { p++; continue; }
01216                 AN.argaddress = t;
01217                 AddWild(BHEAD p[1],ARGTOARG,0);
01218                 p += 2;
01219             }
01220         }
01221         newpat = pattern + pattern[1];
01222         if ( newpat >= AN.patstop ) {
01223             if ( AN.UseFindOnly == 0 ) {
01224                 if ( FindOnce(BHEAD AN.findTerm,AN.findPattern) ) {
01225                     AT.WorkPointer = oldworkpointer;
01226                     AT.pWorkPointer = oww;
01227                     AN.UsedOtherFind = 1;
01228                     return(1);
01229                 }
01230                 j = 0;
01231             }
01232             else {
01233                 AT.WorkPointer = oldworkpointer;
01234                 AT.pWorkPointer = oww;
01235                 return(1);
01236             }
01237         }
01238         else j = ScanFunctions(BHEAD newpat,inter,par);
01239         if ( j ) {
01240             AT.WorkPointer = oldworkpointer;
01241             AT.pWorkPointer = oww;
01242             return(j); /* Full match. Return our success */
01243         }
01244         AN.RepFunNum -= 2;
01245     }
01246
01247     /* No (deeper) match. -> reset wildcards and continue */
01248
01249     if ( wc && nwstore > 0 ) {
01250         j = nwstore;
01251         m = AN.WildValue;
01252         t = thewildcards + ntwa; r = AT.WildMask;
01253         if ( j > 0 ) {
01254             do {
01255                 *m++ = *t++; *m++ = *t++; *m++ = *t++; *m++ = *t++; *r++ = *t++;
01256             } while ( --j > 0 );
01257         }
01258         C->numrhs = *t++;
01259         C->Pointer = C->Buffer + oldcpointer;
01260     }
01261 }
01262 }
01263     goto NoSuccess;
01264 }
01265 /*
01266 #] Case 1:
01267 #[ Case 2: One -ARGWILD. Fix its length.
01268 */
01269 if ( funnycount == 1 ) {
01270     funnycount = tcount - argcount; /* Number or arguments to be eaten */
01271     for ( k = 0; k <= type; k++ ) {
01272         if ( k == 0 ) {
01273             a = AT.pWorkSpace+oww; t = fun + FUNHEAD;
01274             while ( t < tstop ) { *a++ = t; NEXTARG(t); }
01275         }
01276         else {
01277             a = AT.pWorkSpace+oww+tcount; t = fun + FUNHEAD;
01278             while ( t < tstop ) { *--a = t; NEXTARG(t); }
01279         }
01280     }
01281     for ( i = 0; i < tcount; i++ ) { /* The various cycles */
01282         p = pattern + FUNHEAD;
01283         a = AT.pWorkSpace+oww;
01284         wc = 0;
01285         for ( j = 0; j < tcount; j++, a++ ) { /* The arguments */
01286             t = *a;
01287             if ( *p == -ARGWILD ) {
01288                 wc = 1;
01289                 AN.argaddress = (WORD *)a;

```

```

01289         if ( CheckWild(BHEAD p[1],ARLTOARL,funnycount,(WORD *)a) ) break;
01290         AddWild(BHEAD p[1],ARLTOARL,funnycount);
01291         j += funnycount-1; a += funnycount-1;
01292     }
01293     else if ( MatchArgument(BHEAD t,p) == 0 ) break;
01294     NEXTARG(p);
01295 }
01296 if ( j >= tcount ) { /* Match! */
01297
01298     /* Continue with other functions. Make sure of the funnies */
01299
01300     AN.RepFunList[AN.RepFunNum++] = offset;
01301     AN.RepFunList[AN.RepFunNum++] = 0;
01302     newpat = pattern + pattern[1];
01303     if ( newpat >= AN.patstop ) {
01304         if ( AN.UseFindOnly == 0 ) {
01305             if ( FindOnce(BHEAD AN.findTerm,AN.findPattern) ) {
01306                 AT.WorkPointer = oldworkpointer;
01307                 AT.pWorkPointer = ow;
01308                 AN.UsedOtherFind = 1;
01309                 return(1);
01310             }
01311             j = 0;
01312         }
01313         else {
01314             AT.WorkPointer = oldworkpointer;
01315             AT.pWorkPointer = ow;
01316             return(1);
01317         }
01318     }
01319     else j = ScanFunctions(BHEAD newpat,inter,par);
01320     if ( j ) {
01321         AT.WorkPointer = oldworkpointer;
01322         AT.pWorkPointer = ow;
01323         return(j); /* Full match. Return our success */
01324     }
01325     AN.RepFunNum -= 2;
01326 }
01327
01328 /* No (deeper) match. -> reset wildcards and continue */
01329
01330 if ( wc ) {
01331     j = nwstore;
01332     m = AN.WildValue;
01333     t = thewildcards + ntw; r = AT.WildMask;
01334     if ( j > 0 ) {
01335         do {
01336             *m++ = *t++; *m++ = *t++; *m++ = *t++; *m++ = *t++; *r++ = *t++;
01337         } while ( --j > 0 );
01338     }
01339     C->numrhs = *t++;
01340     C->Pointer = C->Buffer + oldcpointer;
01341 }
01342 a = AT.pWorkSpace+ow;
01343 t = *a;
01344 for ( j = 1; j < tcount; j++ ) { *a = a[1]; a++; }
01345 *a = t;
01346 }
01347 }
01348 goto NoSuccess;
01349 }
01350 /*
01351 #] Case 2:
01352 #[ Case 3: More than one -ARGWILD. Complicated.
01353 */
01354
01355 sumeat = tcount - argcount; /* Total number to be eaten by Funnies */
01356 /*
01357 In the first funnycount elements of 'thewildcards' we arrange
01358 for the summing over the various possibilities.
01359 The renumbering table is in thewildcards[2*funnycount]
01360 The multiplicity table is in thewildcards[funnycount]
01361 The number of arguments for each is in thewildcards[]
01362 */
01363 p = pattern+FUNHEAD;
01364 for ( i = funnycount; i < ntw; i++ ) thewildcards[i] = -1;
01365 multiplicity = thewildcards + funnycount;
01366 renum = multiplicity + funnycount;
01367 j = 0;
01368 while ( p < pstop ) {
01369     if ( *p != -ARGWILD ) { p++; continue; }
01370     p++;
01371     if ( renum[*p] < 0 ) {
01372         renum[*p] = j;
01373         multiplicity[j] = 1;
01374         j++;
01375     }

```

```

01376         else multiplicity[renum[*p]]++;
01377         p++;
01378     }
01379     /*
01380     Strategy: First 'declared' has a tendency to be smaller
01381     */
01382     for ( i = 1; i < AN.NumTotWildArgs; i++ ) {
01383         if ( renum[i] < 0 ) continue;
01384         for ( j = i+1; j <= AN.NumTotWildArgs; j++ ) {
01385             if ( renum[j] < 0 ) continue;
01386             if ( renum[i] < renum[j] ) continue;
01387             k = multiplicity[renum[i]];
01388             multiplicity[renum[i]] = multiplicity[renum[j]];
01389             multiplicity[renum[j]] = k;
01390             k = renum[i]; renum[i] = renum[j]; renum[j] = k;
01391         }
01392     }
01393     for ( i = 0; i < funnycount; i++ ) thewildcards[i] = 0;
01394     iraise = funnycount-1;
01395     for ( ;; ) {
01396         for ( i = 0, j = sumeat; i < iraise; i++ )
01397             j -= thewildcards[i]*multiplicity[i];
01398         if ( j < 0 || j % multiplicity[iraise] != 0 ) {
01399             if ( j > 0 ) {
01400                 thewildcards[iraise-1]++;
01401                 continue;
01402             }
01403             itop = iraise-1;
01404             while ( itop > 0 && j < 0 ) {
01405                 j += thewildcards[itop]*multiplicity[itop];
01406                 thewildcards[itop] = 0;
01407                 itop--;
01408             }
01409             if ( itop <= 0 && j <= 0 ) break;
01410             thewildcards[itop]++;
01411             continue;
01412         }
01413         thewildcards[iraise] = j / multiplicity[iraise];
01414     }
01415     for ( k = 0; k <= type; k++ ) {
01416         if ( k == 0 ) {
01417             a = AT.pWorkSpace+oww; t = fun + FUNHEAD;
01418             while ( t < tstop ) { *a++ = t; NEXTARG(t); }
01419         }
01420         else {
01421             a = AT.pWorkSpace+oww+tcount; t = fun + FUNHEAD;
01422             while ( t < tstop ) { *--a = t; NEXTARG(t); }
01423         }
01424         for ( i = 0; i < tcount; i++ ) { /* The various cycles */
01425             p = pattern + FUNHEAD;
01426             a = AT.pWorkSpace+oww;
01427             wc = 0;
01428             for ( j = 0; j < tcount; j++, a++ ) { /* The arguments */
01429                 t = *a;
01430                 if ( *p == -ARGWILD ) {
01431                     wc = thewildcards[renum[p[1]]];
01432                     AN.argaddress = (WORD *)a;
01433                     if ( CheckWild(BHEAD p[1],ARLTOARL,wc,(WORD *)a) ) break;
01434                     AddWild(BHEAD p[1],ARLTOARL,wc);
01435                     j += wc-1; a += wc-1; wc = 1;
01436                 }
01437                 else if ( MatchArgument(BHEAD t,p) == 0 ) break;
01438                 NEXTARG(p);
01439             }
01440             if ( j >= tcount ) { /* Match! */
01441
01442                 /* Continue with other functions. Make sure of the funnies */
01443
01444                 AN.RepFunList[AN.RepFunNum++] = offset;
01445                 AN.RepFunList[AN.RepFunNum++] = 0;
01446                 newpat = pattern + pattern[1];
01447                 if ( newpat >= AN.patstop ) {
01448                     if ( AN.UseFindOnly == 0 ) {
01449                         if ( FindOnce(BHEAD AN.findTerm,AN.findPattern) ) {
01450                             AT.WorkPointer = oldworkpointer;
01451                             AT.pWorkPointer = oww;
01452                             AN.UsedOtherFind = 1;
01453                             return(1);
01454                         }
01455                         j = 0;
01456                     }
01457                     else {
01458                         AT.WorkPointer = oldworkpointer;
01459                         AT.pWorkPointer = oww;
01460                         return(1);
01461                     }
01462                 }

```

```

01463         else j = ScanFunctions(BHEAD newpat,inter,par);
01464         if ( j ) {
01465             AT.WorkPointer = oldworkpointer;
01466             AT.pWorkPointer = oww;
01467             return(j); /* Full match. Return our success */
01468         }
01469         AN.RepFunNum -= 2;
01470     }
01471
01472     /* No (deeper) match. -> reset wildcards and continue */
01473
01474     if ( wc ) {
01475         j = nwstore;
01476         m = AN.WildValue;
01477         t = thewildcards + ntw; r = AT.WildMask;
01478         if ( j > 0 ) {
01479             do {
01480                 *m++ = *t++; *m++ = *t++; *m++ = *t++; *m++ = *t++; *r++ = *t++;
01481             } while ( --j > 0 );
01482         }
01483         C->numrhs = *t++;
01484         C->Pointer = C->Buffer + oldcpointer;
01485     }
01486     a = AT.pWorkSpace+oww;
01487     t = *a;
01488     for ( j = 1; j < tcount; j++ ) { *a = a[1]; a++; }
01489     *a = t;
01490 }
01491 }
01492 (thewildcards[iraise-1])++;
01493 }
01494 /*
01495 #] Case 3:
01496 */
01497 NoSuccess:
01498     if ( oldwilval > 0 ) {
01499 nomatch:;
01500         j = nwstore;
01501         m = AN.WildValue;
01502         t = lowlevel; r = AT.WildMask;
01503         if ( j > 0 ) {
01504             do {
01505                 *m++ = *t++; *m++ = *t++; *m++ = *t++; *m++ = *t++; *r++ = *t++;
01506             } while ( --j > 0 );
01507         }
01508         C->numrhs = *t++;
01509         C->Pointer = C->Buffer + oldcpointer;
01510     }
01511     AT.WorkPointer = oldworkpointer;
01512     AT.pWorkPointer = oww;
01513     return(0);
01514 }
01515
01516 /*
01517 #] FunMatchCy :
01518 #[ FunMatchSy :
01519
01520     Matching of (anti)symmetric functions.
01521     Like MatchE, but now for general functions.
01522 */
01523
01524 int FunMatchSy(PHEAD WORD *pattern, WORD *fun, WORD *inter, WORD par)
01525 {
01526     GETBIDENTITY
01527     WORD *t, *tstop, *p, *pstop, *m, *r, *oldworkpointer = AT.WorkPointer;
01528     WORD **a, *thewildcards, oldwilval = 0;
01529     WORD newvalue, *lowlevel = 0, num, assig;
01530     WORD *cycles;
01531     LONG oww = AT.pWorkPointer, lhpar, lhfunnies;
01532     int argcount = 0, funnycount = 0, tcount = 0, signs = 0, signfun = 0, signo;
01533     int type = 0, pnun, i, j, k, nwstore, iraise, cou2;
01534     CBUF *C = cbuf+AT.ebufnum;
01535     int ntw = 3*AN.NumTotWildArgs+1;
01536     LONG oldcpointer = C->Pointer - C->Buffer;
01537     WORD offset = fun-AN.terstart, *newpat;
01538
01539     if ( (functions[fun[0]-FUNCTION].symmetric & ~REVERSEORDER) == RCYCLESYMMETRIC ) type = 1;
01540     pnun = pattern[0];
01541     nwstore = (AN.WildValue[-SUBEXPSIZE+1]-SUBEXPSIZE)/4;
01542     if ( pnun > FUNCTION + WILDOFFSET ) {
01543         pnun -= WILDOFFSET;
01544         if ( CheckWild(BHEAD pnun,FUNTOFUN,fun[0],&newvalue) ) return(0);
01545         oldwilval = 1;
01546         t = lowlevel = oldworkpointer;
01547         m = AN.WildValue;
01548         i = nwstore;
01549         r = AT.WildMask;

```

```

01550         if ( i > 0 ) {
01551             do {
01552                 *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *r++;
01553             } while ( --i > 0 );
01554         }
01555         *t++ = C->numrhs;
01556         if ( t >= AT.WorkTop ) {
01557             MLOCK(ErrorMessageLock);
01558             MesWork();
01559             MUNLOCK(ErrorMessageLock);
01560             return(-1);
01561         }
01562         AT.WorkPointer = t;
01563         AddWild(BHEAD pnun,FUNTOFUN,newvalue);
01564     }
01565     if ( (functions[pnum-FUNCTION].symmetric & ~REVERSEORDER) == RCYCLESYMMETRIC ) type = 1;
01566
01567     /* Try for a straight match. After all, both have been normalized */
01568
01569     if ( fun[l] == pattern[l] ) {
01570         i = fun[l]-FUNHEAD; p = pattern+FUNHEAD; t = fun + FUNHEAD;
01571         while ( --i >= 0 ) { if ( *p++ != *t++ ) break; }
01572         if ( i < 0 ) goto quicky;
01573     }
01574
01575     /* First we have to make an inventory. Are there -ARGWILD pointers? */
01576
01577     p = pattern + FUNHEAD;
01578     pstop = pattern + pattern[l];
01579     while ( p < pstop ) {
01580         if ( *p == -ARGWILD ) { p += 2; funnycount++; }
01581         else { NEXTARG(p); argcount++; }
01582     }
01583     t = fun + FUNHEAD;
01584     tstop = fun + fun[l];
01585     while ( t < tstop ) { NEXTARG(t); tcount++; }
01586
01587     if ( argcount > tcount ) return(0);
01588     if ( argcount < tcount && funnycount == 0 ) return(0);
01589     if ( argcount == 0 && tcount == 0 && funnycount == 0 ) {
01590 quicky:
01591         if ( AN.SignCheck && signs != AN.ExpectedSign ) goto NoSuccess;
01592         AN.RepFunList[AN.RepFunNum++] = offset;
01593         AN.RepFunList[AN.RepFunNum++] = signs;
01594         newpat = pattern + pattern[l];
01595         if ( newpat >= AN.patstop ) {
01596             if ( AN.UseFindOnly == 0 ) {
01597                 if ( FindOnce(BHEAD AN.findTerm,AN.findPattern) ) {
01598                     AT.WorkPointer = oldworkpointer;
01599                     AN.UsedOtherFind = 1;
01600                     return(1);
01601                 }
01602                 j = 0;
01603             }
01604             else {
01605                 AT.WorkPointer = oldworkpointer;
01606                 return(1);
01607             }
01608         }
01609         else j = ScanFunctions(BHEAD newpat,inter,par);
01610         if ( j ) {
01611             AT.WorkPointer = oldworkpointer;
01612             return(j);
01613         }
01614         goto NoSuccess;
01615     }
01616
01617     /* Store the wildcard assignments */
01618
01619     WantAddPointers(tcount+argcount+funnycount);
01620     AT.pWorkPointer += tcount+argcount+funnycount;
01621     thewildcards = t = AT.WorkPointer;
01622     t += ntw;
01623     if ( oldwilval ) lowlevel = oldworkpointer;
01624     else lowlevel = t;
01625     m = AN.WildValue;
01626     i = nwstore; assig = 0;
01627     if ( i > 0 ) {
01628         r = AT.WildMask;
01629         do {
01630             assig += *r;
01631             *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *m++; *t++ = *r++;
01632         } while ( --i > 0 );
01633         *t++ = C->numrhs;
01634     }
01635     if ( t >= AT.WorkTop ) {
01636         MLOCK(ErrorMessageLock);

```

```

01637         MesWork();
01638         MUNLOCK(ErrorMessageLock);
01639         return(-1);
01640     }
01641     AT.WorkPointer = t;
01642
01643     /* Store pointers to the arguments */
01644
01645     t = fun + FUNHEAD; a = AT.pWorkSpace+oww;
01646     while ( t < tstop ) { *a++ = t; NEXTARG(t) }
01647     lhpars = a-AT.pWorkSpace;
01648     t = pattern + FUNHEAD;
01649     while ( t < pstop ) {
01650         if ( *t != -ARGWILD ) *a++ = t;
01651         NEXTARG(t)
01652     }
01653     lhfunnies = a-AT.pWorkSpace;
01654     t = pattern + FUNHEAD; cou2 = 0;
01655     while ( t < pstop ) {
01656         cou2++;
01657         if ( *t == -ARGWILD ) {
01658             *a++ = t;
01659         }
01660     }
01661     signfun: last ?a: tcount-argcount: number of arguments in ?a (assume one ?a)
01662     argcount+funnycount-cou2: arguments after ?a.
01663     Together tells whether moving ?a to end of list is even or odd
01664     */
01665     signfun = ((argcount+funnycount-cou2)*(tcount-argcount)) & 1;
01666     NEXTARG(t)
01667 }
01668 signs += signfun;
01669 if ( funnycount > 0 ) {
01670     if ( ( (functions[fun[0]-FUNCTION].symmetric & ~REVERSEORDER) == SYMMETRIC )
01671         || ( (functions[fun[0]-FUNCTION].symmetric & ~REVERSEORDER) == ANTISYMMETRIC )
01672         || ( (functions[pnum-FUNCTION].symmetric & ~REVERSEORDER) == SYMMETRIC )
01673         || ( (functions[pnum-FUNCTION].symmetric & ~REVERSEORDER) == ANTISYMMETRIC ) ) {
01674         AT.WorkPointer = oldworkpointer;
01675         AT.pWorkPointer = oww;
01676         MLOCK(ErrorMessageLock);
01677         MesPrint("Sorry: no argument field wildcards yet in (anti)symmetric functions");
01678         MUNLOCK(ErrorMessageLock);
01679         Terminate(-1);
01680     }
01681 }
01682 /*
01683 Sort the regular arguments by
01684 1: no wildcards, fast.
01685 2: wildcards that have been assigned.
01686 3: general arguments.
01687 4: wildcards without an assignment.
01688 */
01689 iraise = argcount;
01690 for ( i = 0; i < iraise; i++ ) {
01691     t = AT.pWorkSpace[i+lhpars];
01692     if ( *t > 0 ) { /* Category 3: general argument */
01693         continue;
01694     }
01695     else if ( *t <= -FUNCTION ) {
01696         if ( *t > -FUNCTION - WILDOFFSET ) goto cat1;
01697         type = FUNTOFUN; num = -*t - WILDOFFSET;
01698     }
01699     else if ( *t == -SYMBOL ) {
01700         if ( t[1] < 2*MAXPOWER ) goto cat1;
01701         type = SYMTOSYM; num = t[1] - 2*MAXPOWER;
01702     }
01703     else if ( *t == -INDEX ) {
01704         if ( t[1] < AM.OffsetIndex + WILDOFFSET ) goto cat1;
01705         type = INDTOIND; num = t[1] - WILDOFFSET;
01706     }
01707     else if ( *t == -VECTOR || *t == -MINVECTOR ) {
01708         if ( t[1] < AM.OffsetVector + WILDOFFSET ) goto cat1;
01709         type = VECTOVEC; num = t[1] - WILDOFFSET;
01710     }
01711     else goto cat1; /* Things like -SNUMBER etc. */
01712 }
01713 /*
01714 Now we have a wildcard and have to see whether it was assigned
01715 */
01716 m = AN.WildValue;
01717 j = nwstore;
01718 r = AT.WildMask;
01719 while ( --j >= 0 ) {
01720     if ( m[2] == num && *r ) {
01721         if ( type == *m ) break;
01722         if ( type == SYMTOSYM ) {
01723             if ( *m == SYMTONUM || *m == SYMTOSUB ) break;
01724         }
01725     }
01726 }

```

```

01724         else if ( type == INDTOIND ) {
01725             if ( *m == INDTOSUB ) break;
01726         }
01727         else if ( type == VECTOVEC ) {
01728             if ( *m == VECTOMIN || *m == VECTOSUB ) break;
01729         }
01730     }
01731     m += 4; r++;
01732 }
01733 if ( j < 0 ) { /* Category 4: Wildcard that was not assigned */
01734     a = AT.pWorkSpace+lhpars;
01735     iraise--;
01736     if ( iraise != i ) signs++;
01737     m = a[iraise];
01738     a[iraise] = a[i];
01739     a[i] = m; i--;
01740 }
01741 else { /* Category 2: Wildcard that was assigned */
01742     for ( j = 0; j < tcount; j++ ) {
01743         if ( MatchArgument(BHEAD AT.pWorkSpace[oww+j],t) ) {
01744             k = nwstore;
01745             r = AT.WildMask;
01746             num = 0;
01747             while ( --k >= 0 ) num += *r++;
01748             if ( num == assig ) { /* no wildcards were changed */
01749                 goto oneless;
01750             }
01751             break;
01752         }
01753     }
01754     if ( j >= tcount ) goto NoSuccess;
01755     j = nwstore;
01756     m = AN.WildValue;
01757     t = thewildcards + ntw; r = AT.WildMask;
01758     if ( j > 0 ) {
01759         do { /* undo assignment */
01760             *m++ = *t++; *m++ = *t++; *m++ = *t++; *m++ = *t++; *r++ = *t++;
01761         } while ( --j > 0 );
01762     }
01763     C->numrhs = *t++;
01764 }
01765 continue;
01766 cat1:
01767 for ( j = 0; j < tcount; j++ ) {
01768     m = AT.pWorkSpace[j+oww];
01769     if ( *t != *m ) continue;
01770     if ( *t < 0 ) {
01771         if ( *t <= -FUNCTION ) break;
01772         if ( t[1] == m[1] ) break;
01773     }
01774     else {
01775         k = *t; r = t;
01776         while ( --k >= 0 && *m++ == *r++ ) {}
01777         if ( k < 0 ) break;
01778     }
01779 }
01780 if ( j >= tcount ) goto NoSuccess; /* Even the fixed ones don't match */
01781 oneless:
01782     signs += j - i;
01783 /*
01784     The next statements replace the one that is commented out
01785 */
01786     tcount--;
01787     while ( j < tcount ) {
01788         AT.pWorkSpace[oww+j] = AT.pWorkSpace[oww+j+1]; j++;
01789     }
01790 /*
01791     AT.pWorkSpace[oww+j] = AT.pWorkSpace[oww+(-tcount)];
01792 */
01793     argcount--; j = i;
01794     while ( j < argcount ) {
01795         AT.pWorkSpace[lhpars+j] = AT.pWorkSpace[lhpars+j+1]; j++;
01796     }
01797     iraise--; i--;
01798 }
01799 /*
01800     Now we see whether there are any ARGWILD objects that have been
01801     assigned already. In that case the work simplifies considerably.
01802     Currently (12-nov-2001) only in (R)CYCLIC functions; hence we do not
01803     test the sign!
01804 */
01805     for ( i = 0; i < funnycount; i++ ) {
01806         k = AT.pWorkSpace[lhfunnies+i][1];
01807         m = AN.WildValue;
01808         j = nwstore;
01809         r = AT.WildMask;
01810         while ( --j >= 0 ) {

```

```

01811         if ( *m == ARGTOARG && m[2] == k ) break;
01812         m += 4; r++;
01813     }
01814     if ( *r == 0 ) continue; /* not assigned yet */
01815     m = cbuf[AT.ebufnum].rhs[m[3]];
01816     if ( *m > 0 ) { /* Tensor arguments */
01817         j = *m;
01818         if ( j > tcount - argcount ) goto NoSuccess;
01819         while ( --j >= 0 ) {
01820             m++;
01821             if ( *m < 0 ) type = -VECTOR;
01822             else if ( *m < AM.OffsetIndex ) type = -SNUMBER;
01823             else type = -INDEX;
01824             a = AT.pWorkSpace+oww;
01825             for ( k = 0; k < tcount; k++ ) {
01826                 if ( a[k][0] != type || a[k][1] != *m ) continue;
01827                 a[k] = a[--tcount];
01828                 goto nextjarg;
01829             }
01830             goto NoSuccess;
01831 nextjarg;;
01832         }
01833     }
01834     else {
01835         m++;
01836         while ( *m ) {
01837             for ( k = 0; k < tcount; k++ ) {
01838                 t = AT.pWorkSpace[oww+k];
01839                 if ( *t != *m ) continue;
01840                 r = m;
01841                 if ( *r < 0 ) {
01842                     if ( *r < -FUNCTION ) goto nextargw;
01843                     else if ( r[1] == t[1] ) goto nextargw;
01844                 }
01845                 else {
01846                     j = *r;
01847                     while ( --j >= 0 && *r++ == *t++ ) {}
01848                     if ( j < 0 ) goto nextargw;
01849                 }
01850             }
01851             goto NoSuccess;
01852 nextargw;;
01853             AT.pWorkSpace[oww+k] = AT.pWorkSpace[oww+(--tcount)];
01854             NEXTARG(m)
01855         }
01856     }
01857     AT.pWorkSpace[lhfunnies+i] = AT.pWorkSpace[lhfunnies+(--funnycount)];
01858 }
01859 if ( tcount == 0 ) {
01860     if ( argcount > 0 ) goto NoSuccess;
01861     for ( i = 0; i < funnycount; i++ ) {
01862         AddWild(BHEAD AT.pWorkSpace[lhfunnies+i][1],ARGTOARG,0);
01863     }
01864     goto quicky;
01865 }
01866 /*
01867 We have now in lhpars first iraise elements with a dubious nature.
01868 Then argcount-iraise wildcards that have not been assigned.
01869 In lhfunnies we have funnycount ARGTOARG objects. ( (R)CYCLIC only )
01870
01871 First work our way through the 'dubious' objects
01872 We check whether assig changes.
01873 */
01874 for ( i = 0; i < iraise; i++ ) {
01875     for ( j = 0; j < tcount; j++ ) {
01876         if ( MatchArgument(BHEAD AT.pWorkSpace[oww+j],AT.pWorkSpace[lhpars+i]) ) {
01877             k = nwstore;
01878             r = AT.WildMask;
01879             num = 0;
01880             while ( --k >= 0 ) num += *r++;
01881             if ( num == assig ) { /* no wildcards were changed */
01882                 signs += j-i;
01883                 AT.pWorkSpace[oww+j] = AT.pWorkSpace[oww+(--tcount)];
01884                 if ( tcount > j ) signs += tcount-j-1;
01885                 argcount--;
01886                 a = AT.pWorkSpace + lhpars;
01887                 for ( j = i; j < argcount; j++ ) a[j] = a[j+1];
01888                 iraise--;
01889                 goto nexttiraise;
01890             }
01891             else { /* We cannot use this yet */
01892                 j = nwstore;
01893                 m = AN.WildValue;
01894                 t = thewildcards + ntwa; r = AT.WildMask;
01895                 if ( j > 0 ) {
01896                     do { /* undo assignment */
01897                         *m++ = *t++; *m++ = *t++; *m++ = *t++; *m++ = *t++; *r++ = *t++;

```



```

01898             } while ( --j > 0 );
01899         }
01900         C->numrhs = *t++;
01901         C->Pointer = C->Buffer + oldcpointer;
01902         goto nexttiraise;
01903     }
01904 }
01905 }
01906 goto NoSuccess;
01907 nexttiraise:;
01908 }
01909 /*
01910 Now all leftover patterns have unassigned wildcards in them.
01911 From now on we are in potential factorial territory.
01912
01913 Strategy:
01914 1: cycle through the regular objects.
01915 2: save wildcard settings
01916 3: divide the ARGWILDS
01917 4: make permutations of leftover arguments
01918 5: try them all
01919 */
01920 cycles = AT.WorkPointer;
01921 for ( i = 0; i < tcount; i++ ) cycles[i] = tcount-i;
01922 AT.WorkPointer += tcount;
01923 signo = 0;
01924 /*MesPrint("<1> signs = %d",signs);*/
01925 for ( ;; ) {
01926     WORD oRepFunNum = AN.RepFunNum;
01927     for ( j = 0; j < argcount; j++ ) {
01928         if ( MatchArgument (BHEAD AT.pWorkSpace[oww+j],AT.pWorkSpace[lhpars+j]) == 0 ) {
01929             break;
01930         }
01931     }
01932     if ( j >= argcount ) {
01933         /*
01934         Thus far we have a match. Now the funnies
01935         */
01936         if ( funnycount ) {
01937             AT.WorkPointer = oldworkpointer;
01938             AT.pWorkPointer = oww;
01939             MLOCK(ErrorMessageLock);
01940             MesPrint("Sorry: no argument field wildcards yet in (anti)symmetric functions");
01941             MUNLOCK(ErrorMessageLock);
01942         /*
01943         Bugfix 31-oct-2001, reported by Kasper Peeters
01944         We returned here with value -1 but that is not caught.
01945         Extra note (12-nov-2001): the sign becomes a bit problematic
01946         if we have funnies. No more than one allowed in antisymmetric
01947         functions, or we have serious problems.
01948         */
01949         Terminate(-1);
01950     }
01951
01952     AN.RepFunList[AN.RepFunNum++] = offset;
01953     if ( ( (functions[fun[0]-FUNCTION].symmetric & ~REVERSEORDER) == ANTISYMMETRIC )
01954         || ( (functions[pnum-FUNCTION].symmetric & ~REVERSEORDER) == ANTISYMMETRIC ) ) {
01955         AN.RepFunList[AN.RepFunNum++] = ( signs + signo ) & 1;
01956     }
01957     else {
01958         AN.RepFunList[AN.RepFunNum++] = 0;
01959     }
01960     newpat = pattern + pattern[1];
01961     if ( newpat >= AN.patstop ) {
01962         WORD countsgn, sgn = 0;
01963         for ( countsgn = oRepFunNum+1; countsgn < AN.RepFunNum; countsgn += 2 ) {
01964             if ( AN.RepFunList[countsgn] ) sgn ^= 1;
01965         }
01966         if ( AN.SignCheck == 0 || sgn == AN.ExpectedSign ) {
01967             AT.WorkPointer = oldworkpointer;
01968             AT.pWorkPointer = oww;
01969             return(1);
01970         }
01971         if ( AN.UseFindOnly == 0 ) {
01972             if ( FindOnce(BHEAD AN.findTerm,AN.findPattern) ) {
01973                 AT.WorkPointer = oldworkpointer;
01974                 AT.pWorkPointer = oww;
01975                 AN.UsedOtherFind = 1;
01976                 return(1);
01977             }
01978         }
01979         j = 0;
01980     }
01981     else j = ScanFunctions(BHEAD newpat,inter,par);
01982     if ( j ) {
01983         WORD countsgn, sgn = 0;
01984         for ( countsgn = oRepFunNum+1; countsgn < AN.RepFunNum; countsgn += 2 ) {

```

```

01985             if ( AN.RepFunList[countsgn] ) sgn ^= 1;
01986         }
01987         if ( AN.SignCheck == 0 || sgn == AN.ExpectedSign ) {
01988             AT.WorkPointer = oldworkpointer;
01989             AT.pWorkPointer = oww;
01990             return(j);
01991         }
01992     }
01993     AN.RepFunNum = oRepFunNum;
01994     i = argcount - 1;
01995 }
01996 else i = j;
01997 j = nwstore;
01998 m = AN.WildValue;
01999 t = thewildcards + ntw; r = AT.WildMask;
02000 if ( j > 0 ) {
02001     do {
02002         *m++ = *t++; *m++ = *t++; *m++ = *t++; *m++ = *t++; *r++ = *t++;
02003     } while ( --j > 0 );
02004 }
02005 C->numrhs = *t++;
02006 C->Pointer = C->Buffer + oldcpointer;
02007 /*
02008     On to the next cycle
02009 */
02010 a = AT.pWorkSpace + oww;
02011 for ( j = i+1, t = a[i]; j < tcount; j++ ) a[j-1] = a[j];
02012 a[tcount-1] = t; cycles[i]--;
02013 signo += tcount - i - 1;
02014 while ( cycles[i] <= 0 ) {
02015     cycles[i] = tcount - i;
02016     i--;
02017     if ( i < 0 ) goto NoSuccess;
02018 /*
02019     MLOCK(ErrorMessageLock);
02020     MesPrint("Cycle i = %d",i);
02021     MUNLOCK(ErrorMessageLock);
02022 */
02023     for ( j = i+1, t = a[i]; j < tcount; j++ ) a[j-1] = a[j];
02024     a[tcount-1] = t; cycles[i]--;
02025     signo += tcount - i - 1;
02026 }
02027 }
02028 NoSuccess:
02029 if ( oldwilval > 0 ) {
02030     j = nwstore;
02031     m = AN.WildValue;
02032     t = lowlevel; r = AT.WildMask;
02033     if ( j > 0 ) {
02034         do {
02035             *m++ = *t++; *m++ = *t++; *m++ = *t++; *m++ = *t++; *r++ = *t++;
02036         } while ( --j > 0 );
02037     }
02038     C->numrhs = *t++;
02039     C->Pointer = C->Buffer + oldcpointer;
02040 }
02041 AT.WorkPointer = oldworkpointer;
02042 AT.pWorkPointer = oww;
02043 return(0);
02044 }
02045
02046 /*
02047 #] FunMatchSy :
02048 #[ MatchArgument :
02049 */
02050
02051 int MatchArgument(PHEAD WORD *arg, WORD *pat)
02052 {
02053     GETBIDENTITY
02054     WORD *m = pat, *t = arg, i, j, newvalue;
02055     WORD *argmstop = pat, *argtstop = arg;
02056     WORD *cto, *cfrom, *csav, ci;
02057     WORD oRepFunNum, *oRepFunList;
02058     WORD *oterstart, *oterstop, *opatstop;
02059     WORD wildargs, wildeat;
02060     WORD *mtrmstop, *ttrmstop, *msubstop, msizcoef;
02061     WORD *wildargtaken;
02062     int wc = 1;
02063
02064     NEXTARG(argmstop);
02065     NEXTARG(argtstop);
02066 /*
02067 #[ Both fast :
02068 */
02069     if ( *m < 0 && *t < 0 ) {
02070         if ( *t <= -FUNCTION ) {
02071             if ( *t == *m ) {}

```

```

02072         else if ( *m <= -FUNCTION-WILDOFFSET
02073         && functions[-*t-FUNCTION].spec
02074         == functions[-*m-FUNCTION-WILDOFFSET].spec ) {
02075             i = -*m - WILDOFFSET; wc = 2;
02076             if ( CheckWild(BHEAD i,FUNTOFUN,-*t,&newvalue) ) {
02077                 return(0);
02078             }
02079             AddWild(BHEAD i,FUNTOFUN,newvalue);
02080         }
02081         else if ( *m == -SYMBOL && m[1] >= 2*MAXPOWER ) {
02082             i = m[1] - 2*MAXPOWER;
02083             AN.argaddress = AT.FunArg;
02084             AT.FunArg[ARGHEAD+1] = -*t;
02085             if ( CheckWild(BHEAD i,SYMTOSUB,1,AN.argaddress) ) return(0);
02086             AddWild(BHEAD i,SYMTOSUB,0);
02087         }
02088         else return(0);
02089     }
02090     else if ( *t == *m ) {
02091         if ( t[1] == m[1] ) {}
02092         else if ( *t == -SYMBOL ) {
02093             j = SYMTOSYM;
02094             SymAll:
02095             if ( ( i = m[1] - 2*MAXPOWER ) < 0 ) return(0);
02096             wc = 2;
02097             if ( CheckWild(BHEAD i,j,t[1],&newvalue) ) return(0);
02098             AddWild(BHEAD i,j,newvalue);
02099         }
02100         else if ( *t == -INDEX ) {
02101             IndAll:
02102             i = m[1] - WILDOFFSET;
02103             if ( i < AM.OffsetIndex || i >= WILDOFFSET+AM.OffsetIndex )
02104                 return(0);
02105             /* We kill the summed over indices here */
02106             wc = 2;
02107             if ( CheckWild(BHEAD i,INDTOIND,t[1],&newvalue) ) return(0);
02108             AddWild(BHEAD i,INDTOIND,newvalue);
02109         }
02110         else if ( *t == -VECTOR || *t == -MINVECTOR ) {
02111             i = m[1] - WILDOFFSET;
02112             if ( i < AM.OffsetVector ) return(0);
02113             wc = 2;
02114             if ( CheckWild(BHEAD i,VECTOVEC,t[1],&newvalue) ) return(0);
02115             AddWild(BHEAD i,VECTOVEC,newvalue);
02116         }
02117         else return(0);
02118     }
02119     else if ( *m == -INDEX && m[1] >= AM.OffsetIndex+WILDOFFSET
02120     && m[1] < AM.OffsetIndex+(WILDOFFSET<1) ) {
02121         if ( *t == -VECTOR ) goto IndAll;
02122         if ( *t == -SNUMBER && t[1] >= 0 && t[1] < AM.OffsetIndex ) goto IndAll;
02123         if ( *t == -MINVECTOR ) {
02124             i = m[1] - WILDOFFSET;
02125             AN.argaddress = AT.MinVecArg;
02126             AT.MinVecArg[ARGHEAD+3] = t[1];
02127             wc = 2;
02128             if ( CheckWild(BHEAD i,INDTOSUB,1,AN.argaddress) ) return(0);
02129             AddWild(BHEAD i,INDTOSUB,(WORD)0);
02130         }
02131         else return(0);
02132     }
02133     else if ( *m == -SYMBOL && m[1] >= 2*MAXPOWER && *t == -SNUMBER ) {
02134         j = SYMTONUM;
02135         goto SymAll;
02136     }
02137     else if ( *m == -VECTOR && *t == -MINVECTOR &&
02138     ( i = m[1] - WILDOFFSET ) >= AM.OffsetVector ) {
02139         wc = 2;
02140         /*
02141         AN.argaddress = AT.MinVecArg;
02142         AT.MinVecArg[ARGHEAD+3] = t[1];
02143         if ( CheckWild(BHEAD i,VECTOSUB,1,AN.argaddress) ) return(0);
02144         AddWild(BHEAD i,VECTOSUB,(WORD)0);
02145         */
02146         if ( CheckWild(BHEAD i,VECTOMIN,t[1],&newvalue) ) return(0);
02147         AddWild(BHEAD i,VECTOMIN,newvalue);
02148     }
02149     else if ( *m == -MINVECTOR && *t == -VECTOR &&
02150     ( i = m[1] - WILDOFFSET ) >= AM.OffsetVector ) {
02151         wc = 2;
02152         /*
02153         AN.argaddress = AT.MinVecArg;
02154         AT.MinVecArg[ARGHEAD+3] = t[1];
02155         if ( CheckWild(BHEAD i,VECTOSUB,1,AN.argaddress) ) return(0);
02156         AddWild(BHEAD i,VECTOSUB,(WORD)0);
02157         */
02158         if ( CheckWild(BHEAD i,VECTOMIN,t[1],&newvalue) ) return(0);
02159         AddWild(BHEAD i,VECTOMIN,newvalue);

```

```

02159     }
02160     else return(0);
02161 }
02162 /*
02163 #] Both fast :
02164 #[ Fast arg :
02165 */
02166 else if ( *m > 0 && *t <= -FUNCTION ) {
02167     if ( ( m[ARGHEAD]+ARGHEAD == *m ) && m[*m-1] == 3
02168         && m[*m-2] == 1 && m[*m-3] == 1 && m[ARGHEAD+1] >= FUNCTION
02169         && m[ARGHEAD+2] == *m-ARGHEAD-4 ) { /* Check for f(?a) etc */
02170         WORD *mmmst, *mmm, mmmi;
02171         if ( m[ARGHEAD+1] >= FUNCTION+WILDOFFSET ) {
02172             mmmi = *m - WILDOFFSET;
02173             wc = 2;
02174             if ( CheckWild(BHEAD mmmi,FUNTOFUN,-*t,&newvalue) ) return(0);
02175             AddWild(BHEAD mmmi,FUNTOFUN,newvalue);
02176         }
02177         else if ( m[ARGHEAD+1] != -*t ) return(0);
02178     /*
02179         Only arguments allowed are ?a etc.
02180     */
02181     mmmst = m+*m-3;
02182     mmm = m + ARGHEAD + FUNHEAD + 1;
02183     while ( mmm < mmmst ) {
02184         if ( *mmm != -ARGWILD ) return(0);
02185         mmmi = 0;
02186         AN.argaddress = t; wc = 2;
02187         if ( CheckWild(BHEAD mmm[1],ARGTOARG,mmmi,t) ) return(0);
02188         AddWild(BHEAD mmm[1],ARGTOARG,mmmi);
02189         mmm += 2;
02190     }
02191 }
02192 else return(0);
02193 }
02194 /*
02195 #] Fast arg :
02196 #[ Fast pat :
02197 */
02198 else if ( *m < 0 && *t > 0 ) {
02199     if ( *m == -SYMBOL ) { /* SYMTOSUB */
02200         if ( m[1] < 2*MAXPOWER ) return(0);
02201         i = m[1] - 2*MAXPOWER;
02202         AN.argaddress = t; wc = 2;
02203         if ( CheckWild(BHEAD i,SYMTOSUB,1,AN.argaddress) ) return(0);
02204         AddWild(BHEAD i,SYMTOSUB,0);
02205     }
02206     else if ( *m == -VECTOR ) {
02207         if ( ( i = m[1] - WILDOFFSET ) < AM.OffsetVector ) return(0);
02208         AN.argaddress = t; wc = 2;
02209         if ( CheckWild(BHEAD i,VECTOSUB,1,t) ) return(0);
02210         AddWild(BHEAD i,VECTOSUB,(WORD)0);
02211     }
02212     else if ( *m == -INDEX ) {
02213         if ( ( i = m[1] - WILDOFFSET ) < AM.OffsetIndex ) return(0);
02214         if ( i >= AM.OffsetIndex + WILDOFFSET ) return(0);
02215         AN.argaddress = t; wc = 2;
02216         if ( CheckWild(BHEAD i,INDTOSUB,1,AN.argaddress) ) return(0);
02217         AddWild(BHEAD i,INDTOSUB,(WORD)0);
02218     }
02219     else return(0);
02220 }
02221 /*
02222 #] Fast pat :
02223 #[ Both general :
02224 */
02225 else if ( *m > 0 && *t > 0 ) {
02226     i = *m;
02227     do { if ( *m++ != *t++ ) break; } while ( --i > 0 );
02228     if ( i > 0 ) {
02229     /*
02230         Not an exact match here.
02231         We have to hope that the pattern contains a composite wildcard.
02232     */
02233         m = pat; t = arg;
02234         m += ARGHEAD; t += ARGHEAD; /* Point at (first?) term */
02235         mtrmstop = m + *m;
02236         ttrmstop = t + *t;
02237         if ( mtrmstop < argmstop ) return(0); /* More than one term */
02238         msizcoef = mtrmstop[-1];
02239         if ( msizcoef < 0 ) msizcoef = -msizcoef;
02240         msubstop = mtrmstop - msizcoef;
02241         m++;
02242         if ( m >= msubstop ) return(0); /* Only coefficient */
02243     /*
02244         Here we have a composite term. It can match provided it
02245         matches the entire argument. This argument must be a

```

```

02246         single term also and the coefficients should match
02247         (more or less).
02248         The matching takes:
02249         1: Match the functions etc. Nothing can be left.
02250         2: Match dotproducts and symbols. ONLY must match
02251             and nothing may be left.
02252         For safety it is best to take the term out and put it
02253         in workspace.
02254     */
02255     if ( argtstop > ttrmstop ) return(0);
02256     m--;
02257
02258     oterstart = AN.terstart;
02259     oterstop = AN.terstop;
02260     opatstop = AN.patstop;
02261     oRepFunList = AN.RepFunList;
02262     oRepFunNum = AN.RepFunNum;
02263     AN.RepFunNum = 0;
02264     wildargtaken = AT.WorkPointer;
02265     AN.RepFunList = wildargtaken + AN.NumTotWildArgs;
02266     AT.WorkPointer = (WORD *) ((UBYTE *) (AN.RepFunList)) + AM.MaxTer/2;
02267     csav = cto = AT.WorkPointer;
02268     cfrom = t;
02269     ci = *t;
02270     while ( --ci >= 0 ) *cto++ = *cfrom++;
02271     AT.WorkPointer = cto;
02272     ci = msizcoef;
02273     cfrom = mtrmstop;
02274     while ( --ci >= 0 ) {
02275         if ( *--cfrom != *--cto ) {
02276             AT.WorkPointer = wildargtaken;
02277             AN.RepFunList = oRepFunList;
02278             AN.RepFunNum = oRepFunNum;
02279             AN.terstart = oterstart;
02280             AN.terstop = oterstop;
02281             AN.patstop = opatstop;
02282             return(0);
02283         }
02284     }
02285     *m -= msizcoef;
02286     wildargs = AN.WildArgs;
02287     wildeat = AN.WildEat;
02288     for ( i = 0; i < wildargs; i++ ) wildargtaken[i] = AT.WildArgTaken[i];
02289     AN.ForFindOnly = 0; AN.UseFindOnly = 1;
02290     AN.nogroundlevel++;
02291     if ( FindRest(BHEAD csav,m) && ( AN.UsedOtherFind || FindOnly(BHEAD csav,m) ) ) { }
02292     else {
02293         *m += msizcoef;
02294         AT.WorkPointer = wildargtaken;
02295         AN.RepFunList = oRepFunList;
02296         AN.RepFunNum = oRepFunNum;
02297         AN.terstart = oterstart;
02298         AN.terstop = oterstop;
02299         AN.patstop = opatstop;
02300         AN.WildArgs = wildargs;
02301         AN.WildEat = wildeat;
02302         AN.nogroundlevel--;
02303         for ( i = 0; i < wildargs; i++ ) AT.WildArgTaken[i] = wildargtaken[i];
02304         return(0);
02305     }
02306     AN.nogroundlevel--;
02307     AN.WildArgs = wildargs;
02308     AN.WildEat = wildeat;
02309     for ( i = 0; i < wildargs; i++ ) AT.WildArgTaken[i] = wildargtaken[i];
02310     Substitute(BHEAD csav,m,1);
02311     cto = csav;
02312     cfrom = cto + *cto - msizcoef;
02313     cto++;
02314     *m += msizcoef;
02315     AT.WorkPointer = wildargtaken;
02316     AN.RepFunList = oRepFunList;
02317     AN.RepFunNum = oRepFunNum;
02318     AN.terstart = oterstart;
02319     AN.terstop = oterstop;
02320     AN.patstop = opatstop;
02321     if ( *cto != SUBEXPRESSION ) return(0);
02322     cto += cto[1];
02323     if ( cto < cfrom ) return(0);
02324 }
02325 }
02326 /*
02327 #] Both general :
02328 */
02329 else return(0);
02330 /*
02331 And now the success: (wc = 2 means that there was a wildcard involved)
02332 */

```

```

02333     return (wc);
02334 }
02335
02336 /*
02337  #] MatchArgument :
02338  */

```

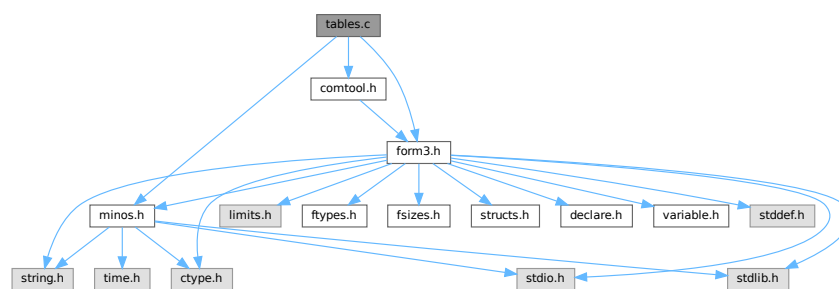
4.140 tables.c File Reference

```

#include "form3.h"
#include "minos.h"
#include "comtool.h"

```

Include dependency graph for tables.c:



Functions

- void [ClearTableTree](#) (TABLES T)
- int [InsTableTree](#) (TABLES T, WORD *tp)
- void [RedoTableTree](#) (TABLES T, int newsize)
- int [FindTableTree](#) (TABLES T, WORD *tp, int inc)
- WORD [DoTableExpansion](#) (WORD *term, WORD level)
- int [CoTableBase](#) (UBYTE *s)
- int [FlipTable](#) (FUNCTIONS f, int type)
- int [SpareTable](#) (TABLES TT)
- DBASE * [FindTB](#) (UBYTE *name)
- int [CoTBcreate](#) (UBYTE *s)
- int [CoTBopen](#) (UBYTE *s)
- int [CoTBaddto](#) (UBYTE *s)
- int [CoTBenter](#) (UBYTE *s)
- int [CoTestUse](#) (UBYTE *s)
- int [CheckTableDeclarations](#) (DBASE *d)
- int [CoTBload](#) (UBYTE *ss)
- WORD [TestUse](#) (WORD *term, WORD level)
- int [CoTBaudit](#) (UBYTE *s)
- int [CoTBon](#) (UBYTE *s)
- int [CoTBoff](#) (UBYTE *s)
- int [CoTBcleanup](#) (UBYTE *s)
- int [CoTBreplace](#) (UBYTE *s)
- int [CoTBuse](#) (UBYTE *s)
- int [CoTApply](#) (UBYTE *s)
- int [CoTBhelp](#) (UBYTE *s)

- void [ReWorkT](#) (WORD *term, WORD *funcs, WORD numfuncs)
- WORD [Apply](#) (WORD *term, WORD level)
- int [ApplyExec](#) (WORD *term, int maxtogo, WORD level)
- WORD [ApplyReset](#) (WORD level)
- WORD [TableReset](#) (void)
- int [ReleaseTB](#) (void)

Variables

- char * [helptb](#) []

4.140.1 Detailed Description

Contains all functions that deal with the table bases on the 'FORM level' The low level database routines are in [minos.c](#)

Definition in file [tables.c](#).

4.140.2 Function Documentation

4.140.2.1 ClearTableTree()

```
void ClearTableTree (
    TABLES T )
```

Definition at line 72 of file [tables.c](#).

4.140.2.2 InsTableTree()

```
int InsTableTree (
    TABLES T,
    WORD * tp )
```

Definition at line 103 of file [tables.c](#).

4.140.2.3 RedoTableTree()

```
void RedoTableTree (
    TABLES T,
    int newsiz )
```

Definition at line 266 of file [tables.c](#).

4.140.2.4 FindTableTree()

```
int FindTableTree (
    TABLES T,
    WORD * tp,
    int inc )
```

Definition at line 291 of file [tables.c](#).

4.140.2.5 DoTableExpansion()

```
WORD DoTableExpansion (
    WORD * term,
    WORD level )
```

Definition at line 334 of file [tables.c](#).

4.140.2.6 CoTableBase()

```
int CoTableBase (
    UBYTE * s )
```

Definition at line 627 of file [tables.c](#).

4.140.2.7 FlipTable()

```
int FlipTable (
    FUNCTIONS f,
    int type )
```

Definition at line 671 of file [tables.c](#).

4.140.2.8 SpareTable()

```
int SpareTable (
    TABLES TT )
```

Definition at line 694 of file [tables.c](#).

4.140.2.9 FindTB()

```
DBASE * FindTB (
    UBYTE * name )
```

Definition at line 734 of file [tables.c](#).

4.140.2.10 CoTBcreate()

```
int CoTBcreate (
    UBYTE * s )
```

Definition at line 756 of file [tables.c](#).

4.140.2.11 CoTBopen()

```
int CoTBopen (
    UBYTE * s )
```

Definition at line 772 of file [tables.c](#).

4.140.2.12 CoTBaddto()

```
int CoTBaddto (
    UBYTE * s )
```

Definition at line 801 of file [tables.c](#).

4.140.2.13 CoTBenter()

```
int CoTBenter (
    UBYTE * s )
```

Definition at line 974 of file [tables.c](#).

4.140.2.14 CoTestUse()

```
int CoTestUse (
    UBYTE * s )
```

Definition at line 1133 of file [tables.c](#).

4.140.2.15 CheckTableDeclarations()

```
int CheckTableDeclarations (
    DBASE * d )
```

Definition at line 1178 of file [tables.c](#).

4.140.2.16 CoTBload()

```
int CoTBload (
    UBYTE * ss )
```

Definition at line 1252 of file [tables.c](#).

4.140.2.17 TestUse()

```
WORD TestUse (
    WORD * term,
    WORD level )
```

Definition at line 1414 of file [tables.c](#).

4.140.2.18 CoTBaudit()

```
int CoTBaudit (
    UBYTE * s )
```

Definition at line 1473 of file [tables.c](#).

4.140.2.19 CoTBon()

```
int CoTBon (
    UBYTE * s )
```

Definition at line 1513 of file [tables.c](#).

4.140.2.20 CoTBoff()

```
int CoTBoff (
    UBYTE * s )
```

Definition at line 1544 of file [tables.c](#).

4.140.2.21 CoTBcleanup()

```
int CoTBcleanup (
    UBYTE * s )
```

Definition at line 1575 of file [tables.c](#).

4.140.2.22 CoTBreplace()

```
int CoTBreplace (
    UBYTE * s )
```

Definition at line 1587 of file [tables.c](#).

4.140.2.23 CoTBuse()

```
int CoTBuse (
    UBYTE * s )
```

Definition at line 1602 of file [tables.c](#).

4.140.2.24 CoApply()

```
int CoApply (
    UBYTE * s )
```

Definition at line 1796 of file [tables.c](#).

4.140.2.25 CoTBhelp()

```
int CoTBhelp (
    UBYTE * s )
```

Definition at line 1883 of file [tables.c](#).

4.140.2.26 ReWorkT()

```
void ReWorkT (
    WORD * term,
    WORD * funcs,
    WORD numfuncs )
```

Definition at line 1899 of file [tables.c](#).

4.140.2.27 Apply()

```
WORD Apply (
    WORD * term,
    WORD level )
```

Definition at line 1980 of file [tables.c](#).

4.140.2.28 ApplyExec()

```
int ApplyExec (
    WORD * term,
    int maxtogo,
    WORD level )
```

Definition at line 2039 of file [tables.c](#).

4.140.2.29 ApplyReset()

```
WORD ApplyReset (
    WORD level )
```

Definition at line 2256 of file [tables.c](#).

4.140.2.30 TableReset()

```
WORD TableReset (
    void )
```

Definition at line 2292 of file [tables.c](#).

4.140.2.31 ReleaseTB()

```
int ReleaseTB (
    void )
```

Definition at line 2319 of file [tables.c](#).

4.140.3 Variable Documentation

4.140.3.1 helptb

char* helptb[]

Definition at line 1851 of file [tables.c](#).

4.141 tables.c

[Go to the documentation of this file.](#)

```

00001
00006 /* #[ License : */
00007 /*
00008 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00009 *   When using this file you are requested to refer to the publication
00010 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00011 *   This is considered a matter of courtesy as the development was paid
00012 *   for by FOM the Dutch physics granting agency and we would like to
00013 *   be able to track its scientific use to convince FOM of its value
00014 *   for the community.
00015 *
00016 *   This file is part of FORM.
00017 *
00018 *   FORM is free software: you can redistribute it and/or modify it under the
00019 *   terms of the GNU General Public License as published by the Free Software
00020 *   Foundation, either version 3 of the License, or (at your option) any later
00021 *   version.
00022 *
00023 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00024 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00025 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00026 *   details.
00027 *
00028 *   You should have received a copy of the GNU General Public License along
00029 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00030 */
00031 /* #[ License : */
00032 /*
00033 *   #[ Includes :
00034 *
00035 *   File contains the routines for the tree structure of sparse tables
00036 *   We insert elements by
00037 *   InsTableTree(T,tp) with T the TABLES element and tp the pointer
00038 *   to the indices.
00039 *   We look for elements with
00040 *   FindTableTree(T,tp,inc) with T the TABLES element, tp the pointer to the
00041 *   indices or the function arguments and inc tells which of these options.
00042 *   The tree is cleared with ClearTableTree(T) and we rebuild the tree
00043 *   after a .store in which we lost a part of the table with
00044 *   RedoTableTree(T,newsize)
00045 *
00046 *   In T->tablepointers we have the lists of indices for each element.
00047 *   Additionally for each element there is an extension. There are
00048 *   TABLEEXTENSION WORDs reserved for that. The old system had two words
00049 *   One for the element in the rhs of the compile buffer and one for
00050 *   an additional rhs in case the original would be overwritten by a new
00051 *   definition, but the old was fixed by .global and hence it should be possible
00052 *   to restore it.
00053 *   New use (new = 24-sep-2001)
00054 *   rhs1,numCompBuffer1,rhs2,numCompBuffer2,usage
00055 *   Hence TABLEEXTENSION will be 5. Note that for 64 bits the use of the
00056 *   compiler buffer is overdoing it a bit, but it would be too complicated
00057 *   to try to give it special code.
00058 */
00059
00060 #include "form3.h"
00061 #include "minos.h"
00062 #include "comtool.h"
00063
00064 /* static UBYTE *sparse = (UBYTE *) "sparse"; */
00065 static UBYTE *tablebase = (UBYTE *) "tablebase";
00066
00067 /*
00068 *   #[ Includes :
00069 *   #[ ClearTableTree :
00070 */

```

```

00071
00072 void ClearTableTree(TABLES T)
00073 {
00074     COMPTREE *root;
00075     if ( T->boomlijst == 0 ) {
00076         T->MaxTreeSize = 125;
00077         T->boomlijst = (COMPTREE *)Malloc1(T->MaxTreeSize*sizeof(COMPTREE),
00078             "ClearTableTree");
00079     }
00080     root = T->boomlijst;
00081     T->numtree = 0;
00082     T->rootnum = 0;
00083     root->left = -1;
00084     root->right = -1;
00085     root->parent = -1;
00086     root->blnce = 0;
00087     root->value = -1;
00088     root->usage = 0;
00089 }
00090
00091 /*
00092 #] ClearTableTree :
00093 #[ InsTableTree :
00094
00095 int InsTableTree(TABLES T,WORD *,arglist)
00096     Searches for the element specified by the list of arguments.
00097     If found, it returns -(the offset in T->tablepointers)
00098     If not found, it will allocate a new element, balance the tree if
00099     necessary and return the number of the element in the boomlijst
00100     This number is always > 0, because we start from 1.
00101 */
00102
00103 int InsTableTree(TABLES T, WORD *tp)
00104 {
00105     COMPTREE *boomlijst, *q, *p, *s;
00106     WORD *v1, *v2, *v3, xstop;
00107     int ip, iq, is;
00108     if ( T->numtree + 1 >= T->MaxTreeSize ) {
00109         if ( T->MaxTreeSize == 0 ) ClearTableTree(T);
00110         else {
00111             is = T->MaxTreeSize * 2;
00112             s = (COMPTREE *)Malloc1(is*sizeof(COMPTREE),"InsTableTree");
00113             for ( ip = 0; ip < T->MaxTreeSize; ip++ ) { s[ip] = T->boomlijst[ip]; }
00114             if ( T->boomlijst ) M_free(T->boomlijst,"InsTableTree");
00115             T->boomlijst = s;
00116             T->MaxTreeSize = is;
00117         }
00118     }
00119     boomlijst = T->boomlijst;
00120     q = boomlijst + T->rootnum;
00121     if ( q->right == -1 ) { /* First element */
00122         T->numtree++;
00123         s = boomlijst+T->numtree;
00124         q->right = T->numtree;
00125         s->parent = T->rootnum;
00126         s->left = s->right = -1;
00127         s->blnce = 0;
00128         s->value = tp - T->tablepointers;
00129         s->usage = 0;
00130         return(T->numtree);
00131     }
00132     ip = q->right;
00133     if ( T->numind >= 0 ) xstop = T->numind;
00134     else xstop = *tp + 1;
00135     while ( ip >= 0 ) {
00136         p = boomlijst + ip;
00137         v1 = T->tablepointers + p->value;
00138         v2 = tp; v3 = tp + xstop;
00139         while ( *v1 == *v2 && v2 < v3 ) { v1++; v2++; }
00140         if ( v2 >= v3 ) return(-p->value);
00141         if ( *v1 > *v2 ) {
00142             iq = p->right;
00143             if ( iq >= 0 ) { ip = iq; }
00144             else {
00145                 T->numtree++;
00146                 is = T->numtree;
00147                 p->right = is;
00148                 s = boomlijst + is;
00149                 s->parent = ip; s->left = s->right = -1;
00150                 s->blnce = 0; s->value = tp - T->tablepointers;
00151                 s->usage = 0;
00152                 p->blnce++;
00153                 if ( p->blnce == 0 ) return(T->numtree);
00154                 goto balance;
00155             }
00156         }
00157         else if ( *v1 < *v2 ) {

```

```

00158         iq = p->left;
00159         if ( iq >= 0 ) { ip = iq; }
00160         else {
00161             T->numtree++;
00162             is = T->numtree;
00163             s = boomlijst+is;
00164             p->left = is;
00165             s->parent = ip; s->left = s->right = -1;
00166             s->blnce = 0; s->value = tp - T->tablepointers;
00167             s->usage = 0;
00168             p->blnce--;
00169             if ( p->blnce == 0 ) return(T->numtree);
00170             goto balance;
00171         }
00172     }
00173 }
00174 MesPrint("Serious problems in InsTableTree!\n");
00175 Terminate(-1);
00176 return(0);
00177 balance:;
00178 for (;;) {
00179     p = boomlijst + ip;
00180     iq = p->parent;
00181     if ( iq == T->rootnum ) break;
00182     q = boomlijst + iq;
00183     if ( ip == q->left ) q->blnce--;
00184     else q->blnce++;
00185     if ( q->blnce == 0 ) break;
00186     if ( q->blnce == -2 ) {
00187         if ( p->blnce == -1 ) { /* single rotation */
00188             q->left = p->right;
00189             p->right = iq;
00190             p->parent = q->parent;
00191             q->parent = ip;
00192             if ( boomlijst[p->parent].left == iq ) boomlijst[p->parent].left = ip;
00193             else boomlijst[p->parent].right = ip;
00194             if ( q->left >= 0 ) boomlijst[q->left].parent = iq;
00195             q->blnce = p->blnce = 0;
00196         }
00197         else { /* double rotation */
00198             s = boomlijst + is;
00199             q->left = s->right;
00200             p->right = s->left;
00201             s->right = iq;
00202             s->left = ip;
00203             if ( p->right >= 0 ) boomlijst[p->right].parent = ip;
00204             if ( q->left >= 0 ) boomlijst[q->left].parent = iq;
00205             s->parent = q->parent;
00206             q->parent = is;
00207             p->parent = is;
00208             if ( boomlijst[s->parent].left == iq )
00209                 boomlijst[s->parent].left = is;
00210             else boomlijst[s->parent].right = is;
00211             if ( s->blnce > 0 ) { q->blnce = s->blnce = 0; p->blnce = -1; }
00212             else if ( s->blnce < 0 ) { p->blnce = s->blnce = 0; q->blnce = 1; }
00213             else { p->blnce = s->blnce = q->blnce = 0; }
00214         }
00215         break;
00216     }
00217     else if ( q->blnce == 2 ) {
00218         if ( p->blnce == 1 ) { /* single rotation */
00219             q->right = p->left;
00220             p->left = iq;
00221             p->parent = q->parent;
00222             q->parent = ip;
00223             if ( boomlijst[p->parent].left == iq ) boomlijst[p->parent].left = ip;
00224             else boomlijst[p->parent].right = ip;
00225             if ( q->right >= 0 ) boomlijst[q->right].parent = iq;
00226             q->blnce = p->blnce = 0;
00227         }
00228         else { /* double rotation */
00229             s = boomlijst + is;
00230             q->right = s->left;
00231             p->left = s->right;
00232             s->left = iq;
00233             s->right = ip;
00234             if ( p->left >= 0 ) boomlijst[p->left].parent = ip;
00235             if ( q->right >= 0 ) boomlijst[q->right].parent = iq;
00236             s->parent = q->parent;
00237             q->parent = is;
00238             p->parent = is;
00239             if ( boomlijst[s->parent].left == iq ) boomlijst[s->parent].left = is;
00240             else boomlijst[s->parent].right = is;
00241             if ( s->blnce < 0 ) { q->blnce = s->blnce = 0; p->blnce = 1; }
00242             else if ( s->blnce > 0 ) { p->blnce = s->blnce = 0; q->blnce = -1; }
00243             else { p->blnce = s->blnce = q->blnce = 0; }
00244         }
00245     }

```

```

00245         break;
00246     }
00247     is = ip; ip = iq;
00248 }
00249 return(T->numtree);
00250 }
00251
00252 /*
00253 #] InsTableTree :
00254 #[ RedoTableTree :
00255
00256     To be used when a sparse table is trimmed due to a .store
00257     We rebuild the tree. In the future one could try to become faster
00258     at the cost of quite some complexity.
00259     We need to keep the first 'size' elements in the boomlijst.
00260     Kill all others and reconstruct the tree with the original ordering.
00261     This is very complicated! Because .store will either keep the whole
00262     table or remove the whole table we should not come here often.
00263     Hence we choose the slow solution for now.
00264 */
00265
00266 void RedoTableTree(TABLES T, int newsiz)
00267 {
00268     WORD *tp;
00269     int i;
00270     ClearTableTree(T);
00271     for ( i = 0, tp = T->tablepointers; i < newsiz; i++ ) {
00272         InsTableTree(T, tp);
00273         tp += ABS(T->numind)+TABLEEXTENSION;
00274     }
00275 }
00276
00277 /*
00278 #] RedoTableTree :
00279 #[ FindTableTree :
00280
00281     int FindTableTree(TABLES T, WORD *, arglist, int, inc)
00282         Searches for the element specified by the list of arguments.
00283         If found, it returns the offset in T->tablepointers
00284         If not found, it will return -1
00285         The list here is from the list of function arguments. Hence it
00286         has pairs of numbers -SNUMBER, index
00287         Actually inc says how many numbers there are and the above case is
00288         for inc = 2. For inc = 1 we have just a list of indices.
00289 */
00290
00291 int FindTableTree(TABLES T, WORD *tp, int inc)
00292 {
00293     COMPTREE *boomlijst = T->boomlijst, *q = boomlijst + T->rootnum, *p;
00294     WORD *v1, *v2, *v3, xstop;
00295     int ip, iq;
00296     if ( q->right == -1 ) return(-1);
00297     ip = q->right;
00298     if ( inc > 1 ) tp += inc-1;
00299     if ( T->numind >= 0 ) xstop = T->numind;
00300     else { /* We have to read the number of arguments first */
00301         if ( *tp <= 0 ) return(-1); /* Cannot be! */
00302         xstop = *tp+1;
00303     }
00304     while ( ip >= 0 ) {
00305         p = boomlijst + ip;
00306         v1 = T->tablepointers + p->value;
00307         v2 = tp; v3 = v1 + xstop;
00308         while ( *v1 == *v2 && v1 < v3 ) { v1++; v2 += inc; }
00309         if ( v1 == v3 ) {
00310             p->usage++;
00311             return(p->value);
00312         }
00313         if ( *v1 > *v2 ) {
00314             iq = p->right;
00315             if ( iq >= 0 ) { ip = iq; }
00316             else return(-1);
00317         }
00318         else if ( *v1 < *v2 ) {
00319             iq = p->left;
00320             if ( iq >= 0 ) { ip = iq; }
00321             else return(-1);
00322         }
00323     }
00324     MesPrint("Serious problems in FindTableTree\n");
00325     Terminate(-1);
00326     return(-1);
00327 }
00328
00329 /*
00330 #] FindTableTree :
00331 #[ DoTableExpansion :

```

```

00332 */
00333
00334 WORD DoTableExpansion(WORD *term, WORD level)
00335 {
00336     GETIDENTITY
00337     WORD *t, *tstop, *stopper, *termout, *m, *mm, *tp, *r, xx;
00338     WORD numsubexp, numbuf;
00339     TABLES T = 0;
00340     int i, j, num;
00341     AN.TeInFun = AR.TePos = 0;
00342     tstop = term + *term;
00343     stopper = tstop - ABS(tstop[-1]);
00344     t = term+1;
00345     while ( t < stopper ) {
00346         if ( *t != TABLEFUNCTION ) { t += t[1]; continue; }
00347         if ( t[FUNHEAD] > -FUNCTION ) { t += t[1]; continue; }
00348         T = functions[-t[FUNHEAD]-FUNCTION].tabl;
00349         if ( T == 0 ) { t += t[1]; continue; }
00350         if ( T->sparse ) T = T->sparse;
00351         if ( t[1] == FUNHEAD+2 && t[FUNHEAD+1] <= -FUNCTION ) break;
00352         if ( t[1] < FUNHEAD+1+2*ABS(T->numind) ) { t += t[1]; continue; }
00353         for ( i = 0; i < ABS(T->numind); i++ ) {
00354             if ( t[FUNHEAD+1+2*i] != -SYMBOL ) break;
00355         }
00356         if ( i >= ABS(T->numind) ) break;
00357         t += t[1];
00358     }
00359     if ( t >= stopper ) {
00360         MesPrint("Internal error: Missing table_ function");
00361         Terminate(-1);
00362     }
00363     /*
00364     Table in T. Now collect the numbers of the symbols;
00365     */
00366     termout = AT.WorkPointer;
00367     if ( T->sparse ) {
00368         for ( i = 0; i < T->totind; i++ ) {
00369             /*
00370             Loop over all table elements
00371             */
00372             m = termout + 1; mm = term + 1;
00373             while ( mm < t ) *m++ = *mm++;
00374             r = m;
00375             if ( t[1] == FUNHEAD+2 && t[FUNHEAD+1] <= -FUNCTION ) {
00376                 *m++ = -t[FUNHEAD+1];
00377                 tp = T->tablepointers + (ABS(T->numind)+TABLEEXTENSION)*i;
00378                 if ( T->numind < 0 ) {
00379                     xx = tp[0]+1;
00380                     *m++ = FUNHEAD+xx*2;
00381                     for ( j = 2; j < FUNHEAD; j++ ) *m++ = 0;
00382                     for ( j = 0; j < xx; j++ ) {
00383                         *m++ = -SNUMBER; *m++ = *tp++;
00384                     }
00385                 }
00386                 else {
00387                     *m++ = FUNHEAD+T->numind*2;
00388                     for ( j = 2; j < FUNHEAD; j++ ) *m++ = 0;
00389                     for ( j = 0; j < T->numind; j++ ) {
00390                         *m++ = -SNUMBER; *m++ = *tp++;
00391                     }
00392                 }
00393             }
00394             else if ( T->numind < 0 ) {
00395                 tp = T->tablepointers + (ABS(T->numind)+TABLEEXTENSION)*i;
00396                 xx = tp[0]+1;
00397                 *m++ = SYMBOL; *m++ = 2+xx*2; mm = t + FUNHEAD+1;
00398                 for ( j = 0; j < xx; j++, mm += 2, tp++ ) {
00399                     if ( *tp != 0 ) { *m++ = mm[1]; *m++ = *tp; }
00400                 }
00401                 r[1] = m-r;
00402                 if ( r[1] == 2 ) m = r;
00403             }
00404             else {
00405                 *m++ = SYMBOL; *m++ = 2+T->numind*2; mm = t + FUNHEAD+1;
00406                 tp = T->tablepointers + (T->numind+TABLEEXTENSION)*i;
00407                 for ( j = 0; j < T->numind; j++, mm += 2, tp++ ) {
00408                     if ( *tp != 0 ) { *m++ = mm[1]; *m++ = *tp; }
00409                 }
00410                 r[1] = m-r;
00411                 if ( r[1] == 2 ) m = r;
00412             }
00413             /*
00414             The next code replaces this old code
00415
00416             *m++ = SUBEXPRESSION;
00417             *m++ = SUBEXPSIZE;
00418             *m++ = *tp;

```



```

00419         *m++ = 1;
00420         *m++ = T->bufnum;
00421         FILLSUB(m);
00422         mm = t + t[1];
00423
00424         We had forgotten to take the parameters into account.
00425         Hence the subexpression prototype for wildcards was missed
00426         Now we slow things down a little bit, but we do not run
00427         any risks. There is still one problem. We have not checked
00428         that the prototype matches.
00429     */
00430     tp = T->tablepointers + (ABS(T->numind)+TABLEEXTENSION)*i
00431         +ABS(T->numind);
00432     numsubexp = tp[0]; numbuf = tp[1];
00433     r = m;
00434     #ifdef WITHPTHREADS
00435         tp = T->prototype[identity];
00436     #else
00437         tp = T->prototype;
00438     #endif
00439     for ( j = 0; j < tp[1]; j++ ) *m++ = tp[j];
00440     r[2] = numsubexp; r[4] = numbuf;
00441     /*
00442     r = m;
00443     tp = T->tablepointers + (ABS(T->numind)+TABLEEXTENSION)*i;
00444     *m++ = -t[FUNHEAD];
00445     if ( T->numind < 0 ) {
00446         xx = tp[0]+1;
00447         *m++ = t[1] - xx - T->numind - 1;
00448     }
00449     else {
00450         xx = T->numind;
00451         *m++ = t[1] - 1;
00452     }
00453     for ( j = 2; j < FUNHEAD; j++ ) *m++ = t[j];
00454     for ( j = 0; j < xx; j++ ) {
00455         *m++ = -SNUMBER; *m++ = *tp++;
00456     }
00457     tp = t + FUNHEAD + 1 + 2*T->numind;
00458     mm = t + t[1];
00459     while ( tp < mm ) *m++ = *tp++;
00460     r[1] = m-r;
00461     */
00462     /*
00463     From now on is old code
00464     */
00465     mm = t + t[1];
00466     while ( mm < tstop ) *m++ = *mm++;
00467     *termout = m - termout;
00468     AT.WorkPointer = m;
00469     if ( Generator(BHEAD termout,level) ) {
00470         MesCall("DoTableExpand");
00471         return(-1);
00472     }
00473     AT.WorkPointer = termout;
00474 }
00475 }
00476 else {
00477     for ( i = 0; i < T->totind; i++ ) {
00478     #if TABLEEXTENSION == 2
00479         if ( T->tablepointers[i] < 0 ) continue;
00480     #else
00481         if ( T->tablepointers[TABLEEXTENSION*i] < 0 ) continue;
00482     #endif
00483     m = termout + 1; mm = term + 1;
00484     while ( mm < t ) *m++ = *mm++;
00485     r = m;
00486     if ( t[1] == FUNHEAD+2 && t[FUNHEAD+1] <= -FUNCTION ) {
00487         *m++ = -t[FUNHEAD+1];
00488         *m++ = FUNHEAD+T->numind*2;
00489         for ( j = 2; j < FUNHEAD; j++ ) *m++ = 0;
00490         tp = T->tablepointers + (T->numind+TABLEEXTENSION)*i;
00491         for ( j = 0; j < T->numind; j++ ) {
00492             if ( j > 0 ) {
00493                 num = T->mm[j].mini + ( i % T->mm[j-1].size ) / T->mm[j].size;
00494             }
00495             else {
00496                 num = T->mm[j].mini + i / T->mm[j].size;
00497             }
00498             *m++ = -SNUMBER; *m++ = num;
00499         }
00500     }
00501     else {
00502         *m++ = SYMBOL; *m++ = 2+T->numind*2; mm = t + FUNHEAD+1;
00503         for ( j = 0; j < T->numind; j++, mm += 2 ) {
00504             if ( j > 0 ) {
00505                 num = T->mm[j].mini + ( i % T->mm[j-1].size ) / T->mm[j].size;

```

```

00506         }
00507         else {
00508             num = T->mm[j].mini + i / T->mm[j].size;
00509         }
00510         if ( num != 0 ) { *m++ = mm[1]; *m++ = num; }
00511     }
00512     r[1] = m-r;
00513     if ( r[1] == 2 ) m = r;
00514 }
00515 /*
00516     The next code replaces this old code
00517
00518     *m++ = SUBEXPRESSION;
00519     *m++ = SUBEXPSIZE;
00520     *m++ = *tp;
00521     *m++ = 1;
00522     *m++ = T->bufnum;
00523     FILLSUB(m);
00524     mm = t + t[1];
00525
00526     We had forgotten to take the parameters into account.
00527     Hence the subexpression prototype for wildcards was missed
00528     Now we slow things down a little bit, but we do not run
00529     any risks. There is still one problem. We have not checked
00530     that the prototype matches.
00531 */
00532     r = m;
00533     *m++ = -t[FUNHEAD];
00534     *m++ = t[1] - 1;
00535     for ( j = 2; j < FUNHEAD; j++ ) *m++ = t[j];
00536     for ( j = 0; j < T->numind; j++ ) {
00537         if ( j > 0 ) {
00538             num = T->mm[j].mini + ( i % T->mm[j-1].size ) / T->mm[j].size;
00539         }
00540         else {
00541             num = T->mm[j].mini + i / T->mm[j].size;
00542         }
00543         *m++ = -SNUMBER; *m++ = num;
00544     }
00545     tp = t + FUNHEAD + 1 + 2*T->numind;
00546     mm = t + t[1];
00547     while ( tp < mm ) *m++ = *tp++;
00548     r[1] = m - r;
00549 /*
00550     From now on is old code
00551 */
00552     while ( mm < tstop ) *m++ = *mm++;
00553     *termout = m - termout;
00554     AT.WorkPointer = m;
00555     if ( Generator(BHEAD termout, level) ) {
00556         MesCall("DoTableExpand");
00557         return(-1);
00558     }
00559 }
00560 }
00561 return(0);
00562 }
00563
00564 /*
00565 #] DoTableExpansion :
00566 #[ TableBase :
00567
00568 File with all the database related things.
00569 We have the routines for the generic database command
00570 TableBase,options;
00571 TB,options;
00572 Options are:
00573     Open "File.tbl";                Open for R/W
00574     Open "File.tbl", readonly;      Open for R
00575     Create "File.tbl";              Create for write
00576     Load "File.tbl", tablename;     Loads stubs of table
00577     Load "File.tbl";                Loads stubs of all tables
00578     Enter "File.tbl", tablename;     Loads whole table
00579     Enter "File.tbl";               Loads all tables
00580     Audit "File.tbl", options;       Print list of contents
00581     Replace "File.tbl", tablename;    Saves a table (with overwrite)
00582     Replace "File.tbl", table element; Saves a table element
00583     Cleanup "File.tbl";              Makes tables contingent
00584     AddTo "File.tbl" tablename;       Add if not yet there.
00585     AddTo "File.tbl" table element;   Add if not yet there.
00586     Delete "File.tbl" tablename;
00587     Delete "File.tbl" table element;
00588
00589     On/Off substitute;
00590     On/Off compress "File.tbl";
00591     id tbl_(f?,?a) = f(?a);
00592     When a tbl_ is used, automatically the corresponding element is compiled

```

```

00593     at the start of the next module.
00594     if TB,On,substitute [tablename], use of table RHS (if loaded)
00595     if TB,Off,substitute [tablename], use of tbl_(table,...);
00596
00597
00598     Still needed: Something like OverLoad to allow loading parts of a table
00599     from more than one file. Date stamps needed? In that case we need a touch
00600     command as well.
00601
00602     If we put all our diagrams inside, we have to go outside the concept
00603     of tables.
00604
00605     #] TableBase :
00606     #[ CoTableBase :
00607
00608     To be followed by ,subkey
00609 */
00610 static KEYWORD tboptions[] = {
00611     {"addto",          (TFUN)CoTBaddto,      0,    PARTEST}
00612     ,{"audit",         (TFUN)CoTBaudit,      0,    PARTEST}
00613     ,{"cleanup",       (TFUN)CoTbcleanup,    0,    PARTEST}
00614     ,{"create",        (TFUN)CoTBcreate,     0,    PARTEST}
00615     ,{"enter",         (TFUN)CoTBenter,      0,    PARTEST}
00616     ,{"help",          (TFUN)CoTBhelp,       0,    PARTEST}
00617     ,{"load",          (TFUN)CoTBload,       0,    PARTEST}
00618     ,{"off",           (TFUN)CoTBoff,        0,    PARTEST}
00619     ,{"on",            (TFUN)CoTBon,         0,    PARTEST}
00620     ,{"open",          (TFUN)CoTBopen,       0,    PARTEST}
00621     ,{"replace",       (TFUN)CoTBreplace,    0,    PARTEST}
00622     ,{"use",           (TFUN)CoTBuse,        0,    PARTEST}
00623 };
00624
00625 static UBYTE *tablebasename = 0;
00626
00627 int CoTableBase(UBYTE *s)
00628 {
00629     UBYTE *option, c, *t;
00630     int i,optlistsize = sizeof(tboptions)/sizeof(KEYWORD), error = 0;
00631     while ( *s == ' ' ) s++;
00632     if ( *s != '"' ) {
00633         if ( ( tolower(*s) == 'h' ) && ( tolower(s[1]) == 'e' )
00634             && ( tolower(s[2]) == 'l' ) && ( tolower(s[3]) == 'p' )
00635             && ( FG.cTable[s[4]] > 1 ) ) {
00636             CoTBhelp(s);
00637             return(0);
00638         }
00639         proper::;
00640         MesPrint("&Proper syntax: TableBase \"filename\" options");
00641         return(1);
00642     }
00643     s++; tablebasename = s;
00644     while ( *s && *s != '"' ) s++;
00645     if ( *s != '"' ) goto proper;
00646     t = s; s++; *t = 0;
00647     while ( *s == ' ' || *s == '\\t' || *s == ',' ) s++;
00648     option = s;
00649     while ( FG.cTable[*s] == 0 ) s++;
00650     c = *s; *s = 0;
00651     for ( i = 0; i < optlistsize; i++ ) {
00652         if ( StrICmp(option, (UBYTE *) (tboptions[i].name)) == 0 ) {
00653             *s = c;
00654             while ( *s == ',' ) s++;
00655             error = (tboptions[i].func)(s);
00656             *t = '"';
00657             return(error);
00658         }
00659     }
00660     MesPrint("&Unrecognized option %s in TableBase statement",option);
00661     return(1);
00662 }
00663
00664 /*
00665     #] CoTableBase :
00666     #[ FlipTable :
00667
00668     Flips the table between use as 'stub' and regular use
00669 */
00670
00671 int FlipTable(FUNCTIONS f, int type)
00672 {
00673     TABLES T, TT;
00674     T = f->tabl;
00675     if ( ( TT = T->spare ) == 0 ) {
00676         MesPrint("Error: trying to change mode on a table that has no tablebase");
00677         return(-1);
00678     }
00679     if ( TT->mode == type ) f->tabl = TT;

```

```

00680     return(0);
00681 }
00682
00683 /*
00684 #] FlipTable :
00685 #[ SpareTable :
00686
00687     Creates a spare element for a table. This is used in the table bases.
00688     It is a (thus far) empty copy of the TT table.
00689     By using FlipTable we can switch between them and alter which version of
00690     a table we will be using. Note that this also causes some extra work in the
00691     ResetVariables and the Globalize routines.
00692 */
00693
00694 int SpareTable(TABLES TT)
00695 {
00696     TABLES T;
00697     T = (TABLES)Malloc1(sizeof(struct TableEs),"table");
00698     T->defined = T->mdefined = 0; T->sparse = TT->sparse; T->mm = 0; T->flags = 0;
00699     T->numtree = 0; T->rootnum = 0; T->MaxTreeSize = 0;
00700     T->boomlijst = 0;
00701     T->strict = TT->strict;
00702     T->bounds = TT->bounds;
00703     T->bufnum = inicbufs();
00704     T->argtail = TT->argtail;
00705     T->spare = TT;
00706     T->bufferssize = 8;
00707     T->buffers = (WORD *)Malloc1(sizeof(WORD)*T->bufferssize,"SpareTable buffers");
00708     T->buffersfill = 0;
00709     T->buffers[T->buffersfill++] = T->bufnum;
00710     T->mode = 0;
00711     T->numind = TT->numind;
00712     T->totind = 0;
00713     T->prototype = TT->prototype;
00714     T->pattern = TT->pattern;
00715     T->tablepointers = 0;
00716     T->reserved = 0;
00717     T->tablenum = 0;
00718     T->numdummies = 0;
00719     T->mm = (MINMAX *)Malloc1(ABS(T->numind)*sizeof(MINMAX),"table dimensions");
00720     T->flags = (WORD *)Malloc1(ABS(T->numind)*sizeof(WORD),"table flags");
00721     ClearTableTree(T);
00722     TT->spare = T;
00723     TT->mode = 1;
00724     return(0);
00725 }
00726
00727 /*
00728 #] SpareTable :
00729 #[ FindTB :
00730
00731     Looks for a tablebase with the given name in the active tablebases.
00732 */
00733
00734 DBASE *FindTB(UBYTE *name)
00735 {
00736     DBASE *d;
00737     int i;
00738     for ( i = 0; i < NumTableBases; i++ ) {
00739         d = tablebases+i;
00740         if ( d->name && ( StrCmp(name,(UBYTE *) (d->name)) == 0 ) ) { return(d); }
00741     }
00742     return(0);
00743 }
00744
00745 /*
00746 #] FindTB :
00747 #[ CoTBcreate :
00748
00749     Creates a new tablebase.
00750     Error is when there is already an active tablebase by this name.
00751     If a file with the given name exists already, but it does not correspond
00752     to an active table base, its contents will be lost.
00753     Note that tablebasename is a static variable, defined in CoTableBase
00754 */
00755
00756 int CoTBcreate(UBYTE *s)
00757 {
00758     DUMMYUSE(s);
00759     if ( FindTB(tablebasename) != 0 ) {
00760         MesPrint("&There is already an open TableBase with the name %s",tablebasename);
00761         return(-1);
00762     }
00763     NewDbase((char *)tablebasename,0);
00764     return(0);
00765 }
00766

```

```

00767 /*
00768 #] CoTBcreate :
00769 #[ CoTBopen :
00770 */
00771
00772 int CoTBopen(UBYTE *s)
00773 {
00774     DBASE *d;
00775     MLONG rw = 1;
00776
00777     SkipSpaces(&s);
00778
00779     if ( *s ) {
00780         if ( ConsumeOption(&s,"readonly") != 0 ) {
00781             rw = 0;
00782         } else {
00783             MesPrint("&Invalid option for TableBase open: %s, ignoring", s);
00784         }
00785     }
00786
00787     if ( ( d = FindTB(tablebasename) ) != 0 ) {
00788         MesPrint("&There is already an open TableBase with the name %s",tablebasename);
00789         return(-1);
00790     }
00791     d = GetDbase((char *)tablebasename, rw);
00792     if ( CheckTableDeclarations(d) ) return(-1);
00793     return(0);
00794 }
00795
00796 /*
00797 #] CoTBopen :
00798 #[ CoTBaddto :
00799 */
00800
00801 int CoTBaddto(UBYTE *s)
00802 {
00803     GETIDENTITY
00804     DBASE *d;
00805     UBYTE *tablename, c, *t, elementstring[ELEMENTSIZE+20], *ss, *es;
00806     WORD type, funnum, lbrac, first, num, *expr, *w;
00807     TABLES T = 0;
00808     MLONG basenumber;
00809     LONG x;
00810     int i, j, error = 0, sum;
00811     if ( ( d = FindTB(tablebasename) ) == 0 ) {
00812         MesPrint("&No open tablebase with the name %s",tablebasename);
00813         return(-1);
00814     }
00815
00816     if ( ( d->rwmode ) == 0 ) {
00817         MesPrint("&Tablebase with the name %s opened in read only mode",tablebasename);
00818         return(-1);
00819     }
00820     AO.DollarOutSizeBuffer = 32;
00821     AO.DollarOutBuffer = (UBYTE *)Malloc1(AO.DollarOutSizeBuffer,
00822         "TableOutBuffer");
00823
00824     /*
00825     Now loop through the names and start adding
00826     */
00827     while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
00828     while ( *s ) {
00829         tablename = s;
00830         if ( ( s = SkipAName(s) ) == 0 ) goto tableabort;
00831         c = *s; *s = 0;
00832         if ( ( GetVar(tablename,&type,&funnum,CFUNCTION,NOAUTO) == NAMENOTFOUND )
00833             || ( T = functions[funnum].tabl ) == 0 ) {
00834             MesPrint("&%s should be a previously declared table",tablename);
00835             *s = c; goto tableabort;
00836         }
00837         if ( T->sparse == 0 ) {
00838             MesPrint("&%s should be a sparse table",tablename);
00839             *s = c; goto tableabort;
00840         }
00841         basenumber = AddTableName(d,(char *)tablename,T);
00842         if ( T->sparse && ( T->mode == 1 ) ) T = T->sparse;
00843         if ( basenumber < 0 ) basenumber = -basenumber;
00844         else if ( basenumber == 0 ) { *s = c; goto tableabort; }
00845         *s = c;
00846         if ( *s == '(' ) { /* Addition of single element */
00847             s++; es = s;
00848             for ( i = 0, w = AT.WorkPointer; i < ABS(T->numind); i++ ) {
00849                 ParseSignedNumber(x,s);
00850                 if ( FG.cTable[s[-1]] != 1 || ( *s != ',' && *s != ')' ) ) {
00851                     MesPrint("&Table arguments in TableBase addto statement should be numbers");
00852                     return(1);
00853                 }
00854                 *w++ = x;

```

```

00854         if ( *s == ' ' ) break;
00855         s++;
00856     }
00857     if ( *s != ' ' || i < ( ABS(T->numind) - 1 ) ) {
00858         MesPrint("&Incorrect number of table arguments in TableBase addto statement. Should be
%d"
00859             ,ABS(T->numind));
00860         error = 1;
00861     }
00862     c = *s; *s = 0;
00863     i = FindTableTree(T,AT.WorkPointer,1);
00864     if ( i < 0 ) {
00865         MesPrint("&Element %s has not been defined",es);
00866         error = 1;
00867         *s++ = c;
00868     }
00869     else if ( ExistsObject(d,basenumber,(char *)es) ) {}
00870     else {
00871         int dict = AO.CurrentDictionary;
00872         AO.CurrentDictionary = 0;
00873         sum = i + ABS(T->numind);
00874         /*
00875             See also commentary below
00876         */
00877         AO.DollarInOutBuffer = 1;
00878         AO.PrintType = 1;
00879         ss = AO.DollarOutBuffer;
00880         *ss = 0;
00881         AO.OutInBuffer = 1;
00882         #if ( TABLEEXTENSION == 2 )
00883             expr = cbuf[T->bufnum].rhs[T->tablepointers[sum]];
00884         #else
00885             expr = cbuf[T->tablepointers[sum+1]].rhs[T->tablepointers[sum]];
00886         #endif
00887         lbrac = 0; first = 0;
00888         while ( *expr ) {
00889             if ( WriteTerm(expr,&lbrac,first,PRINTON,0) ) {
00890                 error = 1; break;
00891             }
00892             expr += *expr;
00893         }
00894         AO.OutInBuffer = 0;
00895         AddObject(d,basenumber,(char *)es,(char *) (AO.DollarOutBuffer));
00896         *s++ = c;
00897         AO.CurrentDictionary = dict;
00898     }
00899 }
00900 else {
00901     /*
00902         Now we have to start looping through all defined elements of this table.
00903         We have to construct the arguments in text format.
00904     */
00905     for ( i = 0; i < T->totind; i++ ) {
00906         #if ( TABLEEXTENSION == 2 )
00907             if ( !T->sparse && T->tablepointers[i] < 0 ) continue;
00908         #else
00909             if ( !T->sparse && T->tablepointers[TABLEEXTENSION+i] < 0 ) continue;
00910         #endif
00911         sum = i * ( ABS(T->numind) + TABLEEXTENSION );
00912         t = elementstring;
00913         for ( j = 0; j < ABS(T->numind); j++, sum++ ) {
00914             if ( j > 0 ) *t++ = ',';
00915             num = T->tablepointers[sum];
00916             t = NumCopy(num,t);
00917             if ( ( t - elementstring ) >= ELEMENTSIZE ) {
00918                 MesPrint("&Table element specification takes more than %ld characters and cannot
be handled",
00919                     (MLONG) ELEMENTSIZE);
00920                 goto tableabort;
00921             }
00922         }
00923         if ( ExistsObject(d,basenumber,(char *)elementstring) ) { continue; }
00924         /*
00925             We have the number in basenumber and the element in elementstring.
00926             Now we need the rhs. We can use the code from WriteDollarToBuffer.
00927             Main complication: in the table compiler buffer there can be
00928             brackets. The dollars do not have those.....
00929         */
00930         AO.DollarInOutBuffer = 1;
00931         AO.PrintType = 1;
00932         ss = AO.DollarOutBuffer;
00933         *ss = 0;
00934         AO.OutInBuffer = 1;
00935         #if ( TABLEEXTENSION == 2 )
00936             expr = cbuf[T->bufnum].rhs[T->tablepointers[sum]];
00937         #else
00938             expr = cbuf[T->tablepointers[sum+1]].rhs[T->tablepointers[sum]];

```

```

00939 #endif
00940         lbrac = 0; first = 0;
00941         while ( *expr ) {
00942             if ( WriteTerm(expr,&lbrac,first,PRINTON,0) ) {
00943                 error = 1; break;
00944             }
00945             expr += *expr;
00946         }
00947         AO.OutInBuffer = 0;
00948         AddObject(d,basenumber, (char *)elementstring, (char *) (AO.DollarOutBuffer));
00949     }
00950 }
00951 while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
00952 }
00953 if ( WriteIniInfo(d) ) goto tableabort;
00954 M_free(AO.DollarOutBuffer,"DollarOutBuffer");
00955 AO.DollarOutBuffer = 0;
00956 AO.DollarOutSizeBuffer = 0;
00957 return(error);
00958 tableabort:;
00959 M_free(AO.DollarOutBuffer,"DollarOutBuffer");
00960 AO.DollarOutBuffer = 0;
00961 AO.DollarOutSizeBuffer = 0;
00962 AO.OutInBuffer = 0;
00963 return(1);
00964 }
00965
00966 /*
00967 #] CoTBaddto :
00968 #[ CoTBenter :
00969
00970 Loads the elements of the tables specified into memory and sends them
00971 one by one to the compiler as Fill statements.
00972 */
00973
00974 int CoTBenter(UBYTE *s)
00975 {
00976     DBASE *d;
00977     MLONG basenumber;
00978     UBYTE *arguments, *rhs, *buffer, *t, *u, c, *tablename;
00979     LONG size;
00980     int i, j, error = 0, error1 = 0, printall = 0;
00981     TABLES T = 0;
00982     WORD type, funnum;
00983     int dict = AO.CurrentDictionary;
00984     AO.CurrentDictionary = 0;
00985     if ( ( d = FindTB(tablebasename) ) == 0 ) {
00986         MesPrint("&No open tablebase with the name %s, check for existence of file or try readonly
mode when opening.",tablebasename);
00987         error = -1;
00988         goto Endofall;
00989     }
00990     while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
00991     if ( *s == '!' ) { printall = 1; s++; }
00992     while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
00993     if ( *s ) {
00994         while ( *s ) {
00995             tablename = s;
00996             if ( ( s = SkipAName(s) ) == 0 ) { error = 1; goto Endofall; }
00997             c = *s; *s = 0;
00998             if ( ( GetVar(tablename,&type,&funnum,CFUNCTION,NOAUTO) == NAMENOTFOUND )
00999                 || ( T = functions[funnum].tabl ) == 0 ) {
01000                 MesPrint("&%s should be a previously declared table",tablename);
01001                 basenumber = 0;
01002             }
01003             else if ( T->sparse == 0 ) {
01004                 MesPrint("&%s should be a sparse table",tablename);
01005                 basenumber = 0;
01006             }
01007             else { basenumber = GetTableName(d,(char *)tablename); }
01008             if ( T->sparse == 0 ) { SpareTable(T); }
01009             if ( basenumber > 0 ) {
01010                 for ( i = 0; i < d->info.numberofindexblocks; i++ ) {
01011                     for ( j = 0; j < NUMOBJECTS; j++ ) {
01012                         if ( basenumber != d->iblocks[i]->objects[j].tablename )
01013                             continue;
01014                         arguments = (UBYTE *) (d->iblocks[i]->objects[j].element);
01015                         rhs = (UBYTE *) ReadObject(d,basenumber, (char *)arguments);
01016                         if ( printall ) {
01017                             if ( rhs ) {
01018                                 MesPrint("&%s(%s) = %s",tablename,arguments,rhs);
01019                             }
01020                             else {
01021                                 MesPrint("&%s(%s) = 0",tablename,arguments);
01022                             }
01023                         }
01024                     }
01025                 }
01026             }
01027         }
01028     }
01029     if ( rhs ) {

```

```

01025         u = rhs; while ( *u ) u++;
01026         size = u-rhs;
01027         u = arguments; while ( *u ) u++;
01028         size += u-arguments;
01029         u = tablename; while ( *u ) u++;
01030         size += u-tablename;
01031         size += 6;
01032         buffer = (UBYTE *)Malloc1(size,"TableBase copy");
01033         t = tablename; u = buffer;
01034         while ( *t ) *u++ = *t++;
01035         *u++ = '(';
01036         t = arguments;
01037         while ( *t ) *u++ = *t++;
01038         *u++ = ')'; *u++ = '=';
01039         t = rhs;
01040         while ( *t ) *u++ = *t++;
01041         if ( t == rhs ) *u++ = '0';
01042         *u++ = 0; *u = 0;
01043         M_free(rhs,"rhs in TBenter");
01044
01045         error1 = CoFill(buffer);
01046
01047         if ( error1 < 0 ) goto Endofall;
01048         if ( error1 != 0 ) error = error1;
01049         M_free(buffer,"TableBase copy");
01050     }
01051 }
01052 }
01053 }
01054 *s = c;
01055 while ( *s == ',' || *s == ' ' || *s == '\\t' ) s++;
01056 }
01057 }
01058 else {
01059     s = (UBYTE *) (d->tablenames); basenumber = 0;
01060     while ( *s ) {
01061         basenumber++;
01062         tablename = s; while ( *s ) s++; s++;
01063         while ( *s ) s++;
01064         s++;
01065         if ( ( GetVar(tablename,&type,&funnum,CFUNCTION,NOAUTO) == NAMENOTFOUND )
01066             || ( T = functions[funnum].tabl ) == 0 ) {
01067             MesPrint("%s should be a previously declared table",tablename);
01068         }
01069         else if ( T->sparse == 0 ) {
01070             MesPrint("%s should be a sparse table",tablename);
01071         }
01072         if ( T->sparse == 0 ) { SpareTable(T); }
01073         for ( i = 0; i < d->info.numberofindexblocks; i++ ) {
01074             for ( j = 0; j < NUMOBJECTS; j++ ) {
01075                 if ( d->iblocks[i]->objects[j].tablenumber == basenumber ) {
01076                     arguments = (UBYTE *) (d->iblocks[i]->objects[j].element);
01077                     rhs = (UBYTE *) ReadObject(d,basenumber,(char *)arguments);
01078                     if ( printall ) {
01079                         if ( rhs ) {
01080                             MesPrint("%s%s = %s",tablename,arguments,rhs);
01081                         }
01082                         else {
01083                             MesPrint("%s%s = 0",tablename,arguments);
01084                         }
01085                     }
01086                     if ( rhs ) {
01087                         u = rhs; while ( *u ) u++;
01088                         size = u-rhs;
01089                         u = arguments; while ( *u ) u++;
01090                         size += u-arguments;
01091                         u = tablename; while ( *u ) u++;
01092                         size += u-tablename;
01093                         size += 6;
01094                         buffer = (UBYTE *) Malloc1(size,"TableBase copy");
01095                         t = tablename; u = buffer;
01096                         while ( *t ) *u++ = *t++;
01097                         *u++ = '(';
01098                         t = arguments;
01099                         while ( *t ) *u++ = *t++;
01100                         *u++ = ')'; *u++ = '=';
01101                         t = rhs;
01102                         while ( *t ) *u++ = *t++;
01103                         if ( t == rhs ) *u++ = '0';
01104                         *u++ = 0; *u = 0;
01105                         M_free(rhs,"rhs in TBenter");
01106
01107                         error1 = CoFill(buffer);
01108
01109                         if ( error1 < 0 ) goto Endofall;
01110                         if ( error1 != 0 ) error = error1;
01111                         M_free(buffer,"TableBase copy");

```



```

01112     }
01113     }
01114     }
01115     }
01116     }
01117 }
01118 Endofall;;
01119 AO.CurrentDictionary = dict;
01120 return(error);
01121 }
01122
01123 /*
01124 #] CoTBenter :
01125 #[ CoTestUse :
01126
01127     Possibly to be followed by names of tables.
01128     We make an array of TABLES structs to be tested in AC.usedtables.
01129     Note: only sparse tables are allowed.
01130     No arguments means all tables.
01131 */
01132
01133 int CoTestUse(UBYTE *s)
01134 {
01135     GETIDENTITY
01136     UBYTE *tablename, c;
01137     WORD type, funnum, *w;
01138     TABLES T;
01139     int error = 0;
01140     w = AT.WorkPointer;
01141     *w++ = TYPETESTUSE; *w++ = 2;
01142     while ( *s ) {
01143         tablename = s;
01144         if ( ( s = SkipAName(s) ) == 0 ) return(1);
01145         c = *s; *s = 0;
01146         if ( ( GetVar(tablename,&type,&funnum,CFUNCTION,NOAUTO) == NAMENOTFOUND )
01147             || ( T = functions[funnum].tabl ) == 0 ) {
01148             MesPrint("&%s should be a previously declared table",tablename);
01149             error = 1;
01150         }
01151         else if ( T->sparse == 0 ) {
01152             MesPrint("&%s should be a sparse table",tablename);
01153             error = 1;
01154         }
01155         *w++ = funnum + FUNCTION;
01156         *s = c;
01157         while ( *s == ' ' || *s == ',' || *s == '\\t' ) s++;
01158     }
01159     AT.WorkPointer[1] = w - AT.WorkPointer;
01160 /*
01161     if ( AT.WorkPointer[1] > 2 ) {
01162         AddNtoL(AT.WorkPointer[1],AT.WorkPointer);
01163     }
01164 */
01165     AddNtoL(AT.WorkPointer[1],AT.WorkPointer);
01166     return(error);
01167 }
01168
01169 /*
01170 #] CoTestUse :
01171 #[ CheckTableDeclarations :
01172
01173     Checks that all tables in a tablebase have identical properties to
01174     possible previous declarations. If they have not been declared
01175     before, they are declared here.
01176 */
01177
01178 int CheckTableDeclarations(DBASE *d)
01179 {
01180     WORD type, funnum;
01181     UBYTE *s, *ss, *t, *command = 0;
01182     int k, error = 0, error1, i;
01183     TABLES T;
01184     LONG commandsize = 0;
01185
01186     s = (UBYTE *) (d->tablenames);
01187     for ( k = 0; k < d->topnumber; k++ ) {
01188         if ( GetVar(s,&type,&funnum,ANYTYPE,NOAUTO) == NAMENOTFOUND ) {
01189             /*
01190                 We have to declare the table
01191             */
01192             ss = s; i = 0; while ( *ss ) { ss++; i++; } /* name */
01193             ss++; while ( *ss ) { ss++; i++; } /* tail */
01194             if ( commandsize == 0 ) {
01195                 commandsize = i + 15;
01196                 if ( commandsize < 100 ) commandsize = 100;
01197             }
01198             if ( (i+11) > commandsize ) {

```

```

01199         if ( command ) { M_free(command,"table command"); command = 0; }
01200         commandsize = i+10;
01201     }
01202     if ( command == 0 ) {
01203         command = (UBYTE *)Malloc1(commandsize,"table command");
01204     }
01205     t = command; ss = tablebase; while ( *ss ) *t++ = *ss++;
01206     *t++ = ','; while ( *s ) *t++ = *s++;
01207     s++; while ( *s ) *t++ = *s++;
01208     *t++ = ')'; *t = 0; s++;
01209     error1 = DoTable(command,1);
01210     if ( error1 ) error = error1;
01211 }
01212 else if ( ( type != CFUNCTION )
01213         || ( ( T = functions[funnum].tabl ) == 0 )
01214         || ( T->sparse == 0 ) ) {
01215     MesPrint("%s has been declared previously, but not as a sparse table.",s);
01216     error = 1;
01217     while ( *s ) s++;
01218     s++;
01219     while ( *s ) s++;
01220     s++;
01221 }
01222 else {
01223 /*
01224     Test dimension and argtail. There should be an exact match.
01225     We are not going to rename arguments when reading the elements.
01226 */
01227     ss = s;
01228     while ( *s ) s++;
01229     s++;
01230     if ( StrCmp(s,T->argtail) ) {
01231         MesPrint("&Declaration of table %s in %s different from previous
declaration",ss,d->name);
01232         error = 1;
01233     }
01234     while ( *s ) s++;
01235     s++;
01236 }
01237 }
01238 if ( command ) { M_free(command,"table command"); }
01239 return(error);
01240 }
01241
01242 /*
01243 #] CheckTableDeclarations :
01244 #[ CoTbload :
01245
01246     Loads the table stubbs of the specified tables in the indicated
01247     tablebase. Syntax:
01248     TableBase "tablebasename.tbl" load [tablename(s)];
01249     If no tables are specified all tables are taken.
01250 */
01251
01252 int CoTbload(UBYTE *ss)
01253 {
01254     DBASE *d;
01255     UBYTE *s, *name, *t, *r, *command, *arguments, *tail;
01256     LONG commandsize;
01257     int num, cs, es, ns, ts, i, j, error = 0, error1;
01258     if ( ( d = FindTB(tablebasename) ) == 0 ) {
01259         MesPrint("&No open tablebase with the name %s",tablebasename);
01260         return(-1);
01261     }
01262     commandsize = 120;
01263     command = (UBYTE *)Malloc1(commandsize,"Fill command");
01264     AC.vetofilling = 1;
01265     if ( *ss ) {
01266         while ( *ss == ',' || *ss == ' ' || *ss == '\t' ) ss++;
01267         while ( *ss ) {
01268             name = ss; ss = SkipAName(ss); *ss = 0;
01269             s = (UBYTE *) (d->tablenames);
01270             num = 0; ns = 0;
01271             while ( *s ) {
01272                 num++;
01273                 if ( StrCmp(s,name) ) {
01274                     while ( *s ) s++;
01275                     s++;
01276                     while ( *s ) s++;
01277                     s++;
01278                     num++;
01279                     continue;
01280                 }
01281                 name = s; while ( *s ) s++; ns = s-name; s++;
01282                 tail = s; while ( *s ) s++; ts = s-tail; s++;
01283                 tail++; while ( FG.cTable[*tail] == 1 ) tail++;
01284 /*

```

```

01285         Go through all elements
01286     */
01287     for ( i = 0; i < d->info.numberofindexblocks; i++ ) {
01288         for ( j = 0; j < NUMOBJECTS; j++ ) {
01289             if ( d->iblocks[i]->objects[j].tablename == num ) {
01290                 t = arguments = (UBYTE *) (d->iblocks[i]->objects[j].element);
01291                 while ( *t ) t++;
01292                 es = t - arguments;
01293                 cs = 2*es + 2*ns + ts + 10;
01294                 if ( cs > commandsize ) {
01295                     commandsize = 2*cs;
01296                     if ( command ) M_free(command,"Fill command");
01297                     command = (UBYTE *) Malloc1(commandsize,"Fill command");
01298                 }
01299                 r = command; t = name; while ( *t ) *r++ = *t++;
01300                 *r++ = '('; t = arguments; while ( *t ) *r++ = *t++;
01301                 *r++ = '='; *r++ = '='; *r++ = 't'; *r++ = 'b'; *r++ = 'l';
01302                 *r++ = '_'; *r++ = '('; t = name; while ( *t ) *r++ = *t++;
01303                 *r++ = ','; t = arguments; while ( *t ) *r++ = *t++;
01304                 t = tail; while ( *t ) {
01305                     if ( *t == '?' && r[-1] != ',' ) {
01306                         t++;
01307                         if ( FG.cTable[*t] == 0 || *t == '$' || *t == '[' ) {
01308                             t = SkipAName(t);
01309                             if ( *t == '[' ) {
01310                                 SKIPBRA1(t);
01311                             }
01312                         }
01313                         else if ( *t == '{' ) {
01314                             SKIPBRA2(t);
01315                         }
01316                         else if ( *t ) { *r++ = *t++; continue; }
01317                     }
01318                     else *r++ = *t++;
01319                 }
01320                 *r++ = ')'; *r = 0;
01321     /*
01322         Still to do: replacemode or no replacemode?
01323     */
01324         AC.vetotablebasefill = 1;
01325         error1 = CoFill(command);
01326         AC.vetotablebasefill = 0;
01327         if ( error1 < 0 ) goto finishup;
01328         if ( error1 != 0 ) error = error1;
01329     }
01330 }
01331 }
01332 break;
01333 }
01334 while ( *ss == ',' || *ss == ' ' || *ss == '\t' ) ss++;
01335 }
01336 }
01337 else { /* do all of them */
01338     s = (UBYTE *) (d->tablenames);
01339     num = 0; ns = 0;
01340     while ( *s ) {
01341         num++;
01342         name = s; while ( *s ) s++; ns = s - name; s++;
01343         tail = s; while ( *s ) s++; ts = s - tail; s++;
01344         tail++; while ( FG.cTable[*tail] == 1 ) tail++;
01345     /*
01346         Go through all elements
01347     */
01348     for ( i = 0; i < d->info.numberofindexblocks; i++ ) {
01349         for ( j = 0; j < NUMOBJECTS; j++ ) {
01350             if ( d->iblocks[i]->objects[j].tablename == num ) {
01351                 t = arguments = (UBYTE *) (d->iblocks[i]->objects[j].element);
01352                 while ( *t ) t++;
01353                 es = t - arguments;
01354                 cs = 2*es + 2*ns + ts + 10;
01355                 if ( cs > commandsize ) {
01356                     commandsize = 2*cs;
01357                     if ( command ) M_free(command,"Fill command");
01358                     command = (UBYTE *) Malloc1(commandsize,"Fill command");
01359                 }
01360                 r = command; t = name; while ( *t ) *r++ = *t++;
01361                 *r++ = '('; t = arguments; while ( *t ) *r++ = *t++;
01362                 *r++ = '='; *r++ = '='; *r++ = 't'; *r++ = 'b'; *r++ = 'l';
01363                 *r++ = '_'; *r++ = '('; t = name; while ( *t ) *r++ = *t++;
01364                 *r++ = ','; t = arguments; while ( *t ) *r++ = *t++;
01365                 t = tail; while ( *t ) {
01366                     if ( *t == '?' && r[-1] != ',' ) {
01367                         t++;
01368                         if ( FG.cTable[*t] == 0 || *t == '$' || *t == '[' ) {
01369                             t = SkipAName(t);
01370                             if ( *t == '[' ) {
01371                                 SKIPBRA1(t);

```

```

01372             }
01373             }
01374             else if ( *t == '{' ) {
01375                 SKIPBRA2(t);
01376             }
01377             else if ( *t ) { *r++ = *t++; continue; }
01378             }
01379             else *r++ = *t++;
01380             }
01381             *r++ = '>'; *r = 0;
01382             /*
01383             Still to do: replacemode or no replacemode?
01384             */
01385             AC.vetotablebasefill = 1;
01386             error1 = CoFill(command);
01387             AC.vetotablebasefill = 0;
01388             if ( error1 < 0 ) goto finishup;
01389             if ( error1 != 0 ) error = error1;
01390             }
01391             }
01392             }
01393             }
01394             }
01395             finishup:;
01396             AC.vetofilling = 0;
01397             if ( command ) M_free(command,"Fill command");
01398             return(error);
01399             }
01400             /*
01401             #] CoTBlod :
01402             #[ TestUse :
01403             #[ TestUse :
01404             Look for tbl_(tablename,arguments)
01405             if tablename is encountered, check first whether the element is in
01406             use already. If not, check in the tables in AC.usedtables.
01407             If the element is not there, add it to AC.usedtables.
01408             We need the arguments of TestUse to see for which tables it is to be done
01409             */
01410             WORD TestUse(WORD *term, WORD level)
01411             {
01412             WORD *tstop, *t, *m, *tstart, tabnum;
01413             WORD *funs, numfuns, error = 0;
01414             TABLES T;
01415             LONG i;
01416             CBUF *C = cbuf+AM.rbufnum;
01417             int isp;
01418             numfuns = C->lhs[level][1] - 2;
01419             funs = C->lhs[level] + 2;
01420             GETSTOP(term,tstop);
01421             t = term+1;
01422             while ( t < tstop ) {
01423             if ( *t != TABLESTUB ) { t += t[1]; continue; }
01424             tstart = t;
01425             m = t + FUNHEAD;
01426             t += t[1];
01427             if ( *m >= -FUNCTION ) continue;
01428             tabnum = -*m;
01429             if ( ( T = functions[tabnum-FUNCTION].tabl ) == 0 ) continue;
01430             if ( T->sparse == 0 ) continue;
01431             /*
01432             Check whether we have to test this one
01433             */
01434             if ( numfuns > 0 ) {
01435             for ( i = 0; i < numfuns; i++ ) {
01436             if ( tabnum == funs[i] ) break;
01437             }
01438             if ( i >= numfuns && numfuns > 0 ) continue;
01439             }
01440             /*
01441             Test whether the element has been defined already.
01442             If not, mark it as used.
01443             Note: we only allow sparse tables (for now)
01444             */
01445             m++;
01446             for ( i = 0; i < ABS(T->numind); i++, m += 2 ) {
01447             if ( m >= t || *m != -SNUMBER ) break;
01448             }
01449             if ( ( i == ABS(T->numind) ) &&
01450             ( ( isp = FindTableTree(T,tstart+FUNHEAD+1,2) ) >= 0 ) ) {
01451             if ( ( T->tablepointers[isp+ABS(T->numind)+4] & ELEMENTLOADED ) == 0 ) {
01452             T->tablepointers[isp+ABS(T->numind)+4] |= ELEMENTUSED;
01453             }
01454             }
01455             }

```

```

01459     }
01460     else {
01461         MesPrint("TestUse: Encountered a table element inside tbl_ that does not correspond to a
tablebase element");
01462         error = -1;
01463     }
01464 }
01465 return(error);
01466 }
01467
01468 /*
01469 #] TestUse :
01470 #[ CoTBaudit :
01471 */
01472
01473 int CoTBaudit(UBYTE *s)
01474 {
01475     DBASE *d;
01476     UBYTE *name, *tail;
01477     int i, j, error = 0, num;
01478
01479     if ( ( d = FindTB(tablebasename) ) == 0 ) {
01480         MesPrint("&No open tablebase with the name %s",tablebasename);
01481         return(-1);
01482     }
01483     while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
01484     while ( *s ) {
01485         /*
01486             Get the options here
01487             They will mainly involve the sorting of the output.
01488         */
01489         s++;
01490     }
01491     s = (UBYTE *) (d->tablenames); num = 0;
01492     while ( *s ) {
01493         num++;
01494         name = s; while ( *s ) s++; s++;
01495         tail = s; while ( *s ) s++; s++;
01496         MesPrint("Table,sparse,%s%s",name,tail);
01497         for ( i = 0; i < d->info.numberofindexblocks; i++ ) {
01498             for ( j = 0; j < NUMOBJECTS; j++ ) {
01499                 if ( d->iblocks[i]->objects[j].tablenumber == num ) {
01500                     MesPrint("    %s(%s)",name,d->iblocks[i]->objects[j].element);
01501                 }
01502             }
01503         }
01504     }
01505     return(error);
01506 }
01507
01508 /*
01509 #] CoTBaudit :
01510 #[ CoTBon :
01511 */
01512
01513 int CoTBon(UBYTE *s)
01514 {
01515     DBASE *d;
01516     UBYTE *ss, c;
01517     int error = 0;
01518     if ( ( d = FindTB(tablebasename) ) == 0 ) {
01519         MesPrint("&No open tablebase with the name %s",tablebasename);
01520         return(-1);
01521     }
01522     while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
01523     while ( *s ) {
01524         ss = SkipAName(s);
01525         c = *ss; *ss = 0;
01526         if ( StrICmp(s,(UBYTE *) ("compress")) == 0 ) {
01527             d->mode &= ~NOCOMPRESS;
01528         }
01529         else {
01530             MesPrint("&subkey %s not defined in TableBase On statement");
01531             error = 1;
01532         }
01533         *ss = c; s = ss;
01534         while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
01535     }
01536     return(error);
01537 }
01538
01539 /*
01540 #] CoTBon :
01541 #[ CoTBoff :
01542 */
01543
01544 int CoTBoff(UBYTE *s)

```

```

01545 {
01546     DBASE *d;
01547     UBYTE *ss, c;
01548     int error = 0;
01549     if ( ( d = FindTB(tablebasename) ) == 0 ) {
01550         MesPrint("&No open tablebase with the name %s",tablebasename);
01551         return(-1);
01552     }
01553     while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
01554     while ( *s ) {
01555         ss = SkipAName(s);
01556         c = *ss; *ss = 0;
01557         if ( StrICmp(s, (UBYTE *)("compress")) == 0 ) {
01558             d->mode |= NOCOMPRESS;
01559         }
01560         else {
01561             MesPrint("&subkey %s not defined in TableBase Off statement");
01562             error = 1;
01563         }
01564         *ss = c; s = ss;
01565         while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
01566     }
01567     return(error);
01568 }
01569
01570 /*
01571     #] CoTBoff :
01572     #[ CoTBcleanup :
01573 */
01574
01575 int CoTBcleanup(UBYTE *s)
01576 {
01577     DUMMYUSE(s);
01578     MesPrint("&TableBase Cleanup statement not yet implemented");
01579     return(1);
01580 }
01581
01582 /*
01583     #] CoTBcleanup :
01584     #[ CoTBreplace :
01585 */
01586
01587 int CoTBreplace(UBYTE *s)
01588 {
01589     DUMMYUSE(s);
01590     MesPrint("&TableBase Replace statement not yet implemented");
01591     return(1);
01592 }
01593
01594 /*
01595     #] CoTBreplace :
01596     #[ CoTBuse :
01597
01598     Here the actual table use as determined in TestUse causes the needed
01599     table elements to be loaded
01600 */
01601
01602 int CoTBuse(UBYTE *s)
01603 {
01604     GETIDENTITY
01605     DBASE *d;
01606     MLONG basenumber;
01607     UBYTE *arguments, *rhs, *buffer, *t, *u, c, *tablename, *p;
01608     LONG size, sum, x;
01609     int i, j, error = 0, error1 = 0, k;
01610     TABLES T = 0;
01611     WORD type, funnum, mode, *w;
01612     if ( ( d = FindTB(tablebasename) ) == 0 ) {
01613         MesPrint("&No open tablebase with the name %s",tablebasename);
01614         return(-1);
01615     }
01616     while ( *s == ',' || *s == ' ' || *s == '\t' ) s++;
01617     if ( *s ) {
01618         while ( *s ) {
01619             tablename = s;
01620             if ( ( s = SkipAName(s) ) == 0 ) return(1);
01621             c = *s; *s = 0;
01622             if ( ( GetVar(tablename,&type,&funnum,CFUNCTION,NOAUTO) == NAMENOTFOUND )
01623                 || ( T = functions[funnum].tabl ) == 0 ) {
01624                 MesPrint("&%s should be a previously declared table",tablename);
01625                 basenumber = 0;
01626             }
01627             else if ( T->sparse == 0 ) {
01628                 MesPrint("&%s should be a sparse table",tablename);
01629                 basenumber = 0;
01630             }
01631             else { basenumber = GetTableName(d, (char *)tablename); }

```

```

01632 /*      if ( T->sparse == 0 ) { SpareTable(T); } */
01633 if ( basenumber > 0 ) {
01634     for ( i = 0; i < d->info.numberofindexblocks; i++ ) {
01635         for ( j = 0; j < NUMOBJECTS; j++ ) {
01636             if ( d->iblocks[i]->objects[j].tablenumber != basenumber ) continue;
01637             arguments = p = (UBYTE *) (d->iblocks[i]->objects[j].element);
01638 /*
01639             Now translate the arguments and see whether we need
01640             this one....
01641 */
01642             for ( k = 0, w = AT.WorkPointer; k < ABS(T->numind); k++ ) {
01643                 ParseSignedNumber(x,p);
01644                 *w++ = x; p++;
01645             }
01646             sum = FindTableTree(T,AT.WorkPointer,1);
01647             if ( sum < 0 ) {
01648                 MesPrint("Table %s in tablebase %s has not been loaded properly"
01649                     ,tablename,tablebasename);
01650                 error = 1;
01651                 continue;
01652             }
01653             sum += ABS(T->numind) + 4;
01654             mode = T->tablepointers[sum];
01655             if ( ( mode & ELEMENTLOADED ) == ELEMENTLOADED ) {
01656                 T->tablepointers[sum] &= ~ELEMENTUSED;
01657                 continue;
01658             }
01659             if ( ( mode & ELEMENTUSED ) == 0 ) continue;
01660 /*
01661             We need this one!
01662 */
01663             rhs = (UBYTE *)ReadijObject(d,i,j,(char *)arguments);
01664             if ( rhs ) {
01665                 u = rhs; while ( *u ) u++;
01666                 size = u-rhs;
01667                 u = arguments; while ( *u ) u++;
01668                 size += u-arguments;
01669                 u = tablename; while ( *u ) u++;
01670                 size += u-tablename;
01671                 size += 6;
01672                 buffer = (UBYTE *)Malloca(size,"TableBase copy");
01673                 t = tablename; u = buffer;
01674                 while ( *t ) *u++ = *t++;
01675                 *u++ = '(';
01676                 t = arguments;
01677                 while ( *t ) *u++ = *t++;
01678                 *u++ = ')'; *u++ = '=';
01679                 t = rhs;
01680                 while ( *t ) *u++ = *t++;
01681                 if ( t == rhs ) { *u++ = '0'; }
01682                 *u++ = 0; *u = 0;
01683                 M_free(rhs,"rhs in TBase xxx");
01684
01685                 error1 = CoFill(buffer);
01686
01687                 if ( error1 < 0 ) { return(error); }
01688                 if ( error1 != 0 ) error = error1;
01689                 M_free(buffer,"TableBase copy");
01690             }
01691             T->tablepointers[sum] &= ~ELEMENTUSED;
01692             T->tablepointers[sum] |= ELEMENTLOADED;
01693         }
01694     }
01695     }
01696     *s = c;
01697     while ( *s == ',' || *s == ' ' || *s == '\\t' ) s++;
01698 }
01699 }
01700 else {
01701     s = (UBYTE *) (d->tablenames); basenumber = 0;
01702     while ( *s ) {
01703         basenumber++;
01704         tablename = s;
01705         while ( *s ) s++;
01706         s++;
01707         while ( *s ) s++;
01708         s++;
01709         if ( ( GetVar(tablename,&type,&funnum,CFUNCTION,NOAUTO) == NAMENOTFOUND )
01710             || ( T = functions[funnum].tabl ) == 0 ) {
01711             MesPrint("&%s should be a previously declared table",tablename);
01712         }
01713         else if ( T->sparse == 0 ) {
01714             MesPrint("&%s should be a sparse table",tablename);
01715         }
01716         if ( T->sparse && T->mode == 0 ) {
01717             MesPrint("In table %s we have a problem with stubb orders in CoTBase",tablename);
01718             error = -1;

```

```

01719     }
01720 /*     if ( T->spare == 0 ) { SpareTable(T); } */
01721     for ( i = 0; i < d->info.numberofindexblocks; i++ ) {
01722         for ( j = 0; j < NUMOBJECTS; j++ ) {
01723             if ( d->iblocks[i]->objects[j].tablename == basenumber ) {
01724                 arguments = p = (UBYTE *) (d->iblocks[i]->objects[j].element);
01725 /*
01726                 Now translate the arguments and see whether we need
01727                 this one....
01728 */
01729                 for ( k = 0, w = AT.WorkPointer; k < ABS(T->numind); k++ ) {
01730                     ParseSignedNumber(x,p);
01731                     *w++ = x; p++;
01732                 }
01733                 sum = FindTableTree(T,AT.WorkPointer,1);
01734                 if ( sum < 0 ) {
01735                     MesPrint("Table %s in tablebase %s has not been loaded properly"
01736                             ,tablename,tablebasename);
01737                     error = 1;
01738                     continue;
01739                 }
01740                 sum += ABS(T->numind) + 4;
01741                 mode = T->tablepointers[sum];
01742                 if ( ( mode & ELEMENTLOADED ) == ELEMENTLOADED ) {
01743                     T->tablepointers[sum] &= ~ELEMENTUSED;
01744                     continue;
01745                 }
01746                 if ( ( mode & ELEMENTUSED ) == 0 ) continue;
01747 /*
01748                 We need this one!
01749 */
01750                 rhs = (UBYTE *)ReadijObject(d,i,j,(char *)arguments);
01751                 if ( rhs ) {
01752                     u = rhs; while ( *u ) u++;
01753                     size = u-rhs;
01754                     u = arguments; while ( *u ) u++;
01755                     size += u-arguments;
01756                     u = tablename; while ( *u ) u++;
01757                     size += u-tablename;
01758                     size += 6;
01759                     buffer = (UBYTE *)Malloc1(size,"TableBase copy");
01760                     t = tablename; u = buffer;
01761                     while ( *t ) *u++ = *t++;
01762                     *u++ = '(';
01763                     t = arguments;
01764                     while ( *t ) *u++ = *t++;
01765                     *u++ = ')'; *u++ = '=';
01766
01767                     t = rhs;
01768                     while ( *t ) *u++ = *t++;
01769                     if ( t == rhs ) { *u++ = '0'; }
01770                     *u++ = 0; *u = 0;
01771                     M_free(rhs,"rhs in TBuse");
01772
01773                     error1 = CoFill(buffer);
01774
01775                     if ( error1 < 0 ) { return(error); }
01776                     if ( error1 != 0 ) error = error1;
01777                     M_free(buffer,"TableBase copy");
01778                 }
01779                 T->tablepointers[sum] &= ~ELEMENTUSED;
01780                 T->tablepointers[sum] |= ELEMENTLOADED;
01781             }
01782         }
01783     }
01784 }
01785 }
01786 return(error);
01787 }
01788
01789 /*
01790 #] CoTBuse :
01791 #[ CoApply :
01792
01793 Possibly to be followed by names of tables.
01794 */
01795
01796 int CoApply(UBYTE *s)
01797 {
01798     GETIDENTITY
01799     UBYTE *tablename, c;
01800     WORD type, funnum, *w;
01801     TABLES T;
01802     LONG maxtogo = MAXPOSITIVE;
01803     int error = 0;
01804     w = AT.WorkPointer;
01805     if ( FG.cTable[*s] == 1 ) {

```



```

01806         maxtogo = 0;
01807         while ( FG.cTable[*s] == 1 ) {
01808             maxtogo = maxtogo*10 + (*s-'0');
01809             s++;
01810         }
01811         while ( *s == ',' ) s++;
01812         if ( maxtogo > MAXPOSITIVE || maxtogo < 0 ) maxtogo = MAXPOSITIVE;
01813     }
01814     *w++ = TYPEAPPLY; *w++ = 3; *w++ = maxtogo;
01815     while ( *s ) {
01816         tablename = s;
01817         if ( ( s = SkipAName(s) ) == 0 ) return(1);
01818         c = *s; *s = 0;
01819         if ( ( GetVar(tablename,&type,&funnum,CFUNCTION,NOAUTO) == NAMENOTFOUND )
01820             || ( T = functions[funnum].tbl ) == 0 ) {
01821             MesPrint("&%s should be a previously declared table",tablename);
01822             error = 1;
01823         }
01824         else if ( T->sparse == 0 ) {
01825             MesPrint("&%s should be a sparse table",tablename);
01826             error = 1;
01827         }
01828         *w++ = funnum + FUNCTION;
01829         *s = c;
01830         while ( *s == ' ' || *s == ',' || *s == '\\t' ) s++;
01831     }
01832     AT.WorkPointer[1] = w - AT.WorkPointer;
01833     /*
01834     if ( AT.WorkPointer[1] > 2 ) {
01835         AddNtoL(AT.WorkPointer[1],AT.WorkPointer);
01836     }
01837     */
01838     AddNtoL(AT.WorkPointer[1],AT.WorkPointer);
01839     /*
01840     AT.WorkPointer[0] = TYPEAPPLYRESET;
01841     AddNtoL(AT.WorkPointer[1],AT.WorkPointer);
01842     */
01843     return(error);
01844 }
01845
01846 /*
01847 #] CoApply :
01848 #[ CoTBhelp :
01849 */
01850
01851 char *helptb[] = {
01852     "The TableBase statement is used as follows:"
01853     ,"TableBase \"file.tbl\" keyword subkey(s)"
01854     ,"    in which we have"
01855     ,"Keyword    Subkey(s)    Action"
01856     ,"open                Opens file.tbl for R/W"
01857     ,"create              Creates file.tbl for R/W. Old contents are lost"
01858     ,"load                Loads all stubs of all tables"
01859     ,"load    tablename(s) Loads all stubs the tables mentioned"
01860     ,"enter                Loads all stubs and rhs of all tables"
01861     ,"enter    tablename(s) Loads all stubs and rhs of the tables mentioned"
01862     ,"audit                Prints list of contents"
01863     /* ,"replace tablename    saves a table (with overwrite)" */
01864     /* ,"replace tableelement saves a table element (with overwrite)" */
01865     /* ,"cleanup                makes tables contingent" */
01866     ,"addto    tablename    adds all elements if not yet there"
01867     ,"addto    tableelement adds element if not yet there"
01868     /* ,"delete    tablename    removes table from tablebase" */
01869     /* ,"delete    tableelement removes element from tablebase" */
01870     ,"on                elements are stored in gzip format (default)"
01871     ,"off               elements are stored in uncompressed format"
01872     ,"use                compiles all needed elements"
01873     ,"use    tablename(s) compiles all needed elements of these tables"
01874     ,""
01875     ,"Related commands are:"
01876     ,"testuse            marks which tbl_ elements occur for all tables"
01877     ,"testuse    tablename(s) marks which tbl_ elements occur for given tables"
01878     ,"apply                replaces tbl_ if rhs available"
01879     ,"apply    tablename(s) replaces tbl_ for given tables if rhs available"
01880     ,""
01881     };
01882
01883 int CoTBhelp(UBYTE *s)
01884 {
01885     int i, ii = sizeof(helptb)/sizeof(char *);
01886     DUMMYUSE(s);
01887     for ( i = 0; i < ii; i++ ) MesPrint("%s",helptb[i]);
01888     return(0);
01889 }
01890
01891 /*
01892 #] CoTBhelp :

```

```

01893     #[ ReWorkT :
01894
01895     Replaces the STUBBS of the functions in the list.
01896     This gains one space. Hence we have to be very careful
01897 */
01898
01899 void ReWorkT(WORD *term, WORD *funcs, WORD numfuncs)
01900 {
01901     WORD *tstop, *tend, *m, *t, *tt, *mm, *mmm, *r, *rr;
01902     int i, j;
01903     tend = term + *term; tstop = tend - ABS(tend[-1]);
01904     m = t = term+1;
01905     while ( t < tstop ) {
01906         if ( *t == TABLESTUB ) {
01907             for ( i = 0; i < numfuncs; i++ ) {
01908                 if ( -t[FUNHEAD] == funcs[i] ) break;
01909             }
01910             if ( numfuncs == 0 || i < numfuncs ) { /* Hit */
01911                 i = t[1] - 1;
01912                 *m++ = -t[FUNHEAD]; *m++ = i; t += 2; i -= FUNHEAD;
01913                 if ( m < t ) { for ( j = 0; j < FUNHEAD-2; j++ ) *m++ = *t++; }
01914                 else { m += FUNHEAD-2; t += FUNHEAD-2; }
01915                 t++;
01916                 while ( i-- > 0 ) { *m++ = *t++; }
01917                 tt = t; mm = m;
01918                 if ( mm < tt ) {
01919                     while ( tt < tend ) *mm++ = *tt++;
01920                     *term = mm - term;
01921                     tend = term + *term; tstop = tend - ABS(tend[-1]);
01922                     t = m;
01923                 }
01924             }
01925             else { goto inc; }
01926         }
01927         else if ( *t >= FUNCTION ) {
01928             tt = t + t[1];
01929             mm = m;
01930             for ( j = 0; j < FUNHEAD; j++ ) {
01931                 if ( m == t ) { m++; t++; }
01932                 else *m++ = *t++;
01933             }
01934             while ( t < tt ) {
01935                 if ( *t <= -FUNCTION ) {
01936                     if ( m == t ) { m++; t++; }
01937                     else *m++ = *t++;
01938                 }
01939                 else if ( *t < 0 ) {
01940                     if ( m == t ) { m += 2; t += 2; }
01941                     else { *m++ = *t++; *m++ = *t++; }
01942                 }
01943                 else {
01944                     rr = t + *t; mmm = m;
01945                     for ( j = 0; j < ARGHEAD; j++ ) {
01946                         if ( m == t ) { m++; t++; }
01947                         else *m++ = *t++;
01948                     }
01949                     while ( t < rr ) {
01950                         r = t + *t;
01951                         ReWorkT(t, funcs, numfuncs);
01952                         j = *t;
01953                         if ( m == t ) { m += j; t += j; }
01954                         else { while ( j-- >= 0 ) *m++ = *t++; }
01955                         t = r;
01956                     }
01957                     *mmm = m-mmm;
01958                 }
01959             }
01960             mm[1] = m - mm;
01961             t = tt;
01962         }
01963         else {
01964             inc:
01965                 j = t[1];
01966                 if ( m < t ) { while ( j-- >= 0 ) *m++ = *t++; }
01967                 else { m += j; t += j; }
01968             }
01969         if ( m < t ) {
01970             while ( t < tend ) *m++ = *t++;
01971             *term = m - term;
01972         }
01973     }
01974
01975 /*
01976     #] ReWorkT :
01977     #[ Apply :
01978 */
01979

```

```

01980 WORD Apply(WORD *term, WORD level)
01981 {
01982     WORD *funcs, numfuncs;
01983     TABLES T;
01984     int i, j;
01985     CBUF *C = cbuf+AM.rbufnum;
01986     /*
01987      Point the tables in the proper direction
01988     */
01989     numfuncs = C->lhs[level][1] - 2;
01990     funcs = C->lhs[level] + 2;
01991     if ( numfuncs > 0 ) {
01992         for ( i = MAXBUILTINFUNCTION-FUNCTION; i < AC.FunctionList.num; i++ ) {
01993             if ( ( T = functions[i].tabl ) != 0 ) {
01994                 for ( j = 0; j < numfuncs; j++ ) {
01995                     if ( i == (funcs[j]-FUNCTION) && T->spare ) {
01996                         FlipTable(&(functions[i]),0);
01997                         break;
01998                     }
01999                 }
02000             }
02001         }
02002     }
02003     else {
02004         for ( i = MAXBUILTINFUNCTION-FUNCTION; i < AC.FunctionList.num; i++ ) {
02005             if ( ( T = functions[i].tabl ) != 0 ) {
02006                 if ( T->spare ) FlipTable(&(functions[i]),0);
02007             }
02008         }
02009     }
02010     /*
02011      Now the replacements everywhere of
02012      id tbl_(table,?a) = table(?a);
02013      Actually, this has to be done recursively.
02014      Note that we actually gain one space.
02015     */
02016     ReWorkT(term, funcs, numfuncs);
02017     return(0);
02018 }
02019
02020 /*
02021 #] Apply :
02022 #[ ApplyExec :
02023
02024 Replaces occurrences of tbl_(table,indices,pattern) by the proper
02025 rhs of table(indices,pattern). It does this up to maxtogo times
02026 in the given term. It starts with the occurrences inside the
02027 arguments of functions. If necessary it finishes at groundlevel.
02028 An infinite number of tries is indicated by maxtogo = 2^15-1 or 2^31-1.
02029 The occurrences are replaced by subexpressions. This allows TestSub
02030 to finish the job properly.
02031
02032 The main trick here is T = T->spare which turns to the proper rhs.
02033
02034 The return value is the number of substitutions that can still be made
02035 based on maxtogo. Hence, if the returnvalue is different from maxtogo
02036 there was a substitution.
02037 */
02038
02039 int ApplyExec(WORD *term, int maxtogo, WORD level)
02040 {
02041     GETIDENTITY
02042     WORD rhsnumber, *Tpattern, *funcs, numfuncs, funnum;
02043     WORD ii, *t, *tl, *w, *p, *m, *ml, *u, *r, tbufnum, csize, wilds;
02044     NESTING NN;
02045     int i, j, isp, stilltogo;
02046     CBUF *C;
02047     TABLES T;
02048     /*
02049      Startup. We need NestPoin for when we have to replace something deep down.
02050     */
02051     t = term;
02052     m = t + *t;
02053     csize = ABS(m[-1]);
02054     m -= csize;
02055     AT.NestPoin->termsize = t;
02056     if ( AT.NestPoin == AT.Nest ) AN.EndNest = t + *t;
02057     t++;
02058     /*
02059      First we look inside function arguments. Also when clean!
02060     */
02061     while ( t < m ) {
02062         if ( *t < FUNCTION ) { t += t[1]; continue; }
02063         if ( functions[*t-FUNCTION].spec > 0 ) { t += t[1]; continue; }
02064         AT.NestPoin->funsize = t;
02065         r = t + t[1];
02066         t += FUNHEAD;

```

```

02067     while ( t < r ) {
02068         if ( *t < 0 ) { NEXTARG(t); continue; }
02069         AT.NestPoin->argsize = t1 = t;
02070         u = t + *t;
02071         t += ARGHEAD;
02072         AT.NestPoin++;
02073         while ( t < u ) {
02074             /*
02075                 Now we loop over the terms inside a function argument
02076                 This defines a recursion and we have to call ApplyExec again.
02077                 The real problem is when we catch something and we have
02078                 to insert a subexpression pointer. This may use more or
02079                 less space and the whole term has to be readjusted.
02080                 This is why we have the NestPoin variables. They tell us
02081                 where the sizes of the term, the function and the arguments
02082                 are sitting, and also where the dirty flags are.
02083                 This readjusting is of course done in the groundlevel code.
02084                 Here we worry about the maxtogo count.
02085             */
02086             stilltogo = ApplyExec(t,maxtogo,level);
02087             if ( stilltogo != maxtogo ) {
02088                 if ( stilltogo <= 0 ) {
02089                     AT.NestPoin--;
02090                     return(stilltogo);
02091                 }
02092                 maxtogo = stilltogo;
02093                 u = t1 + *t1;
02094                 m = term + *term - csize;
02095             }
02096             t += *t;
02097         }
02098         AT.NestPoin--;
02099     }
02100 }
02101 /*
02102     Now we look at the ground level
02103 */
02104 C = cbuf+AM.rbufnum;
02105 t = term + 1;
02106 while ( t < m ) {
02107     if ( *t != TABLESTUB ) { t += t[1]; continue; }
02108     funnum = -t[FUNHEAD];
02109     if ( ( funnum < FUNCTION )
02110         || ( funnum >= FUNCTION+WILDOffset )
02111         || ( ( T = functions[funnum-FUNCTION].tab1 ) == 0 )
02112         || ( T->sparse == 0 )
02113         || ( T->spare == 0 ) ) { t += t[1]; continue; }
02114     numfuns = C->lhs[level][1] - 3;
02115     funs = C->lhs[level] + 3;
02116     if ( numfuns > 0 ) {
02117         for ( i = 0; i < numfuns; i++ ) {
02118             if ( funs[i] == funnum ) break;
02119         }
02120         if ( i >= numfuns ) { t += t[1]; continue; }
02121     }
02122     r = t + t[1];
02123     AT.NestPoin->funsize = t + 1;
02124     t1 = t;
02125     t += FUNHEAD + 1;
02126     /*
02127         Test whether the table catches
02128         Test 1: index arguments and range. isp will be the number
02129         of the element in the table.
02130     */
02131     T = T->sparse;
02132     #ifdef WITHPTHREADS
02133     Tpattern = T->pattern[identity];
02134     #else
02135     Tpattern = T->pattern;
02136     #endif
02137     p = Tpattern+FUNHEAD+1;
02138     for ( i = 0; i < ABS(T->numind); i++, t += 2 ) {
02139         if ( *t != -SNUMBER ) break;
02140     }
02141     if ( i < ABS(T->numind) ) { t = r; continue; }
02142     isp = FindTableTree(T,t1+FUNHEAD+1,2);
02143     if ( isp < 0 ) { t = r; continue; }
02144     rhsnumber = T->tablepointers[isp+ABS(T->numind)];
02145     #if ( TABLEEXTENSION == 2 )
02146     tbufnum = T->bunum;
02147     #else
02148     tbufnum = T->tablepointers[isp+ABS(T->numind)+1];
02149     #endif
02150     t = t1+FUNHEAD+2;
02151     ii = ABS(T->numind);
02152     while ( --ii >= 0 ) {
02153         *p = *t; t += 2; p += 2;

```

```

02154     }
02155     /*
02156         If there are more arguments we have to do some
02157         pattern matching. This should be easy. We adapted the
02158         pattern, so that the array indices match already.
02159     */
02160     #ifdef WITHPTHREADS
02161         AN.FullProto = T->prototype[identity];
02162     #else
02163         AN.FullProto = T->prototype;
02164     #endif
02165     AN.WildValue = AN.FullProto + SUBEXPSIZE;
02166     AN.WildStop = AN.FullProto+AN.FullProto[1];
02167     ClearWild(BHEAD0);
02168     AN.RepFunNum = 0;
02169     AN.RepFunList = AN.EndNest;
02170     AT.WorkPointer = (WORD *)(((UBYTE *) (AN.EndNest)) + AM.MaxTer/2);
02171     /*
02172         The RepFunList is after the term but not very relevant.
02173         We need because MatchFunction uses it
02174     */
02175     if ( AT.WorkPointer + t1[1] >= AT.WorkTop ) { MesWork(); }
02176     wilds = 0;
02177     w = AT.WorkPointer;
02178     *w++ = -t1[FUNHEAD];
02179     *w++ = t1[1] - 1;
02180     for ( i = 2; i < FUNHEAD; i++ ) *w++ = t1[i];
02181     t = t1 + FUNHEAD+1;
02182     while ( t < r ) *w++ = *t++;
02183     t = AT.WorkPointer;
02184     AT.WorkPointer = w;
02185     if ( MatchFunction(BHEAD Tpattern,t,&wilds) > 0 ) {
02186     /*
02187         Here we caught one. Now we should worry about:
02188         1: inserting the subexpression pointer with its wildcards
02189         2: NestPoin because we may not be at the lowest level
02190         The function starts at t1.
02191     */
02192     #ifdef WITHPTHREADS
02193         m1 = T->prototype[identity];
02194     #else
02195         m1 = T->prototype;
02196     #endif
02197     m1[2] = rhsnumber;
02198     m1[4] = tbufnum;
02199     t = t1;
02200     j = t[1];
02201     i = m1[1];
02202     if ( j > i ) {
02203         j = i - j;
02204         NCOPY(t,m1,i);
02205         m1 = AN.EndNest;
02206         while ( r < m1 ) *t++ = *r++;
02207         AN.EndNest = t;
02208         *term += j;
02209         NN = AT.NestPoin;
02210         while ( NN > AT.Nest ) {
02211             NN--;
02212             NN->termsize[0] += j;
02213             NN->funsize[1] += j;
02214             NN->argsize[0] += j;
02215             NN->funsize[2] |= DIRTYFLAG;
02216             NN->argsize[1] |= DIRTYFLAG;
02217         }
02218         m += j;
02219     }
02220     else if ( j < i ) {
02221         j = i-j;
02222         t = AN.EndNest;
02223         while ( t >= r ) { t[j] = *t; t--; }
02224         t = t1;
02225         NCOPY(t,m1,i);
02226         AN.EndNest += j;
02227         *term += j;
02228         NN = AT.NestPoin;
02229         while ( NN > AT.Nest ) {
02230             NN--;
02231             NN->termsize[0] += j;
02232             NN->funsize[1] += j;
02233             NN->argsize[0] += j;
02234             NN->funsize[2] |= DIRTYFLAG;
02235             NN->argsize[1] |= DIRTYFLAG;
02236         }
02237         m += j;
02238     }
02239     else {
02240         NCOPY(t,m1,j);

```

```

02241         }
02242         r = t1 + t1[1];
02243         maxtogo--;
02244         if ( maxtogo <= 0 ) return(maxtogo);
02245     }
02246     t = r;
02247 }
02248 return(maxtogo);
02249 }
02250
02251 /*
02252 #] ApplyExec :
02253 #[ ApplyReset :
02254 */
02255
02256 WORD ApplyReset(WORD level)
02257 {
02258     WORD *funcs, numfuncs;
02259     TABLES T;
02260     int i, j;
02261     CBUF *C = cbuf+AM.rbufnum;
02262
02263     numfuncs = C->lhs[level][1] - 2;
02264     funcs = C->lhs[level] + 2;
02265     if ( numfuncs > 0 ) {
02266         for ( i = MAXBUILTINFUNCTION-FUNCTION; i < AC.FunctionList.num; i++ ) {
02267             if ( ( T = functions[i].tabl ) != 0 ) {
02268                 for ( j = 0; j < numfuncs; j++ ) {
02269                     if ( i == (funcs[j]-FUNCTION) && T->spare ) {
02270                         FlipTable(&(functions[i]),1);
02271                         break;
02272                     }
02273                 }
02274             }
02275         }
02276     }
02277     else {
02278         for ( i = MAXBUILTINFUNCTION-FUNCTION; i < AC.FunctionList.num; i++ ) {
02279             if ( ( T = functions[i].tabl ) != 0 ) {
02280                 if ( T->spare ) FlipTable(&(functions[i]),1);
02281             }
02282         }
02283     }
02284     return(0);
02285 }
02286
02287 /*
02288 #] ApplyReset :
02289 #[ TableReset :
02290 */
02291
02292 WORD TableReset(void)
02293 {
02294     TABLES T;
02295     int i;
02296
02297     for ( i = MAXBUILTINFUNCTION-FUNCTION; i < AC.FunctionList.num; i++ ) {
02298         if ( ( T = functions[i].tabl ) != 0 && T->spare && T->mode == 0 ) {
02299             functions[i].tabl = T->spare;
02300         }
02301     }
02302     return(0);
02303 }
02304
02305 /*
02306 #] TableReset :
02307 #[ LoadTableElement :
02308 ?????
02309 int LoadTableElement(DBASE *d, TABLE *T, WORD num)
02310 {
02311 }
02312
02313 #] LoadTableElement :
02314 #[ ReleaseTB :
02315
02316 Releases all TableBases
02317 */
02318
02319 int ReleaseTB(void)
02320 {
02321     DBASE *d;
02322     int i;
02323     for ( i = NumTableBases - 1; i >= 0; i-- ) {
02324         d = tablebases+i;
02325         fclose(d->handle);
02326         FreeTableBase(d);
02327     }

```

```

02328     return(0);
02329 }
02330
02331 /*
02332  #] ReleaseTB :
02333 */

```

4.142 threads.c File Reference

4.142.1 Detailed Description

Routines for the interface of FORM with the pthreads library

This is the main part of the parallelization of TFORM. It is important to also look in the files [structs.h](#) and [variable.h](#) because the treatment of the A and B structs is essential (these structs are used by means of the macros AM, AP, AC, AS, AR, AT, AN, AO and AX). Also the definitions and use of the macros BHEAD and PHEAD should be looked up.

The sources are set up in such a way that if WITHPTHREADS isn't defined there is no trace of pthread parallelization. The reason is that TFORM is far more memory hungry than sequential FORM.

Special attention should also go to the locks. The proper use of the locks is essential and determines whether TFORM can work at all. We use the LOCK/UNLOCK macros which are empty in the case of sequential FORM. These locks are at many places in the source files when workers can interfere with each others data or with the data of the master.

Definition in file [threads.c](#).

4.143 threads.c

[Go to the documentation of this file.](#)

```

00001
00022 /* #] License : */
00023 /*
00024  * Copyright (C) 1984-2023 J.A.M. Vermaseren
00025  * When using this file you are requested to refer to the publication
00026  * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00027  * This is considered a matter of courtesy as the development was paid
00028  * for by FOM the Dutch physics granting agency and we would like to
00029  * be able to track its scientific use to convince FOM of its value
00030  * for the community.
00031  *
00032  * This file is part of FORM.
00033  *
00034  * FORM is free software: you can redistribute it and/or modify it under the
00035  * terms of the GNU General Public License as published by the Free Software
00036  * Foundation, either version 3 of the License, or (at your option) any later
00037  * version.
00038  *
00039  * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00040  * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00041  * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00042  * details.
00043  *
00044  * You should have received a copy of the GNU General Public License along
00045  * with FORM. If not, see <http://www.gnu.org/licenses/>.
00046  */
00047 /* #] License : */
00048
00049 #ifndef WITHPTHREADS
00050
00051 #define WHOLEBRACKETS
00052 /*
00053  #] Variables :
00054
00055  The sortbot additions are from 17-may-2007 and after. They constitute

```

```

00056     an attempt to make the final merge sorting faster for the master.
00057     This way the master has only one compare per term.
00058     It does add some complexity, but the final merge routine (MasterMerge)
00059     is much simpler for the sortbots. On the other hand the original merging is
00060     for a large part a copy of the MergePatches routine in sort.c and hence
00061     even though complex the bad part has been thoroughly debugged.
00062 */
00063
00064 #include "form3.h"
00065
00066 #ifdef WITH_ALARM
00067 // This is only required if we are blocking SIG_ALARM in the worker threads.
00068 #include <signal.h>
00069 #endif
00070
00071 #ifdef WITHFLOAT
00072 #include <gmp.h>
00073
00074 int PackFloat(WORD *,mpf_t);
00075 int UnpackFloat(mpf_t, WORD *);
00076 void RatToFloat(mpf_t result, UWORD *formrat, int ratsize);
00077 #endif
00078
00079 static int numberofthreads;
00080 static int numberofworkers;
00081 static int identityofthreads = 0;
00082 static int *listofavailables;
00083 static int topofavailables = 0;
00084 static pthread_key_t identitykey;
00085 static INILOCK(numberofthreadslock)
00086 static INILOCK(availabilitylock)
00087 static pthread_t *threadpointers = 0;
00088 static pthread_mutex_t *wakeuplocks;
00089 static pthread_mutex_t *wakeupmasterthreadlocks;
00090 static pthread_cond_t *wakeupconditions;
00091 static pthread_condattr_t *wakeupconditionattributes;
00092 static int *wakeup;
00093 static int *wakeupmasterthread;
00094 static INILOCK(wakeupmasterlock)
00095 static pthread_cond_t wakeupmasterconditions = PTHREAD_COND_INITIALIZER;
00096 static pthread_cond_t *wakeupmasterthreadconditions;
00097 static int wakeupmaster = 0;
00098 static int identityretval;
00099 /* static INILOCK(clearclocklock) */
00100 static LONG *timerinfo;
00101 static LONG *sumtimerinfo;
00102 static int numberclaimed;
00103
00104 static THREADBUCKET **threadbuckets, **freebuckets;
00105 static int numthreadbuckets;
00106 static int numberoffullbuckets;
00107
00108 /* static int numberbusy = 0; */
00109
00110 INILOCK(dummylock)
00111 INIRWLOCK(dummyrwlock)
00112 static pthread_cond_t dummywakeupcondition = PTHREAD_COND_INITIALIZER;
00113
00114 #ifdef WITHSORTBOTS
00115 static POSITION SortBotPosition;
00116 static int numberofsortbots;
00117 static INILOCK(wakeupsortbotlock)
00118 static pthread_cond_t wakeupsortbotconditions = PTHREAD_COND_INITIALIZER;
00119 static int topsortbotavailables = 0;
00120 static LONG numberofterms;
00121 #endif
00122
00123 /*
00124     #] Variables :
00125     #[ Identity :
00126         #[ StartIdentity :
00127 */
00128 void StartIdentity(void)
00129 {
00130     pthread_key_create(&identitykey, FinishIdentity);
00131 }
00132
00133 /*
00134     #] StartIdentity :
00135     #[ FinishIdentity :
00136 */
00137 void FinishIdentity(void *keyp)
00138 {
00139     DUMMYUSE(keyp);
00140     /* free(keyp); */
00141 }
00142
00143
00144
00145
00146
00147
00148
00149
00150
00151

```



```

00152 /*
00153     #] FinishIdentity :
00154     #[ SetIdentity :
00155 */
00160 int SetIdentity(int *identityretval)
00161 {
00162 /*
00163 #ifdef _MSC_VER
00164     printf("addr %d\n",&numberofthreadslock);
00165     printf("size %d\n",sizeof(numberofthreadslock));
00166 #endif
00167 */
00168     LOCK(numberofthreadslock);
00169     *identityretval = identityofthreads++;
00170     UNLOCK(numberofthreadslock);
00171     pthread_setspecific(identitykey, (void *)identityretval);
00172     return(*identityretval);
00173 }
00174
00175 /*
00176     #] SetIdentity :
00177     #[ WhoAmI :
00178 */
00179
00190 int WhoAmI(void)
00191 {
00192     int *identity;
00193 /*
00194     First a fast exit for when there is at most one thread
00195 */
00196     if ( identityofthreads <= 1 ) return(0);
00197 /*
00198     Now the reading of the key.
00199     According to the book the statement should read:
00200
00201     pthread_getspecific(identitykey, (void **)(&identity));
00202
00203     but according to the information in pthread.h it is:
00204 */
00205     identity = (int *)pthread_getspecific(identitykey);
00206     return(*identity);
00207 }
00208
00209 /*
00210     #] WhoAmI :
00211     #[ BeginIdentities :
00212 */
00218 void BeginIdentities(void)
00219 {
00220     StartIdentity();
00221     SetIdentity(&identityretval);
00222 }
00223
00224 /*
00225     #] BeginIdentities :
00226     #] Identity :
00227     #[ StartHandleLock :
00228 */
00235 void StartHandleLock(void)
00236 {
00237     AM.handlelock = dummyrwlock;
00238 }
00239
00240 /*
00241     #] StartHandleLock :
00242     #[ StartAllThreads :
00243 */
00261 int StartAllThreads(int number)
00262 {
00263     int identity, j, dummy, mul;
00264     ALLPRIVATES *B;
00265     pthread_t thethread;
00266     identity = WhoAmI();
00267
00268 #ifdef WITHSORTBOTS
00269     timerinfo = (LONG *)Malloca1(sizeof(LONG)*number*2,"timerinfo");
00270     sumtimerinfo = (LONG *)Malloca1(sizeof(LONG)*number*2,"sumtimerinfo");
00271     for ( j = 0; j < number*2; j++ ) { timerinfo[j] = 0; sumtimerinfo[j] = 0; }
00272     mul = 2;
00273 #else
00274     timerinfo = (LONG *)Malloca1(sizeof(LONG)*number,"timerinfo");
00275     sumtimerinfo = (LONG *)Malloca1(sizeof(LONG)*number,"sumtimerinfo");
00276     for ( j = 0; j < number; j++ ) { timerinfo[j] = 0; sumtimerinfo[j] = 0; }
00277     mul = 1;
00278 #endif
00279
00280     listofavaiables = (int *)Malloca1(sizeof(int)*(number+1),"listofavaiables");

```

```

00281     threadpointers = (pthread_t *)Malloc1(sizeof(pthread_t)*number*mul,"threadpointers");
00282     AB = (ALLPRIVATES **)Malloc1(sizeof(ALLPRIVATES *)*number*mul,"Private structs");
00283
00284     wakeup = (int *)Malloc1(sizeof(int)*number*mul,"wakeup");
00285     wakeuplocks = (pthread_mutex_t *)Malloc1(sizeof(pthread_mutex_t)*number*mul,"wakeuplocks");
00286     wakeupconditions = (pthread_cond_t
*)Malloc1(sizeof(pthread_cond_t)*number*mul,"wakeupconditions");
00287     wakeupconditionattributes = (pthread_condattr_t *)
00288         Malloc1(sizeof(pthread_condattr_t)*number*mul,"wakeupconditionattributes");
00289
00290     wakeupmasterthread = (int *)Malloc1(sizeof(int)*number*mul,"wakeupmasterthread");
00291     wakeupmasterthreadlocks = (pthread_mutex_t
*)Malloc1(sizeof(pthread_mutex_t)*number*mul,"wakeupmasterthreadlocks");
00292     wakeupmasterthreadconditions = (pthread_cond_t
*)Malloc1(sizeof(pthread_cond_t)*number*mul,"wakeupmasterthread");
00293
00294     numberofthreads = number;
00295     numberofworkers = number - 1;
00296     threadpointers[identity] = pthread_self();
00297     toposavailables = 0;
00298
00299 #ifdef WITH_ALARM
00300     /* During thread creation, we block SIGALRM on the main thread. The created
00301        threads will inherit this. This is required for #timeout to work properly
00302        in TFORM: only the main thread should recieve SIGALRM. */
00303     sigset_t sig_set;
00304     sigemptyset(&sig_set);
00305     sigaddset(&sig_set, SIGALRM);
00306     pthread_sigmask(SIG_BLOCK, &sig_set, NULL);
00307 #endif
00308
00309     for ( j = 1; j < number; j++ ) {
00310         if ( pthread_create(&thethread,NULL,RunThread,(void *)(&dummy)) )
00311             goto failure;
00312     }
00313     /*
00314     Now we initialize the master at the same time that the workers are doing so.
00315     */
00316     B = InitializeOneThread(identity);
00317     AR.infile = &(AR.Fscr[0]);
00318     AR.outfile = &(AR.Fscr[1]);
00319     AR.hidefile = &(AR.Fscr[2]);
00320     AM.sbuflock = dummylock;
00321     AS.inputslock = dummylock;
00322     AS.outputslock = dummylock;
00323     AS.MaxExprSizeLock = dummylock;
00324     AP.PreVarLock = dummylock;
00325     AC.halfmodlock = dummylock;
00326     MakeThreadBuckets(number,0);
00327     /*
00328     Now we wait for the workers to finish their startup.
00329     We don't want to initialize the sortbots yet and run the risk that
00330     some of them may end up with a lower number than one of the workers.
00331     */
00332     MasterWaitAll();
00333 #ifdef WITHSORTBOTS
00334     if ( numberofworkers > 2 ) {
00335         numberofsortbots = numberofworkers-2;
00336         for ( j = numberofworkers+1; j < 2*numberofworkers-1; j++ ) {
00337             if ( pthread_create(&thethread,NULL,RunSortBot,(void *)(&dummy)) )
00338                 goto failure;
00339         }
00340     }
00341     else {
00342         numberofsortbots = 0;
00343     }
00344     MasterWaitAllSortBots();
00345     DefineSortBotTree();
00346 #endif
00347     IniSortBlocks(number-1);
00348     AS.MasterSort = 0;
00349     AM.storefilelock = dummylock;
00350
00351 #ifdef WITH_ALARM
00352     /* Now we allow the main thread to recieve SIGALRM again. */
00353     pthread_sigmask(SIG_UNBLOCK, &sig_set, NULL);
00354 #endif
00355     /*
00356     MesPrint("AB = %x %x %x  %d",AB[0],AB[1],AB[2], identityofthreads);
00357     */
00358     return(0);
00359 failure:
00360     MesPrint("Cannot start %d threads",number);
00361     Terminate(-1);
00362     return(-1);
00363 }
00364 }

```

```

00365
00366 /*
00367 #] StartAllThreads :
00368 #[ InitializeOneThread :
00369 */
00373 UBYTE *scratchname[] = { (UBYTE *)"scratchsize",
00374                           (UBYTE *)"scratchsize",
00375                           (UBYTE *)"hidesize" };
00402 ALLPRIVATES *InitializeOneThread(int identity)
00403 {
00404     WORD *t, *ScratchBuf;
00405     int i, j, bsize, *bp;
00406     LONG ScratchSize[3], IOsize;
00407     ALLPRIVATES *B;
00408     UBYTE *s;
00409
00410     wakeup[identity] = 0;
00411     wakeuplocks[identity] = dummylock;
00412     pthread_condattr_init(&(wakeupconditionattributes[identity]));
00413     pthread_condattr_setpshared(&(wakeupconditionattributes[identity]),PTHREAD_PROCESS_PRIVATE);
00414     wakeupconditions[identity] = dummywakeupcondition;
00415     pthread_cond_init(&(wakeupconditions[identity]),&(wakeupconditionattributes[identity]));
00416     wakeupmasterthread[identity] = 0;
00417     wakeupmasterthreadlocks[identity] = dummylock;
00418     wakeupmasterthreadconditions[identity] = dummywakeupcondition;
00419
00420     bsize = sizeof(ALLPRIVATES);
00421     bsize = (bsize+sizeof(int)-1)/sizeof(int);
00422     B = (ALLPRIVATES *)Malloc1(sizeof(int)*bsize,"B struct");
00423     for ( bp = (int *)B, j = 0; j < bsize; j++ ) *bp++ = 0;
00424
00425     AB[identity] = B;
00426 /*
00427     12-jun-2007 JV:
00428     For the timing one has to know a few things:
00429     The POSIX standard is that there is only a single process ID and that
00430     getrusage returns the time of all the threads together.
00431     Under Linux there are two methods though: The older LinuxThreads and NPTL.
00432     LinuxThreads gives each thread its own process id. This makes that we
00433     can time the threads with getrusage, and hence this was done. Under NPTL
00434     this has been 'corrected' and suddenly getruage doesn't work anymore the
00435     way it used to. Now we need
00436     clock_gettime(CLOCK_THREAD_CPUTIME_ID,&timing)
00437     which is declared in <time.h> and we need -lrt extra in the link statement.
00438     (this is at least the case on blade02 at DESY-Zeuthen).
00439     See also the code in tools.c at the routine Timer.
00440     We may still have to include more stuff there.
00441 */
00442     if ( identity > 0 ) TimeCPU(0);
00443
00444 #ifdef WITHSORTBOTS
00445
00446     if ( identity > numberofworkers ) {
00447 /*
00448         Some workspace is needed when we have a PolyFun and we have to add
00449         two terms and the new result is going to be longer than the old result.
00450 */
00451         LONG length = AM.WorkSize*sizeof(WORD)/8+AM.MaxTer*2;
00452         AT.WorkSpace = (WORD *)Malloc1(length,"WorkSpace");
00453         AT.WorkTop = AT.WorkSpace + length/sizeof(WORD);
00454         AT.WorkPointer = AT.WorkSpace;
00455         AT.identity = identity;
00456 /*
00457         The SB struct gets treated in IniSortBlocks.
00458         The SortBotIn variables will be defined DefineSortBotTree.
00459 */
00460         if ( AN.SoScratC == 0 ) {
00461             AN.SoScratC = (UWORD *)Malloc1(2*(AM.MaxTal+2)*sizeof(UWORD),"Scratch in SortBot");
00462         }
00463         AT.SS = (SORTING *)Malloc1(sizeof(SORTING),"dummy sort buffer");
00464         AT.SS->PolyFlag = 0;
00465
00466         AT.comsym[0] = 8;
00467         AT.comsym[1] = SYMBOL;
00468         AT.comsym[2] = 4;
00469         AT.comsym[3] = 0;
00470         AT.comsym[4] = 1;
00471         AT.comsym[5] = 1;
00472         AT.comsym[6] = 1;
00473         AT.comsym[7] = 3;
00474         AT.comnum[0] = 4;
00475         AT.comnum[1] = 1;
00476         AT.comnum[2] = 1;
00477         AT.comnum[3] = 3;
00478         AT.comfun[0] = FUNHEAD+4;
00479         AT.comfun[1] = FUNCTION;
00480         AT.comfun[2] = FUNHEAD;

```

```

00481         AT.comfun[3] = 0;
00482 #if FUNHEAD > 3
00483         for ( i = 4; i <= FUNHEAD; i++ )
00484             AT.comfun[i] = 0;
00485 #endif
00486         AT.comfun[FUNHEAD+1] = 1;
00487         AT.comfun[FUNHEAD+2] = 1;
00488         AT.comfun[FUNHEAD+3] = 3;
00489         AT.comind[0] = 7;
00490         AT.comind[1] = INDEX;
00491         AT.comind[2] = 3;
00492         AT.comind[3] = 0;
00493         AT.comind[4] = 1;
00494         AT.comind[5] = 1;
00495         AT.comind[6] = 3;
00496
00497         AT.inprimelist = -1;
00498         AT.sizeprimelist = 0;
00499         AT.primelist = 0;
00500         AT.bracketinfo = 0;
00501
00502         AR.CompareRoutine = (COMPAREDUMMY) (&Compare1);
00503
00504         AR.sLevel = 0;
00505         AR.wranfia = 0;
00506         AR.wranfcall = 0;
00507         AR.wranfnpair1 = NPAIR1;
00508         AR.wranfnpair2 = NPAIR2;
00509         AN.NumFunSorts = 5;
00510         AN.MaxFunSorts = 5;
00511         AN.SplitScratch = 0;
00512         AN.SplitScratchSize = AN.InScratch = 0;
00513         AN.SplitScratch1 = 0;
00514         AN.SplitScratchSize1 = AN.InScratch1 = 0;
00515
00516         AN.FunSorts = (SORTING **)Malloc1((AN.NumFunSorts+1)*sizeof(SORTING *),"FunSort pointers");
00517         for ( i = 0; i <= AN.NumFunSorts; i++ ) AN.FunSorts[i] = 0;
00518         AN.FunSorts[0] = AT.S0 = AT.SS;
00519         AN.idfunctionflag = 0;
00520         AN.tryterm = 0;
00521
00522         return(B);
00523     }
00524     if ( identity == 0 && AN.SoScratC == 0 ) {
00525         AN.SoScratC = (UWORD *)Malloc1(2*(AM.MaxTal+2)*sizeof(UWORD),"Scratch in SortBot");
00526     }
00527 #endif
00528     AR.CurDum = AM.IndDum;
00529     for ( j = 0; j < 3; j++ ) {
00530         if ( identity == 0 ) {
00531             if ( j == 2 ) {
00532                 ScratchSize[j] = AM.HideSize;
00533             }
00534             else {
00535                 ScratchSize[j] = AM.ScratSize;
00536             }
00537             if ( ScratchSize[j] < 10*AM.MaxTer ) ScratchSize[j] = 10 * AM.MaxTer;
00538         }
00539         else {
00540             /*
00541                 ScratchSize[j] = AM.ScratSize / (numberofthreads-1);
00542                 ScratchSize[j] = ScratchSize[j] / 20;
00543                 if ( ScratchSize[j] < 10*AM.MaxTer ) ScratchSize[j] = 10 * AM.MaxTer;
00544             */
00545             if ( j == 1 ) ScratchSize[j] = AM.ThreadScratOutSize;
00546             else ScratchSize[j] = AM.ThreadScratSize;
00547             if ( ScratchSize[j] < 4*AM.MaxTer ) ScratchSize[j] = 4 * AM.MaxTer;
00548             AR.Fscr[j].name = 0;
00549         }
00550         ScratchSize[j] = ( ScratchSize[j] + 255 ) / 256;
00551         ScratchSize[j] = ScratchSize[j] * 256;
00552         ScratchBuf = (WORD *)Malloc1(ScratchSize[j]*sizeof(WORD),(char *) (scratchname[j]));
00553         AR.Fscr[j].POsize = ScratchSize[j] * sizeof(WORD);
00554         AR.Fscr[j].POfull = AR.Fscr[j].POfill = AR.Fscr[j].PObuffer = ScratchBuf;
00555         AR.Fscr[j].POstop = AR.Fscr[j].PObuffer + ScratchSize[j];
00556         PUTZERO(AR.Fscr[j].POposition);
00557         AR.Fscr[j].pthreadlock = dummylock;
00558         AR.Fscr[j].wPOsize = AR.Fscr[j].POsize;
00559         AR.Fscr[j].wPObuffer = AR.Fscr[j].PObuffer;
00560         AR.Fscr[j].wPOfill = AR.Fscr[j].POfill;
00561         AR.Fscr[j].wPOfull = AR.Fscr[j].POfull;
00562         AR.Fscr[j].wPOstop = AR.Fscr[j].POstop;
00563     }
00564     AR.InInBuf = 0;
00565     AR.InHiBuf = 0;
00566     AR.Fscr[0].handle = -1;
00567     AR.Fscr[1].handle = -1;

```

```

00568     AR.Fscr[2].handle = -1;
00569     AR.FoStage4[0].handle = -1;
00570     AR.FoStage4[1].handle = -1;
00571     IOsize = AM.S0->file.POSize;
00572     #ifdef WITHZLIB
00573     AR.FoStage4[0].ziosize = IOsize;
00574     AR.FoStage4[1].ziosize = IOsize;
00575     AR.FoStage4[0].ziobuffer = 0;
00576     AR.FoStage4[1].ziobuffer = 0;
00577     #endif
00578     AR.FoStage4[0].POsize = ((IOsize+sizeof(WORD)-1)/sizeof(WORD))*sizeof(WORD);
00579     AR.FoStage4[1].POsize = ((IOsize+sizeof(WORD)-1)/sizeof(WORD))*sizeof(WORD);
00580
00581     AR.hidefile = &(AR.Fscr[2]);
00582     AR.StoreData.Handle = -1;
00583     AR.SortType = AC.SortType;
00584
00585     AN.IndDum = AM.IndDum;
00586
00587     if ( identity > 0 ) {
00588         s = (UBYTE *) (FG.fname); i = 0;
00589         while ( *s ) { s++; i++; }
00590         s = (UBYTE *) Malloc1(sizeof(char)*(i+12), "name for Fscr[0] file");
00591         snprintf((char *)s, i+12, "%s.%d", FG.fname, identity);
00592         s[i-3] = 's'; s[i-2] = 'c'; s[i-1] = '0';
00593         AR.Fscr[0].name = (char *)s;
00594         s = (UBYTE *) (FG.fname); i = 0;
00595         while ( *s ) { s++; i++; }
00596         s = (UBYTE *) Malloc1(sizeof(char)*(i+12), "name for Fscr[1] file");
00597         snprintf((char *)s, i+12, "%s.%d", FG.fname, identity);
00598         s[i-3] = 's'; s[i-2] = 'c'; s[i-1] = '1';
00599         AR.Fscr[1].name = (char *)s;
00600     }
00601
00602     AR.CompressBuffer = (WORD *) Malloc1((AM.CompressSize+10)*sizeof(WORD), "compresssize");
00603     AR.ComprTop = AR.CompressBuffer + AM.CompressSize;
00604     AR.CompareRoutine = (COMPAREDUMMY) (&Compare1);
00605     /*
00606     Here we make all allocations for the struct AT
00607     (which is AB[identity].T or B->T with B = AB+identity).
00608     */
00609     AT.WorkSpace = (WORD *) Malloc1(AM.WorkSize*sizeof(WORD), "WorkSpace");
00610     AT.WorkTop = AT.WorkSpace + AM.WorkSize;
00611     AT.WorkPointer = AT.WorkSpace;
00612
00613     AT.Nest = (NESTING) Malloc1((LONG) sizeof(struct NeStIng)*AM.maxFlevels, "functionlevels");
00614     AT.NestStop = AT.Nest + AM.maxFlevels;
00615     AT.NestPoin = AT.Nest;
00616
00617     AT.WildMask = (WORD *) Malloc1((LONG) AM.MaxWildcards*sizeof(WORD), "maxwildcards");
00618
00619     LOCK(availabilitylock);
00620     AT.ebufnum = inicbufs(); /* Buffer for extras during execution */
00621     AT.fbufnum = inicbufs(); /* Buffer for caching in factorization */
00622     AT.allbufnum = inicbufs(); /* Buffer for id,all */
00623     AT.aebufnum = inicbufs(); /* Buffer for id,all */
00624     UNLOCK(availabilitylock);
00625
00626     AT.RepCount = (int *) Malloc1((LONG) ((AM.RepMax+3)*sizeof(int)), "repeat buffers");
00627     AN.RepPoint = AT.RepCount;
00628     AN.polysortflag = 0;
00629     AN.subsubveto = 0;
00630     AN.tryterm = 0;
00631     AT.RepTop = AT.RepCount + AM.RepMax;
00632
00633     AT.WildArgTaken = (WORD *) Malloc1((LONG) AC.WildcardBufferSize*sizeof(WORD)/2
00634         , "argument list names");
00635     AT.WildcardBufferSize = AC.WildcardBufferSize;
00636     AT.previousEfactor = 0;
00637
00638     AT.identity = identity;
00639     AT.LoadBalancing = 0;
00640     /*
00641     Still to do: the SS stuff.
00642         the Fscr[3]
00643         the FoStage4[2]
00644     */
00645     if ( AT.WorkSpace == 0 ||
00646         AT.Nest == 0 ||
00647         AT.WildMask == 0 ||
00648         AT.RepCount == 0 ||
00649         AT.WildArgTaken == 0 ) goto OnError;
00650     /*
00651     And initializations
00652     */
00653     AT.comsym[0] = 8;
00654     AT.comsym[1] = SYMBOL;

```

```

00655     AT.comsym[2] = 4;
00656     AT.comsym[3] = 0;
00657     AT.comsym[4] = 1;
00658     AT.comsym[5] = 1;
00659     AT.comsym[6] = 1;
00660     AT.comsym[7] = 3;
00661     AT.comnum[0] = 4;
00662     AT.comnum[1] = 1;
00663     AT.comnum[2] = 1;
00664     AT.comnum[3] = 3;
00665     AT.comfun[0] = FUNHEAD+4;
00666     AT.comfun[1] = FUNCTION;
00667     AT.comfun[2] = FUNHEAD;
00668     AT.comfun[3] = 0;
00669     #if FUNHEAD > 3
00670         for ( i = 4; i <= FUNHEAD; i++ )
00671             AT.comfun[i] = 0;
00672     #endif
00673     AT.comfun[FUNHEAD+1] = 1;
00674     AT.comfun[FUNHEAD+2] = 1;
00675     AT.comfun[FUNHEAD+3] = 3;
00676     AT.comind[0] = 7;
00677     AT.comind[1] = INDEX;
00678     AT.comind[2] = 3;
00679     AT.comind[3] = 0;
00680     AT.comind[4] = 1;
00681     AT.comind[5] = 1;
00682     AT.comind[6] = 3;
00683     AT.locwildvalue[0] = SUBEXPRESSION;
00684     AT.locwildvalue[1] = SUBEXPSIZE;
00685     for ( i = 2; i < SUBEXPSIZE; i++ ) AT.locwildvalue[i] = 0;
00686     AT.mulpat[0] = TYPMULT;
00687     AT.mulpat[1] = SUBEXPSIZE+3;
00688     AT.mulpat[2] = 0;
00689     AT.mulpat[3] = SUBEXPRESSION;
00690     AT.mulpat[4] = SUBEXPSIZE;
00691     AT.mulpat[5] = 0;
00692     AT.mulpat[6] = 1;
00693     for ( i = 7; i < SUBEXPSIZE+5; i++ ) AT.mulpat[i] = 0;
00694     AT.proexp[0] = SUBEXPSIZE+4;
00695     AT.proexp[1] = EXPRESSION;
00696     AT.proexp[2] = SUBEXPSIZE;
00697     AT.proexp[3] = -1;
00698     AT.proexp[4] = 1;
00699     for ( i = 5; i < SUBEXPSIZE+1; i++ ) AT.proexp[i] = 0;
00700     AT.proexp[SUBEXPSIZE+1] = 1;
00701     AT.proexp[SUBEXPSIZE+2] = 1;
00702     AT.proexp[SUBEXPSIZE+3] = 3;
00703     AT.proexp[SUBEXPSIZE+4] = 0;
00704     AT.dummysubexp[0] = SUBEXPRESSION;
00705     AT.dummysubexp[1] = SUBEXPSIZE+4;
00706     for ( i = 2; i < SUBEXPSIZE; i++ ) AT.dummysubexp[i] = 0;
00707     AT.dummysubexp[SUBEXPSIZE] = WILDDUMMY;
00708     AT.dummysubexp[SUBEXPSIZE+1] = 4;
00709     AT.dummysubexp[SUBEXPSIZE+2] = 0;
00710     AT.dummysubexp[SUBEXPSIZE+3] = 0;
00711
00712     AT.MinVecArg[0] = 7+ARGHEAD;
00713     AT.MinVecArg[ARGHEAD] = 7;
00714     AT.MinVecArg[1+ARGHEAD] = INDEX;
00715     AT.MinVecArg[2+ARGHEAD] = 3;
00716     AT.MinVecArg[3+ARGHEAD] = 0;
00717     AT.MinVecArg[4+ARGHEAD] = 1;
00718     AT.MinVecArg[5+ARGHEAD] = 1;
00719     AT.MinVecArg[6+ARGHEAD] = -3;
00720     t = AT.FuncArg;
00721     *t++ = 4+ARGHEAD+FUNHEAD;
00722     for ( i = 1; i < ARGHEAD; i++ ) *t++ = 0;
00723     *t++ = 4+FUNHEAD;
00724     *t++ = 0;
00725     *t++ = FUNHEAD;
00726     for ( i = 2; i < FUNHEAD; i++ ) *t++ = 0;
00727     *t++ = 1; *t++ = 1; *t++ = 3;
00728
00729     AT.inprimelist = -1;
00730     AT.sizeprimelist = 0;
00731     AT.primelist = 0;
00732     AT.nfac = AT.nBer = 0;
00733     AT.factorials = 0;
00734     AT.bernoullis = 0;
00735     AR.wranfia = 0;
00736     AR.wranfcall = 0;
00737     AR.wranfnpair1 = NPAIR1;
00738     AR.wranfnpair2 = NPAIR2;
00739     AR.wranfseed = 0;
00740     AN.SplitScratch = 0;
00741     AN.SplitScratchSize = AN.InScratch = 0;

```

```

00742     AN.SplitScratch1 = 0;
00743     AN.SplitScratchSize1 = AN.InScratch1 = 0;
00744     /*
00745     Now the sort buffers. They depend on which thread. The master
00746     inherits the sortbuffer from AM.S0
00747     */
00748     if ( identity == 0 ) {
00749         AT.S0 = AM.S0;
00750     }
00751     else {
00752     /*
00753         For the moment we don't have special settings.
00754         They may become costly in virtual memory.
00755     */
00756         AT.S0 = AllocSort(AM.S0->LargeSize*sizeof(WORD)/numberofworkers
00757                           ,AM.S0->SmallSize*sizeof(WORD)/numberofworkers
00758                           ,AM.S0->SmallEsize*sizeof(WORD)/numberofworkers
00759                           ,AM.S0->TermsInSmall
00760                           ,AM.S0->MaxPatches
00761                           ,AM.S0->MaxPatches/numberofworkers /*
00762                           ,AM.S0->MaxFpatches/numberofworkers
00763                           ,AM.S0->file.POsize
00764                           ,0);
00765     }
00766     AR.CompressPointer = AR.CompressBuffer;
00767     /*
00768     Install the store caches (15-aug-2006 JV)
00769     */
00770     AT.StoreCache = AT.StoreCacheAlloc = 0;
00771     if ( AM.NumStoreCaches > 0 ) {
00772         STORECACHE sa, sb;
00773         LONG size;
00774         size = sizeof(struct StOrEcAcHe)+AM.SizeStoreCache;
00775         size = ((size-1)/sizeof(size_t)+1)*sizeof(size_t);
00776         AT.StoreCacheAlloc = (STORECACHE)Malloc1(size*AM.NumStoreCaches,"StoreCaches");
00777         sa = AT.StoreCache = AT.StoreCacheAlloc;
00778         for ( i = 0; i < AM.NumStoreCaches; i++ ) {
00779             sb = (STORECACHE)(void *)((UBYTE *)sa+size);
00780             if ( i == AM.NumStoreCaches-1 ) {
00781                 sa->next = 0;
00782             }
00783             else {
00784                 sa->next = sb;
00785             }
00786             SETBASEPOSITION(sa->position,-1);
00787             SETBASEPOSITION(sa->toppos,-1);
00788             sa = sb;
00789         }
00790     }
00791
00792     ReserveTempFiles(2);
00793     return(B);
00794 OnError:;
00795     MLOCK(ErrorMessageLock);
00796     MesPrint("Error initializing thread %d",identity);
00797     MUNLOCK(ErrorMessageLock);
00798     Terminate(-1);
00799     return(B);
00800 }
00801
00802 /*
00803     #] InitializeOneThread :
00804     #[ FinalizeOneThread :
00805     */
00806 void FinalizeOneThread(int identity)
00807 {
00808     timerinfo[identity] = TimeCPU(1);
00809 }
00810
00811 /*
00812     #] FinalizeOneThread :
00813     #[ ClearAllThreads :
00814     */
00815 void ClearAllThreads(void)
00816 {
00817     int i;
00818     MasterWaitAll();
00819     for ( i = 1; i <= numberofworkers; i++ ) {
00820         WakeupThread(i,CLEARCLOCK);
00821     }
00822 #ifdef WITHSORTBOTS
00823     for ( i = numberofworkers+1; i <= numberofworkers+numberofsortbots; i++ ) {
00824         WakeupThread(i,CLEARCLOCK);
00825     }
00826 #endif
00827 }
00828
00829
00830

```

```

00845 /*
00846     #] ClearAllThreads :
00847     #[ TerminateAllThreads :
00848 */
00855 void TerminateAllThreads(void)
00856 {
00857     int i;
00858     for ( i = 1; i <= numberofworkers; i++ ) {
00859         GetThread(i);
00860         WakeupThread(i, TERMINATETHREAD);
00861     }
00862 #ifdef WITHSORTBOTS
00863     for ( i = numberofworkers+1; i <= numberofworkers+numberofsortbots; i++ ) {
00864         WakeupThread(i, TERMINATETHREAD);
00865     }
00866 #endif
00867     for ( i = 1; i <= numberofworkers; i++ ) {
00868         pthread_join(threadpointers[i], NULL);
00869     }
00870 #ifdef WITHSORTBOTS
00871     for ( i = numberofworkers+1; i <= numberofworkers+numberofsortbots; i++ ) {
00872         pthread_join(threadpointers[i], NULL);
00873     }
00874 #endif
00875 }
00876
00877 /*
00878     #] TerminateAllThreads :
00879     #[ MakeThreadBuckets :
00880 */
00906 int MakeThreadBuckets(int number, int par)
00907 {
00908     int i;
00909     LONG sizethreadbuckets;
00910     THREADBUCKET *thr;
00911 /*
00912     First we need a decent estimate. Not all terms should be maximal.
00913     Note that AM.MaxTer is in bytes!!!
00914     Maybe we should try to limit the size here a bit more effectively.
00915     This is a great consumer of memory.
00916 */
00917     sizethreadbuckets = ( AC.ThreadBucketSize + 1 ) * AM.MaxTer + 2*sizeof(WORD);
00918     if ( AC.ThreadBucketSize >= 250 )         sizethreadbuckets /= 4;
00919     else if ( AC.ThreadBucketSize >= 90 )     sizethreadbuckets /= 3;
00920     else if ( AC.ThreadBucketSize >= 40 )     sizethreadbuckets /= 2;
00921     sizethreadbuckets /= sizeof(WORD);
00922
00923     if ( par == 0 ) {
00924         numthreadbuckets = 2*(number-1);
00925         threadbuckets = (THREADBUCKET **)Mallc1(numthreadbuckets*sizeof(THREADBUCKET
00926 *), "threadbuckets");
00927         freebuckets = (THREADBUCKET **)Mallc1(numthreadbuckets*sizeof(THREADBUCKET
00928 *), "threadbuckets");
00929     }
00930     if ( par > 0 ) {
00931         if ( sizethreadbuckets <= threadbuckets[0]->threadbuffersize ) return(0);
00932         for ( i = 0; i < numthreadbuckets; i++ ) {
00933             thr = threadbuckets[i];
00934             M_free(thr->deferbuffer, "deferbuffer");
00935         }
00936     }
00937     else {
00938         for ( i = 0; i < numthreadbuckets; i++ ) {
00939             threadbuckets[i] = (THREADBUCKET *)Mallc1(sizeof(THREADBUCKET), "threadbuckets");
00940             threadbuckets[i]->lock = dummylock;
00941         }
00942     }
00943     for ( i = 0; i < numthreadbuckets; i++ ) {
00944         thr = threadbuckets[i];
00945         thr->threadbuffersize = sizethreadbuckets;
00946         thr->free = BUCKETFREE;
00947         thr->deferbuffer = (POSITION *)Mallc1(2*sizethreadbuckets*sizeof(WORD)
00948 + (AC.ThreadBucketSize+1)*sizeof(POSITION), "deferbuffer");
00949         thr->threadbuffer = (WORD *) (thr->deferbuffer+AC.ThreadBucketSize+1);
00950         thr->compressbuffer = (WORD *) (thr->threadbuffer+sizethreadbuckets);
00951         thr->busy = BUCKETPREPARINGTERM;
00952         thr->usenum = thr->totnum = 0;
00953         thr->type = BUCKETDOINGTERMS;
00954     }
00955     return(0);
00956 }
00957 /*
00958     #] MakeThreadBuckets :
00959     #[ GetTimerInfo :
00960 */

```



```

00966 int GetTimerInfo(LONG** ti, LONG** sti)
00967 {
00968     *ti = timerinfo;
00969     *sti = sumtimerinfo;
00970 #ifdef WITHSORTBOTS
00971     return AM.totalnumberofthreads*2;
00972 #else
00973     return AM.totalnumberofthreads;
00974 #endif
00975 }
00976
00977 /*
00978 #] GetTimerInfo :
00979 #[ WriteTimerInfo :
00980 */
00981
00982 void WriteTimerInfo(LONG* ti, LONG* sti)
00983 {
00984     int i;
00985 #ifdef WITHSORTBOTS
00986     int max = AM.totalnumberofthreads*2;
00987 #else
00988     int max = AM.totalnumberofthreads;
00989 #endif
00990     for ( i=0; i<max; ++i ) {
00991         timerinfo[i] = ti[i];
00992         sumtimerinfo[i] = sti[i];
00993     }
00994 }
00995
01000 /*
01001 #] WriteTimerInfo :
01002 #[ GetWorkerTimes :
01003 */
01004
01005 LONG GetWorkerTimes(void)
01006 {
01007     LONG retval = 0;
01008     int i;
01009     for ( i = 1; i <= numberofworkers; i++ ) retval += timerinfo[i] + sumtimerinfo[i];
01010 #ifdef WITHSORTBOTS
01011     for ( i = numberofworkers+1; i <= numberofworkers+numberofsortbots; i++ )
01012         retval += timerinfo[i] + sumtimerinfo[i];
01013 #endif
01014     return(retval);
01015 }
01016
01021 /*
01022 #] GetWorkerTimes :
01023 #[ UpdateOneThread :
01024 */
01025
01026 int UpdateOneThread(int identity)
01027 {
01028     ALLPRIVATES *B = AB[identity], *B0 = AB[0];
01029     AR.GetFile = AR0.GetFile;
01030     AR.KeptInHold = AR0.KeptInHold;
01031     AR.CurExpr = AR0.CurExpr;
01032     AR.SortType = AC.SortType;
01033     if ( AT.WildcardBufferSize < AC.WildcardBufferSize ) {
01034         M_free(AT.WildArgTaken, "argument list names");
01035         AT.WildcardBufferSize = AC.WildcardBufferSize;
01036         AT.WildArgTaken = (WORD *)Malloc1((LONG)AC.WildcardBufferSize*sizeof(WORD)/2
01037             , "argument list names");
01038         if ( AT.WildArgTaken == 0 ) return(-1);
01039     }
01040     return(0);
01041 }
01042
01048 /*
01049 #] UpdateOneThread :
01050 #[ LoadOneThread :
01051 */
01052
01053 int LoadOneThread(int from, int identity, THREADBUCKET *thr, int par)
01054 {
01055     WORD *t1, *t2;
01056     ALLPRIVATES *B = AB[identity], *B0 = AB[from];
01057
01058     AR.DefPosition = AR0.DefPosition;
01059     AR.NoCompress = AR0.NoCompress;
01060     AR.gzipCompress = AR0.gzipCompress;
01061     AR.BraceOn = AR0.BraceOn;
01062     AR.CurDum = AR0.CurDum;
01063     AR.DeferFlag = AR0.DeferFlag;
01064     AR.TePos = 0;
01065     AR.sLevel = AR0.sLevel;
01066     AR.Stage4Name = AR0.Stage4Name;
01067     AR.GetOneFile = AR0.GetOneFile;
01068     AR.PolyFun = AR0.PolyFun;

```

```

01081     AR.PolyFunInv = AR0.PolyFunInv;
01082     AR.PolyFunType = AR0.PolyFunType;
01083     AR.PolyFunExp = AR0.PolyFunExp;
01084     AR.PolyFunVar = AR0.PolyFunVar;
01085     AR.PolyFunPow = AR0.PolyFunPow;
01086     AR.Eside = AR0.Eside;
01087     AR.Cnumlhs = AR0.Cnumlhs;
01088 /*
01089     AR.MaxBracket = AR0.MaxBracket;
01090
01091     The compressbuffer contents are mainly relevant for keep brackets
01092     We should do this only if there is a keep brackets statement
01093     We may however still need the compressbuffer for expressions in the rhs.
01094 */
01095     if ( par >= 1 ) {
01096 /*
01097         We may not need this %%%% 7-apr-2006
01098 */
01099         t1 = AR.CompressBuffer; t2 = AR0.CompressBuffer;
01100         while ( t2 < AR0.CompressPointer ) *t1++ = *t2++;
01101         AR.CompressPointer = t1;
01102     }
01103     else {
01104         AR.CompressPointer = AR.CompressBuffer;
01105     }
01106     if ( AR.DeferFlag ) {
01107         if ( AR.infile->handle < 0 ) {
01108             AR.infile->POfill = AR0.infile->POfill;
01109         }
01110         else {
01111 /*
01112             We have to set the value of PPosition to something that will
01113             force a read in the first try.
01114 */
01115             AR.infile->POfull = AR.infile->POfill = AR.infile->PObuffer;
01116         }
01117     }
01118     if ( par == 0 ) {
01119         AN.threadbuck = thr;
01120         AN.ninterms = thr->firstterm;
01121     }
01122     else if ( par == 1 ) {
01123         WORD *tstop;
01124         t1 = thr->threadbuffer; tstop = t1 + *t1;
01125         t2 = AT.WorkPointer;
01126         while ( t1 < tstop ) *t2++ = *t1++;
01127         AN.ninterms = thr->firstterm;
01128     }
01129     AN.TeInFun = 0;
01130     AN.ncmod = AC.ncmod;
01131     AT.BrackBuf = AT0.BrackBuf;
01132     AT.bracketindexflag = AT0.bracketindexflag;
01133     AN.PolyFunTodo = 0;
01134 /*
01135     The relevant variables and the term are in their place.
01136     There is nothing more to do.
01137 */
01138     return(0);
01139 }
01140
01141 /*
01142 #] LoadOneThread :
01143 #[ BalanceRunThread :
01144 */
01145 int BalanceRunThread(PHEAD int identity, WORD *term, WORD level)
01146 {
01147     GETBIDENTITY
01148     ALLPRIVATES *BB;
01149     WORD *t, *tti;
01150     int i, *ti, *tti;
01151
01152     LoadOneThread(AT.identity,identity,0,2);
01153 /*
01154     Extra loading if needed. Quantities changed in Generator.
01155     Like the level that has to be passed.
01156 */
01157     BB = AB[identity];
01158     BB->R.level = level;
01159     BB->T.TMbuff = AT.TMbuff;
01160     ti = AT.RepCount; tti = BB->T.RepCount;
01161     i = AN.RepPoint - AT.RepCount;
01162     BB->N.RepPoint = BB->T.RepCount + i;
01163     for ( ; i >= 0; i-- ) tti[i] = ti[i];
01164
01165     t = term; i = *term;
01166     tt = BB->T.WorkSpace;

```

```

01183     NCOPY(tt,t,i);
01184     BB->T.WorkPointer = tt;
01185
01186     WakeupThread(identity,HIGHERLEVELGENERATION);
01187
01188     return(0);
01189 }
01190
01191 /*
01192  #] BalanceRunThread :
01193  #[ SetWorkerFiles :
01194  */
01195 void SetWorkerFiles(void)
01200 {
01201     int id;
01202     ALLPRIVATES *B, *B0 = AB[0];
01203     for ( id = 1; id < AM.totalnumberofthreads; id++ ) {
01204         B = AB[id];
01205         AR.infile = &(AR.Fscr[0]);
01206         AR.outfile = &(AR.Fscr[1]);
01207         AR.hidefile = &(AR.Fscr[2]);
01208         AR.infile->handle = AR0.infile->handle;
01209         AR.hidefile->handle = AR0.hidefile->handle;
01210         if ( AR.infile->handle < 0 ) {
01211             AR.infile->PObuffer = AR0.infile->PObuffer;
01212             AR.infile->POnstop = AR0.infile->POnstop;
01213             AR.infile->POfill = AR0.infile->POfill;
01214             AR.infile->POfull = AR0.infile->POfull;
01215             AR.infile->POsize = AR0.infile->POsize;
01216             AR.InInBuf = AR0.InInBuf;
01217             AR.infile->POposition = AR0.infile->POposition;
01218             AR.infile->filesize = AR0.infile->filesize;
01219         }
01220         else {
01221             AR.infile->PObuffer = AR.infile->wPObuffer;
01222             AR.infile->POnstop = AR.infile->wPOnstop;
01223             AR.infile->POfill = AR.infile->wPOfill;
01224             AR.infile->POfull = AR.infile->wPOfull;
01225             AR.infile->POsize = AR.infile->wPOsize;
01226             AR.InInBuf = 0;
01227             PUTZERO(AR.infile->POposition);
01228         }
01229     /*
01230     If there is some writing, it better happens to ones own outfile.
01231     Currently this is to be done only for InParallel.
01232     Merging of the outputs is then done by the CopyExpression routine.
01233     */
01234     {
01235         AR.outfile->PObuffer = AR.outfile->wPObuffer;
01236         AR.outfile->POnstop = AR.outfile->wPOnstop;
01237         AR.outfile->POfill = AR.outfile->wPOfill;
01238         AR.outfile->POfull = AR.outfile->wPOfull;
01239         AR.outfile->POsize = AR.outfile->wPOsize;
01240         PUTZERO(AR.outfile->POposition);
01241     }
01242     if ( AR.hidefile->handle < 0 ) {
01243         AR.hidefile->PObuffer = AR0.hidefile->PObuffer;
01244         AR.hidefile->POnstop = AR0.hidefile->POnstop;
01245         AR.hidefile->POfill = AR0.hidefile->POfill;
01246         AR.hidefile->POfull = AR0.hidefile->POfull;
01247         AR.hidefile->POsize = AR0.hidefile->POsize;
01248         AR.InHiBuf = AR0.InHiBuf;
01249         AR.hidefile->POposition = AR0.hidefile->POposition;
01250         AR.hidefile->filesize = AR0.hidefile->filesize;
01251     }
01252     else {
01253         AR.hidefile->PObuffer = AR.hidefile->wPObuffer;
01254         AR.hidefile->POnstop = AR.hidefile->wPOnstop;
01255         AR.hidefile->POfill = AR.hidefile->wPOfill;
01256         AR.hidefile->POfull = AR.hidefile->wPOfull;
01257         AR.hidefile->POsize = AR.hidefile->wPOsize;
01258         AR.InHiBuf = 0;
01259         PUTZERO(AR.hidefile->POposition);
01260     }
01261 }
01262 if ( AR0.StoreData.dirtyflag ) {
01263     for ( id = 1; id < AM.totalnumberofthreads; id++ ) {
01264         B = AB[id];
01265         AR.StoreData = AR0.StoreData;
01266     }
01267 }
01268 }
01269
01270 /*
01271  #] SetWorkerFiles :
01272  #[ RunThread :
01273  */

```

```

01281 void *RunThread(void *dummy)
01282 {
01283     WORD *term, *ttin, *tt, *ttco, *oldwork;
01284     int identity, wakeupsignal, identityretv, i, toberelaxed, errorcode;
01285     ALLPRIVATES *B;
01286     THREADBUCKET *thr;
01287     POSITION *ppdef;
01288     EXPRESSIONS e;
01289     DUMMYUSE(dummy);
01290     identity = SetIdentity(&identityretv);
01291     threadpointers[identity] = pthread_self();
01292     B = InitializeOneThread(identity);
01293     while ( ( wakeupsignal = ThreadWait(identity) ) > 0 ) {
01294         switch ( wakeupsignal ) {
01295             /*
01296              *      #[ STARTNEWEXPRESSION :
01297              */
01298             case STARTNEWEXPRESSION:
01299                 /*
01300                  *      Set up the sort routines etc.
01301                  *      Start with getting some buffers synchronized with the compiler
01302                  */
01303                 if ( UpdateOneThread(identity) ) {
01304                     MLOCK(ErrorMessageLock);
01305                     MesPrint("Update error in starting expression in thread %d in module
01306                             %d",identity,AC.CModule);
01307                     MUNLOCK(ErrorMessageLock);
01308                     Terminate(-1);
01309                 }
01310                 AR.DeferFlag = AC.ComDefer;
01311                 AR.sLevel = AS.sLevel;
01312                 AR.MaxDum = AM.IndDum;
01313                 AR.expchanged = AB[0]->R.expchanged;
01314                 AR.expflags = AB[0]->R.expflags;
01315                 AR.PolyFun = AB[0]->R.PolyFun;
01316                 AR.PolyFunInv = AB[0]->R.PolyFunInv;
01317                 AR.PolyFunType = AB[0]->R.PolyFunType;
01318                 AR.PolyFunExp = AB[0]->R.PolyFunExp;
01319                 AR.PolyFunVar = AB[0]->R.PolyFunVar;
01320                 AR.PolyFunPow = AB[0]->R.PolyFunPow;
01321                 /*
01322                  *      Now fire up the sort buffer.
01323                  */
01324                 NewSort (BHEAD0);
01325                 break;
01326             /*
01327              *      #[ STARTNEWEXPRESSION :
01328              *      #[ LOWESTLEVELGENERATION :
01329              */
01330             case LOWESTLEVELGENERATION:
01331                 #ifdef INNERTEST
01332                 if ( AC.InnerTest ) {
01333                     if ( StrCmp(AC.TestValue,(UBYTE *)INNERTEST) == 0 ) {
01334                         MesPrint("Testing(Worker%d): value = %s",AT.identity,AC.TestValue);
01335                     }
01336                 }
01337                 #endif
01338                 e = Expressions + AR.CurExpr;
01339                 thr = AN.threadbuck;
01340                 ppdef = thr->deferbuffer;
01341                 ttin = thr->threadbuffer;
01342                 ttco = thr->compressbuffer;
01343                 term = AT.WorkPointer;
01344                 thr->usenum = 0;
01345                 toberelaxed = 0;
01346                 AN.inputnumber = thr->firstterm;
01347                 AN.ninterms = thr->firstterm;
01348                 do {
01349                     thr->usenum++; /* For if the master wants to steal the bucket */
01350                     tt = term; i = *ttin;
01351                     NCOPY(tt,ttin,i);
01352                     AT.WorkPointer = tt;
01353                     if ( AR.DeferFlag ) {
01354                         tt = AR.CompressBuffer; i = *ttco;
01355                         NCOPY(tt,ttco,i);
01356                         AR.CompressPointer = tt;
01357                         AR.DefPosition = ppdef[0]; ppdef++;
01358                     }
01359                     if ( thr->free == BUCKETTERMINATED ) {
01360                         /*
01361                          *      The next statement allows the master to steal the bucket
01362                          *      for load balancing purposes. We do still execute the current
01363                          *      term, but afterwards we drop out.
01364                          *      Once we have written the release code, we cannot use this
01365                          *      bucket anymore. Hence the exit to the label bucketstolen.
01366                          */
01367                         if ( thr->usenum == thr->totnum ) {

```

```

01367         thr->free = BUCKETCOMINGFREE;
01368     }
01369     else {
01370         thr->free = BUCKETRELEASED;
01371         tobereleased = 1;
01372     }
01373 }
01374 /*
01375     What if we want to steal and we set thr->free while
01376     the thread is inside the next code for a long time?
01377 if ( AT.LoadBalancing ) {
01378 */
01379     LOCK(thr->lock);
01380     thr->busy = BUCKETDOINGTERM;
01381     UNLOCK(thr->lock);
01382 /*
01383     }
01384     else {
01385         thr->busy = BUCKETDOINGTERM;
01386     }
01387 */
01388     AN.RepPoint = AT.RepCount + 1;
01389
01390     if ( ( e->vflags & ISFACTORIZED ) != 0 && term[1] == HAAKJE ) {
01391         StoreTerm(BHEAD term);
01392     }
01393     else {
01394         if ( AR.DeferFlag ) {
01395             AR.CurDum = AN.IndDum = Expressions[AR.CurExpr].numdummies + AM.IndDum;
01396         }
01397         else {
01398             AN.IndDum = AM.IndDum;
01399             AR.CurDum = ReNumber(BHEAD term);
01400         }
01401         if ( AC.SymChangeFlag ) MarkDirty(term,DIRTYSYMFLAG);
01402         if ( AN.ncmod ) {
01403             if ( ( AC.modmode & ALSOFUNARGS ) != 0 ) MarkDirty(term,DIRTYFLAG);
01404             else if ( AR.PolyFun ) PolyFunDirty(BHEAD term);
01405         }
01406         else if ( AC.PolyRatFunChanged ) PolyFunDirty(BHEAD term);
01407         if ( ( AP.PreDebug & THREADSDEBUG ) != 0 ) {
01408             MLOCK(ErrorMessageLock);
01409             MesPrint("Thread %w executing term:");
01410             PrintTerm(term,"LLG");
01411             MUNLOCK(ErrorMessageLock);
01412         }
01413         if ( ( AR.PolyFunType == 2 ) && ( AC.PolyRatFunChanged == 0 )
01414             && ( e->status == LOCALEXPRESSION || e->status == GLOBALEXPRESSION ) ) {
01415             PolyFunClean(BHEAD term);
01416         }
01417         if ( Generator(BHEAD term,0) ) {
01418             LowerSortLevel();
01419             MLOCK(ErrorMessageLock);
01420             MesPrint("Error in processing one term in thread %d in module
01421 %d",identity,AC.CModule);
01422             MUNLOCK(ErrorMessageLock);
01423             Terminate(-1);
01424         }
01425         AN.ninterms++;
01426     }
01427     if ( AT.LoadBalancing ) { */
01428         LOCK(thr->lock);
01429         thr->busy = BUCKETPREPARINGTERM;
01430         UNLOCK(thr->lock);
01431     }
01432     else {
01433         thr->busy = BUCKETPREPARINGTERM;
01434     }
01435 */
01436     if ( thr->free == BUCKETTERMINATED ) {
01437         if ( thr->usenum == thr->totnum ) {
01438             thr->free = BUCKETCOMINGFREE;
01439         }
01440         else {
01441             thr->free = BUCKETRELEASED;
01442             tobereleased = 1;
01443         }
01444     }
01445     if ( tobereleased ) goto bucketstolen;
01446 } while ( *ttin );
01447 thr->free = BUCKETCOMINGFREE;
01448 bucketstolen;;
01449 /*
01450     if ( AT.LoadBalancing ) { */
01451         LOCK(thr->lock);
01452         thr->busy = BUCKETTOBERELEASED;
01453         UNLOCK(thr->lock);

```

```

01453 /*          }
01454             else {
01455                 thr->busy = BUCKETTOBERELEASED;
01456             }
01457 */
01458         AT.WorkPointer = term;
01459         break;
01460 /*
01461         #] LOWESTLEVELGENERATION :
01462         #[ FINISHEXPRESSION :
01463 */
01464 #ifdef WITHSORTBOTS
01465         case CLAIMOUTPUT:
01466             LOCK(AT.SB.MasterBlockLock[1]);
01467             break;
01468 #endif
01469         case FINISHEXPRESSION:
01470 /*
01471             Finish the sort
01472
01473             Start with claiming the first block
01474             Once we have claimed it we can let the master know that
01475             everything is all right.
01476 */
01477             LOCK(AT.SB.MasterBlockLock[1]);
01478             ThreadClaimedBlock(identity);
01479 /*
01480             Entry for when we work with sortbots
01481 */
01482 #ifdef WITHSORTBOTS
01483             /* fall through */
01484             case FINISHEXPRESSION2:
01485 #endif
01486 /*
01487             Now we may need here an fsync on the sort file
01488 */
01489             if ( AC.ThreadSortFileSynch ) {
01490                 if ( AT.S0->file.handle >= 0 ) {
01491                     SynchFile(AT.S0->file.handle);
01492                 }
01493             }
01494             AT.SB.FillBlock = 1;
01495             AT.SB.MasterFill[1] = AT.SB.MasterStart[1];
01496             errorcode = EndSort(BHEAD AT.S0->sBuffer,0);
01497             UNLOCK(AT.SB.MasterBlockLock[AT.SB.FillBlock]);
01498             UpdateMaxSize();
01499             if ( errorcode ) {
01500                 MLOCK(ErrorMessageLock);
01501                 MesPrint("Error terminating sort in thread %d in module %d",identity,AC.CModule);
01502                 MUNLOCK(ErrorMessageLock);
01503                 Terminate(-1);
01504             }
01505             break;
01506 /*
01507             #] FINISHEXPRESSION :
01508             #[ CLEANUPEXPRESSION :
01509 */
01510         case CLEANUPEXPRESSION:
01511 /*
01512             Cleanup everything and wait for the next expression
01513 */
01514             if ( AR.outfile->handle >= 0 ) {
01515                 CloseFile(AR.outfile->handle);
01516                 AR.outfile->handle = -1;
01517                 remove(AR.outfile->name);
01518                 AR.outfile->POfill = AR.outfile->POfull = AR.outfile->PObuffer;
01519                 PUTZERO(AR.outfile->POposition);
01520                 PUTZERO(AR.outfile->filesize);
01521             }
01522             else {
01523                 AR.outfile->POfill = AR.outfile->POfull = AR.outfile->PObuffer;
01524                 PUTZERO(AR.outfile->POposition);
01525                 PUTZERO(AR.outfile->filesize);
01526             }
01527             {
01528                 CBUF *C = cbuf+AT.ebufnum;
01529                 WORD **w, ii;
01530                 if ( C->numrhs > 0 || C->numlhs > 0 ) {
01531                     if ( C->rhs ) {
01532                         w = C->rhs; ii = C->numrhs;
01533                         do { *w++ = 0; } while ( --ii > 0 );
01534                     }
01535                     if ( C->lhs ) {
01536                         w = C->lhs; ii = C->numlhs;
01537                         do { *w++ = 0; } while ( --ii > 0 );
01538                     }
01539                     C->numlhs = C->numrhs = 0;

```

```

01540             ClearTree(AT.ebufnum);
01541             C->Pointer = C->Buffer;
01542         }
01543     }
01544     break;
01545 /*
01546     #] CLEANUPEXPRESSION :
01547     #[ HIGHERLEVELGENERATION :
01548 */
01549     case HIGHERLEVELGENERATION:
01550 /*
01551         When foliating halfway the tree.
01552         This should only be needed in a second level load balancing
01553 */
01554         term = AT.WorkSpace; AT.WorkPointer = term + *term;
01555         if ( Generator(BHEAD term,AR.level) ) {
01556             LowerSortLevel();
01557             MLOCK(ErrorMessageLock);
01558             MesPrint("Error in load balancing one term at level %d in thread %d in module
01559 %d",AR.level,AT.identity,AC.CModule);
01559             MUNLOCK(ErrorMessageLock);
01560             Terminate(-1);
01561         }
01562         AT.WorkPointer = term;
01563         break;
01564 /*
01565     #] HIGHERLEVELGENERATION :
01566     #[ STARTNEWMODULE :
01567 */
01568     case STARTNEWMODULE:
01569 /*
01570         For resetting variables.
01571 */
01572         SpecialCleanup(B);
01573         break;
01574 /*
01575     #] STARTNEWMODULE :
01576     #[ TERMINATETHREAD :
01577 */
01578     case TERMINATETHREAD:
01579         goto EndOfThread;
01580 /*
01581     #] TERMINATETHREAD :
01582     #[ DOONEEXPRESSION :
01583
01584         When a thread has to do a complete (not too big) expression.
01585         The number of the expression to be done is in AR.exprtodo.
01586         The code is mostly taken from Processor. The only difference
01587         is with what to do with the output.
01588         The output should go to the scratch buffer of the worker
01589         (which is free at the right moment). If this buffer is too
01590         small we have a problem. We could write to file or give the
01591         master what we have and from now on the master has to collect
01592         pieces until things are complete.
01593         Note: this assumes that the expressions don't keep their order.
01594         If they have to keep their order, don't use this feature.
01595 */
01596     case DOONEEXPRESSION: {
01597         POSITION position, outposition;
01598         FILEHANDLE *fi, *fout, *oldoutfile;
01599         LONG dd = 0;
01600         WORD oldBracketOn = AR.BracketOn;
01601         WORD *oldBrackBuf = AT.BrackBuf;
01602         WORD oldbracketindexflag = AT.bracketindexflag;
01603         WORD fromspectator = 0;
01604         e = Expressions + AR.exprtodo;
01605         i = AR.exprtodo;
01606         AR.CurExpr = i;
01607         AR.SortType = AC.SortType;
01608         AR.expchanged = 0;
01609         if ( ( e->vflags & ISFACTORIZED ) != 0 ) {
01610             AR.BracketOn = 1;
01611             AT.BrackBuf = AM.BracketFactors;
01612             AT.bracketindexflag = 1;
01613         }
01614         position = AS.OldOnFile[i];
01615         if ( e->status == HIDDENLEXPRESSION || e->status == HIDDENEXPRESSION ) {
01616             AR.GetFile = 2; fi = AR.hidefile;
01617         }
01618         else {
01619             AR.GetFile = 0; fi = AR.infile;
01620         }
01621         PUTZERO(fi->PPosition);
01622         if ( fi->handle >= 0 ) {

```

```

01626         fi->POfill = fi->POfull = fi->PObuffer;
01627     }
01628 */
01629     SetScratch(fi,&position);
01630     term = oldwork = AT.WorkPointer;
01631     AR.CompressPointer = AR.CompressBuffer;
01632     AR.CompressPointer[0] = 0;
01633     AR.KeptInHold = 0;
01634     if ( GetTerm(BHEAD term) <= 0 ) {
01635         MLOCK(ErrorMessageLock);
01636         MesPrint("Expression %d has problems in scratchfile (t)",i);
01637         MUNLOCK(ErrorMessageLock);
01638         Terminate(-1);
01639     }
01640     if ( AT.bracketindexflag > 0 ) OpenBracketIndex(i);
01641     term[3] = i;
01642     if ( term[5] < 0 ) {
01643         fromspectator = -term[5];
01644         PUTZERO (AM.SpectatorFiles[fromspectator-1].readpos);
01645         term[5] = AC.cbunum;
01646     }
01647     PUTZERO(outposition);
01648     fout = AR.outfile;
01649     fout->POfill = fout->POfull = fout->PObuffer;
01650     fout->POposition = outposition;
01651     if ( fout->handle >= 0 ) {
01652         fout->POposition = outposition;
01653     }
01654 /*
01655     The next statement is needed because we need the system
01656     to believe that the expression is at position zero for
01657     the moment. In this worker, with no memory of other expressions,
01658     it is. This is needed for when a bracket index is made
01659     because there e->onfile is an offset. Afterwards, when the
01660     expression is written to its final location in the masters
01661     output e->onfile will get its real value.
01662 */
01663     PUTZERO(e->onfile);
01664     if ( PutOut(BHEAD term,&outposition,fout,0) < 0 ) goto ProcErr;
01665
01666     AR.DeferFlag = AC.ComDefer;
01667
01668     AR.sLevel = AB[0]->R.sLevel;
01669     term = AT.WorkPointer;
01670     NewSort(BHEAD0);
01671     AR.MaxDum = AM.IndDum;
01672     AN.ninterms = 0;
01673     if ( fromspectator ) {
01674         while ( GetFromSpectator(term,fromspectator-1) ) {
01675             AT.WorkPointer = term + *term;
01676             AN.RepPoint = AT.RepCount + 1;
01677             AN.IndDum = AM.IndDum;
01678             AR.CurDum = ReNumber(BHEAD term);
01679             if ( AC.SymChangeFlag ) MarkDirty(term,DIRTYSYMLAG);
01680             if ( AN.ncmod ) {
01681                 if ( ( AC.modmode & ALSOFUNARGS ) != 0 ) MarkDirty(term,DIRTYFLAG);
01682                 else if ( AR.PolyFun ) PolyFunDirty(BHEAD term);
01683             }
01684             else if ( AC.PolyRatFunChanged ) PolyFunDirty(BHEAD term);
01685             if ( ( AR.PolyFunType == 2 ) && ( AC.PolyRatFunChanged == 0 )
01686                 && ( e->status == LOCALEXPRESSION || e->status == GLOBALEXPRESSION ) ) {
01687                 PolyFunClean(BHEAD term);
01688             }
01689             if ( Generator(BHEAD term,0) ) {
01690                 LowerSortLevel(); goto ProcErr;
01691             }
01692         }
01693     }
01694     else {
01695         while ( GetTerm(BHEAD term) ) {
01696             SeekScratch(fi,&position);
01697             AN.ninterms++; dd = AN.deferskipped;
01698             if ( ( e->vflags & ISFACTORIZED ) != 0 && term[1] == HAAKJE ) {
01699                 StoreTerm(BHEAD term);
01700             }
01701             else {
01702                 if ( AC.CollectFun && *term <= (AM.MaxTer/(2*(LONG)sizeof(WORD))) ) {
01703                     if ( GetMoreTerms(term) < 0 ) {
01704                         LowerSortLevel(); goto ProcErr;
01705                     }
01706                     SeekScratch(fi,&position);
01707                 }
01708                 AT.WorkPointer = term + *term;
01709                 AN.RepPoint = AT.RepCount + 1;
01710                 if ( AR.DeferFlag ) {
01711                     AR.CurDum = AN.IndDum = Expressions[AR.exprtodo].numdummies;
01712                 }

```



```

01713         else {
01714             AN.IndDum = AM.IndDum;
01715             AR.CurDum = ReNumber(BHEAD term);
01716         }
01717         if ( AC.SymChangeFlag ) MarkDirty(term,DIRTYSYMFLAG);
01718         if ( AN.ncmod ) {
01719             if ( ( AC.modmode & ALSOFUNARGS ) != 0 ) MarkDirty(term,DIRTYFLAG);
01720             else if ( AR.PolyFun ) PolyFunDirty(BHEAD term);
01721         }
01722         else if ( AC.PolyRatFunChanged ) PolyFunDirty(BHEAD term);
01723         if ( ( AR.PolyFunType == 2 ) && ( AC.PolyRatFunChanged == 0 )
01724             && ( e->status == LOCALEXPRESSION || e->status == GLOBALEXPRESSION ) ) {
01725             PolyFunClean(BHEAD term);
01726         }
01727         if ( Generator(BHEAD term,0) ) {
01728             LowerSortLevel(); goto ProcErr;
01729         }
01730         AN.ninterms += dd;
01731     }
01732     SetScratch(fi,&position);
01733     if ( fi == AR.hidefile ) {
01734         AR.InHiBuf = (fi->POfull-fi->PObuffer)
01735             -DIFBASE(position,fi->POposition)/sizeof(WORD);
01736     }
01737     else {
01738         AR.InInBuf = (fi->POfull-fi->PObuffer)
01739             -DIFBASE(position,fi->POposition)/sizeof(WORD);
01740     }
01741 }
01742 }
01743 AN.ninterms += dd;
01744 if ( EndSort(BHEAD AT.S0->sBuffer,0) < 0 ) goto ProcErr;
01745 e->numdummies = AR.MaxDum - AM.IndDum;
01746 AR.BracketOn = oldBracketOn;
01747 AT.BrackBuf = oldBrackBuf;
01748 if ( ( e->vflags & TOBEFACTORED ) != 0 )
01749     poly_factorize_expression(e);
01750 else if ( ( ( e->vflags & TOBEUNFACTORED ) != 0 )
01751     && ( ( e->vflags & ISFACTORIZED ) != 0 ) )
01752     poly_unfactorize_expression(e);
01753 if ( AT.S0->TermsLeft ) e->vflags &= ~ISZERO;
01754 else e->vflags |= ISZERO;
01755 if ( AR.expchanged == 0 ) e->vflags |= ISUNMODIFIED;
01756 if ( AT.S0->TermsLeft ) AR.expflags |= ISZERO;
01757 if ( AR.expchanged ) AR.expflags |= ISUNMODIFIED;
01758 AR.GetFile = 0;
01759 AT.bracketindexflag = oldbracketindexflag;
01760 /*
01761     Now copy the whole thing from fout to AR0.outfile
01762     Do this in one go to keep the lock occupied as short as possible
01763 */
01764 SeekScratch(fout,&outposition);
01765 LOCK(AS.outputslock);
01766 oldoutfile = AB[0]->R.outfile;
01767 if ( e->status == INTOHIDELEXPRESSION || e->status == INTOHIDEGEXPRESSION ) {
01768     AB[0]->R.outfile = AB[0]->R.hidefile;
01769 }
01770 SeekScratch(AB[0]->R.outfile,&position);
01771 e->onfile = position;
01772 if ( CopyExpression(fout,AB[0]->R.outfile) < 0 ) {
01773     AB[0]->R.outfile = oldoutfile;
01774     UNLOCK(AS.outputslock);
01775     MLOCK(ErrorMessageLock);
01776     MesPrint("Error copying output of 'InParallel' expression to master. Thread:
01777 %d",identity);
01778     MUNLOCK(ErrorMessageLock);
01779     goto ProcErr;
01780 }
01781 AB[0]->R.outfile = oldoutfile;
01782 AB[0]->R.hidefile->POfull = AB[0]->R.hidefile->POfill;
01783 AB[0]->R.expflags = AR.expflags;
01784 UNLOCK(AS.outputslock);
01785 if ( fout->handle >= 0 ) { /* Now get rid of the file */
01786     CloseFile(fout->handle);
01787     fout->handle = -1;
01788     remove(fout->name);
01789     PUTZERO(fout->POposition);
01790     PUTZERO(fout->filesize);
01791     fout->POfill = fout->POfull = fout->PObuffer;
01792 }
01793 UpdateMaxSize();
01794
01795 AT.WorkPointer = oldwork;
01796
01797 } break;
01798 /*

```

```

01799      #] DOONEEXPRESSION :
01800      #[ DOBRACKETS :
01801
01802          In case we have a bracket index we can have the worker treat
01803          one or more of the entries in the bracket index.
01804          The advantage is that identical terms will meet each other
01805          sooner in the sorting and hence fewer compares will be needed.
01806          Also this way the master doesn't need to fill the buckets.
01807          The main problem is the load balancing which can become very
01808          bad when there is a long tail without things outside the bracket.
01809
01810          We get sent:
01811          1: The number of the first bracket to be done
01812          2: The number of the last bracket to be done
01813      */
01814      case DOBRACKETS: {
01815          BRACKETINFO *binfo;
01816          BRACKETINDEX *bi;
01817          FILEHANDLE *fi;
01818          POSITION stoppos, where;
01819          e = Expressions + AR.CurExpr;
01820          binfo = e->bracketinfo;
01821          thr = AN.threadbuck;
01822          bi = &(binfo->indexbuffer[thr->firstbracket]);
01823          if ( AR.GetFile == 2 ) fi = AR.hidefile;
01824          else fi = AR.infile;
01825          where = bi->start;
01826          ADD2POS(where, AS.OldOnFile[AR.CurExpr]);
01827          SetScratch(fi, &(where));
01828          stoppos = binfo->indexbuffer[thr->lastbracket].next;
01829          ADD2POS(stoppos, AS.OldOnFile[AR.CurExpr]);
01830          AN.ninterms = thr->firstterm;
01831      /*
01832          Now we have to put the 'value' of the bracket in the
01833          Compress buffer.
01834      */
01835          ttco = AR.CompressBuffer;
01836          tt = binfo->bracketbuffer + bi->bracket;
01837          i = *tt;
01838          NCOPY(ttco, tt, i)
01839          AR.CompressPointer = ttco;
01840          term = AT.WorkPointer;
01841          while ( GetTerm(BHEAD term) ) {
01842              SeekScratch(fi, &where);
01843              AT.WorkPointer = term + *term;
01844              AN.IndDum = AM.IndDum;
01845              AR.CurDum = ReNumber(BHEAD term);
01846              if ( AC.SymChangeFlag ) MarkDirty(term, DIRTYSYMFLAG);
01847              if ( AN.ncmod ) {
01848                  if ( ( AC.modmode & ALSOFUNARGS ) != 0 ) MarkDirty(term, DIRTYFLAG);
01849                  else if ( AR.PolyFun ) PolyFunDirty(BHEAD term);
01850              }
01851              else if ( AC.PolyRatFunChanged ) PolyFunDirty(BHEAD term);
01852              if ( ( AR.PolyFunType == 2 ) && ( AC.PolyRatFunChanged == 0 )
01853                  && ( e->status == LOCALEXPRESSION || e->status == GLOBALEXPRESSION ) ) {
01854                  PolyFunClean(BHEAD term);
01855              }
01856              if ( ( AP.PreDebug & THREADSDEBUG ) != 0 ) {
01857                  MLOCK(ErrorMessageLock);
01858                  MesPrint("Thread %w executing term:");
01859                  PrintTerm(term, "DoBrackets");
01860                  MUNLOCK(ErrorMessageLock);
01861              }
01862              AT.WorkPointer = term + *term;
01863              if ( Generator(BHEAD term, 0) ) {
01864                  LowerSortLevel();
01865                  MLOCK(ErrorMessageLock);
01866                  MesPrint("Error in processing one term in thread %d in module
01867                      %d", identity, AC.CModule);
01867                  MUNLOCK(ErrorMessageLock);
01868                  Terminate(-1);
01869              }
01870              AN.ninterms++;
01871              SetScratch(fi, &(where));
01872              if ( ISGEPOS(where, stoppos) ) break;
01873          }
01874          AT.WorkPointer = term;
01875          thr->free = BUCKETCOMINGFREE;
01876          break;
01877      }
01878  /*
01879      #] DOBRACKETS :
01880      #[ CLEARCLOCK :
01881
01882      The program only comes here after a .clear
01883  */
01884      case CLEARCLOCK:

```

```

01885  /*          LOCK(clearclocklock); */
01886          sumtimerinfo[identity] += TimeCPU(1);
01887          timerinfo[identity] = TimeCPU(0);
01888  /*          UNLOCK(clearclocklock); */
01889          break;
01890  /*
01891      #] CLEARCLOCK :
01892      #[ MCTSEXPANDTREE :
01893  */
01894      case MCTSEXPANDTREE:
01895          AT.optimtimes = AB[0]->T.optimtimes;
01896          find_Horner_MCTS_expand_tree();
01897          break;
01898  /*
01899      #] MCTSEXPANDTREE :
01900      #[ OPTIMIZEEXPRESSION :
01901  */
01902      case OPTIMIZEEXPRESSION:
01903          optimize_expression_given_Horner();
01904          break;
01905  /*
01906      #] OPTIMIZEEXPRESSION :
01907  */
01908      default:
01909          MLOCK(ErrorMessageLock);
01910          MesPrint("Illegal wakeup signal %d for thread %d",wakeupsignal,identity);
01911          MUNLOCK(ErrorMessageLock);
01912          Terminate(-1);
01913          break;
01914  }
01915  /* we need the following update in case we are using checkpoints. then we
01916     need to readjust the clocks when recovering using this information */
01917  timerinfo[identity] = TimeCPU(1);
01918  }
01919 EndOfThread;;
01920  /*
01921      This is the end of the thread. We cleanup and exit.
01922      If we are using flint, call the per-thread cleanup function. This keep valgrind happy.
01923  */
01924  #ifdef WITHFLINT
01925      flint_final_cleanup_thread();
01926  #endif
01927      FinalizeOneThread(identity);
01928      return(0);
01929 ProcErr:
01930      Terminate(-1);
01931      return(0);
01932  }
01933
01934  /*
01935      #] RunThread :
01936      #[ RunSortBot :
01937  */
01938  #ifdef WITHSORTBOTS
01939  void *RunSortBot(void *dummy)
01940  {
01941      int identity, wakeupsignal, identityretv;
01942      ALLPRIVATES *B, *BB;
01943      DUMMYUSE(dummy);
01944      identity = SetIdentity(&identityretv);
01945      threadpointers[identity] = pthread_self();
01946      B = InitializeOneThread(identity);
01947      while ( ( wakeupsignal = SortBotWait(identity) ) > 0 ) {
01948          switch ( wakeupsignal ) {
01949              /*
01950                  #[ INISORTBOT :
01951              */
01952              case INISORTBOT:
01953                  AR.CompressBuffer = AB[0]->R.CompressBuffer;
01954                  AR.ComprTop = AB[0]->R.ComprTop;
01955                  AR.CompressPointer = AB[0]->R.CompressPointer;
01956                  AR.CurExpr = AB[0]->R.CurExpr;
01957                  AR.PolyFun = AB[0]->R.PolyFun;
01958                  AR.PolyFunInv = AB[0]->R.PolyFunInv;
01959                  AR.PolyFunType = AB[0]->R.PolyFunType;
01960                  AR.PolyFunExp = AB[0]->R.PolyFunExp;
01961                  AR.PolyFunVar = AB[0]->R.PolyFunVar;
01962                  AR.PolyFunPow = AB[0]->R.PolyFunPow;
01963                  AR.SortType = AC.SortType;
01964                  if ( AR.PolyFun == 0 ) { AT.SS->PolyFlag = 0; }
01965                  else if ( AR.PolyFunType == 1 ) { AT.SS->PolyFlag = 1; }
01966                  else if ( AR.PolyFunType == 2 ) {
01967                      if ( AR.PolyFunExp == 2
01968                          || AR.PolyFunExp == 3 ) AT.SS->PolyFlag = 1;
01969                      else
01970                          AT.SS->PolyFlag = 2;
01971                  }

```

```

01979         AT.SS->PolyWise = 0;
01980         AN.ncmod = AC.ncmod;
01981         LOCK(AT.SB.MasterBlockLock[1]);
01982         BB = AB[AT.SortBotIn1];
01983         LOCK(BB->T.SB.MasterBlockLock[BB->T.SB.MasterNumBlocks]);
01984         BB = AB[AT.SortBotIn2];
01985         LOCK(BB->T.SB.MasterBlockLock[BB->T.SB.MasterNumBlocks]);
01986         AT.SB.FillBlock = 1;
01987         AT.SB.MasterFill[1] = AT.SB.MasterStart[1];
01988         SETBASEPOSITION(AN.theposition,0);
01989         break;
01990 /*
01991         #] INISORTBOT :
01992         #[ RUNSORTBOT :
01993 */
01994         case RUNSORTBOT:
01995             SortBotMerge(B);
01996             break;
01997 /*
01998         #] RUNSORTBOT :
01999         #[ TERMINATETHREAD :
02000 */
02001         case TERMINATETHREAD:
02002             goto EndOfThread;
02003 /*
02004         #] TERMINATETHREAD :
02005         #[ CLEARCLOCK :
02006
02007         The program only comes here after a .clear
02008 */
02009         case CLEARCLOCK:
02010             LOCK(clearclocklock); /*
02011             sumtimerinfo[identity] += TimeCPU(1);
02012             timerinfo[identity] = TimeCPU(0);
02013             UNLOCK(clearclocklock); */
02014             break;
02015 /*
02016         #] CLEARCLOCK :
02017 */
02018         default:
02019             MLOCK(ErrorMessageLock);
02020             MesPrint("Illegal wakeup signal %d for thread %d",wakeupsignal,identity);
02021             MUNLOCK(ErrorMessageLock);
02022             Terminate(-1);
02023             break;
02024     }
02025 }
02026 EndOfThread;;
02027 /*
02028     This is the end of the thread. We cleanup and exit.
02029     If we are using flint, call the per-thread cleanup function. This keep valgrind happy.
02030 */
02031 #ifdef WITHFLINT
02032     flint_final_cleanup_thread();
02033 #endif
02034     FinalizeOneThread(identity);
02035     return(0);
02036 }
02037 #endif
02038
02039 /*
02040     #] RunSortBot :
02041     #[ IAmAvailable :
02042 */
02043 void IAmAvailable(int identity)
02044 {
02045     int top;
02046     LOCK(availabilitylock);
02047     top = topofavailables;
02048     listofavailables[topofavailables++] = identity;
02049     if ( top == 0 ) {
02050         UNLOCK(availabilitylock);
02051         LOCK(wakeupmasterlock);
02052         wakeupmaster = identity;
02053         pthread_cond_signal(&wakeupmasterconditions);
02054         UNLOCK(wakeupmasterlock);
02055     }
02056     else {
02057         UNLOCK(availabilitylock);
02058     }
02059 }
02060
02061 /*
02062     #] IAmAvailable :
02063     #[ GetAvailableThread :
02064 */

```

```

02085 int GetAvailableThread(void)
02086 {
02087     int retval = -1;
02088     LOCK(availabilitylock);
02089     if ( topofavailables > 0 ) retval = listofavailables[--topofavailables];
02090     UNLOCK(availabilitylock);
02091     if ( retval >= 0 ) {
02092         /*
02093             Make sure the thread is indeed waiting and not between
02094             saying that it is available and starting to wait.
02095         */
02096         LOCK(wakeuplocks[retval]);
02097         UNLOCK(wakeuplocks[retval]);
02098     }
02099     return(retval);
02100 }
02101
02102 /*
02103     #] GetAvailableThread :
02104     #[ ConditionalGetAvailableThread :
02105 */
02113 int ConditionalGetAvailableThread(void)
02114 {
02115     int retval = -1;
02116     if ( topofavailables > 0 ) {
02117         LOCK(availabilitylock);
02118         if ( topofavailables > 0 ) {
02119             retval = listofavailables[--topofavailables];
02120         }
02121         UNLOCK(availabilitylock);
02122         if ( retval >= 0 ) {
02123             /*
02124                 Make sure the thread is indeed waiting and not between
02125                 saying that it is available and starting to wait.
02126             */
02127             LOCK(wakeuplocks[retval]);
02128             UNLOCK(wakeuplocks[retval]);
02129         }
02130     }
02131     return(retval);
02132 }
02133
02134 /*
02135     #] ConditionalGetAvailableThread :
02136     #[ GetThread :
02137 */
02147 int GetThread(int identity)
02148 {
02149     int retval = -1, j;
02150     LOCK(availabilitylock);
02151     for ( j = 0; j < topofavailables; j++ ) {
02152         if ( identity == listofavailables[j] ) break;
02153     }
02154     if ( j < topofavailables ) {
02155         --topofavailables;
02156         for ( ; j < topofavailables; j++ ) {
02157             listofavailables[j] = listofavailables[j+1];
02158         }
02159         retval = identity;
02160     }
02161     UNLOCK(availabilitylock);
02162     return(retval);
02163 }
02164
02165 /*
02166     #] GetThread :
02167     #[ ThreadWait :
02168 */
02178 int ThreadWait(int identity)
02179 {
02180     int retval, top, j;
02181     LOCK(wakeuplocks[identity]);
02182     LOCK(availabilitylock);
02183     top = topofavailables;
02184     for ( j = topofavailables; j > 0; j-- )
02185         listofavailables[j] = listofavailables[j-1];
02186     listofavailables[0] = identity;
02187     topofavailables++;
02188     if ( top == 0 || topofavailables == numberofworkers ) {
02189         UNLOCK(availabilitylock);
02190         LOCK(wakeupmasterlock);
02191         wakeupmaster = identity;
02192         pthread_cond_signal(&wakeupmasterconditions);
02193         UNLOCK(wakeupmasterlock);
02194     }
02195     else {
02196         UNLOCK(availabilitylock);

```

```

02197     }
02198     while ( wakeup[identity] == 0 ) {
02199         pthread_cond_wait(&(wakeupconditions[identity]),&(wakeuplocks[identity]));
02200     }
02201     retval = wakeup[identity];
02202     wakeup[identity] = 0;
02203     UNLOCK(wakeuplocks[identity]);
02204     return(retval);
02205 }
02206
02207 /*
02208  #] ThreadWait :
02209  #[ SortBotWait :
02210  */
02211
02212 #ifdef WITHSORTBOTS
02222 int SortBotWait(int identity)
02223 {
02224     int retval;
02225     LOCK(wakeuplocks[identity]);
02226     LOCK(availabilitylock);
02227     topsortbotavailables++;
02228     if ( topsortbotavailables >= numberofsortbots ) {
02229         UNLOCK(availabilitylock);
02230         LOCK(wakeupsortbotlock);
02231         wakeupmaster = identity;
02232         pthread_cond_signal(&wakeupsortbotconditions);
02233         UNLOCK(wakeupsortbotlock);
02234     }
02235     else {
02236         UNLOCK(availabilitylock);
02237     }
02238     while ( wakeup[identity] == 0 ) {
02239         pthread_cond_wait(&(wakeupconditions[identity]),&(wakeuplocks[identity]));
02240     }
02241     retval = wakeup[identity];
02242     wakeup[identity] = 0;
02243     UNLOCK(wakeuplocks[identity]);
02244     return(retval);
02245 }
02246
02247 #endif
02248
02249 /*
02250  #] SortBotWait :
02251  #[ ThreadClaimedBlock :
02252  */
02263 int ThreadClaimedBlock(int identity)
02264 {
02265     LOCK(availabilitylock);
02266     numberclaimed++;
02267     if ( numberclaimed >= numberofworkers ) {
02268         UNLOCK(availabilitylock);
02269         LOCK(wakeupmasterlock);
02270         wakeupmaster = identity;
02271         pthread_cond_signal(&wakeupmasterconditions);
02272         UNLOCK(wakeupmasterlock);
02273     }
02274     else {
02275         UNLOCK(availabilitylock);
02276     }
02277     return(0);
02278 }
02279
02280 /*
02281  #] ThreadClaimedBlock :
02282  #[ MasterWait :
02283  */
02291 int MasterWait(void)
02292 {
02293     int retval;
02294     LOCK(wakeupmasterlock);
02295     while ( wakeupmaster == 0 ) {
02296         pthread_cond_wait(&wakeupmasterconditions,&wakeupmasterlock);
02297     }
02298     retval = wakeupmaster;
02299     wakeupmaster = 0;
02300     UNLOCK(wakeupmasterlock);
02301     return(retval);
02302 }
02303
02304 /*
02305  #] MasterWait :
02306  #[ MasterWaitThread :
02307  */
02314 int MasterWaitThread(int identity)
02315 {

```

```

02316     int retval;
02317     LOCK(wakeupmasterthreadlocks[identity]);
02318     while ( wakeupmasterthread[identity] == 0 ) {
02319         pthread_cond_wait(&(wakeupmasterthreadconditions[identity])
02320             ,&(wakeupmasterthreadlocks[identity]));
02321     }
02322     retval = wakeupmasterthread[identity];
02323     wakeupmasterthread[identity] = 0;
02324     UNLOCK(wakeupmasterthreadlocks[identity]);
02325     return(retval);
02326 }
02327
02328 /*
02329  #] MasterWaitThread :
02330  #[ MasterWaitAll :
02331  */
02332 void MasterWaitAll(void)
02333 {
02334     LOCK(wakeupmasterlock);
02335     while ( topofavailables < numberofworkers ) {
02336         pthread_cond_wait(&wakeupmasterconditions,&wakeupmasterlock);
02337     }
02338     UNLOCK(wakeupmasterlock);
02339     return;
02340 }
02341
02342 /*
02343  #] MasterWaitAll :
02344  #[ MasterWaitAllSortBots :
02345  */
02346 #ifdef WITHSORTBOTS
02347 void MasterWaitAllSortBots(void)
02348 {
02349     LOCK(wakeupsortbotlock);
02350     while ( topsortbotavailables < numberofsortbots ) {
02351         pthread_cond_wait(&wakeupsortbotconditions,&wakeupsortbotlock);
02352     }
02353     UNLOCK(wakeupsortbotlock);
02354     return;
02355 }
02356 #endif
02357
02358 /*
02359  #] MasterWaitAllSortBots :
02360  #[ MasterWaitAllBlocks :
02361  */
02362 void MasterWaitAllBlocks(void)
02363 {
02364     LOCK(wakeupmasterlock);
02365     while ( numberclaimed < numberofworkers ) {
02366         pthread_cond_wait(&wakeupmasterconditions,&wakeupmasterlock);
02367     }
02368     UNLOCK(wakeupmasterlock);
02369     return;
02370 }
02371
02372 /*
02373  #] MasterWaitAllBlocks :
02374  #[ WakeupThread :
02375  */
02376 void WakeupThread(int identity, int signalnumber)
02377 {
02378     if ( signalnumber == 0 ) {
02379         MLOCK(ErrorMessageLock);
02380         MesPrint("Illegal wakeup signal for thread %d",identity);
02381         MUNLOCK(ErrorMessageLock);
02382         Terminate(-1);
02383     }
02384     LOCK(wakeuplocks[identity]);
02385     wakeup[identity] = signalnumber;
02386     pthread_cond_signal(&(wakeupconditions[identity]));
02387     UNLOCK(wakeuplocks[identity]);
02388 }
02389
02390 /*
02391  #] WakeupThread :
02392  #[ WakeupMasterFromThread :
02393  */
02394 void WakeupMasterFromThread(int identity, int signalnumber)
02395 {
02396     if ( signalnumber == 0 ) {
02397         MLOCK(ErrorMessageLock);
02398         MesPrint("Illegal wakeup signal for master %d",identity);
02399         MUNLOCK(ErrorMessageLock);

```

```

02436         Terminate(-1);
02437     }
02438     LOCK(wakeupmasterthreadlocks[identity]);
02439     wakeupmasterthread[identity] = signalnumber;
02440     pthread_cond_signal(&(wakeupmasterthreadconditions[identity]));
02441     UNLOCK(wakeupmasterthreadlocks[identity]);
02442 }
02443
02444 /*
02445  #] WakeupMasterFromThread :
02446  #[ SendOneBucket :
02447  */
02453 int SendOneBucket(int type)
02454 {
02455     ALLPRIVATES *B0 = AB[0];
02456     THREADBUCKET *thr = 0;
02457     int j, k, id;
02458     for ( j = 0; j < numthreadbuckets; j++ ) {
02459         if ( threadbuckets[j]->free == BUCKETFILLED ) {
02460             thr = threadbuckets[j];
02461             for ( k = j+1; k < numthreadbuckets; k++ )
02462                 threadbuckets[k-1] = threadbuckets[k];
02463             threadbuckets[numthreadbuckets-1] = thr;
02464             break;
02465         }
02466     }
02467     AN0.ninterms++;
02468     while ( ( id = GetAvailableThread() ) < 0 ) { MasterWait(); }
02469 /*
02470     Prepare the thread. Give it the term and variables.
02471  */
02472     LoadOneThread(0,id,thr,0);
02473     thr->busy = BUCKETASSIGNED;
02474     thr->free = BUCKETINUSE;
02475     numberoffullbuckets--;
02476 /*
02477     And signal the thread to run.
02478     Form now on we may only interfere with this bucket
02479     1: after it has been marked BUCKETCOMINGFREE
02480     2: when thr->busy == BUCKETDOINGTERM and then only when protected by
02481         thr->lock. This would be for load balancing.
02482  */
02483     WakeupThread(id,type);
02484 /* AN0.ninterms += thr->ddterms; */
02485     return(0);
02486 }
02487
02488 /*
02489  #] SendOneBucket :
02490  #[ InParallelProcessor :
02491  */
02511 int InParallelProcessor(void)
02512 {
02513     GETIDENTITY
02514     int i, id, retval = 0, num = 0;
02515     EXPRESSIONS e;
02516     if ( numberofworkers >= 2 ) {
02517         SetWorkerFiles();
02518         for ( i = 0; i < NumExpressions; i++ ) {
02519             e = Expressions+i;
02520             if ( e->partodo <= 0 ) continue;
02521             if ( e->status == LOCALEXPRESSION || e->status == GLOBALEXPRESSION
02522                 || e->status == UNHIDELEXPRESSION || e->status == UNHIDEGEXPRESSION
02523                 || e->status == INTOHIDELEXPRESSION || e->status == INTOHIDEGEXPRESSION ) {
02524             }
02525             else {
02526                 e->partodo = 0;
02527                 continue;
02528             }
02529             if ( e->counter == 0 ) { /* Expression with zero terms */
02530                 e->partodo = 0;
02531                 continue;
02532             }
02533 /*
02534             This expression should go to an idle worker
02535          */
02536             while ( ( id = GetAvailableThread() ) < 0 ) { MasterWait(); }
02537             LoadOneThread(0,id,0,-1);
02538             AB[id]->R.exprtoto = i;
02539             WakeupThread(id,DOONEEXPRESSION);
02540             num++;
02541         }
02542 /*
02543         Now we have to wait for all workers to finish
02544     */
02545         if ( num > 0 ) MasterWaitAll();
02546     }

```



```

02547         if ( AC.CollectFun ) AR.DeferFlag = 0;
02548     }
02549     else {
02550         for ( i = 0; i < NumExpressions; i++ ) {
02551             Expressions[i].partodo = 0;
02552         }
02553     }
02554     return(retval);
02555 }
02556 /*
02557  #] InParallelProcessor :
02558  #[ ThreadsProcessor :
02559 */
02585 int ThreadsProcessor(EXPRESSIONS e, WORD LastExpression, WORD fromspectator)
02586 {
02587     ALLPRIVATES *B0 = AB[0], *B = B0;
02588     int id, oldgzipCompress, endofinput = 0, j, still, k, defcount = 0, bra = 0, first = 1;
02589     LONG dd = 0, ddd, thrbufsiz, thrbufsiz0, thrbufsiz2, numbucket = 0, numpasses;
02590     LONG num, i;
02591     WORD *oldworkpointer = AT0.WorkPointer, *tt, *ttco = 0, *tl = 0, ter, *tstop = 0, *t2;
02592     THREADBUCKET *thr = 0;
02593     FILEHANDLE *oldoutfile = AR0.outfile;
02594     GETTERM GetTermP = &GetTerm;
02595     POSITION eonfile = AS.OldOnFile[e-Expressions];
02596     numberoffullbuckets = 0;
02597 /*
02598     Start up all threads. The lock needs to be around the whole loop
02599     to keep processes from terminating quickly and putting themselves
02600     in the list of available threads again.
02601 */
02602     AM.tracebackflag = 1;
02603
02604     AS.sLevel = AR0.sLevel;
02605     LOCK(availabilitylock);
02606     toposofavailables = 0;
02607     for ( id = 1; id <= numberofworkers; id++ ) {
02608         WakeupThread(id, STARTNEWEXPRESSION);
02609     }
02610     UNLOCK(availabilitylock);
02611     NewSort(BHEAD0);
02612     AN0.ninterms = 1;
02613 /*
02614     Now for redefine
02615 */
02616     if ( AC.numpfirstnum > 0 ) {
02617         for ( j = 0; j < AC.numpfirstnum; j++ ) {
02618             AC.inputnumbers[j] = -1;
02619         }
02620     }
02621     MasterWaitAll();
02622 /*
02623     Determine a reasonable bucketsize.
02624     This is based on the value of AC.ThreadBucketSize and the number
02625     of terms. We want at least 5 buckets per worker at the moment.
02626     Some research should show whether this is reasonable.
02627
02628     The number of terms in the expression is in e->counter
02629 */
02630     thrbufsiz2 = thrbufsiz = AC.ThreadBucketSize-1;
02631     if ( ( e->counter / ( numberofworkers * 5 ) ) < thrbufsiz ) {
02632         thrbufsiz = e->counter / ( numberofworkers * 5 ) - 1;
02633         if ( thrbufsiz < 0 ) thrbufsiz = 0;
02634     }
02635     thrbufsiz0 = thrbufsiz;
02636     numpasses = 5; /* this is just for trying */
02637     thrbufsiz = thrbufsiz0 / ( 2 << numpasses );
02638 /*
02639     Mark all buckets as free and take the first.
02640 */
02641     for ( j = 0; j < numthreadbuckets; j++ )
02642         threadbuckets[j]->free = BUCKETFREE;
02643     thr = threadbuckets[0];
02644 /*
02645     #[ Whole brackets :
02646
02647     First we look whether we have to work with entire brackets
02648     This is the case when there is a non-NULL address in e->bracketinfo.
02649     Of course we shouldn't have interference from a collect or keep statement.
02650 */
02651 #ifdef WHOLEBRACKETS
02652     if ( e->bracketinfo && AC.CollectFun == 0 && AR0.DeferFlag == 0 ) {
02653         FILEHANDLE *curfile;
02654         int didone = 0;
02655         LONG num, n;
02656         AN0.expr = e;
02657         for ( n = 0; n < e->bracketinfo->indexfill; n++ ) {

```

```

02658         num = TreatIndexEntry(B0,n);
02659         if ( num > 0 ) {
02660             didone = 1;
02661         /*
02662             This bracket can be sent off.
02663             1: Look for an empty bucket
02664         */
02665         ReTry;;
02666         for ( j = 0; j < numthreadbuckets; j++ ) {
02667             switch ( threadbuckets[j]->free ) {
02668                 case BUCKETFREE:
02669                     thr = threadbuckets[j];
02670                     goto Found1;
02671                 case BUCKETCOMINGFREE:
02672                     thr = threadbuckets[j];
02673                     thr->free = BUCKETFREE;
02674                     for ( k = j+1; k < numthreadbuckets; k++ )
02675                         threadbuckets[k-1] = threadbuckets[k];
02676                     threadbuckets[numthreadbuckets-1] = thr;
02677                     j--;
02678                     break;
02679                 default:
02680                     break;
02681             }
02682         }
02683         Found1;;
02684         if ( j < numthreadbuckets ) {
02685             /*
02686                 Found an empty bucket. Fill it.
02687             */
02688             thr->firstbracket = n;
02689             thr->lastbracket = n + num - 1;
02690             thr->type = BUCKETDOINGBRACKET;
02691             thr->free = BUCKETFILLED;
02692             thr->firstterm = AN0.ninterms;
02693             for ( j = n; j < n+num; j++ ) {
02694                 AN0.ninterms += e->bracketinfo->indexbuffer[j].termsinbracket;
02695             }
02696             n += num-1;
02697             numberoffullbuckets++;
02698             if ( topofavailables > 0 ) {
02699                 SendOneBucket(DOBRACKETS);
02700             }
02701         }
02702         /*
02703             All buckets are in use.
02704             Look/wait for an idle worker. Give it a bucket.
02705             After that, retry for a bucket
02706         */
02707         else {
02708             while ( topofavailables <= 0 ) {
02709                 MasterWait();
02710             }
02711             SendOneBucket(DOBRACKETS);
02712             goto ReTry;
02713         }
02714     }
02715 }
02716 if ( didone ) {
02717     /*
02718         And now put the input back in the original position.
02719     */
02720     switch ( e->status ) {
02721         case UNHIDELEXPRESSION:
02722         case UNHIDEGEXPRESSION:
02723         case DROPHLEXPRESSION:
02724         case DROPHGEXPRESSION:
02725         case HIDDENLEXPRESSION:
02726         case HIDDENGEXPRESSION:
02727             curfile = ARO.hidefile;
02728             break;
02729         default:
02730             curfile = ARO.infile;
02731             break;
02732     }
02733     SetScratch(curfile,&eonfile);
02734     GetTerm(B0,AT0.WorkPointer);
02735     /*
02736         Now we point the GetTerm that is used to the one that is selective
02737     */
02738     GetTermP = &GetTerm2;
02739     /*
02740         Next wait till there is a bucket available and initialize thr to it.
02741     */
02742     for(;;) {
02743         for ( j = 0; j < numthreadbuckets; j++ ) {
02744             switch ( threadbuckets[j]->free ) {

```

```

02745         case BUCKETFREE:
02746             thr = threadbuckets[j];
02747             goto Found2;
02748         case BUCKETCOMINGFREE:
02749             thr = threadbuckets[j];
02750             thr->free = BUCKETFREE;
02751             for ( k = j+1; k < numthreadbuckets; k++ )
02752                 threadbuckets[k-1] = threadbuckets[k];
02753             threadbuckets[numthreadbuckets-1] = thr;
02754             j--;
02755             break;
02756         default:
02757             break;
02758     }
02759 }
02760 while ( topofavailables <= 0 ) {
02761     MasterWait();
02762 }
02763 while ( topofavailables > 0 && numberoffullbuckets > 0 ) {
02764     SendOneBucket(DOBRACKETS);
02765 }
02766 }
02767 Found2:;
02768 while ( numberoffullbuckets > 0 ) {
02769     while ( topofavailables <= 0 ) {
02770         MasterWait();
02771     }
02772     while ( topofavailables > 0 && numberoffullbuckets > 0 ) {
02773         SendOneBucket(DOBRACKETS);
02774     }
02775 }
02776 /*
02777     Disable the 'warming up' with smaller buckets.
02778
02779     numpasses = 0;
02780     thrbufsiz = thrbufsiz0;
02781 */
02782     AN0.lastinindex = -1;
02783 }
02784 MasterWaitAll();
02785 }
02786 #endif
02787 /*
02788     #] Whole brackets :
02789
02790     Now the loop to start a bucket
02791 */
02792 for(;;) {
02793     if ( fromspectator ) {
02794         ter = GetFromSpectator(thr->threadbuffer,fromspectator-1);
02795         if ( ter == 0 ) fromspectator = 0;
02796     }
02797     else {
02798         ter = GetTermP(B0,thr->threadbuffer);
02799     }
02800     /*
02801         At this point we could check whether the input term is
02802         just an expression that resides in a scratch file.
02803         If this is the case we should store the current input info
02804         (file and buffer content) and redirect the input.
02805         At the end we can go back to where we were.
02806         There are two possibilities: we are in the same scratchfile
02807         as the main input and the other expression is still in the
02808         input buffer, or we have to do some reading. The reading is
02809         of course done in GetTerm.
02810
02811         How to set this up can also be studied in TestSub (in file proces.c)
02812         where it checks for EXPRESSION.
02813     */
02814     if ( ter < 0 ) break;
02815     if ( ter == 0 ) { endofinput = 1; goto Finalize; }
02816     dd = AN0.deferskipped;
02817     if ( AR0.DeferFlag ) {
02818         defcount = 0;
02819         thr->deferbuffer[defcount++] = AR0.DefPosition;
02820         ttco = thr->compressbuffer; t1 = AR0.CompressBuffer; j = *t1;
02821         NCOPY(ttco,t1,j);
02822     }
02823     else if ( first && ( AC.CollectFun == 0 ) ) { /* Brackets ? */
02824         first = 0;
02825         t1 = tstop = thr->threadbuffer;
02826         tstop += *tstop; tstop -= ABS(tstop[-1]);
02827         t1++;
02828         while ( t1 < tstop ) {
02829             if ( t1[0] == HAAKJE ) { bra = 1; break; }
02830             t1 += t1[1];
02831         }

```

```

02832         t1 = thr->threadbuffer;
02833     }
02834     /*
02835     Check whether we have a collect,function going. If so execute it.
02836     */
02837     if ( AC.CollectFun && *(thr->threadbuffer) < (AM.MaxTer/((LONG)sizeof(WORD))-10) ) {
02838         if ( ( dd = GetMoreTerms(thr->threadbuffer) ) < 0 ) {
02839             LowerSortLevel(); goto ProcErr;
02840         }
02841     }
02842     /*
02843     Check whether we have a priority task:
02844     */
02845     if ( topofavailables > 0 && numberoffullbuckets > 0 ) SendOneBucket(LOWESTLEVELGENERATION);
02846     /*
02847     Now put more terms in the bucket. Position tt after the first term
02848     */
02849     tt = thr->threadbuffer; tt += *tt;
02850     thr->totnum = 1;
02851     thr->usenum = 0;
02852     /*
02853     Next we worry about the 'slow startup' in which we make the initial
02854     buckets smaller, so that we get all threads busy as soon as possible.
02855     */
02856     if ( numpasses > 0 ) {
02857         numbucket++;
02858         if ( numbucket >= numberofworkers ) {
02859             numbucket = 0;
02860             numpasses--;
02861             if ( numpasses == 0 ) thrbufsiz = thrbufsiz0;
02862             else thrbufsiz = thrbufsiz0 / (2 « numpasses);
02863         }
02864         thrbufsiz2 = thrbufsiz + thrbufsiz/5; /* for completing brackets */
02865     }
02866     /*
02867     we have already 1+dd terms
02868     */
02869     while ( ( dd < thrbufsiz ) &&
02870         ( tt - thr->threadbuffer ) < ( thr->threadbuffersize - AM.MaxTer/((LONG)sizeof(WORD)) - 2
02871     ) ) {
02872         /*
02873         First check:
02874         */
02875         if ( topofavailables > 0 && numberoffullbuckets > 0 )
02876             SendOneBucket(LOWESTLEVELGENERATION);
02877         /*
02878         There is room in the bucket. Fill yet another term.
02879         */
02880         if ( GetTermP(B0,tt) == 0 ) { endofinput = 1; break; }
02881         dd++;
02882         thr->totnum++;
02883         dd += AN0.deferskipped;
02884         if ( AR0.DeferFlag ) {
02885             thr->deferbuffer[defcount++] = AR0.DefPosition;
02886             t1 = AR0.CompressBuffer; j = *t1;
02887             NCOPY(ttco,t1,j);
02888         }
02889         if ( AC.CollectFun && *tt < (AM.MaxTer/((LONG)sizeof(WORD))-10) ) {
02890             if ( ( ddd = GetMoreTerms(tt) ) < 0 ) {
02891                 LowerSortLevel(); goto ProcErr;
02892             }
02893             dd += ddd;
02894         }
02895         t1 = tt;
02896         tt += *tt;
02897     }
02898     /*
02899     Check whether there are regular brackets and if we have no DeferFlag
02900     and no collect, we try to add more terms till we finish the current
02901     bracket. We should however not overdo it. Let us say: up to 20%
02902     more terms are allowed.
02903     */
02904     if ( bra ) {
02905         tstop = t1 + *t1; tstop -= ABS(tstop[-1]);
02906         t2 = t1+1;
02907         while ( t2 < tstop ) {
02908             if ( t2[0] == HAAKJE ) { break; }
02909             t2 += t2[1];
02910         }
02911         if ( t2[0] == HAAKJE ) {
02912             t2 += t2[1]; num = t2 - t1;
02913             while ( ( dd < thrbufsiz2 ) &&
02914                 ( tt - thr->threadbuffer ) < ( thr->threadbuffersize - AM.MaxTer - 2 ) ) {
02915                 /*
02916                 First check:
02917                 */
02918                 if ( topofavailables > 0 && numberoffullbuckets > 0 )

```

```

        SendOneBucket (LOWESTLEVELGENERATION);
02917 /*
02918         There is room in the bucket. Fill yet another term.
02919 */
02920         if ( GetTermP (B0,tt) == 0 ) { endofinput = 1; break; }
02921 /*
02922         Same bracket?
02923 */
02924         tstop = tt + *tt; tstop -= ABS(tstop[-1]);
02925         if ( tstop-tt < num ) { /* Different: abort */
02926             AR0.KeptInHold = 1;
02927             break;
02928         }
02929         for ( i = 1; i < num; i++ ) {
02930             if ( t1[i] != tt[i] ) break;
02931         }
02932         if ( i < num ) { /* Different: abort */
02933             AR0.KeptInHold = 1;
02934             break;
02935         }
02936 /*
02937         Same bracket. We need this term.
02938 */
02939         dd++;
02940         thr->totnum++;
02941         tt += *tt;
02942     }
02943 }
02944 }
02945 thr->ddterms = dd; /* total number of terms including keep brackets */
02946 thr->firstterm = AN0.ninterms;
02947 AN0.ninterms += dd;
02948 *tt = 0; /* mark end of bucket */
02949 thr->free = BUCKETFILLED;
02950 thr->type = BUCKETDOINGTERMS;
02951 numberoffullbuckets++;
02952 if ( topofavailables <= 0 && endofinput == 0 ) {
02953 /*
02954     Problem: topofavailables may already be > 0, but the
02955     thread has not yet gone into waiting. Can the signal get lost?
02956     How can we tell that a thread is waiting for a signal?
02957
02958     All threads are busy. Try to load up another bucket.
02959     In the future we could be more sophisticated.
02960     At the moment we load a complete bucket which could be
02961     1000 terms or even more.
02962     In principle it is better to keep a full bucket ready
02963     and check after each term we put in the next bucket. That
02964     way we don't waste time of the workers.
02965 */
02966     for ( j = 0; j < numthreadbuckets; j++ ) {
02967         switch ( threadbuckets[j]->free ) {
02968             case BUCKETFREE:
02969                 thr = threadbuckets[j];
02970                 if ( !endofinput ) goto NextBucket;
02971 /*
02972                 If we are at the end of the input we mark
02973                 the free buckets in a special way. That way
02974                 we don't keep running into them.
02975 */
02976                 thr->free = BUCKETATEND;
02977                 break;
02978             case BUCKETCOMINGFREE:
02979                 thr = threadbuckets[j];
02980                 thr->free = BUCKETFREE;
02981 /*
02982                 Bucket has just been finished.
02983                 Put at the end of the list. We don't want
02984                 an early bucket to wait to be treated last.
02985 */
02986                 for ( k = j+1; k < numthreadbuckets; k++ )
02987                     threadbuckets[k-1] = threadbuckets[k];
02988                 threadbuckets[numthreadbuckets-1] = thr;
02989                 j--; /* we have to redo the same number j. */
02990                 break;
02991             default:
02992                 break;
02993         }
02994     }
02995 /*
02996     We have no free bucket or we are at the end.
02997     The only thing we can do now is wait for a worker to come free,
02998     provided there are still buckets to send.
02999 */
03000 }
03001 /*
03002     Look for the next bucket to send. There is at least one full bucket!

```

```

03003 */
03004     for ( j = 0; j < numthreadbuckets; j++ ) {
03005         if ( threadbuckets[j]->free == BUCKETFILLED ) {
03006             thr = threadbuckets[j];
03007             for ( k = j+1; k < numthreadbuckets; k++ )
03008                 threadbuckets[k-1] = threadbuckets[k];
03009             threadbuckets[numthreadbuckets-1] = thr;
03010             break;
03011         }
03012     }
03013 */
03014     Wait for a thread to become available
03015     The bucket we are going to use is in thr.
03016 */
03017 DoBucket;;
03018     AN0.ninterms++;
03019     while ( ( id = GetAvailableThread() ) < 0 ) { MasterWait(); }
03020 */
03021     Prepare the thread. Give it the term and variables.
03022 */
03023     LoadOneThread(0,id,thr,0);
03024     LOCK(thr->lock);
03025     thr->busy = BUCKETASSIGNED;
03026     UNLOCK(thr->lock);
03027     thr->free = BUCKETINUSE;
03028     numberoffullbuckets--;
03029 */
03030     And signal the thread to run.
03031     Form now on we may only interfere with this bucket
03032     1: after it has been marked BUCKETCOMINGFREE
03033     2: when thr->busy == BUCKETDOINGTERM and then only when protected by
03034        thr->lock. This would be for load balancing.
03035 */
03036     WakeupThread(id,LOWESTLEVELGENERATION);
03037 */
03038     AN0.ninterms += thr->ddterms; */
03039 */
03040     Now look whether there is another bucket filled and a worker available
03041 */
03042     if ( topofavailables > 0 ) { /* there is a worker */
03043         for ( j = 0; j < numthreadbuckets; j++ ) {
03044             if ( threadbuckets[j]->free == BUCKETFILLED ) {
03045                 thr = threadbuckets[j];
03046                 for ( k = j+1; k < numthreadbuckets; k++ )
03047                     threadbuckets[k-1] = threadbuckets[k];
03048                 threadbuckets[numthreadbuckets-1] = thr;
03049                 goto DoBucket; /* and we found a bucket */
03050             }
03051         }
03052     }
03053     no bucket is loaded but there is a thread available
03054     find a bucket to load. If there is none (all are USED or ATEND)
03055     we jump out of the loop.
03056 */
03057     for ( j = 0; j < numthreadbuckets; j++ ) {
03058         switch ( threadbuckets[j]->free ) {
03059             case BUCKETFREE:
03060                 thr = threadbuckets[j];
03061                 if ( !endofinput ) goto NextBucket;
03062                 thr->free = BUCKETATEND;
03063                 break;
03064             case BUCKETCOMINGFREE:
03065                 thr = threadbuckets[j];
03066                 if ( endofinput ) {
03067                     thr->free = BUCKETATEND;
03068                 }
03069                 else {
03070                     thr->free = BUCKETFREE;
03071                     for ( k = j+1; k < numthreadbuckets; k++ )
03072                         threadbuckets[k-1] = threadbuckets[k];
03073                     threadbuckets[numthreadbuckets-1] = thr;
03074                     j--;
03075                 }
03076                 break;
03077             default:
03078                 break;
03079         }
03080     }
03081     if ( j >= numthreadbuckets ) break;
03082 }
03083 */
03084     No worker available.
03085     Look for a bucket to load.
03086     Its number will be in "still"
03087 */
03088 Finalize;;
03089     still = -1;

```

```

03090         for ( j = 0; j < numthreadbuckets; j++ ) {
03091             switch ( threadbuckets[j]->free ) {
03092                 case BUCKETFREE:
03093                     thr = threadbuckets[j];
03094                     if ( !endofinput ) goto NextBucket;
03095                     thr->free = BUCKETATEND;
03096                     break;
03097                 case BUCKETCOMINGFREE:
03098                     thr = threadbuckets[j];
03099                     if ( endofinput ) thr->free = BUCKETATEND;
03100                     else {
03101                         thr->free = BUCKETFREE;
03102                         for ( k = j+1; k < numthreadbuckets; k++ )
03103                             threadbuckets[k-1] = threadbuckets[k];
03104                         threadbuckets[numthreadbuckets-1] = thr;
03105                         j--;
03106                     }
03107                     break;
03108                 case BUCKETFILLED:
03109                     if ( still < 0 ) still = j;
03110                     break;
03111                 default:
03112                     break;
03113             }
03114         }
03115         if ( still < 0 ) {
03116             /*
03117              * No buckets to be executed and no buckets FREE.
03118              * We must be at the end. Break out of the loop.
03119              */
03120             break;
03121         }
03122         thr = threadbuckets[still];
03123         for ( k = still+1; k < numthreadbuckets; k++ )
03124             threadbuckets[k-1] = threadbuckets[k];
03125         threadbuckets[numthreadbuckets-1] = thr;
03126         goto DoBucket;
03127     }
03128 NextBucket:;
03129     }
03130     /*
03131      * Now the stage one load balancing.
03132      * If the load has been readjusted we have again filled buckets.
03133      * In that case we jump back in the loop.
03134      *
03135      * Tricky point: when do the workers see the new value of AT.LoadBalancing?
03136      * It should activate the locks on thr->busy
03137      */
03138     if ( AC.ThreadBalancing ) {
03139         for ( id = 1; id <= numberofworkers; id++ ) {
03140             AB[id]->T.LoadBalancing = 1;
03141         }
03142         if ( LoadReadjusted() ) goto Finalize;
03143         for ( id = 1; id <= numberofworkers; id++ ) {
03144             AB[id]->T.LoadBalancing = 0;
03145         }
03146     }
03147     if ( AC.ThreadBalancing ) {
03148         /*
03149          * The AS.Balancing flag should have Generator look for
03150          * free workers and apply the "buro" method.
03151          *
03152          * There is still a serious problem.
03153          * When for instance a sum_, there may be space created in a local
03154          * compiler buffer for a wildcard substitution or whatever.
03155          * Compiler buffer execution scribble space.....
03156          * This isn't copied along?
03157          * Look up ebufnum. There are 12 places with AddRHS!
03158          * Problem: one process allocates in ebuf. Then term is given to
03159          * other process. It would like to use from this ebuf, but the sender
03160          * finishes first and removes the ebuf (and/or overwrites it).
03161          *
03162          * Other problem: local $ variables aren't copied along.
03163          */
03164         AS.Balancing = 0;
03165     }
03166     MasterWaitAll();
03167     AS.Balancing = 0;
03168     /*
03169      * When we deal with the last expression we can now remove the input
03170      * scratch file. This saves potentially much disk space (up to 1/3)
03171      */
03172     if ( LastExpression ) {
03173         UpdateMaxSize();
03174         if ( AR0.infile->handle >= 0 ) {
03175             CloseFile(AR0.infile->handle);
03176             AR0.infile->handle = -1;

```

```

03177         remove(AR0.infile->name);
03178         PUTZERO(AR0.infile->POposition);
03179         AR0.infile->POfill = AR0.infile->POfull = AR0.infile->PObuffer;
03180     }
03181 }
03182 /*
03183 We order the threads to finish in the MasterMerge routine
03184 It will start with waiting for all threads to finish.
03185 One could make an administration in which threads that have
03186 finished can start already with the final sort but
03187 1: The load balancing should not make this super urgent
03188 2: It would definitely not be very compatible with the second
03189 stage load balancing.
03190 */
03191 oldgzipCompress = AR0.gzipCompress;
03192 AR0.gzipCompress = 0;
03193 if ( AR0.outtohide ) AR0.outfile = AR0.hidefile;
03194 if ( MasterMerge() < 0 ) {
03195     if ( AR0.outtohide ) AR0.outfile = oldoutfile;
03196     AR0.gzipCompress = oldgzipCompress;
03197     goto ProcErr;
03198 }
03199 if ( AR0.outtohide ) AR0.outfile = oldoutfile;
03200 AR0.gzipCompress = oldgzipCompress;
03201 /*
03202 Now wait for all threads to be ready to give them the cleaning up signal.
03203 With the new MasterMerge routine we can do the cleanup already automatically
03204 avoiding having to send these signals.
03205 */
03206 MasterWaitAll();
03207 AR0.sLevel--;
03208 for ( id = 1; id < AM.totalnumberofthreads; id++ ) {
03209     if ( GetThread(id) > 0 ) WakeupThread(id,CLEANUPEXPRESSION);
03210 }
03211 e->numdummies = 0;
03212 for ( id = 1; id < AM.totalnumberofthreads; id++ ) {
03213     if ( AB[id]->R.MaxDum - AM.IndDum > e->numdummies )
03214         e->numdummies = AB[id]->R.MaxDum - AM.IndDum;
03215     AR0.expchanged |= AB[id]->R.expchanged;
03216 }
03217 /*
03218 And wait for all to be clean.
03219 */
03220 MasterWaitAll();
03221 AT0.WorkPointer = oldworkpointer;
03222 return(0);
03223 ProcErr:;
03224 return(-1);
03225 }
03226
03227 /*
03228 #] ThreadsProcessor :
03229 #[ LoadReadjusted :
03230 */
03231 int LoadReadjusted(void)
03232 {
03233     ALLPRIVATES *B0 = AB[0];
03234     THREADBUCKET *thr = 0, *thrtogo = 0;
03235     int numtogo, numfree, numbusy, n, nperbucket, extra, i, j, u, bus;
03236     LONG numinput;
03237     WORD *t1, *c1, *t2, *c2, *t3;
03238
03239     /*
03240     Start with waiting for at least one free processor.
03241     We don't want the master competing for time when all are busy.
03242     */
03243     while ( topofavailables <= 0 ) MasterWait();
03244
03245     /*
03246     Now look for the fullest bucket and make a list of free buckets
03247     The bad part is that most numbers can change at any moment.
03248     */
03249 restart:;
03250     numtogo = 0;
03251     numfree = 0;
03252     numbusy = 0;
03253     for ( j = 0; j < numthreadbuckets; j++ ) {
03254         thr = threadbuckets[j];
03255         if ( thr->free == BUCKETFREE || thr->free == BUCKETATEND
03256             || thr->free == BUCKETCOMINGFREE ) {
03257             freebuckets[numfree++] = thr;
03258         }
03259         else if ( thr->type != BUCKETDOINGTERMS ) {}
03260         else if ( thr->totnum > 1 ) { /* never steal from a bucket with one term */
03261             LOCK(thr->lock);
03262             bus = thr->busy;
03263             UNLOCK(thr->lock);
03264             if ( thr->free == BUCKETINUSE ) {
03265                 n = thr->totnum-thr->usenum;

```



```

03282         if ( bus == BUCKETASSIGNED ) numbusy++;
03283     else if ( ( bus != BUCKETASSIGNED )
03284         && ( n > numtogo ) ) {
03285         numtogo = n;
03286         thrtogo = thr;
03287     }
03288 }
03289 else if ( bus == BUCKETTOBERELEASED
03290     && thr->free == BUCKETRELEASED ) {
03291     freebuckets[numfree++] = thr;
03292     thr->free = BUCKETATEND;
03293     LOCK(thr->lock);
03294     thr->busy = BUCKETPREPARINGTERM;
03295     UNLOCK(thr->lock);
03296 }
03297 }
03298 }
03299 if ( numfree == 0 ) return(0); /* serious problem */
03300 if ( numtogo > 0 ) { /* provisionally there is something to be stolen */
03301     thr = thrtogo;
03302 /*
03303     If the number has changed there is good progress.
03304     Maybe there is another thread that needs assistance.
03305     We start all over.
03306 */
03307     if ( thr->totnum-thr->usenum < numtogo ) goto restart;
03308 /*
03309     If the thread is in the term loading phase
03310     (thr->busy == BUCKETPREPARINGTERM) we better stay away from it.
03311     We wait now for the thread to be busy, and don't allow it
03312     now to drop out of this state till we are done here.
03313     This all depends on whether AT.LoadBalancing == 1 is seen by
03314     the thread.
03315 */
03316     LOCK(thr->lock);
03317     if ( thr->busy != BUCKETDOINGTERM ) {
03318         UNLOCK(thr->lock);
03319         goto restart;
03320     }
03321     if ( thr->totnum-thr->usenum < numtogo ) {
03322         UNLOCK(thr->lock);
03323         goto restart;
03324     }
03325     thr->free = BUCKETTERMINATED;
03326 /*
03327     The above will signal the thread we want to terminate.
03328     Next all effort goes into making sure the landing is soft.
03329     Unfortunately we don't want to wait for a signal, because the thread
03330     may be working for a long time on a single term.
03331 */
03332     if ( thr->usenum == thr->totnum ) {
03333 /*
03334         Terminated in the mean time or by now working on the
03335         last term. Try again.
03336 */
03337         thr->free = BUCKETATEND;
03338         UNLOCK(thr->lock);
03339         goto restart;
03340     }
03341     goto intercepted;
03342 }
03343 /* This has always been commented. Indeed no lock is held here. */
03344 /* UNLOCK(thr->lock); */
03345 if ( numbusy > 0 ) {
03346     /* JD: this avoids large runtimes for tform tests under valgrind.
03347        What seems to happen is we return from here, goto Finalize, and
03348        end up in LoadReadjusted again without the threads having a
03349        chance to update their busy status. Then we end up here again.
03350        Sleep the thread for, say, lus to allow threads to aquire the lock. */
03351     struct timespec sleeptime;
03352     sleeptime.tv_sec = 0;
03353     sleeptime.tv_nsec = 1000L;
03354     nanosleep(&sleeptime, NULL);
03355     return(1); /* Wait a bit.... */
03356 }
03357 return(0);
03358 intercepted;
03359 /*
03360     We intercepted one successfully. Now it becomes interesting. Action:
03361     1: determine how many terms per free bucket.
03362     2: find the first untreated term.
03363     3: put the terms in the free buckets.
03364
03365     Remember: we still have the lock to avoid interference from the thread
03366     that is being robbed. We were holding it and then jumped here with
03367     goto intercepted.
03368 */

```

```

03369     numinput = thr->firstterm + thr->usenum;
03370     nperbucket = numtogo / numfree;
03371     extra = numtogo - nperbucket*numfree;
03372     if ( AR0.DeferFlag ) {
03373         t1 = thr->threadbuffer; c1 = thr->compressbuffer; u = thr->usenum;
03374         for ( n = 0; n < thr->usenum; n++ ) { t1 += *t1; c1 += *c1; }
03375         t3 = t1;
03376         if ( extra > 0 ) {
03377             for ( i = 0; i < extra; i++ ) {
03378                 thrtogo = freebuckets[i];
03379                 t2 = thrtogo->threadbuffer;
03380                 c2 = thrtogo->compressbuffer;
03381                 thrtogo->free = BUCKETFILLED;
03382                 thrtogo->type = BUCKETDOINGTERMS;
03383                 thrtogo->totnum = nperbucket+1;
03384                 thrtogo->ddterms = 0;
03385                 thrtogo->usenum = 0;
03386                 thrtogo->busy = BUCKETASSIGNED;
03387                 thrtogo->firstterm = numinput;
03388                 numinput += nperbucket+1;
03389                 for ( n = 0; n <= nperbucket; n++ ) {
03390                     j = *t1; NCOPY(t2,t1,j);
03391                     j = *c1; NCOPY(c2,c1,j);
03392                     thrtogo->deferbuffer[n] = thr->deferbuffer[u++];
03393                 }
03394                 *t2 = *c2 = 0;
03395             }
03396         }
03397         if ( nperbucket > 0 ) {
03398             for ( i = extra; i < numfree; i++ ) {
03399                 thrtogo = freebuckets[i];
03400                 t2 = thrtogo->threadbuffer;
03401                 c2 = thrtogo->compressbuffer;
03402                 thrtogo->free = BUCKETFILLED;
03403                 thrtogo->type = BUCKETDOINGTERMS;
03404                 thrtogo->totnum = nperbucket;
03405                 thrtogo->ddterms = 0;
03406                 thrtogo->usenum = 0;
03407                 thrtogo->busy = BUCKETASSIGNED;
03408                 thrtogo->firstterm = numinput;
03409                 numinput += nperbucket;
03410                 for ( n = 0; n < nperbucket; n++ ) {
03411                     j = *t1; NCOPY(t2,t1,j);
03412                     j = *c1; NCOPY(c2,c1,j);
03413                     thrtogo->deferbuffer[n] = thr->deferbuffer[u++];
03414                 }
03415                 *t2 = *c2 = 0;
03416             }
03417         }
03418     }
03419     else {
03420         t1 = thr->threadbuffer;
03421         for ( n = 0; n < thr->usenum; n++ ) { t1 += *t1; }
03422         t3 = t1;
03423         if ( extra > 0 ) {
03424             for ( i = 0; i < extra; i++ ) {
03425                 thrtogo = freebuckets[i];
03426                 t2 = thrtogo->threadbuffer;
03427                 thrtogo->free = BUCKETFILLED;
03428                 thrtogo->type = BUCKETDOINGTERMS;
03429                 thrtogo->totnum = nperbucket+1;
03430                 thrtogo->ddterms = 0;
03431                 thrtogo->usenum = 0;
03432                 thrtogo->busy = BUCKETASSIGNED;
03433                 thrtogo->firstterm = numinput;
03434                 numinput += nperbucket+1;
03435                 for ( n = 0; n <= nperbucket; n++ ) {
03436                     j = *t1; NCOPY(t2,t1,j);
03437                 }
03438                 *t2 = 0;
03439             }
03440         }
03441         if ( nperbucket > 0 ) {
03442             for ( i = extra; i < numfree; i++ ) {
03443                 thrtogo = freebuckets[i];
03444                 t2 = thrtogo->threadbuffer;
03445                 thrtogo->free = BUCKETFILLED;
03446                 thrtogo->type = BUCKETDOINGTERMS;
03447                 thrtogo->totnum = nperbucket;
03448                 thrtogo->ddterms = 0;
03449                 thrtogo->usenum = 0;
03450                 thrtogo->busy = BUCKETASSIGNED;
03451                 thrtogo->firstterm = numinput;
03452                 numinput += nperbucket;
03453                 for ( n = 0; n < nperbucket; n++ ) {
03454                     j = *t1; NCOPY(t2,t1,j);
03455                 }

```

```

03456         *t2 = 0;
03457     }
03458 }
03459 }
03460 *t3 = 0; /* This is some form of extra insurance */
03461 if ( thr->free == BUCKETRELEASED && thr->busy == BUCKETTOBERELEASED ) {
03462     thr->free = BUCKETATEND; thr->busy = BUCKETPREPARINGTERM;
03463 }
03464 UNLOCK(thr->lock);
03465 return(1);
03466 }
03467
03468 /*
03469 #] LoadReadjusted :
03470 #[ SortStrategy :
03471 */
03504 /*
03505 #] SortStrategy :
03506 #[ PutToMaster :
03507 */
03530 int PutToMaster(PHEAD WORD *term)
03531 {
03532     int i,j,nexti,ret = 0;
03533     WORD *t, *fill, *top, zero = 0;
03534     if ( term == 0 ) { /* Mark the end of the expression */
03535         t = &zero; j = 1;
03536     }
03537     else {
03538         t = term; ret = j = *term;
03539         if ( j == 0 ) { j = 1; } /* Just in case there is a spurious end */
03540     }
03541     i = AT.SB.FillBlock; /* The block we are working at */
03542     fill = AT.SB.MasterFill[i]; /* Where we are filling */
03543     top = AT.SB.MasterStop[i]; /* End of the block */
03544     while ( j > 0 ) {
03545         while ( j > 0 && fill < top ) {
03546             *fill++ = *t++; j--;
03547         }
03548         if ( j > 0 ) {
03549             /*
03550              We reached the end of the block.
03551              Get the next block and release this block.
03552              The order is important. This is why there should be at least
03553              4 blocks or deadlocks can occur.
03554             */
03555             nexti = i+1;
03556             if ( nexti > AT.SB.MasterNumBlocks ) {
03557                 nexti = 1;
03558             }
03559             LOCK(AT.SB.MasterBlockLock[nexti]);
03560             UNLOCK(AT.SB.MasterBlockLock[i]);
03561             AT.SB.MasterFill[i] = AT.SB.MasterStart[i];
03562             AT.SB.FillBlock = i = nexti;
03563             fill = AT.SB.MasterStart[i];
03564             top = AT.SB.MasterStop[i];
03565         }
03566     }
03567     AT.SB.MasterFill[i] = fill;
03568     return(ret);
03569 }
03570
03571 /*
03572 #] PutToMaster :
03573 #[ SortBotOut :
03574 */
03575
03576 #ifdef WITHSORTBOTS
03577
03586 int
03587 SortBotOut(PHEAD WORD *term)
03588 {
03589     WORD im;
03590
03591     if ( AT.identity != 0 ) return(PutToMaster(BHEAD term));
03592
03593     if ( term == 0 ) {
03594         if ( FlushOut(&SortBotPosition,AR.outfile,1) ) return(-1);
03595         ADDPOS(AT.SS->SizeInFile[0],1);
03596         return(0);
03597     }
03598     else {
03599         numerofterms++;
03600         if ( ( im = PutOut(BHEAD term,&SortBotPosition,AR.outfile,1) ) < 0 ) {
03601             MLOCK(ErrorMessageLock);
03602             MesPrint("Called from MasterMerge/SortBotOut");
03603             MUNLOCK(ErrorMessageLock);
03604             return(-1);

```

```

03605     }
03606     ADDPOS(AT.SS->SizeInFile[0],im);
03607     return(im);
03608 }
03609 }
03610
03611 #endif
03612
03613 /*
03614  #] SortBotOut :
03615  #[ MasterMerge :
03616 */
03634 int MasterMerge(void)
03635 {
03636     ALLPRIVATES *B0 = AB[0], *B = 0;
03637     SORTING *S = AT0.SS;
03638     WORD **poin, **poin2, ul, k, i, im, *m1, j;
03639     WORD lpat, mpat, level, l1, l2, r1, r2, r3, c;
03640     WORD *m2, *m3, r31, r33, ki, *rr;
03641     UWORD *coef;
03642     POSITION position;
03643     FILEHANDLE *fin, *fout;
03644     #ifdef WITHSORTBOTS
03645     if ( numberofworkers > 2 ) return(SortBotMasterMerge());
03646     #endif
03647     fin = &S->file;
03648     if ( AR0.PolyFun == 0 ) { S->PolyFlag = 0; }
03649     else if ( AR0.PolyFunType == 1 ) { S->PolyFlag = 1; }
03650     else if ( AR0.PolyFunType == 2 ) {
03651         if ( AR0.PolyFunExp == 2
03652             || AR0.PolyFunExp == 3 ) S->PolyFlag = 1;
03653         else S->PolyFlag = 2;
03654     }
03655     S->TermsLeft = 0;
03656     coef = AN0.SoScratC;
03657     poin = S->poina; poin2 = S->poin2a;
03658     rr = AR0.CompressPointer;
03659     *rr = 0;
03660 /*
03661     #[ Setup :
03662 */
03663     S->inNum = numberofthreads;
03664     fout = AR0.outfile;
03665 /*
03666     Load the patches. The threads have to finish their sort first.
03667 */
03668     S->lPatch = S->inNum - 1;
03669 /*
03670     Claim all zero blocks. We need them anyway.
03671     In principle the workers should never get into these.
03672     We also claim all last blocks. This is a safety procedure that
03673     should prevent the workers from working their way around the clock
03674     before the master gets started again.
03675 */
03676     AS.MasterSort = 1;
03677     numberclaimed = 0;
03678     for ( i = 1; i <= S->lPatch; i++ ) {
03679         B = AB[i];
03680         LOCK(AT.SB.MasterBlockLock[0]);
03681         LOCK(AT.SB.MasterBlockLock[AT.SB.MasterNumBlocks]);
03682     }
03683 /*
03684     Now wake up the threads and have them start their final sorting.
03685     They should start with claiming their block and the master is
03686     not allowed to continue until that has been done.
03687     This waiting of the master will be done below in MasterWaitAllBlocks
03688 */
03689     for ( i = 0; i < S->lPatch; i++ ) {
03690         GetThread(i+1);
03691         WakeupThread(i+1,FINISHEXPRESSION);
03692     }
03693 /*
03694     Prepare the output file.
03695 */
03696     if ( fout->handle >= 0 ) {
03697         PUTZERO(position);
03698         SeekFile(fout->handle,&position,SEEK_END);
03699         ADDPOS(position,((fout->POfill-fout->PObuffer)*sizeof(WORD)));
03700     }
03701     else {
03702         SETBASEPOSITION(position, (fout->POfill-fout->PObuffer)*sizeof(WORD));
03703     }
03704 /*
03705     Wait for all threads to finish loading their first block.
03706 */
03707     MasterWaitAllBlocks();
03708 /*

```

```

03709      Claim all first blocks.
03710      We don't release the last blocks.
03711      The strategy is that we always keep the previous block.
03712      In principle it looks like it isn't needed for the last block but
03713      actually it is to keep the front from overrunning the tail when writing.
03714  */
03715      for ( i = 1; i <= S->lPatch; i++ ) {
03716          B = AB[i];
03717          LOCK(AT.SB.MasterBlockLock[1]);
03718          AT.SB.MasterBlock = 1;
03719      }
03720  /*
03721      #] Setup :
03722
03723      Now construct the tree:
03724  */
03725      lpat = 1;
03726      do { lpat <= 1; } while ( lpat < S->lPatch );
03727      mpat = ( lpat » 1 ) - 1;
03728      k = lpat - S->lPatch;
03729  /*
03730      k is the number of empty places in the tree. they will
03731      be at the even positions from 2 to 2*k
03732  */
03733      for ( i = 1; i < lpat; i++ ) { S->tree[i] = -1; }
03734      for ( i = 1; i <= k; i++ ) {
03735          im = ( i * 2 ) - 1;
03736          poin[im] = AB[i]->T.SB.MasterStart[AB[i]->T.SB.MasterBlock];
03737          poin2[im] = poin[im] + *(poin[im]);
03738          S->used[i] = im;
03739          S->ktoi[im] = i-1;
03740          S->tree[mpat+i] = 0;
03741          poin[im-1] = poin2[im-1] = 0;
03742      }
03743      for ( i = (k*2)+1; i <= lpat; i++ ) {
03744          S->used[i-k] = i;
03745          S->ktoi[i] = i-k-1;
03746          poin[i] = AB[i-k]->T.SB.MasterStart[AB[i-k]->T.SB.MasterBlock];
03747          poin2[i] = poin[i] + *(poin[i]);
03748      }
03749  /*
03750      the array poin tells the position of the i-th element of the S->tree
03751      'S->used' is a stack with the S->tree elements that need to be entered
03752      into the S->tree. at the beginning this is S->lPatch. during the
03753      sort there will be only very few elements.
03754      poin2 is the next value of poin. it has to be determined
03755      before the comparisons as the position or the size of the
03756      term indicated by poin may change.
03757      S->ktoi translates a S->tree element back to its stream number.
03758
03759      start the sort
03760  */
03761      level = S->lPatch;
03762  /*
03763      introduce one term
03764  */
03765      OneTerm:
03766      k = S->used[level];
03767      i = k + lpat - 1;
03768      if ( !(poin[k]) ) {
03769          do { if ( !(i »= 1) ) goto EndOfMerge; } while ( !S->tree[i] );
03770          if ( S->tree[i] == -1 ) {
03771              S->tree[i] = 0;
03772              level--;
03773              goto OneTerm;
03774          }
03775          k = S->tree[i];
03776          S->used[level] = k;
03777          S->tree[i] = 0;
03778      }
03779  /*
03780      move terms down the tree
03781  */
03782      while ( i »= 1 ) {
03783          if ( S->tree[i] > 0 ) {
03784              if ( ( c = CompareTerms(B0, poin[S->tree[i]],poin[k],(WORD)0) ) > 0 ) {
03785  /*
03786                  S->tree[i] is the smaller. Exchange and go on.
03787  */
03788                  S->used[level] = S->tree[i];
03789                  S->tree[i] = k;
03790                  k = S->used[level];
03791              }
03792              else if ( !c ) { /* Terms are equal */
03793  /*
03794                  S->TermsLeft--;
03795                  Here the terms are equal and their coefficients

```

```

03796             have to be added.
03797 */
03798     l1 = *( m1 = poin[S->tree[i]] );
03799     l2 = *( m2 = poin[k] );
03800     if ( S->PolyWise ) { /* Here we work with PolyFun */
03801         WORD *ttl, *w;
03802         ttl = m1;
03803         m1 += S->PolyWise;
03804         m2 += S->PolyWise;
03805         if ( S->PolyFlag == 2 ) {
03806             w = poly_ratfun_add(B0,m1,m2);
03807             if ( *ttl + w[1] - m1[1] > AM.MaxTer/((LONG)sizeof(WORD)) ) {
03808                 MLOCK(ErrorMessageLock);
03809                 MesPrint("Term too complex in PolyRatFun addition. MaxTermSize of %10l is
too small",AM.MaxTer);
03810                 MUNLOCK(ErrorMessageLock);
03811                 Terminate(-1);
03812             }
03813             AT0.WorkPointer = w;
03814             if ( w[FUNHEAD] == -SNUMBER && w[FUNHEAD+1] == 0 && w[1] > FUNHEAD ) {
03815                 goto cancelled;
03816             }
03817         }
03818         else {
03819             w = AT0.WorkPointer;
03820             if ( w + m1[1] + m2[1] > AT0.WorkTop ) {
03821                 MLOCK(ErrorMessageLock);
03822                 MesPrint("MasterMerge: A Workspace of %10l is too small",AM.WorkSize);
03823                 MUNLOCK(ErrorMessageLock);
03824                 Terminate(-1);
03825             }
03826             AddArgs(B0,m1,m2,w);
03827         }
03828         r1 = w[1];
03829         if ( r1 <= FUNHEAD
03830             || ( w[FUNHEAD] == -SNUMBER && w[FUNHEAD+1] == 0 ) )
03831             { goto cancelled; }
03832         if ( r1 == m1[1] ) {
03833             NCOPY(m1,w,r1);
03834         }
03835         else if ( r1 < m1[1] ) {
03836             r2 = m1[1] - r1;
03837             m2 = w + r1;
03838             m1 += m1[1];
03839             while ( --r1 >= 0 ) *--m1 = *--m2;
03840             m2 = m1 - r2;
03841             r1 = S->PolyWise;
03842             while ( --r1 >= 0 ) *--m1 = *--m2;
03843             *m1 -= r2;
03844             poin[S->tree[i]] = m1;
03845         }
03846         else {
03847             r2 = r1 - m1[1];
03848             m2 = ttl - r2;
03849             r1 = S->PolyWise;
03850             m1 = ttl;
03851             *m1 += r2;
03852             poin[S->tree[i]] = m2;
03853             NCOPY(m2,m1,r1);
03854             r1 = w[1];
03855             NCOPY(m2,w,r1);
03856         }
03857     }
03858 #ifdef WITHFLOAT
03859     else if ( AT.SortFloatMode ) {
03860         WORD *term1, *term2;
03861         term1 = poin[S->tree[i]];
03862         term2 = poin[k];
03863         if ( MergeWithFloat(B0, &term1,&term2) == 0 )
03864             goto cancelled;
03865         poin[S->tree[i]] = term1;
03866     }
03867 #endif
03868     else {
03869         r1 = *( m1 += l1 - 1 );
03870         m1 -= ABS(r1) - 1;
03871         r1 = ( ( r1 > 0 ) ? (r1-1) : (r1+1) ) » 1;
03872         r2 = *( m2 += l2 - 1 );
03873         m2 -= ABS(r2) - 1;
03874         r2 = ( ( r2 > 0 ) ? (r2-1) : (r2+1) ) » 1;
03875
03876         if ( AddRat(B0, (UWORD *)m1,r1, (UWORD *)m2,r2,coef,&r3) ) {
03877             MLOCK(ErrorMessageLock);
03878             MesCall("MasterMerge");
03879             MUNLOCK(ErrorMessageLock);
03880             SETERROR(-1)
03881         }

```

```

03882
03883
03884     if ( AN.ncmod != 0 ) {
03885         if ( ( AC.modmode & POSNEG ) != 0 ) {
03886             NormalModulus(coef,&r3);
03887         }
03888         else if ( BigLong(coef,r3,(UWORD *)AC.cmod,ABS(AN.ncmod)) >= 0 ) {
03889             WORD ii;
03890             SubPLon(coef,r3,(UWORD *)AC.cmod,ABS(AN.ncmod),coef,&r3);
03891             coef[r3] = 1;
03892             for ( ii = 1; ii < r3; ii++ ) coef[r3+ii] = 0;
03893         }
03894     }
03895     r3 *= 2;
03896     r33 = ( r3 > 0 ) ? ( r3 + 1 ) : ( r3 - 1 );
03897     if ( r3 < 0 ) r3 = -r3;
03898     if ( r1 < 0 ) r1 = -r1;
03899     r1 *= 2;
03900     r31 = r3 - r1;
03901     if ( !r3 ) { /* Terms cancel */
03902         ul = S->used[level] = S->tree[i];
03903         S->tree[i] = -1;
03904         /*
03905          * We skip to the next term in stream ul
03906          */
03907         im = *poin2[ul];
03908         poin[ul] = poin2[ul];
03909         ki = S->ktoi[ul];
03910         if ( (poin[ul] + im + COMPINC) >=
03911             AB[ki+1]->T.SB.MasterStop[AB[ki+1]->T.SB.MasterBlock]
03912             && im > 0 ) {
03913             /*
03914              * We made it to the end of the block. We have to
03915              * release the previous block and claim the next.
03916              */
03917             B = AB[ki+1];
03918             i = AT.SB.MasterBlock;
03919             if ( i == 1 ) {
03920                 UNLOCK(AT.SB.MasterBlockLock[AT.SB.MasterNumBlocks]);
03921             }
03922             else {
03923                 UNLOCK(AT.SB.MasterBlockLock[i-1]);
03924             }
03925             if ( i == AT.SB.MasterNumBlocks ) {
03926                 /*
03927                  * Move the remainder down into block 0
03928                  */
03929                 WORD *from, *to;
03930                 to = AT.SB.MasterStart[1];
03931                 from = AT.SB.MasterStop[i];
03932                 while ( from > poin[ul] ) *--to = *--from;
03933                 poin[ul] = to;
03934                 i = 1;
03935             }
03936             else { i++; }
03937             LOCK(AT.SB.MasterBlockLock[i]);
03938             AT.SB.MasterBlock = i;
03939             poin2[ul] = poin[ul] + im;
03940         }
03941         else {
03942             poin2[ul] += im;
03943         }
03944         S->used[++level] = k;
03945     }
03946     else if ( !r31 ) { /* copy coef into term1 */
03947         goto CopCof2;
03948     }
03949     else if ( r31 < 0 ) { /* copy coef into term1
03950                          * and adjust the length of term1 */
03951         goto CopCof2;
03952     }
03953     else {
03954         /*
03955          * this is the dreaded calamity.
03956          * is there enough space?
03957          */
03958         if ( (poin[S->tree[i]]+11+r31) >= poin2[S->tree[i]] ) {
03959             /*
03960              * no space! now the special trick for which
03961              * we left 2*maxlng spaces open at the beginning
03962              * of each patch.
03963              */
03964             if ( (11 + r31)*((LONG)sizeof(WORD)) >= AM.MaxTer ) {
03965                 MLOCK(ErrorMessageLock);
03966                 MesPrint("MasterMerge: Coefficient overflow during sort");
03967                 MUNLOCK(ErrorMessageLock);
03968                 goto ReturnError;

```

```

03969             }
03970             m2 = poin[S->tree[i]];
03971             m3 = ( poin[S->tree[i]] -= r31 );
03972             do { *m3++ = *m2++; } while ( m2 < m1 );
03973             m1 = m3;
03974         }
03975 CopCoef:
03976         *(poin[S->tree[i]]) += r31;
03977 CopCof2:
03978         m2 = (WORD *)coef; im = r3;
03979         NCOPY(m1,m2,im);
03980         *m1 = r33;
03981     }
03982 }
03983 /*
03984     Now skip to the next term in stream k.
03985 */
03986 NextTerm:
03987     im = poin2[k][0];
03988     poin[k] = poin2[k];
03989     ki = S->ktoi[k];
03990     if ( (poin[k] + im + COMPINC) >=
03991         AB[ki+1]->T.SB.MasterStop[AB[ki+1]->T.SB.MasterBlock]
03992         && im > 0 ) {
03993 /*
03994     We made it to the end of the block. We have to
03995     release the previous block and claim the next.
03996 */
03997         B = AB[ki+1];
03998         i = AT.SB.MasterBlock;
03999         if ( i == 1 ) {
04000             UNLOCK(AT.SB.MasterBlockLock[AT.SB.MasterNumBlocks]);
04001         }
04002         else {
04003             UNLOCK(AT.SB.MasterBlockLock[i-1]);
04004         }
04005         if ( i == AT.SB.MasterNumBlocks ) {
04006 /*
04007             Move the remainder down into block 0
04008 */
04009             WORD *from, *to;
04010             to = AT.SB.MasterStart[1];
04011             from = AT.SB.MasterStop[i];
04012             while ( from > poin[k] ) *--to = *--from;
04013             poin[k] = to;
04014             i = 1;
04015         }
04016         else { i++; }
04017         LOCK(AT.SB.MasterBlockLock[i]);
04018         AT.SB.MasterBlock = i;
04019         poin2[k] = poin[k] + im;
04020     }
04021     else {
04022         poin2[k] += im;
04023     }
04024     goto OneTerm;
04025 }
04026 }
04027 else if ( S->tree[i] < 0 ) {
04028     S->tree[i] = k;
04029     level--;
04030     goto OneTerm;
04031 }
04032 }
04033 /*
04034 found the smallest in the set. indicated by k.
04035 write to its destination.
04036 */
04037 S->TermsLeft++;
04038 if ( ( im = PutOut(B0,poin[k],&position,fout,1) ) < 0 ) {
04039     MLOCK(ErrorMessageLock);
04040     MesPrint("Called from MasterMerge with k = %d (stream %d)",k,S->ktoi[k]);
04041     MUNLOCK(ErrorMessageLock);
04042     goto ReturnError;
04043 }
04044 ADDPOS(S->SizeInFile[0],im);
04045 goto NextTerm;
04046 EndOfMerge:
04047 if ( FlushOut(&position,fout,1) ) goto ReturnError;
04048 ADDPOS(S->SizeInFile[0],1);
04049 CloseFile(fin->handle);
04050 remove(fin->name);
04051 fin->handle = -1;
04052 position = S->SizeInFile[0];
04053 MULPOS(position,sizeof(WORD));
04054 S->GenTerms = 0;
04055 for ( j = 1; j <= numberofworkers; j++ ) {

```



```

04056         S->GenTerms += AB[j]->T.SS->GenTerms;
04057     }
04058     WriteStats(&position,STATSPOSTSORT,NOCHECKLOGTYPE);
04059     Expressions[ARO.CurExpr].counter = S->TermsLeft;
04060     Expressions[ARO.CurExpr].size = position;
04061 /*
04062     Release all locks
04063 */
04064     for ( i = 1; i <= S->lPatch; i++ ) {
04065         B = AB[i];
04066         UNLOCK(AT.SB.MasterBlockLock[0]);
04067         if ( AT.SB.MasterBlock == 1 ) {
04068             UNLOCK(AT.SB.MasterBlockLock[AT.SB.MasterNumBlocks]);
04069         }
04070         else {
04071             UNLOCK(AT.SB.MasterBlockLock[AT.SB.MasterBlock-1]);
04072         }
04073         UNLOCK(AT.SB.MasterBlockLock[AT.SB.MasterBlock]);
04074     }
04075     AS.MasterSort = 0;
04076     return(0);
04077 ReturnError:
04078     for ( i = 1; i <= S->lPatch; i++ ) {
04079         B = AB[i];
04080         UNLOCK(AT.SB.MasterBlockLock[0]);
04081         if ( AT.SB.MasterBlock == 1 ) {
04082             UNLOCK(AT.SB.MasterBlockLock[AT.SB.MasterNumBlocks]);
04083         }
04084         else {
04085             UNLOCK(AT.SB.MasterBlockLock[AT.SB.MasterBlock-1]);
04086         }
04087         UNLOCK(AT.SB.MasterBlockLock[AT.SB.MasterBlock]);
04088     }
04089     AS.MasterSort = 0;
04090     return(-1);
04091 }
04092
04093 /*
04094     #] MasterMerge :
04095     #[ SortBotMasterMerge :
04096 */
04097
04098 #ifdef WITHSORTBOTS
04099
04115 int SortBotMasterMerge(void)
04116 {
04117     FILEHANDLE *fin, *fout;
04118     ALLPRIVATES *B = AB[0], *BB;
04119     POSITION position;
04120     SORTING *S = AT.SS;
04121     int i, j;
04122 /*
04123     Get the sortbots get to claim their writing blocks.
04124     We have to wait till all have been claimed because they also have to
04125     claim the last writing blocks of the workers to prevent the head of
04126     the circular buffer to overrun the tail.
04127
04128     Before waiting we can do some needed initializations.
04129     Also the master has to claim the last writing blocks of its input.
04130 */
04131     topsortbotavailables = 0;
04132     for ( i = numberofworkers+1; i <= numberofworkers+numberofsortbots; i++ ) {
04133         WakeupThread(i,INISORTBOT);
04134     }
04135
04136     AS.MasterSort = 1;
04137     fout = AR.outfile;
04138     numberofterms = 0;
04139     AR.CompressPointer[0] = 0;
04140     numberclaimed = 0;
04141     BB = AB[AT.SortBotIn1];
04142     LOCK(BB->T.SB.MasterBlockLock[BB->T.SB.MasterNumBlocks]);
04143     BB = AB[AT.SortBotIn2];
04144     LOCK(BB->T.SB.MasterBlockLock[BB->T.SB.MasterNumBlocks]);
04145
04146     MasterWaitAllSortBots();
04147 /*
04148     Now we can start up the workers. They will claim their writing blocks.
04149     Here the master will wait till all writing blocks have been claimed.
04150 */
04151     for ( i = 1; i <= numberofworkers; i++ ) {
04152         j = GetThread(i);
04153         WakeupThread(i,FINISHEXPRESSION);
04154     }
04155 /*
04156     Prepare the output file in the mean time.
04157 */

```

```

04158     if ( fout->handle >= 0 ) {
04159         PUTZERO(SortBotPosition);
04160         SeekFile(fout->handle,&SortBotPosition,SEEK_END);
04161         ADDPOS(SortBotPosition,((fout->POfill-fout->PObuffer)*sizeof(WORD)));
04162     }
04163     else {
04164         SETBASEPOSITION(SortBotPosition,(fout->POfill-fout->PObuffer)*sizeof(WORD));
04165     }
04166     MasterWaitAllBlocks();
04167     /*
04168     Now we can start the sortbots after which the master goes in
04169     sortbot mode to do its part of the job (the very final merge and
04170     the writing to output file).
04171     */
04172     topsortbotavailables = 0;
04173     for ( i = numberofworkers+1; i <= numberofworkers+numberofsortbots; i++ ) {
04174         WakeupThread(i,RUNSORTBOT);
04175     }
04176     if ( SortBotMerge(BHEAD0) ) {
04177         MLOCK(ErrorMessageLock);
04178         MesPrint("Called from SortBotMasterMerge");
04179         MUNLOCK(ErrorMessageLock);
04180         AS.MasterSort = 0;
04181         return(-1);
04182     }
04183     /*
04184     And next the cleanup
04185     */
04186     if ( S->file.handle >= 0 )
04187     {
04188         fin = &S->file;
04189         CloseFile(fin->handle);
04190         remove(fin->name);
04191         fin->handle = -1;
04192     }
04193     position = S->SizeInFile[0];
04194     MULPOS(position,sizeof(WORD));
04195     S->GenTerms = 0;
04196     for ( j = 1; j <= numberofworkers; j++ ) {
04197         S->GenTerms += AB[j]->T.SS->GenTerms;
04198     }
04199     S->TermsLeft = numberofterms;
04200     WriteStats(&position,STATSPOSTSORT,NOCHECKLOGTYPE);
04201     Expressions[AR.CurExpr].counter = S->TermsLeft;
04202     Expressions[AR.CurExpr].size = position;
04203     AS.MasterSort = 0;
04204     /*
04205     The next statement is to prevent one of the sortbots not having
04206     completely cleaned up before the next module starts.
04207     If this statement is omitted every once in a while one of the sortbots
04208     is still running when the next expression starts and misses its
04209     initialization. The result is usually disastrous.
04210     */
04211     MasterWaitAllSortBots();
04212
04213     return(0);
04214 }
04215
04216 #endif
04217
04218 /*
04219 #] SortBotMasterMerge :
04220 #[ SortBotMerge :
04221 */
04222
04223 #ifdef WITHSORTBOTS
04224
04225 int SortBotMerge(PHEAD0)
04226 {
04227     GETBIDENTITY
04228     ALLPRIVATES *Bin1 = AB[AT.SortBotIn1],*Bin2 = AB[AT.SortBotIn2];
04229     WORD *term1, *term2, *next, *wp;
04230     int blin1, blin2; /* Current block numbers */
04231     int error = 0;
04232     WORD l1, l2, *m1, *m2, *w, r1, r2, r3, r33, r3l, *ttl, ii;
04233     WORD *to, *from, im, c;
04234     UWORD *coef;
04235     SORTING *S = AT.SS;
04236     #ifdef WITHFLOAT
04237     WORD *fun1, *fun2, *fun3, *tstop1, *tstop2, size1, size2, size3, l3, jj, *m3;
04238     #endif
04239     /*
04240     Set the pointers to the input terms and the output space
04241     */
04242     coef = AN.SoScratC;
04243     blin1 = 1;
04244     blin2 = 1;

```

```

04250     if ( AT.identity == 0 ) {
04251         wp = AT.WorkPointer;
04252     }
04253     else {
04254         wp = AT.WorkPointer = AT.WorkSpace;
04255     }
04256     /*
04257     Get the locks for reading the input
04258     This means that we can start once these locks have been cleared
04259     which means that there will be input.
04260     */
04261     LOCK(Bin1->T.SB.MasterBlockLock[blin1]);
04262     LOCK(Bin2->T.SB.MasterBlockLock[blin2]);
04263
04264     term1 = Bin1->T.SB.MasterStart[blin1];
04265     term2 = Bin2->T.SB.MasterStart[blin2];
04266     AT.SB.FillBlock = 1;
04267     /*
04268     Now the main loop. Keep going until one of the two hits the end.
04269     */
04270     while ( *term1 && *term2 ) {
04271         if ( ( c = CompareTerms(BHEAD term1,term2,(WORD)0) ) > 0 ) {
04272             /*
04273             #[ One is smallest :
04274             */
04275             if ( SortBotOut(BHEAD term1) < 0 ) {
04276                 MLOCK(ErrorMessageLock);
04277                 MesPrint("Called from SortBotMerge with thread = %d",AT.identity);
04278                 MUNLOCK(ErrorMessageLock);
04279                 error = -1;
04280                 goto ReturnError;
04281             }
04282             im = *term1;
04283             next = term1 + im;
04284             if ( next >= Bin1->T.SB.MasterStop[blin1] || ( *next &&
04285                 next+*next+COMPINC > Bin1->T.SB.MasterStop[blin1] ) ) {
04286                 if ( blin1 == 1 ) {
04287                     UNLOCK(Bin1->T.SB.MasterBlockLock[Bin1->T.SB.MasterNumBlocks]);
04288                 }
04289                 else {
04290                     UNLOCK(Bin1->T.SB.MasterBlockLock[blin1-1]);
04291                 }
04292                 if ( blin1 == Bin1->T.SB.MasterNumBlocks ) {
04293                     /*
04294                     Move the remainder down into block 0
04295                     */
04296                     to = Bin1->T.SB.MasterStart[1];
04297                     from = Bin1->T.SB.MasterStop[Bin1->T.SB.MasterNumBlocks];
04298                     while ( from > next ) *--to = *--from;
04299                     next = to;
04300                     blin1 = 1;
04301                 }
04302                 else {
04303                     blin1++;
04304                 }
04305                 LOCK(Bin1->T.SB.MasterBlockLock[blin1]);
04306                 Bin1->T.SB.MasterBlock = blin1;
04307             }
04308             term1 = next;
04309             /*
04310             #[ One is smallest :
04311             */
04312         }
04313         else if ( c < 0 ) {
04314             /*
04315             #[ Two is smallest :
04316             */
04317             if ( SortBotOut(BHEAD term2) < 0 ) {
04318                 MLOCK(ErrorMessageLock);
04319                 MesPrint("Called from SortBotMerge with thread = %d",AT.identity);
04320                 MUNLOCK(ErrorMessageLock);
04321                 error = -1;
04322                 goto ReturnError;
04323             }
04324             next2: im = *term2;
04325             next = term2 + im;
04326             if ( next >= Bin2->T.SB.MasterStop[blin2] || ( *next
04327                 && next+*next+COMPINC > Bin2->T.SB.MasterStop[blin2] ) ) {
04328                 if ( blin2 == 1 ) {
04329                     UNLOCK(Bin2->T.SB.MasterBlockLock[Bin2->T.SB.MasterNumBlocks]);
04330                 }
04331                 else {
04332                     UNLOCK(Bin2->T.SB.MasterBlockLock[blin2-1]);
04333                 }
04334                 if ( blin2 == Bin2->T.SB.MasterNumBlocks ) {
04335                     /*
04336                     Move the remainder down into block 0

```

```

04337 */
04338         to = Bin2->T.SB.MasterStart[1];
04339         from = Bin2->T.SB.MasterStop[Bin2->T.SB.MasterNumBlocks];
04340         while ( from > next ) *--to = *--from;
04341         next = to;
04342         blin2 = 1;
04343     }
04344     else {
04345         blin2++;
04346     }
04347     LOCK(Bin2->T.SB.MasterBlockLock[blin2]);
04348     Bin2->T.SB.MasterBlock = blin2;
04349 }
04350 term2 = next;
04351 /*
04352     #] Two is smallest :
04353 */
04354 }
04355 else {
04356 /*
04357     #] Equal :
04358 */
04359     l1 = *( m1 = term1 );
04360     l2 = *( m2 = term2 );
04361     if ( S->PolyWise ) { /* Here we work with PolyFun */
04362         ttl = m1;
04363         m1 += S->PolyWise;
04364         m2 += S->PolyWise;
04365         if ( S->PolyFlag == 2 ) {
04366             AT.WorkPointer = wp;
04367             w = poly_ratfun_add(BHEAD m1,m2);
04368             if ( *ttl + w[1] - m1[1] > AM.MaxTer/((LONG)sizeof(WORD)) ) {
04369                 MLOCK(ErrorMessageLock);
04370                 MesPrint("Term too complex in PolyRatFun addition. MaxTermSize of %10l is too
small",AM.MaxTer);
04371                 MUNLOCK(ErrorMessageLock);
04372                 Terminate(-1);
04373             }
04374             AT.WorkPointer = wp;
04375             if ( w[FUNHEAD] == -SNUMBER && w[FUNHEAD+1] == 0 && w[1] > FUNHEAD ) {
04376                 goto cancelled;
04377             }
04378         }
04379         else {
04380             w = wp;
04381             if ( w + m1[1] + m2[1] > AT.WorkTop ) {
04382                 MLOCK(ErrorMessageLock);
04383                 MesPrint("SortBotMerge(%d): A MaxtermSize of %10l is too small",
04384                     AT.identity,AM.MaxTer/sizeof(WORD));
04385                 MesPrint("m1[1] = %d, m2[1] = %d, Space =
%1",m1[1],m2[1],(LONG) (AT.WorkTop-wp));
04386                 PrintTerm(term1,"term1");
04387                 PrintTerm(term2,"term2");
04388                 MesPrint("PolyWise = %d",S->PolyWise);
04389                 MUNLOCK(ErrorMessageLock);
04390                 Terminate(-1);
04391             }
04392             AddArgs(BHEAD m1,m2,w);
04393         }
04394         r1 = w[1];
04395         if ( r1 <= FUNHEAD
04396             || ( w[FUNHEAD] == -SNUMBER && w[FUNHEAD+1] == 0 ) )
04397             { goto cancelled; }
04398         if ( r1 == m1[1] ) {
04399             NCOPY(m1,w,r1);
04400         }
04401         else if ( r1 < m1[1] ) {
04402             r2 = m1[1] - r1;
04403             m2 = w + r1;
04404             m1 += m1[1];
04405             while ( --r1 >= 0 ) *--m1 = *--m2;
04406             m2 = m1 - r2;
04407             r1 = S->PolyWise;
04408             while ( --r1 >= 0 ) *--m1 = *--m2;
04409             *m1 -= r2;
04410             term1 = m1;
04411         }
04412         else {
04413             r2 = r1 - m1[1];
04414             m2 = ttl - r2;
04415             r1 = S->PolyWise;
04416             m1 = ttl;
04417             *m1 += r2;
04418             term1 = m2;
04419             NCOPY(m2,m1,r1);
04420             r1 = w[1];
04421             NCOPY(m2,w,r1);

```

```

04422     }
04423     }
04424     #ifdef WITHFLOAT
04425     else if ( AT.SortFloatMode ) {
04426     /*
04427         The terms are in m1/term1 and m2/term2 and their length in l1 and l2.
04428         We have to locate the floats which have already been
04429         verified in the compare routine. Once we have the new
04430         term we can jump to the code that writes away the
04431         result after adding the rationals.
04432     */
04433         tstop1 = m1+l1; size1 = tstop1[-1]; tstop1 -= ABS(size1);
04434         tstop2 = m2+l2; size2 = tstop2[-1]; tstop2 -= ABS(size2);
04435         if ( AT.SortFloatMode == 3 ) {
04436             fun1 = m1+l1;
04437             while ( fun1[0] != FLOATFUN && fun1+fun1[1] < tstop1 ) {
04438                 fun1 += fun1[1];
04439             }
04440             if ( size1 < 0 ) fun1[FUNHEAD+3] = -fun1[FUNHEAD+3];
04441             UnpackFloat(aux1,fun1);
04442             fun2 = m2+l1;
04443             while ( fun2[0] != FLOATFUN && fun2+fun2[1] < tstop2 ) {
04444                 fun2 += fun2[1];
04445             }
04446             if ( size2 < 0 ) fun2[FUNHEAD+3] = -fun2[FUNHEAD+3];
04447             UnpackFloat(aux2,fun2);
04448         }
04449         else if ( AT.SortFloatMode == 1 ) {
04450             fun1 = m1+l1;
04451             while ( fun1[0] != FLOATFUN && fun1+fun1[1] < tstop1 ) {
04452                 fun1 += fun1[1];
04453             }
04454             if ( size1 < 0 ) fun1[FUNHEAD+3] = -fun1[FUNHEAD+3];
04455             UnpackFloat(aux1,fun1);
04456             fun2 = tstop2;
04457             RatToFloat(aux2,(UWORD *)fun2,size2);
04458         }
04459         else if ( AT.SortFloatMode == 2 ) {
04460             fun1 = tstop1;
04461             RatToFloat(aux1,(UWORD *)fun1,size1);
04462             fun2 = m2+l1;
04463             while ( fun2[0] != FLOATFUN && fun2+fun2[1] < tstop2 ) {
04464                 fun2 += fun2[1];
04465             }
04466             if ( size2 < 0 ) fun2[FUNHEAD+3] = -fun2[FUNHEAD+3];
04467             UnpackFloat(aux2,fun2);
04468         }
04469         else {
04470             MLOCK(ErrorMessageLock);
04471             MesPrint("Illegal value %d for AT.SortFloatMode in
SortBotMerge.",AT.SortFloatMode);
04472             MUNLOCK(ErrorMessageLock);
04473             Terminate(-1);
04474             return(0);
04475         }
04476         mpf_add(aux3,aux1,aux2);
04477         size3 = mpf_sgn(aux3);
04478         if ( size3 == 0 ) { /* Cancelling! Rare! */
04479             goto cancelled;
04480         }
04481         else if ( size3 < 0 ) mpf_neg(aux3,aux3);
04482         fun3 = TermMalloc("MasterMerge");
04483         PackFloat(fun3,aux3);
04484         l3 = fun3[1]+(fun1-m1)+3; /* new size */
04485
04486         if ( l3 != l1 ) {
04487         /*
04488             Copy it all to wp
04489         */
04490             if ( (l3)*(LONG)sizeof(WORD) >= AM.MaxTer ) {
04491                 MLOCK(ErrorMessageLock);
04492                 MesPrint("MasterMerge: Coefficient overflow during sort");
04493                 MUNLOCK(ErrorMessageLock);
04494                 goto ReturnError;
04495             }
04496             m3 = wp; m2 = term1;
04497             while ( m2 < fun1 ) *m3++ = *m2++;
04498             for ( jj = 0; jj < fun3[1]; jj++ ) *m3++ = fun3[jj];
04499             *m3++ = 1; *m3++ = 1;
04500             *m3++ = size3 < 0 ? -3: 3;
04501             *wp = m3-wp;
04502             TermFree(fun3,"MasterMerge");
04503             goto PutOutwp;
04504         }
04505         for ( jj = 0; jj < fun3[1]; jj++ ) fun1[jj] = fun3[jj];
04506         fun1 += fun3[1];
04507         *fun1++ = 1; *fun1++ = 1;

```

```

04508         *funl++ = size3 < 0 ? -3: 3;
04509         *term1 = funl-term1;
04510         TermFree(fun3,"MasterMerge");
04511     }
04512 #endif
04513     else {
04514         r1 = *( m1 += l1 - 1 );
04515         m1 -= ABS(r1) - 1;
04516         r1 = ( ( r1 > 0 ) ? (r1-1) : (r1+1) ) » 1;
04517         r2 = *( m2 += l2 - 1 );
04518         m2 -= ABS(r2) - 1;
04519         r2 = ( ( r2 > 0 ) ? (r2-1) : (r2+1) ) » 1;
04520
04521         if ( AddRat(BHEAD (UWORD *)m1,r1,(UWORD *)m2,r2,coef,&r3) ) {
04522             MLOCK(ErrorMessageLock);
04523             MesCall("SortBotMerge");
04524             MUNLOCK(ErrorMessageLock);
04525             SETERROR(-1)
04526         }
04527
04528         if ( AN.ncmod != 0 ) {
04529             if ( ( AC.modmode & POSNEG ) != 0 ) {
04530                 NormalModulus(coef,&r3);
04531             }
04532             else if ( BigLong(coef,r3,(UWORD *)AC.cmod,ABS(AN.ncmod)) >= 0 ) {
04533                 SubPLon(coef,r3,(UWORD *)AC.cmod,ABS(AN.ncmod),coef,&r3);
04534                 coef[r3] = 1;
04535                 for ( ii = 1; ii < r3; ii++ ) coef[r3+ii] = 0;
04536             }
04537         }
04538         if ( !r3 ) { goto cancelled; }
04539         r3 *= 2;
04540         r33 = ( r3 > 0 ) ? ( r3 + 1 ) : ( r3 - 1 );
04541         if ( r3 < 0 ) r3 = -r3;
04542         if ( r1 < 0 ) r1 = -r1;
04543         r1 *= 2;
04544         r31 = r3 - r1;
04545         if ( !r31 ) { /* copy coef into term1 */
04546             m2 = (WORD *)coef; im = r3;
04547             NCOPY(m1,m2,im);
04548             *m1 = r33;
04549         }
04550         else {
04551             to = wp; from = term1;
04552             while ( from < m1 ) *to++ = *from++;
04553             from = (WORD *)coef; im = r3;
04554             NCOPY(to,from,im);
04555             *to++ = r33;
04556             wp[0] = to - wp;
04557 PutOutwp:
04558             if ( SortBotOut(BHEAD wp) < 0 ) {
04559                 MLOCK(ErrorMessageLock);
04560                 MesPrint("Called from SortBotMerge with thread = %d",AT.identity);
04561                 MUNLOCK(ErrorMessageLock);
04562                 error = -1;
04563                 goto ReturnError;
04564             }
04565             goto cancelled;
04566         }
04567     }
04568     if ( SortBotOut(BHEAD term1) < 0 ) {
04569         MLOCK(ErrorMessageLock);
04570         MesPrint("Called from SortBotMerge with thread = %d",AT.identity);
04571         MUNLOCK(ErrorMessageLock);
04572         error = -1;
04573         goto ReturnError;
04574     }
04575     cancelled;; /* Now we need two new terms */
04576     im = *term1;
04577     next = term1 + im;
04578     if ( next >= Bin1->T.SB.MasterStop[blin1] || ( *next &&
04579         next+*next+COMPINC > Bin1->T.SB.MasterStop[blin1] ) ) {
04580         if ( blin1 == 1 ) {
04581             UNLOCK(Bin1->T.SB.MasterBlockLock[Bin1->T.SB.MasterNumBlocks]);
04582         }
04583         else {
04584             UNLOCK(Bin1->T.SB.MasterBlockLock[blin1-1]);
04585         }
04586         if ( blin1 == Bin1->T.SB.MasterNumBlocks ) {
04587             /*
04588              Move the remainder down into block 0
04589             */
04590             to = Bin1->T.SB.MasterStart[1];
04591             from = Bin1->T.SB.MasterStop[Bin1->T.SB.MasterNumBlocks];
04592             while ( from > next ) *--to = *--from;
04593             next = to;
04594             blin1 = 1;

```

```

04595         }
04596         else {
04597             blin1++;
04598         }
04599         LOCK(Bin1->T.SB.MasterBlockLock[blin1]);
04600         Bin1->T.SB.MasterBlock = blin1;
04601     }
04602     term1 = next;
04603     goto next2;
04604 /*
04605     #] Equal :
04606 */
04607 }
04608 }
04609 /*
04610 Copy the tail
04611 */
04612 if ( *term1 ) {
04613 /*
04614     #] Tail in one :
04615 */
04616     while ( *term1 ) {
04617         if ( SortBotOut(BHEAD term1) < 0 ) {
04618             MLOCK(ErrorMessageLock);
04619             MesPrint("Called from SortBotMerge with thread = %d",AT.identity);
04620             MUNLOCK(ErrorMessageLock);
04621             error = -1;
04622             goto ReturnError;
04623         }
04624         im = *term1;
04625         next = term1 + im;
04626         if ( next >= Bin1->T.SB.MasterStop[blin1] || ( *next &&
04627             next+*next+COMPINC > Bin1->T.SB.MasterStop[blin1] ) ) {
04628             if ( blin1 == 1 ) {
04629                 UNLOCK(Bin1->T.SB.MasterBlockLock[Bin1->T.SB.MasterNumBlocks]);
04630             }
04631             else {
04632                 UNLOCK(Bin1->T.SB.MasterBlockLock[blin1-1]);
04633             }
04634             if ( blin1 == Bin1->T.SB.MasterNumBlocks ) {
04635 /*
04636             Move the remainder down into block 0
04637 */
04638                 to = Bin1->T.SB.MasterStart[1];
04639                 from = Bin1->T.SB.MasterStop[Bin1->T.SB.MasterNumBlocks];
04640                 while ( from > next ) *--to = *--from;
04641                 next = to;
04642                 blin1 = 1;
04643             }
04644             else {
04645                 blin1++;
04646             }
04647             LOCK(Bin1->T.SB.MasterBlockLock[blin1]);
04648             Bin1->T.SB.MasterBlock = blin1;
04649         }
04650         term1 = next;
04651     }
04652 /*
04653     #] Tail in one :
04654 */
04655 }
04656 else if ( *term2 ) {
04657 /*
04658     #] Tail in two :
04659 */
04660     while ( *term2 ) {
04661         if ( SortBotOut(BHEAD term2) < 0 ) {
04662             MLOCK(ErrorMessageLock);
04663             MesPrint("Called from SortBotMerge with thread = %d",AT.identity);
04664             MUNLOCK(ErrorMessageLock);
04665             error = -1;
04666             goto ReturnError;
04667         }
04668         im = *term2;
04669         next = term2 + im;
04670         if ( next >= Bin2->T.SB.MasterStop[blin2] || ( *next
04671             && next+*next+COMPINC > Bin2->T.SB.MasterStop[blin2] ) ) {
04672             if ( blin2 == 1 ) {
04673                 UNLOCK(Bin2->T.SB.MasterBlockLock[Bin2->T.SB.MasterNumBlocks]);
04674             }
04675             else {
04676                 UNLOCK(Bin2->T.SB.MasterBlockLock[blin2-1]);
04677             }
04678             if ( blin2 == Bin2->T.SB.MasterNumBlocks ) {
04679 /*
04680             Move the remainder down into block 0
04681 */

```

```

04682         to = Bin2->T.SB.MasterStart[1];
04683         from = Bin2->T.SB.MasterStop[Bin2->T.SB.MasterNumBlocks];
04684         while ( from > next ) *--to = *--from;
04685         next = to;
04686         blin2 = 1;
04687     }
04688     else {
04689         blin2++;
04690     }
04691     LOCK(Bin2->T.SB.MasterBlockLock[blin2]);
04692     Bin2->T.SB.MasterBlock = blin2;
04693 }
04694 term2 = next;
04695 }
04696 /*
04697     #] Tail in two :
04698 */
04699 }
04700 SortBotOut(BHEAD 0);
04701 ReturnError;;
04702 /*
04703     Release all locks
04704 */
04705 UNLOCK(Bin1->T.SB.MasterBlockLock[blin1]);
04706 if ( blin1 > 1 ) {
04707     UNLOCK(Bin1->T.SB.MasterBlockLock[blin1-1]);
04708 }
04709 else {
04710     UNLOCK(Bin1->T.SB.MasterBlockLock[Bin1->T.SB.MasterNumBlocks]);
04711 }
04712 UNLOCK(Bin2->T.SB.MasterBlockLock[blin2]);
04713 if ( blin2 > 1 ) {
04714     UNLOCK(Bin2->T.SB.MasterBlockLock[blin2-1]);
04715 }
04716 else {
04717     UNLOCK(Bin2->T.SB.MasterBlockLock[Bin2->T.SB.MasterNumBlocks]);
04718 }
04719 if ( AT.identity > 0 ) {
04720     UNLOCK(AT.SB.MasterBlockLock[AT.SB.FillBlock]);
04721 }
04722 /*
04723     And that was all folks
04724 */
04725 return(error);
04726 }
04727
04728 #endif
04729
04730 /*
04731     #] SortBotMerge :
04732     #[ IniSortBlocks :
04733 */
04734
04735 static int SortBlocksInitialized = 0;
04736
04737 int IniSortBlocks(int numworkers)
04738 {
04739     ALLPRIVATES *B;
04740     SORTING *S;
04741     LONG totalsize, workersize, blocksize, numberofterms;
04742     int maxter, id, j;
04743     int numberofblocks = NUMBEROFBLOCKSINSERT, numparts;
04744     WORD *w;
04745
04746     if ( SortBlocksInitialized ) return(0);
04747     SortBlocksInitialized = 1;
04748     if ( numworkers == 0 ) return(0);
04749
04750 #ifdef WITHSORTBOTS
04751     if ( numworkers > 2 ) {
04752         numparts = 2*numworkers - 2;
04753         numberofblocks = numberofblocks/2;
04754     }
04755     else {
04756         numparts = numworkers;
04757     }
04758 #else
04759     numparts = numworkers;
04760 #endif
04761
04762     S = AM.S0;
04763     totalsize = S->LargeSize + S->SmallEsize;
04764     workersize = totalsize / numparts;
04765     maxter = AM.MaxTer/sizeof(WORD);
04766     blocksize = ( workersize - maxter )/numberofblocks;
04767     numberofterms = blocksize / maxter;
04768     if ( numberofterms < MINIMUMNUMBEROFTERMS ) {
04769

```



```

04775         This should have been taken care of in RecalcSetups.
04776  */
04777         MesPrint("We have a problem with the size of the blocks in IniSortBlocks");
04778         Terminate(-1);
04779     }
04780  /*
04781     Layout:  For each worker
04782              block 0: size is maxter WORDS
04783              numberofblocks blocks of size blocksize WORDS
04784  */
04785     w = S->lBuffer;
04786     if ( w == 0 ) w = S->sBuffer;
04787     for ( id = 1; id <= numparts; id++ ) {
04788         B = AB[id];
04789         AT.SB.MasterBlockLock = (pthread_mutex_t *)Malloc1(
04790             sizeof(pthread_mutex_t)*(numberofblocks+1), "MasterBlockLock");
04791         AT.SB.MasterStart = (WORD **)Malloc1(sizeof(WORD *)*(numberofblocks+1)*3, "MasterBlock");
04792         AT.SB.MasterFill = AT.SB.MasterStart + (numberofblocks+1);
04793         AT.SB.MasterStop = AT.SB.MasterFill + (numberofblocks+1);
04794         AT.SB.MasterNumBlocks = numberofblocks;
04795         AT.SB.MasterBlock = 0;
04796         AT.SB.FillBlock = 0;
04797         AT.SB.MasterFill[0] = AT.SB.MasterStart[0] = w;
04798         w += maxter;
04799         AT.SB.MasterStop[0] = w;
04800         AT.SB.MasterBlockLock[0] = dummylock;
04801         for ( j = 1; j <= numberofblocks; j++ ) {
04802             AT.SB.MasterFill[j] = AT.SB.MasterStart[j] = w;
04803             w += blocksize;
04804             AT.SB.MasterStop[j] = w;
04805             AT.SB.MasterBlockLock[j] = dummylock;
04806         }
04807     }
04808     if ( w > S->sTop2 ) {
04809         MesPrint("Counting problem in IniSortBlocks");
04810         Terminate(-1);
04811     }
04812     return(0);
04813 }
04814
04815  /*
04816     #] IniSortBlocks :
04817     #[ UpdateSortBlocks :
04818  */
04819
04824 int UpdateSortBlocks(int numworkers)
04825 {
04826     ALLPRIVATES *B;
04827     SORTING *S;
04828     LONG totalsize, workersize, blocksize, numberofterms;
04829     int maxter, id, j;
04830     int numberofblocks = NUMBEROFBLOCKSINSORT, numparts;
04831     WORD *w;
04832
04833     if ( numworkers == 0 ) return(0);
04834
04835     #ifdef WITHSORTBOTS
04836     if ( numworkers > 2 ) {
04837         numparts = 2*numworkers - 2;
04838         numberofblocks = numberofblocks/2;
04839     }
04840     else {
04841         numparts = numworkers;
04842     }
04843     #else
04844     numparts = numworkers;
04845     #endif
04846     S = AM.S0;
04847     totalsize = S->LargeSize + S->SmallEsize;
04848     workersize = totalsize / numparts;
04849     maxter = AM.MaxTer/sizeof(WORD);
04850     blocksize = ( workersize - maxter )/numberofblocks;
04851     numberofterms = blocksize / maxter;
04852     if ( numberofterms < MINIMUMNUMBEROFTERMS ) {
04853  /*
04854         This should have been taken care of in RecalcSetups.
04855  */
04856         MesPrint("We have a problem with the size of the blocks in UpdateSortBlocks");
04857         Terminate(-1);
04858     }
04859  /*
04860     Layout:  For each worker
04861              block 0: size is maxter WORDS
04862              numberofblocks blocks of size blocksize WORDS
04863  */
04864     w = S->lBuffer;
04865     if ( w == 0 ) w = S->sBuffer;

```

```

04866     for ( id = 1; id <= numparts; id++ ) {
04867         B = AB[id];
04868         AT.SB.MasterFill[0] = AT.SB.MasterStart[0] = w;
04869         w += maxter;
04870         AT.SB.MasterStop[0] = w;
04871         for ( j = 1; j <= numberofblocks; j++ ) {
04872             AT.SB.MasterFill[j] = AT.SB.MasterStart[j] = w;
04873             w += blocksize;
04874             AT.SB.MasterStop[j] = w;
04875         }
04876     }
04877     if ( w > S->sTop2 ) {
04878         MesPrint("Counting problem in UpdateSortBlocks");
04879         Terminate(-1);
04880     }
04881     return(0);
04882 }
04883
04884 /*
04885  #] UpdateSortBlocks :
04886  #[ DefineSortBotTree :
04887  */
04888
04889 #ifndef WITHSORTBOTS
04890
04891 void DefineSortBotTree(void)
04892 {
04893     ALLPRIVATES *B;
04894     int n, i, from;
04895     if ( numberofworkers <= 2 ) return;
04896     n = numberofworkers*2-2;
04897     for ( i = numberofworkers+1, from = 1; i <= n; i++ ) {
04898         B = AB[i];
04899         AT.SortBotIn1 = from++;
04900         AT.SortBotIn2 = from++;
04901     }
04902     B = AB[0];
04903     AT.SortBotIn1 = from++;
04904     AT.SortBotIn2 = from++;
04905 }
04906
04907 #endif
04908
04909 /*
04910  #] DefineSortBotTree :
04911  #[ GetTerm2 :
04912  */
04913
04914 Routine does a GetTerm but only when a bracket index is involved and
04915 only from brackets that have been judged not suitable for treatment
04916 as complete brackets by a single worker. Whether or not a bracket should
04917 be treated by a single worker is decided by TreatIndexEntry
04918
04919 /*
04920 WORD GetTerm2(PHEAD WORD *term)
04921 {
04922     WORD *ttco, *tt, retval;
04923     LONG n,i;
04924     FILEHANDLE *fi;
04925     EXPRESSIONS e = AN.expr;
04926     BRACKETINFO *b = e->bracketinfo;
04927     BRACKETINDEX *bi = b->indexbuffer;
04928     POSITION where, eofnfile = AS.OldOnFile[e-Expressions], bstart, bnext;
04929
04930     /*
04931     1: Get the current position.
04932     */
04933     switch ( e->status ) {
04934         case UNHIDELEXPRESSION:
04935         case UNHIDEGEXPRESSION:
04936         case DROPHLEXPRESSION:
04937         case DROPHGEXPRESSION:
04938         case HIDDENLEXPRESSION:
04939         case HIDDENGEXPRESSION:
04940             fi = AR.hidefile;
04941             break;
04942         default:
04943             fi = AR.infile;
04944             break;
04945     }
04946     if ( AR.KeptInHold ) {
04947         retval = GetTerm(BHEAD term);
04948         return(retval);
04949     }
04950     SeekScratch(fi,&where);
04951     if ( AN.lastinindex < 0 ) {
04952         /*
04953         We have to test whether we have to do the first bracket
04954         */

```

```

04958         if ( ( n = TreatIndexEntry(BHEAD 0) ) <= 0 ) {
04959             AN.lastindex = n;
04960             where = bi[n].start;
04961             ADD2POS(where,eonfile);
04962             SetScratch(fi,&where);
04963         /*
04964             Put the bracket in the Compress buffer.
04965         */
04966             ttco = AR.CompressBuffer;
04967             tt = b->bracketbuffer + bi[0].bracket;
04968             i = *tt;
04969             NCOPY(ttco,tt,i)
04970             AR.CompressPointer = ttco;
04971             retval = GetTerm(BHEAD term);
04972             return(retval);
04973         }
04974         else AN.lastindex = n-1;
04975     }
04976     /*
04977     2: Find the corresponding index number
04978     a: test whether it is in the current bracket
04979     */
04980     n = AN.lastindex;
04981     bstart = bi[n].start;
04982     ADD2POS(bstart,eonfile);
04983     bnext = bi[n].next;
04984     ADD2POS(bnext,eonfile);
04985     if ( ISLESSPOS(bstart,where) && ISLESSPOS(where,bnext) ) {
04986         retval = GetTerm(BHEAD term);
04987         return(retval);
04988     }
04989     for ( n++; n < b->indexfill; n++ ) {
04990         i = TreatIndexEntry(BHEAD n);
04991         if ( i <= 0 ) {
04992             /*
04993                 Put the bracket in the Compress buffer.
04994             */
04995             ttco = AR.CompressBuffer;
04996             tt = b->bracketbuffer + bi[n].bracket;
04997             i = *tt;
04998             NCOPY(ttco,tt,i)
04999             AR.CompressPointer = ttco;
05000             AN.lastindex = n;
05001             where = bi[n].start;
05002             ADD2POS(where,eonfile);
05003             SetScratch(fi,&(where));
05004             retval = GetTerm(BHEAD term);
05005             return(retval);
05006         }
05007         else n += i - 1;
05008     }
05009     return(0);
05010 }
05011
05012 /*
05013 #] GetTerm2 :
05014 #[ TreatIndexEntry :
05015 */
05024 int TreatIndexEntry(PHEAD LONG n)
05025 {
05026     BRACKETINFO *b = AN.expr->bracketinfo;
05027     LONG numbra = b->indexfill - 1, i;
05028     LONG totterms;
05029     BRACKETINDEX *bi;
05030     POSITION pos1, average;
05031     /*
05032     1: number of the bracket should be such that there is one bucket
05033         for each worker remaining.
05034     */
05035     if ( ( numbra - n ) <= numberofworkers ) return(0);
05036     /*
05037     2: size of the bracket contents should be less than what remains in
05038         the expression divided by the number of workers.
05039     */
05040     bi = b->indexbuffer;
05041     DIFPOS(pos1,bi[numbra].next,bi[n].next); /* Size of what remains */
05042     BASEPOSITION(average) = DIVPOS(pos1,(3*numberofworkers));
05043     DIFPOS(pos1,bi[n].next,bi[n].start); /* Size of the current bracket */
05044     if ( ISLESSPOS(average,pos1) ) return(0);
05045     /*
05046     It passes.
05047     Now check whether we can do more brackets
05048     */
05049     totterms = bi->termsinbracket;
05050     if ( totterms > 2*AC.ThreadBucketSize ) return(1);
05051     for ( i = 1; i < numbra-n; i++ ) {
05052         DIFPOS(pos1,bi[n+i].next,bi[n].start); /* Size of the combined brackets */

```

```

05053         if ( ISLESSPOS(average,pos1) ) return(i);
05054         totterms += bi->termsinbracket;
05055         if ( totterms > 2*AC.ThreadBucketSize ) return(i+1);
05056     }
05057     /*
05058     We have a problem at the end of the system. Just do one.
05059     */
05060     return(1);
05061 }
05062 /*
05063 #] TreatIndexEntry :
05064 #[ SetHideFiles :
05065 */
05066 void SetHideFiles(void) {
05067     int i;
05068     ALLPRIVATES *B, *B0 = AB[0];
05069     for ( i = 1; i <= numberofworkers; i++ ) {
05070         B = AB[i];
05071         AR.hidefile->handle = AR0.hidefile->handle;
05072         if ( AR.hidefile->handle < 0 ) {
05073             AR.hidefile->PObuffer = AR0.hidefile->PObuffer;
05074             AR.hidefile->POstop = AR0.hidefile->POstop;
05075             AR.hidefile->POfill = AR0.hidefile->POfill;
05076             AR.hidefile->POfull = AR0.hidefile->POfull;
05077             AR.hidefile->POsize = AR0.hidefile->POsize;
05078             AR.hidefile->POposition = AR0.hidefile->POposition;
05079             AR.hidefile->filesize = AR0.hidefile->filesize;
05080         }
05081         else {
05082             AR.hidefile->PObuffer = AR.hidefile->wPObuffer;
05083             AR.hidefile->POstop = AR.hidefile->wPOstop;
05084             AR.hidefile->POfill = AR.hidefile->wPOfill;
05085             AR.hidefile->POfull = AR.hidefile->wPOfull;
05086             AR.hidefile->POsize = AR.hidefile->wPOsize;
05087             PUTZERO(AR.hidefile->POposition);
05088         }
05089     }
05090 }
05091 }
05092 }
05093 /*
05094 #] SetHideFiles :
05095 #[ IniFbufs :
05096 */
05097 void IniFbufs(void)
05098 {
05099     int i;
05100     for ( i = 0; i < AM.totalnumberofthreads; i++ ) {
05101         IniFbuffer(AB[i]->T.fbufnum);
05102     }
05103 }
05104 /*
05105 #] IniFbufs :
05106 #[ SetMods :
05107 */
05108 void SetMods(void)
05109 {
05110     ALLPRIVATES *B;
05111     int i, n, j;
05112     for ( j = 0; j < AM.totalnumberofthreads; j++ ) {
05113         B = AB[j];
05114         AN.ncmod = AC.ncmod;
05115         if ( AN.ncmod != 0 ) M_free(AN.ncmod, "AN.ncmod");
05116         n = ABS(AN.ncmod);
05117         AN.ncmod = (UWORD *)Malloc1(sizeof(WORD)*n, "AN.ncmod");
05118         for ( i = 0; i < n; i++ ) AN.ncmod[i] = AC.ncmod[i];
05119     }
05120 }
05121 /*
05122 #] SetMods :
05123 #[ UnSetMods :
05124 */
05125 void UnSetMods(void)
05126 {
05127     ALLPRIVATES *B;
05128     int j;
05129     for ( j = 0; j < AM.totalnumberofthreads; j++ ) {
05130         B = AB[j];
05131         if ( AN.ncmod != 0 ) M_free(AN.ncmod, "AN.ncmod");
05132         AN.ncmod = 0;
05133     }
05134 }

```

```

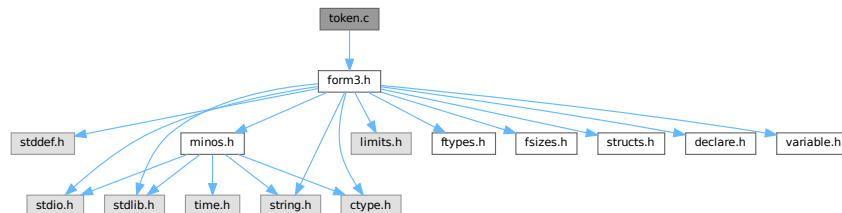
05140 }
05141
05142 /*
05143  #] UnSetMods :
05144  #[ find_Horner_MCTS_expand_tree_threaded :
05145 */
05146
05147 void find_Horner_MCTS_expand_tree_threaded(void) {
05148     int id;
05149     while (( id = GetAvailableThread() ) < 0)
05150         MasterWait();
05151     WakeupThread(id,MCTSEXPAENDTREE);
05152 }
05153
05154 /*
05155  #] find_Horner_MCTS_expand_tree_threaded :
05156  #[ optimize_expression_given_Horner_threaded :
05157 */
05158
05159 extern void optimize_expression_given_Horner_threaded(void) {
05160     int id;
05161     while (( id = GetAvailableThread() ) < 0)
05162         MasterWait();
05163     WakeupThread(id,OPTIMIZEEXPRESSION);
05164 }
05165
05166 /*
05167  #] optimize_expression_given_Horner_threaded :
05168 */
05169
05170 #endif

```

4.144 token.c File Reference

```
#include "form3.h"
```

Include dependency graph for token.c:



Macros

- #define [CHECKPOLY](#) {if(polyflag)MesPrint("&Illegal use of polynomial function"); polyflag = 0; }

Functions

- int [tokenize](#) (UBYTE *in, WORD leftright)
- void [WriteTokens](#) (SBYTE *in)
- int [simp1token](#) (SBYTE *s)
- int [simpwtoken](#) (SBYTE *s)
- int [simp2token](#) (SBYTE *s)
- int [simp3atoken](#) (SBYTE *s, int mode)
- int [simp3btoken](#) (SBYTE *s, int mode)
- int [simp4token](#) (SBYTE *s)
- int [simp5token](#) (SBYTE *s, int mode)
- int [simp6token](#) (SBYTE *tokens, int mode)

Variables

- `char * ttypes []`

4.144.1 Detailed Description

The tokenizer. This is a part of the compiler that does an intermediate type of translation. It does look up the names etc and can do a number of optimizations. The resulting output is a stream of bytes which can be processed by the code generator (in the file [compiler.c](#))

Definition in file [token.c](#).

4.144.2 Macro Definition Documentation

4.144.2.1 CHECKPOLY

```
#define CHECKPOLY {if(polyflag)MesPrint("&Illegal use of polynomial function"); polyflag = 0;
}
```

Definition at line 56 of file [token.c](#).

4.144.3 Function Documentation

4.144.3.1 tokenize()

```
int tokenize (
    UBYTE * in,
    WORD leftright )
```

Definition at line 58 of file [token.c](#).

4.144.3.2 WriteTokens()

```
void WriteTokens (
    SBYTE * in )
```

Definition at line 657 of file [token.c](#).

4.144.3.3 simp1token()

```
int simp1token (
    SBYTE * s )
```

Definition at line 705 of file [token.c](#).

4.144.3.4 simpwtoken()

```
int simpwtoken (
    SBYTE * s )
```

Definition at line 794 of file [token.c](#).

4.144.3.5 simp2token()

```
int simp2token (
    SBYTE * s )
```

Definition at line 948 of file [token.c](#).

4.144.3.6 simp3atoken()

```
int simp3atoken (
    SBYTE * s,
    int mode )
```

Definition at line 1196 of file [token.c](#).

4.144.3.7 simp3btoken()

```
int simp3btoken (
    SBYTE * s,
    int mode )
```

Definition at line 1437 of file [token.c](#).

4.144.3.8 simp4token()

```
int simp4token (
    SBYTE * s )
```

Definition at line 1822 of file [token.c](#).

4.144.3.9 simp5token()

```
int simp5token (
    SBYTE * s,
    int mode )
```

Definition at line 1982 of file [token.c](#).

4.144.3.10 simp6token()

```
int simp6token (
    SBYTE * tokens,
    int mode )
```

Definition at line 2028 of file [token.c](#).

4.144.4 Variable Documentation

4.144.4.1 ttypes

```
char* ttypes[]
```

Initial value:

```
= { "\n", "S", "I", "V", "F", "set", "E", "dotp", "#",
    "sub", "d_", "S", "dub", "(", ")", "?", "??", ".", "[", "]",
    ",", "(", ")", ":", ":", "/", "^", "+", "-", "!", "end", "{", "}",
    "N_?", "conj", "()", "#d", "^d", "_", "snum" }
```

Definition at line 652 of file [token.c](#).

4.145 token.c

[Go to the documentation of this file.](#)

```
00001
00008 /* # [ License : */
00009 /*
00010 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00011 * When using this file you are requested to refer to the publication
00012 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00013 * This is considered a matter of courtesy as the development was paid
00014 * for by FOM the Dutch physics granting agency and we would like to
00015 * be able to track its scientific use to convince FOM of its value
00016 * for the community.
00017 *
00018 * This file is part of FORM.
00019 *
00020 * FORM is free software: you can redistribute it and/or modify it under the
00021 * terms of the GNU General Public License as published by the Free Software
00022 * Foundation, either version 3 of the License, or (at your option) any later
00023 * version.
00024 *
00025 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00026 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00027 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00028 * details.
00029 *
00030 * You should have received a copy of the GNU General Public License along
00031 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00032 */
00033 /* # [ License : */
00034 /*
00035 * # [ Includes :
00036 */
00037
00038 #include "form3.h"
00039
00040 /*
00041 * #] Includes :
00042 * # [ Compiler :
00043 * # [ tokenize :
00044
00045 * Takes the input in 'in' and translates it into tokens.
00046 * The tokens are put in the token buffer which starts at 'AC.tokens'
00047 * and runs till 'AC.toptokens'
00048 * We may assume that the various types of brackets match properly.
00049 * object = -1: after , or (
00050 * object = 0: name/variable/number etc is allowed
```



```

00051         object = 1: variable.
00052         object = 2: number
00053         object = 3: ) after subexpression
00054 */
00055
00056 #define CHECKPOLY {if(polyflag)MesPrint("&Illegal use of polynomial function"); polyflag = 0; }
00057
00058 int tokenize(UBYTE *in, WORD leftright)
00059 {
00060     int error = 0, object, funlevel = 0, bracelevel = 0, explevel = 0, numexp;
00061     int polyflag = 0;
00062     WORD number, type;
00063     UBYTE *s = in, c;
00064     SBYTE *out, *outtop, num[MAXNUMSIZE], *t;
00065     LONG i;
00066     if ( AC.tokens == 0 ) {
00067         SBYTE **ppp = &(AC.tokens); /* to avoid a compiler warning */
00068         SBYTE **pppp = &(AC.toptokens);
00069         DoubleBuffer((void **)ppp, (void **)pppp, sizeof(SBYTE), "start tokens");
00070     }
00071     out = AC.tokens;
00072     outtop = AC.toptokens - MAXNUMSIZE;
00073     AC.dumnumflag = 0;
00074     object = 0;
00075     while ( *in ) {
00076         if ( out > outtop ) {
00077             LONG oldsize = (LONG)(out - AC.tokens);
00078             SBYTE **ppp = &(AC.tokens); /* to avoid a compiler warning */
00079             SBYTE **pppp = &(AC.toptokens);
00080             DoubleBuffer((void **)ppp, (void **)pppp, sizeof(SBYTE), "expand tokens");
00081             out = AC.tokens + oldsize;
00082             outtop = AC.toptokens - MAXNUMSIZE;
00083         }
00084         switch ( FG.cTable[*in] ) {
00085             case 0: /* a-zA-Z */
00086                 CHECKPOLY
00087                 s = in++;
00088                 while ( FG.cTable[*in] == 0 || FG.cTable[*in] == 1
00089                     || *in == '_' ) in++;
00090             dovariable: c = *in; *in = 0;
00091                 if ( object > 0 ) {
00092                     MesPrint("&Illegal position for %s",s);
00093                     if ( !error ) error = 1;
00094                 }
00095                 if ( out > AC.tokens && ( out[-1] == TWILDCARD || out[-1] == TNOT ) ) {
00096                     type = GetName(AC.varnames,s,&number,NOAUTO);
00097                 }
00098                 else {
00099                     type = GetName(AC.varnames,s,&number,WITHAUTO);
00100                 }
00101                 if ( type < 0 )
00102                     type = GetName(AC.exprnames,s,&number,NOAUTO);
00103                 switch ( type ) {
00104                     case CSYMBOL: *out++ = TSYMBOL; break;
00105                     case CINDEX:
00106                         if ( number >= (AM.IndDum-AM.OffsetIndex) ) {
00107                             if ( c != '?' ) {
00108                                 MesPrint("&Generated indices should be of the type Nnumber_?");
00109                                 error = 1;
00110                             }
00111                             else {
00112                                 *in++ = c; c = *in; *in = 0;
00113                                 AC.dumnumflag = 1;
00114                             }
00115                         }
00116                         *out++ = TINDEX;
00117                         break;
00118                     case CVECTOR: *out++ = TVECTOR; break;
00119                     case CFUNCTION:
00120 #ifdef WITHMPI
00121                         /*
00122                          * In the preprocessor, random functions in #svar=... and #inside
00123                          * may cause troubles, because the program flow on a slave may be
00124                          * different from those on others. We set AC.RhsExprInModuleFlag in order
00125                          * to make the change of $-variable be done on the master and thus keep the
00126                          * consistency among the master and all slave processes. The previous value
00127                          * of AC.RhsExprInModuleFlag will be restored after #svar=... and #inside.
00128                          */
00129                         if ( AP.PreAssignFlag || AP.PreInsideLevel ) {
00130                             switch ( number + FUNCTION ) {
00131                                 case RANDOMFUNCTION:
00132                                 case RANPERM:
00133                                     AC.RhsExprInModuleFlag = 1;
00134                             }
00135                         }
00136 #endif
00137                         *out++ = TFUNCTION;

```

```

00138             break;
00139         case CSET: *out++ = TSET; break;
00140         case CEXPRESSION: *out++ = TEXPRESSION;
00141             if ( leftright == LHSIDE ) {
00142                 if ( !error ) error = 1;
00143                 MesPrint("&Expression not allowed in LH-side of
substitution: %s",s);
00144             }
00145         /*[06nov2003 mt]:*/
00146         #ifdef WITHMPI
00147             else { /*RHSide*/
00148                 /* NOTE: We always set AC.RhsExprInModuleFlag regardless
of
00149                 * AP.PreAssignFlag or AP.PreInsideLevel because we
have to detect
00150                 * RHS expressions even in those cases. */
00151                 AC.RhsExprInModuleFlag = 1;
00152             }
00153             if ( !AP.PreAssignFlag && !AP.PreInsideLevel )
00154                 Expressions[number].vflags |= ISINRHS;
00155         #endif
00156         /*:[06nov2003 mt]*/
00157             if ( AC.exprfillwarning == 0 ) {
00158                 AC.exprfillwarning = 1;
00159             }
00160             break;
00161             case CDELTA: *out++ = TDELTA; *in = c;
00162                 object = 1; continue;
00163             case CDUBIOUS: *out++ = TDUBIOUS; break;
00164             default: *out++ = TDUBIOUS;
00165                 if ( !error ) error = 1;
00166                 MesPrint("&Undeclared variable %s",s);
00167                 number = AddDubious(s);
00168                 break;
00169             }
00170             object = 1;
00171         donumber: i = 0;
00172             do { num[i++] = (SBYTE)(number & 0x7F); number >>= 7; } while ( number );
00173             while ( --i >= 0 ) *out++ = num[i];
00174             *in = c;
00175             break;
00176             case 1: /* 0-9 */
00177             {
00178                 #ifdef WITHFLOAT
00179                     int spec;
00180                     UBYTE *in2;
00181                 #endif
00182                 CHECKPOLY
00183                 s = in;
00184                 while ( *s == '0' && FG.cTable[s[1]] == 1 ) s++;
00185                 in = s+1; i = 1;
00186                 while ( FG.cTable[*in] == 1 ) { in++; i++; }
00187                 if ( object > 0 ) {
00188                     c = *in; *in = 0;
00189                     MesPrint("&Illegal position for %s",s);
00190                     *in = c;
00191                     if ( !error ) error = 1;
00192                 }
00193                 if ( i == 1 && *in == '_' && ( *s == '5' || *s == '6'
|| *s == '7' ) ) {
00194                     in++; *out++ = TSGAMMA; *out++ = (SBYTE)(*s - '4');
00195                     object = 1;
00196                     break;
00197                 }
00198             }
00199         #ifdef WITHFLOAT
00200             in2 = CheckFloat(in,&spec);
00201             if ( in2 > in ) {
00202                 if ( spec == 1 ) {
00203                     *out++ = TNUMBER; *out++ = 0; s = in2;
00204                 }
00205                 else if ( spec == -1 ) {
00206                     MesPrint("&The floating point system has not been started: %s",in);
00207                     if ( !error ) error = 1;
00208                 }
00209                 else {
00210                     in = in2;
00211                 }
00212             }
00213             dofloat:
00214                 while ( out + (in-s) >= AC.toptokens ) {
00215                     LONG oldsize = (LONG)(out - AC.tokens);
00216                     SBYTE **ppp = &(AC.tokens); /* to avoid a compiler warning */
00217                     SBYTE **pppp = &(AC.toptokens);
00218                     DoubleBuffer((void **)ppp,(void **)pppp,sizeof(SBYTE),"more tokens");
00219                     out = AC.tokens + oldsize;
00220                     outtop = AC.toptokens - MAXNUMSIZE;
00221                 }
00222                 *out++ = TFLOAT;
00223                 while ( s < in ) *out++ = *s++;

```

```

00222     }
00223     }
00224     else
00225 #endif
00226     {
00227         *out++ = TNUMBER;
00228         if ( ( i & 1 ) != 0 ) *out++ = (SBYTE) (*s++ - '0');
00229         while ( out + (in-s)/2 >= AC.toptokens ) {
00230             LONG oldsize = (LONG) (out - AC.tokens);
00231             SBYTE **ppp = &(AC.tokens); /* to avoid a compiler warning */
00232             SBYTE **pppp = &(AC.toptokens);
00233             DoubleBuffer((void **)ppp, (void **)pppp, sizeof(SBYTE), "more tokens");
00234             out = AC.tokens + oldsize;
00235             outtop = AC.toptokens - MAXNUMSIZE;
00236         }
00237         while ( s < in ) { /* We store in base 100 */
00238             *out++ = (SBYTE) (( *s - '0' ) * 10 + ( s[1] - '0' ));
00239             s += 2;
00240         }
00241     }
00242     object = 2;
00243 }
00244 break;
00245 case 2: /* . $ _ ? # ' */
00246     CHECKPOLY
00247     if ( *in == '?' ) {
00248         if ( leftright == LHSIDE ) {
00249             if ( object == 1 ) { /* follows a name */
00250                 in++; *out++ = TWILDCARD;
00251                 if ( FG.cTable[in[0]] == 0 || in[0] == '[' || in[0] == '{' ) object = 0;
00252             }
00253             else if ( object == -1 ) { /* follows comma or ( */
00254                 in++; s = in;
00255                 while ( FG.cTable[*in] == 0 || FG.cTable[*in] == 1 ) in++;
00256                 c = *in; *in = 0;
00257                 if ( FG.cTable[*s] != 0 ) {
00258                     MesPrint("&Illegal name for argument list variable %s", s);
00259                     error = 1;
00260                 }
00261                 else {
00262                     i = AddWildcardName((UBYTE *)s);
00263                     *in = c;
00264                     *out++ = TWILDARG;
00265                     *out++ = (SBYTE)i;
00266                 }
00267                 object = 1;
00268             }
00269             else {
00270                 MesPrint("&Illegal position for ?");
00271                 error = 1;
00272                 in++;
00273             }
00274         }
00275         else {
00276             if ( object != -1 ) goto IllPos;
00277             in++;
00278             if ( FG.cTable[*in] == 0 || FG.cTable[*in] == 1 ) {
00279                 s = in;
00280                 while ( FG.cTable[*in] == 0 || FG.cTable[*in] == 1 ) in++;
00281                 c = *in; *in = 0;
00282                 i = GetWildcardName((UBYTE *)s);
00283                 if ( i <= 0 ) {
00284                     MesPrint("&Undefined argument list variable %s", s);
00285                     error = 1;
00286                 }
00287                 *in = c;
00288                 *out++ = TWILDARG;
00289                 *out++ = (SBYTE)i;
00290             }
00291             else {
00292                 if ( AC.vectorlikeLHS == 0 ) {
00293                     MesPrint("&Generated index ? only allowed in vector substitution", s);
00294                     error = 1;
00295                 }
00296                 *out++ = TGENINDEX;
00297             }
00298             object = 1;
00299         }
00300     }
00301     else if ( *in == '.' ) {
00302         if ( object == 1 ) { /* follows a name */
00303             *out++ = TDOT;
00304             object = 0;
00305             in++;
00306         }
00307 #ifdef WITHFLOAT
00308         else if ( object == 0 || object == -1 ) {

```

```

00309 /*
00310                                     Test for floating point number
00311 */
00312 int spec;
00313 s = CheckFloat(in,&spec);
00314 if ( s > in ) {
00315     if ( spec == 1 ) {
00316         *out++ = TNUMBER; *out++ = 0;
00317         object = 2; in = s;
00318     }
00319     else if ( spec == -1 ) {
00320         MesPrint("&The floating point system has not been started: %s",in);
00321         if ( !error ) error = 1;
00322     }
00323     else {
00324         UBYTE *a = s; s = in; in = a;
00325         goto dofloat;
00326     }
00327 }
00328     else goto IllPos;
00329 }
00330 #endif
00331     else goto IllPos;
00332 }
00333 else if ( *in == '$' ) { /* $ variable */
00334     in++;
00335     s = in;
00336     if ( FG.cTable[*in] == 0 ) {
00337         while ( FG.cTable[*in] == 0 || FG.cTable[*in] == 1 ) in++;
00338         if ( *in == '_' && AP.PreAssignFlag == 2 ) in++;
00339         c = *in; *in = 0;
00340         if ( object > 0 ) {
00341             if ( object != 1 || leftright == RHSIDE ) {
00342                 MesPrint("&Illegal position for %s",s);
00343                 if ( !error ) error = 1;
00344             } /* else can be assignment in wildcard */
00345             else {
00346                 if ( ( number = GetDollar(s) ) < 0 ) {
00347                     number = AddDollar(s,0,0,0);
00348                 }
00349             }
00350         }
00351         else if ( ( number = GetDollar(s) ) < 0 ) {
00352             MesPrint("&Undefined variable %s",s);
00353             if ( !error ) error = 1;
00354             number = AddDollar(s,0,0,0);
00355         }
00356         *out++ = TDOLLAR;
00357         object = 1;
00358         if ( ( AC.exprfillwarning == 0 ) &&
00359             ( out > AC.tokens+1 ) && ( out[-2] != TWILDCARD ) ) {
00360             AC.exprfillwarning = 1;
00361         }
00362         goto donumber;
00363     }
00364     else {
00365         MesPrint("&Illegal name for $ variable after %s",in);
00366         if ( !error ) error = 1;
00367     }
00368 }
00369 else if ( *in == '#' ) {
00370     in++;
00371     if ( object == 1 ) { /* follows a name */
00372         *out++ = TCONJUGATE;
00373     }
00374     else {
00375         MesPrint("&Illegal position for %#",in);
00376         error = 1;
00377     }
00378 }
00379     else goto IllPos;
00380     break;
00381 case 3: /* [ ] */
00382     CHECKPOLY
00383     if ( *in == '[' ) {
00384         if ( object == 1 ) { /* after name */
00385             t = out-1;
00386             if ( *t == RPARENTHESIS ) {
00387                 *out++ = LBRACE; *out++ = LPARENTHESIS;
00388                 bracelevel++; explevel = bracelevel;
00389             }
00390             else {
00391                 while ( *t >= 0 && t > AC.tokens ) t--;
00392                 if ( *t == TEXPRESSION ) {
00393                     *out++ = LBRACE; *out++ = LPARENTHESIS;
00394                     bracelevel++; explevel = bracelevel;
00395                 }

```

```

00396         else { *out++ = LBACE; bracelevel++; }
00397     }
00398     object = 0;
00399 }
00400 else { /* name. find matching */
00401     s = in;
00402     in = SkipAName(in);
00403     goto dovariable;
00404 }
00405 }
00406 else {
00407     if ( explevel > 0 && explevel == bracelevel ) {
00408         *out++ = RPARENTHESIS; explevel = 0;
00409     }
00410     *out++ = RBACE; object = 1; bracelevel--;
00411 }
00412 in++;
00413 break;
00414 case 4: /* ( ) = ; , */
00415     if ( *in == '(' ) {
00416         if ( funlevel >= AM.MaxParLevel ) {
00417             MesPrint("&More than %d levels of parentheses", AM.MaxParLevel);
00418             return(-1);
00419         }
00420         if ( object == 1 ) { /* After name -> function,vector */
00421             AC.tokenarglevel[funlevel++] = TYPEISFUN;
00422             *out++ = TFUNOPEN;
00423             if ( polyflag ) {
00424                 if ( in[1] != ')' && in[1] != ',' ) {
00425                     *out++ = TNUMBER; *out++ = (SBYTE) (polyflag);
00426                     *out++ = TCOMMA;
00427                     *out++ = LPARENTHESIS;
00428                 }
00429                 else {
00430                     *out++ = LPARENTHESIS;
00431                     *out++ = TNUMBER; *out++ = (SBYTE) (polyflag);
00432                 }
00433                 polyflag = 0;
00434             }
00435             else if ( in[1] != ')' && in[1] != ',' ) {
00436                 *out++ = LPARENTHESIS;
00437             }
00438         }
00439         else if ( object <= 0 ) {
00440             CHECKPOLY
00441             AC.tokenarglevel[funlevel++] = TYPEISSUB;
00442             *out++ = LPARENTHESIS;
00443         }
00444         else {
00445             polyflag = 0;
00446             AC.tokenarglevel[funlevel++] = TYPEISMYSTERY;
00447             MesPrint("&Illegal position for (: %s", in);
00448             if ( error >= 0 ) error = -1;
00449         }
00450         object = -1;
00451     }
00452     else if ( *in == ')' ) {
00453         funlevel--;
00454         if ( funlevel < 0 ) {
00455             /* if ( funflag == 0 ) { */
00456                 MesPrint("&There is an unmatched parenthesis");
00457                 if ( error >= 0 ) error = -1;
00458             /* } */
00459         }
00460         else if ( object <= 0
00461             && ( AC.tokenarglevel[funlevel] != TYPEISFUN
00462             || out[-1] != TFUNOPEN ) ) {
00463             MesPrint("&Illegal position for closing parenthesis.");
00464             if ( error >= 0 ) error = -1;
00465             if ( AC.tokenarglevel[funlevel] == TYPEISFUN ) object = 1;
00466             else object = 3;
00467         }
00468         else {
00469             if ( AC.tokenarglevel[funlevel] == TYPEISFUN ) {
00470                 if ( out[-1] == TFUNOPEN ) out--;
00471                 else {
00472                     if ( out[-1] != TCOMMA ) *out++ = RPARENTHESIS;
00473                     *out++ = TFUNCLOSE;
00474                 }
00475                 object = 1;
00476             }
00477             else if ( AC.tokenarglevel[funlevel] == TYPEISSUB ) {
00478                 *out++ = RPARENTHESIS;
00479                 object = 3;
00480             }
00481         }
00482     }

```

```

00483         else if ( *in == ',' ) {
00484             if ( /* object > 0 && */ funlevel > 0 &&
00485                 AC.tokenarglevel[funlevel-1] == TYPEISFUN ) {
00486                 if ( out[-1] != TFUNOPEN && out[-1] != TCOMMA )
00487                     *out++ = RPARENTHESIS;
00488                 else { *out++ = TNUMBER; *out++ = 0; }
00489                 *out++ = TCOMMA;
00490                 if ( in[1] != ',' && in[1] != ')' )
00491                     *out++ = LPARENTHESIS;
00492                 else if ( in[1] == ')' ) {
00493                     *out++ = TNUMBER; *out++ = 0;
00494                 }
00495             }
00496         /*
00497         else if ( object > 0 ) {
00498         }
00499         */
00500         else {
00501             MesPrint("&Illegal position for comma: %s",in);
00502             MesPrint("&Forgotten ; ?");
00503             if ( error >= 0 ) error = -1;
00504         }
00505         object = -1;
00506     }
00507     else goto IllPos;
00508     in++;
00509     break;
00510 case 5: /* + - * % / ^ : */
00511     CHECKPOLY
00512     if ( *in == ':' || *in == '%' ) goto IllPos;
00513     if ( *in == '*' || *in == '/' || *in == '^' ) {
00514         if ( object <= 0 ) {
00515             MesPrint("&Illegal position for operator: %s",in);
00516             if ( error >= 0 ) error = -1;
00517         }
00518         else if ( *in == '*' ) *out++ = TMULTIPLY;
00519         else if ( *in == '/' ) *out++ = TDIVIDE;
00520         else *out++ = TPOWER;
00521         in++;
00522     }
00523     else {
00524         i = 1;
00525         while ( *in == '+' || *in == '-' ) {
00526             if ( *in == '-' ) i = -i;
00527             in++;
00528         }
00529         if ( i == 1 ) {
00530             if ( out > AC.tokens && out[-1] != TFUNOPEN &&
00531                 out[-1] != LPARENTHESIS && out[-1] != TCOMMA
00532                 && out[-1] != LBRACE )
00533                 *out++ = TPLUS;
00534             }
00535             else *out++ = TMINUS;
00536         }
00537         object = 0;
00538         break;
00539 case 6: /* Whitespace */
00540     in++; break;
00541 case 7: /* { | } */
00542     CHECKPOLY
00543     if ( *in == '{' ) {
00544         if ( object > 0 ) {
00545             MesPrint("&Illegal position for %s",in);
00546             if ( !error ) error = 1;
00547         }
00548         s = in+1;
00549         SKIPBRA2(in)
00550         number = DoTempSet(s,in);
00551         in++;
00552         if ( number >= 0 ) {
00553             *out++ = TSET;
00554             i = 0;
00555             do { num[i++] = (SBYTE)(number & 0x7F); number >>= 7; } while ( number );
00556             while ( --i >= 0 ) *out++ = num[i];
00557         }
00558         else if ( error == 0 ) error = 1;
00559         object = 1;
00560     }
00561     else goto IllPos;
00562     break;
00563 case 8: /* ! & < > */
00564     CHECKPOLY
00565     if ( *in != '!' || leftright == RHSIDE
00566         || object != 1 || out[-1] != TWILDCARD ) goto IllPos;
00567     *out++ = TNOT;
00568     if ( FG.cTable[in[1]] == 0 || in[1] == '[' || in[1] == '{' ) object = 0;
00569     in++;

```

```

00570             break;
00571         default:
00572     IllPos:      MesPrint("&Illegal character at this position: %s",in);
00573                 if ( error >= 0 ) error = -1;
00574                 in++;
00575                 polyflag = 0;
00576                 break;
00577     }
00578 }
00579 *out++ = TENDOFIT;
00580 AC.endoftokens = out;
00581 if ( funlevel > 0 || bracelevel != 0 ) {
00582     if ( funlevel > 0 ) MesPrint("&Unmatched parentheses");
00583     if ( bracelevel != 0 ) MesPrint("&Unmatched braces");
00584     return(-1);
00585 }
00586 if ( AC.TokensWriteFlag ) WriteTokens(AC.tokens);
00587 /*
00588     Simplify fixed set elements
00589 */
00590 if ( error == 0 && simpltoken(AC.tokens) ) error = 1;
00591 /*
00592     Collect wildcards for the prototype. Simplify the leftover wildcards
00593 */
00594 if ( error == 0 && leftright == LHSIDE && simpwtoken(AC.tokens) )
00595     error = 1;
00596 /*
00597     Now prepare the set[n] objects in the RHS.
00598 */
00599 if ( error == 0 && leftright == RHSIDE && simp4token(AC.tokens) )
00600     error = 1;
00601 /*
00602     Simplify simple function arguments (and 1/fac_ and 1/invfac_)
00603 */
00604 if ( error == 0 && simp2token(AC.tokens) ) error = 1;
00605 /*
00606     Next we try to remove composite denominators or exponents and
00607     replace them by their internal functions. This may involve expanding
00608     the buffer. The return code of 3a is negative if there is an error
00609     and positive if indeed we need to do some work.
00610     simp3btoken does the work
00611 */
00612 numexp = 0;
00613 if ( error == 0 && ( numexp = simp3atoken(AC.tokens,leftright) ) < 0 )
00614     error = 1;
00615 if ( numexp > 0 ) {
00616     SBYTE *tt;
00617     out = AC.tokens;
00618     while ( *out != TENDOFIT ) out++;
00619     while ( out+numexp*9+20 > outtop ) {
00620         LONG oldsize = (LONG)(out - AC.tokens);
00621         SBYTE **ppp = &(AC.tokens); /* to avoid a compiler warning */
00622         SBYTE **pppp = &(AC.toptokens);
00623         DoubleBuffer((void **)ppp,(void **)pppp,sizeof(SBYTE),"out tokens");
00624         out = AC.tokens + oldsize;
00625         outtop = AC.toptokens - MAXNUMSIZE;
00626     }
00627     tt = out + numexp*9+20;
00628     while ( out >= AC.tokens ) { *tt-- = *out--; }
00629     while ( tt >= AC.tokens ) { *tt-- = EMPTY; }
00630     if ( error == 0 && simp3btoken(AC.tokens,leftright) ) error = 1;
00631     if ( error == 0 && simp2token(AC.tokens) ) error = 1;
00632 }
00633 /*
00634     In simp5token we test for special cases like sumvariables that are
00635     already wildcards, etc.
00636 */
00637 if ( error == 0 && simp5token(AC.tokens,leftright) ) error = 1;
00638 /*
00639     In simp6token we test for special cases like factorized expressions
00640     that occur in the RHS in an improper way.
00641 */
00642 if ( error == 0 && simp6token(AC.tokens,leftright) ) error = 1;
00643
00644 return(error);
00645 }
00646
00647 /*
00648     #] tokenize :
00649     #[ WriteTokens :
00650 */
00651
00652 char *ttypes[] = { "\n", "S", "I", "V", "F", "set", "E", "dotp", "#",
00653     "sub", "d_", "$", "dub", "(", ")", "?", "??", ".", "[", "]",
00654     ", ", "(", ")", "*", "/", "^", "+", "-", "!", "end", "{", "}",
00655     "N_?", "conj", "()", "#d", "^d", "_", "snum" };
00656

```

```

00657 void WriteTokens(SBYTE *in)
00658 {
00659     int numinline = 0, x, n = sizeof(ttypes)/sizeof(char *);
00660     char outbuf[81], *s, *out, c;
00661     out = outbuf;
00662     while ( *in != TENDOFIT ) {
00663         if ( *in < 0 ) {
00664             if ( *in >= -n ) {
00665                 s = ttypes[-*in];
00666                 while ( *s ) { *out++ = *s++; numinline++; }
00667             }
00668             else {
00669                 *out++ = '-'; x = -*in; numinline++;
00670                 goto writenumber;
00671             }
00672         }
00673         else {
00674             x = *in;
00675 writenumber:
00676             s = out;
00677             do {
00678                 *out++ = (char)(( x % 10 ) + '0');
00679                 numinline++;
00680                 x = x / 10;
00681             } while ( x );
00682             c = out[-1]; out[-1] = *s; *s = c;
00683         }
00684         if ( numinline > 70 ) {
00685             *out = 0;
00686             MesPrint("%s",outbuf);
00687             out = outbuf; numinline = 0;
00688         }
00689         else {
00690             *out++ = ' '; numinline++;
00691         }
00692         in++;
00693     }
00694     if ( numinline > 0 ) { *out = 0; MesPrint("%s",outbuf); }
00695 }
00696
00697 /*
00698     #] WriteTokens :
00699     #[ simpltoken :
00700
00701     Routine substitutes set elements if possible.
00702     This means sets with a fixed argument like setname[3].
00703 */
00704
00705 int simpltoken(SBYTE *s)
00706 {
00707     int error = 0, n, i, base;
00708     WORD numsub;
00709     SBYTE *fill = s, *start, *t, numtab[10];
00710     SETS set;
00711     while ( *s != TENDOFIT ) {
00712         if ( *s == RBRACE ) {
00713             start = fill-1;
00714             while ( *start != LBRACE ) start--;
00715             t = start - 1;
00716             while ( *t >= 0 ) t--;
00717             if ( *t == TSET && ( start[1] == TNUMBER || start[1] == TNUMBER1 ) ) {
00718                 base = start[1] == TNUMBER ? 100: 128;
00719                 start += 2;
00720                 numsub = *start++;
00721                 while ( *start >= 0 && start < fill )
00722                     { numsub = base*numsub + *start++; }
00723                 if ( start == fill ) {
00724                     start = t;
00725                     t++; n = *t++; while ( *t >= 0 ) { n = 128*n + *t++; }
00726                     set = Sets+n;
00727                     if ( ( set->type != CRANGE )
00728                         && ( numsub > 0 && numsub <= set->last-set->first ) ) {
00729                         fill = start;
00730                         n = SetElements[set->first+numsub-1];
00731                         switch (set->type) {
00732                             case CSYMBOL:
00733                                 if ( n > MAXPOWER ) {
00734                                     n -= 2*MAXPOWER;
00735                                     if ( n < 0 ) { n = -n; *fill++ = TMINUS; }
00736                                     *fill++ = TNUMBER1;
00737                                 }
00738                                 else *fill++ = TSYMBOL;
00739                                 break;
00740                             case CINDEX:
00741                                 if ( n < AM.OffsetIndex ) *fill++ = TNUMBER1;
00742                                 else {
00743                                     *fill++ = TINDEX;

```



```

00744             n -= AM.OffsetIndex;
00745         }
00746         break;
00747     case CVECTOR: *fill++ = TVECTOR;
00748         n -= AM.OffsetVector; break;
00749     case CFUNCTION: *fill++ = TFUNCTION;
00750         n -= FUNCTION; break;
00751     case CNUMBER: *fill++ = TNUMBER1; break;
00752     case CDUBIOUS: *fill++ = TDUBIOUS; n = 1; break;
00753     }
00754     i = 0;
00755     if ( n < 0 ) {
00756         MesPrint("Value of n = %d",n);
00757     }
00758     do { numtab[i++] = (SBYTE)(n & 0x7F); n >>= 7; } while ( n );
00759     while ( --i >= 0 ) *fill++ = numtab[i];
00760     }
00761     else {
00762         MesPrint("&Illegal element %d in set",numsub);
00763         error++;
00764     }
00765     s++; continue;
00766     }
00767     }
00768     *fill++ = *s++;
00769     }
00770     else *fill++ = *s++;
00771     }
00772     *fill++ = TENDOFIT;
00773     return(error);
00774 }
00775
00776 /*
00777     #] simpltoken :
00778     #[ simpwtoken :
00779
00780     Only to be called in the LHS.
00781     Hunts down the wildcards and writes them to the wildcardbuffer.
00782     Next it causes the ProtoType to be constructed.
00783     All wildcards are simplified into the trailing TWILDCARD,
00784     because the specifics are stored in the prototype.
00785     These specifics also include the transfer of wildcard values
00786     to $variables.
00787
00788     Types of wildcards:
00789     a?, a?set, a?!set, a?set[i], A?set1?set2, ?a
00790     After this we can strip the set information.
00791     We still need the ? because of the wildcarding offset in code generation
00792 */
00793
00794 int simpwtoken(SBYTE *s)
00795 {
00796     int error = 0, first = 1, notflag;
00797     WORD num, numto, numdollar, *w = AC.WildC, *wstart, *wtop;
00798     SBYTE *fill = s, *t, *v, *s0 = s;
00799     while ( *s != TENDOFIT ) {
00800         if ( *s == TWILDCARD ) {
00801             notflag = 0; t = fill;
00802             while ( t > s0 && t[-1] >= 0 ) t--;
00803             v = t; num = 0; *fill++ = *s++;
00804             while ( *v >= 0 ) num = 128*num + *v++;
00805             if ( t > s0 ) t--;
00806             AC.NwildC += 4;
00807             if ( AC.NwildC > 4*AM.MaxWildcards ) goto firsterr;
00808             switch ( *t ) {
00809                 case TSYMBOL:
00810                 case TDUBIOUS:
00811                     *w++ = SYMTOSYM; *w++ = 4; *w++ = num; *w++ = num; break;
00812                 case TINDEX:
00813                     num += AM.OffsetIndex;
00814                     *w++ = INDTOIND; *w++ = 4; *w++ = num; *w++ = num; break;
00815                 case TVECTOR:
00816                     num += AM.OffsetVector;
00817                     *w++ = VECTOVEC; *w++ = 4; *w++ = num; *w++ = num; break;
00818                 case TFUNCTION:
00819                     num += FUNCTION;
00820                     *w++ = FUNTOFUN; *w++ = 4; *w++ = num; *w++ = num; break;
00821                 default:
00822                     MesPrint("&Illegal type of wildcard in LHS");
00823                     error = -1;
00824                     *w++ = SYMTOSYM; *w++ = 4; *w++ = num; *w++ = num; break;
00825             }
00826         }
00827     }
00828     /*
00829     Now the sets. The s pointer sits after the ?
00830     */
00831     wstart = w;

```

```

00831         if ( *s == TNOT && s[1] == TSET ) { notflag = 1; s++; }
00832     if ( *s == TSET ) {
00833         s++; num = 0; while ( *s >= 0 ) num = 128*num + *s++;
00834         if ( notflag == 0 && *s == TWILDCARD && s[1] == TSET ) {
00835             s += 2; numto = 0; while ( *s >= 0 ) numto = 128*numto + *s++;
00836             if ( num < AM.NumFixedSets || numto < AM.NumFixedSets
00837                 || Sets[num].type == CRANGE || Sets[numto].type == CRANGE ) {
00838                 MesPrint("&This type of set not allowed in this wildcard construction");
00839                 error = 1;
00840             }
00841             else {
00842                 AC.NwildC += 4;
00843                 if ( AC.NwildC > 4*AM.MaxWildcards ) goto firsterr;
00844                 *w++ = FROMSET; *w++ = 4; *w++ = num; *w++ = numto;
00845                 wstart = w;
00846             }
00847         }
00848     else if ( notflag == 0 && *s == LBRACE && s[1] == TSYMBOL ) {
00849         if ( num < AM.NumFixedSets || Sets[num].type == CRANGE ) {
00850             MesPrint("&This type of set not allowed in this wildcard construction");
00851             error = 1;
00852         }
00853         v = s; s += 2;
00854         numto = 0; while ( *s >= 0 ) numto = 128*numto + *s++;
00855         if ( *s == TWILDCARD ) s++; /* most common mistake */
00856         if ( *s == RBRACE ) {
00857             s++;
00858             AC.NwildC += 8;
00859             if ( AC.NwildC > 4*AM.MaxWildcards ) goto firsterr;
00860             *w++ = SETTONUM; *w++ = 4; *w++ = num; *w++ = numto;
00861             wstart = w;
00862             *w++ = SYMTOSYM; *w++ = 4; *w++ = numto; *w++ = 0;
00863         }
00864     else if ( *s == TDOLLAR ) {
00865         s++; numdollar = 0;
00866         while ( *s >= 0 ) numdollar = 128*numdollar + *s++;
00867         if ( *s == RBRACE ) {
00868             s++;
00869             AC.NwildC += 12;
00870             if ( AC.NwildC > 4*AM.MaxWildcards ) goto firsterr;
00871             *w++ = SETTONUM; *w++ = 4; *w++ = num; *w++ = numto;
00872             wstart = w;
00873             *w++ = SYMTOSYM; *w++ = 4; *w++ = numto; *w++ = 0;
00874             *w++ = LOADDOLLAR; *w++ = 4; *w++ = numdollar;
00875             *w++ = numdollar;
00876         }
00877         else { s = v; goto singlewild; }
00878     }
00879     else { s = v; goto singlewild; }
00880 }
00881 singlewild:
00882     num += notflag * 2*WILDOFFSET;
00883     AC.NwildC += 4;
00884     if ( AC.NwildC > 4*AM.MaxWildcards ) goto firsterr;
00885     *w++ = FROMSET; *w++ = 4; *w++ = num; *w++ = -WILDOFFSET;
00886     wstart = w;
00887 }
00888 }
00889 else if ( *s != TDOLLAR && *s != TENDOFIT && *s != RPARENTHESIS
00890 && *s != RBRACE && *s != TCOMMA && *s != TFUNCLOSE && *s != TMULTIPLY
00891 && *s != TPOWER && *s != TDIVIDE && *s != TPLUS && *s != TMINUS
00892 && *s != TPOWER1 && *s != TEMPTY && *s != TFUNOPEN && *s != TDOT ) {
00893     MesPrint("&Illegal type of wildcard in LHS");
00894     error = -1;
00895 }
00896 if ( *s == TDOLLAR ) {
00897     s++; numdollar = 0;
00898     while ( *s >= 0 ) numdollar = 128*numdollar + *s++;
00899     AC.NwildC += 4;
00900     if ( AC.NwildC > 4*AM.MaxWildcards ) goto firsterr;
00901     wtop = w + 4;
00902     if ( wstart < w ) {
00903         while ( w > wstart ) { w[4] = w[0]; w--; }
00904     }
00905     *w++ = LOADDOLLAR; *w++ = 4; *w++ = numdollar; *w++ = numdollar;
00906     w = wtop;
00907 }
00908 }
00909 else if ( *s == TWILDARG ) {
00910     *fill++ = *s++;
00911     num = 0;
00912     while ( *s >= 0 ) { num = 128*num + *s; *fill++ = *s++; }
00913     AC.NwildC += 4;
00914     if ( AC.NwildC > 4*AM.MaxWildcards ) {
00915 firsterr:         if ( first ) {
00916                     MesPrint("&More than %d wildcards",AM.MaxWildcards);
00917                     error = -1;

```

```

00918         first = 0;
00919     }
00920 }
00921 else { *w++ = ARGTOARG; *w++ = 4; *w++ = num; *w++ = -1; }
00922 if ( *s == TDOLLAR ) {
00923     s++; num = 0; while ( *s >= 0 ) num = 128*num + *s++;
00924     AC.NwildC += 4;
00925     if ( AC.NwildC > 4*AM.MaxWildcards ) goto firsterr;
00926     *w++ = LOADDOLLAR; *w++ = 4; *w++ = num; *w++ = num;
00927 }
00928 }
00929 else *fill++ = *s++;
00930 }
00931 *fill++ = TENDOFIT;
00932 AC.WildC = w;
00933 return(error);
00934 }
00935
00936 /*
00937     #] simpwtoken :
00938     #[ simp2token :
00939
00940     Deals with function arguments.
00941     The tokenizer has given function arguments extra parentheses.
00942     We remove the double parentheses.
00943     Next we remove the parentheses around the simple arguments.
00944
00945     It also replaces /fac_() by *invfac_() and /invfac_() by *fac_()
00946 */
00947
00948 int simp2token(SBYTE *s)
00949 {
00950     SBYTE *to, *fill, *t, *v, *w, *s0 = s, *vv;
00951     int error = 0, n;
00952     /*
00953         Set substitutions
00954     */
00955     fill = to = s;
00956     while ( *s != TENDOFIT ) {
00957         if ( *s == LPARENTHESIS && s[1] == LPARENTHESIS ) {
00958             t = s+1; n = 0;
00959             while ( n >= 0 ) {
00960                 t++;
00961                 if ( *t == LPARENTHESIS ) n++;
00962                 else if ( *t == RPARENTHESIS ) n--;
00963             }
00964             if ( t[1] == RPARENTHESIS ) {
00965                 *t = EMPTY; s++;
00966             }
00967             *fill++ = *s++;
00968         }
00969         else if ( *s == EMPTY ) s++;
00970         else if ( *s == AM.facnum && ( fill > (s0+1) ) && fill[-2] == TDIVIDE
00971             && fill[-1] == TFUNCTION ) {
00972             fill[-2] = TMULTIPLY; *fill++ = (SBYTE)(AM.invfacnum); s++;
00973         }
00974         else if ( *s == AM.invfacnum && ( fill > (s0+1) ) && fill[-2] == TDIVIDE
00975             && fill[-1] == TFUNCTION ) {
00976             fill[-2] = TMULTIPLY; *fill++ = (SBYTE)(AM.facnum); s++;
00977         }
00978         else *fill++ = *s++;
00979     }
00980     *fill++ = TENDOFIT;
00981     /*
00982         Second round: try to locate 'simple' arguments and strip their brackets
00983
00984         We add (9-feb-2010) to the simple arguments integers of any size
00985     */
00986     fill = s = to;
00987     while ( *s != TENDOFIT ) {
00988         if ( *s == LPARENTHESIS ) {
00989             t = s; n = 0;
00990             while ( n >= 0 ) {
00991                 t++;
00992                 if ( *t == LPARENTHESIS ) n++;
00993                 else if ( *t == RPARENTHESIS ) n--;
00994             }
00995             if ( t[1] == TFUNCLOSE && s[1] != TWILDARG ) { /* Check for last argument in sum */
00996                 v = fill - 1; n = 0;
00997                 while ( n >= 0 && v >= to ) {
00998                     if ( *v == TFUNOPEN ) n--;
00999                     else if ( *v == TFUNCLOSE ) n++;
01000                     v--;
01001                 }
01002                 if ( v > to ) {
01003                     while ( *v >= 0 ) v--;
01004                     if ( *v == TFUNCTION ) { v++;

```

```

01005         n = 0; while ( *v >= 0 && v < fill ) n = 128*n + *v++;
01006         if ( n == AM.sumnum || n == AM.sumpnum ) {
01007             *fill++ = *s++; continue;
01008         }
01009         else if ( ( n == (FIRSTBRACKET-FUNCTION)
01010             || n == (TERMSINEXPR-FUNCTION)
01011             || n == (SIZEOFFUNCTION-FUNCTION)
01012             || n == (NUMFACTORS-FUNCTION)
01013             || n == (GCDFUNCTION-FUNCTION)
01014             || n == (DIVFUNCTION-FUNCTION)
01015             || n == (REMFUNCTON-FUNCTION)
01016             || n == (INVERSEFUNCTION-FUNCTION)
01017             || n == (MULFUNCTION-FUNCTION)
01018             || n == (FACTORIN-FUNCTION)
01019             || n == (FIRSTTERM-FUNCTION)
01020             || n == (CONTENTTERM-FUNCTION) )
01021             && fill[-1] == TFUNOPEN ) {
01022             v = s+1;
01023             if ( *v == TEXPRESSON ) {
01024                 v++;
01025                 n = 0; while ( *v >= 0 ) n = 128*n + *v++;
01026                 if ( v == t ) {
01027                     *t = EMPTY; s++;
01028                 }
01029             }
01030         }
01031     }
01032 }
01033 }
01034 if ( ( fill > to )
01035     && ( ( fill[-1] == TFUNOPEN || fill[-1] == TCOMMA )
01036         && ( t[1] == TFUNCLOSE || t[1] == TCOMMA ) ) ) {
01037     v = s + 1;
01038     switch ( *v ) {
01039     case TMINUS:
01040         v++;
01041         if ( *v == TVECTOR ) {
01042             w = v+1; while ( *w >= 0 ) w++;
01043             if ( w == t ) {
01044                 *t = EMPTY; s++;
01045             }
01046         }
01047     else {
01048         if ( *v == TNUMBER || *v == TNUMBER1 ) {
01049             if ( BITSINWORD == 16 ) { ULONG x; WORD base;
01050                 base = ( *v == TNUMBER ) ? 100: 128;
01051                 vv = v+1; x = 0; while ( *vv >= 0 ) { x = x*base + *vv++; }
01052                 if ( ( vv != t ) || ( ( vv - v ) > 4 ) || ( x > (MAXPOSITIVE+1) ) )
01053                     *fill++ = *s++;
01054                 else { *t = EMPTY; s++; break; }
01055             }
01056             else if ( BITSINWORD == 32 ) { ULONG x; WORD base;
01057                 base = ( *v == TNUMBER ) ? 100: 128;
01058                 vv = v+1; x = 0; while ( *vv >= 0 ) { x = x*base + *vv++; }
01059                 if ( ( vv != t ) || ( ( vv - v ) > 6 ) || ( x > (MAXPOSITIVE+1) ) )
01060                     *fill++ = *s++;
01061                 else { *t = EMPTY; s++; break; }
01062             }
01063             else {
01064                 if ( ( v+2 == t ) || ( v+3 == t && v[2] >= 0 ) )
01065                     { *t = EMPTY; s++; break; }
01066                 else *fill++ = *s++;
01067             }
01068         }
01069         else if ( *v == LPARENTHESIS && t[-1] == RPARENTHESIS ) {
01070             w = v; n = 0;
01071             while ( n >= 0 ) {
01072                 w++;
01073                 if ( *w == LPARENTHESIS ) n++;
01074                 else if ( *w == RPARENTHESIS ) n--;
01075             }
01076             if ( w == ( t-1 ) ) { *t = EMPTY; s++; }
01077             else *fill++ = *s++;
01078         }
01079         else *fill++ = *s++;
01080         break;
01081     }
01082     /* fall through */
01083     case TSETNUM:
01084         v++; while ( *v >= 0 ) v++;
01085         goto tcommon;
01086     case TSYMBOL:
01087         if ( ( v[1] == COEFFSYMBOL || v[1] == NUMERATORSYMBOL
01088             || v[1] == DENOMINATORSYMBOL ) && v[2] < 0 ) {
01089             *fill++ = *s++; break;
01090         }
01091     /* fall through */

```

```

01092         case TSET:
01093         case TVECTOR:
01094         case TINDEX:
01095         case TFUNCTION:
01096         case TDOLLAR:
01097         case TDUBIOUS:
01098         case TSGAMMA:
01099 tcommon: v++; while ( *v >= 0 ) v++;
01100         if ( v == t || ( v[0] == TWILDCARD && v+1 == t ) )
01101         { *t = EMPTY; s++; }
01102         else *fill++ = s++;
01103         break;
01104     case TGENINDEX:
01105         v++;
01106         if ( v == t ) { *t = EMPTY; s++; }
01107         else *fill++ = s++;
01108         break;
01109     case TNUMBER:
01110     case TNUMBER1:
01111         if ( BITSINWORD == 16 ) { ULONG x; WORD base;
01112             base = ( *v == TNUMBER ) ? 100: 128;
01113             vv = v+1; x = 0; while ( *vv >= 0 ) { x = x*base + *vv++; }
01114             if ( ( vv != t ) || ( ( vv - v ) > 4 ) || ( x > MAXPOSITIVE ) )
01115                 *fill++ = s++;
01116             else { *t = EMPTY; s++; break; }
01117         }
01118         else if ( BITSINWORD == 32 ) { ULONG x; WORD base;
01119             base = ( *v == TNUMBER ) ? 100: 128;
01120             vv = v+1; x = 0; while ( *vv >= 0 ) { x = x*base + *vv++; }
01121             if ( ( vv != t ) || ( ( vv - v ) > 6 ) || ( x > MAXPOSITIVE ) )
01122                 *fill++ = s++;
01123             else { *t = EMPTY; s++; break; }
01124         }
01125         else {
01126             if ( ( v+2 == t ) || ( v+3 == t && v[2] >= 0 ) )
01127                 { *t = EMPTY; s++; break; }
01128             else *fill++ = s++;
01129         }
01130         break;
01131     case TWILDARG:
01132         v++; while ( *v >= 0 ) v++;
01133         if ( v == t ) { *t = EMPTY; s++; }
01134         else *fill++ = s++;
01135         break;
01136     case TEXPRESSON:
01137 /*
01138         First establish that there is only the expression
01139         in this argument.
01140 */
01141         vv = s+1;
01142         while ( vv < t ) {
01143             if ( *vv != TEXPRESSON ) break;
01144             vv++; while ( *vv >= 0 ) vv++;
01145         }
01146         if ( vv < t ) { *fill++ = s++; break; }
01147 /*
01148         Find the function
01149 */
01150         w = fill-1; n = 0;
01151         while ( n >= 0 && w >= to ) {
01152             if ( *w == TFUNOPEN ) n--;
01153             else if ( *w == TFUNCLOSE ) n++;
01154             w--;
01155         }
01156         w--; while ( w > to && *w >= 0 ) w--;
01157         if ( *w != TFUNCTION ) { *fill++ = s++; break; }
01158         w++; n = 0;
01159         while ( *w >= 0 ) { n = 128*n + *w++; }
01160         if ( n == GCDFUNCTION-FUNCTION
01161             || n == DIVFUNCTION-FUNCTION
01162             || n == REMFUNCTION-FUNCTION
01163             || n == INVERSEFUNCTION-FUNCTION
01164             || n == MULFUNCTION-FUNCTION ) {
01165             *t = EMPTY; s++;
01166         }
01167         else *fill++ = s++;
01168         break;
01169     default: *fill++ = s++; break;
01170 }
01171 }
01172     else *fill++ = s++;
01173 }
01174     else if ( *s == EMPTY ) s++;
01175     else *fill++ = s++;
01176 }
01177 *fill++ = TENDOFIT;
01178 return(error);

```

```

01179 }
01180
01181 /*
01182     #] simp2token :
01183     #[ simp3atoken :
01184
01185     We hunt for denominators and exponents that seem hidden.
01186     For the denominators we have to recognize:
01187         /fun /fun() /fun^power /fun()^power
01188         /set[n] /set[n]() /set[n]^power /set[n]()^power
01189         /symbol^power (power no number or symbol wildcard)
01190         /dotpr^power (id)
01191         /#^power (id)
01192         /() /()^power
01193         /vect /index /vect(anything) /vect(anything)^power
01194 */
01195
01196 int simp3atoken(SBYTE *s, int mode)
01197 {
01198     int error = 0, n, numexp = 0, denom, base, numprot, i;
01199     SBYTE *t, c;
01200     LONG num;
01201     WORD *prot;
01202     if ( mode == RHSIDE ) {
01203         prot = AC.ProtoType;
01204         numprot = prot[1] - SUBEXPSIZE;
01205         prot += SUBEXPSIZE;
01206     }
01207     else { prot = 0; numprot = 0; }
01208     while ( *s != TENDOFIT ) {
01209         denom = 1;
01210         if ( *s == TDIVIDE ) { denom = -1; s++; }
01211         c = *s;
01212         switch(c) {
01213             case TSYMBOL:
01214             case TNUMBER:
01215             case TNUMBER1:
01216                 s++; while ( *s >= 0 ) s++; /* skip the object */
01217                 if ( *s == TWILDCARD ) s++; /* and the possible wildcard */
01218             dosymbol:
01219                 if ( *s != TPOWER ) continue; /* No power -> done */
01220                 s++; /* Skip the power */
01221                 if ( *s == TMINUS ) s++; /* negative: no difference here */
01222                 if ( *s == TNUMBER || *s == TNUMBER1 ) {
01223                     base = *s == TNUMBER ? 100: 128; /* NUMBER = base 100 */
01224                     s++; /* Now we compose the power */
01225                     num = s++; /* If the number is way too large */
01226                     while ( *s >= 0 ) { /* it may look like not too big */
01227                         if ( num > MAXPOWER ) break; /* Hence... */
01228                         num = base*num + *s++;
01229                     }
01230                     while ( *s >= 0 ) s++; /* Finish the number if needed */
01231                     if ( *s == TPOWER ) goto doublepower;
01232                     if ( num <= MAXPOWER ) continue; /* Simple case */
01233                 }
01234                 else if ( *s == TSYMBOL && c != TNUMBER && c != TNUMBER1 ) {
01235                     s++; n = 0; while ( *s >= 0 ) { n = 128*n + *s++; }
01236                     if ( *s == TWILDCARD ) { s++;
01237                         if ( *s == TPOWER ) goto doublepower;
01238                         continue; }
01239                 }
01240                 /*
01241                 Now we have to test whether n happens to be a wildcard
01242                 */
01243                 if ( mode == RHSIDE ) {
01244                     n += 2*MAXPOWER;
01245                     for ( i = 0; i < numprot; i += 4 ) {
01246                         if ( prot[i+2] == n && prot[i] == SYMTOSYM ) break;
01247                     }
01248                     if ( i < numprot ) break;
01249                     if ( *s == TPOWER ) goto doublepower;
01250                 }
01251                 numexp++;
01252                 break;
01253             case TINDEX:
01254                 s++; while ( *s >= 0 ) s++;
01255                 if ( *s == TWILDCARD ) s++;
01256             doindex:
01257                 if ( denom < 0 || *s == TPOWER ) {
01258                     MesPrint("&Index to a power or in denominator is illegal");
01259                     error = 1;
01260                 }
01261                 break;
01262             case TVECTOR:
01263                 s++; while ( *s >= 0 ) s++;
01264                 if ( *s == TWILDCARD ) s++;
01265             dovector:

```

```

01266         if ( *s == TFUNOPEN ) {
01267             s++; n = 1;
01268             for(;;) {
01269                 if ( *s == TFUNOPEN ) {
01270                     n++;
01271                     MesPrint("&Illegal vector index");
01272                     error = 1;
01273                 }
01274                 else if ( *s == TFUNCLOSE ) {
01275                     n--;
01276                     if ( n <= 0 ) break;
01277                 }
01278                 s++;
01279             }
01280             s++;
01281         }
01282         else if ( *s == TDOT ) goto dodot;
01283         if ( denom < 0 || *s == TPOWER || *s == TPOWER1 ) numexp++;
01284         break;
01285     case TFUNCTION:
01286         s++; while ( *s >= 0 ) s++;
01287         if ( *s == TWILDCARD ) s++;
01288     dofunction:
01289         t = s;
01290         if ( *t == TFUNOPEN ) {
01291             t++; n = 1;
01292             for(;;) {
01293                 if ( *t == TFUNOPEN ) n++;
01294                 else if ( *t == TFUNCLOSE ) { if ( --n <= 0 ) break; }
01295                 t++;
01296             }
01297             t++; s++;
01298         }
01299         if ( denom < 0 || *t == TPOWER || *t == TPOWER1 ) numexp++;
01300         break;
01301     #ifdef WITHFLOAT
01302     case TFLOAT:
01303         s++; while ( *s >= 0 ) s++;
01304         if ( denom < 0 || *s == TPOWER || *s == TPOWER1 ) numexp++;
01305         break;
01306     #endif
01307     case TEXPRESSION:
01308         s++; while ( *s >= 0 ) s++;
01309         t = s;
01310         if ( *t == TFUNOPEN ) {
01311             t++; n = 1;
01312             for(;;) {
01313                 if ( *t == TFUNOPEN ) n++;
01314                 else if ( *t == TFUNCLOSE ) { if ( --n <= 0 ) break; }
01315                 t++;
01316             }
01317             t++;
01318         }
01319         if ( *t == LBRACE ) {
01320             t++; n = 1;
01321             for(;;) {
01322                 if ( *t == LBRACE ) n++;
01323                 else if ( *t == RBRACE ) { if ( --n <= 0 ) break; }
01324                 t++;
01325             }
01326             t++;
01327         }
01328         if ( denom < 0 || ( ( *t == TPOWER || *t == TPOWER1 )
01329             && t[1] == TMINUS ) ) numexp++;
01330         break;
01331     case TDOLLAR:
01332         s++; while ( *s >= 0 ) s++;
01333         if ( denom < 0 || ( ( *s == TPOWER || *s == TPOWER1 )
01334             && s[1] == TMINUS ) ) numexp++;
01335         break;
01336     case LPARENTHESIS:
01337         s++; n = 1; t = s;
01338         for(;;) {
01339             if ( *t == LPARENTHESIS ) n++;
01340             else if ( *t == RPARENTHESIS ) { if ( --n <= 0 ) break; }
01341             t++;
01342         }
01343         t++;
01344         if ( denom > 0 && ( *t == TPOWER || *t == TPOWER1 ) ) {
01345             if ( ( t[1] == TNUMBER || t[1] == TNUMBER1 ) && t[2] >= 0
01346                 && t[3] < 0 ) break;
01347             numexp++;
01348         }
01349         else if ( denom < 0 && ( *t == TPOWER || *t == TPOWER1 ) ) {
01350             if ( t[1] == TMINUS && ( t[2] == TNUMBER
01351                 || t[2] == TNUMBER1 ) && t[3] >= 0
01352                 && t[4] < 0 ) break;

```

```

01353         numexp++;
01354     }
01355     else if ( denom < 0 || ( ( *t == TPOWER || *t == TPOWER1 )
01356 && ( t[1] == TMINUS || t[1] == LPARENTHESIS ) ) ) numexp++;
01357     break;
01358 case TSET:
01359     s++; n = *s++; while ( *s >= 0 ) { n = 128*n + *s++; }
01360     n = Sets[n].type;
01361     switch ( n ) {
01362     case CSYMBOL: goto dosymbol;
01363     case CINDEX: goto doindex;
01364     case CVECTOR: goto dovector;
01365     case CFUNCTION: goto dofunction;
01366     case CNUMBER: goto dosymbol;
01367     case CMODEL:
01368         if ( denom < 0 || *s == TPOWER ) {
01369             MesPrint("&A model to a power or in denominator is illegal");
01370             error = 1;
01371         }
01372         break;
01373     case ANYTYPE:
01374         if ( denom < 0 || *s == TPOWER ) {
01375             MesPrint("&A set without type to a power or in denominator is illegal");
01376             error = 1;
01377         }
01378         break;
01379     default: error = 1; break;
01380     }
01381     break;
01382 case TDOT:
01383     s++;
01384     if ( *s == TVECTOR ) { s++; while ( *s >= 0 ) s++; }
01385     else if ( *s == TSET ) {
01386         s++; n = *s++; while ( *s >= 0 ) { n = 128*n + *s++; }
01387         if ( Sets[n].type != CVECTOR ) {
01388             MesPrint("&Set in dotproduct is not a set of vectors");
01389             error = 1;
01390         }
01391         if ( *s == LBRACE ) {
01392             s++; n = 1;
01393             for(;;) {
01394                 if ( *s == LBRACE ) n++;
01395                 else if ( *s == RBRACE ) { if ( --n <= 0 ) break; }
01396                 s++;
01397             }
01398             s++;
01399         }
01400         else {
01401             MesPrint("&Set without argument in dotproduct");
01402             error = 1;
01403         }
01404     }
01405     else if ( *s == TSETNUM ) {
01406         s++; n = *s++; while ( *s >= 0 ) { n = 128*n + *s++; }
01407         if ( *s != TVECTOR ) goto nodot;
01408         s++; n = *s++; while ( *s >= 0 ) { n = 128*n + *s++; }
01409         if ( Sets[n].type != CVECTOR ) {
01410             MesPrint("&Set in dotproduct is not a set of vectors");
01411             error = 1;
01412         }
01413     }
01414     else {
01415         MesPrint("&Illegal second element in dotproduct");
01416         error = 1;
01417         s++; while ( *s >= 0 ) s++;
01418     }
01419     goto dosymbol;
01420 default:
01421     s++; while ( *s >= 0 ) s++;
01422     break;
01423 }
01424 }
01425 if ( error ) return(-1);
01426 return(numexp);
01427 doublepower:
01428     MesPrint("&Dubious notation with object^power1^power2");
01429     return(-1);
01430 }
01431
01432 /*
01433     #] simp3atoken :
01434     #[ simp3btoken :
01435 */
01436
01437 int simp3btoken(SBYTE *s, int mode)
01438 {
01439     int error = 0, i, numprot, n, denom, base, inset = 0, dotp, sube = 0;

```



```

01440     SBYTE *t, c, *fill, *ff, *ss;
01441     LONG num;
01442     WORD *prot;
01443     if ( mode == RHSIDE ) {
01444         prot = AC.ProtoType;
01445         numprot = prot[1] - SUBEXPSIZE;
01446         prot += SUBEXPSIZE;
01447     }
01448     else { prot = 0; numprot = 0; }
01449     fill = s;
01450     while ( *s == EMPTY ) s++;
01451     while ( *s != TENDOFIT ) {
01452         if ( *s == EMPTY ) { s++; continue; }
01453         denom = 1;
01454         if ( *s == TDIVIDE ) { denom = -1; *fill++ = *s++; }
01455         ff = fill; ss = s; c = *s;
01456         if ( c == TSETNUM ) {
01457             *fill++ = *s++; while ( *s >= 0 ) *fill++ = *s++;
01458             c = *s;
01459         }
01460         dotp = 0;
01461         switch(c) {
01462             case TSYMBOL:
01463             case TNUMBER:
01464             case TNUMBER1:
01465                 *fill++ = *s++;
01466                 while ( *s >= 0 ) *fill++ = *s++;
01467                 if ( *s == TWILDCARD ) *fill++ = *s++;
01468 dosymbol:
01469                 t = s;
01470                 if ( *s != TPOWER ) continue;
01471                 *fill++ = *s++;
01472                 if ( *s == TMINUS ) *fill++ = *s++;
01473                 if ( *s == TPLUS ) s++;
01474                 if ( *s == TSETNUM ) {
01475                     *fill++ = *s++; while ( *s >= 0 ) *fill++ = *s++;
01476                     inset = 1;
01477                 }
01478                 else inset = 0;
01479                 if ( *s == TNUMBER || *s == TNUMBER1 ) {
01480                     base = *s == TNUMBER ? 100: 128;
01481                     *fill++ = *s++;
01482                     num = *s++; *fill++ = num;
01483                     while ( *s >= 0 ) {
01484                         if ( num > MAXPOWER ) break;
01485                         *fill++ = *s;
01486                         num = base*num + *s++;
01487                     }
01488                     while ( *s >= 0 ) *fill++ = *s++;
01489                     if ( num <= MAXPOWER ) continue;
01490                     goto putexpl;
01491                 }
01492                 else if ( *s == TSYMBOL && c != TNUMBER && c != TNUMBER1 ) {
01493                     *fill++ = *s++;
01494                     n = 0; while ( *s >= 0 ) { n = 128*n + *s; *fill++ = *s++; }
01495                     if ( *s == TWILDCARD ) { *fill++ = *s++;
01496                         if ( *s == TPOWER ) goto doublepower;
01497                     break; }
01498 /*
01499     Now we have to test whether n happens to be a wildcard
01500 */
01501                 if ( mode == RHSIDE && inset == 0 ) {
01502 /*
01503                     n += WILDOFFSET;*/
01504                     for ( i = 0; i < numprot; i += 4 ) {
01505                         if ( prot[i+2] == n && prot[i] == SYMTOSYM ) break;
01506                     }
01507                     if ( i < numprot ) break;
01508                 }
01509 putexpl:
01510                 fill = ff;
01511                 if ( denom < 0 ) fill[-1] = TMULTIPLY;
01512                 *fill++ = TFUNCTION; *fill++ = (SBYTE)(AM.expnum); *fill++ = TFUNOPEN;
01513                 if ( dotp ) *fill++ = LPARENTHESIS;
01514                 while ( ss < t ) *fill++ = *ss++;
01515                 if ( dotp ) *fill++ = RPARENTHESIS;
01516                 *fill++ = TCOMMA;
01517                 ss++; /* Skip TPOWER */
01518                 if ( *ss == TMINUS ) { denom = -denom; ss++; }
01519                 if ( denom < 0 ) {
01520                     *fill++ = LPARENTHESIS;
01521                     *fill++ = TMINUS;
01522                     while ( ss < s ) *fill++ = *ss++;
01523                     *fill++ = RPARENTHESIS;
01524                 }
01525                 else {
01526                     while ( ss < s ) *fill++ = *ss++;
01527                 }

```

```

01527         *fill++ = TFUNCLOSE;
01528         if ( *ss == TPOWER ) goto doublepower;
01529     }
01530     else { /* other objects can be composite */
01531         goto dofunpower;
01532     }
01533     break;
01534 case TINDEX:
01535     *fill++ = *s++; while ( *s >= 0 ) *fill++ = *s++;
01536     if ( *s == TWILDCARD ) *fill++ = *s++;
01537     break;
01538 case TVECTOR:
01539     *fill++ = *s++; while ( *s >= 0 ) *fill++ = *s++;
01540     if ( *s == TWILDCARD ) *fill++ = *s++;
01541 dovector:
01542     if ( *s == TFUNOPEN ) {
01543         while ( *s != TFUNCLOSE ) *fill++ = *s++;
01544         *fill++ = *s++;
01545     }
01546     else if ( *s == TDOT ) goto dodot;
01547     t = s;
01548     goto dofunpower;
01549 #ifdef WITHFLOAT
01550 case TFLOAT:
01551     *fill++ = *s++; while ( *s >= 0 ) *fill++ = *s++;
01552     t = s;
01553     sube = 0;
01554     goto dofunpower;
01555 #endif
01556 case TFUNCTION:
01557     *fill++ = *s++; while ( *s >= 0 ) *fill++ = *s++;
01558     if ( *s == TWILDCARD ) *fill++ = *s++;
01559 dofunction:
01560     t = s;
01561     if ( *t == TFUNOPEN ) {
01562         t++; n = 1;
01563         for(;;) {
01564             if ( *t == TFUNOPEN ) n++;
01565             else if ( *t == TFUNCLOSE ) { if ( --n <= 0 ) break; }
01566             t++;
01567         }
01568         t++; *fill++ = *s++;
01569     }
01570     sube = 0;
01571 dofunpower:
01572     if ( *t == TPOWER || *t == TPOWER1 ) {
01573         if ( sube ) {
01574             if ( ( t[1] == TNUMBER || t[1] == TNUMBER1 )
01575                 && denom > 0 ) {
01576                 if ( t[2] >= 0 && t[3] < 0 ) { sube = 0; break; }
01577             }
01578             else if ( t[1] == TMINUS && denom < 0 &&
01579                 ( t[2] == TNUMBER || t[2] == TNUMBER1 ) ) {
01580                 if ( t[2] >= 0 && t[3] < 0 ) { sube = 0; break; }
01581             }
01582             sube = 0;
01583         }
01584         fill = ff;
01585         *fill++ = TFUNCTION; *fill++ = (SBYTE) (AM.expnum); *fill++ = TFUNOPEN;
01586         *fill++ = LPARENTHESIS;
01587         while ( ss < t ) *fill++ = *ss++;
01588         t++;
01589         *fill++ = RPARENTHESIS; *fill++ = TCOMMA;
01590         if ( *t == TMINUS ) { t++; denom = -denom; }
01591         *fill++ = LPARENTHESIS;
01592         if ( denom < 0 ) *fill++ = TMINUS;
01593         if ( *t == LPARENTHESIS ) {
01594             *fill++ = *t++; n = 0;
01595             while ( n >= 0 ) {
01596                 if ( *t == LPARENTHESIS ) n++;
01597                 else if ( *t == RPARENTHESIS ) n--;
01598                 *fill++ = *t++;
01599             }
01600         }
01601         else if ( *t == TFUNCTION || *t == TDUBIOUS ) {
01602             *fill++ = *t++; while ( *t >= 0 ) *fill++ = *t++;
01603             if ( *t == TWILDCARD ) *fill++ = *t++;
01604             if ( *t == TFUNOPEN ) {
01605                 *fill++ = *t++; n = 0;
01606                 while ( n >= 0 ) {
01607                     if ( *t == TFUNOPEN ) n++;
01608                     else if ( *t == TFUNCLOSE ) n--;
01609                     *fill++ = *t++;
01610                 }
01611             }
01612         }
01613         else if ( *t == TSET ) {

```

```

01614         *fill++ = *t++; n = 0;
01615         while ( *t >= 0 ) { n = 128*n + *t; *fill++ = *t++; }
01616         if ( *t == LBRACE ) {
01617             if ( n < AM.NumFixedSets || Sets[n].type == CRANGE ) {
01618                 MesPrint("&This type of usage of sets is not allowed");
01619                 error = 1;
01620             }
01621             *fill++ = *t++; n = 0;
01622             while ( n >= 0 ) {
01623                 if ( *t == LBRACE ) n++;
01624                 else if ( *t == RBRACE ) n--;
01625                 *fill++ = *t++;
01626             }
01627         }
01628     }
01629     else if ( *t == TEXPRESSION ) {
01630         *fill++ = *t++; while ( *t >= 0 ) *fill++ = *t++;
01631         if ( *t == TFUNOPEN ) {
01632             *fill++ = *t++; n = 0;
01633             while ( n >= 0 ) {
01634                 if ( *t == TFUNOPEN ) n++;
01635                 else if ( *t == TFUNCLOSE ) n--;
01636                 *fill++ = *t++;
01637             }
01638         }
01639         if ( *t == LBRACE ) {
01640             *fill++ = *t++; n = 0;
01641             while ( n >= 0 ) {
01642                 if ( *t == LBRACE ) n++;
01643                 else if ( *t == RBRACE ) n--;
01644                 *fill++ = *t++;
01645             }
01646         }
01647     }
01648     else if ( *t == TVECTOR ) {
01649         *fill++ = *t++; while ( *t >= 0 ) *fill++ = *t++;
01650         if ( *t == TFUNOPEN ) {
01651             *fill++ = *t++; n = 0;
01652             while ( n >= 0 ) {
01653                 if ( *t == TFUNOPEN ) n++;
01654                 else if ( *t == TFUNCLOSE ) n--;
01655                 *fill++ = *t++;
01656             }
01657         }
01658     }
01659     else if ( *t == TDOT ) {
01660         *fill++ = *t++;
01661         if ( *t == TVECTOR || *t == TDUBIOUS ) {
01662             *fill++ = *t++; while ( *t >= 0 ) *fill++ = *t++;
01663         }
01664         else if ( *t == TSET ) {
01665             *fill++ = *t++; num = 0;
01666             while ( *t >= 0 ) { num = 128*num + *t; *fill++ = *t++; }
01667             if ( Sets[num].type != CVECTOR ) {
01668                 MesPrint("&Illegal set type in dotproduct");
01669                 error = 1;
01670             }
01671             if ( *t == LBRACE ) {
01672                 *fill++ = *t++; n = 0;
01673                 while ( n >= 0 ) {
01674                     if ( *t == LBRACE ) n++;
01675                     else if ( *t == RBRACE ) n--;
01676                     *fill++ = *t++;
01677                 }
01678             }
01679         }
01680         else if ( *t == TSETNUM ) {
01681             *fill++ = *t++;
01682             while ( *t >= 0 ) { *fill++ = *t++; }
01683             *fill++ = *t++;
01684             while ( *t >= 0 ) { *fill++ = *t++; }
01685         }
01686     }
01687     else {
01688         MesPrint("&Illegal second element in dotproduct");
01689         error = 1;
01690     }
01691 }
01692 else {
01693     *fill++ = *t++; while ( *t >= 0 ) *fill++ = *t++;
01694     if ( *t == TWILDCARD ) *fill++ = *t++;
01695 }
01696 *fill++ = RPARENTHESIS; *fill++ = TFUNCLOSE;
01697 if ( *t == TPOWER ) goto doublepower;
01698 while ( fill > ff ) *--t = *--fill;
01699 s = t;
01700 }
01701 else if ( denom < 0 ) {

```

```

01701         fill = ff; ff[-1] = TMULTIPLY;
01702         *fill++ = TFUNCTION; *fill++ = (SBYTE) (AM.denomnum);
01703         *fill++ = TFUNOPEN; *fill++ = LPARENTHESIS;
01704         while ( ss < t ) *fill++ = *ss++;
01705         *fill++ = RPARENTHESIS; *fill++ = TFUNCLOSE;
01706         while ( fill > ff ) *--t = *--fill;
01707         s = t; denom = 1; sube = 0;
01708         break;
01709     }
01710     sube = 0;
01711     break;
01712 case TEXPRESSION:
01713     *fill++ = *s++; while ( *s >= 0 ) *fill++ = *s++;
01714     t = s;
01715     if ( *t == TFUNOPEN ) {
01716         t++; n = 1;
01717         for(;;) {
01718             if ( *t == TFUNOPEN ) n++;
01719             else if ( *t == TFUNCLOSE ) { if ( --n <= 0 ) break; }
01720             t++;
01721         }
01722         t++;
01723     }
01724     if ( *t == LBRACE ) {
01725         t++; n = 1;
01726         for(;;) {
01727             if ( *t == LBRACE ) n++;
01728             else if ( *t == RBRACE ) { if ( --n <= 0 ) break; }
01729             t++;
01730         }
01731         t++;
01732     }
01733     if ( t > s || denom < 0 || ( ( *t == TPOWER || *t == TPOWER1 )
01734         && t[1] == TMINUS ) ) goto dofunpower;
01735     else goto dosymbol;
01736 case TDOLLAR:
01737     *fill++ = *s++; while ( *s >= 0 ) *fill++ = *s++;
01738     goto dosymbol;
01739 case LPARENTHESIS:
01740     *fill++ = *s++; n = 1; t = s;
01741     for(;;) {
01742         if ( *t == LPARENTHESIS ) n++;
01743         else if ( *t == RPARENTHESIS ) { if ( --n <= 0 ) break; }
01744         t++;
01745     }
01746     t++; sube = 1;
01747     goto dofunpower;
01748 case TSET:
01749     *fill++ = *s++; n = *s++; *fill++ = (SBYTE)n;
01750     while ( *s >= 0 ) { *fill++ = *s; n = 128*n + *s++; }
01751     n = Sets[n].type;
01752     switch ( n ) {
01753         case CSYMBOL: goto dosymbol;
01754         case CINDEX: break;
01755         case CVECTOR: goto dovector;
01756         case CFUNCTION: goto dofunction;
01757         case CNUMBER: goto dosymbol;
01758         case CMODEL: break;
01759         case ANYTYPE: break;
01760         default: error = 1; break;
01761     }
01762     break;
01763 case TDOT:
01764     *fill++ = *s++;
01765     if ( *s == TVECTOR ) {
01766         *fill++ = *s++; while ( *s >= 0 ) *fill++ = *s++;
01767     }
01768     else if ( *s == TSET ) {
01769         *fill++ = *s++; n = *s++; *fill++ = (SBYTE)n;
01770         while ( *s >= 0 ) { *fill++ = *s; n = 128*n + *s++; }
01771         if ( *s == LBRACE ) {
01772             if ( n < AM.NumFixedSets || Sets[n].type == CRANGE ) {
01773                 MesPrint("&This type of usage of sets is not allowed");
01774                 error = 1;
01775             }
01776             *fill++ = *s++; n = 1;
01777             for(;;) {
01778                 if ( *s == LBRACE ) n++;
01779                 else if ( *s == RBRACE ) { if ( --n <= 0 ) break; }
01780                 *fill++ = *s++;
01781             }
01782             *fill++ = *s++;
01783         }
01784         else {
01785             MesPrint("&Set without argument in dotproduct");
01786             error = 1;
01787         }
01788     }

```

```

01788     }
01789     else if ( *s == TSETNUM ) {
01790         *fill++ = *s++; while ( *s >= 0 ) *fill++ = *s++;
01791         if ( *s != TVECTOR ) goto nodot;
01792         *fill++ = *s++; while ( *s >= 0 ) *fill++ = *s++;
01793     }
01794     else {
01795 nodot:    MesPrint("&Illegal second element in dotproduct");
01796         error = 1;
01797         *fill++ = *s++;
01798         while ( *s >= 0 ) *fill++ = *s++;
01799     }
01800     dotp = 1;
01801     goto dosymbol;
01802 default:
01803     *fill++ = *s++;
01804     while ( *s >= 0 ) *fill++ = *s++;
01805     break;
01806 }
01807 }
01808 *fill = TENDOFIT;
01809 return(error);
01810 doublepower:
01811 MesPrint("&Dubious notation with power of power");
01812 return(-1);
01813 }
01814
01815 /*
01816     #] simp3btoken :
01817     #[ simp4token :
01818
01819     Deal with the set[n] objects in the RHS.
01820 */
01821
01822 int simp4token(SBYTE *s)
01823 {
01824     int error = 0, n, nsym, settype;
01825     WORD i, *w, *wstop, level;
01826     SBYTE *const s0 = s;
01827     SBYTE *fill = s, *s1, *s2, *s3, type, slbuf[10];
01828     SBYTE *tbuf = s, *t, *t1;
01829
01830     while ( *s != TENDOFIT ) {
01831         if ( *s != TSET ) {
01832             if ( *s == TEMPTY ) s++;
01833             else *fill++ = *s++;
01834             continue;
01835         }
01836         if ( fill >= (s0+1) && fill[-1] == TWILDCARD ) { *fill++ = *s++; continue; }
01837         if ( fill >= (s0+2) && fill[-1] == TNOT && fill[-2] == TWILDCARD ) { *fill++ = *s++; continue; }
01838
01839         s1 = s++; n = 0; while ( *s >= 0 ) { n = 128*n + *s++; }
01840         i = Sets[n].type;
01841         if ( *s != LBRACE ) { while ( s1 < s ) *fill++ = *s1++; continue; }
01842         if ( n < AM.NumFixedSets || i == CRANGE ) {
01843             MesPrint("&It is not allowed to refer to individual elements of built in or ranged sets");
01844             error = 1;
01845         }
01846         s++;
01847         if ( *s != TSYMBOL && *s != TDOLLAR ) {
01848             MesPrint("&Set index in RHS is not a wildcard symbol or $-variable");
01849             error = 1;
01850             while ( s1 < s ) *fill++ = *s1++;
01851             continue;
01852         }
01853         settype = ( *s == TDOLLAR );
01854         s++; nsym = 0; s2 = s;
01855         while ( *s >= 0 ) nsym = 128*nsym + *s++;
01856         if ( *s != RBRACE ) {
01857             MesPrint("&Improper set argument in RHS");
01858             error = 1;
01859             while ( s1 < s ) *fill++ = *s1++;
01860             continue;
01861         }
01862         s++;
01863         /*
01864         Verify that nsym is a wildcard
01865         */
01866         if ( !settype ) {
01867             w = AC.ProtoType; wstop = w + w[1]; w += SUBEXPSIZE;
01868             while ( w < wstop ) {
01869                 if ( *w == SYMTOSYM && w[2] == nsym ) break;
01870                 w += w[1];
01871             }
01872             if ( w >= wstop ) {
01873                 It could still be a summation parameter!

```

```

01874 */
01875         t = fill - 1;
01876         while ( t >= tbuf ) {
01877             if ( *t == TFUNCLOSE ) {
01878                 level = 1; t--;
01879                 while ( t >= tbuf ) {
01880                     if ( *t == TFUNCLOSE ) level++;
01881                     else if ( *t == TFUNOPEN ) {
01882                         level--;
01883                         if ( level == 0 ) break;
01884                     }
01885                     t--;
01886                 }
01887             }
01888             else if ( *t == RBRACE ) {
01889                 level = 1; t--;
01890                 while ( t >= tbuf ) {
01891                     if ( *t == RBRACE ) level++;
01892                     else if ( *t == LBRACE ) {
01893                         level--;
01894                         if ( level == 0 ) break;
01895                     }
01896                     t--;
01897                 }
01898             }
01899             else if ( *t == RPARENTHESIS ) {
01900                 level = 1; t--;
01901                 while ( t >= tbuf ) {
01902                     if ( *t == RPARENTHESIS ) level++;
01903                     else if ( *t == LPARENTHESIS ) {
01904                         level--;
01905                         if ( level == 0 ) break;
01906                     }
01907                     t--;
01908                 }
01909             }
01910             else if ( *t == TFUNOPEN ) {
01911                 t1 = t-1;
01912                 while ( *t1 > 0 && t1 > tbuf ) t1--;
01913                 if ( *t1 == TFUNCTION ) {
01914                     t1++; level = 0;
01915                     while ( *t1 > 0 ) level = level*128+*t1++;
01916                     if ( level == (SUMF1-FUNCTION)
01917                         || level == (SUMF2-FUNCTION) ) {
01918                         t1 = t + 1;
01919                         if ( *t1 == LPARENTHESIS ) t1++;
01920                         if ( *t1 == TSYMBOL ) {
01921                             if ( ( t1[1] == COEFFSYMBOL
01922                                 || t1[1] == NUMERATORSYMBOL
01923                                 || t1[1] == DENOMINATORSYMBOL )
01924                                 && t1[2] < 0 ) {}
01925                             else {
01926                                 t1++; level = 0;
01927                                 while ( *t1 >= 0 && t1 < fill ) level = 128*level + *t1++;
01928                                 if ( level == nsym && t1 < fill ) {
01929                                     if ( t[1] == LPARENTHESIS
01930                                         && *t1 == RPARENTHESIS && t1[1] == TCOMMA ) break;
01931                                     if ( t[1] != LPARENTHESIS && *t1 == TCOMMA ) break;
01932                                 }
01933                             }
01934                         }
01935                     }
01936                 }
01937             }
01938             t--;
01939         }
01940         if ( t < tbuf ) {
01941             fill--;
01942             MesPrint("&Set index in RHS is not a wildcard symbol");
01943             error = 1;
01944             while ( s1 < s ) *fill++ = *s1++;
01945             continue;
01946         }
01947     }
01948 }
01949 /*
01950 Now replace by a set marker: TSETNUM,nsym,TYPE,setnumber
01951 */
01952 switch ( i ) {
01953     case CSYMBOL: type = TSYMBOL; break;
01954     case CINDEX: type = TINDEX; break;
01955     case CVECTOR: type = TVECTOR; break;
01956     case CFUNCTION: type = TFUNCTION; break;
01957     case CNUMBER: type = TNUMBER1; break;
01958     case CDUBIOUS: type = TDUBIOUS; break;
01959     default:
01960         MesPrint("&Unknown set type in simp4token");

```

```

01961         error = 1; type = CDUBIOUS; break;
01962     }
01963     s3 = slbuf; s1++;
01964     while ( *s1 >= 0 ) *s3++ = *s1++;
01965     *s3 = -1; s1 = slbuf;
01966     if ( settype ) *fill++ = TSETDOL;
01967     else           *fill++ = TSETNUM;
01968     while ( *s2 >= 0 ) *fill++ = *s2++;
01969     *fill++ = type; while ( *s1 >= 0 ) *fill++ = *s1++;
01970 }
01971 *fill++ = TENDOFIT;
01972 return(error);
01973 }
01974
01975 /*
01976     #] simp4token :
01977     #[ simp5token :
01978
01979     Making sure that first argument of sumfunction is not a wildcard already
01980 */
01981
01982 int simp5token(SBYTE *s, int mode)
01983 {
01984     int error = 0, n, type;
01985     WORD *w, *wstop;
01986     if ( mode == RHSIDE ) {
01987         while ( *s != TENDOFIT ) {
01988             if ( *s == TFUNCTION ) {
01989                 s++; n = 0; while ( *s >= 0 ) n = 128*n + *s++;
01990                 if ( n == AM.sumnum || n == AM.sumpnum ) {
01991                     if ( *s != TFUNOPEN ) continue;
01992                     s++;
01993                     if ( *s != TSYMBOL && *s != TINDEX ) continue;
01994                     type = *s++;
01995                     n = 0; while ( *s >= 0 ) n = 128*n + *s++;
01996                     if ( type == TINDEX ) n += AM.OffsetIndex;
01997                     if ( *s != TCOMMA ) continue;
01998                     w = AC.ProtoType;
01999                     wstop = w + w[1];
02000                     w += SUBEXPSIZE;
02001                     while ( w < wstop ) {
02002                         if ( w[2] == n ) {
02003                             if ( ( type == TSYMBOL && ( w[0] == SYMTOSYM
02004                                 || w[0] == SYMTONUM || w[0] == SYMTOSUB ) ) || (
02005                                 type == TINDEX && ( w[0] == INDTOIND
02006                                 || w[0] == INDTOSUB ) ) ) {
02007                                 error = 1;
02008                                 MesPrint("&Parameter of sum function is already a wildcard");
02009                             }
02010                         }
02011                         w += w[1];
02012                     }
02013                 }
02014             }
02015             else s++;
02016         }
02017     }
02018     return(error);
02019 }
02020
02021 /*
02022     #] simp5token :
02023     #[ simp6token :
02024
02025     Making sure that factorized expressions are used properly
02026 */
02027
02028 int simp6token(SBYTE *tokens, int mode)
02029 {
02030     /* EXPRESSIONS e = Expressions; */
02031     int error = 0, n;
02032     int level = 0, haveone = 0;
02033     SBYTE *s = tokens, *ss;
02034     LONG numterms;
02035     WORD funnum = 0;
02036     GETIDENTITY
02037     if ( mode == RHSIDE ) {
02038         while ( *s == TPLUS || *s == TMINUS ) s++;
02039         numterms = 1;
02040         while ( *s != TENDOFIT ) {
02041             if ( *s == LPARENTHESIS ) level++;
02042             else if ( *s == RPARENTHESIS ) level--;
02043             else if ( *s == TFUNOPEN ) level++;
02044             else if ( *s == TFUNCLOSE ) level--;
02045             else if ( ( *s == TPLUS || *s == TMINUS ) && level == 0 ) {
02046                 /*
02047                     Special exception: x^-1 etc.

```

```

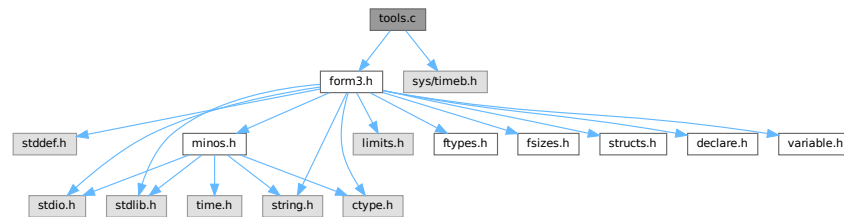
02048 */
02049         if ( s[-1] != TPOWER && s[-1] != TPLUS && s[-1] != TMINUS ) {
02050             numterms++;
02051         }
02052     }
02053     else if ( *s == TEXPRESSION ) {
02054         ss = s;
02055         s++; n = 0; while ( *s >= 0 ) n = 128*n + *s++;
02056
02057         if ( Expressions[n].status == STOREDEXPRESSION ) {
02058             POSITION position;
02059             /*
02060             #ifdef WITHPTHREADS
02061                 RENUMBER renumber;
02062             #endif
02063             */
02064             RENUMBER renumber;
02065
02066             WORD TMproto[SUBEXPSIZE];
02067             TMproto[0] = EXPRESSION;
02068             TMproto[1] = SUBEXPSIZE;
02069             TMproto[2] = n;
02070             TMproto[3] = 1;
02071             { int ie; for ( ie = 4; ie < SUBEXPSIZE; ie++ ) TMproto[ie] = 0; }
02072             AT.TMaddr = TMproto;
02073             PUTZERO(position);
02074             /*
02075             if ( (
02076             #ifdef WITHPTHREADS
02077                 renumber =
02078             #endif
02079                 GetTable(n,&position,0) ) == 0 )
02080             */
02081             if ( ( renumber = GetTable(n,&position,0) ) == 0 )
02082             {
02083                 error = 1;
02084                 MesPrint("&Problems getting information about stored expression %s(4)"
02085                     ,EXPRNAME(n));
02086             }
02087             /*
02088             #ifdef WITHPTHREADS
02089             */
02090             if ( renumber->sympb.lo != AN.dummyrenumlist )
02091                 M_free(renumber->sympb.lo,"VarSpace");
02092             M_free(renumber,"Renumber");
02093             /*
02094             #endif
02095             */
02096         }
02097
02098         if ( ( ( AS.Oldvflags[n] & ISFACTORIZED ) != 0 ) && *s != LBRACE ) {
02099             if ( level == 0 ) {
02100                 haveone = 1;
02101             }
02102             else if ( error == 0 ) {
02103                 if ( ss[-1] != TFUNOPEN || funnum != NUMFACTORS-FUNCTION ) {
02104                     MesPrint("&Illegal use of factorized expression(s) in RHS");
02105                     error = 1;
02106                 }
02107             }
02108         }
02109         continue;
02110     }
02111     else if ( *s == TFUNCTION ) {
02112         s++; funnum = 0; while ( *s >= 0 ) funnum = 128*fnum + *s++;
02113         continue;
02114     }
02115     s++;
02116 }
02117 if ( haveone ) {
02118     if ( numterms > 1 ) {
02119         MesPrint("&Factorized expression in RHS in an expression of more than one term.");
02120         error = 1;
02121     }
02122     else if ( AC.ToBeInFactors == 0 ) {
02123         MesPrint("&Attempt to put a factorized expression inside an unfactorized
expression.");
02124         error = 1;
02125     }
02126 }
02127 }
02128 return(error);
02129 }
02130
02131 /*
02132     #] simp6token :
02133     #] Compiler :

```


02134 */

4.146 tools.c File Reference

```
#include "form3.h"
#include <sys/timeb.h>
Include dependency graph for tools.c:
```



Macros

- #define [FILLVALUE](#) 0
- #define [COPYFILEBUFSIZE](#) 40960L
- #define [TERMMEMSTARTNUM](#) 16
- #define [TERMEXTRAWORDS](#) 10
- #define [NUMBERMEMSTARTNUM](#) 16
- #define [NUMBEREXTRAWORDS](#) 10L
- #define [DODOUBLE](#)(x)
- #define [DOEXPAND](#)(x)

Functions

- UBYTE * [LoadInputFile](#) (UBYTE *filename, int type)
- UBYTE [ReadFromStream](#) (STREAM *stream)
- UBYTE [GetFromStream](#) (STREAM *stream)
- UBYTE [LookInStream](#) (STREAM *stream)
- STREAM * [OpenStream](#) (UBYTE *name, int type, int prevarmode, int raiselow)
- int [LocateFile](#) (UBYTE **name, int type)
- STREAM * [CloseStream](#) (STREAM *stream)
- STREAM * [CreateStream](#) (UBYTE *where)
- LONG [GetStreamPosition](#) (STREAM *stream)
- void [PositionStream](#) (STREAM *stream, LONG position)
- int [ReverseStatements](#) (STREAM *stream)
- void [StartFiles](#) (void)
- int [OpenFile](#) (char *name)
- int [OpenAddFile](#) (char *name)
- int [ReOpenFile](#) (char *name)
- int [CreateFile](#) (char *name)
- int [CreateLogFile](#) (char *name)
- void [CloseFile](#) (int handle)
- int [CopyFile](#) (char *source, char *dest)

- int [CreateHandle](#) (void)
- LONG [ReadFile](#) (int handle, UBYTE *buffer, LONG size)
- LONG [ReadPosFile](#) (PHEAD [FILEHANDLE](#) *fi, UBYTE *buffer, LONG size, [POSITION](#) *pos)
- LONG [WriteFileToFile](#) (int handle, UBYTE *buffer, LONG size)
- void [SeekFile](#) (int handle, [POSITION](#) *offset, int origin)
- LONG [TellFile](#) (int handle)
- void [TELLFILE](#) (int handle, [POSITION](#) *position)
- void [FlushFile](#) (int handle)
- int [GetPosFile](#) (int handle, fpos_t *pospointer)
- int [SetPosFile](#) (int handle, fpos_t *pospointer)
- void [SynchFile](#) (int handle)
- void [TruncateFile](#) (int handle)
- int [GetChannel](#) (char *name, int mode)
- int [GetAppendChannel](#) (char *name)
- int [CloseChannel](#) (char *name)
- void [UpdateMaxSize](#) (void)
- int [StrCmp](#) (UBYTE *s1, UBYTE *s2)
- int [StrlCmp](#) (UBYTE *s1, UBYTE *s2)
- int [StrHlCmp](#) (UBYTE *s1, UBYTE *s2)
- int [StrlCont](#) (UBYTE *s1, UBYTE *s2)
- int [CmpArray](#) (WORD *t1, WORD *t2, WORD n)
- int [ConWord](#) (UBYTE *s1, UBYTE *s2)
- int [StrLen](#) (UBYTE *s)
- void [NumToStr](#) (UBYTE *s, LONG x)
- void [WriteString](#) (int type, UBYTE *str, int num)
- void [WriteUnfinString](#) (int type, UBYTE *str, int num)
- UBYTE * [AddToString](#) (UBYTE *outstring, UBYTE *extrastring, int par)
- UBYTE * [strDup1](#) (UBYTE *instring, char *ifwrong)
- UBYTE * [EndOfToken](#) (UBYTE *s)
- UBYTE * [ToToken](#) (UBYTE *s)
- UBYTE * [SkipField](#) (UBYTE *s, int level)
- WORD [ReadSnum](#) (UBYTE **p)
- UBYTE * [NumCopy](#) (WORD y, UBYTE *to)
- char * [LongCopy](#) (LONG y, char *to)
- char * [LongLongCopy](#) (off_t *y, char *to)
- UBYTE * [MakeDate](#) (void)
- int [set_in](#) (UBYTE ch, [set_of_char](#) set)
- [one_byte](#) [set_set](#) (UBYTE ch, [set_of_char](#) set)
- [one_byte](#) [set_del](#) (UBYTE ch, [set_of_char](#) set)
- [one_byte](#) [set_sub](#) ([set_of_char](#) set, [set_of_char](#) set1, [set_of_char](#) set2)
- void [iniTools](#) (void)
- void * [Malloc](#) (LONG size)
- void * [Malloc1](#) (LONG size, const char *messageifwrong)
- void [M_free](#) (void *x, const char *where)
- void [M_check1](#) (void)
- void [M_print](#) (void)
- void [TermMallocAddMemory](#) (PHEAD0)
- void [NumberMallocAddMemory](#) (PHEAD0)
- void [CacheNumberMallocAddMemory](#) (PHEAD0)
- void * [FromList](#) ([LIST](#) *L)
- void * [From0List](#) ([LIST](#) *L)
- void * [FromVarList](#) ([LIST](#) *L)
- int [DoubleList](#) (void ***lijst, int *oldsize, int objectsize, char *nameoftype)
- int [DoubleLList](#) (void ***lijst, LONG *oldsize, int objectsize, char *nameoftype)
- void [DoubleBuffer](#) (void **start, void **stop, int size, char *text)

- void [ExpandBuffer](#) (void **buffer, LONG *oldsize, int type)
- LONG [iexp](#) (LONG x, int p)
- void [ToGeneral](#) (WORD *r, WORD *m, WORD par)
- int [ToFast](#) (WORD *r, WORD *m)
- WORD [ToPolyFunGeneral](#) (PHEAD WORD *term)
- int [IsLikeVector](#) (WORD *arg)
- int [AreArgsEqual](#) (WORD *arg1, WORD *arg2)
- int [CompareArgs](#) (WORD *arg1, WORD *arg2)
- int [CompArg](#) (WORD *s1, WORD *s2)
- LONG [TimeWallClock](#) (WORD par)
- LONG [TimeChildren](#) (WORD par)
- LONG [TimeCPU](#) (WORD par)
- int [Crash](#) (void)
- int [TestTerm](#) (WORD *term)
- int [DistrN](#) (int n, int *cpl, int ncpl, int *scratch)

Variables

- FILES ** [filelist](#)
- int [numinfilelist](#) = 0
- int [filelistsize](#) = 0
- WRITEFILE [WriteFile](#) = &WriteFileToFile

4.146.1 Detailed Description

Low level routines for many types of task. There are routines for manipulating the input system (streams and files) routines for string manipulation, the memory allocation interface, and the clock. The last is the most sensitive to ports. In the past nearly every port to another OS or computer gave trouble. Nowadays it is slightly better but the poor POSIX compliance of LINUX again gave problems for the multithreaded version.

Definition in file [tools.c](#).

4.146.2 Macro Definition Documentation

4.146.2.1 FILLVALUE

```
#define FILLVALUE 0
```

Definition at line 49 of file [tools.c](#).

4.146.2.2 TERMMEMSTARTNUM

```
#define TERMMEMSTARTNUM 16
```

Provides memory for one term (or one small polynomial) This means that the memory is limited to a buffer of size AM.MaxTer plus a few extra words. In parallel versions, each worker has its own memory pool.

The way we use the memory is by: term = TermMalloc(BHEAD0); and later we free it by TermFree(BHEAD term);

Layout: We have a list of available pointers to buffers: AT.TermMemHeap Its size is AT.TermMemMax We take from the top (indicated by AT.TermMemTop). When we run out of buffers we assign new ones (doubling the amount) and we have to extend the AT.TermMemHeap array. Important: There is no checking that the returned memory is legal, ie is memory that was handed out earlier.

Definition at line 2547 of file [tools.c](#).

4.146.2.3 TERMEXTRAWORDS

```
#define TERMEXTRAWORDS 10
```

Definition at line 2548 of file [tools.c](#).

4.146.2.4 NUMBERMEMSTARTNUM

```
#define NUMBERMEMSTARTNUM 16
```

Provides memory for one Long number This means that the memory is limited to a buffer of size AM.MaxTal In parallel versions, each worker has its own memory pool.

The way we use the memory is by: num = NumberMalloc(BHEAD0); Number = AT.NumberMemHeap[num]; and later we free it by NumberFree(BHEAD num);

Layout: We have a list of available pointers to buffers: AT.NumberMemHeap Its size is AT.NumberMemMax We take from the top (indicated by AT.NumberMemTop). When we run out of buffers we assign new ones (doubling the amount) and we have to extend the AT.NumberMemHeap array. Important: There is no checking on the returned memory!!!!

Definition at line 2647 of file [tools.c](#).

4.146.2.5 NUMBEREXTRAWORDS

```
#define NUMBEREXTRAWORDS 10L
```

Definition at line 2648 of file [tools.c](#).

4.146.2.6 DODOUBLE

```
#define DODOUBLE(  
    x )
```

Value:

```
{ x *s, *t, *u; if ( *start ) { \
    oldsize = *(x **)stop - *(x **)start; newsize = 2*oldsize; \
    t = u = (x *)Mallc1(newsize*sizeof(x),text); s = *(x **)start; \
    for ( i = 0; i < oldsize; i++ ) { *t++ = *s++; } M_free(*start,"double"); } \
    else { newsize = 100; u = (x *)Mallc1(newsize*sizeof(x),text); } \
    *start = (void *)u; *stop = (void *) (u+newsize); }
```

Definition at line 2970 of file [tools.c](#).

4.146.2.7 DOEXPAND

```
#define DOEXPAND(  
    x )
```

Value:

```
{ x *newbuffer, *t, *m; \
    t = newbuffer = (x *)Mallc1((newsize+2)*type,"ExpandBuffer"); \
    if ( *buffer ) { m = (x *)*buffer; i = *oldsize; \
        while ( --i >= 0 ) { *t++ = *m++; } M_free(*buffer,"ExpandBuffer"); \
    } *buffer = newbuffer; *oldsize = newsize; }
```

Definition at line 2996 of file [tools.c](#).

4.146.3 Function Documentation

4.146.3.1 LoadInputFile()

```
UBYTE * LoadInputFile (
    UBYTE * filename,
    int type )
```

Definition at line 112 of file [tools.c](#).

4.146.3.2 ReadFromStream()

```
UBYTE ReadFromStream (
    STREAM * stream )
```

Definition at line 151 of file [tools.c](#).

4.146.3.3 GetFromStream()

```
UBYTE GetFromStream (
    STREAM * stream )
```

Definition at line 286 of file [tools.c](#).

4.146.3.4 LookInStream()

```
UBYTE LookInStream (
    STREAM * stream )
```

Definition at line 309 of file [tools.c](#).

4.146.3.5 OpenStream()

```
STREAM * OpenStream (
    UBYTE * name,
    int type,
    int prevarmode,
    int raiselow )
```

Definition at line 321 of file [tools.c](#).

4.146.3.6 LocateFile()

```
int LocateFile (
    UBYTE ** name,
    int type )
```

Definition at line 576 of file [tools.c](#).

4.146.3.7 CloseStream()

```
STREAM * CloseStream (
    STREAM * stream )
```

Definition at line 663 of file [tools.c](#).

4.146.3.8 CreateStream()

```
STREAM * CreateStream (
    UBYTE * where )
```

Definition at line 782 of file [tools.c](#).

4.146.3.9 GetStreamPosition()

```
LONG GetStreamPosition (
    STREAM * stream )
```

Definition at line 813 of file [tools.c](#).

4.146.3.10 PositionStream()

```
void PositionStream (
    STREAM * stream,
    LONG position )
```

Definition at line 823 of file [tools.c](#).

4.146.3.11 ReverseStatements()

```
int ReverseStatements (
    STREAM * stream )
```

Definition at line 855 of file [tools.c](#).

4.146.3.12 StartFiles()

```
void StartFiles (
    void )
```

Definition at line 956 of file [tools.c](#).

4.146.3.13 OpenFile()

```
int OpenFile (
    char * name )
```

Definition at line 983 of file [tools.c](#).

4.146.3.14 OpenAddFile()

```
int OpenAddFile (
    char * name )
```

Definition at line 1002 of file [tools.c](#).

4.146.3.15 ReOpenFile()

```
int ReOpenFile (
    char * name )
```

Definition at line 1023 of file [tools.c](#).

4.146.3.16 CreateFile()

```
int CreateFile (
    char * name )
```

Definition at line 1043 of file [tools.c](#).

4.146.3.17 CreateLogFile()

```
int CreateLogFile (
    char * name )
```

Definition at line 1060 of file [tools.c](#).

4.146.3.18 CloseFile()

```
void CloseFile (
    int handle )
```

Definition at line 1078 of file [tools.c](#).

4.146.3.19 CopyFile()

```
int CopyFile (
    char * source,
    char * dest )
```

Copies a file with name *source to a file named *dest. The involved files must not be open. Returns non-zero if an error occurred. Uses if possible the combined large and small sorting buffers as cache.

Definition at line 1101 of file [tools.c](#).

Referenced by [DoRecovery\(\)](#).

4.146.3.20 CreateHandle()

```
int CreateHandle (
    void )
```

Definition at line 1162 of file [tools.c](#).

4.146.3.21 ReadFile()

```
LONG ReadFile (
    int handle,
    UBYTE * buffer,
    LONG size )
```

Definition at line 1217 of file [tools.c](#).

4.146.3.22 ReadPosFile()

```
LONG ReadPosFile (
    PHEAD FILEHANDLE * fi,
    UBYTE * buffer,
    LONG size,
    POSITION * pos )
```

Definition at line 1267 of file [tools.c](#).

4.146.3.23 WriteFileToFile()

```
LONG WriteFileToFile (
    int handle,
    UBYTE * buffer,
    LONG size )
```

Definition at line 1333 of file [tools.c](#).

4.146.3.24 SeekFile()

```
void SeekFile (
    int handle,
    POSITION * offset,
    int origin )
```

Definition at line 1371 of file [tools.c](#).

4.146.3.25 TellFile()

```
LONG TellFile (
    int handle )
```

Definition at line 1396 of file [tools.c](#).

4.146.3.26 TELLFILE()

```
void TELLFILE (
    int handle,
    POSITION * position )
```

Definition at line 1406 of file [tools.c](#).

4.146.3.27 FlushFile()

```
void FlushFile (
    int handle )
```

Definition at line 1420 of file [tools.c](#).

4.146.3.28 GetPosFile()

```
int GetPosFile (
    int handle,
    fpos_t * pospointer )
```

Definition at line 1434 of file [tools.c](#).

4.146.3.29 SetPosFile()

```
int SetPosFile (
    int handle,
    fpos_t * pospointer )
```

Definition at line 1448 of file [tools.c](#).

4.146.3.30 SynchFile()

```
void SynchFile (
    int handle )
```

Definition at line 1468 of file [tools.c](#).

4.146.3.31 TruncateFile()

```
void TruncateFile (
    int handle )
```

Definition at line 1490 of file [tools.c](#).

4.146.3.32 GetChannel()

```
int GetChannel (
    char * name,
    int mode )
```

Definition at line [1510](#) of file [tools.c](#).

4.146.3.33 GetAppendChannel()

```
int GetAppendChannel (
    char * name )
```

Definition at line [1546](#) of file [tools.c](#).

4.146.3.34 CloseChannel()

```
int CloseChannel (
    char * name )
```

Definition at line [1576](#) of file [tools.c](#).

4.146.3.35 UpdateMaxSize()

```
void UpdateMaxSize (
    void )
```

Definition at line [1610](#) of file [tools.c](#).

4.146.3.36 StrCmp()

```
int StrCmp (
    UBYTE * s1,
    UBYTE * s2 )
```

Definition at line [1700](#) of file [tools.c](#).

4.146.3.37 StrICmp()

```
int StrICmp (
    UBYTE * s1,
    UBYTE * s2 )
```

Definition at line [1711](#) of file [tools.c](#).

4.146.3.38 StrHICmp()

```
int StrHICmp (
    UBYTE * s1,
    UBYTE * s2 )
```

Definition at line 1722 of file [tools.c](#).

4.146.3.39 StrICont()

```
int StrICont (
    UBYTE * s1,
    UBYTE * s2 )
```

Definition at line 1733 of file [tools.c](#).

4.146.3.40 CmpArray()

```
int CmpArray (
    WORD * t1,
    WORD * t2,
    WORD n )
```

Definition at line 1745 of file [tools.c](#).

4.146.3.41 ConWord()

```
int ConWord (
    UBYTE * s1,
    UBYTE * s2 )
```

Definition at line 1759 of file [tools.c](#).

4.146.3.42 StrLen()

```
int StrLen (
    UBYTE * s )
```

Definition at line 1771 of file [tools.c](#).

4.146.3.43 NumToStr()

```
void NumToStr (
    UBYTE * s,
    LONG x )
```

Definition at line 1783 of file [tools.c](#).

4.146.3.44 WriteString()

```
void WriteString (
    int type,
    UBYTE * str,
    int num )
```

Definition at line [1807](#) of file [tools.c](#).

4.146.3.45 WriteUnfinString()

```
void WriteUnfinString (
    int type,
    UBYTE * str,
    int num )
```

Definition at line [1841](#) of file [tools.c](#).

4.146.3.46 AddToString()

```
UBYTE * AddToString (
    UBYTE * outstring,
    UBYTE * extrastring,
    int par )
```

Definition at line [1866](#) of file [tools.c](#).

4.146.3.47 strDup1()

```
UBYTE * strDup1 (
    UBYTE * instring,
    char * ifwrong )
```

Definition at line [1904](#) of file [tools.c](#).

4.146.3.48 EndOfToken()

```
UBYTE * EndOfToken (
    UBYTE * s )
```

Definition at line [1919](#) of file [tools.c](#).

4.146.3.49 ToToken()

```
UBYTE * ToToken (
    UBYTE * s )
```

Definition at line [1931](#) of file [tools.c](#).

4.146.3.50 SkipField()

```
UBYTE * SkipField (
    UBYTE * s,
    int level )
```

Definition at line 1946 of file [tools.c](#).

4.146.3.51 ReadSnum()

```
WORD ReadSnum (
    UBYTE ** p )
```

Definition at line 1973 of file [tools.c](#).

4.146.3.52 NumCopy()

```
UBYTE * NumCopy (
    WORD y,
    UBYTE * to )
```

Definition at line 1997 of file [tools.c](#).

4.146.3.53 LongCopy()

```
char * LongCopy (
    LONG y,
    char * to )
```

Definition at line 2024 of file [tools.c](#).

4.146.3.54 LongLongCopy()

```
char * LongLongCopy (
    off_t * y,
    char * to )
```

Definition at line 2051 of file [tools.c](#).

4.146.3.55 MakeDate()

```
UBYTE * MakeDate (
    void )
```

Definition at line 2090 of file [tools.c](#).

4.146.3.56 set_in()

```
int set_in (
    UBYTE ch,
    set_of_char set )
```

Definition at line 2112 of file [tools.c](#).

4.146.3.57 set_set()

```
one_byte set_set (
    UBYTE ch,
    set_of_char set )
```

Definition at line 2132 of file [tools.c](#).

4.146.3.58 set_del()

```
one_byte set_del (
    UBYTE ch,
    set_of_char set )
```

Definition at line 2153 of file [tools.c](#).

4.146.3.59 set_sub()

```
one_byte set_sub (
    set_of_char set,
    set_of_char set1,
    set_of_char set2 )
```

Definition at line 2175 of file [tools.c](#).

4.146.3.60 iniTools()

```
void iniTools (
    void )
```

Definition at line 2201 of file [tools.c](#).

4.146.3.61 Malloc()

```
void * Malloc (
    LONG size )
```

Definition at line 2221 of file [tools.c](#).

4.146.3.62 Malloc1()

```
void * Malloc1 (
    LONG size,
    const char * messageifwrong )
```

Definition at line 2303 of file [tools.c](#).

4.146.3.63 M_free()

```
void M_free (
    void * x,
    const char * where )
```

Definition at line 2383 of file [tools.c](#).

4.146.3.64 M_check1()

```
void M_check1 (
    void )
```

Definition at line 2516 of file [tools.c](#).

4.146.3.65 M_print()

```
void M_print (
    void )
```

Definition at line 2517 of file [tools.c](#).

4.146.3.66 TermMallocAddMemory()

```
void TermMallocAddMemory (
    PHEAD0 )
```

Definition at line 2550 of file [tools.c](#).

4.146.3.67 NumberMallocAddMemory()

```
void NumberMallocAddMemory (
    PHEAD0 )
```

Definition at line 2654 of file [tools.c](#).

4.146.3.68 CacheNumberMallocAddMemory()

```
void CacheNumberMallocAddMemory (
    PHEAD0 )
```

Definition at line 2728 of file [tools.c](#).

4.146.3.69 FromList()

```
void * FromList (
    LIST * L )
```

Definition at line 2780 of file [tools.c](#).

4.146.3.70 From0List()

```
void * From0List (
    LIST * L )
```

Definition at line 2806 of file [tools.c](#).

4.146.3.71 FromVarList()

```
void * FromVarList (
    LIST * L )
```

Definition at line 2835 of file [tools.c](#).

4.146.3.72 DoubleList()

```
int DoubleList (
    void *** lijst,
    int * oldsize,
    int objectsize,
    char * nameoftype )
```

Definition at line 2877 of file [tools.c](#).

4.146.3.73 DoubleLList()

```
int DoubleLList (
    void *** lijst,
    LONG * oldsize,
    int objectsize,
    char * nameoftype )
```

Definition at line 2929 of file [tools.c](#).

4.146.3.74 DoubleBuffer()

```
void DoubleBuffer (
    void ** start,
    void ** stop,
    int size,
    char * text )
```

Definition at line 2977 of file [tools.c](#).

4.146.3.75 ExpandBuffer()

```
void ExpandBuffer (
    void ** buffer,
    LONG * oldsize,
    int type )
```

Definition at line [3002](#) of file [tools.c](#).

4.146.3.76 iexp()

```
LONG iexp (
    LONG x,
    int p )
```

Definition at line [3026](#) of file [tools.c](#).

4.146.3.77 ToGeneral()

```
void ToGeneral (
    WORD * r,
    WORD * m,
    WORD par )
```

Definition at line [3057](#) of file [tools.c](#).

4.146.3.78 ToFast()

```
int ToFast (
    WORD * r,
    WORD * m )
```

Definition at line [3101](#) of file [tools.c](#).

4.146.3.79 ToPolyFunGeneral()

```
WORD ToPolyFunGeneral (
    PHEAD WORD * term )
```

Definition at line [3154](#) of file [tools.c](#).

4.146.3.80 IsLikeVector()

```
int IsLikeVector (
    WORD * arg )
```

Definition at line [3229](#) of file [tools.c](#).

4.146.3.81 AreArgsEqual()

```
int AreArgsEqual (
    WORD * arg1,
    WORD * arg2 )
```

Definition at line [3257](#) of file [tools.c](#).

4.146.3.82 CompareArgs()

```
int CompareArgs (
    WORD * arg1,
    WORD * arg2 )
```

Definition at line [3276](#) of file [tools.c](#).

4.146.3.83 CompArg()

```
int CompArg (
    WORD * s1,
    WORD * s2 )
```

Definition at line [3307](#) of file [tools.c](#).

4.146.3.84 TimeWallClock()

```
LONG TimeWallClock (
    WORD par )
```

Returns the wall-clock time.

Parameters

<i>par</i>	If zero, the wall-clock time will be reset to 0.
------------	--

Returns

The wall-clock time in centiseconds.

Definition at line [3477](#) of file [tools.c](#).

Referenced by [DoCheckpoint\(\)](#), [DoRecovery\(\)](#), [find_Horner_MCTS\(\)](#), [optimize_expression_given_Horner\(\)](#), [optimize_greedy\(\)](#), and [WriteStats\(\)](#).

4.146.3.85 TimeChildren()

```
LONG TimeChildren (
    WORD par )
```

Definition at line [3533](#) of file [tools.c](#).

4.146.3.86 TimeCPU()

```
LONG TimeCPU (
    WORD par )
```

Returns the CPU time.

Parameters

<i>par</i>	If zero, the CPU time will be reset to 0.
------------	---

Returns

The CPU time in milliseconds.

Definition at line 3551 of file [tools.c](#).

Referenced by [PF_GetSlaveTimes\(\)](#), [PF_Processor\(\)](#), and [WriteStats\(\)](#).

4.146.3.87 Crash()

```
int Crash (
    void )
```

Definition at line 3834 of file [tools.c](#).

4.146.3.88 TestTerm()

```
int TestTerm (
    WORD * term )
```

Tests the consistency of the term. Returns 0 when the term is OK. Any nonzero value is trouble. In the current version the testing isn't 100% complete. For instance, we don't check the validity of the symbols nor do we check the range of their powers. Etc. This should be extended when the need is there.

Parameters

<i>term</i>	the term to be tested
-------------	-----------------------

Definition at line 3862 of file [tools.c](#).

References [TestTerm\(\)](#).

Referenced by [TestTerm\(\)](#).

Here is the call graph for this function:



4.146.3.89 DistrN()

```
int DistrN (
    int n,
    int * cpl,
    int ncpl,
    int * scratch )
```

Definition at line 4048 of file [tools.c](#).

4.146.4 Variable Documentation

4.146.4.1 filelist

```
FILES** filelist
```

Definition at line 65 of file [tools.c](#).

4.146.4.2 numinfilelist

```
int numinfilelist = 0
```

Definition at line 66 of file [tools.c](#).

4.146.4.3 filelistsize

```
int filelistsize = 0
```

Definition at line 67 of file [tools.c](#).

4.146.4.4 WriteFile

```
WRITEFILE WriteFile = &WriteFileToFile
```

Definition at line 1357 of file [tools.c](#).

4.147 tools.c

[Go to the documentation of this file.](#)

```

00001
00011 /* #[ License : */
00012 /*
00013 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00014 *   When using this file you are requested to refer to the publication
00015 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00016 *   This is considered a matter of courtesy as the development was paid
00017 *   for by FOM the Dutch physics granting agency and we would like to
00018 *   be able to track its scientific use to convince FOM of its value
00019 *   for the community.
00020 *
00021 *   This file is part of FORM.
00022 *
00023 *   FORM is free software: you can redistribute it and/or modify it under the
00024 *   terms of the GNU General Public License as published by the Free Software
00025 *   Foundation, either version 3 of the License, or (at your option) any later
00026 *   version.
00027 *
00028 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00029 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00030 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00031 *   details.
00032 *
00033 *   You should have received a copy of the GNU General Public License along
00034 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00035 */
00036 /* #] License : */
00037 /*
00038 *   #[ Includes :
00039 *   Note: TERMMALLOCDEBUG tests part of the TermMalloc and NumberMalloc
00040 *   system. To work properly it needs MEMORYMACROS in declare.h
00041 *   not to be defined to make sure that all calls will be diverted
00042 *   to the routines here.
00043 #define TERMMALLOCDEBUG
00044 #define FILLVALUE 126
00045 #define MALLOCDEBUGOUTPUT
00046 #define MALLOCDEBUG 1
00047 */
00048 #ifndef FILLVALUE
00049     #define FILLVALUE 0
00050 #endif
00051
00052 /*
00053 *   The enhanced malloc debugger, see comments in the beginning of the
00054 *   file mallocprotect.h
00055 *   MALLOCPROTECT == -1 -- protect left side, used block is left-aligned.
00056 *   MALLOCPROTECT == 0 -- protect both sides, used block is left-aligned;
00057 *   MALLOCPROTECT == 1 -- protect both sides, used block is right-aligned;
00058 *   ATTENTION! The macro MALLOCPROTECT must be defined
00059 *   BEFORE #include mallocprotect.h
00060 #define MALLOCPROTECT 1
00061 */
00062
00063 #include "form3.h"
00064
00065 FILES **filelist;
00066 int numinfilelist = 0;
00067 int filelistsize = 0;
00068 #ifdef MALLOCDEBUG
00069 #define BANNER (4*sizeof(LONG))
00070 void *malloclist[60000];
00071 LONG mallocsizes[60000];
00072 char *mallocstrings[60000];
00073 int nummalloclist = 0;
00074 #endif
00075
00076 #ifdef GPP
00077 extern "C" getdtablesize();
00078 #endif
00079
00080 #ifdef WITHSTATS
00081 LONG numwrites = 0;
00082 LONG numreads = 0;
00083 LONG numseeks = 0;
00084 LONG nummallocs = 0;
00085 LONG numfrees = 0;
00086 #endif
00087
00088 #ifdef MALLOCPROTECT
00089 #ifdef TRAP_SIGNALS
00090 #error "MALLOCPROTECT":  undefine "TRAP_SIGNALS" in unix.h first!
00091 #endif

```

```

00092 #include "mallocprotect.h"
00093
00094 #ifdef M_alloc
00095 #undef M_alloc
00096 #endif
00097
00098 #define M_alloc mprotectMalloc
00099
00100 #endif
00101
00102 #ifdef TERMMALLOCDEBUG
00103 WORD **DebugHeap1, **DebugHeap2;
00104 #endif
00105
00106 /*
00107     #] Includes :
00108     #[ Streams :
00109         #[ LoadInputFile :
00110 */
00111
00112 UBYTE *LoadInputFile(UBYTE *filename, int type)
00113 {
00114     int handle;
00115     LONG filesize;
00116     UBYTE *buffer, *name = filename;
00117     POSITION scrpos;
00118     handle = LocateFile(&name,type);
00119     if ( handle < 0 ) return(0);
00120     PUTZERO(scrpos);
00121     SeekFile(handle,&scrpos,SEEK_END);
00122     TELLFILE(handle,&scrpos);
00123     filesize = BASEPOSITION(scrpos);
00124     PUTZERO(scrpos);
00125     SeekFile(handle,&scrpos,SEEK_SET);
00126     buffer = (UBYTE *)Malloca(filesize+2,"LoadInputFile");
00127     if ( ReadFile(handle,buffer,filesize) != filesize ) {
00128         Error1("Read error for file ",name);
00129         M_free(buffer,"LoadInputFile");
00130         if ( name != filename ) M_free(name,"FromLoadInputFile");
00131         CloseFile(handle);
00132         return(0);
00133     }
00134     CloseFile(handle);
00135     if ( type == PROCEDUREFILE || type == SETUPFILE ) {
00136         buffer[filesize] = '\n';
00137         buffer[filesize+1] = 0;
00138     }
00139     else {
00140         buffer[filesize] = 0;
00141     }
00142     if ( name != filename ) M_free(name,"FromLoadInputFile");
00143     return(buffer);
00144 }
00145
00146 /*
00147     #] LoadInputFile :
00148     #[ ReadFromStream :
00149 */
00150
00151 UBYTE ReadFromStream(STREAM *stream)
00152 {
00153     UBYTE c;
00154     POSITION scrpos;
00155     #ifdef WITHPIPE
00156     if ( stream->type == PIPESTREAM ) {
00157     #ifndef WITHMPI
00158         FILE *f;
00159         int cc;
00160         RWLOCKR(AM.handlelock);
00161         f = (FILE *) (filelist[stream->handle]);
00162         UNRWLOCK(AM.handlelock);
00163         cc = getc(f);
00164         if ( cc == EOF ) return(ENDOFSTREAM);
00165         c = (UBYTE)cc;
00166     #else
00167         if ( stream->pointer >= stream->top ) {
00168             /* The master reads the pipe and broadcasts it to the slaves. */
00169             LONG len;
00170             if ( PF.me == MASTER ) {
00171                 FILE *f;
00172                 UBYTE *p, *end;
00173                 RWLOCKR(AM.handlelock);
00174                 f = (FILE *) filelist[stream->handle];
00175                 UNRWLOCK(AM.handlelock);
00176                 p = stream->buffer;
00177                 end = stream->buffer + stream->buffersize;
00178                 while ( p < end ) {

```

```

00179         int cc = getc(f);
00180         if ( cc == EOF ) {
00181             break;
00182         }
00183         *p++ = (UBYTE)cc;
00184     }
00185     len = p - stream->buffer;
00186     PF_BroadcastNumber(len);
00187 }
00188 else {
00189     len = PF_BroadcastNumber(0);
00190 }
00191 if ( len > 0 ) {
00192     PF_Bcast(stream->buffer, len);
00193 }
00194 stream->pointer = stream->buffer;
00195 stream->inbuffer = len;
00196 stream->top = stream->buffer + stream->inbuffer;
00197 if ( stream->pointer == stream->top ) return ENDOFSTREAM;
00198 }
00199 c = (UBYTE)*stream->pointer++;
00200 #endif
00201 if ( stream->eqnum == 1 ) { stream->eqnum = 0; stream->linenumber++; }
00202 if ( c == LINEFEED ) stream->eqnum = 1;
00203 return(c);
00204 }
00205 #endif
00206 /*[14apr2004 mt]:*/
00207 #ifdef WITHEXTERNALCHANNEL
00208     if ( stream->type == EXTERNALCHANNELSTREAM ) {
00209         int cc;
00210         cc = getcFromExtChannel();
00211         /*[18may20006 mt]:*/
00212         /*if ( cc == EOF ) return(ENDOFSTREAM);*/
00213         if ( cc < 0 ) {
00214             if( cc == EOF )
00215                 return(ENDOFSTREAM);
00216             else{
00217                 Error0("No current external channel");
00218                 Terminate(-1);
00219             }
00220         }/*if ( cc < 0 )*/
00221         /*:[18may20006 mt]:*/
00222         c = (UBYTE)cc;
00223         if ( stream->eqnum == 1 ) { stream->eqnum = 0; stream->linenumber++; }
00224         if ( c == LINEFEED ) stream->eqnum = 1;
00225         return(c);
00226     }
00227 #endif /*ifdef WITHEXTERNALCHANNEL*/
00228 /*:[14apr2004 mt]:*/
00229     if ( stream->type == INPUTSTREAM ) {
00230         if ( stream->pointer < stream->top ) {
00231             c = *stream->pointer++;
00232         }
00233         else {
00234             if ( ReadFile(stream->handle,&c,1) != 1 ) {
00235                 return(ENDOFSTREAM);
00236             }
00237             if ( stream->fileposition == 0 ) {
00238                 if ( !stream->buffer ) {
00239                     stream->buffer = (UBYTE *)Malloc1(stream->buffer, "input stream buffer");
00240                     stream->pointer = stream->top = stream->buffer;
00241                 }
00242             }
00243             else {
00244                 if ( stream->top - stream->buffer >= stream->buffer ) {
00245                     LONG oldsize = stream->buffer;
00246                     DoubleBuffer((void**)&stream->buffer, (void**)&stream->top, sizeof(UBYTE), "double input stream buffer");
00247                     stream->buffer = stream->top - stream->buffer;
00248                     stream->pointer = stream->top = stream->buffer + oldsize;
00249                 }
00250             }
00251             *stream->pointer = c;
00252             stream->pointer = ++stream->top;
00253             stream->inbuffer++;
00254         }
00255     }
00256     if ( stream->eqnum == 1 ) { stream->eqnum = 0; stream->linenumber++; }
00257     if ( c == LINEFEED ) stream->eqnum = 1;
00258     return(c);
00259 }
00260 if ( stream->pointer >= stream->top ) {
00261     if ( stream->type != FILESTREAM ) return(ENDOFSTREAM);
00262     if ( stream->fileposition != stream->bufferposition+stream->inbuffer ) {
00263         stream->fileposition = stream->bufferposition+stream->inbuffer;
00264         SETBASEPOSITION(scrpos, stream->fileposition);

```

```

00265         SeekFile(stream->handle, &scrpos, SEEK_SET);
00266     }
00267     stream->bufferposition = stream->fileposition;
00268     stream->inbuffer = ReadFile(stream->handle,
00269         stream->buffer, stream->buffersize);
00270     if ( stream->inbuffer <= 0 ) return(EOFSTREAM);
00271     stream->top = stream->buffer + stream->inbuffer;
00272     stream->pointer = stream->buffer;
00273     stream->fileposition = stream->bufferposition + stream->inbuffer;
00274 }
00275 if ( stream->eqnum == 1 ) { stream->eqnum = 0; stream->linenumber++; }
00276 c = *(stream->pointer)++;
00277 if ( c == LINEFEED ) stream->eqnum = 1;
00278 return(c);
00279 }
00280
00281 /*
00282     #] ReadFromStream :
00283     #[ GetFromStream :
00284 */
00285
00286 UBYTE GetFromStream(STREAM *stream)
00287 {
00288     UBYTE c1, c2;
00289     if ( stream->isnextchar > 0 ) {
00290         return(stream->nextchar[--stream->isnextchar]);
00291     }
00292     c1 = ReadFromStream(stream);
00293     if ( c1 == LINEFEED || c1 == CARRIAGERETURN ) {
00294         c2 = ReadFromStream(stream);
00295         if ( c2 == c1 || ( c2 != LINEFEED && c2 != CARRIAGERETURN ) ) {
00296             stream->isnextchar = 1;
00297             stream->nextchar[0] = c2;
00298         }
00299         return(LINEFEED);
00300     }
00301     else return(c1);
00302 }
00303
00304 /*
00305     #] GetFromStream :
00306     #[ LookInStream :
00307 */
00308
00309 UBYTE LookInStream(STREAM *stream)
00310 {
00311     UBYTE c = GetFromStream(stream);
00312     UngetFromStream(stream, c);
00313     return(c);
00314 }
00315
00316 /*
00317     #] LookInStream :
00318     #[ OpenStream :
00319 */
00320
00321 STREAM *OpenStream(UBYTE *name, int type, int prevarmode, int raiselow)
00322 {
00323     STREAM *stream;
00324     UBYTE *rhsosvariable, *s, *newname, c;
00325     POSITION scrpos;
00326     int handle, num;
00327     LONG filesize;
00328     switch ( type ) {
00329         case REVERSEFILESTREAM:
00330         case FILESTREAM:
00331     }
00332     /*
00333         Notice that FILESTREAM is only used for text files:
00334         The #include files and the main input file (.frm)
00335         Hence we do not worry about files longer than 2 Gbytes.
00336     */
00337     newname = name;
00338     handle = LocateFile(&newname, -1);
00339     if ( handle < 0 ) return(0);
00340     PUTZERO(scrpos);
00341     SeekFile(handle, &scrpos, SEEK_END);
00342     TELLFILE(handle, &scrpos);
00343     filesize = BASEPOSITION(scrpos);
00344     PUTZERO(scrpos);
00345     SeekFile(handle, &scrpos, SEEK_SET);
00346     if ( filesize > AM.MaxStreamSize && type == FILESTREAM )
00347         filesize = AM.MaxStreamSize;
00348     stream = CreateStream((UBYTE *) "filestream");
00349     /*
00350         The extra +1 in the Malloc1 is potentially needed in ReverseStatements!
00351     */
00352     stream->buffer = (UBYTE *) Malloc1(filesize+1, "name of input stream");

```



```

00352     stream->inbuffer = ReadFile(handle,stream->buffer,filesize);
00353     if ( type == REVERSEFILESTREAM ) {
00354         if ( ReverseStatements(stream) ) {
00355             M_free(stream->buffer,"name of input stream");
00356             return(0);
00357         }
00358     }
00359     stream->top = stream->buffer + stream->inbuffer;
00360     stream->pointer = stream->buffer;
00361     stream->handle = handle;
00362     stream->bufferSize = fileSize;
00363     stream->fileposition = stream->inbuffer;
00364     if ( newname != name ) stream->name = newname;
00365     else if ( name ) stream->name = strdup(name,"name of input stream");
00366     else
00367         stream->name = 0;
00368     stream->prevline = stream->linenumber = 1;
00369     stream->eqnum = 0;
00370     break;
00371 case PREVARSTREAM:
00372     if ( ( rhsOfVariable = GetPreVar(name,WITHERROR) ) == 0 ) return(0);
00373     stream = CreateStream((UBYTE *)"var-stream");
00374     stream->buffer = stream->pointer = s = rhsOfVariable;
00375     while ( *s ) s++;
00376     stream->top = s;
00377     stream->inbuffer = s - stream->buffer;
00378     stream->name = AC.CurrentStream->name;
00379     stream->linenumber = AC.CurrentStream->linenumber;
00380     stream->prevline = AC.CurrentStream->prevline;
00381     stream->eqnum = AC.CurrentStream->eqnum;
00382     stream->pname = strdup(name,"stream->pname");
00383     stream->olddelay = AP.AllowDelay;
00384     s = stream->pname; while ( *s ) s++;
00385     while ( s[-1] == '+' || s[-1] == '-' ) s--;
00386     *s = 0;
00387     UnsetAllowDelay();
00388     break;
00389 case DOLLARSTREAM:
00390     if ( ( num = GetDollar(name) ) < 0 ) {
00391         WORD numfac = 0;
00392         /*
00393          Here we have to test first whether we have $x[1], $x[0]
00394          or just an undefined $x.
00395          */
00396         s = name; while ( *s && *s != '[' ) s++;
00397         if ( *s == 0 ) return(0);
00398         c = *s; *s = 0;
00399         if ( ( num = GetDollar(name) ) < 0 ) return(0);
00400         *s = c;
00401         s++;
00402         if ( *s == 0 || FG.cTable[*s] != 1 || *s == ']' ) {
00403             MesPrint("@Illegal factor number for dollar variable");
00404             return(0);
00405         }
00406         while ( *s && FG.cTable[*s] == 1 ) {
00407             numfac = 10*numfac+*s---'0';
00408         }
00409         if ( *s != '[' || s[1] != 0 ) {
00410             MesPrint("@Illegal factor number for $ variable");
00411             return(0);
00412         }
00413         stream = CreateStream((UBYTE *)"dollar-stream");
00414         stream->buffer = stream->pointer = s = WriteDollarFactorToBuffer(num,numfac,1);
00415     }
00416     else {
00417         stream = CreateStream((UBYTE *)"dollar-stream");
00418         stream->buffer = stream->pointer = s = WriteDollarToBuffer(num,1);
00419     }
00420     while ( *s ) s++;
00421     stream->top = s;
00422     stream->inbuffer = s - stream->buffer;
00423     stream->name = AC.CurrentStream->name;
00424     stream->linenumber = AC.CurrentStream->linenumber;
00425     stream->prevline = AC.CurrentStream->prevline;
00426     stream->eqnum = AC.CurrentStream->eqnum;
00427     stream->pname = strdup(name,"stream->pname");
00428     s = stream->pname; while ( *s ) s++;
00429     while ( s[-1] == '+' || s[-1] == '-' ) s--;
00430     *s = 0;
00431     /* We 'stole' the buffer. Later we can free it. */
00432     AO.DollarOutSizeBuffer = 0;
00433     AO.DollarOutBuffer = 0;
00434     AO.DollarInOutBuffer = 0;
00435     break;
00436 case PREREADSTREAM:
00437 case PREREADSTREAM2:
00438 case PREREADSTREAM3:

```

```

00439         case PRECALCSTREAM:
00440             stream = CreateStream((UBYTE *)"calculator");
00441             stream->buffer = stream->pointer = s = name;
00442             while ( *s ) s++;
00443             stream->top = s;
00444             stream->inbuffer = s - stream->buffer;
00445             stream->name = AC.CurrentStream->name;
00446             stream->linenumber = AC.CurrentStream->linenumber;
00447             stream->prevline = AC.CurrentStream->prevline;
00448             stream->eqnum = 0;
00449             break;
00450 #ifndef WITHPIPE
00451         case PIPESTREAM:
00452             stream = CreateStream((UBYTE *)"pipe");
00453 #ifndef WITHMPI
00454             {
00455                 FILE *f;
00456                 if ( ( f = popen((char *)name,"r") ) == 0 ) {
00457                     Error0("@Cannot create pipe");
00458                 }
00459                 stream->handle = CreateHandle();
00460                 RWLOCKW(AM.handlelock);
00461                 filelist[stream->handle] = (FILES *)f;
00462                 UNRWLOCK(AM.handlelock);
00463             }
00464             stream->buffer = stream->top = 0;
00465             stream->inbuffer = 0;
00466 #else
00467             {
00468                 /* Only the master opens the pipe. */
00469                 FILE *f;
00470                 if ( PF.me == MASTER ) {
00471                     f = popen((char *)name, "r");
00472                     PF_BroadcastNumber(f == 0);
00473                     if ( f == 0 ) Error0("@Cannot create pipe");
00474                 }
00475                 else {
00476                     if ( PF_BroadcastNumber(0) ) Error0("@Cannot create pipe");
00477                     f = (FILE *)123; /* dummy */
00478                 }
00479                 stream->handle = CreateHandle();
00480                 RWLOCKW(AM.handlelock);
00481                 filelist[stream->handle] = (FILES *)f;
00482                 UNRWLOCK(AM.handlelock);
00483             }
00484             /* stream->buffer as a send/receive buffer. */
00485             stream->buffer = (UBYTE *)Malloca1(stream->buffer, "pipe buffer");
00486             stream->inbuffer = 0;
00487             stream->top = stream->buffer;
00488             stream->pointer = stream->buffer;
00489 #endif
00490             stream->name = strDup1((UBYTE *)"pipe","pipe");
00491             stream->prevline = stream->linenumber = 1;
00492             stream->eqnum = 0;
00493             break;
00494 #endif
00495 #endif
00496 /*[14apr2004 mt]:*/
00497 #ifndef WITHEXTERNALCHANNEL
00498         case EXTERNALCHANNELSTREAM:
00499             /*Block*/
00500             int n, *tmpn;
00501             if( (n=getCurrentExternalChannel()) == 0 )
00502                 Error0("@No current external channel");
00503             stream = CreateStream((UBYTE *)"externalchannel");
00504             stream->handle = CreateHandle();
00505             tmpn = (int *)Malloca1(sizeof(int),"external channel handle");
00506             *tmpn = n;
00507             RWLOCKW(AM.handlelock);
00508             filelist[stream->handle] = (FILES *)tmpn;
00509             UNRWLOCK(AM.handlelock);
00510             /*Block*/
00511             stream->buffer = stream->top = 0;
00512             stream->inbuffer = 0;
00513             stream->name = strDup1((UBYTE *)"externalchannel","externalchannel");
00514             stream->prevline = stream->linenumber = 1;
00515             stream->eqnum = 0;
00516             break;
00517 #endif /*ifdef WITHEXTERNALCHANNEL*/
00518 /*:[14apr2004 mt]:*/
00519         case INPUTSTREAM:
00520             /*
00521              * Assume that "name" stores a file descriptor (UNIX) or a FILE
00522              * pointer (Windows). We don't close it automatically on closing
00523              * the INPUTSTREAM stream (e.g., for stdin).
00524              */
00525             stream = CreateStream((UBYTE *)"input stream");

```

```

00526         stream->handle = CreateHandle();
00527     {
00528         FILES *f = (FILES *)Malloca(sizeof(int),"input stream handle");
00529         /* NOTE: in both cases name=0 indicates stdin. */
00530 #ifdef UNIX
00531         f->descriptor = (int)(ssize_t)name;
00532 #else
00533         f = name ? (FILES *)name : stdin;
00534 #endif
00535         RWLOCKW(AM.handlelock);
00536         filelist[stream->handle] = f;
00537         UNRWLOCK(AM.handlelock);
00538     }
00539     stream->buffer = stream->pointer = stream->top = 0;
00540     stream->inbuffer = 0;
00541     stream->name = strdup((UBYTE *) (name ? "INPUT" : "STDIN"),"input stream name");
00542     stream->prevline = stream->linenumber = 1;
00543     stream->eqnum = 0;
00544     /*
00545      * fileposition == -1: default
00546      * fileposition == 0: cache the input
00547      * See also: ReadFromStream, TryFileSetups
00548      */
00549     stream->fileposition = -1;
00550     break;
00551 default:
00552     return(0);
00553 }
00554 stream->bufferposition = 0;
00555 stream->isnextchar = 0;
00556 stream->type = type;
00557 stream->previousNoShowInput = AC.NoShowInput;
00558 stream->afterwards = raiselow;
00559 if ( AC.CurrentStream ) stream->previous = AC.CurrentStream - AC.Streams;
00560 else stream->previous = -1;
00561 stream->FoldName = 0;
00562 if ( prevarmode == 0 ) stream->prevvars = -1;
00563 else if ( prevarmode > 0 ) stream->prevvars = NumPre;
00564 else if ( prevarmode < 0 ) stream->prevvars = -prevarmode-1;
00565 AC.CurrentStream = stream;
00566 if ( type == PREREADSTREAM || type == PREREADSTREAM3 || type == PRECALCSTREAM
00567     || type == DOLLARSTREAM ) AC.NoShowInput = 1;
00568 return(stream);
00569 }
00570
00571 /*
00572     #] OpenStream :
00573     #[ LocateFile :
00574 */
00575
00576 int LocateFile(UBYTE **name, int type)
00577 {
00578     int handle, namesize, i;
00579     UBYTE *s, *to, *u1, *u2, *newname, *indir;
00580     handle = OpenFile((char *) (*name));
00581     if ( handle >= 0 ) return(handle);
00582     if ( type == SETUPFILE && AM.SetupFile ) {
00583         handle = OpenFile((char *) (AM.SetupFile));
00584         if ( handle >= 0 ) return(handle);
00585         MesPrint("Could not open setup file %s", (char *) (AM.SetupFile));
00586     }
00587     namesize = 4; s = *name;
00588     while ( *s ) { s++; namesize++; }
00589     if ( type == SETUPFILE ) indir = AM.SetupDir;
00590     else indir = AM.IncDir;
00591     if ( indir ) {
00592
00593         s = indir; i = 0;
00594         while ( *s ) { s++; i++; }
00595         newname = (UBYTE *)Malloca(namesize+i,"LocateFile");
00596         s = indir; to = newname;
00597         while ( *s ) *to++ = *s++;
00598         if ( to > newname && to[-1] != SEPARATOR ) *to++ = SEPARATOR;
00599         s = *name;
00600         while ( *s ) *to++ = *s++;
00601         *to = 0;
00602         handle = OpenFile((char *) newname);
00603         if ( handle >= 0 ) {
00604             *name = newname;
00605             return(handle);
00606         }
00607         M_free(newname,"LocateFile, incdir/file");
00608     }
00609     if ( type == SETUPFILE ) {
00610         handle = OpenFile(setupfilename);
00611         if ( handle >= 0 ) return(handle);
00612         s = (UBYTE *)getenv("FORMSETUP");

```

```

00613         if ( s ) {
00614             handle = OpenFile((char *)s);
00615             if ( handle >= 0 ) return(handle);
00616             MesPrint("Could not open setup file %s",s);
00617         }
00618     }
00619     if ( type != SETUPFILE && AM.Path ) {
00620         ul = AM.Path;
00621         while ( *ul ) {
00622             u2 = ul; i = 0;
00623 #ifndef WINDOWS
00624             while ( *ul && *ul != ';' ) {
00625                 ul++; i++;
00626             }
00627 #else
00628             while ( *ul && *ul != ':' ) {
00629                 if ( *ul == '\\\' ) ul++;
00630                 ul++; i++;
00631             }
00632 #endif
00633             newname = (UBYTE *)Malloc1(namesize+i,"LocateFile");
00634             s = u2; to = newname;
00635             while ( s < ul ) {
00636 #ifndef WINDOWS
00637                 if ( *s == '\\\' ) s++;
00638 #endif
00639                 *to++ = *s++;
00640             }
00641             if ( to > newname && to[-1] != SEPARATOR ) *to++ = SEPARATOR;
00642             s = *name;
00643             while ( *s ) *to++ = *s++;
00644             *to = 0;
00645             handle = OpenFile((char *)newname);
00646             if ( handle >= 0 ) {
00647                 *name = newname;
00648                 return(handle);
00649             }
00650             M_free(newname,"LocateFile Path/file");
00651             if ( *ul ) ul++;
00652         }
00653     }
00654     if ( type != SETUPFILE && type >= -1 ) Error1("LocateFile: Cannot find file",*name);
00655     return(-1);
00656 }
00657
00658 /*
00659     #] LocateFile :
00660     #[ CloseStream :
00661 */
00662
00663 STREAM *CloseStream(STREAM *stream)
00664 {
00665     int newstr = stream->previous, sgn;
00666     UBYTE *t, numbuf[24];
00667     LONG x;
00668     if ( stream->FoldName ) {
00669         M_free(stream->FoldName,"stream->FoldName");
00670         stream->FoldName = 0;
00671     }
00672     if ( stream->type == FILESTREAM || stream->type == REVERSEFILESTREAM ) {
00673         CloseFile(stream->handle);
00674         if ( stream->buffer != 0 ) M_free(stream->buffer,"name of input stream");
00675         stream->buffer = 0;
00676     }
00677 #ifndef WITHPIPE
00678     else if ( stream->type == PIPESTREAM ) {
00679         RWLOCKW(AM.handlelock);
00680 #endif
00681 #ifndef WITHMPI
00682         if ( PF.me == MASTER )
00683 #endif
00684         pclose((FILE *) (filelist[stream->handle]));
00685         filelist[stream->handle] = 0;
00686         numinfilelist--;
00687         UNRWLOCK(AM.handlelock);
00688 #ifndef WITHMPI
00689         if ( stream->buffer != 0 ) {
00690             M_free(stream->buffer,"pipe buffer");
00691             stream->buffer = 0;
00692         }
00693 #endif
00694 #endif
00695 /*[14apr2004 mt]:*/
00696 #ifndef WITHEXTERNALCHANNEL
00697         else if ( stream->type == EXTERNALCHANNELSTREAM ) {
00698             int *tmpn;
00699             RWLOCKW(AM.handlelock);

```

```

00700         tmpn = (int *) (filelist[stream->handle]);
00701         filelist[stream->handle] = 0;
00702         numinfilelist--;
00703         UNRWLOCK(AM.handlelock);
00704         M_free(tmpn, "external channel handle");
00705     }
00706 #endif /*ifdef WITHEXTERNALCHANNEL*/
00707 /*:[14apr2004 mt]*/
00708     else if ( stream->type == INPUTSTREAM ) {
00709         FILES *f;
00710         RWLOCKW(AM.handlelock);
00711         f = filelist[stream->handle];
00712         filelist[stream->handle] = 0;
00713         numinfilelist--;
00714         UNRWLOCK(AM.handlelock);
00715         M_free(f, "input stream handle");
00716     }
00717     else if ( stream->type == PREVARSTREAM && (
00718 stream->afterwards == PRERAISEAFTER || stream->afterwards == PRELOWERAFTER ) ) {
00719         t = stream->buffer; x = 0; sgn = 1;
00720         while ( *t == '-' || *t == '+' ) {
00721             if ( *t == '-' ) sgn = -sgn;
00722             t++;
00723         }
00724         if ( FG.cTable[*t] == 1 ) {
00725             while ( *t && FG.cTable[*t] == 1 ) x = 10*x + *t++ - '0';
00726             if ( *t == 0 ) {
00727                 if ( stream->afterwards == PRERAISEAFTER ) x = sgn*x + 1;
00728                 else x = sgn*x - 1;
00729                 NumToStr(numbuf, x);
00730                 PutPreVar(stream->pname, numbuf, 0, 1);
00731             }
00732         }
00733     }
00734     else if ( stream->type == DOLLARSTREAM && (
00735 stream->afterwards == PRERAISEAFTER || stream->afterwards == PRELOWERAFTER ) ) {
00736         if ( stream->afterwards == PRERAISEAFTER ) x = 1;
00737         else x = -1;
00738         DollarRaiseLow(stream->pname, x);
00739     }
00740     else if ( stream->type == PRECALCSTREAM || stream->type == DOLLARSTREAM ) {
00741         if ( stream->buffer ) M_free(stream->buffer, "stream->buffer");
00742         stream->buffer = 0;
00743     }
00744     if ( stream->name && stream->type != PREVARSTREAM
00745 && stream->type != PREREADSTREAM && stream->type != PREREADSTREAM2 && stream->type !=
PREREADSTREAM3
00746 && stream->type != PRECALCSTREAM && stream->type != DOLLARSTREAM ) {
00747         M_free(stream->name, "stream->name");
00748     }
00749     stream->name = 0;
00750 /* if ( stream->type != FILESTREAM ) */
00751 AC.NoShowInput = stream->previousNoShowInput;
00752 stream->buffer = 0; /* To make sure we will not reuse it */
00753 stream->pointer = 0;
00754 /*
00755 Look whether we have to pop preprocessor variables.
00756 */
00757     if ( stream->prevars >= 0 ) {
00758         while ( NumPre > stream->prevars ) {
00759             NumPre--;
00760             M_free(PreVar[NumPre].name, "PreVar[NumPre].name");
00761             PreVar[NumPre].name = PreVar[NumPre].value = 0;
00762         }
00763     }
00764     if ( stream->type == PREVARSTREAM ) {
00765         AP.AllowDelay = stream->olddelay;
00766         ClearMacro(stream->pname);
00767         M_free(stream->pname, "stream->pname");
00768     }
00769     else if ( stream->type == DOLLARSTREAM ) {
00770         M_free(stream->pname, "stream->pname");
00771     }
00772     AC.NumStreams--;
00773     if ( newstr >= 0 ) return(AC.Streams + newstr);
00774     else return(0);
00775 }
00776 /*
00777 #] CloseStream :
00778 #[ CreateStream :
00779 */
00780
00781
00782 STREAM *CreateStream(UBYTE *where)
00783 {
00784     STREAM *newstreams;
00785     int numnewstreams, i;

```

```

00786     int offset;
00787     if ( AC.NumStreams >= AC.MaxNumStreams ) {
00788         if ( AC.MaxNumStreams == 0 ) numnewstreams = 10;
00789         else numnewstreams = 2*AC.MaxNumStreams;
00790         newstreams = (STREAM *)Malloc1(sizeof(STREAM)*(numnewstreams+1),"CreateStream");
00791         if ( AC.MaxNumStreams > 0 ) {
00792             offset = AC.CurrentStream - AC.Streams;
00793             for ( i = 0; i < AC.MaxNumStreams; i++ ) {
00794                 newstreams[i] = AC.Streams[i];
00795             }
00796             AC.CurrentStream = newstreams + offset;
00797         }
00798         else newstreams[0].previous = -1;
00799         AC.MaxNumStreams = numnewstreams;
00800         if ( AC.Streams ) M_free(AC.Streams, (char *)where);
00801         AC.Streams = newstreams;
00802     }
00803     newstreams = AC.Streams+AC.NumStreams++;
00804     newstreams->name = 0;
00805     return(newstreams);
00806 }
00807
00808 /*
00809     #] CreateStream :
00810     #[ GetStreamPosition :
00811 */
00812
00813 LONG GetStreamPosition(STREAM *stream)
00814 {
00815     return(stream->bufferposition + ((LONG)stream->pointer-(LONG)stream->buffer));
00816 }
00817
00818 /*
00819     #] GetStreamPosition :
00820     #[ PositionStream :
00821 */
00822
00823 void PositionStream(STREAM *stream, LONG position)
00824 {
00825     POSITION scrpos;
00826     if ( position >= stream->bufferposition
00827         && position < stream->bufferposition + stream->inbuffer ) {
00828         stream->pointer = stream->buffer + (position-stream->bufferposition);
00829     }
00830     else if ( stream->type == FILESTREAM ) {
00831         SETBASEPOSITION(scrpos,position);
00832         SeekFile(stream->handle,&scrpos,SEEK_SET);
00833         stream->inbuffer = ReadFile(stream->handle,stream->buffer,stream->buffersize);
00834         stream->pointer = stream->buffer;
00835         stream->top = stream->buffer + stream->inbuffer;
00836         stream->bufferposition = position;
00837         stream->fileposition = position + stream->inbuffer;
00838         stream->isnextchar = 0;
00839     }
00840     else {
00841         Error0("Illegal position for stream");
00842         Terminate(-1);
00843     }
00844 }
00845
00846 /*
00847     #] PositionStream :
00848     #[ ReverseStatements :
00849
00850     Reverses the order of the statements in the buffer.
00851     We allocate an extra buffer and copy a bit to and from.
00852     Note that there are some nasties that cannot be resolved.
00853 */
00854
00855 int ReverseStatements(STREAM *stream)
00856 {
00857     UBYTE *spare = (UBYTE *)Malloc1((stream->inbuffer+1)*sizeof(UBYTE),"Reverse copy");
00858     UBYTE *top = stream->buffer + stream->inbuffer, *in, *s, *ss, *out;
00859     out = spare+stream->inbuffer+1;
00860     in = stream->buffer;
00861     while ( in < top ) {
00862         s = in;
00863         if ( *s == AP.ComChar ) {
00864             toeol;
00865             for(;;) {
00866                 if ( s == top ) { *--out = '\n'; break; }
00867                 if ( *s == '\\\\' ) {
00868                     s++;
00869                     if ( s >= top ) { /* This is an error! */
00870                         MesPrint("@Irregular end of reverse include file.");
00871                         return(1);
00872                     }

```

```

00873     }
00874     else if ( *s == '\n' ) {
00875         s++; ss = s;
00876         while ( ss > in ) *--out = *--ss;
00877         in = s;
00878         if ( out[0] == AP.ComChar && ss+6 < s && out[3] == '#' ) {
00879             /*
00880                 For folds we have to exchange begin and end
00881             */
00882                 if ( out[4] == '[' ) out[4] = ']';
00883                 else if ( out[4] == ']' ) out[4] = '[';
00884             }
00885             break;
00886         }
00887         s++;
00888     }
00889     continue;
00890 }
00891 while ( s < top && ( *s == ' ' || *s == '\t' ) ) s++;
00892 if ( *s == '#' ) { /* preprocessor instruction */
00893     goto toeol; /* read to end of line */
00894 }
00895 if ( *s == '.' ) { /* end-of-module instruction */
00896     goto toeol; /* read to end of line */
00897 }
00898 /*
00899     Here we have a regular statement. In principle we scan to ; and its \n
00900     but there are special cases.
00901     1: ; inside a string (in print ".....;")
00902     2: multiple statements on one line.
00903     3: ; + commentary after some blanks.
00904     4: 'var' can cause problems.....
00905 */
00906 while ( s < top ) {
00907     if ( *s == ';' ) {
00908         s++;
00909         while ( s < top && ( *s == ' ' || *s == '\t' ) ) s++;
00910         while ( s < top && *s == '\n' ) s++;
00911         if ( s >= top && s[-1] != '\n' ) *s++ = '\n';
00912         ss = s;
00913         while ( ss > in ) *--out = *--ss;
00914         in = s;
00915         break;
00916     }
00917     else if ( *s == '"' ) {
00918         s++;
00919         while ( s < top ) {
00920             if ( *s == '"' ) break;
00921             if ( *s == '\\ ' ) { s++; }
00922             s++;
00923         }
00924         if ( s >= top ) goto irrend;
00925     }
00926     else if ( *s == '\\ ' ) {
00927         s++;
00928         if ( s >= top ) goto irrend;
00929     }
00930     s++;
00931 }
00932 if ( in < top ) { /* Like blank lines at the end */
00933     if ( s >= top && s[-1] != '\n' ) *s++ = '\n';
00934     ss = s;
00935     while ( ss > in ) *--out = *--ss;
00936     in = s;
00937 }
00938 }
00939 if ( out == spare ) stream->inbuffer++;
00940 if ( out > spare+1 ) {
00941     MesPrint("@Internal error in #reverseinclude instruction.");
00942     return(1);
00943 }
00944 memcpy((void *) (stream->buffer), (void *) out, (size_t) (stream->inbuffer*sizeof(UBYTE)));
00945 M_free(spare, "Reverse copy");
00946 return(0);
00947 }
00948
00949 /*
00950     #] ReverseStatements :
00951     #] Streams :
00952     #[ Files :
00953     #[ StartFiles :
00954 */
00955
00956 void StartFiles(void)
00957 {
00958     int i = CreateHandle();
00959     filelist[i] = Ustdout;

```

```

00960     AM.StdOut = i;
00961     AC.StoreHandle = -1;
00962     AC.LogHandle = -1;
00963 #ifndef WITHPTHREADS
00964     AR.Fscr[0].handle = -1;
00965     AR.Fscr[1].handle = -1;
00966     AR.Fscr[2].handle = -1;
00967     AR.FoStage4[0].handle = -1;
00968     AR.FoStage4[1].handle = -1;
00969     AR.infile = &(AR.Fscr[0]);
00970     AR.outfile = &(AR.Fscr[1]);
00971     AR.hidefile = &(AR.Fscr[2]);
00972     AR.StoreData.Handle = -1;
00973 #endif
00974     AC.Streams = 0;
00975     AC.MaxNumStreams = 0;
00976 }
00977
00978 /*
00979     #] StartFiles :
00980     #[ OpenFile :
00981 */
00982
00983 int OpenFile(char *name)
00984 {
00985     FILES *f;
00986     int i;
00987
00988     if ( ( f = Uopen(name,"rb") ) == 0 ) return(-1);
00989 /* Usetbuf(f,0); */
00990     i = CreateHandle();
00991     RWLOCKW(AM.handlelock);
00992     filelist[i] = f;
00993     UNRWLOCK(AM.handlelock);
00994     return(i);
00995 }
00996
00997 /*
00998     #] OpenFile :
00999     #[ OpenAddFile :
01000 */
01001
01002 int OpenAddFile(char *name)
01003 {
01004     FILES *f;
01005     int i;
01006     POSITION scrpos;
01007     if ( ( f = Uopen(name,"a+b") ) == 0 ) return(-1);
01008 /* Usetbuf(f,0); */
01009     i = CreateHandle();
01010     RWLOCKW(AM.handlelock);
01011     filelist[i] = f;
01012     UNRWLOCK(AM.handlelock);
01013     TELLFILE(i,&scrpos);
01014     SeekFile(i,&scrpos,SEEK_SET);
01015     return(i);
01016 }
01017
01018 /*
01019     #] OpenAddFile :
01020     #[ ReOpenFile :
01021 */
01022
01023 int ReOpenFile(char *name)
01024 {
01025     FILES *f;
01026     int i;
01027     POSITION scrpos;
01028     if ( ( f = Uopen(name,"r+b") ) == 0 ) return(-1);
01029     i = CreateHandle();
01030     RWLOCKW(AM.handlelock);
01031     filelist[i] = f;
01032     UNRWLOCK(AM.handlelock);
01033     TELLFILE(i,&scrpos);
01034     SeekFile(i,&scrpos,SEEK_SET);
01035     return(i);
01036 }
01037
01038 /*
01039     #] ReOpenFile :
01040     #[ CreateFile :
01041 */
01042
01043 int CreateFile(char *name)
01044 {
01045     FILES *f;
01046     int i;

```



```

01047     if ( ( f = Uopen(name,"w+b") ) == 0 ) return(-1);
01048     i = CreateHandle();
01049     RWLOCKW(AM.handlelock);
01050     filelist[i] = f;
01051     UNRWLOCK(AM.handlelock);
01052     return(i);
01053 }
01054
01055 /*
01056     #] CreateFile :
01057     #[ CreateLogFile :
01058 */
01059
01060 int CreateLogFile(char *name)
01061 {
01062     FILES *f;
01063     int i;
01064     if ( ( f = Uopen(name,"w+b") ) == 0 ) return(-1);
01065     Usetbuf(f,0);
01066     i = CreateHandle();
01067     RWLOCKW(AM.handlelock);
01068     filelist[i] = f;
01069     UNRWLOCK(AM.handlelock);
01070     return(i);
01071 }
01072
01073 /*
01074     #] CreateLogFile :
01075     #[ CloseFile :
01076 */
01077
01078 void CloseFile(int handle)
01079 {
01080     if ( handle >= 0 ) {
01081         FILES *f; /* we need this variable to be thread-safe */
01082         RWLOCKW(AM.handlelock);
01083         f = filelist[handle];
01084         filelist[handle] = 0;
01085         numinfilelist--;
01086         UNRWLOCK(AM.handlelock);
01087         Uclose(f);
01088     }
01089 }
01090
01091 /*
01092     #] CloseFile :
01093     #[ CopyFile :
01094 */
01095
01101 int CopyFile(char *source, char *dest)
01102 {
01103     #define COPYFILEBUFSIZE 40960L
01104     FILE *in, *out;
01105     size_t countin, countout, sumcount;
01106     char *buffer = NULL;
01107
01108     sumcount = (AM.S0->LargeSize+AM.S0->SmallEsize)*sizeof(WORD);
01109     if ( sumcount <= COPYFILEBUFSIZE ) {
01110         sumcount = COPYFILEBUFSIZE;
01111         buffer = (char*)Malloc1(sumcount, "file copy buffer");
01112     }
01113     else {
01114         buffer = (char *) (AM.S0->lBuffer);
01115     }
01116
01117     in = fopen(source, "rb");
01118     if ( in == NULL ) {
01119         perror("CopyFile: ");
01120         return(1);
01121     }
01122     out = fopen(dest, "wb");
01123     if ( out == NULL ) {
01124         perror("CopyFile: ");
01125         return(2);
01126     }
01127
01128     while ( !feof(in) ) {
01129         countin = fread(buffer, 1, sumcount, in);
01130         if ( countin != sumcount ) {
01131             if ( ferror(in) ) {
01132                 perror("CopyFile: ");
01133                 return(3);
01134             }
01135         }
01136         countout = fwrite(buffer, 1, countin, out);
01137         if ( countin != countout ) {
01138             perror("CopyFile: ");

```

```

01139         return(4);
01140     }
01141 }
01142
01143 fclose(in);
01144 fclose(out);
01145 if ( sumcount <= COPYFILEBUFSIZE ) {
01146     M_free(buffer, "file copy buffer");
01147 }
01148 return(0);
01149 }
01150
01151 /*
01152     #] CopyFile :
01153     #[ CreateHandle :
01154
01155     We need a lock here.
01156     Problem: the same lock is needed inside Malloc1 and M_free which
01157     is used in DoubleList when we use MALLOCDEBUG
01158
01159     Conclusion: MALLOCDEBUG will have to be a bit unsafe
01160 */
01161
01162 int CreateHandle(void)
01163 {
01164     int i, j;
01165     #ifndef MALLOCDEBUG
01166         RWLOCKW(AM.handlelock);
01167     #endif
01168     if ( filelistsize == 0 ) {
01169         filelistsize = 10;
01170         filelist = (FILES **)Malloc1(sizeof(FILES *)*filelistsize, "file handle");
01171         for ( j = 0; j < filelistsize; j++ ) filelist[j] = 0;
01172         numinfilelist = 1;
01173         i = 0;
01174     }
01175     else if ( numinfilelist >= filelistsize ) {
01176         void **fl = (void **)filelist;
01177         i = filelistsize;
01178         if ( DoubleList((void **)(fl), &filelistsize, (int)sizeof(FILES *),
01179             "list of open files") != 0 ) Terminate(-1);
01180         filelist = (FILES **)fl;
01181         for ( j = i; j < filelistsize; j++ ) filelist[j] = 0;
01182         numinfilelist = i + 1;
01183     }
01184     else {
01185         i = filelistsize;
01186         for ( j = 0; j < filelistsize; j++ ) {
01187             if ( filelist[j] == 0 ) { i = j; break; }
01188         }
01189         numinfilelist++;
01190     }
01191     filelist[i] = (FILES *) (filelist); /* Just for now to not get into problems */
01192     /*
01193     The next code is not needed when we use open.
01194     It may be needed when we use fopen.
01195     fopen is used in minos.c without this central administration.
01196     */
01197     if ( numinfilelist > MAX_OPEN_FILES ) {
01198         #ifndef MALLOCDEBUG
01199             UNRWLOCK(AM.handlelock);
01200         #endif
01201         MesPrint("More than %d open files", MAX_OPEN_FILES);
01202         Error0("System limit. This limit is not due to FORM!");
01203     }
01204     else {
01205         #ifndef MALLOCDEBUG
01206             UNRWLOCK(AM.handlelock);
01207         #endif
01208     }
01209     return(i);
01210 }
01211
01212 /*
01213     #] CreateHandle :
01214     #[ ReadFile :
01215     */
01216
01217 LONG ReadFile(int handle, UBYTE *buffer, LONG size)
01218 {
01219     LONG inbuf = 0, r;
01220     FILES *f;
01221     char *b;
01222     b = (char *)buffer;
01223     for(;;) { /* Gotta do difficult because of VMS! */
01224         RWLOCKR(AM.handlelock);
01225         f = filelist[handle];

```

```

01226     UNRWLOCK(AM.handlelock);
01227 #ifdef WITHSTATS
01228     numreads++;
01229 #endif
01230     r = Uread(b,1,size,f);
01231     if ( r < 0 ) return(r);
01232     if ( r == 0 ) return(inbuf);
01233     inbuf += r;
01234     if ( r == size ) return(inbuf);
01235     if ( r > size ) return(-1);
01236     size -= r;
01237     b += r;
01238 }
01239 }
01240
01241 /*
01242     #] ReadFile :
01243     #[ ReadPosFile :
01244
01245     Gets words from a file(handle).
01246     First tries to get the information from the buffers.
01247     Reads a file at a position. Updates the position.
01248     Places a lock in the case of multithreading.
01249     Exists for multiple reading from the same file.
01250     size is the number of WORDs to read!!!!
01251
01252     We may need some strategy in the caching. This routine is used from
01253     GetOneTerm only. The problem is when it reads brackets and the
01254     brackets are read backwards. This is very uneconomical because
01255     each time it may read a large buffer.
01256     On the other hand, reading piece by piece in GetOneTerm takes
01257     much overhead as well.
01258     Two strategies come to mind:
01259     1: keep things as they are but limit the size of the buffers.
01260     2: have the position of 'pos' at about 1/3 of the buffer.
01261         this is of course guess work.
01262     Currently we have implemented the first method by creating the
01263     setup parameter threadscratchsize with the default value 100K.
01264     In the test program much bigger values gave a slower program.
01265 */
01266 LONG ReadPosFile(PHEAD FILEHANDLE *fi, UBYTE *buffer, LONG size, POSITION *pos)
01267 {
01268     GETBIDENTITY
01269     LONG i, retval = 0;
01270     WORD *b = (WORD *)buffer, *t;
01271
01272     if ( fi->handle < 0 ) {
01273         fi->POfill = (WORD *)((UBYTE *) (fi->PObuffer) + BASEPOSITION(*pos));
01274         t = fi->POfill;
01275         while ( size > 0 && fi->POfill < fi->POfull ) { *b++ = *t++; size--; }
01276     }
01277     else {
01278         if ( ISLESSPOS(*pos,fi->POposition) || ISGEPOSINC(*pos,fi->POposition,
01279             ((UBYTE *) (fi->POfull)-(UBYTE *) (fi->PObuffer))) ) {
01280             /*
01281                 The start is not inside the buffer. Fill the buffer.
01282             */
01283
01284             fi->POposition = *pos;
01285             LOCK(AS.inputslock);
01286             SeekFile(fi->handle,pos,SEEK_SET);
01287             retval = ReadFile(fi->handle,(UBYTE *) (fi->PObuffer),fi->POsize);
01288             UNLOCK(AS.inputslock);
01289             fi->POfull = fi->PObuffer+retval/sizeof(WORD);
01290             fi->POfill = fi->PObuffer;
01291             if ( fi != AR.hidefile ) AR.InInBuf = retval/sizeof(WORD);
01292             else AR.InHiBuf = retval/sizeof(WORD);
01293         }
01294         else {
01295             fi->POfill = (WORD *)((UBYTE *) (fi->PObuffer) + DIFBASE(*pos,fi->POposition));
01296         }
01297         if ( fi->POfill + size <= fi->POfull ) {
01298             t = fi->POfill;
01299             while ( size > 0 ) { *b++ = *t++; size--; }
01300         }
01301         else {
01302             for (;;) {
01303                 i = fi->POfull - fi->POfill; t = fi->POfill;
01304                 if ( i > size ) i = size;
01305                 size -= i;
01306                 while ( --i >= 0 ) *b++ = *t++;
01307                 if ( size == 0 ) break;
01308                 ADDPOS(fi->POposition,(UBYTE *) (fi->POfull)-(UBYTE *) (fi->PObuffer));
01309                 LOCK(AS.inputslock);
01310                 SeekFile(fi->handle,&(fi->POposition),SEEK_SET);
01311                 retval = ReadFile(fi->handle,(UBYTE *) (fi->PObuffer),fi->POsize);
01312             }

```

```

01313         UNLOCK(AS.inputslock);
01314         fi->POfull = fi->PObuffer+retval/sizeof(WORD);
01315         fi->POfill = fi->PObuffer;
01316         if ( fi != AR.hidefile ) AR.InInBuf = retval/sizeof(WORD);
01317         else                     AR.InHiBuf = retval/sizeof(WORD);
01318         if ( retval == 0 ) { t = fi->POfill; break; }
01319     }
01320 }
01321 }
01322 retval = (UBYTE *)b - buffer;
01323 fi->POfill = t;
01324 ADDPOS(*pos,retval);
01325 return(retval);
01326 }
01327
01328 /*
01329     #] ReadPosFile :
01330     #[ WriteFile :
01331 */
01332
01333 LONG WriteFileToFile(int handle, UBYTE *buffer, LONG size)
01334 {
01335     FILES *f;
01336     LONG retval, totalwritten = 0, stilltowrite;
01337     RWLOCKR(AM.handlelock);
01338     f = filelist[handle];
01339     UNRWLOCK(AM.handlelock);
01340     while ( totalwritten < size ) {
01341         stilltowrite = size - totalwritten;
01342 #ifdef WITHSTATS
01343         numwrites++;
01344 #endif
01345         retval = Uwrite((char *)buffer+totalwritten,1,stilltowrite,f);
01346         if ( retval < 0 ) return(retval);
01347         if ( retval == 0 ) return(totalwritten);
01348         totalwritten += retval;
01349     }
01350 /*
01351 if ( handle == AC.LogHandle || handle == ERROROUT ) FlushFile(handle);
01352 */
01353     return(totalwritten);
01354 }
01355 #ifndef WITHMPI
01356 /*[17nov2005]:*/
01357 WRITEFILE WriteFile = &WriteFileToFile;
01358 /*
01359 LONG (*WriteFile)(int handle, UBYTE *buffer, LONG size) = &WriteFileToFile;
01360 */
01361 /*:[17nov2005]:*/
01362 #else
01363 WRITEFILE WriteFile = &PF_WriteFileToFile;
01364 #endif
01365
01366 /*
01367     #] WriteFile :
01368     #[ SeekFile :
01369 */
01370
01371 void SeekFile(int handle, POSITION *offset, int origin)
01372 {
01373     FILES *f;
01374     RWLOCKR(AM.handlelock);
01375     f = filelist[handle];
01376     UNRWLOCK(AM.handlelock);
01377 #ifdef WITHSTATS
01378     numseeks++;
01379 #endif
01380     if ( origin == SEEK_SET ) {
01381         Useek(f,BASEPOSITION(*offset),origin);
01382         SETBASEPOSITION(*offset,(Utell(f)));
01383         return;
01384     }
01385     else if ( origin == SEEK_END ) {
01386         Useek(f,0,origin);
01387     }
01388     SETBASEPOSITION(*offset,(Utell(f)));
01389 }
01390
01391 /*
01392     #] SeekFile :
01393     #[ TellFile :
01394 */
01395
01396 LONG TellFile(int handle)
01397 {
01398     POSITION pos;
01399     TELLFILE(handle,&pos);

```

```

01400 #ifdef WITHSTATS
01401     numseeks++;
01402 #endif
01403     return (BASEPOSITION(pos));
01404 }
01405
01406 void TELLFILE(int handle, POSITION *position)
01407 {
01408     FILES *f;
01409     RWLOCKR(AM.handlelock);
01410     f = filelist[handle];
01411     UNRWLOCK(AM.handlelock);
01412     SETBASEPOSITION(*position, (Utell(f)));
01413 }
01414
01415 /*
01416     #] TellFile :
01417     #[ FlushFile :
01418 */
01419
01420 void FlushFile(int handle)
01421 {
01422     FILES *f;
01423     RWLOCKR(AM.handlelock);
01424     f = filelist[handle];
01425     UNRWLOCK(AM.handlelock);
01426     Uflush(f);
01427 }
01428
01429 /*
01430     #] FlushFile :
01431     #[ GetPosFile :
01432 */
01433
01434 int GetPosFile(int handle, fpos_t *pospointer)
01435 {
01436     FILES *f;
01437     RWLOCKR(AM.handlelock);
01438     f = filelist[handle];
01439     UNRWLOCK(AM.handlelock);
01440     return (Ugetpos(f, pospointer));
01441 }
01442
01443 /*
01444     #] GetPosFile :
01445     #[ SetPosFile :
01446 */
01447
01448 int SetPosFile(int handle, fpos_t *pospointer)
01449 {
01450     FILES *f;
01451     RWLOCKR(AM.handlelock);
01452     f = filelist[handle];
01453     UNRWLOCK(AM.handlelock);
01454     return (Usetpos(f, (fpos_t *)pospointer));
01455 }
01456
01457 /*
01458     #] SetPosFile :
01459     #[ SynchFile :
01460
01461     It may be that when we use many sort files at the same time there
01462     is a big traffic jam in the cache. This routine is experimental,
01463     just to see whether this improves the situation.
01464     It could also be that the internal disk of the Quad opteron norma
01465     is very slow.
01466 */
01467
01468 void SynchFile(int handle)
01469 {
01470     FILES *f;
01471     if ( handle >= 0 ) {
01472         RWLOCKR(AM.handlelock);
01473         f = filelist[handle];
01474         UNRWLOCK(AM.handlelock);
01475         Usync(f);
01476     }
01477 }
01478
01479 /*
01480     #] SynchFile :
01481     #[ TruncateFile :
01482
01483     It may be that when we use many sort files at the same time there
01484     is a big traffic jam in the cache. This routine is experimental,
01485     just to see whether this improves the situation.
01486     It could also be that the internal disk of the Quad opteron norma

```

```

01487         is very slow.
01488     */
01489
01490 void TruncateFile(int handle)
01491 {
01492     FILES *f;
01493     if ( handle >= 0 ) {
01494         RWLOCKR(AM.handlelock);
01495         f = filelist[handle];
01496         UNRWLOCK(AM.handlelock);
01497         Utruncate(f);
01498     }
01499 }
01500
01501 /*
01502     #] TruncateFile :
01503     #[ GetChannel :
01504
01505     Checks whether we have this file already. If so, we return its
01506     handle. If not and mode == 0, we open the file first and add it
01507     to the buffers.
01508 */
01509
01510 int GetChannel(char *name,int mode)
01511 {
01512     CHANNEL *ch;
01513     int i;
01514     FILES *f;
01515     for ( i = 0; i < NumOutputChannels; i++ ) {
01516         if ( channels[i].name == 0 ) continue;
01517         if ( StrCmp((UBYTE *)name,(UBYTE *) (channels[i].name)) == 0 ) return(channels[i].handle);
01518     }
01519     if ( mode == 1 ) {
01520         MesPrint("&File %s in print statement is not open",name);
01521         MesPrint("    You should open it first with a #write or #append instruction");
01522         return(-1);
01523     }
01524     for ( i = 0; i < NumOutputChannels; i++ ) {
01525         if ( channels[i].name == 0 ) break;
01526     }
01527     if ( i < NumOutputChannels ) { ch = &(channels[i]); }
01528     else { ch = (CHANNEL *)FromList(&AC.Channellist); }
01529     ch->name = (char *)strDup1((UBYTE *)name,"name of channel");
01530     ch->handle = CreateFile(name);
01531     RWLOCKR(AM.handlelock);
01532     f = filelist[ch->handle];
01533     UNRWLOCK(AM.handlelock);
01534     Usetbuf(f,0); /* We turn the buffer off!!!!!!*/
01535     return(ch->handle);
01536 }
01537
01538 /*
01539     #] GetChannel :
01540     #[ GetAppendChannel :
01541
01542     Checks whether we have this file already. If so, we return its
01543     handle. If not, we open the file first and add it to the buffers.
01544 */
01545
01546 int GetAppendChannel(char *name)
01547 {
01548     CHANNEL *ch;
01549     int i;
01550     FILES *f;
01551     for ( i = 0; i < NumOutputChannels; i++ ) {
01552         if ( channels[i].name == 0 ) continue;
01553         if ( StrCmp((UBYTE *)name,(UBYTE *) (channels[i].name)) == 0 ) return(channels[i].handle);
01554     }
01555     for ( i = 0; i < NumOutputChannels; i++ ) {
01556         if ( channels[i].name == 0 ) break;
01557     }
01558     if ( i < NumOutputChannels ) { ch = &(channels[i]); }
01559     else { ch = (CHANNEL *)FromList(&AC.Channellist); }
01560     ch->name = (char *)strDup1((UBYTE *)name,"name of channel");
01561     ch->handle = OpenAddFile(name);
01562     RWLOCKR(AM.handlelock);
01563     f = filelist[ch->handle];
01564     UNRWLOCK(AM.handlelock);
01565     Usetbuf(f,0); /* We turn the buffer off!!!!!!*/
01566     return(ch->handle);
01567 }
01568
01569 /*
01570     #] GetAppendChannel :
01571     #[ CloseChannel :
01572
01573     Checks whether we have this file already. If so, we close it.

```

```

01574 */
01575
01576 int CloseChannel(char *name)
01577 {
01578     int i;
01579     for ( i = 0; i < NumOutputChannels; i++ ) {
01580         if ( channels[i].name == 0 ) continue;
01581         if ( channels[i].name[0] == 0 ) continue;
01582         if ( StrCmp((UBYTE *)name, (UBYTE *) (channels[i].name)) == 0 ) {
01583             CloseFile(channels[i].handle);
01584             M_free(channels[i].name, "CloseChannel");
01585             channels[i].name = 0;
01586             return(0);
01587         }
01588     }
01589     return(0);
01590 }
01591
01592 /*
01593     #] CloseChannel :
01594     #[ UpdateMaxSize :
01595
01596     Updates the maximum size of the combined input/output/hide scratch
01597     files, the sort files and the .str file.
01598     The result becomes only visible with either
01599         ON totalsize;
01600         #: totalsize ON;
01601     or the -T in the command tail.
01602
01603     To be called, whenever a file is closed/removed or truncated to zero.
01604
01605     We have no provisions yet for expressions that remain inside the
01606     small or large buffer during the sort. The space they use there is
01607     currently ignored.
01608 */
01609
01610 void UpdateMaxSize(void)
01611 {
01612     POSITION position, sumsize;
01613     int i;
01614     FILEHANDLE *scr;
01615 #ifndef WITHMPI
01616     /* Currently, it works only on the master. The sort files on the slaves
01617      * are ignored. (TU 11 Oct 2011) */
01618     if ( PF.me != MASTER ) return;
01619 #endif
01620     PUTZERO(sumsize);
01621     if ( AM.PrintTotalSize ) {
01622         /*
01623             First the three scratch files
01624         */
01625 #ifdef WITHPTHREADS
01626         scr = AB[0]->R.Fscr;
01627     #else
01628         scr = AR.Fscr;
01629     #endif
01630         for ( i = 0; i <=2; i++ ) {
01631             if ( scr[i].handle < 0 ) {
01632                 SETBASEPOSITION(position, (scr[i].POfull-scr[i].PObuffer)*sizeof(WORD));
01633             }
01634             else {
01635                 position = scr[i].filesize;
01636             }
01637             ADD2POS(sumsize, position);
01638         }
01639         /*
01640             Now the sort file(s)
01641         */
01642 #ifdef WITHPTHREADS
01643         {
01644             int j;
01645             ALLPRIVATES *B;
01646             for ( j = 0; j < AM.totalnumberofthreads; j++ ) {
01647                 B = AB[j];
01648                 if ( AT.SS && AT.SS->file.handle >= 0 ) {
01649                     position = AT.SS->file.filesize;
01650                 }
01651                 MLOCK(ErrorMessageLock);
01652                 MesPrint("%d: %10p", j, &(AT.SS->file.filesize));
01653                 MUNLOCK(ErrorMessageLock);
01654                 /*
01655                     ADD2POS(sumsize, position);
01656                 */
01657                 if ( AR.FoStage4[0].handle >= 0 ) {
01658                     position = AR.FoStage4[0].filesize;
01659                     ADD2POS(sumsize, position);
01660                 }

```

```

01661     }
01662   }
01663   #else
01664     if ( AT.SS && AT.SS->file.handle >= 0 ) {
01665       position = AT.SS->file.filesize;
01666       ADD2POS(sumsize,position);
01667     }
01668     if ( AR.FoStage4[0].handle >= 0 ) {
01669       position = AR.FoStage4[0].filesize;
01670       ADD2POS(sumsize,position);
01671     }
01672   #endif
01673   /*
01674       And of course the str file.
01675   */
01676   ADD2POS(sumsize,AC.StoreFileSize);
01677   /*
01678       Finally the test whether it is bigger
01679   */
01680   if ( ISLESSPOS(AS.MaxExprSize,sumsize) ) {
01681     #ifdef WITHPTHREADS
01682       LOCK(AS.MaxExprSizeLock);
01683       if ( ISLESSPOS(AS.MaxExprSize,sumsize) ) AS.MaxExprSize = sumsize;
01684       UNLOCK(AS.MaxExprSizeLock);
01685     #else
01686       AS.MaxExprSize = sumsize;
01687     #endif
01688   }
01689   }
01690   return;
01691 }
01692
01693 /*
01694     #] UpdateMaxSize :
01695     #] Files :
01696     #[ Strings :
01697     #[ StrCmp :
01698 */
01699
01700 int StrCmp(UBYTE *s1, UBYTE *s2)
01701 {
01702     while ( *s1 && *s1 == *s2 ) { s1++; s2++; }
01703     return((int)*s1-(int)*s2);
01704 }
01705
01706 /*
01707     #] StrCmp :
01708     #[ StrICmp :
01709 */
01710
01711 int StrICmp(UBYTE *s1, UBYTE *s2)
01712 {
01713     while ( *s1 && tolower(*s1) == tolower(*s2) ) { s1++; s2++; }
01714     return((int)tolower(*s1)-(int)tolower(*s2));
01715 }
01716
01717 /*
01718     #] StrICmp :
01719     #[ StrHICmp :
01720 */
01721
01722 int StrHICmp(UBYTE *s1, UBYTE *s2)
01723 {
01724     while ( *s1 && tolower(*s1) == *s2 ) { s1++; s2++; }
01725     return((int)tolower(*s1)-(int)(*s2));
01726 }
01727
01728 /*
01729     #] StrHICmp :
01730     #[ StrICont :
01731 */
01732
01733 int StrICont(UBYTE *s1, UBYTE *s2)
01734 {
01735     while ( *s1 && tolower(*s1) == tolower(*s2) ) { s1++; s2++; }
01736     if ( *s1 == 0 ) return(0);
01737     return((int)tolower(*s1)-(int)tolower(*s2));
01738 }
01739
01740 /*
01741     #] StrICont :
01742     #[ CmpArray :
01743 */
01744
01745 int CmpArray(WORD *t1, WORD *t2, WORD n)
01746 {
01747     int i,x;

```



```

01748     for ( i = 0; i < n; i++ ) {
01749         if ( ( x = (int)(t1[i]-t2[i]) ) != 0 ) return(x);
01750     }
01751     return(0);
01752 }
01753
01754 /*
01755     #] CmpArray :
01756     #[ ConWord :
01757 */
01758
01759 int ConWord(UBYTE *s1, UBYTE *s2)
01760 {
01761     while ( *s1 && ( tolower(*s1) == tolower(*s2) ) ) { s1++; s2++; }
01762     if ( *s1 == 0 ) return(1);
01763     return(0);
01764 }
01765
01766 /*
01767     #] ConWord :
01768     #[ StrLen :
01769 */
01770
01771 int StrLen(UBYTE *s)
01772 {
01773     int i = 0;
01774     while ( *s ) { s++; i++; }
01775     return(i);
01776 }
01777
01778 /*
01779     #] StrLen :
01780     #[ NumToStr :
01781 */
01782
01783 void NumToStr(UBYTE *s, LONG x)
01784 {
01785     UBYTE *t, str[24];
01786     ULONG xx;
01787     t = str;
01788     if ( x < 0 ) { *s++ = '-'; xx = -x; }
01789     else xx = x;
01790     do {
01791         *t++ = xx % 10 + '0';
01792         xx /= 10;
01793     } while ( xx );
01794     while ( t > str ) *s++ = *--t;
01795     *s = 0;
01796 }
01797
01798 /*
01799     #] NumToStr :
01800     #[ WriteString :
01801
01802     Writes a characterstring to the various outputs.
01803     The action may depend on the flags involved.
01804     The type of output is given by type, the string by str and the
01805     number of characters in it by num
01806 */
01807 void WriteString(int type, UBYTE *str, int num)
01808 {
01809     int error = 0;
01810
01811     if ( num > 0 && str[num-1] == 0 ) { num--; }
01812     else if ( num <= 0 || str[num-1] != LINEFEED ) {
01813         AddLineFeed(str,num);
01814     }
01815     /*[15apr2004 mt]:*/
01816     if(type == EXTERNALCHANNELOUT){
01817         if(WriteFile(0,str,num) != num) error = 1;
01818     }else
01819     /*:[15apr2004 mt]*/
01820     if ( AM.silent == 0 || type == ERROROUT ) {
01821         if ( type == INPUTOUT ) {
01822             if ( !AM.FileOnlyFlag && WriteFile(AM.Stdout, (UBYTE *) "      ",4) != 4 ) error = 1;
01823             if ( AC.LogHandle >= 0 && WriteFile(AC.LogHandle, (UBYTE *) "      ",4) != 4 ) error = 1;
01824         }
01825         if ( !AM.FileOnlyFlag && WriteFile(AM.Stdout,str,num) != num ) error = 1;
01826         if ( AC.LogHandle >= 0 && WriteFile(AC.LogHandle,str,num) != num ) error = 1;
01827     }
01828     if ( error ) Terminate(-1);
01829 }
01830
01831 /*
01832     #] WriteString :
01833     #[ WriteUnfinString :
01834

```

```

01835         Writes a characterstring to the various outputs.
01836         The action may depend on the flags involved.
01837         The type of output is given by type, the string by str and the
01838         number of characters in it by num
01839     */
01840
01841 void WriteUnfinString(int type, UBYTE *str, int num)
01842 {
01843     int error = 0;
01844
01845     /*[15apr2004 mt]:*/
01846     if(type == EXTERNALCHANNELOUT){
01847         if(WriteFile(0,str,num) != num) error = 1;
01848     }else
01849     /*:[15apr2004 mt]*/
01850     if ( AM.silent == 0 || type == ERROROUT ) {
01851         if ( type == INPUTOUT ) {
01852             if ( !AM.FileOnlyFlag && WriteFile(AM.Stdout,(UBYTE *)"      ",4) != 4 ) error = 1;
01853             if ( AC.LogHandle >= 0 && WriteFile(AC.LogHandle,(UBYTE *)"      ",4) != 4 ) error = 1;
01854         }
01855         if ( !AM.FileOnlyFlag && WriteFile(AM.Stdout,str,num) != num ) error = 1;
01856         if ( AC.LogHandle >= 0 && WriteFile(AC.LogHandle,str,num) != num ) error = 1;
01857     }
01858     if ( error ) Terminate(-1);
01859 }
01860
01861 /*
01862     #] WriteUnfinString :
01863     #[ AddToString :
01864 */
01865
01866 UBYTE *AddToString(UBYTE *outstring, UBYTE *extrastring, int par)
01867 {
01868     UBYTE *s = extrastring, *t, *newstring;
01869     int n, nn;
01870     while ( *s ) { s++; }
01871     n = s-extrastring;
01872     if ( outstring == 0 ) {
01873         s = extrastring;
01874         t = outstring = (UBYTE *)Malloca1(n+1,"AddToString");
01875         NCOPY(t,s,n)
01876         *t++ = 0;
01877         return(outstring);
01878     }
01879     else {
01880         t = outstring;
01881         while ( *t ) t++;
01882         nn = t - outstring;
01883         t = newstring = (UBYTE *)Malloca1(n+nn+2,"AddToString");
01884         s = outstring;
01885         NCOPY(t,s,nn)
01886         if ( par == 1 ) *t++ = ',';
01887         s = extrastring;
01888         NCOPY(t,s,n)
01889         *t = 0;
01890         M_free(outstring,"AddToString");
01891         return(newstring);
01892     }
01893 }
01894
01895 /*
01896     #] AddToString :
01897     #[ strDupl :
01898
01899     string duplication with message passing for Malloca1, allowing
01900     this routine to give a more detailed error message if there
01901     is not enough memory.
01902 */
01903
01904 UBYTE *strDupl(UBYTE *instring, char *ifwrong)
01905 {
01906     UBYTE *s = instring, *to;
01907     while ( *s ) s++;
01908     to = s = (UBYTE *)Malloca1((s-instring)+1,ifwrong);
01909     while ( *instring ) *to++ = *instring++;
01910     *to = 0;
01911     return(s);
01912 }
01913
01914 /*
01915     #] strDupl :
01916     #[ EndOfToken :
01917 */
01918
01919 UBYTE *EndOfToken(UBYTE *s)
01920 {
01921     UBYTE c;

```

```

01922     while ( ( c = (UBYTE)(FG.cTable[*s]) ) == 0 || c == 1 ) s++;
01923     return(s);
01924 }
01925
01926 /*
01927     #] EndOfToken :
01928     #[ ToToken :
01929 */
01930
01931 UBYTE *ToToken(UBYTE *s)
01932 {
01933     UBYTE c;
01934     while ( *s && ( c = (UBYTE)(FG.cTable[*s]) ) != 0 && c != 1 ) s++;
01935     return(s);
01936 }
01937
01938 /*
01939     #] ToToken :
01940     #[ SkipField :
01941
01942     Skips from s to the end of a declaration field.
01943     par is the number of parentheses that still has to be closed.
01944 */
01945
01946 UBYTE *SkipField(UBYTE *s, int level)
01947 {
01948     while ( *s ) {
01949         if ( *s == ',' && level == 0 ) return(s);
01950         if ( *s == '(' ) level++;
01951         else if ( *s == ')' ) { level--; if ( level < 0 ) level = 0; }
01952         else if ( *s == '[' ) {
01953             SKIPBRA1(s)
01954         }
01955         else if ( *s == '{' ) {
01956             SKIPBRA2(s)
01957         }
01958         s++;
01959     }
01960     return(s);
01961 }
01962
01963 /*
01964     #] SkipField :
01965     #[ ReadSnum :          WORD ReadSnum(p)
01966
01967     Reads a number that should fit in a word.
01968     The number should be unsigned and a negative return value
01969     indicates an irregularity.
01970
01971 */
01972
01973 WORD ReadSnum(UBYTE **p)
01974 {
01975     LONG x = 0;
01976     UBYTE *s;
01977     s = *p;
01978     if ( FG.cTable[*s] == 1 ) {
01979         do {
01980             x = ( x << 3 ) + ( x << 1 ) + ( *s++ - '0' );
01981             if ( x > MAXPOSITIVE ) return(-1);
01982         } while ( FG.cTable[*s] == 1 );
01983         *p = s;
01984         return((WORD)x);
01985     }
01986     else return(-1);
01987 }
01988
01989 /*
01990     #] ReadSnum :
01991     #[ NumCopy :
01992
01993     Adds the decimal representation of a number to a string.
01994
01995 */
01996
01997 UBYTE *NumCopy(WORD y, UBYTE *to)
01998 {
01999     UBYTE *s;
02000     WORD i = 0, j;
02001     UWORD x;
02002     if ( y < 0 ) {
02003         *to++ = '-';
02004     }
02005     x = WordAbs(y);
02006     s = to;
02007     do { *s++ = (UBYTE)((x % 10) + '0'); i++; } while ( ( x /= 10 ) != 0 );
02008     *s-- = '\0';

```

```

02009     j = ( i - 1 ) >> 1;
02010     while ( j >= 0 ) {
02011         i = to[j]; to[j] = s[-j]; s[-j] = (UBYTE)i; j--;
02012     }
02013     return(s+1);
02014 }
02015
02016 /*
02017     #] NumCopy :
02018     #[ LongCopy :
02019
02020     Adds the decimal representation of a number to a string.
02021
02022 */
02023
02024 char *LongCopy(LONG y, char *to)
02025 {
02026     char *s;
02027     WORD i = 0, j;
02028     ULONG x;
02029     if ( y < 0 ) {
02030         *to++ = '-';
02031     }
02032     x = LongAbs(y);
02033     s = to;
02034     do { *s++ = (x % 10) + '0'; i++; } while ( ( x /= 10 ) != 0 );
02035     *s-- = '\0';
02036     j = ( i - 1 ) >> 1;
02037     while ( j >= 0 ) {
02038         i = to[j]; to[j] = s[-j]; s[-j] = (char)i; j--;
02039     }
02040     return(s+1);
02041 }
02042
02043 /*
02044     #] LongCopy :
02045     #[ LongLongCopy :
02046
02047     Adds the decimal representation of a number to a string.
02048     Bugfix feb 2003. y was not pointer!
02049 */
02050
02051 char *LongLongCopy(off_t *y, char *to)
02052 {
02053     /*
02054      * This code fails to print the maximum negative value on systems with two's
02055      * complement. To fix this, we need the unsigned version of off_t with the
02056      * same size, but unfortunately it is undefined. On the other hand, if a
02057      * system is configured with a 64-bit off_t, in practice one never reaches
02058      * 2^63 ~ 10^18 as of 2016. If one really reach such a big number, then it
02059      * would be the time to move on a 128-bit off_t.
02060      */
02061     off_t x = *y;
02062     char *s;
02063     WORD i = 0, j;
02064     if ( x < 0 ) { x = -x; *to++ = '-'; }
02065     s = to;
02066     do { *s++ = (x % 10) + '0'; i++; } while ( ( x /= 10 ) != 0 );
02067     *s-- = '\0';
02068     j = ( i - 1 ) >> 1;
02069     while ( j >= 0 ) {
02070         i = to[j]; to[j] = s[-j]; s[-j] = (char)i; j--;
02071     }
02072     return(s+1);
02073 }
02074
02075 /*
02076     #] LongLongCopy :
02077     #[ MakeDate :
02078
02079     Routine produces a string with the date and time of the run
02080 */
02081
02082 #ifdef ANSI
02083 #else
02084 #ifdef mBSD
02085 #else
02086 static char notime[] = "";
02087 #endif
02088 #endif
02089
02090 UBYTE *MakeDate(void)
02091 {
02092 #ifdef ANSI
02093     time_t tp;
02094     time(&tp);
02095     return((UBYTE *)ctime(&tp));

```

```

02096 #else
02097 #ifdef mBSD
02098     time_t tp;
02099     time(&tp);
02100     return((UBYTE *)ctime(&tp));
02101 #else
02102     return((UBYTE *)notime);
02103 #endif
02104 #endif
02105 }
02106
02107 /*
02108     #] MakeDate :
02109     #[ set_in :
02110         Returns 1 if ch is in set ; 0 if ch is not in set:
02111 */
02112 int set_in(UBYTE ch, set_of_char set)
02113 {
02114     set += ch/8;
02115     switch (ch % 8){
02116         case 0: return(set->bit_0);
02117         case 1: return(set->bit_1);
02118         case 2: return(set->bit_2);
02119         case 3: return(set->bit_3);
02120         case 4: return(set->bit_4);
02121         case 5: return(set->bit_5);
02122         case 6: return(set->bit_6);
02123         case 7: return(set->bit_7);
02124     }/*switch (ch % 8)*/
02125     return(-1);
02126 }/*set_in*/
02127
02128     #] set_in :
02129     #[ set_set :
02130         sets ch into set; returns *set:
02131 */
02132 one_byte set_set(UBYTE ch, set_of_char set)
02133 {
02134     one_byte tmp=(one_byte)set;
02135     set += ch/8;
02136     switch (ch % 8){
02137         case 0: set->bit_0=1;break;
02138         case 1: set->bit_1=1;break;
02139         case 2: set->bit_2=1;break;
02140         case 3: set->bit_3=1;break;
02141         case 4: set->bit_4=1;break;
02142         case 5: set->bit_5=1;break;
02143         case 6: set->bit_6=1;break;
02144         case 7: set->bit_7=1;break;
02145     }
02146     return(tmp);
02147 }/*set_set*/
02148
02149     #] set_set :
02150     #[ set_del :
02151         deletes ch from set; returns *set:
02152 */
02153 one_byte set_del(UBYTE ch, set_of_char set)
02154 {
02155     one_byte tmp=(one_byte)set;
02156     set += ch/8;
02157     switch (ch % 8){
02158         case 0: set->bit_0=0;break;
02159         case 1: set->bit_1=0;break;
02160         case 2: set->bit_2=0;break;
02161         case 3: set->bit_3=0;break;
02162         case 4: set->bit_4=0;break;
02163         case 5: set->bit_5=0;break;
02164         case 6: set->bit_6=0;break;
02165         case 7: set->bit_7=0;break;
02166     }
02167     return(tmp);
02168 }/*set_del*/
02169
02170     #] set_del :
02171     #[ set_sub :
02172         returns *set = set1\set2. This function may be used for initialising,
02173         set_sub(a,a,a) => now a is empty set :
02174 */
02175 one_byte set_sub(set_of_char set, set_of_char set1, set_of_char set2)
02176 {
02177     one_byte tmp=(one_byte)set;
02178     int i=0,j=0;
02179     while(j=0,i++<32)
02180     while(j<9)
02181         switch (j++){
02182             case 0: set->bit_0=(set1->bit_0&&(!set2->bit_0));break;

```

```

02183         case 1: set->bit_1=(set1->bit_1&&(!set2->bit_1));break;
02184         case 2: set->bit_2=(set1->bit_2&&(!set2->bit_2));break;
02185         case 3: set->bit_3=(set1->bit_3&&(!set2->bit_3));break;
02186         case 4: set->bit_4=(set1->bit_4&&(!set2->bit_4));break;
02187         case 5: set->bit_5=(set1->bit_5&&(!set2->bit_5));break;
02188         case 6: set->bit_6=(set1->bit_6&&(!set2->bit_6));break;
02189         case 7: set->bit_7=(set1->bit_7&&(!set2->bit_7));break;
02190         case 8: set++;set1++;set2++;
02191     };
02192     return(tmp);
02193 }/*set_sub*/
02194 /*
02195     #] set_sub :
02196     #] Strings :
02197     #[ Mixed :
02198     #[ iniTools :
02199 */
02200
02201 void iniTools(void)
02202 {
02203     #ifdef MALLOCPROTECT
02204         if ( mprotectInit() ) exit(0);
02205     #endif
02206     return;
02207 }
02208
02209 /*
02210     #] iniTools :
02211     #[ Malloc :
02212
02213     Malloc routine with built in error checking.
02214     This saves lots of messages.
02215 */
02216 #ifdef MALLOCDEBUG
02217 char *dummymessage = "Malloc";
02218 INILOCK(MallocLock)
02219 #endif
02220
02221 void *Malloc(LONG size)
02222 {
02223     void *mem;
02224     #ifdef MALLOCDEBUG
02225     char *t, *u;
02226     int i;
02227     LOCK(MallocLock);
02228     /* MLOCK(ErrorMessageLock); */
02229     if ( size == 0 ) {
02230         MesPrint("Asking for 0 bytes in Malloc");
02231     }
02232     #endif
02233     if ( ( size & 7 ) != 0 ) { size = size - ( size&7 ) + 8; }
02234     #ifdef MALLOCDEBUG
02235     size += 2*BANNER;
02236     #endif
02237     mem = (void *)M_alloc(size);
02238     if ( mem == 0 ) {
02239         #ifndef MALLOCDEBUG
02240             MLOCK(ErrorMessageLock);
02241         #endif
02242         Error0("No memory!");
02243         #ifndef MALLOCDEBUG
02244             MUNLOCK(ErrorMessageLock);
02245         #else
02246             /* MUNLOCK(ErrorMessageLock); */
02247         #endif
02248         #ifdef MALLOCDEBUG
02249             UNLOCK(MallocLock);
02250         #endif
02251         Terminate(-1);
02252     }
02253     #ifdef MALLOCDEBUG
02254     mallocsizes[nummalloclist] = size;
02255     mallocstrings[nummalloclist] = dummymessage;
02256     malloclist[nummalloclist++] = mem;
02257     if ( filelist ) MesPrint("Mem0 at 0x%x, %l bytes",mem,size);
02258     {
02259         int i = nummalloclist-1;
02260         while ( --i >= 0 ) {
02261             if ( (char *)mem < (((char *)malloclist[i]) + mallocsizes[i])
02262                 && (char *)malloclist[i] < ((char *)mem + size) ) {
02263                 if ( filelist ) MesPrint("This memory overlaps with the block at 0x%x"
02264                     ,malloclist[i]);
02265             }
02266         }
02267     }
02268     t = (char *)mem;
02269     u = t + size;

```

```

02270     for ( i = 0; i < (int)BANNER; i++ ) { *t++ = FILLVALUE; *--u = FILLVALUE; }
02271     mem = (void *)t;
02272     {
02273         int j = nummalloclist-1, i;
02274         while ( --j >= 0 ) {
02275             t = (char *) (malloclist[j]);
02276             u = t + malloclistsizes[j];
02277             for ( i = 0; i < (int)BANNER; i++ ) {
02278                 u--;
02279                 if ( *t != FILLVALUE || *u != FILLVALUE ) {
02280                     MesPrint("Writing outside memory for %s",malloclist[i]);
02281                     /* MUNLOCK(ErrorMessageLock); */
02282                     UNLOCK(MallocLock);
02283                     Terminate(-1);
02284                 }
02285                 t--;
02286             }
02287         }
02288     }
02289     /* MUNLOCK(ErrorMessageLock); */
02290     UNLOCK(MallocLock);
02291 #endif
02292     return(mem);
02293 }
02294
02295 /*
02296     #] Malloc :
02297     #[ Malloc1 :
02298
02299     Malloc with more detailed error message.
02300     Gives the user some idea of what is happening.
02301 */
02302
02303 void *Malloc1(LONG size, const char *messageifwrong)
02304 {
02305     void *mem;
02306 #ifdef MALLOCDDEBUG
02307     char *t, *u;
02308     int i;
02309     LOCK(MallocLock);
02310     /* MLOCK(ErrorMessageLock); */
02311     if ( size == 0 ) {
02312         MesPrint("%wAsking for 0 bytes in Malloc1");
02313     }
02314 #endif
02315 #ifdef WITHSTATS
02316     nummallocs++;
02317 #endif
02318     if ( ( size & 7 ) != 0 ) { size = size - ( size&7 ) + 8; }
02319 #ifdef MALLOCDDEBUG
02320     size += 2*BANNER;
02321 #endif
02322     mem = (void *)M_alloc(size);
02323     if ( mem == 0 ) {
02324 #ifndef MALLOCDDEBUG
02325         MLOCK(ErrorMessageLock);
02326 #endif
02327         MesPrint("Attempted to allocate %l bytes.", size);
02328         Error1("No memory while allocating ", (UBYTE *)messageifwrong);
02329 #ifndef MALLOCDDEBUG
02330         MUNLOCK(ErrorMessageLock);
02331 #endif
02332         /* MUNLOCK(ErrorMessageLock); */
02333 #endif
02334 #ifdef MALLOCDDEBUG
02335         UNLOCK(MallocLock);
02336 #endif
02337         Terminate(-1);
02338     }
02339 #ifdef MALLOCDDEBUG
02340     malloclistsizes[nummalloclist] = size;
02341     malloclistsizes[nummalloclist] = (char *)messageifwrong;
02342     malloclist[nummalloclist++] = mem;
02343     if ( AC.MemDebugFlag && filelist ) MesPrint("%wMem1 at 0x%x: %l bytes.
02344 %s",mem,size,messageifwrong);
02345     {
02346         int i = nummalloclist-1;
02347         while ( --i >= 0 ) {
02348             if ( (char *)mem < ((char *)malloclist[i]) + malloclistsizes[i]
02349                 && (char *) (malloclist[i]) < ((char *)mem + size) ) {
02349                 if ( filelist ) MesPrint("This memory overlaps with the block at 0x%x"
02350 ,malloclist[i]);
02351             }
02352         }
02353     }
02354
02355 #ifdef MALLOCDDEBUGOUTPUT

```

```

02356     printf ("Malloc1: %s, allocated %li bytes at %.8lx\n",messageifwrong,size,(unsigned long)mem);
02357     fflush (stdout);
02358 #endif
02359
02360     t = (char *)mem;
02361     u = t + size;
02362     for ( i = 0; i < (int)BANNER; i++ ) { *t++ = FILLVALUE; *--u = FILLVALUE; }
02363     mem = (void *)t;
02364     M_check();
02365     /* MUNLOCK(ErrorMessageLock); */
02366     UNLOCK(MallocLock);
02367 #endif
02368 /*
02369     if ( size > 500000000L ) {
02370         MLOCK(ErrorMessageLock);
02371         MesPrint("Malloc1: %s, allocated %l bytes\n",messageifwrong,size);
02372         MUNLOCK(ErrorMessageLock);
02373     }
02374 */
02375     return(mem);
02376 }
02377
02378 /*
02379     #] Malloc1 :
02380     #[ M_free :
02381 */
02382
02383 void M_free(void *x, const char *where)
02384 {
02385     #ifdef MALLOCDDEBUG
02386     char *t = (char *)x;
02387     int i, j, k;
02388     LONG size = 0;
02389     x = (void *)(((char *)x)-BANNER);
02390     /* MLOCK(ErrorMessageLock); */
02391     if ( AC.MemDebugFlag ) MesPrint("%wFreeing 0x%x: %s",x,where);
02392     LOCK(MallocLock);
02393     for ( i = nummalloclist-1; i >= 0; i-- ) {
02394         if ( x == malloclist[i] ) {
02395             size = malloclist[i];
02396             for ( j = i+1; j < nummalloclist; j++ ) {
02397                 malloclist[j-1] = malloclist[j];
02398                 malloclist[j-1] = malloclist[j];
02399                 malloclist[j-1] = malloclist[j];
02400             }
02401             nummalloclist--;
02402             break;
02403         }
02404     }
02405     if ( i < 0 ) {
02406         unsigned int xx = ((ULONG)x);
02407         printf("Error returning non-allocated address: 0x%x from %s\n"
02408             ,xx,where);
02409         /* MUNLOCK(ErrorMessageLock); */
02410         UNLOCK(MallocLock);
02411         exit(-1);
02412     }
02413     else {
02414         for ( k = 0, j = 0; k < (int)BANNER; k++ ) {
02415             if ( *--t != FILLVALUE ) j++;
02416         }
02417         if ( j ) {
02418             LONG *tt = (LONG *)x;
02419             MesPrint("%w!!!! Banner has been written in !!!!!: %x %x %x %x",
02420                 tt[0],tt[1],tt[2],tt[3]);
02421         }
02422         t += size;
02423         for ( k = 0, j = 0; k < (int)BANNER; k++ ) {
02424             if ( *--t != FILLVALUE ) j++;
02425         }
02426         if ( j ) {
02427             LONG *tt = (LONG *)x;
02428             MesPrint("%w!!!! Tail has been written in !!!!!: %x %x %x %x",
02429                 tt[0],tt[1],tt[2],tt[3]);
02430         }
02431         M_check();
02432         /* MUNLOCK(ErrorMessageLock); */
02433         UNLOCK(MallocLock);
02434     }
02435 #else
02436     DUMMYUSE(where);
02437 #endif
02438 #ifdef WITHSTATS
02439     numfrees++;
02440 #endif
02441     if ( x ) {
02442     #ifdef MALLOCDDEBUGOUTPUT

```



```

02443         printf ("M_free: %s, memory freed at %.8lx\n",where,(unsigned long)x);
02444         fflush(stdout);
02445     #endif
02446
02447     #ifdef MALLOCPROTECT
02448         mprotectFree((void *)x);
02449     #else
02450         free(x);
02451     #endif
02452 }
02453 }
02454
02455 /*
02456     #] M_free :
02457     #[ M_check :
02458 */
02459
02460 #ifdef MALLOCDEBUG
02461
02462 void M_check1() { MesPrint("Checking Malloc"); M_check(); }
02463
02464 void M_check()
02465 {
02466     int i,j,k,error = 0;
02467     char *t;
02468     LONG *tt;
02469     for ( i = 0; i < nummalloclist; i++ ) {
02470         t = (char *) (malloclist[i]);
02471         for ( k = 0, j = 0; k < (int)BANNER; k++ ) {
02472             if ( *t++ != FILLVALUE ) j++;
02473         }
02474         if ( j ) {
02475             tt = (LONG *) (malloclist[i]);
02476             MesPrint("%w!!!! Banner %d (%s) has been written in !!!!!: %x %x %x %x",
02477                 i,mallocstrings[i],tt[0],tt[1],tt[2],tt[3]);
02478             tt[0] = tt[1] = tt[2] = tt[3] = 0;
02479             error = 1;
02480         }
02481         t = (char *) (malloclist[i]) + malloccsizes[i];
02482         for ( k = 0, j = 0; k < (int)BANNER; k++ ) {
02483             if ( *--t != FILLVALUE ) j++;
02484         }
02485         if ( j ) {
02486             tt = (LONG *)t;
02487             MesPrint("%w!!!! Tail %d (%s) has been written in !!!!!: %x %x %x %x",
02488                 i,mallocstrings[i],tt[0],tt[1],tt[2],tt[3]);
02489             tt[0] = tt[1] = tt[2] = tt[3] = 0;
02490             error = 1;
02491         }
02492         if ( ( mallocstrings[i][0] == ' ' ) || ( mallocstrings[i][0] == '#' ) ) {
02493             MesPrint("%w!!!! Funny mallocstring");
02494             error = 1;
02495         }
02496     }
02497     if ( error ) {
02498         M_print();
02499         /* MUNLOCK(ErrorMessageLock); */
02500         UNLOCK(MallocLock);
02501         Terminate(-1);
02502     }
02503 }
02504
02505 void M_print()
02506 {
02507     int i;
02508     MesPrint("We have the following memory allocations left:");
02509     for ( i = 0; i < nummalloclist; i++ ) {
02510         MesPrint("0x%x: %l bytes. number %d: '%s'",malloclist[i],malloccsizes[i],i,mallocstrings[i]);
02511     }
02512 }
02513
02514 #else
02515
02516 void M_check1(void) {}
02517 void M_print(void) {}
02518
02519 #endif
02520
02521 /*
02522     #] M_check :
02523     #[ TermMalloc :
02524 */
02547 #define TERMMEMSTARTNUM 16
02548 #define TERMEXTRAWORDS 10
02549
02550 void TermMallocAddMemory (PHEAD0)
02551 {

```

```

02552     WORD *newbufs;
02553     int i, extra;
02554     if ( AT.TermMemMax == 0 ) extra = TERMMEMSTARTNUM;
02555     else extra = AT.TermMemMax;
02556     if ( AT.TermMemHeap ) M_free(AT.TermMemHeap,"TermMalloc");
02557     newbufs = (WORD *)M_malloc1(extra*(AM.MaxTer+TERMEXTRAWORDS*sizeof(WORD)), "TermMalloc");
02558     AT.TermMemHeap = (WORD **)M_malloc1((extra+AT.TermMemMax)*sizeof(WORD *), "TermMalloc");
02559     for ( i = 0; i < extra; i++ ) {
02560         AT.TermMemHeap[i] = newbufs + i*(AM.MaxTer/sizeof(WORD)+TERMEXTRAWORDS);
02561     }
02562 #ifdef TERMMALLOCDEBUG
02563     DebugHeap2 = (WORD **)M_malloc1((extra+AT.TermMemMax)*sizeof(WORD *), "TermMalloc");
02564     for ( i = 0; i < AT.TermMemMax; i++ ) { DebugHeap2[i] = DebugHeap1[i]; }
02565     for ( i = 0; i < extra; i++ ) {
02566         DebugHeap2[i+AT.TermMemMax] = newbufs + i*(AM.MaxTer/sizeof(WORD)+TERMEXTRAWORDS);
02567     }
02568     if ( DebugHeap1 ) M_free(DebugHeap1, "TermMalloc");
02569     DebugHeap1 = DebugHeap2;
02570 #endif
02571     AT.TermMemTop = extra;
02572     AT.TermMemMax += extra;
02573 #ifdef TERMMALLOCDEBUG
02574     MesPrint("AT.TermMemMax is now %l", AT.TermMemMax);
02575 #endif
02576 }
02577
02578 #ifndef MEMORYMACROS
02579
02580 WORD *TermMalloc2(PHEAD char *text)
02581 {
02582     if ( AT.TermMemTop <= 0 ) TermMallocAddMemory(BHEAD0);
02583
02584 #ifdef TERMMALLOCDEBUG
02585     MesPrint("TermMalloc: %s, %d", text, (AT.TermMemMax-AT.TermMemTop));
02586 #endif
02587
02588 #ifdef MALLOCDEBUGOUTPUT
02589     MesPrint("TermMalloc: %s, %l/%l (%x)", text, AT.TermMemTop, AT.TermMemMax, AT.TermMemHeap[AT.TermMemTop-1]);
02590 #endif
02591
02592     DUMMYUSE(text);
02593     return (AT.TermMemHeap[--AT.TermMemTop]);
02594 }
02595
02596 void TermFree2(PHEAD WORD *TermMem, char *text)
02597 {
02598 #ifdef TERMMALLOCDEBUG
02599
02600     int i;
02601
02602     for ( i = 0; i < AT.TermMemMax; i++ ) {
02603         if ( TermMem == DebugHeap1[i] ) break;
02604     }
02605     if ( i >= AT.TermMemMax ) {
02606         MesPrint(" ERROR: TermFree called with an address not given by TermMalloc.");
02607         Terminate(-1);
02608     }
02609 #endif
02610     DUMMYUSE(text);
02611     AT.TermMemHeap[AT.TermMemTop++] = TermMem;
02612
02613 #ifdef TERMMALLOCDEBUG
02614     MesPrint("TermFree: %s, %d", text, (AT.TermMemMax-AT.TermMemTop));
02615 #endif
02616 #ifdef MALLOCDEBUGOUTPUT
02617     MesPrint("TermFree: %s, %l/%l (%x)", text, AT.TermMemTop, AT.TermMemMax, TermMem);
02618 #endif
02619 }
02620
02621 #endif
02622
02623 /*
02624     #] TermMalloc :
02625     #[ NumberMalloc :
02626 */
02647 #define NUMBERMEMSTARTNUM 16
02648 #define NUMBEREXTRAWORDS 10L
02649
02650 #ifdef TERMMALLOCDEBUG
02651 UWORD **DebugHeap3, **DebugHeap4;
02652 #endif
02653
02654 void NumberMallocAddMemory(PHEAD0)
02655 {
02656     UWORD *newbufs;
02657     WORD extra;

```

```

02658     int i;
02659     if ( AT.NumberMemMax == 0 ) extra = NUMBERMEMSTARTNUM;
02660     else extra = AT.NumberMemMax;
02661     if ( AT.NumberMemHeap ) M_free(AT.NumberMemHeap,"NumberMalloc");
02662     newbufs = (UWORD *)Malloc1(extra*(AM.MaxTal+NUMBEREXTRAWORDS)*sizeof(UWORD),"NumberMalloc");
02663     AT.NumberMemHeap = (UWORD **)Malloc1((extra+AT.NumberMemMax)*sizeof(UWORD *),"NumberMalloc");
02664     for ( i = 0; i < extra; i++ ) {
02665         AT.NumberMemHeap[i] = newbufs + i*(LONG) (AM.MaxTal+NUMBEREXTRAWORDS);
02666     }
02667 #ifdef TERMMALLOCDEBUG
02668     DebugHeap4 = (UWORD **)Malloc1((extra+AT.NumberMemMax)*sizeof(WORD *),"NumberMalloc");
02669     for ( i = 0; i < AT.NumberMemMax; i++ ) { DebugHeap4[i] = DebugHeap3[i]; }
02670     for ( i = 0; i < extra; i++ ) {
02671         DebugHeap4[i+AT.NumberMemMax] = newbufs + i*(LONG) (AM.MaxTal+NUMBEREXTRAWORDS);
02672     }
02673     if ( DebugHeap3 ) M_free(DebugHeap3,"NumberMalloc");
02674     DebugHeap3 = DebugHeap4;
02675 #endif
02676     AT.NumberMemTop = extra;
02677     AT.NumberMemMax += extra;
02678 /*
02679 MesPrint("AT.NumberMemMax is now %l",AT.NumberMemMax);
02680 */
02681 }
02682
02683 #ifndef MEMORYMACROS
02684
02685 UWORD *NumberMalloc2(PHEAD char *text)
02686 {
02687     if ( AT.NumberMemTop <= 0 ) NumberMallocAddMemory(BHEAD text);
02688
02689 #ifdef MALLOCDEBUGOUTPUT
02690     if ( (AT.NumberMemMax-AT.NumberMemTop) > 10 )
02691         MesPrint("NumberMalloc: %s, %l/%l
02692 (%x)",text,AT.NumberMemTop,AT.NumberMemMax,AT.NumberMemHeap[AT.NumberMemTop-1]);
02692 #endif
02693
02694     DUMMYUSE(text);
02695     return (AT.NumberMemHeap[--AT.NumberMemTop]);
02696 }
02697
02698 void NumberFree2(PHEAD UWORD *NumberMem, char *text)
02699 {
02700 #ifdef TERMMALLOCDEBUG
02701     int i;
02702     for ( i = 0; i < AT.NumberMemMax; i++ ) {
02703         if ( NumberMem == DebugHeap3[i] ) break;
02704     }
02705     if ( i >= AT.NumberMemMax ) {
02706         MesPrint(" ERROR: NumberFree called with an address not given by NumberMalloc.");
02707         Terminate(-1);
02708     }
02709 #endif
02710     DUMMYUSE(text);
02711     AT.NumberMemHeap[AT.NumberMemTop++] = NumberMem;
02712
02713 #ifdef MALLOCDEBUGOUTPUT
02714     if ( (AT.NumberMemMax-AT.NumberMemTop) > 10 )
02715         MesPrint("NumberFree: %s, %l/%l (%x)",text,AT.NumberMemTop,AT.NumberMemMax,NumberMem);
02716 #endif
02717 }
02718
02719 #endif
02720
02721 /*
02722     #] NumberMalloc :
02723     #[ CacheNumberMalloc :
02724
02725     Similar to NumberMalloc
02726 */
02727
02728 void CacheNumberMallocAddMemory(PHEAD0)
02729 {
02730     UWORD *newbufs;
02731     WORD extra;
02732     int i;
02733     if ( AT.CacheNumberMemMax == 0 ) extra = NUMBERMEMSTARTNUM;
02734     else extra = AT.CacheNumberMemMax;
02735     if ( AT.CacheNumberMemHeap ) M_free(AT.CacheNumberMemHeap,"NumberMalloc");
02736     newbufs = (UWORD *)Malloc1(extra*(AM.MaxTal+NUMBEREXTRAWORDS)*sizeof(UWORD),"CacheNumberMalloc");
02737     AT.CacheNumberMemHeap = (UWORD **)Malloc1((extra+AT.NumberMemMax)*sizeof(UWORD
02738 *),"CacheNumberMalloc");
02739     for ( i = 0; i < extra; i++ ) {
02740         AT.CacheNumberMemHeap[i] = newbufs + i*(LONG) (AM.MaxTal+NUMBEREXTRAWORDS);
02741     }
02742     AT.CacheNumberMemTop = extra;
02743     AT.CacheNumberMemMax += extra;

```

```

02743 }
02744
02745 #ifndef MEMORYMACROS
02746
02747 UWORD *CacheNumberMalloc2(PHEAD char *text)
02748 {
02749     if ( AT.CacheNumberMemTop <= 0 ) CacheNumberMallocAddMemory(BHEAD0);
02750
02751 #ifdef MALLOCDEBUGOUTPUT
02752     MesPrint("NumberMalloc: %s, %l/%l
02753             (%x)",text,AT.NumberMemTop,AT.NumberMemMax,AT.NumberMemHeap[AT.NumberMemTop-1]);
02754 #endif
02755     DUMMYUSE(text);
02756     return(AT.CacheNumberMemHeap[--AT.CacheNumberMemTop]);
02757 }
02758
02759 void CacheNumberFree2(PHEAD UWORD *NumberMem, char *text)
02760 {
02761     DUMMYUSE(text);
02762     AT.CacheNumberMemHeap[AT.CacheNumberMemTop++] = NumberMem;
02763
02764 #ifdef MALLOCDEBUGOUTPUT
02765     MesPrint("NumberFree: %s, %l/%l (%x)",text,AT.NumberMemTop,AT.NumberMemMax,NumberMem);
02766 #endif
02767 }
02768
02769 #endif
02770
02771 /*
02772     #] CacheNumberMalloc :
02773     #[ FromList :
02774
02775     Returns the next object in a list.
02776     If the list has been exhausted we double it (like a realloc)
02777     If the list has not been initialized yet we start with 12 elements.
02778 */
02779 void *FromList(LIST *L)
02780 {
02781     void *newlist;
02782     int i, *old, *newL;
02783     if ( L->num >= L->maxnum || L->lijst == 0 ) {
02784         if ( L->maxnum == 0 ) L->maxnum = 12;
02785         else if ( L->lijst ) L->maxnum *= 2;
02786         newlist = Malloc1(L->maxnum * L->size,L->message);
02787         if ( L->lijst ) {
02788             i = ( L->num * L->size ) / sizeof(int);
02789             old = (int *)L->lijst; newL = (int *)newlist;
02790             while ( --i >= 0 ) *newL++ = *old++;
02791             if ( L->lijst ) M_free(L->lijst,"L->lijst FromList");
02792         }
02793         L->lijst = newlist;
02794     }
02795     return( ((char *) (L->lijst)) + L->size * (L->num)++ );
02796 }
02797
02798 /*
02799     #] FromList :
02800     #[ From0List :
02801
02802     Same as FromList, but we zero excess variables.
02803 */
02804 void *From0List(LIST *L)
02805 {
02806     void *newlist;
02807     int i, *old, *newL;
02808     if ( L->num >= L->maxnum || L->lijst == 0 ) {
02809         if ( L->maxnum == 0 ) L->maxnum = 12;
02810         else if ( L->lijst ) L->maxnum *= 2;
02811         newlist = Malloc1(L->maxnum * L->size,L->message);
02812         i = ( L->num * L->size ) / sizeof(int);
02813         old = (int *) (L->lijst); newL = (int *) newlist;
02814         while ( --i >= 0 ) *newL++ = *old++;
02815         i = ( L->maxnum - L->num ) / sizeof(int);
02816         while ( --i >= 0 ) *newL++ = 0;
02817         if ( L->lijst ) M_free(L->lijst,"L->lijst From0List");
02818         L->lijst = newlist;
02819     }
02820     return( ((char *) (L->lijst)) + L->size * (L->num)++ );
02821 }
02822
02823 /*
02824     #] From0List :
02825     #[ FromVarList :
02826
02827
02828

```

```

02829     Returns the next object in a list of variables.
02830     If the list has been exhausted we double it (like a realloc)
02831     If the list has not been initialized yet we start with 12 elements.
02832     We allow at most MAXVARIABLES elements!
02833  */
02834
02835 void *FromVarList(LIST *L)
02836 {
02837     void *newlist;
02838     int i, *old, *newL;
02839     if ( L->num >= L->maxnum || L->lijst == 0 ) {
02840         if ( L->maxnum == 0 ) L->maxnum = 12;
02841         else if ( L->lijst ) {
02842             L->maxnum *= 2;
02843             if ( L == &(AP.DollarList) ) {
02844                 if ( L->maxnum > MAXDOLLARVARIABLES ) L->maxnum = MAXDOLLARVARIABLES;
02845                 if ( L->num >= MAXDOLLARVARIABLES ) {
02846                     MesPrint("!!!More than %1 objects in list of $-variables",
02847                             MAXDOLLARVARIABLES);
02848                     Terminate(-1);
02849                 }
02850             }
02851             else {
02852                 if ( L->maxnum > MAXVARIABLES ) L->maxnum = MAXVARIABLES;
02853                 if ( L->num >= MAXVARIABLES ) {
02854                     MesPrint("!!!More than %1 objects in list of variables",
02855                             MAXVARIABLES);
02856                     Terminate(-1);
02857                 }
02858             }
02859             newlist = Malloc1(L->maxnum * L->size, L->message);
02860             if ( L->lijst ) {
02861                 i = ( L->num * L->size ) / sizeof(int);
02862                 old = (int *) (L->lijst); newL = (int *) newlist;
02863                 while ( --i >= 0 ) *newL++ = *old++;
02864                 if ( L->lijst ) M_free(L->lijst, "L->lijst from VarList");
02865             }
02866             L->lijst = newlist;
02867         }
02868     }
02869     return( ((char *) (L->lijst)) + L->size * ((L->num)++) );
02870 }
02871
02872 /*
02873     #] FromVarList :
02874     #[ DoubleList :
02875  */
02876
02877 int DoubleList(void ***lijst, int *oldsize, int objectsize, char *nameoftype)
02878 {
02879     void **newlist;
02880     LONG i, newsize, fullsize;
02881     void **to, **from;
02882     static LONG maxlistsize = (LONG) (MAXPOSITIVE);
02883     if ( *lijst == 0 ) {
02884         if ( *oldsize > 0 ) newsize = *oldsize;
02885         else newsize = 100;
02886     }
02887     else newsize = *oldsize * 2;
02888     if ( newsize > maxlistsize ) {
02889         if ( *oldsize == maxlistsize ) {
02890             MesPrint("No memory for extra space in %s", nameoftype);
02891             return(-1);
02892         }
02893         newsize = maxlistsize;
02894     }
02895     fullsize = ( newsize * objectsize + sizeof(void *) - 1 ) & (~sizeof(void *));
02896     newlist = (void **) Malloc1(fullsize, nameoftype);
02897     if ( *lijst ) { /* Now some punning. DANGEROUS CODE in principle */
02898         to = newlist; from = *lijst; i = (*oldsize * objectsize) / sizeof(void *);
02899     }
02900     #ifdef MALLOCDEBUG
02901     if ( filelist ) MesPrint("    oldsize: %1, objectsize: %d, fullsize: %1"
02902                             , *oldsize, objectsize, fullsize);
02903     #endif
02904     /*
02905         while ( --i >= 0 ) *to++ = *from++;
02906     */
02907     if ( *lijst ) M_free(*lijst, "DoubleLList");
02908     *lijst = newlist;
02909     *oldsize = newsize;
02910     return(0);
02911 }
02912
02913 int error;
02914 LONG lsize = *oldsize;
02915 maxlistsize = (LONG) (MAXPOSITIVE);

```

```

02916     error = DoubleLinkedList(lijst,&lsize,objectsize,nameoftype);
02917     *oldsize = lsize;
02918     maxlistsize = (LONG)(MAXLONG);
02919
02920     return(error);
02921 */
02922 }
02923
02924 /*
02925     #] DoubleList :
02926     #[ DoubleLinkedList :
02927 */
02928
02929 int DoubleLinkedList(void ***lijst, LONG *oldsize, int objectsize, char *nameoftype)
02930 {
02931     void **newlist;
02932     LONG i, newsize, fullsize;
02933     void **to, **from;
02934     static LONG maxlistsize = (LONG)(MAXLONG);
02935     if ( *lijst == 0 ) {
02936         if ( *oldsize > 0 ) newsize = *oldsize;
02937         else newsize = 100;
02938     }
02939     else newsize = *oldsize * 2;
02940     if ( newsize > maxlistsize ) {
02941         if ( *oldsize == maxlistsize ) {
02942             MesPrint("No memory for extra space in %s",nameoftype);
02943             return(-1);
02944         }
02945         newsize = maxlistsize;
02946     }
02947     fullsize = ( newsize * objectsize + sizeof(void *)-1 ) & (~sizeof(void *));
02948     newlist = (void **)Malloca1(fullsize,nameoftype);
02949     if ( *lijst ) { /* Now some punning. DANGEROUS CODE in principle */
02950         to = newlist; from = *lijst; i = (*oldsize * objectsize)/sizeof(void *);
02951     }
02952     #ifdef MALLOCDEBUG
02953     if ( filelist ) MesPrint("    oldsize: %l, objectsize: %d, fullsize: %l"
02954         ,*oldsize,objectsize,fullsize);
02955     #endif
02956     /*
02957         while ( --i >= 0 ) *to++ = *from++;
02958     */
02959     if ( *lijst ) M_free(*lijst,"DoubleLinkedList");
02960     *lijst = newlist;
02961     *oldsize = newsize;
02962     return(0);
02963 }
02964
02965 /*
02966     #] DoubleLinkedList :
02967     #[ DoubleBuffer :
02968 */
02969
02970 #define DODOUBLE(x) { x *s, *t, *u; if ( *start ) { \
02971     oldsize = *(x **)stop - *(x **)start; newsize = 2*oldsize; \
02972     t = u = (x *)Malloca1(newsize*sizeof(x),text); s = *(x **)start; \
02973     for ( i = 0; i < oldsize; i++ ) { *t++ = *s++; } M_free(*start,"double"); } \
02974     else { newsize = 100; u = (x *)Malloca1(newsize*sizeof(x),text); } \
02975     *start = (void *)u; *stop = (void *) (u+newsize); }
02976
02977 void DoubleBuffer(void **start, void **stop, int size, char *text)
02978 {
02979     LONG oldsize, newsize, i;
02980     if ( size == sizeof(char) ) DODOUBLE(char)
02981     else if ( size == sizeof(short) ) DODOUBLE(short)
02982     else if ( size == sizeof(int) ) DODOUBLE(int)
02983     else if ( size == sizeof(LONG) ) DODOUBLE(LONG)
02984     else if ( size % sizeof(int) == 0 ) DODOUBLE(int)
02985     else {
02986         MesPrint("---Cannot handle doubling buffers of size %d",size);
02987         Terminate(-1);
02988     }
02989 }
02990
02991 /*
02992     #] DoubleBuffer :
02993     #[ ExpandBuffer :
02994 */
02995
02996 #define DOEXPAND(x) { x *newbuffer, *t, *m; \
02997     t = newbuffer = (x *)Malloca1((newsize+2)*type,"ExpandBuffer"); \
02998     if ( *buffer ) { m = (x *)*buffer; i = *oldsize; \
02999         while ( --i >= 0 ) { *t++ = *m++; } M_free(*buffer,"ExpandBuffer"); \
03000     } *buffer = newbuffer; *oldsize = newsize; }
03001
03002 void ExpandBuffer(void **buffer, LONG *oldsize, int type)

```

```

03003 {
03004     LONG newsize, i;
03005     if ( *oldsize <= 0 ) { newsize = 100; }
03006     else newsize = 2*(*oldsize);
03007     if ( type == sizeof(char) ) DOEXPAND(char)
03008     else if ( type == sizeof(short) ) DOEXPAND(short)
03009     else if ( type == sizeof(int) ) DOEXPAND(int)
03010     else if ( type == sizeof(LONG) ) DOEXPAND(LONG)
03011     else if ( type == sizeof(POSITION) ) DOEXPAND(POSITION)
03012     else {
03013         MesPrint("---Cannot handle expanding buffers with objects of size %d",type);
03014         Terminate(-1);
03015     }
03016 }
03017
03018 /*
03019     #] ExpandBuffer :
03020     #[ iexp :
03021
03022     Raises the long integer y to the power p.
03023     Returnvalue is long, regardless of overflow.
03024 */
03025
03026 LONG iexp(LONG x, int p)
03027 {
03028     int sign;
03029     ULONG y;
03030     ULONG ux;
03031     if ( x == 0 ) return(0);
03032     if ( p == 0 ) return(1);
03033     sign = x < 0 ? -1 : 1;
03034     if ( sign < 0 && ( p & 1 ) == 0 ) sign = 1;
03035     ux = LongAbs(x);
03036     if ( ux == 1 ) return(sign);
03037     if ( p < 0 ) return(0);
03038     y = 1;
03039     while ( p ) {
03040         if ( ( p & 1 ) != 0 ) y *= ux;
03041         p >>= 1;
03042         ux = ux*ux;
03043     }
03044     if ( sign < 0 ) y = -y;
03045     return ULONGToLong(y);
03046 }
03047
03048 /*
03049     #] iexp :
03050     #[ ToGeneral :
03051
03052     Convert a fast argument to a general argument
03053     Input in r, output in m.
03054     If par == 0 we need the argument header also.
03055 */
03056
03057 void ToGeneral(WORD *r, WORD *m, WORD par)
03058 {
03059     WORD *mm = m, j, k;
03060     if ( par ) m++;
03061     else { m[1] = 0; m += ARGHEAD + 1; }
03062     j = -*r++;
03063     k = 3;
03064     /* JV: Bugfix 1-feb-2016. Old code assumed FUNHEAD to be 2 */
03065     if ( j >= FUNCTION ) { *m++ = j; *m++ = FUNHEAD; FILLFUN(m) }
03066     else {
03067         switch ( j ) {
03068             case SYMBOL: *m++ = j; *m++ = 4; *m++ = *r++; *m++ = 1; break;
03069             case SNUMBER:
03070                 if ( *r > 0 ) { *m++ = *r; *m++ = 1; *m++ = 3; }
03071                 else if ( *r == 0 ) { m--; }
03072                 else { *m++ = -*r; *m++ = 1; *m++ = -3; }
03073                 goto MakeSize;
03074             case MINVECTOR:
03075                 k = -k;
03076                 /* fall through */
03077             case INDEX:
03078             case VECTOR:
03079                 *m++ = INDEX; *m++ = 3; *m++ = *r++;
03080                 break;
03081         }
03082     }
03083     *m++ = 1; *m++ = 1; *m++ = k;
03084 MakeSize:
03085     *mm = m-mm;
03086     if ( !par ) mm[ARGHEAD] = *mm-ARGHEAD;
03087 }
03088
03089 /*

```

```

03090      #] ToGeneral :
03091      #[ ToFast :
03092
03093      Checks whether an argument can be converted to fast notation
03094      If this can be done it does it.
03095      Important: m should be allowed to be equal to r!
03096      Return value is 1 if conversion took place.
03097      If there was conversion the answer is in m.
03098      If there was no conversion m hasn't been touched.
03099  */
03100
03101  int ToFast(WORD *r, WORD *m)
03102  {
03103      WORD i;
03104      if ( *r == ARGHEAD ) { *m++ = -SNUMBER; *m++ = 0; return(1); }
03105      if ( *r != r[ARGHEAD]+ARGHEAD ) return(0); /* > 1 term */
03106      r += ARGHEAD;
03107      if ( *r == 4 ) {
03108          if ( r[2] != 1 || r[1] <= 0 ) return(0);
03109          *m++ = -SNUMBER; *m = ( r[3] < 0 ) ? -r[1] : r[1]; return(1);
03110      }
03111      i = *r - 1;
03112      if ( r[i-1] != 1 || r[i-2] != 1 ) return(0);
03113      if ( r[i] != 3 ) {
03114          if ( r[i] == -3 && r[2] == *r-4 && r[2] == 3 && r[1] == INDEX
03115              && r[3] < MINSPEC ) {}
03116          else return(0);
03117      }
03118      else if ( r[2] != *r - 4 ) return(0);
03119      r++;
03120      if ( *r >= FUNCTION ) {
03121          if ( r[1] <= FUNHEAD ) { *m++ = -*r; return(1); }
03122      }
03123      else if ( *r == SYMBOL ) {
03124          if ( r[1] == 4 && r[3] == 1 )
03125              { *m++ = -SYMBOL; *m++ = r[2]; return(1); }
03126      }
03127      else if ( *r == INDEX ) {
03128          if ( r[1] == 3 ) {
03129              if ( r[2] >= MINSPEC ) {
03130                  if ( r[2] >= 0 && r[2] < AM.OffsetIndex ) *m++ = -SNUMBER;
03131                  else *m++ = -INDEX;
03132              }
03133              else {
03134                  if ( r[5] == -3 ) *m++ = -MINVECTOR;
03135                  else *m++ = -VECTOR;
03136              }
03137              *m++ = r[2];
03138              return(1);
03139          }
03140      }
03141      return(0);
03142  }
03143  /*
03144  #] ToFast :
03145  #[ ToPolyFunGeneral :
03146
03147      Routine forces a polyratfun into general notation if needed.
03148      If no action was needed, the return value is zero.
03149      A positive return value indicates how many arguments were converted.
03150      The new term overwrite the old.
03151  */
03152  /*
03153
03154  WORD ToPolyFunGeneral(PHEAD WORD *term)
03155  {
03156      WORD *t = term+1, *tt, *to, *tol, *termout, *tstop, *tnext;
03157      WORD numarg, i, change = 0;
03158      tstop = term + *term; tstop -= ABS(tstop[-1]);
03159      termout = to = AT.WorkPointer;
03160      to++;
03161      while ( t < tstop ) { /* go through the subterms */
03162          if ( *t == AR.PolyFun ) {
03163              tt = t+FUNHEAD; tnext = t + t[1];
03164              numarg = 0;
03165              while ( tt < tnext ) { numarg++; NEXTARG(tt); }
03166              if ( numarg == 2 ) { /* this needs attention */
03167                  tt = t + FUNHEAD;
03168                  tol = to;
03169                  i = FUNHEAD; NCOPY(to,t,i);
03170                  while ( tt < tnext ) { /* Do the arguments */
03171                      if ( *tt > 0 ) {
03172                          i = *tt; NCOPY(to,tt,i);
03173                      }
03174                      else if ( *tt == -SYMBOL ) {
03175                          tol[1] += 6+ARGHEAD; tol[2] |= MUSTCLEANPRF; change++;
03176                          *to++ = 8+ARGHEAD; *to++ = 0; FILLARG(to);

```



```

03177         *to++ = 8; *to++ = SYMBOL; *to++ = 4; *to++ = tt[1];
03178         *to++ = 1; *to++ = 1; *to++ = 1; *to++ = 3;
03179         tt += 2;
03180     }
03181     else if ( *tt == -SNUMBER ) {
03182         if ( tt[1] > 0 ) {
03183             tol[1] += 2*ARGHEAD; tol[2] |= MUSTCLEANPRF; change++;
03184             *to++ = 4*ARGHEAD; *to++ = 0; FILLARG(to);
03185             *to++ = 4; *to++ = tt[1]; *to++ = 1; *to++ = 3;
03186             tt += 2;
03187         }
03188         else if ( tt[1] < 0 ) {
03189             tol[1] += 2*ARGHEAD; tol[2] |= MUSTCLEANPRF; change++;
03190             *to++ = 4*ARGHEAD; *to++ = 0; FILLARG(to);
03191             *to++ = 4; *to++ = -tt[1]; *to++ = 1; *to++ = -3;
03192             tt += 2;
03193         }
03194         else {
03195             MLOCK(ErrorMessageLock);
03196             MesPrint("Internal error: Zero in PolyRatFun");
03197             MUNLOCK(ErrorMessageLock);
03198             Terminate(-1);
03199         }
03200     }
03201 }
03202 t = tnext;
03203 continue;
03204 }
03205 }
03206 i = t[1]; NCOPY(to,t,i)
03207 }
03208 if ( change ) {
03209     tt = term + *term;
03210     while ( t < tt ) *to++ = *t++;
03211     *termout = to - termout;
03212     t = term; i = *termout; tt = termout;
03213     NCOPY(t,tt,i)
03214     AT.WorkPointer = term + *term;
03215 }
03216 return(change);
03217 }
03218
03219 /*
03220     #] ToPolyFunGeneral :
03221     #[ IsLikeVector :
03222
03223     Routine determines whether a function argument is like a vector.
03224     Returnvalue: 1: is vector or index
03225                 0: is not vector or index
03226                 -1: may be an index
03227 */
03228
03229 int IsLikeVector(WORD *arg)
03230 {
03231     WORD *sstop, *t, *tstop;
03232     if ( *arg < 0 ) {
03233         if ( *arg == -VECTOR || *arg == -INDEX ) return(1);
03234         if ( *arg == -SNUMBER && arg[1] >= 0 && arg[1] < AM.OffsetIndex )
03235             return(-1);
03236         return(0);
03237     }
03238     sstop = arg + *arg; arg += ARGHEAD;
03239     while ( arg < sstop ) {
03240         t = arg + *arg;
03241         tstop = t - ABS(t[-1]);
03242         arg++;
03243         while ( arg < tstop ) {
03244             if ( *arg == INDEX ) return(1);
03245             arg += arg[1];
03246         }
03247         arg = t;
03248     }
03249     return(0);
03250 }
03251
03252 /*
03253     #] IsLikeVector :
03254     #[ AreArgsEqual :
03255 */
03256
03257 int AreArgsEqual(WORD *arg1, WORD *arg2)
03258 {
03259     int i;
03260     if ( *arg2 != *arg1 ) return(0);
03261     if ( *arg1 > 0 ) {
03262         i = *arg1;
03263         while ( --i > 0 ) { if ( arg1[i] != arg2[i] ) return(0); }

```

```

03264         return(1);
03265     }
03266     else if ( *arg1 <= -FUNCTION ) return(1);
03267     else if ( arg1[1] == arg2[1] ) return(1);
03268     return(0);
03269 }
03270
03271 /*
03272     #] AreArgsEqual :
03273     #[ CompareArgs :
03274 */
03275
03276 int CompareArgs(WORD *arg1, WORD *arg2)
03277 {
03278     int i1,i2;
03279     if ( *arg1 > 0 ) {
03280         if ( *arg2 < 0 ) return(-1);
03281         i1 = *arg1-ARGHEAD; arg1 += ARGHEAD;
03282         i2 = *arg2-ARGHEAD; arg2 += ARGHEAD;
03283         while ( i1 > 0 && i2 > 0 ) {
03284             if ( *arg1 != *arg2 ) return((int) (*arg1)-(int) (*arg2));
03285             i1--; i2--; arg1++; arg2++;
03286         }
03287         return(i1-i2);
03288     }
03289     else if ( *arg2 > 0 ) return(1);
03290     else {
03291         if ( *arg1 != *arg2 ) {
03292             if ( *arg1 < *arg2 ) return(-1);
03293             else return(1);
03294         }
03295         if ( *arg1 <= -FUNCTION ) return(0);
03296         return((int) (arg1[1])-(int) (arg2[1]));
03297     }
03298 }
03299
03300 /*
03301     #] CompareArgs :
03302     #[ CompArg :
03303
03304     returns 1 if arg1 comes first, -1 if arg2 comes first, 0 if equal
03305 */
03306
03307 int CompArg(WORD *s1, WORD *s2)
03308 {
03309     GETIDENTITY
03310     WORD *st1, *st2, x[7];
03311     int k;
03312     if ( *s1 < 0 ) {
03313         if ( *s2 < 0 ) {
03314             if ( *s1 <= -FUNCTION && *s2 <= -FUNCTION ) {
03315                 if ( *s1 > *s2 ) return(-1);
03316                 if ( *s1 < *s2 ) return(1);
03317                 return(0);
03318             }
03319             if ( *s1 > *s2 ) return(1);
03320             if ( *s1 < *s2 ) return(-1);
03321             if ( *s1 <= -FUNCTION ) return(0);
03322             s1++; s2++;
03323             if ( *s1 > *s2 ) return(1);
03324             if ( *s1 < *s2 ) return(-1);
03325             return(0);
03326         }
03327         x[1] = AT.comsym[3];
03328         x[2] = AT.comnum[1];
03329         x[3] = AT.comnum[3];
03330         x[4] = AT.comind[3];
03331         x[5] = AT.comind[6];
03332         x[6] = AT.comfun[1];
03333         if ( *s1 == -SYMBOL ) {
03334             AT.comsym[3] = s1[1];
03335             st1 = AT.comsym+8; s1 = AT.comsym;
03336         }
03337         else if ( *s1 == -SNUMBER ) {
03338             if ( s1[1] < 0 ) {
03339                 AT.comnum[1] = -s1[1]; AT.comnum[3] = -3;
03340             }
03341             else {
03342                 AT.comnum[1] = s1[1]; AT.comnum[3] = 3;
03343             }
03344             st1 = AT.comnum+4;
03345             s1 = AT.comnum;
03346         }
03347         else if ( *s1 == -INDEX || *s1 == -VECTOR ) {
03348             AT.comind[3] = s1[1]; AT.comind[6] = 3;
03349             st1 = AT.comind+7; s1 = AT.comind;
03350         }

```

```

03351     else if ( *s1 == -MINVECTOR ) {
03352         AT.comind[3] = s1[1]; AT.comind[6] = -3;
03353         st1 = AT.comind+7; s1 = AT.comind;
03354     }
03355     else if ( *s1 <= -FUNCTION ) {
03356         AT.comfun[1] = -*s1;
03357         st1 = AT.comfun+FUNHEAD+4; s1 = AT.comfun;
03358     }
03359 /*
03360     Symmetrize during compilation of id statement when properorder
03361     needs this one. Code added 10-nov-2001
03362 */
03363     else if ( *s1 == -ARGWILD ) {
03364         return(-1);
03365     }
03366     else { goto argerror; }
03367     st2 = s2 + *s2; s2 += ARGHEAD;
03368     goto docompare;
03369 }
03370 else if ( *s2 < 0 ) {
03371     x[1] = AT.comsym[3];
03372     x[2] = AT.comnum[1];
03373     x[3] = AT.comnum[3];
03374     x[4] = AT.comind[3];
03375     x[5] = AT.comind[6];
03376     x[6] = AT.comfun[1];
03377     if ( *s2 == -SYMBOL ) {
03378         AT.comsym[3] = s2[1];
03379         st2 = AT.comsym+8; s2 = AT.comsym;
03380     }
03381     else if ( *s2 == -SNUMBER ) {
03382         if ( s2[1] < 0 ) {
03383             AT.comnum[1] = -s2[1]; AT.comnum[3] = -3;
03384             st2 = AT.comnum+4;
03385         }
03386         else if ( s2[1] == 0 ) {
03387             st2 = AT.comnum+4; s2 = st2;
03388         }
03389         else {
03390             AT.comnum[1] = s2[1]; AT.comnum[3] = 3;
03391             st2 = AT.comnum+4;
03392         }
03393         s2 = AT.comnum;
03394     }
03395     else if ( *s2 == -INDEX || *s2 == -VECTOR ) {
03396         AT.comind[3] = s2[1]; AT.comind[6] = 3;
03397         st2 = AT.comind+7; s2 = AT.comind;
03398     }
03399     else if ( *s2 == -MINVECTOR ) {
03400         AT.comind[3] = s2[1]; AT.comind[6] = -3;
03401         st2 = AT.comind+7; s2 = AT.comind;
03402     }
03403     else if ( *s2 <= -FUNCTION ) {
03404         AT.comfun[1] = -*s2;
03405         st2 = AT.comfun+FUNHEAD+4; s2 = AT.comfun;
03406     }
03407 /*
03408     Symmetrize during compilation of id statement when properorder
03409     needs this one. Code added 10-nov-2001
03410 */
03411     else if ( *s2 == -ARGWILD ) {
03412         return(1);
03413     }
03414     else { goto argerror; }
03415     st1 = s1 + *s1; s1 += ARGHEAD;
03416     goto docompare;
03417 }
03418 else {
03419     x[1] = AT.comsym[3];
03420     x[2] = AT.comnum[1];
03421     x[3] = AT.comnum[3];
03422     x[4] = AT.comind[3];
03423     x[5] = AT.comind[6];
03424     x[6] = AT.comfun[1];
03425     st1 = s1 + *s1; st2 = s2 + *s2;
03426     s1 += ARGHEAD; s2 += ARGHEAD;
03427 docompare:
03428     while ( s1 < st1 && s2 < st2 ) {
03429         if ( ( k = CompareTerms(BHEAD s1,s2,(WORD)2) ) != 0 ) {
03430             AT.comsym[3] = x[1];
03431             AT.comnum[1] = x[2];
03432             AT.comnum[3] = x[3];
03433             AT.comind[3] = x[4];
03434             AT.comind[6] = x[5];
03435             AT.comfun[1] = x[6];
03436             return(-k);
03437         }

```

```

03438         s1 += *s1; s2 += *s2;
03439     }
03440     AT.comsym[3] = x[1];
03441     AT.comnum[1] = x[2];
03442     AT.comnum[3] = x[3];
03443     AT.comind[3] = x[4];
03444     AT.comind[6] = x[5];
03445     AT.comfun[1] = x[6];
03446     if ( s1 < st1 ) return(1);
03447     if ( s2 < st2 ) return(-1);
03448 }
03449 return(0);
03450
03451 argerror:
03452     MesPrint("Illegal type of short function argument in Normalize");
03453     Terminate(-1); return(0);
03454 }
03455
03456 /*
03457     #] CompArg :
03458     #[ TimeWallClock :
03459 */
03460
03461 #ifdef HAVE_CLOCK_GETTIME
03462 #include <time.h> /* for clock_gettime() */
03463 #else
03464 #ifdef HAVE_GETTIMEOFDAY
03465 #include <sys/time.h> /* for gettimeofday() */
03466 #else
03467 #include <sys/timeb.h> /* for ftime() */
03468 #endif
03469 #endif
03470
03471 LONG TimeWallClock(WORD par)
03472 {
03473     /*
03474      * NOTE: this function is not thread-safe. Operations on tp are not atomic.
03475      */
03476
03477 #ifdef HAVE_CLOCK_GETTIME
03478     struct timespec ts;
03479     clock_gettime(CLOCK_MONOTONIC, &ts);
03480
03481     if ( par ) {
03482         return(((LONG)(ts.tv_sec)-AM.OldSecTime)*100 +
03483             ((LONG)(ts.tv_nsec / 1000000)-AM.OldMilliTime)/10);
03484     }
03485     else {
03486         AM.OldSecTime = (LONG)(ts.tv_sec);
03487         AM.OldMilliTime = (LONG)(ts.tv_nsec / 1000000);
03488         return(0L);
03489     }
03490 #else
03491 #ifdef HAVE_GETTIMEOFDAY
03492     struct timeval t;
03493     LONG sec, msec;
03494     gettimeofday(&t, NULL);
03495     sec = (LONG)t.tv_sec;
03496     msec = (LONG)(t.tv_usec/1000);
03497     if ( par ) {
03498         return (sec-AM.OldSecTime)*100 + (msec-AM.OldMilliTime)/10;
03499     }
03500     else {
03501         AM.OldSecTime = sec;
03502         AM.OldMilliTime = msec;
03503         return(0L);
03504     }
03505 #else
03506     struct timeb tp;
03507     ftime(&tp);
03508
03509     if ( par ) {
03510         return(((LONG)(tp.time)-AM.OldSecTime)*100 +
03511             ((LONG)(tp.millitm)-AM.OldMilliTime)/10);
03512     }
03513     else {
03514         AM.OldSecTime = (LONG)(tp.time);
03515         AM.OldMilliTime = (LONG)(tp.millitm);
03516         return(0L);
03517     }
03518 #endif
03519 #endif
03520 }
03521
03522 #] TimeWallClock :
03523 #[ TimeChildren :

```

```

03531 */
03532
03533 LONG TimeChildren(WORD par)
03534 {
03535     if ( par ) return(Timer(1)-AM.OldChildTime);
03536     AM.OldChildTime = Timer(1);
03537     return(0L);
03538 }
03539
03540 /*
03541     #] TimeChildren :
03542     #[ TimeCPU :
03543 */
03544
03551 LONG TimeCPU(WORD par)
03552 {
03553     GETIDENTITY
03554     if ( par ) return(Timer(0)-AR.OldTime);
03555     AR.OldTime = Timer(0);
03556     return(0L);
03557 }
03558
03559 /*
03560     #] TimeCPU :
03561     #[ Timer :
03562 */
03563 #if defined(WINDOWS)
03564
03565 LONG Timer(int par)
03566 {
03567     #ifndef WITHPTHREADS
03568         static int initialized = 0;
03569         static HANDLE hProcess;
03570         FILETIME ftCreate, ftExit, ftKernel, ftUser;
03571         DUMMYUSE(par);
03572
03573         if ( !initialized ) {
03574             hProcess = OpenProcess(PROCESS_QUERY_INFORMATION, FALSE, GetCurrentProcessId());
03575         }
03576         if ( GetProcessTimes(hProcess, &ftCreate, &ftExit, &ftKernel, &ftUser) ) {
03577             PFILETIME pftKernel = &ftKernel; /* to avoid strict-aliasing rule warnings */
03578             PFILETIME pftUser = &ftUser;
03579             __int64 t = *(__int64 *)pftKernel + *(__int64 *)pftUser; /* in 100 nsec. */
03580             return (LONG)(t / 10000); /* in msec. */
03581         }
03582         return 0;
03583     #else
03584         LONG lResult = 0;
03585         HANDLE hThread;
03586         FILETIME ftCreate, ftExit, ftKernel, ftUser;
03587         DUMMYUSE(par);
03588
03589         hThread = OpenThread(THREAD_QUERY_INFORMATION, FALSE, GetCurrentThreadId());
03590         if ( hThread ) {
03591             if ( GetThreadTimes(hThread, &ftCreate, &ftExit, &ftKernel, &ftUser) ) {
03592                 PFILETIME pftKernel = &ftKernel; /* to avoid strict-aliasing rule warnings */
03593                 PFILETIME pftUser = &ftUser;
03594                 __int64 t = *(__int64 *)pftKernel + *(__int64 *)pftUser; /* in 100 nsec. */
03595                 lResult = (LONG)(t / 10000); /* in msec. */
03596             }
03597             CloseHandle(hThread);
03598         }
03599         return lResult;
03600     #endif
03601 }
03602
03603 #elif defined(UNIX)
03604 #include <sys/time.h>
03605 #include <sys/resource.h>
03606 #ifndef WITHPOSIXCLOCK
03607 #include <time.h>
03608 /*
03609     And include -lrt in the link statement (on blade02)
03610 */
03611 #endif
03612
03613 LONG Timer(int par)
03614 {
03615     #ifndef WITHPOSIXCLOCK
03616         /*
03617             Only to be used in combination with WITHPTHREADS
03618             This clock seems to be supported by the standard.
03619             The getusage clock returns according to the standard only the combined
03620             time of the whole process. But in older versions of Linux LinuxThreads
03621             is used which gives a separate id to each thread and individual timings.
03622             In NPTL we get, according to the standard, one combined timing.
03623             To get individual timings we need to use

```

```

03624         clock_gettime(CLOCK_THREAD_CPUTIME_ID, &timing)
03625     with timing of the time
03626     struct timespec {
03627         time_t tv_sec;           Seconds.
03628         long tv_nsec;           Nanoseconds.
03629     };
03630
03631     /*
03632     struct timespec t;
03633     if ( par == 0 ) {
03634         if ( clock_gettime(CLOCK_THREAD_CPUTIME_ID, &t) ) {
03635             MesPrint("Error in getting timing information");
03636         }
03637         return (LONG)t.tv_sec * 1000 + (LONG)t.tv_nsec / 1000000;
03638     }
03639     return(0);
03640 #else
03641     struct rusage rusage;
03642     if ( par == 1 ) {
03643         getrusage(RUSAGE_CHILDREN, &rusage);
03644         return(((LONG) (rusage.ru_utime.tv_sec) + (LONG) (rusage.ru_stime.tv_sec)) * 1000
03645             + (rusage.ru_utime.tv_usec / 1000 + rusage.ru_stime.tv_usec / 1000));
03646     }
03647     else {
03648         getrusage(RUSAGE_SELF, &rusage);
03649         return(((LONG) (rusage.ru_utime.tv_sec) + (LONG) (rusage.ru_stime.tv_sec)) * 1000
03650             + (rusage.ru_utime.tv_usec / 1000 + rusage.ru_stime.tv_usec / 1000));
03651     }
03652 #endif
03653 }
03654
03655 #elif defined(SUN)
03656 #define _TIME_T_
03657 #include <sys/time.h>
03658 #include <sys/resource.h>
03659
03660 LONG Timer(int par)
03661 {
03662     struct rusage rusage;
03663     if ( par == 1 ) {
03664         getrusage(RUSAGE_CHILDREN, &rusage);
03665         return(((LONG) (rusage.ru_utime.tv_sec) + (LONG) (rusage.ru_stime.tv_sec)) * 1000
03666             + (rusage.ru_utime.tv_usec / 1000 + rusage.ru_stime.tv_usec / 1000));
03667     }
03668     else {
03669         getrusage(RUSAGE_SELF, &rusage);
03670         return(((LONG) (rusage.ru_utime.tv_sec) + (LONG) (rusage.ru_stime.tv_sec)) * 1000
03671             + (rusage.ru_utime.tv_usec / 1000 + rusage.ru_stime.tv_usec / 1000));
03672     }
03673 }
03674
03675 #elif defined(RS6K)
03676 #include <sys/time.h>
03677 #include <sys/resource.h>
03678
03679 LONG Timer(int par)
03680 {
03681     struct rusage rusage;
03682     if ( par == 1 ) {
03683         getrusage(RUSAGE_CHILDREN, &rusage);
03684         return(((LONG) (rusage.ru_utime.tv_sec) + (LONG) (rusage.ru_stime.tv_sec)) * 1000
03685             + (rusage.ru_utime.tv_usec / 1000 + rusage.ru_stime.tv_usec / 1000));
03686     }
03687     else {
03688         getrusage(RUSAGE_SELF, &rusage);
03689         return(((LONG) (rusage.ru_utime.tv_sec) + (LONG) (rusage.ru_stime.tv_sec)) * 1000
03690             + (rusage.ru_utime.tv_usec / 1000 + rusage.ru_stime.tv_usec / 1000));
03691     }
03692 }
03693
03694 #elif defined(ANSI)
03695 LONG Timer(int par)
03696 {
03697     #ifdef ALPHA
03698         /* clock_t t, tikken = clock(); */
03699         /* MesPrint("ALPHA-clock = %l", (LONG)tikken); */
03700         /* t = tikken % CLOCKS_PER_SEC; */
03701         /* tikken /= CLOCKS_PER_SEC; */
03702         /* tikken *= 1000; */
03703         /* tikken += (t * 1000) / CLOCKS_PER_SEC; */
03704         /* return((LONG)tikken); */
03705         /* #define _TIME_T_ */
03706         #include <sys/time.h>
03707         #include <sys/resource.h>
03708         struct rusage rusage;
03709         if ( par == 1 ) {
03710             getrusage(RUSAGE_CHILDREN, &rusage);

```

```

03711         return(((LONG)(rusage.ru_utime.tv_sec)+(LONG)(rusage.ru_stime.tv_sec))*1000
03712                + (rusage.ru_utime.tv_usec/1000+rusage.ru_stime.tv_usec/1000));
03713     }
03714     else {
03715         getrusage(RUSAGE_SELF,&rusage);
03716         return(((LONG)(rusage.ru_utime.tv_sec)+(LONG)(rusage.ru_stime.tv_sec))*1000
03717                + (rusage.ru_utime.tv_usec/1000+rusage.ru_stime.tv_usec/1000));
03718     }
03719 #else
03720 #ifdef DEC_STATION
03721     clock_t tikken = clock();
03722     return((LONG)tikken/1000);
03723 #else
03724     clock_t t, tikken = clock();
03725     t = tikken % CLK_TCK;
03726     tikken /= CLK_TCK;
03727     tikken *= 1000;
03728     tikken += (t*1000)/CLK_TCK;
03729     return(tikken);
03730 #endif
03731 #endif
03732 }
03733 #elif defined(VMS)
03734
03735 #include <time.h>
03736 void times(tbuffer_t *buffer);
03737
03738 LONG
03739 Timer(int par)
03740 {
03741     tbuffer_t buffer;
03742     if ( par == 1 ) { return(0); }
03743     else {
03744         times(&buffer);
03745         return(buffer.proc_user_time * 10);
03746     }
03747 }
03748
03749 #elif defined(mBSD)
03750
03751 #ifdef MICROTIME
03752 /*
03753     There is only a CP time clock in microseconds here
03754     This can cause problems with AO.wrap around
03755 */
03756 #else
03757 #ifdef mBSD2
03758 #include <sys/types.h>
03759 #include <sys/times.h>
03760 #include <time.h>
03761 LONG pretime = 0;
03762 #else
03763 #define _TIME_T_
03764 #include <sys/time.h>
03765 #include <sys/resource.h>
03766 #endif
03767 #endif
03768
03769 LONG Timer(int par)
03770 {
03771     #ifdef MICROTIME
03772         LONG t;
03773         if ( par == 1 ) { return(0); }
03774         t = clock();
03775         if ( ( AO.wrapnum & 1 ) != 0 ) t ^= 0x80000000;
03776         if ( t < 0 ) {
03777             t ^= 0x80000000;
03778             wrapnum++;
03779             AO.wrap += 2147584;
03780         }
03781         return(AO.wrap+(t/1000));
03782     #else
03783     #ifdef mBSD2
03784         struct tms buffer;
03785         LONG ret;
03786         ULONG a1, a2, a3, a4;
03787         if ( par == 1 ) { return(0); }
03788         times(&buffer);
03789         a1 = (ULONG)buffer.tms_utime;
03790         a2 = a1 » 16;
03791         a3 = a1 & 0xFFFFL;
03792         a3 *= 1000;
03793         a2 = 1000*a2 + (a3 » 16);
03794         a3 &= 0xFFFFL;
03795         a4 = a2/CLK_TCK;
03796         a2 %= CLK_TCK;
03797         a3 += a2 « 16;

```

```

03798     ret = (LONG)((a4 << 16) + a3 / CLK_TCK);
03799 /* ret = ((LONG)buffer.tms_utime * 1000)/CLK_TCK; */
03800     return(ret);
03801 #else
03802 #ifdef REALTIME
03803     struct timeval tp;
03804     struct timezone tzp;
03805     if ( par == 1 ) { return(0); }
03806     gettimeofday(&tp,&tzp); */
03807     return(tp.tv_sec*1000+tp.tv_usec/1000);
03808 #else
03809     struct rusage rusage;
03810     if ( par == 1 ) {
03811         getrusage(RUSAGE_CHILDREN,&rusage);
03812         return((rusage.ru_utime.tv_sec+rusage.ru_stime.tv_sec)*1000
03813             +(rusage.ru_utime.tv_usec/1000+rusage.ru_stime.tv_usec/1000));
03814     }
03815     else {
03816         getrusage(RUSAGE_SELF,&rusage);
03817         return((rusage.ru_utime.tv_sec+rusage.ru_stime.tv_sec)*1000
03818             +(rusage.ru_utime.tv_usec/1000+rusage.ru_stime.tv_usec/1000));
03819     }
03820 #endif
03821 #endif
03822 #endif
03823 }
03824
03825 #endif
03826
03827 /*
03828     #] Timer :
03829     #[ Crash :
03830
03831     Routine for debugging purposes
03832 */
03833
03834 int Crash(void)
03835 {
03836     int retval;
03837 #ifdef DEBUGGING
03838     int *zero = 0;
03839     retval = *zero;
03840 #else
03841     retval = 0;
03842 #endif
03843     return(retval);
03844 }
03845
03846 /*
03847     #] Crash :
03848     #[ TestTerm :
03849 */
03850
03862 int TestTerm(WORD *term)
03863 {
03864     int errorcode = 0, coeffsize;
03865     WORD *t, *tt, *tstop, *endterm, *targ, *targstop, *funstop, *argterm;
03866     endterm = term + *term;
03867     coeffsize = ABS(endterm[-1]);
03868     if ( coeffsize >= *term ) {
03869         MLOCK(ErrorMessageLock);
03870         MesPrint("TestTerm: Internal inconsistency in term. Coefficient too big.");
03871         MUNLOCK(ErrorMessageLock);
03872         errorcode = 1;
03873         goto finish;
03874     }
03875     if ( ( coeffsize < 3 ) || ( ( coeffsize & 1 ) != 1 ) ) {
03876         MLOCK(ErrorMessageLock);
03877         MesPrint("TestTerm: Internal inconsistency in term. Wrong size coefficient.");
03878         MUNLOCK(ErrorMessageLock);
03879         errorcode = 2;
03880         goto finish;
03881     }
03882     t = term+1;
03883     tstop = endterm - coeffsize;
03884     while ( t < tstop ) {
03885         switch ( *t ) {
03886             case SYMBOL:
03887             case DOTPRODUCT:
03888             case INDEX:
03889             case VECTOR:
03890             case DELTA:
03891             case HAAKJE:
03892                 break;
03893             case SNUMBER:
03894             case LNUMBER:
03895                 MLOCK(ErrorMessageLock);

```



```

03896         MesPrint("TestTerm: Internal inconsistency in term. L or S number");
03897         MUNLOCK(ErrorMessageLock);
03898         errorcode = 3;
03899         goto finish;
03900         break;
03901     case EXPRESSION:
03902     case SUBEXPRESSION:
03903     case DOLLAREXPRESSION:
03904 /*
03905         MLOCK(ErrorMessageLock);
03906         MesPrint("TestTerm: Internal inconsistency in term. Expression survives.");
03907         MUNLOCK(ErrorMessageLock);
03908         errorcode = 4;
03909         goto finish;
03910 */
03911         break;
03912     case SETSET:
03913     case MINVECTOR:
03914     case SETEXP:
03915     case ARGFIELD:
03916         MLOCK(ErrorMessageLock);
03917         MesPrint("TestTerm: Internal inconsistency in term. Illegal subterm.");
03918         MUNLOCK(ErrorMessageLock);
03919         errorcode = 5;
03920         goto finish;
03921         break;
03922     case ARGWILD:
03923         break;
03924     default:
03925         if ( *t <= 0 ) {
03926             MLOCK(ErrorMessageLock);
03927             MesPrint("TestTerm: Internal inconsistency in term. Illegal subterm number.");
03928             MUNLOCK(ErrorMessageLock);
03929             errorcode = 6;
03930             goto finish;
03931         }
03932 /*
03933         This is a regular function.
03934 */
03935         if ( *t-FUNCTION >= NumFunctions ) {
03936             MLOCK(ErrorMessageLock);
03937             MesPrint("TestTerm: Internal inconsistency in term. Illegal function number");
03938             MUNLOCK(ErrorMessageLock);
03939             errorcode = 7;
03940             goto finish;
03941         }
03942         funstop = t + t[1];
03943         if ( funstop > tstop ) goto subtermssize;
03944         if ( t[2] != 0 ) {
03945             MLOCK(ErrorMessageLock);
03946             MesPrint("TestTerm: Internal inconsistency in term. Dirty flag nonzero.");
03947             MUNLOCK(ErrorMessageLock);
03948             errorcode = 8;
03949             goto finish;
03950         }
03951         targ = t + FUNHEAD;
03952         if ( targ > funstop ) {
03953             MLOCK(ErrorMessageLock);
03954             MesPrint("TestTerm: Internal inconsistency in term. Illegal function size.");
03955             MUNLOCK(ErrorMessageLock);
03956             errorcode = 9;
03957             goto finish;
03958         }
03959         if ( functions[*t-FUNCTION].spec >= TENSORFUNCTION ) {
03960         }
03961         else {
03962             while ( targ < funstop ) {
03963                 if ( *targ < 0 ) {
03964                     if ( *targ <= -(FUNCTION+NumFunctions) ) {
03965                         MLOCK(ErrorMessageLock);
03966                         MesPrint("TestTerm: Internal inconsistency in term. Illegal function
number in argument.");
03967                         MUNLOCK(ErrorMessageLock);
03968                         errorcode = 10;
03969                         goto finish;
03970                     }
03971                     if ( *targ <= -FUNCTION ) { targ++; }
03972                     else {
03973                         if ( ( *targ != -SYMBOL ) && ( *targ != -VECTOR )
03974                             && ( *targ != -MINVECTOR )
03975                             && ( *targ != -SNUMBER )
03976                             && ( *targ != -ARGWILD )
03977                             && ( *targ != -INDEX ) ) {
03978                             MLOCK(ErrorMessageLock);
03979                             MesPrint("TestTerm: Internal inconsistency in term. Illegal object in
argument.");
03980                             MUNLOCK(ErrorMessageLock);

```

```

03981                 errorcode = 11;
03982                 goto finish;
03983             }
03984             targ += 2;
03985         }
03986     }
03987     else if ( ( *targ < ARGHEAD ) || ( targ+*targ > funstop ) ) {
03988         MLOCK(ErrorMessageLock);
03989         MesPrint("TestTerm: Internal inconsistency in term. Illegal size of
argument.");
03990         MUNLOCK(ErrorMessageLock);
03991         errorcode = 12;
03992         goto finish;
03993     }
03994     else if ( targ[1] != 0 ) {
03995         MLOCK(ErrorMessageLock);
03996         MesPrint("TestTerm: Internal inconsistency in term. Dirty flag in argument.");
03997         MUNLOCK(ErrorMessageLock);
03998         errorcode = 13;
03999         goto finish;
04000     }
04001     else {
04002         targstop = targ + *targ;
04003         argterm = targ + ARGHEAD;
04004         while ( argterm < targstop ) {
04005             if ( ( *argterm < 4 ) || ( argterm + *argterm > targstop ) ) {
04006                 MLOCK(ErrorMessageLock);
04007                 MesPrint("TestTerm: Internal inconsistency in term. Illegal termsize
in argument.");
04008                 MUNLOCK(ErrorMessageLock);
04009                 errorcode = 14;
04010                 goto finish;
04011             }
04012             if ( TestTerm(argterm) != 0 ) {
04013                 MLOCK(ErrorMessageLock);
04014                 MesPrint("TestTerm: Internal inconsistency in term. Called from
TestTerm.");
04015                 MUNLOCK(ErrorMessageLock);
04016                 errorcode = 15;
04017                 goto finish;
04018             }
04019             argterm += *argterm;
04020         }
04021         targ = targstop;
04022     }
04023 }
04024 }
04025 break;
04026 }
04027 tt = t + t[1];
04028 if ( tt > tstop ) {
04029 subtermsize:
04030     MLOCK(ErrorMessageLock);
04031     MesPrint("TestTerm: Internal inconsistency in term. Illegal subterm size.");
04032     MUNLOCK(ErrorMessageLock);
04033     errorcode = 100;
04034     goto finish;
04035 }
04036 t = tt;
04037 }
04038 return(errorcode);
04039 finish:
04040     return(errorcode);
04041 }
04042
04043 /*
04044     #] TestTerm :
04045     #[ DistrN :
04046 */
04047
04048 int DistrN(int n, int *cpl, int ncpl, int *scratch)
04049 {
04050     /*
04051     Divides n objects over ncpl bins (cpl), each time returning one
04052     of those distributions until there are no more after which the
04053     routine returns the value zero (otherwise one).
04054     The array scratch (size n) is kept for the intermediate information.
04055     The whole starts with scratch[0] == -2;
04056     */
04057     int i, j;
04058     if ( ncpl == 0 ) {
04059         if ( scratch[0] == -2 ) { scratch[0] = 0; return(1); }
04060         else return(0);
04061     }
04062     if ( scratch[0] == ncpl-1 ) {
04063         return(0);
04064     }

```

```

04065     else if ( scratch[0] == -2 ) {
04066         for ( i = 0; i < n; i++ ) scratch[i] = 0;
04067     }
04068     else {
04069         j = n-1;
04070         while ( j >= 0 ) {
04071             scratch[j]++;
04072             if ( scratch[j] < ncpl ) break;
04073             j--;
04074         }
04075         j++;
04076         while ( j < n ) { scratch[j] = scratch[j-1]; j++; }
04077     }
04078     for ( i = 0; i < ncpl; i++ ) cpl[i] = 0;
04079     for ( i = 0; i < n; i++ ) { cpl[scratch[i]]++; }
04080     return(1);
04081 }
04082
04083 /*
04084     #] DistrN :
04085     #] Mixed :
04086 */

```

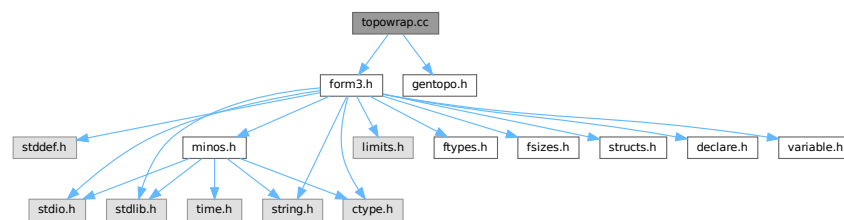
4.148 topowrap.cc File Reference

```

#include "form3.h"
#include "gentopo.h"

```

Include dependency graph for topowrap.cc:



Data Structures

- struct [ToPoTyPe](#)

Macros

- #define [MAXPOINTS](#) 120

Typedefs

- typedef struct [ToPoTyPe](#) **TOPOTYPE**

Functions

- int [GenerateVertices](#) (TOPOTYPE *TopoInf, int pointsremaining, int level)
- WORD [GenerateTopologies](#) (PHEAD WORD nloops, WORD nlegs, WORD setvert, WORD setmax)
- void [toForm](#) (T_EGraph *egraph)

4.148.1 Detailed Description

routines for conversion of topology and diagram output to FORM notation

Definition in file [topowrap.cc](#).

4.148.2 Macro Definition Documentation

4.148.2.1 MAXPOINTS

```
#define MAXPOINTS 120
```

Definition at line 44 of file [topowrap.cc](#).

4.148.3 Function Documentation

4.148.3.1 GenerateVertices()

```
int GenerateVertices (
    TOPOTYPE * TopoInf,
    int pointsremaining,
    int level )
```

Definition at line 72 of file [topowrap.cc](#).

4.148.3.2 GenerateTopologies()

```
WORD GenerateTopologies (
    PHEAD WORD nloops,
    WORD nlegs,
    WORD setvert,
    WORD setmax )
```

Definition at line 123 of file [topowrap.cc](#).

4.148.3.3 toForm()

```
void toForm (
    T_EGraph * egraph )
```

Definition at line 178 of file [topowrap.cc](#).

4.149 topowrap.cc

[Go to the documentation of this file.](#)

```

00001
00006 /* #[ License : */
00007 /*
00008 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00009 *   When using this file you are requested to refer to the publication
00010 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00011 *   This is considered a matter of courtesy as the development was paid
00012 *   for by FOM the Dutch physics granting agency and we would like to
00013 *   be able to track its scientific use to convince FOM of its value
00014 *   for the community.
00015 *
00016 *   This file is part of FORM.
00017 *
00018 *   FORM is free software: you can redistribute it and/or modify it under the
00019 *   terms of the GNU General Public License as published by the Free Software
00020 *   Foundation, either version 3 of the License, or (at your option) any later
00021 *   version.
00022 *
00023 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00024 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00025 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00026 *   details.
00027 *
00028 *   You should have received a copy of the GNU General Public License along
00029 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00030 */
00031 /*
00032     #[ License :
00033     #[ includes :
00034 */
00035
00036 extern "C" {
00037 #include "form3.h"
00038 }
00039
00040 #include "gentopo.h"
00041
00042 //using namespace std;
00043
00044 #define MAXPOINTS 120
00045
00046 typedef struct ToPoTyPe {
00047     int cldeg[MAXPOINTS], cnum[MAXPOINTS], clext[MAXPOINTS];
00048     int ncl, nloops, nlegs, npadding;
00049     WORD *vert;
00050     WORD *vertmax;
00051     WORD nvert;
00052     WORD sopi;
00053 } TOPOTYPE;
00054
00055 /*
00056     #[ includes :
00057     #[ GenerateVertices :
00058
00059     Routine is to be used recursively to work its way through a list
00060     of possible vertices. The array of vertices is in TopoInf->vert
00061     with TopoInf->nvert the number of possible vertices.
00062     Currently we allow in TopoInf->vert only vertices with 3 or more edges.
00063
00064     We work with a point system. Each n-point vertex contributes n-2 points.
00065     When all points are assigned, we can call mgraph->generate().
00066
00067     The number of vertices of a given number of edges is stored in
00068     TopoInf->cnum[..] but the loop that determines how many there are
00069     may be limited by the corresponding element in TopoInf->vertmax[level]
00070 */
00071
00072 int GenerateVertices(TOPOTYPE *TopoInf, int pointsremaining, int level)
00073 {
00074     int i, j;
00075
00076     for ( i = pointsremaining, j = 0; i >= 0; i -= TopoInf->vert[level]-2, j++ ) {
00077         if ( TopoInf->vertmax && TopoInf->vertmax[level] >= 0
00078             && j > TopoInf->vertmax[level] ) break;
00079         if ( i == 0 ) { // We got one!
00080             T_MGraph *mgraph;
00081             TopoInf->cldeg[TopoInf->ncl] = TopoInf->vert[level];
00082             TopoInf->cnum[TopoInf->ncl] = j;
00083             TopoInf->clext[TopoInf->ncl] = 0;
00084             TopoInf->ncl++;
00085
00086             mgraph = new T_MGraph(0, TopoInf->ncl, TopoInf->cldeg,

```

```

00087         TopoInf->clnum, TopoInf->clext, TopoInf->sopi);
00088
00089         mgraph->generate();
00090
00091         delete mgraph;
00092
00093         TopoInf->ncl--;
00094
00095         break;
00096     }
00097     if ( level < TopoInf->nvert-1 ) {
00098         if ( j > 0 ) {
00099             TopoInf->cldeg[TopoInf->ncl] = TopoInf->vert[level];
00100             TopoInf->clnum[TopoInf->ncl] = j;
00101             TopoInf->clext[TopoInf->ncl] = 0;
00102             TopoInf->ncl++;
00103         }
00104         if ( GenerateVertices(TopoInf,i,level+1) < 0 ) return(-1);
00105         if ( j > 0 ) { TopoInf->ncl--; }
00106     }
00107 }
00108 return(0);
00109 }
00110
00111 /*
00112  #] GenerateVertices :
00113  #[ GenerateTopologies :
00114
00115  Note that setmax, option1 and option2 are optional.
00116  Default values are -1,0,0
00117
00118  vert is a pointer to a set of numbers indicating the types of vertices
00119  that are allowed.
00120  nvert is the number of elements in vert.
00121 */
00122
00123 WORD GenerateTopologies(PHEAD WORD nloops, WORD nlegs, WORD setvert, WORD setmax)
00124 {
00125     TOPOTYPE TopoInf;
00126     int i, points, identical = 0;
00127     DUMMYUSE(AT.nfac);
00128
00129     if ( nlegs == -2 ) {
00130         nlegs = 2;
00131         identical = 1;
00132     }
00133     TopoInf.vert = &(SetElements[Sets[setvert].first]);
00134     TopoInf.nvert = Sets[setvert].last-Sets[setvert].first;
00135
00136     if ( setmax >= 0 ) TopoInf.vertmax = &(SetElements[Sets[setmax].first]);
00137     else TopoInf.vertmax = 0;
00138
00139     // point counting: an n-point vertex counts for n-2 points.
00140
00141     points = 2*nloops-2+nlegs;
00142     if ( points >= MAXPOINTS ) {
00143         MLOCK(ErrorMessageLock);
00144         MesPrint("GenerateTopologies: %d loops and %d legs considered excessive",nloops,nlegs);
00145         MUNLOCK(ErrorMessageLock);
00146         Terminate(-1);
00147     }
00148
00149     // First the external nodes.
00150
00151     for ( i = 0; i < nlegs; i++ ) {
00152         TopoInf.cldeg[i] = 1; TopoInf.clnum[i] = 1; TopoInf.clext[i] = 1;
00153     }
00154     if ( identical == 1 ) { /* Only propagator topologies..... */
00155         nlegs = 1;
00156         TopoInf.clnum[0] = 2;
00157     }
00158     TopoInf.ncl = nlegs;
00159     TopoInf.sopi = 1; // For now
00160     if ( GenerateVertices(&TopoInf,points,0) != 0 ) {
00161         MLOCK(ErrorMessageLock);
00162         MesPrint("Called from GenerateTopologies with %d loops and %d legs considered
00163 excessive",nloops,nlegs);
00164         MUNLOCK(ErrorMessageLock);
00165         Terminate(-1);
00166     }
00167     return(0);
00168 }
00169
00170 /*
00171  #] GenerateTopologies :
00172  #[ toForm :
00173 */

```

```

00173
00174 //=====
00175 // Output for FROM called by genTopo each time a new graph is
00176 // generated
00177 //
00178 void toForm(T_EGraph *egraph)
00179 {
00180     GETIDENTITY;
00181     //
00182     // skipext : boolean; whether to skip external node/line in the output.
00183     //
00184     // const int skipext = 1;
00185
00186     int n, lg, ed, i, fromset;
00187
00188     WORD *termout = AT.WorkPointer;
00189     WORD *t, *tt, *ttstop, *ttend, *ttail;
00190
00191     t = termout+1;
00192
00193     // First pick up part of the original term
00194
00195     tt = AT.TopologiesTerm + 1;
00196     i = AT.TopologiesStart - tt;
00197     NCOPY(t,tt,i)
00198     ttail = tt+tt[1];
00199
00200     // Now the vertices/nodes
00201     // Options are in AT.TopologiesOptions[]
00202
00203     for ( n = 0; n < egraph->nNodes; n++ ) {
00204         if ( ( AT.TopologiesOptions[1] & 1 ) == 1 && egraph->nodes[n].ext ) continue;
00205         tt = t;
00206         *t++ = NODEFUNCTION; t++; FILLFUN(t);
00207         *t++ = -SNUMBER; *t++ = n;
00208         for ( lg = 0; lg < egraph->nodes[n].deg; lg++ ) {
00209             ed = egraph->nodes[n].edges[lg];
00210             if ( ed >= 0 ) { *t++ = -VECTOR; }
00211             else { ed = -ed; *t++ = -MINVECTOR; }
00212
00213             // Now we need to pick up the proper set element.
00214
00215             fromset = egraph->edges[ed].ext ? AT.setexterntopo : AT.setinterntopo;
00216             *t++ = SetElements[Sets[fromset].first+egraph->edges[ed].momn-1];
00217         }
00218         tt[1] = t-tt;
00219     }
00220
00221     if ( ( AT.TopologiesOptions[0] & 1 ) == 1 ) {
00222         // Note that the edges count from 1.
00223         for ( n = 1; n <= egraph->nEdges; n++ ) {
00224             if ( ( AT.TopologiesOptions[1] & 1 ) == 1 && egraph->edges[n].ext ) continue;
00225             tt = t;
00226             *t++ = EDGE; t++; FILLFUN(t);
00227             *t++ = -SNUMBER; *t++ = egraph->edges[n].nodes[0];
00228             *t++ = -SNUMBER; *t++ = egraph->edges[n].nodes[1];
00229             *t++ = -VECTOR;
00230             fromset = egraph->edges[n].ext ? AT.setexterntopo : AT.setinterntopo;
00231             *t++ = SetElements[Sets[fromset].first+egraph->edges[n].momn-1];
00232             tt[1] = t-tt;
00233         }
00234     }
00235
00236     // Now the tail end
00237
00238     ttend = AT.TopologiesTerm; ttend = ttend+ttend[0];
00239     ttstop = ttend - ABS(ttend[-1]);
00240     i = ttstop - ttail;
00241     NCOPY(t,ttail,i)
00242
00243     // Finally the coefficient
00244     // The topological coefficient should be in egraph->wsum, egraph->nwsum
00245     // as a FORM Long number
00246
00247     #ifdef WITHFACTOR
00248     if ( egraph->nwsum == 1 && egraph->wsum[0] == 1 ) {
00249         while ( ttstop < ttend ) *t++ = *ttstop++;
00250     }
00251     else {
00252         WORD newsize;
00253         newsize = ttend[-1];
00254         newsize = REDLENG(newsize);
00255         if ( Divvy(BHEAD (UWORD *)ttstop,&newsize,egraph->wsum,egraph->nwsum) )
00256             goto OnError;
00257         newsize = INCLENG(newsize);
00258         i = ABS(newsize)-1;
00259         NCOPY(t,ttstop,i)

```

```

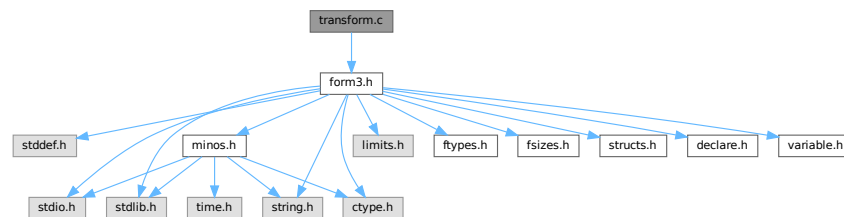
00260         *t++ = newsize;
00261     }
00262 #else
00263     while (ttstop < ttend ) *t++ = *ttstop++;
00264 #endif
00265     *termout = t - termout;
00266
00267     AT.WorkPointer = t;
00268
00269     if ( Generator(BHEAD termout,AT.TopologiesLevel) < 0 ) {
00270 // OnError:
00271         MLOCK(ErrorMessageLock);
00272         MesPrint("Called from the topologies routine toForm");
00273         MUNLOCK(ErrorMessageLock);
00274         Terminate(-1);
00275     }
00276
00277     AT.WorkPointer = termout;
00278 }
00279
00280 /*
00281 #] toForm :
00282 */
00283

```

4.150 transform.c File Reference

```
#include "form3.h"
```

Include dependency graph for transform.c:



Functions

- int [CoTransform](#) (UBYTE *in)
- WORD [RunTransform](#) (PHEAD WORD *term, WORD *params)
- WORD [RunEncode](#) (PHEAD WORD *fun, WORD *args, WORD *info)
- WORD [RunDecode](#) (PHEAD WORD *fun, WORD *args, WORD *info)
- WORD [RunReplace](#) (PHEAD WORD *fun, WORD *args, WORD *info)
- WORD [RunImplode](#) (WORD *fun, WORD *args)
- WORD [RunExplode](#) (PHEAD WORD *fun, WORD *args)
- WORD [RunPermute](#) (PHEAD WORD *fun, WORD *args, WORD *info)
- WORD [RunReverse](#) (PHEAD WORD *fun, WORD *args)
- WORD [RunDedup](#) (PHEAD WORD *fun, WORD *args)
- WORD [RunCycle](#) (PHEAD WORD *fun, WORD *args, WORD *info)
- WORD [RunAddArg](#) (PHEAD WORD *fun, WORD *args)
- WORD [RunMulArg](#) (PHEAD WORD *fun, WORD *args)
- WORD [RunIsLyndon](#) (PHEAD WORD *fun, WORD *args, int par)
- WORD [RunToLyndon](#) (PHEAD WORD *fun, WORD *args, int par)
- WORD [RunDropArg](#) (PHEAD WORD *fun, WORD *args)
- WORD [RunSelectArg](#) (PHEAD WORD *fun, WORD *args)

- WORD [RunZtoHArg](#) (PHEAD WORD *fun, WORD *args)
- WORD [RunHtoZArg](#) (PHEAD WORD *fun, WORD *args)
- int [TestArgNum](#) (int n, int totarg, WORD *args)
- WORD [PutArgInScratch](#) (WORD *arg, UWORD *scrat)
- UBYTE * [ReadRange](#) (UBYTE *s, WORD *out, int par)
- int [FindRange](#) (PHEAD WORD *args, WORD *arg1, WORD *arg2, WORD totarg)

4.150.1 Detailed Description

Routines that deal with the transform statement and its varieties.

Definition in file [transform.c](#).

4.150.2 Function Documentation

4.150.2.1 CoTransform()

```
int CoTransform (  
    UBYTE * in )
```

Definition at line [87](#) of file [transform.c](#).

4.150.2.2 RunTransform()

```
WORD RunTransform (  
    PHEAD WORD * term,  
    WORD * params )
```

Definition at line [732](#) of file [transform.c](#).

4.150.2.3 RunEncode()

```
WORD RunEncode (  
    PHEAD WORD * fun,  
    WORD * args,  
    WORD * info )
```

Definition at line [982](#) of file [transform.c](#).

4.150.2.4 RunDecode()

```
WORD RunDecode (  
    PHEAD WORD * fun,  
    WORD * args,  
    WORD * info )
```

Definition at line [1169](#) of file [transform.c](#).

4.150.2.5 RunReplace()

```
WORD RunReplace (
    PHEAD WORD * fun,
    WORD * args,
    WORD * info )
```

Definition at line 1339 of file [transform.c](#).

4.150.2.6 RunImplode()

```
WORD RunImplode (
    WORD * fun,
    WORD * args )
```

Definition at line 1817 of file [transform.c](#).

4.150.2.7 RunExplode()

```
WORD RunExplode (
    PHEAD WORD * fun,
    WORD * args )
```

Definition at line 2018 of file [transform.c](#).

4.150.2.8 RunPermute()

```
WORD RunPermute (
    PHEAD WORD * fun,
    WORD * args,
    WORD * info )
```

Definition at line 2132 of file [transform.c](#).

4.150.2.9 RunReverse()

```
WORD RunReverse (
    PHEAD WORD * fun,
    WORD * args )
```

Definition at line 2359 of file [transform.c](#).

4.150.2.10 RunDedup()

```
WORD RunDedup (
    PHEAD WORD * fun,
    WORD * args )
```

Definition at line 2446 of file [transform.c](#).

4.150.2.11 RunCycle()

```
WORD RunCycle (
    PHEAD WORD * fun,
    WORD * args,
    WORD * info )
```

Definition at line 2535 of file [transform.c](#).

4.150.2.12 RunAddArg()

```
WORD RunAddArg (
    PHEAD WORD * fun,
    WORD * args )
```

Definition at line 2687 of file [transform.c](#).

4.150.2.13 RunMulArg()

```
WORD RunMulArg (
    PHEAD WORD * fun,
    WORD * args )
```

Definition at line 2776 of file [transform.c](#).

4.150.2.14 RunIsLyndon()

```
WORD RunIsLyndon (
    PHEAD WORD * fun,
    WORD * args,
    int par )
```

Definition at line 2917 of file [transform.c](#).

4.150.2.15 RunToLyndon()

```
WORD RunToLyndon (
    PHEAD WORD * fun,
    WORD * args,
    int par )
```

Definition at line 2995 of file [transform.c](#).

4.150.2.16 RunDropArg()

```
WORD RunDropArg (
    PHEAD WORD * fun,
    WORD * args )
```

Definition at line 3107 of file [transform.c](#).

4.150.2.17 RunSelectArg()

```
WORD RunSelectArg (
    PHEAD WORD * fun,
    WORD * args )
```

Definition at line 3133 of file [transform.c](#).

4.150.2.18 RunZtoHArg()

```
WORD RunZtoHArg (
    PHEAD WORD * fun,
    WORD * args )
```

Definition at line 3161 of file [transform.c](#).

4.150.2.19 RunHtoZArg()

```
WORD RunHtoZArg (
    PHEAD WORD * fun,
    WORD * args )
```

Definition at line 3216 of file [transform.c](#).

4.150.2.20 TestArgNum()

```
int TestArgNum (
    int n,
    int totarg,
    WORD * args )
```

Definition at line 3281 of file [transform.c](#).

4.150.2.21 PutArgInScratch()

```
WORD PutArgInScratch (
    WORD * arg,
    UWORD * scrat )
```

Definition at line 3350 of file [transform.c](#).

4.150.2.22 ReadRange()

```
UBYTE * ReadRange (
    UBYTE * s,
    WORD * out,
    int par )
```

Definition at line 3386 of file [transform.c](#).

4.150.2.23 FindRange()

```
int FindRange (
    PHEAD WORD * args,
    WORD * arg1,
    WORD * arg2,
    WORD totarg )
```

Definition at line 3546 of file [transform.c](#).

4.151 transform.c

[Go to the documentation of this file.](#)

```
00001
00005 /* #[] License : */
00006 /*
00007  * Copyright (C) 1984-2023 J.A.M. Vermaseren
00008  * When using this file you are requested to refer to the publication
00009  * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00010  * This is considered a matter of courtesy as the development was paid
00011  * for by FOM the Dutch physics granting agency and we would like to
00012  * be able to track its scientific use to convince FOM of its value
00013  * for the community.
00014  *
00015  * This file is part of FORM.
00016  *
00017  * FORM is free software: you can redistribute it and/or modify it under the
00018  * terms of the GNU General Public License as published by the Free Software
00019  * Foundation, either version 3 of the License, or (at your option) any later
00020  * version.
00021  *
00022  * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00023  * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00024  * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00025  * details.
00026  *
00027  * You should have received a copy of the GNU General Public License along
00028  * with FORM. If not, see <http://www.gnu.org/licenses/>.
00029  */
00030 /* #[] License : */
00031 /*
00032  #[] Includes : transform.c
00033  */
00034
00035 #include "form3.h"
00036
00037 /*
00038  #[] Includes :
00039  #[] Transform :
00040  #[] Intro :
00041
00042  Here are the routines for the transform statement. This is a
00043  group of transformations on function arguments or groups of
00044  function arguments. The purpose of this command is that it
00045  avoids repetitive pattern matching.
00046  Syntax:
00047      Transform,SetOfFunctions,OneOrMoreTransformations;
00048  Each transformation is given by
00049      Replace(argfirst,arglast)=(,,)
00050      Encode(argfirst,arglast):base=#
00051      Decode(argfirst,arglast):base=#
00052      Implode(argfirst,arglast)
00053      Explode(argfirst,arglast)
00054      Permute(cycle)(cycle)(cycle)...(cycle)
00055      Reverse(argfirst,arglast)
00056      Dedup(argfirst,arglast)
00057      Cycle(argfirst,arglast)=+/-num
00058      IsLyndon(argfirst,arglast)=(yes,no)
00059      ToLyndon(argfirst,arglast)=(yes,no)
00060  In replace the extra information is
00061      a replace_() without the name of the replace_ function.
00062      This can be as in (0,1,1,0) or (xarg_,1-xarg_) to indicate
00063      a symbolic argument or (x,y,y,x) to exchange x and y, etc.
00064  In Encode and Decode argfirst is the most significant 'word' and
00065  arglast is the least significant 'word'.
00066  Note that we need to introduce the generic symbolic arguments xarg_,
```

```

00067     parg_, iarg_ and farg_.
00068     Examples:
00069         Transform,{H,E}
00070             ,Replace(1:'WEIGHT')=(0,1,1,0)
00071             ,Encode(1:'WEIGHT')=base(2);
00072         Transform,{H,E}
00073             ,Decode(1:'WEIGHT')=base(3)
00074             ,Replace(1:'WEIGHT')=(2,-1,1,0,0,1);
00075     Others that can be added:
00076         symmetrize?
00077
00078     6-may-2016: Changed MAXPOSITIVE2 into MAXPOSITIVE4. This makes room
00079         for the use of dollar variables as arguments.
00080
00081     #] Intro :
00082     #[ CoTransform :
00083     */
00084
00085     static WORD tranarray[10] = { SUBEXPRESSION, SUBEXPSIZE, 0, 1, 0, 0, 0, 0, 0, 0 };
00086
00087     int CoTransform(UBYTE *in)
00088     {
00089         GETIDENTITY
00090         UBYTE *s = in, c, *ss, *Tempbuf;
00091         WORD number, type, i, *work = AT.WorkPointer+2, *wp, range[2], one = 1;
00092         WORD numdol, *wstart;
00093         int error = 0, irhs;
00094         LONG x;
00095         while ( *in == ',' ) in++;
00096         wp = work + 1;
00097     /*
00098     #[ Sets :
00099
00100     First the set specification(s). No sets means all functions (dangerous!)
00101     */
00102     for(;;) {
00103         if ( *in == '{' ) {
00104             s = in+1;
00105             SKIPBRA2(in)
00106             number = DoTempSet(s,in);
00107             in++;
00108             if ( *in != ',' ) {
00109                 c = in[1]; in[1] = 0;
00110                 MesPrint("& %s: A set in a transform statement should be followed by a comma",s);
00111                 in[1] = c; in++;
00112                 if ( error == 0 ) error = 1;
00113             }
00114         }
00115         else if ( *in == '[' || FG.cTable[*in] == 0 ) {
00116             s = in;
00117             in = SkipAName(in);
00118             if ( *in != ',' ) break;
00119             c = *in; *in = 0;
00120             type = GetName(AC.varnames,s,&number,NOAUTO);
00121             if ( type == CFUNCTION ) { number += MAXVARIABLES + FUNCTION; }
00122             else if ( type != CSET ) {
00123                 MesPrint("& %s: A transform statement starts with sets of functions",s);
00124                 if ( error == 0 ) error = 1;
00125             }
00126             *in++ = c;
00127         }
00128         else {
00129             MesPrint("&Illegal syntax in Transform statement",s);
00130             if ( error == 0 ) error = 1;
00131             return(error);
00132         }
00133         if ( number >= 0 ) {
00134             if ( number < MAXVARIABLES ) {
00135                 /*
00136                 Check that this is a set of functions
00137                 */
00138                 if ( Sets[number].type != CFUNCTION ) {
00139                     MesPrint("&A set in a transform statement should be a set of functions");
00140                     if ( error == 0 ) error = 1;
00141                 }
00142             }
00143         }
00144         else if ( error == 0 ) error = 1;
00145     /*
00146     Now write the number to the right place
00147     */
00148         *wp++ = number;
00149         while ( *in == ',' ) in++;
00150     }
00151     *work = wp - work;
00152     work = wp; wp++;
00153     */

```

```

00154     #] Sets :
00155
00156     Now we should loop over the various transformations
00157 */
00158     while ( *s ) {
00159         in = s;
00160         if ( FG.cTable[*in] != 0 ) {
00161             MesPrint("&Illegal character in Transform statement");
00162             if ( error == 0 ) error = 1;
00163             return(error);
00164         }
00165         in = SkipAName(in);
00166         if ( *in == '>' || *in == '<' || *in == '+' || *in == '-' ) in++;
00167         ss = in;
00168         c = *ss; *ss = 0;
00169         if ( c != '(' ) {
00170             MesPrint("&Illegal syntax in specifying a transformation inside a Transform statement");
00171             if ( error == 0 ) error = 1;
00172             return(error);
00173         }
00174     /*
00175     #[ replace :
00176 */
00177     if ( StrICmp(s, (UBYTE *) "replace") == 0 ) {
00178     /*
00179         Subkeys: (,,) as in replace_(,,)
00180         The idea here is to read the subkeys as the argument
00181         of a replace_ function.
00182         We put the whole together as in the multiply statement (which
00183         could just be a replace_(....)) and compile it.
00184         Then we expand the tree with Generator and check the complete
00185         expression for legality.
00186 */
00187         type = REPLACEARG;
00188     doreplace:
00189         *ss = c;
00190         if ( ( in = ReadRange(in, range, 0) ) == 0 ) {
00191             if ( error == 0 ) error = 1;
00192             return(error);
00193         }
00194         in++;
00195     /*
00196         We have replace(#,#)=(...), and we want dum_(...) (DUMFUN)
00197         to send to the compiler. The pointer is after the '=';
00198 */
00199         s = in;
00200         if ( *s != '(' ) {
00201             MesPrint("&");
00202             if ( error == 0 ) error = 1;
00203             return(error);
00204         }
00205         SKIPBRA3(in);
00206         if ( *in != ')' ) {
00207             MesPrint("&");
00208             if ( error == 0 ) error = 1;
00209             return(error);
00210         }
00211         in++;
00212         if ( *in != ',' && *in != '\0' ) {
00213             MesPrint("&");
00214             if ( error == 0 ) error = 1;
00215             return(error);
00216         }
00217         i = in - s;
00218         ss = Tempbuf = (UBYTE *) Malloc1(i+5, "CoTransform/replace");
00219         *ss++ = 'd'; *ss++ = 'u'; *ss++ = 'm'; *ss++ = '_';
00220         NCOPY(ss, s, i)
00221         *ss++ = 0;
00222         AC.ProtoType = tranarray;
00223         tranarray[4] = AC.cbufnum;
00224         irhs = CompileAlgebra(Tempbuf, RHSIDE, AC.ProtoType);
00225         M_free(Tempbuf, "CoTransform/replace");
00226         if ( irhs < 0 ) {
00227             if ( error == 0 ) error = 1;
00228             return(error);
00229         }
00230         tranarray[2] = irhs;
00231     /*
00232         The result of the compilation goes through Generator during
00233         execution, because that takes care of $-variables.
00234         This is why we could not use replace_ and had to use dum_.
00235 */
00236         *wp++ = ARGGRANGE;
00237         *wp++ = range[0];
00238         *wp++ = range[1];
00239         *wp++ = type;
00240         *wp++ = SUBEXPSIZE+4;

```

```

00241         for ( i = 0; i < SUBEXPSIZE; i++ ) *wp++ = tranarray[i];
00242         *wp++ = 1;
00243         *wp++ = 1;
00244         *wp++ = 3;
00245         *work = wp-work;
00246         work = wp; *wp++ = 0;
00247         s = in;
00248     }
00249 /*
00250     #] replace :
00251     #[ encode/decode :
00252 */
00253     else if ( StrICmp(s,(UBYTE *)"decode" ) == 0 ) {
00254         type = DECODEARG;
00255         goto doencode;
00256     }
00257     else if ( StrICmp(s,(UBYTE *)"encode" ) == 0 ) {
00258         type = ENCODEARG;
00259 doencode:    *ss = c;
00260         if ( ( in = ReadRange(in,range,2) ) == 0 ) {
00261             if ( error == 0 ) error = 1;
00262             return(error);
00263         }
00264         in++;
00265         s = in; while ( FG.cTable[*in] == 0 ) in++;
00266         c = *in; *in = 0;
00267 /*
00268         Subkeys: base=# or base=$var
00269 */
00270         if ( StrICmp(s,(UBYTE *)"base" ) == 0 ) {
00271             *in = c;
00272             if ( *in != '=' ) {
00273                 MesPrint("&Illegal base specification in encode/decode transformation");
00274                 if ( error == 0 ) error = 1;
00275                 return(error);
00276             }
00277             in++;
00278             if ( *in == '$' ) {
00279                 in++; ss = in;
00280                 in = SkipAName(in);
00281                 c = *in; *in = 0;
00282                 if ( GetName(AC.dollarnames,ss,&numdol,NOAUTO) != CDOLLAR ) {
00283                     MesPrint("&%s is undefined",ss-1);
00284                     numdol = AddDollar(ss,DOLINDEX,&one,1);
00285                     return(1);
00286                 }
00287                 *in = c;
00288                 x = -numdol;
00289             }
00290             else {
00291                 x = 0;
00292                 while ( FG.cTable[*in] == 1 ) {
00293                     x = 10*x + *in++ - '0';
00294                     if ( x > MAXPOSITIVE4 ) {
00295 illsize:      MesPrint("&Illegal value for base in encode/decode transformation");
00296                     if ( error == 0 ) error = 1;
00297                     return(error);
00298                 }
00299             }
00300             if ( x <= 1 ) goto illsize;
00301         }
00302         if ( *in != ',' && *in != '\0' ) {
00303             MesPrint("&Illegal termination of transformation");
00304             if ( error == 0 ) error = 1;
00305             return(error);
00306         }
00307     }
00308     else {
00309         MesPrint("&Illegal option in encode/decode transformation");
00310         if ( error == 0 ) error = 1;
00311         return(error);
00312     }
00313 /*
00314     Now we can put the whole statement together
00315     We have the set(s) in work up to wp and the range in range.
00316     The base is in x and the type tells whether it is encode or decode.
00317 */
00318     *wp++ = ARGRANGE;
00319     *wp++ = range[0];
00320     *wp++ = range[1];
00321     *wp++ = type;
00322     *wp++ = 4;
00323     *wp++ = BASECODE;
00324     *wp++ = (WORD)x;
00325     *work = wp-work;
00326     work = wp; *wp++ = 0;
00327     s = in;

```



```

00328     }
00329  /*
00330     #] encode/decode :
00331     #[ implode :
00332  */
00333     else if ( StrICmp(s, (UBYTE *)"implode") == 0
00334             || StrICmp(s, (UBYTE *)"tosumnotation") == 0 ) {
00335  /*
00336         Subkeys: ?
00337  */
00338         type = IMplodeARG;
00339         *ss = c;
00340         if ( ( in = ReadRange(in, range, 1) ) == 0 ) {
00341             if ( error == 0 ) error = 1;
00342             return(error);
00343         }
00344         *wp++ = ARGRANGE;
00345         *wp++ = range[0];
00346         *wp++ = range[1];
00347         *wp++ = type;
00348         *work = wp-work;
00349         work = wp; *wp++ = 0;
00350         s = in;
00351     }
00352  /*
00353     #] implode :
00354     #[ explode :
00355  */
00356     else if ( StrICmp(s, (UBYTE *)"explode") == 0
00357             || StrICmp(s, (UBYTE *)"tointegralnotation") == 0 ) {
00358  /*
00359         Subkeys: ?
00360  */
00361         type = EXPLODEARG;
00362         *ss = c;
00363         if ( ( in = ReadRange(in, range, 1) ) == 0 ) {
00364             if ( error == 0 ) error = 1;
00365             return(error);
00366         }
00367         *wp++ = ARGRANGE;
00368         *wp++ = range[0];
00369         *wp++ = range[1];
00370         *wp++ = type;
00371         *work = wp-work;
00372         work = wp; *wp++ = 0;
00373         s = in;
00374     }
00375  /*
00376     #] explode :
00377     #[ permute :
00378  */
00379     else if ( StrICmp(s, (UBYTE *)"permute") == 0 ) {
00380         type = PERMUTEARG;
00381         *ss = c;
00382         *wp++ = ARGRANGE;
00383         *wp++ = 1;
00384         *wp++ = MAXPOSITIVE4;
00385         *wp++ = type;
00386  /*
00387         Now a sequence of cycles
00388  */
00389         do {
00390             wstart = wp; wp++;
00391             do {
00392                 in++;
00393                 if ( *in == '$' ) {
00394                     WORD number; UBYTE *t;
00395                     in++; t = in;
00396                     while ( FG.cTable[*in] < 2 ) in++;
00397                     c = *in; *in = 0;
00398                     if ( ( number = GetDollar(t) ) < 0 ) {
00399                         MesPrint("&Undefined variable %s", t);
00400                         if ( !error ) error = 1;
00401                         number = AddDollar(t, 0, 0, 0);
00402                     }
00403                     *in = c;
00404                     *wp++ = -number-1;
00405                 }
00406             } else {
00407                 x = 0;
00408                 while ( FG.cTable[*in] == 1 ) {
00409                     x = 10*x + *in++ - '0';
00410                     if ( x > MAXPOSITIVE4 ) {
00411                         MesPrint("&value in permute transformation too large");
00412                         if ( error == 0 ) error = 1;
00413                         return(error);
00414                     }

```

```

00415         }
00416         if ( x == 0 ) {
00417             MesPrint("&value 0 in permute transformation not allowed");
00418             if ( error == 0 ) error = 1;
00419             return(error);
00420         }
00421         *wp++ = (WORD)x-1;
00422     }
00423     } while ( *in == ',' );
00424     if ( *in != ')' ) {
00425         MesPrint("&Illegal syntax in permute transformation");
00426         if ( error == 0 ) error = 1;
00427         return(error);
00428     }
00429     in++;
00430     if ( *in != ',' && *in != '(' && *in != '\0' ) {
00431         MesPrint("&Illegal ending in permute transformation");
00432         if ( error == 0 ) error = 1;
00433         return(error);
00434     }
00435     *wstart = wp-wstart;
00436     if ( *wstart == 1 ) wstart--;
00437     } while ( *in == '(' );
00438     *work = wp-work;
00439     work = wp; *wp++ = 0;
00440     s = in;
00441 }
00442 /*
00443 #] permute :
00444 #[ reverse :
00445 */
00446 else if ( StrICmp(s, (UBYTE *) "reverse") == 0 ) {
00447     type = REVERSEARG;
00448     *ss = c;
00449     if ( ( in = ReadRange(in, range, 1) ) == 0 ) {
00450         if ( error == 0 ) error = 1;
00451         return(error);
00452     }
00453     *wp++ = ARGRANGE;
00454     *wp++ = range[0];
00455     *wp++ = range[1];
00456     *wp++ = type;
00457     *work = wp-work;
00458     work = wp; *wp++ = 0;
00459     s = in;
00460 }
00461 /*
00462 #] reverse :
00463 #[ dedup :
00464 */
00465 else if ( StrICmp(s, (UBYTE *) "dedup") == 0 ) {
00466     type = DEDUPARG;
00467     *ss = c;
00468     if ( ( in = ReadRange(in, range, 1) ) == 0 ) {
00469         if ( error == 0 ) error = 1;
00470         return(error);
00471     }
00472     *wp++ = ARGRANGE;
00473     *wp++ = range[0];
00474     *wp++ = range[1];
00475     *wp++ = type;
00476     *work = wp-work;
00477     work = wp; *wp++ = 0;
00478     s = in;
00479 }
00480 /*
00481 #] dedup :
00482 #[ cycle :
00483 */
00484 else if ( StrICmp(s, (UBYTE *) "cycle") == 0 ) {
00485     type = CYCLEARG;
00486     *ss = c;
00487     if ( ( in = ReadRange(in, range, 0) ) == 0 ) {
00488         if ( error == 0 ) error = 1;
00489         return(error);
00490     }
00491     *wp++ = ARGRANGE;
00492     *wp++ = range[0];
00493     *wp++ = range[1];
00494     *wp++ = type;
00495 /*
00496 Now a sequence of cycles
00497 */
00498     in++;
00499     if ( *in == '+' ) {
00500     }
00501     else if ( *in == '-' ) {

```

```

00502         one = -1;
00503     }
00504     else {
00505         MesPrint("&Cycle in a Transform statement should be followed by +/-number/$");
00506         if ( error == 0 ) error = 1;
00507         return(error);
00508     }
00509     in++; x = 0;
00510     if ( *in == '$' ) {
00511         UBYTE *si = in;
00512         in++; si = in;
00513         while ( FG.cTable[*in] == 0 || FG.cTable[*in] == 1 ) in++;
00514         c = *in; *in = 0;
00515         if ( ( x = GetDollar(si) ) < 0 ) {
00516             MesPrint("&Undefined $-variable in transform,cycle statement.");
00517             error = 1;
00518         }
00519         *in = c;
00520         if ( one < 0 ) x += MAXPOSITIVE4;
00521         x += MAXPOSITIVE2;
00522         *wp++ = x;
00523     }
00524     else {
00525         while ( FG.cTable[*in] == 1 ) {
00526             x = 10*x + *in++ - '0';
00527             if ( x > MAXPOSITIVE4 ) {
00528                 MesPrint("&Number in cycle in a Transform statement too big");
00529                 if ( error == 0 ) error = 1;
00530                 return(error);
00531             }
00532         }
00533         *wp++ = x*one;
00534     }
00535     *work = wp-work;
00536     work = wp; *wp++ = 0;
00537     s = in;
00538 }
00539 /*
00540 #] cycle :
00541 #[ islyndon/tolyndon :
00542 */
00543 else if ( StrICmp(s,(UBYTE *)"islyndon" ) == 0 ) {
00544     type = ISLYNDON;
00545     goto doreplace;
00546 }
00547 else if ( StrICmp(s,(UBYTE *)"islyndon<" ) == 0 ) {
00548     type = ISLYNDON;
00549     goto doreplace;
00550 }
00551 else if ( StrICmp(s,(UBYTE *)"islyndon-" ) == 0 ) {
00552     type = ISLYNDON;
00553     goto doreplace;
00554 }
00555 else if ( StrICmp(s,(UBYTE *)"islyndon>" ) == 0 ) {
00556     type = ISLYNDONR;
00557     goto doreplace;
00558 }
00559 else if ( StrICmp(s,(UBYTE *)"islyndon+" ) == 0 ) {
00560     type = ISLYNDONR;
00561     goto doreplace;
00562 }
00563 else if ( StrICmp(s,(UBYTE *)"tolyndon" ) == 0 ) {
00564     type = TOLYNDON;
00565     goto doreplace;
00566 }
00567 else if ( StrICmp(s,(UBYTE *)"tolyndon<" ) == 0 ) {
00568     type = TOLYNDON;
00569     goto doreplace;
00570 }
00571 else if ( StrICmp(s,(UBYTE *)"tolyndon-" ) == 0 ) {
00572     type = TOLYNDON;
00573     goto doreplace;
00574 }
00575 else if ( StrICmp(s,(UBYTE *)"tolyndon>" ) == 0 ) {
00576     type = TOLYNDONR;
00577     goto doreplace;
00578 }
00579 else if ( StrICmp(s,(UBYTE *)"tolyndon+" ) == 0 ) {
00580     type = TOLYNDONR;
00581     goto doreplace;
00582 }
00583 /*
00584 #] islyndon/tolyndon :
00585 #[ addarg :
00586 */
00587 else if ( StrICmp(s,(UBYTE *)"addargs" ) == 0 ) {
00588     type = ADDARG;

```

```

00589         *ss = c;
00590         if ( ( in = ReadRange(in,range,1) ) == 0 ) {
00591             if ( error == 0 ) error = 1;
00592             return(error);
00593         }
00594         *wp++ = ARGGRANGE;
00595         *wp++ = range[0];
00596         *wp++ = range[1];
00597         *wp++ = type;
00598         *work = wp-work;
00599         work = wp; *wp++ = 0;
00600         s = in;
00601     }
00602     /*
00603     #] addarg :
00604     #[ mularg :
00605     */
00606     else if ( ( StrICmp(s,(UBYTE *)"mulargs" ) == 0 )
00607         || ( StrICmp(s,(UBYTE *)"multiplyargs" ) == 0 ) ) {
00608         type = MULTIPLYARG;
00609         *ss = c;
00610         if ( ( in = ReadRange(in,range,1) ) == 0 ) {
00611             if ( error == 0 ) error = 1;
00612             return(error);
00613         }
00614         *wp++ = ARGGRANGE;
00615         *wp++ = range[0];
00616         *wp++ = range[1];
00617         *wp++ = type;
00618         *work = wp-work;
00619         work = wp; *wp++ = 0;
00620         s = in;
00621     }
00622     /*
00623     #] mularg :
00624     #[ droparg :
00625     */
00626     else if ( StrICmp(s,(UBYTE *)"dropargs" ) == 0 ) {
00627         type = DROPARG;
00628         *ss = c;
00629         if ( ( in = ReadRange(in,range,1) ) == 0 ) {
00630             if ( error == 0 ) error = 1;
00631             return(error);
00632         }
00633         *wp++ = ARGGRANGE;
00634         *wp++ = range[0];
00635         *wp++ = range[1];
00636         *wp++ = type;
00637         *work = wp-work;
00638         work = wp; *wp++ = 0;
00639         s = in;
00640     }
00641     /*
00642     #] droparg :
00643     #[ selectarg :
00644     */
00645     else if ( StrICmp(s,(UBYTE *)"selectargs" ) == 0 ) {
00646         type = SELECTARG;
00647         *ss = c;
00648         if ( ( in = ReadRange(in,range,1) ) == 0 ) {
00649             if ( error == 0 ) error = 1;
00650             return(error);
00651         }
00652         *wp++ = ARGGRANGE;
00653         *wp++ = range[0];
00654         *wp++ = range[1];
00655         *wp++ = type;
00656         *work = wp-work;
00657         work = wp; *wp++ = 0;
00658         s = in;
00659     }
00660     /*
00661     #] selectarg :
00662     #[ ZtoH :
00663     */
00664     else if ( StrICmp(s,(UBYTE *)"ztoh" ) == 0 ) {
00665         /*
00666         Subkeys: ?
00667         */
00668         type = ZTOHARG;
00669         *ss = c;
00670         if ( ( in = ReadRange(in,range,1) ) == 0 ) {
00671             if ( error == 0 ) error = 1;
00672             return(error);
00673         }
00674         *wp++ = ARGGRANGE;
00675         *wp++ = range[0];

```

```

00676         *wp++ = range[1];
00677         *wp++ = type;
00678         *work = wp-work;
00679         work = wp; *wp++ = 0;
00680         s = in;
00681     }
00682     /*
00683     #] ZtoH :
00684     #[ HtoZ :
00685     */
00686     else if ( StrICmp(s, (UBYTE *) "htoz") == 0 ) {
00687     /*
00688         Subkeys: ?
00689     */
00690         type = HTOZARG;
00691         *ss = c;
00692         if ( ( in = ReadRange(in, range, 1) ) == 0 ) {
00693             if ( error == 0 ) error = 1;
00694             return(error);
00695         }
00696         *wp++ = ARGGRANGE;
00697         *wp++ = range[0];
00698         *wp++ = range[1];
00699         *wp++ = type;
00700         *work = wp-work;
00701         work = wp; *wp++ = 0;
00702         s = in;
00703     }
00704     /*
00705     #] HtoZ :
00706     */
00707     else {
00708         MesPrint("&Unknown transformation inside a Transform statement: %s", s);
00709         *ss = c;
00710         if ( error == 0 ) error = 1;
00711         return(error);
00712     }
00713     while ( *s == ',' ) s++;
00714 }
00715 AT.WorkPointer[0] = TYPETRANSFORM;
00716 AT.WorkPointer[1] = i = wp - AT.WorkPointer;
00717 AddNtoL(i, AT.WorkPointer);
00718 return(error);
00719 }
00720
00721 /*
00722     #] CoTransform :
00723     #[ RunTransform :
00724
00725     Executes the transform statement.
00726     This routine hunts down the functions and sends them to the various
00727     action routines.
00728     params: size, #set1, ..., #setn, transformations
00729
00730     */
00731
00732 WORD RunTransform(PHEAD WORD *term, WORD *params)
00733 {
00734     WORD *t, *tstop, *w, *m, *out, *in, *tt, retval;
00735     WORD *fun, *args, *info, *infoend, *onetransform, *funs, *endfun;
00736     WORD *thearg = 0, *iterm, *newterm, *nt, *oldwork = AT.WorkPointer, sign = 1;
00737     int i;
00738     out = tstop = term + *term;
00739     tstop -= ABS(tstop[-1]);
00740     in = term;
00741     t = term + 1;
00742     while ( t < tstop ) {
00743         endfun = onetransform = params + *params;
00744         funs = params + 1;
00745         if ( *t < FUNCTION ) {}
00746         else if ( funs == endfun ) { /* we do all functions */
00747             hit:;
00748             while ( in < t ) *out++ = *in++;
00749             tt = t + t[1]; fun = out;
00750             while ( in < tt ) *out++ = *in++;
00751             do {
00752                 args = onetransform + 1;
00753                 info = args; while ( *info <= MAXRANGEINDICATOR ) {
00754                     if ( *info == ALLARGS ) info++;
00755                     else if ( *info == NUMARG ) info += 2;
00756                     else if ( *info == ARGGRANGE ) info += 3;
00757                     else if ( *info == MAKEARGS ) info += 3;
00758                 }
00759                 switch ( *info ) {
00760                     case REPLACEARG:
00761                         if ( RunReplace(BHEAD fun, args, info) ) goto abo;
00762                         out = fun + fun[1];

```

```

00763         break;
00764     case ENCODEARG:
00765         if ( RunEncode(BHEAD fun,args,info) ) goto abo;
00766         out = fun + fun[1];
00767         break;
00768     case DECODEARG:
00769         if ( RunDecode(BHEAD fun,args,info) ) goto abo;
00770         out = fun + fun[1];
00771         break;
00772     case IMplodeARG:
00773         if ( RunImplode(fun,args) ) goto abo;
00774         out = fun + fun[1];
00775         break;
00776     case EXPLODEARG:
00777         if ( RunExplode(BHEAD fun,args) ) goto abo;
00778         out = fun + fun[1];
00779         break;
00780     case PERMUTEARG:
00781         if ( RunPermute(BHEAD fun,args,info) ) goto abo;
00782         out = fun + fun[1];
00783         break;
00784     case REVERSEARG:
00785         if ( RunReverse(BHEAD fun,args) ) goto abo;
00786         out = fun + fun[1];
00787         break;
00788     case DEDUPARG:
00789         if ( RunDedup(BHEAD fun,args) ) goto abo;
00790         out = fun + fun[1];
00791         break;
00792     case CYCLEARG:
00793         if ( RunCycle(BHEAD fun,args,info) ) goto abo;
00794         out = fun + fun[1];
00795         break;
00796     case ADDARG:
00797         if ( RunAddArg(BHEAD fun,args) ) goto abo;
00798         out = fun + fun[1];
00799         break;
00800     case MULTIPLYARG:
00801         if ( RunMulArg(BHEAD fun,args) ) goto abo;
00802         out = fun + fun[1];
00803         break;
00804     case ISLYNDON:
00805         if ( ( retval = RunIsLyndon(BHEAD fun,args,1) ) < -1 ) goto abo;
00806         goto returnvalues;
00807         break;
00808     case ISLYNDONR:
00809         if ( ( retval = RunIsLyndon(BHEAD fun,args,-1) ) < -1 ) goto abo;
00810         goto returnvalues;
00811         break;
00812     case TOLYNDON:
00813         if ( ( retval = RunToLyndon(BHEAD fun,args,1) ) < -1 ) goto abo;
00814         goto returnvalues;
00815         break;
00816     case TOLYNDONR:
00817         if ( ( retval = RunToLyndon(BHEAD fun,args,-1) ) < -1 ) goto abo;
00818     returnvalues;;
00819
00820         out = fun + fun[1];
00821         if ( retval == -1 ) break;
00822
00823         /*
00824         Work out the yes/no stuff
00825
00826         AT.WorkPointer += 2*AM.MaxTer;
00827         if ( AT.WorkPointer > AT.WorkTop ) {
00828             MLOCK(ErrorMessageLock);
00829             MesWork();
00830             MUNLOCK(ErrorMessageLock);
00831             return(-1);
00832         }
00833         item = AT.WorkPointer;
00834         info++;
00835         for ( i = 0; i < *info; i++ ) item[i] = info[i];
00836         AT.WorkPointer = item + *item;
00837         AR.Eside = LHSIDEX;
00838         NewSort(BHEAD0);
00839         if ( Generator(BHEAD item,AR.Cnumlhs) ) {
00840             LowerSortLevel();
00841             AT.WorkPointer = oldwork;
00842             return(-1);
00843         }
00844         newterm = AT.WorkPointer;
00845         if ( EndSort(BHEAD newterm,1) < 0 ) {}
00846         if ( ( *newterm && *(newterm+*newterm) != 0 ) || *newterm == 0 ) {
00847             MLOCK(ErrorMessageLock);
00848             MesPrint("&yes/no information in islyndon/tolyndon does not evaluate into
a single term");
00849             MUNLOCK(ErrorMessageLock);
00850             return(-1);

```

```

00849     }
00850     AR.Eside = RHSIDE;
00851     i = *newterm; tt = iterm; nt = newterm;
00852     NCOPY(tt,nt,i);
00853     AT.WorkPointer = iterm + *iterm;
00854     info = iterm + 1;
00855     infoend = info+info[1];
00856     info += FUNHEAD;
00857
00858     if ( retval == 0 ) {
00859         /*
00860             Need second argument (=no)
00861         */
00862         if ( info >= infoend ) {
00863             abortlyndon;;
00864             MLOCK(ErrorMessageLock);
00865             MesPrint("There should be a yes and a no argument in
00866                 islyndon/tolyndon");
00867             MUNLOCK(ErrorMessageLock);
00868             Terminate(-1);
00869         }
00870         NEXTARG(info)
00871         if ( info >= infoend ) goto abortlyndon;
00872         thearg = info;
00873     }
00874     else if ( retval == 1 ) {
00875         /*
00876             Need first argument (=yes)
00877         */
00878         if ( info >= infoend ) goto abortlyndon;
00879         thearg = info;
00880         NEXTARG(info)
00881         if ( info >= infoend ) goto abortlyndon;
00882     }
00883     NEXTARG(info)
00884     if ( info < infoend ) goto abortlyndon;
00885     /*
00886         The argument in thearg needs to be copied
00887         We did not pull it through generator to guarantee
00888         that it is a single argument.
00889         The easiest way is to let the routine Normalize
00890         do the job and put everything in an exponent function
00891         with the power one.
00892     */
00893     if ( *thearg == -SNUMBER && thearg[1] == 0 ) {
00894         *term = 0; return(0);
00895     }
00896     if ( *thearg == -SNUMBER && thearg[1] == 1 ) { }
00897     else {
00898         fun = out;
00899         *out++ = EXPONENT; out++; *out++ = 1; FILLFUN3(out);
00900         COPY1ARG(out,thearg);
00901         *out++ = -SNUMBER; *out++ = 1;
00902         fun[1] = out-fun;
00903     }
00904     break;
00905 case DROPARG:
00906     if ( RunDropArg(BHEAD fun,args) ) goto abo;
00907     out = fun + fun[1];
00908     break;
00909 case SELECTARG:
00910     if ( RunSelectArg(BHEAD fun,args) ) goto abo;
00911     out = fun + fun[1];
00912     break;
00913 case ZTOHARG:
00914     {
00915         WORD s = RunZtoHArg(BHEAD fun,args);
00916         if ( s < 0 ) goto abo;
00917         if ( s == 1 ) sign = -sign;
00918         out = fun + fun[1];
00919     }
00920     break;
00921 case HTOZARG:
00922     {
00923         WORD s = RunHtoZArg(BHEAD fun,args);
00924         if ( s < 0 ) goto abo;
00925         if ( s == 1 ) sign = -sign;
00926         out = fun + fun[1];
00927     }
00928     break;
00929 default:
00930     MLOCK(ErrorMessageLock);
00931     MesPrint("Irregular code in execution of transform statement");
00932     MUNLOCK(ErrorMessageLock);
00933     Terminate(-1);
00934 }
00935 onetransform += *onetransform;

```

```

00935         } while ( *onetransform );
00936     }
00937     else {
00938         while ( funs < endfun ) { /* sum over sets */
00939             if ( *funs > MAXVARIABLES ) {
00940                 if ( *t == *funs-MAXVARIABLES ) goto hit;
00941             }
00942             else {
00943                 w = SetElements + Sets[*funs].first;
00944                 m = SetElements + Sets[*funs].last;
00945                 while ( w < m ) { /* sum over set elements */
00946                     if ( *w == *t ) goto hit;
00947                     w++;
00948                 }
00949             }
00950             funs++;
00951         }
00952     }
00953     t += t[1];
00954 }
00955 tt = term + *term; while ( in < tt ) *out++ = *in++;
00956 if ( sign == -1 ) out[-1] = -out[-1];
00957 *tt = i = out - tt;
00958 /*
00959 Now copy the whole thing back
00960 */
00961 NCOPY(term,tt,i)
00962 return(0);
00963 abo:
00964 MLOCK(ErrorMessageLock);
00965 MesCall("RunTransform");
00966 MUNLOCK(ErrorMessageLock);
00967 return(-1);
00968 }
00969
00970 /*
00971 #] RunTransform :
00972 #[ RunEncode :
00973
00974 The info is given by
00975     ENCODEARG,size,BASECODE,num
00976 and possibly more codes to follow.
00977 Only one range is allowed and for now, it should be fully numerical
00978 If the range is in reverse order, we need to either revert it
00979 first or work with an array of pointers.
00980 */
00981
00982 WORD RunEncode(PHEAD WORD *fun, WORD *args, WORD *info)
00983 {
00984     WORD base, *f, *funstop, *fun1, *t, size1, size2, size3, *arg;
00985     int num, num1, num2, n, i, i1, i2;
00986     UWORD *scrat1, *scrat2, *scrat3;
00987     WORD *tt, *tstop, totarg, arg1, arg2;
00988     if ( functions[fun[0]-FUNCTION].spec != 0 ) return(0);
00989     if ( *args != ARG RANGE ) {
00990         MLOCK(ErrorMessageLock);
00991         MesPrint("Illegal range encountered in RunEncode");
00992         MUNLOCK(ErrorMessageLock);
00993         Terminate(-1);
00994     }
00995     tt = fun+FUNHEAD; tstop = fun+fun[1]; totarg = 0;
00996     while ( tt < tstop ) { totarg++; NEXTARG(tt); }
00997     if ( FindRange(BHEAD args,&arg1,&arg2,totarg) ) return(-1);
00998     if ( arg1 > totarg || arg2 > totarg ) return(0);
00999
01000     if ( info[2] == BASECODE ) {
01001         base = info[3];
01002         if ( base <= 0 ) { /* is a dollar variable */
01003             i1 = -base;
01004             base = DolToNumber(BHEAD i1);
01005             if ( AN.ErrorInDollar || base < 2 ) {
01006                 MLOCK(ErrorMessageLock);
01007                 MesPrint("$%s does not have a number value > 1 in base/encode/transform statement in
module %1",
01008                     DOLLARNAME(Dollars,i1),AC.CModule);
01009                 MUNLOCK(ErrorMessageLock);
01010                 Terminate(-1);
01011             }
01012         }
01013     }
01014 /*
01015 Compute number of pointers needed and make sure there is space
01016 */
01017 if ( arg1 > arg2 ) { num1 = arg2; num2 = arg1; }
01018 else { num1 = arg1; num2 = arg2; }
01019 num = num2-num1+1;
01020 WantAddPointers(num);
01021 */

```



```

01021         Collect the pointers in pWorkspace
01022     */
01023     n = 1; funstop = fun+fun[1]; f = fun+FUNHEAD;
01024     while ( n < num1 ) {
01025         if ( f >= funstop ) return(0);
01026         NEXTARG(f);
01027         n++;
01028     }
01029     fun1 = f; i = 0;
01030     while ( n <= num2 ) {
01031         if ( f >= funstop ) return(0);
01032         if ( *f != -SNUMBER ) {
01033             if ( *f < 0 ) return(0);
01034             t = f + *f - 1;
01035             i1 = ABS(*t);
01036             if ( (*f-i1) != (ARGHEAD+1) ) return(0); /* Not numerical */
01037             i1 = (i1-1)/2 - 1;
01038             t--;
01039             while ( i1 > 0 ) {
01040                 if ( *t != 0 ) return(0); /* Not an integer */
01041                 t--; i1--;
01042             }
01043         }
01044         AT.pWorkspace[AT.pWorkPointer+i] = f;
01045         i++;
01046         NEXTARG(f);
01047         n++;
01048     }
01049     /*
01050     f points now to after the arguments; fun1 at the first.
01051     Now check whether we need to revert the order
01052     */
01053     if ( arg1 > arg2 ) {
01054         i1 = 0; i2 = i-1;
01055         while ( i1 < i2 ) {
01056             t = AT.pWorkspace[AT.pWorkPointer+i1];
01057             AT.pWorkspace[AT.pWorkPointer+i1] = AT.pWorkspace[AT.pWorkPointer+i2];
01058             AT.pWorkspace[AT.pWorkPointer+i2] = t;
01059             i1++; i2--;
01060         }
01061     }
01062     /*
01063     Now we can put the thing together.
01064     x = arg1;
01065     x = base*x+arg2
01066     x = base*x+arg3 etc.
01067     We need three scratch arrays for long integers
01068         (see NumberMalloc in tools.c).
01069     */
01070     scrat1 = NumberMalloc("RunEncode");
01071     scrat2 = NumberMalloc("RunEncode");
01072     scrat3 = NumberMalloc("RunEncode");
01073     arg = AT.pWorkspace[AT.pWorkPointer];
01074     size1 = PutArgInScratch(arg,scrat1);
01075     i--;
01076     while ( i > 0 ) {
01077         if ( MulLong(scrat1,size1,(UWORD *)(&base),1,scrat2,&size2) ) {
01078             NumberFree(scrat3,"RunEncode");
01079             NumberFree(scrat2,"RunEncode");
01080             NumberFree(scrat1,"RunEncode");
01081             goto CalledFrom;
01082         }
01083         NEXTARG(arg);
01084         size3 = PutArgInScratch(arg,scrat3);
01085         if ( AddLong(scrat2,size2,scrat3,size3,scrat1,&size1) ) {
01086             NumberFree(scrat3,"RunEncode");
01087             NumberFree(scrat2,"RunEncode");
01088             NumberFree(scrat1,"RunEncode");
01089             goto CalledFrom;
01090         }
01091         i--;
01092     }
01093     /*
01094     Now put the output in place. There are two cases, one being much
01095     faster than the other. Hence we program both.
01096     Fast: it fits inside the old location.
01097     Slow: it does not.
01098     The total space is f-fun1
01099     */
01100     if ( size1 == 0 ) { /* Fits! */
01101         *fun1++ = -SNUMBER; *fun1++ = 0;
01102         while ( f < funstop ) *fun1++ = *f++;
01103         fun[1] = funstop-fun;
01104     }
01105     else if ( size1 == 1 && scrat1[0] <= MAXPOSITIVE ) { /* Fits! */
01106         *fun1++ = -SNUMBER; *fun1++ = scrat1[0];
01107         while ( f < funstop ) *fun1++ = *f++;

```

```

01108         fun[l] = funl-fun;
01109     }
01110     else if ( sizel == -1 && scratl[0] <= MAXPOSITIVE+1 ) { /* Fits! */
01111         *funl++ = -SNUMBER;
01112         if ( scratl[0] < MAXPOSITIVE ) *funl++ = scratl[0];
01113         else *funl++ = (WORD) (MAXPOSITIVE+1);
01114         while ( f < funstop ) *funl++ = *f++;
01115         fun[l] = funl-fun;
01116     }
01117     else if ( ABS(sizel)*2+2+ARGHEAD <= f-funl ) { /* Fits! */
01118         if ( sizel < 0 ) { size2 = sizel*2-1; sizel = -sizel; size3 = -size2; }
01119         else { size2 = 2*sizel+1; size3 = size2; }
01120         *funl++ = size3+ARGHEAD+1;
01121         *funl++ = 0; FILLARG(funl);
01122         *funl++ = size3+1;
01123         for ( i = 0; i < sizel; i++ ) *funl++ = scratl[i];
01124         *funl++ = 1;
01125         for ( i = 1; i < sizel; i++ ) *funl++ = 0;
01126         *funl++ = size2;
01127         while ( f < funstop ) *funl++ = *f++;
01128         fun[l] = funl-fun;
01129     }
01130     else { /* Does not fit */
01131         t = funstop;
01132         if ( sizel < 0 ) { size2 = sizel*2-1; sizel = -sizel; size3 = -size2; }
01133         else { size2 = 2*sizel+1; size3 = size2; }
01134         *t++ = size3+ARGHEAD+1;
01135         *t++ = 0; FILLARG(t);
01136         *t++ = size3+1;
01137         for ( i = 0; i < sizel; i++ ) *t++ = scratl[i];
01138         *t++ = 1;
01139         for ( i = 1; i < sizel; i++ ) *t++ = 0;
01140         *t++ = size2;
01141         while ( f < funstop ) *t++ = *f++;
01142         f = funstop;
01143         while ( f < t ) *funl++ = *f++;
01144         fun[l] = funl-fun;
01145     }
01146     NumberFree(scrat3,"RunEncode");
01147     NumberFree(scrat2,"RunEncode");
01148     NumberFree(scratl,"RunEncode");
01149 }
01150 else {
01151     MLOCK(ErrorMessageLock);
01152     MesPrint("Unimplemented type of encoding encountered in RunEncode");
01153     MUNLOCK(ErrorMessageLock);
01154     Terminate(-1);
01155 }
01156 return(0);
01157 CalledFrom:
01158 MLOCK(ErrorMessageLock);
01159 MesCall("RunEncode");
01160 MUNLOCK(ErrorMessageLock);
01161 return(-1);
01162 }
01163
01164 /*
01165     #] RunEncode :
01166     #[ RunDecode :
01167 */
01168
01169 WORD RunDecode(PHEAD WORD *fun, WORD *args, WORD *info)
01170 {
01171     WORD base, num, num1, num2, n, *f, *funstop, *funl, sizel, size2, size3, *t;
01172     WORD i1, i2, i, sig;
01173     UWORD *scratl, *scrat2, *scrat3;
01174     WORD *tt, *tstop, totarg, arg1, arg2;
01175     if ( functions[fun[0]-FUNCTION].spec != 0 ) return(0);
01176     if ( *args != ARGRANGE ) {
01177         MLOCK(ErrorMessageLock);
01178         MesPrint("Illegal range encountered in RunDecode");
01179         MUNLOCK(ErrorMessageLock);
01180         Terminate(-1);
01181     }
01182     tt = fun+FUNHEAD; tstop = fun+fun[l]; totarg = 0;
01183     while ( tt < tstop ) { totarg++; NEXTARG(tt); }
01184     if ( FindRange(BHEAD args,&arg1,&arg2,totarg) ) return(-1);
01185     if ( arg1 > totarg && arg2 > totarg ) return(0);
01186     if ( info[2] == BASECODE ) {
01187         base = info[3];
01188         if ( base <= 0 ) { /* is a dollar variable */
01189             i1 = -base;
01190             base = DolToNumber(BHEAD i1);
01191             if ( AN.ErrorInDollar || base < 2 ) {
01192                 MLOCK(ErrorMessageLock);
01193                 MesPrint("%$s does not have a number value > 1 in base/decode/transform statement in
module %1",

```

```

01194             DOLLARNAME(Dollars,i1),AC.CModule);
01195             MUNLOCK(ErrorMessageLock);
01196             Terminate(-1);
01197         }
01198     }
01199     /*
01200     Compute number of output arguments needed
01201     */
01202     if ( arg1 > arg2 ) { num1 = arg2; num2 = arg1; }
01203     else { num1 = arg1; num2 = arg2; }
01204     num = num2-num1+1;
01205     if ( num <= 1 ) return(0);
01206     /*
01207     Find argument num1
01208     */
01209     funstop = fun + fun[1];
01210     f = fun + FUNHEAD; n = 1;
01211     while ( f < funstop ) {
01212         if ( n == num1 ) break;
01213         NEXTARG(f); n++;
01214     }
01215     if ( f >= funstop ) return(0); /* not enough arguments */
01216     /*
01217     Check that f is integer
01218     */
01219     if ( *f == -SNUMBER ) {}
01220     else if ( *f < 0 ) return(0);
01221     else {
01222         t = f + *f - 1;
01223         i1 = ABS(*t);
01224         if ( (*f-i1) != (ARGHEAD+1) ) return(0); /* Not numerical */
01225         i1 = (i1-1)/2 - 1;
01226         t--;
01227         while ( i1 > 0 ) {
01228             if ( *t != 0 ) return(0); /* Not an integer */
01229             t--; i1--;
01230         }
01231     }
01232     fun1 = f;
01233     /*
01234     The argument that should be decoded is in fun1
01235     We have to copy it to scratch
01236     */
01237     scrat1 = NumberMalloc("RunEncode");
01238     scrat2 = NumberMalloc("RunEncode");
01239     scrat3 = NumberMalloc("RunEncode");
01240     size1 = PutArgInScratch(fun1,scrat1);
01241     if ( size1 < 0 ) { sig = -1; size1 = -size1; }
01242     else sig = 1;
01243     /*
01244     We can check first whether this number can be decoded
01245     */
01246     scrat2[0] = base; size2 = 1;
01247     if ( RaisPow(BHEAD scrat2,&size2,num) ) {
01248         NumberFree(scrat3,"RunEncode");
01249         NumberFree(scrat2,"RunEncode");
01250         NumberFree(scrat1,"RunEncode");
01251         goto CalledFrom;
01252     }
01253     if ( BigLong(scrat1,size1,scrat2,size2) >= 0 ) { /* Number too big */
01254         NumberFree(scrat3,"RunEncode");
01255         NumberFree(scrat2,"RunEncode");
01256         NumberFree(scrat1,"RunEncode");
01257         return(0);
01258     }
01259     /*
01260     We need num*2 spaces
01261     */
01262     if ( *fun1 > num*2 ) { /* shrink space */
01263         t = fun1 + 2*num; f = fun1 + *fun1;
01264         while ( f < funstop ) *t++ = *f++;
01265         fun[1] = t - fun;
01266     }
01267     else if ( *fun1 < num*2 ) { /* case includes -SNUMBER */
01268         if ( *fun1 < 0 ) { /* expand space from -SNUMBER */
01269             fun[1] += (num-1)*2;
01270             t = funstop + (num-1)*2;
01271         }
01272         else { /* expand space from general argument */
01273             fun[1] += 2*num - *fun1;
01274             t = funstop + 2*num - *fun1;
01275         }
01276         f = funstop;
01277         while ( f > fun1 ) *--t = *--f;
01278     }
01279     /*
01280     Now there is space for num -SNUMBER arguments filled from the top.

```

```

01281 */
01282     for ( i = num-1; i >= 0; i-- ) {
01283         DivLong(scrat1,size1,(UWORD *)(&base),1,scrat2,&size2,scrat3,&size3);
01284         fun1[2*i] = -SNUMBER;
01285         if ( size3 == 0 ) fun1[2*i+1] = 0;
01286         else fun1[2*i+1] = (WORD)(scrat3[0])*sig;
01287         for ( i1 = 0; i1 < size2; i1++ ) scrat1[i1] = scrat2[i1];
01288         size1 = size2;
01289     }
01290     if ( size2 != 0 ) {
01291         MLOCK(ErrorMessageLock);
01292         MesPrint("RunDecode: number to be decoded is too big");
01293         MUNLOCK(ErrorMessageLock);
01294         NumberFree(scrat3,"RunEncode");
01295         NumberFree(scrat2,"RunEncode");
01296         NumberFree(scrat1,"RunEncode");
01297         goto CalledFrom;
01298     }
01299 /*
01300     Now check whether we should change the order of the arguments
01301 */
01302     if ( arg1 > arg2 ) {
01303         i1 = 1; i2 = 2*num-1;
01304         while ( i2 > i1 ) {
01305             i = fun1[i1]; fun1[i1] = fun1[i2]; fun1[i2] = i;
01306             i1 += 2; i2 -= 2;
01307         }
01308     }
01309     NumberFree(scrat3,"RunEncode");
01310     NumberFree(scrat2,"RunEncode");
01311     NumberFree(scrat1,"RunEncode");
01312 }
01313 else {
01314     MLOCK(ErrorMessageLock);
01315     MesPrint("Unimplemented type of encoding encountered in RunDecode");
01316     MUNLOCK(ErrorMessageLock);
01317     Terminate(-1);
01318 }
01319 return(0);
01320 CalledFrom:
01321     MLOCK(ErrorMessageLock);
01322     MesCall("RunDecode");
01323     MUNLOCK(ErrorMessageLock);
01324     return(-1);
01325 }
01326
01327 /*
01328     #] RunDecode :
01329     #[ RunReplace :
01330
01331     Gets the function, passes the arguments and looks whether they
01332     need to be treated. If so, the exact treatment is found in info.
01333     The info is given as if it is a function of type REPLACEMENT but
01334     its name is REPLACEARG (which is NOT a function).
01335     It is performed on the arguments.
01336     The output is at first written after fun and in the end overwrites fun.
01337 */
01338
01339 WORD RunReplace(PHEAD WORD *fun, WORD *args, WORD *info)
01340 {
01341     int n = 0, i, dirty = 0, totarg, nfix, nwild, ngeneral;
01342     WORD *t, *tt, *u, *tstop, *info1, *infoend, *oldwork = AT.WorkPointer;
01343     WORD *term, *newterm, *nt, *term1, *term2;
01344     WORD wild[4], mask, *term3, *term4, *oldmask = AT.WildMask;
01345     WORD n1, n2, doanyway;
01346     info++;
01347     t = fun; tstop = fun + fun[1]; u = tstop;
01348     for ( i = 0; i < FUNHEAD; i++ ) *u++ = *t++;
01349     tt = t;
01350     if ( functions[fun[0]-FUNCTION].spec != TENSORFUNCTION ) {
01351         totarg = 0;
01352         while ( tt < tstop ) { totarg++; NEXTARG(tt); }
01353     }
01354     else {
01355         totarg = tstop - tt;
01356     }
01357 /*
01358     Now get the info through Generator to bring it to standard form.
01359     info points at a single term that should be sent to Generator.
01360
01361     We want to put the information in the WorkSpace but fun etc lies there
01362     already. This means that we have to move the WorkPointer quite high up.
01363 */
01364     AT.WorkPointer += 2*AM.MaxTer;
01365     if ( AT.WorkPointer > AT.WorkTop ) {
01366         MLOCK(ErrorMessageLock);
01367         MesWork();

```

```

01368     MUNLOCK(ErrorMessageLock);
01369     return(-1);
01370 }
01371 term = AT.WorkPointer;
01372 for ( i = 0; i < *info; i++ ) term[i] = info[i];
01373 AT.WorkPointer = term + *term;
01374 AR.Eside = LHSIDEX;
01375 NewSort(BHEAD0);
01376 if ( Generator(BHEAD term,AR.Cnumlhs) ) {
01377     LowerSortLevel();
01378     AT.WorkPointer = oldwork;
01379     return(-1);
01380 }
01381 newterm = AT.WorkPointer;
01382 if ( EndSort(BHEAD newterm,1) < 0 ) {}
01383 if ( ( *newterm && *(newterm+*newterm) != 0 ) || *newterm == 0 ) {
01384     MLOCK(ErrorMessageLock);
01385     MesPrint("&information in replace transformation does not evaluate into a single term");
01386     MUNLOCK(ErrorMessageLock);
01387     return(-1);
01388 }
01389 AR.Eside = RHSIDE;
01390 i = *newterm; tt = term; nt = newterm;
01391 NCOPY(tt,nt,i);
01392 AT.WorkPointer = term + *term;
01393 info = term + 1;
01394
01395 term1 = term + *term;
01396 term2 = term1+1;
01397 *term2++ = REPLACEMENT;
01398 term2++; FILLFUN(term2)
01399 /*
01400 First we count the different types of objects
01401 */
01402 infoend = info + info[1];
01403 info1 = info + FUNHEAD;
01404 nfix = nwild = ngeneral = 0;
01405 while ( info1 < infoend ) {
01406     if ( *info1 == -SNUMBER ) {
01407         nfix++;
01408         info1 += 2; NEXTARG(info1)
01409     }
01410     else if ( *info1 <= -FUNCTION ) {
01411         if ( *info1 == -WILDARGFUN ) {
01412             nwild++;
01413             info1++; NEXTARG(info1)
01414         }
01415         else {
01416             *term2++ = *info1++; COPY1ARG(term2,info1)
01417             ngeneral++;
01418         }
01419     }
01420     else if ( *info1 == -INDEX ) {
01421         if ( info1[1] == WILDARGINDEX + AM.OffsetIndex ) {
01422             nwild++;
01423             info1 += 2; NEXTARG(info1)
01424         }
01425         else {
01426             *term2++ = *info1++; *term2++ = *info1++; COPY1ARG(term2,info1)
01427             ngeneral++;
01428         }
01429     }
01430     else if ( *info1 == -SYMBOL ) {
01431         if ( info1[1] == WILDARGSYMBOL ) {
01432             nwild++;
01433             info1 += 2; NEXTARG(info1)
01434         }
01435         else {
01436             *term2++ = *info1++; *term2++ = *info1++; COPY1ARG(term2,info1)
01437             ngeneral++;
01438         }
01439     }
01440     else if ( *info1 == -MINVECTOR || *info1 == -VECTOR ) {
01441         if ( info1[1] == WILDARGVECTOR + AM.OffsetVector ) {
01442             nwild++;
01443             info1 += 2; NEXTARG(info1)
01444         }
01445         else {
01446             *term2++ = *info1++; *term2++ = *info1++; COPY1ARG(term2,info1)
01447             ngeneral++;
01448         }
01449     }
01450     else {
01451         MLOCK(ErrorMessageLock);
01452         MesPrint("&irregular code found in replace transformation (RunReplace)");
01453         MUNLOCK(ErrorMessageLock);
01454         Terminate(-1);

```

```

01455     }
01456 }
01457 AT.WorkPointer = term2;
01458 *term1 = term2 - term1;
01459 term1[2] = *term1 - 1;
01460 /*
01461 And now stepping through the arguments
01462 */
01463 while ( t < tstop ) {
01464     n++; /* The number of the argument. Now check whether we need it */
01465     if ( TestArgNum(n,totarg,args) == 0 ) {
01466         if ( functions[fun[0]-FUNCTION].spec != TENSORFUNCTION ) {
01467             if ( *t <= -FUNCTION ) { *u++ = *t++; }
01468             else if ( *t < 0 ) { *u++ = *t++; *u++ = *t++; }
01469             else { i = *t; NCOPY(u,t,i) }
01470         }
01471         else *u++ = *t++;
01472         continue;
01473     }
01474 /*
01475 Here we have in info effectively a replace_ function, but with
01476 additionally the possibility of integer arguments. We treat those first
01477 and for the rest we have to do some pattern matching.
01478 Note that the compilation routine should check that there is an
01479 even number of arguments in the replace function.
01480
01481 First we go for number -> something
01482 */
01483 doanyway = 0;
01484 if ( nfix > 0 ) {
01485     if ( functions[fun[0]-FUNCTION].spec != TENSORFUNCTION ) {
01486         if ( *t == -SNUMBER ) {
01487             info1 = info + FUNHEAD;
01488             while ( info1 < infoend ) {
01489                 if ( *info1 == -SNUMBER ) {
01490                     if ( info1[1] == t[1] ) {
01491                         if ( info1[2] == -SNUMBER ) {
01492                             *u++ = -SNUMBER; *u++ = info1[3];
01493                             info1 += 4;
01494                         }
01495                         else {
01496                             info1 += 2;
01497                             if ( info1[0] <= -FUNCTION ) i = 1;
01498                             else if ( info1[0] < 0 ) i = 2;
01499                             else i = *info1;
01500                             NCOPY(u,info1,i)
01501                         }
01502                         t += 2; goto nextt;
01503                     }
01504                     info1 += 2;
01505                     NEXTARG(info1);
01506                 }
01507                 else {
01508                     NEXTARG(info1);
01509                     NEXTARG(info1);
01510                 }
01511             }
01512 /*
01513 Here we had no match in the style of 1->2. It could however
01514 be that xarg_ does something
01515 */
01516 doanyway = 1; n2 = t[1];
01517 }
01518 }
01519 else { /* Tensor */
01520     if ( *t < AM.OffsetIndex && *t >= 0 ) {
01521         info1 = info + FUNHEAD;
01522         while ( info1 < infoend ) {
01523             if ( ( *info1 == -SNUMBER ) && ( info1[1] == *t )
01524                 && ( ( info1[2] == -SNUMBER ) && ( info1[3] >= 0 )
01525                     && ( info1[3] < AM.OffsetIndex )
01526                     || ( info1[2] == -INDEX || info1[2] == -VECTOR
01527                         || info1[2] == -MINVECTOR ) ) ) {
01528                 *u++ = info1[3];
01529                 info1 += 4;
01530                 t++; goto nextt;
01531             }
01532             else {
01533                 NEXTARG(info1);
01534                 NEXTARG(info1);
01535             }
01536         }
01537     }
01538 }
01539 }
01540 else if ( *t == -SNUMBER ) {
01541     doanyway = 1; n2 = t[1];

```

```

01542     }
01543     /*
01544     First we try to catch those elements that have an exact match
01545     in the traditional replace_ part.
01546     This means that *t should be less than zero and match an entry
01547     in the replace_ function that we prepared.
01548     */
01549     if ( ngeneral > 0 ) {
01550         if ( functions[fun[0]-FUNCTION].spec != TENSORFUNCTION ) {
01551             if ( *t < 0 ) {
01552                 term3 = term1 + *term1;
01553                 term4 = term1 + FUNHEAD;
01554                 while ( term4 < term3 ) {
01555                     if ( *term4 == *t && ( *t <= -FUNCTION ||
01556                         ( t[1] == term4[1] ) ) ) break;
01557                     NEXTARG(term4)
01558                 }
01559                 if ( term4 < term3 ) goto dothisnow;
01560             }
01561         }
01562         else {
01563             term3 = term1 + *term1;
01564             term4 = term1 + FUNHEAD;
01565             while ( term4 < term3 ) {
01566                 if ( ( term4[1] == *t ) &&
01567                     ( ( *term4 == -INDEX || *term4 == -VECTOR ||
01568                       ( *term4 == -SYMBOL && term4[1] < AM.OffsetIndex
01569                         && term4[1] >= 0 ) ) ) ) break;
01570                 NEXTARG(term4)
01571             }
01572             if ( term4 < term3 ) goto dothisnow;
01573         }
01574     }
01575     /*
01576     First we eliminate the fixed arguments and make a 'new info'
01577     If there is anything left we can continue.
01578     Now we look for whole argument wildcards (arg_, parg_, iarg_ or farg_)
01579     */
01580     if ( nwild > 0 ) {
01581         /*
01582         If we have f(a)*replace_(xarg_,b(xarg_)) this gives f(b(a))
01583         In testing the wildcard we have CheckWild do the work.
01584         This means that we have to set up the special variables
01585         (AT.WildMask,AN.WildValue,AN.NumWild)
01586         */
01587         wild[1] = 4;
01588         info1 = info + FUNHEAD;
01589         while ( info1 < infoend ) {
01590             if ( *info1 == -SYMBOL && info1[1] == WILDARGSYMBOL
01591                 && ( functions[fun[0]-FUNCTION].spec != TENSORFUNCTION ) ) {
01592                 wild[0] = SYMTOSUB;
01593                 wild[2] = WILDARGSYMBOL;
01594                 wild[3] = 0;
01595                 AN.WildValue = wild;
01596                 AT.WildMask = &mask;
01597                 mask = 0;
01598                 AN.NumWild = 1;
01599                 if ( *t == -SYMBOL || ( *t > 0 && CheckWild(BHEAD WILDARGSYMBOL,SYMTOSUB,1,t) == 0
01600                     || doanyway ) ) {
01601                     || doanyway ) {
01602                         /*
01603                         We put the part in replace in a function and make
01604                         a replace_(xarg_,(t argument)).
01605                         */
01606                         n1 = SYMBOL; n2 = WILDARGSYMBOL;
01607                         info1 += 2;
01608                         getthisone;;
01609                         term3 = term2+1;
01610                         if ( functions[fun[0]-FUNCTION].spec != TENSORFUNCTION ) {
01611                             *term3++ = DUMFUN; term3++; FILLFUN(term3)
01612                             COPY1ARG(term3,info1)
01613                         }
01614                         else {
01615                             *term3++ = fun[0]; term3++; FILLFUN(term3)
01616                             *term3++ = *info1;
01617                         }
01618                         term2[2] = term3 - term2 - 1;
01619                         tt = term3;
01620                         *term3++ = REPLACEMENT;
01621                         term3++; FILLFUN(term3)
01622                         *term3++ = -n1;
01623                         if ( n1 < FUNCTION ) *term3++ = n2;
01624                         if ( functions[fun[0]-FUNCTION].spec != TENSORFUNCTION ) {
01625                             term4 = t;
01626                             COPY1ARG(term3,term4)
01627                         }

```

```

01628         else {
01629             *term3++ = *t;
01630         }
01631         tt[1] = term3 - tt;
01632         *term3++ = 1; *term3++ = 1; *term3++ = 3;
01633         *term2 = term3 - term2;
01634
01635         AT.WorkPointer = term3;
01636         NewSort(BHEAD0);
01637         if ( Generator(BHEAD term2,AR.Cnumlhs) ) {
01638             LowerSortLevel();
01639             AT.WorkPointer = oldwork;
01640             AT.WildMask = oldmask;
01641             return(-1);
01642         }
01643         term4 = AT.WorkPointer;
01644         if ( EndSort(BHEAD term4,1) < 0 ) {}
01645         if ( ( *term4 && *(term4+*term4) != 0 ) || *term4 == 0 ) {
01646             MLOCK(ErrorMessageLock);
01647             MesPrint("information in replace transformation does not evaluate into a
single term");
01648             MUNLOCK(ErrorMessageLock);
01649             return(-1);
01650         }
01651         /*
01652         Now we can copy the new function argument to the output u
01653         */
01654         i = term4[2]-FUNHEAD;
01655         term3 = term4+FUNHEAD+1;
01656         NCOPY(u,term3,i)
01657         if ( functions[fun[0]-FUNCTION].spec != TENSORFUNCTION ) {
01658             NEXTARG(t)
01659         }
01660         else t++;
01661         AT.WorkPointer = term2;
01662
01663         goto nextt;
01664     }
01665     info1 += 2; NEXTARG(info1)
01666 }
01667 else if ( ( *info1 == -INDEX )
01668 && ( info[1] == WILDARGINDEX + AM.OffsetIndex ) ) {
01669     wild[0] = INDTO SUB;
01670     wild[2] = WILDARGINDEX+AM.OffsetIndex;
01671     wild[3] = 0;
01672     AN.WildValue = wild;
01673     AT.WildMask = &mask;
01674     mask = 0;
01675     AN.NumWild = 1;
01676     if ( ( functions[fun[0]-FUNCTION].spec == TENSORFUNCTION )
01677 || ( *t == -INDEX || ( *t > 0 && CheckWild(BHEAD WILDARGINDEX,INDTO SUB,1,t) == 0 ) ) ) {
01678     /*
01679         We put the part in replace in a function and make
01680         a replace_(xarg_,(t argument)).
01681         */
01682         n1 = INDEX; n2 = WILDARGINDEX+AM.OffsetIndex;
01683         info1 += 2;
01684         goto getthisone;
01685     }
01686     info1 += 2; NEXTARG(info1)
01687 }
01688 else if ( ( *info1 == -VECTOR )
01689 && ( info[1] == WILDARGVECTOR + AM.OffsetVector ) ) {
01690     wild[0] = VECTO SUB;
01691     wild[2] = WILDARGVECTOR+AM.OffsetVector;
01692     wild[3] = 0;
01693     AN.WildValue = wild;
01694     AT.WildMask = &mask;
01695     mask = 0;
01696     AN.NumWild = 1;
01697     if ( functions[fun[0]-FUNCTION].spec == TENSORFUNCTION ) {
01698         if ( *t < MINSPEC ) {
01699             n1 = VECTOR; n2 = WILDARGVECTOR+AM.OffsetVector;
01700             info1 += 2;
01701             goto getthisone;
01702         }
01703     }
01704     else if ( *t == -VECTOR || *t == -MINVECTOR ||
01705 ( *t > 0 && CheckWild(BHEAD WILDARGVECTOR,VECTO SUB,1,t) == 0 ) ) {
01706     /*
01707         We put the part in replace in a function and make
01708         a replace_(xarg_,(t argument)).
01709         */
01710         n1 = VECTOR; n2 = WILDARGVECTOR+AM.OffsetVector;
01711         info1 += 2;
01712         goto getthisone;

```



```

01713         }
01714         info1 += 2; NEXTARG(info1)
01715     }
01716     else if ( *info1 == -WILDARGFUN ) {
01717         wild[0] = FUNTOFUN;
01718         wild[2] = WILDARGFUN;
01719         wild[3] = 0;
01720         AN.WildValue = wild;
01721         AT.WildMask = &mask;
01722         mask = 0;
01723         AN.NumWild = 1;
01724         if ( *t <= -FUNCTION || ( *t > 0 && CheckWild(BHEAD WILDARGFUN,FUNTOFUN,1,t) == 0
    ) ) {
01725     /*
01726         We put the part in replace in a function and make
01727         a replace_(xarg_,(t argument)).
01728     */
01729         n2 = n1 = -WILDARGFUN; /* n2 is to keep the compiler quiet */
01730         info1++;
01731         goto getthisone;
01732     }
01733     info1++; NEXTARG(info1)
01734 }
01735     else {
01736         NEXTARG(info1) NEXTARG(info1)
01737     }
01738 }
01739 }
01740 if ( ngeneral > 0 ) {
01741     /*
01742         They are all in a replace_ function.
01743         Compose the whole thing into a term with replace_()*dum_(arg)
01744         which will be given to Generator.
01745         If we have f(a(x))*replace_(x,b) this gives f(a(b))
01746     */
01747     dothisnow;
01748     term3 = term2; term4 = term1; i = *term1;
01749     NCOPY(term3,term4,i)
01750     term4 = term3;
01751     if ( functions[fun[0]-FUNCTION].spec != TENSORFUNCTION ) {
01752         *term3++ = DUMFUN; term3++; FILLFUN(term3);
01753         tt = t;
01754         COPY1ARG(term3,tt)
01755     }
01756     else {
01757         *term3++ = fun[0]; term3++; FILLFUN(term3); *term3++ = *t;
01758     }
01759     term4[1] = term3-term4;
01760     *term3++ = 1; *term3++ = 1; *term3++ = 3;
01761     *term2 = term3-term2;
01762     AT.WorkPointer = term3;
01763     NewSort(BHEAD0);
01764     if ( Generator(BHEAD term2,AR.Cnumlhs) ) {
01765         LowerSortLevel();
01766         AT.WorkPointer = oldwork;
01767         AT.WildMask = oldmask;
01768         return(-1);
01769     }
01770     term4 = AT.WorkPointer;
01771     if ( EndSort(BHEAD term4,1) < 0 ) {}
01772     if ( ( *term4 && *(term4+*term4) != 0 ) || *term4 == 0 ) {
01773         MLOCK(ErrorMessageLock);
01774         MesPrint("&information in replace transformation does not evaluate into a single
term");
01775         MUNLOCK(ErrorMessageLock);
01776         return(-1);
01777     }
01778     /*
01779         Now we can copy the new function argument to the output u
01780     */
01781     i = term4[2]-FUNHEAD;
01782     term3 = term4+FUNHEAD+1;
01783     NCOPY(u,term3,i)
01784     NEXTARG(t)
01785     AT.WorkPointer = term2;
01786
01787     goto nextt;
01788 }
01789
01790     /*
01791         No catch. Copy the argument and continue.
01792     */
01793     if ( functions[fun[0]-FUNCTION].spec != TENSORFUNCTION ) {
01794         if ( *t <= -FUNCTION ) { *u++ = *t++; }
01795         else if ( *t < 0 ) { *u++ = *t++; *u++ = *t++; }
01796         else { i = *t; NCOPY(u,t,i) }
01797     }

```

```

01798         else {
01799             *u++ = *t++;
01800         }
01801 nexttt::
01802     }
01803     i = u - tstop; tstop[1] = i; tstop[2] = dirty;
01804     t = fun; u = tstop; NCOPY(t,u,i)
01805     AT.WorkPointer = oldwork;
01806     AT.WildMask = oldmask;
01807     return(0);
01808 }
01809
01810 /*
01811     #] RunReplace :
01812     #[ RunImplode :
01813
01814     Note that we restrict ourselves to short integers and/or single symbols
01815 */
01816
01817 WORD RunImplode(WORD *fun, WORD *args)
01818 {
01819     GETIDENTITY
01820     WORD *tt, *tstop, totarg, arg1, arg2, num1, num2, il, n;
01821     WORD *f, *t, *ttt, *t4, *ff, *fff;
01822     WORD moveup, numzero, outspace;
01823     if ( functions[fun[0]-FUNCTION].spec != 0 ) return(0);
01824     if ( *args != ARGRANGE ) {
01825         MLOCK(ErrorMessageLock);
01826         MesPrint("Illegal range encountered in RunImplode");
01827         MUNLOCK(ErrorMessageLock);
01828         Terminate(-1);
01829     }
01830     tt = fun+FUNHEAD; tstop = fun+fun[1]; totarg = 0;
01831     while ( tt < tstop ) { totarg++; NEXTARG(tt); }
01832     if ( FindRange(BHEAD args,&arg1,&arg2,totarg) ) return(-1);
01833 /*
01834     Get the proper range in forward direction and the number of arguments
01835 */
01836     if ( arg1 > arg2 ) { num1 = arg2; num2 = arg1; }
01837     else { num1 = arg1; num2 = arg2; }
01838     if ( num1 > totarg || num2 > totarg ) return(0);
01839 /*
01840     We need, for the most general case 4 spots for each:
01841         x,pow,coef,sign
01842     Hence we put these in the workspace above the term after tstop
01843 */
01844     n = 1; f = fun+FUNHEAD;
01845     while ( n < num1 ) {
01846         if ( f >= tstop ) return(0);
01847         NEXTARG(f);
01848         n++;
01849     }
01850     ff = f;
01851 /*
01852     We are now at the first argument to be done
01853     Go through the terms and test their validity.
01854     If one of them doesn't conform to the rules we don't do anything.
01855     The terms to be done are put in special notation after the function.
01856     Notation: numsymbol, power, |coef|, sign
01857     If numsymbol is negative there is no symbol.
01858     We do it this way because otherwise stepping backwards (as in range=(4,1))
01859     would be very difficult.
01860 */
01861     tt = tstop;
01862     while ( n <= num2 ) {
01863         if ( f >= tstop ) return(0);
01864         if ( *f == -SNUMBER ) { *tt++ = -1; *tt++ = 0;
01865             if ( f[1] < 0 ) { *tt++ = -f[1]; *tt++ = -1; }
01866             else { *tt++ = f[1]; *tt++ = 1; }
01867             f += 2;
01868         }
01869         else if ( *f == -SYMBOL ) { *tt++ = f[1]; *tt++ = 1; *tt++ = 1; *tt++ = 1; f += 2; }
01870         else if ( *f < 0 ) return(0);
01871         else {
01872             if ( *f != ( f[ARGHEAD]+ARGHEAD ) ) return(0); /* Not a single term */
01873             t = f + *f - 1;
01874             il = ABS(*t);
01875             if ( ( il > 3 ) || ( t[-1] != 1 ) ) return(0); /* Not an integer or too big */
01876             if ( ( UWORD)(t[-2]) > MAXPOSITIVE4 ) return(0); /* number too big */
01877             if ( f[ARGHEAD] == il+1 ) { /* numerical which is fine */
01878                 *tt++ = -1; *tt++ = 0; *tt++ = t[-2];
01879                 if ( *t < 0 ) { *tt++ = -1; }
01880                 else { *tt++ = 1; }
01881             }
01882             else if ( ( f[ARGHEAD+1] != SYMBOL )
01883                 || ( f[ARGHEAD+2] != 4 )
01884                 || ( ( f[ARGHEAD+1]+f[ARGHEAD+2] ) < ( t-il ) ) ) return(0);

```

```

01885             /* not a single symbol with a coefficient */
01886         else {
01887             *tt++ = f[ARGHEAD+3];
01888             *tt++ = f[ARGHEAD+4];
01889             *tt++ = t[-2];
01890             if ( *t < 0 ) { *tt++ = -1; }
01891             else { *tt++ = 1; }
01892         }
01893         f += *f;
01894     }
01895     n++;
01896 }
01897 fff = f;
01898 /*
01899 At this point we can do the implosion.
01900 Requirement: no coefficient shall take more than one word.
01901 (a stricter requirement may be needed to keep the explosion contained)
01902 */
01903 if ( arg1 > arg2 ) {
01904 /*
01905     Work backward.
01906 */
01907     t = tt - 4; numzero = 0;
01908     while ( t >= tstop ) {
01909         if ( t[2] == 0 ) numzero++;
01910         else {
01911             if ( numzero > 0 ) {
01912                 t[2] += numzero;
01913                 t4 = t+4;
01914                 ttt = t4 + 4*numzero;
01915                 while ( ttt < tt ) *t4++ = *ttt++;
01916                 tt -= 4*numzero;
01917                 numzero = 0;
01918             }
01919         }
01920         t -= 4;
01921     }
01922 }
01923 else {
01924     t = tstop;
01925     numzero = 0; ttt = t;
01926     while ( t < tt ) {
01927         if ( t[2] == 0 ) numzero++;
01928         else {
01929             if ( numzero > 0 ) {
01930                 t[2] += numzero;
01931                 t4 = t;
01932                 while ( t4 < tt ) *ttt++ = *t4++;
01933                 tt -= 4*numzero;
01934                 t -= 4*numzero;
01935                 ttt = t + 4;
01936                 numzero = 0;
01937             }
01938             else {
01939                 ttt = t + 4;
01940             }
01941         }
01942         t += 4;
01943     }
01944 /*
01945     We may have numzero > 0 at the end. We leave them.
01946     Output space is currently from tstop to tt
01947 */
01948 }
01949 /*
01950 Now we compute the real output space needed
01951 */
01952 t = tstop; outspace = 0;
01953 while ( t < tt ) {
01954     if ( t[0] == -1 ) {
01955         if ( t[2] > MAXPOSITIVE4 ) { return(0); /* Number too big */ }
01956         outspace += 2;
01957     }
01958     else if ( t[1] == 1 && t[2] == 1 && t[3] == 1 ) { outspace += 2; }
01959     else { outspace += 8 + ARGHEAD; }
01960     t += 4;
01961 }
01962 if ( outspace < (fff-ff) ) {
01963     t = tstop;
01964     while ( t < tt ) {
01965         if ( t[0] == -1 ) { *ff++ = -SNUMBER; *ff++ = t[2]*t[3]; }
01966         else if ( t[1] == 1 && t[2] == 1 && t[3] == 1 ) {
01967             *ff++ = -SYMBOL; *ff++ = t[0];
01968         }
01969         else {
01970             *ff++ = 8+ARGHEAD; *ff++ = 0; FILLARG(ff);
01971             *ff++ = 8; *ff++ = SYMBOL; *ff++ = 4; *ff++ = t[0]; *ff++ = t[1];

```

```

01972         *ff++ = t[2]; *ff++ = 1; *ff++ = t[3] > 0 ? 3: -3;
01973     }
01974     t += 4;
01975 }
01976 while ( fff < tstop ) *ff++ = *fff++;
01977 fun[1] = ff - fun;
01978 }
01979 else if ( outspace > (fff-ff) ) {
01980 /*
01981     Move the answer up by the required amount.
01982     Move the tail to its new location
01983     Move in things as for outspace == (fff-ff)
01984 */
01985     moveup = outspace-(fff-ff);
01986     ttt = tt + moveup;
01987     t = tt;
01988     while ( t > fff ) *--ttt = *--t;
01989     tt += moveup; tstop += moveup;
01990     fff += moveup;
01991     fun[1] += moveup;
01992     goto moveinto;
01993 }
01994 else {
01995 moveinto:
01996     t = tstop;
01997     while ( t < tt ) {
01998         if ( t[0] == -1 ) { *ff++ = -SNUMBER; *ff++ = t[2]*t[3]; }
01999         else if ( t[1] == 1 && t[2] == 1 && t[3] == 1 ) {
02000             *ff++ = -SYMBOL; *ff++ = t[0];
02001         }
02002         else {
02003             *ff++ = 8+ARGHEAD; *ff++ = 0; FILLARG(ff);
02004             *ff++ = 8; *ff++ = SYMBOL; *ff++ = 4; *ff++ = t[0]; *ff++ = t[1];
02005             *ff++ = t[2]; *ff++ = 1; *ff++ = t[3] > 0 ? 3: -3;
02006         }
02007         t += 4;
02008     }
02009 }
02010 return(0);
02011 }
02012
02013 /*
02014     #] RunImplode :
02015     #[ RunExplode :
02016 */
02017
02018 WORD RunExplode(PHEAD WORD *fun, WORD *args)
02019 {
02020     WORD arg1, arg2, num1, num2, *tt, *tstop, totarg, *tonew, *newfun;
02021     WORD *ff, *f;
02022     int reverse = 0, iarg, i, numzero;
02023     if ( functions[fun[0]-FUNCTION].spec != 0 ) return(0);
02024     if ( *args != ARGGRANGE ) {
02025         MLOCK(ErrorMessageLock);
02026         MesPrint("Illegal range encountered in RunExplode");
02027         MUNLOCK(ErrorMessageLock);
02028         Terminate(-1);
02029     }
02030     tt = fun+FUNHEAD; tstop = fun+fun[1]; totarg = 0;
02031     while ( tt < tstop ) { totarg++; NEXTARG(tt); }
02032     if ( FindRange(BHEAD args,&arg1,&arg2,totarg) ) return(-1);
02033 /*
02034     Get the proper range in forward direction and the number of arguments
02035 */
02036     if ( arg1 > arg2 ) { num1 = arg2; num2 = arg1; reverse = 1; }
02037     else { num1 = arg1; num2 = arg2; }
02038     if ( num1 > totarg || num2 > totarg ) return(0);
02039     if ( tstop + AM.MaxTer > AT.WorkTop ) goto OverWork;
02040 /*
02041     We will make the new function after the old one in the workspace
02042     Find the first argument
02043 */
02044     tonew = newfun = tstop;
02045     ff = fun + FUNHEAD; iarg = 0;
02046     while ( ff < tstop ) {
02047         iarg++;
02048         if ( iarg == num1 ) {
02049             i = ff - fun; f = fun;
02050             NCOPY(tonew,f,i)
02051             break;
02052         }
02053         NEXTARG(ff)
02054     }
02055 /*
02056     We have reached the first argument to be done
02057 */
02058     while ( iarg <= num2 ) {

```

```

02059     if ( *ff == -SYMBOL || ( *ff == -SNUMBER && ff[1] == 0 ) )
02060     { *tonew++ = *ff++; *tonew++ = *ff++; }
02061     else if ( *ff == -SNUMBER ) {
02062         numzero = ABS(ff[1])-1;
02063         if ( reverse ) {
02064             *tonew++ = -SNUMBER; *tonew++ = ff[1] < 0 ? -1: 1;
02065             while ( numzero > 0 ) {
02066                 *tonew++ = -SNUMBER; *tonew++ = 0; numzero--;
02067             }
02068         }
02069         else {
02070             while ( numzero > 0 ) {
02071                 *tonew++ = -SNUMBER; *tonew++ = 0; numzero--;
02072             }
02073             *tonew++ = -SNUMBER; *tonew++ = ff[1] < 0 ? -1: 1;
02074         }
02075         ff += 2;
02076     }
02077     else if ( *ff < 0 ) { return(0); }
02078     else {
02079         if ( *ff != ARGHEAD+8 || ff[ARGHEAD] != 8
02080             || ff[ARGHEAD+1] != SYMBOL || ABS(ff[ARGHEAD+7]) != 3
02081             || ff[ARGHEAD+6] != 1 ) return(0);
02082         numzero = ff[ARGHEAD+5];
02083         if ( numzero >= MAXPOSITIVE4 ) return(0);
02084         numzero--;
02085         if ( reverse ) {
02086             if ( ff[ARGHEAD+7] > 0 ) { *tonew++ = -SNUMBER; *tonew++ = 1; }
02087             else {
02088                 *tonew++ = ARGHEAD+8; *tonew++ = 0; FILLARG(tonew)
02089                 *tonew++ = 8; *tonew++ = SYMBOL; *tonew++ = ff[ARGHEAD+3];
02090                 *tonew++ = ff[ARGHEAD+4]; *tonew++ = 1; *tonew++ = 1;
02091                 *tonew++ = -3;
02092             }
02093             while ( numzero > 0 ) {
02094                 *tonew++ = -SNUMBER; *tonew++ = 0; numzero--;
02095             }
02096         }
02097         else {
02098             while ( numzero > 0 ) {
02099                 *tonew++ = -SNUMBER; *tonew++ = 0; numzero--;
02100             }
02101             *tonew++ = ARGHEAD+8; *tonew++ = 0; FILLARG(tonew)
02102             *tonew++ = 8; *tonew++ = SYMBOL; *tonew++ = 4;
02103             *tonew++ = ff[ARGHEAD+3]; *tonew++ = ff[ARGHEAD+4];
02104             *tonew++ = 1; *tonew++ = 1;
02105             if ( ff[ARGHEAD+7] > 0 ) *tonew++ = 3;
02106             else *tonew++ = -3;
02107         }
02108         ff += *ff;
02109     }
02110     if ( tonew > AT.WorkTop ) goto OverWork;
02111     iarg++;
02112 }
02113 /*
02114 Copy the tail, settle the size and copy the whole thing back.
02115 */
02116 while ( ff < tstop ) *tonew++ = *ff++;
02117 i = newfun[1] = tonew-newfun;
02118 NCOPY(fun,newfun,i)
02119 return(0);
02120 OverWork::
02121 MLOCK(ErrorMessageLock);
02122 MesWork();
02123 MUNLOCK(ErrorMessageLock);
02124 return(-1);
02125 }
02126
02127 /*
02128 #] RunExplode :
02129 #[ RunPermute :
02130 */
02131
02132 WORD RunPermute(PHEAD WORD *fun, WORD *args, WORD *info)
02133 {
02134     WORD *tt, totarg, *tstop, arg1, arg2, n, num, i, *f, *f1, *f2, *infostop;
02135     WORD *in, *iw, withdollar;
02136     DOLLARS d;
02137     if ( *args != ARGGRANGE ) {
02138         MLOCK(ErrorMessageLock);
02139         MesPrint("Illegal range encountered in RunPermute");
02140         MUNLOCK(ErrorMessageLock);
02141         Terminate(-1);
02142     }
02143     if ( functions[fun[0]-FUNCTION].spec != TENSORFUNCTION ) {
02144         tt = fun+FUNHEAD; tstop = fun+fun[1]; totarg = 0;
02145         while ( tt < tstop ) { totarg++; NEXTARG(tt); }

```

```

02146     arg1 = 1; arg2 = totarg;
02147  /*
02148     We need to:
02149     1: get pointers to the arguments
02150     2: permute the pointers
02151     3: copy the arguments to safe territory in the new order
02152     4: copy this new order back in situ.
02153  */
02154     num = arg2-arg1+1;
02155     WantAddPointers(num); /* Guarantees the presence of enough pointers */
02156     f = fun+FUNHEAD; n = 1; i = 0;
02157     while ( n < arg1 ) { n++; NEXTARG(f) }
02158     f1 = f;
02159     while ( n <= arg2 ) { AT.pWorkSpace[AT.pWorkPointer+i++] = f; n++; NEXTARG(f) }
02160  /*
02161     Now the permutations
02162  */
02163     info++;
02164     while ( *info ) {
02165         infostop = info + *info;
02166         info++;
02167         if ( *info > totarg ) return(0);
02168  /*
02169         Now we have a look whether there are dollar variables to be expanded
02170         We also shift out all values that are out of range.
02171  */
02172         withddollar = 0; in = info;
02173         while ( in < infostop ) {
02174             if ( *in < 0 ) { /* Dollar variable -(number+1) */
02175                 d = Dollars - *in - 1;
02176 #ifdef WITHPTHREADS
02177                 {
02178                     int nummodopt, dtype = -1, numdollar = -*in-1;
02179                     if ( AS.MultiThreaded && ( AC.mparallelflag == PARALLELFLAG ) ) {
02180                         for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
02181                             if ( numdollar == ModOptdollars[nummodopt].number ) break;
02182                         }
02183                         if ( nummodopt < NumModOptdollars ) {
02184                             dtype = ModOptdollars[nummodopt].type;
02185                             if ( dtype == MODLOCAL ) {
02186                                 d = ModOptdollars[nummodopt].dstruct+AT.identity;
02187                             }
02188                             else {
02189                                 LOCK(d->pthreadlockread);
02190                             }
02191                         }
02192                     }
02193                 }
02194 #endif
02195                 if ( ( d->type == DOLNUMBER || d->type == DOLTERMS )
02196                     && d->where[0] == 4 && d->where[4] == 0 ) {
02197                     if ( d->where[3] < 0 || d->where[2] != 1 || d->where[1] > totarg ) return(0);
02198                 }
02199                 else if ( d->type == DOLWILDARGS ) {
02200                     iw = d->where+1;
02201                     while ( *iw ) {
02202                         if ( *iw == -SNUMBER ) {
02203                             if ( iw[1] <= 0 || iw[1] > totarg ) return(0);
02204                         }
02205                         else goto IllType;
02206                         iw += 2;
02207                     }
02208                 }
02209                 else {
02210 IllType:
02211                     MLOCK(ErrorMessageLock);
02212                     MesPrint("Illegal type of $-variable in RunPermute");
02213                     MUNLOCK(ErrorMessageLock);
02214                     Terminate(-1);
02215                 }
02216                 withddollar++;
02217             }
02218             else if ( *in > totarg ) return(0);
02219             in++;
02220         }
02221         if ( withddollar ) { /* We need some space for a copy */
02222             WORD *incopy, *tocopy;
02223             incopy = TermMalloc("RunPermute");
02224             tocopy = incopy+1; in = info;
02225             while ( in < infostop ) {
02226                 if ( *in < 0 ) {
02227                     d = Dollars - *in - 1;
02228 #ifdef WITHPTHREADS
02229                     {
02230                         int nummodopt, dtype = -1, numdollar = -*in-1;
02231                         if ( AS.MultiThreaded && ( AC.mparallelflag == PARALLELFLAG ) ) {
02232                             for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {

```

```

02233         if ( numdollar == ModOptdollars[nummodopt].number ) break;
02234     }
02235     if ( nummodopt < NumModOptdollars ) {
02236         dtype = ModOptdollars[nummodopt].type;
02237         if ( dtype == MODLOCAL ) {
02238             d = ModOptdollars[nummodopt].dstruct+AT.identity;
02239         }
02240         else {
02241             LOCK(d->pthreadlockread);
02242         }
02243     }
02244 }
02245 }
02246 #endif
02247     if ( d->type == DOLNUMBER || d->type == DOLTERMS ) {
02248         *tocopy++ = d->where[1] - 1;
02249     }
02250     else if ( d->type == DOLWILDARGS ) {
02251         iw = d->where+1;
02252         while ( *iw ) {
02253             *tocopy++ = iw[1] - 1;
02254             iw += 2;
02255         }
02256     }
02257     in++;
02258 }
02259     else *tocopy++ = *in++;
02260 }
02261 *tocopy = 0;
02262 *incopy = tocopy - incopy;
02263 in = incopy+1;
02264 tt = AT.pWorkSpace[AT.pWorkPointer+*in];
02265 in++;
02266 while ( in < tocopy ) {
02267     if ( *in > totarg ) return(0);
02268     AT.pWorkSpace[AT.pWorkPointer+in[-1]] = AT.pWorkSpace[AT.pWorkPointer+*in];
02269     in++;
02270 }
02271 AT.pWorkSpace[AT.pWorkPointer+in[-1]] = tt;
02272 TermFree(incopy,"RunPermute");
02273 info = infostop;
02274 }
02275 else {
02276     tt = AT.pWorkSpace[AT.pWorkPointer+*info];
02277     info++;
02278     while ( info < infostop ) {
02279         if ( *info > totarg ) return(0);
02280         AT.pWorkSpace[AT.pWorkPointer+info[-1]] = AT.pWorkSpace[AT.pWorkPointer+*info];
02281         info++;
02282     }
02283     AT.pWorkSpace[AT.pWorkPointer+info[-1]] = tt;
02284 }
02285 }
02286 /*
02287 info++;
02288 while ( *info ) {
02289     infostop = info + *info;
02290     info++;
02291     if ( *info > totarg ) return(0);
02292     tt = AT.pWorkSpace[AT.pWorkPointer+*info];
02293     info++;
02294     while ( info < infostop ) {
02295         if ( *info > totarg ) return(0);
02296         AT.pWorkSpace[AT.pWorkPointer+info[-1]] = AT.pWorkSpace[AT.pWorkPointer+*info];
02297         info++;
02298     }
02299     AT.pWorkSpace[AT.pWorkPointer+info[-1]] = tt;
02300 }
02301 */
02302 /*
02303 And the final cleanup
02304 */
02305 if ( tstop+(f-f1) > AT.WorkTop ) goto OverWork;
02306 f2 = tstop;
02307 for ( i = 0; i < num; i++ ) { f = AT.pWorkSpace[AT.pWorkPointer+i]; COPY1ARG(f2,f) }
02308 i = f2 - tstop;
02309 NCOPY(f1,tstop,i)
02310 }
02311 else { /* tensors */
02312     tt = fun+FUNHEAD; tstop = fun+fun[1]; totarg = tstop-tt;
02313     arg1 = 1; arg2 = totarg;
02314     num = arg2-arg1+1;
02315     WantAddPointers(num); /* Guarantees the presence of enough pointers */
02316     f = fun+FUNHEAD; n = 1; i = 0;
02317     while ( n < arg1 ) { n++; f++; }
02318     f1 = f;
02319     while ( n <= arg2 ) { AT.pWorkSpace[AT.pWorkPointer+i++] = f; n++; f++; }

```

```

02320 /*
02321     Now the permutations
02322 */
02323     info++;
02324     while ( *info ) {
02325         infostop = info + *info;
02326         info++;
02327         if ( *info > totarg ) return(0);
02328         tt = AT.pWorkSpace[AT.pWorkPointer+*info];
02329         info++;
02330         while ( info < infostop ) {
02331             if ( *info > totarg ) return(0);
02332             AT.pWorkSpace[AT.pWorkPointer+info[-1]] = AT.pWorkSpace[AT.pWorkPointer+*info];
02333             info++;
02334         }
02335         AT.pWorkSpace[AT.pWorkPointer+info[-1]] = tt;
02336     }
02337 /*
02338     And the final cleanup
02339 */
02340     if ( tstop+(f-f1) > AT.WorkTop ) goto OverWork;
02341     f2 = tstop;
02342     for ( i = 0; i < num; i++ ) { f = AT.pWorkSpace[AT.pWorkPointer+i]; *f2++ = *f++; }
02343     i = f2 - tstop;
02344     NCOPY(f1,tstop,i)
02345 }
02346 return(0);
02347 OverWork:;
02348 MLOCK(ErrorMessageLock);
02349 MesWork();
02350 MUNLOCK(ErrorMessageLock);
02351 return(-1);
02352 }
02353
02354 /*
02355     #] RunPermute :
02356     #[ RunReverse :
02357 */
02358
02359 WORD RunReverse(PHEAD WORD *fun, WORD *args)
02360 {
02361     WORD *tt, totarg, *tstop, arg1, arg2, n, num, i, *f, *f1, *f2, i1, i2;
02362     if ( *args != ARGRANGE ) {
02363         MLOCK(ErrorMessageLock);
02364         MesPrint("Illegal range encountered in RunReverse");
02365         MUNLOCK(ErrorMessageLock);
02366         Terminate(-1);
02367     }
02368     if ( functions[fun[0]-FUNCTION].spec != TENSORFUNCTION ) {
02369         tt = fun+FUNHEAD; tstop = fun+fun[1]; totarg = 0;
02370         while ( tt < tstop ) { totarg++; NEXTARG(tt); }
02371         if ( FindRange(BHEAD args,&arg1,&arg2,totarg) ) return(-1);
02372 /*
02373     We need to:
02374     1: get pointers to the arguments
02375     2: reverse the order of the pointers
02376     3: copy the arguments to safe territory in the new order
02377     4: copy this new order back in situ.
02378 */
02379         if ( arg2 < arg1 ) { n = arg1; arg1 = arg2; arg2 = n; }
02380         if ( arg2 > totarg ) return(0);
02381
02382         num = arg2-arg1+1;
02383         WantAddPointers(num); /* Guarantees the presence of enough pointers */
02384         f = fun+FUNHEAD; n = 1; i = 0;
02385         while ( n < arg1 ) { n++; NEXTARG(f) }
02386         f1 = f;
02387         while ( n <= arg2 ) { AT.pWorkSpace[AT.pWorkPointer+i++] = f; n++; NEXTARG(f) }
02388         i1 = i-1; i2 = 0;
02389         while ( i1 > i2 ) {
02390             tt = AT.pWorkSpace[AT.pWorkPointer+i1];
02391             AT.pWorkSpace[AT.pWorkPointer+i1] = AT.pWorkSpace[AT.pWorkPointer+i2];
02392             AT.pWorkSpace[AT.pWorkPointer+i2] = tt;
02393             i1--; i2++;
02394         }
02395         if ( tstop+(f-f1) > AT.WorkTop ) goto OverWork;
02396         f2 = tstop;
02397         for ( i = 0; i < num; i++ ) { f = AT.pWorkSpace[AT.pWorkPointer+i]; COPY1ARG(f2,f) }
02398         i = f2 - tstop;
02399         NCOPY(f1,tstop,i)
02400     }
02401     else { /* Tensors */
02402         tt = fun+FUNHEAD; tstop = fun+fun[1]; totarg = tstop - tt;
02403         if ( FindRange(BHEAD args,&arg1,&arg2,totarg) ) return(-1);
02404 /*
02405     We need to:
02406     1: get pointers to the arguments

```



```

02407         2: reverse the order of the pointers
02408         3: copy the arguments to safe territory in the new order
02409         4: copy this new order back in situ.
02410 */
02411     if ( arg2 < arg1 ) { n = arg1; arg1 = arg2; arg2 = n; }
02412     if ( arg2 > totarg ) return(0);
02413
02414     num = arg2-arg1+1;
02415     WantAddPointers(num); /* Guarantees the presence of enough pointers */
02416     f = fun+FUNHEAD; n = 1; i = 0;
02417     while ( n < arg1 ) { n++; f++; }
02418     f1 = f;
02419     while ( n <= arg2 ) { AT.pWorkSpace[AT.pWorkPointer+i++] = f; n++; f++; }
02420     i1 = i-1; i2 = 0;
02421     while ( i1 > i2 ) {
02422         tt = AT.pWorkSpace[AT.pWorkPointer+i1];
02423         AT.pWorkSpace[AT.pWorkPointer+i1] = AT.pWorkSpace[AT.pWorkPointer+i2];
02424         AT.pWorkSpace[AT.pWorkPointer+i2] = tt;
02425         i1--; i2++;
02426     }
02427     if ( tstop+(f-f1) > AT.WorkTop ) goto OverWork;
02428     f2 = tstop;
02429     for ( i = 0; i < num; i++ ) { f = AT.pWorkSpace[AT.pWorkPointer+i]; *f2++ = *f++; }
02430     i = f2 - tstop;
02431     NCOPY(f1,tstop,i)
02432 }
02433 return(0);
02434 OverWork:;
02435 MLOCK(ErrorMessageLock);
02436 MesWork();
02437 MUNLOCK(ErrorMessageLock);
02438 return(-1);
02439 }
02440
02441 /*
02442     #] RunReverse :
02443     #[ RunDedup :
02444 */
02445
02446 WORD RunDedup(PHEAD WORD *fun, WORD *args)
02447 {
02448     WORD *tt, totarg, *tstop, arg1, arg2, n, i, j,k, *f, *f1, *f2, *fd, *fstart;
02449     if ( *args != ARGRANGE ) {
02450         MLOCK(ErrorMessageLock);
02451         MesPrint("Illegal range encountered in RunDedup");
02452         MUNLOCK(ErrorMessageLock);
02453         Terminate(-1);
02454     }
02455     if ( functions[fun[0]-FUNCTION].spec != TENSORFUNCTION ) {
02456         tt = fun+FUNHEAD; tstop = fun+fun[1]; totarg = 0;
02457         while ( tt < tstop ) { totarg++; NEXTARG(tt); }
02458         if ( FindRange(BHEAD args,&arg1,&arg2,totarg) ) return(-1);
02459
02460         if ( arg2 < arg1 ) { n = arg1; arg1 = arg2; arg2 = n; }
02461         if ( arg2 > totarg ) return(0);
02462
02463         f = fun+FUNHEAD; n = 1;
02464         while ( n < arg1 ) { n++; NEXTARG(f) }
02465         f1 = f; // fast forward to first element in range
02466         i = 0; // new argument count
02467         fstart = f1;
02468
02469         for (; n <= arg2; n++ ) {
02470             f2 = fstart;
02471             for ( j = 0; j < i; j++ ) { // check all previous terms
02472                 fd = f2;
02473                 NEXTARG(fd)
02474                 for ( k = 0; k < fd-f2; k++ ) // byte comparison of args
02475                     if ( f2[k] != f[k] ) break;
02476
02477                 if ( k == fd-f2 ) break; // duplicate arg
02478                 f2 = fd;
02479             }
02480
02481             if ( j == i ) {
02482                 // unique factor, copy in situ
02483                 COPY1ARG(f1,f)
02484                 i++;
02485             } else {
02486                 NEXTARG(f)
02487             }
02488         }
02489
02490         // move the terms from after the range
02491         for ( j = n; j <= totarg; j++ ) {
02492             COPY1ARG(f1,f)
02493         }

```

```

02494     fun[1] = f1 - fun; // resize function
02495 }
02496 else { /* Tensors */
02497     tt = fun+FUNHEAD; tstop = fun+fun[1]; totarg = tstop - tt;
02498     if ( FindRange(BHEAD args,&arg1,&arg2,totarg) ) return(-1);
02500     if ( arg2 < arg1 ) { n = arg1; arg1 = arg2; arg2 = n; }
02501     if ( arg2 > totarg ) return(0);
02503     f = fun+FUNHEAD;
02504     i = arg1; // new argument count
02505     n = i;
02507     for (; n <= arg2; n++) {
02508         for ( j = arg1; j < i; j++ ) { // check all previous terms
02509             if ( f[n-1] == f[j-1] ) break; // duplicate arg
02510         }
02511         if ( j == i ) {
02512             // unique factor, copy in situ
02513             f[i-1] = f[n-1];
02514             i++;
02515         }
02516     }
02517     // move the terms from after the range
02518     for (j = n; j <= totarg; j++, i++) {
02519         f[i-1] = f[j-1];
02520     }
02521     fun[1] = f + i - 1 - fun; // resize function
02522 }
02523 return(0);
02524 }
02525 /*
02526     #] RunDedup :
02527     #[ RunCycle :
02528 */
02529 WORD RunCycle(PHEAD WORD *fun, WORD *args, WORD *info)
02530 {
02531     WORD *tt, totarg, *tstop, arg1, arg2, n, num, i, j, *f, *f1, *f2, x, ncyc, cc;
02532     if ( *args != ARGGRANGE ) {
02533         MLOCK(ErrorMessageLock);
02534         MesPrint("Illegal range encountered in RunCycle");
02535         MUNLOCK(ErrorMessageLock);
02536         Terminate(-1);
02537     }
02538     ncyc = info[1];
02539     if ( ncyc >= MAXPOSITIVE2 ) { /* $ variable */
02540         ncyc -= MAXPOSITIVE2;
02541         if ( ncyc >= MAXPOSITIVE4 ) {
02542             ncyc -= MAXPOSITIVE4; /* -$ */
02543             cc = -1;
02544         }
02545         else cc = 1;
02546         ncyc = DolToNumber(BHEAD ncyc);
02547         if ( AN.ErrorInDollar ) {
02548             MesPrint(" Error in Dollar variable in transform,cycle()=$");
02549             return(-1);
02550         }
02551         if ( ncyc >= MAXPOSITIVE4 || ncyc <= -MAXPOSITIVE4 ) {
02552             MesPrint(" Illegal value from Dollar variable in transform,cycle()=$");
02553             return(-1);
02554         }
02555         ncyc *= cc;
02556     }
02557     if ( functions[fun[0]-FUNCTION].spec != TENSORFUNCTION ) {
02558         tt = fun+FUNHEAD; tstop = fun+fun[1]; totarg = 0;
02559         while ( tt < tstop ) { totarg++; NEXTARG(tt); }
02560         if ( FindRange(BHEAD args,&arg1,&arg2,totarg) ) return(-1);
02561         if ( arg1 > arg2 ) { n = arg1; arg1 = arg2; arg2 = n; }
02562         if ( arg2 > totarg ) return(0);
02563     }
02564     /*
02565     We need to:
02566     1: get pointers to the arguments
02567     2: cycle the pointers
02568     3: copy the arguments to safe territory in the new order
02569     4: copy this new order back in situ.
02570     */
02571     num = arg2-arg1+1;
02572     WantAddPointers(num); /* Guarantees the presence of enough pointers */
02573     f = fun+FUNHEAD; n = 1; i = 0;
02574     while ( n < arg1 ) { n++; NEXTARG(f) }
02575     f1 = f;

```

```

02581     while ( n <= arg2 ) { AT.pWorkspace[AT.pWorkPointer+i++] = f; n++; NEXTARG(f) }
02582 /*
02583     Now the cycle(s). First minimize the number of cycles.
02584 */
02585     x = ncy;
02586     if ( x >= i ) {
02587         x %= i;
02588         if ( x > i/2 ) x -= i;
02589     }
02590     else if ( x <= -i ) {
02591         x = -((-x) % i);
02592         if ( x <= -i/2 ) x += i;
02593     }
02594     while ( x ) {
02595         if ( x > 0 ) {
02596             tt = AT.pWorkspace[AT.pWorkPointer+i-1];
02597             for ( j = i-1; j > 0; j-- )
02598                 AT.pWorkspace[AT.pWorkPointer+j] = AT.pWorkspace[AT.pWorkPointer+j-1];
02599             AT.pWorkspace[AT.pWorkPointer] = tt;
02600             x--;
02601         }
02602         else {
02603             tt = AT.pWorkspace[AT.pWorkPointer];
02604             for ( j = 1; j < i; j++ )
02605                 AT.pWorkspace[AT.pWorkPointer+j-1] = AT.pWorkspace[AT.pWorkPointer+j];
02606             AT.pWorkspace[AT.pWorkPointer+j-1] = tt;
02607             x++;
02608         }
02609     }
02610 /*
02611     And the final cleanup
02612 */
02613     if ( tstop+(f-f1) > AT.WorkTop ) goto OverWork;
02614     f2 = tstop;
02615     for ( i = 0; i < num; i++ ) { f = AT.pWorkspace[AT.pWorkPointer+i]; COPY1ARG(f2,f) }
02616     i = f2 - tstop;
02617     NCOPY(f1,tstop,i)
02618 }
02619 else { /* Tensors */
02620     tt = fun+FUNHEAD; tstop = fun+fun[1]; totarg = tstop - tt;
02621     if ( FindRange(BHEAD args,&arg1,&arg2,totarg) ) return(-1);
02622     if ( arg1 > arg2 ) { n = arg1; arg1 = arg2; arg2 = n; }
02623     if ( arg2 > totarg ) return(0);
02624 /*
02625     We need to:
02626     1: get pointers to the arguments
02627     2: cycle the pointers
02628     3: copy the arguments to safe territory in the new order
02629     4: copy this new order back in situ.
02630 */
02631     num = arg2-arg1+1;
02632     WantAddPointers(num); /* Guarantees the presence of enough pointers */
02633     f = fun+FUNHEAD; n = 1; i = 0;
02634     while ( n < arg1 ) { n++; f++; }
02635     f1 = f;
02636     while ( n <= arg2 ) { AT.pWorkspace[AT.pWorkPointer+i++] = f; n++; f++; }
02637 /*
02638     Now the cycle(s). First minimize the number of cycles.
02639 */
02640     x = ncy;
02641     if ( x >= i ) {
02642         x %= i;
02643         if ( x > i/2 ) x -= i;
02644     }
02645     else if ( x <= -i ) {
02646         x = -((-x) % i);
02647         if ( x <= -i/2 ) x += i;
02648     }
02649     while ( x ) {
02650         if ( x > 0 ) {
02651             tt = AT.pWorkspace[AT.pWorkPointer+i-1];
02652             for ( j = i-1; j > 0; j-- )
02653                 AT.pWorkspace[AT.pWorkPointer+j] = AT.pWorkspace[AT.pWorkPointer+j-1];
02654             AT.pWorkspace[AT.pWorkPointer] = tt;
02655             x--;
02656         }
02657         else {
02658             tt = AT.pWorkspace[AT.pWorkPointer];
02659             for ( j = 1; j < i; j++ )
02660                 AT.pWorkspace[AT.pWorkPointer+j-1] = AT.pWorkspace[AT.pWorkPointer+j];
02661             AT.pWorkspace[AT.pWorkPointer+j-1] = tt;
02662             x++;
02663         }
02664     }
02665 /*
02666     And the final cleanup
02667 */

```

```

02668     if ( tstop+(f-f1) > AT.WorkTop ) goto OverWork;
02669     f2 = tstop;
02670     for ( i = 0; i < num; i++ ) { f = AT.pWorkspace[AT.pWorkPointer+i]; *f2++ = *f++; }
02671     i = f2 - tstop;
02672     NCOPY(f1,tstop,i)
02673 }
02674 return(0);
02675 OverWork:;
02676 MLOCK(ErrorMessageLock);
02677 MesWork();
02678 MUNLOCK(ErrorMessageLock);
02679 return(-1);
02680 }
02681
02682 /*
02683     #] RunCycle :
02684     #[ RunAddArg :
02685 */
02686
02687 WORD RunAddArg(PHEAD WORD *fun, WORD *args)
02688 {
02689     WORD *tt, totarg, *tstop, arg1, arg2, n, num, *f, *f1, *f2;
02690     WORD scribble[10*ARGHEAD];
02691     LONG space;
02692     if ( *args != ARGRANGE ) {
02693         MLOCK(ErrorMessageLock);
02694         MesPrint("Illegal range encountered in RunAddArg");
02695         MUNLOCK(ErrorMessageLock);
02696         Terminate(-1);
02697     }
02698     if ( functions[fun[0]-FUNCTION].spec == TENSORFUNCTION ) {
02699         MLOCK(ErrorMessageLock);
02700         MesPrint("Illegal attempt to add arguments of a tensor in AddArg");
02701         MUNLOCK(ErrorMessageLock);
02702         Terminate(-1);
02703     }
02704     tt = fun+FUNHEAD; tstop = fun+fun[1]; totarg = 0;
02705     while ( tt < tstop ) { totarg++; NEXTARG(tt); }
02706     /* ignore functions with no arguments */
02707     if ( totarg == 0 ) return(0);
02708     if ( FindRange(BHEAD args,&arg1,&arg2,totarg) ) return(-1);
02709 /*
02710     We need to:
02711         1: establish that we actually need to add something
02712         2: start a sort
02713         3: if needed, convert arguments to long arguments
02714         4: send (terms in) argument to StoreTerm
02715         5: EndSort and copy the result back into the function
02716     Note that the function is in the workspace, above the term and no
02717     relevant information is trailing it.
02718 */
02719     if ( arg2 < arg1 ) { n = arg1; arg1 = arg2; arg2 = n; }
02720     if ( arg2 > totarg ) return(0);
02721     num = arg2-arg1+1;
02722     if ( num == 1 ) return(0);
02723     f = fun+FUNHEAD; n = 1;
02724     while ( n < arg1 ) { n++; NEXTARG(f) }
02725     f1 = f;
02726     NewSort(BHEAD0);
02727     while ( n <= arg2 ) {
02728         if ( *f > 0 ) {
02729             f2 = f + *f; f += ARGHEAD;
02730             while ( f < f2 ) { StoreTerm(BHEAD f); f += *f; }
02731         }
02732         else if ( *f == -SNUMBER && f[1] == 0 ) {
02733             f+= 2;
02734         }
02735         else {
02736             ToGeneral(f,scribble,1);
02737             StoreTerm(BHEAD scribble);
02738             NEXTARG(f);
02739         }
02740         n++;
02741     }
02742     if ( EndSort(BHEAD tstop+ARGHEAD,1) < 0 ) return(-1);
02743     num = 0;
02744     f2 = tstop+ARGHEAD;
02745     while ( *f2 ) { f2 += *f2; num++; }
02746     *tstop = f2-tstop;
02747     for ( n = 1; n < ARGHEAD; n++ ) tstop[n] = 0;
02748     if ( num == 1 && ToFast(tstop,tstop) == 1 ) {
02749         f2 = tstop; NEXTARG(f2);
02750     }
02751     if ( *tstop == ARGHEAD ) {
02752         *tstop = -SNUMBER; tstop[1] = 0;
02753         f2 = tstop+2;
02754     }

```

```

02755 /*
02756 Copy the trailing arguments after the new argument, then copy the whole back.
02757 */
02758 while ( f < tstop ) *f2++ = *f++;
02759 while ( f < f2 ) *f1++ = *f++;
02760 space = f1 - fun;
02761 if ( (space+8)*sizeof(WORD) > (UWORD)AM.MaxTer ) {
02762     MLOCK(ErrorMessageLock);
02763     MesWork();
02764     MUNLOCK(ErrorMessageLock);
02765     return(-1);
02766 }
02767 fun[1] = (WORD)space;
02768 return(0);
02769 }
02770
02771 /*
02772 #] RunAddArg :
02773 #[ RunMulArg :
02774 */
02775
02776 WORD RunMulArg(PHEAD WORD *fun, WORD *args)
02777 {
02778     WORD *t, totarg, *tstop, arg1, arg2, n, *f, nb, *m, i, *w;
02779     WORD *scratch, argbuf[20], argsize, *where, *newterm;
02780     LONG oldcpointer_pos;
02781     CBUF *C = cbuf + AT.ebufnum;
02782     if ( *args != ARGRANGE ) {
02783         MLOCK(ErrorMessageLock);
02784         MesPrint("Illegal range encountered in RunMulArg");
02785         MUNLOCK(ErrorMessageLock);
02786         Terminate(-1);
02787     }
02788     if ( functions[fun[0]-FUNCTION].spec == TENSORFUNCTION ) {
02789         MLOCK(ErrorMessageLock);
02790         MesPrint("Illegal attempt to multiply arguments of a tensor in MulArg");
02791         MUNLOCK(ErrorMessageLock);
02792         Terminate(-1);
02793     }
02794     t = fun+FUNHEAD; tstop = fun+fun[1]; totarg = 0;
02795     while ( t < tstop ) { totarg++; NEXTARG(t); }
02796     /* ignore functions with no arguments */
02797     if ( totarg == 0 ) return(0);
02798     if ( FindRange(BHEAD args,&arg1,&arg2,totarg) ) return(-1);
02799     if ( arg2 < arg1 ) { n = arg1; arg1 = arg2; arg2 = n; }
02800     if ( arg1 > totarg ) return(0);
02801     if ( arg2 < 1 ) return(0);
02802     if ( arg1 < 1 ) arg1 = 1;
02803     if ( arg2 > totarg ) arg2 = totarg;
02804     if ( arg1 == arg2 ) return(0);
02805 /*
02806 Now we move the arguments to a compiler buffer
02807 Then we create a term in the workspace that is the product of
02808 subexpression pointers to the objects in the compiler buffer.
02809 Next we let Generator work out that term.
02810 Finally we pick up the results from EndSort and put it in the function.
02811 */
02812 f = fun+FUNHEAD; n = 1;
02813 while ( n < arg1 ) { n++; NEXTARG(f) }
02814 t = f;
02815 if ( fun >= AT.WorkSpace && fun < AT.WorkTop ) {
02816     if ( AT.WorkPointer < fun+fun[1] ) AT.WorkPointer = fun+fun[1];
02817 }
02818 scratch = AT.WorkPointer;
02819 w = scratch+1;
02820 oldcpointer_pos = C->Pointer-C->Buffer;
02821 nb = C->numrhs;
02822 while ( n <= arg2 ) {
02823     if ( *t > 0 ) {
02824         argsize = *t - ARGHEAD; where = t + ARGHEAD; t += *t;
02825     }
02826     else if ( *t <= -FUNCTION ) {
02827         argbuf[0] = FUNHEAD+4; argbuf[1] = -*t++; argbuf[2] = FUNHEAD;
02828         for ( i = 2; i < FUNHEAD; i++ ) argbuf[i+1] = 0;
02829         argbuf[FUNHEAD+1] = 1;
02830         argbuf[FUNHEAD+2] = 1;
02831         argbuf[FUNHEAD+3] = 3;
02832         argsize = argbuf[0];
02833         where = argbuf;
02834     }
02835     else if ( *t == -SYMBOL ) {
02836         argbuf[0] = 8; argbuf[1] = SYMBOL; argbuf[2] = 4;
02837         argbuf[3] = t[1]; argbuf[4] = 1;
02838         argbuf[5] = 1; argbuf[6] = 1; argbuf[7] = 3;
02839         argsize = 8; t += 2;
02840         where = argbuf;
02841     }

```

```

02842     else if ( *t == -VECTOR || *t == -MINVECTOR ) {
02843         argbuf[0] = 7; argbuf[1] = INDEX; argbuf[2] = 3;
02844         argbuf[3] = t[1];
02845         argbuf[4] = 1; argbuf[5] = 1;
02846         if ( *t == -MINVECTOR ) argbuf[6] = -3;
02847         else argbuf[6] = 3;
02848         argsize = 7; t += 2;
02849         where = argbuf;
02850     }
02851     else if ( *t == -INDEX ) {
02852         argbuf[0] = 7; argbuf[1] = INDEX; argbuf[2] = 3;
02853         argbuf[3] = t[1];
02854         argbuf[4] = 1; argbuf[5] = 1; argbuf[6] = 3;
02855         argsize = 7; t += 2;
02856         where = argbuf;
02857     }
02858     else if ( *t == -SNUMBER ) {
02859         if ( t[1] < 0 ) {
02860             argbuf[0] = 4; argbuf[1] = -t[1]; argbuf[2] = 1; argbuf[3] = -3;
02861         }
02862         else {
02863             argbuf[0] = 4; argbuf[1] = t[1]; argbuf[2] = 1; argbuf[3] = 3;
02864         }
02865         argsize = 4; t += 2;
02866         where = argbuf;
02867     }
02868     else {
02869         /* unreachable */
02870         return(1);
02871     }
02872 /*
02873     Now add the argbuf to AT.ebufnum
02874 */
02875     m = AddRHS(AT.ebufnum,1);
02876     while ( (m + argsize + 10) > C->Top ) m = DoubleCbuffer(AT.ebufnum,m,17);
02877     for ( i = 0; i < argsize; i++ ) m[i] = where[i];
02878     m[i] = 0;
02879     C->Pointer = m + i + 1;
02880     n++;
02881     *w++ = SUBEXPRESSION; *w++ = SUBEXPSIZE; *w++ = C->numrhs; *w++ = 1;
02882     *w++ = AT.ebufnum; FILLSUB(w);
02883 }
02884 *w++ = 1; *w++ = 1; *w++ = 3;
02885 *scratch = w-scratch;
02886 AT.WorkPointer = w;
02887 NewSort(BHEAD0);
02888 Generator(BHEAD scratch,AR.Cnumlhs);
02889 newterm = AT.WorkPointer;
02890 EndSort(BHEAD newterm+ARGHEAD,1);
02891 C->Pointer = C->Buffer+oldcpointer_pos;
02892 C->numrhs = nb;
02893 w = newterm+ARGHEAD; while ( *w ) w += *w;
02894 *newterm = w-newterm; newterm[1] = 0;
02895 if ( ToFast(newterm,newterm) ) {
02896     if ( *newterm <= -FUNCTION ) w = newterm+1;
02897     else w = newterm+2;
02898 }
02899 while ( t < tstop ) *w++ = *t++;
02900 i = w - newterm;
02901 t = newterm; NCOPY(f,t,i);
02902 fun[1] = f-fun;
02903 AT.WorkPointer = scratch;
02904 if ( AT.WorkPointer > AT.WorkSpace && AT.WorkPointer < f ) AT.WorkPointer = f;
02905 return(0);
02906 }
02907 /*
02908 #] RunMulArg :
02909 #[ RunIsLyndon :
02910
02911     Determines whether the range constitutes a Lyndon word.
02912     The two cases of ordering are distinguished by the order of
02913     the numbers of the arguments in the range.
02914 */
02915 WORD RunIsLyndon(PHEAD WORD *fun, WORD *args, int par)
02916 {
02917     WORD *tt, totarg, *tstop, arg1, arg2, arg, num, *f, n, i;
02918 /* WORD *f1; */
02919     WORD sign, i1, i2, retval;
02920     if ( fun[0] <= GAMMASEVEN && fun[0] >= GAMMA ) return(0);
02921     if ( *args != ARGRANGE ) {
02922         MLOCK(ErrorMessageLock);
02923         MesPrint("Illegal range encountered in RunIsLyndon");
02924         MUNLOCK(ErrorMessageLock);
02925         Terminate(-1);
02926     }
02927 }

```

```

02929     tt = fun+FUNHEAD; tstop = fun+fun[1]; totarg = 0;
02930     while ( tt < tstop ) { totarg++; NEXTARG(tt); }
02931     if ( FindRange(BHEAD args,&arg1,&arg2,totarg) ) return(-1);
02932     if ( arg1 > totarg || arg2 > totarg ) return(-1);
02933     /*
02934     Now make a list of the relevant arguments.
02935     */
02936     if ( arg1 == arg2 ) return(1);
02937     if ( arg2 < arg1 ) { /* greater, rather than smaller */
02938         arg = arg1; arg1 = arg2; arg2 = arg; sign = 1;
02939     }
02940     else sign = 0;
02941     num = arg2-arg1+1;
02942     WantAddPointers(num); /* Guarantees the presence of enough pointers */
02943     f = fun+FUNHEAD; n = 1; i = 0;
02944     while ( n < arg1 ) { n++; NEXTARG(f) }
02945     /* f1 = f; */
02946     while ( n <= arg2 ) { AT.pWorkSpace[AT.pWorkPointer+i++] = f; n++; NEXTARG(f) }
02947     /*
02948     If sign == 1 we should alter the order of the pointers first
02949     */
02950     if ( sign ) {
02951         i1 = i-1; i2 = 0;
02952         while ( i1 > i2 ) {
02953             tt = AT.pWorkSpace[AT.pWorkPointer+i1];
02954             AT.pWorkSpace[AT.pWorkPointer+i1] = AT.pWorkSpace[AT.pWorkPointer+i2];
02955             AT.pWorkSpace[AT.pWorkPointer+i2] = tt;
02956             i1--; i2++;
02957         }
02958     }
02959     /*
02960     The argument range is from f1 to f and the num pointers to the arguments
02961     are in AT.pWorkSpace[AT.pWorkPointer] to AT.pWorkSpace[AT.pWorkPointer+num-1]
02962     */
02963     for ( i1 = 1; i1 < num; i1++ ) {
02964         retval = par * CompArg(AT.pWorkSpace[AT.pWorkPointer+i1],
02965                               AT.pWorkSpace[AT.pWorkPointer]);
02966         if ( retval > 0 ) continue;
02967         if ( retval < 0 ) return(0);
02968         for ( i2 = 1; i2 < num; i2++ ) {
02969             retval = par * CompArg(AT.pWorkSpace[AT.pWorkPointer+(i1+i2)%num],
02970                                   AT.pWorkSpace[AT.pWorkPointer+i2]);
02971             if ( retval < 0 ) return(0);
02972             if ( retval > 0 ) goto nextil;
02973         }
02974     }
02975     /*
02976     If we come here the sequence is not unique.
02977     */
02978     return(0);
02979 nextil:;
02980     }
02981     return(1);
02982 }
02983 /*
02984 #] RunIsLyndon :
02985 #[ RunToLyndon :
02986
02987 Determines whether the range constitutes a Lyndon word.
02988 If not, we rotate it to a Lyndon word. If this is not possible
02989 we return the noLyndon condition.
02990 The two cases of ordering are distinguished by the order of
02991 the numbers of the arguments in the range.
02992 */
02993 WORD RunToLyndon(PHEAD WORD *fun, WORD *args, int par)
02994 {
02995     WORD *tt, totarg, *tstop, arg1, arg2, arg, num, *f, *f1, *f2, n, i;
02996     WORD sign, i1, i2, retval, unique;
02997     if ( fun[0] <= GAMMASEVEN && fun[0] >= GAMMA ) return(0);
02998     if ( *args != ARGRANGE ) {
02999         MLOCK(ErrorMessageLock);
03000         MesPrint("Illegal range encountered in RunToLyndon");
03001         MUNLOCK(ErrorMessageLock);
03002         Terminate(-1);
03003     }
03004     tt = fun+FUNHEAD; tstop = fun+fun[1]; totarg = 0;
03005     while ( tt < tstop ) { totarg++; NEXTARG(tt); }
03006     if ( FindRange(BHEAD args,&arg1,&arg2,totarg) ) return(-1);
03007     if ( arg1 > totarg || arg2 > totarg ) return(-1);
03008     /*
03009     Now make a list of the relevant arguments.
03010     */
03011     if ( arg1 == arg2 ) return(1);
03012     if ( arg2 < arg1 ) { /* greater, rather than smaller */
03013         arg = arg1; arg1 = arg2; arg2 = arg; sign = 1;
03014     }
03015     else sign = 0;

```

```

03016     }
03017     else sign = 0;
03018
03019     num = arg2-arg1+1;
03020     WantAddPointers((2*num)); /* Guarantees the presence of enough pointers */
03021     f = fun+FUNHEAD; n = 1; i = 0;
03022     while ( n < arg1 ) { n++; NEXTARG(f) }
03023     f1 = f;
03024     while ( n <= arg2 ) { AT.pWorkSpace[AT.pWorkPointer+i++] = f; n++; NEXTARG(f) }
03025 /*
03026     If sign == 1 we should alter the order of the pointers first
03027 */
03028     if ( sign ) {
03029         i1 = i-1; i2 = 0;
03030         while ( i1 > i2 ) {
03031             tt = AT.pWorkSpace[AT.pWorkPointer+i1];
03032             AT.pWorkSpace[AT.pWorkPointer+i1] = AT.pWorkSpace[AT.pWorkPointer+i2];
03033             AT.pWorkSpace[AT.pWorkPointer+i2] = tt;
03034             i1--; i2++;
03035         }
03036     }
03037 /*
03038     The argument range is from f1 to f and the num pointers to the arguments
03039     are in AT.pWorkSpace[AT.pWorkPointer] to AT.pWorkSpace[AT.pWorkPointer+num-1]
03040 */
03041     unique = 1;
03042     for ( i1 = 1; i1 < num; i1++ ) {
03043         retval = par * CompArg(AT.pWorkSpace[AT.pWorkPointer+i1],
03044                               AT.pWorkSpace[AT.pWorkPointer]);
03045         if ( retval > 0 ) continue;
03046         if ( retval < 0 ) {
03047 Rotate;
03048 /*
03049             Rotate so that i1 becomes the zero element. Then start again.
03050 */
03051             for ( i2 = 0; i2 < num; i2++ ) {
03052                 AT.pWorkSpace[AT.pWorkPointer+num+i2] =
03053                     AT.pWorkSpace[AT.pWorkPointer+(i1+i2)%num];
03054             }
03055             for ( i2 = 0; i2 < num; i2++ ) {
03056                 AT.pWorkSpace[AT.pWorkPointer+i2] =
03057                     AT.pWorkSpace[AT.pWorkPointer+i2+num];
03058             }
03059             i1 = 0;
03060             goto nextil;
03061         }
03062         for ( i2 = 1; i2 < num; i2++ ) {
03063             retval = par * CompArg(AT.pWorkSpace[AT.pWorkPointer+(i1+i2)%num],
03064                                   AT.pWorkSpace[AT.pWorkPointer+i2]);
03065             if ( retval < 0 ) goto Rotate;
03066             if ( retval > 0 ) goto nextil;
03067         }
03068 /*
03069             If we come here the sequence is not unique.
03070 */
03071             unique = 0;
03072 nextil;
03073         }
03074         if ( sign ) {
03075             i1 = i-1; i2 = 0;
03076             while ( i1 > i2 ) {
03077                 tt = AT.pWorkSpace[AT.pWorkPointer+i1];
03078                 AT.pWorkSpace[AT.pWorkPointer+i1] = AT.pWorkSpace[AT.pWorkPointer+i2];
03079                 AT.pWorkSpace[AT.pWorkPointer+i2] = tt;
03080                 i1--; i2++;
03081             }
03082         }
03083 /*
03084             Now rewrite the arguments into the proper order
03085 */
03086             if ( tstop+(f-f1) > AT.WorkTop ) goto OverWork;
03087             f2 = tstop;
03088             for ( i = 0; i < num; i++ ) { f = AT.pWorkSpace[AT.pWorkPointer+i]; COPY1ARG(f2,f) }
03089             i = f2 - tstop;
03090             NCOPY(f1,tstop,i)
03091 /*
03092             The return value indicates whether we have a Lyndon word
03093 */
03094             return(unique);
03095 OverWork;
03096             MLOCK(ErrorMessageLock);
03097             MesWork();
03098             MUNLOCK(ErrorMessageLock);
03099             return(-2);
03100 }
03101
03102 /*

```



```

03103         #] RunToLyndon :
03104         #[ RunDropArg :
03105     */
03106
03107 WORD RunDropArg(PHEAD WORD *fun, WORD *args)
03108 {
03109     WORD *t, *tstop, *f, totarg, arg1, arg2, n;
03110
03111     t = fun+FUNHEAD; tstop = fun+fun[1]; totarg = 0;
03112     while ( t < tstop ) { totarg++; NEXTARG(t); }
03113     if ( FindRange(BHEAD args,&arg1,&arg2,totarg) ) return(-1);
03114     if ( arg2 < arg1 ) { n = arg1; arg1 = arg2; arg2 = n; }
03115     if ( arg1 > totarg ) return(0);
03116     if ( arg2 < 1 ) return(0);
03117     if ( arg1 < 1 ) arg1 = 1;
03118     if ( arg2 > totarg ) arg2 = totarg;
03119     f = fun+FUNHEAD; n = 1;
03120     while ( n < arg1 ) { n++; NEXTARG(f) }
03121     t = f;
03122     while ( n <= arg2 ) { n++; NEXTARG(t) }
03123     while ( t < tstop ) *f++ = *t++;
03124     fun[1] = f-fun;
03125     return(0);
03126 }
03127
03128 /*
03129     #] RunDropArg :
03130     #[ RunSelectArg :
03131 */
03132
03133 WORD RunSelectArg(PHEAD WORD *fun, WORD *args)
03134 {
03135     WORD *t, *tstop, *f, *tt, totarg, arg1, arg2, n;
03136
03137     t = fun+FUNHEAD; tstop = fun+fun[1]; totarg = 0;
03138     while ( t < tstop ) { totarg++; NEXTARG(t); }
03139     if ( FindRange(BHEAD args,&arg1,&arg2,totarg) ) return(-1);
03140     if ( arg2 < arg1 ) { n = arg1; arg1 = arg2; arg2 = n; }
03141     if ( arg1 > totarg ) return(0);
03142     if ( arg2 < 1 ) return(0);
03143     if ( arg1 < 1 ) arg1 = 1;
03144     if ( arg2 > totarg ) arg2 = totarg;
03145     f = fun+FUNHEAD; n = 1; t = f;
03146     while ( n < arg1 ) { n++; NEXTARG(t) }
03147     while ( n <= arg2 ) {
03148         tt = t; NEXTARG(tt)
03149         while ( t < tt ) *f++ = *t++;
03150         n++;
03151     }
03152     fun[1] = f-fun;
03153     return(0);
03154 }
03155
03156 /*
03157     #] RunSelectArg :
03158     #[ RunZtoHArg :
03159 */
03160
03161 WORD RunZtoHArg(PHEAD WORD *fun, WORD *args)
03162 {
03163     WORD *tt, totarg, *tstop, arg1, arg2, n, i, *f, *f1, sign = 0;
03164     WORD *t, *t1, *t2, *t3;
03165     if ( *args != ARGGRANGE ) {
03166         MLOCK(ErrorMessageLock);
03167         MesPrint("Illegal range encountered in RunZtoHArg.");
03168         MUNLOCK(ErrorMessageLock);
03169         Terminate(-1);
03170     }
03171     if ( functions[fun[0]-FUNCTION].spec != 0 ) {
03172         MLOCK(ErrorMessageLock);
03173         MesPrint("The ZtoH transformation can only be executed on regular functions with nonzero
integer arguments.");
03174         MUNLOCK(ErrorMessageLock);
03175         Terminate(-1);
03176     }
03177     tt = fun+FUNHEAD; tstop = fun+fun[1]; totarg = 0;
03178     while ( tt < tstop ) { totarg++; NEXTARG(tt); }
03179     if ( FindRange(BHEAD args,&arg1,&arg2,totarg) ) return(-1);
03180 /*
03181     Check the arguments. Should be -SNUMBER x!=0
03182 */
03183     f = fun+FUNHEAD; n = 1;
03184     while ( n < arg1 ) { n++; NEXTARG(f) }
03185     f1 = f;
03186     for ( i = arg1; i <= arg2; i++, f += 2 ) {
03187         if ( *f != -SNUMBER || f[1] == 0 ) return(-1);
03188     }

```

```

03189 /*
03190     Now we need a copy.
03191 */
03192 t = f1; t1 = t2 = tt = TermMalloc("RunZtoHArg");
03193 while ( t < f ) { *t1++ = *t++; *t1++ = *t++; }
03194 t = f1;
03195 while ( t2 < t1 ) {
03196     t += 2;
03197     if ( t2[1] < 0 ) {
03198         t3 = t;
03199         while ( t3 < f ) { t3[1] = -t3[1]; t3 += 2; }
03200     }
03201     t2 += 2;
03202 }
03203 TermFree(tt,"RunZtoHArg");
03204 /*
03205     Now the overall sign.
03206 */
03207 while ( f1 < f ) { if ( f1[1] < 0 ) sign = 1-sign; f1 += 2; }
03208 return(sign);
03209 }
03210
03211 /*
03212     #] RunZtoHArg :
03213     #[ RunHtoZArg :
03214 */
03215
03216 WORD RunHtoZArg(PHEAD WORD *fun, WORD *args)
03217 {
03218     WORD *tt, totarg, *tstop, arg1, arg2, n, i, *f, *f1, *f2, sign = 0;
03219     WORD *t, *t1, *t2;
03220     if ( *args != ARGGRANGE ) {
03221         MLOCK(ErrorMessageLock);
03222         MesPrint("Illegal range encountered in RunZtoHArg.");
03223         MUNLOCK(ErrorMessageLock);
03224         Terminate(-1);
03225     }
03226     if ( functions[fun[0]-FUNCTION].spec != 0 ) {
03227         MLOCK(ErrorMessageLock);
03228         MesPrint("The HtoZ transformation can only be executed on regular functions with nonzero
integer arguments.");
03229         MUNLOCK(ErrorMessageLock);
03230         Terminate(-1);
03231     }
03232     tt = fun+FUNHEAD; tstop = fun+fun[1]; totarg = 0;
03233     while ( tt < tstop ) { totarg++; NEXTARG(tt); }
03234     if ( FindRange(BHEAD args,&arg1,&arg2,totarg) ) return(-1);
03235 /*
03236     Check the arguments. Should be -SNUMBER x!=0
03237 */
03238 f = fun+FUNHEAD; n = 1;
03239 while ( n < arg1 ) { n++; NEXTARG(f) }
03240 f2 = f1 = f;
03241 for ( i = arg1; i <= arg2; i++, f += 2 ) {
03242     if ( *f != -SNUMBER || f[1] == 0 ) return(-1);
03243 }
03244 /*
03245     First the overall sign.
03246 */
03247 while ( f2 < f ) { if ( f2[1] < 0 ) sign = 1-sign; f2 += 2; }
03248 /*
03249     Now we need a copy.
03250 */
03251 t = f1; t1 = tt = TermMalloc("RunHtoZArg");
03252 while ( t < f ) { *t1++ = *t++; *t1++ = *t++; }
03253 /*
03254     Now the transformation.
03255 */
03256 t = f1; t2 = tt + 2;
03257 while ( t2 < t1 ) {
03258     t += 2;
03259     if ( t2[-1] < 0 ) t[1] = -t[1];
03260     t2 += 2;
03261 }
03262 TermFree(tt,"RunHtoZArg");
03263 return(sign);
03264 }
03265
03266 /*
03267     #] RunHtoZArg :
03268     #[ TestArgNum :
03269
03270     Looks whether argument n is contained in any of the ranges
03271     specified in args. Args contains objects of the types
03272         ALLARGS
03273         NUMARG,num
03274         ARGGRANGE,num1,num2

```

```

03275         The object MAKEARGS,num1,num2 is skipped
03276         Any other object terminates the range specifications.
03277
03278         Currently only ARGGRANGE is used (10-may-2016)
03279  */
03280
03281 int TestArgNum(int n, int totarg, WORD *args)
03282 {
03283     GETIDENTITY
03284     WORD x1, x2;
03285     for(;;) {
03286         switch ( *args ) {
03287             case ALLARGS:
03288                 return(1);
03289             case NUMARG:
03290                 if ( n == args[1] ) return(1);
03291                 if ( args[1] >= MAXPOSITIVE4 ) {
03292                     x1 = args[1]-MAXPOSITIVE4;
03293                     if ( totarg-x1 == n ) return(1);
03294                 }
03295                 args += 2;
03296                 break;
03297             case ARGGRANGE:
03298                 if ( args[1] >= MAXPOSITIVE2 ) {
03299                     x1 = args[1] - MAXPOSITIVE2;
03300                     if ( x1 > MAXPOSITIVE4 ) {
03301                         x1 = x1 - MAXPOSITIVE4;
03302                         x1 = DolToNumber(BHEAD x1);
03303                         x1 = totarg - x1;
03304                     }
03305                     else {
03306                         x1 = DolToNumber(BHEAD x1);
03307                     }
03308                 }
03309                 else if ( args[1] >= MAXPOSITIVE4 ) {
03310                     x1 = totarg-(args[1]-MAXPOSITIVE4);
03311                 }
03312                 else x1 = args[1];
03313                 if ( args[2] >= MAXPOSITIVE2 ) {
03314                     x2 = args[2] - MAXPOSITIVE2;
03315                     if ( x2 > MAXPOSITIVE4 ) {
03316                         x2 = x2 - MAXPOSITIVE4;
03317                         x2 = DolToNumber(BHEAD x2);
03318                         x2 = totarg - x2;
03319                     }
03320                     else {
03321                         x2 = DolToNumber(BHEAD x2);
03322                     }
03323                 }
03324                 else if ( args[2] >= MAXPOSITIVE4 ) {
03325                     x2 = totarg-(args[2]-MAXPOSITIVE4);
03326                 }
03327                 else x2 = args[2];
03328                 if ( x1 >= x2 ) {
03329                     if ( n >= x2 && n <= x1 ) return(1);
03330                 }
03331                 else {
03332                     if ( n >= x1 && n <= x2 ) return(1);
03333                 }
03334                 args += 3;
03335                 break;
03336             case MAKEARGS:
03337                 args += 3;
03338                 break;
03339             default:
03340                 return(0);
03341         }
03342     }
03343 }
03344
03345  /*
03346     #] TestArgNum :
03347     #[ PutArgInScratch :
03348  */
03349
03350 WORD PutArgInScratch(WORD *arg,UWORD *scrat)
03351 {
03352     WORD size, *t, i;
03353     if ( *arg == -SNUMBER ) {
03354         scrat[0] = ABS(arg[1]);
03355         if ( arg[1] < 0 ) size = -1;
03356         else size = 1;
03357     }
03358     else {
03359         t = arg+*arg-1;
03360         if ( *t < 0 ) { i = ((*t)-1)/2; size = -i; }
03361         else { i = ( *t -1)/2; size = i; }

```

```

03362         t = arg+ARGHEAD+1;
03363         NCOPY(scrat,t,i);
03364     }
03365     return(size);
03366 }
03367
03368 /*
03369     #] PutArgInScratch :
03370     #[ ReadRange :
03371
03372     Comes in at the bracket and leaves at the = sign
03373     Ranges can be:
03374     #1,#2 with # numbers. If the second is smaller than the
03375         first we work it backwards.
03376     first,#2 or #2,first
03377     #1,last or last,#1
03378     first,last or last,first
03379     First is represented by 1. Last is represented by MAXPOSITIVE4.
03380
03381     par = 0: we need the = after.
03382     par = 1: we need a , or '\0' after.
03383     par = 2: we need a :
03384 */
03385
03386 UBYTE *ReadRange(UBYTE *s, WORD *out, int par)
03387 {
03388     UBYTE *in = s, *ss, c;
03389     LONG x1, x2;
03390
03391     SKIPBRA3(in)
03392     if ( par == 0 && in[1] != '=' ) {
03393         MesPrint("%A range in this type of transform statement should be followed by an = sign");
03394         return(0);
03395     }
03396     else if ( par == 1 && in[1] != ',' && in[1] != '\0' ) {
03397         MesPrint("%A range in this type of transform statement should be followed by a comma or
end-of-statement");
03398         return(0);
03399     }
03400     else if ( par == 2 && in[1] != ':' ) {
03401         MesPrint("%A range in this type of transform statement should be followed by a :");
03402         return(0);
03403     }
03404     s++;
03405     if ( FG.cTable[*s] == 0 ) {
03406         ss = s; while ( FG.cTable[*s] == 0 ) s++;
03407         c = *s; *s = 0;
03408         if ( StrICmp(ss,(UBYTE *)"first") == 0 ) {
03409             *s = c;
03410             x1 = 1;
03411         }
03412         else if ( StrICmp(ss,(UBYTE *)"last") == 0 ) {
03413             *s = c;
03414             if ( c == '-' ) {
03415                 s++;
03416                 if ( *s == '$' ) {
03417                     s++; ss = s;
03418                     while ( FG.cTable[*s] == 0 || FG.cTable[*s] == 1 ) s++;
03419                     c = *s; *s = 0;
03420                     if ( ( x1 = GetDollar(ss) ) < 0 ) goto Error;
03421                     *s = c;
03422                     x1 += MAXPOSITIVE2;
03423                 }
03424                 else {
03425                     x1 = 0;
03426                     while ( *s >= '0' && *s <= '9' ) {
03427                         x1 = 10*x1 + *s++ - '0';
03428                         if ( x1 >= MAXPOSITIVE4 ) {
03429                             MesPrint("&Fixed range indicator bigger than %1", (LONG)MAXPOSITIVE4);
03430                             return(0);
03431                         }
03432                     }
03433                 }
03434                 x1 += MAXPOSITIVE4;
03435             }
03436             else x1 = MAXPOSITIVE4;
03437         }
03438         else {
03439             MesPrint("&Illegal keyword inside range specification");
03440             return(0);
03441         }
03442     }
03443     else if ( FG.cTable[*s] == 1 ) {
03444         x1 = 0;
03445         while ( *s >= '0' && *s <= '9' ) {
03446             x1 = x1*10 + *s++ - '0';
03447             if ( x1 >= MAXPOSITIVE4 ) {

```

```

03448         MesPrint("&Fixed range indicator bigger than %1", (LONG)MAXPOSITIVE4);
03449         return(0);
03450     }
03451 }
03452 }
03453 else if ( *s == '$' ) {
03454     s++; ss = s;
03455     while ( FG.cTable[*s] == 0 || FG.cTable[*s] == 1 ) s++;
03456     c = *s; *s = 0;
03457     if ( ( x1 = GetDollar(ss) ) < 0 ) goto Error;
03458     *s = c;
03459     x1 += MAXPOSITIVE2;
03460 }
03461 else {
03462     MesPrint("&Illegal character in range specification");
03463     return(0);
03464 }
03465 if ( *s != ',' ) {
03466     MesPrint("&A range is two indicators, separated by a comma or blank");
03467     return(0);
03468 }
03469 s++;
03470 if ( FG.cTable[*s] == 0 ) {
03471     ss = s; while ( FG.cTable[*s] == 0 ) s++;
03472     c = *s; *s = 0;
03473     if ( StrICmp(ss, (UBYTE *) "first" ) == 0 ) {
03474         *s = c;
03475         x2 = 1;
03476     }
03477     else if ( StrICmp(ss, (UBYTE *) "last" ) == 0 ) {
03478         *s = c;
03479         if ( c == '-' ) {
03480             s++;
03481             if ( *s == '$' ) {
03482                 s++; ss = s;
03483                 while ( FG.cTable[*s] == 0 || FG.cTable[*s] == 1 ) s++;
03484                 c = *s; *s = 0;
03485                 if ( ( x2 = GetDollar(ss) ) < 0 ) goto Error;
03486                 *s = c;
03487                 x2 += MAXPOSITIVE2;
03488             }
03489             else {
03490                 x2 = 0;
03491                 while ( *s >= '0' && *s <= '9' ) {
03492                     x2 = 10*x2 + *s++ - '0';
03493                     if ( x2 >= MAXPOSITIVE4 ) {
03494                         MesPrint("&Fixed range indicator bigger than %1", (LONG)MAXPOSITIVE4);
03495                         return(0);
03496                     }
03497                 }
03498             }
03499             x2 += MAXPOSITIVE4;
03500         }
03501         else x2 = MAXPOSITIVE4;
03502     }
03503     else {
03504         MesPrint("&Illegal keyword inside range specification");
03505         return(0);
03506     }
03507 }
03508 else if ( FG.cTable[*s] == 1 ) {
03509     x2 = 0;
03510     while ( *s >= '0' && *s <= '9' ) {
03511         x2 = x2*10 + *s++ - '0';
03512         if ( x2 >= MAXPOSITIVE4 ) {
03513             MesPrint("&Fixed range indicator bigger than %1", (LONG)MAXPOSITIVE4);
03514             return(0);
03515         }
03516     }
03517 }
03518 else if ( *s == '$' ) {
03519     s++; ss = s;
03520     while ( FG.cTable[*s] == 0 || FG.cTable[*s] == 1 ) s++;
03521     c = *s; *s = 0;
03522     if ( ( x2 = GetDollar(ss) ) < 0 ) goto Error;
03523     *s = c;
03524     x2 += MAXPOSITIVE2;
03525 }
03526 else {
03527     MesPrint("&Illegal character in range specification");
03528     return(0);
03529 }
03530 if ( s < in ) {
03531     MesPrint("&A range is two indicators, separated by a comma or blank between parentheses");
03532     return(0);
03533 }
03534 out[0] = x1; out[1] = x2;

```

```

03535     return(in+1);
03536 Error:
03537     MesPrint("&Undefined variable %s in range",ss);
03538     return(0);
03539 }
03540
03541 /*
03542     #] ReadRange :
03543     #[ FindRange :
03544 */
03545
03546 int FindRange(PHEAD WORD *args, WORD *arg1, WORD *arg2, WORD totarg)
03547 {
03548     WORD n[2], fromlast, i;
03549     for ( i = 0; i < 2; i++ ) {
03550         n[i] = args[i+1];
03551         fromlast = 0;
03552         if ( n[i] >= MAXPOSITIVE2 ) { /* This is a dollar variable */
03553             n[i] -= MAXPOSITIVE2;
03554             if ( n[i] >= MAXPOSITIVE4 ) {
03555                 fromlast = 1;
03556                 n[i] -= MAXPOSITIVE4; /* Now we have the number of the dollar variable */
03557             }
03558             n[i] = DolToNumber(BHEAD n[i]);
03559             if ( AN.ErrorInDollar ) {
03560                 MLOCK(ErrorMessageLock);
03561                 MesPrint("Illegal $ value in range while executing transform statement.");
03562                 MUNLOCK(ErrorMessageLock);
03563                 return(-1);
03564             }
03565             if ( fromlast ) n[i] = totarg-n[i];
03566         }
03567         else if ( n[i] >= MAXPOSITIVE4 ) { n[i] = totarg-(n[i]-MAXPOSITIVE4); }
03568         if ( n[i] <= 0 ) {
03569             MLOCK(ErrorMessageLock);
03570             MesPrint("Illegal non-positive value in range (%d) while executing transform statement.",
03571 i+1);
03572             MUNLOCK(ErrorMessageLock);
03573             return(-1);
03574         }
03575     }
03576     *arg1 = n[0];
03577     *arg2 = n[1];
03578     return(0);
03579 }
03580 /*
03581     #] FindRange :
03582     #[ Transform :
03583 */

```

4.152 unix.h File Reference

Macros

- #define [LINEFEED](#) '\n'
- #define [CARRIAGERETURN](#) 0x0D
- #define [WITHPIPE](#)
- #define [WITHSYSTEM](#)
- #define [WITHEXTERNALCHANNEL](#)
- #define [TRAP SIGNALS](#)
- #define [P_term](#)(code) exit(((int)(code<0?-code:code))
- #define [SEPARATOR](#) '/'
- #define [ALTSEPARATOR](#) '/'
- #define [PATHSEPARATOR](#) ':'
- #define [WITH_ENV](#)
- #define [WITH_ALARM](#)

4.152.1 Detailed Description

Settings for Unix-like systems.

Definition in file [unix.h](#).

4.152.2 Macro Definition Documentation

4.152.2.1 LINEFEED

```
#define LINEFEED '\n'
```

Definition at line 33 of file [unix.h](#).

4.152.2.2 CARRIAGERETURN

```
#define CARRIAGERETURN 0x0D
```

Definition at line 34 of file [unix.h](#).

4.152.2.3 WITHPIPE

```
#define WITHPIPE
```

Definition at line 36 of file [unix.h](#).

4.152.2.4 WITHSYSTEM

```
#define WITHSYSTEM
```

Definition at line 37 of file [unix.h](#).

4.152.2.5 WITHEXTERNALCHANNEL

```
#define WITHEXTERNALCHANNEL
```

Definition at line 46 of file [unix.h](#).

4.152.2.6 TRAP SIGNALS

```
#define TRAP SIGNALS
```

Definition at line 49 of file [unix.h](#).

4.152.2.7 P_term

```
#define P_term(  
    code ) exit((int) (code<0?-code:code))
```

Definition at line 52 of file [unix.h](#).

4.152.2.8 SEPARATOR

```
#define SEPARATOR '/'
```

Definition at line 54 of file [unix.h](#).

4.152.2.9 ALTSEPARATOR

```
#define ALTSEPARATOR '/'
```

Definition at line 55 of file [unix.h](#).

4.152.2.10 PATHSEPARATOR

```
#define PATHSEPARATOR ':'
```

Definition at line 56 of file [unix.h](#).

4.152.2.11 WITH_ENV

```
#define WITH_ENV
```

Definition at line 57 of file [unix.h](#).

4.152.2.12 WITH_ALARM

```
#define WITH_ALARM
```

Definition at line 58 of file [unix.h](#).

4.153 unix.h

[Go to the documentation of this file.](#)

```
00001
00006 /* # [ License : */
00007 /*
00008 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00009 * When using this file you are requested to refer to the publication
00010 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00011 * This is considered a matter of courtesy as the development was paid
00012 * for by FOM the Dutch physics granting agency and we would like to
00013 * be able to track its scientific use to convince FOM of its value
00014 * for the community.
00015 *
00016 * This file is part of FORM.
00017 *
00018 * FORM is free software: you can redistribute it and/or modify it under the
00019 * terms of the GNU General Public License as published by the Free Software
00020 * Foundation, either version 3 of the License, or (at your option) any later
00021 * version.
00022 *
00023 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00024 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00025 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00026 * details.
00027 *
```



```

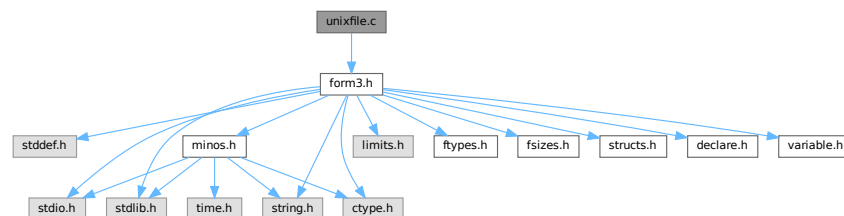
00028 *   You should have received a copy of the GNU General Public License along
00029 *   with FORM.  If not, see <http://www.gnu.org/licenses/>.
00030 */
00031 /* #] License : */
00032
00033 #define LINEFEED '\n'
00034 #define CARRIAGERETURN 0x0D
00035
00036 #define WITHPIPE
00037 #define WITHSYSTEM
00038
00039 /*[13jul2005 mt]:*/
00040 /*With SAFESIGNAL defined, write() and read() syscalls are wrapped by
00041 the errno checkup*/
00042 /*#define SAFESIGNAL*/
00043 /*:[13jul2005 mt]*/
00044
00045 /*[29apr2004 mt]:*/
00046 #define WITHEXTERNALCHANNEL
00047 /*
00048 */
00049 #define TRAP SIGNALS
00050 /*:[29apr2004 mt]*/
00051
00052 #define P_term(code)    exit((int)(code<0?-code:code))
00053
00054 #define SEPARATOR '/'
00055 #define ALTSEPARATOR '/'
00056 #define PATHSEPARATOR ':'
00057 #define WITH_ENV
00058 #define WITH_ALARM

```

4.154 unixfile.c File Reference

#include "form3.h"

Include dependency graph for unixfile.c:



4.154.1 Detailed Description

The interface to a fast variety of file routines in the unix system.

Definition in file [unixfile.c](#).

4.155 unixfile.c

[Go to the documentation of this file.](#)

```

00001
00005 /* #] License : */
00006 /*
00007 *   Copyright (C) 1984-2023 J.A.M. Vermaseren
00008 *   When using this file you are requested to refer to the publication
00009 *   J.A.M.Vermaseren "New features of FORM" math-ph/0010025

```

```

00010 *   This is considered a matter of courtesy as the development was paid
00011 *   for by FOM the Dutch physics granting agency and we would like to
00012 *   be able to track its scientific use to convince FOM of its value
00013 *   for the community.
00014 *
00015 *   This file is part of FORM.
00016 *
00017 *   FORM is free software: you can redistribute it and/or modify it under the
00018 *   terms of the GNU General Public License as published by the Free Software
00019 *   Foundation, either version 3 of the License, or (at your option) any later
00020 *   version.
00021 *
00022 *   FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00023 *   WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00024 *   FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00025 *   details.
00026 *
00027 *   You should have received a copy of the GNU General Public License along
00028 *   with FORM. If not, see <http://www.gnu.org/licenses/>.
00029 */
00030 /* #] License : */
00031 /*
00032     #[ Includes :
00033
00034     File with direct interface to the UNIX functions.
00035     This makes a big difference in speed!
00036 */
00037 #include "form3.h"
00038
00039
00040 /*[13jul2005 mt]:*/
00041 #ifdef SAFESIGNAL
00042 /*[15oct2004 mt]:*/
00043 /*To access errno variable and EINTR constant:*/
00044 #include <errno.h>
00045 /*:[15oct2004 mt]*/
00046 #endif
00047 /*:[13jul2005 mt]*/
00048
00049 #ifdef UFILES
00050
00051 FILES TheStdout = { 1 };
00052 FILES *Ustdout = &TheStdout;
00053
00054 #ifdef GPP
00055 extern "C" open();
00056 extern "C" close();
00057 extern "C" read();
00058 extern "C" write();
00059 extern "C" lseek();
00060 #endif
00061
00062 /*
00063     #] Includes :
00064     #[ Uopen :
00065 */
00066
00067 FILES *Uopen(char *filename, char *mode)
00068 {
00069     FILES *f = (FILES *)Malloc1(sizeof(FILES), "Uopen");
00070     int flags = 0, rights = 0644;
00071     while ( *mode ) {
00072         if ( *mode == 'r' ) { flags |= O_RDONLY; }
00073         else if ( *mode == 'w' ) { flags |= O_CREAT | O_TRUNC; }
00074         else if ( *mode == 'a' ) { flags |= O_APPEND; }
00075         else if ( *mode == 'b' ) { }
00076         else if ( *mode == '+' ) { flags |= O_RDWR; }
00077         mode++;
00078     }
00079     f->descriptor = open(filename, flags, rights);
00080     if ( f->descriptor >= 0 ) return(f);
00081     if ( ( flags & O_APPEND ) != 0 ) {
00082         flags |= O_CREAT;
00083         f->descriptor = open(filename, flags, rights);
00084         if ( f->descriptor >= 0 ) return(f);
00085     }
00086     M_free(f, "Uopen");
00087     return(0);
00088 }
00089
00090 /*
00091     #] Uopen :
00092     #[ Uclose :
00093 */
00094
00095 int Uclose(FILES *f)
00096 {

```

```

00097     int retval;
00098     if ( f ) {
00099         retval = close(f->descriptor);
00100         M_free(f,"Uclose");
00101         return(retval);
00102     }
00103     return(EOF);
00104 }
00105
00106 /*
00107     #] Uclose :
00108     #[ Uread :
00109 */
00110
00111
00112 size_t Uread(char *ptr, size_t size, size_t nobj, FILES *f)
00113 {
00114     /*[13jul2005 mt]:*/
00115     #ifdef SAFESIGNAL
00116
00117     /*[15oct2004 mt]:*/
00118     /*Operation read() can be interrupted by a signal. Note, this is rather unlikely,
00119     so we do not save size*nobj for future attempts*/
00120     size_t ret;
00121     /*If the syscall is interrupted by a signal before it
00122     succeeded in getting any progress, it must be repeated:*/
00123     while( ( ret=read(f->descriptor,ptr,size*nobj)<1)&&(errno == EINTR) );
00124     return(ret);
00125     #else
00126     #ifdef DEEPDEBUG
00127     {
00128         POSITION pos;
00129         SETBASEPOSITION(pos,lseek(f->descriptor,0L,SEEK_CUR));
00130         MesPrint("handle %d: reading %ld bytes from position %p\n",f->descriptor,size*nobj,pos);
00131     }
00132     #endif
00133
00134     return(read(f->descriptor,ptr,size*nobj));
00135
00136     #endif
00137     /*:[15oct2004 mt]:*/
00138     /*:[13jul2005 mt]:*/
00139 }
00140
00141 /*
00142     #] Uread :
00143     #[ Uwrite :
00144 */
00145
00146 size_t Uwrite(char *ptr, size_t size, size_t nobj, FILES *f)
00147 {
00148     /*[13jul2005 mt]:*/
00149     #ifdef SAFESIGNAL
00150     /*[15oct2004 mt]:*/
00151     /*Operation write() can be interrupted by a signal. */
00152     size_t ret;
00153     size_t thesize=size*nobj;
00154     /*If the syscall is interrupted by a signal before it
00155     succeeded in getting any progress, it must be repeated:*/
00156     while( ( ret=write(f->descriptor,ptr,thesize)<1 )&&(errno == EINTR) );
00157
00158     return(ret);
00159     #else
00160     #ifdef DEEPDEBUG
00161     {
00162         POSITION pos;
00163         SETBASEPOSITION(pos,lseek(f->descriptor,0L,SEEK_CUR));
00164         MesPrint("handle %d: writing %ld bytes to position %p\n",f->descriptor,size*nobj,pos);
00165     }
00166     #endif
00167
00168     return(write(f->descriptor,ptr,size*nobj));
00169
00170     /*:[15oct2004 mt]:*/
00171     /*:[13jul2005 mt]:*/
00172 }
00173
00174 /*
00175     #] Uwrite :
00176     #[ Useek :
00177 */
00178
00179 int Useek(FILES *f, off_t offset, int origin)
00180 {
00181     if ( f && ( lseek(f->descriptor,offset,origin) >= 0 ) ) return(0);
00182     return(-1);
00183 }

```

```

00184
00185 /*
00186     #] Useek :
00187     #[ Utell :
00188 */
00189
00190 off_t Utell(FILE *f)
00191 {
00192     if ( f ) return( (off_t) lseek(f->descriptor, 0L, SEEK_CUR) );
00193     else return(-1);
00194 }
00195
00196 /*
00197     #] Utell :
00198     #[ Uflush :
00199 */
00200
00201 void Uflush(FILE *f) { DUMMYUSE(f); }
00202
00203 /*
00204     #] Uflush :
00205     #[ Ugetpos :
00206 */
00207
00208 int Ugetpos(FILE *f, fpos_t *ptr)
00209 {
00210     DUMMYUSE(f); DUMMYUSE(ptr);
00211     return(-1);
00212 }
00213
00214 /*
00215     #] Ugetpos :
00216     #[ Usetpos :
00217 */
00218
00219 int Usetpos(FILE *f, fpos_t *ptr)
00220 {
00221     DUMMYUSE(f); DUMMYUSE(ptr);
00222     return(-1);
00223 }
00224
00225 /*
00226     #] Usetpos :
00227     #[ Usetbuf :
00228 */
00229
00230 void Usetbuf(FILE *f, char *ptr) { DUMMYUSE(f); DUMMYUSE(ptr); }
00231
00232 /*
00233     #] Usetbuf :
00234 */
00235 #endif

```

4.156 variable.h File Reference

This graph shows which files directly or indirectly include this file:



Macros

- #define [chartype](#) FG.cTable
- #define [Procedures](#) ((PROCEDURE *) (AP.ProcList.lijst))
- #define [NumProcedures](#) AP.ProcList.num
- #define [MaxProcedures](#) AP.ProcList.maxnum
- #define [DoLoops](#) ((DOLOOP *) (AP.LoopList.lijst))
- #define [NumDoLoops](#) AP.LoopList.num
- #define [MaxDoLoops](#) AP.LoopList.maxnum
- #define [PreVar](#) ((PREVAR *) (AP.PreVarList.lijst))

- #define [NumPre](#) AP.PreVarList.num
- #define [MaxNumPre](#) AP.PreVarList.maxnum
- #define [SetElements](#) ((WORD *) (AC.SetElementList.lijst))
- #define [Sets](#) ((SETS) (AC.SetList.lijst))
- #define [functions](#) ((FUNCTIONS) (AC.FunctionList.lijst))
- #define [indices](#) ((INDICES) (AC.IndexList.lijst))
- #define [symbols](#) ((SYMBOLS) (AC.SymbolList.lijst))
- #define [vectors](#) ((VECTORS) (AC.VectorList.lijst))
- #define [tablebases](#) ((DBASE *) (AC.TableBaseList.lijst))
- #define [NumFunctions](#) AC.FunctionList.num
- #define [NumIndices](#) AC.IndexList.num
- #define [NumSymbols](#) AC.SymbolList.num
- #define [NumVectors](#) AC.VectorList.num
- #define [NumSets](#) AC.SetList.num
- #define [NumSetElements](#) AC.SetElementList.num
- #define [NumTableBases](#) AC.TableBaseList.num
- #define [GlobalFunctions](#) AC.FunctionList.numglobal
- #define [GlobalIndices](#) AC.IndexList.numglobal
- #define [GlobalSymbols](#) AC.SymbolList.numglobal
- #define [GlobalVectors](#) AC.VectorList.numglobal
- #define [GlobalSets](#) AC.SetList.numglobal
- #define [GlobalSetElements](#) AC.SetElementList.numglobal
- #define [cbuf](#) ((CBUF *) (AC.cbufList.lijst))
- #define [channels](#) ((CHANNEL *) (AC.ChannelList.lijst))
- #define [NumOutputChannels](#) AC.ChannelList.num
- #define [Dollars](#) ((DOLLARS) (AP.DollarList.lijst))
- #define [NumDollars](#) AP.DollarList.num
- #define [Dubious](#) ((DUBIOUSV) (AC.DubiousList.lijst))
- #define [NumDubious](#) AC.DubiousList.num
- #define [Expressions](#) ((EXPRESSIONS) (AC.ExpressionList.lijst))
- #define [NumExpressions](#) AC.ExpressionList.num
- #define [autofunctions](#) ((FUNCTIONS) (AC.AutoFunctionList.lijst))
- #define [autoindices](#) ((INDICES) (AC.AutoIndexList.lijst))
- #define [autosymbols](#) ((SYMBOLS) (AC.AutoSymbolList.lijst))
- #define [autovectors](#) ((VECTORS) (AC.AutoVectorList.lijst))
- #define [xsymbol](#) (cbuf[AM.sbufnum].rhs)
- #define [numxsymbol](#) (cbuf[AM.sbufnum].numrhs)
- #define [PotModdollars](#) ((WORD *) (AC.PotModDolList.lijst))
- #define [NumPotModdollars](#) AC.PotModDolList.num
- #define [ModOptdollars](#) ((MODOPTDOLLAR *) (AC.ModOptDolList.lijst))
- #define [NumModOptdollars](#) AC.ModOptDolList.num
- #define [BUG](#) A.bug;
- #define [AC](#) A.Cc
- #define [AM](#) A.M
- #define [AN](#) A.N
- #define [AO](#) A.O
- #define [AP](#) A.P
- #define [AR](#) A.R
- #define [AS](#) A.S
- #define [AT](#) A.T
- #define [AX](#) A.X

Variables

- WRITEBUFTOEXTCHANNEL **writeBufToExtChannel**
- GETCFROMEXTCHANNEL **getcFromExtChannel**
- SETTERMINATORFOREXTCHANNEL **setTerminatorForExternalChannel**
- SETKILLMODEFOREXTCHANNEL **setKillModeForExternalChannel**
- WRITEFILE [WriteFile](#)
- ALLGLOBALS [A](#)
- FIXEDGLOBALS [FG](#)
- FIXEDSET [fixedsets](#) []
- char * [setupfilename](#)

4.156.1 Detailed Description

Contains a number of defines to make the coding easier. Especially the defines for the use of the lists are very nice. And of course the AC for A.Cc and AT for either A.T of B->T are indispensable to keep FORM and TFORM in one set of sources.

Definition in file [variable.h](#).

4.156.2 Macro Definition Documentation

4.156.2.1 chartype

```
#define chartype FG.cTable
```

Definition at line [77](#) of file [variable.h](#).

4.156.2.2 Procedures

```
#define Procedures ((PROCEDURE *) (AP.ProcList.lijst))
```

Definition at line [79](#) of file [variable.h](#).

4.156.2.3 NumProcedures

```
#define NumProcedures AP.ProcList.num
```

Definition at line [80](#) of file [variable.h](#).

4.156.2.4 MaxProcedures

```
#define MaxProcedures AP.ProcList.maxnum
```

Definition at line [81](#) of file [variable.h](#).

4.156.2.5 DoLoops

```
#define DoLoops ((DOLOOP *) (AP.LoopList.lijst))
```

Definition at line 82 of file [variable.h](#).

4.156.2.6 NumDoLoops

```
#define NumDoLoops AP.LoopList.num
```

Definition at line 83 of file [variable.h](#).

4.156.2.7 MaxDoLoops

```
#define MaxDoLoops AP.LoopList.maxnum
```

Definition at line 84 of file [variable.h](#).

4.156.2.8 PreVar

```
#define PreVar ((PREVAR *) (AP.PreVarList.lijst))
```

Definition at line 85 of file [variable.h](#).

4.156.2.9 NumPre

```
#define NumPre AP.PreVarList.num
```

Definition at line 86 of file [variable.h](#).

4.156.2.10 MaxNumPre

```
#define MaxNumPre AP.PreVarList.maxnum
```

Definition at line 87 of file [variable.h](#).

4.156.2.11 SetElements

```
#define SetElements ((WORD *) (AC.SetElementList.lijst))
```

Definition at line 88 of file [variable.h](#).

4.156.2.12 Sets

```
#define Sets ((SETS) (AC.SetList.lijst))
```

Definition at line 89 of file [variable.h](#).

4.156.2.13 functions

```
#define functions ((FUNCTIONS) (AC.FunctionList.lijt))
```

Definition at line 90 of file [variable.h](#).

4.156.2.14 indices

```
#define indices ((INDICES) (AC.IndexList.lijt))
```

Definition at line 91 of file [variable.h](#).

4.156.2.15 symbols

```
#define symbols ((SYMBOLS) (AC.SymbolList.lijt))
```

Definition at line 92 of file [variable.h](#).

4.156.2.16 vectors

```
#define vectors ((VECTORS) (AC.VectorList.lijt))
```

Definition at line 93 of file [variable.h](#).

4.156.2.17 tablebases

```
#define tablebases ((DBASE *) (AC.TableBaseList.lijt))
```

Definition at line 94 of file [variable.h](#).

4.156.2.18 NumFunctions

```
#define NumFunctions AC.FunctionList.num
```

Definition at line 95 of file [variable.h](#).

4.156.2.19 NumIndices

```
#define NumIndices AC.IndexList.num
```

Definition at line 96 of file [variable.h](#).

4.156.2.20 NumSymbols

```
#define NumSymbols AC.SymbolList.num
```

Definition at line 97 of file [variable.h](#).

4.156.2.21 NumVectors

```
#define NumVectors AC.VectorList.num
```

Definition at line 98 of file [variable.h](#).

4.156.2.22 NumSets

```
#define NumSets AC.SetList.num
```

Definition at line 99 of file [variable.h](#).

4.156.2.23 NumSetElements

```
#define NumSetElements AC.SetElementList.num
```

Definition at line 100 of file [variable.h](#).

4.156.2.24 NumTableBases

```
#define NumTableBases AC.TableBaseList.num
```

Definition at line 101 of file [variable.h](#).

4.156.2.25 GlobalFunctions

```
#define GlobalFunctions AC.FunctionList.numglobal
```

Definition at line 102 of file [variable.h](#).

4.156.2.26 GlobalIndices

```
#define GlobalIndices AC.IndexList.numglobal
```

Definition at line 103 of file [variable.h](#).

4.156.2.27 GlobalSymbols

```
#define GlobalSymbols AC.SymbolList.numglobal
```

Definition at line 104 of file [variable.h](#).

4.156.2.28 GlobalVectors

```
#define GlobalVectors AC.VectorList.numglobal
```

Definition at line 105 of file [variable.h](#).

4.156.2.29 GlobalSets

```
#define GlobalSets AC.SetList.numglobal
```

Definition at line 106 of file [variable.h](#).

4.156.2.30 GlobalSetElements

```
#define GlobalSetElements AC.SetElementList.numglobal
```

Definition at line 107 of file [variable.h](#).

4.156.2.31 cbuf

```
#define cbuf ((CBUF *) (AC.cbufList.lijt))
```

Definition at line 108 of file [variable.h](#).

4.156.2.32 channels

```
#define channels ((CHANNEL *) (AC.ChannelList.lijt))
```

Definition at line 109 of file [variable.h](#).

4.156.2.33 NumOutputChannels

```
#define NumOutputChannels AC.ChannelList.num
```

Definition at line 110 of file [variable.h](#).

4.156.2.34 Dollars

```
#define Dollars ((DOLLARS) (AP.DollarList.lijt))
```

Definition at line 111 of file [variable.h](#).

4.156.2.35 NumDollars

```
#define NumDollars AP.DollarList.num
```

Definition at line 112 of file [variable.h](#).

4.156.2.36 Dubious

```
#define Dubious ((DUBIOUSV) (AC.DubiousList.lijt))
```

Definition at line 113 of file [variable.h](#).

4.156.2.37 NumDubious

```
#define NumDubious AC.DubiousList.num
```

Definition at line 114 of file [variable.h](#).

4.156.2.38 Expressions

```
#define Expressions ((EXPRESSIONS) (AC.ExpressionList.lijst))
```

Definition at line 115 of file [variable.h](#).

4.156.2.39 NumExpressions

```
#define NumExpressions AC.ExpressionList.num
```

Definition at line 116 of file [variable.h](#).

4.156.2.40 autofunctions

```
#define autofunctions ((FUNCTIONS) (AC.AutoFunctionList.lijst))
```

Definition at line 117 of file [variable.h](#).

4.156.2.41 autoindices

```
#define autoindices ((INDICES) (AC.AutoIndexList.lijst))
```

Definition at line 118 of file [variable.h](#).

4.156.2.42 autosymbols

```
#define autosymbols ((SYMBOLS) (AC.AutoSymbolList.lijst))
```

Definition at line 119 of file [variable.h](#).

4.156.2.43 autovectors

```
#define autovectors ((VECTORS) (AC.AutoVectorList.lijst))
```

Definition at line 120 of file [variable.h](#).

4.156.2.44 xsymbol

```
#define xsymbol (cbuf[AM.sbufnum].rhs)
```

Definition at line 121 of file [variable.h](#).

4.156.2.45 numxsymbol

```
#define numxsymbol (cbuf[AM.sbufnum].numrhs)
```

Definition at line 122 of file [variable.h](#).

4.156.2.46 PotModdollars

```
#define PotModdollars ((WORD *) (AC.PotModDolList.lijst))
```

Definition at line 124 of file [variable.h](#).

4.156.2.47 NumPotModdollars

```
#define NumPotModdollars AC.PotModDolList.num
```

Definition at line 125 of file [variable.h](#).

4.156.2.48 ModOptdollars

```
#define ModOptdollars ((MODOPTDOLLAR *) (AC.ModOptDolList.lijst))
```

Definition at line 126 of file [variable.h](#).

4.156.2.49 NumModOptdollars

```
#define NumModOptdollars AC.ModOptDolList.num
```

Definition at line 127 of file [variable.h](#).

4.156.2.50 BUG

```
#define BUG A.bug;
```

Definition at line 129 of file [variable.h](#).

4.156.2.51 AC

```
#define AC A.Cc
```

Definition at line 145 of file [variable.h](#).

4.156.2.52 AM

```
#define AM A.M
```

Definition at line 146 of file [variable.h](#).

4.156.2.53 AN

```
#define AN A.N
```

Definition at line 147 of file [variable.h](#).

4.156.2.54 AO

```
#define AO A.O
```

Definition at line 148 of file [variable.h](#).

4.156.2.55 AP

```
#define AP A.P
```

Definition at line 149 of file [variable.h](#).

4.156.2.56 AR

```
#define AR A.R
```

Definition at line 150 of file [variable.h](#).

4.156.2.57 AS

```
#define AS A.S
```

Definition at line 151 of file [variable.h](#).

4.156.2.58 AT

```
#define AT A.T
```

Definition at line 152 of file [variable.h](#).

4.156.2.59 AX

```
#define AX A.X
```

Definition at line 153 of file [variable.h](#).

4.156.3 Variable Documentation

4.156.3.1 WriteFile

WRITEFILE WriteFile [extern]

Definition at line 1357 of file [tools.c](#).

4.156.3.2 A

ALLGLOBALS A [extern]

Definition at line 133 of file [inivar.h](#).

4.156.3.3 FG

FIXEDGLOBALS FG [extern]

Definition at line 34 of file [inivar.h](#).

4.156.3.4 fixedsets

FIXEDSET fixedsets[] [extern]

Definition at line 258 of file [inivar.h](#).

4.156.3.5 setupfilename

char* setupfilename [extern]

Definition at line 276 of file [inivar.h](#).

4.157 variable.h

[Go to the documentation of this file.](#)

```

00001 #ifndef __VARIABLE__
00002
00003 #define __VARIABLE__
00004
00013 /* #[ License : */
00014 /*
00015  * Copyright (C) 1984-2023 J.A.M. Vermaseren
00016  * When using this file you are requested to refer to the publication
00017  * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00018  * This is considered a matter of courtesy as the development was paid
00019  * for by FOM the Dutch physics granting agency and we would like to
00020  * be able to track its scientific use to convince FOM of its value
00021  * for the community.
00022  *
00023  * This file is part of FORM.
00024  *
00025  * FORM is free software: you can redistribute it and/or modify it under the
00026  * terms of the GNU General Public License as published by the Free Software
00027  * Foundation, either version 3 of the License, or (at your option) any later
00028  * version.
00029  *
00030  * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00031  * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00032  * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00033  * details.
00034  *
00035  * You should have received a copy of the GNU General Public License along
00036  * with FORM. If not, see <http://www.gnu.org/licenses/>.
00037  */
00038 /* #] License : */
00039
00040 /*See the file extcmd.c*/
00041
00042 #ifdef REMOVEDBY_MT
00043 extern int (*writeBufToExtChannel)(char *buf, size_t n);
00044 extern int (*getcFromExtChannel)();
00045 extern int (*setTerminatorForExternalChannel)(char *newterminator);
00046 #endif
00047 extern WRITEBUFTOEXTCHANNEL writeBufToExtChannel;
00048 extern GETCFROMEXTCHANNEL getcFromExtChannel;
00049 extern SETTERMINATORFOREXTCHANNEL setTerminatorForExternalChannel;
00050 extern SETKILLMODEFOREXTCHANNEL setKillModeForExternalChannel;
00051
00052 /*
00053 extern LONG (*WriteFile)(int handle, UBYTE *buffer, LONG number);
00054 */
00055 extern WRITEFILE WriteFile;
00056 /*:[17nov2005 mt]*/
00057
00058 extern ALLGLOBALS A;
00059 #ifdef WITHPTHREADS
00060 extern ALLPRIVATES **AB;
00061 #endif
00062
00063 extern FIXEDGLOBALS FG;
00064 extern FIXEDSET fixedsets[];
00065
00066 extern char *setupfilename;
00067
00068 EXTERNLOCK(ErrorMessageLock)
00069 EXTERNLOCK(ErrorReadLock)
00070 EXTERNLOCK(dummylock)
00071
00072 #ifdef VMS
00073 #include <stdio.h>
00074 extern FILE **FileStructs;
00075 #endif
00076
00077 #define chartype FG.cTable
00078
00079 #define Procedures ((PROCEDURE *) (AP.ProcList.lijst))
00080 #define NumProcedures AP.ProcList.num
00081 #define MaxProcedures AP.ProcList.maxnum
00082 #define DoLoops ((DOLOOP *) (AP.LoopList.lijst))
00083 #define NumDoLoops AP.LoopList.num
00084 #define MaxDoLoops AP.LoopList.maxnum
00085 #define PreVar ((PREVAR *) (AP.PreVarList.lijst))
00086 #define NumPre AP.PreVarList.num
00087 #define MaxNumPre AP.PreVarList.maxnum
00088 #define SetElements ((WORD *) (AC.SetElementList.lijst))
00089 #define Sets ((SETS) (AC.SetList.lijst))
00090 #define functions ((FUNCTIONS) (AC.FunctionList.lijst))

```

```

00091 #define indices ((INDICES) (AC.IndexList.lijst))
00092 #define symbols ((SYMBOLS) (AC.SymbolList.lijst))
00093 #define vectors ((VECTORS) (AC.VectorList.lijst))
00094 #define tablebases ((DBASE *) (AC.TableBaseList.lijst))
00095 #define NumFunctions AC.FunctionList.num
00096 #define NumIndices AC.IndexList.num
00097 #define NumSymbols AC.SymbolList.num
00098 #define NumVectors AC.VectorList.num
00099 #define NumSets AC.SetList.num
00100 #define NumSetElements AC.SetElementList.num
00101 #define NumTableBases AC.TableBaseList.num
00102 #define GlobalFunctions AC.FunctionList.numglobal
00103 #define GlobalIndices AC.IndexList.numglobal
00104 #define GlobalSymbols AC.SymbolList.numglobal
00105 #define GlobalVectors AC.VectorList.numglobal
00106 #define GlobalSets AC.SetList.numglobal
00107 #define GlobalSetElements AC.SetElementList.numglobal
00108 #define cbuf ((CBUF *) (AC.cbufList.lijst))
00109 #define channels ((CHANNEL *) (AC.ChannelList.lijst))
00110 #define NumOutputChannels AC.ChannelList.num
00111 #define Dollars ((DOLLARS) (AP.DollarList.lijst))
00112 #define NumDollars AP.DollarList.num
00113 #define Dubious ((DUBIOUSV) (AC.DubiousList.lijst))
00114 #define NumDubious AC.DubiousList.num
00115 #define Expressions ((EXPRESSIONS) (AC.ExpressionList.lijst))
00116 #define NumExpressions AC.ExpressionList.num
00117 #define autofunctions ((FUNCTIONS) (AC.AutoFunctionList.lijst))
00118 #define autoindices ((INDICES) (AC.AutoIndexList.lijst))
00119 #define autosymbols ((SYMBOLS) (AC.AutoSymbolList.lijst))
00120 #define autovectors ((VECTORS) (AC.AutoVectorList.lijst))
00121 #define xsymbol (cbuf[AM.sbufnum].rhs)
00122 #define numxsymbol (cbuf[AM.sbufnum].numrhs)
00123
00124 #define PotModdollars ((WORD *) (AC.PotModDolList.lijst))
00125 #define NumPotModdollars AC.PotModDolList.num
00126 #define ModOptdollars ((MODOPTDOLLAR *) (AC.ModOptDolList.lijst))
00127 #define NumModOptdollars AC.ModOptDolList.num
00128
00129 #define BUG A.bug;
00130
00131 #ifndef WITHPTHREADS
00132 #define AC A.Cc
00133 #define AM A.M
00134 #define AO A.O
00135 #define AP A.P
00136 #define AS A.S
00137 #define AX A.X
00138 #define AN B->N
00139 #define AR B->R
00140 #define AT B->T
00141 #define AN0 B0->N
00142 #define AR0 B0->R
00143 #define AT0 B0->T
00144 #else
00145 #define AC A.Cc
00146 #define AM A.M
00147 #define AN A.N
00148 #define AO A.O
00149 #define AP A.P
00150 #define AR A.R
00151 #define AS A.S
00152 #define AT A.T
00153 #define AX A.X
00154 #endif
00155
00156 #endif

```

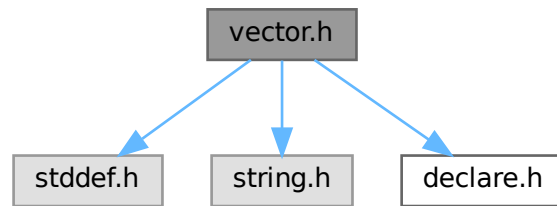
4.158 vector.h File Reference

```

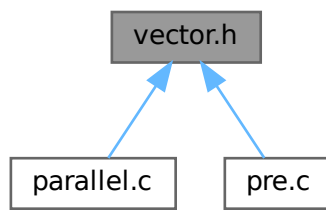
#include <stddef.h>
#include <string.h>
#include "declare.h"

```


Include dependency graph for vector.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define [VectorStruct](#)(T)
- #define [Vector](#)(T, X) [VectorStruct](#)(T) X = { NULL, 0, 0 }
- #define [DeclareVector](#)(T, X) [VectorStruct](#)(T) X
- #define [VectorInit](#)(X)
- #define [VectorFree](#)(X)
- #define [VectorPtr](#)(X) ((X).ptr)
- #define [VectorFront](#)(X) ((X).ptr[0])
- #define [VectorBack](#)(X) ((X).ptr[(X).size - 1])
- #define [VectorSize](#)(X) ((X).size)
- #define [VectorCapacity](#)(X) ((X).capacity)
- #define [VectorEmpty](#)(X) ((X).size == 0)
- #define [VectorClear](#)(X) do { (X).size = 0; } while (0)
- #define [VectorReserve](#)(X, newcapacity)
- #define [VectorPushBack](#)(X, x)
- #define [VectorPushBacks](#)(X, src, n)
- #define [VectorPopBack](#)(X) do { (X).size --; } while (0)
- #define [VectorInsert](#)(X, index, x)
- #define [VectorInserts](#)(X, index, src, n)
- #define [VectorErase](#)(X, index)
- #define [VectorErases](#)(X, index, n)

4.158.1 Detailed Description

An implementation of dynamic array.

Example:

```
size_t i;
Vector(int, vec);
VectorPushBack(vec, 1);
VectorPushBack(vec, 2);
VectorPushBack(vec, 3);
for ( i = 0; i < VectorSize(vec); i++ )
    printf("%d\n", VectorPtr(vec)[i]);
VectorFree(vec);
```

Definition in file [vector.h](#).

4.158.2 Macro Definition Documentation

4.158.2.1 VectorStruct

```
#define VectorStruct(
    T )
```

Value:

```
struct { \
    T *ptr; \
    size_t size; \
    size_t capacity; \
}
```

A struct for vector objects.

Parameters

<i>T</i>	the type of elements.
----------	-----------------------

Definition at line 65 of file [vector.h](#).

4.158.2.2 Vector

```
#define Vector(
    T,
    X ) VectorStruct(T) X = { NULL, 0, 0 }
```

Defines and initialises a vector X of the type T. The user must call [VectorFree\(X\)](#) after the use of X.

Parameters

<i>T</i>	the type of elements.
<i>X</i>	the name of vector object

Definition at line 84 of file [vector.h](#).

4.158.2.3 DeclareVector

```
#define DeclareVector(  
    T,  
    X )  VectorStruct (T) X
```

Declares a vector X of the type T. The user must call [VectorInit\(X\)](#) before the use of X.

Parameters

<i>T</i>	the type of elements.
<i>X</i>	the name of vector object

Definition at line 99 of file [vector.h](#).

4.158.2.4 VectorInit

```
#define VectorInit(  
    X )
```

Value:

```
do { \  
    (X).ptr = NULL; \  
    (X).size = 0; \  
    (X).capacity = 0; \  
} while (0)
```

Initialises a vector X of the type T. The user must call [VectorFree\(X\)](#) after the use of X.

Parameters

<i>X</i>	the vector object.
----------	--------------------

Definition at line 113 of file [vector.h](#).

4.158.2.5 VectorFree

```
#define VectorFree(  
    X )
```

Value:

```
do { \  
    M_free((X).ptr, "VectorFree:" #X); \  
    (X).ptr = NULL; \  
    (X).size = 0; \  
    (X).capacity = 0; \  
} while (0)
```

Frees the buffer allocated by the vector X.

Parameters

<i>X</i>	the vector object.
----------	--------------------

Definition at line 130 of file [vector.h](#).

4.158.2.6 VectorPtr

```
#define VectorPtr(  
    X )    ((X).ptr)
```

Returns the pointer to the buffer for the vector X. NULL when [VectorCapacity\(X\)](#) == 0.

Parameters

X	the vector object.
---	--------------------

Returns

the pointer to the allocated buffer for the vector.

Definition at line 150 of file [vector.h](#).

4.158.2.7 VectorFront

```
#define VectorFront(  
    X )    ((X).ptr[0])
```

Returns the first element of the vector X. Undefined when [VectorSize\(X\)](#) == 0.

Parameters

X	the vector object.
---	--------------------

Returns

the first element of the vector.

Definition at line 165 of file [vector.h](#).

4.158.2.8 VectorBack

```
#define VectorBack(  
    X )    ((X).ptr[(X).size - 1])
```

Returns the last element of the vector X. Undefined when [VectorSize\(X\)](#) == 0.

Parameters

X	the vector object.
---	--------------------

Returns

the last element of the vector.

Definition at line 180 of file [vector.h](#).

4.158.2.9 VectorSize

```
#define VectorSize(  
    X )    ((X).size)
```

Returns the size of the vector X.

Parameters

<i>X</i>	the vector object.
----------	--------------------

Returns

the size of the vector.

Definition at line 194 of file [vector.h](#).

4.158.2.10 VectorCapacity

```
#define VectorCapacity(  
    X )    ((X).capacity)
```

Returns the capacity (the number of the already allocated elements) of the vector X.

Parameters

<i>X</i>	the vector object.
----------	--------------------

Returns

the capacity of the vector.

Definition at line 208 of file [vector.h](#).

4.158.2.11 VectorEmpty

```
#define VectorEmpty(  
    X )    ((X).size == 0)
```

Returns true the size of the vector X is zero.

Parameters

<i>X</i>	the vector object.
----------	--------------------

Returns

true if the vector has no elements, false otherwise.

Definition at line 222 of file [vector.h](#).

4.158.2.12 VectorClear

```
#define VectorClear(  
    X )    do { (X).size = 0; } while (0)
```

Sets the size of the vector *X* to zero.

Parameters

<i>X</i>	the vector object.
----------	--------------------

Definition at line 235 of file [vector.h](#).

4.158.2.13 VectorReserve

```
#define VectorReserve(  
    X,  
    newcapacity )
```

Value:

```
do { \
    size_t v_tmp_newcapacity_ = (newcapacity); \
    if ( (X).capacity < v_tmp_newcapacity_ ) { \
        void *v_tmp_newptr_; \
        v_tmp_newcapacity_ = (v_tmp_newcapacity_ * 3) / 2; \
        if ( v_tmp_newcapacity_ < 4 ) v_tmp_newcapacity_ = 4; \
        v_tmp_newptr_ = Malloc1(sizeof((X).ptr[0]) * v_tmp_newcapacity_, "VectorReserve:" #X); \
        if ( (X).ptr != NULL ) { \
            memcpy(v_tmp_newptr_, (X).ptr, (X).size * sizeof((X).ptr[0])); \
            M_free((X).ptr, "VectorReserve:" #X); \
        } \
        (X).ptr = v_tmp_newptr_; \
        (X).capacity = v_tmp_newcapacity_; \
    } \
} while (0)
```

Requires that the capacity of the vector *X* is equal to or larger than *newcapacity*.

Parameters

<i>X</i>	the vector object.
<i>newcapacity</i>	the capacity to be reserved.

Definition at line 249 of file [vector.h](#).

4.158.2.14 VectorPushBack

```
#define VectorPushBack(  
    X,  
    x )
```

Value:

```
do { \
    VectorReserve((X), (X).size + 1); \
    (X).ptr[(X).size++] = (x); \
} while (0)
```

Adds an element *x* at the end of the vector *X*.

Parameters

<i>X</i>	the vector object.
<i>x</i>	the element to be added.

Definition at line 277 of file [vector.h](#).

4.158.2.15 VectorPushBacks

```
#define VectorPushBacks(  
    X,  
    src,  
    n )
```

Value:

```
do { \
    size_t v_tmp_n_ = (n); \
    VectorReserve((X), (X).size + v_tmp_n_); \
    memcpy((X).ptr + (X).size, (src), v_tmp_n_ * sizeof((X).ptr[0])); \
    (X).size += v_tmp_n_; \
} while (0)
```

Adds an *n* elements of *src* at the end of the vector *X*.

Parameters

<i>X</i>	the vector object.
<i>src</i>	the starting address of the buffer storing elements to be added.
<i>n</i>	the number of elements to be added.

Definition at line 295 of file [vector.h](#).

4.158.2.16 VectorPopBack

```
#define VectorPopBack(  
    X ) do { (X).size --; } while (0)
```

Removes the last element of the vector *X*. [VectorSize\(X\)](#) must be > 0 .

Parameters

<i>X</i>	the vector object.
----------	--------------------

Definition at line 314 of file [vector.h](#).

4.158.2.17 VectorInsert

```
#define VectorInsert(
    X,
    index,
    x )
```

Value:

```
do { \
    size_t v_tmp_index_ = (index); \
    VectorReserve((X), (X).size + 1); \
    memmove((X).ptr + v_tmp_index_ + 1, (X).ptr + v_tmp_index_, ((X).size - v_tmp_index_) *
sizeof((X).ptr[0])); \
    (X).ptr[v_tmp_index_] = (x); \
    (X).size++; \
} while (0)
```

Inserts an element *x* at the specified index of the vector *X*. The index must be $0 \leq \text{index} < \text{VectorSize}(X)$.

Parameters

<i>X</i>	the vector object.
<i>index</i>	the position at which the element will be inserted.
<i>x</i>	the element to be inserted.

Definition at line 330 of file [vector.h](#).

4.158.2.18 VectorInserts

```
#define VectorInserts(
    X,
    index,
    src,
    n )
```

Value:

```
do { \
    size_t v_tmp_index_ = (index), v_tmp_n_ = (n); \
    VectorReserve((X), (X).size + v_tmp_n_); \
    memmove((X).ptr + v_tmp_index_ + v_tmp_n_, (X).ptr + v_tmp_index_, ((X).size - v_tmp_index_) *
sizeof((X).ptr[0])); \
    memcpy((X).ptr + v_tmp_index_, (src), v_tmp_n_ * sizeof((X).ptr[0])); \
    (X).size += v_tmp_n_; \
} while (0)
```

Inserts an *n* elements of *src* at the specified index of the vector *X*. The index must be $0 \leq \text{index} < \text{VectorSize}(X)$.

Parameters

<i>X</i>	the vector object.
<i>index</i>	the position at which the elements will be inserted.
<i>src</i>	the starting address of the buffer storing elements to be inserted.
<i>n</i>	the number of elements to be inserted.

Definition at line 353 of file [vector.h](#).

4.158.2.19 VectorErase

```
#define VectorErase(  
    X,  
    index )
```

Value:

```
do { \
    size_t v_tmp_index_ = (index); \
    memmove((X).ptr + v_tmp_index_, (X).ptr + v_tmp_index_ + 1, ((X).size - v_tmp_index_ - 1) * \
    sizeof((X).ptr[0])); \
    (X).size--; \
} while (0)
```

Removes an element at the specified index of the vector X. The index must be $0 \leq \text{index} < \text{VectorSize}(X)$.

Parameters

<i>X</i>	the vector object.
<i>index</i>	the position of the element to be removed.

Definition at line 374 of file [vector.h](#).

4.158.2.20 VectorErases

```
#define VectorErases(  
    X,  
    index,  
    n )
```

Value:

```
do { \
    size_t v_tmp_index_ = (index), v_tmp_n_ = (n); \
    memmove((X).ptr + v_tmp_index_, (X).ptr + v_tmp_index_ + v_tmp_n_, ((X).size - v_tmp_index_ - 1) * \
    sizeof((X).ptr[0])); \
    (X).size -= v_tmp_n_; \
} while (0)
```

Removes an n elements at the specified index of the vector X. The index must be $0 \leq \text{index} < \text{VectorSize}(X) - n + 1$.

Parameters

<i>X</i>	the vector object.
<i>index</i>	the starting position of the elements to be removed.
<i>n</i>	the number of elements to be removed.

Definition at line 394 of file [vector.h](#).

4.159 vector.h

[Go to the documentation of this file.](#)

```

00001 #ifndef VECTOR_H_
00002 #define VECTOR_H_
00003
00020 /* # License : */
00021 /*
00022 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00023 * When using this file you are requested to refer to the publication
00024 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00025 * This is considered a matter of courtesy as the development was paid
00026 * for by FOM the Dutch physics granting agency and we would like to
00027 * be able to track its scientific use to convince FOM of its value
00028 * for the community.
00029 *
00030 * This file is part of FORM.
00031 *
00032 * FORM is free software: you can redistribute it and/or modify it under the
00033 * terms of the GNU General Public License as published by the Free Software
00034 * Foundation, either version 3 of the License, or (at your option) any later
00035 * version.
00036 *
00037 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00038 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00039 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00040 * details.
00041 *
00042 * You should have received a copy of the GNU General Public License along
00043 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00044 */
00045 /* # License : */
00046 /*
00047 # Includes :
00048 */
00049
00050 #include <stddef.h>
00051 #include <string.h>
00052 #include "declare.h"
00053
00054 /*
00055 # Includes :
00056 # Vector :
00057 # VectorStruct :
00058 */
00059
00065 #define VectorStruct(T) \
00066     struct { \
00067         T *ptr; \
00068         size_t size; \
00069         size_t capacity; \
00070     }
00071
00072 /*
00073 # VectorStruct :
00074 # Vector :
00075 */
00076
00084 #define Vector(T, X) \
00085     VectorStruct(T) X = { NULL, 0, 0 }
00086
00087 /*
00088 # Vector :
00089 # DeclareVector :
00090 */
00091
00099 #define DeclareVector(T, X) \
00100     VectorStruct(T) X
00101
00102 /*
00103 # DeclareVector :
00104 # VectorInit :
00105 */
00106
00113 #define VectorInit(X) \
00114     do { \
00115         (X).ptr = NULL; \
00116         (X).size = 0; \
00117         (X).capacity = 0; \
00118     } while (0)
00119
00120 /*
00121 # VectorInit :
00122 # VectorFree :
00123 */
00124
00130 #define VectorFree(X) \
00131     do { \
00132         M_free((X).ptr, "VectorFree:" #X); \
00133         (X).ptr = NULL; \

```

```

00134         (X).size = 0; \
00135         (X).capacity = 0; \
00136     } while (0)
00137
00138     /*
00139         #] VectorFree :
00140         #[ VectorPtr :
00141     */
00142
00150 #define VectorPtr(X) \
00151     ((X).ptr)
00152
00153     /*
00154         #] VectorPtr :
00155         #[ VectorFront :
00156     */
00157
00165 #define VectorFront(X) \
00166     ((X).ptr[0])
00167
00168     /*
00169         #] VectorFront :
00170         #[ VectorBack :
00171     */
00172
00180 #define VectorBack(X) \
00181     ((X).ptr[(X).size - 1])
00182
00183     /*
00184         #] VectorBack :
00185         #[ VectorSize :
00186     */
00187
00194 #define VectorSize(X) \
00195     ((X).size)
00196
00197     /*
00198         #] VectorSize :
00199         #[ VectorCapacity :
00200     */
00201
00208 #define VectorCapacity(X) \
00209     ((X).capacity)
00210
00211     /*
00212         #] VectorCapacity :
00213         #[ VectorEmpty :
00214     */
00215
00222 #define VectorEmpty(X) \
00223     ((X).size == 0)
00224
00225     /*
00226         #] VectorEmpty :
00227         #[ VectorClear :
00228     */
00229
00235 #define VectorClear(X) \
00236     do { (X).size = 0; } while (0)
00237
00238     /*
00239         #] VectorClear :
00240         #[ VectorReserve :
00241     */
00242
00249 #define VectorReserve(X, newcapacity) \
00250     do { \
00251         size_t v_tmp_newcapacity_ = (newcapacity); \
00252         if ( (X).capacity < v_tmp_newcapacity_ ) { \
00253             void *v_tmp_newptr_; \
00254             v_tmp_newcapacity_ = (v_tmp_newcapacity_ * 3) / 2; \
00255             if ( v_tmp_newcapacity_ < 4 ) v_tmp_newcapacity_ = 4; \
00256             v_tmp_newptr_ = Malloc1(sizeof((X).ptr[0]) * v_tmp_newcapacity_, "VectorReserve:" #X); \
00257             if ( (X).ptr != NULL ) { \
00258                 memcpy(v_tmp_newptr_, (X).ptr, (X).size * sizeof((X).ptr[0])); \
00259                 M_free((X).ptr, "VectorReserve:" #X); \
00260             } \
00261             (X).ptr = v_tmp_newptr_; \
00262             (X).capacity = v_tmp_newcapacity_; \
00263         } \
00264     } while (0)
00265
00266     /*
00267         #] VectorReserve :
00268         #[ VectorPushBack :
00269     */
00270

```

```

00277 #define VectorPushBack(X, x) \
00278     do { \
00279         VectorReserve((X), (X).size + 1); \
00280         (X).ptr[(X).size++] = (x); \
00281     } while (0)
00282
00283 /*
00284     #] VectorPushBack :
00285     #[ VectorPushBacks :
00286 */
00287
00295 #define VectorPushBacks(X, src, n) \
00296     do { \
00297         size_t v_tmp_n_ = (n); \
00298         VectorReserve((X), (X).size + v_tmp_n_); \
00299         memcpy((X).ptr + (X).size, (src), v_tmp_n_ * sizeof((X).ptr[0])); \
00300         (X).size += v_tmp_n_; \
00301     } while (0)
00302
00303 /*
00304     #] VectorPushBacks :
00305     #[ VectorPopBack :
00306 */
00307
00314 #define VectorPopBack(X) \
00315     do { (X).size --; } while (0)
00316
00317 /*
00318     #] VectorPopBack :
00319     #[ VectorInsert :
00320 */
00321
00330 #define VectorInsert(X, index, x) \
00331     do { \
00332         size_t v_tmp_index_ = (index); \
00333         VectorReserve((X), (X).size + 1); \
00334         memmove((X).ptr + v_tmp_index_ + 1, (X).ptr + v_tmp_index_, ((X).size - v_tmp_index_) *
sizeof((X).ptr[0])); \
00335         (X).ptr[v_tmp_index_] = (x); \
00336         (X).size++; \
00337     } while (0)
00338
00339 /*
00340     #] VectorInsert :
00341     #[ VectorInserts :
00342 */
00343
00353 #define VectorInserts(X, index, src, n) \
00354     do { \
00355         size_t v_tmp_index_ = (index), v_tmp_n_ = (n); \
00356         VectorReserve((X), (X).size + v_tmp_n_); \
00357         memmove((X).ptr + v_tmp_index_ + v_tmp_n_, (X).ptr + v_tmp_index_, ((X).size - v_tmp_index_) *
sizeof((X).ptr[0])); \
00358         memcpy((X).ptr + v_tmp_index_, (src), v_tmp_n_ * sizeof((X).ptr[0])); \
00359         (X).size += v_tmp_n_; \
00360     } while (0)
00361
00362 /*
00363     #] VectorInserts :
00364     #[ VectorErase :
00365 */
00366
00374 #define VectorErase(X, index) \
00375     do { \
00376         size_t v_tmp_index_ = (index); \
00377         memmove((X).ptr + v_tmp_index_, (X).ptr + v_tmp_index_ + 1, ((X).size - v_tmp_index_ - 1) *
sizeof((X).ptr[0])); \
00378         (X).size--; \
00379     } while (0)
00380
00381 /*
00382     #] VectorErase :
00383     #[ VectorErases :
00384 */
00385
00394 #define VectorErases(X, index, n) \
00395     do { \
00396         size_t v_tmp_index_ = (index), v_tmp_n_ = (n); \
00397         memmove((X).ptr + v_tmp_index_, (X).ptr + v_tmp_index_ + v_tmp_n_, ((X).size - v_tmp_index_ -
1) * sizeof((X).ptr[0])); \
00398         (X).size -= v_tmp_n_; \
00399     } while (0)
00400
00401 /*
00402     #] VectorErases :
00403     #[ Vector :
00404 */

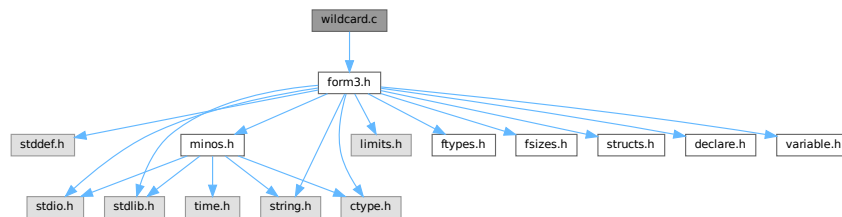
```

```
00405 #endif /* VECTOR_H_ */
```

4.160 wildcard.c File Reference

```
#include "form3.h"
```

Include dependency graph for wildcard.c:



Macros

- `#define` [DEBUG\(x\)](#)

Functions

- WORD [WildFill](#) (PHEAD WORD *to, WORD *from, WORD *sub)
- WORD [ResolveSet](#) (PHEAD WORD *from, WORD *to, WORD *subs)
- void [ClearWild](#) (PHEAD0)
- WORD [AddWild](#) (PHEAD WORD oldnumber, WORD type, WORD newnumber)
- WORD [CheckWild](#) (PHEAD WORD oldnumber, WORD type, WORD newnumber, WORD *newval)
- int [DenToFunction](#) (WORD *term, WORD numfun)

4.160.1 Detailed Description

Contains the functions that deal with the wildcards. During the pattern matching there are two steps: 1: check that a wildcard substitution is correct (if there was already an assignment for this variable, it is the same; it is part of the proper set; it is the proper type of variables, etc.) 2: make the assignment In addition we have to be able to clear assignments. During execution we have to make the actual replacements (WildFill)

Definition in file [wildcard.c](#).

4.160.2 Macro Definition Documentation

4.160.2.1 DEBUG

```
#define DEBUG(
    x )
```

Definition at line 44 of file [wildcard.c](#).

4.160.3 Function Documentation

4.160.3.1 WildFill()

```
WORD WildFill (
    PHEAD WORD * to,
    WORD * from,
    WORD * sub )
```

Definition at line 65 of file [wildcard.c](#).

4.160.3.2 ResolveSet()

```
WORD ResolveSet (
    PHEAD WORD * from,
    WORD * to,
    WORD * subs )
```

Definition at line 1361 of file [wildcard.c](#).

4.160.3.3 ClearWild()

```
void ClearWild (
    PHEAD0 )
```

Definition at line 1518 of file [wildcard.c](#).

4.160.3.4 AddWild()

```
WORD AddWild (
    PHEAD WORD oldnumber,
    WORD type,
    WORD newnumber )
```

Definition at line 1544 of file [wildcard.c](#).

4.160.3.5 CheckWild()

```
WORD CheckWild (
    PHEAD WORD oldnumber,
    WORD type,
    WORD newnumber,
    WORD * newval )
```

Definition at line 1791 of file [wildcard.c](#).

4.160.3.6 DenToFunction()

```
int DenToFunction (
    WORD * term,
    WORD numFun )
```

Definition at line 2556 of file [wildcard.c](#).

4.161 wildcard.c

[Go to the documentation of this file.](#)

```
00001
00012 /* #[ License : */
00013 /*
00014 * Copyright (C) 1984-2023 J.A.M. Vermaseren
00015 * When using this file you are requested to refer to the publication
00016 * J.A.M.Vermaseren "New features of FORM" math-ph/0010025
00017 * This is considered a matter of courtesy as the development was paid
00018 * for by FOM the Dutch physics granting agency and we would like to
00019 * be able to track its scientific use to convince FOM of its value
00020 * for the community.
00021 *
00022 * This file is part of FORM.
00023 *
00024 * FORM is free software: you can redistribute it and/or modify it under the
00025 * terms of the GNU General Public License as published by the Free Software
00026 * Foundation, either version 3 of the License, or (at your option) any later
00027 * version.
00028 *
00029 * FORM is distributed in the hope that it will be useful, but WITHOUT ANY
00030 * WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS
00031 * FOR A PARTICULAR PURPOSE. See the GNU General Public License for more
00032 * details.
00033 *
00034 * You should have received a copy of the GNU General Public License along
00035 * with FORM. If not, see <http://www.gnu.org/licenses/>.
00036 */
00037 /* #] License : */
00038 /*
00039 #[ Includes : wildcard.c
00040 */
00041
00042 #include "form3.h"
00043
00044 #define DEBUG(x)
00045
00046 /*
00047 #define DEBUG(x) x
00048
00049 #] Includes :
00050 #[ Wildcards :
00051 #[ WildFill : WORD WildFill(to,from,sub)
00052
00053 Takes the term in from and puts it into to while
00054 making wildcard substitutions.
00055 The return value is the number of words put in to.
00056 The length as the first word of from is not copied.
00057
00058 There are two possible algorithms:
00059 1: For each element in `from': scan sub.
00060 2: For each wildcard in sub replace elements in term.
00061 The original algorithm used 1:
00062
00063 */
00064
00065 WORD WildFill(PHEAD WORD *to, WORD *from, WORD *sub)
00066 {
00067 GETBIDENTITY
00068 WORD i, j, *s, *t, *m, len, dflag, odirt, adirt;
00069 WORD *r, *u, *v, *w, *z, *zst, *zz, *subs, *accu, na, dirty = 0, *tstop;
00070 WORD *temp = 0, *uu, *oldcpointer, sgn;
00071 WORD subcount, setflag, *setlist = 0, si;
00072 accu = oldcpointer = AR.CompressPointer;
00073 t = sub;
00074 t += sub[1];
00075 s = sub + SUBEXPSIZE;
00076 i = 0;
```

```

00077     while ( s < t && *s != FROMBRAC ) {
00078         i++; s += s[1];
00079     }
00080     if ( !i ) {          /* No wildcards -> done quickly */
00081         j = i = *from;
00082         NCOPY(to,from,i);
00083         if ( dirty ) AN.WildDirt = dirty;
00084         return(j);
00085     }
00086     sgn = 0;
00087     subs = sub + SUBEXPSIZE;
00088     t = from;
00089     GETSTOP(t,r);
00090     t++;
00091     m = to + 1;
00092     if ( t < r ) do {
00093         uu = u = t + t[1];
00094         setflag = 0;
00095     ReSwitch:
00096         switch ( *t ) {
00097             case SYMBOL:
00098             /*
00099                 #[ SYMBOLS :
00100             */
00101                 z = accu;
00102                 *m++ = *t++;
00103                 *m++ = *t++;
00104                 v = m;
00105                 while ( t < u ) {
00106                     *m = *t;
00107                     for ( si = 0; si < setflag; si += 2 ) {
00108                         if ( t == temp + setlist[si] ) goto sspow;
00109                     }
00110                     s = subs;
00111                     for ( j = 0; j < i; j++ ) {
00112                         if ( *t == s[2] ) {
00113                             if ( *s == SYMTOSYM ) {
00114                                 *m = s[3]; dirty = 1;
00115                                 break;
00116                             }
00117                             else if ( *s == SYMTONUM ) {
00118                                 dirty = 1;
00119                                 zst = z;
00120                                 *z++ = SNUMBER;
00121                                 *z++ = 4;
00122                                 *z++ = s[3];
00123                                 w = z;
00124                                 *z++ = *t;
00125                                 if ( ABS(*t) >= 2*MAXPOWER ) {
00126     DoPow:
00127                                     s = subs;
00128                                     for ( j = 0; j < i; j++ ) {
00129                                         if ( ( *s == SYMTONUM ) &&
00130                                             ( ABS(*t) - 2*MAXPOWER ) == s[2] ) {
00131                                             dirty = 1;
00132                                             *w = s[3];
00133                                             if ( *t < 0 ) *w = -*w;
00134                                             break;
00135                                         }
00136                                         if ( ( *s == SYMTOSYM ) &&
00137                                             ( ABS(*t) - 2*MAXPOWER ) == s[2] ) {
00138                                             dirty = 1;
00139                                             zz = z;
00140                                             while ( --zz >= zst ) {
00141                                                 zz[1+FUNHEAD+ARGHEAD] = *zz;
00142                                             }
00143                                             w += 1+FUNHEAD+ARGHEAD;
00144                                             *zst = EXPONENT;
00145                                             zst[2] = DIRTYFLAG;
00146                                             zst[FUNHEAD+ARGHEAD] = WORDDIF(z,zst)+4;
00147                                             zst[1+FUNHEAD] = 1;
00148                                             zst[FUNHEAD] = WORDDIF(z,zst)+4+ARGHEAD;
00149                                             z += FUNHEAD+ARGHEAD+1;
00150                                             *w = 1; /* exponent -> 1 */
00151                                             *z++ = 1;
00152                                             *z++ = 1;
00153                                             *z++ = 3;
00154                                             if ( *t > 0 ) {
00155                                                 *z++ = -SYMBOL;
00156                                                 *z++ = s[3];
00157                                             }
00158                                             else {
00159                                                 *z++ = ARGHEAD+8;
00160                                                 *z++ = 1;
00161                                                 *z++ = 8;
00162                                                 *z++ = SYMBOL;
00163                                                 *z++ = 4;
00164                                                 *z++ = s[3];

```



```

00164             *z++ = 1;
00165             *z++ = 1;
00166             *z++ = 1;
00167             *z++ = -3;
00168         }
00169         zst[1] = WORDDIF(z,zst);
00170         break;
00171     }
00172     if ( *s == SYMTOSUB &&
00173         ( ABS(*t) - 2*MAXPOWER ) == s[2] ) {
00174         dirty = 1;
00175         zz = z;
00176         while ( --zz >= zst ) {
00177             zz[1+FUNHEAD+ARGHEAD] = *zz;
00178         }
00179         w += 1+FUNHEAD+ARGHEAD;
00180         *zst = EXPONENT;
00181         zst[2] = DIRTYFLAG;
00182         zst[FUNHEAD+ARGHEAD] = WORDDIF(z,zst)+4;
00183         zst[1+FUNHEAD] = 1;
00184         zst[FUNHEAD] = WORDDIF(z,zst)+4+ARGHEAD;
00185         z += FUNHEAD+ARGHEAD+1;
00186         *w = 1; /* exponent -> 1 */
00187         *z++ = 1;
00188         *z++ = 1;
00189         *z++ = 3;
00190         *z++ = 4+SUBEXPSIZE+ARGHEAD;
00191         *z++ = 1;
00192         *z++ = 4+SUBEXPSIZE;
00193         *z++ = SUBEXPRESSION;
00194         *z++ = SUBEXPSIZE;
00195         *z++ = s[3];
00196         *z++ = 1;
00197         *z++ = AT.ebufnum;
00198         FILLSUB(z)
00199         *z++ = 1;
00200         *z++ = 1;
00201         *z++ = *t > 0 ? 3: -3;
00202         zst[1] = WORDDIF(z,zst);
00203         break;
00204     }
00205     s += s[1];
00206 }
00207 }
00208 if ( !*w ) z = w - 3;
00209 t++;
00210 goto Seven;
00211 }
00212 else if ( *s == SYMTOSUB ) {
00213     dirty = 1;
00214     zst = z;
00215     *z++ = SUBEXPRESSION;
00216     *z++ = SUBEXPSIZE;
00217     *z++ = s[3];
00218     w = z;
00219     *z++ = *++t;
00220     *z++ = AT.ebufnum;
00221     FILLSUB(z)
00222     goto DoPow;
00223 }
00224 }
00225 s += s[1];
00226 }
00227 sspow:
00228 s = subs;
00229 *++m = *++t;
00230 for ( si = 0; si < setflag; si += 2 ) {
00231     if ( t == temp + setlist[si] ) {
00232         t++; m++;
00233         goto Seven;
00234     }
00235 }
00236 for ( j = 0; j < i; j++ ) {
00237     if ( ( ABS(*t) - 2*MAXPOWER ) == s[2] ) {
00238         if ( *s == SYMTONUM ) {
00239             dirty = 1;
00240             *m = s[3];
00241             if ( *t < 0 ) *m = -*m;
00242             break;
00243         }
00244         else if ( *s == SYMTOSYM ) {
00245             dirty = 1;
00246             *z++ = EXPONENT;
00247             if ( *t < 0 ) *z++ = FUNHEAD+ARGHEAD+10;
00248             else *z++ = 4+FUNHEAD;
00249             *z++ = 0;
00250             FILLFUN3(z)

```

```

00251             *z++ = -SYMBOL;
00252             *z++ = m[-1];
00253             if ( *t < 0 ) {
00254                 *z++ = ARGHEAD+8;
00255                 *z++ = 0;
00256                 *z++ = 8;
00257                 *z++ = SYMBOL;
00258                 *z++ = 4;
00259                 *z++ = s[3];
00260                 *z++ = 1;
00261                 *z++ = 1;
00262                 *z++ = 1;
00263                 *z = -3;
00264             }
00265             else {
00266                 *z++ = -SYMBOL;
00267                 *z++ = s[3];
00268             }
00269             m -= 2;
00270             break;
00271         }
00272         else if ( *s == SYMTOSUB ) {
00273             zst = z;
00274             *z++ = SYMBOL;
00275             *z++ = 4;
00276             *z++ = *-m;
00277             w = z;
00278             *z++ = *t;
00279             goto MakeExp;
00280         }
00281     }
00282     s += s[1];
00283 }
00284 t++;
00285 if ( *m ) m++;
00286 else m--;
00287 Seven;;
00288 }
00289 j = WORDDIF(m,v);
00290 if ( !j ) m -= 2;
00291 else v[-1] = j + 2;
00292 s = accu;
00293 while ( s < z ) *m++ = *s++;
00294 break;
00295 /*
00296 #] SYMBOLS :
00297 */
00298 case DOTPRODUCT:
00299 /*
00300 #[ DOTPRODUCTS :
00301 */
00302     *m++ = *t++;
00303     *m++ = *t++;
00304     v = m;
00305     z = accu;
00306     while ( t < u ) {
00307         *m = *t;
00308         subcount = 0;
00309         /* Process the first vector of the DOTPRODUCT */
00310         for ( si = 0; si < setflag; si += 2 ) {
00311             if ( t == temp + setlist[si] ) goto ss2;
00312         }
00313         s = subs;
00314         for ( j = 0; j < i; j++ ) {
00315             if ( *t == s[2] ) {
00316                 if ( *s == VECTOVEC ) {
00317                     *m = s[3]; dirty = 1; break;
00318                 }
00319                 if ( *s == VECTOMIN ) {
00320                     *m = s[3];
00321                     dirty = 1;
00322                     if ( ( ABS(t[2]) - 2*MAXPOWER ) < 0 ) {
00323                         /* The power is a number */
00324                         sgn += t[2];
00325                     }
00326                     else {
00327                         /* The power is a wildcard. Put a -, resolve later. */
00328                         sgn++;
00329                     }
00330                     break;
00331                 }
00332                 if ( *s == VECTOSUB ) {
00333                     *m = s[3]; dirty = 1; subcount = 1; break;
00334                 }
00335             }
00336             s += s[1];
00337         }

```

```

00338 ss2:
00339     ***m = ***t;
00340     s = subs;
00341     /* Process the second vector of the DOTPRODUCT */
00342     for ( si = 0; si < setflag; si += 2 ) {
00343         if ( t == temp + setlist[si] ) goto ss3;
00344     }
00345     for ( j = 0; j < i; j++ ) {
00346         if ( *t == s[2] ) {
00347             if ( *s == VECTOVEC ) {
00348                 *m = s[3]; dirty = 1; break;
00349             }
00350             if ( *s == VECTOMIN ) {
00351                 *m = s[3];
00352                 dirty = 1;
00353                 if ( ( ABS(t[1]) - 2*MAXPOWER ) < 0 ) {
00354                     /* The power is a number */
00355                     sgn += t[1];
00356                 }
00357                 else {
00358                     /* The power is a wildcard. Put a -, resolve later. */
00359                     sgn++;
00360                 }
00361                 break;
00362             }
00363             if ( *s == VECTOSUB ) {
00364                 *m = s[3]; dirty = 1; subcount += 2; break;
00365             }
00366         }
00367         s += s[1];
00368     }
00369 ss3:
00370     ***m = ***t;
00371     /* Process the power */
00372     if ( ( ABS(*t) - 2*MAXPOWER ) < 0 ) goto RegPow;
00373     s = subs;
00374     for ( j = 0; j < i; j++ ) {
00375         if ( ( ABS(*t) - 2*MAXPOWER ) == s[2] ) {
00376             if ( *s == SYMTONUM ) {
00377                 *m = s[3];
00378                 /* Since the power is a wildcard, sgn is 0,1,2 depending whether
00379                    there were 0,1,2 VECTOMIN in the DOTPRODUCT. Multiply by the
00380                    power, which is positive currently. */
00381                 sgn *= *m;
00382                 /* Now flip the sign of the power, if the wildcard came with a - */
00383                 if ( *t < 0 ) *m = -*m;
00384                 dirty = 1;
00385                 break;
00386             }
00387             if ( *s <= SYMTOSUB ) {
00388
00389                 Here we put together a power function with the proper
00390                 arguments. Note that a p?.q? resolves to a single power.
00391
00392                 m -= 2;
00393                 *z++ = EXPONENT;
00394                 w = z;
00395                 if ( subcount == 0 ) {
00396                     *z++ = 17+FUNHEAD+2*ARGHEAD;
00397                     *z++ = DIRTYFLAG;
00398                     FILLFUN3(z)
00399                     *z++ = 9+ARGHEAD;
00400                     *z++ = 0;
00401                     FILLARG(z)
00402                     *z++ = 9;
00403                     *z++ = DOTPRODUCT;
00404                     *z++ = 5;
00405                     *z++ = *m;
00406                     *z++ = m[1];
00407                     *z++ = 1;
00408                     *z++ = 1;
00409                     *z++ = 1;
00410                     *z++ = 3;
00411                     if ( *s == SYMTOSYM ) {
00412                         *z++ = 8+ARGHEAD;
00413                         *z++ = 0;
00414                         FILLARG(z)
00415                         *z++ = 8;
00416                         *z++ = SYMBOL;
00417                         *z++ = 4;
00418                         *z++ = s[3];
00419                         *z++ = 1;
00420                     }
00421                     else {
00422                         *z++ = 4+SUBEXPSIZE+ARGHEAD;
00423                         *z++ = 1;
00424                         FILLARG(z)
00425                         *z++ = 4+SUBEXPSIZE;

```

```

00425         *z++ = SUBEXPRESSION;
00426         *z++ = SUBEXPSIZE;
00427         *z++ = s[3];
00428         *z++ = 1;
00429         *z++ = AT.ebufnum;
00430         FILLSUB(z)
00431     }
00432     *z++ = 1; *z++ = 1;
00433     *z++ = ( s[2] > 0 ) ? 3: -3;
00434 }
00435 else if ( subcount == 3 ) {
00436     *z++ = 20+2*SUBEXPSIZE+FUNHEAD+2*ARGHEAD;
00437     *z++ = DIRTYFLAG;
00438     FILLFUN3(z)
00439     *z++ = 12+2*SUBEXPSIZE+ARGHEAD;
00440     *z++ = 1;
00441     *z++ = 12+2*SUBEXPSIZE;
00442     *z++ = SUBEXPRESSION;
00443     *z++ = 4+SUBEXPSIZE;
00444     *z++ = *m + 1;
00445     *z++ = 1;
00446     *z++ = AT.ebufnum;
00447     FILLSUB(z)
00448     *z++ = INDTOIND;
00449     *z++ = 4;
00450     *z++ = FUNNYVEC;
00451     *z++ = ++AR.CurDum;

00452
00453     *z++ = SUBEXPRESSION;
00454     *z++ = 4+SUBEXPSIZE;
00455     *z++ = m[1] + 1;
00456     *z++ = 1;
00457     *z++ = AT.ebufnum;
00458     FILLSUB(z)
00459     *z++ = INDTOIND;
00460     *z++ = 4;
00461     *z++ = FUNNYVEC;
00462     *z++ = AR.CurDum;
00463     *z++ = 1; *z++ = 1; *z++ = 3;
00464 }
00465 else {
00466     if ( subcount == 2 ) {
00467         j = *m; *m = m[1]; m[1] = j;
00468     }
00469     *z++ = 16+SUBEXPSIZE+FUNHEAD+2*ARGHEAD;
00470     *z++ = DIRTYFLAG;
00471     FILLFUN3(z)
00472     *z++ = 8+SUBEXPSIZE+ARGHEAD;
00473     *z++ = 1;
00474     *z++ = 8+SUBEXPSIZE;
00475     *z++ = SUBEXPRESSION;
00476     *z++ = 4+SUBEXPSIZE;
00477     *z++ = *m + 1;
00478     *z++ = 1;
00479     *z++ = AT.ebufnum;
00480     FILLSUB(z)
00481     *z++ = INDTOIND;
00482     *z++ = 4;
00483     *z++ = FUNNYVEC;
00484     *z++ = m[1];
00485     *z++ = 1; *z++ = 1; *z++ = 3;
00486 }
00487 if ( *s == SYMTOSYM ) {
00488     if ( s[2] > 0 ) {
00489         *z++ = -SYMBOL;
00490         *z++ = s[3];
00491         t++;
00492         *w = z-w+1;
00493         goto NextDot;
00494     }
00495     *z++ = 8+ARGHEAD;
00496     *z++ = 0;
00497     *z++ = 8;
00498     *z++ = SYMBOL;
00499     *z++ = 4;
00500     *z++ = s[3];
00501     *z++ = 1;
00502 }
00503 else {
00504     *z++ = 4+SUBEXPSIZE+ARGHEAD;
00505     *z++ = 1;
00506     *z++ = 4+SUBEXPSIZE;
00507     *z++ = SUBEXPRESSION;
00508     *z++ = SUBEXPSIZE;
00509     *z++ = s[3];
00510     *z++ = 1;
00511     *z++ = AT.ebufnum;

```

```

00512             FILLSUB(z)
00513         }
00514         *z++ = 1; *z++ = 1;
00515         *z++ = ( s[2] > 0 ) ? 3: -3;
00516         t++;
00517         *w = z-w+1;
00518         goto NextDot;
00519     }
00520 }
00521 s += s[1];
00522 }
00523 RegPow:
00524 if ( *m ) m++;
00525 else { m -= 2; subcount = 0; }
00526 t++;
00527 if ( subcount ) {
00528     m -= 3;
00529     if ( subcount == 3 ) {
00530         if ( m[2] < 0 ) {
00531             j = (-m[2]) * (2*SUBEXPSIZE+8);
00532             *z++ = DENOMINATOR;
00533             *z++ = j + 8 + FUNHEAD + ARGHEAD;
00534             *z++ = DIRTYFLAG;
00535             FILLFUN3(z)
00536             *z++ = j + 8 + ARGHEAD;
00537             *z++ = 1;
00538             *z++ = j + 8;
00539             while ( m[2] < 0 ) {
00540                 (m[2])++;
00541                 *z++ = SUBEXPRESSION;
00542                 *z++ = 4+SUBEXPSIZE;
00543                 *z++ = *m + 1;
00544                 *z++ = 1;
00545                 *z++ = AT.ebufnum;
00546                 FILLSUB(z)
00547                 *z++ = INDTOIND;
00548                 *z++ = 4;
00549                 *z++ = FUNNYVEC;
00550                 *z++ = ++AR.CurDum;
00551                 *z++ = SUBEXPRESSION;
00552                 *z++ = 8+SUBEXPSIZE;
00553                 *z++ = m[1] + 1;
00554                 *z++ = 1;
00555                 *z++ = AT.ebufnum;
00556                 FILLSUB(z)
00557                 *z++ = INDTOIND;
00558                 *z++ = 4;
00559                 *z++ = FUNNYVEC;
00560                 *z++ = AR.CurDum;
00561                 *z++ = SYMTOSYM; /* Needed to avoid */
00562                 *z++ = 4; /* problems with */
00563                 *z++ = 1000; /* conversion to */
00564                 *z++ = 1000; /* square of subexp*/
00565             }
00566             *z++ = 1; *z++ = 1; *z++ = 3;
00567         }
00568         else {
00569             while ( m[2] > 0 ) {
00570                 (m[2])--;
00571                 *z++ = SUBEXPRESSION;
00572                 *z++ = 4+SUBEXPSIZE;
00573                 *z++ = *m + 1;
00574                 *z++ = 1;
00575                 *z++ = AT.ebufnum;
00576                 FILLSUB(z)
00577                 *z++ = INDTOIND;
00578                 *z++ = 4;
00579                 *z++ = FUNNYVEC;
00580                 *z++ = ++AR.CurDum;
00581                 *z++ = SUBEXPRESSION;
00582                 *z++ = 4+SUBEXPSIZE;
00583                 *z++ = m[1] + 1;
00584                 *z++ = 1;
00585                 *z++ = AT.ebufnum;
00586                 FILLSUB(z)
00587                 *z++ = INDTOIND;
00588                 *z++ = 4;
00589                 *z++ = FUNNYVEC;
00590                 *z++ = AR.CurDum;
00591             }
00592         }
00593     }
00594     else {
00595         if ( subcount == 2 ) {
00596             j = *m; *m = m[1]; m[1] = j;
00597         }
00598         if ( m[2] < 0 ) {
00599             *z++ = DENOMINATOR;

```

```

00599             *z++ = 8+SUBEXPSIZE+FUNHEAD+ARGHEAD;
00600             *z++ = DIRTYFLAG;
00601             FILLFUN3(z)
00602             *z++ = 8+SUBEXPSIZE+ARGHEAD;
00603             *z++ = 1;
00604             *z++ = 8+SUBEXPSIZE;
00605         }
00606         *z++ = SUBEXPRESSION;
00607         *z++ = 4+SUBEXPSIZE;
00608         *z++ = *m + 1;
00609         *z++ = ABS(m[2]);
00610         *z++ = AT.ebufnum;
00611         FILLSUB(z)
00612         *z++ = INDTOIND;
00613         *z++ = 4;
00614         *z++ = FUNNYVEC;
00615         *z++ = m[1];
00616         if ( m[2] < 0 ) {
00617             *z++ = 1; *z++ = 1; *z++ = 3;
00618         }
00619     }
00620 }
00621 NextDot;;
00622     }
00623     if ( m <= v ) m = v - 2;
00624     else v[-1] = WORDDIF(m,v) + 2;
00625     if ( z > accu ) {
00626         j = WORDDIF(z,accu);
00627         z = accu;
00628         NCOPY(m,z,j);
00629     }
00630     break;
00631 /*
00632 #] DOTPRODUCTS :
00633 */
00634 case SETSET:
00635 /*
00636 #[ SETS :
00637 */
00638     temp = accu + ((AR.ComprTop - accu)»1)&(-2));
00639     if ( ResolveSet(BHEAD t,temp,sub) ) {
00640         Terminate(-1);
00641     }
00642     setlist = t + 2 + t[3];
00643     setflag = t[1] - 2 - t[3]; /* Number of elements * 2 */
00644     t = temp; u = t + t[1];
00645     goto ReSwitch;
00646 /*
00647 #] SETS :
00648 */
00649 case VECTOR:
00650 /*
00651 #[ VECTORS :
00652 */
00653     *m++ = *t++;
00654     *m++ = *t++;
00655     v = m;
00656     z = accu;
00657     while ( t < u ) {
00658         *m = *t;
00659         for ( si = 0; si < setflag; si += 2 ) {
00660             if ( t == temp + setlist[si] ) goto ss4;
00661         }
00662         s = subs;
00663         for ( j = 0; j < i; j++ ) {
00664             if ( *t == s[2] ) {
00665                 if ( *s == INDTOIND || *s == VECTOVEC ) {
00666                     *m = s[3]; dirty = 1; break;
00667                 }
00668                 if ( *s == VECTOMIN ) {
00669                     *m = s[3]; dirty = 1; sgn++; break;
00670                 }
00671                 else if ( *s == VECTOSUB ) {
00672                     *z++ = SUBEXPRESSION;
00673                     *z++ = 4+SUBEXPSIZE;
00674                     *z++ = s[3]+1;
00675                     *z++ = 1;
00676                     *z++ = AT.ebufnum;
00677                     FILLSUB(z)
00678                     *z++ = VECTOVEC;
00679                     *z++ = 4;
00680                     *z++ = FUNNYVEC;
00681                     *z++ = *++t;
00682                     m--;
00683                     s = subs;
00684                     for ( j = 0; j < i; j++ ) {
00685                         if ( z[-1] == s[2] ) {

```

```

00686         if ( *s == INDTOIND || *s == VECTOVEC ) {
00687             z[-1] = s[3];
00688             break;
00689         }
00690         if ( *s == INDTOSUB || *s == VECTOSUB ) {
00691             z[-1] = ++AR.CurDum;
00692             *z++ = SUBEXPRESSION;
00693             *z++ = 4+SUBEXPSIZE;
00694             *z++ = s[3]+1;
00695             *z++ = 1;
00696             *z++ = AT.ebufnum;
00697             FILLSUB(z)
00698             if ( *s == INDTOSUB ) *z++ = INDTOIND;
00699             else *z++ = VECTOSUB;
00700             *z++ = 4;
00701             *z++ = FUNNYVEC;
00702             *z++ = AR.CurDum;
00703             break;
00704         }
00705     }
00706     s += s[1];
00707 }
00708 dirty = 1;
00709 break;
00710 }
00711 else if ( *s == INDTOSUB ) {
00712     *z++ = SUBEXPRESSION;
00713     *z++ = 4+SUBEXPSIZE;
00714     *z++ = s[3]+1;
00715     *z++ = 1;
00716     *z++ = AT.ebufnum;
00717     FILLSUB(z)
00718     *z++ = INDTOIND;
00719     *z++ = 4;
00720     *z++ = FUNNYVEC;
00721     m -= 2;
00722     *z++ = m[1];
00723     dirty = 1;
00724     t++;
00725     break;
00726 }
00727 }
00728 s += s[1];
00729 }
00730 ss4:
00731     m++; t++;
00732 }
00733 if ( m <= v ) m = v-2;
00734 else v[-1] = WORDDIF(m,v)+2;
00735 if ( z > accu ) {
00736     j = WORDDIF(z,accu); z = accu;
00737     NCOPY(m,z,j);
00738 }
00739 break;
00740 /*
00741 #] VECTORS :
00742 case INDEX:
00743 /*
00744 #[ INDEX :
00745 */
00746     *m++ = *t++;
00747     *m++ = *t++;
00748     v = m;
00749     z = accu;
00750     while ( t < u ) {
00751         *m = *t;
00752         for ( si = 0; si < setflag; si += 2 ) {
00753             if ( t == temp + setlist[si] ) goto ss5;
00754         }
00755         s = subs;
00756         for ( j = 0; j < i; j++ ) {
00757             if ( *t == s[2] ) {
00758                 if ( *s == INDTOIND || *s == VECTOVEC )
00759                     { *m = s[3]; dirty = 1; break; }
00760                 if ( *s == VECTOMIN )
00761                     { *m = s[3]; dirty = 1; sgn++; break; }
00762                 else if ( *s == VECTOSUB || *s == INDTOSUB ) {
00763                     *z++ = SUBEXPRESSION;
00764                     *z++ = SUBEXPSIZE;
00765                     *z++ = s[3];
00766                     *z++ = 1;
00767                     *z++ = AT.ebufnum;
00768                     FILLSUB(z)
00769                     m--;
00770                     dirty = 1;
00771                     break;
00772                 }

```

```

00773         }
00774         s += s[1];
00775     }
00776 ss5:      m++; t++;
00777     }
00778     if ( m <= v ) m = v-2;
00779     else v[-1] = WORDDIF(m,v)+2;
00780     if ( z > accu ) {
00781         j = WORDDIF(z,accu); z = accu;
00782         NCOPY(m,z,j);
00783     }
00784     break;
00785 /*
00786     #] INDEX :
00787 */
00788     case DELTA:
00789     case LEVICIVITA:
00790     case GAMMA:
00791 /*
00792     #[ SPECIALS :
00793 */
00794     v = m;
00795     *m++ = *t++;
00796     *m++ = *t++;
00797     #if FUNHEAD > 2
00798         if ( t[-2] != DELTA ) *m++ = *t++;
00799     #endif
00800 Tensors:
00801     COPYFUN3(m,t)
00802     z = accu;
00803     while ( t < u ) {
00804         *m = *t;
00805         for ( si = 0; si < setflag; si += 2 ) {
00806             if ( t == temp + setlist[si] ) goto ss6;
00807         }
00808         s = subs;
00809         if ( *m == FUNNYWILD ) {
00810             CBUF *C = cbuf+AT.ebufnum;
00811             t++;
00812             for ( j = 0; j < i; j++ ) {
00813                 if ( *s == ARGTOARG && *t == s[2] ) {
00814                     v[2] |= DIRTYFLAG;
00815                     if ( s[3] < 0 ) { /* empty */
00816                         t++; break;
00817                     }
00818                     w = C->rhs[s[3]];
00819                     DEBUG(MesPrint("Thread %w(a): s[3] = %d, w=(%d,%d,%d,%d)",s[3],w[0],w[1],w[2],w[3]));
00820                     j = *w++;
00821                     if ( j > 0 ) {
00822                         NCOPY(m,w,j);
00823                     }
00824                     else {
00825                         while ( *w ) {
00826                             if ( *w == -INDEX || *w == -VECTOR
00827                                 || *w == -MINVECTOR
00828                                 || ( *w == -SNUMBER && w[1] >= 0
00829                                     && w[1] < AM.OffsetIndex ) ) {
00830                                 if ( *w == -MINVECTOR ) sgn++;
00831                                 w++;
00832                                 *m++ = *w++;
00833                             }
00834                             else {
00835                                 MLOCK(ErrorMessageLock);
00836                                 DEBUG(MesPrint("Thread %w(aa): *w = %d",*w));
00837                                 MesPrint("Illegal substitution of argument field in
tensor");
00838                                 MUNLOCK(ErrorMessageLock);
00839                                 SETERROR(-1)
00840                             }
00841                         }
00842                     }
00843                     t++;
00844                     break;
00845                 }
00846                 s += s[1];
00847             }
00848         }
00849         else {
00850             for ( j = 0; j < i; j++ ) {
00851                 if ( *t == s[2] ) {
00852                     if ( *s == INDTOIND || *s == VECTOVEC )
00853                         { *m = s[3]; dirty = 1; break; }
00854                     if ( *s == VECTOMIN )
00855                         { *m = s[3]; dirty = 1; sgn++; break; }
00856                     else if ( *s == VECTOSUB || *s == INDTOSUB ) {
00857                         *m = ++AR.CurDum;
00858                         *z++ = SUBEXPRESSION;

```



```

00859                                     *z++ = 4+SUBEXPSIZE;
00860                                     *z++ = s[3]+1;
00861                                     *z++ = 1;
00862                                     *z++ = AT.ebufnum;
00863                                     FILLSUB(z)
00864                                     *z++ = INDTOIND;
00865                                     *z++ = 4;
00866                                     *z++ = FUNNYVEC;
00867                                     *z++ = AR.CurDum;
00868                                     dirty = 1;
00869                                     break;
00870                                 }
00871                             }
00872                             s += s[1];
00873                         }
00874                         if ( j < i && *v != DELTA ) v[2] |= DIRTYFLAG;
00875 ss6:                             m++; t++;
00876                     }
00877                 }
00878                 v[1] = WORDDIF(m,v);
00879                 if ( z > accu ) {
00880                     j = WORDDIF(z,accu); z = accu;
00881                     NCOPY(m,z,j);
00882                 }
00883                 break;
00884 /*
00885 #] SPECIALS :
00886 */
00887 case SUBEXPRESSION:
00888 /*
00889 #[ SUBEXPRESSION :
00890 */
00891     dirty = 1;
00892     tstop = t + t[1];
00893     *m++ = *t++;
00894     *m++ = *t++;
00895     *m++ = *t++;
00896     *m++ = *t++;
00897     if ( t[-1] >= 2*MAXPOWER || t[-1] <= -2*MAXPOWER ) {
00898         s = subs;
00899         for ( j = 0; j < i; j++ ) {
00900             if ( *s == SYMTONUM &&
00901                 ( ABS(t[-1]) - 2*MAXPOWER ) == s[2] ) {
00902                 m[-1] = s[3];
00903                 if ( t[-1] < 0 ) m[-1] = -m[-1];
00904                 break;
00905             }
00906             s += s[1];
00907         }
00908     }
00909     *m++ = *t++;
00910     COPYSUB(m,t)
00911     while ( t < tstop ) {
00912         for ( si = 0; si < setflag; si += 2 ) {
00913             if ( t == temp + setlist[si] - 2 ) goto ss7;
00914         }
00915         s = subs;
00916         for ( j = 0; j < i; j++ ) {
00917             if ( s[2] == t[2] ) {
00918                 if ( ( *s <= SYMTOSUB && *t <= SYMTOSUB )
00919                     || ( *s == *t && *s < FROMBRAC )
00920                     || ( *s == VECTOVEC && ( *t == VECTOSUB || *t == VECTOMIN ) )
00921                     || ( *s == VECTOSUB && ( *t == VECTOVEC || *t == VECTOMIN ) )
00922                     || ( *s == VECTOMIN && ( *t == VECTOSUB || *t == VECTOVEC ) )
00923                     || ( *s == INDTOIND && *t == INDTOIND )
00924                     || ( *s == INDTOIND && *t == INDTOIND ) ) {
00925                     WORD *vv = m;
00926 /*
00927                     *t = *s; Wrong!!! Overwrites compiler buffer */
00928                     j = t[1];
00929                     NCOPY(m,t,j);
00930                     vv[0] = s[0];
00931                     vv[3] = s[3];
00932                     goto sr7;
00933                 }
00934             }
00935             s += s[1];
00936 ss7:             j = t[1];
00937             NCOPY(m,t,j);
00938 sr7:;
00939         }
00940         break;
00941 /*
00942 #] SUBEXPRESSION :
00943 */
00944 case EXPRESSION:
00945 /*

```

```

00946      #[] EXPRESSION :
00947      */
00948          dirty = 1;
00949          tstop = t + t[1];
00950          v = m;
00951          *m++ = *t++;
00952          *m++ = *t++;
00953          *m++ = *t++;
00954          *m++ = *t++;
00955          s = subs;
00956          for ( j = 0; j < i; j++ ) {
00957              if ( ( ABS(t[-1]) - 2*MAXPOWER ) == s[2] ) {
00958                  if ( *s == SYMTONUM ) {
00959                      m[-1] = s[3];
00960                      if ( t[-1] < 0 ) m[-1] = -m[-1];
00961                      break;
00962                  }
00963                  else if ( *s <= SYMTOSUB ) {
00964                      MLOCK(ErrorMessageLock);
00965                      MesPrint("Wildcard power of expression should be a number");
00966                      MUNLOCK(ErrorMessageLock);
00967                      SETERROR(-1)
00968                  }
00969              }
00970              s += s[1];
00971          }
00972          *m++ = *t++;
00973          COPYSUB(m,t)
00974          while ( t < tstop && *t != WILDCARDS ) {
00975              j = t[1];
00976              NCOPY(m,t,j);
00977          }
00978          if ( t < tstop && *t == WILDCARDS ) {
00979              *m++ = *t;
00980              s = sub;
00981              j = s[1];
00982              *m++ = j+2;
00983              NCOPY(m,s,j);
00984              t += t[1];
00985          }
00986          if ( t < tstop && *t == FROMBRAC ) {
00987              w = m;
00988              *m++ = *t;
00989              *m++ = t[1];
00990              if ( WildFill(BHEAD m,t+2,sub) < 0 ) {
00991                  MLOCK(ErrorMessageLock);
00992                  MesCall("WildFill");
00993                  MUNLOCK(ErrorMessageLock);
00994                  SETERROR(-1)
00995              }
00996              m += *m;
00997              w[1] = m - w;
00998              t += t[1];
00999          }
01000          while ( t < tstop ) {
01001              j = t[1];
01002              NCOPY(m,t,j);
01003          }
01004          v[1] = m-v;
01005          break;
01006      /*
01007      #[] EXPRESSION :
01008      */
01009      default:
01010      /*
01011      #[] FUNCTIONS :
01012      */
01013          if ( *t >= FUNCTION ) {
01014              dflag = 0;
01015              na = 0;
01016              *m = *t;
01017              for ( si = 0; si < setflag; si += 2 ) {
01018                  if ( t == temp + setlist[si] ) {
01019                      dflag = DIRTYFLAG; goto ss8;
01020                  }
01021              }
01022              s = subs;
01023              for ( j = 0; j < i; j++ ) {
01024                  if ( *s == FUNTOFUN && *t == s[2] )
01025                      { *m = s[3]; dirty = 1; dflag = DIRTYFLAG; break; }
01026                  s += s[1];
01027              }
01028          ss8:
01029              v = m;
01030              if ( *t >= FUNCTION && functions[*t-FUNCTION].spec
01031                  >= TENSORFUNCTION ) {
01032                  if ( *m < FUNCTION || functions[*m-FUNCTION].spec
01033                      < TENSORFUNCTION ) {

```

```

01033             MLOCK(ErrorMessageLock);
01034             MesPrint("Illegal wildcarding of regular function to tensorfunction");
01035             MUNLOCK(ErrorMessageLock);
01036             SETERROR(-1)
01037         }
01038         m++; t++;
01039         *m++ = *t++;
01040         *m++ = *t++ | dflag;
01041         goto Tensors;
01042     }
01043     m++; t++;
01044     *m++ = *t++;
01045     *m++ = *t++ | dflag;
01046     COPYFUN3(m,t)
01047     z = accu;
01048     while ( t < u ) {          /* do an argument */
01049         if ( *t < 0 ) {
01050             /*
01051              *# Simple arguments :
01052              */
01053             CBUF *C = cbuf+AT.ebufnum;
01054             for ( si = 0; si < setflag; si += 2 ) {
01055                 if ( *t <= -FUNCTION ) {
01056                     if ( t == temp + setlist[si] ) {
01057                         v[2] |= DIRTYFLAG; goto ss10; }
01058                     }
01059                     else {
01060                         if ( t == temp + setlist[si]-1 ) {
01061                             v[2] |= DIRTYFLAG; goto ss9; }
01062                         }
01063                     }
01064                 if ( *t == -ARGWILD ) {
01065                     s = subs;
01066                     for ( j = 0; j < i; j++ ) {
01067                         if ( *s == ARGTOARG && s[2] == t[1] ) break;
01068                         s += s[1];
01069                     }
01070                     v[2] |= DIRTYFLAG;
01071                     w = C->rhs[s[3]];
01072                     DEBUG(MesPrint("Thread %w(b): s[3] = %d, w=(%d,%d,%d,%d)",s[3],w[0],w[1],w[2],w[3]));
01073                     if ( *w == 0 ) {
01074                         w++;
01075                         while ( *w ) {
01076                             if ( *w > 0 ) j = *w;
01077                             else if ( *w <= -FUNCTION ) j = 1;
01078                             else j = 2;
01079                             NCOPY(m,w,j);
01080                         }
01081                     }
01082                     else {
01083                         j = *w++;
01084                         while ( --j >= 0 ) {
01085                             if ( *w < MINSPEC ) *m++ = -VECTOR;
01086                             else if ( *w >= 0 && *w < AM.OffsetIndex )
01087                                 *m++ = -SNUMBER;
01088                             else *m++ = -INDEX;
01089                             *m++ = *w++;
01090                         }
01091                     }
01092                     t += 2;
01093                     dirty = 1;
01094                     if ( ( *v == NUMARGSFUN || *v == NUMTERMSFUN )
01095                         && t >= u && m == v + FUNHEAD ) {
01096                         m = v;
01097                         *m++ = SNUMBER; *m++ = 3; *m++ = 0;
01098                         break;
01099                     }
01100                 }
01101                 else if ( *t <= -FUNCTION ) {
01102                     *m = *t;
01103                     s = subs;
01104                     for ( j = 0; j < i; j++ ) {
01105                         if ( -*t == s[2] ) {
01106                             if ( *s == FUNTOFUN )
01107                                 { *m = -s[3]; dirty = 1; v[2] |= DIRTYFLAG; break; }
01108                             }
01109                         s += s[1];
01110                     }
01111                     m++; t++;
01112                 }
01113                 else if ( *t == -SYMBOL ) {
01114                     *m++ = *t++;
01115                     *m = *t;
01116                     s = subs;
01117                     for ( j = 0; j < i; j++ ) {
01118                         if ( *t == s[2] && *s <= SYMTOSUB ) {
01119                             dirty = 1; v[2] |= DIRTYFLAG;

```

```

01120         if ( AR.PolyFunType == 2 && v[0] == AR.PolyFun )
01121             v[2] |= MUSTCLEANPRF;
01122         if ( *s == SYMTOSYM ) *m = s[3];
01123     else if ( *s == SYMTONUM ) {
01124         m[-1] = -SNUMBER;
01125         *m = s[3];
01126     }
01127     else if ( *s == SYMTOSUB ) {
01128         ToSub:
01129         m--;
01130         w = C->rhs[s[3]];
01131         DEBUG(MesPrint("Thread %w(c): s[3] = %d, w=(%d,%d,%d,%d)",s[3],w[0],w[1],w[2],w[3]));
01132         s = m;
01133         m += 2;
01134         while ( *w ) {
01135             j = *w;
01136             NCOPY(m,w,j);
01137         }
01138         *s = WORDDIF(m,s);
01139         s[1] = 0;
01140         *m = 0;
01141         if ( t[-1] == -MINVECTOR ) {
01142             w = s+2;
01143             while ( *w ) {
01144                 w += *w;
01145                 w[-1] = -w[-1];
01146             }
01147             if ( ToFast(s,s) ) {
01148                 if ( *s <= -FUNCTION ) m = s;
01149                 else m = s + 1;
01150             }
01151             else m--;
01152         }
01153         break;
01154     }
01155     s += s[1];
01156 }
01157 m++; t++;
01158 }
01159 else if ( *t == -INDEX ) {
01160     *m++ = *t++;
01161     *m = *t;
01162     s = subs;
01163     for ( j = 0; j < i; j++ ) {
01164         if ( *t == s[2] ) {
01165             if ( *s == INDTOIND || *s == VECTOVEC ) {
01166                 *m = s[3];
01167                 if ( *m < MINSPEC ) m[-1] = -VECTOR;
01168                 else if ( *m >= 0 && *m < AM.OffsetIndex )
01169                     m[-1] = -SNUMBER;
01170                 else m[-1] = -INDEX;
01171             }
01172             else if ( *s == VECTOSUB || *s == INDTOSUB ) {
01173                 m[-1] = -INDEX;
01174                 *m = ++AR.CurDum;
01175                 *z++ = SUBEXPRESSION;
01176                 *z++ = 4+SUBEXPSIZE;
01177                 *z++ = s[3]+1;
01178                 *z++ = 1;
01179                 *z++ = AT.ebufnum;
01180                 FILLSUB(z);
01181                 *z++ = INDTOIND;
01182                 *z++ = 4;
01183                 *z++ = FUNNYVEC;
01184                 *z++ = AR.CurDum;
01185             }
01186             v[2] |= DIRTYFLAG; dirty = 1;
01187             break;
01188         }
01189         s += s[1];
01190     }
01191     m++; t++;
01192 }
01193 else if ( *t == -VECTOR || *t == -MINVECTOR ) {
01194     *m++ = *t++;
01195     *m = *t;
01196     s = subs;
01197     for ( j = 0; j < i; j++ ) {
01198         if ( *t == s[2] ) {
01199             if ( *s == VECTOVEC ) *m = s[3];
01200             else if ( *s == VECTOMIN ) {
01201                 *m = s[3];
01202                 if ( t[-1] == -VECTOR )
01203                     m[-1] = -MINVECTOR;
01204                 else
01205                     m[-1] = -VECTOR;
01206             }

```

```

01207         else if ( *s == VECTOSUB ) goto ToSub;
01208         dirty = 1; v[2] |= DIRTYFLAG;
01209         break;
01210     }
01211     s += s[1];
01212 }
01213 m++; t++;
01214 }
01215 else if ( *t == -SNUMBER ) {
01216     *m++ = *t++;
01217     *m = *t;
01218     s = subs;
01219     for ( j = 0; j < i; j++ ) {
01220         if ( *t == s[2] && *s >= NUMTONUM && *s <= NUMTOSUB ) {
01221             dirty = 1; v[2] |= DIRTYFLAG;
01222             if ( *s == NUMTONUM ) *m = s[3];
01223             else if ( *s == NUMTOSYM ) {
01224                 m[-1] = -SYMBOL;
01225                 *m = s[3];
01226             }
01227             else if ( *s == NUMTOIND ) {
01228                 m[-1] = -INDEX;
01229                 *m = s[3];
01230             }
01231             else if ( *s == NUMTOSUB ) goto ToSub;
01232             break;
01233         }
01234         s += s[1];
01235     }
01236     m++; t++;
01237 }
01238 else {
01239     *m++ = *t++;
01240     *m++ = *t++;
01241 }
01242 na = WORDDIFF(z,accu);
01243 /*
01244 #] Simple arguments :
01245 */
01246 }
01247 else {
01248     w = m;
01249     zz = t;
01250     NEXTARG(zz)
01251     odirt = AN.WildDirt; AN.WildDirt = 0;
01252     AR.CompressPointer = accu + na;
01253     for ( j = 0; j < ARGHEAD; j++ ) *m++ = *t++;
01254     j = 0;
01255     adirt = 0;
01256     while ( t < zz ) { /* do a term */
01257         if ( ( len = WildFill(BHEAD m,t,sub) ) < 0 ) {
01258             MLOCK(ErrorMessageLock);
01259             MesCall("WildFill");
01260             MUNLOCK(ErrorMessageLock);
01261             SETERROR(-1)
01262         }
01263         if ( AN.WildDirt ) {
01264             adirt = AN.WildDirt;
01265             AN.WildDirt = 0;
01266         }
01267         m += len;
01268         t += *t;
01269     }
01270     *w = WORDDIFF(m,w); /* Fill parameter length */
01271     if ( adirt ) {
01272         dirty = w[1] = 1; v[2] |= DIRTYFLAG;
01273         if ( AR.PolyFunType == 2 && v[0] == AR.PolyFun )
01274             v[2] |= MUSTCLEANPRF;
01275         AN.WildDirt = adirt;
01276     }
01277     else {
01278         AN.WildDirt = odirt;
01279     }
01280     if ( ToFast(w,w) ) {
01281         if ( *w <= -FUNCTION ) {
01282             if ( *w == NUMARGSFUN || *w == NUMTERMSFUN ) {
01283                 *w = -SNUMBER; w[1] = 0; m = w + 2;
01284             }
01285             else m = w+1;
01286         }
01287         else m = w+2;
01288     }
01289     AR.CompressPointer = oldcpointer;
01290 }
01291 }
01292 v[1] = WORDDIFF(m,v); /* Fill function length */
01293 s = accu;

```

```

01294          NCOPY(m,s,na);
01295  /*
01296          Now some code to speed up a few special cases
01297  */
01298          if ( v[0] == EXPONENT ) {
01299              if ( v[1] == FUNHEAD+4 && v[FUNHEAD] == -SYMBOL &&
01300                  v[FUNHEAD+2] == -SNUMBER && v[FUNHEAD+3] < MAXPOWER
01301                  && v[FUNHEAD+3] > -MAXPOWER ) {
01302                  v[0] = SYMBOL;
01303                  v[1] = 4;
01304                  v[2] = v[FUNHEAD+1];
01305                  v[3] = v[FUNHEAD+3];
01306                  m = v+4;
01307              }
01308              else if ( v[1] == FUNHEAD+ARGHEAD+11
01309                      && v[FUNHEAD] == ARGHEAD+9
01310                      && v[FUNHEAD+ARGHEAD] == 9
01311                      && v[FUNHEAD+ARGHEAD+1] == DOTPRODUCT
01312                      && v[FUNHEAD+ARGHEAD+8] == 3
01313                      && v[FUNHEAD+ARGHEAD+7] == 1
01314                      && v[FUNHEAD+ARGHEAD+6] == 1
01315                      && v[FUNHEAD+ARGHEAD+5] == 1
01316                      && v[FUNHEAD+ARGHEAD+9] == -SNUMBER
01317                      && v[FUNHEAD+ARGHEAD+10] < MAXPOWER
01318                      && v[FUNHEAD+ARGHEAD+10] > -MAXPOWER ) {
01319                  v[0] = DOTPRODUCT;
01320                  v[1] = 5;
01321                  v[2] = v[FUNHEAD+ARGHEAD+3];
01322                  v[3] = v[FUNHEAD+ARGHEAD+4];
01323                  v[4] = v[FUNHEAD+ARGHEAD+10];
01324                  m = v+5;
01325              }
01326          }
01327      }
01328      else { while ( t < u ) *m++ = *t++; }
01329  /*
01330      #] FUNCTIONS :
01331  */
01332      {
01333          t = uu;
01334      } while ( t < r );
01335      t = from; /* Copy coefficient */
01336      t += *t;
01337      if ( r < t ) do { *m++ = *r++; } while ( r < t );
01338      if ( ( sgn & 1 ) != 0 ) m[-1] = -m[-1];
01339      *to = WORDDIF(m,to);
01340      if ( dirty ) AN.WildDirt = dirty;
01341      return(*to);
01342  }
01343
01344  /*
01345      #] WildFill :
01346      #[ ResolveSet :          WORD ResolveSet(from,to,subs)
01347
01348      The set syntax is:
01349      SET,length,subterm,where,whichmember[,where,whichmember]
01350
01351      setlength is 2*n+1 with n the number of set substitutions.
01352      length = setlength + subtermlength + 2
01353
01354      At 'where' is the number of the set and 'whichmember' is the
01355      number of the element. This is still a symbol/dollar and we
01356      have to find the substitution in the wildcards.
01357      The output is the subterm in which the setelements have been
01358      substituted. This is ready for further wildcard substitutions.
01359  */
01360
01361  WORD ResolveSet(PHEAD WORD *from, WORD *to, WORD *subs)
01362  {
01363      GETBIDENTITY
01364      WORD *m, *s, *w, j, i, ii, i3, flag, num;
01365      DOLLARS d = 0;
01366      #ifdef WITHPTHREADS
01367      int nummodopt, dtype = -1;
01368      #endif
01369      m = to; /* pointer in output */
01370      s = from + 2;
01371      w = s + s[1];
01372      while ( s < w ) *m++ = *s++;
01373      j = (from[1] - WORDDIF(w,from) ) » 1;
01374      m = subs + subs[1];
01375      subs += SUBEXPSIZE;
01376      s = subs;
01377      i = 0;
01378      while ( s < m ) { i++; s += s[1]; }
01379      m = to;
01380      if ( *m >= FUNCTION && functions[*m-FUNCTION].spec

```

```

01381     >= TENSORFUNCTION ) flag = 0;
01382     else flag = 1;
01383     while ( --j >= 0 ) {
01384         if ( w[l] >= 0 ) {
01385             s = subs;
01386             for ( ii = 0; ii < i; ii++ ) {
01387                 if ( *s == SYMTONUM && s[2] == w[l] ) { num = s[3]; goto GotOne; }
01388                 s += s[1];
01389             }
01390             MLOCK(ErrorMessageLock);
01391             MesPrint(" Unresolved setelement during substitution");
01392             MUNLOCK(ErrorMessageLock);
01393             return(-1);
01394         }
01395         else { /* Dollar ! */
01396             d = Dollars - w[l];
01397 #ifdef WITHPTHREADS
01398             if ( AS.MultiThreaded ) {
01399                 for ( nummodopt = 0; nummodopt < NumModOptdollars; nummodopt++ ) {
01400                     if ( -w[l] == ModOptdollars[nummodopt].number ) break;
01401                 }
01402                 if ( nummodopt < NumModOptdollars ) {
01403                     dtype = ModOptdollars[nummodopt].type;
01404                     if ( dtype == MODLOCAL ) {
01405                         d = ModOptdollars[nummodopt].dstruct+AT.identity;
01406                     }
01407                     else {
01408                         LOCK(d->pthreadslackread);
01409                     }
01410                 }
01411             }
01412 #endif
01413             if ( d->type == DOLNUMBER || d->type == DOLTERMS ) {
01414                 if ( d->where[0] == 4 && d->where[3] == 3 && d->where[2] == 1
01415                     && d->where[1] > 0 && d->where[4] == 0 ) {
01416                     num = d->where[1]; goto GotOne;
01417                 }
01418             }
01419             else if ( d->type == DOLINDEX ) {
01420                 if ( d->index > 0 && d->index < AM.OffsetIndex ) {
01421                     num = d->index; goto GotOne;
01422                 }
01423             }
01424             else if ( d->type == DOLARGUMENT ) {
01425                 if ( d->where[0] == -SNUMBER && d->where[1] > 0 ) {
01426                     num = d->where[1]; goto GotOne;
01427                 }
01428             }
01429             else if ( d->type == DOLWILDARGS ) {
01430                 if ( d->where[0] == 1 &&
01431                     d->where[1] > 0 && d->where[1] < AM.OffsetIndex ) {
01432                     num = d->where[1]; goto GotOne;
01433                 }
01434                 if ( d->where[0] == 0 && d->where[1] < 0 && d->where[3] == 0 ) {
01435                     if ( ( d->where[1] == -SNUMBER && d->where[2] > 0 )
01436                         || ( d->where[1] == -INDEX && d->where[2] > 0
01437                             && d->where[2] < AM.OffsetIndex ) ) {
01438                         num = d->where[2]; goto GotOne;
01439                     }
01440                 }
01441             }
01442 #ifdef WITHPTHREADS
01443             if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadslackread); }
01444 #endif
01445             MLOCK(ErrorMessageLock);
01446             MesPrint("Unusable type of variable %s in set substitution",
01447                 AC.dollarnames->namebuffer+d->name);
01448             MUNLOCK(ErrorMessageLock);
01449             return(-1);
01450         }
01451     GotOne:;
01452 #ifdef WITHPTHREADS
01453     if ( dtype > 0 && dtype != MODLOCAL ) { UNLOCK(d->pthreadslackread); }
01454 #endif
01455     ii = m[*w];
01456     if ( ii >= 2*MAXPOWER ) i3 = ii - 2*MAXPOWER;
01457     else if ( ii <= -2*MAXPOWER ) i3 = -ii - 2*MAXPOWER;
01458     else i3 = ( ii >= 0 ) ? ii: -ii - 1;
01459
01460     if ( num > ( Sets[i3].last - Sets[i3].first ) || num <= 0 ) {
01461         MLOCK(ErrorMessageLock);
01462         MesPrint("Array bound check during set substitution");
01463         MesPrint("    value is %d",num);
01464         MUNLOCK(ErrorMessageLock);
01465         return(-1);
01466     }
01467     m[*w] = (SetElements+Sets[i3].first)[num-1];

```

```

01468         if ( Sets[i3].type == CSYMBOL && m[*w] > MAXPOWER ) {
01469             if ( ii >= 2*MAXPOWER ) m[*w] -= 2*MAXPOWER;
01470             else if ( ii <= -2*MAXPOWER ) m[*w] = -(m[*w] - 2*MAXPOWER);
01471             else {
01472                 m[*w] -= MAXPOWER;
01473                 if ( m[*w] < MAXPOWER ) m[*w] -= 2*MAXPOWER;
01474                 if ( flag ) MakeDirty(m,m+*w,1);
01475             }
01476         }
01477         else if ( Sets[i3].type == CSYMBOL ) {
01478             if ( ii >= 2*MAXPOWER ) m[*w] += 2*MAXPOWER;
01479             else if ( ii <= -2*MAXPOWER ) m[*w] = -m[*w] - 2*MAXPOWER;
01480             else if ( ii < 0 ) m[*w] = - m[*w];
01481         }
01482         else if ( ii < 0 ) m[*w] = - m[*w];
01483         w += 2;
01484     }
01485     m = to;
01486     if ( *m >= FUNCTION && functions[*m-FUNCTION].spec
01487     >= TENSORFUNCTION ) {
01488         w = from + 2 + from[3];
01489         if ( *w == 0 ) { /* We had function -> tensor */
01490             m = from + 2 + FUNHEAD; s = to + FUNHEAD;
01491             while ( m < w ) {
01492                 if ( *m == -INDEX || *m == -VECTOR ) {}
01493                 else if ( *m == -ARGWILD ) { *s++ = FUNNYWILD; }
01494                 else {
01495                     MLOCK(ErrorMessageLock);
01496                     MesPrint("Illegal argument in tensor after set substitution");
01497                     MUNLOCK(ErrorMessageLock);
01498                     SETERROR(-1)
01499                 }
01500                 *s++ = m[1];
01501                 m += 2;
01502             }
01503             to[1] = WORDDIFF(s,to);
01504         }
01505     }
01506     return(0);
01507 }
01508
01509 /*
01510     #] ResolveSet :
01511     #[ ClearWild :          void ClearWild()
01512
01513     Clears the current wildcard settings and makes them ready for
01514     CheckWild and AddWild.
01515
01516 */
01517 void ClearWild(PHEAD0)
01518 {
01519     GETBIDENTITY
01520     WORD n, nn, *w;
01521     n = (AN.WildValue[-SUBEXPSIZE+1]-SUBEXPSIZE)/4; /* Number of wildcards */
01522     AN.NumWild = nn = n;
01523     if ( n > 0 ) {
01524         w = AT.WildMask;
01525         do { *w++ = 0; } while ( --n > 0 );
01526         w = AN.WildValue;
01527         do {
01528             if ( *w == SYMTONUM ) *w = SYMTOSYM;
01529             w += w[1];
01530         } while ( --nn > 0 );
01531     }
01532 }
01533
01534
01535 /*
01536     #] ClearWild :
01537     #[ AddWild :          WORD AddWild(oldnumber,type,newnumber)
01538
01539     Adds a wildcard assignment.
01540     Extra parameter in AN.argaddress;
01541
01542 */
01543 WORD AddWild(PHEAD WORD oldnumber, WORD type, WORD newnumber)
01544 {
01545     GETBIDENTITY
01546     WORD *w, *m, n, k, i = -1;
01547     CBUF *C = cbuf+AT.ebufnum;
01548     WORD eattensor = type & EATTENSOR;
01549     type = type & ~EATTENSOR;
01550     DEBUG(WORD *mm;)
01551     AN.WildReserve = 0;
01552     m = AT.WildMask;
01553     w = AN.WildValue;

```



```

01555     n = AN.NumWild;
01556     if ( n <= 0 ) { return(-1); }
01557     if ( type <= SYMTOSUB ) {
01558         do {
01559             if ( w[2] == oldnumber && *w <= SYMTOSUB ) {
01560                 if ( n > 1 && w[4] == SETTONUM ) i = w[7];
01561                 *w = type;
01562                 if ( *m != 2 ) *m = 1;
01563                 if ( type != SYMTOSUB ) {
01564                     if ( type == SYMTONUM ) AN.MaskPointer = m;
01565                     w[3] = newnumber;
01566                     goto FlipOn;
01567                 }
01568                 m = AddRHS(AT.ebufnum,1);
01569                 w[3] = C->numrhs;
01570                 w = AN.argaddress;
01571             DEBUG(mm = m; )
01572             n = *w - ARGHEAD;
01573             w += ARGHEAD;
01574             while ( (m + n + 10) > C->Top ) m = DoubleCbuffer(AT.ebufnum,m,4);
01575             while ( --n >= 0 ) *m++ = *w++;
01576             *m++ = 0;
01577             C->rhs[C->numrhs+1] = m;
01578             DEBUG(MesPrint("Thread %w(d): m=(%d,%d,%d,%d) (%d)",mm[0],mm[1],mm[2],mm[3],C->numrhs); )
01579             C->Pointer = m;
01580             goto FlipOn;
01581         }
01582         m++; w += w[1];
01583     } while ( --n > 0 );
01584 }
01585 else if ( type == ARGTOARG ) {
01586     do {
01587         if ( w[2] == oldnumber && *w == ARGTOARG ) {
01588             *m = 1;
01589             m = AddRHS(AT.ebufnum,1);
01590             w[3] = C->numrhs;
01591             w = AN.argaddress;
01592             DEBUG(mm=m; )
01593             if ( eattensor ) {
01594                 n = newnumber;
01595                 *m++ = n;
01596                 w = AN.argaddress;
01597             }
01598             else {
01599                 while ( --newnumber >= 0 ) { NEXTARG(w) }
01600                 n = WORDDIF(w,AN.argaddress);
01601                 w = AN.argaddress;
01602                 *m++ = 0;
01603             }
01604             while ( (m + n + 10) > C->Top ) m = DoubleCbuffer(AT.ebufnum,m,5);
01605             DEBUG(if ( mm != m-1 ) MesPrint("Thread %w(e): Alarm!"); mm = m-1; )
01606             while ( --n >= 0 ) *m++ = *w++;
01607             *m++ = 0;
01608             C->rhs[C->numrhs+1] = m;
01609             C->Pointer = m;
01610             DEBUG(MesPrint("Thread %w(e): w=(%d,%d,%d,%d) (%d)",mm[0],mm[1],mm[2],mm[3],C->numrhs); )
01611             return(0);
01612         }
01613         m++; w += w[1];
01614     } while ( --n > 0 );
01615 }
01616 else if ( type == ARLTOARL ) {
01617     do {
01618         if ( w[2] == oldnumber && *w == ARGTOARG ) {
01619             WORD **a;
01620             *m = 1;
01621             m = AddRHS(AT.ebufnum,1);
01622             w[3] = C->numrhs;
01623             DEBUG(mm=m; )
01624             a = (WORD **)(AN.argaddress); n = 0; k = newnumber;
01625             while ( --newnumber >= 0 ) {
01626                 w = *a++;
01627                 if ( *w > 0 ) n += *w;
01628                 else if ( *w <= -FUNCTION ) n++;
01629                 else n += 2;
01630             }
01631             *m++ = 0;
01632             while ( (m + n + 10) > C->Top ) m = DoubleCbuffer(AT.ebufnum,m,6);
01633             DEBUG(if ( mm != m-1 ) MesPrint("Thread %w(f): Alarm!"); mm = m-1; )
01634             a = (WORD **)(AN.argaddress);
01635             while ( --k >= 0 ) {
01636                 w = *a++;
01637                 if ( *w > 0 ) { n = *w; NCOPY(m,w,n); }
01638                 else if ( *w <= -FUNCTION ) *m++ = *w++;
01639                 else { *m++ = *w++; *m++ = *w++; }
01640             }
01641             *m++ = 0;

```

```

01642         C->rhs[C->numrhs+1] = m;
01643 DEBUG(MesPrint("Thread %w(f): w=(%d,%d,%d,%d) (%d) ",mm[0],mm[1],mm[2],mm[3],C->numrhs);)
01644         C->Pointer = m;
01645         return(0);
01646     }
01647     m++; w += w[1];
01648     } while ( --n > 0 );
01649 }
01650 else if ( type == VECTOSUB || type == INDTOSUB ) {
01651     WORD *ss, *sstop, *tt, *ttstop, j, *v1, *v2 = 0;
01652     do {
01653         if ( w[2] == oldnumber && ( *w == type ||
01654             ( type == VECTOSUB && ( *w == VECTOVEC || *w == VECTOMIN ) )
01655             || ( type == INDTOIND && *w == INDTOIND ) ) ) {
01656             if ( n > 1 && w[4] == SETTONUM ) i = w[7];
01657             *w = type;
01658             *m = 1;
01659             m = AddRHS(AT.ebufnum,1);
01660             w[3] = C->numrhs;
01661             w = AN.argaddress;
01662             n = *w - ARGHEAD;
01663             w += ARGHEAD;
01664             while ( (m + n + 10) > C->Top ) m = DoubleCbuffer(AT.ebufnum,m,7);
01665             while ( --n >= 0 ) *m++ = *w++;
01666             *m++ = 0;
01667             C->rhs[C->numrhs+1] = m;
01668             C->Pointer = m;
01669             m = AddRHS(AT.ebufnum,1);
01670             w = AN.argaddress;
01671             n = *w - ARGHEAD;
01672             w += ARGHEAD;
01673             while ( (m + n + 10) > C->Top ) m = DoubleCbuffer(AT.ebufnum,m,8);
01674             sstop = w + n;
01675             while ( w < sstop ) { /* Run over terms */
01676                 tt = w + *w; ttstop = tt - ABS(tt[-1]);
01677                 ss = m; m++; w++;
01678                 while ( w < ttstop ) { /* Subterms */
01679                     if ( *w != INDEX ) {
01680                         j = w[1];
01681                         NCOPY(m,w,j);
01682                     }
01683                     else {
01684                         v1 = m;
01685                         *m++ = *w++;
01686                         *m++ = j = *w++;
01687                         j -= 2;
01688                         while ( --j >= 0 ) {
01689                             if ( *w >= MINSPEC ) *m++ = *w++;
01690                             else v2 = w++;
01691                         }
01692                         j = WORDDIFF(m,v1);
01693                         if ( j != v1[1] ) {
01694                             if ( j <= 2 ) m -= 2;
01695                             else v1[1] = j;
01696                             *m++ = VECTOR;
01697                             *m++ = 4;
01698                             *m++ = *v2;
01699                             *m++ = FUNNYVEC;
01700                         }
01701                     }
01702                 }
01703                 while ( w < tt ) *m++ = *w++;
01704                 *ss = WORDDIFF(m,ss);
01705             }
01706             *m++ = 0;
01707             C->rhs[C->numrhs+1] = m;
01708             C->Pointer = m;
01709             if ( m > C->Top ) {
01710                 MLOCK(ErrorMessageLock);
01711                 MesPrint("Internal problems with extra compiler buffer");
01712                 MUNLOCK(ErrorMessageLock);
01713                 Terminate(-1);
01714             }
01715             goto FlipOn;
01716         }
01717         m++; w += w[1];
01718     } while ( --n > 0 );
01719 }
01720 else {
01721     do {
01722         if ( w[2] == oldnumber && ( *w == type || ( type == VECTOVEC
01723             && ( *w == VECTOMIN || *w == VECTOSUB ) ) || ( type == VECTOMIN
01724             && ( *w == VECTOVEC || *w == VECTOSUB ) )
01725             || ( type == INDTOIND && *w == INDTOIND ) ) ) {
01726             if ( n > 1 && w[4] == SETTONUM ) i = w[7];
01727             *w = type;
01728             w[3] = newnumber;

```

```

01729         *m = 1;
01730         goto FlipOn;
01731     }
01732     m++; w += w[1];
01733     } while ( --n > 0 );
01734 }
01735 MLOCK(ErrorMessageLock);
01736 MesPrint("Bug in AddWild.");
01737 MUNLOCK(ErrorMessageLock);
01738 return(-1);
01739 FlipOn:
01740 if ( i >= 0 ) {
01741     m = AT.WildMask;
01742     w = AN.WildValue;
01743     n = AN.NumWild;
01744     while ( --n >= 0 ) {
01745         if ( w[2] == i && *w == SYMTONUM ) {
01746             *m = 2;
01747             return(0);
01748         }
01749         m++; w += w[1];
01750     }
01751     MLOCK(ErrorMessageLock);
01752     MesPrint(" Bug in AddWild with passing set[i]");
01753     MUNLOCK(ErrorMessageLock);
01754     /*
01755      For the moment we want to crash here. That is easier with debugging.
01756     */
01757     #ifdef WITHPTHREADS
01758     { WORD *s = 0;
01759       *s++ = 1;
01760     }
01761     #endif
01762     Terminate(-1);
01763 }
01764 return(0);
01765 }
01766
01767 /*
01768 #] AddWild :
01769 #[ CheckWild :      WORD CheckWild(oldnumber,type,newnumber,newval)
01770
01771 Tests whether a wildcard assignment is allowed.
01772 A return value of zero means that it is allowed (nihil obstat).
01773 If the variable has been assigned already its existing
01774 assignment is returned in AN.oldvalue and AN.oldtype, which are
01775 global variables.
01776
01777 Note the special problem with name?set[i]. Here we have to pass
01778 an extra assignment. This cannot be done via globals as we
01779 call CheckWild sometimes twice before calling AddWild.
01780 Trick: Check the assignment of the number and if OK put it
01781 in place, but don't alter the used flag (if needed).
01782 Then AddWild can alter the used flag but the value is there.
01783 As long as this trick is 'hanging' we turn on the flag:
01784 'AN.WildReserve' which is either turned off by AddWild or by
01785 a failing call to CheckWild.
01786
01787 With ARGTOARG the tensors give the number of arguments
01788 or-ed with EATTENSOR which is at least 8192.
01789 */
01790
01791 WORD CheckWild(PHEAD WORD oldnumber, WORD type, WORD newnumber, WORD *newval)
01792 {
01793     GETBIDENTITY
01794     WORD *w, *m, *s, n, old2, inset;
01795     WORD n2, oldval, dirty, i, j, notflag = 0, retblock = 0;
01796     CBUF *C = cbuf+AT.ebufnum;
01797     WORD eattensor = type & EATTENSOR;
01798     type = type & ~EATTENSOR;
01799     m = AT.WildMask;
01800     w = AN.WildValue;
01801     n = AN.NumWild;
01802     if ( n <= 0 ) { AN.oldtype = -1; AN.WildReserve = 0; return(-1); }
01803     switch ( type ) {
01804     case SYMTONUM :
01805         *newval = newnumber;
01806         do {
01807             if ( w[2] == oldnumber && *w <= SYMTOSUB ) {
01808                 old2 = *w;
01809                 if ( !*m ) goto TestSet;
01810                 AN.MaskPointer = m;
01811                 if ( *w == SYMTONUM && w[3] == newnumber ) {
01812                     return(0);
01813                 }
01814                 AN.oldtype = old2; AN.oldvalue = w[3]; goto NoMatch;
01815             }

```

```

01816         m++; w += w[1];
01817     } while ( --n > 0 );
01818     break;
01819 case SYMTOSYM :
01820     *newval = newnumber;
01821     do {
01822         if ( w[2] == oldnumber && *w <= SYMTOSUB ) {
01823             old2 = *w;
01824             if ( *w == SYMTOSYM ) {
01825                 if ( !*m ) goto TestSet;
01826                 if ( newnumber >= 0 && (w+4) < AN.WildStop
01827                     && ( w[4] == FROMSET || w[4] == SETTONUM )
01828                     && w[7] >= 0 ) goto TestSet;
01829                 if ( w[3] == newnumber ) return(0);
01830             }
01831             else {
01832                 if ( !*m ) goto TestSet;
01833             }
01834             goto NoM;
01835         }
01836         m++; w += w[1];
01837     } while ( --n > 0 );
01838     break;
01839 case SYMTOSUB :
01840 /*
01841     Now newval contains the pointer to the argument.
01842 */
01843 {
01844 /*
01845     Search for vector or index nature. If so: reject.
01846 */
01847     WORD *ss, *sstop, *tt, *ttstop;
01848     ss = newval;
01849     sstop = ss + *ss;
01850     ss += ARGHEAD;
01851     while ( ss < sstop ) {
01852         tt = ss + *ss;
01853         ttstop = tt - ABS(tt[-1]);
01854         ss++;
01855         while ( ss < ttstop ) {
01856             if ( *ss == INDEX ) goto NoMatch;
01857             ss += ss[1];
01858         }
01859         ss = tt;
01860     }
01861 }
01862 do {
01863     if ( w[2] == oldnumber && *w <= SYMTOSUB ) {
01864         old2 = *w;
01865         if ( *w == SYMTONUM || *w == SYMTOSYM ) {
01866             if ( !*m ) {
01867                 s = w + w[1];
01868                 if ( s >= AN.WildStop || *s != SETTONUM )
01869                     goto TestSet;
01870             }
01871         }
01872         else if ( *w == SYMTOSUB ) {
01873             if ( !*m ) {
01874                 s = w + w[1];
01875                 if ( s >= AN.WildStop || *s != SETTONUM )
01876                     goto TestSet;
01877             }
01878             n = *newval - 2;
01879             newval += 2;
01880             m = C->rhs[w[3]];
01881             if ( (C->rhs[w[3]+1] - m - 1) == n ) {
01882                 while ( n > 0 ) {
01883                     if ( *m != *newval ) {
01884                         m++; newval++; break;
01885                     }
01886                     m++; newval++;
01887                     n--;
01888                 }
01889                 if ( n <= 0 ) return(0);
01890             }
01891         }
01892         AN.oldtype = old2; AN.oldvalue = w[3]; goto NoMatch;
01893     }
01894     m++; w += w[1];
01895 } while ( --n > 0 );
01896 break;
01897 case ARGTOARG :
01898 do {
01899     if ( w[2] == oldnumber && *w == ARGTOARG ) {
01900         if ( !*m ) return(0); /* nihil obstat */
01901         m = C->rhs[w[3]];
01902         if ( eattensor ) {

```

```

01903         n = newnumber;
01904         if ( *m != 0 ) {
01905             if ( n == *m ) {
01906                 m++;
01907                 while ( --n >= 0 ) {
01908                     if ( *m != *newval ) {
01909                         m++; newval++; break;
01910                     }
01911                     m++; newval++;
01912                 }
01913                 if ( n < 0 ) return(0);
01914             }
01915         }
01916         else {
01917             m++;
01918             while ( --n >= 0 ) {
01919                 if ( *newval != m[1] || ( *m != -INDEX
01920                     && *m != -VECTOR && *m != -SNUMBER ) ) break;
01921                 m += 2;
01922                 newval++;
01923             }
01924             if ( n < 0 && *m == 0 ) return(0);
01925         }
01926     }
01927     else {
01928         i = newnumber;
01929         if ( *m != 0 ) { /* Tensor field */
01930             if ( *m == i ) {
01931                 m++;
01932                 while ( --i >= 0 ) {
01933                     if ( *m != newval[1]
01934                         || ( *newval != -VECTOR
01935                             && *newval != -INDEX
01936                             && *newval != -SNUMBER ) ) break;
01937                     newval += 2;
01938                     m++;
01939                 }
01940                 if ( i < 0 ) return(0);
01941             }
01942         }
01943         else {
01944             m++;
01945             s = newval;
01946             while ( --i >= 0 ) { NEXTARG(s) }
01947             n = WORDDIFF(s,newval);
01948             while ( --n >= 0 ) {
01949                 if ( *m != *newval ) {
01950                     m++; newval++; break;
01951                 }
01952                 m++; newval++;
01953             }
01954             if ( n < 0 && *m == 0 ) return(0);
01955         }
01956     }
01957     AN.oldtype = *w; AN.oldvalue = w[3]; goto NoMatch;
01958 }
01959 m++; w += w[1];
01960 } while ( --n > 0 );
01961 break;
01962 case ARLTOARL :
01963     do {
01964         if ( w[2] == oldnumber && *w == ARGTOARG ) {
01965             WORD **a;
01966             if ( !*m ) return(0); /* nihil obstat */
01967             m = C->rhs[w[3]];
01968             i = newnumber;
01969             a = (WORD **)newval;
01970             if ( *m != 0 ) { /* Tensor field */
01971                 if ( *m == i ) {
01972                     m++;
01973                     while ( --i >= 0 ) {
01974                         s = *a++;
01975                         if ( *m != s[1]
01976                             || ( *s != -VECTOR
01977                                 && *s != -INDEX
01978                                 && *s != -SNUMBER ) ) break;
01979                         m++;
01980                     }
01981                     if ( i < 0 ) return(0);
01982                 }
01983             }
01984         }
01985         else {
01986             m++;
01987             while ( --i >= 0 ) {
01988                 s = *a++;
01989                 if ( *s > 0 ) {

```

```

01990             while ( --n >= 0 ) {
01991                 if ( *s != *m ) {
01992                     s++; m++; break;
01993                 }
01994                 s++; m++;
01995             }
01996             if ( n >= 0 ) break;
01997         }
01998         else if ( *s <= -FUNCTION ) {
01999             if ( *s != *m ) {
02000                 s++; m++; break;
02001             }
02002             s++; m++;
02003         }
02004         else {
02005             if ( *s != *m ) {
02006                 s++; m++; break;
02007             }
02008             s++; m++;
02009             if ( *s != *m ) {
02010                 s++; m++; break;
02011             }
02012             s++; m++;
02013         }
02014     }
02015     if ( i < 0 && *m == 0 ) return(0);
02016 }
02017 AN.oldtype = *w; AN.oldvalue = w[3]; goto NoMatch;
02018 }
02019 m++; w += w[1];
02020 } while ( --n > 0 );
02021 break;
02022 case VECTOSUB :
02023 case INDTOSUB :
02024 /*
02025     Now newval contains the pointer to the argument(s).
02026 */
02027 {
02028 /*
02029     Search for vector or index nature. If not so: reject.
02030 */
02031 WORD *ss, *sstop, *tt, *ttstop, count, jt;
02032 ss = newval;
02033 sstop = ss + *ss;
02034 ss += ARGHEAD;
02035 while ( ss < sstop ) {
02036     tt = ss + *ss;
02037     ttstop = tt - ABS(tt[-1]);
02038     ss++;
02039     count = 0;
02040     while ( ss < ttstop ) {
02041         if ( *ss == INDEX ) {
02042             jt = ss[1] - 2; ss += 2;
02043             while ( --jt >= 0 ) {
02044                 if ( *ss < MINSPEC ) count++;
02045                 ss++;
02046             }
02047         }
02048         else ss += ss[1];
02049     }
02050     if ( count != 1 ) goto NoMatch;
02051     ss = tt;
02052 }
02053 }
02054 do {
02055     if ( w[2] == oldnumber ) {
02056         old2 = *w;
02057         if ( ( type == VECTOSUB && ( *w == VECTOVEC || *w == VECTOMIN ) )
02058             || ( type == INDTOSUB && *w == INDTOIND ) ) {
02059             if ( !*m ) goto TestSet;
02060             AN.oldtype = old2; AN.oldvalue = w[3]; goto NoMatch;
02061         }
02062         else if ( *w == type ) {
02063             if ( !*m ) goto TestSet;
02064             if ( type != INDTOIND && type != INDTOSUB ) { /* Prevent double index */
02065                 n = *newval - 2;
02066                 newval += 2;
02067                 m = C->rhs[w[3]];
02068                 if ( (C->rhs[w[3]+1] - m - 1) == n ) {
02069                     while ( n > 0 ) {
02070                         if ( *m != *newval ) {
02071                             m++; newval++; break;
02072                         }
02073                         m++; newval++;
02074                         n--;
02075                     }
02076                     if ( n <= 0 ) return(0);

```

```

02077         }
02078     }
02079     AN.oldtype = old2; AN.oldvalue = w[3]; goto NoMatch;
02080 }
02081 }
02082     m++; w += w[1];
02083 } while ( --n > 0 );
02084 break;
02085 default :
02086     *newval = newnumber;
02087     do {
02088         if ( w[2] == oldnumber ) {
02089             if ( *w == type ) {
02090                 old2 = *w;
02091                 if ( !*m ) goto TestSet;
02092                 if ( newnumber >= 0 && (w+4) < AN.WildStop &&
02093                     ( w[4] == FROMSET || w[4] == SETTONUM )
02094                     && w[7] >= 0 ) goto TestSet;
02095                 if ( newnumber < 0 && *w == VECTOVEC
02096                     && (w+4) < AN.WildStop && ( w[4] == FROMSET
02097                     || w[4] == SETTONUM ) && w[7] >= 0 ) goto TestSet;
02098 /*
02099         The next statement kills multiple indices -> vector
02100 */
02101         if ( *w == INDTOIND && w[3] < 0 ) goto NoMatch;
02102         if ( w[3] == newnumber ) {
02103             if ( *w != FUNTOFUN || newnumber < FUNCTION
02104                 || functions[newnumber-FUNCTION].spec ==
02105                 functions[oldnumber-FUNCTION].spec )
02106                 return(0);
02107         }
02108         AN.oldtype = old2; AN.oldvalue = w[3]; goto NoMatch;
02109     }
02110     else if ( ( type == VECTOVEC &&
02111         ( *w == VECTOSUB || *w == VECTOMIN ) )
02112         || ( type == INDTOIND && *w == INDTOSUB ) ) {
02113         if ( *m ) goto NoMatch;
02114         old2 = *w;
02115         goto TestSet;
02116     }
02117     else if ( type == VECTOMIN &&
02118         ( *w == VECTOSUB || *w == VECTOVEC ) ) {
02119         if ( *m ) goto NoMatch;
02120         old2 = *w;
02121         goto TestSet;
02122     }
02123 }
02124     m++; w += w[1];
02125     if ( n > 1 && ( *w == FROMSET
02126         || *w == SETTONUM ) ) { n--; m++; w += w[1]; }
02127 } while ( --n > 0 );
02128 break;
02129 }
02130 AN.oldtype = -1;
02131 AN.oldvalue = -1;
02132 AN.WildReserve = 0;
02133 MLOCK(ErrorMessageLock);
02134 MesPrint("Inconsistency in Wildcard prototype.");
02135 MUNLOCK(ErrorMessageLock);
02136 return(-1);
02137 NoMatch:
02138     AN.WildReserve = 0;
02139     return(1+retblock);
02140 /*
02141     Here we test the compatibility with a set specification.
02142 */
02143 TestSet:
02144     dirty = *m;
02145     oldval = w[3];
02146     w += w[1];
02147     if ( w < AN.WildStop && ( *w == FROMSET || *w == SETTONUM ) ) {
02148         WORD k;
02149         s = w;
02150         j = w[2]; n2 = w[3];
02151 /*
02152         if SETTONUM: x?j[n2]
02153         if FROMSET:  x?j?n2    or x?j and n2 = -WOLDOFFSET.
02154 */
02155         if ( j > WILDOFFSET ) {
02156             j -= 2*WILDOFFSET;
02157             notflag = 1;
02158 /*
02159         ???????
02160 */
02161         AN.oldtype = -1;
02162         AN.oldvalue = -1;
02163     }

```

```

02164         if ( j < AM.NumFixedSets ) {      /* special set */
02165             retblock = 1;
02166             switch ( j ) {
02167                 case POS_:
02168                     if ( type != SYMTONUM ||
02169                         newnumber <= 0 ) goto NoMnot;
02170                     break;
02171                 case POS0_:
02172                     if ( type != SYMTONUM ||
02173                         newnumber < 0 ) goto NoMnot;
02174                     break;
02175                 case NEG_:
02176                     if ( type != SYMTONUM ||
02177                         newnumber >= 0 ) goto NoMnot;
02178                     break;
02179                 case NEG0_:
02180                     if ( type != SYMTONUM ||
02181                         newnumber > 0 ) goto NoMnot;
02182                     break;
02183                 case EVEN_:
02184                     if ( type != SYMTONUM ||
02185                         ( newnumber & 1 ) != 0 ) goto NoMnot;
02186                     break;
02187                 case ODD_:
02188                     if ( type != SYMTONUM ||
02189                         ( newnumber & 1 ) == 0 ) goto NoMnot;
02190                     break;
02191                 case Z_:
02192                     if ( type != SYMTONUM ) goto NoMnot;
02193                     break;
02194                 case SYMBOL_:
02195                     if ( type != SYMTOSYM ) goto NoMnot;
02196                     break;
02197                 case FIXED_:
02198                     if ( type != INDTOIND ||
02199                         newnumber >= AM.OffsetIndex ||
02200                         newnumber < 0 ) goto NoMnot;
02201                     break;
02202                 case INDEX_:
02203                     if ( type != INDTOIND ||
02204                         newnumber < 0 ) goto NoMnot;
02205                     break;
02206                 case Q_:
02207                     if ( type == SYMTONUM ) break;
02208                     if ( type == SYMTOSUB ) {
02209                         WORD *ss, *sstop;
02210                         ss = newval;
02211                         sstop = ss + *ss;
02212                         ss += ARGHEAD;
02213                         if ( ss >= sstop ) break;
02214                         if ( ss + *ss < sstop ) goto NoMnot;
02215                         if ( ABS(sstop[-1]) == ss[0]-1 ) break;
02216                     }
02217                     goto NoMnot;
02218                 case DUMMYINDEX_:
02219                     if ( type != INDTOIND ||
02220                         newnumber < AM.IndDum || newnumber >= AM.IndDum+MAXDUMMIES ) goto NoMnot;
02221                     break;
02222                 case VECTOR_:
02223                     if ( type != VECTOVEC ) goto NoMnot;
02224                     break;
02225                 default:
02226                     goto NoMnot;
02227             }
02228 Mnot:
02229             if ( notflag ) goto NoM;
02230             return(0);
02231 NoMnot:
02232             if ( !notflag ) goto NoM;
02233             return(0);
02234         }
02235     else if ( Sets[j].type == CRANGE ) {
02236         if ( ( type == SYMTONUM )
02237             || ( type == INDTOIND && ( newnumber > 0
02238                 && newnumber <= AM.OffsetIndex ) ) ) {
02239             if ( Sets[j].first < MAXPOWER ) {
02240                 if ( newnumber >= Sets[j].first ) goto NoMnot;
02241             }
02242             else if ( Sets[j].first < 3*MAXPOWER ) {
02243                 if ( newnumber+2*MAXPOWER > Sets[j].first ) goto NoMnot;
02244             }
02245             if ( Sets[j].last > -MAXPOWER ) {
02246                 if ( newnumber <= Sets[j].last ) goto NoMnot;
02247             }
02248             else if ( Sets[j].last > -3*MAXPOWER ) {
02249                 if ( newnumber-2*MAXPOWER < Sets[j].last ) goto NoMnot;
02250             }
02251         }

```



```

02251         goto Mnot;
02252     }
02253     goto NoMnot;
02254 }
02255 /*
02256 Now we have to determine which set element
02257 */
02258 w = SetElements + Sets[j].first;
02259 m = SetElements + Sets[j].last;
02260 if ( w == m ) {
02261     /*
02262         The set is empty! This is not a match unless we have !{}, in
02263         which case it is.
02264     */
02265     if ( notflag ) return 0;
02266     AN.oldtype = old2;
02267     AN.oldvalue = oldval;
02268     goto NoMatch;
02269 }
02270
02271 if ( ( Sets[j].flags & ORDEREDSET ) == ORDEREDSET ) {
02272     /*
02273         We search first and ask questions later
02274     */
02275     i = BinarySearch(w, Sets[j].last-Sets[j].first, newnumber);
02276     if ( i < 0 ) { /* no matter what, it is not in the set. */
02277         goto NoMnot;
02278     }
02279     else {
02280         /*
02281             We can set the proper parameters now to make only the
02282             checks for the given set element.
02283             After that we jump into the appropriate loop.
02284         */
02285         w = m = SetElements + i;
02286         i++;
02287         if ( Sets[j].type == -1 || Sets[j].type == CNUMBER ) {
02288             goto insideloop1;
02289         }
02290         else {
02291             goto insideloop2;
02292         }
02293     }
02294 }
02295 i = 1;
02296 if ( Sets[j].type == -1 || Sets[j].type == CNUMBER ) {
02297     do {
02298         insideloop1:
02299         if ( notflag ) {
02300             switch ( type ) {
02301                 case SYMTOSYM:
02302                     if ( Sets[j].type == CNUMBER ) {}
02303                     else {
02304                         if ( *w == newnumber ) goto NoMatch;
02305                     }
02306                     break;
02307                 case SYMTONUM:
02308                 case INDTOIND:
02309                     if ( *w == newnumber ) goto NoMatch;
02310                     break;
02311                 default:
02312                     break;
02313             }
02314         }
02315         else if ( type != SYMTONUM && type != INDTOIND
02316             && type != SYMTOSYM ) goto NoMatch;
02317         else if ( type == SYMTOSYM && Sets[j].type == CNUMBER ) goto NoMatch;
02318         else if ( *w == newnumber ) {
02319             if ( *s == SETTONUM ) {
02320                 if ( n2 == oldnumber && type
02321                     <= SYMTOSUB ) goto NoMatch;
02322                 m = AT.WildMask;
02323                 w = AN.WildValue;
02324                 n = AN.NumWild;
02325                 while ( --n >= 0 ) {
02326                     if ( w[2] == n2 && *w <= SYMTOSUB ) {
02327                         if ( !*m ) {
02328                             *w = SYMTONUM;
02329                             w[3] = i;
02330                             AN.WildReserve = 1;
02331                             return(0);
02332                         }
02333                         if ( *w != SYMTONUM )
02334                             goto NoMatch;
02335                         if ( w[3] == i ) return(0);
02336                         i = w[3];
02337                         j = (SetElements + Sets[j].first)[i];

```

```

02338             if ( j == n2 ) return(0);
02339             goto NoMatch;
02340         }
02341         m++; w += w[1];
02342     }
02343 }
02344 else if ( n2 >= 0 ) {
02345     *newval = *(w - Sets[j].first + Sets[n2].first);
02346     if ( *newval > MAXPOWER ) *newval -= 2*MAXPOWER;
02347     if ( dirty && *newval != oldval ) {
02348         *newval = oldval; goto NoMatch;
02349     }
02350 }
02351 return(0);
02352 }
02353 i++;
02354 } while ( ++w < m );
02355 }
02356 else {
02357     do {
02358         insideloop2:
02359         inset = *w;
02360         if ( notflag ) {
02361             switch ( type ) {
02362                 case SYMTONUM:
02363                 case SYMTOSYM:
02364                     if ( ( type == SYMTOSYM && *w == newnumber )
02365                         || ( type == SYMTONUM && *w-2*MAXPOWER == newnumber ) ) {
02366                         goto NoMatch;
02367                     }
02368                     /* fall through */
02369                 case SYMTOSUB:
02370                     if ( *w < 0 ) {
02371                         WORD *mm = AT.WildMask, *mmm, *part;
02372                         WORD *ww = AN.WildValue;
02373                         WORD nn = AN.NumWild;
02374                         k = -*w;
02375                         while ( --nn >= 0 ) {
02376                             if ( *mm && ww[2] == k && ww[0] == type ) {
02377                                 if ( type != SYMTOSUB ) {
02378                                     if ( ww[3] == newnumber ) goto NoMatch;
02379                                 }
02380                                 else {
02381                                     mmm = C->rhs[ww[3]];
02382                                     nn = *newval-2;
02383                                     part = newval+2;
02384                                     if ( (C->rhs[ww[3]+1]-mmm-1) == nn ) {
02385                                         while ( --nn >= 0 ) {
02386                                             if ( *mmm != *part ) {
02387                                                 mmm++; part++; break;
02388                                             }
02389                                             mmm++; part++;
02390                                         }
02391                                         if ( nn < 0 ) goto NoMatch;
02392                                     }
02393                                 }
02394                                 break;
02395                             }
02396                             mm++; ww += ww[1];
02397                         }
02398                     }
02399                     break;
02400                 case VECTOMIN:
02401                     if ( type == VECTOMIN ) {
02402                         if ( inset >= AM.OffsetVector ) { i++; continue; }
02403                         inset += WILDMASK;
02404                     }
02405                     /* fall through */
02406                 case VECTOVEC:
02407                     if ( inset == newnumber ) goto NoMatch;
02408                     /* fall through */
02409                 case VECTOSUB:
02410                     if ( inset - WILDOFFSET >= AM.OffsetVector ) {
02411                         WORD *mm = AT.WildMask, *mmm, *part;
02412                         WORD *ww = AN.WildValue;
02413                         WORD nn = AN.NumWild;
02414                         k = inset - WILDOFFSET;
02415                         while ( --nn >= 0 ) {
02416                             if ( *mm && ww[2] == k && ww[0] == type ) {
02417                                 if ( type == VECTOVEC ) {
02418                                     if ( ww[3] == newnumber ) goto NoMatch;
02419                                 }
02420                                 else {
02421                                     mmm = C->rhs[ww[3]];
02422                                     nn = *newval-2;
02423                                     part = newval+2;
02424                                     if ( (C->rhs[ww[3]+1]-mmm-1) == nn ) {

```

```

02425             while ( --nn >= 0 ) {
02426                 if ( *mmm != *part ) {
02427                     mmm++; part++; break;
02428                 }
02429                 mmm++; part++;
02430             }
02431             if ( nn < 0 ) goto NoMatch;
02432         }
02433     }
02434     break;
02435 }
02436 mm++; ww += ww[1];
02437 }
02438 }
02439 break;
02440 case INDTOIND:
02441     if ( *w == newnumber ) goto NoMatch;
02442     /* fall through */
02443 case INTOSUB:
02444     if ( *w - (WORD)WILDMASK >= AM.OffsetIndex ) {
02445         WORD *mm = AT.WildMask, *mmm, *part;
02446         WORD *ww = AN.WildValue;
02447         WORD nn = AN.NumWild;
02448         k = *w - WILDMASK;
02449         while ( --nn >= 0 ) {
02450             if ( *mm && ww[2] == k && ww[0] == type ) {
02451                 if ( type == INDTOIND ) {
02452                     if ( ww[3] == newnumber ) goto NoMatch;
02453                 }
02454                 else {
02455                     mmm = C->rhs[ww[3]];
02456                     nn = *newval-2;
02457                     part = newval+2;
02458                     if ( (C->rhs[ww[3]+1]-mmm-1) == nn ) {
02459                         while ( --nn >= 0 ) {
02460                             if ( *mmm != *part ) {
02461                                 mmm++; part++; break;
02462                             }
02463                             mmm++; part++;
02464                         }
02465                         if ( nn < 0 ) goto NoMatch;
02466                     }
02467                 }
02468                 break;
02469             }
02470             mm++; ww += ww[1];
02471         }
02472     }
02473     break;
02474 case FUNTOFUN:
02475     if ( *w == newnumber ) goto NoMatch;
02476     if ( ( type == FUNTOFUN &&
02477           ( k = *w - WILDMASK ) > FUNCTION ) ) {
02478         WORD *mm = AT.WildMask;
02479         WORD *ww = AN.WildValue;
02480         WORD nn = AN.NumWild;
02481         while ( --nn >= 0 ) {
02482             if ( *mm && ww[2] == k && ww[0] == type ) {
02483                 if ( ww[3] == newnumber ) goto NoMatch;
02484                 break;
02485             }
02486             mm++; ww += ww[1];
02487         }
02488     }
02489     default:
02490         break;
02491 }
02492 }
02493 else {
02494     if ( type == VECTOMIN ) {
02495         if ( inset >= AM.OffsetVector ) { i++; continue; }
02496         inset += WILDMASK;
02497     }
02498     if ( ( inset == newnumber && type != SYMTONUM ) ||
02499         ( type == SYMTONUM && inset-2*MAXPOWER == newnumber ) ) {
02500         if ( *s == SETTONUM ) {
02501             if ( n2 == oldnumber && type
02502                 <= SYMTOSUB ) goto NoMatch;
02503             m = AT.WildMask;
02504             w = AN.WildValue;
02505             n = AN.NumWild;
02506             while ( --n >= 0 ) {
02507                 if ( w[2] == n2 && *w <= SYMTOSUB ) {
02508                     if ( !*m ) {
02509                         *w = SYMTONUM;
02510                         w[3] = i;
02511                         AN.WildReserve = 1;

```

```

02512             return(0);
02513         }
02514         if ( *w != SYMTONUM )
02515             goto NoMatch;
02516         if ( w[3] == i ) return(0);
02517         i = w[3];
02518         j = (SetElements + Sets[j].first)[i];
02519         if ( j == n2 ) return(0);
02520         goto NoMatch;
02521     }
02522     m++; w += w[1];
02523 }
02524 }
02525 else if ( n2 >= 0 ) {
02526     *newval = *(w - Sets[j].first + Sets[n2].first);
02527     if ( *newval > MAXPOWER ) *newval -= 2*MAXPOWER;
02528     if ( dirty && *newval != oldval ) {
02529         *newval = oldval; goto NoMatch;
02530     }
02531 }
02532 return(0);
02533 }
02534 }
02535 i++;
02536 } while ( ++w < m );
02537 }
02538 if ( notflag ) return(0);
02539 AN.oldtype = old2; AN.oldvalue = oldval; goto NoMatch;
02540 }
02541 else { return(0); }
02542
02543 NoM:
02544 AN.oldtype = old2; AN.oldvalue = w[3]; goto NoMatch;
02545 }
02546
02547 /*
02548     #] CheckWild :
02549     #] Wildcards :
02550     #[ DenToFunction :
02551
02552     Renames the denominator function into a function with the given number.
02553     For the syntax see     Denominators,function;
02554 */
02555
02556 int DenToFunction(WORD *term, WORD numfun)
02557 {
02558     int action = 0;
02559     WORD *t, *tstop, *tnext, *arg, *argstop, *targ;
02560     t = term+1;
02561     tstop = term + *term; tstop -= ABS(tstop[-1]);
02562     while ( t < tstop ) {
02563         if ( *t == DENOMINATOR ) {
02564             *t = numfun; t[2] |= DIRTYFLAG; action = 1;
02565         }
02566         tnext = t + t[1];
02567         if ( *t >= FUNCTION && functions[*t-FUNCTION].spec <= 0 ) {
02568             arg = t + FUNHEAD;
02569             while ( arg < tnext ) {
02570                 if ( *arg > 0 ) {
02571                     targ = arg + ARGHEAD; argstop = arg + *arg;
02572                     while ( targ < argstop ) {
02573                         if ( DenToFunction(targ,numfun) ) {
02574                             arg[1] |= DIRTYFLAG; t[2] |= DIRTYFLAG; action = 1;
02575                         }
02576                         targ += *targ;
02577                     }
02578                     arg = argstop;
02579                 }
02580                 else if ( *arg <= -FUNCTION ) arg++;
02581                 else arg += 2;
02582             }
02583         }
02584         t = tnext;
02585     }
02586     return(action);
02587 }
02588
02589 /*
02590     #] DenToFunction :
02591 */

```

Index

- `_CHECK`
 - `parallel.c`, [2067](#)
 - `__CHECK`
 - `parallel.c`, [2068](#)
 - `~AEdge`
 - `AEdge`, [8](#)
 - `~ANode`
 - `ANode`, [12](#)
 - `~AStack`
 - `AStack`, [27](#)
 - `~Assign`
 - `Assign`, [17](#)
 - `~ECand`
 - `ECand`, [94](#)
 - `~EEdge`
 - `EEdge`, [96](#)
 - `~EGraph`
 - `EGraph`, [104](#)
 - `~ENode`
 - `ENode`, [115](#)
 - `~EStack`
 - `EStack`, [119](#)
 - `~Interaction`
 - `Interaction`, [160](#)
 - `~MCBlock`
 - `MCBlock`, [193](#)
 - `~MCBridge`
 - `MCBridge`, [195](#)
 - `~MCOpi`
 - `MCOpi`, [203](#)
 - `~MConn`
 - `MConn`, [197](#)
 - `~MGraph`
 - `MGraph`, [207](#)
 - `~MNodeClass`
 - `MNodeClass`, [221](#)
 - `~MOrbits`
 - `MOrbits`, [237](#)
 - `~Model`
 - `Model`, [226](#)
 - `~NCand`
 - `NCand`, [266](#)
 - `~NStack`
 - `NStack`, [275](#)
 - `~Options`
 - `Options`, [297](#)
 - `~Output`
 - `Output`, [304](#)
 - `~PNodeClass`
 - `PNodeClass`, [335](#)
 - `~Particle`
 - `Particle`, [323](#)
 - `~Process`
 - `Process`, [364](#)
 - `~SGroup`
 - `SGroup`, [386](#)
 - `~SProcess`
 - `SProcess`, [404](#)
 - `~T_EGraph`
 - `T_EGraph`, [442](#)
 - `~T_MGraph`
 - `T_MGraph`, [447](#)
 - `~T_MNodeClass`
 - `T_MNodeClass`, [453](#)
 - `~fmpz`
 - `flint::fmpz`, [138](#)
 - `~mpoly`
 - `flint::mpoly`, [240](#)
 - `~mpoly_ctx`
 - `flint::mpoly_ctx`, [241](#)
 - `~mpoly_factor`
 - `flint::mpoly_factor`, [242](#)
 - `~poly`
 - `flint::poly`, [338](#)
 - `poly`, [341](#)
 - `~poly_factor`
 - `flint::poly_factor`, [357](#)
- `A`
 - `inivar.h`, [1683](#)
 - `variable.h`, [3212](#)
- `a`
 - `MODNUM`, [235](#)
- `a3`
 - `TrAcEs`, [477](#)
- `a4`
 - `TrAcEs`, [477](#)
- `ABS`
 - `declare.h`, [800](#)
- `ABSFUNCTIO`
 - `ftypes.h`, [1396](#)
- `AC`
 - `variable.h`, [3210](#)
- `accu`
 - `TrAcEn`, [471](#)
 - `TrAcEs`, [474](#)
- `AccumGCD`
 - `declare.h`, [824](#)
 - `reken.c`, [2557](#)

- accup
 - TrAcEn, [471](#)
 - TrAcEs, [474](#)
- acode
 - Particle, [326](#)
 - PInput, [333](#)
- ACOSFUNCTION
 - ftypes.h, [1400](#)
- ACOSHFUNCTION
 - ftypes.h, [1401](#)
- active
 - FiLe, [130](#)
 - PF_BUFFER, [330](#)
- activenames
 - C_const, [46](#)
- ad
 - TrAcEs, [476](#)
- add
 - COST, [76](#)
 - Fraction, [140](#), [141](#)
 - poly, [349](#)
- Add2Com
 - declare.h, [814](#)
- Add2ComStrings
 - compcomm.c, [617](#)
 - declare.h, [927](#)
- ADD2POS
 - declare.h, [812](#)
- Add3Com
 - declare.h, [815](#)
- Add4Com
 - declare.h, [815](#)
- Add5Com
 - declare.h, [815](#)
- add_factor
 - factorized_poly, [126](#)
- ADDARG
 - ftypes.h, [1448](#)
- AddArgs
 - declare.h, [824](#)
 - sort.c, [2726](#)
- AddArrayIndex
 - declare.h, [822](#)
 - sch.c, [2657](#)
- addArtic
 - MConn, [198](#)
- addBlock
 - MConn, [198](#)
- addBlockSelf
 - MConn, [199](#)
- addBridge
 - MConn, [198](#)
- AddCoef
 - declare.h, [824](#)
 - sort.c, [2724](#)
- AddComString
 - compcomm.c, [617](#)
 - declare.h, [956](#)
- AddDictionary
 - declare.h, [1020](#)
 - dict.c, [1076](#)
- AddDollar
 - declare.h, [891](#)
 - names.c, [1835](#)
- AddDubious
 - declare.h, [892](#)
 - names.c, [1835](#)
- addEdge
 - Assign, [18](#)
- AddExpression
 - declare.h, [891](#)
 - names.c, [1836](#)
- addFLine
 - EGraph, [108](#)
- AddFloats
 - float.c, [1287](#)
- AddFunction
 - declare.h, [892](#)
 - names.c, [1832](#)
- addGroup
 - SGroup, [387](#)
- AddIndex
 - declare.h, [892](#)
 - names.c, [1831](#)
- addInteraction
 - Model, [227](#)
- addInteractionEnd
 - Model, [227](#)
- AddLHS
 - comtool.c, [764](#)
 - declare.h, [924](#)
- AddLineFeed
 - declare.h, [802](#)
- AddLong
 - declare.h, [825](#)
 - reken.c, [2558](#)
- AddName
 - declare.h, [888](#)
 - names.c, [1827](#)
- AddNtoC
 - comtool.c, [766](#)
 - declare.h, [926](#)
- AddNtoL
 - comtool.c, [765](#)
 - declare.h, [925](#)
- AddObject
 - declare.h, [986](#)
 - minos.c, [1734](#)
- addOPic
 - MConn, [198](#)
- addParticle
 - Model, [226](#)
- addParticleEnd
 - Model, [226](#)
- AddPLon
 - declare.h, [825](#)

- `reken.c`, 2558
- `AddPoly`
 - `declare.h`, 825
 - `sort.c`, 2725
- `ADDPOS`
 - `declare.h`, 811
- `AddPotModdollar`
 - `declare.h`, 967
 - `dollar.c`, 1100
- `AddRat`
 - `declare.h`, 826
 - `reken.c`, 2556
- `AddRatToFloat`
 - `float.c`, 1288
- `AddRHS`
 - `comtool.c`, 765
 - `declare.h`, 925
- `AddSet`
 - `declare.h`, 893
 - `names.c`, 1834
- `AddSymbol`
 - `declare.h`, 891
 - `names.c`, 1831
- `AddTableName`
 - `declare.h`, 986
 - `minos.c`, 1733
- `AddToCB`
 - `declare.h`, 806
- `AddToCom`
 - `compcomm.c`, 617
 - `declare.h`, 927
- `AddToDictionary`
 - `declare.h`, 1020
 - `dict.c`, 1076
- `AddToDollarBuffer`
 - `declare.h`, 961
 - `dollar.c`, 1093
- `AddToIndex`
 - `declare.h`, 984
 - `minos.c`, 1734
- `AddToLine`
 - `declare.h`, 827
 - `sch.c`, 2655
- `AddToPreTypes`
 - `declare.h`, 923
 - `pre.c`, 2332
- `AddToScratch`
 - `declare.h`, 1024
 - `store.c`, 2833
- `AddToString`
 - `declare.h`, 915
 - `tools.c`, 3090
- `AddVector`
 - `declare.h`, 892
 - `names.c`, 1832
- `AddWild`
 - `declare.h`, 827
 - `wildcard.c`, 3228
- `AddWildcardName`
 - `declare.h`, 923
 - `names.c`, 1830
- `AddWithFloat`
 - `float.c`, 1293
- `adjMat`
 - `MGraph`, 212
 - `T_MGraph`, 449
- `AdjustRenumScratch`
 - `declare.h`, 846
 - `reshuf.c`, 2613
- `aebufnum`
 - `T_const`, 434
- `AEdge`, 7
 - `~AEdge`, 8
 - `AEdge`, 8
 - `cand`, 8
 - `DUMMYPADDING`, 8
 - `nlegs`, 8
 - `nodes`, 8
 - `ptcl`, 8
- `aedges`
 - `ANode`, 13
- `aelegs`
 - `ANode`, 13
- `afterwards`
 - `Stream`, 419
- `agraph`
 - `AStack`, 28
 - `SProcess`, 407
- `agrcount`
 - `Process`, 365
 - `SProcess`, 409
- `ald`
 - `EGraph`, 109
- `alfatable1`
 - `compiler.c`, 726
- `all_variables`
 - `poly`, 345
- `ALLARGS`
 - `ftypes.h`, 1446
- `allAssigned`
 - `Assign`, 18
- `allbufnum`
 - `T_const`, 434
- `ALLDIRTY`
 - `ftypes.h`, 1421
- `ALLFUNCTIONS`
 - `ftypes.h`, 1405
- `ALLFUNPOWERS`
 - `ftypes.h`, 1442
- `ALLGLOBALS`
 - `structs.h`, 2908
- `AllGlobals`, 9
 - `Cc`, 10
 - `M`, 10
 - `N`, 10
 - `O`, 11

- P, [11](#)
- R, [10](#)
- S, [10](#)
- T, [11](#)
- X, [11](#)
- ALLINTEGERDOUBLE
 - ftypes.h, [1388](#)
- AllLoops
 - declare.h, [1029](#)
 - lus.c, [1689](#)
- ALLMZVFUNCTIONS
 - ftypes.h, [1405](#)
- AllocFileHandle
 - declare.h, [972](#)
 - setfile.c, [2695](#)
- AllocSetups
 - declare.h, [902](#)
 - setfile.c, [2695](#)
- AllocSort
 - declare.h, [903](#)
 - setfile.c, [2695](#)
- AllocSortFileName
 - declare.h, [903](#)
 - setfile.c, [2695](#)
- AllOnePI
 - lus.c, [1690](#)
- AllowDelay
 - P_const, [315](#)
- allPart
 - Model, [229](#)
- allParticles
 - Model, [228](#)
- AllPaths
 - declare.h, [1030](#)
 - lus.c, [1690](#)
- allsign
 - TrAcEn, [472](#)
 - TrAcEs, [478](#)
- ALLVARIABLES
 - ftypes.h, [1377](#)
- ALSODOLLARS
 - ftypes.h, [1445](#)
- ALSOFUNARGS
 - ftypes.h, [1445](#)
- ALSOPOWERS
 - ftypes.h, [1444](#)
- ALSOREVERSE
 - ftypes.h, [1389](#)
- AltCollectFun
 - C_const, [66](#)
- ALTSEPARATOR
 - fwin.h, [1498](#)
 - unix.h, [3198](#)
- AM
 - variable.h, [3210](#)
- AN
 - variable.h, [3210](#)
- aname
 - Particle, [325](#)
 - PInput, [332](#)
- ANDCOND
 - ftypes.h, [1438](#)
- ANNOUNCE
 - checkpoint.c, [541](#)
- ANode, [11](#)
 - ~ANode, [12](#)
 - aedges, [13](#)
 - aelegs, [13](#)
 - ANode, [12](#)
 - anodes, [13](#)
 - cand, [13](#)
 - deg, [13](#)
 - newleg, [12](#)
 - nlegs, [13](#)
- anodes
 - ANode, [13](#)
- antiParticle
 - Model, [228](#)
- ANTISYMMETRIC
 - ftypes.h, [1433](#)
- ANYTYPE
 - ftypes.h, [1378](#)
- AO
 - variable.h, [3211](#)
- AP
 - variable.h, [3211](#)
- aparticle
 - Particle, [324](#)
- Apply
 - declare.h, [838](#)
 - tables.c, [2969](#)
- ApplyExec
 - declare.h, [838](#)
 - tables.c, [2969](#)
- ApplyReset
 - declare.h, [838](#)
 - tables.c, [2969](#)
- AR
 - variable.h, [3211](#)
- AreArgsEqual
 - declare.h, [900](#)
 - tools.c, [3095](#)
- arg1
 - optimization, [291](#)
- arg2
 - optimization, [291](#)
- argaddress
 - N_const, [246](#)
- argag
 - Options, [302](#)
- ARGBUFFER, [14](#)
 - buffer, [14](#)
 - dollar, [14](#)
 - dummy, [15](#)
 - size, [14](#)
 - type, [15](#)

- ArgDotproductMerge
 - argument.c, [492](#)
 - declare.h, [931](#)
- argemg
 - Options, [302](#)
- ArgFactorize
 - argument.c, [491](#)
 - declare.h, [928](#)
- ARGFIELD
 - ftypes.h, [1392](#)
- ARGHEAD
 - ftypes.h, [1421](#)
- arglevel
 - C_const, [64](#)
- arglist
 - N_const, [251](#)
- arglistsize
 - N_const, [254](#)
- argmg
 - Options, [302](#)
- argnames
 - pReVaR, [360](#)
- ARRANGE
 - ftypes.h, [1446](#)
- args
 - PaRtl, [320](#)
- argsize
 - NeStInG, [269](#)
- argstack
 - C_const, [52](#)
- argsumcheck
 - C_const, [63](#)
- ArgSymbolMerge
 - argument.c, [492](#)
 - declare.h, [931](#)
- argtail
 - TaBIEs, [461](#)
- ARGTOARG
 - ftypes.h, [1419](#)
- argument.c, [489](#), [495](#)
 - ArgDotproductMerge, [492](#)
 - ArgFactorize, [491](#)
 - ArgSymbolMerge, [492](#)
 - ArgumentExplode, [490](#)
 - ArgumentImplode, [490](#)
 - CleanupArgCache, [491](#)
 - execarg, [490](#)
 - execterm, [490](#)
 - FindArg, [491](#)
 - InsertArg, [491](#)
 - MakeInteger, [493](#)
 - MakeMod, [493](#)
 - NEWORDER, [490](#)
 - SortWeights, [494](#)
 - TakeArgContent, [492](#)
- argument_to_poly
 - poly, [347](#)
- ArgumentExplode
 - argument.c, [490](#)
 - declare.h, [981](#)
- ArgumentImplode
 - argument.c, [490](#)
 - declare.h, [981](#)
- ARGWILD
 - ftypes.h, [1391](#)
- ARLTOARL
 - ftypes.h, [1419](#)
- arraysize
 - POLYMOD, [358](#)
- articals
 - MConn, [199](#)
- AS
 - variable.h, [3211](#)
- ASINFUNCTION
 - ftypes.h, [1400](#)
- ASINHFUNCTION
 - ftypes.h, [1401](#)
- Assign, [15](#)
 - ~Assign, [17](#)
 - addEdge, [18](#)
 - allAssigned, [18](#)
 - Assign, [17](#)
 - assignAllVertices, [17](#)
 - assignVertex, [20](#)
 - assignPLeg, [21](#)
 - assignVertex, [18](#)
 - astack, [24](#)
 - candPart, [20](#)
 - candPartClassify, [21](#)
 - checkAG, [17](#)
 - checkOrderCpl, [21](#)
 - checkpoint0, [25](#)
 - cmpNodes, [22](#)
 - cmpPermGraph, [22](#)
 - connect, [19](#)
 - cpleft, [25](#)
 - detEdge, [21](#)
 - edges, [25](#)
 - edgeSym, [22](#)
 - egraph, [23](#)
 - fillEGraph, [19](#)
 - fromMGraph, [18](#)
 - getLegParticle, [19](#)
 - isIsomorphic, [22](#)
 - isOrdLegs, [22](#)
 - isOrdPLeg, [21](#)
 - legEdgeParticle, [19](#)
 - legPart, [20](#)
 - mgraph, [23](#)
 - model, [23](#)
 - nAGraphs, [25](#)
 - nAOPI, [25](#)
 - nEdges, [24](#)
 - nETotal, [24](#)
 - nExtern, [24](#)
 - nNodes, [24](#)

- nodes, 25
- opt, 24
- orbits, 24
- pnclass, 24
- prCand, 17
- proc, 23
- reordLeg, 19
- restoreCouple, 23
- saveCouple, 22
- selectLeg, 18
- selectVertex, 18
- selectVertexSimp, 18
- selUnAssLeg, 20
- selUnAssVertex, 20
- selUnAssVertexSimp, 20
- sproc, 23
- subCouple, 23
- updateCandNode, 21
- wAGraphs, 25
- wAOPI, 25
- assign
 - SProcess, 405
- assignAllVertices
 - Assign, 17
- AssignDollar
 - declare.h, 961
 - dollar.c, 1093
- assigned
 - EGraph, 110
- assignIVertex
 - Assign, 20
- assignPLeg
 - Assign, 21
- assignVertex
 - Assign, 18
- AStack, 26
 - ~AStack, 27
 - agraph, 28
 - AStack, 27
 - checkPoint, 27
 - eSize, 29
 - eStack, 28
 - eStackP, 29
 - nSize, 28
 - nStack, 28
 - nStackP, 28
 - prStack, 27
 - pushEdge, 28
 - pushNode, 28
 - restore, 27
 - setAGraph, 27
- astack
 - Assign, 24
 - Process, 366
 - SProcess, 406
- AT
 - variable.h, 3211
- ATAN2FUNCTION
 - ftypes.h, 1401
- ATANFUNCTION
 - ftypes.h, 1400
- ATANHFUNCTION
 - ftypes.h, 1401
- ATENDOFMODULE
 - ftypes.h, 1377
- atstartup
 - M_const, 190
- AutoDeclareFlag
 - C_const, 54
- AutoFunctionList
 - C_const, 45
- autofunctions
 - variable.h, 3209
- AutoIndexList
 - C_const, 45
- autoindices
 - variable.h, 3209
- AUTONAMES
 - ftypes.h, 1446
- autonames
 - C_const, 45
- AutoSymbolList
 - C_const, 45
- autosymbols
 - variable.h, 3209
- AutoVectorList
 - C_const, 45
- autovectors
 - variable.h, 3209
- AWORDMASK
 - form3.h, 1327
- AX
 - variable.h, 3211
- BACKINOUT
 - declare.h, 806
- balance
 - NaMeNode, 261
- Balancing
 - S_const, 382
- BASECODE
 - ftypes.h, 1449
- BASEPOSITION
 - declare.h, 812
- bconn
 - EGraph, 113
- begin
 - Options, 300
- beginProc
 - Options, 300
- beginSubProc
 - Options, 300
- Bernoulli
 - declare.h, 843
 - reken.c, 2565
- BERNOULLIFUNCTION
 - ftypes.h, 1397

- bernoullis
 - T_const, [430](#)
 - BETAVERSION
 - form3.h, [1326](#)
 - BHEAD
 - structs.h, [2903](#)
 - BHEAD0
 - structs.h, [2903](#)
 - biconnE
 - EGraph, [106](#)
 - biconnME
 - MGraph, [209](#)
 - bicount
 - EGraph, [113](#)
 - MGraph, [215](#)
 - bidef
 - EGraph, [113](#)
 - MGraph, [215](#)
 - BigLong
 - declare.h, [827](#)
 - reken.c, [2558](#)
 - biinitE
 - EGraph, [106](#)
 - bilow
 - EGraph, [113](#)
 - MGraph, [215](#)
 - BinarySearch
 - declare.h, [973](#)
 - sort.c, [2733](#)
 - BinomGen
 - declare.h, [827](#)
 - ratio.c, [2505](#)
 - BINOMIAL
 - ftypes.h, [1395](#)
 - bisearchE
 - EGraph, [106](#)
 - bisearchME
 - MGraph, [209](#)
 - bit_0
 - bit_field, [30](#)
 - bit_1
 - bit_field, [30](#)
 - bit_2
 - bit_field, [30](#)
 - bit_3
 - bit_field, [30](#)
 - bit_4
 - bit_field, [30](#)
 - bit_5
 - bit_field, [30](#)
 - bit_6
 - bit_field, [30](#)
 - bit_7
 - bit_field, [30](#)
 - bit_field, [29](#)
 - bit_0, [30](#)
 - bit_1, [30](#)
 - bit_2, [30](#)
 - bit_3, [30](#)
 - bit_4, [30](#)
 - bit_5, [30](#)
 - bit_6, [30](#)
 - bit_7, [30](#)
- blksp
 - MConn, [200](#)
 - blkstk
 - MConn, [200](#)
 - blkstkptr
 - MConn, [202](#)
 - blnce
 - tree, [479](#)
 - BLOCK
 - ftypes.h, [1404](#)
 - block
 - MGraph, [215](#)
 - blocks
 - MConn, [199](#)
 - BlockSpaces
 - O_const, [282](#)
 - BOOL
 - form3.h, [1333](#)
 - boomlijst
 - CbUf, [73](#)
 - TaBIEs, [461](#)
 - BottomLevel
 - C_const, [56](#)
 - bounds
 - TaBIEs, [463](#)
 - BrackBuf
 - T_const, [429](#)
 - bracket
 - BrAcKeTiNdEx, [32](#)
 - O_const, [279](#)
 - bracketbuffer
 - BrAcKeTiNfO, [33](#)
 - bracketbuffersize
 - BrAcKeTiNfO, [33](#)
 - BRACKETCURRENTEXPR
 - execute.c, [1155](#)
 - BracketFactors
 - M_const, [192](#)
 - bracketfill
 - BrAcKeTiNfO, [33](#)
 - BrAcKeTiNdEx, [31](#)
 - bracket, [32](#)
 - next, [32](#)
 - start, [32](#)
 - termsinbracket, [32](#)
 - bracketindexflag
 - C_const, [59](#)
 - T_const, [435](#)
 - BrAcKeTiNfO, [32](#)
 - bracketbuffer, [33](#)
 - bracketbuffersize, [33](#)
 - bracketfill, [33](#)
 - indexbuffer, [33](#)

- indexbufferize, [33](#)
- indexfill, [34](#)
- SortType, [34](#)
- BracketInfo, [34](#)
 - BracketInfo, [35](#)
 - dummy, [35](#)
 - num_terms, [35](#)
 - operator<, [35](#)
 - p, [36](#)
 - pattern, [35](#)
- bracketinfo
 - ExPrEsSiOn, [122](#)
 - T_const, [432](#)
- BracketNormalize
 - C_const, [59](#)
 - declare.h, [857](#)
 - normal.c, [1878](#)
- BracketOn
 - R_const, [374](#)
- BRACKETOTHEREXPR
 - execute.c, [1155](#)
- bridges
 - MConn, [199](#)
- buff
 - PF_BUFFER, [329](#)
- Buffer
 - CbUf, [71](#)
- buffer
 - ARGBUFFER, [14](#)
 - INSIDEINFO, [158](#)
 - longMultiStruct, [167](#)
 - PRELOAD, [359](#)
 - StOrEcAcHe, [412](#)
 - StreaM, [417](#)
- bufferedInd
 - O_const, [285](#)
- BufferForOutput
 - inivar.h, [1684](#)
- buffernum
 - SuBbUf, [420](#)
- bufferposition
 - StreaM, [418](#)
- buffers
 - TaBIes, [462](#)
- buffersfill
 - TaBIes, [465](#)
- BufferSize
 - CbUf, [73](#)
- bufferize
 - StreaM, [418](#)
- bufferssize
 - TaBIes, [464](#)
- bufIPstruct, [36](#)
 - e, [37](#)
 - i, [37](#)
- bufnum
 - TaBIes, [464](#)
- bufpos
 - longMultiStruct, [167](#)
- BUG
 - variable.h, [3210](#)
- bugtool.c, [535](#), [536](#)
 - ExprStatus, [536](#)
- build_Horner_tree
 - optimize.cc, [1996](#)
- buildTree
 - optimize.cc, [2000](#)
- BUILTINDOLLARS
 - ftypes.h, [1407](#)
- BUILTININDICES
 - ftypes.h, [1407](#)
- BUILTINSYMBOLS
 - ftypes.h, [1407](#)
- BUILTINVECTORS
 - ftypes.h, [1407](#)
- bzero
 - form3.h, [1332](#)
- c1PI
 - MGraph, [212](#)
 - T_MGraph, [449](#)
- c1PINoTadBlock
 - MGraph, [213](#)
- C_const, [37](#)
 - activenames, [46](#)
 - AltCollectFun, [66](#)
 - arglevel, [64](#)
 - argstack, [52](#)
 - argsumcheck, [63](#)
 - AutoDeclareFlag, [54](#)
 - AutoFunctionList, [45](#)
 - AutoIndexList, [45](#)
 - autonames, [45](#)
 - AutoSymbolList, [45](#)
 - AutoVectorList, [45](#)
 - BottomLevel, [56](#)
 - bracketindexflag, [59](#)
 - BracketNormalize, [59](#)
 - cbufList, [44](#)
 - cbufnum, [54](#)
 - ChannelList, [42](#)
 - CheckpointFlag, [62](#)
 - CheckpointInterval, [53](#)
 - CheckpointRunAfter, [51](#)
 - CheckpointRunBefore, [51](#)
 - CheckpointStamp, [53](#)
 - cmod, [47](#)
 - CModule, [53](#)
 - Cnumpows, [66](#)
 - CodesFlag, [57](#)
 - CollectFun, [66](#)
 - CollectPercentage, [69](#)
 - ComDefer, [66](#)
 - Commercial, [70](#)
 - CommuteInSet, [52](#)
 - CompileLevel, [56](#)
 - compiletype, [54](#)

CurrentStream, 46
debugFlags, 70
DidClean, 67
DirtPow, 65
dollarnames, 42
dolooplevel, 62
doloopnest, 51
doloopstack, 51
doloopstacksize, 62
DubiousList, 43
DumNum, 65
dumnumflag, 59
endoftokens, 50
ExpressionList, 43
exprfillwarning, 60
exprnames, 42
extrasym, 51
extrasymbols, 69
ffbufnum, 61
FinalStats, 58
firstconstindex, 54
firstctypemessage, 56
FixIndices, 49
FlintPolyFlag, 62
Fortran90Kind, 51
FunctionList, 43
Functions, 46
funpowers, 57
halfmod, 48
HideLevel, 68
HumanStatsFlag, 62
iBuffer, 48
iBufferSize, 52
idoption, 56
IfCount, 48
IfHeap, 48
IfLevel, 67
IfStack, 48
IfSumCheck, 52
IndexList, 43
Indices, 45
inexprlevel, 64
inexprstack, 52
inexprsumcheck, 64
InnerTest, 69
inparallelflag, 60
insidefirst, 54
insidelevel, 64
insidestack, 52
insidesumcheck, 64
iPointer, 49
IsFortran90, 61
iStop, 49
LabelNames, 49
Labels, 50
IDefDim, 55
IDefDim4, 55
Ihdollarflag, 60
LineLength, 68
LogHandle, 67
IPolyFun, 68
IPolyFunExp, 68
IPolyFunInv, 68
IPolyFunPow, 69
IPolyFunType, 68
IPolyFunVar, 68
ISortType, 58
IUniTrace, 64
IUnitTrace, 66
MaxIf, 56
MaxLabels, 55
MaxNumStreams, 56
MaxSwitch, 67
maxtermlevel, 59
MemDebugFlag, 61
minsidefirst, 55
ModelLevel, 63
models, 46
modelspace, 63
modinverses, 50
modmode, 65
ModOptDolList, 44
modpowers, 47
mparallelflag, 59
mProcessBucketSize, 53
MultiBracketBuf, 51
MultiBracketLevels, 61
MustTestTable, 65
NamesFlag, 57
ncmod, 65
nhalfmod, 65
NoCompress, 61
NoShowInput, 54
npowmod, 65
NumLabels, 55
nummodels, 63
NumStreams, 56
NumWildcardNames, 55
NwildC, 66
OldFactArgFlag, 61
OldGCDflag, 61
OldParallelStats, 58
origin, 63
OutNumberType, 67
OutputMode, 66
OutputSpaces, 66
outsidefun, 57
parallelflag, 59
partodoflag, 60
PolyRatFunChanged, 69
PotModDolList, 44
powmod, 47
PrintBacktraceFlag, 62
ProcessBucketSize, 53
ProcessStats, 59
properorderflag, 60

- ProtoType, [48](#)
- RepLevel, [64](#)
- RepSumCheck, [64](#)
- separators, [42](#)
- SetElementList, [43](#)
- SetList, [43](#)
- SetupFlag, [58](#)
- ShortStats, [54](#)
- ShortStatsMax, [69](#)
- SizeCommutelnSet, [63](#)
- SortReallocateFlag, [62](#)
- SortType, [58](#)
- StatsFlag, [57](#)
- StoreFileSize, [42](#)
- StoreHandle, [68](#)
- Streams, [46](#)
- SwitchArray, [46](#)
- SwitchHeap, [47](#)
- SwitchlnArray, [67](#)
- SwitchLevel, [67](#)
- SymbolList, [44](#)
- Symbols, [45](#)
- SymChangeFlag, [69](#)
- TableBaseList, [44](#)
- tablecheck, [56](#)
- tablefilling, [60](#)
- termlevel, [65](#)
- termstortstack, [47](#)
- termstack, [47](#)
- termsumcheck, [49](#)
- TestValue, [52](#)
- ThreadBalancing, [58](#)
- ThreadBucketSize, [53](#)
- ThreadsFlag, [58](#)
- ThreadSortFileSynch, [59](#)
- ThreadStats, [58](#)
- ToBeInFactors, [69](#)
- tokenarglevel, [50](#)
- tokens, [50](#)
- TokensWriteFlag, [57](#)
- topolynomialflag, [61](#)
- toptokens, [50](#)
- TransEname, [53](#)
- UnsureDollarMode, [57](#)
- varnames, [42](#)
- vectorlikeLHS, [63](#)
- VectorList, [44](#)
- Vectors, [46](#)
- vetofilling, [60](#)
- vetotablebasefill, [60](#)
- WarnFlag, [57](#)
- WhileLevel, [67](#)
- WildC, [48](#)
- WildcardBufferSize, [55](#)
- WildcardNames, [49](#)
- wildflag, [55](#)
- WTimeStatsFlag, [62](#)
- cache_buffer
 - checkpoint.c, [545](#)
 - cache_fill
 - checkpoint.c, [545](#)
 - CACHE_SIZE
 - checkpoint.c, [538](#)
 - CACHED_SNAPSHOT
 - checkpoint.c, [538](#)
 - CacheNumberFree
 - declare.h, [817](#)
 - CacheNumberMalloc
 - declare.h, [816](#)
 - CacheNumberMallocAddMemory
 - declare.h, [999](#)
 - tools.c, [3093](#)
 - CacheNumberMemHeap
 - T_const, [432](#)
 - CacheNumberMemMax
 - T_const, [435](#)
 - CacheNumberMemTop
 - T_const, [435](#)
 - calcHash
 - node, [273](#)
 - CalculateEuler
 - float.c, [1290](#)
 - CalculateMZV
 - float.c, [1290](#)
 - CalculateMZVhalf
 - float.c, [1290](#)
 - CanCommu
 - CbUf, [72](#)
 - cand
 - AEdge, [8](#)
 - ANode, [13](#)
 - candPart
 - Assign, [20](#)
 - candPartClassify
 - Assign, [21](#)
 - CARRIAGERETURN
 - fwin.h, [1497](#)
 - unix.h, [3197](#)
 - caseoffset
 - SWITCH, [422](#)
 - CatchDollar
 - declare.h, [961](#)
 - dollar.c, [1093](#)
 - CBUF
 - structs.h, [2907](#)
 - CbUf, [70](#)
 - boomlijst, [73](#)
 - Buffer, [71](#)
 - BufferSize, [73](#)
 - CanCommu, [72](#)
 - dimension, [73](#)
 - lhs, [72](#)
 - maxlhs, [73](#)
 - maxrhs, [74](#)
 - MaxTreeSize, [74](#)
 - mnumlhs, [74](#)

- mnumrhs, [74](#)
 - numdum, [72](#)
 - numlhs, [73](#)
 - numrhs, [73](#)
 - NumTerms, [72](#)
 - numtree, [74](#)
 - Pointer, [71](#)
 - rhs, [72](#)
 - rootnum, [74](#)
 - Top, [71](#)
- cbuf
 - variable.h, [3208](#)
- cBuffer
 - sOrT, [395](#)
- cBufferSize
 - sOrT, [399](#)
- cbufList
 - C_const, [44](#)
- cbufnum
 - C_const, [54](#)
- Cc
 - AllGlobals, [10](#)
- cComChar
 - P_const, [316](#)
- cdeg
 - PGInput, [331](#)
- CDELETE
 - ftypes.h, [1378](#)
- CDELTA
 - ftypes.h, [1380](#)
- cDiag
 - MGraph, [212](#)
- CDOLLAR
 - ftypes.h, [1380](#)
- CDOTPRODUCT
 - ftypes.h, [1379](#)
- CDOUBLEDOT
 - ftypes.h, [1380](#)
- CDUBIOUS
 - ftypes.h, [1380](#)
- CEXPRESSION
 - ftypes.h, [1379](#)
- CFD
 - minos.c, [1730](#)
- CFUN
 - ftypes.h, [1458](#)
- CFUNCTION
 - ftypes.h, [1379](#)
- cgen
 - SGroup, [388](#)
- ChainIn
 - declare.h, [981](#)
 - function.c, [1473](#)
- ChainOut
 - declare.h, [981](#)
 - function.c, [1473](#)
- CHANNEL
 - structs.h, [2907](#)
- ChAnNeL, [75](#)
 - handle, [75](#)
 - name, [75](#)
- ChannelList
 - C_const, [42](#)
- channels
 - variable.h, [3208](#)
- characters
 - DICTIONARY, [83](#)
- CharOut
 - declare.h, [904](#)
 - pre.c, [2321](#)
- chartype
 - variable.h, [3204](#)
- CHECK
 - parallel.c, [2067](#)
- check_memory
 - poly, [343](#)
- checkAG
 - Assign, [17](#)
- CHECKEXTERN
 - ftypes.h, [1457](#)
- CheckFloat
 - float.c, [1292](#)
- CHECKLOGTYPE
 - ftypes.h, [1390](#)
- checkOrderCpl
 - Assign, [21](#)
- checkPoint
 - AStack, [27](#)
- checkpoint.c, [537](#), [545](#)
 - ANNOUNCE, [541](#)
 - cache_buffer, [545](#)
 - cache_fill, [545](#)
 - CACHE_SIZE, [538](#)
 - CACHED_SNAPSHOT, [538](#)
 - CheckRecoveryFile, [542](#)
 - DeleteRecoveryFile, [542](#)
 - DoCheckpoint, [544](#)
 - DoRecovery, [543](#)
 - flush_cache, [543](#)
 - fwrite_cached, [543](#)
 - InitRecovery, [542](#)
 - R_COPY_B, [539](#)
 - R_COPY_LIST, [540](#)
 - R_COPY_NAMETREE, [541](#)
 - R_COPY_S, [540](#)
 - R_FREE, [538](#)
 - R_FREE_NAMETREE, [539](#)
 - R_FREE_STREAM, [539](#)
 - R_SET, [539](#)
 - RecoveryFilename, [542](#)
 - S_FLUSH_B, [540](#)
 - S_WRITE_B, [539](#)
 - S_WRITE_DOLLAR, [541](#)
 - S_WRITE_LIST, [540](#)
 - S_WRITE_NAMETREE, [541](#)
 - S_WRITE_S, [540](#)

- checkpoint0
 - Assign, [25](#)
- CheckpointFlag
 - C_const, [62](#)
- CheckpointInterval
 - C_const, [53](#)
- CheckpointRunAfter
 - C_const, [51](#)
- CheckpointRunBefore
 - C_const, [51](#)
- CheckpointStamp
 - C_const, [53](#)
- CHECKPOLY
 - token.c, [3052](#)
- CheckPower
 - O_const, [281](#)
- CheckRecoveryFile
 - checkpoint.c, [542](#)
 - declare.h, [996](#)
- CheckTableDeclarations
 - declare.h, [837](#)
 - tables.c, [2967](#)
- CheckWild
 - declare.h, [827](#)
 - wildcard.c, [3228](#)
- childs
 - tree_node, [481](#)
- CHISHOLM
 - ftypes.h, [1389](#)
- Chisholm
 - declare.h, [828](#)
 - opera.c, [1960](#)
- chkMomConsv
 - EGraph, [107](#)
- chkOrd
 - T_MNodeClass, [455](#)
- CINDEX
 - ftypes.h, [1379](#)
- cl2mcl
 - PNodeClass, [337](#)
- cl2nd
 - PNodeClass, [337](#)
 - SProcess, [407](#)
- clCmp
 - MNodeClass, [221](#)
 - T_MNodeClass, [454](#)
- cldeg
 - NCInput, [268](#)
 - ToPoTyPe, [469](#)
- CleanDollarFactors
 - declare.h, [965](#)
 - dollar.c, [1098](#)
- CleanExpr
 - declare.h, [828](#)
 - execute.c, [1156](#)
- CLEANFLAG
 - ftypes.h, [1421](#)
- CleanUp
 - declare.h, [828](#)
 - startup.c, [2804](#)
- cleanup
 - flintinterface.h, [1270](#)
- cleanup_master
 - flintinterface.h, [1270](#)
- CleanupArgCache
 - argument.c, [491](#)
 - declare.h, [930](#)
- CleanUpSort
 - declare.h, [972](#)
 - sort.c, [2733](#)
- CleanupTerm
 - declare.h, [976](#)
 - execute.c, [1159](#)
- CLEAR
 - ftypes.h, [1439](#)
- ClearBracketIndex
 - declare.h, [979](#)
 - index.c, [1674](#)
- clearcbuf
 - comtool.c, [764](#)
 - declare.h, [971](#)
- clearGroup
 - SGroup, [387](#)
- ClearMacro
 - declare.h, [906](#)
 - pre.c, [2324](#)
- CLEARMODULE
 - ftypes.h, [1370](#)
- ClearMZVTables
 - float.c, [1293](#)
- clearnamefill
 - NaMeTree, [265](#)
- clearnodefill
 - NaMeTree, [265](#)
- ClearOptimize
 - declare.h, [969](#)
 - optimize.cc, [2009](#)
- ClearPushback
 - declare.h, [903](#)
 - pre.c, [2320](#)
- ClearSpectators
 - declare.h, [1023](#)
 - spectator.c, [2792](#)
- ClearStore
 - M_const, [192](#)
- ClearTableTree
 - declare.h, [970](#)
 - tables.c, [2965](#)
- ClearTree
 - comtool.c, [767](#)
 - declare.h, [961](#)
- ClearWild
 - declare.h, [828](#)
 - wildcard.c, [3228](#)
- ClearWildcardNames
 - declare.h, [923](#)

- names.c, [1830](#)
- clext
 - ToPoTyPe, [469](#)
- clist
 - Interaction, [161](#)
 - MGraph, [211](#)
 - MNodeClass, [223](#)
 - Process, [366](#)
 - SProcess, [408](#)
 - T_MGraph, [448](#)
 - T_MNodeClass, [455](#)
- clmat
 - MNodeClass, [223](#)
 - T_MNodeClass, [456](#)
- clnum
 - NCInput, [268](#)
 - ToPoTyPe, [469](#)
- clord
 - MNodeClass, [223](#)
- closeAllExternalChannels
 - declare.h, [989](#)
 - extcmd.c, [1193](#)
- CloseChannel
 - declare.h, [897](#)
 - tools.c, [3088](#)
- closeExternalChannel
 - declare.h, [989](#)
 - extcmd.c, [1193](#)
- CloseFile
 - declare.h, [894](#)
 - tools.c, [3085](#)
- CloseStream
 - declare.h, [887](#)
 - tools.c, [3083](#)
- clss
 - MNode, [219](#)
 - T_MNode, [452](#)
- cltyp
 - NCInput, [268](#)
- cmagd
 - NCInput, [268](#)
 - ToPoTyPe, [469](#)
- cmagdeg
 - MNode, [219](#)
 - MNodeClass, [223](#)
 - Particle, [326](#)
 - PNodeClass, [336](#)
- cmind
 - NCInput, [268](#)
 - ToPoTyPe, [469](#)
- cminddeg
 - MNode, [219](#)
 - MNodeClass, [223](#)
 - Particle, [326](#)
 - PNodeClass, [336](#)
- cmmod
 - C_const, [47](#)
 - N_const, [251](#)
- CMODE
 - ftypes.h, [1387](#)
- CMODEL
 - ftypes.h, [1380](#)
- CModule
 - C_const, [53](#)
- cmp
 - node, [272](#)
- CmpArray
 - declare.h, [901](#)
 - tools.c, [3089](#)
- cmpArray
 - T_MNodeClass, [455](#)
- cmpMNCArray
 - MNodeClass, [222](#)
- cmpMom
 - EGraph, [107](#)
- cmpNodes
 - Assign, [22](#)
- cmpPermGraph
 - Assign, [22](#)
- cnamlist
 - MInput, [217](#)
- cnlist
 - Model, [229](#)
- cNoTadBlock
 - MGraph, [213](#)
- cNoTadpole
 - MGraph, [213](#)
- cnum
 - PGInput, [331](#)
- CNUMBER
 - ftypes.h, [1379](#)
- Cnumlhs
 - R_const, [373](#)
- Cnumpows
 - C_const, [66](#)
- CoAntiBracket
 - compcomm.c, [624](#)
 - declare.h, [932](#)
- CoAntiPutInside
 - compcomm.c, [631](#)
 - declare.h, [935](#)
- CoAntiSymmetrize
 - compcomm.c, [620](#)
 - declare.h, [933](#)
- CoApply
 - declare.h, [936](#)
 - tables.c, [2968](#)
- CoArgExplode
 - compcomm.c, [627](#)
 - declare.h, [933](#)
- CoArgImplode
 - compcomm.c, [627](#)
 - declare.h, [933](#)
- CoArgToExtraSymbol
 - compcomm.c, [628](#)
 - declare.h, [1006](#)

- CoArgument
 - compcomm.c, 618
 - declare.h, 933
- CoAssign
 - comexpr.c, 585
 - declare.h, 959
- CoAuto
 - declare.h, 953
 - names.c, 1835
- CoBracket
 - compcomm.c, 624
 - declare.h, 934
- CoBreak
 - compcomm.c, 631
 - declare.h, 953
- CoCanonicalize
 - declare.h, 834
 - diagrams.c, 1052
- CoCase
 - compcomm.c, 631
 - declare.h, 953
- CoCFunction
 - declare.h, 935
 - names.c, 1833
- CoChainin
 - compcomm.c, 622
 - declare.h, 952
- CoChainout
 - compcomm.c, 622
 - declare.h, 952
- CoChisholm
 - compcomm.c, 621
 - declare.h, 952
- CoClearTable
 - compcomm.c, 628
 - declare.h, 952
- CoClearUserFlag
 - compcomm.c, 632
 - declare.h, 1028
- CoCollect
 - compcomm.c, 612
 - declare.h, 935
- CoCommutelnSet
 - declare.h, 893
 - names.c, 1832
- CoCompress
 - compcomm.c, 612
 - declare.h, 935
- CoContract
 - compcomm.c, 617
 - declare.h, 935
- CoCopySpectator
 - declare.h, 1023
 - spectator.c, 2792
- CoCreateAll
 - compcomm.c, 632
 - declare.h, 1029
- CoCreateAllLoops
 - compcomm.c, 632
 - declare.h, 1028
- CoCreateAllPaths
 - compcomm.c, 632
 - declare.h, 1028
- CoCreateSpectator
 - declare.h, 1022
 - spectator.c, 2791
- CoCTable
 - declare.h, 951
 - names.c, 1834
- CoCTensor
 - declare.h, 935
 - names.c, 1833
- CoCycleSymmetrize
 - compcomm.c, 620
 - declare.h, 936
- code
 - OPTIMIZERESULT, 295
- CoDeallocateTable
 - comexpr.c, 585
 - declare.h, 965
- CodeFactors
 - compiler.c, 726
 - declare.h, 960
- CoDefault
 - compcomm.c, 631
 - declare.h, 954
- CodeGenerator
 - compiler.c, 725
 - declare.h, 959
- CoDelete
 - compcomm.c, 613
 - declare.h, 936
- CoDenominators
 - compcomm.c, 628
 - declare.h, 936
- CodesFlag
 - C_const, 57
- codesize
 - OPTIMIZERESULT, 295
- CodeToLine
 - declare.h, 822
 - sch.c, 2656
- CoDimension
 - declare.h, 936
 - names.c, 1832
- CoDiscard
 - compcomm.c, 617
 - declare.h, 936
- CoDisorder
 - comexpr.c, 583
 - declare.h, 937
- CoDo
 - compcomm.c, 629
 - declare.h, 1006
- CoDrop
 - compcomm.c, 615

- declare.h, 937
- CoDropCoefficient
 - compcomm.c, 628
 - declare.h, 937
- CoDropSymbols
 - compcomm.c, 628
 - declare.h, 937
- coeff
 - optimization, 291
- COEFFI
 - ftypes.h, 1436
- coefficient
 - poly, 346
- coefficient_list_divmod
 - poly, 348
- coefficients_modulo
 - poly, 346
- COEFFICIENTSONLY
 - ftypes.h, 1444
- COEFFSYMBOL
 - ftypes.h, 1406
- coefs
 - POLYMOD, 358
- CoElse
 - compcomm.c, 624
 - declare.h, 937
- CoElself
 - compcomm.c, 625
 - declare.h, 937
- CoEmptySpectator
 - declare.h, 1023
 - spectator.c, 2791
- CoEndArgument
 - compcomm.c, 618
 - declare.h, 937
- CoEndDo
 - compcomm.c, 629
 - declare.h, 1006
- CoEndIf
 - compcomm.c, 625
 - declare.h, 938
- CoEndInExpression
 - compcomm.c, 623
 - declare.h, 934
- CoEndInside
 - compcomm.c, 618
 - declare.h, 938
- CoEndModel
 - declare.h, 1028
- CoEndRepeat
 - compcomm.c, 623
 - declare.h, 938
- CoEndSwitch
 - compcomm.c, 631
 - declare.h, 954
- CoEndTerm
 - compcomm.c, 626
 - declare.h, 938
- CoEndWhile
 - compcomm.c, 625
 - declare.h, 938
- CoEvaluate
 - float.c, 1294
- CoExit
 - compcomm.c, 622
 - declare.h, 938
- CoExtraSymbols
 - compcomm.c, 629
 - declare.h, 1006
- CoFactArg
 - compcomm.c, 619
 - declare.h, 938
- CoFactDollar
 - compcomm.c, 629
 - declare.h, 939
- CoFactorize
 - compcomm.c, 630
 - declare.h, 939
- CoFill
 - comexpr.c, 584
 - declare.h, 939
- CoFillExpression
 - comexpr.c, 584
 - declare.h, 940
- CoFindLoop
 - compcomm.c, 625
 - declare.h, 963
- CoFixIndex
 - compcomm.c, 614
 - declare.h, 940
- CoFlags
 - compcomm.c, 613
 - declare.h, 945
- CoFormat
 - compcomm.c, 612
 - declare.h, 940
- CoFromPolynomial
 - compcomm.c, 628
 - declare.h, 1005
- CoFunction
 - declare.h, 893
 - names.c, 1832
- CoFunPowers
 - compcomm.c, 626
 - declare.h, 963
- CoGlobal
 - comexpr.c, 582
 - declare.h, 940
- CoGlobalFactorized
 - comexpr.c, 582
 - declare.h, 940
- CoGoTo
 - compcomm.c, 617
 - declare.h, 940
- CoHide
 - compcomm.c, 616

- declare.h, 948
- Cold
 - comexpr.c, 583
 - declare.h, 940
- ColdExpression
 - comexpr.c, 584
 - declare.h, 959
- ColdNew
 - comexpr.c, 583
 - declare.h, 941
- ColdOld
 - comexpr.c, 582
 - declare.h, 941
- Colf
 - compcomm.c, 624
 - declare.h, 941
- ColfMatch
 - comexpr.c, 583
 - declare.h, 941
- ColfNoMatch
 - comexpr.c, 583
 - declare.h, 941
- ColIndex
 - declare.h, 941
 - names.c, 1831
- ColnExpression
 - compcomm.c, 623
 - declare.h, 934
- ColnParallel
 - compcomm.c, 622
 - declare.h, 934
- ColInside
 - compcomm.c, 618
 - declare.h, 933
- ColInsideFirst
 - compcomm.c, 613
 - declare.h, 941
- ColIntoHide
 - compcomm.c, 616
 - declare.h, 948
- CoKeep
 - compcomm.c, 613
 - declare.h, 942
- CoLabel
 - compcomm.c, 618
 - declare.h, 942
- CollectFun
 - C_const, 66
- CollectOverFlag
 - S_const, 382
- CollectPercentage
 - C_const, 69
- CoLoad
 - declare.h, 942
 - store.c, 2834
- CoLocal
 - comexpr.c, 582
 - declare.h, 942
- CoLocalFactorized
 - comexpr.c, 582
 - declare.h, 942
- CoMakeInteger
 - compcomm.c, 619
 - declare.h, 945
- CoMany
 - comexpr.c, 583
 - declare.h, 942
- combilast
 - SHvariables, 391
- ComChar
 - P_const, 316
- ComDefer
 - C_const, 66
- CoMerge
 - compcomm.c, 627
 - declare.h, 942
- CoMetric
 - compcomm.c, 614
 - declare.h, 943
- comexpr.c, 581, 585
 - CoAssign, 585
 - CoDeallocateTable, 585
 - CoDisorder, 583
 - CoFill, 584
 - CoFillExpression, 584
 - CoGlobal, 582
 - CoGlobalFactorized, 582
 - Cold, 583
 - ColdExpression, 584
 - ColdNew, 583
 - ColdOld, 582
 - ColfMatch, 583
 - ColfNoMatch, 583
 - CoLocal, 582
 - CoLocalFactorized, 582
 - CoMany, 583
 - CoMulti, 583
 - CoMultiply, 584
 - CoOnce, 584
 - CoOnly, 584
 - CoPrintTable, 585
 - CoSelect, 584
 - DoExpr, 582
- comfun
 - T_const, 437
- COMFUNPOWERS
 - ftypes.h, 1442
- comind
 - T_const, 437
- commentchar
 - setfile.c, 2696
- Commercial
 - C_const, 70
- COMMERCIALSIZE
 - fsizes.h, 1345
- Commute

- declare.h, [828](#)
- normal.c, [1876](#)
- commute
 - FUN_INFO, [143](#)
 - FuNcTiOn, [145](#)
- CommutelnSet
 - C_const, [52](#)
- comnum
 - T_const, [437](#)
- CoModel
 - declare.h, [1027](#)
- CoModOption
 - declare.h, [943](#)
 - module.c, [1765](#)
- CoModuleOption
 - declare.h, [943](#)
 - module.c, [1765](#)
- CoModulus
 - compcomm.c, [623](#)
 - declare.h, [943](#)
- CompactifyTree
 - declare.h, [890](#)
 - names.c, [1829](#)
- COMPARE
 - structs.h, [2908](#)
- Compare1
 - declare.h, [830](#)
 - sort.c, [2727](#)
- compare_degree_vector
 - poly, [349](#)
- CompareArgs
 - declare.h, [900](#)
 - tools.c, [3096](#)
- COMPAREDUMMY
 - structs.h, [2906](#)
- CompareFunctions
 - declare.h, [828](#)
 - normal.c, [1876](#)
- CompareHSymbols
 - declare.h, [990](#)
 - sort.c, [2728](#)
- CompareRoutine
 - R_const, [372](#)
- CompareSymbols
 - declare.h, [990](#)
 - sort.c, [2727](#)
- CompareTerms
 - declare.h, [820](#)
- CompArg
 - declare.h, [829](#)
 - tools.c, [3096](#)
- compbuffer
 - SWITCHTABLE, [424](#)
- CompCoef
 - declare.h, [829](#)
 - reken.c, [2563](#)
- compcomm.c, [609](#), [633](#)
 - Add2ComStrings, [617](#)
 - AddComString, [617](#)
 - AddToCom, [617](#)
 - CoAntiBracket, [624](#)
 - CoAntiPutInside, [631](#)
 - CoAntiSymmetrize, [620](#)
 - CoArgExplode, [627](#)
 - CoArgImplode, [627](#)
 - CoArgToExtraSymbol, [628](#)
 - CoArgument, [618](#)
 - CoBracket, [624](#)
 - CoBreak, [631](#)
 - CoCase, [631](#)
 - CoChainin, [622](#)
 - CoChainout, [622](#)
 - CoChisholm, [621](#)
 - CoClearTable, [628](#)
 - CoClearUserFlag, [632](#)
 - CoCollect, [612](#)
 - CoCompress, [612](#)
 - CoContract, [617](#)
 - CoCreateAll, [632](#)
 - CoCreateAllLoops, [632](#)
 - CoCreateAllPaths, [632](#)
 - CoCycleSymmetrize, [620](#)
 - CoDefault, [631](#)
 - CoDelete, [613](#)
 - CoDenominators, [628](#)
 - CoDiscard, [617](#)
 - CoDo, [629](#)
 - CoDrop, [615](#)
 - CoDropCoefficient, [628](#)
 - CoDropSymbols, [628](#)
 - CoElse, [624](#)
 - CoElseif, [625](#)
 - CoEndArgument, [618](#)
 - CoEndDo, [629](#)
 - CoEndIf, [625](#)
 - CoEndInExpression, [623](#)
 - CoEndInside, [618](#)
 - CoEndRepeat, [623](#)
 - CoEndSwitch, [631](#)
 - CoEndTerm, [626](#)
 - CoEndWhile, [625](#)
 - CoExit, [622](#)
 - CoExtraSymbols, [629](#)
 - CoFactArg, [619](#)
 - CoFactDollar, [629](#)
 - CoFactorize, [630](#)
 - CoFindLoop, [625](#)
 - CoFixIndex, [614](#)
 - CoFlags, [613](#)
 - CoFormat, [612](#)
 - CoFromPolynomial, [628](#)
 - CoFunPowers, [626](#)
 - CoGoTo, [617](#)
 - CoHide, [616](#)
 - CoIf, [624](#)
 - CoInExpression, [623](#)

- CoInParallel, [622](#)
- CoInside, [618](#)
- CoInsideFirst, [613](#)
- CoIntoHide, [616](#)
- CoKeep, [613](#)
- CoLabel, [618](#)
- CoMakeInteger, [619](#)
- CoMerge, [627](#)
- CoMetric, [614](#)
- CoModulus, [623](#)
- CoMultiBracket, [624](#)
- CoNFactorize, [630](#)
- CoNoDrop, [615](#)
- CoNoHide, [616](#)
- CoNoIntoHide, [616](#)
- CoNormalize, [618](#)
- CoNoSkip, [616](#)
- CoNotInParallel, [622](#)
- CoNoUnHide, [616](#)
- CoNPrint, [614](#)
- CoNUnFactorize, [630](#)
- CoNWrite, [620](#)
- CoOff, [613](#)
- CoOn, [613](#)
- CoOptimizeOption, [630](#)
- CoPolyFun, [626](#)
- CoPolyRatFun, [626](#)
- CoPopHide, [615](#)
- CoPrint, [614](#)
- CoPrintB, [614](#)
- CoProcessBucket, [627](#)
- CoProperCount, [613](#)
- CoPushHide, [614](#)
- CoPutInside, [630](#)
- CoRatio, [620](#)
- CoRCycleSymmetrize, [620](#)
- CoRedefine, [620](#)
- CoRenumber, [621](#)
- CoRepeat, [623](#)
- CoReplaceLoop, [625](#)
- CoSetExitFlag, [623](#)
- CoSetUserFlag, [632](#)
- CoSkip, [615](#)
- CoSort, [626](#)
- CoSplitArg, [619](#)
- CoSplitFirstArg, [619](#)
- CoSplitLastArg, [619](#)
- CoStuff, [627](#)
- CoSum, [621](#)
- CoSwitch, [631](#)
- CoSymmetrize, [619](#)
- CoTerm, [626](#)
- CoThreadBucket, [627](#)
- CoToPolynomial, [628](#)
- CoToTensor, [621](#)
- CoToVector, [621](#)
- CoTrace4, [621](#)
- CoTraceN, [621](#)
- CoTryReplace, [623](#)
- CoUnFactorize, [630](#)
- CoUnHide, [616](#)
- CoUnitTrace, [626](#)
- CountComp, [624](#)
- CoWhile, [625](#)
- CoWrite, [620](#)
- DoArgPlode, [627](#)
- DoArgument, [618](#)
- DoBrackets, [624](#)
- DoChain, [622](#)
- DoFactorize, [630](#)
- DoFindLoop, [625](#)
- DoInParallel, [622](#)
- DoPrint, [614](#)
- DoPutInside, [631](#)
- DoSymmetrize, [619](#)
- GetDoParam, [629](#)
- GetIfDollarFactor, [629](#)
- SetExpr, [615](#)
- SetExprCases, [615](#)
- setonoff, [612](#)
- CompGroup
 - declare.h, [829](#)
 - function.c, [1472](#)
- CompileAlgebra
 - compiler.c, [725](#)
 - declare.h, [956](#)
- CompileLevel
 - C_const, [56](#)
- compiler.c, [722](#), [727](#)
 - alfatable1, [726](#)
 - CodeFactors, [726](#)
 - CodeGenerator, [725](#)
 - CompileAlgebra, [725](#)
 - CompileStatement, [725](#)
 - CompileSubExpressions, [725](#)
 - CompleteTerm, [726](#)
 - findcommand, [724](#)
 - GenerateFactors, [726](#)
 - inictable, [724](#)
 - insubexpbuffers, [727](#)
 - IsIdStatement, [725](#)
 - IsRHS, [724](#)
 - OPTION0, [723](#)
 - OPTION1, [723](#)
 - OPTION2, [723](#)
 - ParenthesesTest, [724](#)
 - REDUCESUBEXPBUFFERS, [724](#)
 - SkipAName, [724](#)
 - subexpbuffers, [726](#)
 - TestTables, [725](#)
 - topsubexpbuffers, [726](#)
- COMPILERBUFFER
 - fsizes.h, [1346](#)
- CompileStatement
 - compiler.c, [725](#)
 - declare.h, [919](#)

- CompileSubExpressions
 - compiler.c, [725](#)
 - declare.h, [959](#)
- compiletype
 - C_const, [54](#)
- COMPINC
 - fsizes.h, [1349](#)
- CompleteTerm
 - compiler.c, [726](#)
 - declare.h, [960](#)
- complex
 - FuNcTiOn, [145](#)
 - SyMbOl, [425](#)
 - VeCtOr, [484](#)
- compMat
 - MGraph, [208](#)
- ComposeTableNames
 - declare.h, [986](#)
 - minos.c, [1733](#)
- ComPress
 - declare.h, [922](#)
 - sort.c, [2728](#)
- compress.c, [753](#), [754](#)
- COMPRESSBUFFER
 - fsizes.h, [1346](#)
- CompressBuffer
 - R_const, [371](#)
- compression
 - ExPrEsSiOn, [123](#)
- CompressPointer
 - R_const, [372](#)
- CompressSize
 - InDeXeNtRy, [154](#)
 - M_const, [174](#)
- compressSize
 - N_const, [255](#)
- compressSpace
 - N_const, [251](#)
- ComprTop
 - R_const, [372](#)
- COMPTREE
 - structs.h, [2904](#)
- comsym
 - T_const, [436](#)
- comtool.c, [762](#), [768](#)
 - AddLHS, [764](#)
 - AddNtoC, [766](#)
 - AddNtoL, [765](#)
 - AddRHS, [765](#)
 - clearcbuf, [764](#)
 - ClearTree, [767](#)
 - DoubleCbuffer, [764](#)
 - FindTree, [767](#)
 - finishcbuf, [763](#)
 - inibufs, [763](#)
 - IniFbuffer, [767](#)
 - InsTree, [767](#)
 - numcommute, [768](#)
 - RedoTree, [767](#)
- comtool.h, [775](#), [776](#)
- CoMulti
 - comexpr.c, [583](#)
 - declare.h, [943](#)
- CoMultiBracket
 - compcomm.c, [624](#)
 - declare.h, [935](#)
- CoMultiply
 - comexpr.c, [584](#)
 - declare.h, [943](#)
- CoNFactorize
 - compcomm.c, [630](#)
 - declare.h, [939](#)
- CoNFunction
 - declare.h, [944](#)
 - names.c, [1833](#)
- conid
 - EEdge, [99](#)
- CONJUGATION
 - ftypes.h, [1398](#)
- connComp
 - EGraph, [105](#)
- connect
 - Assign, [19](#)
- connectClass
 - MGraph, [209](#)
- connectLeg
 - MGraph, [209](#)
- connectNode
 - MGraph, [209](#)
- connVisit
 - EGraph, [106](#)
- CoNoDrop
 - compcomm.c, [615](#)
 - declare.h, [944](#)
- CoNoHide
 - compcomm.c, [616](#)
 - declare.h, [949](#)
- CoNoIntoHide
 - compcomm.c, [616](#)
 - declare.h, [949](#)
- CoNormalize
 - compcomm.c, [618](#)
 - declare.h, [944](#)
- CoNoSkip
 - compcomm.c, [616](#)
 - declare.h, [944](#)
- CoNotInParallel
 - compcomm.c, [622](#)
 - declare.h, [934](#)
- CoNoUnHide
 - compcomm.c, [616](#)
 - declare.h, [949](#)
- CoNPrint
 - compcomm.c, [614](#)
 - declare.h, [944](#)
- ConstructName

- declare.h, [1026](#)
- pre.c, [2336](#)
- CoNTable
 - declare.h, [951](#)
 - names.c, [1834](#)
- CoNTensor
 - declare.h, [944](#)
 - names.c, [1833](#)
- ContentMerge
 - declare.h, [977](#)
 - execute.c, [1159](#)
- contents
 - DoLoOp, [91](#)
- CONTENTTERM
 - ftypes.h, [1402](#)
- CONTRACT
 - ftypes.h, [1417](#)
- CoNUnFactorize
 - compcomm.c, [630](#)
 - declare.h, [939](#)
- ConvertArgument
 - declare.h, [1011](#)
 - ratio.c, [2510](#)
- convertblock
 - declare.h, [983](#)
 - minos.c, [1731](#)
- ConvertFromPoly
 - declare.h, [1004](#)
 - notation.c, [1942](#)
- convertiniinfo
 - declare.h, [983](#)
 - minos.c, [1731](#)
- convertnamesblock
 - declare.h, [983](#)
 - minos.c, [1731](#)
- ConvertToPoly
 - declare.h, [1004](#)
 - notation.c, [1941](#)
- ConWord
 - declare.h, [901](#)
 - tools.c, [3089](#)
- CoNWrite
 - compcomm.c, [620](#)
 - declare.h, [944](#)
- CoOff
 - compcomm.c, [613](#)
 - declare.h, [945](#)
- CoOn
 - compcomm.c, [613](#)
 - declare.h, [945](#)
- CoOnce
 - comexpr.c, [584](#)
 - declare.h, [945](#)
- CoOnly
 - comexpr.c, [584](#)
 - declare.h, [945](#)
- CoOptimizeOption
 - compcomm.c, [630](#)
- declare.h, [945](#)
- CoParticle
 - declare.h, [1027](#)
- CoPolyFun
 - compcomm.c, [626](#)
 - declare.h, [946](#)
- CoPolyRatFun
 - compcomm.c, [626](#)
 - declare.h, [946](#)
- CoPopHide
 - compcomm.c, [615](#)
 - declare.h, [948](#)
- CoPrint
 - compcomm.c, [614](#)
 - declare.h, [946](#)
- CoPrintB
 - compcomm.c, [614](#)
 - declare.h, [946](#)
- CoPrintTable
 - comexpr.c, [585](#)
 - declare.h, [965](#)
- CoProcessBucket
 - compcomm.c, [627](#)
 - declare.h, [948](#)
- CoProperCount
 - compcomm.c, [613](#)
 - declare.h, [946](#)
- CoPushHide
 - compcomm.c, [614](#)
 - declare.h, [948](#)
- CoPutInside
 - compcomm.c, [630](#)
 - declare.h, [934](#)
- copy
 - EEdge, [97](#)
 - EGraph, [104](#)
 - ENode, [116](#)
 - MNodeClass, [221](#)
 - T_MNodeClass, [454](#)
- COPY1ARG
 - declare.h, [808](#)
- COPYARG
 - declare.h, [807](#)
- CopyArg
 - declare.h, [806](#)
- CopyExpression
 - declare.h, [987](#)
 - store.c, [2837](#)
- CopyFile
 - declare.h, [895](#)
 - tools.c, [3085](#)
- COPYFUN
 - declare.h, [807](#)
- COPYFUN3
 - declare.h, [807](#)
- COPYLONG
 - reken.c, [2555](#)
- COPYSUB

- declare.h, 808
- CopyTree
 - declare.h, 890
 - names.c, 1830
- CoRatio
 - compcomm.c, 620
 - declare.h, 947
- CoRCycleSymmetrize
 - compcomm.c, 620
 - declare.h, 946
- CoRedefine
 - compcomm.c, 620
 - declare.h, 947
- CoRemoveSpectator
 - declare.h, 1023
 - spectator.c, 2791
- CoRenumber
 - compcomm.c, 621
 - declare.h, 947
- CoRepeat
 - compcomm.c, 623
 - declare.h, 947
- CoReplaceLoop
 - compcomm.c, 625
 - declare.h, 963
- CoSave
 - declare.h, 947
 - store.c, 2833
- CoSelect
 - comexpr.c, 584
 - declare.h, 947
- CoSet
 - declare.h, 947
 - names.c, 1834
- CoSetExitFlag
 - compcomm.c, 623
 - declare.h, 948
- CoSetUserFlag
 - compcomm.c, 632
 - declare.h, 1028
- COSFUNCTION
 - ftypes.h, 1400
- COSHFUNCTION
 - ftypes.h, 1401
- CoSkip
 - compcomm.c, 615
 - declare.h, 948
- CoSort
 - compcomm.c, 626
 - declare.h, 949
- CoSplitArg
 - compcomm.c, 619
 - declare.h, 949
- CoSplitFirstArg
 - compcomm.c, 619
 - declare.h, 949
- CoSplitLastArg
 - compcomm.c, 619
 - declare.h, 950
- COST, 75
 - add, 76
 - div, 76
 - mul, 76
 - pow, 76
- CoShuffle
 - compcomm.c, 627
 - declare.h, 943
- CoSum
 - compcomm.c, 621
 - declare.h, 950
- CoSwitch
 - compcomm.c, 631
 - declare.h, 953
- CoSymbol
 - declare.h, 950
 - names.c, 1831
- CoSymmetrize
 - compcomm.c, 619
 - declare.h, 950
- CoTable
 - declare.h, 950
 - names.c, 1833
- CoTableBase
 - declare.h, 936
 - tables.c, 2966
- CoTBaddto
 - declare.h, 954
 - tables.c, 2966
- CoTBaudit
 - declare.h, 954
 - tables.c, 2967
- CoTBCleanup
 - declare.h, 954
 - tables.c, 2968
- CoTBcreate
 - declare.h, 954
 - tables.c, 2966
- CoTBenter
 - declare.h, 954
 - tables.c, 2967
- CoTBhelp
 - declare.h, 955
 - tables.c, 2968
- CoTBload
 - declare.h, 955
 - tables.c, 2967
- CoTBoff
 - declare.h, 955
 - tables.c, 2968
- CoTBon
 - declare.h, 955
 - tables.c, 2967
- CoTBopen
 - declare.h, 955
 - tables.c, 2966
- CoTBreplace

- declare.h, 955
- tables.c, 2968
- CoTBus
 - declare.h, 955
 - tables.c, 2968
- CoTerm
 - compcomm.c, 626
 - declare.h, 950
- CoTestUse
 - declare.h, 956
 - tables.c, 2967
- CoThreadBucket
 - compcomm.c, 627
 - declare.h, 956
- CoToFloat
 - float.c, 1293
- CoToPolynomial
 - compcomm.c, 628
 - declare.h, 1005
- CoToRat
 - float.c, 1293
- CoToSpectator
 - declare.h, 1023
 - spectator.c, 2791
- CoToTensor
 - compcomm.c, 621
 - declare.h, 951
- CoToVector
 - compcomm.c, 621
 - declare.h, 951
- CoTrace4
 - compcomm.c, 621
 - declare.h, 951
- CoTraceN
 - compcomm.c, 621
 - declare.h, 951
- CoTransform
 - declare.h, 952
 - transform.c, 3151
- CoTryReplace
 - compcomm.c, 623
 - declare.h, 952
- COULDCOMMUTE
 - ftypes.h, 1452
- CoUnFactorize
 - compcomm.c, 630
 - declare.h, 939
- CoUnHide
 - compcomm.c, 616
 - declare.h, 949
- CoUnitTrace
 - compcomm.c, 626
 - declare.h, 946
- count
 - PNodeClass, 336
- count_operators
 - optimize.cc, 1993, 1994
- count_operators_cse
 - optimize.cc, 1999
- count_operators_cse_topdown
 - optimize.cc, 2000
- CountComp
 - compcomm.c, 624
 - declare.h, 932
- CountDo
 - declare.h, 831
 - reshuf.c, 2614
- counter
 - ExPrEsSiOn, 122
- CountFun
 - declare.h, 831
 - reshuf.c, 2614
- COUNTFUNCTION
 - ftypes.h, 1397
- countfunnum
 - M_const, 189
- CountTerms1
 - declare.h, 982
 - execute.c, 1160
- coupl
 - FGInput, 127
- couple
 - PNodeClass, 336
- couplings
 - MoDeL, 233
 - VeRtEx, 485
- CoVector
 - declare.h, 953
 - names.c, 1832
- CoVertex
 - declare.h, 1028
- CoWhile
 - compcomm.c, 625
 - declare.h, 953
- CoWrite
 - compcomm.c, 620
 - declare.h, 953
- cple
 - NCInput, 269
 - PGInput, 331
- cplgcp
 - Model, 231
- cplglg
 - Model, 231
- cplgnvl
 - Model, 231
- cplgvl
 - Model, 231
- cpilleft
 - Assign, 25
- cprocedureExtension
 - P_const, 312
- CRANGE
 - ftypes.h, 1380
- Crash
 - declare.h, 982

- tools.c, [3097](#)
- CreateExpression
 - declare.h, [1010](#)
 - ratio.c, [2508](#)
- CreateFile
 - declare.h, [894](#)
 - tools.c, [3085](#)
- CreateHandle
 - declare.h, [895](#)
 - tools.c, [3085](#)
- CreateLogFile
 - declare.h, [894](#)
 - tools.c, [3085](#)
- CreateStream
 - declare.h, [927](#)
 - tools.c, [3084](#)
- CreateVertex
 - declare.h, [1027](#)
- csav
 - SGroup, [388](#)
- CSEEq, [76](#)
 - operator(), [77](#)
- CSEHash, [77](#)
 - operator(), [77](#)
- CSET
 - ftypes.h, [1379](#)
- CSUBEXP
 - ftypes.h, [1380](#)
- csum
 - Interaction, [161](#)
- CSYMBOL
 - ftypes.h, [1379](#)
- cTable
 - FixedGlobals, [137](#)
- CTD
 - minos.c, [1730](#)
- cTerm
 - N_const, [246](#)
- ctloop
 - MCOpI, [204](#)
- ctotal
 - Process, [365](#)
- ctyp
 - PGInput, [331](#)
- CurBufWrt
 - O_const, [280](#)
- curcl
 - MGraph, [212](#)
 - T_MGraph, [450](#)
- CurDictFunWithArgs
 - O_const, [283](#)
- CurDictInDollars
 - O_const, [284](#)
- CurDictNotInFunctions
 - O_const, [284](#)
- CurDictNumbers
 - O_const, [283](#)
- CurDictNumberWarning
 - O_const, [283](#)
- CurDictSpecials
 - O_const, [283](#)
- CurDictVariables
 - O_const, [283](#)
- curdirp
 - setfile.c, [2696](#)
- CurDum
 - R_const, [374](#)
- CurExpr
 - R_const, [377](#)
- current
 - TabIEbAsE, [457](#)
- CURRENTBRACKET
 - execute.c, [1155](#)
- CurrentDictionary
 - O_const, [283](#)
- currentExternalChannel
 - X_const, [488](#)
- currentMODEL
 - TERMINFO, [466](#)
- currentModel
 - TERMINFO, [466](#)
- currentPrompt
 - X_const, [487](#)
- CurrentStream
 - C_const, [46](#)
- currentTerm
 - N_const, [251](#)
- cursortdirp
 - setfile.c, [2696](#)
- cut
 - EEdge, [99](#)
- cvallist
 - lInput, [150](#)
- CVECTOR
 - ftypes.h, [1379](#)
- CVECTOR1
 - ftypes.h, [1380](#)
- cycle
 - DiStRiBuTe, [86](#)
 - PeRmUtE, [327](#)
 - PeRmUtEp, [328](#)
- CYCLE1
 - declare.h, [806](#)
- CYCLEARG
 - ftypes.h, [1448](#)
- CYCLESYMMETRIC
 - ftypes.h, [1433](#)
- d
 - flint::fmpz, [139](#)
 - flint::mpoly, [240](#)
 - flint::mpoly_ctx, [241](#)
 - flint::mpoly_factor, [242](#)
 - flint::poly, [338](#)
 - flint::poly_factor, [357](#)
- DAEMON
 - pre.c, [2319](#)

- daemonize
 - X_const, [488](#)
- data
 - node, [273](#)
- date
 - objects, [287](#)
- dbase, [78](#)
 - fullname, [80](#)
 - handle, [79](#)
 - iblocks, [79](#)
 - info, [78](#)
 - mode, [78](#)
 - name, [79](#)
 - nblocks, [79](#)
 - rwmode, [79](#)
 - tablenamefill, [79](#)
 - tablenames, [80](#)
 - tablenamessize, [79](#)
 - topnumber, [79](#)
- DBGOUT
 - parallel.c, [2068](#)
- DBGOUT_NINTERMS
 - parallel.c, [2068](#)
- dbufnum
 - M_const, [179](#)
- DeAllocFileHandle
 - declare.h, [972](#)
 - setfile.c, [2695](#)
- DEBUG
 - wildcard.c, [3227](#)
- DebugFlag
 - P_const, [316](#)
- debugFlags
 - C_const, [70](#)
- debugflags
 - OPTIMIZE, [294](#)
- DECLARATION
 - ftypes.h, [1376](#)
- declare.h, [777](#), [1031](#)
 - ABS, [800](#)
 - AccumGCD, [824](#)
 - Add2Com, [814](#)
 - Add2ComStrings, [927](#)
 - ADD2POS, [812](#)
 - Add3Com, [815](#)
 - Add4Com, [815](#)
 - Add5Com, [815](#)
 - AddArgs, [824](#)
 - AddArrayIndex, [822](#)
 - AddCoef, [824](#)
 - AddComString, [956](#)
 - AddDictionary, [1020](#)
 - AddDollar, [891](#)
 - AddDubious, [892](#)
 - AddExpression, [891](#)
 - AddFunction, [892](#)
 - AddIndex, [892](#)
 - AddLHS, [924](#)
 - AddLineFeed, [802](#)
 - AddLong, [825](#)
 - AddName, [888](#)
 - AddNtoC, [926](#)
 - AddNtoL, [925](#)
 - AddObject, [986](#)
 - AddPLon, [825](#)
 - AddPoly, [825](#)
 - ADDPOS, [811](#)
 - AddPotModdollar, [967](#)
 - AddRat, [826](#)
 - AddRHS, [925](#)
 - AddSet, [893](#)
 - AddSymbol, [891](#)
 - AddTableName, [986](#)
 - AddToCB, [806](#)
 - AddToCom, [927](#)
 - AddToDictionary, [1020](#)
 - AddToDollarBuffer, [961](#)
 - AddToIndex, [984](#)
 - AddToLine, [827](#)
 - AddToPreTypes, [923](#)
 - AddToScratch, [1024](#)
 - AddToString, [915](#)
 - AddVector, [892](#)
 - AddWild, [827](#)
 - AddWildcardName, [923](#)
 - AdjustRenumScratch, [846](#)
 - AllLoops, [1029](#)
 - AllocFileHandle, [972](#)
 - AllocSetups, [902](#)
 - AllocSort, [903](#)
 - AllocSortFileName, [903](#)
 - AllPaths, [1030](#)
 - Apply, [838](#)
 - ApplyExec, [838](#)
 - ApplyReset, [838](#)
 - AreArgsEqual, [900](#)
 - ArgDotproductMerge, [931](#)
 - ArgFactorize, [928](#)
 - ArgSymbolMerge, [931](#)
 - ArgumentExplode, [981](#)
 - ArgumentImplode, [981](#)
 - AssignDollar, [961](#)
 - BACKINOUT, [806](#)
 - BASEPOSITION, [812](#)
 - Bernoulli, [843](#)
 - BigLong, [827](#)
 - BinarySearch, [973](#)
 - BinomGen, [827](#)
 - BracketNormalize, [857](#)
 - CacheNumberFree, [817](#)
 - CacheNumberMalloc, [816](#)
 - CacheNumberMallocAddMemory, [999](#)
 - CatchDollar, [961](#)
 - ChainIn, [981](#)
 - ChainOut, [981](#)
 - CharOut, [904](#)

CheckRecoveryFile, 996
CheckTableDeclarations, 837
CheckWild, 827
Chisholm, 828
CleanDollarFactors, 965
CleanExpr, 828
CleanUp, 828
CleanupArgCache, 930
CleanupSort, 972
CleanupTerm, 976
ClearBracketIndex, 979
clearcbuf, 971
ClearMacro, 906
ClearOptimize, 969
ClearPushback, 903
ClearSpectators, 1023
ClearTableTree, 970
ClearTree, 961
ClearWild, 828
ClearWildcardNames, 923
closeAllExternalChannels, 989
CloseChannel, 897
closeExternalChannel, 989
CloseFile, 894
CloseStream, 887
CmpArray, 901
CoAntiBracket, 932
CoAntiPutInside, 935
CoAntiSymmetrize, 933
CoApply, 936
CoArgExplode, 933
CoArgImplode, 933
CoArgToExtraSymbol, 1006
CoArgument, 933
CoAssign, 959
CoAuto, 953
CoBracket, 934
CoBreak, 953
CoCanonicalize, 834
CoCase, 953
CoCFunction, 935
CoChainin, 952
CoChainout, 952
CoChisholm, 952
CoClearTable, 952
CoClearUserFlag, 1028
CoCollect, 935
CoCommutelnSet, 893
CoCompress, 935
CoContract, 935
CoCopySpectator, 1023
CoCreateAll, 1029
CoCreateAllLoops, 1028
CoCreateAllPaths, 1028
CoCreateSpectator, 1022
CoCTable, 951
CoCTensor, 935
CoCycleSymmetrize, 936
CoDeallocateTable, 965
CodeFactors, 960
CoDefault, 954
CodeGenerator, 959
CoDelete, 936
CoDenominators, 936
CodeToLine, 822
CoDimension, 936
CoDiscard, 936
CoDisorder, 937
CoDo, 1006
CoDrop, 937
CoDropCoefficient, 937
CoDropSymbols, 937
CoElse, 937
CoElseif, 937
CoEmptySpectator, 1023
CoEndArgument, 937
CoEndDo, 1006
CoEndIf, 938
CoEndInExpression, 934
CoEndInside, 938
CoEndModel, 1028
CoEndRepeat, 938
CoEndSwitch, 954
CoEndTerm, 938
CoEndWhile, 938
CoExit, 938
CoExtraSymbols, 1006
CoFactArg, 938
CoFactDollar, 939
CoFactorize, 939
CoFill, 939
CoFillExpression, 940
CoFindLoop, 963
CoFixIndex, 940
CoFlags, 945
CoFormat, 940
CoFromPolynomial, 1005
CoFunction, 893
CoFunPowers, 963
CoGlobal, 940
CoGlobalFactorized, 940
CoGoTo, 940
CoHide, 948
Cold, 940
ColdExpression, 959
ColdNew, 941
ColdOld, 941
Colf, 941
ColfMatch, 941
ColfNoMatch, 941
ColIndex, 941
ColInExpression, 934
ColInParallel, 934
ColInside, 933
ColInsideFirst, 941
ColIntoHide, 948

CoKeep, [942](#)
CoLabel, [942](#)
CoLoad, [942](#)
CoLocal, [942](#)
CoLocalFactorized, [942](#)
CoMakeInteger, [945](#)
CoMany, [942](#)
CoMerge, [942](#)
CoMetric, [943](#)
Commute, [828](#)
CoModel, [1027](#)
CoModOption, [943](#)
CoModuleOption, [943](#)
CoModulus, [943](#)
CompactifyTree, [890](#)
Compare1, [830](#)
CompareArgs, [900](#)
CompareFunctions, [828](#)
CompareHSymbols, [990](#)
CompareSymbols, [990](#)
CompareTerms, [820](#)
CompArg, [829](#)
CompCoef, [829](#)
CompGroup, [829](#)
CompileAlgebra, [956](#)
CompileStatement, [919](#)
CompileSubExpressions, [959](#)
CompleteTerm, [960](#)
ComposeTableNames, [986](#)
ComPress, [922](#)
CoMulti, [943](#)
CoMultiBracket, [935](#)
CoMultiply, [943](#)
CoNFactorize, [939](#)
CoNFunction, [944](#)
CoNoDrop, [944](#)
CoNoHide, [949](#)
CoNoIntoHide, [949](#)
CoNormalize, [944](#)
CoNoSkip, [944](#)
CoNotInParallel, [934](#)
CoNoUnHide, [949](#)
CoNPrint, [944](#)
ConstructName, [1026](#)
CoNTable, [951](#)
CoNTensor, [944](#)
ContentMerge, [977](#)
CoNUnFactorize, [939](#)
ConvertArgument, [1011](#)
convertblock, [983](#)
ConvertFromPoly, [1004](#)
convertiniinfo, [983](#)
convertnamesblock, [983](#)
ConvertToPoly, [1004](#)
ConWord, [901](#)
CoNWrite, [944](#)
CoOff, [945](#)
CoOn, [945](#)
CoOnce, [945](#)
CoOnly, [945](#)
CoOptimizeOption, [945](#)
CoParticle, [1027](#)
CoPolyFun, [946](#)
CoPolyRatFun, [946](#)
CoPopHide, [948](#)
CoPrint, [946](#)
CoPrintB, [946](#)
CoPrintTable, [965](#)
CoProcessBucket, [948](#)
CoProperCount, [946](#)
CoPushHide, [948](#)
CoPutInside, [934](#)
COPY1ARG, [808](#)
COPYARG, [807](#)
CopyArg, [806](#)
CopyExpression, [987](#)
CopyFile, [895](#)
COPYFUN, [807](#)
COPYFUN3, [807](#)
COPYSUB, [808](#)
CopyTree, [890](#)
CoRatio, [947](#)
CoRCycleSymmetrize, [946](#)
CoRedefine, [947](#)
CoRemoveSpectator, [1023](#)
CoRenumber, [947](#)
CoRepeat, [947](#)
CoReplaceLoop, [963](#)
CoSave, [947](#)
CoSelect, [947](#)
CoSet, [947](#)
CoSetExitFlag, [948](#)
CoSetUserFlag, [1028](#)
CoSkip, [948](#)
CoSort, [949](#)
CoSplitArg, [949](#)
CoSplitFirstArg, [949](#)
CoSplitLastArg, [950](#)
CoStuffle, [943](#)
CoSum, [950](#)
CoSwitch, [953](#)
CoSymbol, [950](#)
CoSymmetrize, [950](#)
CoTable, [950](#)
CoTableBase, [936](#)
CoTBaddto, [954](#)
CoTBaudit, [954](#)
CoTBCleanup, [954](#)
CoTBcreate, [954](#)
CoTBenter, [954](#)
CoTBhelp, [955](#)
CoTBload, [955](#)
CoTBoff, [955](#)
CoTBon, [955](#)
CoTBopen, [955](#)
CoTBreplace, [955](#)

CoTBuse, [955](#)
CoTerm, [950](#)
CoTestUse, [956](#)
CoThreadBucket, [956](#)
CoToPolynomial, [1005](#)
CoToSpectator, [1023](#)
CoToTensor, [951](#)
CoToVector, [951](#)
CoTrace4, [951](#)
CoTraceN, [951](#)
CoTransform, [952](#)
CoTryReplace, [952](#)
CoUnFactorize, [939](#)
CoUnHide, [949](#)
CoUnitTrace, [946](#)
CountComp, [932](#)
CountDo, [831](#)
CountFun, [831](#)
CountTerms1, [982](#)
CoVector, [953](#)
CoVertex, [1028](#)
CoWhile, [953](#)
CoWrite, [953](#)
Crash, [982](#)
CreateExpression, [1010](#)
CreateFile, [894](#)
CreateHandle, [895](#)
CreateLogFile, [894](#)
CreateStream, [927](#)
CreateVertex, [1027](#)
CYCLE1, [806](#)
DeAllocFileHandle, [972](#)
Deferred, [831](#)
defineChannel, [989](#)
DeleteObject, [985](#)
DeleteRecoveryFile, [997](#)
DeleteStore, [832](#)
DenToFunction, [982](#)
DetCommu, [829](#)
DetCurDum, [832](#)
DetVars, [832](#)
DictFromBytes, [1022](#)
DictToBytes, [1022](#)
DIFBASE, [812](#)
DIFPOS, [812](#)
DimensionExpression, [831](#)
DimensionSubterm, [831](#)
DimensionTerm, [831](#)
Distribute, [833](#)
DistrN, [1026](#)
DIVfunction, [1010](#)
DivLong, [833](#)
DIVPOS, [811](#)
DivRat, [833](#)
Divvy, [833](#)
DoArgPlode, [933](#)
DoArgument, [928](#)
DoBrackets, [932](#)
DoBreakDo, [913](#)
DoCall, [912](#)
DoCanonicalize, [834](#)
DoChain, [952](#)
DoCheckpoint, [998](#)
DoClearOptimize, [909](#)
DoClearUserFlag, [1027](#)
DoCommentChar, [915](#)
DoContinueDo, [913](#)
DoDebug, [913](#)
DoDefine, [905](#)
DoDelta, [833](#)
DoDelta3, [834](#)
DoDimension, [892](#)
DoDistrib, [836](#)
DoDo, [913](#)
DoElements, [893](#)
DoElse, [912](#)
DoElseif, [912](#)
DoEnddo, [913](#)
DoEndif, [912](#)
DoEndInside, [914](#)
DoEndNamespace, [1026](#)
DoEndprocedure, [913](#)
DoEndSwitch, [1025](#)
DoesCommu, [829](#)
DoExecStatement, [919](#)
DoExecute, [884](#)
DoExpr, [959](#)
DoExternal, [907](#)
DoFactDollar, [908](#)
DoFactorize, [939](#)
DoFindLoop, [963](#)
DoFromExternal, [907](#)
Dolf, [912](#)
Dolfdef, [911](#)
Dolfndef, [912](#)
DolfStatement, [839](#)
Dolfydef, [911](#)
DoInclude, [907](#)
DoInParallel, [934](#)
DoinParallel, [980](#)
DoInside, [913](#)
DollarFactorize, [965](#)
DOLLARNAME, [810](#)
DollarRaiseLow, [892](#)
DolToFunction, [964](#)
DolToIndex, [965](#)
DolToLong, [965](#)
DolToNumber, [964](#)
DolToSymbol, [964](#)
DolToTensor, [964](#)
DolToTerms, [973](#)
DolToVector, [964](#)
DoMessage, [909](#)
DoModDollar, [980](#)
DoModLocal, [980](#)
DoModMax, [980](#)

- DoModMin, [980](#)
- DoModSum, [980](#)
- DoNamespace, [1026](#)
- DoNoParallel, [979](#)
- DonotinParallel, [981](#)
- DoOnePow, [839](#)
- DoOptimize, [909](#)
- DoParallel, [979](#)
- DoPartitions, [834](#)
- DoPermutations, [836](#)
- DoPipe, [911](#)
- DoPipeStatement, [919](#)
- DoPolyfun, [919](#)
- DoPolyratfun, [919](#)
- DoPrcExtension, [915](#)
- DoPreAdd, [1019](#)
- DoPreAddSeparator, [988](#)
- DoPreAppend, [909](#)
- DoPreAppendPath, [1024](#)
- DoPreAssign, [910](#)
- DoPreBreak, [910](#)
- DoPreCase, [910](#)
- DoPreClose, [914](#)
- DoPreCloseDictionary, [1019](#)
- DoPreCreate, [910](#)
- DoPreDefault, [910](#)
- DoPreEndSwitch, [910](#)
- DoPreExchange, [911](#)
- DoPreOpenDictionary, [1019](#)
- DoPreOut, [909](#)
- DoPrePrependPath, [1024](#)
- DoPrePrintTimes, [914](#)
- DoPreRemove, [914](#)
- DoPreReset, [915](#)
- DoPreRmSeparator, [988](#)
- DoPreShow, [911](#)
- DoPreSortReallocate, [914](#)
- DoPreSwitch, [910](#)
- DoPreUseDictionary, [1019](#)
- DoPreWrite, [914](#)
- DoPrint, [927](#)
- DoProcedure, [914](#)
- DoProcessBucket, [980](#)
- DoPrompt, [908](#)
- DoPutInside, [932](#)
- DoRecovery, [997](#)
- DoRedefine, [905](#)
- DoReverseInclude, [907](#)
- DoRevert, [840](#)
- DoRmExternal, [908](#)
- DoSetExternal, [908](#)
- DoSetExternalAttr, [908](#)
- DoSetRandom, [908](#)
- DoSetups, [888](#)
- DoSetUserFlag, [1027](#)
- DoShattering, [835](#)
- DoShuffle, [820](#)
- DoShuffle, [836](#)
- DoSkipExtraSymbols, [909](#)
- DoStuffle, [836](#)
- DoSumF1, [840](#)
- DoSumF2, [840](#)
- DoSwitch, [1025](#)
- DoSymmetrize, [928](#)
- DoSystem, [911](#)
- DoTable, [950](#)
- DoTableExpansion, [835](#)
- DoTail, [885](#)
- DoTempSet, [893](#)
- DoTerminate, [912](#)
- DoTheta, [841](#)
- DoTimeOutAfter, [909](#)
- DoToExternal, [907](#)
- DoTopologyCanonicalize, [835](#)
- DoubleBuffer, [899](#)
- DoubleCbuffer, [924](#)
- DoubleIfBuffers, [927](#)
- DoubleList, [899](#)
- DoubleLList, [899](#)
- DoubleSwitchBuffers, [1025](#)
- DoUndefine, [907](#)
- DoUse, [1026](#)
- DropCoefficient, [857](#)
- DropSymbols, [857](#)
- DUMMYUSE, [810](#)
- DumpNode, [890](#)
- DumpTree, [890](#)
- EmptyTable, [951](#)
- EndOfToken, [918](#)
- EndSort, [841](#)
- EntVar, [842](#)
- EpfCon, [842](#)
- EpfFind, [842](#)
- EpfGen, [842](#)
- EqualArg, [843](#)
- Error0, [886](#)
- Error1, [886](#)
- Error2, [886](#)
- EvalDoLoopArg, [973](#)
- EvalPrelf, [917](#)
- EXCH, [801](#)
- ExchangeDollars, [975](#)
- ExchangeExpressions, [975](#)
- EXCHINOUT, [806](#)
- EXCHN, [801](#)
- execarg, [883](#)
- ExecInside, [933](#)
- ExecModule, [918](#)
- ExecStore, [918](#)
- execterm, [883](#)
- ExistsObject, [985](#)
- ExpandBuffer, [899](#)
- ExpandRat, [1011](#)
- ExpandTripleDots, [921](#)
- EXPRNAME, [810](#)
- ExprStatus, [999](#)

EXTERNLOCK, [818](#)
EXTERNRWLOCK, [819](#)
ExtraSymbol, [856](#)
ExtraSymFun, [1007](#)
Factorial, [843](#)
FactorIn, [843](#)
FactorInExpr, [843](#)
FILLARG, [807](#)
FILLEXP, [808](#)
FILLFUN, [807](#)
FILLFUN3, [808](#)
FILLSUB, [808](#)
FindAll, [844](#)
FindArg, [930](#)
FindBracket, [979](#)
FindCase, [1025](#)
findcommand, [897](#)
FindDictionary, [1020](#)
FindExtraSymbol, [1022](#)
FindFunction, [1022](#)
FindFunWithArgs, [1022](#)
FindIndex, [1022](#)
FindInIndex, [823](#)
FindInKeyWord, [905](#)
FindKeyWord, [905](#)
FindLocalSubterm, [1004](#)
FindLus, [963](#)
FindMulti, [844](#)
FindOnce, [844](#)
FindOnly, [844](#)
FindRange, [1001](#)
FindRest, [844](#)
FindrNumber, [844](#)
FindSubexpression, [1005](#)
FindSubterm, [1004](#)
FindSymbol, [1021](#)
FindTableNumber, [987](#)
FindTableTree, [971](#)
FindTB, [837](#)
FindTree, [960](#)
FindVar, [839](#)
FindVector, [1021](#)
FiniLine, [845](#)
finishcbuf, [971](#)
FinishShuffle, [836](#)
FinishStuffle, [837](#)
FiniShuffle, [820](#)
FiniTerm, [845](#)
FlipTable, [981](#)
FLUSHCONSOLE, [804](#)
FlushFile, [896](#)
FlushOut, [845](#)
FlushSpectators, [1024](#)
FORM_INLINE, [816](#)
FreeNameTree, [891](#)
FreeTableBase, [986](#)
FromOList, [898](#)
FromList, [898](#)
FromPolyRatFun, [1008](#)
FromVarList, [899](#)
FullCleanUp, [918](#)
FullRenummer, [962](#)
FullSymmetrize, [874](#)
FunLevel, [846](#)
FunMatchCy, [853](#)
FunMatchSy, [853](#)
GarbHand, [846](#)
GCDclean, [1008](#)
GCDfunction, [1007](#)
GCDfunction3, [1007](#)
GcdLong, [846](#)
GCDterms, [1008](#)
GenDiagrams, [835](#)
GenerateFactors, [960](#)
GenerateTopologies, [835](#)
Generator, [847](#)
GenLoops, [1029](#)
GenPaths, [1030](#)
GenTopologies, [834](#)
GetAppendChannel, [897](#)
GetAutoName, [889](#)
GETBIDENTITY, [820](#)
GetBinom, [848](#)
GETCFROMEXTCHANNEL, [821](#)
GetChannel, [897](#)
GetChar, [903](#)
GETCOEF, [801](#)
GetContent, [976](#)
getCurrentExternalChannel, [989](#)
GetDbase, [984](#)
GetDollar, [919](#)
GetDollarNumber, [908](#)
GetDoINum, [967](#)
GetDoParam, [1006](#)
GetFirstBracket, [976](#)
GetFirstTerm, [976](#)
GetFromSpectator, [1023](#)
GetFromStore, [848](#)
GetFromStream, [886](#)
GetFunction, [889](#)
GETIDENTITY, [820](#)
GetIfDollarFactor, [1006](#)
GetIfDollarNum, [838](#)
GetInput, [903](#)
GetLabel, [959](#)
GetLastExprName, [889](#)
GetLong, [848](#)
GetLongModInverses, [867](#)
GetModInverses, [866](#)
GetMoreFromMem, [849](#)
GetMoreTerms, [849](#)
GetName, [888](#)
GetNode, [888](#)
GetNumber, [889](#)
GetOName, [890](#)
GetOneTerm, [849](#)

- GetPosFile, 896
- GetPreVar, 901
- GetRunningTime, 863
- GetSetupPar, 902
- GETSTOP, 801
- GetStreamPosition, 924
- GetTable, 849
- GetTableName, 987
- GETTERM, 821
- GetTerm, 849
- GetVar, 889
- GetWildcardName, 923
- Globalize, 923
- Glue, 849
- HighWarning, 884
- HowMany, 982
- iexp, 899
- INCLENG, 801
- Include, 907
- InFunction, 850
- inibufs, 897
- inictable, 897
- IniBuffer, 1007
- IniLine, 851
- INILOCK, 818
- IniModule, 904
- INIRWLOCK, 819
- IniSpecialModule, 904
- iniTools, 999
- initPresetExternalChannels, 988
- InitRecovery, 996
- IniVars, 851
- iniwranf, 991
- InsertArg, 930
- InsertTerm, 851
- InsideDollar, 973
- InsTableTree, 971
- InsTree, 960
- InvPoly, 1011
- iranf, 991
- ISEQUALPOS, 813
- ISEQUALPOSINC, 811
- IsExponentSign, 1020
- ISGEPOS, 814
- ISGEPOSINC, 811
- IsIdStatement, 956
- ISLESSPOS, 813
- IsLikeVector, 900
- ISMINPOS, 813
- IsMultipleOf, 978
- IsMultiplySign, 1021
- ISNEGPOS, 814
- ISNOTEQUALPOS, 813
- ISNOTZEROPOS, 814
- ISPOSPOS, 814
- IsRHS, 956
- IsSetMember, 978
- ISZEROPOS, 814
- LcmLong, 847
- LinkTree, 890
- LoadInputFile, 903
- LoadInstruction, 905
- LoadModel, 1028
- LoadStatement, 905
- LocalConvertToPoly, 1004
- LocateBase, 871
- LocateFile, 887
- LOCK, 818
- LongCopy, 920
- LongLongCopy, 920
- LongToLine, 852
- LookInStream, 887
- LoopOutput, 1029
- LowerSortLevel, 972
- Lus, 962
- M_alloc, 820
- M_check1, 978
- M_free, 922
- M_print, 978
- MakeDate, 898
- MakeDirty, 852
- MakeDollarInteger, 966
- MakeDollarMod, 966
- MakeDubious, 889
- MakeGlobal, 918
- MakeInteger, 929
- MakeInverses, 866
- MakeLongRational, 970
- MakeMod, 929
- MakeModTable, 853
- MakeNameTree, 891
- MakeRational, 970
- MakeSetupAllocs, 975
- Malloc, 885
- Malloc1, 885
- MarkDirty, 852
- MarkPolyRatFunDirty, 817
- MatchArgument, 853
- MatchCy, 853
- MatchE, 853
- MatchFunction, 854
- MatchIsPossible, 964
- MaX, 800
- MEMORYMACROS, 816
- MergeDotproductLists, 1010
- MergePatches, 854
- MergeSymbolLists, 1010
- MesCall, 920
- MesCerr, 855
- MesComp, 855
- MesNesting, 817
- MessPreNesting, 923
- MesWork, 920
- Min, 800
- MLOCK, 819
- ModuleInstruction, 904

Modulus, [855](#)
Moebius, [991](#)
MoveDummies, [855](#)
MULfunc, [1010](#)
MulLong, [855](#)
Mully, [856](#)
MULPOS, [812](#)
MulRat, [856](#)
MultDo, [856](#)
MultiplyToLine, [1021](#)
MultiplyWithTerm, [1009](#)
MUNLOCK, [820](#)
NameConflict, [894](#)
NCOPY, [803](#)
NCOPYB, [803](#)
NCOPYI, [803](#)
NCOPYI32, [804](#)
NeedNumber, [804](#)
NestingChecksum, [817](#)
NewDbase, [986](#)
NewSort, [856](#)
NEXTARG, [808](#)
NextFileIndex, [823](#)
NextPrime, [991](#)
Normalize, [857](#)
NormalModulus, [866](#)
NormPolyTerm, [1003](#)
NOTSTARTPOS, [813](#)
NumberFree, [817](#)
NumberMalloc, [816](#)
NumberMallocAddMemory, [998](#)
numcommute, [962](#)
NumCopy, [920](#)
NumToStr, [916](#)
OpenAddFile, [894](#)
OpenBracketIndex, [979](#)
OpenDbase, [985](#)
openExternalChannel, [988](#)
OpenFile, [894](#)
OpenInput, [885](#)
OpenStream, [887](#)
OpenTemp, [857](#)
Optimize, [968](#)
optimize_print_code, [1019](#)
Pack, [858](#)
ParenthesesTest, [957](#)
ParseNumber, [802](#)
ParseSign, [803](#)
ParseSignedNumber, [803](#)
PasteFile, [858](#)
PasteTerm, [823](#)
PathOutput, [1030](#)
Permute, [858](#)
PermuteP, [859](#)
poly_div, [1012](#)
poly_factorize_argument, [1015](#)
poly_factorize_dollar, [1016](#)
poly_factorize_expression, [1016](#)
poly_free_poly_vars, [1018](#)
poly_gcd, [1011](#)
poly_inverse, [1013](#)
poly_mul, [1013](#)
poly_ratfun_add, [1013](#)
poly_ratfun_normalize, [1014](#)
poly_rem, [1012](#)
poly_unfactorize_expression, [1017](#)
PolyDiv, [1008](#)
PolyFunClean, [852](#)
PolyFunDirty, [852](#)
PolyFunMul, [859](#)
PolyRatFunSpecial, [972](#)
POPPREASSIGNLEVEL, [818](#)
PopPreVars, [904](#)
PopVariables, [860](#)
PositionStream, [887](#)
pParseObject, [917](#)
PreCalc, [916](#)
PreCmp, [916](#)
PreEq, [916](#)
PreEval, [916](#)
PrelfDollarEval, [977](#)
PrelfEval, [917](#)
PreLoad, [917](#)
PrepPoly, [860](#)
PreProcessor, [898](#)
PreProInstruction, [904](#)
PreRandom, [992](#)
PreSkip, [917](#)
PREV, [810](#)
PrintDeprecation, [863](#)
PrintExtraSymbol, [1005](#)
PrintRunningTime, [863](#)
PrintSeq, [921](#)
PrintSubTerm, [921](#)
PrintSubtermList, [1005](#)
PrintTerm, [921](#)
PrintTermC, [921](#)
PrintTime, [979](#)
PrintWords, [921](#)
ProcessOption, [888](#)
Processor, [861](#)
Product, [862](#)
PrtLong, [862](#)
PrtTerms, [862](#)
PruneExtraSymbols, [1007](#)
PUSHPREASSIGNLEVEL, [818](#)
PutArgInScratch, [1001](#)
PutBracket, [863](#)
PutBracketInIndex, [979](#)
PutExtraSymbols, [1009](#)
PutIn, [863](#)
PutInside, [857](#)
PutInSpectator, [1023](#)
PutInStore, [864](#)
PutInVflags, [876](#)
PutOut, [864](#)

- PutPreVar, [885](#)
- PutTableNames, [986](#)
- PutTermInDollar, [962](#)
- PUTZERO, [812](#)
- Quotient, [865](#)
- RaisPow, [865](#)
- RaisPowCached, [865](#)
- RaisPowMod, [866](#)
- RatioFind, [867](#)
- RatioGen, [867](#)
- RatToLine, [867](#)
- ReadFile, [895](#)
- ReadFromScratch, [1024](#)
- ReadFromStream, [886](#)
- ReadijObject, [985](#)
- ReadIndex, [983](#)
- ReadIniInfo, [984](#)
- ReadObject, [985](#)
- ReadParticle, [1027](#)
- ReadPolyRatFun, [1007](#)
- ReadPosFile, [895](#)
- ReadRange, [1001](#)
- ReadSaveExpression, [993](#)
- ReadSaveHeader, [992](#)
- ReadSaveIndex, [992](#)
- ReadSaveTerm32, [994](#)
- ReadSaveVariables, [995](#)
- ReadSnum, [868](#)
- RecalcSetups, [902](#)
- RecoveryFilename, [997](#)
- REDLENG, [800](#)
- RedoTableTree, [971](#)
- RedoTree, [960](#)
- ReleaseTB, [990](#)
- Remain10, [868](#)
- Remain4, [868](#)
- RemoveDictionary, [1020](#)
- RemoveDollars, [982](#)
- ReNumber, [868](#)
- ReOpenFile, [894](#)
- ReplaceDollar, [891](#)
- ReserveTempFiles, [920](#)
- ResetScratch, [868](#)
- ResetVariables, [923](#)
- ResolveSet, [868](#)
- ReverseStatements, [887](#)
- RevertScratch, [869](#)
- ReWorkT, [838](#)
- RunAddArg, [1002](#)
- RunCycle, [1002](#)
- RunDecode, [1000](#)
- RunDedup, [1003](#)
- RunDropArg, [1003](#)
- RunEncode, [1000](#)
- RunExplode, [1001](#)
- RunHtoZArg, [1003](#)
- RunImplode, [1000](#)
- RunIsLyndon, [1002](#)
- RunMulArg, [1002](#)
- RunPermute, [1001](#)
- RunReplace, [1000](#)
- RunReverse, [1002](#)
- RunSelectArg, [1003](#)
- RunToLyndon, [1002](#)
- RunTransform, [1000](#)
- RunZtoHArg, [1003](#)
- RWLOCKR, [819](#)
- RWLOCKW, [819](#)
- ScanFunctions, [869](#)
- SeekFile, [896](#)
- SeekScratch, [869](#)
- selectExternalChannel, [989](#)
- set_del, [988](#)
- set_in, [987](#)
- set_set, [987](#)
- set_sub, [988](#)
- SETBASELENGTH, [811](#)
- SETBASEPOSITION, [811](#)
- SetDictionaryOptions, [1020](#)
- SetEndHScratch, [869](#)
- SetEndScratch, [869](#)
- SETERROR, [810](#)
- SetExpr, [927](#)
- SetExprCases, [974](#)
- SetFileIndex, [869](#)
- SETKILLMODEFOREXTERNALCHANNEL, [821](#)
- SetMods, [883](#)
- setonoff, [927](#)
- SetPosFile, [896](#)
- SetScratch, [884](#)
- SetSpecialMode, [918](#)
- SETSTARTPOS, [813](#)
- SETTERMINATORFOREXTERNALCHANNEL, [821](#)
- Sflush, [870](#)
- SGN, [800](#)
- ShrinkDictionary, [1021](#)
- Shuffle, [836](#)
- simp1token, [957](#)
- simp2token, [957](#)
- simp3atoken, [957](#)
- simp3btoken, [958](#)
- simp4token, [958](#)
- simp5token, [958](#)
- simp6token, [958](#)
- SimpleSplitMerge, [973](#)
- SimpleSplitMergeRec, [973](#)
- Simplify, [870](#)
- simpwtoken, [957](#)
- SizeOfDollar, [977](#)
- SizeOfExpression, [977](#)
- SkipAName, [958](#)
- SKIPBLANKS, [804](#)
- SKIPBRA1, [805](#)
- SKIPBRA2, [805](#)
- SKIPBRA3, [805](#)

SKIPBRA4, 805
SKIPBRA5, 805
SkipField, 900
SkipName, 1026
SortTheList, 963
SortWeights, 931
SortWild, 870
SpareTable, 884
SpecialCleanup, 883
SplitMerge, 871
StageSort, 922
StartFiles, 898
StartLoops, 1029
StartPrepro, 911
StartVariables, 822
StoreTerm, 872
str_dup, 983
StrCmp, 900
StrCopy, 824
strDup1, 884
StrHICmp, 901
StrICmp, 900
StrICont, 901
StrLen, 901
StudyPattern, 964
StuffAdd, 801
Stuffle, 837
StuffRootAdd, 837
SubPLon, 873
SubsInAll, 974
Substitute, 873
SwitchSplitMerge, 1025
SwitchSplitMergeRec, 1025
SymbolNormalize, 990
SymFind, 873
SymGen, 873
Symmetrize, 874
SynchFile, 896
TableReset, 838
TABLESIZE, 809
TakeArgContent, 928
TakeContent, 1009
TakeDollarContent, 965
TakeExtraSymbols, 1009
TakeIDfunction, 975
TakeLongRoot, 970
TakeModulus, 874
TakeNormalModulus, 874
TakeRatRoot, 970
TakeSymbolContent, 1008
TalToLine, 874
TELLFILE, 822
TellFile, 896
TenVec, 875
TenVecFind, 875
TermAssign, 962
TermFree, 817
Terminate, 810
TerminatImpl, 888
TermMalloc, 816
TermMallocAddMemory, 999
TermRenummer, 875
TermsInBracket, 982
TermsInDollar, 977
TermsInExpression, 977
TestArgNum, 1001
TestDoLoop, 1011
TestDrop, 875
TestEndDoLoop, 1011
TestFunFlag, 990
TestMatch, 876
TestName, 893
TestPartitions, 834
TestSelect, 974
TestSub, 877
TestTables, 958
TestTerm, 999
TestUse, 837
TheDefine, 906
TheUndefine, 906
TimeChildren, 877
TimeCPU, 877
TimeWallClock, 878
ToFast, 902
ToGeneral, 902
tokenize, 957
TOKENTOLINE, 802
TokenToLine, 878
TOLONG, 814
ToPolyFunGeneral, 902
ToStorage, 878
ToToken, 919
Trace4, 878
Trace4Gen, 878
Trace4no, 879
TraceFind, 879
TraceN, 879
TraceNgen, 879
TraceNno, 879
Traces, 879
TransferBuffer, 975
TransformRational, 1021
TranslateExpression, 977
TreatPolyRatFun, 992
Trick, 880
TruncateFile, 897
TryDo, 880
TryEnvironment, 987
TryFileSetups, 975
TryRecover, 802
TwoExprCompare, 978
UngetChar, 802
UngetFromStream, 802
UNLOCK, 819
UnPack, 880
UNRWLOCK, 819

- UnsetAllowDelay, [904](#)
- UnSetDictionary, [1020](#)
- UnSetMods, [883](#)
- UpdateMaxSize, [1005](#)
- UpdatePositions, [978](#)
- VARNAME, [810](#)
- VarStore, [880](#)
- WantAddLongs, [815](#)
- WantAddPointers, [815](#)
- WantAddPositions, [816](#)
- Warning, [884](#)
- WCOPY, [804](#)
- WildDollars, [962](#)
- WildFill, [880](#)
- WORDDIF, [809](#)
- wranf, [991](#)
- WriteAll, [881](#)
- WriteArgument, [881](#)
- WRITEBUFTOEXTCHANNEL, [821](#)
- WriteDictionary, [1021](#)
- WriteDollarFactorToBuffer, [961](#)
- WriteDollarToBuffer, [961](#)
- WriteExpression, [881](#)
- WRITEFILE, [821](#)
- WriteFileToFile, [895](#)
- WriteIndex, [984](#)
- WriteIndexBlock, [983](#)
- WriteIniInfo, [984](#)
- WriteInnerTerm, [881](#)
- WriteLists, [881](#)
- WriteNamesBlock, [984](#)
- WriteObject, [985](#)
- WriteOne, [881](#)
- WriteSetup, [882](#)
- WriteStats, [882](#)
- WriteStoreHeader, [996](#)
- WriteString, [915](#)
- WriteSubTerm, [882](#)
- WriteTerm, [883](#)
- writeToChannel, [989](#)
- WriteTokens, [957](#)
- WriteUnfinString, [915](#)
- WrtPower, [824](#)
- wsizeof, [809](#)
- ZEROARG, [807](#)
- ZeroFillRange, [809](#)
- DeclareVector
 - vector.h, [3216](#)
- DECODEARG
 - ftypes.h, [1447](#)
- DEDUPARG
 - ftypes.h, [1449](#)
- defaultcase
 - SWITCH, [421](#)
- DEFAULTFNAMELENGTH
 - startup.c, [2802](#)
- DEFAULTPROCESSBUCKETSIZE
 - fsizes.h, [1349](#)
- defaulttempfilename
 - startup.c, [2805](#)
- DEFAULTTHREADBUCKETSIZE
 - fsizes.h, [1350](#)
- DEFAULTTHREADLOADBALANCING
 - fsizes.h, [1350](#)
- DEFAULTTHREADS
 - fsizes.h, [1350](#)
- defaultv
 - OptDef, [289](#)
- DeferFlag
 - R_const, [375](#)
- Deferred
 - declare.h, [831](#)
 - proces.c, [2436](#)
- deferskipped
 - N_const, [252](#)
- defineChannel
 - declare.h, [989](#)
 - pre.c, [2334](#)
- defined
 - TaBIEs, [462](#)
- DEFINEVALUE
 - setfile.c, [2694](#)
- DEFINITION
 - ftypes.h, [1376](#)
- defpart
 - MInput, [217](#)
 - Model, [231](#)
- DefPosition
 - R_const, [371](#)
- deg
 - ANode, [13](#)
 - ENode, [117](#)
 - MNode, [219](#)
 - NCand, [267](#)
 - NStack, [275](#)
 - PNodeClass, [336](#)
 - T_ENode, [445](#)
 - T_MNode, [452](#)
- degree
 - poly, [345](#)
- degree_vector
 - poly, [349](#)
- DelayPrevar
 - P_const, [315](#)
- deleted
 - EEdge, [98](#)
- DeleteObject
 - declare.h, [985](#)
 - minos.c, [1735](#)
- DeleteRecoveryFile
 - checkpoint.c, [542](#)
 - declare.h, [997](#)
- DeleteStore
 - declare.h, [832](#)
 - store.c, [2834](#)
- delGroup

- SGroup, [387](#)
- DELTA
 - ftypes.h, [1393](#)
- DELTA2
 - ftypes.h, [1397](#)
- DELTA3
 - ftypes.h, [1395](#)
- deltaEuler
 - float.c, [1289](#)
- deltaEulerC
 - float.c, [1290](#)
- deltaMZV
 - float.c, [1289](#)
- DELTAP
 - ftypes.h, [1397](#)
- den
 - Fraction, [142](#)
- DENOMINATOR
 - ftypes.h, [1393](#)
- DENOMINATORSYMBOL
 - ftypes.h, [1406](#)
- denomnum
 - M_const, [188](#)
- DENSETABLE
 - ftypes.h, [1456](#)
- DenToFunction
 - declare.h, [982](#)
 - wildcard.c, [3228](#)
- derivative
 - poly, [346](#)
- description
 - fixedset, [137](#)
- det
 - ECand, [95](#)
 - EStack, [119](#)
- DetCommu
 - declare.h, [829](#)
 - normal.c, [1877](#)
- DetCurDum
 - declare.h, [832](#)
 - reshuf.c, [2613](#)
- detEdge
 - Assign, [21](#)
- DetVars
 - declare.h, [832](#)
 - store.c, [2835](#)
- DGraph, [80](#)
 - edges, [81](#)
 - nedges, [81](#)
 - nnodes, [81](#)
 - nodes, [81](#)
- DIAGRAMS
 - ftypes.h, [1404](#)
- diagrams.c, [1051](#), [1052](#)
 - CoCanonicalize, [1052](#)
 - DoCanonicalize, [1052](#)
 - DoShattering, [1052](#)
 - DoTopologyCanonicalize, [1052](#)
- diaoffset
 - TERMINFO, [467](#)
- diawrap.cc, [1061](#)
- dict.c, [1073](#), [1078](#)
 - AddDictionary, [1076](#)
 - AddToDictionary, [1076](#)
 - DictFromBytes, [1077](#)
 - DictToBytes, [1077](#)
 - DoPreAdd, [1077](#)
 - DoPreCloseDictionary, [1077](#)
 - DoPreOpenDictionary, [1077](#)
 - DoPreUseDictionary, [1077](#)
 - FindDictionary, [1075](#)
 - FindExtraSymbol, [1075](#)
 - FindFunction, [1075](#)
 - FindFunWithArgs, [1075](#)
 - FindIndex, [1075](#)
 - FindSymbol, [1075](#)
 - FindVector, [1075](#)
 - IsExponentSign, [1074](#)
 - IsMultiplySign, [1074](#)
 - RemoveDictionary, [1076](#)
 - SetDictionaryOptions, [1076](#)
 - ShrinkDictionary, [1077](#)
 - TransformRational, [1074](#)
 - UnSetDictionary, [1076](#)
 - UseDictionary, [1076](#)
- DICT_ALLNUMBERS
 - ftypes.h, [1453](#)
- DICT_DOFUNWITHARGS
 - ftypes.h, [1454](#)
- DICT_DOSPECIALS
 - ftypes.h, [1454](#)
- DICT_DOVARIABLES
 - ftypes.h, [1453](#)
- DICT_FUNCTION
 - ftypes.h, [1455](#)
- DICT_FUNCTION_WITH_ARGUMENTS
 - ftypes.h, [1455](#)
- DICT_INDEX
 - ftypes.h, [1455](#)
- DICT_INDOLLARS
 - ftypes.h, [1454](#)
- DICT_INTEGERNUMBER
 - ftypes.h, [1454](#)
- DICT_INTEGERONLY
 - ftypes.h, [1453](#)
- DICT_NOFUNWITHARGS
 - ftypes.h, [1454](#)
- DICT_NONUMBERS
 - ftypes.h, [1453](#)
- DICT_NOSPECIALS
 - ftypes.h, [1453](#)
- DICT_NOTINDOLLARS
 - ftypes.h, [1454](#)
- DICT_NOVARIABLES
 - ftypes.h, [1453](#)
- DICT_RANGE

- ftypes.h, [1455](#)
- DICT_RATIONALNUMBER
 - ftypes.h, [1454](#)
- DICT_RATIONALONLY
 - ftypes.h, [1453](#)
- DICT_SPECIALCHARACTER
 - ftypes.h, [1455](#)
- DICT_SYMBOL
 - ftypes.h, [1454](#)
- DICT_VECTOR
 - ftypes.h, [1455](#)
- DictFromBytes
 - declare.h, [1022](#)
 - dict.c, [1077](#)
- Dictionaries
 - O_const, [281](#)
- DICTIONARY, [81](#)
 - characters, [83](#)
 - elements, [82](#)
 - funwith, [83](#)
 - gnumelements, [83](#)
 - name, [82](#)
 - numbers, [82](#)
 - numelements, [82](#)
 - ranges, [83](#)
 - sizeelements, [82](#)
 - variables, [82](#)
- DICTIONARY_ELEMENT, [83](#)
 - lhs, [84](#)
 - rhs, [84](#)
 - size, [84](#)
 - type, [84](#)
- DictToBytes
 - declare.h, [1022](#)
 - dict.c, [1077](#)
- DidClean
 - C_const, [67](#)
- DIFBASE
 - declare.h, [812](#)
- DIFPOS
 - declare.h, [812](#)
- dimension
 - CbUf, [73](#)
 - fixedset, [138](#)
 - FuNcTiOn, [147](#)
 - InDeX, [151](#)
 - SeTs, [384](#)
 - SyMbOl, [425](#)
 - VeCtOr, [484](#)
- DimensionExpression
 - declare.h, [831](#)
 - reshuf.c, [2614](#)
- DimensionSubterm
 - declare.h, [831](#)
 - reshuf.c, [2614](#)
- DIMENSIONSYMBOL
 - ftypes.h, [1406](#)
- DimensionTerm
 - declare.h, [831](#)
 - reshuf.c, [2614](#)
- dir
 - EEdge, [100](#)
- dirEdge
 - EGraph, [106](#)
- DirtPow
 - C_const, [65](#)
- DIRTYFLAG
 - ftypes.h, [1421](#)
- dirtyflag
 - FiLeDaTa, [132](#)
- DIRTYSYMFLAG
 - ftypes.h, [1421](#)
- discardDisc
 - MGraph, [214](#)
 - T_MGraph, [450](#)
- discardIso
 - MGraph, [214](#)
 - T_MGraph, [451](#)
- discardOrd
 - T_MGraph, [450](#)
- discardRefine
 - MGraph, [214](#)
 - T_MGraph, [450](#)
- DisOrderFlag
 - N_const, [256](#)
- DISTRIBUTE
 - structs.h, [2907](#)
- DiStRiBuTe, [84](#)
 - cycle, [86](#)
 - n, [85](#)
 - n1, [85](#)
 - n2, [85](#)
 - obj1, [85](#)
 - obj2, [85](#)
 - out, [85](#)
 - sign, [85](#)
- Distribute
 - declare.h, [833](#)
 - symmetr.c, [2936](#)
- DISTRIBUTION
 - ftypes.h, [1394](#)
- DistrN
 - declare.h, [1026](#)
 - tools.c, [3098](#)
- div
 - COST, [76](#)
 - poly, [350](#)
- DivFloats
 - float.c, [1288](#)
- DIVFUNCTION
 - ftypes.h, [1399](#)
- DIVfunction
 - declare.h, [1010](#)
 - ratio.c, [2510](#)
- divides
 - poly, [351](#)

- DivLong
 - declare.h, [833](#)
 - reken.c, [2559](#)
- DivLong_fix_args
 - optimize.cc, [1996](#)
- divmod
 - poly, [351](#)
- divmod_heap
 - poly, [354](#)
- divmod_mpoly
 - flintinterface.h, [1270](#)
- divmod_one_term
 - poly, [353](#)
- divmod_poly
 - flintinterface.h, [1270](#)
- divmod_univar
 - poly, [353](#)
- DIVPOS
 - declare.h, [811](#)
- DivRat
 - declare.h, [833](#)
 - reken.c, [2557](#)
- divrem
 - ratio.c, [2511](#)
- Divvy
 - declare.h, [833](#)
 - reken.c, [2556](#)
- DNode, [86](#)
 - extloop, [86](#)
 - intrct, [86](#)
- do_optimization
 - optimize.cc, [2002](#)
- DO_UFFLE
 - structs.h, [2906](#)
- do_uffle
 - SHvariables, [391](#)
- DOALL
 - ftypes.h, [1452](#)
- DoArgPlode
 - compcomm.c, [627](#)
 - declare.h, [933](#)
- DoArgument
 - compcomm.c, [618](#)
 - declare.h, [928](#)
- DoBrackets
 - compcomm.c, [624](#)
 - declare.h, [932](#)
- DoBreakDo
 - declare.h, [913](#)
 - pre.c, [2326](#)
- DoCall
 - declare.h, [912](#)
 - pre.c, [2325](#)
- DoCanonicalize
 - declare.h, [834](#)
 - diagrams.c, [1052](#)
- DoChain
 - compcomm.c, [622](#)
- declare.h, [952](#)
- DoCheckpoint
 - checkpoint.c, [544](#)
 - declare.h, [998](#)
- DoClearOptimize
 - declare.h, [909](#)
 - pre.c, [2335](#)
- DoClearUserFlag
 - declare.h, [1027](#)
 - pre.c, [2337](#)
- DoCommentChar
 - declare.h, [915](#)
 - pre.c, [2324](#)
- DoContinueDo
 - declare.h, [913](#)
 - pre.c, [2326](#)
- DoDebug
 - declare.h, [913](#)
 - pre.c, [2326](#)
- DoDefine
 - declare.h, [905](#)
 - pre.c, [2324](#)
- DoDelta
 - declare.h, [833](#)
 - normal.c, [1876](#)
- DoDelta3
 - declare.h, [834](#)
 - reshuf.c, [2615](#)
- DoDimension
 - declare.h, [892](#)
 - names.c, [1831](#)
- DoDistrib
 - declare.h, [836](#)
 - reshuf.c, [2615](#)
- DoDo
 - declare.h, [913](#)
 - pre.c, [2326](#)
- DODOUBLE
 - tools.c, [3082](#)
- DoElements
 - declare.h, [893](#)
 - names.c, [1834](#)
- DoElse
 - declare.h, [912](#)
 - pre.c, [2326](#)
- DoElseif
 - declare.h, [912](#)
 - pre.c, [2326](#)
- DoEnddo
 - declare.h, [913](#)
 - pre.c, [2327](#)
- DoEndif
 - declare.h, [912](#)
 - pre.c, [2327](#)
- DoEndInside
 - declare.h, [914](#)
 - pre.c, [2328](#)
- DoEndNamespace

- declare.h, [1026](#)
- pre.c, [2336](#)
- DoEndprocedure
 - declare.h, [913](#)
 - pre.c, [2327](#)
- DoEndSwitch
 - declare.h, [1025](#)
 - if.c, [1657](#)
- DoesCommu
 - declare.h, [829](#)
 - normal.c, [1877](#)
- DOESNOTCOMMUTE
 - ftypes.h, [1453](#)
- DoExecStatement
 - declare.h, [919](#)
 - module.c, [1767](#)
- DoExecute
 - declare.h, [884](#)
 - execute.c, [1156](#)
- DOEXPAND
 - tools.c, [3082](#)
- DoExpr
 - comexpr.c, [582](#)
 - declare.h, [959](#)
- DoExternal
 - declare.h, [907](#)
 - pre.c, [2333](#)
- DoFactDollar
 - declare.h, [908](#)
 - pre.c, [2334](#)
- DoFactorize
 - compcomm.c, [630](#)
 - declare.h, [939](#)
- DoFindLoop
 - compcomm.c, [625](#)
 - declare.h, [963](#)
- DoFromExternal
 - declare.h, [907](#)
 - pre.c, [2334](#)
- Dolf
 - declare.h, [912](#)
 - pre.c, [2327](#)
- Dolfdef
 - declare.h, [911](#)
 - pre.c, [2327](#)
- Dolfndef
 - declare.h, [912](#)
 - pre.c, [2327](#)
- DolfStatement
 - declare.h, [839](#)
 - if.c, [1657](#)
- Dolfydef
 - declare.h, [911](#)
 - pre.c, [2327](#)
- DoInclude
 - declare.h, [907](#)
 - pre.c, [2325](#)
- DoInParallel
 - compcomm.c, [622](#)
 - declare.h, [934](#)
- DoinParallel
 - declare.h, [980](#)
 - module.c, [1767](#)
- DoInside
 - declare.h, [913](#)
 - pre.c, [2328](#)
- DOLARGUMENT
 - ftypes.h, [1440](#)
- DOLINDEX
 - ftypes.h, [1441](#)
- dollar
 - ARGBUFFER, [14](#)
- dollar.c, [1091](#), [1101](#)
 - AddPotModdollar, [1100](#)
 - AddToDollarBuffer, [1093](#)
 - AssignDollar, [1093](#)
 - CatchDollar, [1093](#)
 - CleanDollarFactors, [1098](#)
 - DollarFactorize, [1098](#)
 - DollarRaiseLow, [1096](#)
 - DoToFunction, [1094](#)
 - DoToIndex, [1094](#)
 - DoToLong, [1095](#)
 - DoToNumber, [1094](#)
 - DoToSymbol, [1094](#)
 - DoToTensor, [1094](#)
 - DoToTerms, [1095](#)
 - DoToVector, [1094](#)
 - EvalDoLoopArg, [1097](#)
 - ExchangeDollars, [1095](#)
 - ExecInside, [1095](#)
 - GetDoNum, [1100](#)
 - InsideDollar, [1095](#)
 - IsMultipleOf, [1096](#)
 - IsSetMember, [1096](#)
 - MakeDollarInteger, [1098](#)
 - MakeDollarMod, [1099](#)
 - PreIfDollarEval, [1096](#)
 - PutTermInDollar, [1093](#)
 - SizeOfDollar, [1095](#)
 - STEP2, [1092](#)
 - TakeDollarContent, [1098](#)
 - TermAssign, [1093](#)
 - TermsInDollar, [1095](#)
 - TestDoLoop, [1097](#)
 - TestEndDoLoop, [1097](#)
 - TranslateExpression, [1096](#)
 - TwoExprCompare, [1096](#)
 - WildDollars, [1094](#)
 - WriteDollarFactorToBuffer, [1093](#)
 - WriteDollarToBuffer, [1093](#)
- dollar_buf, [87](#)
- DOLLAREXP2
 - ftypes.h, [1392](#)
- DOLLAREXPRESSON
 - ftypes.h, [1391](#)

- DollarFactorize
 - declare.h, [965](#)
 - dollar.c, [1098](#)
- DOLLARFLAG
 - ftypes.h, [1435](#)
- DollarInOutBuffer
 - O_const, [282](#)
- DollarList
 - P_const, [311](#)
- DOLLARNAME
 - declare.h, [810](#)
- dollarname
 - DoLoOp, [91](#)
- DOLLARNAMES
 - ftypes.h, [1446](#)
- dollarname
 - C_const, [42](#)
- DollarOutBuffer
 - O_const, [279](#)
- DollarOutSizeBuffer
 - O_const, [282](#)
- DollarRaiseLow
 - declare.h, [892](#)
 - dollar.c, [1096](#)
- DoLIaRS, [87](#)
 - factors, [88](#)
 - index, [88](#)
 - name, [88](#)
 - nfactors, [89](#)
 - node, [88](#)
 - numdummies, [89](#)
 - size, [88](#)
 - type, [88](#)
 - where, [88](#)
 - zero, [88](#)
- Dollars
 - variable.h, [3208](#)
- DOLLARSTREAM
 - ftypes.h, [1368](#)
- dollarzero
 - M_const, [190](#)
- DOLNUMBER
 - ftypes.h, [1440](#)
- DOLOOP
 - structs.h, [2906](#)
- DoLoOp, [89](#)
 - contents, [91](#)
 - dollarname, [91](#)
 - errorsinloop, [92](#)
 - firstdollar, [92](#)
 - firstloopcall, [92](#)
 - firstnum, [91](#)
 - incdollar, [92](#)
 - incnum, [91](#)
 - lastdollar, [92](#)
 - lastnum, [91](#)
 - name, [90](#)
 - NoShowInput, [92](#)
 - NumPreTypes, [92](#)
 - p, [90](#)
 - PreIfLevel, [92](#)
 - PreSwitchLevel, [93](#)
 - startlinenumber, [91](#)
 - type, [91](#)
 - vars, [90](#)
- dolooplevel
 - C_const, [62](#)
- doloopnest
 - C_const, [51](#)
- DoLoops
 - variable.h, [3204](#)
- doloopstack
 - C_const, [51](#)
- doloopstacksize
 - C_const, [62](#)
- DOLSUBTERM
 - ftypes.h, [1441](#)
- DOLTERMS
 - ftypes.h, [1441](#)
- DolToFunction
 - declare.h, [964](#)
 - dollar.c, [1094](#)
- DolToIndex
 - declare.h, [965](#)
 - dollar.c, [1094](#)
- DolToLong
 - declare.h, [965](#)
 - dollar.c, [1095](#)
- DolToNumber
 - declare.h, [964](#)
 - dollar.c, [1094](#)
- DolToSymbol
 - declare.h, [964](#)
 - dollar.c, [1094](#)
- DolToTensor
 - declare.h, [964](#)
 - dollar.c, [1094](#)
- DolToTerms
 - declare.h, [973](#)
 - dollar.c, [1095](#)
- DolToVector
 - declare.h, [964](#)
 - dollar.c, [1094](#)
- DOLUNDEFINED
 - ftypes.h, [1440](#)
- DOLWILDARGS
 - ftypes.h, [1441](#)
- DOLZERO
 - ftypes.h, [1441](#)
- DoMessage
 - declare.h, [909](#)
 - pre.c, [2328](#)
- DoModDollar
 - declare.h, [980](#)
 - module.c, [1767](#)
- DoModLocal

- declare.h, 980
- module.c, 1767
- DoModMax
 - declare.h, 980
 - module.c, 1766
- DoModMin
 - declare.h, 980
 - module.c, 1766
- DoModSum
 - declare.h, 980
 - module.c, 1766
- DoNamespace
 - declare.h, 1026
 - pre.c, 2336
- DONE
 - proces.c, 2428
- DoNoParallel
 - declare.h, 979
 - module.c, 1766
- DonotinParallel
 - declare.h, 981
 - module.c, 1767
- DoOnePow
 - declare.h, 839
 - proces.c, 2435
- DoOptimize
 - declare.h, 909
 - pre.c, 2335
- DoParallel
 - declare.h, 979
 - module.c, 1766
- DoPartitions
 - declare.h, 834
 - reshuf.c, 2615
- DoPermutations
 - declare.h, 836
 - reshuf.c, 2616
- DoPipe
 - declare.h, 911
 - pre.c, 2328
- DoPipeStatement
 - declare.h, 919
 - module.c, 1767
- DoPolyfun
 - declare.h, 919
 - module.c, 1766
- DoPolyratfun
 - declare.h, 919
 - module.c, 1766
- DoPrcExtension
 - declare.h, 915
 - pre.c, 2328
- DoPreAdd
 - declare.h, 1019
 - dict.c, 1077
- DoPreAddSeparator
 - declare.h, 988
 - pre.c, 2333
- DoPreAppend
 - declare.h, 909
 - pre.c, 2329
- DoPreAppendPath
 - declare.h, 1024
 - pre.c, 2335
- DoPreAssign
 - declare.h, 910
 - pre.c, 2324
- DoPreBreak
 - declare.h, 910
 - pre.c, 2330
- DoPreCase
 - declare.h, 910
 - pre.c, 2330
- DoPreClose
 - declare.h, 914
 - pre.c, 2329
- DoPreCloseDictionary
 - declare.h, 1019
 - dict.c, 1077
- DoPreCreate
 - declare.h, 910
 - pre.c, 2329
- DoPreDefault
 - declare.h, 910
 - pre.c, 2330
- DoPreEndSwitch
 - declare.h, 910
 - pre.c, 2330
- DoPreExchange
 - declare.h, 911
 - pre.c, 2325
- DoPreOpenDictionary
 - declare.h, 1019
 - dict.c, 1077
- DoPreOut
 - declare.h, 909
 - pre.c, 2328
- DoPrePrependPath
 - declare.h, 1024
 - pre.c, 2336
- DoPrePrintTimes
 - declare.h, 914
 - pre.c, 2328
- DoPreRemove
 - declare.h, 914
 - pre.c, 2329
- DoPreReset
 - declare.h, 915
 - pre.c, 2335
- DoPreRmSeparator
 - declare.h, 988
 - pre.c, 2333
- DoPreShow
 - declare.h, 911
 - pre.c, 2330
- DoPreSortReallocate

- declare.h, 914
- pre.c, 2329
- DoPreSwitch
 - declare.h, 910
 - pre.c, 2330
- DoPreUseDictionary
 - declare.h, 1019
 - dict.c, 1077
- DoPreWrite
 - declare.h, 914
 - pre.c, 2329
- DoPrint
 - compcomm.c, 614
 - declare.h, 927
- DoProcedure
 - declare.h, 914
 - pre.c, 2329
- DoProcessBucket
 - declare.h, 980
 - module.c, 1767
- DoPrompt
 - declare.h, 908
 - pre.c, 2333
- DoPutInside
 - compcomm.c, 631
 - declare.h, 932
- DoRecovery
 - checkpoint.c, 543
 - declare.h, 997
- DoRedefine
 - declare.h, 905
 - pre.c, 2324
- DoReverseInclude
 - declare.h, 907
 - pre.c, 2325
- DoRevert
 - declare.h, 840
 - normal.c, 1877
- DoRmExternal
 - declare.h, 908
 - pre.c, 2334
- DoSetExternal
 - declare.h, 908
 - pre.c, 2333
- DoSetExternalAttr
 - declare.h, 908
 - pre.c, 2333
- DoSetRandom
 - declare.h, 908
 - pre.c, 2335
- DoSetups
 - declare.h, 888
 - setfile.c, 2694
- DoSetUserFlag
 - declare.h, 1027
 - pre.c, 2337
- DoShattering
 - declare.h, 835
- diagrams.c, 1052
- DoShtuffle
 - declare.h, 820
- DoShuffle
 - declare.h, 836
 - reshuf.c, 2616
- DoSkipExtraSymbols
 - declare.h, 909
 - pre.c, 2335
- DoStuffle
 - declare.h, 836
 - reshuf.c, 2616
- DoSumF1
 - declare.h, 840
 - ratio.c, 2506
- DoSumF2
 - declare.h, 840
 - ratio.c, 2506
- DoSwitch
 - declare.h, 1025
 - if.c, 1657
- DoSymmetrize
 - compcomm.c, 619
 - declare.h, 928
- DoSystem
 - declare.h, 911
 - pre.c, 2330
- DoTable
 - declare.h, 950
 - names.c, 1833
- DoTableExpansion
 - declare.h, 835
 - tables.c, 2965
- DoTail
 - declare.h, 885
 - startup.c, 2803
- dotchar
 - setfile.c, 2696
- DoTempSet
 - declare.h, 893
 - names.c, 1835
- DoTerminate
 - declare.h, 912
 - pre.c, 2326
- DoTheta
 - declare.h, 841
 - normal.c, 1876
- DoTimeOutAfter
 - declare.h, 909
 - pre.c, 2336
- DoToExternal
 - declare.h, 907
 - pre.c, 2334
- DoTopologyCanonicalize
 - declare.h, 835
 - diagrams.c, 1052
- DOTPBIT
 - ftypes.h, 1439

- DOTPRODUCT
 - ftypes.h, 1390
- DoubleBuffer
 - declare.h, 899
 - tools.c, 3094
- DoubleCbuffer
 - comtool.c, 764
 - declare.h, 924
- DoubleFlag
 - O_const, 285
- DOUBLEFORTRANMODE
 - ftypes.h, 1387
- DoubleIfBuffers
 - declare.h, 927
 - if.c, 1657
- DoubleList
 - declare.h, 899
 - tools.c, 3094
- DoubleLList
 - declare.h, 899
 - tools.c, 3094
- DOUBLEPRECISIONFLAG
 - ftypes.h, 1387
- DoubleSwitchBuffers
 - declare.h, 1025
 - if.c, 1658
- DoubleTable
 - float.c, 1289
- DoUndefine
 - declare.h, 907
 - pre.c, 2325
- DoUse
 - declare.h, 1026
 - pre.c, 2337
- DROP
 - ftypes.h, 1422
- DROPARG
 - ftypes.h, 1448
- DropCoefficient
 - declare.h, 857
 - normal.c, 1877
- DROPGEXPRESSION
 - ftypes.h, 1423
- DROPHGEXPRESSION
 - ftypes.h, 1424
- DROPHLEXPRESSION
 - ftypes.h, 1424
- DROPLEXPRESSION
 - ftypes.h, 1423
- DROPEDEXPRESSION
 - ftypes.h, 1423
- DROPSPECTATOREXPRESSION
 - ftypes.h, 1425
- DropSymbols
 - declare.h, 857
 - normal.c, 1877
- DuBiOuS, 93
 - dummy, 93
 - name, 93
 - node, 93
- Dubious
 - variable.h, 3208
- DubiousList
 - C_const, 43
- DumFound
 - N_const, 247
- DUMFUN
 - ftypes.h, 1394
- DumFunPlace
 - N_const, 247
- DumInd
 - M_const, 185
- DUMMY
 - ftypes.h, 1438
- dummy
 - ARGBUFFER, 15
 - BracketInfo, 35
 - DuBiOuS, 93
 - MoDeL, 234
- DUMMYBUFFER
 - ftypes.h, 1446
- DUMMYFLAG
 - ftypes.h, 1431
- DUMMYFUN
 - ftypes.h, 1395
- DUMMYINDEX_
 - ftypes.h, 1429
- DUMMYPADDING
 - AEdge, 8
 - EGraph, 112
 - ENode, 118
 - MCBlock, 194
 - MConn, 202
 - Model, 231
 - OptDef, 289
 - Options, 302
 - SProcess, 409
- DUMMYPADDING1
 - EGraph, 114
 - MGraph, 215
- DUMMYPADDING2
 - MGraph, 216
- dummyrenumlist
 - N_const, 248
- dummysubexp
 - T_const, 436
- DUMMYTEN
 - ftypes.h, 1395
- DUMMYUSE
 - declare.h, 810
- DumNum
 - C_const, 65
- dumnumflag
 - C_const, 59
- DUMPINTERMS
 - ftypes.h, 1374

- DumPlace
 - N_const, [247](#)
- DumpNode
 - declare.h, [890](#)
 - names.c, [1829](#)
- DUMPOUTTERMS
 - ftypes.h, [1374](#)
- DUMPTOCOMPILER
 - ftypes.h, [1373](#)
- DUMPTOPARALLEL
 - ftypes.h, [1374](#)
- DUMPTOSORT
 - ftypes.h, [1374](#)
- DumpTree
 - declare.h, [890](#)
 - names.c, [1829](#)
- e
 - bufIPstruct, [37](#)
- e3
 - TrAcEs, [475](#)
- e4
 - TrAcEs, [475](#)
- eat
 - P_const, [315](#)
- EATTENSOR
 - ftypes.h, [1420](#)
- ebufnum
 - T_const, [433](#)
- ECand, [94](#)
 - ~ECand, [94](#)
 - det, [95](#)
 - ECand, [94](#)
 - nplist, [95](#)
 - plist, [95](#)
 - prECand, [95](#)
- eclass
 - SGroup, [388](#)
- econn
 - EGraph, [109](#)
- EDGE
 - ftypes.h, [1404](#)
- edgen
 - EStack, [119](#)
- edges
 - Assign, [25](#)
 - DGraph, [81](#)
 - EGraph, [109](#)
 - ENode, [117](#)
 - MCBlock, [194](#)
 - T_EGraph, [444](#)
 - T_ENode, [445](#)
- edgeSym
 - Assign, [22](#)
- edtype
 - EEdge, [99](#)
- EEdge, [95](#)
 - ~EEdge, [96](#)
 - conid, [99](#)
 - copy, [97](#)
 - cut, [99](#)
 - deleted, [98](#)
 - dir, [100](#)
 - edtype, [99](#)
 - EEdge, [96](#)
 - egraph, [98](#)
 - emom, [99](#)
 - ext, [98](#)
 - extMom, [99](#)
 - id, [98](#)
 - lmom, [99](#)
 - nlegs, [98](#)
 - nodes, [98](#)
 - opicom, [99](#)
 - print, [97](#)
 - ptcl, [98](#)
 - setEMom, [97](#)
 - setId, [97](#)
 - setLMom, [97](#)
 - setType, [97](#)
 - visited, [99](#)
- eers
 - TrAcEs, [475](#)
- EESYMBOL
 - ftypes.h, [1406](#)
- EFLine, [100](#)
 - EFLine, [100](#)
 - elist, [101](#)
 - fkind, [101](#)
 - ftype, [101](#)
 - nlist, [101](#)
 - print, [101](#)
- EGraph, [102](#)
 - ~EGraph, [104](#)
 - addFLine, [108](#)
 - ald, [109](#)
 - assigned, [110](#)
 - bconn, [113](#)
 - biconnE, [106](#)
 - bicount, [113](#)
 - bidef, [113](#)
 - biinitE, [106](#)
 - bilow, [113](#)
 - bisearchE, [106](#)
 - chkMomConsv, [107](#)
 - cmpMom, [107](#)
 - connComp, [105](#)
 - connVisit, [106](#)
 - copy, [104](#)
 - dirEdge, [106](#)
 - DUMMYPADDING, [112](#)
 - DUMMYPADDING1, [114](#)
 - econn, [109](#)
 - edges, [109](#)
 - EGraph, [104](#)
 - endSetExtLoop, [105](#)
 - esym, [110](#)

- extMom, 113
- extMomConsv, 107
- extperm, 110
- findRoot, 106
- flines, 113
- fltrace, 108
- fromDGraph, 104
- fromMGraph, 104
- fsign, 110
- getFLines, 107
- groupLMom, 107
- gSubId, 110
- isExternal, 105
- isFermion, 105
- isOptE, 104
- legParticle, 108
- loopm, 113
- maxdeg, 112
- mgraph, 109
- mId, 109
- model, 108
- multp, 110
- nadj2ptv, 112
- nEdges, 111
- nExtern, 111
- nfines, 114
- nLoops, 112
- nNodes, 111
- nodes, 109
- nopi2p, 112
- nopicomp, 112
- nsym, 110
- nsym1, 110
- opi2plp, 112
- opiCount, 113
- opt, 108
- pId, 111
- prFLines, 107
- print, 104
- proc, 108
- sEdges, 111
- setExtern, 105
- setExtLoop, 105
- sId, 109
- sLoops, 111
- sMaxdeg, 111
- sNodes, 111
- sproc, 109
- totalc, 112
- egraph
 - Assign, 23
 - EEdge, 98
 - ENode, 117
 - MGraph, 212
 - SProcess, 407
 - T_MGraph, 450
- elem
 - SGroup, 388
- element
 - objects, 288
- ELEMENTLOADED
 - ftypes.h, 1444
- elements
 - DICTIONARY, 82
- ELEMENTSIZE
 - minos.h, 1755
- ELEMENTUSED
 - ftypes.h, 1444
- elist
 - EFLine, 101
- emom
 - EEdge, 99
- empty
 - FiLeInDeX, 134
- EMPTYINDEX
 - ftypes.h, 1430
- EMPTYININDEX
 - structs.h, 2903
- emptystring
 - startup.c, 2805
- EmptyTable
 - declare.h, 951
 - names.c, 1834
- EMSYMBOL
 - ftypes.h, 1407
- ENCODEARG
 - ftypes.h, 1447
- END
 - ftypes.h, 1439
- end
 - Options, 300
- endAGraph
 - SProcess, 405
- endianness
 - STOREHEADER, 414
- endmg
 - Options, 302
- endMGraph
 - SProcess, 405
- ENDMODULE
 - ftypes.h, 1370
- EndNest
 - N_const, 245
- ENDOFINPUT
 - ftypes.h, 1369
- ENDOFSTREAM
 - ftypes.h, 1369
- EndOfToken
 - declare.h, 918
 - tools.c, 3090
- endoftokens
 - C_const, 50
- endProc
 - Options, 300
- endSetExtern
 - T_EGraph, 443

- endSetExtLoop
 - EGraph, 105
- EndSort
 - declare.h, 841
 - sort.c, 2721
- endSubProc
 - Options, 300
- endswitch
 - SWITCH, 422
- EndTable
 - float.c, 1289
- ENode, 114
 - ~ENode, 115
 - copy, 116
 - deg, 117
 - DUMMYPADDING, 118
 - edges, 117
 - egraph, 117
 - ENode, 115
 - extloop, 117
 - id, 117
 - initAss, 116
 - intrct, 117
 - klow, 118
 - maxdeg, 117
 - ndtype, 117
 - print, 116
 - setExtern, 116
 - setId, 116
 - setType, 116
 - visited, 118
- entriesinindex
 - iniinfo, 156
- EntVar
 - declare.h, 842
 - names.c, 1829
- EpfCon
 - declare.h, 842
 - opera.c, 1958
- EpfFind
 - declare.h, 842
 - opera.c, 1958
- EpfGen
 - declare.h, 842
 - opera.c, 1958
- eqnidxs
 - optimization, 291
- eqnum
 - StreaM, 419
- EQUAL
 - ftypes.h, 1438
- EqualArg
 - declare.h, 843
 - reshuf.c, 2615
- error
 - MoDeL, 234
 - VeRtEx, 486
- Error0
 - declare.h, 886
 - message.c, 1715
- Error1
 - declare.h, 886
 - message.c, 1715
- Error2
 - declare.h, 886
 - message.c, 1715
- ErrorBlock
 - O_const, 286
- ErrorInDollar
 - N_const, 253
- ERROROUT
 - ftypes.h, 1374
- errorsinloop
 - DoLoOp, 92
- Eside
 - R_const, 376
- eSize
 - AStack, 29
- EStack, 118
 - ~EStack, 119
 - det, 119
 - edgen, 119
 - EStack, 119
 - nplist, 119
 - plist, 119
 - print, 119
- eStack
 - AStack, 28
- eStackP
 - AStack, 29
- esym
 - EGraph, 110
 - MGraph, 212
 - T_EGraph, 444
 - T_MGraph, 449
- EvalDoLoopArg
 - declare.h, 973
 - dollar.c, 1097
- EvalPrelf
 - declare.h, 917
 - pre.c, 2331
- evaluate.c, 1146
- EvaluateEuler
 - float.c, 1294
- EVEN_
 - ftypes.h, 1428
- EXCH
 - declare.h, 801
- ExchangeDollars
 - declare.h, 975
 - dollar.c, 1095
- ExchangeExpressions
 - declare.h, 975
 - execute.c, 1157
- EXCHINOUT
 - declare.h, 806

- EXCHN
 - declare.h, [801](#)
- execarg
 - argument.c, [490](#)
 - declare.h, [883](#)
- ExecInside
 - declare.h, [933](#)
 - dollar.c, [1095](#)
- ExecMode
 - S_const, [382](#)
- ExecModule
 - declare.h, [918](#)
 - module.c, [1765](#)
- ExecStore
 - declare.h, [918](#)
 - module.c, [1765](#)
- execterm
 - argument.c, [490](#)
 - declare.h, [883](#)
- execute.c, [1154](#), [1160](#)
 - BRACKETCURRENTEXPR, [1155](#)
 - BRACKETOTHEREXPR, [1155](#)
 - CleanExpr, [1156](#)
 - CleanupTerm, [1159](#)
 - ContentMerge, [1159](#)
 - CountTerms1, [1160](#)
 - CURRENTBRACKET, [1155](#)
 - DoExecute, [1156](#)
 - ExchangeExpressions, [1157](#)
 - GetContent, [1159](#)
 - GetFirstBracket, [1157](#)
 - GetFirstTerm, [1158](#)
 - MakeGlobal, [1156](#)
 - NOBRACKETACTIVE, [1156](#)
 - PopVariables, [1156](#)
 - PutBracket, [1157](#)
 - PutInVflags, [1156](#)
 - SetMods, [1157](#)
 - SizeOfExpression, [1159](#)
 - SpecialCleanup, [1157](#)
 - TermsInBracket, [1160](#)
 - TermsInExpression, [1159](#)
 - TestDrop, [1156](#)
 - UnSetMods, [1157](#)
 - UpdatePositions, [1160](#)
- EXECUTINGIF
 - ftypes.h, [1372](#)
- EXECUTINGPRESWITCH
 - ftypes.h, [1373](#)
- ExistsObject
 - declare.h, [985](#)
 - minos.c, [1735](#)
- exitflag
 - M_const, [191](#)
- expand_memory
 - poly, [343](#)
- ExpandBuffer
 - declare.h, [899](#)
- tools.c, [3094](#)
- ExpandEuler
 - float.c, [1290](#)
- ExpandMZV
 - float.c, [1290](#)
- ExpandRat
 - declare.h, [1011](#)
 - ratio.c, [2510](#)
- ExpandTripleDots
 - declare.h, [921](#)
 - pre.c, [2323](#)
- expchanged
 - R_const, [376](#)
- ExpectedSign
 - N_const, [258](#)
- expflags
 - R_const, [376](#)
- EXPLODEARG
 - ftypes.h, [1447](#)
- expnum
 - M_const, [188](#)
- EXPONENT
 - ftypes.h, [1393](#)
- exprbufsize
 - ParallelVars, [319](#)
- EXPRESSION
 - ftypes.h, [1391](#)
- ExPrEsSiOn, [120](#)
 - bracketinfo, [122](#)
 - compression, [123](#)
 - counter, [122](#)
 - hidelevel, [122](#)
 - inmem, [122](#)
 - name, [122](#)
 - namesize, [123](#)
 - newbracketinfo, [122](#)
 - node, [123](#)
 - numdummies, [124](#)
 - numfactors, [124](#)
 - onfile, [121](#)
 - printf, [123](#)
 - prototype, [121](#)
 - renum, [121](#)
 - renumlists, [122](#)
 - replace, [123](#)
 - size, [121](#)
 - sizeprototype, [124](#)
 - status, [123](#)
 - uflags, [124](#)
 - vflags, [123](#)
 - whichbuffer, [123](#)
- expression
 - FiLeInDeX, [134](#)
- ExpressionList
 - C_const, [43](#)
- EXPRESSIONNUMBER
 - ftypes.h, [1450](#)
- EXPRESSIONONLY

- ftypes.h, [1378](#)
- Expressions
 - variable.h, [3209](#)
- exprfillwarning
 - C_const, [60](#)
- EXPRHEAD
 - ftypes.h, [1422](#)
- EXPRNAME
 - declare.h, [810](#)
- EXPRNAMES
 - ftypes.h, [1446](#)
- exprnames
 - C_const, [42](#)
- exprnr
 - OPTIMIZERESULT, [296](#)
- exprnumber
 - SpecTatoR, [402](#)
- EXPRSOUT
 - ftypes.h, [1375](#)
- ExprStat
 - FixedGlobals, [135](#)
- ExprStatus
 - bugtool.c, [536](#)
 - declare.h, [999](#)
- exprtodo
 - ParallelVars, [319](#)
- EXPTOEXP
 - ftypes.h, [1419](#)
- ext
 - EEdge, [98](#)
 - T_EEdge, [440](#)
 - T_ENode, [445](#)
 - T_MNode, [452](#)
- extcmd.c, [1192](#), [1194](#)
 - closeAllExternalChannels, [1193](#)
 - closeExternalChannel, [1193](#)
 - getCurrentExternalChannel, [1193](#)
 - initPresetExternalChannels, [1193](#)
 - openExternalChannel, [1193](#)
 - selectExternalChannel, [1193](#)
- EXTERNALCHANNELOUT
 - ftypes.h, [1375](#)
- EXTERNALCHANNELSTREAM
 - ftypes.h, [1368](#)
- externalset
 - TERMINFO, [467](#)
- EXTERNLOCK
 - declare.h, [818](#)
- externonly
 - VerRtEx, [486](#)
- EXTERNRWLOCK
 - declare.h, [819](#)
- EXTEUCLIDEAN
 - ftypes.h, [1403](#)
- extloop
 - DNode, [86](#)
 - ENode, [117](#)
 - MNode, [219](#)
- extMom
 - EEdge, [99](#)
 - EGraph, [113](#)
- extMomConsv
 - EGraph, [107](#)
- extonly
 - Particle, [326](#)
 - PInput, [333](#)
- extperm
 - EGraph, [110](#)
 - SProcess, [409](#)
- EXTRAPARAMETER
 - ftypes.h, [1440](#)
- extrasym
 - C_const, [51](#)
- EXTRASymbol
 - ftypes.h, [1450](#)
- ExtraSymbol
 - declare.h, [856](#)
 - normal.c, [1876](#)
- extrasymbols
 - C_const, [69](#)
- EXTRASymFUN
 - ftypes.h, [1402](#)
- ExtraSymFun
 - declare.h, [1007](#)
 - notation.c, [1943](#)
- extself
 - MGraph, [214](#)
- f
 - sOrT, [397](#)
- FaCdOILaR, [124](#)
 - size, [125](#)
 - type, [125](#)
 - value, [125](#)
 - where, [125](#)
- facnum
 - M_const, [189](#)
- FACTARGFLAG
 - ftypes.h, [1450](#)
- factor
 - factorized_poly, [126](#)
 - TrAcEn, [472](#)
 - TrAcEs, [478](#)
- factor.c, [1213](#), [1214](#)
 - FactorIn, [1213](#)
 - FactorInExpr, [1213](#)
- FACTORIAL
 - ftypes.h, [1395](#)
- Factorial
 - declare.h, [843](#)
 - reken.c, [2564](#)
- factorials
 - T_const, [430](#)
- FACTORIN
 - ftypes.h, [1399](#)
- FactorIn
 - declare.h, [843](#)

- factor.c, 1213
- FactorInExpr
 - declare.h, 843
 - factor.c, 1213
- factorize_mpoly
 - flintinterface.h, 1270
- factorize_poly
 - flintinterface.h, 1271
- factorized_poly, 125
 - add_factor, 126
 - factor, 126
 - power, 126
 - tostring, 126
- FactorMode
 - O_const, 286
- FactorNum
 - O_const, 286
- factors
 - DoLIaRS, 88
- FACTORSYMBOL
 - ftypes.h, 1406
- FALSE_EXPR
 - pre.c, 2320
- FATAL_SIG_ERROR
 - portsignals.h, 2312
- FBUFFERSIZE
 - fsizes.h, 1351
- fbuffersize
 - M_const, 183
- fbufnum
 - T_const, 434
- ff
 - sOrT, 397
- ffbufnum
 - C_const, 61
- FG
 - inivar.h, 1683
 - variable.h, 3212
- FGInput, 126
 - coupl, 127
 - finalc, 127
 - finaln, 127
 - initlc, 127
 - initln, 127
 - nfinal, 127
 - ninitl, 127
- fh
 - SpecTatoR, 402
- FiLe, 128
 - active, 130
 - filesize, 129
 - handle, 130
 - inbuffer, 130
 - name, 129
 - numblocks, 130
 - PObuffer, 129
 - POfill, 129
 - POfull, 129
 - PPosition, 129
 - POsize, 130
 - POstop, 129
- file
 - sOrT, 394
- FiLeDaTa, 131
 - dirtyflag, 132
 - Fill, 132
 - Handle, 132
 - Index, 132
 - Position, 132
- FILEHANDLE
 - structs.h, 2905
- FILEINDEX
 - structs.h, 2904
- FiLeInDeX, 133
 - empty, 134
 - expression, 134
 - next, 134
 - number, 134
- filelist
 - tools.c, 3098
- filelistsize
 - tools.c, 3098
- filenum
 - N_const, 255
- FileOnlyFlag
 - M_const, 177
- fileposition
 - StreamM, 416
- FILES
 - form3.h, 1330
- filesize
 - FiLe, 129
- FILESTREAM
 - ftypes.h, 1367
- Fill
 - FiLeDaTa, 132
- fill
 - PF_BUFFER, 329
- FILLARG
 - declare.h, 807
- fillEGraph
 - Assign, 19
- FILLEXPRESS
 - declare.h, 808
- FILLFUN
 - declare.h, 807
- FILLFUN3
 - declare.h, 808
- fillpoint
 - TabIEbAsE, 457
- FILLSUB
 - declare.h, 808
- FILLVALUE
 - tools.c, 3081
- finalc
 - FGInput, 127

- finaln
 - FGInput, 127
- finalPart
 - Process, 365
- FinalStats
 - C_const, 58
- finalstep
 - TrAcEs, 478
- find_Horner_MCTS
 - optimize.cc, 2000
- find_Horner_MCTS_expand_tree
 - optimize.cc, 2000
- find_optimizations
 - optimize.cc, 2002
- FindAll
 - declare.h, 844
 - pattern.c, 2176
- FindArg
 - argument.c, 491
 - declare.h, 930
- FindBracket
 - declare.h, 979
 - index.c, 1673
- FindCase
 - declare.h, 1025
 - if.c, 1658
- findcommand
 - compiler.c, 724
 - declare.h, 897
- FindDictionary
 - declare.h, 1020
 - dict.c, 1075
- FindExtraSymbol
 - declare.h, 1022
 - dict.c, 1075
- FindFunction
 - declare.h, 1022
 - dict.c, 1075
- FindFunWithArgs
 - declare.h, 1022
 - dict.c, 1075
- FindIndex
 - declare.h, 1022
 - dict.c, 1075
- FindInIndex
 - declare.h, 823
 - store.c, 2837
- FindInKeyWord
 - declare.h, 905
 - pre.c, 2323
- findInteractionCode
 - Model, 227
- findInteractionName
 - Model, 227
- FindKeyWord
 - declare.h, 905
 - pre.c, 2323
- FindLocalSubterm
 - declare.h, 1004
 - notation.c, 1942
- FINDLOOP
 - ftypes.h, 1441
- FindLus
 - declare.h, 963
 - lus.c, 1688
- findMClass
 - Model, 228
- FindMulti
 - declare.h, 844
 - findpat.c, 1225
- FindOnce
 - declare.h, 844
 - findpat.c, 1224
- FindOnly
 - declare.h, 844
 - findpat.c, 1224
- findParticleCode
 - Model, 227
- findParticleName
 - Model, 227
- findpat.c, 1224, 1225
 - FindMulti, 1225
 - FindOnce, 1224
 - FindOnly, 1224
 - FindRest, 1225
- findPattern
 - N_const, 248
- FindRange
 - declare.h, 1001
 - transform.c, 3154
- FindRest
 - declare.h, 844
 - findpat.c, 1225
- FindrNumber
 - declare.h, 844
 - store.c, 2837
- findRoot
 - EGraph, 106
- FindSubexpression
 - declare.h, 1005
 - notation.c, 1943
- FindSubterm
 - declare.h, 1004
 - notation.c, 1942
- FindSymbol
 - declare.h, 1021
 - dict.c, 1075
- FindTableNumber
 - declare.h, 987
 - minos.c, 1734
- FindTableTree
 - declare.h, 971
 - tables.c, 2965
- FindTB
 - declare.h, 837
 - tables.c, 2966

- findTerm
 - N_const, [248](#)
- FindTree
 - comtool.c, [767](#)
 - declare.h, [960](#)
- FindVar
 - declare.h, [839](#)
 - if.c, [1657](#)
- FindVector
 - declare.h, [1021](#)
 - dict.c, [1075](#)
- FinilLine
 - declare.h, [845](#)
 - sch.c, [2655](#)
- FINISH
 - ftypes.h, [1442](#)
- finishcbuf
 - comtool.c, [763](#)
 - declare.h, [971](#)
- finished
 - tree_node, [482](#)
- FinishShuffle
 - declare.h, [836](#)
 - reshuf.c, [2616](#)
- FinishStuffle
 - declare.h, [837](#)
 - reshuf.c, [2617](#)
- finishuf
 - SHvariables, [391](#)
- FINISHUFFLE
 - structs.h, [2906](#)
- FinlShuffle
 - declare.h, [820](#)
- FinlTerm
 - declare.h, [845](#)
 - proces.c, [2433](#)
- first
 - SeTs, [383](#)
- first_variable
 - poly, [345](#)
- FIRSTBRACKET
 - ftypes.h, [1398](#)
- firstconstindex
 - C_const, [54](#)
- firstctypemessage
 - C_const, [56](#)
- firstdollar
 - DoLoOp, [92](#)
- firstindexblock
 - iniinfo, [156](#)
- firstloopcall
 - DoLoOp, [92](#)
- FIRSTMODULE
 - ftypes.h, [1370](#)
- firstnameblock
 - iniinfo, [157](#)
- firstnamespace
 - P_const, [311](#)
- firstnum
 - DoLoOp, [91](#)
- FIRSTTERM
 - ftypes.h, [1402](#)
- FIRSTUSERFUNCTION
 - ftypes.h, [1405](#)
- FIRSTUSERSYMBOL
 - ftypes.h, [1407](#)
- fix_sign_fmpz_mpoly_ratfun
 - flintinterface.h, [1276](#)
- fix_sign_fmpz_poly_ratfun
 - flintinterface.h, [1276](#)
- fixarg
 - optimize.cc, [1995](#)
- FIXED_
 - ftypes.h, [1428](#)
- FIXEDGLOBALS
 - structs.h, [2908](#)
- FixedGlobals, [134](#)
 - cTable, [137](#)
 - ExprStat, [135](#)
 - fname, [136](#)
 - fname2, [136](#)
 - fname2base, [136](#)
 - fname2size, [137](#)
 - fnamebase, [136](#)
 - fnamesize, [136](#)
 - FunNam, [136](#)
 - OperaFind, [135](#)
 - Operation, [135](#)
 - s_one, [136](#)
 - swmes, [136](#)
 - VarType, [135](#)
- fixedset, [137](#)
 - description, [137](#)
 - dimension, [138](#)
 - name, [137](#)
 - type, [138](#)
- fixedsets
 - inivar.h, [1683](#)
 - variable.h, [3212](#)
- FixIndices
 - C_const, [49](#)
- fkind
 - EFLine, [101](#)
- flags
 - FuNcTiOn, [146](#)
 - InDeX, [151](#)
 - indexblock, [152](#)
 - KEYWORD, [163](#)
 - KEYWORDV, [164](#)
 - SeTs, [384](#)
 - SETUPPARAMETERS, [385](#)
 - SpecTatoR, [402](#)
 - SyMbOl, [425](#)
 - TabIEs, [461](#)
 - TERMINFO, [467](#)
 - VeCtOr, [484](#)

- flg0
 - MNodeClass, [224](#)
- flg1
 - MNodeClass, [224](#)
- flg2
 - MNodeClass, [224](#)
- flines
 - EGraph, [113](#)
- flint::fmpz, [138](#)
 - ~fmpz, [138](#)
 - d, [139](#)
 - fmpz, [138](#)
 - print, [139](#)
- flint::mpoly, [239](#)
 - ~mpoly, [240](#)
 - d, [240](#)
 - mpoly, [240](#)
 - print, [240](#)
- flint::mpoly_ctx, [240](#)
 - ~mpoly_ctx, [241](#)
 - d, [241](#)
 - mpoly_ctx, [241](#)
- flint::mpoly_factor, [241](#)
 - ~mpoly_factor, [242](#)
 - d, [242](#)
 - mpoly_factor, [242](#)
- flint::poly, [337](#)
 - ~poly, [338](#)
 - d, [338](#)
 - poly, [338](#)
 - print, [338](#)
- flint::poly_factor, [356](#)
 - ~poly_factor, [357](#)
 - d, [357](#)
 - poly_factor, [357](#)
- flint_div
 - flintwrap.cc, [1279](#)
- flint_factorize_argument
 - flintwrap.cc, [1279](#)
- flint_factorize_dollar
 - flintwrap.cc, [1279](#)
- flint_final_cleanup_master
 - flintwrap.cc, [1279](#)
- flint_final_cleanup_thread
 - flintwrap.cc, [1279](#)
- flint_gcd
 - flintwrap.cc, [1280](#)
- flint_inverse
 - flintwrap.cc, [1280](#)
- flint_mul
 - flintwrap.cc, [1280](#)
- flint_ratfun_add
 - flintwrap.cc, [1280](#)
- flint_ratfun_normalize
 - flintwrap.cc, [1280](#)
- flint_rem
 - flintwrap.cc, [1280](#)
- flint_startup_init
 - flintwrap.cc, [1281](#)
- flintinterface.cc, [1241](#), [1242](#)
 - IFW, [1242](#)
- flintinterface.h, [1268](#), [1276](#)
 - cleanup, [1270](#)
 - cleanup_master, [1270](#)
 - divmod_mpoly, [1270](#)
 - divmod_poly, [1270](#)
 - factorize_mpoly, [1270](#)
 - factorize_poly, [1271](#)
 - fix_sign_fmpz_mpoly_ratfun, [1276](#)
 - fix_sign_fmpz_poly_ratfun, [1276](#)
 - fmpz_get_form, [1271](#)
 - fmpz_set_form, [1272](#)
 - form_sort, [1271](#)
 - from_argument_mpoly, [1271](#)
 - from_argument_poly, [1271](#)
 - gcd_mpoly, [1272](#)
 - gcd_poly, [1272](#)
 - get_variables, [1272](#)
 - inverse_poly, [1272](#)
 - mul_mpoly, [1273](#)
 - mul_poly, [1273](#)
 - ratfun_add_mpoly, [1273](#)
 - ratfun_add_poly, [1273](#)
 - ratfun_normalize_mpoly, [1273](#)
 - ratfun_normalize_poly, [1274](#)
 - ratfun_read_mpoly, [1274](#)
 - ratfun_read_poly, [1274](#)
 - simplify_fmpz, [1275](#)
 - simplify_fmpz_poly, [1276](#)
 - startup_init, [1274](#)
 - to_argument_mpoly, [1274](#), [1275](#)
 - to_argument_poly, [1275](#)
 - var_map_t, [1270](#)
- FlintPolyFlag
 - C_const, [62](#)
- flintwrap.cc, [1278](#), [1281](#)
 - flint_div, [1279](#)
 - flint_factorize_argument, [1279](#)
 - flint_factorize_dollar, [1279](#)
 - flint_final_cleanup_master, [1279](#)
 - flint_final_cleanup_thread, [1279](#)
 - flint_gcd, [1280](#)
 - flint_inverse, [1280](#)
 - flint_mul, [1280](#)
 - flint_ratfun_add, [1280](#)
 - flint_ratfun_normalize, [1280](#)
 - flint_rem, [1280](#)
 - flint_startup_init, [1281](#)
- FlipLONG
 - O_const, [280](#)
- FlipPOINTER
 - O_const, [280](#)
- FlipPOS
 - O_const, [280](#)
- FlipTable
 - declare.h, [981](#)

- tables.c, [2966](#)
- FlipWORD
 - O_const, [280](#)
- flist
 - MNodeClass, [223](#)
 - MOrbits, [239](#)
 - T_MNodeClass, [456](#)
- float.c, [1284](#), [1294](#)
 - AddFloats, [1287](#)
 - AddRatToFloat, [1288](#)
 - AddWithFloat, [1293](#)
 - CalculateEuler, [1290](#)
 - CalculateMZV, [1290](#)
 - CalculateMZVhalf, [1290](#)
 - CheckFloat, [1292](#)
 - ClearMZVTables, [1293](#)
 - CoEvaluate, [1294](#)
 - CoToFloat, [1293](#)
 - CoToRat, [1293](#)
 - deltaEuler, [1289](#)
 - deltaEulerC, [1290](#)
 - deltaMZV, [1289](#)
 - DivFloats, [1288](#)
 - DoubleTable, [1289](#)
 - EndTable, [1289](#)
 - EvaluateEuler, [1294](#)
 - ExpandEuler, [1290](#)
 - ExpandMZV, [1290](#)
 - FloatFunToRat, [1291](#)
 - FloatToInteger, [1287](#)
 - FloatToRat, [1287](#)
 - Form_mpf_clear, [1286](#)
 - Form_mpf_empty, [1291](#)
 - Form_mpf_init, [1286](#)
 - Form_mpf_set_prec_raw, [1286](#)
 - FormtoZ, [1286](#)
 - GMPSPREAD, [1286](#)
 - IntegerToFloat, [1287](#)
 - MergeWithFloat, [1294](#)
 - MulFloats, [1287](#)
 - MulRatToFloat, [1288](#)
 - PackFloat, [1291](#)
 - PrintFloat, [1292](#)
 - RatToFloat, [1291](#)
 - ReadFloat, [1292](#)
 - SetFloatPrecision, [1292](#)
 - SetupMPFTables, [1293](#)
 - SetupMZVTables, [1292](#)
 - SimpleDelta, [1288](#)
 - SimpleDeltaC, [1288](#)
 - SingleTable, [1289](#)
 - TestFloat, [1291](#)
 - ToFloat, [1293](#)
 - ToRat, [1293](#)
 - UnpackFloat, [1291](#)
 - WITHCUTOFF, [1286](#)
 - ZeroTable, [1292](#)
 - ZtoForm, [1287](#)
- FloatFunToRat
 - float.c, [1291](#)
- FLOATMODE
 - ftypes.h, [1431](#)
- FloatToInteger
 - float.c, [1287](#)
- FloatToRat
 - float.c, [1287](#)
- fltrace
 - EGraph, [108](#)
- flush_cache
 - checkpoint.c, [543](#)
- FLUSHCONSOLE
 - declare.h, [804](#)
- FlushFile
 - declare.h, [896](#)
 - tools.c, [3087](#)
- FlushOut
 - declare.h, [845](#)
 - sort.c, [2724](#)
- FlushSpectators
 - declare.h, [1024](#)
 - spectator.c, [2791](#)
- fmpz
 - flint::fmpz, [138](#)
- fmpz_get_form
 - flintinterface.h, [1271](#)
- fmpz_set_form
 - flintinterface.h, [1272](#)
- fname
 - FixedGlobals, [136](#)
- fname2
 - FixedGlobals, [136](#)
- fname2base
 - FixedGlobals, [136](#)
- fname2size
 - FixedGlobals, [137](#)
- fnamebase
 - FixedGlobals, [136](#)
- fnamesize
 - FixedGlobals, [136](#)
- FoldName
 - StreaM, [417](#)
- foralignment
 - T_MNodeClass, [456](#)
- ForFindOnly
 - N_const, [248](#)
- form3.h, [1323](#), [1334](#)
 - AWORDMASK, [1327](#)
 - BETAVERSION, [1326](#)
 - BOOL, [1333](#)
 - bzero, [1332](#)
 - FILES, [1330](#)
 - form_alignof, [1328](#)
 - FULLMAX, [1327](#)
 - GetPID, [1332](#)
 - inline, [1326](#)
 - MAJORVERSION, [1325](#)

- MAX_OPEN_FILES, [1332](#)
- MAXLONG, [1328](#)
- MAXPOSITIVE, [1328](#)
- MAXPOSITIVE2, [1328](#)
- MAXPOSITIVE4, [1328](#)
- MINORVERSION, [1325](#)
- MLONG, [1333](#)
- NDEBUG, [1326](#)
- PADDUMMY, [1328](#)
- PADINT, [1329](#)
- PADLONG, [1329](#)
- PADPOINTER, [1329](#)
- PADPOSITION, [1328](#)
- PADWORD, [1330](#)
- PRODUCTIONDATE, [1325](#)
- RLONG, [1333](#)
- SBYTE, [1333](#)
- SPECMASK, [1327](#)
- STATIC_ASSERT, [1326](#)
- STATIC_ASSERT__1, [1326](#)
- STATIC_ASSERT__2, [1326](#)
- STATIC_ASSERT__3, [1326](#)
- TOPBITONLY, [1327](#)
- TOPLONGBITONLY, [1327](#)
- UBYTE, [1333](#)
- Uclose, [1330](#)
- Uflush, [1330](#)
- Ugetpos, [1331](#)
- UINT, [1333](#)
- Uopen, [1330](#)
- Uread, [1331](#)
- Useek, [1331](#)
- Usetbuf, [1331](#)
- Usetpos, [1332](#)
- Ustdout, [1332](#)
- Usync, [1332](#)
- Utell, [1331](#)
- Utruncate, [1332](#)
- Uwrite, [1331](#)
- WILDMASK, [1327](#)
- WITHSORTBOTS, [1330](#)
- WORDMASK, [1327](#)
- form_alignof
 - form3.h, [1328](#)
- FORM_INLINE
 - declare.h, [816](#)
- Form_mpf_clear
 - float.c, [1286](#)
- Form_mpf_empty
 - float.c, [1291](#)
- Form_mpf_init
 - float.c, [1286](#)
- Form_mpf_set_prec_raw
 - float.c, [1286](#)
- form_sort
 - flintinterface.h, [1271](#)
- FORMNAME
 - startup.c, [2802](#)
- FormtoZ
 - float.c, [1286](#)
- FortDotChar
 - O_const, [286](#)
- FortFirst
 - O_const, [285](#)
- Fortran90Kind
 - C_const, [51](#)
- FortranCont
 - M_const, [188](#)
- FORTTRANCONTINUATIONLINES
 - fsizes.h, [1346](#)
- FORTTRANMODE
 - ftypes.h, [1386](#)
- FoStage4
 - R_const, [371](#)
- fPatches
 - sOrT, [396](#)
- fPatchesStop
 - sOrT, [396](#)
- fPatchN
 - sOrT, [400](#)
- fPatchN1
 - sOrT, [399](#)
- Fraction, [139](#)
 - add, [140](#), [141](#)
 - den, [142](#)
 - Fraction, [140](#)
 - gcd, [141](#)
 - isEq, [141](#)
 - normal, [141](#)
 - num, [142](#)
 - print, [140](#)
 - ratio, [142](#)
 - setValue, [140](#)
 - sub, [141](#)
- freelg
 - MNode, [220](#)
 - T_MNode, [452](#)
- FreeNameTree
 - declare.h, [891](#)
 - names.c, [1830](#)
- FreeTableBase
 - declare.h, [986](#)
 - minos.c, [1733](#)
- from
 - PF_BUFFER, [330](#)
- From0List
 - declare.h, [898](#)
 - tools.c, [3094](#)
- from_argument_mpoly
 - flintinterface.h, [1271](#)
- from_argument_poly
 - flintinterface.h, [1271](#)
- from_coefficient_list
 - poly, [348](#)
- FROMBRAC
 - ftypes.h, [1419](#)

- fromDGraph
 - EGraph, [104](#)
- FROMDISK
 - minos.h, [1755](#)
- FROMFUNCTION
 - ftypes.h, [1445](#)
- fromindex
 - T_const, [439](#)
- FromList
 - declare.h, [898](#)
 - tools.c, [3093](#)
- fromMGraph
 - Assign, [18](#)
 - EGraph, [104](#)
- FROMMODULEOPTION
 - ftypes.h, [1450](#)
- fromPerm
 - MOrbits, [238](#)
- FROMPOINTINSTRUCTION
 - ftypes.h, [1450](#)
- FromPolyRatFun
 - declare.h, [1008](#)
 - ratio.c, [2509](#)
- FROMSET
 - ftypes.h, [1419](#)
- FromStdin
 - M_const, [192](#)
- FromVarList
 - declare.h, [899](#)
 - tools.c, [3094](#)
- Frozen
 - N_const, [245](#)
- Fscr
 - R_const, [371](#)
- fsign
 - EGraph, [110](#)
- fsizes.h, [1340](#), [1353](#)
 - COMMERCIALSIZE, [1345](#)
 - COMPILERBUFFER, [1346](#)
 - COMPINC, [1349](#)
 - COMPRESSBUFFER, [1346](#)
 - DEFAULTPROCESSBUCKETSIZE, [1349](#)
 - DEFAULTTHREADBUCKETSIZE, [1350](#)
 - DEFAULTTHREADLOADBALANCING, [1350](#)
 - DEFAULTTHREADS, [1350](#)
 - FBUFFERSIZE, [1351](#)
 - FORTRANCONTINUATIONLINES, [1346](#)
 - GZIPDEFAULT, [1349](#)
 - HALFMAX, [1343](#)
 - INDENTSPACE, [1351](#)
 - INITNAMESIZE, [1344](#)
 - INITNODESIZE, [1344](#)
 - JUMPRATIO, [1352](#)
 - LARGEBUFFER, [1347](#)
 - MAXBRACKETBUFFERSIZE, [1349](#)
 - MAXCOMBUFRHS, [1350](#)
 - MAXCOUPLINGS, [1352](#)
 - MAXDOLLARVARIABLES, [1342](#)
 - MAXDUMMIES, [1343](#)
 - MAXENAME, [1343](#)
 - MAXFILESTREAMSIZE, [1343](#)
 - MAXFLAGS, [1345](#)
 - MAXFLEVELS, [1348](#)
 - MAXFPATCHES, [1347](#)
 - MAXIF, [1345](#)
 - MAXINNAMETREE, [1342](#)
 - MAXLABELS, [1345](#)
 - MAXLEGS, [1352](#)
 - MAXLEVELS, [1346](#)
 - MAXLHS, [1346](#)
 - MAXLINELENGTH, [1351](#)
 - MAXLOOPS, [1345](#)
 - MAXMATCH, [1345](#)
 - MAXMULTIBRACKETLEVELS, [1351](#)
 - MAXNEST, [1344](#)
 - MAXNUMBERSIZE, [1344](#)
 - MAXNUMSIZE, [1349](#)
 - MAXPARLEVEL, [1344](#)
 - MAXPARTICLES, [1352](#)
 - MAXPATCHES, [1347](#)
 - MAXPOWER, [1342](#)
 - MAXPRENAMESIZE, [1342](#)
 - MAXREPEAT, [1344](#)
 - MAXSAVEFUNCTION, [1343](#)
 - MAXSUBEXPRESSIONS, [1343](#)
 - MAXTABLECOMBUF, [1350](#)
 - MAXTER, [1343](#)
 - MAXVARIABLES, [1342](#)
 - MAXWILDC, [1346](#)
 - MINALLOC, [1352](#)
 - MULTIINDENTSPACE, [1351](#)
 - NORMSIZE, [1344](#)
 - NPAIR1, [1351](#)
 - NPAIR2, [1351](#)
 - NUMFACS, [1345](#)
 - NUMFIXED, [1344](#)
 - NUMOPTIONS, [1352](#)
 - NUMSTORECACHES, [1350](#)
 - NUMTABLEENTRIES, [1346](#)
 - SCRATCHSIZE, [1347](#)
 - SFHSIZE, [1349](#)
 - SHMWINSIZE, [1349](#)
 - SIZEFACS, [1345](#)
 - SIZESTORECACHE, [1351](#)
 - SLARGEBUFFER, [1348](#)
 - SMALLBUFFER, [1346](#)
 - SMALLOVERFLOW, [1347](#)
 - SMAFPATCHES, [1348](#)
 - SMAXPATCHES, [1348](#)
 - SORTIOSIZE, [1347](#)
 - SPECTATORSIZE, [1348](#)
 - SSMALLBUFFER, [1347](#)
 - SSMALLOVERFLOW, [1348](#)
 - SSORTIOSIZE, [1348](#)
 - STERMSSMALL, [1348](#)
 - TABLEEXTENSION, [1349](#)

- TERMSMALL, [1347](#)
- THREADSCRATCHOUTSIZE, [1350](#)
- THREADSCRATCHSIZE, [1350](#)
- WILDOFFSET, [1342](#)
- WORKBUFFER, [1343](#)
- ftype
 - EFLine, [101](#)
- ftypes.h, [1355](#), [1459](#)
 - ABSFUNCTION, [1396](#)
 - ACOSFUNCTION, [1400](#)
 - ACOSHFUNCTION, [1401](#)
 - ADDARG, [1448](#)
 - ALLARGS, [1446](#)
 - ALLDIRTY, [1421](#)
 - ALLFUNCTIONS, [1405](#)
 - ALLFUNPOWERS, [1442](#)
 - ALLINTEGERDOUBLE, [1388](#)
 - ALLMZVFUNCTIONS, [1405](#)
 - ALLVARIABLES, [1377](#)
 - ALSODOLLARS, [1445](#)
 - ALSOFUNARGS, [1445](#)
 - ALSOPOWERS, [1444](#)
 - ALSOREVERSE, [1389](#)
 - ANDCOND, [1438](#)
 - ANTISYMMETRIC, [1433](#)
 - ANYTYPE, [1378](#)
 - ARGFIELD, [1392](#)
 - ARGHEAD, [1421](#)
 - ARGRANGE, [1446](#)
 - ARGTOARG, [1419](#)
 - ARGWILD, [1391](#)
 - ARLTOARL, [1419](#)
 - ASINFUNCTION, [1400](#)
 - ASINHFUNCTION, [1401](#)
 - ATAN2FUNCTION, [1401](#)
 - ATANFUNCTION, [1400](#)
 - ATANHFUNCTION, [1401](#)
 - ATENDOFMODULE, [1377](#)
 - AUTONAMES, [1446](#)
 - BASECODE, [1449](#)
 - BERNOULLIFUNCTION, [1397](#)
 - BINOMIAL, [1395](#)
 - BLOCK, [1404](#)
 - BUILTINDOLLARS, [1407](#)
 - BUILTININDICES, [1407](#)
 - BUILTINSYMBOLS, [1407](#)
 - BUILTINVECTORS, [1407](#)
 - CDELETE, [1378](#)
 - CDELTA, [1380](#)
 - CDOLLAR, [1380](#)
 - CDOTPRODUCT, [1379](#)
 - CDOUBLEDOT, [1380](#)
 - CDUBIOUS, [1380](#)
 - CEXPRESSION, [1379](#)
 - CFUN, [1458](#)
 - CFUNCTION, [1379](#)
 - CHECKEXTERN, [1457](#)
 - CHECKLOGTYPE, [1390](#)
 - CHISHOLM, [1389](#)
 - CINDEX, [1379](#)
 - CLEANFLAG, [1421](#)
 - CLEAR, [1439](#)
 - CLEARMODULE, [1370](#)
 - CMODE, [1387](#)
 - CMODEL, [1380](#)
 - CNUMBER, [1379](#)
 - COEFFI, [1436](#)
 - COEFFICIENTSONLY, [1444](#)
 - COEFFSYMBOL, [1406](#)
 - COMFUNPOWERS, [1442](#)
 - CONJUGATION, [1398](#)
 - CONTENTTERM, [1402](#)
 - CONTRACT, [1417](#)
 - COSFUNCTION, [1400](#)
 - COSHFUNCTION, [1401](#)
 - COULDCOMMUTE, [1452](#)
 - COUNTFUNCTION, [1397](#)
 - CRANGE, [1380](#)
 - CSET, [1379](#)
 - CSUBEXP, [1380](#)
 - CSYMBOL, [1379](#)
 - CVECTOR, [1379](#)
 - CVECTOR1, [1380](#)
 - CYCLEARG, [1448](#)
 - CYCLESYMMETRIC, [1433](#)
 - DECLARATION, [1376](#)
 - DECODEARG, [1447](#)
 - DEDUPARG, [1449](#)
 - DEFINITION, [1376](#)
 - DELTA, [1393](#)
 - DELTA2, [1397](#)
 - DELTA3, [1395](#)
 - DELTAP, [1397](#)
 - DENOMINATOR, [1393](#)
 - DENOMINATORSYMBOL, [1406](#)
 - DENSETABLE, [1456](#)
 - DIAGRAMS, [1404](#)
 - DICT_ALLNUMBERS, [1453](#)
 - DICT_DOFUNWITHARGS, [1454](#)
 - DICT_DOSPECIALS, [1454](#)
 - DICT_DOVARIABLES, [1453](#)
 - DICT_FUNCTION, [1455](#)
 - DICT_FUNCTION_WITH_ARGUMENTS, [1455](#)
 - DICT_INDEX, [1455](#)
 - DICT_INDOLLARS, [1454](#)
 - DICT_INTEGERNUMBER, [1454](#)
 - DICT_INTEGERONLY, [1453](#)
 - DICT_NOFUNWITHARGS, [1454](#)
 - DICT_NONUMBERS, [1453](#)
 - DICT_NOSPECIALS, [1453](#)
 - DICT_NOTINDOLLARS, [1454](#)
 - DICT_NOVARIABLES, [1453](#)
 - DICT_RANGE, [1455](#)
 - DICT_RATIONALNUMBER, [1454](#)
 - DICT_RATIONALONLY, [1453](#)
 - DICT_SPECIALCHARACTER, [1455](#)

- DICTIONARY_SYMBOL, 1454
- DICTIONARY_VECTOR, 1455
- DIMENSIONSYMBOL, 1406
- DIRTYFLAG, 1421
- DIRTYSYMFLAG, 1421
- DISTRIBUTION, 1394
- DIVFUNCTION, 1399
- DOALL, 1452
- DOESNOTCOMMUTE, 1453
- DOLARGUMENT, 1440
- DOLINDEX, 1441
- DOLLAREXPR2, 1392
- DOLLAREXPRESSION, 1391
- DOLLARFLAG, 1435
- DOLLARNAMES, 1446
- DOLLARSTREAM, 1368
- DOLNUMBER, 1440
- DOLSUBTERM, 1441
- DOLTERMS, 1441
- DOLUNDEFINED, 1440
- DOLWILDARGS, 1441
- DOLZERO, 1441
- DOTPBIT, 1439
- DOTPRODUCT, 1390
- DOUBLEFORTRANMODE, 1387
- DOUBLEPRECISIONFLAG, 1387
- DROP, 1422
- DROPARG, 1448
- DROPGEXPRESSION, 1423
- DROPHGEXPRESSION, 1424
- DROPHLEXPRESSION, 1424
- DROPLEXPRESSION, 1423
- DROPPEDEXPRESSION, 1423
- DROPSPECTATOREXPRESSION, 1425
- DUMFUN, 1394
- DUMMY, 1438
- DUMMYBUFFER, 1446
- DUMMYFLAG, 1431
- DUMMYFUN, 1395
- DUMMYINDEX_, 1429
- DUMMYTEN, 1395
- DUMPINTERMS, 1374
- DUMPOUTTERMS, 1374
- DUMPTOCOMPILER, 1373
- DUMPTOPARALLEL, 1374
- DUMPTOSORT, 1374
- EATTENSOR, 1420
- EDGE, 1404
- EESYMBOL, 1406
- ELEMENTLOADED, 1444
- ELEMENTUSED, 1444
- EMPTYINDEX, 1430
- EMSYMBOL, 1407
- ENCODEARG, 1447
- END, 1439
- ENDMODULE, 1370
- ENDOFINPUT, 1369
- ENDOFSTREAM, 1369
- EQUAL, 1438
- ERROROUT, 1374
- EVEN_, 1428
- EXECUTINGIF, 1372
- EXECUTINGPRESWITCH, 1373
- EXPLODEARG, 1447
- EXPONENT, 1393
- EXPRESSION, 1391
- EXPRESSIONNUMBER, 1450
- EXPRESSIONONLY, 1378
- EXPRHEAD, 1422
- EXPRNAMES, 1446
- EXPRSOUT, 1375
- EXPTOEXP, 1419
- EXTERNALCHANNELOUT, 1375
- EXTERNALCHANNELSTREAM, 1368
- EXTEUCLIDEAN, 1403
- EXTRAPARAMETER, 1440
- EXTRASymbol, 1450
- EXTRASymFUN, 1402
- FACTARGFLAG, 1450
- FACTORIAL, 1395
- FACTORIN, 1399
- FACTORSYMBOL, 1406
- FILESTREAM, 1367
- FINDLOOP, 1441
- FINISH, 1442
- FIRSTBRACKET, 1398
- FIRSTMODULE, 1370
- FIRSTTERM, 1402
- FIRSTUSERFUNCTION, 1405
- FIRSTUSERSYMBOL, 1407
- FIXED_, 1428
- FLOATMODE, 1431
- FORTTRANMODE, 1386
- FROMBRAC, 1419
- FROMFUNCTION, 1445
- FROMMODULEOPTION, 1450
- FROMPOINTINSTRUCTION, 1450
- FROMSET, 1419
- FUNBIT, 1439
- FUNCTION, 1392
- FUNCTIONONLY, 1378
- FUNCTIONSORT, 1431
- FUNHEAD, 1422
- FUNNYDOLLAR, 1430
- FUNNYVEC, 1430
- FUNNYWILD, 1430
- FUNTOFUN, 1419
- GAMMA, 1393
- GAMMA1, 1429
- GAMMA5, 1429
- GAMMA6, 1429
- GAMMA7, 1429
- GAMMAFIVE, 1393
- GAMMAFUNCTION, 1427
- GAMMAI, 1393
- GAMMASEVEN, 1394

GAMMASIX, [1394](#)
GCDFUNCTION, [1398](#)
GENCOMMUTE, [1440](#)
GENERALFUNCTION, [1427](#)
GENNONCOMMUTE, [1440](#)
GFIVE, [1436](#)
GIDENT, [1435](#)
GLOBAL, [1439](#)
GLOBALEXPRESSION, [1423](#)
GLOBALMODULE, [1370](#)
GLOBALSPECTATORFLAG, [1455](#)
GMINUS, [1436](#)
GPLUS, [1436](#)
GREATER, [1437](#)
GREATEREQUAL, [1438](#)
HAAKJE, [1393](#)
HAAKJE0, [1392](#)
HEADERFILE, [1369](#)
HIDDENGEXPRESSION, [1424](#)
HIDDENLEXPRESSION, [1424](#)
HIDE, [1422](#)
HIDEGEXPRESSION, [1424](#)
HIDELEXPRESSION, [1424](#)
HTOZARG, [1449](#)
IDFUNCTION, [1403](#)
IDHEAD, [1435](#)
IFDOLLAR, [1437](#)
IFDOLLAREXTRA, [1437](#)
IFEXPRESSION, [1437](#)
IFFLOATNUMBER, [1437](#)
IFISFACTORIZED, [1437](#)
IFOCCURS, [1437](#)
IFUSERFLAG, [1437](#)
IMPLODEARG, [1447](#)
INCEXPRESSION, [1424](#)
INDEX, [1391](#)
INDEX_, [1429](#)
INDEXONLY, [1378](#)
INDTOIND, [1418](#)
INDTOSUB, [1419](#)
INPUTOUT, [1374](#)
INPUTSTREAM, [1369](#)
INTFUNCTION, [1396](#)
INTOHIDE, [1423](#)
INTOHIDEGEXPRESSION, [1425](#)
INTOHIDELEXPRESSION, [1425](#)
INUSE, [1452](#)
INVERSEFACTORIAL, [1395](#)
INVERSEFUNCTION, [1403](#)
INVERSETABLE, [1444](#)
ISCOMPRESSED, [1432](#)
ISFACTORIZED, [1432](#)
ISFORTRAN90, [1389](#)
ISINRHS, [1432](#)
ISLYNDON, [1448](#)
ISLYNDONR, [1448](#)
ISNOTFORTRAN90, [1388](#)
ISUNMODIFIED, [1432](#)
ISYMBOL, [1405](#)
ISZERO, [1431](#)
KEEPZERO, [1432](#)
LBRACE, [1383](#)
LESS, [1438](#)
LESSEQUAL, [1438](#)
LEVICIVITA, [1395](#)
LHSIDE, [1386](#)
LHSIDEX, [1386](#)
LI2FUNCTION, [1401](#)
LINFUNCTION, [1402](#)
LISTEDLOOP, [1375](#)
LNFUNCTION, [1400](#)
LNUMBER, [1392](#)
LOADDOLLAR, [1420](#)
LOCALEXPRESSION, [1423](#)
LONGNUMBER, [1436](#)
LOOKINGFORELSE, [1372](#)
LOOKINGFORENDIF, [1373](#)
LPARENTHESIS, [1382](#)
LRPARENTHESSES, [1385](#)
MAINSORT, [1431](#)
MAKEARGS, [1446](#)
MAKERATIONAL, [1403](#)
MAPLEMODE, [1387](#)
MATCH, [1436](#)
MATCHFUNCTION, [1397](#)
MATHEMATICAMODE, [1387](#)
MAXBUILTINFUNCTION, [1405](#)
MAXFUNCTION, [1396](#)
MAXPOWEROF, [1399](#)
MAXRANGEINDICATOR, [1447](#)
MINFUNCTION, [1396](#)
MINPOWEROF, [1399](#)
MINSPEC, [1430](#)
MINVECTOR, [1391](#)
MIXED, [1377](#)
MIXED2, [1377](#)
MOD2FUNCTION, [1396](#)
MODFUNCTION, [1396](#)
MODLOCAL, [1444](#)
MODMAX, [1443](#)
MODMIN, [1444](#)
MODNONE, [1443](#)
MODSUM, [1443](#)
MODULEINSTACK, [1372](#)
MOEBIUS, [1404](#)
MULFUNCTION, [1403](#)
MULTIPLEOF, [1436](#)
MULTIPLYARG, [1448](#)
MUSTCLEANPRF, [1421](#)
NAMENOTFOUND, [1440](#)
NEG0_, [1428](#)
NEG_, [1428](#)
NEWFACTARG, [1450](#)
NEWLYDEFINEDEXPRESSION, [1427](#)
NEWSTATEMENT, [1373](#)
NMIN4SHIFT, [1390](#)

- NOAUTO, [1377](#)
- NOCHECKLOGTYPE, [1390](#)
- NODEFUNCTION, [1404](#)
- NODOUBLEMASK, [1388](#)
- NOEXTSELF, [1457](#)
- NOFUNPOWERS, [1441](#)
- NOINDEX, [1430](#)
- NOINVERSES, [1445](#)
- NOLYNDON, [1449](#)
- NONODES, [1457](#)
- NOPARALLEL_CONVPOLY, [1371](#)
- NOPARALLEL_DOLLAR, [1371](#)
- NOPARALLEL_NPROC, [1371](#)
- NOPARALLEL_RHS, [1371](#)
- NOPARALLEL_SPECTATOR, [1371](#)
- NOPARALLEL_TBLDOLLAR, [1371](#)
- NOPARALLEL_USER, [1371](#)
- NOQUADMASK, [1388](#)
- NORMALFORMAT, [1388](#)
- NORMALIZEFLAG, [1435](#)
- NOSNAILS, [1457](#)
- NOSPACEFORMAT, [1388](#)
- NOTADPOLES, [1456](#)
- NOTEQUAL, [1438](#)
- NOTRICK, [1389](#)
- NOUNPACK, [1445](#)
- NUMARG, [1446](#)
- NUMARGSFUN, [1395](#)
- NUMERATORSYMBOL, [1406](#)
- NUMERICLOOP, [1375](#)
- NUMFACTORS, [1402](#)
- NUMSPECSETS, [1431](#)
- NUMTERMSFUN, [1398](#)
- NUMTOIND, [1420](#)
- NUMTONUM, [1420](#)
- NUMTOSUB, [1420](#)
- NUMTOSYM, [1420](#)
- O_BACKWARD, [1452](#)
- O_CSE, [1451](#)
- O_CSEGREEDY, [1451](#)
- O_FORWARD, [1451](#)
- O_FORWARDANDBACKWARD, [1452](#)
- O_FORWARDORBACKWARD, [1452](#)
- O_GREEDY, [1451](#)
- O_MCTS, [1451](#)
- O_NONE, [1451](#)
- O_OCCURRENCE, [1451](#)
- O_SIMULATED_ANNEALING, [1451](#)
- ODD_, [1428](#)
- OLDFACTARG, [1450](#)
- OLDSTATEMENT, [1373](#)
- ONEEXPRESSION, [1375](#)
- ONEPARTICLEIRREDUCIBLE, [1456](#)
- ONEPI, [1405](#)
- ONLYFUNCTIONS, [1452](#)
- OPTHEAD, [1452](#)
- ORCOND, [1438](#)
- ORDEREDSET, [1456](#)
- PARALLELFLAG, [1371](#)
- PARTEST, [1377](#)
- PARTITIONS, [1403](#)
- PATTERNFUNCTION, [1397](#)
- PERMUTATIONS, [1403](#)
- PERMUTEARG, [1447](#)
- PFORTRANMODE, [1387](#)
- PHI, [1405](#)
- PIPESTREAM, [1368](#)
- PISYMBOL, [1405](#)
- POLYADD, [1442](#)
- POLYDIV, [1443](#)
- POLYFUN, [1370](#)
- POLYGCD, [1443](#)
- POLYINTFAC, [1443](#)
- POLYMUL, [1442](#)
- POLYNORM, [1443](#)
- POLYREM, [1443](#)
- POLYSUB, [1442](#)
- POS0_, [1428](#)
- POS_, [1427](#)
- POSITIVEONLY, [1445](#)
- POSNEG, [1444](#)
- PRECALCSTREAM, [1368](#)
- PRELOWERAFTER, [1372](#)
- PRENOACTION, [1372](#)
- PREPROONLY, [1373](#)
- PRERAISEAFTER, [1372](#)
- PREREADSTREAM, [1368](#)
- PREREADSTREAM2, [1368](#)
- PREREADSTREAM3, [1368](#)
- PRETYPEDO, [1376](#)
- PRETYPEIF, [1375](#)
- PRETYPEINSIDE, [1376](#)
- PRETYPENONE, [1375](#)
- PRETYPEPROCEDURE, [1376](#)
- PRETYPESWITCH, [1376](#)
- PREVARSTREAM, [1368](#)
- PRIMENUMBER, [1402](#)
- PRINTALL, [1426](#)
- PRINTCONTENT, [1426](#)
- PRINTCONTENTS, [1426](#)
- PRINTLFILE, [1426](#)
- PRINTOFF, [1426](#)
- PRINTON, [1426](#)
- PRINTONEFUNCTION, [1426](#)
- PRINTONETERM, [1426](#)
- PROCEDUREFILE, [1369](#)
- PROPERORDERFLAG, [1442](#)
- PUTFIRST, [1403](#)
- PVFUNV, [1458](#)
- PVFUNWP, [1458](#)
- Q_, [1429](#)
- QUADRUPLEFORTRANMODE, [1388](#)
- QUADRUPLEPRECISIONFLAG, [1388](#)
- RANDOMFUNCTION, [1402](#)
- RANPERM, [1402](#)
- RATIO, [1417](#)

RATIONALMODE, 1431
RBRACE, 1383
RCYCLESYMMETRIC, 1433
READSPECTATORFLAG, 1455
REDEFINEDEXPRESSION, 1427
REDUCEMODE, 1387
REGULAR, 1442
REGULAREXPRESSION, 1427
REGULARSYMBOL, 1450
REMFUNCTION, 1399
REPLACEARG, 1447
REPLACELOOP, 1441
REPLACEMENT, 1394
REVERSEARG, 1447
REVERSEFILESTREAM, 1369
REVERSEFUNCTION, 1394
REVERSEORDER, 1434
RHSIDE, 1386
ROOTFUNCTION, 1398
RPARENTHESIS, 1382
SEARCHINGPRECASE, 1373
SEARCHINGPREENDSWITCH, 1373
SELECTARG, 1449
SEPARATESYMBOL, 1406
SETBIT, 1440
SETEXP, 1392
SETFUNCTION, 1393
SETNUMBER, 1420
SETONLY, 1378
SETSET, 1391
SETTONUM, 1419
SETUPFILE, 1369
SIGFUNCTION, 1396
SIGNFUN, 1396
SINFUNCTION, 1400
SINHFUNCTION, 1401
SIZEOFFUNCTION, 1404
SKIP, 1422
SKIPGEXPRESSION, 1423
SKIPLEXPRESSION, 1423
SKIPUNHIDEGEXPRESSION, 1425
SKIPUNHIDELEXPRESSION, 1425
SNUMBER, 1392
SORT, 1439
SORTANTIPOWER, 1389
SORTHIGHFIRST, 1389
SORTLOWFIRST, 1389
SORTMODULE, 1370
SORTPOWERFIRST, 1389
SPARSETABLE, 1456
SPECIFICATION, 1376
SPECTATOREXPRESSION, 1425
SQRTFUNCTION, 1400
STATEMENT, 1376
STATSMERGETOFILE, 1390
STATSOUT, 1374
STATSPOSTSORT, 1390
STATSSPLITMERGE, 1390
STORE, 1439
STOREDEXPRESSION, 1424
STOREMODULE, 1370
SUBAFTER, 1435
SUBAFTERNOT, 1435
SUBALL, 1434
SUBDISORDER, 1435
SUBEXPR, 1436
SUBEXPRESSION, 1391
SUBEXPSIZE, 1422
SUBMANY, 1434
SUBMASK, 1435
SUBMULTI, 1434
SUBONCE, 1434
SUBONLY, 1434
SUBROUTINEFILE, 1369
SUBSELECT, 1434
SUBSORT, 1431
SUBTERMUSED1, 1421
SUBTERMUSED2, 1421
SUBVECTOR, 1434
SUMF1, 1394
SUMF2, 1394
SUMMEDIND, 1430
SUMNUM1, 1417
SUMNUM2, 1417
SYMBOL, 1390
SYMBOL_, 1428
SYMBOLONLY, 1378
SYMMETRIC, 1433
SYMMETRIZE, 1417
SYMTONUM, 1418
SYMTOSSUB, 1418
SYMTOSSYM, 1418
TABLEBASEFILE, 1370
TABLEFUNCTION, 1398
TABLESTUB, 1399
TAKETRACE, 1417
TANFUNCTION, 1400
TANHFUNCTION, 1401
TCOMMA, 1383
TCONJUGATE, 1385
TDELTA, 1382
TDIVIDE, 1383
TDOLLAR, 1382
TDOT, 1383
TDOTPRODUCT, 1381
TDUBIOUS, 1382
EMPTY, 1385
TENDOFIT, 1384
TENSORFUNCTION, 1427
TENVEC, 1417
TERMFUNCTION, 1398
TERMSINBRACKET, 1399
TERMSINEXPR, 1398
TEXPRESSION, 1381
TFLOAT, 1386
TFUN, 1458

- TFUN1, [1458](#)
- TFUNCLOSE, [1383](#)
- TFUNCTION, [1381](#)
- TFUNOPEN, [1383](#)
- TGENINDEX, [1384](#)
- THETA, [1397](#)
- THETA2, [1397](#)
- THREADSDEBUG, [1374](#)
- TINDEX, [1381](#)
- TMINUS, [1384](#)
- TMPPOLYFUN, [1392](#)
- TMULTIPLY, [1383](#)
- TNOT, [1384](#)
- TNUMBER, [1381](#)
- TNUMBER1, [1385](#)
- TOBEFACTORED, [1432](#)
- TOBEUNFACTORED, [1432](#)
- TOLYNDON, [1448](#)
- TOLYNDONR, [1448](#)
- TOOUTPUT, [1377](#)
- TOPO, [1404](#)
- TOPOLOGIES, [1404](#)
- TOPOLOGIESONLY, [1456](#)
- TOPOLYNOMIALFLAG, [1449](#)
- TPLUS, [1384](#)
- TPOWER, [1384](#)
- TPOWER1, [1385](#)
- TSET, [1381](#)
- TSETCLOSE, [1384](#)
- TSETDOL, [1385](#)
- TSETNUM, [1385](#)
- TSETOPEN, [1384](#)
- TSGAMMA, [1385](#)
- TSUBEXP, [1382](#)
- TSYMBOL, [1381](#)
- TVECTOR, [1381](#)
- TWILDARG, [1382](#)
- TWILDCARD, [1382](#)
- TYPEADJUSTBOUNDS, [1412](#)
- TYPEALLLOOPS, [1416](#)
- TYPEALLPATHS, [1417](#)
- TYPEAPPLY, [1413](#)
- TYPEAPPLYRESET, [1413](#)
- TYPEARG, [1410](#)
- TYPEARGEXPLODE, [1414](#)
- TYPEARGHEADSIZE, [1422](#)
- TYPEARGIMPLODE, [1414](#)
- TYPEARGTOEXTRASymbol, [1416](#)
- TYPEASSIGN, [1411](#)
- TYPECANONICALIZE, [1416](#)
- TYPECHAININ, [1414](#)
- TYPECHAINOUT, [1414](#)
- TYPECHISHOLM, [1409](#)
- TYPECLEARUSERFLAG, [1416](#)
- TYPECOUNT, [1408](#)
- TYPEDENOMINATORS, [1414](#)
- TYPEDETCURDUM, [1413](#)
- TYPEDISCARD, [1409](#)
- TYPEDOLOOP, [1415](#)
- TYPEDROPCOEFFICIENT, [1415](#)
- TYPEDROPSYMBOLS, [1415](#)
- TYPEELIF, [1409](#)
- TYPEELSE, [1409](#)
- TYPEENDDOLOOP, [1415](#)
- TYPEENDIF, [1409](#)
- TYPEENDREPEAT, [1408](#)
- TYPEENDSWITCH, [1416](#)
- TYPEEXIT, [1410](#)
- TYPEEXPRESSION, [1408](#)
- TYPEFACTARG, [1411](#)
- TYPEFACTARG2, [1411](#)
- TYPEFACTOR, [1414](#)
- TYPEFINDLOOP, [1412](#)
- TYPEFPRINT, [1411](#)
- TYPEFROMPOLYNOMIAL, [1415](#)
- TYPEGOTO, [1409](#)
- TYPEIDNEW, [1408](#)
- TYPEIDOLD, [1408](#)
- TYPEIF, [1409](#)
- TYPEINEXPRESSION, [1413](#)
- TYPEINSIDE, [1412](#)
- TYPEISFUN, [1386](#)
- TYPEISMYSTERY, [1386](#)
- TYPEISSUB, [1386](#)
- TYPEMERGE, [1413](#)
- TYPEMULT, [1408](#)
- TYPENORM, [1410](#)
- TYPENORM2, [1410](#)
- TYPENORM3, [1410](#)
- TYPENORM4, [1414](#)
- TYPEOPERATION, [1408](#)
- TYPEPRINT, [1410](#)
- TYPEPUTINSIDE, [1415](#)
- TYPEREDEFPRE, [1411](#)
- TYPERENUMBER, [1412](#)
- TYPEREPEAT, [1408](#)
- TYPEREVERSE, [1410](#)
- TYPESETEXIT, [1410](#)
- TYPESETUSERFLAG, [1416](#)
- YPESORT, [1412](#)
- YPESPLITARG, [1411](#)
- YPESPLITARG2, [1411](#)
- YPESPLITFIRSTARG, [1413](#)
- YPESPLITLASTARG, [1413](#)
- YPESTUFFLE, [1414](#)
- YPESUM, [1409](#)
- YPESUMFIX, [1412](#)
- YPESWITCH, [1416](#)
- TYPETERM, [1412](#)
- TYPETESTUSE, [1413](#)
- TYPETOPOLYNOMIAL, [1415](#)
- TYPETOSPECTATOR, [1416](#)
- TYPETRANSFORM, [1415](#)
- TYPETRY, [1411](#)
- TYPEUNRAVEL, [1412](#)
- UNHIDE, [1422](#)

- UNHIDEGEXPRESSION, [1425](#)
- UNHIDELEXPRESSION, [1425](#)
- UNPACK, [1445](#)
- USEDFLAG, [1430](#)
- VARNAMES, [1445](#)
- VARTYPECOMPLEX, [1433](#)
- VARTYPEIMAGINARY, [1433](#)
- VARTYPEMINUS, [1433](#)
- VARTYPENONE, [1432](#)
- VARTYPEROOTOFUNITY, [1433](#)
- VECTBIT, [1439](#)
- VECTOMIN, [1418](#)
- VECTOR, [1391](#)
- VECTOR_, [1429](#)
- VECTORONLY, [1378](#)
- VECTOSUB, [1418](#)
- VECTOVEC, [1418](#)
- VERTEXFUNCTION, [1427](#)
- VORTRANMODE, [1387](#)
- WILDARGFUN, [1399](#)
- WILDARGINDEX, [1407](#)
- WILDARGSYMBOL, [1406](#)
- WILDARGVECTOR, [1407](#)
- WILDCARDS, [1420](#)
- WILDDUMMY, [1418](#)
- WITHAUTO, [1377](#)
- WITHBLOCKS, [1457](#)
- WITHEDGES, [1457](#)
- WITHERROR, [1367](#)
- WITHINSERTIONS, [1456](#)
- WITHONEPI, [1457](#)
- WITHOUTERROR, [1367](#)
- WITHOUTSEMICOLON, [1372](#)
- WITHSEMICOLON, [1372](#)
- WITHSYMMETRIZE, [1456](#)
- WRITEOUT, [1375](#)
- YESLYNDON, [1449](#)
- Z_, [1428](#)
- ZTOHARG, [1449](#)
- full
 - PF_BUFFER, [329](#)
- FullCleanUp
 - declare.h, [918](#)
 - module.c, [1765](#)
- FULLMAX
 - form3.h, [1327](#)
- fullName
 - dbase, [80](#)
 - P_const, [311](#)
- fullnamesize
 - P_const, [316](#)
- FullProto
 - N_const, [245](#)
- FullRenumbr
 - declare.h, [962](#)
 - reshuf.c, [2613](#)
- FullSymmetrize
 - declare.h, [874](#)
 - function.c, [1473](#)
- FUN_INFO, [142](#)
 - commute, [143](#)
 - location, [143](#)
 - numargs, [143](#)
 - numfunnies, [143](#)
 - numwildcards, [143](#)
 - symmet, [143](#)
 - tensor, [143](#)
- FunArg
 - T_const, [437](#)
- funargs
 - N_const, [248](#)
- FUNBIT
 - ftypes.h, [1439](#)
- func
 - KEYWORD, [163](#)
 - ReNuMbEr, [379](#)
- FUNCTION
 - ftypes.h, [1392](#)
- FuNcTiOn, [144](#)
 - commute, [145](#)
 - complex, [145](#)
 - dimension, [147](#)
 - flags, [146](#)
 - maxnumargs, [147](#)
 - minnumargs, [147](#)
 - name, [145](#)
 - namesize, [146](#)
 - node, [146](#)
 - number, [145](#)
 - spec, [146](#)
 - symmetric, [146](#)
 - symminfo, [145](#)
 - tabl, [145](#)
- function.c, [1471](#), [1474](#)
 - ChainIn, [1473](#)
 - ChainOut, [1473](#)
 - CompGroup, [1472](#)
 - FullSymmetrize, [1473](#)
 - MakeDirty, [1472](#)
 - MarkDirty, [1472](#)
 - MatchFunction, [1473](#)
 - PolyFunClean, [1472](#)
 - PolyFunDirty, [1472](#)
 - ScanFunctions, [1474](#)
 - SymFind, [1473](#)
 - SymGen, [1473](#)
 - Symmetrize, [1472](#)
- FunctionList
 - C_const, [43](#)
- FUNCTIONONLY
 - ftypes.h, [1378](#)
- FUNCTIONS
 - structs.h, [2905](#)
- Functions
 - C_const, [46](#)
- functions

- variable.h, [3205](#)
- FUNCTIONSORT
 - ftypes.h, [1431](#)
- FUNHEAD
 - ftypes.h, [1422](#)
- funinds
 - N_const, [249](#)
- FunInfo
 - N_const, [250](#)
- funisize
 - N_const, [254](#)
- FunLevel
 - declare.h, [846](#)
 - reshuf.c, [2613](#)
- funlocs
 - N_const, [248](#)
- FunMatchCy
 - declare.h, [853](#)
 - symmetr.c, [2936](#)
- FunMatchSy
 - declare.h, [853](#)
 - symmetr.c, [2936](#)
- FunNam
 - FixedGlobals, [136](#)
- funnum
 - ReNuMbEr, [380](#)
- FUNNYDOLLAR
 - ftypes.h, [1430](#)
- FUNNYVEC
 - ftypes.h, [1430](#)
- FUNNYWILD
 - ftypes.h, [1430](#)
- funoffset
 - R_const, [377](#)
- funpowers
 - C_const, [57](#)
- FunScratch
 - N_const, [250](#)
- funsizes
 - NeStInG, [269](#)
- FunSorts
 - N_const, [250](#)
- FUNTOFUN
 - ftypes.h, [1419](#)
- funwith
 - DICTIONARY, [83](#)
- fval
 - OPTIMIZE, [292](#)
- fwin.h, [1497](#), [1498](#)
 - ALTSEPARATOR, [1498](#)
 - CARRIAGEReturn, [1497](#)
 - LINEFEED, [1497](#)
 - P_term, [1497](#)
 - PATHSEPARATOR, [1498](#)
 - SEPARATOR, [1498](#)
 - WITH_ENV, [1498](#)
 - WITHRETURN, [1497](#)
 - WITHSYSTEM, [1497](#)
- fwrite_cached
 - checkpoint.c, [543](#)
- gamm
 - TrAcEs, [476](#)
- GAMMA
 - ftypes.h, [1393](#)
- GAMMA1
 - ftypes.h, [1429](#)
- GAMMA5
 - ftypes.h, [1429](#)
- gamma5
 - TrAcEs, [477](#)
- GAMMA6
 - ftypes.h, [1429](#)
- GAMMA7
 - ftypes.h, [1429](#)
- GAMMAFIVE
 - ftypes.h, [1393](#)
- GAMMAFUNCTION
 - ftypes.h, [1427](#)
- GAMMAI
 - ftypes.h, [1393](#)
- GAMMASEVEN
 - ftypes.h, [1394](#)
- GAMMASIX
 - ftypes.h, [1394](#)
- GarbHand
 - declare.h, [846](#)
 - sort.c, [2729](#)
- gcd
 - Fraction, [141](#)
- gcd_heuristic_failed, [147](#)
- gcd_heuristic_possible
 - polygcd.cc, [2259](#)
- gcd_linear_helper
 - polygcd.cc, [2260](#)
- gcd_mpoly
 - flintinterface.h, [1272](#)
- gcd_poly
 - flintinterface.h, [1272](#)
- GCDclean
 - declare.h, [1008](#)
 - ratio.c, [2509](#)
- GCDFUNCTION
 - ftypes.h, [1398](#)
- GCDfunction
 - declare.h, [1007](#)
 - ratio.c, [2506](#)
- GCDfunction3
 - declare.h, [1007](#)
 - ratio.c, [2506](#)
- GcdLong
 - declare.h, [846](#)
 - reken.c, [2562](#)
- GcdLong_fix_args
 - optimize.cc, [1996](#)
- GCDMAX
 - reken.c, [2555](#)

- GCDterms
 - declare.h, [1008](#)
 - ratio.c, [2508](#)
- gcmmod
 - M_const, [172](#)
- gcNumDollars
 - M_const, [183](#)
- gCnumpows
 - M_const, [186](#)
- gCodesFlag
 - M_const, [180](#)
- gDefDim
 - M_const, [178](#)
- gDefDim4
 - M_const, [178](#)
- GENCOMMUTE
 - ftypes.h, [1440](#)
- GenDiagrams
 - declare.h, [835](#)
- GENERALFUNCTION
 - ftypes.h, [1427](#)
- generate
 - MGraph, [207](#)
 - SProcess, [404](#)
 - T_MGraph, [447](#)
- generate_expression
 - optimize.cc, [2006](#)
- generate_instructions
 - optimize.cc, [1998](#)
- generate_output
 - optimize.cc, [2005](#)
- GenerateFactors
 - compiler.c, [726](#)
 - declare.h, [960](#)
- GenerateTopologies
 - declare.h, [835](#)
 - topowrap.cc, [3146](#)
- GenerateVertices
 - topowrap.cc, [3146](#)
- Generator
 - declare.h, [847](#)
 - proces.c, [2433](#)
- GenLoops
 - declare.h, [1029](#)
 - lus.c, [1689](#)
- genNext
 - SGroup, [387](#)
- GENNONCOMMUTE
 - ftypes.h, [1440](#)
- GenPaths
 - declare.h, [1030](#)
 - lus.c, [1690](#)
- GenSpace
 - sOrT, [398](#)
- GenTerms
 - sOrT, [398](#)
- gentopo.cc, [1499](#)
- gentopo.h, [1517](#)
- GenTopologies
 - declare.h, [834](#)
- get_brackets
 - optimize.cc, [1993](#)
- get_expression
 - optimize.cc, [1992](#)
- get_variables
 - flintinterface.h, [1272](#)
 - poly, [347](#)
- GetAppendChannel
 - declare.h, [897](#)
 - tools.c, [3088](#)
- GetAutoName
 - declare.h, [889](#)
 - names.c, [1828](#)
- GETBIDENTITY
 - declare.h, [820](#)
- GetBinom
 - declare.h, [848](#)
 - reken.c, [2562](#)
- GETCFROMEXTCHANNEL
 - declare.h, [821](#)
- GetChannel
 - declare.h, [897](#)
 - tools.c, [3087](#)
- GetChar
 - declare.h, [903](#)
 - pre.c, [2320](#)
- GETCOEF
 - declare.h, [801](#)
- GetContent
 - declare.h, [976](#)
 - execute.c, [1159](#)
- getCurrentExternalChannel
 - declare.h, [989](#)
 - extcmd.c, [1193](#)
- GetDbase
 - declare.h, [984](#)
 - minos.c, [1733](#)
- getDef
 - Options, [300](#)
- GetDollar
 - declare.h, [919](#)
 - names.c, [1829](#)
- GetDollarNumber
 - declare.h, [908](#)
 - pre.c, [2335](#)
- GetDolNum
 - declare.h, [967](#)
 - dollar.c, [1100](#)
- GetDoParam
 - compcomm.c, [629](#)
 - declare.h, [1006](#)
- GetFile
 - R_const, [374](#)
- GetFirstBracket
 - declare.h, [976](#)
 - execute.c, [1157](#)

- GetFirstTerm
 - declare.h, [976](#)
 - execute.c, [1158](#)
- getFLines
 - EGraph, [107](#)
- GetFromSpectator
 - declare.h, [1023](#)
 - spectator.c, [2792](#)
- GetFromStore
 - declare.h, [848](#)
 - store.c, [2835](#)
- GetFromStream
 - declare.h, [886](#)
 - tools.c, [3083](#)
- GetFunction
 - declare.h, [889](#)
 - names.c, [1828](#)
- GETIDENTITY
 - declare.h, [820](#)
- GetIfDollarFactor
 - compcomm.c, [629](#)
 - declare.h, [1006](#)
- GetIfDollarNum
 - declare.h, [838](#)
 - if.c, [1657](#)
- GetInput
 - declare.h, [903](#)
 - pre.c, [2320](#)
- GetLabel
 - declare.h, [959](#)
 - names.c, [1836](#)
- GetLastExprName
 - declare.h, [889](#)
 - names.c, [1828](#)
- getLegParticle
 - Assign, [19](#)
- GetLong
 - declare.h, [848](#)
 - reken.c, [2562](#)
- GetLongModInverses
 - declare.h, [867](#)
 - reken.c, [2561](#)
- GetModInverses
 - declare.h, [866](#)
 - reken.c, [2560](#)
- GetMoreFromMem
 - declare.h, [849](#)
 - store.c, [2835](#)
- GetMoreTerms
 - declare.h, [849](#)
 - store.c, [2834](#)
- GetName
 - declare.h, [888](#)
 - names.c, [1827](#)
- GetNode
 - declare.h, [888](#)
 - names.c, [1827](#)
- GetNumber
 - declare.h, [889](#)
 - names.c, [1828](#)
- GetOName
 - declare.h, [890](#)
 - names.c, [1828](#)
- GetOneFile
 - R_const, [375](#)
- GetOneTerm
 - declare.h, [849](#)
 - store.c, [2834](#)
- GetPID
 - form3.h, [1332](#)
- GetPosFile
 - declare.h, [896](#)
 - tools.c, [3087](#)
- getpower
 - optimize.cc, [1994](#)
- GetPreVar
 - declare.h, [901](#)
 - pre.c, [2321](#)
- GetRunningTime
 - declare.h, [863](#)
 - startup.c, [2805](#)
- GetSetupPar
 - declare.h, [902](#)
 - setfile.c, [2694](#)
- GETSTOP
 - declare.h, [801](#)
- GetStreamPosition
 - declare.h, [924](#)
 - tools.c, [3084](#)
- GetTable
 - declare.h, [849](#)
 - store.c, [2837](#)
- GetTableName
 - declare.h, [987](#)
 - minos.c, [1734](#)
- GETTERM
 - declare.h, [821](#)
- GetTerm
 - declare.h, [849](#)
 - store.c, [2834](#)
- getValue
 - Options, [298](#)
- GetVar
 - declare.h, [889](#)
 - names.c, [1828](#)
- GetWildcardName
 - declare.h, [923](#)
 - names.c, [1831](#)
- gextrasym
 - M_const, [174](#)
- gextrasymbols
 - M_const, [191](#)
- gFinalStats
 - M_const, [181](#)
- GFIVE
 - ftypes.h, [1436](#)

- gFortran90Kind
 - M_const, [174](#)
- gfunpowers
 - M_const, [179](#)
- ggextrasymp
 - M_const, [174](#)
- ggextrasympols
 - M_const, [192](#)
- ggFinalStats
 - M_const, [181](#)
- ggIndentSpace
 - M_const, [191](#)
- ggNoSpacesInNumbers
 - M_const, [183](#)
- ggnumextrasymp
 - M_const, [184](#)
- ggOldFactArgFlag
 - M_const, [184](#)
- ggOldGCDflag
 - M_const, [184](#)
- ggOldParallelStats
 - M_const, [182](#)
- ggProcessStats
 - M_const, [182](#)
- ggShortStatsMax
 - M_const, [191](#)
- ggStatsFlag
 - M_const, [187](#)
- ggThreadBalancing
 - M_const, [182](#)
- ggThreadBucketSize
 - M_const, [177](#)
- ggThreadsFlag
 - M_const, [181](#)
- ggThreadSortFileSynch
 - M_const, [182](#)
- ggThreadStats
 - M_const, [181](#)
- ggWTimeStatsFlag
 - M_const, [185](#)
- gld
 - T_EGraph, [443](#)
- GIDENT
 - ftypes.h, [1435](#)
- gIndentSpace
 - M_const, [191](#)
- ginsidefirst
 - M_const, [178](#)
- glsFortran90
 - M_const, [183](#)
- gLineLength
 - M_const, [187](#)
- GLOBAL
 - ftypes.h, [1439](#)
- GLOBALEXPRESSION
 - ftypes.h, [1423](#)
- GlobalFunctions
 - variable.h, [3207](#)
- GlobalIndices
 - variable.h, [3207](#)
- Globalize
 - declare.h, [923](#)
 - names.c, [1836](#)
- GLOBALMODULE
 - ftypes.h, [1370](#)
- globalnamefill
 - NaMeTree, [265](#)
- globalnodefill
 - NaMeTree, [265](#)
- GlobalSetElements
 - variable.h, [3208](#)
- GlobalSets
 - variable.h, [3207](#)
- GLOBALSPECTATORFLAG
 - ftypes.h, [1455](#)
- GlobalSymbols
 - variable.h, [3207](#)
- GlobalVectors
 - variable.h, [3207](#)
- Glue
 - declare.h, [849](#)
 - ratio.c, [2506](#)
- GMINUS
 - ftypes.h, [1436](#)
- gmodmode
 - M_const, [186](#)
- GMPSPREAD
 - float.c, [1286](#)
- gNamesFlag
 - M_const, [180](#)
- gncmod
 - M_const, [185](#)
- gNoSpacesInNumbers
 - M_const, [183](#)
- gnpowmod
 - M_const, [186](#)
- GNUCL_PREREQ
 - parallel.h, [2142](#)
- gNumDictionaries
 - O_const, [284](#)
- gnumelements
 - DICTIONARY, [83](#)
- gnumextrasymp
 - M_const, [184](#)
- gNumPre
 - P_const, [315](#)
- gOldFactArgFlag
 - M_const, [184](#)
- gOldGCDflag
 - M_const, [184](#)
- gOldParallelStats
 - M_const, [182](#)
- gOutNumberType
 - M_const, [186](#)
- gOutputMode
 - M_const, [186](#)

- gOutputSpaces
 - M_const, [186](#)
- gparallelflag
 - M_const, [180](#)
- GPLUS
 - ftypes.h, [1436](#)
- gPolyFun
 - M_const, [190](#)
- gPolyFunExp
 - M_const, [190](#)
- gPolyFunInv
 - M_const, [190](#)
- gPolyFunPow
 - M_const, [190](#)
- gPolyFunType
 - M_const, [190](#)
- gPolyFunVar
 - M_const, [190](#)
- gpowmod
 - M_const, [172](#)
- gProcessBucketSize
 - M_const, [176](#)
- gProcessStats
 - M_const, [182](#)
- gproperorderflag
 - M_const, [180](#)
- grcc.cc, [1519](#)
- grcc.h, [1638](#)
- grccmodel
 - MoDeL, [233](#)
- grccparam.h, [1653](#)
- GREATER
 - ftypes.h, [1437](#)
- GREATEREQUAL
 - ftypes.h, [1438](#)
- greedymaxperc
 - OPTIMIZE, [294](#)
- greedyminnum
 - OPTIMIZE, [293](#)
- greedytimelimit
 - OPTIMIZE, [293](#)
- group
 - MGraph, [210](#)
- groupLMom
 - EGraph, [107](#)
- gShortStatsMax
 - M_const, [191](#)
- gSizeCommutelnSet
 - M_const, [181](#)
- gSortType
 - M_const, [180](#)
- gStatsFlag
 - M_const, [180](#)
- gSubId
 - EGraph, [110](#)
- gThreadBalancing
 - M_const, [181](#)
- gThreadBucketSize
 - M_const, [177](#)
- gThreadsFlag
 - M_const, [181](#)
- gThreadSortFileSynch
 - M_const, [182](#)
- gThreadStats
 - M_const, [181](#)
- gTokensWriteFlag
 - M_const, [179](#)
- gUniTrace
 - M_const, [186](#)
- gUnitTrace
 - M_const, [186](#)
- gWTimeStatsFlag
 - M_const, [185](#)
- gzipCompress
 - R_const, [373](#)
- GZIPDEFAULT
 - fsizes.h, [1349](#)
- HAAKJE
 - ftypes.h, [1393](#)
- HAAKJE0
 - ftypes.h, [1392](#)
- HALFMAX
 - fsizes.h, [1343](#)
- halfmod
 - C_const, [48](#)
- Handle
 - FiLeDaTa, [132](#)
- handle
 - ChAnNeL, [75](#)
 - dbase, [79](#)
 - FiLe, [130](#)
 - StreaM, [418](#)
- HANDLERS, [147](#)
 - newhandle, [148](#)
 - newlogonly, [148](#)
 - oldhandle, [148](#)
 - oldlogonly, [148](#)
 - oldprinttype, [148](#)
 - oldsilent, [148](#)
- hash
 - node, [273](#)
- hash_range
 - optimize.cc, [1998](#)
- havesortdir
 - M_const, [192](#)
- HEADERFILE
 - ftypes.h, [1369](#)
- headermark
 - STOREHEADER, [413](#)
- headnode
 - NaMeTree, [266](#)
- helptb
 - tables.c, [2970](#)
- hi
 - VaRrEnUm, [483](#)
- HIDDENGEXPRESSION

- ftypes.h, [1424](#)
- HIDDENEXPRESSION
 - ftypes.h, [1424](#)
- HIDE
 - ftypes.h, [1422](#)
- hidefile
 - R_const, [371](#)
- HIDEGEXPRESSION
 - ftypes.h, [1424](#)
- HideLevel
 - C_const, [68](#)
- hidelevel
 - ExPrEsSiOn, [122](#)
- HIDELEXPRESSION
 - ftypes.h, [1424](#)
- HideSize
 - M_const, [175](#)
- highfirst
 - setfile.c, [2697](#)
- HighWarning
 - declare.h, [884](#)
 - message.c, [1716](#)
- HoldFlag
 - M_const, [188](#)
- horner
 - OPTIMIZE, [292](#)
- Horner_tree
 - optimize.cc, [1997](#)
- hornerdirection
 - OPTIMIZE, [293](#)
- HowMany
 - declare.h, [982](#)
 - if.c, [1657](#)
- hparallelflag
 - M_const, [180](#)
- hProcessBucketSize
 - M_const, [176](#)
- HTOZARG
 - ftypes.h, [1449](#)
- humanBytesSuffix
 - sort.c, [2734](#)
- HumanStatsFlag
 - C_const, [62](#)
- HumanString
 - sort.c, [2720](#)
- HUMANSTRLEN
 - sort.c, [2719](#)
- HUMANSUFFLEN
 - sort.c, [2720](#)
- humanTermsSuffix
 - sort.c, [2734](#)
- i
 - bufIPstruct, [37](#)
- iblocks
 - dbase, [79](#)
- iBufError
 - P_const, [314](#)
- iBuffer
 - C_const, [48](#)
- iBufferSize
 - C_const, [52](#)
- icode
 - lInput, [149](#)
 - Interaction, [162](#)
- id
 - EEdge, [98](#)
 - ENode, [117](#)
 - Interaction, [161](#)
 - MNode, [219](#)
 - Particle, [325](#)
 - Process, [365](#)
 - SProcess, [408](#)
 - T_MNode, [452](#)
- idallflag
 - T_const, [434](#)
- idallmaxnum
 - T_const, [434](#)
- idallnum
 - T_const, [434](#)
- IDFUNCTION
 - ftypes.h, [1403](#)
- idfunctionflag
 - N_const, [258](#)
- IDHEAD
 - ftypes.h, [1435](#)
- idoption
 - C_const, [56](#)
- iexp
 - declare.h, [899](#)
 - tools.c, [3095](#)
- if.c, [1656](#), [1659](#)
 - DoEndSwitch, [1657](#)
 - DolfStatement, [1657](#)
 - DoSwitch, [1657](#)
 - DoubleIfBuffers, [1657](#)
 - DoubleSwitchBuffers, [1658](#)
 - FindCase, [1658](#)
 - FindVar, [1657](#)
 - GetIfDollarNum, [1657](#)
 - HowMany, [1657](#)
 - SwitchSplitMerge, [1658](#)
 - SwitchSplitMergeRec, [1658](#)
- IfCount
 - C_const, [48](#)
- IFDOLLAR
 - ftypes.h, [1437](#)
- IFDOLLAREXTRA
 - ftypes.h, [1437](#)
- IFEXPRESSION
 - ftypes.h, [1437](#)
- IFFLOATNUMBER
 - ftypes.h, [1437](#)
- IfHeap
 - C_const, [48](#)
- IFISFACTORIZED
 - ftypes.h, [1437](#)

- IfLevel
 - C_const, [67](#)
- iflevel
 - SWITCH, [422](#)
- IFOCCURS
 - ftypes.h, [1437](#)
- IfStack
 - C_const, [48](#)
- IfSumCheck
 - C_const, [52](#)
- IFUSERFLAG
 - ftypes.h, [1437](#)
- IFW
 - flintinterface.cc, [1242](#)
- IgnoreDeprecation
 - M_const, [193](#)
- Input, [149](#)
 - cvallist, [150](#)
 - icode, [149](#)
 - name, [149](#)
 - nplistn, [149](#)
 - plistc, [150](#)
 - plistn, [149](#)
- ilist
 - NCand, [267](#)
 - NStack, [276](#)
- IMPLODEARG
 - ftypes.h, [1447](#)
- improve
 - optimization, [291](#)
- INANDOUT
 - minos.h, [1755](#)
- inbuffer
 - FiLe, [130](#)
 - StreaM, [418](#)
- IncDir
 - M_const, [173](#)
- incdollar
 - DoLoOp, [92](#)
- INCEXPRESSION
 - ftypes.h, [1424](#)
- INCLENG
 - declare.h, [801](#)
- Include
 - declare.h, [907](#)
 - pre.c, [2325](#)
- incMat
 - MNodeClass, [222](#)
 - T_MNodeClass, [455](#)
- incnum
 - DoLoOp, [91](#)
- incoef
 - SHvariables, [390](#)
- IndDum
 - M_const, [185](#)
 - N_const, [258](#)
- INDENTSPACE
 - fsizes.h, [1351](#)
- IndentSpace
 - O_const, [285](#)
- INDEX
 - ftypes.h, [1391](#)
- InDeX, [150](#)
 - dimension, [151](#)
 - flags, [151](#)
 - name, [150](#)
 - namesize, [151](#)
 - nmin4, [151](#)
 - node, [151](#)
 - number, [151](#)
 - type, [150](#)
- Index
 - FiLeDaTa, [132](#)
- index
 - DoLIaRS, [88](#)
 - PF_BUFFER, [330](#)
- index.c, [1673](#), [1674](#)
 - ClearBracketIndex, [1674](#)
 - FindBracket, [1673](#)
 - OpenBracketIndex, [1674](#)
 - PutBracketInIndex, [1673](#)
 - PutInside, [1674](#)
- INDEX_
 - ftypes.h, [1429](#)
- indexblock, [152](#)
 - flags, [152](#)
 - objects, [152](#)
 - position, [152](#)
 - previousblock, [152](#)
- indexbuffer
 - BrAcKeTiNfO, [33](#)
- indexbuffersize
 - BrAcKeTiNfO, [33](#)
- INDEXENTRY
 - structs.h, [2904](#)
- InDeXeNtRy, [153](#)
 - CompressSize, [154](#)
 - length, [154](#)
 - name, [155](#)
 - nfunctions, [155](#)
 - nindices, [154](#)
 - nsymbols, [154](#)
 - nvectors, [155](#)
 - position, [154](#)
 - size, [155](#)
 - variables, [154](#)
- indexfill
 - BrAcKeTiNfO, [34](#)
- IndexList
 - C_const, [43](#)
- INDEXONLY
 - ftypes.h, [1378](#)
- indi
 - ReNuMbEr, [379](#)
- Indices
 - C_const, [45](#)

- indices
 - parallel.h, [2143](#)
 - variable.h, [3206](#)
- indnum
 - ReNuMbEr, [379](#)
- INDTOIND
 - ftypes.h, [1418](#)
- INDTOSUB
 - ftypes.h, [1419](#)
- inexprlevel
 - C_const, [64](#)
- inexprstack
 - C_const, [52](#)
- inexprsumcheck
 - C_const, [64](#)
- InFbrack
 - O_const, [285](#)
- infile
 - R_const, [371](#)
- INFILEINDEX
 - structs.h, [2903](#)
- info
 - dbase, [78](#)
- InFunction
 - declare.h, [850](#)
 - proces.c, [2429](#)
- InHiBuf
 - R_const, [372](#)
- inicbufs
 - comtool.c, [763](#)
 - declare.h, [897](#)
- inictable
 - compiler.c, [724](#)
 - declare.h, [897](#)
- IniFbuffer
 - comtool.c, [767](#)
 - declare.h, [1007](#)
- iniinfo, [156](#)
 - entriesinindex, [156](#)
 - firstindexblock, [156](#)
 - firstnameblock, [157](#)
 - lastindexblock, [156](#)
 - lastnameblock, [157](#)
 - numberofindexblocks, [156](#)
 - numberofnamesblocks, [157](#)
 - numberoftables, [156](#)
- IniLine
 - declare.h, [851](#)
 - sch.c, [2655](#)
- INILOCK
 - declare.h, [818](#)
- IniModule
 - declare.h, [904](#)
 - pre.c, [2322](#)
- InInBuf
 - R_const, [372](#)
- INIRWLOCK
 - declare.h, [819](#)
- IniSpecialModule
 - declare.h, [904](#)
 - pre.c, [2322](#)
- init
 - MCBlock, [194](#)
 - MConn, [198](#)
 - MCOpI, [203](#)
 - MGraph, [207](#)
 - MNodeClass, [221](#)
 - T_EGraph, [442](#)
 - T_MNodeClass, [454](#)
- initAss
 - ENode, [116](#)
- initlc
 - FGInput, [127](#)
- initln
 - FGInput, [127](#)
- initlPart
 - Process, [365](#)
- INITNAMESIZE
 - fsizes.h, [1344](#)
- INITNODESIZE
 - fsizes.h, [1344](#)
- iniTools
 - declare.h, [999](#)
 - tools.c, [3092](#)
- initPerm
 - MOrbits, [238](#)
- initPresetExternalChannels
 - declare.h, [988](#)
 - extcmd.c, [1193](#)
- InitRecovery
 - checkpoint.c, [542](#)
 - declare.h, [996](#)
- inivar.h, [1683](#), [1684](#)
 - A, [1683](#)
 - BufferForOutput, [1684](#)
 - FG, [1683](#)
 - fixedsets, [1683](#)
 - setupfilename, [1684](#)
- IniVars
 - declare.h, [851](#)
 - startup.c, [2804](#)
- iniwranf
 - declare.h, [991](#)
 - reken.c, [2565](#)
- inline
 - form3.h, [1326](#)
- inlist
 - TrAcEn, [472](#)
 - TrAcEs, [474](#)
- inmem
 - ExPrEsSiOn, [122](#)
- InnerTest
 - C_const, [69](#)
- inNum
 - sOrT, [400](#)
- InNumMem

- T_const, [433](#)
- InOutBuf
 - P_const, [313](#)
- inparallelflag
 - C_const, [60](#)
- inPatches
 - sOrT, [396](#)
- inprimelist
 - T_const, [439](#)
- InputFileName
 - M_const, [173](#)
- INPUTONLY
 - minos.h, [1756](#)
- INPUTOUT
 - ftypes.h, [1374](#)
- INPUTSTREAM
 - ftypes.h, [1369](#)
- inscbuf
 - INSIDEINFO, [159](#)
- inscheme
 - O_const, [281](#)
- InScratch
 - N_const, [252](#)
- InScratch1
 - N_const, [252](#)
- InsertArg
 - argument.c, [491](#)
 - declare.h, [930](#)
- InsertTerm
 - declare.h, [851](#)
 - proces.c, [2430](#)
- inside
 - P_const, [311](#)
- InsideDollar
 - declare.h, [973](#)
 - dollar.c, [1095](#)
- insidefirst
 - C_const, [54](#)
- INSIDEINFO, [157](#)
 - buffer, [158](#)
 - inscbuf, [159](#)
 - numdollars, [158](#)
 - oldcbuf, [158](#)
 - oldcnumlhs, [159](#)
 - oldcompiletype, [158](#)
 - oldnumpotmoddollars, [158](#)
 - oldparallelflag, [158](#)
 - oldrbuf, [159](#)
 - size, [158](#)
- insidelevel
 - C_const, [64](#)
- insidestack
 - C_const, [52](#)
- insidesumcheck
 - C_const, [64](#)
- INSLENGTH
 - sort.c, [2720](#)
- InsTableTree
 - declare.h, [971](#)
 - tables.c, [2965](#)
- InsTree
 - comtool.c, [767](#)
 - declare.h, [960](#)
- insubexpbuffers
 - compiler.c, [727](#)
- integer_lcoeff
 - poly, [345](#)
- IntegerToFloat
 - float.c, [1287](#)
- Interact
 - M_const, [177](#)
- Interaction, [159](#)
 - ~Interaction, [160](#)
 - clist, [161](#)
 - csum, [161](#)
 - icode, [162](#)
 - id, [161](#)
 - Interaction, [160](#)
 - loop, [162](#)
 - mdl, [161](#)
 - name, [161](#)
 - nclist, [162](#)
 - nlegs, [162](#)
 - nplist, [162](#)
 - nslist, [162](#)
 - plist, [161](#)
 - prInteraction, [161](#)
 - slist, [161](#)
- interacts
 - Model, [230](#)
- internalset
 - TERMINFO, [467](#)
- INTFUNCTION
 - ftypes.h, [1396](#)
- INTOHIDE
 - ftypes.h, [1423](#)
- INTOHIDEGEXPRESSION
 - ftypes.h, [1425](#)
- INTOHIDELEXPRESSION
 - ftypes.h, [1425](#)
- intPart
 - Model, [230](#)
- intracestack
 - N_const, [254](#)
- intrct
 - DNode, [86](#)
 - ENode, [117](#)
- INUSE
 - ftypes.h, [1452](#)
- inverse_poly
 - flintinterface.h, [1272](#)
- INVERSEFACTORIAL
 - ftypes.h, [1395](#)
- INVERSEFUNCTION
 - ftypes.h, [1403](#)
- INVERSETABLE

- ftypes.h, [1444](#)
- invertices
 - MoDeL, [234](#)
- invfacnum
 - M_const, [189](#)
- InvPoly
 - declare.h, [1011](#)
 - ratio.c, [2510](#)
- iPatches
 - sOrT, [397](#)
- iPointer
 - C_const, [49](#)
- iranf
 - declare.h, [991](#)
 - reken.c, [2566](#)
- is_dense_univariate
 - poly, [344](#)
- is_integer
 - poly, [344](#)
- is_monomial
 - poly, [344](#)
- is_one
 - poly, [344](#)
- is_zero
 - poly, [344](#)
- IsBracket
 - O_const, [285](#)
- ISCOMPRESSED
 - ftypes.h, [1432](#)
- isConnected
 - MGraph, [208](#)
- isEq
 - Fraction, [141](#)
- ISEQUALPOS
 - declare.h, [813](#)
- ISEQUALPOSINC
 - declare.h, [811](#)
- IsExponentSign
 - declare.h, [1020](#)
 - dict.c, [1074](#)
- isExternal
 - EGraph, [105](#)
 - MGraph, [207](#)
- ISFACTORIZED
 - ftypes.h, [1432](#)
- isFermion
 - EGraph, [105](#)
- ISFORTRAN90
 - ftypes.h, [1389](#)
- IsFortran90
 - C_const, [61](#)
- ISGEPOS
 - declare.h, [814](#)
- ISGEPOSINC
 - declare.h, [811](#)
- IsIdStatement
 - compiler.c, [725](#)
 - declare.h, [956](#)
- ISINRHS
 - ftypes.h, [1432](#)
- isIsomorphic
 - Assign, [22](#)
 - MGraph, [208](#)
- ISLESSPOS
 - declare.h, [813](#)
- IsLikeVector
 - declare.h, [900](#)
 - tools.c, [3095](#)
- ISLYNDON
 - ftypes.h, [1448](#)
- ISLYNDONR
 - ftypes.h, [1448](#)
- ISMINPOS
 - declare.h, [813](#)
- IsMultipleOf
 - declare.h, [978](#)
 - dollar.c, [1096](#)
- IsMultiplySign
 - declare.h, [1021](#)
 - dict.c, [1074](#)
- ISNEGPOS
 - declare.h, [814](#)
- isNeutral
 - Particle, [324](#)
- isnextchar
 - Stream, [419](#)
- ISNOTEQUALPOS
 - declare.h, [813](#)
- ISNOTFORTRAN90
 - ftypes.h, [1388](#)
- ISNOTZEROPOS
 - declare.h, [814](#)
- isOptE
 - EGraph, [104](#)
- isOptM
 - MGraph, [209](#)
- isOrdLegs
 - Assign, [22](#)
- isOrdPLeg
 - Assign, [21](#)
- ISPOSPOS
 - declare.h, [814](#)
- IsRHS
 - compiler.c, [724](#)
 - declare.h, [956](#)
- IsSetMember
 - declare.h, [978](#)
 - dollar.c, [1096](#)
- iStop
 - C_const, [49](#)
- ISUNMODIFIED
 - ftypes.h, [1432](#)
- ISYMBOL
 - ftypes.h, [1405](#)
- ISZERO
 - ftypes.h, [1431](#)

- ISZEROPOS
 - declare.h, [814](#)
- ival
 - OPTIMIZE, [292](#)
- j3
 - TrAcEs, [475](#)
- j4
 - TrAcEs, [475](#)
- JUMPRATIO
 - fsizes.h, [1352](#)
- jumpratio
 - M_const, [185](#)
- KEEPZERO
 - ftypes.h, [1432](#)
- KeptInHold
 - R_const, [374](#)
- KEYWORD, [163](#)
 - flags, [163](#)
 - func, [163](#)
 - name, [163](#)
 - type, [163](#)
- KEYWORDV, [164](#)
 - flags, [164](#)
 - name, [164](#)
 - type, [164](#)
 - var, [164](#)
- KILL
 - pre.c, [2319](#)
- KILLALL
 - pre.c, [2319](#)
- killSignal
 - X_const, [487](#)
- killWholeGroup
 - X_const, [488](#)
- klow
 - ENode, [118](#)
- kstep
 - TrAcEs, [476](#)
- ktoi
 - sOrT, [396](#)
- l
 - node, [273](#)
- LabelNames
 - C_const, [49](#)
- Labels
 - C_const, [50](#)
- LARGEBUFFER
 - fsizes.h, [1347](#)
- LargeSize
 - sOrT, [397](#)
- last
 - SeTs, [383](#)
- last_monomial
 - poly, [349](#)
- last_monomial_index
 - poly, [349](#)
- lastdollar
 - DoLoOp, [92](#)
- lastindexblock
 - iniinfo, [156](#)
- lastLen
 - longMultiStruct, [168](#)
- lastnameblock
 - iniinfo, [157](#)
- lastnamespace
 - P_const, [311](#)
- lastnum
 - DoLoOp, [91](#)
- LBRACE
 - ftypes.h, [1383](#)
- lBuffer
 - sOrT, [394](#)
- lc3
 - TrAcEs, [477](#)
- lc4
 - TrAcEs, [477](#)
- LcmLong
 - declare.h, [847](#)
 - reken.c, [2563](#)
- lcoeff_multivar
 - poly, [346](#)
- lcoeff_univar
 - poly, [346](#)
- lDefDim
 - C_const, [55](#)
- lDefDim4
 - C_const, [55](#)
- LeaveNegative
 - T_const, [436](#)
- left
 - NaMeNode, [260](#)
 - NoDe, [270](#)
 - tree, [479](#)
- legcouple
 - MoDeL, [233](#)
 - TERMINFO, [466](#)
- legEdgeParticle
 - Assign, [19](#)
- legPart
 - Assign, [20](#)
- legParticle
 - EGraph, [108](#)
- length
 - InDeXeNtRy, [154](#)
- lenLONG
 - STOREHEADER, [413](#)
- lenPOINTER
 - STOREHEADER, [413](#)
- lenPOS
 - STOREHEADER, [413](#)
- lenWORD
 - STOREHEADER, [413](#)
- LESS
 - ftypes.h, [1438](#)

- LESSEQUAL
 - ftypes.h, [1438](#)
- level
 - R_const, [376](#)
 - SHvariables, [391](#)
 - TERMINFO, [467](#)
 - TrAcEn, [472](#)
 - TrAcEs, [478](#)
- LEVICIVITA
 - ftypes.h, [1395](#)
- IFill
 - sOrT, [394](#)
- lhdollarerror
 - P_const, [315](#)
- lhdollarflag
 - C_const, [60](#)
- lhs
 - CbUf, [72](#)
 - DICTIONARY_ELEMENT, [84](#)
- LHSIDE
 - ftypes.h, [1386](#)
- LHSIDEX
 - ftypes.h, [1386](#)
- LI2FUNCTION
 - ftypes.h, [1401](#)
- lijst
 - LIST, [165](#)
- LINEFEED
 - fwin.h, [1497](#)
 - unix.h, [3197](#)
- LineLength
 - C_const, [68](#)
- linenumber
 - StreaM, [416](#)
- LINFUNCTION
 - ftypes.h, [1402](#)
- LinkTree
 - declare.h, [890](#)
 - names.c, [1830](#)
- LIST, [165](#)
 - lijst, [165](#)
 - maxnum, [165](#)
 - message, [165](#)
 - num, [165](#)
 - numclear, [166](#)
 - numglobal, [166](#)
 - numtemp, [166](#)
 - size, [166](#)
- LISTEDLOOP
 - ftypes.h, [1375](#)
- listinprint
 - N_const, [250](#)
- ListPoly
 - T_const, [432](#)
- ListSymbols
 - T_const, [432](#)
- ListSymbolsSize
 - T_const, [435](#)
- lloser
 - NoDe, [271](#)
- lmom
 - EEdge, [99](#)
- LNFUNCTION
 - ftypes.h, [1400](#)
- LNUMBER
 - ftypes.h, [1392](#)
- lo
 - VaRrEnUm, [483](#)
- LOADDOLLAR
 - ftypes.h, [1420](#)
- LoadInputFile
 - declare.h, [903](#)
 - tools.c, [3083](#)
- LoadInstruction
 - declare.h, [905](#)
 - pre.c, [2322](#)
- loadmode
 - PROCEDURE, [362](#)
- LoadModel
 - declare.h, [1028](#)
- LoadStatement
 - declare.h, [905](#)
 - pre.c, [2322](#)
- LocalConvertToPoly
 - declare.h, [1004](#)
 - notation.c, [1942](#)
- LOCALEXPRESSION
 - ftypes.h, [1423](#)
- LocateBase
 - declare.h, [871](#)
 - minos.c, [1732](#)
- LocateFile
 - declare.h, [887](#)
 - tools.c, [3083](#)
- location
 - FUN_INFO, [143](#)
- LOCK
 - declare.h, [818](#)
- locwildvalue
 - T_const, [437](#)
- log
 - ParallelVars, [319](#)
- LogFileName
 - M_const, [173](#)
- LogHandle
 - C_const, [67](#)
- LogType
 - M_const, [187](#)
- LongCopy
 - declare.h, [920](#)
 - tools.c, [3091](#)
- LongLongCopy
 - declare.h, [920](#)
 - tools.c, [3091](#)
- longMultiStruct, [167](#)
 - buffer, [167](#)

- bufpos, 167
- lastLen, 168
- next, 168
- nPacks, 167
- packpos, 167
- LONGNUMBER
 - ftypes.h, 1436
- LongToLine
 - declare.h, 852
 - sch.c, 2656
- LOOKINGFORELSE
 - ftypes.h, 1372
- LOOKINGFORENDIF
 - ftypes.h, 1373
- LookInStream
 - declare.h, 887
 - tools.c, 3083
- loop
 - Interaction, 162
 - MCBlock, 194
 - MCOpI, 204
 - Process, 366
 - SProcess, 408
- LoopList
 - P_const, 311
- loopm
 - EGraph, 113
- LoopOutput
 - declare.h, 1029
 - lus.c, 1689
- LowerSortLevel
 - declare.h, 972
 - sort.c, 2733
- lowfirst
 - setfile.c, 2697
- LPARENTHESIS
 - ftypes.h, 1382
- IPatch
 - sOrT, 399
- IPolyFun
 - C_const, 68
- IPolyFunExp
 - C_const, 68
- IPolyFunInv
 - C_const, 68
- IPolyFunPow
 - C_const, 69
- IPolyFunType
 - C_const, 68
- IPolyFunVar
 - C_const, 68
- LRPARENTHESSES
 - ftypes.h, 1385
- ISortType
 - C_const, 58
- lsrc
 - NoDe, 271
- ITop
 - sOrT, 394
- IUniTrace
 - C_const, 64
- IUnitTrace
 - C_const, 66
- Lus
 - declare.h, 962
 - lus.c, 1688
- lus.c, 1688, 1691
 - AllLoops, 1689
 - AllOnePI, 1690
 - AllPaths, 1690
 - FindLus, 1688
 - GenLoops, 1689
 - GenPaths, 1690
 - LoopOutput, 1689
 - Lus, 1688
 - OutputOnePI, 1691
 - PathOutput, 1690
 - RemoveBridges, 1690
 - SortTheList, 1689
 - StartLoops, 1689
 - TakeOneLine, 1691
- IWorkPointer
 - T_const, 433
- IWorkSize
 - R_const, 373
- IWorkSpace
 - T_const, 429
- M
 - AllGlobals, 10
- m
 - MODNUM, 235
- M_alloc
 - declare.h, 820
- M_check1
 - declare.h, 978
 - tools.c, 3093
- M_const, 168
 - atstartup, 190
 - BracketFactors, 192
 - ClearStore, 192
 - CompressSize, 174
 - countfunnum, 189
 - dbufnum, 179
 - denomnum, 188
 - dollarzero, 190
 - DumInd, 185
 - exitflag, 191
 - expnum, 188
 - facnum, 189
 - fbufferize, 183
 - FileOnlyFlag, 177
 - FortranCont, 188
 - FromStdin, 192
 - gcmmod, 172
 - gcNumDollars, 183
 - gCnumpows, 186

gCodesFlag, 180
gDefDim, 178
gDefDim4, 178
gextrasym, 174
gextrasymbols, 191
gFinalStats, 181
gFortran90Kind, 174
gfunpowers, 179
ggextrasym, 174
ggextrasymbols, 192
ggFinalStats, 181
ggIndentSpace, 191
ggNoSpacesInNumbers, 183
ggnumextrasym, 184
ggOldFactArgFlag, 184
ggOldGCDflag, 184
ggOldParallelStats, 182
ggProcessStats, 182
ggShortStatsMax, 191
ggStatsFlag, 187
ggThreadBalancing, 182
ggThreadBucketSize, 177
ggThreadsFlag, 181
ggThreadSortFileSynch, 182
ggThreadStats, 181
ggWTimeStatsFlag, 185
gIndentSpace, 191
ginsidefirst, 178
glsFortran90, 183
gLineLength, 187
gmodmode, 186
gNamesFlag, 180
gncmod, 185
gNoSpacesInNumbers, 183
gnpowmod, 186
gnumextrasym, 184
gOldFactArgFlag, 184
gOldGCDflag, 184
gOldParallelStats, 182
gOutNumberType, 186
gOutputMode, 186
gOutputSpaces, 186
gparallelflag, 180
gPolyFun, 190
gPolyFunExp, 190
gPolyFunInv, 190
gPolyFunPow, 190
gPolyFunType, 190
gPolyFunVar, 190
gpowmod, 172
gProcessBucketSize, 176
gProcessStats, 182
gproperorderflag, 180
gShortStatsMax, 191
gSizeCommutelnSet, 181
gSortType, 180
gStatsFlag, 180
gThreadBalancing, 181
gThreadBucketSize, 177
gThreadsFlag, 181
gThreadSortFileSynch, 182
gThreadStats, 181
gTokensWriteFlag, 179
gUniTrace, 186
gUnitTrace, 186
gWTimeStatsFlag, 185
havesortdir, 192
HideSize, 175
HoldFlag, 188
hparallelflag, 180
hProcessBucketSize, 176
IgnoreDeprecation, 193
IncDir, 173
IndDum, 185
InputFileName, 173
Interact, 177
invfacnum, 189
jumpratio, 185
LogFileName, 173
LogType, 187
matchfunnum, 189
MaxBracketBufferSize, 176
maxFlevels, 182
MaxParLevel, 177
MaxStreamSize, 175
MaxTal, 185
MaxTer, 174
MaxWildcards, 187
mTraceDum, 187
MultiRun, 183
NumFixedFunctions, 179
NumFixedSets, 178
NumSpectatorFiles, 184
NumStoreCaches, 191
OffsetIndex, 187
OffsetVector, 187
OldChildTime, 176
OldMilliTime, 176
oldnumextrasymbols, 174
OldOrderFlag, 189
OldSecTime, 176
onerhs, 192
Ordering, 188
OutBuffer, 173
OutBufSize, 178
Path, 173
PrintTotalSize, 183
qError, 188
rbufnum, 179
RepMax, 187
resetTimeOnClear, 183
S0, 172
sbufnum, 179
ScratSize, 174
SetupDir, 173
SetupFile, 173

- shmWinSize, 176
- silent, 188
- SIOsize, 175
- SizeForSpectatorFiles, 184
- SizeStoreCache, 175
- SkipClears, 179
- SLargeSize, 175
- SMaxFpatches, 178
- SMaxPatches, 178
- SpectatorFiles, 174
- SpectatorSize, 177
- SSmallEsize, 175
- SSmallSize, 175
- StdOut, 178
- STermsInSmall, 175
- sumnum, 189
- sumpnum, 189
- SumTime, 177
- TempDir, 172
- TempSortDir, 173
- termfunnum, 189
- TimeLimit, 177
- totalnumberofthreads, 180
- tracebackflag, 188
- vectorzero, 192
- Willnd, 185
- WorkSize, 176
- zbufnum, 179
- zeropos, 172
- zerorhs, 192
- M_free
 - declare.h, 922
 - tools.c, 3093
- M_print
 - declare.h, 978
 - tools.c, 3093
- main
 - startup.c, 2804
- MAINSORT
 - ftypes.h, 1431
- MAJORVERSION
 - form3.h, 1325
- MAKEARGS
 - ftypes.h, 1446
- MakeDate
 - declare.h, 898
 - tools.c, 3091
- MakeDirty
 - declare.h, 852
 - function.c, 1472
- MakeDollarInteger
 - declare.h, 966
 - dollar.c, 1098
- MakeDollarMod
 - declare.h, 966
 - dollar.c, 1099
- MakeDubious
 - declare.h, 889
- names.c, 1835
- MakeGlobal
 - declare.h, 918
 - execute.c, 1156
- MakeInteger
 - argument.c, 493
 - declare.h, 929
- MakeInverses
 - declare.h, 866
 - reken.c, 2560
- MakeLongRational
 - declare.h, 970
 - reken.c, 2563
- MakeMod
 - argument.c, 493
 - declare.h, 929
- MakeModTable
 - declare.h, 853
 - reken.c, 2564
- MakeNameTree
 - declare.h, 891
 - names.c, 1830
- MAKERATIONAL
 - ftypes.h, 1403
- MakeRational
 - declare.h, 970
 - reken.c, 2563
- MakeSetupAllocs
 - declare.h, 975
 - setfile.c, 2696
- Malloc
 - declare.h, 885
 - tools.c, 3092
- Malloc1
 - declare.h, 885
 - tools.c, 3092
- mallocprotect.h, 1710
- MAPLEMODE
 - ftypes.h, 1387
- MarkDirty
 - declare.h, 852
 - function.c, 1472
- MarkPolyRatFunDirty
 - declare.h, 817
- MaskPointer
 - N_const, 248
- mass
 - PaRtlcLe, 322
- MASTER
 - parallel.h, 2140
- MATCH
 - ftypes.h, 1436
- match
 - SProcess, 405
- MatchArgument
 - declare.h, 853
 - symmetr.c, 2936
- MatchCy

- declare.h, [853](#)
- symmetr.c, [2936](#)
- MatchE
 - declare.h, [853](#)
 - symmetr.c, [2935](#)
- MATCHFUNCTION
 - ftypes.h, [1397](#)
- MatchFunction
 - declare.h, [854](#)
 - function.c, [1473](#)
- matchfunnum
 - M_const, [189](#)
- MatchIsPossible
 - declare.h, [964](#)
 - smart.c, [2713](#)
- MATHEMATICAMODE
 - ftypes.h, [1387](#)
- MaX
 - declare.h, [800](#)
- MAX_OPEN_FILES
 - form3.h, [1332](#)
- MAXBASES
 - minos.h, [1754](#)
- MaxBracket
 - R_const, [374](#)
- MAXBRACKETBUFFERSIZE
 - fsizes.h, [1349](#)
- MaxBracketBufferSize
 - M_const, [176](#)
- MAXBUILTINFUNCTION
 - ftypes.h, [1405](#)
- maxcase
 - SWITCH, [422](#)
- MAXCOMBUFRHS
 - fsizes.h, [1350](#)
- MAXCOUPLINGS
 - fsizes.h, [1352](#)
- maxcpl
 - Model, [230](#)
- maxdeg
 - EGraph, [112](#)
 - ENode, [117](#)
 - MGraph, [212](#)
 - MNodeClass, [224](#)
 - T_EGraph, [444](#)
 - T_MGraph, [448](#)
 - T_MNodeClass, [456](#)
- MAXDOLLARVARIABLES
 - fsizes.h, [1342](#)
- MaxDoLoops
 - variable.h, [3205](#)
- MaxDum
 - R_const, [376](#)
- MAXDUMMIES
 - fsizes.h, [1343](#)
- MAXENAME
 - fsizes.h, [1343](#)
- MaxExprSize
 - S_const, [381](#)
- MAXFILESTREAMSIZE
 - fsizes.h, [1343](#)
- MAXFLAGS
 - fsizes.h, [1345](#)
- MAXFLEVELS
 - fsizes.h, [1348](#)
- maxFlevels
 - M_const, [182](#)
- MAXFPATCHES
 - fsizes.h, [1347](#)
- MaxFpatches
 - sOrT, [399](#)
- MAXFUNCTION
 - ftypes.h, [1396](#)
- MaxFunSorts
 - N_const, [254](#)
- maxi
 - MiNmAx, [216](#)
- MAXIF
 - fsizes.h, [1345](#)
- MaxIf
 - C_const, [56](#)
- MAXINDEXSIZE
 - minos.h, [1755](#)
- MAXINNAMETREE
 - fsizes.h, [1342](#)
- MAXLABELS
 - fsizes.h, [1345](#)
- MaxLabels
 - C_const, [55](#)
- MAXLEGS
 - fsizes.h, [1352](#)
- MAXLEVELS
 - fsizes.h, [1346](#)
- MAXLHS
 - fsizes.h, [1346](#)
- maxlhs
 - CbUf, [73](#)
- MAXLINELENGTH
 - fsizes.h, [1351](#)
- MAXLONG
 - form3.h, [1328](#)
- maxloop
 - Model, [230](#)
- MAXLOOPS
 - fsizes.h, [1345](#)
- MAXMATCH
 - fsizes.h, [1345](#)
- MAXMULTIBRACKETLEVELS
 - fsizes.h, [1351](#)
- MAXNEST
 - fsizes.h, [1344](#)
- maxnlegs
 - Model, [230](#)
 - Process, [366](#)
- maxnum
 - LIST, [165](#)

- maxnumargs
 - FuNcTiOn, [147](#)
- MAXNUMBEROFNONCOMTERMS
 - normal.c, [1876](#)
- MAXNUMBERSIZE
 - fsizes.h, [1344](#)
- MaxNumPre
 - variable.h, [3205](#)
- MAXNUMSIZE
 - fsizes.h, [1349](#)
- MaxNumStreams
 - C_const, [56](#)
- MAXPARLEVEL
 - fsizes.h, [1344](#)
- MaxParLevel
 - M_const, [177](#)
- MAXPARTICLES
 - fsizes.h, [1352](#)
- MAXPATCHES
 - fsizes.h, [1347](#)
- MaxPatches
 - sOrT, [398](#)
- MAXPOINTS
 - topowrap.cc, [3146](#)
- MAXPOSITIVE
 - form3.h, [1328](#)
- MAXPOSITIVE2
 - form3.h, [1328](#)
- MAXPOSITIVE4
 - form3.h, [1328](#)
- MAXPOWER
 - fsizes.h, [1342](#)
- maxpower
 - STOREHEADER, [415](#)
 - SyMbOl, [424](#)
- MAXPOWEROF
 - ftypes.h, [1399](#)
- MaxPreAssignLevel
 - P_const, [316](#)
- MaxPreIfLevel
 - P_const, [314](#)
- MAXPRENAMESIZE
 - fsizes.h, [1342](#)
- MaxPreTypes
 - P_const, [314](#)
- MaxProcedures
 - variable.h, [3204](#)
- MAXRANGEINDICATOR
 - ftypes.h, [1447](#)
- MaxRenumScratch
 - N_const, [255](#)
- MAXREPEAT
 - fsizes.h, [1344](#)
- maxrhs
 - CbUf, [74](#)
- MAXSAVEFUNCTION
 - fsizes.h, [1343](#)
- MaxStreamSize
 - M_const, [175](#)
- MAXSUBEXPRESSIONS
 - fsizes.h, [1343](#)
- MaxSwitch
 - C_const, [67](#)
- MAXTABLECOMBUF
 - fsizes.h, [1350](#)
- MaxTal
 - M_const, [185](#)
- MAXTER
 - fsizes.h, [1343](#)
- MaxTer
 - M_const, [174](#)
- maxtermlevel
 - C_const, [59](#)
- maxtermsize
 - sOrT, [399](#)
- MaxTreeSize
 - CbUf, [74](#)
 - TaBIes, [464](#)
- maxvar
 - OPTIMIZERESULT, [296](#)
- MAXVARIABLES
 - fsizes.h, [1342](#)
- MAXWILDC
 - fsizes.h, [1346](#)
- MaxWildcards
 - M_const, [187](#)
- mBer
 - T_const, [438](#)
- MCBlock, [193](#)
 - ~MCBlock, [193](#)
 - DUMMYPADDING, [194](#)
 - edges, [194](#)
 - init, [194](#)
 - loop, [194](#)
 - MCBlock, [193](#)
 - nartps, [194](#)
 - nmedges, [194](#)
- MCBridge, [195](#)
 - ~MCBridge, [195](#)
 - MCBridge, [195](#)
 - next, [195](#)
 - nodes, [195](#)
- MCLEANFLAG
 - minos.h, [1755](#)
- MConn, [196](#)
 - ~MConn, [197](#)
 - addArtic, [198](#)
 - addBlock, [198](#)
 - addBlockSelf, [199](#)
 - addBridge, [198](#)
 - addOPlc, [198](#)
 - articuls, [199](#)
 - blksp, [200](#)
 - blkstk, [200](#)
 - blkstkptr, [202](#)
 - blocks, [199](#)

- bridges, [199](#)
- DUMMYPADDING, [202](#)
- init, [198](#)
- MConn, [197](#)
- na1blocks, [201](#)
- narticuls, [201](#)
- nblksp, [202](#)
- nblocks, [201](#)
- nbridges, [201](#)
- nctopic, [200](#)
- ne0bridges, [201](#)
- ne1bridges, [201](#)
- neblocks, [201](#)
- nlpopic, [200](#)
- nopic, [200](#)
- nosp, [202](#)
- nselfloops, [201](#)
- opics, [199](#)
- opisp, [199](#)
- opistk, [200](#)
- opistkptr, [202](#)
- print, [199](#)
- pushEdge, [198](#)
- pushNode, [198](#)
- sedges, [200](#)
- snodes, [200](#)
- mconn
 - MGraph, [215](#)
- MCOp, [202](#)
 - ~MCOp, [203](#)
 - ctloop, [204](#)
 - init, [203](#)
 - loop, [204](#)
 - MCOp, [203](#)
 - nedges, [204](#)
 - next, [204](#)
 - nlegs, [204](#)
 - nnodes, [203](#)
 - nodes, [203](#)
- mcts_best_schemes
 - optimize.cc, [2012](#)
- mcts_expr_score
 - optimize.cc, [2011](#)
- mcts_factorized
 - optimize.cc, [2011](#)
- mcts_root
 - optimize.cc, [2011](#)
- mcts_separated
 - optimize.cc, [2011](#)
- mcts_vars
 - optimize.cc, [2011](#)
- mctsdecaymode
 - OPTIMIZE, [294](#)
- mctsnumexpand
 - OPTIMIZE, [293](#)
- mctsnumkeep
 - OPTIMIZE, [293](#)
- mctsnumrepeat
 - OPTIMIZE, [293](#)
- mctsttimelimit
 - OPTIMIZE, [293](#)
- mdefined
 - TaBIEs, [463](#)
- mdel
 - TrAcEs, [475](#)
- MDIRTYFLAG
 - minos.h, [1755](#)
- mdl
 - Interaction, [161](#)
 - Particle, [325](#)
- mdum
 - TrAcEs, [476](#)
- me
 - ParallelVars, [318](#)
- mean
 - OptDef, [289](#)
- MemDebugFlag
 - C_const, [61](#)
- MEMORYMACROS
 - declare.h, [816](#)
- mepf
 - TrAcEs, [475](#)
- merge_operators
 - optimize.cc, [2001](#)
- MergeDotproductLists
 - declare.h, [1010](#)
 - ratio.c, [2508](#)
- MergePatches
 - declare.h, [854](#)
 - sort.c, [2730](#)
- MergeSymbolLists
 - declare.h, [1010](#)
 - ratio.c, [2508](#)
- MergeWithFloat
 - float.c, [1294](#)
- MesCall
 - declare.h, [920](#)
 - message.c, [1716](#)
- MesCerr
 - declare.h, [855](#)
 - message.c, [1716](#)
- MesComp
 - declare.h, [855](#)
 - message.c, [1716](#)
- MesNesting
 - declare.h, [817](#)
- MesPrint
 - message.c, [1716](#)
- message
 - LIST, [165](#)
- message.c, [1714](#), [1718](#)
 - Error0, [1715](#)
 - Error1, [1715](#)
 - Error2, [1715](#)
 - HighWarning, [1716](#)
 - MesCall, [1716](#)

- MesCerr, [1716](#)
- MesComp, [1716](#)
- MesPrint, [1716](#)
- MesWork, [1715](#)
- PrintSeq, [1717](#)
- PrintSubTerm, [1717](#)
- PrintTerm, [1716](#)
- PrintTermC, [1717](#)
- PrintWords, [1717](#)
- Warning, [1716](#)
- MessPreNesting
 - declare.h, [923](#)
 - pre.c, [2333](#)
- MesWork
 - declare.h, [920](#)
 - message.c, [1715](#)
- method
 - OPTIMIZE, [293](#)
- mfac
 - T_const, [433](#)
- MGraph, [205](#)
 - ~MGraph, [207](#)
 - adjMat, [212](#)
 - biconnME, [209](#)
 - bicount, [215](#)
 - bidef, [215](#)
 - bilow, [215](#)
 - bisearchME, [209](#)
 - block, [215](#)
 - c1PI, [212](#)
 - c1PINoTadBlock, [213](#)
 - cDiag, [212](#)
 - clist, [211](#)
 - cNoTadBlock, [213](#)
 - cNoTadpole, [213](#)
 - compMat, [208](#)
 - connectClass, [209](#)
 - connectLeg, [209](#)
 - connectNode, [209](#)
 - curcl, [212](#)
 - discardDisc, [214](#)
 - discardIso, [214](#)
 - discardRefine, [214](#)
 - DUMMYPADDING1, [215](#)
 - DUMMYPADDING2, [216](#)
 - egraph, [212](#)
 - esym, [212](#)
 - extself, [214](#)
 - generate, [207](#)
 - group, [210](#)
 - init, [207](#)
 - isConnected, [208](#)
 - isExternal, [207](#)
 - isIsomorphic, [208](#)
 - isOptM, [209](#)
 - maxdeg, [212](#)
 - mconn, [215](#)
 - MGraph, [206](#)
 - mld, [210](#)
 - mindeg, [211](#)
 - modmat, [215](#)
 - nCallRefine, [213](#)
 - nClasses, [211](#)
 - nEdges, [211](#)
 - newGraph, [210](#)
 - nExtern, [211](#)
 - ngconn, [213](#)
 - ngen, [213](#)
 - nLoops, [211](#)
 - nNodes, [211](#)
 - nodes, [210](#)
 - nsym, [212](#)
 - opi, [214](#)
 - opiloop, [214](#)
 - opt, [210](#)
 - orbits, [210](#)
 - permMat, [208](#)
 - pld, [211](#)
 - print, [207](#)
 - printAdjMat, [207](#)
 - refineClass, [208](#)
 - selfloop, [214](#)
 - tadblock, [215](#)
 - tadpole, [214](#)
 - visit, [208](#)
 - wscon, [213](#)
 - wsopi, [213](#)
- mgraph
 - Assign, [23](#)
 - EGraph, [109](#)
 - SProcess, [407](#)
- mgrcount
 - Process, [365](#)
 - SProcess, [409](#)
- mld
 - EGraph, [109](#)
 - MGraph, [210](#)
- MiN
 - declare.h, [800](#)
- MINALLOC
 - fsizes.h, [1352](#)
- mincase
 - SWITCH, [422](#)
- mindeg
 - MGraph, [211](#)
 - T_MGraph, [448](#)
- MINFUNCTION
 - ftypes.h, [1396](#)
- mini
 - MiNmAx, [216](#)
- MiNmAx, [216](#)
 - maxi, [216](#)
 - mini, [216](#)
 - size, [216](#)
- minnumargs
 - FuNcTiOn, [147](#)

- MINORVERSION
 - form3.h, [1325](#)
- minos.c, [1729](#), [1736](#)
 - AddObject, [1734](#)
 - AddTableName, [1733](#)
 - AddToIndex, [1734](#)
 - CFD, [1730](#)
 - ComposeTableNames, [1733](#)
 - convertblock, [1731](#)
 - convertiniinfo, [1731](#)
 - convertnamesblock, [1731](#)
 - CTD, [1730](#)
 - DeleteObject, [1735](#)
 - ExistsObject, [1735](#)
 - FindTableNumber, [1734](#)
 - FreeTableBase, [1733](#)
 - GetDbase, [1733](#)
 - GetTableName, [1734](#)
 - LocateBase, [1732](#)
 - minosread, [1731](#)
 - minoswrite, [1731](#)
 - NewDbase, [1733](#)
 - OpenDbase, [1733](#)
 - PutTableNames, [1734](#)
 - ReadijObject, [1735](#)
 - ReadIndex, [1732](#)
 - ReadIniInfo, [1733](#)
 - ReadObject, [1735](#)
 - str_dup, [1731](#)
 - withoutflush, [1736](#)
 - WriteIndex, [1732](#)
 - WriteIndexBlock, [1732](#)
 - WriteIniInfo, [1732](#)
 - WriteNamesBlock, [1732](#)
 - WriteObject, [1734](#)
- minos.h, [1753](#), [1757](#)
 - ELEMENTSIZE, [1755](#)
 - FROMDISK, [1755](#)
 - INANDOUT, [1755](#)
 - INPUTONLY, [1756](#)
 - MAXBASES, [1754](#)
 - MAXINDEXSIZE, [1755](#)
 - MCLEANFLAG, [1755](#)
 - MDIRTYFLAG, [1755](#)
 - minosread, [1756](#)
 - minoswrite, [1756](#)
 - NAMETABLESIZE, [1755](#)
 - NOCOMPRESS, [1756](#)
 - NUMOBJECTS, [1754](#)
 - OUTPUTONLY, [1756](#)
 - TODISK, [1755](#)
 - withoutflush, [1757](#)
- minosread
 - minos.c, [1731](#)
 - minos.h, [1756](#)
- minoswrite
 - minos.c, [1731](#)
 - minos.h, [1756](#)
- minpower
 - SyMbOI, [424](#)
- MINPOWEROF
 - ftypes.h, [1399](#)
- MInput, [217](#)
 - cnamlist, [217](#)
 - defpart, [217](#)
 - name, [217](#)
 - ncouple, [217](#)
- minsidefirst
 - C_const, [55](#)
- MINSPEC
 - ftypes.h, [1430](#)
- minvar
 - OPTIMIZERESULT, [296](#)
- MinVecArg
 - T_const, [437](#)
- MINVECTOR
 - ftypes.h, [1391](#)
- MIXED
 - ftypes.h, [1377](#)
- MIXED2
 - ftypes.h, [1377](#)
- mkCImat
 - MNodeClass, [222](#)
 - T_MNodeClass, [454](#)
- mkFlist
 - MNodeClass, [222](#)
 - T_MNodeClass, [454](#)
- mkNdCl
 - MNodeClass, [222](#)
 - T_MNodeClass, [454](#)
- mkSlaveInfile
 - ParallelVars, [319](#)
- mkSPProcess
 - Process, [364](#)
- MLOCK
 - declare.h, [819](#)
- MLONG
 - form3.h, [1333](#)
- mm
 - TabIEs, [461](#)
- MNode, [218](#)
 - class, [219](#)
 - cmxdeg, [219](#)
 - cmindeg, [219](#)
 - deg, [219](#)
 - extloop, [219](#)
 - freelg, [220](#)
 - id, [219](#)
 - MNode, [218](#)
 - visited, [220](#)
- MNodeClass, [220](#)
 - ~MNodeClass, [221](#)
 - clCmp, [221](#)
 - clist, [223](#)
 - clmat, [223](#)
 - clord, [223](#)

- cmaxdeg, [223](#)
 - cmindeg, [223](#)
 - cmpMNCArray, [222](#)
 - copy, [221](#)
 - flg0, [224](#)
 - flg1, [224](#)
 - flg2, [224](#)
 - flist, [223](#)
 - incMat, [222](#)
 - init, [221](#)
 - maxdeg, [224](#)
 - mkCIMat, [222](#)
 - mkFlist, [222](#)
 - mkNdCI, [222](#)
 - MNodeClass, [221](#)
 - nClasses, [224](#)
 - ndcl, [223](#)
 - nNodes, [224](#)
 - printMat, [221](#)
 - reorder, [222](#)
- mnumlhs
 - CbUf, [74](#)
- mnumrhs
 - CbUf, [74](#)
- mod
 - poly, [351](#)
- MOD2FUNCTION
 - ftypes.h, [1396](#)
- mode
 - dbase, [78](#)
 - TaBIEs, [465](#)
- MoDeL, [232](#)
 - couplings, [233](#)
 - dummy, [234](#)
 - error, [234](#)
 - grccmodel, [233](#)
 - invertices, [234](#)
 - legcouple, [233](#)
 - name, [233](#)
 - ncouplings, [234](#)
 - nparticles, [233](#)
 - nvertices, [233](#)
 - sizecouplings, [234](#)
 - sizevertices, [234](#)
 - vertices, [233](#)
- Model, [225](#)
 - ~Model, [226](#)
 - addInteraction, [227](#)
 - addInteractionEnd, [227](#)
 - addParticle, [226](#)
 - addParticleEnd, [226](#)
 - allPart, [229](#)
 - allParticles, [228](#)
 - antiParticle, [228](#)
 - cnlist, [229](#)
 - cplgcp, [231](#)
 - cplglg, [231](#)
 - cplgnvl, [231](#)
 - cplgvl, [231](#)
 - defpart, [231](#)
 - DUMMYPADDING, [231](#)
 - findInteractionCode, [227](#)
 - findInteractionName, [227](#)
 - findMClass, [228](#)
 - findParticleCode, [227](#)
 - findParticleName, [227](#)
 - interacts, [230](#)
 - intPart, [230](#)
 - maxcpl, [230](#)
 - maxloop, [230](#)
 - maxnlegs, [230](#)
 - Model, [226](#)
 - nallPart, [229](#)
 - name, [229](#)
 - ncouple, [229](#)
 - ncplgcp, [231](#)
 - nInteracts, [230](#)
 - nintPart, [230](#)
 - normalParticle, [228](#)
 - nParticles, [229](#)
 - particleCode, [228](#)
 - particleName, [227](#)
 - particles, [229](#)
 - pdef, [229](#)
 - prModel, [226](#)
 - prParticleArray, [228](#)
 - skipFLine, [231](#)
 - vdef, [230](#)
- model
 - Assign, [23](#)
 - EGraph, [108](#)
 - Options, [301](#)
 - Output, [307](#)
 - Process, [364](#)
 - SProcess, [406](#)
- model.c, [1758](#)
- ModelLevel
 - C_const, [63](#)
- modeloptions
 - R_const, [377](#)
- models
 - C_const, [46](#)
- modelspace
 - C_const, [63](#)
- MODFUNCTION
 - ftypes.h, [1396](#)
- modinverses
 - C_const, [50](#)
- MODLOCAL
 - ftypes.h, [1444](#)
- modmat
 - MGraph, [215](#)
- MODMAX
 - ftypes.h, [1443](#)
- MODMIN
 - ftypes.h, [1444](#)

- modmode
 - C_const, [65](#)
- modn
 - poly, [356](#)
- MODNONE
 - ftypes.h, [1443](#)
- MODNUM, [235](#)
 - a, [235](#)
 - m, [235](#)
 - na, [235](#)
 - nm, [235](#)
- modnum
 - POLYMOD, [358](#)
- MoDoPtDoLIaRS, [236](#)
 - number, [236](#)
 - type, [236](#)
- ModOptdollars
 - variable.h, [3210](#)
- ModOptDolList
 - C_const, [44](#)
- modp
 - poly, [356](#)
- modpowers
 - C_const, [47](#)
- MODSUM
 - ftypes.h, [1443](#)
- module.c, [1764](#), [1768](#)
 - CoModOption, [1765](#)
 - CoModuleOption, [1765](#)
 - DoExecStatement, [1767](#)
 - DoinParallel, [1767](#)
 - DoModDollar, [1767](#)
 - DoModLocal, [1767](#)
 - DoModMax, [1766](#)
 - DoModMin, [1766](#)
 - DoModSum, [1766](#)
 - DoNoParallel, [1766](#)
 - DonotinParallel, [1767](#)
 - DoParallel, [1766](#)
 - DoPipeStatement, [1767](#)
 - DoPolyfun, [1766](#)
 - DoPolyratfun, [1766](#)
 - DoProcessBucket, [1767](#)
 - ExecModule, [1765](#)
 - ExecStore, [1765](#)
 - FullCleanUp, [1765](#)
 - ModuleInstruction, [1765](#)
 - SetSpecialMode, [1765](#)
- MODULEINSTACK
 - ftypes.h, [1372](#)
- ModuleInstruction
 - declare.h, [904](#)
 - module.c, [1765](#)
- Modulus
 - declare.h, [855](#)
 - reken.c, [2564](#)
- MOEBIUS
 - ftypes.h, [1404](#)
- Moebius
 - declare.h, [991](#)
 - reken.c, [2565](#)
- moebiustable
 - R_const, [372](#)
- moebiustablesizes
 - R_const, [377](#)
- momc
 - T_EEdge, [440](#)
- momn
 - T_EEdge, [440](#)
- monomial_compare
 - poly, [349](#)
- monomial_larger, [236](#)
 - monomial_larger, [236](#)
 - operator(), [237](#)
- MOrbits, [237](#)
 - ~MOrbits, [237](#)
 - flist, [239](#)
 - fromPerm, [238](#)
 - initPerm, [238](#)
 - MOrbits, [237](#)
 - nd2or, [239](#)
 - nNodes, [238](#)
 - nOrbits, [238](#)
 - or2nd, [239](#)
 - print, [238](#)
 - toOrbits, [238](#)
- MoveDummies
 - declare.h, [855](#)
 - reshuf.c, [2613](#)
- mparallelflag
 - C_const, [59](#)
- mpi.c, [1777](#), [1794](#)
 - MPI_ERRCODE_CHECK, [1779](#)
 - PF_Bcast, [1782](#)
 - PF_Broadcast, [1788](#)
 - PF_IRecvRbuf, [1782](#)
 - PF_ISendSbuf, [1781](#)
 - PF_LibInit, [1779](#)
 - PF_LibTerminate, [1780](#)
 - PF_LongMultiBroadcast, [1793](#)
 - PF_LongMultiPackImpl, [1791](#)
 - PF_LongMultiUnpackImpl, [1792](#)
 - PF_LongSinglePack, [1789](#)
 - PF_LongSingleReceive, [1790](#)
 - PF_LongSingleSend, [1790](#)
 - PF_LongSingleUnpack, [1789](#)
 - PF_maxDollarChunkSize, [1793](#)
 - PF_Pack, [1785](#)
 - PF_PACKSIZE, [1779](#)
 - PF_PackString, [1786](#)
 - PF_PrepereLongMultiPack, [1791](#)
 - PF_PrepereLongSinglePack, [1788](#)
 - PF_PreperePack, [1785](#)
 - PF_PrintPackBuf, [1784](#)
 - PF_Probe, [1780](#)
 - PF_RawProbe, [1784](#)

- PF_RawRecv, 1783
- PF_RawSend, 1783
- PF_RealTime, 1779
- PF_Receive, 1787
- PF_RecvWbuf, 1781
- PF_Send, 1787
- PF_Unpack, 1785
- PF_UnpackString, 1786
- PF_WaitRbuf, 1782
- MPI_Barrier
 - mpidbg.h, 1816
- MPI_Bcast
 - mpidbg.h, 1816
- MPI_Bsend
 - mpidbg.h, 1813
- MPI_Cancel
 - mpidbg.h, 1816
- MPI_ERRCODE_CHECK
 - mpi.c, 1779
- MPI_Ibsend
 - mpidbg.h, 1814
- MPI_lprobe
 - mpidbg.h, 1815
- MPI_irecv
 - mpidbg.h, 1814
- MPI_Irsend
 - mpidbg.h, 1814
- MPI_Isend
 - mpidbg.h, 1814
- MPI_Issend
 - mpidbg.h, 1814
- MPI_Probe
 - mpidbg.h, 1816
- MPI_Recv
 - mpidbg.h, 1813
- MPI_Rsend
 - mpidbg.h, 1813
- MPI_Send
 - mpidbg.h, 1813
- MPI_Ssend
 - mpidbg.h, 1813
- MPI_Test
 - mpidbg.h, 1814
- MPI_Test_cancelled
 - mpidbg.h, 1816
- MPI_Testall
 - mpidbg.h, 1815
- MPI_Testany
 - mpidbg.h, 1815
- MPI_Testsome
 - mpidbg.h, 1815
- MPI_Wait
 - mpidbg.h, 1814
- MPI_Waitall
 - mpidbg.h, 1815
- MPI_Waitany
 - mpidbg.h, 1815
- MPI_Waitsome
 - mpidbg.h, 1815
- mpidbg.h, 1815
- mpidbg.h, 1811, 1817
 - MPI_Barrier, 1816
 - MPI_Bcast, 1816
 - MPI_Bsend, 1813
 - MPI_Cancel, 1816
 - MPI_Ibsend, 1814
 - MPI_lprobe, 1815
 - MPI_irecv, 1814
 - MPI_Irsend, 1814
 - MPI_Isend, 1814
 - MPI_Issend, 1814
 - MPI_Probe, 1816
 - MPI_Recv, 1813
 - MPI_Rsend, 1813
 - MPI_Send, 1813
 - MPI_Ssend, 1813
 - MPI_Test, 1814
 - MPI_Test_cancelled, 1816
 - MPI_Testall, 1815
 - MPI_Testany, 1815
 - MPI_Testsome, 1815
 - MPI_Wait, 1814
 - MPI_Waitall, 1815
 - MPI_Waitany, 1815
 - MPI_Waitsome, 1815
- MPIDBG_EXTARG, 1813
- MPIDBG_Out, 1813
- MPIDBG_RANK, 1813
- MPIDBG_EXTARG
 - mpidbg.h, 1813
- MPIDBG_Out
 - mpidbg.h, 1813
- MPIDBG_RANK
 - mpidbg.h, 1813
- mpoly
 - flint::mpoly, 240
- mpoly_ctx
 - flint::mpoly_ctx, 241
- mpoly_factor
 - flint::mpoly_factor, 242
- mpower
 - O_const, 284
- mProcessBucketSize
 - C_const, 53
- mTraceDum
 - M_const, 187
- mul
 - COST, 76
 - poly, 350
- mul_heap
 - poly, 352
- mul_mpoly
 - flintinterface.h, 1273
- mul_one_term
 - poly, 352
- mul_poly
 - flintinterface.h, 1273

- mul_univar
 - poly, [352](#)
- MulFloats
 - float.c, [1287](#)
- MULfunc
 - declare.h, [1010](#)
 - ratio.c, [2510](#)
- MULFUNCTION
 - ftypes.h, [1403](#)
- MulLong
 - declare.h, [855](#)
 - reken.c, [2558](#)
- Mully
 - declare.h, [856](#)
 - reken.c, [2556](#)
- mulpat
 - T_const, [437](#)
- MULPOS
 - declare.h, [812](#)
- MulRat
 - declare.h, [856](#)
 - reken.c, [2557](#)
- MulRatToFloat
 - float.c, [1288](#)
- MultDo
 - declare.h, [856](#)
 - reshuf.c, [2614](#)
- MultiBracketBuf
 - C_const, [51](#)
- MultiBracketLevels
 - C_const, [61](#)
- MULTIINDENTSPACE
 - fsizes.h, [1351](#)
- MULTIPLEOF
 - ftypes.h, [1436](#)
- MULTIPLYARG
 - ftypes.h, [1448](#)
- MultiplyToLine
 - declare.h, [1021](#)
 - sch.c, [2656](#)
- MultiplyWithTerm
 - declare.h, [1009](#)
 - ratio.c, [2507](#)
- MultiRun
 - M_const, [183](#)
- MultiThreaded
 - S_const, [382](#)
- multp
 - EGraph, [110](#)
- MUNLOCK
 - declare.h, [820](#)
- MUSTCLEANPRF
 - ftypes.h, [1421](#)
- MustTestTable
 - C_const, [65](#)
- my_random_shuffle
 - optimize.cc, [1992](#)
- mytime.cc, [1825](#)
- mytime.h, [1825](#)
- N
 - AllGlobals, [10](#)
- n
 - DiStRiBuTe, [85](#)
 - PeRmUtE, [327](#)
 - PeRmUtEp, [328](#)
- n1
 - DiStRiBuTe, [85](#)
- n1PI
 - T_MGraph, [449](#)
- n2
 - DiStRiBuTe, [85](#)
- N_const, [242](#)
 - argaddress, [246](#)
 - arglist, [251](#)
 - arglistsize, [254](#)
 - cmod, [251](#)
 - compressSize, [255](#)
 - compressSpace, [251](#)
 - cTerm, [246](#)
 - currentTerm, [251](#)
 - deferskipped, [252](#)
 - DisOrderFlag, [256](#)
 - DumFound, [247](#)
 - DumFunPlace, [247](#)
 - dummyrenumlist, [248](#)
 - DumPlace, [247](#)
 - EndNest, [245](#)
 - ErrorInDollar, [253](#)
 - ExpectedSign, [258](#)
 - filenum, [255](#)
 - findPattern, [248](#)
 - findTerm, [248](#)
 - ForFindOnly, [248](#)
 - Frozen, [245](#)
 - FullProto, [245](#)
 - funargs, [248](#)
 - funinds, [249](#)
 - FunInfo, [250](#)
 - funisize, [254](#)
 - funlocs, [248](#)
 - FunScratch, [250](#)
 - FunSorts, [250](#)
 - idfunctionflag, [258](#)
 - IndDum, [258](#)
 - InScratch, [252](#)
 - InScratch1, [252](#)
 - intracestack, [254](#)
 - listinprint, [250](#)
 - MaskPointer, [248](#)
 - MaxFunSorts, [254](#)
 - MaxRenumScratch, [255](#)
 - nargs, [253](#)
 - ncmod, [258](#)
 - ninterms, [252](#)
 - nogroundlevel, [255](#)
 - NoScrat2, [249](#)

- numfargs, [253](#)
- numflocs, [253](#)
- NumFound, [256](#)
- numfuninfo, [254](#)
- NumFunSorts, [254](#)
- numlistingprint, [257](#)
- numoffuns, [253](#)
- NumTotWildArgs, [252](#)
- numtracesctack, [254](#)
- NumWild, [256](#)
- OldPosIn, [245](#)
- OldPosOut, [245](#)
- oldtype, [255](#)
- oldvalue, [256](#)
- patstop, [246](#)
- patternbuffer, [249](#)
- patternbuffersize, [257](#)
- PoinScratch, [249](#)
- poly_num_vars, [258](#)
- poly_vars, [251](#)
- poly_vars_type, [258](#)
- PolyFunTodo, [257](#)
- PolyNormFlag, [257](#)
- polysortflag, [255](#)
- RenumScratch, [250](#)
- RepFunList, [246](#)
- RepFunNum, [256](#)
- ReplaceScrat, [249](#)
- RepPoint, [246](#)
- RSize, [254](#)
- selecttermundo, [249](#)
- SHcombi, [251](#)
- SHcombisize, [252](#)
- SHvar, [251](#)
- SignCheck, [258](#)
- sizeselecttermundo, [257](#)
- SoScratC, [250](#)
- SplitScratch, [250](#)
- SplitScratch1, [250](#)
- SplitScratchSize, [252](#)
- SplitScratchSize1, [252](#)
- subsubveto, [255](#)
- TelInFun, [256](#)
- terfirstcomm, [247](#)
- termbuffer, [249](#)
- terstart, [247](#)
- terstop, [246](#)
- TeSuOut, [257](#)
- theposition, [245](#)
- tlistbuf, [251](#)
- tlistsize, [255](#)
- tohunt, [253](#)
- tracestack, [249](#)
- tryterm, [259](#)
- UsedFunction, [248](#)
- UsedIndex, [247](#)
- UsedOtherFind, [253](#)
- UsedSymbol, [247](#)
- UsedVector, [247](#)
- UseFindOnly, [253](#)
- WildArgs, [257](#)
- WildDirt, [256](#)
- WildEat, [257](#)
- WildReserve, [256](#)
- WildStop, [246](#)
- WildValue, [246](#)
- na
 - MODNUM, [235](#)
- na1blocks
 - MConn, [201](#)
- nadj2ptv
 - EGraph, [112](#)
- nAGraphs
 - Assign, [25](#)
 - Process, [368](#)
 - SProcess, [410](#)
- nallPart
 - Model, [229](#)
- name
 - ChAnNeL, [75](#)
 - dbase, [79](#)
 - DICTIONARY, [82](#)
 - DoLIARs, [88](#)
 - DoLoOp, [90](#)
 - DuBiOuS, [93](#)
 - ExPrEsSiOn, [122](#)
 - FiLe, [129](#)
 - fixedset, [137](#)
 - FuNcTiOn, [145](#)
 - IInput, [149](#)
 - InDeX, [150](#)
 - InDeXeNtRy, [155](#)
 - Interaction, [161](#)
 - KEYWORD, [163](#)
 - KEYWORDV, [164](#)
 - MInput, [217](#)
 - MoDeL, [233](#)
 - Model, [229](#)
 - NaMeNode, [260](#)
 - NaMeSpAcE, [263](#)
 - OptDef, [289](#)
 - Particle, [325](#)
 - PInput, [332](#)
 - pReVaR, [360](#)
 - PROCEDURE, [362](#)
 - SeTs, [383](#)
 - SpecTatoR, [402](#)
 - StreaM, [417](#)
 - SyMbOl, [424](#)
 - TaBIEbAsE, [458](#)
 - VeCtOr, [484](#)
- nameblock, [259](#)
 - names, [259](#)
 - position, [259](#)
 - previousblock, [259](#)
- namebuffer

- NaMeTree, 264
- NameConflict
 - declare.h, 894
 - names.c, 1836
- namefill
 - NaMeTree, 264
- NAMENODE
 - structs.h, 2904
- NaMeNode, 260
 - balance, 261
 - left, 260
 - name, 260
 - number, 261
 - parent, 260
 - right, 261
 - type, 261
- namenode
 - NaMeTree, 264
- NAMENOTFOUND
 - ftypes.h, 1440
- nameofexpr
 - OPTIMIZERESULT, 295
- names
 - nameblock, 259
- names.c, 1826, 1837
 - AddDollar, 1835
 - AddDubious, 1835
 - AddExpression, 1836
 - AddFunction, 1832
 - AddIndex, 1831
 - AddName, 1827
 - AddSet, 1834
 - AddSymbol, 1831
 - AddVector, 1832
 - AddWildcardName, 1830
 - ClearWildcardNames, 1830
 - CoAuto, 1835
 - CoCFunction, 1833
 - CoCommutelnSet, 1832
 - CoCTable, 1834
 - CoCTensor, 1833
 - CoDimension, 1832
 - CoFunction, 1832
 - CoIndex, 1831
 - CompactifyTree, 1829
 - CoNFunction, 1833
 - CoNTable, 1834
 - CoNTensor, 1833
 - CopyTree, 1830
 - CoSet, 1834
 - CoSymbol, 1831
 - CoTable, 1833
 - CoVector, 1832
 - DoDimension, 1831
 - DoElements, 1834
 - DoTable, 1833
 - DoTempSet, 1835
 - DumpNode, 1829
 - DumpTree, 1829
 - EmptyTable, 1834
 - EntVar, 1829
 - FreeNameTree, 1830
 - GetAutoName, 1828
 - GetDollar, 1829
 - GetFunction, 1828
 - GetLabel, 1836
 - GetLastExprName, 1828
 - GetName, 1827
 - GetNode, 1827
 - GetNumber, 1828
 - GetOName, 1828
 - GetVar, 1828
 - GetWildcardName, 1831
 - Globalize, 1836
 - LinkTree, 1830
 - MakeDubious, 1835
 - MakeNameTree, 1830
 - NameConflict, 1836
 - RemoveDollars, 1836
 - ReplaceDollar, 1835
 - ResetVariables, 1836
 - TestName, 1837
- NamesFlag
 - C_const, 57
- namesize
 - ExPrEsSiOn, 123
 - FuNcTiOn, 146
 - InDeX, 151
 - NaMeTree, 264
 - SeTs, 384
 - SyMbOl, 425
 - VeCtOr, 484
- NaMeSpAcE, 262
 - name, 263
 - next, 262
 - previous, 262
 - usernames, 262
- NAMETABLESIZE
 - minos.h, 1755
- NAMETREE
 - structs.h, 2904
- NaMeTree, 263
 - clearnamefill, 265
 - clearnodefill, 265
 - globalnamefill, 265
 - globalnodefill, 265
 - headnode, 266
 - namebuffer, 264
 - namefill, 264
 - namenode, 264
 - namesize, 264
 - nodefill, 264
 - nodesize, 264
 - oldnamefill, 265
 - oldnodefill, 265
- nAOPI

- Assign, 25
- Process, 368
- SProcess, 410
- nargs
 - N_const, 253
 - PaRtl, 320
 - pReVaR, 360
- narticuls
 - MConn, 201
- nartps
 - MCBlock, 194
- nBer
 - T_const, 438
- nblksp
 - MConn, 202
- nblocks
 - dbase, 79
 - MConn, 201
- nBridges
 - T_MGraph, 449
- nbridges
 - MConn, 201
- nCallRefine
 - MGraph, 213
 - T_MGraph, 450
- NCand, 266
 - ~NCand, 266
 - deg, 267
 - ilist, 267
 - NCand, 266
 - nilist, 267
 - prNCand, 267
 - st, 267
- ncase
 - SWITCHTABLE, 423
- NCInput, 268
 - cldeg, 268
 - clnum, 268
 - cltyp, 268
 - cmaxd, 268
 - cmind, 268
 - cple, 269
 - ptcl, 268
- ncl
 - ToPoTyPe, 470
- nclass
 - PNodeClass, 337
 - SProcess, 407
- nClasses
 - MGraph, 211
 - MNodeClass, 224
 - T_MGraph, 448
 - T_MNodeClass, 455
- nclist
 - Interaction, 162
- ncmod
 - C_const, 65
 - N_const, 258
- NCOPY
 - declare.h, 803
- NCOPYB
 - declare.h, 803
- NCOPYI
 - declare.h, 803
- NCOPYI32
 - declare.h, 804
- ncouple
 - MInput, 217
 - Model, 229
 - SProcess, 408
- ncouplings
 - MoDeL, 234
 - VeRtEx, 486
- ncplgcp
 - Model, 231
- nctopic
 - MConn, 200
- nd2cl
 - PNodeClass, 337
 - SProcess, 407
- nd2or
 - MOrbits, 239
- ndcl
 - MNodeClass, 223
 - T_MNodeClass, 456
- NDEBUG
 - form3.h, 1326
- ndiag
 - T_MGraph, 448
- ndtype
 - ENode, 117
- ne0bridges
 - MConn, 201
- ne1bridges
 - MConn, 201
- neblocks
 - MConn, 201
- neclass
 - SGroup, 388
- nEdges
 - Assign, 24
 - EGraph, 111
 - MGraph, 211
 - SProcess, 408
 - T_EGraph, 443
 - T_MGraph, 447
- nedges
 - DGraph, 81
 - MCOpI, 204
- NeedNumber
 - declare.h, 804
- NEG0_
 - ftypes.h, 1428
- NEG_
 - ftypes.h, 1428
- nElem

- SGroup, [387](#)
- nelem
 - SGroup, [388](#)
- Nest
 - T_const, [428](#)
- NESTING
 - structs.h, [2907](#)
- NeStInG, [269](#)
 - argsize, [269](#)
 - funsize, [269](#)
 - termsize, [269](#)
- NestingChecksum
 - declare.h, [817](#)
- nestingsum
 - SWITCH, [423](#)
- NestPoin
 - T_const, [428](#)
- NestStop
 - T_const, [428](#)
- nETotal
 - Assign, [24](#)
- neutral
 - Particle, [325](#)
- newAGraph
 - Options, [301](#)
- newbracketinfo
 - ExPrEsSiOn, [122](#)
- NEWCODE
 - reshuf.c, [2613](#)
- NewDbase
 - declare.h, [986](#)
 - minos.c, [1733](#)
- NEWFACTARG
 - ftypes.h, [1450](#)
- newGraph
 - MGraph, [210](#)
- newGroup
 - SGroup, [386](#)
- newhandle
 - HANDLERS, [148](#)
- newleg
 - ANode, [12](#)
- newlogonly
 - HANDLERS, [148](#)
- NEWLYDEFINEDEXPRESSION
 - ftypes.h, [1427](#)
- newmaxtermsize
 - sOrT, [400](#)
- newMGraph
 - Options, [301](#)
- NEWORDER
 - argument.c, [490](#)
- NewSort
 - declare.h, [856](#)
 - sort.c, [2721](#)
- NEWSPLITMERGE
 - sort.c, [2719](#)
- NEWSPLITMERGETIMSORT
 - sort.c, [2719](#)
- NEWSTATEMENT
 - ftypes.h, [1373](#)
- NEWTRICK
 - reken.c, [2555](#)
- next
 - BrAcKeTiNdEx, [32](#)
 - FiLeInDeX, [134](#)
 - longMultiStruct, [168](#)
 - MCBridge, [195](#)
 - MCOpi, [204](#)
 - NaMeSpAcE, [262](#)
 - StOrEcAcHe, [412](#)
- NEXTARG
 - declare.h, [808](#)
- nextchar
 - Stream, [419](#)
- nextElem
 - SGroup, [387](#)
- nExtern
 - Assign, [24](#)
 - EGraph, [111](#)
 - MGraph, [211](#)
 - Process, [366](#)
 - SProcess, [408](#)
 - T_EGraph, [444](#)
- NextFileIndex
 - declare.h, [823](#)
 - store.c, [2835](#)
- NextPrime
 - declare.h, [991](#)
 - reken.c, [2565](#)
- nfac
 - T_const, [438](#)
- nfactors
 - DoLIARs, [89](#)
- nfinal
 - FGInput, [127](#)
 - Process, [365](#)
 - SProcess, [407](#)
- nflines
 - EGraph, [114](#)
- nfun
 - PaRtl, [320](#)
- nfunctions
 - InDeXeNtRy, [155](#)
- ngconn
 - MGraph, [213](#)
 - T_MGraph, [450](#)
- ngen
 - MGraph, [213](#)
 - T_MGraph, [450](#)
- ngraphs
 - Process, [367](#)
- nhalfmod
 - C_const, [65](#)
- nilist
 - NCand, [267](#)

- NStack, [276](#)
- nincoef
 - SHvariables, [391](#)
- nindices
 - InDeXeNtRy, [154](#)
- ninitl
 - FGInput, [127](#)
 - Process, [365](#)
 - SProcess, [407](#)
- nInteracts
 - Model, [230](#)
- ninterms
 - N_const, [252](#)
 - sOrT, [398](#)
- nintPart
 - Model, [230](#)
- nlegs
 - AEdge, [8](#)
 - ANode, [13](#)
 - EEdge, [98](#)
 - Interaction, [162](#)
 - MCOpI, [204](#)
 - ToPoTyPe, [470](#)
- nlist
 - EFLine, [101](#)
- nLoops
 - EGraph, [112](#)
 - MGraph, [211](#)
 - T_MGraph, [448](#)
- nloops
 - ToPoTyPe, [470](#)
- nlpopic
 - MConn, [200](#)
- nm
 - MODNUM, [235](#)
- nmedges
 - MCBlock, [194](#)
- nMGraphs
 - Process, [367](#)
 - SProcess, [409](#)
- nmin4
 - InDeX, [151](#)
- NMIN4SHIFT
 - ftypes.h, [1390](#)
- nMOPI
 - Process, [367](#)
 - SProcess, [409](#)
- nNodes
 - Assign, [24](#)
 - EGraph, [111](#)
 - MGraph, [211](#)
 - MNodeClass, [224](#)
 - MOrbits, [238](#)
 - SProcess, [408](#)
 - T_EGraph, [443](#)
 - T_MGraph, [447](#)
 - T_MNodeClass, [455](#)
- nnodes
 - DGraph, [81](#)
 - MCOpI, [203](#)
 - PNodeClass, [337](#)
 - SGroup, [388](#)
- NOAUTO
 - ftypes.h, [1377](#)
- NOBRACKETACTIVE
 - execute.c, [1156](#)
- NOCHECKLOGTYPE
 - ftypes.h, [1390](#)
- NOCOMPRESS
 - minos.h, [1756](#)
- NoCompress
 - C_const, [61](#)
 - R_const, [373](#)
- NODE
 - parallel.c, [2069](#)
- NoDe, [270](#)
 - left, [270](#)
 - lloser, [271](#)
 - lsrc, [271](#)
 - right, [270](#)
 - rloser, [271](#)
 - rsrc, [271](#)
- node, [271](#)
 - calcHash, [273](#)
 - cmp, [272](#)
 - data, [273](#)
 - DoLIaRS, [88](#)
 - DuBiOuS, [93](#)
 - ExPrEsSiOn, [123](#)
 - FuNcTiOn, [146](#)
 - hash, [273](#)
 - InDeX, [151](#)
 - l, [273](#)
 - node, [272](#)
 - operator(), [272](#)
 - r, [273](#)
 - SeTs, [384](#)
 - sign, [273](#)
 - SyMbOl, [425](#)
 - VeCtOr, [484](#)
- NodeEq, [274](#)
 - operator(), [274](#)
- nodefill
 - NaMeTree, [264](#)
- NODEFUNCTION
 - ftypes.h, [1404](#)
- NodeHash, [274](#)
 - operator(), [274](#)
- noden
 - NStack, [275](#)
- nodes
 - AEdge, [8](#)
 - Assign, [25](#)
 - DGraph, [81](#)
 - EEdge, [98](#)
 - EGraph, [109](#)

- MCBridge, [195](#)
- MCOp, [203](#)
- MGraph, [210](#)
- T_EEdge, [440](#)
- T_EGraph, [444](#)
- T_MGraph, [448](#)
- nodesize
 - NaMeTree, [264](#)
- NODOUBLEMASK
 - ftypes.h, [1388](#)
- NOEXTSELF
 - ftypes.h, [1457](#)
- NOFUNPOWERS
 - ftypes.h, [1441](#)
- nogroundlevel
 - N_const, [255](#)
- NOINDEX
 - ftypes.h, [1430](#)
- NOINVERSES
 - ftypes.h, [1445](#)
- NOLYNDON
 - ftypes.h, [1449](#)
- NONODES
 - ftypes.h, [1457](#)
- NOPARALLEL_CONVPOLY
 - ftypes.h, [1371](#)
- NOPARALLEL_DOLLAR
 - ftypes.h, [1371](#)
- NOPARALLEL_NPROC
 - ftypes.h, [1371](#)
- NOPARALLEL_RHS
 - ftypes.h, [1371](#)
- NOPARALLEL_SPECTATOR
 - ftypes.h, [1371](#)
- NOPARALLEL_TBLDOLLAR
 - ftypes.h, [1371](#)
- NOPARALLEL_USER
 - ftypes.h, [1371](#)
- nopi
 - Process, [367](#)
- nopi2p
 - EGraph, [112](#)
- nopic
 - MConn, [200](#)
- nopicomp
 - EGraph, [112](#)
- nopisp
 - MConn, [202](#)
- NOQUADMASK
 - ftypes.h, [1388](#)
- nOrbits
 - MOrbits, [238](#)
- normal
 - Fraction, [141](#)
- normal.c, [1875](#), [1878](#)
 - BracketNormalize, [1878](#)
 - Commute, [1876](#)
 - CompareFunctions, [1876](#)
 - DetCommu, [1877](#)
 - DoDelta, [1876](#)
 - DoesCommu, [1877](#)
 - DoRevert, [1877](#)
 - DoTheta, [1876](#)
 - DropCoefficient, [1877](#)
 - DropSymbols, [1877](#)
 - ExtraSymbol, [1876](#)
 - MAXNUMBEROFNONCOMTERMS, [1876](#)
 - Normalize, [1876](#)
 - SymbolNormalize, [1877](#)
 - TestFunFlag, [1878](#)
 - TreatPolyRatFun, [1877](#)
- NORMALFORMAT
 - ftypes.h, [1388](#)
- Normalize
 - declare.h, [857](#)
 - normal.c, [1876](#)
- normalize
 - poly, [348](#)
- NORMALIZEFLAG
 - ftypes.h, [1435](#)
- NormalModulus
 - declare.h, [866](#)
 - reken.c, [2560](#)
- normalParticle
 - Model, [228](#)
- NormPolyTerm
 - declare.h, [1003](#)
 - notation.c, [1941](#)
- NORMSIZE
 - fsizes.h, [1344](#)
- NoScrat2
 - N_const, [249](#)
- NOSHELL
 - pre.c, [2320](#)
- NoShowInput
 - C_const, [54](#)
 - DoLoOp, [92](#)
- NOSNAILS
 - ftypes.h, [1457](#)
- NOSPACEFORMAT
 - ftypes.h, [1388](#)
- NoSpacesInNumbers
 - O_const, [282](#)
- NOTADPOLES
 - ftypes.h, [1456](#)
- notation.c, [1941](#), [1944](#)
 - ConvertFromPoly, [1942](#)
 - ConvertToPoly, [1941](#)
 - ExtraSymFun, [1943](#)
 - FindLocalSubterm, [1942](#)
 - FindSubexpression, [1943](#)
 - FindSubterm, [1942](#)
 - LocalConvertToPoly, [1942](#)
 - NormPolyTerm, [1941](#)
 - PrintExtraSymbol, [1943](#)
 - PrintSubtermList, [1943](#)

- PruneExtraSymbols, [1943](#)
- NOTEQUAL
 - ftypes.h, [1438](#)
- NOTRICK
 - ftypes.h, [1389](#)
- NOTSTARTPOS
 - declare.h, [813](#)
- NOUNPACK
 - ftypes.h, [1445](#)
- nPacks
 - longMultiStruct, [167](#)
- npadding
 - ToPoTyPe, [470](#)
- NPAIR1
 - fsizes.h, [1351](#)
- NPAIR2
 - fsizes.h, [1351](#)
- nParticles
 - Model, [229](#)
- nparticles
 - MoDeL, [233](#)
 - VeRtEx, [486](#)
- nplist
 - ECand, [95](#)
 - EStack, [119](#)
 - Interaction, [162](#)
- nplistn
 - Input, [149](#)
- npowmod
 - C_const, [65](#)
- nselfloops
 - MConn, [201](#)
- NSIG
 - portsignals.h, [2312](#)
- nSize
 - AStack, [28](#)
- nslist
 - Interaction, [162](#)
- NStack, [275](#)
 - ~NStack, [275](#)
 - deg, [275](#)
 - ilist, [276](#)
 - nilist, [276](#)
 - noden, [275](#)
 - NStack, [275](#)
 - print, [275](#)
 - st, [276](#)
- nStack
 - AStack, [28](#)
- nStackP
 - AStack, [28](#)
- nSubproc
 - Process, [366](#)
- nsym
 - EGraph, [110](#)
 - MGraph, [212](#)
 - T_EGraph, [444](#)
 - T_MGraph, [449](#)
- nsym1
 - EGraph, [110](#)
- nsymbols
 - InDeXeNtRy, [154](#)
- nt3
 - TrAcEs, [474](#)
- nt4
 - TrAcEs, [474](#)
- num
 - Fraction, [142](#)
 - LIST, [165](#)
 - TrAcEn, [472](#)
 - TrAcEs, [477](#)
- num_terms
 - BracketInfo, [35](#)
- num_visits
 - tree_node, [482](#)
- NUMARG
 - ftypes.h, [1446](#)
- numargs
 - FUN_INFO, [143](#)
 - PaRtl, [320](#)
- NUMARGSFUN
 - ftypes.h, [1395](#)
- number
 - FiLeInDeX, [134](#)
 - FuNcTiOn, [145](#)
 - InDeX, [151](#)
 - MoDoPtDoLIaRS, [236](#)
 - NaMeNode, [261](#)
 - PaRtlcLe, [321](#)
 - SyMbOl, [425](#)
 - VeCtOr, [484](#)
- number_of_terms
 - poly, [345](#)
- NUMBEREXTRAWORDS
 - tools.c, [3082](#)
- NumberFree
 - declare.h, [817](#)
- NumberMalloc
 - declare.h, [816](#)
- NumberMallocAddMemory
 - declare.h, [998](#)
 - tools.c, [3093](#)
- NumberMemHeap
 - T_const, [431](#)
- NumberMemMax
 - T_const, [435](#)
- NUMBERMEMSTARTNUM
 - tools.c, [3082](#)
- NumberMemTop
 - T_const, [435](#)
- numberofindexblocks
 - iniinfo, [156](#)
- numberofnamesblocks
 - iniinfo, [157](#)
- numberoftables
 - iniinfo, [156](#)

- numbers
 - DICTIONARY, [82](#)
- numblocks
 - File, [130](#)
- numbufs
 - PF_BUFFER, [330](#)
- numcases
 - SWITCH, [422](#)
- numclear
 - LIST, [166](#)
- numcommute
 - comtool.c, [768](#)
 - declare.h, [962](#)
- numcompares
 - sort.c, [2734](#)
- NumCopy
 - declare.h, [920](#)
 - tools.c, [3091](#)
- numdia
 - TERMINFO, [467](#)
- NumDictionaries
 - O_const, [283](#)
- NumDollars
 - variable.h, [3208](#)
- numdollars
 - INSIDEINFO, [158](#)
- NumDoLoops
 - variable.h, [3205](#)
- NumDubious
 - variable.h, [3208](#)
- numdum
 - CbUf, [72](#)
- numdummies
 - DoLIaRS, [89](#)
 - ExPrEsSiOn, [124](#)
 - TaBIes, [465](#)
- numelements
 - DICTIONARY, [82](#)
- NUMERATORSYMBOL
 - ftypes.h, [1406](#)
- NUMERICALLOOP
 - ftypes.h, [1375](#)
- NUMERICALVALUE
 - setfile.c, [2693](#)
- NumExpressions
 - variable.h, [3209](#)
- numextern
 - TERMINFO, [467](#)
- NUMFACS
 - fsizes.h, [1345](#)
- NUMFACTORS
 - ftypes.h, [1402](#)
- numfactors
 - ExPrEsSiOn, [124](#)
- numfargs
 - N_const, [253](#)
- NUMFIXED
 - fsizes.h, [1344](#)
- NumFixedFunctions
 - M_const, [179](#)
- NumFixedSets
 - M_const, [178](#)
- numflocs
 - N_const, [253](#)
- NumFound
 - N_const, [256](#)
- NumFunctions
 - variable.h, [3206](#)
- numfuninfo
 - N_const, [254](#)
- numfunnies
 - FUN_INFO, [143](#)
- NumFunSorts
 - N_const, [254](#)
- numglobal
 - LIST, [166](#)
- NumInBrack
 - O_const, [282](#)
- numind
 - TaBIes, [463](#)
- NumIndices
 - variable.h, [3206](#)
- numinfilelist
 - tools.c, [3098](#)
- NumLabels
 - C_const, [55](#)
- numlhs
 - CbUf, [73](#)
- numlistinprint
 - N_const, [257](#)
- NumListSymbols
 - T_const, [436](#)
- NumMem
 - T_const, [432](#)
- nummodels
 - C_const, [63](#)
- NumModOptdollars
 - variable.h, [3210](#)
- NUMOBJECTS
 - minos.h, [1754](#)
- numoffuns
 - N_const, [253](#)
- NumOldNumFactors
 - S_const, [382](#)
- NumOldOnFile
 - S_const, [382](#)
- NUMOPTIONS
 - fsizes.h, [1352](#)
- NumOutputChannels
 - variable.h, [3208](#)
- numpart
 - PaRtl, [321](#)
- numpoly
 - T_const, [436](#)
- NumPotModdollars
 - variable.h, [3210](#)

- NumPre
 - variable.h, [3205](#)
- NumPreSwitchStrings
 - P_const, [314](#)
- NumPreTypes
 - DoLoOp, [92](#)
 - P_const, [314](#)
- NumProcedures
 - variable.h, [3204](#)
- numrbufs
 - ParallelVars, [319](#)
- numrhs
 - CbUf, [73](#)
- numsbufs
 - ParallelVars, [319](#)
- NumSetElements
 - variable.h, [3207](#)
- NumSets
 - variable.h, [3207](#)
- NUMSPECSETS
 - ftypes.h, [1431](#)
- NumSpectatorFiles
 - M_const, [184](#)
- NUMSTORECACHES
 - fsizes.h, [1350](#)
- NumStoreCaches
 - M_const, [191](#)
- NumStreams
 - C_const, [56](#)
- numsym
 - POLYMOD, [358](#)
- NumSymbols
 - variable.h, [3206](#)
- NumTableBases
 - variable.h, [3207](#)
- NUMTABLEENTRIES
 - fsizes.h, [1346](#)
- numtables
 - TaBIeBAsE, [458](#)
- numtasks
 - ParallelVars, [318](#)
- numtemp
 - LIST, [166](#)
- NumTerms
 - CbUf, [72](#)
- NUMTERMSFUN
 - ftypes.h, [1398](#)
- NUMTOIND
 - ftypes.h, [1420](#)
- NUMTONUM
 - ftypes.h, [1420](#)
- numtopo
 - TERMINFO, [466](#)
- NumToStr
 - declare.h, [916](#)
 - tools.c, [3089](#)
- NUMTOSUB
 - ftypes.h, [1420](#)
- NUMTOSYM
 - ftypes.h, [1420](#)
- NumTotWildArgs
 - N_const, [252](#)
- numtracesctack
 - N_const, [254](#)
- numtree
 - CbUf, [74](#)
 - TaBIEs, [464](#)
- NumVectors
 - variable.h, [3206](#)
- NumWild
 - N_const, [256](#)
- NumWildcardNames
 - C_const, [55](#)
- numwildcards
 - FUN_INFO, [143](#)
- numxsymbol
 - variable.h, [3209](#)
- nvector
 - InDeXeNtRy, [155](#)
- nvert
 - SProcess, [408](#)
 - ToPoTyPe, [470](#)
- nvertices
 - MoDeL, [233](#)
- NwildC
 - C_const, [66](#)
- O
 - AllGlobals, [11](#)
- O_BACKWARD
 - ftypes.h, [1452](#)
- O_const, [276](#)
 - BlockSpaces, [282](#)
 - bracket, [279](#)
 - bufferedInd, [285](#)
 - CheckPower, [281](#)
 - CurBufWrt, [280](#)
 - CurDictFunWithArgs, [283](#)
 - CurDictInDollars, [284](#)
 - CurDictNotInFunctions, [284](#)
 - CurDictNumbers, [283](#)
 - CurDictNumberWarning, [283](#)
 - CurDictSpecials, [283](#)
 - CurDictVariables, [283](#)
 - CurrentDictionary, [283](#)
 - Dictionaries, [281](#)
 - DollarInOutBuffer, [282](#)
 - DollarOutBuffer, [279](#)
 - DollarOutSizeBuffer, [282](#)
 - DoubleFlag, [285](#)
 - ErrorBlock, [286](#)
 - FactorMode, [286](#)
 - FactorNum, [286](#)
 - FlipLONG, [280](#)
 - FlipPOINTER, [280](#)
 - FlipPOS, [280](#)
 - FlipWORD, [280](#)

- FortDotChar, [286](#)
- FortFirst, [285](#)
- gNumDictionaries, [284](#)
- IndentSpace, [285](#)
- InFbrack, [285](#)
- inscheme, [281](#)
- IsBracket, [285](#)
- mpower, [284](#)
- NoSpacesInNumbers, [282](#)
- NumDictionaries, [283](#)
- NumInBrack, [282](#)
- OptimizationLevel, [286](#)
- Optimize, [282](#)
- OptimizeResult, [278](#)
- OutFill, [279](#)
- OutInBuffer, [282](#)
- OutputLine, [278](#)
- OutSkip, [285](#)
- OutStop, [279](#)
- powerFlag, [284](#)
- PrintType, [285](#)
- RenumberVec, [281](#)
- ResizeData, [280](#)
- resizeFlag, [284](#)
- ResizeLONG, [281](#)
- ResizeNCWORD, [280](#)
- ResizePOINTER, [281](#)
- ResizePOS, [281](#)
- ResizeWORD, [280](#)
- SaveData, [278](#)
- SaveHeader, [278](#)
- schemenum, [284](#)
- SizeDictionaries, [283](#)
- tabstring, [279](#)
- tensorList, [281](#)
- termbuf, [279](#)
- transFlag, [284](#)
- wlen, [282](#)
- wpoin, [279](#)
- wpos, [279](#)
- O_CSE
 - ftypes.h, [1451](#)
- O_CSEGREEDY
 - ftypes.h, [1451](#)
- O_FORWARD
 - ftypes.h, [1451](#)
- O_FORWARDANDBACKWARD
 - ftypes.h, [1452](#)
- O_FORWARDORBACKWARD
 - ftypes.h, [1452](#)
- O_GREEDY
 - ftypes.h, [1451](#)
- O_MCTS
 - ftypes.h, [1451](#)
- O_NONE
 - ftypes.h, [1451](#)
- O_OCCURRENCE
 - ftypes.h, [1451](#)
- O_SIMULATED_ANNEALING
 - ftypes.h, [1451](#)
- obj1
 - DiStRiBuTe, [85](#)
- obj2
 - DiStRiBuTe, [85](#)
- objects, [287](#)
 - date, [287](#)
 - element, [288](#)
 - indexblock, [152](#)
 - PeRmUtE, [327](#)
 - PeRmUtEp, [328](#)
 - position, [287](#)
 - size, [287](#)
 - spare1, [288](#)
 - spare2, [288](#)
 - spare3, [288](#)
 - tablenumber, [287](#)
 - uncompressed, [287](#)
- occurrence_order
 - optimize.cc, [1994](#)
- ODD_
 - ftypes.h, [1428](#)
- OffsetIndex
 - M_const, [187](#)
- OffsetVector
 - M_const, [187](#)
- oldcbuf
 - INSIDEINFO, [158](#)
- OldChildTime
 - M_const, [176](#)
- oldcnumlhs
 - INSIDEINFO, [159](#)
- oldcompiletype
 - INSIDEINFO, [158](#)
- olddelay
 - StreaM, [419](#)
- OLDFACTARG
 - ftypes.h, [1450](#)
- OldFactArgFlag
 - C_const, [61](#)
- OldGCDflag
 - C_const, [61](#)
- oldhandle
 - HANDLERS, [148](#)
- oldlogonly
 - HANDLERS, [148](#)
- OldMilliTime
 - M_const, [176](#)
- oldnamefill
 - NaMeTree, [265](#)
- oldnodefill
 - NaMeTree, [265](#)
- oldnoshowinput
 - StreaM, [419](#)
- oldnumextrasymbols
 - M_const, [174](#)
- OldNumFactors

- S_const, [381](#)
- oldnumpotmoddollars
 - INSIDEINFO, [158](#)
- OldOnFile
 - S_const, [381](#)
- OldOrderFlag
 - M_const, [189](#)
- oldparallelflag
 - INSIDEINFO, [158](#)
- OldParallelStats
 - C_const, [58](#)
- OldPosIn
 - N_const, [245](#)
- OldPosOut
 - N_const, [245](#)
- oldprinttype
 - HANDLERS, [148](#)
- oldrbuf
 - INSIDEINFO, [159](#)
- OldSecTime
 - M_const, [176](#)
- oldsilent
 - HANDLERS, [148](#)
- OLDSTATEMENT
 - ftypes.h, [1373](#)
- OldTime
 - R_const, [372](#)
- oldtype
 - N_const, [255](#)
- Olduflags
 - S_const, [382](#)
- oldvalue
 - N_const, [256](#)
- Oldvflags
 - S_const, [381](#)
- OMPI_SKIP_MPICXX
 - parallel.h, [2143](#)
- one_byte
 - structs.h, [2906](#)
- ONEEXPRESSION
 - ftypes.h, [1375](#)
- ONEPARTICLEIRREDUCIBLE
 - ftypes.h, [1456](#)
- ONEPI
 - ftypes.h, [1405](#)
- onerhs
 - M_const, [192](#)
- onfile
 - ExPrEsSiOn, [121](#)
- ONLYFUNCTIONS
 - ftypes.h, [1452](#)
- ONOFFVALUE
 - setfile.c, [2694](#)
- OpenAddFile
 - declare.h, [894](#)
 - tools.c, [3084](#)
- OpenBracketIndex
 - declare.h, [979](#)
- index.c, [1674](#)
- OpenDbase
 - declare.h, [985](#)
 - minos.c, [1733](#)
- OpenDictionary
 - P_const, [315](#)
- openExternalChannel
 - declare.h, [988](#)
 - extcmd.c, [1193](#)
- OpenFile
 - declare.h, [894](#)
 - tools.c, [3084](#)
- OpenInput
 - declare.h, [885](#)
 - startup.c, [2803](#)
- OpenStream
 - declare.h, [887](#)
 - tools.c, [3083](#)
- OpenTemp
 - declare.h, [857](#)
 - store.c, [2832](#)
- OPER_ADD
 - optimize.cc, [2010](#)
- OPER_COMMA
 - optimize.cc, [2010](#)
- OPER_MUL
 - optimize.cc, [2010](#)
- opera.c, [1957](#), [1961](#)
 - Chisholm, [1960](#)
 - EpfCon, [1958](#)
 - EpfFind, [1958](#)
 - EpfGen, [1958](#)
 - TenVec, [1961](#)
 - TenVecFind, [1960](#)
 - Trace4, [1959](#)
 - Trace4Gen, [1959](#)
 - Trace4no, [1959](#)
 - TraceFind, [1960](#)
 - TraceN, [1960](#)
 - TraceNgen, [1960](#)
 - TraceNno, [1959](#)
 - Traces, [1960](#)
 - Trick, [1959](#)
- OperaFind
 - FixedGlobals, [135](#)
- Operation
 - FixedGlobals, [135](#)
- operator!=
 - poly, [343](#)
- operator<
 - BracketInfo, [35](#)
 - optimization, [290](#)
- operator<<
 - polyfact.cc, [2241](#)
- operator()
 - CSEEq, [77](#)
 - CSEHash, [77](#)
 - monomial_larger, [237](#)

- node, 272
- NodeEq, 274
- NodeHash, 274
- operator+
 - poly, 342
- operator+=
 - poly, 341
- operator-
 - poly, 342
- operator-=
 - poly, 341
- operator/
 - poly, 342
- operator/=
 - poly, 341
- operator=
 - poly, 343
 - tree_node, 481
- operator==
 - poly, 342
- operator%
 - poly, 342
- operator%=
 - poly, 342
- operator[]
 - poly, 343
- operator*
 - poly, 342
- operator*=
 - poly, 341
- opi
 - MGraph, 214
 - T_EGraph, 444
- opi2plp
 - EGraph, 112
- opicmp
 - EEdge, 99
- opiCount
 - EGraph, 113
- opics
 - MConn, 199
- opiloop
 - MGraph, 214
- opisp
 - MConn, 199
- opistk
 - MConn, 200
- opistkptr
 - MConn, 202
- opt
 - Assign, 24
 - EGraph, 108
 - MGraph, 210
 - Output, 307
 - Process, 364
 - SProcess, 406
 - ToPoTyPe, 469
- OptDef, 288
- defaultv, 289
- DUMMYPADDING, 289
- mean, 289
- name, 289
- OPTHEAD
 - ftypes.h, 1452
- optimization, 289
 - arg1, 291
 - arg2, 291
 - coeff, 291
 - eqnidxs, 291
 - improve, 291
 - operator<, 290
 - type, 291
- OptimizationLevel
 - O_const, 286
- OPTIMIZE, 292
 - debugflags, 294
 - fval, 292
 - greedymaxperc, 294
 - greedyminnum, 293
 - greedytimelimit, 293
 - horner, 292
 - hornerdirection, 293
 - ival, 292
 - mctsdecaymode, 294
 - mctsnumexpand, 293
 - mctsnumkeep, 293
 - mctsnumrepeat, 293
 - mctstimelimit, 293
 - method, 293
 - printstats, 294
 - salter, 294
 - schemeflags, 294
 - spare, 294
- Optimize
 - declare.h, 968
 - O_const, 282
 - optimize.cc, 2007
- optimize.cc, 1990, 2012
 - build_Horner_tree, 1996
 - buildTree, 2000
 - ClearOptimize, 2009
 - count_operators, 1993, 1994
 - count_operators_cse, 1999
 - count_operators_cse_topdown, 2000
 - DivLong_fix_args, 1996
 - do_optimization, 2002
 - find_Horner_MCTS, 2000
 - find_Horner_MCTS_expand_tree, 2000
 - find_optimizations, 2002
 - fixarg, 1995
 - GcdLong_fix_args, 1996
 - generate_expression, 2006
 - generate_instructions, 1998
 - generate_output, 2005
 - get_brackets, 1993
 - get_expression, 1992

- getpower, [1994](#)
- hash_range, [1998](#)
- Horner_tree, [1997](#)
- mcts_best_schemes, [2012](#)
- mcts_expr_score, [2011](#)
- mcts_factorized, [2011](#)
- mcts_root, [2011](#)
- mcts_separated, [2011](#)
- mcts_vars, [2011](#)
- merge_operators, [2001](#)
- my_random_shuffle, [1992](#)
- occurrence_order, [1994](#)
- OPER_ADD, [2010](#)
- OPER_COMMA, [2010](#)
- OPER_MUL, [2010](#)
- Optimize, [2007](#)
- optimize_best_Horner_schemes, [2010](#)
- optimize_best_instr, [2011](#)
- optimize_best_num_oper, [2011](#)
- optimize_best_vars, [2011](#)
- optimize_expr, [2010](#)
- optimize_expression_given_Horner, [2005](#)
- optimize_greedy, [2003](#)
- optimize_num_vars, [2010](#)
- optimize_print_code, [2007](#)
- partial_factorize, [2003](#)
- print_instr, [1992](#)
- print_tree, [1998](#)
- recycle_variables, [2004](#)
- simulated_annealing, [2000](#)
- term_compare, [1997](#)
- optimize_best_Horner_schemes
 - optimize.cc, [2010](#)
- optimize_best_instr
 - optimize.cc, [2011](#)
- optimize_best_num_oper
 - optimize.cc, [2011](#)
- optimize_best_vars
 - optimize.cc, [2011](#)
- optimize_expr
 - optimize.cc, [2010](#)
- optimize_expression_given_Horner
 - optimize.cc, [2005](#)
- optimize_greedy
 - optimize.cc, [2003](#)
- optimize_num_vars
 - optimize.cc, [2010](#)
- optimize_print_code
 - declare.h, [1019](#)
 - optimize.cc, [2007](#)
- OPTIMIZERESULT, [295](#)
 - code, [295](#)
 - codesize, [295](#)
 - exprnr, [296](#)
 - maxvar, [296](#)
 - minvar, [296](#)
 - nameofexpr, [295](#)
- OptimizeResult
 - O_const, [278](#)
- optintimes
 - T_const, [435](#)
- option
 - SHvariables, [392](#)
- OPTION0
 - compiler.c, [723](#)
- OPTION1
 - compiler.c, [723](#)
- OPTION2
 - compiler.c, [723](#)
- Options, [296](#)
 - ~Options, [297](#)
 - argag, [302](#)
 - argemg, [302](#)
 - argmg, [302](#)
 - begin, [300](#)
 - beginProc, [300](#)
 - beginSubProc, [300](#)
 - DUMMYPADDING, [302](#)
 - end, [300](#)
 - endmg, [302](#)
 - endProc, [300](#)
 - endSubProc, [300](#)
 - getDef, [300](#)
 - getValue, [298](#)
 - model, [301](#)
 - newAGraph, [301](#)
 - newMGraph, [301](#)
 - Options, [297](#)
 - out, [301](#)
 - outag, [302](#)
 - outmg, [302](#)
 - outModel, [301](#)
 - print, [298](#)
 - printLevel, [299](#)
 - printModel, [299](#)
 - proc, [301](#)
 - setDefaultValue, [298](#)
 - setEndMG, [299](#)
 - setErExit, [299](#)
 - setOutAG, [299](#)
 - setOutMG, [299](#)
 - setOutputF, [298](#)
 - setOutputP, [298](#)
 - setValue, [298](#)
 - sproc, [301](#)
 - time0, [303](#)
 - time1, [303](#)
 - values, [302](#)
- or2nd
 - MOrbits, [239](#)
- orbits
 - Assign, [24](#)
 - MGraph, [210](#)
- ORCOND
 - ftypes.h, [1438](#)
- ORDEREDSET

- ftypes.h, [1456](#)
- Ordering
 - M_const, [188](#)
- origin
 - C_const, [63](#)
- out
 - DiStRiBuTe, [85](#)
 - Options, [301](#)
- outag
 - Options, [302](#)
- outBeginF
 - Output, [305](#)
- outBeginP
 - Output, [305](#)
- OutBuffer
 - M_const, [173](#)
- OutBufSize
 - M_const, [178](#)
- outEGraphF
 - Output, [307](#)
- outEGraphP
 - Output, [307](#)
- outEndF
 - Output, [305](#)
- outEndP
 - Output, [305](#)
- outfile
 - R_const, [371](#)
- OutFill
 - O_const, [279](#)
- outfun
 - SHvariables, [390](#)
- outgrf
 - Output, [308](#)
- outgrfp
 - Output, [308](#)
- outgrp
 - Output, [308](#)
- outgrpp
 - Output, [308](#)
- OutInBuffer
 - O_const, [282](#)
- outmg
 - Options, [302](#)
- outModel
 - Options, [301](#)
- outModelF
 - Output, [307](#)
- outModelP
 - Output, [307](#)
- OutNumberType
 - C_const, [67](#)
- outproc
 - Output, [308](#)
- outProcBegin0
 - Output, [306](#)
- outProcBeginF
 - Output, [306](#)
- outProcBeginP
 - Output, [306](#)
- outProcEndF
 - Output, [306](#)
- outProcEndP
 - Output, [306](#)
- Output, [303](#)
 - ~Output, [304](#)
 - model, [307](#)
 - opt, [307](#)
 - outBeginF, [305](#)
 - outBeginP, [305](#)
 - outEGraphF, [307](#)
 - outEGraphP, [307](#)
 - outEndF, [305](#)
 - outEndP, [305](#)
 - outgrf, [308](#)
 - outgrfp, [308](#)
 - outgrp, [308](#)
 - outgrpp, [308](#)
 - outModelF, [307](#)
 - outModelP, [307](#)
 - outproc, [308](#)
 - outProcBegin0, [306](#)
 - outProcBeginF, [306](#)
 - outProcBeginP, [306](#)
 - outProcEndF, [306](#)
 - outProcEndP, [306](#)
 - Output, [304](#)
 - outSProcBeginF, [306](#)
 - outSProcBeginP, [306](#)
 - proc, [307](#)
 - proclid, [308](#)
 - setOutgrf, [305](#)
 - setOutgrp, [305](#)
 - sproc, [308](#)
- OutputLine
 - O_const, [278](#)
- OutputMode
 - C_const, [66](#)
- outputmode
 - sOrT, [400](#)
- OutputOnePI
 - lus.c, [1691](#)
- OUTPUTONLY
 - minos.h, [1756](#)
- OutputSpaces
 - C_const, [66](#)
- outsidfun
 - C_const, [57](#)
- OutSkip
 - O_const, [285](#)
- outSProcBeginF
 - Output, [306](#)
- outSProcBeginP
 - Output, [306](#)
- OutStop
 - O_const, [279](#)

- outterm
 - SHvariables, [390](#)
- outtohide
 - R_const, [373](#)
- P
 - AllGlobals, [11](#)
- p
 - BracketInfo, [36](#)
 - DoLoOp, [90](#)
 - PROCEDURE, [362](#)
- p1
 - PoSiTiOn, [359](#)
- P_const, [309](#)
 - AllowDelay, [315](#)
 - cComChar, [316](#)
 - ComChar, [316](#)
 - cprocedureExtension, [312](#)
 - DebugFlag, [316](#)
 - DelayPrevar, [315](#)
 - DollarList, [311](#)
 - eat, [315](#)
 - firstnamespace, [311](#)
 - fullname, [311](#)
 - fullnamesize, [316](#)
 - gNumPre, [315](#)
 - iBufError, [314](#)
 - InOutBuf, [313](#)
 - inside, [311](#)
 - lastnamespace, [311](#)
 - lhdollarerror, [315](#)
 - LoopList, [311](#)
 - MaxPreAssignLevel, [316](#)
 - MaxPreIfLevel, [314](#)
 - MaxPreTypes, [314](#)
 - NumPreSwitchStrings, [314](#)
 - NumPreTypes, [314](#)
 - OpenDictionary, [315](#)
 - PreAssignFlag, [313](#)
 - PreAssignLevel, [316](#)
 - PreAssignStack, [312](#)
 - PreContinuation, [313](#)
 - PreDebug, [315](#)
 - preError, [316](#)
 - preFill, [312](#)
 - PreIfLevel, [314](#)
 - PreIfStack, [312](#)
 - PreInsideLevel, [315](#)
 - PreOut, [314](#)
 - PreproFlag, [313](#)
 - preStart, [312](#)
 - preStop, [312](#)
 - PreSwitchLevel, [314](#)
 - PreSwitchModes, [313](#)
 - PreSwitchStrings, [312](#)
 - PreTypes, [313](#)
 - PreVarList, [311](#)
 - procedureExtension, [312](#)
 - ProcList, [311](#)
 - pSize, [313](#)
 - StopWatchZero, [313](#)
- P_term
 - fwin.h, [1497](#)
 - unix.h, [3197](#)
- Pack
 - declare.h, [858](#)
 - reken.c, [2556](#)
- PACK_LONG
 - parallel.c, [2067](#)
- PackFloat
 - float.c, [1291](#)
- packpos
 - longMultiStruct, [167](#)
- padding
 - SWITCH, [423](#)
 - T_EEdge, [441](#)
- PADDUMMY
 - form3.h, [1328](#)
- PADINT
 - form3.h, [1329](#)
- PADLONG
 - form3.h, [1329](#)
- PADPOINTER
 - form3.h, [1329](#)
- PADPOSITION
 - form3.h, [1328](#)
- PADWORD
 - form3.h, [1330](#)
- parallel
 - ParallelVars, [318](#)
- parallel.c, [2064](#), [2090](#)
 - _CHECK, [2067](#)
 - __CHECK, [2068](#)
 - CHECK, [2067](#)
 - DBGOUT, [2068](#)
 - DBGOUT_NINTERMS, [2068](#)
 - NODE, [2069](#)
 - PACK_LONG, [2067](#)
 - PF, [2089](#)
 - PF_BroadcastBuffer, [2079](#)
 - PF_BroadcastCBuf, [2084](#)
 - PF_BroadcastExpFlags, [2084](#)
 - PF_BroadcastExpr, [2085](#)
 - PF_BroadcastModifiedDollars, [2082](#)
 - PF_BroadcastNumber, [2078](#)
 - PF_BroadcastPreDollar, [2080](#)
 - PF_BroadcastRedefinedPreVars, [2083](#)
 - PF_BroadcastRHS, [2085](#)
 - PF_BroadcastString, [2079](#)
 - PF_CollectModifiedDollars, [2081](#)
 - PF_Deferred, [2074](#)
 - PF_EndSort, [2073](#)
 - PF_FlushStdOutBuffer, [2089](#)
 - PF_FreeErrorMessageBuffers, [2089](#)
 - PF_GetSlaveTimes, [2078](#)
 - PF_Init, [2076](#)
 - PF_InParallelProcessor, [2086](#)

- PF_IRecvRbuf, [2071](#)
- PF_LibInit, [2069](#)
- PF_LibTerminate, [2070](#)
- PF_MLock, [2088](#)
- PF_MUnlock, [2088](#)
- PF_Probe, [2070](#)
- PF_Processor, [2075](#)
- PF_RawProbe, [2073](#)
- PF_RawRecv, [2072](#)
- PF_RawSend, [2072](#)
- PF_RealTime, [2069](#)
- PF_RecvFile, [2087](#)
- PF_RecvWbuf, [2070](#)
- PF_RestoreInsideInfo, [2084](#)
- PF_SendFile, [2086](#)
- PF_SNDFILEBUFSIZE, [2068](#)
- PF_STATS_SIZE, [2068](#)
- PF_StoreInsideInfo, [2083](#)
- PF_Terminate, [2077](#)
- PF_WaitRbuf, [2071](#)
- PF_WriteFileToFile, [2088](#)
- PRINTFBUF, [2066](#)
- recvBuffer, [2069](#)
- SWAP, [2066](#)
- UNPACK_LONG, [2067](#)
- parallel.h, [2138](#), [2171](#)
 - GNUC_PREREQ, [2142](#)
 - indices, [2143](#)
 - MASTER, [2140](#)
 - OMPI_SKIP_MPICXX, [2143](#)
 - PF, [2170](#)
 - PF_ANY_MSGTAG, [2143](#)
 - PF_ANY_SOURCE, [2143](#)
 - PF_Bcast, [2144](#)
 - PF_Broadcast, [2149](#)
 - PF_BroadcastBuffer, [2160](#)
 - PF_BroadcastCBuf, [2164](#)
 - PF_BroadcastExpFlags, [2165](#)
 - PF_BroadcastExpr, [2166](#)
 - PF_BroadcastModifiedDollars, [2163](#)
 - PF_BroadcastNumber, [2159](#)
 - PF_BroadcastPreDollar, [2161](#)
 - PF_BroadcastRedefinedPreVars, [2164](#)
 - PF_BroadcastRHS, [2167](#)
 - PF_BroadcastString, [2160](#)
 - PF_BUFFER_MSGTAG, [2141](#)
 - PF_BYTE, [2143](#)
 - PF_CollectModifiedDollars, [2162](#)
 - PF_COMM, [2143](#)
 - PF_DATA_MSGTAG, [2141](#)
 - PF_Deferred, [2155](#)
 - PF_DOLLAR_MSGTAG, [2141](#)
 - PF_EMPTY_MSGTAG, [2142](#)
 - PF_ENDBUFFER_MSGTAG, [2141](#)
 - PF_EndSort, [2154](#)
 - PF_ENDSORT_MSGTAG, [2141](#)
 - PF_FlushStdOutBuffer, [2170](#)
 - PF_GetSlaveTimes, [2159](#)
 - PF_Init, [2157](#)
 - PF_InParallelProcessor, [2167](#)
 - PF_INT, [2143](#)
 - PF_ISendSbuf, [2144](#)
 - PF_LOG_MSGTAG, [2142](#)
 - PF_LongMultiBroadcast, [2154](#)
 - PF_LongMultiPack, [2144](#)
 - PF_LongMultiPackImpl, [2152](#)
 - PF_LongMultiUnpack, [2144](#)
 - PF_LongMultiUnpackImpl, [2153](#)
 - PF_LongSinglePack, [2150](#)
 - PF_LongSingleReceive, [2151](#)
 - PF_LongSingleSend, [2151](#)
 - PF_LongSingleUnpack, [2150](#)
 - PF_maxDollarChunkSize, [2170](#)
 - PF_MISC_MSGTAG, [2142](#)
 - PF_MLock, [2169](#)
 - PF_MUnlock, [2169](#)
 - PF_OPT_COLLECT_MSGTAG, [2142](#)
 - PF_OPT_HORNER_MSGTAG, [2142](#)
 - PF_OPT_MCTS_MSGTAG, [2142](#)
 - PF_Pack, [2146](#)
 - PF_PackString, [2147](#)
 - PF_PrepareLongMultiPack, [2152](#)
 - PF_PrepareLongSinglePack, [2149](#)
 - PF_PreparePack, [2146](#)
 - PF_Processor, [2156](#)
 - PF_RawRecv, [2145](#)
 - PF_RawSend, [2145](#)
 - PF_READY_MSGTAG, [2141](#)
 - PF_Receive, [2149](#)
 - PF_RecvFile, [2168](#)
 - PF_RESET, [2140](#)
 - PF_RestoreInsideInfo, [2166](#)
 - PF_Send, [2148](#)
 - PF_SendFile, [2167](#)
 - PF_STDOUT_MSGTAG, [2142](#)
 - PF_StoreInsideInfo, [2166](#)
 - PF_TERM_MSGTAG, [2141](#)
 - PF_Terminate, [2158](#)
 - PF_TIME, [2140](#)
 - PF_Unpack, [2147](#)
 - PF_UnpackString, [2148](#)
 - PF_WriteFileToFile, [2169](#)
- PARALLELFLAG
 - ftypes.h, [1371](#)
- parallelflag
 - C_const, [59](#)
- ParallelVars, [317](#)
 - exprbufsize, [319](#)
 - exprtodo, [319](#)
 - log, [319](#)
 - me, [318](#)
 - mkSlaveInfile, [319](#)
 - numrbufs, [319](#)
 - numsbufs, [319](#)
 - numtasks, [318](#)
 - parallel, [318](#)

- rbufs, [318](#)
 - rhsInParallel, [318](#)
 - sbuf, [318](#)
 - slavebuf, [318](#)
- parameter
 - SETUPPARAMETERS, [385](#)
- parent
 - NaMeNode, [260](#)
 - tree, [479](#)
- ParenthesesTest
 - compiler.c, [724](#)
 - declare.h, [957](#)
- ParseNumber
 - declare.h, [802](#)
- ParseSign
 - declare.h, [803](#)
- ParseSignedNumber
 - declare.h, [803](#)
- PARTEST
 - ftypes.h, [1377](#)
- PARTI
 - structs.h, [2908](#)
- PaRtl, [320](#)
 - args, [320](#)
 - nargs, [320](#)
 - nfun, [320](#)
 - numargs, [320](#)
 - numpart, [321](#)
 - psize, [320](#)
 - where, [321](#)
- partial_factorize
 - optimize.cc, [2003](#)
- PaRtlcLe, [321](#)
 - mass, [322](#)
 - number, [321](#)
 - spin, [321](#)
 - type, [322](#)
- Particle, [322](#)
 - ~Particle, [323](#)
 - acode, [326](#)
 - aname, [325](#)
 - aparticle, [324](#)
 - cmaxdeg, [326](#)
 - cmindeg, [326](#)
 - extonly, [326](#)
 - id, [325](#)
 - isNeutral, [324](#)
 - mdl, [325](#)
 - name, [325](#)
 - neutral, [325](#)
 - Particle, [323](#)
 - particleCode, [324](#)
 - particleName, [324](#)
 - pcode, [325](#)
 - prParticle, [324](#)
 - pType, [325](#)
 - typeGName, [324](#)
 - typeName, [324](#)
- particle
 - PNodeClass, [336](#)
- particleCode
 - Model, [228](#)
 - Particle, [324](#)
- particleName
 - Model, [227](#)
 - Particle, [324](#)
- particles
 - Model, [229](#)
 - VerTex, [485](#)
- PARTITIONS
 - ftypes.h, [1403](#)
- partitions
 - T_const, [432](#)
- partodoflag
 - C_const, [60](#)
- PasteFile
 - declare.h, [858](#)
 - proces.c, [2432](#)
- PasteTerm
 - declare.h, [823](#)
 - proces.c, [2432](#)
- Patches
 - sOrT, [395](#)
- Path
 - M_const, [173](#)
- PathOutput
 - declare.h, [1030](#)
 - lus.c, [1690](#)
- PATHSEPARATOR
 - fwin.h, [1498](#)
 - unix.h, [3198](#)
- PATHVALUE
 - setfile.c, [2693](#)
- patstop
 - N_const, [246](#)
- pattern
 - BracketInfo, [35](#)
 - TaBIEs, [461](#)
- pattern.c, [2174](#), [2177](#)
 - FindAll, [2176](#)
 - PutInBuffers, [2175](#)
 - SubsInAll, [2176](#)
 - Substitute, [2176](#)
 - TakeIDfunction, [2177](#)
 - TestMatch, [2175](#)
 - TestSelect, [2176](#)
 - TransferBuffer, [2177](#)
- patternbuffer
 - N_const, [249](#)
- patternbuffersize
 - N_const, [257](#)
- PATTERNFUNCTION
 - ftypes.h, [1397](#)
- pBer
 - T_const, [431](#)
- pcode

- Particle, [325](#)
- PGInput, [332](#)
- PInput, [333](#)
- pdef
 - Model, [229](#)
- pdel
 - TrAcEs, [476](#)
- pepf
 - TrAcEs, [475](#)
- PERM
 - structs.h, [2907](#)
- perm
 - TrAcEn, [472](#)
 - TrAcEs, [474](#)
- permg
 - SGroup, [389](#)
- permMat
 - MGraph, [208](#)
- PERMP
 - structs.h, [2907](#)
- perms
 - SGroup, [389](#)
- PERMUTATIONS
 - ftypes.h, [1403](#)
- PeRmUtE, [326](#)
 - cycle, [327](#)
 - n, [327](#)
 - objects, [327](#)
 - sign, [327](#)
- Permute
 - declare.h, [858](#)
 - symmetr.c, [2935](#)
- PERMUTEARG
 - ftypes.h, [1447](#)
- PeRmUtEp, [327](#)
 - cycle, [328](#)
 - n, [328](#)
 - objects, [328](#)
 - sign, [328](#)
- PermuteP
 - declare.h, [859](#)
 - symmetr.c, [2936](#)
- PF
 - parallel.c, [2089](#)
 - parallel.h, [2170](#)
- PF_ANY_MSGTAG
 - parallel.h, [2143](#)
- PF_ANY_SOURCE
 - parallel.h, [2143](#)
- PF_Bcast
 - mpi.c, [1782](#)
 - parallel.h, [2144](#)
- PF_Broadcast
 - mpi.c, [1788](#)
 - parallel.h, [2149](#)
- PF_BroadcastBuffer
 - parallel.c, [2079](#)
 - parallel.h, [2160](#)
- PF_BroadcastCBuf
 - parallel.c, [2084](#)
 - parallel.h, [2164](#)
- PF_BroadcastExpFlags
 - parallel.c, [2084](#)
 - parallel.h, [2165](#)
- PF_BroadcastExpr
 - parallel.c, [2085](#)
 - parallel.h, [2166](#)
- PF_BroadcastModifiedDollars
 - parallel.c, [2082](#)
 - parallel.h, [2163](#)
- PF_BroadcastNumber
 - parallel.c, [2078](#)
 - parallel.h, [2159](#)
- PF_BroadcastPreDollar
 - parallel.c, [2080](#)
 - parallel.h, [2161](#)
- PF_BroadcastRedefinedPreVars
 - parallel.c, [2083](#)
 - parallel.h, [2164](#)
- PF_BroadcastRHS
 - parallel.c, [2085](#)
 - parallel.h, [2167](#)
- PF_BroadcastString
 - parallel.c, [2079](#)
 - parallel.h, [2160](#)
- PF_BUFFER, [328](#)
 - active, [330](#)
 - buff, [329](#)
 - fill, [329](#)
 - from, [330](#)
 - full, [329](#)
 - index, [330](#)
 - numbufs, [330](#)
 - request, [330](#)
 - retstat, [330](#)
 - status, [329](#)
 - stop, [329](#)
 - tag, [330](#)
 - type, [330](#)
- PF_BUFFER_MSGTAG
 - parallel.h, [2141](#)
- PF_BYTE
 - parallel.h, [2143](#)
- PF_CollectModifiedDollars
 - parallel.c, [2081](#)
 - parallel.h, [2162](#)
- PF_COMM
 - parallel.h, [2143](#)
- PF_DATA_MSGTAG
 - parallel.h, [2141](#)
- PF_Deferred
 - parallel.c, [2074](#)
 - parallel.h, [2155](#)
- PF_DOLLAR_MSGTAG
 - parallel.h, [2141](#)
- PF_EMPTY_MSGTAG

- parallel.h, [2142](#)
- PF_ENDBUFFER_MSGTAG
 - parallel.h, [2141](#)
- PF_EndSort
 - parallel.c, [2073](#)
 - parallel.h, [2154](#)
- PF_ENDSORT_MSGTAG
 - parallel.h, [2141](#)
- PF_FlushStdOutBuffer
 - parallel.c, [2089](#)
 - parallel.h, [2170](#)
- PF_FreeErrorMessageBuffers
 - parallel.c, [2089](#)
- PF_GetSlaveTimes
 - parallel.c, [2078](#)
 - parallel.h, [2159](#)
- PF_Init
 - parallel.c, [2076](#)
 - parallel.h, [2157](#)
- PF_InParallelProcessor
 - parallel.c, [2086](#)
 - parallel.h, [2167](#)
- PF_INT
 - parallel.h, [2143](#)
- PF_IRecvRbuf
 - mpi.c, [1782](#)
 - parallel.c, [2071](#)
- PF_ISendSbuf
 - mpi.c, [1781](#)
 - parallel.h, [2144](#)
- PF_LibInit
 - mpi.c, [1779](#)
 - parallel.c, [2069](#)
- PF_LibTerminate
 - mpi.c, [1780](#)
 - parallel.c, [2070](#)
- PF_LOG_MSGTAG
 - parallel.h, [2142](#)
- PF_LongMultiBroadcast
 - mpi.c, [1793](#)
 - parallel.h, [2154](#)
- PF_LongMultiPack
 - parallel.h, [2144](#)
- PF_LongMultiPackImpl
 - mpi.c, [1791](#)
 - parallel.h, [2152](#)
- PF_LongMultiUnpack
 - parallel.h, [2144](#)
- PF_LongMultiUnpackImpl
 - mpi.c, [1792](#)
 - parallel.h, [2153](#)
- PF_LongSinglePack
 - mpi.c, [1789](#)
 - parallel.h, [2150](#)
- PF_LongSingleReceive
 - mpi.c, [1790](#)
 - parallel.h, [2151](#)
- PF_LongSingleSend
 - mpi.c, [1790](#)
 - parallel.h, [2151](#)
- PF_LongSingleUnpack
 - mpi.c, [1789](#)
 - parallel.h, [2150](#)
- PF_maxDollarChunkSize
 - mpi.c, [1793](#)
 - parallel.h, [2170](#)
- PF_MISC_MSGTAG
 - parallel.h, [2142](#)
- PF_MLock
 - parallel.c, [2088](#)
 - parallel.h, [2169](#)
- PF_MUnlock
 - parallel.c, [2088](#)
 - parallel.h, [2169](#)
- PF_OPT_COLLECT_MSGTAG
 - parallel.h, [2142](#)
- PF_OPT_HORNER_MSGTAG
 - parallel.h, [2142](#)
- PF_OPT_MCTS_MSGTAG
 - parallel.h, [2142](#)
- PF_Pack
 - mpi.c, [1785](#)
 - parallel.h, [2146](#)
- PF_PACKSIZE
 - mpi.c, [1779](#)
- PF_PackString
 - mpi.c, [1786](#)
 - parallel.h, [2147](#)
- PF_PrepareLongMultiPack
 - mpi.c, [1791](#)
 - parallel.h, [2152](#)
- PF_PrepareLongSinglePack
 - mpi.c, [1788](#)
 - parallel.h, [2149](#)
- PF_PreparePack
 - mpi.c, [1785](#)
 - parallel.h, [2146](#)
- PF_PrintPackBuf
 - mpi.c, [1784](#)
- PF_Probe
 - mpi.c, [1780](#)
 - parallel.c, [2070](#)
- PF_Processor
 - parallel.c, [2075](#)
 - parallel.h, [2156](#)
- PF_RawProbe
 - mpi.c, [1784](#)
 - parallel.c, [2073](#)
- PF_RawRecv
 - mpi.c, [1783](#)
 - parallel.c, [2072](#)
 - parallel.h, [2145](#)
- PF_RawSend
 - mpi.c, [1783](#)
 - parallel.c, [2072](#)
 - parallel.h, [2145](#)

PF_READY_MSGTAG
 parallel.h, 2141
 PF_RealTime
 mpi.c, 1779
 parallel.c, 2069
 PF_Receive
 mpi.c, 1787
 parallel.h, 2149
 PF_RecvFile
 parallel.c, 2087
 parallel.h, 2168
 PF_RecvWbuf
 mpi.c, 1781
 parallel.c, 2070
 PF_RESET
 parallel.h, 2140
 PF_RestoreInsideInfo
 parallel.c, 2084
 parallel.h, 2166
 PF_Send
 mpi.c, 1787
 parallel.h, 2148
 PF_SendFile
 parallel.c, 2086
 parallel.h, 2167
 PF_SNDFILEBUFSIZE
 parallel.c, 2068
 PF_STATS_SIZE
 parallel.c, 2068
 PF_STDOUT_MSGTAG
 parallel.h, 2142
 PF_StoreInsideInfo
 parallel.c, 2083
 parallel.h, 2166
 PF_TERM_MSGTAG
 parallel.h, 2141
 PF_Terminate
 parallel.c, 2077
 parallel.h, 2158
 PF_TIME
 parallel.h, 2140
 PF_Unpack
 mpi.c, 1785
 parallel.h, 2147
 PF_UnpackString
 mpi.c, 1786
 parallel.h, 2148
 PF_WaitRbuf
 mpi.c, 1782
 parallel.c, 2071
 PF_WriteFileToFile
 parallel.c, 2088
 parallel.h, 2169
 pfac
 T_const, 431
 PFORTTRANMODE
 ftypes.h, 1387
 PGInput, 331
 cdeg, 331
 cnum, 331
 cple, 331
 ctyp, 331
 pcode, 332
 pname, 332
 pgq
 SGroup, 389
 pgr
 SGroup, 389
 PHEAD
 structs.h, 2903
 PHEAD0
 structs.h, 2903
 PHI
 ftypes.h, 1405
 pld
 EGraph, 111
 MGraph, 211
 T_EGraph, 443
 T_MGraph, 447
 PInput, 332
 acode, 333
 aname, 332
 extonly, 333
 name, 332
 pcode, 333
 ptypc, 333
 ptypen, 333
 PIPESTREAM
 ftypes.h, 1368
 PISYMBOL
 ftypes.h, 1405
 plist
 ECand, 95
 EStack, 119
 Interaction, 161
 plistc
 lInput, 150
 plistn
 lInput, 149
 pname
 PGInput, 332
 Stream, 418
 pnclass
 Assign, 24
 SProcess, 406
 PNodeClass, 334
 ~PNodeClass, 335
 cl2mcl, 337
 cl2nd, 337
 cmaxdeg, 336
 cmindeg, 336
 count, 336
 couple, 336
 deg, 336
 nclass, 337
 nd2cl, 337

- nnodes, 337
- particle, 336
- PNodeClass, 335
- prElem, 335
- prPNodeClass, 335
- sproc, 336
- type, 336
- PObuffer
 - FiLe, 129
- POfill
 - FiLe, 129
- POfull
 - FiLe, 129
- poin2a
 - sOrT, 396
- poina
 - sOrT, 396
- PoinFill
 - sOrT, 395
- PoinScratch
 - N_const, 249
- Pointer
 - CbUf, 71
- pointer
 - StreaM, 417
- poly, 339
 - ~poly, 341
 - add, 349
 - all_variables, 345
 - argument_to_poly, 347
 - check_memory, 343
 - coefficient, 346
 - coefficient_list_divmod, 348
 - coefficients_modulo, 346
 - compare_degree_vector, 349
 - degree, 345
 - degree_vector, 349
 - derivative, 346
 - div, 350
 - divides, 351
 - divmod, 351
 - divmod_heap, 354
 - divmod_one_term, 353
 - divmod_univar, 353
 - expand_memory, 343
 - first_variable, 345
 - flint::poly, 338
 - from_coefficient_list, 348
 - get_variables, 347
 - integer_lcoeff, 345
 - is_dense_univariate, 344
 - is_integer, 344
 - is_monomial, 344
 - is_one, 344
 - is_zero, 344
 - last_monomial, 349
 - last_monomial_index, 349
 - lcoeff_multivar, 346
 - lcoeff_univar, 346
 - mod, 351
 - modn, 356
 - modp, 356
 - monomial_compare, 349
 - mul, 350
 - mul_heap, 352
 - mul_one_term, 352
 - mul_univar, 352
 - normalize, 348
 - number_of_terms, 345
 - operator!=, 343
 - operator+, 342
 - operator+=", 341
 - operator-, 342
 - operator-=, 341
 - operator/, 342
 - operator/=", 341
 - operator=, 343
 - operator==, 342
 - operator%, 342
 - operator%=", 342
 - operator[], 343
 - operator*, 342
 - operator*=", 341
 - poly, 340, 341
 - poly_to_argument, 347
 - poly_to_argument_with_den, 347
 - pop_heap, 355
 - push_heap, 355
 - setmod, 346
 - sign, 345
 - simple_poly, 346, 347
 - size_of_form_notation, 348
 - size_of_form_notation_with_den, 348
 - size_of_terms, 356
 - sub, 350
 - terms, 356
 - termscopy, 343
 - to_coefficient_list, 348
 - to_string, 349
 - total_degree, 345
- poly.cc, 2204
- poly.h, 2237
- poly_determine_modulus
 - polywrap.cc, 2279
- poly_div
 - declare.h, 1012
 - polywrap.cc, 2280
- poly_divmod
 - polywrap.cc, 2280
- poly_factor
 - flint::poly_factor, 357
- poly_factorize
 - polywrap.cc, 2284
- poly_factorize_argument
 - declare.h, 1015
 - polywrap.cc, 2285

- poly_factorize_dollar
 - declare.h, [1016](#)
 - polywrap.cc, [2286](#)
- poly_factorize_expression
 - declare.h, [1016](#)
 - polywrap.cc, [2286](#)
- poly_fix_minus_signs
 - polywrap.cc, [2284](#)
- poly_free_poly_vars
 - declare.h, [1018](#)
 - polywrap.cc, [2289](#)
- poly_gcd
 - declare.h, [1011](#)
 - polywrap.cc, [2279](#)
- poly_inverse
 - declare.h, [1013](#)
 - polywrap.cc, [2288](#)
- poly_mul
 - declare.h, [1013](#)
 - polywrap.cc, [2289](#)
- poly_num_vars
 - N_const, [258](#)
- poly_ratfun_add
 - declare.h, [1013](#)
 - polywrap.cc, [2282](#)
- poly_ratfun_normalize
 - declare.h, [1014](#)
 - polywrap.cc, [2283](#)
- poly_ratfun_read
 - polywrap.cc, [2280](#)
- poly_rem
 - declare.h, [1012](#)
 - polywrap.cc, [2280](#)
- poly_sort
 - polywrap.cc, [2281](#)
- poly_to_argument
 - poly, [347](#)
- poly_to_argument_with_den
 - poly, [347](#)
- poly_unfactorize_expression
 - declare.h, [1017](#)
 - polywrap.cc, [2287](#)
- poly_vars
 - N_const, [251](#)
- poly_vars_type
 - N_const, [258](#)
- PolyAct
 - T_const, [438](#)
- POLYADD
 - ftypes.h, [1442](#)
- POLYDIV
 - ftypes.h, [1443](#)
- PolyDiv
 - declare.h, [1008](#)
 - ratio.c, [2509](#)
- polyfact.cc, [2240](#), [2242](#)
 - operator<=, [2241](#)
- polyfact.h, [2257](#)
- PolyFlag
 - sOrT, [399](#)
- POLYFUN
 - ftypes.h, [1370](#)
- PolyFun
 - R_const, [375](#)
- PolyFunClean
 - declare.h, [852](#)
 - function.c, [1472](#)
- PolyFunDirty
 - declare.h, [852](#)
 - function.c, [1472](#)
- PolyFunExp
 - R_const, [376](#)
- PolyFunInv
 - R_const, [375](#)
- PolyFunMul
 - declare.h, [859](#)
 - proces.c, [2438](#)
- PolyFunPow
 - R_const, [376](#)
- PolyFunTodo
 - N_const, [257](#)
- PolyFunType
 - R_const, [375](#)
- PolyFunVar
 - R_const, [376](#)
- POLYGCD
 - ftypes.h, [1443](#)
- polygcd.cc, [2259](#), [2260](#)
 - gcd_heuristic_possible, [2259](#)
 - gcd_linear_helper, [2260](#)
- polygcd.h, [2276](#)
- POLYINTFAC
 - ftypes.h, [1443](#)
- POLYMOD, [357](#)
 - arraysize, [358](#)
 - coefs, [358](#)
 - modnum, [358](#)
 - numsym, [358](#)
 - polysize, [358](#)
- POLYMUL
 - ftypes.h, [1442](#)
- POLYNORM
 - ftypes.h, [1443](#)
- PolyNormFlag
 - N_const, [257](#)
- PolyRatFunChanged
 - C_const, [69](#)
- PolyRatFunSpecial
 - declare.h, [972](#)
 - sort.c, [2733](#)
- POLYREM
 - ftypes.h, [1443](#)
- polysize
 - POLYMOD, [358](#)
- polysortflag
 - N_const, [255](#)

- POLYSUB
 - ftypes.h, 1442
- PolyWise
 - sOrT, 399
- polywrap.cc, 2278, 2289
 - poly_determine_modulus, 2279
 - poly_div, 2280
 - poly_divmod, 2280
 - poly_factorize, 2284
 - poly_factorize_argument, 2285
 - poly_factorize_dollar, 2286
 - poly_factorize_expression, 2286
 - poly_fix_minus_signs, 2284
 - poly_free_poly_vars, 2289
 - poly_gcd, 2279
 - poly_inverse, 2288
 - poly_mul, 2289
 - poly_ratfun_add, 2282
 - poly_ratfun_normalize, 2283
 - poly_ratfun_read, 2280
 - poly_rem, 2280
 - poly_sort, 2281
 - poly_unfactorize_expression, 2287
- POLYWRAP_DENOMPOWER_INCREASE_FACTOR, 2289
- POLYWRAP_DENOMPOWER_INCREASE_FACTOR
 - polywrap.cc, 2289
- pop_heap
 - poly, 355
- POposition
 - File, 129
- POPPREASSIGNLEVEL
 - declare.h, 818
- PopPreVars
 - declare.h, 904
 - pre.c, 2322
- PopVariables
 - declare.h, 860
 - execute.c, 1156
- portsignals.h, 2311, 2315
 - FATAL_SIG_ERROR, 2312
 - NSIG, 2312
 - SIGALRM, 2314
 - SIGEMT, 2313
 - SIGFPE, 2312
 - SIGHUP, 2314
 - SIGILL, 2313
 - SIGINT, 2314
 - SIGLOST, 2313
 - SIGPIPE, 2313
 - SIGPROF, 2314
 - SIGQUIT, 2314
 - SIGSEGV, 2312
 - SIGSYS, 2313
 - SIGTERM, 2313
 - SIGVTALRM, 2314
 - SIGXCPU, 2313
 - SIGXFSZ, 2313
- POS0_
 - ftypes.h, 1428
- POS_
 - ftypes.h, 1427
- PoSiTiOn, 358
 - p1, 359
- Position
 - FiLeDaTa, 132
- position
 - indexblock, 152
 - InDeXeNtRy, 154
 - nameblock, 259
 - objects, 287
 - SpecTatoR, 401
 - StOrEcAcHe, 411
- PositionStream
 - declare.h, 887
 - tools.c, 3084
- POSITIVEONLY
 - ftypes.h, 1445
- POsize
 - File, 130
- POSNEG
 - ftypes.h, 1444
- POstop
 - File, 129
- posWorkPointer
 - T_const, 433
- posWorkSize
 - R_const, 373
- posWorkSpace
 - T_const, 429
- PotModdollars
 - variable.h, 3210
- PotModDolList
 - C_const, 44
- pow
 - COST, 76
- power
 - factorized_poly, 126
- powerFlag
 - O_const, 284
- powmod
 - C_const, 47
- pParseObject
 - declare.h, 917
 - pre.c, 2332
- prCand
 - Assign, 17
- pre.c, 2316, 2338
 - AddToPreTypes, 2332
 - CharOut, 2321
 - ClearMacro, 2324
 - ClearPushback, 2320
 - ConstructName, 2336
 - DAEMON, 2319
 - defineChannel, 2334
 - DoBreakDo, 2326

- DoCall, [2325](#)
- DoClearOptimize, [2335](#)
- DoClearUserFlag, [2337](#)
- DoCommentChar, [2324](#)
- DoContinueDo, [2326](#)
- DoDebug, [2326](#)
- DoDefine, [2324](#)
- DoDo, [2326](#)
- DoElse, [2326](#)
- DoElseif, [2326](#)
- DoEnddo, [2327](#)
- DoEndif, [2327](#)
- DoEndInside, [2328](#)
- DoEndNamespace, [2336](#)
- DoEndprocedure, [2327](#)
- DoExternal, [2333](#)
- DoFactDollar, [2334](#)
- DoFromExternal, [2334](#)
- Dof, [2327](#)
- Dofdef, [2327](#)
- Dofndef, [2327](#)
- Dofydef, [2327](#)
- DoInclude, [2325](#)
- DoInside, [2328](#)
- DoMessage, [2328](#)
- DoNamespace, [2336](#)
- DoOptimize, [2335](#)
- DoPipe, [2328](#)
- DoPrcExtension, [2328](#)
- DoPreAddSeparator, [2333](#)
- DoPreAppend, [2329](#)
- DoPreAppendPath, [2335](#)
- DoPreAssign, [2324](#)
- DoPreBreak, [2330](#)
- DoPreCase, [2330](#)
- DoPreClose, [2329](#)
- DoPreCreate, [2329](#)
- DoPreDefault, [2330](#)
- DoPreEndSwitch, [2330](#)
- DoPreExchange, [2325](#)
- DoPreOut, [2328](#)
- DoPrePrependPath, [2336](#)
- DoPrePrintTimes, [2328](#)
- DoPreRemove, [2329](#)
- DoPreReset, [2335](#)
- DoPreRmSeparator, [2333](#)
- DoPreShow, [2330](#)
- DoPreSortReallocate, [2329](#)
- DoPreSwitch, [2330](#)
- DoPreWrite, [2329](#)
- DoProcedure, [2329](#)
- DoPrompt, [2333](#)
- DoRedefine, [2324](#)
- DoReverseInclude, [2325](#)
- DoRmExternal, [2334](#)
- DoSetExternal, [2333](#)
- DoSetExternalAttr, [2333](#)
- DoSetRandom, [2335](#)
- DoSetUserFlag, [2337](#)
- DoSkipExtraSymbols, [2335](#)
- DoSystem, [2330](#)
- DoTerminate, [2326](#)
- DoTimeOutAfter, [2336](#)
- DoToExternal, [2334](#)
- DoUndefine, [2325](#)
- DoUse, [2337](#)
- EvalPrelf, [2331](#)
- ExpandTripleDots, [2323](#)
- FALSE_EXPR, [2320](#)
- FindInKeyWord, [2323](#)
- FindKeyWord, [2323](#)
- GetChar, [2320](#)
- GetDollarNumber, [2335](#)
- GetInput, [2320](#)
- GetPreVar, [2321](#)
- Include, [2325](#)
- IniModule, [2322](#)
- IniSpecialModule, [2322](#)
- KILL, [2319](#)
- KILLALL, [2319](#)
- LoadInstruction, [2322](#)
- LoadStatement, [2322](#)
- MessPreNesting, [2333](#)
- NOSHELL, [2320](#)
- PopPreVars, [2322](#)
- pParseObject, [2332](#)
- PreCalc, [2332](#)
- PreCmp, [2331](#)
- PreEq, [2332](#)
- PreEval, [2332](#)
- PrelfEval, [2331](#)
- PreLoad, [2331](#)
- PreProcessor, [2322](#)
- PreProInstruction, [2322](#)
- PreSkip, [2331](#)
- PutPreVar, [2321](#)
- SHELL, [2319](#)
- SKIPBUFSIZE, [2319](#)
- SkipName, [2336](#)
- StartPrepro, [2331](#)
- STDERR, [2319](#)
- STRINGIFY, [2319](#)
- STRINGIFY_, [2319](#)
- TERMINAL, [2320](#)
- TheDefine, [2323](#)
- TheUndefine, [2325](#)
- TRUE_EXPR, [2320](#)
- UnsetAllowDelay, [2321](#)
- UserFlags, [2337](#)
- writeToChannel, [2334](#)
- PreAssignFlag
 - P_const, [313](#)
- PreAssignLevel
 - P_const, [316](#)
- PreAssignStack
 - P_const, [312](#)

- PreCalc
 - declare.h, [916](#)
 - pre.c, [2332](#)
- PRECALCSTREAM
 - ftypes.h, [1368](#)
- prECand
 - ECand, [95](#)
- PreCmp
 - declare.h, [916](#)
 - pre.c, [2331](#)
- PreContinuation
 - P_const, [313](#)
- PreDebug
 - P_const, [315](#)
- PreEq
 - declare.h, [916](#)
 - pre.c, [2332](#)
- preError
 - P_const, [316](#)
- PreEval
 - declare.h, [916](#)
 - pre.c, [2332](#)
- preFill
 - P_const, [312](#)
- PrelfDollarEval
 - declare.h, [977](#)
 - dollar.c, [1096](#)
- PrelfEval
 - declare.h, [917](#)
 - pre.c, [2331](#)
- PrelfLevel
 - DoLoOp, [92](#)
 - P_const, [314](#)
- PrelfStack
 - P_const, [312](#)
- PreInsideLevel
 - P_const, [315](#)
- prElem
 - PNodeClass, [335](#)
- PRELOAD, [359](#)
 - buffer, [359](#)
 - size, [359](#)
- PreLoad
 - declare.h, [917](#)
 - pre.c, [2331](#)
- PRELOWERAFTER
 - ftypes.h, [1372](#)
- PRENOACTION
 - ftypes.h, [1372](#)
- PreOut
 - P_const, [314](#)
- PrepPoly
 - declare.h, [860](#)
 - proces.c, [2437](#)
- PreProcessor
 - declare.h, [898](#)
 - pre.c, [2322](#)
- PreproFlag
 - P_const, [313](#)
- PreProInstruction
 - declare.h, [904](#)
 - pre.c, [2322](#)
- PREPROONLY
 - ftypes.h, [1373](#)
- PRERAISEAFTER
 - ftypes.h, [1372](#)
- PreRandom
 - declare.h, [992](#)
 - reken.c, [2566](#)
- PREREADSTREAM
 - ftypes.h, [1368](#)
- PREREADSTREAM2
 - ftypes.h, [1368](#)
- PREREADSTREAM3
 - ftypes.h, [1368](#)
- PreSkip
 - declare.h, [917](#)
 - pre.c, [2331](#)
- preStart
 - P_const, [312](#)
- preStop
 - P_const, [312](#)
- PreSwitchLevel
 - DoLoOp, [93](#)
 - P_const, [314](#)
- PreSwitchModes
 - P_const, [313](#)
- PreSwitchStrings
 - P_const, [312](#)
- PRETYPEDO
 - ftypes.h, [1376](#)
- PRETYPEIF
 - ftypes.h, [1375](#)
- PRETYPEINSIDE
 - ftypes.h, [1376](#)
- PRETYPENONE
 - ftypes.h, [1375](#)
- PRETYPEPROCEDURE
 - ftypes.h, [1376](#)
- PreTypes
 - P_const, [313](#)
- PRETYPESWITCH
 - ftypes.h, [1376](#)
- PREV
 - declare.h, [810](#)
- PREVAR
 - structs.h, [2906](#)
- PreVar
 - variable.h, [3205](#)
- pReVaR, [360](#)
 - argnames, [360](#)
 - name, [360](#)
 - nargs, [360](#)
 - value, [360](#)
 - wildarg, [361](#)
- PreVarList

- P_const, 311
- prevars
 - StreaM, 419
- PREVARSTREAM
 - ftypes.h, 1368
- previous
 - NaMeSpAcE, 262
 - StreaM, 418
- previousblock
 - indexblock, 152
 - nameblock, 259
- previousEfactor
 - T_const, 431
- previousNoShowInput
 - StreaM, 419
- prevline
 - StreaM, 417
- prFLines
 - EGraph, 107
- primelist
 - T_const, 431
- PRIMENUMBER
 - ftypes.h, 1402
- print
 - EEdge, 97
 - EFLine, 101
 - EGraph, 104
 - ENode, 116
 - EStack, 119
 - flint::fmpz, 139
 - flint::mpoly, 240
 - flint::poly, 338
 - Fraction, 140
 - MConn, 199
 - MGraph, 207
 - MOrbits, 238
 - NStack, 275
 - Options, 298
 - SGroup, 386
 - T_EGraph, 442
- print_instr
 - optimize.cc, 1992
- print_tree
 - optimize.cc, 1998
- printAdjMat
 - MGraph, 207
- PRINTALL
 - ftypes.h, 1426
- PrintBacktraceFlag
 - C_const, 62
- PRINTCONTENT
 - ftypes.h, 1426
- PRINTCONTENTS
 - ftypes.h, 1426
- PrintDeprecation
 - declare.h, 863
 - startup.c, 2804
- prInteraction
 - Interaction, 161
- PrintExtraSymbol
 - declare.h, 1005
 - notation.c, 1943
- PRINTFBUF
 - parallel.c, 2066
- printflag
 - ExPrEsSiOn, 123
- PrintFloat
 - float.c, 1292
- printLevel
 - Options, 299
- PRINTLFILE
 - ftypes.h, 1426
- printMat
 - MNodeClass, 221
 - T_MNodeClass, 454
- printModel
 - Options, 299
- PRINTOFF
 - ftypes.h, 1426
- PRINTON
 - ftypes.h, 1426
- PRINTONEFUNCTION
 - ftypes.h, 1426
- PRINTONETERM
 - ftypes.h, 1426
- PrintRunningTime
 - declare.h, 863
 - startup.c, 2805
- printscratch
 - proces.c, 2439
- PrintSeq
 - declare.h, 921
 - message.c, 1717
- printstats
 - OPTIMIZE, 294
- PrintSubTerm
 - declare.h, 921
 - message.c, 1717
- PrintSubtermList
 - declare.h, 1005
 - notation.c, 1943
- PrintTerm
 - declare.h, 921
 - message.c, 1716
- PrintTermC
 - declare.h, 921
 - message.c, 1717
- PrintTime
 - declare.h, 979
 - sch.c, 2657
- PrintTotalSize
 - M_const, 183
- PrintType
 - O_const, 285
- PrintWords
 - declare.h, 921

- message.c, [1717](#)
- prModel
 - Model, [226](#)
- prNCand
 - NCand, [267](#)
- proc
 - Assign, [23](#)
 - EGraph, [108](#)
 - Options, [301](#)
 - Output, [307](#)
 - SProcess, [406](#)
- PROCEDURE, [361](#)
 - loadmode, [362](#)
 - name, [362](#)
 - p, [362](#)
- procedureExtension
 - P_const, [312](#)
- procedureextension
 - setfile.c, [2697](#)
- PROCEDUREFILE
 - ftypes.h, [1369](#)
- Procedures
 - variable.h, [3204](#)
- proces.c, [2427](#), [2439](#)
 - Deferred, [2436](#)
 - DONE, [2428](#)
 - DoOnePow, [2435](#)
 - FiniTerm, [2433](#)
 - Generator, [2433](#)
 - InFunction, [2429](#)
 - InsertTerm, [2430](#)
 - PasteFile, [2432](#)
 - PasteTerm, [2432](#)
 - PolyFunMul, [2438](#)
 - PrepPoly, [2437](#)
 - printscratch, [2439](#)
 - Processor, [2428](#)
 - TestSub, [2429](#)
- Process, [362](#)
 - ~Process, [364](#)
 - agrcount, [365](#)
 - astack, [366](#)
 - clist, [366](#)
 - ctotal, [365](#)
 - finalPart, [365](#)
 - id, [365](#)
 - initlPart, [365](#)
 - loop, [366](#)
 - maxnlegs, [366](#)
 - mgrcount, [365](#)
 - mkSProcess, [364](#)
 - model, [364](#)
 - nAGraphs, [368](#)
 - nAOPI, [368](#)
 - nExtern, [366](#)
 - nfinal, [365](#)
 - ngraphs, [367](#)
 - ninitl, [365](#)
 - nMGraphs, [367](#)
 - nMOPI, [367](#)
 - nopi, [367](#)
 - nSubproc, [366](#)
 - opt, [364](#)
 - Process, [364](#)
 - prProcess, [364](#)
 - sec, [368](#)
 - sproc, [366](#)
 - sptbl, [366](#)
 - wAGraphs, [368](#)
 - wAOPI, [368](#)
 - wgraphs, [367](#)
 - wMGraphs, [367](#)
 - wMOPI, [367](#)
 - wopi, [367](#)
- ProcessBucketSize
 - C_const, [53](#)
- ProcessOption
 - declare.h, [888](#)
 - setfile.c, [2694](#)
- Processor
 - declare.h, [861](#)
 - proces.c, [2428](#)
- ProcessStats
 - C_const, [59](#)
- proclid
 - Output, [308](#)
- ProcList
 - P_const, [311](#)
- Product
 - declare.h, [862](#)
 - reken.c, [2561](#)
- PRODUCTIONDATE
 - form3.h, [1325](#)
- proexp
 - T_const, [437](#)
- PROPERORDERFLAG
 - ftypes.h, [1442](#)
- properorderflag
 - C_const, [60](#)
- ProtoType
 - C_const, [48](#)
- prototype
 - ExPrEsSiOn, [121](#)
 - TaBIEs, [460](#)
- prototypeSize
 - TaBIEs, [463](#)
- prParticle
 - Particle, [324](#)
- prParticleArray
 - Model, [228](#)
- prPNodeClass
 - PNodeClass, [335](#)
- prProcess
 - Process, [364](#)
- prSProcess
 - SProcess, [404](#)

- prStack
 - AStack, [27](#)
- PrtLong
 - declare.h, [862](#)
 - reken.c, [2562](#)
- PrtTerms
 - declare.h, [862](#)
 - sch.c, [2657](#)
- PruneExtraSymbols
 - declare.h, [1007](#)
 - notation.c, [1943](#)
- pSize
 - P_const, [313](#)
- psize
 - PaRtl, [320](#)
- psq
 - SGroup, [389](#)
- psr
 - SGroup, [389](#)
- pStop
 - sOrT, [396](#)
- ptcl
 - AEdge, [8](#)
 - EEdge, [98](#)
 - NCInput, [268](#)
- ptype
 - Particle, [325](#)
- ptypespec
 - PInput, [333](#)
- ptypen
 - PInput, [333](#)
- push_heap
 - poly, [355](#)
- pushEdge
 - AStack, [28](#)
 - MConn, [198](#)
- pushNode
 - AStack, [28](#)
 - MConn, [198](#)
- PUSHPREASSIGNLEVEL
 - declare.h, [818](#)
- PutArgInScratch
 - declare.h, [1001](#)
 - transform.c, [3154](#)
- PutBracket
 - declare.h, [863](#)
 - execute.c, [1157](#)
- PutBracketInIndex
 - declare.h, [979](#)
 - index.c, [1673](#)
- PutExtraSymbols
 - declare.h, [1009](#)
 - ratio.c, [2507](#)
- PUTFIRST
 - ftypes.h, [1403](#)
- PutIn
 - declare.h, [863](#)
 - sort.c, [2722](#)
- PutInBuffers
 - pattern.c, [2175](#)
- PutInside
 - declare.h, [857](#)
 - index.c, [1674](#)
- putinsize
 - sOrT, [398](#)
- PutInSpectator
 - declare.h, [1023](#)
 - spectator.c, [2791](#)
- PutInStore
 - declare.h, [864](#)
 - store.c, [2834](#)
- PutInVflags
 - declare.h, [876](#)
 - execute.c, [1156](#)
- PutOut
 - declare.h, [864](#)
 - sort.c, [2723](#)
- PutPreVar
 - declare.h, [885](#)
 - pre.c, [2321](#)
- PutTableNames
 - declare.h, [986](#)
 - minos.c, [1734](#)
- PutTermInDollar
 - declare.h, [962](#)
 - dollar.c, [1093](#)
- PUTZERO
 - declare.h, [812](#)
- PVFUNV
 - ftypes.h, [1458](#)
- PVFUNWP
 - ftypes.h, [1458](#)
- pWorkPointer
 - T_const, [433](#)
- pWorkSize
 - R_const, [373](#)
- pWorkSpace
 - T_const, [429](#)
- Q_
 - ftypes.h, [1429](#)
- qError
 - M_const, [188](#)
- QUADRUPLEFORTRANMODE
 - ftypes.h, [1388](#)
- QUADRUPLEPRECISIONFLAG
 - ftypes.h, [1388](#)
- Quotient
 - declare.h, [865](#)
 - reken.c, [2561](#)
- R
 - AllGlobals, [10](#)
- r
 - node, [273](#)
- R_const, [369](#)
 - BracketOn, [374](#)

- Cnumlhs, [373](#)
- CompareRoutine, [372](#)
- CompressBuffer, [371](#)
- CompressPointer, [372](#)
- ComprTop, [372](#)
- CurDum, [374](#)
- CurExpr, [377](#)
- DeferFlag, [375](#)
- DefPosition, [371](#)
- Eside, [376](#)
- expchanged, [376](#)
- expflags, [376](#)
- FoStage4, [371](#)
- Fscr, [371](#)
- funoffset, [377](#)
- GetFile, [374](#)
- GetOneFile, [375](#)
- gzipCompress, [373](#)
- hidefile, [371](#)
- infile, [371](#)
- InHiBuf, [372](#)
- InInBuf, [372](#)
- KeptInHold, [374](#)
- level, [376](#)
- IWorkSize, [373](#)
- MaxBracket, [374](#)
- MaxDum, [376](#)
- modeloptions, [377](#)
- moebiustable, [372](#)
- moebiustablesizesize, [377](#)
- NoCompress, [373](#)
- OldTime, [372](#)
- outfile, [371](#)
- outtohide, [373](#)
- PolyFun, [375](#)
- PolyFunExp, [376](#)
- PolyFunInv, [375](#)
- PolyFunPow, [376](#)
- PolyFunType, [375](#)
- PolyFunVar, [376](#)
- posWorkSize, [373](#)
- pWorkSize, [373](#)
- ShortSortCount, [377](#)
- sLevel, [375](#)
- SortType, [377](#)
- Stage4Name, [375](#)
- StoreData, [371](#)
- TePos, [375](#)
- wranfcall, [374](#)
- wranfia, [372](#)
- wranfnpair1, [374](#)
- wranfnpair2, [374](#)
- wranfseed, [373](#)
- R_COPY_B
 - checkpoint.c, [539](#)
- R_COPY_LIST
 - checkpoint.c, [540](#)
- R_COPY_NAMETREE
 - checkpoint.c, [541](#)
- R_COPY_S
 - checkpoint.c, [540](#)
- R_FREE
 - checkpoint.c, [538](#)
- R_FREE_NAMETREE
 - checkpoint.c, [539](#)
- R_FREE_STREAM
 - checkpoint.c, [539](#)
- R_SET
 - checkpoint.c, [539](#)
- RaisPow
 - declare.h, [865](#)
 - reken.c, [2559](#)
- RaisPowCached
 - declare.h, [865](#)
 - reken.c, [2559](#)
- RaisPowMod
 - declare.h, [866](#)
 - reken.c, [2560](#)
- RANDOMFUNCTION
 - ftypes.h, [1402](#)
- ranges
 - DICTIONARY, [83](#)
- RANPERM
 - ftypes.h, [1402](#)
- ratfun_add_mpoly
 - flintinterface.h, [1273](#)
- ratfun_add_poly
 - flintinterface.h, [1273](#)
- ratfun_normalize_mpoly
 - flintinterface.h, [1273](#)
- ratfun_normalize_poly
 - flintinterface.h, [1274](#)
- ratfun_read_mpoly
 - flintinterface.h, [1274](#)
- ratfun_read_poly
 - flintinterface.h, [1274](#)
- RATIO
 - ftypes.h, [1417](#)
- ratio
 - Fraction, [142](#)
- ratio.c, [2504](#), [2511](#)
 - BinomGen, [2505](#)
 - ConvertArgument, [2510](#)
 - CreateExpression, [2508](#)
 - DIVfunction, [2510](#)
 - divrem, [2511](#)
 - DoSumF1, [2506](#)
 - DoSumF2, [2506](#)
 - ExpandRat, [2510](#)
 - FromPolyRatFun, [2509](#)
 - GCDclean, [2509](#)
 - GCDfunction, [2506](#)
 - GCDfunction3, [2506](#)
 - GCDterms, [2508](#)
 - Glue, [2506](#)
 - InvPoly, [2510](#)

- MergeDotproductLists, [2508](#)
- MergeSymbolLists, [2508](#)
- MULfunc, [2510](#)
- MultiplyWithTerm, [2507](#)
- PolyDiv, [2509](#)
- PutExtraSymbols, [2507](#)
- RatioFind, [2505](#)
- RatioGen, [2505](#)
- ReadPolyRatFun, [2508](#)
- TakeContent, [2507](#)
- TakeExtraSymbols, [2507](#)
- TakeSymbolContent, [2509](#)
- TheErrorMessage, [2511](#)
- RatioFind
 - declare.h, [867](#)
 - ratio.c, [2505](#)
- RatioGen
 - declare.h, [867](#)
 - ratio.c, [2505](#)
- RATIONALMODE
 - ftypes.h, [1431](#)
- RatToFloat
 - float.c, [1291](#)
- RatToLine
 - declare.h, [867](#)
 - sch.c, [2656](#)
- RBRACE
 - ftypes.h, [1383](#)
- rbufnum
 - M_const, [179](#)
- rbufs
 - ParallelVars, [318](#)
- RCYCLESYMMETRIC
 - ftypes.h, [1433](#)
- ReadFile
 - declare.h, [895](#)
 - tools.c, [3086](#)
- ReadFloat
 - float.c, [1292](#)
- ReadFromScratch
 - declare.h, [1024](#)
 - store.c, [2833](#)
- ReadFromStream
 - declare.h, [886](#)
 - tools.c, [3083](#)
- ReadIjObject
 - declare.h, [985](#)
 - minos.c, [1735](#)
- ReadIndex
 - declare.h, [983](#)
 - minos.c, [1732](#)
- ReadIniInfo
 - declare.h, [984](#)
 - minos.c, [1733](#)
- ReadObject
 - declare.h, [985](#)
 - minos.c, [1735](#)
- ReadParticle
 - declare.h, [1027](#)
- ReadPolyRatFun
 - declare.h, [1007](#)
 - ratio.c, [2508](#)
- readpos
 - SpecTatoR, [401](#)
- ReadPosFile
 - declare.h, [895](#)
 - tools.c, [3086](#)
- ReadRange
 - declare.h, [1001](#)
 - transform.c, [3154](#)
- ReadSaveExpression
 - declare.h, [993](#)
 - store.c, [2840](#)
- ReadSaveHeader
 - declare.h, [992](#)
 - store.c, [2838](#)
- ReadSaveIndex
 - declare.h, [992](#)
 - store.c, [2838](#)
- ReadSaveTerm32
 - declare.h, [994](#)
 - store.c, [2839](#)
- ReadSaveVariables
 - declare.h, [995](#)
 - store.c, [2839](#)
- ReadSnum
 - declare.h, [868](#)
 - tools.c, [3091](#)
- READSPECTATORFLAG
 - ftypes.h, [1455](#)
- RecalcSetups
 - declare.h, [902](#)
 - setfile.c, [2694](#)
- RecFlag
 - T_const, [438](#)
- RecoveryFilename
 - checkpoint.c, [542](#)
 - declare.h, [997](#)
- recvBuffer
 - parallel.c, [2069](#)
- recycle_variables
 - optimize.cc, [2004](#)
- REDEFINEDEXPRESSION
 - ftypes.h, [1427](#)
- REDLENG
 - declare.h, [800](#)
- RedoTableTree
 - declare.h, [971](#)
 - tables.c, [2965](#)
- RedoTree
 - comtool.c, [767](#)
 - declare.h, [960](#)
- REDUCEMODE
 - ftypes.h, [1387](#)
- REDUCESUBEXPBUFFERS
 - compiler.c, [724](#)

- refineClass
 - MGraph, [208](#)
- REGULAR
 - ftypes.h, [1442](#)
- REGULAREXPRESSION
 - ftypes.h, [1427](#)
- REGULARSYMBOL
 - ftypes.h, [1450](#)
- reken.c, [2553](#), [2566](#)
 - AccumGCD, [2557](#)
 - AddLong, [2558](#)
 - AddPLon, [2558](#)
 - AddRat, [2556](#)
 - Bernoulli, [2565](#)
 - BigLong, [2558](#)
 - CompCoef, [2563](#)
 - COPYLONG, [2555](#)
 - DivLong, [2559](#)
 - DivRat, [2557](#)
 - Divvy, [2556](#)
 - Factorial, [2564](#)
 - GcdLong, [2562](#)
 - GCDMAX, [2555](#)
 - GetBinom, [2562](#)
 - GetLong, [2562](#)
 - GetLongModInverses, [2561](#)
 - GetModInverses, [2560](#)
 - iniwranf, [2565](#)
 - iranf, [2566](#)
 - LcmLong, [2563](#)
 - MakeInverses, [2560](#)
 - MakeLongRational, [2563](#)
 - MakeModTable, [2564](#)
 - MakeRational, [2563](#)
 - Modulus, [2564](#)
 - Moebius, [2565](#)
 - MulLong, [2558](#)
 - Mully, [2556](#)
 - MulRat, [2557](#)
 - NEWTRICK, [2555](#)
 - NextPrime, [2565](#)
 - NormalModulus, [2560](#)
 - Pack, [2556](#)
 - PreRandom, [2566](#)
 - Product, [2561](#)
 - PrtLong, [2562](#)
 - Quotient, [2561](#)
 - RaisPow, [2559](#)
 - RaisPowCached, [2559](#)
 - RaisPowMod, [2560](#)
 - Remain10, [2561](#)
 - Remain4, [2562](#)
 - Simplify, [2557](#)
 - SubPLon, [2558](#)
 - TakeLongRoot, [2563](#)
 - TakeModulus, [2564](#)
 - TakeNormalModulus, [2564](#)
 - TakeRatRoot, [2557](#)
 - UnPack, [2556](#)
 - WARMUP, [2555](#)
 - wranf, [2565](#)
- ReleaseTB
 - declare.h, [990](#)
 - tables.c, [2969](#)
- Remain10
 - declare.h, [868](#)
 - reken.c, [2561](#)
- Remain4
 - declare.h, [868](#)
 - reken.c, [2562](#)
- REMFUNCTION
 - ftypes.h, [1399](#)
- RemoveBridges
 - lus.c, [1690](#)
- RemoveDictionary
 - declare.h, [1020](#)
 - dict.c, [1076](#)
- RemoveDollars
 - declare.h, [982](#)
 - names.c, [1836](#)
- renum
 - ExPrEsSiOn, [121](#)
- RENUMBER
 - structs.h, [2904](#)
- ReNuMbEr, [378](#)
 - func, [379](#)
 - funnum, [380](#)
 - indi, [379](#)
 - indnum, [379](#)
 - startposition, [378](#)
 - symp, [378](#)
 - symnum, [379](#)
 - vecnum, [380](#)
 - vect, [379](#)
- ReNumber
 - declare.h, [868](#)
 - reshuf.c, [2613](#)
- ReNUMBERVec
 - O_const, [281](#)
- renumlists
 - ExPrEsSiOn, [122](#)
- ReNumScratch
 - N_const, [250](#)
- ReOpenFile
 - declare.h, [894](#)
 - tools.c, [3085](#)
- reorder
 - MNodeClass, [222](#)
- reordLeg
 - Assign, [19](#)
- RepCount
 - T_const, [430](#)
- RepFunList
 - N_const, [246](#)
- RepFunNum
 - N_const, [256](#)

- replace
 - ExPrEsSiOn, [123](#)
- REPLACEARG
 - ftypes.h, [1447](#)
- ReplaceDollar
 - declare.h, [891](#)
 - names.c, [1835](#)
- REPLACELOOP
 - ftypes.h, [1441](#)
- REPLACEMENT
 - ftypes.h, [1394](#)
- ReplaceScrat
 - N_const, [249](#)
- RepLevel
 - C_const, [64](#)
- RepMax
 - M_const, [187](#)
- RepPoint
 - N_const, [246](#)
- RepSumCheck
 - C_const, [64](#)
- RepTop
 - T_const, [430](#)
- request
 - PF_BUFFER, [330](#)
- reserved
 - STOREHEADER, [415](#)
 - StreaM, [420](#)
 - TaBIEs, [462](#)
- ReserveTempFiles
 - declare.h, [920](#)
 - startup.c, [2803](#)
- ResetScratch
 - declare.h, [868](#)
 - store.c, [2833](#)
- resetTimeOnClear
 - M_const, [183](#)
- ResetVariables
 - declare.h, [923](#)
 - names.c, [1836](#)
- reshuf.c, [2612](#), [2617](#)
 - AdjustRenumScratch, [2613](#)
 - CountDo, [2614](#)
 - CountFun, [2614](#)
 - DetCurDum, [2613](#)
 - DimensionExpression, [2614](#)
 - DimensionSubterm, [2614](#)
 - DimensionTerm, [2614](#)
 - DoDelta3, [2615](#)
 - DoDistrib, [2615](#)
 - DoPartitions, [2615](#)
 - DoPermutations, [2616](#)
 - DoShuffle, [2616](#)
 - DoStuffle, [2616](#)
 - EqualArg, [2615](#)
 - FinishShuffle, [2616](#)
 - FinishStuffle, [2617](#)
 - FullRenumber, [2613](#)
 - FunLevel, [2613](#)
 - MoveDummies, [2613](#)
 - MultDo, [2614](#)
 - NEWCODE, [2613](#)
 - ReNumber, [2613](#)
 - Shuffle, [2616](#)
 - Stuffle, [2616](#)
 - StuffRootAdd, [2617](#)
 - TestPartitions, [2615](#)
 - TryDo, [2615](#)
- ResizeData
 - O_const, [280](#)
- resizeFlag
 - O_const, [284](#)
- ResizeLONG
 - O_const, [281](#)
- ResizeNCWORD
 - O_const, [280](#)
- ResizePOINTER
 - O_const, [281](#)
- ResizePOS
 - O_const, [281](#)
- ResizeWORD
 - O_const, [280](#)
- ResolveSet
 - declare.h, [868](#)
 - wildcard.c, [3228](#)
- restore
 - AStack, [27](#)
- restoreCouple
 - Assign, [23](#)
- resultAGraph
 - SProcess, [406](#)
- resultMGraph
 - SProcess, [405](#)
- retstat
 - PF_BUFFER, [330](#)
- REVERSEARG
 - ftypes.h, [1447](#)
- REVERSEFILESTREAM
 - ftypes.h, [1369](#)
- REVERSEFUNCTION
 - ftypes.h, [1394](#)
- REVERSEORDER
 - ftypes.h, [1434](#)
- ReverseStatements
 - declare.h, [887](#)
 - tools.c, [3084](#)
- RevertScratch
 - declare.h, [869](#)
 - store.c, [2833](#)
- revision
 - STOREHEADER, [415](#)
- ReWorkT
 - declare.h, [838](#)
 - tables.c, [2968](#)
- right
 - NoDe, [270](#)

- rhs
 - CbUf, [72](#)
 - DICTIONARY_ELEMENT, [84](#)
- RHSIDE
 - ftypes.h, [1386](#)
- rhsInParallel
 - ParallelVars, [318](#)
- right
 - NaMeNode, [261](#)
 - tree, [479](#)
- RLONG
 - form3.h, [1333](#)
- rloser
 - NoDe, [271](#)
- ROOTFUNCTION
 - ftypes.h, [1398](#)
- rootnum
 - CbUf, [74](#)
 - TaBIEs, [464](#)
- RPARENTHESIS
 - ftypes.h, [1382](#)
- rsrc
 - NoDe, [271](#)
- RSsize
 - N_const, [254](#)
- RunAddArg
 - declare.h, [1002](#)
 - transform.c, [3153](#)
- RunCycle
 - declare.h, [1002](#)
 - transform.c, [3152](#)
- RunDecode
 - declare.h, [1000](#)
 - transform.c, [3151](#)
- RunDedup
 - declare.h, [1003](#)
 - transform.c, [3152](#)
- RunDropArg
 - declare.h, [1003](#)
 - transform.c, [3153](#)
- RunEncode
 - declare.h, [1000](#)
 - transform.c, [3151](#)
- RunExplode
 - declare.h, [1001](#)
 - transform.c, [3152](#)
- RunHtoZArg
 - declare.h, [1003](#)
 - transform.c, [3154](#)
- RunImplode
 - declare.h, [1000](#)
 - transform.c, [3152](#)
- RunIsLyndon
 - declare.h, [1002](#)
 - transform.c, [3153](#)
- RunMulArg
 - declare.h, [1002](#)
 - transform.c, [3153](#)
- RunPermute
 - declare.h, [1001](#)
 - transform.c, [3152](#)
- RunReplace
 - declare.h, [1000](#)
 - transform.c, [3151](#)
- RunReverse
 - declare.h, [1002](#)
 - transform.c, [3152](#)
- RunSelectArg
 - declare.h, [1003](#)
 - transform.c, [3153](#)
- RunToLyndon
 - declare.h, [1002](#)
 - transform.c, [3153](#)
- RunTransform
 - declare.h, [1000](#)
 - transform.c, [3151](#)
- RunZtoHArg
 - declare.h, [1003](#)
 - transform.c, [3154](#)
- RWLOCKR
 - declare.h, [819](#)
- RWLOCKW
 - declare.h, [819](#)
- rwmode
 - dbase, [79](#)
- S
 - AllGlobals, [10](#)
- S0
 - M_const, [172](#)
 - T_const, [428](#)
- S_const, [380](#)
 - Balancing, [382](#)
 - CollectOverFlag, [382](#)
 - ExecMode, [382](#)
 - MaxExprSize, [381](#)
 - MultiThreaded, [382](#)
 - NumOldNumFactors, [382](#)
 - NumOldOnFile, [382](#)
 - OldNumFactors, [381](#)
 - OldOnFile, [381](#)
 - Oldvflags, [382](#)
 - Oldvflags, [381](#)
- S_FLUSH_B
 - checkpoint.c, [540](#)
- s_one
 - FixedGlobals, [136](#)
- S_WRITE_B
 - checkpoint.c, [539](#)
- S_WRITE_DOLLAR
 - checkpoint.c, [541](#)
- S_WRITE_LIST
 - checkpoint.c, [540](#)
- S_WRITE_NAMETREE
 - checkpoint.c, [541](#)
- S_WRITE_S
 - checkpoint.c, [540](#)

- salter
 - OPTIMIZE, [294](#)
- saveCouple
 - Assign, [22](#)
- SaveData
 - O_const, [278](#)
- SaveHeader
 - O_const, [278](#)
- SAVEREVISION
 - store.c, [2832](#)
- sBer
 - T_const, [433](#)
- sbuf
 - ParallelVars, [318](#)
- sBuffer
 - sOrT, [394](#)
- sbufnum
 - M_const, [179](#)
- SBYTE
 - form3.h, [1333](#)
- ScanFunctions
 - declare.h, [869](#)
 - function.c, [1474](#)
- sch.c, [2653](#), [2659](#)
 - AddArrayIndex, [2657](#)
 - AddToLine, [2655](#)
 - CodeToLine, [2656](#)
 - FiniLine, [2655](#)
 - IniLine, [2655](#)
 - LongToLine, [2656](#)
 - MultiplyToLine, [2656](#)
 - PrintTime, [2657](#)
 - PrtTerms, [2657](#)
 - RatToLine, [2656](#)
 - StrCopy, [2655](#)
 - TalToLine, [2656](#)
 - TokenToLine, [2656](#)
 - va_arg, [2655](#)
 - va_dcl, [2654](#)
 - va_end, [2654](#)
 - va_list, [2655](#)
 - va_start, [2654](#)
 - WriteAll, [2658](#)
 - WriteArgument, [2657](#)
 - WriteDictionary, [2657](#)
 - WriteExpression, [2658](#)
 - WriteInnerTerm, [2658](#)
 - WriteLists, [2657](#)
 - WriteOne, [2658](#)
 - WriteSubTerm, [2658](#)
 - WriteTerm, [2658](#)
 - WrtPower, [2657](#)
- schemeFlags
 - OPTIMIZE, [294](#)
- schemenum
 - O_const, [284](#)
- SCRATCHSIZE
 - fsizes.h, [1347](#)
- ScratSize
 - M_const, [174](#)
- SEARCHINGPRECASE
 - ftypes.h, [1373](#)
- SEARCHINGPREENDSWITCH
 - ftypes.h, [1373](#)
- sec
 - Process, [368](#)
- sEdges
 - EGraph, [111](#)
- sedges
 - MConn, [200](#)
- SeekFile
 - declare.h, [896](#)
 - tools.c, [3086](#)
- SeekScratch
 - declare.h, [869](#)
 - store.c, [2832](#)
- SELECTARG
 - ftypes.h, [1449](#)
- selectExternalChannel
 - declare.h, [989](#)
 - extcmd.c, [1193](#)
- selectLeg
 - Assign, [18](#)
- selecttermundo
 - N_const, [249](#)
- selectVertex
 - Assign, [18](#)
- selectVertexSimp
 - Assign, [18](#)
- selfloop
 - MGraph, [214](#)
- selOPI
 - T_MGraph, [448](#)
- selUnAssLeg
 - Assign, [20](#)
- selUnAssVertex
 - Assign, [20](#)
- selUnAssVertexSimp
 - Assign, [20](#)
- SEPARATESYMBOL
 - ftypes.h, [1406](#)
- SEPARATOR
 - fwin.h, [1498](#)
 - unix.h, [3197](#)
- separators
 - C_const, [42](#)
- set_del
 - declare.h, [988](#)
 - tools.c, [3092](#)
- set_in
 - declare.h, [987](#)
 - tools.c, [3091](#)
- set_of_char
 - structs.h, [2906](#)
- set_set
 - declare.h, [987](#)

- tools.c, 3092
- set_sub
 - declare.h, 988
 - tools.c, 3092
- setAGraph
 - AStack, 27
- SETBASELENGTH
 - declare.h, 811
- SETBASEPOSITION
 - declare.h, 811
- SETBIT
 - ftypes.h, 1440
- SETBUFSIZE
 - setfile.c, 2694
- setDefaultValue
 - Options, 298
- SetDictionaryOptions
 - declare.h, 1020
 - dict.c, 1076
- SetElementList
 - C_const, 43
- SetElements
 - variable.h, 3205
- setEMom
 - EEdge, 97
- SetEndHScratch
 - declare.h, 869
 - store.c, 2832
- setEndMG
 - Options, 299
- SetEndScratch
 - declare.h, 869
 - store.c, 2832
- setErExit
 - Options, 299
- SETERROR
 - declare.h, 810
- SETEXP
 - ftypes.h, 1392
- SetExpr
 - compcomm.c, 615
 - declare.h, 927
- SetExprCases
 - compcomm.c, 615
 - declare.h, 974
- setExtern
 - EGraph, 105
 - ENode, 116
 - T_EGraph, 443
- setexterntopo
 - T_const, 439
- setExtLoop
 - EGraph, 105
- setfile.c, 2692, 2697
 - AllocFileHandle, 2695
 - AllocSetups, 2695
 - AllocSort, 2695
 - AllocSortFileName, 2695
- commentchar, 2696
- curdirp, 2696
- cursortdirp, 2696
- DeAllocFileHandle, 2695
- DEFINEVALUE, 2694
- DoSetups, 2694
- dotchar, 2696
- GetSetupPar, 2694
- highfirst, 2697
- lowfirst, 2697
- MakeSetupAllocs, 2696
- NUMERICALVALUE, 2693
- ONOFFVALUE, 2694
- PATHVALUE, 2693
- procedureextension, 2697
- ProcessOption, 2694
- RecalcSetups, 2694
- SETBUFSIZE, 2694
- setupparameters, 2697
- STRINGVALUE, 2693
- TryEnvironment, 2696
- TryFileSetups, 2696
- WriteSetup, 2695
- SetFileIndex
 - declare.h, 869
 - store.c, 2835
- SetFloatPrecision
 - float.c, 1292
- SETFUNCTION
 - ftypes.h, 1393
- setId
 - EEdge, 97
 - ENode, 116
- setintern topo
 - T_const, 439
- SETKILLMODEFOREXTERNALCHANNEL
 - declare.h, 821
- SetList
 - C_const, 43
- setLMom
 - EEdge, 97
- setmod
 - poly, 346
- SetMods
 - declare.h, 883
 - execute.c, 1157
- SETNUMBER
 - ftypes.h, 1420
- SETONLY
 - ftypes.h, 1378
- setonoff
 - compcomm.c, 612
 - declare.h, 927
- setOutAG
 - Options, 299
- setOutgrf
 - Output, 305
- setOutgrp

- Output, [305](#)
- setOutMG
 - Options, [299](#)
- setOutputF
 - Options, [298](#)
- setOutputP
 - Options, [298](#)
- SetPosFile
 - declare.h, [896](#)
 - tools.c, [3087](#)
- SeTs, [383](#)
 - dimension, [384](#)
 - first, [383](#)
 - flags, [384](#)
 - last, [383](#)
 - name, [383](#)
 - namesize, [384](#)
 - node, [384](#)
 - type, [383](#)
- Sets
 - variable.h, [3205](#)
- SetScratch
 - declare.h, [884](#)
 - store.c, [2833](#)
- SETSET
 - ftypes.h, [1391](#)
- SetSpecialMode
 - declare.h, [918](#)
 - module.c, [1765](#)
- SETSTARTPOS
 - declare.h, [813](#)
- SETTERMINATORFOREXTERNALCHANNEL
 - declare.h, [821](#)
- SETTONUM
 - ftypes.h, [1419](#)
- setType
 - EEdge, [97](#)
 - ENode, [116](#)
- SetupDir
 - M_const, [173](#)
- SETUPFILE
 - ftypes.h, [1369](#)
- SetupFile
 - M_const, [173](#)
- setupfilename
 - inivar.h, [1684](#)
 - variable.h, [3212](#)
- SetupFlag
 - C_const, [58](#)
- SetupMPFTables
 - float.c, [1293](#)
- SetupMZVTables
 - float.c, [1292](#)
- SETUPPARAMETERS, [384](#)
 - flags, [385](#)
 - parameter, [385](#)
 - type, [385](#)
 - value, [385](#)
- setupparameters
 - setfile.c, [2697](#)
- setValue
 - Fraction, [140](#)
 - Options, [298](#)
- sfact
 - T_const, [433](#)
- SFHSIZE
 - fsizes.h, [1349](#)
- sFill
 - sOrT, [395](#)
- Sflush
 - declare.h, [870](#)
 - sort.c, [2723](#)
- sFun
 - STOREHEADER, [414](#)
- SGN
 - declare.h, [800](#)
- sgn
 - TrAcEn, [472](#)
 - TrAcEs, [476](#)
- SGroup, [385](#)
 - ~SGroup, [386](#)
 - addGroup, [387](#)
 - cgen, [388](#)
 - clearGroup, [387](#)
 - csav, [388](#)
 - delGroup, [387](#)
 - eclass, [388](#)
 - elem, [388](#)
 - genNext, [387](#)
 - neclass, [388](#)
 - nElem, [387](#)
 - nelem, [388](#)
 - newGroup, [386](#)
 - nextElem, [387](#)
 - nnodes, [388](#)
 - permg, [389](#)
 - perms, [389](#)
 - pgq, [389](#)
 - pgr, [389](#)
 - print, [386](#)
 - psq, [389](#)
 - psr, [389](#)
 - SGroup, [386](#)
 - size, [388](#)
- sHalf
 - sOrT, [395](#)
- SHcombi
 - N_const, [251](#)
- SHcombisize
 - N_const, [252](#)
- SHELL
 - pre.c, [2319](#)
- shellname
 - X_const, [487](#)
- SHMWINSIZE
 - fsizes.h, [1349](#)

- shmWinSize
 - M_const, 176
- ShortSortCount
 - R_const, 377
- ShortStats
 - C_const, 54
- ShortStatsMax
 - C_const, 69
- ShrinkDictionary
 - declare.h, 1021
 - dict.c, 1077
- Shuffle
 - declare.h, 836
 - reshuf.c, 2616
- SHvar
 - N_const, 251
- SHvariables, 390
 - combilast, 391
 - do_uffle, 391
 - finishuf, 391
 - incoef, 390
 - level, 391
 - nincoef, 391
 - option, 392
 - outfun, 390
 - outterm, 390
 - stop1, 390
 - stop2, 390
 - ststop1, 391
 - ststop2, 391
 - thefunction, 391
- sld
 - EGraph, 109
- SIGALRM
 - portsignals.h, 2314
- SIGEMT
 - portsignals.h, 2313
- SIGFPE
 - portsignals.h, 2312
- SIGFUNCTION
 - ftypes.h, 1396
- SIGHUP
 - portsignals.h, 2314
- SIGILL
 - portsignals.h, 2313
- SIGINT
 - portsignals.h, 2314
- SIGLOST
 - portsignals.h, 2313
- sign
 - DiStRiBuTe, 85
 - node, 273
 - PeRmUtE, 327
 - PeRmUtEp, 328
 - poly, 345
- sign1
 - TrAcEs, 477
- sign2
 - TrAcEs, 477
- SignCheck
 - N_const, 258
- SIGNFUN
 - ftypes.h, 1396
- SIGPIPE
 - portsignals.h, 2313
- SIGPROF
 - portsignals.h, 2314
- SIGQUIT
 - portsignals.h, 2314
- SIGSEGV
 - portsignals.h, 2312
- SIGSYS
 - portsignals.h, 2313
- SIGTERM
 - portsignals.h, 2313
- SIGVTALRM
 - portsignals.h, 2314
- SIGXCPU
 - portsignals.h, 2313
- SIGXFSZ
 - portsignals.h, 2313
- silent
 - M_const, 188
- simp1token
 - declare.h, 957
 - token.c, 3052
- simp2token
 - declare.h, 957
 - token.c, 3053
- simp3atoken
 - declare.h, 957
 - token.c, 3053
- simp3btoken
 - declare.h, 958
 - token.c, 3053
- simp4token
 - declare.h, 958
 - token.c, 3053
- simp5token
 - declare.h, 958
 - token.c, 3053
- simp6token
 - declare.h, 958
 - token.c, 3053
- simple_poly
 - poly, 346, 347
- SimpleDelta
 - float.c, 1288
- SimpleDeltaC
 - float.c, 1288
- SimpleSplitMerge
 - declare.h, 973
 - sort.c, 2733
- SimpleSplitMergeRec
 - declare.h, 973
 - sort.c, 2733

- Simplify
 - declare.h, [870](#)
 - reken.c, [2557](#)
- simplify_fmpz
 - flintinterface.h, [1275](#)
- simplify_fmpz_poly
 - flintinterface.h, [1276](#)
- simpwtoken
 - declare.h, [957](#)
 - token.c, [3052](#)
- simulated_annealing
 - optimize.cc, [2000](#)
- sInd
 - STOREHEADER, [414](#)
- SINFUNCTION
 - ftypes.h, [1400](#)
- SingleTable
 - float.c, [1289](#)
- SINHFUNCTION
 - ftypes.h, [1401](#)
- SIOsize
 - M_const, [175](#)
- size
 - ARGBUFFER, [14](#)
 - DICTIONARY_ELEMENT, [84](#)
 - DoLIaRS, [88](#)
 - ExPrEsSiOn, [121](#)
 - FaCdOILaR, [125](#)
 - InDeXeNtRy, [155](#)
 - INSIDEINFO, [158](#)
 - LIST, [166](#)
 - MiNmAx, [216](#)
 - objects, [287](#)
 - PRELOAD, [359](#)
 - SGroup, [388](#)
 - TaBIEbAsEsUbInDeX, [459](#)
- size_of_form_notation
 - poly, [348](#)
- size_of_form_notation_with_den
 - poly, [348](#)
- size_of_terms
 - poly, [356](#)
- SizeCommutelnSet
 - C_const, [63](#)
- sizecouplings
 - MoDeL, [234](#)
- SizeDictionaries
 - O_const, [283](#)
- sizeelements
 - DICTIONARY, [82](#)
- SIZEFACS
 - fsizes.h, [1345](#)
- SizeForSpectatorFiles
 - M_const, [184](#)
- SizeInFile
 - sOrT, [394](#)
- SizeOfDollar
 - declare.h, [977](#)
 - dollar.c, [1095](#)
- SizeOfExpression
 - declare.h, [977](#)
 - execute.c, [1159](#)
- SIZEOFFUNCTION
 - ftypes.h, [1404](#)
- sizeprimelist
 - T_const, [439](#)
- sizeprototype
 - ExPrEsSiOn, [124](#)
- sizeselecttermundo
 - N_const, [257](#)
- SIZESTORECACHE
 - fsizes.h, [1351](#)
- SizeStoreCache
 - M_const, [175](#)
- sizevertices
 - MoDeL, [234](#)
- SKIP
 - ftypes.h, [1422](#)
- SkipAName
 - compiler.c, [724](#)
 - declare.h, [958](#)
- SKIPBLANKS
 - declare.h, [804](#)
- SKIPBRA1
 - declare.h, [805](#)
- SKIPBRA2
 - declare.h, [805](#)
- SKIPBRA3
 - declare.h, [805](#)
- SKIPBRA4
 - declare.h, [805](#)
- SKIPBRA5
 - declare.h, [805](#)
- SKIPBUFSIZE
 - pre.c, [2319](#)
- SkipClears
 - M_const, [179](#)
- SkipField
 - declare.h, [900](#)
 - tools.c, [3090](#)
- skipFLine
 - Model, [231](#)
- SKIPGEXPRESSION
 - ftypes.h, [1423](#)
- SKIPLEXPRESSION
 - ftypes.h, [1423](#)
- SkipName
 - declare.h, [1026](#)
 - pre.c, [2336](#)
- SKIPUNHIDEGEXPRESSION
 - ftypes.h, [1425](#)
- SKIPUNHIDELEXPRESSION
 - ftypes.h, [1425](#)
- SLARGEBUFFER
 - fsizes.h, [1348](#)
- SLargeSize

- M_const, 175
- slavebuf
 - ParallelVars, 318
- sLevel
 - R_const, 375
- slist
 - Interaction, 161
- sLoops
 - EGraph, 111
- small_power
 - T_const, 430
- small_power_maxn
 - T_const, 436
- small_power_maxx
 - T_const, 436
- small_power_n
 - T_const, 430
- SMALLBUFFER
 - fsizes.h, 1346
- SmallEsize
 - sOrT, 397
- SMALLOVERFLOW
 - fsizes.h, 1347
- SmallSize
 - sOrT, 397
- smart.c, 2712, 2713
 - MatchIsPossible, 2713
 - StudyPattern, 2713
- sMaxdeg
 - EGraph, 111
- SMA XF PATCHES
 - fsizes.h, 1348
- SMaxFpatches
 - M_const, 178
- SMA XPATCHES
 - fsizes.h, 1348
- SMaxPatches
 - M_const, 178
- sNodes
 - EGraph, 111
- snodes
 - MConn, 200
- SNUMBER
 - ftypes.h, 1392
- sopi
 - ToPoTyPe, 470
- SORT
 - ftypes.h, 1439
- sOrT, 392
 - cBuffer, 395
 - cBufferSize, 399
 - f, 397
 - ff, 397
 - file, 394
 - fPatches, 396
 - fPatchesStop, 396
 - fPatchN, 400
 - fPatchN1, 399
- GenSpace, 398
- GenTerms, 398
- inNum, 400
- inPatches, 396
- iPatches, 397
- ktoi, 396
- LargeSize, 397
- lBuffer, 394
- lFill, 394
- lPatch, 399
- lTop, 394
- MaxFpatches, 399
- MaxPatches, 398
- maxtermsize, 399
- newmaxtermsize, 400
- ninterms, 398
- outputmode, 400
- Patches, 395
- poin2a, 396
- poina, 396
- PoinFill, 395
- PolyFlag, 399
- PolyWise, 399
- pStop, 396
- putinsize, 398
- sBuffer, 394
- sFill, 395
- sHalf, 395
- SizeInFile, 394
- SmallEsize, 397
- SmallSize, 397
- SpaceLeft, 398
- sPointer, 395
- stage4, 400
- stagelevel, 400
- sTerms, 397
- sTop, 395
- sTop2, 395
- Terms2InSmall, 398
- TermsInSmall, 397
- TermsLeft, 398
- tree, 396
- type, 399
- used, 394
- sort.c, 2718, 2735
 - AddArgs, 2726
 - AddCoef, 2724
 - AddPoly, 2725
 - BinarySearch, 2733
 - CleanUpSort, 2733
 - Compare1, 2727
 - CompareHSymbols, 2728
 - CompareSymbols, 2727
 - ComPress, 2728
 - EndSort, 2721
 - FlushOut, 2724
 - GarbHand, 2729
 - humanBytesSuffix, 2734

- HumanString, [2720](#)
- HUMANSTRLEN, [2719](#)
- HUMANSUFFLEN, [2720](#)
- humanTermsSuffix, [2734](#)
- INSLNGTH, [2720](#)
- LowerSortLevel, [2733](#)
- MergePatches, [2730](#)
- NewSort, [2721](#)
- NEWSPLITMERGE, [2719](#)
- NEWSPLITMERGETIMSORT, [2719](#)
- numcompares, [2734](#)
- PolyRatFunSpecial, [2733](#)
- PutIn, [2722](#)
- PutOut, [2723](#)
- Sflush, [2723](#)
- SimpleSplitMerge, [2733](#)
- SimpleSplitMergeRec, [2733](#)
- SortWild, [2732](#)
- SplitMerge, [2729](#)
- StageSort, [2732](#)
- StoreTerm, [2731](#)
- totterms, [2734](#)
- WriteStats, [2720](#)
- SORTANTIPOWER
 - ftypes.h, [1389](#)
- SORTHIGHFIRST
 - ftypes.h, [1389](#)
- SORTING
 - structs.h, [2908](#)
- SORTIOSIZE
 - fsizes.h, [1347](#)
- SORTLOWFIRST
 - ftypes.h, [1389](#)
- SORTMODULE
 - ftypes.h, [1370](#)
- SORTPOWERFIRST
 - ftypes.h, [1389](#)
- SortReallocateFlag
 - C_const, [62](#)
- SortTheList
 - declare.h, [963](#)
 - lus.c, [1689](#)
- SortType
 - BrAckeTiNfO, [34](#)
 - C_const, [58](#)
 - R_const, [377](#)
- SortWeights
 - argument.c, [494](#)
 - declare.h, [931](#)
- SortWild
 - declare.h, [870](#)
 - sort.c, [2732](#)
- SoScratC
 - N_const, [250](#)
- SpaceLeft
 - sOrT, [398](#)
- spare
 - OPTIMIZE, [294](#)
 - Tables, [462](#)
 - VerTEx, [486](#)
- spare1
 - objects, [288](#)
- spare2
 - objects, [288](#)
- spare3
 - objects, [288](#)
- SpareTable
 - declare.h, [884](#)
 - tables.c, [2966](#)
- sparse
 - Tables, [463](#)
- SPARSETABLE
 - ftypes.h, [1456](#)
- spec
 - FuNcTiOn, [146](#)
- SpecialCleanup
 - declare.h, [883](#)
 - execute.c, [1157](#)
- SPECIFICATION
 - ftypes.h, [1376](#)
- SPECMASK
 - form3.h, [1327](#)
- SpecTatoR, [401](#)
 - exprnumber, [402](#)
 - fh, [402](#)
 - flags, [402](#)
 - name, [402](#)
 - position, [401](#)
 - readpos, [401](#)
- spectator.c, [2790](#), [2792](#)
 - ClearSpectators, [2792](#)
 - CoCopySpectator, [2792](#)
 - CoCreateSpectator, [2791](#)
 - CoEmptySpectator, [2791](#)
 - CoRemoveSpectator, [2791](#)
 - CoToSpectator, [2791](#)
 - FlushSpectators, [2791](#)
 - GetFromSpectator, [2792](#)
 - PutInSpectator, [2791](#)
- SPECTATOREXPRESSON
 - ftypes.h, [1425](#)
- SpectatorFiles
 - M_const, [174](#)
- SPECTATORSIZE
 - fsizes.h, [1348](#)
- SpectatorSize
 - M_const, [177](#)
- spin
 - PaRtlcLe, [321](#)
- SplitMerge
 - declare.h, [871](#)
 - sort.c, [2729](#)
- SplitScratch
 - N_const, [250](#)
- SplitScratch1
 - N_const, [250](#)

- SplitScratchSize
 - N_const, [252](#)
- SplitScratchSize1
 - N_const, [252](#)
- sPointer
 - sOrT, [395](#)
- sproc
 - Assign, [23](#)
 - EGraph, [109](#)
 - Options, [301](#)
 - Output, [308](#)
 - PNodeClass, [336](#)
 - Process, [366](#)
- SProcess, [402](#)
 - ~SProcess, [404](#)
 - agraph, [407](#)
 - agrcount, [409](#)
 - assign, [405](#)
 - astack, [406](#)
 - cl2nd, [407](#)
 - clist, [408](#)
 - DUMMYPADDING, [409](#)
 - egraph, [407](#)
 - endAGraph, [405](#)
 - endMGraph, [405](#)
 - extperm, [409](#)
 - generate, [404](#)
 - id, [408](#)
 - loop, [408](#)
 - match, [405](#)
 - mgraph, [407](#)
 - mgrcount, [409](#)
 - model, [406](#)
 - nAGraphs, [410](#)
 - nAOPI, [410](#)
 - nclass, [407](#)
 - ncouple, [408](#)
 - nd2cl, [407](#)
 - nEdges, [408](#)
 - nExtern, [408](#)
 - nfinal, [407](#)
 - ninitl, [407](#)
 - nMGraphs, [409](#)
 - nMOPI, [409](#)
 - nNodes, [408](#)
 - nvert, [408](#)
 - opt, [406](#)
 - pnclass, [406](#)
 - proc, [406](#)
 - prSProcess, [404](#)
 - resultAGraph, [406](#)
 - resultMGraph, [405](#)
 - SProcess, [404](#)
 - tCouple, [409](#)
 - toMNodeClass, [405](#)
 - wAGraphs, [410](#)
 - wAOPI, [410](#)
 - wMGraphs, [409](#)
 - wMOPI, [410](#)
- sptbl
 - Process, [366](#)
- SQRTFUNCTION
 - ftypes.h, [1400](#)
- SS
 - T_const, [428](#)
- SSMALLBUFFER
 - fsizes.h, [1347](#)
- SSmallEsize
 - M_const, [175](#)
- SSMALLOVERFLOW
 - fsizes.h, [1348](#)
- SSmallSize
 - M_const, [175](#)
- SSORTIOSIZE
 - fsizes.h, [1348](#)
- sSym
 - STOREHEADER, [414](#)
- st
 - NCand, [267](#)
 - NStack, [276](#)
- stage4
 - sOrT, [400](#)
- Stage4Name
 - R_const, [375](#)
- stagelevel
 - sOrT, [400](#)
- StageSort
 - declare.h, [922](#)
 - sort.c, [2732](#)
- stap
 - TrAcEs, [476](#)
- start
 - BrAcKeTiNdEx, [32](#)
 - VaRrEnUm, [483](#)
- StartFiles
 - declare.h, [898](#)
 - tools.c, [3084](#)
- startlinenumber
 - DoLoOp, [91](#)
- StartLoops
 - declare.h, [1029](#)
 - lus.c, [1689](#)
- StartMore
 - startup.c, [2803](#)
- startposition
 - ReNuMbEr, [378](#)
- StartPrepro
 - declare.h, [911](#)
 - pre.c, [2331](#)
- startup.c, [2800](#), [2806](#)
 - CleanUp, [2804](#)
 - DEFAULTFNAMELENGTH, [2802](#)
 - defaulttempfilename, [2805](#)
 - DoTail, [2803](#)
 - emptystring, [2805](#)
 - FORMNAME, [2802](#)

- GetRunningTime, [2805](#)
- IniVars, [2804](#)
- main, [2804](#)
- OpenInput, [2803](#)
- PrintDeprecation, [2804](#)
- PrintRunningTime, [2805](#)
- ReserveTempFiles, [2803](#)
- StartMore, [2803](#)
- StartVariables, [2803](#)
- STRINGIFY, [2802](#)
- STRINGIFY__, [2802](#)
- TAKEPATH, [2802](#)
- TerminateImpl, [2804](#)
- VERSIONSTR, [2802](#)
- VERSIONSTR__, [2802](#)
- startup_init
 - flintinterface.h, [1274](#)
- StartVariables
 - declare.h, [822](#)
 - startup.c, [2803](#)
- STATEMENT
 - ftypes.h, [1376](#)
- STATIC_ASSERT
 - form3.h, [1326](#)
- STATIC_ASSERT__1
 - form3.h, [1326](#)
- STATIC_ASSERT__2
 - form3.h, [1326](#)
- STATIC_ASSERT__3
 - form3.h, [1326](#)
- StatsFlag
 - C_const, [57](#)
- STATSMERGETOFILE
 - ftypes.h, [1390](#)
- STATSOUT
 - ftypes.h, [1374](#)
- STATSPOSTSORT
 - ftypes.h, [1390](#)
- STATSSPLITMERGE
 - ftypes.h, [1390](#)
- status
 - ExPrEsSiOn, [123](#)
 - PF_BUFFER, [329](#)
- STDERR
 - pre.c, [2319](#)
- stderrname
 - X_const, [487](#)
- StdOut
 - M_const, [178](#)
- step1
 - TrAcEs, [476](#)
- STEP2
 - dollar.c, [1092](#)
- sTerms
 - sOrT, [397](#)
- STermsInSmall
 - M_const, [175](#)
- TERMSSMALL
 - fsizes.h, [1348](#)
- sTop
 - sOrT, [395](#)
- stop
 - PF_BUFFER, [329](#)
- stop1
 - SHvariables, [390](#)
- sTop2
 - sOrT, [395](#)
- stop2
 - SHvariables, [390](#)
- StopWatchZero
 - P_const, [313](#)
- STORE
 - ftypes.h, [1439](#)
- store.c, [2831](#), [2841](#)
 - AddToScratch, [2833](#)
 - CoLoad, [2834](#)
 - CopyExpression, [2837](#)
 - CoSave, [2833](#)
 - DeleteStore, [2834](#)
 - DetVars, [2835](#)
 - FindInIndex, [2837](#)
 - FindrNumber, [2837](#)
 - GetFromStore, [2835](#)
 - GetMoreFromMem, [2835](#)
 - GetMoreTerms, [2834](#)
 - GetOneTerm, [2834](#)
 - GetTable, [2837](#)
 - GetTerm, [2834](#)
 - NextFileIndex, [2835](#)
 - OpenTemp, [2832](#)
 - PutInStore, [2834](#)
 - ReadFromScratch, [2833](#)
 - ReadSaveExpression, [2840](#)
 - ReadSaveHeader, [2838](#)
 - ReadSaveIndex, [2838](#)
 - ReadSaveTerm32, [2839](#)
 - ReadSaveVariables, [2839](#)
 - ResetScratch, [2833](#)
 - RevertScratch, [2833](#)
 - SAVEREVISION, [2832](#)
 - SeekScratch, [2832](#)
 - SetEndHScratch, [2832](#)
 - SetEndScratch, [2832](#)
 - SetFileIndex, [2835](#)
 - SetScratch, [2833](#)
 - TermRenummer, [2836](#)
 - ToStorage, [2835](#)
 - VarStore, [2836](#)
 - WriteStoreHeader, [2837](#)
- STORECACHE
 - structs.h, [2907](#)
- StOrEcAcHe, [411](#)
 - buffer, [412](#)
 - next, [412](#)
 - position, [411](#)
 - toppos, [411](#)

- StoreCache
 - T_const, [429](#)
- StoreCacheAlloc
 - T_const, [429](#)
- StoreData
 - R_const, [371](#)
- STOREEXPRESSION
 - ftypes.h, [1424](#)
- StoreFileSize
 - C_const, [42](#)
- StoreHandle
 - C_const, [68](#)
- STOREHEADER, [412](#)
 - endianness, [414](#)
 - headermark, [413](#)
 - lenLONG, [413](#)
 - lenPOINTER, [413](#)
 - lenPOS, [413](#)
 - lenWORD, [413](#)
 - maxpower, [415](#)
 - reserved, [415](#)
 - revision, [415](#)
 - sFun, [414](#)
 - sInd, [414](#)
 - sSym, [414](#)
 - sVec, [414](#)
 - wildoffset, [415](#)
- STOREMODULE
 - ftypes.h, [1370](#)
- StoreTerm
 - declare.h, [872](#)
 - sort.c, [2731](#)
- str_dup
 - declare.h, [983](#)
 - minos.c, [1731](#)
- StrCmp
 - declare.h, [900](#)
 - tools.c, [3088](#)
- StrCopy
 - declare.h, [824](#)
 - sch.c, [2655](#)
- strDup1
 - declare.h, [884](#)
 - tools.c, [3090](#)
- STREAM
 - structs.h, [2905](#)
- StreaM, [416](#)
 - afterwards, [419](#)
 - buffer, [417](#)
 - bufferposition, [418](#)
 - bufferize, [418](#)
 - eqnum, [419](#)
 - fileposition, [416](#)
 - FoldName, [417](#)
 - handle, [418](#)
 - inbuffer, [418](#)
 - isnextchar, [419](#)
 - linenumber, [416](#)
 - name, [417](#)
 - nextchar, [419](#)
 - olddelay, [419](#)
 - oldnoshowinput, [419](#)
 - pname, [418](#)
 - pointer, [417](#)
 - prevars, [419](#)
 - previous, [418](#)
 - previousNoShowInput, [419](#)
 - prevline, [417](#)
 - reserved, [420](#)
 - top, [417](#)
 - type, [418](#)
- Streams
 - C_const, [46](#)
- StrHICmp
 - declare.h, [901](#)
 - tools.c, [3088](#)
- StrICmp
 - declare.h, [900](#)
 - tools.c, [3088](#)
- StrICont
 - declare.h, [901](#)
 - tools.c, [3089](#)
- strict
 - TaBIEs, [463](#)
- STRINGIFY
 - pre.c, [2319](#)
 - startup.c, [2802](#)
- STRINGIFY__
 - pre.c, [2319](#)
 - startup.c, [2802](#)
- STRINGVALUE
 - setfile.c, [2693](#)
- StrLen
 - declare.h, [901](#)
 - tools.c, [3089](#)
- structs.h, [2899](#), [2909](#)
 - ALLGLOBALS, [2908](#)
 - BHEAD, [2903](#)
 - BHEAD0, [2903](#)
 - CBUF, [2907](#)
 - CHANNEL, [2907](#)
 - COMPARE, [2908](#)
 - COMPAREDUMMY, [2906](#)
 - COMPTREE, [2904](#)
 - DISTRIBUTE, [2907](#)
 - DO_UFFLE, [2906](#)
 - DOLOOP, [2906](#)
 - EMPTYININDEX, [2903](#)
 - FILEHANDLE, [2905](#)
 - FILEINDEX, [2904](#)
 - FINISHUFFLE, [2906](#)
 - FIXEDGLOBALS, [2908](#)
 - FUNCTIONS, [2905](#)
 - INDEXENTRY, [2904](#)
 - INFILEINDEX, [2903](#)
 - NAMENODE, [2904](#)

- NAMETREE, [2904](#)
- NESTING, [2907](#)
- one_byte, [2906](#)
- PARTI, [2908](#)
- PERM, [2907](#)
- PERMP, [2907](#)
- PHEAD, [2903](#)
- PHEAD0, [2903](#)
- PREVAR, [2906](#)
- RENUMBER, [2904](#)
- set_of_char, [2906](#)
- SORTING, [2908](#)
- STORECACHE, [2907](#)
- STREAM, [2905](#)
- TABLES, [2905](#)
- TRACEN, [2905](#)
- TRACES, [2905](#)
- VARRENUM, [2904](#)
- WCN, [2908](#)
- WCN2, [2908](#)
- ststop1
 - SHvariables, [391](#)
- ststop2
 - SHvariables, [391](#)
- StudyPattern
 - declare.h, [964](#)
 - smart.c, [2713](#)
- StuffAdd
 - declare.h, [801](#)
- Stuffle
 - declare.h, [837](#)
 - reshuf.c, [2616](#)
- StuffRootAdd
 - declare.h, [837](#)
 - reshuf.c, [2617](#)
- sub
 - Fraction, [141](#)
 - poly, [350](#)
- SUBAFTER
 - ftypes.h, [1435](#)
- SUBAFTERNOT
 - ftypes.h, [1435](#)
- SUBALL
 - ftypes.h, [1434](#)
- SuBbUf, [420](#)
 - buffernum, [420](#)
 - subexpnum, [420](#)
- subCouple
 - Assign, [23](#)
- SUBDISORDER
 - ftypes.h, [1435](#)
- subexpbuffers
 - compiler.c, [726](#)
- subexpnum
 - SuBbUf, [420](#)
- SUBEXPR
 - ftypes.h, [1436](#)
- SUBEXPRESSION
 - ftypes.h, [1391](#)
- SUBEXPSIZE
 - ftypes.h, [1422](#)
- subindex
 - TabIEbAsE, [458](#)
- SUBMANY
 - ftypes.h, [1434](#)
- SUBMASK
 - ftypes.h, [1435](#)
- SUBMULTI
 - ftypes.h, [1434](#)
- SUBONCE
 - ftypes.h, [1434](#)
- SUBONLY
 - ftypes.h, [1434](#)
- SubPLon
 - declare.h, [873](#)
 - reken.c, [2558](#)
- SUBROUTINEFILE
 - ftypes.h, [1369](#)
- SUBSELECT
 - ftypes.h, [1434](#)
- SubsInAll
 - declare.h, [974](#)
 - pattern.c, [2176](#)
- SUBSORT
 - ftypes.h, [1431](#)
- Substitute
 - declare.h, [873](#)
 - pattern.c, [2176](#)
- subsubveto
 - N_const, [255](#)
- SUBTERMUSED1
 - ftypes.h, [1421](#)
- SUBTERMUSED2
 - ftypes.h, [1421](#)
- SUBVECTOR
 - ftypes.h, [1434](#)
- sum_results
 - tree_node, [481](#)
- SUMF1
 - ftypes.h, [1394](#)
- SUMF2
 - ftypes.h, [1394](#)
- SUMMEDIND
 - ftypes.h, [1430](#)
- sumnum
 - M_const, [189](#)
- SUMNUM1
 - ftypes.h, [1417](#)
- SUMNUM2
 - ftypes.h, [1417](#)
- sumpnum
 - M_const, [189](#)
- SumTime
 - M_const, [177](#)
- sVec
 - STOREHEADER, [414](#)

- SWAP
 - parallel.c, 2066
- SWITCH, 421
 - caseoffset, 422
 - defaultcase, 421
 - endswitch, 422
 - iflevel, 422
 - maxcase, 422
 - mincase, 422
 - nestingsum, 423
 - numcases, 422
 - padding, 423
 - table, 421
 - tablesize, 422
 - typetable, 422
 - whilelevel, 423
- SwitchArray
 - C_const, 46
- SwitchHeap
 - C_const, 47
- SwitchInArray
 - C_const, 67
- SwitchLevel
 - C_const, 67
- SwitchSplitMerge
 - declare.h, 1025
 - if.c, 1658
- SwitchSplitMergeRec
 - declare.h, 1025
 - if.c, 1658
- SWITCHTABLE, 423
 - compbuffer, 424
 - ncase, 423
 - value, 423
- swmes
 - FixedGlobals, 136
- symb
 - ReNuMbEr, 378
- SYMBOL
 - ftypes.h, 1390
- SyMbOl, 424
 - complex, 425
 - dimension, 425
 - flags, 425
 - maxpower, 424
 - minpower, 424
 - name, 424
 - namesize, 425
 - node, 425
 - number, 425
- SYMBOL_
 - ftypes.h, 1428
- SymbolList
 - C_const, 44
- SymbolNormalize
 - declare.h, 990
 - normal.c, 1877
- SYMBOLONLY
 - ftypes.h, 1378
- Symbols
 - C_const, 45
- symbols
 - variable.h, 3206
- SymChangeFlag
 - C_const, 69
- SymFind
 - declare.h, 873
 - function.c, 1473
- SymGen
 - declare.h, 873
 - function.c, 1473
- symmet
 - FUN_INFO, 143
- symmetr.c, 2935, 2937
 - Distribute, 2936
 - FunMatchCy, 2936
 - FunMatchSy, 2936
 - MatchArgument, 2936
 - MatchCy, 2936
 - MatchE, 2935
 - Permute, 2935
 - PermuteP, 2936
- SYMMETRIC
 - ftypes.h, 1433
- symmetric
 - FuNcTiOn, 146
- SYMMETRIZE
 - ftypes.h, 1417
- Symmetrize
 - declare.h, 874
 - function.c, 1472
- symminfo
 - FuNcTiOn, 145
- symnum
 - ReNuMbEr, 379
- SYMTONUM
 - ftypes.h, 1418
- SYMTOSUB
 - ftypes.h, 1418
- SYMTOSYM
 - ftypes.h, 1418
- SynchFile
 - declare.h, 896
 - tools.c, 3087
- T
 - AllGlobals, 11
- T_const, 426
 - aebufnum, 434
 - allbufnum, 434
 - bernoullis, 430
 - BrackBuf, 429
 - bracketindexflag, 435
 - bracketinfo, 432
 - CacheNumberMemHeap, 432
 - CacheNumberMemMax, 435
 - CacheNumberMemTop, 435

comfun, 437
comind, 437
comnum, 437
comsym, 436
dummysubexp, 436
ebufnum, 433
factorials, 430
fbufnum, 434
fromindex, 439
FunArg, 437
idallflag, 434
idallmaxnum, 434
idallnum, 434
InNumMem, 433
inprimelist, 439
LeaveNegative, 436
ListPoly, 432
ListSymbols, 432
ListSymbolsSize, 435
locwildvalue, 437
IWorkPointer, 433
IWorkSpace, 429
mBer, 438
mfac, 433
MinVecArg, 437
mulpat, 437
nBer, 438
Nest, 428
NestPoin, 428
NestStop, 428
nfac, 438
NumberMemHeap, 431
NumberMemMax, 435
NumberMemTop, 435
NumListSymbols, 436
NumMem, 432
numpoly, 436
optintimes, 435
partitions, 432
pBer, 431
pfac, 431
PolyAct, 438
posWorkPointer, 433
posWorkSpace, 429
previousEfactor, 431
primelist, 431
proexp, 437
pWorkPointer, 433
pWorkSpace, 429
RecFlag, 438
RepCount, 430
RepTop, 430
S0, 428
sBer, 433
setexterntopo, 439
setinterntopo, 439
sfact, 433
sizeprimelist, 439
small_power, 430
small_power_maxn, 436
small_power_maxx, 436
small_power_n, 430
SS, 428
StoreCache, 429
StoreCacheAlloc, 429
TermMemHeap, 431
TermMemMax, 434
TermMemTop, 435
TMaddr, 431
TMbuff, 438
TMdolfac, 438
TMout, 438
TopologiesLevel, 439
TopologiesOptions, 439
TopologiesStart, 432
TopologiesTerm, 432
TrimPower, 436
WildArgTaken, 430
WildcardBufferSize, 434
WildMask, 431
WorkPointer, 430
WorkSpace, 429
WorkTop, 429
T_EEdge, 440
 ext, 440
 momc, 440
 momn, 440
 nodes, 440
 padding, 441
T_EGraph, 441
 ~T_EGraph, 442
 edges, 444
 endSetExtern, 443
 esym, 444
 gld, 443
 init, 442
 maxdeg, 444
 nEdges, 443
 nExtern, 444
 nNodes, 443
 nodes, 444
 nsym, 444
 opi, 444
 pld, 443
 print, 442
 setExtern, 443
 T_EGraph, 442
T_ENode, 445
 deg, 445
 edges, 445
 ext, 445
T_MGraph, 446
 ~T_MGraph, 447
 adjMat, 449
 c1PI, 449
 clist, 448

- curcl, 450
- discardDisc, 450
- discardIso, 451
- discardOrd, 450
- discardRefine, 450
- egraph, 450
- esym, 449
- generate, 447
- maxdeg, 448
- mindeg, 448
- n1PI, 449
- nBridges, 449
- nCallRefine, 450
- nClasses, 448
- ndiag, 448
- nEdges, 447
- ngconn, 450
- ngen, 450
- nLoops, 448
- nNodes, 447
- nodes, 448
- nsym, 449
- pld, 447
- selOPI, 448
- T_MGraph, 447
- wsopi, 449
- wsum, 449
- T_MNode, 451
 - cls, 452
 - deg, 452
 - ext, 452
 - freelg, 452
 - id, 452
 - T_MNode, 451
 - visited, 452
- T_MNodeClass, 453
 - ~T_MNodeClass, 453
 - chkOrd, 455
 - clCmp, 454
 - clist, 455
 - clmat, 456
 - cmpArray, 455
 - copy, 454
 - flist, 456
 - foralignment, 456
 - incMat, 455
 - init, 454
 - maxdeg, 456
 - mkCIMat, 454
 - mkFlist, 454
 - mkNdCI, 454
 - nClasses, 455
 - ndcl, 456
 - nNodes, 455
 - printMat, 454
 - T_MNodeClass, 453
- tabl
 - FuNcTiOn, 145
- table
 - SWITCH, 421
- TaBIEbAsE, 457
 - current, 457
 - fillpoint, 457
 - name, 458
 - numtables, 458
 - subindex, 458
 - tablenumbers, 458
- TABLEBASEFILE
 - ftypes.h, 1370
- TableBaseList
 - C_const, 44
- tablebases
 - variable.h, 3206
- TaBIEbAsEsUbInDeX, 458
 - size, 459
 - where, 459
- tablecheck
 - C_const, 56
- TABLEEXTENSION
 - fsizes.h, 1349
- tablefilling
 - C_const, 60
- TABLEFUNCTION
 - ftypes.h, 1398
- tablenamefill
 - dbase, 79
- tablenames
 - dbase, 80
- tablenamessize
 - dbase, 79
- tablenum
 - TaBIEs, 465
- tablenumber
 - objects, 287
- tablenumbers
 - TaBIEbAsE, 458
- tablepointers
 - TaBIEs, 460
- TableReset
 - declare.h, 838
 - tables.c, 2969
- TABLES
 - structs.h, 2905
- TaBIEs, 459
 - argtail, 461
 - boomlijst, 461
 - bounds, 463
 - buffers, 462
 - buffersfill, 465
 - bufferssize, 464
 - bufnum, 464
 - defined, 462
 - flags, 461
 - MaxTreeSize, 464
 - mdefined, 463
 - mm, 461

- mode, 465
- numdummies, 465
- numind, 463
- numtree, 464
- pattern, 461
- prototype, 460
- prototypeSize, 463
- reserved, 462
- rootnum, 464
- spare, 462
- sparse, 463
- strict, 463
- tablenum, 465
- tablepointers, 460
- totind, 462
- tables.c, 2964, 2970
 - Apply, 2969
 - ApplyExec, 2969
 - ApplyReset, 2969
 - CheckTableDeclarations, 2967
 - ClearTableTree, 2965
 - CoApply, 2968
 - CoTableBase, 2966
 - CoTBaddto, 2966
 - CoTBaudit, 2967
 - CoTbcleanup, 2968
 - CoTBcreate, 2966
 - CoTBenter, 2967
 - CoTBhelp, 2968
 - CoTBload, 2967
 - CoTBoff, 2968
 - CoTBon, 2967
 - CoTBopen, 2966
 - CoTBreplace, 2968
 - CoTBuse, 2968
 - CoTestUse, 2967
 - DoTableExpansion, 2965
 - FindTableTree, 2965
 - FindTB, 2966
 - FlipTable, 2966
 - helpTB, 2970
 - InsTableTree, 2965
 - RedoTableTree, 2965
 - ReleaseTB, 2969
 - ReWorkT, 2968
 - SpareTable, 2966
 - TableReset, 2969
 - TestUse, 2967
- TABSIZE
 - declare.h, 809
- tablesize
 - SWITCH, 422
- TABLESTUB
 - ftypes.h, 1399
- tabstring
 - O_const, 279
- tadblock
 - MGraph, 215
- tadpole
 - MGraph, 214
- tag
 - PF_BUFFER, 330
- TakeArgContent
 - argument.c, 492
 - declare.h, 928
- TakeContent
 - declare.h, 1009
 - ratio.c, 2507
- TakeDollarContent
 - declare.h, 965
 - dollar.c, 1098
- TakeExtraSymbols
 - declare.h, 1009
 - ratio.c, 2507
- TakeIdfunction
 - declare.h, 975
 - pattern.c, 2177
- TakeLongRoot
 - declare.h, 970
 - reken.c, 2563
- TakeModulus
 - declare.h, 874
 - reken.c, 2564
- TakeNormalModulus
 - declare.h, 874
 - reken.c, 2564
- TakeOneLine
 - lus.c, 1691
- TAKEPATH
 - startup.c, 2802
- TakeRatRoot
 - declare.h, 970
 - reken.c, 2557
- TakeSymbolContent
 - declare.h, 1008
 - ratio.c, 2509
- TAKETRACE
 - ftypes.h, 1417
- TalToLine
 - declare.h, 874
 - sch.c, 2656
- TANFUNCTION
 - ftypes.h, 1400
- TANHFUNCTION
 - ftypes.h, 1401
- TCOMMA
 - ftypes.h, 1383
- TCONJUGATE
 - ftypes.h, 1385
- tCouple
 - SProcess, 409
- TDELTA
 - ftypes.h, 1382
- TDIVIDE
 - ftypes.h, 1383
- TDOLLAR

- ftypes.h, 1382
- TDOT
 - ftypes.h, 1383
- TDOTPRODUCT
 - ftypes.h, 1381
- TDUBIOUS
 - ftypes.h, 1382
- TeInFun
 - N_const, 256
- TELLFILE
 - declare.h, 822
 - tools.c, 3086
- TellFile
 - declare.h, 896
 - tools.c, 3086
- TempDir
 - M_const, 172
- TempSortDir
 - M_const, 173
- EMPTY
 - ftypes.h, 1385
- TENDOFIT
 - ftypes.h, 1384
- tensor
 - FUN_INFO, 143
- TENSORFUNCTION
 - ftypes.h, 1427
- tensorList
 - O_const, 281
- TENVEC
 - ftypes.h, 1417
- TenVec
 - declare.h, 875
 - opera.c, 1961
- TenVecFind
 - declare.h, 875
 - opera.c, 1960
- TePos
 - R_const, 375
- terfirstcomm
 - N_const, 247
- term
 - TERMINFO, 466
- term_compare
 - optimize.cc, 1997
- TermAssign
 - declare.h, 962
 - dollar.c, 1093
- termbuf
 - O_const, 279
- termbuffer
 - N_const, 249
- TERMEXTRAWORDS
 - tools.c, 3081
- TermFree
 - declare.h, 817
- TERMFUNCTION
 - ftypes.h, 1398
- termfunnum
 - M_const, 189
- TERMINAL
 - pre.c, 2320
- Terminate
 - declare.h, 810
- TerminatImpl
 - declare.h, 888
 - startup.c, 2804
- TERMINFO, 466
 - currentMODEL, 466
 - currentModel, 466
 - diaoffset, 467
 - externalset, 467
 - flags, 467
 - internalset, 467
 - legcouple, 466
 - level, 467
 - numdia, 467
 - numextern, 467
 - numtopo, 466
 - term, 466
- termlevel
 - C_const, 65
- TermMalloc
 - declare.h, 816
- TermMallocAddMemory
 - declare.h, 999
 - tools.c, 3093
- TermMemHeap
 - T_const, 431
- TermMemMax
 - T_const, 434
- TERMMEMSTARTNUM
 - tools.c, 3081
- TermMemTop
 - T_const, 435
- termp
 - TrAcEn, 471
 - TrAcEs, 474
- TermRenumber
 - declare.h, 875
 - store.c, 2836
- terms
 - poly, 356
- Terms2InSmall
 - sOrT, 398
- termscopy
 - poly, 343
- TERMSINBRACKET
 - ftypes.h, 1399
- TermsInBracket
 - declare.h, 982
 - execute.c, 1160
- termsinbracket
 - BrAcKeTiNdEx, 32
- TermsInDollar
 - declare.h, 977

- dollar.c, [1095](#)
- TERMSINEXPR
 - ftypes.h, [1398](#)
- TermsInExpression
 - declare.h, [977](#)
 - execute.c, [1159](#)
- TermsInSmall
 - sOrT, [397](#)
- termsize
 - NeStInG, [269](#)
- TermsLeft
 - sOrT, [398](#)
- termstortstack
 - C_const, [47](#)
- TERMSSMALL
 - fsizes.h, [1347](#)
- termstack
 - C_const, [47](#)
- termsumcheck
 - C_const, [49](#)
- terstart
 - N_const, [247](#)
- terstop
 - N_const, [246](#)
- TestArgNum
 - declare.h, [1001](#)
 - transform.c, [3154](#)
- TestDoLoop
 - declare.h, [1011](#)
 - dollar.c, [1097](#)
- TestDrop
 - declare.h, [875](#)
 - execute.c, [1156](#)
- TestEndDoLoop
 - declare.h, [1011](#)
 - dollar.c, [1097](#)
- TestFloat
 - float.c, [1291](#)
- TestFunFlag
 - declare.h, [990](#)
 - normal.c, [1878](#)
- TestMatch
 - declare.h, [876](#)
 - pattern.c, [2175](#)
- TestName
 - declare.h, [893](#)
 - names.c, [1837](#)
- TestPartitions
 - declare.h, [834](#)
 - reshuf.c, [2615](#)
- TestSelect
 - declare.h, [974](#)
 - pattern.c, [2176](#)
- TestSub
 - declare.h, [877](#)
 - proces.c, [2429](#)
- TestTables
 - compiler.c, [725](#)
- declare.h, [958](#)
- TestTerm
 - declare.h, [999](#)
 - tools.c, [3097](#)
- TestUse
 - declare.h, [837](#)
 - tables.c, [2967](#)
- TestValue
 - C_const, [52](#)
- TeSuOut
 - N_const, [257](#)
- TEXPRESSION
 - ftypes.h, [1381](#)
- TFLOAT
 - ftypes.h, [1386](#)
- TFUN
 - ftypes.h, [1458](#)
- TFUN1
 - ftypes.h, [1458](#)
- TFUNCLOSE
 - ftypes.h, [1383](#)
- TFUNCTION
 - ftypes.h, [1381](#)
- TFUNOPEN
 - ftypes.h, [1383](#)
- TGENINDEX
 - ftypes.h, [1384](#)
- TheDefine
 - declare.h, [906](#)
 - pre.c, [2323](#)
- TheErrorMessage
 - ratio.c, [2511](#)
- thefunction
 - SHvariables, [391](#)
- theposition
 - N_const, [245](#)
- THETA
 - ftypes.h, [1397](#)
- THETA2
 - ftypes.h, [1397](#)
- TheUndefine
 - declare.h, [906](#)
 - pre.c, [2325](#)
- ThreadBalancing
 - C_const, [58](#)
- ThreadBucketSize
 - C_const, [53](#)
- threads.c, [2997](#)
- THREADSCRATCHOUTSIZE
 - fsizes.h, [1350](#)
- THREADSCRATCHSIZE
 - fsizes.h, [1350](#)
- THREADSDEBUG
 - ftypes.h, [1374](#)
- ThreadsFlag
 - C_const, [58](#)
- ThreadSortFileSynch
 - C_const, [59](#)

- ThreadStats
 - C_const, 58
- time0
 - Options, 303
- time1
 - Options, 303
- TimeChildren
 - declare.h, 877
 - tools.c, 3096
- TimeCPU
 - declare.h, 877
 - tools.c, 3096
- TimeLimit
 - M_const, 177
- timeout
 - X_const, 487
- TimeWallClock
 - declare.h, 878
 - tools.c, 3096
- TINDEX
 - ftypes.h, 1381
- tlistbuf
 - N_const, 251
- tlistsize
 - N_const, 255
- TMaddr
 - T_const, 431
- TMbuff
 - T_const, 438
- TMdolfac
 - T_const, 438
- TMINUS
 - ftypes.h, 1384
- TMout
 - T_const, 438
- TMPPOLYFUN
 - ftypes.h, 1392
- TMULTIPLY
 - ftypes.h, 1383
- TNOT
 - ftypes.h, 1384
- TNUMBER
 - ftypes.h, 1381
- TNUMBER1
 - ftypes.h, 1385
- to_argument_mpoly
 - flintinterface.h, 1274, 1275
- to_argument_poly
 - flintinterface.h, 1275
- to_coefficient_list
 - poly, 348
- to_string
 - poly, 349
- TOBEFACTORED
 - ftypes.h, 1432
- ToBelFactors
 - C_const, 69
- TOBEUNFACTORED
 - ftypes.h, 1432
- TODISK
 - minos.h, 1755
- ToFast
 - declare.h, 902
 - tools.c, 3095
- ToFloat
 - float.c, 1293
- toForm
 - topowrap.cc, 3146
- ToGeneral
 - declare.h, 902
 - tools.c, 3095
- tohunt
 - N_const, 253
- token.c, 3051, 3054
 - CHECKPOLY, 3052
 - simp1token, 3052
 - simp2token, 3053
 - simp3atoken, 3053
 - simp3btoken, 3053
 - simp4token, 3053
 - simp5token, 3053
 - simp6token, 3053
 - simpwtoken, 3052
 - tokenize, 3052
 - ttypes, 3054
 - WriteTokens, 3052
- tokenarglevel
 - C_const, 50
- tokenize
 - declare.h, 957
 - token.c, 3052
- tokens
 - C_const, 50
- TokensWriteFlag
 - C_const, 57
- TOKENTOLINE
 - declare.h, 802
- TokenToLine
 - declare.h, 878
 - sch.c, 2656
- TOLONG
 - declare.h, 814
- TOLYNDON
 - ftypes.h, 1448
- TOLYNDONR
 - ftypes.h, 1448
- toMNodeClass
 - SProcess, 405
- tools.c, 3079, 3099
 - AddToString, 3090
 - AreArgsEqual, 3095
 - CacheNumberMallocAddMemory, 3093
 - CloseChannel, 3088
 - CloseFile, 3085
 - CloseStream, 3083
 - CmpArray, 3089

- CompareArgs, [3096](#)
- CompArg, [3096](#)
- ConWord, [3089](#)
- CopyFile, [3085](#)
- Crash, [3097](#)
- CreateFile, [3085](#)
- CreateHandle, [3085](#)
- CreateLogFile, [3085](#)
- CreateStream, [3084](#)
- DistrN, [3098](#)
- DODOUBLE, [3082](#)
- DOEXPAND, [3082](#)
- DoubleBuffer, [3094](#)
- DoubleList, [3094](#)
- DoubleLList, [3094](#)
- EndOfToken, [3090](#)
- ExpandBuffer, [3094](#)
- filelist, [3098](#)
- filelistsize, [3098](#)
- FILLVALUE, [3081](#)
- FlushFile, [3087](#)
- From0List, [3094](#)
- FromList, [3093](#)
- FromVarList, [3094](#)
- GetAppendChannel, [3088](#)
- GetChannel, [3087](#)
- GetFromStream, [3083](#)
- GetPosFile, [3087](#)
- GetStreamPosition, [3084](#)
- iexp, [3095](#)
- iniTools, [3092](#)
- IsLikeVector, [3095](#)
- LoadInputFile, [3083](#)
- LocateFile, [3083](#)
- LongCopy, [3091](#)
- LongLongCopy, [3091](#)
- LookInStream, [3083](#)
- M_check1, [3093](#)
- M_free, [3093](#)
- M_print, [3093](#)
- MakeDate, [3091](#)
- Malloc, [3092](#)
- Malloc1, [3092](#)
- NUMBEREXTRAWORDS, [3082](#)
- NumberMallocAddMemory, [3093](#)
- NUMBERMEMSTARTNUM, [3082](#)
- NumCopy, [3091](#)
- numinfilelist, [3098](#)
- NumToStr, [3089](#)
- OpenAddFile, [3084](#)
- OpenFile, [3084](#)
- OpenStream, [3083](#)
- PositionStream, [3084](#)
- ReadFile, [3086](#)
- ReadFromStream, [3083](#)
- ReadPosFile, [3086](#)
- ReadSnum, [3091](#)
- ReOpenFile, [3085](#)
- ReverseStatements, [3084](#)
- SeekFile, [3086](#)
- set_del, [3092](#)
- set_in, [3091](#)
- set_set, [3092](#)
- set_sub, [3092](#)
- SetPosFile, [3087](#)
- SkipField, [3090](#)
- StartFiles, [3084](#)
- StrCmp, [3088](#)
- strDup1, [3090](#)
- StrHICmp, [3088](#)
- StrlCmp, [3088](#)
- StrlCont, [3089](#)
- StrLen, [3089](#)
- SynchFile, [3087](#)
- TELLFILE, [3086](#)
- TellFile, [3086](#)
- TERMEXTRAWORDS, [3081](#)
- TermMallocAddMemory, [3093](#)
- TERMMEMSTARTNUM, [3081](#)
- TestTerm, [3097](#)
- TimeChildren, [3096](#)
- TimeCPU, [3096](#)
- TimeWallClock, [3096](#)
- ToFast, [3095](#)
- ToGeneral, [3095](#)
- ToPolyFunGeneral, [3095](#)
- ToToken, [3090](#)
- TruncateFile, [3087](#)
- UpdateMaxSize, [3088](#)
- WriteFile, [3098](#)
- WriteFileToFile, [3086](#)
- WriteString, [3089](#)
- WriteUnfinString, [3090](#)
- toOrbits
 - MOrbits, [238](#)
- TOOUTPUT
 - ftypes.h, [1377](#)
- Top
 - CbUf, [71](#)
- top
 - StreaM, [417](#)
- TOPBITONLY
 - form3.h, [1327](#)
- TOPLONGBITONLY
 - form3.h, [1327](#)
- topnumber
 - dbase, [79](#)
- TOPO
 - ftypes.h, [1404](#)
- TOPOLOGIES
 - ftypes.h, [1404](#)
- TopologiesLevel
 - T_const, [439](#)
- TOPOLOGIESONLY
 - ftypes.h, [1456](#)
- TopologiesOptions

- T_const, [439](#)
- TopologiesStart
 - T_const, [432](#)
- TopologiesTerm
 - T_const, [432](#)
- ToPolyFunGeneral
 - declare.h, [902](#)
 - tools.c, [3095](#)
- TOPOLYNOMIALFLAG
 - ftypes.h, [1449](#)
- topolynomialflag
 - C_const, [61](#)
- ToPoTyPe, [468](#)
 - cldeg, [469](#)
 - clext, [469](#)
 - clnum, [469](#)
 - cmaxd, [469](#)
 - cmind, [469](#)
 - ncl, [470](#)
 - nlegs, [470](#)
 - nloops, [470](#)
 - npadding, [470](#)
 - nvert, [470](#)
 - opt, [469](#)
 - sopi, [470](#)
 - vert, [469](#)
 - vertmax, [469](#)
- topowrap.cc, [3145](#), [3147](#)
 - GenerateTopologies, [3146](#)
 - GenerateVertices, [3146](#)
 - MAXPOINTS, [3146](#)
 - toForm, [3146](#)
- toppos
 - StOrEcAcHe, [411](#)
- topsubexpbuffers
 - compiler.c, [726](#)
- toptokens
 - C_const, [50](#)
- ToRat
 - float.c, [1293](#)
- ToStorage
 - declare.h, [878](#)
 - store.c, [2835](#)
- tostring
 - factorized_poly, [126](#)
- total_degree
 - poly, [345](#)
- totalc
 - EGraph, [112](#)
- totalnumberofthreads
 - M_const, [180](#)
- totermes
 - sort.c, [2734](#)
- totind
 - TaBIes, [462](#)
- ToToken
 - declare.h, [919](#)
 - tools.c, [3090](#)
- TPLUS
 - ftypes.h, [1384](#)
- TPOWER
 - ftypes.h, [1384](#)
- TPOWER1
 - ftypes.h, [1385](#)
- Trace4
 - declare.h, [878](#)
 - opera.c, [1959](#)
- Trace4Gen
 - declare.h, [878](#)
 - opera.c, [1959](#)
- Trace4no
 - declare.h, [879](#)
 - opera.c, [1959](#)
- tracebackflag
 - M_const, [188](#)
- TraceFind
 - declare.h, [879](#)
 - opera.c, [1960](#)
- TRACEN
 - structs.h, [2905](#)
- TrAcEn, [471](#)
 - accu, [471](#)
 - accup, [471](#)
 - allsign, [472](#)
 - factor, [472](#)
 - inlist, [472](#)
 - level, [472](#)
 - num, [472](#)
 - perm, [472](#)
 - sgn, [472](#)
 - termp, [471](#)
- TraceN
 - declare.h, [879](#)
 - opera.c, [1960](#)
- TraceNgen
 - declare.h, [879](#)
 - opera.c, [1960](#)
- TraceNno
 - declare.h, [879](#)
 - opera.c, [1959](#)
- TRACES
 - structs.h, [2905](#)
- TrAcEs, [473](#)
 - a3, [477](#)
 - a4, [477](#)
 - accu, [474](#)
 - accup, [474](#)
 - ad, [476](#)
 - allsign, [478](#)
 - e3, [475](#)
 - e4, [475](#)
 - eers, [475](#)
 - factor, [478](#)
 - finalstep, [478](#)
 - gamm, [476](#)
 - gamma5, [477](#)

- inlist, [474](#)
- j3, [475](#)
- j4, [475](#)
- kstep, [476](#)
- lc3, [477](#)
- lc4, [477](#)
- level, [478](#)
- mdel, [475](#)
- mdum, [476](#)
- mepf, [475](#)
- nt3, [474](#)
- nt4, [474](#)
- num, [477](#)
- pdel, [476](#)
- pepf, [475](#)
- perm, [474](#)
- sgn, [476](#)
- sign1, [477](#)
- sign2, [477](#)
- stap, [476](#)
- step1, [476](#)
- termp, [474](#)
- Traces
 - declare.h, [879](#)
 - opera.c, [1960](#)
- tracestack
 - N_const, [249](#)
- TransEname
 - C_const, [53](#)
- TransferBuffer
 - declare.h, [975](#)
 - pattern.c, [2177](#)
- transFlag
 - O_const, [284](#)
- transform.c, [3150](#), [3155](#)
 - CoTransform, [3151](#)
 - FindRange, [3154](#)
 - PutArgInScratch, [3154](#)
 - ReadRange, [3154](#)
 - RunAddArg, [3153](#)
 - RunCycle, [3152](#)
 - RunDecode, [3151](#)
 - RunDedup, [3152](#)
 - RunDropArg, [3153](#)
 - RunEncode, [3151](#)
 - RunExplode, [3152](#)
 - RunHtoZArg, [3154](#)
 - RunImplode, [3152](#)
 - RunIsLyndon, [3153](#)
 - RunMulArg, [3153](#)
 - RunPermute, [3152](#)
 - RunReplace, [3151](#)
 - RunReverse, [3152](#)
 - RunSelectArg, [3153](#)
 - RunToLyndon, [3153](#)
 - RunTransform, [3151](#)
 - RunZtoHArg, [3154](#)
 - TestArgNum, [3154](#)
- TransformRational
 - declare.h, [1021](#)
 - dict.c, [1074](#)
- TranslateExpression
 - declare.h, [977](#)
 - dollar.c, [1096](#)
- TRAP SIGNALS
 - unix.h, [3197](#)
- TreatPolyRatFun
 - declare.h, [992](#)
 - normal.c, [1877](#)
- tree, [478](#)
 - blnce, [479](#)
 - left, [479](#)
 - parent, [479](#)
 - right, [479](#)
 - sOrT, [396](#)
 - usage, [480](#)
 - value, [479](#)
- tree_node, [480](#)
 - childs, [481](#)
 - finished, [482](#)
 - num_visits, [482](#)
 - operator=, [481](#)
 - sum_results, [481](#)
 - tree_node, [481](#)
 - var, [482](#)
- Trick
 - declare.h, [880](#)
 - opera.c, [1959](#)
- TrimPower
 - T_const, [436](#)
- TRUE_EXPR
 - pre.c, [2320](#)
- TruncateFile
 - declare.h, [897](#)
 - tools.c, [3087](#)
- TryDo
 - declare.h, [880](#)
 - reshuf.c, [2615](#)
- TryEnvironment
 - declare.h, [987](#)
 - setfile.c, [2696](#)
- TryFileSetups
 - declare.h, [975](#)
 - setfile.c, [2696](#)
- TryRecover
 - declare.h, [802](#)
- tryterm
 - N_const, [259](#)
- TSET
 - ftypes.h, [1381](#)
- TSETCLOSE
 - ftypes.h, [1384](#)
- TSETDOL
 - ftypes.h, [1385](#)
- TSETNUM
 - ftypes.h, [1385](#)

- TSETOPEN
 - ftypes.h, [1384](#)
- TSGAMMA
 - ftypes.h, [1385](#)
- TSUBEXP
 - ftypes.h, [1382](#)
- TSYMBOL
 - ftypes.h, [1381](#)
- ttypes
 - token.c, [3054](#)
- TVECTOR
 - ftypes.h, [1381](#)
- TWILDARG
 - ftypes.h, [1382](#)
- TWILDCARD
 - ftypes.h, [1382](#)
- TwoExprCompare
 - declare.h, [978](#)
 - dollar.c, [1096](#)
- type
 - ARGBUFFER, [15](#)
 - DICTIONARY_ELEMENT, [84](#)
 - DoLIaR, [88](#)
 - DoLoOp, [91](#)
 - FaCdOILaR, [125](#)
 - fixedset, [138](#)
 - InDeX, [150](#)
 - KEYWORD, [163](#)
 - KEYWORDV, [164](#)
 - MoDoPiDoLIaR, [236](#)
 - NaMeNode, [261](#)
 - optimization, [291](#)
 - PaRtlcLe, [322](#)
 - PF_BUFFER, [330](#)
 - PNodeClass, [336](#)
 - SeTs, [383](#)
 - SETUPPARAMETERS, [385](#)
 - sOrT, [399](#)
 - StreaM, [418](#)
 - VeRtEx, [486](#)
- TYPEADJUSTBOUNDS
 - ftypes.h, [1412](#)
- TYPEALLLOOPS
 - ftypes.h, [1416](#)
- TYPEALLPATHS
 - ftypes.h, [1417](#)
- TYPEAPPLY
 - ftypes.h, [1413](#)
- TYPEAPPLYRESET
 - ftypes.h, [1413](#)
- TYPEARG
 - ftypes.h, [1410](#)
- TYPEARGEXPLODE
 - ftypes.h, [1414](#)
- TYPEARGHEADSIZE
 - ftypes.h, [1422](#)
- TYPEARGIMPLODE
 - ftypes.h, [1414](#)
- TYPEARGTOEXTRASymbol
 - ftypes.h, [1416](#)
- TYPEASSIGN
 - ftypes.h, [1411](#)
- TYPECANONICALIZE
 - ftypes.h, [1416](#)
- TYPECHAININ
 - ftypes.h, [1414](#)
- TYPECHAINOUT
 - ftypes.h, [1414](#)
- TYPECHISHOLM
 - ftypes.h, [1409](#)
- TYPECLEARUSERFLAG
 - ftypes.h, [1416](#)
- TYPECOUNT
 - ftypes.h, [1408](#)
- TYPEDENOMINATORS
 - ftypes.h, [1414](#)
- TYPEDETCURDUM
 - ftypes.h, [1413](#)
- TYPEDISCARD
 - ftypes.h, [1409](#)
- TYPEDOLOOP
 - ftypes.h, [1415](#)
- TYPEDROPCOEFFICIENT
 - ftypes.h, [1415](#)
- TYPEDROPSYMBOLS
 - ftypes.h, [1415](#)
- TYPEELIF
 - ftypes.h, [1409](#)
- TYPEELSE
 - ftypes.h, [1409](#)
- TYPEENDDOLOOP
 - ftypes.h, [1415](#)
- TYPEENDIF
 - ftypes.h, [1409](#)
- TYPEENDREPEAT
 - ftypes.h, [1408](#)
- TYPEENDSWITCH
 - ftypes.h, [1416](#)
- TYPEEXIT
 - ftypes.h, [1410](#)
- TYPEEXPRESSION
 - ftypes.h, [1408](#)
- TYPEFACTARG
 - ftypes.h, [1411](#)
- TYPEFACTARG2
 - ftypes.h, [1411](#)
- TYPEFACTOR
 - ftypes.h, [1414](#)
- TYPEFINDLOOP
 - ftypes.h, [1412](#)
- TYPEFPRINT
 - ftypes.h, [1411](#)
- TYPEFROMPOLYNOMIAL
 - ftypes.h, [1415](#)
- typeGName
 - Particle, [324](#)

- TYPEGOTO
 - ftypes.h, [1409](#)
- TYPEIDNEW
 - ftypes.h, [1408](#)
- TYPEIDOLD
 - ftypes.h, [1408](#)
- TYPEIF
 - ftypes.h, [1409](#)
- TYPEINEXPRESSION
 - ftypes.h, [1413](#)
- TYPEINSIDE
 - ftypes.h, [1412](#)
- TYPEISFUN
 - ftypes.h, [1386](#)
- TYPEISMYSTERY
 - ftypes.h, [1386](#)
- TYPEISSUB
 - ftypes.h, [1386](#)
- TYPEMERGE
 - ftypes.h, [1413](#)
- TYPEMULT
 - ftypes.h, [1408](#)
- typeName
 - Particle, [324](#)
- TYPENORM
 - ftypes.h, [1410](#)
- TYPENORM2
 - ftypes.h, [1410](#)
- TYPENORM3
 - ftypes.h, [1410](#)
- TYPENORM4
 - ftypes.h, [1414](#)
- TYPEOPERATION
 - ftypes.h, [1408](#)
- TYPEPRINT
 - ftypes.h, [1410](#)
- TYPEPUTINSIDE
 - ftypes.h, [1415](#)
- TYPEREDEFPRE
 - ftypes.h, [1411](#)
- TYPERENUMBER
 - ftypes.h, [1412](#)
- TYPEREPEAT
 - ftypes.h, [1408](#)
- TYPEREVERSE
 - ftypes.h, [1410](#)
- TYPESETEXIT
 - ftypes.h, [1410](#)
- TYPESETUSERFLAG
 - ftypes.h, [1416](#)
- YPESORT
 - ftypes.h, [1412](#)
- YPESPLITARG
 - ftypes.h, [1411](#)
- YPESPLITARG2
 - ftypes.h, [1411](#)
- YPESPLITFIRSTARG
 - ftypes.h, [1413](#)
- YPESPLITLASTARG
 - ftypes.h, [1413](#)
- YPESTUFFLE
 - ftypes.h, [1414](#)
- YPESUM
 - ftypes.h, [1409](#)
- YPESUMFIX
 - ftypes.h, [1412](#)
- YPESWITCH
 - ftypes.h, [1416](#)
- typetable
 - SWITCH, [422](#)
- TYPETERM
 - ftypes.h, [1412](#)
- TYPETESTUSE
 - ftypes.h, [1413](#)
- TYPETOPOLYNOMIAL
 - ftypes.h, [1415](#)
- TYPETOSPECTATOR
 - ftypes.h, [1416](#)
- TYPETRANSFORM
 - ftypes.h, [1415](#)
- TYPETRY
 - ftypes.h, [1411](#)
- TYPEUNRAVEL
 - ftypes.h, [1412](#)
- UBYTE
 - form3.h, [1333](#)
- Uclose
 - form3.h, [1330](#)
- uflags
 - ExPrEsSiOn, [124](#)
- Uflush
 - form3.h, [1330](#)
- Ugetpos
 - form3.h, [1331](#)
- UINT
 - form3.h, [1333](#)
- uncompressed
 - objects, [287](#)
- UngetChar
 - declare.h, [802](#)
- UngetFromStream
 - declare.h, [802](#)
- UNHIDE
 - ftypes.h, [1422](#)
- UNHIDEGEXPRESSION
 - ftypes.h, [1425](#)
- UNHIDELEXPRESSION
 - ftypes.h, [1425](#)
- unix.h, [3196](#), [3198](#)
 - ALTSEPARATOR, [3198](#)
 - CARRIAGEReturn, [3197](#)
 - LINEFEED, [3197](#)
 - P_term, [3197](#)
 - PATHSEPARATOR, [3198](#)
 - SEPARATOR, [3197](#)
 - TRAPsignals, [3197](#)

- WITH_ALARM, 3198
- WITH_ENV, 3198
- WITHEXTERNALCHANNEL, 3197
- WITHPIPE, 3197
- WITHSYSTEM, 3197
- unixfile.c, 3199
- UNLOCK
 - declare.h, 819
- UNPACK
 - ftypes.h, 1445
- UnPack
 - declare.h, 880
 - reken.c, 2556
- UNPACK_LONG
 - parallel.c, 2067
- UnpackFloat
 - float.c, 1291
- UNRWLOCK
 - declare.h, 819
- UnsetAllowDelay
 - declare.h, 904
 - pre.c, 2321
- UnSetDictionary
 - declare.h, 1020
 - dict.c, 1076
- UnSetMods
 - declare.h, 883
 - execute.c, 1157
- UnsureDollarMode
 - C_const, 57
- Uopen
 - form3.h, 1330
- updateCandNode
 - Assign, 21
- UpdateMaxSize
 - declare.h, 1005
 - tools.c, 3088
- UpdatePositions
 - declare.h, 978
 - execute.c, 1160
- Uread
 - form3.h, 1331
- usage
 - tree, 480
- used
 - sOrT, 394
- USEDFLAG
 - ftypes.h, 1430
- UsedFunction
 - N_const, 248
- UseDictionary
 - dict.c, 1076
- UsedIndex
 - N_const, 247
- UsedOtherFind
 - N_const, 253
- UsedSymbol
 - N_const, 247
- UsedVector
 - N_const, 247
- Useek
 - form3.h, 1331
- UseFindOnly
 - N_const, 253
- usenames
 - NaMeSpAcE, 262
- UserFlags
 - pre.c, 2337
- Usetbuf
 - form3.h, 1331
- Usetpos
 - form3.h, 1332
- Ustdout
 - form3.h, 1332
- Usync
 - form3.h, 1332
- Utell
 - form3.h, 1331
- Utruncate
 - form3.h, 1332
- Uwrite
 - form3.h, 1331
- va_arg
 - sch.c, 2655
- va_dcl
 - sch.c, 2654
- va_end
 - sch.c, 2654
- va_list
 - sch.c, 2655
- va_start
 - sch.c, 2654
- value
 - FaCdOILaR, 125
 - pReVaR, 360
 - SETUPPARAMETERS, 385
 - SWITCHTABLE, 423
 - tree, 479
- values
 - Options, 302
- var
 - KEYWORDV, 164
 - tree_node, 482
- var_map_t
 - flintinterface.h, 1270
- variable.h, 3202, 3213
 - A, 3212
 - AC, 3210
 - AM, 3210
 - AN, 3210
 - AO, 3211
 - AP, 3211
 - AR, 3211
 - AS, 3211
 - AT, 3211
 - autofunctions, 3209

- autoindices, [3209](#)
- autosymbols, [3209](#)
- autovectors, [3209](#)
- AX, [3211](#)
- BUG, [3210](#)
- cbuf, [3208](#)
- channels, [3208](#)
- chartype, [3204](#)
- Dollars, [3208](#)
- DoLoops, [3204](#)
- Dubious, [3208](#)
- Expressions, [3209](#)
- FG, [3212](#)
- fixedsets, [3212](#)
- functions, [3205](#)
- GlobalFunctions, [3207](#)
- GlobalIndices, [3207](#)
- GlobalSetElements, [3208](#)
- GlobalSets, [3207](#)
- GlobalSymbols, [3207](#)
- GlobalVectors, [3207](#)
- indices, [3206](#)
- MaxDoLoops, [3205](#)
- MaxNumPre, [3205](#)
- MaxProcedures, [3204](#)
- ModOptdollars, [3210](#)
- NumDollars, [3208](#)
- NumDoLoops, [3205](#)
- NumDubious, [3208](#)
- NumExpressions, [3209](#)
- NumFunctions, [3206](#)
- NumIndices, [3206](#)
- NumModOptdollars, [3210](#)
- NumOutputChannels, [3208](#)
- NumPotModdollars, [3210](#)
- NumPre, [3205](#)
- NumProcedures, [3204](#)
- NumSetElements, [3207](#)
- NumSets, [3207](#)
- NumSymbols, [3206](#)
- NumTableBases, [3207](#)
- NumVectors, [3206](#)
- numxsymbol, [3209](#)
- PotModdollars, [3210](#)
- PreVar, [3205](#)
- Procedures, [3204](#)
- SetElements, [3205](#)
- Sets, [3205](#)
- setupfilename, [3212](#)
- symbols, [3206](#)
- tablebases, [3206](#)
- vectors, [3206](#)
- WriteFile, [3212](#)
- xsymbol, [3209](#)
- variables
 - DICTIONARY, [82](#)
 - InDeXeNtRy, [154](#)
- VARNAME
 - declare.h, [810](#)
- VARNAMES
 - ftypes.h, [1445](#)
- varnames
 - C_const, [42](#)
- VARRENUM
 - structs.h, [2904](#)
- VaRrEnUm, [482](#)
 - hi, [483](#)
 - lo, [483](#)
 - start, [483](#)
- vars
 - DoLoOp, [90](#)
- VarStore
 - declare.h, [880](#)
 - store.c, [2836](#)
- VarType
 - FixedGlobals, [135](#)
- VARTYPECOMPLEX
 - ftypes.h, [1433](#)
- VARTYPEIMAGINARY
 - ftypes.h, [1433](#)
- VARTYPEMINUS
 - ftypes.h, [1433](#)
- VARTYPENONE
 - ftypes.h, [1432](#)
- VARTYPEROOTOFUNITY
 - ftypes.h, [1433](#)
- vdef
 - Model, [230](#)
- vecnum
 - ReNuMbEr, [380](#)
- vect
 - ReNuMbEr, [379](#)
- VECTBIT
 - ftypes.h, [1439](#)
- VECTOMIN
 - ftypes.h, [1418](#)
- VECTOR
 - ftypes.h, [1391](#)
- VeCtOr, [483](#)
 - complex, [484](#)
 - dimension, [484](#)
 - flags, [484](#)
 - name, [484](#)
 - namesize, [484](#)
 - node, [484](#)
 - number, [484](#)
- Vector
 - vector.h, [3216](#)
- vector.h, [3214](#), [3223](#)
 - DeclareVector, [3216](#)
 - Vector, [3216](#)
 - VectorBack, [3218](#)
 - VectorCapacity, [3219](#)
 - VectorClear, [3220](#)
 - VectorEmpty, [3219](#)
 - VectorErase, [3223](#)

- VectorErases, [3223](#)
- VectorFree, [3217](#)
- VectorFront, [3218](#)
- VectorInit, [3217](#)
- VectorInsert, [3222](#)
- VectorInserts, [3222](#)
- VectorPopBack, [3221](#)
- VectorPtr, [3218](#)
- VectorPushBack, [3220](#)
- VectorPushBacks, [3221](#)
- VectorReserve, [3220](#)
- VectorSize, [3219](#)
- VectorStruct, [3216](#)
- VECTOR_
 - ftypes.h, [1429](#)
- VectorBack
 - vector.h, [3218](#)
- VectorCapacity
 - vector.h, [3219](#)
- VectorClear
 - vector.h, [3220](#)
- VectorEmpty
 - vector.h, [3219](#)
- VectorErase
 - vector.h, [3223](#)
- VectorErases
 - vector.h, [3223](#)
- VectorFree
 - vector.h, [3217](#)
- VectorFront
 - vector.h, [3218](#)
- VectorInit
 - vector.h, [3217](#)
- VectorInsert
 - vector.h, [3222](#)
- VectorInserts
 - vector.h, [3222](#)
- vectorlikeLHS
 - C_const, [63](#)
- VectorList
 - C_const, [44](#)
- VECTORONLY
 - ftypes.h, [1378](#)
- VectorPopBack
 - vector.h, [3221](#)
- VectorPtr
 - vector.h, [3218](#)
- VectorPushBack
 - vector.h, [3220](#)
- VectorPushBacks
 - vector.h, [3221](#)
- VectorReserve
 - vector.h, [3220](#)
- Vectors
 - C_const, [46](#)
- vectors
 - variable.h, [3206](#)
- VectorSize
 - vector.h, [3219](#)
- VectorStruct
 - vector.h, [3216](#)
- vectorzero
 - M_const, [192](#)
- VECTOSUB
 - ftypes.h, [1418](#)
- VECTOVEC
 - ftypes.h, [1418](#)
- VERSIONSTR
 - startup.c, [2802](#)
- VERSIONSTR__
 - startup.c, [2802](#)
- vert
 - ToPoTyPe, [469](#)
- VeRtEx, [485](#)
 - couplings, [485](#)
 - error, [486](#)
 - externonly, [486](#)
 - ncouplings, [486](#)
 - nparticles, [486](#)
 - particles, [485](#)
 - spare, [486](#)
 - type, [486](#)
- VERTEXFUNCTION
 - ftypes.h, [1427](#)
- vertices
 - MoDeL, [233](#)
- vertmax
 - ToPoTyPe, [469](#)
- vetofilling
 - C_const, [60](#)
- vetotablebasefill
 - C_const, [60](#)
- vflags
 - ExPrEsSiOn, [123](#)
- visit
 - MGraph, [208](#)
- visited
 - EEdge, [99](#)
 - ENode, [118](#)
 - MNode, [220](#)
 - T_MNode, [452](#)
- VORTRANMODE
 - ftypes.h, [1387](#)
- wAGraphs
 - Assign, [25](#)
 - Process, [368](#)
 - SProcess, [410](#)
- WantAddLongs
 - declare.h, [815](#)
- WantAddPointers
 - declare.h, [815](#)
- WantAddPositions
 - declare.h, [816](#)
- wAOPI
 - Assign, [25](#)
 - Process, [368](#)

- SProcess, [410](#)
- WARMUP
 - reken.c, [2555](#)
- WarnFlag
 - C_const, [57](#)
- Warning
 - declare.h, [884](#)
 - message.c, [1716](#)
- WCN
 - structs.h, [2908](#)
- WCN2
 - structs.h, [2908](#)
- WCOPY
 - declare.h, [804](#)
- wgraphs
 - Process, [367](#)
- where
 - DoLIaRS, [88](#)
 - FaCdOILaR, [125](#)
 - PaRtl, [321](#)
 - TaBIEbAsEsUbInDeX, [459](#)
- whichbuffer
 - ExPrEsSiOn, [123](#)
- WhileLevel
 - C_const, [67](#)
- whilelevel
 - SWITCH, [423](#)
- wildarg
 - pReVaR, [361](#)
- WILDARGFUN
 - ftypes.h, [1399](#)
- WILDARGINDEX
 - ftypes.h, [1407](#)
- WildArgs
 - N_const, [257](#)
- WILDARGSYMBOL
 - ftypes.h, [1406](#)
- WildArgTaken
 - T_const, [430](#)
- WILDARGVECTOR
 - ftypes.h, [1407](#)
- WildC
 - C_const, [48](#)
- wildcard.c, [3227](#), [3229](#)
 - AddWild, [3228](#)
 - CheckWild, [3228](#)
 - ClearWild, [3228](#)
 - DEBUG, [3227](#)
 - DenToFunction, [3228](#)
 - ResolveSet, [3228](#)
 - WildFill, [3228](#)
- WildcardBufferSize
 - C_const, [55](#)
 - T_const, [434](#)
- WildcardNames
 - C_const, [49](#)
- WILDCARDS
 - ftypes.h, [1420](#)
- WildDirt
 - N_const, [256](#)
- WildDollars
 - declare.h, [962](#)
 - dollar.c, [1094](#)
- WILDDUMMY
 - ftypes.h, [1418](#)
- WildEat
 - N_const, [257](#)
- WildFill
 - declare.h, [880](#)
 - wildcard.c, [3228](#)
- wildflag
 - C_const, [55](#)
- WILDMASK
 - form3.h, [1327](#)
- WildMask
 - T_const, [431](#)
- WILDOFFSET
 - fsizes.h, [1342](#)
- wildoffset
 - STOREHEADER, [415](#)
- WildReserve
 - N_const, [256](#)
- WildStop
 - N_const, [246](#)
- WildValue
 - N_const, [246](#)
- Willnd
 - M_const, [185](#)
- WITH_ALARM
 - unix.h, [3198](#)
- WITH_ENV
 - fwin.h, [1498](#)
 - unix.h, [3198](#)
- WITHAUTO
 - ftypes.h, [1377](#)
- WITHBLOCKS
 - ftypes.h, [1457](#)
- WITHCUTOFF
 - float.c, [1286](#)
- WITHEDGES
 - ftypes.h, [1457](#)
- WITHERROR
 - ftypes.h, [1367](#)
- WITHEXTERNALCHANNEL
 - unix.h, [3197](#)
- WITHINSERTIONS
 - ftypes.h, [1456](#)
- WITHONEPI
 - ftypes.h, [1457](#)
- WITHOUTERROR
 - ftypes.h, [1367](#)
- withoutflush
 - minos.c, [1736](#)
 - minos.h, [1757](#)
- WITHOUTSEMICOLON
 - ftypes.h, [1372](#)

- WITHPIPE
 - unix.h, [3197](#)
- WITHRETURN
 - fwin.h, [1497](#)
- WITHSEMICOLON
 - ftypes.h, [1372](#)
- WITHSORTBOTS
 - form3.h, [1330](#)
- WITHSYMMETRIZE
 - ftypes.h, [1456](#)
- WITHSYSTEM
 - fwin.h, [1497](#)
 - unix.h, [3197](#)
- wlen
 - O_const, [282](#)
- wMGraphs
 - Process, [367](#)
 - SProcess, [409](#)
- wMOPI
 - Process, [367](#)
 - SProcess, [410](#)
- wopi
 - Process, [367](#)
- WORDDIF
 - declare.h, [809](#)
- WORDMASK
 - form3.h, [1327](#)
- WORKBUFFER
 - fsizes.h, [1343](#)
- WorkPointer
 - T_const, [430](#)
- WorkSize
 - M_const, [176](#)
- WorkSpace
 - T_const, [429](#)
- WorkTop
 - T_const, [429](#)
- wpoin
 - O_const, [279](#)
- wpos
 - O_const, [279](#)
- wranf
 - declare.h, [991](#)
 - reken.c, [2565](#)
- wranfcall
 - R_const, [374](#)
- wranfia
 - R_const, [372](#)
- wranfnpair1
 - R_const, [374](#)
- wranfnpair2
 - R_const, [374](#)
- wranfseed
 - R_const, [373](#)
- WriteAll
 - declare.h, [881](#)
 - sch.c, [2658](#)
- WriteArgument
 - declare.h, [881](#)
 - sch.c, [2657](#)
- WRITEBUFTOEXTCHANNEL
 - declare.h, [821](#)
- WriteDictionary
 - declare.h, [1021](#)
 - sch.c, [2657](#)
- WriteDollarFactorToBuffer
 - declare.h, [961](#)
 - dollar.c, [1093](#)
- WriteDollarToBuffer
 - declare.h, [961](#)
 - dollar.c, [1093](#)
- WriteExpression
 - declare.h, [881](#)
 - sch.c, [2658](#)
- WRITEFILE
 - declare.h, [821](#)
- WriteFile
 - tools.c, [3098](#)
 - variable.h, [3212](#)
- WriteFileToFile
 - declare.h, [895](#)
 - tools.c, [3086](#)
- WriteIndex
 - declare.h, [984](#)
 - minos.c, [1732](#)
- WriteIndexBlock
 - declare.h, [983](#)
 - minos.c, [1732](#)
- WriteNilInfo
 - declare.h, [984](#)
 - minos.c, [1732](#)
- WriteInnerTerm
 - declare.h, [881](#)
 - sch.c, [2658](#)
- WriteLists
 - declare.h, [881](#)
 - sch.c, [2657](#)
- WriteNamesBlock
 - declare.h, [984](#)
 - minos.c, [1732](#)
- WriteObject
 - declare.h, [985](#)
 - minos.c, [1734](#)
- WriteOne
 - declare.h, [881](#)
 - sch.c, [2658](#)
- WRITEOUT
 - ftypes.h, [1375](#)
- WriteSetup
 - declare.h, [882](#)
 - setfile.c, [2695](#)
- WriteStats
 - declare.h, [882](#)
 - sort.c, [2720](#)
- WriteStoreHeader
 - declare.h, [996](#)

- store.c, [2837](#)
- WriteString
 - declare.h, [915](#)
 - tools.c, [3089](#)
- WriteSubTerm
 - declare.h, [882](#)
 - sch.c, [2658](#)
- WriteTerm
 - declare.h, [883](#)
 - sch.c, [2658](#)
- writeToChannel
 - declare.h, [989](#)
 - pre.c, [2334](#)
- WriteTokens
 - declare.h, [957](#)
 - token.c, [3052](#)
- WriteUnfinString
 - declare.h, [915](#)
 - tools.c, [3090](#)
- WrtPower
 - declare.h, [824](#)
 - sch.c, [2657](#)
- wskon
 - MGraph, [213](#)
- wsizeof
 - declare.h, [809](#)
- wsopi
 - MGraph, [213](#)
 - T_MGraph, [449](#)
- wsum
 - T_MGraph, [449](#)
- WTimeStatsFlag
 - C_const, [62](#)
- X
 - AllGlobals, [11](#)
- X_const, [487](#)
 - currentExternalChannel, [488](#)
 - currentPrompt, [487](#)
 - daemonize, [488](#)
 - killSignal, [487](#)
 - killWholeGroup, [488](#)
 - shellname, [487](#)
 - stderrname, [487](#)
 - timeout, [487](#)
- xsymbol
 - variable.h, [3209](#)
- YESLYNDON
 - ftypes.h, [1449](#)
- Z_
 - ftypes.h, [1428](#)
- zbufnum
 - M_const, [179](#)
- zero
 - DoLIaRS, [88](#)
- ZEROARG
 - declare.h, [807](#)
- ZeroFillRange
 - declare.h, [809](#)
- zeropos
 - M_const, [172](#)
- zerorhs
 - M_const, [192](#)
- ZeroTable
 - float.c, [1292](#)
- ZtoForm
 - float.c, [1287](#)
- ZTOHARG
 - ftypes.h, [1449](#)